NeuraEdge Universal: Complete Line-by-Line Source Reference

Generated from hashed repository source

July 16, 2026

Contents

1 How to read this volume **12**

2 rtl/neuraedge_activation_buffer.sv **13**
- 2.1 Beginner architecture map . . . 13
- 2.2 Complete source . . . 13
- 2.3 Line annotations . . . 13

3 rtl/neuraedge_activation_lut8.sv **17**
- 3.1 Beginner architecture map . . . 17
- 3.2 Complete source . . . 17
- 3.3 Line annotations . . . 19

4 rtl/neuraedge_active_index_buffer.sv **37**
- 4.1 Beginner architecture map . . . 37
- 4.2 Complete source . . . 37
- 4.3 Line annotations . . . 37

5 rtl/neuraedge_alpha_xnor.sv **41**
- 5.1 Beginner architecture map . . . 41
- 5.2 Complete source . . . 41
- 5.3 Line annotations . . . 44

6 rtl/neuraedge_buffered_tensor_core.sv **68**
- 6.1 Beginner architecture map . . . 68
- 6.2 Complete source . . . 68
- 6.3 Line annotations . . . 80

7 rtl/neuraedge_command_pkg.sv **177**
- 7.1 Beginner architecture map . . . 177
- 7.2 Complete source . . . 177
- 7.3 Line annotations . . . 177

8 rtl/neuraedge_descriptor_sequencer.sv **181**
- 8.1 Beginner architecture map . . . 181
- 8.2 Complete source . . . 181
- 8.3 Line annotations . . . 185

9 rtl/neuraedge_gradient_accumulator_bank.sv **216**

9.1 Beginner architecture map . . . 216
9.2 Complete source . . . 216
9.3 Line annotations . . . 216

10 rtl/neuraedge_integer_layernorm.sv 219
10.1 Beginner architecture map . . . 219
10.2 Complete source . . . 219
10.3 Line annotations . . . 228

11 rtl/neuraedge_isqrt64.sv 299
11.1 Beginner architecture map . . . 299
11.2 Complete source . . . 299
11.3 Line annotations . . . 300

12 rtl/neuraedge_lif_bank.sv 309
12.1 Beginner architecture map . . . 309
12.2 Complete source . . . 309
12.3 Line annotations . . . 313

13 rtl/neuraedge_linear_backward8.sv 345
13.1 Beginner architecture map . . . 345
13.2 Complete source . . . 345
13.3 Line annotations . . . 357

14 rtl/neuraedge_log_softmax.sv 452
14.1 Beginner architecture map . . . 452
14.2 Complete source . . . 452
14.3 Line annotations . . . 456

15 rtl/neuraedge_mac2_array_probe.sv 485
15.1 Beginner architecture map . . . 485
15.2 Complete source . . . 485
15.3 Line annotations . . . 486

16 rtl/neuraedge_mac2_u8.sv 490
16.1 Beginner architecture map . . . 490
16.2 Complete source . . . 490
16.3 Line annotations . . . 491

17 rtl/neuraedge_pynq_axi_top.sv 497
17.1 Beginner architecture map . . . 497
17.2 Complete source . . . 497
17.3 Line annotations . . . 505

18 rtl/neuraedge_sgd_optimizer8.sv 562
18.1 Beginner architecture map . . . 562
18.2 Complete source . . . 562
18.3 Line annotations . . . 567

19 rtl/neuraedge_tensor_tile.sv 611

19.1 Beginner architecture map . . . 611
19.2 Complete source . . . 611
19.3 Line annotations . . . 625

20 rtl/neuraedge_udivider.sv 740
20.1 Beginner architecture map . . . 740
20.2 Complete source . . . 740
20.3 Line annotations . . . 742

21 rtl/neuraedge_universal_core.sv 755
21.1 Beginner architecture map . . . 755
21.2 Complete source . . . 755
21.3 Line annotations . . . 769

22 rtl/neuraedge_vector_alu.sv 878
22.1 Beginner architecture map . . . 878
22.2 Complete source . . . 878
22.3 Line annotations . . . 883

23 rtl/neuraedge_weight_bank.sv 928
23.1 Beginner architecture map . . . 928
23.2 Complete source . . . 928
23.3 Line annotations . . . 929

24 software/benchmark_compiler_memory.py 933
24.1 Beginner architecture map . . . 933
24.2 Complete source . . . 933
24.3 Line annotations . . . 934

25 software/evaluate_spikingformer.py 942
25.1 Beginner architecture map . . . 942
25.2 Complete source . . . 942
25.3 Line annotations . . . 945

26 software/export_cifar10_shards.py 975
26.1 Beginner architecture map . . . 975
26.2 Complete source . . . 975
26.3 Line annotations . . . 976

27 software/export_spikingformer_hardware.py 987
27.1 Beginner architecture map . . . 987
27.2 Complete source . . . 987
27.3 Line annotations . . . 990

28 software/generate_debug_report.py 1027
28.1 Beginner architecture map . . . 1027
28.2 Complete source . . . 1027
28.3 Line annotations . . . 1040

29 software/generate_operator_fixtures.py 1162

29.1 Beginner architecture map . . . 1162
29.2 Complete source . . . 1162
29.3 Line annotations . . . 1165

30 software/generate_source_reference.py 1194
30.1 Beginner architecture map . . . 1194
30.2 Complete source . . . 1194
30.3 Line annotations . . . 1206

31 software/generate_spikingformer_manifest.py 1425
31.1 Beginner architecture map . . . 1425
31.2 Complete source . . . 1425
31.3 Line annotations . . . 1429

32 software/generate_tensor_tile_fixture.py 1480
32.1 Beginner architecture map . . . 1480
32.2 Complete source . . . 1480
32.3 Line annotations . . . 1482

33 software/generate_training_coverage.py 1499
33.1 Beginner architecture map . . . 1499
33.2 Complete source . . . 1499
33.3 Line annotations . . . 1499

34 software/neuraedge_compiler.py 1503
34.1 Beginner architecture map . . . 1503
34.2 Complete source . . . 1503
34.3 Line annotations . . . 1512

35 software/neuraedge_golden.py 1585
35.1 Beginner architecture map . . . 1585
35.2 Complete source . . . 1585
35.3 Line annotations . . . 1590

36 software/neuraedge_protocol.py 1624
36.1 Beginner architecture map . . . 1624
36.2 Complete source . . . 1624
36.3 Line annotations . . . 1632

37 software/neuraedge_pynq.py 1688
37.1 Beginner architecture map . . . 1688
37.2 Complete source . . . 1688
37.3 Line annotations . . . 1693

38 software/neuraedge_training.py 1730
38.1 Beginner architecture map . . . 1730
38.2 Complete source . . . 1730
38.3 Line annotations . . . 1748

39 software/run_spikingformer_pynq.py 1918

39.1 Beginner architecture map . . . 1918
39.2 Complete source . . . 1918
39.3 Line annotations . . . 1919

40 software/spikingformer_accelerator.py 1929
40.1 Beginner architecture map . . . 1929
40.2 Complete source . . . 1929
40.3 Line annotations . . . 1938

41 software/spikingformer_parameters.py 2017
41.1 Beginner architecture map . . . 2017
41.2 Complete source . . . 2017
41.3 Line annotations . . . 2019

42 software/spikingformer_quantization.py 2037
42.1 Beginner architecture map . . . 2037
42.2 Complete source . . . 2037
42.3 Line annotations . . . 2040

43 software/spikingformer_reference.py 2070
43.1 Beginner architecture map . . . 2070
43.2 Complete source . . . 2070
43.3 Line annotations . . . 2075

44 software/spikingformer_training_runtime.py 2108
44.1 Beginner architecture map . . . 2108
44.2 Complete source . . . 2108
44.3 Line annotations . . . 2131

45 software/train_spikingformer_pynq.py 2356
45.1 Beginner architecture map . . . 2356
45.2 Complete source . . . 2356
45.3 Line annotations . . . 2360

46 tb/tb_neuraedge_activation_lut8.sv 2398
46.1 Beginner architecture map . . . 2398
46.2 Complete source . . . 2398
46.3 Line annotations . . . 2400

47 tb/tb_neuraedge_alpha_xnor.sv 2411
47.1 Beginner architecture map . . . 2411
47.2 Complete source . . . 2411
47.3 Line annotations . . . 2413

48 tb/tb_neuraedge_buffered_tensor_core.sv 2426
48.1 Beginner architecture map . . . 2426
48.2 Complete source . . . 2426
48.3 Line annotations . . . 2431

49 tb/tb_neuraedge_descriptor_sequencer.sv 2464

49.1 Beginner architecture map . . . 2464
49.2 Complete source . . . 2464
49.3 Line annotations . . . 2466

50 tb/tb_neuraedge_integer_primitives.sv 2484
50.1 Beginner architecture map . . . 2484
50.2 Complete source . . . 2484
50.3 Line annotations . . . 2486

51 tb/tb_neuraedge_lif_bank.sv 2498
51.1 Beginner architecture map . . . 2498
51.2 Complete source . . . 2498
51.3 Line annotations . . . 2500

52 tb/tb_neuraedge_linear_backward8.sv 2517
52.1 Beginner architecture map . . . 2517
52.2 Complete source . . . 2517
52.3 Line annotations . . . 2525

53 tb/tb_neuraedge_mac2_u8.sv 2584
53.1 Beginner architecture map . . . 2584
53.2 Complete source . . . 2584
53.3 Line annotations . . . 2585

54 tb/tb_neuraedge_normalization_units.sv 2593
54.1 Beginner architecture map . . . 2593
54.2 Complete source . . . 2593
54.3 Line annotations . . . 2596

55 tb/tb_neuraedge_pynq_axi_top.sv 2620
55.1 Beginner architecture map . . . 2620
55.2 Complete source . . . 2620
55.3 Line annotations . . . 2624

56 tb/tb_neuraedge_pynq_descriptor.sv 2648
56.1 Beginner architecture map . . . 2648
56.2 Complete source . . . 2648
56.3 Line annotations . . . 2650

57 tb/tb_neuraedge_sgd_optimizer8.sv 2662
57.1 Beginner architecture map . . . 2662
57.2 Complete source . . . 2662
57.3 Line annotations . . . 2664

58 tb/tb_neuraedge_tensor_tile.sv 2676
58.1 Beginner architecture map . . . 2676
58.2 Complete source . . . 2676
58.3 Line annotations . . . 2680

59 tb/tb_neuraedge_universal_core.sv 2704

59.1 Beginner architecture map . . . 2704
59.2 Complete source . . . 2704
59.3 Line annotations . . . 2710

60 tb/tb_neuraedge_vector_alu.sv 2751
60.1 Beginner architecture map . . . 2751
60.2 Complete source . . . 2751
60.3 Line annotations . . . 2753

61 tests/test_build_contract.py 2764
61.1 Beginner architecture map . . . 2764
61.2 Complete source . . . 2764
61.3 Line annotations . . . 2765

62 tests/test_debug_report.py 2774
62.1 Beginner architecture map . . . 2774
62.2 Complete source . . . 2774
62.3 Line annotations . . . 2775

63 tests/test_documentation_generator.py 2786
63.1 Beginner architecture map . . . 2786
63.2 Complete source . . . 2786
63.3 Line annotations . . . 2787

64 tests/test_neuraedge_compiler.py 2796
64.1 Beginner architecture map . . . 2796
64.2 Complete source . . . 2796
64.3 Line annotations . . . 2799

65 tests/test_neuraedge_golden.py 2826
65.1 Beginner architecture map . . . 2826
65.2 Complete source . . . 2826
65.3 Line annotations . . . 2829

66 tests/test_neuraedge_protocol.py 2858
66.1 Beginner architecture map . . . 2858
66.2 Complete source . . . 2858
66.3 Line annotations . . . 2861

67 tests/test_neuraedge_pynq.py 2893
67.1 Beginner architecture map . . . 2893
67.2 Complete source . . . 2893
67.3 Line annotations . . . 2895

68 tests/test_neuraedge_training.py 2912
68.1 Beginner architecture map . . . 2912
68.2 Complete source . . . 2912
68.3 Line annotations . . . 2919

69 tests/test_run_spikingformer_pynq.py 2989

69.1 Beginner architecture map . . . 2989
69.2 Complete source . . . 2989
69.3 Line annotations . . . 2989

70 tests/test_spikingformer_accelerator.py 2992
70.1 Beginner architecture map . . . 2992
70.2 Complete source . . . 2992
70.3 Line annotations . . . 2995

71 tests/test_spikingformer_export.py 3026
71.1 Beginner architecture map . . . 3026
71.2 Complete source . . . 3026
71.3 Line annotations . . . 3027

72 tests/test_spikingformer_manifest.py 3033
72.1 Beginner architecture map . . . 3033
72.2 Complete source . . . 3033
72.3 Line annotations . . . 3034

73 tests/test_spikingformer_parameters.py 3038
73.1 Beginner architecture map . . . 3038
73.2 Complete source . . . 3038
73.3 Line annotations . . . 3039

74 tests/test_spikingformer_reference.py 3049
74.1 Beginner architecture map . . . 3049
74.2 Complete source . . . 3049
74.3 Line annotations . . . 3051

75 tests/test_spikingformer_training_runtime.py 3065
75.1 Beginner architecture map . . . 3065
75.2 Complete source . . . 3065
75.3 Line annotations . . . 3069

76 tests/test_train_spikingformer_pynq.py 3106
76.1 Beginner architecture map . . . 3106
76.2 Complete source . . . 3106
76.3 Line annotations . . . 3107

77 scripts/capture_power_activity.ps1 3116
77.1 Beginner architecture map . . . 3116
77.2 Complete source . . . 3116
77.3 Line annotations . . . 3117

78 scripts/capture_waveforms.ps1 3127
78.1 Beginner architecture map . . . 3127
78.2 Complete source . . . 3127
78.3 Line annotations . . . 3129

79 scripts/deploy_to_pynq.ps1 3140

79.1 Beginner architecture map . . . 3140
79.2 Complete source . . . 3140
79.3 Line annotations . . . 3140

80 scripts/export_cifar10_shards.ps1 3145
80.1 Beginner architecture map . . . 3145
80.2 Complete source . . . 3145
80.3 Line annotations . . . 3145

81 scripts/manage_pynq_training.ps1 3148
81.1 Beginner architecture map . . . 3148
81.2 Complete source . . . 3148
81.3 Line annotations . . . 3149

82 scripts/run_mac2_probe.ps1 3154
82.1 Beginner architecture map . . . 3154
82.2 Complete source . . . 3154
82.3 Line annotations . . . 3154

83 scripts/run_pynq_full.ps1 3157
83.1 Beginner architecture map . . . 3157
83.2 Complete source . . . 3157
83.3 Line annotations . . . 3158

84 scripts/run_regression.ps1 3163
84.1 Beginner architecture map . . . 3163
84.2 Complete source . . . 3163
84.3 Line annotations . . . 3165

85 scripts/run_saif_power.ps1 3178
85.1 Beginner architecture map . . . 3178
85.2 Complete source . . . 3178
85.3 Line annotations . . . 3178

86 scripts/run_verilator_lint.ps1 3183
86.1 Beginner architecture map . . . 3183
86.2 Complete source . . . 3183
86.3 Line annotations . . . 3183

87 scripts/run_vivado_build.ps1 3185
87.1 Beginner architecture map . . . 3185
87.2 Complete source . . . 3185
87.3 Line annotations . . . 3186

88 scripts/analyze_routed_timing.tcl 3191
88.1 Beginner architecture map . . . 3191
88.2 Complete source . . . 3191
88.3 Line annotations . . . 3191

89 scripts/annotate_saif_power.tcl 3193

89.1 Beginner architecture map . . . 3193
89.2 Complete source . . . 3193
89.3 Line annotations . . . 3194

90 scripts/build_pynq_full.tcl 3202
90.1 Beginner architecture map . . . 3202
90.2 Complete source . . . 3202
90.3 Line annotations . . . 3206

91 scripts/build_pynq_ooc.tcl 3238
91.1 Beginner architecture map . . . 3238
91.2 Complete source . . . 3238
91.3 Line annotations . . . 3239

92 scripts/capture_power_activity.tcl 3244
92.1 Beginner architecture map . . . 3244
92.2 Complete source . . . 3244
92.3 Line annotations . . . 3244

93 scripts/capture_waveform.tcl 3247
93.1 Beginner architecture map . . . 3247
93.2 Complete source . . . 3247
93.3 Line annotations . . . 3247

94 scripts/export_pynq_diagrams.tcl 3249
94.1 Beginner architecture map . . . 3249
94.2 Complete source . . . 3249
94.3 Line annotations . . . 3249

95 scripts/finalize_pynq_ooc.tcl 3252
95.1 Beginner architecture map . . . 3252
95.2 Complete source . . . 3252
95.3 Line annotations . . . 3253

96 scripts/report_checkpoint_hierarchy.tcl 3258
96.1 Beginner architecture map . . . 3258
96.2 Complete source . . . 3258
96.3 Line annotations . . . 3258

97 scripts/report_optimizer_dsp.tcl 3261
97.1 Beginner architecture map . . . 3261
97.2 Complete source . . . 3261
97.3 Line annotations . . . 3261

98 scripts/resume_pynq_ooc.tcl 3265
98.1 Beginner architecture map . . . 3265
98.2 Complete source . . . 3265
98.3 Line annotations . . . 3265

99 scripts/synth_mac2_probe.tcl 3268

99.1 Beginner architecture map . . . 3268
99.2 Complete source . . . 3268
99.3 Line annotations . . . 3268

100 `scripts/synth_timing_iteration.tcl` 3271
100.1 Beginner architecture map . . . 3271
100.2 Complete source . . . 3271
100.3 Line annotations . . . 3271

101 Coverage statement 3277

1 How to read this volume

Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.

2 rtl/neuraedge_activation_buffer.sv

Role in the complete system: stores one local tile of unsigned eight-bit activations for repeated tensor-array reads.

2.1 Beginner architecture map

Dataflow responsibility A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_activation_buffer into physical nets and primitives. The source declares 5 input-port lines, 1 output-port lines, 1 clocked process(es), and 0 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls.

Connections to other blocks This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

2.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_activation_buffer #(
5      parameter int DEPTH = 2048,
6      parameter int ADDRESS_BITS = 11
7  ) (
8      input  wire clk,
9      input  wire write_enable,
10     input  wire [ADDRESS_BITS-1:0] write_address,
11     input  wire [63:0] write_data,
12     input  wire [ADDRESS_BITS-1:0] read_address,
13     output logic [63:0] read_data
14 );
15     (* ram_style = "block" *) logic [63:0] memory [0:DEPTH-1];
16     logic [63:0] memory_read_data_r;
17
18     always_ff @(posedge clk) begin
19         if (write_enable) memory[write_address] <= write_data;
20         memory_read_data_r <= memory[read_address];
21         read_data <= memory_read_data_r;
22     end
23
24     initial begin
25         if (DEPTH != (1 << ADDRESS_BITS)) $fatal(1, "activation buffer depth/address mismatch");
26     end
27 endmodule
28
29 `default_nettype wire
```

2.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_activation_buffer #(` This opens neuraedge_activation_buffer, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that stores one local tile of unsigned eight-bit activations for repeated tensor-array reads.

Line 5: `parameter int DEPTH = 2048,` This statement continues through source line 14; the explanation uses the complete expression rather than only the first visual fragment. This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 6: `parameter int ADDRESS_BITS = 11` This is continuation line 2 of the 10-line logical statement that begins at source line 5. Its exact fragment is 'parameter int ADDRESS_BITS = 11'. The important operands are parameter: the value named 'parameter' in this block; ADDRESS_BITS: the value named 'address bits' in this block. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 7: `) (` This is continuation line 3 of the 10-line logical statement that begins at source line 5. Its exact fragment is ') ('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 8: `input wire clk,` This is continuation line 4 of the 10-line logical statement that begins at source line 5. Its exact fragment is 'input wire clk,'. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 9: `input wire write_enable,` This is continuation line 5 of the 10-line logical statement that begins at source line 5. Its exact fragment is 'input wire write_enable,'. The important operands are input: the value named 'input' in this block; write_enable: a control bit that permits the associated state or storage update. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 10: `input wire [ADDRESS_BITS-1:0] write_address,` This is continuation line 6 of the 10-line logical statement that begins at source line 5. Its exact fragment is 'input wire [ADDRESS_BITS-1:0] write_address,'. The important operands are input: the value named 'input' in this block; ADDRESS_BITS: the value named 'address bits' in this block; write_address: the value named 'write address' in this block. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 11: `input wire [63:0] write_data,` This is continuation line 7 of the 10-line logical statement that begins at source line 5. Its exact fragment is 'input wire [63:0] write_data,'. The important operands are input: the value named 'input' in this block; write_data: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 12: `input wire [ADDRESS_BITS-1:0] read_address,` This is continuation line 8 of the 10-line logical statement that begins at source line 5. Its exact fragment is 'input wire [ADDRESS_BITS-1:0] read_address,'. The important

operands are input: the value named 'input' in this block; ADDRESS_BITS: the value named 'address bits' in this block; read_address: the value named 'read address' in this block. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 13: `output logic [63:0] read_data` This is continuation line 9 of the 10-line logical statement that begins at source line 5. Its exact fragment is 'output logic [63:0] read_data'. The important operands are output: the value named 'output' in this block; read_data: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 14: `);` This is continuation line 10 of the 10-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [63:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 15: `(* ram_style = "block" *) logic [63:0] memory [0:DEPTH-1];` Within the module's structural declarations, this exact statement contributes to the block that stores one local tile of unsigned eight-bit activations for repeated tensor-array reads. The important operands are ram_style: the value named 'ram style' in this block; block: the value named 'block' in this block; memory: local FPGA storage addressed by surrounding index logic.

Line 16: `logic [63:0] memory_read_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, memory_read_data_r is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 19: `if (write_enable) memory[write_address] <= write_data;` This condition selects the following hardware branch only when 'write_enable' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are write_enable: a control bit that permits the associated state or storage update. Context: the clocked process that updates registers only on a rising clock edge.

Line 20: `memory_read_data_r <= memory[read_address];` This makes memory_read_data_r, local FPGA storage addressed by surrounding index logic, equal to 'memory[read_address]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are memory: local FPGA storage addressed by surrounding index logic; read_address: the value named 'read address' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 21: `read_data <= memory_read_data_r;` This makes read_data, the payload carried with the surrounding valid/ready handshake, equal to 'memory_read_data_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are memory_read_data_r: local FPGA storage addressed by surrounding index logic. Context: the clocked process that updates registers only on a rising clock edge.

Line 22: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 24: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 25: `if (DEPTH != (1 << ADDRESS_BITS)) $fatal(1, "activation buffer depth/address mismatch");` This condition selects the following hardware branch only when ‘DEPTH != (1 << ADDRESS_BITS)) $fatal(1, "activation buffer depth/address mismatch"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are DEPTH: the value named ’depth’ in this block; ADDRESS_BITS: the value named ’address bits’ in this block; fatal: the value named ’fatal’ in this block. Context: the module’s structural declarations.

Line 26: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 27: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 29: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that stores one local tile of unsigned eight-bit activations for repeated tensor-array reads.

3 rtl/neuraedge_activation_lut8.sv

Role in the complete system: loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

3.1 Beginner architecture map

Dataflow responsibility Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_activation_lut8 into physical nets and primitives. The source declares 9 input-port lines, 6 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls.

Connections to other blocks This source instantiates end as if, unique as case, endcase as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

3.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_activation_lut8 (
5      input  wire clk,
6      input  wire rst_n,
7      input  wire clear_table,
8      input  wire load_valid,
9      output logic load_ready,
10     input  wire [7:0] load_address,
11     input  wire signed [15:0] load_value,
12     input  wire input_valid,
13     output logic input_ready,
14     input  wire [63:0] input_indices,
15     output logic output_valid,
16     input  wire output_ready,
17     output logic [127:0] output_values,
18     output logic config_error,
19     output logic [8:0] loaded_entries
20 );
21     // Replication gives eight independent lookup ports. A 256-entry table is
22     // small enough for distributed RAM and preserves one eight-value vector per
23     // cycle for GELU, SiLU, sigmoid, tanh, or a model-specific quantized curve.
24     (* ram_style = "distributed" *) logic signed [15:0] lut_mem0 [0:255];
25     (* ram_style = "distributed" *) logic signed [15:0] lut_mem1 [0:255];
26     (* ram_style = "distributed" *) logic signed [15:0] lut_mem2 [0:255];
27     (* ram_style = "distributed" *) logic signed [15:0] lut_mem3 [0:255];
28     (* ram_style = "distributed" *) logic signed [15:0] lut_mem4 [0:255];
29     (* ram_style = "distributed" *) logic signed [15:0] lut_mem5 [0:255];
30     (* ram_style = "distributed" *) logic signed [15:0] lut_mem6 [0:255];
31     (* ram_style = "distributed" *) logic signed [15:0] lut_mem7 [0:255];
32     logic [255:0] loaded_bitmap;
33     logic load_count_pending_r;
34     logic load_count_new_r;
35     logic [1:0] result_fifo_count;
36     logic result_fifo_write_pointer;
37     logic result_fifo_read_pointer;
38     logic [127:0] result_fifo_values [0:1];
39     logic [7:0] result_fifo_loaded_masks [0:1];
40     logic [63:0] lookup_indices_r;
41     logic lookup_index_valid_r;
42     logic [127:0] lookup_mid_values_r;
43     logic [127:0] lookup_loaded_candidates_r;
44     logic [31:0] lookup_high_nibbles_r;
45     logic lookup_mid_valid_r;
46     logic [127:0] lookup_result_r;
47     logic [7:0] lookup_result_loaded_r;
48     logic lookup_result_valid_r;
```

```
    logic [127:0] lookup_values_comb;
    logic [127:0] lookup_loaded_candidates_comb;
    wire signed [15:0] lookup_value0 = lut_mem0[lookup_indices_r[7:0]];
    wire signed [15:0] lookup_value1 = lut_mem1[lookup_indices_r[15:8]];
    wire signed [15:0] lookup_value2 = lut_mem2[lookup_indices_r[23:16]];
    wire signed [15:0] lookup_value3 = lut_mem3[lookup_indices_r[31:24]];
    wire signed [15:0] lookup_value4 = lut_mem4[lookup_indices_r[39:32]];
    wire signed [15:0] lookup_value5 = lut_mem5[lookup_indices_r[47:40]];
    wire signed [15:0] lookup_value6 = lut_mem6[lookup_indices_r[55:48]];
    wire signed [15:0] lookup_value7 = lut_mem7[lookup_indices_r[63:56]];
    wire result_fifo_pop = output_valid && output_ready;
    // Occupancy, rather than downstream ready, controls the lookup stages.
    // This deliberately permits one recovery bubble after a completely full
    // queue and prevents AXI backpressure from becoming a chip-wide data path.
    wire result_fifo_space = result_fifo_count != 2;
    wire result_fifo_push = lookup_result_valid_r && result_fifo_space;
    wire lookup_result_ready = !lookup_result_valid_r || result_fifo_space;
    wire lookup_mid_transfer = lookup_mid_valid_r && lookup_result_ready;
    wire lookup_mid_ready = !lookup_mid_valid_r || lookup_result_ready;
    wire lookup_index_transfer = lookup_index_valid_r && lookup_mid_ready;
    wire lookup_index_ready = !lookup_index_valid_r || lookup_mid_ready;
    wire lookup_input_accept = input_valid && input_ready;

    always_comb begin
        // The load and evaluate datapaths are independent at the acceptance
        // edge: a just-written entry is visible before a newly captured index
        // reaches the lookup stage. Keeping ready independent of peer valid
        // also prevents a valid->ready->controller combinational loop.
        load_ready = result_fifo_count == 0 && !lookup_index_valid_r
            && !lookup_mid_valid_r && !lookup_result_valid_r;
        input_ready = lookup_index_ready;
        output_valid = result_fifo_count != 0;
        output_values = result_fifo_values[result_fifo_read_pointer];
        config_error = output_valid
            && !(&result_fifo_loaded_masks[result_fifo_read_pointer]);

        lookup_values_comb = {lookup_value7, lookup_value6, lookup_value5,
                              lookup_value4, lookup_value3, lookup_value2,
                              lookup_value1, lookup_value0};
        for (int lane = 0; lane < 8; lane++) begin
            for (int group = 0; group < 16; group++) begin
                lookup_loaded_candidates_comb[lane*16+group] =
                    loaded_bitmap[group*16
                        + int'(lookup_indices_r[lane*8 +: 4])];
            end
        end
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            result_fifo_count <= 2'd0;
            result_fifo_write_pointer <= 1'b0;
            result_fifo_read_pointer <= 1'b0;
            result_fifo_values[0] <= 128'd0;
            result_fifo_values[1] <= 128'd0;
            result_fifo_loaded_masks[0] <= 8'd0;
            result_fifo_loaded_masks[1] <= 8'd0;
            lookup_indices_r <= 64'd0;
            lookup_index_valid_r <= 1'b0;
            lookup_mid_values_r <= 128'd0;
            lookup_loaded_candidates_r <= 128'd0;
            lookup_high_nibbles_r <= 32'd0;
            lookup_mid_valid_r <= 1'b0;
            lookup_result_r <= 128'd0;
            lookup_result_loaded_r <= 8'd0;
            lookup_result_valid_r <= 1'b0;
            loaded_bitmap <= 256'd0;
            loaded_entries <= 9'd0;
            load_count_pending_r <= 1'b0;
            load_count_new_r <= 1'b0;
        end else if (clear_table) begin
            result_fifo_count <= 2'd0;
            result_fifo_write_pointer <= 1'b0;
            result_fifo_read_pointer <= 1'b0;
            lookup_index_valid_r <= 1'b0;
            lookup_mid_valid_r <= 1'b0;
            lookup_result_valid_r <= 1'b0;
            loaded_bitmap <= 256'd0;
            loaded_entries <= 9'd0;
            load_count_pending_r <= 1'b0;
            load_count_new_r <= 1'b0;
        end else begin
            load_count_pending_r <= 1'b0;
            if (load_count_pending_r && load_count_new_r) begin
                loaded_entries <= loaded_entries + 1'b1;
            end
            if (load_valid && load_ready) begin
```

```
                lut_mem0[load_address] <= load_value;
                lut_mem1[load_address] <= load_value;
                lut_mem2[load_address] <= load_value;
                lut_mem3[load_address] <= load_value;
                lut_mem4[load_address] <= load_value;
                lut_mem5[load_address] <= load_value;
                lut_mem6[load_address] <= load_value;
                lut_mem7[load_address] <= load_value;
                load_count_pending_r <= 1'b1;
                load_count_new_r <= !loaded_bitmap[load_address];
                loaded_bitmap[load_address] <= 1'b1;
            end

            if (lookup_input_accept) lookup_indices_r <= input_indices;
            unique case ({lookup_input_accept, lookup_index_transfer})
                2'b10: lookup_index_valid_r <= 1'b1;
                2'b01: lookup_index_valid_r <= 1'b0;
                default: begin end
            endcase

            if (lookup_index_transfer) begin
                lookup_mid_values_r <= lookup_values_comb;
                lookup_loaded_candidates_r <= lookup_loaded_candidates_comb;
                for (int lane = 0; lane < 8; lane++) begin
                    lookup_high_nibbles_r[lane*4 +: 4]
                        <= lookup_indices_r[lane*8+4 +: 4];
                end
            end
            unique case ({lookup_index_transfer, lookup_mid_transfer})
                2'b10: lookup_mid_valid_r <= 1'b1;
                2'b01: lookup_mid_valid_r <= 1'b0;
                default: begin end
            endcase

            if (lookup_mid_transfer) begin
                lookup_result_r <= lookup_mid_values_r;
                for (int lane = 0; lane < 8; lane++) begin
                    lookup_result_loaded_r[lane]
                        <= lookup_loaded_candidates_r[lane*16
                            + int'(lookup_high_nibbles_r[lane*4 +: 4])];
                end
            end
            unique case ({lookup_mid_transfer, result_fifo_push})
                2'b10: lookup_result_valid_r <= 1'b1;
                2'b01: lookup_result_valid_r <= 1'b0;
                default: begin end
            endcase

            if (result_fifo_push) begin
                for (int lane = 0; lane < 8; lane++) begin
                    result_fifo_values[result_fifo_write_pointer][lane*16 +: 16]
                        <= lookup_result_loaded_r[lane]
                        ? lookup_result_r[lane*16 +: 16] : 16'd0;
                end
                result_fifo_loaded_masks[result_fifo_write_pointer]
                    <= lookup_result_loaded_r;
                result_fifo_write_pointer <= !result_fifo_write_pointer;
            end

            if (result_fifo_pop) result_fifo_read_pointer <= !result_fifo_read_pointer;

            unique case ({result_fifo_push, result_fifo_pop})
                2'b10: result_fifo_count <= result_fifo_count + 1'b1;
                2'b01: result_fifo_count <= result_fifo_count - 1'b1;
                default: begin end
            endcase
        end
    end
endmodule

`default_nettype wire
```

3.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_activation_lut8 (` This opens neuraedge_activation_lut8, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 5: `input wire clk,` This declares clk as a input port with width one bit. It carries the clock that makes all registers advance together across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 6: `input wire rst_n,` This declares rst_n as a input port with width one bit. It carries the active-low synchronous reset across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 7: `input wire clear_table,` This declares clear_table as a input port with width one bit. It carries the value named 'clear table' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 8: `input wire load_valid,` This declares load_valid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 9: `output logic load_ready,` This declares load_ready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 10: `input wire [7:0] load_address,` This declares load_address as a input port with width [7:0]. It carries the value named 'load address' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 11: `input wire signed [15:0] load_value,` This declares load_value as a input port with width [15:0]. It carries the value named 'load value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 12: `input wire input_valid,` This declares input_valid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 13: `output logic input_ready,` This declares input_ready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 14: `input wire [63:0] input_indices,` This declares input_indices as a input port with width [63:0]. It carries the value named 'input indices' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 15: `output logic output_valid,` This declares output_valid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 16: `input wire output_ready,` This declares output_ready as a input port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 17: `output logic [127:0] output_values,` This declares output_values as a output port with width [127:0]. It carries the value named 'output values' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 18: `output logic config_error,` This declares config_error as a output port with width one bit. It carries the value named 'config error' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 19: `output logic [8:0] loaded_entries` This declares loaded_entries as a output port with width [8:0]. It carries the value named 'loaded entries' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 20: `);` Within the module's structural declarations, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The expression is a literal or structural operator and has no hidden software state.

Line 21: `// Replication gives eight independent lookup ports. A 256-entry table is` Design intent: Replication gives eight independent lookup ports. A 256-entry table is This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 22: `// small enough for distributed RAM and preserves one eight-value vector per` Design intent: small enough for distributed RAM and preserves one eight-value vector per This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 23: `// cycle for GELU, SiLU, sigmoid, tanh, or a model-specific quantized curve.` Design intent: cycle for GELU, SiLU, sigmoid, tanh, or a model-specific quantized curve. This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 24: `(* ram_style = "distributed" *) logic signed [15:0] lut_mem0 [0:255];` Within the module's structural declarations, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block; lut_mem0: the value named 'lut mem0' in this block.

Line 25: `(* ram_style = "distributed" *) logic signed [15:0] lut_mem1 [0:255];` Within the module's structural declarations, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block; lut_mem1: the value named 'lut mem1' in this block.

Line 26: `(* ram_style = "distributed" *) logic signed [15:0] lut_mem2 [0:255];` Within the module's structural declarations, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block; lut_mem2: the value named 'lut mem2' in this block.

Line 27: `(* ram_style = "distributed" *) logic signed [15:0] lut_mem3 [0:255];` Within the module's structural declarations, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block; lut_mem3: the value named 'lut mem3' in this block.

Line 28: `(* ram_style = "distributed" *) logic signed [15:0] lut_mem4 [0:255];` Within the module's structural declarations, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block; lut_mem4: the value named 'lut mem4' in this block.

Line 29: `(* ram_style = "distributed" *) logic signed [15:0] lut_mem5 [0:255];` Within the module's structural declarations, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block; lut_mem5: the value named 'lut mem5' in this block.

Line 30: `(* ram_style = "distributed" *) logic signed [15:0] lut_mem6 [0:255];` Within the module's structural declarations, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block; lut_mem6: the value named 'lut mem6' in this block.

Line 31: `(* ram_style = "distributed" *) logic signed [15:0] lut_mem7 [0:255];` Within the module's structural declarations, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block; lut_mem7: the value named 'lut mem7' in this block.

Line 32: `logic [255:0] loaded_bitmap;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, loaded_bitmap is the value named 'loaded bitmap' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `logic load_count_pending_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_count_pending_r is the value named 'load count pending r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic load_count_new_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_count_new_r is the value named 'load count new r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 35: `logic [1:0] result_fifo_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `logic result_fifo_write_pointer;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_write_pointer is the value named 'result fifo write pointer' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 37: `logic result_fifo_read_pointer;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_read_pointer is the value named 'result fifo read pointer' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 38: `logic [127:0] result_fifo_values [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_values is the value named 'result fifo values' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 39: `logic [7:0] result_fifo_loaded_masks [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_loaded_masks is the value named 'result fifo loaded masks' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 40: `logic [63:0] lookup_indices_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_indices_r is the value named 'lookup indices r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 41: `logic lookup_index_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_index_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 42: `logic [127:0] lookup_mid_values_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_mid_values_r is the value named 'lookup mid values r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 43: `logic [127:0] lookup_loaded_candidates_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_loaded_candidates_r is the value named 'lookup loaded candidates r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 44: `logic [31:0] lookup_high_nibbles_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_high_nibbles_r is the value named 'lookup high nibbles r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 45: `logic lookup_mid_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_mid_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 46: `logic [127:0] lookup_result_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_result_r is the value named 'lookup result r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 47: `logic [7:0] lookup_result_loaded_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_result_loaded_r is the value named 'lookup result loaded r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 48: `logic lookup_result_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_result_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 49: `logic [127:0] lookup_values_comb;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_values_comb is the value named 'lookup values comb' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 50: `logic [127:0] lookup_loaded_candidates_comb;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_loaded_candidates_comb is the value named 'lookup loaded candidates comb' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 51: `wire signed [15:0] lookup_value0 = lut_mem0[lookup_indices_r[7:0]];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_value0 is the value named 'lookup value0' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 52: `wire signed [15:0] lookup_value1 = lut_mem1[lookup_indices_r[15:8]];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_value1 is the value named 'lookup value1' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 53: `wire signed [15:0] lookup_value2 = lut_mem2[lookup_indices_r[23:16]];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_value2 is the value named 'lookup value2' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 54: `wire signed [15:0] lookup_value3 = lut_mem3[lookup_indices_r[31:24]];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_value3 is the value named 'lookup value3' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 55: `wire signed [15:0] lookup_value4 = lut_mem4[lookup_indices_r[39:32]];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_value4 is the value named 'lookup value4' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 56: `wire signed [15:0] lookup_value5 = lut_mem5[lookup_indices_r[47:40]];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_value5 is the value named 'lookup value5' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 57: `wire signed [15:0] lookup_value6 = lut_mem6[lookup_indices_r[55:48]];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_value6 is the value named 'lookup value6' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 58: `wire signed [15:0] lookup_value7 = lut_mem7[lookup_indices_r[63:56]];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_value7 is the value named 'lookup value7' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 59: `wire result_fifo_pop = output_valid && output_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_pop is the value named 'result fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 60: `// Occupancy, rather than downstream ready, controls the lookup stages.` Design intent: Occupancy, rather than downstream ready, controls the lookup stages. This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 61: `// This deliberately permits one recovery bubble after a completely full` Design intent: This deliberately permits one recovery bubble after a completely full This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 62: `// queue and prevents AXI backpressure from becoming a chip-wide data path.` Design intent: queue and prevents AXI backpressure from becoming a chip-wide data path. This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 63: `wire result_fifo_space = result_fifo_count != 2;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_space is the value named 'result fifo space' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 64: `wire result_fifo_push = lookup_result_valid_r && result_fifo_space;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_push is the value named 'result fifo push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 65: `wire lookup_result_ready = !lookup_result_valid_r || result_fifo_space;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_result_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 66: `wire lookup_mid_transfer = lookup_mid_valid_r && lookup_result_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_mid_transfer is the value named 'lookup mid transfer' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 67: `wire lookup_mid_ready = !lookup_mid_valid_r || lookup_result_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_mid_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 68: `wire lookup_index_transfer = lookup_index_valid_r && lookup_mid_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_index_transfer is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 69: `wire lookup_index_ready = !lookup_index_valid_r || lookup_mid_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_index_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 70: `wire lookup_input_accept = input_valid && input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lookup_input_accept is the value named 'lookup input accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 72: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 73: `// The load and evaluate datapaths are independent at the acceptance` Design intent: The load and evaluate datapaths are independent at the acceptance This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 74: `// edge: a just-written entry is visible before a newly captured index` Design intent: edge: a just-written entry is visible before a newly captured index This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 75: `// reaches the lookup stage. Keeping ready independent of peer valid` Design intent: reaches the lookup stage. Keeping ready independent of peer valid This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 76: `// also prevents a valid->ready->controller combinational loop.` Design intent: also prevents a valid->ready->controller combinational loop. This comment anchors the surrounding logic in the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

Line 77: `load_ready = result_fifo_count == 0 && !lookup_index_valid_r` This statement continues through source line 78; the explanation uses the complete expression rather than only the first visual fragment. This makes load_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_fifo_count == 0 && !lookup_index_valid_r && !lookup_mid_valid_r && !lookup_result_valid_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_fifo_count: an integer count retained for control or measurable telemetry; lookup_index_valid_r: the producer's statement that its current payload is meaningful; lookup_mid_valid_r: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 78: `&& !lookup_mid_valid_r && !lookup_result_valid_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 77. Its exact fragment is '&& !lookup_mid_valid_r && !lookup_result_valid_r;'. The important operands are lookup_mid_valid_r: the producer's statement that its current payload is meaningful; lookup_result_valid_r: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This makes load_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_fifo_count == 0 && !lookup_index_valid_r && !lookup_mid_valid_r && !lookup_result_valid_r'. It is driven

by combinational or procedural logic in the current evaluation. The important operands are result_fifo_count: an integer count retained for control or measurable telemetry; lookup_index_valid_r: the producer's statement that its current payload is meaningful; lookup_mid_valid_r: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 79: `input_ready = lookup_index_ready;` This makes input_ready, the consumer's statement that it can accept a payload this cycle, equal to 'lookup_index_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lookup_index_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 80: `output_valid = result_fifo_count != 0;` This makes output_valid, the producer's statement that its current payload is meaningful, equal to 'result_fifo_count != 0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_fifo_count: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 81: `output_values = result_fifo_values[result_fifo_read_pointer];` This makes output_values, the value named 'output values' in this block, equal to 'result_fifo_values[result_fifo_read_pointer]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_fifo_values: the value named 'result fifo values' in this block; result_fifo_read_pointer: the value named 'result fifo read pointer' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 82: `config_error = output_valid` This statement continues through source line 83; the explanation uses the complete expression rather than only the first visual fragment. This makes config_error, the value named 'config error' in this block, equal to 'output_valid && !(&result_fifo_loaded_masks[result_fifo_read_pointer])'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_valid: the producer's statement that its current payload is meaningful; result_fifo_loaded_masks: the value named 'result fifo loaded masks' in this block; result_fifo_read_pointer: the value named 'result fifo read pointer' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 83: `&& !(&result_fifo_loaded_masks[result_fifo_read_pointer]);` This is continuation line 2 of the 2-line logical statement that begins at source line 82. Its exact fragment is '&& !(&result_fifo_loaded_masks[result_fifo_read_pointer]);'. The important operands are result_fifo_loaded_masks: the value named 'result fifo loaded masks' in this block; result_fifo_read_pointer: the value named 'result fifo read pointer' in this block. Read together, the complete operation is: This makes config_error, the value named 'config error' in this block, equal to 'output_valid && !(&result_fifo_loaded_masks[result_fifo_read_pointer])'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_valid: the producer's statement that its current payload is meaningful; result_fifo_loaded_masks: the value named 'result fifo loaded masks' in this block; result_fifo_read_pointer: the value named 'result fifo read pointer' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 85: `lookup_values_comb = {lookup_value7, lookup_value6, lookup_value5,` This statement continues through source line 87; the explanation uses the complete expression rather than only the first visual fragment. This makes lookup_values_comb, the value named 'lookup values comb' in this block, equal to '{lookup_value7, lookup_value6, lookup_value5, lookup_value4, lookup_value3, lookup_value2, lookup_value1, lookup_value0}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lookup_value7: the value named 'lookup value7' in this block; lookup_value6: the value named 'lookup value6' in this block; lookup_value5: the value named 'lookup value5' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 86: `lookup_value4, lookup_value3, lookup_value2,` This is continuation line 2 of the 3-line logical statement that begins at source line 85. Its exact fragment is 'lookup_value4, lookup_value3, lookup_value2,'. The important operands are lookup_value4: the value named 'lookup value4' in this block; lookup_value3: the value named 'lookup value3' in this block; lookup_value2: the value named 'lookup value2' in this block. Read together, the complete operation is: This makes lookup_values_comb, the value named 'lookup values comb' in this block, equal to '{lookup_value7, lookup_value6, lookup_value5, lookup_value4, lookup_value3, lookup_value2, lookup_value1, lookup_value0}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lookup_value7: the

value named 'lookup value7' in this block; lookup_value6: the value named 'lookup value6' in this block; lookup_value5: the value named 'lookup value5' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 87: `lookup_value1, lookup_value0};` This is continuation line 3 of the 3-line logical statement that begins at source line 85. Its exact fragment is 'lookup_value1, lookup_value0};'. The important operands are lookup_value1: the value named 'lookup value1' in this block; lookup_value0: the value named 'lookup value0' in this block. Read together, the complete operation is: This makes lookup_values_comb, the value named 'lookup values comb' in this block, equal to '{lookup_value7, lookup_value6, lookup_value5, lookup_value4, lookup_value3, lookup_value2, lookup_value1, lookup_value0}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lookup_value7: the value named 'lookup value7' in this block; lookup_value6: the value named 'lookup value6' in this block; lookup_value5: the value named 'lookup value5' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 88: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 89: `for (int group = 0; group < 16; group++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 90: `lookup_loaded_candidates_comb[lane*16+group] =` This statement continues through source line 92; the explanation uses the complete expression rather than only the first visual fragment. Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_loaded_candidates_comb: the value named 'lookup loaded candidates comb' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; group: the value named 'group' in this block.

Line 91: `loaded_bitmap[group*16` This is continuation line 2 of the 3-line logical statement that begins at source line 90. Its exact fragment is 'loaded_bitmap[group*16'. The important operands are loaded_bitmap: the value named 'loaded bitmap' in this block; group: the value named 'group' in this block. Read together, the complete operation is: Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_loaded_candidates_comb: the value named 'lookup loaded candidates comb' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; group: the value named 'group' in this block.

Line 92: `+ int'(lookup_indices_r[lane*8 +: 4])];` This is continuation line 3 of the 3-line logical statement that begins at source line 90. Its exact fragment is '+ int'(lookup_indices_r[lane*8 +: 4])];'. The important operands are lookup_indices_r: the value named 'lookup indices r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_loaded_candidates_comb: the value named 'lookup loaded candidates comb' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; group: the value named 'group' in this block.

Line 93: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 94: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 95: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 97: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 98: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 99: `result_fifo_count <= 2'd0;` This makes result_fifo_count, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 100: `result_fifo_write_pointer <= 1'b0;` This makes result_fifo_write_pointer, the value named 'result fifo write pointer' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 101: `result_fifo_read_pointer <= 1'b0;` This makes result_fifo_read_pointer, the value named 'result fifo read pointer' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 102: `result_fifo_values[0] <= 128'd0;` This makes result_fifo_values[0], the value named 'result fifo values' in this block, equal to '128'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 103: `result_fifo_values[1] <= 128'd0;` This makes result_fifo_values[1], the value named 'result fifo values' in this block, equal to '128'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 104: `result_fifo_loaded_masks[0] <= 8'd0;` This makes result_fifo_loaded_masks[0], the value named 'result fifo loaded masks' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 105: `result_fifo_loaded_masks[1] <= 8'd0;` This makes result_fifo_loaded_masks[1], the value named 'result fifo loaded masks' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 106: `lookup_indices_r <= 64'd0;` This makes lookup_indices_r, the value named 'lookup indices r' in this block, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 107: `lookup_index_valid_r <= 1'b0;` This makes lookup_index_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 108: `lookup_mid_values_r <= 128'd0;` This makes lookup_mid_values_r, the value named 'lookup mid values r' in this block, equal to '128'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 109: `lookup_loaded_candidates_r <= 128'd0;` This makes lookup_loaded_candidates_r, the value named 'lookup loaded candidates r' in this block, equal to '128'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 110: `lookup_high_nibbles_r <= 32'd0;` This makes lookup_high_nibbles_r, the value named 'lookup high nibbles r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 111: `lookup_mid_valid_r <= 1'b0;` This makes lookup_mid_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 112: `lookup_result_r <= 128'd0;` This makes lookup_result_r, the value named 'lookup result r' in this block, equal to '128'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 113: `lookup_result_loaded_r <= 8'd0;` This makes lookup_result_loaded_r, the value named 'lookup result loaded r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 114: `lookup_result_valid_r <= 1'b0;` This makes lookup_result_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 115: `loaded_bitmap <= 256'd0;` This makes loaded_bitmap, the value named 'loaded bitmap' in this block, equal to '256'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 116: `loaded_entries <= 9'd0;` This makes loaded_entries, the value named 'loaded entries' in this block, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 117: `load_count_pending_r <= 1'b0;` This makes load_count_pending_r, the value named 'load count pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 118: `load_count_new_r <= 1'b0;` This makes load_count_new_r, the value named 'load count new r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 119: `end else if (clear_table) begin` This condition selects the following hardware branch only when ‘clear_table‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_table: the value named ’clear table’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 120: `result_fifo_count <= 2'd0;` This makes result_fifo_count, an integer count retained for control or measurable telemetry, equal to ‘2’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 121: `result_fifo_write_pointer <= 1'b0;` This makes result_fifo_write_pointer, the value named ’result fifo write pointer’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 122: `result_fifo_read_pointer <= 1'b0;` This makes result_fifo_read_pointer, the value named ’result fifo read pointer’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 123: `lookup_index_valid_r <= 1'b0;` This makes lookup_index_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 124: `lookup_mid_valid_r <= 1'b0;` This makes lookup_mid_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 125: `lookup_result_valid_r <= 1'b0;` This makes lookup_result_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 126: `loaded_bitmap <= 256'd0;` This makes loaded_bitmap, the value named ’loaded bitmap’ in this block, equal to ‘256’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 127: `loaded_entries <= 9'd0;` This makes loaded_entries, the value named ’loaded entries’ in this block, equal to ‘9’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 128: `load_count_pending_r <= 1'b0;` This makes load_count_pending_r, the value named ’load count pending r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 129: `load_count_new_r <= 1'b0;` This makes load_count_new_r, the value named ’load count new r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 130: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 131: `load_count_pending_r <= 1'b0;` This makes load_count_pending_r, the value named 'load count pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 132: `if (load_count_pending_r && load_count_new_r) begin` This condition selects the following hardware branch only when 'load_count_pending_r && load_count_new_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_count_pending_r: the value named 'load count pending r' in this block; load_count_new_r: the value named 'load count new r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 133: `loaded_entries <= loaded_entries + 1'b1;` This makes loaded_entries, the value named 'loaded entries' in this block, equal to 'loaded_entries + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are loaded_entries: the value named 'loaded entries' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 134: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 135: `if (load_valid && load_ready) begin` This condition selects the following hardware branch only when 'load_valid && load_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_valid: the producer's statement that its current payload is meaningful; load_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 136: `lut_mem0[load_address] <= load_value;` This makes lut_mem0[load_address], the value named 'lut mem0' in this block, equal to 'load_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_value: the value named 'load value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 137: `lut_mem1[load_address] <= load_value;` This makes lut_mem1[load_address], the value named 'lut mem1' in this block, equal to 'load_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_value: the value named 'load value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 138: `lut_mem2[load_address] <= load_value;` This makes lut_mem2[load_address], the value named 'lut mem2' in this block, equal to 'load_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_value: the value named 'load value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 139: `lut_mem3[load_address] <= load_value;` This makes lut_mem3[load_address], the value named 'lut mem3' in this block, equal to 'load_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_value: the value named 'load value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 140: `lut_mem4[load_address] <= load_value;` This makes lut_mem4[load_address], the value named 'lut mem4' in this block, equal to 'load_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_value: the value named 'load value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 141: `lut_mem5[load_address] <= load_value;` This makes lut_mem5[load_address], the value named 'lut mem5' in this block, equal to 'load_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_value: the value named 'load value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 142: `lut_mem6[load_address] <= load_value;` This makes lut_mem6[load_address], the value named 'lut mem6' in this block, equal to 'load_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_value: the value named 'load value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 143: `lut_mem7[load_address] <= load_value;` This makes lut_mem7[load_address], the value named 'lut mem7' in this block, equal to 'load_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_value: the value named 'load value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 144: `load_count_pending_r <= 1'b1;` This makes load_count_pending_r, the value named 'load count pending r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 145: `load_count_new_r <= !loaded_bitmap[load_address];` This makes load_count_new_r, the value named 'load count new r' in this block, equal to '!loaded_bitmap[load_address]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are loaded_bitmap: the value named 'loaded bitmap' in this block; load_address: the value named 'load address' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 146: `loaded_bitmap[load_address] <= 1'b1;` This makes loaded_bitmap[load_address], the value named 'loaded bitmap' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 147: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 149: `if (lookup_input_accept) lookup_indices_r <= input_indices;` This condition selects the following hardware branch only when 'lookup_input_accept' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lookup_input_accept: the value named 'lookup input accept' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 150: `unique case ({lookup_input_accept, lookup_index_transfer})` This starts a multiplexer/state decode selected by '{lookup_input_accept, lookup_index_transfer}'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 151: `2'b10: lookup_index_valid_r <= 1'b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are b10: the value named 'b10' in this block; lookup_index_valid_r: the producer's statement that its current payload is meaningful; b1: the value named 'b1' in this block.

Line 152: `2'b01: lookup_index_valid_r <= 1'b0;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are b01: the value named 'b01' in this block; lookup_index_valid_r: the producer's statement that its current payload is meaningful; b0: the value named 'b0' in this block.

Line 153: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin end‘; context is the clocked process that updates registers only on a rising clock edge.

Line 154: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 156: `if (lookup_index_transfer) begin` This condition selects the following hardware branch only when ‘lookup_index_transfer‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lookup_index_transfer: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge.

Line 157: `lookup_mid_values_r <= lookup_values_comb;` This makes lookup_mid_values_r, the value named ’lookup mid values r’ in this block, equal to ‘lookup_values_comb‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are lookup_values_comb: the value named ’lookup values comb’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 158: `lookup_loaded_candidates_r <= lookup_loaded_candidates_comb;` This makes lookup_loaded_candidates_r, the value named ’lookup loaded candidates r’ in this block, equal to ‘lookup_loaded_candidates_comb‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are lookup_loaded_candidates_comb: the value named ’lookup loaded candidates comb’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 159: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 160: `lookup_high_nibbles_r[lane*4 +: 4]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_high_nibbles_r: the value named ’lookup high nibbles r’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 161: `<= lookup_indices_r[lane*8+4 +: 4];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_indices_r: the value named ’lookup indices r’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 162: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 163: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 164: `unique case ({lookup_index_transfer, lookup_mid_transfer})` This starts a multiplexer/state decode selected by ‘{lookup_index_transfer, lookup_mid_transfer}‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 165: `2’b10: lookup_mid_valid_r <= 1’b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are b10: the value named ’b10’ in this block; lookup_mid_valid_r: the producer’s statement that its current payload is meaningful; b1: the value named ’b1’ in this block.

Line 166: `2'b01: lookup_mid_valid_r <= 1'b0;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are b01: the value named 'b01' in this block; lookup_mid_valid_r: the producer's statement that its current payload is meaningful; b0: the value named 'b0' in this block.

Line 167: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the clocked process that updates registers only on a rising clock edge.

Line 168: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 170: `if (lookup_mid_transfer) begin` This condition selects the following hardware branch only when 'lookup_mid_tran is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lookup_mid_transfer: the value named 'lookup mid transfer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 171: `lookup_result_r <= lookup_mid_values_r;` This makes lookup_result_r, the value named 'lookup result r' in this block, equal to 'lookup_mid_values_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lookup_mid_values_r: the value named 'lookup mid values r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 172: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 173: `lookup_result_loaded_r[lane]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_result_loaded_r: the value named 'lookup result loaded r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 174: `<= lookup_loaded_candidates_r[lane*16` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_loaded_candidates_r: the value named 'lookup loaded candidates r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 175: `+ int'(lookup_high_nibbles_r[lane*4 +: 4])];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_high_nibbles_r: the value named 'lookup high nibbles r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 176: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 177: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 178: `unique case ({lookup_mid_transfer, result_fifo_push})` This starts a multiplexer/state decode selected by '{lookup_mid_transfer, result_fifo_push}'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 179: `2'b10: lookup_result_valid_r <= 1'b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are b10: the value named 'b10' in this block; lookup_result_valid_r: the producer's statement that its current payload is meaningful; b1: the value named 'b1' in this block.

Line 180: `2'b01: lookup_result_valid_r <= 1'b0;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are b01: the value named 'b01' in this block; lookup_result_valid_r: the producer's statement that its current payload is meaningful; b0: the value named 'b0' in this block.

Line 181: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the clocked process that updates registers only on a rising clock edge.

Line 182: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 184: `if (result_fifo_push) begin` This condition selects the following hardware branch only when 'result_fifo_push' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_fifo_push: the value named 'result fifo push' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 185: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 186: `result_fifo_values[result_fifo_write_pointer][lane*16 +: 16]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are result_fifo_values: the value named 'result fifo values' in this block; result_fifo_write_pointer: the value named 'result fifo write pointer' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 187: `<= lookup_result_loaded_r[lane]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_result_loaded_r: the value named 'lookup result loaded r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 188: `? lookup_result_r[lane*16 +: 16] : 16'd0;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_result_r: the value named 'lookup result r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; d0: the value named 'd0' in this block.

Line 189: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 190: `result_fifo_loaded_masks[result_fifo_write_pointer]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are result_fifo_loaded_masks: the value named 'result fifo loaded masks' in this block; result_fifo_write_pointer: the value named 'result fifo write pointer' in this block.

Line 191: `<= lookup_result_loaded_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are lookup_result_loaded_r: the value named 'lookup result loaded r' in this block.

Line 192: `result_fifo_write_pointer <= !result_fifo_write_pointer;` This makes result_fifo_write_pointer, the value named 'result fifo write pointer' in this block, equal to '!result_fifo_write_pointer'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_fifo_write_pointer: the value named 'result fifo write pointer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 193: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 195: `if (result_fifo_pop) result_fifo_read_pointer <= !result_fifo_read_pointer;` This condition selects the following hardware branch only when 'result_fifo_pop' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_fifo_pop: the value named 'result fifo pop' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 197: `unique case ({result_fifo_push, result_fifo_pop})` This starts a multiplexer/state decode selected by '{result_fifo_push, result_fifo_pop}'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 198: `2'b10: result_fifo_count <= result_fifo_count + 1'b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are b10: the value named 'b10' in this block; result_fifo_count: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block.

Line 199: `2'b01: result_fifo_count <= result_fifo_count - 1'b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes. The important operands are b01: the value named 'b01' in this block; result_fifo_count: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block.

Line 200: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the clocked process that updates registers only on a rising clock edge.

Line 201: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 202: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 203: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 204: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 206: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that loads and evaluates a programmable 256-entry activation lookup table on eight lanes.

4 rtl/neuraedge_active_index_buffer.sv

Role in the complete system: stores the reduction indices that remain after all-zero activation positions are compacted.

4.1 Beginner architecture map

Dataflow responsibility While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_active_index_buffer into physical nets and primitives. The source declares 6 input-port lines, 1 output-port lines, 1 clocked process(es), and 0 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls.

Connections to other blocks This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

4.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  // Simple-dual-port storage for the compacted list of nonzero activation tiles.
5  // Both ports are synchronous, matching the canonical Vivado BRAM template.
6  module neuraedge_active_index_buffer #(
7      parameter int DEPTH = 2048,
8      parameter int ADDRESS_BITS = 11
9  ) (
10     input  wire clk,
11     input  wire write_enable,
12     input  wire [ADDRESS_BITS-1:0] write_address,
13     input  wire [ADDRESS_BITS-1:0] write_data,
14     input  wire read_enable,
15     input  wire [ADDRESS_BITS-1:0] read_address,
16     output logic [ADDRESS_BITS-1:0] read_data
17 );
18     (* ram_style = "block" *) logic [ADDRESS_BITS-1:0] memory [0:DEPTH-1];
19
20     always_ff @(posedge clk) begin
21         if (write_enable) memory[write_address] <= write_data;
22         if (read_enable) read_data <= memory[read_address];
23     end
24
25     initial begin
26         if (DEPTH != (1 << ADDRESS_BITS)) begin
27             $fatal(1, "active-index depth must equal 2^ADDRESS_BITS");
28         end
29     end
30 endmodule
31
32 `default_nettype wire
```

4.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `// Simple-dual-port storage for the compacted list of nonzero activation tiles.` Design intent: Simple-dual-port storage for the compacted list of nonzero activation tiles. This comment anchors the surrounding logic in the block that stores the reduction indices that remain after all-zero activation positions are compacted.

Line 5: `// Both ports are synchronous, matching the canonical Vivado BRAM template.` Design intent: Both ports are synchronous, matching the canonical Vivado BRAM template. This comment anchors the surrounding logic in the block that stores the reduction indices that remain after all-zero activation positions are compacted.

Line 6: `module neuraedge_active_index_buffer #(` This opens neuraedge_active_index_buffer, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that stores the reduction indices that remain after all-zero activation positions are compacted.

Line 7: `parameter int DEPTH = 2048,` This statement continues through source line 17; the explanation uses the complete expression rather than only the first visual fragment. This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 8: `parameter int ADDRESS_BITS = 11` This is continuation line 2 of the 11-line logical statement that begins at source line 7. Its exact fragment is 'parameter int ADDRESS_BITS = 11'. The important operands are parameter: the value named 'parameter' in this block; ADDRESS_BITS: the value named 'address bits' in this block. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 9: `) (` This is continuation line 3 of the 11-line logical statement that begins at source line 7. Its exact fragment is ') ('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 10: `input wire clk,` This is continuation line 4 of the 11-line logical statement that begins at source line 7. Its exact fragment is 'input wire clk,'. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 11: `input wire write_enable,` This is continuation line 5 of the 11-line logical statement that begins at source line 7. Its exact fragment is 'input wire write_enable,'. The important operands are input: the value named 'input' in this block; write_enable: a control bit that permits the associated state or storage update. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 12: `input wire [ADDRESS_BITS-1:0] write_address,` This is continuation line 6 of the 11-line logical statement that begins at source line 7. Its exact fragment is 'input wire [ADDRESS_BITS-1:0] write_address,'. The important operands are input: the value named 'input' in this block; ADDRESS_BITS: the value named 'address bits'

in this block; write_address: the value named 'write address' in this block. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 13: `input wire [ADDRESS_BITS-1:0] write_data,` This is continuation line 7 of the 11-line logical statement that begins at source line 7. Its exact fragment is 'input wire [ADDRESS_BITS-1:0] write_data,'. The important operands are input: the value named 'input' in this block; ADDRESS_BITS: the value named 'address bits' in this block; write_data: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 14: `input wire read_enable,` This is continuation line 8 of the 11-line logical statement that begins at source line 7. Its exact fragment is 'input wire read_enable,'. The important operands are input: the value named 'input' in this block; read_enable: a control bit that permits the associated state or storage update. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 15: `input wire [ADDRESS_BITS-1:0] read_address,` This is continuation line 9 of the 11-line logical statement that begins at source line 7. Its exact fragment is 'input wire [ADDRESS_BITS-1:0] read_address,'. The important operands are input: the value named 'input' in this block; ADDRESS_BITS: the value named 'address bits' in this block; read_address: the value named 'read address' in this block. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 16: `output logic [ADDRESS_BITS-1:0] read_data` This is continuation line 10 of the 11-line logical statement that begins at source line 7. Its exact fragment is 'output logic [ADDRESS_BITS-1:0] read_data'. The important operands are output: the value named 'output' in this block; ADDRESS_BITS: the value named 'address bits' in this block; read_data: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 17: `);` This is continuation line 11 of the 11-line logical statement that begins at source line 7. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes DEPTH to 2048, parameter int ADDRESS_BITS = 11) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [ADDRESS_BITS-1:0] write_data, input wire read_enable, input wire [ADDRESS_BITS-1:0] read_address, output logic [ADDRESS_BITS-1:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 18: `(* ram_style = "block" *) logic [ADDRESS_BITS-1:0] memory [0:DEPTH-1];` Within the module's structural declarations, this exact statement contributes to the block that stores the reduction indices that remain after all-zero activation positions are compacted. The important operands are ram_style: the value named 'ram style' in this block; block: the value named 'block' in this block; ADDRESS_BITS: the value named 'address bits' in this block.

Line 20: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 21: `if (write_enable) memory[write_address] <= write_data;` This condition selects the following hardware branch only when 'write_enable' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are write_enable: a control bit that permits the associated state or storage update. Context: the clocked process that updates registers only on a rising clock edge.

Line 22: `if (read_enable) read_data <= memory[read_address];` This condition selects the following hardware branch only when 'read_enable' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_enable: a control bit that permits the associated state or storage update. Context: the clocked process that updates registers only on a rising clock edge.

Line 23: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 25: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 26: `if (DEPTH != (1 << ADDRESS_BITS)) begin` This condition selects the following hardware branch only when 'DEPTH != (1 << ADDRESS_BITS)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are DEPTH: the value named 'depth' in this block; ADDRESS_BITS: the value named 'address bits' in this block. Context: the module's structural declarations.

Line 27: `$fatal(1, "active-index depth must equal 2^ADDRESS_BITS");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 28: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 29: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 30: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 32: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that stores the reduction indices that remain after all-zero activation positions are compacted.

5 rtl/neuraedge_alpha_xnor.sv

Role in the complete system: computes alpha-weighted XNOR spiking similarity and streams its accumulated scores.

5.1 Beginner architecture map

Dataflow responsibility The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_alpha_xnor into physical nets and primitives. The source declares 8 input-port lines, 6 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_ACCUMULATE, ST_IDLE, ST_MASK_CHUNK, ST_OUTPUT, ST_POPCOUNT, ST_POPCOUNT_FINAL, ST_POPCOUNT_HALF, ST_POPCOUNT_PARTIAL, ST_SCORE_ADD ST_SCORE_LATCH, ST_SCORE_MULTIPLY, ST_VALIDATE. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates end as for, unique as case. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

5.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_alpha_xnor #(
5      parameter int VECTOR_BITS = 256
6  ) (
7      input  wire clk,
8      input  wire rst_n,
9      input  wire input_valid,
10     output logic input_ready,
11     input  wire [VECTOR_BITS-1:0] query_spikes,
12     input  wire [VECTOR_BITS-1:0] key_spikes,
13     input  wire [8:0] active_bits,
14     input  wire [8:0] alpha_q8,
15     output logic output_valid,
16     input  wire output_ready,
17     output logic [17:0] score_q8,
18     output logic [8:0] spike_matches,
19     output logic [8:0] zero_matches,
20     output logic config_error
21 );
22     localparam int CHUNK_BITS = 32;
23     localparam int CHUNKS = VECTOR_BITS / CHUNK_BITS;
24
25     typedef enum logic [3:0] {
26         ST_IDLE = 4'd0,
27         ST_MASK_CHUNK = 4'd1,
28         ST_ACCUMULATE = 4'd2,
29         ST_SCORE_MULTIPLY = 4'd3,
30         ST_SCORE_ADD = 4'd4,
31         ST_OUTPUT = 4'd5,
32         ST_SCORE_LATCH = 4'd6,
33         ST_POPCOUNT = 4'd7,
34         ST_POPCOUNT_PARTIAL = 4'd8,
35         ST_VALIDATE = 4'd9,
36         ST_POPCOUNT_HALF = 4'd10,
37         ST_POPCOUNT_FINAL = 4'd11
38     } state_t;
39
40     state_t state;
41     logic [VECTOR_BITS-1:0] query_r;
42     logic [VECTOR_BITS-1:0] key_r;
43     logic [8:0] active_bits_r;
```

```
    logic [8:0] alpha_q8_r;
    logic [$clog2(CHUNKS)-1:0] chunk_index;
    logic [31:0] spike_mask_r;
    logic [31:0] zero_mask_r;
    logic [8:0] spike_count_r;
    logic [8:0] zero_count_r;
    logic [5:0] spike_chunk_count_r;
    logic [5:0] zero_chunk_count_r;
    logic [2:0] spike_nibble_count_r [0:7];
    logic [2:0] zero_nibble_count_r [0:7];
    logic [3:0] spike_pair_count_r [0:3];
    logic [3:0] zero_pair_count_r [0:3];
    logic [4:0] spike_half_count_r [0:1];
    logic [4:0] zero_half_count_r [0:1];
    logic [8:0] final_spike_count_r;
    logic [8:0] final_zero_count_r;
    (* dont_touch = "yes" *) logic [8:0] score_spike_operand_r;
    (* dont_touch = "yes" *) logic [8:0] score_zero_operand_r;
    logic [17:0] alpha_product_r;

    (* use_dsp = "yes" *) wire [17:0] alpha_product =
        score_zero_operand_r * alpha_q8_r;

    function automatic logic [2:0] popcount4(input logic [3:0] value);
        logic [1:0] low_pair;
        logic [1:0] high_pair;
        begin
            low_pair = value[0] + value[1];
            high_pair = value[2] + value[3];
            popcount4 = low_pair + high_pair;
        end
    endfunction

    always_comb begin
        input_ready = state == ST_IDLE;
        output_valid = state == ST_OUTPUT;
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            state <= ST_IDLE;
            query_r <= '0;
            key_r <= '0;
            active_bits_r <= 9'd0;
            alpha_q8_r <= 9'd0;
            chunk_index <= '0;
            spike_mask_r <= 32'd0;
            zero_mask_r <= 32'd0;
            spike_count_r <= 9'd0;
            zero_count_r <= 9'd0;
            spike_chunk_count_r <= 6'd0;
            zero_chunk_count_r <= 6'd0;
            for (int nibble = 0; nibble < 8; nibble++) begin
                spike_nibble_count_r[nibble] <= 3'd0;
                zero_nibble_count_r[nibble] <= 3'd0;
            end
            for (int pair = 0; pair < 4; pair++) begin
                spike_pair_count_r[pair] <= 4'd0;
                zero_pair_count_r[pair] <= 4'd0;
            end
            for (int half = 0; half < 2; half++) begin
                spike_half_count_r[half] <= 5'd0;
                zero_half_count_r[half] <= 5'd0;
            end
            final_spike_count_r <= 9'd0;
            final_zero_count_r <= 9'd0;
            score_spike_operand_r <= 9'd0;
            score_zero_operand_r <= 9'd0;
            alpha_product_r <= 18'd0;
            score_q8 <= 18'd0;
            spike_matches <= 9'd0;
            zero_matches <= 9'd0;
            config_error <= 1'b0;
        end else begin
            unique case (state)
                ST_IDLE: begin
                    if (input_valid) begin
                        query_r <= query_spikes;
                        key_r <= key_spikes;
                        active_bits_r <= active_bits;
                        alpha_q8_r <= alpha_q8;
                        state <= ST_VALIDATE;
                    end
                end

                ST_VALIDATE: begin
                        if (active_bits_r == 0
```

```
                    || active_bits_r > 9'(VECTOR_BITS)
                    || alpha_q8_r > 256) begin
                config_error <= 1'b1;
                score_q8 <= 18'd0;
                spike_matches <= 9'd0;
                zero_matches <= 9'd0;
                state <= ST_OUTPUT;
            end else begin
                config_error <= 1'b0;
                chunk_index <= '0;
                spike_count_r <= 9'd0;
                zero_count_r <= 9'd0;
                state <= ST_MASK_CHUNK;
            end
        end

        ST_MASK_CHUNK: begin
            for (int bit_index = 0; bit_index < CHUNK_BITS; bit_index++) begin
                logic [8:0] global_bit;
                logic query_bit;
                logic key_bit;
                global_bit = {chunk_index, 5'd0} + 9'(bit_index);
                query_bit = query_r[chunk_index*CHUNK_BITS + bit_index];
                key_bit = key_r[chunk_index*CHUNK_BITS + bit_index];
                spike_mask_r[bit_index] <= global_bit < active_bits_r
                    && query_bit && key_bit;
                zero_mask_r[bit_index] <= global_bit < active_bits_r
                    && !query_bit && !key_bit;
            end
            state <= ST_POPCOUNT_PARTIAL;
        end

        ST_POPCOUNT_PARTIAL: begin
            for (int nibble = 0; nibble < 8; nibble++) begin
                spike_nibble_count_r[nibble] <=
                    popcount4(spike_mask_r[nibble*4 +: 4]);
                zero_nibble_count_r[nibble] <=
                    popcount4(zero_mask_r[nibble*4 +: 4]);
            end
            state <= ST_POPCOUNT;
        end

        ST_POPCOUNT: begin
            for (int pair = 0; pair < 4; pair++) begin
                spike_pair_count_r[pair] <= spike_nibble_count_r[2*pair]
                    + spike_nibble_count_r[2*pair+1];
                zero_pair_count_r[pair] <= zero_nibble_count_r[2*pair]
                    + zero_nibble_count_r[2*pair+1];
            end
            state <= ST_POPCOUNT_HALF;
        end

        ST_POPCOUNT_HALF: begin
            spike_half_count_r[0] <= spike_pair_count_r[0]
                + spike_pair_count_r[1];
            spike_half_count_r[1] <= spike_pair_count_r[2]
                + spike_pair_count_r[3];
            zero_half_count_r[0] <= zero_pair_count_r[0]
                + zero_pair_count_r[1];
            zero_half_count_r[1] <= zero_pair_count_r[2]
                + zero_pair_count_r[3];
            state <= ST_POPCOUNT_FINAL;
        end

        ST_POPCOUNT_FINAL: begin
            spike_chunk_count_r <= spike_half_count_r[0]
                + spike_half_count_r[1];
            zero_chunk_count_r <= zero_half_count_r[0]
                + zero_half_count_r[1];
            state <= ST_ACCUMULATE;
        end

        ST_ACCUMULATE: begin
            logic [8:0] spike_total;
            logic [8:0] zero_total;
            spike_total = spike_count_r + {3'd0, spike_chunk_count_r};
            zero_total = zero_count_r + {3'd0, zero_chunk_count_r};
            spike_count_r <= spike_total;
            zero_count_r <= zero_total;
            if (chunk_index == $clog2(CHUNKS)'(CHUNKS - 1)) begin
                final_spike_count_r <= spike_total;
                final_zero_count_r <= zero_total;
                spike_matches <= spike_total;
                zero_matches <= zero_total;
                state <= ST_SCORE_LATCH;
            end else begin
                chunk_index <= chunk_index + 1'b1;
```

```
                        state <= ST_MASK_CHUNK;
                    end
                end

                ST_SCORE_LATCH: begin
                    score_spike_operand_r <= final_spike_count_r;
                    score_zero_operand_r <= final_zero_count_r;
                    state <= ST_SCORE_MULTIPLY;
                end

                ST_SCORE_MULTIPLY: begin
                    alpha_product_r <= alpha_product;
                    state <= ST_SCORE_ADD;
                end

                ST_SCORE_ADD: begin
                    score_q8 <= {score_spike_operand_r, 8'd0} + alpha_product_r;
                    state <= ST_OUTPUT;
                end

                ST_OUTPUT: if (output_ready) state <= ST_IDLE;
                default: state <= ST_IDLE;
            endcase
        end
    end

    initial begin
        if (VECTOR_BITS != 256) $fatal(1, "neuraedge_alpha_xnor currently requires VECTOR_BITS=256");
        if (VECTOR_BITS % CHUNK_BITS != 0) $fatal(1, "VECTOR_BITS must be divisible by 32");
    end
endmodule

`default_nettype wire
```

5.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_alpha_xnor #(` This opens neuraedge_alpha_xnor, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores.

Line 5: `parameter int VECTOR_BITS = 256` This statement continues through source line 21; the explanation uses the complete expression rather than only the first visual fragment. This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 6: `) (` This is continuation line 2 of the 17-line logical statement that begins at source line 5. Its exact fragment is ‘) (‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 7: `input wire clk,` This is continuation line 3 of the 17-line logical statement that begins at source line 5. Its exact fragment is ‘input wire clk,‘. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes VECTOR_BITS to

256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 8: `input wire rst_n,` This is continuation line 4 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'input wire rst_n,'. The important operands are input: the value named 'input' in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 9: `input wire input_valid,` This is continuation line 5 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'input wire input_valid,'. The important operands are input: the value named 'input' in this block; input_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 10: `output logic input_ready,` This is continuation line 6 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'output logic input_ready,'. The important operands are output: the value named 'output' in this block; input_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 11: `input wire [VECTOR_BITS-1:0] query_spikes,` This is continuation line 7 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'input wire [VECTOR_BITS-1:0] query_spikes,'. The important operands are input: the value named 'input' in this block; VECTOR_BITS: the value named 'vector bits' in this block; query_spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 12: `input wire [VECTOR_BITS-1:0] key_spikes,` This is continuation line 8 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'input wire [VECTOR_BITS-1:0] key_spikes,'. The important operands are input: the value named 'input' in this block; VECTOR_BITS: the value named 'vector bits' in this block; key_spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 13: `input wire [8:0] active_bits,` This is continuation line 9 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'input wire [8:0] active_bits,'. The important operands are input: the value named 'input' in this block; active_bits: the value named 'active bits' in this block. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 14: `input wire [8:0] alpha_q8,` This is continuation line 10 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'input wire [8:0] alpha_q8,'. The important operands are input: the value named 'input' in this block; alpha_q8: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 15: `output logic output_valid,` This is continuation line 11 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'output logic output_valid,'. The important operands are output: the value named 'output' in this block; output_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 16: `input wire output_ready,` This is continuation line 12 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'input wire output_ready,'. The important operands are input: the value named 'input' in this block; output_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 17: `output logic [17:0] score_q8,` This is continuation line 13 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'output logic [17:0] score_q8,'. The important operands are output: the value named 'output' in this block; score_q8: the value named 'score q8' in this block. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 18: `output logic [8:0] spike_matches,` This is continuation line 14 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'output logic [8:0] spike_matches,'. The important operands are output: the value named 'output' in this block; spike_matches: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error

) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 19: `output logic [8:0] zero_matches,` This is continuation line 15 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'output logic [8:0] zero_matches,'. The important operands are output: the value named 'output' in this block; zero_matches: the value named 'zero matches' in this block. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 20: `output logic config_error` This is continuation line 16 of the 17-line logical statement that begins at source line 5. Its exact fragment is 'output logic config_error'. The important operands are output: the value named 'output' in this block; config_error: the value named 'config error' in this block. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 21: `);` This is continuation line 17 of the 17-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes VECTOR_BITS to 256) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [VECTOR_BITS-1:0] query_spikes, input wire [VECTOR_BITS-1:0] key_spikes, input wire [8:0] active_bits, input wire [8:0] alpha_q8, output logic output_valid, input wire output_ready, output logic [17:0] score_q8, output logic [8:0] spike_matches, output logic [8:0] zero_matches, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; VECTOR_BITS denotes the value named 'vector bits' in this block.

Line 22: `localparam int CHUNK_BITS = 32;` This fixes CHUNK_BITS to 32 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CHUNK_BITS denotes the value named 'chunk bits' in this block.

Line 23: `localparam int CHUNKS = VECTOR_BITS / CHUNK_BITS;` This fixes CHUNKS to VECTOR_BITS / CHUNK_BITS during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CHUNKS denotes the value named 'chunks' in this block.

Line 25: `typedef enum logic [3:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 26: `ST_IDLE = 4'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 27: `ST_MASK_CHUNK = 4'd1,` This assigns a fixed binary encoding to state ST_MASK_CHUNK, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 28: `ST_ACCUMULATE = 4'd2,` This assigns a fixed binary encoding to state ST_ACCUMULATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 29: `ST_SCORE_MULTIPLY = 4'd3,` This assigns a fixed binary encoding to state ST_SCORE_MULTIPLY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 30: `ST_SCORE_ADD = 4'd4,` This assigns a fixed binary encoding to state ST_SCORE_ADD, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 31: `ST_OUTPUT = 4'd5,` This assigns a fixed binary encoding to state ST_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 32: `ST_SCORE_LATCH = 4'd6,` This assigns a fixed binary encoding to state ST_SCORE_LATCH, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 33: `ST_POPCOUNT = 4'd7,` This assigns a fixed binary encoding to state ST_POPCOUNT, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 34: `ST_POPCOUNT_PARTIAL = 4'd8,` This assigns a fixed binary encoding to state ST_POPCOUNT_PARTIAL, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 35: `ST_VALIDATE = 4'd9,` This assigns a fixed binary encoding to state ST_VALIDATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 36: `ST_POPCOUNT_HALF = 4'd10,` This assigns a fixed binary encoding to state ST_POPCOUNT_HALF, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 37: `ST_POPCOUNT_FINAL = 4'd11` This statement continues through source line 38; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_POPCOUNT_FINAL, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 38: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 37. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_POPCOUNT_FINAL, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 40: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 41: `logic [VECTOR_BITS-1:0] query_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, query_r is the value named 'query r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 42: `logic [VECTOR_BITS-1:0] key_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, key_r is the value named 'key r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 43: `logic [8:0] active_bits_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_bits_r is the value named 'active bits r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 44: `logic [8:0] alpha_q8_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_q8_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 45: `logic [$clog2(CHUNKS)-1:0] chunk_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, chunk_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 46: `logic [31:0] spike_mask_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, spike_mask_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 47: `logic [31:0] zero_mask_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_mask_r is the value named 'zero mask r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 48: `logic [8:0] spike_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, spike_count_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 49: `logic [8:0] zero_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 50: `logic [5:0] spike_chunk_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, spike_chunk_count_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 51: `logic [5:0] zero_chunk_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_chunk_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 52: `logic [2:0] spike_nibble_count_r [0:7];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, spike_nibble_count_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 53: `logic [2:0] zero_nibble_count_r [0:7];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_nibble_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 54: `logic [3:0] spike_pair_count_r [0:3];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, spike_pair_count_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 55: `logic [3:0] zero_pair_count_r [0:3];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_pair_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 56: `logic [4:0] spike_half_count_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, spike_half_count_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 57: `logic [4:0] zero_half_count_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_half_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 58: `logic [8:0] final_spike_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, final_spike_count_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 59: `logic [8:0] final_zero_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, final_zero_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 60: `(* dont_touch = "yes" *) logic [8:0] score_spike_operand_r;` Within the module's structural declarations, this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; score_spike_operand_r: a binary event produced when a neuron crosses its threshold.

Line 61: `(* dont_touch = "yes" *) logic [8:0] score_zero_operand_r;` Within the module's structural declarations, this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; score_zero_operand_r: the value named 'score zero operand r' in this block.

Line 62: `logic [17:0] alpha_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_product_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 64: `(* use_dsp = "yes" *) wire [17:0] alpha_product =` Within the module's structural declarations, this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; alpha_product: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 65: `score_zero_operand_r * alpha_q8_r;` Within the module's structural declarations, this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are score_zero_operand_r: the value named 'score zero operand r' in this block; alpha_q8_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 67: `function automatic logic [2:0] popcount4(input logic [3:0] value);` Within the combinational helper popcount4, this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; popcount4: the value named 'popcount4' in this block.

Line 68: `logic [1:0] low_pair;` This declares hardware signal storage or wiring in the combinational helper popcount4. Specifically, low_pair is the value named 'low pair' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 69: `logic [1:0] high_pair;` This declares hardware signal storage or wiring in the combinational helper popcount4. Specifically, high_pair is the value named 'high pair' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 70: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper popcount4; it has no arithmetic value of its own.

Line 71: `low_pair = value[0] + value[1];` This makes low_pair, the value named 'low pair' in this block, equal to 'value[0] + value[1]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount4.

Line 72: `high_pair = value[2] + value[3];` This makes high_pair, the value named 'high pair' in this block, equal to 'value[2] + value[3]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount4.

Line 73: `popcount4 = low_pair + high_pair;` This makes popcount4, the value named 'popcount4' in this block, equal to 'low_pair + high_pair'. It is driven by combinational or procedural logic in the current evaluation. The important operands are low_pair: the value named 'low pair' in this block; high_pair: the value named 'high pair' in this block. Context: the combinational helper popcount4.

Line 74: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper popcount4; it has no arithmetic value of its own.

Line 75: `endfunction` Within the combinational helper popcount4, this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are endfunction: the value named 'endfunction' in this block.

Line 77: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 78: `input_ready = state == ST_IDLE;` This makes input_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_IDLE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 79: `output_valid = state == ST_OUTPUT;` This makes output_valid, the producer's statement that its current payload is meaningful, equal to 'state == ST_OUTPUT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT: the value named 'st output' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 80: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 82: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 83: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 84: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 85: `query_r <= '0;` This makes query_r, the value named 'query r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 86: `key_r <= '0;` This makes key_r, the value named 'key r' in this block, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 87: `active_bits_r <= 9'd0;` This makes active_bits_r, the value named 'active bits r' in this block, equal to ‘9'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 88: `alpha_q8_r <= 9'd0;` This makes alpha_q8_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to ‘9'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 89: `chunk_index <= '0;` This makes chunk_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 90: `spike_mask_r <= 32'd0;` This makes spike_mask_r, a binary event produced when a neuron crosses its threshold, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 91: `zero_mask_r <= 32'd0;` This makes zero_mask_r, the value named 'zero mask r' in this block, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 92: `spike_count_r <= 9'd0;` This makes spike_count_r, a binary event produced when a neuron crosses its threshold, equal to ‘9'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 93: `zero_count_r <= 9'd0;` This makes zero_count_r, an integer count retained for control or measurable telemetry, equal to ‘9'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 94: `spike_chunk_count_r <= 6'd0;` This makes spike_chunk_count_r, a binary event produced when a neuron crosses its threshold, equal to ‘6'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 95: `zero_chunk_count_r <= 6'd0;` This makes zero_chunk_count_r, an integer count retained for control or measurable telemetry, equal to ‘6'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 96: `for (int nibble = 0; nibble < 8; nibble++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 97: `spike_nibble_count_r[nibble] <= 3'd0;` This makes spike_nibble_count_r[nibble], a binary event produced when a neuron crosses its threshold, equal to ‘3'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 98: `zero_nibble_count_r[nibble] <= 3'd0;` This makes zero_nibble_count_r[nibble], an integer count retained for control or measurable telemetry, equal to '3'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 99: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 100: `for (int pair = 0; pair < 4; pair++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 101: `spike_pair_count_r[pair] <= 4'd0;` This makes spike_pair_count_r[pair], a binary event produced when a neuron crosses its threshold, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 102: `zero_pair_count_r[pair] <= 4'd0;` This makes zero_pair_count_r[pair], an integer count retained for control or measurable telemetry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 103: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 104: `for (int half = 0; half < 2; half++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 105: `spike_half_count_r[half] <= 5'd0;` This makes spike_half_count_r[half], a binary event produced when a neuron crosses its threshold, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 106: `zero_half_count_r[half] <= 5'd0;` This makes zero_half_count_r[half], an integer count retained for control or measurable telemetry, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 107: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 108: `final_spike_count_r <= 9'd0;` This makes final_spike_count_r, a binary event produced when a neuron crosses its threshold, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 109: `final_zero_count_r <= 9'd0;` This makes final_zero_count_r, an integer count retained for control or measurable telemetry, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 110: `score_spike_operand_r <= 9'd0;` This makes score_spike_operand_r, a binary event produced when a neuron crosses its threshold, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 111: `score_zero_operand_r <= 9'd0;` This makes score_zero_operand_r, the value named 'score zero operand r' in this block, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 112: `alpha_product_r <= 18'd0;` This makes alpha_product_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to '18'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 113: `score_q8 <= 18'd0;` This makes score_q8, the value named 'score q8' in this block, equal to '18'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 114: `spike_matches <= 9'd0;` This makes spike_matches, a binary event produced when a neuron crosses its threshold, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 115: `zero_matches <= 9'd0;` This makes zero_matches, the value named 'zero matches' in this block, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 116: `config_error <= 1'b0;` This makes config_error, the value named 'config error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 117: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 118: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 119: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 120: `if (input_valid) begin` This condition selects the following hardware branch only when 'input_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 121: `query_r <= query_spikes;` This makes query_r, the value named 'query r' in this block, equal to 'query_spikes'. It updates after the active clock edge together with other nonblocking assignments. The important operands are query_spikes: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 122: `key_r <= key_spikes;` This makes key_r, the value named 'key r' in this block, equal to 'key_spikes'. It updates after the active clock edge together with other nonblocking assignments. The important operands are key_spikes: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 123: `active_bits_r <= active_bits;` This makes active_bits_r, the value named 'active bits r' in this block, equal to 'active_bits'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_bits: the value named 'active bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 124: `alpha_q8_r <= alpha_q8;` This makes alpha_q8_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to 'alpha_q8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are alpha_q8: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 125: `state <= ST_VALIDATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_VALIDATE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VALIDATE: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 126: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 127: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 129: `ST_VALIDATE: begin` This is the ST_VALIDATE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 130: `if (active_bits_r == 0` Within the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase., this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are active_bits_r: the value named 'active bits r' in this block.

Line 131: `|| active_bits_r > 9'(VECTOR_BITS)` Within the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase., this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are active_bits_r: the value named 'active bits r' in this block; VECTOR_BITS: the value named 'vector bits' in this block.

Line 132: `|| alpha_q8_r > 256) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase., this exact statement contributes to the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores. The important operands are alpha_q8_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 133: `config_error <= 1'b1;` This makes config_error, the value named 'config error' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 134: `score_q8 <= 18'd0;` This makes score_q8, the value named 'score q8' in this block, equal to '18'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 135: `spike_matches <= 9'd0;` This makes spike_matches, a binary event produced when a neuron crosses its threshold, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 136: `zero_matches <= 9'd0;` This makes zero_matches, the value named 'zero matches' in this block, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 137: `state <= ST_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT: the value named 'st output' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 138: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 139: `config_error <= 1'b0;` This makes config_error, the value named 'config error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 140: `chunk_index <= '0;` This makes chunk_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 141: `spike_count_r <= 9'd0;` This makes spike_count_r, a binary event produced when a neuron crosses its threshold, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 142: `zero_count_r <= 9'd0;` This makes zero_count_r, an integer count retained for control or measurable telemetry, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 143: `state <= ST_MASK_CHUNK;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MASK_CHUNK'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MASK_CHUNK: the value named 'st mask chunk' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 144: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 145: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 147: `ST_MASK_CHUNK: begin` This is the ST_MASK_CHUNK case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 148: `for (int bit_index = 0; bit_index < CHUNK_BITS; bit_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 149: `logic [8:0] global_bit;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase.. Specifically, global_bit is the value named 'global bit' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 150: `logic query_bit;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase.. Specifically, query_bit is the value named 'query bit' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 151: `logic key_bit;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase.. Specifically, key_bit is the value named 'key bit' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 152: `global_bit = {chunk_index, 5'd0} + 9'(bit_index);` This makes global_bit, the value named 'global bit' in this block, equal to '{chunk_index, 5'd0} + 9'(bit_index)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are chunk_index: an integer selecting one tensor element, lane, bank, or state entry; d0: the value named 'd0' in this block; bit_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 153: `query_bit = query_r[chunk_index*CHUNK_BITS + bit_index];` This makes query_bit, the value named 'query bit' in this block, equal to 'query_r[chunk_index*CHUNK_BITS + bit_index]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are query_r: the value named 'query r' in this block; chunk_index: an integer selecting one tensor element, lane, bank, or state entry; CHUNK_BITS: the value named 'chunk bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 154: `key_bit = key_r[chunk_index*CHUNK_BITS + bit_index];` This makes key_bit, the value named 'key bit' in this block, equal to 'key_r[chunk_index*CHUNK_BITS + bit_index]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are key_r: the value named 'key r' in this block; chunk_index: an integer selecting one tensor element, lane, bank, or state entry; CHUNK_BITS: the value named 'chunk bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 155: `spike_mask_r[bit_index] <= global_bit < active_bits_r` This statement continues through source line 156; the explanation uses the complete expression rather than only the first visual fragment. This makes spike_mask_r[bit_index], a binary event produced when a neuron crosses its threshold, equal to ‘global_bit < active_bits_r && query_bit && key_bit‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are global_bit: the value named ’global bit’ in this block; active_bits_r: the value named ’active bits r’ in this block; query_bit: the value named ’query bit’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 156: `&& query_bit && key_bit;` This is continuation line 2 of the 2-line logical statement that begins at source line 155. Its exact fragment is ‘&& query_bit && key_bit;‘. The important operands are query_bit: the value named ’query bit’ in this block; key_bit: the value named ’key bit’ in this block. Read together, the complete operation is: This makes spike_mask_r[bit_index], a binary event produced when a neuron crosses its threshold, equal to ‘global_bit < active_bits_r && query_bit && key_bit‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are global_bit: the value named ’global bit’ in this block; active_bits_r: the value named ’active bits r’ in this block; query_bit: the value named ’query bit’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 157: `zero_mask_r[bit_index] <= global_bit < active_bits_r` This statement continues through source line 158; the explanation uses the complete expression rather than only the first visual fragment. This makes zero_mask_r[bit_index], the value named ’zero mask r’ in this block, equal to ‘global_bit < active_bits_r && !query_bit && !key_bit‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are global_bit: the value named ’global bit’ in this block; active_bits_r: the value named ’active bits r’ in this block; query_bit: the value named ’query bit’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 158: `&& !query_bit && !key_bit;` This is continuation line 2 of the 2-line logical statement that begins at source line 157. Its exact fragment is ‘&& !query_bit && !key_bit;‘. The important operands are query_bit: the value named ’query bit’ in this block; key_bit: the value named ’key bit’ in this block. Read together, the complete operation is: This makes zero_mask_r[bit_index], the value named ’zero mask r’ in this block, equal to ‘global_bit < active_bits_r && !query_bit && !key_bit‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are global_bit: the value named ’global bit’ in this block; active_bits_r: the value named ’active bits r’ in this block; query_bit: the value named ’query bit’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 159: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 160: `state <= ST_POPCOUNT_PARTIAL;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_POPCOUNT_PARTIAL‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_POPCOUNT_PARTIAL: the value named ’st popcount partial’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase..

Line 161: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MASK_CHUNK, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 163: `ST_POPCOUNT_PARTIAL: begin` This is the ST_POPCOUNT_PARTIAL case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_PARTIAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 164: `for (int nibble = 0; nibble < 8; nibble++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_PARTIAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 165: `spike_nibble_count_r[nibble] <=` This statement continues through source line 166; the explanation uses the complete expression rather than only the first visual fragment. This makes spike_nibble_count_r[nibble], a binary event produced when a neuron crosses its threshold, equal to ‘popcount4(spike_mask_r[nibble*4 +: 4])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are popcount4: the value named ’popcount4’ in this block; spike_mask_r: a binary event produced when a neuron crosses its threshold; nibble: the value named ’nibble’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_PARTIAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 166: `popcount4(spike_mask_r[nibble*4 +: 4]);` This is continuation line 2 of the 2-line logical statement that begins at source line 165. Its exact fragment is ‘popcount4(spike_mask_r[nibble*4 +: 4]);‘. The important operands are popcount4: the value named ’popcount4’ in this block; spike_mask_r: a binary event produced when a neuron crosses its threshold; nibble: the value named ’nibble’ in this block. Read together, the complete operation is: This makes spike_nibble_count_r[nibble], a binary event produced when a neuron crosses its threshold, equal to ‘popcount4(spike_mask_r[nibble*4 +: 4])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are popcount4: the value named ’popcount4’ in this block; spike_mask_r: a binary event produced when a neuron crosses its threshold; nibble: the value named ’nibble’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_PARTIAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 167: `zero_nibble_count_r[nibble] <=` This statement continues through source line 168; the explanation uses the complete expression rather than only the first visual fragment. This makes zero_nibble_count_r[nibble], an integer count retained for control or measurable telemetry, equal to ‘popcount4(zero_mask_r[nibble*4 +: 4])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are popcount4: the value named ’popcount4’ in this block; zero_mask_r: the value named ’zero mask r’ in this block; nibble: the value named ’nibble’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_PARTIAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 168: `popcount4(zero_mask_r[nibble*4 +: 4]);` This is continuation line 2 of the 2-line logical statement that begins at source line 167. Its exact fragment is ‘popcount4(zero_mask_r[nibble*4 +: 4]);‘. The important operands are popcount4: the value named ’popcount4’ in this block; zero_mask_r: the value named ’zero mask r’ in this block; nibble: the value named ’nibble’ in this block. Read together, the complete operation is: This makes zero_nibble_count_r[nibble], an integer count retained for control or measurable telemetry, equal to ‘popcount4(zero_mask_r[nibble*4 +: 4])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are popcount4: the value named ’popcount4’ in this block; zero_mask_r: the value named ’zero mask r’ in this block; nibble: the value named ’nibble’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_PARTIAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 169: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_PARTIAL, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 170: `state <= ST_POPCOUNT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_POPCOUNT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_POPCOUNT: the value named ’st popcount’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_PARTIAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 171: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_PARTIAL, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 173: `ST_POPCOUNT: begin` This is the ST_POPCOUNT case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT, the machine updates a segmented telemetry counter without a long carry chain..

Line 174: `for (int pair = 0; pair < 4; pair++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT, the machine updates a segmented telemetry counter without a long carry chain..

Line 175: `spike_pair_count_r[pair] <= spike_nibble_count_r[2*pair]` This statement continues through source line 176; the explanation uses the complete expression rather than only the first visual fragment. This makes spike_pair_count_r[pai a binary event produced when a neuron crosses its threshold, equal to 'spike_nibble_count_r[2*pair] + spike_nibble_count_r[2*pai It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_nibble_count_r: a binary event produced when a neuron crosses its threshold; pair: the value named 'pair' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT, the machine updates a segmented telemetry counter without a long carry chain..

Line 176: `+ spike_nibble_count_r[2*pair+1];` This is continuation line 2 of the 2-line logical statement that begins at source line 175. Its exact fragment is '+ spike_nibble_count_r[2*pair+1];'. The important operands are spike_nibble_count_r: a binary event produced when a neuron crosses its threshold; pair: the value named 'pair' in this block. Read together, the complete operation is: This makes spike_pair_count_r[pair], a binary event produced when a neuron crosses its threshold, equal to 'spike_nibble_count_r[2*pair] + spike_nibble_count_r[2*pair+1]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_nibble_count_r: a binary event produced when a neuron crosses its threshold; pair: the value named 'pair' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT, the machine updates a segmented telemetry counter without a long carry chain..

Line 177: `zero_pair_count_r[pair] <= zero_nibble_count_r[2*pair]` This statement continues through source line 178; the explanation uses the complete expression rather than only the first visual fragment. This makes zero_pair_count_r[pair an integer count retained for control or measurable telemetry, equal to 'zero_nibble_count_r[2*pair] + zero_nibble_count_r[2*pair- It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_nibble_count_r: an integer count retained for control or measurable telemetry; pair: the value named 'pair' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT, the machine updates a segmented telemetry counter without a long carry chain..

Line 178: `+ zero_nibble_count_r[2*pair+1];` This is continuation line 2 of the 2-line logical statement that begins at source line 177. Its exact fragment is '+ zero_nibble_count_r[2*pair+1];'. The important operands are zero_nibble_count_r: an integer count retained for control or measurable telemetry; pair: the value named 'pair' in this block. Read together, the complete operation is: This makes zero_pair_count_r[pair], an integer count retained for control or measurable telemetry, equal to 'zero_nibble_count_r[2*pair] + zero_nibble_count_r[2*pair+1]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_nibble_count_r: an integer count retained for control or measurable telemetry; pair: the value named 'pair' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT, the machine updates a segmented telemetry counter without a long carry chain..

Line 179: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 180: `state <= ST_POPCOUNT_HALF;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_POPCOUNT_HALF‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_POPCOUNT_HALF: the value named ’st popcount half’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT, the machine updates a segmented telemetry counter without a long carry chain..

Line 181: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 183: `ST_POPCOUNT_HALF: begin` This is the ST_POPCOUNT_HALF case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 184: `spike_half_count_r[0] <= spike_pair_count_r[0]` This statement continues through source line 185; the explanation uses the complete expression rather than only the first visual fragment. This makes spike_half_count_r[0], a binary event produced when a neuron crosses its threshold, equal to ‘spike_pair_count_r[0] + spike_pair_count_r[1]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_pair_count_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 185: `+ spike_pair_count_r[1];` This is continuation line 2 of the 2-line logical statement that begins at source line 184. Its exact fragment is ‘+ spike_pair_count_r[1];‘. The important operands are spike_pair_count_r: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This makes spike_half_count_r[0], a binary event produced when a neuron crosses its threshold, equal to ‘spike_pair_count_r[0] + spike_pair_count_r[1]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_pair_count_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 186: `spike_half_count_r[1] <= spike_pair_count_r[2]` This statement continues through source line 187; the explanation uses the complete expression rather than only the first visual fragment. This makes spike_half_count_r[1], a binary event produced when a neuron crosses its threshold, equal to ‘spike_pair_count_r[2] + spike_pair_count_r[3]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_pair_count_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 187: `+ spike_pair_count_r[3];` This is continuation line 2 of the 2-line logical statement that begins at source line 186. Its exact fragment is ‘+ spike_pair_count_r[3];‘. The important operands are spike_pair_count_r: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This makes spike_half_count_r[1], a binary event produced when a neuron crosses its threshold, equal to ‘spike_pair_count_r[2] + spike_pair_count_r[3]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_pair_count_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 188: `zero_half_count_r[0] <= zero_pair_count_r[0]` This statement continues through source line 189; the explanation uses the complete expression rather than only the first visual fragment. This makes zero_half_count_r[0], an integer count retained for control or measurable telemetry, equal to ‘zero_pair_count_r[0] + zero_pair_count_r[1]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_pair_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 189: `+ zero_pair_count_r[1];` This is continuation line 2 of the 2-line logical statement that begins at source line 188. Its exact fragment is '+ zero_pair_count_r[1];'. The important operands are zero_pair_count_r: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This makes zero_half_count_r[0], an integer count retained for control or measurable telemetry, equal to 'zero_pair_count_r[0] + zero_pair_count_r[1]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_pair_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 190: `zero_half_count_r[1] <= zero_pair_count_r[2]` This statement continues through source line 191; the explanation uses the complete expression rather than only the first visual fragment. This makes zero_half_count_r[1], an integer count retained for control or measurable telemetry, equal to 'zero_pair_count_r[2] + zero_pair_count_r[3]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_pair_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 191: `+ zero_pair_count_r[3];` This is continuation line 2 of the 2-line logical statement that begins at source line 190. Its exact fragment is '+ zero_pair_count_r[3];'. The important operands are zero_pair_count_r: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This makes zero_half_count_r[1], an integer count retained for control or measurable telemetry, equal to 'zero_pair_count_r[2] + zero_pair_count_r[3]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_pair_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 192: `state <= ST_POPCOUNT_FINAL;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_POPCOUNT_FINAL'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_POPCOUNT_FINAL: the value named 'st popcount final' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain..

Line 193: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_HALF, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 195: `ST_POPCOUNT_FINAL: begin` This is the ST_POPCOUNT_FINAL case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_FINAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 196: `spike_chunk_count_r <= spike_half_count_r[0]` This statement continues through source line 197; the explanation uses the complete expression rather than only the first visual fragment. This makes spike_chunk_count_r, a binary event produced when a neuron crosses its threshold, equal to 'spike_half_count_r[0] + spike_half_count_r[1]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_half_count_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_FINAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 197: `+ spike_half_count_r[1];` This is continuation line 2 of the 2-line logical statement that begins at source line 196. Its exact fragment is '+ spike_half_count_r[1];'. The important operands are spike_half_count_r: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This makes spike_chunk_count_r, a binary event produced when a neuron crosses its threshold, equal to 'spike_half_count_r[0] + spike_half_count_r[1]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_half_count_r: a binary event produced when a neuron crosses its threshold. Context: the

clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_FINAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 198: `zero_chunk_count_r <= zero_half_count_r[0]` This statement continues through source line 199; the explanation uses the complete expression rather than only the first visual fragment. This makes zero_chunk_count_r, an integer count retained for control or measurable telemetry, equal to ‘zero_half_count_r[0] + zero_half_count_r[1]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_half_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_FINAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 199: `+ zero_half_count_r[1];` This is continuation line 2 of the 2-line logical statement that begins at source line 198. Its exact fragment is ‘+ zero_half_count_r[1];‘. The important operands are zero_half_count_r: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This makes zero_chunk_count_r, an integer count retained for control or measurable telemetry, equal to ‘zero_half_count_r[0] + zero_half_count_r[1]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_half_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_FINAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 200: `state <= ST_ACCUMULATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_ACCUMULATE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ACCUMULATE: the value named ’st accumulate’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_FINAL, the machine updates a segmented telemetry counter without a long carry chain..

Line 201: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_POPCOUNT_FINAL, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 203: `ST_ACCUMULATE: begin` This is the ST_ACCUMULATE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 204: `logic [8:0] spike_total;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase.. Specifically, spike_total is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 205: `logic [8:0] zero_total;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase.. Specifically, zero_total is the value named ’zero total’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 206: `spike_total = spike_count_r + {3'd0, spike_chunk_count_r};` This makes spike_total, a binary event produced when a neuron crosses its threshold, equal to ‘spike_count_r + {3’d0, spike_chunk_count_r}‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are spike_count_r: a binary event produced when a neuron crosses its threshold; d0: the value named ’d0’ in this block; spike_chunk_count_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 207: `zero_total = zero_count_r + {3'd0, zero_chunk_count_r};` This makes zero_total, the value named ’zero total’ in this block, equal to ‘zero_count_r + {3’d0, zero_chunk_count_r}‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are zero_count_r: an integer count retained for control or measurable telemetry; d0: the value named ’d0’ in this block; zero_chunk_count_r: an integer count retained

for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 208: `spike_count_r <= spike_total;` This makes spike_count_r, a binary event produced when a neuron crosses its threshold, equal to ‘spike_total‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_total: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 209: `zero_count_r <= zero_total;` This makes zero_count_r, an integer count retained for control or measurable telemetry, equal to ‘zero_total‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_total: the value named ’zero total’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 210: `if (chunk_index == $clog2(CHUNKS)'(CHUNKS - 1)) begin` This condition selects the following hardware branch only when ‘chunk_index == $clog2(CHUNKS)'(CHUNKS - 1)‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are chunk_index: an integer selecting one tensor element, lane, bank, or state entry; clog2: the value named ’clog2’ in this block; CHUNKS: the value named ’chunks’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 211: `final_spike_count_r <= spike_total;` This makes final_spike_count_r, a binary event produced when a neuron crosses its threshold, equal to ‘spike_total‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_total: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 212: `final_zero_count_r <= zero_total;` This makes final_zero_count_r, an integer count retained for control or measurable telemetry, equal to ‘zero_total‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_total: the value named ’zero total’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 213: `spike_matches <= spike_total;` This makes spike_matches, a binary event produced when a neuron crosses its threshold, equal to ‘spike_total‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_total: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 214: `zero_matches <= zero_total;` This makes zero_matches, the value named ’zero matches’ in this block, equal to ‘zero_total‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_total: the value named ’zero total’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 215: `state <= ST_SCORE_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_SCORE_LATCH‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SCORE_LATCH: the value named ’st score latch’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 216: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 217: `chunk_index <= chunk_index + 1'b1;` This makes chunk_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘chunk_index + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are chunk_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 218: `state <= ST_MASK_CHUNK;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_MASK_CHUNK‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MASK_CHUNK: the value named ’st mask chunk’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase..

Line 219: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 220: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 222: `ST_SCORE_LATCH: begin` This is the ST_SCORE_LATCH case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_LATCH, the machine performs the named control phase..

Line 223: `score_spike_operand_r <= final_spike_count_r;` This makes score_spike_operand_r, a binary event produced when a neuron crosses its threshold, equal to ‘final_spike_count_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are final_spike_count_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_LATCH, the machine performs the named control phase..

Line 224: `score_zero_operand_r <= final_zero_count_r;` This makes score_zero_operand_r, the value named ’score zero operand r’ in this block, equal to ‘final_zero_count_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are final_zero_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_LATCH, the machine performs the named control phase..

Line 225: `state <= ST_SCORE_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_SCORE_MULTIPLY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SCORE_MULTIPLY: the value named ’st score multiply’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_LATCH, the machine performs the named control phase..

Line 226: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_LATCH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 228: `ST_SCORE_MULTIPLY: begin` This is the ST_SCORE_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_MULTIPLY, the machine performs the named control phase..

Line 229: `alpha_product_r <= alpha_product;` This makes alpha_product_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to ‘alpha_product‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are alpha_product: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_MULTIPLY, the machine performs the named control phase..

Line 230: `state <= ST_SCORE_ADD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_SCORE_ADD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SCORE_ADD: the value named ’st score add’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_MULTIPLY, the machine performs the named control phase..

Line 231: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 233: `ST_SCORE_ADD: begin` This is the ST_SCORE_ADD case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_ADD, the machine performs the named control phase..

Line 234: `score_q8 <= {score_spike_operand_r, 8'd0} + alpha_product_r;` This makes score_q8, the value named ’score q8’ in this block, equal to ‘{score_spike_operand_r, 8’d0} + alpha_product_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are score_spike_operand_r: a binary event produced when a neuron crosses its threshold; d0: the value named ’d0’ in this block; alpha_product_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_ADD, the machine performs the named control phase..

Line 235: `state <= ST_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT: the value named ’st output’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_ADD, the machine performs the named control phase..

Line 236: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SCORE_ADD, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 238: `ST_OUTPUT: if (output_ready) state <= ST_IDLE;` This is the ST_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘if (output_ready) state <= ST_IDLE;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 239: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘state <= ST_IDLE;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 240: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 241: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 242: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 244: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 245: `if (VECTOR_BITS != 256) $fatal(1, "neuraedge_alpha_xnor currently requires VECTOR_BITS=256");` This condition selects the following hardware branch only when ‘VECTOR_BITS != 256) $fatal(1, "neuraedge_alpha_xnor currently requires VECTOR_BITS=256"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are VECTOR_BITS: the value named ’vector bits’ in this block; fatal: the value named ’fatal’ in this block; neuraedge_alpha_xnor: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the module’s structural declarations.

Line 246: `if (VECTOR_BITS % CHUNK_BITS != 0) $fatal(1, "VECTOR_BITS must be divisible by 32");` This condition selects the following hardware branch only when ‘VECTOR_BITS % CHUNK_BITS != 0) $fatal(1, "VECTOR_BITS must be divisible by 32"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are VECTOR_BITS: the value named ’vector bits’ in this block; CHUNK_BITS: the value named ’chunk bits’ in this block; fatal: the value named ’fatal’ in this block. Context: the module’s structural declarations.

Line 247: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 248: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 250: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that computes alpha-weighted XNOR spiking similarity and streams its accumulated scores.

6 rtl/neuraedge_buffered_tensor_core.sv

Role in the complete system: loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

6.1 Beginner architecture map

Dataflow responsibility This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_buffered_tensor_core into physical nets and primitives. The source declares 12 input-port lines, 17 output-port lines, 3 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_COMPUTE_ACCOUNT, ST_COMPUTE_CONFIG, ST_COMPUTE_DRAIN, ST_COMPUTE_OUTPUT, ST_COMPUTE_RUN, ST_COMPUTE_START, ST_COMPUTE ST_COMPUTE_VALIDATED, ST_DECODE_COMMAND, ST_IDLE, ST_LOAD_ACTIVATIONS, ST_LOAD_ACTIV ST_LOAD_CONFIG, ST_LOAD_WEIGHTS_DOT8, ST_LOAD_WEIGHTS_OUTER, ST_PREDECODE_COMMAN A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates neuraedge_active_index_buffer as u_active_index_buffer, neuraedge_activation_buffer as u_activation_buffer, neuraedge_weight_bank as u_weight_bank, neuraedge_tensor_tile as u_tensor_tile, unique as case, end as if, else as if, endcase as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

6.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_buffered_tensor_core #(
5      parameter int MAX_OUTER_ISSUES = 2048,
6      parameter int MAX_DOT8_ISSUES = 256,
7      parameter int COLUMN_LANES = 16
8  ) (
9      input  wire clk,
10     input  wire rst_n,
11
12     input  wire command_valid,
13     output logic command_ready,
14     input  wire [41:0] command_data,
15     output logic command_done,
16     output logic status_error,
17     output logic status_busy,
18
19     input  wire s_axis_tvalid,
20     output logic s_axis_tready,
21     input  wire [63:0] s_axis_tdata,
22     input  wire s_axis_tlast,
23
24     output logic m_axis_tvalid,
25     input  wire m_axis_tready,
26     output logic [63:0] m_axis_tdata,
27     output logic m_axis_tlast,
28
29     output logic [11:0] status_loaded_activations,
30     output logic [11:0] status_loaded_weights,
31     output logic [11:0] status_active_issues,
32     output logic [31:0] status_input_beats,
33     output logic [31:0] status_output_beats,
34     output logic [31:0] status_compute_cycles,
```

```
35      output logic [31:0] status_skipped_issues,
36      output logic [63:0] status_dense_macs,
37      output logic [63:0] status_spike_adds
38  );
39      import neuraedge_command_pkg::*;
40
41      localparam int WEIGHT_BANKS = 8;
42
43      typedef enum logic [3:0] {
44          ST_IDLE = 4'd0,
45          ST_LOAD_ACTIVATIONS = 4'd1,
46          ST_LOAD_WEIGHTS_OUTER = 4'd2,
47          ST_LOAD_WEIGHTS_DOT8 = 4'd3,
48          ST_COMPUTE_START = 4'd4,
49          ST_COMPUTE_RUN = 4'd5,
50          ST_COMPUTE_OUTPUT = 4'd6,
51          ST_LOAD_ACTIVATION_DRAIN = 4'd7,
52          ST_DECODE_COMMAND = 4'd8,
53          ST_COMPUTE_CONFIG = 4'd9,
54          ST_LOAD_CONFIG = 4'd10,
55          ST_COMPUTE_VALIDATE = 4'd11,
56          ST_COMPUTE_VALIDATED = 4'd12,
57          ST_PREDECODE_COMMAND = 4'd13,
58          ST_COMPUTE_ACCOUNT = 4'd14,
59          ST_COMPUTE_DRAIN = 4'd15
60      } state_t;
61
62      state_t state;
63      logic [41:0] captured_command_r;
64      logic [3:0] decoded_opcode_r;
65      logic [11:0] decoded_issue_count_r;
66      logic decoded_outer_issue_error_r;
67      logic decoded_dot8_issue_error_r;
68      (* dont_touch = "yes" *) logic command_done_pending_r;
69      (* dont_touch = "yes" *) logic error_event_r;
70      (* dont_touch = "yes" *) logic clear_error_r;
71      logic [3:0] pending_load_opcode_r;
72      logic [11:0] pending_load_issue_count_r;
73      logic [7:0] pending_load_activation_zp_r;
74
75      // Eight 256-deep, 128-bit banks are the physical form of a 32 KiB weight
76      // tile. In
77      // outer-product mode one bank supplies each consecutive K position. In
78      // Dot8 mode reads all eight banks together, exposing 8x16 weight bytes.
79      logic activation_write_enable;
80      logic [10:0] activation_write_address;
81      logic [10:0] activation_read_address;
82      wire [63:0] activation_read_data;
83      logic [WEIGHT_BANKS-1:0] weight_write_enable;
84      logic [7:0] weight_write_address;
85      logic [1:0] weight_write_segment;
86      logic [WEIGHT_BANKS-1:0] weight_commit_enable_r;
87      logic [7:0] weight_commit_address_r;
88      logic [1:0] weight_commit_segment_r;
89      logic [63:0] weight_commit_data_r;
90      logic [7:0] weight_read_address;
91      wire [COLUMN_LANES*8-1:0] weight_read_data [0:WEIGHT_BANKS-1];
92
93      logic [11:0] load_requested_issues;
94      logic [14:0] load_beats_remaining_r;
95      logic load_finish_pending_r;
96      logic [3:0] load_segment_index_r;
97      logic [11:0] load_item_index;
98      logic [7:0] load_activation_zero_point;
99      logic weight_format_dot8;
100     logic activation_classify_valid_r;
101     logic activation_classify_nonzero_r;
102     logic [10:0] activation_classify_index_r;
103     logic activation_load_success_r;
104     wire active_index_write_enable;
105     wire [10:0] active_index_read_data;
106     wire active_index_read_enable;
107     wire [10:0] active_mapped_index;
108
109     (* dont_touch = "yes" *) logic [1:0] compute_mode_r;
110     (* dont_touch = "yes" *) logic [3:0] compute_rows_r;
111     (* dont_touch = "yes" *) logic [5:0] compute_columns_r;
112     logic [11:0] compute_requested_issues;
113     logic compute_validation_error_r;
114     (* dont_touch = "yes" *) logic [7:0] compute_activation_zp_r;
115     (* dont_touch = "yes" *) logic [7:0] compute_weight_zp_r;
116     logic [11:0] schedule_pointer;
117     logic [11:0] schedule_remaining_r;
118     logic schedule_active_r;
119     logic schedule_last_r;
120     logic compute_empty_r;
121     logic compute_read_request;
```

```
    logic index_request_valid_r;
    logic index_request_last_r;
    logic [10:0] index_request_address_r;
    logic index_valid_r;
    logic index_last_r;
    logic map_valid_r;
    logic map_last_r;
    logic [10:0] mapped_index_r;
    logic read_valid_r;
    logic [2:0] read_bank_select_r;
    logic read_last_r;
    logic memory_valid_r;
    logic [2:0] memory_bank_select_r;
    logic memory_last_r;
    logic capture_valid_r;
    logic capture_last_r;
    logic [63:0] capture_activation_r;
    logic [COLUMN_LANES*8-1:0] capture_outer_weights_r;
    logic [COLUMN_LANES*8*8-1:0] capture_dot_weights_r;
    logic [1:0] issue_fifo_count_r;
    logic issue_fifo_write_pointer_r;
    logic issue_fifo_read_pointer_r;
    logic issue_fifo_last_r [0:1];
    logic [63:0] issue_fifo_activation_r [0:1];
    logic [COLUMN_LANES*8-1:0] issue_fifo_outer_weights_r [0:1];
    logic [COLUMN_LANES*8*8-1:0] issue_fifo_dot_weights_r [0:1];
    (* dont_touch = "yes" *) logic tile_last_issue_accepted_r;
    logic issue_fifo_available;
    logic capture_stage_available;
    logic memory_stage_available;
    logic read_stage_available;
    logic map_stage_available;
    logic index_stage_available;
    logic index_request_stage_available;

    logic tile_start;
    logic tile_config_error;
    logic tile_issue_valid;
    wire tile_issue_ready;
    logic [63:0] tile_activation_vector;
    logic [COLUMN_LANES*8-1:0] tile_outer_weight_vector;
    logic [COLUMN_LANES*8*8-1:0] tile_dot_weight_vector;
    logic tile_issue_last;
    wire tile_result_valid;
    logic tile_result_ready;
    wire signed [47:0] tile_result_value;
    wire [3:0] tile_result_row;
    wire [5:0] tile_result_column;
    wire tile_result_last;
    // This two-entry skid FIFO breaks the external READY path before it reaches
    // the tile's 128-entry result store and replicated output-state controls.
    // A registered tile READY can remain asserted for one final transfer while
    // downstream backpressure arrives; the second entry is the space reserved
    // for that transfer, so no result is dropped and sustained throughput stays
    // at one result per clock whenever the parent accepts one per clock.
    logic tile_result_ready_r;
    logic [1:0] tensor_output_fifo_count_r;
    logic tensor_output_fifo_write_pointer_r;
    logic tensor_output_fifo_read_pointer_r;
    logic [63:0] tensor_output_fifo_data_r [0:1];
    logic tensor_output_fifo_last_r [0:1];
    wire [63:0] tile_debug_dense_macs;
    wire [63:0] tile_debug_spike_adds;
    logic [23:0] pending_dense_macs_low_r;
    logic pending_dense_macs_carry_r;
    logic [23:0] pending_spike_adds_low_r;
    logic pending_spike_adds_carry_r;
    wire issue_fifo_push = capture_valid_r && issue_fifo_available;
    wire issue_fifo_pop = issue_fifo_count_r != 0 && tile_issue_ready;
    wire tensor_output_fifo_push = state == ST_COMPUTE_OUTPUT
        && tile_result_valid && tile_result_ready_r;
    wire tensor_output_fifo_pop = tensor_output_fifo_count_r != 0
        && m_axis_tready;
    wire tensor_output_fifo_head_last = tensor_output_fifo_last_r[
        tensor_output_fifo_read_pointer_r];
    logic [1:0] tensor_output_fifo_count_next;

    function automatic logic all_bytes_equal64(
        input logic [63:0] values,
        input logic [7:0] reference
    );
        logic equal_all;
        begin
            equal_all = 1'b1;
            for (int lane = 0; lane < 8; lane++) begin
                equal_all = equal_all && (values[lane*8 +: 8] == reference);
            end
```

```
            all_bytes_equal64 = equal_all;
        end
    endfunction

    function automatic logic [63:0] repeat_byte(input logic [7:0] value);
        begin
            repeat_byte = {8{value}};
        end
    endfunction

    // Tile activity is at most 19 bits per command.  The low 24-bit addition
    // and its carry are registered on the final output beat; the following
    // accounting state updates the independent high 40 bits.
    function automatic logic [24:0] add_u19_to_u24(
        input logic [23:0] value,
        input logic [18:0] increment
    );
        begin
            add_u19_to_u24 = {1'b0, value} + {6'd0, increment};
        end
    endfunction

    assign tile_start = (state == ST_COMPUTE_START);
    assign active_index_write_enable = activation_classify_valid_r
        && activation_classify_nonzero_r;
    // Each accepted request generates exactly one synchronous BRAM response.
    // The following local register stage holds that response during downstream
    // stalls, so an unaccepted address can never overwrite accepted metadata.
    assign active_index_read_enable = index_request_valid_r && !compute_empty_r;
    assign active_mapped_index = compute_empty_r
        ? 11'd0 : active_index_read_data;

    neuraedge_active_index_buffer #(
        .DEPTH(MAX_OUTER_ISSUES),
        .ADDRESS_BITS(11)
    ) u_active_index_buffer (
        .clk(clk),
        .write_enable(active_index_write_enable),
        .write_address(status_active_issues[10:0]),
        .write_data(activation_classify_index_r),
        .read_enable(active_index_read_enable),
        .read_address(index_request_address_r),
        .read_data(active_index_read_data)
    );

    neuraedge_activation_buffer u_activation_buffer (
        .clk(clk),
        .write_enable(activation_write_enable),
        .write_address(activation_write_address),
        .write_data(s_axis_tdata),
        .read_address(activation_read_address),
        .read_data(activation_read_data)
    );

    for (genvar bank = 0; bank < WEIGHT_BANKS; bank++) begin : g_weight_bank
        neuraedge_weight_bank u_weight_bank (
            .clk(clk),
            .write_enable(weight_commit_enable_r[bank]),
            .write_address(weight_commit_address_r),
            .write_segment(weight_commit_segment_r),
            .write_data(weight_commit_data_r),
            .read_address(weight_read_address),
            .read_data(weight_read_data[bank])
        );
    end

    neuraedge_tensor_tile #(.COLUMN_LANES(COLUMN_LANES)) u_tensor_tile (
        .clk(clk),
        .rst_n(rst_n),
        .start(tile_start),
        .compute_mode(compute_mode_r),
        .active_rows(compute_rows_r),
        .active_columns(compute_columns_r),
        .activation_zero_point(compute_activation_zp_r),
        .weight_zero_point(compute_weight_zp_r),
        .config_error(tile_config_error),
        .issue_valid(tile_issue_valid),
        .issue_ready(tile_issue_ready),
        .activation_vector(tile_activation_vector),
        .outer_weight_vector(tile_outer_weight_vector),
        .dot_weight_vector(tile_dot_weight_vector),
        .issue_last(tile_issue_last),
        .busy(),
        .result_valid(tile_result_valid),
        .result_ready(tile_result_ready),
        .result_value(tile_result_value),
        .result_row(tile_result_row),
```

```
        .result_column(tile_result_column),
        .result_last(tile_result_last),
        .debug_accepted_issues(),
        .debug_skipped_issues(),
        .debug_retired_issues(),
        .debug_dense_macs(tile_debug_dense_macs),
        .debug_spike_adds(tile_debug_spike_adds)
    );

`ifndef SYNTHESIS
    // The tile's mathematical per-command maximum is 262,144, so its debug
    // counters must remain 19-bit quantities.  Keep the fast accumulator's
    // width assumption checked in every RTL simulation.
    always_ff @(posedge clk) begin
        if (rst_n && tile_result_valid && tile_result_ready && tile_result_last
                && ((|tile_debug_dense_macs[63:19])
                    || (|tile_debug_spike_adds[63:19]))) begin
            $fatal(1, "tile activity counter exceeded its 19-bit contract");
        end
    end
`endif

    always_comb begin
        command_ready = (state == ST_IDLE);
        status_busy = (state != ST_IDLE);
        s_axis_tready = !load_finish_pending_r
            && (state == ST_LOAD_ACTIVATIONS
                || state == ST_LOAD_WEIGHTS_OUTER
                || state == ST_LOAD_WEIGHTS_DOT8);

        m_axis_tvalid = tensor_output_fifo_count_r != 0;
        m_axis_tdata = tensor_output_fifo_data_r[
            tensor_output_fifo_read_pointer_r];
        m_axis_tlast = tensor_output_fifo_head_last;
        tile_result_ready = state == ST_COMPUTE_OUTPUT
            && tile_result_ready_r;

        activation_write_enable = state == ST_LOAD_ACTIVATIONS
            && s_axis_tvalid && s_axis_tready;
        activation_write_address = load_item_index[10:0];
        weight_write_enable = '0;
        weight_write_address = 8'd0;
        weight_write_segment = {1'b0, load_segment_index_r[0]};
        if (state == ST_LOAD_WEIGHTS_OUTER && s_axis_tvalid && s_axis_tready) begin
            weight_write_enable[load_item_index[2:0]] = 1'b1;
            weight_write_address = load_item_index[10:3];
        end else if (state == ST_LOAD_WEIGHTS_DOT8 && s_axis_tvalid && s_axis_tready) begin
            weight_write_enable[load_segment_index_r[3:1]] = 1'b1;
            weight_write_address = load_item_index[7:0];
        end

        // A registered two-entry queue terminates tile READY before it can
        // propagate through every BRAM-response stage. Deliberately refusing
        // a same-cycle push while full costs at most one recovery cycle and
        // keeps the queue credit independent of downstream combinational logic.
        issue_fifo_available = issue_fifo_count_r != 2;
        capture_stage_available = !capture_valid_r || issue_fifo_available;
        memory_stage_available = !memory_valid_r || capture_stage_available;
        read_stage_available = !read_valid_r || memory_stage_available;
        map_stage_available = !map_valid_r || read_stage_available;
        index_stage_available = !index_valid_r || map_stage_available;
        index_request_stage_available = !index_request_valid_r
            || index_stage_available;
        compute_read_request = state == ST_COMPUTE_RUN
            && schedule_active_r
            && index_request_stage_available;
        activation_read_address = mapped_index_r;
        weight_read_address = compute_mode_r == MODE_DENSE_DOT8
            ? mapped_index_r[7:0] : mapped_index_r[10:3];

        tile_issue_valid = issue_fifo_count_r != 0;
        tile_issue_last = issue_fifo_last_r[issue_fifo_read_pointer_r];
        tile_activation_vector =
            issue_fifo_activation_r[issue_fifo_read_pointer_r];
        tile_outer_weight_vector =
            issue_fifo_outer_weights_r[issue_fifo_read_pointer_r];
        tile_dot_weight_vector =
            issue_fifo_dot_weights_r[issue_fifo_read_pointer_r];

        tensor_output_fifo_count_next = tensor_output_fifo_count_r;
        unique case ({tensor_output_fifo_push, tensor_output_fifo_pop})
            2'b10: tensor_output_fifo_count_next
                = tensor_output_fifo_count_r + 1'b1;
            2'b01: tensor_output_fifo_count_next
                = tensor_output_fifo_count_r - 1'b1;
            default: begin end
        endcase
```

```
    end

    // The FIFO owns the timing boundary independently of the command FSM.  Its
    // READY register is derived from post-transfer occupancy, which permits a
    // simultaneous dequeue/enqueue every clock and deasserts early enough to
    // reserve the second slot when the downstream interface stops.
    always_ff @(posedge clk) begin
        if (!rst_n) begin
            tile_result_ready_r <= 1'b0;
            tensor_output_fifo_count_r <= 2'd0;
            tensor_output_fifo_write_pointer_r <= 1'b0;
            tensor_output_fifo_read_pointer_r <= 1'b0;
            for (int output_entry = 0; output_entry < 2; output_entry++) begin
                tensor_output_fifo_data_r[output_entry] <= 64'd0;
                tensor_output_fifo_last_r[output_entry] <= 1'b0;
            end
        end else begin
            if (state == ST_COMPUTE_START) begin
                tensor_output_fifo_count_r <= 2'd0;
                tensor_output_fifo_write_pointer_r <= 1'b0;
                tensor_output_fifo_read_pointer_r <= 1'b0;
            end else begin
                tensor_output_fifo_count_r <= tensor_output_fifo_count_next;
                if (tensor_output_fifo_push) begin
                    tensor_output_fifo_data_r[tensor_output_fifo_write_pointer_r]
                        <= {5'd0, tile_result_last, tile_result_column,
                            tile_result_row, tile_result_value};
                    tensor_output_fifo_last_r[tensor_output_fifo_write_pointer_r]
                        <= tile_result_last;
                    tensor_output_fifo_write_pointer_r
                        <= !tensor_output_fifo_write_pointer_r;
                end
                if (tensor_output_fifo_pop) begin
                    tensor_output_fifo_read_pointer_r
                        <= !tensor_output_fifo_read_pointer_r;
                end
            end

            tile_result_ready_r <= (state == ST_COMPUTE_OUTPUT
                    && tensor_output_fifo_count_next != 2)
                || (state == ST_COMPUTE_RUN && tile_last_issue_accepted_r);
        end
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            state <= ST_IDLE;
            captured_command_r <= 42'd0;
            decoded_opcode_r <= 4'd0;
            decoded_issue_count_r <= 12'd0;
            decoded_outer_issue_error_r <= 1'b0;
            decoded_dot8_issue_error_r <= 1'b0;
            command_done_pending_r <= 1'b0;
            error_event_r <= 1'b0;
            clear_error_r <= 1'b0;
            if (state == ST_COMPUTE_RUN && tile_config_error) begin
                error_event_r <= 1'b1;
            end
            pending_load_opcode_r <= 4'd0;
            pending_load_issue_count_r <= 12'd0;
            pending_load_activation_zp_r <= 8'd0;
            command_done <= 1'b0;
            status_error <= 1'b0;
            status_loaded_activations <= 12'd0;
            status_loaded_weights <= 12'd0;
            status_active_issues <= 12'd0;
            status_input_beats <= 32'd0;
            status_output_beats <= 32'd0;
            status_compute_cycles <= 32'd0;
            status_skipped_issues <= 32'd0;
            status_dense_macs <= 64'd0;
            status_spike_adds <= 64'd0;
            pending_dense_macs_low_r <= 24'd0;
            pending_dense_macs_carry_r <= 1'b0;
            pending_spike_adds_low_r <= 24'd0;
            pending_spike_adds_carry_r <= 1'b0;
            load_requested_issues <= 12'd0;
            load_beats_remaining_r <= 15'd0;
            load_finish_pending_r <= 1'b0;
            load_segment_index_r <= 4'd0;
            load_item_index <= 12'd0;
            load_activation_zero_point <= 8'd0;
            weight_format_dot8 <= 1'b0;
            weight_commit_enable_r <= '0;
            weight_commit_address_r <= 8'd0;
            weight_commit_segment_r <= 2'd0;
            weight_commit_data_r <= 64'd0;
```

```
            activation_classify_valid_r <= 1'b0;
            activation_classify_nonzero_r <= 1'b0;
            activation_classify_index_r <= 11'd0;
            activation_load_success_r <= 1'b0;
            compute_mode_r <= MODE_DENSE_OUTER;
            compute_rows_r <= 4'd1;
            compute_columns_r <= 6'd1;
            compute_requested_issues <= 12'd0;
            compute_validation_error_r <= 1'b0;
            compute_activation_zp_r <= 8'd0;
            compute_weight_zp_r <= 8'd0;
            schedule_pointer <= 12'd0;
            schedule_remaining_r <= 12'd0;
            schedule_active_r <= 1'b0;
            schedule_last_r <= 1'b0;
            compute_empty_r <= 1'b0;
            index_request_valid_r <= 1'b0;
            index_request_last_r <= 1'b0;
            index_request_address_r <= 11'd0;
            index_valid_r <= 1'b0;
            index_last_r <= 1'b0;
            map_valid_r <= 1'b0;
            map_last_r <= 1'b0;
            mapped_index_r <= 11'd0;
            read_valid_r <= 1'b0;
            read_bank_select_r <= 3'd0;
            read_last_r <= 1'b0;
            memory_valid_r <= 1'b0;
            memory_bank_select_r <= 3'd0;
            memory_last_r <= 1'b0;
            capture_valid_r <= 1'b0;
            capture_last_r <= 1'b0;
            capture_activation_r <= 64'd0;
            capture_outer_weights_r <= '0;
            capture_dot_weights_r <= '0;
            issue_fifo_count_r <= 2'd0;
            issue_fifo_write_pointer_r <= 1'b0;
            issue_fifo_read_pointer_r <= 1'b0;
            tile_last_issue_accepted_r <= 1'b0;
            for (int fifo_entry = 0; fifo_entry < 2; fifo_entry++) begin
                issue_fifo_last_r[fifo_entry] <= 1'b0;
                issue_fifo_activation_r[fifo_entry] <= 64'd0;
                issue_fifo_outer_weights_r[fifo_entry] <= '0;
                issue_fifo_dot_weights_r[fifo_entry] <= '0;
            end
        end else begin
            // Register the accepted write request before it reaches sixteen
            // BRAM enable pins. This remains a one-beat-per-cycle pipeline:
            // the banks commit the prior request while this stage captures
            // the next one.
            weight_commit_enable_r <= weight_write_enable;
            if (|weight_write_enable) begin
                weight_commit_address_r <= weight_write_address;
                weight_commit_segment_r <= weight_write_segment;
                weight_commit_data_r <= s_axis_tdata;
            end
            command_done <= command_done_pending_r;
            command_done_pending_r <= 1'b0;
            if (clear_error_r) status_error <= 1'b0;
            else if (error_event_r) status_error <= 1'b1;
            error_event_r <= 1'b0;
            clear_error_r <= 1'b0;
            activation_classify_valid_r <= 1'b0;
            tile_last_issue_accepted_r <= tile_issue_valid
                && tile_issue_ready && tile_issue_last;

            if (issue_fifo_push) begin
                issue_fifo_last_r[issue_fifo_write_pointer_r] <= capture_last_r;
                issue_fifo_activation_r[issue_fifo_write_pointer_r]
                    <= capture_activation_r;
                issue_fifo_outer_weights_r[issue_fifo_write_pointer_r]
                    <= capture_outer_weights_r;
                issue_fifo_dot_weights_r[issue_fifo_write_pointer_r]
                    <= capture_dot_weights_r;
                issue_fifo_write_pointer_r <= !issue_fifo_write_pointer_r;
            end
            if (issue_fifo_pop) begin
                issue_fifo_read_pointer_r <= !issue_fifo_read_pointer_r;
            end
            unique case ({issue_fifo_push, issue_fifo_pop})
                2'b10: issue_fifo_count_r <= issue_fifo_count_r + 1'b1;
                2'b01: issue_fifo_count_r <= issue_fifo_count_r - 1'b1;
                default: begin end
            endcase

            if (activation_classify_valid_r && activation_classify_nonzero_r) begin
                status_active_issues <= status_active_issues + 1'b1;
```

```
557             end
558
559             if (state == ST_COMPUTE_START || state == ST_COMPUTE_RUN
560                     || state == ST_COMPUTE_OUTPUT
561                     || state == ST_COMPUTE_DRAIN) begin
562                 status_compute_cycles <= status_compute_cycles + 1'b1;
563             end
564             if (tensor_output_fifo_pop) begin
565                 status_output_beats <= status_output_beats + 1'b1;
566             end
567
568             unique case (state)
569                 ST_IDLE: begin
570                     map_valid_r <= 1'b0;
571                     read_valid_r <= 1'b0;
572                     memory_valid_r <= 1'b0;
573                     issue_fifo_count_r <= 2'd0;
574                     issue_fifo_write_pointer_r <= 1'b0;
575                     issue_fifo_read_pointer_r <= 1'b0;
576                     if (command_valid) begin
577                         captured_command_r <= command_data[41:0];
578                         state <= ST_PREDECODE_COMMAND;
579                     end
580                 end
581
582                 ST_PREDECODE_COMMAND: begin
583                     decoded_opcode_r <= captured_command_r[3:0];
584                     decoded_issue_count_r <= captured_command_r[25:14];
585                     decoded_outer_issue_error_r <= captured_command_r[25:14] == 0
586                         || captured_command_r[25:14] > 12'(MAX_OUTER_ISSUES);
587                     decoded_dot8_issue_error_r <= captured_command_r[25:14] == 0
588                         || captured_command_r[25:14] > 12'(MAX_DOT8_ISSUES);
589                     state <= ST_DECODE_COMMAND;
590                 end
591
592                 ST_DECODE_COMMAND: begin
593                     // Default back to idle for immediate commands and rejected
594                     // requests. Valid streaming/compute commands override this.
595                     state <= ST_IDLE;
596                     begin
597                         logic [3:0] opcode;
598                         logic [11:0] issue_count;
599                         opcode = decoded_opcode_r;
600                         issue_count = decoded_issue_count_r;
601
602                         unique case (opcode)
603                             CMD_LOAD_ACTIVATIONS: begin
604                                 if (decoded_outer_issue_error_r) begin
605                                     error_event_r <= 1'b1;
606                                     status_loaded_activations <= 12'd0;
607                                     status_active_issues <= 12'd0;
608                                     command_done_pending_r <= 1'b1;
609                                 end else begin
610                                     pending_load_opcode_r <= opcode;
611                                     pending_load_issue_count_r <= issue_count;
612                                     pending_load_activation_zp_r
613                                         <= captured_command_r[33:26];
614                                     state <= ST_LOAD_CONFIG;
615                                 end
616                             end
617
618                             CMD_LOAD_WEIGHTS_OUTER: begin
619                                 if (decoded_outer_issue_error_r) begin
620                                     error_event_r <= 1'b1;
621                                     status_loaded_weights <= 12'd0;
622                                     command_done_pending_r <= 1'b1;
623                                 end else begin
624                                     pending_load_opcode_r <= opcode;
625                                     pending_load_issue_count_r <= issue_count;
626                                     state <= ST_LOAD_CONFIG;
627                                 end
628                             end
629
630                             CMD_LOAD_WEIGHTS_DOT8: begin
631                                 if (decoded_dot8_issue_error_r) begin
632                                     error_event_r <= 1'b1;
633                                     status_loaded_weights <= 12'd0;
634                                     command_done_pending_r <= 1'b1;
635                                 end else begin
636                                     pending_load_opcode_r <= opcode;
637                                     pending_load_issue_count_r <= issue_count;
638                                     state <= ST_LOAD_CONFIG;
639                                 end
640                             end
641
642                             CMD_COMPUTE_DENSE_OUTER,
643                             CMD_COMPUTE_DENSE_DOT8,
```

```
                        CMD_COMPUTE_BINARY,
                        CMD_COMPUTE_TERNARY: begin
                            unique case (opcode)
                                CMD_COMPUTE_DENSE_OUTER:
                                    compute_mode_r <= MODE_DENSE_OUTER;
                                CMD_COMPUTE_DENSE_DOT8:
                                    compute_mode_r <= MODE_DENSE_DOT8;
                                CMD_COMPUTE_BINARY:
                                    compute_mode_r <= MODE_BINARY_WEIGHTED;
                                default: compute_mode_r <= MODE_TERNARY_WEIGHTED;
                            endcase
                            compute_rows_r <= captured_command_r[7:4];
                            compute_columns_r <= captured_command_r[13:8];
                            compute_requested_issues <= issue_count;
                            compute_activation_zp_r <= captured_command_r[33:26];
                            compute_weight_zp_r <= captured_command_r[41:34];
                            state <= ST_COMPUTE_VALIDATE;
                        end

                        CMD_CLEAR_STATUS: begin
                            clear_error_r <= 1'b1;
                            status_input_beats <= 32'd0;
                            status_output_beats <= 32'd0;
                            status_compute_cycles <= 32'd0;
                            status_skipped_issues <= 32'd0;
                            status_dense_macs <= 64'd0;
                            status_spike_adds <= 64'd0;
                            command_done_pending_r <= 1'b1;
                        end

                        default: begin
                            error_event_r <= 1'b1;
                            command_done_pending_r <= 1'b1;
                        end
                    endcase
                end
            end

            ST_LOAD_CONFIG: begin
                load_requested_issues <= pending_load_issue_count_r;
                load_segment_index_r <= 4'd0;
                load_item_index <= 12'd0;
                load_finish_pending_r <= 1'b0;
                unique case (pending_load_opcode_r)
                    CMD_LOAD_ACTIVATIONS: begin
                        load_beats_remaining_r <=
                            {3'd0, pending_load_issue_count_r};
                        load_activation_zero_point <= pending_load_activation_zp_r;
                        activation_load_success_r <= 1'b0;
                        status_active_issues <= 12'd0;
                        state <= ST_LOAD_ACTIVATIONS;
                    end
                    CMD_LOAD_WEIGHTS_OUTER: begin
                        load_beats_remaining_r <=
                            {2'd0, pending_load_issue_count_r, 1'b0};
                        weight_format_dot8 <= 1'b0;
                        state <= ST_LOAD_WEIGHTS_OUTER;
                    end
                    default: begin
                        load_beats_remaining_r <=
                            {2'd0, pending_load_issue_count_r[8:0], 4'd0};
                        weight_format_dot8 <= 1'b1;
                        state <= ST_LOAD_WEIGHTS_DOT8;
                    end
                endcase
            end

            ST_COMPUTE_CONFIG: begin
                compute_empty_r <= (status_active_issues == 0);
                schedule_remaining_r <= status_active_issues == 0
                    ? 12'd1 : status_active_issues;
                schedule_active_r <= 1'b1;
                schedule_last_r <= status_active_issues <= 1;
                schedule_pointer <= 12'd0;
                status_compute_cycles <= 32'd0;
                status_output_beats <= 32'd0;
                state <= ST_COMPUTE_START;
            end

            ST_COMPUTE_VALIDATE: begin
                // End the multi-field legality predicate at one local
                // register instead of placing it on every parameter mux.
                compute_validation_error_r <= compute_requested_issues == 0
                    || compute_rows_r == 0
                    || compute_rows_r > 8
                    || compute_columns_r == 0
                    || compute_columns_r > 6'(COLUMN_LANES)
```

```
                        || (compute_mode_r == MODE_DENSE_DOT8
                            && compute_rows_r != 1)
                        || compute_requested_issues != status_loaded_activations
                        || compute_requested_issues != status_loaded_weights
                        || (compute_mode_r == MODE_DENSE_DOT8)
                            != weight_format_dot8;
                    state <= ST_COMPUTE_VALIDATED;
                end

                ST_COMPUTE_VALIDATED: begin
                    if (compute_validation_error_r) begin
                        error_event_r <= 1'b1;
                        command_done_pending_r <= 1'b1;
                        state <= ST_IDLE;
                    end else begin
                        state <= ST_COMPUTE_CONFIG;
                    end
                end

                ST_LOAD_ACTIVATIONS: begin
                    if (load_finish_pending_r) begin
                        load_finish_pending_r <= 1'b0;
                        state <= ST_LOAD_ACTIVATION_DRAIN;
                    end else if (s_axis_tvalid && s_axis_tready) begin
                        logic beat_is_last;
                        logic byte_vector_is_zero;
                        beat_is_last = load_beats_remaining_r == 15'd1;
                        byte_vector_is_zero = all_bytes_equal64(
                            s_axis_tdata, load_activation_zero_point);
                        activation_classify_valid_r <= 1'b1;
                        activation_classify_nonzero_r <= !byte_vector_is_zero;
                        activation_classify_index_r <= load_item_index[10:0];
                        status_input_beats <= status_input_beats + 1'b1;
                        load_beats_remaining_r <= load_beats_remaining_r - 1'b1;
                        load_item_index <= load_item_index + 1'b1;
                        status_loaded_activations <= beat_is_last
                            ? load_requested_issues : 12'd0;

                        if (s_axis_tlast != beat_is_last) error_event_r <= 1'b1;
                        if (beat_is_last || s_axis_tlast) begin
                            activation_load_success_r <= beat_is_last;
                            load_finish_pending_r <= 1'b1;
                        end
                    end
                end

                ST_LOAD_ACTIVATION_DRAIN: begin
                    if (!activation_classify_valid_r) begin
                        if (!activation_load_success_r) status_active_issues <= 12'd0;
                        command_done_pending_r <= 1'b1;
                        state <= ST_IDLE;
                    end
                end

                ST_LOAD_WEIGHTS_OUTER: begin
                    if (load_finish_pending_r) begin
                        load_finish_pending_r <= 1'b0;
                        command_done_pending_r <= 1'b1;
                        state <= ST_IDLE;
                    end else if (s_axis_tvalid && s_axis_tready) begin
                        logic [1:0] word_segment;
                        logic beat_is_last;
                        word_segment = {1'b0, load_segment_index_r[0]};
                        beat_is_last = load_beats_remaining_r == 15'd1;
                        status_input_beats <= status_input_beats + 1'b1;
                        load_beats_remaining_r <= load_beats_remaining_r - 1'b1;
                        status_loaded_weights <= beat_is_last
                            ? load_requested_issues : 12'd0;
                        if (word_segment == 2'd1) begin
                            load_item_index <= load_item_index + 1'b1;
                            load_segment_index_r <= 4'd0;
                        end else begin
                            load_segment_index_r <= 4'd1;
                        end

                        if (s_axis_tlast != beat_is_last) error_event_r <= 1'b1;
                        if (beat_is_last || s_axis_tlast) begin
                            load_finish_pending_r <= 1'b1;
                        end
                    end
                end

                ST_LOAD_WEIGHTS_DOT8: begin
                    if (load_finish_pending_r) begin
                        load_finish_pending_r <= 1'b0;
                        command_done_pending_r <= 1'b1;
                        state <= ST_IDLE;
```

```
818                     end else if (s_axis_tvalid && s_axis_tready) begin
819                         logic beat_is_last;
820                         beat_is_last = load_beats_remaining_r == 15'd1;
821                         status_input_beats <= status_input_beats + 1'b1;
822                         load_beats_remaining_r <= load_beats_remaining_r - 1'b1;
823                         status_loaded_weights <= beat_is_last
824                             ? load_requested_issues : 12'd0;
825                         if (load_segment_index_r == 4'd15) begin
826                             load_item_index <= load_item_index + 1'b1;
827                             load_segment_index_r <= 4'd0;
828                         end else begin
829                             load_segment_index_r <= load_segment_index_r + 1'b1;
830                         end
831
832                         if (s_axis_tlast != beat_is_last) error_event_r <= 1'b1;
833                         if (beat_is_last || s_axis_tlast) begin
834                             load_finish_pending_r <= 1'b1;
835                         end
836                     end
837                 end
838
839                 ST_COMPUTE_START: begin
840                     schedule_pointer <= 12'd0;
841                     index_request_valid_r <= 1'b0;
842                     index_valid_r <= 1'b0;
843                     map_valid_r <= 1'b0;
844                     read_valid_r <= 1'b0;
845                     memory_valid_r <= 1'b0;
846                     capture_valid_r <= 1'b0;
847                     issue_fifo_count_r <= 2'd0;
848                     issue_fifo_write_pointer_r <= 1'b0;
849                     issue_fifo_read_pointer_r <= 1'b0;
850                     state <= ST_COMPUTE_RUN;
851                 end
852
853                 ST_COMPUTE_RUN: begin
854                     if (capture_stage_available) begin
855                         capture_valid_r <= memory_valid_r;
856                         if (memory_valid_r) begin
857                             capture_last_r <= memory_last_r;
858                             if (compute_empty_r) begin
859                                 capture_activation_r <= (compute_mode_r == MODE_DENSE_OUTER
860                                         || compute_mode_r == MODE_DENSE_DOT8)
861                                     ? repeat_byte(compute_activation_zp_r) : 64'd0;
862                                 capture_outer_weights_r <= '0;
863                                 capture_dot_weights_r <= '0;
864                             end else begin
865                                 capture_activation_r <= activation_read_data;
866                                 capture_outer_weights_r
867                                     <= weight_read_data[memory_bank_select_r];
868                                 capture_dot_weights_r <= {
869                                     weight_read_data[7], weight_read_data[6],
870                                     weight_read_data[5], weight_read_data[4],
871                                     weight_read_data[3], weight_read_data[2],
872                                     weight_read_data[1], weight_read_data[0]
873                                 };
874                             end
875                         end
876                     end
877
878                     if (memory_stage_available) begin
879                         memory_valid_r <= read_valid_r;
880                         if (read_valid_r) begin
881                             memory_bank_select_r <= read_bank_select_r;
882                             memory_last_r <= read_last_r;
883                         end
884                     end
885
886                     if (read_stage_available) begin
887                         read_valid_r <= map_valid_r;
888                         if (map_valid_r) begin
889                             read_bank_select_r <= mapped_index_r[2:0];
890                             read_last_r <= map_last_r;
891                         end
892                     end
893
894                     if (map_stage_available) begin
895                         map_valid_r <= index_valid_r;
896                         if (index_valid_r) begin
897                             mapped_index_r <= active_mapped_index;
898                             map_last_r <= index_last_r;
899                         end
900                     end
901
902                     if (index_stage_available) begin
903                         index_valid_r <= index_request_valid_r;
904                         if (index_request_valid_r) begin
```

```
                            index_last_r <= index_request_last_r;
                        end
                    end

                    if (index_request_stage_available) begin
                        index_request_valid_r <= compute_read_request;
                        if (compute_read_request) begin
                            index_request_address_r <= schedule_pointer[10:0];
                            index_request_last_r <= schedule_last_r;
                            schedule_last_r <= schedule_remaining_r == 12'd2;
                            schedule_remaining_r <= schedule_remaining_r - 1'b1;
                            if (schedule_remaining_r == 12'd1) begin
                                schedule_active_r <= 1'b0;
                            end
                            schedule_pointer <= schedule_pointer + 1'b1;
                        end
                    end

                    if (tile_last_issue_accepted_r) begin
                        state <= ST_COMPUTE_OUTPUT;
                    end
                end

                ST_COMPUTE_OUTPUT: begin
                    status_skipped_issues <= {20'd0, compute_requested_issues}
                        - {20'd0, status_active_issues};
                    if (tensor_output_fifo_push) begin
                        if (tile_result_last) begin
                            {pending_dense_macs_carry_r,
                             pending_dense_macs_low_r}
                                <= add_u19_to_u24(
                                    status_dense_macs[23:0],
                                    tile_debug_dense_macs[18:0]);
                            {pending_spike_adds_carry_r,
                             pending_spike_adds_low_r}
                                <= add_u19_to_u24(
                                    status_spike_adds[23:0],
                                    tile_debug_spike_adds[18:0]);
                            state <= ST_COMPUTE_DRAIN;
                        end
                    end
                end

                ST_COMPUTE_DRAIN: begin
                    if (tensor_output_fifo_pop
                            && tensor_output_fifo_head_last) begin
                        state <= ST_COMPUTE_ACCOUNT;
                    end
                end

                ST_COMPUTE_ACCOUNT: begin
                    status_dense_macs <= {
                        status_dense_macs[63:24]
                            + {{39{1'b0}}, pending_dense_macs_carry_r},
                        pending_dense_macs_low_r
                    };
                    status_spike_adds <= {
                        status_spike_adds[63:24]
                            + {{39{1'b0}}, pending_spike_adds_carry_r},
                        pending_spike_adds_low_r
                    };
                    command_done_pending_r <= 1'b1;
                    state <= ST_IDLE;
                end

                default: state <= ST_IDLE;
            endcase
        end
    end

    initial begin
        if (MAX_OUTER_ISSUES < 8 || MAX_OUTER_ISSUES > 2048) begin
            $fatal(1, "MAX_OUTER_ISSUES must be in [8,2048]");
        end
        if (MAX_DOT8_ISSUES != MAX_OUTER_ISSUES / 8) begin
            $fatal(1, "MAX_DOT8_ISSUES must equal MAX_OUTER_ISSUES/8");
        end
    end
endmodule

`default_nettype wire
```

6.3 Line annotations

Line 1: `‘timescale 1ns / 1ps` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `‘default_nettype none` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_buffered_tensor_core #(` This opens neuraedge_buffered_tensor_core, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 5: `parameter int MAX_OUTER_ISSUES = 2048,` This statement continues through source line 38; the explanation uses the complete expression rather than only the first visual fragment. This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 6: `parameter int MAX_DOT8_ISSUES = 256,` This is continuation line 2 of the 34-line logical statement that begins at source line 5. Its exact fragment is ‘parameter int MAX_DOT8_ISSUES = 256,‘. The important operands are parameter: the value named 'parameter' in this block; MAX_DOT8_ISSUES: the value named 'max dot8 issues' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 7: `parameter int COLUMN_LANES = 16` This is continuation line 3 of the 34-line logical statement that begins at source line 5. Its exact fragment is ‘parameter int COLUMN_LANES = 16‘. The important operands are parameter: the value named 'parameter' in this block; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 8: `) (` This is continuation line 4 of the 34-line logical statement that begins at source line 5. Its exact fragment is ‘) (‘. The expression is a literal or structural operator and has no hidden software state. Read together,

the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 9: `input wire clk,` This is continuation line 5 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'input wire clk,'. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 10: `input wire rst_n,` This is continuation line 6 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'input wire rst_n,'. The important operands are input: the value named 'input' in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 12: `input wire command_valid,` This is continuation line 8 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'input wire command_valid,'. The important operands are input: the value named 'input' in this block; command_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 13: `output logic command_ready,` This is continuation line 9 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic command_ready,'. The important operands are output: the value named 'output' in this block; command_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES

= 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 14: `input wire [41:0] command_data,` This is continuation line 10 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'input wire [41:0] command_data,'. The important operands are input: the value named 'input' in this block; command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 15: `output logic command_done,` This is continuation line 11 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic command_done,'. The important operands are output: the value named 'output' in this block; command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 16: `output logic status_error,` This is continuation line 12 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic status_error,'. The important operands are output: the value named 'output' in this block; status_error: the value named 'status error' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 17: `output logic status_busy,` This is continuation line 13 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic status_busy,'. The important operands are output: the value named 'output' in this block; status_busy: the value named 'status busy' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic

command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 19: `input wire s_axis_tvalid,` This is continuation line 15 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'input wire s_axis_tvalid,'. The important operands are input: the value named 'input' in this block; s_axis_tvalid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 20: `output logic s_axis_tready,` This is continuation line 16 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic s_axis_tready,'. The important operands are output: the value named 'output' in this block; s_axis_tready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 21: `input wire [63:0] s_axis_tdata,` This is continuation line 17 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'input wire [63:0] s_axis_tdata,'. The important operands are input: the value named 'input' in this block; s_axis_tdata: the value named 's axis tdata' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 22: `input wire s_axis_tlast,` This is continuation line 18 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'input wire s_axis_tlast,'. The important operands are input: the value named 'input' in this block; s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output

logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 24: `output logic m_axis_tvalid,` This is continuation line 20 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic m_axis_tvalid,'. The important operands are output: the value named 'output' in this block; m_axis_tvalid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 25: `input wire m_axis_tready,` This is continuation line 21 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'input wire m_axis_tready,'. The important operands are input: the value named 'input' in this block; m_axis_tready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 26: `output logic [63:0] m_axis_tdata,` This is continuation line 22 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [63:0] m_axis_tdata,'. The important operands are output: the value named 'output' in this block; m_axis_tdata: the value named 'm axis tdata' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 27: `output logic m_axis_tlast,` This is continuation line 23 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic m_axis_tlast,'. The important operands are output: the value named 'output' in this block; m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire

s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 29: `output logic [11:0] status_loaded_activations,` This is continuation line 25 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [11:0] status_loaded_activations,'. The important operands are output: the value named 'output' in this block; status_loaded_activations: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 30: `output logic [11:0] status_loaded_weights,` This is continuation line 26 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [11:0] status_loaded_weights,'. The important operands are output: the value named 'output' in this block; status_loaded_weights: a learned coefficient multiplied by an activation. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 31: `output logic [11:0] status_active_issues,` This is continuation line 27 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [11:0] status_active_issues,'. The important operands are output: the value named 'output' in this block; status_active_issues: the value named 'status active issues' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 32: `output logic [31:0] status_input_beats,` This is continuation line 28 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] status_input_beats,'. The important operands are output: the value named 'output' in this block; status_input_beats: the value named 'status input beats' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire

[63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 33: `output logic [31:0] status_output_beats,` This is continuation line 29 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] status_output_beats,'. The important operands are output: the value named 'output' in this block; status_output_beats: the value named 'status output beats' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 34: `output logic [31:0] status_compute_cycles,` This is continuation line 30 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] status_compute_cycles,'. The important operands are output: the value named 'output' in this block; status_compute_cycles: the value named 'status compute cycles' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 35: `output logic [31:0] status_skipped_issues,` This is continuation line 31 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] status_skipped_issues,'. The important operands are output: the value named 'output' in this block; status_skipped_issues: the value named 'status skipped issues' in this block. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 36: `output logic [63:0] status_dense_macs,` This is continuation line 32 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [63:0] status_dense_macs,'. The important operands are output: the value named 'output' in this block; status_dense_macs: a count of ordinary activation-times-weight operations actually issued. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready,

input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 37: `output logic [63:0] status_spike_adds` This is continuation line 33 of the 34-line logical statement that begins at source line 5. Its exact fragment is 'output logic [63:0] status_spike_adds'. The important operands are output: the value named 'output' in this block; status_spike_adds: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 38: `);` This is continuation line 34 of the 34-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes MAX_OUTER_ISSUES to 2048, parameter int MAX_DOT8_ISSUES = 256, parameter int COLUMN_LANES = 16) (input wire clk, input wire rst_n, input wire command_valid, output logic command_ready, input wire [41:0] command_data, output logic command_done, output logic status_error, output logic status_busy, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic [11:0] status_loaded_activations, output logic [11:0] status_loaded_weights, output logic [11:0] status_active_issues, output logic [31:0] status_input_beats, output logic [31:0] status_output_beats, output logic [31:0] status_compute_cycles, output logic [31:0] status_skipped_issues, output logic [63:0] status_dense_macs, output logic [63:0] status_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_OUTER_ISSUES denotes the value named 'max outer issues' in this block.

Line 39: `import neuraedge_command_pkg::*;` This imports shared package names into the current module so opcodes and mode constants have exactly one definition. The import changes compile-time name visibility; it does not create registers or consume a clock cycle.

Line 41: `localparam int WEIGHT_BANKS = 8;` This fixes WEIGHT_BANKS to 8 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; WEIGHT_BANKS denotes a learned coefficient multiplied by an activation.

Line 43: `typedef enum logic [3:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 44: `ST_IDLE = 4'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 45: `ST_LOAD_ACTIVATIONS = 4'd1,` This assigns a fixed binary encoding to state ST_LOAD_ACTIVATIONS, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 46: `ST_LOAD_WEIGHTS_OUTER = 4'd2,` This assigns a fixed binary encoding to state ST_LOAD_WEIGHTS_OUTER, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 47: `ST_LOAD_WEIGHTS_DOT8 = 4'd3,` This assigns a fixed binary encoding to state ST_LOAD_WEIGHTS_DOT8, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 48: `ST_COMPUTE_START = 4'd4,` This assigns a fixed binary encoding to state ST_COMPUTE_START, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 49: `ST_COMPUTE_RUN = 4'd5,` This assigns a fixed binary encoding to state ST_COMPUTE_RUN, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 50: `ST_COMPUTE_OUTPUT = 4'd6,` This assigns a fixed binary encoding to state ST_COMPUTE_OUTPUT, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 51: `ST_LOAD_ACTIVATION_DRAIN = 4'd7,` This assigns a fixed binary encoding to state ST_LOAD_ACTIVATION_DRAI which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 52: `ST_DECODE_COMMAND = 4'd8,` This assigns a fixed binary encoding to state ST_DECODE_COMMAND, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 53: `ST_COMPUTE_CONFIG = 4'd9,` This assigns a fixed binary encoding to state ST_COMPUTE_CONFIG, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 54: `ST_LOAD_CONFIG = 4'd10,` This assigns a fixed binary encoding to state ST_LOAD_CONFIG, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 55: `ST_COMPUTE_VALIDATE = 4'd11,` This assigns a fixed binary encoding to state ST_COMPUTE_VALIDATE, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 56: `ST_COMPUTE_VALIDATED = 4'd12,` This assigns a fixed binary encoding to state ST_COMPUTE_VALIDATED, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 57: `ST_PREDECODE_COMMAND = 4'd13,` This assigns a fixed binary encoding to state ST_PREDECODE_COMMAND, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 58: `ST_COMPUTE_ACCOUNT = 4'd14,` This assigns a fixed binary encoding to state ST_COMPUTE_ACCOUNT, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 59: `ST_COMPUTE_DRAIN = 4'd15` This statement continues through source line 60; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_COMPUTE_DRAIN, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 60: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 59. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_COMPUTE_DRAIN, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 62: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 63: `logic [41:0] captured_command_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, captured_command_r is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 64: `logic [3:0] decoded_opcode_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, decoded_opcode_r is the value named 'decoded opcode r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 65: `logic [11:0] decoded_issue_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, decoded_issue_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 66: `logic decoded_outer_issue_error_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, decoded_outer_issue_error_r is the value named 'decoded outer issue error r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 67: `logic decoded_dot8_issue_error_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, decoded_dot8_issue_error_r is the value named 'decoded dot8 issue error r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 68: `(* dont_touch = "yes" *) logic command_done_pending_r;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; command_done_pending_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 69: `(* dont_touch = "yes" *) logic error_event_r;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; error_event_r: the value named 'error event r' in this block.

Line 70: `(* dont_touch = "yes" *) logic clear_error_r;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; clear_error_r: the value named 'clear error r' in this block.

Line 71: `logic [3:0] pending_load_opcode_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pending_load_opcode_r is the value named 'pending load opcode r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 72: `logic [11:0] pending_load_issue_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pending_load_issue_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 73: `logic [7:0] pending_load_activation_zp_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pending_load_activation_zp_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 75: `// Eight 256-deep, 128-bit banks are the physical form of a 32 KiB weight` Design intent: Eight 256-deep, 128-bit banks are the physical form of a 32 KiB weight This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 76: `// tile. In` Design intent: tile. In This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 77: `// outer-product mode one bank supplies each consecutive K position. In` Design intent: outer-product mode one bank supplies each consecutive K position. In This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 78: `// Dot8 mode reads all eight banks together, exposing 8x16 weight bytes.` Design intent: Dot8 mode reads all eight banks together, exposing 8x16 weight bytes. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 79: `logic activation_write_enable;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_write_enable is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 80: `logic [10:0] activation_write_address;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_write_address is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 81: `logic [10:0] activation_read_address;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_read_address is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 82: `wire [63:0] activation_read_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_read_data is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 83: `logic [WEIGHT_BANKS-1:0] weight_write_enable;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_write_enable is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 84: `logic [7:0] weight_write_address;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_write_address is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 85: `logic [1:0] weight_write_segment;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_write_segment is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 86: `logic [WEIGHT_BANKS-1:0] weight_commit_enable_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_commit_enable_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 87: `logic [7:0] weight_commit_address_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_commit_address_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 88: `logic [1:0] weight_commit_segment_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_commit_segment_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 89: `logic [63:0] weight_commit_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_commit_data_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 90: `logic [7:0] weight_read_address;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_read_address is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 91: `wire [COLUMN_LANES*8-1:0] weight_read_data [0:WEIGHT_BANKS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_read_data is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 93: `logic [11:0] load_requested_issues;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_requested_issues is the value named 'load requested issues' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 94: `logic [14:0] load_beats_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_beats_remaining_r is the value named 'load beats remaining r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 95: `logic load_finish_pending_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_finish_pending_r is the value named 'load finish pending r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 96: `logic [3:0] load_segment_index_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_segment_index_r is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 97: `logic [11:0] load_item_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_item_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 98: `logic [7:0] load_activation_zero_point;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_activation_zero_point is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 99: `logic weight_format_dot8;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_format_dot8 is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 100: `logic activation_classify_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_classify_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 101: `logic activation_classify_nonzero_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_classify_nonzero_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 102: `logic [10:0] activation_classify_index_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_classify_index_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 103: `logic activation_load_success_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_load_success_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 104: `wire active_index_write_enable;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_index_write_enable is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 105: `wire [10:0] active_index_read_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_index_read_data is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 106: `wire active_index_read_enable;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_index_read_enable is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 107: `wire [10:0] active_mapped_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_mapped_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 109: `(* dont_touch = "yes" *) logic [1:0] compute_mode_r;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; compute_mode_r: the value named 'compute mode r' in this block.

Line 110: `(* dont_touch = "yes" *) logic [3:0] compute_rows_r;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; compute_rows_r: one output-row position processed by the tensor tile.

Line 111: `(* dont_touch = "yes" *) logic [5:0] compute_columns_r;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; compute_columns_r: one output-feature position processed in parallel.

Line 112: `logic [11:0] compute_requested_issues;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, compute_requested_issues is the value named 'compute requested issues' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 113: `logic compute_validation_error_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, compute_validation_error_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 114: `(* dont_touch = "yes" *) logic [7:0] compute_activation_zp_r;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; compute_activation_zp_r: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 115: `(* dont_touch = "yes" *) logic [7:0] compute_weight_zp_r;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; compute_weight_zp_r: a learned coefficient multiplied by an activation.

Line 116: `logic [11:0] schedule_pointer;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, schedule_pointer is the value named 'schedule pointer' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 117: `logic [11:0] schedule_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, schedule_remaining_r is the value named 'schedule remaining r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 118: `logic schedule_active_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, schedule_active_r is the value named 'schedule active r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 119: `logic schedule_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, schedule_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 120: `logic compute_empty_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, compute_empty_r is the value named 'compute empty r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 121: `logic compute_read_request;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, compute_read_request is the value named 'compute read request' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 122: `logic index_request_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, index_request_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 123: `logic index_request_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, index_request_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 124: `logic [10:0] index_request_address_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, index_request_address_r is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 125: `logic index_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, index_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 126: `logic index_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, index_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 127: `logic map_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, map_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 128: `logic map_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, map_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 129: `logic [10:0] mapped_index_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mapped_index_r is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 130: `logic read_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, read_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 131: `logic [2:0] read_bank_select_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, read_bank_select_r is the value named 'read bank select r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 132: `logic read_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, read_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 133: `logic memory_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, memory_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 134: `logic [2:0] memory_bank_select_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, memory_bank_select_r is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 135: `logic memory_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, memory_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 136: `logic capture_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, capture_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 137: `logic capture_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, capture_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 138: `logic [63:0] capture_activation_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, capture_activation_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 139: `logic [COLUMN_LANES*8-1:0] capture_outer_weights_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, capture_outer_weights_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 140: `logic [COLUMN_LANES*8*8-1:0] capture_dot_weights_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, capture_dot_weights_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 141: `logic [1:0] issue_fifo_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 142: `logic issue_fifo_write_pointer_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_write_pointer_r is the value named 'issue fifo write pointer r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 143: `logic issue_fifo_read_pointer_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_read_pointer_r is the value named 'issue fifo read pointer r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 144: `logic issue_fifo_last_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 145: `logic [63:0] issue_fifo_activation_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_activation_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 146: `logic [COLUMN_LANES*8-1:0] issue_fifo_outer_weights_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_outer_weights_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 147: `logic [COLUMN_LANES*8*8-1:0] issue_fifo_dot_weights_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_dot_weights_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 148: `(* dont_touch = "yes" *) logic tile_last_issue_accepted_r;` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; tile_last_issue_accepted_r: the AXI-Stream marker for the final beat of a packet or command payload.

Line 149: `logic issue_fifo_available;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_available is the value named 'issue fifo available' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 150: `logic capture_stage_available;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, capture_stage_available is the value named 'capture stage available' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 151: `logic memory_stage_available;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, memory_stage_available is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 152: `logic read_stage_available;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, read_stage_available is the value named 'read stage available' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 153: `logic map_stage_available;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, map_stage_available is the value named 'map stage available' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 154: `logic index_stage_available;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, index_stage_available is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 155: `logic index_request_stage_available;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, index_request_stage_available is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 157: `logic tile_start;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_start is the value named 'tile start' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 158: `logic tile_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_config_error is the value named 'tile config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 159: `logic tile_issue_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_issue_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 160: `wire tile_issue_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_issue_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 161: `logic [63:0] tile_activation_vector;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_activation_vector is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 162: `logic [COLUMN_LANES*8-1:0] tile_outer_weight_vector;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_outer_weight_vector is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 163: `logic [COLUMN_LANES*8*8-1:0] tile_dot_weight_vector;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_dot_weight_vector is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 164: `logic tile_issue_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_issue_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 165: `wire tile_result_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_result_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 166: `logic tile_result_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_result_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 167: `wire signed [47:0] tile_result_value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_result_value is the value named 'tile result value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 168: `wire [3:0] tile_result_row;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_result_row is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 169: `wire [5:0] tile_result_column;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_result_column is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 170: `wire tile_result_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_result_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 171: `// This two-entry skid FIFO breaks the external READY path before it reaches` Design intent: This two-entry skid FIFO breaks the external READY path before it reaches This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 172: `// the tile's 128-entry result store and replicated output-state controls.` Design intent: the tile's 128-entry result store and replicated output-state controls. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 173: `// A registered tile READY can remain asserted for one final transfer while` Design intent: A registered tile READY can remain asserted for one final transfer while This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 174: `// downstream backpressure arrives; the second entry is the space reserved` Design intent: downstream backpressure arrives; the second entry is the space reserved This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 175: `// for that transfer, so no result is dropped and sustained throughput stays` Design intent: for that transfer, so no result is dropped and sustained throughput stays This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 176: `// at one result per clock whenever the parent accepts one per clock.` Design intent: at one result per clock whenever the parent accepts one per clock. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 177: `logic tile_result_ready_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_result_ready_r is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 178: `logic [1:0] tensor_output_fifo_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 179: `logic tensor_output_fifo_write_pointer_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_write_pointer_r is the value named 'tensor output fifo write pointer r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 180: `logic tensor_output_fifo_read_pointer_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_read_pointer_r is the value named 'tensor output fifo read pointer r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 181: `logic [63:0] tensor_output_fifo_data_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 182: `logic tensor_output_fifo_last_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 183: `wire [63:0] tile_debug_dense_macs;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_debug_dense_macs is a count of ordinary activation-times-weight operations actually issued. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 184: `wire [63:0] tile_debug_spike_adds;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tile_debug_spike_adds is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 185: `logic [23:0] pending_dense_macs_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pending_dense_macs_low_r is a count of ordinary activation-times-weight operations actually issued. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 186: `logic pending_dense_macs_carry_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pending_dense_macs_carry_r is a count of ordinary activation-times-weight operations actually issued. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 187: `logic [23:0] pending_spike_adds_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pending_spike_adds_low_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 188: `logic pending_spike_adds_carry_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pending_spike_adds_carry_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 189: `wire issue_fifo_push = capture_valid_r && issue_fifo_available;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_push is the value named 'issue fifo push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 190: `wire issue_fifo_pop = issue_fifo_count_r != 0 && tile_issue_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_fifo_pop is the value named 'issue fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 191: `wire tensor_output_fifo_push = state == ST_COMPUTE_OUTPUT` This statement continues through source line 192; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_push is the value named 'tensor output fifo push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 192: `&& tile_result_valid && tile_result_ready_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 191. Its exact fragment is '&& tile_result_valid && tile_result_ready_r;'. The important operands are tile_result_valid: the producer's statement that its current payload is meaningful; tile_result_ready_r: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_push is the value named 'tensor output fifo push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 193: `wire tensor_output_fifo_pop = tensor_output_fifo_count_r != 0` This statement continues through source line 194; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_pop is the value named 'tensor output fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 194: `&& m_axis_tready;` This is continuation line 2 of the 2-line logical statement that begins at source line 193. Its exact fragment is '&& m_axis_tready;'. The important operands are m_axis_tready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_pop is the value named 'tensor output fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 195: `wire tensor_output_fifo_head_last = tensor_output_fifo_last_r[` This statement continues through source line 196; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_head_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 196: `tensor_output_fifo_read_pointer_r];` This is continuation line 2 of the 2-line logical statement that begins at source line 195. Its exact fragment is 'tensor_output_fifo_read_pointer_r];'. The important operands are tensor_output_fifo_read_pointer_r: the value named 'tensor output fifo read pointer r' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_head_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 197: `logic [1:0] tensor_output_fifo_count_next;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_output_fifo_count_next is the value named 'tensor output fifo count next' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 199: `function automatic logic all_bytes_equal64(` Within the combinational helper all bytes equal64, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; all_bytes_equal64: the value named 'all bytes equal64' in this block.

Line 200: `input logic [63:0] values,` This declares values as a input port with width [63:0]. It carries the value named 'values' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 201: `input logic [7:0] reference` This declares reference as a input port with width [7:0]. It carries the value named 'reference' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 202: `);` Within the combinational helper all bytes equal64, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The expression is a literal or structural operator and has no hidden software state.

Line 203: `logic equal_all;` This declares hardware signal storage or wiring in the combinational helper all bytes equal64. Specifically, equal_all is the value named 'equal all' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 204: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper all bytes equal64; it has no arithmetic value of its own.

Line 205: `equal_all = 1'b1;` This makes equal_all, the value named 'equal all' in this block, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the combinational helper all bytes equal64.

Line 206: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational helper all bytes equal64.

Line 207: `equal_all = equal_all && (values[lane*8 +: 8] == reference);` This makes equal_all, the value named 'equal all' in this block, equal to 'equal_all && (values[lane*8 +: 8] == reference)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are equal_all: the value named 'equal all' in this block; values: the value named 'values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational helper all bytes equal64.

Line 208: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper all bytes equal64; it has no arithmetic value of its own.

Line 209: `all_bytes_equal64 = equal_all;` This makes all_bytes_equal64, the value named 'all bytes equal64' in this block, equal to 'equal_all'. It is driven by combinational or procedural logic in the current evaluation. The important operands are equal_all: the value named 'equal all' in this block. Context: the combinational helper all bytes equal64.

Line 210: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper all bytes equal64; it has no arithmetic value of its own.

Line 211: `endfunction` Within the combinational helper all bytes equal64, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are endfunction: the value named 'endfunction' in this block.

Line 213: `function automatic logic [63:0] repeat_byte(input logic [7:0] value);` Within the combinational helper repeat byte, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; repeat_byte: the value named 'repeat byte' in this block.

Line 214: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper repeat byte; it has no arithmetic value of its own.

Line 215: `repeat_byte = {8{value}};` This makes repeat_byte, the value named 'repeat byte' in this block, equal to '{8{value}}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper repeat byte.

Line 216: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper repeat byte; it has no arithmetic value of its own.

Line 217: `endfunction` Within the combinational helper repeat byte, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are endfunction: the value named 'endfunction' in this block.

Line 219: `// Tile activity is at most 19 bits per command. The low 24-bit addition` Design intent: Tile activity is at most 19 bits per command. The low 24-bit addition This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 220: `// and its carry are registered on the final output beat; the following` Design intent: and its carry are registered on the final output beat; the following This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 221: `// accounting state updates the independent high 40 bits.` Design intent: accounting state updates the independent high 40 bits. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 222: `function automatic logic [24:0] add_u19_to_u24(` Within the combinational helper add u19 to u24, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; add_u19_to_u24: the value named 'add u19 to u24' in this block.

Line 223: `input logic [23:0] value,` This declares value as a input port with width [23:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 224: `input logic [18:0] increment` This declares increment as a input port with width [18:0]. It carries the value named 'increment' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 225: `);` Within the combinational helper add u19 to u24, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The expression is a literal or structural operator and has no hidden software state.

Line 226: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper add u19 to u24; it has no arithmetic value of its own.

Line 227: `add_u19_to_u24 = {1'b0, value} + {6'd0, increment};` This makes add_u19_to_u24, the value named 'add u19 to u24' in this block, equal to '{1'b0, value} + {6'd0, increment}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block; value: the value named 'value' in this block; d0: the value named 'd0' in this block. Context: the combinational helper add u19 to u24.

Line 228: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper add u19 to u24; it has no arithmetic value of its own.

Line 229: `endfunction` Within the combinational helper add u19 to u24, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are endfunction: the value named 'endfunction' in this block.

Line 231: `assign tile_start = (state == ST_COMPUTE_START);` This makes tile_start, the value named 'tile start' in this block, equal to '(state == ST_COMPUTE_START)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_START: the value named 'st compute start' in this block. Context: the module's structural declarations.

Line 232: `assign active_index_write_enable = activation_classify_valid_r` This statement continues through source line 233; the explanation uses the complete expression rather than only the first visual fragment. This makes active_index_write_enable, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘activation_classify_valid_r && activation_classify_nonzero_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are activation_classify_valid_r: the producer's statement that its current payload is meaningful; activation_classify_nonzero_r: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the module's structural declarations.

Line 233: `&& activation_classify_nonzero_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 232. Its exact fragment is ‘&& activation_classify_nonzero_r;‘. The important operands are activation_classify_nonzero_r: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This makes active_index_write_enable, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘activation_classify_valid_r && activation_classify_nonzero_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are activation_classify_valid_r: the producer's statement that its current payload is meaningful; activation_classify_nonzero_r: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the module's structural declarations.

Line 234: `// Each accepted request generates exactly one synchronous BRAM response.` Design intent: Each accepted request generates exactly one synchronous BRAM response. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 235: `// The following local register stage holds that response during downstream` Design intent: The following local register stage holds that response during downstream This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 236: `// stalls, so an unaccepted address can never overwrite accepted metadata.` Design intent: stalls, so an unaccepted address can never overwrite accepted metadata. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 237: `assign active_index_read_enable = index_request_valid_r && !compute_empty_r;` This makes active_index_read_enable, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘index_request_valid_r && !compute_empty_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are index_request_valid_r: the producer's statement that its current payload is meaningful; compute_empty_r: the value named 'compute empty r' in this block. Context: the module's structural declarations.

Line 238: `assign active_mapped_index = compute_empty_r` This statement continues through source line 239; the explanation uses the complete expression rather than only the first visual fragment. This makes active_mapped_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘compute_empty_r ? 11'd0 : active_index_read_data‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are compute_empty_r: the value named 'compute empty r' in this block; d0: the value named 'd0' in this block; active_index_read_data: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 239: `? 11'd0 : active_index_read_data;` This is continuation line 2 of the 2-line logical statement that begins at source line 238. Its exact fragment is ‘? 11'd0 : active_index_read_data;‘. The important operands are d0: the value named 'd0' in this block; active_index_read_data: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes active_mapped_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘compute_empty_r ? 11'd0 : active_index_read_data‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are compute_empty_r: the value named 'compute empty r' in this block; d0: the value named 'd0' in this block; active_index_read_data: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 241: `neuraedge_active_index_buffer #(` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are neuraedge_active_index_buffer: local storage that decouples producer and consumer timing.

Line 242: `.DEPTH(MAX_OUTER_ISSUES),` This connects the child instance's named port 'DEPTH' to parent expression 'MAX_OUTER_ISSUES'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'max outer issues' in this block.

Line 243: `.ADDRESS_BITS(11)` This connects the child instance's named port 'ADDRESS_BITS' to parent expression '11'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named '11' in this block.

Line 244: `) u_active_index_buffer (` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are u_active_index_buffer: local storage that decouples producer and consumer timing.

Line 245: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 246: `.write_enable(active_index_write_enable),` This connects the child instance's named port 'write_enable' to parent expression 'active_index_write_enable'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an integer selecting one tensor element, lane, bank, or state entry.

Line 247: `.write_address(status_active_issues[10:0]),` This connects the child instance's named port 'write_address' to parent expression 'status_active_issues[10:0]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status active issues' in this block.

Line 248: `.write_data(activation_classify_index_r),` This connects the child instance's named port 'write_data' to parent expression 'activation_classify_index_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 249: `.read_enable(active_index_read_enable),` This connects the child instance's named port 'read_enable' to parent expression 'active_index_read_enable'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an integer selecting one tensor element, lane, bank, or state entry.

Line 250: `.read_address(index_request_address_r),` This connects the child instance's named port 'read_address' to parent expression 'index_request_address_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an integer selecting one tensor element, lane, bank, or state entry.

Line 251: `.read_data(active_index_read_data)` This connects the child instance's named port 'read_data' to parent expression 'active_index_read_data'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an integer selecting one tensor element, lane, bank, or state entry.

Line 252: `);` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The expression is a literal or structural operator and has no hidden software state.

Line 254: `neuraedge_activation_buffer u_activation_buffer (` This instantiates module neuraedge_activation_buffer as u_activation_buffer. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 255: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 256: `.write_enable(activation_write_enable),` This connects the child instance's named port 'write_enable' to parent expression 'activation_write_enable'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 257: `.write_address(activation_write_address),` This connects the child instance's named port 'write_address' to parent expression 'activation_write_address'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 258: `.write_data(s_axis_tdata),` This connects the child instance's named port 'write_data' to parent expression 's_axis_tdata'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 's axis tdata' in this block.

Line 259: `.read_address(activation_read_address),` This connects the child instance's named port 'read_address' to parent expression 'activation_read_address'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 260: `.read_data(activation_read_data)` This connects the child instance's named port 'read_data' to parent expression 'activation_read_data'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 261: `);` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The expression is a literal or structural operator and has no hidden software state.

Line 263: `for (genvar bank = 0; bank < WEIGHT_BANKS; bank++) begin : g_weight_bank` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 264: `neuraedge_weight_bank u_weight_bank (` This instantiates module neuraedge_weight_bank as u_weight_bank. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 265: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 266: `.write_enable(weight_commit_enable_r[bank]),` This connects the child instance's named port 'write_enable' to parent expression 'weight_commit_enable_r[bank]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 267: `.write_address(weight_commit_address_r),` This connects the child instance's named port 'write_address' to parent expression 'weight_commit_address_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 268: `.write_segment(weight_commit_segment_r),` This connects the child instance's named port 'write_segment' to parent expression 'weight_commit_segment_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 269: `.write_data(weight_commit_data_r),` This connects the child instance's named port 'write_data' to parent expression 'weight_commit_data_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 270: `.read_address(weight_read_address),` This connects the child instance's named port 'read_address' to parent expression 'weight_read_address'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 271: `.read_data(weight_read_data[bank])` This connects the child instance's named port 'read_data' to parent expression 'weight_read_data[bank]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 272: `);` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The expression is a literal or structural operator and has no hidden software state.

Line 273: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 275: `neuraedge_tensor_tile #(.COLUMN_LANES(COLUMN_LANES)) u_tensor_tile (` This instantiates module neuraedge_tensor_tile as u_tensor_tile. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 276: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 277: `.rst_n(rst_n),` This connects the child instance's named port 'rst_n' to parent expression 'rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 278: `.start(tile_start),` This connects the child instance's named port 'start' to parent expression 'tile_start'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'tile start' in this block.

Line 279: `.compute_mode(compute_mode_r),` This connects the child instance's named port 'compute_mode' to parent expression 'compute_mode_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'compute mode r' in this block.

Line 280: `.active_rows(compute_rows_r),` This connects the child instance's named port 'active_rows' to parent expression 'compute_rows_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one output-row position processed by the tensor tile.

Line 281: `.active_columns(compute_columns_r),` This connects the child instance's named port 'active_columns' to parent expression 'compute_columns_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one output-feature position processed in parallel.

Line 282: `.activation_zero_point(compute_activation_zp_r),` This connects the child instance's named port 'activation_zero_point' to parent expression 'compute_activation_zp_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 283: `.weight_zero_point(compute_weight_zp_r),` This connects the child instance's named port 'weight_zero_point' to parent expression 'compute_weight_zp_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 284: `.config_error(tile_config_error),` This connects the child instance's named port 'config_error' to parent expression 'tile_config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'tile config error' in this block.

Line 285: `.issue_valid(tile_issue_valid),` This connects the child instance's named port 'issue_valid' to parent expression 'tile_issue_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 286: `.issue_ready(tile_issue_ready),` This connects the child instance's named port 'issue_ready' to parent expression 'tile_issue_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 287: `.activation_vector(tile_activation_vector),` This connects the child instance's named port 'activation_vector' to parent expression 'tile_activation_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 288: `.outer_weight_vector(tile_outer_weight_vector),` This connects the child instance's named port 'outer_weight_vector' to parent expression 'tile_outer_weight_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 289: `.dot_weight_vector(tile_dot_weight_vector),` This connects the child instance's named port 'dot_weight_vector to parent expression 'tile_dot_weight_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 290: `.issue_last(tile_issue_last),` This connects the child instance's named port 'issue_last' to parent expression 'tile_issue_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 291: `.busy(),` This connects the child instance's named port 'busy' to parent expression ''. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'busy' in this block.

Line 292: `.result_valid(tile_result_valid),` This connects the child instance's named port 'result_valid' to parent expression 'tile_result_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 293: `.result_ready(tile_result_ready),` This connects the child instance's named port 'result_ready' to parent expression 'tile_result_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 294: `.result_value(tile_result_value),` This connects the child instance's named port 'result_value' to parent expression 'tile_result_value'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'tile result value' in this block.

Line 295: `.result_row(tile_result_row),` This connects the child instance's named port 'result_row' to parent expression 'tile_result_row'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one output-row position processed by the tensor tile.

Line 296: `.result_column(tile_result_column),` This connects the child instance's named port 'result_column' to parent expression 'tile_result_column'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one output-feature position processed in parallel.

Line 297: `.result_last(tile_result_last),` This connects the child instance's named port 'result_last' to parent expression 'tile_result_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 298: `.debug_accepted_issues(),` This connects the child instance's named port 'debug_accepted_issues' to parent expression ''. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug accepted issues' in this block.

Line 299: `.debug_skipped_issues(),` This connects the child instance's named port 'debug_skipped_issues' to parent expression ''. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug skipped issues' in this block.

Line 300: `.debug_retired_issues(),` This connects the child instance's named port 'debug_retired_issues' to parent expression ''. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug retired issues' in this block.

Line 301: `.debug_dense_macs(tile_debug_dense_macs),` This connects the child instance's named port 'debug_dense_macs' to parent expression 'tile_debug_dense_macs'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a count of ordinary activation-times-weight operations actually issued.

Line 302: `.debug_spike_adds(tile_debug_spike_adds)` This connects the child instance's named port 'debug_spike_adds' to parent expression 'tile_debug_spike_adds'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 303: `);` Within the module's structural declarations, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The expression is a literal or structural operator and has no hidden software state.

Line 305: `` `ifndef SYNTHESIS `` This SystemVerilog preprocessor line controls compilation of the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 306: `// The tile's mathematical per-command maximum is 262,144, so its debug` Design intent: The tile's mathematical per-command maximum is 262,144, so its debug This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 307: `// counters must remain 19-bit quantities. Keep the fast accumulator's` Design intent: counters must remain 19-bit quantities. Keep the fast accumulator's This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 308: `// width assumption checked in every RTL simulation.` Design intent: width assumption checked in every RTL simulation. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 309: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 310: `if (rst_n && tile_result_valid && tile_result_ready && tile_result_last` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are rst_n: the active-low synchronous reset; tile_result_valid: the producer's statement that its current payload is meaningful; tile_result_ready: the consumer's statement that it can accept a payload this cycle.

Line 311: `&& ((|tile_debug_dense_macs[63:19])` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tile_debug_dense_macs: a count of ordinary activation-times-weight operations actually issued.

Line 312: `|| (|tile_debug_spike_adds[63:19]))) begin` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tile_debug_spike_adds: a binary event produced when a neuron crosses its threshold.

Line 313: `$fatal(1, "tile activity counter exceeded its 19-bit contract");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 314: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 315: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 316: `` `endif `` This SystemVerilog preprocessor line controls compilation of the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 318: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 319: `command_ready = (state == ST_IDLE);` This makes command_ready, the consumer's statement that it can accept a payload this cycle, equal to '(state == ST_IDLE)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 320: `status_busy = (state != ST_IDLE);` This makes status_busy, the value named 'status busy' in this block, equal to '(state != ST_IDLE)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 321: `s_axis_tready = !load_finish_pending_r` This statement continues through source line 324; the explanation uses the complete expression rather than only the first visual fragment. This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '!load_finish_pending_r && (state == ST_LOAD_ACTIVATIONS || state == ST_LOAD_WEIGHTS_OUTER || state == ST_LOAD_WEIGHTS_DOT8)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_finish_pending_r: the value named 'load finish pending r' in this block; state: the finite-state-machine phase retained in flip-flops; ST_LOAD_ACTIVATIONS:

an input feature value consumed by a neuron or multiply-accumulate operation. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 322: `&& (state == ST_LOAD_ACTIVATIONS` This is continuation line 2 of the 4-line logical statement that begins at source line 321. Its exact fragment is '&& (state == ST_LOAD_ACTIVATIONS'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '!load_finish_pending_r && (state == ST_LOAD_ACTIVATIONS || state == ST_LOAD_WEIGHTS_OUTER || state == ST_LOAD_WEIGHTS_DOT8)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_finish_pending_r: the value named 'load finish pending r' in this block; state: the finite-state-machine phase retained in flip-flops; ST_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 323: `|| state == ST_LOAD_WEIGHTS_OUTER` This is continuation line 3 of the 4-line logical statement that begins at source line 321. Its exact fragment is '|| state == ST_LOAD_WEIGHTS_OUTER'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '!load_finish_pending_r && (state == ST_LOAD_ACTIVATIONS || state == ST_LOAD_WEIGHTS_OUTER || state == ST_LOAD_WEIGHTS_DOT8)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_finish_pending_r: the value named 'load finish pending r' in this block; state: the finite-state-machine phase retained in flip-flops; ST_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 324: `|| state == ST_LOAD_WEIGHTS_DOT8);` This is continuation line 4 of the 4-line logical statement that begins at source line 321. Its exact fragment is '|| state == ST_LOAD_WEIGHTS_DOT8);'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '!load_finish_pending_r && (state == ST_LOAD_ACTIVATIONS || state == ST_LOAD_WEIGHTS_OUTER || state == ST_LOAD_WEIGHTS_DOT8)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_finish_pending_r: the value named 'load finish pending r' in this block; state: the finite-state-machine phase retained in flip-flops; ST_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 326: `m_axis_tvalid = tensor_output_fifo_count_r != 0;` This makes m_axis_tvalid, the producer's statement that its current payload is meaningful, equal to 'tensor_output_fifo_count_r != 0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are tensor_output_fifo_count_r: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 327: `m_axis_tdata = tensor_output_fifo_data_r[` This statement continues through source line 328; the explanation uses the complete expression rather than only the first visual fragment. This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to 'tensor_output_fifo_data_r[tensor_output_fifo_read_pointer_r]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are tensor_output_fifo_data_r: the payload carried with the surrounding valid/ready handshake; tensor_output_fifo_read_pointer_r: the value named 'tensor output fifo read pointer r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 328: `tensor_output_fifo_read_pointer_r];` This is continuation line 2 of the 2-line logical statement that begins at source line 327. Its exact fragment is 'tensor_output_fifo_read_pointer_r];'. The important operands are tensor_output_fifo_read_pointer_r: the value named 'tensor output fifo read pointer r' in this block. Read together, the complete operation is: This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to 'tensor_output_fifo_data_r[tensor_output_fifo_read_pointer_r]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are tensor_output_fifo_data_r: the payload carried with the surrounding

valid/ready handshake; tensor_output_fifo_read_pointer_r: the value named 'tensor output fifo read pointer r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 329: `m_axis_tlast = tensor_output_fifo_head_last;` This makes m_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'tensor_output_fifo_head_last'. It is driven by combinational or procedural logic in the current evaluation. The important operands are tensor_output_fifo_head_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 330: `tile_result_ready = state == ST_COMPUTE_OUTPUT` This statement continues through source line 331; the explanation uses the complete expression rather than only the first visual fragment. This makes tile_result_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_COMPUTE_OUTPUT && tile_result_ready_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_OUTPUT: the value named 'st compute output' in this block; tile_result_ready_r: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 331: `&& tile_result_ready_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 330. Its exact fragment is '&& tile_result_ready_r;'. The important operands are tile_result_ready_r: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This makes tile_result_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_COMPUTE_OUTPUT && tile_result_ready_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_OUTPUT: the value named 'st compute output' in this block; tile_result_ready_r: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 333: `activation_write_enable = state == ST_LOAD_ACTIVATIONS` This statement continues through source line 334; the explanation uses the complete expression rather than only the first visual fragment. This makes activation_write_enable, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'state == ST_LOAD_ACTIVATIONS && s_axis_tvalid && s_axis_tready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; s_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 334: `&& s_axis_tvalid && s_axis_tready;` This is continuation line 2 of the 2-line logical statement that begins at source line 333. Its exact fragment is '&& s_axis_tvalid && s_axis_tready;'. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful; s_axis_tready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This makes activation_write_enable, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'state == ST_LOAD_ACTIVATIONS && s_axis_tvalid && s_axis_tready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; s_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 335: `activation_write_address = load_item_index[10:0];` This makes activation_write_address, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'load_item_index[10:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_item_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 336: `weight_write_enable = '0;` This makes weight_write_enable, a learned coefficient multiplied by an activation, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 337: `weight_write_address = 8'd0;` This makes weight_write_address, a learned coefficient multiplied by an activation, equal to '8'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 338: `weight_write_segment = {1'b0, load_segment_index_r[0]};` This makes weight_write_segment, a learned coefficient multiplied by an activation, equal to '{1'b0, load_segment_index_r[0]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block; load_segment_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 339: `if (state == ST_LOAD_WEIGHTS_OUTER && s_axis_tvalid && s_axis_tready) begin` This condition selects the following hardware branch only when 'state == ST_LOAD_WEIGHTS_OUTER && s_axis_tvalid && s_axis_tready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation; s_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 340: `weight_write_enable[load_item_index[2:0]] = 1'b1;` This makes weight_write_enable[load_item_index[2:0]], a learned coefficient multiplied by an activation, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 341: `weight_write_address = load_item_index[10:3];` This makes weight_write_address, a learned coefficient multiplied by an activation, equal to 'load_item_index[10:3]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_item_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 342: `end else if (state == ST_LOAD_WEIGHTS_DOT8 && s_axis_tvalid && s_axis_tready) begin` This condition selects the following hardware branch only when 'state == ST_LOAD_WEIGHTS_DOT8 && s_axis_tvalid && s_axis_tready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation; s_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 343: `weight_write_enable[load_segment_index_r[3:1]] = 1'b1;` This makes weight_write_enable[load_segment_inc a learned coefficient multiplied by an activation, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 344: `weight_write_address = load_item_index[7:0];` This makes weight_write_address, a learned coefficient multiplied by an activation, equal to 'load_item_index[7:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_item_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 345: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 347: `// A registered two-entry queue terminates tile READY before it can` Design intent: A registered two-entry queue terminates tile READY before it can This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 348: `// propagate through every BRAM-response stage. Deliberately refusing` Design intent: propagate through every BRAM-response stage. Deliberately refusing This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 349: `// a same-cycle push while full costs at most one recovery cycle and` Design intent: a same-cycle push while full costs at most one recovery cycle and This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 350: `// keeps the queue credit independent of downstream combinational logic.` Design intent: keeps the queue credit independent of downstream combinational logic. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 351: `issue_fifo_available = issue_fifo_count_r != 2;` This makes issue_fifo_available, the value named 'issue fifo available' in this block, equal to 'issue_fifo_count_r != 2'. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_fifo_count_r: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 352: `capture_stage_available = !capture_valid_r || issue_fifo_available;` This makes capture_stage_available the value named 'capture stage available' in this block, equal to '!capture_valid_r || issue_fifo_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are capture_valid_r: the producer's statement that its current payload is meaningful; issue_fifo_available: the value named 'issue fifo available' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 353: `memory_stage_available = !memory_valid_r || capture_stage_available;` This makes memory_stage_availab local FPGA storage addressed by surrounding index logic, equal to '!memory_valid_r || capture_stage_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are memory_valid_r: the producer's statement that its current payload is meaningful; capture_stage_available: the value named 'capture stage available' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 354: `read_stage_available = !read_valid_r || memory_stage_available;` This makes read_stage_available, the value named 'read stage available' in this block, equal to '!read_valid_r || memory_stage_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are read_valid_r: the producer's statement that its current payload is meaningful; memory_stage_available: local FPGA storage addressed by surrounding index logic. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 355: `map_stage_available = !map_valid_r || read_stage_available;` This makes map_stage_available, the value named 'map stage available' in this block, equal to '!map_valid_r || read_stage_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are map_valid_r: the producer's statement that its current payload is meaningful; read_stage_available: the value named 'read stage available' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 356: `index_stage_available = !index_valid_r || map_stage_available;` This makes index_stage_available, an integer selecting one tensor element, lane, bank, or state entry, equal to '!index_valid_r || map_stage_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are index_valid_r: the producer's statement that its current payload is meaningful; map_stage_available: the value named 'map stage available' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 357: `index_request_stage_available = !index_request_valid_r` This statement continues through source line 358; the explanation uses the complete expression rather than only the first visual fragment. This makes index_request_stage_available, an integer selecting one tensor element, lane, bank, or state entry, equal to '!index_request_valid_r || index_stage_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are index_request_valid_r: the producer's statement that its current payload is meaningful; index_stage_available:

an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 358: `|| index_stage_available;` This is continuation line 2 of the 2-line logical statement that begins at source line 357. Its exact fragment is '|| index_stage_available;'. The important operands are index_stage_available: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes index_request_stage_available, an integer selecting one tensor element, lane, bank, or state entry, equal to '!index_request_valid_r || index_stage_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are index_request_valid_r: the producer's statement that its current payload is meaningful; index_stage_available: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 359: `compute_read_request = state == ST_COMPUTE_RUN` This statement continues through source line 361; the explanation uses the complete expression rather than only the first visual fragment. This makes compute_read_request, the value named 'compute read request' in this block, equal to 'state == ST_COMPUTE_RUN && schedule_active_r && index_request_stage_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_RUN: the value named 'st compute run' in this block; schedule_active_r: the value named 'schedule active r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 360: `&& schedule_active_r` This is continuation line 2 of the 3-line logical statement that begins at source line 359. Its exact fragment is '&& schedule_active_r'. The important operands are schedule_active_r: the value named 'schedule active r' in this block. Read together, the complete operation is: This makes compute_read_request, the value named 'compute read request' in this block, equal to 'state == ST_COMPUTE_RUN && schedule_active_r && index_request_stage_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_RUN: the value named 'st compute run' in this block; schedule_active_r: the value named 'schedule active r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 361: `&& index_request_stage_available;` This is continuation line 3 of the 3-line logical statement that begins at source line 359. Its exact fragment is '&& index_request_stage_available;'. The important operands are index_request_stage_available: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes compute_read_request, the value named 'compute read request' in this block, equal to 'state == ST_COMPUTE_RUN && schedule_active_r && index_request_stage_available'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_RUN: the value named 'st compute run' in this block; schedule_active_r: the value named 'schedule active r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 362: `activation_read_address = mapped_index_r;` This makes activation_read_address, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'mapped_index_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are mapped_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 363: `weight_read_address = compute_mode_r == MODE_DENSE_DOT8` This statement continues through source line 364; the explanation uses the complete expression rather than only the first visual fragment. This makes weight_read_address, a learned coefficient multiplied by an activation, equal to 'compute_mode_r == MODE_DENSE_DOT8 ? mapped_index_r[7:0] : mapped_index_r[10:3]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are compute_mode_r: the value named 'compute mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; mapped_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 364: `? mapped_index_r[7:0] : mapped_index_r[10:3];` This is continuation line 2 of the 2-line logical statement that begins at source line 363. Its exact fragment is '? mapped_index_r[7:0] : mapped_index_r[10:3];'. The important operands are mapped_index_r: an integer selecting one tensor element, lane, bank, or state entry. Read

together, the complete operation is: This makes weight_read_address, a learned coefficient multiplied by an activation, equal to ‘compute_mode_r == MODE_DENSE_DOT8 ? mapped_index_r[7:0] : mapped_index_r[10:3]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are compute_mode_r: the value named ’compute mode r’ in this block; MODE_DENSE_DOT8: the value named ’mode dense dot8’ in this block; mapped_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 366: `tile_issue_valid = issue_fifo_count_r != 0;` This makes tile_issue_valid, the producer’s statement that its current payload is meaningful, equal to ‘issue_fifo_count_r != 0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_fifo_count_r: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 367: `tile_issue_last = issue_fifo_last_r[issue_fifo_read_pointer_r];` This makes tile_issue_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘issue_fifo_last_r[issue_fifo_read_pointer_r]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_fifo_last_r: the AXI-Stream marker for the final beat of a packet or command payload; issue_fifo_read_pointer_r: the value named ’issue fifo read pointer r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 368: `tile_activation_vector =` This statement continues through source line 369; the explanation uses the complete expression rather than only the first visual fragment. This makes tile_activation_vector, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘issue_fifo_activation_r[issue_fifo_read_pointer_r]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_fifo_activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; issue_fifo_read_pointer_r: the value named ’issue fifo read pointer r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 369: `issue_fifo_activation_r[issue_fifo_read_pointer_r];` This is continuation line 2 of the 2-line logical statement that begins at source line 368. Its exact fragment is ‘issue_fifo_activation_r[issue_fifo_read_pointer_r];‘. The important operands are issue_fifo_activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; issue_fifo_read_pointer_r: the value named ’issue fifo read pointer r’ in this block. Read together, the complete operation is: This makes tile_activation_vector, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘issue_fifo_activation_r[issue_fifo_read_pointer_r]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_fifo_activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; issue_fifo_read_pointer_r: the value named ’issue fifo read pointer r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 370: `tile_outer_weight_vector =` This statement continues through source line 371; the explanation uses the complete expression rather than only the first visual fragment. This makes tile_outer_weight_vector, a learned coefficient multiplied by an activation, equal to ‘issue_fifo_outer_weights_r[issue_fifo_read_pointer_r]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_fifo_outer_weights_r: a learned coefficient multiplied by an activation; issue_fifo_read_pointer_r: the value named ’issue fifo read pointer r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 371: `issue_fifo_outer_weights_r[issue_fifo_read_pointer_r];` This is continuation line 2 of the 2-line logical statement that begins at source line 370. Its exact fragment is ‘issue_fifo_outer_weights_r[issue_fifo_read_pointer_r];‘. The important operands are issue_fifo_outer_weights_r: a learned coefficient multiplied by an activation; issue_fifo_read_pointer_ the value named ’issue fifo read pointer r’ in this block. Read together, the complete operation is: This makes tile_outer_weight_vector, a learned coefficient multiplied by an activation, equal to ‘issue_fifo_outer_weights_r[issue_fifo_read_p It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_fifo_outer_weights_r: a learned coefficient multiplied by an activation; issue_fifo_read_pointer_r: the value named ’issue fifo read pointer r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 372: `tile_dot_weight_vector =` This statement continues through source line 373; the explanation uses the complete expression rather than only the first visual fragment. This makes tile_dot_weight_vector, a learned coefficient

multiplied by an activation, equal to ‘issue_fifo_dot_weights_r[issue_fifo_read_pointer_r]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_fifo_dot_weights_r: a learned coefficient multiplied by an activation; issue_fifo_read_pointer_r: the value named ’issue fifo read pointer r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 373: `issue_fifo_dot_weights_r[issue_fifo_read_pointer_r];` This is continuation line 2 of the 2-line logical statement that begins at source line 372. Its exact fragment is ‘issue_fifo_dot_weights_r[issue_fifo_read_pointer_r];‘. The important operands are issue_fifo_dot_weights_r: a learned coefficient multiplied by an activation; issue_fifo_read_pointer_r the value named ’issue fifo read pointer r’ in this block. Read together, the complete operation is: This makes tile_dot_weight_vector, a learned coefficient multiplied by an activation, equal to ‘issue_fifo_dot_weights_r[issue_fifo_read_poin It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_fifo_dot_weights_r: a learned coefficient multiplied by an activation; issue_fifo_read_pointer_r: the value named ’issue fifo read pointer r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 375: `tensor_output_fifo_count_next = tensor_output_fifo_count_r;` This makes tensor_output_fifo_count_next, the value named ’tensor output fifo count next’ in this block, equal to ‘tensor_output_fifo_count_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are tensor_output_fifo_count_r: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 376: `unique case ({tensor_output_fifo_push, tensor_output_fifo_pop})` This starts a multiplexer/state decode selected by ‘{tensor_output_fifo_push, tensor_output_fifo_pop}‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 377: `2’b10: tensor_output_fifo_count_next` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are b10: the value named ’b10’ in this block; tensor_output_fifo_count_next: the value named ’tensor output fifo count next’ in this block.

Line 378: `= tensor_output_fifo_count_r + 1’b1;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_count_r: an integer count retained for control or measurable telemetry; b1: the value named ’b1’ in this block.

Line 379: `2’b01: tensor_output_fifo_count_next` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are b01: the value named ’b01’ in this block; tensor_output_fifo_count_next: the value named ’tensor output fifo count next’ in this block.

Line 380: `= tensor_output_fifo_count_r - 1’b1;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_count_r: an integer count retained for control or measurable telemetry; b1: the value named ’b1’ in this block.

Line 381: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin end‘; context is the combinational process whose outputs respond without waiting for a clock edge.

Line 382: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 383: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 385: `// The FIFO owns the timing boundary independently of the command FSM. Its` Design intent: The FIFO owns the timing boundary independently of the command FSM. Its This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 386: `// READY register is derived from post-transfer occupancy, which permits a` Design intent: READY register is derived from post-transfer occupancy, which permits a This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 387: `// simultaneous dequeue/enqueue every clock and deasserts early enough to` Design intent: simultaneous dequeue/enqueue every clock and deasserts early enough to This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 388: `// reserve the second slot when the downstream interface stops.` Design intent: reserve the second slot when the downstream interface stops. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 389: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 390: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 391: `tile_result_ready_r <= 1'b0;` This makes tile_result_ready_r, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 392: `tensor_output_fifo_count_r <= 2'd0;` This makes tensor_output_fifo_count_r, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 393: `tensor_output_fifo_write_pointer_r <= 1'b0;` This makes tensor_output_fifo_write_pointer_r, the value named 'tensor output fifo write pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 394: `tensor_output_fifo_read_pointer_r <= 1'b0;` This makes tensor_output_fifo_read_pointer_r, the value named 'tensor output fifo read pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 395: `for (int output_entry = 0; output_entry < 2; output_entry++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 396: `tensor_output_fifo_data_r[output_entry] <= 64'd0;` This makes tensor_output_fifo_data_r[output_entry], the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 397: `tensor_output_fifo_last_r[output_entry] <= 1'b0;` This makes tensor_output_fifo_last_r[output_entry], the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 398: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 399: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 400: `if (state == ST_COMPUTE_START) begin` This condition selects the following hardware branch only when 'state == ST_COMPUTE_START' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_START: the value named 'st compute start' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 401: `tensor_output_fifo_count_r <= 2'd0;` This makes tensor_output_fifo_count_r, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 402: `tensor_output_fifo_write_pointer_r <= 1'b0;` This makes tensor_output_fifo_write_pointer_r, the value named 'tensor output fifo write pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 403: `tensor_output_fifo_read_pointer_r <= 1'b0;` This makes tensor_output_fifo_read_pointer_r, the value named 'tensor output fifo read pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 404: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 405: `tensor_output_fifo_count_r <= tensor_output_fifo_count_next;` This makes tensor_output_fifo_count_r, an integer count retained for control or measurable telemetry, equal to 'tensor_output_fifo_count_next'. It updates after the active clock edge together with other nonblocking assignments. The important operands are tensor_output_fifo_count_next: the value named 'tensor output fifo count next' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 406: `if (tensor_output_fifo_push) begin` This condition selects the following hardware branch only when 'tensor_output_fifo_push' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tensor_output_fifo_push: the value named 'tensor output fifo push' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 407: `tensor_output_fifo_data_r[tensor_output_fifo_write_pointer_r]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_data_r: the payload carried with the surrounding valid/ready handshake; tensor_output_fifo_write_pointer_r: the value named 'tensor output fifo write pointer r' in this block.

Line 408: `<= {5'd0, tile_result_last, tile_result_column,` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are d0: the value named 'd0' in this block; tile_result_last: the AXI-Stream marker for the final beat of a packet or command payload; tile_result_column: one output-feature position processed in parallel.

Line 409: `tile_result_row, tile_result_value};` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tile_result_row: one output-row position processed by the tensor tile; tile_result_value: the value named 'tile result value' in this block.

Line 410: `tensor_output_fifo_last_r[tensor_output_fifo_write_pointer_r]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_last_r: the AXI-Stream marker for the final beat of a packet or command payload; tensor_output_fifo_write_pointer_r: the value named 'tensor output fifo write pointer r' in this block.

Line 411: `<= tile_result_last;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tile_result_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 412: `tensor_output_fifo_write_pointer_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_write_pointer_r: the value named 'tensor output fifo write pointer r' in this block.

Line 413: `<= !tensor_output_fifo_write_pointer_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_write_pointer_r: the value named 'tensor output fifo write pointer r' in this block.

Line 414: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 415: `if (tensor_output_fifo_pop) begin` This condition selects the following hardware branch only when 'tensor_output_fifo_pop' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tensor_output_fifo_pop: the value named 'tensor output fifo pop' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 416: `tensor_output_fifo_read_pointer_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_read_pointer_r: the value named 'tensor output fifo read pointer r' in this block.

Line 417: `<= !tensor_output_fifo_read_pointer_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_read_pointer_r: the value named 'tensor output fifo read pointer r' in this block.

Line 418: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 419: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 421: `tile_result_ready_r <= (state == ST_COMPUTE_OUTPUT` This statement continues through source line 423; the explanation uses the complete expression rather than only the first visual fragment. This makes tile_result_ready_r, the consumer's statement that it can accept a payload this cycle, equal to '(state == ST_COMPUTE_OUTPUT && tensor_output_fifo_count_next != 2) || (state == ST_COMPUTE_RUN && tile_last_issue_accepted_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_OUTPUT: the value named 'st compute output' in this block; tensor_output_fifo_count_next: the value named 'tensor output fifo count next' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 422: `&& tensor_output_fifo_count_next != 2)` This is continuation line 2 of the 3-line logical statement that begins at source line 421. Its exact fragment is '&& tensor_output_fifo_count_next != 2)'. The important operands are tensor_output_fifo_count_next: the value named 'tensor output fifo count next' in this block. Read together, the complete operation is: This makes tile_result_ready_r, the consumer's statement that it can accept a payload this cycle, equal to '(state == ST_COMPUTE_OUTPUT && tensor_output_fifo_count_next != 2) || (state == ST_COMPUTE_RUN && tile_last_issue_accepted_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_OUTPUT: the value named 'st compute output' in this block; tensor_output_fifo_count_next: the value named 'tensor output fifo count next' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 423: `|| (state == ST_COMPUTE_RUN && tile_last_issue_accepted_r);` This is continuation line 3 of the 3-line logical statement that begins at source line 421. Its exact fragment is '|| (state == ST_COMPUTE_RUN && tile_last_issue_accepted_r);'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_RUN: the value named 'st compute run' in this block; tile_last_issue_accepted_r: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This makes tile_result_ready_r, the consumer's statement that it can accept a payload this cycle, equal to '(state == ST_COMPUTE_OUTPUT && tensor_output_fifo_count_next != 2) || (state == ST_COMPUTE_RUN && tile_last_issue_ac
It updates after the active clock edge together with other nonblocking assignments. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_OUTPUT: the value named 'st compute output' in this block; tensor_output_fifo_count_next: the value named 'tensor output fifo count next' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 424: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 425: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 427: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 428: `if (!rst_n) begin` This is the reset branch for ‘(!rst_n‘. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 429: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 430: `captured_command_r <= 42'd0;` This makes captured_command_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘42’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 431: `decoded_opcode_r <= 4'd0;` This makes decoded_opcode_r, the value named ’decoded opcode r’ in this block, equal to ‘4’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 432: `decoded_issue_count_r <= 12'd0;` This makes decoded_issue_count_r, an integer count retained for control or measurable telemetry, equal to ‘12’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 433: `decoded_outer_issue_error_r <= 1'b0;` This makes decoded_outer_issue_error_r, the value named ’decoded outer issue error r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 434: `decoded_dot8_issue_error_r <= 1'b0;` This makes decoded_dot8_issue_error_r, the value named ’decoded dot8 issue error r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 435: `command_done_pending_r <= 1'b0;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 436: `error_event_r <= 1'b0;` This makes error_event_r, the value named ’error event r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 437: `clear_error_r <= 1'b0;` This makes clear_error_r, the value named ’clear error r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 438: `if (state == ST_COMPUTE_RUN && tile_config_error) begin` This condition selects the following hardware branch only when ‘state == ST_COMPUTE_RUN && tile_config_error‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_RUN: the value named ’st compute run’ in this block; tile_config_error: the value named ’tile config error’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 439: `error_event_r <= 1'b1;` This makes error_event_r, the value named 'error event r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 440: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 441: `pending_load_opcode_r <= 4'd0;` This makes pending_load_opcode_r, the value named 'pending load opcode r' in this block, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 442: `pending_load_issue_count_r <= 12'd0;` This makes pending_load_issue_count_r, an integer count retained for control or measurable telemetry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 443: `pending_load_activation_zp_r <= 8'd0;` This makes pending_load_activation_zp_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 444: `command_done <= 1'b0;` This makes command_done, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 445: `status_error <= 1'b0;` This makes status_error, the value named 'status error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 446: `status_loaded_activations <= 12'd0;` This makes status_loaded_activations, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 447: `status_loaded_weights <= 12'd0;` This makes status_loaded_weights, a learned coefficient multiplied by an activation, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 448: `status_active_issues <= 12'd0;` This makes status_active_issues, the value named 'status active issues' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 449: `status_input_beats <= 32'd0;` This makes status_input_beats, the value named 'status input beats' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 450: `status_output_beats <= 32'd0;` This makes status_output_beats, the value named 'status output beats' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 451: `status_compute_cycles <= 32'd0;` This makes status_compute_cycles, the value named 'status compute cycles' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 452: `status_skipped_issues <= 32'd0;` This makes status_skipped_issues, the value named 'status skipped issues' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 453: `status_dense_macs <= 64'd0;` This makes status_dense_macs, a count of ordinary activation-times-weight operations actually issued, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 454: `status_spike_adds <= 64'd0;` This makes status_spike_adds, a binary event produced when a neuron crosses its threshold, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 455: `pending_dense_macs_low_r <= 24'd0;` This makes pending_dense_macs_low_r, a count of ordinary activation-times-weight operations actually issued, equal to '24'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 456: `pending_dense_macs_carry_r <= 1'b0;` This makes pending_dense_macs_carry_r, a count of ordinary activation-times-weight operations actually issued, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 457: `pending_spike_adds_low_r <= 24'd0;` This makes pending_spike_adds_low_r, a binary event produced when a neuron crosses its threshold, equal to '24'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 458: `pending_spike_adds_carry_r <= 1'b0;` This makes pending_spike_adds_carry_r, a binary event produced when a neuron crosses its threshold, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 459: `load_requested_issues <= 12'd0;` This makes load_requested_issues, the value named 'load requested issues' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 460: `load_beats_remaining_r <= 15'd0;` This makes load_beats_remaining_r, the value named 'load beats remaining r' in this block, equal to '15'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 461: `load_finish_pending_r <= 1'b0;` This makes load_finish_pending_r, the value named 'load finish pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 462: `load_segment_index_r <= 4'd0;` This makes load_segment_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 463: `load_item_index <= 12'd0;` This makes load_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 464: `load_activation_zero_point <= 8'd0;` This makes load_activation_zero_point, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 465: `weight_format_dot8 <= 1'b0;` This makes weight_format_dot8, a learned coefficient multiplied by an activation, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 466: `weight_commit_enable_r <= '0;` This makes weight_commit_enable_r, a learned coefficient multiplied by an activation, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 467: `weight_commit_address_r <= 8'd0;` This makes weight_commit_address_r, a learned coefficient multiplied by an activation, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 468: `weight_commit_segment_r <= 2'd0;` This makes weight_commit_segment_r, a learned coefficient multiplied by an activation, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 469: `weight_commit_data_r <= 64'd0;` This makes weight_commit_data_r, a learned coefficient multiplied by an activation, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 470: `activation_classify_valid_r <= 1'b0;` This makes activation_classify_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 471: `activation_classify_nonzero_r <= 1'b0;` This makes activation_classify_nonzero_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 472: `activation_classify_index_r <= 11'd0;` This makes activation_classify_index_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '11'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 473: `activation_load_success_r <= 1'b0;` This makes activation_load_success_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 474: `compute_mode_r <= MODE_DENSE_OUTER;` This makes compute_mode_r, the value named 'compute mode r' in this block, equal to 'MODE_DENSE_OUTER'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_DENSE_OUTER: the value named 'mode dense outer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 475: `compute_rows_r <= 4'd1;` This makes compute_rows_r, one output-row position processed by the tensor tile, equal to '4'd1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 476: `compute_columns_r <= 6'd1;` This makes compute_columns_r, one output-feature position processed in parallel, equal to '6'd1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 477: `compute_requested_issues <= 12'd0;` This makes compute_requested_issues, the value named 'compute requested issues' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 478: `compute_validation_error_r <= 1'b0;` This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 479: `compute_activation_zp_r <= 8'd0;` This makes compute_activation_zp_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 480: `compute_weight_zp_r <= 8'd0;` This makes compute_weight_zp_r, a learned coefficient multiplied by an activation, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 481: `schedule_pointer <= 12'd0;` This makes schedule_pointer, the value named 'schedule pointer' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 482: `schedule_remaining_r <= 12'd0;` This makes schedule_remaining_r, the value named 'schedule remaining r' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 483: `schedule_active_r <= 1'b0;` This makes schedule_active_r, the value named 'schedule active r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 484: `schedule_last_r <= 1'b0;` This makes schedule_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 485: `compute_empty_r <= 1'b0;` This makes compute_empty_r, the value named 'compute empty r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 486: `index_request_valid_r <= 1'b0;` This makes index_request_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 487: `index_request_last_r <= 1'b0;` This makes index_request_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 488: `index_request_address_r <= 11'd0;` This makes index_request_address_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '11'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 489: `index_valid_r <= 1'b0;` This makes index_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 490: `index_last_r <= 1'b0;` This makes index_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 491: `map_valid_r <= 1'b0;` This makes map_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 492: `map_last_r <= 1'b0;` This makes map_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 493: `mapped_index_r <= 11'd0;` This makes mapped_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '11'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 494: `read_valid_r <= 1'b0;` This makes read_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 495: `read_bank_select_r <= 3'd0;` This makes read_bank_select_r, the value named 'read bank select r' in this block, equal to ‘3'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 496: `read_last_r <= 1'b0;` This makes read_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 497: `memory_valid_r <= 1'b0;` This makes memory_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 498: `memory_bank_select_r <= 3'd0;` This makes memory_bank_select_r, local FPGA storage addressed by surrounding index logic, equal to ‘3'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 499: `memory_last_r <= 1'b0;` This makes memory_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 500: `capture_valid_r <= 1'b0;` This makes capture_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 501: `capture_last_r <= 1'b0;` This makes capture_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 502: `capture_activation_r <= 64'd0;` This makes capture_activation_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘64'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 503: `capture_outer_weights_r <= '0;` This makes capture_outer_weights_r, a learned coefficient multiplied by an activation, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 504: `capture_dot_weights_r <= '0;` This makes capture_dot_weights_r, a learned coefficient multiplied by an activation, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 505: `issue_fifo_count_r <= 2'd0;` This makes issue_fifo_count_r, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 506: `issue_fifo_write_pointer_r <= 1'b0;` This makes issue_fifo_write_pointer_r, the value named 'issue fifo write pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 507: `issue_fifo_read_pointer_r <= 1'b0;` This makes issue_fifo_read_pointer_r, the value named 'issue fifo read pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 508: `tile_last_issue_accepted_r <= 1'b0;` This makes tile_last_issue_accepted_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 509: `for (int fifo_entry = 0; fifo_entry < 2; fifo_entry++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 510: `issue_fifo_last_r[fifo_entry] <= 1'b0;` This makes issue_fifo_last_r[fifo_entry], the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 511: `issue_fifo_activation_r[fifo_entry] <= 64'd0;` This makes issue_fifo_activation_r[fifo_entry], an input feature value consumed by a neuron or multiply-accumulate operation, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 512: `issue_fifo_outer_weights_r[fifo_entry] <= '0;` This makes issue_fifo_outer_weights_r[fifo_entry], a learned coefficient multiplied by an activation, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 513: `issue_fifo_dot_weights_r[fifo_entry] <= '0;` This makes issue_fifo_dot_weights_r[fifo_entry], a learned coefficient multiplied by an activation, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 514: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 515: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 516: `// Register the accepted write request before it reaches sixteen` Design intent: Register the accepted write request before it reaches sixteen This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 517: `// BRAM enable pins. This remains a one-beat-per-cycle pipeline:` Design intent: BRAM enable pins. This remains a one-beat-per-cycle pipeline: This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 518: `// the banks commit the prior request while this stage captures` Design intent: the banks commit the prior request while this stage captures This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 519: `// the next one.` Design intent: the next one. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 520: `weight_commit_enable_r <= weight_write_enable;` This makes weight_commit_enable_r, a learned coefficient multiplied by an activation, equal to ‘weight_write_enable‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_write_enable: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 521: `if (|weight_write_enable) begin` This condition selects the following hardware branch only when ‘|weight_write_enable‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are weight_write_enable: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 522: `weight_commit_address_r <= weight_write_address;` This makes weight_commit_address_r, a learned coefficient multiplied by an activation, equal to ‘weight_write_address‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_write_address: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 523: `weight_commit_segment_r <= weight_write_segment;` This makes weight_commit_segment_r, a learned coefficient multiplied by an activation, equal to ‘weight_write_segment‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_write_segment: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 524: `weight_commit_data_r <= s_axis_tdata;` This makes weight_commit_data_r, a learned coefficient multiplied by an activation, equal to ‘s_axis_tdata‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 525: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 526: `command_done <= command_done_pending_r;` This makes command_done, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘command_done_pending_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_done_pending_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge.

Line 527: `command_done_pending_r <= 1'b0;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 528: `if (clear_error_r) status_error <= 1'b0;` This condition selects the following hardware branch only when ‘clear_error_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_error_r: the value named 'clear error r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 529: `else if (error_event_r) status_error <= 1'b1;` This condition selects the following hardware branch only when ‘error_event_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are error_event_r: the value named 'error event r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 530: `error_event_r <= 1'b0;` This makes error_event_r, the value named 'error event r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 531: `clear_error_r <= 1'b0;` This makes clear_error_r, the value named 'clear error r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 532: `activation_classify_valid_r <= 1'b0;` This makes activation_classify_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 533: `tile_last_issue_accepted_r <= tile_issue_valid` This statement continues through source line 534; the explanation uses the complete expression rather than only the first visual fragment. This makes tile_last_issue_accepted_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘tile_issue_valid && tile_issue_ready && tile_issue_last‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are tile_issue_valid: the producer's statement that its current payload is meaningful; tile_issue_ready: the consumer's statement that it can accept a payload this cycle; tile_issue_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 534: `&& tile_issue_ready && tile_issue_last;` This is continuation line 2 of the 2-line logical statement that begins at source line 533. Its exact fragment is ‘&& tile_issue_ready && tile_issue_last;‘. The important operands are tile_issue_ready: the consumer's statement that it can accept a payload this cycle; tile_issue_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This makes tile_last_issue_accepted_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘tile_issue_valid && tile_issue_ready && tile_issue_last‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are tile_issue_valid: the producer's statement that its current payload is meaningful; tile_issue_ready: the consumer's statement that it can accept a payload this cycle; tile_issue_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 536: `if (issue_fifo_push) begin` This condition selects the following hardware branch only when ‘issue_fifo_push‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are issue_fifo_push: the value named 'issue fifo push' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 537: `issue_fifo_last_r[issue_fifo_write_pointer_r] <= capture_last_r;` This makes issue_fifo_last_r[issue_fifo_ the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘capture_last_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are capture_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 538: `issue_fifo_activation_r[issue_fifo_write_pointer_r]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are issue_fifo_activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; issue_fifo_write_pointer_r: the value named 'issue fifo write pointer r' in this block.

Line 539: `<= capture_activation_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are capture_activation_r: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 540: `issue_fifo_outer_weights_r[issue_fifo_write_pointer_r]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are issue_fifo_outer_weights_r: a learned coefficient multiplied by an activation; issue_fifo_write_pointer_r: the value named 'issue fifo write pointer r' in this block.

Line 541: `<= capture_outer_weights_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are capture_outer_weights_r: a learned coefficient multiplied by an activation.

Line 542: `issue_fifo_dot_weights_r[issue_fifo_write_pointer_r]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are issue_fifo_dot_weights_r: a learned coefficient multiplied by an activation; issue_fifo_write_pointer_r: the value named 'issue fifo write pointer r' in this block.

Line 543: `<= capture_dot_weights_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are capture_dot_weights_r: a learned coefficient multiplied by an activation.

Line 544: `issue_fifo_write_pointer_r <= !issue_fifo_write_pointer_r;` This makes issue_fifo_write_pointer_r, the value named 'issue fifo write pointer r' in this block, equal to '!issue_fifo_write_pointer_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_fifo_write_pointer_r: the value named 'issue fifo write pointer r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 545: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 546: `if (issue_fifo_pop) begin` This condition selects the following hardware branch only when 'issue_fifo_pop' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are issue_fifo_pop: the value named 'issue fifo pop' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 547: `issue_fifo_read_pointer_r <= !issue_fifo_read_pointer_r;` This makes issue_fifo_read_pointer_r, the value named 'issue fifo read pointer r' in this block, equal to '!issue_fifo_read_pointer_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_fifo_read_pointer_r: the value named 'issue fifo read pointer r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 548: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 549: `unique case ({issue_fifo_push, issue_fifo_pop})` This starts a multiplexer/state decode selected by ‘{issue_fifo_push, issue_fifo_pop}‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 550: `2’b10: issue_fifo_count_r <= issue_fifo_count_r + 1’b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are b10: the value named ’b10’ in this block; issue_fifo_count_r: an integer count retained for control or measurable telemetry; b1: the value named ’b1’ in this block.

Line 551: `2’b01: issue_fifo_count_r <= issue_fifo_count_r - 1’b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are b01: the value named ’b01’ in this block; issue_fifo_count_r: an integer count retained for control or measurable telemetry; b1: the value named ’b1’ in this block.

Line 552: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin end‘; context is the clocked process that updates registers only on a rising clock edge.

Line 553: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 555: `if (activation_classify_valid_r && activation_classify_nonzero_r) begin` This condition selects the following hardware branch only when ‘activation_classify_valid_r && activation_classify_nonzero_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are activation_classify_valid_r: the producer’s statement that its current payload is meaningful; activation_classify_nonzero_r: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge.

Line 556: `status_active_issues <= status_active_issues + 1’b1;` This makes status_active_issues, the value named ’status active issues’ in this block, equal to ‘status_active_issues + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_active_issues: the value named ’status active issues’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 557: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 559: `if (state == ST_COMPUTE_START || state == ST_COMPUTE_RUN` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_START: the value named ’st compute start’ in this block; ST_COMPUTE_RUN: the value named ’st compute run’ in this block.

Line 560: `|| state == ST_COMPUTE_OUTPUT` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_OUTPUT: the value named ’st compute output’ in this block.

Line 561: `|| state == ST_COMPUTE_DRAIN) begin` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMPUTE_DRAIN: the value named 'st compute drain' in this block.

Line 562: `status_compute_cycles <= status_compute_cycles + 1'b1;` This makes status_compute_cycles, the value named 'status compute cycles' in this block, equal to 'status_compute_cycles + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_compute_cycles: the value named 'status compute cycles' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 563: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 564: `if (tensor_output_fifo_pop) begin` This condition selects the following hardware branch only when 'tensor_output_fifo_pop' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tensor_output_fifo_pop: the value named 'tensor output fifo pop' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 565: `status_output_beats <= status_output_beats + 1'b1;` This makes status_output_beats, the value named 'status output beats' in this block, equal to 'status_output_beats + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_output_beats: the value named 'status output beats' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 566: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 568: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 569: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 570: `map_valid_r <= 1'b0;` This makes map_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 571: `read_valid_r <= 1'b0;` This makes read_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 572: `memory_valid_r <= 1'b0;` This makes memory_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 573: `issue_fifo_count_r <= 2'd0;` This makes issue_fifo_count_r, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 574: `issue_fifo_write_pointer_r <= 1'b0;` This makes issue_fifo_write_pointer_r, the value named 'issue fifo write pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 575: `issue_fifo_read_pointer_r <= 1'b0;` This makes issue_fifo_read_pointer_r, the value named 'issue fifo read pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 576: `if (command_valid) begin` This condition selects the following hardware branch only when 'command_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are command_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 577: `captured_command_r <= command_data[41:0];` This makes captured_command_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'command_data[41:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 578: `state <= ST_PREDECODE_COMMAND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_PREDECODE_COMMAND'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PREDECODE_COMMAND: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 579: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 580: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 582: `ST_PREDECODE_COMMAND: begin` This is the ST_PREDECODE_COMMAND case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_PREDECODE_COMMAND, the machine performs the named control phase..

Line 583: `decoded_opcode_r <= captured_command_r[3:0];` This makes decoded_opcode_r, the value named 'decoded opcode r' in this block, equal to 'captured_command_r[3:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREDECODE_COMMAND, the machine performs the named control phase..

Line 584: `decoded_issue_count_r <= captured_command_r[25:14];` This makes decoded_issue_count_r, an integer count retained for control or measurable telemetry, equal to ‘captured_command_r[25:14]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREDECODE_COMMAND, the machine performs the named control phase..

Line 585: `decoded_outer_issue_error_r <= captured_command_r[25:14] == 0` This statement continues through source line 586; the explanation uses the complete expression rather than only the first visual fragment. This makes decoded_outer_issue_error_r, the value named ’decoded outer issue error r’ in this block, equal to ‘captured_command_r[25:14] == 0 || captured_command_r[25:14] > 12’(MAX_OUTER_ISSUES)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; MAX_OUTER_ISSUES: the value named ’max outer issues’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREDECODE_COMMAND, the machine performs the named control phase..

Line 586: `|| captured_command_r[25:14] > 12’(MAX_OUTER_ISSUES);` This is continuation line 2 of the 2-line logical statement that begins at source line 585. Its exact fragment is ‘|| captured_command_r[25:14] > 12’(MAX_OUTER_ISSUES);‘. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; MAX_OUTER_ISSUES: the value named ’max outer issues’ in this block. Read together, the complete operation is: This makes decoded_outer_issue_error_r, the value named ’decoded outer issue error r’ in this block, equal to ‘captured_command_r[25:14] == 0 || captured_command_r[25:14] > 12’(MAX_OUTER_ISSUES)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; MAX_OUTER_ISSUES: the value named ’max outer issues’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREDECODE_COMMAND, the machine performs the named control phase..

Line 587: `decoded_dot8_issue_error_r <= captured_command_r[25:14] == 0` This statement continues through source line 588; the explanation uses the complete expression rather than only the first visual fragment. This makes decoded_dot8_issue_error_r, the value named ’decoded dot8 issue error r’ in this block, equal to ‘captured_command_r[25:14] == 0 || captured_command_r[25:14] > 12’(MAX_DOT8_ISSUES)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; MAX_DOT8_ISSUES: the value named ’max dot8 issues’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREDECODE_COMMAND, the machine performs the named control phase..

Line 588: `|| captured_command_r[25:14] > 12’(MAX_DOT8_ISSUES);` This is continuation line 2 of the 2-line logical statement that begins at source line 587. Its exact fragment is ‘|| captured_command_r[25:14] > 12’(MAX_DOT8_ISSUES);‘. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; MAX_DOT8_ISSUES: the value named ’max dot8 issues’ in this block. Read together, the complete operation is: This makes decoded_dot8_issue_error_r, the value named ’decoded dot8 issue error r’ in this block, equal to ‘captured_command_r[25:14] == 0 || captured_command_r[25:14] > 12’(MAX_DOT8_ISSUES)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; MAX_DOT8_ISSUES: the value named ’max dot8 issues’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREDECODE_COMMAND, the machine performs the named control phase..

Line 589: `state <= ST_DECODE_COMMAND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_DECODE_COMMAND‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECODE_COMMAND: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREDECODE_COMMAND, the machine performs the named control phase..

Line 590: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PREDECODE_COMMAND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 592: `ST_DECODE_COMMAND: begin` This is the ST_DECODE_COMMAND case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECODE_COMMAND, the machine performs the named control phase..

Line 593: `// Default back to idle for immediate commands and rejected` Design intent: Default back to idle for immediate commands and rejected This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 594: `// requests. Valid streaming/compute commands override this.` Design intent: requests. Valid streaming/compute commands override this. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 595: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECODE_COMMAND, the machine performs the named control phase..

Line 596: `begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECODE_COMMAND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 597: `logic [3:0] opcode;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_DECODE_COMMAND, the machine performs the named control phase.. Specifically, opcode is the value named ’opcode’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 598: `logic [11:0] issue_count;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_DECODE_COMMAND, the machine performs the named control phase.. Specifically, issue_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 599: `opcode = decoded_opcode_r;` This makes opcode, the value named ’opcode’ in this block, equal to ‘decoded_opcode_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are decoded_opcode_r: the value named ’decoded opcode r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECODE_COMMAND, the machine performs the named control phase..

Line 600: `issue_count = decoded_issue_count_r;` This makes issue_count, an integer count retained for control or measurable telemetry, equal to ‘decoded_issue_count_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are decoded_issue_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECODE_COMMAND, the machine performs the named control phase..

Line 602: `unique case (opcode)` This starts a multiplexer/state decode selected by ‘opcode‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 603: `CMD_LOAD_ACTIVATIONS: begin` This is the CMD_LOAD_ACTIVATIONS case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 604: `if (decoded_outer_issue_error_r) begin` This condition selects the following hardware branch only when ‘decoded_outer_issue_error_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are decoded_outer_issue_error_r: the value named ’decoded outer issue error r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 605: `error_event_r <= 1'b1;` This makes error_event_r, the value named ’error event r’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 606: `status_loaded_activations <= 12'd0;` This makes status_loaded_activations, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘12’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 607: `status_active_issues <= 12'd0;` This makes status_active_issues, the value named ’status active issues’ in this block, equal to ‘12’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 608: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 609: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 610: `pending_load_opcode_r <= opcode;` This makes pending_load_opcode_r, the value named ’pending load opcode r’ in this block, equal to ‘opcode‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are opcode: the value named ’opcode’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 611: `pending_load_issue_count_r <= issue_count;` This makes pending_load_issue_count_r, an integer count retained for control or measurable telemetry, equal to ‘issue_count‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 612: `pending_load_activation_zp_r` Within the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are pending_load_activation_zp_r: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 613: `<= captured_command_r[33:26];` Within the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions,

schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 614: `state <= ST_LOAD_CONFIG;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_CONFIG‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_CONFIG: the value named ’st load config’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 615: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 616: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 618: `CMD_LOAD_WEIGHTS_OUTER: begin` This is the CMD_LOAD_WEIGHTS_OUTER case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 619: `if (decoded_outer_issue_error_r) begin` This condition selects the following hardware branch only when ‘decoded_outer_issue_error_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are decoded_outer_issue_error_r: the value named ’decoded outer issue error r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 620: `error_event_r <= 1’b1;` This makes error_event_r, the value named ’error event r’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 621: `status_loaded_weights <= 12’d0;` This makes status_loaded_weights, a learned coefficient multiplied by an activation, equal to ‘12’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 622: `command_done_pending_r <= 1’b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 623: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 624: `pending_load_opcode_r <= opcode;` This makes pending_load_opcode_r, the value named ’pending load opcode r’ in this block, equal to ‘opcode‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are opcode: the value named ’opcode’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 625: `pending_load_issue_count_r <= issue_count;` This makes pending_load_issue_count_r, an integer count retained for control or measurable telemetry, equal to ‘issue_count‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 626: `state <= ST_LOAD_CONFIG;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_CONFIG‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_CONFIG: the value named ’st load config’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 627: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 628: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 630: `CMD_LOAD_WEIGHTS_DOT8: begin` This is the CMD_LOAD_WEIGHTS_DOT8 case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 631: `if (decoded_dot8_issue_error_r) begin` This condition selects the following hardware branch only when ‘decoded_dot8_issue_error_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are decoded_dot8_issue_error_r: the value named ’decoded dot8 issue error r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 632: `error_event_r <= 1’b1;` This makes error_event_r, the value named ’error event r’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 633: `status_loaded_weights <= 12’d0;` This makes status_loaded_weights, a learned coefficient multiplied by an activation, equal to ‘12’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 634: `command_done_pending_r <= 1’b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 635: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 636: `pending_load_opcode_r <= opcode;` This makes pending_load_opcode_r, the value named 'pending load opcode r' in this block, equal to 'opcode'. It updates after the active clock edge together with other nonblocking assignments. The important operands are opcode: the value named 'opcode' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 637: `pending_load_issue_count_r <= issue_count;` This makes pending_load_issue_count_r, an integer count retained for control or measurable telemetry, equal to 'issue_count'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 638: `state <= ST_LOAD_CONFIG;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LOAD_CONFIG'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_CONFIG: the value named 'st load config' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 639: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 640: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 642: `CMD_COMPUTE_DENSE_OUTER,` Within the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are CMD_COMPUTE_DENSE_OUTER: the value named 'cmd compute dense outer' in this block.

Line 643: `CMD_COMPUTE_DENSE_DOT8,` Within the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are CMD_COMPUTE_DENSE_DOT8: the value named 'cmd compute dense dot8' in this block.

Line 644: `CMD_COMPUTE_BINARY,` Within the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are CMD_COMPUTE_BINARY: the value named 'cmd compute binary' in this block.

Line 645: `CMD_COMPUTE_TERNARY: begin` This is the CMD_COMPUTE_TERNARY case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_TERNARY, the machine issues arithmetic work using values already held on chip..

Line 646: `unique case (opcode)` This starts a multiplexer/state decode selected by 'opcode'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 647: `CMD_COMPUTE_DENSE_OUTER:` This is the CMD_COMPUTE_DENSE_OUTER case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with "; context is the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_DENSE_OUTER, the machine issues arithmetic work using values already held on chip..

Line 648: `compute_mode_r <= MODE_DENSE_OUTER;` This makes compute_mode_r, the value named 'compute mode r' in this block, equal to 'MODE_DENSE_OUTER'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_DENSE_OUTER: the value named 'mode dense outer' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_DENSE_OU the machine issues arithmetic work using values already held on chip..

Line 649: `CMD_COMPUTE_DENSE_DOT8:` This is the CMD_COMPUTE_DENSE_DOT8 case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with "; context is the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_DENSE_DOT8, the machine issues arithmetic work using values already held on chip..

Line 650: `compute_mode_r <= MODE_DENSE_DOT8;` This makes compute_mode_r, the value named 'compute mode r' in this block, equal to 'MODE_DENSE_DOT8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_DENSE_DOT8, the machine issues arithmetic work using values already held on chip..

Line 651: `CMD_COMPUTE_BINARY:` This is the CMD_COMPUTE_BINARY case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with "; context is the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_BINARY, the machine issues arithmetic work using values already held on chip..

Line 652: `compute_mode_r <= MODE_BINARY_WEIGHTED;` This makes compute_mode_r, the value named 'compute mode r' in this block, equal to 'MODE_BINARY_WEIGHTED'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_BINARY_WEIGHTED: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_BINARY, the machine issues arithmetic work using values already held on chip..

Line 653: `default: compute_mode_r <= MODE_TERNARY_WEIGHTED;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'compute_mode_r <= MODE_TERNARY_WEIGHTED;'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_BINARY, the machine issues arithmetic work using values already held on chip..

Line 654: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_BINARY, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 655: `compute_rows_r <= captured_command_r[7:4];` This makes compute_rows_r, one output-row position processed by the tensor tile, equal to 'captured_command_r[7:4]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge.

Line 656: `compute_columns_r <= captured_command_r[13:8];` This makes compute_columns_r, one output-feature position processed in parallel, equal to 'captured_command_r[13:8]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge.

Line 657: `compute_requested_issues <= issue_count;` This makes compute_requested_issues, the value named 'compute requested issues' in this block, equal to 'issue_count'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge.

Line 658: `compute_activation_zp_r <= captured_command_r[33:26];` This makes compute_activation_zp_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'captured_command_r[33:26]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge.

Line 659: `compute_weight_zp_r <= captured_command_r[41:34];` This makes compute_weight_zp_r, a learned coefficient multiplied by an activation, equal to 'captured_command_r[41:34]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge.

Line 660: `state <= ST_COMPUTE_VALIDATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_COMPUTE_VALIDATE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMPUTE_VALIDATE: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 661: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 663: `CMD_CLEAR_STATUS: begin` This is the CMD_CLEAR_STATUS case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 664: `clear_error_r <= 1'b1;` This makes clear_error_r, the value named 'clear error r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 665: `status_input_beats <= 32'd0;` This makes status_input_beats, the value named 'status input beats' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 666: `status_output_beats <= 32'd0;` This makes status_output_beats, the value named 'status output beats' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 667: `status_compute_cycles <= 32'd0;` This makes status_compute_cycles, the value named 'status compute cycles' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 668: `status_skipped_issues <= 32'd0;` This makes status_skipped_issues, the value named 'status skipped issues' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 669: `status_dense_macs <= 64'd0;` This makes status_dense_macs, a count of ordinary activation-times-weight operations actually issued, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 670: `status_spike_adds <= 64'd0;` This makes status_spike_adds, a binary event produced when a neuron crosses its threshold, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 671: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 672: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 674: `default: begin` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 675: `error_event_r <= 1'b1;` This makes error_event_r, the value named 'error event r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 676: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 677: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 678: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 679: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 680: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 682: `ST_LOAD_CONFIG: begin` This is the ST_LOAD_CONFIG case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_CONFIG, the machine stores incoming operands into local FPGA memory..

Line 683: `load_requested_issues <= pending_load_issue_count_r;` This makes load_requested_issues, the value named 'load requested issues' in this block, equal to 'pending_load_issue_count_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are pending_load_issue_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_CONFIG, the machine stores incoming operands into local FPGA memory..

Line 684: `load_segment_index_r <= 4'd0;` This makes load_segment_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_CONFIG, the machine stores incoming operands into local FPGA memory..

Line 685: `load_item_index <= 12'd0;` This makes load_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_CONFIG, the machine stores incoming operands into local FPGA memory..

Line 686: `load_finish_pending_r <= 1'b0;` This makes load_finish_pending_r, the value named 'load finish pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_CONFIG, the machine stores incoming operands into local FPGA memory..

Line 687: `unique case (pending_load_opcode_r)` This starts a multiplexer/state decode selected by 'pending_load_opcode_r 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 688: `CMD_LOAD_ACTIVATIONS: begin` This is the CMD_LOAD_ACTIVATIONS case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 689: `load_beats_remaining_r <=` This statement continues through source line 690; the explanation uses the complete expression rather than only the first visual fragment. This makes load_beats_remaining_r, the value named 'load beats remaining r' in this block, equal to '{3'd0, pending_load_issue_count_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; pending_load_issue_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 690: `{3'd0, pending_load_issue_count_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 689. Its exact fragment is '{3'd0, pending_load_issue_count_r};'. The important operands are d0: the value named 'd0' in this block; pending_load_issue_count_r: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This makes load_beats_remaining_r, the value named 'load beats remaining r' in this block, equal to '{3'd0, pending_load_issue_count_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; pending_load_issue_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 691: `load_activation_zero_point <= pending_load_activation_zp_r;` This makes load_activation_zero_point, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'pending_load_activation_zp_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are pending_load_activation_zp_r: an input feature value consumed by a neuron or multiply-accumulate operation. Context:

the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 692: `activation_load_success_r <= 1'b0;` This makes activation_load_success_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 693: `status_active_issues <= 12'd0;` This makes status_active_issues, the value named 'status active issues' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 694: `state <= ST_LOAD_ACTIVATIONS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LOAD_ACTIVATIONS'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 695: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 696: `CMD_LOAD_WEIGHTS_OUTER: begin` This is the CMD_LOAD_WEIGHTS_OUTER case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 697: `load_beats_remaining_r <=` This statement continues through source line 698; the explanation uses the complete expression rather than only the first visual fragment. This makes load_beats_remaining_r, the value named 'load beats remaining r' in this block, equal to '{2'd0, pending_load_issue_count_r, 1'b0}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; pending_load_issue_count_r: an integer count retained for control or measurable telemetry; b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 698: `{2'd0, pending_load_issue_count_r, 1'b0};` This is continuation line 2 of the 2-line logical statement that begins at source line 697. Its exact fragment is '{2'd0, pending_load_issue_count_r, 1'b0};'. The important operands are d0: the value named 'd0' in this block; pending_load_issue_count_r: an integer count retained for control or measurable telemetry; b0: the value named 'b0' in this block. Read together, the complete operation is: This makes load_beats_remaining_r, the value named 'load beats remaining r' in this block, equal to '{2'd0, pending_load_issue_count_r, 1'b0}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; pending_load_issue_count_r: an integer count retained for control or measurable telemetry; b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 699: `weight_format_dot8 <= 1'b0;` This makes weight_format_dot8, a learned coefficient multiplied by an activation, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 700: `state <= ST_LOAD_WEIGHTS_OUTER;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_WEIGHTS_OUTER‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 701: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 702: `default: begin` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 703: `load_beats_remaining_r <=` This statement continues through source line 704; the explanation uses the complete expression rather than only the first visual fragment. This makes load_beats_remaining_r, the value named ’load beats remaining r’ in this block, equal to ‘{2’d0, pending_load_issue_count_r[8:0], 4’d0}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; pending_load_issue_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 704: `{2’d0, pending_load_issue_count_r[8:0], 4’d0};` This is continuation line 2 of the 2-line logical statement that begins at source line 703. Its exact fragment is ‘{2’d0, pending_load_issue_count_r[8:0], 4’d0};‘. The important operands are d0: the value named ’d0’ in this block; pending_load_issue_count_r: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This makes load_beats_remaining_r, the value named ’load beats remaining r’ in this block, equal to ‘{2’d0, pending_load_issue_count_r[8:0], 4’d0}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; pending_load_issue_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 705: `weight_format_dot8 <= 1’b1;` This makes weight_format_dot8, a learned coefficient multiplied by an activation, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 706: `state <= ST_LOAD_WEIGHTS_DOT8;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_WEIGHTS_DOT8‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 707: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 708: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 709: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 711: `ST_COMPUTE_CONFIG: begin` This is the ST_COMPUTE_CONFIG case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 712: `compute_empty_r <= (status_active_issues == 0);` This makes compute_empty_r, the value named 'compute empty r' in this block, equal to ‘(status_active_issues == 0)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_active_issues: the value named 'status active issues' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 713: `schedule_remaining_r <= status_active_issues == 0` This statement continues through source line 714; the explanation uses the complete expression rather than only the first visual fragment. This makes schedule_remaining_r, the value named 'schedule remaining r' in this block, equal to ‘status_active_issues == 0 ? 12'd1 : status_active_issues‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_active_issues: the value named 'status active issues' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 714: `? 12'd1 : status_active_issues;` This is continuation line 2 of the 2-line logical statement that begins at source line 713. Its exact fragment is ‘? 12'd1 : status_active_issues;‘. The important operands are d1: the value named 'd1' in this block; status_active_issues: the value named 'status active issues' in this block. Read together, the complete operation is: This makes schedule_remaining_r, the value named 'schedule remaining r' in this block, equal to ‘status_active_issues == 0 ? 12'd1 : status_active_issues‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_active_issues: the value named 'status active issues' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 715: `schedule_active_r <= 1'b1;` This makes schedule_active_r, the value named 'schedule active r' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 716: `schedule_last_r <= status_active_issues <= 1;` This makes schedule_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘status_active_issues <= 1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_active_issues: the value named 'status active issues' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 717: `schedule_pointer <= 12'd0;` This makes schedule_pointer, the value named 'schedule pointer' in this block, equal to ‘12'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 718: `status_compute_cycles <= 32'd0;` This makes status_compute_cycles, the value named 'status compute cycles' in this block, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 719: `status_output_beats <= 32'd0;` This makes status_output_beats, the value named 'status output beats' in this block, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The

important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 720: `state <= ST_COMPUTE_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_COMPUTE_START'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMPUTE_START: the value named 'st compute start' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip..

Line 721: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_CONFIG, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 723: `ST_COMPUTE_VALIDATE: begin` This is the ST_COMPUTE_VALIDATE case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 724: `// End the multi-field legality predicate at one local` Design intent: End the multi-field legality predicate at one local This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 725: `// register instead of placing it on every parameter mux.` Design intent: register instead of placing it on every parameter mux. This comment anchors the surrounding logic in the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

Line 726: `compute_validation_error_r <= compute_requested_issues == 0` This statement continues through source line 736; the explanation uses the complete expression rather than only the first visual fragment. This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_
!= weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 727: `|| compute_rows_r == 0` This is continuation line 2 of the 11-line logical statement that begins at source line 726. Its exact fragment is '|| compute_rows_r == 0'. The important operands are compute_rows_r: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_DOT8
!= weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 728: `|| compute_rows_r > 8` This is continuation line 3 of the 11-line logical statement that begins at source line 726. Its exact fragment is '|| compute_rows_r > 8'. The important operands are compute_rows_r: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_DOT8 != weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 729: `|| compute_columns_r == 0` This is continuation line 4 of the 11-line logical statement that begins at source line 726. Its exact fragment is '|| compute_columns_r == 0'. The important operands are compute_columns_r: one output-feature position processed in parallel. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_DOT8 != weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 730: `|| compute_columns_r > 6'(COLUMN_LANES)` This is continuation line 5 of the 11-line logical statement that begins at source line 726. Its exact fragment is '|| compute_columns_r > 6'(COLUMN_LANES)'. The important operands are compute_columns_r: one output-feature position processed in parallel; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_DOT8) != weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 731: `|| (compute_mode_r == MODE_DENSE_DOT8` This is continuation line 6 of the 11-line logical statement that begins at source line 726. Its exact fragment is '|| (compute_mode_r == MODE_DENSE_DOT8'. The important operands are compute_mode_r: the value named 'compute mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_DOT8 != weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 732: `&& compute_rows_r != 1)` This is continuation line 7 of the 11-line logical statement that begins at source line 726. Its exact fragment is '&& compute_rows_r != 1)'. The important operands are compute_rows_r:

one output-row position processed by the tensor tile. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_ != weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 733: `|| compute_requested_issues != status_loaded_activations` This is continuation line 8 of the 11-line logical statement that begins at source line 726. Its exact fragment is '|| compute_requested_issues != status_loaded_activations'. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; status_loaded_activations: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_DOT8) != weight_format_dot8' It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 734: `|| compute_requested_issues != status_loaded_weights` This is continuation line 9 of the 11-line logical statement that begins at source line 726. Its exact fragment is '|| compute_requested_issues != status_loaded_weights'. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; status_loaded_weights: a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_DOT8) != weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 735: `|| (compute_mode_r == MODE_DENSE_DOT8)` This is continuation line 10 of the 11-line logical statement that begins at source line 726. Its exact fragment is '|| (compute_mode_r == MODE_DENSE_DOT8)'. The important operands are compute_mode_r: the value named 'compute mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_DOT8 != weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 736: `!= weight_format_dot8;` This is continuation line 11 of the 11-line logical statement that begins at source line 726. Its exact fragment is '!= weight_format_dot8;'. The important operands are weight_format_dot8: a learned co-

efficient multiplied by an activation. Read together, the complete operation is: This makes compute_validation_error_r, the producer's statement that its current payload is meaningful, equal to 'compute_requested_issues == 0 || compute_rows_r == 0 || compute_rows_r > 8 || compute_columns_r == 0 || compute_columns_r > 6'(COLUMN_LANES) || (compute_mode_r == MODE_DENSE_DOT8 && compute_rows_r != 1) || compute_requested_issues != status_loaded_activations || compute_requested_issues != status_loaded_weights || (compute_mode_r == MODE_DENSE_DOT8 != weight_format_dot8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_requested_issues: the value named 'compute requested issues' in this block; compute_rows_r: one output-row position processed by the tensor tile; compute_columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 737: `state <= ST_COMPUTE_VALIDATED;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_COMPUTE_VALIDATED'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMPUTE_VALIDATED: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip..

Line 738: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATE, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 740: `ST_COMPUTE_VALIDATED: begin` This is the ST_COMPUTE_VALIDATED case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATED, the machine issues arithmetic work using values already held on chip..

Line 741: `if (compute_validation_error_r) begin` This condition selects the following hardware branch only when 'compute_validation_error_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are compute_validation_error_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATED, the machine issues arithmetic work using values already held on chip..

Line 742: `error_event_r <= 1'b1;` This makes error_event_r, the value named 'error event r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATED, the machine issues arithmetic work using values already held on chip..

Line 743: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATED, the machine issues arithmetic work using values already held on chip..

Line 744: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATED, the machine issues arithmetic work using values already held on chip..

Line 745: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATED, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 746: `state <= ST_COMPUTE_CONFIG;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_COMPUTE_CONFIG‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMPUTE_CONFIG: the value named ’st compute config’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATED, the machine issues arithmetic work using values already held on chip..

Line 747: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATED, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 748: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_VALIDATED, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 750: `ST_LOAD_ACTIVATIONS: begin` This is the ST_LOAD_ACTIVATIONS case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 751: `if (load_finish_pending_r) begin` This condition selects the following hardware branch only when ‘load_finish_pending_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_finish_pending_r: the value named ’load finish pending r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 752: `load_finish_pending_r <= 1’b0;` This makes load_finish_pending_r, the value named ’load finish pending r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 753: `state <= ST_LOAD_ACTIVATION_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_ACTIVATION_DRAIN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_ACTIVATION_DRAIN: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 754: `end else if (s_axis_tvalid && s_axis_tready) begin` This condition selects the following hardware branch only when ‘s_axis_tvalid && s_axis_tready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tvalid: the producer’s statement that its current payload is meaningful; s_axis_tready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 755: `logic beat_is_last;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory.. Specifically, beat_is_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 756: `logic byte_vector_is_zero;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory.. Specifically, byte_vector_is_zero is the value named ’byte vector is zero’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 757: `beat_is_last = load_beats_remaining_r == 15'd1;` This makes beat_is_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘load_beats_remaining_r == 15'd1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_beats_remaining_r: the value named 'load beats remaining r' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 758: `byte_vector_is_zero = all_bytes_equal64(` This statement continues through source line 759; the explanation uses the complete expression rather than only the first visual fragment. This makes byte_vector_is_zero, the value named 'byte vector is zero' in this block, equal to ‘all_bytes_equal64(s_axis_tdata, load_activation_zero_point)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are all_bytes_equal64: the value named 'all bytes equal64' in this block; s_axis_tdata: the value named 's axis tdata' in this block; load_activation_zero_po an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 759: `s_axis_tdata, load_activation_zero_point);` This is continuation line 2 of the 2-line logical statement that begins at source line 758. Its exact fragment is ‘s_axis_tdata, load_activation_zero_point);‘. The important operands are s_axis_tdata: the value named 's axis tdata' in this block; load_activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This makes byte_vector_is_zero, the value named 'byte vector is zero' in this block, equal to ‘all_bytes_equal64(s_axis_tdata, load_activation_zero_point)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are all_bytes_equal64: the value named 'all bytes equal64' in this block; s_axis_tdata: the value named 's axis tdata' in this block; load_activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 760: `activation_classify_valid_r <= 1'b1;` This makes activation_classify_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 761: `activation_classify_nonzero_r <= !byte_vector_is_zero;` This makes activation_classify_nonzero_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘!byte_vector_is_zero‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are byte_vector_is_zero: the value named 'byte vector is zero' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 762: `activation_classify_index_r <= load_item_index[10:0];` This makes activation_classify_index_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘load_item_index[10:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_item_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 763: `status_input_beats <= status_input_beats + 1'b1;` This makes status_input_beats, the value named 'status input beats' in this block, equal to ‘status_input_beats + 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_input_beats: the value named 'status input beats' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 764: `load_beats_remaining_r <= load_beats_remaining_r - 1'b1;` This makes load_beats_remaining_r, the value named 'load beats remaining r' in this block, equal to ‘load_beats_remaining_r - 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_beats_remaining_r:

the value named 'load beats remaining r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 765: `load_item_index <= load_item_index + 1'b1;` This makes load_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'load_item_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_item_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 766: `status_loaded_activations <= beat_is_last` This statement continues through source line 767; the explanation uses the complete expression rather than only the first visual fragment. This makes status_loaded_activations, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'beat_is_last ? load_requested_issues : 12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload; load_requested_issues: the value named 'load requested issues' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 767: `? load_requested_issues : 12'd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 766. Its exact fragment is '? load_requested_issues : 12'd0;'. The important operands are load_requested_issues: the value named 'load requested issues' in this block; d0: the value named 'd0' in this block. Read together, the complete operation is: This makes status_loaded_activations, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'beat_is_last ? load_requested_issues : 12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload; load_requested_issues: the value named 'load requested issues' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 769: `if (s_axis_tlast != beat_is_last) error_event_r <= 1'b1;` This condition selects the following hardware branch only when 's_axis_tlast != beat_is_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 770: `if (beat_is_last || s_axis_tlast) begin` This condition selects the following hardware branch only when 'beat_is_last || s_axis_tlast' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload; s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 771: `activation_load_success_r <= beat_is_last;` This makes activation_load_success_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'beat_is_last'. It updates after the active clock edge together with other nonblocking assignments. The important operands are beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 772: `load_finish_pending_r <= 1'b1;` This makes load_finish_pending_r, the value named 'load finish pending r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory..

Line 773: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 774: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 775: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATIONS, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 777: `ST_LOAD_ACTIVATION_DRAIN: begin` This is the ST_LOAD_ACTIVATION_DRAIN case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATION_DRAIN, the machine stores incoming operands into local FPGA memory..

Line 778: `if (!activation_classify_valid_r) begin` This condition selects the following hardware branch only when ‘!activation_classify_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are activation_classify_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATION_DRAIN, the machine stores incoming operands into local FPGA memory..

Line 779: `if (!activation_load_success_r) status_active_issues <= 12'd0;` This condition selects the following hardware branch only when ‘!activation_load_success_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are activation_load_success_r: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATION_DRAIN, the machine stores incoming operands into local FPGA memory..

Line 780: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATION_DRAIN, the machine stores incoming operands into local FPGA memory..

Line 781: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATION_DRAIN, the machine stores incoming operands into local FPGA memory..

Line 782: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATION_DRAIN, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 783: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_ACTIVATION_DRAIN, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 785: `ST_LOAD_WEIGHTS_OUTER: begin` This is the ST_LOAD_WEIGHTS_OUTER case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 786: `if (load_finish_pending_r) begin` This condition selects the following hardware branch only when ‘load_finish_pending_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_finish_pending_r: the value named ’load finish pending r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 787: `load_finish_pending_r <= 1'b0;` This makes load_finish_pending_r, the value named ’load finish pending r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 788: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 789: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 790: `end else if (s_axis_tvalid && s_axis_tready) begin` This condition selects the following hardware branch only when ‘s_axis_tvalid && s_axis_tready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tvalid: the producer’s statement that its current payload is meaningful; s_axis_tready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 791: `logic [1:0] word_segment;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.. Specifically, word_segment is the value named ’word segment’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 792: `logic beat_is_last;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.. Specifically, beat_is_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 793: `word_segment = {1'b0, load_segment_index_r[0]};` This makes word_segment, the value named ’word segment’ in this block, equal to ‘{1’b0, load_segment_index_r[0]}‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block; load_segment_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 794: `beat_is_last = load_beats_remaining_r == 15'd1;` This makes beat_is_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘load_beats_remaining_r == 15’d1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_beats_remaining_r: the value named ’load beats remaining r’ in this block; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 795: `status_input_beats <= status_input_beats + 1'b1;` This makes status_input_beats, the value named 'status input beats' in this block, equal to 'status_input_beats + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_input_beats: the value named 'status input beats' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 796: `load_beats_remaining_r <= load_beats_remaining_r - 1'b1;` This makes load_beats_remaining_r, the value named 'load beats remaining r' in this block, equal to 'load_beats_remaining_r - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_beats_remaining_r: the value named 'load beats remaining r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 797: `status_loaded_weights <= beat_is_last` This statement continues through source line 798; the explanation uses the complete expression rather than only the first visual fragment. This makes status_loaded_weights, a learned coefficient multiplied by an activation, equal to 'beat_is_last ? load_requested_issues : 12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload; load_requested_issues: the value named 'load requested issues' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 798: `? load_requested_issues : 12'd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 797. Its exact fragment is '? load_requested_issues : 12'd0;'. The important operands are load_requested_issues: the value named 'load requested issues' in this block; d0: the value named 'd0' in this block. Read together, the complete operation is: This makes status_loaded_weights, a learned coefficient multiplied by an activation, equal to 'beat_is_last ? load_requested_issues : 12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload; load_requested_issues: the value named 'load requested issues' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 799: `if (word_segment == 2'd1) begin` This condition selects the following hardware branch only when 'word_segment == 2'd1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are word_segment: the value named 'word segment' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 800: `load_item_index <= load_item_index + 1'b1;` This makes load_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'load_item_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_item_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 801: `load_segment_index_r <= 4'd0;` This makes load_segment_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 802: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 803: `load_segment_index_r <= 4'd1;` This makes load_segment_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '4'd1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 804: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 806: `if (s_axis_tlast != beat_is_last) error_event_r <= 1'b1;` This condition selects the following hardware branch only when 's_axis_tlast != beat_is_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 807: `if (beat_is_last || s_axis_tlast) begin` This condition selects the following hardware branch only when 'beat_is_last || s_axis_tlast' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload; s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 808: `load_finish_pending_r <= 1'b1;` This makes load_finish_pending_r, the value named 'load finish pending r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory..

Line 809: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 810: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 811: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_OUTER, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 813: `ST_LOAD_WEIGHTS_DOT8: begin` This is the ST_LOAD_WEIGHTS_DOT8 case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 814: `if (load_finish_pending_r) begin` This condition selects the following hardware branch only when 'load_finish_pending_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_finish_pending_r: the value named 'load finish pending r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 815: `load_finish_pending_r <= 1'b0;` This makes load_finish_pending_r, the value named 'load finish pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 816: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 817: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 818: `end else if (s_axis_tvalid && s_axis_tready) begin` This condition selects the following hardware branch only when 's_axis_tvalid && s_axis_tready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful; s_axis_tready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 819: `logic beat_is_last;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory.. Specifically, beat_is_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 820: `beat_is_last = load_beats_remaining_r == 15'd1;` This makes beat_is_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'load_beats_remaining_r == 15'd1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_beats_remaining_r: the value named 'load beats remaining r' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 821: `status_input_beats <= status_input_beats + 1'b1;` This makes status_input_beats, the value named 'status input beats' in this block, equal to 'status_input_beats + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_input_beats: the value named 'status input beats' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 822: `load_beats_remaining_r <= load_beats_remaining_r - 1'b1;` This makes load_beats_remaining_r, the value named 'load beats remaining r' in this block, equal to 'load_beats_remaining_r - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_beats_remaining_r: the value named 'load beats remaining r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 823: `status_loaded_weights <= beat_is_last` This statement continues through source line 824; the explanation uses the complete expression rather than only the first visual fragment. This makes status_loaded_weights, a learned coefficient multiplied by an activation, equal to 'beat_is_last ? load_requested_issues : 12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are beat_is_last:

the AXI-Stream marker for the final beat of a packet or command payload; load_requested_issues: the value named 'load requested issues' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 824: `? load_requested_issues : 12'd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 823. Its exact fragment is '? load_requested_issues : 12'd0;'. The important operands are load_requested_issues: the value named 'load requested issues' in this block; d0: the value named 'd0' in this block. Read together, the complete operation is: This makes status_loaded_weights, a learned coefficient multiplied by an activation, equal to 'beat_is_last ? load_requested_issues : 12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload; load_requested_issues: the value named 'load requested issues' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 825: `if (load_segment_index_r == 4'd15) begin` This condition selects the following hardware branch only when 'load_segment_index_r == 4'd15' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_segment_index_r: an integer selecting one tensor element, lane, bank, or state entry; d15: the value named 'd15' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 826: `load_item_index <= load_item_index + 1'b1;` This makes load_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'load_item_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_item_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 827: `load_segment_index_r <= 4'd0;` This makes load_segment_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 828: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 829: `load_segment_index_r <= load_segment_index_r + 1'b1;` This makes load_segment_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to 'load_segment_index_r + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_segment_index_r: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 830: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 832: `if (s_axis_tlast != beat_is_last) error_event_r <= 1'b1;` This condition selects the following hardware branch only when 's_axis_tlast != beat_is_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 833: `if (beat_is_last || s_axis_tlast) begin` This condition selects the following hardware branch only when ‘beat_is_last || s_axis_tlast‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are beat_is_last: the AXI-Stream marker for the final beat of a packet or command payload; s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 834: `load_finish_pending_r <= 1'b1;` This makes load_finish_pending_r, the value named 'load finish pending r' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory..

Line 835: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 836: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 837: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_WEIGHTS_DOT8, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 839: `ST_COMPUTE_START: begin` This is the ST_COMPUTE_START case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 840: `schedule_pointer <= 12'd0;` This makes schedule_pointer, the value named 'schedule pointer' in this block, equal to ‘12'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 841: `index_request_valid_r <= 1'b0;` This makes index_request_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 842: `index_valid_r <= 1'b0;` This makes index_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 843: `map_valid_r <= 1'b0;` This makes map_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 844: `read_valid_r <= 1'b0;` This makes read_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 845: `memory_valid_r <= 1'b0;` This makes memory_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 846: `capture_valid_r <= 1'b0;` This makes capture_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 847: `issue_fifo_count_r <= 2'd0;` This makes issue_fifo_count_r, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 848: `issue_fifo_write_pointer_r <= 1'b0;` This makes issue_fifo_write_pointer_r, the value named 'issue fifo write pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 849: `issue_fifo_read_pointer_r <= 1'b0;` This makes issue_fifo_read_pointer_r, the value named 'issue fifo read pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 850: `state <= ST_COMPUTE_RUN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_COMPUTE_RUN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMPUTE_RUN: the value named 'st compute run' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip..

Line 851: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_START, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 853: `ST_COMPUTE_RUN: begin` This is the ST_COMPUTE_RUN case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 854: `if (capture_stage_available) begin` This condition selects the following hardware branch only when 'capture_stage_available' is true. Both branches exist as gates; the condition controls a multiplexer rather than

making software jump. The important operands are capture_stage_available: the value named 'capture stage available' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 855: `capture_valid_r <= memory_valid_r;` This makes capture_valid_r, the producer's statement that its current payload is meaningful, equal to 'memory_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are memory_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 856: `if (memory_valid_r) begin` This condition selects the following hardware branch only when 'memory_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are memory_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 857: `capture_last_r <= memory_last_r;` This makes capture_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'memory_last_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are memory_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 858: `if (compute_empty_r) begin` This condition selects the following hardware branch only when 'compute_empty_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are compute_empty_r: the value named 'compute empty r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 859: `capture_activation_r <= (compute_mode_r == MODE_DENSE_OUTER` This statement continues through source line 861; the explanation uses the complete expression rather than only the first visual fragment. This makes capture_activation_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '(compute_mode_r == MODE_DENSE_OUTER || compute_mode_r == MODE_DENSE_DOT8) ? repeat_byte(compute_activatio : 64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_mode_r: the value named 'compute mode r' in this block; MODE_DENSE_OUTER: the value named 'mode dense outer' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 860: `|| compute_mode_r == MODE_DENSE_DOT8)` This is continuation line 2 of the 3-line logical statement that begins at source line 859. Its exact fragment is '|| compute_mode_r == MODE_DENSE_DOT8)'. The important operands are compute_mode_r: the value named 'compute mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Read together, the complete operation is: This makes capture_activation_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '(compute_mode_r == MODE_DENSE_OUTER || compute_mode_r == MODE_DENSE_DOT8) ? repeat_byte(compute_activation_zp_r) : 64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_mode_r: the value named 'compute mode r' in this block; MODE_DENSE_OUTER: the value named 'mode dense outer' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 861: `? repeat_byte(compute_activation_zp_r) : 64'd0;` This is continuation line 3 of the 3-line logical statement that begins at source line 859. Its exact fragment is '? repeat_byte(compute_activation_zp_r) : 64'd0;'. The important operands are repeat_byte: the value named 'repeat byte' in this block; compute_activation_zp_r: an input feature value consumed by a neuron or multiply-accumulate operation; d0: the value named 'd0' in this block. Read together, the complete operation is: This makes capture_activation_r, an input feature value consumed

by a neuron or multiply-accumulate operation, equal to ‘(compute_mode_r == MODE_DENSE_OUTER || compute_mode_r == MODE_DENSE_DOT8) ? repeat_byte(compute_activation_zp_r) : 64’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_mode_r: the value named ’compute mode r’ in this block; MODE_DENSE_OUTER: the value named ’mode dense outer’ in this block; MODE_DENSE_DOT8: the value named ’mode dense dot8’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 862: `capture_outer_weights_r <= ’0;` This makes capture_outer_weights_r, a learned coefficient multiplied by an activation, equal to ‘’0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 863: `capture_dot_weights_r <= ’0;` This makes capture_dot_weights_r, a learned coefficient multiplied by an activation, equal to ‘’0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 864: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 865: `capture_activation_r <= activation_read_data;` This makes capture_activation_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘activation_read_data‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are activation_read_data: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 866: `capture_outer_weights_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are capture_outer_weights_r: a learned coefficient multiplied by an activation.

Line 867: `<= weight_read_data[memory_bank_select_r];` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are weight_read_data: a learned coefficient multiplied by an activation; memory_bank_select_r: local FPGA storage addressed by surrounding index logic.

Line 868: `capture_dot_weights_r <= {` This statement continues through source line 873; the explanation uses the complete expression rather than only the first visual fragment. This makes capture_dot_weights_r, a learned coefficient multiplied by an activation, equal to ‘{ weight_read_data[7], weight_read_data[6], weight_read_data[5], weight_read_data[4], weight_read_data[3], weight_read_data[2], weight_read_data[1], weight_read_data[0] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 869: `weight_read_data[7], weight_read_data[6],` This is continuation line 2 of the 6-line logical statement that begins at source line 868. Its exact fragment is ‘weight_read_data[7], weight_read_data[6],‘. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Read together, the complete operation

is: This makes capture_dot_weights_r, a learned coefficient multiplied by an activation, equal to ‘{ weight_read_data[7], weight_read_data[6], weight_read_data[5], weight_read_data[4], weight_read_data[3], weight_read_data[2], weight_read_data weight_read_data[0] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 870: `weight_read_data[5], weight_read_data[4],` This is continuation line 3 of the 6-line logical statement that begins at source line 868. Its exact fragment is ‘weight_read_data[5], weight_read_data[4],‘. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes capture_dot_weights_r, a learned coefficient multiplied by an activation, equal to ‘{ weight_read_data[7], weight_read_data[6], weight_read_data[5], weight_read_data[4], weight_read_data[3], weight_read_data[2], weight_read_data weight_read_data[0] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 871: `weight_read_data[3], weight_read_data[2],` This is continuation line 4 of the 6-line logical statement that begins at source line 868. Its exact fragment is ‘weight_read_data[3], weight_read_data[2],‘. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes capture_dot_weights_r, a learned coefficient multiplied by an activation, equal to ‘{ weight_read_data[7], weight_read_data[6], weight_read_data[5], weight_read_data[4], weight_read_data[3], weight_read_data[2], weight_read_data weight_read_data[0] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 872: `weight_read_data[1], weight_read_data[0]` This is continuation line 5 of the 6-line logical statement that begins at source line 868. Its exact fragment is ‘weight_read_data[1], weight_read_data[0]‘. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes capture_dot_weights_r, a learned coefficient multiplied by an activation, equal to ‘{ weight_read_data[7], weight_read_data[6], weight_read_data[5], weight_read_data[4], weight_read_data[3], weight_read_data[2], weight_read_data weight_read_data[0] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 873: `};` This is continuation line 6 of the 6-line logical statement that begins at source line 868. Its exact fragment is ‘};‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This makes capture_dot_weights_r, a learned coefficient multiplied by an activation, equal to ‘{ weight_read_data[7], weight_read_data[6], weight_read_data[5], weight_read_data[4], weight_read_data[3], weight_read_data[2], weight_read_data[1], weight_read_data[0] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_read_data: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 874: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 875: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 876: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 878: `if (memory_stage_available) begin` This condition selects the following hardware branch only when ‘memory_stage_available‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are memory_stage_available: local FPGA storage addressed by surrounding index logic. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 879: `memory_valid_r <= read_valid_r;` This makes memory_valid_r, the producer's statement that its current payload is meaningful, equal to ‘read_valid_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are read_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 880: `if (read_valid_r) begin` This condition selects the following hardware branch only when ‘read_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 881: `memory_bank_select_r <= read_bank_select_r;` This makes memory_bank_select_r, local FPGA storage addressed by surrounding index logic, equal to ‘read_bank_select_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are read_bank_select_r: the value named 'read bank select r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 882: `memory_last_r <= read_last_r;` This makes memory_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘read_last_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are read_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 883: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 884: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 886: `if (read_stage_available) begin` This condition selects the following hardware branch only when ‘read_stage_available‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_stage_available: the value named 'read stage available' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 887: `read_valid_r <= map_valid_r;` This makes read_valid_r, the producer's statement that its current payload is meaningful, equal to ‘map_valid_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are map_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 888: `if (map_valid_r) begin` This condition selects the following hardware branch only when ‘map_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are map_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 889: `read_bank_select_r <= mapped_index_r[2:0];` This makes read_bank_select_r, the value named ’read bank select r’ in this block, equal to ‘mapped_index_r[2:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are mapped_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 890: `read_last_r <= map_last_r;` This makes read_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘map_last_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are map_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 891: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 892: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 894: `if (map_stage_available) begin` This condition selects the following hardware branch only when ‘map_stage_availa is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are map_stage_available: the value named ’map stage available’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 895: `map_valid_r <= index_valid_r;` This makes map_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘index_valid_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are index_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 896: `if (index_valid_r) begin` This condition selects the following hardware branch only when ‘index_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are index_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 897: `mapped_index_r <= active_mapped_index;` This makes mapped_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘active_mapped_index‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_mapped_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 898: `map_last_r <= index_last_r;` This makes map_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘index_last_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are index_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 899: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 900: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 902: `if (index_stage_available) begin` This condition selects the following hardware branch only when 'index_stage_available' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are index_stage_available: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 903: `index_valid_r <= index_request_valid_r;` This makes index_valid_r, the producer's statement that its current payload is meaningful, equal to 'index_request_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are index_request_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 904: `if (index_request_valid_r) begin` This condition selects the following hardware branch only when 'index_request_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are index_request_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 905: `index_last_r <= index_request_last_r;` This makes index_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'index_request_last_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are index_request_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 906: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 907: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 909: `if (index_request_stage_available) begin` This condition selects the following hardware branch only when 'index_request_stage_available' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are index_request_stage_available: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 910: `index_request_valid_r <= compute_read_request;` This makes index_request_valid_r, the producer's statement that its current payload is meaningful, equal to 'compute_read_request'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_read_request: the value named 'compute read request' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 911: `if (compute_read_request) begin` This condition selects the following hardware branch only when ‘compute_read_request‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are compute_read_request: the value named ’compute read request’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 912: `index_request_address_r <= schedule_pointer[10:0];` This makes index_request_address_r, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘schedule_pointer[10:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are schedule_pointer: the value named ’schedule pointer’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 913: `index_request_last_r <= schedule_last_r;` This makes index_request_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘schedule_last_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are schedule_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 914: `schedule_last_r <= schedule_remaining_r == 12'd2;` This makes schedule_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘schedule_remaining_r == 12’d2‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are schedule_remaining_r: the value named ’schedule remaining r’ in this block; d2: the value named ’d2’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 915: `schedule_remaining_r <= schedule_remaining_r - 1'b1;` This makes schedule_remaining_r, the value named ’schedule remaining r’ in this block, equal to ‘schedule_remaining_r - 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are schedule_remaining_r: the value named ’schedule remaining r’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 916: `if (schedule_remaining_r == 12'd1) begin` This condition selects the following hardware branch only when ‘schedule_remaining_r == 12’d1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are schedule_remaining_r: the value named ’schedule remaining r’ in this block; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 917: `schedule_active_r <= 1'b0;` This makes schedule_active_r, the value named ’schedule active r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 918: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 919: `schedule_pointer <= schedule_pointer + 1'b1;` This makes schedule_pointer, the value named ’schedule pointer’ in this block, equal to ‘schedule_pointer + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are schedule_pointer: the value named ’schedule pointer’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 920: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 921: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 923: `if (tile_last_issue_accepted_r) begin` This condition selects the following hardware branch only when ‘tile_last_issue_accepted_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tile_last_issue_accepted_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 924: `state <= ST_COMPUTE_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_COMPUTE_OUTPUT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMPUTE_OUTPUT: the value named ’st compute output’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip..

Line 925: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 926: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_RUN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 928: `ST_COMPUTE_OUTPUT: begin` This is the ST_COMPUTE_OUTPUT case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip..

Line 929: `status_skipped_issues <= {20'd0, compute_requested_issues}` This statement continues through source line 930; the explanation uses the complete expression rather than only the first visual fragment. This makes status_skipped_issues, the value named ’status skipped issues’ in this block, equal to ‘{20’d0, compute_requested_issues} - {20’d0, status_active_issues}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; compute_requested_issues: the value named ’compute requested issues’ in this block; status_active_issues: the value named ’status active issues’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip..

Line 930: `- {20'd0, status_active_issues};` This is continuation line 2 of the 2-line logical statement that begins at source line 929. Its exact fragment is ‘- {20’d0, status_active_issues};‘. The important operands are d0: the value named ’d0’ in this block; status_active_issues: the value named ’status active issues’ in this block. Read together, the complete operation is: This makes status_skipped_issues, the value named ’status skipped issues’ in this block, equal to ‘{20’d0, compute_requested_issues} - {20’d0, status_active_issues}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; compute_requested_issues: the value named ’compute requested issues’ in this block; status_active_issues: the value named ’status active issues’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip..

Line 931: `if (tensor_output_fifo_push) begin` This condition selects the following hardware branch only when ‘tensor_output_fifo_push‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tensor_output_fifo_push: the value named ’tensor output fifo

push' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip..

Line 932: `if (tile_result_last) begin` This condition selects the following hardware branch only when 'tile_result_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tile_result_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip..

Line 933: `{pending_dense_macs_carry_r,` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are pending_dense_macs_carry_r: a count of ordinary activation-times-weight operations actually issued.

Line 934: `pending_dense_macs_low_r}` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are pending_dense_macs_low_r: a count of ordinary activation-times-weight operations actually issued.

Line 935: `<= add_u19_to_u24(` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are add_u19_to_u24: the value named 'add u19 to u24' in this block.

Line 936: `status_dense_macs[23:0],` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are status_dense_macs: a count of ordinary activation-times-weight operations actually issued.

Line 937: `tile_debug_dense_macs[18:0]);` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tile_debug_dense_macs: a count of ordinary activation-times-weight operations actually issued.

Line 938: `{pending_spike_adds_carry_r,` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are pending_spike_adds_carry_r: a binary event produced when a neuron crosses its threshold.

Line 939: `pending_spike_adds_low_r}` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are pending_spike_adds_low_r: a binary event produced when a neuron crosses its threshold.

Line 940: `<= add_u19_to_u24(` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules

the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are add_u19_to_u24: the value named 'add u19 to u24' in this block.

Line 941: `status_spike_adds[23:0],` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are status_spike_adds: a binary event produced when a neuron crosses its threshold.

Line 942: `tile_debug_spike_adds[18:0]);` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tile_debug_spike_adds: a binary event produced when a neuron crosses its threshold.

Line 943: `state <= ST_COMPUTE_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_COMPUTE_DRAIN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMPUTE_DRAIN: the value named 'st compute drain' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip..

Line 944: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 945: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 946: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_OUTPUT, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 948: `ST_COMPUTE_DRAIN: begin` This is the ST_COMPUTE_DRAIN case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_DRAIN, the machine issues arithmetic work using values already held on chip..

Line 949: `if (tensor_output_fifo_pop` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_DRAIN, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_pop: the value named 'tensor output fifo pop' in this block.

Line 950: `&& tensor_output_fifo_head_last) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_DRAIN, the machine issues arithmetic work using values already held on chip., this exact statement contributes to the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO. The important operands are tensor_output_fifo_head_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 951: `state <= ST_COMPUTE_ACCOUNT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_COMPUTE_ACCOUNT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMPUTE_ACCOUNT: the value named 'st compute account' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_DRAIN, the machine issues arithmetic work using values already held on chip..

Line 952: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_DRAIN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 953: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_DRAIN, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 955: `ST_COMPUTE_ACCOUNT: begin` This is the ST_COMPUTE_ACCOUNT case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 956: `status_dense_macs <= {` This statement continues through source line 960; the explanation uses the complete expression rather than only the first visual fragment. This makes status_dense_macs, a count of ordinary activation-times-weight operations actually issued, equal to ‘{ status_dense_macs[63:24] + {{39{1’b0}}, pending_dense_macs_carry_r}, pending_dense_macs_low_r }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_dense_macs: a count of ordinary activation-times-weight operations actually issued; b0: the value named ’b0’ in this block; pending_dense_macs_carry_r: a count of ordinary activation-times-weight operations actually issued. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 957: `status_dense_macs[63:24]` This is continuation line 2 of the 5-line logical statement that begins at source line 956. Its exact fragment is ‘status_dense_macs[63:24]‘. The important operands are status_dense_macs: a count of ordinary activation-times-weight operations actually issued. Read together, the complete operation is: This makes status_dense_macs, a count of ordinary activation-times-weight operations actually issued, equal to ‘{ status_dense_macs[63:24] + {{39{1’b0}}, pending_dense_macs_carry_r}, pending_dense_macs_low_r }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_dense_macs: a count of ordinary activation-times-weight operations actually issued; b0: the value named ’b0’ in this block; pending_dense_macs_carry_r: a count of ordinary activation-times-weight operations actually issued. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 958: `+ {{39{1’b0}}, pending_dense_macs_carry_r},` This is continuation line 3 of the 5-line logical statement that begins at source line 956. Its exact fragment is ‘+ {{39{1’b0}}, pending_dense_macs_carry_r},‘. The important operands are b0: the value named ’b0’ in this block; pending_dense_macs_carry_r: a count of ordinary activation-times-weight operations actually issued. Read together, the complete operation is: This makes status_dense_macs, a count of ordinary activation-times-weight operations actually issued, equal to ‘{ status_dense_macs[63:24] + {{39{1’b0}}, pending_dense_macs_carry_r}, pending_dense_macs_low_r }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_dense_macs: a count of ordinary activation-times-weight operations actually issued; b0: the value named ’b0’ in this block; pending_dense_macs_carry_r: a count of ordinary activation-times-weight operations actually issued. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 959: `pending_dense_macs_low_r` This is continuation line 4 of the 5-line logical statement that begins at source line 956. Its exact fragment is ‘pending_dense_macs_low_r‘. The important operands are pending_dense_macs_low_r: a count of ordinary activation-times-weight operations actually issued. Read together, the complete operation is: This makes status_dense_macs, a count of ordinary activation-times-weight operations actually issued, equal to ‘{ status_dense_macs[63:24] + {{39{1’b0}}, pending_dense_macs_carry_r}, pending_dense_macs_low_r }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_dense_macs: a count of ordinary activation-times-weight operations actually issued; b0: the value named ’b0’ in this block; pending_dense_macs_carry_r: a count of ordinary activation-times-weight operations actually issued. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 960: `};` This is continuation line 5 of the 5-line logical statement that begins at source line 956. Its exact fragment is '};'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This makes status_dense_macs, a count of ordinary activation-times-weight operations actually issued, equal to '{ status_dense_macs[63:24] + {{39{1'b0}}, pending_dense_macs_carry_r}, pending_dense_macs_low_r }'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_dense_macs: a count of ordinary activation-times-weight operations actually issued; b0: the value named 'b0' in this block; pending_dense_macs_carry_r: a count of ordinary activation-times-weight operations actually issued. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 961: `status_spike_adds <= {` This statement continues through source line 965; the explanation uses the complete expression rather than only the first visual fragment. This makes status_spike_adds, a binary event produced when a neuron crosses its threshold, equal to '{ status_spike_adds[63:24] + {{39{1'b0}}, pending_spike_adds_carry_r}, pending_spike_adds_low_r }'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_spike_adds: a binary event produced when a neuron crosses its threshold; b0: the value named 'b0' in this block; pending_spike_adds_carry_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 962: `status_spike_adds[63:24]` This is continuation line 2 of the 5-line logical statement that begins at source line 961. Its exact fragment is 'status_spike_adds[63:24]'. The important operands are status_spike_adds: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This makes status_spike_adds, a binary event produced when a neuron crosses its threshold, equal to '{ status_spike_adds[63:24] + {{39{1'b0}}, pending_spike_adds_carry_r}, pending_spike_adds_low_r }'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_spike_adds: a binary event produced when a neuron crosses its threshold; b0: the value named 'b0' in this block; pending_spike_adds_carry_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 963: `+ {{39{1'b0}}, pending_spike_adds_carry_r},` This is continuation line 3 of the 5-line logical statement that begins at source line 961. Its exact fragment is '+ {{39{1'b0}}, pending_spike_adds_carry_r},'. The important operands are b0: the value named 'b0' in this block; pending_spike_adds_carry_r: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This makes status_spike_adds, a binary event produced when a neuron crosses its threshold, equal to '{ status_spike_adds[63:24] + {{39{1'b0}}, pending_spike_adds_carry_r}, pending_spike_adds_low_r }'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_spike_adds: a binary event produced when a neuron crosses its threshold; b0: the value named 'b0' in this block; pending_spike_adds_carry_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 964: `pending_spike_adds_low_r` This is continuation line 4 of the 5-line logical statement that begins at source line 961. Its exact fragment is 'pending_spike_adds_low_r'. The important operands are pending_spike_adds_low_r: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This makes status_spike_adds, a binary event produced when a neuron crosses its threshold, equal to '{ status_spike_adds[63:24] + {{39{1'b0}}, pending_spike_adds_carry_r}, pending_spike_adds_low_r }'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_spike_adds: a binary event produced when a neuron crosses its threshold; b0: the value named 'b0' in this block; pending_spike_adds_carry_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 965: `};` This is continuation line 5 of the 5-line logical statement that begins at source line 961. Its exact fragment is '};'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This makes status_spike_adds, a binary event produced when a neuron crosses its threshold, equal to '{ status_spike_adds[63:24] + {{39{1'b0}}, pending_spike_adds_carry_r}, pending_spike_adds_low_r }'.

It updates after the active clock edge together with other nonblocking assignments. The important operands are status_spike_adds: a binary event produced when a neuron crosses its threshold; b0: the value named 'b0' in this block; pending_spike_adds_carry_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 966: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 967: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 968: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 970: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'state <= ST_IDLE;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip..

Line 971: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPUTE_ACCOUNT, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 972: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 973: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 975: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 976: `if (MAX_OUTER_ISSUES < 8 || MAX_OUTER_ISSUES > 2048) begin` This condition selects the following hardware branch only when 'MAX_OUTER_ISSUES < 8 || MAX_OUTER_ISSUES > 2048' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are MAX_OUTER_ISSUES: the value named 'max outer issues' in this block. Context: the module's structural declarations.

Line 977: `$fatal(1, "MAX_OUTER_ISSUES must be in [8,2048]");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 978: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 979: `if (MAX_DOT8_ISSUES != MAX_OUTER_ISSUES / 8) begin` This condition selects the following hardware branch only when ‘MAX_DOT8_ISSUES != MAX_OUTER_ISSUES / 8‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are MAX_DOT8_ISSUES: the value named ’max dot8 issues’ in this block; MAX_OUTER_ISSUES: the value named ’max outer issues’ in this block. Context: the module’s structural declarations.

Line 980: `$fatal(1, "MAX_DOT8_ISSUES must equal MAX_OUTER_ISSUES/8");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 981: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 982: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 983: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 985: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO.

7 rtl/neuraedge_command_pkg.sv

Role in the complete system: defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

7.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

What becomes FPGA hardware Vivado elaborates module(s) declared here into physical nets and primitives. The source declares 0 input-port lines, 0 output-port lines, 0 clocked process(es), and 0 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls.

Connections to other blocks This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

7.2 Complete source

```
1  `timescale 1ns / 1ps
2
3  package neuraedge_command_pkg;
4      localparam logic [3:0] CMD_TRAIN_LINEAR_BACKWARD = 4'h0;
5      localparam logic [3:0] CMD_LOAD_ACTIVATIONS = 4'h1;
6      localparam logic [3:0] CMD_LOAD_WEIGHTS_OUTER = 4'h2;
7      localparam logic [3:0] CMD_LOAD_WEIGHTS_DOT8 = 4'h3;
8      localparam logic [3:0] CMD_COMPUTE_DENSE_OUTER = 4'h4;
9      localparam logic [3:0] CMD_COMPUTE_DENSE_DOT8 = 4'h5;
10     localparam logic [3:0] CMD_COMPUTE_BINARY = 4'h6;
11     localparam logic [3:0] CMD_COMPUTE_TERNARY = 4'h7;
12     localparam logic [3:0] CMD_ALPHA_XNOR = 4'h8;
13     localparam logic [3:0] CMD_VECTOR_ALU = 4'h9;
14     localparam logic [3:0] CMD_LAYERNORM = 4'ha;
15     localparam logic [3:0] CMD_LOG_SOFTMAX = 4'hb;
16     localparam logic [3:0] CMD_LIF_STEP = 4'hc;
17     localparam logic [3:0] CMD_SGD_UPDATE = 4'hd;
18     localparam logic [3:0] CMD_ACTIVATION_LUT = 4'he;
19     localparam logic [3:0] CMD_CLEAR_STATUS = 4'hf;
20
21     localparam logic [1:0] MODE_DENSE_OUTER = 2'd0;
22     localparam logic [1:0] MODE_DENSE_DOT8 = 2'd1;
23     localparam logic [1:0] MODE_BINARY_WEIGHTED = 2'd2;
24     localparam logic [1:0] MODE_TERNARY_WEIGHTED = 2'd3;
25
26     // The 128-bit command word is deliberately fixed-width so software can
27     // submit it as four ordinary 32-bit AXI-Lite register writes.
28     // [3:0]   opcode
29     // [7:4]   active output rows
30     // [13:8]  active output columns
31     // [25:14] reduction issue count
32     // [33:26] activation zero point
33     // [41:34] weight zero point
34     // [127:42] tensor commands: reserved and required to be zero
35     // Other operators reinterpret the payload as documented in
36     // docs/stream_protocol.md; opcode placement never changes.
37 endpackage
```

7.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 3: `package neuraedge_command_pkg;` This opens neuraedge_command_pkg;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 4: `localparam logic [3:0] CMD_TRAIN_LINEAR_BACKWARD = 4'h0;` This fixes CMD_TRAIN_LINEAR_BACKWARD to 4'h0 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_TRAIN_LINEAR_BACKW denotes the value named 'cmd train linear backward' in this block.

Line 5: `localparam logic [3:0] CMD_LOAD_ACTIVATIONS = 4'h1;` This fixes CMD_LOAD_ACTIVATIONS to 4'h1 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_LOAD_ACTIVATIONS denotes an input feature value consumed by a neuron or multiply-accumulate operation.

Line 6: `localparam logic [3:0] CMD_LOAD_WEIGHTS_OUTER = 4'h2;` This fixes CMD_LOAD_WEIGHTS_OUTER to 4'h2 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_LOAD_WEIGHTS_OUTER denotes a learned coefficient multiplied by an activation.

Line 7: `localparam logic [3:0] CMD_LOAD_WEIGHTS_DOT8 = 4'h3;` This fixes CMD_LOAD_WEIGHTS_DOT8 to 4'h3 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_LOAD_WEIGHTS_DOT8 denotes a learned coefficient multiplied by an activation.

Line 8: `localparam logic [3:0] CMD_COMPUTE_DENSE_OUTER = 4'h4;` This fixes CMD_COMPUTE_DENSE_OUTER to 4'h4 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_COMPUTE_DENSE_OUTE denotes the value named 'cmd compute dense outer' in this block.

Line 9: `localparam logic [3:0] CMD_COMPUTE_DENSE_DOT8 = 4'h5;` This fixes CMD_COMPUTE_DENSE_DOT8 to 4'h5 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_COMPUTE_DENSE_DOT8 denotes the value named 'cmd compute dense dot8' in this block.

Line 10: `localparam logic [3:0] CMD_COMPUTE_BINARY = 4'h6;` This fixes CMD_COMPUTE_BINARY to 4'h6 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_COMPUTE_BINARY denotes the value named 'cmd compute binary' in this block.

Line 11: `localparam logic [3:0] CMD_COMPUTE_TERNARY = 4'h7;` This fixes CMD_COMPUTE_TERNARY to 4'h7 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_COMPUTE_TERNARY denotes the value named 'cmd compute ternary' in this block.

Line 12: `localparam logic [3:0] CMD_ALPHA_XNOR = 4'h8;` This fixes CMD_ALPHA_XNOR to 4'h8 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_ALPHA_XNOR denotes the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 13: `localparam logic [3:0] CMD_VECTOR_ALU = 4'h9;` This fixes CMD_VECTOR_ALU to 4'h9 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_VECTOR_ALU denotes the value named 'cmd vector alu' in this block.

Line 14: `localparam logic [3:0] CMD_LAYERNORM = 4'ha;` This fixes CMD_LAYERNORM to 4'ha during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_LAYERNORM denotes the value named 'cmd layernorm' in this block.

Line 15: `localparam logic [3:0] CMD_LOG_SOFTMAX = 4'hb;` This fixes CMD_LOG_SOFTMAX to 4'hb during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_LOG_SOFTMAX denotes the value named 'cmd log softmax' in this block.

Line 16: `localparam logic [3:0] CMD_LIF_STEP = 4'hc;` This fixes CMD_LIF_STEP to 4'hc during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_LIF_STEP denotes the value named 'cmd lif step' in this block.

Line 17: `localparam logic [3:0] CMD_SGD_UPDATE = 4'hd;` This fixes CMD_SGD_UPDATE to 4'hd during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_SGD_UPDATE denotes the value named 'cmd sgd update' in this block.

Line 18: `localparam logic [3:0] CMD_ACTIVATION_LUT = 4'he;` This fixes CMD_ACTIVATION_LUT to 4'he during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_ACTIVATION_LUT denotes an input feature value consumed by a neuron or multiply-accumulate operation.

Line 19: `localparam logic [3:0] CMD_CLEAR_STATUS = 4'hf;` This fixes CMD_CLEAR_STATUS to 4'hf during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CMD_CLEAR_STATUS denotes the value named 'cmd clear status' in this block.

Line 21: `localparam logic [1:0] MODE_DENSE_OUTER = 2'd0;` This fixes MODE_DENSE_OUTER to 2'd0 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_DENSE_OUTER denotes the value named 'mode dense outer' in this block.

Line 22: `localparam logic [1:0] MODE_DENSE_DOT8 = 2'd1;` This fixes MODE_DENSE_DOT8 to 2'd1 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_DENSE_DOT8 denotes the value named 'mode dense dot8' in this block.

Line 23: `localparam logic [1:0] MODE_BINARY_WEIGHTED = 2'd2;` This fixes MODE_BINARY_WEIGHTED to 2'd2 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_BINARY_WEIGHTED denotes a learned coefficient multiplied by an activation.

Line 24: `localparam logic [1:0] MODE_TERNARY_WEIGHTED = 2'd3;` This fixes MODE_TERNARY_WEIGHTED to 2'd3 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_TERNARY_WEIGHTED denotes a learned coefficient multiplied by an activation.

Line 26: `// The 128-bit command word is deliberately fixed-width so software can` Design intent: The 128-bit command word is deliberately fixed-width so software can This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 27: `// submit it as four ordinary 32-bit AXI-Lite register writes.` Design intent: submit it as four ordinary 32-bit AXI-Lite register writes. This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 28: `// [3:0] opcode` Design intent: [3:0] opcode This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 29: `// [7:4] active output rows` Design intent: [7:4] active output rows This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 30: `// [13:8] active output columns` Design intent: [13:8] active output columns This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 31: `// [25:14] reduction issue count` Design intent: [25:14] reduction issue count This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 32: `// [33:26] activation zero point` Design intent: [33:26] activation zero point This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 33: `// [41:34] weight zero point` Design intent: [41:34] weight zero point This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 34: `// [127:42] tensor commands: reserved and required to be zero` Design intent: [127:42] tensor commands: reserved and required to be zero This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 35: `// Other operators reinterpret the payload as documented in` Design intent: Other operators reinterpret the payload as documented in This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 36: `// docs/stream_protocol.md; opcode placement never changes.` Design intent: docs/stream_protocol.md; opcode placement never changes. This comment anchors the surrounding logic in the block that defines the shared command opcodes and tensor-mode encodings used by RTL and host software.

Line 37: `endpackage` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

8 rtl/neuraedge_descriptor_sequencer.sv

Role in the complete system: turns a multi-record DMA program into checked core commands and payload/output streams.

8.1 Beginner architecture map

Dataflow responsibility A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_descriptor_sequencer into physical nets and primitives. The source declares 16 input-port lines, 14 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_COMMAND_HIGH, ST_COMMAND_LOW, ST_ERROR_DRAIN ST_HEADER, ST_PAYLOAD, ST_SUBMIT, ST_WAIT. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates unique as case, endcase as if, end as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

8.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  // Converts one DMA stream of descriptor records into ordinary core commands.
5  // A record is header, command[63:0], command[127:64], then input_count payload
6  // beats. Header [31:0] is input_count, [62:32] is output_count, and [63] marks
7  // the final record. External TLAST appears only on the final program input beat;
8  // the sequencer regenerates the per-command TLAST required by the core.
9  module neuraedge_descriptor_sequencer (
10     input  wire clk,
11     input  wire rst_n,
12
13     input  wire program_start,
14     input  wire clear_status,
15     input  wire [31:0] program_output_beats,
16     output logic program_active,
17     output logic program_done,
18     output logic program_error,
19     output logic [31:0] program_records,
20
21     input  wire s_axis_tvalid,
22     output logic s_axis_tready,
23     input  wire [63:0] s_axis_tdata,
24     input  wire s_axis_tlast,
25
26     output logic m_axis_tvalid,
27     input  wire m_axis_tready,
28     output logic [63:0] m_axis_tdata,
29     output logic m_axis_tlast,
30
31     output logic core_command_valid,
32     input  wire core_command_ready,
33     output logic [127:0] core_command_data,
34     input  wire core_command_done,
35     input  wire core_status_error,
36
37     output logic core_s_valid,
38     input  wire core_s_ready,
39     output logic [63:0] core_s_data,
40     output logic core_s_last,
41
42     input  wire core_m_valid,
43     output logic core_m_ready,
```

```
    input  wire [63:0] core_m_data,
    input  wire core_m_last
);
    typedef enum logic [3:0] {
        ST_HEADER = 4'd0,
        ST_COMMAND_LOW = 4'd1,
        ST_COMMAND_HIGH = 4'd2,
        ST_SUBMIT = 4'd3,
        ST_PAYLOAD = 4'd4,
        ST_WAIT = 4'd5,
        ST_ERROR_DRAIN = 4'd6
    } state_t;

    state_t state;
    logic [127:0] descriptor_command_r;
    logic [31:0] input_remaining_r;
    logic input_zero_r;
    logic input_last_r;
    logic [30:0] output_remaining_r;
    logic output_zero_r;
    logic output_last_r;
    logic [31:0] program_output_remaining_r;
    logic program_output_zero_r;
    logic program_output_last_r;
    logic record_final_r;
    logic command_done_seen_r;
    logic command_error_seen_r;
    logic [63:0] input_fifo_data_r [0:1];
    logic input_fifo_last_r [0:1];
    logic input_fifo_write_pointer_r;
    logic input_fifo_read_pointer_r;
    logic [1:0] input_fifo_count_r;
    logic output_buffer_valid_r;
    logic output_buffer_overrun_r;
    logic [63:0] output_buffer_data_r;
    logic output_buffer_last_r;
    // Malformed descriptor detection is quarantined for one cycle.  The
    // pending bit immediately blocks every handshake; on the following edge a
    // local decode clears the FIFO/buffer and either stops or drains input.
    // This separates error detection from a wide multi-register clear cone.
    logic error_pending_r;
    logic error_input_was_last_r;

    wire input_accept = s_axis_tvalid && s_axis_tready;
    wire payload_input_accept = input_accept && state == ST_PAYLOAD;
    wire core_input_accept = core_s_valid && core_s_ready;
    wire output_accept = m_axis_tvalid && m_axis_tready;
    wire core_output_accept = core_m_valid && core_m_ready;
    wire core_output_overrun = core_output_accept
        && (output_zero_r || (output_last_r && output_buffer_valid_r));
    wire record_outputs_complete = output_zero_r;
    wire record_command_complete = command_done_seen_r;

    always_comb begin
        core_command_valid = program_active && !error_pending_r
            && state == ST_SUBMIT;
        core_command_data = descriptor_command_r;

        s_axis_tready = 1'b0;
        core_s_valid = program_active && !error_pending_r
            && (state == ST_PAYLOAD || state == ST_WAIT)
            && input_fifo_count_r != 0;
        core_s_data = input_fifo_data_r[input_fifo_read_pointer_r];
        core_s_last = input_fifo_last_r[input_fifo_read_pointer_r];
        if (program_active && !error_pending_r) begin
            unique case (state)
                ST_HEADER,
                ST_COMMAND_LOW,
                ST_COMMAND_HIGH: s_axis_tready = 1'b1;
                ST_PAYLOAD: s_axis_tready = input_fifo_count_r != 2;
                ST_ERROR_DRAIN: s_axis_tready = 1'b1;
                default: begin end
            endcase
        end

        m_axis_tvalid = program_active && !error_pending_r
            && output_buffer_valid_r && !output_buffer_overrun_r;
        m_axis_tdata = output_buffer_data_r;
        m_axis_tlast = program_output_last_r;
        core_m_ready = program_active && !error_pending_r
            && (state == ST_PAYLOAD || state == ST_WAIT)
            && (!output_buffer_valid_r || m_axis_tready);
    end

`define NE_ENTER_PROGRAM_ERROR(INPUT_WAS_LAST) \
    begin \
        program_error <= 1'b1; \
```

```
131         error_pending_r <= 1'b1; \
132         error_input_was_last_r <= INPUT_WAS_LAST; \
133     end
134
135     always_ff @(posedge clk) begin
136         if (!rst_n) begin
137             state <= ST_HEADER;
138             descriptor_command_r <= 128'd0;
139             input_remaining_r <= 32'd0;
140             input_zero_r <= 1'b1;
141             input_last_r <= 1'b0;
142             output_remaining_r <= 31'd0;
143             output_zero_r <= 1'b1;
144             output_last_r <= 1'b0;
145             program_output_remaining_r <= 32'd0;
146             program_output_zero_r <= 1'b1;
147             program_output_last_r <= 1'b0;
148             record_final_r <= 1'b0;
149             command_done_seen_r <= 1'b0;
150             command_error_seen_r <= 1'b0;
151             input_fifo_data_r[0] <= 64'd0;
152             input_fifo_data_r[1] <= 64'd0;
153             input_fifo_last_r[0] <= 1'b0;
154             input_fifo_last_r[1] <= 1'b0;
155             input_fifo_write_pointer_r <= 1'b0;
156             input_fifo_read_pointer_r <= 1'b0;
157             input_fifo_count_r <= 2'd0;
158             output_buffer_valid_r <= 1'b0;
159             output_buffer_overrun_r <= 1'b0;
160             output_buffer_data_r <= 64'd0;
161             output_buffer_last_r <= 1'b0;
162             error_pending_r <= 1'b0;
163             error_input_was_last_r <= 1'b0;
164             program_active <= 1'b0;
165             program_done <= 1'b0;
166             if (clear_status && !program_active) program_error <= 1'b0;
167             program_error <= 1'b0;
168             program_records <= 32'd0;
169         end else begin
170             program_done <= 1'b0;
171             unique case ({payload_input_accept, core_input_accept})
172                 2'b10: input_fifo_count_r <= input_fifo_count_r + 1'b1;
173                 2'b01: input_fifo_count_r <= input_fifo_count_r - 1'b1;
174                 default: begin end
175             endcase
176             if (payload_input_accept) begin
177                 input_fifo_data_r[input_fifo_write_pointer_r] <= s_axis_tdata;
178                 input_fifo_last_r[input_fifo_write_pointer_r]
179                     <= input_last_r;
180                 input_fifo_write_pointer_r <= !input_fifo_write_pointer_r;
181             end
182             if (core_input_accept) begin
183                 input_fifo_read_pointer_r <= !input_fifo_read_pointer_r;
184             end
185             unique case ({core_output_accept, output_accept})
186                 2'b10: output_buffer_valid_r <= 1'b1;
187                 2'b01: output_buffer_valid_r <= 1'b0;
188                 default: begin end
189             endcase
190             if (core_output_accept) begin
191                 output_buffer_data_r <= core_m_data;
192                 output_buffer_last_r <= core_m_last;
193                 output_buffer_overrun_r <= core_output_overrun;
194             end
195             if (clear_status && !program_active) program_error <= 1'b0;
196             if (program_start) begin
197                 if (program_active) begin
198                     program_error <= 1'b1;
199                     program_done <= 1'b1;
200                 end else begin
201                     state <= ST_HEADER;
202                     descriptor_command_r <= 128'd0;
203                     input_remaining_r <= 32'd0;
204                     input_zero_r <= 1'b1;
205                     input_last_r <= 1'b0;
206                     output_remaining_r <= 31'd0;
207                     output_zero_r <= 1'b1;
208                     output_last_r <= 1'b0;
209                     program_output_remaining_r <= program_output_beats;
210                     program_output_zero_r <= program_output_beats == 0;
211                     program_output_last_r <= program_output_beats == 1;
212                     record_final_r <= 1'b0;
213                     command_done_seen_r <= 1'b0;
214                     command_error_seen_r <= 1'b0;
215                     input_fifo_write_pointer_r <= 1'b0;
216                     input_fifo_read_pointer_r <= 1'b0;
217                     input_fifo_count_r <= 2'd0;
```

```
218                     output_buffer_valid_r <= 1'b0;
219                     output_buffer_overrun_r <= 1'b0;
220                     error_pending_r <= 1'b0;
221                     error_input_was_last_r <= 1'b0;
222                     program_active <= 1'b1;
223                     program_error <= 1'b0;
224                     program_records <= 32'd0;
225                 end
226             end
227
228             if (error_pending_r) begin
229                 error_pending_r <= 1'b0;
230                 input_fifo_count_r <= 2'd0;
231                 output_buffer_valid_r <= 1'b0;
232                 output_buffer_overrun_r <= 1'b0;
233                 if (error_input_was_last_r) begin
234                     program_active <= 1'b0;
235                     program_done <= 1'b1;
236                     state <= ST_HEADER;
237                 end else begin
238                     state <= ST_ERROR_DRAIN;
239                 end
240             end else if (program_active) begin
241                 if (output_buffer_valid_r && output_buffer_overrun_r) begin
242                     `NE_ENTER_PROGRAM_ERROR(record_final_r)
243                 end else begin
244                     if (core_command_done) begin
245                         command_done_seen_r <= 1'b1;
246                         command_error_seen_r <= core_status_error;
247                     end
248                     if (output_accept) begin
249                         if (output_buffer_last_r != output_last_r
250                                 || program_output_zero_r) begin
251                             `NE_ENTER_PROGRAM_ERROR(record_final_r)
252                         end else begin
253                             output_remaining_r <= output_remaining_r - 1'b1;
254                             program_output_remaining_r
255                                 <= program_output_remaining_r - 1'b1;
256                             if (output_last_r) begin
257                                 output_zero_r <= 1'b1;
258                                 output_last_r <= 1'b0;
259                             end else if (output_remaining_r == 2) begin
260                                 output_last_r <= 1'b1;
261                             end
262                             if (program_output_last_r) begin
263                                 program_output_zero_r <= 1'b1;
264                                 program_output_last_r <= 1'b0;
265                             end else if (program_output_remaining_r == 2) begin
266                                 program_output_last_r <= 1'b1;
267                             end
268                         end
269                     end
270
271                     unique case (state)
272                     ST_HEADER: if (input_accept) begin
273                         input_remaining_r <= s_axis_tdata[31:0];
274                         input_zero_r <= s_axis_tdata[31:0] == 0;
275                         input_last_r <= s_axis_tdata[31:0] == 1;
276                         output_remaining_r <= s_axis_tdata[62:32];
277                         output_zero_r <= s_axis_tdata[62:32] == 0;
278                         output_last_r <= s_axis_tdata[62:32] == 1;
279                         record_final_r <= s_axis_tdata[63];
280                         command_done_seen_r <= 1'b0;
281                         command_error_seen_r <= 1'b0;
282                         if (s_axis_tlast) `NE_ENTER_PROGRAM_ERROR(1'b1)
283                         else state <= ST_COMMAND_LOW;
284                     end
285
286                     ST_COMMAND_LOW: if (input_accept) begin
287                         descriptor_command_r[63:0] <= s_axis_tdata;
288                         if (s_axis_tlast) `NE_ENTER_PROGRAM_ERROR(1'b1)
289                         else state <= ST_COMMAND_HIGH;
290                     end
291
292                     ST_COMMAND_HIGH: if (input_accept) begin
293                         descriptor_command_r[127:64] <= s_axis_tdata;
294                         if (s_axis_tlast != (record_final_r
295                                 && input_zero_r)) begin
296                             `NE_ENTER_PROGRAM_ERROR(s_axis_tlast)
297                         end else begin
298                             state <= ST_SUBMIT;
299                         end
300                     end
301
302                     ST_SUBMIT: if (core_command_ready) begin
303                         program_records <= program_records + 1'b1;
304                         if (input_zero_r) state <= ST_WAIT;
```

```
                        else state <= ST_PAYLOAD;
                    end

                    ST_PAYLOAD: if (input_accept) begin
                        if (s_axis_tlast != (record_final_r
                                && input_last_r)) begin
                            `NE_ENTER_PROGRAM_ERROR(s_axis_tlast)
                        end else begin
                            input_remaining_r <= input_remaining_r - 1'b1;
                            if (input_last_r) begin
                                input_zero_r <= 1'b1;
                                input_last_r <= 1'b0;
                                state <= ST_WAIT;
                            end else if (input_remaining_r == 2) begin
                                input_last_r <= 1'b1;
                            end
                        end
                    end

                    ST_WAIT: begin
                        if (record_command_complete && record_outputs_complete) begin
                            command_done_seen_r <= 1'b0;
                            command_error_seen_r <= 1'b0;
                            if (command_error_seen_r) begin
                                `NE_ENTER_PROGRAM_ERROR(record_final_r)
                            end else if (record_final_r) begin
                                if (!program_output_zero_r) begin
                                    `NE_ENTER_PROGRAM_ERROR(1'b1)
                                end else begin
                                    program_active <= 1'b0;
                                    program_done <= 1'b1;
                                    state <= ST_HEADER;
                                end
                            end else begin
                                state <= ST_HEADER;
                            end
                        end
                    end

                    ST_ERROR_DRAIN: if (input_accept && s_axis_tlast) begin
                        program_active <= 1'b0;
                        program_done <= 1'b1;
                        state <= ST_HEADER;
                    end

                        default: `NE_ENTER_PROGRAM_ERROR(1'b0)
                    endcase
                end
            end
        end
    end
endmodule

`undef NE_ENTER_PROGRAM_ERROR

`default_nettype wire
```

8.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `// Converts one DMA stream of descriptor records into ordinary core commands.` Design intent: Converts one DMA stream of descriptor records into ordinary core commands. This comment anchors the surrounding logic in the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 5: `// A record is header, command[63:0], command[127:64], then input_count payload` Design intent: A record is header, command[63:0], command[127:64], then input_count payload This comment anchors the surrounding logic in the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 6: `// beats. Header [31:0] is input_count, [62:32] is output_count, and [63] marks` Design intent: beats. Header [31:0] is input_count, [62:32] is output_count, and [63] marks This comment anchors the surrounding

logic in the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 7: `// the final record. External TLAST appears only on the final program input beat;` Design intent: the final record. External TLAST appears only on the final program input beat; This comment anchors the surrounding logic in the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 8: `// the sequencer regenerates the per-command TLAST required by the core.` Design intent: the sequencer regenerates the per-command TLAST required by the core. This comment anchors the surrounding logic in the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 9: `module neuraedge_descriptor_sequencer (` This opens neuraedge_descriptor_sequencer, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 10: `input wire clk,` This declares clk as a input port with width one bit. It carries the clock that makes all registers advance together across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 11: `input wire rst_n,` This declares rst_n as a input port with width one bit. It carries the active-low synchronous reset across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 13: `input wire program_start,` This declares program_start as a input port with width one bit. It carries the value named 'program start' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 14: `input wire clear_status,` This declares clear_status as a input port with width one bit. It carries the value named 'clear status' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 15: `input wire [31:0] program_output_beats,` This declares program_output_beats as a input port with width [31:0]. It carries the value named 'program output beats' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 16: `output logic program_active,` This declares program_active as a output port with width one bit. It carries the value named 'program active' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 17: `output logic program_done,` This declares program_done as a output port with width one bit. It carries the value named 'program done' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 18: `output logic program_error,` This declares program_error as a output port with width one bit. It carries the value named 'program error' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 19: `output logic [31:0] program_records,` This declares program_records as a output port with width [31:0]. It carries the number of descriptor records accepted since program start across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 21: `input wire s_axis_tvalid,` This declares s_axis_tvalid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 22: `output logic s_axis_tready,` This declares s_axis_tready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 23: `input wire [63:0] s_axis_tdata,` This declares s_axis_tdata as a input port with width [63:0]. It carries the value named 's axis tdata' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 24: `input wire s_axis_tlast,` This declares s_axis_tlast as a input port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 26: `output logic m_axis_tvalid,` This declares m_axis_tvalid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 27: `input wire m_axis_tready,` This declares m_axis_tready as a input port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 28: `output logic [63:0] m_axis_tdata,` This declares m_axis_tdata as a output port with width [63:0]. It carries the value named 'm axis tdata' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 29: `output logic m_axis_tlast,` This declares m_axis_tlast as a output port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 31: `output logic core_command_valid,` This declares core_command_valid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 32: `input wire core_command_ready,` This declares core_command_ready as a input port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 33: `output logic [127:0] core_command_data,` This declares core_command_data as a output port with width [127:0]. It carries the 128-bit instruction selecting an engine, shape, mode, and numeric parameters across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 34: `input wire core_command_done,` This declares core_command_done as a input port with width one bit. It carries the 128-bit instruction selecting an engine, shape, mode, and numeric parameters across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 35: `input wire core_status_error,` This declares core_status_error as a input port with width one bit. It carries the value named 'core status error' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 37: `output logic core_s_valid,` This declares core_s_valid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 38: `input wire core_s_ready,` This declares core_s_ready as a input port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 39: `output logic [63:0] core_s_data,` This declares core_s_data as a output port with width [63:0]. It carries the payload carried with the surrounding valid/ready handshake across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 40: `output logic core_s_last,` This declares core_s_last as a output port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 42: `input wire core_m_valid,` This declares core_m_valid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 43: `output logic core_m_ready,` This declares core_m_ready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 44: `input wire [63:0] core_m_data,` This declares core_m_data as a input port with width [63:0]. It carries the payload carried with the surrounding valid/ready handshake across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 45: `input wire core_m_last` This declares core_m_last as a input port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 46: `);` Within the module's structural declarations, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The expression is a literal or structural operator and has no hidden software state.

Line 47: `typedef enum logic [3:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 48: `ST_HEADER = 4'd0,` This assigns a fixed binary encoding to state ST_HEADER, which accepts and validates the descriptor record header. The encoding is visible in waveforms even though the source uses the readable name.

Line 49: `ST_COMMAND_LOW = 4'd1,` This assigns a fixed binary encoding to state ST_COMMAND_LOW, which captures command bits 63 through 0. The encoding is visible in waveforms even though the source uses the readable name.

Line 50: `ST_COMMAND_HIGH = 4'd2,` This assigns a fixed binary encoding to state ST_COMMAND_HIGH, which captures command bits 127 through 64 and checks input TLAST. The encoding is visible in waveforms even though the source uses the readable name.

Line 51: `ST_SUBMIT = 4'd3,` This assigns a fixed binary encoding to state ST_SUBMIT, which holds command-valid until the universal core accepts the instruction. The encoding is visible in waveforms even though the source uses the readable name.

Line 52: `ST_PAYLOAD = 4'd4,` This assigns a fixed binary encoding to state ST_PAYLOAD, which forwards exactly the declared number of input beats to the selected engine. The encoding is visible in waveforms even though the source uses the readable name.

Line 53: `ST_WAIT = 4'd5,` This assigns a fixed binary encoding to state ST_WAIT, which waits for both command completion and the declared output count. The encoding is visible in waveforms even though the source uses the readable name.

Line 54: `ST_ERROR_DRAIN = 4'd6` This statement continues through source line 55; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_ERROR_DRAIN, which discards remaining DMA input until TLAST so the next transfer starts aligned. The encoding is visible in waveforms even though the source uses the readable name.

Line 55: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 54. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_ERROR_DRAIN, which discards remaining DMA input until TLAST so the next transfer starts aligned. The encoding is visible in waveforms even though the source uses the readable name.

Line 57: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 58: `logic [127:0] descriptor_command_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, descriptor_command_r is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 59: `logic [31:0] input_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_remaining_r is the number of command payload beats still expected. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 60: `logic input_zero_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_zero_r is the value named 'input zero r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 61: `logic input_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 62: `logic [30:0] output_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_remaining_r is the number of command result beats still expected. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 63: `logic output_zero_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_zero_r is the value named 'output zero r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 64: `logic output_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 65: `logic [31:0] program_output_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_output_remaining_r is the number of result beats left in the complete descriptor program. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 66: `logic program_output_zero_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_output_zero_r is the value named 'program output zero r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 67: `logic program_output_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_output_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 68: `logic record_final_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, record_final_r is the value named 'record final r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 69: `logic command_done_seen_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_done_seen_r is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 70: `logic command_error_seen_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_error_seen_r is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 71: `logic [63:0] input_fifo_data_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 72: `logic input_fifo_last_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 73: `logic input_fifo_write_pointer_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_write_pointer_r is the value named 'input fifo write pointer r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 74: `logic input_fifo_read_pointer_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_read_pointer_r is the value named 'input fifo read pointer r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 75: `logic [1:0] input_fifo_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 76: `logic output_buffer_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_buffer_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 77: `logic output_buffer_overrun_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_buffer_overrun_r is local storage that decouples producer and consumer timing. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 78: `logic [63:0] output_buffer_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_buffer_data_r is local storage that decouples producer and consumer timing. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 79: `logic output_buffer_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_buffer_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 80: `// Malformed descriptor detection is quarantined for one cycle. The` Design intent: Malformed descriptor detection is quarantined for one cycle. The This comment anchors the surrounding logic in the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 81: `// pending bit immediately blocks every handshake; on the following edge a` Design intent: pending bit immediately blocks every handshake; on the following edge a This comment anchors the surrounding logic in the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 82: `// local decode clears the FIFO/buffer and either stops or drains input.` Design intent: local decode clears the FIFO/buffer and either stops or drains input. This comment anchors the surrounding logic in the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 83: `// This separates error detection from a wide multi-register clear cone.` Design intent: This separates error detection from a wide multi-register clear cone. This comment anchors the surrounding logic in the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 84: `logic error_pending_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, error_pending_r is the value named 'error pending r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 85: `logic error_input_was_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, error_input_was_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 87: `wire input_accept = s_axis_tvalid && s_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_accept is the value named 'input accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 88: `wire payload_input_accept = input_accept && state == ST_PAYLOAD;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, payload_input_accept is stream data belonging to the current command. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 89: `wire core_input_accept = core_s_valid && core_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_input_accept is the value named 'core input accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 90: `wire output_accept = m_axis_tvalid && m_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_accept is the value named 'output accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 91: `wire core_output_accept = core_m_valid && core_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_accept is the value named 'core output accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 92: `wire core_output_overrun = core_output_accept` This statement continues through source line 93; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_overrun is the value named 'core output overrun' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 93: `&& (output_zero_r || (output_last_r && output_buffer_valid_r));` This is continuation line 2 of the 2-line logical statement that begins at source line 92. Its exact fragment is '&& (output_zero_r || (output_last_r && output_buffer_valid_r));'. The important operands are output_zero_r: the value named 'output zero r' in this block; output_last_r: the AXI-Stream marker for the final beat of a packet or command payload; output_buffer_valid_r: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_overrun is the value named 'core output overrun' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 94: `wire record_outputs_complete = output_zero_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, record_outputs_complete is the value named 'record outputs complete' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 95: `wire record_command_complete = command_done_seen_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, record_command_complete is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 97: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 98: `core_command_valid = program_active && !error_pending_r` This statement continues through source line 99; the explanation uses the complete expression rather than only the first visual fragment. This makes core_command_valid, the producer's statement that its current payload is meaningful, equal to 'program_active && !error_pending_r && state == ST_SUBMIT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; state: the finite-state-machine phase retained in flip-flops. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 99: `&& state == ST_SUBMIT;` This is continuation line 2 of the 2-line logical statement that begins at source line 98. Its exact fragment is '&& state == ST_SUBMIT;'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_SUBMIT: the value named 'st submit' in this block. Read together, the complete operation is: This makes core_command_valid, the producer's statement that its current payload is meaningful, equal to 'program_active && !error_pending_r && state == ST_SUBMIT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; state: the finite-state-machine phase retained in flip-flops. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 100: `core_command_data = descriptor_command_r;` This makes core_command_data, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'descriptor_command_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are descriptor_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 102: `s_axis_tready = 1'b0;` This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 103: `core_s_valid = program_active && !error_pending_r` This statement continues through source line 105; the explanation uses the complete expression rather than only the first visual fragment. This makes core_s_valid, the producer's statement that its current payload is meaningful, equal to 'program_active && !error_pending_r && (state == ST_PAYLOAD || state == ST_WAIT) && input_fifo_count_r != 0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; state: the finite-state-machine phase retained in flip-flops. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 104: `&& (state == ST_PAYLOAD || state == ST_WAIT)` This is continuation line 2 of the 3-line logical statement that begins at source line 103. Its exact fragment is '&& (state == ST_PAYLOAD || state == ST_WAIT)'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_PAYLOAD: stream data belonging to the current command; ST_WAIT: the value named 'st wait' in this block. Read together, the complete operation is: This makes core_s_valid, the producer's statement that its current payload is meaningful, equal to 'program_active && !error_pending_r && (state == ST_PAYLOAD || state == ST_WAIT) && input_fifo_count_r != 0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; state: the finite-state-machine phase retained in flip-flops. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 105: `&& input_fifo_count_r != 0;` This is continuation line 3 of the 3-line logical statement that begins at source line 103. Its exact fragment is '&& input_fifo_count_r != 0;'. The important operands are input_fifo_count_r: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This makes core_s_valid, the producer's statement that its current payload is meaningful, equal to 'program_active && !error_pending_r && (state == ST_PAYLOAD || state == ST_WAIT) && input_fifo_count_r != 0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; state: the finite-state-machine phase retained in flip-flops. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 106: `core_s_data = input_fifo_data_r[input_fifo_read_pointer_r];` This makes core_s_data, the payload carried with the surrounding valid/ready handshake, equal to 'input_fifo_data_r[input_fifo_read_pointer_r]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_fifo_data_r: the payload carried with the surrounding valid/ready handshake; input_fifo_read_pointer_r: the value named 'input fifo read pointer r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 107: `core_s_last = input_fifo_last_r[input_fifo_read_pointer_r];` This makes core_s_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'input_fifo_last_r[input_fifo_read_pointer_r]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_fifo_last_r: the AXI-Stream marker for the final beat of a packet or command payload; input_fifo_read_pointer_r: the value named 'input fifo read pointer r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 108: `if (program_active && !error_pending_r) begin` This condition selects the following hardware branch only when 'program_active && !error_pending_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 109: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 110: `ST_HEADER,` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are ST_HEADER: the value named 'st header' in this block.

Line 111: `ST_COMMAND_LOW,` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are ST_COMMAND_LOW: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 112: `ST_COMMAND_HIGH: s_axis_tready = 1'b1;` This is the ST_COMMAND_HIGH case branch, which captures command bits 127 through 64 and checks input TLAST. The same-cycle action begins with 's_axis_tready =

1'b1;‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_COMMAND_HIGH, the machine captures command bits 127 through 64 and checks input TLAST..

Line 113: `ST_PAYLOAD: s_axis_tready = input_fifo_count_r != 2;` This is the ST_PAYLOAD case branch, which forwards exactly the declared number of input beats to the selected engine. The same-cycle action begins with ‘s_axis_tready = input_fifo_count_r != 2;‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine..

Line 114: `ST_ERROR_DRAIN: s_axis_tready = 1'b1;` This is the ST_ERROR_DRAIN case branch, which discards remaining DMA input until TLAST so the next transfer starts aligned. The same-cycle action begins with ‘s_axis_tready = 1'b1;‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned..

Line 115: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin end‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned..

Line 116: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned.; it has no arithmetic value of its own.

Line 117: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 119: `m_axis_tvalid = program_active && !error_pending_r` This statement continues through source line 120; the explanation uses the complete expression rather than only the first visual fragment. This makes m_axis_tvalid, the producer's statement that its current payload is meaningful, equal to ‘program_active && !error_pending_r && output_buffer_valid_r && !output_buffer_overrun_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; output_buffer_valid_r: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 120: `&& output_buffer_valid_r && !output_buffer_overrun_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 119. Its exact fragment is ‘&& output_buffer_valid_r && !output_buffer_overrun_r;‘. The important operands are output_buffer_valid_r: the producer's statement that its current payload is meaningful; output_buffer_overrun_r: local storage that decouples producer and consumer timing. Read together, the complete operation is: This makes m_axis_tvalid, the producer's statement that its current payload is meaningful, equal to ‘program_active && !error_pending_r && output_buffer_valid_r && !output_buffer_overrun_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; output_buffer_valid_r: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 121: `m_axis_tdata = output_buffer_data_r;` This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to ‘output_buffer_data_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_buffer_data_r: local storage that decouples producer and consumer timing. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 122: `m_axis_tlast = program_output_last_r;` This makes m_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘program_output_last_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_output_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 123: `core_m_ready = program_active && !error_pending_r` This statement continues through source line 125; the explanation uses the complete expression rather than only the first visual fragment. This makes core_m_ready, the consumer's statement that it can accept a payload this cycle, equal to 'program_active && !error_pending_r && (state == ST_PAYLOAD || state == ST_WAIT) && (!output_buffer_valid_r || m_axis_tready)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; state: the finite-state-machine phase retained in flip-flops. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 124: `&& (state == ST_PAYLOAD || state == ST_WAIT)` This is continuation line 2 of the 3-line logical statement that begins at source line 123. Its exact fragment is '&& (state == ST_PAYLOAD || state == ST_WAIT)'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_PAYLOAD: stream data belonging to the current command; ST_WAIT: the value named 'st wait' in this block. Read together, the complete operation is: This makes core_m_ready, the consumer's statement that it can accept a payload this cycle, equal to 'program_active && !error_pending_r && (state == ST_PAYLOAD || state == ST_WAIT) && (!output_buffer_valid_r || m_axis_tready)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; state: the finite-state-machine phase retained in flip-flops. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 125: `&& (!output_buffer_valid_r || m_axis_tready);` This is continuation line 3 of the 3-line logical statement that begins at source line 123. Its exact fragment is '&& (!output_buffer_valid_r || m_axis_tready);'. The important operands are output_buffer_valid_r: the producer's statement that its current payload is meaningful; m_axis_tready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This makes core_m_ready, the consumer's statement that it can accept a payload this cycle, equal to 'program_active && !error_pending_r && (state == ST_PAYLOAD || state == ST_WAIT) && (!output_buffer_valid_r || m_axis_tready)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; error_pending_r: the value named 'error pending r' in this block; state: the finite-state-machine phase retained in flip-flops. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 126: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 128: `` `define NE_ENTER_PROGRAM_ERROR(INPUT_WAS_LAST) \ `` This SystemVerilog preprocessor line controls compilation of the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 129: `begin \` Within the module's structural declarations, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The expression is a literal or structural operator and has no hidden software state.

Line 130: `program_error <= 1'b1; \` This makes program_error, the value named 'program error' in this block, equal to '1'b1; \'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 131: `error_pending_r <= 1'b1; \` This makes error_pending_r, the value named 'error pending r' in this block, equal to '1'b1; \'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 132: `error_input_was_last_r <= INPUT_WAS_LAST; \` This makes error_input_was_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'INPUT_WAS_LAST; \'. It updates after the active clock edge together with other nonblocking assignments. The important operands are INPUT_WAS_LAST: the AXI-Stream marker for the final beat of a packet or command payload. Context: the module's structural declarations.

Line 133: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 135: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 136: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 137: `state <= ST_HEADER;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_HEADER'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_HEADER: the value named 'st header' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 138: `descriptor_command_r <= 128'd0;` This makes descriptor_command_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 139: `input_remaining_r <= 32'd0;` This makes input_remaining_r, the number of command payload beats still expected, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 140: `input_zero_r <= 1'b1;` This makes input_zero_r, the value named 'input zero r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 141: `input_last_r <= 1'b0;` This makes input_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 142: `output_remaining_r <= 31'd0;` This makes output_remaining_r, the number of command result beats still expected, equal to '31'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 143: `output_zero_r <= 1'b1;` This makes output_zero_r, the value named 'output zero r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 144: `output_last_r <= 1'b0;` This makes output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 145: `program_output_remaining_r <= 32'd0;` This makes program_output_remaining_r, the number of result beats left in the complete descriptor program, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 146: `program_output_zero_r <= 1'b1;` This makes program_output_zero_r, the value named 'program output zero r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 147: `program_output_last_r <= 1'b0;` This makes program_output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 148: `record_final_r <= 1'b0;` This makes record_final_r, the value named 'record final r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 149: `command_done_seen_r <= 1'b0;` This makes command_done_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 150: `command_error_seen_r <= 1'b0;` This makes command_error_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 151: `input_fifo_data_r[0] <= 64'd0;` This makes input_fifo_data_r[0], the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 152: `input_fifo_data_r[1] <= 64'd0;` This makes input_fifo_data_r[1], the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 153: `input_fifo_last_r[0] <= 1'b0;` This makes input_fifo_last_r[0], the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 154: `input_fifo_last_r[1] <= 1'b0;` This makes input_fifo_last_r[1], the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 155: `input_fifo_write_pointer_r <= 1'b0;` This makes input_fifo_write_pointer_r, the value named 'input fifo write pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 156: `input_fifo_read_pointer_r <= 1'b0;` This makes input_fifo_read_pointer_r, the value named 'input fifo read pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 157: `input_fifo_count_r <= 2'd0;` This makes input_fifo_count_r, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 158: `output_buffer_valid_r <= 1'b0;` This makes output_buffer_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 159: `output_buffer_overrun_r <= 1'b0;` This makes output_buffer_overrun_r, local storage that decouples producer and consumer timing, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 160: `output_buffer_data_r <= 64'd0;` This makes output_buffer_data_r, local storage that decouples producer and consumer timing, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 161: `output_buffer_last_r <= 1'b0;` This makes output_buffer_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 162: `error_pending_r <= 1'b0;` This makes error_pending_r, the value named 'error pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 163: `error_input_was_last_r <= 1'b0;` This makes error_input_was_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 164: `program_active <= 1'b0;` This makes program_active, the value named 'program active' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 165: `program_done <= 1'b0;` This makes program_done, the value named 'program done' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 166: `if (clear_status && !program_active) program_error <= 1'b0;` This condition selects the following hardware branch only when 'clear_status && !program_active' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_status: the value named 'clear status' in this block; program_active: the value named 'program active' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 167: `program_error <= 1'b0;` This makes program_error, the value named 'program error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 168: `program_records <= 32'd0;` This makes program_records, the number of descriptor records accepted since program start, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 169: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 170: `program_done <= 1'b0;` This makes program_done, the value named 'program done' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 171: `unique case ({payload_input_accept, core_input_accept})` This starts a multiplexer/state decode selected by '{payload_input_accept, core_input_accept}'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 172: `2'b10: input_fifo_count_r <= input_fifo_count_r + 1'b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are b10: the value named 'b10' in this block; input_fifo_count_r: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block.

Line 173: `2'b01: input_fifo_count_r <= input_fifo_count_r - 1'b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are b01: the value named 'b01' in this block; input_fifo_count_r: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block.

Line 174: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the clocked process that updates registers only on a rising clock edge.

Line 175: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 176: `if (payload_input_accept) begin` This condition selects the following hardware branch only when 'payload_input_accept' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are payload_input_accept: stream data belonging to the current command. Context: the clocked process that updates registers only on a rising clock edge.

Line 177: `input_fifo_data_r[input_fifo_write_pointer_r] <= s_axis_tdata;` This makes input_fifo_data_r[input_fifo_ the payload carried with the surrounding valid/ready handshake, equal to 's_axis_tdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 178: `input_fifo_last_r[input_fifo_write_pointer_r]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are input_fifo_last_r: the AXI-Stream marker for the final beat of a packet or command payload; input_fifo_write_pointer_r: the value named 'input fifo write pointer r' in this block.

Line 179: `<= input_last_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are input_last_r: the AXI-Stream marker for the final beat of a packet or command payload.

Line 180: `input_fifo_write_pointer_r <= !input_fifo_write_pointer_r;` This makes input_fifo_write_pointer_r, the value named 'input fifo write pointer r' in this block, equal to '!input_fifo_write_pointer_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_fifo_write_pointer_r: the value named 'input fifo write pointer r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 181: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 182: `if (core_input_accept) begin` This condition selects the following hardware branch only when 'core_input_accept' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are core_input_accept: the value named 'core input accept' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 183: `input_fifo_read_pointer_r <= !input_fifo_read_pointer_r;` This makes input_fifo_read_pointer_r, the value named 'input fifo read pointer r' in this block, equal to '!input_fifo_read_pointer_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_fifo_read_pointer_r: the value named 'input fifo read pointer r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 184: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 185: `unique case ({core_output_accept, output_accept})` This starts a multiplexer/state decode selected by '{core_output_accept, output_accept}'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 186: `2'b10: output_buffer_valid_r <= 1'b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are b10: the value named 'b10' in this block; output_buffer_valid_r: the producer's statement that its current payload is meaningful; b1: the value named 'b1' in this block.

Line 187: `2'b01: output_buffer_valid_r <= 1'b0;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are b01: the value named 'b01' in this block; output_buffer_valid_r: the producer's statement that its current payload is meaningful; b0: the value named 'b0' in this block.

Line 188: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the clocked process that updates registers only on a rising clock edge.

Line 189: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 190: `if (core_output_accept) begin` This condition selects the following hardware branch only when 'core_output_accep is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are core_output_accept: the value named 'core output accept' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 191: `output_buffer_data_r <= core_m_data;` This makes output_buffer_data_r, local storage that decouples producer and consumer timing, equal to ‘core_m_data‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are core_m_data: the payload carried with the surrounding valid/ready handshake. Context: the clocked process that updates registers only on a rising clock edge.

Line 192: `output_buffer_last_r <= core_m_last;` This makes output_buffer_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘core_m_last‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are core_m_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 193: `output_buffer_overrun_r <= core_output_overrun;` This makes output_buffer_overrun_r, local storage that decouples producer and consumer timing, equal to ‘core_output_overrun‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are core_output_overrun: the value named ’core output overrun’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 194: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 195: `if (clear_status && !program_active) program_error <= 1'b0;` This condition selects the following hardware branch only when ‘clear_status && !program_active‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_status: the value named ’clear status’ in this block; program_active: the value named ’program active’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 196: `if (program_start) begin` This condition selects the following hardware branch only when ‘program_start‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are program_start: the value named ’program start’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 197: `if (program_active) begin` This condition selects the following hardware branch only when ‘program_active‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are program_active: the value named ’program active’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 198: `program_error <= 1'b1;` This makes program_error, the value named ’program error’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 199: `program_done <= 1'b1;` This makes program_done, the value named ’program done’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 200: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 201: `state <= ST_HEADER;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_HEADER‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_HEADER: the value named ’st header’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 202: `descriptor_command_r <= 128'd0;` This makes descriptor_command_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 203: `input_remaining_r <= 32'd0;` This makes input_remaining_r, the number of command payload beats still expected, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 204: `input_zero_r <= 1'b1;` This makes input_zero_r, the value named 'input zero r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 205: `input_last_r <= 1'b0;` This makes input_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 206: `output_remaining_r <= 31'd0;` This makes output_remaining_r, the number of command result beats still expected, equal to '31'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 207: `output_zero_r <= 1'b1;` This makes output_zero_r, the value named 'output zero r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 208: `output_last_r <= 1'b0;` This makes output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 209: `program_output_remaining_r <= program_output_beats;` This makes program_output_remaining_r, the number of result beats left in the complete descriptor program, equal to 'program_output_beats'. It updates after the active clock edge together with other nonblocking assignments. The important operands are program_output_beats: the value named 'program output beats' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 210: `program_output_zero_r <= program_output_beats == 0;` This makes program_output_zero_r, the value named 'program output zero r' in this block, equal to 'program_output_beats == 0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are program_output_beats: the value named 'program output beats' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 211: `program_output_last_r <= program_output_beats == 1;` This makes program_output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'program_output_beats == 1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are program_output_beats: the value named 'program output beats' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 212: `record_final_r <= 1'b0;` This makes record_final_r, the value named 'record final r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 213: `command_done_seen_r <= 1'b0;` This makes command_done_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 214: `command_error_seen_r <= 1'b0;` This makes command_error_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 215: `input_fifo_write_pointer_r <= 1'b0;` This makes input_fifo_write_pointer_r, the value named 'input fifo write pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 216: `input_fifo_read_pointer_r <= 1'b0;` This makes input_fifo_read_pointer_r, the value named 'input fifo read pointer r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 217: `input_fifo_count_r <= 2'd0;` This makes input_fifo_count_r, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 218: `output_buffer_valid_r <= 1'b0;` This makes output_buffer_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 219: `output_buffer_overrun_r <= 1'b0;` This makes output_buffer_overrun_r, local storage that decouples producer and consumer timing, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 220: `error_pending_r <= 1'b0;` This makes error_pending_r, the value named 'error pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 221: `error_input_was_last_r <= 1'b0;` This makes error_input_was_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 222: `program_active <= 1'b1;` This makes program_active, the value named 'program active' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 223: `program_error <= 1'b0;` This makes program_error, the value named 'program error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 224: `program_records <= 32'd0;` This makes program_records, the number of descriptor records accepted since program start, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 225: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 226: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 228: `if (error_pending_r) begin` This condition selects the following hardware branch only when 'error_pending_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are error_pending_r: the value named 'error pending r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 229: `error_pending_r <= 1'b0;` This makes error_pending_r, the value named 'error pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 230: `input_fifo_count_r <= 2'd0;` This makes input_fifo_count_r, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 231: `output_buffer_valid_r <= 1'b0;` This makes output_buffer_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 232: `output_buffer_overrun_r <= 1'b0;` This makes output_buffer_overrun_r, local storage that decouples producer and consumer timing, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 233: `if (error_input_was_last_r) begin` This condition selects the following hardware branch only when 'error_input_was_last_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are error_input_was_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 234: `program_active <= 1'b0;` This makes program_active, the value named 'program active' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 235: `program_done <= 1'b1;` This makes program_done, the value named 'program done' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 236: `state <= ST_HEADER;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_HEADER‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_HEADER: the value named ’st header’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 237: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 238: `state <= ST_ERROR_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_ERROR_DRAIN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ERROR_DRAIN: the value named ’st error drain’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 239: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 240: `end else if (program_active) begin` This condition selects the following hardware branch only when ‘program_active‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are program_active: the value named ’program active’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 241: `if (output_buffer_valid_r && output_buffer_overrun_r) begin` This condition selects the following hardware branch only when ‘output_buffer_valid_r && output_buffer_overrun_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_buffer_valid_r: the producer’s statement that its current payload is meaningful; output_buffer_overrun_r: local storage that decouples producer and consumer timing. Context: the clocked process that updates registers only on a rising clock edge.

Line 242: `‘NE_ENTER_PROGRAM_ERROR(record_final_r)` This SystemVerilog preprocessor line controls compilation of the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 243: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 244: `if (core_command_done) begin` This condition selects the following hardware branch only when ‘core_command_don is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are core_command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge.

Line 245: `command_done_seen_r <= 1’b1;` This makes command_done_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 246: `command_error_seen_r <= core_status_error;` This makes command_error_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘core_status_error‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are core_status_error: the value named ’core status error’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 247: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 248: `if (output_accept) begin` This condition selects the following hardware branch only when ‘output_accept‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_accept: the value named ’output accept’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 249: `if (output_buffer_last_r != output_last_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are output_buffer_last_r: the AXI-Stream marker for the final beat of a packet or command payload; output_last_r: the AXI-Stream marker for the final beat of a packet or command payload.

Line 250: `|| program_output_zero_r) begin` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are program_output_zero_r: the value named ’program output zero r’ in this block.

Line 251: `‘NE_ENTER_PROGRAM_ERROR(record_final_r)` This SystemVerilog preprocessor line controls compilation of the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 252: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 253: `output_remaining_r <= output_remaining_r - 1’b1;` This makes output_remaining_r, the number of command result beats still expected, equal to ‘output_remaining_r - 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_remaining_r: the number of command result beats still expected; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 254: `program_output_remaining_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are program_output_remaining_r: the number of result beats left in the complete descriptor program.

Line 255: `<= program_output_remaining_r - 1’b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are program_output_remaining_r: the number of result beats left in the complete descriptor program; b1: the value named ’b1’ in this block.

Line 256: `if (output_last_r) begin` This condition selects the following hardware branch only when ‘output_last_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 257: `output_zero_r <= 1’b1;` This makes output_zero_r, the value named ’output zero r’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 258: `output_last_r <= 1’b0;` This makes output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 259: `end else if (output_remaining_r == 2) begin` This condition selects the following hardware branch only when ‘output_remaining_r == 2‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_remaining_r: the number of command result beats still expected. Context: the clocked process that updates registers only on a rising clock edge.

Line 260: `output_last_r <= 1'b1;` This makes output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 261: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 262: `if (program_output_last_r) begin` This condition selects the following hardware branch only when ‘program_output_last_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are program_output_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 263: `program_output_zero_r <= 1'b1;` This makes program_output_zero_r, the value named ’program output zero r’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 264: `program_output_last_r <= 1'b0;` This makes program_output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 265: `end else if (program_output_remaining_r == 2) begin` This condition selects the following hardware branch only when ‘program_output_remaining_r == 2‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are program_output_remaining_r: the number of result beats left in the complete descriptor program. Context: the clocked process that updates registers only on a rising clock edge.

Line 266: `program_output_last_r <= 1'b1;` This makes program_output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 267: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 268: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 269: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 271: `unique case (state)` This starts a multiplexer/state decode selected by ‘state‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 272: `ST_HEADER: if (input_accept) begin` This is the ST_HEADER case branch, which accepts and validates the descriptor record header. The same-cycle action begins with ‘if (input_accept) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 273: `input_remaining_r <= s_axis_tdata[31:0];` This makes input_remaining_r, the number of command payload beats still expected, equal to ‘s_axis_tdata[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 274: `input_zero_r <= s_axis_tdata[31:0] == 0;` This makes input_zero_r, the value named ’input zero r’ in this block, equal to ‘s_axis_tdata[31:0] == 0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 275: `input_last_r <= s_axis_tdata[31:0] == 1;` This makes input_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘s_axis_tdata[31:0] == 1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 276: `output_remaining_r <= s_axis_tdata[62:32];` This makes output_remaining_r, the number of command result beats still expected, equal to ‘s_axis_tdata[62:32]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 277: `output_zero_r <= s_axis_tdata[62:32] == 0;` This makes output_zero_r, the value named ’output zero r’ in this block, equal to ‘s_axis_tdata[62:32] == 0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 278: `output_last_r <= s_axis_tdata[62:32] == 1;` This makes output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘s_axis_tdata[62:32] == 1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 279: `record_final_r <= s_axis_tdata[63];` This makes record_final_r, the value named ’record final r’ in this block, equal to ‘s_axis_tdata[63]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 280: `command_done_seen_r <= 1'b0;` This makes command_done_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 281: `command_error_seen_r <= 1'b0;` This makes command_error_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 282: `if (s_axis_tlast) 'NE_ENTER_PROGRAM_ERROR(1'b1)` This condition selects the following hardware branch only when 's_axis_tlast) 'NE_ENTER_PROGRAM_ERROR(1'b1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; NE_ENTER_PROGRAM_ERROR: the value named 'ne enter program error' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header..

Line 283: `else state <= ST_COMMAND_LOW;` Within the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header., this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMMAND_LOW: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 284: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_HEADER, the machine accepts and validates the descriptor record header.; it has no arithmetic value of its own.

Line 286: `ST_COMMAND_LOW: if (input_accept) begin` This is the ST_COMMAND_LOW case branch, which captures command bits 63 through 0. The same-cycle action begins with 'if (input_accept) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_LOW, the machine captures command bits 63 through 0..

Line 287: `descriptor_command_r[63:0] <= s_axis_tdata;` This makes descriptor_command_r[63:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 's_axis_tdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_LOW, the machine captures command bits 63 through 0..

Line 288: `if (s_axis_tlast) 'NE_ENTER_PROGRAM_ERROR(1'b1)` This condition selects the following hardware branch only when 's_axis_tlast) 'NE_ENTER_PROGRAM_ERROR(1'b1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; NE_ENTER_PROGRAM_ERROR: the value named 'ne enter program error' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_LOW, the machine captures command bits 63 through 0..

Line 289: `else state <= ST_COMMAND_HIGH;` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_LOW, the machine captures command bits 63 through 0., this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_COMMAND_HIGH: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 290: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_LOW, the machine captures command bits 63 through 0.; it has no arithmetic value of its own.

Line 292: `ST_COMMAND_HIGH: if (input_accept) begin` This is the ST_COMMAND_HIGH case branch, which captures command bits 127 through 64 and checks input TLAST. The same-cycle action begins with ‘if (input_accept) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_HIGH, the machine captures command bits 127 through 64 and checks input TLAST..

Line 293: `descriptor_command_r[127:64] <= s_axis_tdata;` This makes descriptor_command_r[127:64], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘s_axis_tdata‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_HIGH, the machine captures command bits 127 through 64 and checks input TLAST..

Line 294: `if (s_axis_tlast != (record_final_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_HIGH, the machine captures command bits 127 through 64 and checks input TLAST., this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; record_final_r: the value named ’record final r’ in this block.

Line 295: `&& input_zero_r)) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_HIGH, the machine captures command bits 127 through 64 and checks input TLAST., this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are input_zero_r: the value named ’input zero r’ in this block.

Line 296: `‘NE_ENTER_PROGRAM_ERROR(s_axis_tlast)` This SystemVerilog preprocessor line controls compilation of the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 297: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_HIGH, the machine captures command bits 127 through 64 and checks input TLAST.; it has no arithmetic value of its own.

Line 298: `state <= ST_SUBMIT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_SUBMIT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SUBMIT: the value named ’st submit’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_HIGH, the machine captures command bits 127 through 64 and checks input TLAST..

Line 299: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_HIGH, the machine captures command bits 127 through 64 and checks input TLAST.; it has no arithmetic value of its own.

Line 300: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMAND_HIGH, the machine captures command bits 127 through 64 and checks input TLAST.; it has no arithmetic value of its own.

Line 302: `ST_SUBMIT: if (core_command_ready) begin` This is the ST_SUBMIT case branch, which holds command-valid until the universal core accepts the instruction. The same-cycle action begins with ‘if (core_command_ready) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_SUBMIT, the machine holds command-valid until the universal core accepts the instruction..

Line 303: `program_records <= program_records + 1’b1;` This makes program_records, the number of descriptor records accepted since program start, equal to ‘program_records + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are program_records: the number of descriptor records accepted since program start; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBMIT, the machine holds command-valid until the universal core accepts the instruction..

Line 304: `if (input_zero_r) state <= ST_WAIT;` This condition selects the following hardware branch only when ‘input_zero_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_zero_r: the value named ’input zero r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBMIT, the machine holds command-valid until the universal core accepts the instruction..

Line 305: `else state <= ST_PAYLOAD;` Within the clocked process that updates registers only on a rising clock edge Within state ST_SUBMIT, the machine holds command-valid until the universal core accepts the instruction., this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_PAYLOAD: stream data belonging to the current command.

Line 306: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SUBMIT, the machine holds command-valid until the universal core accepts the instruction.; it has no arithmetic value of its own.

Line 308: `ST_PAYLOAD: if (input_accept) begin` This is the ST_PAYLOAD case branch, which forwards exactly the declared number of input beats to the selected engine. The same-cycle action begins with ‘if (input_accept) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine..

Line 309: `if (s_axis_tlast != (record_final_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine., this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; record_final_r: the value named ’record final r’ in this block.

Line 310: `&& input_last_r)) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine., this exact statement contributes to the block that turns a multi-record DMA program into checked core commands and payload/output streams. The important operands are input_last_r: the AXI-Stream marker for the final beat of a packet or command payload.

Line 311: `‘NE_ENTER_PROGRAM_ERROR(s_axis_tlast)` This SystemVerilog preprocessor line controls compilation of the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 312: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine.; it has no arithmetic value of its own.

Line 313: `input_remaining_r <= input_remaining_r - 1’b1;` This makes input_remaining_r, the number of command payload beats still expected, equal to ‘input_remaining_r - 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_remaining_r: the number of command payload beats still expected; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine..

Line 314: `if (input_last_r) begin` This condition selects the following hardware branch only when ‘input_last_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine..

Line 315: `input_zero_r <= 1'b1;` This makes input_zero_r, the value named 'input zero r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine..

Line 316: `input_last_r <= 1'b0;` This makes input_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine..

Line 317: `state <= ST_WAIT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_WAIT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_WAIT: the value named 'st wait' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine..

Line 318: `end else if (input_remaining_r == 2) begin` This condition selects the following hardware branch only when 'input_remaining_r == 2' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_remaining_r: the number of command payload beats still expected. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine..

Line 319: `input_last_r <= 1'b1;` This makes input_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine..

Line 320: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine.; it has no arithmetic value of its own.

Line 321: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine.; it has no arithmetic value of its own.

Line 322: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PAYLOAD, the machine forwards exactly the declared number of input beats to the selected engine.; it has no arithmetic value of its own.

Line 324: `ST_WAIT: begin` This is the ST_WAIT case branch, which waits for both command completion and the declared output count. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 325: `if (record_command_complete && record_outputs_complete) begin` This condition selects the following hardware branch only when 'record_command_complete && record_outputs_complete' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are record_command_complete: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; record_outputs_complete: the value named 'record outputs complete' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 326: `command_done_seen_r <= 1'b0;` This makes command_done_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 327: `command_error_seen_r <= 1'b0;` This makes command_error_seen_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 328: `if (command_error_seen_r) begin` This condition selects the following hardware branch only when ‘command_error_seen_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are command_error_seen_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 329: `‘NE_ENTER_PROGRAM_ERROR(record_final_r)` This SystemVerilog preprocessor line controls compilation of the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 330: `end else if (record_final_r) begin` This condition selects the following hardware branch only when ‘record_final_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are record_final_r: the value named ’record final r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 331: `if (!program_output_zero_r) begin` This condition selects the following hardware branch only when ‘!program_output_zero_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are program_output_zero_r: the value named ’program output zero r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 332: `‘NE_ENTER_PROGRAM_ERROR(1'b1)` This SystemVerilog preprocessor line controls compilation of the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 333: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 334: `program_active <= 1'b0;` This makes program_active, the value named ’program active’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 335: `program_done <= 1'b1;` This makes program_done, the value named ’program done’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 336: `state <= ST_HEADER;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_HEADER‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_HEADER: the value named ’st header’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 337: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 338: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 339: `state <= ST_HEADER;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_HEADER‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_HEADER: the value named ’st header’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count..

Line 340: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 341: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 342: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 344: `ST_ERROR_DRAIN: if (input_accept && s_axis_tlast) begin` This is the ST_ERROR_DRAIN case branch, which discards remaining DMA input until TLAST so the next transfer starts aligned. The same-cycle action begins with ‘if (input_accept && s_axis_tlast) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned..

Line 345: `program_active <= 1’b0;` This makes program_active, the value named ’program active’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned..

Line 346: `program_done <= 1’b1;` This makes program_done, the value named ’program done’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned..

Line 347: `state <= ST_HEADER;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_HEADER‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_HEADER: the value named ’st header’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned..

Line 348: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned.; it has no arithmetic value of its own.

Line 350: `default: `NE_ENTER_PROGRAM_ERROR(1'b0)` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with "NE_ENTER_PROGRAM_ERROR(1'b0)‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned..

Line 351: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ERROR_DRAIN, the machine discards remaining DMA input until TLAST so the next transfer starts aligned.; it has no arithmetic value of its own.

Line 352: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 353: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 354: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 355: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 356: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 358: `‘undef NE_ENTER_PROGRAM_ERROR` This SystemVerilog preprocessor line controls compilation of the block that turns a multi-record DMA program into checked core commands and payload/output streams.

Line 360: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that turns a multi-record DMA program into checked core commands and payload/output streams.

9 rtl/neuraedge_gradient_accumulator_bank.sv

Role in the complete system: stores one lane of exact cross-command backward gradients in inferred distributed memory.

9.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to stores one lane of exact cross-command backward gradients in inferred distributed memory.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_gradient_accumulator_bank into physical nets and primitives. The source declares 5 input-port lines, 1 output-port lines, 1 clocked process(es), and 0 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls.

Connections to other blocks This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

9.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  // Stores one lane of exact 47-bit backward-gradient partial sums.
5  module neuraedge_gradient_accumulator_bank (
6      input  wire clk,
7      input  wire write_enable,
8      input  wire [3:0] write_address,
9      input  wire signed [46:0] write_data,
10     input  wire [3:0] read_address,
11     output wire signed [46:0] read_data
12 );
13     // One asynchronous read and one synchronous write match a 7-series
14     // distributed-RAM inference template. There is intentionally no reset:
15     // the parent selects zero until a complete non-accumulating command has
16     // established valid contents for every active address.
17     (* ram_style = "distributed" *)
18     logic signed [46:0] memory [0:15];
19
20     assign read_data = memory[read_address];
21
22     always_ff @(posedge clk) begin
23         if (write_enable) memory[write_address] <= write_data;
24     end
25 endmodule
26
27 `default_nettype wire
```

9.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `// Stores one lane of exact 47-bit backward-gradient partial sums.` Design intent: Stores one lane of exact 47-bit backward-gradient partial sums. This comment anchors the surrounding logic in the block that stores one lane of exact cross-command backward gradients in inferred distributed memory.

Line 5: `module neuraedge_gradient_accumulator_bank (` This opens neuraedge_gradient_accumulator_bank, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that stores one lane of exact cross-command backward gradients in inferred distributed memory.

Line 6: `input wire clk,` This declares clk as a input port with width one bit. It carries the clock that makes all registers advance together across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 7: `input wire write_enable,` This declares write_enable as a input port with width one bit. It carries a control bit that permits the associated state or storage update across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 8: `input wire [3:0] write_address,` This declares write_address as a input port with width [3:0]. It carries the value named 'write address' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 9: `input wire signed [46:0] write_data,` This declares write_data as a input port with width [46:0]. It carries the payload carried with the surrounding valid/ready handshake across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 10: `input wire [3:0] read_address,` This declares read_address as a input port with width [3:0]. It carries the value named 'read address' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 11: `output wire signed [46:0] read_data` This declares read_data as a output port with width [46:0]. It carries the payload carried with the surrounding valid/ready handshake across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 12: `);` Within the module's structural declarations, this exact statement contributes to the block that stores one lane of exact cross-command backward gradients in inferred distributed memory. The expression is a literal or structural operator and has no hidden software state.

Line 13: `// One asynchronous read and one synchronous write match a 7-series` Design intent: One asynchronous read and one synchronous write match a 7-series This comment anchors the surrounding logic in the block that stores one lane of exact cross-command backward gradients in inferred distributed memory.

Line 14: `// distributed-RAM inference template. There is intentionally no reset:` Design intent: distributed-RAM inference template. There is intentionally no reset: This comment anchors the surrounding logic in the block that stores one lane of exact cross-command backward gradients in inferred distributed memory.

Line 15: `// the parent selects zero until a complete non-accumulating command has` Design intent: the parent selects zero until a complete non-accumulating command has This comment anchors the surrounding logic in the block that stores one lane of exact cross-command backward gradients in inferred distributed memory.

Line 16: `// established valid contents for every active address.` Design intent: established valid contents for every active address. This comment anchors the surrounding logic in the block that stores one lane of exact cross-command backward gradients in inferred distributed memory.

Line 17: `(* ram_style = "distributed" *)` Within the module's structural declarations, this exact statement contributes to the block that stores one lane of exact cross-command backward gradients in inferred distributed memory. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block.

Line 18: `logic signed [46:0] memory [0:15];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, memory is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `assign read_data = memory[read_address];` This makes read_data, the payload carried with the surrounding valid/ready handshake, equal to 'memory[read_address]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are memory: local FPGA storage addressed by surrounding index logic; read_address: the value named 'read address' in this block. Context: the module's structural declarations.

Line 22: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 23: `if (write_enable) memory[write_address] <= write_data;` This condition selects the following hardware branch only when 'write_enable' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are write_enable: a control bit that permits the associated state or storage update. Context: the clocked process that updates registers only on a rising clock edge.

Line 24: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 25: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 27: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that stores one lane of exact cross-command backward gradients in inferred distributed memory.

10 rtl/neuraedge_integer_layernorm.sv

Role in the complete system: evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

10.1 Beginner architecture map

Dataflow responsibility The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_integer_layernorm into physical nets and primitives. The source declares 12 input-port lines, 12 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_IDLE, ST_LOAD, ST_MEAN_ABSOLUTE_HIGH, ST_MEAN_A ST_MEAN_COMMIT, ST_MEAN_PREPARE, ST_MEAN_START, ST_MEAN_WAIT, ST_OUTPUT_BETA, ST_OUTPUT_CENTER, ST_OUTPUT_GAMMA_CROSS, ST_OUTPUT_GAMMA_LATCH, ST_OUTPUT_GAMM ST_OUTPUT_GAMMA_MULTIPLY, ST_OUTPUT_GAMMA_OPERANDS, ST_OUTPUT_HOLD, ST_OUTPUT_L ST_OUTPUT_NORMALIZE_ABSOLUTE and additional states. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates neuraedge_udivider as u_divider, neuraedge_isqrt64 as u_sqrt, begin as if, else as if, unique as case, end as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

10.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_integer_layernorm #(
5      parameter int MAX_FEATURES = 2048,
6      parameter int VALUE_BITS = 24
7  ) (
8      input  wire clk,
9      input  wire rst_n,
10     input  wire start_valid,
11     output logic start_ready,
12     input  wire [11:0] feature_count,
13     input  wire [31:0] epsilon,
14     input  wire sample_valid,
15     output logic sample_ready,
16     input  wire signed [VALUE_BITS-1:0] sample_value,
17     input  wire signed [VALUE_BITS-1:0] sample_gamma_q15,
18     input  wire signed [VALUE_BITS-1:0] sample_beta,
19     input  wire sample_last,
20     output logic result_valid,
21     input  wire result_ready,
22     output logic signed [31:0] result_value,
23     output logic [11:0] result_index,
24     output logic result_last,
25     output logic config_error,
26     output logic [31:0] cycle_count,
27     output logic signed [31:0] debug_mean,
28     output logic [31:0] debug_variance,
29     output logic [31:0] debug_sigma,
30     output logic [31:0] debug_reciprocal_q31
31 );
32     // The normalization datapath is deliberately deeply registered.  Each state
33     // performs one memory access, multiply, carry-chain operation, or comparison;
34     // no state hides a multi-operation arithmetic expression from timing analysis.
35     typedef enum logic [5:0] {
36         ST_IDLE = 6'd0,
37         ST_LOAD = 6'd1,
```

```
        ST_MEAN_START = 6'd2,
        ST_MEAN_WAIT = 6'd3,
        ST_VARIANCE_READ = 6'd4,
        ST_VARIANCE_CENTER = 6'd5,
        ST_VARIANCE_SQUARE = 6'd6,
        ST_VARIANCE_ACCUMULATE = 6'd7,
        ST_VARIANCE_START = 6'd8,
        ST_VARIANCE_WAIT = 6'd9,
        ST_SQRT_START = 6'd10,
        ST_SQRT_WAIT = 6'd11,
        ST_RECIPROCAL_START = 6'd12,
        ST_RECIPROCAL_WAIT = 6'd13,
        ST_OUTPUT_READ = 6'd14,
        ST_OUTPUT_CENTER = 6'd15,
        ST_OUTPUT_NORMALIZE_MULTIPLY = 6'd16,
        ST_OUTPUT_NORMALIZE_ABSOLUTE = 6'd17,
        ST_OUTPUT_NORMALIZE_ROUND = 6'd18,
        ST_OUTPUT_NORMALIZE_SIGN = 6'd19,
        ST_OUTPUT_GAMMA_MULTIPLY = 6'd20,
        ST_OUTPUT_SCALE_ABSOLUTE = 6'd21,
        ST_OUTPUT_SCALE_ROUND = 6'd22,
        ST_OUTPUT_SCALE_SIGN = 6'd23,
        ST_OUTPUT_BETA = 6'd24,
        ST_OUTPUT_SATURATE = 6'd25,
        ST_OUTPUT_HOLD = 6'd26,
        ST_VARIANCE_LATCH = 6'd27,
        ST_OUTPUT_LATCH = 6'd28,
        ST_OUTPUT_NORMALIZE_LATCH = 6'd29,
        ST_OUTPUT_GAMMA_LATCH = 6'd30,
        ST_VARIANCE_SQUARE_LATCH = 6'd31,
        ST_OUTPUT_NORMALIZE_OPERANDS = 6'd32,
        ST_OUTPUT_NORMALIZE_CLAMP = 6'd33,
        ST_OUTPUT_GAMMA_OPERANDS = 6'd34,
        ST_OUTPUT_GAMMA_CROSS = 6'd35,
        ST_OUTPUT_GAMMA_LOW = 6'd36,
        ST_MEAN_ADJUST = 6'd37,
        ST_MEAN_COMMIT = 6'd38,
        ST_VARIANCE_SQUARE_PARTIAL = 6'd39,
        ST_VARIANCE_SQUARE_ADD = 6'd40,
        ST_MEAN_PREPARE = 6'd41,
        ST_VARIANCE_ACCUMULATE_HIGH = 6'd42,
        ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH = 6'd43,
        ST_MEAN_ABSOLUTE_HIGH = 6'd44
    } state_t;

    state_t state;
    (* ram_style = "block" *) logic signed [VALUE_BITS-1:0] value_mem [0:MAX_FEATURES-1];
    (* ram_style = "block" *) logic signed [VALUE_BITS-1:0] gamma_mem [0:MAX_FEATURES-1];
    (* ram_style = "block" *) logic signed [VALUE_BITS-1:0] beta_mem [0:MAX_FEATURES-1];

    logic [11:0] feature_count_r;
    logic [31:0] epsilon_r;
    logic [11:0] load_index;
    logic [11:0] samples_remaining_r;
    logic [11:0] variance_index;
    logic [11:0] variance_remaining_r;
    logic [11:0] output_index_r;
    logic [11:0] output_remaining_r;
    logic signed [47:0] sum_r;
    logic [63:0] mean_numerator_r;
    logic [31:0] mean_absolute_low_r;
    logic mean_absolute_carry_r;
    logic [63:0] variance_sum_r;
    logic [31:0] variance_sum_low_r;
    logic variance_sum_carry_r;
    logic signed [31:0] mean_r;
    logic mean_negative_r;
    logic mean_has_remainder_r;
    logic [31:0] mean_quotient_r;
    logic [31:0] mean_magnitude_r;
    logic [31:0] variance_r;
    logic [31:0] sigma_r;
    logic [31:0] reciprocal_q31_r;

    logic signed [VALUE_BITS-1:0] output_value_r;
    logic signed [VALUE_BITS-1:0] output_gamma_r;
    logic signed [VALUE_BITS-1:0] output_beta_r;
    logic signed [VALUE_BITS-1:0] output_value_ram_r;
    logic signed [VALUE_BITS-1:0] output_gamma_ram_r;
    logic signed [VALUE_BITS-1:0] output_beta_ram_r;
    logic signed [VALUE_BITS-1:0] variance_value_ram_r;
    logic signed [VALUE_BITS-1:0] variance_value_r;
    logic signed [24:0] variance_centered_r;
    logic [23:0] variance_magnitude_r;
    logic [35:0] variance_square_low_product_r;
    logic [35:0] variance_square_high_product_r;
    logic [49:0] variance_square_r;
```

```
    logic [49:0] variance_square_latched_r;
    (* dont_touch = "yes" *) logic signed [24:0] output_centered_r;
    logic signed [24:0] normalized_centered_operand_r;
    logic [31:0] normalized_reciprocal_operand_r;
    logic signed [41:0] normalized_low_product_r;
    logic signed [41:0] normalized_high_product_r;
    logic signed [57:0] normalized_product_r;
    logic normalized_negative_r;
    logic [31:0] normalized_magnitude_low_r;
    logic normalized_magnitude_carry_r;
    logic [63:0] normalized_magnitude_r;
    logic [63:0] normalized_rounded_magnitude_r;
    logic [31:0] normalized_clamped_magnitude_r;
    (* dont_touch = "yes" *) logic signed [31:0] normalized_r;
    logic signed [31:0] gamma_normalized_operand_r;
    logic signed [VALUE_BITS-1:0] gamma_scale_operand_r;
    logic signed [33:0] gamma_partial_ll_r;
    logic signed [24:0] gamma_partial_lh_r;
    logic signed [32:0] gamma_partial_hl_r;
    logic signed [23:0] gamma_partial_hh_r;
    logic signed [33:0] gamma_cross_r;
    logic signed [49:0] gamma_low_cross_r;
    logic signed [55:0] gamma_product_r;
    logic scale_negative_r;
    logic [55:0] scale_magnitude_r;
    logic [55:0] scale_rounded_magnitude_r;
    logic signed [55:0] scaled_r;
    logic signed [63:0] scaled_beta_r;
    logic result_last_r;

    // A difference between two signed 24-bit values fits in 25 bits and its
    // magnitude fits in 24 unsigned bits. Two 24 x 12 DSP products therefore
    // form the exact square without a multi-DSP combinational cascade.
    (* use_dsp = "yes" *) wire [35:0] variance_square_low_product =
        variance_magnitude_r[23:0] * variance_magnitude_r[11:0];
    (* use_dsp = "yes" *) wire [35:0] variance_square_high_product =
        variance_magnitude_r[23:0] * variance_magnitude_r[23:12];

    // A 24-bit sample minus its mean needs only 25 signed bits. Splitting the
    // unsigned Q31 reciprocal into two 16-bit limbs lets each exact partial
    // product fit one DSP48E1 without an unregistered PCIN cascade.
    (* use_dsp = "yes" *) wire signed [41:0] normalized_low_product =
        normalized_centered_operand_r
        * $signed({1'b0, normalized_reciprocal_operand_r[15:0]});
    (* use_dsp = "yes" *) wire signed [41:0] normalized_high_product =
        normalized_centered_operand_r
        * $signed({1'b0, normalized_reciprocal_operand_r[31:16]});

    // The signed 32 x 24 gamma multiply is four exact base-2^16 limb
    // products. Low limbs are made explicitly positive by a leading zero;
    // high limbs retain the sign of the original two's-complement operand.
    wire signed [16:0] gamma_normalized_low =
        $signed({1'b0, gamma_normalized_operand_r[15:0]});
    wire signed [15:0] gamma_normalized_high =
        $signed(gamma_normalized_operand_r[31:16]);
    wire signed [16:0] gamma_scale_low =
        $signed({1'b0, gamma_scale_operand_r[15:0]});
    wire signed [7:0] gamma_scale_high =
        $signed(gamma_scale_operand_r[23:16]);
    (* use_dsp = "yes" *) wire signed [33:0] gamma_partial_ll =
        gamma_normalized_low * gamma_scale_low;
    (* use_dsp = "yes" *) wire signed [24:0] gamma_partial_lh =
        gamma_normalized_low * gamma_scale_high;
    (* use_dsp = "yes" *) wire signed [32:0] gamma_partial_hl =
        gamma_normalized_high * gamma_scale_low;
    (* use_dsp = "yes" *) wire signed [23:0] gamma_partial_hh =
        gamma_normalized_high * gamma_scale_high;

    logic divider_input_valid;
    wire divider_input_ready;
    logic [63:0] divider_numerator;
    logic [31:0] divider_denominator;
    wire divider_output_valid;
    logic divider_output_ready;
    wire [63:0] divider_quotient;
    wire [31:0] divider_remainder;
    wire divider_zero;

    logic sqrt_input_valid;
    wire sqrt_input_ready;
    logic [63:0] sqrt_radicand;
    wire sqrt_output_valid;
    logic sqrt_output_ready;
    wire [31:0] sqrt_root;

    neuraedge_udivider u_divider (
        .clk(clk), .rst_n(rst_n),
```

```
        .input_valid(divider_input_valid), .input_ready(divider_input_ready),
        .numerator(divider_numerator), .denominator(divider_denominator),
        .output_valid(divider_output_valid), .output_ready(divider_output_ready),
        .quotient(divider_quotient), .remainder(divider_remainder),
        .divide_by_zero(divider_zero)
    );

    neuraedge_isqrt64 u_sqrt (
        .clk(clk), .rst_n(rst_n),
        .input_valid(sqrt_input_valid), .input_ready(sqrt_input_ready),
        .radicand(sqrt_radicand), .output_valid(sqrt_output_valid),
        .output_ready(sqrt_output_ready), .root(sqrt_root),
        .remainder()
    );

    function automatic logic signed [31:0] saturate_s32(
        input logic signed [63:0] value
    );
        begin
            if (value[63:32] == {32{value[31]}}) saturate_s32 = value[31:0];
            else if (value[63]) saturate_s32 = 32'sh80000000;
            else saturate_s32 = 32'sh7fffffff;
        end
    endfunction

    always_comb begin
        start_ready = state == ST_IDLE;
        sample_ready = state == ST_LOAD;
        result_valid = state == ST_OUTPUT_HOLD;
        result_index = output_index_r;
        result_last = result_last_r;

        divider_input_valid = state == ST_MEAN_START
            || state == ST_VARIANCE_START || state == ST_RECIPROCAL_START;
        divider_output_ready = state == ST_MEAN_WAIT
            || state == ST_VARIANCE_WAIT || state == ST_RECIPROCAL_WAIT;
        divider_numerator = 64'd0;
        divider_denominator = {20'd0, feature_count_r};
        if (state == ST_MEAN_START) begin
            divider_numerator = mean_numerator_r;
        end else if (state == ST_VARIANCE_START) begin
            divider_numerator = variance_sum_r;
        end else if (state == ST_RECIPROCAL_START) begin
            divider_numerator = 64'd2147483647;
            divider_denominator = sigma_r;
        end

        sqrt_input_valid = state == ST_SQRT_START;
        sqrt_output_ready = state == ST_SQRT_WAIT;
        sqrt_radicand = {32'd0, variance_r} + {32'd0, epsilon_r};

        debug_mean = mean_r;
        debug_variance = variance_r;
        debug_sigma = sigma_r;
        debug_reciprocal_q31 = reciprocal_q31_r;
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            state <= ST_IDLE;
            feature_count_r <= 12'd0;
            epsilon_r <= 32'd0;
            load_index <= 12'd0;
            samples_remaining_r <= 12'd0;
            variance_index <= 12'd0;
            variance_remaining_r <= 12'd0;
            output_index_r <= 12'd0;
            output_remaining_r <= 12'd0;
            sum_r <= 48'sd0;
            mean_numerator_r <= 64'd0;
            mean_absolute_low_r <= 32'd0;
            mean_absolute_carry_r <= 1'b0;
            variance_sum_r <= 64'd0;
            variance_sum_low_r <= 32'd0;
            variance_sum_carry_r <= 1'b0;
            mean_r <= 32'sd0;
            mean_negative_r <= 1'b0;
            mean_has_remainder_r <= 1'b0;
            mean_quotient_r <= 32'd0;
            mean_magnitude_r <= 32'd0;
            variance_r <= 32'd0;
            sigma_r <= 32'd1;
            reciprocal_q31_r <= 32'd0;
            output_value_r <= '0;
            output_gamma_r <= '0;
            output_beta_r <= '0;
            output_value_ram_r <= '0;
```

```
            output_gamma_ram_r <= '0;
            output_beta_ram_r <= '0;
            variance_value_ram_r <= '0;
            variance_value_r <= '0;
            variance_centered_r <= 25'sd0;
            variance_magnitude_r <= 24'd0;
            variance_square_low_product_r <= 36'd0;
            variance_square_high_product_r <= 36'd0;
            variance_square_r <= 50'd0;
            variance_square_latched_r <= 50'd0;
            output_centered_r <= 25'sd0;
            normalized_centered_operand_r <= 25'sd0;
            normalized_reciprocal_operand_r <= 32'd0;
            normalized_low_product_r <= 42'sd0;
            normalized_high_product_r <= 42'sd0;
            normalized_product_r <= 58'sd0;
            normalized_negative_r <= 1'b0;
            normalized_magnitude_low_r <= 32'd0;
            normalized_magnitude_carry_r <= 1'b0;
            normalized_magnitude_r <= 64'd0;
            normalized_rounded_magnitude_r <= 64'd0;
            normalized_clamped_magnitude_r <= 32'd0;
            normalized_r <= 32'sd0;
            gamma_normalized_operand_r <= 32'sd0;
            gamma_scale_operand_r <= '0;
            gamma_partial_ll_r <= 34'sd0;
            gamma_partial_lh_r <= 25'sd0;
            gamma_partial_hl_r <= 33'sd0;
            gamma_partial_hh_r <= 24'sd0;
            gamma_cross_r <= 34'sd0;
            gamma_low_cross_r <= 50'sd0;
            gamma_product_r <= 56'sd0;
            scale_negative_r <= 1'b0;
            scale_magnitude_r <= 56'd0;
            scale_rounded_magnitude_r <= 56'd0;
            scaled_r <= 56'sd0;
            scaled_beta_r <= 64'sd0;
            result_last_r <= 1'b0;
            result_value <= 32'sd0;
            config_error <= 1'b0;
            cycle_count <= 32'd0;
        end else begin
            if (state != ST_IDLE) cycle_count <= cycle_count + 1'b1;
            unique case (state)
                ST_IDLE: begin
                    if (start_valid) begin
                        cycle_count <= 32'd0;
                        config_error <= feature_count == 0
                            || feature_count > 12'(MAX_FEATURES);
                        feature_count_r <= feature_count;
                        epsilon_r <= epsilon;
                        // Clearing on every accepted start keeps feature-count
                        // validation out of the 48-bit sum register enable.
                        sum_r <= 48'sd0;
                        if (feature_count == 0
                                || feature_count > 12'(MAX_FEATURES)) begin
                            state <= ST_IDLE;
                        end else begin
                            load_index <= 12'd0;
                            samples_remaining_r <= feature_count;
                            state <= ST_LOAD;
                        end
                    end
                end

                ST_LOAD: begin
                    if (sample_valid) begin
                        logic signed [47:0] sample_extended;
                        logic expected_last;
                        sample_extended = {{(48-VALUE_BITS){sample_value[VALUE_BITS-1]}}, sample_value};
                        expected_last = samples_remaining_r == 12'd1;
                        value_mem[load_index[10:0]] <= sample_value;
                        gamma_mem[load_index[10:0]] <= sample_gamma_q15;
                        beta_mem[load_index[10:0]] <= sample_beta;
                        sum_r <= sum_r + sample_extended;
                        if (sample_last != expected_last) begin
                            config_error <= 1'b1;
                        end
                        if (expected_last) begin
                            state <= ST_MEAN_PREPARE;
                        end else if (sample_last) begin
                            config_error <= 1'b1;
                            state <= ST_IDLE;
                        end else begin
                            load_index <= load_index + 1'b1;
                            samples_remaining_r <= samples_remaining_r - 1'b1;
                        end
```

```
                    end
                end

                ST_MEAN_PREPARE: begin
                    logic [32:0] absolute_low;
                    if (sum_r < 0) begin
                        absolute_low = {1'b0, ~sum_r[31:0]} + 1'b1;
                        mean_absolute_low_r <= absolute_low[31:0];
                        mean_absolute_carry_r <= absolute_low[32];
                    end else begin
                        mean_absolute_low_r <= sum_r[31:0];
                        mean_absolute_carry_r <= 1'b0;
                    end
                    state <= ST_MEAN_ABSOLUTE_HIGH;
                end

                ST_MEAN_ABSOLUTE_HIGH: begin
                    logic [15:0] absolute_high;
                    if (sum_r < 0) begin
                        absolute_high = ~sum_r[47:32]
                            + 16'(mean_absolute_carry_r);
                        mean_numerator_r <= {
                            16'd0, absolute_high, mean_absolute_low_r};
                    end else begin
                        mean_numerator_r <= {
                            16'd0, sum_r[47:32], mean_absolute_low_r};
                    end
                    state <= ST_MEAN_START;
                end

                ST_MEAN_START: begin
                    if (divider_input_ready) state <= ST_MEAN_WAIT;
                end

                ST_MEAN_WAIT: begin
                    if (divider_output_valid) begin
                        mean_negative_r <= sum_r < 0;
                        mean_has_remainder_r <= divider_remainder != 0;
                        mean_quotient_r <= divider_quotient[31:0];
                        config_error <= config_error || divider_zero;
                        state <= ST_MEAN_ADJUST;
                    end
                end

                ST_MEAN_ADJUST: begin
                    mean_magnitude_r <= mean_quotient_r
                        + (mean_negative_r && mean_has_remainder_r);
                    state <= ST_MEAN_COMMIT;
                end

                ST_MEAN_COMMIT: begin
                        mean_r <= mean_negative_r
                            ? -$signed(mean_magnitude_r)
                            : $signed(mean_magnitude_r);
                        variance_index <= 12'd0;
                        variance_remaining_r <= feature_count_r;
                        variance_sum_r <= 64'd0;
                        state <= ST_VARIANCE_READ;
                end

                ST_VARIANCE_READ: begin
                    variance_value_ram_r <= value_mem[variance_index[10:0]];
                    state <= ST_VARIANCE_LATCH;
                end

                ST_VARIANCE_LATCH: begin
                    variance_value_r <= variance_value_ram_r;
                    state <= ST_VARIANCE_CENTER;
                end

                ST_VARIANCE_CENTER: begin
                    variance_centered_r
                        <= $signed({variance_value_r[23], variance_value_r})
                            - $signed({mean_r[31], mean_r[23:0]});
                    state <= ST_VARIANCE_SQUARE;
                end

                ST_VARIANCE_SQUARE: begin
                    variance_magnitude_r <= variance_centered_r[24]
                        ? (~variance_centered_r[23:0] + 1'b1)
                        : variance_centered_r[23:0];
                    state <= ST_VARIANCE_SQUARE_PARTIAL;
                end

                ST_VARIANCE_SQUARE_PARTIAL: begin
                    variance_square_low_product_r <= variance_square_low_product;
                    variance_square_high_product_r <= variance_square_high_product;
```

```
                    state <= ST_VARIANCE_SQUARE_ADD;
                end

                ST_VARIANCE_SQUARE_ADD: begin
                    variance_square_r <= {14'd0, variance_square_low_product_r}
                        + {2'd0, variance_square_high_product_r, 12'd0};
                    state <= ST_VARIANCE_SQUARE_LATCH;
                end

                ST_VARIANCE_SQUARE_LATCH: begin
                    variance_square_latched_r <= variance_square_r;
                    state <= ST_VARIANCE_ACCUMULATE;
                end

                ST_VARIANCE_ACCUMULATE: begin
                    logic [32:0] low_sum;
                    low_sum = {1'b0, variance_sum_r[31:0]}
                        + {1'b0, variance_square_latched_r[31:0]};
                    variance_sum_low_r <= low_sum[31:0];
                    variance_sum_carry_r <= low_sum[32];
                    state <= ST_VARIANCE_ACCUMULATE_HIGH;
                end

                ST_VARIANCE_ACCUMULATE_HIGH: begin
                    logic [32:0] high_sum;
                    high_sum = {1'b0, variance_sum_r[63:32]}
                        + {1'b0, 14'd0, variance_square_latched_r[49:32]}
                        + variance_sum_carry_r;
                    variance_sum_r <= {high_sum[31:0], variance_sum_low_r};
                    if (high_sum[32]) config_error <= 1'b1;
                    if (variance_remaining_r == 12'd1) begin
                        state <= ST_VARIANCE_START;
                    end else begin
                        variance_index <= variance_index + 1'b1;
                        variance_remaining_r <= variance_remaining_r - 1'b1;
                        state <= ST_VARIANCE_READ;
                    end
                end

                ST_VARIANCE_START: begin
                    if (divider_input_ready) state <= ST_VARIANCE_WAIT;
                end

                ST_VARIANCE_WAIT: begin
                    if (divider_output_valid) begin
                        variance_r <= divider_quotient[31:0];
                        config_error <= config_error || divider_zero;
                        state <= ST_SQRT_START;
                    end
                end

                ST_SQRT_START: begin
                    if (sqrt_input_ready) state <= ST_SQRT_WAIT;
                end

                ST_SQRT_WAIT: begin
                    if (sqrt_output_valid) begin
                        sigma_r <= sqrt_root == 0 ? 32'd1 : sqrt_root;
                        state <= ST_RECIPROCAL_START;
                    end
                end

                ST_RECIPROCAL_START: begin
                    if (divider_input_ready) state <= ST_RECIPROCAL_WAIT;
                end

                ST_RECIPROCAL_WAIT: begin
                    if (divider_output_valid) begin
                        reciprocal_q31_r <= divider_quotient[31:0];
                        output_index_r <= 12'd0;
                        output_remaining_r <= feature_count_r;
                        config_error <= config_error || divider_zero;
                        state <= ST_OUTPUT_READ;
                    end
                end

                ST_OUTPUT_READ: begin
                    output_value_ram_r <= value_mem[output_index_r[10:0]];
                    output_gamma_ram_r <= gamma_mem[output_index_r[10:0]];
                    output_beta_ram_r <= beta_mem[output_index_r[10:0]];
                    state <= ST_OUTPUT_LATCH;
                end

                ST_OUTPUT_LATCH: begin
                    output_value_r <= output_value_ram_r;
                    output_gamma_r <= output_gamma_ram_r;
                    output_beta_r <= output_beta_ram_r;
```

```
560                     state <= ST_OUTPUT_CENTER;
561                 end
562
563                 ST_OUTPUT_CENTER: begin
564                     output_centered_r
565                         <= $signed({output_value_r[23], output_value_r})
566                             - $signed({mean_r[31], mean_r[23:0]});
567                     state <= ST_OUTPUT_NORMALIZE_OPERANDS;
568                 end
569
570                 ST_OUTPUT_NORMALIZE_OPERANDS: begin
571                     normalized_centered_operand_r <= output_centered_r;
572                     normalized_reciprocal_operand_r <= reciprocal_q31_r;
573                     state <= ST_OUTPUT_NORMALIZE_MULTIPLY;
574                 end
575
576                 ST_OUTPUT_NORMALIZE_MULTIPLY: begin
577                     normalized_low_product_r <= normalized_low_product;
578                     normalized_high_product_r <= normalized_high_product;
579                     state <= ST_OUTPUT_NORMALIZE_LATCH;
580                 end
581
582                 ST_OUTPUT_NORMALIZE_LATCH: begin
583                     normalized_product_r <=
584                         {{16{normalized_low_product_r[41]}}, normalized_low_product_r}
585                         + ($signed({{16{normalized_high_product_r[41]}},
586                             normalized_high_product_r}) <<< 16);
587                     state <= ST_OUTPUT_NORMALIZE_ABSOLUTE;
588                 end
589
590                 ST_OUTPUT_NORMALIZE_ABSOLUTE: begin
591                     logic [32:0] magnitude_low;
592                     normalized_negative_r <= normalized_product_r[57];
593                     magnitude_low = normalized_product_r[57]
594                         ? {1'b0, ~normalized_product_r[31:0]} + 1'b1
595                         : {1'b0, normalized_product_r[31:0]};
596                     normalized_magnitude_low_r <= magnitude_low[31:0];
597                     normalized_magnitude_carry_r <= magnitude_low[32];
598                     state <= ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH;
599                 end
600
601                 ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH: begin
602                     logic [25:0] magnitude_high;
603                     magnitude_high = normalized_negative_r
604                         ? ~normalized_product_r[57:32]
605                             + 26'(normalized_magnitude_carry_r)
606                         : normalized_product_r[57:32];
607                     normalized_magnitude_r <= {
608                         6'd0, magnitude_high, normalized_magnitude_low_r};
609                     state <= ST_OUTPUT_NORMALIZE_ROUND;
610                 end
611
612                 ST_OUTPUT_NORMALIZE_ROUND: begin
613                     normalized_rounded_magnitude_r <=
614                         (normalized_magnitude_r + 64'd1073741824) >> 31;
615                     state <= ST_OUTPUT_NORMALIZE_CLAMP;
616                 end
617
618                 ST_OUTPUT_NORMALIZE_CLAMP: begin
619                     if (normalized_negative_r) begin
620                         normalized_clamped_magnitude_r <=
621                             normalized_rounded_magnitude_r > 64'h0000000080000000
622                             ? 32'h80000000
623                             : normalized_rounded_magnitude_r[31:0];
624                     end else begin
625                         normalized_clamped_magnitude_r <=
626                             normalized_rounded_magnitude_r > 64'h000000007fffffff
627                             ? 32'h7fffffff
628                             : normalized_rounded_magnitude_r[31:0];
629                     end
630                     state <= ST_OUTPUT_NORMALIZE_SIGN;
631                 end
632
633                 ST_OUTPUT_NORMALIZE_SIGN: begin
634                     if (normalized_negative_r) begin
635                         normalized_r <= normalized_clamped_magnitude_r[31]
636                             ? 32'sh80000000
637                             : -$signed(normalized_clamped_magnitude_r);
638                     end else begin
639                         normalized_r <= $signed(normalized_clamped_magnitude_r);
640                     end
641                     state <= ST_OUTPUT_GAMMA_OPERANDS;
642                 end
643
644                 ST_OUTPUT_GAMMA_OPERANDS: begin
645                     gamma_normalized_operand_r <= normalized_r;
646                     gamma_scale_operand_r <= output_gamma_r;
```

```
                    state <= ST_OUTPUT_GAMMA_MULTIPLY;
                end

                ST_OUTPUT_GAMMA_MULTIPLY: begin
                    gamma_partial_ll_r <= gamma_partial_ll;
                    gamma_partial_lh_r <= gamma_partial_lh;
                    gamma_partial_hl_r <= gamma_partial_hl;
                    gamma_partial_hh_r <= gamma_partial_hh;
                    state <= ST_OUTPUT_GAMMA_CROSS;
                end

                ST_OUTPUT_GAMMA_CROSS: begin
                    gamma_cross_r <=
                        {{9{gamma_partial_lh_r[24]}}, gamma_partial_lh_r}
                        + {{1{gamma_partial_hl_r[32]}}, gamma_partial_hl_r};
                    state <= ST_OUTPUT_GAMMA_LOW;
                end

                ST_OUTPUT_GAMMA_LOW: begin
                    gamma_low_cross_r <=
                        {{16{gamma_partial_ll_r[33]}}, gamma_partial_ll_r}
                        + ($signed({{16{gamma_cross_r[33]}}, gamma_cross_r}) <<< 16);
                    state <= ST_OUTPUT_GAMMA_LATCH;
                end

                ST_OUTPUT_GAMMA_LATCH: begin
                    gamma_product_r <=
                        {{6{gamma_low_cross_r[49]}}, gamma_low_cross_r}
                        + ($signed({{32{gamma_partial_hh_r[23]}},
                            gamma_partial_hh_r}) <<< 32);
                    state <= ST_OUTPUT_SCALE_ABSOLUTE;
                end

                ST_OUTPUT_SCALE_ABSOLUTE: begin
                    scale_negative_r <= gamma_product_r[55];
                    scale_magnitude_r <= gamma_product_r[55]
                        ? -gamma_product_r : gamma_product_r;
                    state <= ST_OUTPUT_SCALE_ROUND;
                end

                ST_OUTPUT_SCALE_ROUND: begin
                    scale_rounded_magnitude_r <=
                        (scale_magnitude_r + 56'd16384) >> 15;
                    state <= ST_OUTPUT_SCALE_SIGN;
                end

                ST_OUTPUT_SCALE_SIGN: begin
                    scaled_r <= scale_negative_r
                        ? -$signed(scale_rounded_magnitude_r)
                        : $signed(scale_rounded_magnitude_r);
                    state <= ST_OUTPUT_BETA;
                end

                ST_OUTPUT_BETA: begin
                    scaled_beta_r <= {{8{scaled_r[55]}}, scaled_r}
                        + {{(64-VALUE_BITS){output_beta_r[VALUE_BITS-1]}}, output_beta_r};
                    state <= ST_OUTPUT_SATURATE;
                end

                ST_OUTPUT_SATURATE: begin
                    result_value <= saturate_s32(scaled_beta_r);
                    result_last_r <= output_remaining_r == 12'd1;
                    state <= ST_OUTPUT_HOLD;
                end

                ST_OUTPUT_HOLD: begin
                    if (result_ready) begin
                        if (result_last) begin
                            state <= ST_IDLE;
                        end else begin
                            output_index_r <= output_index_r + 1'b1;
                            output_remaining_r <= output_remaining_r - 1'b1;
                            state <= ST_OUTPUT_READ;
                        end
                    end
                end

                default: state <= ST_IDLE;
            endcase
        end
    end

    initial begin
        if (MAX_FEATURES < 1 || MAX_FEATURES > 2048) $fatal(1, "MAX_FEATURES must be in [1,2048]");
        if (VALUE_BITS != 24) $fatal(1, "neuraedge_integer_layernorm currently requires VALUE_BITS=24");
    end
endmodule
```

```
734
735 `default_nettype wire
```

10.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_integer_layernorm #(` This opens neuraedge_integer_layernorm, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 5: `parameter int MAX_FEATURES = 2048,` This statement continues through source line 31; the explanation uses the complete expression rather than only the first visual fragment. This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 6: `parameter int VALUE_BITS = 24` This is continuation line 2 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'parameter int VALUE_BITS = 24'. The important operands are parameter: the value named 'parameter' in this block; VALUE_BITS: the value named 'value bits' in this block. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 7: `) (` This is continuation line 3 of the 27-line logical statement that begins at source line 5. Its exact fragment is ') ('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 8: `input wire clk,` This is continuation line 4 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire clk,'. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic

start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 9: `input wire rst_n,` This is continuation line 5 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire rst_n,'. The important operands are input: the value named 'input' in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 10: `input wire start_valid,` This is continuation line 6 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire start_valid,'. The important operands are input: the value named 'input' in this block; start_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 11: `output logic start_ready,` This is continuation line 7 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic start_ready,'. The important operands are output: the value named 'output' in this block; start_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 12: `input wire [11:0] feature_count,` This is continuation line 8 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire [11:0] feature_count,'. The important operands are input: the value named 'input' in this block; feature_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic

[31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 13: `input wire [31:0] epsilon,` This is continuation line 9 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire [31:0] epsilon,'. The important operands are input: the value named 'input' in this block; epsilon: the value named 'epsilon' in this block. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 14: `input wire sample_valid,` This is continuation line 10 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire sample_valid,'. The important operands are input: the value named 'input' in this block; sample_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 15: `output logic sample_ready,` This is continuation line 11 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic sample_ready,'. The important operands are output: the value named 'output' in this block; sample_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 16: `input wire signed [VALUE_BITS-1:0] sample_value,` This is continuation line 12 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [VALUE_BITS-1:0] sample_value,'. The important operands are input: the value named 'input' in this block; VALUE_BITS: the value named 'value bits' in this block; sample_value: the value named 'sample value' in this block. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 17: `input wire signed [VALUE_BITS-1:0] sample_gamma_q15,` This is continuation line 13 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [VALUE_BITS-1:0] sample_gamma_q15,'. The important operands are input: the value named 'input' in this block; VALUE_BITS: the value named 'value bits' in this block; sample_gamma_q15: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 18: `input wire signed [VALUE_BITS-1:0] sample_beta,` This is continuation line 14 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [VALUE_BITS-1:0] sample_beta,'. The important operands are input: the value named 'input' in this block; VALUE_BITS: the value named 'value bits' in this block; sample_beta: the learned additive LayerNorm parameter. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 19: `input wire sample_last,` This is continuation line 15 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire sample_last,'. The important operands are input: the value named 'input' in this block; sample_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 20: `output logic result_valid,` This is continuation line 16 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic result_valid,'. The important operands are output: the value named 'output' in this block; result_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 21: `input wire result_ready,` This is continuation line 17 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'input wire result_ready,'. The important operands are input: the value named 'input' in this block; result_ready: the consumer's statement that it can accept a payload this cycle. Read together,

the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 22: `output logic signed [31:0] result_value,` This is continuation line 18 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic signed [31:0] result_value,'. The important operands are output: the value named 'output' in this block; result_value: the value named 'result value' in this block. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 23: `output logic [11:0] result_index,` This is continuation line 19 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic [11:0] result_index,'. The important operands are output: the value named 'output' in this block; result_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 24: `output logic result_last,` This is continuation line 20 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic result_last,'. The important operands are output: the value named 'output' in this block; result_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 25: `output logic config_error,` This is continuation line 21 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic config_error,'. The important operands are output: the value named 'output' in this block; config_error: the value named 'config error' in this block. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output

logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 26: `output logic [31:0] cycle_count,` This is continuation line 22 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] cycle_count,'. The important operands are output: the value named 'output' in this block; cycle_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 27: `output logic signed [31:0] debug_mean,` This is continuation line 23 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic signed [31:0] debug_mean,'. The important operands are output: the value named 'output' in this block; debug_mean: the arithmetic average used by normalization. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 28: `output logic [31:0] debug_variance,` This is continuation line 24 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] debug_variance,'. The important operands are output: the value named 'output' in this block; debug_variance: the average squared distance from the mean. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 29: `output logic [31:0] debug_sigma,` This is continuation line 25 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] debug_sigma,'. The important operands are output: the value named 'output' in this block; debug_sigma: the value named 'debug sigma' in this block. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 30: `output logic [31:0] debug_reciprocal_q31` This is continuation line 26 of the 27-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] debug_reciprocal_q31'. The important operands are output: the value named 'output' in this block; debug_reciprocal_q31: the value named 'debug reciprocal q31' in this block. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 31: `);` This is continuation line 27 of the 27-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes MAX_FEATURES to 2048, parameter int VALUE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [11:0] feature_count, input wire [31:0] epsilon, input wire sample_valid, output logic sample_ready, input wire signed [VALUE_BITS-1:0] sample_value, input wire signed [VALUE_BITS-1:0] sample_gamma_q15, input wire signed [VALUE_BITS-1:0] sample_beta, input wire sample_last, output logic result_valid, input wire result_ready, output logic signed [31:0] result_value, output logic [11:0] result_index, output logic result_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [31:0] debug_mean, output logic [31:0] debug_variance, output logic [31:0] debug_sigma, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_FEATURES denotes the value named 'max features' in this block.

Line 32: `// The normalization datapath is deliberately deeply registered. Each state` Design intent: The normalization datapath is deliberately deeply registered. Each state This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 33: `// performs one memory access, multiply, carry-chain operation, or comparison;` Design intent: performs one memory access, multiply, carry-chain operation, or comparison; This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 34: `// no state hides a multi-operation arithmetic expression from timing analysis.` Design intent: no state hides a multi-operation arithmetic expression from timing analysis. This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 35: `typedef enum logic [5:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 36: `ST_IDLE = 6'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 37: `ST_LOAD = 6'd1,` This assigns a fixed binary encoding to state ST_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 38: `ST_MEAN_START = 6'd2,` This assigns a fixed binary encoding to state ST_MEAN_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 39: `ST_MEAN_WAIT = 6'd3,` This assigns a fixed binary encoding to state ST_MEAN_WAIT, which waits for both command completion and the declared output count. The encoding is visible in waveforms even though the source uses the readable name.

Line 40: `ST_VARIANCE_READ = 6'd4,` This assigns a fixed binary encoding to state ST_VARIANCE_READ, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 41: `ST_VARIANCE_CENTER = 6'd5,` This assigns a fixed binary encoding to state ST_VARIANCE_CENTER, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 42: `ST_VARIANCE_SQUARE = 6'd6,` This assigns a fixed binary encoding to state ST_VARIANCE_SQUARE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 43: `ST_VARIANCE_ACCUMULATE = 6'd7,` This assigns a fixed binary encoding to state ST_VARIANCE_ACCUMULATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 44: `ST_VARIANCE_START = 6'd8,` This assigns a fixed binary encoding to state ST_VARIANCE_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 45: `ST_VARIANCE_WAIT = 6'd9,` This assigns a fixed binary encoding to state ST_VARIANCE_WAIT, which waits for both command completion and the declared output count. The encoding is visible in waveforms even though the source uses the readable name.

Line 46: `ST_SQRT_START = 6'd10,` This assigns a fixed binary encoding to state ST_SQRT_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 47: `ST_SQRT_WAIT = 6'd11,` This assigns a fixed binary encoding to state ST_SQRT_WAIT, which waits for both command completion and the declared output count. The encoding is visible in waveforms even though the source uses the readable name.

Line 48: `ST_RECIPROCAL_START = 6'd12,` This assigns a fixed binary encoding to state ST_RECIPROCAL_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 49: `ST_RECIPROCAL_WAIT = 6'd13,` This assigns a fixed binary encoding to state ST_RECIPROCAL_WAIT, which waits for both command completion and the declared output count. The encoding is visible in waveforms even though the source uses the readable name.

Line 50: `ST_OUTPUT_READ = 6'd14,` This assigns a fixed binary encoding to state ST_OUTPUT_READ, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 51: `ST_OUTPUT_CENTER = 6'd15,` This assigns a fixed binary encoding to state ST_OUTPUT_CENTER, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 52: `ST_OUTPUT_NORMALIZE_MULTIPLY = 6'd16,` This assigns a fixed binary encoding to state ST_OUTPUT_NORMALIZE which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 53: `ST_OUTPUT_NORMALIZE_ABSOLUTE = 6'd17,` This assigns a fixed binary encoding to state ST_OUTPUT_NORMALIZE which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 54: `ST_OUTPUT_NORMALIZE_ROUND = 6'd18,` This assigns a fixed binary encoding to state ST_OUTPUT_NORMALIZE_R which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 55: `ST_OUTPUT_NORMALIZE_SIGN = 6'd19,` This assigns a fixed binary encoding to state ST_OUTPUT_NORMALIZE_SI which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 56: `ST_OUTPUT_GAMMA_MULTIPLY = 6'd20,` This assigns a fixed binary encoding to state ST_OUTPUT_GAMMA_MULTI which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 57: `ST_OUTPUT_SCALE_ABSOLUTE = 6'd21,` This assigns a fixed binary encoding to state ST_OUTPUT_SCALE_ABSOLU which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 58: `ST_OUTPUT_SCALE_ROUND = 6'd22,` This assigns a fixed binary encoding to state ST_OUTPUT_SCALE_ROUND, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 59: `ST_OUTPUT_SCALE_SIGN = 6'd23,` This assigns a fixed binary encoding to state ST_OUTPUT_SCALE_SIGN, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 60: `ST_OUTPUT_BETA = 6'd24,` This assigns a fixed binary encoding to state ST_OUTPUT_BETA, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 61: `ST_OUTPUT_SATURATE = 6'd25,` This assigns a fixed binary encoding to state ST_OUTPUT_SATURATE, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 62: `ST_OUTPUT_HOLD = 6'd26,` This assigns a fixed binary encoding to state ST_OUTPUT_HOLD, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 63: `ST_VARIANCE_LATCH = 6'd27,` This assigns a fixed binary encoding to state ST_VARIANCE_LATCH, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 64: `ST_OUTPUT_LATCH = 6'd28,` This assigns a fixed binary encoding to state ST_OUTPUT_LATCH, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 65: `ST_OUTPUT_NORMALIZE_LATCH = 6'd29,` This assigns a fixed binary encoding to state ST_OUTPUT_NORMALIZE_L which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 66: `ST_OUTPUT_GAMMA_LATCH = 6'd30,` This assigns a fixed binary encoding to state ST_OUTPUT_GAMMA_LATCH, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 67: `ST_VARIANCE_SQUARE_LATCH = 6'd31,` This assigns a fixed binary encoding to state ST_VARIANCE_SQUARE_LAT which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 68: `ST_OUTPUT_NORMALIZE_OPERANDS = 6'd32,` This assigns a fixed binary encoding to state ST_OUTPUT_NORMALIZE which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 69: `ST_OUTPUT_NORMALIZE_CLAMP = 6'd33,` This assigns a fixed binary encoding to state ST_OUTPUT_NORMALIZE_C which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 70: `ST_OUTPUT_GAMMA_OPERANDS = 6'd34,` This assigns a fixed binary encoding to state ST_OUTPUT_GAMMA_OPERA which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 71: `ST_OUTPUT_GAMMA_CROSS = 6'd35,` This assigns a fixed binary encoding to state ST_OUTPUT_GAMMA_CROSS, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 72: `ST_OUTPUT_GAMMA_LOW = 6'd36,` This assigns a fixed binary encoding to state ST_OUTPUT_GAMMA_LOW, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 73: `ST_MEAN_ADJUST = 6'd37,` This assigns a fixed binary encoding to state ST_MEAN_ADJUST, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 74: `ST_MEAN_COMMIT = 6'd38,` This assigns a fixed binary encoding to state ST_MEAN_COMMIT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 75: `ST_VARIANCE_SQUARE_PARTIAL = 6'd39,` This assigns a fixed binary encoding to state ST_VARIANCE_SQUARE_PA which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 76: `ST_VARIANCE_SQUARE_ADD = 6'd40,` This assigns a fixed binary encoding to state ST_VARIANCE_SQUARE_ADD, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 77: `ST_MEAN_PREPARE = 6'd41,` This assigns a fixed binary encoding to state ST_MEAN_PREPARE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 78: `ST_VARIANCE_ACCUMULATE_HIGH = 6'd42,` This assigns a fixed binary encoding to state ST_VARIANCE_ACCUMULA which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 79: `ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH = 6'd43,` This assigns a fixed binary encoding to state ST_OUTPUT_NORMA which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 80: `ST_MEAN_ABSOLUTE_HIGH = 6'd44` This statement continues through source line 81; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_MEAN_ABSOLUTE_HIGH, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 81: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 80. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_MEAN_ABSOLUTE_HIGH, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 83: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 84: `(* ram_style = "block" *) logic signed [VALUE_BITS-1:0] value_mem [0:MAX_FEATURES-1];` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are ram_style: the value named 'ram style' in this block; block: the value named 'block' in this block; VALUE_BITS: the value named 'value bits' in this block.

Line 85: `(* ram_style = "block" *) logic signed [VALUE_BITS-1:0] gamma_mem [0:MAX_FEATURES-1];` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are ram_style: the value named 'ram style' in this block; block: the value named 'block' in this block; VALUE_BITS: the value named 'value bits' in this block.

Line 86: `(* ram_style = "block" *) logic signed [VALUE_BITS-1:0] beta_mem [0:MAX_FEATURES-1];` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are ram_style: the value named 'ram style' in this block; block: the value named 'block' in this block; VALUE_BITS: the value named 'value bits' in this block.

Line 88: `logic [11:0] feature_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, feature_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 89: `logic [31:0] epsilon_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, epsilon_r is the value named 'epsilon r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 90: `logic [11:0] load_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 91: `logic [11:0] samples_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, samples_remaining_r is the value named 'samples remaining r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 92: `logic [11:0] variance_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_index is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 93: `logic [11:0] variance_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_remaining_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 94: `logic [11:0] output_index_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_index_r is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 95: `logic [11:0] output_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_remaining_r is the number of command result beats still expected. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 96: `logic signed [47:0] sum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sum_r is the value named 'sum r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 97: `logic [63:0] mean_numerator_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mean_numerator_r is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 98: `logic [31:0] mean_absolute_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mean_absolute_low_r is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 99: `logic mean_absolute_carry_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mean_absolute_carry_r is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 100: `logic [63:0] variance_sum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_sum_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 101: `logic [31:0] variance_sum_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_sum_low_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 102: `logic variance_sum_carry_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_sum_carry_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 103: `logic signed [31:0] mean_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mean_r is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 104: `logic mean_negative_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mean_negative_r is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 105: `logic mean_has_remainder_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mean_has_remainder_r is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 106: `logic [31:0] mean_quotient_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mean_quotient_r is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 107: `logic [31:0] mean_magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mean_magnitude_r is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 108: `logic [31:0] variance_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 109: `logic [31:0] sigma_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sigma_r is the value named 'sigma r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 110: `logic [31:0] reciprocal_q31_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reciprocal_q31_r is the value named 'reciprocal q31 r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 112: `logic signed [VALUE_BITS-1:0] output_value_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_value_r is the value named 'output value r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 113: `logic signed [VALUE_BITS-1:0] output_gamma_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_gamma_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 114: `logic signed [VALUE_BITS-1:0] output_beta_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_beta_r is the learned additive LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 115: `logic signed [VALUE_BITS-1:0] output_value_ram_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_value_ram_r is the value named 'output value ram r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 116: `logic signed [VALUE_BITS-1:0] output_gamma_ram_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_gamma_ram_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 117: `logic signed [VALUE_BITS-1:0] output_beta_ram_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_beta_ram_r is the learned additive LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 118: `logic signed [VALUE_BITS-1:0] variance_value_ram_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_value_ram_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 119: `logic signed [VALUE_BITS-1:0] variance_value_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_value_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 120: `logic signed [24:0] variance_centered_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_centered_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 121: `logic [23:0] variance_magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_magnitude_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 122: `logic [35:0] variance_square_low_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_square_low_product_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 123: `logic [35:0] variance_square_high_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_square_high_product_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 124: `logic [49:0] variance_square_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_square_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 125: `logic [49:0] variance_square_latched_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, variance_square_latched_r is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 126: `(* dont_touch = "yes" *) logic signed [24:0] output_centered_r;` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; output_centered_r: the value named 'output centered r' in this block.

Line 127: `logic signed [24:0] normalized_centered_operand_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_centered_operand_r is the value named 'normalized centered operand r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 128: `logic [31:0] normalized_reciprocal_operand_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_reciprocal_operand_r is the value named 'normalized reciprocal operand r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 129: `logic signed [41:0] normalized_low_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_low_product_r is the value named 'normalized low product r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 130: `logic signed [41:0] normalized_high_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_high_product_r is the value named 'normalized high product r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 131: `logic signed [57:0] normalized_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_product_r is the value named 'normalized product r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 132: `logic normalized_negative_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_negative_r is the value named 'normalized negative r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 133: `logic [31:0] normalized_magnitude_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_magnitude_low_r is the value named 'normalized magnitude low r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 134: `logic normalized_magnitude_carry_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_magnitude_carry_r is the value named 'normalized magnitude carry r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 135: `logic [63:0] normalized_magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_magnitude_r is the value named 'normalized magnitude r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 136: `logic [63:0] normalized_rounded_magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_rounded_magnitude_r is the value named 'normalized rounded magnitude r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 137: `logic [31:0] normalized_clamped_magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, normalized_clamped_magnitude_r is the value named 'normalized clamped magnitude r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 138: `(* dont_touch = "yes" *) logic signed [31:0] normalized_r;` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; normalized_r: the value named 'normalized r' in this block.

Line 139: `logic signed [31:0] gamma_normalized_operand_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_normalized_operand_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 140: `logic signed [VALUE_BITS-1:0] gamma_scale_operand_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_scale_operand_r is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 141: `logic signed [33:0] gamma_partial_ll_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_partial_ll_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 142: `logic signed [24:0] gamma_partial_lh_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_partial_lh_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 143: `logic signed [32:0] gamma_partial_hl_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_partial_hl_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 144: `logic signed [23:0] gamma_partial_hh_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_partial_hh_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 145: `logic signed [33:0] gamma_cross_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_cross_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 146: `logic signed [49:0] gamma_low_cross_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_low_cross_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 147: `logic signed [55:0] gamma_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_product_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 148: `logic scale_negative_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, scale_negative_r is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 149: `logic [55:0] scale_magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, scale_magnitude_r is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 150: `logic [55:0] scale_rounded_magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, scale_rounded_magnitude_r is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 151: `logic signed [55:0] scaled_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, scaled_r is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 152: `logic signed [63:0] scaled_beta_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, scaled_beta_r is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 153: `logic result_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 155: `// A difference between two signed 24-bit values fits in 25 bits and its` Design intent: A difference between two signed 24-bit values fits in 25 bits and its This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 156: `// magnitude fits in 24 unsigned bits. Two 24 x 12 DSP products therefore` Design intent: magnitude fits in 24 unsigned bits. Two 24 x 12 DSP products therefore This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 157: `// form the exact square without a multi-DSP combinational cascade.` Design intent: form the exact square without a multi-DSP combinational cascade. This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 158: `(* use_dsp = "yes" *) wire [35:0] variance_square_low_product =` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; variance_square_low_product: the average squared distance from the mean.

Line 159: `variance_magnitude_r[23:0] * variance_magnitude_r[11:0];` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are variance_magnitude_r: the average squared distance from the mean.

Line 160: `(* use_dsp = "yes" *) wire [35:0] variance_square_high_product =` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; variance_square_high_product: the average squared distance from the mean.

Line 161: `variance_magnitude_r[23:0] * variance_magnitude_r[23:12];` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are variance_magnitude_r: the average squared distance from the mean.

Line 163: `// A 24-bit sample minus its mean needs only 25 signed bits. Splitting the` Design intent: A 24-bit sample minus its mean needs only 25 signed bits. Splitting the This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 164: `// unsigned Q31 reciprocal into two 16-bit limbs lets each exact partial` Design intent: unsigned Q31 reciprocal into two 16-bit limbs lets each exact partial This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 165: `// product fit one DSP48E1 without an unregistered PCIN cascade.` Design intent: product fit one DSP48E1 without an unregistered PCIN cascade. This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 166: `(* use_dsp = "yes" *) wire signed [41:0] normalized_low_product =` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; normalized_low_product: the value named 'normalized low product' in this block.

Line 167: `normalized_centered_operand_r` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are normalized_centered_operand_r: the value named 'normalized centered operand r' in this block.

Line 168: `* $signed({1'b0, normalized_reciprocal_operand_r[15:0]});` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are b0: the value named 'b0' in this block; normalized_reciprocal_operand_r: the value named 'normalized reciprocal operand r' in this block.

Line 169: `(* use_dsp = "yes" *) wire signed [41:0] normalized_high_product =` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; normalized_high_product: the value named 'normalized high product' in this block.

Line 170: `normalized_centered_operand_r` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are normalized_centered_operand_r: the value named 'normalized centered operand r' in this block.

Line 171: `* $signed({1'b0, normalized_reciprocal_operand_r[31:16]});` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are b0: the value named 'b0' in this block; normalized_reciprocal_operand_r: the value named 'normalized reciprocal operand r' in this block.

Line 173: `// The signed 32 x 24 gamma multiply is four exact base-2^16 limb` Design intent: The signed 32 x 24 gamma multiply is four exact base-2^16 limb This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 174: `// products. Low limbs are made explicitly positive by a leading zero;` Design intent: products. Low limbs are made explicitly positive by a leading zero; This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 175: `// high limbs retain the sign of the original two's-complement operand.` Design intent: high limbs retain the sign of the original two's-complement operand. This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 176: `wire signed [16:0] gamma_normalized_low =` This statement continues through source line 177; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_normalized_low is the learned multiplicative LayerNorm parameter; gamma_normalized_operand_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 177: `$signed({1'b0, gamma_normalized_operand_r[15:0]});` This is continuation line 2 of the 2-line logical statement that begins at source line 176. Its exact fragment is '$signed({1'b0, gamma_normalized_operand_r[15:0]});'. The important operands are b0: the value named 'b0' in this block; gamma_normalized_operand_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_normalized_low is the learned multiplicative LayerNorm parameter; gamma_normalized_operand_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 178: `wire signed [15:0] gamma_normalized_high =` This statement continues through source line 179; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_normalized_high is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 179: `$signed(gamma_normalized_operand_r[31:16]);` This is continuation line 2 of the 2-line logical statement that begins at source line 178. Its exact fragment is '$signed(gamma_normalized_operand_r[31:16]);'. The important operands are gamma_normalized_operand_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically,

gamma_normalized_high is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 180: `wire signed [16:0] gamma_scale_low =` This statement continues through source line 181; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_scale_low is the real-value spacing represented by one quantized integer step; gamma_scale_operand_r is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 181: `$signed({1'b0, gamma_scale_operand_r[15:0]});` This is continuation line 2 of the 2-line logical statement that begins at source line 180. Its exact fragment is '$signed({1'b0, gamma_scale_operand_r[15:0]});'. The important operands are b0: the value named 'b0' in this block; gamma_scale_operand_r: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_scale_low is the real-value spacing represented by one quantized integer step; gamma_scale_operand_r is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 182: `wire signed [7:0] gamma_scale_high =` This statement continues through source line 183; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_scale_high is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 183: `$signed(gamma_scale_operand_r[23:16]);` This is continuation line 2 of the 2-line logical statement that begins at source line 182. Its exact fragment is '$signed(gamma_scale_operand_r[23:16]);'. The important operands are gamma_scale_operand_r: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gamma_scale_high is the real-value spacing represented by one quantized integer step. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 184: `(* use_dsp = "yes" *) wire signed [33:0] gamma_partial_ll =` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; gamma_partial_ll: the learned multiplicative LayerNorm parameter.

Line 185: `gamma_normalized_low * gamma_scale_low;` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are gamma_normalized_low: the learned multiplicative LayerNorm parameter; gamma_scale_low: the real-value spacing represented by one quantized integer step.

Line 186: `(* use_dsp = "yes" *) wire signed [24:0] gamma_partial_lh =` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; gamma_partial_lh: the learned multiplicative LayerNorm parameter.

Line 187: `gamma_normalized_low * gamma_scale_high;` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are gamma_normalized_low: the learned multiplicative LayerNorm parameter; gamma_scale_high: the real-value spacing represented by one quantized integer step.

Line 188: `(* use_dsp = "yes" *) wire signed [32:0] gamma_partial_hl =` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are use_dsp: the value named

'use dsp' in this block; yes: the value named 'yes' in this block; gamma_partial_hl: the learned multiplicative LayerNorm parameter.

Line 189: `gamma_normalized_high * gamma_scale_low;` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are gamma_normalized_high: the learned multiplicative LayerNorm parameter; gamma_scale_low: the real-value spacing represented by one quantized integer step.

Line 190: `(* use_dsp = "yes" *) wire signed [23:0] gamma_partial_hh =` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; gamma_partial_hh: the learned multiplicative LayerNorm parameter.

Line 191: `gamma_normalized_high * gamma_scale_high;` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are gamma_normalized_high: the learned multiplicative LayerNorm parameter; gamma_scale_high: the real-value spacing represented by one quantized integer step.

Line 193: `logic divider_input_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 194: `wire divider_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 195: `logic [63:0] divider_numerator;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_numerator is the value named 'divider numerator' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 196: `logic [31:0] divider_denominator;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_denominator is the value named 'divider denominator' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 197: `wire divider_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 198: `logic divider_output_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 199: `wire [63:0] divider_quotient;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_quotient is the value named 'divider quotient' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 200: `wire [31:0] divider_remainder;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_remainder is the value named 'divider remainder' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 201: `wire divider_zero;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_zero is the value named 'divider zero' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 203: `logic sqrt_input_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 204: `wire sqrt_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 205: `logic [63:0] sqrt_radicand;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_radicand is the value named 'sqrt radicand' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 206: `wire sqrt_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 207: `logic sqrt_output_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 208: `wire [31:0] sqrt_root;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_root is the value named 'sqrt root' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 210: `neuraedge_udivider u_divider (` This instantiates module neuraedge_udivider as u_divider. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 211: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 212: `.input_valid(divider_input_valid), .input_ready(divider_input_ready),` This connects the child instance's named port 'input_valid' to parent expression 'divider_input_valid), .input_ready(divider_input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 213: `.numerator(divider_numerator), .denominator(divider_denominator),` This connects the child instance's named port 'numerator' to parent expression 'divider_numerator), .denominator(divider_denominator'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'divider numerator), .denominator(divider denominator' in this block.

Line 214: `.output_valid(divider_output_valid), .output_ready(divider_output_ready),` This connects the child instance's named port 'output_valid' to parent expression 'divider_output_valid), .output_ready(divider_output_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 215: `.quotient(divider_quotient), .remainder(divider_remainder),` This connects the child instance's named port 'quotient' to parent expression 'divider_quotient), .remainder(divider_remainder'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'divider quotient), .remainder(divider remainder' in this block.

Line 216: `.divide_by_zero(divider_zero)` This connects the child instance's named port 'divide_by_zero' to parent expression 'divider_zero'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'divider zero' in this block.

Line 217: `);` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The expression is a literal or structural operator and has no hidden software state.

Line 219: `neuraedge_isqrt64 u_sqrt (` This instantiates module neuraedge_isqrt64 as u_sqrt. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 220: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 221: `.input_valid(sqrt_input_valid), .input_ready(sqrt_input_ready),` This connects the child instance's named port 'input_valid' to parent expression 'sqrt_input_valid), .input_ready(sqrt_input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 222: `.radicand(sqrt_radicand), .output_valid(sqrt_output_valid),` This connects the child instance's named port 'radicand' to parent expression 'sqrt_radicand), .output_valid(sqrt_output_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 223: `.output_ready(sqrt_output_ready), .root(sqrt_root),` This connects the child instance's named port 'output_ready' to parent expression 'sqrt_output_ready), .root(sqrt_root'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 224: `.remainder()` This connects the child instance's named port 'remainder' to parent expression ''. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'remainder' in this block.

Line 225: `);` Within the module's structural declarations, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The expression is a literal or structural operator and has no hidden software state.

Line 227: `function automatic logic signed [31:0] saturate_s32(` Within the combinational helper saturate s32, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_s32: the value named 'saturate s32' in this block.

Line 228: `input logic signed [63:0] value` This declares value as a input port with width [63:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 229: `);` Within the combinational helper saturate s32, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The expression is a literal or structural operator and has no hidden software state.

Line 230: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32; it has no arithmetic value of its own.

Line 231: `if (value[63:32] == {32{value[31]}}) saturate_s32 = value[31:0];` This condition selects the following hardware branch only when 'value[63:32] == {32{value[31]}}' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the combinational helper saturate s32.

Line 232: `else if (value[63]) saturate_s32 = 32'sh80000000;` This condition selects the following hardware branch only when ‘value[63]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the combinational helper saturate s32.

Line 233: `else saturate_s32 = 32'sh7fffffff;` Within the combinational helper saturate s32, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are saturate_s32: the value named 'saturate s32' in this block; sh7fffffff: the value named 'sh7fffffff' in this block.

Line 234: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32; it has no arithmetic value of its own.

Line 235: `endfunction` Within the combinational helper saturate s32, this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are endfunction: the value named 'endfunction' in this block.

Line 237: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 238: `start_ready = state == ST_IDLE;` This makes start_ready, the consumer's statement that it can accept a payload this cycle, equal to ‘state == ST_IDLE‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 239: `sample_ready = state == ST_LOAD;` This makes sample_ready, the consumer's statement that it can accept a payload this cycle, equal to ‘state == ST_LOAD‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD: the value named 'st load' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 240: `result_valid = state == ST_OUTPUT_HOLD;` This makes result_valid, the producer's statement that its current payload is meaningful, equal to ‘state == ST_OUTPUT_HOLD‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT_HOLD: the value named 'st output hold' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 241: `result_index = output_index_r;` This makes result_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 242: `result_last = result_last_r;` This makes result_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘result_last_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 244: `divider_input_valid = state == ST_MEAN_START` This statement continues through source line 245; the explanation uses the complete expression rather than only the first visual fragment. This makes divider_input_valid, the producer's statement that its current payload is meaningful, equal to ‘state == ST_MEAN_START || state == ST_VARIANCE_START || state == ST_RECIPROCAL_START‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_MEAN_START: the arithmetic average used by normalization; ST_VARIANCE_START: the average squared distance from the mean. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 245: `|| state == ST_VARIANCE_START || state == ST_RECIPROCAL_START;` This is continuation line 2 of the 2-line logical statement that begins at source line 244. Its exact fragment is '|| state == ST_VARIANCE_START || state == ST_RECIPROCAL_START;'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_VARIANCE_START: the average squared distance from the mean; ST_RECIPROCAL_START: the value named 'st reciprocal start' in this block. Read together, the complete operation is: This makes divider_input_valid, the producer's statement that its current payload is meaningful, equal to 'state == ST_MEAN_START || state == ST_VARIANCE_START || state == ST_RECIPROCAL_START'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_MEAN_START: the arithmetic average used by normalization; ST_VARIANCE_START: the average squared distance from the mean. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 246: `divider_output_ready = state == ST_MEAN_WAIT` This statement continues through source line 247; the explanation uses the complete expression rather than only the first visual fragment. This makes divider_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_MEAN_WAIT || state == ST_VARIANCE_WAIT || state == ST_RECIPROCAL_WAIT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_MEAN_WAIT: the arithmetic average used by normalization; ST_VARIANCE_WAIT: the average squared distance from the mean. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 247: `|| state == ST_VARIANCE_WAIT || state == ST_RECIPROCAL_WAIT;` This is continuation line 2 of the 2-line logical statement that begins at source line 246. Its exact fragment is '|| state == ST_VARIANCE_WAIT || state == ST_RECIPROCAL_WAIT;'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_VARIANCE_WAIT: the average squared distance from the mean; ST_RECIPROCAL_WAIT: the value named 'st reciprocal wait' in this block. Read together, the complete operation is: This makes divider_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_MEAN_WAIT || state == ST_VARIANCE_WAIT || state == ST_RECIPROCAL_WAIT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_MEAN_WAIT: the arithmetic average used by normalization; ST_VARIANCE_WAIT: the average squared distance from the mean. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 248: `divider_numerator = 64'd0;` This makes divider_numerator, the value named 'divider numerator' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 249: `divider_denominator = {20'd0, feature_count_r};` This makes divider_denominator, the value named 'divider denominator' in this block, equal to '{20'd0, feature_count_r}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block; feature_count_r: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 250: `if (state == ST_MEAN_START) begin` This condition selects the following hardware branch only when 'state == ST_MEAN_START' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_MEAN_START: the arithmetic average used by normalization. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 251: `divider_numerator = mean_numerator_r;` This makes divider_numerator, the value named 'divider numerator' in this block, equal to 'mean_numerator_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are mean_numerator_r: the arithmetic average used by normalization. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 252: `end else if (state == ST_VARIANCE_START) begin` This condition selects the following hardware branch only when 'state == ST_VARIANCE_START' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in

flip-flops; ST_VARIANCE_START: the average squared distance from the mean. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 253: `divider_numerator = variance_sum_r;` This makes divider_numerator, the value named 'divider numerator' in this block, equal to 'variance_sum_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are variance_sum_r: the average squared distance from the mean. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 254: `end else if (state == ST_RECIPROCAL_START) begin` This condition selects the following hardware branch only when 'state == ST_RECIPROCAL_START' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_RECIPROCAL_START: the value named 'st reciprocal start' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 255: `divider_numerator = 64'd2147483647;` This makes divider_numerator, the value named 'divider numerator' in this block, equal to '64'd2147483647'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d2147483647: the value named 'd2147483647' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 256: `divider_denominator = sigma_r;` This makes divider_denominator, the value named 'divider denominator' in this block, equal to 'sigma_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sigma_r: the value named 'sigma r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 257: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 259: `sqrt_input_valid = state == ST_SQRT_START;` This makes sqrt_input_valid, the producer's statement that its current payload is meaningful, equal to 'state == ST_SQRT_START'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_SQRT_START: the value named 'st sqrt start' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 260: `sqrt_output_ready = state == ST_SQRT_WAIT;` This makes sqrt_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_SQRT_WAIT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_SQRT_WAIT: the value named 'st sqrt wait' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 261: `sqrt_radicand = {32'd0, variance_r} + {32'd0, epsilon_r};` This makes sqrt_radicand, the value named 'sqrt radicand' in this block, equal to '{32'd0, variance_r} + {32'd0, epsilon_r}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block; variance_r: the average squared distance from the mean; epsilon_r: the value named 'epsilon r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 263: `debug_mean = mean_r;` This makes debug_mean, the arithmetic average used by normalization, equal to 'mean_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are mean_r: the arithmetic average used by normalization. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 264: `debug_variance = variance_r;` This makes debug_variance, the average squared distance from the mean, equal to 'variance_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are variance_r: the average squared distance from the mean. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 265: `debug_sigma = sigma_r;` This makes debug_sigma, the value named 'debug sigma' in this block, equal to 'sigma_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sigma_r: the value named 'sigma r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 266: `debug_reciprocal_q31 = reciprocal_q31_r;` This makes debug_reciprocal_q31, the value named 'debug reciprocal q31' in this block, equal to 'reciprocal_q31_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are reciprocal_q31_r: the value named 'reciprocal q31 r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 267: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 269: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 270: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 271: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 272: `feature_count_r <= 12'd0;` This makes feature_count_r, an integer count retained for control or measurable telemetry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 273: `epsilon_r <= 32'd0;` This makes epsilon_r, the value named 'epsilon r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 274: `load_index <= 12'd0;` This makes load_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 275: `samples_remaining_r <= 12'd0;` This makes samples_remaining_r, the value named 'samples remaining r' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 276: `variance_index <= 12'd0;` This makes variance_index, the average squared distance from the mean, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 277: `variance_remaining_r <= 12'd0;` This makes variance_remaining_r, the average squared distance from the mean, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 278: `output_index_r <= 12'd0;` This makes output_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 279: `output_remaining_r <= 12'd0;` This makes output_remaining_r, the number of command result beats still expected, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 280: `sum_r <= 48'sd0;` This makes sum_r, the value named 'sum r' in this block, equal to '48'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 281: `mean_numerator_r <= 64'd0;` This makes mean_numerator_r, the arithmetic average used by normalization, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 282: `mean_absolute_low_r <= 32'd0;` This makes mean_absolute_low_r, the arithmetic average used by normalization, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 283: `mean_absolute_carry_r <= 1'b0;` This makes mean_absolute_carry_r, the arithmetic average used by normalization, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 284: `variance_sum_r <= 64'd0;` This makes variance_sum_r, the average squared distance from the mean, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 285: `variance_sum_low_r <= 32'd0;` This makes variance_sum_low_r, the average squared distance from the mean, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 286: `variance_sum_carry_r <= 1'b0;` This makes variance_sum_carry_r, the average squared distance from the mean, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 287: `mean_r <= 32'sd0;` This makes mean_r, the arithmetic average used by normalization, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 288: `mean_negative_r <= 1'b0;` This makes mean_negative_r, the arithmetic average used by normalization, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 289: `mean_has_remainder_r <= 1'b0;` This makes mean_has_remainder_r, the arithmetic average used by normalization, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 290: `mean_quotient_r <= 32'd0;` This makes mean_quotient_r, the arithmetic average used by normalization, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 291: `mean_magnitude_r <= 32'd0;` This makes mean_magnitude_r, the arithmetic average used by normalization, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 292: `variance_r <= 32'd0;` This makes variance_r, the average squared distance from the mean, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 293: `sigma_r <= 32'd1;` This makes sigma_r, the value named 'sigma r' in this block, equal to ‘32'd1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 294: `reciprocal_q31_r <= 32'd0;` This makes reciprocal_q31_r, the value named 'reciprocal q31 r' in this block, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 295: `output_value_r <= '0;` This makes output_value_r, the value named 'output value r' in this block, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 296: `output_gamma_r <= '0;` This makes output_gamma_r, the learned multiplicative LayerNorm parameter, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 297: `output_beta_r <= '0;` This makes output_beta_r, the learned additive LayerNorm parameter, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 298: `output_value_ram_r <= '0;` This makes output_value_ram_r, the value named 'output value ram r' in this block, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 299: `output_gamma_ram_r <= '0;` This makes output_gamma_ram_r, the learned multiplicative LayerNorm parameter, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 300: `output_beta_ram_r <= '0;` This makes output_beta_ram_r, the learned additive LayerNorm parameter, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 301: `variance_value_ram_r <= '0;` This makes variance_value_ram_r, the average squared distance from the mean, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 302: `variance_value_r <= '0;` This makes variance_value_r, the average squared distance from the mean, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 303: `variance_centered_r <= 25'sd0;` This makes variance_centered_r, the average squared distance from the mean, equal to ‘25'sd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 304: `variance_magnitude_r <= 24'd0;` This makes variance_magnitude_r, the average squared distance from the mean, equal to ‘24'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 305: `variance_square_low_product_r <= 36'd0;` This makes variance_square_low_product_r, the average squared distance from the mean, equal to ‘36'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 306: `variance_square_high_product_r <= 36'd0;` This makes variance_square_high_product_r, the average squared distance from the mean, equal to ‘36'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 307: `variance_square_r <= 50'd0;` This makes variance_square_r, the average squared distance from the mean, equal to ‘50'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 308: `variance_square_latched_r <= 50'd0;` This makes variance_square_latched_r, the average squared distance from the mean, equal to ‘50'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 309: `output_centered_r <= 25'sd0;` This makes output_centered_r, the value named 'output centered r' in this block, equal to ‘25'sd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 310: `normalized_centered_operand_r <= 25'sd0;` This makes normalized_centered_operand_r, the value named 'normalized centered operand r' in this block, equal to ‘25'sd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 311: `normalized_reciprocal_operand_r <= 32'd0;` This makes normalized_reciprocal_operand_r, the value named 'normalized reciprocal operand r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 312: `normalized_low_product_r <= 42'sd0;` This makes normalized_low_product_r, the value named 'normalized low product r' in this block, equal to '42'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 313: `normalized_high_product_r <= 42'sd0;` This makes normalized_high_product_r, the value named 'normalized high product r' in this block, equal to '42'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 314: `normalized_product_r <= 58'sd0;` This makes normalized_product_r, the value named 'normalized product r' in this block, equal to '58'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 315: `normalized_negative_r <= 1'b0;` This makes normalized_negative_r, the value named 'normalized negative r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 316: `normalized_magnitude_low_r <= 32'd0;` This makes normalized_magnitude_low_r, the value named 'normalized magnitude low r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 317: `normalized_magnitude_carry_r <= 1'b0;` This makes normalized_magnitude_carry_r, the value named 'normalized magnitude carry r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 318: `normalized_magnitude_r <= 64'd0;` This makes normalized_magnitude_r, the value named 'normalized magnitude r' in this block, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 319: `normalized_rounded_magnitude_r <= 64'd0;` This makes normalized_rounded_magnitude_r, the value named 'normalized rounded magnitude r' in this block, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 320: `normalized_clamped_magnitude_r <= 32'd0;` This makes normalized_clamped_magnitude_r, the value named 'normalized clamped magnitude r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 321: `normalized_r <= 32'sd0;` This makes normalized_r, the value named 'normalized r' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 322: `gamma_normalized_operand_r <= 32'sd0;` This makes gamma_normalized_operand_r, the learned multiplicative LayerNorm parameter, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 323: `gamma_scale_operand_r <= '0;` This makes gamma_scale_operand_r, the real-value spacing represented by one quantized integer step, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 324: `gamma_partial_ll_r <= 34'sd0;` This makes gamma_partial_ll_r, the learned multiplicative LayerNorm parameter, equal to '34'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 325: `gamma_partial_lh_r <= 25'sd0;` This makes gamma_partial_lh_r, the learned multiplicative LayerNorm parameter, equal to '25'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 326: `gamma_partial_hl_r <= 33'sd0;` This makes gamma_partial_hl_r, the learned multiplicative LayerNorm parameter, equal to '33'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 327: `gamma_partial_hh_r <= 24'sd0;` This makes gamma_partial_hh_r, the learned multiplicative LayerNorm parameter, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 328: `gamma_cross_r <= 34'sd0;` This makes gamma_cross_r, the learned multiplicative LayerNorm parameter, equal to '34'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 329: `gamma_low_cross_r <= 50'sd0;` This makes gamma_low_cross_r, the learned multiplicative LayerNorm parameter, equal to '50'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 330: `gamma_product_r <= 56'sd0;` This makes gamma_product_r, the learned multiplicative LayerNorm parameter, equal to '56'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 331: `scale_negative_r <= 1'b0;` This makes scale_negative_r, the real-value spacing represented by one quantized integer step, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 332: `scale_magnitude_r <= 56'd0;` This makes scale_magnitude_r, the real-value spacing represented by one quantized integer step, equal to '56'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 333: `scale_rounded_magnitude_r <= 56'd0;` This makes scale_rounded_magnitude_r, the real-value spacing represented by one quantized integer step, equal to '56'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 334: `scaled_r <= 56'sd0;` This makes scaled_r, the real-value spacing represented by one quantized integer step, equal to '56'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 335: `scaled_beta_r <= 64'sd0;` This makes scaled_beta_r, the real-value spacing represented by one quantized integer step, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 336: `result_last_r <= 1'b0;` This makes result_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 337: `result_value <= 32'sd0;` This makes result_value, the value named 'result value' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 338: `config_error <= 1'b0;` This makes config_error, the value named 'config error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 339: `cycle_count <= 32'd0;` This makes cycle_count, an integer count retained for control or measurable telemetry, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 340: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 341: `if (state != ST_IDLE) cycle_count <= cycle_count + 1'b1;` This condition selects the following hardware branch only when 'state != ST_IDLE' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 342: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 343: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 344: `if (start_valid) begin` This condition selects the following hardware branch only when 'start_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are start_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 345: `cycle_count <= 32'd0;` This makes cycle_count, an integer count retained for control or measurable telemetry, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 346: `config_error <= feature_count == 0` This statement continues through source line 347; the explanation uses the complete expression rather than only the first visual fragment. This makes config_error, the value named 'config error' in this block, equal to 'feature_count == 0 || feature_count > 12'(MAX_FEATURES)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are feature_count: an integer count retained for control or measurable telemetry; MAX_FEATURES: the value named 'max features' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 347: `|| feature_count > 12'(MAX_FEATURES);` This is continuation line 2 of the 2-line logical statement that begins at source line 346. Its exact fragment is '|| feature_count > 12'(MAX_FEATURES);'. The important operands are feature_count: an integer count retained for control or measurable telemetry; MAX_FEATURES: the value named 'max features' in this block. Read together, the complete operation is: This makes config_error, the value named 'config error' in this block, equal to 'feature_count == 0 || feature_count > 12'(MAX_FEATURES)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are feature_count: an integer count retained for control or measurable telemetry; MAX_FEATURES: the value named 'max features' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 348: `feature_count_r <= feature_count;` This makes feature_count_r, an integer count retained for control or measurable telemetry, equal to 'feature_count'. It updates after the active clock edge together with other nonblocking assignments. The important operands are feature_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 349: `epsilon_r <= epsilon;` This makes epsilon_r, the value named 'epsilon r' in this block, equal to 'epsilon'. It updates after the active clock edge together with other nonblocking assignments. The important operands are epsilon: the value named 'epsilon' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 350: `// Clearing on every accepted start keeps feature-count` Design intent: Clearing on every accepted start keeps feature-count This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 351: `// validation out of the 48-bit sum register enable.` Design intent: validation out of the 48-bit sum register enable. This comment anchors the surrounding logic in the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

Line 352: `sum_r <= 48'sd0;` This makes sum_r, the value named 'sum r' in this block, equal to '48'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 353: `if (feature_count == 0` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are feature_count: an integer count retained for control or measurable telemetry.

Line 354: `|| feature_count > 12'(MAX_FEATURES)) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are feature_count: an integer count retained for control or measurable telemetry; MAX_FEATURES: the value named 'max features' in this block.

Line 355: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 356: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 357: `load_index <= 12'd0;` This makes load_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 358: `samples_remaining_r <= feature_count;` This makes samples_remaining_r, the value named 'samples remaining r' in this block, equal to 'feature_count'. It updates after the active clock edge together with other nonblocking assignments. The important operands are feature_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 359: `state <= ST_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LOAD'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD: the value named 'st load' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 360: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 361: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 362: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 364: `ST_LOAD: begin` This is the ST_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 365: `if (sample_valid) begin` This condition selects the following hardware branch only when 'sample_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sample_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 366: `logic signed [47:0] sample_extended;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.. Specifically, sample_extended is the value named 'sample extended' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 367: `logic expected_last;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.. Specifically, expected_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 368: `sample_extended = {{(48-VALUE_BITS){sample_value[VALUE_BITS-1]}}, sample_value};` This makes sample_extended, the value named 'sample extended' in this block, equal to '{{(48-VALUE_BITS){sample_value[VALUE_BITS-1]}}, sample_value}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are VALUE_BITS: the value named 'value bits' in this block; sample_value: the value named 'sample value' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 369: `expected_last = samples_remaining_r == 12'd1;` This makes expected_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'samples_remaining_r == 12'd1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are samples_remaining_r: the value named 'samples remaining r' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 370: `value_mem[load_index[10:0]] <= sample_value;` This makes value_mem[load_index[10:0]], local FPGA storage addressed by surrounding index logic, equal to 'sample_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sample_value: the value named 'sample value' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 371: `gamma_mem[load_index[10:0]] <= sample_gamma_q15;` This makes gamma_mem[load_index[10:0]], the learned multiplicative LayerNorm parameter, equal to 'sample_gamma_q15'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sample_gamma_q15: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 372: `beta_mem[load_index[10:0]] <= sample_beta;` This makes beta_mem[load_index[10:0]], the learned additive LayerNorm parameter, equal to 'sample_beta'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sample_beta: the learned additive LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 373: `sum_r <= sum_r + sample_extended;` This makes sum_r, the value named 'sum r' in this block, equal to 'sum_r + sample_extended'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sum_r: the value named 'sum r' in this block; sample_extended: the value named 'sample extended' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 374: `if (sample_last != expected_last) begin` This condition selects the following hardware branch only when 'sample_last != expected_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sample_last: the AXI-Stream marker for the final beat of a packet or command payload; expected_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 375: `config_error <= 1'b1;` This makes config_error, the value named 'config error' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 376: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 377: `if (expected_last) begin` This condition selects the following hardware branch only when 'expected_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are expected_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 378: `state <= ST_MEAN_PREPARE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MEAN_PREPARE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MEAN_PREPARE: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 379: `end else if (sample_last) begin` This condition selects the following hardware branch only when 'sample_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sample_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 380: `config_error <= 1'b1;` This makes config_error, the value named 'config error' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 381: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 382: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 383: `load_index <= load_index + 1'b1;` This makes load_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'load_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 384: `samples_remaining_r <= samples_remaining_r - 1'b1;` This makes samples_remaining_r, the value named 'samples remaining r' in this block, equal to 'samples_remaining_r - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are samples_remaining_r: the value named 'samples remaining r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 385: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 386: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 387: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 389: `ST_MEAN_PREPARE: begin` This is the ST_MEAN_PREPARE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase..

Line 390: `logic [32:0] absolute_low;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase.. Specifically, absolute_low is the value named ’absolute low’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 391: `if (sum_r < 0) begin` This condition selects the following hardware branch only when ‘sum_r $<$ 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sum_r: the value named ’sum r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase..

Line 392: `absolute_low = {1'b0, ~sum_r[31:0]} + 1'b1;` This makes absolute_low, the value named ’absolute low’ in this block, equal to ‘{1’b0, ~sum_r[31:0]} + 1’b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block; sum_r: the value named ’sum r’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase..

Line 393: `mean_absolute_low_r <= absolute_low[31:0];` This makes mean_absolute_low_r, the arithmetic average used by normalization, equal to ‘absolute_low[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are absolute_low: the value named ’absolute low’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase..

Line 394: `mean_absolute_carry_r <= absolute_low[32];` This makes mean_absolute_carry_r, the arithmetic average used by normalization, equal to ‘absolute_low[32]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are absolute_low: the value named ’absolute low’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase..

Line 395: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 396: `mean_absolute_low_r <= sum_r[31:0];` This makes mean_absolute_low_r, the arithmetic average used by normalization, equal to ‘sum_r[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are sum_r: the value named ’sum r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase..

Line 397: `mean_absolute_carry_r <= 1'b0;` This makes mean_absolute_carry_r, the arithmetic average used by normalization, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase..

Line 398: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 399: `state <= ST_MEAN_ABSOLUTE_HIGH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_MEAN_ABSOLUTE_HIGH‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MEAN_ABSOLUTE_HIGH: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase..

Line 400: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_PREPARE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 402: `ST_MEAN_ABSOLUTE_HIGH: begin` This is the ST_MEAN_ABSOLUTE_HIGH case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase..

Line 403: `logic [15:0] absolute_high;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase.. Specifically, absolute_high is the value named 'absolute high' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 404: `if (sum_r < 0) begin` This condition selects the following hardware branch only when ‘sum_r < 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sum_r: the value named 'sum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase..

Line 405: `absolute_high = ~sum_r[47:32]` This statement continues through source line 406; the explanation uses the complete expression rather than only the first visual fragment. This makes absolute_high, the value named 'absolute high' in this block, equal to ‘~sum_r[47:32] + 16'(mean_absolute_carry_r)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are sum_r: the value named 'sum r' in this block; mean_absolute_carry_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase..

Line 406: `+ 16'(mean_absolute_carry_r);` This is continuation line 2 of the 2-line logical statement that begins at source line 405. Its exact fragment is ‘+ 16'(mean_absolute_carry_r);‘. The important operands are mean_absolute_carry_r: the arithmetic average used by normalization. Read together, the complete operation is: This makes absolute_high, the value named 'absolute high' in this block, equal to ‘~sum_r[47:32] + 16'(mean_absolute_carry_r)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are sum_r: the value named 'sum r' in this block; mean_absolute_carry_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase..

Line 407: `mean_numerator_r <= {` This statement continues through source line 408; the explanation uses the complete expression rather than only the first visual fragment. This makes mean_numerator_r, the arithmetic average used by normalization, equal to ‘{ 16'd0, absolute_high, mean_absolute_low_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this

block; absolute_high: the value named 'absolute high' in this block; mean_absolute_low_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase..

Line 408: `16'd0, absolute_high, mean_absolute_low_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 407. Its exact fragment is '16'd0, absolute_high, mean_absolute_low_r};'. The important operands are d0: the value named 'd0' in this block; absolute_high: the value named 'absolute high' in this block; mean_absolute_low_r: the arithmetic average used by normalization. Read together, the complete operation is: This makes mean_numerator_r, the arithmetic average used by normalization, equal to '{ 16'd0, absolute_high, mean_absolute_low_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; absolute_high: the value named 'absolute high' in this block; mean_absolute_low_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase..

Line 409: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 410: `mean_numerator_r <= {` This statement continues through source line 411; the explanation uses the complete expression rather than only the first visual fragment. This makes mean_numerator_r, the arithmetic average used by normalization, equal to '{ 16'd0, sum_r[47:32], mean_absolute_low_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; sum_r: the value named 'sum r' in this block; mean_absolute_low_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase..

Line 411: `16'd0, sum_r[47:32], mean_absolute_low_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 410. Its exact fragment is '16'd0, sum_r[47:32], mean_absolute_low_r};'. The important operands are d0: the value named 'd0' in this block; sum_r: the value named 'sum r' in this block; mean_absolute_low_r: the arithmetic average used by normalization. Read together, the complete operation is: This makes mean_numerator_r, the arithmetic average used by normalization, equal to '{ 16'd0, sum_r[47:32], mean_absolute_low_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; sum_r: the value named 'sum r' in this block; mean_absolute_low_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase..

Line 412: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 413: `state <= ST_MEAN_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MEAN_START'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MEAN_START: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase..

Line 414: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ABSOLUTE_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 416: `ST_MEAN_START: begin` This is the ST_MEAN_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_START, the machine performs the named control phase..

Line 417: `if (divider_input_ready) state <= ST_MEAN_WAIT;` This condition selects the following hardware branch only when ‘divider_input_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are divider_input_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_START, the machine performs the named control phase..

Line 418: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_START, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 420: `ST_MEAN_WAIT: begin` This is the ST_MEAN_WAIT case branch, which waits for both command completion and the declared output count. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_WAIT, the machine waits for both command completion and the declared output count..

Line 421: `if (divider_output_valid) begin` This condition selects the following hardware branch only when ‘divider_output_valid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are divider_output_valid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_WAIT, the machine waits for both command completion and the declared output count..

Line 422: `mean_negative_r <= sum_r < 0;` This makes mean_negative_r, the arithmetic average used by normalization, equal to ‘sum_r $<$ 0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are sum_r: the value named ’sum r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_WAIT, the machine waits for both command completion and the declared output count..

Line 423: `mean_has_remainder_r <= divider_remainder != 0;` This makes mean_has_remainder_r, the arithmetic average used by normalization, equal to ‘divider_remainder != 0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are divider_remainder: the value named ’divider remainder’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_WAIT, the machine waits for both command completion and the declared output count..

Line 424: `mean_quotient_r <= divider_quotient[31:0];` This makes mean_quotient_r, the arithmetic average used by normalization, equal to ‘divider_quotient[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are divider_quotient: the value named ’divider quotient’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_WAIT, the machine waits for both command completion and the declared output count..

Line 425: `config_error <= config_error || divider_zero;` This makes config_error, the value named ’config error’ in this block, equal to ‘config_error || divider_zero‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are config_error: the value named ’config error’ in this block; divider_zero: the value named ’divider zero’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_WAIT, the machine waits for both command completion and the declared output count..

Line 426: `state <= ST_MEAN_ADJUST;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_MEAN_ADJUST‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MEAN_ADJUST: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_WAIT, the machine waits for both command completion and the declared output count..

Line 427: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 428: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 430: `ST_MEAN_ADJUST: begin` This is the ST_MEAN_ADJUST case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ADJUST, the machine performs the named control phase..

Line 431: `mean_magnitude_r <= mean_quotient_r` This statement continues through source line 432; the explanation uses the complete expression rather than only the first visual fragment. This makes mean_magnitude_r, the arithmetic average used by normalization, equal to ‘mean_quotient_r + (mean_negative_r && mean_has_remainder_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are mean_quotient_r: the arithmetic average used by normalization; mean_negative_r: the arithmetic average used by normalization; mean_has_remainder_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ADJUST, the machine performs the named control phase..

Line 432: `+ (mean_negative_r && mean_has_remainder_r);` This is continuation line 2 of the 2-line logical statement that begins at source line 431. Its exact fragment is ‘+ (mean_negative_r && mean_has_remainder_r);‘. The important operands are mean_negative_r: the arithmetic average used by normalization; mean_has_remainder_r: the arithmetic average used by normalization. Read together, the complete operation is: This makes mean_magnitude_r, the arithmetic average used by normalization, equal to ‘mean_quotient_r + (mean_negative_r && mean_has_remainder_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are mean_quotient_r: the arithmetic average used by normalization; mean_negative_r: the arithmetic average used by normalization; mean_has_remainder_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ADJUST, the machine performs the named control phase..

Line 433: `state <= ST_MEAN_COMMIT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_MEAN_COMMIT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MEAN_COMMIT: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ADJUST, the machine performs the named control phase..

Line 434: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_ADJUST, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 436: `ST_MEAN_COMMIT: begin` This is the ST_MEAN_COMMIT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_COMMIT, the machine performs the named control phase..

Line 437: `mean_r <= mean_negative_r` This statement continues through source line 439; the explanation uses the complete expression rather than only the first visual fragment. This makes mean_r, the arithmetic average used by normalization, equal to ‘mean_negative_r ? -$signed(mean_magnitude_r) : $signed(mean_magnitude_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are mean_negative_r: the arithmetic average used by normalization; mean_magnitude_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_COMMIT, the machine performs the named control phase..

Line 438: `? -$signed(mean_magnitude_r)` This is continuation line 2 of the 3-line logical statement that begins at source line 437. Its exact fragment is ‘? -$signed(mean_magnitude_r)‘. The important operands are mean_magnitude_r: the arithmetic average used by normalization. Read together, the complete operation is: This makes mean_r, the arithmetic average used by normalization, equal to ‘mean_negative_r ? -$signed(mean_magnitude_r) : $signed(mean_magnitude_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are

mean_negative_r: the arithmetic average used by normalization; mean_magnitude_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_COMMIT, the machine performs the named control phase..

Line 439: `: $signed(mean_magnitude_r);` This is continuation line 3 of the 3-line logical statement that begins at source line 437. Its exact fragment is ': $signed(mean_magnitude_r);'. The important operands are mean_magnitude_r: the arithmetic average used by normalization. Read together, the complete operation is: This makes mean_r, the arithmetic average used by normalization, equal to 'mean_negative_r ? -$signed(mean_magnitude_r) : $signed(mean_magnitude_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mean_negative_r: the arithmetic average used by normalization; mean_magnitude_r: the arithmetic average used by normalization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_COMMIT, the machine performs the named control phase..

Line 440: `variance_index <= 12'd0;` This makes variance_index, the average squared distance from the mean, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_COMMIT, the machine performs the named control phase..

Line 441: `variance_remaining_r <= feature_count_r;` This makes variance_remaining_r, the average squared distance from the mean, equal to 'feature_count_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are feature_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_COMMIT, the machine performs the named control phase..

Line 442: `variance_sum_r <= 64'd0;` This makes variance_sum_r, the average squared distance from the mean, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_COMMIT, the machine performs the named control phase..

Line 443: `state <= ST_VARIANCE_READ;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_VARIANCE_READ'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_READ: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_COMMIT, the machine performs the named control phase..

Line 444: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MEAN_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 446: `ST_VARIANCE_READ: begin` This is the ST_VARIANCE_READ case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_READ, the machine performs the named control phase..

Line 447: `variance_value_ram_r <= value_mem[variance_index[10:0]];` This makes variance_value_ram_r, the average squared distance from the mean, equal to 'value_mem[variance_index[10:0]]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are value_mem: local FPGA storage addressed by surrounding index logic; variance_index: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_READ, the machine performs the named control phase..

Line 448: `state <= ST_VARIANCE_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_VARIANCE_LATCH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_LATCH: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_READ, the machine performs the named control phase..

Line 449: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_READ, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 451: `ST_VARIANCE_LATCH: begin` This is the ST_VARIANCE_LATCH case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_LATCH, the machine performs the named control phase..

Line 452: `variance_value_r <= variance_value_ram_r;` This makes variance_value_r, the average squared distance from the mean, equal to 'variance_value_ram_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are variance_value_ram_r: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_LATCH, the machine performs the named control phase..

Line 453: `state <= ST_VARIANCE_CENTER;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_VARIANCE_CENTER'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_CENTER: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_LATCH, the machine performs the named control phase..

Line 454: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_LATCH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 456: `ST_VARIANCE_CENTER: begin` This is the ST_VARIANCE_CENTER case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_CENTER, the machine performs the named control phase..

Line 457: `variance_centered_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_CENTER, the machine performs the named control phase., this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are variance_centered_r: the average squared distance from the mean.

Line 458: `<= $signed({variance_value_r[23], variance_value_r})` Within the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_CENTER, the machine performs the named control phase., this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are variance_value_r: the average squared distance from the mean.

Line 459: `- $signed({mean_r[31], mean_r[23:0]});` Within the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_CENTER, the machine performs the named control phase., this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are mean_r: the arithmetic average used by normalization.

Line 460: `state <= ST_VARIANCE_SQUARE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_VARIANCE_SQUARE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_SQUARE: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_CENTER, the machine performs the named control phase..

Line 461: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_CENTER, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 463: `ST_VARIANCE_SQUARE: begin` This is the ST_VARIANCE_SQUARE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE, the machine performs the named control phase..

Line 464: `variance_magnitude_r <= variance_centered_r[24]` This statement continues through source line 466; the explanation uses the complete expression rather than only the first visual fragment. This makes variance_magnitude_r, the average squared distance from the mean, equal to ‘variance_centered_r[24] ? (~variance_centered_r[23:0] + 1’b1) : variance_centered_r[23:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are variance_centered_r: the average squared distance from the mean; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE, the machine performs the named control phase..

Line 465: `? (~variance_centered_r[23:0] + 1'b1)` This is continuation line 2 of the 3-line logical statement that begins at source line 464. Its exact fragment is ‘? (~variance_centered_r[23:0] + 1’b1)‘. The important operands are variance_centered_r: the average squared distance from the mean; b1: the value named ’b1’ in this block. Read together, the complete operation is: This makes variance_magnitude_r, the average squared distance from the mean, equal to ‘variance_centered_r[24] ? (~variance_centered_r[23:0] + 1’b1) : variance_centered_r[23:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are variance_centered_r: the average squared distance from the mean; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE, the machine performs the named control phase..

Line 466: `: variance_centered_r[23:0];` This is continuation line 3 of the 3-line logical statement that begins at source line 464. Its exact fragment is ‘: variance_centered_r[23:0];‘. The important operands are variance_centered_r: the average squared distance from the mean. Read together, the complete operation is: This makes variance_magnitude_r, the average squared distance from the mean, equal to ‘variance_centered_r[24] ? (~variance_centered_r[23:0] + 1’b1) : variance_centered_r[23:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are variance_centered_r: the average squared distance from the mean; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE, the machine performs the named control phase..

Line 467: `state <= ST_VARIANCE_SQUARE_PARTIAL;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VARIANCE_SQUARE_PARTIAL‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_SQUARE_PARTIAL: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE, the machine performs the named control phase..

Line 468: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 470: `ST_VARIANCE_SQUARE_PARTIAL: begin` This is the ST_VARIANCE_SQUARE_PARTIAL case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_PARTIAL, the machine performs the named control phase..

Line 471: `variance_square_low_product_r <= variance_square_low_product;` This makes variance_square_low_product_
the average squared distance from the mean, equal to ‘variance_square_low_product‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are variance_square_low_product: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_PARTIAL, the machine performs the named control phase..

Line 472: `variance_square_high_product_r <= variance_square_high_product;` This makes variance_square_high_produ
the average squared distance from the mean, equal to ‘variance_square_high_product‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are variance_square_high_product: the

average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_PARTIAL, the machine performs the named control phase..

Line 473: `state <= ST_VARIANCE_SQUARE_ADD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VARIANCE_SQUARE_ADD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_SQUARE_ADD: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_PARTIAL, the machine performs the named control phase..

Line 474: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_PARTIAL, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 476: `ST_VARIANCE_SQUARE_ADD: begin` This is the ST_VARIANCE_SQUARE_ADD case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_ADD, the machine performs the named control phase..

Line 477: `variance_square_r <= {14'd0, variance_square_low_product_r}` This statement continues through source line 478; the explanation uses the complete expression rather than only the first visual fragment. This makes variance_square_r, the average squared distance from the mean, equal to ‘{14’d0, variance_square_low_product_r} + {2’d0, variance_square_high_product_r, 12’d0}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; variance_square_low_product_r: the average squared distance from the mean; variance_square_high_product_r: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_ADD, the machine performs the named control phase..

Line 478: `+ {2'd0, variance_square_high_product_r, 12'd0};` This is continuation line 2 of the 2-line logical statement that begins at source line 477. Its exact fragment is ‘+ {2’d0, variance_square_high_product_r, 12’d0};‘. The important operands are d0: the value named ’d0’ in this block; variance_square_high_product_r: the average squared distance from the mean. Read together, the complete operation is: This makes variance_square_r, the average squared distance from the mean, equal to ‘{14’d0, variance_square_low_product_r} + {2’d0, variance_square_high_product_r, 12’d0}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; variance_square_low_product_r: the average squared distance from the mean; variance_square_high_product_r: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_ADD, the machine performs the named control phase..

Line 479: `state <= ST_VARIANCE_SQUARE_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VARIANCE_SQUARE_LATCH‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_SQUARE_LATCH: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_ADD, the machine performs the named control phase..

Line 480: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_ADD, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 482: `ST_VARIANCE_SQUARE_LATCH: begin` This is the ST_VARIANCE_SQUARE_LATCH case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_LATCH, the machine performs the named control phase..

Line 483: `variance_square_latched_r <= variance_square_r;` This makes variance_square_latched_r, the average squared distance from the mean, equal to ‘variance_square_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are variance_square_r: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_LAT the machine performs the named control phase..

Line 484: `state <= ST_VARIANCE_ACCUMULATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VARIANCE_ACCUMULATE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_ACCUMULATE: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_LATCH, the machine performs the named control phase..

Line 485: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_SQUARE_LATCH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 487: `ST_VARIANCE_ACCUMULATE: begin` This is the ST_VARIANCE_ACCUMULATE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE, the machine performs the named control phase..

Line 488: `logic [32:0] low_sum;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE, the machine performs the named control phase.. Specifically, low_sum is the value named ’low sum’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 489: `low_sum = {1'b0, variance_sum_r[31:0]}` This statement continues through source line 490; the explanation uses the complete expression rather than only the first visual fragment. This makes low_sum, the value named ’low sum’ in this block, equal to ‘{1’b0, variance_sum_r[31:0]} + {1’b0, variance_square_latched_r[31:0]}‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block; variance_sum_r: the average squared distance from the mean; variance_square_latched_r: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE, the machine performs the named control phase..

Line 490: `+ {1'b0, variance_square_latched_r[31:0]};` This is continuation line 2 of the 2-line logical statement that begins at source line 489. Its exact fragment is ‘+ {1’b0, variance_square_latched_r[31:0]};‘. The important operands are b0: the value named ’b0’ in this block; variance_square_latched_r: the average squared distance from the mean. Read together, the complete operation is: This makes low_sum, the value named ’low sum’ in this block, equal to ‘{1’b0, variance_sum_r[31:0]} + {1’b0, variance_square_latched_r[31:0]}‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block; variance_sum_r: the average squared distance from the mean; variance_square_latched_r: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE, the machine performs the named control phase..

Line 491: `variance_sum_low_r <= low_sum[31:0];` This makes variance_sum_low_r, the average squared distance from the mean, equal to ‘low_sum[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are low_sum: the value named ’low sum’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE, the machine performs the named control phase..

Line 492: `variance_sum_carry_r <= low_sum[32];` This makes variance_sum_carry_r, the average squared distance from the mean, equal to ‘low_sum[32]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are low_sum: the value named ’low sum’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE, the machine performs the named control phase..

Line 493: `state <= ST_VARIANCE_ACCUMULATE_HIGH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VARIANCE_ACCUMULATE_HIGH‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_ACCUMULATE_HIGH: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE, the machine performs the named control phase..

Line 494: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 496: `ST_VARIANCE_ACCUMULATE_HIGH: begin` This is the ST_VARIANCE_ACCUMULATE_HIGH case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 497: `logic [32:0] high_sum;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase.. Specifically, high_sum is the value named ’high sum’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 498: `high_sum = {1'b0, variance_sum_r[63:32]}` This statement continues through source line 500; the explanation uses the complete expression rather than only the first visual fragment. This makes high_sum, the value named ’high sum’ in this block, equal to ‘{1’b0, variance_sum_r[63:32]} + {1’b0, 14’d0, variance_square_latched_r[49:32]} + variance_sum_carry_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block; variance_sum_r: the average squared distance from the mean; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 499: `+ {1'b0, 14'd0, variance_square_latched_r[49:32]}` This is continuation line 2 of the 3-line logical statement that begins at source line 498. Its exact fragment is ‘+ {1’b0, 14’d0, variance_square_latched_r[49:32]}‘. The important operands are b0: the value named ’b0’ in this block; d0: the value named ’d0’ in this block; variance_square_latched_r: the average squared distance from the mean. Read together, the complete operation is: This makes high_sum, the value named ’high sum’ in this block, equal to ‘{1’b0, variance_sum_r[63:32]} + {1’b0, 14’d0, variance_square_latched_r[49:32]} + variance_sum_carry_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block; variance_sum_r: the average squared distance from the mean; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 500: `+ variance_sum_carry_r;` This is continuation line 3 of the 3-line logical statement that begins at source line 498. Its exact fragment is ‘+ variance_sum_carry_r;‘. The important operands are variance_sum_carry_r: the average squared distance from the mean. Read together, the complete operation is: This makes high_sum, the value named ’high sum’ in this block, equal to ‘{1’b0, variance_sum_r[63:32]} + {1’b0, 14’d0, variance_square_latched_r[49:32]} + variance_sum_carry_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block; variance_sum_r: the average squared distance from the mean; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 501: `variance_sum_r <= {high_sum[31:0], variance_sum_low_r};` This makes variance_sum_r, the average squared distance from the mean, equal to ‘{high_sum[31:0], variance_sum_low_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are high_sum: the value named ’high sum’ in this block; variance_sum_low_r: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 502: `if (high_sum[32]) config_error <= 1'b1;` This condition selects the following hardware branch only when ‘high_sum[32]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are high_sum: the value named 'high sum' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 503: `if (variance_remaining_r == 12'd1) begin` This condition selects the following hardware branch only when ‘variance_remaining_r == 12'd1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are variance_remaining_r: the average squared distance from the mean; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 504: `state <= ST_VARIANCE_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VARIANCE_START‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_START: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 505: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 506: `variance_index <= variance_index + 1'b1;` This makes variance_index, the average squared distance from the mean, equal to ‘variance_index + 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are variance_index: the average squared distance from the mean; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 507: `variance_remaining_r <= variance_remaining_r - 1'b1;` This makes variance_remaining_r, the average squared distance from the mean, equal to ‘variance_remaining_r - 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are variance_remaining_r: the average squared distance from the mean; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 508: `state <= ST_VARIANCE_READ;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VARIANCE_READ‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VARIANCE_READ: the average squared distance from the mean. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase..

Line 509: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 510: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_ACCUMULATE_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 512: `ST_VARIANCE_START: begin` This is the ST_VARIANCE_START case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_START, the machine performs the named control phase..

Line 513: `if (divider_input_ready) state <= ST_VARIANCE_WAIT;` This condition selects the following hardware branch only when ‘divider_input_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are divider_input_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_START, the machine performs the named control phase..

Line 514: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_START, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 516: `ST_VARIANCE_WAIT: begin` This is the ST_VARIANCE_WAIT case branch, which waits for both command completion and the declared output count. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_WAIT, the machine waits for both command completion and the declared output count..

Line 517: `if (divider_output_valid) begin` This condition selects the following hardware branch only when ‘divider_output_valid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are divider_output_valid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_WAIT, the machine waits for both command completion and the declared output count..

Line 518: `variance_r <= divider_quotient[31:0];` This makes variance_r, the average squared distance from the mean, equal to ‘divider_quotient[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are divider_quotient: the value named ’divider quotient’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_WAIT, the machine waits for both command completion and the declared output count..

Line 519: `config_error <= config_error || divider_zero;` This makes config_error, the value named ’config error’ in this block, equal to ‘config_error || divider_zero‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are config_error: the value named ’config error’ in this block; divider_zero: the value named ’divider zero’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_WAIT, the machine waits for both command completion and the declared output count..

Line 520: `state <= ST_SQRT_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_SQRT_START‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SQRT_START: the value named ’st sqrt start’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_WAIT, the machine waits for both command completion and the declared output count..

Line 521: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 522: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VARIANCE_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 524: `ST_SQRT_START: begin` This is the ST_SQRT_START case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_SQRT_START, the machine performs the named control phase..

Line 525: `if (sqrt_input_ready) state <= ST_SQRT_WAIT;` This condition selects the following hardware branch only when ‘sqrt_input_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sqrt_input_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SQRT_START, the machine performs the named control phase..

Line 526: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SQRT_START, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 528: `ST_SQRT_WAIT: begin` This is the ST_SQRT_WAIT case branch, which waits for both command completion and the declared output count. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_SQRT_WAIT, the machine waits for both command completion and the declared output count..

Line 529: `if (sqrt_output_valid) begin` This condition selects the following hardware branch only when ‘sqrt_output_valid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sqrt_output_valid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SQRT_WAIT, the machine waits for both command completion and the declared output count..

Line 530: `sigma_r <= sqrt_root == 0 ? 32'd1 : sqrt_root;` This makes sigma_r, the value named ’sigma r’ in this block, equal to ‘sqrt_root == 0 ? 32’d1 : sqrt_root‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are sqrt_root: the value named ’sqrt root’ in this block; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SQRT_WAIT, the machine waits for both command completion and the declared output count..

Line 531: `state <= ST_RECIPROCAL_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_RECIPROCAL_START‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_RECIPROCAL_START: the value named ’st reciprocal start’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SQRT_WAIT, the machine waits for both command completion and the declared output count..

Line 532: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SQRT_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 533: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SQRT_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 535: `ST_RECIPROCAL_START: begin` This is the ST_RECIPROCAL_START case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_START, the machine performs the named control phase..

Line 536: `if (divider_input_ready) state <= ST_RECIPROCAL_WAIT;` This condition selects the following hardware branch only when ‘divider_input_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are divider_input_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_START, the machine performs the named control phase..

Line 537: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_START, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 539: `ST_RECIPROCAL_WAIT: begin` This is the ST_RECIPROCAL_WAIT case branch, which waits for both command completion and the declared output count. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 540: `if (divider_output_valid) begin` This condition selects the following hardware branch only when ‘divider_output_valid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are divider_output_valid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 541: `reciprocal_q31_r <= divider_quotient[31:0];` This makes reciprocal_q31_r, the value named ’reciprocal q31 r’ in this block, equal to ‘divider_quotient[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are divider_quotient: the value named ’divider quotient’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 542: `output_index_r <= 12'd0;` This makes output_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘12’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 543: `output_remaining_r <= feature_count_r;` This makes output_remaining_r, the number of command result beats still expected, equal to ‘feature_count_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are feature_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 544: `config_error <= config_error || divider_zero;` This makes config_error, the value named ’config error’ in this block, equal to ‘config_error || divider_zero‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are config_error: the value named ’config error’ in this block; divider_zero: the value named ’divider zero’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 545: `state <= ST_OUTPUT_READ;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_READ‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_READ: the value named ’st output read’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 546: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 547: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 549: `ST_OUTPUT_READ: begin` This is the ST_OUTPUT_READ case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure..

Line 550: `output_value_ram_r <= value_mem[output_index_r[10:0]];` This makes output_value_ram_r, the value named 'output value ram r' in this block, equal to 'value_mem[output_index_r[10:0]]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are value_mem: local FPGA storage addressed by surrounding index logic; output_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure..

Line 551: `output_gamma_ram_r <= gamma_mem[output_index_r[10:0]];` This makes output_gamma_ram_r, the learned multiplicative LayerNorm parameter, equal to 'gamma_mem[output_index_r[10:0]]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_mem: the learned multiplicative LayerNorm parameter; output_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure..

Line 552: `output_beta_ram_r <= beta_mem[output_index_r[10:0]];` This makes output_beta_ram_r, the learned additive LayerNorm parameter, equal to 'beta_mem[output_index_r[10:0]]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are beta_mem: the learned additive LayerNorm parameter; output_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure..

Line 553: `state <= ST_OUTPUT_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_LATCH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_LATCH: the value named 'st output latch' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure..

Line 554: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 556: `ST_OUTPUT_LATCH: begin` This is the ST_OUTPUT_LATCH case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure..

Line 557: `output_value_r <= output_value_ram_r;` This makes output_value_r, the value named 'output value r' in this block, equal to 'output_value_ram_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_value_ram_r: the value named 'output value ram r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure..

Line 558: `output_gamma_r <= output_gamma_ram_r;` This makes output_gamma_r, the learned multiplicative LayerNorm parameter, equal to 'output_gamma_ram_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_gamma_ram_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure..

Line 559: `output_beta_r <= output_beta_ram_r;` This makes output_beta_r, the learned additive LayerNorm parameter, equal to 'output_beta_ram_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_beta_ram_r: the learned additive LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure..

Line 560: `state <= ST_OUTPUT_CENTER;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_CENTER‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_CENTER: the value named ’st output center’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure..

Line 561: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 563: `ST_OUTPUT_CENTER: begin` This is the ST_OUTPUT_CENTER case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_CENTER, the machine presents computed results under valid/ready backpressure..

Line 564: `output_centered_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_CENTER, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are output_centered_r: the value named ’output centered r’ in this block.

Line 565: `<= $signed({output_value_r[23], output_value_r})` Within the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_CENTER, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are output_value_r: the value named ’output value r’ in this block.

Line 566: `- $signed({mean_r[31], mean_r[23:0]});` Within the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_CENTER, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating. The important operands are mean_r: the arithmetic average used by normalization.

Line 567: `state <= ST_OUTPUT_NORMALIZE_OPERANDS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_NORMALIZE_OPERANDS‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_NORMALIZE_OPERANDS: the value named ’st output normalize operands’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_CENTER, the machine presents computed results under valid/ready backpressure..

Line 568: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_CENTER, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 570: `ST_OUTPUT_NORMALIZE_OPERANDS: begin` This is the ST_OUTPUT_NORMALIZE_OPERANDS case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_OPERAND the machine presents computed results under valid/ready backpressure..

Line 571: `normalized_centered_operand_r <= output_centered_r;` This makes normalized_centered_operand_r, the value named ’normalized centered operand r’ in this block, equal to ‘output_centered_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_centered_r: the value named ’output centered r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 572: `normalized_reciprocal_operand_r <= reciprocal_q31_r;` This makes normalized_reciprocal_operand_r, the value named 'normalized reciprocal operand r' in this block, equal to 'reciprocal_q31_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are reciprocal_q31_r: the value named 'reciprocal q31 r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 573: `state <= ST_OUTPUT_NORMALIZE_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_NORMALIZE_MULTIPLY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_NORMALIZE_MULTIPLY: the value named 'st output normalize multiply' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 574: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_OPERANDS, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 576: `ST_OUTPUT_NORMALIZE_MULTIPLY: begin` This is the ST_OUTPUT_NORMALIZE_MULTIPLY case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_MULTIPLY the machine presents computed results under valid/ready backpressure..

Line 577: `normalized_low_product_r <= normalized_low_product;` This makes normalized_low_product_r, the value named 'normalized low product r' in this block, equal to 'normalized_low_product'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_low_product: the value named 'normalized low product' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 578: `normalized_high_product_r <= normalized_high_product;` This makes normalized_high_product_r, the value named 'normalized high product r' in this block, equal to 'normalized_high_product'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_high_product: the value named 'normalized high product' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 579: `state <= ST_OUTPUT_NORMALIZE_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_NORMALIZE_LATCH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_NORMALIZE_LATCH: the value named 'st output normalize latch' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 580: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_MULTIPLY, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 582: `ST_OUTPUT_NORMALIZE_LATCH: begin` This is the ST_OUTPUT_NORMALIZE_LATCH case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_LATCH, the machine presents computed results under valid/ready backpressure..

Line 583: `normalized_product_r <=` This statement continues through source line 586; the explanation uses the complete expression rather than only the first visual fragment. This makes normalized_product_r, the value named 'normalized product r' in this block, equal to '{{16{normalized_low_product_r[41]}}, normalized_low_product_r} + ($signed({{16{normalized_high_product_r[41]}}, normalized_high_product_r}) <<< 16)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_low_product_r: the value named 'normalized low product r' in this block; normalized_high_product_r: the value named 'normalized high product r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_LATCH, the machine presents computed results under valid/ready backpressure..

Line 584: `{{16{normalized_low_product_r[41]}}, normalized_low_product_r}` This is continuation line 2 of the 4-line logical statement that begins at source line 583. Its exact fragment is '{{16{normalized_low_product_r[41]}}, normalized_low_product_r}'. The important operands are normalized_low_product_r: the value named 'normalized low product r' in this block. Read together, the complete operation is: This makes normalized_product_r, the value named 'normalized product r' in this block, equal to '{{16{normalized_low_product_r[41]}}, normalized_low_product_r} + ($signed({{16{normalized_high_product_r[41]}}, normalized_high_product_r}) <<< 16)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_low_product_r: the value named 'normalized low product r' in this block; normalized_high_product_r: the value named 'normalized high product r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_LATCH, the machine presents computed results under valid/ready backpressure..

Line 585: `+ ($signed({{16{normalized_high_product_r[41]}},` This is continuation line 3 of the 4-line logical statement that begins at source line 583. Its exact fragment is '+ ($signed({{16{normalized_high_product_r[41]}},'. The important operands are normalized_high_product_r: the value named 'normalized high product r' in this block. Read together, the complete operation is: This makes normalized_product_r, the value named 'normalized product r' in this block, equal to '{{16{normalized_low_product_r[41]}}, normalized_low_product_r} + ($signed({{16{normalized_high_product_r}) <<< 16)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_low_product_r: the value named 'normalized low product r' in this block; normalized_high_product_r: the value named 'normalized high product r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_LATCH, the machine presents computed results under valid/ready backpressure..

Line 586: `normalized_high_product_r}) <<< 16);` This is continuation line 4 of the 4-line logical statement that begins at source line 583. Its exact fragment is 'normalized_high_product_r}) <<< 16);'. The important operands are normalized_high_product_r: the value named 'normalized high product r' in this block. Read together, the complete operation is: This makes normalized_product_r, the value named 'normalized product r' in this block, equal to '{{16{normalized_low_product_r[41]}}, normalized_low_product_r} + ($signed({{16{normalized_high_product_r[41]}}, normalized_high_product_r}) <<< 16)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_low_product_r: the value named 'normalized low product r' in this block; normalized_high_product_r: the value named 'normalized high product r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_LATCH, the machine presents computed results under valid/ready backpressure..

Line 587: `state <= ST_OUTPUT_NORMALIZE_ABSOLUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_NORMALIZE_ABSOLUTE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_NORMALIZE_ABSOLUTE: the value named 'st output normalize absolute' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_LATCH, the machine presents computed results under valid/ready backpressure..

Line 588: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_LATCH, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 590: `ST_OUTPUT_NORMALIZE_ABSOLUTE: begin` This is the ST_OUTPUT_NORMALIZE_ABSOLUTE case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; con-

text is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 591: `logic [32:0] magnitude_low;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure.. Specifically, magnitude_low is the value named 'magnitude low' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 592: `normalized_negative_r <= normalized_product_r[57];` This makes normalized_negative_r, the value named 'normalized negative r' in this block, equal to 'normalized_product_r[57]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_product_r: the value named 'normalized product r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 593: `magnitude_low = normalized_product_r[57]` This statement continues through source line 595; the explanation uses the complete expression rather than only the first visual fragment. This makes magnitude_low, the value named 'magnitude low' in this block, equal to 'normalized_product_r[57] ? {1'b0, ~normalized_product_r[31:0]} + 1'b1 : {1'b0, normalized_product_r[31:0]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are normalized_product_r: the value named 'normalized product r' in this block; b0: the value named 'b0' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 594: `? {1'b0, ~normalized_product_r[31:0]} + 1'b1` This is continuation line 2 of the 3-line logical statement that begins at source line 593. Its exact fragment is '? {1'b0, ~normalized_product_r[31:0]} + 1'b1'. The important operands are b0: the value named 'b0' in this block; normalized_product_r: the value named 'normalized product r' in this block; b1: the value named 'b1' in this block. Read together, the complete operation is: This makes magnitude_low, the value named 'magnitude low' in this block, equal to 'normalized_product_r[57] ? {1'b0, ~normalized_product_r[31:0]} + 1'b1 : {1'b0, normalized_product_r[31:0]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are normalized_product_r: the value named 'normalized product r' in this block; b0: the value named 'b0' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 595: `: {1'b0, normalized_product_r[31:0]};` This is continuation line 3 of the 3-line logical statement that begins at source line 593. Its exact fragment is ': {1'b0, normalized_product_r[31:0]};'. The important operands are b0: the value named 'b0' in this block; normalized_product_r: the value named 'normalized product r' in this block. Read together, the complete operation is: This makes magnitude_low, the value named 'magnitude low' in this block, equal to 'normalized_product_r[57] ? {1'b0, ~normalized_product_r[31:0]} + 1'b1 : {1'b0, normalized_product_r[31:0]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are normalized_product_r: the value named 'normalized product r' in this block; b0: the value named 'b0' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 596: `normalized_magnitude_low_r <= magnitude_low[31:0];` This makes normalized_magnitude_low_r, the value named 'normalized magnitude low r' in this block, equal to 'magnitude_low[31:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are magnitude_low: the value named 'magnitude low' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 597: `normalized_magnitude_carry_r <= magnitude_low[32];` This makes normalized_magnitude_carry_r, the value named 'normalized magnitude carry r' in this block, equal to 'magnitude_low[32]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are magnitude_low: the value named

'magnitude low' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 598: `state <= ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH: the value named 'st output normalize absolute high' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 599: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 601: `ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH: begin` This is the ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLU the machine presents computed results under valid/ready backpressure..

Line 602: `logic [25:0] magnitude_high;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH, the machine presents computed results under valid/ready backpressure.. Specifically, magnitude_high is the value named 'magnitude high' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 603: `magnitude_high = normalized_negative_r` This statement continues through source line 606; the explanation uses the complete expression rather than only the first visual fragment. This makes magnitude_high, the value named 'magnitude high' in this block, equal to 'normalized_negative_r ? ~normalized_product_r[57:32] + 26'(normalized_magnitude_carry_r) : normalized_product_r[57:32]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are normalized_negative_r: the value named 'normalized negative r' in this block; normalized_product_r: the value named 'normalized product r' in this block; normalized_magnitude_carry_r: the value named 'normalized magnitude carry r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH, the machine presents computed results under valid/ready backpressure..

Line 604: `? ~normalized_product_r[57:32]` This is continuation line 2 of the 4-line logical statement that begins at source line 603. Its exact fragment is '? ~normalized_product_r[57:32]'. The important operands are normalized_product_r: the value named 'normalized product r' in this block. Read together, the complete operation is: This makes magnitude_high, the value named 'magnitude high' in this block, equal to 'normalized_negative_r ? ~normalized_product_r[57:32] + 26'(normalized_magnitude_carry_r) : normalized_product_r[57:32]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are normalized_negative_r: the value named 'normalized negative r' in this block; normalized_product_r: the value named 'normalized product r' in this block; normalized_magnitude_carry_r: the value named 'normalized magnitude carry r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH, the machine presents computed results under valid/ready backpressure..

Line 605: `+ 26'(normalized_magnitude_carry_r)` This is continuation line 3 of the 4-line logical statement that begins at source line 603. Its exact fragment is '+ 26'(normalized_magnitude_carry_r)'. The important operands are normalized_magnitude_carry_r: the value named 'normalized magnitude carry r' in this block. Read together, the complete operation is: This makes magnitude_high, the value named 'magnitude high' in this block, equal to 'normalized_negative_r ? ~normalized_product_r[57:32] + 26'(normalized_magnitude_carry_r) : normalized_product_r[57:32]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are normalized_negative_r: the value named 'normalized negative r' in this block; normalized_product_r: the value named 'normalized product r' in this block; normalized_magnitude_carry_r: the value named 'normalized magnitude carry r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_A the machine presents computed results under valid/ready backpressure..

Line 606: `: normalized_product_r[57:32];` This is continuation line 4 of the 4-line logical statement that begins at source line 603. Its exact fragment is ‘: normalized_product_r[57:32];‘. The important operands are normalized_product_r: the value named 'normalized product r' in this block. Read together, the complete operation is: This makes magnitude_high, the value named 'magnitude high' in this block, equal to ‘normalized_negative_r ? ~normalized_product_r[57:32] + 26'(normalized_magnitude_carry_r) : normalized_product_r[57:32]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are normalized_negative_r: the value named 'normalized negative r' in this block; normalized_product_r: the value named 'normalized product r' in this block; normalized_magnitude_carry_r: the value named 'normalized magnitude carry r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH, the machine presents computed results under valid/ready backpressure..

Line 607: `normalized_magnitude_r <= {` This statement continues through source line 608; the explanation uses the complete expression rather than only the first visual fragment. This makes normalized_magnitude_r, the value named 'normalized magnitude r' in this block, equal to ‘{ 6'd0, magnitude_high, normalized_magnitude_low_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; magnitude_high: the value named 'magnitude high' in this block; normalized_magnitude_low_r: the value named 'normalized magnitude low r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH, the machine presents computed results under valid/ready backpressure..

Line 608: `6'd0, magnitude_high, normalized_magnitude_low_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 607. Its exact fragment is ‘6'd0, magnitude_high, normalized_magnitude_low_r};‘. The important operands are d0: the value named 'd0' in this block; magnitude_high: the value named 'magnitude high' in this block; normalized_magnitude_low_r: the value named 'normalized magnitude low r' in this block. Read together, the complete operation is: This makes normalized_magnitude_r, the value named 'normalized magnitude r' in this block, equal to ‘{ 6'd0, magnitude_high, normalized_magnitude_low_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; magnitude_high: the value named 'magnitude high' in this block; normalized_magnitude_low_r: the value named 'normalized magnitude low r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH, the machine presents computed results under valid/ready backpressure..

Line 609: `state <= ST_OUTPUT_NORMALIZE_ROUND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_NORMALIZE_ROUND‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_NORMALIZE_ROUND: the value named 'st output normalize round' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH, the machine presents computed results under valid/ready backpressure..

Line 610: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ABSOLUTE_HIGH, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 612: `ST_OUTPUT_NORMALIZE_ROUND: begin` This is the ST_OUTPUT_NORMALIZE_ROUND case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ROUND, the machine presents computed results under valid/ready backpressure..

Line 613: `normalized_rounded_magnitude_r <=` This statement continues through source line 614; the explanation uses the complete expression rather than only the first visual fragment. This makes normalized_rounded_magnitude_r, the value named 'normalized rounded magnitude r' in this block, equal to ‘(normalized_magnitude_r + 64'd1073741824) >> 31‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_magnitude_r: the value named 'normalized magnitude r' in this block; d1073741824: the value named 'd1073741824' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ROUND, the machine presents computed results under valid/ready backpressure..

Line 614: `(normalized_magnitude_r + 64'd1073741824) >> 31;` This is continuation line 2 of the 2-line logical statement that begins at source line 613. Its exact fragment is '(normalized_magnitude_r + 64'd1073741824) >> 31;'. The important operands are normalized_magnitude_r: the value named 'normalized magnitude r' in this block; d1073741824: the value named 'd1073741824' in this block. Read together, the complete operation is: This makes normalized_rounded_magnitude_r, the value named 'normalized rounded magnitude r' in this block, equal to '(normalized_magnitude_r + 64'd1073741824) >> 31'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_magnitude_r: the value named 'normalized magnitude r' in this block; d1073741824: the value named 'd1073741824' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ROUND, the machine presents computed results under valid/ready backpressure..

Line 615: `state <= ST_OUTPUT_NORMALIZE_CLAMP;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_NORMALIZE_CLAMP'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_NORMALIZE_CLAMP: the value named 'st output normalize clamp' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ROUND, the machine presents computed results under valid/ready backpressure..

Line 616: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_ROUND, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 618: `ST_OUTPUT_NORMALIZE_CLAMP: begin` This is the ST_OUTPUT_NORMALIZE_CLAMP case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 619: `if (normalized_negative_r) begin` This condition selects the following hardware branch only when 'normalized_negative_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are normalized_negative_r: the value named 'normalized negative r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 620: `normalized_clamped_magnitude_r <=` This statement continues through source line 623; the explanation uses the complete expression rather than only the first visual fragment. This makes normalized_clamped_magnitude_r, the value named 'normalized clamped magnitude r' in this block, equal to 'normalized_rounded_magnitude_r > 64'h0000000080000000 ? 32'h80000000 : normalized_rounded_magnitude_r[31:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block; h0000000080000000: the value named 'h0000000080000000' in this block; h80000000: the value named 'h80000000' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 621: `normalized_rounded_magnitude_r > 64'h0000000080000000` This is continuation line 2 of the 4-line logical statement that begins at source line 620. Its exact fragment is 'normalized_rounded_magnitude_r > 64'h0000000080000000'. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block; h0000000080000000: the value named 'h0000000080000000' in this block. Read together, the complete operation is: This makes normalized_clamped_magnitude_r, the value named 'normalized clamped magnitude r' in this block, equal to 'normalized_rounded_magnitude_r > 64'h0000000080000000 ? 32'h80000000 : normalized_rounded_magnitude_r[31:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block; h0000000080000000: the value named 'h0000000080000000' in this block; h80000000: the value named 'h80000000' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 622: `? 32'h80000000` This is continuation line 3 of the 4-line logical statement that begins at source line 620. Its exact fragment is '? 32'h80000000'. The important operands are h80000000: the value named 'h80000000' in this block. Read together, the complete operation is: This makes normalized_clamped_magnitude_r, the value named 'normalized clamped magnitude r' in this block, equal to 'normalized_rounded_magnitude_r > 64'h0000000080000000 ? 32'h80000000 : normalized_rounded_magnitude_r[31:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block; h0000000080000000: the value named 'h0000000080000000' in this block; h80000000: the value named 'h80000000' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 623: `: normalized_rounded_magnitude_r[31:0];` This is continuation line 4 of the 4-line logical statement that begins at source line 620. Its exact fragment is ': normalized_rounded_magnitude_r[31:0];'. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block. Read together, the complete operation is: This makes normalized_clamped_magnitude_r, the value named 'normalized clamped magnitude r' in this block, equal to 'normalized_rounded_magnitude_r > 64'h0000000080000000 ? 32'h80000000 : normalized_rounded_magnitude_r[31:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block; h0000000080000000: the value named 'h0000000080000000' in this block; h80000000: the value named 'h80000000' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 624: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 625: `normalized_clamped_magnitude_r <=` This statement continues through source line 628; the explanation uses the complete expression rather than only the first visual fragment. This makes normalized_clamped_magnitude_r, the value named 'normalized clamped magnitude r' in this block, equal to 'normalized_rounded_magnitude_r > 64'h000000007fffffff ? 32'h7fffffff : normalized_rounded_magnitude_r[31:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block; h000000007fffffff: the value named 'h000000007fffffff' in this block; h7fffffff: the value named 'h7fffffff' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 626: `normalized_rounded_magnitude_r > 64'h000000007fffffff` This is continuation line 2 of the 4-line logical statement that begins at source line 625. Its exact fragment is 'normalized_rounded_magnitude_r > 64'h000000007fffffff'. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block; h000000007fffffff: the value named 'h000000007fffffff' in this block. Read together, the complete operation is: This makes normalized_clamped_magnitude_r, the value named 'normalized clamped magnitude r' in this block, equal to 'normalized_rounded_magnitude_r > 64'h000000007fffffff ? 32'h7fffffff : normalized_rounded_magnitude_r[31:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block; h000000007fffffff: the value named 'h000000007fffffff' in this block; h7fffffff: the value named 'h7fffffff' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 627: `? 32'h7fffffff` This is continuation line 3 of the 4-line logical statement that begins at source line 625. Its exact fragment is '? 32'h7fffffff'. The important operands are h7fffffff: the value named 'h7fffffff' in this block. Read together, the complete operation is: This makes normalized_clamped_magnitude_r, the value named 'normalized clamped magnitude r' in this block, equal to 'normalized_rounded_magnitude_r > 64'h000000007fffffff ? 32'h7fffffff : normalized_rounded_magnitude_r[31:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_rounded_magnitude_r: the value named 'normalized rounded magnitude r' in this block; h000000007fffffff: the value named 'h000000007fffffff' in this block; h7fffffff: the value named 'h7fffffff' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 628: `: normalized_rounded_magnitude_r[31:0];` This is continuation line 4 of the 4-line logical statement that begins at source line 625. Its exact fragment is ‘: normalized_rounded_magnitude_r[31:0];‘. The important operands are normalized_rounded_magnitude_r: the value named ’normalized rounded magnitude r’ in this block. Read together, the complete operation is: This makes normalized_clamped_magnitude_r, the value named ’normalized clamped magnitude r’ in this block, equal to ‘normalized_rounded_magnitude_r > 64’h000000007fffffff ? 32’h7fffffff : normalized_rounded_magnitude_r[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_rounded_magnitude_r: the value named ’normalized rounded magnitude r’ in this block; h000000007fffffff: the value named ’h000000007fffffff’ in this block; h7fffffff: the value named ’h7fffffff’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 629: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 630: `state <= ST_OUTPUT_NORMALIZE_SIGN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_NORMALIZE_SIGN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_NORMALIZE_SIGN: the value named ’st output normalize sign’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure..

Line 631: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_CLAMP, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 633: `ST_OUTPUT_NORMALIZE_SIGN: begin` This is the ST_OUTPUT_NORMALIZE_SIGN case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 634: `if (normalized_negative_r) begin` This condition selects the following hardware branch only when ‘normalized_negative_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are normalized_negative_r: the value named ’normalized negative r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 635: `normalized_r <= normalized_clamped_magnitude_r[31]` This statement continues through source line 637; the explanation uses the complete expression rather than only the first visual fragment. This makes normalized_r, the value named ’normalized r’ in this block, equal to ‘normalized_clamped_magnitude_r[31] ? 32’sh80000000 : -$signed(normalized_clamped_magnitude_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_clamped_magnitude_r: the value named ’normalized clamped magnitude r’ in this block; sh80000000: the value named ’sh80000000’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 636: `? 32'sh80000000` This is continuation line 2 of the 3-line logical statement that begins at source line 635. Its exact fragment is ‘? 32’sh80000000‘. The important operands are sh80000000: the value named ’sh80000000’ in this block. Read together, the complete operation is: This makes normalized_r, the value named ’normalized r’ in this block, equal to ‘normalized_clamped_magnitude_r[31] ? 32’sh80000000 : -$signed(normalized_clamped_magnitude_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_clamped_magnitude_r: the value named ’normalized clamped magnitude r’ in this block; sh80000000: the value named ’sh80000000’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 637: `: -$signed(normalized_clamped_magnitude_r);` This is continuation line 3 of the 3-line logical statement that begins at source line 635. Its exact fragment is ': -$signed(normalized_clamped_magnitude_r);'. The important operands are normalized_clamped_magnitude_r: the value named 'normalized clamped magnitude r' in this block. Read together, the complete operation is: This makes normalized_r, the value named 'normalized r' in this block, equal to 'normalized_clamped_magnitude_r[31] ? 32'sh80000000 : -$signed(normalized_clamped_magnitude_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_clamped_magnitude_r: the value named 'normalized clamped magnitude r' in this block; sh80000000: the value named 'sh80000000' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 638: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 639: `normalized_r <= $signed(normalized_clamped_magnitude_r);` This makes normalized_r, the value named 'normalized r' in this block, equal to '$signed(normalized_clamped_magnitude_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_clamped_magnitude_r: the value named 'normalized clamped magnitude r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 640: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 641: `state <= ST_OUTPUT_GAMMA_OPERANDS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_GAMMA_OPERANDS'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_GAMMA_OPERANDS: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 642: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_NORMALIZE_SIGN, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 644: `ST_OUTPUT_GAMMA_OPERANDS: begin` This is the ST_OUTPUT_GAMMA_OPERANDS case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 645: `gamma_normalized_operand_r <= normalized_r;` This makes gamma_normalized_operand_r, the learned multiplicative LayerNorm parameter, equal to 'normalized_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are normalized_r: the value named 'normalized r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 646: `gamma_scale_operand_r <= output_gamma_r;` This makes gamma_scale_operand_r, the real-value spacing represented by one quantized integer step, equal to 'output_gamma_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_gamma_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 647: `state <= ST_OUTPUT_GAMMA_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_GAMMA_MULTIPLY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_GAMMA_MULTIPLY: the learned multiplicative

LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 648: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_OPERANDS, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 650: `ST_OUTPUT_GAMMA_MULTIPLY: begin` This is the ST_OUTPUT_GAMMA_MULTIPLY case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 651: `gamma_partial_ll_r <= gamma_partial_ll;` This makes gamma_partial_ll_r, the learned multiplicative LayerNorm parameter, equal to 'gamma_partial_ll'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_partial_ll: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 652: `gamma_partial_lh_r <= gamma_partial_lh;` This makes gamma_partial_lh_r, the learned multiplicative LayerNorm parameter, equal to 'gamma_partial_lh'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_partial_lh: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 653: `gamma_partial_hl_r <= gamma_partial_hl;` This makes gamma_partial_hl_r, the learned multiplicative LayerNorm parameter, equal to 'gamma_partial_hl'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_partial_hl: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 654: `gamma_partial_hh_r <= gamma_partial_hh;` This makes gamma_partial_hh_r, the learned multiplicative LayerNorm parameter, equal to 'gamma_partial_hh'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_partial_hh: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 655: `state <= ST_OUTPUT_GAMMA_CROSS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_GAMMA_CROSS'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_GAMMA_CROSS: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 656: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_MULTIPLY, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 658: `ST_OUTPUT_GAMMA_CROSS: begin` This is the ST_OUTPUT_GAMMA_CROSS case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_CROSS, the machine presents computed results under valid/ready backpressure..

Line 659: `gamma_cross_r <=` This statement continues through source line 661; the explanation uses the complete expression rather than only the first visual fragment. This makes gamma_cross_r, the learned multiplicative LayerNorm parameter, equal to '{{9{gamma_partial_lh_r[24]}}, gamma_partial_lh_r} + {{1{gamma_partial_hl_r[32]}},

gamma_partial_hl_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_partial_lh_r: the learned multiplicative LayerNorm parameter; gamma_partial_hl_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_CROSS, the machine presents computed results under valid/ready backpressure..

Line 660: `{{9{gamma_partial_lh_r[24]}}, gamma_partial_lh_r}` This is continuation line 2 of the 3-line logical statement that begins at source line 659. Its exact fragment is ‘{{9{gamma_partial_lh_r[24]}}, gamma_partial_lh_r}‘. The important operands are gamma_partial_lh_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This makes gamma_cross_r, the learned multiplicative LayerNorm parameter, equal to ‘{{9{gamma_partial_lh_r[24]}}, gamma_partial_lh_r} + {{1{gamma_partial_hl_r[32]}}, gamma_partial_hl_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_partial_lh_r: the learned multiplicative LayerNorm parameter; gamma_partial_hl_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_CROSS, the machine presents computed results under valid/ready backpressure..

Line 661: `+ {{1{gamma_partial_hl_r[32]}}, gamma_partial_hl_r};` This is continuation line 3 of the 3-line logical statement that begins at source line 659. Its exact fragment is ‘+ {{1{gamma_partial_hl_r[32]}}, gamma_partial_hl_r};‘. The important operands are gamma_partial_hl_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This makes gamma_cross_r, the learned multiplicative LayerNorm parameter, equal to ‘{{9{gamma_partial_lh_r[24]}}, gamma_partial_lh_r} + {{1{gamma_partial_hl_r[32]}}, gamma_partial_hl_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_partial_lh_r: the learned multiplicative LayerNorm parameter; gamma_partial_hl_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_CROSS, the machine presents computed results under valid/ready backpressure..

Line 662: `state <= ST_OUTPUT_GAMMA_LOW;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_GAMMA_LOW‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_GAMMA_LOW: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_CROSS, the machine presents computed results under valid/ready backpressure..

Line 663: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_CROSS, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 665: `ST_OUTPUT_GAMMA_LOW: begin` This is the ST_OUTPUT_GAMMA_LOW case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LOW, the machine presents computed results under valid/ready backpressure..

Line 666: `gamma_low_cross_r <=` This statement continues through source line 668; the explanation uses the complete expression rather than only the first visual fragment. This makes gamma_low_cross_r, the learned multiplicative LayerNorm parameter, equal to ‘{{16{gamma_partial_ll_r[33]}}, gamma_partial_ll_r} + ($signed({{16{gamma_cross_r[33]}}, gamma_cross_r}) <<< 16)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_partial_ll_r: the learned multiplicative LayerNorm parameter; gamma_cross_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LOW, the machine presents computed results under valid/ready backpressure..

Line 667: `{{16{gamma_partial_ll_r[33]}}, gamma_partial_ll_r}` This is continuation line 2 of the 3-line logical statement that begins at source line 666. Its exact fragment is ‘{{16{gamma_partial_ll_r[33]}}, gamma_partial_ll_r}‘. The important operands are gamma_partial_ll_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This makes gamma_low_cross_r, the learned multiplicative LayerNorm parameter, equal to ‘{{16{gamma_partial_ll_r[33]}}, gamma_partial_ll_r} + ($signed({{16{gamma_cross_r[33]}}, gamma_cross_r}) <<< 16)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands

are gamma_partial_ll_r: the learned multiplicative LayerNorm parameter; gamma_cross_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LOW, the machine presents computed results under valid/ready backpressure..

Line 668: `+ ($signed({{16{gamma_cross_r[33]}}, gamma_cross_r}) <<< 16);` This is continuation line 3 of the 3-line logical statement that begins at source line 666. Its exact fragment is '+ ($signed({{16{gamma_cross_r[33]}}, gamma_cross_r}) <<< 16);'. The important operands are gamma_cross_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This makes gamma_low_cross_r, the learned multiplicative LayerNorm parameter, equal to '{{16{gamma_partial_ll_r[33]}}, gamma_partial_ll_r} + ($signed({{16{gamma_cross_r[33]}}, gamma_cross_r}) <<< 16)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_partial_ll_r: the learned multiplicative LayerNorm parameter; gamma_cross_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LOW, the machine presents computed results under valid/ready backpressure..

Line 669: `state <= ST_OUTPUT_GAMMA_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_GAMMA_LATCH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_GAMMA_LATCH: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LOW, the machine presents computed results under valid/ready backpressure..

Line 670: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LOW, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 672: `ST_OUTPUT_GAMMA_LATCH: begin` This is the ST_OUTPUT_GAMMA_LATCH case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LATCH, the machine presents computed results under valid/ready backpressure..

Line 673: `gamma_product_r <=` This statement continues through source line 676; the explanation uses the complete expression rather than only the first visual fragment. This makes gamma_product_r, the learned multiplicative LayerNorm parameter, equal to '{{6{gamma_low_cross_r[49]}}, gamma_low_cross_r} + ($signed({{32{gamma_partial_hh_r[23]}}, gamma_partial_hh_r}) <<< 32)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_low_cross_r: the learned multiplicative LayerNorm parameter; gamma_partial_hh_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LATCH, the machine presents computed results under valid/ready backpressure..

Line 674: `{{6{gamma_low_cross_r[49]}}, gamma_low_cross_r}` This is continuation line 2 of the 4-line logical statement that begins at source line 673. Its exact fragment is '{{6{gamma_low_cross_r[49]}}, gamma_low_cross_r}'. The important operands are gamma_low_cross_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This makes gamma_product_r, the learned multiplicative LayerNorm parameter, equal to '{{6{gamma_low_cross_r[49]}}, gamma_low_cross_r} + ($signed({{32{gamma_partial_hh_r[23]}}, gamma_partial_hh_r}) <<< 32)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_low_cross_r: the learned multiplicative LayerNorm parameter; gamma_partial_hh_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LATCH, the machine presents computed results under valid/ready backpressure..

Line 675: `+ ($signed({{32{gamma_partial_hh_r[23]}},` This is continuation line 3 of the 4-line logical statement that begins at source line 673. Its exact fragment is '+ ($signed({{32{gamma_partial_hh_r[23]}},'. The important operands are gamma_partial_hh_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This makes gamma_product_r, the learned multiplicative LayerNorm parameter, equal to '{{6{gamma_low_cross_r[49]}}, gamma_low_cross_r} + ($signed({{32{gamma_partial_hh_r[23]}}, gamma_partial_hh_r}) <<< 32)'. It updates after the active clock edge together with other nonblocking assignments. The important operands

are gamma_low_cross_r: the learned multiplicative LayerNorm parameter; gamma_partial_hh_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LATCH, the machine presents computed results under valid/ready backpressure..

Line 676: `gamma_partial_hh_r}) <<< 32);` This is continuation line 4 of the 4-line logical statement that begins at source line 673. Its exact fragment is 'gamma_partial_hh_r}) <<< 32);'. The important operands are gamma_partial_hh_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This makes gamma_product_r, the learned multiplicative LayerNorm parameter, equal to '{{6{gamma_low_cross_r[49]}}, gamma_low_cross_r} + ($signed({{32{gamma_partial_hh_r[23]}}, gamma_partial_hh_r}) <<< 32)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_low_cross_r: the learned multiplicative LayerNorm parameter; gamma_partial_hh_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LATCH, the machine presents computed results under valid/ready backpressure..

Line 677: `state <= ST_OUTPUT_SCALE_ABSOLUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_SCALE_ABSOLUTE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_SCALE_ABSOLUTE: the real-value spacing represented by one quantized integer step. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LATCH, the machine presents computed results under valid/ready backpressure..

Line 678: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_GAMMA_LATCH, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 680: `ST_OUTPUT_SCALE_ABSOLUTE: begin` This is the ST_OUTPUT_SCALE_ABSOLUTE case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 681: `scale_negative_r <= gamma_product_r[55];` This makes scale_negative_r, the real-value spacing represented by one quantized integer step, equal to 'gamma_product_r[55]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_product_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 682: `scale_magnitude_r <= gamma_product_r[55]` This statement continues through source line 683; the explanation uses the complete expression rather than only the first visual fragment. This makes scale_magnitude_r, the real-value spacing represented by one quantized integer step, equal to 'gamma_product_r[55] ? -gamma_product_r : gamma_product_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_product_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 683: `? -gamma_product_r : gamma_product_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 682. Its exact fragment is '? -gamma_product_r : gamma_product_r;'. The important operands are gamma_product_r: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This makes scale_magnitude_r, the real-value spacing represented by one quantized integer step, equal to 'gamma_product_r[55] ? -gamma_product_r : gamma_product_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gamma_product_r: the learned multiplicative LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 684: `state <= ST_OUTPUT_SCALE_ROUND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_SCALE_ROUND'. It updates after the active clock edge together with other nonblocking

assignments. The important operands are ST_OUTPUT_SCALE_ROUND: the real-value spacing represented by one quantized integer step. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ABSOLUTE, the machine presents computed results under valid/ready backpressure..

Line 685: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ABSOLUTE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 687: `ST_OUTPUT_SCALE_ROUND: begin` This is the ST_OUTPUT_SCALE_ROUND case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ROUND, the machine presents computed results under valid/ready backpressure..

Line 688: `scale_rounded_magnitude_r <=` This statement continues through source line 689; the explanation uses the complete expression rather than only the first visual fragment. This makes scale_rounded_magnitude_r, the real-value spacing represented by one quantized integer step, equal to '(scale_magnitude_r + 56'd16384) >> 15'. It updates after the active clock edge together with other nonblocking assignments. The important operands are scale_magnitude_r: the real-value spacing represented by one quantized integer step; d16384: the value named 'd16384' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ROUND, the machine presents computed results under valid/ready backpressure..

Line 689: `(scale_magnitude_r + 56'd16384) >> 15;` This is continuation line 2 of the 2-line logical statement that begins at source line 688. Its exact fragment is '(scale_magnitude_r + 56'd16384) >> 15;'. The important operands are scale_magnitude_r: the real-value spacing represented by one quantized integer step; d16384: the value named 'd16384' in this block. Read together, the complete operation is: This makes scale_rounded_magnitude_r, the real-value spacing represented by one quantized integer step, equal to '(scale_magnitude_r + 56'd16384) >> 15'. It updates after the active clock edge together with other nonblocking assignments. The important operands are scale_magnitude_r: the real-value spacing represented by one quantized integer step; d16384: the value named 'd16384' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ROUND, the machine presents computed results under valid/ready backpressure..

Line 690: `state <= ST_OUTPUT_SCALE_SIGN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_SCALE_SIGN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_SCALE_SIGN: the real-value spacing represented by one quantized integer step. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ROUND, the machine presents computed results under valid/ready backpressure..

Line 691: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_ROUND, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 693: `ST_OUTPUT_SCALE_SIGN: begin` This is the ST_OUTPUT_SCALE_SIGN case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 694: `scaled_r <= scale_negative_r` This statement continues through source line 696; the explanation uses the complete expression rather than only the first visual fragment. This makes scaled_r, the real-value spacing represented by one quantized integer step, equal to 'scale_negative_r ? -$signed(scale_rounded_magnitude_r) : $signed(scale_rounded_magnitu
It updates after the active clock edge together with other nonblocking assignments. The important operands are scale_negative_r: the real-value spacing represented by one quantized integer step; scale_rounded_magnitude_r: the real-value spacing represented by one quantized integer step. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 695: `? -$signed(scale_rounded_magnitude_r)` This is continuation line 2 of the 3-line logical statement that begins at source line 694. Its exact fragment is '? -$signed(scale_rounded_magnitude_r)'. The important operands are scale_rounded_magnitude_r: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This makes scaled_r, the real-value spacing represented by one quantized integer step, equal to 'scale_negative_r ? -$signed(scale_rounded_magnitude_r) : $signed(scale_rounded_magnitude_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are scale_negative_r: the real-value spacing represented by one quantized integer step; scale_rounded_magnitude_r: the real-value spacing represented by one quantized integer step. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 696: `: $signed(scale_rounded_magnitude_r);` This is continuation line 3 of the 3-line logical statement that begins at source line 694. Its exact fragment is ': $signed(scale_rounded_magnitude_r);'. The important operands are scale_rounded_magnitude_r: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This makes scaled_r, the real-value spacing represented by one quantized integer step, equal to 'scale_negative_r ? -$signed(scale_rounded_magnitude_r) : $signed(scale_rounded_magnitude_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are scale_negative_r: the real-value spacing represented by one quantized integer step; scale_rounded_magnitude_r: the real-value spacing represented by one quantized integer step. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 697: `state <= ST_OUTPUT_BETA;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_BETA'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_BETA: the learned additive LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_SIGN, the machine presents computed results under valid/ready backpressure..

Line 698: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SCALE_SIGN, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 700: `ST_OUTPUT_BETA: begin` This is the ST_OUTPUT_BETA case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_BETA, the machine presents computed results under valid/ready backpressure..

Line 701: `scaled_beta_r <= {{8{scaled_r[55]}}, scaled_r}` This statement continues through source line 702; the explanation uses the complete expression rather than only the first visual fragment. This makes scaled_beta_r, the real-value spacing represented by one quantized integer step, equal to '{{8{scaled_r[55]}}, scaled_r} + {{(64-VALUE_BITS){output_beta_r[VALUE_BITS-1]}}, output_beta_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are scaled_r: the real-value spacing represented by one quantized integer step; VALUE_BITS: the value named 'value bits' in this block; output_beta_r: the learned additive LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_BETA, the machine presents computed results under valid/ready backpressure..

Line 702: `+ {{(64-VALUE_BITS){output_beta_r[VALUE_BITS-1]}}, output_beta_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 701. Its exact fragment is '+ {{(64-VALUE_BITS){output_beta_r[VALUE_1]}}, output_beta_r};'. The important operands are VALUE_BITS: the value named 'value bits' in this block; output_beta_r: the learned additive LayerNorm parameter. Read together, the complete operation is: This makes scaled_beta_r, the real-value spacing represented by one quantized integer step, equal to '{{8{scaled_r[55]}}, scaled_r} + {{(64-VALUE_BITS){output_beta_r[VALUE_BITS-1]}}, output_beta_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are scaled_r: the real-value spacing represented by one quantized integer step; VALUE_BITS: the value named 'value bits' in this block; output_beta_r: the learned additive LayerNorm parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_BETA, the machine presents computed results under valid/ready backpressure..

Line 703: `state <= ST_OUTPUT_SATURATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_SATURATE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_SATURATE: the value named ’st output saturate’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_BETA, the machine presents computed results under valid/ready backpressure..

Line 704: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_BETA, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 706: `ST_OUTPUT_SATURATE: begin` This is the ST_OUTPUT_SATURATE case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SATURATE, the machine presents computed results under valid/ready backpressure..

Line 707: `result_value <= saturate_s32(scaled_beta_r);` This makes result_value, the value named ’result value’ in this block, equal to ‘saturate_s32(scaled_beta_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_s32: the value named ’saturate s32’ in this block; scaled_beta_r: the real-value spacing represented by one quantized integer step. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SATURATE, the machine presents computed results under valid/ready backpressure..

Line 708: `result_last_r <= output_remaining_r == 12'd1;` This makes result_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘output_remaining_r == 12’d1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_remaining_r: the number of command result beats still expected; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SATURATE, the machine presents computed results under valid/ready backpressure..

Line 709: `state <= ST_OUTPUT_HOLD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_HOLD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_HOLD: the value named ’st output hold’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SATURATE, the machine presents computed results under valid/ready backpressure..

Line 710: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_SATURATE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 712: `ST_OUTPUT_HOLD: begin` This is the ST_OUTPUT_HOLD case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 713: `if (result_ready) begin` This condition selects the following hardware branch only when ‘result_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 714: `if (result_last) begin` This condition selects the following hardware branch only when ‘result_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 715: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 716: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 717: `output_index_r <= output_index_r + 1'b1;` This makes output_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index_r + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_index_r: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 718: `output_remaining_r <= output_remaining_r - 1'b1;` This makes output_remaining_r, the number of command result beats still expected, equal to ‘output_remaining_r - 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_remaining_r: the number of command result beats still expected; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 719: `state <= ST_OUTPUT_READ;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_READ‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_READ: the value named ’st output read’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 720: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 721: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 722: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 724: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘state <= ST_IDLE;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 725: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 726: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 727: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 729: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 730: `if (MAX_FEATURES < 1 || MAX_FEATURES > 2048) $fatal(1, "MAX_FEATURES must be in [1,2048]");` This condition selects the following hardware branch only when ‘MAX_FEATURES $<$ 1 || MAX_FEATURES $>$ 2048) $fatal(1, "MAX_FEATURES must be in [1,2048]"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are MAX_FEATURES: the value named ’max features’ in this block; fatal: the value named ’fatal’ in this block; must: the value named ’must’ in this block. Context: the module’s structural declarations.

Line 731: `if (VALUE_BITS != 24) $fatal(1, "neuraedge_integer_layernorm currently requires VALUE_BITS=24");` This condition selects the following hardware branch only when ‘VALUE_BITS != 24) $fatal(1, "neuraedge_integer_layernorm currently requires VALUE_BITS=24"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are VALUE_BITS: the value named ’value bits’ in this block; fatal: the value named ’fatal’ in this block; neuraedge_integer_layernorm: the value named ’neuraedge integer layernorm’ in this block. Context: the module’s structural declarations.

Line 732: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 733: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 735: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating.

11 rtl/neuraedge_isqrt64.sv

Role in the complete system: computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm.

11.1 Beginner architecture map

Dataflow responsibility One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_isqrt64 into physical nets and primitives. The source declares 5 input-port lines, 4 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_IDLE, ST_OUTPUT, ST_ROOT. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates unique as case. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

11.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_isqrt64 (
5      input  wire clk,
6      input  wire rst_n,
7      input  wire input_valid,
8      output logic input_ready,
9      input  wire [63:0] radicand,
10     output logic output_valid,
11     input  wire output_ready,
12     output logic [31:0] root,
13     output logic [32:0] remainder
14 );
15     typedef enum logic [1:0] {
16         ST_IDLE = 2'd0,
17         ST_ROOT = 2'd1,
18         ST_OUTPUT = 2'd2
19     } state_t;
20
21     state_t state;
22     logic [63:0] radicand_r;
23     logic [31:0] root_r;
24     // Before the final digit, remainder < 2*partial_root+1 < 2^32. The final
25     // remainder may need bit 32 and is written directly to the output port.
26     logic [31:0] remainder_r;
27     logic [5:0] pair_count;
28
29     always_comb begin
30         input_ready = state == ST_IDLE;
31         output_valid = state == ST_OUTPUT;
32     end
33
34     always_ff @(posedge clk) begin
35         if (!rst_n) begin
36             state <= ST_IDLE;
37             radicand_r <= 64'd0;
38             root_r <= 32'd0;
39             remainder_r <= 32'd0;
40             pair_count <= 6'd0;
41             root <= 32'd0;
42             remainder <= 33'd0;
43         end else begin
44             unique case (state)
45                 ST_IDLE: begin
46                     if (input_valid) begin
```

```
                        radicand_r <= radicand;
                        root_r <= 32'd0;
                        remainder_r <= 32'd0;
                        pair_count <= 6'd0;
                        state <= ST_ROOT;
                    end
                end

                ST_ROOT: begin
                    logic [33:0] shifted_remainder;
                    logic [33:0] trial_divisor;
                    logic [33:0] trial_difference;
                    logic [32:0] next_remainder;
                    logic [31:0] next_root;
                    shifted_remainder = {remainder_r, radicand_r[63:62]};
                    trial_divisor = {root_r, 2'b01};
                    // One widened subtraction produces both the candidate
                    // remainder and its unsigned borrow. This removes the
                    // serial compare-then-subtract carry chains.
                    trial_difference = shifted_remainder - trial_divisor;
                    if (!trial_difference[33]) begin
                        next_remainder = trial_difference[32:0];
                        next_root = {root_r[30:0], 1'b1};
                    end else begin
                        next_remainder = shifted_remainder[32:0];
                        next_root = {root_r[30:0], 1'b0};
                    end
                    radicand_r <= {radicand_r[61:0], 2'b00};
                    remainder_r <= next_remainder[31:0];
                    root_r <= next_root;

                    if (pair_count == 6'd31) begin
                        root <= next_root;
                        remainder <= next_remainder;
                        state <= ST_OUTPUT;
                    end else begin
                        pair_count <= pair_count + 1'b1;
                    end
                end

                ST_OUTPUT: begin
                    if (output_ready) state <= ST_IDLE;
                end

                default: state <= ST_IDLE;
            endcase
        end
    end
endmodule

`default_nettype wire
```

11.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_isqrt64 (` This opens neuraedge_isqrt64, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm.

Line 5: `input wire clk,` This declares clk as a input port with width one bit. It carries the clock that makes all registers advance together across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 6: `input wire rst_n,` This declares rst_n as a input port with width one bit. It carries the active-low synchronous reset across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 7: `input wire input_valid,` This declares input_valid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 8: `output logic input_ready,` This declares input_ready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 9: `input wire [63:0] radicand,` This declares radicand as a input port with width [63:0]. It carries the value named 'radicand' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 10: `output logic output_valid,` This declares output_valid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 11: `input wire output_ready,` This declares output_ready as a input port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 12: `output logic [31:0] root,` This declares root as a output port with width [31:0]. It carries the value named 'root' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 13: `output logic [32:0] remainder` This declares remainder as a output port with width [32:0]. It carries the value named 'remainder' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 14: `);` Within the module's structural declarations, this exact statement contributes to the block that computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm. The expression is a literal or structural operator and has no hidden software state.

Line 15: `typedef enum logic [1:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 16: `ST_IDLE = 2'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 17: `ST_ROOT = 2'd1,` This assigns a fixed binary encoding to state ST_ROOT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 18: `ST_OUTPUT = 2'd2` This statement continues through source line 19; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 19: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 18. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 21: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 22: `logic [63:0] radicand_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, radicand_r is the value named 'radicand r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic [31:0] root_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, root_r is the value named 'root r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `// Before the final digit, remainder < 2*partial_root+1 < 2^32. The final` Design intent: Before the final digit, remainder < 2*partial_root+1 < 2^32. The final This comment anchors the surrounding logic in the block that computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm.

Line 25: `// remainder may need bit 32 and is written directly to the output port.` Design intent: remainder may need bit 32 and is written directly to the output port. This comment anchors the surrounding logic in the block that computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm.

Line 26: `logic [31:0] remainder_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, remainder_r is the value named 'remainder r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic [5:0] pair_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pair_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 30: `input_ready = state == ST_IDLE;` This makes input_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_IDLE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 31: `output_valid = state == ST_OUTPUT;` This makes output_valid, the producer's statement that its current payload is meaningful, equal to 'state == ST_OUTPUT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT: the value named 'st output' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 32: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 34: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 35: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 36: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 37: `radicand_r <= 64'd0;` This makes radicand_r, the value named ’radicand r’ in this block, equal to ‘64’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 38: `root_r <= 32'd0;` This makes root_r, the value named ’root r’ in this block, equal to ‘32’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 39: `remainder_r <= 32'd0;` This makes remainder_r, the value named ’remainder r’ in this block, equal to ‘32’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 40: `pair_count <= 6'd0;` This makes pair_count, an integer count retained for control or measurable telemetry, equal to ‘6’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 41: `root <= 32'd0;` This makes root, the value named ’root’ in this block, equal to ‘32’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 42: `remainder <= 33'd0;` This makes remainder, the value named ’remainder’ in this block, equal to ‘33’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 43: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 44: `unique case (state)` This starts a multiplexer/state decode selected by ‘state‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 45: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 46: `if (input_valid) begin` This condition selects the following hardware branch only when ‘input_valid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_valid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 47: `radicand_r <= radicand;` This makes radicand_r, the value named ’radicand r’ in this block, equal to ‘radicand‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are radicand: the value named ’radicand’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 48: `root_r <= 32'd0;` This makes root_r, the value named 'root r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 49: `remainder_r <= 32'd0;` This makes remainder_r, the value named 'remainder r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 50: `pair_count <= 6'd0;` This makes pair_count, an integer count retained for control or measurable telemetry, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 51: `state <= ST_ROOT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ROOT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ROOT: the value named 'st root' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 52: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 53: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 55: `ST_ROOT: begin` This is the ST_ROOT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 56: `logic [33:0] shifted_remainder;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.. Specifically, shifted_remainder is the value named 'shifted remainder' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 57: `logic [33:0] trial_divisor;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.. Specifically, trial_divisor is the value named 'trial divisor' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 58: `logic [33:0] trial_difference;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.. Specifically, trial_difference is the value named 'trial difference' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 59: `logic [32:0] next_remainder;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.. Specifically, next_remainder is the value named 'next remainder' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 60: `logic [31:0] next_root;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.. Specifically, next_root is the value named 'next root' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 61: `shifted_remainder = {remainder_r, radicand_r[63:62]};` This makes shifted_remainder, the value named 'shifted remainder' in this block, equal to '{remainder_r, radicand_r[63:62]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are remainder_r: the value named 'remainder r' in this block; radicand_r: the value named 'radicand r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 62: `trial_divisor = {root_r, 2'b01};` This makes trial_divisor, the value named 'trial divisor' in this block, equal to '{root_r, 2'b01}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are root_r: the value named 'root r' in this block; b01: the value named 'b01' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 63: `// One widened subtraction produces both the candidate` Design intent: One widened subtraction produces both the candidate This comment anchors the surrounding logic in the block that computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm.

Line 64: `// remainder and its unsigned borrow. This removes the` Design intent: remainder and its unsigned borrow. This removes the This comment anchors the surrounding logic in the block that computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm.

Line 65: `// serial compare-then-subtract carry chains.` Design intent: serial compare-then-subtract carry chains. This comment anchors the surrounding logic in the block that computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm.

Line 66: `trial_difference = shifted_remainder - trial_divisor;` This makes trial_difference, the value named 'trial difference' in this block, equal to 'shifted_remainder - trial_divisor'. It is driven by combinational or procedural logic in the current evaluation. The important operands are shifted_remainder: the value named 'shifted remainder' in this block; trial_divisor: the value named 'trial divisor' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 67: `if (!trial_difference[33]) begin` This condition selects the following hardware branch only when '!trial_difference[33]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are trial_difference: the value named 'trial difference' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 68: `next_remainder = trial_difference[32:0];` This makes next_remainder, the value named 'next remainder' in this block, equal to 'trial_difference[32:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are trial_difference: the value named 'trial difference' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 69: `next_root = {root_r[30:0], 1'b1};` This makes next_root, the value named 'next root' in this block, equal to '{root_r[30:0], 1'b1}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are root_r: the value named 'root r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 70: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 71: `next_remainder = shifted_remainder[32:0];` This makes next_remainder, the value named 'next remainder' in this block, equal to 'shifted_remainder[32:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are shifted_remainder: the value named 'shifted remainder' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 72: `next_root = {root_r[30:0], 1'b0};` This makes next_root, the value named 'next root' in this block, equal to '{root_r[30:0], 1'b0}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are root_r: the value named 'root r' in this block; b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 73: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 74: `radicand_r <= {radicand_r[61:0], 2'b00};` This makes radicand_r, the value named 'radicand r' in this block, equal to '{radicand_r[61:0], 2'b00}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are radicand_r: the value named 'radicand r' in this block; b00: the value named 'b00' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 75: `remainder_r <= next_remainder[31:0];` This makes remainder_r, the value named 'remainder r' in this block, equal to 'next_remainder[31:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_remainder: the value named 'next remainder' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 76: `root_r <= next_root;` This makes root_r, the value named 'root r' in this block, equal to 'next_root'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_root: the value named 'next root' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 78: `if (pair_count == 6'd31) begin` This condition selects the following hardware branch only when 'pair_count == 6'd31' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pair_count: an integer count retained for control or measurable telemetry; d31: the value named 'd31' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 79: `root <= next_root;` This makes root, the value named 'root' in this block, equal to 'next_root'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_root: the value named 'next root' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 80: `remainder <= next_remainder;` This makes remainder, the value named 'remainder' in this block, equal to 'next_remainder'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_remainder: the value named 'next remainder' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 81: `state <= ST_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT: the value named ’st output’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 82: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 83: `pair_count <= pair_count + 1'b1;` This makes pair_count, an integer count retained for control or measurable telemetry, equal to ‘pair_count + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are pair_count: an integer count retained for control or measurable telemetry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase..

Line 84: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 85: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROOT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 87: `ST_OUTPUT: begin` This is the ST_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 88: `if (output_ready) state <= ST_IDLE;` This condition selects the following hardware branch only when ‘output_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 89: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 91: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘state <= ST_IDLE;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 92: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 93: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 94: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 95: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 97: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm.

12 rtl/neuraedge_lif_bank.sv

Role in the complete system: stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting.

12.1 Beginner architecture map

Dataflow responsibility Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_lif_bank into physical nets and primitives. The source declares 11 input-port lines, 7 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_COMMIT, ST_COUNT_DONE, ST_COUNT_LOW, ST_COUNT_WORD1, ST_COUNT_WORD2, ST_COUNT_WORD3, ST_GROUP_LOAD, ST_IDLE, ST_INTEGRAT ST_LEAK, ST_LEAK_SUBTRACT, ST_OUTPUT, ST_SATURATE, ST_THRESHOLD_COMPARE. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates unique as case, end as for, endcase as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

12.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_lif_bank #(
5      parameter int LANES = 32,
6      parameter int LANES_PER_CYCLE = 8,
7      parameter int MEMBRANE_BITS = 24,
8      parameter int CURRENT_BITS = 16
9  ) (
10     input  wire clk,
11     input  wire rst_n,
12     input  wire clear_state,
13     input  wire clear_counters,
14     input  wire input_valid,
15     output logic input_ready,
16     input  wire [LANES*CURRENT_BITS-1:0] current_vector,
17     input  wire [4:0] leak_shift,
18     input  wire signed [MEMBRANE_BITS-1:0] threshold,
19     input  wire signed [MEMBRANE_BITS-1:0] reset_value,
20     output logic output_valid,
21     input  wire output_ready,
22     output logic [LANES-1:0] spikes,
23     output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector,
24     output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector,
25     output logic [31:0] timestep_count,
26     output logic [63:0] total_spike_count
27 );
28     localparam int GROUPS = LANES / LANES_PER_CYCLE;
29
30     typedef enum logic [3:0] {
31         ST_IDLE = 4'd0,
32         ST_GROUP_LOAD = 4'd1,
33         ST_LEAK = 4'd2,
34         ST_INTEGRATE = 4'd3,
35         ST_SATURATE = 4'd4,
36         ST_COMMIT = 4'd5,
37         ST_OUTPUT = 4'd6,
38         ST_LEAK_SUBTRACT = 4'd7,
39         ST_THRESHOLD_COMPARE = 4'd8,
40         ST_COUNT_LOW = 4'd9,
41         ST_COUNT_WORD1 = 4'd10,
42         ST_COUNT_WORD2 = 4'd11,
43         ST_COUNT_WORD3 = 4'd12,
```

```
        ST_COUNT_DONE = 4'd13
    } state_t;

    state_t state;
    logic signed [MEMBRANE_BITS-1:0] membrane [0:LANES-1];
    logic signed [MEMBRANE_BITS-1:0] pre_reset_membrane [0:LANES-1];
    logic [LANES*CURRENT_BITS-1:0] current_vector_r;
    logic [4:0] leak_shift_r;
    logic signed [MEMBRANE_BITS-1:0] threshold_r;
    logic signed [MEMBRANE_BITS-1:0] reset_value_r;
    logic [$clog2(GROUPS)-1:0] group_index;
    logic [LANES-1:0] spike_work_r;
    logic [6:0] step_spike_count_r;
    logic [6:0] completed_step_spike_count_r;
    logic [16:0] counter_sum_r;
    logic signed [MEMBRANE_BITS-1:0] group_membrane_r [0:LANES_PER_CYCLE-1];
    logic signed [CURRENT_BITS-1:0] group_current_r [0:LANES_PER_CYCLE-1];
    logic signed [MEMBRANE_BITS:0] leak_shifted_r [0:LANES_PER_CYCLE-1];
    logic signed [MEMBRANE_BITS:0] leaked_r [0:LANES_PER_CYCLE-1];
    logic signed [MEMBRANE_BITS:0] integrated_r [0:LANES_PER_CYCLE-1];
    logic signed [MEMBRANE_BITS-1:0] saturated_r [0:LANES_PER_CYCLE-1];
    logic [LANES_PER_CYCLE-1:0] group_spikes_r;

    function automatic logic signed [MEMBRANE_BITS-1:0] saturate_membrane(
        input logic signed [MEMBRANE_BITS:0] value
    );
        begin
            unique case (value[MEMBRANE_BITS:MEMBRANE_BITS-1])
                2'b01: saturate_membrane = {1'b0, {(MEMBRANE_BITS-1){1'b1}}};
                2'b10: saturate_membrane = {1'b1, {(MEMBRANE_BITS-1){1'b0}}};
                default: saturate_membrane = value[MEMBRANE_BITS-1:0];
            endcase
        end
    endfunction

    function automatic logic [3:0] popcount8(input logic [7:0] value);
        logic [1:0] pair0;
        logic [1:0] pair1;
        logic [1:0] pair2;
        logic [1:0] pair3;
        logic [2:0] half0;
        logic [2:0] half1;
        begin
            pair0 = value[0] + value[1];
            pair1 = value[2] + value[3];
            pair2 = value[4] + value[5];
            pair3 = value[6] + value[7];
            half0 = pair0 + pair1;
            half1 = pair2 + pair3;
            popcount8 = half0 + half1;
        end
    endfunction

    always_comb begin
        input_ready = state == ST_IDLE;
        output_valid = state == ST_OUTPUT;
        for (int lane = 0; lane < LANES; lane++) begin
            membrane_vector[lane*MEMBRANE_BITS +: MEMBRANE_BITS] = membrane[lane];
            pre_reset_membrane_vector[lane*MEMBRANE_BITS +: MEMBRANE_BITS]
                = pre_reset_membrane[lane];
        end
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            state <= ST_IDLE;
            current_vector_r <= '0;
            leak_shift_r <= 5'd0;
            threshold_r <= '0;
            reset_value_r <= '0;
            group_index <= '0;
            spike_work_r <= '0;
            step_spike_count_r <= 7'd0;
            completed_step_spike_count_r <= 7'd0;
            counter_sum_r <= 17'd0;
            group_spikes_r <= '0;
            spikes <= '0;
            timestep_count <= 32'd0;
            total_spike_count <= 64'd0;
            for (int lane = 0; lane < LANES; lane++) begin
                membrane[lane] <= '0;
                pre_reset_membrane[lane] <= '0;
            end
            for (int lane = 0; lane < LANES_PER_CYCLE; lane++) begin
                group_membrane_r[lane] <= '0;
                group_current_r[lane] <= '0;
                leak_shifted_r[lane] <= '0;
```

```
                leaked_r[lane] <= '0;
                integrated_r[lane] <= '0;
                saturated_r[lane] <= '0;
            end
        end else if (clear_state) begin
            state <= ST_IDLE;
            spike_work_r <= '0;
            spikes <= '0;
            timestep_count <= 32'd0;
            for (int lane = 0; lane < LANES; lane++) begin
                membrane[lane] <= reset_value;
                pre_reset_membrane[lane] <= reset_value;
            end
        end else begin
            unique case (state)
                ST_IDLE: begin
                    if (input_valid) begin
                        current_vector_r <= current_vector;
                        leak_shift_r <= leak_shift;
                        threshold_r <= threshold;
                        reset_value_r <= reset_value;
                        group_index <= '0;
                        spike_work_r <= '0;
                        step_spike_count_r <= 7'd0;
                        state <= ST_GROUP_LOAD;
                    end
                end

                ST_GROUP_LOAD: begin
                    for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin
                        logic [$clog2(LANES)-1:0] lane;
                        lane = $clog2(LANES)'(
                            group_index * LANES_PER_CYCLE + local_lane);
                        group_membrane_r[local_lane] <= membrane[lane];
                        group_current_r[local_lane] <= $signed(
                            current_vector_r[lane*CURRENT_BITS +: CURRENT_BITS]);
                    end
                    state <= ST_LEAK;
                end

                ST_LEAK: begin
                    for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin
                        logic signed [MEMBRANE_BITS:0] membrane_extended;
                        membrane_extended = {
                            group_membrane_r[local_lane][MEMBRANE_BITS-1],
                            group_membrane_r[local_lane]};
                        leak_shifted_r[local_lane] <=
                            membrane_extended >>> leak_shift_r;
                    end
                    state <= ST_LEAK_SUBTRACT;
                end

                ST_LEAK_SUBTRACT: begin
                    for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin
                        logic signed [MEMBRANE_BITS:0] membrane_extended;
                        membrane_extended = {
                            group_membrane_r[local_lane][MEMBRANE_BITS-1],
                            group_membrane_r[local_lane]};
                        leaked_r[local_lane] <= membrane_extended
                            - leak_shifted_r[local_lane];
                    end
                    state <= ST_INTEGRATE;
                end

                ST_INTEGRATE: begin
                    for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin
                        integrated_r[local_lane] <= leaked_r[local_lane]
                            + $signed({{(MEMBRANE_BITS+1-CURRENT_BITS){
                                group_current_r[local_lane][CURRENT_BITS-1]}},
                                group_current_r[local_lane]});
                    end
                    state <= ST_SATURATE;
                end

                ST_SATURATE: begin
                    for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin
                        saturated_r[local_lane] <=
                            saturate_membrane(integrated_r[local_lane]);
                    end
                    state <= ST_THRESHOLD_COMPARE;
                end

                ST_THRESHOLD_COMPARE: begin
                    for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin
                        group_spikes_r[local_lane] <=
                            saturated_r[local_lane] >= threshold_r;
                    end
```

```
                    state <= ST_COMMIT;
                end

                ST_COMMIT: begin
                    logic [LANES-1:0] next_spikes;
                    logic [3:0] group_spike_count;
                    next_spikes = spike_work_r;
                    for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin
                        logic [$clog2(LANES)-1:0] lane;
                        lane = $clog2(LANES)'(
                            group_index * LANES_PER_CYCLE + local_lane);
                        pre_reset_membrane[lane] <= saturated_r[local_lane];
                        if (group_spikes_r[local_lane]) begin
                            membrane[lane] <= reset_value_r;
                        end else begin
                            membrane[lane] <= saturated_r[local_lane];
                        end
                    end
                    next_spikes[group_index*LANES_PER_CYCLE +: LANES_PER_CYCLE]
                        = group_spikes_r;
                    group_spike_count = popcount8(group_spikes_r);
                    spike_work_r <= next_spikes;
                    step_spike_count_r <= step_spike_count_r
                        + {3'd0, group_spike_count};

                    if (group_index == $clog2(GROUPS)'(GROUPS - 1)) begin
                        spikes <= next_spikes;
                        timestep_count <= timestep_count + 1'b1;
                        completed_step_spike_count_r <= step_spike_count_r
                            + {3'd0, group_spike_count};
                        state <= ST_COUNT_LOW;
                    end else begin
                        group_index <= group_index + 1'b1;
                        state <= ST_GROUP_LOAD;
                    end
                end

                // The cumulative diagnostic counter is deliberately updated
                // as four 16-bit words. A monolithic 64-bit addition placed a
                // popcount and long carry chain on the 200 MHz neuron path.
                ST_COUNT_LOW: begin
                    counter_sum_r <= {1'b0, total_spike_count[15:0]}
                        + {10'd0, completed_step_spike_count_r};
                    state <= ST_COUNT_WORD1;
                end

                ST_COUNT_WORD1: begin
                    total_spike_count[15:0] <= counter_sum_r[15:0];
                    counter_sum_r <= {1'b0, total_spike_count[31:16]}
                        + {16'd0, counter_sum_r[16]};
                    state <= ST_COUNT_WORD2;
                end

                ST_COUNT_WORD2: begin
                    total_spike_count[31:16] <= counter_sum_r[15:0];
                    counter_sum_r <= {1'b0, total_spike_count[47:32]}
                        + {16'd0, counter_sum_r[16]};
                    state <= ST_COUNT_WORD3;
                end

                ST_COUNT_WORD3: begin
                    total_spike_count[47:32] <= counter_sum_r[15:0];
                    counter_sum_r <= {1'b0, total_spike_count[63:48]}
                        + {16'd0, counter_sum_r[16]};
                    state <= ST_COUNT_DONE;
                end

                ST_COUNT_DONE: begin
                    total_spike_count[63:48] <= counter_sum_r[15:0];
                    state <= ST_OUTPUT;
                end

                ST_OUTPUT: if (output_ready) state <= ST_IDLE;
                default: state <= ST_IDLE;
            endcase
            if (clear_counters) total_spike_count <= 64'd0;
        end
    end

    initial begin
        if (LANES != 32) $fatal(1, "neuraedge_lif_bank currently requires LANES=32");
        if (LANES_PER_CYCLE != 8) $fatal(1, "neuraedge_lif_bank currently requires LANES_PER_CYCLE=8");
        if (LANES % LANES_PER_CYCLE != 0) $fatal(1, "LANES_PER_CYCLE must divide LANES");
        if (CURRENT_BITS >= MEMBRANE_BITS) $fatal(1, "CURRENT_BITS must be smaller than MEMBRANE_BITS");
    end
endmodule

```

```
305 `default_nettype wire
```

12.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_lif_bank #(` This opens neuraedge_lif_bank, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting.

Line 5: `parameter int LANES = 32,` This statement continues through source line 27; the explanation uses the complete expression rather than only the first visual fragment. This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 6: `parameter int LANES_PER_CYCLE = 8,` This is continuation line 2 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'parameter int LANES_PER_CYCLE = 8,'. The important operands are parameter: the value named 'parameter' in this block; LANES_PER_CYCLE: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 7: `parameter int MEMBRANE_BITS = 24,` This is continuation line 3 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'parameter int MEMBRANE_BITS = 24,'. The important operands are parameter: the value named 'parameter' in this block; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 8: `parameter int CURRENT_BITS = 16` This is continuation line 4 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'parameter int CURRENT_BITS = 16'. The important operands are parameter: the value named 'parameter' in this block; CURRENT_BITS: the value named 'current bits' in this block. Read together,

the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 9: `) (` This is continuation line 5 of the 23-line logical statement that begins at source line 5. Its exact fragment is ‘) (‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 10: `input wire clk,` This is continuation line 6 of the 23-line logical statement that begins at source line 5. Its exact fragment is ‘input wire clk,‘. The important operands are input: the value named ’input’ in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 11: `input wire rst_n,` This is continuation line 7 of the 23-line logical statement that begins at source line 5. Its exact fragment is ‘input wire rst_n,‘. The important operands are input: the value named ’input’ in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 12: `input wire clear_state,` This is continuation line 8 of the 23-line logical statement that begins at source line 5. Its exact fragment is ‘input wire clear_state,‘. The important operands are input: the value named ’input’ in this block; clear_state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during

elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 13: `input wire clear_counters,` This is continuation line 9 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'input wire clear_counters,'. The important operands are input: the value named 'input' in this block; clear_counters: a hardware telemetry register that accumulates measured events. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 14: `input wire input_valid,` This is continuation line 10 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'input wire input_valid,'. The important operands are input: the value named 'input' in this block; input_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 15: `output logic input_ready,` This is continuation line 11 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'output logic input_ready,'. The important operands are output: the value named 'output' in this block; input_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 16: `input wire [LANES*CURRENT_BITS-1:0] current_vector,` This is continuation line 12 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'input wire [LANES*CURRENT_BITS-1:0] current_vector,'. The important operands are input: the value named 'input' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; CURRENT_BITS: the value named 'current bits' in this block. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 17: `input wire [4:0] leak_shift,` This is continuation line 13 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'input wire [4:0] leak_shift,'. The important operands are input: the value named 'input' in this block; leak_shift: the retained fraction of prior membrane state. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 18: `input wire signed [MEMBRANE_BITS-1:0] threshold,` This is continuation line 14 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [MEMBRANE_BITS-1:0] threshold,'. The important operands are input: the value named 'input' in this block; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps; threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 19: `input wire signed [MEMBRANE_BITS-1:0] reset_value,` This is continuation line 15 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [MEMBRANE_BITS-1:0] reset_value,'. The important operands are input: the value named 'input' in this block; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps; reset_value: the value named 'reset value' in this block. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 20: `output logic output_valid,` This is continuation line 16 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'output logic output_valid,'. The important operands are output: the value named 'output' in this block; output_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 21: `input wire output_ready,` This is continuation line 17 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'input wire output_ready,'. The important operands are input: the value named 'input' in this block; output_ready: the consumer's statement that it can accept a payload this cycle. Read together,

the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 22: `output logic [LANES-1:0] spikes,` This is continuation line 18 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'output logic [LANES-1:0] spikes,'. The important operands are output: the value named 'output' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 23: `output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector,` This is continuation line 19 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector,'. The important operands are output: the value named 'output' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 24: `output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector,` This is continuation line 20 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector,'. The important operands are output: the value named 'output' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 25: `output logic [31:0] timestep_count,` This is continuation line 21 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] timestep_count,'. The important operands are output: the value named 'output' in this block; timestep_count: one discrete SNN update instant. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state,

input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 26: `output logic [63:0] total_spike_count` This is continuation line 22 of the 23-line logical statement that begins at source line 5. Its exact fragment is 'output logic [63:0] total_spike_count'. The important operands are output: the value named 'output' in this block; total_spike_count: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 27: `);` This is continuation line 23 of the 23-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes LANES to 32, parameter int LANES_PER_CYCLE = 8, parameter int MEMBRANE_BITS = 24, parameter int CURRENT_BITS = 16) (input wire clk, input wire rst_n, input wire clear_state, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*CURRENT_BITS-1:0] current_vector, input wire [4:0] leak_shift, input wire signed [MEMBRANE_BITS-1:0] threshold, input wire signed [MEMBRANE_BITS-1:0] reset_value, output logic output_valid, input wire output_ready, output logic [LANES-1:0] spikes, output logic [LANES*MEMBRANE_BITS-1:0] membrane_vector, output logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector, output logic [31:0] timestep_count, output logic [63:0] total_spike_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 28: `localparam int GROUPS = LANES / LANES_PER_CYCLE;` This fixes GROUPS to LANES / LANES_PER_CYCLE during elaboration. Vivado uses that constant to shape or simplify the generated circuit; GROUPS denotes the value named 'groups' in this block.

Line 30: `typedef enum logic [3:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 31: `ST_IDLE = 4'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 32: `ST_GROUP_LOAD = 4'd1,` This assigns a fixed binary encoding to state ST_GROUP_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 33: `ST_LEAK = 4'd2,` This assigns a fixed binary encoding to state ST_LEAK, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 34: `ST_INTEGRATE = 4'd3,` This assigns a fixed binary encoding to state ST_INTEGRATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 35: `ST_SATURATE = 4'd4,` This assigns a fixed binary encoding to state ST_SATURATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 36: `ST_COMMIT = 4'd5,` This assigns a fixed binary encoding to state ST_COMMIT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 37: `ST_OUTPUT = 4'd6,` This assigns a fixed binary encoding to state ST_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 38: `ST_LEAK_SUBTRACT = 4'd7,` This assigns a fixed binary encoding to state ST_LEAK_SUBTRACT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 39: `ST_THRESHOLD_COMPARE = 4'd8,` This assigns a fixed binary encoding to state ST_THRESHOLD_COMPARE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 40: `ST_COUNT_LOW = 4'd9,` This assigns a fixed binary encoding to state ST_COUNT_LOW, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 41: `ST_COUNT_WORD1 = 4'd10,` This assigns a fixed binary encoding to state ST_COUNT_WORD1, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 42: `ST_COUNT_WORD2 = 4'd11,` This assigns a fixed binary encoding to state ST_COUNT_WORD2, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 43: `ST_COUNT_WORD3 = 4'd12,` This assigns a fixed binary encoding to state ST_COUNT_WORD3, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 44: `ST_COUNT_DONE = 4'd13` This statement continues through source line 45; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_COUNT_DONE, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 45: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 44. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_COUNT_DONE, which updates a segmented telemetry counter without a long carry chain. The encoding is visible in waveforms even though the source uses the readable name.

Line 47: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 48: `logic signed [MEMBRANE_BITS-1:0] membrane [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, membrane is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 49: `logic signed [MEMBRANE_BITS-1:0] pre_reset_membrane [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pre_reset_membrane is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 50: `logic [LANES*CURRENT_BITS-1:0] current_vector_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, current_vector_r is the value named 'current vector r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 51: `logic [4:0] leak_shift_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, leak_shift_r is the retained fraction of prior membrane state. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 52: `logic signed [MEMBRANE_BITS-1:0] threshold_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, threshold_r is the membrane level at which a neuron emits a spike. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 53: `logic signed [MEMBRANE_BITS-1:0] reset_value_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reset_value_r is the value named 'reset value r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 54: `logic [$clog2(GROUPS)-1:0] group_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, group_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 55: `logic [LANES-1:0] spike_work_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, spike_work_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 56: `logic [6:0] step_spike_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, step_spike_count_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 57: `logic [6:0] completed_step_spike_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, completed_step_spike_count_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 58: `logic [16:0] counter_sum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, counter_sum_r is a hardware telemetry register that accumulates measured events. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 59: `logic signed [MEMBRANE_BITS-1:0] group_membrane_r [0:LANES_PER_CYCLE-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, group_membrane_r is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 60: `logic signed [CURRENT_BITS-1:0] group_current_r [0:LANES_PER_CYCLE-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, group_current_r is the value named 'group current r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 61: `logic signed [MEMBRANE_BITS:0] leak_shifted_r [0:LANES_PER_CYCLE-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, leak_shifted_r is the retained fraction of prior membrane state. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 62: `logic signed [MEMBRANE_BITS:0] leaked_r [0:LANES_PER_CYCLE-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, leaked_r is the retained fraction of prior membrane state. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 63: `logic signed [MEMBRANE_BITS:0] integrated_r [0:LANES_PER_CYCLE-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, integrated_r is the value named 'integrated r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 64: `logic signed [MEMBRANE_BITS-1:0] saturated_r [0:LANES_PER_CYCLE-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, saturated_r is the value named 'saturated r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 65: `logic [LANES_PER_CYCLE-1:0] group_spikes_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, group_spikes_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 67: `function automatic logic signed [MEMBRANE_BITS-1:0] saturate_membrane(` Within the combinational helper saturate membrane, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps.

Line 68: `input logic signed [MEMBRANE_BITS:0] value` This declares value as a input port with width [MEMBRANE_BITS:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 69: `);` Within the combinational helper saturate membrane, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The expression is a literal or structural operator and has no hidden software state.

Line 70: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate membrane; it has no arithmetic value of its own.

Line 71: `unique case (value[MEMBRANE_BITS:MEMBRANE_BITS-1])` This starts a multiplexer/state decode selected by ‘value[MEMBRANE_BITS:MEMBRANE_BITS-1]‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 72: `2'b01: saturate_membrane = {1'b0, {(MEMBRANE_BITS-1){1'b1}}};` Within the combinational helper saturate membrane, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are b01: the value named 'b01' in this block; saturate_membrane: the persistent LIF neuron state carrying charge between timesteps; b0: the value named 'b0' in this block.

Line 73: `2'b10: saturate_membrane = {1'b1, {(MEMBRANE_BITS-1){1'b0}}};` Within the combinational helper saturate membrane, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are b10: the value named 'b10' in this block; saturate_membrane: the persistent LIF neuron state carrying charge between timesteps; b1: the value named 'b1' in this block.

Line 74: `default:  saturate_membrane = value[MEMBRANE_BITS-1:0];` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'saturate_membrane = value[MEMBRANE_BITS-1:0];'; context is the combinational helper saturate membrane.

Line 75: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate membrane; it has no arithmetic value of its own.

Line 76: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate membrane; it has no arithmetic value of its own.

Line 77: `endfunction` Within the combinational helper saturate membrane, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are endfunction: the value named 'endfunction' in this block.

Line 79: `function automatic logic [3:0] popcount8(input logic [7:0] value);` Within the combinational helper popcount8, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; popcount8: the value named 'popcount8' in this block.

Line 80: `logic [1:0] pair0;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, pair0 is the value named 'pair0' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 81: `logic [1:0] pair1;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, pair1 is the value named 'pair1' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 82: `logic [1:0] pair2;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, pair2 is the value named 'pair2' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 83: `logic [1:0] pair3;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, pair3 is the value named 'pair3' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 84: `logic [2:0] half0;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, half0 is the value named 'half0' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 85: `logic [2:0] half1;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, half1 is the value named 'half1' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 86: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper popcount8; it has no arithmetic value of its own.

Line 87: `pair0 = value[0] + value[1];` This makes pair0, the value named 'pair0' in this block, equal to 'value[0] + value[1]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount8.

Line 88: `pair1 = value[2] + value[3];` This makes pair1, the value named 'pair1' in this block, equal to 'value[2] + value[3]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount8.

Line 89: `pair2 = value[4] + value[5];` This makes pair2, the value named 'pair2' in this block, equal to 'value[4] + value[5]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount8.

Line 90: `pair3 = value[6] + value[7];` This makes pair3, the value named 'pair3' in this block, equal to 'value[6] + value[7]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount8.

Line 91: `half0 = pair0 + pair1;` This makes half0, the value named 'half0' in this block, equal to 'pair0 + pair1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pair0: the value named 'pair0' in this block; pair1: the value named 'pair1' in this block. Context: the combinational helper popcount8.

Line 92: `half1 = pair2 + pair3;` This makes half1, the value named 'half1' in this block, equal to 'pair2 + pair3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pair2: the value named 'pair2' in this block; pair3: the value named 'pair3' in this block. Context: the combinational helper popcount8.

Line 93: `popcount8 = half0 + half1;` This makes popcount8, the value named 'popcount8' in this block, equal to 'half0 + half1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are half0: the value named 'half0' in this block; half1: the value named 'half1' in this block. Context: the combinational helper popcount8.

Line 94: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper popcount8; it has no arithmetic value of its own.

Line 95: `endfunction` Within the combinational helper popcount8, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are endfunction: the value named 'endfunction' in this block.

Line 97: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 98: `input_ready = state == ST_IDLE;` This makes input_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_IDLE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 99: `output_valid = state == ST_OUTPUT;` This makes output_valid, the producer's statement that its current payload is meaningful, equal to 'state == ST_OUTPUT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT: the value named 'st output' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 100: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 101: `membrane_vector[lane*MEMBRANE_BITS +: MEMBRANE_BITS] = membrane[lane];` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are membrane_vector: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps.

Line 102: `pre_reset_membrane_vector[lane*MEMBRANE_BITS +: MEMBRANE_BITS]` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are pre_reset_membrane_vector: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps.

Line 103: `= pre_reset_membrane[lane];` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 104: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 105: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 107: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 108: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 109: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 110: `current_vector_r <= '0;` This makes current_vector_r, the value named 'current vector r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 111: `leak_shift_r <= 5'd0;` This makes leak_shift_r, the retained fraction of prior membrane state, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 112: `threshold_r <= '0;` This makes threshold_r, the membrane level at which a neuron emits a spike, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 113: `reset_value_r <= '0;` This makes reset_value_r, the value named 'reset value r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 114: `group_index <= '0;` This makes group_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 115: `spike_work_r <= '0;` This makes spike_work_r, a binary event produced when a neuron crosses its threshold, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 116: `step_spike_count_r <= 7'd0;` This makes step_spike_count_r, a binary event produced when a neuron crosses its threshold, equal to ‘7'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 117: `completed_step_spike_count_r <= 7'd0;` This makes completed_step_spike_count_r, a binary event produced when a neuron crosses its threshold, equal to ‘7'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 118: `counter_sum_r <= 17'd0;` This makes counter_sum_r, a hardware telemetry register that accumulates measured events, equal to ‘17'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 119: `group_spikes_r <= '0;` This makes group_spikes_r, a binary event produced when a neuron crosses its threshold, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 120: `spikes <= '0;` This makes spikes, a binary event produced when a neuron crosses its threshold, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 121: `timestep_count <= 32'd0;` This makes timestep_count, one discrete SNN update instant, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 122: `total_spike_count <= 64'd0;` This makes total_spike_count, a binary event produced when a neuron crosses its threshold, equal to ‘64'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 123: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 124: `membrane[lane] <= '0;` This makes membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 125: `pre_reset_membrane[lane] <= '0;` This makes pre_reset_membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 126: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 127: `for (int lane = 0; lane < LANES_PER_CYCLE; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 128: `group_membrane_r[lane] <= '0;` This makes group_membrane_r[lane], the persistent LIF neuron state carrying charge between timesteps, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 129: `group_current_r[lane] <= '0;` This makes group_current_r[lane], the value named 'group current r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 130: `leak_shifted_r[lane] <= '0;` This makes leak_shifted_r[lane], the retained fraction of prior membrane state, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 131: `leaked_r[lane] <= '0;` This makes leaked_r[lane], the retained fraction of prior membrane state, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 132: `integrated_r[lane] <= '0;` This makes integrated_r[lane], the value named 'integrated r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 133: `saturated_r[lane] <= '0;` This makes saturated_r[lane], the value named 'saturated r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 134: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 135: `end else if (clear_state) begin` This condition selects the following hardware branch only when 'clear_state' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_state: the finite-state-machine phase retained in flip-flops. Context: the clocked process that updates registers only on a rising clock edge.

Line 136: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 137: `spike_work_r <= '0;` This makes spike_work_r, a binary event produced when a neuron crosses its threshold, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 138: `spikes <= '0;` This makes spikes, a binary event produced when a neuron crosses its threshold, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 139: `timestep_count <= 32'd0;` This makes timestep_count, one discrete SNN update instant, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 140: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 141: `membrane[lane] <= reset_value;` This makes membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to ‘reset_value‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are reset_value: the value named 'reset value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 142: `pre_reset_membrane[lane] <= reset_value;` This makes pre_reset_membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to ‘reset_value‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are reset_value: the value named 'reset value' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 143: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 144: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 145: `unique case (state)` This starts a multiplexer/state decode selected by ‘state‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 146: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 147: `if (input_valid) begin` This condition selects the following hardware branch only when ‘input_valid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 148: `current_vector_r <= current_vector;` This makes current_vector_r, the value named 'current vector r' in this block, equal to ‘current_vector‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are current_vector: the value named 'current vector' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 149: `leak_shift_r <= leak_shift;` This makes leak_shift_r, the retained fraction of prior membrane state, equal to ‘leak_shift‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are leak_shift: the retained fraction of prior membrane state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 150: `threshold_r <= threshold;` This makes threshold_r, the membrane level at which a neuron emits a spike, equal to ‘threshold‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are threshold: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 151: `reset_value_r <= reset_value;` This makes reset_value_r, the value named ’reset value r’ in this block, equal to ‘reset_value‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are reset_value: the value named ’reset value’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 152: `group_index <= '0;` This makes group_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘’0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 153: `spike_work_r <= '0;` This makes spike_work_r, a binary event produced when a neuron crosses its threshold, equal to ‘’0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 154: `step_spike_count_r <= 7'd0;` This makes step_spike_count_r, a binary event produced when a neuron crosses its threshold, equal to ‘7’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 155: `state <= ST_GROUP_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_GROUP_LOAD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_GROUP_LOAD: the value named ’st group load’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 156: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 157: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 159: `ST_GROUP_LOAD: begin` This is the ST_GROUP_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory..

Line 160: `for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory..

Line 161: `logic [$clog2(LANES)-1:0] lane;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory.. Specifically, lane is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 162: `lane = $clog2(LANES)'(` This statement continues through source line 163; the explanation uses the complete expression rather than only the first visual fragment. This makes lane, one parallel arithmetic element within a vector or tensor datapath, equal to '$clog2(LANES)'(group_index * LANES_PER_CYCLE + local_lane)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are clog2: the value named 'clog2' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; group_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory..

Line 163: `group_index * LANES_PER_CYCLE + local_lane);` This is continuation line 2 of the 2-line logical statement that begins at source line 162. Its exact fragment is 'group_index * LANES_PER_CYCLE + local_lane);'. The important operands are group_index: an integer selecting one tensor element, lane, bank, or state entry; LANES_PER_CYCLE: one parallel arithmetic element within a vector or tensor datapath; local_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes lane, one parallel arithmetic element within a vector or tensor datapath, equal to '$clog2(LANES)'(group_index * LANES_PER_CYCLE + local_lane)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are clog2: the value named 'clog2' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; group_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory..

Line 164: `group_membrane_r[local_lane] <= membrane[lane];` This makes group_membrane_r[local_lane], the persistent LIF neuron state carrying charge between timesteps, equal to 'membrane[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory..

Line 165: `group_current_r[local_lane] <= $signed(` This statement continues through source line 166; the explanation uses the complete expression rather than only the first visual fragment. This makes group_current_r[local_lane], the value named 'group current r' in this block, equal to '$signed(current_vector_r[lane*CURRENT_BITS +: CURRENT_BITS])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are current_vector_r: the value named 'current vector r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; CURRENT_BITS: the value named 'current bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory..

Line 166: `current_vector_r[lane*CURRENT_BITS +: CURRENT_BITS]);` This is continuation line 2 of the 2-line logical statement that begins at source line 165. Its exact fragment is 'current_vector_r[lane*CURRENT_BITS +: CURRENT_BITS]);'. The important operands are current_vector_r: the value named 'current vector r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; CURRENT_BITS: the value named 'current bits' in this block. Read together, the complete operation is: This makes group_current_r[local_lane], the value named 'group current r' in this block, equal to '$signed(current_vector_r[lane*CURRENT_BITS +: CURRENT_BITS])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are current_vector_r: the value named 'current vector r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; CURRENT_BITS: the value named 'current bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory..

Line 167: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands

into local FPGA memory.; it has no arithmetic value of its own.

Line 168: `state <= ST_LEAK;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LEAK‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LEAK: the retained fraction of prior membrane state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory..

Line 169: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_GROUP_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 171: `ST_LEAK: begin` This is the ST_LEAK case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase..

Line 172: `for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase..

Line 173: `logic signed [MEMBRANE_BITS:0] membrane_extended;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase.. Specifically, membrane_extended is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 174: `membrane_extended = {` This statement continues through source line 176; the explanation uses the complete expression rather than only the first visual fragment. This makes membrane_extended, the persistent LIF neuron state carrying charge between timesteps, equal to ‘{ group_membrane_r[local_lane][MEMBRANE_BITS-1], group_membrane_r[local_lane]}‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase..

Line 175: `group_membrane_r[local_lane][MEMBRANE_BITS-1],` This is continuation line 2 of the 3-line logical statement that begins at source line 174. Its exact fragment is ‘group_membrane_r[local_lane][MEMBRANE_BITS-1],‘. The important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This makes membrane_extended, the persistent LIF neuron state carrying charge between timesteps, equal to ‘{ group_membrane_r[local_lane][ME
1], group_membrane_r[local_lane]}‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase..

Line 176: `group_membrane_r[local_lane]};` This is continuation line 3 of the 3-line logical statement that begins at source line 174. Its exact fragment is ‘group_membrane_r[local_lane]};‘. The important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes membrane_extended, the persistent LIF neuron state carrying charge between timesteps, equal to ‘{ group_membrane_r[local_lane][MEMBRANE_BITS-1], group_membrane_r[local_lane]}‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF

neuron state carrying charge between timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase..

Line 177: `leak_shifted_r[local_lane] <=` This statement continues through source line 178; the explanation uses the complete expression rather than only the first visual fragment. This makes leak_shifted_r[local_lane], the retained fraction of prior membrane state, equal to 'membrane_extended >>> leak_shift_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are membrane_extended: the persistent LIF neuron state carrying charge between timesteps; leak_shift_r: the retained fraction of prior membrane state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase..

Line 178: `membrane_extended >>> leak_shift_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 177. Its exact fragment is 'membrane_extended >>> leak_shift_r;'. The important operands are membrane_extended: the persistent LIF neuron state carrying charge between timesteps; leak_shift_r: the retained fraction of prior membrane state. Read together, the complete operation is: This makes leak_shifted_r[local_lane], the retained fraction of prior membrane state, equal to 'membrane_extended >>> leak_shift_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are membrane_extended: the persistent LIF neuron state carrying charge between timesteps; leak_shift_r: the retained fraction of prior membrane state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase..

Line 179: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 180: `state <= ST_LEAK_SUBTRACT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LEAK_SUBTRACT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LEAK_SUBTRACT: the retained fraction of prior membrane state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase..

Line 181: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LEAK, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 183: `ST_LEAK_SUBTRACT: begin` This is the ST_LEAK_SUBTRACT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase..

Line 184: `for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase..

Line 185: `logic signed [MEMBRANE_BITS:0] membrane_extended;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase.. Specifically, membrane_extended is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 186: `membrane_extended = {` This statement continues through source line 188; the explanation uses the complete expression rather than only the first visual fragment. This makes membrane_extended, the persistent LIF neuron state carrying charge between timesteps, equal to '{ group_membrane_r[local_lane][MEMBRANE_BITS-1], group_membrane_r[local_lane]}'. It is driven by combinational or procedural logic in the current evaluation. The

important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase..

Line 187: `group_membrane_r[local_lane][MEMBRANE_BITS-1],` This is continuation line 2 of the 3-line logical statement that begins at source line 186. Its exact fragment is 'group_membrane_r[local_lane][MEMBRANE_BITS-1],'. The important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This makes membrane_extended, the persistent LIF neuron state carrying charge between timesteps, equal to '{ group_membrane_r[local_lane][ME 1], group_membrane_r[local_lane]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase..

Line 188: `group_membrane_r[local_lane]};` This is continuation line 3 of the 3-line logical statement that begins at source line 186. Its exact fragment is 'group_membrane_r[local_lane]};'. The important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes membrane_extended, the persistent LIF neuron state carrying charge between timesteps, equal to '{ group_membrane_r[local_lane][MEMBRANE_BITS-1], group_membrane_r[local_lane]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are group_membrane_r: the persistent LIF neuron state carrying charge between timesteps; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase..

Line 189: `leaked_r[local_lane] <= membrane_extended` This statement continues through source line 190; the explanation uses the complete expression rather than only the first visual fragment. This makes leaked_r[local_lane], the retained fraction of prior membrane state, equal to 'membrane_extended - leak_shifted_r[local_lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are membrane_extended: the persistent LIF neuron state carrying charge between timesteps; leak_shifted_r: the retained fraction of prior membrane state; local_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase..

Line 190: `- leak_shifted_r[local_lane];` This is continuation line 2 of the 2-line logical statement that begins at source line 189. Its exact fragment is '- leak_shifted_r[local_lane];'. The important operands are leak_shifted_r: the retained fraction of prior membrane state; local_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes leaked_r[local_lane], the retained fraction of prior membrane state, equal to 'membrane_extended - leak_shifted_r[local_lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are membrane_extended: the persistent LIF neuron state carrying charge between timesteps; leak_shifted_r: the retained fraction of prior membrane state; local_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase..

Line 191: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 192: `state <= ST_INTEGRATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_INTEGRATE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_INTEGRATE: the value named 'st integrate' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase..

Line 193: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LEAK_SUBTRACT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 195: `ST_INTEGRATE: begin` This is the ST_INTEGRATE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_INTEGRATE, the machine performs the named control phase..

Line 196: `for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_INTEGRATE, the machine performs the named control phase..

Line 197: `integrated_r[local_lane] <= leaked_r[local_lane]` This statement continues through source line 200; the explanation uses the complete expression rather than only the first visual fragment. This makes integrated_r[local_lane], the value named 'integrated r' in this block, equal to 'leaked_r[local_lane] + $signed({{(MEMBRANE_BITS+1-CURRENT_BITS){ group_current_r[local_lane][CURRENT_BITS-1]}}, group_current_r[local_lane]})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are leaked_r: the retained fraction of prior membrane state; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_INTEGRATE, the machine performs the named control phase..

Line 198: `+ $signed({{(MEMBRANE_BITS+1-CURRENT_BITS){` This is continuation line 2 of the 4-line logical statement that begins at source line 197. Its exact fragment is '+ $signed({{(MEMBRANE_BITS+1-CURRENT_BITS){'. The important operands are MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps; CURRENT_BITS: the value named 'current bits' in this block. Read together, the complete operation is: This makes integrated_r[local_lane], the value named 'integrated r' in this block, equal to 'leaked_r[local_lane] + $signed({{(MEMBRANE_BITS+CURRENT_BITS){ group_current_r[local_lane][CURRENT_BITS-1]}}, group_current_r[local_lane]})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are leaked_r: the retained fraction of prior membrane state; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_INTEGRATE, the machine performs the named control phase..

Line 199: `group_current_r[local_lane][CURRENT_BITS-1]}},` This is continuation line 3 of the 4-line logical statement that begins at source line 197. Its exact fragment is 'group_current_r[local_lane][CURRENT_BITS-1]}},'. The important operands are group_current_r: the value named 'group current r' in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath; CURRENT_BITS: the value named 'current bits' in this block. Read together, the complete operation is: This makes integrated_r[local_lane], the value named 'integrated r' in this block, equal to 'leaked_r[local_lane] + $signed({{(MEMBRANE_BITS+1-CURRENT_BITS){ group_current_r[local_lane][CURRENT
1]}}, group_current_r[local_lane]})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are leaked_r: the retained fraction of prior membrane state; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_INTEGRATE, the machine performs the named control phase..

Line 200: `group_current_r[local_lane]});` This is continuation line 4 of the 4-line logical statement that begins at source line 197. Its exact fragment is 'group_current_r[local_lane]});'. The important operands are group_current_r: the value named 'group current r' in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes integrated_r[local_lane], the value named 'integrated r' in this block, equal to 'leaked_r[local_lane] + $signed({{(MEMBRANE_BITS+1-CURRENT_BITS){ group_current_r[local_lane][CUR
1]}}, group_current_r[local_lane]})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are leaked_r: the retained fraction of prior membrane state; local_lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between

timesteps. Context: the clocked process that updates registers only on a rising clock edge Within state ST_INTEGRATE, the machine performs the named control phase..

Line 201: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_INTEGRATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 202: `state <= ST_SATURATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_SATURATE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SATURATE: the value named 'st saturate' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_INTEGRATE, the machine performs the named control phase..

Line 203: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_INTEGRATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 205: `ST_SATURATE: begin` This is the ST_SATURATE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 206: `for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 207: `saturated_r[local_lane] <=` This statement continues through source line 208; the explanation uses the complete expression rather than only the first visual fragment. This makes saturated_r[local_lane], the value named 'saturated r' in this block, equal to 'saturate_membrane(integrated_r[local_lane])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_membrane: the persistent LIF neuron state carrying charge between timesteps; integrated_r: the value named 'integrated r' in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 208: `saturate_membrane(integrated_r[local_lane]);` This is continuation line 2 of the 2-line logical statement that begins at source line 207. Its exact fragment is 'saturate_membrane(integrated_r[local_lane]);'. The important operands are saturate_membrane: the persistent LIF neuron state carrying charge between timesteps; integrated_r: the value named 'integrated r' in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes saturated_r[local_lane], the value named 'saturated r' in this block, equal to 'saturate_membrane(integrated_r[local_lane])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_membrane: the persistent LIF neuron state carrying charge between timesteps; integrated_r: the value named 'integrated r' in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 209: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 210: `state <= ST_THRESHOLD_COMPARE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_THRESHOLD_COMPARE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_THRESHOLD_COMPARE: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 211: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 213: `ST_THRESHOLD_COMPARE: begin` This is the ST_THRESHOLD_COMPARE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_THRESHOLD_COMPARE, the machine performs the named control phase..

Line 214: `for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_THRESHOLD_COMPARE, the machine performs the named control phase..

Line 215: `group_spikes_r[local_lane] <=` This statement continues through source line 216; the explanation uses the complete expression rather than only the first visual fragment. This makes group_spikes_r[local_lane], a binary event produced when a neuron crosses its threshold, equal to ‘saturated_r[local_lane] >= threshold_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturated_r: the value named ’saturated r’ in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath; threshold_r: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_THRESHOLD_COMPARE, the machine performs the named control phase..

Line 216: `saturated_r[local_lane] >= threshold_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 215. Its exact fragment is ‘saturated_r[local_lane] >= threshold_r;‘. The important operands are saturated_r: the value named ’saturated r’ in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath; threshold_r: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This makes group_spikes_r[local_lane], a binary event produced when a neuron crosses its threshold, equal to ‘saturated_r[local_lane] >= threshold_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturated_r: the value named ’saturated r’ in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath; threshold_r: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_THRESHOLD_COMPARE, the machine performs the named control phase..

Line 217: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_THRESHOLD_COMPARE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 218: `state <= ST_COMMIT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_COMMIT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMMIT: the value named ’st commit’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_THRESHOLD_COMPARE, the machine performs the named control phase..

Line 219: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_THRESHOLD_COMPARE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 221: `ST_COMMIT: begin` This is the ST_COMMIT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 222: `logic [LANES-1:0] next_spikes;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control

phase.. Specifically, next_spikes is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 223: `logic [3:0] group_spike_count;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.. Specifically, group_spike_count is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 224: `next_spikes = spike_work_r;` This makes next_spikes, a binary event produced when a neuron crosses its threshold, equal to ‘spike_work_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are spike_work_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 225: `for (int local_lane = 0; local_lane < LANES_PER_CYCLE; local_lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 226: `logic [$clog2(LANES)-1:0] lane;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.. Specifically, lane is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 227: `lane = $clog2(LANES)'(` This statement continues through source line 228; the explanation uses the complete expression rather than only the first visual fragment. This makes lane, one parallel arithmetic element within a vector or tensor datapath, equal to ‘$clog2(LANES)'(group_index * LANES_PER_CYCLE + local_lane)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are clog2: the value named 'clog2' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; group_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 228: `group_index * LANES_PER_CYCLE + local_lane);` This is continuation line 2 of the 2-line logical statement that begins at source line 227. Its exact fragment is ‘group_index * LANES_PER_CYCLE + local_lane);‘. The important operands are group_index: an integer selecting one tensor element, lane, bank, or state entry; LANES_PER_CYCLE: one parallel arithmetic element within a vector or tensor datapath; local_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes lane, one parallel arithmetic element within a vector or tensor datapath, equal to ‘$clog2(LANES)'(group_index * LANES_PER_CYCLE + local_lane)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are clog2: the value named 'clog2' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; group_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 229: `pre_reset_membrane[lane] <= saturated_r[local_lane];` This makes pre_reset_membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to ‘saturated_r[local_lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturated_r: the value named 'saturated r' in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 230: `if (group_spikes_r[local_lane]) begin` This condition selects the following hardware branch only when ‘group_spikes_r[local_lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are group_spikes_r: a binary event produced when a neuron crosses its threshold; local_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 231: `membrane[lane] <= reset_value_r;` This makes membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to ‘reset_value_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are reset_value_r: the value named ’reset value r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 232: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 233: `membrane[lane] <= saturated_r[local_lane];` This makes membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to ‘saturated_r[local_lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturated_r: the value named ’saturated r’ in this block; local_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 234: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 235: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 236: `next_spikes[group_index*LANES_PER_CYCLE +: LANES_PER_CYCLE]` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are next_spikes: a binary event produced when a neuron crosses its threshold; group_index: an integer selecting one tensor element, lane, bank, or state entry; LANES_PER_CYCLE: one parallel arithmetic element within a vector or tensor datapath.

Line 237: `= group_spikes_r;` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting. The important operands are group_spikes_r: a binary event produced when a neuron crosses its threshold.

Line 238: `group_spike_count = popcount8(group_spikes_r);` This makes group_spike_count, a binary event produced when a neuron crosses its threshold, equal to ‘popcount8(group_spikes_r)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are popcount8: the value named ’popcount8’ in this block; group_spikes_r: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 239: `spike_work_r <= next_spikes;` This makes spike_work_r, a binary event produced when a neuron crosses its threshold, equal to ‘next_spikes‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_spikes: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 240: `step_spike_count_r <= step_spike_count_r` This statement continues through source line 241; the explanation uses the complete expression rather than only the first visual fragment. This makes step_spike_count_r, a binary event produced when a neuron crosses its threshold, equal to ‘step_spike_count_r + {3’d0, group_spike_count}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are step_spike_count_r: a binary event produced when a neuron crosses its threshold; d0: the value named ’d0’ in this

block; group_spike_count: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 241: `+ {3'd0, group_spike_count};` This is continuation line 2 of the 2-line logical statement that begins at source line 240. Its exact fragment is '+ {3'd0, group_spike_count};'. The important operands are d0: the value named 'd0' in this block; group_spike_count: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This makes step_spike_count_r, a binary event produced when a neuron crosses its threshold, equal to 'step_spike_count_r + {3'd0, group_spike_count}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are step_spike_count_r: a binary event produced when a neuron crosses its threshold; d0: the value named 'd0' in this block; group_spike_count: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 243: `if (group_index == $clog2(GROUPS)'(GROUPS - 1)) begin` This condition selects the following hardware branch only when 'group_index == $clog2(GROUPS)'(GROUPS - 1)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are group_index: an integer selecting one tensor element, lane, bank, or state entry; clog2: the value named 'clog2' in this block; GROUPS: the value named 'groups' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 244: `spikes <= next_spikes;` This makes spikes, a binary event produced when a neuron crosses its threshold, equal to 'next_spikes'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_spikes: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 245: `timestep_count <= timestep_count + 1'b1;` This makes timestep_count, one discrete SNN update instant, equal to 'timestep_count + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are timestep_count: one discrete SNN update instant; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 246: `completed_step_spike_count_r <= step_spike_count_r` This statement continues through source line 247; the explanation uses the complete expression rather than only the first visual fragment. This makes completed_step_spike_cour a binary event produced when a neuron crosses its threshold, equal to 'step_spike_count_r + {3'd0, group_spike_count}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are step_spike_count_r: a binary event produced when a neuron crosses its threshold; d0: the value named 'd0' in this block; group_spike_count: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 247: `+ {3'd0, group_spike_count};` This is continuation line 2 of the 2-line logical statement that begins at source line 246. Its exact fragment is '+ {3'd0, group_spike_count};'. The important operands are d0: the value named 'd0' in this block; group_spike_count: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This makes completed_step_spike_count_r, a binary event produced when a neuron crosses its threshold, equal to 'step_spike_count_r + {3'd0, group_spike_count}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are step_spike_count_r: a binary event produced when a neuron crosses its threshold; d0: the value named 'd0' in this block; group_spike_count: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 248: `state <= ST_COUNT_LOW;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_COUNT_LOW'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COUNT_LOW: the value named 'st count low' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 249: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 250: `group_index <= group_index + 1'b1;` This makes group_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘group_index + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are group_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 251: `state <= ST_GROUP_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_GROUP_LOAD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_GROUP_LOAD: the value named ’st group load’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 252: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 253: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 255: `// The cumulative diagnostic counter is deliberately updated` Design intent: The cumulative diagnostic counter is deliberately updated This comment anchors the surrounding logic in the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting.

Line 256: `// as four 16-bit words. A monolithic 64-bit addition placed a` Design intent: as four 16-bit words. A monolithic 64-bit addition placed a This comment anchors the surrounding logic in the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting.

Line 257: `// popcount and long carry chain on the 200 MHz neuron path.` Design intent: popcount and long carry chain on the 200 MHz neuron path. This comment anchors the surrounding logic in the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting.

Line 258: `ST_COUNT_LOW: begin` This is the ST_COUNT_LOW case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_LOW, the machine updates a segmented telemetry counter without a long carry chain..

Line 259: `counter_sum_r <= {1'b0, total_spike_count[15:0]}` This statement continues through source line 260; the explanation uses the complete expression rather than only the first visual fragment. This makes counter_sum_r, a hardware telemetry register that accumulates measured events, equal to ‘{1’b0, total_spike_count[15:0]} + {10’d0, completed_step_spike_count_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block; total_spike_count: a binary event produced when a neuron crosses its threshold; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_LOW, the machine updates a segmented telemetry counter without a long carry chain..

Line 260: `+ {10'd0, completed_step_spike_count_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 259. Its exact fragment is ‘+ {10’d0, completed_step_spike_count_r};‘. The important operands are d0: the value named ’d0’ in this block; completed_step_spike_count_r: a binary event produced

when a neuron crosses its threshold. Read together, the complete operation is: This makes counter_sum_r, a hardware telemetry register that accumulates measured events, equal to ‘{1’b0, total_spike_count[15:0]} + {10’d0, completed_step_spike_count_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block; total_spike_count: a binary event produced when a neuron crosses its threshold; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_LOW, the machine updates a segmented telemetry counter without a long carry chain..

Line 261: `state <= ST_COUNT_WORD1;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_COUNT_WORD1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COUNT_WORD1: the value named ’st count word1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_LOW, the machine updates a segmented telemetry counter without a long carry chain..

Line 262: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_LOW, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 264: `ST_COUNT_WORD1: begin` This is the ST_COUNT_WORD1 case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD1, the machine updates a segmented telemetry counter without a long carry chain..

Line 265: `total_spike_count[15:0] <= counter_sum_r[15:0];` This makes total_spike_count[15:0], a binary event produced when a neuron crosses its threshold, equal to ‘counter_sum_r[15:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are counter_sum_r: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD1, the machine updates a segmented telemetry counter without a long carry chain..

Line 266: `counter_sum_r <= {1’b0, total_spike_count[31:16]}` This statement continues through source line 267; the explanation uses the complete expression rather than only the first visual fragment. This makes counter_sum_r, a hardware telemetry register that accumulates measured events, equal to ‘{1’b0, total_spike_count[31:16]} + {16’d0, counter_sum_r[16]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block; total_spike_count: a binary event produced when a neuron crosses its threshold; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD1, the machine updates a segmented telemetry counter without a long carry chain..

Line 267: `+ {16’d0, counter_sum_r[16]};` This is continuation line 2 of the 2-line logical statement that begins at source line 266. Its exact fragment is ‘+ {16’d0, counter_sum_r[16]};‘. The important operands are d0: the value named ’d0’ in this block; counter_sum_r: a hardware telemetry register that accumulates measured events. Read together, the complete operation is: This makes counter_sum_r, a hardware telemetry register that accumulates measured events, equal to ‘{1’b0, total_spike_count[31:16]} + {16’d0, counter_sum_r[16]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block; total_spike_count: a binary event produced when a neuron crosses its threshold; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD1, the machine updates a segmented telemetry counter without a long carry chain..

Line 268: `state <= ST_COUNT_WORD2;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_COUNT_WORD2‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COUNT_WORD2: the value named ’st count word2’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD1, the machine updates a segmented telemetry counter without a long carry chain..

Line 269: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD1, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 271: `ST_COUNT_WORD2: begin` This is the ST_COUNT_WORD2 case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD2, the machine updates a segmented telemetry counter without a long carry chain..

Line 272: `total_spike_count[31:16] <= counter_sum_r[15:0];` This makes total_spike_count[31:16], a binary event produced when a neuron crosses its threshold, equal to ‘counter_sum_r[15:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are counter_sum_r: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD2, the machine updates a segmented telemetry counter without a long carry chain..

Line 273: `counter_sum_r <= {1'b0, total_spike_count[47:32]}` This statement continues through source line 274; the explanation uses the complete expression rather than only the first visual fragment. This makes counter_sum_r, a hardware telemetry register that accumulates measured events, equal to ‘{1’b0, total_spike_count[47:32]} + {16’d0, counter_sum_r[16]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block; total_spike_count: a binary event produced when a neuron crosses its threshold; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD2, the machine updates a segmented telemetry counter without a long carry chain..

Line 274: `+ {16'd0, counter_sum_r[16]};` This is continuation line 2 of the 2-line logical statement that begins at source line 273. Its exact fragment is ‘+ {16’d0, counter_sum_r[16]};‘. The important operands are d0: the value named ’d0’ in this block; counter_sum_r: a hardware telemetry register that accumulates measured events. Read together, the complete operation is: This makes counter_sum_r, a hardware telemetry register that accumulates measured events, equal to ‘{1’b0, total_spike_count[47:32]} + {16’d0, counter_sum_r[16]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block; total_spike_count: a binary event produced when a neuron crosses its threshold; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD2, the machine updates a segmented telemetry counter without a long carry chain..

Line 275: `state <= ST_COUNT_WORD3;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_COUNT_WORD3‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COUNT_WORD3: the value named ’st count word3’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD2, the machine updates a segmented telemetry counter without a long carry chain..

Line 276: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD2, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 278: `ST_COUNT_WORD3: begin` This is the ST_COUNT_WORD3 case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD3, the machine updates a segmented telemetry counter without a long carry chain..

Line 279: `total_spike_count[47:32] <= counter_sum_r[15:0];` This makes total_spike_count[47:32], a binary event produced when a neuron crosses its threshold, equal to ‘counter_sum_r[15:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are counter_sum_r: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD3, the machine updates a segmented telemetry counter without a long carry chain..

Line 280: `counter_sum_r <= {1'b0, total_spike_count[63:48]}` This statement continues through source line 281; the explanation uses the complete expression rather than only the first visual fragment. This makes counter_sum_r, a hardware telemetry register that accumulates measured events, equal to '{1'b0, total_spike_count[63:48]} + {16'd0, counter_sum_r[16]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block; total_spike_count: a binary event produced when a neuron crosses its threshold; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD3, the machine updates a segmented telemetry counter without a long carry chain..

Line 281: `+ {16'd0, counter_sum_r[16]};` This is continuation line 2 of the 2-line logical statement that begins at source line 280. Its exact fragment is '+ {16'd0, counter_sum_r[16]};'. The important operands are d0: the value named 'd0' in this block; counter_sum_r: a hardware telemetry register that accumulates measured events. Read together, the complete operation is: This makes counter_sum_r, a hardware telemetry register that accumulates measured events, equal to '{1'b0, total_spike_count[63:48]} + {16'd0, counter_sum_r[16]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block; total_spike_count: a binary event produced when a neuron crosses its threshold; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD3, the machine updates a segmented telemetry counter without a long carry chain..

Line 282: `state <= ST_COUNT_DONE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_COUNT_DONE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COUNT_DONE: the value named 'st count done' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD3, the machine updates a segmented telemetry counter without a long carry chain..

Line 283: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_WORD3, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 285: `ST_COUNT_DONE: begin` This is the ST_COUNT_DONE case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_DONE, the machine updates a segmented telemetry counter without a long carry chain..

Line 286: `total_spike_count[63:48] <= counter_sum_r[15:0];` This makes total_spike_count[63:48], a binary event produced when a neuron crosses its threshold, equal to 'counter_sum_r[15:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are counter_sum_r: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_DONE, the machine updates a segmented telemetry counter without a long carry chain..

Line 287: `state <= ST_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT: the value named 'st output' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_DONE, the machine updates a segmented telemetry counter without a long carry chain..

Line 288: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COUNT_DONE, the machine updates a segmented telemetry counter without a long carry chain.; it has no arithmetic value of its own.

Line 290: `ST_OUTPUT: if (output_ready) state <= ST_IDLE;` This is the ST_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'if (output_ready) state <= ST_IDLE;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 291: `default:  state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘state <= ST_IDLE;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 292: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 293: `if (clear_counters) total_spike_count <= 64'd0;` This condition selects the following hardware branch only when ‘clear_counters‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_counters: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge.

Line 294: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 295: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 297: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 298: `if (LANES != 32) $fatal(1, "neuraedge_lif_bank currently requires LANES=32");` This condition selects the following hardware branch only when ‘LANES != 32) $fatal(1, "neuraedge_lif_bank currently requires LANES=32"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are LANES: one parallel arithmetic element within a vector or tensor datapath; fatal: the value named 'fatal' in this block; neuraedge_lif_bank: the value named 'neuraedge lif bank' in this block. Context: the module's structural declarations.

Line 299: `if (LANES_PER_CYCLE != 8) $fatal(1, "neuraedge_lif_bank currently requires LANES_PER_CYCLE=8");` This condition selects the following hardware branch only when ‘LANES_PER_CYCLE != 8) $fatal(1, "neuraedge_lif_bank currently requires LANES_PER_CYCLE=8"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are LANES_PER_CYCLE: one parallel arithmetic element within a vector or tensor datapath; fatal: the value named 'fatal' in this block; neuraedge_lif_bank: the value named 'neuraedge lif bank' in this block. Context: the module's structural declarations.

Line 300: `if (LANES % LANES_PER_CYCLE != 0) $fatal(1, "LANES_PER_CYCLE must divide LANES");` This condition selects the following hardware branch only when ‘LANES % LANES_PER_CYCLE != 0) $fatal(1, "LANES_PER_CYCLE must divide LANES"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are LANES: one parallel arithmetic element within a vector or tensor datapath; LANES_PER_CYCLE: one parallel arithmetic element within a vector or tensor datapath; fatal: the value named 'fatal' in this block. Context: the module's structural declarations.

Line 301: `if (CURRENT_BITS >= MEMBRANE_BITS) $fatal(1, "CURRENT_BITS must be smaller than MEMBRANE_BITS");` This condition selects the following hardware branch only when ‘CURRENT_BITS >= MEMBRANE_BITS) $fatal(1, "CURRENT_BITS must be smaller than MEMBRANE_BITS"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are CURRENT_BITS: the value named 'current bits' in this block; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps; fatal: the value named 'fatal' in this block. Context: the module's structural declarations.

Line 302: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 303: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 305: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting.

13 rtl/neuraedge_linear_backward8.sv

Role in the complete system: computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

13.1 Beginner architecture map

Dataflow responsibility A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_linear_backward8 into physical nets and primitives. The source declares 22 input-port lines, 10 output-port lines, 1 clocked process(es), and 2 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_ADD_ACCUMULATOR, ST_COMMIT, ST_CONFIGURE, ST_DONE, ST_IDLE, ST_LOAD_COMMIT_DRAIN, ST_LOAD_DY, ST_LOAD_W, ST_LOAD_X, ST_MULTIPLY, ST_OUTPUT_DW, ST_OUTPUT_DX, ST_PAIR, ST_PRODUCT, ST_REDUCE, ST_REDUCTION_SUM, ST_REGISTER_OPERANDS, ST_ROUND_BIAS and additional states. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates begin as if, else as if, endfunction as for, neuraedge_gradient_accumulator_ as u_bank, end as for, end as if, unique as case, endcase as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

13.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  // Exact tiled reverse-mode engine for Y = XW:
5  //   dX = dY * transpose(W), dW = transpose(X) * dY.
6  // The external tile remains 8x8x8, while a 4x4 DSP array traverses two
7  // result-column groups and up to two reduction groups. One shared 47-bit
8  // accumulator bank preserves cross-command partial sums until the compiler
9  // requests the final rounded output.
10 module neuraedge_linear_backward8 #(
11     parameter int LANES = 8
12 ) (
13     input  wire clk,
14     input  wire rst_n,
15     input  wire clear_counters,
16
17     input  wire start_valid,
18     output logic start_ready,
19     input  wire [3:0] active_rows,
20     input  wire [3:0] active_inputs,
21     input  wire [3:0] active_outputs,
22     input  wire [4:0] input_gradient_shift,
23     input  wire [4:0] weight_gradient_shift,
24     input  wire [1:0] gradient_mode,
25     input  wire accumulate_result,
26     input  wire suppress_output,
27     input  wire reserved_nonzero,
28
29     input  wire s_axis_tvalid,
30     output logic s_axis_tready,
31     input  wire [63:0] s_axis_tdata,
32     input  wire s_axis_tlast,
33
34     output logic m_axis_tvalid,
```

```
35      input  wire m_axis_tready,
36      output logic [63:0] m_axis_tdata,
37      output logic m_axis_tlast,
38
39      output logic busy,
40      output logic done,
41      output logic config_error,
42      (* extract_enable = "no" *)
43      output logic [31:0] completed_tiles,
44      output logic [31:0] active_cycles
45  );
46      localparam int RESULT_LANES = 4;
47      localparam int REDUCTION_LANES = 4;
48
49      localparam logic [1:0] MODE_BOTH = 2'd0;
50      localparam logic [1:0] MODE_INPUT_GRADIENT = 2'd1;
51      localparam logic [1:0] MODE_WEIGHT_GRADIENT = 2'd2;
52
53      typedef enum logic [4:0] {
54          ST_IDLE = 5'd0,
55          ST_LOAD_X = 5'd1,
56          ST_LOAD_W = 5'd2,
57          ST_LOAD_DY = 5'd3,
58          ST_MULTIPLY = 5'd4,
59          ST_PRODUCT = 5'd5,
60          ST_PAIR = 5'd6,
61          ST_REDUCE = 5'd7,
62          ST_COMMIT = 5'd8,
63          ST_OUTPUT_DX = 5'd9,
64          ST_OUTPUT_DW = 5'd10,
65          ST_SWITCH_DW = 5'd11,
66          ST_DONE = 5'd12,
67          ST_ROUND_MAGNITUDE = 5'd13,
68          ST_ROUND_BIAS = 5'd14,
69          ST_ROUND_SHIFT = 5'd15,
70          ST_ROUND_SIGN = 5'd16,
71          ST_SATURATE = 5'd17,
72          ST_ADD_ACCUMULATOR = 5'd18,
73          ST_STORE_ACCUMULATOR = 5'd19,
74          ST_REDUCTION_SUM = 5'd20,
75          ST_STORE_RESULT = 5'd21,
76          ST_REGISTER_OPERANDS = 5'd22,
77          ST_CONFIGURE = 5'd23,
78          ST_ROUND_MAGNITUDE_HIGH = 5'd24,
79          ST_ROUND_BIAS_HIGH = 5'd25,
80          ST_ROUND_SIGN_HIGH = 5'd26,
81          ST_LOAD_COMMIT_DRAIN = 5'd27
82      } state_t;
83
84      // Keep the compact encoding, but route next-state decisions to D rather
85      // than the tighter-setup CE pin used by automatic enable extraction.
86      (* fsm_encoding = "sequential", extract_enable = "no" *) state_t state;
87      logic [3:0] rows_r;
88      logic [3:0] inputs_r;
89      logic [3:0] outputs_r;
90      logic [4:0] dw_shift_r;
91      logic [4:0] result_shift_r;
92      logic [46:0] rounding_bias_r;
93      logic [1:0] mode_r;
94      logic [1:0] compute_mode_r;
95      logic suppress_output_r;
96      logic accumulate_result_r;
97      logic start_configuration_invalid_r;
98      logic [1:0] requested_accumulator_mode_r;
99      logic [3:0] requested_accumulator_rows_r;
100     logic [3:0] requested_accumulator_columns_r;
101     logic [3:0] compute_result_row_count_r;
102     logic [3:0] compute_result_column_count_r;
103     logic [3:0] compute_reduction_count_r;
104
105     // Each lane owns a preserved copy of the mode-select bit. This limits the
106     // physical fanout into the four DSP lanes while retaining one logical mode.
107     (* keep = "true", dont_touch = "true" *)
108     logic [RESULT_LANES-1:0] input_gradient_lane_r;
109
110     // These banks are loaded under several FSM states.  Implementing the hold
111     // condition on D keeps a distant state decode off thousands of FDRE CE
112     // pins and lets Vivado place each small input mux beside its register.
113     (* extract_enable = "no" *)
114     logic signed [15:0] activation_r [0:LANES-1][0:LANES-1];
115     (* extract_enable = "no" *)
116     logic signed [15:0] weight_r [0:LANES-1][0:LANES-1];
117     (* extract_enable = "no" *)
118     logic signed [15:0] output_gradient_r [0:LANES-1][0:LANES-1];
119
120     // Four banks provide the four independent reads and writes needed by the
121     // result lanes. Each bank stores one column from both four-column groups
```

```
    // for every row: address = {row, column_group}. Commands that do not
    // accumulate select mathematical zero instead of clearing the RAM. This
    // preserves exact accumulation while removing a clear/hold/store mux and
    // its FSM control route from 3,008 individual flip-flops.
    wire signed [46:0] accumulator_bank_read_data
        [0:RESULT_LANES-1];
    wire signed [46:0] accumulator_bank_write_data
        [0:RESULT_LANES-1];
    wire [RESULT_LANES-1:0] accumulator_bank_write_enable;
    logic accumulator_valid_r;
    logic [1:0] accumulator_mode_r;
    logic [3:0] accumulator_rows_r;
    logic [3:0] accumulator_columns_r;

    logic signed [31:0] result_row_buffer_r [0:LANES-1];
    // Eight signed 16x16 products fit exactly in 35 bits. Keeping this
    // command-local sum at its mathematical width shortens the carry chain;
    // it is sign-extended only when added to the retained 47-bit accumulator.
    logic signed [34:0] row_partial_r [0:RESULT_LANES-1];
    logic signed [33:0] reduction_sum_r [0:RESULT_LANES-1];
    logic signed [46:0] selected_accumulator_r [0:RESULT_LANES-1];
    logic signed [47:0] accumulated_wide_r [0:RESULT_LANES-1];
    logic commit_lane_valid_r [0:RESULT_LANES-1];
    logic [46:0] rounding_value_r [0:RESULT_LANES-1];
    logic rounding_negative_r [0:RESULT_LANES-1];
    logic rounding_carry_r [0:RESULT_LANES-1];
    logic signed [31:0] saturated_result_r [0:RESULT_LANES-1];

    logic [3:0] load_row_r;
    logic load_half_r;
    logic load_final_r;
    // Stream VALID previously participated directly in the D-input mux for
    // all 3,072 operand-bank bits.  This elastic stage accepts one beat every
    // clock, while the row-local commit enables below terminate control near
    // the destination registers.  The payload stage can consume its old beat
    // and capture the next operand class on the same edge.
    logic input_stage_valid_r;
    logic [63:0] input_stage_data_r;
    logic input_stage_last_r;
    logic [63:0] load_commit_data_r;
    logic [1:0] activation_write_enable_r [0:LANES-1];
    logic [1:0] weight_write_enable_r [0:LANES-1];
    logic [1:0] output_gradient_write_enable_r [0:LANES-1];
    // Keep the row hold as a D-input mux.  A dedicated CE would put the
    // multi-state row-update decode on the FDRE CE pin, whose setup time is
    // tighter than the data input on 7-series devices.
    (* extract_enable = "no" *) logic [3:0] result_row_r;
    logic column_group_r;
    logic reduction_group_r;
    logic [1:0] output_word_r;

    wire [2:0] load_row_index = load_row_r[2:0];
    wire [2:0] result_row_index = result_row_r[2:0];
    wire [3:0] accumulator_address = {result_row_index, column_group_r};
    wire [3:0] result_row_count = compute_result_row_count_r;
    wire [3:0] result_column_count = compute_result_column_count_r;
    wire [3:0] reduction_count = compute_reduction_count_r;
    wire input_load_state = state == ST_LOAD_X || state == ST_LOAD_W
        || state == ST_LOAD_DY;
    wire input_stage_consume = input_stage_valid_r && input_load_state;
    wire input_stage_ready = !input_stage_valid_r || input_stage_consume;
    wire input_final_buffered = state == ST_LOAD_DY && input_stage_valid_r
        && load_half_r && load_final_r;
    wire [1:0] requested_accumulator_mode = gradient_mode == MODE_WEIGHT_GRADIENT
        ? MODE_WEIGHT_GRADIENT : MODE_INPUT_GRADIENT;
    wire [3:0] requested_accumulator_rows = gradient_mode == MODE_WEIGHT_GRADIENT
        ? active_inputs : active_rows;
    wire [3:0] requested_accumulator_columns = gradient_mode == MODE_WEIGHT_GRADIENT
        ? active_outputs : active_inputs;
    wire dimensions_invalid = active_rows == 0 || active_rows > 4'(LANES)
        || active_inputs == 0 || active_inputs > 4'(LANES)
        || active_outputs == 0 || active_outputs > 4'(LANES);
    wire accumulation_shape_invalid = accumulate_result && (
        gradient_mode == MODE_BOTH
        || !accumulator_valid_r
        || accumulator_mode_r != requested_accumulator_mode
        || accumulator_rows_r != requested_accumulator_rows
        || accumulator_columns_r != requested_accumulator_columns);
    wire start_configuration_invalid = reserved_nonzero || dimensions_invalid
        || gradient_mode == 2'd3 || accumulation_shape_invalid;

    logic signed [15:0] multiplier_left [0:RESULT_LANES-1][0:REDUCTION_LANES-1];
    logic signed [15:0] multiplier_right [0:RESULT_LANES-1][0:REDUCTION_LANES-1];
    (* keep = "true", dont_touch = "true" *)
    logic signed [15:0] multiplier_left_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];
    (* keep = "true", dont_touch = "true" *)
    logic signed [15:0] multiplier_right_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];
```

```
    logic signed [15:0] multiplier_left_dsp_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];
    logic signed [15:0] multiplier_right_dsp_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];
    (* use_dsp = "yes" *) wire signed [31:0]
        multiplier_product [0:RESULT_LANES-1][0:REDUCTION_LANES-1];
    logic signed [31:0] product_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];
    logic signed [32:0] pair_sum_r [0:RESULT_LANES-1][0:1];
    // This local pulse drives only the eight pair-reduction DSP registers.
    // Registering it near the array avoids routing the shared FSM decode to
    // DSP clock-enable pins, whose setup requirement is especially tight.
    logic pair_update_r;

    wire [3:0] result_column_index [0:RESULT_LANES-1];
    wire [3:0] reduction_index [0:REDUCTION_LANES-1];
    logic [191:0] packed_weight_gradient;

    function automatic logic signed [46:0] saturate_s47(
        input logic signed [47:0] value
    );
        begin
            if (value[47] == value[46]) saturate_s47 = value[46:0];
            else if (value[47]) saturate_s47 = {1'b1, {46{1'b0}}};
            else saturate_s47 = {1'b0, {46{1'b1}}};
        end
    endfunction

    function automatic logic [46:0] round_bias(
        input logic [4:0] shift
    );
        begin
            if (shift == 0) round_bias = 47'd0;
            else round_bias = 47'd1 << (shift - 1'b1);
        end
    endfunction

    function automatic logic signed [31:0] saturate_s32(
        input logic signed [46:0] value
    );
        begin
            // A signed value fits in 32 bits exactly when every discarded bit
            // is a copy of bit 31.  This is equivalent to range comparisons,
            // but uses equality/reduction logic instead of two carry chains.
            if (value[46:31] == {16{value[31]}}) begin
                saturate_s32 = value[31:0];
            end else if (value[46]) begin
                saturate_s32 = 32'sh80000000;
            end else begin
                saturate_s32 = 32'sh7fffffff;
            end
        end
    endfunction

    function automatic logic signed [23:0] saturate_s24(
        input logic signed [46:0] value
    );
        begin
            if (value[46:23] == {24{value[23]}}) begin
                saturate_s24 = value[23:0];
            end else if (value[46]) begin
                saturate_s24 = 24'sh800000;
            end else begin
                saturate_s24 = 24'sh7fffff;
            end
        end
    endfunction

    function automatic logic signed [31:0] saturate_s24_extended(
        input logic signed [46:0] value
    );
        logic signed [23:0] saturated;
        begin
            saturated = saturate_s24(value);
            saturate_s24_extended = {{8{saturated[23]}}, saturated};
        end
    endfunction

    for (genvar bank = 0; bank < RESULT_LANES; bank++) begin : g_accumulator_bank
        assign accumulator_bank_write_enable[bank] =
            state == ST_STORE_ACCUMULATOR && commit_lane_valid_r[bank];
        assign accumulator_bank_write_data[bank] =
            saturate_s47(accumulated_wide_r[bank]);

        neuraedge_gradient_accumulator_bank u_bank (
            .clk(clk),
            .write_enable(accumulator_bank_write_enable[bank]),
            .write_address(accumulator_address),
            .write_data(accumulator_bank_write_data[bank]),
            .read_address(accumulator_address),
```

```
            .read_data(accumulator_bank_read_data[bank])
        );
    end

    for (genvar result_lane = 0; result_lane < RESULT_LANES;
            result_lane++) begin : g_result_lane
        assign result_column_index[result_lane] =
            (column_group_r ? 4'd4 : 4'd0) + 4'(result_lane);
        for (genvar reduction_lane = 0;
                reduction_lane < REDUCTION_LANES; reduction_lane++) begin : g_multiplier
            assign multiplier_product[result_lane][reduction_lane] =
                multiplier_left_dsp_r[result_lane][reduction_lane]
                * multiplier_right_dsp_r[result_lane][reduction_lane];
        end
    end

    for (genvar reduction_lane = 0; reduction_lane < REDUCTION_LANES;
            reduction_lane++) begin : g_reduction_index
        assign reduction_index[reduction_lane] =
            (reduction_group_r ? 4'd4 : 4'd0) + 4'(reduction_lane);
    end

    always_comb begin
        for (int result_lane = 0; result_lane < RESULT_LANES;
                result_lane++) begin
            for (int reduction_lane = 0; reduction_lane < REDUCTION_LANES;
                    reduction_lane++) begin
                multiplier_left[result_lane][reduction_lane] = 16'sd0;
                multiplier_right[result_lane][reduction_lane] = 16'sd0;
                if (result_row_r < result_row_count
                        && result_column_index[result_lane] < result_column_count
                        && reduction_index[reduction_lane] < reduction_count) begin
                    if (input_gradient_lane_r[result_lane]) begin
                        multiplier_left[result_lane][reduction_lane] =
                            output_gradient_r[result_row_index]
                                [reduction_index[reduction_lane][2:0]];
                        multiplier_right[result_lane][reduction_lane] =
                            weight_r[result_column_index[result_lane][2:0]]
                                [reduction_index[reduction_lane][2:0]];
                    end else begin
                        multiplier_left[result_lane][reduction_lane] =
                            activation_r[reduction_index[reduction_lane][2:0]]
                                [result_row_index];
                        multiplier_right[result_lane][reduction_lane] =
                            output_gradient_r[reduction_index[reduction_lane][2:0]]
                                [result_column_index[result_lane][2:0]];
                    end
                end
            end
        end

        packed_weight_gradient = '0;
        for (int lane = 0; lane < LANES; lane++) begin
            packed_weight_gradient[lane*24 +: 24] =
                result_row_buffer_r[lane][23:0];
        end
    end

    always_comb begin
        start_ready = state == ST_IDLE;
        busy = state != ST_IDLE;
        done = state == ST_DONE;
        s_axis_tready = input_load_state && input_stage_ready
            && !input_final_buffered;
        m_axis_tvalid = state == ST_OUTPUT_DX || state == ST_OUTPUT_DW;
        m_axis_tdata = 64'd0;
        m_axis_tlast = 1'b0;
        // compute_mode_r is fixed before an output row starts.  Selecting data
        // with it avoids putting the large FSM state decode in the stream-data
        // path while preserving the same value whenever tvalid is asserted.
        if (compute_mode_r == MODE_INPUT_GRADIENT) begin
            m_axis_tdata = {
                result_row_buffer_r[output_word_r*2 + 1],
                result_row_buffer_r[output_word_r*2]
            };
            m_axis_tlast = mode_r == MODE_INPUT_GRADIENT
                && result_row_r == rows_r - 1'b1 && output_word_r == 3;
        end else begin
            m_axis_tdata = packed_weight_gradient[output_word_r*64 +: 64];
            m_axis_tlast = result_row_r == inputs_r - 1'b1
                && output_word_r == 2;
        end
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            state <= ST_IDLE;
```

```
            rows_r <= 4'd0;
            inputs_r <= 4'd0;
            outputs_r <= 4'd0;
            dw_shift_r <= 5'd0;
            result_shift_r <= 5'd0;
            rounding_bias_r <= 47'd0;
            mode_r <= MODE_BOTH;
            compute_mode_r <= MODE_INPUT_GRADIENT;
            suppress_output_r <= 1'b0;
            accumulate_result_r <= 1'b0;
            start_configuration_invalid_r <= 1'b0;
            requested_accumulator_mode_r <= MODE_INPUT_GRADIENT;
            requested_accumulator_rows_r <= 4'd0;
            requested_accumulator_columns_r <= 4'd0;
            compute_result_row_count_r <= 4'd0;
            compute_result_column_count_r <= 4'd0;
            compute_reduction_count_r <= 4'd0;
            input_gradient_lane_r <= {RESULT_LANES{1'b1}};
            load_row_r <= 4'd0;
            load_half_r <= 1'b0;
            load_final_r <= 1'b0;
            input_stage_valid_r <= 1'b0;
            input_stage_data_r <= 64'd0;
            input_stage_last_r <= 1'b0;
            load_commit_data_r <= 64'd0;
            result_row_r <= 4'd0;
            column_group_r <= 1'b0;
            reduction_group_r <= 1'b0;
            output_word_r <= 2'd0;
            accumulator_valid_r <= 1'b0;
            accumulator_mode_r <= MODE_INPUT_GRADIENT;
            accumulator_rows_r <= 4'd0;
            accumulator_columns_r <= 4'd0;
            pair_update_r <= 1'b0;
            config_error <= 1'b0;
            completed_tiles <= 32'd0;
            active_cycles <= 32'd0;
            for (int row = 0; row < LANES; row++) begin
                result_row_buffer_r[row] <= 32'sd0;
                activation_write_enable_r[row] <= 2'b00;
                weight_write_enable_r[row] <= 2'b00;
                output_gradient_write_enable_r[row] <= 2'b00;
                for (int column = 0; column < LANES; column++) begin
                    activation_r[row][column] <= 16'sd0;
                    weight_r[row][column] <= 16'sd0;
                    output_gradient_r[row][column] <= 16'sd0;
                end
            end
            for (int result_lane = 0; result_lane < RESULT_LANES;
                    result_lane++) begin
                row_partial_r[result_lane] <= 35'sd0;
                reduction_sum_r[result_lane] <= 34'sd0;
                selected_accumulator_r[result_lane] <= 47'sd0;
                accumulated_wide_r[result_lane] <= 48'sd0;
                commit_lane_valid_r[result_lane] <= 1'b0;
                rounding_value_r[result_lane] <= 47'd0;
                rounding_negative_r[result_lane] <= 1'b0;
                rounding_carry_r[result_lane] <= 1'b0;
                saturated_result_r[result_lane] <= 32'sd0;
                for (int reduction_lane = 0;
                        reduction_lane < REDUCTION_LANES; reduction_lane++) begin
                    multiplier_left_r[result_lane][reduction_lane] <= 16'sd0;
                    multiplier_right_r[result_lane][reduction_lane] <= 16'sd0;
                    multiplier_left_dsp_r[result_lane][reduction_lane] <= 16'sd0;
                    multiplier_right_dsp_r[result_lane][reduction_lane] <= 16'sd0;
                    product_r[result_lane][reduction_lane] <= 32'sd0;
                end
                pair_sum_r[result_lane][0] <= 33'sd0;
                pair_sum_r[result_lane][1] <= 33'sd0;
            end
        end else begin
            if (state != ST_IDLE) active_cycles <= active_cycles + 1'b1;
            pair_update_r <= state == ST_PRODUCT;

            // The single elastic slot is fall-through only at clock edges: an
            // old buffered beat can be consumed while the next stream beat is
            // captured. On the final dY beat READY is withheld because there is
            // no following operand class to prefetch.
            if (input_stage_ready) begin
                input_stage_valid_r <= s_axis_tvalid && s_axis_tready;
                if (s_axis_tvalid && s_axis_tready) begin
                    input_stage_data_r <= s_axis_tdata;
                    input_stage_last_r <= s_axis_tlast;
                end
            end

            // Apply the previous beat's row-local commit and prepare one-hot
```

```
            // enables for the beat consumed this cycle.  Each enable drives
            // only four 16-bit destinations, replacing the former chip-wide
            // stream-VALID/state route into all three operand banks.
            for (int row = 0; row < LANES; row++) begin
                for (int half = 0; half < 2; half++) begin
                    if (activation_write_enable_r[row][half]) begin
                        for (int lane = 0; lane < 4; lane++) begin
                            activation_r[row][half*4 + lane]
                                <= load_commit_data_r[lane*16 +: 16];
                        end
                    end
                    if (weight_write_enable_r[row][half]) begin
                        for (int lane = 0; lane < 4; lane++) begin
                            weight_r[row][half*4 + lane]
                                <= load_commit_data_r[lane*16 +: 16];
                        end
                    end
                    if (output_gradient_write_enable_r[row][half]) begin
                        for (int lane = 0; lane < 4; lane++) begin
                            output_gradient_r[row][half*4 + lane]
                                <= load_commit_data_r[lane*16 +: 16];
                        end
                    end
                end
                activation_write_enable_r[row] <= 2'b00;
                weight_write_enable_r[row] <= 2'b00;
                output_gradient_write_enable_r[row] <= 2'b00;
            end
            if (input_stage_valid_r && input_load_state) begin
                load_commit_data_r <= input_stage_data_r;
                unique case (state)
                    ST_LOAD_X: activation_write_enable_r[load_row_index]
                        [load_half_r] <= 1'b1;
                    ST_LOAD_W: weight_write_enable_r[load_row_index]
                        [load_half_r] <= 1'b1;
                    ST_LOAD_DY: output_gradient_write_enable_r[load_row_index]
                        [load_half_r] <= 1'b1;
                    default: begin end
                endcase
            end

            if (pair_update_r) begin
                for (int result_lane = 0; result_lane < RESULT_LANES;
                        result_lane++) begin
                    pair_sum_r[result_lane][0] <=
                        {product_r[result_lane][0][31], product_r[result_lane][0]}
                        + {product_r[result_lane][1][31], product_r[result_lane][1]};
                    pair_sum_r[result_lane][1] <=
                        {product_r[result_lane][2][31], product_r[result_lane][2]}
                        + {product_r[result_lane][3][31], product_r[result_lane][3]};
                end
            end
            unique case (state)
                ST_IDLE: begin
                    if (start_valid) begin
                        rows_r <= active_rows;
                        inputs_r <= active_inputs;
                        outputs_r <= active_outputs;
                        dw_shift_r <= weight_gradient_shift;
                        if (gradient_mode == MODE_WEIGHT_GRADIENT) begin
                            result_shift_r <= weight_gradient_shift;
                            rounding_bias_r <= round_bias(weight_gradient_shift);
                        end else begin
                            result_shift_r <= input_gradient_shift;
                            rounding_bias_r <= round_bias(input_gradient_shift);
                        end
                        mode_r <= gradient_mode;
                        compute_mode_r <= requested_accumulator_mode;
                        suppress_output_r <= suppress_output;
                        accumulate_result_r <= accumulate_result;
                        start_configuration_invalid_r
                            <= start_configuration_invalid;
                        requested_accumulator_mode_r
                            <= requested_accumulator_mode;
                        requested_accumulator_rows_r
                            <= requested_accumulator_rows;
                        requested_accumulator_columns_r
                            <= requested_accumulator_columns;
                        input_gradient_lane_r <= {RESULT_LANES{
                            requested_accumulator_mode
                                == MODE_INPUT_GRADIENT}};
                        if (requested_accumulator_mode
                                == MODE_INPUT_GRADIENT) begin
                            compute_result_row_count_r <= active_rows;
                            compute_result_column_count_r <= active_inputs;
                            compute_reduction_count_r <= active_outputs;
                        end else begin
```

```
                    compute_result_row_count_r <= active_inputs;
                    compute_result_column_count_r <= active_outputs;
                    compute_reduction_count_r <= active_rows;
                end
                load_row_r <= 4'd0;
                load_half_r <= 1'b0;
                result_row_r <= 4'd0;
                column_group_r <= 1'b0;
                reduction_group_r <= 1'b0;
                output_word_r <= 2'd0;
                config_error <= 1'b0;
                for (int row = 0; row < LANES; row++) begin
                    result_row_buffer_r[row] <= 32'sd0;
                end
                state <= ST_CONFIGURE;
            end
        end

        ST_CONFIGURE: begin
            config_error <= start_configuration_invalid_r;
            if (start_configuration_invalid_r) begin
                state <= ST_DONE;
            end else begin
                accumulator_valid_r <= 1'b1;
                accumulator_mode_r <= requested_accumulator_mode_r;
                accumulator_rows_r <= requested_accumulator_rows_r;
                accumulator_columns_r
                    <= requested_accumulator_columns_r;
                if (mode_r == MODE_INPUT_GRADIENT) begin
                    load_final_r <= inputs_r == 4'd1;
                    state <= ST_LOAD_W;
                end else begin
                    load_final_r <= rows_r == 4'd1;
                    state <= ST_LOAD_X;
                end
            end
        end

        ST_LOAD_X: if (input_stage_valid_r) begin
            if (input_stage_last_r) config_error <= 1'b1;
            if (!load_half_r) load_half_r <= 1'b1;
            else if (load_final_r) begin
                load_row_r <= 4'd0;
                load_half_r <= 1'b0;
                if (mode_r == MODE_WEIGHT_GRADIENT) begin
                    load_final_r <= rows_r == 4'd1;
                    state <= ST_LOAD_DY;
                end else begin
                    load_final_r <= inputs_r == 4'd1;
                    state <= ST_LOAD_W;
                end
            end else begin
                load_row_r <= load_row_r + 1'b1;
                load_half_r <= 1'b0;
                load_final_r <= load_row_r == rows_r - 4'd2;
            end
        end

        ST_LOAD_W: if (input_stage_valid_r) begin
            if (input_stage_last_r) config_error <= 1'b1;
            if (!load_half_r) load_half_r <= 1'b1;
            else if (load_final_r) begin
                load_row_r <= 4'd0;
                load_half_r <= 1'b0;
                load_final_r <= rows_r == 4'd1;
                state <= ST_LOAD_DY;
            end else begin
                load_row_r <= load_row_r + 1'b1;
                load_half_r <= 1'b0;
                load_final_r <= load_row_r == inputs_r - 4'd2;
            end
        end

        ST_LOAD_DY: if (input_stage_valid_r) begin
            if (input_stage_last_r != (load_half_r && load_final_r)) begin
                config_error <= 1'b1;
            end
            if (!load_half_r) load_half_r <= 1'b1;
            else if (load_final_r) begin
                result_row_r <= 4'd0;
                column_group_r <= 1'b0;
                reduction_group_r <= 1'b0;
                state <= ST_LOAD_COMMIT_DRAIN;
            end else begin
                load_row_r <= load_row_r + 1'b1;
                load_half_r <= 1'b0;
                load_final_r <= load_row_r == rows_r - 4'd2;
```

```
                    end
                end

                // The final dY payload has reached the commit registers. One
                // dedicated cycle lets its row-local write land before the DSP
                // operand-selection logic is allowed to read that bank.
                ST_LOAD_COMMIT_DRAIN: state <= ST_MULTIPLY;

                ST_MULTIPLY: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        for (int reduction_lane = 0;
                                reduction_lane < REDUCTION_LANES;
                                reduction_lane++) begin
                            multiplier_left_r[result_lane][reduction_lane]
                                <= multiplier_left[result_lane][reduction_lane];
                            multiplier_right_r[result_lane][reduction_lane]
                                <= multiplier_right[result_lane][reduction_lane];
                        end
                    end
                    state <= ST_REGISTER_OPERANDS;
                end

                ST_REGISTER_OPERANDS: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        for (int reduction_lane = 0;
                                reduction_lane < REDUCTION_LANES;
                                reduction_lane++) begin
                            multiplier_left_dsp_r[result_lane][reduction_lane]
                                <= multiplier_left_r[result_lane][reduction_lane];
                            multiplier_right_dsp_r[result_lane][reduction_lane]
                                <= multiplier_right_r[result_lane][reduction_lane];
                        end
                    end
                    state <= ST_PRODUCT;
                end

                ST_PRODUCT: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        for (int reduction_lane = 0;
                                reduction_lane < REDUCTION_LANES;
                                reduction_lane++) begin
                            product_r[result_lane][reduction_lane]
                                <= multiplier_product[result_lane][reduction_lane];
                        end
                    end
                    state <= ST_PAIR;
                end

                ST_PAIR: begin
                    state <= ST_REDUCTION_SUM;
                end

                ST_REDUCTION_SUM: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        reduction_sum_r[result_lane] <=
                            {pair_sum_r[result_lane][0][32],
                             pair_sum_r[result_lane][0]}
                            + {pair_sum_r[result_lane][1][32],
                               pair_sum_r[result_lane][1]};
                    end
                    state <= ST_REDUCE;
                end

                ST_REDUCE: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        if (!reduction_group_r) begin
                            row_partial_r[result_lane]
                                <= {reduction_sum_r[result_lane][33],
                                    reduction_sum_r[result_lane]};
                        end else begin
                            row_partial_r[result_lane] <=
                                row_partial_r[result_lane]
                                + {reduction_sum_r[result_lane][33],
                                   reduction_sum_r[result_lane]};
                        end
                    end
                    state <= ST_COMMIT;
                end

                ST_COMMIT: begin
                    if (!reduction_group_r && reduction_count > 4'd4) begin
                        reduction_group_r <= 1'b1;
```

```
                        state <= ST_MULTIPLY;
                    end else begin
                        for (int result_lane = 0; result_lane < RESULT_LANES;
                                result_lane++) begin
                            if (result_column_index[result_lane]
                                    < result_column_count) begin
                                if (accumulate_result_r) begin
                                    selected_accumulator_r[result_lane]
                                        <= accumulator_bank_read_data
                                            [result_lane];
                                end else begin
                                    selected_accumulator_r[result_lane]
                                        <= 47'sd0;
                                end
                                commit_lane_valid_r[result_lane] <= 1'b1;
                            end else begin
                                selected_accumulator_r[result_lane] <= 47'sd0;
                                commit_lane_valid_r[result_lane] <= 1'b0;
                            end
                        end
                        state <= ST_ADD_ACCUMULATOR;
                    end
                end

                ST_ADD_ACCUMULATOR: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        accumulated_wide_r[result_lane] <=
                            {selected_accumulator_r[result_lane][46],
                             selected_accumulator_r[result_lane]}
                            + {{13{row_partial_r[result_lane][34]}},
                               row_partial_r[result_lane]};
                    end
                    state <= ST_STORE_ACCUMULATOR;
                end

                ST_STORE_ACCUMULATOR: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        if (commit_lane_valid_r[result_lane]) begin
                            rounding_value_r[result_lane]
                                <= saturate_s47(accumulated_wide_r[result_lane]);
                            rounding_negative_r[result_lane]
                                <= accumulated_wide_r[result_lane][47];
                            if (accumulated_wide_r[result_lane][47]
                                    != accumulated_wide_r[result_lane][46]) begin
                                config_error <= 1'b1;
                            end
                        end else begin
                            rounding_value_r[result_lane] <= 47'd0;
                            rounding_negative_r[result_lane] <= 1'b0;
                        end
                    end
                    reduction_group_r <= 1'b0;
                    state <= ST_ROUND_MAGNITUDE;
                end

                ST_ROUND_MAGNITUDE: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        if (rounding_negative_r[result_lane]) begin
                            {rounding_carry_r[result_lane],
                             rounding_value_r[result_lane][23:0]} <=
                                {1'b0, ~rounding_value_r[result_lane][23:0]}
                                + 25'd1;
                        end else rounding_carry_r[result_lane] <= 1'b0;
                    end
                    state <= ST_ROUND_MAGNITUDE_HIGH;
                end

                ST_ROUND_MAGNITUDE_HIGH: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        if (rounding_negative_r[result_lane]) begin
                            rounding_value_r[result_lane][46:24] <=
                                (~rounding_value_r[result_lane][46:24])
                                + rounding_carry_r[result_lane];
                        end
                    end
                    state <= ST_ROUND_BIAS;
                end

                ST_ROUND_BIAS: begin
                    for (int result_lane = 0; result_lane < RESULT_LANES;
                            result_lane++) begin
                        {rounding_carry_r[result_lane],
                         rounding_value_r[result_lane][23:0]} <=
```

```
                {1'b0, rounding_value_r[result_lane][23:0]}
                + {1'b0, rounding_bias_r[23:0]};
        end
        state <= ST_ROUND_BIAS_HIGH;
    end

    ST_ROUND_BIAS_HIGH: begin
        for (int result_lane = 0; result_lane < RESULT_LANES;
                result_lane++) begin
            rounding_value_r[result_lane][46:24] <=
                rounding_value_r[result_lane][46:24]
                + rounding_bias_r[46:24]
                + rounding_carry_r[result_lane];
        end
        state <= ST_ROUND_SHIFT;
    end

    ST_ROUND_SHIFT: begin
        for (int result_lane = 0; result_lane < RESULT_LANES;
                result_lane++) begin
            rounding_value_r[result_lane]
                <= rounding_value_r[result_lane] >> result_shift_r;
        end
        state <= ST_ROUND_SIGN;
    end

    ST_ROUND_SIGN: begin
        for (int result_lane = 0; result_lane < RESULT_LANES;
                result_lane++) begin
            if (rounding_negative_r[result_lane]) begin
                {rounding_carry_r[result_lane],
                 rounding_value_r[result_lane][23:0]} <=
                    {1'b0, ~rounding_value_r[result_lane][23:0]}
                    + 25'd1;
            end else rounding_carry_r[result_lane] <= 1'b0;
        end
        state <= ST_ROUND_SIGN_HIGH;
    end

    ST_ROUND_SIGN_HIGH: begin
        for (int result_lane = 0; result_lane < RESULT_LANES;
                result_lane++) begin
            if (rounding_negative_r[result_lane]) begin
                rounding_value_r[result_lane][46:24] <=
                    (~rounding_value_r[result_lane][46:24])
                    + rounding_carry_r[result_lane];
            end
        end
        state <= ST_SATURATE;
    end

    ST_SATURATE: begin
        for (int result_lane = 0; result_lane < RESULT_LANES;
                result_lane++) begin
            if (result_column_index[result_lane]
                    < result_column_count) begin
                if (input_gradient_lane_r[result_lane]) begin
                    saturated_result_r[result_lane]
                        <= saturate_s32($signed(
                            rounding_value_r[result_lane]));
                end else begin
                    saturated_result_r[result_lane]
                        <= saturate_s24_extended($signed(
                            rounding_value_r[result_lane]));
                end
            end else begin
                saturated_result_r[result_lane] <= 32'sd0;
            end
        end
        state <= ST_STORE_RESULT;
    end

    ST_STORE_RESULT: begin
        for (int result_lane = 0; result_lane < RESULT_LANES;
                result_lane++) begin
            if (column_group_r) begin
                result_row_buffer_r[result_lane + RESULT_LANES]
                    <= saturated_result_r[result_lane];
            end else begin
                result_row_buffer_r[result_lane]
                    <= saturated_result_r[result_lane];
            end
        end

        if (!column_group_r) begin
            column_group_r <= 1'b1;
            state <= ST_MULTIPLY;
```

```
                    end else if (suppress_output_r) begin
                        column_group_r <= 1'b0;
                        if (result_row_r == result_row_count - 1'b1) begin
                            if (compute_mode_r == MODE_INPUT_GRADIENT
                                    && mode_r == MODE_BOTH) begin
                                state <= ST_SWITCH_DW;
                            end else begin
                                completed_tiles <= completed_tiles + 1'b1;
                                state <= ST_DONE;
                            end
                        end else begin
                            result_row_r <= result_row_r + 1'b1;
                            state <= ST_MULTIPLY;
                        end
                    end else begin
                        output_word_r <= 2'd0;
                        if (compute_mode_r == MODE_INPUT_GRADIENT) begin
                            state <= ST_OUTPUT_DX;
                        end else begin
                            state <= ST_OUTPUT_DW;
                        end
                    end
                end

                ST_OUTPUT_DX: if (m_axis_tready) begin
                    if (output_word_r == 3) begin
                        output_word_r <= 2'd0;
                        column_group_r <= 1'b0;
                        reduction_group_r <= 1'b0;
                        if (result_row_r == rows_r - 1'b1) begin
                            if (mode_r == MODE_BOTH) state <= ST_SWITCH_DW;
                            else begin
                                completed_tiles <= completed_tiles + 1'b1;
                                accumulator_valid_r <= 1'b0;
                                state <= ST_DONE;
                            end
                        end else begin
                            result_row_r <= result_row_r + 1'b1;
                            state <= ST_MULTIPLY;
                        end
                    end else output_word_r <= output_word_r + 1'b1;
                end

                ST_OUTPUT_DW: if (m_axis_tready) begin
                    if (output_word_r == 2) begin
                        output_word_r <= 2'd0;
                        column_group_r <= 1'b0;
                        reduction_group_r <= 1'b0;
                        if (result_row_r == inputs_r - 1'b1) begin
                            completed_tiles <= completed_tiles + 1'b1;
                            accumulator_valid_r <= 1'b0;
                            state <= ST_DONE;
                        end else begin
                            result_row_r <= result_row_r + 1'b1;
                            state <= ST_MULTIPLY;
                        end
                    end else output_word_r <= output_word_r + 1'b1;
                end

                ST_SWITCH_DW: begin
                    for (int row = 0; row < LANES; row++) begin
                        result_row_buffer_r[row] <= 32'sd0;
                    end
                    compute_mode_r <= MODE_WEIGHT_GRADIENT;
                    input_gradient_lane_r <= {RESULT_LANES{1'b0}};
                    compute_result_row_count_r <= inputs_r;
                    compute_result_column_count_r <= outputs_r;
                    compute_reduction_count_r <= rows_r;
                    result_shift_r <= dw_shift_r;
                    rounding_bias_r <= round_bias(dw_shift_r);
                    accumulator_mode_r <= MODE_WEIGHT_GRADIENT;
                    accumulator_rows_r <= inputs_r;
                    accumulator_columns_r <= outputs_r;
                    result_row_r <= 4'd0;
                    column_group_r <= 1'b0;
                    reduction_group_r <= 1'b0;
                    output_word_r <= 2'd0;
                    state <= ST_MULTIPLY;
                end

                ST_DONE: state <= ST_IDLE;
                default: state <= ST_IDLE;
            endcase

            if (clear_counters) begin
                completed_tiles <= 32'd0;
                active_cycles <= 32'd0;
```

```
                accumulator_valid_r <= 1'b0;
            end
        end
    end

    initial begin
        if (LANES != 8) begin
            $fatal(1, "neuraedge_linear_backward8 currently requires LANES=8");
        end
    end
endmodule

`default_nettype wire
```

13.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `// Exact tiled reverse-mode engine for Y = XW:` Design intent: Exact tiled reverse-mode engine for Y = XW: This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 5: `// dX = dY * transpose(W), dW = transpose(X) * dY.` Design intent: dX = dY * transpose(W), dW = transpose(X) * dY. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 6: `// The external tile remains 8x8x8, while a 4x4 DSP array traverses two` Design intent: The external tile remains 8x8x8, while a 4x4 DSP array traverses two This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 7: `// result-column groups and up to two reduction groups. One shared 47-bit` Design intent: result-column groups and up to two reduction groups. One shared 47-bit This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 8: `// accumulator bank preserves cross-command partial sums until the compiler` Design intent: accumulator bank preserves cross-command partial sums until the compiler This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 9: `// requests the final rounded output.` Design intent: requests the final rounded output. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 10: `module neuraedge_linear_backward8 #(` This opens neuraedge_linear_backward8, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 11: `parameter int LANES = 8` This statement continues through source line 45; the explanation uses the complete expression rather than only the first visual fragment. This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output

logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 12: `) (` This is continuation line 2 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘) (‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 13: `input wire clk,` This is continuation line 3 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘input wire clk,‘. The important operands are input: the value named ’input’ in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 14: `input wire rst_n,` This is continuation line 4 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘input wire rst_n,‘. The important operands are input: the value named ’input’ in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 15: `input wire clear_counters,` This is continuation line 5 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘input wire clear_counters,‘. The important operands are input: the value named ’input’ in this block; clear_counters: a hardware telemetry register that accumulates measured events. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 17: `input wire start_valid,` This is continuation line 7 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘input wire start_valid,‘. The important operands are input: the value named ’input’ in

this block; start_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 18: `output logic start_ready,` This is continuation line 8 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'output logic start_ready,'. The important operands are output: the value named 'output' in this block; start_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 19: `input wire [3:0] active_rows,` This is continuation line 9 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire [3:0] active_rows,'. The important operands are input: the value named 'input' in this block; active_rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 20: `input wire [3:0] active_inputs,` This is continuation line 10 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire [3:0] active_inputs,'. The important operands are input: the value named 'input' in this block; active_inputs: the value named 'active inputs' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 21: `input wire [3:0] active_outputs,` This is continuation line 11 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire [3:0] active_outputs,'. The important operands are input: the value named 'input' in this block; active_outputs: the value named 'active outputs' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input

wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 22: `input wire [4:0] input_gradient_shift,` This is continuation line 12 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire [4:0] input_gradient_shift,'. The important operands are input: the value named 'input' in this block; input_gradient_shift: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 23: `input wire [4:0] weight_gradient_shift,` This is continuation line 13 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire [4:0] weight_gradient_shift,'. The important operands are input: the value named 'input' in this block; weight_gradient_shift: a learned coefficient multiplied by an activation. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 24: `input wire [1:0] gradient_mode,` This is continuation line 14 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire [1:0] gradient_mode,'. The important operands are input: the value named 'input' in this block; gradient_mode: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 25: `input wire accumulate_result,` This is continuation line 15 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire accumulate_result,'. The important operands are input: the value named 'input' in this block; accumulate_result: the value named 'accumulate result' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 26: `input wire suppress_output,` This is continuation line 16 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire suppress_output,'. The important operands are input: the value named 'input' in this block; suppress_output: the value named 'suppress output' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 27: `input wire reserved_nonzero,` This is continuation line 17 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire reserved_nonzero,'. The important operands are input: the value named 'input' in this block; reserved_nonzero: the value named 'reserved nonzero' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 29: `input wire s_axis_tvalid,` This is continuation line 19 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire s_axis_tvalid,'. The important operands are input: the value named 'input' in this block; s_axis_tvalid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 30: `output logic s_axis_tready,` This is continuation line 20 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'output logic s_axis_tready,'. The important operands are output: the value named 'output' in this block; s_axis_tready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 31: `input wire [63:0] s_axis_tdata,` This is continuation line 21 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire [63:0] s_axis_tdata,'. The important operands are input: the value named 'input' in this block; s_axis_tdata: the value named 's axis tdata' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs,

input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 32: `input wire s_axis_tlast,` This is continuation line 22 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire s_axis_tlast,'. The important operands are input: the value named 'input' in this block; s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 34: `output logic m_axis_tvalid,` This is continuation line 24 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'output logic m_axis_tvalid,'. The important operands are output: the value named 'output' in this block; m_axis_tvalid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 35: `input wire m_axis_tready,` This is continuation line 25 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'input wire m_axis_tready,'. The important operands are input: the value named 'input' in this block; m_axis_tready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 36: `output logic [63:0] m_axis_tdata,` This is continuation line 26 of the 35-line logical statement that begins at source line 11. Its exact fragment is 'output logic [63:0] m_axis_tdata,'. The important operands are output: the value named 'output' in this block; m_axis_tdata: the value named 'm axis tdata' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles,

output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 37: `output logic m_axis_tlast,` This is continuation line 27 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘output logic m_axis_tlast,‘. The important operands are output: the value named ’output’ in this block; m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 39: `output logic busy,` This is continuation line 29 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘output logic busy,‘. The important operands are output: the value named ’output’ in this block; busy: the value named ’busy’ in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 40: `output logic done,` This is continuation line 30 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘output logic done,‘. The important operands are output: the value named ’output’ in this block; done: the value named ’done’ in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 41: `output logic config_error,` This is continuation line 31 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘output logic config_error,‘. The important operands are output: the value named ’output’ in this block; config_error: the value named ’config error’ in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 42: `(* extract_enable = "no" *)` This is continuation line 32 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘(* extract_enable = "no" *)‘. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named ’no’ in this block. Read together,

the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 43: `output logic [31:0] completed_tiles,` This is continuation line 33 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘output logic [31:0] completed_tiles,‘. The important operands are output: the value named ’output’ in this block; completed_tiles: the value named ’completed tiles’ in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 44: `output logic [31:0] active_cycles` This is continuation line 34 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘output logic [31:0] active_cycles‘. The important operands are output: the value named ’output’ in this block; active_cycles: the value named ’active cycles’ in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 45: `);` This is continuation line 35 of the 35-line logical statement that begins at source line 11. Its exact fragment is ‘);‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire start_valid, output logic start_ready, input wire [3:0] active_rows, input wire [3:0] active_inputs, input wire [3:0] active_outputs, input wire [4:0] input_gradient_shift, input wire [4:0] weight_gradient_shift, input wire [1:0] gradient_mode, input wire accumulate_result, input wire suppress_output, input wire reserved_nonzero, input wire s_axis_tvalid, output logic s_axis_tready, input wire [63:0] s_axis_tdata, input wire s_axis_tlast, output logic m_axis_tvalid, input wire m_axis_tready, output logic [63:0] m_axis_tdata, output logic m_axis_tlast, output logic busy, output logic done, output logic config_error, (* extract_enable = "no" *) output logic [31:0] completed_tiles, output logic [31:0] active_cycles) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 46: `localparam int RESULT_LANES = 4;` This fixes RESULT_LANES to 4 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; RESULT_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 47: `localparam int REDUCTION_LANES = 4;` This fixes REDUCTION_LANES to 4 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; REDUCTION_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 49: `localparam logic [1:0] MODE_BOTH = 2'd0;` This fixes MODE_BOTH to 2'd0 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_BOTH denotes the value named 'mode both' in this block.

Line 50: `localparam logic [1:0] MODE_INPUT_GRADIENT = 2'd1;` This fixes MODE_INPUT_GRADIENT to 2'd1 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_INPUT_GRADIENT denotes the signed loss derivative used to update a learned parameter.

Line 51: `localparam logic [1:0] MODE_WEIGHT_GRADIENT = 2'd2;` This fixes MODE_WEIGHT_GRADIENT to 2'd2 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_WEIGHT_GRADIENT denotes a learned coefficient multiplied by an activation.

Line 53: `typedef enum logic [4:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 54: `ST_IDLE = 5'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 55: `ST_LOAD_X = 5'd1,` This assigns a fixed binary encoding to state ST_LOAD_X, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 56: `ST_LOAD_W = 5'd2,` This assigns a fixed binary encoding to state ST_LOAD_W, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 57: `ST_LOAD_DY = 5'd3,` This assigns a fixed binary encoding to state ST_LOAD_DY, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 58: `ST_MULTIPLY = 5'd4,` This assigns a fixed binary encoding to state ST_MULTIPLY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 59: `ST_PRODUCT = 5'd5,` This assigns a fixed binary encoding to state ST_PRODUCT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 60: `ST_PAIR = 5'd6,` This assigns a fixed binary encoding to state ST_PAIR, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 61: `ST_REDUCE = 5'd7,` This assigns a fixed binary encoding to state ST_REDUCE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 62: `ST_COMMIT = 5'd8,` This assigns a fixed binary encoding to state ST_COMMIT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 63: `ST_OUTPUT_DX = 5'd9,` This assigns a fixed binary encoding to state ST_OUTPUT_DX, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 64: `ST_OUTPUT_DW = 5'd10,` This assigns a fixed binary encoding to state ST_OUTPUT_DW, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 65: `ST_SWITCH_DW = 5'd11,` This assigns a fixed binary encoding to state ST_SWITCH_DW, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 66: `ST_DONE = 5'd12,` This assigns a fixed binary encoding to state ST_DONE, which raises completion for one cycle and returns control to idle. The encoding is visible in waveforms even though the source uses the readable name.

Line 67: `ST_ROUND_MAGNITUDE = 5'd13,` This assigns a fixed binary encoding to state ST_ROUND_MAGNITUDE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 68: `ST_ROUND_BIAS = 5'd14,` This assigns a fixed binary encoding to state ST_ROUND_BIAS, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 69: `ST_ROUND_SHIFT = 5'd15,` This assigns a fixed binary encoding to state ST_ROUND_SHIFT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 70: `ST_ROUND_SIGN = 5'd16,` This assigns a fixed binary encoding to state ST_ROUND_SIGN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 71: `ST_SATURATE = 5'd17,` This assigns a fixed binary encoding to state ST_SATURATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 72: `ST_ADD_ACCUMULATOR = 5'd18,` This assigns a fixed binary encoding to state ST_ADD_ACCUMULATOR, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 73: `ST_STORE_ACCUMULATOR = 5'd19,` This assigns a fixed binary encoding to state ST_STORE_ACCUMULATOR, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 74: `ST_REDUCTION_SUM = 5'd20,` This assigns a fixed binary encoding to state ST_REDUCTION_SUM, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 75: `ST_STORE_RESULT = 5'd21,` This assigns a fixed binary encoding to state ST_STORE_RESULT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 76: `ST_REGISTER_OPERANDS = 5'd22,` This assigns a fixed binary encoding to state ST_REGISTER_OPERANDS, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 77: `ST_CONFIGURE = 5'd23,` This assigns a fixed binary encoding to state ST_CONFIGURE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 78: `ST_ROUND_MAGNITUDE_HIGH = 5'd24,` This assigns a fixed binary encoding to state ST_ROUND_MAGNITUDE_HIGH which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 79: `ST_ROUND_BIAS_HIGH = 5'd25,` This assigns a fixed binary encoding to state ST_ROUND_BIAS_HIGH, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 80: `ST_ROUND_SIGN_HIGH = 5'd26,` This assigns a fixed binary encoding to state ST_ROUND_SIGN_HIGH, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 81: `ST_LOAD_COMMIT_DRAIN = 5'd27` This statement continues through source line 82; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_LOAD_COMMIT_DRAIN, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 82: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 81. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_LOAD_COMMIT_DRAIN, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 84: `// Keep the compact encoding, but route next-state decisions to D rather` Design intent: Keep the compact encoding, but route next-state decisions to D rather This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 85: `// than the tighter-setup CE pin used by automatic enable extraction.` Design intent: than the tighter-setup CE pin used by automatic enable extraction. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 86: `(* fsm_encoding = "sequential", extract_enable = "no" *) state_t state;` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are fsm_encoding: the value named 'fsm encoding' in this block; sequential: the value named 'sequential' in this block; extract_enable: a control bit that permits the associated state or storage update.

Line 87: `logic [3:0] rows_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rows_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 88: `logic [3:0] inputs_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, inputs_r is the value named 'inputs r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 89: `logic [3:0] outputs_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, outputs_r is the value named 'outputs r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 90: `logic [4:0] dw_shift_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dw_shift_r is the value named 'dw shift r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 91: `logic [4:0] result_shift_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_shift_r is the value named 'result shift r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 92: `logic [46:0] rounding_bias_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rounding_bias_r is an additive learned offset applied after accumulation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 93: `logic [1:0] mode_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mode_r is the value named 'mode r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 94: `logic [1:0] compute_mode_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, compute_mode_r is the value named 'compute mode r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 95: `logic suppress_output_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, suppress_output_r is the value named 'suppress output r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 96: `logic accumulate_result_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulate_result_r is the value named 'accumulate result r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 97: `logic start_configuration_invalid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, start_configuration_invalid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 98: `logic [1:0] requested_accumulator_mode_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, requested_accumulator_mode_r is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 99: `logic [3:0] requested_accumulator_rows_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, requested_accumulator_rows_r is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 100: `logic [3:0] requested_accumulator_columns_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, requested_accumulator_columns_r is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 101: `logic [3:0] compute_result_row_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, compute_result_row_count_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 102: `logic [3:0] compute_result_column_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, compute_result_column_count_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 103: `logic [3:0] compute_reduction_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, compute_reduction_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 105: `// Each lane owns a preserved copy of the mode-select bit. This limits the` Design intent: Each lane owns a preserved copy of the mode-select bit. This limits the This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 106: `// physical fanout into the four DSP lanes while retaining one logical mode.` Design intent: physical fanout into the four DSP lanes while retaining one logical mode. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 107: `(* keep = "true", dont_touch = "true" *)` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are keep: the value named 'keep' in this block; dont_touch: the value named 'dont touch' in this block.

Line 108: `logic [RESULT_LANES-1:0] input_gradient_lane_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_gradient_lane_r is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 110: `// These banks are loaded under several FSM states. Implementing the hold` Design intent: These banks are loaded under several FSM states. Implementing the hold This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 111: `// condition on D keeps a distant state decode off thousands of FDRE CE` Design intent: condition on D keeps a distant state decode off thousands of FDRE CE This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 112: `// pins and lets Vivado place each small input mux beside its register.` Design intent: pins and lets Vivado place each small input mux beside its register. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 113: `(* extract_enable = "no" *)` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block.

Line 114: `logic signed [15:0] activation_r [0:LANES-1][0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 115: `(* extract_enable = "no" *)` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block.

Line 116: `logic signed [15:0] weight_r [0:LANES-1][0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 117: `(* extract_enable = "no" *)` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block.

Line 118: `logic signed [15:0] output_gradient_r [0:LANES-1][0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_gradient_r is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 120: `// Four banks provide the four independent reads and writes needed by the` Design intent: Four banks provide the four independent reads and writes needed by the This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 121: `// result lanes. Each bank stores one column from both four-column groups` Design intent: result lanes. Each bank stores one column from both four-column groups This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 122: `// for every row: address = {row, column_group}. Commands that do not` Design intent: for every row: address = {row, column_group}. Commands that do not This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 123: `// accumulate select mathematical zero instead of clearing the RAM. This` Design intent: accumulate select mathematical zero instead of clearing the RAM. This This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 124: `// preserves exact accumulation while removing a clear/hold/store mux and` Design intent: preserves exact accumulation while removing a clear/hold/store mux and This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 125: `// its FSM control route from 3,008 individual flip-flops.` Design intent: its FSM control route from 3,008 individual flip-flops. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 126: `wire signed [46:0] accumulator_bank_read_data` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are accumulator_bank_read_data: a widened running sum that prevents intermediate overflow.

Line 127: `[0:RESULT_LANES-1];` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are RESULT_LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 128: `wire signed [46:0] accumulator_bank_write_data` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are accumulator_bank_write_data: a widened running sum that prevents intermediate overflow.

Line 129: `[0:RESULT_LANES-1];` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are RESULT_LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 130: `wire [RESULT_LANES-1:0] accumulator_bank_write_enable;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulator_bank_write_enable is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 131: `logic accumulator_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulator_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 132: `logic [1:0] accumulator_mode_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulator_mode_r is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 133: `logic [3:0] accumulator_rows_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulator_rows_r is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 134: `logic [3:0] accumulator_columns_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulator_columns_r is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 136: `logic signed [31:0] result_row_buffer_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_row_buffer_r is local storage that decouples producer and consumer timing. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 137: `// Eight signed 16x16 products fit exactly in 35 bits. Keeping this` Design intent: Eight signed 16x16 products fit exactly in 35 bits. Keeping this This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 138: `// command-local sum at its mathematical width shortens the carry chain;` Design intent: command-local sum at its mathematical width shortens the carry chain; This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 139: `// it is sign-extended only when added to the retained 47-bit accumulator.` Design intent: it is sign-extended only when added to the retained 47-bit accumulator. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 140: `logic signed [34:0] row_partial_r [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, row_partial_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 141: `logic signed [33:0] reduction_sum_r [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reduction_sum_r is the value named 'reduction sum r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 142: `logic signed [46:0] selected_accumulator_r [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, selected_accumulator_r is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 143: `logic signed [47:0] accumulated_wide_r [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulated_wide_r is the value named 'accumulated wide r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 144: `logic commit_lane_valid_r [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, commit_lane_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 145: `logic [46:0] rounding_value_r [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rounding_value_r is the value named 'rounding value r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 146: `logic rounding_negative_r [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rounding_negative_r is the value named 'rounding negative r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 147: `logic rounding_carry_r [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rounding_carry_r is the value named 'rounding carry r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 148: `logic signed [31:0] saturated_result_r [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, saturated_result_r is the value named 'saturated result r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 150: `logic [3:0] load_row_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_row_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 151: `logic load_half_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_half_r is the value named 'load half r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 152: `logic load_final_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_final_r is the value named 'load final r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 153: `// Stream VALID previously participated directly in the D-input mux for` Design intent: Stream VALID previously participated directly in the D-input mux for This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 154: `// all 3,072 operand-bank bits. This elastic stage accepts one beat every` Design intent: all 3,072 operand-bank bits. This elastic stage accepts one beat every This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 155: `// clock, while the row-local commit enables below terminate control near` Design intent: clock, while the row-local commit enables below terminate control near This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 156: `// the destination registers. The payload stage can consume its old beat` Design intent: the destination registers. The payload stage can consume its old beat This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 157: `// and capture the next operand class on the same edge.` Design intent: and capture the next operand class on the same edge. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 158: `logic input_stage_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_stage_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 159: `logic [63:0] input_stage_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_stage_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 160: `logic input_stage_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_stage_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 161: `logic [63:0] load_commit_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_commit_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 162: `logic [1:0] activation_write_enable_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_write_enable_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 163: `logic [1:0] weight_write_enable_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_write_enable_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 164: `logic [1:0] output_gradient_write_enable_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_gradient_write_enable_r is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 165: `// Keep the row hold as a D-input mux. A dedicated CE would put the` Design intent: Keep the row hold as a D-input mux. A dedicated CE would put the This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 166: `// multi-state row-update decode on the FDRE CE pin, whose setup time is` Design intent: multi-state row-update decode on the FDRE CE pin, whose setup time is This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 167: `// tighter than the data input on 7-series devices.` Design intent: tighter than the data input on 7-series devices. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 168: `(* extract_enable = "no" *) logic [3:0] result_row_r;` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block; result_row_r: one output-row position processed by the tensor tile.

Line 169: `logic column_group_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, column_group_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 170: `logic reduction_group_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reduction_group_r is the value named 'reduction group r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 171: `logic [1:0] output_word_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_word_r is the value named 'output word r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 173: `wire [2:0] load_row_index = load_row_r[2:0];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_row_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 174: `wire [2:0] result_row_index = result_row_r[2:0];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_row_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 175: `wire [3:0] accumulator_address = {result_row_index, column_group_r};` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulator_address is a widened running sum that prevents intermediate overflow; column_group_r} is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 176: `wire [3:0] result_row_count = compute_result_row_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_row_count is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 177: `wire [3:0] result_column_count = compute_result_column_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_column_count is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 178: `wire [3:0] reduction_count = compute_reduction_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reduction_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 179: `wire input_load_state = state == ST_LOAD_X || state == ST_LOAD_W` This statement continues through source line 180; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_load_state is the finite-state-machine phase retained in flip-flops. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 180: `|| state == ST_LOAD_DY;` This is continuation line 2 of the 2-line logical statement that begins at source line 179. Its exact fragment is '|| state == ST_LOAD_DY;'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD_DY: the value named 'st load dy' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_load_state is the finite-state-machine phase retained in flip-flops. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 181: `wire input_stage_consume = input_stage_valid_r && input_load_state;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_stage_consume is the value named 'input stage consume' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 182: `wire input_stage_ready = !input_stage_valid_r || input_stage_consume;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_stage_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 183: `wire input_final_buffered = state == ST_LOAD_DY && input_stage_valid_r` This statement continues through source line 184; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_final_buffered is

local storage that decouples producer and consumer timing. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 184: `&& load_half_r && load_final_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 183. Its exact fragment is '&& load_half_r && load_final_r;'. The important operands are load_half_r: the value named 'load half r' in this block; load_final_r: the value named 'load final r' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_final_buffered is local storage that decouples producer and consumer timing. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 185: `wire [1:0] requested_accumulator_mode = gradient_mode == MODE_WEIGHT_GRADIENT` This statement continues through source line 186; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, requested_accumulator_mode is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 186: `? MODE_WEIGHT_GRADIENT : MODE_INPUT_GRADIENT;` This is continuation line 2 of the 2-line logical statement that begins at source line 185. Its exact fragment is '? MODE_WEIGHT_GRADIENT : MODE_INPUT_GRADIENT;'. The important operands are MODE_WEIGHT_GRADIENT: a learned coefficient multiplied by an activation; MODE_INPUT_GI the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, requested_accumulator_mode is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 187: `wire [3:0] requested_accumulator_rows = gradient_mode == MODE_WEIGHT_GRADIENT` This statement continues through source line 188; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, requested_accumulator_rows is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 188: `? active_inputs : active_rows;` This is continuation line 2 of the 2-line logical statement that begins at source line 187. Its exact fragment is '? active_inputs : active_rows;'. The important operands are active_inputs: the value named 'active inputs' in this block; active_rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, requested_accumulator_rows is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 189: `wire [3:0] requested_accumulator_columns = gradient_mode == MODE_WEIGHT_GRADIENT` This statement continues through source line 190; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, requested_accumulator_columns is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 190: `? active_outputs : active_inputs;` This is continuation line 2 of the 2-line logical statement that begins at source line 189. Its exact fragment is '? active_outputs : active_inputs;'. The important operands are active_outputs: the value named 'active outputs' in this block; active_inputs: the value named 'active inputs' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, requested_accumulator_columns is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 191: `wire dimensions_invalid = active_rows == 0 || active_rows > 4'(LANES)` This statement continues through source line 193; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dimensions_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 192: `|| active_inputs == 0 || active_inputs > 4'(LANES)` This is continuation line 2 of the 3-line logical statement that begins at source line 191. Its exact fragment is '|| active_inputs == 0 || active_inputs > 4'(LANES)'. The important operands are active_inputs: the value named 'active inputs' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dimensions_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 193: `|| active_outputs == 0 || active_outputs > 4'(LANES);` This is continuation line 3 of the 3-line logical statement that begins at source line 191. Its exact fragment is '|| active_outputs == 0 || active_outputs > 4'(LANES);'. The important operands are active_outputs: the value named 'active outputs' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dimensions_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 194: `wire accumulation_shape_invalid = accumulate_result && (` This statement continues through source line 199; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulation_shape_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 195: `gradient_mode == MODE_BOTH` This is continuation line 2 of the 6-line logical statement that begins at source line 194. Its exact fragment is 'gradient_mode == MODE_BOTH'. The important operands are gradient_mode: the signed loss derivative used to update a learned parameter; MODE_BOTH: the value named 'mode both' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulation_shape_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 196: `|| !accumulator_valid_r` This is continuation line 3 of the 6-line logical statement that begins at source line 194. Its exact fragment is '|| !accumulator_valid_r'. The important operands are accumulator_valid_r: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulation_shape_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 197: `|| accumulator_mode_r != requested_accumulator_mode` This is continuation line 4 of the 6-line logical statement that begins at source line 194. Its exact fragment is '|| accumulator_mode_r != requested_accumulator_mode'. The important operands are accumulator_mode_r: a widened running sum that prevents intermediate overflow; requested_accumulator_mode: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulation_shape_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 198: `|| accumulator_rows_r != requested_accumulator_rows` This is continuation line 5 of the 6-line logical statement that begins at source line 194. Its exact fragment is '|| accumulator_rows_r != requested_accumulator_rows'. The important operands are accumulator_rows_r: a widened running sum that prevents intermediate overflow; requested_accumulator_rows: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulation_shape_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 199: `|| accumulator_columns_r != requested_accumulator_columns);` This is continuation line 6 of the 6-line logical statement that begins at source line 194. Its exact fragment is '|| accumulator_columns_r != requested_accumulator_col

The important operands are accumulator_columns_r: a widened running sum that prevents intermediate overflow; requested_accumulator_columns: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulation_shape_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 200: `wire start_configuration_invalid = reserved_nonzero || dimensions_invalid` This statement continues through source line 201; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, start_configuration_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 201: `|| gradient_mode == 2'd3 || accumulation_shape_invalid;` This is continuation line 2 of the 2-line logical statement that begins at source line 200. Its exact fragment is '|| gradient_mode == 2'd3 || accumulation_shape_invalid;'. The important operands are gradient_mode: the signed loss derivative used to update a learned parameter; d3: the value named 'd3' in this block; accumulation_shape_invalid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, start_configuration_invalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 203: `logic signed [15:0] multiplier_left [0:RESULT_LANES-1][0:REDUCTION_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_left is the value named 'multiplier left' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 204: `logic signed [15:0] multiplier_right [0:RESULT_LANES-1][0:REDUCTION_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_right is the value named 'multiplier right' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 205: `(* keep = "true", dont_touch = "true" *)` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are keep: the value named 'keep' in this block; dont_touch: the value named 'dont touch' in this block.

Line 206: `logic signed [15:0] multiplier_left_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_left_r is the value named 'multiplier left r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 207: `(* keep = "true", dont_touch = "true" *)` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are keep: the value named 'keep' in this block; dont_touch: the value named 'dont touch' in this block.

Line 208: `logic signed [15:0] multiplier_right_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_right_r is the value named 'multiplier right r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 209: `logic signed [15:0] multiplier_left_dsp_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_left_dsp_r is the value named 'multiplier left dsp r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 210: `logic signed [15:0] multiplier_right_dsp_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_right_dsp_r is the value named 'multiplier right dsp r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 211: `(* use_dsp = "yes" *) wire signed [31:0]` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block.

Line 212: `multiplier_product [0:RESULT_LANES-1][0:REDUCTION_LANES-1];` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_product: the value named 'multiplier product' in this block; RESULT_LANES: one parallel arithmetic element within a vector or tensor datapath; REDUCTION_LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 213: `logic signed [31:0] product_r [0:RESULT_LANES-1][0:REDUCTION_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_r is the value named 'product r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 214: `logic signed [32:0] pair_sum_r [0:RESULT_LANES-1][0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pair_sum_r is the value named 'pair sum r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 215: `// This local pulse drives only the eight pair-reduction DSP registers.` Design intent: This local pulse drives only the eight pair-reduction DSP registers. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 216: `// Registering it near the array avoids routing the shared FSM decode to` Design intent: Registering it near the array avoids routing the shared FSM decode to This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 217: `// DSP clock-enable pins, whose setup requirement is especially tight.` Design intent: DSP clock-enable pins, whose setup requirement is especially tight. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 218: `logic pair_update_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pair_update_r is the value named 'pair update r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 220: `wire [3:0] result_column_index [0:RESULT_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_column_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 221: `wire [3:0] reduction_index [0:REDUCTION_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reduction_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 222: `logic [191:0] packed_weight_gradient;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, packed_weight_gradient is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 224: `function automatic logic signed [46:0] saturate_s47(` Within the combinational helper saturate s47, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_s47: the value named 'saturate s47' in this block.

Line 225: `input logic signed [47:0] value` This declares value as a input port with width [47:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 226: `);` Within the combinational helper saturate s47, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The expression is a literal or structural operator and has no hidden software state.

Line 227: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s47; it has no arithmetic value of its own.

Line 228: `if (value[47] == value[46]) saturate_s47 = value[46:0];` This condition selects the following hardware branch only when 'value[47] == value[46]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the combinational helper saturate s47.

Line 229: `else if (value[47]) saturate_s47 = {1'b1, {46{1'b0}}};` This condition selects the following hardware branch only when 'value[47]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the combinational helper saturate s47.

Line 230: `else saturate_s47 = {1'b0, {46{1'b1}}};` Within the combinational helper saturate s47, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are saturate_s47: the value named 'saturate s47' in this block; b0: the value named 'b0' in this block; b1: the value named 'b1' in this block.

Line 231: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s47; it has no arithmetic value of its own.

Line 232: `endfunction` Within the combinational helper saturate s47, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are endfunction: the value named 'endfunction' in this block.

Line 234: `function automatic logic [46:0] round_bias(` Within the combinational helper round bias, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; round_bias: an additive learned offset applied after accumulation.

Line 235: `input logic [4:0] shift` This declares shift as a input port with width [4:0]. It carries the value named 'shift' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 236: `);` Within the combinational helper round bias, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The expression is a literal or structural operator and has no hidden software state.

Line 237: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper round bias; it has no arithmetic value of its own.

Line 238: `if (shift == 0) round_bias = 47'd0;` This condition selects the following hardware branch only when 'shift == 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are shift: the value named 'shift' in this block. Context: the combinational helper round bias.

Line 239: `else round_bias = 47'd1 << (shift - 1'b1);` Within the combinational helper round bias, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are round_bias: an additive learned offset applied after accumulation; d1: the value named 'd1' in this block; shift: the value named 'shift' in this block.

Line 240: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper round bias; it has no arithmetic value of its own.

Line 241: `endfunction` Within the combinational helper round bias, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are endfunction: the value named 'endfunction' in this block.

Line 243: `function automatic logic signed [31:0] saturate_s32(` Within the combinational helper saturate s32, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_s32: the value named 'saturate s32' in this block.

Line 244: `input logic signed [46:0] value` This declares value as a input port with width [46:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 245: `);` Within the combinational helper saturate s32, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The expression is a literal or structural operator and has no hidden software state.

Line 246: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32; it has no arithmetic value of its own.

Line 247: `// A signed value fits in 32 bits exactly when every discarded bit` Design intent: A signed value fits in 32 bits exactly when every discarded bit This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 248: `// is a copy of bit 31. This is equivalent to range comparisons,` Design intent: is a copy of bit 31. This is equivalent to range comparisons, This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 249: `// but uses equality/reduction logic instead of two carry chains.` Design intent: but uses equality/reduction logic instead of two carry chains. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 250: `if (value[46:31] == {16{value[31]}}) begin` This condition selects the following hardware branch only when 'value[46:31] == {16{value[31]}}' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the combinational helper saturate s32.

Line 251: `saturate_s32 = value[31:0];` This makes saturate_s32, the value named 'saturate s32' in this block, equal to 'value[31:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper saturate s32.

Line 252: `end else if (value[46]) begin` This condition selects the following hardware branch only when ‘value[46]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named ’value’ in this block. Context: the combinational helper saturate s32.

Line 253: `saturate_s32 = 32'sh80000000;` This makes saturate_s32, the value named ’saturate s32’ in this block, equal to ‘32’sh80000000‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are sh80000000: the value named ’sh80000000’ in this block. Context: the combinational helper saturate s32.

Line 254: `end else begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32; it has no arithmetic value of its own.

Line 255: `saturate_s32 = 32'sh7fffffff;` This makes saturate_s32, the value named ’saturate s32’ in this block, equal to ‘32’sh7fffffff‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are sh7fffffff: the value named ’sh7fffffff’ in this block. Context: the combinational helper saturate s32.

Line 256: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32; it has no arithmetic value of its own.

Line 257: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32; it has no arithmetic value of its own.

Line 258: `endfunction` Within the combinational helper saturate s32, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are endfunction: the value named ’endfunction’ in this block.

Line 260: `function automatic logic signed [23:0] saturate_s24(` Within the combinational helper saturate s24, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are function: the value named ’function’ in this block; automatic: the value named ’automatic’ in this block; saturate_s24: the value named ’saturate s24’ in this block.

Line 261: `input logic signed [46:0] value` This declares value as a input port with width [46:0]. It carries the value named ’value’ in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 262: `);` Within the combinational helper saturate s24, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The expression is a literal or structural operator and has no hidden software state.

Line 263: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24; it has no arithmetic value of its own.

Line 264: `if (value[46:23] == {24{value[23]}}) begin` This condition selects the following hardware branch only when ‘value[46:23] == {24{value[23]}}‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named ’value’ in this block. Context: the combinational helper saturate s24.

Line 265: `saturate_s24 = value[23:0];` This makes saturate_s24, the value named ’saturate s24’ in this block, equal to ‘value[23:0]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named ’value’ in this block. Context: the combinational helper saturate s24.

Line 266: `end else if (value[46]) begin` This condition selects the following hardware branch only when ‘value[46]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named ’value’ in this block. Context: the combinational helper saturate s24.

Line 267: `saturate_s24 = 24'sh800000;` This makes saturate_s24, the value named 'saturate s24' in this block, equal to '24'sh800000'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sh800000: the value named 'sh800000' in this block. Context: the combinational helper saturate s24.

Line 268: `end else begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24; it has no arithmetic value of its own.

Line 269: `saturate_s24 = 24'sh7fffff;` This makes saturate_s24, the value named 'saturate s24' in this block, equal to '24'sh7fffff'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sh7fffff: the value named 'sh7fffff' in this block. Context: the combinational helper saturate s24.

Line 270: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24; it has no arithmetic value of its own.

Line 271: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24; it has no arithmetic value of its own.

Line 272: `endfunction` Within the combinational helper saturate s24, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are endfunction: the value named 'endfunction' in this block.

Line 274: `function automatic logic signed [31:0] saturate_s24_extended(` Within the combinational helper saturate s24 extended, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_s24_extended: the value named 'saturate s24 extended' in this block.

Line 275: `input logic signed [46:0] value` This declares value as a input port with width [46:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 276: `);` Within the combinational helper saturate s24 extended, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The expression is a literal or structural operator and has no hidden software state.

Line 277: `logic signed [23:0] saturated;` This declares hardware signal storage or wiring in the combinational helper saturate s24 extended. Specifically, saturated is the value named 'saturated' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 278: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24 extended; it has no arithmetic value of its own.

Line 279: `saturated = saturate_s24(value);` This makes saturated, the value named 'saturated' in this block, equal to 'saturate_s24(value)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_s24: the value named 'saturate s24' in this block; value: the value named 'value' in this block. Context: the combinational helper saturate s24 extended.

Line 280: `saturate_s24_extended = {{8{saturated[23]}}, saturated};` This makes saturate_s24_extended, the value named 'saturate s24 extended' in this block, equal to '{{8{saturated[23]}}, saturated}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturated: the value named 'saturated' in this block. Context: the combinational helper saturate s24 extended.

Line 281: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24 extended; it has no arithmetic value of its own.

Line 282: `endfunction` Within the combinational helper saturate s24 extended, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are endfunction: the value named 'endfunction' in this block.

Line 284: `for (genvar bank = 0; bank < RESULT_LANES; bank++) begin : g_accumulator_bank` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 285: `assign accumulator_bank_write_enable[bank] =` This statement continues through source line 286; the explanation uses the complete expression rather than only the first visual fragment. This makes accumulator_bank_write_enable[ba a widened running sum that prevents intermediate overflow, equal to 'state == ST_STORE_ACCUMULATOR && commit_lane_valid_r[bank]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_STORE_ACCUMULATOR: a widened running sum that prevents intermediate overflow; commit_lane_valid_r: the producer's statement that its current payload is meaningful. Context: the module's structural declarations.

Line 286: `state == ST_STORE_ACCUMULATOR && commit_lane_valid_r[bank];` This is continuation line 2 of the 2-line logical statement that begins at source line 285. Its exact fragment is 'state == ST_STORE_ACCUMULATOR && commit_lane_valid_r[bank];'. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_STORE_ACCUMULATOR: a widened running sum that prevents intermediate overflow; commit_lane_valid_r: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This makes accumulator_bank_write_enable[bank], a widened running sum that prevents intermediate overflow, equal to 'state == ST_STORE_ACCUMULATOR && commit_lane_valid_r[bank]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_STORE_ACCUMULATOR: a widened running sum that prevents intermediate overflow; commit_lane_valid_r: the producer's statement that its current payload is meaningful. Context: the module's structural declarations.

Line 287: `assign accumulator_bank_write_data[bank] =` This statement continues through source line 288; the explanation uses the complete expression rather than only the first visual fragment. This makes accumulator_bank_write_data[bank], a widened running sum that prevents intermediate overflow, equal to 'saturate_s47(accumulated_wide_r[bank])'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_s47: the value named 'saturate s47' in this block; accumulated_wide_r: the value named 'accumulated wide r' in this block; bank: the value named 'bank' in this block. Context: the module's structural declarations.

Line 288: `saturate_s47(accumulated_wide_r[bank]);` This is continuation line 2 of the 2-line logical statement that begins at source line 287. Its exact fragment is 'saturate_s47(accumulated_wide_r[bank]);'. The important operands are saturate_s47: the value named 'saturate s47' in this block; accumulated_wide_r: the value named 'accumulated wide r' in this block; bank: the value named 'bank' in this block. Read together, the complete operation is: This makes accumulator_bank_write_data[bank], a widened running sum that prevents intermediate overflow, equal to 'saturate_s47(accumulated_wide_r[bank])'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_s47: the value named 'saturate s47' in this block; accumulated_wide_r: the value named 'accumulated wide r' in this block; bank: the value named 'bank' in this block. Context: the module's structural declarations.

Line 290: `neuraedge_gradient_accumulator_bank u_bank (` This instantiates module neuraedge_gradient_accumulator_ban as u_bank. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 291: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 292: `.write_enable(accumulator_bank_write_enable[bank]),` This connects the child instance's named port 'write_enable' to parent expression 'accumulator_bank_write_enable[bank]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a widened running sum that prevents intermediate overflow.

Line 293: `.write_address(accumulator_address),` This connects the child instance's named port 'write_address' to parent expression 'accumulator_address'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a widened running sum that prevents intermediate overflow.

Line 294: `.write_data(accumulator_bank_write_data[bank]),` This connects the child instance's named port 'write_data' to parent expression 'accumulator_bank_write_data[bank]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a widened running sum that prevents intermediate overflow.

Line 295: `.read_address(accumulator_address),` This connects the child instance's named port 'read_address' to parent expression 'accumulator_address'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a widened running sum that prevents intermediate overflow.

Line 296: `.read_data(accumulator_bank_read_data[bank])` This connects the child instance's named port 'read_data' to parent expression 'accumulator_bank_read_data[bank]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a widened running sum that prevents intermediate overflow.

Line 297: `);` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The expression is a literal or structural operator and has no hidden software state.

Line 298: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 300: `for (genvar result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 301: `result_lane++) begin : g_result_lane` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath; g_result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 302: `assign result_column_index[result_lane] =` This statement continues through source line 303; the explanation uses the complete expression rather than only the first visual fragment. This makes result_column_index[result_lane], an integer selecting one tensor element, lane, bank, or state entry, equal to '(column_group_r ? 4'd4 : 4'd0) + 4'(result_lane)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are column_group_r: one output-feature position processed in parallel; d4: the value named 'd4' in this block; d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 303: `(column_group_r ? 4'd4 : 4'd0) + 4'(result_lane);` This is continuation line 2 of the 2-line logical statement that begins at source line 302. Its exact fragment is '(column_group_r ? 4'd4 : 4'd0) + 4'(result_lane);'. The important operands are column_group_r: one output-feature position processed in parallel; d4: the value named 'd4' in this block; d0: the value named 'd0' in this block. Read together, the complete operation is: This makes result_column_index[result_lane], an integer selecting one tensor element, lane, bank, or state entry, equal to '(column_group_r ? 4'd4 : 4'd0) + 4'(result_lane)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are column_group_r: one output-feature position processed in parallel; d4: the value named 'd4' in this block; d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 304: `for (genvar reduction_lane = 0;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 305: `reduction_lane < REDUCTION_LANES; reduction_lane++) begin : g_multiplier` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath; REDUCTION_LANES: one parallel arithmetic element within a vector or tensor datapath; g_multiplier: the value named 'g multiplier' in this block.

Line 306: `assign multiplier_product[result_lane][reduction_lane] =` This statement continues through source line 308; the explanation uses the complete expression rather than only the first visual fragment. This makes multiplier_product[result_lane][reduction_lane], the value named 'multiplier product' in this block, equal to 'multiplier_left_dsp_r[resul * multiplier_right_dsp_r[result_lane][reduction_lane]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are multiplier_left_dsp_r: the value named 'multiplier left dsp r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the module's structural declarations.

Line 307: `multiplier_left_dsp_r[result_lane][reduction_lane]` This is continuation line 2 of the 3-line logical statement that begins at source line 306. Its exact fragment is 'multiplier_left_dsp_r[result_lane][reduction_lane]'. The important operands are multiplier_left_dsp_r: the value named 'multiplier left dsp r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes multiplier_product[result_lane][reduction_lane], the value named 'multiplier product' in this block, equal to 'multiplier_left_dsp_r[result_lane][reduction_lane] * multiplier_right_dsp_r[result_lane][reduction_lane]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are multiplier_left_dsp_r: the value named 'multiplier left dsp r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the module's structural declarations.

Line 308: `* multiplier_right_dsp_r[result_lane][reduction_lane];` This is continuation line 3 of the 3-line logical statement that begins at source line 306. Its exact fragment is '* multiplier_right_dsp_r[result_lane][reduction_lane];'. The important operands are multiplier_right_dsp_r: the value named 'multiplier right dsp r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes multiplier_product[result_lane][reduction_lane], the value named 'multiplier product' in this block, equal to 'multiplier_left_dsp_r[result_lane][reduction_lane] * multiplier_right_dsp_r[result_lane][reduction_lane]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are multiplier_left_dsp_r: the value named 'multiplier left dsp r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the module's structural declarations.

Line 309: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 310: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 312: `for (genvar reduction_lane = 0; reduction_lane < REDUCTION_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 313: `reduction_lane++) begin : g_reduction_index` Within the module's structural declarations, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath; g_reduction_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 314: `assign reduction_index[reduction_lane] =` This statement continues through source line 315; the explanation uses the complete expression rather than only the first visual fragment. This makes reduction_index[reduction_lane], an integer selecting one tensor element, lane, bank, or state entry, equal to '(reduction_group_r ? 4'd4 : 4'd0) + 4'(reduction_lane)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are

reduction_group_r: the value named 'reduction group r' in this block; d4: the value named 'd4' in this block; d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 315: `(reduction_group_r ? 4'd4 : 4'd0) + 4'(reduction_lane);` This is continuation line 2 of the 2-line logical statement that begins at source line 314. Its exact fragment is '(reduction_group_r ? 4'd4 : 4'd0) + 4'(reduction_lane);'. The important operands are reduction_group_r: the value named 'reduction group r' in this block; d4: the value named 'd4' in this block; d0: the value named 'd0' in this block. Read together, the complete operation is: This makes reduction_index[reduction_lane], an integer selecting one tensor element, lane, bank, or state entry, equal to '(reduction_group_r ? 4'd4 : 4'd0) + 4'(reduction_lane)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are reduction_group_r: the value named 'reduction group r' in this block; d4: the value named 'd4' in this block; d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 316: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 318: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 319: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 320: `result_lane++) begin` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 321: `for (int reduction_lane = 0; reduction_lane < REDUCTION_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 322: `reduction_lane++) begin` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 323: `multiplier_left[result_lane][reduction_lane] = 16'sd0;` This makes multiplier_left[result_lane][reduction_la the value named 'multiplier left' in this block, equal to '16'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 324: `multiplier_right[result_lane][reduction_lane] = 16'sd0;` This makes multiplier_right[result_lane][reduction the value named 'multiplier right' in this block, equal to '16'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 325: `if (result_row_r < result_row_count` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_row_r: one output-row position processed by the tensor tile; result_row_count: one output-row position processed by the tensor tile.

Line 326: `&& result_column_index[result_lane] < result_column_count` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_column_index: an integer selecting one tensor element, lane, bank, or state entry; result_lane: one parallel arithmetic element within a vector or tensor datapath; result_column_count: one output-feature position processed in parallel.

Line 327: `&& reduction_index[reduction_lane] < reduction_count) begin` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_count: an integer count retained for control or measurable telemetry.

Line 328: `if (input_gradient_lane_r[result_lane]) begin` This condition selects the following hardware branch only when ‘input_gradient_lane_r[result_lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_gradient_lane_r: the signed loss derivative used to update a learned parameter; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 329: `multiplier_left[result_lane][reduction_lane] =` This statement continues through source line 331; the explanation uses the complete expression rather than only the first visual fragment. This makes multiplier_left[result_lane][reduc the value named ’multiplier left’ in this block, equal to ‘output_gradient_r[result_row_index] [reduction_index[reduction_lane][2:0] It is driven by combinational or procedural logic in the current evaluation. The important operands are output_gradient_r: the signed loss derivative used to update a learned parameter; result_row_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 330: `output_gradient_r[result_row_index]` This is continuation line 2 of the 3-line logical statement that begins at source line 329. Its exact fragment is ‘output_gradient_r[result_row_index]‘. The important operands are output_gradient_r: the signed loss derivative used to update a learned parameter; result_row_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes multiplier_left[result_lane][reduction_lane], the value named ’multiplier left’ in this block, equal to ‘output_gradient_r[result_row_inde [reduction_index[reduction_lane][2:0]]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_gradient_r: the signed loss derivative used to update a learned parameter; result_row_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 331: `[reduction_index[reduction_lane][2:0]];` This is continuation line 3 of the 3-line logical statement that begins at source line 329. Its exact fragment is ‘[reduction_index[reduction_lane][2:0]];‘. The important operands are reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes multiplier_left[result_lane][reduction_lane], the value named ’multiplier left’ in this block, equal to ‘output_gradient_r[result_row_inde [reduction_index[reduction_lane][2:0]]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_gradient_r: the signed loss derivative used to update a learned parameter; result_row_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 332: `multiplier_right[result_lane][reduction_lane] =` This statement continues through source line 334; the explanation uses the complete expression rather than only the first visual fragment. This makes multiplier_right[result_lane][redu the value named ’multiplier right’ in this block, equal to ‘weight_r[result_column_index[result_lane][2:0]] [reduction_index[reductio It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_r: a learned coefficient multiplied by an activation; result_column_index: an integer selecting one tensor element, lane,

bank, or state entry; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 333: `weight_r[result_column_index[result_lane][2:0]]` This is continuation line 2 of the 3-line logical statement that begins at source line 332. Its exact fragment is 'weight_r[result_column_index[result_lane][2:0]]'. The important operands are weight_r: a learned coefficient multiplied by an activation; result_column_index: an integer selecting one tensor element, lane, bank, or state entry; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes multiplier_right[result_lane][reduction_lane], the value named 'multiplier right' in this block, equal to 'weight_r[result_column_index[result_lane][2:0]] [reduction_index[reduction_lane][2
It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_r: a learned coefficient multiplied by an activation; result_column_index: an integer selecting one tensor element, lane, bank, or state entry; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 334: `[reduction_index[reduction_lane][2:0]];` This is continuation line 3 of the 3-line logical statement that begins at source line 332. Its exact fragment is '[reduction_index[reduction_lane][2:0]];'. The important operands are reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes multiplier_right[result_lane][reduction_lane], the value named 'multiplier right' in this block, equal to 'weight_r[result_column_index[re
[reduction_index[reduction_lane][2:0]]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_r: a learned coefficient multiplied by an activation; result_column_index: an integer selecting one tensor element, lane, bank, or state entry; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 335: `end else begin` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 336: `multiplier_left[result_lane][reduction_lane] =` This statement continues through source line 338; the explanation uses the complete expression rather than only the first visual fragment. This makes multiplier_left[result_lane][reduc
the value named 'multiplier left' in this block, equal to 'activation_r[reduction_index[reduction_lane][2:0]] [result_row_index]'.
It is driven by combinational or procedural logic in the current evaluation. The important operands are activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 337: `activation_r[reduction_index[reduction_lane][2:0]]` This is continuation line 2 of the 3-line logical statement that begins at source line 336. Its exact fragment is 'activation_r[reduction_index[reduction_lane][2:0]]'. The important operands are activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes multiplier_left[result_lane][reduction_lane], the value named 'multiplier left' in this block, equal to 'activation_r[reduction_index[reducti
[result_row_index]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 338: `[result_row_index];` This is continuation line 3 of the 3-line logical statement that begins at source line 336. Its exact fragment is '[result_row_index];'. The important operands are result_row_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes multiplier_left[result_lane][reduction_lane], the value named 'multiplier left' in this block, equal to 'activation_r[reduction_index[reducti
[result_row_index]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 339: `multiplier_right[result_lane][reduction_lane] =` This statement continues through source line 341; the explanation uses the complete expression rather than only the first visual fragment. This makes multiplier_right[result_lane][redu the value named 'multiplier right' in this block, equal to 'output_gradient_r[reduction_index[reduction_lane][2:0]] [result_column_index[result_lane][2:0]]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_gradient_r: the signed loss derivative used to update a learned parameter; reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 340: `output_gradient_r[reduction_index[reduction_lane][2:0]]` This is continuation line 2 of the 3-line logical statement that begins at source line 339. Its exact fragment is 'output_gradient_r[reduction_index[reduction_lane][2:0]]'. The important operands are output_gradient_r: the signed loss derivative used to update a learned parameter; reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes multiplier_right[result_lane][reduction_lane], the value named 'multiplier right' in this block, equal to 'output_gradient_r[reduction_ind [result_column_index[result_lane][2:0]]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_gradient_r: the signed loss derivative used to update a learned parameter; reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 341: `[result_column_index[result_lane][2:0]];` This is continuation line 3 of the 3-line logical statement that begins at source line 339. Its exact fragment is '[result_column_index[result_lane][2:0]];'. The important operands are result_column_index: an integer selecting one tensor element, lane, bank, or state entry; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes multiplier_right[result_lane][reduction_lane], the value named 'multiplier right' in this block, equal to 'output_gradient_r[reduction_index[reduction_lane][2:0]] [result_column_index[result_lane][2:0]]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_gradient_r: the signed loss derivative used to update a learned parameter; reduction_index: an integer selecting one tensor element, lane, bank, or state entry; reduction_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 342: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 343: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 344: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 345: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 347: `packed_weight_gradient = '0;` This makes packed_weight_gradient, a learned coefficient multiplied by an activation, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 348: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 349: `packed_weight_gradient[lane*24 +: 24] =` This statement continues through source line 350; the explanation uses the complete expression rather than only the first visual fragment. Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are packed_weight_gradient: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; result_row_buffer_r: local storage that decouples producer and consumer timing.

Line 350: `result_row_buffer_r[lane][23:0];` This is continuation line 2 of the 2-line logical statement that begins at source line 349. Its exact fragment is 'result_row_buffer_r[lane][23:0];'. The important operands are result_row_buffer_r: local storage that decouples producer and consumer timing; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are packed_weight_gradient: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; result_row_buffer_r: local storage that decouples producer and consumer timing.

Line 351: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 352: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 354: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 355: `start_ready = state == ST_IDLE;` This makes start_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_IDLE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 356: `busy = state != ST_IDLE;` This makes busy, the value named 'busy' in this block, equal to 'state != ST_IDLE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 357: `done = state == ST_DONE;` This makes done, the value named 'done' in this block, equal to 'state == ST_DONE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_DONE: the value named 'st done' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 358: `s_axis_tready = input_load_state && input_stage_ready` This statement continues through source line 359; the explanation uses the complete expression rather than only the first visual fragment. This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to 'input_load_state && input_stage_ready && !input_final_buffered'. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_load_state: the finite-state-machine phase retained in flip-flops; input_stage_ready: the consumer's statement that it can accept a payload this cycle; input_final_buffered: local storage that decouples producer and consumer timing. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 359: `&& !input_final_buffered;` This is continuation line 2 of the 2-line logical statement that begins at source line 358. Its exact fragment is '&& !input_final_buffered;'. The important operands are input_final_buffered: local storage that decouples producer and consumer timing. Read together, the complete operation is: This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to 'input_load_state && input_stage_ready && !input_final_buffered'. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_load_state: the finite-state-machine phase retained in flip-flops; input_stage_ready:

the consumer's statement that it can accept a payload this cycle; input_final_buffered: local storage that decouples producer and consumer timing. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 360: `m_axis_tvalid = state == ST_OUTPUT_DX || state == ST_OUTPUT_DW;` This makes m_axis_tvalid, the producer's statement that its current payload is meaningful, equal to ‘state == ST_OUTPUT_DX || state == ST_OUTPUT_DW‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT_DX: the value named 'st output dx' in this block; ST_OUTPUT_DW: the value named 'st output dw' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 361: `m_axis_tdata = 64'd0;` This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to ‘64'd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 362: `m_axis_tlast = 1'b0;` This makes m_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1'b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 363: `// compute_mode_r is fixed before an output row starts. Selecting data` Design intent: compute_mode_r is fixed before an output row starts. Selecting data This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 364: `// with it avoids putting the large FSM state decode in the stream-data` Design intent: with it avoids putting the large FSM state decode in the stream-data This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 365: `// path while preserving the same value whenever tvalid is asserted.` Design intent: path while preserving the same value whenever tvalid is asserted. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 366: `if (compute_mode_r == MODE_INPUT_GRADIENT) begin` This condition selects the following hardware branch only when ‘compute_mode_r == MODE_INPUT_GRADIENT‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are compute_mode_r: the value named 'compute mode r' in this block; MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 367: `m_axis_tdata = {` This statement continues through source line 370; the explanation uses the complete expression rather than only the first visual fragment. This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to ‘{ result_row_buffer_r[output_word_r*2 + 1], result_row_buffer_r[output_word_r*2] }‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_row_buffer_r: local storage that decouples producer and consumer timing; output_word_r: the value named 'output word r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 368: `result_row_buffer_r[output_word_r*2 + 1],` This is continuation line 2 of the 4-line logical statement that begins at source line 367. Its exact fragment is ‘result_row_buffer_r[output_word_r*2 + 1],‘. The important operands are result_row_buffer_r: local storage that decouples producer and consumer timing; output_word_r: the value named 'output word r' in this block. Read together, the complete operation is: This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to ‘{ result_row_buffer_r[output_word_r*2 + 1], result_row_buffer_r[output_word_r*2 }‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_row_buffer_r: local storage that decouples producer and consumer timing; output_word_r: the value named 'output word r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 369: `result_row_buffer_r[output_word_r*2]` This is continuation line 3 of the 4-line logical statement that begins at source line 367. Its exact fragment is 'result_row_buffer_r[output_word_r*2]'. The important operands are result_row_buffer_r: local storage that decouples producer and consumer timing; output_word_r: the value named 'output word r' in this block. Read together, the complete operation is: This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to '{ result_row_buffer_r[output_word_r*2 + 1], result_row_buffer_r[output_word_r*2] }'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_row_buffer_r: local storage that decouples producer and consumer timing; output_word_r: the value named 'output word r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 370: `};` This is continuation line 4 of the 4-line logical statement that begins at source line 367. Its exact fragment is '};'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to '{ result_row_buffer_r[output_word_r*2 + 1], result_row_buffer_r[output_word_r*2] }'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_row_buffer_r: local storage that decouples producer and consumer timing; output_word_r: the value named 'output word r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 371: `m_axis_tlast = mode_r == MODE_INPUT_GRADIENT` This statement continues through source line 372; the explanation uses the complete expression rather than only the first visual fragment. This makes m_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'mode_r == MODE_INPUT_GRADIENT && result_row_r == rows_r - 1'b1 && output_word_r == 3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are mode_r: the value named 'mode r' in this block; MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter; result_row_r: one output-row position processed by the tensor tile. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 372: `&& result_row_r == rows_r - 1'b1 && output_word_r == 3;` This is continuation line 2 of the 2-line logical statement that begins at source line 371. Its exact fragment is '&& result_row_r == rows_r - 1'b1 && output_word_r == 3;'. The important operands are result_row_r: one output-row position processed by the tensor tile; rows_r: one output-row position processed by the tensor tile; b1: the value named 'b1' in this block. Read together, the complete operation is: This makes m_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'mode_r == MODE_INPUT_GRADIENT && result_row_r == rows_r - 1'b1 && output_word_r == 3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are mode_r: the value named 'mode r' in this block; MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter; result_row_r: one output-row position processed by the tensor tile. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 373: `end else begin` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 374: `m_axis_tdata = packed_weight_gradient[output_word_r*64 +: 64];` This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to 'packed_weight_gradient[output_word_r*64 +: 64]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are packed_weight_gradient: a learned coefficient multiplied by an activation; output_word_r: the value named 'output word r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 375: `m_axis_tlast = result_row_r == inputs_r - 1'b1` This statement continues through source line 376; the explanation uses the complete expression rather than only the first visual fragment. This makes m_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'result_row_r == inputs_r - 1'b1 && output_word_r == 2'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_row_r: one output-row position processed by the tensor tile; inputs_r: the value named 'inputs r' in this block; b1: the value named 'b1' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 376: `&& output_word_r == 2;` This is continuation line 2 of the 2-line logical statement that begins at source line 375. Its exact fragment is '&& output_word_r == 2;'. The important operands are output_word_r: the value

named 'output word r' in this block. Read together, the complete operation is: This makes m_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'result_row_r == inputs_r - 1'b1 && output_word_r == 2'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_row_r: one output-row position processed by the tensor tile; inputs_r: the value named 'inputs r' in this block; b1: the value named 'b1' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 377: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 378: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 380: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 381: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 382: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 383: `rows_r <= 4'd0;` This makes rows_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 384: `inputs_r <= 4'd0;` This makes inputs_r, the value named 'inputs r' in this block, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 385: `outputs_r <= 4'd0;` This makes outputs_r, the value named 'outputs r' in this block, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 386: `dw_shift_r <= 5'd0;` This makes dw_shift_r, the value named 'dw shift r' in this block, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 387: `result_shift_r <= 5'd0;` This makes result_shift_r, the value named 'result shift r' in this block, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 388: `rounding_bias_r <= 47'd0;` This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to '47'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 389: `mode_r <= MODE_BOTH;` This makes mode_r, the value named 'mode r' in this block, equal to 'MODE_BOTH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_BOTH: the value named 'mode both' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 390: `compute_mode_r <= MODE_INPUT_GRADIENT;` This makes compute_mode_r, the value named 'compute mode r' in this block, equal to 'MODE_INPUT_GRADIENT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge.

Line 391: `suppress_output_r <= 1'b0;` This makes suppress_output_r, the value named 'suppress output r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 392: `accumulate_result_r <= 1'b0;` This makes accumulate_result_r, the value named 'accumulate result r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 393: `start_configuration_invalid_r <= 1'b0;` This makes start_configuration_invalid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 394: `requested_accumulator_mode_r <= MODE_INPUT_GRADIENT;` This makes requested_accumulator_mode_r, a widened running sum that prevents intermediate overflow, equal to 'MODE_INPUT_GRADIENT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge.

Line 395: `requested_accumulator_rows_r <= 4'd0;` This makes requested_accumulator_rows_r, a widened running sum that prevents intermediate overflow, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 396: `requested_accumulator_columns_r <= 4'd0;` This makes requested_accumulator_columns_r, a widened running sum that prevents intermediate overflow, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 397: `compute_result_row_count_r <= 4'd0;` This makes compute_result_row_count_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 398: `compute_result_column_count_r <= 4'd0;` This makes compute_result_column_count_r, one output-feature position processed in parallel, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 399: `compute_reduction_count_r <= 4'd0;` This makes compute_reduction_count_r, an integer count retained for control or measurable telemetry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 400: `input_gradient_lane_r <= {RESULT_LANES{1'b1}};` This makes input_gradient_lane_r, the signed loss derivative used to update a learned parameter, equal to '{RESULT_LANES{1'b1}}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are RESULT_LANES: one parallel arithmetic element within a vector or tensor datapath; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 401: `load_row_r <= 4'd0;` This makes load_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 402: `load_half_r <= 1'b0;` This makes load_half_r, the value named 'load half r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 403: `load_final_r <= 1'b0;` This makes load_final_r, the value named 'load final r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 404: `input_stage_valid_r <= 1'b0;` This makes input_stage_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 405: `input_stage_data_r <= 64'd0;` This makes input_stage_data_r, the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 406: `input_stage_last_r <= 1'b0;` This makes input_stage_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 407: `load_commit_data_r <= 64'd0;` This makes load_commit_data_r, the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 408: `result_row_r <= 4'd0;` This makes result_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 409: `column_group_r <= 1'b0;` This makes column_group_r, one output-feature position processed in parallel, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 410: `reduction_group_r <= 1'b0;` This makes reduction_group_r, the value named 'reduction group r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 411: `output_word_r <= 2'd0;` This makes output_word_r, the value named 'output word r' in this block, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 412: `accumulator_valid_r <= 1'b0;` This makes accumulator_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 413: `accumulator_mode_r <= MODE_INPUT_GRADIENT;` This makes accumulator_mode_r, a widened running sum that prevents intermediate overflow, equal to 'MODE_INPUT_GRADIENT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge.

Line 414: `accumulator_rows_r <= 4'd0;` This makes accumulator_rows_r, a widened running sum that prevents intermediate overflow, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 415: `accumulator_columns_r <= 4'd0;` This makes accumulator_columns_r, a widened running sum that prevents intermediate overflow, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 416: `pair_update_r <= 1'b0;` This makes pair_update_r, the value named 'pair update r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 417: `config_error <= 1'b0;` This makes config_error, the value named 'config error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 418: `completed_tiles <= 32'd0;` This makes completed_tiles, the value named 'completed tiles' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 419: `active_cycles <= 32'd0;` This makes active_cycles, the value named 'active cycles' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 420: `for (int row = 0; row < LANES; row++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 421: `result_row_buffer_r[row] <= 32'sd0;` This makes result_row_buffer_r[row], local storage that decouples producer and consumer timing, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 422: `activation_write_enable_r[row] <= 2'b00;` This makes activation_write_enable_r[row], an input feature value consumed by a neuron or multiply-accumulate operation, equal to '2'b00'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b00: the value named 'b00' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 423: `weight_write_enable_r[row] <= 2'b00;` This makes weight_write_enable_r[row], a learned coefficient multiplied by an activation, equal to '2'b00'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b00: the value named 'b00' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 424: `output_gradient_write_enable_r[row] <= 2'b00;` This makes output_gradient_write_enable_r[row], the signed loss derivative used to update a learned parameter, equal to '2'b00'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b00: the value named 'b00' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 425: `for (int column = 0; column < LANES; column++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 426: `activation_r[row][column] <= 16'sd0;` This makes activation_r[row][column], an input feature value consumed by a neuron or multiply-accumulate operation, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 427: `weight_r[row][column] <= 16'sd0;` This makes weight_r[row][column], a learned coefficient multiplied by an activation, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 428: `output_gradient_r[row][column] <= 16'sd0;` This makes output_gradient_r[row][column], the signed loss derivative used to update a learned parameter, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 429: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 430: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 431: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 432: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 433: `row_partial_r[result_lane] <= 35'sd0;` This makes row_partial_r[result_lane], one output-row position processed by the tensor tile, equal to '35'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 434: `reduction_sum_r[result_lane] <= 34'sd0;` This makes reduction_sum_r[result_lane], the value named 'reduction sum r' in this block, equal to '34'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 435: `selected_accumulator_r[result_lane] <= 47'sd0;` This makes selected_accumulator_r[result_lane], a widened running sum that prevents intermediate overflow, equal to '47'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 436: `accumulated_wide_r[result_lane] <= 48'sd0;` This makes accumulated_wide_r[result_lane], the value named 'accumulated wide r' in this block, equal to '48'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 437: `commit_lane_valid_r[result_lane] <= 1'b0;` This makes commit_lane_valid_r[result_lane], the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 438: `rounding_value_r[result_lane] <= 47'd0;` This makes rounding_value_r[result_lane], the value named 'rounding value r' in this block, equal to '47'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 439: `rounding_negative_r[result_lane] <= 1'b0;` This makes rounding_negative_r[result_lane], the value named 'rounding negative r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 440: `rounding_carry_r[result_lane] <= 1'b0;` This makes rounding_carry_r[result_lane], the value named 'rounding carry r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 441: `saturated_result_r[result_lane] <= 32'sd0;` This makes saturated_result_r[result_lane], the value named 'saturated result r' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 442: `for (int reduction_lane = 0;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 443: `reduction_lane < REDUCTION_LANES; reduction_lane++) begin` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath; REDUCTION_LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 444: `multiplier_left_r[result_lane][reduction_lane] <= 16'sd0;` This makes multiplier_left_r[result_lane][reduct the value named 'multiplier left r' in this block, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 445: `multiplier_right_r[result_lane][reduction_lane] <= 16'sd0;` This makes multiplier_right_r[result_lane][red the value named 'multiplier right r' in this block, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 446: `multiplier_left_dsp_r[result_lane][reduction_lane] <= 16'sd0;` This makes multiplier_left_dsp_r[result_la the value named 'multiplier left dsp r' in this block, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 447: `multiplier_right_dsp_r[result_lane][reduction_lane] <= 16'sd0;` This makes multiplier_right_dsp_r[result the value named 'multiplier right dsp r' in this block, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 448: `product_r[result_lane][reduction_lane] <= 32'sd0;` This makes product_r[result_lane][reduction_lane], the value named 'product r' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 449: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 450: `pair_sum_r[result_lane][0] <= 33'sd0;` This makes pair_sum_r[result_lane][0], the value named 'pair sum r' in this block, equal to '33'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 451: `pair_sum_r[result_lane][1] <= 33'sd0;` This makes pair_sum_r[result_lane][1], the value named 'pair sum r' in this block, equal to '33'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 452: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 453: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 454: `if (state != ST_IDLE) active_cycles <= active_cycles + 1'b1;` This condition selects the following hardware branch only when 'state != ST_IDLE' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 455: `pair_update_r <= state == ST_PRODUCT;` This makes pair_update_r, the value named 'pair update r' in this block, equal to 'state == ST_PRODUCT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_PRODUCT: the value named 'st product' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 457: `// The single elastic slot is fall-through only at clock edges: an` Design intent: The single elastic slot is fall-through only at clock edges: an This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 458: `// old buffered beat can be consumed while the next stream beat is` Design intent: old buffered beat can be consumed while the next stream beat is This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 459: `// captured. On the final dY beat READY is withheld because there is` Design intent: captured. On the final dY beat READY is withheld because there is This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 460: `// no following operand class to prefetch.` Design intent: no following operand class to prefetch. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 461: `if (input_stage_ready) begin` This condition selects the following hardware branch only when 'input_stage_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_stage_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 462: `input_stage_valid_r <= s_axis_tvalid && s_axis_tready;` This makes input_stage_valid_r, the producer's statement that its current payload is meaningful, equal to 's_axis_tvalid && s_axis_tready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful; s_axis_tready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 463: `if (s_axis_tvalid && s_axis_tready) begin` This condition selects the following hardware branch only when 's_axis_tvalid && s_axis_tready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful; s_axis_tready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 464: `input_stage_data_r <= s_axis_tdata;` This makes input_stage_data_r, the payload carried with the surrounding valid/ready handshake, equal to 's_axis_tdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 465: `input_stage_last_r <= s_axis_tlast;` This makes input_stage_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 's_axis_tlast'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 466: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 467: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 469: `// Apply the previous beat's row-local commit and prepare one-hot` Design intent: Apply the previous beat's row-local commit and prepare one-hot This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 470: `// enables for the beat consumed this cycle. Each enable drives` Design intent: enables for the beat consumed this cycle. Each enable drives This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 471: `// only four 16-bit destinations, replacing the former chip-wide` Design intent: only four 16-bit destinations, replacing the former chip-wide This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 472: `// stream-VALID/state route into all three operand banks.` Design intent: stream-VALID/state route into all three operand banks. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 473: `for (int row = 0; row < LANES; row++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 474: `for (int half = 0; half < 2; half++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 475: `if (activation_write_enable_r[row][half]) begin` This condition selects the following hardware branch only when ‘activation_write_enable_r[row][half]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are activation_write_enable_r: an input feature value consumed by a neuron or multiply-accumulate operation; row: one output-row position processed by the tensor tile; half: the value named ’half’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 476: `for (int lane = 0; lane < 4; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 477: `activation_r[row][half*4 + lane]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; row: one output-row position processed by the tensor tile; half: the value named ’half’ in this block.

Line 478: `<= load_commit_data_r[lane*16 +: 16];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are load_commit_data_r: the payload carried with the surrounding valid/ready handshake; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 479: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 480: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 481: `if (weight_write_enable_r[row][half]) begin` This condition selects the following hardware branch only when ‘weight_write_enable_r[row][half]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are weight_write_enable_r: a learned coefficient multiplied by an activation; row: one output-row position processed by the tensor tile; half: the value named ’half’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 482: `for (int lane = 0; lane < 4; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 483: `weight_r[row][half*4 + lane]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are weight_r: a learned coefficient multiplied by an activation; row: one output-row position processed by the tensor tile; half: the value named 'half' in this block.

Line 484: `<= load_commit_data_r[lane*16 +: 16];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are load_commit_data_r: the payload carried with the surrounding valid/ready handshake; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 485: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 486: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 487: `if (output_gradient_write_enable_r[row][half]) begin` This condition selects the following hardware branch only when ‘output_gradient_write_enable_r[row][half]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_gradient_write_enable_r: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile; half: the value named 'half' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 488: `for (int lane = 0; lane < 4; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 489: `output_gradient_r[row][half*4 + lane]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are output_gradient_r: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile; half: the value named 'half' in this block.

Line 490: `<= load_commit_data_r[lane*16 +: 16];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are load_commit_data_r: the payload carried with the surrounding valid/ready handshake; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 491: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 492: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 493: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 494: `activation_write_enable_r[row] <= 2'b00;` This makes activation_write_enable_r[row], an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘2'b00‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b00: the value named 'b00' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 495: `weight_write_enable_r[row] <= 2'b00;` This makes weight_write_enable_r[row], a learned coefficient multiplied by an activation, equal to '2'b00'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b00: the value named 'b00' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 496: `output_gradient_write_enable_r[row] <= 2'b00;` This makes output_gradient_write_enable_r[row], the signed loss derivative used to update a learned parameter, equal to '2'b00'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b00: the value named 'b00' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 497: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 498: `if (input_stage_valid_r && input_load_state) begin` This condition selects the following hardware branch only when 'input_stage_valid_r && input_load_state' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_stage_valid_r: the producer's statement that its current payload is meaningful; input_load_state: the finite-state-machine phase retained in flip-flops. Context: the clocked process that updates registers only on a rising clock edge.

Line 499: `load_commit_data_r <= input_stage_data_r;` This makes load_commit_data_r, the payload carried with the surrounding valid/ready handshake, equal to 'input_stage_data_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_stage_data_r: the payload carried with the surrounding valid/ready handshake. Context: the clocked process that updates registers only on a rising clock edge.

Line 500: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 501: `ST_LOAD_X: activation_write_enable_r[load_row_index]` This is the ST_LOAD_X case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'activation_write_enable_r[load_row_index context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 502: `[load_half_r] <= 1'b1;` Within the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are load_half_r: the value named 'load half r' in this block; b1: the value named 'b1' in this block.

Line 503: `ST_LOAD_W: weight_write_enable_r[load_row_index]` This is the ST_LOAD_W case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'weight_write_enable_r[load_row_index]'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 504: `[load_half_r] <= 1'b1;` Within the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are load_half_r: the value named 'load half r' in this block; b1: the value named 'b1' in this block.

Line 505: `ST_LOAD_DY: output_gradient_write_enable_r[load_row_index]` This is the ST_LOAD_DY case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'output_gradient_write_enable_r[loac context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 506: `[load_half_r] <= 1'b1;` Within the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are load_half_r: the value named 'load half r' in this block; b1: the value named 'b1' in this block.

Line 507: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 508: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 509: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 511: `if (pair_update_r) begin` This condition selects the following hardware branch only when 'pair_update_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pair_update_r: the value named 'pair update r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 512: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 513: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 514: `pair_sum_r[result_lane][0] <=` This statement continues through source line 516; the explanation uses the complete expression rather than only the first visual fragment. This makes pair_sum_r[result_lane][0], the value named 'pair sum r' in this block, equal to '{product_r[result_lane][0][31], product_r[result_lane][0]} + {product_r[result_lane][1][31], product_r[result_lane][1]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge.

Line 515: `{product_r[result_lane][0][31], product_r[result_lane][0]}` This is continuation line 2 of the 3-line logical statement that begins at source line 514. Its exact fragment is '{product_r[result_lane][0][31], product_r[result_lane][0]}'. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes pair_sum_r[result_lane][0], the value named 'pair sum r' in this block, equal to '{product_r[result_lane][0][31], product_r[result_lane][0]} + {product_r[result_lane][1][31], product_r[result_lane][1]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge.

Line 516: `+ {product_r[result_lane][1][31], product_r[result_lane][1]};` This is continuation line 3 of the 3-line logical statement that begins at source line 514. Its exact fragment is '+ {product_r[result_lane][1][31], product_r[result_lane][1]};'. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes

pair_sum_r[result_lane][0], the value named 'pair sum r' in this block, equal to '{product_r[result_lane][0][31], product_r[result_lane][0]} + {product_r[result_lane][1][31], product_r[result_lane][1]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge.

Line 517: `pair_sum_r[result_lane][1] <=` This statement continues through source line 519; the explanation uses the complete expression rather than only the first visual fragment. This makes pair_sum_r[result_lane][1], the value named 'pair sum r' in this block, equal to '{product_r[result_lane][2][31], product_r[result_lane][2]} + {product_r[result_lane][3][31], product_r[result_lane][3]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge.

Line 518: `{product_r[result_lane][2][31], product_r[result_lane][2]}` This is continuation line 2 of the 3-line logical statement that begins at source line 517. Its exact fragment is '{product_r[result_lane][2][31], product_r[result_lane][2]}'. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes pair_sum_r[result_lane][1], the value named 'pair sum r' in this block, equal to '{product_r[result_lane][2][31], product_r[result_lane][2]} + {product_r[result_lane][3][31], product_r[result_lane][3]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge.

Line 519: `+ {product_r[result_lane][3][31], product_r[result_lane][3]};` This is continuation line 3 of the 3-line logical statement that begins at source line 517. Its exact fragment is '+ {product_r[result_lane][3][31], product_r[result_lane][3]};'. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes pair_sum_r[result_lane][1], the value named 'pair sum r' in this block, equal to '{product_r[result_lane][2][31], product_r[result_lane][2]} + {product_r[result_lane][3][31], product_r[result_lane][3]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge.

Line 520: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 521: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 522: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 523: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 524: `if (start_valid) begin` This condition selects the following hardware branch only when 'start_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are start_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 525: `rows_r <= active_rows;` This makes rows_r, one output-row position processed by the tensor tile, equal to ‘active_rows‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_rows: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 526: `inputs_r <= active_inputs;` This makes inputs_r, the value named ’inputs r’ in this block, equal to ‘active_inputs‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_inputs: the value named ’active inputs’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 527: `outputs_r <= active_outputs;` This makes outputs_r, the value named ’outputs r’ in this block, equal to ‘active_outputs‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_outputs: the value named ’active outputs’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 528: `dw_shift_r <= weight_gradient_shift;` This makes dw_shift_r, the value named ’dw shift r’ in this block, equal to ‘weight_gradient_shift‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 529: `if (gradient_mode == MODE_WEIGHT_GRADIENT) begin` This condition selects the following hardware branch only when ‘gradient_mode == MODE_WEIGHT_GRADIENT‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are gradient_mode: the signed loss derivative used to update a learned parameter; MODE_WEIGHT_GRADIENT: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 530: `result_shift_r <= weight_gradient_shift;` This makes result_shift_r, the value named ’result shift r’ in this block, equal to ‘weight_gradient_shift‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 531: `rounding_bias_r <= round_bias(weight_gradient_shift);` This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to ‘round_bias(weight_gradient_shift)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are round_bias: an additive learned offset applied after accumulation; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 532: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 533: `result_shift_r <= input_gradient_shift;` This makes result_shift_r, the value named ’result shift r’ in this block, equal to ‘input_gradient_shift‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 534: `rounding_bias_r <= round_bias(input_gradient_shift);` This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to ‘round_bias(input_gradient_shift)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are round_bias: an additive learned offset applied after accumulation; input_gradient_shift: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 535: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 536: `mode_r <= gradient_mode;` This makes mode_r, the value named ’mode r’ in this block, equal to ‘gradient_mode‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are gradient_mode: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 537: `compute_mode_r <= requested_accumulator_mode;` This makes compute_mode_r, the value named ’compute mode r’ in this block, equal to ‘requested_accumulator_mode‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are requested_accumulator_mode: a widened running sum that prevents intermediate overflow. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 538: `suppress_output_r <= suppress_output;` This makes suppress_output_r, the value named ’suppress output r’ in this block, equal to ‘suppress_output‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are suppress_output: the value named ’suppress output’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 539: `accumulate_result_r <= accumulate_result;` This makes accumulate_result_r, the value named ’accumulate result r’ in this block, equal to ‘accumulate_result‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are accumulate_result: the value named ’accumulate result’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 540: `start_configuration_invalid_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are start_configuration_invalid_r: the producer’s statement that its current payload is meaningful.

Line 541: `<= start_configuration_invalid;` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are start_configuration_invalid: the producer’s statement that its current payload is meaningful.

Line 542: `requested_accumulator_mode_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are requested_accumulator_mode_r: a widened running sum that prevents intermediate overflow.

Line 543: `<= requested_accumulator_mode;` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are requested_accumulator_mode: a widened running sum that prevents intermediate overflow.

Line 544: `requested_accumulator_rows_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are requested_accumulator_rows_r: a widened running sum that prevents intermediate overflow.

Line 545: `<= requested_accumulator_rows;` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are requested_accumulator_rows: a widened running sum that prevents intermediate overflow.

Line 546: `requested_accumulator_columns_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are requested_accumulator_columns_r: a widened running sum that prevents intermediate overflow.

Line 547: `<= requested_accumulator_columns;` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are requested_accumulator_columns: a widened running sum that prevents intermediate overflow.

Line 548: `input_gradient_lane_r <= {RESULT_LANES{` This statement continues through source line 550; the explanation uses the complete expression rather than only the first visual fragment. This makes input_gradient_lane_r, the signed loss derivative used to update a learned parameter, equal to ‘{RESULT_LANES{ requested_accumulator_mode == MODE_INPUT_GRADIENT}}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are RESULT_LANES: one parallel arithmetic element within a vector or tensor datapath; requested_accumulator_mode: a widened running sum that prevents intermediate overflow; MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 549: `requested_accumulator_mode` This is continuation line 2 of the 3-line logical statement that begins at source line 548. Its exact fragment is ‘requested_accumulator_mode‘. The important operands are requested_accumulator_mode: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This makes input_gradient_lane_r, the signed loss derivative used to update a learned parameter, equal to ‘{RESULT_LANES{ requested_accumulator_mode == MODE_INPUT_GRADIENT}}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are RESULT_LANES: one parallel arithmetic element within a vector or tensor datapath; requested_accumulator_mode: a widened running sum that prevents intermediate overflow; MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 550: `== MODE_INPUT_GRADIENT}};` This is continuation line 3 of the 3-line logical statement that begins at source line 548. Its exact fragment is ‘== MODE_INPUT_GRADIENT}};‘. The important operands are MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This makes input_gradient_lane_r, the signed loss derivative used to update a learned parameter, equal to ‘{RESULT_LANES{ requested_accumulator_mode == MODE_INPUT_GRADIENT}}‘. It updates after the active clock edge together

with other nonblocking assignments. The important operands are RESULT_LANES: one parallel arithmetic element within a vector or tensor datapath; requested_accumulator_mode: a widened running sum that prevents intermediate overflow; MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 551: `if (requested_accumulator_mode` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are requested_accumulator_mode: a widened running sum that prevents intermediate overflow.

Line 552: `== MODE_INPUT_GRADIENT) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter.

Line 553: `compute_result_row_count_r <= active_rows;` This makes compute_result_row_count_r, one output-row position processed by the tensor tile, equal to 'active_rows'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_rows: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 554: `compute_result_column_count_r <= active_inputs;` This makes compute_result_column_count_r, one output-feature position processed in parallel, equal to 'active_inputs'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_inputs: the value named 'active inputs' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 555: `compute_reduction_count_r <= active_outputs;` This makes compute_reduction_count_r, an integer count retained for control or measurable telemetry, equal to 'active_outputs'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_outputs: the value named 'active outputs' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 556: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 557: `compute_result_row_count_r <= active_inputs;` This makes compute_result_row_count_r, one output-row position processed by the tensor tile, equal to 'active_inputs'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_inputs: the value named 'active inputs' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 558: `compute_result_column_count_r <= active_outputs;` This makes compute_result_column_count_r, one output-feature position processed in parallel, equal to 'active_outputs'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_outputs: the value named 'active outputs' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 559: `compute_reduction_count_r <= active_rows;` This makes compute_reduction_count_r, an integer count retained for control or measurable telemetry, equal to 'active_rows'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_rows: one output-row position processed by the

tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 560: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 561: `load_row_r <= 4'd0;` This makes load_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 562: `load_half_r <= 1'b0;` This makes load_half_r, the value named 'load half r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 563: `result_row_r <= 4'd0;` This makes result_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 564: `column_group_r <= 1'b0;` This makes column_group_r, one output-feature position processed in parallel, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 565: `reduction_group_r <= 1'b0;` This makes reduction_group_r, the value named 'reduction group r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 566: `output_word_r <= 2'd0;` This makes output_word_r, the value named 'output word r' in this block, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 567: `config_error <= 1'b0;` This makes config_error, the value named 'config error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 568: `for (int row = 0; row < LANES; row++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 569: `result_row_buffer_r[row] <= 32'sd0;` This makes result_row_buffer_r[row], local storage that decouples producer and consumer timing, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 570: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 571: `state <= ST_CONFIGURE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_CONFIGURE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_CONFIGURE: the value named ’st configure’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 572: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 573: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 575: `ST_CONFIGURE: begin` This is the ST_CONFIGURE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 576: `config_error <= start_configuration_invalid_r;` This makes config_error, the value named ’config error’ in this block, equal to ‘start_configuration_invalid_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are start_configuration_invalid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 577: `if (start_configuration_invalid_r) begin` This condition selects the following hardware branch only when ‘start_configuration_invalid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are start_configuration_invalid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 578: `state <= ST_DONE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_DONE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DONE: the value named ’st done’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 579: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 580: `accumulator_valid_r <= 1’b1;` This makes accumulator_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 581: `accumulator_mode_r <= requested_accumulator_mode_r;` This makes accumulator_mode_r, a widened running sum that prevents intermediate overflow, equal to ‘requested_accumulator_mode_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are requested_accumulator_mode_r: a widened running sum that prevents intermediate overflow. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 582: `accumulator_rows_r <= requested_accumulator_rows_r;` This makes accumulator_rows_r, a widened running sum that prevents intermediate overflow, equal to ‘requested_accumulator_rows_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are requested_accumulator_rows_r: a widened running sum that prevents intermediate overflow. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 583: `accumulator_columns_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are accumulator_columns_r: a widened running sum that prevents intermediate overflow.

Line 584: `<= requested_accumulator_columns_r;` Within the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are requested_accumulator_columns_r: a widened running sum that prevents intermediate overflow.

Line 585: `if (mode_r == MODE_INPUT_GRADIENT) begin` This condition selects the following hardware branch only when ‘mode_r == MODE_INPUT_GRADIENT‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are mode_r: the value named ’mode r’ in this block; MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 586: `load_final_r <= inputs_r == 4'd1;` This makes load_final_r, the value named ’load final r’ in this block, equal to ‘inputs_r == 4’d1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are inputs_r: the value named ’inputs r’ in this block; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 587: `state <= ST_LOAD_W;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_W‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_W: the value named ’st load w’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 588: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 589: `load_final_r <= rows_r == 4'd1;` This makes load_final_r, the value named ’load final r’ in this block, equal to ‘rows_r == 4’d1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are rows_r: one output-row position processed by the tensor tile; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 590: `state <= ST_LOAD_X;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_X‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_X: the value named ’st load x’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase..

Line 591: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 592: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 593: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_CONFIGURE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 595: `ST_LOAD_X: if (input_stage_valid_r) begin` This is the ST_LOAD_X case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘if (input_stage_valid_r) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 596: `if (input_stage_last_r) config_error <= 1'b1;` This condition selects the following hardware branch only when ‘input_stage_last_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_stage_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 597: `if (!load_half_r) load_half_r <= 1'b1;` This condition selects the following hardware branch only when ‘!load_half_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_half_r: the value named 'load half r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 598: `else if (load_final_r) begin` This condition selects the following hardware branch only when ‘load_final_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_final_r: the value named 'load final r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 599: `load_row_r <= 4'd0;` This makes load_row_r, one output-row position processed by the tensor tile, equal to ‘4'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 600: `load_half_r <= 1'b0;` This makes load_half_r, the value named 'load half r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 601: `if (mode_r == MODE_WEIGHT_GRADIENT) begin` This condition selects the following hardware branch only when ‘mode_r == MODE_WEIGHT_GRADIENT‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are mode_r: the value named 'mode r' in this block; MODE_WEIGHT_GRADIENT: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 602: `load_final_r <= rows_r == 4'd1;` This makes load_final_r, the value named 'load final r' in this block, equal to ‘rows_r == 4'd1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are rows_r: one output-row position processed by the tensor tile; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 603: `state <= ST_LOAD_DY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_DY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_DY: the value named ’st load dy’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 604: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 605: `load_final_r <= inputs_r == 4'd1;` This makes load_final_r, the value named ’load final r’ in this block, equal to ‘inputs_r == 4’d1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are inputs_r: the value named ’inputs r’ in this block; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 606: `state <= ST_LOAD_W;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_W‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_W: the value named ’st load w’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 607: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 608: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 609: `load_row_r <= load_row_r + 1'b1;` This makes load_row_r, one output-row position processed by the tensor tile, equal to ‘load_row_r + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_row_r: one output-row position processed by the tensor tile; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 610: `load_half_r <= 1'b0;` This makes load_half_r, the value named ’load half r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 611: `load_final_r <= load_row_r == rows_r - 4'd2;` This makes load_final_r, the value named ’load final r’ in this block, equal to ‘load_row_r == rows_r - 4’d2‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_row_r: one output-row position processed by the tensor tile; rows_r: one output-row position processed by the tensor tile; d2: the value named ’d2’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory..

Line 612: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 613: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_X, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 615: `ST_LOAD_W: if (input_stage_valid_r) begin` This is the ST_LOAD_W case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘if (input_stage_valid_r) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 616: `if (input_stage_last_r) config_error <= 1'b1;` This condition selects the following hardware branch only when ‘input_stage_last_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_stage_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 617: `if (!load_half_r) load_half_r <= 1'b1;` This condition selects the following hardware branch only when ‘!load_half_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_half_r: the value named 'load half r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 618: `else if (load_final_r) begin` This condition selects the following hardware branch only when ‘load_final_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_final_r: the value named 'load final r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 619: `load_row_r <= 4'd0;` This makes load_row_r, one output-row position processed by the tensor tile, equal to ‘4'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 620: `load_half_r <= 1'b0;` This makes load_half_r, the value named 'load half r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 621: `load_final_r <= rows_r == 4'd1;` This makes load_final_r, the value named 'load final r' in this block, equal to ‘rows_r == 4'd1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are rows_r: one output-row position processed by the tensor tile; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 622: `state <= ST_LOAD_DY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LOAD_DY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_DY: the value named 'st load dy' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 623: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 624: `load_row_r <= load_row_r + 1'b1;` This makes load_row_r, one output-row position processed by the tensor tile, equal to ‘load_row_r + 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_row_r: one output-row position processed by the tensor tile; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 625: `load_half_r <= 1'b0;` This makes load_half_r, the value named 'load half r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 626: `load_final_r <= load_row_r == inputs_r - 4'd2;` This makes load_final_r, the value named 'load final r' in this block, equal to 'load_row_r == inputs_r - 4'd2'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_row_r: one output-row position processed by the tensor tile; inputs_r: the value named 'inputs r' in this block; d2: the value named 'd2' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory..

Line 627: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 628: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_W, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 630: `ST_LOAD_DY: if (input_stage_valid_r) begin` This is the ST_LOAD_DY case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'if (input_stage_valid_r) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 631: `if (input_stage_last_r != (load_half_r && load_final_r)) begin` This condition selects the following hardware branch only when 'input_stage_last_r != (load_half_r && load_final_r)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_stage_last_r: the AXI-Stream marker for the final beat of a packet or command payload; load_half_r: the value named 'load half r' in this block; load_final_r: the value named 'load final r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 632: `config_error <= 1'b1;` This makes config_error, the value named 'config error' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 633: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 634: `if (!load_half_r) load_half_r <= 1'b1;` This condition selects the following hardware branch only when '!load_half_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_half_r: the value named 'load half r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 635: `else if (load_final_r) begin` This condition selects the following hardware branch only when 'load_final_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_final_r: the value named 'load final r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 636: `result_row_r <= 4'd0;` This makes result_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 637: `column_group_r <= 1'b0;` This makes column_group_r, one output-feature position processed in parallel, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 638: `reduction_group_r <= 1'b0;` This makes reduction_group_r, the value named 'reduction group r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 639: `state <= ST_LOAD_COMMIT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LOAD_COMMIT_DRAIN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_COMMIT_DRAIN: the value named 'st load commit drain' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 640: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 641: `load_row_r <= load_row_r + 1'b1;` This makes load_row_r, one output-row position processed by the tensor tile, equal to 'load_row_r + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_row_r: one output-row position processed by the tensor tile; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 642: `load_half_r <= 1'b0;` This makes load_half_r, the value named 'load half r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 643: `load_final_r <= load_row_r == rows_r - 4'd2;` This makes load_final_r, the value named 'load final r' in this block, equal to 'load_row_r == rows_r - 4'd2'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_row_r: one output-row position processed by the tensor tile; rows_r: one output-row position processed by the tensor tile; d2: the value named 'd2' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory..

Line 644: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 645: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_DY, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 647: `// The final dY payload has reached the commit registers. One` Design intent: The final dY payload has reached the commit registers. One This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 648: `// dedicated cycle lets its row-local write land before the DSP` Design intent: dedicated cycle lets its row-local write land before the DSP This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 649: `// operand-selection logic is allowed to read that bank.` Design intent: operand-selection logic is allowed to read that bank. This comment anchors the surrounding logic in the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

Line 650: `ST_LOAD_COMMIT_DRAIN: state <= ST_MULTIPLY;` This is the ST_LOAD_COMMIT_DRAIN case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'state <= ST_MULTIPLY;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_COMMIT_DRAIN, the machine stores incoming operands into local FPGA memory..

Line 652: `ST_MULTIPLY: begin` This is the ST_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 653: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 654: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 655: `for (int reduction_lane = 0;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 656: `reduction_lane < REDUCTION_LANES;` Within the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath; REDUCTION_LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 657: `reduction_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 658: `multiplier_left_r[result_lane][reduction_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_left_r: the value named 'multiplier left r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 659: `<= multiplier_left[result_lane][reduction_lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_left: the value named 'multiplier left' in this

block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 660: `multiplier_right_r[result_lane][reduction_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_right_r: the value named 'multiplier right r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 661: `<= multiplier_right[result_lane][reduction_lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_right: the value named 'multiplier right' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 662: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 663: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 664: `state <= ST_REGISTER_OPERANDS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_REGISTER_OPERANDS'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_REGISTER_OPERANDS: the value named 'st register operands' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 665: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 667: `ST_REGISTER_OPERANDS: begin` This is the ST_REGISTER_OPERANDS case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase..

Line 668: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase..

Line 669: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 670: `for (int reduction_lane = 0;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase..

Line 671: `reduction_lane < REDUCTION_LANES;` Within the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath; REDUCTION_LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 672: `reduction_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 673: `multiplier_left_dsp_r[result_lane][reduction_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_left_dsp_r: the value named 'multiplier left dsp r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 674: `<= multiplier_left_r[result_lane][reduction_lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_left_r: the value named 'multiplier left r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 675: `multiplier_right_dsp_r[result_lane][reduction_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_right_dsp_r: the value named 'multiplier right dsp r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 676: `<= multiplier_right_r[result_lane][reduction_lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_right_r: the value named 'multiplier right r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 677: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 678: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 679: `state <= ST_PRODUCT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_PRODUCT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PRODUCT: the value named 'st product' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase..

Line 680: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_REGISTER_OPERANDS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 682: `ST_PRODUCT: begin` This is the ST_PRODUCT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase..

Line 683: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase..

Line 684: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 685: `for (int reduction_lane = 0;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase..

Line 686: `reduction_lane < REDUCTION_LANES;` Within the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath; REDUCTION_LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 687: `reduction_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 688: `product_r[result_lane][reduction_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are product_r: the value named 'product r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 689: `<= multiplier_product[result_lane][reduction_lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are multiplier_product: the value named 'multiplier product' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 690: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 691: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 692: `state <= ST_PAIR;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_PAIR‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PAIR: the value named ’st pair’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase..

Line 693: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 695: `ST_PAIR: begin` This is the ST_PAIR case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_PAIR, the machine performs the named control phase..

Line 696: `state <= ST_REDUCTION_SUM;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_REDUCTION_SUM‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_REDUCTION_SUM: the value named ’st reduction sum’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PAIR, the machine performs the named control phase..

Line 697: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PAIR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 699: `ST_REDUCTION_SUM: begin` This is the ST_REDUCTION_SUM case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase..

Line 700: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase..

Line 701: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 702: `reduction_sum_r[result_lane] <=` This statement continues through source line 706; the explanation uses the complete expression rather than only the first visual fragment. This makes reduction_sum_r[result_lane], the value named ’reduction sum r’ in this block, equal to ‘{pair_sum_r[result_lane][0][32], pair_sum_r[result_lane][0]} + {pair_sum_r[result_lane][1][32], pair_sum_r[result_lane][1]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are pair_sum_r: the value named ’pair sum r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase..

Line 703: `{pair_sum_r[result_lane][0][32],` This is continuation line 2 of the 5-line logical statement that begins at source line 702. Its exact fragment is ‘{pair_sum_r[result_lane][0][32],‘. The important operands are pair_sum_r: the value named ’pair sum r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes reduction_sum_r[result_lane], the value named ’reduction sum r’ in this block, equal to ‘{pair_sum_r[result_lane][0][32], pair_sum_r[result_lane][0]} + {pair_sum_r[result_lane][1][32], pair_sum_r[result_lane][1]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are pair_sum_r: the value named ’pair sum r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase..

Line 704: `pair_sum_r[result_lane][0]}` This is continuation line 3 of the 5-line logical statement that begins at source line 702. Its exact fragment is 'pair_sum_r[result_lane][0]}'. The important operands are pair_sum_r: the value named 'pair sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes reduction_sum_r[result_lane], the value named 'reduction sum r' in this block, equal to '{pair_sum_r[result_lane][0][32], pair_sum_r[result_lane][0]} + {pair_sum_r[result_lane][1][32], pair_sum_r[result_lane][1]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are pair_sum_r: the value named 'pair sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase..

Line 705: `+ {pair_sum_r[result_lane][1][32],` This is continuation line 4 of the 5-line logical statement that begins at source line 702. Its exact fragment is '+ {pair_sum_r[result_lane][1][32],'. The important operands are pair_sum_r: the value named 'pair sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes reduction_sum_r[result_lane], the value named 'reduction sum r' in this block, equal to '{pair_sum_r[result_lane][0][32], pair_sum_r[result_lane][0]} + {pair_sum_r[result_lane][1][32], pair_sum_r[result_lane][1]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are pair_sum_r: the value named 'pair sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase..

Line 706: `pair_sum_r[result_lane][1]};` This is continuation line 5 of the 5-line logical statement that begins at source line 702. Its exact fragment is 'pair_sum_r[result_lane][1]};'. The important operands are pair_sum_r: the value named 'pair sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes reduction_sum_r[result_lane], the value named 'reduction sum r' in this block, equal to '{pair_sum_r[result_lane][0][32], pair_sum_r[result_lane][0]} + {pair_sum_r[result_lane][1][32], pair_sum_r[result_lane][1]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are pair_sum_r: the value named 'pair sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase..

Line 707: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 708: `state <= ST_REDUCE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_REDUCE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_REDUCE: the value named 'st reduce' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase..

Line 709: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_REDUCTION_SUM, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 711: `ST_REDUCE: begin` This is the ST_REDUCE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase..

Line 712: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase..

Line 713: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 714: `if (!reduction_group_r) begin` This condition selects the following hardware branch only when '!reduction_group_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are reduction_group_r: the value named 'reduction group r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase..

Line 715: `row_partial_r[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are row_partial_r: one output-row position processed by the tensor tile; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 716: `<= {reduction_sum_r[result_lane][33],` Within the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_sum_r: the value named 'reduction sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 717: `reduction_sum_r[result_lane]};` Within the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are reduction_sum_r: the value named 'reduction sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 718: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 719: `row_partial_r[result_lane] <=` This statement continues through source line 722; the explanation uses the complete expression rather than only the first visual fragment. This makes row_partial_r[result_lane], one output-row position processed by the tensor tile, equal to 'row_partial_r[result_lane] + {reduction_sum_r[result_lane][33], reduction_sum_r[result_lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are row_partial_r: one output-row position processed by the tensor tile; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_sum_r: the value named 'reduction sum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase..

Line 720: `row_partial_r[result_lane]` This is continuation line 2 of the 4-line logical statement that begins at source line 719. Its exact fragment is 'row_partial_r[result_lane]'. The important operands are row_partial_r: one output-row position processed by the tensor tile; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes row_partial_r[result_lane], one output-row position processed by the tensor tile, equal to 'row_partial_r[result_lane] + {reduction_sum_r[result_lane][33], reduction_sum_r[result_lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are row_partial_r: one output-row position processed by the tensor tile; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_sum_r: the value named 'reduction sum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase..

Line 721: `+ {reduction_sum_r[result_lane][33],` This is continuation line 3 of the 4-line logical statement that begins at source line 719. Its exact fragment is '+ {reduction_sum_r[result_lane][33],'. The important operands are

reduction_sum_r: the value named 'reduction sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes row_partial_r[result_lane], one output-row position processed by the tensor tile, equal to ‘row_partial_r[result_lane] + {reduction_sum_r[result_lane][33], reduction_sum_r[result_lane]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are row_partial_r: one output-row position processed by the tensor tile; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_sum_r: the value named 'reduction sum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase..

Line 722: `reduction_sum_r[result_lane]};` This is continuation line 4 of the 4-line logical statement that begins at source line 719. Its exact fragment is ‘reduction_sum_r[result_lane]};‘. The important operands are reduction_sum_r: the value named 'reduction sum r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes row_partial_r[result_lane], one output-row position processed by the tensor tile, equal to ‘row_partial_r[result_lane] + {reduction_sum_r[result_lane][33], reduction_sum_r[result_lane]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are row_partial_r: one output-row position processed by the tensor tile; result_lane: one parallel arithmetic element within a vector or tensor datapath; reduction_sum_r: the value named 'reduction sum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase..

Line 723: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 724: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 725: `state <= ST_COMMIT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_COMMIT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMMIT: the value named 'st commit' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase..

Line 726: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_REDUCE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 728: `ST_COMMIT: begin` This is the ST_COMMIT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 729: `if (!reduction_group_r && reduction_count > 4'd4) begin` This condition selects the following hardware branch only when ‘!reduction_group_r && reduction_count > 4'd4‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are reduction_group_r: the value named 'reduction group r' in this block; reduction_count: an integer count retained for control or measurable telemetry; d4: the value named 'd4' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 730: `reduction_group_r <= 1'b1;` This makes reduction_group_r, the value named 'reduction group r' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 731: `state <= ST_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_MULTIPLY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MULTIPLY: the value named ’st multiply’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 732: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 733: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 734: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 735: `if (result_column_index[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_column_index: an integer selecting one tensor element, lane, bank, or state entry; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 736: `< result_column_count) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_column_count: one output-feature position processed in parallel.

Line 737: `if (accumulate_result_r) begin` This condition selects the following hardware branch only when ‘accumulate_result_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are accumulate_result_r: the value named ’accumulate result r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 738: `selected_accumulator_r[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are selected_accumulator_r: a widened running sum that prevents intermediate overflow; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 739: `<= accumulator_bank_read_data` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are accumulator_bank_read_data: a widened running sum that prevents intermediate overflow.

Line 740: `[result_lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 741: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 742: `selected_accumulator_r[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are selected_accumulator_r: a widened running sum that prevents intermediate overflow; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 743: `<= 47'sd0;` Within the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are sd0: the value named 'sd0' in this block.

Line 744: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 745: `commit_lane_valid_r[result_lane] <= 1'b1;` This makes commit_lane_valid_r[result_lane], the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 746: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 747: `selected_accumulator_r[result_lane] <= 47'sd0;` This makes selected_accumulator_r[result_lane], a widened running sum that prevents intermediate overflow, equal to '47'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 748: `commit_lane_valid_r[result_lane] <= 1'b0;` This makes commit_lane_valid_r[result_lane], the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 749: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 750: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 751: `state <= ST_ADD_ACCUMULATOR;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ADD_ACCUMULATOR'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ADD_ACCUMULATOR: a widened running sum that prevents intermediate overflow. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase..

Line 752: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 753: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 755: `ST_ADD_ACCUMULATOR: begin` This is the ST_ADD_ACCUMULATOR case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase..

Line 756: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase..

Line 757: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 758: `accumulated_wide_r[result_lane] <=` This statement continues through source line 762; the explanation uses the complete expression rather than only the first visual fragment. This makes accumulated_wide_r[result_lane], the value named 'accumulated wide r' in this block, equal to '{selected_accumulator_r[result_lane][46], selected_accumulator_r[result_ + {{13{row_partial_r[result_lane][34]}}, row_partial_r[result_lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are selected_accumulator_r: a widened running sum that prevents intermediate overflow; result_lane: one parallel arithmetic element within a vector or tensor datapath; row_partial_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase..

Line 759: `{selected_accumulator_r[result_lane][46],` This is continuation line 2 of the 5-line logical statement that begins at source line 758. Its exact fragment is '{selected_accumulator_r[result_lane][46],'. The important operands are selected_accumulator_r: a widened running sum that prevents intermediate overflow; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes accumulated_wide_r[result_lane], the value named 'accumulated wide r' in this block, equal to '{selected_accumulator_r[result_lane][46], selected_accumulator_r[result_lane]} + {{13{row_partial_r[result_lane][34]}}, row_partial_r[result_lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are selected_accumulator_r: a widened running sum that prevents intermediate overflow; result_lane: one parallel arithmetic element within a vector or tensor datapath; row_partial_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase..

Line 760: `selected_accumulator_r[result_lane]}` This is continuation line 3 of the 5-line logical statement that begins at source line 758. Its exact fragment is 'selected_accumulator_r[result_lane]}'. The important operands are selected_accumulator_r: a widened running sum that prevents intermediate overflow; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes accumulated_wide_r[result_lane], the value named 'accumulated wide r' in this block, equal to '{selected_accumulator_r[result_lane][46], selected_accumulator_r[result_lane]} + {{13{row_partial_r[result_lane][34]}}, row_partial_r[result_lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are selected_accumulator_r: a widened running sum that prevents intermediate overflow; result_lane: one parallel arithmetic element within a vector or tensor datapath; row_partial_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase..

Line 761: `+ {{13{row_partial_r[result_lane][34]}},` This is continuation line 4 of the 5-line logical statement that begins at source line 758. Its exact fragment is '+ {{13{row_partial_r[result_lane][34]}},'. The important operands are row_partial_r: one output-row position processed by the tensor tile; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes accumulated_wide_r[result_lane], the value named 'accumulated wide r' in this block, equal to '{selected_accumulator_r[result_lane][46], selected_accumulator_r[result_lane]} + {{13{row_partial_r[result_lane][34]}}, row_partial_r[result_lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are selected_accumulator_r: a widened running sum that prevents intermediate overflow; result_lane: one parallel arithmetic element within a vector or tensor datapath; row_partial_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase..

Line 762: `row_partial_r[result_lane]};` This is continuation line 5 of the 5-line logical statement that begins at source line 758. Its exact fragment is 'row_partial_r[result_lane]};'. The important operands are row_partial_r: one output-row position processed by the tensor tile; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes accumulated_wide_r[result_lane], the value named 'accumulated wide r' in this block, equal to '{selected_accumulator_r[result_lane][46], selected_accumulator_r[result_lane]} + {{13{row_partial_r[result_lane][34]}}, row_partial_r[result_lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are selected_accumulator_r: a widened running sum that prevents intermediate overflow; result_lane: one parallel arithmetic element within a vector or tensor datapath; row_partial_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase..

Line 763: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 764: `state <= ST_STORE_ACCUMULATOR;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_STORE_ACCUMULATOR'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_STORE_ACCUMULATOR: a widened running sum that prevents intermediate overflow. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase..

Line 765: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ADD_ACCUMULATOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 767: `ST_STORE_ACCUMULATOR: begin` This is the ST_STORE_ACCUMULATOR case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase..

Line 768: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase..

Line 769: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 770: `if (commit_lane_valid_r[result_lane]) begin` This condition selects the following hardware branch only when 'commit_lane_valid_r[result_lane]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are commit_lane_valid_r: the producer's statement that its

current payload is meaningful; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase..

Line 771: `rounding_value_r[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 772: `<= saturate_s47(accumulated_wide_r[result_lane]);` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are saturate_s47: the value named 'saturate s47' in this block; accumulated_wide_r: the value named 'accumulated wide r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 773: `rounding_negative_r[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_negative_r: the value named 'rounding negative r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 774: `<= accumulated_wide_r[result_lane][47];` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are accumulated_wide_r: the value named 'accumulated wide r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 775: `if (accumulated_wide_r[result_lane][47]` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are accumulated_wide_r: the value named 'accumulated wide r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 776: `!= accumulated_wide_r[result_lane][46]) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are accumulated_wide_r: the value named 'accumulated wide r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 777: `config_error <= 1'b1;` This makes config_error, the value named 'config error' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase..

Line 778: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 779: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 780: `rounding_value_r[result_lane] <= 47'd0;` This makes rounding_value_r[result_lane], the value named 'rounding value r' in this block, equal to '47'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase..

Line 781: `rounding_negative_r[result_lane] <= 1'b0;` This makes rounding_negative_r[result_lane], the value named 'rounding negative r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase..

Line 782: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 783: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 784: `reduction_group_r <= 1'b0;` This makes reduction_group_r, the value named 'reduction group r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase..

Line 785: `state <= ST_ROUND_MAGNITUDE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ROUND_MAGNITUDE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ROUND_MAGNITUDE: the value named 'st round magnitude' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase..

Line 786: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_ACCUMULATOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 788: `ST_ROUND_MAGNITUDE: begin` This is the ST_ROUND_MAGNITUDE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase..

Line 789: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase..

Line 790: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 791: `if (rounding_negative_r[result_lane]) begin` This condition selects the following hardware branch only when 'rounding_negative_r[result_lane]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are rounding_negative_r: the value named 'rounding negative r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase..

Line 792: `{rounding_carry_r[result_lane],` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_carry_r: the value named 'rounding carry r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 793: `rounding_value_r[result_lane][23:0]} <=` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 794: `{1'b0, ~rounding_value_r[result_lane][23:0]}` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are b0: the value named 'b0' in this block; rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 795: `+ 25'd1;` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are d1: the value named 'd1' in this block.

Line 796: `end else rounding_carry_r[result_lane] <= 1'b0;` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 797: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 798: `state <= ST_ROUND_MAGNITUDE_HIGH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ROUND_MAGNITUDE_HIGH'. It updates after the active clock edge together with other non-blocking assignments. The important operands are ST_ROUND_MAGNITUDE_HIGH: the value named 'st round magnitude high' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase..

Line 799: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 801: `ST_ROUND_MAGNITUDE_HIGH: begin` This is the ST_ROUND_MAGNITUDE_HIGH case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase..

Line 802: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase..

Line 803: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 804: `if (rounding_negative_r[result_lane]) begin` This condition selects the following hardware branch only when ‘rounding_negative_r[result_lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are rounding_negative_r: the value named ’rounding negative r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase..

Line 805: `rounding_value_r[result_lane][46:24] <=` This statement continues through source line 807; the explanation uses the complete expression rather than only the first visual fragment. This makes rounding_value_r[result_lane][46:24], the value named ’rounding value r’ in this block, equal to ‘(~rounding_value_r[result_lane][46:24]) + rounding_carry_r[result_lane] It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named ’rounding value r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_carry_r: the value named ’rounding carry r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase..

Line 806: `(~rounding_value_r[result_lane][46:24])` This is continuation line 2 of the 3-line logical statement that begins at source line 805. Its exact fragment is ‘(~rounding_value_r[result_lane][46:24])‘. The important operands are rounding_value_r: the value named ’rounding value r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes rounding_value_r[result_lane][46:24], the value named ’rounding value r’ in this block, equal to ‘(~rounding_value_r[result_lane][46:24]) + rounding_carry_r[result_lane] It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named ’rounding value r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_carry_r: the value named ’rounding carry r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase..

Line 807: `+ rounding_carry_r[result_lane];` This is continuation line 3 of the 3-line logical statement that begins at source line 805. Its exact fragment is ‘+ rounding_carry_r[result_lane];‘. The important operands are rounding_carry_r: the value named ’rounding carry r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes rounding_value_r[result_lane][46:24], the value named ’rounding value r’ in this block, equal to ‘(~rounding_value_r[result_lane][46:24]) + rounding_carry_r[result_lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named ’rounding value r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_carry_r: the value named ’rounding carry r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase..

Line 808: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 809: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 810: `state <= ST_ROUND_BIAS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_ROUND_BIAS‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ROUND_BIAS: an additive learned offset applied after accumulation. Context: the clocked

process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase..

Line 811: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_MAGNITUDE_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 813: `ST_ROUND_BIAS: begin` This is the ST_ROUND_BIAS case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase..

Line 814: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase..

Line 815: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 816: `{rounding_carry_r[result_lane],` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_carry_r: the value named 'rounding carry r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 817: `rounding_value_r[result_lane][23:0]} <=` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 818: `{1'b0, rounding_value_r[result_lane][23:0]}` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are b0: the value named 'b0' in this block; rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 819: `+ {1'b0, rounding_bias_r[23:0]};` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are b0: the value named 'b0' in this block; rounding_bias_r: an additive learned offset applied after accumulation.

Line 820: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 821: `state <= ST_ROUND_BIAS_HIGH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ROUND_BIAS_HIGH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ROUND_BIAS_HIGH: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase..

Line 822: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 824: `ST_ROUND_BIAS_HIGH: begin` This is the ST_ROUND_BIAS_HIGH case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase..

Line 825: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase..

Line 826: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 827: `rounding_value_r[result_lane][46:24] <=` This statement continues through source line 830; the explanation uses the complete expression rather than only the first visual fragment. This makes rounding_value_r[result_lane][46:24], the value named 'rounding value r' in this block, equal to 'rounding_value_r[result_lane][46:24] + rounding_bias_r[46:24] + rounding_carry_r[result_lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_bias_r: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase..

Line 828: `rounding_value_r[result_lane][46:24]` This is continuation line 2 of the 4-line logical statement that begins at source line 827. Its exact fragment is 'rounding_value_r[result_lane][46:24]'. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes rounding_value_r[result_lane][46:24], the value named 'rounding value r' in this block, equal to 'rounding_value_r[result_lane][46:24] + rounding_bias_r[46:24] + rounding_carry_r[result_lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_bias_r: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase..

Line 829: `+ rounding_bias_r[46:24]` This is continuation line 3 of the 4-line logical statement that begins at source line 827. Its exact fragment is '+ rounding_bias_r[46:24]'. The important operands are rounding_bias_r: an additive learned offset applied after accumulation. Read together, the complete operation is: This makes rounding_value_r[result_lane][46:24], the value named 'rounding value r' in this block, equal to 'rounding_value_r[result_lane][46:24] + rounding_bias_r[46:24] + rounding_carry_r[result_lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_bias_r: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase..

Line 830: `+ rounding_carry_r[result_lane];` This is continuation line 4 of the 4-line logical statement that begins at source line 827. Its exact fragment is '+ rounding_carry_r[result_lane];'. The important operands are rounding_carry_r: the value named 'rounding carry r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes rounding_value_r[result_lane][46:24], the value named 'rounding value r' in this block, equal to 'rounding_value_r[result_lane][46:24] + rounding_bias_r[46:24] + rounding_carry_r[result_lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_bias_r: an additive learned offset applied

after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase..

Line 831: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 832: `state <= ST_ROUND_SHIFT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ROUND_SHIFT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ROUND_SHIFT: the value named 'st round shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase..

Line 833: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 835: `ST_ROUND_SHIFT: begin` This is the ST_ROUND_SHIFT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SHIFT, the machine performs the named control phase..

Line 836: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SHIFT, the machine performs the named control phase..

Line 837: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SHIFT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 838: `rounding_value_r[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SHIFT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 839: `<= rounding_value_r[result_lane] >> result_shift_r;` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SHIFT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; result_shift_r: the value named 'result shift r' in this block.

Line 840: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SHIFT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 841: `state <= ST_ROUND_SIGN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ROUND_SIGN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ROUND_SIGN: the value named 'st round sign' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SHIFT, the machine performs the named control phase..

Line 842: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SHIFT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 844: `ST_ROUND_SIGN: begin` This is the ST_ROUND_SIGN case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase..

Line 845: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase..

Line 846: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 847: `if (rounding_negative_r[result_lane]) begin` This condition selects the following hardware branch only when ‘rounding_negative_r[result_lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are rounding_negative_r: the value named ’rounding negative r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase..

Line 848: `{rounding_carry_r[result_lane],` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_carry_r: the value named ’rounding carry r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 849: `rounding_value_r[result_lane][23:0]} <=` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_value_r: the value named ’rounding value r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 850: `{1’b0, ~rounding_value_r[result_lane][23:0]}` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are b0: the value named ’b0’ in this block; rounding_value_r: the value named ’rounding value r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 851: `+ 25’d1;` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are d1: the value named ’d1’ in this block.

Line 852: `end else rounding_carry_r[result_lane] <= 1’b0;` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 853: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 854: `state <= ST_ROUND_SIGN_HIGH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_ROUND_SIGN_HIGH‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ROUND_SIGN_HIGH: the value named ’st round sign high’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase..

Line 855: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 857: `ST_ROUND_SIGN_HIGH: begin` This is the ST_ROUND_SIGN_HIGH case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase..

Line 858: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase..

Line 859: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 860: `if (rounding_negative_r[result_lane]) begin` This condition selects the following hardware branch only when ‘rounding_negative_r[result_lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are rounding_negative_r: the value named ’rounding negative r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase..

Line 861: `rounding_value_r[result_lane][46:24] <=` This statement continues through source line 863; the explanation uses the complete expression rather than only the first visual fragment. This makes rounding_value_r[result_lane][46:24], the value named ’rounding value r’ in this block, equal to ‘(~rounding_value_r[result_lane][46:24]) + rounding_carry_r[result_lane] It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named ’rounding value r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_carry_r: the value named ’rounding carry r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase..

Line 862: `(~rounding_value_r[result_lane][46:24])` This is continuation line 2 of the 3-line logical statement that begins at source line 861. Its exact fragment is ‘(~rounding_value_r[result_lane][46:24])‘. The important operands are rounding_value_r: the value named ’rounding value r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes rounding_value_r[result_lane][46:24], the value named ’rounding value r’ in this block, equal to ‘(~rounding_value_r[result_lane][46:24]) + rounding_carry_r[result_lane] It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named ’rounding value r’ in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_carry_r: the value named ’rounding carry r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase..

Line 863: `+ rounding_carry_r[result_lane];` This is continuation line 3 of the 3-line logical statement that begins at source line 861. Its exact fragment is '+ rounding_carry_r[result_lane];'. The important operands are rounding_carry_r: the value named 'rounding carry r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes rounding_value_r[result_lane][46:24], the value named 'rounding value r' in this block, equal to '(~rounding_value_r[result_lane][46:24]) + rounding_carry_r[result_lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath; rounding_carry_r: the value named 'rounding carry r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase..

Line 864: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 865: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 866: `state <= ST_SATURATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_SATURATE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SATURATE: the value named 'st saturate' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase..

Line 867: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_SIGN_HIGH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 869: `ST_SATURATE: begin` This is the ST_SATURATE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 870: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 871: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 872: `if (result_column_index[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_column_index: an integer selecting one tensor element, lane, bank, or state entry; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 873: `< result_column_count) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_column_count: one output-feature position processed in parallel.

Line 874: `if (input_gradient_lane_r[result_lane]) begin` This condition selects the following hardware branch only when ‘input_gradient_lane_r[result_lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_gradient_lane_r: the signed loss derivative used to update a learned parameter; result_lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 875: `saturated_result_r[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are saturated_result_r: the value named 'saturated result r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 876: `<= saturate_s32($signed(` Within the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are saturate_s32: the value named 'saturate s32' in this block.

Line 877: `rounding_value_r[result_lane]));` Within the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 878: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 879: `saturated_result_r[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are saturated_result_r: the value named 'saturated result r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 880: `<= saturate_s24_extended($signed(` Within the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are saturate_s24_extended: the value named 'saturate s24 extended' in this block.

Line 881: `rounding_value_r[result_lane]));` Within the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are rounding_value_r: the value named 'rounding value r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 882: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 883: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 884: `saturated_result_r[result_lane] <= 32'sd0;` This makes saturated_result_r[result_lane], the value named 'saturated result r' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 885: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 886: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 887: `state <= ST_STORE_RESULT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_STORE_RESULT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_STORE_RESULT: the value named 'st store result' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase..

Line 888: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 890: `ST_STORE_RESULT: begin` This is the ST_STORE_RESULT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 891: `for (int result_lane = 0; result_lane < RESULT_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 892: `result_lane++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 893: `if (column_group_r) begin` This condition selects the following hardware branch only when 'column_group_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are column_group_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 894: `result_row_buffer_r[result_lane + RESULT_LANES]` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_row_buffer_r: local storage that decouples producer and consumer timing; result_lane: one parallel arithmetic element within a vector or tensor datapath; RESULT_LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 895: `<= saturated_result_r[result_lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four

signed DSP array. The important operands are saturated_result_r: the value named 'saturated result r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 896: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 897: `result_row_buffer_r[result_lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are result_row_buffer_r: local storage that decouples producer and consumer timing; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 898: `<= saturated_result_r[result_lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are saturated_result_r: the value named 'saturated result r' in this block; result_lane: one parallel arithmetic element within a vector or tensor datapath.

Line 899: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 900: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 902: `if (!column_group_r) begin` This condition selects the following hardware branch only when '!column_group_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are column_group_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 903: `column_group_r <= 1'b1;` This makes column_group_r, one output-feature position processed in parallel, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 904: `state <= ST_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MULTIPLY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MULTIPLY: the value named 'st multiply' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 905: `end else if (suppress_output_r) begin` This condition selects the following hardware branch only when 'suppress_output_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are suppress_output_r: the value named 'suppress output r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 906: `column_group_r <= 1'b0;` This makes column_group_r, one output-feature position processed in parallel, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 907: `if (result_row_r == result_row_count - 1'b1) begin` This condition selects the following hardware branch only when ‘result_row_r == result_row_count - 1’b1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_row_r: one output-row position processed by the tensor tile; result_row_count: one output-row position processed by the tensor tile; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 908: `if (compute_mode_r == MODE_INPUT_GRADIENT` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are compute_mode_r: the value named ’compute mode r’ in this block; MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter.

Line 909: `&& mode_r == MODE_BOTH) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase., this exact statement contributes to the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array. The important operands are mode_r: the value named ’mode r’ in this block; MODE_BOTH: the value named ’mode both’ in this block.

Line 910: `state <= ST_SWITCH_DW;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_SWITCH_DW‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SWITCH_DW: the value named ’st switch dw’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 911: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 912: `completed_tiles <= completed_tiles + 1'b1;` This makes completed_tiles, the value named ’completed tiles’ in this block, equal to ‘completed_tiles + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are completed_tiles: the value named ’completed tiles’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 913: `state <= ST_DONE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_DONE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DONE: the value named ’st done’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 914: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 915: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 916: `result_row_r <= result_row_r + 1'b1;` This makes result_row_r, one output-row position processed by the tensor tile, equal to ‘result_row_r + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_row_r: one output-row position processed by the tensor tile; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 917: `state <= ST_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_MULTIPLY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MULTIPLY: the value named ’st multiply’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 918: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 919: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 920: `output_word_r <= 2'd0;` This makes output_word_r, the value named ’output word r’ in this block, equal to ‘2’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 921: `if (compute_mode_r == MODE_INPUT_GRADIENT) begin` This condition selects the following hardware branch only when ‘compute_mode_r == MODE_INPUT_GRADIENT‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are compute_mode_r: the value named ’compute mode r’ in this block; MODE_INPUT_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 922: `state <= ST_OUTPUT_DX;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_DX‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DX: the value named ’st output dx’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 923: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 924: `state <= ST_OUTPUT_DW;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_DW‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DW: the value named ’st output dw’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 925: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 926: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 927: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 929: `ST_OUTPUT_DX: if (m_axis_tready) begin` This is the ST_OUTPUT_DX case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘if (m_axis_tready) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 930: `if (output_word_r == 3) begin` This condition selects the following hardware branch only when ‘output_word_r == 3‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_word_r: the value named ’output word r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 931: `output_word_r <= 2'd0;` This makes output_word_r, the value named ’output word r’ in this block, equal to ‘2’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 932: `column_group_r <= 1'b0;` This makes column_group_r, one output-feature position processed in parallel, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 933: `reduction_group_r <= 1'b0;` This makes reduction_group_r, the value named ’reduction group r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 934: `if (result_row_r == rows_r - 1'b1) begin` This condition selects the following hardware branch only when ‘result_row_r == rows_r - 1’b1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_row_r: one output-row position processed by the tensor tile; rows_r: one output-row position processed by the tensor tile; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 935: `if (mode_r == MODE_BOTH) state <= ST_SWITCH_DW;` This condition selects the following hardware branch only when ‘mode_r == MODE_BOTH‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are mode_r: the value named ’mode r’ in this block; MODE_BOTH: the value named ’mode both’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 936: `else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 937: `completed_tiles <= completed_tiles + 1'b1;` This makes completed_tiles, the value named ’completed tiles’ in this block, equal to ‘completed_tiles + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are completed_tiles: the value named ’completed tiles’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 938: `accumulator_valid_r <= 1'b0;` This makes accumulator_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 939: `state <= ST_DONE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_DONE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DONE: the value named ’st done’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 940: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 941: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 942: `result_row_r <= result_row_r + 1'b1;` This makes result_row_r, one output-row position processed by the tensor tile, equal to ‘result_row_r + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_row_r: one output-row position processed by the tensor tile; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 943: `state <= ST_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_MULTIPLY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MULTIPLY: the value named ’st multiply’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure..

Line 944: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 945: `end else output_word_r <= output_word_r + 1'b1;` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 946: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DX, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 948: `ST_OUTPUT_DW: if (m_axis_tready) begin` This is the ST_OUTPUT_DW case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘if (m_axis_tready) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 949: `if (output_word_r == 2) begin` This condition selects the following hardware branch only when ‘output_word_r == 2‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_word_r: the value named ’output word r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 950: `output_word_r <= 2'd0;` This makes output_word_r, the value named ’output word r’ in this block, equal to ‘2’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 951: `column_group_r <= 1'b0;` This makes column_group_r, one output-feature position processed in parallel, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 952: `reduction_group_r <= 1'b0;` This makes reduction_group_r, the value named 'reduction group r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 953: `if (result_row_r == inputs_r - 1'b1) begin` This condition selects the following hardware branch only when ‘result_row_r == inputs_r - 1'b1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_row_r: one output-row position processed by the tensor tile; inputs_r: the value named 'inputs r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 954: `completed_tiles <= completed_tiles + 1'b1;` This makes completed_tiles, the value named 'completed tiles' in this block, equal to ‘completed_tiles + 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are completed_tiles: the value named 'completed tiles' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 955: `accumulator_valid_r <= 1'b0;` This makes accumulator_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 956: `state <= ST_DONE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_DONE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DONE: the value named 'st done' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 957: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 958: `result_row_r <= result_row_r + 1'b1;` This makes result_row_r, one output-row position processed by the tensor tile, equal to ‘result_row_r + 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_row_r: one output-row position processed by the tensor tile; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 959: `state <= ST_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_MULTIPLY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MULTIPLY: the value named 'st multiply' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure..

Line 960: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 961: `end else output_word_r <= output_word_r + 1'b1;` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 962: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DW, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 964: `ST_SWITCH_DW: begin` This is the ST_SWITCH_DW case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 965: `for (int row = 0; row < LANES; row++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 966: `result_row_buffer_r[row] <= 32'sd0;` This makes result_row_buffer_r[row], local storage that decouples producer and consumer timing, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 967: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 968: `compute_mode_r <= MODE_WEIGHT_GRADIENT;` This makes compute_mode_r, the value named 'compute mode r' in this block, equal to 'MODE_WEIGHT_GRADIENT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_WEIGHT_GRADIENT: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 969: `input_gradient_lane_r <= {RESULT_LANES{1'b0}};` This makes input_gradient_lane_r, the signed loss derivative used to update a learned parameter, equal to '{RESULT_LANES{1'b0}}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are RESULT_LANES: one parallel arithmetic element within a vector or tensor datapath; b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 970: `compute_result_row_count_r <= inputs_r;` This makes compute_result_row_count_r, one output-row position processed by the tensor tile, equal to 'inputs_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are inputs_r: the value named 'inputs r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 971: `compute_result_column_count_r <= outputs_r;` This makes compute_result_column_count_r, one output-feature position processed in parallel, equal to 'outputs_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are outputs_r: the value named 'outputs r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 972: `compute_reduction_count_r <= rows_r;` This makes compute_reduction_count_r, an integer count retained for control or measurable telemetry, equal to ‘rows_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are rows_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 973: `result_shift_r <= dw_shift_r;` This makes result_shift_r, the value named ’result shift r’ in this block, equal to ‘dw_shift_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are dw_shift_r: the value named ’dw shift r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 974: `rounding_bias_r <= round_bias(dw_shift_r);` This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to ‘round_bias(dw_shift_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are round_bias: an additive learned offset applied after accumulation; dw_shift_r: the value named ’dw shift r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 975: `accumulator_mode_r <= MODE_WEIGHT_GRADIENT;` This makes accumulator_mode_r, a widened running sum that prevents intermediate overflow, equal to ‘MODE_WEIGHT_GRADIENT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_WEIGHT_GRADIENT: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 976: `accumulator_rows_r <= inputs_r;` This makes accumulator_rows_r, a widened running sum that prevents intermediate overflow, equal to ‘inputs_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are inputs_r: the value named ’inputs r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 977: `accumulator_columns_r <= outputs_r;` This makes accumulator_columns_r, a widened running sum that prevents intermediate overflow, equal to ‘outputs_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are outputs_r: the value named ’outputs r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 978: `result_row_r <= 4'd0;` This makes result_row_r, one output-row position processed by the tensor tile, equal to ‘4’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 979: `column_group_r <= 1'b0;` This makes column_group_r, one output-feature position processed in parallel, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 980: `reduction_group_r <= 1'b0;` This makes reduction_group_r, the value named ’reduction group r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 981: `output_word_r <= 2'd0;` This makes output_word_r, the value named ’output word r’ in this block, equal to ‘2’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 982: `state <= ST_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_MULTIPLY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MULTIPLY: the value named ’st multiply’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase..

Line 983: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SWITCH_DW, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 985: `ST_DONE: state <= ST_IDLE;` This is the ST_DONE case branch, which raises completion for one cycle and returns control to idle. The same-cycle action begins with ‘state <= ST_IDLE;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DONE, the machine raises completion for one cycle and returns control to idle..

Line 986: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘state <= ST_IDLE;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DONE, the machine raises completion for one cycle and returns control to idle..

Line 987: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DONE, the machine raises completion for one cycle and returns control to idle.; it has no arithmetic value of its own.

Line 989: `if (clear_counters) begin` This condition selects the following hardware branch only when ‘clear_counters‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_counters: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge.

Line 990: `completed_tiles <= 32’d0;` This makes completed_tiles, the value named ’completed tiles’ in this block, equal to ‘32’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 991: `active_cycles <= 32’d0;` This makes active_cycles, the value named ’active cycles’ in this block, equal to ‘32’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 992: `accumulator_valid_r <= 1’b0;` This makes accumulator_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 993: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 994: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 995: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 997: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 998: `if (LANES != 8) begin` This condition selects the following hardware branch only when ‘LANES != 8‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the module’s structural declarations.

Line 999: `$fatal(1, "neuraedge_linear_backward8 currently requires LANES=8");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 1000: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 1001: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 1002: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 1004: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array.

14 rtl/neuraedge_log_softmax.sv

Role in the complete system: implements power-of-two integer softmax normalization without floating-point exponent hardware.

14.1 Beginner architecture map

Dataflow responsibility The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_log_softmax into physical nets and primitives. The source declares 9 input-port lines, 11 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_EXPONENT_ACCUMULATE, ST_EXPONENT_CLAMP, ST_EXPONENT_DIFFERENCE, ST_EXPONENT_LATCH, ST_EXPONENT_QUANTIZE, ST_EXPONENT_READ, ST_EXPONENT_SHIFT, ST_IDLE, ST_LOAD, ST_OUTPUT_HOLD, ST_OUTPUT_LATCH, ST_OUTPUT_MULTI ST_OUTPUT_OPERANDS, ST_OUTPUT_PRODUCT_ADD, ST_OUTPUT_READ, ST_OUTPUT_ROUND, ST_RECIPROCAL_START, ST_RECIPROCAL_WAIT. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates neuraedge_udivider as u_divider, unique as case. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

14.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_log_softmax #(
5      parameter int MAX_ITEMS = 256,
6      parameter int SCORE_BITS = 24
7  ) (
8      input  wire clk,
9      input  wire rst_n,
10     input  wire start_valid,
11     output logic start_ready,
12     input  wire [8:0] item_count,
13     input  wire [4:0] difference_shift,
14     input  wire score_valid,
15     output logic score_ready,
16     input  wire signed [SCORE_BITS-1:0] score_value,
17     input  wire score_last,
18     output logic probability_valid,
19     input  wire probability_ready,
20     output logic [15:0] probability_q15,
21     output logic [8:0] probability_index,
22     output logic probability_last,
23     output logic config_error,
24     output logic [31:0] cycle_count,
25     output logic signed [SCORE_BITS-1:0] debug_maximum_score,
26     output logic [23:0] debug_denominator,
27     output logic [31:0] debug_reciprocal_q31
28 );
29     typedef enum logic [4:0] {
30         ST_IDLE = 5'd0,
31         ST_LOAD = 5'd1,
32         ST_EXPONENT_READ = 5'd2,
33         ST_EXPONENT_DIFFERENCE = 5'd3,
34         ST_EXPONENT_QUANTIZE = 5'd4,
35         ST_EXPONENT_ACCUMULATE = 5'd5,
36         ST_RECIPROCAL_START = 5'd6,
37         ST_RECIPROCAL_WAIT = 5'd7,
38         ST_OUTPUT_READ = 5'd8,
39         ST_OUTPUT_MULTIPLY = 5'd9,
```

```
        ST_OUTPUT_ROUND = 5'd10,
        ST_OUTPUT_HOLD = 5'd11,
        ST_EXPONENT_LATCH = 5'd12,
        ST_OUTPUT_LATCH = 5'd13,
        ST_OUTPUT_OPERANDS = 5'd14,
        ST_OUTPUT_PRODUCT_ADD = 5'd15,
        ST_EXPONENT_SHIFT = 5'd16,
        ST_EXPONENT_CLAMP = 5'd17
    } state_t;

    (* fsm_encoding = "one_hot" *) state_t state;
    (* ram_style = "block" *) logic signed [SCORE_BITS-1:0] score_mem [0:MAX_ITEMS-1];
    (* ram_style = "block" *) logic [15:0] exponent_mem [0:MAX_ITEMS-1];
    logic [8:0] item_count_r;
    logic [4:0] difference_shift_r;
    logic [8:0] load_index;
    logic [8:0] load_remaining_r;
    logic [8:0] exponent_index;
    logic [8:0] exponent_remaining_r;
    logic [8:0] output_index_r;
    logic [8:0] output_remaining_r;
    logic signed [SCORE_BITS-1:0] maximum_score_r;
    logic [23:0] denominator_r;
    logic [31:0] reciprocal_q31_r;
    logic [15:0] output_exponent_r;
    logic [15:0] output_exponent_ram_r;
    logic [15:0] multiply_exponent_r;
    logic [31:0] multiply_reciprocal_r;
    logic signed [SCORE_BITS-1:0] exponent_score_r;
    logic signed [SCORE_BITS-1:0] exponent_score_ram_r;
    logic [SCORE_BITS:0] exponent_difference_r;
    logic [SCORE_BITS:0] shifted_difference_r;
    logic [4:0] exponent_r;
    logic [15:0] unnormalized_r;
    logic [31:0] probability_product_r;
    logic [31:0] probability_low_product_r;
    logic [31:0] probability_high_product_r;
    logic probability_last_r;
    (* use_dsp = "yes" *) wire [31:0] probability_low_product =
        multiply_exponent_r * multiply_reciprocal_r[15:0];
    (* use_dsp = "yes" *) wire [31:0] probability_high_product =
        multiply_exponent_r * multiply_reciprocal_r[31:16];

    logic divider_input_valid;
    wire divider_input_ready;
    wire divider_output_valid;
    logic divider_output_ready;
    wire [31:0] divider_quotient;
    wire divider_zero;

    neuraedge_udivider #(.NUMERATOR_BITS(32)) u_divider (
        .clk(clk), .rst_n(rst_n),
        .input_valid(divider_input_valid), .input_ready(divider_input_ready),
        .numerator(32'd2147483647), .denominator({8'd0, denominator_r}),
        .output_valid(divider_output_valid), .output_ready(divider_output_ready),
        .quotient(divider_quotient), .remainder(),
        .divide_by_zero(divider_zero)
    );

    always_comb begin
        start_ready = state == ST_IDLE;
        score_ready = state == ST_LOAD;
        probability_valid = state == ST_OUTPUT_HOLD;
        probability_index = output_index_r;
        probability_last = probability_last_r;
        divider_input_valid = state == ST_RECIPROCAL_START;
        divider_output_ready = state == ST_RECIPROCAL_WAIT;
        debug_maximum_score = maximum_score_r;
        debug_denominator = denominator_r;
        debug_reciprocal_q31 = reciprocal_q31_r;
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            state <= ST_IDLE;
            item_count_r <= 9'd0;
            difference_shift_r <= 5'd0;
            load_index <= 9'd0;
            load_remaining_r <= 9'd0;
            exponent_index <= 9'd0;
            exponent_remaining_r <= 9'd0;
            output_index_r <= 9'd0;
            output_remaining_r <= 9'd0;
            maximum_score_r <= {1'b1, {(SCORE_BITS-1){1'b0}}};
            denominator_r <= 24'd0;
            reciprocal_q31_r <= 32'd0;
            output_exponent_r <= 16'd0;
```

```
            output_exponent_ram_r <= 16'd0;
            multiply_exponent_r <= 16'd0;
            multiply_reciprocal_r <= 32'd0;
            exponent_score_r <= '0;
            exponent_score_ram_r <= '0;
            exponent_difference_r <= '0;
            shifted_difference_r <= '0;
            exponent_r <= 5'd0;
            unnormalized_r <= 16'd0;
            probability_product_r <= 32'd0;
            probability_low_product_r <= 32'd0;
            probability_high_product_r <= 32'd0;
            probability_last_r <= 1'b0;
            probability_q15 <= 16'd0;
            config_error <= 1'b0;
            cycle_count <= 32'd0;
        end else begin
            if (state != ST_IDLE) cycle_count <= cycle_count + 1'b1;
            unique case (state)
                ST_IDLE: begin
                    if (start_valid) begin
                        cycle_count <= 32'd0;
                        config_error <= item_count == 0
                            || item_count > 9'(MAX_ITEMS);
                        if (item_count == 0 || item_count > 9'(MAX_ITEMS)) begin
                            state <= ST_IDLE;
                        end else begin
                            item_count_r <= item_count;
                            difference_shift_r <= difference_shift;
                            load_index <= 9'd0;
                            load_remaining_r <= item_count;
                            maximum_score_r <= {1'b1, {(SCORE_BITS-1){1'b0}}};
                            denominator_r <= 24'd0;
                            state <= ST_LOAD;
                        end
                    end
                end

                ST_LOAD: begin
                    if (score_valid) begin
                        logic expected_last;
                        expected_last = load_remaining_r == 9'd1;
                        score_mem[load_index[7:0]] <= score_value;
                        if (score_value > maximum_score_r) maximum_score_r <= score_value;
                        if (score_last != expected_last) config_error <= 1'b1;
                        if (expected_last) begin
                            exponent_index <= 9'd0;
                            exponent_remaining_r <= item_count_r;
                            state <= ST_EXPONENT_READ;
                        end else if (score_last) begin
                            config_error <= 1'b1;
                            state <= ST_IDLE;
                        end else begin
                            load_index <= load_index + 1'b1;
                            load_remaining_r <= load_remaining_r - 1'b1;
                        end
                    end
                end

                ST_EXPONENT_READ: begin
                    exponent_score_ram_r <= score_mem[exponent_index[7:0]];
                    state <= ST_EXPONENT_LATCH;
                end

                ST_EXPONENT_LATCH: begin
                    exponent_score_r <= exponent_score_ram_r;
                    state <= ST_EXPONENT_DIFFERENCE;
                end

                ST_EXPONENT_DIFFERENCE: begin
                    exponent_difference_r <=
                        $signed(maximum_score_r) - $signed(exponent_score_r);
                    state <= ST_EXPONENT_SHIFT;
                end

                ST_EXPONENT_SHIFT: begin
                    shifted_difference_r <= exponent_difference_r >> difference_shift_r;
                    state <= ST_EXPONENT_CLAMP;
                end

                ST_EXPONENT_CLAMP: begin
                    exponent_r <= shifted_difference_r > 15
                        ? 5'd15 : shifted_difference_r[4:0];
                    state <= ST_EXPONENT_QUANTIZE;
                end

                ST_EXPONENT_QUANTIZE: begin
```

```
                    unnormalized_r <= 16'd1 << (15 - exponent_r);
                    state <= ST_EXPONENT_ACCUMULATE;
                end

                ST_EXPONENT_ACCUMULATE: begin
                    exponent_mem[exponent_index[7:0]] <= unnormalized_r;
                    denominator_r <= denominator_r + {8'd0, unnormalized_r};
                    if (exponent_remaining_r == 9'd1) begin
                        state <= ST_RECIPROCAL_START;
                    end else begin
                        exponent_index <= exponent_index + 1'b1;
                        exponent_remaining_r <= exponent_remaining_r - 1'b1;
                        state <= ST_EXPONENT_READ;
                    end
                end

                ST_RECIPROCAL_START: begin
                    if (divider_input_ready) state <= ST_RECIPROCAL_WAIT;
                end

                ST_RECIPROCAL_WAIT: begin
                    if (divider_output_valid) begin
                        reciprocal_q31_r <= divider_quotient;
                        output_index_r <= 9'd0;
                        output_remaining_r <= item_count_r;
                        config_error <= config_error || divider_zero;
                        state <= ST_OUTPUT_READ;
                    end
                end

                ST_OUTPUT_READ: begin
                    output_exponent_ram_r <= exponent_mem[output_index_r[7:0]];
                    state <= ST_OUTPUT_LATCH;
                end

                ST_OUTPUT_LATCH: begin
                    output_exponent_r <= output_exponent_ram_r;
                    state <= ST_OUTPUT_OPERANDS;
                end

                ST_OUTPUT_OPERANDS: begin
                    multiply_exponent_r <= output_exponent_r;
                    multiply_reciprocal_r <= reciprocal_q31_r;
                    state <= ST_OUTPUT_MULTIPLY;
                end

                ST_OUTPUT_MULTIPLY: begin
                    probability_low_product_r <= probability_low_product;
                    probability_high_product_r <= probability_high_product;
                    state <= ST_OUTPUT_PRODUCT_ADD;
                end

                ST_OUTPUT_PRODUCT_ADD: begin
                    probability_product_r <= probability_low_product_r
                        + {probability_high_product_r[15:0], 16'd0};
                    state <= ST_OUTPUT_ROUND;
                end

                ST_OUTPUT_ROUND: begin
                    probability_q15 <= probability_product_r[31:16]
                        + 16'(probability_product_r[15]);
                    probability_last_r <= output_remaining_r == 9'd1;
                    state <= ST_OUTPUT_HOLD;
                end

                ST_OUTPUT_HOLD: begin
                    if (probability_ready) begin
                        if (probability_last) begin
                            state <= ST_IDLE;
                        end else begin
                            output_index_r <= output_index_r + 1'b1;
                            output_remaining_r <= output_remaining_r - 1'b1;
                            state <= ST_OUTPUT_READ;
                        end
                    end
                end

                default: state <= ST_IDLE;
            endcase
        end
    end

    initial begin
        if (MAX_ITEMS < 1 || MAX_ITEMS > 256) $fatal(1, "MAX_ITEMS must be in [1,256]");
        if (SCORE_BITS != 24) $fatal(1, "neuraedge_log_softmax currently requires SCORE_BITS=24");
    end
endmodule
```

```
301
302 `default_nettype wire
```

14.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_log_softmax #(` This opens neuraedge_log_softmax, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that implements power-of-two integer softmax normalization without floating-point exponent hardware.

Line 5: `parameter int MAX_ITEMS = 256,` This statement continues through source line 28; the explanation uses the complete expression rather than only the first visual fragment. This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 6: `parameter int SCORE_BITS = 24` This is continuation line 2 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'parameter int SCORE_BITS = 24'. The important operands are parameter: the value named 'parameter' in this block; SCORE_BITS: the value named 'score bits' in this block. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 7: `) (` This is continuation line 3 of the 24-line logical statement that begins at source line 5. Its exact fragment is ') ('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 8: `input wire clk,` This is continuation line 4 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'input wire clk,'. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output

logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 9: `input wire rst_n,` This is continuation line 5 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'input wire rst_n,'. The important operands are input: the value named 'input' in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 10: `input wire start_valid,` This is continuation line 6 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'input wire start_valid,'. The important operands are input: the value named 'input' in this block; start_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 11: `output logic start_ready,` This is continuation line 7 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic start_ready,'. The important operands are output: the value named 'output' in this block; start_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 12: `input wire [8:0] item_count,` This is continuation line 8 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'input wire [8:0] item_count,'. The important operands are input: the value named 'input' in this block; item_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 13: `input wire [4:0] difference_shift,` This is continuation line 9 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'input wire [4:0] difference_shift,'. The important operands are input: the value named 'input' in this block; difference_shift: the value named 'difference shift' in this block. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input

wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 14: `input wire score_valid,` This is continuation line 10 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'input wire score_valid,'. The important operands are input: the value named 'input' in this block; score_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 15: `output logic score_ready,` This is continuation line 11 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic score_ready,'. The important operands are output: the value named 'output' in this block; score_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 16: `input wire signed [SCORE_BITS-1:0] score_value,` This is continuation line 12 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [SCORE_BITS-1:0] score_value,'. The important operands are input: the value named 'input' in this block; SCORE_BITS: the value named 'score bits' in this block; score_value: the value named 'score value' in this block. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 17: `input wire score_last,` This is continuation line 13 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'input wire score_last,'. The important operands are input: the value named 'input' in this block; score_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 18: `output logic probability_valid,` This is continuation line 14 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic probability_valid,'. The important operands are output: the value

named 'output' in this block; probability_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 19: `input wire probability_ready,` This is continuation line 15 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'input wire probability_ready,'. The important operands are input: the value named 'input' in this block; probability_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 20: `output logic [15:0] probability_q15,` This is continuation line 16 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic [15:0] probability_q15,'. The important operands are output: the value named 'output' in this block; probability_q15: the value named 'probability q15' in this block. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 21: `output logic [8:0] probability_index,` This is continuation line 17 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic [8:0] probability_index,'. The important operands are output: the value named 'output' in this block; probability_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 22: `output logic probability_last,` This is continuation line 18 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic probability_last,'. The important operands are output: the value named 'output' in this block; probability_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 23: `output logic config_error,` This is continuation line 19 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic config_error,'. The important operands are output: the value named 'output' in this block; config_error: the value named 'config error' in this block. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 24: `output logic [31:0] cycle_count,` This is continuation line 20 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] cycle_count,'. The important operands are output: the value named 'output' in this block; cycle_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 25: `output logic signed [SCORE_BITS-1:0] debug_maximum_score,` This is continuation line 21 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic signed [SCORE_BITS-1:0] debug_maximum_score,'. The important operands are output: the value named 'output' in this block; SCORE_BITS: the value named 'score bits' in this block; debug_maximum_score: the value named 'debug maximum score' in this block. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 26: `output logic [23:0] debug_denominator,` This is continuation line 22 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic [23:0] debug_denominator,'. The important operands are output: the value named 'output' in this block; debug_denominator: the value named 'debug denominator' in this block. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 27: `output logic [31:0] debug_reciprocal_q31` This is continuation line 23 of the 24-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] debug_reciprocal_q31'. The important operands are output: the value named 'output' in this block; debug_reciprocal_q31: the value named 'debug reciprocal q31' in this block. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready,

output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 28: `);` This is continuation line 24 of the 24-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes MAX_ITEMS to 256, parameter int SCORE_BITS = 24) (input wire clk, input wire rst_n, input wire start_valid, output logic start_ready, input wire [8:0] item_count, input wire [4:0] difference_shift, input wire score_valid, output logic score_ready, input wire signed [SCORE_BITS-1:0] score_value, input wire score_last, output logic probability_valid, input wire probability_ready, output logic [15:0] probability_q15, output logic [8:0] probability_index, output logic probability_last, output logic config_error, output logic [31:0] cycle_count, output logic signed [SCORE_BITS-1:0] debug_maximum_score, output logic [23:0] debug_denominator, output logic [31:0] debug_reciprocal_q31) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAX_ITEMS denotes the value named 'max items' in this block.

Line 29: `typedef enum logic [4:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 30: `ST_IDLE = 5'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 31: `ST_LOAD = 5'd1,` This assigns a fixed binary encoding to state ST_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 32: `ST_EXPONENT_READ = 5'd2,` This assigns a fixed binary encoding to state ST_EXPONENT_READ, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 33: `ST_EXPONENT_DIFFERENCE = 5'd3,` This assigns a fixed binary encoding to state ST_EXPONENT_DIFFERENCE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 34: `ST_EXPONENT_QUANTIZE = 5'd4,` This assigns a fixed binary encoding to state ST_EXPONENT_QUANTIZE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 35: `ST_EXPONENT_ACCUMULATE = 5'd5,` This assigns a fixed binary encoding to state ST_EXPONENT_ACCUMULATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 36: `ST_RECIPROCAL_START = 5'd6,` This assigns a fixed binary encoding to state ST_RECIPROCAL_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 37: `ST_RECIPROCAL_WAIT = 5'd7,` This assigns a fixed binary encoding to state ST_RECIPROCAL_WAIT, which waits for both command completion and the declared output count. The encoding is visible in waveforms even though the source uses the readable name.

Line 38: `ST_OUTPUT_READ = 5'd8,` This assigns a fixed binary encoding to state ST_OUTPUT_READ, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 39: `ST_OUTPUT_MULTIPLY = 5'd9,` This assigns a fixed binary encoding to state ST_OUTPUT_MULTIPLY, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 40: `ST_OUTPUT_ROUND = 5'd10,` This assigns a fixed binary encoding to state ST_OUTPUT_ROUND, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 41: `ST_OUTPUT_HOLD = 5'd11,` This assigns a fixed binary encoding to state ST_OUTPUT_HOLD, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 42: `ST_EXPONENT_LATCH = 5'd12,` This assigns a fixed binary encoding to state ST_EXPONENT_LATCH, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 43: `ST_OUTPUT_LATCH = 5'd13,` This assigns a fixed binary encoding to state ST_OUTPUT_LATCH, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 44: `ST_OUTPUT_OPERANDS = 5'd14,` This assigns a fixed binary encoding to state ST_OUTPUT_OPERANDS, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 45: `ST_OUTPUT_PRODUCT_ADD = 5'd15,` This assigns a fixed binary encoding to state ST_OUTPUT_PRODUCT_ADD, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 46: `ST_EXPONENT_SHIFT = 5'd16,` This assigns a fixed binary encoding to state ST_EXPONENT_SHIFT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 47: `ST_EXPONENT_CLAMP = 5'd17` This statement continues through source line 48; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_EXPONENT_CLAMP, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 48: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 47. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_EXPONENT_CLAMP, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 50: `(* fsm_encoding = "one_hot" *) state_t state;` Within the module's structural declarations, this exact statement contributes to the block that implements power-of-two integer softmax normalization without floating-point exponent hardware. The important operands are fsm_encoding: the value named 'fsm encoding' in this block; one_hot: the value named 'one hot' in this block; state_t: the finite-state-machine phase retained in flip-flops.

Line 51: `(* ram_style = "block" *) logic signed [SCORE_BITS-1:0] score_mem [0:MAX_ITEMS-1];` Within the module's structural declarations, this exact statement contributes to the block that implements power-of-two integer softmax normalization without floating-point exponent hardware. The important operands are ram_style: the value named 'ram style' in this block; block: the value named 'block' in this block; SCORE_BITS: the value named 'score bits' in this block.

Line 52: `(* ram_style = "block" *) logic [15:0] exponent_mem [0:MAX_ITEMS-1];` Within the module's structural declarations, this exact statement contributes to the block that implements power-of-two integer softmax normalization without floating-point exponent hardware. The important operands are ram_style: the value named 'ram style' in this block; block: the value named 'block' in this block; exponent_mem: local FPGA storage addressed by surrounding index logic.

Line 53: `logic [8:0] item_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, item_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 54: `logic [4:0] difference_shift_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, difference_shift_r is the value named 'difference shift r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 55: `logic [8:0] load_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 56: `logic [8:0] load_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_remaining_r is the value named 'load remaining r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 57: `logic [8:0] exponent_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, exponent_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 58: `logic [8:0] exponent_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, exponent_remaining_r is the value named 'exponent remaining r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 59: `logic [8:0] output_index_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_index_r is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 60: `logic [8:0] output_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_remaining_r is the number of command result beats still expected. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 61: `logic signed [SCORE_BITS-1:0] maximum_score_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, maximum_score_r is the value named 'maximum score r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 62: `logic [23:0] denominator_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, denominator_r is the value named 'denominator r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 63: `logic [31:0] reciprocal_q31_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reciprocal_q31_r is the value named 'reciprocal q31 r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 64: `logic [15:0] output_exponent_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_exponent_r is the value named 'output exponent r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 65: `logic [15:0] output_exponent_ram_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_exponent_ram_r is the value named 'output exponent ram r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 66: `logic [15:0] multiply_exponent_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiply_exponent_r is the value named 'multiply exponent r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 67: `logic [31:0] multiply_reciprocal_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiply_reciprocal_r is the value named 'multiply reciprocal r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 68: `logic signed [SCORE_BITS-1:0] exponent_score_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, exponent_score_r is the value named 'exponent score r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 69: `logic signed [SCORE_BITS-1:0] exponent_score_ram_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, exponent_score_ram_r is the value named 'exponent score ram r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 70: `logic [SCORE_BITS:0] exponent_difference_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, exponent_difference_r is the value named 'exponent difference r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 71: `logic [SCORE_BITS:0] shifted_difference_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, shifted_difference_r is the value named 'shifted difference r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 72: `logic [4:0] exponent_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, exponent_r is the value named 'exponent r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 73: `logic [15:0] unnormalized_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, unnormalized_r is the value named 'unnormalized r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 74: `logic [31:0] probability_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, probability_product_r is the value named 'probability product r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 75: `logic [31:0] probability_low_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, probability_low_product_r is the value named 'probability low product r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 76: `logic [31:0] probability_high_product_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, probability_high_product_r is the value named 'probability high product r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 77: `logic probability_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, probability_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 78: `(* use_dsp = "yes" *) wire [31:0] probability_low_product =` Within the module's structural declarations, this exact statement contributes to the block that implements power-of-two integer softmax normalization without floating-point exponent hardware. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; probability_low_product: the value named 'probability low product' in this block.

Line 79: `multiply_exponent_r * multiply_reciprocal_r[15:0];` Within the module's structural declarations, this exact statement contributes to the block that implements power-of-two integer softmax normalization without floating-point exponent hardware. The important operands are multiply_exponent_r: the value named 'multiply exponent r' in this block; multiply_reciprocal_r: the value named 'multiply reciprocal r' in this block.

Line 80: `(* use_dsp = "yes" *) wire [31:0] probability_high_product =` Within the module's structural declarations, this exact statement contributes to the block that implements power-of-two integer softmax normalization without floating-point exponent hardware. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; probability_high_product: the value named 'probability high product' in this block.

Line 81: `multiply_exponent_r * multiply_reciprocal_r[31:16];` Within the module's structural declarations, this exact statement contributes to the block that implements power-of-two integer softmax normalization without floating-point exponent hardware. The important operands are multiply_exponent_r: the value named 'multiply exponent r' in this block; multiply_reciprocal_r: the value named 'multiply reciprocal r' in this block.

Line 83: `logic divider_input_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 84: `wire divider_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 85: `wire divider_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 86: `logic divider_output_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 87: `wire [31:0] divider_quotient;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_quotient is the value named 'divider quotient' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 88: `wire divider_zero;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divider_zero is the value named 'divider zero' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 90: `neuraedge_udivider #(.NUMERATOR_BITS(32)) u_divider (` This instantiates module neuraedge_udivider as u_divider. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 91: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port ‘clk‘ to parent expression ‘clk), .rst_n(rst_n‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 92: `.input_valid(divider_input_valid), .input_ready(divider_input_ready),` This connects the child instance's named port 'input_valid' to parent expression 'divider_input_valid), .input_ready(divider_input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 93: `.numerator(32'd2147483647), .denominator({8'd0, denominator_r}),` This connects the child instance's named port 'numerator' to parent expression '32'd2147483647), .denominator({8'd0, denominator_r}'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named '32'd2147483647), .denominator({8'd0, denominator r}' in this block.

Line 94: `.output_valid(divider_output_valid), .output_ready(divider_output_ready),` This connects the child instance's named port 'output_valid' to parent expression 'divider_output_valid), .output_ready(divider_output_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 95: `.quotient(divider_quotient), .remainder(),` This connects the child instance's named port 'quotient' to parent expression 'divider_quotient), .remainder('. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'divider quotient), .remainder(' in this block.

Line 96: `.divide_by_zero(divider_zero)` This connects the child instance's named port 'divide_by_zero' to parent expression 'divider_zero'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'divider zero' in this block.

Line 97: `);` Within the module's structural declarations, this exact statement contributes to the block that implements power-of-two integer softmax normalization without floating-point exponent hardware. The expression is a literal or structural operator and has no hidden software state.

Line 99: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 100: `start_ready = state == ST_IDLE;` This makes start_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_IDLE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 101: `score_ready = state == ST_LOAD;` This makes score_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_LOAD'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LOAD: the value named 'st load' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 102: `probability_valid = state == ST_OUTPUT_HOLD;` This makes probability_valid, the producer's statement that its current payload is meaningful, equal to 'state == ST_OUTPUT_HOLD'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT_HOLD: the value named 'st output hold' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 103: `probability_index = output_index_r;` This makes probability_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'output_index_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 104: `probability_last = probability_last_r;` This makes probability_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘probability_last_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are probability_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 105: `divider_input_valid = state == ST_RECIPROCAL_START;` This makes divider_input_valid, the producer’s statement that its current payload is meaningful, equal to ‘state == ST_RECIPROCAL_START‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_RECIPROCAL_START: the value named ’st reciprocal start’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 106: `divider_output_ready = state == ST_RECIPROCAL_WAIT;` This makes divider_output_ready, the consumer’s statement that it can accept a payload this cycle, equal to ‘state == ST_RECIPROCAL_WAIT‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_RECIPROCAL_WAIT: the value named ’st reciprocal wait’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 107: `debug_maximum_score = maximum_score_r;` This makes debug_maximum_score, the value named ’debug maximum score’ in this block, equal to ‘maximum_score_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are maximum_score_r: the value named ’maximum score r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 108: `debug_denominator = denominator_r;` This makes debug_denominator, the value named ’debug denominator’ in this block, equal to ‘denominator_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are denominator_r: the value named ’denominator r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 109: `debug_reciprocal_q31 = reciprocal_q31_r;` This makes debug_reciprocal_q31, the value named ’debug reciprocal q31’ in this block, equal to ‘reciprocal_q31_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are reciprocal_q31_r: the value named ’reciprocal q31 r’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 110: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 112: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with ‘<=‘ inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator’s notion of cycles and persistent state.

Line 113: `if (!rst_n) begin` This is the reset branch for ‘(!rst_n‘. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 114: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 115: `item_count_r <= 9'd0;` This makes item_count_r, an integer count retained for control or measurable telemetry, equal to ‘9’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 116: `difference_shift_r <= 5'd0;` This makes difference_shift_r, the value named 'difference shift r' in this block, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 117: `load_index <= 9'd0;` This makes load_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 118: `load_remaining_r <= 9'd0;` This makes load_remaining_r, the value named 'load remaining r' in this block, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 119: `exponent_index <= 9'd0;` This makes exponent_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 120: `exponent_remaining_r <= 9'd0;` This makes exponent_remaining_r, the value named 'exponent remaining r' in this block, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 121: `output_index_r <= 9'd0;` This makes output_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 122: `output_remaining_r <= 9'd0;` This makes output_remaining_r, the number of command result beats still expected, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 123: `maximum_score_r <= {1'b1, {(SCORE_BITS-1){1'b0}}};` This makes maximum_score_r, the value named 'maximum score r' in this block, equal to '{1'b1, {(SCORE_BITS-1){1'b0}}}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block; SCORE_BITS: the value named 'score bits' in this block; b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 124: `denominator_r <= 24'd0;` This makes denominator_r, the value named 'denominator r' in this block, equal to '24'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 125: `reciprocal_q31_r <= 32'd0;` This makes reciprocal_q31_r, the value named 'reciprocal q31 r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 126: `output_exponent_r <= 16'd0;` This makes output_exponent_r, the value named 'output exponent r' in this block, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 127: `output_exponent_ram_r <= 16'd0;` This makes output_exponent_ram_r, the value named 'output exponent ram r' in this block, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 128: `multiply_exponent_r <= 16'd0;` This makes multiply_exponent_r, the value named 'multiply exponent r' in this block, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 129: `multiply_reciprocal_r <= 32'd0;` This makes multiply_reciprocal_r, the value named 'multiply reciprocal r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 130: `exponent_score_r <= '0;` This makes exponent_score_r, the value named 'exponent score r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 131: `exponent_score_ram_r <= '0;` This makes exponent_score_ram_r, the value named 'exponent score ram r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 132: `exponent_difference_r <= '0;` This makes exponent_difference_r, the value named 'exponent difference r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 133: `shifted_difference_r <= '0;` This makes shifted_difference_r, the value named 'shifted difference r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 134: `exponent_r <= 5'd0;` This makes exponent_r, the value named 'exponent r' in this block, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 135: `unnormalized_r <= 16'd0;` This makes unnormalized_r, the value named 'unnormalized r' in this block, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 136: `probability_product_r <= 32'd0;` This makes probability_product_r, the value named 'probability product r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 137: `probability_low_product_r <= 32'd0;` This makes probability_low_product_r, the value named 'probability low product r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 138: `probability_high_product_r <= 32'd0;` This makes probability_high_product_r, the value named 'probability high product r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 139: `probability_last_r <= 1'b0;` This makes probability_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 140: `probability_q15 <= 16'd0;` This makes probability_q15, the value named 'probability q15' in this block, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 141: `config_error <= 1'b0;` This makes config_error, the value named 'config error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 142: `cycle_count <= 32'd0;` This makes cycle_count, an integer count retained for control or measurable telemetry, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 143: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 144: `if (state != ST_IDLE) cycle_count <= cycle_count + 1'b1;` This condition selects the following hardware branch only when 'state != ST_IDLE' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 145: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 146: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 147: `if (start_valid) begin` This condition selects the following hardware branch only when 'start_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are start_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 148: `cycle_count <= 32'd0;` This makes cycle_count, an integer count retained for control or measurable telemetry, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 149: `config_error <= item_count == 0` This statement continues through source line 150; the explanation uses the complete expression rather than only the first visual fragment. This makes config_error, the value named 'config error' in this block, equal to 'item_count == 0 || item_count > 9'(MAX_ITEMS)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are item_count: an integer count retained for control or measurable telemetry; MAX_ITEMS: the value named 'max items' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 150: `|| item_count > 9'(MAX_ITEMS);` This is continuation line 2 of the 2-line logical statement that begins at source line 149. Its exact fragment is '|| item_count > 9'(MAX_ITEMS);'. The important operands are item_count: an integer count retained for control or measurable telemetry; MAX_ITEMS: the value named 'max items' in this block. Read together, the complete operation is: This makes config_error, the value named 'config error' in this block, equal to 'item_count == 0 || item_count > 9'(MAX_ITEMS)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are item_count: an integer count retained for control or measurable telemetry; MAX_ITEMS: the value named 'max items' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 151: `if (item_count == 0 || item_count > 9'(MAX_ITEMS)) begin` This condition selects the following hardware branch only when 'item_count == 0 || item_count > 9'(MAX_ITEMS)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are item_count: an integer count retained for control or measurable telemetry; MAX_ITEMS: the value named 'max items' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 152: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 153: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 154: `item_count_r <= item_count;` This makes item_count_r, an integer count retained for control or measurable telemetry, equal to 'item_count'. It updates after the active clock edge together with other nonblocking assignments. The important operands are item_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 155: `difference_shift_r <= difference_shift;` This makes difference_shift_r, the value named 'difference shift r' in this block, equal to 'difference_shift'. It updates after the active clock edge together with other nonblocking assignments. The important operands are difference_shift: the value named 'difference shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 156: `load_index <= 9'd0;` This makes load_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 157: `load_remaining_r <= item_count;` This makes load_remaining_r, the value named 'load remaining r' in this block, equal to 'item_count'. It updates after the active clock edge together with other nonblocking assignments. The important operands are item_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 158: `maximum_score_r <= {1'b1, {(SCORE_BITS-1){1'b0}}};` This makes maximum_score_r, the value named 'maximum score r' in this block, equal to '{1'b1, {(SCORE_BITS-1){1'b0}}}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block; SCORE_BITS: the value named 'score bits' in this block; b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 159: `denominator_r <= 24'd0;` This makes denominator_r, the value named 'denominator r' in this block, equal to '24'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 160: `state <= ST_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LOAD'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD: the value named 'st load' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 161: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 162: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 163: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 165: `ST_LOAD: begin` This is the ST_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 166: `if (score_valid) begin` This condition selects the following hardware branch only when 'score_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are score_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 167: `logic expected_last;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.. Specifically, expected_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 168: `expected_last = load_remaining_r == 9'd1;` This makes expected_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'load_remaining_r == 9'd1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are load_remaining_r: the value named 'load remaining r' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 169: `score_mem[load_index[7:0]] <= score_value;` This makes score_mem[load_index[7:0]], local FPGA storage addressed by surrounding index logic, equal to 'score_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are score_value: the value named 'score value' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 170: `if (score_value > maximum_score_r) maximum_score_r <= score_value;` This condition selects the following hardware branch only when ‘score_value > maximum_score_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are score_value: the value named ’score value’ in this block; maximum_score_r: the value named ’maximum score r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 171: `if (score_last != expected_last) config_error <= 1'b1;` This condition selects the following hardware branch only when ‘score_last != expected_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are score_last: the AXI-Stream marker for the final beat of a packet or command payload; expected_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 172: `if (expected_last) begin` This condition selects the following hardware branch only when ‘expected_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are expected_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 173: `exponent_index <= 9'd0;` This makes exponent_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘9’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 174: `exponent_remaining_r <= item_count_r;` This makes exponent_remaining_r, the value named ’exponent remaining r’ in this block, equal to ‘item_count_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are item_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 175: `state <= ST_EXPONENT_READ;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_EXPONENT_READ‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_EXPONENT_READ: the value named ’st exponent read’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 176: `end else if (score_last) begin` This condition selects the following hardware branch only when ‘score_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are score_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 177: `config_error <= 1'b1;` This makes config_error, the value named ’config error’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 178: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 179: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 180: `load_index <= load_index + 1'b1;` This makes load_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'load_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 181: `load_remaining_r <= load_remaining_r - 1'b1;` This makes load_remaining_r, the value named 'load remaining r' in this block, equal to 'load_remaining_r - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are load_remaining_r: the value named 'load remaining r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory..

Line 182: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 183: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 184: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 186: `ST_EXPONENT_READ: begin` This is the ST_EXPONENT_READ case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_READ, the machine performs the named control phase..

Line 187: `exponent_score_ram_r <= score_mem[exponent_index[7:0]];` This makes exponent_score_ram_r, the value named 'exponent score ram r' in this block, equal to 'score_mem[exponent_index[7:0]]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are score_mem: local FPGA storage addressed by surrounding index logic; exponent_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_READ, the machine performs the named control phase..

Line 188: `state <= ST_EXPONENT_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_EXPONENT_LATCH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_EXPONENT_LATCH: the value named 'st exponent latch' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_READ, the machine performs the named control phase..

Line 189: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_READ, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 191: `ST_EXPONENT_LATCH: begin` This is the ST_EXPONENT_LATCH case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_LATCH, the machine performs the named control phase..

Line 192: `exponent_score_r <= exponent_score_ram_r;` This makes exponent_score_r, the value named 'exponent score r' in this block, equal to 'exponent_score_ram_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are exponent_score_ram_r: the value named 'exponent score ram r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_LATCH, the machine performs the named control phase..

Line 193: `state <= ST_EXPONENT_DIFFERENCE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_EXPONENT_DIFFERENCE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_EXPONENT_DIFFERENCE: the value named 'st exponent difference' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_LATCH, the machine performs the named control phase..

Line 194: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_LATCH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 196: `ST_EXPONENT_DIFFERENCE: begin` This is the ST_EXPONENT_DIFFERENCE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_DIFFERENCE, the machine performs the named control phase..

Line 197: `exponent_difference_r <=` This statement continues through source line 198; the explanation uses the complete expression rather than only the first visual fragment. This makes exponent_difference_r, the value named 'exponent difference r' in this block, equal to '$signed(maximum_score_r) - $signed(exponent_score_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are maximum_score_r: the value named 'maximum score r' in this block; exponent_score_r: the value named 'exponent score r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_DIFFERENCE, the machine performs the named control phase..

Line 198: `$signed(maximum_score_r) - $signed(exponent_score_r);` This is continuation line 2 of the 2-line logical statement that begins at source line 197. Its exact fragment is '$signed(maximum_score_r) - $signed(exponent_score_r);'. The important operands are maximum_score_r: the value named 'maximum score r' in this block; exponent_score_r: the value named 'exponent score r' in this block. Read together, the complete operation is: This makes exponent_difference_r, the value named 'exponent difference r' in this block, equal to '$signed(maximum_score_r) - $signed(exponent_score_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are maximum_score_r: the value named 'maximum score r' in this block; exponent_score_r: the value named 'exponent score r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_DIFFERENCE, the machine performs the named control phase..

Line 199: `state <= ST_EXPONENT_SHIFT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_EXPONENT_SHIFT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_EXPONENT_SHIFT: the value named 'st exponent shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_DIFFERENCE, the machine performs the named control phase..

Line 200: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_DIFFERENCE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 202: `ST_EXPONENT_SHIFT: begin` This is the ST_EXPONENT_SHIFT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_SHIFT, the machine performs the named control phase..

Line 203: `shifted_difference_r <= exponent_difference_r >> difference_shift_r;` This makes shifted_difference_r, the value named 'shifted difference r' in this block, equal to 'exponent_difference_r >> difference_shift_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are exponent_difference_r: the value named 'exponent difference r' in this block; difference_shift_r: the value named 'difference shift r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_SHIFT, the machine performs the named control phase..

Line 204: `state <= ST_EXPONENT_CLAMP;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_EXPONENT_CLAMP'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_EXPONENT_CLAMP: the value named 'st exponent clamp' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_SHIFT, the machine performs the named control phase..

Line 205: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_SHIFT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 207: `ST_EXPONENT_CLAMP: begin` This is the ST_EXPONENT_CLAMP case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_CLAMP, the machine performs the named control phase..

Line 208: `exponent_r <= shifted_difference_r > 15` This statement continues through source line 209; the explanation uses the complete expression rather than only the first visual fragment. This makes exponent_r, the value named 'exponent r' in this block, equal to 'shifted_difference_r > 15 ? 5'd15 : shifted_difference_r[4:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are shifted_difference_r: the value named 'shifted difference r' in this block; d15: the value named 'd15' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_CLAMP, the machine performs the named control phase..

Line 209: `? 5'd15 : shifted_difference_r[4:0];` This is continuation line 2 of the 2-line logical statement that begins at source line 208. Its exact fragment is '? 5'd15 : shifted_difference_r[4:0];'. The important operands are d15: the value named 'd15' in this block; shifted_difference_r: the value named 'shifted difference r' in this block. Read together, the complete operation is: This makes exponent_r, the value named 'exponent r' in this block, equal to 'shifted_difference_r > 15 ? 5'd15 : shifted_difference_r[4:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are shifted_difference_r: the value named 'shifted difference r' in this block; d15: the value named 'd15' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_CLAMP, the machine performs the named control phase..

Line 210: `state <= ST_EXPONENT_QUANTIZE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_EXPONENT_QUANTIZE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_EXPONENT_QUANTIZE: the value named 'st exponent quantize' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_CLAMP, the machine performs the named control phase..

Line 211: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_CLAMP, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 213: `ST_EXPONENT_QUANTIZE: begin` This is the ST_EXPONENT_QUANTIZE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_QUANTIZE, the machine performs the named control phase..

Line 214: `unnormalized_r <= 16'd1 << (15 - exponent_r);` This makes unnormalized_r, the value named 'unnormalized r' in this block, equal to '16'd1 << (15 - exponent_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d1: the value named 'd1' in this block; exponent_r: the value named 'exponent r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_QUANTIZE, the machine performs the named control phase..

Line 215: `state <= ST_EXPONENT_ACCUMULATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_EXPONENT_ACCUMULATE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_EXPONENT_ACCUMULATE: the value named 'st exponent accumulate' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_QUANTIZE, the machine performs the named control phase..

Line 216: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_QUANTIZE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 218: `ST_EXPONENT_ACCUMULATE: begin` This is the ST_EXPONENT_ACCUMULATE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase..

Line 219: `exponent_mem[exponent_index[7:0]] <= unnormalized_r;` This makes exponent_mem[exponent_index[7:0]], local FPGA storage addressed by surrounding index logic, equal to 'unnormalized_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are unnormalized_r: the value named 'unnormalized r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase..

Line 220: `denominator_r <= denominator_r + {8'd0, unnormalized_r};` This makes denominator_r, the value named 'denominator r' in this block, equal to 'denominator_r + {8'd0, unnormalized_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are denominator_r: the value named 'denominator r' in this block; d0: the value named 'd0' in this block; unnormalized_r: the value named 'unnormalized r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase..

Line 221: `if (exponent_remaining_r == 9'd1) begin` This condition selects the following hardware branch only when 'exponent_remaining_r == 9'd1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are exponent_remaining_r: the value named 'exponent remaining r' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase..

Line 222: `state <= ST_RECIPROCAL_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_RECIPROCAL_START'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_RECIPROCAL_START: the value named 'st reciprocal start' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase..

Line 223: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 224: `exponent_index <= exponent_index + 1'b1;` This makes exponent_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'exponent_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are exponent_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers

only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase..

Line 225: `exponent_remaining_r <= exponent_remaining_r - 1'b1;` This makes exponent_remaining_r, the value named 'exponent remaining r' in this block, equal to 'exponent_remaining_r - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are exponent_remaining_r: the value named 'exponent remaining r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase..

Line 226: `state <= ST_EXPONENT_READ;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_EXPONENT_READ'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_EXPONENT_READ: the value named 'st exponent read' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase..

Line 227: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 228: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_EXPONENT_ACCUMULATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 230: `ST_RECIPROCAL_START: begin` This is the ST_RECIPROCAL_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_START, the machine performs the named control phase..

Line 231: `if (divider_input_ready) state <= ST_RECIPROCAL_WAIT;` This condition selects the following hardware branch only when 'divider_input_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are divider_input_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_START, the machine performs the named control phase..

Line 232: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_START, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 234: `ST_RECIPROCAL_WAIT: begin` This is the ST_RECIPROCAL_WAIT case branch, which waits for both command completion and the declared output count. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 235: `if (divider_output_valid) begin` This condition selects the following hardware branch only when 'divider_output_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are divider_output_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 236: `reciprocal_q31_r <= divider_quotient;` This makes reciprocal_q31_r, the value named 'reciprocal q31 r' in this block, equal to 'divider_quotient'. It updates after the active clock edge together with other nonblocking assignments. The important operands are divider_quotient: the value named 'divider quotient' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 237: `output_index_r <= 9'd0;` This makes output_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 238: `output_remaining_r <= item_count_r;` This makes output_remaining_r, the number of command result beats still expected, equal to 'item_count_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are item_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 239: `config_error <= config_error || divider_zero;` This makes config_error, the value named 'config error' in this block, equal to 'config_error || divider_zero'. It updates after the active clock edge together with other nonblocking assignments. The important operands are config_error: the value named 'config error' in this block; divider_zero: the value named 'divider zero' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 240: `state <= ST_OUTPUT_READ;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_READ'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_READ: the value named 'st output read' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count..

Line 241: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 242: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RECIPROCAL_WAIT, the machine waits for both command completion and the declared output count.; it has no arithmetic value of its own.

Line 244: `ST_OUTPUT_READ: begin` This is the ST_OUTPUT_READ case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure..

Line 245: `output_exponent_ram_r <= exponent_mem[output_index_r[7:0]];` This makes output_exponent_ram_r, the value named 'output exponent ram r' in this block, equal to 'exponent_mem[output_index_r[7:0]]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are exponent_mem: local FPGA storage addressed by surrounding index logic; output_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure..

Line 246: `state <= ST_OUTPUT_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_LATCH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_LATCH: the value named 'st output latch' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure..

Line 247: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_READ, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 249: `ST_OUTPUT_LATCH: begin` This is the ST_OUTPUT_LATCH case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure..

Line 250: `output_exponent_r <= output_exponent_ram_r;` This makes output_exponent_r, the value named ’output exponent r’ in this block, equal to ‘output_exponent_ram_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_exponent_ram_r: the value named ’output exponent ram r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure..

Line 251: `state <= ST_OUTPUT_OPERANDS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_OPERANDS‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_OPERANDS: the value named ’st output operands’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure..

Line 252: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_LATCH, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 254: `ST_OUTPUT_OPERANDS: begin` This is the ST_OUTPUT_OPERANDS case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 255: `multiply_exponent_r <= output_exponent_r;` This makes multiply_exponent_r, the value named ’multiply exponent r’ in this block, equal to ‘output_exponent_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_exponent_r: the value named ’output exponent r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 256: `multiply_reciprocal_r <= reciprocal_q31_r;` This makes multiply_reciprocal_r, the value named ’multiply reciprocal r’ in this block, equal to ‘reciprocal_q31_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are reciprocal_q31_r: the value named ’reciprocal q31 r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 257: `state <= ST_OUTPUT_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_MULTIPLY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_MULTIPLY: the value named ’st output multiply’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_OPERANDS, the machine presents computed results under valid/ready backpressure..

Line 258: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_OPERANDS, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 260: `ST_OUTPUT_MULTIPLY: begin` This is the ST_OUTPUT_MULTIPLY case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 261: `probability_low_product_r <= probability_low_product;` This makes probability_low_product_r, the value named 'probability low product r' in this block, equal to 'probability_low_product'. It updates after the active clock edge together with other nonblocking assignments. The important operands are probability_low_product: the value named 'probability low product' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 262: `probability_high_product_r <= probability_high_product;` This makes probability_high_product_r, the value named 'probability high product r' in this block, equal to 'probability_high_product'. It updates after the active clock edge together with other nonblocking assignments. The important operands are probability_high_product: the value named 'probability high product' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 263: `state <= ST_OUTPUT_PRODUCT_ADD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_PRODUCT_ADD'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_PRODUCT_ADD: the value named 'st output product add' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_MULTIPLY, the machine presents computed results under valid/ready backpressure..

Line 264: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_MULTIPLY, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 266: `ST_OUTPUT_PRODUCT_ADD: begin` This is the ST_OUTPUT_PRODUCT_ADD case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PRODUCT_ADD, the machine presents computed results under valid/ready backpressure..

Line 267: `probability_product_r <= probability_low_product_r` This statement continues through source line 268; the explanation uses the complete expression rather than only the first visual fragment. This makes probability_product_r, the value named 'probability product r' in this block, equal to 'probability_low_product_r + {probability_high_product_r[15:0], 16'd0}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are probability_low_product_r: the value named 'probability low product r' in this block; probability_high_product_r: the value named 'probability high product r' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PRODUCT_ADD, the machine presents computed results under valid/ready backpressure..

Line 268: `+ {probability_high_product_r[15:0], 16'd0};` This is continuation line 2 of the 2-line logical statement that begins at source line 267. Its exact fragment is '+ {probability_high_product_r[15:0], 16'd0};'. The important operands are probability_high_product_r: the value named 'probability high product r' in this block; d0: the value named 'd0' in this block. Read together, the complete operation is: This makes probability_product_r, the value named 'probability product r' in this block, equal to 'probability_low_product_r + {probability_high_product_r[15:0], 16'd0}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are probability_low_product_r: the value named 'probability low product r' in this block; probability_high_product_r: the value named 'probability high product r' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PRODUCT_ADD, the machine presents computed results under valid/ready backpressure..

Line 269: `state <= ST_OUTPUT_ROUND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_ROUND'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_ROUND: the value named 'st output round' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PRODUCT_ADD, the machine presents computed results under valid/ready backpressure..

Line 270: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PRODUCT_ADD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 272: `ST_OUTPUT_ROUND: begin` This is the ST_OUTPUT_ROUND case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_ROUND, the machine presents computed results under valid/ready backpressure..

Line 273: `probability_q15 <= probability_product_r[31:16]` This statement continues through source line 274; the explanation uses the complete expression rather than only the first visual fragment. This makes probability_q15, the value named ’probability q15’ in this block, equal to ‘probability_product_r[31:16] + 16’(probability_product_r[15])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are probability_product_r: the value named ’probability product r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_ROUND, the machine presents computed results under valid/ready backpressure..

Line 274: `+ 16'(probability_product_r[15]);` This is continuation line 2 of the 2-line logical statement that begins at source line 273. Its exact fragment is ‘+ 16’(probability_product_r[15]);‘. The important operands are probability_product_r: the value named ’probability product r’ in this block. Read together, the complete operation is: This makes probability_q15, the value named ’probability q15’ in this block, equal to ‘probability_product_r[31:16] + 16’(probability_product_r[15])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are probability_product_r: the value named ’probability product r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_ROUND, the machine presents computed results under valid/ready backpressure..

Line 275: `probability_last_r <= output_remaining_r == 9'd1;` This makes probability_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘output_remaining_r == 9’d1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_remaining_r: the number of command result beats still expected; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_ROUND, the machine presents computed results under valid/ready backpressure..

Line 276: `state <= ST_OUTPUT_HOLD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_HOLD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_HOLD: the value named ’st output hold’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_ROUND, the machine presents computed results under valid/ready backpressure..

Line 277: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_ROUND, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 279: `ST_OUTPUT_HOLD: begin` This is the ST_OUTPUT_HOLD case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 280: `if (probability_ready) begin` This condition selects the following hardware branch only when ‘probability_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are probability_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 281: `if (probability_last) begin` This condition selects the following hardware branch only when ‘probability_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are probability_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 282: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 283: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 284: `output_index_r <= output_index_r + 1'b1;` This makes output_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index_r + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_index_r: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 285: `output_remaining_r <= output_remaining_r - 1'b1;` This makes output_remaining_r, the number of command result beats still expected, equal to ‘output_remaining_r - 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_remaining_r: the number of command result beats still expected; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 286: `state <= ST_OUTPUT_READ;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_READ‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_READ: the value named ’st output read’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 287: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 288: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 289: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 291: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘state <= ST_IDLE;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure..

Line 292: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_HOLD, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 293: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 294: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 296: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 297: `if (MAX_ITEMS < 1 || MAX_ITEMS > 256) $fatal(1, "MAX_ITEMS must be in [1,256]");` This condition selects the following hardware branch only when ‘MAX_ITEMS < 1 || MAX_ITEMS > 256) $fatal(1, "MAX_ITEMS must be in [1,256]"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are MAX_ITEMS: the value named ’max items’ in this block; fatal: the value named ’fatal’ in this block; must: the value named ’must’ in this block. Context: the module’s structural declarations.

Line 298: `if (SCORE_BITS != 24) $fatal(1, "neuraedge_log_softmax currently requires SCORE_BITS=24");` This condition selects the following hardware branch only when ‘SCORE_BITS != 24) $fatal(1, "neuraedge_log_softmax currently requires SCORE_BITS=24"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are SCORE_BITS: the value named ’score bits’ in this block; fatal: the value named ’fatal’ in this block; neuraedge_log_softmax: the value named ’neuraedge log softmax’ in this block. Context: the module’s structural declarations.

Line 299: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 300: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 302: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that implements power-of-two integer softmax normalization without floating-point exponent hardware.

15 rtl/neuraedge_mac2_array_probe.sv

Role in the complete system: instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior.

15.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_mac2_array_probe into physical nets and primitives. The source declares 3 input-port lines, 1 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls.

Connections to other blocks This source instantiates neuraedge_mac2_u8 as u_mac. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

15.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  // Synthesis-only probe used to prove packed-MAC density on the target FPGA.
5  module neuraedge_mac2_array_probe #(
6      parameter int PAIRS = 128
7  ) (
8      input  wire        clk,
9      input  wire        rst_n,
10     input  wire        step,
11     output logic [31:0] checksum
12 );
13     logic [31:0] stimulus [0:PAIRS-1];
14     wire [15:0] product_lo [0:PAIRS-1];
15     wire [15:0] product_hi [0:PAIRS-1];
16     wire product_valid [0:PAIRS-1];
17
18     for (genvar pair_idx = 0; pair_idx < PAIRS; pair_idx++) begin : g_pair
19         neuraedge_mac2_u8 u_mac (
20             .clk(clk),
21             .rst_n(rst_n),
22             .enable(step),
23             .activation(stimulus[pair_idx][7:0]),
24             .weight_lo(stimulus[pair_idx][15:8]),
25             .weight_hi(stimulus[pair_idx][23:16]),
26             .product_valid(product_valid[pair_idx]),
27             .product_lo(product_lo[pair_idx]),
28             .product_hi(product_hi[pair_idx])
29         );
30
31         always_ff @(posedge clk) begin
32             if (!rst_n) begin
33                 stimulus[pair_idx] <= 32'h6d2b79f5 ^ (pair_idx * 32'h9e3779b9);
34             end else if (step) begin
35                 stimulus[pair_idx] <= {stimulus[pair_idx][30:0],
36                     stimulus[pair_idx][31] ^ stimulus[pair_idx][21]
37                     ^ stimulus[pair_idx][1] ^ stimulus[pair_idx][0]};
38             end
39         end
40     end
41
42     always_comb begin
43         checksum = 32'd0;
44         for (int pair_idx = 0; pair_idx < PAIRS; pair_idx++) begin
45             checksum = checksum ^ {product_hi[pair_idx], product_lo[pair_idx]};
46         end
47     end
48 endmodule
```

```
49
50 `default_nettype wire
```

15.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `// Synthesis-only probe used to prove packed-MAC density on the target FPGA.` Design intent: Synthesis-only probe used to prove packed-MAC density on the target FPGA. This comment anchors the surrounding logic in the block that instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior.

Line 5: `module neuraedge_mac2_array_probe #(` This opens neuraedge_mac2_array_probe, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior.

Line 6: `parameter int PAIRS = 128` This statement continues through source line 12; the explanation uses the complete expression rather than only the first visual fragment. This fixes PAIRS to 128) (input wire clk, input wire rst_n, input wire step, output logic [31:0] checksum) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; PAIRS denotes the value named 'pairs' in this block.

Line 7: `) (` This is continuation line 2 of the 7-line logical statement that begins at source line 6. Its exact fragment is ') ('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes PAIRS to 128) (input wire clk, input wire rst_n, input wire step, output logic [31:0] checksum) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; PAIRS denotes the value named 'pairs' in this block.

Line 8: `input wire clk,` This is continuation line 3 of the 7-line logical statement that begins at source line 6. Its exact fragment is 'input wire clk,'. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes PAIRS to 128) (input wire clk, input wire rst_n, input wire step, output logic [31:0] checksum) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; PAIRS denotes the value named 'pairs' in this block.

Line 9: `input wire rst_n,` This is continuation line 4 of the 7-line logical statement that begins at source line 6. Its exact fragment is 'input wire rst_n,'. The important operands are input: the value named 'input' in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes PAIRS to 128) (input wire clk, input wire rst_n, input wire step, output logic [31:0] checksum) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; PAIRS denotes the value named 'pairs' in this block.

Line 10: `input wire step,` This is continuation line 5 of the 7-line logical statement that begins at source line 6. Its exact fragment is 'input wire step,'. The important operands are input: the value named 'input' in this block; step: the value named 'step' in this block. Read together, the complete operation is: This fixes PAIRS to 128) (input wire clk, input wire rst_n, input wire step, output logic [31:0] checksum) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; PAIRS denotes the value named 'pairs' in this block.

Line 11: `output logic [31:0] checksum` This is continuation line 6 of the 7-line logical statement that begins at source line 6. Its exact fragment is 'output logic [31:0] checksum'. The important operands are output: the value named 'output' in this block; checksum: the value named 'checksum' in this block. Read together, the complete operation is: This fixes PAIRS to 128) (input wire clk, input wire rst_n, input wire step, output logic [31:0] checksum) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; PAIRS denotes the value named 'pairs' in this block.

Line 12: `);` This is continuation line 7 of the 7-line logical statement that begins at source line 6. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes PAIRS to 128) (input wire clk, input wire rst_n, input wire step, output logic [31:0] checksum) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; PAIRS denotes the value named 'pairs' in this block.

Line 13: `logic [31:0] stimulus [0:PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stimulus is the value named 'stimulus' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `wire [15:0] product_lo [0:PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_lo is the value named 'product lo' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `wire [15:0] product_hi [0:PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_hi is the value named 'product hi' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `wire product_valid [0:PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `for (genvar pair_idx = 0; pair_idx < PAIRS; pair_idx++) begin : g_pair` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 19: `neuraedge_mac2_u8 u_mac (` This instantiates module neuraedge_mac2_u8 as u_mac. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 20: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 21: `.rst_n(rst_n),` This connects the child instance's named port 'rst_n' to parent expression 'rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 22: `.enable(step),` This connects the child instance's named port 'enable' to parent expression 'step'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'step' in this block.

Line 23: `.activation(stimulus[pair_idx][7:0]),` This connects the child instance's named port 'activation' to parent expression 'stimulus[pair_idx][7:0]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'stimulus' in this block.

Line 24: `.weight_lo(stimulus[pair_idx][15:8]),` This connects the child instance's named port 'weight_lo' to parent expression 'stimulus[pair_idx][15:8]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'stimulus' in this block.

Line 25: `.weight_hi(stimulus[pair_idx][23:16]),` This connects the child instance's named port 'weight_hi' to parent expression 'stimulus[pair_idx][23:16]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'stimulus' in this block.

Line 26: `.product_valid(product_valid[pair_idx]),` This connects the child instance's named port 'product_valid' to parent expression 'product_valid[pair_idx]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 27: `.product_lo(product_lo[pair_idx]),` This connects the child instance's named port 'product_lo' to parent expression 'product_lo[pair_idx]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'product lo' in this block.

Line 28: `.product_hi(product_hi[pair_idx])` This connects the child instance's named port 'product_hi' to parent expression 'product_hi[pair_idx]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'product hi' in this block.

Line 29: `);` Within the module's structural declarations, this exact statement contributes to the block that instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior. The expression is a literal or structural operator and has no hidden software state.

Line 31: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 32: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 33: `stimulus[pair_idx] <= 32'h6d2b79f5 ^ (pair_idx * 32'h9e3779b9);` This makes stimulus[pair_idx], the value named 'stimulus' in this block, equal to '32'h6d2b79f5 ^ (pair_idx * 32'h9e3779b9)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are h6d2b79f5: the value named 'h6d2b79f5' in this block; pair_idx: the value named 'pair idx' in this block; h9e3779b9: the value named 'h9e3779b9' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 34: `end else if (step) begin` This condition selects the following hardware branch only when 'step' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are step: the value named 'step' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 35: `stimulus[pair_idx] <= {stimulus[pair_idx][30:0],` This statement continues through source line 37; the explanation uses the complete expression rather than only the first visual fragment. This makes stimulus[pair_idx], the value named 'stimulus' in this block, equal to '{stimulus[pair_idx][30:0], stimulus[pair_idx][31] ^ stimulus[pair_idx][21] ^ stimulus[pair_idx][1] ^ stimulus[pair_idx][0]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stimulus: the value named 'stimulus' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 36: `stimulus[pair_idx][31] ^ stimulus[pair_idx][21]` This is continuation line 2 of the 3-line logical statement that begins at source line 35. Its exact fragment is 'stimulus[pair_idx][31] ^ stimulus[pair_idx][21]'. The important operands are stimulus: the value named 'stimulus' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes stimulus[pair_idx], the value named 'stimulus' in this block, equal to '{stimulus[pair_idx][30:0], stimulus[pair_idx][31] ^ stimulus[pair_idx][21] ^ stimulus[pair_idx][1] ^ stimulus[pair_idx][0]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stimulus: the value named 'stimulus' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 37: `^ stimulus[pair_idx][1] ^ stimulus[pair_idx][0]};` This is continuation line 3 of the 3-line logical statement that begins at source line 35. Its exact fragment is '^ stimulus[pair_idx][1] ^ stimulus[pair_idx][0]};'. The important operands are stimulus: the value named 'stimulus' in this block; pair_idx: the value named 'pair idx' in

this block. Read together, the complete operation is: This makes stimulus[pair_idx], the value named 'stimulus' in this block, equal to '{stimulus[pair_idx][30:0], stimulus[pair_idx][31] ^ stimulus[pair_idx][21] ^ stimulus[pair_idx][1] ^ stimulus[pair_idx][0]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stimulus: the value named 'stimulus' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 38: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 39: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 40: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 42: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 43: `checksum = 32'd0;` This makes checksum, the value named 'checksum' in this block, equal to '32'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 44: `for (int pair_idx = 0; pair_idx < PAIRS; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 45: `checksum = checksum ^ {product_hi[pair_idx], product_lo[pair_idx]};` This makes checksum, the value named 'checksum' in this block, equal to 'checksum ^ {product_hi[pair_idx], product_lo[pair_idx]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are checksum: the value named 'checksum' in this block; product_hi: the value named 'product hi' in this block; pair_idx: the value named 'pair idx' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 46: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 47: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 48: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 50: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior.

16 rtl/neuraedge_mac2_u8.sv

Role in the complete system: packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

16.1 Beginner architecture map

Dataflow responsibility Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_mac2_u8 into physical nets and primitives. The source declares 6 input-port lines, 3 output-port lines, 1 clocked process(es), and 0 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls.

Connections to other blocks This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

16.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  // Computes two exact unsigned 8-bit products in one 25 x 18 multiplier.
5  // The upper weight starts at bit 17, leaving bit 16 as a guard between the
6  // two 16-bit products. Since 255 * 255 is smaller than 2^16, the low product
7  // can never carry into the high product field.
8  module neuraedge_mac2_u8 (
9      input  wire         clk,
10     input  wire         rst_n,
11     input  wire         enable,
12     input  wire [7:0]   activation,
13     input  wire [7:0]   weight_lo,
14     input  wire [7:0]   weight_hi,
15     output wire         product_valid,
16     output logic [15:0] product_lo,
17     output logic [15:0] product_hi
18 );
19     logic signed [24:0] packed_weight_r;
20     logic signed [17:0] extended_activation_r;
21     logic signed [47:0] unsigned_correction_r;
22     (* dont_touch = "yes" *) logic input_valid_r;
23     (* dont_touch = "yes" *) logic dsp_product_valid_r;
24     (* dont_touch = "yes" *) logic product_valid_r;
25     /* verilator lint_off UNUSEDSIGNAL */
26     (* use_dsp = "yes" *) logic signed [47:0] packed_product;
27     /* verilator lint_on UNUSEDSIGNAL */
28
29     assign product_valid = product_valid_r;
30
31     always_ff @(posedge clk) begin
32         if (!rst_n) begin
33             packed_weight_r <= '0;
34             extended_activation_r <= '0;
35             unsigned_correction_r <= '0;
36             input_valid_r <= 1'b0;
37             dsp_product_valid_r <= 1'b0;
38             packed_product <= '0;
39             product_valid_r <= 1'b0;
40             product_lo <= 16'd0;
41             product_hi <= 16'd0;
42         end else begin
43             input_valid_r <= enable;
44             dsp_product_valid_r <= input_valid_r;
45             product_valid_r <= dsp_product_valid_r;
46             // The tile holds operands stable outside a valid issue. Sampling
47             // every cycle keeps the DSP A/B input registers enabled locally;
48             // input_valid_r still gates product retirement, so invalid cycles
```

```
            // cannot change any architectural result.
            packed_weight_r <= $signed({weight_hi, 9'b0, weight_lo});
            extended_activation_r <= $signed({10'b0, activation});
            unsigned_correction_r <= weight_hi[7]
                ? $signed({15'b0, activation, 25'b0})
                : 48'sd0;
            // Invalid products are discarded by the valid pipeline. Updating
            // the DSP and unpack registers every clock makes all DSP48E1
            // A/B/M/P clock enables local constants instead of a 64-DSP
            // control tree spanning the tensor tile.
            packed_product <= (packed_weight_r * extended_activation_r)
                + unsigned_correction_r;
            product_lo <= packed_product[15:0];
            product_hi <= packed_product[32:17];
        end
    end
endmodule

`default_nettype wire
```

16.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `// Computes two exact unsigned 8-bit products in one 25 x 18 multiplier.` Design intent: Computes two exact unsigned 8-bit products in one 25 x 18 multiplier. This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 5: `// The upper weight starts at bit 17, leaving bit 16 as a guard between the` Design intent: The upper weight starts at bit 17, leaving bit 16 as a guard between the This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 6: `// two 16-bit products. Since 255 * 255 is smaller than 2^16, the low product` Design intent: two 16-bit products. Since 255 * 255 is smaller than 2^16, the low product This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 7: `// can never carry into the high product field.` Design intent: can never carry into the high product field. This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 8: `module neuraedge_mac2_u8 (` This opens neuraedge_mac2_u8, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 9: `input wire clk,` This declares clk as a input port with width one bit. It carries the clock that makes all registers advance together across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 10: `input wire rst_n,` This declares rst_n as a input port with width one bit. It carries the active-low synchronous reset across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 11: `input wire enable,` This declares enable as a input port with width one bit. It carries the value named 'enable' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 12: `input wire [7:0] activation,` This declares activation as a input port with width [7:0]. It carries an input feature value consumed by a neuron or multiply-accumulate operation across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 13: `input wire [7:0] weight_lo,` This declares weight_lo as a input port with width [7:0]. It carries a learned coefficient multiplied by an activation across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 14: `input wire [7:0] weight_hi,` This declares weight_hi as a input port with width [7:0]. It carries a learned coefficient multiplied by an activation across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 15: `output wire product_valid,` This declares product_valid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 16: `output logic [15:0] product_lo,` This declares product_lo as a output port with width [15:0]. It carries the value named 'product lo' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 17: `output logic [15:0] product_hi` This declares product_hi as a output port with width [15:0]. It carries the value named 'product hi' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 18: `);` Within the module's structural declarations, this exact statement contributes to the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication. The expression is a literal or structural operator and has no hidden software state.

Line 19: `logic signed [24:0] packed_weight_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, packed_weight_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic signed [17:0] extended_activation_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, extended_activation_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic signed [47:0] unsigned_correction_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, unsigned_correction_r is the value named 'unsigned correction r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `(* dont_touch = "yes" *) logic input_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; input_valid_r: the producer's statement that its current payload is meaningful.

Line 23: `(* dont_touch = "yes" *) logic dsp_product_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; dsp_product_valid_r: the producer's statement that its current payload is meaningful.

Line 24: `(* dont_touch = "yes" *) logic product_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; product_valid_r: the producer's statement that its current payload is meaningful.

Line 25: `/* verilator lint_off UNUSEDSIGNAL */` Within the module's structural declarations, this exact statement contributes to the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication. The important operands are verilator: the value named 'verilator' in this block; lint_off: the value named 'lint off' in this block; UNUSEDSIGNAL: the value named 'unusedsignal' in this block.

Line 26: `(* use_dsp = "yes" *) logic signed [47:0] packed_product;` Within the module's structural declarations, this exact statement contributes to the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; packed_product: the value named 'packed product' in this block.

Line 27: `/* verilator lint_on UNUSEDSIGNAL */` Within the module's structural declarations, this exact statement contributes to the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication. The important operands are verilator: the value named 'verilator' in this block; lint_on: the value named 'lint on' in this block; UNUSEDSIGNAL: the value named 'unusedsignal' in this block.

Line 29: `assign product_valid = product_valid_r;` This makes product_valid, the producer's statement that its current payload is meaningful, equal to ‘product_valid_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are product_valid_r: the producer's statement that its current payload is meaningful. Context: the module's structural declarations.

Line 31: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with ‘<=‘ inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 32: `if (!rst_n) begin` This is the reset branch for ‘(!rst_n‘. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 33: `packed_weight_r <= '0;` This makes packed_weight_r, a learned coefficient multiplied by an activation, equal to ‘’0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 34: `extended_activation_r <= '0;` This makes extended_activation_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘’0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 35: `unsigned_correction_r <= '0;` This makes unsigned_correction_r, the value named 'unsigned correction r' in this block, equal to ‘’0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 36: `input_valid_r <= 1'b0;` This makes input_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 37: `dsp_product_valid_r <= 1'b0;` This makes dsp_product_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 38: `packed_product <= '0;` This makes packed_product, the value named 'packed product' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 39: `product_valid_r <= 1'b0;` This makes product_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 40: `product_lo <= 16'd0;` This makes product_lo, the value named 'product lo' in this block, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 41: `product_hi <= 16'd0;` This makes product_hi, the value named 'product hi' in this block, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 42: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 43: `input_valid_r <= enable;` This makes input_valid_r, the producer's statement that its current payload is meaningful, equal to 'enable'. It updates after the active clock edge together with other nonblocking assignments. The important operands are enable: the value named 'enable' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 44: `dsp_product_valid_r <= input_valid_r;` This makes dsp_product_valid_r, the producer's statement that its current payload is meaningful, equal to 'input_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 45: `product_valid_r <= dsp_product_valid_r;` This makes product_valid_r, the producer's statement that its current payload is meaningful, equal to 'dsp_product_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dsp_product_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 46: `// The tile holds operands stable outside a valid issue. Sampling` Design intent: The tile holds operands stable outside a valid issue. Sampling This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 47: `// every cycle keeps the DSP A/B input registers enabled locally;` Design intent: every cycle keeps the DSP A/B input registers enabled locally; This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 48: `// input_valid_r still gates product retirement, so invalid cycles` Design intent: input_valid_r still gates product retirement, so invalid cycles This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 49: `// cannot change any architectural result.` Design intent: cannot change any architectural result. This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 50: `packed_weight_r <= $signed({weight_hi, 9'b0, weight_lo});` This makes packed_weight_r, a learned coefficient multiplied by an activation, equal to '$signed({weight_hi, 9'b0, weight_lo})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_hi: a learned coefficient multiplied by an activation; b0: the value named 'b0' in this block; weight_lo: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 51: `extended_activation_r <= $signed({10'b0, activation});` This makes extended_activation_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '$signed({10'b0, activation})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block; activation: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge.

Line 52: `unsigned_correction_r <= weight_hi[7]` This statement continues through source line 54; the explanation uses the complete expression rather than only the first visual fragment. This makes unsigned_correction_r, the value named 'unsigned correction r' in this block, equal to 'weight_hi[7] ? $signed({15'b0, activation, 25'b0}) : 48'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_hi: a learned coefficient multiplied by an activation; b0: the value named 'b0' in this block; activation: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge.

Line 53: `? $signed({15'b0, activation, 25'b0})` This is continuation line 2 of the 3-line logical statement that begins at source line 52. Its exact fragment is '? $signed({15'b0, activation, 25'b0})'. The important operands are b0: the value named 'b0' in this block; activation: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This makes unsigned_correction_r, the value named 'unsigned correction r' in this block, equal to 'weight_hi[7] ? $signed({15'b0, activation, 25'b0}) : 48'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_hi: a learned coefficient multiplied by an activation; b0: the value named 'b0' in this block; activation: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge.

Line 54: `: 48'sd0;` This is continuation line 3 of the 3-line logical statement that begins at source line 52. Its exact fragment is ': 48'sd0;'. The important operands are sd0: the value named 'sd0' in this block. Read together, the complete operation is: This makes unsigned_correction_r, the value named 'unsigned correction r' in this block, equal to 'weight_hi[7] ? $signed({15'b0, activation, 25'b0}) : 48'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_hi: a learned coefficient multiplied by an activation; b0: the value named 'b0' in this block; activation: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge.

Line 55: `// Invalid products are discarded by the valid pipeline. Updating` Design intent: Invalid products are discarded by the valid pipeline. Updating This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 56: `// the DSP and unpack registers every clock makes all DSP48E1` Design intent: the DSP and unpack registers every clock makes all DSP48E1 This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 57: `// A/B/M/P clock enables local constants instead of a 64-DSP` Design intent: A/B/M/P clock enables local constants instead of a 64-DSP This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 58: `// control tree spanning the tensor tile.` Design intent: control tree spanning the tensor tile. This comment anchors the surrounding logic in the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

Line 59: `packed_product <= (packed_weight_r * extended_activation_r)` This statement continues through source line 60; the explanation uses the complete expression rather than only the first visual fragment. This makes packed_product, the value named 'packed product' in this block, equal to '(packed_weight_r * extended_activation_r) + unsigned_correction_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are packed_weight_r: a learned coefficient multiplied by an activation; extended_activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; unsigned_correction_r: the value named 'unsigned correction r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 60: `+ unsigned_correction_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 59. Its exact fragment is '+ unsigned_correction_r;'. The important operands are unsigned_correction_r: the value named 'unsigned correction r' in this block. Read together, the complete operation is: This makes packed_product, the value named 'packed product' in this block, equal to '(packed_weight_r * extended_activation_r) + unsigned_correction_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are packed_weight_r: a learned coefficient multiplied by an activation; extended_activation_r: an input feature value consumed by a neuron or multiply-accumulate operation; unsigned_correction_r: the value named 'unsigned correction r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 61: `product_lo <= packed_product[15:0];` This makes product_lo, the value named 'product lo' in this block, equal to 'packed_product[15:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are packed_product: the value named 'packed product' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 62: `product_hi <= packed_product[32:17];` This makes product_hi, the value named 'product hi' in this block, equal to 'packed_product[32:17]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are packed_product: the value named 'packed product' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 63: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 64: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 65: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 67: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication.

17 rtl/neuraedge_pynq_axi_top.sv

Role in the complete system: provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

17.1 Beginner architecture map

Dataflow responsibility This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_pynq_axi_top into physical nets and primitives. The source declares 19 input-port lines, 14 output-port lines, 3 clocked process(es), and 4 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls.

Connections to other blocks This source instantiates unique as case, neuraedge_descriptor_sequencer as u_descriptor_sequenc neuraedge_universal_core as u_core, end as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

17.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_pynq_axi_top #(
5      parameter int AXI_ADDR_BITS = 8
6  ) (
7      (* X_INTERFACE_INFO = "xilinx.com:signal:clock:1.0 aclk CLK" *)
8      (* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME aclk, ASSOCIATED_BUSIF S_AXI:S_AXIS:M_AXIS, ASSOCIATED_RESET aresetn, FREQ_HZ 200000000" *)
9      input  wire aclk,
10     (* X_INTERFACE_INFO = "xilinx.com:signal:reset:1.0 aresetn RST" *)
11     (* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME aresetn, POLARITY ACTIVE_LOW" *)
12     input  wire aresetn,
13
14     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI AWADDR" *)
15     (* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME S_AXI, PROTOCOL AXI4LITE, DATA_WIDTH 32, ADDR_WIDTH 8, FREQ_HZ 200000000, HAS_BURST 0,
        HAS_LOCK 0, HAS_PROT 0, HAS_CACHE 0, HAS_QOS 0, HAS_REGION 0, SUPPORTS_NARROW_BURST 0" *)
16     input  wire [AXI_ADDR_BITS-1:0] s_axi_awaddr,
17     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI AWVALID" *)
18     input  wire s_axi_awvalid,
19     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI AWREADY" *)
20     output logic s_axi_awready,
21     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI WDATA" *)
22     input  wire [31:0] s_axi_wdata,
23     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI WSTRB" *)
24     input  wire [3:0] s_axi_wstrb,
25     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI WVALID" *)
26     input  wire s_axi_wvalid,
27     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI WREADY" *)
28     output logic s_axi_wready,
29     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI BRESP" *)
30     output logic [1:0] s_axi_bresp,
31     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI BVALID" *)
32     output logic s_axi_bvalid,
33     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI BREADY" *)
34     input  wire s_axi_bready,
35     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI ARADDR" *)
36     input  wire [AXI_ADDR_BITS-1:0] s_axi_araddr,
37     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI ARVALID" *)
38     input  wire s_axi_arvalid,
39     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI ARREADY" *)
40     output logic s_axi_arready,
41     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI RDATA" *)
42     output logic [31:0] s_axi_rdata,
43     (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI RRESP" *)
44     output logic [1:0] s_axi_rresp,
```

```
    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI RVALID" *)
    output logic s_axi_rvalid,
    (* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI RREADY" *)
    input  wire s_axi_rready,

    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TVALID" *)
    (* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME S_AXIS, TDATA_NUM_BYTES 8, HAS_TKEEP 1, HAS_TLAST 1, FREQ_HZ 200000000" *)
    input  wire s_axis_tvalid,
    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TREADY" *)
    output logic s_axis_tready,
    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TDATA" *)
    input  wire [63:0] s_axis_tdata,
    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TKEEP" *)
    input  wire [7:0] s_axis_tkeep,
    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TLAST" *)
    input  wire s_axis_tlast,
    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TVALID" *)
    (* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME M_AXIS, TDATA_NUM_BYTES 8, HAS_TKEEP 1, HAS_TLAST 1, FREQ_HZ 200000000" *)
    output logic m_axis_tvalid,
    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TREADY" *)
    input  wire m_axis_tready,
    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TDATA" *)
    output logic [63:0] m_axis_tdata,
    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TKEEP" *)
    output logic [7:0] m_axis_tkeep,
    (* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TLAST" *)
    output logic m_axis_tlast,
    (* X_INTERFACE_INFO = "xilinx.com:signal:interrupt:1.0 interrupt INTERRUPT" *)
    (* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME interrupt, SENSITIVITY LEVEL_HIGH" *)
    output logic interrupt
);
    localparam logic [7:0] ADDR_CONTROL = 8'h00;
    localparam logic [7:0] ADDR_STATUS = 8'h04;
    localparam logic [7:0] ADDR_ACTIVE_OPCODE = 8'h08;
    localparam logic [7:0] ADDR_COMMAND_COUNT = 8'h0c;
    localparam logic [7:0] ADDR_COMMAND0 = 8'h10;
    localparam logic [7:0] ADDR_COMMAND1 = 8'h14;
    localparam logic [7:0] ADDR_COMMAND2 = 8'h18;
    localparam logic [7:0] ADDR_COMMAND3 = 8'h1c;
    localparam logic [7:0] ADDR_INPUT_BEATS = 8'h20;
    localparam logic [7:0] ADDR_OUTPUT_BEATS = 8'h24;
    localparam logic [7:0] ADDR_TARGET_CLOCK = 8'h28;
    localparam logic [7:0] ADDR_CAPABILITIES = 8'h2c;
    localparam logic [7:0] ADDR_VERSION = 8'h30;
    localparam logic [7:0] ADDR_ID = 8'h34;
    localparam logic [7:0] ADDR_TENSOR_LOADED = 8'h38;
    localparam logic [7:0] ADDR_TENSOR_ACTIVE = 8'h3c;
    localparam logic [7:0] ADDR_TENSOR_CYCLES = 8'h40;
    localparam logic [7:0] ADDR_TENSOR_SKIPPED = 8'h44;
    localparam logic [7:0] ADDR_DENSE_MACS_LOW = 8'h48;
    localparam logic [7:0] ADDR_DENSE_MACS_HIGH = 8'h4c;
    localparam logic [7:0] ADDR_SPIKE_ADDS_LOW = 8'h50;
    localparam logic [7:0] ADDR_SPIKE_ADDS_HIGH = 8'h54;
    localparam logic [7:0] ADDR_LAYERNORM_CYCLES = 8'h58;
    localparam logic [7:0] ADDR_LAYERNORM_MEAN = 8'h5c;
    localparam logic [7:0] ADDR_LAYERNORM_VARIANCE = 8'h60;
    localparam logic [7:0] ADDR_LAYERNORM_SIGMA = 8'h64;
    localparam logic [7:0] ADDR_SOFTMAX_CYCLES = 8'h68;
    localparam logic [7:0] ADDR_SOFTMAX_MAXIMUM = 8'h6c;
    localparam logic [7:0] ADDR_SOFTMAX_DENOMINATOR = 8'h70;
    localparam logic [7:0] ADDR_LIF_SPIKES_LOW = 8'h74;
    localparam logic [7:0] ADDR_LIF_SPIKES_HIGH = 8'h78;
    localparam logic [7:0] ADDR_OPTIMIZER_UPDATES = 8'h7c;
    localparam logic [7:0] ADDR_LUT_ENTRIES = 8'h80;
    localparam logic [7:0] ADDR_LAYERNORM_RECIPROCAL = 8'h84;
    localparam logic [7:0] ADDR_SOFTMAX_RECIPROCAL = 8'h88;
    localparam logic [7:0] ADDR_PROGRAM_RECORDS = 8'h8c;
    localparam logic [7:0] ADDR_PROGRAM_OUTPUT_BEATS = 8'h90;
    localparam logic [7:0] ADDR_TRAINING_TILES = 8'h94;
    localparam logic [7:0] ADDR_TRAINING_CYCLES = 8'h98;

    logic [31:0] command_register [0:3];
    logic [127:0] submitted_command;
    logic command_pending;
    logic interrupt_enable;
    logic done_sticky;
    logic wrapper_error;
    logic program_start_r;
    logic program_clear_r;
    logic [31:0] program_output_beats_r;
    wire program_active;
    wire program_done;
    wire program_error;
    wire [31:0] program_records;

    logic aw_hold;
    logic [AXI_ADDR_BITS-1:0] awaddr_hold;
```

```
132     logic w_hold;
133     logic [31:0] wdata_hold;
134     logic [3:0] wstrb_hold;
135
136     wire core_command_ready;
137     wire core_command_done;
138     wire sequencer_core_command_valid;
139     wire [127:0] sequencer_core_command_data;
140     wire sequencer_core_s_valid;
141     wire [63:0] sequencer_core_s_data;
142     wire sequencer_core_s_last;
143     wire sequencer_core_m_ready;
144     wire core_busy;
145     wire core_error;
146     wire [3:0] core_active_opcode;
147     wire [31:0] core_command_count;
148     wire [31:0] core_input_beats;
149     wire [31:0] core_output_beats;
150     wire [11:0] core_tensor_loaded_activations;
151     wire [11:0] core_tensor_loaded_weights;
152     wire [11:0] core_tensor_active_issues;
153     wire [31:0] core_tensor_compute_cycles;
154     wire [31:0] core_tensor_skipped_issues;
155     wire [63:0] core_tensor_dense_macs;
156     wire [63:0] core_tensor_spike_adds;
157     wire [31:0] core_layernorm_cycles;
158     wire signed [31:0] core_layernorm_mean;
159     wire [31:0] core_layernorm_variance;
160     wire [31:0] core_layernorm_sigma;
161     wire [31:0] core_layernorm_reciprocal;
162     wire [31:0] core_softmax_cycles;
163     wire signed [23:0] core_softmax_maximum;
164     wire [23:0] core_softmax_denominator;
165     wire [31:0] core_softmax_reciprocal;
166     wire [63:0] core_lif_total_spikes;
167     wire [31:0] core_optimizer_updates;
168     wire [8:0] core_lut_loaded_entries;
169     wire [31:0] core_training_tiles;
170     wire [31:0] core_training_cycles;
171     logic core_s_valid;
172     wire core_s_ready;
173     wire core_raw_s_ready;
174     logic core_s_ready_r;
175     logic next_core_s_valid;
176     logic next_core_s_ready;
177     logic [63:0] core_s_data;
178     logic core_s_last;
179     logic [1:0] input_fifo_count;
180     logic input_fifo_write_pointer;
181     logic input_fifo_read_pointer;
182     logic [63:0] input_fifo_data [0:1];
183     logic input_fifo_last [0:1];
184     logic core_input_capture_valid;
185     logic [63:0] core_input_capture_data;
186     logic core_input_capture_last;
187     wire core_raw_m_valid;
188     wire core_raw_m_ready;
189     logic core_raw_m_ready_r;
190     wire [63:0] core_raw_m_data;
191     wire core_raw_m_last;
192     logic core_m_valid;
193     logic next_core_m_valid;
194     logic next_core_raw_m_ready;
195     wire core_m_ready;
196     wire sequencer_s_ready;
197     wire sequencer_m_valid;
198     wire [63:0] sequencer_m_data;
199     wire sequencer_m_last;
200     logic core_m_ready_r;
201     logic [63:0] core_m_data;
202     logic core_m_last;
203     logic core_output_capture_valid;
204     logic [63:0] core_output_capture_data;
205     logic core_output_capture_last;
206     logic [1:0] output_fifo_count;
207     logic output_fifo_write_pointer;
208     logic output_fifo_read_pointer;
209     logic [63:0] output_fifo_data [0:1];
210     logic output_fifo_last [0:1];
211     logic output_register_valid_r;
212     logic [63:0] output_register_data_r;
213     logic output_register_last_r;
214     logic [1:0] next_output_fifo_count;
215     logic next_core_output_capture_valid;
216     logic next_core_m_ready;
217
218     wire input_fifo_push = s_axis_tvalid && s_axis_tready;
```

```
    wire input_fifo_pop = input_fifo_count != 0 && !core_input_capture_valid;
    wire wrapper_input_ready = program_active ? sequencer_s_ready : core_s_ready;
    wire wrapper_output_valid = program_active ? sequencer_m_valid : core_m_valid;
    wire [63:0] wrapper_output_data = program_active
        ? sequencer_m_data : core_m_data;
    wire wrapper_output_last = program_active ? sequencer_m_last : core_m_last;
    wire core_output_capture_push = wrapper_output_valid && core_m_ready_r;
    wire core_output_capture_pop = core_output_capture_valid
        && output_fifo_count != 2;
    wire output_fifo_push = core_output_capture_pop;
    wire output_register_ready = !output_register_valid_r || m_axis_tready;
    wire output_fifo_pop = output_fifo_count != 0 && output_register_ready;
    wire selected_core_s_valid = program_active
        ? sequencer_core_s_valid : core_input_capture_valid;
    wire [63:0] selected_core_s_data = program_active
        ? sequencer_core_s_data : core_input_capture_data;
    wire selected_core_s_last = program_active
        ? sequencer_core_s_last : core_input_capture_last;
    wire core_s_boundary_push = selected_core_s_valid && core_s_ready_r;
    wire core_s_boundary_pop = core_s_valid && core_raw_s_ready;
    wire core_m_boundary_push = core_raw_m_valid && core_raw_m_ready_r;
    wire core_m_boundary_pop = core_m_valid && core_m_ready;
    assign core_s_ready = core_s_ready_r;
    assign core_m_ready = program_active
        ? sequencer_core_m_ready : core_m_ready_r;
    assign core_raw_m_ready = core_raw_m_ready_r;

    always_comb begin
        next_core_s_valid = core_s_valid;
        unique case ({core_s_boundary_push, core_s_boundary_pop})
            2'b10: next_core_s_valid = 1'b1;
            2'b01: next_core_s_valid = 1'b0;
            default: begin end
        endcase
        next_core_s_ready = !next_core_s_valid || core_raw_s_ready;
    end

    always_comb begin
        next_core_m_valid = core_m_valid;
        unique case ({core_m_boundary_push, core_m_boundary_pop})
            2'b10: next_core_m_valid = 1'b1;
            2'b01: next_core_m_valid = 1'b0;
            default: begin end
        endcase
        next_core_raw_m_ready = !next_core_m_valid || core_m_ready;
    end

    // Registered credit prevents a child engine's READY state from becoming
    // a payload clock-enable while retaining simultaneous consume/refill.
    always_ff @(posedge aclk) begin
        if (!aresetn) begin
            core_s_valid <= 1'b0;
            core_s_ready_r <= 1'b1;
            core_s_data <= 64'd0;
            core_s_last <= 1'b0;
        end else begin
            core_s_valid <= next_core_s_valid;
            core_s_ready_r <= next_core_s_ready;
            if (core_s_ready_r) begin
                core_s_data <= selected_core_s_data;
                core_s_last <= selected_core_s_last;
            end
        end
    end

    // This elastic boundary isolates every child engine's result-valid path
    // from the descriptor and wrapper FIFOs while retaining one beat/cycle.
    always_ff @(posedge aclk) begin
        if (!aresetn) begin
            core_m_valid <= 1'b0;
            core_raw_m_ready_r <= 1'b1;
            core_m_data <= 64'd0;
            core_m_last <= 1'b0;
        end else begin
            core_m_valid <= next_core_m_valid;
            core_raw_m_ready_r <= next_core_raw_m_ready;
            if (core_raw_m_ready_r) begin
                core_m_data <= core_raw_m_data;
                core_m_last <= core_raw_m_last;
            end
        end
    end

    always_comb begin
        next_output_fifo_count = output_fifo_count;
        unique case ({output_fifo_push, output_fifo_pop})
            2'b10: next_output_fifo_count = output_fifo_count + 1'b1;
```

```
306             2'b01: next_output_fifo_count = output_fifo_count - 1'b1;
307             default: begin end
308         endcase
309
310         next_core_output_capture_valid = core_output_capture_valid;
311         unique case ({core_output_capture_push, core_output_capture_pop})
312             2'b10: next_core_output_capture_valid = 1'b1;
313             2'b01: next_core_output_capture_valid = 1'b0;
314             default: begin end
315         endcase
316         // This next-cycle credit is conservative when a full FIFO pops: the
317         // resulting single restart bubble prevents any combinational READY
318         // path while preserving one result per cycle during steady traffic.
319         next_core_m_ready = !next_core_output_capture_valid
320             || next_output_fifo_count != 2;
321     end
322
323     // Two entries isolate external backpressure from every internal engine while
324     // still allowing simultaneous enqueue/dequeue at one result per clock.
325     function automatic logic [31:0] apply_write_strobes(
326         input logic [31:0] old_value,
327         input logic [31:0] new_value,
328         input logic [3:0] strobes
329     );
330         logic [31:0] merged;
331         begin
332             merged = old_value;
333             for (int byte_index = 0; byte_index < 4; byte_index++) begin
334                 if (strobes[byte_index]) begin
335                     merged[byte_index*8 +: 8] = new_value[byte_index*8 +: 8];
336                 end
337             end
338             apply_write_strobes = merged;
339         end
340     endfunction
341
342     function automatic logic [31:0] register_read(input logic [7:0] address);
343         begin
344             unique case (address)
345                 ADDR_CONTROL: register_read = {
346                     23'd0, interrupt_enable, 5'd0, program_active,
347                     1'b0, command_pending};
348                 ADDR_STATUS: register_read = {
349                     26'd0, program_active, interrupt, command_pending,
350                     done_sticky, core_error || wrapper_error || program_error,
351                     core_busy || command_pending || program_active};
352                 ADDR_ACTIVE_OPCODE: register_read = {28'd0, core_active_opcode};
353                 ADDR_COMMAND_COUNT: register_read = core_command_count;
354                 ADDR_COMMAND0: register_read = command_register[0];
355                 ADDR_COMMAND1: register_read = command_register[1];
356                 ADDR_COMMAND2: register_read = command_register[2];
357                 ADDR_COMMAND3: register_read = command_register[3];
358                 ADDR_INPUT_BEATS: register_read = core_input_beats;
359                 ADDR_OUTPUT_BEATS: register_read = core_output_beats;
360                 ADDR_TARGET_CLOCK: register_read = 32'd200000000;
361                 // Bits correspond to tensor, binary/ternary, alpha-XNOR,
362                 // vector/LUT, normalization, LIF, optimizer, and backward tile.
363                 ADDR_CAPABILITIES: register_read = 32'h000001ff;
364                 ADDR_VERSION: register_read = 32'h00010007;
365                 ADDR_ID: register_read = 32'h4e455552;
366                 ADDR_TENSOR_LOADED: register_read = {
367                     4'd0, core_tensor_loaded_weights,
368                     4'd0, core_tensor_loaded_activations};
369                 ADDR_TENSOR_ACTIVE: register_read = {20'd0, core_tensor_active_issues};
370                 ADDR_TENSOR_CYCLES: register_read = core_tensor_compute_cycles;
371                 ADDR_TENSOR_SKIPPED: register_read = core_tensor_skipped_issues;
372                 ADDR_DENSE_MACS_LOW: register_read = core_tensor_dense_macs[31:0];
373                 ADDR_DENSE_MACS_HIGH: register_read = core_tensor_dense_macs[63:32];
374                 ADDR_SPIKE_ADDS_LOW: register_read = core_tensor_spike_adds[31:0];
375                 ADDR_SPIKE_ADDS_HIGH: register_read = core_tensor_spike_adds[63:32];
376                 ADDR_LAYERNORM_CYCLES: register_read = core_layernorm_cycles;
377                 ADDR_LAYERNORM_MEAN: register_read = core_layernorm_mean;
378                 ADDR_LAYERNORM_VARIANCE: register_read = core_layernorm_variance;
379                 ADDR_LAYERNORM_SIGMA: register_read = core_layernorm_sigma;
380                 ADDR_SOFTMAX_CYCLES: register_read = core_softmax_cycles;
381                 ADDR_SOFTMAX_MAXIMUM: register_read =
382                     {{8{core_softmax_maximum[23]}}, core_softmax_maximum};
383                 ADDR_SOFTMAX_DENOMINATOR: register_read =
384                     {8'd0, core_softmax_denominator};
385                 ADDR_LIF_SPIKES_LOW: register_read = core_lif_total_spikes[31:0];
386                 ADDR_LIF_SPIKES_HIGH: register_read = core_lif_total_spikes[63:32];
387                 ADDR_OPTIMIZER_UPDATES: register_read = core_optimizer_updates;
388                 ADDR_LUT_ENTRIES: register_read = {23'd0, core_lut_loaded_entries};
389                 ADDR_LAYERNORM_RECIPROCAL: register_read = core_layernorm_reciprocal;
390                 ADDR_SOFTMAX_RECIPROCAL: register_read = core_softmax_reciprocal;
391                 ADDR_PROGRAM_RECORDS: register_read = program_records;
392                 ADDR_PROGRAM_OUTPUT_BEATS: register_read = program_output_beats_r;
```

```
                ADDR_TRAINING_TILES: register_read = core_training_tiles;
                ADDR_TRAINING_CYCLES: register_read = core_training_cycles;
                default: register_read = 32'd0;
            endcase
        end
    endfunction

    neuraedge_descriptor_sequencer u_descriptor_sequencer (
        .clk(aclk), .rst_n(aresetn),
        .program_start(program_start_r),
        .clear_status(program_clear_r),
        .program_output_beats(program_output_beats_r),
        .program_active(program_active), .program_done(program_done),
        .program_error(program_error), .program_records(program_records),
        .s_axis_tvalid(core_input_capture_valid),
        .s_axis_tready(sequencer_s_ready),
        .s_axis_tdata(core_input_capture_data),
        .s_axis_tlast(core_input_capture_last),
        .m_axis_tvalid(sequencer_m_valid),
        .m_axis_tready(core_m_ready_r),
        .m_axis_tdata(sequencer_m_data),
        .m_axis_tlast(sequencer_m_last),
        .core_command_valid(sequencer_core_command_valid),
        .core_command_ready(core_command_ready),
        .core_command_data(sequencer_core_command_data),
        .core_command_done(core_command_done),
        .core_status_error(core_error),
        .core_s_valid(sequencer_core_s_valid), .core_s_ready(core_s_ready),
        .core_s_data(sequencer_core_s_data), .core_s_last(sequencer_core_s_last),
        .core_m_valid(core_m_valid), .core_m_ready(sequencer_core_m_ready),
        .core_m_data(core_m_data), .core_m_last(core_m_last)
    );

    neuraedge_universal_core u_core (
        .clk(aclk), .rst_n(aresetn),
        .command_valid(program_active
            ? sequencer_core_command_valid : command_pending),
        .command_ready(core_command_ready),
        .command_data(program_active
            ? sequencer_core_command_data : submitted_command),
        .command_done(core_command_done),
        .status_busy(core_busy), .status_error(core_error),
        .status_active_opcode(core_active_opcode),
        .status_command_count(core_command_count),
        .status_input_beats(core_input_beats), .status_output_beats(core_output_beats),
        .debug_tensor_loaded_activations(core_tensor_loaded_activations),
        .debug_tensor_loaded_weights(core_tensor_loaded_weights),
        .debug_tensor_active_issues(core_tensor_active_issues),
        .debug_tensor_compute_cycles(core_tensor_compute_cycles),
        .debug_tensor_skipped_issues(core_tensor_skipped_issues),
        .debug_tensor_dense_macs(core_tensor_dense_macs),
        .debug_tensor_spike_adds(core_tensor_spike_adds),
        .debug_layernorm_cycles(core_layernorm_cycles),
        .debug_layernorm_mean(core_layernorm_mean),
        .debug_layernorm_variance(core_layernorm_variance),
        .debug_layernorm_sigma(core_layernorm_sigma),
        .debug_layernorm_reciprocal(core_layernorm_reciprocal),
        .debug_softmax_cycles(core_softmax_cycles),
        .debug_softmax_maximum(core_softmax_maximum),
        .debug_softmax_denominator(core_softmax_denominator),
        .debug_softmax_reciprocal(core_softmax_reciprocal),
        .debug_lif_total_spikes(core_lif_total_spikes),
        .debug_optimizer_updates(core_optimizer_updates),
        .debug_lut_loaded_entries(core_lut_loaded_entries),
        .debug_training_tiles(core_training_tiles),
        .debug_training_cycles(core_training_cycles),
        .s_axis_tvalid(core_s_valid), .s_axis_tready(core_raw_s_ready),
        .s_axis_tdata(core_s_data), .s_axis_tlast(core_s_last),
        .m_axis_tvalid(core_raw_m_valid), .m_axis_tready(core_raw_m_ready),
        .m_axis_tdata(core_raw_m_data), .m_axis_tlast(core_raw_m_last)
    );

    always_comb begin
        // A two-entry elastic FIFO makes the external ready path local while
        // still accepting one input beat per clock during simultaneous pop/push.
        s_axis_tready = input_fifo_count != 2;
        s_axi_awready = !aw_hold && !s_axi_bvalid;
        s_axi_wready = !w_hold && !s_axi_bvalid;
        s_axi_bresp = 2'b00;
        s_axi_arready = !s_axi_rvalid;
        s_axi_rresp = 2'b00;
        m_axis_tvalid = output_register_valid_r;
        m_axis_tdata = output_register_data_r;
        m_axis_tkeep = 8'hff;
        m_axis_tlast = output_register_last_r;
        interrupt = interrupt_enable && done_sticky;
    end
```

```
480
481     always_ff @(posedge aclk) begin
482         if (!aresetn) begin
483             for (int word_index = 0; word_index < 4; word_index++) begin
484                 command_register[word_index] <= 32'd0;
485             end
486             submitted_command <= 128'd0;
487             command_pending <= 1'b0;
488             interrupt_enable <= 1'b0;
489             done_sticky <= 1'b0;
490             wrapper_error <= 1'b0;
491             program_start_r <= 1'b0;
492             program_clear_r <= 1'b0;
493             program_output_beats_r <= 32'd0;
494             aw_hold <= 1'b0;
495             awaddr_hold <= '0;
496             w_hold <= 1'b0;
497             wdata_hold <= 32'd0;
498             wstrb_hold <= 4'd0;
499             s_axi_bvalid <= 1'b0;
500             s_axi_rvalid <= 1'b0;
501             s_axi_rdata <= 32'd0;
502             output_fifo_count <= 2'd0;
503             core_output_capture_valid <= 1'b0;
504             core_m_ready_r <= 1'b1;
505             core_output_capture_data <= 64'd0;
506             core_output_capture_last <= 1'b0;
507             output_fifo_write_pointer <= 1'b0;
508             output_fifo_read_pointer <= 1'b0;
509             output_fifo_data[0] <= 64'd0;
510             output_fifo_data[1] <= 64'd0;
511             output_fifo_last[0] <= 1'b0;
512             output_fifo_last[1] <= 1'b0;
513             output_register_valid_r <= 1'b0;
514             output_register_data_r <= 64'd0;
515             output_register_last_r <= 1'b0;
516             input_fifo_count <= 2'd0;
517             input_fifo_write_pointer <= 1'b0;
518             input_fifo_read_pointer <= 1'b0;
519             input_fifo_data[0] <= 64'd0;
520             input_fifo_data[1] <= 64'd0;
521             input_fifo_last[0] <= 1'b0;
522             input_fifo_last[1] <= 1'b0;
523             core_input_capture_valid <= 1'b0;
524             core_input_capture_data <= 64'd0;
525             core_input_capture_last <= 1'b0;
526         end else begin
527             program_start_r <= 1'b0;
528             program_clear_r <= 1'b0;
529             // The prefetch stage decouples payload storage from deep core-ready
530             // logic. At 200 MHz it accepts one 64-bit beat every two cycles.
531             if (input_fifo_pop) begin
532                 core_input_capture_data <= input_fifo_data[input_fifo_read_pointer];
533                 core_input_capture_last <= input_fifo_last[input_fifo_read_pointer];
534                 core_input_capture_valid <= 1'b1;
535             end else if (core_input_capture_valid && wrapper_input_ready) begin
536                 core_input_capture_valid <= 1'b0;
537             end
538             if (input_fifo_push) begin
539                 input_fifo_data[input_fifo_write_pointer] <= s_axis_tdata;
540                 input_fifo_last[input_fifo_write_pointer] <= s_axis_tlast;
541                 input_fifo_write_pointer <= !input_fifo_write_pointer;
542             end
543             if (input_fifo_pop) begin
544                 input_fifo_read_pointer <= !input_fifo_read_pointer;
545             end
546             unique case ({input_fifo_push, input_fifo_pop})
547                 2'b10: input_fifo_count <= input_fifo_count + 1'b1;
548                 2'b01: input_fifo_count <= input_fifo_count - 1'b1;
549                 default: begin end
550             endcase
551             // Sampling an invalid payload is harmless because the separate
552             // valid bit qualifies it. Enabling on local registered credit,
553             // rather than deep VALID && READY logic, keeps all 65 payload
554             // clock-enables local to this wrapper.
555             if (core_m_ready_r) begin
556                 core_output_capture_data <= wrapper_output_data;
557                 core_output_capture_last <= wrapper_output_last;
558             end
559             core_output_capture_valid <= next_core_output_capture_valid;
560             core_m_ready_r <= next_core_m_ready;
561             if (output_fifo_push) begin
562                 output_fifo_data[output_fifo_write_pointer]
563                     <= core_output_capture_data;
564                 output_fifo_last[output_fifo_write_pointer]
565                     <= core_output_capture_last;
566                 output_fifo_write_pointer <= !output_fifo_write_pointer;
```

```
            end
            if (output_fifo_pop) begin
                output_fifo_read_pointer <= !output_fifo_read_pointer;
            end
            output_fifo_count <= next_output_fifo_count;
            if (output_register_ready) begin
                output_register_valid_r <= output_fifo_count != 0;
                if (output_fifo_count != 0) begin
                    output_register_data_r
                        <= output_fifo_data[output_fifo_read_pointer];
                    output_register_last_r
                        <= output_fifo_last[output_fifo_read_pointer];
                end
            end
            if (command_pending && !program_active && core_command_ready) begin
                command_pending <= 1'b0;
            end
            if ((!program_active && core_command_done) || program_done) begin
                done_sticky <= 1'b1;
            end
            if (input_fifo_push && s_axis_tkeep != 8'hff) begin
                wrapper_error <= 1'b1;
            end

            if (s_axi_awvalid && s_axi_awready) begin
                aw_hold <= 1'b1;
                awaddr_hold <= s_axi_awaddr;
            end
            if (s_axi_wvalid && s_axi_wready) begin
                w_hold <= 1'b1;
                wdata_hold <= s_axi_wdata;
                wstrb_hold <= s_axi_wstrb;
            end

            if (aw_hold && w_hold && !s_axi_bvalid) begin
                unique case (awaddr_hold[7:0])
                    ADDR_CONTROL: begin
                        if (wstrb_hold[1]) interrupt_enable <= wdata_hold[8];
                        if (wstrb_hold[0] && wdata_hold[1]) begin
                            done_sticky <= 1'b0;
                            wrapper_error <= 1'b0;
                            program_clear_r <= 1'b1;
                        end
                        if (wstrb_hold[0] && wdata_hold[0]) begin
                            if (command_pending || program_active) begin
                                wrapper_error <= 1'b1;
                            end else begin
                                submitted_command <= {command_register[3], command_register[2],
                                                      command_register[1], command_register[0]};
                                command_pending <= 1'b1;
                                done_sticky <= 1'b0;
                            end
                        end
                        if (wstrb_hold[0] && wdata_hold[2]) begin
                            if (command_pending || program_active) begin
                                wrapper_error <= 1'b1;
                            end else begin
                                program_start_r <= 1'b1;
                                done_sticky <= 1'b0;
                            end
                        end
                    end
                    ADDR_COMMAND0: command_register[0] <= apply_write_strobes(
                        command_register[0], wdata_hold, wstrb_hold);
                    ADDR_COMMAND1: command_register[1] <= apply_write_strobes(
                        command_register[1], wdata_hold, wstrb_hold);
                    ADDR_COMMAND2: command_register[2] <= apply_write_strobes(
                        command_register[2], wdata_hold, wstrb_hold);
                    ADDR_COMMAND3: command_register[3] <= apply_write_strobes(
                        command_register[3], wdata_hold, wstrb_hold);
                    ADDR_PROGRAM_OUTPUT_BEATS: program_output_beats_r
                        <= apply_write_strobes(
                            program_output_beats_r, wdata_hold, wstrb_hold);
                    default: wrapper_error <= 1'b1;
                endcase
                aw_hold <= 1'b0;
                w_hold <= 1'b0;
                s_axi_bvalid <= 1'b1;
            end else if (s_axi_bvalid && s_axi_bready) begin
                s_axi_bvalid <= 1'b0;
            end

            if (s_axi_arvalid && s_axi_arready) begin
                s_axi_rdata <= register_read(s_axi_araddr[7:0]);
                s_axi_rvalid <= 1'b1;
            end else if (s_axi_rvalid && s_axi_rready) begin
                s_axi_rvalid <= 1'b0;
```

```
            end
        end
    end

    initial begin
        if (AXI_ADDR_BITS < 8) $fatal(1, "AXI_ADDR_BITS must be at least 8");
    end
endmodule

`default_nettype wire
```

17.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_pynq_axi_top #(` This opens neuraedge_pynq_axi_top, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 5: `parameter int AXI_ADDR_BITS = 8` This fixes AXI_ADDR_BITS to 8 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; AXI_ADDR_BITS denotes the value named 'axi addr bits' in this block.

Line 6: `) (` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The expression is a literal or structural operator and has no hidden software state.

Line 7: `(* X_INTERFACE_INFO = "xilinx.com:signal:clock:1.0 aclk CLK" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 8: `(* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME aclk, ASSOCIATED_BUSIF S_AXI:S_AXIS:M_AXIS, ASSOCIATED_`
`...` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_PARAMETER: the value named 'x interface parameter' in this block; XIL_INTERFACENAME: the value named 'xil interfacename' in this block; aclk: the AXI and accelerator clock.

Line 9: `input wire aclk,` This declares aclk as a input port with width one bit. It carries the AXI and accelerator clock across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 10: `(* X_INTERFACE_INFO = "xilinx.com:signal:reset:1.0 aresetn RST" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 11: `(* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME aresetn, POLARITY ACTIVE_LOW" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_PARAMETE
the value named 'x interface parameter' in this block; XIL_INTERFACENAME: the value named 'xil interfacename' in this block; aresetn: the active-low AXI reset.

Line 12: `input wire aresetn,` This declares aresetn as a input port with width one bit. It carries the active-low AXI reset across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 14: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI AWADDR" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 15: `(* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME S_AXI, PROTOCOL AXI4LITE, DATA_WIDTH 32, ADDR_WIDTH 8, FREQ_H...` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_PARAMETER: the value named 'x interface parameter' in this block; XIL_INTERFACENAME: the value named 'xil interfacename' in this block; S_AXI: the value named 's axi' in this block.

Line 16: `input wire [AXI_ADDR_BITS-1:0] s_axi_awaddr,` This declares s_axi_awaddr as a input port with width [AXI_ADDR_BITS-1:0]. It carries the value named 's axi awaddr' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 17: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI AWVALID" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 18: `input wire s_axi_awvalid,` This declares s_axi_awvalid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 19: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI AWREADY" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 20: `output logic s_axi_awready,` This declares s_axi_awready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 21: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI WDATA" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 22: `input wire [31:0] s_axi_wdata,` This declares s_axi_wdata as a input port with width [31:0]. It carries the value named 's axi wdata' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 23: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI WSTRB" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 24: `input wire [3:0] s_axi_wstrb,` This declares s_axi_wstrb as a input port with width [3:0]. It carries the value named 's axi wstrb' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 25: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI WVALID" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 26: `input wire s_axi_wvalid,` This declares s_axi_wvalid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 27: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI WREADY" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 28: `output logic s_axi_wready,` This declares s_axi_wready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 29: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI BRESP" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 30: `output logic [1:0] s_axi_bresp,` This declares s_axi_bresp as a output port with width [1:0]. It carries the value named 's axi bresp' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 31: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI BVALID" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 32: `output logic s_axi_bvalid,` This declares s_axi_bvalid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 33: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI BREADY" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 34: `input wire s_axi_bready,` This declares s_axi_bready as a input port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 35: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI ARADDR" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 36: `input wire [AXI_ADDR_BITS-1:0] s_axi_araddr,` This declares s_axi_araddr as a input port with width [AXI_ADDR_BITS-1:0]. It carries the value named 's axi araddr' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 37: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI ARVALID" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 38: `input wire s_axi_arvalid,` This declares s_axi_arvalid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 39: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI ARREADY" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 40: `output logic s_axi_arready,` This declares s_axi_arready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 41: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI RDATA" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 42: `output logic [31:0] s_axi_rdata,` This declares s_axi_rdata as a output port with width [31:0]. It carries the value named 's axi rdata' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 43: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI RRESP" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 44: `output logic [1:0] s_axi_rresp,` This declares s_axi_rresp as a output port with width [1:0]. It carries the value named 's axi rresp' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 45: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI RVALID" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 46: `output logic s_axi_rvalid,` This declares s_axi_rvalid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 47: `(* X_INTERFACE_INFO = "xilinx.com:interface:aximm:1.0 S_AXI RREADY" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 48: `input wire s_axi_rready,` This declares s_axi_rready as a input port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 50: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TVALID" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 51: `(* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME S_AXIS, TDATA_NUM_BYTES 8, HAS_TKEEP 1, HAS_TLAST 1, FREQ_HZ ...` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_PARAMETER: the value named 'x interface parameter' in this block; XIL_INTERFACENAME the value named 'xil interfacename' in this block; S_AXIS: the value named 's axis' in this block.

Line 52: `input wire s_axis_tvalid,` This declares s_axis_tvalid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 53: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TREADY" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 54: `output logic s_axis_tready,` This declares s_axis_tready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 55: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TDATA" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 56: `input wire [63:0] s_axis_tdata,` This declares s_axis_tdata as a input port with width [63:0]. It carries the value named 's axis tdata' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 57: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TKEEP" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 58: `input wire [7:0] s_axis_tkeep,` This declares s_axis_tkeep as a input port with width [7:0]. It carries the value named 's axis tkeep' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 59: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 S_AXIS TLAST" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 60: `input wire s_axis_tlast,` This declares s_axis_tlast as a input port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 61: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TVALID" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 62: `(* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME M_AXIS, TDATA_NUM_BYTES 8, HAS_TKEEP 1, HAS_TLAST 1, FREQ_HZ ...` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_PARAMETER: the value named 'x interface parameter' in this block; XIL_INTERFACENAME the value named 'xil interfacename' in this block; M_AXIS: the value named 'm axis' in this block.

Line 63: `output logic m_axis_tvalid,` This declares m_axis_tvalid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 64: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TREADY" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 65: `input wire m_axis_tready,` This declares m_axis_tready as a input port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 66: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TDATA" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 67: `output logic [63:0] m_axis_tdata,` This declares m_axis_tdata as a output port with width [63:0]. It carries the value named 'm axis tdata' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 68: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TKEEP" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 69: `output logic [7:0] m_axis_tkeep,` This declares m_axis_tkeep as a output port with width [7:0]. It carries the value named 'm axis tkeep' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 70: `(* X_INTERFACE_INFO = "xilinx.com:interface:axis:1.0 M_AXIS TLAST" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 71: `output logic m_axis_tlast,` This declares m_axis_tlast as a output port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 72: `(* X_INTERFACE_INFO = "xilinx.com:signal:interrupt:1.0 interrupt INTERRUPT" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_INFO: the value named 'x interface info' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 73: `(* X_INTERFACE_PARAMETER = "XIL_INTERFACENAME interrupt, SENSITIVITY LEVEL_HIGH" *)` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are X_INTERFACE_PARA the value named 'x interface parameter' in this block; XIL_INTERFACENAME: the value named 'xil interfacename' in this block; interrupt: the hardware notification that wakes software after completion or error.

Line 74: `output logic interrupt` This declares interrupt as a output port with width one bit. It carries the hardware notification that wakes software after completion or error across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 75: `);` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The expression is a literal or structural operator and has no hidden software state.

Line 76: `localparam logic [7:0] ADDR_CONTROL = 8'h00;` This fixes ADDR_CONTROL to 8'h00 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_CONTROL denotes the value named 'addr control' in this block.

Line 77: `localparam logic [7:0] ADDR_STATUS = 8'h04;` This fixes ADDR_STATUS to 8'h04 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_STATUS denotes the value named 'addr status' in this block.

Line 78: `localparam logic [7:0] ADDR_ACTIVE_OPCODE = 8'h08;` This fixes ADDR_ACTIVE_OPCODE to 8'h08 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_ACTIVE_OPCODE denotes the value named 'addr active opcode' in this block.

Line 79: `localparam logic [7:0] ADDR_COMMAND_COUNT = 8'h0c;` This fixes ADDR_COMMAND_COUNT to 8'h0c during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_COMMAND_COUNT denotes the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 80: `localparam logic [7:0] ADDR_COMMAND0 = 8'h10;` This fixes ADDR_COMMAND0 to 8'h10 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_COMMAND0 denotes the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 81: `localparam logic [7:0] ADDR_COMMAND1 = 8'h14;` This fixes ADDR_COMMAND1 to 8'h14 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_COMMAND1 denotes the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 82: `localparam logic [7:0] ADDR_COMMAND2 = 8'h18;` This fixes ADDR_COMMAND2 to 8'h18 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_COMMAND2 denotes the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 83: `localparam logic [7:0] ADDR_COMMAND3 = 8'h1c;` This fixes ADDR_COMMAND3 to 8'h1c during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_COMMAND3 denotes the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 84: `localparam logic [7:0] ADDR_INPUT_BEATS = 8'h20;` This fixes ADDR_INPUT_BEATS to 8'h20 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_INPUT_BEATS denotes the value named 'addr input beats' in this block.

Line 85: `localparam logic [7:0] ADDR_OUTPUT_BEATS = 8'h24;` This fixes ADDR_OUTPUT_BEATS to 8'h24 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_OUTPUT_BEATS denotes the value named 'addr output beats' in this block.

Line 86: `localparam logic [7:0] ADDR_TARGET_CLOCK = 8'h28;` This fixes ADDR_TARGET_CLOCK to 8'h28 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_TARGET_CLOCK denotes the value named 'addr target clock' in this block.

Line 87: `localparam logic [7:0] ADDR_CAPABILITIES = 8'h2c;` This fixes ADDR_CAPABILITIES to 8'h2c during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_CAPABILITIES denotes the value named 'addr capabilities' in this block.

Line 88: `localparam logic [7:0] ADDR_VERSION = 8'h30;` This fixes ADDR_VERSION to 8'h30 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_VERSION denotes the value named 'addr version' in this block.

Line 89: `localparam logic [7:0] ADDR_ID = 8'h34;` This fixes ADDR_ID to 8'h34 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_ID denotes the value named 'addr id' in this block.

Line 90: `localparam logic [7:0] ADDR_TENSOR_LOADED = 8'h38;` This fixes ADDR_TENSOR_LOADED to 8'h38 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_TENSOR_LOADED denotes the value named 'addr tensor loaded' in this block.

Line 91: `localparam logic [7:0] ADDR_TENSOR_ACTIVE = 8'h3c;` This fixes ADDR_TENSOR_ACTIVE to 8'h3c during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_TENSOR_ACTIVE denotes the value named 'addr tensor active' in this block.

Line 92: `localparam logic [7:0] ADDR_TENSOR_CYCLES = 8'h40;` This fixes ADDR_TENSOR_CYCLES to 8'h40 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_TENSOR_CYCLES denotes the value named 'addr tensor cycles' in this block.

Line 93: `localparam logic [7:0] ADDR_TENSOR_SKIPPED = 8'h44;` This fixes ADDR_TENSOR_SKIPPED to 8'h44 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_TENSOR_SKIPPED denotes the value named 'addr tensor skipped' in this block.

Line 94: `localparam logic [7:0] ADDR_DENSE_MACS_LOW = 8'h48;` This fixes ADDR_DENSE_MACS_LOW to 8'h48 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_DENSE_MACS_LOW denotes a count of ordinary activation-times-weight operations actually issued.

Line 95: `localparam logic [7:0] ADDR_DENSE_MACS_HIGH = 8'h4c;` This fixes ADDR_DENSE_MACS_HIGH to 8'h4c during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_DENSE_MACS_HIGH denotes a count of ordinary activation-times-weight operations actually issued.

Line 96: `localparam logic [7:0] ADDR_SPIKE_ADDS_LOW = 8'h50;` This fixes ADDR_SPIKE_ADDS_LOW to 8'h50 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_SPIKE_ADDS_LOW denotes a binary event produced when a neuron crosses its threshold.

Line 97: `localparam logic [7:0] ADDR_SPIKE_ADDS_HIGH = 8'h54;` This fixes ADDR_SPIKE_ADDS_HIGH to 8'h54 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_SPIKE_ADDS_HIGH denotes a binary event produced when a neuron crosses its threshold.

Line 98: `localparam logic [7:0] ADDR_LAYERNORM_CYCLES = 8'h58;` This fixes ADDR_LAYERNORM_CYCLES to 8'h58 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_LAYERNORM_CYCLES denotes the value named 'addr layernorm cycles' in this block.

Line 99: `localparam logic [7:0] ADDR_LAYERNORM_MEAN = 8'h5c;` This fixes ADDR_LAYERNORM_MEAN to 8'h5c during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_LAYERNORM_MEAN denotes the arithmetic average used by normalization.

Line 100: `localparam logic [7:0] ADDR_LAYERNORM_VARIANCE = 8'h60;` This fixes ADDR_LAYERNORM_VARIANCE to 8'h60 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_LAYERNORM_VARIANC denotes the average squared distance from the mean.

Line 101: `localparam logic [7:0] ADDR_LAYERNORM_SIGMA = 8'h64;` This fixes ADDR_LAYERNORM_SIGMA to 8'h64 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_LAYERNORM_SIGMA denotes the value named 'addr layernorm sigma' in this block.

Line 102: `localparam logic [7:0] ADDR_SOFTMAX_CYCLES = 8'h68;` This fixes ADDR_SOFTMAX_CYCLES to 8'h68 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_SOFTMAX_CYCLES denotes the value named 'addr softmax cycles' in this block.

Line 103: `localparam logic [7:0] ADDR_SOFTMAX_MAXIMUM = 8'h6c;` This fixes ADDR_SOFTMAX_MAXIMUM to 8'h6c during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_SOFTMAX_MAXIMUM denotes the value named 'addr softmax maximum' in this block.

Line 104: `localparam logic [7:0] ADDR_SOFTMAX_DENOMINATOR = 8'h70;` This fixes ADDR_SOFTMAX_DENOMINATOR to 8'h70 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_SOFTMAX_DENOMINA denotes the value named 'addr softmax denominator' in this block.

Line 105: `localparam logic [7:0] ADDR_LIF_SPIKES_LOW = 8'h74;` This fixes ADDR_LIF_SPIKES_LOW to 8'h74 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_LIF_SPIKES_LOW denotes a binary event produced when a neuron crosses its threshold.

Line 106: `localparam logic [7:0] ADDR_LIF_SPIKES_HIGH = 8'h78;` This fixes ADDR_LIF_SPIKES_HIGH to 8'h78 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_LIF_SPIKES_HIGH denotes a binary event produced when a neuron crosses its threshold.

Line 107: `localparam logic [7:0] ADDR_OPTIMIZER_UPDATES = 8'h7c;` This fixes ADDR_OPTIMIZER_UPDATES to 8'h7c during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_OPTIMIZER_UPDATES denotes the value named 'addr optimizer updates' in this block.

Line 108: `localparam logic [7:0] ADDR_LUT_ENTRIES = 8'h80;` This fixes ADDR_LUT_ENTRIES to 8'h80 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_LUT_ENTRIES denotes the value named 'addr lut entries' in this block.

Line 109: `localparam logic [7:0] ADDR_LAYERNORM_RECIPROCAL = 8'h84;` This fixes ADDR_LAYERNORM_RECIPROCA to 8'h84 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_LAYERNORM_RECIPRO denotes the value named 'addr layernorm reciprocal' in this block.

Line 110: `localparam logic [7:0] ADDR_SOFTMAX_RECIPROCAL = 8'h88;` This fixes ADDR_SOFTMAX_RECIPROCAL to 8'h88 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_SOFTMAX_RECIPROCA denotes the value named 'addr softmax reciprocal' in this block.

Line 111: `localparam logic [7:0] ADDR_PROGRAM_RECORDS = 8'h8c;` This fixes ADDR_PROGRAM_RECORDS to 8'h8c during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_PROGRAM_RECORDS denotes the number of descriptor records accepted since program start.

Line 112: `localparam logic [7:0] ADDR_PROGRAM_OUTPUT_BEATS = 8'h90;` This fixes ADDR_PROGRAM_OUTPUT_BEA to 8'h90 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_PROGRAM_OUTPUT_B denotes the value named 'addr program output beats' in this block.

Line 113: `localparam logic [7:0] ADDR_TRAINING_TILES = 8'h94;` This fixes ADDR_TRAINING_TILES to 8'h94 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_TRAINING_TILES denotes the value named 'addr training tiles' in this block.

Line 114: `localparam logic [7:0] ADDR_TRAINING_CYCLES = 8'h98;` This fixes ADDR_TRAINING_CYCLES to 8'h98 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ADDR_TRAINING_CYCLES denotes the value named 'addr training cycles' in this block.

Line 116: `logic [31:0] command_register [0:3];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_register is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 117: `logic [127:0] submitted_command;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, submitted_command is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 118: `logic command_pending;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_pending is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 119: `logic interrupt_enable;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, interrupt_enable is the hardware notification that wakes software after completion or error. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 120: `logic done_sticky;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, done_sticky is the value named 'done sticky' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 121: `logic wrapper_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wrapper_error is the value named 'wrapper error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 122: `logic program_start_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_start_r is the value named 'program start r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 123: `logic program_clear_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_clear_r is the value named 'program clear r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 124: `logic [31:0] program_output_beats_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_output_beats_r is the value named 'program output beats r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 125: `wire program_active;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_active is the value named 'program active' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 126: `wire program_done;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_done is the value named 'program done' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 127: `wire program_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_error is the value named 'program error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 128: `wire [31:0] program_records;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_records is the number of descriptor records accepted since program start. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 130: `logic aw_hold;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, aw_hold is the value named 'aw hold' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 131: `logic [AXI_ADDR_BITS-1:0] awaddr_hold;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, awaddr_hold is the value named 'awaddr hold' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 132: `logic w_hold;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, w_hold is the value named 'w hold' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 133: `logic [31:0] wdata_hold;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wdata_hold is the value named 'wdata hold' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 134: `logic [3:0] wstrb_hold;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wstrb_hold is the value named 'wstrb hold' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 136: `wire core_command_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_command_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 137: `wire core_command_done;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_command_done is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 138: `wire sequencer_core_command_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_core_command_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 139: `wire [127:0] sequencer_core_command_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_core_command_data is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 140: `wire sequencer_core_s_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_core_s_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 141: `wire [63:0] sequencer_core_s_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_core_s_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 142: `wire sequencer_core_s_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_core_s_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 143: `wire sequencer_core_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_core_m_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 144: `wire core_busy;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_busy is the value named 'core busy' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 145: `wire core_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_error is the value named 'core error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 146: `wire [3:0] core_active_opcode;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_active_opcode is the value named 'core active opcode' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 147: `wire [31:0] core_command_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_command_count is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 148: `wire [31:0] core_input_beats;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_input_beats is the value named 'core input beats' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 149: `wire [31:0] core_output_beats;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_beats is the value named 'core output beats' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 150: `wire [11:0] core_tensor_loaded_activations;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_tensor_loaded_activations is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 151: `wire [11:0] core_tensor_loaded_weights;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_tensor_loaded_weights is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 152: `wire [11:0] core_tensor_active_issues;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_tensor_active_issues is the value named 'core tensor active issues' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 153: `wire [31:0] core_tensor_compute_cycles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_tensor_compute_cycles is the value named 'core tensor compute cycles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 154: `wire [31:0] core_tensor_skipped_issues;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_tensor_skipped_issues is the value named 'core tensor skipped issues' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 155: `wire [63:0] core_tensor_dense_macs;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_tensor_dense_macs is a count of ordinary activation-times-weight operations actually issued. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 156: `wire [63:0] core_tensor_spike_adds;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_tensor_spike_adds is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 157: `wire [31:0] core_layernorm_cycles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_layernorm_cycles is the value named 'core layernorm cycles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 158: `wire signed [31:0] core_layernorm_mean;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_layernorm_mean is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 159: `wire [31:0] core_layernorm_variance;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_layernorm_variance is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 160: `wire [31:0] core_layernorm_sigma;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_layernorm_sigma is the value named 'core layernorm sigma' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 161: `wire [31:0] core_layernorm_reciprocal;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_layernorm_reciprocal is the value named 'core layernorm reciprocal' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 162: `wire [31:0] core_softmax_cycles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_softmax_cycles is the value named 'core softmax cycles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 163: `wire signed [23:0] core_softmax_maximum;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_softmax_maximum is the value named 'core softmax maximum' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 164: `wire [23:0] core_softmax_denominator;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_softmax_denominator is the value named 'core softmax denominator' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 165: `wire [31:0] core_softmax_reciprocal;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_softmax_reciprocal is the value named 'core softmax reciprocal' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 166: `wire [63:0] core_lif_total_spikes;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_lif_total_spikes is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 167: `wire [31:0] core_optimizer_updates;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_optimizer_updates is the value named 'core optimizer updates' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 168: `wire [8:0] core_lut_loaded_entries;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_lut_loaded_entries is the value named 'core lut loaded entries' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 169: `wire [31:0] core_training_tiles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_training_tiles is the value named 'core training tiles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 170: `wire [31:0] core_training_cycles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_training_cycles is the value named 'core training cycles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 171: `logic core_s_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 172: `wire core_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 173: `wire core_raw_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_raw_s_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 174: `logic core_s_ready_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_ready_r is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 175: `logic next_core_s_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, next_core_s_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 176: `logic next_core_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, next_core_s_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 177: `logic [63:0] core_s_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 178: `logic core_s_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 179: `logic [1:0] input_fifo_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 180: `logic input_fifo_write_pointer;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_write_pointer is the value named 'input fifo write pointer' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 181: `logic input_fifo_read_pointer;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_read_pointer is the value named 'input fifo read pointer' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 182: `logic [63:0] input_fifo_data [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 183: `logic input_fifo_last [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 184: `logic core_input_capture_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_input_capture_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 185: `logic [63:0] core_input_capture_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_input_capture_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 186: `logic core_input_capture_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_input_capture_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 187: `wire core_raw_m_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_raw_m_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 188: `wire core_raw_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_raw_m_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 189: `logic core_raw_m_ready_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_raw_m_ready_r is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 190: `wire [63:0] core_raw_m_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_raw_m_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 191: `wire core_raw_m_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_raw_m_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 192: `logic core_m_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 193: `logic next_core_m_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, next_core_m_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 194: `logic next_core_raw_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, next_core_raw_m_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 195: `wire core_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 196: `wire sequencer_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_s_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 197: `wire sequencer_m_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_m_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 198: `wire [63:0] sequencer_m_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_m_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 199: `wire sequencer_m_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sequencer_m_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 200: `logic core_m_ready_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_ready_r is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 201: `logic [63:0] core_m_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 202: `logic core_m_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 203: `logic core_output_capture_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_capture_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 204: `logic [63:0] core_output_capture_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_capture_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 205: `logic core_output_capture_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_capture_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 206: `logic [1:0] output_fifo_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_fifo_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 207: `logic output_fifo_write_pointer;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_fifo_write_pointer is the value named 'output fifo write pointer' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 208: `logic output_fifo_read_pointer;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_fifo_read_pointer is the value named 'output fifo read pointer' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 209: `logic [63:0] output_fifo_data [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_fifo_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 210: `logic output_fifo_last [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_fifo_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 211: `logic output_register_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_register_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 212: `logic [63:0] output_register_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_register_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 213: `logic output_register_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_register_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 214: `logic [1:0] next_output_fifo_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, next_output_fifo_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 215: `logic next_core_output_capture_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, next_core_output_capture_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 216: `logic next_core_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, next_core_m_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 218: `wire input_fifo_push = s_axis_tvalid && s_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_push is the value named 'input fifo push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 219: `wire input_fifo_pop = input_fifo_count != 0 && !core_input_capture_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_fifo_pop is the value named 'input fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 220: `wire wrapper_input_ready = program_active ? sequencer_s_ready : core_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wrapper_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 221: `wire wrapper_output_valid = program_active ? sequencer_m_valid : core_m_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wrapper_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 222: `wire [63:0] wrapper_output_data = program_active` This statement continues through source line 223; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wrapper_output_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 223: `? sequencer_m_data : core_m_data;` This is continuation line 2 of the 2-line logical statement that begins at source line 222. Its exact fragment is '? sequencer_m_data : core_m_data;'. The important operands are sequencer_m_data: the payload carried with the surrounding valid/ready handshake; core_m_data: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wrapper_output_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 224: `wire wrapper_output_last = program_active ? sequencer_m_last : core_m_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wrapper_output_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 225: `wire core_output_capture_push = wrapper_output_valid && core_m_ready_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_capture_push is the value named 'core output capture push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 226: `wire core_output_capture_pop = core_output_capture_valid` This statement continues through source line 227; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_capture_pop is the value named 'core output capture pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 227: `&& output_fifo_count != 2;` This is continuation line 2 of the 2-line logical statement that begins at source line 226. Its exact fragment is '&& output_fifo_count != 2;'. The important operands are output_fifo_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_capture_pop is the value named 'core output capture pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 228: `wire output_fifo_push = core_output_capture_pop;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_fifo_push is the value named 'output fifo push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 229: `wire output_register_ready = !output_register_valid_r || m_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_register_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 230: `wire output_fifo_pop = output_fifo_count != 0 && output_register_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_fifo_pop is the value named 'output fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 231: `wire selected_core_s_valid = program_active` This statement continues through source line 232; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, selected_core_s_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 232: `? sequencer_core_s_valid : core_input_capture_valid;` This is continuation line 2 of the 2-line logical statement that begins at source line 231. Its exact fragment is '? sequencer_core_s_valid : core_input_capture_valid;'. The important operands are sequencer_core_s_valid: the producer's statement that its current payload is meaningful; core_input_capture_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, selected_core_s_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 233: `wire [63:0] selected_core_s_data = program_active` This statement continues through source line 234; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, selected_core_s_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 234: `? sequencer_core_s_data : core_input_capture_data;` This is continuation line 2 of the 2-line logical statement that begins at source line 233. Its exact fragment is '? sequencer_core_s_data : core_input_capture_data;'. The important operands are sequencer_core_s_data: the payload carried with the surrounding valid/ready handshake; core_input_capture_data: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, selected_core_s_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 235: `wire selected_core_s_last = program_active` This statement continues through source line 236; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, selected_core_s_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 236: `? sequencer_core_s_last : core_input_capture_last;` This is continuation line 2 of the 2-line logical statement that begins at source line 235. Its exact fragment is '? sequencer_core_s_last : core_input_capture_last;'. The important operands are sequencer_core_s_last: the AXI-Stream marker for the final beat of a packet or command payload; core_input_capture_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, selected_core_s_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 237: `wire core_s_boundary_push = selected_core_s_valid && core_s_ready_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_boundary_push is the value named 'core s boundary push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 238: `wire core_s_boundary_pop = core_s_valid && core_raw_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_boundary_pop is the value named 'core s boundary pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 239: `wire core_m_boundary_push = core_raw_m_valid && core_raw_m_ready_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_boundary_push is the value named 'core m boundary push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 240: `wire core_m_boundary_pop = core_m_valid && core_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_boundary_pop is the value named 'core m boundary pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 241: `assign core_s_ready = core_s_ready_r;` This makes core_s_ready, the consumer's statement that it can accept a payload this cycle, equal to 'core_s_ready_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_s_ready_r: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 242: `assign core_m_ready = program_active` This statement continues through source line 243; the explanation uses the complete expression rather than only the first visual fragment. This makes core_m_ready, the consumer's statement that it can accept a payload this cycle, equal to 'program_active ? sequencer_core_m_ready : core_m_ready_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; sequencer_core_m_ready: the consumer's statement that it can accept a payload this cycle; core_m_ready_r: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 243: `? sequencer_core_m_ready : core_m_ready_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 242. Its exact fragment is '? sequencer_core_m_ready : core_m_ready_r;'. The important operands are sequencer_core_m_ready: the consumer's statement that it can accept a payload this cycle; core_m_ready_r: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This makes core_m_ready, the consumer's statement that it can accept a payload this cycle, equal to 'program_active ? sequencer_core_m_ready : core_m_ready_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are program_active: the value named 'program active' in this block; sequencer_core_m_ready: the consumer's statement that it can accept a payload this cycle; core_m_ready_r: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 244: `assign core_raw_m_ready = core_raw_m_ready_r;` This makes core_raw_m_ready, the consumer's statement that it can accept a payload this cycle, equal to 'core_raw_m_ready_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_raw_m_ready_r: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 246: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 247: `next_core_s_valid = core_s_valid;` This makes next_core_s_valid, the producer's statement that its current payload is meaningful, equal to 'core_s_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_s_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 248: `unique case ({core_s_boundary_push, core_s_boundary_pop})` This starts a multiplexer/state decode selected by '{core_s_boundary_push, core_s_boundary_pop}'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 249: `2'b10: next_core_s_valid = 1'b1;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b10: the value named 'b10' in this block; next_core_s_valid: the producer's statement that its current payload is meaningful; b1: the value named 'b1' in this block.

Line 250: `2'b01: next_core_s_valid = 1'b0;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b01: the value named 'b01' in this block; next_core_s_valid: the producer's statement that its current payload is meaningful; b0: the value named 'b0' in this block.

Line 251: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the combinational process whose outputs respond without waiting for a clock edge.

Line 252: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 253: `next_core_s_ready = !next_core_s_valid || core_raw_s_ready;` This makes next_core_s_ready, the consumer's statement that it can accept a payload this cycle, equal to '!next_core_s_valid || core_raw_s_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are next_core_s_valid: the producer's statement that its current payload is meaningful; core_raw_s_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 254: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 256: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 257: `next_core_m_valid = core_m_valid;` This makes next_core_m_valid, the producer's statement that its current payload is meaningful, equal to 'core_m_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_m_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 258: `unique case ({core_m_boundary_push, core_m_boundary_pop})` This starts a multiplexer/state decode selected by ‘{core_m_boundary_push, core_m_boundary_pop}‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 259: `2'b10: next_core_m_valid = 1'b1;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b10: the value named ’b10’ in this block; next_core_m_valid: the producer’s statement that its current payload is meaningful; b1: the value named ’b1’ in this block.

Line 260: `2'b01: next_core_m_valid = 1'b0;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b01: the value named ’b01’ in this block; next_core_m_valid: the producer’s statement that its current payload is meaningful; b0: the value named ’b0’ in this block.

Line 261: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin end‘; context is the combinational process whose outputs respond without waiting for a clock edge.

Line 262: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 263: `next_core_raw_m_ready = !next_core_m_valid || core_m_ready;` This makes next_core_raw_m_ready, the consumer’s statement that it can accept a payload this cycle, equal to ‘!next_core_m_valid || core_m_ready‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are next_core_m_valid: the producer’s statement that its current payload is meaningful; core_m_ready: the consumer’s statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 264: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 266: `// Registered credit prevents a child engine's READY state from becoming` Design intent: Registered credit prevents a child engine’s READY state from becoming This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 267: `// a payload clock-enable while retaining simultaneous consume/refill.` Design intent: a payload clock-enable while retaining simultaneous consume/refill. This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 268: `always_ff @(posedge aclk) begin` This opens clocked sequential logic. Everything assigned with ‘<=‘ inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator’s notion of cycles and persistent state.

Line 269: `if (!aresetn) begin` This is the reset branch for ‘(!aresetn‘. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 270: `core_s_valid <= 1'b0;` This makes core_s_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 271: `core_s_ready_r <= 1'b1;` This makes core_s_ready_r, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 272: `core_s_data <= 64'd0;` This makes core_s_data, the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 273: `core_s_last <= 1'b0;` This makes core_s_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 274: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 275: `core_s_valid <= next_core_s_valid;` This makes core_s_valid, the producer's statement that its current payload is meaningful, equal to 'next_core_s_valid'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_core_s_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 276: `core_s_ready_r <= next_core_s_ready;` This makes core_s_ready_r, the consumer's statement that it can accept a payload this cycle, equal to 'next_core_s_ready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_core_s_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 277: `if (core_s_ready_r) begin` This condition selects the following hardware branch only when 'core_s_ready_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are core_s_ready_r: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 278: `core_s_data <= selected_core_s_data;` This makes core_s_data, the payload carried with the surrounding valid/ready handshake, equal to 'selected_core_s_data'. It updates after the active clock edge together with other nonblocking assignments. The important operands are selected_core_s_data: the payload carried with the surrounding valid/ready handshake. Context: the clocked process that updates registers only on a rising clock edge.

Line 279: `core_s_last <= selected_core_s_last;` This makes core_s_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'selected_core_s_last'. It updates after the active clock edge together with other nonblocking assignments. The important operands are selected_core_s_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 280: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 281: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 282: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 284: `// This elastic boundary isolates every child engine's result-valid path` Design intent: This elastic boundary isolates every child engine's result-valid path This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 285: `// from the descriptor and wrapper FIFOs while retaining one beat/cycle.` Design intent: from the descriptor and wrapper FIFOs while retaining one beat/cycle. This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 286: `always_ff @(posedge aclk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 287: `if (!aresetn) begin` This is the reset branch for '(!aresetn'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 288: `core_m_valid <= 1'b0;` This makes core_m_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 289: `core_raw_m_ready_r <= 1'b1;` This makes core_raw_m_ready_r, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 290: `core_m_data <= 64'd0;` This makes core_m_data, the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 291: `core_m_last <= 1'b0;` This makes core_m_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 292: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 293: `core_m_valid <= next_core_m_valid;` This makes core_m_valid, the producer's statement that its current payload is meaningful, equal to 'next_core_m_valid'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_core_m_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 294: `core_raw_m_ready_r <= next_core_raw_m_ready;` This makes core_raw_m_ready_r, the consumer's statement that it can accept a payload this cycle, equal to 'next_core_raw_m_ready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_core_raw_m_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 295: `if (core_raw_m_ready_r) begin` This condition selects the following hardware branch only when 'core_raw_m_read is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are core_raw_m_ready_r: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 296: `core_m_data <= core_raw_m_data;` This makes core_m_data, the payload carried with the surrounding valid/ready handshake, equal to ‘core_raw_m_data‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are core_raw_m_data: the payload carried with the surrounding valid/ready handshake. Context: the clocked process that updates registers only on a rising clock edge.

Line 297: `core_m_last <= core_raw_m_last;` This makes core_m_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘core_raw_m_last‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are core_raw_m_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 298: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 299: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 300: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 302: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 303: `next_output_fifo_count = output_fifo_count;` This makes next_output_fifo_count, an integer count retained for control or measurable telemetry, equal to ‘output_fifo_count‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_fifo_count: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 304: `unique case ({output_fifo_push, output_fifo_pop})` This starts a multiplexer/state decode selected by ‘{output_fifo_push, output_fifo_pop}‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 305: `2'b10: next_output_fifo_count = output_fifo_count + 1'b1;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b10: the value named 'b10' in this block; next_output_fifo_count: an integer count retained for control or measurable telemetry; output_fifo_count: an integer count retained for control or measurable telemetry.

Line 306: `2'b01: next_output_fifo_count = output_fifo_count - 1'b1;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b01: the value named 'b01' in this block; next_output_fifo_count: an integer count retained for control or measurable telemetry; output_fifo_count: an integer count retained for control or measurable telemetry.

Line 307: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin end‘; context is the combinational process whose outputs respond without waiting for a clock edge.

Line 308: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 310: `next_core_output_capture_valid = core_output_capture_valid;` This makes next_core_output_capture_valid the producer's statement that its current payload is meaningful, equal to 'core_output_capture_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_output_capture_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 311: `unique case ({core_output_capture_push, core_output_capture_pop})` This starts a multiplexer/state decode selected by '{core_output_capture_push, core_output_capture_pop}'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 312: `2'b10: next_core_output_capture_valid = 1'b1;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b10: the value named 'b10' in this block; next_core_output_capture_valid: the producer's statement that its current payload is meaningful; b1: the value named 'b1' in this block.

Line 313: `2'b01: next_core_output_capture_valid = 1'b0;` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b01: the value named 'b01' in this block; next_core_output_capture_valid: the producer's statement that its current payload is meaningful; b0: the value named 'b0' in this block.

Line 314: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the combinational process whose outputs respond without waiting for a clock edge.

Line 315: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 316: `// This next-cycle credit is conservative when a full FIFO pops: the` Design intent: This next-cycle credit is conservative when a full FIFO pops: the This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 317: `// resulting single restart bubble prevents any combinational READY` Design intent: resulting single restart bubble prevents any combinational READY This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 318: `// path while preserving one result per cycle during steady traffic.` Design intent: path while preserving one result per cycle during steady traffic. This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 319: `next_core_m_ready = !next_core_output_capture_valid` This statement continues through source line 320; the explanation uses the complete expression rather than only the first visual fragment. This makes next_core_m_ready, the consumer's statement that it can accept a payload this cycle, equal to '!next_core_output_capture_valid || next_output_fifo_co != 2'. It is driven by combinational or procedural logic in the current evaluation. The important operands are next_core_output_capture_valid: the producer's statement that its current payload is meaningful; next_output_fifo_count: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 320: `|| next_output_fifo_count != 2;` This is continuation line 2 of the 2-line logical statement that begins at source line 319. Its exact fragment is '|| next_output_fifo_count != 2;'. The important operands are next_output_fifo_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This makes next_core_m_ready, the consumer's statement that it can accept a payload this cycle, equal to '!next_core_output_capture_valid || next_output_fifo_count != 2'. It is driven by combinational or procedural logic in the current evaluation. The important operands are next_core_output_capture_valid: the producer's statement that its current payload is meaningful;

next_output_fifo_count: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 321: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 323: `// Two entries isolate external backpressure from every internal engine while` Design intent: Two entries isolate external backpressure from every internal engine while This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 324: `// still allowing simultaneous enqueue/dequeue at one result per clock.` Design intent: still allowing simultaneous enqueue/dequeue at one result per clock. This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 325: `function automatic logic [31:0] apply_write_strobes(` Within the combinational helper apply write strobes, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; apply_write_strobes: the value named 'apply write strobes' in this block.

Line 326: `input logic [31:0] old_value,` This declares old_value as a input port with width [31:0]. It carries the value named 'old value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 327: `input logic [31:0] new_value,` This declares new_value as a input port with width [31:0]. It carries the value named 'new value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 328: `input logic [3:0] strobes` This declares strobes as a input port with width [3:0]. It carries the value named 'strobes' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 329: `);` Within the combinational helper apply write strobes, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The expression is a literal or structural operator and has no hidden software state.

Line 330: `logic [31:0] merged;` This declares hardware signal storage or wiring in the combinational helper apply write strobes. Specifically, merged is the value named 'merged' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 331: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper apply write strobes; it has no arithmetic value of its own.

Line 332: `merged = old_value;` This makes merged, the value named 'merged' in this block, equal to ‘old_value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are old_value: the value named 'old value' in this block. Context: the combinational helper apply write strobes.

Line 333: `for (int byte_index = 0; byte_index < 4; byte_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational helper apply write strobes.

Line 334: `if (strobes[byte_index]) begin` This condition selects the following hardware branch only when ‘strobes[byte_inde is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are strobes: the value named 'strobes' in this block; byte_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational helper apply write strobes.

Line 335: `merged[byte_index*8 +: 8] = new_value[byte_index*8 +: 8];` Within the combinational helper apply write strobes, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are merged: the value named 'merged' in this block; byte_index: an integer selecting one tensor element, lane, bank, or state entry; new_value: the value named 'new value' in this block.

Line 336: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper apply write strobes; it has no arithmetic value of its own.

Line 337: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper apply write strobes; it has no arithmetic value of its own.

Line 338: `apply_write_strobes = merged;` This makes apply_write_strobes, the value named 'apply write strobes' in this block, equal to ‘merged‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are merged: the value named 'merged' in this block. Context: the combinational helper apply write strobes.

Line 339: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper apply write strobes; it has no arithmetic value of its own.

Line 340: `endfunction` Within the combinational helper apply write strobes, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are endfunction: the value named 'endfunction' in this block.

Line 342: `function automatic logic [31:0] register_read(input logic [7:0] address);` Within the combinational helper register read, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; register_read: the value named 'register read' in this block.

Line 343: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper register read; it has no arithmetic value of its own.

Line 344: `unique case (address)` This starts a multiplexer/state decode selected by ‘address‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 345: `ADDR_CONTROL: register_read = {` This statement continues through source line 347; the explanation uses the complete expression rather than only the first visual fragment. This is the ADDR_CONTROL case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = { 23'd0, interrupt_enable, 5'd0, program_active, 1'b0, command_pending};‘; context is the combinational helper register read Within state ADDR_CONTROL, the machine performs the named control phase..

Line 346: `23'd0, interrupt_enable, 5'd0, program_active,` This is continuation line 2 of the 3-line logical statement that begins at source line 345. Its exact fragment is ‘23'd0, interrupt_enable, 5'd0, program_active,‘. The important operands are d0: the value named 'd0' in this block; interrupt_enable: the hardware notification that wakes software after completion or error; program_active: the value named 'program active' in this block. Read together, the complete operation is: This is the ADDR_CONTROL case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = { 23'd0, interrupt_enable, 5'd0, program_active, 1'b0, command_pending};‘; context is the combinational helper register read Within state ADDR_CONTROL, the machine performs the named control phase..

Line 347: `1'b0, command_pending};` This is continuation line 3 of the 3-line logical statement that begins at source line 345. Its exact fragment is '1'b0, command_pending};'. The important operands are b0: the value named 'b0' in this block; command_pending: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This is the ADDR_CONTROL case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = { 23'd0, interrupt_enable, 5'd0, program_active, 1'b0, command_pending};'; context is the combinational helper register read Within state ADDR_CONTROL, the machine performs the named control phase..

Line 348: `ADDR_STATUS: register_read = {` This statement continues through source line 351; the explanation uses the complete expression rather than only the first visual fragment. This is the ADDR_STATUS case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = { 26'd0, program_active, interrupt, command_pending, done_sticky, core_error || wrapper_error || program_error, core_busy || command_pending || program_active};'; context is the combinational helper register read Within state ADDR_STATUS, the machine performs the named control phase..

Line 349: `26'd0, program_active, interrupt, command_pending,` This is continuation line 2 of the 4-line logical statement that begins at source line 348. Its exact fragment is '26'd0, program_active, interrupt, command_pending,'. The important operands are d0: the value named 'd0' in this block; program_active: the value named 'program active' in this block; interrupt: the hardware notification that wakes software after completion or error. Read together, the complete operation is: This is the ADDR_STATUS case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = { 26'd0, program_active, interrupt, command_pending, done_sticky, core_error || wrapper_error || program_error, core_busy || command_pending || program_active};'; context is the combinational helper register read Within state ADDR_STATUS, the machine performs the named control phase..

Line 350: `done_sticky, core_error || wrapper_error || program_error,` This is continuation line 3 of the 4-line logical statement that begins at source line 348. Its exact fragment is 'done_sticky, core_error || wrapper_error || program_error,'. The important operands are done_sticky: the value named 'done sticky' in this block; core_error: the value named 'core error' in this block; wrapper_error: the value named 'wrapper error' in this block. Read together, the complete operation is: This is the ADDR_STATUS case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = { 26'd0, program_active, interrupt, command_pending, done_sticky, core_error || wrapper_error || program_error, core_busy || command_pending || program_active};'; context is the combinational helper register read Within state ADDR_STATUS, the machine performs the named control phase..

Line 351: `core_busy || command_pending || program_active};` This is continuation line 4 of the 4-line logical statement that begins at source line 348. Its exact fragment is 'core_busy || command_pending || program_active};'. The important operands are core_busy: the value named 'core busy' in this block; command_pending: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; program_active: the value named 'program active' in this block. Read together, the complete operation is: This is the ADDR_STATUS case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = { 26'd0, program_active, interrupt, command_pending, done_sticky, core_error || wrapper_error || program_error, core_busy || command_pending || program_active};'; context is the combinational helper register read Within state ADDR_STATUS, the machine performs the named control phase..

Line 352: `ADDR_ACTIVE_OPCODE: register_read = {28'd0, core_active_opcode};` This is the ADDR_ACTIVE_OPCODE case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = {28'd0, core_active_opcode};'; context is the combinational helper register read Within state ADDR_ACTIVE_OPCODE, the machine performs the named control phase..

Line 353: `ADDR_COMMAND_COUNT: register_read = core_command_count;` This is the ADDR_COMMAND_COUNT case branch, which updates a segmented telemetry counter without a long carry chain. The same-cycle action begins with 'register_read = core_command_count;'; context is the combinational helper register read Within state ADDR_COMMAND_COUNT, the machine updates a segmented telemetry counter without a long carry chain..

Line 354: `ADDR_COMMAND0: register_read = command_register[0];` This is the ADDR_COMMAND0 case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = command_register[0];';

context is the combinational helper register read Within state ADDR_COMMAND0, the machine performs the named control phase..

Line 355: `ADDR_COMMAND1: register_read = command_register[1];` This is the ADDR_COMMAND1 case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = command_register[1];‘; context is the combinational helper register read Within state ADDR_COMMAND1, the machine performs the named control phase..

Line 356: `ADDR_COMMAND2: register_read = command_register[2];` This is the ADDR_COMMAND2 case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = command_register[2];‘; context is the combinational helper register read Within state ADDR_COMMAND2, the machine performs the named control phase..

Line 357: `ADDR_COMMAND3: register_read = command_register[3];` This is the ADDR_COMMAND3 case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = command_register[3];‘; context is the combinational helper register read Within state ADDR_COMMAND3, the machine performs the named control phase..

Line 358: `ADDR_INPUT_BEATS: register_read = core_input_beats;` This is the ADDR_INPUT_BEATS case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = core_input_beats;‘; context is the combinational helper register read Within state ADDR_INPUT_BEATS, the machine performs the named control phase..

Line 359: `ADDR_OUTPUT_BEATS: register_read = core_output_beats;` This is the ADDR_OUTPUT_BEATS case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘register_read = core_output_beats;‘; context is the combinational helper register read Within state ADDR_OUTPUT_BEATS, the machine presents computed results under valid/ready backpressure..

Line 360: `ADDR_TARGET_CLOCK: register_read = 32'd200000000;` This is the ADDR_TARGET_CLOCK case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = 32'd200000000;‘; context is the combinational helper register read Within state ADDR_TARGET_CLOCK, the machine performs the named control phase..

Line 361: `// Bits correspond to tensor, binary/ternary, alpha-XNOR,` Design intent: Bits correspond to tensor, binary/ternary, alpha-XNOR, This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 362: `// vector/LUT, normalization, LIF, optimizer, and backward tile.` Design intent: vector/LUT, normalization, LIF, optimizer, and backward tile. This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 363: `ADDR_CAPABILITIES: register_read = 32'h000001ff;` This is the ADDR_CAPABILITIES case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = 32'h000001ff;‘; context is the combinational helper register read Within state ADDR_CAPABILITIES, the machine performs the named control phase..

Line 364: `ADDR_VERSION: register_read = 32'h00010007;` This is the ADDR_VERSION case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = 32'h00010007;‘; context is the combinational helper register read Within state ADDR_VERSION, the machine performs the named control phase..

Line 365: `ADDR_ID: register_read = 32'h4e455552;` This is the ADDR_ID case branch, which performs this explicitly named phase. The same-cycle action begins with ‘register_read = 32'h4e455552;‘; context is the combinational helper register read Within state ADDR_ID, the machine performs the named control phase..

Line 366: `ADDR_TENSOR_LOADED: register_read = {` This statement continues through source line 368; the explanation uses the complete expression rather than only the first visual fragment. This is the ADDR_TENSOR_LOADED case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'register_read = { 4'd0, core_tensor_loaded_weights, 4'd0, core_tensor_loaded_activations};'; context is the combinational helper register read Within state ADDR_TENSOR_LOADED, the machine stores incoming operands into local FPGA memory..

Line 367: `4'd0, core_tensor_loaded_weights,` This is continuation line 2 of the 3-line logical statement that begins at source line 366. Its exact fragment is '4'd0, core_tensor_loaded_weights,'. The important operands are d0: the value named 'd0' in this block; core_tensor_loaded_weights: a learned coefficient multiplied by an activation. Read together, the complete operation is: This is the ADDR_TENSOR_LOADED case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'register_read = { 4'd0, core_tensor_loaded_weights, 4'd0, core_tensor_loaded_activations};'; context is the combinational helper register read Within state ADDR_TENSOR_LOADED, the machine stores incoming operands into local FPGA memory..

Line 368: `4'd0, core_tensor_loaded_activations};` This is continuation line 3 of the 3-line logical statement that begins at source line 366. Its exact fragment is '4'd0, core_tensor_loaded_activations};'. The important operands are d0: the value named 'd0' in this block; core_tensor_loaded_activations: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This is the ADDR_TENSOR_LOADED case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'register_read = { 4'd0, core_tensor_loaded_weights, 4'd0, core_tensor_loaded_activations};'; context is the combinational helper register read Within state ADDR_TENSOR_LOADED, the machine stores incoming operands into local FPGA memory..

Line 369: `ADDR_TENSOR_ACTIVE: register_read = {20'd0, core_tensor_active_issues};` This is the ADDR_TENSOR_A case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = {20'd0, core_tensor_active_issues};'; context is the combinational helper register read Within state ADDR_TENSOR_ACTIVE, the machine performs the named control phase..

Line 370: `ADDR_TENSOR_CYCLES: register_read = core_tensor_compute_cycles;` This is the ADDR_TENSOR_CYCLES case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_tensor_compute_ context is the combinational helper register read Within state ADDR_TENSOR_CYCLES, the machine performs the named control phase..

Line 371: `ADDR_TENSOR_SKIPPED: register_read = core_tensor_skipped_issues;` This is the ADDR_TENSOR_SKIPPEI case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_tensor_skipped_i context is the combinational helper register read Within state ADDR_TENSOR_SKIPPED, the machine performs the named control phase..

Line 372: `ADDR_DENSE_MACS_LOW: register_read = core_tensor_dense_macs[31:0];` This is the ADDR_DENSE_MACS_L case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_tensor_dense_ma context is the combinational helper register read Within state ADDR_DENSE_MACS_LOW, the machine performs the named control phase..

Line 373: `ADDR_DENSE_MACS_HIGH: register_read = core_tensor_dense_macs[63:32];` This is the ADDR_DENSE_MACS case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_tensor_dense_ma context is the combinational helper register read Within state ADDR_DENSE_MACS_HIGH, the machine performs the named control phase..

Line 374: `ADDR_SPIKE_ADDS_LOW: register_read = core_tensor_spike_adds[31:0];` This is the ADDR_SPIKE_ADDS_L case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_tensor_spike_ad context is the combinational helper register read Within state ADDR_SPIKE_ADDS_LOW, the machine performs the named control phase..

Line 375: `ADDR_SPIKE_ADDS_HIGH: register_read = core_tensor_spike_adds[63:32];` This is the ADDR_SPIKE_ADDS_ case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_tensor_spike_ad context is the combinational helper register read Within state ADDR_SPIKE_ADDS_HIGH, the machine performs the named control phase..

Line 376: `ADDR_LAYERNORM_CYCLES: register_read = core_layernorm_cycles;` This is the ADDR_LAYERNORM_CYCLE case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_layernorm_cycles context is the combinational helper register read Within state ADDR_LAYERNORM_CYCLES, the machine performs the named control phase..

Line 377: `ADDR_LAYERNORM_MEAN: register_read = core_layernorm_mean;` This is the ADDR_LAYERNORM_MEAN case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_layernorm_mean; context is the combinational helper register read Within state ADDR_LAYERNORM_MEAN, the machine performs the named control phase..

Line 378: `ADDR_LAYERNORM_VARIANCE: register_read = core_layernorm_variance;` This is the ADDR_LAYERNORM_VA case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_layernorm_varian context is the combinational helper register read Within state ADDR_LAYERNORM_VARIANCE, the machine performs the named control phase..

Line 379: `ADDR_LAYERNORM_SIGMA: register_read = core_layernorm_sigma;` This is the ADDR_LAYERNORM_SIGMA case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_layernorm_sigma context is the combinational helper register read Within state ADDR_LAYERNORM_SIGMA, the machine performs the named control phase..

Line 380: `ADDR_SOFTMAX_CYCLES: register_read = core_softmax_cycles;` This is the ADDR_SOFTMAX_CYCLES case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_softmax_cycles;'; context is the combinational helper register read Within state ADDR_SOFTMAX_CYCLES, the machine performs the named control phase..

Line 381: `ADDR_SOFTMAX_MAXIMUM: register_read =` This statement continues through source line 382; the explanation uses the complete expression rather than only the first visual fragment. This is the ADDR_SOFTMAX_MAXIMUM case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = {{8{core_softmax_ma core_softmax_maximum};'; context is the combinational helper register read Within state ADDR_SOFTMAX_MAXIMUM, the machine performs the named control phase..

Line 382: `{{8{core_softmax_maximum[23]}}, core_softmax_maximum};` This is continuation line 2 of the 2-line logical statement that begins at source line 381. Its exact fragment is '{{8{core_softmax_maximum[23]}}, core_softmax_maximum};'. The important operands are core_softmax_maximum: the value named 'core softmax maximum' in this block. Read together, the complete operation is: This is the ADDR_SOFTMAX_MAXIMUM case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = {{8{core_softmax_maximum[23]}}, core_softmax_maximun context is the combinational helper register read Within state ADDR_SOFTMAX_MAXIMUM, the machine performs the named control phase..

Line 383: `ADDR_SOFTMAX_DENOMINATOR: register_read =` This statement continues through source line 384; the explanation uses the complete expression rather than only the first visual fragment. This is the ADDR_SOFTMAX_DENOMINATOR case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = {8'd0, core_softmax_denominator};'; context is the combinational helper register read Within state ADDR_SOFTMAX_DENOMINATO the machine performs the named control phase..

Line 384: `{8'd0, core_softmax_denominator};` This is continuation line 2 of the 2-line logical statement that begins at source line 383. Its exact fragment is '{8'd0, core_softmax_denominator};'. The important operands are d0: the value named 'd0' in this block; core_softmax_denominator: the value named 'core softmax denominator' in this block. Read together, the complete operation is: This is the ADDR_SOFTMAX_DENOMINATOR case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = {8'd0, core_softmax_denominator};';

context is the combinational helper register read Within state ADDR_SOFTMAX_DENOMINATOR, the machine performs the named control phase..

Line 385: `ADDR_LIF_SPIKES_LOW: register_read = core_lif_total_spikes[31:0];` This is the ADDR_LIF_SPIKES_LOW case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_lif_total_spikes[3 context is the combinational helper register read Within state ADDR_LIF_SPIKES_LOW, the machine performs the named control phase..

Line 386: `ADDR_LIF_SPIKES_HIGH: register_read = core_lif_total_spikes[63:32];` This is the ADDR_LIF_SPIKES_H case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_lif_total_spikes[6 context is the combinational helper register read Within state ADDR_LIF_SPIKES_HIGH, the machine performs the named control phase..

Line 387: `ADDR_OPTIMIZER_UPDATES: register_read = core_optimizer_updates;` This is the ADDR_OPTIMIZER_UPDA case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_optimizer_update context is the combinational helper register read Within state ADDR_OPTIMIZER_UPDATES, the machine performs the named control phase..

Line 388: `ADDR_LUT_ENTRIES: register_read = {23'd0, core_lut_loaded_entries};` This is the ADDR_LUT_ENTRIES case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = {23'd0, core_lut_loaded_entries};'; context is the combinational helper register read Within state ADDR_LUT_ENTRIES, the machine performs the named control phase..

Line 389: `ADDR_LAYERNORM_RECIPROCAL: register_read = core_layernorm_reciprocal;` This is the ADDR_LAYERNORM case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_layernorm_recipr context is the combinational helper register read Within state ADDR_LAYERNORM_RECIPROCAL, the machine performs the named control phase..

Line 390: `ADDR_SOFTMAX_RECIPROCAL: register_read = core_softmax_reciprocal;` This is the ADDR_SOFTMAX_RECI case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_softmax_reciproc context is the combinational helper register read Within state ADDR_SOFTMAX_RECIPROCAL, the machine performs the named control phase..

Line 391: `ADDR_PROGRAM_RECORDS: register_read = program_records;` This is the ADDR_PROGRAM_RECORDS case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = program_records;'; context is the combinational helper register read Within state ADDR_PROGRAM_RECORDS, the machine performs the named control phase..

Line 392: `ADDR_PROGRAM_OUTPUT_BEATS: register_read = program_output_beats_r;` This is the ADDR_PROGRAM_OUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'register_read = program_output_beats_r;'; context is the combinational helper register read Within state ADDR_PROGRAM_OUTP the machine presents computed results under valid/ready backpressure..

Line 393: `ADDR_TRAINING_TILES: register_read = core_training_tiles;` This is the ADDR_TRAINING_TILES case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_training_tiles;'; context is the combinational helper register read Within state ADDR_TRAINING_TILES, the machine performs the named control phase..

Line 394: `ADDR_TRAINING_CYCLES: register_read = core_training_cycles;` This is the ADDR_TRAINING_CYCLES case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = core_training_cycles;'; context is the combinational helper register read Within state ADDR_TRAINING_CYCLES, the machine performs the named control phase..

Line 395: `default: register_read = 32'd0;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'register_read = 32'd0;'; context is the combinational helper register read Within state ADDR_TRAINING_CYCLES, the machine performs the named control phase..

Line 396: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational helper register read Within state ADDR_TRAINING_CYCLES, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 397: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper register read; it has no arithmetic value of its own.

Line 398: `endfunction` Within the combinational helper register read, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are endfunction: the value named 'endfunction' in this block.

Line 400: `neuraedge_descriptor_sequencer u_descriptor_sequencer (` This instantiates module neuraedge_descriptor_sec as u_descriptor_sequencer. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 401: `.clk(aclk), .rst_n(aresetn),` This connects the child instance's named port 'clk' to parent expression 'aclk), .rst_n(aresetn'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 402: `.program_start(program_start_r),` This connects the child instance's named port 'program_start' to parent expression 'program_start_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'program start r' in this block.

Line 403: `.clear_status(program_clear_r),` This connects the child instance's named port 'clear_status' to parent expression 'program_clear_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'program clear r' in this block.

Line 404: `.program_output_beats(program_output_beats_r),` This connects the child instance's named port 'program_output_beats' to parent expression 'program_output_beats_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'program output beats r' in this block.

Line 405: `.program_active(program_active), .program_done(program_done),` This connects the child instance's named port 'program_active' to parent expression 'program_active), .program_done(program_done'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'program active), .program done(program done' in this block.

Line 406: `.program_error(program_error), .program_records(program_records),` This connects the child instance's named port 'program_error' to parent expression 'program_error), .program_records(program_records'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the number of descriptor records accepted since program start.

Line 407: `.s_axis_tvalid(core_input_capture_valid),` This connects the child instance's named port 's_axis_tvalid' to parent expression 'core_input_capture_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 408: `.s_axis_tready(sequencer_s_ready),` This connects the child instance's named port 's_axis_tready' to parent expression 'sequencer_s_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 409: `.s_axis_tdata(core_input_capture_data),` This connects the child instance's named port 's_axis_tdata' to parent expression 'core_input_capture_data'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the payload carried with the surrounding valid/ready handshake.

Line 410: `.s_axis_tlast(core_input_capture_last),` This connects the child instance's named port 's_axis_tlast' to parent expression 'core_input_capture_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 411: `.m_axis_tvalid(sequencer_m_valid),` This connects the child instance's named port 'm_axis_tvalid' to parent expression 'sequencer_m_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 412: `.m_axis_tready(core_m_ready_r),` This connects the child instance's named port 'm_axis_tready' to parent expression 'core_m_ready_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 413: `.m_axis_tdata(sequencer_m_data),` This connects the child instance's named port 'm_axis_tdata' to parent expression 'sequencer_m_data'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the payload carried with the surrounding valid/ready handshake.

Line 414: `.m_axis_tlast(sequencer_m_last),` This connects the child instance's named port 'm_axis_tlast' to parent expression 'sequencer_m_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 415: `.core_command_valid(sequencer_core_command_valid),` This connects the child instance's named port 'core_command_valid' to parent expression 'sequencer_core_command_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 416: `.core_command_ready(core_command_ready),` This connects the child instance's named port 'core_command_ready' to parent expression 'core_command_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 417: `.core_command_data(sequencer_core_command_data),` This connects the child instance's named port 'core_command_data' to parent expression 'sequencer_core_command_data'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 418: `.core_command_done(core_command_done),` This connects the child instance's named port 'core_command_done' to parent expression 'core_command_done'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 419: `.core_status_error(core_error),` This connects the child instance's named port 'core_status_error' to parent expression 'core_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core error' in this block.

Line 420: `.core_s_valid(sequencer_core_s_valid), .core_s_ready(core_s_ready),` This connects the child instance's named port 'core_s_valid' to parent expression 'sequencer_core_s_valid), .core_s_ready(core_s_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 421: `.core_s_data(sequencer_core_s_data), .core_s_last(sequencer_core_s_last),` This connects the child instance's named port 'core_s_data' to parent expression 'sequencer_core_s_data), .core_s_last(sequencer_core_s_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 422: `.core_m_valid(core_m_valid), .core_m_ready(sequencer_core_m_ready),` This connects the child instance's named port 'core_m_valid' to parent expression 'core_m_valid), .core_m_ready(sequencer_core_m_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 423: `.core_m_data(core_m_data), .core_m_last(core_m_last)` This connects the child instance's named port 'core_m_data' to parent expression 'core_m_data), .core_m_last(core_m_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 424: `);` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The expression is a literal or structural operator and has no hidden software state.

Line 426: `neuraedge_universal_core u_core (` This instantiates module neuraedge_universal_core as u_core. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 427: `.clk(aclk), .rst_n(aresetn),` This connects the child instance's named port 'clk' to parent expression 'aclk), .rst_n(aresetn'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 428: `.command_valid(program_active` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are command_valid: the producer's statement that its current payload is meaningful; program_active: the value named 'program active' in this block.

Line 429: `? sequencer_core_command_valid : command_pending),` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are sequencer_core_command_valid: the producer's statement that its current payload is meaningful; command_pending: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 430: `.command_ready(core_command_ready),` This connects the child instance's named port 'command_ready' to parent expression 'core_command_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 431: `.command_data(program_active` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; program_active: the value named 'program active' in this block.

Line 432: `? sequencer_core_command_data : submitted_command),` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are sequencer_core_command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; submitted_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 433: `.command_done(core_command_done),` This connects the child instance's named port 'command_done' to parent expression 'core_command_done'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 434: `.status_busy(core_busy), .status_error(core_error),` This connects the child instance's named port 'status_busy' to parent expression 'core_busy), .status_error(core_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core busy), .status error(core error' in this block.

Line 435: `.status_active_opcode(core_active_opcode),` This connects the child instance's named port 'status_active_opcode' to parent expression 'core_active_opcode'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core active opcode' in this block.

Line 436: `.status_command_count(core_command_count),` This connects the child instance's named port 'status_command_count' to parent expression 'core_command_count'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 437: `.status_input_beats(core_input_beats), .status_output_beats(core_output_beats),` This connects the child instance's named port 'status_input_beats' to parent expression 'core_input_beats), .status_output_beats(core_ou
The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core input beats), .status output beats(core output beats' in this block.

Line 438: `.debug_tensor_loaded_activations(core_tensor_loaded_activations),` This connects the child instance's named port 'debug_tensor_loaded_activations' to parent expression 'core_tensor_loaded_activations'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 439: `.debug_tensor_loaded_weights(core_tensor_loaded_weights),` This connects the child instance's named port 'debug_tensor_loaded_weights' to parent expression 'core_tensor_loaded_weights'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 440: `.debug_tensor_active_issues(core_tensor_active_issues),` This connects the child instance's named port 'debug_tensor_active_issues' to parent expression 'core_tensor_active_issues'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core tensor active issues' in this block.

Line 441: `.debug_tensor_compute_cycles(core_tensor_compute_cycles),` This connects the child instance's named port 'debug_tensor_compute_cycles' to parent expression 'core_tensor_compute_cycles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core tensor compute cycles' in this block.

Line 442: `.debug_tensor_skipped_issues(core_tensor_skipped_issues),` This connects the child instance's named port 'debug_tensor_skipped_issues' to parent expression 'core_tensor_skipped_issues'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core tensor skipped issues' in this block.

Line 443: `.debug_tensor_dense_macs(core_tensor_dense_macs),` This connects the child instance's named port 'debug_tensor_dense_macs' to parent expression 'core_tensor_dense_macs'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a count of ordinary activation-times-weight operations actually issued.

Line 444: `.debug_tensor_spike_adds(core_tensor_spike_adds),` This connects the child instance's named port 'debug_tensor_spike_adds' to parent expression 'core_tensor_spike_adds'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 445: `.debug_layernorm_cycles(core_layernorm_cycles),` This connects the child instance's named port 'debug_layernorm_cycles' to parent expression 'core_layernorm_cycles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core layernorm cycles' in this block.

Line 446: `.debug_layernorm_mean(core_layernorm_mean),` This connects the child instance's named port 'debug_layernorm_mean' to parent expression 'core_layernorm_mean'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the arithmetic average used by normalization.

Line 447: `.debug_layernorm_variance(core_layernorm_variance),` This connects the child instance's named port 'debug_layernorm_variance' to parent expression 'core_layernorm_variance'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the average squared distance from the mean.

Line 448: `.debug_layernorm_sigma(core_layernorm_sigma),` This connects the child instance's named port 'debug_layernorm_sigma' to parent expression 'core_layernorm_sigma'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core layernorm sigma' in this block.

Line 449: `.debug_layernorm_reciprocal(core_layernorm_reciprocal),` This connects the child instance's named port 'debug_layernorm_reciprocal' to parent expression 'core_layernorm_reciprocal'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core layernorm reciprocal' in this block.

Line 450: `.debug_softmax_cycles(core_softmax_cycles),` This connects the child instance's named port 'debug_softmax_cycles' to parent expression 'core_softmax_cycles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core softmax cycles' in this block.

Line 451: `.debug_softmax_maximum(core_softmax_maximum),` This connects the child instance's named port 'debug_softmax_maximum' to parent expression 'core_softmax_maximum'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core softmax maximum' in this block.

Line 452: `.debug_softmax_denominator(core_softmax_denominator),` This connects the child instance's named port 'debug_softmax_denominator' to parent expression 'core_softmax_denominator'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core softmax denominator' in this block.

Line 453: `.debug_softmax_reciprocal(core_softmax_reciprocal),` This connects the child instance's named port 'debug_softmax_reciprocal' to parent expression 'core_softmax_reciprocal'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core softmax reciprocal' in this block.

Line 454: `.debug_lif_total_spikes(core_lif_total_spikes),` This connects the child instance's named port 'debug_lif_total_spikes' to parent expression 'core_lif_total_spikes'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 455: `.debug_optimizer_updates(core_optimizer_updates),` This connects the child instance's named port 'debug_optimizer_updates' to parent expression 'core_optimizer_updates'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core optimizer updates' in this block.

Line 456: `.debug_lut_loaded_entries(core_lut_loaded_entries),` This connects the child instance's named port 'debug_lut_loaded_entries' to parent expression 'core_lut_loaded_entries'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core lut loaded entries' in this block.

Line 457: `.debug_training_tiles(core_training_tiles),` This connects the child instance's named port 'debug_training_tiles' to parent expression 'core_training_tiles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core training tiles' in this block.

Line 458: `.debug_training_cycles(core_training_cycles),` This connects the child instance's named port 'debug_training_cycles' to parent expression 'core_training_cycles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core training cycles' in this block.

Line 459: `.s_axis_tvalid(core_s_valid), .s_axis_tready(core_raw_s_ready),` This connects the child instance's named port ‘s_axis_tvalid‘ to parent expression ‘core_s_valid), .s_axis_tready(core_raw_s_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 460: `.s_axis_tdata(core_s_data), .s_axis_tlast(core_s_last),` This connects the child instance's named port ‘s_axis_tdata‘ to parent expression ‘core_s_data), .s_axis_tlast(core_s_last‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 461: `.m_axis_tvalid(core_raw_m_valid), .m_axis_tready(core_raw_m_ready),` This connects the child instance's named port ‘m_axis_tvalid‘ to parent expression ‘core_raw_m_valid), .m_axis_tready(core_raw_m_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 462: `.m_axis_tdata(core_raw_m_data), .m_axis_tlast(core_raw_m_last)` This connects the child instance's named port ‘m_axis_tdata‘ to parent expression ‘core_raw_m_data), .m_axis_tlast(core_raw_m_last‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 463: `);` Within the module's structural declarations, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The expression is a literal or structural operator and has no hidden software state.

Line 465: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 466: `// A two-entry elastic FIFO makes the external ready path local while` Design intent: A two-entry elastic FIFO makes the external ready path local while This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 467: `// still accepting one input beat per clock during simultaneous pop/push.` Design intent: still accepting one input beat per clock during simultaneous pop/push. This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 468: `s_axis_tready = input_fifo_count != 2;` This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to ‘input_fifo_count != 2‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_fifo_count: an integer count retained for control or measurable telemetry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 469: `s_axi_awready = !aw_hold && !s_axi_bvalid;` This makes s_axi_awready, the consumer's statement that it can accept a payload this cycle, equal to ‘!aw_hold && !s_axi_bvalid‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are aw_hold: the value named 'aw hold' in this block; s_axi_bvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 470: `s_axi_wready = !w_hold && !s_axi_bvalid;` This makes s_axi_wready, the consumer's statement that it can accept a payload this cycle, equal to ‘!w_hold && !s_axi_bvalid‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are w_hold: the value named 'w hold' in this block; s_axi_bvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 471: `s_axi_bresp = 2'b00;` This makes s_axi_bresp, the value named 's axi bresp' in this block, equal to '2'b00'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b00: the value named 'b00' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 472: `s_axi_arready = !s_axi_rvalid;` This makes s_axi_arready, the consumer's statement that it can accept a payload this cycle, equal to '!s_axi_rvalid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are s_axi_rvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 473: `s_axi_rresp = 2'b00;` This makes s_axi_rresp, the value named 's axi rresp' in this block, equal to '2'b00'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b00: the value named 'b00' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 474: `m_axis_tvalid = output_register_valid_r;` This makes m_axis_tvalid, the producer's statement that its current payload is meaningful, equal to 'output_register_valid_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_register_valid_r: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 475: `m_axis_tdata = output_register_data_r;` This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to 'output_register_data_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_register_data_r: the payload carried with the surrounding valid/ready handshake. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 476: `m_axis_tkeep = 8'hff;` This makes m_axis_tkeep, the value named 'm axis tkeep' in this block, equal to '8'hff'. It is driven by combinational or procedural logic in the current evaluation. The important operands are hff: the value named 'hff' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 477: `m_axis_tlast = output_register_last_r;` This makes m_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'output_register_last_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_register_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 478: `interrupt = interrupt_enable && done_sticky;` This makes interrupt, the hardware notification that wakes software after completion or error, equal to 'interrupt_enable && done_sticky'. It is driven by combinational or procedural logic in the current evaluation. The important operands are interrupt_enable: the hardware notification that wakes software after completion or error; done_sticky: the value named 'done sticky' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 479: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 481: `always_ff @(posedge aclk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 482: `if (!aresetn) begin` This is the reset branch for '(!aresetn'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 483: `for (int word_index = 0; word_index < 4; word_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 484: `command_register[word_index] <= 32'd0;` This makes command_register[word_index], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 485: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 486: `submitted_command <= 128'd0;` This makes submitted_command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 487: `command_pending <= 1'b0;` This makes command_pending, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 488: `interrupt_enable <= 1'b0;` This makes interrupt_enable, the hardware notification that wakes software after completion or error, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 489: `done_sticky <= 1'b0;` This makes done_sticky, the value named 'done sticky' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 490: `wrapper_error <= 1'b0;` This makes wrapper_error, the value named 'wrapper error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 491: `program_start_r <= 1'b0;` This makes program_start_r, the value named 'program start r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 492: `program_clear_r <= 1'b0;` This makes program_clear_r, the value named 'program clear r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 493: `program_output_beats_r <= 32'd0;` This makes program_output_beats_r, the value named 'program output beats r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 494: `aw_hold <= 1'b0;` This makes aw_hold, the value named 'aw hold' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 495: `awaddr_hold <= '0;` This makes awaddr_hold, the value named 'awaddr hold' in this block, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 496: `w_hold <= 1'b0;` This makes w_hold, the value named 'w hold' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 497: `wdata_hold <= 32'd0;` This makes wdata_hold, the value named 'wdata hold' in this block, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 498: `wstrb_hold <= 4'd0;` This makes wstrb_hold, the value named 'wstrb hold' in this block, equal to ‘4'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 499: `s_axi_bvalid <= 1'b0;` This makes s_axi_bvalid, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 500: `s_axi_rvalid <= 1'b0;` This makes s_axi_rvalid, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 501: `s_axi_rdata <= 32'd0;` This makes s_axi_rdata, the value named 's axi rdata' in this block, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 502: `output_fifo_count <= 2'd0;` This makes output_fifo_count, an integer count retained for control or measurable telemetry, equal to ‘2'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 503: `core_output_capture_valid <= 1'b0;` This makes core_output_capture_valid, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 504: `core_m_ready_r <= 1'b1;` This makes core_m_ready_r, the consumer's statement that it can accept a payload this cycle, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 505: `core_output_capture_data <= 64'd0;` This makes core_output_capture_data, the payload carried with the surrounding valid/ready handshake, equal to ‘64'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 506: `core_output_capture_last <= 1'b0;` This makes core_output_capture_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 507: `output_fifo_write_pointer <= 1'b0;` This makes output_fifo_write_pointer, the value named 'output fifo write pointer' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 508: `output_fifo_read_pointer <= 1'b0;` This makes output_fifo_read_pointer, the value named 'output fifo read pointer' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 509: `output_fifo_data[0] <= 64'd0;` This makes output_fifo_data[0], the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 510: `output_fifo_data[1] <= 64'd0;` This makes output_fifo_data[1], the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 511: `output_fifo_last[0] <= 1'b0;` This makes output_fifo_last[0], the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 512: `output_fifo_last[1] <= 1'b0;` This makes output_fifo_last[1], the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 513: `output_register_valid_r <= 1'b0;` This makes output_register_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 514: `output_register_data_r <= 64'd0;` This makes output_register_data_r, the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 515: `output_register_last_r <= 1'b0;` This makes output_register_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 516: `input_fifo_count <= 2'd0;` This makes input_fifo_count, an integer count retained for control or measurable telemetry, equal to '2'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 517: `input_fifo_write_pointer <= 1'b0;` This makes input_fifo_write_pointer, the value named 'input fifo write pointer' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 518: `input_fifo_read_pointer <= 1'b0;` This makes input_fifo_read_pointer, the value named 'input fifo read pointer' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 519: `input_fifo_data[0] <= 64'd0;` This makes input_fifo_data[0], the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 520: `input_fifo_data[1] <= 64'd0;` This makes input_fifo_data[1], the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 521: `input_fifo_last[0] <= 1'b0;` This makes input_fifo_last[0], the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 522: `input_fifo_last[1] <= 1'b0;` This makes input_fifo_last[1], the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 523: `core_input_capture_valid <= 1'b0;` This makes core_input_capture_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 524: `core_input_capture_data <= 64'd0;` This makes core_input_capture_data, the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 525: `core_input_capture_last <= 1'b0;` This makes core_input_capture_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 526: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 527: `program_start_r <= 1'b0;` This makes program_start_r, the value named 'program start r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 528: `program_clear_r <= 1'b0;` This makes program_clear_r, the value named 'program clear r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 529: `// The prefetch stage decouples payload storage from deep core-ready` Design intent: The prefetch stage decouples payload storage from deep core-ready This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 530: `// logic. At 200 MHz it accepts one 64-bit beat every two cycles.` Design intent: logic. At 200 MHz it accepts one 64-bit beat every two cycles. This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 531: `if (input_fifo_pop) begin` This condition selects the following hardware branch only when ‘input_fifo_pop‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_fifo_pop: the value named 'input fifo pop' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 532: `core_input_capture_data <= input_fifo_data[input_fifo_read_pointer];` This makes core_input_capture_d the payload carried with the surrounding valid/ready handshake, equal to ‘input_fifo_data[input_fifo_read_pointer]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_fifo_data: the payload carried with the surrounding valid/ready handshake; input_fifo_read_pointer: the value named 'input fifo read pointer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 533: `core_input_capture_last <= input_fifo_last[input_fifo_read_pointer];` This makes core_input_capture_la the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘input_fifo_last[input_fifo_read_pointer]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_fifo_last: the AXI-Stream marker for the final beat of a packet or command payload; input_fifo_read_pointer: the value named 'input fifo read pointer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 534: `core_input_capture_valid <= 1'b1;` This makes core_input_capture_valid, the producer's statement that its current payload is meaningful, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 535: `end else if (core_input_capture_valid && wrapper_input_ready) begin` This condition selects the following hardware branch only when ‘core_input_capture_valid && wrapper_input_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are core_input_capture_valid: the producer's statement that its current payload is meaningful; wrapper_input_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 536: `core_input_capture_valid <= 1'b0;` This makes core_input_capture_valid, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 537: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 538: `if (input_fifo_push) begin` This condition selects the following hardware branch only when ‘input_fifo_push‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_fifo_push: the value named 'input fifo push' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 539: `input_fifo_data[input_fifo_write_pointer] <= s_axis_tdata;` This makes input_fifo_data[input_fifo_write_ the payload carried with the surrounding valid/ready handshake, equal to 's_axis_tdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 540: `input_fifo_last[input_fifo_write_pointer] <= s_axis_tlast;` This makes input_fifo_last[input_fifo_write_p the AXI-Stream marker for the final beat of a packet or command payload, equal to 's_axis_tlast'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 541: `input_fifo_write_pointer <= !input_fifo_write_pointer;` This makes input_fifo_write_pointer, the value named 'input fifo write pointer' in this block, equal to '!input_fifo_write_pointer'. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_fifo_write_pointer: the value named 'input fifo write pointer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 542: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 543: `if (input_fifo_pop) begin` This condition selects the following hardware branch only when 'input_fifo_pop' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_fifo_pop: the value named 'input fifo pop' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 544: `input_fifo_read_pointer <= !input_fifo_read_pointer;` This makes input_fifo_read_pointer, the value named 'input fifo read pointer' in this block, equal to '!input_fifo_read_pointer'. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_fifo_read_pointer: the value named 'input fifo read pointer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 545: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 546: `unique case ({input_fifo_push, input_fifo_pop})` This starts a multiplexer/state decode selected by '{input_fifo_push, input_fifo_pop}'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 547: `2'b10: input_fifo_count <= input_fifo_count + 1'b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b10: the value named 'b10' in this block; input_fifo_count: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block.

Line 548: `2'b01: input_fifo_count <= input_fifo_count - 1'b1;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are b01: the value named 'b01' in this block; input_fifo_count: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block.

Line 549: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the clocked process that updates registers only on a rising clock edge.

Line 550: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 551: `// Sampling an invalid payload is harmless because the separate` Design intent: Sampling an invalid payload is harmless because the separate This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 552: `// valid bit qualifies it. Enabling on local registered credit,` Design intent: valid bit qualifies it. Enabling on local registered credit, This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 553: `// rather than deep VALID && READY logic, keeps all 65 payload` Design intent: rather than deep VALID && READY logic, keeps all 65 payload This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 554: `// clock-enables local to this wrapper.` Design intent: clock-enables local to this wrapper. This comment anchors the surrounding logic in the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

Line 555: `if (core_m_ready_r) begin` This condition selects the following hardware branch only when ‘core_m_ready_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are core_m_ready_r: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 556: `core_output_capture_data <= wrapper_output_data;` This makes core_output_capture_data, the payload carried with the surrounding valid/ready handshake, equal to ‘wrapper_output_data‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are wrapper_output_data: the payload carried with the surrounding valid/ready handshake. Context: the clocked process that updates registers only on a rising clock edge.

Line 557: `core_output_capture_last <= wrapper_output_last;` This makes core_output_capture_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘wrapper_output_last‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are wrapper_output_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 558: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 559: `core_output_capture_valid <= next_core_output_capture_valid;` This makes core_output_capture_valid, the producer's statement that its current payload is meaningful, equal to ‘next_core_output_capture_valid‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_core_output_capture_va the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 560: `core_m_ready_r <= next_core_m_ready;` This makes core_m_ready_r, the consumer's statement that it can accept a payload this cycle, equal to ‘next_core_m_ready‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_core_m_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 561: `if (output_fifo_push) begin` This condition selects the following hardware branch only when ‘output_fifo_push‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_fifo_push: the value named 'output fifo push' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 562: `output_fifo_data[output_fifo_write_pointer]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are output_fifo_data: the payload carried with the surrounding valid/ready handshake; output_fifo_write_pointer: the value named 'output fifo write pointer' in this block.

Line 563: `<= core_output_capture_data;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are core_output_capture_data: the payload carried with the surrounding valid/ready handshake.

Line 564: `output_fifo_last[output_fifo_write_pointer]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are output_fifo_last: the AXI-Stream marker for the final beat of a packet or command payload; output_fifo_write_pointer: the value named 'output fifo write pointer' in this block.

Line 565: `<= core_output_capture_last;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are core_output_capture_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 566: `output_fifo_write_pointer <= !output_fifo_write_pointer;` This makes output_fifo_write_pointer, the value named 'output fifo write pointer' in this block, equal to '!output_fifo_write_pointer'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_fifo_write_pointer: the value named 'output fifo write pointer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 567: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 568: `if (output_fifo_pop) begin` This condition selects the following hardware branch only when 'output_fifo_pop' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_fifo_pop: the value named 'output fifo pop' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 569: `output_fifo_read_pointer <= !output_fifo_read_pointer;` This makes output_fifo_read_pointer, the value named 'output fifo read pointer' in this block, equal to '!output_fifo_read_pointer'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_fifo_read_pointer: the value named 'output fifo read pointer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 570: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 571: `output_fifo_count <= next_output_fifo_count;` This makes output_fifo_count, an integer count retained for control or measurable telemetry, equal to 'next_output_fifo_count'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_output_fifo_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge.

Line 572: `if (output_register_ready) begin` This condition selects the following hardware branch only when 'output_register_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_register_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 573: `output_register_valid_r <= output_fifo_count != 0;` This makes output_register_valid_r, the producer's statement that its current payload is meaningful, equal to 'output_fifo_count != 0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_fifo_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge.

Line 574: `if (output_fifo_count != 0) begin` This condition selects the following hardware branch only when 'output_fifo_count != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_fifo_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge.

Line 575: `output_register_data_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are output_register_data_r: the payload carried with the surrounding valid/ready handshake.

Line 576: `<= output_fifo_data[output_fifo_read_pointer];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are output_fifo_data: the payload carried with the surrounding valid/ready handshake; output_fifo_read_pointer: the value named 'output fifo read pointer' in this block.

Line 577: `output_register_last_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are output_register_last_r: the AXI-Stream marker for the final beat of a packet or command payload.

Line 578: `<= output_fifo_last[output_fifo_read_pointer];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are output_fifo_last: the AXI-Stream marker for the final beat of a packet or command payload; output_fifo_read_pointer: the value named 'output fifo read pointer' in this block.

Line 579: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 580: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 581: `if (command_pending && !program_active && core_command_ready) begin` This condition selects the following hardware branch only when 'command_pending && !program_active && core_command_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are command_pending: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; program_active: the value named 'program active' in this block; core_command_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 582: `command_pending <= 1'b0;` This makes command_pending, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 583: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 584: `if ((!program_active && core_command_done) || program_done) begin` This condition selects the following hardware branch only when ‘(!program_active && core_command_done) || program_done‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are program_active: the value named 'program active' in this block; core_command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; program_done: the value named 'program done' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 585: `done_sticky <= 1'b1;` This makes done_sticky, the value named 'done sticky' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 586: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 587: `if (input_fifo_push && s_axis_tkeep != 8'hff) begin` This condition selects the following hardware branch only when ‘input_fifo_push && s_axis_tkeep != 8'hff‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_fifo_push: the value named 'input fifo push' in this block; s_axis_tkeep: the value named 's axis tkeep' in this block; hff: the value named 'hff' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 588: `wrapper_error <= 1'b1;` This makes wrapper_error, the value named 'wrapper error' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 589: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 591: `if (s_axi_awvalid && s_axi_awready) begin` This condition selects the following hardware branch only when ‘s_axi_awvalid && s_axi_awready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axi_awvalid: the producer's statement that its current payload is meaningful; s_axi_awready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 592: `aw_hold <= 1'b1;` This makes aw_hold, the value named 'aw hold' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 593: `awaddr_hold <= s_axi_awaddr;` This makes awaddr_hold, the value named 'awaddr hold' in this block, equal to ‘s_axi_awaddr‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axi_awaddr: the value named 's axi awaddr' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 594: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 595: `if (s_axi_wvalid && s_axi_wready) begin` This condition selects the following hardware branch only when ‘s_axi_wvalid && s_axi_wready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axi_wvalid: the producer's statement that its current payload is meaningful; s_axi_wready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 596: `w_hold <= 1'b1;` This makes w_hold, the value named 'w hold' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 597: `wdata_hold <= s_axi_wdata;` This makes wdata_hold, the value named 'wdata hold' in this block, equal to 's_axi_wdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axi_wdata: the value named 's axi wdata' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 598: `wstrb_hold <= s_axi_wstrb;` This makes wstrb_hold, the value named 'wstrb hold' in this block, equal to 's_axi_wstrb'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axi_wstrb: the value named 's axi wstrb' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 599: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 601: `if (aw_hold && w_hold && !s_axi_bvalid) begin` This condition selects the following hardware branch only when 'aw_hold && w_hold && !s_axi_bvalid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are aw_hold: the value named 'aw hold' in this block; w_hold: the value named 'w hold' in this block; s_axi_bvalid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 602: `unique case (awaddr_hold[7:0])` This starts a multiplexer/state decode selected by 'awaddr_hold[7:0]'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 603: `ADDR_CONTROL: begin` This is the ADDR_CONTROL case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 604: `if (wstrb_hold[1]) interrupt_enable <= wdata_hold[8];` This condition selects the following hardware branch only when 'wstrb_hold[1]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are wstrb_hold: the value named 'wstrb hold' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 605: `if (wstrb_hold[0] && wdata_hold[1]) begin` This condition selects the following hardware branch only when 'wstrb_hold[0] && wdata_hold[1]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are wstrb_hold: the value named 'wstrb hold' in this block; wdata_hold: the value named 'wdata hold' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 606: `done_sticky <= 1'b0;` This makes done_sticky, the value named 'done sticky' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 607: `wrapper_error <= 1'b0;` This makes wrapper_error, the value named 'wrapper error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 608: `program_clear_r <= 1'b1;` This makes program_clear_r, the value named 'program clear r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 609: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 610: `if (wstrb_hold[0] && wdata_hold[0]) begin` This condition selects the following hardware branch only when ‘wstrb_hold[0] && wdata_hold[0]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are wstrb_hold: the value named ’wstrb hold’ in this block; wdata_hold: the value named ’wdata hold’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 611: `if (command_pending || program_active) begin` This condition selects the following hardware branch only when ‘command_pending || program_active‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are command_pending: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; program_active: the value named ’program active’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 612: `wrapper_error <= 1'b1;` This makes wrapper_error, the value named ’wrapper error’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 613: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 614: `submitted_command <= {command_register[3], command_register[2],` This statement continues through source line 615; the explanation uses the complete expression rather than only the first visual fragment. This makes submitted_command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘{command_register[3], command_register[2], command_register[1], command_register[0]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_register: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 615: `command_register[1], command_register[0]};` This is continuation line 2 of the 2-line logical statement that begins at source line 614. Its exact fragment is ‘command_register[1], command_register[0]};‘. The important operands are command_register: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This makes submitted_command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘{command_register[3], command_register[2], command_register[1], command_register[0]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_register: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 616: `command_pending <= 1'b1;` This makes command_pending, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 617: `done_sticky <= 1'b0;` This makes done_sticky, the value named ’done sticky’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 618: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 619: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 620: `if (wstrb_hold[0] && wdata_hold[2]) begin` This condition selects the following hardware branch only when ‘wstrb_hold[0] && wdata_hold[2]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are wstrb_hold: the value named 'wstrb hold' in this block; wdata_hold: the value named 'wdata hold' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 621: `if (command_pending || program_active) begin` This condition selects the following hardware branch only when ‘command_pending || program_active‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are command_pending: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; program_active: the value named 'program active' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 622: `wrapper_error <= 1'b1;` This makes wrapper_error, the value named 'wrapper error' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 623: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 624: `program_start_r <= 1'b1;` This makes program_start_r, the value named 'program start r' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 625: `done_sticky <= 1'b0;` This makes done_sticky, the value named 'done sticky' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase..

Line 626: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 627: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 628: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ADDR_CONTROL, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 629: `ADDR_COMMAND0: command_register[0] <= apply_write_strobes(` This statement continues through source line 630; the explanation uses the complete expression rather than only the first visual fragment. This is the ADDR_COMMAND0 case branch, which performs this explicitly named phase. The same-cycle action begins with 'command_register[0] <= apply_write_strobes(command_register[0], wdata_hold, wstrb_hold);'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_COMMAND0, the machine performs the named control phase..

Line 630: `command_register[0], wdata_hold, wstrb_hold);` This is continuation line 2 of the 2-line logical statement that begins at source line 629. Its exact fragment is 'command_register[0], wdata_hold, wstrb_hold);'. The important operands are command_register: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; wdata_hold: the value named 'wdata hold' in this block; wstrb_hold: the value named 'wstrb hold' in this block. Read together, the complete operation is: This is the ADDR_COMMAND0 case branch, which performs this explicitly named phase. The same-cycle action begins with 'command_register[0] <= apply_write_strobes(command_register[0], wdata_hold, wstrb_hold);'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_COMMAND0, the machine performs the named control phase..

Line 631: `ADDR_COMMAND1: command_register[1] <= apply_write_strobes(` This statement continues through source line 632; the explanation uses the complete expression rather than only the first visual fragment. This is the ADDR_COMMAND1 case branch, which performs this explicitly named phase. The same-cycle action begins with 'command_register[1] <= apply_write_strobes(command_register[1], wdata_hold, wstrb_hold);'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_COMMAND1, the machine performs the named control phase..

Line 632: `command_register[1], wdata_hold, wstrb_hold);` This is continuation line 2 of the 2-line logical statement that begins at source line 631. Its exact fragment is 'command_register[1], wdata_hold, wstrb_hold);'. The important operands are command_register: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; wdata_hold: the value named 'wdata hold' in this block; wstrb_hold: the value named 'wstrb hold' in this block. Read together, the complete operation is: This is the ADDR_COMMAND1 case branch, which performs this explicitly named phase. The same-cycle action begins with 'command_register[1] <= apply_write_strobes(command_register[1], wdata_hold, wstrb_hold);'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_COMMAND1, the machine performs the named control phase..

Line 633: `ADDR_COMMAND2: command_register[2] <= apply_write_strobes(` This statement continues through source line 634; the explanation uses the complete expression rather than only the first visual fragment. This is the ADDR_COMMAND2 case branch, which performs this explicitly named phase. The same-cycle action begins with 'command_register[2] <= apply_write_strobes(command_register[2], wdata_hold, wstrb_hold);'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_COMMAND2, the machine performs the named control phase..

Line 634: `command_register[2], wdata_hold, wstrb_hold);` This is continuation line 2 of the 2-line logical statement that begins at source line 633. Its exact fragment is 'command_register[2], wdata_hold, wstrb_hold);'. The important operands are command_register: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; wdata_hold: the value named 'wdata hold' in this block; wstrb_hold: the value named 'wstrb hold' in this block. Read together, the complete operation is: This is the ADDR_COMMAND2 case branch, which performs this explicitly named phase. The same-cycle action begins with 'command_register[2] <= apply_write_strobes(command_register[2], wdata_hold, wstrb_hold);'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_COMMAND2, the machine performs the named control phase..

Line 635: `ADDR_COMMAND3: command_register[3] <= apply_write_strobes(` This statement continues through source line 636; the explanation uses the complete expression rather than only the first visual fragment. This is the ADDR_COMMAND3 case branch, which performs this explicitly named phase. The same-cycle action begins with 'command_register[3] <= apply_write_strobes(command_register[3], wdata_hold, wstrb_hold);'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_COMMAND3, the machine performs the named control phase..

Line 636: `command_register[3], wdata_hold, wstrb_hold);` This is continuation line 2 of the 2-line logical statement that begins at source line 635. Its exact fragment is 'command_register[3], wdata_hold, wstrb_hold);'. The important operands are command_register: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; wdata_hold: the value named 'wdata hold' in this block; wstrb_hold: the value named 'wstrb hold' in this block. Read together, the complete operation is: This is the ADDR_COMMAND3 case branch, which performs this explicitly named phase. The same-cycle action begins with 'command_register[3] <= apply_write_strobes(command_register[3], wdata_hold, wstrb_hold);'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_COMMAND3, the machine performs the named control phase..

Line 637: `ADDR_PROGRAM_OUTPUT_BEATS: program_output_beats_r` This is the ADDR_PROGRAM_OUTPUT_BEATS case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'program_output_beats_r'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_PROGRAM_OUTPUT_BEATS, the machine presents computed results under valid/ready backpressure..

Line 638: `<= apply_write_strobes(` Within the clocked process that updates registers only on a rising clock edge Within state ADDR_PROGRAM_OUTPUT_BEATS, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are apply_write_strobes: the value named 'apply write strobes' in this block.

Line 639: `program_output_beats_r, wdata_hold, wstrb_hold);` Within the clocked process that updates registers only on a rising clock edge Within state ADDR_PROGRAM_OUTPUT_BEATS, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ. The important operands are program_output_beats_r: the value named 'program output beats r' in this block; wdata_hold: the value named 'wdata hold' in this block; wstrb_hold: the value named 'wstrb hold' in this block.

Line 640: `default: wrapper_error <= 1'b1;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'wrapper_error <= 1'b1;'; context is the clocked process that updates registers only on a rising clock edge Within state ADDR_PROGRAM_OUTPUT_BEATS, the machine presents computed results under valid/ready backpressure..

Line 641: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ADDR_PROGRAM_OUTPUT_BEATS, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 642: `aw_hold <= 1'b0;` This makes aw_hold, the value named 'aw hold' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 643: `w_hold <= 1'b0;` This makes w_hold, the value named 'w hold' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 644: `s_axi_bvalid <= 1'b1;` This makes s_axi_bvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 645: `end else if (s_axi_bvalid && s_axi_bready) begin` This condition selects the following hardware branch only when 's_axi_bvalid && s_axi_bready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axi_bvalid: the producer's statement that its current payload is meaningful; s_axi_bready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 646: `s_axi_bvalid <= 1'b0;` This makes s_axi_bvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 647: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 649: `if (s_axi_arvalid && s_axi_arready) begin` This condition selects the following hardware branch only when 's_axi_arvalid && s_axi_arready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axi_arvalid: the producer's statement that its current payload is meaningful; s_axi_arready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 650: `s_axi_rdata <= register_read(s_axi_araddr[7:0]);` This makes s_axi_rdata, the value named 's axi rdata' in this block, equal to 'register_read(s_axi_araddr[7:0])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are register_read: the value named 'register read' in this block; s_axi_araddr: the value named 's axi araddr' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 651: `s_axi_rvalid <= 1'b1;` This makes s_axi_rvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 652: `end else if (s_axi_rvalid && s_axi_rready) begin` This condition selects the following hardware branch only when 's_axi_rvalid && s_axi_rready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axi_rvalid: the producer's statement that its current payload is meaningful; s_axi_rready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 653: `s_axi_rvalid <= 1'b0;` This makes s_axi_rvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 654: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 655: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 656: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 658: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 659: `if (AXI_ADDR_BITS < 8) $fatal(1, "AXI_ADDR_BITS must be at least 8");` This condition selects the following hardware branch only when 'AXI_ADDR_BITS < 8) $fatal(1, "AXI_ADDR_BITS must be at least 8"' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are AXI_ADDR_BITS: the value named 'axi addr bits' in this block; fatal: the value named 'fatal' in this block; must: the value named 'must' in this block. Context: the module's structural declarations.

Line 660: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 661: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 663: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ.

18 rtl/neuraedge_sgd_optimizer8.sv

Role in the complete system: applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state.

18.1 Beginner architecture map

Dataflow responsibility For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_sgd_optimizer8 into physical nets and primitives. The source declares 12 input-port lines, 6 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_DECAY_ABSOLUTE, ST_DECAY_BIAS, ST_DECAY_MULTIPL ST_DECAY_PRODUCT, ST_DECAY_PRODUCT_RELAY, ST_DECAY_ROUND, ST_DECAY_SATURATE, ST_DECAY_SIGN, ST_IDLE, ST_MOMENTUM_ABSOLUTE, ST_MOMENTUM_BIAS, ST_MOMENTUM_MULTIP ST_MOMENTUM_PRODUCT, ST_MOMENTUM_PRODUCT_RELAY, ST_MOMENTUM_ROUND, ST_MOMENTU ST_MOMENTUM_SIGN, ST_OUTPUT and additional states. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates begin as if, else as if, endfunction as for, unique as case, endcase as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

18.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_sgd_optimizer8 #(
5      parameter int LANES = 8
6  ) (
7      input  wire clk,
8      input  wire rst_n,
9      input  wire clear_counters,
10     input  wire input_valid,
11     output logic input_ready,
12     input  wire [LANES*16-1:0] weight_vector,
13     input  wire [LANES*24-1:0] gradient_vector,
14     input  wire [LANES*24-1:0] velocity_vector,
15     input  wire [17:0] learning_rate_q16,
16     input  wire [16:0] momentum_q16,
17     input  wire [16:0] weight_decay_q16,
18     input  wire [4:0] update_right_shift,
19     output logic output_valid,
20     input  wire output_ready,
21     output logic [LANES*16-1:0] updated_weight_vector,
22     output logic [LANES*24-1:0] updated_velocity_vector,
23     output logic config_error,
24     output logic [31:0] update_count
25 );
26     typedef enum logic [4:0] {
27         ST_IDLE = 5'd0,
28         ST_MOMENTUM_MULTIPLY = 5'd1,
29         ST_MOMENTUM_ABSOLUTE = 5'd2,
30         ST_MOMENTUM_ROUND = 5'd3,
31         ST_MOMENTUM_SIGN = 5'd4,
32         ST_DECAY_MULTIPLY = 5'd5,
33         ST_DECAY_ABSOLUTE = 5'd6,
34         ST_DECAY_ROUND = 5'd7,
35         ST_DECAY_SIGN = 5'd8,
36         ST_VELOCITY = 5'd9,
37         ST_UPDATE_MULTIPLY = 5'd10,
```

```
        ST_UPDATE_ABSOLUTE = 5'd11,
        ST_UPDATE_ROUND = 5'd12,
        ST_UPDATE_SIGN = 5'd13,
        ST_UPDATE_SUBTRACT = 5'd14,
        ST_OUTPUT = 5'd15,
        ST_MOMENTUM_PRODUCT = 5'd16,
        ST_DECAY_PRODUCT = 5'd17,
        ST_UPDATE_PRODUCT = 5'd18,
        ST_VELOCITY_SUM = 5'd19,
        ST_VELOCITY_SATURATE = 5'd20,
        ST_MOMENTUM_SATURATE = 5'd21,
        ST_DECAY_SATURATE = 5'd22,
        ST_UPDATE_SATURATE = 5'd23,
        ST_MOMENTUM_BIAS = 5'd24,
        ST_DECAY_BIAS = 5'd25,
        ST_UPDATE_BIAS = 5'd26,
        ST_UPDATE_DIRECTION = 5'd27,
        ST_UPDATE_DIRECTION_SATURATE = 5'd28,
        ST_MOMENTUM_PRODUCT_RELAY = 5'd29,
        ST_DECAY_PRODUCT_RELAY = 5'd30,
        ST_UPDATE_PRODUCT_RELAY = 5'd31
    } state_t;

    state_t state;
    logic signed [15:0] weight_r [0:LANES-1];
    logic signed [23:0] gradient_r [0:LANES-1];
    logic signed [23:0] velocity_r [0:LANES-1];
    logic signed [23:0] next_velocity_r [0:LANES-1];
    logic signed [23:0] momentum_term_r [0:LANES-1];
    logic signed [23:0] decay_term_r [0:LANES-1];
    logic signed [24:0] two_term_sum_r [0:LANES-1];
    logic signed [25:0] velocity_sum_r [0:LANES-1];
    logic signed [41:0] product_r [0:LANES-1];
    logic product_negative_r [0:LANES-1];
    logic [41:0] product_magnitude_r [0:LANES-1];
    logic [41:0] rounded_magnitude_r [0:LANES-1];
    logic signed [41:0] signed_rounded_r [0:LANES-1];
    logic signed [41:0] update_value_r [0:LANES-1];
    logic signed [41:0] weight_difference_r [0:LANES-1];
    logic [17:0] learning_rate_r;
    logic [16:0] momentum_r;
    logic [16:0] weight_decay_r;
    logic [4:0] update_right_shift_r;
    logic [4:0] round_shift_counter_r;
    logic [41:0] update_rounding_bias_r;

    logic signed [23:0] multiplier_left [0:LANES-1];
    logic signed [17:0] multiplier_right [0:LANES-1];
    logic signed [23:0] multiplier_operand_left_r [0:LANES-1];
    logic signed [17:0] multiplier_operand_right_r [0:LANES-1];
    (* use_dsp = "yes" *) wire signed [41:0] multiplier_product [0:LANES-1];
    (* use_dsp = "yes" *) logic signed [41:0]
        multiplier_product_pipeline_r [0:LANES-1];

    function automatic logic signed [23:0] saturate_s24(input logic signed [41:0] value);
        begin
            if (value > 42'sd8388607) saturate_s24 = 24'sh7fffff;
            else if (value < -42'sd8388608) saturate_s24 = 24'sh800000;
            else saturate_s24 = value[23:0];
        end
    endfunction

    function automatic logic signed [15:0] saturate_s16(input logic signed [41:0] value);
        begin
            if (value > 42'sd32767) saturate_s16 = 16'sh7fff;
            else if (value < -42'sd32768) saturate_s16 = 16'sh8000;
            else saturate_s16 = value[15:0];
        end
    endfunction

    for (genvar lane = 0; lane < LANES; lane++) begin : g_optimizer_lane
        assign multiplier_product[lane] =
            multiplier_operand_left_r[lane] * multiplier_operand_right_r[lane];
    end

    always_comb begin
        input_ready = state == ST_IDLE;
        output_valid = state == ST_OUTPUT;
        for (int lane = 0; lane < LANES; lane++) begin
            multiplier_left[lane] = 24'sd0;
            multiplier_right[lane] = 18'sd0;
            unique case (state)
                ST_MOMENTUM_MULTIPLY: begin
                    multiplier_left[lane] = velocity_r[lane];
                    multiplier_right[lane] = $signed({1'b0, momentum_r});
                end
                ST_DECAY_MULTIPLY: begin
```

```
                    multiplier_left[lane] = {{8{weight_r[lane][15]}}, weight_r[lane]};
                    multiplier_right[lane] = $signed({1'b0, weight_decay_r});
                end
                ST_UPDATE_MULTIPLY: begin
                    multiplier_left[lane] = momentum_term_r[lane];
                    multiplier_right[lane] = $signed(learning_rate_r);
                end
                default: begin
                    multiplier_left[lane] = 24'sd0;
                    multiplier_right[lane] = 18'sd0;
                end
            endcase
        end
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            state <= ST_IDLE;
            updated_weight_vector <= '0;
            updated_velocity_vector <= '0;
            learning_rate_r <= 18'd0;
            momentum_r <= 17'd0;
            weight_decay_r <= 17'd0;
            update_right_shift_r <= 5'd16;
            round_shift_counter_r <= 5'd0;
            update_rounding_bias_r <= 42'd32768;
            config_error <= 1'b0;
            update_count <= 32'd0;
            for (int lane = 0; lane < LANES; lane++) begin
                weight_r[lane] <= 16'sd0;
                gradient_r[lane] <= 24'sd0;
                velocity_r[lane] <= 24'sd0;
                next_velocity_r[lane] <= 24'sd0;
                momentum_term_r[lane] <= 24'sd0;
                decay_term_r[lane] <= 24'sd0;
                two_term_sum_r[lane] <= 25'sd0;
                velocity_sum_r[lane] <= 26'sd0;
                product_r[lane] <= 42'sd0;
                product_negative_r[lane] <= 1'b0;
                product_magnitude_r[lane] <= 42'd0;
                rounded_magnitude_r[lane] <= 42'd0;
                signed_rounded_r[lane] <= 42'sd0;
                update_value_r[lane] <= 42'sd0;
                weight_difference_r[lane] <= 42'sd0;
                multiplier_operand_left_r[lane] <= 24'sd0;
                multiplier_operand_right_r[lane] <= 18'sd0;
                multiplier_product_pipeline_r[lane] <= 42'sd0;
            end
        end else begin
            unique case (state)
                ST_IDLE: begin
                    if (input_valid) begin
                        config_error <= learning_rate_q16 > 18'd131071
                            || momentum_q16 > 17'd65536
                            || weight_decay_q16 > 17'd65536;
                        learning_rate_r <= learning_rate_q16;
                        momentum_r <= momentum_q16;
                        weight_decay_r <= weight_decay_q16;
                        update_right_shift_r <= update_right_shift;
                        update_rounding_bias_r <= update_right_shift == 0
                            ? 42'd0 : (42'd1 << (update_right_shift - 1'b1));
                        for (int lane = 0; lane < LANES; lane++) begin
                            weight_r[lane] <= $signed(weight_vector[lane*16 +: 16]);
                            gradient_r[lane] <= $signed(gradient_vector[lane*24 +: 24]);
                            velocity_r[lane] <= $signed(velocity_vector[lane*24 +: 24]);
                        end
                        state <= ST_MOMENTUM_MULTIPLY;
                    end
                end

                ST_MOMENTUM_MULTIPLY: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        multiplier_operand_left_r[lane] <= multiplier_left[lane];
                        multiplier_operand_right_r[lane] <= multiplier_right[lane];
                    end
                    state <= ST_MOMENTUM_PRODUCT;
                end

                ST_MOMENTUM_PRODUCT: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        multiplier_product_pipeline_r[lane]
                            <= multiplier_product[lane];
                    end
                    state <= ST_MOMENTUM_PRODUCT_RELAY;
                end

                ST_MOMENTUM_PRODUCT_RELAY: begin
```

```
                    for (int lane = 0; lane < LANES; lane++) begin
                        product_r[lane] <= multiplier_product_pipeline_r[lane];
                    end
                    state <= ST_MOMENTUM_ABSOLUTE;
                end

                ST_MOMENTUM_ABSOLUTE: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        product_negative_r[lane] <= product_r[lane][41];
                        product_magnitude_r[lane] <= product_r[lane][41]
                            ? -product_r[lane] : product_r[lane];
                    end
                    state <= ST_MOMENTUM_BIAS;
                end

                ST_MOMENTUM_BIAS: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        rounded_magnitude_r[lane]
                            <= product_magnitude_r[lane] + 42'd32768;
                    end
                    state <= ST_MOMENTUM_ROUND;
                end

                ST_MOMENTUM_ROUND: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        rounded_magnitude_r[lane] <= rounded_magnitude_r[lane] >> 16;
                    end
                    state <= ST_MOMENTUM_SIGN;
                end

                ST_MOMENTUM_SIGN: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        signed_rounded_r[lane] <= product_negative_r[lane]
                            ? -$signed(rounded_magnitude_r[lane])
                            : $signed(rounded_magnitude_r[lane]);
                    end
                    state <= ST_MOMENTUM_SATURATE;
                end

                ST_MOMENTUM_SATURATE: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        momentum_term_r[lane] <= saturate_s24(signed_rounded_r[lane]);
                    end
                    state <= ST_DECAY_MULTIPLY;
                end

                ST_DECAY_MULTIPLY: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        multiplier_operand_left_r[lane] <= multiplier_left[lane];
                        multiplier_operand_right_r[lane] <= multiplier_right[lane];
                    end
                    state <= ST_DECAY_PRODUCT;
                end

                ST_DECAY_PRODUCT: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        multiplier_product_pipeline_r[lane]
                            <= multiplier_product[lane];
                    end
                    state <= ST_DECAY_PRODUCT_RELAY;
                end

                ST_DECAY_PRODUCT_RELAY: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        product_r[lane] <= multiplier_product_pipeline_r[lane];
                    end
                    state <= ST_DECAY_ABSOLUTE;
                end

                ST_DECAY_ABSOLUTE: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        product_negative_r[lane] <= product_r[lane][41];
                        product_magnitude_r[lane] <= product_r[lane][41]
                            ? -product_r[lane] : product_r[lane];
                    end
                    state <= ST_DECAY_BIAS;
                end

                ST_DECAY_BIAS: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        rounded_magnitude_r[lane]
                            <= product_magnitude_r[lane] + 42'd32768;
                    end
                    state <= ST_DECAY_ROUND;
                end

                ST_DECAY_ROUND: begin
```

```
                    for (int lane = 0; lane < LANES; lane++) begin
                        rounded_magnitude_r[lane] <= rounded_magnitude_r[lane] >> 16;
                    end
                    state <= ST_DECAY_SIGN;
                end

                ST_DECAY_SIGN: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        signed_rounded_r[lane] <= product_negative_r[lane]
                            ? -$signed(rounded_magnitude_r[lane])
                            : $signed(rounded_magnitude_r[lane]);
                    end
                    state <= ST_DECAY_SATURATE;
                end

                ST_DECAY_SATURATE: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        decay_term_r[lane] <= saturate_s24(signed_rounded_r[lane]);
                    end
                    state <= ST_VELOCITY;
                end

                ST_VELOCITY: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        two_term_sum_r[lane] <=
                            {gradient_r[lane][23], gradient_r[lane]}
                            + {momentum_term_r[lane][23], momentum_term_r[lane]};
                    end
                    state <= ST_VELOCITY_SUM;
                end

                ST_VELOCITY_SUM: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        velocity_sum_r[lane] <=
                            {two_term_sum_r[lane][24], two_term_sum_r[lane]};
                    end
                    state <= ST_VELOCITY_SATURATE;
                end

                ST_VELOCITY_SATURATE: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        next_velocity_r[lane] <= saturate_s24(
                            {{16{velocity_sum_r[lane][25]}}, velocity_sum_r[lane]});
                    end
                    state <= ST_UPDATE_DIRECTION;
                end

                ST_UPDATE_DIRECTION: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        two_term_sum_r[lane] <=
                            {next_velocity_r[lane][23], next_velocity_r[lane]}
                            + {decay_term_r[lane][23], decay_term_r[lane]};
                    end
                    state <= ST_UPDATE_DIRECTION_SATURATE;
                end

                ST_UPDATE_DIRECTION_SATURATE: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        // The retained velocity excludes weight decay. This
                        // scratch register holds only the SGDW update direction.
                        momentum_term_r[lane] <= saturate_s24(
                            {{17{two_term_sum_r[lane][24]}}, two_term_sum_r[lane]});
                    end
                    state <= ST_UPDATE_MULTIPLY;
                end

                ST_UPDATE_MULTIPLY: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        multiplier_operand_left_r[lane] <= multiplier_left[lane];
                        multiplier_operand_right_r[lane] <= multiplier_right[lane];
                    end
                    state <= ST_UPDATE_PRODUCT;
                end

                ST_UPDATE_PRODUCT: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        multiplier_product_pipeline_r[lane]
                            <= multiplier_product[lane];
                    end
                    state <= ST_UPDATE_PRODUCT_RELAY;
                end

                ST_UPDATE_PRODUCT_RELAY: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        product_r[lane] <= multiplier_product_pipeline_r[lane];
                    end
                    state <= ST_UPDATE_ABSOLUTE;
```

```
                end

                ST_UPDATE_ABSOLUTE: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        product_negative_r[lane] <= product_r[lane][41];
                        product_magnitude_r[lane] <= product_r[lane][41]
                            ? -product_r[lane] : product_r[lane];
                    end
                    state <= ST_UPDATE_BIAS;
                end

                ST_UPDATE_BIAS: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        rounded_magnitude_r[lane] <=
                            product_magnitude_r[lane] + update_rounding_bias_r;
                    end
                    round_shift_counter_r <= update_right_shift_r;
                    state <= ST_UPDATE_ROUND;
                end

                ST_UPDATE_ROUND: begin
                    if (round_shift_counter_r == 0) begin
                        state <= ST_UPDATE_SIGN;
                    end else begin
                        for (int lane = 0; lane < LANES; lane++) begin
                            rounded_magnitude_r[lane]
                                <= rounded_magnitude_r[lane] >> 1;
                        end
                        round_shift_counter_r <= round_shift_counter_r - 1'b1;
                        if (round_shift_counter_r == 1) state <= ST_UPDATE_SIGN;
                    end
                end

                ST_UPDATE_SIGN: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        update_value_r[lane] <= product_negative_r[lane]
                            ? -$signed(rounded_magnitude_r[lane])
                            : $signed(rounded_magnitude_r[lane]);
                    end
                    state <= ST_UPDATE_SUBTRACT;
                end

                ST_UPDATE_SUBTRACT: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        weight_difference_r[lane] <=
                            {{26{weight_r[lane][15]}}, weight_r[lane]}
                            - update_value_r[lane];
                        updated_velocity_vector[lane*24 +: 24] <= next_velocity_r[lane];
                    end
                    state <= ST_UPDATE_SATURATE;
                end

                ST_UPDATE_SATURATE: begin
                    for (int lane = 0; lane < LANES; lane++) begin
                        updated_weight_vector[lane*16 +: 16] <=
                            saturate_s16(weight_difference_r[lane]);
                    end
                    update_count <= update_count + LANES;
                    state <= ST_OUTPUT;
                end

                ST_OUTPUT: begin
                    if (output_ready) state <= ST_IDLE;
                end

                default: state <= ST_IDLE;
            endcase
            if (clear_counters) update_count <= 32'd0;
        end
    end

    initial begin
        if (LANES != 8) $fatal(1, "neuraedge_sgd_optimizer8 currently requires LANES=8");
    end
endmodule

`default_nettype wire
```

18.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_sgd_optimizer8 #(` This opens neuraedge_sgd_optimizer8, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state.

Line 5: `parameter int LANES = 8` This statement continues through source line 25; the explanation uses the complete expression rather than only the first visual fragment. This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 6: `) (` This is continuation line 2 of the 21-line logical statement that begins at source line 5. Its exact fragment is ‘) (‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 7: `input wire clk,` This is continuation line 3 of the 21-line logical statement that begins at source line 5. Its exact fragment is ‘input wire clk,‘. The important operands are input: the value named ’input’ in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 8: `input wire rst_n,` This is continuation line 4 of the 21-line logical statement that begins at source line 5. Its exact fragment is ‘input wire rst_n,‘. The important operands are input: the value named ’input’ in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 9: `input wire clear_counters,` This is continuation line 5 of the 21-line logical statement that begins at source line 5. Its exact fragment is ‘input wire clear_counters,‘. The important operands are input: the value named ’input’ in this block; clear_counters: a hardware telemetry register that accumulates measured events. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready,

output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 10: `input wire input_valid,` This is continuation line 6 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'input wire input_valid,'. The important operands are input: the value named 'input' in this block; input_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 11: `output logic input_ready,` This is continuation line 7 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'output logic input_ready,'. The important operands are output: the value named 'output' in this block; input_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 12: `input wire [LANES*16-1:0] weight_vector,` This is continuation line 8 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'input wire [LANES*16-1:0] weight_vector,'. The important operands are input: the value named 'input' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; weight_vector: a learned coefficient multiplied by an activation. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 13: `input wire [LANES*24-1:0] gradient_vector,` This is continuation line 9 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'input wire [LANES*24-1:0] gradient_vector,'. The important operands are input: the value named 'input' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; gradient_vector: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 14: `input wire [LANES*24-1:0] velocity_vector,` This is continuation line 10 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'input wire [LANES*24-1:0] velocity_vector,'. The important operands are input: the value named 'input' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; velocity_vector: the momentum history retained by the SGD optimizer. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire

[16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 15: `input wire [17:0] learning_rate_q16,` This is continuation line 11 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'input wire [17:0] learning_rate_q16,'. The important operands are input: the value named 'input' in this block; learning_rate_q16: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 16: `input wire [16:0] momentum_q16,` This is continuation line 12 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'input wire [16:0] momentum_q16,'. The important operands are input: the value named 'input' in this block; momentum_q16: the value named 'momentum q16' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 17: `input wire [16:0] weight_decay_q16,` This is continuation line 13 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'input wire [16:0] weight_decay_q16,'. The important operands are input: the value named 'input' in this block; weight_decay_q16: a learned coefficient multiplied by an activation. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 18: `input wire [4:0] update_right_shift,` This is continuation line 14 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'input wire [4:0] update_right_shift,'. The important operands are input: the value named 'input' in this block; update_right_shift: the value named 'update right shift' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 19: `output logic output_valid,` This is continuation line 15 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'output logic output_valid,'. The important operands are output: the value named 'output' in this block; output_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input

wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 20: `input wire output_ready,` This is continuation line 16 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'input wire output_ready,'. The important operands are input: the value named 'input' in this block; output_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 21: `output logic [LANES*16-1:0] updated_weight_vector,` This is continuation line 17 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'output logic [LANES*16-1:0] updated_weight_vector,'. The important operands are output: the value named 'output' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; updated_weight_vector: a learned coefficient multiplied by an activation. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 22: `output logic [LANES*24-1:0] updated_velocity_vector,` This is continuation line 18 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'output logic [LANES*24-1:0] updated_velocity_vector,'. The important operands are output: the value named 'output' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; updated_velocity_vector: the momentum history retained by the SGD optimizer. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 23: `output logic config_error,` This is continuation line 19 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'output logic config_error,'. The important operands are output: the value named 'output' in this block; config_error: the value named 'config error' in this block. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 24: `output logic [31:0] update_count` This is continuation line 20 of the 21-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] update_count'. The important operands are output:

the value named 'output' in this block; update_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 25: `);` This is continuation line 21 of the 21-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes LANES to 8) (input wire clk, input wire rst_n, input wire clear_counters, input wire input_valid, output logic input_ready, input wire [LANES*16-1:0] weight_vector, input wire [LANES*24-1:0] gradient_vector, input wire [LANES*24-1:0] velocity_vector, input wire [17:0] learning_rate_q16, input wire [16:0] momentum_q16, input wire [16:0] weight_decay_q16, input wire [4:0] update_right_shift, output logic output_valid, input wire output_ready, output logic [LANES*16-1:0] updated_weight_vector, output logic [LANES*24-1:0] updated_velocity_vector, output logic config_error, output logic [31:0] update_count) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 26: `typedef enum logic [4:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 27: `ST_IDLE = 5'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 28: `ST_MOMENTUM_MULTIPLY = 5'd1,` This assigns a fixed binary encoding to state ST_MOMENTUM_MULTIPLY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 29: `ST_MOMENTUM_ABSOLUTE = 5'd2,` This assigns a fixed binary encoding to state ST_MOMENTUM_ABSOLUTE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 30: `ST_MOMENTUM_ROUND = 5'd3,` This assigns a fixed binary encoding to state ST_MOMENTUM_ROUND, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 31: `ST_MOMENTUM_SIGN = 5'd4,` This assigns a fixed binary encoding to state ST_MOMENTUM_SIGN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 32: `ST_DECAY_MULTIPLY = 5'd5,` This assigns a fixed binary encoding to state ST_DECAY_MULTIPLY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 33: `ST_DECAY_ABSOLUTE = 5'd6,` This assigns a fixed binary encoding to state ST_DECAY_ABSOLUTE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 34: `ST_DECAY_ROUND = 5'd7,` This assigns a fixed binary encoding to state ST_DECAY_ROUND, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 35: `ST_DECAY_SIGN = 5'd8,` This assigns a fixed binary encoding to state ST_DECAY_SIGN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 36: `ST_VELOCITY = 5'd9,` This assigns a fixed binary encoding to state ST_VELOCITY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 37: `ST_UPDATE_MULTIPLY = 5'd10,` This assigns a fixed binary encoding to state ST_UPDATE_MULTIPLY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 38: `ST_UPDATE_ABSOLUTE = 5'd11,` This assigns a fixed binary encoding to state ST_UPDATE_ABSOLUTE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 39: `ST_UPDATE_ROUND = 5'd12,` This assigns a fixed binary encoding to state ST_UPDATE_ROUND, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 40: `ST_UPDATE_SIGN = 5'd13,` This assigns a fixed binary encoding to state ST_UPDATE_SIGN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 41: `ST_UPDATE_SUBTRACT = 5'd14,` This assigns a fixed binary encoding to state ST_UPDATE_SUBTRACT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 42: `ST_OUTPUT = 5'd15,` This assigns a fixed binary encoding to state ST_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 43: `ST_MOMENTUM_PRODUCT = 5'd16,` This assigns a fixed binary encoding to state ST_MOMENTUM_PRODUCT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 44: `ST_DECAY_PRODUCT = 5'd17,` This assigns a fixed binary encoding to state ST_DECAY_PRODUCT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 45: `ST_UPDATE_PRODUCT = 5'd18,` This assigns a fixed binary encoding to state ST_UPDATE_PRODUCT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 46: `ST_VELOCITY_SUM = 5'd19,` This assigns a fixed binary encoding to state ST_VELOCITY_SUM, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 47: `ST_VELOCITY_SATURATE = 5'd20,` This assigns a fixed binary encoding to state ST_VELOCITY_SATURATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 48: `ST_MOMENTUM_SATURATE = 5'd21,` This assigns a fixed binary encoding to state ST_MOMENTUM_SATURATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 49: `ST_DECAY_SATURATE = 5'd22,` This assigns a fixed binary encoding to state ST_DECAY_SATURATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 50: `ST_UPDATE_SATURATE = 5'd23,` This assigns a fixed binary encoding to state ST_UPDATE_SATURATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 51: `ST_MOMENTUM_BIAS = 5'd24,` This assigns a fixed binary encoding to state ST_MOMENTUM_BIAS, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 52: `ST_DECAY_BIAS = 5'd25,` This assigns a fixed binary encoding to state ST_DECAY_BIAS, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 53: `ST_UPDATE_BIAS = 5'd26,` This assigns a fixed binary encoding to state ST_UPDATE_BIAS, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 54: `ST_UPDATE_DIRECTION = 5'd27,` This assigns a fixed binary encoding to state ST_UPDATE_DIRECTION, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 55: `ST_UPDATE_DIRECTION_SATURATE = 5'd28,` This assigns a fixed binary encoding to state ST_UPDATE_DIRECTION_ which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 56: `ST_MOMENTUM_PRODUCT_RELAY = 5'd29,` This assigns a fixed binary encoding to state ST_MOMENTUM_PRODUCT_ which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 57: `ST_DECAY_PRODUCT_RELAY = 5'd30,` This assigns a fixed binary encoding to state ST_DECAY_PRODUCT_RELAY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 58: `ST_UPDATE_PRODUCT_RELAY = 5'd31` This statement continues through source line 59; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_UPDATE_PRODUCT_RELAY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 59: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 58. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_UPDATE_PRODUCT_RELAY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 61: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 62: `logic signed [15:0] weight_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 63: `logic signed [23:0] gradient_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gradient_r is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 64: `logic signed [23:0] velocity_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, velocity_r is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 65: `logic signed [23:0] next_velocity_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, next_velocity_r is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 66: `logic signed [23:0] momentum_term_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, momentum_term_r is the value named 'momentum term r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 67: `logic signed [23:0] decay_term_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, decay_term_r is the value named 'decay term r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 68: `logic signed [24:0] two_term_sum_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, two_term_sum_r is the value named 'two term sum r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 69: `logic signed [25:0] velocity_sum_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, velocity_sum_r is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 70: `logic signed [41:0] product_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_r is the value named 'product r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 71: `logic product_negative_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_negative_r is the value named 'product negative r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 72: `logic [41:0] product_magnitude_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_magnitude_r is the value named 'product magnitude r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 73: `logic [41:0] rounded_magnitude_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rounded_magnitude_r is the value named 'rounded magnitude r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 74: `logic signed [41:0] signed_rounded_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, signed_rounded_r is the value named 'signed rounded r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 75: `logic signed [41:0] update_value_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, update_value_r is the value named 'update value r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 76: `logic signed [41:0] weight_difference_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_difference_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 77: `logic [17:0] learning_rate_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, learning_rate_r is the fixed-point multiplier controlling update magnitude. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 78: `logic [16:0] momentum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, momentum_r is the value named 'momentum r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 79: `logic [16:0] weight_decay_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_decay_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 80: `logic [4:0] update_right_shift_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, update_right_shift_r is the value named 'update right shift r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 81: `logic [4:0] round_shift_counter_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, round_shift_counter_r is a hardware telemetry register that accumulates measured events. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 82: `logic [41:0] update_rounding_bias_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, update_rounding_bias_r is an additive learned offset applied after accumulation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 84: `logic signed [23:0] multiplier_left [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_left is the value named 'multiplier left' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 85: `logic signed [17:0] multiplier_right [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_right is the value named 'multiplier right' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 86: `logic signed [23:0] multiplier_operand_left_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_operand_left_r is the value named 'multiplier operand left r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 87: `logic signed [17:0] multiplier_operand_right_r [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_operand_right_r is the value named 'multiplier operand right r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 88: `(* use_dsp = "yes" *) wire signed [41:0] multiplier_product [0:LANES-1];` Within the module's structural declarations, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; multiplier_product: the value named 'multiplier product' in this block.

Line 89: `(* use_dsp = "yes" *) logic signed [41:0]` Within the module's structural declarations, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block.

Line 90: `multiplier_product_pipeline_r [0:LANES-1];` Within the module's structural declarations, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are multiplier_product_pipeline_r: the value named 'multiplier product pipeline r' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 92: `function automatic logic signed [23:0] saturate_s24(input logic signed [41:0] value);` Within the combinational helper saturate s24, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_s24: the value named 'saturate s24' in this block.

Line 93: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24; it has no arithmetic value of its own.

Line 94: `if (value > 42'sd8388607) saturate_s24 = 24'sh7fffff;` This condition selects the following hardware branch only when 'value > 42'sd8388607' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; sd8388607: the value named 'sd8388607' in this block. Context: the combinational helper saturate s24.

Line 95: `else if (value < -42'sd8388608) saturate_s24 = 24'sh800000;` This condition selects the following hardware branch only when 'value < -42'sd8388608' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; sd8388608: the value named 'sd8388608' in this block. Context: the combinational helper saturate s24.

Line 96: `else saturate_s24 = value[23:0];` Within the combinational helper saturate s24, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are saturate_s24: the value named 'saturate s24' in this block; value: the value named 'value' in this block.

Line 97: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24; it has no arithmetic value of its own.

Line 98: `endfunction` Within the combinational helper saturate s24, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are endfunction: the value named 'endfunction' in this block.

Line 100: `function automatic logic signed [15:0] saturate_s16(input logic signed [41:0] value);` Within the combinational helper saturate s16, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_s16: the value named 'saturate s16' in this block.

Line 101: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s16; it has no arithmetic value of its own.

Line 102: `if (value > 42'sd32767) saturate_s16 = 16'sh7fff;` This condition selects the following hardware branch only when 'value > 42'sd32767' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; sd32767: the value named 'sd32767' in this block. Context: the combinational helper saturate s16.

Line 103: `else if (value < -42'sd32768) saturate_s16 = 16'sh8000;` This condition selects the following hardware branch only when 'value < -42'sd32768' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; sd32768: the value named 'sd32768' in this block. Context: the combinational helper saturate s16.

Line 104: `else saturate_s16 = value[15:0];` Within the combinational helper saturate s16, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are saturate_s16: the value named 'saturate s16' in this block; value: the value named 'value' in this block.

Line 105: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s16; it has no arithmetic value of its own.

Line 106: `endfunction` Within the combinational helper saturate s16, this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are endfunction: the value named 'endfunction' in this block.

Line 108: `for (genvar lane = 0; lane < LANES; lane++) begin : g_optimizer_lane` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 109: `assign multiplier_product[lane] =` This statement continues through source line 110; the explanation uses the complete expression rather than only the first visual fragment. This makes multiplier_product[lane], the value named 'multiplier product' in this block, equal to 'multiplier_operand_left_r[lane] * multiplier_operand_right_r[lane]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are multiplier_operand_left_r: the value named 'multiplier operand left r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; multiplier_operand_right_r: the value named 'multiplier operand right r' in this block. Context: the module's structural declarations.

Line 110: `multiplier_operand_left_r[lane] * multiplier_operand_right_r[lane];` This is continuation line 2 of the 2-line logical statement that begins at source line 109. Its exact fragment is 'multiplier_operand_left_r[lane] * multiplier_operand_right_r[lane];'. The important operands are multiplier_operand_left_r: the value named 'multiplier operand left r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; multiplier_operand_right_r: the value named 'multiplier operand right r' in this block. Read together, the complete operation is: This makes multiplier_product[lane], the value named 'multiplier product' in this block, equal to 'multiplier_operand_left_r[lane] * multiplier_operand_right_r[lane]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are multiplier_operand_left_r: the value named 'multiplier operand left r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; multiplier_operand_right_r: the value named 'multiplier operand right r' in this block. Context: the module's structural declarations.

Line 111: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 113: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 114: `input_ready = state == ST_IDLE;` This makes input_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_IDLE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 115: `output_valid = state == ST_OUTPUT;` This makes output_valid, the producer's statement that its current payload is meaningful, equal to 'state == ST_OUTPUT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT: the value named 'st output' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 116: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 117: `multiplier_left[lane] = 24'sd0;` This makes multiplier_left[lane], the value named 'multiplier left' in this block, equal to '24'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 118: `multiplier_right[lane] = 18'sd0;` This makes multiplier_right[lane], the value named 'multiplier right' in this block, equal to '18'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 119: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 120: `ST_MOMENTUM_MULTIPLY: begin` This is the ST_MOMENTUM_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase..

Line 121: `multiplier_left[lane] = velocity_r[lane];` This makes multiplier_left[lane], the value named 'multiplier left' in this block, equal to 'velocity_r[lane]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are velocity_r: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase..

Line 122: `multiplier_right[lane] = $signed({1'b0, momentum_r});` This makes multiplier_right[lane], the value named 'multiplier right' in this block, equal to '$signed({1'b0, momentum_r})'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block; momentum_r: the value named 'momentum r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase..

Line 123: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 124: `ST_DECAY_MULTIPLY: begin` This is the ST_DECAY_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase..

Line 125: `multiplier_left[lane] = {{8{weight_r[lane][15]}}, weight_r[lane]};` This makes multiplier_left[lane], the value named 'multiplier left' in this block, equal to '{{8{weight_r[lane][15]}}, weight_r[lane]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_r: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase..

Line 126: `multiplier_right[lane] = $signed({1'b0, weight_decay_r});` This makes multiplier_right[lane], the value named 'multiplier right' in this block, equal to '$signed({1'b0, weight_decay_r})'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block; weight_decay_r: a learned coefficient multiplied by an activation. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase..

Line 127: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 128: `ST_UPDATE_MULTIPLY: begin` This is the ST_UPDATE_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 129: `multiplier_left[lane] = momentum_term_r[lane];` This makes multiplier_left[lane], the value named 'multiplier left' in this block, equal to 'momentum_term_r[lane]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are momentum_term_r: the value named 'momentum term r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 130: `multiplier_right[lane] = $signed(learning_rate_r);` This makes multiplier_right[lane], the value named 'multiplier right' in this block, equal to '$signed(learning_rate_r)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are learning_rate_r: the fixed-point multiplier controlling update magnitude. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 131: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 132: `default: begin` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 133: `multiplier_left[lane] = 24'sd0;` This makes multiplier_left[lane], the value named 'multiplier left' in this block, equal to '24'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 134: `multiplier_right[lane] = 18'sd0;` This makes multiplier_right[lane], the value named 'multiplier right' in this block, equal to '18'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 135: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 136: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 137: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 138: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 140: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 141: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 142: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 143: `updated_weight_vector <= '0;` This makes updated_weight_vector, a learned coefficient multiplied by an activation, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 144: `updated_velocity_vector <= '0;` This makes updated_velocity_vector, the momentum history retained by the SGD optimizer, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 145: `learning_rate_r <= 18'd0;` This makes learning_rate_r, the fixed-point multiplier controlling update magnitude, equal to '18'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 146: `momentum_r <= 17'd0;` This makes momentum_r, the value named 'momentum r' in this block, equal to '17'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 147: `weight_decay_r <= 17'd0;` This makes weight_decay_r, a learned coefficient multiplied by an activation, equal to '17'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 148: `update_right_shift_r <= 5'd16;` This makes update_right_shift_r, the value named 'update right shift r' in this block, equal to '5'd16'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d16: the value named 'd16' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 149: `round_shift_counter_r <= 5'd0;` This makes round_shift_counter_r, a hardware telemetry register that accumulates measured events, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 150: `update_rounding_bias_r <= 42'd32768;` This makes update_rounding_bias_r, an additive learned offset applied after accumulation, equal to '42'd32768'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d32768: the value named 'd32768' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 151: `config_error <= 1'b0;` This makes config_error, the value named 'config error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 152: `update_count <= 32'd0;` This makes update_count, an integer count retained for control or measurable telemetry, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 153: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 154: `weight_r[lane] <= 16'sd0;` This makes weight_r[lane], a learned coefficient multiplied by an activation, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 155: `gradient_r[lane] <= 24'sd0;` This makes gradient_r[lane], the signed loss derivative used to update a learned parameter, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 156: `velocity_r[lane] <= 24'sd0;` This makes velocity_r[lane], the momentum history retained by the SGD optimizer, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 157: `next_velocity_r[lane] <= 24'sd0;` This makes next_velocity_r[lane], the momentum history retained by the SGD optimizer, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 158: `momentum_term_r[lane] <= 24'sd0;` This makes momentum_term_r[lane], the value named 'momentum term r' in this block, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 159: `decay_term_r[lane] <= 24'sd0;` This makes decay_term_r[lane], the value named 'decay term r' in this block, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 160: `two_term_sum_r[lane] <= 25'sd0;` This makes two_term_sum_r[lane], the value named 'two term sum r' in this block, equal to '25'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 161: `velocity_sum_r[lane] <= 26'sd0;` This makes velocity_sum_r[lane], the momentum history retained by the SGD optimizer, equal to '26'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 162: `product_r[lane] <= 42'sd0;` This makes product_r[lane], the value named 'product r' in this block, equal to '42'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 163: `product_negative_r[lane] <= 1'b0;` This makes product_negative_r[lane], the value named 'product negative r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 164: `product_magnitude_r[lane] <= 42'd0;` This makes product_magnitude_r[lane], the value named 'product magnitude r' in this block, equal to '42'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 165: `rounded_magnitude_r[lane] <= 42'd0;` This makes rounded_magnitude_r[lane], the value named 'rounded magnitude r' in this block, equal to '42'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 166: `signed_rounded_r[lane] <= 42'sd0;` This makes signed_rounded_r[lane], the value named 'signed rounded r' in this block, equal to '42'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 167: `update_value_r[lane] <= 42'sd0;` This makes update_value_r[lane], the value named 'update value r' in this block, equal to '42'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 168: `weight_difference_r[lane] <= 42'sd0;` This makes weight_difference_r[lane], a learned coefficient multiplied by an activation, equal to '42'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 169: `multiplier_operand_left_r[lane] <= 24'sd0;` This makes multiplier_operand_left_r[lane], the value named 'multiplier operand left r' in this block, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 170: `multiplier_operand_right_r[lane] <= 18'sd0;` This makes multiplier_operand_right_r[lane], the value named 'multiplier operand right r' in this block, equal to '18'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 171: `multiplier_product_pipeline_r[lane] <= 42'sd0;` This makes multiplier_product_pipeline_r[lane], the value named 'multiplier product pipeline r' in this block, equal to '42'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 172: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 173: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 174: `unique case (state)` This starts a multiplexer/state decode selected by ‘state‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 175: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 176: `if (input_valid) begin` This condition selects the following hardware branch only when ‘input_valid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_valid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 177: `config_error <= learning_rate_q16 > 18'd131071` This statement continues through source line 179; the explanation uses the complete expression rather than only the first visual fragment. This makes config_error, the value named ’config error’ in this block, equal to ‘learning_rate_q16 > 18’d131071 || momentum_q16 > 17’d65536 || weight_decay_q16 > 17’d65536‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude; d131071: the value named ’d131071’ in this block; momentum_q16: the value named ’momentum q16’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 178: `|| momentum_q16 > 17'd65536` This is continuation line 2 of the 3-line logical statement that begins at source line 177. Its exact fragment is ‘|| momentum_q16 > 17’d65536‘. The important operands are momentum_q16: the value named ’momentum q16’ in this block; d65536: the value named ’d65536’ in this block. Read together, the complete operation is: This makes config_error, the value named ’config error’ in this block, equal to ‘learning_rate_q16 > 18’d131071 || momentum_q16 > 17’d65536 || weight_decay_q16 > 17’d65536‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude; d131071: the value named ’d131071’ in this block; momentum_q16: the value named ’momentum q16’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 179: `|| weight_decay_q16 > 17'd65536;` This is continuation line 3 of the 3-line logical statement that begins at source line 177. Its exact fragment is ‘|| weight_decay_q16 > 17’d65536;‘. The important operands are weight_decay_q16: a learned coefficient multiplied by an activation; d65536: the value named ’d65536’ in this block. Read together, the complete operation is: This makes config_error, the value named ’config error’ in this block, equal to ‘learning_rate_q16 > 18’d131071 || momentum_q16 > 17’d65536 || weight_decay_q16 > 17’d65536‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude; d131071: the value named ’d131071’ in this block; momentum_q16: the value named ’momentum q16’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 180: `learning_rate_r <= learning_rate_q16;` This makes learning_rate_r, the fixed-point multiplier controlling update magnitude, equal to ‘learning_rate_q16‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 181: `momentum_r <= momentum_q16;` This makes momentum_r, the value named 'momentum r' in this block, equal to 'momentum_q16'. It updates after the active clock edge together with other nonblocking assignments. The important operands are momentum_q16: the value named 'momentum q16' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 182: `weight_decay_r <= weight_decay_q16;` This makes weight_decay_r, a learned coefficient multiplied by an activation, equal to 'weight_decay_q16'. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_decay_q16: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 183: `update_right_shift_r <= update_right_shift;` This makes update_right_shift_r, the value named 'update right shift r' in this block, equal to 'update_right_shift'. It updates after the active clock edge together with other nonblocking assignments. The important operands are update_right_shift: the value named 'update right shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 184: `update_rounding_bias_r <= update_right_shift == 0` This statement continues through source line 185; the explanation uses the complete expression rather than only the first visual fragment. This makes update_rounding_bias_r, an additive learned offset applied after accumulation, equal to 'update_right_shift == 0 ? 42'd0 : (42'd1 << (update_right_shift - 1'b1))'. It updates after the active clock edge together with other nonblocking assignments. The important operands are update_right_shift: the value named 'update right shift' in this block; d0: the value named 'd0' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 185: `? 42'd0 : (42'd1 << (update_right_shift - 1'b1));` This is continuation line 2 of the 2-line logical statement that begins at source line 184. Its exact fragment is '? 42'd0 : (42'd1 << (update_right_shift - 1'b1));'. The important operands are d0: the value named 'd0' in this block; d1: the value named 'd1' in this block; update_right_shift: the value named 'update right shift' in this block. Read together, the complete operation is: This makes update_rounding_bias_r, an additive learned offset applied after accumulation, equal to 'update_right_shift == 0 ? 42'd0 : (42'd1 << (update_right_shift - 1'b1))'. It updates after the active clock edge together with other nonblocking assignments. The important operands are update_right_shift: the value named 'update right shift' in this block; d0: the value named 'd0' in this block; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 186: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 187: `weight_r[lane] <= $signed(weight_vector[lane*16 +: 16]);` This makes weight_r[lane], a learned coefficient multiplied by an activation, equal to '$signed(weight_vector[lane*16 +: 16])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_vector: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 188: `gradient_r[lane] <= $signed(gradient_vector[lane*24 +: 24]);` This makes gradient_r[lane], the signed loss derivative used to update a learned parameter, equal to '$signed(gradient_vector[lane*24 +: 24])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gradient_vector: the signed loss derivative used to update a learned parameter; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 189: `velocity_r[lane] <= $signed(velocity_vector[lane*24 +: 24]);` This makes velocity_r[lane], the momentum history retained by the SGD optimizer, equal to '$signed(velocity_vector[lane*24 +: 24])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are velocity_vector: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 190: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 191: `state <= ST_MOMENTUM_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MOMENTUM_MULTIPLY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MOMENTUM_MULTIPLY: the value named 'st momentum multiply' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 192: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 193: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 195: `ST_MOMENTUM_MULTIPLY: begin` This is the ST_MOMENTUM_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase..

Line 196: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase..

Line 197: `multiplier_operand_left_r[lane] <= multiplier_left[lane];` This makes multiplier_operand_left_r[lane], the value named 'multiplier operand left r' in this block, equal to 'multiplier_left[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_left: the value named 'multiplier left' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase..

Line 198: `multiplier_operand_right_r[lane] <= multiplier_right[lane];` This makes multiplier_operand_right_r[lane], the value named 'multiplier operand right r' in this block, equal to 'multiplier_right[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_right: the value named 'multiplier right' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase..

Line 199: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 200: `state <= ST_MOMENTUM_PRODUCT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MOMENTUM_PRODUCT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MOMENTUM_PRODUCT: the value named 'st momentum product' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase..

Line 201: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 203: `ST_MOMENTUM_PRODUCT: begin` This is the ST_MOMENTUM_PRODUCT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT, the machine performs the named control phase..

Line 204: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT, the machine performs the named control phase..

Line 205: `multiplier_product_pipeline_r[lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are multiplier_product_pipeline_r: the value named 'multiplier product pipeline r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 206: `<= multiplier_product[lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are multiplier_product: the value named 'multiplier product' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 207: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 208: `state <= ST_MOMENTUM_PRODUCT_RELAY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MOMENTUM_PRODUCT_RELAY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MOMENTUM_PRODUCT_RELAY: the value named 'st momentum product relay' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT, the machine performs the named control phase..

Line 209: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 211: `ST_MOMENTUM_PRODUCT_RELAY: begin` This is the ST_MOMENTUM_PRODUCT_RELAY case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT_RELAY, the machine performs the named control phase..

Line 212: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT_RELAY, the machine performs the named control phase..

Line 213: `product_r[lane] <= multiplier_product_pipeline_r[lane];` This makes product_r[lane], the value named 'product r' in this block, equal to 'multiplier_product_pipeline_r[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_product_pipeline_r: the value named 'multiplier product pipeline r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT_R the machine performs the named control phase..

Line 214: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT_RELAY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 215: `state <= ST_MOMENTUM_ABSOLUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MOMENTUM_ABSOLUTE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MOMENTUM_ABSOLUTE: the value named 'st momentum absolute' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT_RELAY, the machine performs the named control phase..

Line 216: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_PRODUCT_RELAY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 218: `ST_MOMENTUM_ABSOLUTE: begin` This is the ST_MOMENTUM_ABSOLUTE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ABSOLUTE, the machine performs the named control phase..

Line 219: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ABSOLUTE, the machine performs the named control phase..

Line 220: `product_negative_r[lane] <= product_r[lane][41];` This makes product_negative_r[lane], the value named 'product negative r' in this block, equal to 'product_r[lane][41]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ABSOLUTE, the machine performs the named control phase..

Line 221: `product_magnitude_r[lane] <= product_r[lane][41]` This statement continues through source line 222; the explanation uses the complete expression rather than only the first visual fragment. This makes product_magnitude_r[lane], the value named 'product magnitude r' in this block, equal to 'product_r[lane][41] ? -product_r[lane] : product_r[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ABSOLUTE, the machine performs the named control phase..

Line 222: `? -product_r[lane] : product_r[lane];` This is continuation line 2 of the 2-line logical statement that begins at source line 221. Its exact fragment is '? -product_r[lane] : product_r[lane];'. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes product_magnitude_r[lane], the value named 'product magnitude r' in this block, equal to 'product_r[lane][41] ? -product_r[lane] : product_r[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ABSOLUTE, the machine performs the named control phase..

Line 223: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ABSOLUTE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 224: `state <= ST_MOMENTUM_BIAS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MOMENTUM_BIAS'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MOMENTUM_BIAS: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ABSOLUTE, the machine performs the named control phase..

Line 225: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ABSOLUTE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 227: `ST_MOMENTUM_BIAS: begin` This is the ST_MOMENTUM_BIAS case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_BIAS, the machine performs the named control phase..

Line 228: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_BIAS, the machine performs the named control phase..

Line 229: `rounded_magnitude_r[lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_BIAS, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 230: `<= product_magnitude_r[lane] + 42'd32768;` Within the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_BIAS, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are product_magnitude_r: the value named 'product magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; d32768: the value named 'd32768' in this block.

Line 231: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_BIAS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 232: `state <= ST_MOMENTUM_ROUND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MOMENTUM_ROUND'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MOMENTUM_ROUND: the value named 'st momentum round' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_BIAS, the machine performs the named control phase..

Line 233: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_BIAS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 235: `ST_MOMENTUM_ROUND: begin` This is the ST_MOMENTUM_ROUND case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ROUND, the machine performs the named control phase..

Line 236: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ROUND, the machine performs the named control phase..

Line 237: `rounded_magnitude_r[lane] <= rounded_magnitude_r[lane] >> 16;` This makes rounded_magnitude_r[lane], the value named 'rounded magnitude r' in this block, equal to 'rounded_magnitude_r[lane] >> 16'. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ROUND, the machine performs the named control phase..

Line 238: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ROUND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 239: `state <= ST_MOMENTUM_SIGN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MOMENTUM_SIGN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MOMENTUM_SIGN: the value named 'st momentum sign' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ROUND, the machine performs the named control phase..

Line 240: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_ROUND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 242: `ST_MOMENTUM_SIGN: begin` This is the ST_MOMENTUM_SIGN case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SIGN, the machine performs the named control phase..

Line 243: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SIGN, the machine performs the named control phase..

Line 244: `signed_rounded_r[lane] <= product_negative_r[lane]` This statement continues through source line 246; the explanation uses the complete expression rather than only the first visual fragment. This makes signed_rounded_r[lane], the value named 'signed rounded r' in this block, equal to 'product_negative_r[lane] ? -$signed(rounded_magnitude_r[lane]) : $signed(rounded_magnitude_r[lane])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named 'product negative r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; rounded_magnitude_r: the value named 'rounded magnitude r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SIGN, the machine performs the named control phase..

Line 245: `? -$signed(rounded_magnitude_r[lane])` This is continuation line 2 of the 3-line logical statement that begins at source line 244. Its exact fragment is '? -$signed(rounded_magnitude_r[lane])'. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes signed_rounded_r[lane], the value named 'signed rounded r' in this block, equal to 'product_negative_r[lane] ? -$signed(rounded_magnitude_r[lane]) : $signed(rounded_magnitude_r[lane])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named 'product negative r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; rounded_magnitude_r: the value named 'rounded magnitude r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SIGN, the machine performs the named control phase..

Line 246: `: $signed(rounded_magnitude_r[lane]);` This is continuation line 3 of the 3-line logical statement that begins at source line 244. Its exact fragment is ': $signed(rounded_magnitude_r[lane]);'. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes signed_rounded_r[lane], the value named 'signed rounded r' in this block, equal to 'product_negative_r[lane] ? -$signed(rounded_magnitude_r[lane]) : $signed(rounded_magnitude_r[lane])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named 'product negative r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; rounded_magnitude_r: the value named 'rounded magnitude r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SIGN, the machine performs the named control phase..

Line 247: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 248: `state <= ST_MOMENTUM_SATURATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MOMENTUM_SATURATE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MOMENTUM_SATURATE: the value named 'st momentum saturate' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SIGN, the machine performs the named control phase..

Line 249: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 251: `ST_MOMENTUM_SATURATE: begin` This is the ST_MOMENTUM_SATURATE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SATURATE, the machine performs the named control phase..

Line 252: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SATURATE, the machine performs the named control phase..

Line 253: `momentum_term_r[lane] <= saturate_s24(signed_rounded_r[lane]);` This makes momentum_term_r[lane], the value named 'momentum term r' in this block, equal to 'saturate_s24(signed_rounded_r[lane])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_s24: the value named 'saturate s24' in this block; signed_rounded_r: the value named 'signed rounded r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SATURATE, the machine performs the named control phase..

Line 254: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 255: `state <= ST_DECAY_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_DECAY_MULTIPLY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECAY_MULTIPLY: the value named 'st decay multiply' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SATURATE, the machine performs the named control phase..

Line 256: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MOMENTUM_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 258: `ST_DECAY_MULTIPLY: begin` This is the ST_DECAY_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase..

Line 259: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase..

Line 260: `multiplier_operand_left_r[lane] <= multiplier_left[lane];` This makes multiplier_operand_left_r[lane], the value named 'multiplier operand left r' in this block, equal to ‘multiplier_left[lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_left: the value named 'multiplier left' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase..

Line 261: `multiplier_operand_right_r[lane] <= multiplier_right[lane];` This makes multiplier_operand_right_r[lane], the value named 'multiplier operand right r' in this block, equal to ‘multiplier_right[lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_right: the value named 'multiplier right' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase..

Line 262: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 263: `state <= ST_DECAY_PRODUCT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_DECAY_PRODUCT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECAY_PRODUCT: the value named 'st decay product' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase..

Line 264: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 266: `ST_DECAY_PRODUCT: begin` This is the ST_DECAY_PRODUCT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT, the machine performs the named control phase..

Line 267: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT, the machine performs the named control phase..

Line 268: `multiplier_product_pipeline_r[lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are multiplier_product_pipeline_r: the value named 'multiplier product pipeline r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 269: `<= multiplier_product[lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are multiplier_product: the value named 'multiplier product' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 270: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 271: `state <= ST_DECAY_PRODUCT_RELAY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_DECAY_PRODUCT_RELAY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECAY_PRODUCT_RELAY: the value named 'st decay product relay' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT, the machine performs the named control phase..

Line 272: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 274: `ST_DECAY_PRODUCT_RELAY: begin` This is the ST_DECAY_PRODUCT_RELAY case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT_RELAY, the machine performs the named control phase..

Line 275: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT_RELAY, the machine performs the named control phase..

Line 276: `product_r[lane] <= multiplier_product_pipeline_r[lane];` This makes product_r[lane], the value named 'product r' in this block, equal to 'multiplier_product_pipeline_r[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_product_pipeline_r: the value named 'multiplier product pipeline r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT_RELAY, the machine performs the named control phase..

Line 277: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT_RELAY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 278: `state <= ST_DECAY_ABSOLUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_DECAY_ABSOLUTE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECAY_ABSOLUTE: the value named 'st decay absolute' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT_RELAY, the machine performs the named control phase..

Line 279: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_PRODUCT_RELAY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 281: `ST_DECAY_ABSOLUTE: begin` This is the ST_DECAY_ABSOLUTE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ABSOLUTE, the machine performs the named control phase..

Line 282: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ABSOLUTE, the machine performs the named control phase..

Line 283: `product_negative_r[lane] <= product_r[lane][41];` This makes product_negative_r[lane], the value named 'product negative r' in this block, equal to 'product_r[lane][41]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ABSOLUTE, the machine performs the named control phase..

Line 284: `product_magnitude_r[lane] <= product_r[lane][41]` This statement continues through source line 285; the explanation uses the complete expression rather than only the first visual fragment. This makes product_magnitude_r[lane], the value named 'product magnitude r' in this block, equal to 'product_r[lane][41] ? -product_r[lane] : product_r[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ABSOLUTE, the machine performs the named control phase..

Line 285: `? -product_r[lane] : product_r[lane];` This is continuation line 2 of the 2-line logical statement that begins at source line 284. Its exact fragment is '? -product_r[lane] : product_r[lane];'. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes product_magnitude_r[lane], the value named 'product magnitude r' in this block, equal to 'product_r[lane][41] ? -product_r[lane] : product_r[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ABSOLUTE, the machine performs the named control phase..

Line 286: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ABSOLUTE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 287: `state <= ST_DECAY_BIAS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_DECAY_BIAS'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECAY_BIAS: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ABSOLUTE, the machine performs the named control phase..

Line 288: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ABSOLUTE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 290: `ST_DECAY_BIAS: begin` This is the ST_DECAY_BIAS case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_BIAS, the machine performs the named control phase..

Line 291: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_BIAS, the machine performs the named control phase..

Line 292: `rounded_magnitude_r[lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_BIAS, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 293: `<= product_magnitude_r[lane] + 42'd32768;` Within the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_BIAS, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are product_magnitude_r: the value named 'product magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; d32768: the value named 'd32768' in this block.

Line 294: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_BIAS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 295: `state <= ST_DECAY_ROUND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_DECAY_ROUND'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECAY_ROUND: the value named 'st decay round' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_BIAS, the machine performs the named control phase..

Line 296: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_BIAS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 298: `ST_DECAY_ROUND: begin` This is the ST_DECAY_ROUND case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ROUND, the machine performs the named control phase..

Line 299: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ROUND, the machine performs the named control phase..

Line 300: `rounded_magnitude_r[lane] <= rounded_magnitude_r[lane] >> 16;` This makes rounded_magnitude_r[lane], the value named 'rounded magnitude r' in this block, equal to 'rounded_magnitude_r[lane] >> 16'. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ROUND, the machine performs the named control phase..

Line 301: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ROUND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 302: `state <= ST_DECAY_SIGN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_DECAY_SIGN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECAY_SIGN: the value named 'st decay sign' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ROUND, the machine performs the named control phase..

Line 303: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_ROUND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 305: `ST_DECAY_SIGN: begin` This is the ST_DECAY_SIGN case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SIGN, the machine performs the named control phase..

Line 306: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SIGN, the machine performs the named control phase..

Line 307: `signed_rounded_r[lane] <= product_negative_r[lane]` This statement continues through source line 309; the explanation uses the complete expression rather than only the first visual fragment. This makes signed_rounded_r[lane], the value named ’signed rounded r’ in this block, equal to ‘product_negative_r[lane] ? -$signed(rounded_magnitude_r[lane]) : $signed(rounded_magnitude_r[lane])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named ’product negative r’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; rounded_magnitude_r: the value named ’rounded magnitude r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SIGN, the machine performs the named control phase..

Line 308: `? -$signed(rounded_magnitude_r[lane])` This is continuation line 2 of the 3-line logical statement that begins at source line 307. Its exact fragment is ‘? -$signed(rounded_magnitude_r[lane])‘. The important operands are rounded_magnitude_r: the value named ’rounded magnitude r’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes signed_rounded_r[lane], the value named ’signed rounded r’ in this block, equal to ‘product_negative_r[lane] ? -$signed(rounded_magnitude_r[lane]) : $signed(rounded_magnitude_r[lane])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named ’product negative r’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; rounded_magnitude_r: the value named ’rounded magnitude r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SIGN, the machine performs the named control phase..

Line 309: `: $signed(rounded_magnitude_r[lane]);` This is continuation line 3 of the 3-line logical statement that begins at source line 307. Its exact fragment is ‘: $signed(rounded_magnitude_r[lane]);‘. The important operands are rounded_magnitude_r: the value named ’rounded magnitude r’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes signed_rounded_r[lane], the value named ’signed rounded r’ in this block, equal to ‘product_negative_r[lane] ? -$signed(rounded_magnitude_r[lane]) : $signed(rounded_magnitude_r[lane])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named ’product negative r’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; rounded_magnitude_r: the value named ’rounded magnitude r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SIGN, the machine performs the named control phase..

Line 310: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 311: `state <= ST_DECAY_SATURATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_DECAY_SATURATE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECAY_SATURATE: the value named ’st decay saturate’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SIGN, the machine performs the named control phase..

Line 312: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 314: `ST_DECAY_SATURATE: begin` This is the ST_DECAY_SATURATE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SATURATE, the machine performs the named control phase..

Line 315: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SATURATE, the machine performs the named control phase..

Line 316: `decay_term_r[lane] <= saturate_s24(signed_rounded_r[lane]);` This makes decay_term_r[lane], the value named ’decay term r’ in this block, equal to ‘saturate_s24(signed_rounded_r[lane])‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_s24: the value named ’saturate s24’ in this block; signed_rounded_r: the value named ’signed rounded r’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SATURATE, the machine performs the named control phase..

Line 317: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 318: `state <= ST_VELOCITY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VELOCITY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VELOCITY: the momentum history retained by the SGD optimizer. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SATURATE, the machine performs the named control phase..

Line 319: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DECAY_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 321: `ST_VELOCITY: begin` This is the ST_VELOCITY case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY, the machine performs the named control phase..

Line 322: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY, the machine performs the named control phase..

Line 323: `two_term_sum_r[lane] <=` This statement continues through source line 325; the explanation uses the complete expression rather than only the first visual fragment. This makes two_term_sum_r[lane], the value named ’two term sum r’ in this block, equal to ‘{gradient_r[lane][23], gradient_r[lane]} + {momentum_term_r[lane][23], momentum_term_r[lane]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are gradient_r: the signed loss derivative used to update a learned parameter; lane: one parallel arithmetic element within a vector or tensor datapath; momentum_term_r: the value named ’momentum term r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY, the machine performs the named control phase..

Line 324: `{gradient_r[lane][23], gradient_r[lane]}` This is continuation line 2 of the 3-line logical statement that begins at source line 323. Its exact fragment is ‘{gradient_r[lane][23], gradient_r[lane]}‘. The important operands are gradient_r: the signed loss derivative used to update a learned parameter; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes two_term_sum_r[lane], the value named ’two term sum r’ in this block, equal to ‘{gradient_r[lane][23], gradient_r[lane]} + {momentum_term_r[lane][23], momentum_term_r[lane]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are gradient_r: the signed loss derivative used to update a learned parameter; lane: one parallel

arithmetic element within a vector or tensor datapath; momentum_term_r: the value named 'momentum term r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY, the machine performs the named control phase..

Line 325: `+ {momentum_term_r[lane][23], momentum_term_r[lane]};` This is continuation line 3 of the 3-line logical statement that begins at source line 323. Its exact fragment is '+ {momentum_term_r[lane][23], momentum_term_r[lane]};'. The important operands are momentum_term_r: the value named 'momentum term r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes two_term_sum_r[lane], the value named 'two term sum r' in this block, equal to '{gradient_r[lane][23], gradient_r[lane]} + {momentum_term_r[lane][23], momentum_term_r[lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are gradient_r: the signed loss derivative used to update a learned parameter; lane: one parallel arithmetic element within a vector or tensor datapath; momentum_term_r: the value named 'momentum term r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY, the machine performs the named control phase..

Line 326: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 327: `state <= ST_VELOCITY_SUM;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_VELOCITY_SUM'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VELOCITY_SUM: the momentum history retained by the SGD optimizer. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY, the machine performs the named control phase..

Line 328: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 330: `ST_VELOCITY_SUM: begin` This is the ST_VELOCITY_SUM case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SUM, the machine performs the named control phase..

Line 331: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SUM, the machine performs the named control phase..

Line 332: `velocity_sum_r[lane] <=` This statement continues through source line 333; the explanation uses the complete expression rather than only the first visual fragment. This makes velocity_sum_r[lane], the momentum history retained by the SGD optimizer, equal to '{two_term_sum_r[lane][24], two_term_sum_r[lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are two_term_sum_r: the value named 'two term sum r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SUM, the machine performs the named control phase..

Line 333: `{two_term_sum_r[lane][24], two_term_sum_r[lane]};` This is continuation line 2 of the 2-line logical statement that begins at source line 332. Its exact fragment is '{two_term_sum_r[lane][24], two_term_sum_r[lane]};'. The important operands are two_term_sum_r: the value named 'two term sum r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes velocity_sum_r[lane], the momentum history retained by the SGD optimizer, equal to '{two_term_sum_r[lane][24], two_term_sum_r[lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are two_term_sum_r: the value named 'two term sum r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SUM, the machine performs the named control phase..

Line 334: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SUM, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 335: `state <= ST_VELOCITY_SATURATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_VELOCITY_SATURATE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VELOCITY_SATURATE: the momentum history retained by the SGD optimizer. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SUM, the machine performs the named control phase..

Line 336: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SUM, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 338: `ST_VELOCITY_SATURATE: begin` This is the ST_VELOCITY_SATURATE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SATURATE, the machine performs the named control phase..

Line 339: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SATURATE, the machine performs the named control phase..

Line 340: `next_velocity_r[lane] <= saturate_s24(` This statement continues through source line 341; the explanation uses the complete expression rather than only the first visual fragment. This makes next_velocity_r[lane], the momentum history retained by the SGD optimizer, equal to 'saturate_s24({{16{velocity_sum_r[lane][25]}}, velocity_sum_r[lane]})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_s24: the value named 'saturate s24' in this block; velocity_sum_r: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SATURATE, the machine performs the named control phase..

Line 341: `{{16{velocity_sum_r[lane][25]}}, velocity_sum_r[lane]});` This is continuation line 2 of the 2-line logical statement that begins at source line 340. Its exact fragment is '{{16{velocity_sum_r[lane][25]}}, velocity_sum_r[lane]});'. The important operands are velocity_sum_r: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes next_velocity_r[lane], the momentum history retained by the SGD optimizer, equal to 'saturate_s24({{16{velocity_sum_r[lane][25]}}, velocity_sum_r[lane]})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_s24: the value named 'saturate s24' in this block; velocity_sum_r: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SATURATE, the machine performs the named control phase..

Line 342: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 343: `state <= ST_UPDATE_DIRECTION;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_UPDATE_DIRECTION'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_DIRECTION: the value named 'st update direction' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SATURATE, the machine performs the named control phase..

Line 344: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VELOCITY_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 346: `ST_UPDATE_DIRECTION: begin` This is the ST_UPDATE_DIRECTION case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION, the machine performs the named control phase..

Line 347: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION, the machine performs the named control phase..

Line 348: `two_term_sum_r[lane] <=` This statement continues through source line 350; the explanation uses the complete expression rather than only the first visual fragment. This makes two_term_sum_r[lane], the value named 'two term sum r' in this block, equal to '{next_velocity_r[lane][23], next_velocity_r[lane]} + {decay_term_r[lane][23], decay_term_r[lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_velocity_r: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath; decay_term_r: the value named 'decay term r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION, the machine performs the named control phase..

Line 349: `{next_velocity_r[lane][23], next_velocity_r[lane]}` This is continuation line 2 of the 3-line logical statement that begins at source line 348. Its exact fragment is '{next_velocity_r[lane][23], next_velocity_r[lane]}'. The important operands are next_velocity_r: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes two_term_sum_r[lane], the value named 'two term sum r' in this block, equal to '{next_velocity_r[lane][23], next_velocity_r[lane]} + {decay_term_r[lane][23], decay_term_r[lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_velocity_r: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath; decay_term_r: the value named 'decay term r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION, the machine performs the named control phase..

Line 350: `+ {decay_term_r[lane][23], decay_term_r[lane]};` This is continuation line 3 of the 3-line logical statement that begins at source line 348. Its exact fragment is '+ {decay_term_r[lane][23], decay_term_r[lane]};'. The important operands are decay_term_r: the value named 'decay term r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes two_term_sum_r[lane], the value named 'two term sum r' in this block, equal to '{next_velocity_r[lane][23], next_velocity_r[lane]} + {decay_term_r[lane][23], decay_term_r[lane]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_velocity_r: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath; decay_term_r: the value named 'decay term r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION, the machine performs the named control phase..

Line 351: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 352: `state <= ST_UPDATE_DIRECTION_SATURATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_UPDATE_DIRECTION_SATURATE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_DIRECTION_SATURATE: the value named 'st update direction saturate' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION, the machine performs the named control phase..

Line 353: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 355: `ST_UPDATE_DIRECTION_SATURATE: begin` This is the ST_UPDATE_DIRECTION_SATURATE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION_SATURATE, the machine performs the named control phase..

Line 356: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION_SATURATE, the machine performs the named control phase..

Line 357: `// The retained velocity excludes weight decay. This` Design intent: The retained velocity excludes weight decay. This This comment anchors the surrounding logic in the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state.

Line 358: `// scratch register holds only the SGDW update direction.` Design intent: scratch register holds only the SGDW update direction. This comment anchors the surrounding logic in the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state.

Line 359: `momentum_term_r[lane] <= saturate_s24(` This statement continues through source line 360; the explanation uses the complete expression rather than only the first visual fragment. This makes momentum_term_r[lane], the value named 'momentum term r' in this block, equal to 'saturate_s24({{17{two_term_sum_r[lane][24]}}, two_term_sum_r[lane]} It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_s24: the value named 'saturate s24' in this block; two_term_sum_r: the value named 'two term sum r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION_SATURATE, the machine performs the named control phase..

Line 360: `{{17{two_term_sum_r[lane][24]}}, two_term_sum_r[lane]});` This is continuation line 2 of the 2-line logical statement that begins at source line 359. Its exact fragment is '{{17{two_term_sum_r[lane][24]}}, two_term_sum_r[lane]}) The important operands are two_term_sum_r: the value named 'two term sum r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes momentum_term_r[lane], the value named 'momentum term r' in this block, equal to 'saturate_s24({{17{two_term_sum_r[lane][24]}}, two_term_sum_r[lane]})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_s24: the value named 'saturate s24' in this block; two_term_sum_r: the value named 'two term sum r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION_SATURATE, the machine performs the named control phase..

Line 361: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 362: `state <= ST_UPDATE_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_UPDATE_MULTIPLY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_MULTIPLY: the value named 'st update multiply' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION_SATURATE, the machine performs the named control phase..

Line 363: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_DIRECTION_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 365: `ST_UPDATE_MULTIPLY: begin` This is the ST_UPDATE_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 366: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 367: `multiplier_operand_left_r[lane] <= multiplier_left[lane];` This makes multiplier_operand_left_r[lane], the value named ’multiplier operand left r’ in this block, equal to ‘multiplier_left[lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_left: the value named ’multiplier left’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 368: `multiplier_operand_right_r[lane] <= multiplier_right[lane];` This makes multiplier_operand_right_r[lane], the value named ’multiplier operand right r’ in this block, equal to ‘multiplier_right[lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_right: the value named ’multiplier right’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 369: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 370: `state <= ST_UPDATE_PRODUCT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_UPDATE_PRODUCT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_PRODUCT: the value named ’st update product’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase..

Line 371: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 373: `ST_UPDATE_PRODUCT: begin` This is the ST_UPDATE_PRODUCT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT, the machine performs the named control phase..

Line 374: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT, the machine performs the named control phase..

Line 375: `multiplier_product_pipeline_r[lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are multiplier_product_pipeline_r: the value named ’multiplier product pipeline r’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 376: `<= multiplier_product[lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are multiplier_product: the value named 'multiplier product' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 377: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 378: `state <= ST_UPDATE_PRODUCT_RELAY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_UPDATE_PRODUCT_RELAY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_PRODUCT_RELAY: the value named 'st update product relay' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT, the machine performs the named control phase..

Line 379: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 381: `ST_UPDATE_PRODUCT_RELAY: begin` This is the ST_UPDATE_PRODUCT_RELAY case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT_RELAY, the machine performs the named control phase..

Line 382: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT_RELAY, the machine performs the named control phase..

Line 383: `product_r[lane] <= multiplier_product_pipeline_r[lane];` This makes product_r[lane], the value named 'product r' in this block, equal to 'multiplier_product_pipeline_r[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_product_pipeline_r: the value named 'multiplier product pipeline r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT_RELA the machine performs the named control phase..

Line 384: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT_RELAY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 385: `state <= ST_UPDATE_ABSOLUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_UPDATE_ABSOLUTE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_ABSOLUTE: the value named 'st update absolute' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT_RELAY, the machine performs the named control phase..

Line 386: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_PRODUCT_RELAY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 388: `ST_UPDATE_ABSOLUTE: begin` This is the ST_UPDATE_ABSOLUTE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ABSOLUTE, the machine performs the named control phase..

Line 389: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ABSOLUTE, the machine performs the named control phase..

Line 390: `product_negative_r[lane] <= product_r[lane][41];` This makes product_negative_r[lane], the value named 'product negative r' in this block, equal to 'product_r[lane][41]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ABSOLUTE, the machine performs the named control phase..

Line 391: `product_magnitude_r[lane] <= product_r[lane][41]` This statement continues through source line 392; the explanation uses the complete expression rather than only the first visual fragment. This makes product_magnitude_r[lane], the value named 'product magnitude r' in this block, equal to 'product_r[lane][41] ? -product_r[lane] : product_r[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ABSOLUTE, the machine performs the named control phase..

Line 392: `? -product_r[lane] : product_r[lane];` This is continuation line 2 of the 2-line logical statement that begins at source line 391. Its exact fragment is '? -product_r[lane] : product_r[lane];'. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes product_magnitude_r[lane], the value named 'product magnitude r' in this block, equal to 'product_r[lane][41] ? -product_r[lane] : product_r[lane]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_r: the value named 'product r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ABSOLUTE, the machine performs the named control phase..

Line 393: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ABSOLUTE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 394: `state <= ST_UPDATE_BIAS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_UPDATE_BIAS'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_BIAS: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ABSOLUTE, the machine performs the named control phase..

Line 395: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ABSOLUTE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 397: `ST_UPDATE_BIAS: begin` This is the ST_UPDATE_BIAS case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_BIAS, the machine performs the named control phase..

Line 398: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_BIAS, the machine performs the named control phase..

Line 399: `rounded_magnitude_r[lane] <=` This statement continues through source line 400; the explanation uses the complete expression rather than only the first visual fragment. This makes rounded_magnitude_r[lane], the value named 'rounded magnitude r' in this block, equal to 'product_magnitude_r[lane] + update_rounding_bias_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_magnitude_r: the value named 'product magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; update_rounding_bias_r: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_BIAS, the machine performs the named control phase..

Line 400: `product_magnitude_r[lane] + update_rounding_bias_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 399. Its exact fragment is 'product_magnitude_r[lane] + update_rounding_bias_r;'. The important operands are product_magnitude_r: the value named 'product magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; update_rounding_bias_r: an additive learned offset applied after accumulation. Read together, the complete operation is: This makes rounded_magnitude_r[lane], the value named 'rounded magnitude r' in this block, equal to 'product_magnitude_r[lane] + update_rounding_bias_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_magnitude_r: the value named 'product magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; update_rounding_bias_r: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_BIAS, the machine performs the named control phase..

Line 401: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_BIAS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 402: `round_shift_counter_r <= update_right_shift_r;` This makes round_shift_counter_r, a hardware telemetry register that accumulates measured events, equal to 'update_right_shift_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are update_right_shift_r: the value named 'update right shift r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_BIAS, the machine performs the named control phase..

Line 403: `state <= ST_UPDATE_ROUND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_UPDATE_ROUND'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_ROUND: the value named 'st update round' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_BIAS, the machine performs the named control phase..

Line 404: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_BIAS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 406: `ST_UPDATE_ROUND: begin` This is the ST_UPDATE_ROUND case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase..

Line 407: `if (round_shift_counter_r == 0) begin` This condition selects the following hardware branch only when 'round_shift_counter_r == 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are round_shift_counter_r: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase..

Line 408: `state <= ST_UPDATE_SIGN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_UPDATE_SIGN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_SIGN: the value named 'st update sign' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase..

Line 409: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 410: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase..

Line 411: `rounded_magnitude_r[lane]` Within the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 412: `<= rounded_magnitude_r[lane] >> 1;` Within the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 413: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 414: `round_shift_counter_r <= round_shift_counter_r - 1'b1;` This makes round_shift_counter_r, a hardware telemetry register that accumulates measured events, equal to 'round_shift_counter_r - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are round_shift_counter_r: a hardware telemetry register that accumulates measured events; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase..

Line 415: `if (round_shift_counter_r == 1) state <= ST_UPDATE_SIGN;` This condition selects the following hardware branch only when 'round_shift_counter_r == 1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are round_shift_counter_r: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase..

Line 416: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 417: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_ROUND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 419: `ST_UPDATE_SIGN: begin` This is the ST_UPDATE_SIGN case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SIGN, the machine performs the named control phase..

Line 420: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SIGN, the machine performs the named control phase..

Line 421: `update_value_r[lane] <= product_negative_r[lane]` This statement continues through source line 423; the explanation uses the complete expression rather than only the first visual fragment. This makes update_value_r[lane], the value named 'update value r' in this block, equal to 'product_negative_r[lane] ? -$signed(rounded_magnitude_r[lane]) : $signed(rounded_magnitude_r[lane])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named 'product negative r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; rounded_magnitude_r: the value named 'rounded magnitude r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SIGN, the machine performs the named control phase..

Line 422: `? -$signed(rounded_magnitude_r[lane])` This is continuation line 2 of the 3-line logical statement that begins at source line 421. Its exact fragment is '? -$signed(rounded_magnitude_r[lane])'. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes update_value_r[lane], the value named 'update value r' in this block, equal to 'product_negative_r[lane] ? -$signed(rounded_magnitude_r[lane]) : $signed(rounded_magnitude_r[lane])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named 'product negative r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; rounded_magnitude_r: the value named 'rounded magnitude r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SIGN, the machine performs the named control phase..

Line 423: `: $signed(rounded_magnitude_r[lane]);` This is continuation line 3 of the 3-line logical statement that begins at source line 421. Its exact fragment is ': $signed(rounded_magnitude_r[lane]);'. The important operands are rounded_magnitude_r: the value named 'rounded magnitude r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes update_value_r[lane], the value named 'update value r' in this block, equal to 'product_negative_r[lane] ? -$signed(rounded_magnitude_r[lane]) : $signed(rounded_magnitude_r[lane])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named 'product negative r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; rounded_magnitude_r: the value named 'rounded magnitude r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SIGN, the machine performs the named control phase..

Line 424: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 425: `state <= ST_UPDATE_SUBTRACT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_UPDATE_SUBTRACT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_SUBTRACT: the value named 'st update subtract' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SIGN, the machine performs the named control phase..

Line 426: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 428: `ST_UPDATE_SUBTRACT: begin` This is the ST_UPDATE_SUBTRACT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SUBTRACT, the machine performs the named control phase..

Line 429: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SUBTRACT, the machine performs the named control phase..

Line 430: `weight_difference_r[lane] <=` This statement continues through source line 432; the explanation uses the complete expression rather than only the first visual fragment. This makes weight_difference_r[lane], a learned coefficient multiplied by an activation, equal to ‘{{26{weight_r[lane][15]}}, weight_r[lane]} - update_value_r[lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_r: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; update_value_r: the value named 'update value r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SUBTRACT, the machine performs the named control phase..

Line 431: `{{26{weight_r[lane][15]}}, weight_r[lane]}` This is continuation line 2 of the 3-line logical statement that begins at source line 430. Its exact fragment is ‘{{26{weight_r[lane][15]}}, weight_r[lane]}‘. The important operands are weight_r: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes weight_difference_r[lane], a learned coefficient multiplied by an activation, equal to ‘{{26{weight_r[lane][15]}}, weight_r[lane]} - update_value_r[lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_r: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; update_value_r: the value named 'update value r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SUBTRACT, the machine performs the named control phase..

Line 432: `- update_value_r[lane];` This is continuation line 3 of the 3-line logical statement that begins at source line 430. Its exact fragment is ‘- update_value_r[lane];‘. The important operands are update_value_r: the value named 'update value r' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes weight_difference_r[lane], a learned coefficient multiplied by an activation, equal to ‘{{26{weight_r[lane][15]}}, weight_r[lane]} - update_value_r[lane]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_r: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; update_value_r: the value named 'update value r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SUBTRACT, the machine performs the named control phase..

Line 433: `updated_velocity_vector[lane*24 +: 24] <= next_velocity_r[lane];` Within the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SUBTRACT, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are updated_velocity_vector: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath; next_velocity_r: the momentum history retained by the SGD optimizer.

Line 434: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SUBTRACT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 435: `state <= ST_UPDATE_SATURATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_UPDATE_SATURATE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_UPDATE_SATURATE: the value named 'st update saturate' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SUBTRACT, the machine performs the named control phase..

Line 436: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SUBTRACT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 438: `ST_UPDATE_SATURATE: begin` This is the ST_UPDATE_SATURATE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SATURATE, the machine performs the named control phase..

Line 439: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware

rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SATURATE, the machine performs the named control phase..

Line 440: `updated_weight_vector[lane*16 +: 16] <=` This statement continues through source line 441; the explanation uses the complete expression rather than only the first visual fragment. Within the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are updated_weight_vector: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; saturate_s16: the value named 'saturate s16' in this block.

Line 441: `saturate_s16(weight_difference_r[lane]);` This is continuation line 2 of the 2-line logical statement that begins at source line 440. Its exact fragment is 'saturate_s16(weight_difference_r[lane]);'. The important operands are saturate_s16: the value named 'saturate s16' in this block; weight_difference_r: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: Within the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SATURATE, the machine performs the named control phase., this exact statement contributes to the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state. The important operands are updated_weight_vector: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; saturate_s16: the value named 'saturate s16' in this block.

Line 442: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 443: `update_count <= update_count + LANES;` This makes update_count, an integer count retained for control or measurable telemetry, equal to 'update_count + LANES'. It updates after the active clock edge together with other nonblocking assignments. The important operands are update_count: an integer count retained for control or measurable telemetry; LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SATURATE, the machine performs the named control phase..

Line 444: `state <= ST_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT: the value named 'st output' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SATURATE, the machine performs the named control phase..

Line 445: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_UPDATE_SATURATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 447: `ST_OUTPUT: begin` This is the ST_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 448: `if (output_ready) state <= ST_IDLE;` This condition selects the following hardware branch only when 'output_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 449: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 451: `default:  state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘state <= ST_IDLE;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 452: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 453: `if (clear_counters) update_count <= 32'd0;` This condition selects the following hardware branch only when ‘clear_counters‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_counters: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge.

Line 454: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 455: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 457: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 458: `if (LANES != 8) $fatal(1, "neuraedge_sgd_optimizer8 currently requires LANES=8");` This condition selects the following hardware branch only when ‘LANES != 8) $fatal(1, "neuraedge_sgd_optimizer8 currently requires LANES=8"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are LANES: one parallel arithmetic element within a vector or tensor datapath; fatal: the value named 'fatal' in this block; neuraedge_sgd_optimizer8: the value named 'neuraedge sgd optimizer8' in this block. Context: the module's structural declarations.

Line 459: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 460: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 462: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state.

19 rtl/neuraedge_tensor_tile.sv

Role in the complete system: implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

19.1 Beginner architecture map

Dataflow responsibility The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_tensor_tile into physical nets and primitives. The source declares 22 input-port lines, 13 output-port lines, 1 clocked process(es), and 3 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_ACCUMULATE_OUTPUT_COUNT, ST_CAPTURE_TARGET, ST_COMBINE_ZERO_POINT, ST_DRAIN, ST_IDLE, ST_LATCH_ZERO_POINT, ST_OUTPUT_PIPELINE, ST_PREPARE_OUTPUT_COUNT, ST_PREPARE_ZERO_POINT, ST_RESTORE_ADD, ST_RESTORE_MULTIPL ST_RESTORE_RELAY, ST_RUN, ST_SPARSE_DRAIN. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates end as endfunction, unique as case, end as for, neuraedge_mac2_u8 as u_mac2, end as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

19.2 Complete source

```
`timescale 1ns / 1ps
`default_nettype none

module neuraedge_tensor_tile #(
    parameter int ROW_LANES = 8,
    parameter int COLUMN_LANES = 16,
    parameter int ACC_BITS = 32
) (
    input  wire clk,
    input  wire rst_n,

    input  wire start,
    input  wire [1:0] compute_mode,
    input  wire [3:0] active_rows,
    input  wire [5:0] active_columns,
    input  wire [7:0] activation_zero_point,
    input  wire [7:0] weight_zero_point,
    output logic config_error,

    input  wire issue_valid,
    output logic issue_ready,
    input  wire [ROW_LANES*8-1:0] activation_vector,
    input  wire [COLUMN_LANES*8-1:0] outer_weight_vector,
    input  wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector,
    input  wire issue_last,

    output logic busy,
    output logic result_valid,
    input  wire result_ready,
    output logic signed [47:0] result_value,
    output logic [3:0] result_row,
    output logic [5:0] result_column,
    output logic result_last,

    output logic [31:0] debug_accepted_issues,
    output logic [31:0] debug_skipped_issues,
    output logic [31:0] debug_retired_issues,
    output logic [63:0] debug_dense_macs,
    output logic [63:0] debug_spike_adds
```

```
);
    localparam logic [1:0] MODE_DENSE_OUTER = 2'd0;
    localparam logic [1:0] MODE_DENSE_DOT8 = 2'd1;
    localparam logic [1:0] MODE_BINARY_WEIGHTED = 2'd2;
    localparam logic [1:0] MODE_TERNARY_WEIGHTED = 2'd3;
    localparam int COLUMN_PAIRS = COLUMN_LANES / 2;
    localparam int RESULT_FIFO_DEPTH = ROW_LANES * COLUMN_LANES;
    localparam int RESULT_FIFO_ADDRESS_BITS = $clog2(RESULT_FIFO_DEPTH);
    localparam int RESULT_FIFO_COUNT_BITS = $clog2(RESULT_FIFO_DEPTH + 1);

    typedef enum logic [3:0] {
        ST_IDLE = 4'd0,
        ST_RUN = 4'd1,
        ST_DRAIN = 4'd2,
        ST_PREPARE_ZERO_POINT = 4'd3,
        ST_RESTORE_MULTIPLY = 4'd4,
        ST_OUTPUT_PIPELINE = 4'd5,
        ST_LATCH_ZERO_POINT = 4'd6,
        ST_RESTORE_ADD = 4'd7,
        ST_SPARSE_DRAIN = 4'd8,
        ST_CAPTURE_TARGET = 4'd9,
        ST_COMBINE_ZERO_POINT = 4'd10,
        ST_PREPARE_OUTPUT_COUNT = 4'd11,
        ST_ACCUMULATE_OUTPUT_COUNT = 4'd12,
        ST_RESTORE_RELAY = 4'd13
    } state_t;

    state_t state;
    (* dont_touch = "yes" *) logic [1:0] mode_r;
    (* dont_touch = "yes" *) logic [3:0] rows_r;
    logic [ROW_LANES-1:0] active_row_mask_r;
    logic [5:0] columns_r;
    logic [7:0] activation_zp_r;
    logic [7:0] weight_zp_r;

    logic signed [ACC_BITS-1:0] accumulator [0:ROW_LANES-1][0:COLUMN_LANES-1];
    // Keep one physical clear source per row. A command-start state bit driving
    // every accumulator bit creates a long, high-fanout control path after
    // placement; these registered row-local pulses perform the same clear one
    // cycle after start, before the issue pipeline can produce a delta.
    (* dont_touch = "yes", max_fanout = 96 *)
    logic accumulator_clear_row_r [0:ROW_LANES-1];
    // Column sums use the same delayed-clear scheme. One register per column
    // keeps the command-start decode out of a 512-bit distributed CE network.
    (* dont_touch = "yes", max_fanout = 48 *)
    logic column_sum_clear_r [0:COLUMN_LANES-1];
    (* dont_touch = "yes" *) logic accumulate_delta_valid_r;
    logic signed [20:0] accumulate_delta_r [0:ROW_LANES-1][0:COLUMN_LANES-1];
    logic [31:0] row_sum [0:ROW_LANES-1];
    logic [31:0] column_sum [0:COLUMN_LANES-1];
    logic sum_update_valid_r;
    logic [15:0] row_increment_r [0:ROW_LANES-1];
    logic [15:0] column_increment_r [0:COLUMN_LANES-1];
    logic [31:0] effective_reduction;

    logic [11:0] mac_issue_count;
    // The counter is updated in both run and command-boundary states.  A local
    // D-input hold avoids a long FSM-to-CE route across the MAC array.
    (* extract_enable = "no" *) logic [11:0] mac_retire_count;
    logic [11:0] target_mac_retires;
    // One command can execute at most 2048*8*16 = 262144 useful operations,
    // so 19 bits represent the exact per-command telemetry range. The
    // increment is registered before accumulation to keep popcount and lane
    // sizing logic out of the counter carry chain at 200 MHz.
    logic [18:0] debug_dense_macs_r;
    logic [18:0] debug_spike_adds_r;
    logic [7:0] metric_increment_r;
    logic metric_increment_valid_r;
    logic [3:0] sparse_active_count_r;
    logic sparse_metric_valid_r;
    logic [7:0] dense_macs_per_issue_r;
    (* dont_touch = "yes" *) logic drain_complete_r;
    (* use_dsp = "yes" *) logic [15:0] zero_point_product_r;
    logic [31:0] zero_restore_reduction_operand_r;
    logic [15:0] zero_restore_zp_operand_r;
    (* use_dsp = "yes" *) logic [31:0] zero_restore_low_product_r;
    (* use_dsp = "yes" *) logic [31:0] zero_restore_high_product_r;
    logic [31:0] zero_restore_low_relay_r;
    logic [31:0] zero_restore_high_relay_r;
    logic [47:0] zero_point_restore_r;
    logic [7:0] launch_total_r;
    (* extract_enable = "no" *) logic [7:0] launch_remaining_r;
    logic [3:0] launch_rows_remaining_r;
    (* extract_enable = "no" *) logic [5:0] launch_columns_remaining_r;
    (* extract_enable = "no" *) logic [3:0] launch_row_r;
    (* extract_enable = "no" *) logic [5:0] launch_column_r;
    logic pread_valid_r;
```

```
    logic signed [ACC_BITS-1:0] pread_accumulator_by_row_r [0:ROW_LANES-1];
    logic [31:0] pread_row_sum_r;
    logic [31:0] pread_column_sum_r;
    logic [3:0] pread_row_r;
    logic [5:0] pread_column_r;
    (* dont_touch = "yes" *) logic p0_valid_r;
    logic signed [63:0] p0_raw_r;
    logic [31:0] p0_row_sum_r;
    logic [31:0] p0_column_sum_r;
    logic [7:0] p0_row_zp_low_r;
    logic [7:0] p0_row_zp_high_r;
    logic [7:0] p0_column_zp_low_r;
    logic [7:0] p0_column_zp_high_r;
    logic [3:0] p0_row_r;
    logic [5:0] p0_column_r;
    (* dont_touch = "yes" *) logic p1_valid_r;
    logic signed [63:0] p1_raw_r;
    logic [23:0] p1_row_product_low_r;
    logic [23:0] p1_row_product_high_r;
    logic [23:0] p1_column_product_low_r;
    logic [23:0] p1_column_product_high_r;
    logic [3:0] p1_row_r;
    logic [5:0] p1_column_r;
    (* dont_touch = "yes" *) logic p2_valid_r;
    logic signed [63:0] p2_raw_r;
    logic signed [63:0] p2_row_correction_r;
    logic signed [63:0] p2_column_correction_r;
    logic [3:0] p2_row_r;
    logic [5:0] p2_column_r;
    (* dont_touch = "yes" *) logic p3_valid_r;
    logic signed [63:0] p3_corrected_r;
    logic signed [63:0] p3_column_correction_r;
    logic [3:0] p3_row_r;
    logic [5:0] p3_column_r;
    (* dont_touch = "yes" *) logic p4_valid_r;
    logic signed [63:0] p4_corrected_r;
    logic [3:0] p4_row_r;
    logic [5:0] p4_column_r;
    (* dont_touch = "yes" *) logic p5_valid_r;
    logic signed [47:0] p5_result_r;
    logic [3:0] p5_row_r;
    logic [5:0] p5_column_r;
    logic output_pipeline_enable;
    logic [RESULT_FIFO_COUNT_BITS-1:0] result_fifo_count_r;
    (* dont_touch = "yes" *) logic result_fifo_has_space_r;
    (* dont_touch = "yes" *) logic result_fifo_nonempty_r;
    logic [RESULT_FIFO_ADDRESS_BITS-1:0] result_fifo_write_pointer_r;
    logic [RESULT_FIFO_ADDRESS_BITS-1:0] result_fifo_read_pointer_r;
    logic [RESULT_FIFO_ADDRESS_BITS-1:0] result_fifo_last_address_r;
    // One command emits at most 8*16 records. Buffering that complete tile
    // removes downstream backpressure from every arithmetic pipeline CE while
    // retaining an asynchronous read suitable for a stable AXI-Stream output.
    (* ram_style = "distributed" *) logic [57:0] result_fifo_payload_r
        [0:RESULT_FIFO_DEPTH-1];
    logic result_stage_valid_r;
    logic [57:0] result_stage_payload_r;
    logic result_stage_last_r;
    wire result_fifo_push = state == ST_OUTPUT_PIPELINE
        && p5_valid_r && result_fifo_has_space_r;
    wire result_fifo_pop = state == ST_OUTPUT_PIPELINE
        && result_fifo_nonempty_r
        && (!result_stage_valid_r
            || (result_stage_valid_r && result_ready));
    wire result_stage_pop = state == ST_OUTPUT_PIPELINE
        && result_stage_valid_r && result_ready;

    // Zero-point restoration is computed once per tile. Row and column products
    // are independent and therefore form a throughput-one correction pipeline.
    (* use_dsp = "yes" *) wire [23:0] p0_row_product_low =
        p0_row_sum_r[15:0] * p0_row_zp_low_r;
    (* use_dsp = "yes" *) wire [23:0] p0_row_product_high =
        p0_row_sum_r[31:16] * p0_row_zp_high_r;
    (* use_dsp = "yes" *) wire [23:0] p0_column_product_low =
        p0_column_sum_r[15:0] * p0_column_zp_low_r;
    (* use_dsp = "yes" *) wire [23:0] p0_column_product_high =
        p0_column_sum_r[31:16] * p0_column_zp_high_r;

    logic [7:0] mac_activation [0:ROW_LANES-1][0:COLUMN_PAIRS-1];
    logic [7:0] mac_weight_lo [0:ROW_LANES-1][0:COLUMN_PAIRS-1];
    logic [7:0] mac_weight_hi [0:ROW_LANES-1][0:COLUMN_PAIRS-1];
    wire [15:0] mac_product_lo [0:ROW_LANES-1][0:COLUMN_PAIRS-1];
    wire [15:0] mac_product_hi [0:ROW_LANES-1][0:COLUMN_PAIRS-1];
    wire mac_product_valid [0:ROW_LANES-1][0:COLUMN_PAIRS-1];
    logic dot_pair_valid_r;
    logic dot_half_valid_r;
    logic dot_sum_valid_r;
    logic [16:0] dot_pair_lo_r [0:COLUMN_PAIRS-1][0:3];
```

```
    logic [16:0] dot_pair_hi_r [0:COLUMN_PAIRS-1][0:3];
    logic [17:0] dot_half_lo_r [0:COLUMN_PAIRS-1][0:1];
    logic [17:0] dot_half_hi_r [0:COLUMN_PAIRS-1][0:1];
    logic [19:0] dot_sum_lo_r [0:COLUMN_PAIRS-1];
    logic [19:0] dot_sum_hi_r [0:COLUMN_PAIRS-1];
    logic dot8_pair_valid_r;
    logic [8:0] dot8_row_pair_r [0:3];
    // As with the retire counter, keep the classify-state hold on the data
    // input so each pair register can be implemented near its adder.
    (* extract_enable = "no" *)
    logic [8:0] dot8_column_pair_r [0:COLUMN_LANES-1][0:3];
    logic dot8_increment_half_valid_r;
    logic [9:0] dot8_row_half_r [0:1];
    logic [9:0] dot8_column_half_r [0:COLUMN_LANES-1][0:1];

    logic issue_accept;
    logic input_done_r;
    logic issue_match_valid_r;
    logic [7:0] issue_zero_matches_r;
    logic issue_match_last_r;
    logic [ROW_LANES*8-1:0] issue_match_activation_vector_r;
    logic [COLUMN_LANES*8-1:0] issue_match_outer_weight_vector_r;
    logic [COLUMN_LANES*ROW_LANES*8-1:0] issue_match_dot_weight_vector_r;
    logic [7:0] issue_zero_matches_comb;

    function automatic logic [3:0] popcount8(input logic [7:0] value);
        logic [1:0] pair0;
        logic [1:0] pair1;
        logic [1:0] pair2;
        logic [1:0] pair3;
        logic [2:0] half0;
        logic [2:0] half1;
        begin
            pair0 = value[0] + value[1];
            pair1 = value[2] + value[3];
            pair2 = value[4] + value[5];
            pair3 = value[6] + value[7];
            half0 = pair0 + pair1;
            half1 = pair2 + pair3;
            popcount8 = half0 + half1;
        end
    endfunction
    (* dont_touch = "yes" *) logic classify_valid_r;
    (* dont_touch = "yes" *) logic classify_is_zero_r;
    logic classify_last_r;
    logic [ROW_LANES*8-1:0] classify_activation_vector_r;
    logic [COLUMN_LANES*8-1:0] classify_outer_weight_vector_r;
    logic [COLUMN_LANES*ROW_LANES*8-1:0] classify_dot_weight_vector_r;
    logic dense_issue_enable;
    (* dont_touch = "yes" *) logic dense_issue_enable_r;
    logic [ROW_LANES*8-1:0] dense_activation_vector_r;
    logic [7:0] dense_weight_lo_r [0:ROW_LANES-1][0:COLUMN_PAIRS-1];
    logic [7:0] dense_weight_hi_r [0:ROW_LANES-1][0:COLUMN_PAIRS-1];
    logic dense_outer_tag0_r;
    logic dense_outer_tag1_r;
    logic dense_outer_tag2_r;
    logic sparse_prepare_valid_r;
    logic [ROW_LANES-1:0] sparse_prepare_active_r;
    logic [ROW_LANES-1:0] sparse_prepare_negative_r;
    logic [COLUMN_LANES*8-1:0] sparse_prepare_weight_vector_r;
    (* dont_touch = "yes" *) logic sparse_delta_valid_r;
    logic signed [8:0] sparse_delta_r [0:ROW_LANES-1][0:COLUMN_LANES-1];
    wire dense_product_valid = mac_product_valid[0][0];
    wire dense_accumulate_valid = dense_outer_tag2_r || dot_sum_valid_r;

    function automatic logic [15:0] sum_u8x8(input logic [63:0] values);
        logic [8:0] pair0;
        logic [8:0] pair1;
        logic [8:0] pair2;
        logic [8:0] pair3;
        logic [9:0] half0;
        logic [9:0] half1;
        begin
            pair0 = values[7:0] + values[15:8];
            pair1 = values[23:16] + values[31:24];
            pair2 = values[39:32] + values[47:40];
            pair3 = values[55:48] + values[63:56];
            half0 = pair0 + pair1;
            half1 = pair2 + pair3;
            sum_u8x8 = {6'd0, half0} + {6'd0, half1};
        end
    endfunction

    function automatic logic [19:0] sum_u16x8(
        input logic [15:0] value0,
        input logic [15:0] value1,
        input logic [15:0] value2,
```

```
        input logic [15:0] value3,
        input logic [15:0] value4,
        input logic [15:0] value5,
        input logic [15:0] value6,
        input logic [15:0] value7
    );
        logic [16:0] pair0;
        logic [16:0] pair1;
        logic [16:0] pair2;
        logic [16:0] pair3;
        logic [17:0] half0;
        logic [17:0] half1;
        begin
            pair0 = value0 + value1;
            pair1 = value2 + value3;
            pair2 = value4 + value5;
            pair3 = value6 + value7;
            half0 = pair0 + pair1;
            half1 = pair2 + pair3;
            sum_u16x8 = {2'd0, half0} + {2'd0, half1};
        end
    endfunction

    always_comb begin
        for (int lane = 0; lane < ROW_LANES; lane++) begin
            unique case (mode_r)
                MODE_DENSE_OUTER,
                MODE_DENSE_DOT8: issue_zero_matches_comb[lane] =
                    activation_vector[lane*8 +: 8] == activation_zp_r;
                MODE_BINARY_WEIGHTED,
                MODE_TERNARY_WEIGHTED: issue_zero_matches_comb[lane] =
                    activation_vector[lane*8 +: 8] == 8'd0;
                default: issue_zero_matches_comb[lane] = 1'b1;
            endcase
        end
    end

    assign issue_accept = issue_valid && issue_ready;
    assign dense_issue_enable = dense_issue_enable_r;

    always_comb begin
        for (int lane = 0; lane < ROW_LANES; lane++) begin
            for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin
                mac_activation[lane][pair_idx] =
                    dense_activation_vector_r[lane*8 +: 8];
                mac_weight_lo[lane][pair_idx] = dense_weight_lo_r[lane][pair_idx];
                mac_weight_hi[lane][pair_idx] = dense_weight_hi_r[lane][pair_idx];
            end
        end
    end

    for (genvar lane = 0; lane < ROW_LANES; lane++) begin : g_mac_lane
        for (genvar pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin : g_mac_pair
            neuraedge_mac2_u8 u_mac2 (
                .clk(clk),
                .rst_n(rst_n),
                .enable(dense_issue_enable),
                .activation(mac_activation[lane][pair_idx]),
                .weight_lo(mac_weight_lo[lane][pair_idx]),
                .weight_hi(mac_weight_hi[lane][pair_idx]),
                .product_valid(mac_product_valid[lane][pair_idx]),
                .product_lo(mac_product_lo[lane][pair_idx]),
                .product_hi(mac_product_hi[lane][pair_idx])
            );
        end
    end

    always_comb begin
        issue_ready = (state == ST_RUN) && !input_done_r;
        busy = (state != ST_IDLE);
        result_valid = (state == ST_OUTPUT_PIPELINE) && result_stage_valid_r;
        result_row = result_stage_payload_r[51:48];
        result_column = result_stage_payload_r[57:52];
        result_last = result_stage_last_r;
        result_value = result_stage_payload_r[47:0];
        debug_dense_macs = {45'd0, debug_dense_macs_r};
        debug_spike_adds = {45'd0, debug_spike_adds_r};
        // The FIFO holds the largest legal tile, so all arithmetic stages can
        // advance every cycle regardless of downstream READY.
        output_pipeline_enable = 1'b1;
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            state <= ST_IDLE;
            mode_r <= MODE_DENSE_OUTER;
            rows_r <= 4'd1;
```

```
            active_row_mask_r <= {{(ROW_LANES-1){1'b0}}, 1'b1};
            columns_r <= 6'd1;
            activation_zp_r <= 8'd0;
            weight_zp_r <= 8'd0;
            config_error <= 1'b0;
            effective_reduction <= 32'd0;
            mac_issue_count <= 12'd0;
            mac_retire_count <= 12'd0;
            target_mac_retires <= 12'd0;
            drain_complete_r <= 1'b0;
            zero_point_product_r <= 16'd0;
            zero_restore_reduction_operand_r <= 32'd0;
            zero_restore_zp_operand_r <= 16'd0;
            zero_restore_low_product_r <= 32'd0;
            zero_restore_high_product_r <= 32'd0;
            zero_restore_low_relay_r <= 32'd0;
            zero_restore_high_relay_r <= 32'd0;
            zero_point_restore_r <= 48'd0;
            launch_total_r <= 8'd0;
            launch_remaining_r <= 8'd0;
            launch_rows_remaining_r <= 4'd0;
            launch_columns_remaining_r <= 6'd0;
            launch_row_r <= 4'd0;
            launch_column_r <= 6'd0;
            pread_valid_r <= 1'b0;
            pread_row_sum_r <= 32'd0;
            pread_column_sum_r <= 32'd0;
            pread_row_r <= 4'd0;
            pread_column_r <= 6'd0;
            p0_valid_r <= 1'b0;
            p0_raw_r <= 64'sd0;
            p0_row_sum_r <= 32'd0;
            p0_column_sum_r <= 32'd0;
            p0_row_zp_low_r <= 8'd0;
            p0_row_zp_high_r <= 8'd0;
            p0_column_zp_low_r <= 8'd0;
            p0_column_zp_high_r <= 8'd0;
            p0_row_r <= 4'd0;
            p0_column_r <= 6'd0;
            p1_valid_r <= 1'b0;
            p1_raw_r <= 64'sd0;
            p1_row_product_low_r <= 24'd0;
            p1_row_product_high_r <= 24'd0;
            p1_column_product_low_r <= 24'd0;
            p1_column_product_high_r <= 24'd0;
            p1_row_r <= 4'd0;
            p1_column_r <= 6'd0;
            p2_valid_r <= 1'b0;
            p2_raw_r <= 64'sd0;
            p2_row_correction_r <= 64'sd0;
            p2_column_correction_r <= 64'sd0;
            p2_row_r <= 4'd0;
            p2_column_r <= 6'd0;
            p3_valid_r <= 1'b0;
            p3_corrected_r <= 64'sd0;
            p3_column_correction_r <= 64'sd0;
            p3_row_r <= 4'd0;
            p3_column_r <= 6'd0;
            p4_valid_r <= 1'b0;
            p4_corrected_r <= 64'sd0;
            p4_row_r <= 4'd0;
            p4_column_r <= 6'd0;
            p5_valid_r <= 1'b0;
            p5_result_r <= 48'sd0;
            p5_row_r <= 4'd0;
            p5_column_r <= 6'd0;
            result_fifo_count_r <= '0;
            result_fifo_has_space_r <= 1'b1;
            result_fifo_nonempty_r <= 1'b0;
            result_fifo_write_pointer_r <= '0;
            result_fifo_read_pointer_r <= '0;
            result_fifo_last_address_r <= '0;
            result_stage_valid_r <= 1'b0;
            result_stage_payload_r <= '0;
            result_stage_last_r <= 1'b0;
            accumulate_delta_valid_r <= 1'b0;
            debug_accepted_issues <= 32'd0;
            debug_skipped_issues <= 32'd0;
            debug_retired_issues <= 32'd0;
            debug_dense_macs_r <= 19'd0;
            debug_spike_adds_r <= 19'd0;
            metric_increment_r <= 8'd0;
            metric_increment_valid_r <= 1'b0;
            sparse_active_count_r <= 4'd0;
            sparse_metric_valid_r <= 1'b0;
            dense_macs_per_issue_r <= 8'd0;
            dot_sum_valid_r <= 1'b0;
```

```
            dot_pair_valid_r <= 1'b0;
            dot_half_valid_r <= 1'b0;
            dot8_pair_valid_r <= 1'b0;
            dot8_increment_half_valid_r <= 1'b0;
            sum_update_valid_r <= 1'b0;
            input_done_r <= 1'b0;
            issue_match_valid_r <= 1'b0;
            issue_zero_matches_r <= 8'd0;
            issue_match_last_r <= 1'b0;
            issue_match_activation_vector_r <= '0;
            issue_match_outer_weight_vector_r <= '0;
            issue_match_dot_weight_vector_r <= '0;
            classify_valid_r <= 1'b0;
            classify_is_zero_r <= 1'b0;
            classify_last_r <= 1'b0;
            classify_activation_vector_r <= '0;
            classify_outer_weight_vector_r <= '0;
            classify_dot_weight_vector_r <= '0;
            dense_issue_enable_r <= 1'b0;
            dense_activation_vector_r <= '0;
            dense_outer_tag0_r <= 1'b0;
            dense_outer_tag1_r <= 1'b0;
            dense_outer_tag2_r <= 1'b0;
            sparse_prepare_valid_r <= 1'b0;
            sparse_prepare_active_r <= '0;
            sparse_prepare_negative_r <= '0;
            sparse_prepare_weight_vector_r <= '0;
            sparse_delta_valid_r <= 1'b0;
            for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin
                dot_sum_lo_r[pair_idx] <= 20'd0;
                dot_sum_hi_r[pair_idx] <= 20'd0;
                for (int reduce_pair = 0; reduce_pair < 4; reduce_pair++) begin
                    dot_pair_lo_r[pair_idx][reduce_pair] <= 17'd0;
                    dot_pair_hi_r[pair_idx][reduce_pair] <= 17'd0;
                end
                for (int reduce_half = 0; reduce_half < 2; reduce_half++) begin
                    dot_half_lo_r[pair_idx][reduce_half] <= 18'd0;
                    dot_half_hi_r[pair_idx][reduce_half] <= 18'd0;
                end
            end
            for (int pair_idx = 0; pair_idx < 4; pair_idx++) begin
                dot8_row_pair_r[pair_idx] <= 9'd0;
            end
            dot8_row_half_r[0] <= 10'd0;
            dot8_row_half_r[1] <= 10'd0;
            for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                accumulator_clear_row_r[row_idx] <= 1'b0;
                pread_accumulator_by_row_r[row_idx] <= '0;
                row_sum[row_idx] <= 32'd0;
                row_increment_r[row_idx] <= 16'd0;
                for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin
                    dense_weight_lo_r[row_idx][pair_idx] <= 8'd0;
                    dense_weight_hi_r[row_idx][pair_idx] <= 8'd0;
                end
                for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                    accumulator[row_idx][column_idx] <= '0;
                    accumulate_delta_r[row_idx][column_idx] <= 21'sd0;
                    sparse_delta_r[row_idx][column_idx] <= 9'sd0;
                end
            end
            for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                column_sum_clear_r[column_idx] <= 1'b0;
                column_sum[column_idx] <= 32'd0;
                column_increment_r[column_idx] <= 16'd0;
                for (int pair_idx = 0; pair_idx < 4; pair_idx++) begin
                    dot8_column_pair_r[column_idx][pair_idx] <= 9'd0;
                end
                dot8_column_half_r[column_idx][0] <= 10'd0;
                dot8_column_half_r[column_idx][1] <= 10'd0;
            end
        end else begin
            if (result_fifo_push) begin
                result_fifo_payload_r[result_fifo_write_pointer_r]
                    <= {p5_column_r, p5_row_r, p5_result_r};
                result_fifo_write_pointer_r <= result_fifo_write_pointer_r + 1'b1;
            end
            if (result_fifo_pop) begin
                result_stage_payload_r
                    <= result_fifo_payload_r[result_fifo_read_pointer_r];
                result_stage_last_r
                    <= result_fifo_read_pointer_r == result_fifo_last_address_r;
                result_stage_valid_r <= 1'b1;
                result_fifo_read_pointer_r <= result_fifo_read_pointer_r + 1'b1;
            end else if (result_stage_pop) begin
                result_stage_valid_r <= 1'b0;
            end
            unique case ({result_fifo_push, result_fifo_pop})
```

```
562                 2'b10: begin
563                     result_fifo_count_r <= result_fifo_count_r + 1'b1;
564                     result_fifo_has_space_r
565                         <= result_fifo_count_r
566                             < RESULT_FIFO_COUNT_BITS'(RESULT_FIFO_DEPTH - 1);
567                     result_fifo_nonempty_r <= 1'b1;
568                 end
569                 2'b01: begin
570                     result_fifo_count_r <= result_fifo_count_r - 1'b1;
571                     result_fifo_has_space_r <= 1'b1;
572                     result_fifo_nonempty_r <= result_fifo_count_r != 1;
573                 end
574                 default: begin end
575             endcase
576
577             // Unconditional products let Vivado absorb these registers into
578             // the DSP48E1 PREG stage. The FSM presents operands one cycle
579             // before ST_RESTORE_ADD consumes the registered products.
580             zero_restore_low_product_r <=
581                 zero_restore_reduction_operand_r[15:0] * zero_restore_zp_operand_r;
582             zero_restore_high_product_r <=
583                 zero_restore_reduction_operand_r[31:16] * zero_restore_zp_operand_r;
584             // This exact unsigned 8x8 product maps to one DSP48E1 PREG. It
585             // replaces four LUT multipliers plus a reconstruction carry chain.
586             zero_point_product_r <= activation_zp_r * weight_zp_r;
587             // Dedicated copies have one DSP load each, allowing Vivado to use
588             // the four BREG input stages instead of routing shared zero points.
589             p0_row_zp_low_r <= weight_zp_r;
590             p0_row_zp_high_r <= weight_zp_r;
591             p0_column_zp_low_r <= activation_zp_r;
592             p0_column_zp_high_r <= activation_zp_r;
593
594             // The corresponding valid bit is the architectural qualifier, so
595             // these pre-read registers may sample continuously. Free-running
596             // them removes a 256-register launch_remaining clock-enable tree;
597             // samples outside ST_OUTPUT_PIPELINE are never consumed.
598             for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
599                 pread_accumulator_by_row_r[row_idx]
600                     <= accumulator[row_idx][launch_column_r[3:0]];
601             end
602             pread_row_sum_r <= row_sum[launch_row_r[2:0]];
603             pread_column_sum_r <= column_sum[launch_column_r[3:0]];
604             pread_row_r <= launch_row_r;
605             pread_column_r <= launch_column_r;
606
607             dense_issue_enable_r <= 1'b0;
608             dense_outer_tag0_r <= dense_issue_enable_r
609                 && mode_r == MODE_DENSE_OUTER;
610             dense_outer_tag1_r <= dense_outer_tag0_r;
611             dense_outer_tag2_r <= dense_outer_tag1_r;
612             sparse_prepare_valid_r <= 1'b0;
613             sparse_delta_valid_r <= 1'b0;
614             accumulate_delta_valid_r <= 1'b0;
615             metric_increment_valid_r <= 1'b0;
616             sparse_metric_valid_r <= 1'b0;
617             if (metric_increment_valid_r) begin
618                 if (mode_r == MODE_DENSE_OUTER || mode_r == MODE_DENSE_DOT8) begin
619                     debug_dense_macs_r <= debug_dense_macs_r
620                         + {11'd0, metric_increment_r};
621                 end else begin
622                     debug_spike_adds_r <= debug_spike_adds_r
623                         + {11'd0, metric_increment_r};
624                 end
625             end
626             if (sparse_metric_valid_r) begin
627                 metric_increment_r <= sparse_active_count_r * columns_r;
628                 metric_increment_valid_r <= 1'b1;
629             end
630             for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
631                 accumulator_clear_row_r[row_idx] <= 1'b0;
632             end
633             for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
634                 column_sum_clear_r[column_idx] <= 1'b0;
635             end
636             classify_valid_r <= issue_match_valid_r;
637             issue_match_valid_r <= issue_accept;
638             // The queue head is stable whenever it is not accepted, so capture
639             // its payload unconditionally and use issue_match_valid_r as the
640             // only validity qualifier. This removes issue_accept from the CE
641             // pins of 1,216 payload registers without changing pipeline latency.
642             issue_zero_matches_r <= issue_zero_matches_comb;
643             issue_match_last_r <= issue_last;
644             issue_match_activation_vector_r <= activation_vector;
645             issue_match_outer_weight_vector_r <= outer_weight_vector;
646             issue_match_dot_weight_vector_r <= dot_weight_vector;
647             if (issue_match_valid_r) begin
648                 classify_is_zero_r <= &issue_zero_matches_r;
```

```
                classify_last_r <= issue_match_last_r;
                classify_activation_vector_r <= issue_match_activation_vector_r;
                classify_outer_weight_vector_r <= issue_match_outer_weight_vector_r;
                classify_dot_weight_vector_r <= issue_match_dot_weight_vector_r;
            end
            dot8_pair_valid_r <= 1'b0;
            dot8_increment_half_valid_r <= 1'b0;
            dot_pair_valid_r <= dense_product_valid && mode_r == MODE_DENSE_DOT8;
            dot_half_valid_r <= dot_pair_valid_r;
            dot_sum_valid_r <= dot_half_valid_r;
            sum_update_valid_r <= 1'b0;
            for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                if (accumulator_clear_row_r[row_idx]) begin
                    row_sum[row_idx] <= 32'd0;
                end else if (sum_update_valid_r) begin
                    row_sum[row_idx] <= row_sum[row_idx]
                        + {16'd0, row_increment_r[row_idx]};
                end
            end
            for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                if (column_sum_clear_r[column_idx]) begin
                    column_sum[column_idx] <= 32'd0;
                end else if (sum_update_valid_r) begin
                    column_sum[column_idx] <= column_sum[column_idx]
                        + {16'd0, column_increment_r[column_idx]};
                end
            end
            if (dot8_pair_valid_r) begin
                dot8_increment_half_valid_r <= 1'b1;
                dot8_row_half_r[0] <= dot8_row_pair_r[0] + dot8_row_pair_r[1];
                dot8_row_half_r[1] <= dot8_row_pair_r[2] + dot8_row_pair_r[3];
                for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                    dot8_column_half_r[column_idx][0] <=
                        dot8_column_pair_r[column_idx][0]
                        + dot8_column_pair_r[column_idx][1];
                    dot8_column_half_r[column_idx][1] <=
                        dot8_column_pair_r[column_idx][2]
                        + dot8_column_pair_r[column_idx][3];
                end
            end
            if (dot8_increment_half_valid_r) begin
                row_increment_r[0] <= {6'd0, dot8_row_half_r[0]}
                    + {6'd0, dot8_row_half_r[1]};
                for (int row_idx = 1; row_idx < ROW_LANES; row_idx++) begin
                    row_increment_r[row_idx] <= 16'd0;
                end
                for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                    column_increment_r[column_idx]
                        <= {6'd0, dot8_column_half_r[column_idx][0]}
                            + {6'd0, dot8_column_half_r[column_idx][1]};
                end
                sum_update_valid_r <= 1'b1;
            end
            if (dense_product_valid && mode_r == MODE_DENSE_DOT8) begin
                for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin
                    for (int reduce_pair = 0; reduce_pair < 4; reduce_pair++) begin
                        dot_pair_lo_r[pair_idx][reduce_pair] <=
                            mac_product_lo[2*reduce_pair][pair_idx]
                            + mac_product_lo[2*reduce_pair+1][pair_idx];
                        dot_pair_hi_r[pair_idx][reduce_pair] <=
                            mac_product_hi[2*reduce_pair][pair_idx]
                            + mac_product_hi[2*reduce_pair+1][pair_idx];
                    end
                end
            end
            if (dot_pair_valid_r) begin
                for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin
                    dot_half_lo_r[pair_idx][0] <= dot_pair_lo_r[pair_idx][0]
                        + dot_pair_lo_r[pair_idx][1];
                    dot_half_lo_r[pair_idx][1] <= dot_pair_lo_r[pair_idx][2]
                        + dot_pair_lo_r[pair_idx][3];
                    dot_half_hi_r[pair_idx][0] <= dot_pair_hi_r[pair_idx][0]
                        + dot_pair_hi_r[pair_idx][1];
                    dot_half_hi_r[pair_idx][1] <= dot_pair_hi_r[pair_idx][2]
                        + dot_pair_hi_r[pair_idx][3];
                end
            end
            if (dot_half_valid_r) begin
                for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin
                    dot_sum_lo_r[pair_idx] <= {2'd0, dot_half_lo_r[pair_idx][0]}
                        + {2'd0, dot_half_lo_r[pair_idx][1]};
                    dot_sum_hi_r[pair_idx] <= {2'd0, dot_half_hi_r[pair_idx][0]}
                        + {2'd0, dot_half_hi_r[pair_idx][1]};
                end
            end

            if (sparse_prepare_valid_r) begin
```

```
                sparse_delta_valid_r <= 1'b1;
                sparse_active_count_r <= popcount8(sparse_prepare_active_r);
                sparse_metric_valid_r <= 1'b1;
                for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                    for (int column_idx = 0; column_idx < COLUMN_LANES;
                            column_idx++) begin
                        logic signed [7:0] signed_weight;
                        logic signed [8:0] extended_weight;
                        signed_weight = $signed(sparse_prepare_weight_vector_r[
                            column_idx*8 +: 8]);
                        extended_weight = {signed_weight[7], signed_weight};
                        if (sparse_prepare_active_r[row_idx]) begin
                            sparse_delta_r[row_idx][column_idx] <=
                                sparse_prepare_negative_r[row_idx]
                                ? -extended_weight : extended_weight;
                        end else begin
                            sparse_delta_r[row_idx][column_idx] <= 9'sd0;
                        end
                    end
                end
            end

            if (dense_accumulate_valid) begin
                mac_retire_count <= mac_retire_count + 1'b1;
                debug_retired_issues <= debug_retired_issues + 1'b1;
            end
            for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                    if (accumulator_clear_row_r[row_idx]) begin
                        accumulator[row_idx][column_idx] <= '0;
                    end else if (accumulate_delta_valid_r) begin
                        accumulator[row_idx][column_idx] <=
                            accumulator[row_idx][column_idx]
                            + {{(ACC_BITS-21){accumulate_delta_r[row_idx][column_idx][20]}},
                               accumulate_delta_r[row_idx][column_idx]};
                    end
                end
            end
            if (dense_outer_tag2_r) begin
                accumulate_delta_valid_r <= 1'b1;
                for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                    for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin
                        accumulate_delta_r[row_idx][2*pair_idx] <=
                            $signed({5'd0, mac_product_lo[row_idx][pair_idx]});
                        accumulate_delta_r[row_idx][2*pair_idx+1] <=
                            $signed({5'd0, mac_product_hi[row_idx][pair_idx]});
                    end
                end
            end else if (dot_sum_valid_r) begin
                accumulate_delta_valid_r <= 1'b1;
                for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                    for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin
                        accumulate_delta_r[row_idx][2*pair_idx] <=
                            row_idx == 0
                            ? $signed({1'b0, dot_sum_lo_r[pair_idx]}) : 21'sd0;
                        accumulate_delta_r[row_idx][2*pair_idx+1] <=
                            row_idx == 0
                            ? $signed({1'b0, dot_sum_hi_r[pair_idx]}) : 21'sd0;
                    end
                end
            end else if (sparse_delta_valid_r) begin
                accumulate_delta_valid_r <= 1'b1;
                for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                    for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                        accumulate_delta_r[row_idx][column_idx] <=
                            {{12{sparse_delta_r[row_idx][column_idx][8]}},
                              sparse_delta_r[row_idx][column_idx]};
                    end
                end
            end

            unique case (state)
                ST_IDLE: begin
                    if (start) begin
                        mode_r <= compute_mode;
                        rows_r <= active_rows;
                        for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                            active_row_mask_r[row_idx] <= row_idx < active_rows;
                        end
                        columns_r <= active_columns;
                        dense_macs_per_issue_r <= compute_mode == MODE_DENSE_DOT8
                            ? {2'd0, active_columns}
                                * 8'd8
                            : {4'd0, active_rows}
                                * {2'd0, active_columns};
                        activation_zp_r <= activation_zero_point;
                        weight_zp_r <= weight_zero_point;
```

```
                    config_error <= active_rows == 0
                        || active_rows > 4'(ROW_LANES)
                        || active_columns == 0
                        || active_columns > 6'(COLUMN_LANES)
                        || (compute_mode == MODE_DENSE_DOT8 && active_rows != 1);
                    effective_reduction <= 32'd0;
                    mac_issue_count <= 12'd0;
                    mac_retire_count <= 12'd0;
                    target_mac_retires <= 12'd0;
                    drain_complete_r <= 1'b0;
                    zero_restore_reduction_operand_r <= 32'd0;
                    zero_restore_zp_operand_r <= 16'd0;
                    zero_point_restore_r <= 48'd0;
                    launch_total_r <= 8'd0;
                    launch_remaining_r <= 8'd0;
                    launch_rows_remaining_r <= 4'd0;
                    launch_columns_remaining_r <= 6'd0;
                    launch_row_r <= 4'd0;
                    launch_column_r <= 6'd0;
                    pread_valid_r <= 1'b0;
                    p0_valid_r <= 1'b0;
                    p1_valid_r <= 1'b0;
                    p2_valid_r <= 1'b0;
                    p3_valid_r <= 1'b0;
                    p4_valid_r <= 1'b0;
                    p5_valid_r <= 1'b0;
                    result_fifo_count_r <= '0;
                    result_fifo_has_space_r <= 1'b1;
                    result_fifo_nonempty_r <= 1'b0;
                    result_fifo_write_pointer_r <= '0;
                    result_fifo_read_pointer_r <= '0;
                    result_fifo_last_address_r <= '0;
                    result_stage_valid_r <= 1'b0;
                    result_stage_payload_r <= '0;
                    result_stage_last_r <= 1'b0;
                    dot_sum_valid_r <= 1'b0;
                    dot_pair_valid_r <= 1'b0;
                    dot_half_valid_r <= 1'b0;
                    dot8_pair_valid_r <= 1'b0;
                    dot8_increment_half_valid_r <= 1'b0;
                    sum_update_valid_r <= 1'b0;
                    input_done_r <= 1'b0;
                    issue_match_valid_r <= 1'b0;
                    issue_zero_matches_r <= 8'd0;
                    issue_match_last_r <= 1'b0;
                    classify_valid_r <= 1'b0;
                    classify_is_zero_r <= 1'b0;
                    classify_last_r <= 1'b0;
                    dense_issue_enable_r <= 1'b0;
                    dense_outer_tag0_r <= 1'b0;
                    dense_outer_tag1_r <= 1'b0;
                    dense_outer_tag2_r <= 1'b0;
                    sparse_prepare_valid_r <= 1'b0;
                    sparse_delta_valid_r <= 1'b0;
                    accumulate_delta_valid_r <= 1'b0;
                    debug_accepted_issues <= 32'd0;
                    debug_skipped_issues <= 32'd0;
                    debug_retired_issues <= 32'd0;
                    debug_dense_macs_r <= 19'd0;
                    debug_spike_adds_r <= 19'd0;
                    metric_increment_r <= 8'd0;
                    metric_increment_valid_r <= 1'b0;
                    sparse_active_count_r <= 4'd0;
                    sparse_metric_valid_r <= 1'b0;
                    for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                        accumulator_clear_row_r[row_idx] <= 1'b1;
                    end
                    for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                        column_sum_clear_r[column_idx] <= 1'b1;
                    end
                    state <= ST_RUN;
                end
            end

            ST_RUN: begin
                if (classify_valid_r) begin
                    debug_accepted_issues <= debug_accepted_issues + 1'b1;
                    if (classify_is_zero_r) begin
                        debug_skipped_issues <= debug_skipped_issues + 1'b1;
                    end else if (mode_r == MODE_DENSE_OUTER) begin
                        dense_issue_enable_r <= 1'b1;
                        for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                            // Inactive rows are never emitted. Computing
                            // them removes active_rows from every DSP input
                            // mux and is therefore both safe and faster.
                            dense_activation_vector_r[row_idx*8 +: 8]
                                <= classify_activation_vector_r[row_idx*8 +: 8];
```

```
                    for (int pair_idx = 0; pair_idx < COLUMN_PAIRS;
                            pair_idx++) begin
                        // Inactive columns are never retired, so
                        // computing them is harmless. Loading all
                        // lanes removes active_columns from every
                        // DSP operand mux and shortens the routed
                        // data path without changing any result.
                        dense_weight_lo_r[row_idx][pair_idx]
                            <= classify_outer_weight_vector_r[
                                (2*pair_idx)*8 +: 8];
                        dense_weight_hi_r[row_idx][pair_idx]
                            <= classify_outer_weight_vector_r[
                                (2*pair_idx+1)*8 +: 8];
                    end
                end
                mac_issue_count <= mac_issue_count + 1'b1;
                metric_increment_r <= dense_macs_per_issue_r;
                metric_increment_valid_r <= 1'b1;
                sum_update_valid_r <= 1'b1;
                for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                    row_increment_r[row_idx]
                        <= {8'd0, classify_activation_vector_r[row_idx*8 +: 8]};
                end
                for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                    column_increment_r[column_idx]
                        <= {8'd0, classify_outer_weight_vector_r[column_idx*8 +: 8]};
                end
            end else if (mode_r == MODE_DENSE_DOT8) begin
                dense_issue_enable_r <= 1'b1;
                dense_activation_vector_r <= classify_activation_vector_r;
                for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                    for (int pair_idx = 0; pair_idx < COLUMN_PAIRS;
                            pair_idx++) begin
                        dense_weight_lo_r[row_idx][pair_idx]
                            <= classify_dot_weight_vector_r[
                                ((2*pair_idx)*ROW_LANES+row_idx)*8 +: 8];
                        dense_weight_hi_r[row_idx][pair_idx]
                            <= classify_dot_weight_vector_r[
                                ((2*pair_idx+1)*ROW_LANES+row_idx)*8 +: 8];
                    end
                end
                mac_issue_count <= mac_issue_count + 1'b1;
                metric_increment_r <= dense_macs_per_issue_r;
                metric_increment_valid_r <= 1'b1;
                dot8_pair_valid_r <= 1'b1;
                for (int pair_idx = 0; pair_idx < 4; pair_idx++) begin
                    dot8_row_pair_r[pair_idx] <=
                        classify_activation_vector_r[(2*pair_idx)*8 +: 8]
                        + classify_activation_vector_r[(2*pair_idx+1)*8 +: 8];
                end
                for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin
                    for (int pair_idx = 0; pair_idx < 4; pair_idx++) begin
                        dot8_column_pair_r[column_idx][pair_idx] <=
                            classify_dot_weight_vector_r[
                                (column_idx*8+2*pair_idx)*8 +: 8]
                            + classify_dot_weight_vector_r[
                                (column_idx*8+2*pair_idx+1)*8 +: 8];
                    end
                end
            end else begin
                sparse_prepare_valid_r <= 1'b1;
                sparse_prepare_weight_vector_r
                    <= classify_outer_weight_vector_r;
                for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin
                    logic signed [7:0] spike_value;
                    spike_value = $signed(
                        classify_activation_vector_r[row_idx*8 +: 8]);
                    sparse_prepare_active_r[row_idx]
                        <= active_row_mask_r[row_idx] && spike_value != 0;
                    sparse_prepare_negative_r[row_idx]
                        <= mode_r == MODE_TERNARY_WEIGHTED
                            && spike_value < 0;
                end
                debug_retired_issues <= debug_retired_issues + 1'b1;
            end

            if (classify_last_r) begin
                launch_row_r <= 4'd0;
                launch_column_r <= 6'd0;
                pread_valid_r <= 1'b0;
                p0_valid_r <= 1'b0;
                p1_valid_r <= 1'b0;
                p2_valid_r <= 1'b0;
                p3_valid_r <= 1'b0;
                p4_valid_r <= 1'b0;
                p5_valid_r <= 1'b0;
                if (mode_r == MODE_DENSE_OUTER || mode_r == MODE_DENSE_DOT8) begin
```

```
                    state <= ST_CAPTURE_TARGET;
                end else begin
                    state <= ST_SPARSE_DRAIN;
                end
            end
        end

        if (issue_accept) begin
            if (issue_last) input_done_r <= 1'b1;
        end
    end

    ST_DRAIN: begin
        // End the 12-bit equality at one local register before it
        // controls the physically replicated state network.
        drain_complete_r <= mac_retire_count == target_mac_retires;
        if (drain_complete_r) begin
            state <= ST_PREPARE_ZERO_POINT;
        end
    end

    ST_CAPTURE_TARGET: begin
        target_mac_retires <= mac_issue_count;
        effective_reduction <= mode_r == MODE_DENSE_DOT8
            ? {17'd0, mac_issue_count, 3'd0}
            : {20'd0, mac_issue_count};
        drain_complete_r <= 1'b0;
        state <= ST_DRAIN;
    end

    ST_SPARSE_DRAIN: begin
        state <= ST_PREPARE_OUTPUT_COUNT;
    end

    ST_PREPARE_ZERO_POINT: begin
        state <= ST_LATCH_ZERO_POINT;
    end

    ST_LATCH_ZERO_POINT: begin
        state <= ST_COMBINE_ZERO_POINT;
    end

    ST_COMBINE_ZERO_POINT: begin
        zero_restore_reduction_operand_r <= effective_reduction;
        zero_restore_zp_operand_r <= zero_point_product_r;
        state <= ST_RESTORE_MULTIPLY;
    end

    ST_RESTORE_MULTIPLY: begin
        state <= ST_RESTORE_RELAY;
    end

    ST_RESTORE_RELAY: begin
        // Isolate the two multiplier PREGs from the reconstruction
        // adder. Vivado can absorb these registers into the adder's
        // DSP input stage, replacing one DSP-to-DSP combinational
        // timing path with two independently timed pipeline paths.
        zero_restore_low_relay_r <= zero_restore_low_product_r;
        zero_restore_high_relay_r <= zero_restore_high_product_r;
        state <= ST_RESTORE_ADD;
    end

    ST_RESTORE_ADD: begin
        zero_point_restore_r <= {16'd0, zero_restore_low_relay_r}
            + {zero_restore_high_relay_r, 16'd0};
        state <= ST_PREPARE_OUTPUT_COUNT;
    end

    ST_PREPARE_OUTPUT_COUNT: begin
        launch_total_r <= {2'd0, columns_r};
        launch_rows_remaining_r <= rows_r;
        launch_columns_remaining_r <= columns_r;
        state <= ST_ACCUMULATE_OUTPUT_COUNT;
    end

    ST_ACCUMULATE_OUTPUT_COUNT: begin
        if (launch_rows_remaining_r <= 4'd1) begin
            launch_remaining_r <= launch_total_r;
            // Output addresses start at zero. Truncation is exact
            // for the 128-result case: 128 mod 128 minus one is 127.
            result_fifo_last_address_r
                <= launch_total_r[RESULT_FIFO_ADDRESS_BITS-1:0] - 1'b1;
            state <= ST_OUTPUT_PIPELINE;
        end else begin
            launch_total_r <= launch_total_r
                + {2'd0, columns_r};
            launch_rows_remaining_r <= launch_rows_remaining_r - 1'b1;
```

```
                    end
                end

                ST_OUTPUT_PIPELINE: begin
                    if (output_pipeline_enable) begin
                        p5_valid_r <= p4_valid_r;
                        if (p4_valid_r) begin
                            p5_result_r <= 48'(p4_corrected_r
                                + $signed({16'd0, zero_point_restore_r}));
                            p5_row_r <= p4_row_r;
                            p5_column_r <= p4_column_r;
                        end

                        p4_valid_r <= p3_valid_r;
                        if (p3_valid_r) begin
                            p4_corrected_r <= p3_corrected_r
                                - p3_column_correction_r;
                            p4_row_r <= p3_row_r;
                            p4_column_r <= p3_column_r;
                        end

                        p3_valid_r <= p2_valid_r;
                        if (p2_valid_r) begin
                            p3_corrected_r <= p2_raw_r - p2_row_correction_r;
                            p3_column_correction_r <= p2_column_correction_r;
                            p3_row_r <= p2_row_r;
                            p3_column_r <= p2_column_r;
                        end

                        p2_valid_r <= p1_valid_r;
                        if (p1_valid_r) begin
                            p2_raw_r <= p1_raw_r;
                            p2_row_correction_r <= $signed({40'd0,
                                p1_row_product_low_r})
                                + $signed({24'd0, p1_row_product_high_r, 16'd0});
                            p2_column_correction_r <= $signed({40'd0,
                                p1_column_product_low_r})
                                + $signed({24'd0, p1_column_product_high_r, 16'd0});
                            p2_row_r <= p1_row_r;
                            p2_column_r <= p1_column_r;
                        end

                        p1_valid_r <= p0_valid_r;
                        if (p0_valid_r) begin
                            p1_raw_r <= p0_raw_r;
                            p1_row_product_low_r <= p0_row_product_low;
                            p1_row_product_high_r <= p0_row_product_high;
                            p1_column_product_low_r <= p0_column_product_low;
                            p1_column_product_high_r <= p0_column_product_high;
                            p1_row_r <= p0_row_r;
                            p1_column_r <= p0_column_r;
                        end

                        // Split the 128-entry accumulator read into a 16-column
                        // selection and an 8-row selection. This adds one fill
                        // cycle while preserving one result per clock.
                        p0_valid_r <= pread_valid_r;
                        if (pread_valid_r) begin
                            p0_raw_r <= {{(64-ACC_BITS){
                                pread_accumulator_by_row_r[pread_row_r[2:0]][ACC_BITS-1]}},
                                pread_accumulator_by_row_r[pread_row_r[2:0]]};
                            p0_row_sum_r <= pread_row_sum_r;
                            p0_column_sum_r <= pread_column_sum_r;
                            p0_row_r <= pread_row_r;
                            p0_column_r <= pread_column_r;
                        end

                        pread_valid_r <= launch_remaining_r != 0;
                        if (launch_remaining_r != 0) begin
                            launch_remaining_r <= launch_remaining_r - 1'b1;
                            launch_row_r <= launch_row_r
                                + (launch_columns_remaining_r == 6'd1);
                            if (launch_columns_remaining_r == 6'd1) begin
                                launch_columns_remaining_r <= columns_r;
                                launch_column_r <= 6'd0;
                            end else begin
                                launch_columns_remaining_r <=
                                    launch_columns_remaining_r - 1'b1;
                                launch_column_r <= launch_column_r + 1'b1;
                            end
                        end else begin
                            pread_valid_r <= 1'b0;
                        end
                    end
                    if (result_stage_pop && result_stage_last_r) state <= ST_IDLE;
                end

```

```
                default: state <= ST_IDLE;
            endcase
        end
    end

    initial begin
        if (ROW_LANES != 8) $fatal(1, "neuraedge_tensor_tile currently requires ROW_LANES=8");
        if (COLUMN_LANES != 16) $fatal(1, "neuraedge_tensor_tile currently requires COLUMN_LANES=16");
        if (COLUMN_LANES % 2 != 0) $fatal(1, "COLUMN_LANES must be even");
        if (ACC_BITS < 32 || ACC_BITS > 48) $fatal(1, "ACC_BITS must be in [32,48]");
    end
endmodule

`default_nettype wire
```

19.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_tensor_tile #(` This opens neuraedge_tensor_tile, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 5: `parameter int ROW_LANES = 8,` This statement continues through source line 40; the explanation uses the complete expression rather than only the first visual fragment. This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 6: `parameter int COLUMN_LANES = 16,` This is continuation line 2 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'parameter int COLUMN_LANES = 16,'. The important operands are parameter: the value named 'parameter' in this block; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 7: `parameter int ACC_BITS = 32` This is continuation line 3 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'parameter int ACC_BITS = 32'. The important operands are parameter: the value named 'parameter' in this block; ACC_BITS: the value named 'acc bits' in this block. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32

) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 8: `) (` This is continuation line 4 of the 36-line logical statement that begins at source line 5. Its exact fragment is ‘) (‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 9: `input wire clk,` This is continuation line 5 of the 36-line logical statement that begins at source line 5. Its exact fragment is ‘input wire clk,‘. The important operands are input: the value named ’input’ in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 10: `input wire rst_n,` This is continuation line 6 of the 36-line logical statement that begins at source line 5. Its exact fragment is ‘input wire rst_n,‘. The important operands are input: the value named ’input’ in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 12: `input wire start,` This is continuation line 8 of the 36-line logical statement that begins at source line 5. Its exact fragment is ‘input wire start,‘. The important operands are input: the value named ’input’ in this block; start: the value named ’start’ in this block. Read together, the complete operation is: This fixes ROW_LANES to

8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 13: `input wire [1:0] compute_mode,` This is continuation line 9 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire [1:0] compute_mode,'. The important operands are input: the value named 'input' in this block; compute_mode: the value named 'compute mode' in this block. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 14: `input wire [3:0] active_rows,` This is continuation line 10 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire [3:0] active_rows,'. The important operands are input: the value named 'input' in this block; active_rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 15: `input wire [5:0] active_columns,` This is continuation line 11 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire [5:0] active_columns,'. The important operands are input: the value named 'input' in this block; active_columns: one output-feature position processed in parallel. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 16: `input wire [7:0] activation_zero_point,` This is continuation line 12 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire [7:0] activation_zero_point,'. The important operands are input: the value named 'input' in this block; activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 17: `input wire [7:0] weight_zero_point,` This is continuation line 13 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire [7:0] weight_zero_point,'. The important operands are input: the value named 'input' in this block; weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 18: `output logic config_error,` This is continuation line 14 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic config_error,'. The important operands are output: the value named 'output' in this block; config_error: the value named 'config error' in this block. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 20: `input wire issue_valid,` This is continuation line 16 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire issue_valid,'. The important operands are input: the value named 'input' in this block; issue_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant

to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 21: `output logic issue_ready,` This is continuation line 17 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic issue_ready,'. The important operands are output: the value named 'output' in this block; issue_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 22: `input wire [ROW_LANES*8-1:0] activation_vector,` This is continuation line 18 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire [ROW_LANES*8-1:0] activation_vector,'. The important operands are input: the value named 'input' in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; activation_vector: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 23: `input wire [COLUMN_LANES*8-1:0] outer_weight_vector,` This is continuation line 19 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire [COLUMN_LANES*8-1:0] outer_weight_vector,'. The important operands are input: the value named 'input' in this block; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath; outer_weight_vector: a learned coefficient multiplied by an activation. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 24: `input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector,` This is continuation line 20 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector,'. The important operands are input: the value named 'input' in this block; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point,

input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 25: `input wire issue_last,` This is continuation line 21 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire issue_last,'. The important operands are input: the value named 'input' in this block; issue_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 27: `output logic busy,` This is continuation line 23 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic busy,'. The important operands are output: the value named 'output' in this block; busy: the value named 'busy' in this block. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 28: `output logic result_valid,` This is continuation line 24 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic result_valid,'. The important operands are output: the value named 'output' in this block; result_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 29: `input wire result_ready,` This is continuation line 25 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'input wire result_ready,'. The important operands are input: the value named 'input' in this block; result_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32

) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 30: `output logic signed [47:0] result_value,` This is continuation line 26 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic signed [47:0] result_value,'. The important operands are output: the value named 'output' in this block; result_value: the value named 'result value' in this block. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 31: `output logic [3:0] result_row,` This is continuation line 27 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic [3:0] result_row,'. The important operands are output: the value named 'output' in this block; result_row: one output-row position processed by the tensor tile. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 32: `output logic [5:0] result_column,` This is continuation line 28 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic [5:0] result_column,'. The important operands are output: the value named 'output' in this block; result_column: one output-feature position processed in parallel. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 33: `output logic result_last,` This is continuation line 29 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic result_last,'. The important operands are output: the value named 'output' in this block; result_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 35: `output logic [31:0] debug_accepted_issues,` This is continuation line 31 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] debug_accepted_issues,'. The important operands are output: the value named 'output' in this block; debug_accepted_issues: the value named 'debug accepted issues' in this block. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 36: `output logic [31:0] debug_skipped_issues,` This is continuation line 32 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] debug_skipped_issues,'. The important operands are output: the value named 'output' in this block; debug_skipped_issues: the value named 'debug skipped issues' in this block. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 37: `output logic [31:0] debug_retired_issues,` This is continuation line 33 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic [31:0] debug_retired_issues,'. The important operands are output: the value named 'output' in this block; debug_retired_issues: the value named 'debug retired issues' in this block. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic

[63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 38: `output logic [63:0] debug_dense_macs,` This is continuation line 34 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic [63:0] debug_dense_macs,'. The important operands are output: the value named 'output' in this block; debug_dense_macs: a count of ordinary activation-times-weight operations actually issued. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 39: `output logic [63:0] debug_spike_adds` This is continuation line 35 of the 36-line logical statement that begins at source line 5. Its exact fragment is 'output logic [63:0] debug_spike_adds'. The important operands are output: the value named 'output' in this block; debug_spike_adds: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 40: `);` This is continuation line 36 of the 36-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes ROW_LANES to 8, parameter int COLUMN_LANES = 16, parameter int ACC_BITS = 32) (input wire clk, input wire rst_n, input wire start, input wire [1:0] compute_mode, input wire [3:0] active_rows, input wire [5:0] active_columns, input wire [7:0] activation_zero_point, input wire [7:0] weight_zero_point, output logic config_error, input wire issue_valid, output logic issue_ready, input wire [ROW_LANES*8-1:0] activation_vector, input wire [COLUMN_LANES*8-1:0] outer_weight_vector, input wire [COLUMN_LANES*ROW_LANES*8-1:0] dot_weight_vector, input wire issue_last, output logic busy, output logic result_valid, input wire result_ready, output logic signed [47:0] result_value, output logic [3:0] result_row, output logic [5:0] result_column, output logic result_last, output logic [31:0] debug_accepted_issues, output logic [31:0] debug_skipped_issues, output logic [31:0] debug_retired_issues, output logic [63:0] debug_dense_macs, output logic [63:0] debug_spike_adds) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 41: `localparam logic [1:0] MODE_DENSE_OUTER = 2'd0;` This fixes MODE_DENSE_OUTER to 2'd0 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_DENSE_OUTER denotes the value named 'mode dense outer' in this block.

Line 42: `localparam logic [1:0] MODE_DENSE_DOT8 = 2'd1;` This fixes MODE_DENSE_DOT8 to 2'd1 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_DENSE_DOT8 denotes the value named 'mode dense dot8' in this block.

Line 43: `localparam logic [1:0] MODE_BINARY_WEIGHTED = 2'd2;` This fixes MODE_BINARY_WEIGHTED to 2'd2 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_BINARY_WEIGHTED denotes a learned coefficient multiplied by an activation.

Line 44: `localparam logic [1:0] MODE_TERNARY_WEIGHTED = 2'd3;` This fixes MODE_TERNARY_WEIGHTED to 2'd3 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MODE_TERNARY_WEIGHTED denotes a learned coefficient multiplied by an activation.

Line 45: `localparam int COLUMN_PAIRS = COLUMN_LANES / 2;` This fixes COLUMN_PAIRS to COLUMN_LANES / 2 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; COLUMN_PAIRS denotes one output-feature position processed in parallel.

Line 46: `localparam int RESULT_FIFO_DEPTH = ROW_LANES * COLUMN_LANES;` This fixes RESULT_FIFO_DEPTH to ROW_LANES * COLUMN_LANES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; RESULT_FIFO_DEPTH denotes the value named 'result fifo depth' in this block.

Line 47: `localparam int RESULT_FIFO_ADDRESS_BITS = $clog2(RESULT_FIFO_DEPTH);` This fixes RESULT_FIFO_ADDRE to $clog2(RESULT_FIFO_DEPTH) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; RESULT_FIFO_ADDRESS_BITS denotes the value named 'result fifo address bits' in this block.

Line 48: `localparam int RESULT_FIFO_COUNT_BITS = $clog2(RESULT_FIFO_DEPTH + 1);` This fixes RESULT_FIFO_COUN to $clog2(RESULT_FIFO_DEPTH + 1) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; RESULT_FIFO_COUNT_BITS denotes the value named 'result fifo count bits' in this block.

Line 50: `typedef enum logic [3:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 51: `ST_IDLE = 4'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 52: `ST_RUN = 4'd1,` This assigns a fixed binary encoding to state ST_RUN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 53: `ST_DRAIN = 4'd2,` This assigns a fixed binary encoding to state ST_DRAIN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 54: `ST_PREPARE_ZERO_POINT = 4'd3,` This assigns a fixed binary encoding to state ST_PREPARE_ZERO_POINT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 55: `ST_RESTORE_MULTIPLY = 4'd4,` This assigns a fixed binary encoding to state ST_RESTORE_MULTIPLY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 56: `ST_OUTPUT_PIPELINE = 4'd5,` This assigns a fixed binary encoding to state ST_OUTPUT_PIPELINE, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 57: `ST_LATCH_ZERO_POINT = 4'd6,` This assigns a fixed binary encoding to state ST_LATCH_ZERO_POINT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 58: `ST_RESTORE_ADD = 4'd7,` This assigns a fixed binary encoding to state ST_RESTORE_ADD, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 59: `ST_SPARSE_DRAIN = 4'd8,` This assigns a fixed binary encoding to state ST_SPARSE_DRAIN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 60: `ST_CAPTURE_TARGET = 4'd9,` This assigns a fixed binary encoding to state ST_CAPTURE_TARGET, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 61: `ST_COMBINE_ZERO_POINT = 4'd10,` This assigns a fixed binary encoding to state ST_COMBINE_ZERO_POINT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 62: `ST_PREPARE_OUTPUT_COUNT = 4'd11,` This assigns a fixed binary encoding to state ST_PREPARE_OUTPUT_COUN which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 63: `ST_ACCUMULATE_OUTPUT_COUNT = 4'd12,` This assigns a fixed binary encoding to state ST_ACCUMULATE_OUTPUT which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 64: `ST_RESTORE_RELAY = 4'd13` This statement continues through source line 65; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_RESTORE_RELAY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 65: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 64. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_RESTORE_RELAY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 67: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 68: `(* dont_touch = "yes" *) logic [1:0] mode_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; mode_r: the value named 'mode r' in this block.

Line 69: `(* dont_touch = "yes" *) logic [3:0] rows_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; rows_r: one output-row position processed by the tensor tile.

Line 70: `logic [ROW_LANES-1:0] active_row_mask_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_row_mask_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 71: `logic [5:0] columns_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, columns_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 72: `logic [7:0] activation_zp_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_zp_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 73: `logic [7:0] weight_zp_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_zp_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 75: `logic signed [ACC_BITS-1:0] accumulator [0:ROW_LANES-1][0:COLUMN_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulator is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 76: `// Keep one physical clear source per row. A command-start state bit driving` Design intent: Keep one physical clear source per row. A command-start state bit driving This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 77: `// every accumulator bit creates a long, high-fanout control path after` Design intent: every accumulator bit creates a long, high-fanout control path after This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 78: `// placement; these registered row-local pulses perform the same clear one` Design intent: placement; these registered row-local pulses perform the same clear one This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 79: `// cycle after start, before the issue pipeline can produce a delta.` Design intent: cycle after start, before the issue pipeline can produce a delta. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 80: `(* dont_touch = "yes", max_fanout = 96 *)` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; max_fanout: the value named 'max fanout' in this block.

Line 81: `logic accumulator_clear_row_r [0:ROW_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulator_clear_row_r is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 82: `// Column sums use the same delayed-clear scheme. One register per column` Design intent: Column sums use the same delayed-clear scheme. One register per column This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 83: `// keeps the command-start decode out of a 512-bit distributed CE network.` Design intent: keeps the command-start decode out of a 512-bit distributed CE network. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 84: `(* dont_touch = "yes", max_fanout = 48 *)` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; max_fanout: the value named 'max fanout' in this block.

Line 85: `logic column_sum_clear_r [0:COLUMN_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, column_sum_clear_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 86: `(* dont_touch = "yes" *) logic accumulate_delta_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; accumulate_delta_valid_r: the producer's statement that its current payload is meaningful.

Line 87: `logic signed [20:0] accumulate_delta_r [0:ROW_LANES-1][0:COLUMN_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulate_delta_r is the value named 'accumulate delta r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 88: `logic [31:0] row_sum [0:ROW_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, row_sum is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 89: `logic [31:0] column_sum [0:COLUMN_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, column_sum is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 90: `logic sum_update_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sum_update_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 91: `logic [15:0] row_increment_r [0:ROW_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, row_increment_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 92: `logic [15:0] column_increment_r [0:COLUMN_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, column_increment_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 93: `logic [31:0] effective_reduction;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, effective_reduction is the value named 'effective reduction' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 95: `logic [11:0] mac_issue_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mac_issue_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 96: `// The counter is updated in both run and command-boundary states. A local` Design intent: The counter is updated in both run and command-boundary states. A local This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 97: `// D-input hold avoids a long FSM-to-CE route across the MAC array.` Design intent: D-input hold avoids a long FSM-to-CE route across the MAC array. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 98: `(* extract_enable = "no" *) logic [11:0] mac_retire_count;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block; mac_retire_count: an integer count retained for control or measurable telemetry.

Line 99: `logic [11:0] target_mac_retires;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, target_mac_retires is the value named 'target mac retires' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 100: `// One command can execute at most 2048*8*16 = 262144 useful operations,` Design intent: One command can execute at most 2048*8*16 = 262144 useful operations, This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 101: `// so 19 bits represent the exact per-command telemetry range. The` Design intent: so 19 bits represent the exact per-command telemetry range. The This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 102: `// increment is registered before accumulation to keep popcount and lane` Design intent: increment is registered before accumulation to keep popcount and lane This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 103: `// sizing logic out of the counter carry chain at 200 MHz.` Design intent: sizing logic out of the counter carry chain at 200 MHz. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 104: `logic [18:0] debug_dense_macs_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, debug_dense_macs_r is a count of ordinary activation-times-weight operations actually issued. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 105: `logic [18:0] debug_spike_adds_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, debug_spike_adds_r is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 106: `logic [7:0] metric_increment_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, metric_increment_r is the value named 'metric increment r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 107: `logic metric_increment_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, metric_increment_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 108: `logic [3:0] sparse_active_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sparse_active_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 109: `logic sparse_metric_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sparse_metric_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 110: `logic [7:0] dense_macs_per_issue_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_macs_per_issue_r is a count of ordinary activation-times-weight operations actually issued. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 111: `(* dont_touch = "yes" *) logic drain_complete_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; drain_complete_r: the value named 'drain complete r' in this block.

Line 112: `(* use_dsp = "yes" *) logic [15:0] zero_point_product_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; zero_point_product_r: the integer code representing real numeric zero in affine quantization.

Line 113: `logic [31:0] zero_restore_reduction_operand_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_restore_reduction_operand_r is the value named 'zero restore reduction operand r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 114: `logic [15:0] zero_restore_zp_operand_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_restore_zp_operand_r is the value named 'zero restore zp operand r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 115: `(* use_dsp = "yes" *) logic [31:0] zero_restore_low_product_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; zero_restore_low_product_r: the value named 'zero restore low product r' in this block.

Line 116: `(* use_dsp = "yes" *) logic [31:0] zero_restore_high_product_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; zero_restore_high_product_r: the value named 'zero restore high product r' in this block.

Line 117: `logic [31:0] zero_restore_low_relay_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_restore_low_relay_r is the value named 'zero restore low relay r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 118: `logic [31:0] zero_restore_high_relay_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_restore_high_relay_r is the value named 'zero restore high relay r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 119: `logic [47:0] zero_point_restore_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_point_restore_r is the integer code representing real numeric zero in affine quantization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 120: `logic [7:0] launch_total_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, launch_total_r is the value named 'launch total r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 121: `(* extract_enable = "no" *) logic [7:0] launch_remaining_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block; launch_remaining_r: the value named 'launch remaining r' in this block.

Line 122: `logic [3:0] launch_rows_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, launch_rows_remaining_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 123: `(* extract_enable = "no" *) logic [5:0] launch_columns_remaining_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block; launch_columns_remaining_r: one output-feature position processed in parallel.

Line 124: `(* extract_enable = "no" *) logic [3:0] launch_row_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block; launch_row_r: one output-row position processed by the tensor tile.

Line 125: `(* extract_enable = "no" *) logic [5:0] launch_column_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block; launch_column_r: one output-feature position processed in parallel.

Line 126: `logic pread_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pread_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 127: `logic signed [ACC_BITS-1:0] pread_accumulator_by_row_r [0:ROW_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pread_accumulator_by_row_r is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 128: `logic [31:0] pread_row_sum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pread_row_sum_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 129: `logic [31:0] pread_column_sum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pread_column_sum_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 130: `logic [3:0] pread_row_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pread_row_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 131: `logic [5:0] pread_column_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pread_column_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 132: `(* dont_touch = "yes" *) logic p0_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; p0_valid_r: the producer's statement that its current payload is meaningful.

Line 133: `logic signed [63:0] p0_raw_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p0_raw_r is the value named 'p0 raw r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 134: `logic [31:0] p0_row_sum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p0_row_sum_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 135: `logic [31:0] p0_column_sum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p0_column_sum_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 136: `logic [7:0] p0_row_zp_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p0_row_zp_low_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 137: `logic [7:0] p0_row_zp_high_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p0_row_zp_high_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 138: `logic [7:0] p0_column_zp_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p0_column_zp_low_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 139: `logic [7:0] p0_column_zp_high_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p0_column_zp_high_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 140: `logic [3:0] p0_row_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p0_row_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 141: `logic [5:0] p0_column_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p0_column_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 142: `(* dont_touch = "yes" *) logic p1_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; p1_valid_r: the producer's statement that its current payload is meaningful.

Line 143: `logic signed [63:0] p1_raw_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p1_raw_r is the value named 'p1 raw r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 144: `logic [23:0] p1_row_product_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p1_row_product_low_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 145: `logic [23:0] p1_row_product_high_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p1_row_product_high_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 146: `logic [23:0] p1_column_product_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p1_column_product_low_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 147: `logic [23:0] p1_column_product_high_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p1_column_product_high_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 148: `logic [3:0] p1_row_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p1_row_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 149: `logic [5:0] p1_column_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p1_column_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 150: `(* dont_touch = "yes" *) logic p2_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; p2_valid_r: the producer's statement that its current payload is meaningful.

Line 151: `logic signed [63:0] p2_raw_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p2_raw_r is the value named 'p2 raw r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 152: `logic signed [63:0] p2_row_correction_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p2_row_correction_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 153: `logic signed [63:0] p2_column_correction_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p2_column_correction_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 154: `logic [3:0] p2_row_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p2_row_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 155: `logic [5:0] p2_column_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p2_column_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 156: `(* dont_touch = "yes" *) logic p3_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; p3_valid_r: the producer's statement that its current payload is meaningful.

Line 157: `logic signed [63:0] p3_corrected_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p3_corrected_r is the value named 'p3 corrected r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 158: `logic signed [63:0] p3_column_correction_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p3_column_correction_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 159: `logic [3:0] p3_row_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p3_row_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 160: `logic [5:0] p3_column_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p3_column_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 161: `(* dont_touch = "yes" *) logic p4_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; p4_valid_r: the producer's statement that its current payload is meaningful.

Line 162: `logic signed [63:0] p4_corrected_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p4_corrected_r is the value named 'p4 corrected r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 163: `logic [3:0] p4_row_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p4_row_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 164: `logic [5:0] p4_column_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p4_column_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 165: `(* dont_touch = "yes" *) logic p5_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; p5_valid_r: the producer's statement that its current payload is meaningful.

Line 166: `logic signed [47:0] p5_result_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p5_result_r is the value named 'p5 result r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 167: `logic [3:0] p5_row_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p5_row_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 168: `logic [5:0] p5_column_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, p5_column_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 169: `logic output_pipeline_enable;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_pipeline_enable is a control bit that permits the associated state or storage update. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 170: `logic [RESULT_FIFO_COUNT_BITS-1:0] result_fifo_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 171: `(* dont_touch = "yes" *) logic result_fifo_has_space_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; result_fifo_has_space_r: the value named 'result fifo has space r' in this block.

Line 172: `(* dont_touch = "yes" *) logic result_fifo_nonempty_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; result_fifo_nonempty_r: the value named 'result fifo nonempty r' in this block.

Line 173: `logic [RESULT_FIFO_ADDRESS_BITS-1:0] result_fifo_write_pointer_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_write_pointer_r is the value named 'result fifo write pointer r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 174: `logic [RESULT_FIFO_ADDRESS_BITS-1:0] result_fifo_read_pointer_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_read_pointer_r is the value named 'result fifo read pointer r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 175: `logic [RESULT_FIFO_ADDRESS_BITS-1:0] result_fifo_last_address_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_last_address_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 176: `// One command emits at most 8*16 records. Buffering that complete tile` Design intent: One command emits at most 8*16 records. Buffering that complete tile This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 177: `// removes downstream backpressure from every arithmetic pipeline CE while` Design intent: removes downstream backpressure from every arithmetic pipeline CE while This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 178: `// retaining an asynchronous read suitable for a stable AXI-Stream output.` Design intent: retaining an asynchronous read suitable for a stable AXI-Stream output. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 179: `(* ram_style = "distributed" *) logic [57:0] result_fifo_payload_r` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are ram_style: the value named 'ram style' in this block; distributed: the value named 'distributed' in this block; result_fifo_payload_r: stream data belonging to the current command.

Line 180: `[0:RESULT_FIFO_DEPTH-1];` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are RESULT_FIFO_DEPTH: the value named 'result fifo depth' in this block.

Line 181: `logic result_stage_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_stage_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 182: `logic [57:0] result_stage_payload_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_stage_payload_r is stream data belonging to the current command. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 183: `logic result_stage_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_stage_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 184: `wire result_fifo_push = state == ST_OUTPUT_PIPELINE` This statement continues through source line 185; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_push is the value named 'result fifo push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 185: `&& p5_valid_r && result_fifo_has_space_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 184. Its exact fragment is '&& p5_valid_r && result_fifo_has_space_r;'. The important operands are p5_valid_r: the producer's statement that its current payload is meaningful; result_fifo_has_space_r: the value named 'result fifo has space r' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_push is the value named 'result fifo push' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 186: `wire result_fifo_pop = state == ST_OUTPUT_PIPELINE` This statement continues through source line 189; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_pop is the value named 'result fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 187: `&& result_fifo_nonempty_r` This is continuation line 2 of the 4-line logical statement that begins at source line 186. Its exact fragment is '&& result_fifo_nonempty_r'. The important operands are result_fifo_nonempty_r: the value named 'result fifo nonempty r' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_pop is the value named 'result fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 188: `&& (!result_stage_valid_r` This is continuation line 3 of the 4-line logical statement that begins at source line 186. Its exact fragment is '&& (!result_stage_valid_r'. The important operands are result_stage_valid_r: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_pop is the value named 'result fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 189: `|| (result_stage_valid_r && result_ready));` This is continuation line 4 of the 4-line logical statement that begins at source line 186. Its exact fragment is '|| (result_stage_valid_r && result_ready));'. The important operands are result_stage_valid_r: the producer's statement that its current payload is meaningful; result_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_fifo_pop is the value named 'result fifo pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 190: `wire result_stage_pop = state == ST_OUTPUT_PIPELINE` This statement continues through source line 191; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_stage_pop is the value named 'result stage pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 191: `&& result_stage_valid_r && result_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 190. Its exact fragment is '&& result_stage_valid_r && result_ready;'. The important operands are result_stage_valid_r: the producer's statement that its current payload is meaningful; result_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_stage_pop is the value named 'result stage pop' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 193: `// Zero-point restoration is computed once per tile. Row and column products` Design intent: Zero-point restoration is computed once per tile. Row and column products This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 194: `// are independent and therefore form a throughput-one correction pipeline.` Design intent: are independent and therefore form a throughput-one correction pipeline. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 195: `(* use_dsp = "yes" *) wire [23:0] p0_row_product_low =` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; p0_row_product_low: one output-row position processed by the tensor tile.

Line 196: `p0_row_sum_r[15:0] * p0_row_zp_low_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are p0_row_sum_r: one output-row position processed by the tensor tile; p0_row_zp_low_r: one output-row position processed by the tensor tile.

Line 197: `(* use_dsp = "yes" *) wire [23:0] p0_row_product_high =` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; p0_row_product_high: one output-row position processed by the tensor tile.

Line 198: `p0_row_sum_r[31:16] * p0_row_zp_high_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are p0_row_sum_r: one output-row position processed by the tensor tile; p0_row_zp_high_r: one output-row position processed by the tensor tile.

Line 199: `(* use_dsp = "yes" *) wire [23:0] p0_column_product_low =` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; p0_column_product_low: one output-feature position processed in parallel.

Line 200: `p0_column_sum_r[15:0] * p0_column_zp_low_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are p0_column_sum_r: one output-feature position processed in parallel; p0_column_zp_low_r: one output-feature position processed in parallel.

Line 201: `(* use_dsp = "yes" *) wire [23:0] p0_column_product_high =` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; p0_column_product_high: one output-feature position processed in parallel.

Line 202: `p0_column_sum_r[31:16] * p0_column_zp_high_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are p0_column_sum_r: one output-feature position processed in parallel; p0_column_zp_high_r: one output-feature position processed in parallel.

Line 204: `logic [7:0] mac_activation [0:ROW_LANES-1][0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mac_activation is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 205: `logic [7:0] mac_weight_lo [0:ROW_LANES-1][0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mac_weight_lo is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 206: `logic [7:0] mac_weight_hi [0:ROW_LANES-1][0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mac_weight_hi is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 207: `wire [15:0] mac_product_lo [0:ROW_LANES-1][0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mac_product_lo is the value named 'mac product lo' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 208: `wire [15:0] mac_product_hi [0:ROW_LANES-1][0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mac_product_hi is the value named 'mac product hi' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 209: `wire mac_product_valid [0:ROW_LANES-1][0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, mac_product_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 210: `logic dot_pair_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_pair_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 211: `logic dot_half_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_half_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 212: `logic dot_sum_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_sum_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 213: `logic [16:0] dot_pair_lo_r [0:COLUMN_PAIRS-1][0:3];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_pair_lo_r is the value named 'dot pair lo r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 214: `logic [16:0] dot_pair_hi_r [0:COLUMN_PAIRS-1][0:3];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_pair_hi_r is the value named 'dot pair hi r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 215: `logic [17:0] dot_half_lo_r [0:COLUMN_PAIRS-1][0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_half_lo_r is the value named 'dot half lo r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 216: `logic [17:0] dot_half_hi_r [0:COLUMN_PAIRS-1][0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_half_hi_r is the value named 'dot half hi r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 217: `logic [19:0] dot_sum_lo_r [0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_sum_lo_r is the value named 'dot sum lo r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 218: `logic [19:0] dot_sum_hi_r [0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_sum_hi_r is the value named 'dot sum hi r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 219: `logic dot8_pair_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot8_pair_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 220: `logic [8:0] dot8_row_pair_r [0:3];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot8_row_pair_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 221: `// As with the retire counter, keep the classify-state hold on the data` Design intent: As with the retire counter, keep the classify-state hold on the data This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 222: `// input so each pair register can be implemented near its adder.` Design intent: input so each pair register can be implemented near its adder. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 223: `(* extract_enable = "no" *)` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block.

Line 224: `logic [8:0] dot8_column_pair_r [0:COLUMN_LANES-1][0:3];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot8_column_pair_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 225: `logic dot8_increment_half_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot8_increment_half_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 226: `logic [9:0] dot8_row_half_r [0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot8_row_half_r is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 227: `logic [9:0] dot8_column_half_r [0:COLUMN_LANES-1][0:1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot8_column_half_r is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 229: `logic issue_accept;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_accept is the value named 'issue accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 230: `logic input_done_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_done_r is the value named 'input done r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 231: `logic issue_match_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_match_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 232: `logic [7:0] issue_zero_matches_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_zero_matches_r is the value named 'issue zero matches r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 233: `logic issue_match_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_match_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 234: `logic [ROW_LANES*8-1:0] issue_match_activation_vector_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_match_activation_vector_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 235: `logic [COLUMN_LANES*8-1:0] issue_match_outer_weight_vector_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_match_outer_weight_vector_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 236: `logic [COLUMN_LANES*ROW_LANES*8-1:0] issue_match_dot_weight_vector_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_match_dot_weight_vector_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 237: `logic [7:0] issue_zero_matches_comb;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_zero_matches_comb is the value named 'issue zero matches comb' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 239: `function automatic logic [3:0] popcount8(input logic [7:0] value);` Within the combinational helper popcount8, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; popcount8: the value named 'popcount8' in this block.

Line 240: `logic [1:0] pair0;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, pair0 is the value named 'pair0' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 241: `logic [1:0] pair1;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, pair1 is the value named 'pair1' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 242: `logic [1:0] pair2;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, pair2 is the value named 'pair2' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 243: `logic [1:0] pair3;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, pair3 is the value named 'pair3' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 244: `logic [2:0] half0;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, half0 is the value named 'half0' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 245: `logic [2:0] half1;` This declares hardware signal storage or wiring in the combinational helper popcount8. Specifically, half1 is the value named 'half1' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 246: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper popcount8; it has no arithmetic value of its own.

Line 247: `pair0 = value[0] + value[1];` This makes pair0, the value named 'pair0' in this block, equal to 'value[0] + value[1]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount8.

Line 248: `pair1 = value[2] + value[3];` This makes pair1, the value named 'pair1' in this block, equal to 'value[2] + value[3]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount8.

Line 249: `pair2 = value[4] + value[5];` This makes pair2, the value named 'pair2' in this block, equal to 'value[4] + value[5]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount8.

Line 250: `pair3 = value[6] + value[7];` This makes pair3, the value named 'pair3' in this block, equal to 'value[6] + value[7]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper popcount8.

Line 251: `half0 = pair0 + pair1;` This makes half0, the value named 'half0' in this block, equal to 'pair0 + pair1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pair0: the value named 'pair0' in this block; pair1: the value named 'pair1' in this block. Context: the combinational helper popcount8.

Line 252: `half1 = pair2 + pair3;` This makes half1, the value named 'half1' in this block, equal to 'pair2 + pair3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pair2: the value named 'pair2' in this block; pair3: the value named 'pair3' in this block. Context: the combinational helper popcount8.

Line 253: `popcount8 = half0 + half1;` This makes popcount8, the value named 'popcount8' in this block, equal to 'half0 + half1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are half0: the value named 'half0' in this block; half1: the value named 'half1' in this block. Context: the combinational helper popcount8.

Line 254: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper popcount8; it has no arithmetic value of its own.

Line 255: `endfunction` Within the combinational helper popcount8, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are endfunction: the value named 'endfunction' in this block.

Line 256: `(* dont_touch = "yes" *) logic classify_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; classify_valid_r: the producer's statement that its current payload is meaningful.

Line 257: `(* dont_touch = "yes" *) logic classify_is_zero_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; classify_is_zero_r: the value named 'classify is zero r' in this block.

Line 258: `logic classify_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, classify_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 259: `logic [ROW_LANES*8-1:0] classify_activation_vector_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, classify_activation_vector_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 260: `logic [COLUMN_LANES*8-1:0] classify_outer_weight_vector_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, classify_outer_weight_vector_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 261: `logic [COLUMN_LANES*ROW_LANES*8-1:0] classify_dot_weight_vector_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, classify_dot_weight_vector_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 262: `logic dense_issue_enable;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_issue_enable is a control bit that permits the associated state or storage update. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 263: `(* dont_touch = "yes" *) logic dense_issue_enable_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; dense_issue_enable_r: a control bit that permits the associated state or storage update.

Line 264: `logic [ROW_LANES*8-1:0] dense_activation_vector_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_activation_vector_r is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 265: `logic [7:0] dense_weight_lo_r [0:ROW_LANES-1][0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_weight_lo_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 266: `logic [7:0] dense_weight_hi_r [0:ROW_LANES-1][0:COLUMN_PAIRS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_weight_hi_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 267: `logic dense_outer_tag0_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_outer_tag0_r is the value named 'dense outer tag0 r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 268: `logic dense_outer_tag1_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_outer_tag1_r is the value named 'dense outer tag1 r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 269: `logic dense_outer_tag2_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_outer_tag2_r is the value named 'dense outer tag2 r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 270: `logic sparse_prepare_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sparse_prepare_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 271: `logic [ROW_LANES-1:0] sparse_prepare_active_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sparse_prepare_active_r is the value named 'sparse prepare active r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 272: `logic [ROW_LANES-1:0] sparse_prepare_negative_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sparse_prepare_negative_r is the value named 'sparse prepare negative r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 273: `logic [COLUMN_LANES*8-1:0] sparse_prepare_weight_vector_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sparse_prepare_weight_vector_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 274: `(* dont_touch = "yes" *) logic sparse_delta_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; sparse_delta_valid_r: the producer's statement that its current payload is meaningful.

Line 275: `logic signed [8:0] sparse_delta_r [0:ROW_LANES-1][0:COLUMN_LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sparse_delta_r is the value named 'sparse delta r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 276: `wire dense_product_valid = mac_product_valid[0][0];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_product_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 277: `wire dense_accumulate_valid = dense_outer_tag2_r || dot_sum_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dense_accumulate_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 279: `function automatic logic [15:0] sum_u8x8(input logic [63:0] values);` Within the combinational helper sum u8x8, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; sum_u8x8: the value named 'sum u8x8' in this block.

Line 280: `logic [8:0] pair0;` This declares hardware signal storage or wiring in the combinational helper sum u8x8. Specifically, pair0 is the value named 'pair0' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 281: `logic [8:0] pair1;` This declares hardware signal storage or wiring in the combinational helper sum u8x8. Specifically, pair1 is the value named 'pair1' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 282: `logic [8:0] pair2;` This declares hardware signal storage or wiring in the combinational helper sum u8x8. Specifically, pair2 is the value named 'pair2' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 283: `logic [8:0] pair3;` This declares hardware signal storage or wiring in the combinational helper sum u8x8. Specifically, pair3 is the value named 'pair3' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 284: `logic [9:0] half0;` This declares hardware signal storage or wiring in the combinational helper sum u8x8. Specifically, half0 is the value named 'half0' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 285: `logic [9:0] half1;` This declares hardware signal storage or wiring in the combinational helper sum u8x8. Specifically, half1 is the value named 'half1' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 286: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper sum u8x8; it has no arithmetic value of its own.

Line 287: `pair0 = values[7:0] + values[15:8];` This makes pair0, the value named 'pair0' in this block, equal to 'values[7:0] + values[15:8]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are values: the value named 'values' in this block. Context: the combinational helper sum u8x8.

Line 288: `pair1 = values[23:16] + values[31:24];` This makes pair1, the value named 'pair1' in this block, equal to 'values[23:16] + values[31:24]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are values: the value named 'values' in this block. Context: the combinational helper sum u8x8.

Line 289: `pair2 = values[39:32] + values[47:40];` This makes pair2, the value named 'pair2' in this block, equal to 'values[39:32] + values[47:40]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are values: the value named 'values' in this block. Context: the combinational helper sum u8x8.

Line 290: `pair3 = values[55:48] + values[63:56];` This makes pair3, the value named 'pair3' in this block, equal to 'values[55:48] + values[63:56]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are values: the value named 'values' in this block. Context: the combinational helper sum u8x8.

Line 291: `half0 = pair0 + pair1;` This makes half0, the value named 'half0' in this block, equal to 'pair0 + pair1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pair0: the value named 'pair0' in this block; pair1: the value named 'pair1' in this block. Context: the combinational helper sum u8x8.

Line 292: `half1 = pair2 + pair3;` This makes half1, the value named 'half1' in this block, equal to 'pair2 + pair3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pair2: the value named 'pair2' in this block; pair3: the value named 'pair3' in this block. Context: the combinational helper sum u8x8.

Line 293: `sum_u8x8 = {6'd0, half0} + {6'd0, half1};` This makes sum_u8x8, the value named 'sum u8x8' in this block, equal to '{6'd0, half0} + {6'd0, half1}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block; half0: the value named 'half0' in this block; half1: the value named 'half1' in this block. Context: the combinational helper sum u8x8.

Line 294: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper sum u8x8; it has no arithmetic value of its own.

Line 295: `endfunction` Within the combinational helper sum u8x8, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are endfunction: the value named 'endfunction' in this block.

Line 297: `function automatic logic [19:0] sum_u16x8(` Within the combinational helper sum u16x8, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; sum_u16x8: the value named 'sum u16x8' in this block.

Line 298: `input logic [15:0] value0,` This declares value0 as a input port with width [15:0]. It carries the value named 'value0' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 299: `input logic [15:0] value1,` This declares value1 as a input port with width [15:0]. It carries the value named 'value1' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 300: `input logic [15:0] value2,` This declares value2 as a input port with width [15:0]. It carries the value named 'value2' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 301: `input logic [15:0] value3,` This declares value3 as a input port with width [15:0]. It carries the value named 'value3' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 302: `input logic [15:0] value4,` This declares value4 as a input port with width [15:0]. It carries the value named 'value4' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 303: `input logic [15:0] value5,` This declares value5 as a input port with width [15:0]. It carries the value named 'value5' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 304: `input logic [15:0] value6,` This declares value6 as a input port with width [15:0]. It carries the value named 'value6' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 305: `input logic [15:0] value7` This declares value7 as a input port with width [15:0]. It carries the value named 'value7' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 306: `);` Within the combinational helper sum u16x8, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The expression is a literal or structural operator and has no hidden software state.

Line 307: `logic [16:0] pair0;` This declares hardware signal storage or wiring in the combinational helper sum u16x8. Specifically, pair0 is the value named 'pair0' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 308: `logic [16:0] pair1;` This declares hardware signal storage or wiring in the combinational helper sum u16x8. Specifically, pair1 is the value named 'pair1' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 309: `logic [16:0] pair2;` This declares hardware signal storage or wiring in the combinational helper sum u16x8. Specifically, pair2 is the value named 'pair2' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 310: `logic [16:0] pair3;` This declares hardware signal storage or wiring in the combinational helper sum u16x8. Specifically, pair3 is the value named 'pair3' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 311: `logic [17:0] half0;` This declares hardware signal storage or wiring in the combinational helper sum u16x8. Specifically, half0 is the value named 'half0' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 312: `logic [17:0] half1;` This declares hardware signal storage or wiring in the combinational helper sum u16x8. Specifically, half1 is the value named 'half1' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 313: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper sum u16x8; it has no arithmetic value of its own.

Line 314: `pair0 = value0 + value1;` This makes pair0, the value named 'pair0' in this block, equal to 'value0 + value1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value0: the value named 'value0' in this block; value1: the value named 'value1' in this block. Context: the combinational helper sum u16x8.

Line 315: `pair1 = value2 + value3;` This makes pair1, the value named 'pair1' in this block, equal to 'value2 + value3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value2: the value named 'value2' in this block; value3: the value named 'value3' in this block. Context: the combinational helper sum u16x8.

Line 316: `pair2 = value4 + value5;` This makes pair2, the value named 'pair2' in this block, equal to 'value4 + value5'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value4: the value named 'value4' in this block; value5: the value named 'value5' in this block. Context: the combinational helper sum u16x8.

Line 317: `pair3 = value6 + value7;` This makes pair3, the value named 'pair3' in this block, equal to 'value6 + value7'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value6: the value named 'value6' in this block; value7: the value named 'value7' in this block. Context: the combinational helper sum u16x8.

Line 318: `half0 = pair0 + pair1;` This makes half0, the value named 'half0' in this block, equal to 'pair0 + pair1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pair0: the value named 'pair0' in this block; pair1: the value named 'pair1' in this block. Context: the combinational helper sum u16x8.

Line 319: `half1 = pair2 + pair3;` This makes half1, the value named 'half1' in this block, equal to 'pair2 + pair3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pair2: the value named 'pair2' in this block; pair3: the value named 'pair3' in this block. Context: the combinational helper sum u16x8.

Line 320: `sum_u16x8 = {2'd0, half0} + {2'd0, half1};` This makes sum_u16x8, the value named 'sum u16x8' in this block, equal to '{2'd0, half0} + {2'd0, half1}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block; half0: the value named 'half0' in this block; half1: the value named 'half1' in this block. Context: the combinational helper sum u16x8.

Line 321: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper sum u16x8; it has no arithmetic value of its own.

Line 322: `endfunction` Within the combinational helper sum u16x8, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are endfunction: the value named 'endfunction' in this block.

Line 324: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 325: `for (int lane = 0; lane < ROW_LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 326: `unique case (mode_r)` This starts a multiplexer/state decode selected by ‘mode_r‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 327: `MODE_DENSE_OUTER,` Within the combinational process whose outputs respond without waiting for a clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are MODE_DENSE_OUTER: the value named ’mode dense outer’ in this block.

Line 328: `MODE_DENSE_DOT8: issue_zero_matches_comb[lane] =` This statement continues through source line 329; the explanation uses the complete expression rather than only the first visual fragment. This is the MODE_DENSE_DOT8 case branch, which performs this explicitly named phase. The same-cycle action begins with ‘issue_zero_matches_comb[lane] = activation_vector[lane*8 +: 8] == activation_zp_r;‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state MODE_DENSE_DOT8, the machine performs the named control phase..

Line 329: `activation_vector[lane*8 +: 8] == activation_zp_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 328. Its exact fragment is ‘activation_vector[lane*8 +: 8] == activation_zp_r;‘. The important operands are activation_vector: an input feature value consumed by a neuron or multiply-accumulate operation; lane: one parallel arithmetic element within a vector or tensor datapath; activation_zp_r: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This is the MODE_DENSE_DOT8 case branch, which performs this explicitly named phase. The same-cycle action begins with ‘issue_zero_matches_comb[lane] = activation_vector[lane*8 +: 8] == activation_zp_r;‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state MODE_DENSE_DOT8, the machine performs the named control phase..

Line 330: `MODE_BINARY_WEIGHTED,` Within the combinational process whose outputs respond without waiting for a clock edge Within state MODE_DENSE_DOT8, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are MODE_BINARY_WEIGHTED: a learned coefficient multiplied by an activation.

Line 331: `MODE_TERNARY_WEIGHTED: issue_zero_matches_comb[lane] =` This statement continues through source line 332; the explanation uses the complete expression rather than only the first visual fragment. This is the MODE_TERNARY_WE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘issue_zero_matches_comb[lane] = activation_vector[lane*8 +: 8] == 8’d0;‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state MODE_TERNARY_WEIGHTED, the machine performs the named control phase..

Line 332: `activation_vector[lane*8 +: 8] == 8’d0;` This is continuation line 2 of the 2-line logical statement that begins at source line 331. Its exact fragment is ‘activation_vector[lane*8 +: 8] == 8’d0;‘. The important operands are activation_vector: an input feature value consumed by a neuron or multiply-accumulate operation; lane: one parallel arithmetic element within a vector or tensor datapath; d0: the value named ’d0’ in this block. Read together, the complete operation is: This is the MODE_TERNARY_WEIGHTED case branch, which performs this explicitly named phase. The same-cycle action begins with ‘issue_zero_matches_comb[lane] = activation_vector[lane*8 +: 8] == 8’d0;‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state MODE_TERNARY_WEIGHTED, the machine performs the named control phase..

Line 333: `default: issue_zero_matches_comb[lane] = 1’b1;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘issue_zero_matches_comb[lane] = 1’b1;‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state MODE_TERNARY_WEIGHTED, the machine performs the named control phase..

Line 334: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state MODE_TERNARY_WEIGHTED, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 335: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 336: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 338: `assign issue_accept = issue_valid && issue_ready;` This makes issue_accept, the value named 'issue accept' in this block, equal to 'issue_valid && issue_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_valid: the producer's statement that its current payload is meaningful; issue_ready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 339: `assign dense_issue_enable = dense_issue_enable_r;` This makes dense_issue_enable, a control bit that permits the associated state or storage update, equal to 'dense_issue_enable_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are dense_issue_enable_r: a control bit that permits the associated state or storage update. Context: the module's structural declarations.

Line 341: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 342: `for (int lane = 0; lane < ROW_LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 343: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 344: `mac_activation[lane][pair_idx] =` This statement continues through source line 345; the explanation uses the complete expression rather than only the first visual fragment. This makes mac_activation[lane][pair_idx], an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'dense_activation_vector_r[lane*8 +: 8]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are dense_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 345: `dense_activation_vector_r[lane*8 +: 8];` This is continuation line 2 of the 2-line logical statement that begins at source line 344. Its exact fragment is 'dense_activation_vector_r[lane*8 +: 8];'. The important operands are dense_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes mac_activation[lane][pair_idx], an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'dense_activation_vector_r[lane*8 +: 8]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are dense_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 346: `mac_weight_lo[lane][pair_idx] = dense_weight_lo_r[lane][pair_idx];` This makes mac_weight_lo[lane][pair_ a learned coefficient multiplied by an activation, equal to 'dense_weight_lo_r[lane][pair_idx]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are dense_weight_lo_r: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; pair_idx: the value named 'pair idx' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 347: `mac_weight_hi[lane][pair_idx] = dense_weight_hi_r[lane][pair_idx];` This makes mac_weight_hi[lane][pair_ a learned coefficient multiplied by an activation, equal to 'dense_weight_hi_r[lane][pair_idx]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are dense_weight_hi_r: a learned coefficient

multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; pair_idx: the value named 'pair idx' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 348: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 349: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 350: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 352: `for (genvar lane = 0; lane < ROW_LANES; lane++) begin : g_mac_lane` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 353: `for (genvar pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin : g_mac_pair` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 354: `neuraedge_mac2_u8 u_mac2 (` This instantiates module neuraedge_mac2_u8 as u_mac2. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 355: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 356: `.rst_n(rst_n),` This connects the child instance's named port 'rst_n' to parent expression 'rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 357: `.enable(dense_issue_enable),` This connects the child instance's named port 'enable' to parent expression 'dense_issue_enable'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a control bit that permits the associated state or storage update.

Line 358: `.activation(mac_activation[lane][pair_idx]),` This connects the child instance's named port 'activation' to parent expression 'mac_activation[lane][pair_idx]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 359: `.weight_lo(mac_weight_lo[lane][pair_idx]),` This connects the child instance's named port 'weight_lo' to parent expression 'mac_weight_lo[lane][pair_idx]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 360: `.weight_hi(mac_weight_hi[lane][pair_idx]),` This connects the child instance's named port 'weight_hi' to parent expression 'mac_weight_hi[lane][pair_idx]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 361: `.product_valid(mac_product_valid[lane][pair_idx]),` This connects the child instance's named port 'product_valid' to parent expression 'mac_product_valid[lane][pair_idx]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 362: `.product_lo(mac_product_lo[lane][pair_idx]),` This connects the child instance's named port 'product_lo' to parent expression 'mac_product_lo[lane][pair_idx]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'mac product lo' in this block.

Line 363: `.product_hi(mac_product_hi[lane][pair_idx])` This connects the child instance's named port 'product_hi' to parent expression 'mac_product_hi[lane][pair_idx]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'mac product hi' in this block.

Line 364: `);` Within the module's structural declarations, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The expression is a literal or structural operator and has no hidden software state.

Line 365: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 366: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 368: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 369: `issue_ready = (state == ST_RUN) && !input_done_r;` This makes issue_ready, the consumer's statement that it can accept a payload this cycle, equal to '(state == ST_RUN) && !input_done_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_RUN: the value named 'st run' in this block; input_done_r: the value named 'input done r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 370: `busy = (state != ST_IDLE);` This makes busy, the value named 'busy' in this block, equal to '(state != ST_IDLE)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 371: `result_valid = (state == ST_OUTPUT_PIPELINE) && result_stage_valid_r;` This makes result_valid, the producer's statement that its current payload is meaningful, equal to '(state == ST_OUTPUT_PIPELINE) && result_stage_valid_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT_PIPELINE: the value named 'st output pipeline' in this block; result_stage_valid_r: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 372: `result_row = result_stage_payload_r[51:48];` This makes result_row, one output-row position processed by the tensor tile, equal to 'result_stage_payload_r[51:48]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_stage_payload_r: stream data belonging to the current command. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 373: `result_column = result_stage_payload_r[57:52];` This makes result_column, one output-feature position processed in parallel, equal to 'result_stage_payload_r[57:52]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_stage_payload_r: stream data belonging to the current command. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 374: `result_last = result_stage_last_r;` This makes result_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'result_stage_last_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_stage_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 375: `result_value = result_stage_payload_r[47:0];` This makes result_value, the value named 'result value' in this block, equal to 'result_stage_payload_r[47:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_stage_payload_r: stream data belonging to the current command. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 376: `debug_dense_macs = {45'd0, debug_dense_macs_r};` This makes debug_dense_macs, a count of ordinary activation-times-weight operations actually issued, equal to '{45'd0, debug_dense_macs_r}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block; debug_dense_macs_r: a count of ordinary activation-times-weight operations actually issued. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 377: `debug_spike_adds = {45'd0, debug_spike_adds_r};` This makes debug_spike_adds, a binary event produced when a neuron crosses its threshold, equal to '{45'd0, debug_spike_adds_r}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block; debug_spike_adds_r: a binary event produced when a neuron crosses its threshold. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 378: `// The FIFO holds the largest legal tile, so all arithmetic stages can` Design intent: The FIFO holds the largest legal tile, so all arithmetic stages can This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 379: `// advance every cycle regardless of downstream READY.` Design intent: advance every cycle regardless of downstream READY. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 380: `output_pipeline_enable = 1'b1;` This makes output_pipeline_enable, a control bit that permits the associated state or storage update, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 381: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 383: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 384: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 385: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 386: `mode_r <= MODE_DENSE_OUTER;` This makes mode_r, the value named 'mode r' in this block, equal to 'MODE_DENSE_OUTER'. It updates after the active clock edge together with other nonblocking assignments. The important operands are MODE_DENSE_OUTER: the value named 'mode dense outer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 387: `rows_r <= 4'd1;` This makes rows_r, one output-row position processed by the tensor tile, equal to '4'd1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 388: `active_row_mask_r <= {{(ROW_LANES-1){1'b0}}, 1'b1};` This makes active_row_mask_r, one output-row position processed by the tensor tile, equal to '{{(ROW_LANES-1){1'b0}}, 1'b1}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; b0: the value named 'b0' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 389: `columns_r <= 6'd1;` This makes columns_r, one output-feature position processed in parallel, equal to '6'd1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 390: `activation_zp_r <= 8'd0;` This makes activation_zp_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 391: `weight_zp_r <= 8'd0;` This makes weight_zp_r, a learned coefficient multiplied by an activation, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 392: `config_error <= 1'b0;` This makes config_error, the value named 'config error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 393: `effective_reduction <= 32'd0;` This makes effective_reduction, the value named 'effective reduction' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 394: `mac_issue_count <= 12'd0;` This makes mac_issue_count, an integer count retained for control or measurable telemetry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 395: `mac_retire_count <= 12'd0;` This makes mac_retire_count, an integer count retained for control or measurable telemetry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 396: `target_mac_retires <= 12'd0;` This makes target_mac_retires, the value named 'target mac retires' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 397: `drain_complete_r <= 1'b0;` This makes drain_complete_r, the value named 'drain complete r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 398: `zero_point_product_r <= 16'd0;` This makes zero_point_product_r, the integer code representing real numeric zero in affine quantization, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 399: `zero_restore_reduction_operand_r <= 32'd0;` This makes zero_restore_reduction_operand_r, the value named 'zero restore reduction operand r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 400: `zero_restore_zp_operand_r <= 16'd0;` This makes zero_restore_zp_operand_r, the value named 'zero restore zp operand r' in this block, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 401: `zero_restore_low_product_r <= 32'd0;` This makes zero_restore_low_product_r, the value named 'zero restore low product r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 402: `zero_restore_high_product_r <= 32'd0;` This makes zero_restore_high_product_r, the value named 'zero restore high product r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 403: `zero_restore_low_relay_r <= 32'd0;` This makes zero_restore_low_relay_r, the value named 'zero restore low relay r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 404: `zero_restore_high_relay_r <= 32'd0;` This makes zero_restore_high_relay_r, the value named 'zero restore high relay r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 405: `zero_point_restore_r <= 48'd0;` This makes zero_point_restore_r, the integer code representing real numeric zero in affine quantization, equal to '48'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 406: `launch_total_r <= 8'd0;` This makes launch_total_r, the value named 'launch total r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 407: `launch_remaining_r <= 8'd0;` This makes launch_remaining_r, the value named 'launch remaining r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 408: `launch_rows_remaining_r <= 4'd0;` This makes launch_rows_remaining_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 409: `launch_columns_remaining_r <= 6'd0;` This makes launch_columns_remaining_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 410: `launch_row_r <= 4'd0;` This makes launch_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 411: `launch_column_r <= 6'd0;` This makes launch_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 412: `pread_valid_r <= 1'b0;` This makes pread_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 413: `pread_row_sum_r <= 32'd0;` This makes pread_row_sum_r, one output-row position processed by the tensor tile, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 414: `pread_column_sum_r <= 32'd0;` This makes pread_column_sum_r, one output-feature position processed in parallel, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 415: `pread_row_r <= 4'd0;` This makes pread_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 416: `pread_column_r <= 6'd0;` This makes pread_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 417: `p0_valid_r <= 1'b0;` This makes p0_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 418: `p0_raw_r <= 64'sd0;` This makes p0_raw_r, the value named 'p0 raw r' in this block, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 419: `p0_row_sum_r <= 32'd0;` This makes p0_row_sum_r, one output-row position processed by the tensor tile, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 420: `p0_column_sum_r <= 32'd0;` This makes p0_column_sum_r, one output-feature position processed in parallel, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 421: `p0_row_zp_low_r <= 8'd0;` This makes p0_row_zp_low_r, one output-row position processed by the tensor tile, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 422: `p0_row_zp_high_r <= 8'd0;` This makes p0_row_zp_high_r, one output-row position processed by the tensor tile, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 423: `p0_column_zp_low_r <= 8'd0;` This makes p0_column_zp_low_r, one output-feature position processed in parallel, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 424: `p0_column_zp_high_r <= 8'd0;` This makes p0_column_zp_high_r, one output-feature position processed in parallel, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 425: `p0_row_r <= 4'd0;` This makes p0_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 426: `p0_column_r <= 6'd0;` This makes p0_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 427: `p1_valid_r <= 1'b0;` This makes p1_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 428: `p1_raw_r <= 64'sd0;` This makes p1_raw_r, the value named 'p1 raw r' in this block, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 429: `p1_row_product_low_r <= 24'd0;` This makes p1_row_product_low_r, one output-row position processed by the tensor tile, equal to '24'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 430: `p1_row_product_high_r <= 24'd0;` This makes p1_row_product_high_r, one output-row position processed by the tensor tile, equal to '24'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 431: `p1_column_product_low_r <= 24'd0;` This makes p1_column_product_low_r, one output-feature position processed in parallel, equal to '24'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 432: `p1_column_product_high_r <= 24'd0;` This makes p1_column_product_high_r, one output-feature position processed in parallel, equal to '24'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 433: `p1_row_r <= 4'd0;` This makes p1_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 434: `p1_column_r <= 6'd0;` This makes p1_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 435: `p2_valid_r <= 1'b0;` This makes p2_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 436: `p2_raw_r <= 64'sd0;` This makes p2_raw_r, the value named 'p2 raw r' in this block, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 437: `p2_row_correction_r <= 64'sd0;` This makes p2_row_correction_r, one output-row position processed by the tensor tile, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 438: `p2_column_correction_r <= 64'sd0;` This makes p2_column_correction_r, one output-feature position processed in parallel, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 439: `p2_row_r <= 4'd0;` This makes p2_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 440: `p2_column_r <= 6'd0;` This makes p2_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 441: `p3_valid_r <= 1'b0;` This makes p3_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 442: `p3_corrected_r <= 64'sd0;` This makes p3_corrected_r, the value named 'p3 corrected r' in this block, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 443: `p3_column_correction_r <= 64'sd0;` This makes p3_column_correction_r, one output-feature position processed in parallel, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 444: `p3_row_r <= 4'd0;` This makes p3_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 445: `p3_column_r <= 6'd0;` This makes p3_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 446: `p4_valid_r <= 1'b0;` This makes p4_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 447: `p4_corrected_r <= 64'sd0;` This makes p4_corrected_r, the value named 'p4 corrected r' in this block, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 448: `p4_row_r <= 4'd0;` This makes p4_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 449: `p4_column_r <= 6'd0;` This makes p4_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 450: `p5_valid_r <= 1'b0;` This makes p5_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 451: `p5_result_r <= 48'sd0;` This makes p5_result_r, the value named 'p5 result r' in this block, equal to '48'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 452: `p5_row_r <= 4'd0;` This makes p5_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 453: `p5_column_r <= 6'd0;` This makes p5_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 454: `result_fifo_count_r <= '0;` This makes result_fifo_count_r, an integer count retained for control or measurable telemetry, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 455: `result_fifo_has_space_r <= 1'b1;` This makes result_fifo_has_space_r, the value named 'result fifo has space r' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 456: `result_fifo_nonempty_r <= 1'b0;` This makes result_fifo_nonempty_r, the value named 'result fifo nonempty r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 457: `result_fifo_write_pointer_r <= '0;` This makes result_fifo_write_pointer_r, the value named 'result fifo write pointer r' in this block, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 458: `result_fifo_read_pointer_r <= '0;` This makes result_fifo_read_pointer_r, the value named 'result fifo read pointer r' in this block, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 459: `result_fifo_last_address_r <= '0;` This makes result_fifo_last_address_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 460: `result_stage_valid_r <= 1'b0;` This makes result_stage_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 461: `result_stage_payload_r <= '0;` This makes result_stage_payload_r, stream data belonging to the current command, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 462: `result_stage_last_r <= 1'b0;` This makes result_stage_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 463: `accumulate_delta_valid_r <= 1'b0;` This makes accumulate_delta_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 464: `debug_accepted_issues <= 32'd0;` This makes debug_accepted_issues, the value named 'debug accepted issues' in this block, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 465: `debug_skipped_issues <= 32'd0;` This makes debug_skipped_issues, the value named 'debug skipped issues' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 466: `debug_retired_issues <= 32'd0;` This makes debug_retired_issues, the value named 'debug retired issues' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 467: `debug_dense_macs_r <= 19'd0;` This makes debug_dense_macs_r, a count of ordinary activation-times-weight operations actually issued, equal to '19'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 468: `debug_spike_adds_r <= 19'd0;` This makes debug_spike_adds_r, a binary event produced when a neuron crosses its threshold, equal to '19'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 469: `metric_increment_r <= 8'd0;` This makes metric_increment_r, the value named 'metric increment r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 470: `metric_increment_valid_r <= 1'b0;` This makes metric_increment_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 471: `sparse_active_count_r <= 4'd0;` This makes sparse_active_count_r, an integer count retained for control or measurable telemetry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 472: `sparse_metric_valid_r <= 1'b0;` This makes sparse_metric_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 473: `dense_macs_per_issue_r <= 8'd0;` This makes dense_macs_per_issue_r, a count of ordinary activation-times-weight operations actually issued, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 474: `dot_sum_valid_r <= 1'b0;` This makes dot_sum_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 475: `dot_pair_valid_r <= 1'b0;` This makes dot_pair_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 476: `dot_half_valid_r <= 1'b0;` This makes dot_half_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 477: `dot8_pair_valid_r <= 1'b0;` This makes dot8_pair_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 478: `dot8_increment_half_valid_r <= 1'b0;` This makes dot8_increment_half_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 479: `sum_update_valid_r <= 1'b0;` This makes sum_update_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 480: `input_done_r <= 1'b0;` This makes input_done_r, the value named 'input done r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 481: `issue_match_valid_r <= 1'b0;` This makes issue_match_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 482: `issue_zero_matches_r <= 8'd0;` This makes issue_zero_matches_r, the value named 'issue zero matches r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 483: `issue_match_last_r <= 1'b0;` This makes issue_match_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 484: `issue_match_activation_vector_r <= '0;` This makes issue_match_activation_vector_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 485: `issue_match_outer_weight_vector_r <= '0;` This makes issue_match_outer_weight_vector_r, a learned coefficient multiplied by an activation, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 486: `issue_match_dot_weight_vector_r <= '0;` This makes issue_match_dot_weight_vector_r, a learned coefficient multiplied by an activation, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 487: `classify_valid_r <= 1'b0;` This makes classify_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 488: `classify_is_zero_r <= 1'b0;` This makes classify_is_zero_r, the value named 'classify is zero r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 489: `classify_last_r <= 1'b0;` This makes classify_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 490: `classify_activation_vector_r <= '0;` This makes classify_activation_vector_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 491: `classify_outer_weight_vector_r <= '0;` This makes classify_outer_weight_vector_r, a learned coefficient multiplied by an activation, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 492: `classify_dot_weight_vector_r <= '0;` This makes classify_dot_weight_vector_r, a learned coefficient multiplied by an activation, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 493: `dense_issue_enable_r <= 1'b0;` This makes dense_issue_enable_r, a control bit that permits the associated state or storage update, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 494: `dense_activation_vector_r <= '0;` This makes dense_activation_vector_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 495: `dense_outer_tag0_r <= 1'b0;` This makes dense_outer_tag0_r, the value named 'dense outer tag0 r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 496: `dense_outer_tag1_r <= 1'b0;` This makes dense_outer_tag1_r, the value named 'dense outer tag1 r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 497: `dense_outer_tag2_r <= 1'b0;` This makes dense_outer_tag2_r, the value named 'dense outer tag2 r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 498: `sparse_prepare_valid_r <= 1'b0;` This makes sparse_prepare_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 499: `sparse_prepare_active_r <= '0;` This makes sparse_prepare_active_r, the value named 'sparse prepare active r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 500: `sparse_prepare_negative_r <= '0;` This makes sparse_prepare_negative_r, the value named 'sparse prepare negative r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 501: `sparse_prepare_weight_vector_r <= '0;` This makes sparse_prepare_weight_vector_r, a learned coefficient multiplied by an activation, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 502: `sparse_delta_valid_r <= 1'b0;` This makes sparse_delta_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 503: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 504: `dot_sum_lo_r[pair_idx] <= 20'd0;` This makes dot_sum_lo_r[pair_idx], the value named 'dot sum lo r' in this block, equal to '20'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 505: `dot_sum_hi_r[pair_idx] <= 20'd0;` This makes dot_sum_hi_r[pair_idx], the value named 'dot sum hi r' in this block, equal to '20'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 506: `for (int reduce_pair = 0; reduce_pair < 4; reduce_pair++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 507: `dot_pair_lo_r[pair_idx][reduce_pair] <= 17'd0;` This makes dot_pair_lo_r[pair_idx][reduce_pair], the value named 'dot pair lo r' in this block, equal to '17'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 508: `dot_pair_hi_r[pair_idx][reduce_pair] <= 17'd0;` This makes dot_pair_hi_r[pair_idx][reduce_pair], the value named 'dot pair hi r' in this block, equal to '17'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 509: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 510: `for (int reduce_half = 0; reduce_half < 2; reduce_half++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 511: `dot_half_lo_r[pair_idx][reduce_half] <= 18'd0;` This makes dot_half_lo_r[pair_idx][reduce_half], the value named 'dot half lo r' in this block, equal to '18'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 512: `dot_half_hi_r[pair_idx][reduce_half] <= 18'd0;` This makes dot_half_hi_r[pair_idx][reduce_half], the value named 'dot half hi r' in this block, equal to '18'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 513: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 514: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 515: `for (int pair_idx = 0; pair_idx < 4; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 516: `dot8_row_pair_r[pair_idx] <= 9'd0;` This makes dot8_row_pair_r[pair_idx], one output-row position processed by the tensor tile, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 517: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 518: `dot8_row_half_r[0] <= 10'd0;` This makes dot8_row_half_r[0], one output-row position processed by the tensor tile, equal to '10'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 519: `dot8_row_half_r[1] <= 10'd0;` This makes dot8_row_half_r[1], one output-row position processed by the tensor tile, equal to '10'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 520: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 521: `accumulator_clear_row_r[row_idx] <= 1'b0;` This makes accumulator_clear_row_r[row_idx], a widened running sum that prevents intermediate overflow, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 522: `pread_accumulator_by_row_r[row_idx] <= '0;` This makes pread_accumulator_by_row_r[row_idx], a widened running sum that prevents intermediate overflow, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 523: `row_sum[row_idx] <= 32'd0;` This makes row_sum[row_idx], one output-row position processed by the tensor tile, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 524: `row_increment_r[row_idx] <= 16'd0;` This makes row_increment_r[row_idx], one output-row position processed by the tensor tile, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 525: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 526: `dense_weight_lo_r[row_idx][pair_idx] <= 8'd0;` This makes dense_weight_lo_r[row_idx][pair_idx], a learned coefficient multiplied by an activation, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 527: `dense_weight_hi_r[row_idx][pair_idx] <= 8'd0;` This makes dense_weight_hi_r[row_idx][pair_idx], a learned coefficient multiplied by an activation, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 528: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 529: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 530: `accumulator[row_idx][column_idx] <= '0;` This makes accumulator[row_idx][column_idx], a widened running sum that prevents intermediate overflow, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 531: `accumulate_delta_r[row_idx][column_idx] <= 21'sd0;` This makes accumulate_delta_r[row_idx][column_idx], the value named 'accumulate delta r' in this block, equal to '21'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 532: `sparse_delta_r[row_idx][column_idx] <= 9'sd0;` This makes sparse_delta_r[row_idx][column_idx], the value named 'sparse delta r' in this block, equal to '9'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 533: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 534: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 535: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 536: `column_sum_clear_r[column_idx] <= 1'b0;` This makes column_sum_clear_r[column_idx], one output-feature position processed in parallel, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 537: `column_sum[column_idx] <= 32'd0;` This makes column_sum[column_idx], one output-feature position processed in parallel, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 538: `column_increment_r[column_idx] <= 16'd0;` This makes column_increment_r[column_idx], one output-feature position processed in parallel, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 539: `for (int pair_idx = 0; pair_idx < 4; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 540: `dot8_column_pair_r[column_idx][pair_idx] <= 9'd0;` This makes dot8_column_pair_r[column_idx][pair_idx], one output-feature position processed in parallel, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 541: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 542: `dot8_column_half_r[column_idx][0] <= 10'd0;` This makes dot8_column_half_r[column_idx][0], one output-feature position processed in parallel, equal to '10'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 543: `dot8_column_half_r[column_idx][1] <= 10'd0;` This makes dot8_column_half_r[column_idx][1], one output-feature position processed in parallel, equal to '10'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 544: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 545: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 546: `if (result_fifo_push) begin` This condition selects the following hardware branch only when ‘result_fifo_push‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_fifo_push: the value named ’result fifo push’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 547: `result_fifo_payload_r[result_fifo_write_pointer_r]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are result_fifo_payload_r: stream data belonging to the current command; result_fifo_write_pointer_r: the value named ’result fifo write pointer r’ in this block.

Line 548: `<= {p5_column_r, p5_row_r, p5_result_r};` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are p5_column_r: one output-feature position processed in parallel; p5_row_r: one output-row position processed by the tensor tile; p5_result_r: the value named ’p5 result r’ in this block.

Line 549: `result_fifo_write_pointer_r <= result_fifo_write_pointer_r + 1'b1;` This makes result_fifo_write_pointer the value named ’result fifo write pointer r’ in this block, equal to ‘result_fifo_write_pointer_r + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_fifo_write_pointer_r: the value named ’result fifo write pointer r’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 550: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 551: `if (result_fifo_pop) begin` This condition selects the following hardware branch only when ‘result_fifo_pop‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_fifo_pop: the value named ’result fifo pop’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 552: `result_stage_payload_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are result_stage_payload_r: stream data belonging to the current command.

Line 553: `<= result_fifo_payload_r[result_fifo_read_pointer_r];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are result_fifo_payload_r: stream data belonging to the current command; result_fifo_read_pointer_r: the value named ’result fifo read pointer r’ in this block.

Line 554: `result_stage_last_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are result_stage_last_r: the AXI-Stream marker for the final beat of a packet or command payload.

Line 555: `<= result_fifo_read_pointer_r == result_fifo_last_address_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are result_fifo_read_pointer_r: the value named 'result fifo read pointer r' in this block; result_fifo_last_address_r: the AXI-Stream marker for the final beat of a packet or command payload.

Line 556: `result_stage_valid_r <= 1'b1;` This makes result_stage_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 557: `result_fifo_read_pointer_r <= result_fifo_read_pointer_r + 1'b1;` This makes result_fifo_read_pointer_r, the value named 'result fifo read pointer r' in this block, equal to 'result_fifo_read_pointer_r + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_fifo_read_pointer_r: the value named 'result fifo read pointer r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 558: `end else if (result_stage_pop) begin` This condition selects the following hardware branch only when 'result_stage_pop' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_stage_pop: the value named 'result stage pop' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 559: `result_stage_valid_r <= 1'b0;` This makes result_stage_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 560: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 561: `unique case ({result_fifo_push, result_fifo_pop})` This starts a multiplexer/state decode selected by '{result_fifo_push, result_fifo_pop}'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 562: `2'b10: begin` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are b10: the value named 'b10' in this block.

Line 563: `result_fifo_count_r <= result_fifo_count_r + 1'b1;` This makes result_fifo_count_r, an integer count retained for control or measurable telemetry, equal to 'result_fifo_count_r + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_fifo_count_r: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 564: `result_fifo_has_space_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are result_fifo_has_space_r: the value named 'result fifo has space r' in this block.

Line 565: `<= result_fifo_count_r` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are result_fifo_count_r: an integer count retained for control or measurable telemetry.

Line 566: `< RESULT_FIFO_COUNT_BITS'(RESULT_FIFO_DEPTH - 1);` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are RESULT_FIFO_COUNT_BITS: the value named 'result fifo count bits' in this block; RESULT_FIFO_DEPTH: the value named 'result fifo depth' in this block.

Line 567: `result_fifo_nonempty_r <= 1'b1;` This makes result_fifo_nonempty_r, the value named 'result fifo nonempty r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 568: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 569: `2'b01: begin` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are b01: the value named 'b01' in this block.

Line 570: `result_fifo_count_r <= result_fifo_count_r - 1'b1;` This makes result_fifo_count_r, an integer count retained for control or measurable telemetry, equal to 'result_fifo_count_r - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_fifo_count_r: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 571: `result_fifo_has_space_r <= 1'b1;` This makes result_fifo_has_space_r, the value named 'result fifo has space r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 572: `result_fifo_nonempty_r <= result_fifo_count_r != 1;` This makes result_fifo_nonempty_r, the value named 'result fifo nonempty r' in this block, equal to 'result_fifo_count_r != 1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_fifo_count_r: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge.

Line 573: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 574: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the clocked process that updates registers only on a rising clock edge.

Line 575: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 577: `// Unconditional products let Vivado absorb these registers into` Design intent: Unconditional products let Vivado absorb these registers into This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 578: `// the DSP48E1 PREG stage. The FSM presents operands one cycle` Design intent: the DSP48E1 PREG stage. The FSM presents operands one cycle This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 579: `// before ST_RESTORE_ADD consumes the registered products.` Design intent: before ST_RESTORE_ADD consumes the registered products. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 580: `zero_restore_low_product_r <=` This statement continues through source line 581; the explanation uses the complete expression rather than only the first visual fragment. This makes zero_restore_low_product_r, the value named 'zero restore low product r' in this block, equal to 'zero_restore_reduction_operand_r[15:0] * zero_restore_zp_operand_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_restore_reduction_operand_r: the value named 'zero restore reduction operand r' in this block; zero_restore_zp_operand_r: the value named 'zero restore zp operand r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 581: `zero_restore_reduction_operand_r[15:0] * zero_restore_zp_operand_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 580. Its exact fragment is 'zero_restore_reduction_operand_r[15:0] * zero_restore_zp_operand_r;'. The important operands are zero_restore_reduction_operand_r: the value named 'zero restore reduction operand r' in this block; zero_restore_zp_operand_r: the value named 'zero restore zp operand r' in this block. Read together, the complete operation is: This makes zero_restore_low_product_r, the value named 'zero restore low product r' in this block, equal to 'zero_restore_reduction_operand_r[15:0] * zero_restore_zp_operand_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_restore_reduction_operand_r: the value named 'zero restore reduction operand r' in this block; zero_restore_zp_operand_r: the value named 'zero restore zp operand r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 582: `zero_restore_high_product_r <=` This statement continues through source line 583; the explanation uses the complete expression rather than only the first visual fragment. This makes zero_restore_high_product_r, the value named 'zero restore high product r' in this block, equal to 'zero_restore_reduction_operand_r[31:16] * zero_restore_zp_operand_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_restore_reduction_operand_r: the value named 'zero restore reduction operand r' in this block; zero_restore_zp_operand_r: the value named 'zero restore zp operand r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 583: `zero_restore_reduction_operand_r[31:16] * zero_restore_zp_operand_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 582. Its exact fragment is 'zero_restore_reduction_operand_r[31:16] * zero_restore_zp_operand_r;'. The important operands are zero_restore_reduction_operand_r: the value named 'zero restore reduction operand r' in this block; zero_restore_zp_operand_r: the value named 'zero restore zp operand r' in this block. Read together, the complete operation is: This makes zero_restore_high_product_r, the value named 'zero restore high product r' in this block, equal to 'zero_restore_reduction_operand_r[31:16] * zero_restore_zp_operand_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_restore_reduction_operand_r: the value named 'zero restore reduction operand r' in this block; zero_restore_zp_operand_r: the value named 'zero restore zp operand r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 584: `// This exact unsigned 8x8 product maps to one DSP48E1 PREG. It` Design intent: This exact unsigned 8x8 product maps to one DSP48E1 PREG. It This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 585: `// replaces four LUT multipliers plus a reconstruction carry chain.` Design intent: replaces four LUT multipliers plus a reconstruction carry chain. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 586: `zero_point_product_r <= activation_zp_r * weight_zp_r;` This makes zero_point_product_r, the integer code representing real numeric zero in affine quantization, equal to 'activation_zp_r * weight_zp_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are activation_zp_r: an input feature value consumed by a neuron or multiply-accumulate operation; weight_zp_r: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 587: `// Dedicated copies have one DSP load each, allowing Vivado to use` Design intent: Dedicated copies have one DSP load each, allowing Vivado to use This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 588: `// the four BREG input stages instead of routing shared zero points.` Design intent: the four BREG input stages instead of routing shared zero points. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 589: `p0_row_zp_low_r <= weight_zp_r;` This makes p0_row_zp_low_r, one output-row position processed by the tensor tile, equal to 'weight_zp_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_zp_r: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 590: `p0_row_zp_high_r <= weight_zp_r;` This makes p0_row_zp_high_r, one output-row position processed by the tensor tile, equal to 'weight_zp_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_zp_r: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 591: `p0_column_zp_low_r <= activation_zp_r;` This makes p0_column_zp_low_r, one output-feature position processed in parallel, equal to 'activation_zp_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are activation_zp_r: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge.

Line 592: `p0_column_zp_high_r <= activation_zp_r;` This makes p0_column_zp_high_r, one output-feature position processed in parallel, equal to 'activation_zp_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are activation_zp_r: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge.

Line 594: `// The corresponding valid bit is the architectural qualifier, so` Design intent: The corresponding valid bit is the architectural qualifier, so This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 595: `// these pre-read registers may sample continuously. Free-running` Design intent: these pre-read registers may sample continuously. Free-running This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 596: `// them removes a 256-register launch_remaining clock-enable tree;` Design intent: them removes a 256-register launch_remaining clock-enable tree; This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 597: `// samples outside ST_OUTPUT_PIPELINE are never consumed.` Design intent: samples outside ST_OUTPUT_P are never consumed. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 598: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 599: `pread_accumulator_by_row_r[row_idx]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are pread_accumulator_by_row_r: a widened running sum that prevents intermediate overflow; row_idx: one output-row position processed by the tensor tile.

Line 600: `<= accumulator[row_idx][launch_column_r[3:0]];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulator: a widened running sum that prevents intermediate overflow; row_idx: one output-row position processed by the tensor tile; launch_column_r: one output-feature position processed in parallel.

Line 601: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 602: `pread_row_sum_r <= row_sum[launch_row_r[2:0]];` This makes pread_row_sum_r, one output-row position processed by the tensor tile, equal to 'row_sum[launch_row_r[2:0]]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are row_sum: one output-row position processed by the tensor tile; launch_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge.

Line 603: `pread_column_sum_r <= column_sum[launch_column_r[3:0]];` This makes pread_column_sum_r, one output-feature position processed in parallel, equal to 'column_sum[launch_column_r[3:0]]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are column_sum: one output-feature position processed in parallel; launch_column_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 604: `pread_row_r <= launch_row_r;` This makes pread_row_r, one output-row position processed by the tensor tile, equal to 'launch_row_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge.

Line 605: `pread_column_r <= launch_column_r;` This makes pread_column_r, one output-feature position processed in parallel, equal to 'launch_column_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_column_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 607: `dense_issue_enable_r <= 1'b0;` This makes dense_issue_enable_r, a control bit that permits the associated state or storage update, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 608: `dense_outer_tag0_r <= dense_issue_enable_r` This statement continues through source line 609; the explanation uses the complete expression rather than only the first visual fragment. This makes dense_outer_tag0_r, the value named 'dense outer tag0 r' in this block, equal to 'dense_issue_enable_r && mode_r == MODE_DENSE_OUTER'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dense_issue_enable_r: a control bit that permits the associated state or storage update; mode_r: the value named 'mode r' in this block; MODE_DENSE_OUTER: the value named 'mode dense outer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 609: `&& mode_r == MODE_DENSE_OUTER;` This is continuation line 2 of the 2-line logical statement that begins at source line 608. Its exact fragment is '&& mode_r == MODE_DENSE_OUTER;'. The important operands are mode_r: the value named 'mode r' in this block; MODE_DENSE_OUTER: the value named 'mode dense outer' in this block. Read together, the complete operation is: This makes dense_outer_tag0_r, the value named 'dense outer tag0 r' in this block, equal to 'dense_issue_enable_r && mode_r == MODE_DENSE_OUTER'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dense_issue_enable_r: a control bit that permits the associated state or storage update; mode_r: the value named 'mode r' in this block; MODE_DENSE_OUTER: the value named 'mode dense outer' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 610: `dense_outer_tag1_r <= dense_outer_tag0_r;` This makes dense_outer_tag1_r, the value named 'dense outer tag1 r' in this block, equal to 'dense_outer_tag0_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dense_outer_tag0_r: the value named 'dense outer tag0 r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 611: `dense_outer_tag2_r <= dense_outer_tag1_r;` This makes dense_outer_tag2_r, the value named 'dense outer tag2 r' in this block, equal to 'dense_outer_tag1_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dense_outer_tag1_r: the value named 'dense outer tag1 r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 612: `sparse_prepare_valid_r <= 1'b0;` This makes sparse_prepare_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 613: `sparse_delta_valid_r <= 1'b0;` This makes sparse_delta_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 614: `accumulate_delta_valid_r <= 1'b0;` This makes accumulate_delta_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 615: `metric_increment_valid_r <= 1'b0;` This makes metric_increment_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 616: `sparse_metric_valid_r <= 1'b0;` This makes sparse_metric_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 617: `if (metric_increment_valid_r) begin` This condition selects the following hardware branch only when 'metric_increment_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are metric_increment_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 618: `if (mode_r == MODE_DENSE_OUTER || mode_r == MODE_DENSE_DOT8) begin` This condition selects the following hardware branch only when 'mode_r == MODE_DENSE_OUTER || mode_r == MODE_DENSE_DOT8' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are mode_r: the value named 'mode r' in this block; MODE_DENSE_OUTER: the value named 'mode dense outer' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 619: `debug_dense_macs_r <= debug_dense_macs_r` This statement continues through source line 620; the explanation uses the complete expression rather than only the first visual fragment. This makes debug_dense_macs_r, a count of ordinary activation-times-weight operations actually issued, equal to 'debug_dense_macs_r + {11'd0, metric_increment_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are debug_dense_macs_r: a count of ordinary activation-times-weight operations actually issued; d0: the value named 'd0' in this block; metric_increment_r: the value named 'metric increment r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 620: `+ {11'd0, metric_increment_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 619. Its exact fragment is '+ {11'd0, metric_increment_r};'. The important operands are d0: the value named 'd0' in this block; metric_increment_r: the value named 'metric increment r' in this block. Read together, the complete operation is: This makes debug_dense_macs_r, a count of ordinary activation-times-weight operations actually issued, equal to 'debug_dense_macs_r + {11'd0, metric_increment_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are debug_dense_macs_r: a count of ordinary activation-times-weight operations actually issued; d0: the value named 'd0' in this block; metric_increment_r: the value named 'metric increment r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 621: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 622: `debug_spike_adds_r <= debug_spike_adds_r` This statement continues through source line 623; the explanation uses the complete expression rather than only the first visual fragment. This makes debug_spike_adds_r, a binary event produced when a neuron crosses its threshold, equal to 'debug_spike_adds_r + {11'd0, metric_increment_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are debug_spike_adds_r: a binary event produced when a neuron crosses its threshold; d0: the value named 'd0' in this block; metric_increment_r: the value named 'metric increment r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 623: `+ {11'd0, metric_increment_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 622. Its exact fragment is '+ {11'd0, metric_increment_r};'. The important operands are d0: the value named 'd0' in this block; metric_increment_r: the value named 'metric increment r' in this block. Read together, the complete operation is: This makes debug_spike_adds_r, a binary event produced when a neuron crosses its threshold, equal to 'debug_spike_adds_r + {11'd0, metric_increment_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are debug_spike_adds_r: a binary event produced when a neuron crosses its threshold; d0: the value named 'd0' in this block; metric_increment_r: the value named 'metric increment r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 624: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 625: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 626: `if (sparse_metric_valid_r) begin` This condition selects the following hardware branch only when 'sparse_metric_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sparse_metric_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 627: `metric_increment_r <= sparse_active_count_r * columns_r;` This makes metric_increment_r, the value named 'metric increment r' in this block, equal to 'sparse_active_count_r * columns_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sparse_active_count_r: an integer count retained for control or measurable telemetry; columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 628: `metric_increment_valid_r <= 1'b1;` This makes metric_increment_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 629: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 630: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 631: `accumulator_clear_row_r[row_idx] <= 1'b0;` This makes accumulator_clear_row_r[row_idx], a widened running sum that prevents intermediate overflow, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 632: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 633: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 634: `column_sum_clear_r[column_idx] <= 1'b0;` This makes column_sum_clear_r[column_idx], one output-feature position processed in parallel, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 635: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 636: `classify_valid_r <= issue_match_valid_r;` This makes classify_valid_r, the producer's statement that its current payload is meaningful, equal to 'issue_match_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_match_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 637: `issue_match_valid_r <= issue_accept;` This makes issue_match_valid_r, the producer's statement that its current payload is meaningful, equal to 'issue_accept'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_accept: the value named 'issue accept' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 638: `// The queue head is stable whenever it is not accepted, so capture` Design intent: The queue head is stable whenever it is not accepted, so capture This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 639: `// its payload unconditionally and use issue_match_valid_r as the` Design intent: its payload unconditionally and use issue_match_valid_r as the This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 640: `// only validity qualifier. This removes issue_accept from the CE` Design intent: only validity qualifier. This removes issue_accept from the CE This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 641: `// pins of 1,216 payload registers without changing pipeline latency.` Design intent: pins of 1,216 payload registers without changing pipeline latency. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 642: `issue_zero_matches_r <= issue_zero_matches_comb;` This makes issue_zero_matches_r, the value named 'issue zero matches r' in this block, equal to 'issue_zero_matches_comb'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_zero_matches_comb: the value named 'issue zero matches comb' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 643: `issue_match_last_r <= issue_last;` This makes issue_match_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'issue_last'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 644: `issue_match_activation_vector_r <= activation_vector;` This makes issue_match_activation_vector_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'activation_vector'. It updates after the active clock edge together with other nonblocking assignments. The important operands are activation_vector: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge.

Line 645: `issue_match_outer_weight_vector_r <= outer_weight_vector;` This makes issue_match_outer_weight_vector_ a learned coefficient multiplied by an activation, equal to 'outer_weight_vector'. It updates after the active clock edge together with other nonblocking assignments. The important operands are outer_weight_vector: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 646: `issue_match_dot_weight_vector_r <= dot_weight_vector;` This makes issue_match_dot_weight_vector_r, a learned coefficient multiplied by an activation, equal to 'dot_weight_vector'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_weight_vector: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 647: `if (issue_match_valid_r) begin` This condition selects the following hardware branch only when 'issue_match_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are issue_match_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 648: `classify_is_zero_r <= &issue_zero_matches_r;` This makes classify_is_zero_r, the value named 'classify is zero r' in this block, equal to '&issue_zero_matches_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_zero_matches_r: the value named 'issue zero matches r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 649: `classify_last_r <= issue_match_last_r;` This makes classify_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'issue_match_last_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_match_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 650: `classify_activation_vector_r <= issue_match_activation_vector_r;` This makes classify_activation_vector_ an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'issue_match_activation_vector_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_match_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge.

Line 651: `classify_outer_weight_vector_r <= issue_match_outer_weight_vector_r;` This makes classify_outer_weight_ a learned coefficient multiplied by an activation, equal to 'issue_match_outer_weight_vector_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_match_outer_weight_vector_r: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 652: `classify_dot_weight_vector_r <= issue_match_dot_weight_vector_r;` This makes classify_dot_weight_vector a learned coefficient multiplied by an activation, equal to ‘issue_match_dot_weight_vector_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are issue_match_dot_weight_vector_r: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 653: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 654: `dot8_pair_valid_r <= 1'b0;` This makes dot8_pair_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 655: `dot8_increment_half_valid_r <= 1'b0;` This makes dot8_increment_half_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 656: `dot_pair_valid_r <= dense_product_valid && mode_r == MODE_DENSE_DOT8;` This makes dot_pair_valid_r, the producer's statement that its current payload is meaningful, equal to ‘dense_product_valid && mode_r == MODE_DENSE_DOT8‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are dense_product_valid: the producer's statement that its current payload is meaningful; mode_r: the value named 'mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 657: `dot_half_valid_r <= dot_pair_valid_r;` This makes dot_half_valid_r, the producer's statement that its current payload is meaningful, equal to ‘dot_pair_valid_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_pair_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 658: `dot_sum_valid_r <= dot_half_valid_r;` This makes dot_sum_valid_r, the producer's statement that its current payload is meaningful, equal to ‘dot_half_valid_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_half_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 659: `sum_update_valid_r <= 1'b0;` This makes sum_update_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 660: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 661: `if (accumulator_clear_row_r[row_idx]) begin` This condition selects the following hardware branch only when ‘accumulator_clear_row_r[row_idx]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are accumulator_clear_row_r: a widened running sum that prevents intermediate overflow; row_idx: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge.

Line 662: `row_sum[row_idx] <= 32'd0;` This makes row_sum[row_idx], one output-row position processed by the tensor tile, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 663: `end else if (sum_update_valid_r) begin` This condition selects the following hardware branch only when 'sum_update_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sum_update_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 664: `row_sum[row_idx] <= row_sum[row_idx]` This statement continues through source line 665; the explanation uses the complete expression rather than only the first visual fragment. This makes row_sum[row_idx], one output-row position processed by the tensor tile, equal to 'row_sum[row_idx] + {16'd0, row_increment_r[row_idx]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are row_sum: one output-row position processed by the tensor tile; row_idx: one output-row position processed by the tensor tile; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 665: `+ {16'd0, row_increment_r[row_idx]};` This is continuation line 2 of the 2-line logical statement that begins at source line 664. Its exact fragment is '+ {16'd0, row_increment_r[row_idx]};'. The important operands are d0: the value named 'd0' in this block; row_increment_r: one output-row position processed by the tensor tile; row_idx: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes row_sum[row_idx], one output-row position processed by the tensor tile, equal to 'row_sum[row_idx] + {16'd0, row_increment_r[row_idx]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are row_sum: one output-row position processed by the tensor tile; row_idx: one output-row position processed by the tensor tile; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 666: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 667: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 668: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 669: `if (column_sum_clear_r[column_idx]) begin` This condition selects the following hardware branch only when 'column_sum_clear_r[column_idx]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are column_sum_clear_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 670: `column_sum[column_idx] <= 32'd0;` This makes column_sum[column_idx], one output-feature position processed in parallel, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 671: `end else if (sum_update_valid_r) begin` This condition selects the following hardware branch only when 'sum_update_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sum_update_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 672: `column_sum[column_idx] <= column_sum[column_idx]` This statement continues through source line 673; the explanation uses the complete expression rather than only the first visual fragment. This makes column_sum[column_idx], one output-feature position processed in parallel, equal to ‘column_sum[column_idx] + {16’d0, column_increment_r[column_idx]}‘ It updates after the active clock edge together with other nonblocking assignments. The important operands are column_sum: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 673: `+ {16'd0, column_increment_r[column_idx]};` This is continuation line 2 of the 2-line logical statement that begins at source line 672. Its exact fragment is ‘+ {16’d0, column_increment_r[column_idx]};‘. The important operands are d0: the value named ’d0’ in this block; column_increment_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes column_sum[column_idx], one output-feature position processed in parallel, equal to ‘column_sum[column_idx] + {16’d0, column_increment_r[column_idx]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are column_sum: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel; d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 674: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 675: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 676: `if (dot8_pair_valid_r) begin` This condition selects the following hardware branch only when ‘dot8_pair_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are dot8_pair_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 677: `dot8_increment_half_valid_r <= 1'b1;` This makes dot8_increment_half_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 678: `dot8_row_half_r[0] <= dot8_row_pair_r[0] + dot8_row_pair_r[1];` This makes dot8_row_half_r[0], one output-row position processed by the tensor tile, equal to ‘dot8_row_pair_r[0] + dot8_row_pair_r[1]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot8_row_pair_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge.

Line 679: `dot8_row_half_r[1] <= dot8_row_pair_r[2] + dot8_row_pair_r[3];` This makes dot8_row_half_r[1], one output-row position processed by the tensor tile, equal to ‘dot8_row_pair_r[2] + dot8_row_pair_r[3]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot8_row_pair_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge.

Line 680: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 681: `dot8_column_half_r[column_idx][0] <=` This statement continues through source line 683; the explanation uses the complete expression rather than only the first visual fragment. This makes dot8_column_half_r[column_idx][0], one output-feature position processed in parallel, equal to ‘dot8_column_pair_r[column_idx][0] + dot8_column_pair_r[column_id It updates after the active clock edge together with other nonblocking assignments. The important operands are

dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 682: `dot8_column_pair_r[column_idx][0]` This is continuation line 2 of the 3-line logical statement that begins at source line 681. Its exact fragment is ‘dot8_column_pair_r[column_idx][0]‘. The important operands are dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes dot8_column_half_r[column_idx][0], one output-feature position processed in parallel, equal to ‘dot8_column_pair_r[column_idx][0] + dot8_column_pair_r[column_idx][1
It updates after the active clock edge together with other nonblocking assignments. The important operands are dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 683: `+ dot8_column_pair_r[column_idx][1];` This is continuation line 3 of the 3-line logical statement that begins at source line 681. Its exact fragment is ‘+ dot8_column_pair_r[column_idx][1];‘. The important operands are dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes dot8_column_half_r[column_idx][0], one output-feature position processed in parallel, equal to ‘dot8_column_pair_r[column_idx][0] + dot8_column_pair_r[column_idx][1
It updates after the active clock edge together with other nonblocking assignments. The important operands are dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 684: `dot8_column_half_r[column_idx][1] <=` This statement continues through source line 686; the explanation uses the complete expression rather than only the first visual fragment. This makes dot8_column_half_r[column_idx][1], one output-feature position processed in parallel, equal to ‘dot8_column_pair_r[column_idx][2] + dot8_column_pair_r[column_id
It updates after the active clock edge together with other nonblocking assignments. The important operands are dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 685: `dot8_column_pair_r[column_idx][2]` This is continuation line 2 of the 3-line logical statement that begins at source line 684. Its exact fragment is ‘dot8_column_pair_r[column_idx][2]‘. The important operands are dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes dot8_column_half_r[column_idx][1], one output-feature position processed in parallel, equal to ‘dot8_column_pair_r[column_idx][2] + dot8_column_pair_r[column_idx][3
It updates after the active clock edge together with other nonblocking assignments. The important operands are dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 686: `+ dot8_column_pair_r[column_idx][3];` This is continuation line 3 of the 3-line logical statement that begins at source line 684. Its exact fragment is ‘+ dot8_column_pair_r[column_idx][3];‘. The important operands are dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes dot8_column_half_r[column_idx][1], one output-feature position processed in parallel, equal to ‘dot8_column_pair_r[column_idx][2] + dot8_column_pair_r[column_idx][3
It updates after the active clock edge together with other nonblocking assignments. The important operands are dot8_column_pair_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 687: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 688: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 689: `if (dot8_increment_half_valid_r) begin` This condition selects the following hardware branch only when ‘dot8_increment_half_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are dot8_increment_half_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 690: `row_increment_r[0] <= {6'd0, dot8_row_half_r[0]}` This statement continues through source line 691; the explanation uses the complete expression rather than only the first visual fragment. This makes row_increment_r[0], one output-row position processed by the tensor tile, equal to '{6'd0, dot8_row_half_r[0]} + {6'd0, dot8_row_half_r[1]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; dot8_row_half_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge.

Line 691: `+ {6'd0, dot8_row_half_r[1]};` This is continuation line 2 of the 2-line logical statement that begins at source line 690. Its exact fragment is '+ {6'd0, dot8_row_half_r[1]};'. The important operands are d0: the value named 'd0' in this block; dot8_row_half_r: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes row_increment_r[0], one output-row position processed by the tensor tile, equal to '{6'd0, dot8_row_half_r[0]} + {6'd0, dot8_row_half_r[1]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; dot8_row_half_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge.

Line 692: `for (int row_idx = 1; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 693: `row_increment_r[row_idx] <= 16'd0;` This makes row_increment_r[row_idx], one output-row position processed by the tensor tile, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 694: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 695: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 696: `column_increment_r[column_idx]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are column_increment_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel.

Line 697: `<= {6'd0, dot8_column_half_r[column_idx][0]}` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are d0: the value named 'd0' in this block; dot8_column_half_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel.

Line 698: `+ {6'd0, dot8_column_half_r[column_idx][1]};` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are d0: the value named 'd0' in this block; dot8_column_half_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel.

Line 699: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 700: `sum_update_valid_r <= 1'b1;` This makes sum_update_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 701: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 702: `if (dense_product_valid && mode_r == MODE_DENSE_DOT8) begin` This condition selects the following hardware branch only when 'dense_product_valid && mode_r == MODE_DENSE_DOT8' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are dense_product_valid: the producer's statement that its current payload is meaningful; mode_r: the value named 'mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 703: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 704: `for (int reduce_pair = 0; reduce_pair < 4; reduce_pair++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 705: `dot_pair_lo_r[pair_idx][reduce_pair] <=` This statement continues through source line 707; the explanation uses the complete expression rather than only the first visual fragment. This makes dot_pair_lo_r[pair_idx][reduce_pair], the value named 'dot pair lo r' in this block, equal to 'mac_product_lo[2*reduce_pair][pair_idx] + mac_product_lo[2*reduce_pair+ It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_product_lo: the value named 'mac product lo' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 706: `mac_product_lo[2*reduce_pair][pair_idx]` This is continuation line 2 of the 3-line logical statement that begins at source line 705. Its exact fragment is 'mac_product_lo[2*reduce_pair][pair_idx]'. The important operands are mac_product_lo: the value named 'mac product lo' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes dot_pair_lo_r[pair_idx][reduce_pair], the value named 'dot pair lo r' in this block, equal to 'mac_product_lo[2*reduce_pair][pair_idx] + mac_product_lo[2*reduce_pair+1][pair_idx]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_product_lo: the value named 'mac product lo' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 707: `+ mac_product_lo[2*reduce_pair+1][pair_idx];` This is continuation line 3 of the 3-line logical statement that begins at source line 705. Its exact fragment is '+ mac_product_lo[2*reduce_pair+1][pair_idx];'. The important operands are mac_product_lo: the value named 'mac product lo' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes dot_pair_lo_r[pair_idx][reduce_pair], the value named 'dot pair lo r' in this block, equal to 'mac_product_lo[2*reduce_pair][pair_idx] + mac_product_lo[2*reduce_pair+1][pair_idx]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_product_lo: the value named 'mac product lo' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 708: `dot_pair_hi_r[pair_idx][reduce_pair] <=` This statement continues through source line 710; the explanation uses the complete expression rather than only the first visual fragment. This makes dot_pair_hi_r[pair_idx][reduce_pair],

the value named 'dot pair hi r' in this block, equal to 'mac_product_hi[2*reduce_pair][pair_idx] + mac_product_hi[2*reduce_pair- It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_product_hi: the value named 'mac product hi' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 709: `mac_product_hi[2*reduce_pair][pair_idx]` This is continuation line 2 of the 3-line logical statement that begins at source line 708. Its exact fragment is 'mac_product_hi[2*reduce_pair][pair_idx]'. The important operands are mac_product_hi: the value named 'mac product hi' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes dot_pair_hi_r[pair_idx][reduce_pair], the value named 'dot pair hi r' in this block, equal to 'mac_product_hi[2*reduce_pair][pair_idx] + mac_product_hi[2*reduce_pair+1][pair_idx]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_product_hi: the value named 'mac product hi' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 710: `+ mac_product_hi[2*reduce_pair+1][pair_idx];` This is continuation line 3 of the 3-line logical statement that begins at source line 708. Its exact fragment is '+ mac_product_hi[2*reduce_pair+1][pair_idx];'. The important operands are mac_product_hi: the value named 'mac product hi' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes dot_pair_hi_r[pair_idx][reduce_pair], the value named 'dot pair hi r' in this block, equal to 'mac_product_hi[2*reduce_pair][pair_idx] + mac_product_hi[2*reduce_pair+1][pair_idx]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_product_hi: the value named 'mac product hi' in this block; reduce_pair: the value named 'reduce pair' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 711: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 712: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 713: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 714: `if (dot_pair_valid_r) begin` This condition selects the following hardware branch only when 'dot_pair_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are dot_pair_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 715: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 716: `dot_half_lo_r[pair_idx][0] <= dot_pair_lo_r[pair_idx][0]` This statement continues through source line 717; the explanation uses the complete expression rather than only the first visual fragment. This makes dot_half_lo_r[pair_idx the value named 'dot half lo r' in this block, equal to 'dot_pair_lo_r[pair_idx][0] + dot_pair_lo_r[pair_idx][1]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_pair_lo_r: the value named 'dot pair lo r' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 717: `+ dot_pair_lo_r[pair_idx][1];` This is continuation line 2 of the 2-line logical statement that begins at source line 716. Its exact fragment is '+ dot_pair_lo_r[pair_idx][1];'. The important operands are dot_pair_lo_r: the value named 'dot pair lo r' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the

complete operation is: This makes dot_half_lo_r[pair_idx][0], the value named 'dot half lo r' in this block, equal to 'dot_pair_lo_r[pair_idx][0] + dot_pair_lo_r[pair_idx][1]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_pair_lo_r: the value named 'dot pair lo r' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 718: `dot_half_lo_r[pair_idx][1] <= dot_pair_lo_r[pair_idx][2]` This statement continues through source line 719; the explanation uses the complete expression rather than only the first visual fragment. This makes dot_half_lo_r[pair_idx the value named 'dot half lo r' in this block, equal to 'dot_pair_lo_r[pair_idx][2] + dot_pair_lo_r[pair_idx][3]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_pair_lo_r: the value named 'dot pair lo r' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 719: `+ dot_pair_lo_r[pair_idx][3];` This is continuation line 2 of the 2-line logical statement that begins at source line 718. Its exact fragment is '+ dot_pair_lo_r[pair_idx][3];'. The important operands are dot_pair_lo_r: the value named 'dot pair lo r' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes dot_half_lo_r[pair_idx][1], the value named 'dot half lo r' in this block, equal to 'dot_pair_lo_r[pair_idx][2] + dot_pair_lo_r[pair_idx][3]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_pair_lo_r: the value named 'dot pair lo r' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 720: `dot_half_hi_r[pair_idx][0] <= dot_pair_hi_r[pair_idx][0]` This statement continues through source line 721; the explanation uses the complete expression rather than only the first visual fragment. This makes dot_half_hi_r[pair_idx the value named 'dot half hi r' in this block, equal to 'dot_pair_hi_r[pair_idx][0] + dot_pair_hi_r[pair_idx][1]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_pair_hi_r: the value named 'dot pair hi r' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 721: `+ dot_pair_hi_r[pair_idx][1];` This is continuation line 2 of the 2-line logical statement that begins at source line 720. Its exact fragment is '+ dot_pair_hi_r[pair_idx][1];'. The important operands are dot_pair_hi_r: the value named 'dot pair hi r' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes dot_half_hi_r[pair_idx][0], the value named 'dot half hi r' in this block, equal to 'dot_pair_hi_r[pair_idx][0] + dot_pair_hi_r[pair_idx][1]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_pair_hi_r: the value named 'dot pair hi r' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 722: `dot_half_hi_r[pair_idx][1] <= dot_pair_hi_r[pair_idx][2]` This statement continues through source line 723; the explanation uses the complete expression rather than only the first visual fragment. This makes dot_half_hi_r[pair_idx the value named 'dot half hi r' in this block, equal to 'dot_pair_hi_r[pair_idx][2] + dot_pair_hi_r[pair_idx][3]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_pair_hi_r: the value named 'dot pair hi r' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 723: `+ dot_pair_hi_r[pair_idx][3];` This is continuation line 2 of the 2-line logical statement that begins at source line 722. Its exact fragment is '+ dot_pair_hi_r[pair_idx][3];'. The important operands are dot_pair_hi_r: the value named 'dot pair hi r' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes dot_half_hi_r[pair_idx][1], the value named 'dot half hi r' in this block, equal to 'dot_pair_hi_r[pair_idx][2] + dot_pair_hi_r[pair_idx][3]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dot_pair_hi_r: the value named 'dot pair hi r' in this block; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 724: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 725: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 726: `if (dot_half_valid_r) begin` This condition selects the following hardware branch only when ‘dot_half_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are dot_half_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 727: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 728: `dot_sum_lo_r[pair_idx] <= {2'd0, dot_half_lo_r[pair_idx][0]}` This statement continues through source line 729; the explanation uses the complete expression rather than only the first visual fragment. This makes dot_sum_lo_r[pair_idx], the value named ’dot sum lo r’ in this block, equal to ‘{2’d0, dot_half_lo_r[pair_idx][0]} + {2’d0, dot_half_lo_r[pair_idx][1]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; dot_half_lo_r: the value named ’dot half lo r’ in this block; pair_idx: the value named ’pair idx’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 729: `+ {2'd0, dot_half_lo_r[pair_idx][1]};` This is continuation line 2 of the 2-line logical statement that begins at source line 728. Its exact fragment is ‘+ {2’d0, dot_half_lo_r[pair_idx][1]};‘. The important operands are d0: the value named ’d0’ in this block; dot_half_lo_r: the value named ’dot half lo r’ in this block; pair_idx: the value named ’pair idx’ in this block. Read together, the complete operation is: This makes dot_sum_lo_r[pair_idx], the value named ’dot sum lo r’ in this block, equal to ‘{2’d0, dot_half_lo_r[pair_idx][0]} + {2’d0, dot_half_lo_r[pair_idx][1]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; dot_half_lo_r: the value named ’dot half lo r’ in this block; pair_idx: the value named ’pair idx’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 730: `dot_sum_hi_r[pair_idx] <= {2'd0, dot_half_hi_r[pair_idx][0]}` This statement continues through source line 731; the explanation uses the complete expression rather than only the first visual fragment. This makes dot_sum_hi_r[pair_idx], the value named ’dot sum hi r’ in this block, equal to ‘{2’d0, dot_half_hi_r[pair_idx][0]} + {2’d0, dot_half_hi_r[pair_idx][1]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; dot_half_hi_r: the value named ’dot half hi r’ in this block; pair_idx: the value named ’pair idx’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 731: `+ {2'd0, dot_half_hi_r[pair_idx][1]};` This is continuation line 2 of the 2-line logical statement that begins at source line 730. Its exact fragment is ‘+ {2’d0, dot_half_hi_r[pair_idx][1]};‘. The important operands are d0: the value named ’d0’ in this block; dot_half_hi_r: the value named ’dot half hi r’ in this block; pair_idx: the value named ’pair idx’ in this block. Read together, the complete operation is: This makes dot_sum_hi_r[pair_idx], the value named ’dot sum hi r’ in this block, equal to ‘{2’d0, dot_half_hi_r[pair_idx][0]} + {2’d0, dot_half_hi_r[pair_idx][1]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; dot_half_hi_r: the value named ’dot half hi r’ in this block; pair_idx: the value named ’pair idx’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 732: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 733: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 735: `if (sparse_prepare_valid_r) begin` This condition selects the following hardware branch only when ‘sparse_prepare_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sparse_prepare_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 736: `sparse_delta_valid_r <= 1'b1;` This makes sparse_delta_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 737: `sparse_active_count_r <= popcount8(sparse_prepare_active_r);` This makes sparse_active_count_r, an integer count retained for control or measurable telemetry, equal to ‘popcount8(sparse_prepare_active_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are popcount8: the value named ’popcount8’ in this block; sparse_prepare_active_r: the value named ’sparse prepare active r’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 738: `sparse_metric_valid_r <= 1'b1;` This makes sparse_metric_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 739: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 740: `for (int column_idx = 0; column_idx < COLUMN_LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 741: `column_idx++) begin` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are column_idx: one output-feature position processed in parallel.

Line 742: `logic signed [7:0] signed_weight;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge. Specifically, signed_weight is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 743: `logic signed [8:0] extended_weight;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge. Specifically, extended_weight is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 744: `signed_weight = $signed(sparse_prepare_weight_vector_r[` This statement continues through source line 745; the explanation uses the complete expression rather than only the first visual fragment. This makes signed_weight, a learned coefficient multiplied by an activation, equal to ‘$signed(sparse_prepare_weight_vector_r[column_idx*8 +: 8])‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are sparse_prepare_weight_vector_r: a learned coefficient multiplied by an activation; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 745: `column_idx*8 +: 8]);` This is continuation line 2 of the 2-line logical statement that begins at source line 744. Its exact fragment is ‘column_idx*8 +: 8]);‘. The important operands are column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes signed_weight, a learned coefficient multiplied by an activation, equal to ‘$signed(sparse_prepare_weight_vector_r[column_idx*8 +: 8])‘. It is driven by combinational

or procedural logic in the current evaluation. The important operands are sparse_prepare_weight_vector_r: a learned coefficient multiplied by an activation; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 746: `extended_weight = {signed_weight[7], signed_weight};` This makes extended_weight, a learned coefficient multiplied by an activation, equal to '{signed_weight[7], signed_weight}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are signed_weight: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 747: `if (sparse_prepare_active_r[row_idx]) begin` This condition selects the following hardware branch only when 'sparse_prepare_active_r[row_idx]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sparse_prepare_active_r: the value named 'sparse prepare active r' in this block; row_idx: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge.

Line 748: `sparse_delta_r[row_idx][column_idx] <=` This statement continues through source line 750; the explanation uses the complete expression rather than only the first visual fragment. This makes sparse_delta_r[row_idx][column_idx], the value named 'sparse delta r' in this block, equal to 'sparse_prepare_negative_r[row_idx] ? -extended_weight : extended_weight'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sparse_prepare_negative_r: the value named 'sparse prepare negative r' in this block; row_idx: one output-row position processed by the tensor tile; extended_weight: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 749: `sparse_prepare_negative_r[row_idx]` This is continuation line 2 of the 3-line logical statement that begins at source line 748. Its exact fragment is 'sparse_prepare_negative_r[row_idx]'. The important operands are sparse_prepare_negative_r: the value named 'sparse prepare negative r' in this block; row_idx: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes sparse_delta_r[row_idx][column_idx], the value named 'sparse delta r' in this block, equal to 'sparse_prepare_negative_r[row_idx] ? -extended_weight : extended_weight'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sparse_prepare_negative_r: the value named 'sparse prepare negative r' in this block; row_idx: one output-row position processed by the tensor tile; extended_weight: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 750: `? -extended_weight : extended_weight;` This is continuation line 3 of the 3-line logical statement that begins at source line 748. Its exact fragment is '? -extended_weight : extended_weight;'. The important operands are extended_weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes sparse_delta_r[row_idx][column_idx], the value named 'sparse delta r' in this block, equal to 'sparse_prepare_negative_r[row_idx] ? -extended_weight : extended_weight'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sparse_prepare_negative_r: the value named 'sparse prepare negative r' in this block; row_idx: one output-row position processed by the tensor tile; extended_weight: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge.

Line 751: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 752: `sparse_delta_r[row_idx][column_idx] <= 9'sd0;` This makes sparse_delta_r[row_idx][column_idx], the value named 'sparse delta r' in this block, equal to '9'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 753: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 754: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 755: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 756: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 758: `if (dense_accumulate_valid) begin` This condition selects the following hardware branch only when ‘dense_accumulate_valid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are dense_accumulate_valid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 759: `mac_retire_count <= mac_retire_count + 1'b1;` This makes mac_retire_count, an integer count retained for control or measurable telemetry, equal to ‘mac_retire_count + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_retire_count: an integer count retained for control or measurable telemetry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 760: `debug_retired_issues <= debug_retired_issues + 1'b1;` This makes debug_retired_issues, the value named ’debug retired issues’ in this block, equal to ‘debug_retired_issues + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are debug_retired_issues: the value named ’debug retired issues’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 761: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 762: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 763: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 764: `if (accumulator_clear_row_r[row_idx]) begin` This condition selects the following hardware branch only when ‘accumulator_clear_row_r[row_idx]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are accumulator_clear_row_r: a widened running sum that prevents intermediate overflow; row_idx: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge.

Line 765: `accumulator[row_idx][column_idx] <= '0;` This makes accumulator[row_idx][column_idx], a widened running sum that prevents intermediate overflow, equal to ‘’0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 766: `end else if (accumulate_delta_valid_r) begin` This condition selects the following hardware branch only when ‘accumulate_delta_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are accumulate_delta_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 767: `accumulator[row_idx][column_idx] <=` This statement continues through source line 770; the explanation uses the complete expression rather than only the first visual fragment. This makes accumulator[row_idx][column_idx], a widened running sum that prevents intermediate overflow, equal to 'accumulator[row_idx][column_idx] + {{(ACC_BITS-21){accumulate_delta_r[row_idx][column_idx][20]}}, accumulate_delta_r[row_idx][column_idx]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are accumulator: a widened running sum that prevents intermediate overflow; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 768: `accumulator[row_idx][column_idx]` This is continuation line 2 of the 4-line logical statement that begins at source line 767. Its exact fragment is 'accumulator[row_idx][column_idx]'. The important operands are accumulator: a widened running sum that prevents intermediate overflow; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes accumulator[row_idx][column_idx], a widened running sum that prevents intermediate overflow, equal to 'accumulator[row_idx][column_idx] + {{(ACC_BITS-21){accumulate_delta_r[row_idx][column_idx][20]}}, accumulate_delta_r[row_idx][column_idx]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are accumulator: a widened running sum that prevents intermediate overflow; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 769: `+ {{(ACC_BITS-21){accumulate_delta_r[row_idx][column_idx][20]}},` This is continuation line 3 of the 4-line logical statement that begins at source line 767. Its exact fragment is '+ {{(ACC_BITS-21){accumulate_delta_r[row_idx] The important operands are ACC_BITS: the value named 'acc bits' in this block; accumulate_delta_r: the value named 'accumulate delta r' in this block; row_idx: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes accumulator[row_idx][column_idx], a widened running sum that prevents intermediate overflow, equal to 'accumulator[row_idx][column_idx] + {{(ACC_BITS-21){accumulate_delta_r[row_idx][column_idx][20]}}, accumulate_delta_r[row_idx][column_idx]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are accumulator: a widened running sum that prevents intermediate overflow; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 770: `accumulate_delta_r[row_idx][column_idx]};` This is continuation line 4 of the 4-line logical statement that begins at source line 767. Its exact fragment is 'accumulate_delta_r[row_idx][column_idx]};'. The important operands are accumulate_delta_r: the value named 'accumulate delta r' in this block; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes accumulator[row_idx][column_idx], a widened running sum that prevents intermediate overflow, equal to 'accumulator[row_idx][column_idx] + {{(ACC_BITS-21){accumulate_delta_r[row_idx][column_idx][20]}}, accumulate_delta_r[row_idx][column_idx]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are accumulator: a widened running sum that prevents intermediate overflow; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 771: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 772: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 773: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 774: `if (dense_outer_tag2_r) begin` This condition selects the following hardware branch only when 'dense_outer_tag2_ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are dense_outer_tag2_r: the value named 'dense outer tag2 r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 775: `accumulate_delta_valid_r <= 1'b1;` This makes accumulate_delta_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 776: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 777: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 778: `accumulate_delta_r[row_idx][2*pair_idx] <=` This statement continues through source line 779; the explanation uses the complete expression rather than only the first visual fragment. Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulate_delta_r: the value named 'accumulate delta r' in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 779: `$signed({5'd0, mac_product_lo[row_idx][pair_idx]});` This is continuation line 2 of the 2-line logical statement that begins at source line 778. Its exact fragment is '$signed({5'd0, mac_product_lo[row_idx][pair_idx]});'. The important operands are d0: the value named 'd0' in this block; mac_product_lo: the value named 'mac product lo' in this block; row_idx: one output-row position processed by the tensor tile. Read together, the complete operation is: Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulate_delta_r: the value named 'accumulate delta r' in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 780: `accumulate_delta_r[row_idx][2*pair_idx+1] <=` This statement continues through source line 781; the explanation uses the complete expression rather than only the first visual fragment. Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulate_delta_r: the value named 'accumulate delta r' in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 781: `$signed({5'd0, mac_product_hi[row_idx][pair_idx]});` This is continuation line 2 of the 2-line logical statement that begins at source line 780. Its exact fragment is '$signed({5'd0, mac_product_hi[row_idx][pair_idx]});'. The important operands are d0: the value named 'd0' in this block; mac_product_hi: the value named 'mac product hi' in this block; row_idx: one output-row position processed by the tensor tile. Read together, the complete operation is: Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulate_delta_r: the value named 'accumulate delta r' in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 782: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 783: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 784: `end else if (dot_sum_valid_r) begin` This condition selects the following hardware branch only when ‘dot_sum_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are dot_sum_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 785: `accumulate_delta_valid_r <= 1'b1;` This makes accumulate_delta_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 786: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 787: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 788: `accumulate_delta_r[row_idx][2*pair_idx] <=` This statement continues through source line 790; the explanation uses the complete expression rather than only the first visual fragment. Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulate_delta_r: the value named ’accumulate delta r’ in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named ’pair idx’ in this block.

Line 789: `row_idx == 0` This is continuation line 2 of the 3-line logical statement that begins at source line 788. Its exact fragment is ‘row_idx == 0‘. The important operands are row_idx: one output-row position processed by the tensor tile. Read together, the complete operation is: Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulate_delta_r: the value named ’accumulate delta r’ in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named ’pair idx’ in this block.

Line 790: `? $signed({1'b0, dot_sum_lo_r[pair_idx]}) : 21'sd0;` This is continuation line 3 of the 3-line logical statement that begins at source line 788. Its exact fragment is ‘? $signed({1’b0, dot_sum_lo_r[pair_idx]}) : 21’sd0;‘. The important operands are b0: the value named ’b0’ in this block; dot_sum_lo_r: the value named ’dot sum lo r’ in this block; pair_idx: the value named ’pair idx’ in this block. Read together, the complete operation is: Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulate_delta_r: the value named ’accumulate delta r’ in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named ’pair idx’ in this block.

Line 791: `accumulate_delta_r[row_idx][2*pair_idx+1] <=` This statement continues through source line 793; the explanation uses the complete expression rather than only the first visual fragment. Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulate_delta_r: the value named ’accumulate delta r’ in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named ’pair idx’ in this block.

Line 792: `row_idx == 0` This is continuation line 2 of the 3-line logical statement that begins at source line 791. Its exact fragment is ‘row_idx == 0‘. The important operands are row_idx: one output-row position processed by the tensor tile. Read together, the complete operation is: Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary,

and ternary arithmetic array. The important operands are accumulate_delta_r: the value named 'accumulate delta r' in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 793: `? $signed({1'b0, dot_sum_hi_r[pair_idx]}) : 21'sd0;` This is continuation line 3 of the 3-line logical statement that begins at source line 791. Its exact fragment is '? $signed({1'b0, dot_sum_hi_r[pair_idx]}) : 21'sd0;'. The important operands are b0: the value named 'b0' in this block; dot_sum_hi_r: the value named 'dot sum hi r' in this block; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are accumulate_delta_r: the value named 'accumulate delta r' in this block; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 794: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 795: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 796: `end else if (sparse_delta_valid_r) begin` This condition selects the following hardware branch only when 'sparse_delta_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sparse_delta_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 797: `accumulate_delta_valid_r <= 1'b1;` This makes accumulate_delta_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 798: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 799: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 800: `accumulate_delta_r[row_idx][column_idx] <=` This statement continues through source line 802; the explanation uses the complete expression rather than only the first visual fragment. This makes accumulate_delta_r[row_idx][column_ the value named 'accumulate delta r' in this block, equal to '{{12{sparse_delta_r[row_idx][column_idx][8]}}, sparse_delta_r[row_i It updates after the active clock edge together with other nonblocking assignments. The important operands are sparse_delta_r: the value named 'sparse delta r' in this block; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 801: `{{12{sparse_delta_r[row_idx][column_idx][8]}},` This is continuation line 2 of the 3-line logical statement that begins at source line 800. Its exact fragment is '{{12{sparse_delta_r[row_idx][column_idx][8]}},'. The important operands are sparse_delta_r: the value named 'sparse delta r' in this block; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes accumulate_delta_r[row_idx][column_idx], the value named 'accumulate delta r' in this block, equal to '{{12{sparse_delta_r[row_idx][column_idx][8]}}, sparse_delta_r[row_idx][column_idx]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sparse_delta_r: the value named 'sparse delta r' in this block; row_idx: one output-row position processed by the tensor tile; column_idx: one

output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 802: `sparse_delta_r[row_idx][column_idx]};` This is continuation line 3 of the 3-line logical statement that begins at source line 800. Its exact fragment is ‘sparse_delta_r[row_idx][column_idx]};‘. The important operands are sparse_delta_r: the value named ’sparse delta r’ in this block; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Read together, the complete operation is: This makes accumulate_delta_r[row_idx][column_idx], the value named ’accumulate delta r’ in this block, equal to ‘{{12{sparse_delta_r[row_idx][column_idx][8]}}, sparse_delta_r[row_idx][column_idx]}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are sparse_delta_r: the value named ’sparse delta r’ in this block; row_idx: one output-row position processed by the tensor tile; column_idx: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge.

Line 803: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 804: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 805: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 807: `unique case (state)` This starts a multiplexer/state decode selected by ‘state‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 808: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 809: `if (start) begin` This condition selects the following hardware branch only when ‘start‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are start: the value named ’start’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 810: `mode_r <= compute_mode;` This makes mode_r, the value named ’mode r’ in this block, equal to ‘compute_mode‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_mode: the value named ’compute mode’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 811: `rows_r <= active_rows;` This makes rows_r, one output-row position processed by the tensor tile, equal to ‘active_rows‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_rows: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 812: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 813: `active_row_mask_r[row_idx] <= row_idx < active_rows;` This makes active_row_mask_r[row_idx], one output-row position processed by the tensor tile, equal to 'row_idx < active_rows'. It updates after the active clock edge together with other nonblocking assignments. The important operands are row_idx: one output-row position processed by the tensor tile; active_rows: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 814: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 815: `columns_r <= active_columns;` This makes columns_r, one output-feature position processed in parallel, equal to 'active_columns'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_columns: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 816: `dense_macs_per_issue_r <= compute_mode == MODE_DENSE_DOT8` This statement continues through source line 820; the explanation uses the complete expression rather than only the first visual fragment. This makes dense_macs_per_issue_ a count of ordinary activation-times-weight operations actually issued, equal to 'compute_mode == MODE_DENSE_DOT8 ? {2'd0, active_columns} * 8'd8 : {4'd0, active_rows} * {2'd0, active_columns}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_mode: the value named 'compute mode' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 817: `? {2'd0, active_columns}` This is continuation line 2 of the 5-line logical statement that begins at source line 816. Its exact fragment is '? {2'd0, active_columns}'. The important operands are d0: the value named 'd0' in this block; active_columns: one output-feature position processed in parallel. Read together, the complete operation is: This makes dense_macs_per_issue_r, a count of ordinary activation-times-weight operations actually issued, equal to 'compute_mode == MODE_DENSE_DOT8 ? {2'd0, active_columns} * 8'd8 : {4'd0, active_rows} * {2'd0, active_columns}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_mode: the value named 'compute mode' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 818: `* 8'd8` This is continuation line 3 of the 5-line logical statement that begins at source line 816. Its exact fragment is '* 8'd8'. The important operands are d8: the value named 'd8' in this block. Read together, the complete operation is: This makes dense_macs_per_issue_r, a count of ordinary activation-times-weight operations actually issued, equal to 'compute_mode == MODE_DENSE_DOT8 ? {2'd0, active_columns} * 8'd8 : {4'd0, active_rows} * {2'd0, active_columns}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_mode: the value named 'compute mode' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 819: `: {4'd0, active_rows}` This is continuation line 4 of the 5-line logical statement that begins at source line 816. Its exact fragment is ': {4'd0, active_rows}'. The important operands are d0: the value named 'd0' in this block; active_rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes dense_macs_per_issue_r, a count of ordinary activation-times-weight operations actually issued, equal to 'compute_mode == MODE_DENSE_DOT8 ? {2'd0, active_columns} * 8'd8 : {4'd0, active_rows} * {2'd0, active_columns}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_mode: the value named 'compute mode' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates reg-

isters only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 820: `* {2'd0, active_columns};` This is continuation line 5 of the 5-line logical statement that begins at source line 816. Its exact fragment is '* {2'd0, active_columns};'. The important operands are d0: the value named 'd0' in this block; active_columns: one output-feature position processed in parallel. Read together, the complete operation is: This makes dense_macs_per_issue_r, a count of ordinary activation-times-weight operations actually issued, equal to 'compute_mode == MODE_DENSE_DOT8 ? {2'd0, active_columns} * 8'd8 : {4'd0, active_rows} * {2'd0, active_columns}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are compute_mode: the value named 'compute mode' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 821: `activation_zp_r <= activation_zero_point;` This makes activation_zp_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'activation_zero_point'. It updates after the active clock edge together with other nonblocking assignments. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 822: `weight_zp_r <= weight_zero_point;` This makes weight_zp_r, a learned coefficient multiplied by an activation, equal to 'weight_zero_point'. It updates after the active clock edge together with other nonblocking assignments. The important operands are weight_zero_point: a learned coefficient multiplied by an activation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 823: `config_error <= active_rows == 0` This statement continues through source line 827; the explanation uses the complete expression rather than only the first visual fragment. This makes config_error, the value named 'config error' in this block, equal to 'active_rows == 0 || active_rows > 4'(ROW_LANES) || active_columns == 0 || active_columns > 6'(COLUMN_LANES) || (compute_mode == MODE_DENSE_DOT8 && active_rows != 1)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_rows: one output-row position processed by the tensor tile; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; active_columns: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 824: `|| active_rows > 4'(ROW_LANES)` This is continuation line 2 of the 5-line logical statement that begins at source line 823. Its exact fragment is '|| active_rows > 4'(ROW_LANES)'. The important operands are active_rows: one output-row position processed by the tensor tile; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes config_error, the value named 'config error' in this block, equal to 'active_rows == 0 || active_rows > 4'(ROW_LANES) || active_columns == 0 || active_columns > 6'(COLUMN_LANES) || (compute_mode == MODE_DENSE_DOT8 && active_rows != 1)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_rows: one output-row position processed by the tensor tile; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; active_columns: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 825: `|| active_columns == 0` This is continuation line 3 of the 5-line logical statement that begins at source line 823. Its exact fragment is '|| active_columns == 0'. The important operands are active_columns: one output-feature position processed in parallel. Read together, the complete operation is: This makes config_error, the value named 'config error' in this block, equal to 'active_rows == 0 || active_rows > 4'(ROW_LANES) || active_columns == 0 || active_columns > 6'(COLUMN_LANES) || (compute_mode == MODE_DENSE_DOT8 && active_rows != 1)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_rows: one output-row position processed by the tensor tile; ROW_LANES: one parallel arithmetic element within

a vector or tensor datapath; active_columns: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 826: `|| active_columns > 6'(COLUMN_LANES)` This is continuation line 4 of the 5-line logical statement that begins at source line 823. Its exact fragment is '|| active_columns > 6'(COLUMN_LANES)'. The important operands are active_columns: one output-feature position processed in parallel; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes config_error, the value named 'config error' in this block, equal to 'active_rows == 0 || active_rows > 4'(ROW_LANES) || active_columns == 0 || active_columns > 6'(COLUMN_LANES) || (compute_mode == MODE_DENSE_DOT8 && active_rows != 1)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_rows: one output-row position processed by the tensor tile; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; active_columns: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 827: `|| (compute_mode == MODE_DENSE_DOT8 && active_rows != 1);` This is continuation line 5 of the 5-line logical statement that begins at source line 823. Its exact fragment is '|| (compute_mode == MODE_DENSE_DOT8 && active_rows != 1);'. The important operands are compute_mode: the value named 'compute mode' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; active_rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes config_error, the value named 'config error' in this block, equal to 'active_rows == 0 || active_rows > 4'(ROW_LANES) || active_columns == 0 || active_columns > 6'(COLUMN_LANES) || (compute_mode == MODE_DENSE_DOT8 && active_rows != 1)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are active_rows: one output-row position processed by the tensor tile; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; active_columns: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 828: `effective_reduction <= 32'd0;` This makes effective_reduction, the value named 'effective reduction' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 829: `mac_issue_count <= 12'd0;` This makes mac_issue_count, an integer count retained for control or measurable telemetry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 830: `mac_retire_count <= 12'd0;` This makes mac_retire_count, an integer count retained for control or measurable telemetry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 831: `target_mac_retires <= 12'd0;` This makes target_mac_retires, the value named 'target mac retires' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 832: `drain_complete_r <= 1'b0;` This makes drain_complete_r, the value named 'drain complete r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important

operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 833: `zero_restore_reduction_operand_r <= 32'd0;` This makes zero_restore_reduction_operand_r, the value named 'zero restore reduction operand r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 834: `zero_restore_zp_operand_r <= 16'd0;` This makes zero_restore_zp_operand_r, the value named 'zero restore zp operand r' in this block, equal to '16'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 835: `zero_point_restore_r <= 48'd0;` This makes zero_point_restore_r, the integer code representing real numeric zero in affine quantization, equal to '48'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 836: `launch_total_r <= 8'd0;` This makes launch_total_r, the value named 'launch total r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 837: `launch_remaining_r <= 8'd0;` This makes launch_remaining_r, the value named 'launch remaining r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 838: `launch_rows_remaining_r <= 4'd0;` This makes launch_rows_remaining_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 839: `launch_columns_remaining_r <= 6'd0;` This makes launch_columns_remaining_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 840: `launch_row_r <= 4'd0;` This makes launch_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 841: `launch_column_r <= 6'd0;` This makes launch_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 842: `pread_valid_r <= 1'b0;` This makes pread_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 843: `p0_valid_r <= 1'b0;` This makes p0_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 844: `p1_valid_r <= 1'b0;` This makes p1_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 845: `p2_valid_r <= 1'b0;` This makes p2_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 846: `p3_valid_r <= 1'b0;` This makes p3_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 847: `p4_valid_r <= 1'b0;` This makes p4_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 848: `p5_valid_r <= 1'b0;` This makes p5_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 849: `result_fifo_count_r <= '0;` This makes result_fifo_count_r, an integer count retained for control or measurable telemetry, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 850: `result_fifo_has_space_r <= 1'b1;` This makes result_fifo_has_space_r, the value named 'result fifo has space r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 851: `result_fifo_nonempty_r <= 1'b0;` This makes result_fifo_nonempty_r, the value named 'result fifo nonempty r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 852: `result_fifo_write_pointer_r <= '0;` This makes result_fifo_write_pointer_r, the value named 'result fifo write pointer r' in this block, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 853: `result_fifo_read_pointer_r <= '0;` This makes result_fifo_read_pointer_r, the value named 'result fifo read pointer r' in this block, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 854: `result_fifo_last_address_r <= '0;` This makes result_fifo_last_address_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 855: `result_stage_valid_r <= 1'b0;` This makes result_stage_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 856: `result_stage_payload_r <= '0;` This makes result_stage_payload_r, stream data belonging to the current command, equal to ''0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 857: `result_stage_last_r <= 1'b0;` This makes result_stage_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 858: `dot_sum_valid_r <= 1'b0;` This makes dot_sum_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 859: `dot_pair_valid_r <= 1'b0;` This makes dot_pair_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 860: `dot_half_valid_r <= 1'b0;` This makes dot_half_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 861: `dot8_pair_valid_r <= 1'b0;` This makes dot8_pair_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 862: `dot8_increment_half_valid_r <= 1'b0;` This makes dot8_increment_half_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 863: `sum_update_valid_r <= 1'b0;` This makes sum_update_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 864: `input_done_r <= 1'b0;` This makes input_done_r, the value named 'input done r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 865: `issue_match_valid_r <= 1'b0;` This makes issue_match_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 866: `issue_zero_matches_r <= 8'd0;` This makes issue_zero_matches_r, the value named 'issue zero matches r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 867: `issue_match_last_r <= 1'b0;` This makes issue_match_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 868: `classify_valid_r <= 1'b0;` This makes classify_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 869: `classify_is_zero_r <= 1'b0;` This makes classify_is_zero_r, the value named 'classify is zero r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 870: `classify_last_r <= 1'b0;` This makes classify_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that

updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 871: `dense_issue_enable_r <= 1'b0;` This makes dense_issue_enable_r, a control bit that permits the associated state or storage update, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 872: `dense_outer_tag0_r <= 1'b0;` This makes dense_outer_tag0_r, the value named ’dense outer tag0 r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 873: `dense_outer_tag1_r <= 1'b0;` This makes dense_outer_tag1_r, the value named ’dense outer tag1 r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 874: `dense_outer_tag2_r <= 1'b0;` This makes dense_outer_tag2_r, the value named ’dense outer tag2 r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 875: `sparse_prepare_valid_r <= 1'b0;` This makes sparse_prepare_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 876: `sparse_delta_valid_r <= 1'b0;` This makes sparse_delta_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 877: `accumulate_delta_valid_r <= 1'b0;` This makes accumulate_delta_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 878: `debug_accepted_issues <= 32'd0;` This makes debug_accepted_issues, the value named ’debug accepted issues’ in this block, equal to ‘32’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 879: `debug_skipped_issues <= 32'd0;` This makes debug_skipped_issues, the value named ’debug skipped issues’ in this block, equal to ‘32’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers

only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 880: `debug_retired_issues <= 32'd0;` This makes debug_retired_issues, the value named 'debug retired issues' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 881: `debug_dense_macs_r <= 19'd0;` This makes debug_dense_macs_r, a count of ordinary activation-times-weight operations actually issued, equal to '19'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 882: `debug_spike_adds_r <= 19'd0;` This makes debug_spike_adds_r, a binary event produced when a neuron crosses its threshold, equal to '19'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 883: `metric_increment_r <= 8'd0;` This makes metric_increment_r, the value named 'metric increment r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 884: `metric_increment_valid_r <= 1'b0;` This makes metric_increment_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 885: `sparse_active_count_r <= 4'd0;` This makes sparse_active_count_r, an integer count retained for control or measurable telemetry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 886: `sparse_metric_valid_r <= 1'b0;` This makes sparse_metric_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 887: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 888: `accumulator_clear_row_r[row_idx] <= 1'b1;` This makes accumulator_clear_row_r[row_idx], a widened running sum that prevents intermediate overflow, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 889: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 890: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 891: `column_sum_clear_r[column_idx] <= 1'b1;` This makes column_sum_clear_r[column_idx], one output-feature position processed in parallel, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 892: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 893: `state <= ST_RUN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_RUN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_RUN: the value named 'st run' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 894: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 895: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 897: `ST_RUN: begin` This is the ST_RUN case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 898: `if (classify_valid_r) begin` This condition selects the following hardware branch only when ‘classify_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are classify_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 899: `debug_accepted_issues <= debug_accepted_issues + 1'b1;` This makes debug_accepted_issues, the value named 'debug accepted issues' in this block, equal to ‘debug_accepted_issues + 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are debug_accepted_issues: the value named 'debug accepted issues' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 900: `if (classify_is_zero_r) begin` This condition selects the following hardware branch only when ‘classify_is_zero_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are classify_is_zero_r: the value named 'classify is zero r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 901: `debug_skipped_issues <= debug_skipped_issues + 1'b1;` This makes debug_skipped_issues, the value named 'debug skipped issues' in this block, equal to 'debug_skipped_issues + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are debug_skipped_issues: the value named 'debug skipped issues' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 902: `end else if (mode_r == MODE_DENSE_OUTER) begin` This condition selects the following hardware branch only when 'mode_r == MODE_DENSE_OUTER' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are mode_r: the value named 'mode r' in this block; MODE_DENSE_OUTER: the value named 'mode dense outer' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 903: `dense_issue_enable_r <= 1'b1;` This makes dense_issue_enable_r, a control bit that permits the associated state or storage update, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 904: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 905: `// Inactive rows are never emitted. Computing` Design intent: Inactive rows are never emitted. Computing This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 906: `// them removes active_rows from every DSP input` Design intent: them removes active_rows from every DSP input This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 907: `// mux and is therefore both safe and faster.` Design intent: mux and is therefore both safe and faster. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 908: `dense_activation_vector_r[row_idx*8 +: 8]` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dense_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; row_idx: one output-row position processed by the tensor tile.

Line 909: `<= classify_activation_vector_r[row_idx*8 +: 8];` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; row_idx: one output-row position processed by the tensor tile.

Line 910: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 911: `pair_idx++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are pair_idx: the value named 'pair idx' in this block.

Line 912: `// Inactive columns are never retired, so` Design intent: Inactive columns are never retired, so This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 913: `// computing them is harmless. Loading all` Design intent: computing them is harmless. Loading all This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 914: `// lanes removes active_columns from every` Design intent: lanes removes active_columns from every This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 915: `// DSP operand mux and shortens the routed` Design intent: DSP operand mux and shortens the routed This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 916: `// data path without changing any result.` Design intent: data path without changing any result. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 917: `dense_weight_lo_r[row_idx][pair_idx]` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dense_weight_lo_r: a learned coefficient multiplied by an activation; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 918: `<= classify_outer_weight_vector_r[` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are classify_outer_weight_vector_r: a learned coefficient multiplied by an activation.

Line 919: `(2*pair_idx)*8 +: 8];` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are pair_idx: the value named 'pair idx' in this block.

Line 920: `dense_weight_hi_r[row_idx][pair_idx]` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dense_weight_hi_r: a learned coefficient multiplied by an activation; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 921: `<= classify_outer_weight_vector_r[` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are classify_outer_weight_vector_r: a learned coefficient multiplied by an activation.

Line 922: `(2*pair_idx+1)*8 +: 8];` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are pair_idx: the value named 'pair idx' in this block.

Line 923: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 924: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 925: `mac_issue_count <= mac_issue_count + 1'b1;` This makes mac_issue_count, an integer count retained for control or measurable telemetry, equal to 'mac_issue_count + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_issue_count: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 926: `metric_increment_r <= dense_macs_per_issue_r;` This makes metric_increment_r, the value named 'metric increment r' in this block, equal to 'dense_macs_per_issue_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dense_macs_per_issue_r: a count of ordinary activation-times-weight operations actually issued. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 927: `metric_increment_valid_r <= 1'b1;` This makes metric_increment_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 928: `sum_update_valid_r <= 1'b1;` This makes sum_update_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 929: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 930: `row_increment_r[row_idx]` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are row_increment_r: one output-row position processed by the tensor tile; row_idx: one output-row position processed by the tensor tile.

Line 931: `<= {8'd0, classify_activation_vector_r[row_idx*8 +: 8]};` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are d0: the value named 'd0' in this block; classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; row_idx: one output-row position processed by the tensor tile.

Line 932: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 933: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 934: `column_increment_r[column_idx]` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are column_increment_r: one output-feature position processed in parallel; column_idx: one output-feature position processed in parallel.

Line 935: `<= {8'd0, classify_outer_weight_vector_r[column_idx*8 +: 8]};` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are d0: the value named 'd0' in this block; classify_outer_weight_vector_r: a learned coefficient multiplied by an activation; column_idx: one output-feature position processed in parallel.

Line 936: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 937: `end else if (mode_r == MODE_DENSE_DOT8) begin` This condition selects the following hardware branch only when 'mode_r == MODE_DENSE_DOT8' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are mode_r: the value named 'mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 938: `dense_issue_enable_r <= 1'b1;` This makes dense_issue_enable_r, a control bit that permits the associated state or storage update, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 939: `dense_activation_vector_r <= classify_activation_vector_r;` This makes dense_activation_vector_r, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'classify_activation_vector_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 940: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 941: `for (int pair_idx = 0; pair_idx < COLUMN_PAIRS;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 942: `pair_idx++) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are pair_idx: the value named 'pair idx' in this block.

Line 943: `dense_weight_lo_r[row_idx][pair_idx]` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dense_weight_lo_r: a learned coefficient multiplied by an activation; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 944: `<= classify_dot_weight_vector_r[` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are classify_dot_weight_vector_r: a learned coefficient multiplied by an activation.

Line 945: `((2*pair_idx)*ROW_LANES+row_idx)*8 +: 8];` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are pair_idx: the value named 'pair idx' in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; row_idx: one output-row position processed by the tensor tile.

Line 946: `dense_weight_hi_r[row_idx][pair_idx]` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are dense_weight_hi_r: a learned coefficient multiplied by an activation; row_idx: one output-row position processed by the tensor tile; pair_idx: the value named 'pair idx' in this block.

Line 947: `<= classify_dot_weight_vector_r[` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are classify_dot_weight_vector_r: a learned coefficient multiplied by an activation.

Line 948: `((2*pair_idx+1)*ROW_LANES+row_idx)*8 +: 8];` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are pair_idx: the value named 'pair idx' in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; row_idx: one output-row position processed by the tensor tile.

Line 949: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 950: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 951: `mac_issue_count <= mac_issue_count + 1'b1;` This makes mac_issue_count, an integer count retained for control or measurable telemetry, equal to 'mac_issue_count + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_issue_count: an integer count retained for control or measurable telemetry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 952: `metric_increment_r <= dense_macs_per_issue_r;` This makes metric_increment_r, the value named 'metric increment r' in this block, equal to 'dense_macs_per_issue_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are dense_macs_per_issue_r: a count of ordinary activation-times-weight operations actually issued. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 953: `metric_increment_valid_r <= 1'b1;` This makes metric_increment_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 954: `dot8_pair_valid_r <= 1'b1;` This makes dot8_pair_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 955: `for (int pair_idx = 0; pair_idx < 4; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 956: `dot8_row_pair_r[pair_idx] <=` This statement continues through source line 958; the explanation uses the complete expression rather than only the first visual fragment. This makes dot8_row_pair_r[pair_idx], one output-row position processed by the tensor tile, equal to 'classify_activation_vector_r[(2*pair_idx)*8 +: 8] + classify_activation_vector_r[(2 +: 8]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 957: `classify_activation_vector_r[(2*pair_idx)*8 +: 8]` This is continuation line 2 of the 3-line logical statement that begins at source line 956. Its exact fragment is 'classify_activation_vector_r[(2*pair_idx)*8 +: 8]'. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes dot8_row_pair_r[pair_idx], one output-row position processed by the tensor tile, equal to 'classify_activation_vector_r[(2*p +: 8] + classify_activation_vector_r[(2*pair_idx+1)*8 +: 8]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 958: `+ classify_activation_vector_r[(2*pair_idx+1)*8 +: 8];` This is continuation line 3 of the 3-line logical statement that begins at source line 956. Its exact fragment is '+ classify_activation_vector_r[(2*pair_idx+1)*8 +: 8];'. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; pair_idx: the value named 'pair idx' in this block. Read together, the complete operation is: This makes dot8_row_pair_r[pair_idx], one output-row position processed by the tensor tile, equal to 'classify_activation_vector_r[(2*pair_idx)*8 +: 8] + classify_activation_vector_r[(2*pair_idx+1)*8 +: 8]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; pair_idx: the value named 'pair idx' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 959: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 960: `for (int column_idx = 0; column_idx < COLUMN_LANES; column_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 961: `for (int pair_idx = 0; pair_idx < 4; pair_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 962: `dot8_column_pair_r[column_idx][pair_idx] <=` This statement continues through source line 966; the explanation uses the complete expression rather than only the first visual fragment. This makes dot8_column_pair_r[column_idx][p one output-feature position processed in parallel, equal to ‘classify_dot_weight_vector_r[(column_idx*8+2*pair_idx)*8 +: 8] + classify_dot_weight_vector_r[(column_idx*8+2*pair_idx+1)*8 +: 8]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are classify_dot_weight_vector_r: a learned coefficient multiplied by an activation; column_idx: one output-feature position processed in parallel; pair_idx: the value named ’pair idx’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 963: `classify_dot_weight_vector_r[` This is continuation line 2 of the 5-line logical statement that begins at source line 962. Its exact fragment is ‘classify_dot_weight_vector_r[‘. The important operands are classify_dot_weight_vector_r: a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes dot8_column_pair_r[column_idx][pair_idx], one output-feature position processed in parallel, equal to ‘classify_dot_weight_vector_r[(column_idx*8+2*pair_idx)*8 +: 8] + classify_dot_weight_vector_r[(column_idx*8+2*pair_idx+1)*8 +: 8]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are classify_dot_weight_vector_r: a learned coefficient multiplied by an activation; column_idx: one output-feature position processed in parallel; pair_idx: the value named ’pair idx’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 964: `(column_idx*8+2*pair_idx)*8 +: 8]` This is continuation line 3 of the 5-line logical statement that begins at source line 962. Its exact fragment is ‘(column_idx*8+2*pair_idx)*8 +: 8]‘. The important operands are column_idx: one output-feature position processed in parallel; pair_idx: the value named ’pair idx’ in this block. Read together, the complete operation is: This makes dot8_column_pair_r[column_idx][pair_idx], one output-feature position processed in parallel, equal to ‘classify_dot_weight_vector_r[(column_idx*8+2*pair_idx)*8 +: 8] + classify_dot_weight_vector_r[(column_idx*8+2*pair_idx+1)*8 +: 8]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are classify_dot_weight_vector_r: a learned coefficient multiplied by an activation; column_idx: one output-feature position processed in parallel; pair_idx: the value named ’pair idx’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 965: `+ classify_dot_weight_vector_r[` This is continuation line 4 of the 5-line logical statement that begins at source line 962. Its exact fragment is ‘+ classify_dot_weight_vector_r[‘. The important operands are classify_dot_weight_vector_r: a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes dot8_column_pair_r[column_idx][pair_idx], one output-feature position processed in parallel, equal to ‘classify_dot_weight_vector_r[(column_idx*8+2*pair_idx)*8 +: 8] + classify_dot_weight_vector_r[(column_idx*8+2*pair_idx+1)*8 +: 8]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are classify_dot_weight_vector_r: a learned coefficient multiplied by an activation; column_idx: one output-feature position processed in parallel; pair_idx: the value named ’pair idx’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 966: `(column_idx*8+2*pair_idx+1)*8 +: 8];` This is continuation line 5 of the 5-line logical statement that begins at source line 962. Its exact fragment is ‘(column_idx*8+2*pair_idx+1)*8 +: 8];‘. The important operands are column_idx: one output-feature position processed in parallel; pair_idx: the value named ’pair idx’ in this block. Read together, the complete operation is: This makes dot8_column_pair_r[column_idx][pair_idx], one output-feature position processed in parallel, equal to ‘classify_dot_weight_vector_r[(column_idx*8+2*pair_idx)*8 +: 8] + classify_dot_weight_vector_r[(column_idx*8+2*pair_idx+1)*8 +: 8]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are classify_dot_weight_vector_r: a learned coefficient multiplied by an activation; column_idx: one output-feature position processed in parallel; pair_idx: the value named ’pair idx’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 967: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 968: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 969: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 970: `sparse_prepare_valid_r <= 1'b1;` This makes sparse_prepare_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 971: `sparse_prepare_weight_vector_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are sparse_prepare_weight_vector_r: a learned coefficient multiplied by an activation.

Line 972: `<= classify_outer_weight_vector_r;` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are classify_outer_weight_vector_r: a learned coefficient multiplied by an activation.

Line 973: `for (int row_idx = 0; row_idx < ROW_LANES; row_idx++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 974: `logic signed [7:0] spike_value;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.. Specifically, spike_value is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 975: `spike_value = $signed(` This statement continues through source line 976; the explanation uses the complete expression rather than only the first visual fragment. This makes spike_value, a binary event produced when a neuron crosses its threshold, equal to '$signed(classify_activation_vector_r[row_idx*8 +: 8])'. It is driven by combinational or procedural logic in the current evaluation. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; row_idx: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 976: `classify_activation_vector_r[row_idx*8 +: 8]);` This is continuation line 2 of the 2-line logical statement that begins at source line 975. Its exact fragment is 'classify_activation_vector_r[row_idx*8 +: 8]);'. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; row_idx: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes spike_value, a binary event produced when a neuron crosses its threshold, equal to '$signed(classify_activation_vector_r[row_idx*8 +: 8])'. It is driven by combinational or procedural logic in the current evaluation. The important operands are classify_activation_vector_r: an input feature value consumed by a neuron or multiply-accumulate operation; row_idx: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 977: `sparse_prepare_active_r[row_idx]` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important

operands are sparse_prepare_active_r: the value named 'sparse prepare active r' in this block; row_idx: one output-row position processed by the tensor tile.

Line 978: `<= active_row_mask_r[row_idx] && spike_value != 0;` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are active_row_mask_r: one output-row position processed by the tensor tile; row_idx: one output-row position processed by the tensor tile; spike_value: a binary event produced when a neuron crosses its threshold.

Line 979: `sparse_prepare_negative_r[row_idx]` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are sparse_prepare_negative_r: the value named 'sparse prepare negative r' in this block; row_idx: one output-row position processed by the tensor tile.

Line 980: `<= mode_r == MODE_TERNARY_WEIGHTED` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are mode_r: the value named 'mode r' in this block; MODE_TERNARY_WEIGHTED: a learned coefficient multiplied by an activation.

Line 981: `&& spike_value < 0;` Within the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are spike_value: a binary event produced when a neuron crosses its threshold.

Line 982: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 983: `debug_retired_issues <= debug_retired_issues + 1'b1;` This makes debug_retired_issues, the value named 'debug retired issues' in this block, equal to 'debug_retired_issues + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are debug_retired_issues: the value named 'debug retired issues' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 984: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 986: `if (classify_last_r) begin` This condition selects the following hardware branch only when 'classify_last_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are classify_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 987: `launch_row_r <= 4'd0;` This makes launch_row_r, one output-row position processed by the tensor tile, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 988: `launch_column_r <= 6'd0;` This makes launch_column_r, one output-feature position processed in parallel, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 989: `pread_valid_r <= 1'b0;` This makes pread_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 990: `p0_valid_r <= 1'b0;` This makes p0_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 991: `p1_valid_r <= 1'b0;` This makes p1_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 992: `p2_valid_r <= 1'b0;` This makes p2_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 993: `p3_valid_r <= 1'b0;` This makes p3_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 994: `p4_valid_r <= 1'b0;` This makes p4_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 995: `p5_valid_r <= 1'b0;` This makes p5_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 996: `if (mode_r == MODE_DENSE_OUTER || mode_r == MODE_DENSE_DOT8) begin` This condition selects the following hardware branch only when 'mode_r == MODE_DENSE_OUTER || mode_r == MODE_DENSE_DOT8' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are mode_r: the value named 'mode r' in this block; MODE_DENSE_OUTER: the value named 'mode dense outer' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 997: `state <= ST_CAPTURE_TARGET;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_CAPTURE_TARGET'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_CAPTURE_TARGET: the value named 'st capture target' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 998: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 999: `state <= ST_SPARSE_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_SPARSE_DRAIN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SPARSE_DRAIN: the value named ’st sparse drain’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 1000: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1001: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1002: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1004: `if (issue_accept) begin` This condition selects the following hardware branch only when ‘issue_accept‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are issue_accept: the value named ’issue accept’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 1005: `if (issue_last) input_done_r <= 1'b1;` This condition selects the following hardware branch only when ‘issue_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are issue_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase..

Line 1006: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1007: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1009: `ST_DRAIN: begin` This is the ST_DRAIN case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DRAIN, the machine performs the named control phase..

Line 1010: `// End the 12-bit equality at one local register before it` Design intent: End the 12-bit equality at one local register before it This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1011: `// controls the physically replicated state network.` Design intent: controls the physically replicated state network. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1012: `drain_complete_r <= mac_retire_count == target_mac_retires;` This makes drain_complete_r, the value named 'drain complete r' in this block, equal to 'mac_retire_count == target_mac_retires'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_retire_count: an integer count retained for control or measurable telemetry; target_mac_retires: the value named 'target mac retires' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DRAIN, the machine performs the named control phase..

Line 1013: `if (drain_complete_r) begin` This condition selects the following hardware branch only when 'drain_complete_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are drain_complete_r: the value named 'drain complete r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DRAIN, the machine performs the named control phase..

Line 1014: `state <= ST_PREPARE_ZERO_POINT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_PREPARE_ZERO_POINT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PREPARE_ZERO_POINT: the integer code representing real numeric zero in affine quantization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DRAIN, the machine performs the named control phase..

Line 1015: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DRAIN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1016: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DRAIN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1018: `ST_CAPTURE_TARGET: begin` This is the ST_CAPTURE_TARGET case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE_TARGET, the machine performs the named control phase..

Line 1019: `target_mac_retires <= mac_issue_count;` This makes target_mac_retires, the value named 'target mac retires' in this block, equal to 'mac_issue_count'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mac_issue_count: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE_TARGET, the machine performs the named control phase..

Line 1020: `effective_reduction <= mode_r == MODE_DENSE_DOT8` This statement continues through source line 1022; the explanation uses the complete expression rather than only the first visual fragment. This makes effective_reduction, the value named 'effective reduction' in this block, equal to 'mode_r == MODE_DENSE_DOT8 ? {17'd0, mac_issue_count, 3'd0} : {20'd0, mac_issue_count}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mode_r: the value named 'mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE_TARGET, the machine performs the named control phase..

Line 1021: `? {17'd0, mac_issue_count, 3'd0}` This is continuation line 2 of the 3-line logical statement that begins at source line 1020. Its exact fragment is '? {17'd0, mac_issue_count, 3'd0}'. The important operands are d0: the value named 'd0' in this block; mac_issue_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This makes effective_reduction, the value named 'effective reduction' in this block, equal to 'mode_r == MODE_DENSE_DOT8 ? {17'd0, mac_issue_count, 3'd0} : {20'd0, mac_issue_count}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mode_r: the value named 'mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE_TARGET, the machine performs the named control phase..

Line 1022: `: {20'd0, mac_issue_count};` This is continuation line 3 of the 3-line logical statement that begins at source line 1020. Its exact fragment is ': {20'd0, mac_issue_count};'. The important operands are d0: the value named 'd0' in this block; mac_issue_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This makes effective_reduction, the value named 'effective reduction' in this block, equal to 'mode_r == MODE_DENSE_DOT8 ? {17'd0, mac_issue_count, 3'd0} : {20'd0, mac_issue_count}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are mode_r: the value named 'mode r' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE_TARGET, the machine performs the named control phase..

Line 1023: `drain_complete_r <= 1'b0;` This makes drain_complete_r, the value named 'drain complete r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE_TARGET, the machine performs the named control phase..

Line 1024: `state <= ST_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_DRAIN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DRAIN: the value named 'st drain' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE_TARGET, the machine performs the named control phase..

Line 1025: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE_TARGET, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1027: `ST_SPARSE_DRAIN: begin` This is the ST_SPARSE_DRAIN case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SPARSE_DRAIN, the machine performs the named control phase..

Line 1028: `state <= ST_PREPARE_OUTPUT_COUNT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_PREPARE_OUTPUT_COUNT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PREPARE_OUTPUT_COUNT: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SPARSE_DRAIN, the machine performs the named control phase..

Line 1029: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SPARSE_DRAIN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1031: `ST_PREPARE_ZERO_POINT: begin` This is the ST_PREPARE_ZERO_POINT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_PREPARE_ZERO_POINT, the machine performs the named control phase..

Line 1032: `state <= ST_LATCH_ZERO_POINT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LATCH_ZERO_POINT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LATCH_ZERO_POINT: the integer code representing real numeric zero in affine quantization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREPARE_ZERO_POINT, the machine performs the named control phase..

Line 1033: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PREPARE_ZERO_POINT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1035: `ST_LATCH_ZERO_POINT: begin` This is the ST_LATCH_ZERO_POINT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LATCH_ZERO_POINT, the machine performs the named control phase..

Line 1036: `state <= ST_COMBINE_ZERO_POINT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_COMBINE_ZERO_POINT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMBINE_ZERO_POINT: the integer code representing real numeric zero in affine quantization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LATCH_ZERO_POINT, the machine performs the named control phase..

Line 1037: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LATCH_ZERO_POINT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1039: `ST_COMBINE_ZERO_POINT: begin` This is the ST_COMBINE_ZERO_POINT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMBINE_ZERO_POINT, the machine performs the named control phase..

Line 1040: `zero_restore_reduction_operand_r <= effective_reduction;` This makes zero_restore_reduction_operand_r, the value named ’zero restore reduction operand r’ in this block, equal to ‘effective_reduction‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are effective_reduction: the value named ’effective reduction’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMBINE_ZERO_POINT, the machine performs the named control phase..

Line 1041: `zero_restore_zp_operand_r <= zero_point_product_r;` This makes zero_restore_zp_operand_r, the value named ’zero restore zp operand r’ in this block, equal to ‘zero_point_product_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_point_product_r: the integer code representing real numeric zero in affine quantization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMBINE_ZERO_POINT, the machine performs the named control phase..

Line 1042: `state <= ST_RESTORE_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_RESTORE_MULTIPLY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_RESTORE_MULTIPLY: the value named ’st restore multiply’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMBINE_ZERO_POINT, the machine performs the named control phase..

Line 1043: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMBINE_ZERO_POINT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1045: `ST_RESTORE_MULTIPLY: begin` This is the ST_RESTORE_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_MULTIPLY, the machine performs the named control phase..

Line 1046: `state <= ST_RESTORE_RELAY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_RESTORE_RELAY‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_RESTORE_RELAY: the value named ’st restore relay’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_MULTIPLY, the machine performs the named control phase..

Line 1047: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1049: `ST_RESTORE_RELAY: begin` This is the ST_RESTORE_RELAY case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_RELAY, the machine performs the named control phase..

Line 1050: `// Isolate the two multiplier PREGs from the reconstruction` Design intent: Isolate the two multiplier PREGs from the reconstruction This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1051: `// adder. Vivado can absorb these registers into the adder's` Design intent: adder. Vivado can absorb these registers into the adder's This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1052: `// DSP input stage, replacing one DSP-to-DSP combinational` Design intent: DSP input stage, replacing one DSP-to-DSP combinational This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1053: `// timing path with two independently timed pipeline paths.` Design intent: timing path with two independently timed pipeline paths. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1054: `zero_restore_low_relay_r <= zero_restore_low_product_r;` This makes zero_restore_low_relay_r, the value named 'zero restore low relay r' in this block, equal to ‘zero_restore_low_product_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_restore_low_product_r: the value named 'zero restore low product r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_RELAY, the machine performs the named control phase..

Line 1055: `zero_restore_high_relay_r <= zero_restore_high_product_r;` This makes zero_restore_high_relay_r, the value named 'zero restore high relay r' in this block, equal to ‘zero_restore_high_product_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_restore_high_product_r: the value named 'zero restore high product r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_RELAY, the machine performs the named control phase..

Line 1056: `state <= ST_RESTORE_ADD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_RESTORE_ADD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_RESTORE_ADD: the value named 'st restore add' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_RELAY, the machine performs the named control phase..

Line 1057: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_RELAY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1059: `ST_RESTORE_ADD: begin` This is the ST_RESTORE_ADD case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_ADD, the machine performs the named control phase..

Line 1060: `zero_point_restore_r <= {16'd0, zero_restore_low_relay_r}` This statement continues through source line 1061; the explanation uses the complete expression rather than only the first visual fragment. This makes zero_point_restore_r, the integer code representing real numeric zero in affine quantization, equal to ‘{16'd0, zero_restore_low_rela + {zero_restore_high_relay_r, 16'd0}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; zero_restore_low_relay_r: the value named 'zero restore low relay r' in this block; zero_restore_high_relay_r: the value named 'zero restore high relay r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_ADD, the machine performs the named control phase..

Line 1061: `+ {zero_restore_high_relay_r, 16'd0};` This is continuation line 2 of the 2-line logical statement that begins at source line 1060. Its exact fragment is '+ {zero_restore_high_relay_r, 16'd0};'. The important operands are zero_restore_high_relay_r: the value named 'zero restore high relay r' in this block; d0: the value named 'd0' in this block. Read together, the complete operation is: This makes zero_point_restore_r, the integer code representing real numeric zero in affine quantization, equal to '{16'd0, zero_restore_low_relay_r} + {zero_restore_high_relay_r, 16'd0}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; zero_restore_low_relay_r: the value named 'zero restore low relay r' in this block; zero_restore_high_relay_r: the value named 'zero restore high relay r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_ADD, the machine performs the named control phase..

Line 1062: `state <= ST_PREPARE_OUTPUT_COUNT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_PREPARE_OUTPUT_COUNT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PREPARE_OUTPUT_COUNT: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_ADD, the machine performs the named control phase..

Line 1063: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_ADD, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1065: `ST_PREPARE_OUTPUT_COUNT: begin` This is the ST_PREPARE_OUTPUT_COUNT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_PREPARE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1066: `launch_total_r <= {2'd0, columns_r};` This makes launch_total_r, the value named 'launch total r' in this block, equal to '{2'd0, columns_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREPARE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1067: `launch_rows_remaining_r <= rows_r;` This makes launch_rows_remaining_r, one output-row position processed by the tensor tile, equal to 'rows_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are rows_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREPARE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1068: `launch_columns_remaining_r <= columns_r;` This makes launch_columns_remaining_r, one output-feature position processed in parallel, equal to 'columns_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREPARE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1069: `state <= ST_ACCUMULATE_OUTPUT_COUNT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ACCUMULATE_OUTPUT_COUNT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ACCUMULATE_OUTPUT_COUNT: an integer count retained for control or measurable telemetry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PREPARE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1070: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PREPARE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1072: `ST_ACCUMULATE_OUTPUT_COUNT: begin` This is the ST_ACCUMULATE_OUTPUT_COUNT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1073: `if (launch_rows_remaining_r <= 4'd1) begin` This statement continues through source line 1074; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when 'launch_rows_remaining_r <= 4'd1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are launch_rows_remaining_r: one output-row position processed by the tensor tile; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1074: `launch_remaining_r <= launch_total_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 1073. Its exact fragment is 'launch_remaining_r <= launch_total_r;'. The important operands are launch_remaining_r: the value named 'launch remaining r' in this block; launch_total_r: the value named 'launch total r' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'launch_rows_remaining_r <= 4'd1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are launch_rows_remaining_r: one output-row position processed by the tensor tile; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1075: `// Output addresses start at zero. Truncation is exact` Design intent: Output addresses start at zero. Truncation is exact This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1076: `// for the 128-result case: 128 mod 128 minus one is 127.` Design intent: for the 128-result case: 128 mod 128 minus one is 127. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1077: `result_fifo_last_address_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are result_fifo_last_address_r: the AXI-Stream marker for the final beat of a packet or command payload.

Line 1078: `<= launch_total_r[RESULT_FIFO_ADDRESS_BITS-1:0] - 1'b1;` Within the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array. The important operands are launch_total_r: the value named 'launch total r' in this block; RESULT_FIFO_ADDRESS_BITS: the value named 'result fifo address bits' in this block; b1: the value named 'b1' in this block.

Line 1079: `state <= ST_OUTPUT_PIPELINE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_PIPELINE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_PIPELINE: the value named 'st output pipeline' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1080: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1081: `launch_total_r <= launch_total_r` This statement continues through source line 1082; the explanation uses the complete expression rather than only the first visual fragment. This makes launch_total_r, the value named 'launch total r' in this block, equal to 'launch_total_r + {2'd0, columns_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_total_r: the value named 'launch total r' in this block; d0: the value named 'd0' in this block; columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1082: `+ {2'd0, columns_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 1081. Its exact fragment is '+ {2'd0, columns_r};'. The important operands are d0: the value named 'd0' in this block; columns_r: one output-feature position processed in parallel. Read together, the complete operation is: This makes launch_total_r, the value named 'launch total r' in this block, equal to 'launch_total_r + {2'd0, columns_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_total_r: the value named 'launch total r' in this block; d0: the value named 'd0' in this block; columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1083: `launch_rows_remaining_r <= launch_rows_remaining_r - 1'b1;` This makes launch_rows_remaining_r, one output-row position processed by the tensor tile, equal to 'launch_rows_remaining_r - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_rows_remaining_r: one output-row position processed by the tensor tile; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure..

Line 1084: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1085: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ACCUMULATE_OUTPUT_COUNT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1087: `ST_OUTPUT_PIPELINE: begin` This is the ST_OUTPUT_PIPELINE case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1088: `if (output_pipeline_enable) begin` This condition selects the following hardware branch only when 'output_pipeline_enable' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_pipeline_enable: a control bit that permits the associated state or storage update. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1089: `p5_valid_r <= p4_valid_r;` This makes p5_valid_r, the producer's statement that its current payload is meaningful, equal to 'p4_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p4_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1090: `if (p4_valid_r) begin` This condition selects the following hardware branch only when 'p4_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are p4_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1091: `p5_result_r <= 48'(p4_corrected_r` This statement continues through source line 1092; the explanation uses the complete expression rather than only the first visual fragment. This makes p5_result_r, the value named 'p5 result r' in this block, equal to '48'(p4_corrected_r + $signed({16'd0, zero_point_restore_r}))'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p4_corrected_r: the value named 'p4 corrected r' in this block; d0: the value named 'd0' in this block; zero_point_restore_r: the integer code representing real numeric zero in affine quantization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1092: `+ $signed({16'd0, zero_point_restore_r}));` This is continuation line 2 of the 2-line logical statement that begins at source line 1091. Its exact fragment is '+ $signed({16'd0, zero_point_restore_r}));'. The important operands are d0: the value named 'd0' in this block; zero_point_restore_r: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This makes p5_result_r, the value named 'p5 result r' in this block, equal to '48'(p4_corrected_r + $signed({16'd0, zero_point_restore_r}))'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p4_corrected_r: the value named 'p4 corrected r' in this block; d0: the value named 'd0' in this block; zero_point_restore_r: the integer code representing real numeric zero in affine quantization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1093: `p5_row_r <= p4_row_r;` This makes p5_row_r, one output-row position processed by the tensor tile, equal to 'p4_row_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p4_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1094: `p5_column_r <= p4_column_r;` This makes p5_column_r, one output-feature position processed in parallel, equal to 'p4_column_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p4_column_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1095: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1097: `p4_valid_r <= p3_valid_r;` This makes p4_valid_r, the producer's statement that its current payload is meaningful, equal to 'p3_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p3_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1098: `if (p3_valid_r) begin` This condition selects the following hardware branch only when 'p3_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are p3_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1099: `p4_corrected_r <= p3_corrected_r` This statement continues through source line 1100; the explanation uses the complete expression rather than only the first visual fragment. This makes p4_corrected_r, the value named 'p4 corrected r' in this block, equal to 'p3_corrected_r - p3_column_correction_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p3_corrected_r: the value named 'p3 corrected r' in this block; p3_column_correction_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1100: `- p3_column_correction_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 1099. Its exact fragment is '- p3_column_correction_r;'. The important operands are p3_column_correction_r: one output-feature position processed in parallel. Read together, the complete operation is: This makes p4_corrected_r, the value named 'p4 corrected r' in this block, equal to 'p3_corrected_r - p3_column_correction_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p3_corrected_r: the value named 'p3 corrected r' in this block; p3_column_correction_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1101: `p4_row_r <= p3_row_r;` This makes p4_row_r, one output-row position processed by the tensor tile, equal to 'p3_row_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p3_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1102: `p4_column_r <= p3_column_r;` This makes p4_column_r, one output-feature position processed in parallel, equal to 'p3_column_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p3_column_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1103: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1105: `p3_valid_r <= p2_valid_r;` This makes p3_valid_r, the producer's statement that its current payload is meaningful, equal to 'p2_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p2_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1106: `if (p2_valid_r) begin` This condition selects the following hardware branch only when 'p2_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are p2_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1107: `p3_corrected_r <= p2_raw_r - p2_row_correction_r;` This makes p3_corrected_r, the value named 'p3 corrected r' in this block, equal to 'p2_raw_r - p2_row_correction_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p2_raw_r: the value named 'p2 raw r' in this block; p2_row_correction_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1108: `p3_column_correction_r <= p2_column_correction_r;` This makes p3_column_correction_r, one output-feature position processed in parallel, equal to 'p2_column_correction_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p2_column_correction_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1109: `p3_row_r <= p2_row_r;` This makes p3_row_r, one output-row position processed by the tensor tile, equal to 'p2_row_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p2_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1110: `p3_column_r <= p2_column_r;` This makes p3_column_r, one output-feature position processed in parallel, equal to ‘p2_column_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p2_column_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1111: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1113: `p2_valid_r <= p1_valid_r;` This makes p2_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘p1_valid_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p1_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1114: `if (p1_valid_r) begin` This condition selects the following hardware branch only when ‘p1_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are p1_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1115: `p2_raw_r <= p1_raw_r;` This makes p2_raw_r, the value named ’p2 raw r’ in this block, equal to ‘p1_raw_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p1_raw_r: the value named ’p1 raw r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1116: `p2_row_correction_r <= $signed({40'd0,` This statement continues through source line 1118; the explanation uses the complete expression rather than only the first visual fragment. This makes p2_row_correction_r, one output-row position processed by the tensor tile, equal to ‘$signed({40’d0, p1_row_product_low_r}) + $signed({24’d0, p1_row_product_high_r, 16’d0})‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; p1_row_product_low_r: one output-row position processed by the tensor tile; p1_row_product_high_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1117: `p1_row_product_low_r})` This is continuation line 2 of the 3-line logical statement that begins at source line 1116. Its exact fragment is ‘p1_row_product_low_r})‘. The important operands are p1_row_product_low_r: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes p2_row_correction_r, one output-row position processed by the tensor tile, equal to ‘$signed({40’d0, p1_row_product_low_r}) + $signed({24’d0, p1_row_product_high_r, 16’d0})‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; p1_row_product_low_r: one output-row position processed by the tensor tile; p1_row_product_high_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1118: `+ $signed({24'd0, p1_row_product_high_r, 16'd0});` This is continuation line 3 of the 3-line logical statement that begins at source line 1116. Its exact fragment is ‘+ $signed({24’d0, p1_row_product_high_r, 16’d0});‘. The important operands are d0: the value named ’d0’ in this block; p1_row_product_high_r: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes p2_row_correction_r, one output-row position processed by the tensor tile, equal to ‘$signed({40’d0, p1_row_product_low_r}) + $signed({24’d0, p1_row_product_high_r, 16’d0})‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block; p1_row_product_low_r: one output-row position processed by the tensor tile; p1_row_product_high_r: one output-row position processed by the tensor tile. Context:

the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1119: `p2_column_correction_r <= $signed({40'd0,` This statement continues through source line 1121; the explanation uses the complete expression rather than only the first visual fragment. This makes p2_column_correction_r, one output-feature position processed in parallel, equal to '$signed({40'd0, p1_column_product_low_r}) + $signed({24'd0, p1_column_product_high_r, 16'd0})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; p1_column_product_low_r: one output-feature position processed in parallel; p1_column_product_high_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1120: `p1_column_product_low_r})` This is continuation line 2 of the 3-line logical statement that begins at source line 1119. Its exact fragment is 'p1_column_product_low_r})'. The important operands are p1_column_product_low_r: one output-feature position processed in parallel. Read together, the complete operation is: This makes p2_column_correction_r, one output-feature position processed in parallel, equal to '$signed({40'd0, p1_column_product_low_r}) + $signed({24'd0, p1_column_product_high_r, 16'd0})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; p1_column_product_low_r: one output-feature position processed in parallel; p1_column_product_high_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1121: `+ $signed({24'd0, p1_column_product_high_r, 16'd0});` This is continuation line 3 of the 3-line logical statement that begins at source line 1119. Its exact fragment is '+ $signed({24'd0, p1_column_product_high_r, 16'd0});'. The important operands are d0: the value named 'd0' in this block; p1_column_product_high_r: one output-feature position processed in parallel. Read together, the complete operation is: This makes p2_column_correction_r, one output-feature position processed in parallel, equal to '$signed({40'd0, p1_column_product_low_r}) + $signed({24'd0, p1_column_product_high_r, 16'd0})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; p1_column_product_low_r: one output-feature position processed in parallel; p1_column_product_high_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1122: `p2_row_r <= p1_row_r;` This makes p2_row_r, one output-row position processed by the tensor tile, equal to 'p1_row_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p1_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1123: `p2_column_r <= p1_column_r;` This makes p2_column_r, one output-feature position processed in parallel, equal to 'p1_column_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p1_column_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1124: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1126: `p1_valid_r <= p0_valid_r;` This makes p1_valid_r, the producer's statement that its current payload is meaningful, equal to 'p0_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are p0_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1127: `if (p0_valid_r) begin` This condition selects the following hardware branch only when ‘p0_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are p0_valid_r: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1128: `p1_raw_r <= p0_raw_r;` This makes p1_raw_r, the value named ’p1 raw r’ in this block, equal to ‘p0_raw_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p0_raw_r: the value named ’p0 raw r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1129: `p1_row_product_low_r <= p0_row_product_low;` This makes p1_row_product_low_r, one output-row position processed by the tensor tile, equal to ‘p0_row_product_low‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p0_row_product_low: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1130: `p1_row_product_high_r <= p0_row_product_high;` This makes p1_row_product_high_r, one output-row position processed by the tensor tile, equal to ‘p0_row_product_high‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p0_row_product_high: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1131: `p1_column_product_low_r <= p0_column_product_low;` This makes p1_column_product_low_r, one output-feature position processed in parallel, equal to ‘p0_column_product_low‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p0_column_product_low: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1132: `p1_column_product_high_r <= p0_column_product_high;` This makes p1_column_product_high_r, one output-feature position processed in parallel, equal to ‘p0_column_product_high‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p0_column_product_high: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1133: `p1_row_r <= p0_row_r;` This makes p1_row_r, one output-row position processed by the tensor tile, equal to ‘p0_row_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p0_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1134: `p1_column_r <= p0_column_r;` This makes p1_column_r, one output-feature position processed in parallel, equal to ‘p0_column_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are p0_column_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1135: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1137: `// Split the 128-entry accumulator read into a 16-column` Design intent: Split the 128-entry accumulator read into a 16-column This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1138: `// selection and an 8-row selection. This adds one fill` Design intent: selection and an 8-row selection. This adds one fill This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1139: `// cycle while preserving one result per clock.` Design intent: cycle while preserving one result per clock. This comment anchors the surrounding logic in the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

Line 1140: `p0_valid_r <= pread_valid_r;` This makes p0_valid_r, the producer's statement that its current payload is meaningful, equal to 'pread_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are pread_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1141: `if (pread_valid_r) begin` This condition selects the following hardware branch only when 'pread_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pread_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1142: `p0_raw_r <= {{(64-ACC_BITS){` This statement continues through source line 1144; the explanation uses the complete expression rather than only the first visual fragment. This makes p0_raw_r, the value named 'p0 raw r' in this block, equal to '{{(64-ACC_BITS){ pread_accumulator_by_row_r[pread_row_r[2:0]][ACC_BITS-1]}}, pread_accumulator_by_row_r[pread_row_r[2:0]]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ACC_BITS: the value named 'acc bits' in this block; pread_accumulator_by_row_r: a widened running sum that prevents intermediate overflow; pread_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1143: `pread_accumulator_by_row_r[pread_row_r[2:0]][ACC_BITS-1]}},` This is continuation line 2 of the 3-line logical statement that begins at source line 1142. Its exact fragment is 'pread_accumulator_by_row_r[pread_row_r[2:0]][ACC_1]}},'. The important operands are pread_accumulator_by_row_r: a widened running sum that prevents intermediate overflow; pread_row_r: one output-row position processed by the tensor tile; ACC_BITS: the value named 'acc bits' in this block. Read together, the complete operation is: This makes p0_raw_r, the value named 'p0 raw r' in this block, equal to '{{(64-ACC_BITS){ pread_accumulator_by_row_r[pread_row_r[2:0]][ACC_BITS-1]}}, pread_accumulator_by_row_r[pread_row_r[2:0]]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ACC_BITS: the value named 'acc bits' in this block; pread_accumulator_by_row_r: a widened running sum that prevents intermediate overflow; pread_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1144: `pread_accumulator_by_row_r[pread_row_r[2:0]]};` This is continuation line 3 of the 3-line logical statement that begins at source line 1142. Its exact fragment is 'pread_accumulator_by_row_r[pread_row_r[2:0]]};'. The important operands are pread_accumulator_by_row_r: a widened running sum that prevents intermediate overflow; pread_row_r: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes p0_raw_r, the value named 'p0 raw r' in this block, equal to '{{(64-ACC_BITS){ pread_accumulator_by_row_r[pread_row_r[2:0]][ACC_BITS-1]}}, pread_accumulator_by_row_r[pread_row_r[2:0]]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ACC_BITS: the value named 'acc bits' in this block; pread_accumulator_by_row_r: a widened running sum that prevents intermediate overflow; pread_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1145: `p0_row_sum_r <= pread_row_sum_r;` This makes p0_row_sum_r, one output-row position processed by the tensor tile, equal to 'pread_row_sum_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are pread_row_sum_r: one output-row position processed by the tensor tile.

Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1146: `p0_column_sum_r <= pread_column_sum_r;` This makes p0_column_sum_r, one output-feature position processed in parallel, equal to ‘pread_column_sum_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are pread_column_sum_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1147: `p0_row_r <= pread_row_r;` This makes p0_row_r, one output-row position processed by the tensor tile, equal to ‘pread_row_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are pread_row_r: one output-row position processed by the tensor tile. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1148: `p0_column_r <= pread_column_r;` This makes p0_column_r, one output-feature position processed in parallel, equal to ‘pread_column_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are pread_column_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1149: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1151: `pread_valid_r <= launch_remaining_r != 0;` This makes pread_valid_r, the producer's statement that its current payload is meaningful, equal to ‘launch_remaining_r != 0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_remaining_r: the value named 'launch remaining r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1152: `if (launch_remaining_r != 0) begin` This condition selects the following hardware branch only when ‘launch_remaining_r != 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are launch_remaining_r: the value named 'launch remaining r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1153: `launch_remaining_r <= launch_remaining_r - 1'b1;` This makes launch_remaining_r, the value named 'launch remaining r' in this block, equal to ‘launch_remaining_r - 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_remaining_r: the value named 'launch remaining r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1154: `launch_row_r <= launch_row_r` This statement continues through source line 1155; the explanation uses the complete expression rather than only the first visual fragment. This makes launch_row_r, one output-row position processed by the tensor tile, equal to ‘launch_row_r + (launch_columns_remaining_r == 6'd1)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_row_r: one output-row position processed by the tensor tile; launch_columns_remaining_r: one output-feature position processed in parallel; d1: the value named 'd1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1155: `+ (launch_columns_remaining_r == 6'd1);` This is continuation line 2 of the 2-line logical statement that begins at source line 1154. Its exact fragment is ‘+ (launch_columns_remaining_r == 6'd1);‘. The important operands are launch_columns_remaining_r: one output-feature position processed in parallel; d1: the value named 'd1'

in this block. Read together, the complete operation is: This makes launch_row_r, one output-row position processed by the tensor tile, equal to ‘launch_row_r + (launch_columns_remaining_r == 6’d1)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_row_r: one output-row position processed by the tensor tile; launch_columns_remaining_r: one output-feature position processed in parallel; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1156: `if (launch_columns_remaining_r == 6'd1) begin` This condition selects the following hardware branch only when ‘launch_columns_remaining_r == 6’d1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are launch_columns_remaining_r: one output-feature position processed in parallel; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1157: `launch_columns_remaining_r <= columns_r;` This makes launch_columns_remaining_r, one output-feature position processed in parallel, equal to ‘columns_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are columns_r: one output-feature position processed in parallel. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1158: `launch_column_r <= 6'd0;` This makes launch_column_r, one output-feature position processed in parallel, equal to ‘6’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1159: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1160: `launch_columns_remaining_r <=` This statement continues through source line 1161; the explanation uses the complete expression rather than only the first visual fragment. This makes launch_columns_remaining_r, one output-feature position processed in parallel, equal to ‘launch_columns_remaining_r - 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_columns_remaining_r: one output-feature position processed in parallel; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1161: `launch_columns_remaining_r - 1'b1;` This is continuation line 2 of the 2-line logical statement that begins at source line 1160. Its exact fragment is ‘launch_columns_remaining_r - 1’b1;‘. The important operands are launch_columns_remaining_r: one output-feature position processed in parallel; b1: the value named ’b1’ in this block. Read together, the complete operation is: This makes launch_columns_remaining_r, one output-feature position processed in parallel, equal to ‘launch_columns_remaining_r - 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_columns_remaining_r: one output-feature position processed in parallel; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1162: `launch_column_r <= launch_column_r + 1'b1;` This makes launch_column_r, one output-feature position processed in parallel, equal to ‘launch_column_r + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are launch_column_r: one output-feature position processed in parallel; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1163: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1164: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1165: `pread_valid_r <= 1'b0;` This makes pread_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1166: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1167: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1168: `if (result_stage_pop && result_stage_last_r) state <= ST_IDLE;` This condition selects the following hardware branch only when 'result_stage_pop && result_stage_last_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_stage_pop: the value named 'result stage pop' in this block; result_stage_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1169: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1171: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'state <= ST_IDLE;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure..

Line 1172: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_PIPELINE, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1173: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 1174: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 1176: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 1177: `if (ROW_LANES != 8) $fatal(1, "neuraedge_tensor_tile currently requires ROW_LANES=8");` This condition selects the following hardware branch only when ‘ROW_LANES != 8) $fatal(1, "neuraedge_tensor_tile currently requires ROW_LANES=8"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; fatal: the value named ’fatal’ in this block; neuraedge_tensor_tile: the value named ’neuraedge tensor tile’ in this block. Context: the module’s structural declarations.

Line 1178: `if (COLUMN_LANES != 16) $fatal(1, "neuraedge_tensor_tile currently requires COLUMN_LANES=16");` This condition selects the following hardware branch only when ‘COLUMN_LANES != 16) $fatal(1, "neuraedge_tensor_tile currently requires COLUMN_LANES=16"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath; fatal: the value named ’fatal’ in this block; neuraedge_tensor_tile: the value named ’neuraedge tensor tile’ in this block. Context: the module’s structural declarations.

Line 1179: `if (COLUMN_LANES % 2 != 0) $fatal(1, "COLUMN_LANES must be even");` This condition selects the following hardware branch only when ‘COLUMN_LANES % 2 != 0) $fatal(1, "COLUMN_LANES must be even"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath; fatal: the value named ’fatal’ in this block; must: the value named ’must’ in this block. Context: the module’s structural declarations.

Line 1180: `if (ACC_BITS < 32 || ACC_BITS > 48) $fatal(1, "ACC_BITS must be in [32,48]");` This condition selects the following hardware branch only when ‘ACC_BITS $<$ 32 || ACC_BITS $>$ 48) $fatal(1, "ACC_BITS must be in [32,48]"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are ACC_BITS: the value named ’acc bits’ in this block; fatal: the value named ’fatal’ in this block; must: the value named ’must’ in this block. Context: the module’s structural declarations.

Line 1181: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 1182: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 1184: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array.

20 rtl/neuraedge_udivider.sv

Role in the complete system: performs deterministic unsigned integer division one quotient bit per cycle.

20.1 Beginner architecture map

Dataflow responsibility A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_udivider into physical nets and primitives. The source declares 6 input-port lines, 5 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_CAPTURE, ST_IDLE, ST_OUTPUT, ST_SHIFT, ST_SUBTRACT, ST_VALIDATE. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates unique as case, end as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

20.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_udivider #(
5      parameter int NUMERATOR_BITS = 64,
6      parameter int DENOMINATOR_BITS = 32
7  ) (
8      input  wire clk,
9      input  wire rst_n,
10     input  wire input_valid,
11     output logic input_ready,
12     input  wire [NUMERATOR_BITS-1:0] numerator,
13     input  wire [DENOMINATOR_BITS-1:0] denominator,
14     output logic output_valid,
15     input  wire output_ready,
16     output logic [NUMERATOR_BITS-1:0] quotient,
17     output logic [DENOMINATOR_BITS-1:0] remainder,
18     output logic divide_by_zero
19 );
20     localparam int INDEX_BITS = $clog2(NUMERATOR_BITS);
21
22     typedef enum logic [2:0] {
23         ST_IDLE = 3'd0,
24         ST_CAPTURE = 3'd1,
25         ST_SHIFT = 3'd2,
26         ST_SUBTRACT = 3'd3,
27         ST_OUTPUT = 3'd4,
28         ST_VALIDATE = 3'd5
29     } state_t;
30
31     state_t state;
32     logic [NUMERATOR_BITS-1:0] numerator_r;
33     logic [DENOMINATOR_BITS-1:0] denominator_r;
34     logic [NUMERATOR_BITS-1:0] quotient_r;
35     logic [DENOMINATOR_BITS-1:0] remainder_r;
36     logic [DENOMINATOR_BITS:0] shifted_remainder_r;
37     (* extract_enable = "no" *) logic [INDEX_BITS-1:0] bit_index;
38     logic denominator_zero_r;
39
40     always_comb begin
41         input_ready = state == ST_IDLE;
42         output_valid = state == ST_OUTPUT;
43     end
44
45     always_ff @(posedge clk) begin
46         if (!rst_n) begin
47             state <= ST_IDLE;
48             numerator_r <= '0;
```

```
            denominator_r <= '0;
            quotient_r <= '0;
            remainder_r <= '0;
            shifted_remainder_r <= '0;
            bit_index <= '0;
            denominator_zero_r <= 1'b0;
            quotient <= '0;
            remainder <= '0;
            divide_by_zero <= 1'b0;
        end else begin
            unique case (state)
                ST_IDLE: begin
                    if (input_valid) begin
                        numerator_r <= numerator;
                        denominator_r <= denominator;
                        state <= ST_CAPTURE;
                    end
                end

                ST_CAPTURE: begin
                    denominator_zero_r <= denominator_r == 0;
                    divide_by_zero <= denominator_r == 0;
                    quotient_r <= '0;
                    remainder_r <= '0;
                    state <= ST_VALIDATE;
                end

                ST_VALIDATE: begin
                    if (denominator_zero_r) begin
                        quotient <= {NUMERATOR_BITS{1'b1}};
                        remainder <= numerator_r[DENOMINATOR_BITS-1:0];
                        state <= ST_OUTPUT;
                    end else begin
                        bit_index <= $clog2(NUMERATOR_BITS)'(NUMERATOR_BITS - 1);
                        state <= ST_SHIFT;
                    end
                end

                ST_SHIFT: begin
                    shifted_remainder_r <= {
                        remainder_r,
                        numerator_r[bit_index]
                    };
                    state <= ST_SUBTRACT;
                end

                ST_SUBTRACT: begin
                    logic [DENOMINATOR_BITS-1:0] next_remainder;
                    logic [NUMERATOR_BITS-1:0] next_quotient;
                    next_quotient = quotient_r;
                    if (shifted_remainder_r >= {1'b0, denominator_r}) begin
                        next_remainder = shifted_remainder_r[DENOMINATOR_BITS-1:0]
                            - denominator_r;
                        next_quotient[bit_index] = 1'b1;
                    end else begin
                        next_remainder = shifted_remainder_r[DENOMINATOR_BITS-1:0];
                        next_quotient[bit_index] = 1'b0;
                    end
                    // A completed subtraction is strictly less than the
                    // denominator, so the stored remainder needs no guard bit.
                    remainder_r <= next_remainder;
                    quotient_r <= next_quotient;

                    if (bit_index == 0) begin
                        quotient <= next_quotient;
                        remainder <= next_remainder;
                        state <= ST_OUTPUT;
                    end else begin
                        bit_index <= bit_index - 1'b1;
                        state <= ST_SHIFT;
                    end
                end

                ST_OUTPUT: begin
                    if (output_ready) state <= ST_IDLE;
                end

                default: state <= ST_IDLE;
            endcase
        end
    end

    initial begin
        if (NUMERATOR_BITS < DENOMINATOR_BITS) begin
            $fatal(1, "NUMERATOR_BITS must be at least DENOMINATOR_BITS");
        end
        if (NUMERATOR_BITS < 2 || (1 << INDEX_BITS) != NUMERATOR_BITS) begin
```

```
136             $fatal(1, "NUMERATOR_BITS must be a power of two");
137         end
138     end
139 endmodule
140
141 `default_nettype wire
```

20.3 Line annotations

Line 1: `timescale 1ns / 1ps This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `default_nettype none This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: module neuraedge_udivider #(This opens neuraedge_udivider, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that performs deterministic unsigned integer division one quotient bit per cycle.

Line 5: parameter int NUMERATOR_BITS = 64, This statement continues through source line 19; the explanation uses the complete expression rather than only the first visual fragment. This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 6: parameter int DENOMINATOR_BITS = 32 This is continuation line 2 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'parameter int DENOMINATOR_BITS = 32'. The important operands are parameter: the value named 'parameter' in this block; DENOMINATOR_BITS: the value named 'denominator bits' in this block. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 7:) (This is continuation line 3 of the 15-line logical statement that begins at source line 5. Its exact fragment is ') ('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 8: input wire clk, This is continuation line 4 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'input wire clk,'. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 9: `input wire rst_n,` This is continuation line 5 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'input wire rst_n,'. The important operands are input: the value named 'input' in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 10: `input wire input_valid,` This is continuation line 6 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'input wire input_valid,'. The important operands are input: the value named 'input' in this block; input_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 11: `output logic input_ready,` This is continuation line 7 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'output logic input_ready,'. The important operands are output: the value named 'output' in this block; input_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 12: `input wire [NUMERATOR_BITS-1:0] numerator,` This is continuation line 8 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'input wire [NUMERATOR_BITS-1:0] numerator,'. The important operands are input: the value named 'input' in this block; NUMERATOR_BITS: the value named 'numerator bits' in this block; numerator: the value named 'numerator' in this block. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 13: `input wire [DENOMINATOR_BITS-1:0] denominator,` This is continuation line 9 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'input wire [DENOMINATOR_BITS-1:0] denominator,'. The important operands are input: the value named 'input' in this block; DENOMINATOR_BITS: the value named 'denominator bits' in this block; denominator: the value named 'denominator' in this block. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 14: `output logic output_valid,` This is continuation line 10 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'output logic output_valid,'. The important operands are output: the value named 'output' in this block; output_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready,

output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 15: `input wire output_ready,` This is continuation line 11 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'input wire output_ready,'. The important operands are input: the value named 'input' in this block; output_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 16: `output logic [NUMERATOR_BITS-1:0] quotient,` This is continuation line 12 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'output logic [NUMERATOR_BITS-1:0] quotient,'. The important operands are output: the value named 'output' in this block; NUMERATOR_BITS: the value named 'numerator bits' in this block; quotient: the value named 'quotient' in this block. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 17: `output logic [DENOMINATOR_BITS-1:0] remainder,` This is continuation line 13 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'output logic [DENOMINATOR_BITS-1:0] remainder,'. The important operands are output: the value named 'output' in this block; DENOMINATOR_BITS: the value named 'denominator bits' in this block; remainder: the value named 'remainder' in this block. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 18: `output logic divide_by_zero` This is continuation line 14 of the 15-line logical statement that begins at source line 5. Its exact fragment is 'output logic divide_by_zero'. The important operands are output: the value named 'output' in this block; divide_by_zero: the value named 'divide by zero' in this block. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 19: `);` This is continuation line 15 of the 15-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes NUMERATOR_BITS to 64, parameter int DENOMINATOR_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [NUMERATOR_BITS-1:0] numerator, input wire [DENOMINATOR_BITS-1:0] denominator, output logic output_valid, input wire output_ready, output logic [NUMERATOR_BITS-1:0] quotient, output logic [DENOMINATOR_BITS-1:0] remainder, output logic divide_by_zero) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; NUMERATOR_BITS denotes the value named 'numerator bits' in this block.

Line 20: `localparam int INDEX_BITS = $clog2(NUMERATOR_BITS);` This fixes INDEX_BITS to $clog2(NUMERATOR_BITS during elaboration. Vivado uses that constant to shape or simplify the generated circuit; INDEX_BITS denotes an integer selecting one tensor element, lane, bank, or state entry.

Line 22: `typedef enum logic [2:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 23: `ST_IDLE = 3'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 24: `ST_CAPTURE = 3'd1,` This assigns a fixed binary encoding to state ST_CAPTURE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 25: `ST_SHIFT = 3'd2,` This assigns a fixed binary encoding to state ST_SHIFT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 26: `ST_SUBTRACT = 3'd3,` This assigns a fixed binary encoding to state ST_SUBTRACT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 27: `ST_OUTPUT = 3'd4,` This assigns a fixed binary encoding to state ST_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 28: `ST_VALIDATE = 3'd5` This statement continues through source line 29; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_VALIDATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 29: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 28. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_VALIDATE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 31: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that performs deterministic unsigned integer division one quotient bit per cycle. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 32: `logic [NUMERATOR_BITS-1:0] numerator_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, numerator_r is the value named 'numerator r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `logic [DENOMINATOR_BITS-1:0] denominator_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, denominator_r is the value named 'denominator r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic [NUMERATOR_BITS-1:0] quotient_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, quotient_r is the value named 'quotient r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 35: `logic [DENOMINATOR_BITS-1:0] remainder_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, remainder_r is the value named 'remainder r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `logic [DENOMINATOR_BITS:0] shifted_remainder_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, shifted_remainder_r is the value named 'shifted remainder r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 37: `(* extract_enable = "no" *) logic [INDEX_BITS-1:0] bit_index;` Within the module's structural declarations, this exact statement contributes to the block that performs deterministic unsigned integer division one quotient bit per cycle. The important operands are extract_enable: a control bit that permits the associated state or storage update; no: the value named 'no' in this block; INDEX_BITS: an integer selecting one tensor element, lane, bank, or state entry.

Line 38: `logic denominator_zero_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, denominator_zero_r is the value named 'denominator zero r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 40: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 41: `input_ready = state == ST_IDLE;` This makes input_ready, the consumer's statement that it can accept a payload this cycle, equal to ‘state == ST_IDLE‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 42: `output_valid = state == ST_OUTPUT;` This makes output_valid, the producer's statement that its current payload is meaningful, equal to ‘state == ST_OUTPUT‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT: the value named 'st output' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 43: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 45: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with ‘<=‘ inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 46: `if (!rst_n) begin` This is the reset branch for ‘(!rst_n‘. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 47: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 48: `numerator_r <= '0;` This makes numerator_r, the value named 'numerator r' in this block, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 49: `denominator_r <= '0;` This makes denominator_r, the value named 'denominator r' in this block, equal to ‘'0‘. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 50: `quotient_r <= '0;` This makes quotient_r, the value named 'quotient r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 51: `remainder_r <= '0;` This makes remainder_r, the value named 'remainder r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 52: `shifted_remainder_r <= '0;` This makes shifted_remainder_r, the value named 'shifted remainder r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 53: `bit_index <= '0;` This makes bit_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 54: `denominator_zero_r <= 1'b0;` This makes denominator_zero_r, the value named 'denominator zero r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 55: `quotient <= '0;` This makes quotient, the value named 'quotient' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 56: `remainder <= '0;` This makes remainder, the value named 'remainder' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 57: `divide_by_zero <= 1'b0;` This makes divide_by_zero, the value named 'divide by zero' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 58: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 59: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 60: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 61: `if (input_valid) begin` This condition selects the following hardware branch only when 'input_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 62: `numerator_r <= numerator;` This makes numerator_r, the value named 'numerator r' in this block, equal to 'numerator'. It updates after the active clock edge together with other nonblocking assignments. The important operands are numerator: the value named 'numerator' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 63: `denominator_r <= denominator;` This makes denominator_r, the value named 'denominator r' in this block, equal to 'denominator'. It updates after the active clock edge together with other nonblocking assignments. The important operands are denominator: the value named 'denominator' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 64: `state <= ST_CAPTURE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_CAPTURE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_CAPTURE: the value named 'st capture' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 65: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 66: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 68: `ST_CAPTURE: begin` This is the ST_CAPTURE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE, the machine performs the named control phase..

Line 69: `denominator_zero_r <= denominator_r == 0;` This makes denominator_zero_r, the value named 'denominator zero r' in this block, equal to 'denominator_r == 0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are denominator_r: the value named 'denominator r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE, the machine performs the named control phase..

Line 70: `divide_by_zero <= denominator_r == 0;` This makes divide_by_zero, the value named 'divide by zero' in this block, equal to 'denominator_r == 0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are denominator_r: the value named 'denominator r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE, the machine performs the named control phase..

Line 71: `quotient_r <= '0;` This makes quotient_r, the value named 'quotient r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE, the machine performs the named control phase..

Line 72: `remainder_r <= '0;` This makes remainder_r, the value named 'remainder r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE, the machine performs the named control phase..

Line 73: `state <= ST_VALIDATE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VALIDATE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VALIDATE: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE, the machine performs the named control phase..

Line 74: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_CAPTURE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 76: `ST_VALIDATE: begin` This is the ST_VALIDATE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 77: `if (denominator_zero_r) begin` This condition selects the following hardware branch only when ‘denominator_zero_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are denominator_zero_r: the value named ’denominator zero r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 78: `quotient <= {NUMERATOR_BITS{1'b1}};` This makes quotient, the value named ’quotient’ in this block, equal to ‘{NUMERATOR_BITS{1’b1}}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are NUMERATOR_BITS: the value named ’numerator bits’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 79: `remainder <= numerator_r[DENOMINATOR_BITS-1:0];` This makes remainder, the value named ’remainder’ in this block, equal to ‘numerator_r[DENOMINATOR_BITS-1:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are numerator_r: the value named ’numerator r’ in this block; DENOMINATOR_BITS: the value named ’denominator bits’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 80: `state <= ST_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT: the value named ’st output’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 81: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 82: `bit_index <= $clog2(NUMERATOR_BITS)'(NUMERATOR_BITS - 1);` This makes bit_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘$clog2(NUMERATOR_BITS)’(NUMERATOR_BITS - 1)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are clog2: the value named ’clog2’ in this block; NUMERATOR_BITS: the value named ’numerator bits’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 83: `state <= ST_SHIFT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_SHIFT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SHIFT: the value named ’st shift’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase..

Line 84: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 85: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VALIDATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 87: `ST_SHIFT: begin` This is the ST_SHIFT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase..

Line 88: `shifted_remainder_r <= {` This statement continues through source line 91; the explanation uses the complete expression rather than only the first visual fragment. This makes shifted_remainder_r, the value named ’shifted remainder r’ in this block, equal to ‘{ remainder_r, numerator_r[bit_index] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are remainder_r: the value named ’remainder r’ in this block; numerator_r: the value named ’numerator r’ in this block; bit_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase..

Line 89: `remainder_r,` This is continuation line 2 of the 4-line logical statement that begins at source line 88. Its exact fragment is ‘remainder_r,‘. The important operands are remainder_r: the value named ’remainder r’ in this block. Read together, the complete operation is: This makes shifted_remainder_r, the value named ’shifted remainder r’ in this block, equal to ‘{ remainder_r, numerator_r[bit_index] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are remainder_r: the value named ’remainder r’ in this block; numerator_r: the value named ’numerator r’ in this block; bit_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase..

Line 90: `numerator_r[bit_index]` This is continuation line 3 of the 4-line logical statement that begins at source line 88. Its exact fragment is ‘numerator_r[bit_index]‘. The important operands are numerator_r: the value named ’numerator r’ in this block; bit_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes shifted_remainder_r, the value named ’shifted remainder r’ in this block, equal to ‘{ remainder_r, numerator_r[bit_index] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are remainder_r: the value named ’remainder r’ in this block; numerator_r: the value named ’numerator r’ in this block; bit_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase..

Line 91: `};` This is continuation line 4 of the 4-line logical statement that begins at source line 88. Its exact fragment is ‘};‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This makes shifted_remainder_r, the value named ’shifted remainder r’ in this block, equal to ‘{ remainder_r, numerator_r[bit_index] }‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are remainder_r: the value named ’remainder r’ in this block; numerator_r: the value named ’numerator r’ in this block; bit_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase..

Line 92: `state <= ST_SUBTRACT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_SUBTRACT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SUBTRACT: the value named ’st subtract’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase..

Line 93: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 95: `ST_SUBTRACT: begin` This is the ST_SUBTRACT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 96: `logic [DENOMINATOR_BITS-1:0] next_remainder;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase.. Specifically, next_remainder is the value named ’next remainder’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 97: `logic [NUMERATOR_BITS-1:0] next_quotient;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase.. Specifically, next_quotient is the value named ’next quotient’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 98: `next_quotient = quotient_r;` This makes next_quotient, the value named ’next quotient’ in this block, equal to ‘quotient_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are quotient_r: the value named ’quotient r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 99: `if (shifted_remainder_r >= {1'b0, denominator_r}) begin` This condition selects the following hardware branch only when ‘shifted_remainder_r >= {1’b0, denominator_r}‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are shifted_remainder_r: the value named ’shifted remainder r’ in this block; b0: the value named ’b0’ in this block; denominator_r: the value named ’denominator r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 100: `next_remainder = shifted_remainder_r[DENOMINATOR_BITS-1:0]` This statement continues through source line 101; the explanation uses the complete expression rather than only the first visual fragment. This makes next_remainder, the value named ’next remainder’ in this block, equal to ‘shifted_remainder_r[DENOMINATOR_BITS-1:0] - denominator_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are shifted_remainder_r: the value named ’shifted remainder r’ in this block; DENOMINATOR_BITS: the value named ’denominator bits’ in this block; denominator_r: the value named ’denominator r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 101: `- denominator_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 100. Its exact fragment is ‘- denominator_r;‘. The important operands are denominator_r: the value named ’denominator r’ in this block. Read together, the complete operation is: This makes next_remainder, the value named ’next remainder’ in this block, equal to ‘shifted_remainder_r[DENOMINATOR_BITS-1:0] - denominator_r‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are shifted_remainder_r: the value named ’shifted remainder r’ in this block; DENOMINATOR_BITS: the value named ’denominator bits’ in this block; denominator_r: the value named ’denominator r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 102: `next_quotient[bit_index] = 1'b1;` This makes next_quotient[bit_index], the value named ’next quotient’ in this block, equal to ‘1’b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 103: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 104: `next_remainder = shifted_remainder_r[DENOMINATOR_BITS-1:0];` This makes next_remainder, the value named 'next remainder' in this block, equal to 'shifted_remainder_r[DENOMINATOR_BITS-1:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are shifted_remainder_r: the value named 'shifted remainder r' in this block; DENOMINATOR_BITS: the value named 'denominator bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 105: `next_quotient[bit_index] = 1'b0;` This makes next_quotient[bit_index], the value named 'next quotient' in this block, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 106: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 107: `// A completed subtraction is strictly less than the` Design intent: A completed subtraction is strictly less than the This comment anchors the surrounding logic in the block that performs deterministic unsigned integer division one quotient bit per cycle.

Line 108: `// denominator, so the stored remainder needs no guard bit.` Design intent: denominator, so the stored remainder needs no guard bit. This comment anchors the surrounding logic in the block that performs deterministic unsigned integer division one quotient bit per cycle.

Line 109: `remainder_r <= next_remainder;` This makes remainder_r, the value named 'remainder r' in this block, equal to 'next_remainder'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_remainder: the value named 'next remainder' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 110: `quotient_r <= next_quotient;` This makes quotient_r, the value named 'quotient r' in this block, equal to 'next_quotient'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_quotient: the value named 'next quotient' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 112: `if (bit_index == 0) begin` This condition selects the following hardware branch only when 'bit_index == 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are bit_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 113: `quotient <= next_quotient;` This makes quotient, the value named 'quotient' in this block, equal to 'next_quotient'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_quotient: the value named 'next quotient' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 114: `remainder <= next_remainder;` This makes remainder, the value named 'remainder' in this block, equal to 'next_remainder'. It updates after the active clock edge together with other nonblocking assignments. The important operands are next_remainder: the value named 'next remainder' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 115: `state <= ST_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT: the value named 'st output' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 116: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 117: `bit_index <= bit_index - 1'b1;` This makes bit_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'bit_index - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are bit_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 118: `state <= ST_SHIFT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_SHIFT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SHIFT: the value named 'st shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase..

Line 119: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 120: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SUBTRACT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 122: `ST_OUTPUT: begin` This is the ST_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 123: `if (output_ready) state <= ST_IDLE;` This condition selects the following hardware branch only when 'output_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 124: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 126: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'state <= ST_IDLE;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 127: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 128: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 129: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 131: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 132: `if (NUMERATOR_BITS < DENOMINATOR_BITS) begin` This condition selects the following hardware branch only when ‘NUMERATOR_BITS < DENOMINATOR_BITS‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are NUMERATOR_BITS: the value named 'numerator bits' in this block; DENOMINATOR_BITS: the value named 'denominator bits' in this block. Context: the module's structural declarations.

Line 133: `$fatal(1, "NUMERATOR_BITS must be at least DENOMINATOR_BITS");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 134: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 135: `if (NUMERATOR_BITS < 2 || (1 << INDEX_BITS) != NUMERATOR_BITS) begin` This condition selects the following hardware branch only when ‘NUMERATOR_BITS < 2 || (1 << INDEX_BITS) != NUMERATOR_BITS‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are NUMERATOR_BITS: the value named 'numerator bits' in this block; INDEX_BITS: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 136: `$fatal(1, "NUMERATOR_BITS must be a power of two");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 137: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 138: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 139: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 141: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that performs deterministic unsigned integer division one quotient bit per cycle.

21 rtl/neuraedge_universal_core.sv

Role in the complete system: decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

21.1 Beginner architecture map

Dataflow responsibility This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_universal_core into physical nets and primitives. The source declares 8 input-port lines, 33 output-port lines, 3 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_ALPHA_DRAIN, ST_ALPHA_LOAD, ST_ALPHA_OUTPUT, ST_ALPHA_START, ST_DECODE_COMMAND, ST_IDLE, ST_LAYERNORM_LOAD0, ST_LAYERNORM_LOAD1 ST_LAYERNORM_OUTPUT, ST_LAYERNORM_SEND, ST_LAYERNORM_START, ST_LIF_CLEAR, ST_LIF_LO ST_LIF_OUTPUT, ST_LIF_START, ST_LUT_CLEAR, ST_LUT_EVAL_LOAD, ST_LUT_EVAL_OUTPUT and additional states. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates neuraedge_buffered_tensor_core as u_tensor_core, neuraedge_alpha_x as u_alpha_xnor, neuraedge_vector_alu as u_vector_alu, neuraedge_integer_layernorm as u_layernorm, neuraedge_log_softmax as u_softmax, neuraedge_lif_bank as u_lif_bank, neuraedge_sgd_optimizer8 as u_optimizer, neuraedge_linear_backward8 as u_linear_backward, neuraedge_activation_lut8 as u_activation_lut, unique as case, end as if, else as if. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

21.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_universal_core (
5      input  wire clk,
6      input  wire rst_n,
7      input  wire command_valid,
8      output logic command_ready,
9      input  wire [127:0] command_data,
10     output logic command_done,
11     output logic status_busy,
12     output logic status_error,
13     output logic [3:0] status_active_opcode,
14     output logic [31:0] status_command_count,
15     output logic [31:0] status_input_beats,
16     output logic [31:0] status_output_beats,
17     output wire [11:0] debug_tensor_loaded_activations,
18     output wire [11:0] debug_tensor_loaded_weights,
19     output wire [11:0] debug_tensor_active_issues,
20     output wire [31:0] debug_tensor_compute_cycles,
21     output wire [31:0] debug_tensor_skipped_issues,
22     output wire [63:0] debug_tensor_dense_macs,
23     output wire [63:0] debug_tensor_spike_adds,
24     output wire [31:0] debug_layernorm_cycles,
25     output wire signed [31:0] debug_layernorm_mean,
26     output wire [31:0] debug_layernorm_variance,
27     output wire [31:0] debug_layernorm_sigma,
28     output wire [31:0] debug_layernorm_reciprocal,
29     output wire [31:0] debug_softmax_cycles,
30     output wire signed [23:0] debug_softmax_maximum,
31     output wire [23:0] debug_softmax_denominator,
```

```
32      output wire [31:0] debug_softmax_reciprocal,
33      output wire [63:0] debug_lif_total_spikes,
34      output wire [31:0] debug_optimizer_updates,
35      output wire [8:0] debug_lut_loaded_entries,
36      output wire [31:0] debug_training_tiles,
37      output wire [31:0] debug_training_cycles,
38
39      input  wire s_axis_tvalid,
40      output logic s_axis_tready,
41      input  wire [63:0] s_axis_tdata,
42      input  wire s_axis_tlast,
43      output logic m_axis_tvalid,
44      input  wire m_axis_tready,
45      output logic [63:0] m_axis_tdata,
46      output logic m_axis_tlast
47  );
48      import neuraedge_command_pkg::*;
49
50      typedef enum logic [5:0] {
51          ST_IDLE = 6'd0,
52          ST_TENSOR = 6'd1,
53          ST_ALPHA_LOAD = 6'd2,
54          ST_ALPHA_START = 6'd3,
55          ST_ALPHA_OUTPUT = 6'd4,
56          ST_VECTOR_LOAD = 6'd5,
57          ST_VECTOR_START = 6'd6,
58          ST_VECTOR_OUTPUT = 6'd7,
59          ST_LAYERNORM_START = 6'd8,
60          ST_LAYERNORM_LOAD0 = 6'd9,
61          ST_LAYERNORM_LOAD1 = 6'd10,
62          ST_LAYERNORM_SEND = 6'd11,
63          ST_LAYERNORM_OUTPUT = 6'd12,
64          ST_SOFTMAX_START = 6'd13,
65          ST_SOFTMAX_LOAD = 6'd14,
66          ST_SOFTMAX_OUTPUT = 6'd15,
67          ST_LIF_LOAD = 6'd16,
68          ST_LIF_CLEAR = 6'd17,
69          ST_LIF_START = 6'd18,
70          ST_LIF_OUTPUT = 6'd19,
71          ST_OPTIMIZER_LOAD = 6'd20,
72          ST_OPTIMIZER_START = 6'd21,
73          ST_OPTIMIZER_OUTPUT = 6'd22,
74          ST_LUT_LOAD = 6'd23,
75          ST_LUT_EVAL_LOAD = 6'd24,
76          ST_LUT_EVAL_START = 6'd25,
77          ST_LUT_EVAL_OUTPUT = 6'd26,
78          ST_LUT_CLEAR = 6'd27,
79          ST_ALPHA_DRAIN = 6'd28,
80          ST_OPTIMIZER_COMMIT = 6'd29,
81          ST_VECTOR_DRAIN = 6'd30,
82          ST_DECODE_COMMAND = 6'd31,
83          ST_TRAIN_START = 6'd32,
84          ST_TRAIN_RUN = 6'd33,
85          ST_OUTPUT_DRAIN = 6'd34
86      } state_t;
87
88      // One-hot recoding turns every engine-select decision into a single state
89      // bit instead of a shared five-bit decode with hundreds of loads.
90      (* fsm_encoding = "one_hot", extract_enable = "no" *) state_t state;
91      logic status_error_r;
92      (* dont_touch = "yes" *) logic error_event_r;
93      (* dont_touch = "yes" *) logic clear_error_r;
94      (* dont_touch = "yes" *) logic stream_protocol_error_event_r;
95      (* dont_touch = "yes" *) logic clear_counters_event_r;
96      (* dont_touch = "yes" *) logic command_done_pending_r;
97      (* dont_touch = "yes" *) logic input_beat_accepted_r;
98      (* dont_touch = "yes" *) logic output_beat_accepted_r;
99      logic [7:0] stream_beat_index;
100     logic [11:0] stream_item_index;
101     logic [11:0] stream_items_remaining_r;
102     logic stream_last_pending_r;
103     logic [3:0] output_word_index;
104     wire [9:0] lif_trace_bit_offset =
105         ({6'd0, output_word_index} - 10'd1) << 6;
106     logic [108:0] captured_command_r;
107     logic [3:0] train_reserved_partial_r;
108     logic train_reserved_nonzero_r;
109
110     typedef enum logic [3:0] {
111         RESULT_TENSOR = 4'd0,
112         RESULT_ALPHA = 4'd1,
113         RESULT_VECTOR = 4'd2,
114         RESULT_LAYERNORM = 4'd3,
115         RESULT_SOFTMAX = 4'd4,
116         RESULT_LIF = 4'd5,
117         RESULT_OPTIMIZER = 4'd6,
118         RESULT_TRAIN = 4'd7,
```

```
        RESULT_LUT = 4'd8
    } result_source_t;

    // Each engine writes a private result register before the shared output
    // mux.  The following elastic register then arbitrates those local banks.
    // Splitting formatting and arbitration across clocks removes the former
    // child-to-wrapper nine-way combinational path while preserving one result
    // beat per clock after the two stages fill.
    logic result_input_valid;
    logic result_input_last;
    logic result_prestage_valid_r;
    result_source_t result_prestage_source_r;
    logic result_prestage_last_r;
    logic [63:0] result_tensor_data_r;
    logic [63:0] result_alpha_data_r;
    logic [63:0] result_vector_data_r;
    logic [63:0] result_layernorm_data_r;
    logic [63:0] result_softmax_data_r;
    logic [63:0] result_lif_data_r;
    logic [63:0] result_optimizer_data_r;
    logic [63:0] result_train_data_r;
    logic [63:0] result_lut_data_r;
    logic result_output_valid_r;
    logic [63:0] result_output_data_r;
    logic result_output_last_r;
    logic command_had_output_r;
    logic final_output_accepted_r;
    logic completion_error_r;

    wire result_output_ready = !result_output_valid_r || m_axis_tready;
    wire result_prestage_ready = !result_prestage_valid_r
        || result_output_ready;
    wire result_input_accept = result_input_valid && result_prestage_ready;
    wire result_output_accept = result_output_valid_r && m_axis_tready;

    wire command_is_tensor = captured_command_r[3:0] >= CMD_LOAD_ACTIVATIONS
        && captured_command_r[3:0] <= CMD_COMPUTE_TERNARY;
    wire command_routes_tensor = command_is_tensor
        || captured_command_r[3:0] == CMD_CLEAR_STATUS;

    logic tensor_command_valid;
    wire tensor_command_ready;
    wire tensor_command_done;
    wire tensor_status_error;
    logic tensor_s_valid;
    wire tensor_s_ready;
    logic tensor_m_ready;
    wire tensor_m_valid;
    wire [63:0] tensor_m_data;
    wire tensor_m_last;

    neuraedge_buffered_tensor_core u_tensor_core (
        .clk(clk), .rst_n(rst_n),
        .command_valid(tensor_command_valid), .command_ready(tensor_command_ready),
        .command_data(captured_command_r[41:0]), .command_done(tensor_command_done),
        .status_error(tensor_status_error), .status_busy(),
        .s_axis_tvalid(tensor_s_valid), .s_axis_tready(tensor_s_ready),
        .s_axis_tdata(s_axis_tdata), .s_axis_tlast(s_axis_tlast),
        .m_axis_tvalid(tensor_m_valid), .m_axis_tready(tensor_m_ready),
        .m_axis_tdata(tensor_m_data), .m_axis_tlast(tensor_m_last),
        .status_loaded_activations(debug_tensor_loaded_activations),
        .status_loaded_weights(debug_tensor_loaded_weights),
        .status_active_issues(debug_tensor_active_issues),
        .status_input_beats(),
        .status_output_beats(),
        .status_compute_cycles(debug_tensor_compute_cycles),
        .status_skipped_issues(debug_tensor_skipped_issues),
        .status_dense_macs(debug_tensor_dense_macs),
        .status_spike_adds(debug_tensor_spike_adds)
    );

    logic [255:0] alpha_query_r;
    logic [255:0] alpha_key_r;
    logic [8:0] alpha_active_bits_r;
    logic [8:0] alpha_q8_r;
    logic alpha_load_valid_r;
    logic [2:0] alpha_load_index_r;
    logic [63:0] alpha_load_data_r;
    logic alpha_load_final_r;
    logic alpha_load_last_r;
    logic alpha_input_valid;
    wire alpha_input_ready;
    wire alpha_output_valid;
    logic alpha_output_ready;
    wire [17:0] alpha_score;
    wire [8:0] alpha_spike_matches;
    wire [8:0] alpha_zero_matches;
```

```
    wire alpha_config_error;

    neuraedge_alpha_xnor u_alpha_xnor (
        .clk(clk), .rst_n(rst_n),
        .input_valid(alpha_input_valid), .input_ready(alpha_input_ready),
        .query_spikes(alpha_query_r), .key_spikes(alpha_key_r),
        .active_bits(alpha_active_bits_r), .alpha_q8(alpha_q8_r),
        .output_valid(alpha_output_valid), .output_ready(alpha_output_ready),
        .score_q8(alpha_score), .spike_matches(alpha_spike_matches),
        .zero_matches(alpha_zero_matches), .config_error(alpha_config_error)
    );

    logic [3:0] vector_operation_r;
    logic [255:0] vector_operand_a_r;
    logic [255:0] vector_operand_b_r;
    logic signed [15:0] vector_multiplier_r;
    logic [4:0] vector_shift_r;
    logic signed [15:0] vector_zero_point_r;
    logic signed [31:0] vector_threshold_r;
    logic [31:0] vector_window_r;
    logic signed [31:0] vector_clamp_min_r;
    logic signed [31:0] vector_clamp_max_r;
    (* dont_touch = "yes" *) logic vector_load_valid_r;
    logic [3:0] vector_load_index_r;
    logic [63:0] vector_load_data_r;
    logic vector_load_final_r;
    logic vector_load_last_r;
    logic vector_input_valid;
    wire vector_input_ready;
    wire vector_output_valid;
    logic vector_output_ready;
    wire [255:0] vector_result;
    wire vector_config_error;

    neuraedge_vector_alu u_vector_alu (
        .clk(clk), .rst_n(rst_n),
        .input_valid(vector_input_valid), .input_ready(vector_input_ready),
        .operation(vector_operation_r), .operand_a(vector_operand_a_r),
        .operand_b(vector_operand_b_r), .multiplier_q15(vector_multiplier_r),
        .right_shift(vector_shift_r), .output_zero_point(vector_zero_point_r),
        .threshold(vector_threshold_r), .surrogate_window(vector_window_r),
        .clamp_minimum(vector_clamp_min_r), .clamp_maximum(vector_clamp_max_r),
        .output_valid(vector_output_valid), .output_ready(vector_output_ready),
        .result(vector_result), .config_error(vector_config_error)
    );

    logic [11:0] ln_count_r;
    logic [31:0] ln_epsilon_r;
    logic [63:0] ln_sample_word0_r;
    logic signed [23:0] ln_sample_value_r;
    logic signed [23:0] ln_sample_gamma_r;
    logic signed [23:0] ln_sample_beta_r;
    logic ln_start_valid;
    wire ln_start_ready;
    logic ln_sample_valid;
    wire ln_sample_ready;
    logic ln_sample_last;
    wire ln_result_valid;
    logic ln_result_ready;
    wire signed [31:0] ln_result_value;
    wire [11:0] ln_result_index;
    wire ln_result_last;
    wire ln_config_error;
    logic ln_start_accept_r;

    neuraedge_integer_layernorm u_layernorm (
        .clk(clk), .rst_n(rst_n),
        .start_valid(ln_start_valid), .start_ready(ln_start_ready),
        .feature_count(ln_count_r), .epsilon(ln_epsilon_r),
        .sample_valid(ln_sample_valid), .sample_ready(ln_sample_ready),
        .sample_value(ln_sample_value_r), .sample_gamma_q15(ln_sample_gamma_r),
        .sample_beta(ln_sample_beta_r), .sample_last(ln_sample_last),
        .result_valid(ln_result_valid), .result_ready(ln_result_ready),
        .result_value(ln_result_value), .result_index(ln_result_index),
        .result_last(ln_result_last), .config_error(ln_config_error),
        .cycle_count(debug_layernorm_cycles),
        .debug_mean(debug_layernorm_mean),
        .debug_variance(debug_layernorm_variance),
        .debug_sigma(debug_layernorm_sigma),
        .debug_reciprocal_q31(debug_layernorm_reciprocal)
    );

    logic [8:0] softmax_count_r;
    logic [4:0] softmax_shift_r;
    logic softmax_start_accept_r;
    logic softmax_start_valid;
    wire softmax_start_ready;
```

```
    logic softmax_score_valid;
    wire softmax_score_ready;
    logic signed [23:0] softmax_score_value;
    logic softmax_score_last;
    wire softmax_probability_valid;
    logic softmax_probability_ready;
    wire [15:0] softmax_probability;
    wire [8:0] softmax_probability_index;
    wire softmax_probability_last;
    wire softmax_config_error;

    neuraedge_log_softmax u_softmax (
        .clk(clk), .rst_n(rst_n),
        .start_valid(softmax_start_valid), .start_ready(softmax_start_ready),
        .item_count(softmax_count_r), .difference_shift(softmax_shift_r),
        .score_valid(softmax_score_valid), .score_ready(softmax_score_ready),
        .score_value(softmax_score_value), .score_last(softmax_score_last),
        .probability_valid(softmax_probability_valid),
        .probability_ready(softmax_probability_ready),
        .probability_q15(softmax_probability),
        .probability_index(softmax_probability_index),
        .probability_last(softmax_probability_last), .config_error(softmax_config_error),
        .cycle_count(debug_softmax_cycles),
        .debug_maximum_score(debug_softmax_maximum),
        .debug_denominator(debug_softmax_denominator),
        .debug_reciprocal_q31(debug_softmax_reciprocal)
    );

    logic [511:0] lif_current_r;
    logic [4:0] lif_leak_shift_r;
    logic signed [23:0] lif_threshold_r;
    logic signed [23:0] lif_reset_r;
    logic lif_clear_requested_r;
    logic [7:0] lif_load_word_select_r;
    logic lif_load_pending_r;
    logic [2:0] lif_load_index_r;
    logic [63:0] lif_load_data_r;
    logic lif_load_last_r;
    logic lif_load_final_r;
    logic lif_clear_state;
    logic lif_input_valid;
    wire lif_input_ready;
    wire lif_output_valid;
    logic lif_output_ready;
    wire [31:0] lif_spikes;
    wire [32*24-1:0] lif_pre_reset_membrane;
    wire [31:0] lif_timesteps;
    logic lif_trace_requested_r;

    neuraedge_lif_bank u_lif_bank (
        .clk(clk), .rst_n(rst_n), .clear_state(lif_clear_state),
        .clear_counters(clear_counters_event_r),
        .input_valid(lif_input_valid), .input_ready(lif_input_ready),
        .current_vector(lif_current_r), .leak_shift(lif_leak_shift_r),
        .threshold(lif_threshold_r), .reset_value(lif_reset_r),
        .output_valid(lif_output_valid), .output_ready(lif_output_ready),
        .spikes(lif_spikes), .membrane_vector(),
        .pre_reset_membrane_vector(lif_pre_reset_membrane),
        .timestep_count(lif_timesteps),
        .total_spike_count(debug_lif_total_spikes)
    );

    logic [127:0] optimizer_weight_r;
    logic [191:0] optimizer_gradient_r;
    logic [191:0] optimizer_velocity_r;
    logic [17:0] optimizer_lr_r;
    logic [16:0] optimizer_momentum_r;
    logic [16:0] optimizer_decay_r;
    logic [4:0] optimizer_update_shift_r;
    logic [63:0] optimizer_load_data_r;
    logic optimizer_load_last_r;
    logic [2:0] optimizer_load_index_r;
    logic optimizer_load_final_r;
    (* dont_touch = "yes" *) logic optimizer_commit_valid_r;
    logic optimizer_input_valid;
    wire optimizer_input_ready;
    wire optimizer_output_valid;
    logic optimizer_output_ready;
    wire [127:0] optimizer_updated_weight;
    wire [191:0] optimizer_updated_velocity;
    wire optimizer_config_error;

    neuraedge_sgd_optimizer8 u_optimizer (
        .clk(clk), .rst_n(rst_n), .clear_counters(clear_counters_event_r),
        .input_valid(optimizer_input_valid), .input_ready(optimizer_input_ready),
        .weight_vector(optimizer_weight_r), .gradient_vector(optimizer_gradient_r),
        .velocity_vector(optimizer_velocity_r), .learning_rate_q16(optimizer_lr_r),
```

```
        .momentum_q16(optimizer_momentum_r), .weight_decay_q16(optimizer_decay_r),
        .update_right_shift(optimizer_update_shift_r),
        .output_valid(optimizer_output_valid), .output_ready(optimizer_output_ready),
        .updated_weight_vector(optimizer_updated_weight),
        .updated_velocity_vector(optimizer_updated_velocity),
        .config_error(optimizer_config_error),
        .update_count(debug_optimizer_updates)
    );

    logic train_start_valid;
    wire train_start_ready;
    logic train_s_valid;
    wire train_s_ready;
    wire train_m_valid;
    logic train_m_ready;
    wire [63:0] train_m_data;
    wire train_m_last;
    wire train_done;
    wire train_config_error;
    wire train_busy;

    neuraedge_linear_backward8 u_linear_backward (
        .clk(clk), .rst_n(rst_n), .clear_counters(clear_counters_event_r),
        .start_valid(train_start_valid), .start_ready(train_start_ready),
        .active_rows(captured_command_r[7:4]),
        .active_inputs(captured_command_r[11:8]),
        .active_outputs(captured_command_r[15:12]),
        .input_gradient_shift(captured_command_r[20:16]),
        .weight_gradient_shift(captured_command_r[25:21]),
        .gradient_mode(captured_command_r[27:26]),
        .accumulate_result(captured_command_r[28]),
        .suppress_output(captured_command_r[29]),
        .reserved_nonzero(train_reserved_nonzero_r),
        .s_axis_tvalid(train_s_valid), .s_axis_tready(train_s_ready),
        .s_axis_tdata(s_axis_tdata), .s_axis_tlast(s_axis_tlast),
        .m_axis_tvalid(train_m_valid), .m_axis_tready(train_m_ready),
        .m_axis_tdata(train_m_data), .m_axis_tlast(train_m_last),
        .busy(train_busy), .done(train_done), .config_error(train_config_error),
        .completed_tiles(debug_training_tiles),
        .active_cycles(debug_training_cycles)
    );

    logic lut_clear;
    logic lut_load_valid;
    wire lut_load_ready;
    logic [7:0] lut_load_address;
    logic signed [15:0] lut_load_value;
    logic lut_load_pending_r;
    logic [7:0] lut_load_address_r;
    logic signed [15:0] lut_load_value_r;
    logic lut_load_last_r;
    logic lut_load_final_r;
    logic lut_input_valid;
    wire lut_input_ready;
    logic [63:0] lut_indices_r;
    wire lut_output_valid;
    logic lut_output_ready;
    wire [127:0] lut_output_values;
    wire lut_config_error;
    logic lut_clear_requested_r;
    logic decode_error_pending_r;

    neuraedge_activation_lut8 u_activation_lut (
        .clk(clk), .rst_n(rst_n), .clear_table(lut_clear),
        .load_valid(lut_load_valid), .load_ready(lut_load_ready),
        .load_address(lut_load_address), .load_value(lut_load_value),
        .input_valid(lut_input_valid), .input_ready(lut_input_ready),
        .input_indices(lut_indices_r), .output_valid(lut_output_valid),
        .output_ready(lut_output_ready), .output_values(lut_output_values),
        .config_error(lut_config_error),
        .loaded_entries(debug_lut_loaded_entries)
    );

    // Keep each engine's transfer event local.  A shared OR-of-valids is fine
    // for the single accounting bit, but using it in unrelated FSM branches
    // creates false cross-engine control cones after one-hot optimization.
    wire tensor_result_accept = state == ST_TENSOR
        && tensor_m_valid && result_prestage_ready;
    wire alpha_result_accept = state == ST_ALPHA_OUTPUT
        && alpha_output_valid && result_prestage_ready;
    wire vector_result_accept = state == ST_VECTOR_OUTPUT
        && vector_output_valid && result_prestage_ready;
    wire layernorm_result_accept = state == ST_LAYERNORM_OUTPUT
        && ln_result_valid && result_prestage_ready;
    wire softmax_result_accept = state == ST_SOFTMAX_OUTPUT
        && softmax_probability_valid && result_prestage_ready;
    wire lif_result_accept = state == ST_LIF_OUTPUT
```

```
        && lif_output_valid && result_prestage_ready;
    wire optimizer_result_accept = state == ST_OPTIMIZER_OUTPUT
        && optimizer_output_valid && result_prestage_ready;
    wire train_result_accept = state == ST_TRAIN_RUN
        && train_m_valid && result_prestage_ready;
    wire lut_result_accept = state == ST_LUT_EVAL_OUTPUT
        && lut_output_valid && result_prestage_ready;

    always_comb begin
        command_ready = state == ST_IDLE;
        status_busy = state != ST_IDLE || train_busy;
        status_error = status_error_r || tensor_status_error;
        s_axis_tready = 1'b0;
        m_axis_tvalid = result_output_valid_r;
        m_axis_tdata = result_output_data_r;
        m_axis_tlast = result_output_last_r;
        result_input_valid = 1'b0;
        result_input_last = 1'b0;

        tensor_command_valid = state == ST_DECODE_COMMAND && command_routes_tensor;
        tensor_s_valid = 1'b0;
        tensor_m_ready = 1'b0;
        alpha_input_valid = 1'b0;
        alpha_output_ready = 1'b0;
        vector_input_valid = 1'b0;
        vector_output_ready = 1'b0;
        ln_start_valid = 1'b0;
        ln_sample_valid = 1'b0;
        ln_sample_last = stream_last_pending_r;
        ln_result_ready = 1'b0;
        softmax_start_valid = 1'b0;
        softmax_score_valid = 1'b0;
        softmax_score_value = s_axis_tdata[23:0];
        softmax_score_last = stream_last_pending_r;
        softmax_probability_ready = 1'b0;
        lif_clear_state = 1'b0;
        lif_input_valid = 1'b0;
        lif_output_ready = 1'b0;
        optimizer_input_valid = 1'b0;
        optimizer_output_ready = 1'b0;
        train_start_valid = 1'b0;
        train_s_valid = 1'b0;
        train_m_ready = 1'b0;
        lut_clear = 1'b0;
        lut_load_valid = 1'b0;
        lut_load_address = lut_load_address_r;
        lut_load_value = lut_load_value_r;
        lut_input_valid = 1'b0;
        lut_output_ready = 1'b0;

        unique case (state)
            ST_TENSOR: begin
                tensor_s_valid = s_axis_tvalid;
                s_axis_tready = tensor_s_ready;
                result_input_valid = tensor_m_valid;
                result_input_last = tensor_m_last;
                tensor_m_ready = result_prestage_ready;
            end
            ST_ALPHA_LOAD: s_axis_tready = !(alpha_load_valid_r && alpha_load_final_r);
            ST_VECTOR_LOAD: s_axis_tready = !vector_load_final_r;
            ST_OPTIMIZER_LOAD, ST_LUT_EVAL_LOAD: s_axis_tready = 1'b1;
            ST_LIF_LOAD: s_axis_tready = !lif_load_pending_r;
            ST_ALPHA_START: alpha_input_valid = 1'b1;
            ST_ALPHA_OUTPUT: begin
                result_input_valid = alpha_output_valid;
                result_input_last = 1'b1;
                alpha_output_ready = result_prestage_ready;
            end
            ST_VECTOR_START: vector_input_valid = 1'b1;
            ST_VECTOR_OUTPUT: begin
                result_input_valid = vector_output_valid;
                result_input_last = output_word_index == 3;
                vector_output_ready = result_prestage_ready
                    && output_word_index == 3;
            end
            ST_LAYERNORM_START: ln_start_valid = 1'b1;
            ST_LAYERNORM_LOAD0, ST_LAYERNORM_LOAD1: s_axis_tready = 1'b1;
            ST_LAYERNORM_SEND: ln_sample_valid = 1'b1;
            ST_LAYERNORM_OUTPUT: begin
                result_input_valid = ln_result_valid;
                result_input_last = ln_result_last;
                ln_result_ready = result_prestage_ready;
            end
            ST_SOFTMAX_START: softmax_start_valid = 1'b1;
            ST_SOFTMAX_LOAD: begin
                softmax_score_valid = s_axis_tvalid;
                // Entry to this state is delayed until the child accepted its
```

```
                // start command, and the child remains ready until score_last.
                // Making that invariant explicit breaks a ready-to-state loop.
                s_axis_tready = softmax_score_ready;
            end
            ST_SOFTMAX_OUTPUT: begin
                result_input_valid = softmax_probability_valid;
                result_input_last = softmax_probability_last;
                softmax_probability_ready = result_prestage_ready;
            end
            ST_LIF_CLEAR: lif_clear_state = 1'b1;
            ST_LIF_START: lif_input_valid = 1'b1;
            ST_LIF_OUTPUT: begin
                result_input_valid = lif_output_valid;
                result_input_last = !lif_trace_requested_r
                    || output_word_index == 12;
                lif_output_ready = result_prestage_ready
                    && (!lif_trace_requested_r || output_word_index == 12);
            end
            ST_OPTIMIZER_START: optimizer_input_valid = 1'b1;
            ST_OPTIMIZER_OUTPUT: begin
                result_input_valid = optimizer_output_valid;
                result_input_last = output_word_index == 4'd7;
                optimizer_output_ready = result_prestage_ready
                    && output_word_index == 4'd7;
            end
            ST_TRAIN_START: train_start_valid = 1'b1;
            ST_TRAIN_RUN: begin
                train_s_valid = s_axis_tvalid;
                s_axis_tready = train_s_ready;
                result_input_valid = train_m_valid;
                result_input_last = train_m_last;
                train_m_ready = result_prestage_ready;
            end
            ST_LUT_LOAD: begin
                lut_load_valid = lut_load_pending_r;
                s_axis_tready = !lut_load_pending_r;
            end
            ST_LUT_EVAL_START: lut_input_valid = 1'b1;
            ST_LUT_EVAL_OUTPUT: begin
                result_input_valid = lut_output_valid;
                result_input_last = output_word_index == 1;
                lut_output_ready = result_prestage_ready
                    && output_word_index == 1;
            end
            ST_LUT_CLEAR: lut_clear = lut_clear_requested_r;
            default: begin end
        endcase
    end

    // A bank samples whenever its source owns an available prestage slot.  The
    // write condition deliberately excludes child VALID: during a downstream
    // stall the full prestage freezes the bank, while an available slot may
    // harmlessly sample an invalid payload.  This keeps the wide bank enable
    // local to the one-hot core state instead of broadcasting a remote child
    // FSM bit, and it preserves the payload associated with every valid token.
    always_ff @(posedge clk) begin
        if (result_prestage_ready) begin
            unique case (state)
                ST_TENSOR: result_tensor_data_r <= tensor_m_data;
                ST_ALPHA_OUTPUT: result_alpha_data_r <= {28'd0,
                    alpha_zero_matches, alpha_spike_matches, alpha_score};
                ST_VECTOR_OUTPUT: result_vector_data_r
                    <= vector_result[output_word_index*64 +: 64];
                ST_LAYERNORM_OUTPUT: result_layernorm_data_r <= {19'd0,
                    ln_result_last, ln_result_index, ln_result_value};
                ST_SOFTMAX_OUTPUT: result_softmax_data_r <= {38'd0,
                    softmax_probability_last, softmax_probability_index,
                    softmax_probability};
                ST_LIF_OUTPUT: begin
                    if (output_word_index == 0) begin
                        result_lif_data_r <= {lif_timesteps, lif_spikes};
                    end else begin
                        result_lif_data_r <= lif_pre_reset_membrane[
                            lif_trace_bit_offset +: 64];
                    end
                end
                ST_OPTIMIZER_OUTPUT: result_optimizer_data_r <= {24'd0,
                    optimizer_updated_velocity[output_word_index*24 +: 24],
                    optimizer_updated_weight[output_word_index*16 +: 16]};
                ST_TRAIN_RUN: result_train_data_r <= train_m_data;
                ST_LUT_EVAL_OUTPUT: result_lut_data_r
                    <= lut_output_values[output_word_index*64 +: 64];
                default: begin end
            endcase
        end
    end

```

```
    // Two-entry elastic output pipeline.  The first stage captures each
    // engine into a source-local register; the second stage performs only the
    // registered-bank selection.  Data and tag registers intentionally have
    // no reset mux because their valid bits are the architectural qualifier.
    always_ff @(posedge clk) begin
        if (!rst_n) begin
            result_prestage_valid_r <= 1'b0;
            result_output_valid_r <= 1'b0;
        end else begin
            if (result_output_ready) begin
                result_output_valid_r <= result_prestage_valid_r;
                if (result_prestage_valid_r) begin
                    result_output_last_r <= result_prestage_last_r;
                    unique case (result_prestage_source_r)
                        RESULT_TENSOR: result_output_data_r <= result_tensor_data_r;
                        RESULT_ALPHA: result_output_data_r <= result_alpha_data_r;
                        RESULT_VECTOR: result_output_data_r <= result_vector_data_r;
                        RESULT_LAYERNORM: result_output_data_r
                            <= result_layernorm_data_r;
                        RESULT_SOFTMAX: result_output_data_r <= result_softmax_data_r;
                        RESULT_LIF: result_output_data_r <= result_lif_data_r;
                        RESULT_OPTIMIZER: result_output_data_r
                            <= result_optimizer_data_r;
                        RESULT_TRAIN: result_output_data_r <= result_train_data_r;
                        RESULT_LUT: result_output_data_r <= result_lut_data_r;
                        default: result_output_data_r <= 64'd0;
                    endcase
                end
            end

            if (result_prestage_ready) begin
                result_prestage_valid_r <= result_input_valid;
                result_prestage_last_r <= result_input_last;
                unique case (state)
                    ST_TENSOR: result_prestage_source_r <= RESULT_TENSOR;
                    ST_ALPHA_OUTPUT: result_prestage_source_r <= RESULT_ALPHA;
                    ST_VECTOR_OUTPUT: result_prestage_source_r <= RESULT_VECTOR;
                    ST_LAYERNORM_OUTPUT: result_prestage_source_r
                        <= RESULT_LAYERNORM;
                    ST_SOFTMAX_OUTPUT: result_prestage_source_r <= RESULT_SOFTMAX;
                    ST_LIF_OUTPUT: result_prestage_source_r <= RESULT_LIF;
                    ST_OPTIMIZER_OUTPUT: result_prestage_source_r
                        <= RESULT_OPTIMIZER;
                    ST_TRAIN_RUN: result_prestage_source_r <= RESULT_TRAIN;
                    ST_LUT_EVAL_OUTPUT: result_prestage_source_r <= RESULT_LUT;
                    default: result_prestage_source_r <= RESULT_TENSOR;
                endcase
            end
        end
    end

    always_ff @(posedge clk) begin
        if (!rst_n) begin
            state <= ST_IDLE;
            command_done <= 1'b0;
            command_done_pending_r <= 1'b0;
            input_beat_accepted_r <= 1'b0;
            output_beat_accepted_r <= 1'b0;
            status_error_r <= 1'b0;
            error_event_r <= 1'b0;
            clear_error_r <= 1'b0;
            stream_protocol_error_event_r <= 1'b0;
            clear_counters_event_r <= 1'b0;
            status_active_opcode <= 4'd0;
            status_command_count <= 32'd0;
            status_input_beats <= 32'd0;
            status_output_beats <= 32'd0;
            stream_beat_index <= 8'd0;
            stream_item_index <= 12'd0;
            stream_items_remaining_r <= 12'd0;
            stream_last_pending_r <= 1'b0;
            output_word_index <= 4'd0;
            captured_command_r <= 109'd0;
            train_reserved_partial_r <= 4'd0;
            train_reserved_nonzero_r <= 1'b0;
            command_had_output_r <= 1'b0;
            final_output_accepted_r <= 1'b0;
            completion_error_r <= 1'b0;
            alpha_query_r <= '0;
            alpha_key_r <= '0;
            alpha_active_bits_r <= 9'd0;
            alpha_q8_r <= 9'd0;
            alpha_load_valid_r <= 1'b0;
            alpha_load_index_r <= 3'd0;
            alpha_load_data_r <= 64'd0;
            alpha_load_final_r <= 1'b0;
            alpha_load_last_r <= 1'b0;
```

```
            vector_operation_r <= 4'd0;
            vector_operand_a_r <= '0;
            vector_operand_b_r <= '0;
            vector_multiplier_r <= 16'sd0;
            vector_shift_r <= 5'd0;
            vector_zero_point_r <= 16'sd0;
            vector_threshold_r <= 32'sd0;
            vector_window_r <= 32'd0;
            vector_clamp_min_r <= 32'sh80000000;
            vector_clamp_max_r <= 32'sh7fffffff;
            vector_load_valid_r <= 1'b0;
            vector_load_index_r <= 4'd0;
            vector_load_data_r <= 64'd0;
            vector_load_final_r <= 1'b0;
            vector_load_last_r <= 1'b0;
            ln_count_r <= 12'd0;
            ln_epsilon_r <= 32'd0;
            ln_sample_word0_r <= 64'd0;
            ln_sample_value_r <= 24'sd0;
            ln_sample_gamma_r <= 24'sd0;
            ln_sample_beta_r <= 24'sd0;
            ln_start_accept_r <= 1'b0;
            softmax_count_r <= 9'd0;
            softmax_shift_r <= 5'd0;
            softmax_start_accept_r <= 1'b0;
            lif_current_r <= '0;
            lif_leak_shift_r <= 5'd0;
            lif_threshold_r <= 24'sd0;
            lif_reset_r <= 24'sd0;
            lif_clear_requested_r <= 1'b0;
            lif_trace_requested_r <= 1'b0;
            lif_load_word_select_r <= 8'd0;
            lif_load_pending_r <= 1'b0;
            lif_load_index_r <= 3'd0;
            lif_load_data_r <= 64'd0;
            lif_load_last_r <= 1'b0;
            lif_load_final_r <= 1'b0;
            optimizer_weight_r <= '0;
            optimizer_gradient_r <= '0;
            optimizer_velocity_r <= '0;
            optimizer_lr_r <= 18'd0;
            optimizer_momentum_r <= 17'd0;
            optimizer_decay_r <= 17'd0;
            optimizer_update_shift_r <= 5'd16;
            optimizer_load_data_r <= 64'd0;
            optimizer_load_last_r <= 1'b0;
            optimizer_load_index_r <= 3'd0;
            optimizer_load_final_r <= 1'b0;
            optimizer_commit_valid_r <= 1'b0;
            lut_load_pending_r <= 1'b0;
            lut_load_address_r <= 8'd0;
            lut_load_value_r <= 16'sd0;
            lut_load_last_r <= 1'b0;
            lut_load_final_r <= 1'b0;
            lut_indices_r <= 64'd0;
            lut_clear_requested_r <= 1'b0;
            decode_error_pending_r <= 1'b0;
        end else begin
            if (clear_error_r) status_error_r <= 1'b0;
            else if (error_event_r) status_error_r <= 1'b1;
            error_event_r <= 1'b0;
            clear_error_r <= 1'b0;
            if (stream_protocol_error_event_r) error_event_r <= 1'b1;
            stream_protocol_error_event_r <= 1'b0;
            clear_counters_event_r <= 1'b0;
            vector_load_valid_r <= 1'b0;
            command_done <= command_done_pending_r;
            command_done_pending_r <= 1'b0;
            optimizer_commit_valid_r <= 1'b0;
            input_beat_accepted_r <= s_axis_tvalid && s_axis_tready;
            output_beat_accepted_r <= m_axis_tvalid && m_axis_tready;
            softmax_start_accept_r <= softmax_start_valid && softmax_start_ready;
            ln_start_accept_r <= ln_start_valid && ln_start_ready;
            if (result_input_accept) command_had_output_r <= 1'b1;
            if (result_output_accept && result_output_last_r) begin
                final_output_accepted_r <= 1'b1;
            end
            lif_load_word_select_r <= 8'd0;
            lif_load_pending_r <= 1'b0;
            for (int lif_word = 0; lif_word < 8; lif_word++) begin
                if (lif_load_word_select_r[lif_word]) begin
                    lif_current_r[lif_word*64 +: 64] <= lif_load_data_r;
                end
            end
            alpha_load_valid_r <= 1'b0;
            if (alpha_load_valid_r) begin
                if (alpha_load_last_r != alpha_load_final_r) begin
```

```
                    stream_protocol_error_event_r <= 1'b1;
                end
                if (alpha_load_index_r < 4) begin
                    alpha_query_r[alpha_load_index_r*64 +: 64] <= alpha_load_data_r;
                end else begin
                    alpha_key_r[(int'(alpha_load_index_r)-4)*64 +: 64]
                        <= alpha_load_data_r;
                end
            end
            if (optimizer_commit_valid_r) begin
                optimizer_weight_r[optimizer_load_index_r*16 +: 16]
                    <= optimizer_load_data_r[15:0];
                optimizer_gradient_r[optimizer_load_index_r*24 +: 24]
                    <= optimizer_load_data_r[39:16];
                optimizer_velocity_r[optimizer_load_index_r*24 +: 24]
                    <= optimizer_load_data_r[63:40];
            end
            if (vector_load_valid_r) begin
                if (vector_load_last_r != vector_load_final_r) begin
                    stream_protocol_error_event_r <= 1'b1;
                end
                if (vector_load_index_r == 0) begin
                    vector_clamp_min_r <= vector_load_data_r[31:0];
                    vector_clamp_max_r <= vector_load_data_r[63:32];
                end else if (vector_load_index_r <= 4) begin
                    vector_operand_a_r[(int'(vector_load_index_r)-1)*64 +: 64]
                        <= vector_load_data_r;
                end else begin
                    vector_operand_b_r[(int'(vector_load_index_r)-5)*64 +: 64]
                        <= vector_load_data_r;
                end
            end
            if (clear_counters_event_r) begin
                status_input_beats <= 32'd0;
                status_output_beats <= 32'd0;
            end else begin
                if (input_beat_accepted_r) begin
                    status_input_beats <= status_input_beats + 1'b1;
                end
                if (output_beat_accepted_r) begin
                    status_output_beats <= status_output_beats + 1'b1;
                end
            end

            unique case (state)
                ST_IDLE: begin
                    if (command_valid && command_ready) begin
                        captured_command_r <= command_data[108:0];
                        // Stage the wide reserved-field reduction across the
                        // existing capture/decode clocks for 200 MHz timing.
                        train_reserved_partial_r[0] <= |command_data[61:30];
                        train_reserved_partial_r[1] <= |command_data[93:62];
                        train_reserved_partial_r[2] <= |command_data[125:94];
                        train_reserved_partial_r[3] <= |command_data[127:126];
                        status_active_opcode <= command_data[3:0];
                        status_command_count <= status_command_count + 1'b1;
                        stream_beat_index <= 8'd0;
                        stream_item_index <= 12'd0;
                        stream_items_remaining_r <= 12'd0;
                        stream_last_pending_r <= 1'b0;
                        output_word_index <= 4'd0;
                        command_had_output_r <= 1'b0;
                        final_output_accepted_r <= 1'b0;
                        completion_error_r <= 1'b0;
                        state <= ST_DECODE_COMMAND;
                    end
                end

                ST_DECODE_COMMAND: begin
                    unique case (captured_command_r[3:0])
                    CMD_TRAIN_LINEAR_BACKWARD: begin
                        train_reserved_nonzero_r <= |train_reserved_partial_r;
                        state <= ST_TRAIN_START;
                    end
                    CMD_LOAD_ACTIVATIONS, CMD_LOAD_WEIGHTS_OUTER,
                    CMD_LOAD_WEIGHTS_DOT8, CMD_COMPUTE_DENSE_OUTER,
                    CMD_COMPUTE_DENSE_DOT8, CMD_COMPUTE_BINARY,
                    CMD_COMPUTE_TERNARY: begin
                        if (tensor_command_ready) state <= ST_TENSOR;
                    end
                    CMD_ALPHA_XNOR: begin
                        alpha_active_bits_r <= captured_command_r[50:42];
                        alpha_q8_r <= captured_command_r[59:51];
                        // ST_ALPHA_LOAD commits all four query words and all
                        // four key words before start, so retaining the prior
                        // command here cannot expose stale operand bits.
                        state <= ST_ALPHA_LOAD;
```

```
                    end
                    CMD_VECTOR_ALU: begin
                        vector_operation_r <= captured_command_r[7:4];
                        vector_multiplier_r <= captured_command_r[23:8];
                        vector_shift_r <= captured_command_r[28:24];
                        vector_zero_point_r <= captured_command_r[44:29];
                        vector_threshold_r <= captured_command_r[76:45];
                        vector_window_r <= captured_command_r[108:77];
                        state <= ST_VECTOR_LOAD;
                    end
                    CMD_LAYERNORM: begin
                        ln_count_r <= captured_command_r[15:4];
                        stream_items_remaining_r <= captured_command_r[15:4];
                        stream_last_pending_r <= captured_command_r[15:4] == 1;
                        ln_epsilon_r <= captured_command_r[47:16];
                        state <= ST_LAYERNORM_START;
                    end
                    CMD_LOG_SOFTMAX: begin
                        softmax_count_r <= captured_command_r[12:4];
                        stream_items_remaining_r <= {3'd0, captured_command_r[12:4]};
                        stream_last_pending_r <= captured_command_r[12:4] == 1;
                        softmax_shift_r <= captured_command_r[17:13];
                        state <= ST_SOFTMAX_START;
                    end
                    CMD_LIF_STEP: begin
                        lif_leak_shift_r <= captured_command_r[8:4];
                        lif_threshold_r <= captured_command_r[32:9];
                        lif_reset_r <= captured_command_r[56:33];
                        lif_clear_requested_r <= captured_command_r[57];
                        lif_trace_requested_r <= captured_command_r[58];
                        // The protocol always loads all eight words before
                        // starting the child, so a 512-bit command-time clear
                        // would only add a global mux to every current bit.
                        lif_load_word_select_r <= 8'd0;
                        lif_load_pending_r <= 1'b0;
                        lif_load_index_r <= 3'd0;
                        lif_load_final_r <= 1'b0;
                        state <= ST_LIF_LOAD;
                    end
                    CMD_SGD_UPDATE: begin
                        optimizer_lr_r <= captured_command_r[21:4];
                        optimizer_momentum_r <= captured_command_r[38:22];
                        optimizer_decay_r <= captured_command_r[55:39];
                        optimizer_update_shift_r <= captured_command_r[60:56];
                        state <= ST_OPTIMIZER_LOAD;
                    end
                    CMD_ACTIVATION_LUT: begin
                        stream_items_remaining_r <= captured_command_r[19:8];
                        stream_last_pending_r <= captured_command_r[19:8] == 1;
                        if (captured_command_r[7:4] == 0) begin
                            lut_clear_requested_r <= 1'b1;
                            decode_error_pending_r <= 1'b0;
                            state <= ST_LUT_CLEAR;
                        end else if (captured_command_r[7:4] == 1
                                && captured_command_r[19:8] != 0) begin
                            lut_load_pending_r <= 1'b0;
                            state <= ST_LUT_LOAD;
                        end else if (captured_command_r[7:4] == 2) begin
                            state <= ST_LUT_EVAL_LOAD;
                        end else begin
                            lut_clear_requested_r <= 1'b0;
                            decode_error_pending_r <= 1'b1;
                            state <= ST_LUT_CLEAR;
                        end
                    end
                    CMD_CLEAR_STATUS: begin
                        if (tensor_command_ready) begin
                            clear_error_r <= 1'b1;
                            clear_counters_event_r <= 1'b1;
                            state <= ST_TENSOR;
                        end
                    end
                    default: begin
                        lut_clear_requested_r <= 1'b0;
                        decode_error_pending_r <= 1'b1;
                        state <= ST_LUT_CLEAR;
                    end
                    endcase
                end

                    ST_TENSOR: begin
                        if (tensor_command_done) begin
                            if (command_had_output_r || tensor_result_accept) begin
                                completion_error_r <= tensor_status_error;
                                state <= ST_OUTPUT_DRAIN;
                            end else begin
                                if (tensor_status_error) error_event_r <= 1'b1;
```

```
                command_done_pending_r <= 1'b1;
                state <= ST_IDLE;
            end
        end
    end
    ST_ALPHA_LOAD: begin
        if (alpha_load_valid_r && alpha_load_final_r) begin
            alpha_load_final_r <= 1'b0;
            state <= ST_ALPHA_DRAIN;
        end else if (s_axis_tvalid) begin
            alpha_load_valid_r <= 1'b1;
            alpha_load_index_r <= stream_beat_index[2:0];
            alpha_load_data_r <= s_axis_tdata;
            alpha_load_final_r <= stream_beat_index == 7;
            alpha_load_last_r <= s_axis_tlast;
            if (stream_beat_index != 7) begin
                stream_beat_index <= stream_beat_index + 1'b1;
            end
        end
    end
    ST_ALPHA_DRAIN: if (!alpha_load_valid_r) state <= ST_ALPHA_START;
    ST_ALPHA_START: if (alpha_input_ready) state <= ST_ALPHA_OUTPUT;
    ST_ALPHA_OUTPUT: if (alpha_result_accept) begin
        completion_error_r <= alpha_config_error;
        state <= ST_OUTPUT_DRAIN;
    end
    ST_VECTOR_LOAD: begin
        if (vector_load_final_r) begin
            vector_load_final_r <= 1'b0;
            state <= ST_VECTOR_DRAIN;
        end else if (s_axis_tvalid) begin
            vector_load_valid_r <= 1'b1;
            vector_load_index_r <= stream_beat_index[3:0];
            vector_load_data_r <= s_axis_tdata;
            vector_load_final_r <= stream_beat_index == 8;
            vector_load_last_r <= s_axis_tlast;
            if (stream_beat_index != 8) begin
                stream_beat_index <= stream_beat_index + 1'b1;
            end
        end
    end
    // The registered final payload commits at this edge.
    ST_VECTOR_DRAIN: state <= ST_VECTOR_START;
    ST_VECTOR_START: if (vector_input_ready) begin
        output_word_index <= 4'd0;
        state <= ST_VECTOR_OUTPUT;
    end
    ST_VECTOR_OUTPUT: if (vector_result_accept) begin
        if (output_word_index == 3) begin
            completion_error_r <= vector_config_error;
            state <= ST_OUTPUT_DRAIN;
        end else output_word_index <= output_word_index + 1'b1;
    end
    ST_LAYERNORM_START: if (ln_start_accept_r) state <= ST_LAYERNORM_LOAD0;
    ST_LAYERNORM_LOAD0: if (s_axis_tvalid) begin
        ln_sample_word0_r <= s_axis_tdata;
        state <= ST_LAYERNORM_LOAD1;
    end
    ST_LAYERNORM_LOAD1: if (s_axis_tvalid) begin
        ln_sample_value_r <= ln_sample_word0_r[23:0];
        ln_sample_gamma_r <= ln_sample_word0_r[47:24];
        ln_sample_beta_r <= {s_axis_tdata[7:0], ln_sample_word0_r[63:48]};
        if (s_axis_tlast != stream_last_pending_r) begin
            error_event_r <= 1'b1;
        end
        state <= ST_LAYERNORM_SEND;
    end
    ST_LAYERNORM_SEND: if (ln_sample_ready) begin
        if (stream_last_pending_r) begin
            state <= ST_LAYERNORM_OUTPUT;
        end else begin
            stream_item_index <= stream_item_index + 1'b1;
            stream_items_remaining_r <= stream_items_remaining_r - 1'b1;
            stream_last_pending_r <= stream_items_remaining_r == 12'd2;
            state <= ST_LAYERNORM_LOAD0;
        end
    end
    ST_LAYERNORM_OUTPUT: if (layernorm_result_accept
            && ln_result_last) begin
        completion_error_r <= ln_config_error;
        state <= ST_OUTPUT_DRAIN;
    end
    ST_SOFTMAX_START: if (softmax_start_accept_r) state <= ST_SOFTMAX_LOAD;
    ST_SOFTMAX_LOAD: if (s_axis_tvalid) begin
        if (s_axis_tlast != stream_last_pending_r) begin
            error_event_r <= 1'b1;
        end
```

```
                        if (stream_last_pending_r) begin
                            state <= ST_SOFTMAX_OUTPUT;
                        end else begin
                            stream_item_index <= stream_item_index + 1'b1;
                            stream_items_remaining_r <= stream_items_remaining_r - 1'b1;
                            stream_last_pending_r <= stream_items_remaining_r == 12'd2;
                        end
                    end
                    ST_SOFTMAX_OUTPUT: if (softmax_result_accept
                            && softmax_probability_last) begin
                        completion_error_r <= softmax_config_error;
                        state <= ST_OUTPUT_DRAIN;
                    end
                    ST_LIF_LOAD: begin
                        if (lif_load_pending_r) begin
                            if (lif_load_last_r != lif_load_final_r) begin
                                error_event_r <= 1'b1;
                            end
                            if (lif_load_final_r) begin
                                if (lif_clear_requested_r) state <= ST_LIF_CLEAR;
                                else state <= ST_LIF_START;
                            end else begin
                                lif_load_index_r <= lif_load_index_r + 1'b1;
                            end
                        end else if (s_axis_tvalid) begin
                            lif_load_pending_r <= 1'b1;
                            lif_load_word_select_r <= 8'b1 << lif_load_index_r;
                            lif_load_data_r <= s_axis_tdata;
                            lif_load_last_r <= s_axis_tlast;
                            lif_load_final_r <= lif_load_index_r == 7;
                        end
                    end
                    ST_LIF_CLEAR: state <= ST_LIF_START;
                    ST_LIF_START: if (lif_input_ready) begin
                        output_word_index <= 4'd0;
                        state <= ST_LIF_OUTPUT;
                    end
                    ST_LIF_OUTPUT: if (lif_result_accept) begin
                        if (!lif_trace_requested_r || output_word_index == 12) begin
                            completion_error_r <= 1'b0;
                            state <= ST_OUTPUT_DRAIN;
                        end else begin
                            output_word_index <= output_word_index + 1'b1;
                        end
                    end
                    ST_OPTIMIZER_LOAD: if (s_axis_tvalid) begin
                        optimizer_load_data_r <= s_axis_tdata;
                        optimizer_load_last_r <= s_axis_tlast;
                        optimizer_load_index_r <= stream_beat_index[2:0];
                        optimizer_load_final_r <= stream_beat_index == 7;
                        optimizer_commit_valid_r <= 1'b1;
                        state <= ST_OPTIMIZER_COMMIT;
                    end
                    ST_OPTIMIZER_COMMIT: begin
                        if (optimizer_load_last_r != optimizer_load_final_r) begin
                            error_event_r <= 1'b1;
                        end
                        if (optimizer_load_final_r) begin
                            state <= ST_OPTIMIZER_START;
                        end else begin
                            stream_beat_index <= stream_beat_index + 1'b1;
                            state <= ST_OPTIMIZER_LOAD;
                        end
                    end
                    ST_OPTIMIZER_START: if (optimizer_input_ready) begin
                        output_word_index <= 4'd0;
                        state <= ST_OPTIMIZER_OUTPUT;
                    end
                    ST_OPTIMIZER_OUTPUT: if (optimizer_result_accept) begin
                        if (output_word_index == 4'd7) begin
                            completion_error_r <= optimizer_config_error;
                            state <= ST_OUTPUT_DRAIN;
                        end else output_word_index <= output_word_index + 1'b1;
                    end
                    ST_TRAIN_START: if (train_start_ready) state <= ST_TRAIN_RUN;
                    ST_TRAIN_RUN: if (train_done) begin
                        if (command_had_output_r || train_result_accept) begin
                            completion_error_r <= train_config_error;
                            state <= ST_OUTPUT_DRAIN;
                        end else begin
                            if (train_config_error) error_event_r <= 1'b1;
                            command_done_pending_r <= 1'b1;
                            state <= ST_IDLE;
                        end
                    end
                    ST_LUT_LOAD: begin
                        if (lut_load_pending_r && lut_load_ready) begin
```

```
                            lut_load_pending_r <= 1'b0;
                            if (lut_load_last_r != lut_load_final_r) begin
                                error_event_r <= 1'b1;
                            end
                            if (lut_load_final_r) begin
                                command_done_pending_r <= 1'b1;
                                state <= ST_IDLE;
                            end else begin
                                stream_item_index <= stream_item_index + 1'b1;
                                stream_items_remaining_r
                                    <= stream_items_remaining_r - 1'b1;
                                stream_last_pending_r
                                    <= stream_items_remaining_r == 12'd2;
                            end
                        end else if (!lut_load_pending_r && s_axis_tvalid) begin
                            lut_load_pending_r <= 1'b1;
                            lut_load_address_r <= s_axis_tdata[7:0];
                            lut_load_value_r <= s_axis_tdata[23:8];
                            lut_load_last_r <= s_axis_tlast;
                            lut_load_final_r <= stream_last_pending_r;
                        end
                    end
                    ST_LUT_EVAL_LOAD: if (s_axis_tvalid) begin
                        lut_indices_r <= s_axis_tdata;
                        if (!s_axis_tlast) error_event_r <= 1'b1;
                        state <= ST_LUT_EVAL_START;
                    end
                    ST_LUT_EVAL_START: if (lut_input_ready) begin
                        output_word_index <= 4'd0;
                        state <= ST_LUT_EVAL_OUTPUT;
                    end
                    ST_LUT_EVAL_OUTPUT: begin
                        if (lut_result_accept && output_word_index == 1) begin
                            completion_error_r <= lut_config_error;
                            state <= ST_OUTPUT_DRAIN;
                        end else if (lut_result_accept) begin
                            output_word_index <= output_word_index + 1'b1;
                        end
                    end
                    ST_OUTPUT_DRAIN: if (final_output_accepted_r) begin
                        if (completion_error_r) error_event_r <= 1'b1;
                        command_done_pending_r <= 1'b1;
                        state <= ST_IDLE;
                    end
                    ST_LUT_CLEAR: begin
                        if (decode_error_pending_r) error_event_r <= 1'b1;
                        lut_clear_requested_r <= 1'b0;
                        decode_error_pending_r <= 1'b0;
                        command_done_pending_r <= 1'b1;
                        state <= ST_IDLE;
                    end
                default: state <= ST_IDLE;
            endcase
        end
    end
endmodule

`default_nettype wire
```

21.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_universal_core (` This opens neuraedge_universal_core, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 5: `input wire clk,` This declares clk as a input port with width one bit. It carries the clock that makes all registers advance together across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 6: `input wire rst_n,` This declares rst_n as a input port with width one bit. It carries the active-low synchronous reset across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 7: `input wire command_valid,` This declares command_valid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 8: `output logic command_ready,` This declares command_ready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 9: `input wire [127:0] command_data,` This declares command_data as a input port with width [127:0]. It carries the 128-bit instruction selecting an engine, shape, mode, and numeric parameters across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 10: `output logic command_done,` This declares command_done as a output port with width one bit. It carries the 128-bit instruction selecting an engine, shape, mode, and numeric parameters across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 11: `output logic status_busy,` This declares status_busy as a output port with width one bit. It carries the value named 'status busy' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 12: `output logic status_error,` This declares status_error as a output port with width one bit. It carries the value named 'status error' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 13: `output logic [3:0] status_active_opcode,` This declares status_active_opcode as a output port with width [3:0]. It carries the value named 'status active opcode' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 14: `output logic [31:0] status_command_count,` This declares status_command_count as a output port with width [31:0]. It carries the 128-bit instruction selecting an engine, shape, mode, and numeric parameters across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 15: `output logic [31:0] status_input_beats,` This declares status_input_beats as a output port with width [31:0]. It carries the value named 'status input beats' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 16: `output logic [31:0] status_output_beats,` This declares status_output_beats as a output port with width [31:0]. It carries the value named 'status output beats' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 17: `output wire [11:0] debug_tensor_loaded_activations,` This declares debug_tensor_loaded_activations as a output port with width [11:0]. It carries an input feature value consumed by a neuron or multiply-accumulate operation across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 18: `output wire [11:0] debug_tensor_loaded_weights,` This declares debug_tensor_loaded_weights as a output port with width [11:0]. It carries a learned coefficient multiplied by an activation across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 19: `output wire [11:0] debug_tensor_active_issues,` This declares debug_tensor_active_issues as a output port with width [11:0]. It carries the value named 'debug tensor active issues' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 20: `output wire [31:0] debug_tensor_compute_cycles,` This declares debug_tensor_compute_cycles as a output port with width [31:0]. It carries the value named 'debug tensor compute cycles' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 21: `output wire [31:0] debug_tensor_skipped_issues,` This declares debug_tensor_skipped_issues as a output port with width [31:0]. It carries the value named 'debug tensor skipped issues' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 22: `output wire [63:0] debug_tensor_dense_macs,` This declares debug_tensor_dense_macs as a output port with width [63:0]. It carries a count of ordinary activation-times-weight operations actually issued across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 23: `output wire [63:0] debug_tensor_spike_adds,` This declares debug_tensor_spike_adds as a output port with width [63:0]. It carries a binary event produced when a neuron crosses its threshold across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 24: `output wire [31:0] debug_layernorm_cycles,` This declares debug_layernorm_cycles as a output port with width [31:0]. It carries the value named 'debug layernorm cycles' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 25: `output wire signed [31:0] debug_layernorm_mean,` This declares debug_layernorm_mean as a output port with width [31:0]. It carries the arithmetic average used by normalization across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 26: `output wire [31:0] debug_layernorm_variance,` This declares debug_layernorm_variance as a output port with width [31:0]. It carries the average squared distance from the mean across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 27: `output wire [31:0] debug_layernorm_sigma,` This declares debug_layernorm_sigma as a output port with width [31:0]. It carries the value named 'debug layernorm sigma' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 28: `output wire [31:0] debug_layernorm_reciprocal,` This declares debug_layernorm_reciprocal as a output port with width [31:0]. It carries the value named 'debug layernorm reciprocal' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 29: `output wire [31:0] debug_softmax_cycles,` This declares debug_softmax_cycles as a output port with width [31:0]. It carries the value named 'debug softmax cycles' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 30: `output wire signed [23:0] debug_softmax_maximum,` This declares debug_softmax_maximum as a output port with width [23:0]. It carries the value named 'debug softmax maximum' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 31: `output wire [23:0] debug_softmax_denominator,` This declares debug_softmax_denominator as a output port with width [23:0]. It carries the value named 'debug softmax denominator' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 32: `output wire [31:0] debug_softmax_reciprocal,` This declares debug_softmax_reciprocal as a output port with width [31:0]. It carries the value named 'debug softmax reciprocal' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 33: `output wire [63:0] debug_lif_total_spikes,` This declares debug_lif_total_spikes as a output port with width [63:0]. It carries a binary event produced when a neuron crosses its threshold across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 34: `output wire [31:0] debug_optimizer_updates,` This declares debug_optimizer_updates as a output port with width [31:0]. It carries the value named 'debug optimizer updates' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 35: `output wire [8:0] debug_lut_loaded_entries,` This declares debug_lut_loaded_entries as a output port with width [8:0]. It carries the value named 'debug lut loaded entries' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 36: `output wire [31:0] debug_training_tiles,` This declares debug_training_tiles as a output port with width [31:0]. It carries the value named 'debug training tiles' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 37: `output wire [31:0] debug_training_cycles,` This declares debug_training_cycles as a output port with width [31:0]. It carries the value named 'debug training cycles' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 39: `input wire s_axis_tvalid,` This declares s_axis_tvalid as a input port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 40: `output logic s_axis_tready,` This declares s_axis_tready as a output port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 41: `input wire [63:0] s_axis_tdata,` This declares s_axis_tdata as a input port with width [63:0]. It carries the value named 's axis tdata' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 42: `input wire s_axis_tlast,` This declares s_axis_tlast as a input port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 43: `output logic m_axis_tvalid,` This declares m_axis_tvalid as a output port with width one bit. It carries the producer's statement that its current payload is meaningful across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 44: `input wire m_axis_tready,` This declares m_axis_tready as a input port with width one bit. It carries the consumer's statement that it can accept a payload this cycle across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 45: `output logic [63:0] m_axis_tdata,` This declares m_axis_tdata as a output port with width [63:0]. It carries the value named 'm axis tdata' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 46: `output logic m_axis_tlast` This declares m_axis_tlast as a output port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 47: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 48: `import neuraedge_command_pkg::*;` This imports shared package names into the current module so opcodes and mode constants have exactly one definition. The import changes compile-time name visibility; it does not create registers or consume a clock cycle.

Line 50: `typedef enum logic [5:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 51: `ST_IDLE = 6'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 52: `ST_TENSOR = 6'd1,` This assigns a fixed binary encoding to state ST_TENSOR, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 53: `ST_ALPHA_LOAD = 6'd2,` This assigns a fixed binary encoding to state ST_ALPHA_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 54: `ST_ALPHA_START = 6'd3,` This assigns a fixed binary encoding to state ST_ALPHA_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 55: `ST_ALPHA_OUTPUT = 6'd4,` This assigns a fixed binary encoding to state ST_ALPHA_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 56: `ST_VECTOR_LOAD = 6'd5,` This assigns a fixed binary encoding to state ST_VECTOR_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 57: `ST_VECTOR_START = 6'd6,` This assigns a fixed binary encoding to state ST_VECTOR_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 58: `ST_VECTOR_OUTPUT = 6'd7,` This assigns a fixed binary encoding to state ST_VECTOR_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 59: `ST_LAYERNORM_START = 6'd8,` This assigns a fixed binary encoding to state ST_LAYERNORM_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 60: `ST_LAYERNORM_LOAD0 = 6'd9,` This assigns a fixed binary encoding to state ST_LAYERNORM_LOAD0, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 61: `ST_LAYERNORM_LOAD1 = 6'd10,` This assigns a fixed binary encoding to state ST_LAYERNORM_LOAD1, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 62: `ST_LAYERNORM_SEND = 6'd11,` This assigns a fixed binary encoding to state ST_LAYERNORM_SEND, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 63: `ST_LAYERNORM_OUTPUT = 6'd12,` This assigns a fixed binary encoding to state ST_LAYERNORM_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 64: `ST_SOFTMAX_START = 6'd13,` This assigns a fixed binary encoding to state ST_SOFTMAX_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 65: `ST_SOFTMAX_LOAD = 6'd14,` This assigns a fixed binary encoding to state ST_SOFTMAX_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 66: `ST_SOFTMAX_OUTPUT = 6'd15,` This assigns a fixed binary encoding to state ST_SOFTMAX_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 67: `ST_LIF_LOAD = 6'd16,` This assigns a fixed binary encoding to state ST_LIF_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 68: `ST_LIF_CLEAR = 6'd17,` This assigns a fixed binary encoding to state ST_LIF_CLEAR, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 69: `ST_LIF_START = 6'd18,` This assigns a fixed binary encoding to state ST_LIF_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 70: `ST_LIF_OUTPUT = 6'd19,` This assigns a fixed binary encoding to state ST_LIF_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 71: `ST_OPTIMIZER_LOAD = 6'd20,` This assigns a fixed binary encoding to state ST_OPTIMIZER_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 72: `ST_OPTIMIZER_START = 6'd21,` This assigns a fixed binary encoding to state ST_OPTIMIZER_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 73: `ST_OPTIMIZER_OUTPUT = 6'd22,` This assigns a fixed binary encoding to state ST_OPTIMIZER_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 74: `ST_LUT_LOAD = 6'd23,` This assigns a fixed binary encoding to state ST_LUT_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 75: `ST_LUT_EVAL_LOAD = 6'd24,` This assigns a fixed binary encoding to state ST_LUT_EVAL_LOAD, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 76: `ST_LUT_EVAL_START = 6'd25,` This assigns a fixed binary encoding to state ST_LUT_EVAL_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 77: `ST_LUT_EVAL_OUTPUT = 6'd26,` This assigns a fixed binary encoding to state ST_LUT_EVAL_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 78: `ST_LUT_CLEAR = 6'd27,` This assigns a fixed binary encoding to state ST_LUT_CLEAR, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 79: `ST_ALPHA_DRAIN = 6'd28,` This assigns a fixed binary encoding to state ST_ALPHA_DRAIN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 80: `ST_OPTIMIZER_COMMIT = 6'd29,` This assigns a fixed binary encoding to state ST_OPTIMIZER_COMMIT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 81: `ST_VECTOR_DRAIN = 6'd30,` This assigns a fixed binary encoding to state ST_VECTOR_DRAIN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 82: `ST_DECODE_COMMAND = 6'd31,` This assigns a fixed binary encoding to state ST_DECODE_COMMAND, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 83: `ST_TRAIN_START = 6'd32,` This assigns a fixed binary encoding to state ST_TRAIN_START, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 84: `ST_TRAIN_RUN = 6'd33,` This assigns a fixed binary encoding to state ST_TRAIN_RUN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 85: `ST_OUTPUT_DRAIN = 6'd34` This statement continues through source line 86; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_OUTPUT_DRAIN, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 86: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 85. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_OUTPUT_DRAIN, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 88: `// One-hot recoding turns every engine-select decision into a single state` Design intent: One-hot recoding turns every engine-select decision into a single state This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 89: `// bit instead of a shared five-bit decode with hundreds of loads.` Design intent: bit instead of a shared five-bit decode with hundreds of loads. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 90: `(* fsm_encoding = "one_hot", extract_enable = "no" *) state_t state;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are fsm_encoding: the value named 'fsm encoding' in this block; one_hot: the value named 'one hot' in this block; extract_enable: a control bit that permits the associated state or storage update.

Line 91: `logic status_error_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_error_r is the value named 'status error r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 92: `(* dont_touch = "yes" *) logic error_event_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; error_event_r: the value named 'error event r' in this block.

Line 93: `(* dont_touch = "yes" *) logic clear_error_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; clear_error_r: the value named 'clear error r' in this block.

Line 94: `(* dont_touch = "yes" *) logic stream_protocol_error_event_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; stream_protocol_error_event_r: the value named 'stream protocol error event r' in this block.

Line 95: `(* dont_touch = "yes" *) logic clear_counters_event_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; clear_counters_event_r: a hardware telemetry register that accumulates measured events.

Line 96: `(* dont_touch = "yes" *) logic command_done_pending_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; command_done_pending_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 97: `(* dont_touch = "yes" *) logic input_beat_accepted_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; input_beat_accepted_r: the value named 'input beat accepted r' in this block.

Line 98: `(* dont_touch = "yes" *) logic output_beat_accepted_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; output_beat_accepted_r: the value named 'output beat accepted r' in this block.

Line 99: `logic [7:0] stream_beat_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stream_beat_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 100: `logic [11:0] stream_item_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stream_item_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 101: `logic [11:0] stream_items_remaining_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stream_items_remaining_r is the value named 'stream items remaining r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 102: `logic stream_last_pending_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stream_last_pending_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 103: `logic [3:0] output_word_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_word_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 104: `wire [9:0] lif_trace_bit_offset =` This statement continues through source line 105; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_trace_bit_offset is the value named 'lif trace bit offset' in this block; output_word_index} is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 105: `({6'd0, output_word_index} - 10'd1) << 6;` This is continuation line 2 of the 2-line logical statement that begins at source line 104. Its exact fragment is '({6'd0, output_word_index} - 10'd1) << 6;'. The important operands are d0: the value named 'd0' in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry; d1: the value named 'd1' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_trace_bit_offset is the value named 'lif trace bit offset' in this block; output_word_index} is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 106: `logic [108:0] captured_command_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, captured_command_r is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 107: `logic [3:0] train_reserved_partial_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_reserved_partial_r is the value named 'train reserved partial r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 108: `logic train_reserved_nonzero_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_reserved_nonzero_r is the value named 'train reserved nonzero r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 110: `typedef enum logic [3:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 111: `RESULT_TENSOR = 4'd0,` This assigns a fixed binary encoding to state RESULT_TENSOR, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 112: `RESULT_ALPHA = 4'd1,` This assigns a fixed binary encoding to state RESULT_ALPHA, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 113: `RESULT_VECTOR = 4'd2,` This assigns a fixed binary encoding to state RESULT_VECTOR, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 114: `RESULT_LAYERNORM = 4'd3,` This assigns a fixed binary encoding to state RESULT_LAYERNORM, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 115: `RESULT_SOFTMAX = 4'd4,` This assigns a fixed binary encoding to state RESULT_SOFTMAX, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 116: `RESULT_LIF = 4'd5,` This assigns a fixed binary encoding to state RESULT_LIF, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 117: `RESULT_OPTIMIZER = 4'd6,` This assigns a fixed binary encoding to state RESULT_OPTIMIZER, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 118: `RESULT_TRAIN = 4'd7,` This assigns a fixed binary encoding to state RESULT_TRAIN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 119: `RESULT_LUT = 4'd8` This statement continues through source line 120; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state RESULT_LUT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 120: `} result_source_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 119. Its exact fragment is '} result_source_t;'. The important operands are result_source_t: the value named 'result source t' in this block. Read together, the complete operation is: This assigns a fixed binary encoding to state RESULT_LUT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 122: `// Each engine writes a private result register before the shared output` Design intent: Each engine writes a private result register before the shared output This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 123: `// mux. The following elastic register then arbitrates those local banks.` Design intent: mux. The following elastic register then arbitrates those local banks. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 124: `// Splitting formatting and arbitration across clocks removes the former` Design intent: Splitting formatting and arbitration across clocks removes the former This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 125: `// child-to-wrapper nine-way combinational path while preserving one result` Design intent: child-to-wrapper nine-way combinational path while preserving one result This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 126: `// beat per clock after the two stages fill.` Design intent: beat per clock after the two stages fill. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 127: `logic result_input_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 128: `logic result_input_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_input_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 129: `logic result_prestage_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_prestage_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 130: `result_source_t result_prestage_source_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are result_source_t: the value named 'result source t' in this block; result_prestage_source_r: the value named 'result prestage source r' in this block.

Line 131: `logic result_prestage_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_prestage_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 132: `logic [63:0] result_tensor_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_tensor_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 133: `logic [63:0] result_alpha_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_alpha_data_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 134: `logic [63:0] result_vector_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_vector_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 135: `logic [63:0] result_layernorm_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_layernorm_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 136: `logic [63:0] result_softmax_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_softmax_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 137: `logic [63:0] result_lif_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_lif_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 138: `logic [63:0] result_optimizer_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_optimizer_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 139: `logic [63:0] result_train_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_train_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 140: `logic [63:0] result_lut_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_lut_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 141: `logic result_output_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_output_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 142: `logic [63:0] result_output_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_output_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 143: `logic result_output_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_output_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 144: `logic command_had_output_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_had_output_r is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 145: `logic final_output_accepted_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, final_output_accepted_r is the value named 'final output accepted r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 146: `logic completion_error_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, completion_error_r is the value named 'completion error r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 148: `wire result_output_ready = !result_output_valid_r || m_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 149: `wire result_prestage_ready = !result_prestage_valid_r` This statement continues through source line 150; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_prestage_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 150: `|| result_output_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 149. Its exact fragment is '|| result_output_ready;'. The important operands are result_output_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_prestage_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 151: `wire result_input_accept = result_input_valid && result_prestage_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_input_accept is the value named 'result input accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 152: `wire result_output_accept = result_output_valid_r && m_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_output_accept is the value named 'result output accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 154: `wire command_is_tensor = captured_command_r[3:0] >= CMD_LOAD_ACTIVATIONS` This statement continues through source line 155; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_is_tensor is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 155: `&& captured_command_r[3:0] <= CMD_COMPUTE_TERNARY;` This is continuation line 2 of the 2-line logical statement that begins at source line 154. Its exact fragment is '&& captured_command_r[3:0] <= CMD_COMPUTE_TERNARY;'. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; CMD_COMPUTE_TERNARY: the value named 'cmd compute ternary' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_is_tensor is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 156: `wire command_routes_tensor = command_is_tensor` This statement continues through source line 157; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_routes_tensor is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 157: `|| captured_command_r[3:0] == CMD_CLEAR_STATUS;` This is continuation line 2 of the 2-line logical statement that begins at source line 156. Its exact fragment is '|| captured_command_r[3:0] == CMD_CLEAR_STATUS;'. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; CMD_CLEAR_STATUS: the value named 'cmd clear status' in this block. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_routes_tensor is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 159: `logic tensor_command_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_command_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 160: `wire tensor_command_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_command_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 161: `wire tensor_command_done;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_command_done is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 162: `wire tensor_status_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_status_error is the value named 'tensor status error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 163: `logic tensor_s_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_s_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 164: `wire tensor_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_s_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 165: `logic tensor_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_m_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 166: `wire tensor_m_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_m_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 167: `wire [63:0] tensor_m_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_m_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 168: `wire tensor_m_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_m_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 170: `neuraedge_buffered_tensor_core u_tensor_core (` This instantiates module neuraedge_buffered_tensor_core as u_tensor_core. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 171: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port ‘clk‘ to parent expression ‘clk), .rst_n(rst_n‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 172: `.command_valid(tensor_command_valid), .command_ready(tensor_command_ready),` This connects the child instance's named port ‘command_valid‘ to parent expression ‘tensor_command_valid), .command_ready(tensor_command_r The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 173: `.command_data(captured_command_r[41:0]), .command_done(tensor_command_done),` This connects the child instance's named port ‘command_data‘ to parent expression ‘captured_command_r[41:0]), .command_done(tensor_comm The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 174: `.status_error(tensor_status_error), .status_busy(),` This connects the child instance's named port ‘status_error‘ to parent expression ‘tensor_status_error), .status_busy(‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'tensor status error), .status busy(' in this block.

Line 175: `.s_axis_tvalid(tensor_s_valid), .s_axis_tready(tensor_s_ready),` This connects the child instance's named port ‘s_axis_tvalid‘ to parent expression ‘tensor_s_valid), .s_axis_tready(tensor_s_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 176: `.s_axis_tdata(s_axis_tdata), .s_axis_tlast(s_axis_tlast),` This connects the child instance's named port ‘s_axis_tdata‘ to parent expression ‘s_axis_tdata), .s_axis_tlast(s_axis_tlast‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 177: `.m_axis_tvalid(tensor_m_valid), .m_axis_tready(tensor_m_ready),` This connects the child instance's named port ‘m_axis_tvalid‘ to parent expression ‘tensor_m_valid), .m_axis_tready(tensor_m_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 178: `.m_axis_tdata(tensor_m_data), .m_axis_tlast(tensor_m_last),` This connects the child instance's named port ‘m_axis_tdata‘ to parent expression ‘tensor_m_data), .m_axis_tlast(tensor_m_last‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 179: `.status_loaded_activations(debug_tensor_loaded_activations),` This connects the child instance's named port 'status_loaded_activations' to parent expression 'debug_tensor_loaded_activations'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 180: `.status_loaded_weights(debug_tensor_loaded_weights),` This connects the child instance's named port 'status_loaded_weights' to parent expression 'debug_tensor_loaded_weights'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 181: `.status_active_issues(debug_tensor_active_issues),` This connects the child instance's named port 'status_active_issues' to parent expression 'debug_tensor_active_issues'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug tensor active issues' in this block.

Line 182: `.status_input_beats(),` This connects the child instance's named port 'status_input_beats' to parent expression ''. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status input beats' in this block.

Line 183: `.status_output_beats(),` This connects the child instance's named port 'status_output_beats' to parent expression ''. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status output beats' in this block.

Line 184: `.status_compute_cycles(debug_tensor_compute_cycles),` This connects the child instance's named port 'status_compute_cycles' to parent expression 'debug_tensor_compute_cycles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug tensor compute cycles' in this block.

Line 185: `.status_skipped_issues(debug_tensor_skipped_issues),` This connects the child instance's named port 'status_skipped_issues' to parent expression 'debug_tensor_skipped_issues'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug tensor skipped issues' in this block.

Line 186: `.status_dense_macs(debug_tensor_dense_macs),` This connects the child instance's named port 'status_dense_macs' to parent expression 'debug_tensor_dense_macs'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a count of ordinary activation-times-weight operations actually issued.

Line 187: `.status_spike_adds(debug_tensor_spike_adds)` This connects the child instance's named port 'status_spike_adds' to parent expression 'debug_tensor_spike_adds'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 188: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 190: `logic [255:0] alpha_query_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_query_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 191: `logic [255:0] alpha_key_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_key_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 192: `logic [8:0] alpha_active_bits_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_active_bits_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 193: `logic [8:0] alpha_q8_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_q8_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 194: `logic alpha_load_valid_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_load_valid_r is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 195: `logic [2:0] alpha_load_index_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_load_index_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 196: `logic [63:0] alpha_load_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_load_data_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 197: `logic alpha_load_final_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_load_final_r is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 198: `logic alpha_load_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_load_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 199: `logic alpha_input_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 200: `wire alpha_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 201: `wire alpha_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 202: `logic alpha_output_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 203: `wire [17:0] alpha_score;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_score is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 204: `wire [8:0] alpha_spike_matches;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_spike_matches is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 205: `wire [8:0] alpha_zero_matches;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_zero_matches is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 206: `wire alpha_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_config_error is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 208: `neuraedge_alpha_xnor u_alpha_xnor (` This instantiates module neuraedge_alpha_xnor as u_alpha_xnor. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 209: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 210: `.input_valid(alpha_input_valid), .input_ready(alpha_input_ready),` This connects the child instance's named port 'input_valid' to parent expression 'alpha_input_valid), .input_ready(alpha_input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 211: `.query_spikes(alpha_query_r), .key_spikes(alpha_key_r),` This connects the child instance's named port 'query_spikes' to parent expression 'alpha_query_r), .key_spikes(alpha_key_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 212: `.active_bits(alpha_active_bits_r), .alpha_q8(alpha_q8_r),` This connects the child instance's named port 'active_bits' to parent expression 'alpha_active_bits_r), .alpha_q8(alpha_q8_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 213: `.output_valid(alpha_output_valid), .output_ready(alpha_output_ready),` This connects the child instance's named port 'output_valid' to parent expression 'alpha_output_valid), .output_ready(alpha_output_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 214: `.score_q8(alpha_score), .spike_matches(alpha_spike_matches),` This connects the child instance's named port 'score_q8' to parent expression 'alpha_score), .spike_matches(alpha_spike_matches'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 215: `.zero_matches(alpha_zero_matches), .config_error(alpha_config_error)` This connects the child instance's named port 'zero_matches' to parent expression 'alpha_zero_matches), .config_error(alpha_config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 216: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 218: `logic [3:0] vector_operation_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_operation_r is the value named 'vector operation r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 219: `logic [255:0] vector_operand_a_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_operand_a_r is the value named 'vector operand a r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 220: `logic [255:0] vector_operand_b_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_operand_b_r is the value named 'vector operand b r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 221: `logic signed [15:0] vector_multiplier_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_multiplier_r is the value named 'vector multiplier r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 222: `logic [4:0] vector_shift_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_shift_r is the value named 'vector shift r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 223: `logic signed [15:0] vector_zero_point_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_zero_point_r is the integer code representing real numeric zero in affine quantization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 224: `logic signed [31:0] vector_threshold_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_threshold_r is the membrane level at which a neuron emits a spike. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 225: `logic [31:0] vector_window_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_window_r is the value named 'vector window r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 226: `logic signed [31:0] vector_clamp_min_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_clamp_min_r is the value named 'vector clamp min r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 227: `logic signed [31:0] vector_clamp_max_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_clamp_max_r is the value named 'vector clamp max r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 228: `(* dont_touch = "yes" *) logic vector_load_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; vector_load_valid_r: the producer's statement that its current payload is meaningful.

Line 229: `logic [3:0] vector_load_index_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_load_index_r is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 230: `logic [63:0] vector_load_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_load_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 231: `logic vector_load_final_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_load_final_r is the value named 'vector load final r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 232: `logic vector_load_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_load_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 233: `logic vector_input_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 234: `wire vector_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 235: `wire vector_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 236: `logic vector_output_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 237: `wire [255:0] vector_result;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_result is the value named 'vector result' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 238: `wire vector_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_config_error is the value named 'vector config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 240: `neuraedge_vector_alu u_vector_alu (` This instantiates module neuraedge_vector_alu as u_vector_alu. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 241: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 242: `.input_valid(vector_input_valid), .input_ready(vector_input_ready),` This connects the child instance's named port 'input_valid' to parent expression 'vector_input_valid), .input_ready(vector_input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 243: `.operation(vector_operation_r), .operand_a(vector_operand_a_r),` This connects the child instance's named port 'operation' to parent expression 'vector_operation_r), .operand_a(vector_operand_a_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'vector operation r), .operand a(vector operand a r' in this block.

Line 244: `.operand_b(vector_operand_b_r), .multiplier_q15(vector_multiplier_r),` This connects the child instance's named port 'operand_b' to parent expression 'vector_operand_b_r), .multiplier_q15(vector_multiplier_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'vector operand b r), .multiplier q15(vector multiplier r' in this block.

Line 245: `.right_shift(vector_shift_r), .output_zero_point(vector_zero_point_r),` This connects the child instance's named port ‘right_shift‘ to parent expression ‘vector_shift_r), .output_zero_point(vector_zero_point_r‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the integer code representing real numeric zero in affine quantization.

Line 246: `.threshold(vector_threshold_r), .surrogate_window(vector_window_r),` This connects the child instance's named port ‘threshold‘ to parent expression ‘vector_threshold_r), .surrogate_window(vector_window_r‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the membrane level at which a neuron emits a spike.

Line 247: `.clamp_minimum(vector_clamp_min_r), .clamp_maximum(vector_clamp_max_r),` This connects the child instance's named port ‘clamp_minimum‘ to parent expression ‘vector_clamp_min_r), .clamp_maximum(vector_clamp_max_r‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named ’vector clamp min r), .clamp maximum(vector clamp max r’ in this block.

Line 248: `.output_valid(vector_output_valid), .output_ready(vector_output_ready),` This connects the child instance's named port ‘output_valid‘ to parent expression ‘vector_output_valid), .output_ready(vector_output_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 249: `.result(vector_result), .config_error(vector_config_error)` This connects the child instance's named port ‘result‘ to parent expression ‘vector_result), .config_error(vector_config_error‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named ’vector result), .config error(vector config error’ in this block.

Line 250: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 252: `logic [11:0] ln_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 253: `logic [31:0] ln_epsilon_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_epsilon_r is the value named ’ln epsilon r’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 254: `logic [63:0] ln_sample_word0_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_word0_r is the value named ’ln sample word0 r’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 255: `logic signed [23:0] ln_sample_value_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_value_r is the value named ’ln sample value r’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 256: `logic signed [23:0] ln_sample_gamma_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_gamma_r is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 257: `logic signed [23:0] ln_sample_beta_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_beta_r is the learned additive LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 258: `logic ln_start_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_start_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 259: `wire ln_start_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_start_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 260: `logic ln_sample_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 261: `wire ln_sample_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 262: `logic ln_sample_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 263: `wire ln_result_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 264: `logic ln_result_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 265: `wire signed [31:0] ln_result_value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_value is the value named 'ln result value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 266: `wire [11:0] ln_result_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 267: `wire ln_result_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 268: `wire ln_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_config_error is the value named 'ln config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 269: `logic ln_start_accept_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_start_accept_r is the value named 'ln start accept r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 271: `neuraedge_integer_layernorm u_layernorm (` This instantiates module neuraedge_integer_layernorm as u_layernorm. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 272: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 273: `.start_valid(ln_start_valid), .start_ready(ln_start_ready),` This connects the child instance's named port ‘start_valid‘ to parent expression ‘ln_start_valid), .start_ready(ln_start_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 274: `.feature_count(ln_count_r), .epsilon(ln_epsilon_r),` This connects the child instance's named port ‘feature_count‘ to parent expression ‘ln_count_r), .epsilon(ln_epsilon_r‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'ln count r), .epsilon(ln epsilon r' in this block.

Line 275: `.sample_valid(ln_sample_valid), .sample_ready(ln_sample_ready),` This connects the child instance's named port ‘sample_valid‘ to parent expression ‘ln_sample_valid), .sample_ready(ln_sample_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 276: `.sample_value(ln_sample_value_r), .sample_gamma_q15(ln_sample_gamma_r),` This connects the child instance's named port ‘sample_value‘ to parent expression ‘ln_sample_value_r), .sample_gamma_q15(ln_sample_gamma_r‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the learned multiplicative LayerNorm parameter.

Line 277: `.sample_beta(ln_sample_beta_r), .sample_last(ln_sample_last),` This connects the child instance's named port ‘sample_beta‘ to parent expression ‘ln_sample_beta_r), .sample_last(ln_sample_last‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 278: `.result_valid(ln_result_valid), .result_ready(ln_result_ready),` This connects the child instance's named port ‘result_valid‘ to parent expression ‘ln_result_valid), .result_ready(ln_result_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 279: `.result_value(ln_result_value), .result_index(ln_result_index),` This connects the child instance's named port ‘result_value‘ to parent expression ‘ln_result_value), .result_index(ln_result_index‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an integer selecting one tensor element, lane, bank, or state entry.

Line 280: `.result_last(ln_result_last), .config_error(ln_config_error),` This connects the child instance's named port ‘result_last‘ to parent expression ‘ln_result_last), .config_error(ln_config_error‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 281: `.cycle_count(debug_layernorm_cycles),` This connects the child instance's named port ‘cycle_count‘ to parent expression ‘debug_layernorm_cycles‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug layernorm cycles' in this block.

Line 282: `.debug_mean(debug_layernorm_mean),` This connects the child instance's named port ‘debug_mean‘ to parent expression ‘debug_layernorm_mean‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the arithmetic average used by normalization.

Line 283: `.debug_variance(debug_layernorm_variance),` This connects the child instance's named port ‘debug_variance‘ to parent expression ‘debug_layernorm_variance‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the average squared distance from the mean.

Line 284: `.debug_sigma(debug_layernorm_sigma),` This connects the child instance's named port ‘debug_sigma‘ to parent expression ‘debug_layernorm_sigma‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug layernorm sigma' in this block.

Line 285: `.debug_reciprocal_q31(debug_layernorm_reciprocal)` This connects the child instance's named port 'debug_reciprocal_q31' to parent expression 'debug_layernorm_reciprocal'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug layernorm reciprocal' in this block.

Line 286: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 288: `logic [8:0] softmax_count_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_count_r is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 289: `logic [4:0] softmax_shift_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_shift_r is the value named 'softmax shift r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 290: `logic softmax_start_accept_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_start_accept_r is the value named 'softmax start accept r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 291: `logic softmax_start_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_start_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 292: `wire softmax_start_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_start_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 293: `logic softmax_score_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_score_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 294: `wire softmax_score_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_score_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 295: `logic signed [23:0] softmax_score_value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_score_value is the value named 'softmax score value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 296: `logic softmax_score_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_score_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 297: `wire softmax_probability_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_probability_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 298: `logic softmax_probability_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_probability_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 299: `wire [15:0] softmax_probability;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_probability is the value named 'softmax probability' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 300: `wire [8:0] softmax_probability_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_probability_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 301: `wire softmax_probability_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_probability_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 302: `wire softmax_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_config_error is the value named 'softmax config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 304: `neuraedge_log_softmax u_softmax (` This instantiates module neuraedge_log_softmax as u_softmax. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 305: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 306: `.start_valid(softmax_start_valid), .start_ready(softmax_start_ready),` This connects the child instance's named port 'start_valid' to parent expression 'softmax_start_valid), .start_ready(softmax_start_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 307: `.item_count(softmax_count_r), .difference_shift(softmax_shift_r),` This connects the child instance's named port 'item_count' to parent expression 'softmax_count_r), .difference_shift(softmax_shift_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'softmax count r), .difference shift(softmax shift r' in this block.

Line 308: `.score_valid(softmax_score_valid), .score_ready(softmax_score_ready),` This connects the child instance's named port 'score_valid' to parent expression 'softmax_score_valid), .score_ready(softmax_score_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 309: `.score_value(softmax_score_value), .score_last(softmax_score_last),` This connects the child instance's named port 'score_value' to parent expression 'softmax_score_value), .score_last(softmax_score_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 310: `.probability_valid(softmax_probability_valid),` This connects the child instance's named port 'probability_valid' to parent expression 'softmax_probability_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 311: `.probability_ready(softmax_probability_ready),` This connects the child instance's named port 'probability_ready' to parent expression 'softmax_probability_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 312: `.probability_q15(softmax_probability),` This connects the child instance's named port 'probability_q15' to parent expression 'softmax_probability'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'softmax probability' in this block.

Line 313: `.probability_index(softmax_probability_index),` This connects the child instance's named port 'probability_index' to parent expression 'softmax_probability_index'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an integer selecting one tensor element, lane, bank, or state entry.

Line 314: `.probability_last(softmax_probability_last), .config_error(softmax_config_error),` This connects the child instance's named port 'probability_last' to parent expression 'softmax_probability_last), .config_error(softmax_con The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 315: `.cycle_count(debug_softmax_cycles),` This connects the child instance's named port 'cycle_count' to parent expression 'debug_softmax_cycles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug softmax cycles' in this block.

Line 316: `.debug_maximum_score(debug_softmax_maximum),` This connects the child instance's named port 'debug_maximum_score' to parent expression 'debug_softmax_maximum'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug softmax maximum' in this block.

Line 317: `.debug_denominator(debug_softmax_denominator),` This connects the child instance's named port 'debug_denominator' to parent expression 'debug_softmax_denominator'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug softmax denominator' in this block.

Line 318: `.debug_reciprocal_q31(debug_softmax_reciprocal)` This connects the child instance's named port 'debug_reciprocal_q31' to parent expression 'debug_softmax_reciprocal'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug softmax reciprocal' in this block.

Line 319: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 321: `logic [511:0] lif_current_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_current_r is the value named 'lif current r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 322: `logic [4:0] lif_leak_shift_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_leak_shift_r is the retained fraction of prior membrane state. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 323: `logic signed [23:0] lif_threshold_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_threshold_r is the membrane level at which a neuron emits a spike. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 324: `logic signed [23:0] lif_reset_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_reset_r is the value named 'lif reset r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 325: `logic lif_clear_requested_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_clear_requested_r is the value named 'lif clear requested r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 326: `logic [7:0] lif_load_word_select_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_load_word_select_r is the value named 'lif load word select r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 327: `logic lif_load_pending_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_load_pending_r is the value named 'lif load pending r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 328: `logic [2:0] lif_load_index_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_load_index_r is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 329: `logic [63:0] lif_load_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_load_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 330: `logic lif_load_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_load_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 331: `logic lif_load_final_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_load_final_r is the value named 'lif load final r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 332: `logic lif_clear_state;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_clear_state is the finite-state-machine phase retained in flip-flops. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 333: `logic lif_input_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 334: `wire lif_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 335: `wire lif_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 336: `logic lif_output_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 337: `wire [31:0] lif_spikes;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_spikes is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 338: `wire [32*24-1:0] lif_pre_reset_membrane;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_pre_reset_membrane is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 339: `wire [31:0] lif_timesteps;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_timesteps is one discrete SNN update instant. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 340: `logic lif_trace_requested_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_trace_requested_r is the value named 'lif trace requested r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 342: `neuraedge_lif_bank u_lif_bank (` This instantiates module neuraedge_lif_bank as u_lif_bank. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 343: `.clk(clk), .rst_n(rst_n), .clear_state(lif_clear_state),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n), .clear_state(lif_clear_state'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 344: `.clear_counters(clear_counters_event_r),` This connects the child instance's named port 'clear_counters' to parent expression 'clear_counters_event_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a hardware telemetry register that accumulates measured events.

Line 345: `.input_valid(lif_input_valid), .input_ready(lif_input_ready),` This connects the child instance's named port 'input_valid' to parent expression 'lif_input_valid), .input_ready(lif_input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 346: `.current_vector(lif_current_r), .leak_shift(lif_leak_shift_r),` This connects the child instance's named port 'current_vector' to parent expression 'lif_current_r), .leak_shift(lif_leak_shift_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the retained fraction of prior membrane state.

Line 347: `.threshold(lif_threshold_r), .reset_value(lif_reset_r),` This connects the child instance's named port 'threshold' to parent expression 'lif_threshold_r), .reset_value(lif_reset_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the membrane level at which a neuron emits a spike.

Line 348: `.output_valid(lif_output_valid), .output_ready(lif_output_ready),` This connects the child instance's named port 'output_valid' to parent expression 'lif_output_valid), .output_ready(lif_output_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 349: `.spikes(lif_spikes), .membrane_vector(),` This connects the child instance's named port 'spikes' to parent expression 'lif_spikes), .membrane_vector('. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the persistent LIF neuron state carrying charge between timesteps.

Line 350: `.pre_reset_membrane_vector(lif_pre_reset_membrane),` This connects the child instance's named port 'pre_reset_membrane_vector' to parent expression 'lif_pre_reset_membrane'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the persistent LIF neuron state carrying charge between timesteps.

Line 351: `.timestep_count(lif_timesteps),` This connects the child instance's named port 'timestep_count' to parent expression 'lif_timesteps'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one discrete SNN update instant.

Line 352: `.total_spike_count(debug_lif_total_spikes)` This connects the child instance's named port 'total_spike_count' to parent expression 'debug_lif_total_spikes'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 353: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 355: `logic [127:0] optimizer_weight_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_weight_r is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 356: `logic [191:0] optimizer_gradient_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_gradient_r is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 357: `logic [191:0] optimizer_velocity_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_velocity_r is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 358: `logic [17:0] optimizer_lr_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_lr_r is the value named 'optimizer lr r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 359: `logic [16:0] optimizer_momentum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_momentum_r is the value named 'optimizer momentum r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 360: `logic [16:0] optimizer_decay_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_decay_r is the value named 'optimizer decay r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 361: `logic [4:0] optimizer_update_shift_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_update_shift_r is the value named 'optimizer update shift r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 362: `logic [63:0] optimizer_load_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_load_data_r is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 363: `logic optimizer_load_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_load_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 364: `logic [2:0] optimizer_load_index_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_load_index_r is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 365: `logic optimizer_load_final_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_load_final_r is the value named 'optimizer load final r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 366: `(* dont_touch = "yes" *) logic optimizer_commit_valid_r;` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are dont_touch: the value named 'dont touch' in this block; yes: the value named 'yes' in this block; optimizer_commit_valid_r: the producer's statement that its current payload is meaningful.

Line 367: `logic optimizer_input_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 368: `wire optimizer_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 369: `wire optimizer_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 370: `logic optimizer_output_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 371: `wire [127:0] optimizer_updated_weight;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_updated_weight is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 372: `wire [191:0] optimizer_updated_velocity;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_updated_velocity is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 373: `wire optimizer_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_config_error is the value named 'optimizer config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 375: `neuraedge_sgd_optimizer8 u_optimizer (` This instantiates module neuraedge_sgd_optimizer8 as u_optimizer. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 376: `.clk(clk), .rst_n(rst_n), .clear_counters(clear_counters_event_r),` This connects the child instance's named port ‘clk‘ to parent expression ‘clk), .rst_n(rst_n), .clear_counters(clear_counters_event_r‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 377: `.input_valid(optimizer_input_valid), .input_ready(optimizer_input_ready),` This connects the child instance's named port ‘input_valid‘ to parent expression ‘optimizer_input_valid), .input_ready(optimizer_input_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 378: `.weight_vector(optimizer_weight_r), .gradient_vector(optimizer_gradient_r),` This connects the child instance's named port ‘weight_vector‘ to parent expression ‘optimizer_weight_r), .gradient_vector(optimizer_gradient_r‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 379: `.velocity_vector(optimizer_velocity_r), .learning_rate_q16(optimizer_lr_r),` This connects the child instance's named port ‘velocity_vector‘ to parent expression ‘optimizer_velocity_r), .learning_rate_q16(optimizer_lr_r‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the momentum history retained by the SGD optimizer.

Line 380: `.momentum_q16(optimizer_momentum_r), .weight_decay_q16(optimizer_decay_r),` This connects the child instance's named port ‘momentum_q16‘ to parent expression ‘optimizer_momentum_r), .weight_decay_q16(optimizer_decay
The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 381: `.update_right_shift(optimizer_update_shift_r),` This connects the child instance's named port 'update_right_shift' to parent expression 'optimizer_update_shift_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'optimizer update shift r' in this block.

Line 382: `.output_valid(optimizer_output_valid), .output_ready(optimizer_output_ready),` This connects the child instance's named port 'output_valid' to parent expression 'optimizer_output_valid), .output_ready(optimizer_output_re The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 383: `.updated_weight_vector(optimizer_updated_weight),` This connects the child instance's named port 'updated_weight_vector' to parent expression 'optimizer_updated_weight'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 384: `.updated_velocity_vector(optimizer_updated_velocity),` This connects the child instance's named port 'updated_velocity_vector' to parent expression 'optimizer_updated_velocity'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the momentum history retained by the SGD optimizer.

Line 385: `.config_error(optimizer_config_error),` This connects the child instance's named port 'config_error' to parent expression 'optimizer_config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'optimizer config error' in this block.

Line 386: `.update_count(debug_optimizer_updates)` This connects the child instance's named port 'update_count' to parent expression 'debug_optimizer_updates'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug optimizer updates' in this block.

Line 387: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 389: `logic train_start_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_start_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 390: `wire train_start_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_start_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 391: `logic train_s_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_s_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 392: `wire train_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_s_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 393: `wire train_m_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_m_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 394: `logic train_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_m_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 395: `wire [63:0] train_m_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_m_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 396: `wire train_m_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_m_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 397: `wire train_done;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_done is the value named 'train done' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 398: `wire train_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_config_error is the value named 'train config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 399: `wire train_busy;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_busy is the value named 'train busy' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 401: `neuraedge_linear_backward8 u_linear_backward (` This instantiates module neuraedge_linear_backward8 as u_linear_backward. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 402: `.clk(clk), .rst_n(rst_n), .clear_counters(clear_counters_event_r),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n), .clear_counters(clear_counters_event_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 403: `.start_valid(train_start_valid), .start_ready(train_start_ready),` This connects the child instance's named port 'start_valid' to parent expression 'train_start_valid), .start_ready(train_start_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 404: `.active_rows(captured_command_r[7:4]),` This connects the child instance's named port 'active_rows' to parent expression 'captured_command_r[7:4]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 405: `.active_inputs(captured_command_r[11:8]),` This connects the child instance's named port 'active_inputs' to parent expression 'captured_command_r[11:8]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 406: `.active_outputs(captured_command_r[15:12]),` This connects the child instance's named port 'active_outputs' to parent expression 'captured_command_r[15:12]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 407: `.input_gradient_shift(captured_command_r[20:16]),` This connects the child instance's named port 'input_gradient_shift' to parent expression 'captured_command_r[20:16]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 408: `.weight_gradient_shift(captured_command_r[25:21]),` This connects the child instance's named port 'weight_gradient_shift' to parent expression 'captured_command_r[25:21]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 409: `.gradient_mode(captured_command_r[27:26]),` This connects the child instance's named port 'gradient_mode' to parent expression 'captured_command_r[27:26]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 410: `.accumulate_result(captured_command_r[28]),` This connects the child instance's named port 'accumulate_result' to parent expression 'captured_command_r[28]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 411: `.suppress_output(captured_command_r[29]),` This connects the child instance's named port 'suppress_output' to parent expression 'captured_command_r[29]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 412: `.reserved_nonzero(train_reserved_nonzero_r),` This connects the child instance's named port 'reserved_nonzero' to parent expression 'train_reserved_nonzero_r'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'train reserved nonzero r' in this block.

Line 413: `.s_axis_tvalid(train_s_valid), .s_axis_tready(train_s_ready),` This connects the child instance's named port 's_axis_tvalid' to parent expression 'train_s_valid), .s_axis_tready(train_s_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 414: `.s_axis_tdata(s_axis_tdata), .s_axis_tlast(s_axis_tlast),` This connects the child instance's named port 's_axis_tdata' to parent expression 's_axis_tdata), .s_axis_tlast(s_axis_tlast'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 415: `.m_axis_tvalid(train_m_valid), .m_axis_tready(train_m_ready),` This connects the child instance's named port 'm_axis_tvalid' to parent expression 'train_m_valid), .m_axis_tready(train_m_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 416: `.m_axis_tdata(train_m_data), .m_axis_tlast(train_m_last),` This connects the child instance's named port 'm_axis_tdata' to parent expression 'train_m_data), .m_axis_tlast(train_m_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 417: `.busy(train_busy), .done(train_done), .config_error(train_config_error),` This connects the child instance's named port 'busy' to parent expression 'train_busy), .done(train_done), .config_error(train_config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'train busy), .done(train done), .config error(train config error' in this block.

Line 418: `.completed_tiles(debug_training_tiles),` This connects the child instance's named port 'completed_tiles' to parent expression 'debug_training_tiles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug training tiles' in this block.

Line 419: `.active_cycles(debug_training_cycles)` This connects the child instance's named port 'active_cycles' to parent expression 'debug_training_cycles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug training cycles' in this block.

Line 420: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 422: `logic lut_clear;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_clear is the value named 'lut clear' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 423: `logic lut_load_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_load_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 424: `wire lut_load_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_load_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 425: `logic [7:0] lut_load_address;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_load_address is the value named 'lut load address' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 426: `logic signed [15:0] lut_load_value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_load_value is the value named 'lut load value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 427: `logic lut_load_pending_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_load_pending_r is the value named 'lut load pending r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 428: `logic [7:0] lut_load_address_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_load_address_r is the value named 'lut load address r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 429: `logic signed [15:0] lut_load_value_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_load_value_r is the value named 'lut load value r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 430: `logic lut_load_last_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_load_last_r is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 431: `logic lut_load_final_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_load_final_r is the value named 'lut load final r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 432: `logic lut_input_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 433: `wire lut_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 434: `logic [63:0] lut_indices_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_indices_r is the value named 'lut indices r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 435: `wire lut_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 436: `logic lut_output_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 437: `wire [127:0] lut_output_values;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_output_values is the value named 'lut output values' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 438: `wire lut_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_config_error is the value named 'lut config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 439: `logic lut_clear_requested_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_clear_requested_r is the value named 'lut clear requested r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 440: `logic decode_error_pending_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, decode_error_pending_r is the value named 'decode error pending r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 442: `neuraedge_activation_lut8 u_activation_lut (` This instantiates module neuraedge_activation_lut8 as u_activation_lut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 443: `.clk(clk), .rst_n(rst_n), .clear_table(lut_clear),` This connects the child instance's named port ‘clk‘ to parent expression ‘clk), .rst_n(rst_n), .clear_table(lut_clear‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 444: `.load_valid(lut_load_valid), .load_ready(lut_load_ready),` This connects the child instance's named port ‘load_valid‘ to parent expression ‘lut_load_valid), .load_ready(lut_load_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 445: `.load_address(lut_load_address), .load_value(lut_load_value),` This connects the child instance's named port ‘load_address‘ to parent expression ‘lut_load_address), .load_value(lut_load_value‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'lut load address), .load value(lut load value' in this block.

Line 446: `.input_valid(lut_input_valid), .input_ready(lut_input_ready),` This connects the child instance's named port ‘input_valid‘ to parent expression ‘lut_input_valid), .input_ready(lut_input_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 447: `.input_indices(lut_indices_r), .output_valid(lut_output_valid),` This connects the child instance's named port ‘input_indices‘ to parent expression ‘lut_indices_r), .output_valid(lut_output_valid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 448: `.output_ready(lut_output_ready), .output_values(lut_output_values),` This connects the child instance's named port 'output_ready' to parent expression 'lut_output_ready), .output_values(lut_output_values'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 449: `.config_error(lut_config_error),` This connects the child instance's named port 'config_error' to parent expression 'lut_config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'lut config error' in this block.

Line 450: `.loaded_entries(debug_lut_loaded_entries)` This connects the child instance's named port 'loaded_entries' to parent expression 'debug_lut_loaded_entries'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug lut loaded entries' in this block.

Line 451: `);` Within the module's structural declarations, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The expression is a literal or structural operator and has no hidden software state.

Line 453: `// Keep each engine's transfer event local. A shared OR-of-valids is fine` Design intent: Keep each engine's transfer event local. A shared OR-of-valids is fine This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 454: `// for the single accounting bit, but using it in unrelated FSM branches` Design intent: for the single accounting bit, but using it in unrelated FSM branches This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 455: `// creates false cross-engine control cones after one-hot optimization.` Design intent: creates false cross-engine control cones after one-hot optimization. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 456: `wire tensor_result_accept = state == ST_TENSOR` This statement continues through source line 457; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_result_accept is the value named 'tensor result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 457: `&& tensor_m_valid && result_prestage_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 456. Its exact fragment is '&& tensor_m_valid && result_prestage_ready;'. The important operands are tensor_m_valid: the producer's statement that its current payload is meaningful; result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, tensor_result_accept is the value named 'tensor result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 458: `wire alpha_result_accept = state == ST_ALPHA_OUTPUT` This statement continues through source line 459; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_result_accept is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 459: `&& alpha_output_valid && result_prestage_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 458. Its exact fragment is '&& alpha_output_valid && result_prestage_ready;'. The important operands are alpha_output_valid: the producer's statement that its current payload is meaningful; result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_result_accept is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 460: `wire vector_result_accept = state == ST_VECTOR_OUTPUT` This statement continues through source line 461; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_result_accept is the value named 'vector result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 461: `&& vector_output_valid && result_prestage_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 460. Its exact fragment is '&& vector_output_valid && result_prestage_ready;'. The important operands are vector_output_valid: the producer's statement that its current payload is meaningful; result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_result_accept is the value named 'vector result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 462: `wire layernorm_result_accept = state == ST_LAYERNORM_OUTPUT` This statement continues through source line 463; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, layernorm_result_accept is the value named 'layernorm result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 463: `&& ln_result_valid && result_prestage_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 462. Its exact fragment is '&& ln_result_valid && result_prestage_ready;'. The important operands are ln_result_valid: the producer's statement that its current payload is meaningful; result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, layernorm_result_accept is the value named 'layernorm result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 464: `wire softmax_result_accept = state == ST_SOFTMAX_OUTPUT` This statement continues through source line 465; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_result_accept is the value named 'softmax result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 465: `&& softmax_probability_valid && result_prestage_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 464. Its exact fragment is '&& softmax_probability_valid && result_prestage_ready;'. The important operands are softmax_probability_valid: the producer's statement that its current payload is meaningful; result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, softmax_result_accept is the value named 'softmax result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 466: `wire lif_result_accept = state == ST_LIF_OUTPUT` This statement continues through source line 467; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_result_accept is the value named 'lif result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 467: `&& lif_output_valid && result_prestage_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 466. Its exact fragment is '&& lif_output_valid && result_prestage_ready;'. The important operands are lif_output_valid: the producer's statement that its current payload is meaningful; result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lif_result_accept is the value named 'lif result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 468: `wire optimizer_result_accept = state == ST_OPTIMIZER_OUTPUT` This statement continues through source line 469; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_result_accept is the value named 'optimizer result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 469: `&& optimizer_output_valid && result_prestage_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 468. Its exact fragment is '&& optimizer_output_valid && result_prestage_ready;'. The important operands are optimizer_output_valid: the producer's statement that its current payload is meaningful; result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_result_accept is the value named 'optimizer result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 470: `wire train_result_accept = state == ST_TRAIN_RUN` This statement continues through source line 471; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_result_accept is the value named 'train result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 471: `&& train_m_valid && result_prestage_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 470. Its exact fragment is '&& train_m_valid && result_prestage_ready;'. The important operands are train_m_valid: the producer's statement that its current payload is meaningful; result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_result_accept is the value named 'train result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 472: `wire lut_result_accept = state == ST_LUT_EVAL_OUTPUT` This statement continues through source line 473; the explanation uses the complete expression rather than only the first visual fragment. This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_result_accept is the value named 'lut result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 473: `&& lut_output_valid && result_prestage_ready;` This is continuation line 2 of the 2-line logical statement that begins at source line 472. Its exact fragment is '&& lut_output_valid && result_prestage_ready;'. The important operands are lut_output_valid: the producer's statement that its current payload is meaningful; result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lut_result_accept is the value named 'lut result accept' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 475: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 476: `command_ready = state == ST_IDLE;` This makes command_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_IDLE'. It is driven by combinational or procedural logic in the current

evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 477: `status_busy = state != ST_IDLE || train_busy;` This makes status_busy, the value named 'status busy' in this block, equal to 'state != ST_IDLE || train_busy'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block; train_busy: the value named 'train busy' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 478: `status_error = status_error_r || tensor_status_error;` This makes status_error, the value named 'status error' in this block, equal to 'status_error_r || tensor_status_error'. It is driven by combinational or procedural logic in the current evaluation. The important operands are status_error_r: the value named 'status error r' in this block; tensor_status_error: the value named 'tensor status error' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 479: `s_axis_tready = 1'b0;` This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 480: `m_axis_tvalid = result_output_valid_r;` This makes m_axis_tvalid, the producer's statement that its current payload is meaningful, equal to 'result_output_valid_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_output_valid_r: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 481: `m_axis_tdata = result_output_data_r;` This makes m_axis_tdata, the value named 'm axis tdata' in this block, equal to 'result_output_data_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_output_data_r: the payload carried with the surrounding valid/ready handshake. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 482: `m_axis_tlast = result_output_last_r;` This makes m_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'result_output_last_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_output_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 483: `result_input_valid = 1'b0;` This makes result_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 484: `result_input_last = 1'b0;` This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 486: `tensor_command_valid = state == ST_DECODE_COMMAND && command_routes_tensor;` This makes tensor_command_valid, the producer's statement that its current payload is meaningful, equal to 'state == ST_DECODE_COMMAN && command_routes_tensor'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_DECODE_COMMAND: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command_routes_tensor: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 487: `tensor_s_valid = 1'b0;` This makes tensor_s_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 488: `tensor_m_ready = 1'b0;` This makes tensor_m_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 489: `alpha_input_valid = 1'b0;` This makes alpha_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 490: `alpha_output_ready = 1'b0;` This makes alpha_output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 491: `vector_input_valid = 1'b0;` This makes vector_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 492: `vector_output_ready = 1'b0;` This makes vector_output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 493: `ln_start_valid = 1'b0;` This makes ln_start_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 494: `ln_sample_valid = 1'b0;` This makes ln_sample_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 495: `ln_sample_last = stream_last_pending_r;` This makes ln_sample_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'stream_last_pending_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are stream_last_pending_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 496: `ln_result_ready = 1'b0;` This makes ln_result_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 497: `softmax_start_valid = 1'b0;` This makes softmax_start_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 498: `softmax_score_valid = 1'b0;` This makes softmax_score_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 499: `softmax_score_value = s_axis_tdata[23:0];` This makes softmax_score_value, the value named 'softmax score value' in this block, equal to 's_axis_tdata[23:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 500: `softmax_score_last = stream_last_pending_r;` This makes softmax_score_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'stream_last_pending_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are stream_last_pending_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 501: `softmax_probability_ready = 1'b0;` This makes softmax_probability_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 502: `lif_clear_state = 1'b0;` This makes lif_clear_state, the finite-state-machine phase retained in flip-flops, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 503: `lif_input_valid = 1'b0;` This makes lif_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 504: `lif_output_ready = 1'b0;` This makes lif_output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 505: `optimizer_input_valid = 1'b0;` This makes optimizer_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 506: `optimizer_output_ready = 1'b0;` This makes optimizer_output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 507: `train_start_valid = 1'b0;` This makes train_start_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 508: `train_s_valid = 1'b0;` This makes train_s_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 509: `train_m_ready = 1'b0;` This makes train_m_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 510: `lut_clear = 1'b0;` This makes lut_clear, the value named 'lut clear' in this block, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 511: `lut_load_valid = 1'b0;` This makes lut_load_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 512: `lut_load_address = lut_load_address_r;` This makes lut_load_address, the value named 'lut load address' in this block, equal to 'lut_load_address_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lut_load_address_r: the value named 'lut load address r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 513: `lut_load_value = lut_load_value_r;` This makes lut_load_value, the value named 'lut load value' in this block, equal to 'lut_load_value_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lut_load_value_r: the value named 'lut load value r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 514: `lut_input_valid = 1'b0;` This makes lut_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 515: `lut_output_ready = 1'b0;` This makes lut_output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 517: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 518: `ST_TENSOR: begin` This is the ST_TENSOR case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 519: `tensor_s_valid = s_axis_tvalid;` This makes tensor_s_valid, the producer's statement that its current payload is meaningful, equal to 's_axis_tvalid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 520: `s_axis_tready = tensor_s_ready;` This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to 'tensor_s_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are tensor_s_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 521: `result_input_valid = tensor_m_valid;` This makes result_input_valid, the producer's statement that its current payload is meaningful, equal to 'tensor_m_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are tensor_m_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 522: `result_input_last = tensor_m_last;` This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'tensor_m_last'. It is driven by combinational or procedural logic in the current evaluation. The important operands are tensor_m_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 523: `tensor_m_ready = result_prestage_ready;` This makes tensor_m_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 524: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_TENSOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 525: `ST_ALPHA_LOAD: s_axis_tready = !(alpha_load_valid_r && alpha_load_final_r);` This is the ST_ALPHA_L case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 's_axis_tready = !(alpha_load_valid_r && alpha_load_final_r);'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 526: `ST_VECTOR_LOAD: s_axis_tready = !vector_load_final_r;` This is the ST_VECTOR_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 's_axis_tready = !vector_load_final_r;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 527: `ST_OPTIMIZER_LOAD, ST_LUT_EVAL_LOAD: s_axis_tready = 1'b1;` Within the combinational process whose outputs respond without waiting for a clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are ST_OPTIMIZER_LOAD: the value named 'st optimizer load' in this block; ST_LUT_EVAL_LOAD: the value named 'st lut eval load' in this block; s_axis_tready: the consumer's statement that it can accept a payload this cycle.

Line 528: `ST_LIF_LOAD: s_axis_tready = !lif_load_pending_r;` This is the ST_LIF_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 's_axis_tready = !lif_load_pending_r;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 529: `ST_ALPHA_START: alpha_input_valid = 1'b1;` This is the ST_ALPHA_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'alpha_input_valid = 1'b1;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_ALPHA_START, the machine performs the named control phase..

Line 530: `ST_ALPHA_OUTPUT: begin` This is the ST_ALPHA_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 531: `result_input_valid = alpha_output_valid;` This makes result_input_valid, the producer's statement that its current payload is meaningful, equal to 'alpha_output_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are alpha_output_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 532: `result_input_last = 1'b1;` This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 533: `alpha_output_ready = result_prestage_ready;` This makes alpha_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 534: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 535: `ST_VECTOR_START: vector_input_valid = 1'b1;` This is the ST_VECTOR_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'vector_input_valid = 1'b1;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_VECTOR_START, the machine performs the named control phase..

Line 536: `ST_VECTOR_OUTPUT: begin` This is the ST_VECTOR_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 537: `result_input_valid = vector_output_valid;` This makes result_input_valid, the producer's statement that its current payload is meaningful, equal to 'vector_output_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are vector_output_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 538: `result_input_last = output_word_index == 3;` This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'output_word_index == 3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 539: `vector_output_ready = result_prestage_ready` This statement continues through source line 540; the explanation uses the complete expression rather than only the first visual fragment. This makes vector_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready && output_word_index == 3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 540: `&& output_word_index == 3;` This is continuation line 2 of the 2-line logical statement that begins at source line 539. Its exact fragment is '&& output_word_index == 3;'. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes vector_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready && output_word_index == 3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 541: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 542: `ST_LAYERNORM_START: ln_start_valid = 1'b1;` This is the ST_LAYERNORM_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'ln_start_valid = 1'b1;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LAYERNORM_START, the machine performs the named control phase..

Line 543: `ST_LAYERNORM_LOAD0, ST_LAYERNORM_LOAD1: s_axis_tready = 1'b1;` Within the combinational process whose outputs respond without waiting for a clock edge Within state ST_LAYERNORM_START, the machine performs the named control phase., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are ST_LAYERNORM_LOAD0: the value named 'st layernorm load0' in this block; ST_LAYERNORM_LOAD1: the value named 'st layernorm load1' in this block; s_axis_tready: the consumer's statement that it can accept a payload this cycle.

Line 544: `ST_LAYERNORM_SEND: ln_sample_valid = 1'b1;` This is the ST_LAYERNORM_SEND case branch, which performs this explicitly named phase. The same-cycle action begins with 'ln_sample_valid = 1'b1;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase..

Line 545: `ST_LAYERNORM_OUTPUT: begin` This is the ST_LAYERNORM_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 546: `result_input_valid = ln_result_valid;` This makes result_input_valid, the producer's statement that its current payload is meaningful, equal to 'ln_result_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are ln_result_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 547: `result_input_last = ln_result_last;` This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'ln_result_last'. It is driven by combinational or procedural logic in the current evaluation. The important operands are ln_result_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 548: `ln_result_ready = result_prestage_ready;` This makes ln_result_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 549: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 550: `ST_SOFTMAX_START: softmax_start_valid = 1'b1;` This is the ST_SOFTMAX_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'softmax_start_valid = 1'b1;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_SOFTMAX_START, the machine performs the named control phase..

Line 551: `ST_SOFTMAX_LOAD: begin` This is the ST_SOFTMAX_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 552: `softmax_score_valid = s_axis_tvalid;` This makes softmax_score_valid, the producer's statement that its current payload is meaningful, equal to 's_axis_tvalid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 553: `// Entry to this state is delayed until the child accepted its` Design intent: Entry to this state is delayed until the child accepted its This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 554: `// start command, and the child remains ready until score_last.` Design intent: start command, and the child remains ready until score_last. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 555: `// Making that invariant explicit breaks a ready-to-state loop.` Design intent: Making that invariant explicit breaks a ready-to-state loop. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 556: `s_axis_tready = softmax_score_ready;` This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to 'softmax_score_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are softmax_score_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 557: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 558: `ST_SOFTMAX_OUTPUT: begin` This is the ST_SOFTMAX_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 559: `result_input_valid = softmax_probability_valid;` This makes result_input_valid, the producer's statement that its current payload is meaningful, equal to 'softmax_probability_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are softmax_probability_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting

for a clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 560: `result_input_last = softmax_probability_last;` This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'softmax_probability_last'. It is driven by combinational or procedural logic in the current evaluation. The important operands are softmax_probability_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 561: `softmax_probability_ready = result_prestage_ready;` This makes softmax_probability_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 562: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 563: `ST_LIF_CLEAR: lif_clear_state = 1'b1;` This is the ST_LIF_CLEAR case branch, which performs this explicitly named phase. The same-cycle action begins with 'lif_clear_state = 1'b1;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_CLEAR, the machine performs the named control phase..

Line 564: `ST_LIF_START: lif_input_valid = 1'b1;` This is the ST_LIF_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'lif_input_valid = 1'b1;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_START, the machine performs the named control phase..

Line 565: `ST_LIF_OUTPUT: begin` This is the ST_LIF_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 566: `result_input_valid = lif_output_valid;` This makes result_input_valid, the producer's statement that its current payload is meaningful, equal to 'lif_output_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lif_output_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 567: `result_input_last = !lif_trace_requested_r` This statement continues through source line 568; the explanation uses the complete expression rather than only the first visual fragment. This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '!lif_trace_requested_r || output_word_index == 12'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lif_trace_requested_r: the value named 'lif trace requested r' in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 568: `|| output_word_index == 12;` This is continuation line 2 of the 2-line logical statement that begins at source line 567. Its exact fragment is '|| output_word_index == 12;'. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '!lif_trace_requested_r

|| output_word_index == 12‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are lif_trace_requested_r: the value named ’lif trace requested r’ in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 569: `lif_output_ready = result_prestage_ready` This statement continues through source line 570; the explanation uses the complete expression rather than only the first visual fragment. This makes lif_output_ready, the consumer’s statement that it can accept a payload this cycle, equal to ‘result_prestage_ready && (!lif_trace_requested_r || output_word_index == 12)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer’s statement that it can accept a payload this cycle; lif_trace_requested_r: the value named ’lif trace requested r’ in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 570: `&& (!lif_trace_requested_r || output_word_index == 12);` This is continuation line 2 of the 2-line logical statement that begins at source line 569. Its exact fragment is ‘&& (!lif_trace_requested_r || output_word_index == 12);‘. The important operands are lif_trace_requested_r: the value named ’lif trace requested r’ in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes lif_output_ready, the consumer’s statement that it can accept a payload this cycle, equal to ‘result_prestage_ready && (!lif_trace_requested_r || output_word_index == 12)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer’s statement that it can accept a payload this cycle; lif_trace_requested_r: the value named ’lif trace requested r’ in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 571: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 572: `ST_OPTIMIZER_START: optimizer_input_valid = 1’b1;` This is the ST_OPTIMIZER_START case branch, which performs this explicitly named phase. The same-cycle action begins with ‘optimizer_input_valid = 1’b1;‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_OPTIMIZER_START, the machine performs the named control phase..

Line 573: `ST_OPTIMIZER_OUTPUT: begin` This is the ST_OPTIMIZER_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 574: `result_input_valid = optimizer_output_valid;` This makes result_input_valid, the producer’s statement that its current payload is meaningful, equal to ‘optimizer_output_valid‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are optimizer_output_valid: the producer’s statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 575: `result_input_last = output_word_index == 4’d7;` This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘output_word_index == 4’d7‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry; d7: the value named ’d7’ in this block. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 576: `optimizer_output_ready = result_prestage_ready` This statement continues through source line 577; the explanation uses the complete expression rather than only the first visual fragment. This makes optimizer_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready && output_word_index == 4'd7'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle; output_word_index: an integer selecting one tensor element, lane, bank, or state entry; d7: the value named 'd7' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 577: `&& output_word_index == 4'd7;` This is continuation line 2 of the 2-line logical statement that begins at source line 576. Its exact fragment is '&& output_word_index == 4'd7;'. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry; d7: the value named 'd7' in this block. Read together, the complete operation is: This makes optimizer_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready && output_word_index == 4'd7'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle; output_word_index: an integer selecting one tensor element, lane, bank, or state entry; d7: the value named 'd7' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 578: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 579: `ST_TRAIN_START: train_start_valid = 1'b1;` This is the ST_TRAIN_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'train_start_valid = 1'b1;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_TRAIN_START, the machine performs the named control phase..

Line 580: `ST_TRAIN_RUN: begin` This is the ST_TRAIN_RUN case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 581: `train_s_valid = s_axis_tvalid;` This makes train_s_valid, the producer's statement that its current payload is meaningful, equal to 's_axis_tvalid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 582: `s_axis_tready = train_s_ready;` This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to 'train_s_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are train_s_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 583: `result_input_valid = train_m_valid;` This makes result_input_valid, the producer's statement that its current payload is meaningful, equal to 'train_m_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are train_m_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 584: `result_input_last = train_m_last;` This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'train_m_last'. It is driven by combinational or procedural logic in the current evaluation. The important operands are train_m_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 585: `train_m_ready = result_prestage_ready;` This makes train_m_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 586: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 587: `ST_LUT_LOAD: begin` This is the ST_LUT_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 588: `lut_load_valid = lut_load_pending_r;` This makes lut_load_valid, the producer's statement that its current payload is meaningful, equal to 'lut_load_pending_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lut_load_pending_r: the value named 'lut load pending r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 589: `s_axis_tready = !lut_load_pending_r;` This makes s_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '!lut_load_pending_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lut_load_pending_r: the value named 'lut load pending r' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 590: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 591: `ST_LUT_EVAL_START: lut_input_valid = 1'b1;` This is the ST_LUT_EVAL_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'lut_input_valid = 1'b1;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_EVAL_START, the machine performs the named control phase..

Line 592: `ST_LUT_EVAL_OUTPUT: begin` This is the ST_LUT_EVAL_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 593: `result_input_valid = lut_output_valid;` This makes result_input_valid, the producer's statement that its current payload is meaningful, equal to 'lut_output_valid'. It is driven by combinational or procedural logic in the current evaluation. The important operands are lut_output_valid: the producer's statement that its current payload is meaningful. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 594: `result_input_last = output_word_index == 1;` This makes result_input_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'output_word_index == 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 595: `lut_output_ready = result_prestage_ready` This statement continues through source line 596; the explanation uses the complete expression rather than only the first visual fragment. This makes lut_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready && output_word_index == 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 596: `&& output_word_index == 1;` This is continuation line 2 of the 2-line logical statement that begins at source line 595. Its exact fragment is '&& output_word_index == 1;'. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes lut_output_ready, the consumer's statement that it can accept a payload this cycle, equal to 'result_prestage_ready && output_word_index == 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 597: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 598: `ST_LUT_CLEAR: lut_clear = lut_clear_requested_r;` This is the ST_LUT_CLEAR case branch, which performs this explicitly named phase. The same-cycle action begins with 'lut_clear = lut_clear_requested_r;'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase..

Line 599: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase..

Line 600: `endcase` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 601: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 603: `// A bank samples whenever its source owns an available prestage slot. The` Design intent: A bank samples whenever its source owns an available prestage slot. The This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 604: `// write condition deliberately excludes child VALID: during a downstream` Design intent: write condition deliberately excludes child VALID: during a downstream This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 605: `// stall the full prestage freezes the bank, while an available slot may` Design intent: stall the full prestage freezes the bank, while an available slot may This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 606: `// harmlessly sample an invalid payload. This keeps the wide bank enable` Design intent: harmlessly sample an invalid payload. This keeps the wide bank enable This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 607: `// local to the one-hot core state instead of broadcasting a remote child` Design intent: local to the one-hot core state instead of broadcasting a remote child This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 608: `// FSM bit, and it preserves the payload associated with every valid token.` Design intent: FSM bit, and it preserves the payload associated with every valid token. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 609: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 610: `if (result_prestage_ready) begin` This condition selects the following hardware branch only when 'result_prestage_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_prestage_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 611: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 612: `ST_TENSOR: result_tensor_data_r <= tensor_m_data;` This is the ST_TENSOR case branch, which performs this explicitly named phase. The same-cycle action begins with 'result_tensor_data_r <= tensor_m_data;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 613: `ST_ALPHA_OUTPUT: result_alpha_data_r <= {28'd0,` This statement continues through source line 614; the explanation uses the complete expression rather than only the first visual fragment. This is the ST_ALPHA_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_alpha_data_r <= {28'd0, alpha_zero_matches, alpha_spike_matches, alpha_score};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 614: `alpha_zero_matches, alpha_spike_matches, alpha_score};` This is continuation line 2 of the 2-line logical statement that begins at source line 613. Its exact fragment is 'alpha_zero_matches, alpha_spike_matches, alpha_score};'. The important operands are alpha_zero_matches: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; alpha_spike_matches: a binary event produced when a neuron crosses its threshold; alpha_score: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Read together, the complete operation is: This is the ST_ALPHA_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_alpha_data_r <= {28'd0, alpha_zero_matches, alpha_spike_matches, alpha_score};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 615: `ST_VECTOR_OUTPUT: result_vector_data_r` This is the ST_VECTOR_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_vector_data_r'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 616: `<= vector_result[output_word_index*64 +: 64];` Within the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are vector_result: the value named 'vector result' in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 617: `ST_LAYERNORM_OUTPUT: result_layernorm_data_r <= {19'd0,` This statement continues through source line 618; the explanation uses the complete expression rather than only the first visual fragment. This is the ST_LAYERNORM_OU case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_layernorm_data_r <= {19'd0, ln_result_last, ln_result_index, ln_result_value};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 618: `ln_result_last, ln_result_index, ln_result_value};` This is continuation line 2 of the 2-line logical statement that begins at source line 617. Its exact fragment is 'ln_result_last, ln_result_index, ln_result_value};'. The important operands are ln_result_last: the AXI-Stream marker for the final beat of a packet or command payload; ln_result_index: an integer selecting one tensor element, lane, bank, or state entry; ln_result_value: the value named 'ln result value' in this block. Read together, the complete operation is: This is the ST_LAYERNORM_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_layernorm_data_r <= {19'd0, ln_result_last, ln_result_index, ln_result_value};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 619: `ST_SOFTMAX_OUTPUT: result_softmax_data_r <= {38'd0,` This statement continues through source line 621; the explanation uses the complete expression rather than only the first visual fragment. This is the ST_SOFTMAX_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_softmax_data_r <= {38'd0, softmax_probability_last, softmax_probability_index, softmax_probability};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 620: `softmax_probability_last, softmax_probability_index,` This is continuation line 2 of the 3-line logical statement that begins at source line 619. Its exact fragment is 'softmax_probability_last, softmax_probability_index,'. The important operands are softmax_probability_last: the AXI-Stream marker for the final beat of a packet or command payload; softmax_probability_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This is the ST_SOFTMAX_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_softmax_data_r <= {38'd0, softmax_probability_last, softmax_probability_index, softmax_probability};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 621: `softmax_probability};` This is continuation line 3 of the 3-line logical statement that begins at source line 619. Its exact fragment is 'softmax_probability};'. The important operands are softmax_probability: the value named 'softmax probability' in this block. Read together, the complete operation is: This is the ST_SOFTMAX_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_softmax_data_r <= {38'd0, softmax_probability_last, softmax_probability_index, softmax_probability};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 622: `ST_LIF_OUTPUT: begin` This is the ST_LIF_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 623: `if (output_word_index == 0) begin` This condition selects the following hardware branch only when 'output_word_index == 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than

making software jump. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 624: `result_lif_data_r <= {lif_timesteps, lif_spikes};` This makes result_lif_data_r, the payload carried with the surrounding valid/ready handshake, equal to '{lif_timesteps, lif_spikes}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lif_timesteps: one discrete SNN update instant; lif_spikes: a binary event produced when a neuron crosses its threshold. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 625: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 626: `result_lif_data_r <= lif_pre_reset_membrane[` This statement continues through source line 627; the explanation uses the complete expression rather than only the first visual fragment. This makes result_lif_data_r, the payload carried with the surrounding valid/ready handshake, equal to 'lif_pre_reset_membrane[lif_trace_bit_offset +: 64]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lif_pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; lif_trace_bit_offset: the value named 'lif trace bit offset' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 627: `lif_trace_bit_offset +:  64];` This is continuation line 2 of the 2-line logical statement that begins at source line 626. Its exact fragment is 'lif_trace_bit_offset +: 64];'. The important operands are lif_trace_bit_offset: the value named 'lif trace bit offset' in this block. Read together, the complete operation is: This makes result_lif_data_r, the payload carried with the surrounding valid/ready handshake, equal to 'lif_pre_reset_membrane[lif_trace_bit_offset +: 64]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lif_pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; lif_trace_bit_offset: the value named 'lif trace bit offset' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 628: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 629: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 630: `ST_OPTIMIZER_OUTPUT: result_optimizer_data_r <= {24'd0,` This statement continues through source line 632; the explanation uses the complete expression rather than only the first visual fragment. This is the ST_OPTIMIZER_OUTP case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_optimizer_data_r <= {24'd0, optimizer_updated_velocity[output_word_index*24 +: 24], optimizer_updated_weight[outpu +: 16]};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 631: `optimizer_updated_velocity[output_word_index*24 +:  24],` This is continuation line 2 of the 3-line logical statement that begins at source line 630. Its exact fragment is 'optimizer_updated_velocity[output_word_index*24 +: 24],'. The important operands are optimizer_updated_velocity: the momentum history retained by the SGD optimizer; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This is the ST_OPTIMIZER_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_optimizer_data_r <= {24'd0, optimizer_updated_velocity[ou +: 24], optimizer_updated_weight[output_word_index*16 +: 16]};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 632: `optimizer_updated_weight[output_word_index*16 +: 16]};` This is continuation line 3 of the 3-line logical statement that begins at source line 630. Its exact fragment is 'optimizer_updated_weight[output_word_index*16 +: 16]};'. The important operands are optimizer_updated_weight: a learned coefficient multiplied by an activation; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This is the ST_OPTIMIZER_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_optimizer_data_r <= {24'd0, optimizer_updated_velocity[output_word_inde +: 24], optimizer_updated_weight[output_word_index*16 +: 16]};'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 633: `ST_TRAIN_RUN: result_train_data_r <= train_m_data;` This is the ST_TRAIN_RUN case branch, which performs this explicitly named phase. The same-cycle action begins with 'result_train_data_r <= train_m_data;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 634: `ST_LUT_EVAL_OUTPUT: result_lut_data_r` This is the ST_LUT_EVAL_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_lut_data_r'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 635: `<= lut_output_values[output_word_index*64 +: 64];` Within the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are lut_output_values: the value named 'lut output values' in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 636: `default: begin end` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin end'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 637: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 638: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 639: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 641: `// Two-entry elastic output pipeline. The first stage captures each` Design intent: Two-entry elastic output pipeline. The first stage captures each This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 642: `// engine into a source-local register; the second stage performs only the` Design intent: engine into a source-local register; the second stage performs only the This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 643: `// registered-bank selection. Data and tag registers intentionally have` Design intent: registered-bank selection. Data and tag registers intentionally have This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 644: `// no reset mux because their valid bits are the architectural qualifier.` Design intent: no reset mux because their valid bits are the architectural qualifier. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 645: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 646: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 647: `result_prestage_valid_r <= 1'b0;` This makes result_prestage_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 648: `result_output_valid_r <= 1'b0;` This makes result_output_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 649: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 650: `if (result_output_ready) begin` This condition selects the following hardware branch only when 'result_output_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_output_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 651: `result_output_valid_r <= result_prestage_valid_r;` This makes result_output_valid_r, the producer's statement that its current payload is meaningful, equal to 'result_prestage_valid_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_prestage_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 652: `if (result_prestage_valid_r) begin` This condition selects the following hardware branch only when 'result_prestage_valid_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_prestage_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 653: `result_output_last_r <= result_prestage_last_r;` This makes result_output_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'result_prestage_last_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_prestage_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 654: `unique case (result_prestage_source_r)` This starts a multiplexer/state decode selected by 'result_prestage_sour 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 655: `RESULT_TENSOR: result_output_data_r <= result_tensor_data_r;` This is the RESULT_TENSOR case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r <= result_tensor_data_r;‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_TENSOR, the machine performs the named control phase..

Line 656: `RESULT_ALPHA: result_output_data_r <= result_alpha_data_r;` This is the RESULT_ALPHA case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r <= result_alpha_data_r;‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_ALPHA, the machine performs the named control phase..

Line 657: `RESULT_VECTOR: result_output_data_r <= result_vector_data_r;` This is the RESULT_VECTOR case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r <= result_vector_data_r;‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_VECTOR, the machine performs the named control phase..

Line 658: `RESULT_LAYERNORM: result_output_data_r` This is the RESULT_LAYERNORM case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_LAYERNORM, the machine performs the named control phase..

Line 659: `<= result_layernorm_data_r;` Within the clocked process that updates registers only on a rising clock edge Within state RESULT_LAYERNORM, the machine performs the named control phase., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are result_layernorm_data_r: the payload carried with the surrounding valid/ready handshake.

Line 660: `RESULT_SOFTMAX: result_output_data_r <= result_softmax_data_r;` This is the RESULT_SOFTMAX case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r <= result_softmax_data_r;‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_SOFTMAX, the machine performs the named control phase..

Line 661: `RESULT_LIF: result_output_data_r <= result_lif_data_r;` This is the RESULT_LIF case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r <= result_lif_data_r;‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_LIF, the machine performs the named control phase..

Line 662: `RESULT_OPTIMIZER: result_output_data_r` This is the RESULT_OPTIMIZER case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_OPTIMIZER, the machine performs the named control phase..

Line 663: `<= result_optimizer_data_r;` Within the clocked process that updates registers only on a rising clock edge Within state RESULT_OPTIMIZER, the machine performs the named control phase., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are result_optimizer_data_r: the payload carried with the surrounding valid/ready handshake.

Line 664: `RESULT_TRAIN: result_output_data_r <= result_train_data_r;` This is the RESULT_TRAIN case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r <= result_train_data_r;‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_TRAIN, the machine performs the named control phase..

Line 665: `RESULT_LUT: result_output_data_r <= result_lut_data_r;` This is the RESULT_LUT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r <= result_lut_data_r;‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_LUT, the machine performs the named control phase..

Line 666: `default: result_output_data_r <= 64'd0;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_output_data_r <= 64’d0;‘; context is the clocked process that updates registers only on a rising clock edge Within state RESULT_LUT, the machine performs the named control phase..

Line 667: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state RESULT_LUT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 668: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 669: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 671: `if (result_prestage_ready) begin` This condition selects the following hardware branch only when ‘result_prestage_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_prestage_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 672: `result_prestage_valid_r <= result_input_valid;` This makes result_prestage_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘result_input_valid‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_input_valid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 673: `result_prestage_last_r <= result_input_last;` This makes result_prestage_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘result_input_last‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_input_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 674: `unique case (state)` This starts a multiplexer/state decode selected by ‘state‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 675: `ST_TENSOR: result_prestage_source_r <= RESULT_TENSOR;` This is the ST_TENSOR case branch, which performs this explicitly named phase. The same-cycle action begins with ‘result_prestage_source_r <= RESULT_TENSOR;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 676: `ST_ALPHA_OUTPUT: result_prestage_source_r <= RESULT_ALPHA;` This is the ST_ALPHA_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘result_prestage_source_r <= RESULT_ALPHA;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 677: `ST_VECTOR_OUTPUT: result_prestage_source_r <= RESULT_VECTOR;` This is the ST_VECTOR_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘result_prestage_source_r <= RESULT_VECTOR;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 678: `ST_LAYERNORM_OUTPUT: result_prestage_source_r` This is the ST_LAYERNORM_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_prestage_source_r'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 679: `<= RESULT_LAYERNORM;` Within the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are RESULT_LAYERNORM: the value named 'result layernorm' in this block.

Line 680: `ST_SOFTMAX_OUTPUT: result_prestage_source_r <= RESULT_SOFTMAX;` This is the ST_SOFTMAX_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_prestage_source_r <= RESULT_SOFTMAX;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 681: `ST_LIF_OUTPUT: result_prestage_source_r <= RESULT_LIF;` This is the ST_LIF_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_prestage_source_r <= RESULT_LIF;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 682: `ST_OPTIMIZER_OUTPUT: result_prestage_source_r` This is the ST_OPTIMIZER_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_prestage_source_r'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 683: `<= RESULT_OPTIMIZER;` Within the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are RESULT_OPTIMIZER: the value named 'result optimizer' in this block.

Line 684: `ST_TRAIN_RUN: result_prestage_source_r <= RESULT_TRAIN;` This is the ST_TRAIN_RUN case branch, which performs this explicitly named phase. The same-cycle action begins with 'result_prestage_source_r <= RESULT_TRAIN;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 685: `ST_LUT_EVAL_OUTPUT: result_prestage_source_r <= RESULT_LUT;` This is the ST_LUT_EVAL_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'result_prestage_source_r <= RESULT_LUT;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 686: `default: result_prestage_source_r <= RESULT_TENSOR;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'result_prestage_source_r <= RESULT_TENSOR;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 687: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 688: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 689: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 690: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 692: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 693: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 694: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 695: `command_done <= 1'b0;` This makes command_done, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 696: `command_done_pending_r <= 1'b0;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 697: `input_beat_accepted_r <= 1'b0;` This makes input_beat_accepted_r, the value named 'input beat accepted r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 698: `output_beat_accepted_r <= 1'b0;` This makes output_beat_accepted_r, the value named 'output beat accepted r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 699: `status_error_r <= 1'b0;` This makes status_error_r, the value named 'status error r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 700: `error_event_r <= 1'b0;` This makes error_event_r, the value named 'error event r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 701: `clear_error_r <= 1'b0;` This makes clear_error_r, the value named 'clear error r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 702: `stream_protocol_error_event_r <= 1'b0;` This makes stream_protocol_error_event_r, the value named 'stream protocol error event r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 703: `clear_counters_event_r <= 1'b0;` This makes clear_counters_event_r, a hardware telemetry register that accumulates measured events, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 704: `status_active_opcode <= 4'd0;` This makes status_active_opcode, the value named 'status active opcode' in this block, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 705: `status_command_count <= 32'd0;` This makes status_command_count, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 706: `status_input_beats <= 32'd0;` This makes status_input_beats, the value named 'status input beats' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 707: `status_output_beats <= 32'd0;` This makes status_output_beats, the value named 'status output beats' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 708: `stream_beat_index <= 8'd0;` This makes stream_beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 709: `stream_item_index <= 12'd0;` This makes stream_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 710: `stream_items_remaining_r <= 12'd0;` This makes stream_items_remaining_r, the value named 'stream items remaining r' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 711: `stream_last_pending_r <= 1'b0;` This makes stream_last_pending_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 712: `output_word_index <= 4'd0;` This makes output_word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 713: `captured_command_r <= 109'd0;` This makes captured_command_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '109'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 714: `train_reserved_partial_r <= 4'd0;` This makes train_reserved_partial_r, the value named 'train reserved partial r' in this block, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 715: `train_reserved_nonzero_r <= 1'b0;` This makes train_reserved_nonzero_r, the value named 'train reserved nonzero r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 716: `command_had_output_r <= 1'b0;` This makes command_had_output_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 717: `final_output_accepted_r <= 1'b0;` This makes final_output_accepted_r, the value named 'final output accepted r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 718: `completion_error_r <= 1'b0;` This makes completion_error_r, the value named 'completion error r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 719: `alpha_query_r <= '0;` This makes alpha_query_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 720: `alpha_key_r <= '0;` This makes alpha_key_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 721: `alpha_active_bits_r <= 9'd0;` This makes alpha_active_bits_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 722: `alpha_q8_r <= 9'd0;` This makes alpha_q8_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 723: `alpha_load_valid_r <= 1'b0;` This makes alpha_load_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 724: `alpha_load_index_r <= 3'd0;` This makes alpha_load_index_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to '3'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 725: `alpha_load_data_r <= 64'd0;` This makes alpha_load_data_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 726: `alpha_load_final_r <= 1'b0;` This makes alpha_load_final_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 727: `alpha_load_last_r <= 1'b0;` This makes alpha_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 728: `vector_operation_r <= 4'd0;` This makes vector_operation_r, the value named 'vector operation r' in this block, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 729: `vector_operand_a_r <= '0;` This makes vector_operand_a_r, the value named 'vector operand a r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 730: `vector_operand_b_r <= '0;` This makes vector_operand_b_r, the value named 'vector operand b r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 731: `vector_multiplier_r <= 16'sd0;` This makes vector_multiplier_r, the value named 'vector multiplier r' in this block, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 732: `vector_shift_r <= 5'd0;` This makes vector_shift_r, the value named 'vector shift r' in this block, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 733: `vector_zero_point_r <= 16'sd0;` This makes vector_zero_point_r, the integer code representing real numeric zero in affine quantization, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 734: `vector_threshold_r <= 32'sd0;` This makes vector_threshold_r, the membrane level at which a neuron emits a spike, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 735: `vector_window_r <= 32'd0;` This makes vector_window_r, the value named 'vector window r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 736: `vector_clamp_min_r <= 32'sh80000000;` This makes vector_clamp_min_r, the value named 'vector clamp min r' in this block, equal to '32'sh80000000'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sh80000000: the value named 'sh80000000' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 737: `vector_clamp_max_r <= 32'sh7fffffff;` This makes vector_clamp_max_r, the value named 'vector clamp max r' in this block, equal to '32'sh7fffffff'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sh7fffffff: the value named 'sh7fffffff' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 738: `vector_load_valid_r <= 1'b0;` This makes vector_load_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 739: `vector_load_index_r <= 4'd0;` This makes vector_load_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 740: `vector_load_data_r <= 64'd0;` This makes vector_load_data_r, the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 741: `vector_load_final_r <= 1'b0;` This makes vector_load_final_r, the value named 'vector load final r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 742: `vector_load_last_r <= 1'b0;` This makes vector_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 743: `ln_count_r <= 12'd0;` This makes ln_count_r, an integer count retained for control or measurable telemetry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 744: `ln_epsilon_r <= 32'd0;` This makes ln_epsilon_r, the value named 'ln epsilon r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 745: `ln_sample_word0_r <= 64'd0;` This makes ln_sample_word0_r, the value named 'ln sample word0 r' in this block, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 746: `ln_sample_value_r <= 24'sd0;` This makes ln_sample_value_r, the value named 'ln sample value r' in this block, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 747: `ln_sample_gamma_r <= 24'sd0;` This makes ln_sample_gamma_r, the learned multiplicative LayerNorm parameter, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 748: `ln_sample_beta_r <= 24'sd0;` This makes ln_sample_beta_r, the learned additive LayerNorm parameter, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 749: `ln_start_accept_r <= 1'b0;` This makes ln_start_accept_r, the value named 'ln start accept r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 750: `softmax_count_r <= 9'd0;` This makes softmax_count_r, an integer count retained for control or measurable telemetry, equal to '9'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 751: `softmax_shift_r <= 5'd0;` This makes softmax_shift_r, the value named 'softmax shift r' in this block, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 752: `softmax_start_accept_r <= 1'b0;` This makes softmax_start_accept_r, the value named 'softmax start accept r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 753: `lif_current_r <= '0;` This makes lif_current_r, the value named 'lif current r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 754: `lif_leak_shift_r <= 5'd0;` This makes lif_leak_shift_r, the retained fraction of prior membrane state, equal to '5'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 755: `lif_threshold_r <= 24'sd0;` This makes lif_threshold_r, the membrane level at which a neuron emits a spike, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 756: `lif_reset_r <= 24'sd0;` This makes lif_reset_r, the value named 'lif reset r' in this block, equal to '24'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 757: `lif_clear_requested_r <= 1'b0;` This makes lif_clear_requested_r, the value named 'lif clear requested r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 758: `lif_trace_requested_r <= 1'b0;` This makes lif_trace_requested_r, the value named 'lif trace requested r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 759: `lif_load_word_select_r <= 8'd0;` This makes lif_load_word_select_r, the value named 'lif load word select r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 760: `lif_load_pending_r <= 1'b0;` This makes lif_load_pending_r, the value named 'lif load pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 761: `lif_load_index_r <= 3'd0;` This makes lif_load_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '3'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 762: `lif_load_data_r <= 64'd0;` This makes lif_load_data_r, the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 763: `lif_load_last_r <= 1'b0;` This makes lif_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 764: `lif_load_final_r <= 1'b0;` This makes lif_load_final_r, the value named 'lif load final r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 765: `optimizer_weight_r <= '0;` This makes optimizer_weight_r, a learned coefficient multiplied by an activation, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 766: `optimizer_gradient_r <= '0;` This makes optimizer_gradient_r, the signed loss derivative used to update a learned parameter, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 767: `optimizer_velocity_r <= '0;` This makes optimizer_velocity_r, the momentum history retained by the SGD optimizer, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 768: `optimizer_lr_r <= 18'd0;` This makes optimizer_lr_r, the value named 'optimizer lr r' in this block, equal to '18'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 769: `optimizer_momentum_r <= 17'd0;` This makes optimizer_momentum_r, the value named 'optimizer momentum r' in this block, equal to '17'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 770: `optimizer_decay_r <= 17'd0;` This makes optimizer_decay_r, the value named 'optimizer decay r' in this block, equal to '17'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 771: `optimizer_update_shift_r <= 5'd16;` This makes optimizer_update_shift_r, the value named 'optimizer update shift r' in this block, equal to '5'd16'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d16: the value named 'd16' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 772: `optimizer_load_data_r <= 64'd0;` This makes optimizer_load_data_r, the payload carried with the surrounding valid/ready handshake, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 773: `optimizer_load_last_r <= 1'b0;` This makes optimizer_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 774: `optimizer_load_index_r <= 3'd0;` This makes optimizer_load_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '3'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 775: `optimizer_load_final_r <= 1'b0;` This makes optimizer_load_final_r, the value named 'optimizer load final r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 776: `optimizer_commit_valid_r <= 1'b0;` This makes optimizer_commit_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 777: `lut_load_pending_r <= 1'b0;` This makes lut_load_pending_r, the value named 'lut load pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 778: `lut_load_address_r <= 8'd0;` This makes lut_load_address_r, the value named 'lut load address r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 779: `lut_load_value_r <= 16'sd0;` This makes lut_load_value_r, the value named 'lut load value r' in this block, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 780: `lut_load_last_r <= 1'b0;` This makes lut_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 781: `lut_load_final_r <= 1'b0;` This makes lut_load_final_r, the value named 'lut load final r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 782: `lut_indices_r <= 64'd0;` This makes lut_indices_r, the value named 'lut indices r' in this block, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 783: `lut_clear_requested_r <= 1'b0;` This makes lut_clear_requested_r, the value named 'lut clear requested r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 784: `decode_error_pending_r <= 1'b0;` This makes decode_error_pending_r, the value named 'decode error pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 785: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 786: `if (clear_error_r) status_error_r <= 1'b0;` This condition selects the following hardware branch only when 'clear_error_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_error_r: the value named 'clear error r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 787: `else if (error_event_r) status_error_r <= 1'b1;` This condition selects the following hardware branch only when 'error_event_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are error_event_r: the value named 'error event r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 788: `error_event_r <= 1'b0;` This makes error_event_r, the value named 'error event r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 789: `clear_error_r <= 1'b0;` This makes clear_error_r, the value named 'clear error r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 790: `if (stream_protocol_error_event_r) error_event_r <= 1'b1;` This condition selects the following hardware branch only when ‘stream_protocol_error_event_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are stream_protocol_error_event_r: the value named 'stream protocol error event r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 791: `stream_protocol_error_event_r <= 1'b0;` This makes stream_protocol_error_event_r, the value named 'stream protocol error event r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 792: `clear_counters_event_r <= 1'b0;` This makes clear_counters_event_r, a hardware telemetry register that accumulates measured events, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 793: `vector_load_valid_r <= 1'b0;` This makes vector_load_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 794: `command_done <= command_done_pending_r;` This makes command_done, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘command_done_pending_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_done_pending_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge.

Line 795: `command_done_pending_r <= 1'b0;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 796: `optimizer_commit_valid_r <= 1'b0;` This makes optimizer_commit_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 797: `input_beat_accepted_r <= s_axis_tvalid && s_axis_tready;` This makes input_beat_accepted_r, the value named 'input beat accepted r' in this block, equal to ‘s_axis_tvalid && s_axis_tready‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful; s_axis_tready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 798: `output_beat_accepted_r <= m_axis_tvalid && m_axis_tready;` This makes output_beat_accepted_r, the value named 'output beat accepted r' in this block, equal to ‘m_axis_tvalid && m_axis_tready‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are m_axis_tvalid: the producer's statement that its current payload is meaningful; m_axis_tready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 799: `softmax_start_accept_r <= softmax_start_valid && softmax_start_ready;` This makes softmax_start_accept the value named 'softmax start accept r' in this block, equal to ‘softmax_start_valid && softmax_start_ready‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are softmax_start_valid: the producer's statement that its current payload is meaningful; softmax_start_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 800: `ln_start_accept_r <= ln_start_valid && ln_start_ready;` This makes ln_start_accept_r, the value named 'ln start accept r' in this block, equal to 'ln_start_valid && ln_start_ready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ln_start_valid: the producer's statement that its current payload is meaningful; ln_start_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge.

Line 801: `if (result_input_accept) command_had_output_r <= 1'b1;` This condition selects the following hardware branch only when 'result_input_accept' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_input_accept: the value named 'result input accept' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 802: `if (result_output_accept && result_output_last_r) begin` This condition selects the following hardware branch only when 'result_output_accept && result_output_last_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_output_accept: the value named 'result output accept' in this block; result_output_last_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge.

Line 803: `final_output_accepted_r <= 1'b1;` This makes final_output_accepted_r, the value named 'final output accepted r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 804: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 805: `lif_load_word_select_r <= 8'd0;` This makes lif_load_word_select_r, the value named 'lif load word select r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 806: `lif_load_pending_r <= 1'b0;` This makes lif_load_pending_r, the value named 'lif load pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 807: `for (int lif_word = 0; lif_word < 8; lif_word++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the clocked process that updates registers only on a rising clock edge.

Line 808: `if (lif_load_word_select_r[lif_word]) begin` This condition selects the following hardware branch only when 'lif_load_word_select_r[lif_word]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lif_load_word_select_r: the value named 'lif load word select r' in this block; lif_word: the value named 'lif word' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 809: `lif_current_r[lif_word*64 +:  64] <= lif_load_data_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are lif_current_r: the value named 'lif current r' in this block; lif_word: the value named 'lif word' in this block; lif_load_data_r: the payload carried with the surrounding valid/ready handshake.

Line 810: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 811: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 812: `alpha_load_valid_r <= 1'b0;` This makes alpha_load_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 813: `if (alpha_load_valid_r) begin` This condition selects the following hardware branch only when ‘alpha_load_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are alpha_load_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 814: `if (alpha_load_last_r != alpha_load_final_r) begin` This condition selects the following hardware branch only when ‘alpha_load_last_r != alpha_load_final_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are alpha_load_last_r: the AXI-Stream marker for the final beat of a packet or command payload; alpha_load_final_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the clocked process that updates registers only on a rising clock edge.

Line 815: `stream_protocol_error_event_r <= 1'b1;` This makes stream_protocol_error_event_r, the value named 'stream protocol error event r' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 816: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 817: `if (alpha_load_index_r < 4) begin` This condition selects the following hardware branch only when ‘alpha_load_index_r $<$ 4‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are alpha_load_index_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the clocked process that updates registers only on a rising clock edge.

Line 818: `alpha_query_r[alpha_load_index_r*64 +: 64] <= alpha_load_data_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are alpha_query_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; alpha_load_index_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; alpha_load_data_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 819: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 820: `alpha_key_r[(int'(alpha_load_index_r)-4)*64 +: 64]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are alpha_key_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; alpha_load_index_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 821: `<= alpha_load_data_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are alpha_load_data_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 822: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 823: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 824: `if (optimizer_commit_valid_r) begin` This condition selects the following hardware branch only when ‘optimizer_commit_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are optimizer_commit_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 825: `optimizer_weight_r[optimizer_load_index_r*16 +: 16]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are optimizer_weight_r: a learned coefficient multiplied by an activation; optimizer_load_index_r: an integer selecting one tensor element, lane, bank, or state entry.

Line 826: `<= optimizer_load_data_r[15:0];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are optimizer_load_data_r: the payload carried with the surrounding valid/ready handshake.

Line 827: `optimizer_gradient_r[optimizer_load_index_r*24 +: 24]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are optimizer_gradient_r: the signed loss derivative used to update a learned parameter; optimizer_load_index_r: an integer selecting one tensor element, lane, bank, or state entry.

Line 828: `<= optimizer_load_data_r[39:16];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are optimizer_load_data_r: the payload carried with the surrounding valid/ready handshake.

Line 829: `optimizer_velocity_r[optimizer_load_index_r*24 +: 24]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are optimizer_velocity_r: the momentum history retained by the SGD optimizer; optimizer_load_index_r: an integer selecting one tensor element, lane, bank, or state entry.

Line 830: `<= optimizer_load_data_r[63:40];` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are optimizer_load_data_r: the payload carried with the surrounding valid/ready handshake.

Line 831: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 832: `if (vector_load_valid_r) begin` This condition selects the following hardware branch only when ‘vector_load_valid_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are vector_load_valid_r: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge.

Line 833: `if (vector_load_last_r != vector_load_final_r) begin` This condition selects the following hardware branch only when ‘vector_load_last_r != vector_load_final_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are vector_load_last_r: the AXI-Stream marker for the final beat of a packet or command payload; vector_load_final_r: the value named ’vector load final r’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 834: `stream_protocol_error_event_r <= 1'b1;` This makes stream_protocol_error_event_r, the value named ’stream protocol error event r’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 835: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 836: `if (vector_load_index_r == 0) begin` This condition selects the following hardware branch only when ‘vector_load_index_r == 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are vector_load_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge.

Line 837: `vector_clamp_min_r <= vector_load_data_r[31:0];` This makes vector_clamp_min_r, the value named ’vector clamp min r’ in this block, equal to ‘vector_load_data_r[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are vector_load_data_r: the payload carried with the surrounding valid/ready handshake. Context: the clocked process that updates registers only on a rising clock edge.

Line 838: `vector_clamp_max_r <= vector_load_data_r[63:32];` This makes vector_clamp_max_r, the value named ’vector clamp max r’ in this block, equal to ‘vector_load_data_r[63:32]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are vector_load_data_r: the payload carried with the surrounding valid/ready handshake. Context: the clocked process that updates registers only on a rising clock edge.

Line 839: `end else if (vector_load_index_r <= 4) begin` This statement continues through source line 841; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when ‘vector_load_index_r <= 4) begin vector_operand_a_r[(int’(vector_load_index_r)-1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are vector_load_index_r: an integer selecting one tensor element, lane, bank, or state entry; vector_operand_a_r: the value named ’vector operand a r’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 840: `vector_operand_a_r[(int'(vector_load_index_r)-1)*64 +: 64]` This is continuation line 2 of the 3-line logical statement that begins at source line 839. Its exact fragment is ‘vector_operand_a_r[(int’(vector_load_index_r)-1)*64 +: 64]‘. The important operands are vector_operand_a_r: the value named ’vector operand a r’ in this block; vector_load_index_r: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This condition selects the following hardware branch only when ‘vector_load_index_r <= 4) begin vector_operand_a_r[(int’(vector_load_index_r)-1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are vector_load_index_r: an integer selecting one tensor element, lane, bank, or state entry; vector_operand_a_r: the value named ’vector operand a r’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 841: `<= vector_load_data_r;` This is continuation line 3 of the 3-line logical statement that begins at source line 839. Its exact fragment is ‘<= vector_load_data_r;‘. The important operands are vector_load_data_r: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This condition selects the following hardware branch only when ‘vector_load_index_r <= 4) begin vector_operand_a_r[(int’(vector_load_index_r)-1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are vector_load_index_r: an integer selecting one tensor element, lane, bank, or state entry; vector_operand_a_r: the value named ’vector operand a r’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 842: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 843: `vector_operand_b_r[(int'(vector_load_index_r)-5)*64 +: 64]` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are vector_operand_b_r: the value named 'vector operand b r' in this block; vector_load_index_r: an integer selecting one tensor element, lane, bank, or state entry.

Line 844: `<= vector_load_data_r;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are vector_load_data_r: the payload carried with the surrounding valid/ready handshake.

Line 845: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 846: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 847: `if (clear_counters_event_r) begin` This condition selects the following hardware branch only when ‘clear_counters_event_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are clear_counters_event_r: a hardware telemetry register that accumulates measured events. Context: the clocked process that updates registers only on a rising clock edge.

Line 848: `status_input_beats <= 32'd0;` This makes status_input_beats, the value named 'status input beats' in this block, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 849: `status_output_beats <= 32'd0;` This makes status_output_beats, the value named 'status output beats' in this block, equal to ‘32'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 850: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 851: `if (input_beat_accepted_r) begin` This condition selects the following hardware branch only when ‘input_beat_accepted_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_beat_accepted_r: the value named 'input beat accepted r' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 852: `status_input_beats <= status_input_beats + 1'b1;` This makes status_input_beats, the value named 'status input beats' in this block, equal to ‘status_input_beats + 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_input_beats: the value named 'status input beats' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 853: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 854: `if (output_beat_accepted_r) begin` This condition selects the following hardware branch only when ‘output_beat_accepted_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_beat_accepted_r: the value named ’output beat accepted r’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 855: `status_output_beats <= status_output_beats + 1'b1;` This makes status_output_beats, the value named ’status output beats’ in this block, equal to ‘status_output_beats + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_output_beats: the value named ’status output beats’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 856: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 857: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 859: `unique case (state)` This starts a multiplexer/state decode selected by ‘state‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 860: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 861: `if (command_valid && command_ready) begin` This condition selects the following hardware branch only when ‘command_valid && command_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are command_valid: the producer’s statement that its current payload is meaningful; command_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 862: `captured_command_r <= command_data[108:0];` This makes captured_command_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘command_data[108:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 863: `// Stage the wide reserved-field reduction across the` Design intent: Stage the wide reserved-field reduction across the This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 864: `// existing capture/decode clocks for 200 MHz timing.` Design intent: existing capture/decode clocks for 200 MHz timing. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 865: `train_reserved_partial_r[0] <= |command_data[61:30];` This makes train_reserved_partial_r[0], the value named ’train reserved partial r’ in this block, equal to ‘|command_data[61:30]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 866: `train_reserved_partial_r[1] <= |command_data[93:62];` This makes train_reserved_partial_r[1], the value named 'train reserved partial r' in this block, equal to '|command_data[93:62]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 867: `train_reserved_partial_r[2] <= |command_data[125:94];` This makes train_reserved_partial_r[2], the value named 'train reserved partial r' in this block, equal to '|command_data[125:94]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 868: `train_reserved_partial_r[3] <= |command_data[127:126];` This makes train_reserved_partial_r[3], the value named 'train reserved partial r' in this block, equal to '|command_data[127:126]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 869: `status_active_opcode <= command_data[3:0];` This makes status_active_opcode, the value named 'status active opcode' in this block, equal to 'command_data[3:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are command_data: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 870: `status_command_count <= status_command_count + 1'b1;` This makes status_command_count, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'status_command_count + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are status_command_count: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 871: `stream_beat_index <= 8'd0;` This makes stream_beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 872: `stream_item_index <= 12'd0;` This makes stream_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 873: `stream_items_remaining_r <= 12'd0;` This makes stream_items_remaining_r, the value named 'stream items remaining r' in this block, equal to '12'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 874: `stream_last_pending_r <= 1'b0;` This makes stream_last_pending_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 875: `output_word_index <= 4'd0;` This makes output_word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘4'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 876: `command_had_output_r <= 1'b0;` This makes command_had_output_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 877: `final_output_accepted_r <= 1'b0;` This makes final_output_accepted_r, the value named 'final output accepted r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 878: `completion_error_r <= 1'b0;` This makes completion_error_r, the value named 'completion error r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 879: `state <= ST_DECODE_COMMAND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_DECODE_COMMAND‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DECODE_COMMAND: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 880: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 881: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 883: `ST_DECODE_COMMAND: begin` This is the ST_DECODE_COMMAND case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DECODE_COMMAND, the machine performs the named control phase..

Line 884: `unique case (captured_command_r[3:0])` This starts a multiplexer/state decode selected by ‘captured_command_r ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 885: `CMD_TRAIN_LINEAR_BACKWARD: begin` This is the CMD_TRAIN_LINEAR_BACKWARD case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state CMD_TRAIN_LINEAR_BACKWARD, the machine performs the named control phase..

Line 886: `train_reserved_nonzero_r <= |train_reserved_partial_r;` This makes train_reserved_nonzero_r, the value named 'train reserved nonzero r' in this block, equal to ‘|train_reserved_partial_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are train_reserved_partial_r: the

value named 'train reserved partial r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_TRAIN_LINEAR_BACKWARD, the machine performs the named control phase..

Line 887: `state <= ST_TRAIN_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_TRAIN_START'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_TRAIN_START: the value named 'st train start' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_TRAIN_LINEAR_BACKWARD, the machine performs the named control phase..

Line 888: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_TRAIN_LINEAR_BACKWARD, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 889: `CMD_LOAD_ACTIVATIONS, CMD_LOAD_WEIGHTS_OUTER,` Within the clocked process that updates registers only on a rising clock edge Within state CMD_TRAIN_LINEAR_BACKWARD, the machine performs the named control phase., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are CMD_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; CMD_LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation.

Line 890: `CMD_LOAD_WEIGHTS_DOT8, CMD_COMPUTE_DENSE_OUTER,` Within the clocked process that updates registers only on a rising clock edge Within state CMD_TRAIN_LINEAR_BACKWARD, the machine performs the named control phase., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are CMD_LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation; CMD_COMPUTE_DENSE_OUTER: the value named 'cmd compute dense outer' in this block.

Line 891: `CMD_COMPUTE_DENSE_DOT8, CMD_COMPUTE_BINARY,` Within the clocked process that updates registers only on a rising clock edge Within state CMD_TRAIN_LINEAR_BACKWARD, the machine performs the named control phase., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are CMD_COMPUTE_DENSE_DOT8: the value named 'cmd compute dense dot8' in this block; CMD_COMPUTE_BINARY: the value named 'cmd compute binary' in this block.

Line 892: `CMD_COMPUTE_TERNARY: begin` This is the CMD_COMPUTE_TERNARY case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_TERNARY, the machine issues arithmetic work using values already held on chip..

Line 893: `if (tensor_command_ready) state <= ST_TENSOR;` This condition selects the following hardware branch only when 'tensor_command_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tensor_command_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_TERNARY, the machine issues arithmetic work using values already held on chip..

Line 894: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_COMPUTE_TERNARY, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 895: `CMD_ALPHA_XNOR: begin` This is the CMD_ALPHA_XNOR case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_ALPHA_XNOR, the machine performs the named control phase..

Line 896: `alpha_active_bits_r <= captured_command_r[50:42];` This makes alpha_active_bits_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to ‘captured_command_r[50:42]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ALPHA_XNOR, the machine performs the named control phase..

Line 897: `alpha_q8_r <= captured_command_r[59:51];` This makes alpha_q8_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to ‘captured_command_r[59:51]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ALPHA_XNOR, the machine performs the named control phase..

Line 898: `// ST_ALPHA_LOAD commits all four query words and all` Design intent: ST_ALPHA_LOAD commits all four query words and all This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 899: `// four key words before start, so retaining the prior` Design intent: four key words before start, so retaining the prior This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 900: `// command here cannot expose stale operand bits.` Design intent: command here cannot expose stale operand bits. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 901: `state <= ST_ALPHA_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_ALPHA_LOAD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ALPHA_LOAD: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ALPHA_XNOR, the machine performs the named control phase..

Line 902: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_ALPHA_XNOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 903: `CMD_VECTOR_ALU: begin` This is the CMD_VECTOR_ALU case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state CMD_VECTOR_ALU, the machine performs the named control phase..

Line 904: `vector_operation_r <= captured_command_r[7:4];` This makes vector_operation_r, the value named 'vector operation r' in this block, equal to ‘captured_command_r[7:4]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_VECTOR_ALU, the machine performs the named control phase..

Line 905: `vector_multiplier_r <= captured_command_r[23:8];` This makes vector_multiplier_r, the value named 'vector multiplier r' in this block, equal to ‘captured_command_r[23:8]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_VECTOR_ALU, the machine performs the named control phase..

Line 906: `vector_shift_r <= captured_command_r[28:24];` This makes vector_shift_r, the value named 'vector shift r' in this block, equal to 'captured_command_r[28:24]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_VECTOR_ALU, the machine performs the named control phase..

Line 907: `vector_zero_point_r <= captured_command_r[44:29];` This makes vector_zero_point_r, the integer code representing real numeric zero in affine quantization, equal to 'captured_command_r[44:29]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_VECTOR_ALU, the machine performs the named control phase..

Line 908: `vector_threshold_r <= captured_command_r[76:45];` This makes vector_threshold_r, the membrane level at which a neuron emits a spike, equal to 'captured_command_r[76:45]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_VECTOR_ALU, the machine performs the named control phase..

Line 909: `vector_window_r <= captured_command_r[108:77];` This makes vector_window_r, the value named 'vector window r' in this block, equal to 'captured_command_r[108:77]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_VECTOR_ALU, the machine performs the named control phase..

Line 910: `state <= ST_VECTOR_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_VECTOR_LOAD'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VECTOR_LOAD: the value named 'st vector load' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_VECTOR_ALU, the machine performs the named control phase..

Line 911: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_VECTOR_ALU, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 912: `CMD_LAYERNORM: begin` This is the CMD_LAYERNORM case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_LAYERNORM, the machine performs the named control phase..

Line 913: `ln_count_r <= captured_command_r[15:4];` This makes ln_count_r, an integer count retained for control or measurable telemetry, equal to 'captured_command_r[15:4]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LAYERNORM, the machine performs the named control phase..

Line 914: `stream_items_remaining_r <= captured_command_r[15:4];` This makes stream_items_remaining_r, the value named 'stream items remaining r' in this block, equal to 'captured_command_r[15:4]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LAYERNORM, the machine performs the named control phase..

Line 915: `stream_last_pending_r <= captured_command_r[15:4] == 1;` This makes stream_last_pending_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'captured_command_r[15:4] == 1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context:

the clocked process that updates registers only on a rising clock edge Within state CMD_LAYERNORM, the machine performs the named control phase..

Line 916: `ln_epsilon_r <= captured_command_r[47:16];` This makes ln_epsilon_r, the value named 'ln epsilon r' in this block, equal to 'captured_command_r[47:16]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LAYERNORM, the machine performs the named control phase..

Line 917: `state <= ST_LAYERNORM_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LAYERNORM_START'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LAYERNORM_START: the value named 'st layernorm start' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LAYERNORM, the machine performs the named control phase..

Line 918: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LAYERNORM, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 919: `CMD_LOG_SOFTMAX: begin` This is the CMD_LOG_SOFTMAX case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_LOG_SOFTMAX, the machine performs the named control phase..

Line 920: `softmax_count_r <= captured_command_r[12:4];` This makes softmax_count_r, an integer count retained for control or measurable telemetry, equal to 'captured_command_r[12:4]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOG_SOFTMAX, the machine performs the named control phase..

Line 921: `stream_items_remaining_r <= {3'd0, captured_command_r[12:4]};` This makes stream_items_remaining_r, the value named 'stream items remaining r' in this block, equal to '{3'd0, captured_command_r[12:4]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block; captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOG_SOFTMAX, the machine performs the named control phase..

Line 922: `stream_last_pending_r <= captured_command_r[12:4] == 1;` This makes stream_last_pending_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'captured_command_r[12:4] == 1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOG_SOFTMAX, the machine performs the named control phase..

Line 923: `softmax_shift_r <= captured_command_r[17:13];` This makes softmax_shift_r, the value named 'softmax shift r' in this block, equal to 'captured_command_r[17:13]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOG_SOFTMAX, the machine performs the named control phase..

Line 924: `state <= ST_SOFTMAX_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_SOFTMAX_START'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SOFTMAX_START: the value named 'st softmax start' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LOG_SOFTMAX, the machine performs the named control phase..

Line 925: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LOG_SOFTMAX, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 926: `CMD_LIF_STEP: begin` This is the CMD_LIF_STEP case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 927: `lif_leak_shift_r <= captured_command_r[8:4];` This makes lif_leak_shift_r, the retained fraction of prior membrane state, equal to 'captured_command_r[8:4]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 928: `lif_threshold_r <= captured_command_r[32:9];` This makes lif_threshold_r, the membrane level at which a neuron emits a spike, equal to 'captured_command_r[32:9]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 929: `lif_reset_r <= captured_command_r[56:33];` This makes lif_reset_r, the value named 'lif reset r' in this block, equal to 'captured_command_r[56:33]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 930: `lif_clear_requested_r <= captured_command_r[57];` This makes lif_clear_requested_r, the value named 'lif clear requested r' in this block, equal to 'captured_command_r[57]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 931: `lif_trace_requested_r <= captured_command_r[58];` This makes lif_trace_requested_r, the value named 'lif trace requested r' in this block, equal to 'captured_command_r[58]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 932: `// The protocol always loads all eight words before` Design intent: The protocol always loads all eight words before This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 933: `// starting the child, so a 512-bit command-time clear` Design intent: starting the child, so a 512-bit command-time clear This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 934: `// would only add a global mux to every current bit.` Design intent: would only add a global mux to every current bit. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 935: `lif_load_word_select_r <= 8'd0;` This makes lif_load_word_select_r, the value named 'lif load word select r' in this block, equal to '8'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 936: `lif_load_pending_r <= 1'b0;` This makes lif_load_pending_r, the value named 'lif load pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 937: `lif_load_index_r <= 3'd0;` This makes lif_load_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to '3'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 938: `lif_load_final_r <= 1'b0;` This makes lif_load_final_r, the value named 'lif load final r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 939: `state <= ST_LIF_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LIF_LOAD'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LIF_LOAD: the value named 'st lif load' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase..

Line 940: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_LIF_STEP, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 941: `CMD_SGD_UPDATE: begin` This is the CMD_SGD_UPDATE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state CMD_SGD_UPDATE, the machine performs the named control phase..

Line 942: `optimizer_lr_r <= captured_command_r[21:4];` This makes optimizer_lr_r, the value named 'optimizer lr r' in this block, equal to 'captured_command_r[21:4]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_SGD_UPDATE, the machine performs the named control phase..

Line 943: `optimizer_momentum_r <= captured_command_r[38:22];` This makes optimizer_momentum_r, the value named 'optimizer momentum r' in this block, equal to 'captured_command_r[38:22]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_SGD_UPDATE, the machine performs the named control phase..

Line 944: `optimizer_decay_r <= captured_command_r[55:39];` This makes optimizer_decay_r, the value named 'optimizer decay r' in this block, equal to 'captured_command_r[55:39]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_SGD_UPDATE, the machine performs the named control phase..

Line 945: `optimizer_update_shift_r <= captured_command_r[60:56];` This makes optimizer_update_shift_r, the value named 'optimizer update shift r' in this block, equal to 'captured_command_r[60:56]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_SGD_UPDATE, the machine performs the named control phase..

Line 946: `state <= ST_OPTIMIZER_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OPTIMIZER_LOAD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OPTIMIZER_LOAD: the value named ’st optimizer load’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_SGD_UPDATE, the machine performs the named control phase..

Line 947: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_SGD_UPDATE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 948: `CMD_ACTIVATION_LUT: begin` This is the CMD_ACTIVATION_LUT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 949: `stream_items_remaining_r <= captured_command_r[19:8];` This makes stream_items_remaining_r, the value named ’stream items remaining r’ in this block, equal to ‘captured_command_r[19:8]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 950: `stream_last_pending_r <= captured_command_r[19:8] == 1;` This makes stream_last_pending_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘captured_command_r[19:8] == 1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 951: `if (captured_command_r[7:4] == 0) begin` This condition selects the following hardware branch only when ‘captured_command_r[7:4] == 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 952: `lut_clear_requested_r <= 1’b1;` This makes lut_clear_requested_r, the value named ’lut clear requested r’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 953: `decode_error_pending_r <= 1’b0;` This makes decode_error_pending_r, the value named ’decode error pending r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 954: `state <= ST_LUT_CLEAR;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LUT_CLEAR‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LUT_CLEAR: the value named ’st lut clear’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 955: `end else if (captured_command_r[7:4] == 1` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 956: `&& captured_command_r[19:8] != 0) begin` Within the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 957: `lut_load_pending_r <= 1'b0;` This makes lut_load_pending_r, the value named 'lut load pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 958: `state <= ST_LUT_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LUT_LOAD'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LUT_LOAD: the value named 'st lut load' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 959: `end else if (captured_command_r[7:4] == 2) begin` This condition selects the following hardware branch only when 'captured_command_r[7:4] == 2' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are captured_command_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 960: `state <= ST_LUT_EVAL_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LUT_EVAL_LOAD'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LUT_EVAL_LOAD: the value named 'st lut eval load' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 961: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 962: `lut_clear_requested_r <= 1'b0;` This makes lut_clear_requested_r, the value named 'lut clear requested r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 963: `decode_error_pending_r <= 1'b1;` This makes decode_error_pending_r, the value named 'decode error pending r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 964: `state <= ST_LUT_CLEAR;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LUT_CLEAR'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LUT_CLEAR: the value named 'st lut clear' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase..

Line 965: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 966: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_ACTIVATION_LUT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 967: `CMD_CLEAR_STATUS: begin` This is the CMD_CLEAR_STATUS case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 968: `if (tensor_command_ready) begin` This condition selects the following hardware branch only when ‘tensor_command_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tensor_command_ready: the consumer's statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 969: `clear_error_r <= 1'b1;` This makes clear_error_r, the value named 'clear error r' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 970: `clear_counters_event_r <= 1'b1;` This makes clear_counters_event_r, a hardware telemetry register that accumulates measured events, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 971: `state <= ST_TENSOR;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_TENSOR‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_TENSOR: the value named 'st tensor' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 972: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 973: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 974: `default: begin` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 975: `lut_clear_requested_r <= 1'b0;` This makes lut_clear_requested_r, the value named 'lut clear requested r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 976: `decode_error_pending_r <= 1'b1;` This makes decode_error_pending_r, the value named 'decode error pending r' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 977: `state <= ST_LUT_CLEAR;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LUT_CLEAR‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LUT_CLEAR: the value named ’st lut clear’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase..

Line 978: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 979: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state CMD_CLEAR_STATUS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 980: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 982: `ST_TENSOR: begin` This is the ST_TENSOR case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 983: `if (tensor_command_done) begin` This condition selects the following hardware branch only when ‘tensor_command_done‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tensor_command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 984: `if (command_had_output_r || tensor_result_accept) begin` This condition selects the following hardware branch only when ‘command_had_output_r || tensor_result_accept‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are command_had_output_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; tensor_result_accept: the value named ’tensor result accept’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 985: `completion_error_r <= tensor_status_error;` This makes completion_error_r, the value named ’completion error r’ in this block, equal to ‘tensor_status_error‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are tensor_status_error: the value named ’tensor status error’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 986: `state <= ST_OUTPUT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_DRAIN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DRAIN: the value named ’st output drain’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 987: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 988: `if (tensor_status_error) error_event_r <= 1'b1;` This condition selects the following hardware branch only when ‘tensor_status_error‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are tensor_status_error: the value named ’tensor status error’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 989: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 990: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase..

Line 991: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 992: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 993: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_TENSOR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 994: `ST_ALPHA_LOAD: begin` This is the ST_ALPHA_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 995: `if (alpha_load_valid_r && alpha_load_final_r) begin` This condition selects the following hardware branch only when ‘alpha_load_valid_r && alpha_load_final_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are alpha_load_valid_r: the producer's statement that its current payload is meaningful; alpha_load_final_r: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 996: `alpha_load_final_r <= 1'b0;` This makes alpha_load_final_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 997: `state <= ST_ALPHA_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_ALPHA_DRAIN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ALPHA_DRAIN: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 998: `end else if (s_axis_tvalid) begin` This condition selects the following hardware branch only when ‘s_axis_tvalid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 999: `alpha_load_valid_r <= 1'b1;` This makes alpha_load_valid_r, the producer's statement that its current payload is meaningful, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1000: `alpha_load_index_r <= stream_beat_index[2:0];` This makes alpha_load_index_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to 'stream_beat_index[2:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1001: `alpha_load_data_r <= s_axis_tdata;` This makes alpha_load_data_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to 's_axis_tdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1002: `alpha_load_final_r <= stream_beat_index == 7;` This makes alpha_load_final_r, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to 'stream_beat_index == 7'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1003: `alpha_load_last_r <= s_axis_tlast;` This makes alpha_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 's_axis_tlast'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1004: `if (stream_beat_index != 7) begin` This condition selects the following hardware branch only when 'stream_beat_index != 7' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1005: `stream_beat_index <= stream_beat_index + 1'b1;` This makes stream_beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'stream_beat_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1006: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1007: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1008: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1009: `ST_ALPHA_DRAIN: if (!alpha_load_valid_r) state <= ST_ALPHA_START;` This is the ST_ALPHA_DRAIN case branch, which performs this explicitly named phase. The same-cycle action begins with ‘if (!alpha_load_valid_r) state <= ST_ALPHA_START;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_DRAIN, the machine performs the named control phase..

Line 1010: `ST_ALPHA_START: if (alpha_input_ready) state <= ST_ALPHA_OUTPUT;` This is the ST_ALPHA_START case branch, which performs this explicitly named phase. The same-cycle action begins with ‘if (alpha_input_ready) state <= ST_ALPHA_OUTPUT;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_START, the machine performs the named control phase..

Line 1011: `ST_ALPHA_OUTPUT: if (alpha_result_accept) begin` This is the ST_ALPHA_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘if (alpha_result_accept) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1012: `completion_error_r <= alpha_config_error;` This makes completion_error_r, the value named ’completion error r’ in this block, equal to ‘alpha_config_error‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are alpha_config_error: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1013: `state <= ST_OUTPUT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_DRAIN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DRAIN: the value named ’st output drain’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1014: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ALPHA_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1015: `ST_VECTOR_LOAD: begin` This is the ST_VECTOR_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1016: `if (vector_load_final_r) begin` This condition selects the following hardware branch only when ‘vector_load_final_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are vector_load_final_r: the value named ’vector load final r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1017: `vector_load_final_r <= 1'b0;` This makes vector_load_final_r, the value named ’vector load final r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1018: `state <= ST_VECTOR_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_VECTOR_DRAIN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VECTOR_DRAIN: the value named ’st vector drain’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1019: `end else if (s_axis_tvalid) begin` This condition selects the following hardware branch only when ‘s_axis_tvalid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tvalid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1020: `vector_load_valid_r <= 1'b1;` This makes vector_load_valid_r, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1021: `vector_load_index_r <= stream_beat_index[3:0];` This makes vector_load_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘stream_beat_index[3:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1022: `vector_load_data_r <= s_axis_tdata;` This makes vector_load_data_r, the payload carried with the surrounding valid/ready handshake, equal to ‘s_axis_tdata‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named ’s axis tdata’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1023: `vector_load_final_r <= stream_beat_index == 8;` This makes vector_load_final_r, the value named ’vector load final r’ in this block, equal to ‘stream_beat_index == 8‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1024: `vector_load_last_r <= s_axis_tlast;` This makes vector_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘s_axis_tlast‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1025: `if (stream_beat_index != 8) begin` This condition selects the following hardware branch only when ‘stream_beat_index != 8‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1026: `stream_beat_index <= stream_beat_index + 1'b1;` This makes stream_beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘stream_beat_index + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1027: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1028: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1029: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1030: `// The registered final payload commits at this edge.` Design intent: The registered final payload commits at this edge. This comment anchors the surrounding logic in the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

Line 1031: `ST_VECTOR_DRAIN: state <= ST_VECTOR_START;` This is the ST_VECTOR_DRAIN case branch, which performs this explicitly named phase. The same-cycle action begins with 'state <= ST_VECTOR_START;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_DRAIN, the machine performs the named control phase..

Line 1032: `ST_VECTOR_START: if (vector_input_ready) begin` This is the ST_VECTOR_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'if (vector_input_ready) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_START, the machine performs the named control phase..

Line 1033: `output_word_index <= 4'd0;` This makes output_word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_START, the machine performs the named control phase..

Line 1034: `state <= ST_VECTOR_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_VECTOR_OUTPUT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_VECTOR_OUTPUT: the value named 'st vector output' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_START, the machine performs the named control phase..

Line 1035: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_START, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1036: `ST_VECTOR_OUTPUT: if (vector_result_accept) begin` This is the ST_VECTOR_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'if (vector_result_accept) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1037: `if (output_word_index == 3) begin` This condition selects the following hardware branch only when 'output_word_index == 3' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1038: `completion_error_r <= vector_config_error;` This makes completion_error_r, the value named 'completion error r' in this block, equal to 'vector_config_error'. It updates after the active clock edge together with other nonblocking assignments. The important operands are vector_config_error: the value named 'vector config error' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1039: `state <= ST_OUTPUT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_DRAIN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DRAIN: the value named 'st output drain' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1040: `end else output_word_index <= output_word_index + 1'b1;` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1041: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_VECTOR_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1042: `ST_LAYERNORM_START: if (ln_start_accept_r) state <= ST_LAYERNORM_LOAD0;` This is the ST_LAYERNORM case branch, which performs this explicitly named phase. The same-cycle action begins with 'if (ln_start_accept_r) state <= ST_LAYERNORM_LOAD0;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_START, the machine performs the named control phase..

Line 1043: `ST_LAYERNORM_LOAD0: if (s_axis_tvalid) begin` This is the ST_LAYERNORM_LOAD0 case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'if (s_axis_tvalid) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD0, the machine stores incoming operands into local FPGA memory..

Line 1044: `ln_sample_word0_r <= s_axis_tdata;` This makes ln_sample_word0_r, the value named 'ln sample word0 r' in this block, equal to 's_axis_tdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD0, the machine stores incoming operands into local FPGA memory..

Line 1045: `state <= ST_LAYERNORM_LOAD1;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LAYERNORM_LOAD1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LAYERNORM_LOAD1: the value named 'st layernorm load1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD0, the machine stores incoming operands into local FPGA memory..

Line 1046: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD0, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1047: `ST_LAYERNORM_LOAD1: if (s_axis_tvalid) begin` This is the ST_LAYERNORM_LOAD1 case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'if (s_axis_tvalid) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD1, the machine stores incoming operands into local FPGA memory..

Line 1048: `ln_sample_value_r <= ln_sample_word0_r[23:0];` This makes ln_sample_value_r, the value named 'ln sample value r' in this block, equal to 'ln_sample_word0_r[23:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ln_sample_word0_r: the value named 'ln sample word0 r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD1, the machine stores incoming operands into local FPGA memory..

Line 1049: `ln_sample_gamma_r <= ln_sample_word0_r[47:24];` This makes ln_sample_gamma_r, the learned multiplicative LayerNorm parameter, equal to 'ln_sample_word0_r[47:24]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ln_sample_word0_r: the value named 'ln sample word0 r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD1, the machine stores incoming operands into local FPGA memory..

Line 1050: `ln_sample_beta_r <= {s_axis_tdata[7:0], ln_sample_word0_r[63:48]};` This makes ln_sample_beta_r, the learned additive LayerNorm parameter, equal to '{s_axis_tdata[7:0], ln_sample_word0_r[63:48]}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block; ln_sample_word0_r: the value named 'ln sample word0 r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD1, the machine stores incoming operands into local FPGA memory..

Line 1051: `if (s_axis_tlast != stream_last_pending_r) begin` This condition selects the following hardware branch only when 's_axis_tlast != stream_last_pending_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; stream_last_pending_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD1, the machine stores incoming operands into local FPGA memory..

Line 1052: `error_event_r <= 1'b1;` This makes error_event_r, the value named 'error event r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD1, the machine stores incoming operands into local FPGA memory..

Line 1053: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD1, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1054: `state <= ST_LAYERNORM_SEND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LAYERNORM_SEND'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LAYERNORM_SEND: the value named 'st layernorm send' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD1, the machine stores incoming operands into local FPGA memory..

Line 1055: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_LOAD1, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1056: `ST_LAYERNORM_SEND: if (ln_sample_ready) begin` This is the ST_LAYERNORM_SEND case branch, which performs this explicitly named phase. The same-cycle action begins with 'if (ln_sample_ready) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase..

Line 1057: `if (stream_last_pending_r) begin` This condition selects the following hardware branch only when 'stream_last_pending_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are stream_last_pending_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase..

Line 1058: `state <= ST_LAYERNORM_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LAYERNORM_OUTPUT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LAYERNORM_OUTPUT: the value named ’st layernorm output’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase..

Line 1059: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1060: `stream_item_index <= stream_item_index + 1’b1;` This makes stream_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘stream_item_index + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_item_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase..

Line 1061: `stream_items_remaining_r <= stream_items_remaining_r - 1’b1;` This makes stream_items_remaining_r, the value named ’stream items remaining r’ in this block, equal to ‘stream_items_remaining_r - 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_items_remaining_r: the value named ’stream items remaining r’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase..

Line 1062: `stream_last_pending_r <= stream_items_remaining_r == 12’d2;` This makes stream_last_pending_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘stream_items_remaining_r == 12’d2‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_items_remaining_r: the value named ’stream items remaining r’ in this block; d2: the value named ’d2’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase..

Line 1063: `state <= ST_LAYERNORM_LOAD0;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LAYERNORM_LOAD0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LAYERNORM_LOAD0: the value named ’st layernorm load0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase..

Line 1064: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1065: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_SEND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1066: `ST_LAYERNORM_OUTPUT: if (layernorm_result_accept` This is the ST_LAYERNORM_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘if (layernorm_result_accept‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1067: `&& ln_result_last) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are ln_result_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 1068: `completion_error_r <= ln_config_error;` This makes completion_error_r, the value named 'completion error r' in this block, equal to 'ln_config_error'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ln_config_error: the value named 'ln config error' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1069: `state <= ST_OUTPUT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_DRAIN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DRAIN: the value named 'st output drain' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1070: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LAYERNORM_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1071: `ST_SOFTMAX_START: if (softmax_start_accept_r) state <= ST_SOFTMAX_LOAD;` This is the ST_SOFTMAX_S case branch, which performs this explicitly named phase. The same-cycle action begins with 'if (softmax_start_accept_r) state <= ST_SOFTMAX_LOAD;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_START, the machine performs the named control phase..

Line 1072: `ST_SOFTMAX_LOAD: if (s_axis_tvalid) begin` This is the ST_SOFTMAX_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'if (s_axis_tvalid) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1073: `if (s_axis_tlast != stream_last_pending_r) begin` This condition selects the following hardware branch only when 's_axis_tlast != stream_last_pending_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; stream_last_pending_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1074: `error_event_r <= 1'b1;` This makes error_event_r, the value named 'error event r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1075: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1076: `if (stream_last_pending_r) begin` This condition selects the following hardware branch only when 'stream_last_pending_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are stream_last_pending_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1077: `state <= ST_SOFTMAX_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_SOFTMAX_OUTPUT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SOFTMAX_OUTPUT: the value named 'st softmax output' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1078: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1079: `stream_item_index <= stream_item_index + 1'b1;` This makes stream_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'stream_item_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_item_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1080: `stream_items_remaining_r <= stream_items_remaining_r - 1'b1;` This makes stream_items_remaining_r, the value named 'stream items remaining r' in this block, equal to 'stream_items_remaining_r - 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_items_remaining_r: the value named 'stream items remaining r' in this block; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1081: `stream_last_pending_r <= stream_items_remaining_r == 12'd2;` This makes stream_last_pending_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'stream_items_remaining_r == 12'd2'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_items_remaining_r: the value named 'stream items remaining r' in this block; d2: the value named 'd2' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1082: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1083: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1084: `ST_SOFTMAX_OUTPUT: if (softmax_result_accept` This is the ST_SOFTMAX_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'if (softmax_result_accept'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1085: `&& softmax_probability_last) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are softmax_probability_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 1086: `completion_error_r <= softmax_config_error;` This makes completion_error_r, the value named 'completion error r' in this block, equal to 'softmax_config_error'. It updates after the active clock edge together with other nonblocking assignments. The important operands are softmax_config_error: the value named 'softmax config error' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1087: `state <= ST_OUTPUT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_DRAIN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DRAIN: the value named 'st output drain' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1088: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SOFTMAX_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1089: `ST_LIF_LOAD: begin` This is the ST_LIF_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1090: `if (lif_load_pending_r) begin` This condition selects the following hardware branch only when 'lif_load_pending is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lif_load_pending_r: the value named 'lif load pending r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1091: `if (lif_load_last_r != lif_load_final_r) begin` This condition selects the following hardware branch only when 'lif_load_last_r != lif_load_final_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lif_load_last_r: the AXI-Stream marker for the final beat of a packet or command payload; lif_load_final_r: the value named 'lif load final r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1092: `error_event_r <= 1'b1;` This makes error_event_r, the value named 'error event r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1093: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1094: `if (lif_load_final_r) begin` This condition selects the following hardware branch only when 'lif_load_final_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lif_load_final_r: the value named 'lif load final r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1095: `if (lif_clear_requested_r) state <= ST_LIF_CLEAR;` This condition selects the following hardware branch only when 'lif_clear_requested_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lif_clear_requested_r: the value named 'lif clear requested r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1096: `else state <= ST_LIF_START;` Within the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_LIF_START: the value named 'st lif start' in this block.

Line 1097: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1098: `lif_load_index_r <= lif_load_index_r + 1'b1;` This makes lif_load_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘lif_load_index_r + 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are lif_load_index_r: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1099: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1100: `end else if (s_axis_tvalid) begin` This condition selects the following hardware branch only when ‘s_axis_tvalid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1101: `lif_load_pending_r <= 1'b1;` This makes lif_load_pending_r, the value named 'lif load pending r' in this block, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1102: `lif_load_word_select_r <= 8'b1 << lif_load_index_r;` This makes lif_load_word_select_r, the value named 'lif load word select r' in this block, equal to ‘8'b1 << lif_load_index_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block; lif_load_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1103: `lif_load_data_r <= s_axis_tdata;` This makes lif_load_data_r, the payload carried with the surrounding valid/ready handshake, equal to ‘s_axis_tdata‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1104: `lif_load_last_r <= s_axis_tlast;` This makes lif_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘s_axis_tlast‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1105: `lif_load_final_r <= lif_load_index_r == 7;` This makes lif_load_final_r, the value named 'lif load final r' in this block, equal to ‘lif_load_index_r == 7‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are lif_load_index_r: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1106: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1107: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1108: `ST_LIF_CLEAR: state <= ST_LIF_START;` This is the ST_LIF_CLEAR case branch, which performs this explicitly named phase. The same-cycle action begins with ‘state <= ST_LIF_START;‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LIF_CLEAR, the machine performs the named control phase..

Line 1109: `ST_LIF_START: if (lif_input_ready) begin` This is the ST_LIF_START case branch, which performs this explicitly named phase. The same-cycle action begins with ‘if (lif_input_ready) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LIF_START, the machine performs the named control phase..

Line 1110: `output_word_index <= 4'd0;` This makes output_word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘4'd0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_START, the machine performs the named control phase..

Line 1111: `state <= ST_LIF_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LIF_OUTPUT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LIF_OUTPUT: the value named 'st lif output' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_START, the machine performs the named control phase..

Line 1112: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_START, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1113: `ST_LIF_OUTPUT: if (lif_result_accept) begin` This is the ST_LIF_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘if (lif_result_accept) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1114: `if (!lif_trace_requested_r || output_word_index == 12) begin` This condition selects the following hardware branch only when ‘!lif_trace_requested_r || output_word_index == 12‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lif_trace_requested_r: the value named 'lif trace requested r' in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1115: `completion_error_r <= 1'b0;` This makes completion_error_r, the value named 'completion error r' in this block, equal to ‘1'b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1116: `state <= ST_OUTPUT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_DRAIN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DRAIN: the value named 'st output drain' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1117: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1118: `output_word_index <= output_word_index + 1'b1;` This makes output_word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_word_index + 1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1119: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1120: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LIF_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1121: `ST_OPTIMIZER_LOAD: if (s_axis_tvalid) begin` This is the ST_OPTIMIZER_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘if (s_axis_tvalid) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1122: `optimizer_load_data_r <= s_axis_tdata;` This makes optimizer_load_data_r, the payload carried with the surrounding valid/ready handshake, equal to ‘s_axis_tdata‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1123: `optimizer_load_last_r <= s_axis_tlast;` This makes optimizer_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘s_axis_tlast‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1124: `optimizer_load_index_r <= stream_beat_index[2:0];` This makes optimizer_load_index_r, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘stream_beat_index[2:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1125: `optimizer_load_final_r <= stream_beat_index == 7;` This makes optimizer_load_final_r, the value named 'optimizer load final r' in this block, equal to ‘stream_beat_index == 7‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1126: `optimizer_commit_valid_r <= 1'b1;` This makes optimizer_commit_valid_r, the producer's statement that its current payload is meaningful, equal to ‘1'b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1127: `state <= ST_OPTIMIZER_COMMIT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OPTIMIZER_COMMIT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OPTIMIZER_COMMIT: the value named 'st optimizer commit' in this block.

Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1128: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1129: `ST_OPTIMIZER_COMMIT: begin` This is the ST_OPTIMIZER_COMMIT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase..

Line 1130: `if (optimizer_load_last_r != optimizer_load_final_r) begin` This condition selects the following hardware branch only when 'optimizer_load_last_r != optimizer_load_final_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are optimizer_load_last_r: the AXI-Stream marker for the final beat of a packet or command payload; optimizer_load_final_r: the value named 'optimizer load final r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase..

Line 1131: `error_event_r <= 1'b1;` This makes error_event_r, the value named 'error event r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase..

Line 1132: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1133: `if (optimizer_load_final_r) begin` This condition selects the following hardware branch only when 'optimizer_load_final_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are optimizer_load_final_r: the value named 'optimizer load final r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase..

Line 1134: `state <= ST_OPTIMIZER_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OPTIMIZER_START'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OPTIMIZER_START: the value named 'st optimizer start' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase..

Line 1135: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1136: `stream_beat_index <= stream_beat_index + 1'b1;` This makes stream_beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'stream_beat_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_beat_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase..

Line 1137: `state <= ST_OPTIMIZER_LOAD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OPTIMIZER_LOAD'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OPTIMIZER_LOAD: the value named 'st optimizer load' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase..

Line 1138: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1139: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_COMMIT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1140: `ST_OPTIMIZER_START: if (optimizer_input_ready) begin` This is the ST_OPTIMIZER_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'if (optimizer_input_ready) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_START, the machine performs the named control phase..

Line 1141: `output_word_index <= 4'd0;` This makes output_word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '4'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_START, the machine performs the named control phase..

Line 1142: `state <= ST_OPTIMIZER_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OPTIMIZER_OUTPUT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OPTIMIZER_OUTPUT: the value named 'st optimizer output' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_START, the machine performs the named control phase..

Line 1143: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_START, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1144: `ST_OPTIMIZER_OUTPUT: if (optimizer_result_accept) begin` This is the ST_OPTIMIZER_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'if (optimizer_result_accept) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1145: `if (output_word_index == 4'd7) begin` This condition selects the following hardware branch only when 'output_word_index == 4'd7' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry; d7: the value named 'd7' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1146: `completion_error_r <= optimizer_config_error;` This makes completion_error_r, the value named 'completion error r' in this block, equal to 'optimizer_config_error'. It updates after the active clock edge together with other nonblocking assignments. The important operands are optimizer_config_error: the value named 'optimizer config error' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1147: `state <= ST_OUTPUT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_DRAIN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DRAIN: the value named 'st output drain' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1148: `end else output_word_index <= output_word_index + 1'b1;` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1149: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OPTIMIZER_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1150: `ST_TRAIN_START: if (train_start_ready) state <= ST_TRAIN_RUN;` This is the ST_TRAIN_START case branch, which performs this explicitly named phase. The same-cycle action begins with 'if (train_start_ready) state <= ST_TRAIN_RUN;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_START, the machine performs the named control phase..

Line 1151: `ST_TRAIN_RUN: if (train_done) begin` This is the ST_TRAIN_RUN case branch, which performs this explicitly named phase. The same-cycle action begins with 'if (train_done) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 1152: `if (command_had_output_r || train_result_accept) begin` This condition selects the following hardware branch only when 'command_had_output_r || train_result_accept' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are command_had_output_r: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; train_result_accept: the value named 'train result accept' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 1153: `completion_error_r <= train_config_error;` This makes completion_error_r, the value named 'completion error r' in this block, equal to 'train_config_error'. It updates after the active clock edge together with other nonblocking assignments. The important operands are train_config_error: the value named 'train config error' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 1154: `state <= ST_OUTPUT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_OUTPUT_DRAIN'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DRAIN: the value named 'st output drain' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 1155: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1156: `if (train_config_error) error_event_r <= 1'b1;` This condition selects the following hardware branch only when 'train_config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are train_config_error: the value named 'train config error' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 1157: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 1158: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase..

Line 1159: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1160: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_TRAIN_RUN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1161: `ST_LUT_LOAD: begin` This is the ST_LUT_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1162: `if (lut_load_pending_r && lut_load_ready) begin` This condition selects the following hardware branch only when ‘lut_load_pending_r && lut_load_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lut_load_pending_r: the value named ’lut load pending r’ in this block; lut_load_ready: the consumer’s statement that it can accept a payload this cycle. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1163: `lut_load_pending_r <= 1’b0;` This makes lut_load_pending_r, the value named ’lut load pending r’ in this block, equal to ‘1’b0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named ’b0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1164: `if (lut_load_last_r != lut_load_final_r) begin` This condition selects the following hardware branch only when ‘lut_load_last_r != lut_load_final_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lut_load_last_r: the AXI-Stream marker for the final beat of a packet or command payload; lut_load_final_r: the value named ’lut load final r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1165: `error_event_r <= 1’b1;` This makes error_event_r, the value named ’error event r’ in this block, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1166: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1167: `if (lut_load_final_r) begin` This condition selects the following hardware branch only when ‘lut_load_final_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lut_load_final_r: the value named ’lut load final r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1168: `command_done_pending_r <= 1’b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the

clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1169: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1170: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1171: `stream_item_index <= stream_item_index + 1'b1;` This makes stream_item_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘stream_item_index + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_item_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1172: `stream_items_remaining_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are stream_items_remaining_r: the value named ’stream items remaining r’ in this block.

Line 1173: `<= stream_items_remaining_r - 1'b1;` Within the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are stream_items_remaining_r: the value named ’stream items remaining r’ in this block; b1: the value named ’b1’ in this block.

Line 1174: `stream_last_pending_r` Within the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are stream_last_pending_r: the AXI-Stream marker for the final beat of a packet or command payload.

Line 1175: `<= stream_items_remaining_r == 12'd2;` Within the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory., this exact statement contributes to the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path. The important operands are stream_items_remaining_r: the value named ’stream items remaining r’ in this block; d2: the value named ’d2’ in this block.

Line 1176: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1177: `end else if (!lut_load_pending_r && s_axis_tvalid) begin` This condition selects the following hardware branch only when ‘!lut_load_pending_r && s_axis_tvalid‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lut_load_pending_r: the value named ’lut load pending r’ in this block; s_axis_tvalid: the producer’s statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1178: `lut_load_pending_r <= 1'b1;` This makes lut_load_pending_r, the value named 'lut load pending r' in this block, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1179: `lut_load_address_r <= s_axis_tdata[7:0];` This makes lut_load_address_r, the value named 'lut load address r' in this block, equal to 's_axis_tdata[7:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1180: `lut_load_value_r <= s_axis_tdata[23:8];` This makes lut_load_value_r, the value named 'lut load value r' in this block, equal to 's_axis_tdata[23:8]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1181: `lut_load_last_r <= s_axis_tlast;` This makes lut_load_last_r, the AXI-Stream marker for the final beat of a packet or command payload, equal to 's_axis_tlast'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1182: `lut_load_final_r <= stream_last_pending_r;` This makes lut_load_final_r, the value named 'lut load final r' in this block, equal to 'stream_last_pending_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are stream_last_pending_r: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1183: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1184: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1185: `ST_LUT_EVAL_LOAD: if (s_axis_tvalid) begin` This is the ST_LUT_EVAL_LOAD case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'if (s_axis_tvalid) begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1186: `lut_indices_r <= s_axis_tdata;` This makes lut_indices_r, the value named 'lut indices r' in this block, equal to 's_axis_tdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are s_axis_tdata: the value named 's axis tdata' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1187: `if (!s_axis_tlast) error_event_r <= 1'b1;` This condition selects the following hardware branch only when '!s_axis_tlast' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1188: `state <= ST_LUT_EVAL_START;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LUT_EVAL_START‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LUT_EVAL_START: the value named ’st lut eval start’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_LOAD, the machine stores incoming operands into local FPGA memory..

Line 1189: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_LOAD, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 1190: `ST_LUT_EVAL_START: if (lut_input_ready) begin` This is the ST_LUT_EVAL_START case branch, which performs this explicitly named phase. The same-cycle action begins with ‘if (lut_input_ready) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_START, the machine performs the named control phase..

Line 1191: `output_word_index <= 4'd0;` This makes output_word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘4’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named ’d0’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_START, the machine performs the named control phase..

Line 1192: `state <= ST_LUT_EVAL_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_LUT_EVAL_OUTPUT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LUT_EVAL_OUTPUT: the value named ’st lut eval output’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_START, the machine performs the named control phase..

Line 1193: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_START, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1194: `ST_LUT_EVAL_OUTPUT: begin` This is the ST_LUT_EVAL_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1195: `if (lut_result_accept && output_word_index == 1) begin` This condition selects the following hardware branch only when ‘lut_result_accept && output_word_index == 1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lut_result_accept: the value named ’lut result accept’ in this block; output_word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1196: `completion_error_r <= lut_config_error;` This makes completion_error_r, the value named ’completion error r’ in this block, equal to ‘lut_config_error‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are lut_config_error: the value named ’lut config error’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1197: `state <= ST_OUTPUT_DRAIN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT_DRAIN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT_DRAIN: the value named ’st output drain’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1198: `end else if (lut_result_accept) begin` This condition selects the following hardware branch only when ‘lut_result_accept‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lut_result_accept: the value named ’lut result accept’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1199: `output_word_index <= output_word_index + 1'b1;` This makes output_word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_word_index + 1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_word_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 1200: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1201: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_EVAL_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1202: `ST_OUTPUT_DRAIN: if (final_output_accepted_r) begin` This is the ST_OUTPUT_DRAIN case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘if (final_output_accepted_r) begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DRAIN, the machine presents computed results under valid/ready backpressure..

Line 1203: `if (completion_error_r) error_event_r <= 1'b1;` This condition selects the following hardware branch only when ‘completion_error_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are completion_error_r: the value named ’completion error r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DRAIN, the machine presents computed results under valid/ready backpressure..

Line 1204: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DRAIN, the machine presents computed results under valid/ready backpressure..

Line 1205: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named ’st idle’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DRAIN, the machine presents computed results under valid/ready backpressure..

Line 1206: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT_DRAIN, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 1207: `ST_LUT_CLEAR: begin` This is the ST_LUT_CLEAR case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase..

Line 1208: `if (decode_error_pending_r) error_event_r <= 1'b1;` This condition selects the following hardware branch only when ‘decode_error_pending_r‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are decode_error_pending_r: the value named ’decode

error pending r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase..

Line 1209: `lut_clear_requested_r <= 1'b0;` This makes lut_clear_requested_r, the value named 'lut clear requested r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase..

Line 1210: `decode_error_pending_r <= 1'b0;` This makes decode_error_pending_r, the value named 'decode error pending r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase..

Line 1211: `command_done_pending_r <= 1'b1;` This makes command_done_pending_r, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase..

Line 1212: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase..

Line 1213: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1214: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'state <= ST_IDLE;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase..

Line 1215: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LUT_CLEAR, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 1216: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 1217: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 1218: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 1220: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path.

22 rtl/neuraedge_vector_alu.sv

Role in the complete system: executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

22.1 Beginner architecture map

Dataflow responsibility Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_vector_alu into physical nets and primitives. The source declares 15 input-port lines, 4 output-port lines, 1 clocked process(es), and 1 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control The named control phases include ST_ABSOLUTE, ST_COMPARE, ST_DISTANCE_ABSOLUTE, ST_IDLE, ST_LOAD_LANE, ST_MULTIPLIER_LATCH, ST_MULTIPLY, ST_OFFSET, ST_OFFSET_ADD, ST_OFFSET_EXTEND, ST_OUTPUT, ST_PRECOMPUTE, ST_PRODUCT_CROSS, ST_PRODUCT_LATCH, ST_PRODUCT_LOW, ST_RESTORE_SIGN, ST_RESULT, ST_ROUND_BIAS and additional states. A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update.

Connections to other blocks This source instantiates else as if, unique as case. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

22.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module neuraedge_vector_alu #(
5      parameter int LANES = 8,
6      parameter int DATA_BITS = 32
7  ) (
8      input  wire clk,
9      input  wire rst_n,
10     input  wire input_valid,
11     output logic input_ready,
12     input  wire [3:0] operation,
13     input  wire [LANES*DATA_BITS-1:0] operand_a,
14     input  wire [LANES*DATA_BITS-1:0] operand_b,
15     input  wire signed [15:0] multiplier_q15,
16     input  wire [4:0] right_shift,
17     input  wire signed [15:0] output_zero_point,
18     input  wire signed [DATA_BITS-1:0] threshold,
19     input  wire [DATA_BITS-1:0] surrogate_window,
20     input  wire signed [DATA_BITS-1:0] clamp_minimum,
21     input  wire signed [DATA_BITS-1:0] clamp_maximum,
22     output logic output_valid,
23     input  wire output_ready,
24     output logic [LANES*DATA_BITS-1:0] result,
25     output logic config_error
26 );
27     localparam logic [3:0] OP_ADD_SAT = 4'd0;
28     localparam logic [3:0] OP_RELU = 4'd1;
29     localparam logic [3:0] OP_REQUANTIZE = 4'd2;
30     localparam logic [3:0] OP_BINARY_SPIKE = 4'd3;
31     localparam logic [3:0] OP_TERNARY_SPIKE = 4'd4;
32     localparam logic [3:0] OP_SURROGATE_GRADIENT = 4'd5;
33     localparam logic [3:0] OP_MAXIMUM = 4'd6;
34     localparam logic [3:0] OP_MULTIPLY_Q15 = 4'd7;
35     localparam logic [3:0] OP_CLAMP = 4'd8;
36     localparam logic [3:0] OP_FLEX_REQUANTIZE = 4'd9;
37     localparam logic [3:0] OP_SELECT_ZERO = 4'd10;
38     localparam logic [3:0] OP_SELECT_NONZERO = 4'd11;
39     localparam logic [3:0] OP_COMPARE_GE = 4'd12;
40
```

```
    // One arithmetic transformation is performed per state.  Besides making the
    // latency deterministic, these registers prevent a multiply, absolute value,
    // rounding add, barrel shift, sign restore, and clamp from becoming one path.
    typedef enum logic [4:0] {
        ST_IDLE = 5'd0,
        ST_LOAD_LANE = 5'd1,
        ST_MULTIPLY = 5'd2,
        ST_ABSOLUTE = 5'd3,
        ST_ROUND_BIAS = 5'd4,
        ST_SHIFT = 5'd5,
        ST_RESTORE_SIGN = 5'd6,
        ST_OFFSET = 5'd7,
        ST_PRECOMPUTE = 5'd8,
        ST_COMPARE = 5'd9,
        ST_RESULT = 5'd10,
        ST_OUTPUT = 5'd11,
        ST_MULTIPLIER_LATCH = 5'd12,
        ST_PRODUCT_LATCH = 5'd13,
        ST_PRODUCT_CROSS = 5'd14,
        ST_PRODUCT_LOW = 5'd15,
        ST_OFFSET_ADD = 5'd16,
        ST_OFFSET_EXTEND = 5'd17,
        ST_DISTANCE_ABSOLUTE = 5'd18,
        ST_STORE_RESULT = 5'd19
    } state_t;

    state_t state;
    logic [2:0] lane_index;
    logic [3:0] operation_r;
    logic [LANES*DATA_BITS-1:0] operand_a_r;
    logic [LANES*DATA_BITS-1:0] operand_b_r;
    logic signed [15:0] multiplier_q15_r;
    logic signed [15:0] output_zero_point_r;
    logic signed [31:0] threshold_r;
    logic signed [31:0] negative_threshold_r;
    logic [31:0] surrogate_window_r;
    logic signed [31:0] clamp_minimum_r;
    logic signed [31:0] clamp_maximum_r;
    logic [5:0] shift_amount_r;
    logic [63:0] rounding_bias_r;

    logic signed [31:0] lane_a_r;
    logic signed [31:0] lane_b_r;
    logic signed [31:0] multiplier_left_r;
    logic signed [31:0] multiplier_right_r;
    logic signed [16:0] multiplier_left_low_r;
    logic signed [16:0] multiplier_right_low_r;
    logic signed [15:0] multiplier_left_high_r;
    logic signed [15:0] multiplier_right_high_r;
    (* use_dsp = "yes" *) wire signed [33:0] multiplier_product_ll =
        multiplier_left_low_r * multiplier_right_low_r;
    (* use_dsp = "yes" *) wire signed [32:0] multiplier_product_lh =
        multiplier_left_low_r * multiplier_right_high_r;
    (* use_dsp = "yes" *) wire signed [32:0] multiplier_product_hl =
        multiplier_left_high_r * multiplier_right_low_r;
    (* use_dsp = "yes" *) wire signed [31:0] multiplier_product_hh =
        multiplier_left_high_r * multiplier_right_high_r;
    logic signed [33:0] product_ll_r;
    logic signed [32:0] product_lh_r;
    logic signed [32:0] product_hl_r;
    logic signed [31:0] product_hh_r;
    logic signed [33:0] product_cross_r;
    logic signed [63:0] product_low_r;
    logic signed [63:0] product_latched_r;
    logic product_negative_r;
    logic [63:0] magnitude_r;
    logic [63:0] biased_magnitude_r;
    logic [63:0] shifted_magnitude_r;
    logic signed [63:0] rounded_signed_r;
    logic signed [32:0] offset_left_r;
    logic signed [32:0] offset_right_r;
    logic signed [32:0] offset_sum_r;
    logic signed [63:0] wide_result_r;
    logic signed [31:0] saturated_value_r;
    logic [31:0] distance_r;
    logic signed [32:0] distance_difference_r;
    logic lane_a_negative_r;
    logic lane_a_ge_threshold_r;
    logic lane_a_le_negative_threshold_r;
    logic lane_a_ge_b_r;
    logic lane_a_zero_r;
    logic lane_a_below_clamp_r;
    logic lane_a_above_clamp_r;
    logic saturated_below_clamp_r;
    logic saturated_above_clamp_r;
    logic inside_surrogate_window_r;
    logic signed [31:0] lane_result_r;
```

```
128
129     function automatic logic signed [31:0] saturate_s32(
130         input logic signed [63:0] value
131     );
132         begin
133             // A signed 64-bit number fits in 32 bits exactly when every upper bit
134             // repeats bit 31.  This avoids a long pair of 64-bit comparators.
135             if (value[63:32] == {32{value[31]}}) saturate_s32 = value[31:0];
136             else if (value[63]) saturate_s32 = 32'sh80000000;
137             else saturate_s32 = 32'sh7fffffff;
138         end
139     endfunction
140
141     always_comb begin
142         input_ready = state == ST_IDLE;
143         output_valid = state == ST_OUTPUT;
144     end
145
146     always_ff @(posedge clk) begin
147         if (!rst_n) begin
148             state <= ST_IDLE;
149             lane_index <= 3'd0;
150             operation_r <= OP_ADD_SAT;
151             operand_a_r <= '0;
152             operand_b_r <= '0;
153             multiplier_q15_r <= 16'sd0;
154             output_zero_point_r <= 16'sd0;
155             threshold_r <= 32'sd0;
156             negative_threshold_r <= 32'sd0;
157             surrogate_window_r <= 32'd0;
158             clamp_minimum_r <= 32'sh80000000;
159             clamp_maximum_r <= 32'sh7fffffff;
160             shift_amount_r <= 6'd0;
161             rounding_bias_r <= 64'd0;
162             lane_a_r <= 32'sd0;
163             lane_b_r <= 32'sd0;
164             multiplier_left_r <= 32'sd0;
165             multiplier_right_r <= 32'sd0;
166             multiplier_left_low_r <= 17'sd0;
167             multiplier_right_low_r <= 17'sd0;
168             multiplier_left_high_r <= 16'sd0;
169             multiplier_right_high_r <= 16'sd0;
170             product_ll_r <= 34'sd0;
171             product_lh_r <= 33'sd0;
172             product_hl_r <= 33'sd0;
173             product_hh_r <= 32'sd0;
174             product_cross_r <= 34'sd0;
175             product_low_r <= 64'sd0;
176             product_latched_r <= 64'sd0;
177             product_negative_r <= 1'b0;
178             magnitude_r <= 64'd0;
179             biased_magnitude_r <= 64'd0;
180             shifted_magnitude_r <= 64'd0;
181             rounded_signed_r <= 64'sd0;
182             offset_left_r <= 33'sd0;
183             offset_right_r <= 33'sd0;
184             offset_sum_r <= 33'sd0;
185             wide_result_r <= 64'sd0;
186             saturated_value_r <= 32'sd0;
187             distance_r <= 32'd0;
188             distance_difference_r <= 33'sd0;
189             lane_a_negative_r <= 1'b0;
190             lane_a_ge_threshold_r <= 1'b0;
191             lane_a_le_negative_threshold_r <= 1'b0;
192             lane_a_ge_b_r <= 1'b0;
193             lane_a_zero_r <= 1'b0;
194             lane_a_below_clamp_r <= 1'b0;
195             lane_a_above_clamp_r <= 1'b0;
196             saturated_below_clamp_r <= 1'b0;
197             saturated_above_clamp_r <= 1'b0;
198             inside_surrogate_window_r <= 1'b0;
199             lane_result_r <= 32'sd0;
200             result <= '0;
201             config_error <= 1'b0;
202         end else begin
203             unique case (state)
204                 ST_IDLE: begin
205                     if (input_valid) begin
206                         operation_r <= operation;
207                         operand_a_r <= operand_a;
208                         operand_b_r <= operand_b;
209                         multiplier_q15_r <= multiplier_q15;
210                         output_zero_point_r <= output_zero_point;
211                         threshold_r <= threshold;
212                         negative_threshold_r <= -threshold;
213                         surrogate_window_r <= surrogate_window;
214                         clamp_minimum_r <= clamp_minimum;
```

```
                clamp_maximum_r <= clamp_maximum;
                config_error <= operation > OP_COMPARE_GE
                    || clamp_minimum > clamp_maximum
                    || (operation == OP_FLEX_REQUANTIZE
                        && right_shift == 5'b10000);
                lane_index <= 3'd0;
                if (operation == OP_REQUANTIZE) begin
                    shift_amount_r <= 6'd15 + {1'b0, right_shift};
                    rounding_bias_r <= 64'd16384 << right_shift;
                end else if (operation == OP_FLEX_REQUANTIZE) begin
                    logic signed [5:0] effective_shift;
                    effective_shift = 6'sd15
                        + $signed({right_shift[4], right_shift});
                    if (right_shift == 5'b10000) begin
                        shift_amount_r <= 6'd0;
                        rounding_bias_r <= 64'd0;
                    end else begin
                        shift_amount_r <= effective_shift[5:0];
                        rounding_bias_r <= effective_shift == 0
                            ? 64'd0
                            : (64'd1 << (effective_shift - 1'b1));
                    end
                end else if (operation == OP_MULTIPLY_Q15) begin
                    shift_amount_r <= 6'd15;
                    rounding_bias_r <= 64'd16384;
                end else begin
                    shift_amount_r <= 6'd0;
                    rounding_bias_r <= 64'd0;
                end
                state <= ST_LOAD_LANE;
            end
        end

        ST_LOAD_LANE: begin
            lane_a_r <= $signed(
                operand_a_r[lane_index*DATA_BITS +: DATA_BITS]);
            lane_b_r <= $signed(
                operand_b_r[lane_index*DATA_BITS +: DATA_BITS]);
            state <= ST_MULTIPLIER_LATCH;
        end

        ST_MULTIPLIER_LATCH: begin
            multiplier_left_r <= lane_a_r;
            if (operation_r == OP_REQUANTIZE
                    || operation_r == OP_FLEX_REQUANTIZE) begin
                multiplier_right_r <= {
                    {16{multiplier_q15_r[15]}}, multiplier_q15_r};
            end else if (operation_r == OP_MULTIPLY_Q15) begin
                multiplier_right_r <= lane_b_r;
            end else begin
                multiplier_right_r <= 32'sd0;
            end
            state <= ST_MULTIPLY;
        end

        ST_MULTIPLY: begin
            multiplier_left_low_r <= $signed({1'b0, multiplier_left_r[15:0]});
            multiplier_right_low_r <= $signed({1'b0, multiplier_right_r[15:0]});
            multiplier_left_high_r <= multiplier_left_r[31:16];
            multiplier_right_high_r <= multiplier_right_r[31:16];
            state <= ST_PRODUCT_CROSS;
        end

        ST_PRODUCT_CROSS: begin
            product_ll_r <= multiplier_product_ll;
            product_lh_r <= multiplier_product_lh;
            product_hl_r <= multiplier_product_hl;
            product_hh_r <= multiplier_product_hh;
            state <= ST_PRODUCT_LOW;
        end

        ST_PRODUCT_LOW: begin
            product_cross_r <= $signed({product_lh_r[32], product_lh_r})
                + $signed({product_hl_r[32], product_hl_r});
            product_low_r <= $signed({{30{product_ll_r[33]}}, product_ll_r});
            state <= ST_PRODUCT_LATCH;
        end

        ST_PRODUCT_LATCH: begin
            product_latched_r <= product_low_r
                + ($signed({{30{product_cross_r[33]}}, product_cross_r}) <<< 16)
                + ($signed({{32{product_hh_r[31]}}, product_hh_r}) <<< 32);
            state <= ST_ABSOLUTE;
        end

        ST_ABSOLUTE: begin
            product_negative_r <= product_latched_r[63];
```

```
                    magnitude_r <= product_latched_r[63]
                        ? -product_latched_r : product_latched_r;
                    state <= ST_ROUND_BIAS;
                end

                ST_ROUND_BIAS: begin
                    biased_magnitude_r <= magnitude_r + rounding_bias_r;
                    state <= ST_SHIFT;
                end

                ST_SHIFT: begin
                    shifted_magnitude_r <= biased_magnitude_r >> shift_amount_r;
                    state <= ST_RESTORE_SIGN;
                end

                ST_RESTORE_SIGN: begin
                    rounded_signed_r <= product_negative_r
                        ? -$signed(shifted_magnitude_r)
                        : $signed(shifted_magnitude_r);
                    state <= ST_OFFSET;
                end

                ST_OFFSET: begin
                    unique case (operation_r)
                        OP_ADD_SAT: begin
                            offset_left_r <= {lane_a_r[31], lane_a_r};
                            offset_right_r <= {lane_b_r[31], lane_b_r};
                            state <= ST_OFFSET_ADD;
                        end
                        OP_REQUANTIZE,
                        OP_FLEX_REQUANTIZE: begin
                            offset_left_r <= rounded_signed_r[32:0];
                            offset_right_r <= {{17{output_zero_point_r[15]}},
                                               output_zero_point_r};
                            state <= ST_OFFSET_ADD;
                        end
                        OP_MULTIPLY_Q15: begin
                            wide_result_r <= rounded_signed_r;
                            state <= ST_PRECOMPUTE;
                        end
                        default: begin
                            wide_result_r <= {{32{lane_a_r[31]}}, lane_a_r};
                            state <= ST_PRECOMPUTE;
                        end
                    endcase
                end

                ST_OFFSET_ADD: begin
                    offset_sum_r <= offset_left_r + offset_right_r;
                    state <= ST_OFFSET_EXTEND;
                end

                ST_OFFSET_EXTEND: begin
                    wide_result_r <= {{31{offset_sum_r[32]}}, offset_sum_r};
                    state <= ST_PRECOMPUTE;
                end

                ST_PRECOMPUTE: begin
                    saturated_value_r <= saturate_s32(wide_result_r);
                    distance_difference_r <= {lane_a_r[31], lane_a_r}
                        - {threshold_r[31], threshold_r};
                    lane_a_negative_r <= lane_a_r < 0;
                    lane_a_ge_threshold_r <= lane_a_r >= threshold_r;
                    lane_a_le_negative_threshold_r <= lane_a_r <= negative_threshold_r;
                    lane_a_ge_b_r <= lane_a_r >= lane_b_r;
                    lane_a_zero_r <= lane_a_r == 0;
                    state <= ST_DISTANCE_ABSOLUTE;
                end

                ST_DISTANCE_ABSOLUTE: begin
                    distance_r <= distance_difference_r[32]
                        ? (~distance_difference_r[31:0] + 1'b1)
                        : distance_difference_r[31:0];
                    state <= ST_COMPARE;
                end

                ST_COMPARE: begin
                    inside_surrogate_window_r <= distance_r <= surrogate_window_r;
                    lane_a_below_clamp_r <= lane_a_r < clamp_minimum_r;
                    lane_a_above_clamp_r <= lane_a_r > clamp_maximum_r;
                    saturated_below_clamp_r <= saturated_value_r < clamp_minimum_r;
                    saturated_above_clamp_r <= saturated_value_r > clamp_maximum_r;
                    state <= ST_RESULT;
                end

                ST_RESULT: begin
                    logic signed [31:0] lane_result;
```

```
                    lane_result = 32'sd0;

                    unique case (operation_r)
                        OP_ADD_SAT: lane_result = saturated_value_r;
                        OP_RELU: lane_result = lane_a_negative_r ? 32'sd0 : lane_a_r;
                        OP_REQUANTIZE,
                        OP_FLEX_REQUANTIZE: begin
                            if (saturated_below_clamp_r) begin
                                lane_result = clamp_minimum_r;
                            end else if (saturated_above_clamp_r) begin
                                lane_result = clamp_maximum_r;
                            end else begin
                                lane_result = saturated_value_r;
                            end
                        end
                        OP_BINARY_SPIKE: lane_result = lane_a_ge_threshold_r
                            ? 32'sd1 : 32'sd0;
                        OP_TERNARY_SPIKE: begin
                            if (lane_a_ge_threshold_r) lane_result = 32'sd1;
                            else if (lane_a_le_negative_threshold_r) lane_result = -32'sd1;
                            else lane_result = 32'sd0;
                        end
                        OP_SURROGATE_GRADIENT: lane_result =
                            inside_surrogate_window_r ? lane_b_r : 32'sd0;
                        OP_MAXIMUM: lane_result = lane_a_ge_b_r
                            ? lane_a_r : lane_b_r;
                        OP_MULTIPLY_Q15: lane_result = saturated_value_r;
                        OP_CLAMP: begin
                            if (lane_a_below_clamp_r) lane_result = clamp_minimum_r;
                            else if (lane_a_above_clamp_r) lane_result = clamp_maximum_r;
                            else lane_result = lane_a_r;
                        end
                        OP_SELECT_ZERO: lane_result = lane_a_zero_r
                            ? lane_b_r : 32'sd0;
                        OP_SELECT_NONZERO: lane_result = lane_a_zero_r
                            ? 32'sd0 : lane_b_r;
                        OP_COMPARE_GE: lane_result = lane_a_ge_b_r
                            ? 32'sd1 : 32'sd0;
                        default: lane_result = 32'sd0;
                    endcase
                    // Keep the operation-select mux local to the scalar lane
                    // pipeline. The following state moves this registered value
                    // into the wide retained output vector, avoiding a routed
                    // comparison-to-distant-result path across the device.
                    lane_result_r <= lane_result;
                    state <= ST_STORE_RESULT;
                end

                ST_STORE_RESULT: begin
                    result[lane_index*DATA_BITS +: DATA_BITS] <= lane_result_r;
                    if (lane_index == $clog2(LANES)'(LANES - 1)) begin
                        state <= ST_OUTPUT;
                    end else begin
                        lane_index <= lane_index + 1'b1;
                        state <= ST_LOAD_LANE;
                    end
                end

                ST_OUTPUT: if (output_ready) state <= ST_IDLE;
                default: state <= ST_IDLE;
            endcase
        end
    end

    initial begin
        if (LANES != 8) $fatal(1, "neuraedge_vector_alu currently requires LANES=8");
        if (DATA_BITS != 32) $fatal(1, "neuraedge_vector_alu currently requires DATA_BITS=32");
    end
endmodule

`default_nettype wire
```

22.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_vector_alu #(` This opens neuraedge_vector_alu, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 5: `parameter int LANES = 8,` This statement continues through source line 26; the explanation uses the complete expression rather than only the first visual fragment. This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 6: `parameter int DATA_BITS = 32` This is continuation line 2 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'parameter int DATA_BITS = 32'. The important operands are parameter: the value named 'parameter' in this block; DATA_BITS: the value named 'data bits' in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 7: `) (` This is continuation line 3 of the 22-line logical statement that begins at source line 5. Its exact fragment is ') ('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 8: `input wire clk,` This is continuation line 4 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire clk,'. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 9: `input wire rst_n,` This is continuation line 5 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire rst_n,'. The important operands are input: the value named 'input' in this block; rst_n: the active-low synchronous reset. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire

output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 10: `input wire input_valid,` This is continuation line 6 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire input_valid,'. The important operands are input: the value named 'input' in this block; input_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 11: `output logic input_ready,` This is continuation line 7 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'output logic input_ready,'. The important operands are output: the value named 'output' in this block; input_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 12: `input wire [3:0] operation,` This is continuation line 8 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire [3:0] operation,'. The important operands are input: the value named 'input' in this block; operation: the value named 'operation' in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 13: `input wire [LANES*DATA_BITS-1:0] operand_a,` This is continuation line 9 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire [LANES*DATA_BITS-1:0] operand_a,'. The important operands are input: the value named 'input' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; DATA_BITS: the value named 'data bits' in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 14: `input wire [LANES*DATA_BITS-1:0] operand_b,` This is continuation line 10 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire [LANES*DATA_BITS-1:0] operand_b,'. The important operands are input: the value named 'input' in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; DATA_BITS: the value named 'data bits' in this block. Read together, the complete

operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 15: `input wire signed [15:0] multiplier_q15,` This is continuation line 11 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [15:0] multiplier_q15,'. The important operands are input: the value named 'input' in this block; multiplier_q15: the value named 'multiplier q15' in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 16: `input wire [4:0] right_shift,` This is continuation line 12 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire [4:0] right_shift,'. The important operands are input: the value named 'input' in this block; right_shift: the value named 'right shift' in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 17: `input wire signed [15:0] output_zero_point,` This is continuation line 13 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [15:0] output_zero_point,'. The important operands are input: the value named 'input' in this block; output_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 18: `input wire signed [DATA_BITS-1:0] threshold,` This is continuation line 14 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [DATA_BITS-1:0] threshold,'. The important operands are input: the value named 'input' in this block; DATA_BITS: the value named 'data bits' in this block; threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result,

output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 19: `input wire [DATA_BITS-1:0] surrogate_window,` This is continuation line 15 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire [DATA_BITS-1:0] surrogate_window,'. The important operands are input: the value named 'input' in this block; DATA_BITS: the value named 'data bits' in this block; surrogate_window: the value named 'surrogate window' in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 20: `input wire signed [DATA_BITS-1:0] clamp_minimum,` This is continuation line 16 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [DATA_BITS-1:0] clamp_minimum,'. The important operands are input: the value named 'input' in this block; DATA_BITS: the value named 'data bits' in this block; clamp_minimum: the value named 'clamp minimum' in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 21: `input wire signed [DATA_BITS-1:0] clamp_maximum,` This is continuation line 17 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire signed [DATA_BITS-1:0] clamp_maximum,'. The important operands are input: the value named 'input' in this block; DATA_BITS: the value named 'data bits' in this block; clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 22: `output logic output_valid,` This is continuation line 18 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'output logic output_valid,'. The important operands are output: the value named 'output' in this block; output_valid: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 23: `input wire output_ready,` This is continuation line 19 of the 22-line logical statement that begins at source line 5. Its exact fragment is 'input wire output_ready,'. The important operands are input: the value named 'input' in this block; output_ready: the consumer's statement that it can accept a payload this cycle. Read together, the complete

operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 24: `output logic [LANES*DATA_BITS-1:0] result,` This is continuation line 20 of the 22-line logical statement that begins at source line 5. Its exact fragment is ‘output logic [LANES*DATA_BITS-1:0] result,‘. The important operands are output: the value named ’output’ in this block; LANES: one parallel arithmetic element within a vector or tensor datapath; DATA_BITS: the value named ’data bits’ in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 25: `output logic config_error` This is continuation line 21 of the 22-line logical statement that begins at source line 5. Its exact fragment is ‘output logic config_error‘. The important operands are output: the value named ’output’ in this block; config_error: the value named ’config error’ in this block. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 26: `);` This is continuation line 22 of the 22-line logical statement that begins at source line 5. Its exact fragment is ‘);‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes LANES to 8, parameter int DATA_BITS = 32) (input wire clk, input wire rst_n, input wire input_valid, output logic input_ready, input wire [3:0] operation, input wire [LANES*DATA_BITS-1:0] operand_a, input wire [LANES*DATA_BITS-1:0] operand_b, input wire signed [15:0] multiplier_q15, input wire [4:0] right_shift, input wire signed [15:0] output_zero_point, input wire signed [DATA_BITS-1:0] threshold, input wire [DATA_BITS-1:0] surrogate_window, input wire signed [DATA_BITS-1:0] clamp_minimum, input wire signed [DATA_BITS-1:0] clamp_maximum, output logic output_valid, input wire output_ready, output logic [LANES*DATA_BITS-1:0] result, output logic config_error) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 27: `localparam logic [3:0] OP_ADD_SAT = 4'd0;` This fixes OP_ADD_SAT to 4’d0 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_ADD_SAT denotes the value named ’op add sat’ in this block.

Line 28: `localparam logic [3:0] OP_RELU = 4'd1;` This fixes OP_RELU to 4’d1 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_RELU denotes the value named ’op relu’ in this block.

Line 29: `localparam logic [3:0] OP_REQUANTIZE = 4'd2;` This fixes OP_REQUANTIZE to 4’d2 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_REQUANTIZE denotes the conversion of a wide accumulated result back into a bounded activation code.

Line 30: `localparam logic [3:0] OP_BINARY_SPIKE = 4'd3;` This fixes OP_BINARY_SPIKE to 4'd3 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_BINARY_SPIKE denotes a binary event produced when a neuron crosses its threshold.

Line 31: `localparam logic [3:0] OP_TERNARY_SPIKE = 4'd4;` This fixes OP_TERNARY_SPIKE to 4'd4 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_TERNARY_SPIKE denotes a binary event produced when a neuron crosses its threshold.

Line 32: `localparam logic [3:0] OP_SURROGATE_GRADIENT = 4'd5;` This fixes OP_SURROGATE_GRADIENT to 4'd5 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_SURROGATE_GRADIENT denotes the signed loss derivative used to update a learned parameter.

Line 33: `localparam logic [3:0] OP_MAXIMUM = 4'd6;` This fixes OP_MAXIMUM to 4'd6 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_MAXIMUM denotes the value named 'op maximum' in this block.

Line 34: `localparam logic [3:0] OP_MULTIPLY_Q15 = 4'd7;` This fixes OP_MULTIPLY_Q15 to 4'd7 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_MULTIPLY_Q15 denotes the value named 'op multiply q15' in this block.

Line 35: `localparam logic [3:0] OP_CLAMP = 4'd8;` This fixes OP_CLAMP to 4'd8 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_CLAMP denotes the value named 'op clamp' in this block.

Line 36: `localparam logic [3:0] OP_FLEX_REQUANTIZE = 4'd9;` This fixes OP_FLEX_REQUANTIZE to 4'd9 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_FLEX_REQUANTIZE denotes the conversion of a wide accumulated result back into a bounded activation code.

Line 37: `localparam logic [3:0] OP_SELECT_ZERO = 4'd10;` This fixes OP_SELECT_ZERO to 4'd10 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_SELECT_ZERO denotes the value named 'op select zero' in this block.

Line 38: `localparam logic [3:0] OP_SELECT_NONZERO = 4'd11;` This fixes OP_SELECT_NONZERO to 4'd11 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_SELECT_NONZERO denotes the value named 'op select nonzero' in this block.

Line 39: `localparam logic [3:0] OP_COMPARE_GE = 4'd12;` This fixes OP_COMPARE_GE to 4'd12 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; OP_COMPARE_GE denotes the value named 'op compare ge' in this block.

Line 41: `// One arithmetic transformation is performed per state. Besides making the` Design intent: One arithmetic transformation is performed per state. Besides making the This comment anchors the surrounding logic in the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 42: `// latency deterministic, these registers prevent a multiply, absolute value,` Design intent: latency deterministic, these registers prevent a multiply, absolute value, This comment anchors the surrounding logic in the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 43: `// rounding add, barrel shift, sign restore, and clamp from becoming one path.` Design intent: rounding add, barrel shift, sign restore, and clamp from becoming one path. This comment anchors the surrounding logic in the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 44: `typedef enum logic [4:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 45: `ST_IDLE = 5'd0,` This assigns a fixed binary encoding to state ST_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 46: `ST_LOAD_LANE = 5'd1,` This assigns a fixed binary encoding to state ST_LOAD_LANE, which stores incoming operands into local FPGA memory. The encoding is visible in waveforms even though the source uses the readable name.

Line 47: `ST_MULTIPLY = 5'd2,` This assigns a fixed binary encoding to state ST_MULTIPLY, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 48: `ST_ABSOLUTE = 5'd3,` This assigns a fixed binary encoding to state ST_ABSOLUTE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 49: `ST_ROUND_BIAS = 5'd4,` This assigns a fixed binary encoding to state ST_ROUND_BIAS, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 50: `ST_SHIFT = 5'd5,` This assigns a fixed binary encoding to state ST_SHIFT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 51: `ST_RESTORE_SIGN = 5'd6,` This assigns a fixed binary encoding to state ST_RESTORE_SIGN, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 52: `ST_OFFSET = 5'd7,` This assigns a fixed binary encoding to state ST_OFFSET, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 53: `ST_PRECOMPUTE = 5'd8,` This assigns a fixed binary encoding to state ST_PRECOMPUTE, which issues arithmetic work using values already held on chip. The encoding is visible in waveforms even though the source uses the readable name.

Line 54: `ST_COMPARE = 5'd9,` This assigns a fixed binary encoding to state ST_COMPARE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 55: `ST_RESULT = 5'd10,` This assigns a fixed binary encoding to state ST_RESULT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 56: `ST_OUTPUT = 5'd11,` This assigns a fixed binary encoding to state ST_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 57: `ST_MULTIPLIER_LATCH = 5'd12,` This assigns a fixed binary encoding to state ST_MULTIPLIER_LATCH, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 58: `ST_PRODUCT_LATCH = 5'd13,` This assigns a fixed binary encoding to state ST_PRODUCT_LATCH, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 59: `ST_PRODUCT_CROSS = 5'd14,` This assigns a fixed binary encoding to state ST_PRODUCT_CROSS, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 60: `ST_PRODUCT_LOW = 5'd15,` This assigns a fixed binary encoding to state ST_PRODUCT_LOW, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 61: `ST_OFFSET_ADD = 5'd16,` This assigns a fixed binary encoding to state ST_OFFSET_ADD, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 62: `ST_OFFSET_EXTEND = 5'd17,` This assigns a fixed binary encoding to state ST_OFFSET_EXTEND, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 63: `ST_DISTANCE_ABSOLUTE = 5'd18,` This assigns a fixed binary encoding to state ST_DISTANCE_ABSOLUTE, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 64: `ST_STORE_RESULT = 5'd19` This statement continues through source line 65; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state ST_STORE_RESULT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 65: `} state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 64. Its exact fragment is '} state_t;'. The important operands are state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state ST_STORE_RESULT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 67: `state_t state;` Within the module's structural declarations, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are state_t: the finite-state-machine phase retained in flip-flops; state: the finite-state-machine phase retained in flip-flops.

Line 68: `logic [2:0] lane_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 69: `logic [3:0] operation_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, operation_r is the value named 'operation r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 70: `logic [LANES*DATA_BITS-1:0] operand_a_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, operand_a_r is the value named 'operand a r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 71: `logic [LANES*DATA_BITS-1:0] operand_b_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, operand_b_r is the value named 'operand b r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 72: `logic signed [15:0] multiplier_q15_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_q15_r is the value named 'multiplier q15 r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 73: `logic signed [15:0] output_zero_point_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_zero_point_r is the integer code representing real numeric zero in affine quantization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 74: `logic signed [31:0] threshold_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, threshold_r is the membrane level at which a neuron emits a spike. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 75: `logic signed [31:0] negative_threshold_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, negative_threshold_r is the membrane level at which a neuron emits a spike. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 76: `logic [31:0] surrogate_window_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, surrogate_window_r is the value named 'surrogate window r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 77: `logic signed [31:0] clamp_minimum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clamp_minimum_r is the value named 'clamp minimum r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 78: `logic signed [31:0] clamp_maximum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clamp_maximum_r is the value named 'clamp maximum r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 79: `logic [5:0] shift_amount_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, shift_amount_r is the value named 'shift amount r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 80: `logic [63:0] rounding_bias_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rounding_bias_r is an additive learned offset applied after accumulation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 82: `logic signed [31:0] lane_a_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_a_r is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 83: `logic signed [31:0] lane_b_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_b_r is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 84: `logic signed [31:0] multiplier_left_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_left_r is the value named 'multiplier left r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 85: `logic signed [31:0] multiplier_right_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_right_r is the value named 'multiplier right r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 86: `logic signed [16:0] multiplier_left_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_left_low_r is the value named 'multiplier left low r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 87: `logic signed [16:0] multiplier_right_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_right_low_r is the value named 'multiplier right low r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 88: `logic signed [15:0] multiplier_left_high_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_left_high_r is the value named 'multiplier left high r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 89: `logic signed [15:0] multiplier_right_high_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_right_high_r is the value named 'multiplier right high r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 90: `(* use_dsp = "yes" *) wire signed [33:0] multiplier_product_ll =` Within the module's structural declarations, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; multiplier_product_ll: the value named 'multiplier product ll' in this block.

Line 91: `multiplier_left_low_r * multiplier_right_low_r;` Within the module's structural declarations, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are multiplier_left_low_r: the value named 'multiplier left low r' in this block; multiplier_right_low_r: the value named 'multiplier right low r' in this block.

Line 92: `(* use_dsp = "yes" *) wire signed [32:0] multiplier_product_lh =` Within the module's structural declarations, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; multiplier_product_lh: the value named 'multiplier product lh' in this block.

Line 93: `multiplier_left_low_r * multiplier_right_high_r;` Within the module's structural declarations, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are multiplier_left_low_r: the value named 'multiplier left low r' in this block; multiplier_right_high_r: the value named 'multiplier right high r' in this block.

Line 94: `(* use_dsp = "yes" *) wire signed [32:0] multiplier_product_hl =` Within the module's structural declarations, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; multiplier_product_hl: the value named 'multiplier product hl' in this block.

Line 95: `multiplier_left_high_r * multiplier_right_low_r;` Within the module's structural declarations, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are multiplier_left_high_r: the value named 'multiplier left high r' in this block; multiplier_right_low_r: the value named 'multiplier right low r' in this block.

Line 96: `(* use_dsp = "yes" *) wire signed [31:0] multiplier_product_hh =` Within the module's structural declarations, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are use_dsp: the value named 'use dsp' in this block; yes: the value named 'yes' in this block; multiplier_product_hh: the value named 'multiplier product hh' in this block.

Line 97: `multiplier_left_high_r * multiplier_right_high_r;` Within the module's structural declarations, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are multiplier_left_high_r: the value named 'multiplier left high r' in this block; multiplier_right_high_r: the value named 'multiplier right high r' in this block.

Line 98: `logic signed [33:0] product_ll_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_ll_r is the value named 'product ll r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 99: `logic signed [32:0] product_lh_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_lh_r is the value named 'product lh r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 100: `logic signed [32:0] product_hl_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_hl_r is the value named 'product hl r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 101: `logic signed [31:0] product_hh_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_hh_r is the value named 'product hh r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 102: `logic signed [33:0] product_cross_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_cross_r is the value named 'product cross r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 103: `logic signed [63:0] product_low_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_low_r is the value named 'product low r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 104: `logic signed [63:0] product_latched_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_latched_r is the value named 'product latched r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 105: `logic product_negative_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_negative_r is the value named 'product negative r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 106: `logic [63:0] magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, magnitude_r is the value named 'magnitude r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 107: `logic [63:0] biased_magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, biased_magnitude_r is an additive learned offset applied after accumulation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 108: `logic [63:0] shifted_magnitude_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, shifted_magnitude_r is the value named 'shifted magnitude r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 109: `logic signed [63:0] rounded_signed_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rounded_signed_r is the value named 'rounded signed r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 110: `logic signed [32:0] offset_left_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, offset_left_r is the value named 'offset left r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 111: `logic signed [32:0] offset_right_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, offset_right_r is the value named 'offset right r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 112: `logic signed [32:0] offset_sum_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, offset_sum_r is the value named 'offset sum r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 113: `logic signed [63:0] wide_result_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wide_result_r is the value named 'wide result r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 114: `logic signed [31:0] saturated_value_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, saturated_value_r is the value named 'saturated value r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 115: `logic [31:0] distance_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, distance_r is the value named 'distance r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 116: `logic signed [32:0] distance_difference_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, distance_difference_r is the value named 'distance difference r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 117: `logic lane_a_negative_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_a_negative_r is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 118: `logic lane_a_ge_threshold_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_a_ge_threshold_r is the membrane level at which a neuron emits a spike. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 119: `logic lane_a_le_negative_threshold_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_a_le_negative_threshold_r is the membrane level at which a neuron emits a spike. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 120: `logic lane_a_ge_b_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_a_ge_b_r is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 121: `logic lane_a_zero_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_a_zero_r is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 122: `logic lane_a_below_clamp_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_a_below_clamp_r is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 123: `logic lane_a_above_clamp_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_a_above_clamp_r is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 124: `logic saturated_below_clamp_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, saturated_below_clamp_r is the value named 'saturated below clamp r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 125: `logic saturated_above_clamp_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, saturated_above_clamp_r is the value named 'saturated above clamp r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 126: `logic inside_surrogate_window_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, inside_surrogate_window_r is the value named 'inside surrogate window r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 127: `logic signed [31:0] lane_result_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, lane_result_r is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 129: `function automatic logic signed [31:0] saturate_s32(` Within the combinational helper saturate s32, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_s32: the value named 'saturate s32' in this block.

Line 130: `input logic signed [63:0] value` This declares value as a input port with width [63:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 131: `);` Within the combinational helper saturate s32, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The expression is a literal or structural operator and has no hidden software state.

Line 132: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32; it has no arithmetic value of its own.

Line 133: `// A signed 64-bit number fits in 32 bits exactly when every upper bit` Design intent: A signed 64-bit number fits in 32 bits exactly when every upper bit This comment anchors the surrounding logic in the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 134: `// repeats bit 31. This avoids a long pair of 64-bit comparators.` Design intent: repeats bit 31. This avoids a long pair of 64-bit comparators. This comment anchors the surrounding logic in the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 135: `if (value[63:32] == {32{value[31]}}) saturate_s32 = value[31:0];` This condition selects the following hardware branch only when 'value[63:32] == {32{value[31]}}' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the combinational helper saturate s32.

Line 136: `else if (value[63]) saturate_s32 = 32'sh80000000;` This condition selects the following hardware branch only when 'value[63]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the combinational helper saturate s32.

Line 137: `else saturate_s32 = 32'sh7fffffff;` Within the combinational helper saturate s32, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are saturate_s32: the value named 'saturate s32' in this block; sh7fffffff: the value named 'sh7fffffff' in this block.

Line 138: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32; it has no arithmetic value of its own.

Line 139: `endfunction` Within the combinational helper saturate s32, this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are endfunction: the value named 'endfunction' in this block.

Line 141: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 142: `input_ready = state == ST_IDLE;` This makes input_ready, the consumer's statement that it can accept a payload this cycle, equal to 'state == ST_IDLE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_IDLE: the value named 'st idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 143: `output_valid = state == ST_OUTPUT;` This makes output_valid, the producer's statement that its current payload is meaningful, equal to 'state == ST_OUTPUT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are state: the finite-state-machine phase retained in flip-flops; ST_OUTPUT: the value named 'st output' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 144: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 146: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 147: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the clocked process that updates registers only on a rising clock edge.

Line 148: `state <= ST_IDLE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_IDLE: the value named 'st idle' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 149: `lane_index <= 3'd0;` This makes lane_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '3'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 150: `operation_r <= OP_ADD_SAT;` This makes operation_r, the value named 'operation r' in this block, equal to 'OP_ADD_SAT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are OP_ADD_SAT: the value named 'op add sat' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 151: `operand_a_r <= '0;` This makes operand_a_r, the value named 'operand a r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 152: `operand_b_r <= '0;` This makes operand_b_r, the value named 'operand b r' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 153: `multiplier_q15_r <= 16'sd0;` This makes multiplier_q15_r, the value named 'multiplier q15 r' in this block, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 154: `output_zero_point_r <= 16'sd0;` This makes output_zero_point_r, the integer code representing real numeric zero in affine quantization, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 155: `threshold_r <= 32'sd0;` This makes threshold_r, the membrane level at which a neuron emits a spike, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 156: `negative_threshold_r <= 32'sd0;` This makes negative_threshold_r, the membrane level at which a neuron emits a spike, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 157: `surrogate_window_r <= 32'd0;` This makes surrogate_window_r, the value named 'surrogate window r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 158: `clamp_minimum_r <= 32'sh80000000;` This makes clamp_minimum_r, the value named 'clamp minimum r' in this block, equal to '32'sh80000000'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sh80000000: the value named 'sh80000000' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 159: `clamp_maximum_r <= 32'sh7fffffff;` This makes clamp_maximum_r, the value named 'clamp maximum r' in this block, equal to '32'sh7fffffff'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sh7fffffff: the value named 'sh7fffffff' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 160: `shift_amount_r <= 6'd0;` This makes shift_amount_r, the value named 'shift amount r' in this block, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 161: `rounding_bias_r <= 64'd0;` This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 162: `lane_a_r <= 32'sd0;` This makes lane_a_r, one parallel arithmetic element within a vector or tensor datapath, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 163: `lane_b_r <= 32'sd0;` This makes lane_b_r, one parallel arithmetic element within a vector or tensor datapath, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 164: `multiplier_left_r <= 32'sd0;` This makes multiplier_left_r, the value named 'multiplier left r' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 165: `multiplier_right_r <= 32'sd0;` This makes multiplier_right_r, the value named 'multiplier right r' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 166: `multiplier_left_low_r <= 17'sd0;` This makes multiplier_left_low_r, the value named 'multiplier left low r' in this block, equal to '17'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 167: `multiplier_right_low_r <= 17'sd0;` This makes multiplier_right_low_r, the value named 'multiplier right low r' in this block, equal to '17'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 168: `multiplier_left_high_r <= 16'sd0;` This makes multiplier_left_high_r, the value named 'multiplier left high r' in this block, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 169: `multiplier_right_high_r <= 16'sd0;` This makes multiplier_right_high_r, the value named 'multiplier right high r' in this block, equal to '16'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 170: `product_ll_r <= 34'sd0;` This makes product_ll_r, the value named 'product ll r' in this block, equal to '34'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 171: `product_lh_r <= 33'sd0;` This makes product_lh_r, the value named 'product lh r' in this block, equal to '33'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 172: `product_hl_r <= 33'sd0;` This makes product_hl_r, the value named 'product hl r' in this block, equal to '33'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 173: `product_hh_r <= 32'sd0;` This makes product_hh_r, the value named 'product hh r' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 174: `product_cross_r <= 34'sd0;` This makes product_cross_r, the value named 'product cross r' in this block, equal to '34'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 175: `product_low_r <= 64'sd0;` This makes product_low_r, the value named 'product low r' in this block, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 176: `product_latched_r <= 64'sd0;` This makes product_latched_r, the value named 'product latched r' in this block, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 177: `product_negative_r <= 1'b0;` This makes product_negative_r, the value named 'product negative r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 178: `magnitude_r <= 64'd0;` This makes magnitude_r, the value named 'magnitude r' in this block, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 179: `biased_magnitude_r <= 64'd0;` This makes biased_magnitude_r, an additive learned offset applied after accumulation, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 180: `shifted_magnitude_r <= 64'd0;` This makes shifted_magnitude_r, the value named 'shifted magnitude r' in this block, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 181: `rounded_signed_r <= 64'sd0;` This makes rounded_signed_r, the value named 'rounded signed r' in this block, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 182: `offset_left_r <= 33'sd0;` This makes offset_left_r, the value named 'offset left r' in this block, equal to '33'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 183: `offset_right_r <= 33'sd0;` This makes offset_right_r, the value named 'offset right r' in this block, equal to '33'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 184: `offset_sum_r <= 33'sd0;` This makes offset_sum_r, the value named 'offset sum r' in this block, equal to '33'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 185: `wide_result_r <= 64'sd0;` This makes wide_result_r, the value named 'wide result r' in this block, equal to '64'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 186: `saturated_value_r <= 32'sd0;` This makes saturated_value_r, the value named 'saturated value r' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 187: `distance_r <= 32'd0;` This makes distance_r, the value named 'distance r' in this block, equal to '32'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 188: `distance_difference_r <= 33'sd0;` This makes distance_difference_r, the value named 'distance difference r' in this block, equal to '33'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 189: `lane_a_negative_r <= 1'b0;` This makes lane_a_negative_r, one parallel arithmetic element within a vector or tensor datapath, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 190: `lane_a_ge_threshold_r <= 1'b0;` This makes lane_a_ge_threshold_r, the membrane level at which a neuron emits a spike, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 191: `lane_a_le_negative_threshold_r <= 1'b0;` This makes lane_a_le_negative_threshold_r, the membrane level at which a neuron emits a spike, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 192: `lane_a_ge_b_r <= 1'b0;` This makes lane_a_ge_b_r, one parallel arithmetic element within a vector or tensor datapath, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 193: `lane_a_zero_r <= 1'b0;` This makes lane_a_zero_r, one parallel arithmetic element within a vector or tensor datapath, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 194: `lane_a_below_clamp_r <= 1'b0;` This makes lane_a_below_clamp_r, one parallel arithmetic element within a vector or tensor datapath, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 195: `lane_a_above_clamp_r <= 1'b0;` This makes lane_a_above_clamp_r, one parallel arithmetic element within a vector or tensor datapath, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 196: `saturated_below_clamp_r <= 1'b0;` This makes saturated_below_clamp_r, the value named 'saturated below clamp r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 197: `saturated_above_clamp_r <= 1'b0;` This makes saturated_above_clamp_r, the value named 'saturated above clamp r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 198: `inside_surrogate_window_r <= 1'b0;` This makes inside_surrogate_window_r, the value named 'inside surrogate window r' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 199: `lane_result_r <= 32'sd0;` This makes lane_result_r, one parallel arithmetic element within a vector or tensor datapath, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 200: `result <= '0;` This makes result, the value named 'result' in this block, equal to ''0'. It updates after the active clock edge together with other nonblocking assignments. The expression is a literal or structural operator and has no hidden software state. Context: the clocked process that updates registers only on a rising clock edge.

Line 201: `config_error <= 1'b0;` This makes config_error, the value named 'config error' in this block, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 202: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 203: `unique case (state)` This starts a multiplexer/state decode selected by 'state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 204: `ST_IDLE: begin` This is the ST_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 205: `if (input_valid) begin` This condition selects the following hardware branch only when 'input_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_valid: the producer's statement that its current payload is meaningful. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 206: `operation_r <= operation;` This makes operation_r, the value named 'operation r' in this block, equal to 'operation'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operation: the value named 'operation' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 207: `operand_a_r <= operand_a;` This makes operand_a_r, the value named 'operand a r' in this block, equal to 'operand_a'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operand_a: the value named 'operand a' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 208: `operand_b_r <= operand_b;` This makes operand_b_r, the value named 'operand b r' in this block, equal to 'operand_b'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operand_b: the value named 'operand b' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 209: `multiplier_q15_r <= multiplier_q15;` This makes multiplier_q15_r, the value named 'multiplier q15 r' in this block, equal to 'multiplier_q15'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_q15: the value named 'multiplier q15' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 210: `output_zero_point_r <= output_zero_point;` This makes output_zero_point_r, the integer code representing real numeric zero in affine quantization, equal to 'output_zero_point'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_zero_point: the integer code representing real numeric zero in affine quantization. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 211: `threshold_r <= threshold;` This makes threshold_r, the membrane level at which a neuron emits a spike, equal to 'threshold'. It updates after the active clock edge together with other nonblocking assignments. The important operands are threshold: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 212: `negative_threshold_r <= -threshold;` This makes negative_threshold_r, the membrane level at which a neuron emits a spike, equal to '-threshold'. It updates after the active clock edge together with other nonblocking assignments. The important operands are threshold: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 213: `surrogate_window_r <= surrogate_window;` This makes surrogate_window_r, the value named 'surrogate window r' in this block, equal to 'surrogate_window'. It updates after the active clock edge together with other nonblocking assignments. The important operands are surrogate_window: the value named 'surrogate window' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 214: `clamp_minimum_r <= clamp_minimum;` This makes clamp_minimum_r, the value named 'clamp minimum r' in this block, equal to 'clamp_minimum'. It updates after the active clock edge together with other nonblocking assignments. The important operands are clamp_minimum: the value named 'clamp minimum' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 215: `clamp_maximum_r <= clamp_maximum;` This makes clamp_maximum_r, the value named 'clamp maximum r' in this block, equal to 'clamp_maximum'. It updates after the active clock edge together with other nonblocking assignments. The important operands are clamp_maximum: the value named 'clamp maximum' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 216: `config_error <= operation > OP_COMPARE_GE` This statement continues through source line 219; the explanation uses the complete expression rather than only the first visual fragment. This makes config_error, the value named 'config error' in this block, equal to 'operation > OP_COMPARE_GE || clamp_minimum > clamp_maximum || (operation == OP_FLEX_REQUANTIZE && right_shift == 5'b10000)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operation: the value named 'operation' in this block; OP_COMPARE_GE: the value named 'op compare ge' in this block; clamp_minimum: the value named 'clamp minimum' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 217: `|| clamp_minimum > clamp_maximum` This is continuation line 2 of the 4-line logical statement that begins at source line 216. Its exact fragment is '|| clamp_minimum > clamp_maximum'. The important operands are clamp_minimum: the value named 'clamp minimum' in this block; clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This makes config_error, the value named 'config

error' in this block, equal to 'operation > OP_COMPARE_GE || clamp_minimum > clamp_maximum || (operation == OP_FLEX_REQUANTIZE && right_shift == 5'b10000)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operation: the value named 'operation' in this block; OP_COMPARE_GE: the value named 'op compare ge' in this block; clamp_minimum: the value named 'clamp minimum' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 218: `|| (operation == OP_FLEX_REQUANTIZE` This is continuation line 3 of the 4-line logical statement that begins at source line 216. Its exact fragment is '|| (operation == OP_FLEX_REQUANTIZE'. The important operands are operation: the value named 'operation' in this block; OP_FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This makes config_error, the value named 'config error' in this block, equal to 'operation > OP_COMPARE_GE || clamp_minimum > clamp_maximum || (operation == OP_FLEX_REQUANTIZE && right_shift == 5'b10000)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operation: the value named 'operation' in this block; OP_COMPARE_GE: the value named 'op compare ge' in this block; clamp_minimum: the value named 'clamp minimum' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 219: `&& right_shift == 5'b10000);` This is continuation line 4 of the 4-line logical statement that begins at source line 216. Its exact fragment is '&& right_shift == 5'b10000);'. The important operands are right_shift: the value named 'right shift' in this block; b10000: the value named 'b10000' in this block. Read together, the complete operation is: This makes config_error, the value named 'config error' in this block, equal to 'operation > OP_COMPARE_GE || clamp_minimum > clamp_maximum || (operation == OP_FLEX_REQUANTIZE && right_shift == 5'b10000)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operation: the value named 'operation' in this block; OP_COMPARE_GE: the value named 'op compare ge' in this block; clamp_minimum: the value named 'clamp minimum' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 220: `lane_index <= 3'd0;` This makes lane_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '3'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 221: `if (operation == OP_REQUANTIZE) begin` This condition selects the following hardware branch only when 'operation == OP_REQUANTIZE' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are operation: the value named 'operation' in this block; OP_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 222: `shift_amount_r <= 6'd15 + {1'b0, right_shift};` This makes shift_amount_r, the value named 'shift amount r' in this block, equal to '6'd15 + {1'b0, right_shift}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d15: the value named 'd15' in this block; b0: the value named 'b0' in this block; right_shift: the value named 'right shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 223: `rounding_bias_r <= 64'd16384 << right_shift;` This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to '64'd16384 << right_shift'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d16384: the value named 'd16384' in this block; right_shift: the value named 'right shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 224: `end else if (operation == OP_FLEX_REQUANTIZE) begin` This condition selects the following hardware branch only when 'operation == OP_FLEX_REQUANTIZE' is true. Both branches exist as gates; the condition

controls a multiplexer rather than making software jump. The important operands are operation: the value named 'operation' in this block; OP_FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 225: `logic signed [5:0] effective_shift;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.. Specifically, effective_shift is the value named 'effective shift' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 226: `effective_shift = 6'sd15` This statement continues through source line 227; the explanation uses the complete expression rather than only the first visual fragment. This makes effective_shift, the value named 'effective shift' in this block, equal to '6'sd15 + $signed({right_shift[4], right_shift})'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd15: the value named 'sd15' in this block; right_shift: the value named 'right shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 227: `+ $signed({right_shift[4], right_shift});` This is continuation line 2 of the 2-line logical statement that begins at source line 226. Its exact fragment is '+ $signed({right_shift[4], right_shift});'. The important operands are right_shift: the value named 'right shift' in this block. Read together, the complete operation is: This makes effective_shift, the value named 'effective shift' in this block, equal to '6'sd15 + $signed({right_shift[4], right_shift})'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd15: the value named 'sd15' in this block; right_shift: the value named 'right shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 228: `if (right_shift == 5'b10000) begin` This condition selects the following hardware branch only when 'right_shift == 5'b10000' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are right_shift: the value named 'right shift' in this block; b10000: the value named 'b10000' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 229: `shift_amount_r <= 6'd0;` This makes shift_amount_r, the value named 'shift amount r' in this block, equal to '6'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 230: `rounding_bias_r <= 64'd0;` This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 231: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 232: `shift_amount_r <= effective_shift[5:0];` This makes shift_amount_r, the value named 'shift amount r' in this block, equal to 'effective_shift[5:0]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are effective_shift: the value named 'effective shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 233: `rounding_bias_r <= effective_shift == 0` This statement continues through source line 235; the explanation uses the complete expression rather than only the first visual fragment. This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to ‘effective_shift == 0 ? 64’d0 : (64’d1 << (effective_shift - 1’b1))‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are effective_shift: the value named ’effective shift’ in this block; d0: the value named ’d0’ in this block; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 234: `? 64’d0` This is continuation line 2 of the 3-line logical statement that begins at source line 233. Its exact fragment is ‘? 64’d0‘. The important operands are d0: the value named ’d0’ in this block. Read together, the complete operation is: This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to ‘effective_shift == 0 ? 64’d0 : (64’d1 << (effective_shift - 1’b1))‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are effective_shift: the value named ’effective shift’ in this block; d0: the value named ’d0’ in this block; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 235: `: (64’d1 << (effective_shift - 1’b1));` This is continuation line 3 of the 3-line logical statement that begins at source line 233. Its exact fragment is ‘: (64’d1 << (effective_shift - 1’b1));‘. The important operands are d1: the value named ’d1’ in this block; effective_shift: the value named ’effective shift’ in this block; b1: the value named ’b1’ in this block. Read together, the complete operation is: This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to ‘effective_shift == 0 ? 64’d0 : (64’d1 << (effective_shift - 1’b1))‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are effective_shift: the value named ’effective shift’ in this block; d0: the value named ’d0’ in this block; d1: the value named ’d1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 236: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 237: `end else if (operation == OP_MULTIPLY_Q15) begin` This condition selects the following hardware branch only when ‘operation == OP_MULTIPLY_Q15‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are operation: the value named ’operation’ in this block; OP_MULTIPLY_Q15: the value named ’op multiply q15’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 238: `shift_amount_r <= 6’d15;` This makes shift_amount_r, the value named ’shift amount r’ in this block, equal to ‘6’d15‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d15: the value named ’d15’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 239: `rounding_bias_r <= 64’d16384;` This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to ‘64’d16384‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are d16384: the value named ’d16384’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 240: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 241: `shift_amount_r <= 6’d0;` This makes shift_amount_r, the value named ’shift amount r’ in this block, equal to ‘6’d0‘. It updates after the active clock edge together with other nonblocking assignments. The important

operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 242: `rounding_bias_r <= 64'd0;` This makes rounding_bias_r, an additive learned offset applied after accumulation, equal to '64'd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are d0: the value named 'd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 243: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 244: `state <= ST_LOAD_LANE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LOAD_LANE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_LANE: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data..

Line 245: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 246: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 248: `ST_LOAD_LANE: begin` This is the ST_LOAD_LANE case branch, which stores incoming operands into local FPGA memory. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_LANE, the machine stores incoming operands into local FPGA memory..

Line 249: `lane_a_r <= $signed(` This statement continues through source line 250; the explanation uses the complete expression rather than only the first visual fragment. This makes lane_a_r, one parallel arithmetic element within a vector or tensor datapath, equal to '$signed(operand_a_r[lane_index*DATA_BITS +: DATA_BITS])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operand_a_r: the value named 'operand a r' in this block; lane_index: an integer selecting one tensor element, lane, bank, or state entry; DATA_BITS: the value named 'data bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_LANE, the machine stores incoming operands into local FPGA memory..

Line 250: `operand_a_r[lane_index*DATA_BITS +: DATA_BITS]);` This is continuation line 2 of the 2-line logical statement that begins at source line 249. Its exact fragment is 'operand_a_r[lane_index*DATA_BITS +: DATA_BITS]);'. The important operands are operand_a_r: the value named 'operand a r' in this block; lane_index: an integer selecting one tensor element, lane, bank, or state entry; DATA_BITS: the value named 'data bits' in this block. Read together, the complete operation is: This makes lane_a_r, one parallel arithmetic element within a vector or tensor datapath, equal to '$signed(operand_a_r[lane_index*DATA_BITS +: DATA_BITS])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operand_a_r: the value named 'operand a r' in this block; lane_index: an integer selecting one tensor element, lane, bank, or state entry; DATA_BITS: the value named 'data bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_LANE, the machine stores incoming operands into local FPGA memory..

Line 251: `lane_b_r <= $signed(` This statement continues through source line 252; the explanation uses the complete expression rather than only the first visual fragment. This makes lane_b_r, one parallel arithmetic element within a vector or tensor datapath, equal to '$signed(operand_b_r[lane_index*DATA_BITS +: DATA_BITS])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operand_b_r: the

value named 'operand b r' in this block; lane_index: an integer selecting one tensor element, lane, bank, or state entry; DATA_BITS: the value named 'data bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_LANE, the machine stores incoming operands into local FPGA memory..

Line 252: `operand_b_r[lane_index*DATA_BITS +: DATA_BITS]);` This is continuation line 2 of the 2-line logical statement that begins at source line 251. Its exact fragment is 'operand_b_r[lane_index*DATA_BITS +: DATA_BITS]);'. The important operands are operand_b_r: the value named 'operand b r' in this block; lane_index: an integer selecting one tensor element, lane, bank, or state entry; DATA_BITS: the value named 'data bits' in this block. Read together, the complete operation is: This makes lane_b_r, one parallel arithmetic element within a vector or tensor datapath, equal to '$signed(operand_b_r[lane_index*DATA_BITS +: DATA_BITS])'. It updates after the active clock edge together with other nonblocking assignments. The important operands are operand_b_r: the value named 'operand b r' in this block; lane_index: an integer selecting one tensor element, lane, bank, or state entry; DATA_BITS: the value named 'data bits' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_LANE, the machine stores incoming operands into local FPGA memory..

Line 253: `state <= ST_MULTIPLIER_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MULTIPLIER_LATCH'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MULTIPLIER_LATCH: the value named 'st multiplier latch' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_LANE, the machine stores incoming operands into local FPGA memory..

Line 254: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_LOAD_LANE, the machine stores incoming operands into local FPGA memory.; it has no arithmetic value of its own.

Line 256: `ST_MULTIPLIER_LATCH: begin` This is the ST_MULTIPLIER_LATCH case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase..

Line 257: `multiplier_left_r <= lane_a_r;` This makes multiplier_left_r, the value named 'multiplier left r' in this block, equal to 'lane_a_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase..

Line 258: `if (operation_r == OP_REQUANTIZE` Within the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase., this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are operation_r: the value named 'operation r' in this block; OP_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code.

Line 259: `|| operation_r == OP_FLEX_REQUANTIZE) begin` Within the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase., this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are operation_r: the value named 'operation r' in this block; OP_FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code.

Line 260: `multiplier_right_r <= {` This statement continues through source line 261; the explanation uses the complete expression rather than only the first visual fragment. This makes multiplier_right_r, the value named 'multiplier right r' in this block, equal to '{ {16{multiplier_q15_r[15]}}, multiplier_q15_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_q15_r: the value named 'multiplier q15 r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase..

Line 261: `{16{multiplier_q15_r[15]}}, multiplier_q15_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 260. Its exact fragment is '{16{multiplier_q15_r[15]}}, multiplier_q15_r};'. The important operands are multiplier_q15_r: the value named 'multiplier q15 r' in this block. Read together, the complete operation is: This makes multiplier_right_r, the value named 'multiplier right r' in this block, equal to '{ {16{multiplier_q15_r[15]}}, multiplier_q15_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_q15_r: the value named 'multiplier q15 r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase..

Line 262: `end else if (operation_r == OP_MULTIPLY_Q15) begin` This condition selects the following hardware branch only when 'operation_r == OP_MULTIPLY_Q15' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are operation_r: the value named 'operation r' in this block; OP_MULTIPLY_Q15: the value named 'op multiply q15' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase..

Line 263: `multiplier_right_r <= lane_b_r;` This makes multiplier_right_r, the value named 'multiplier right r' in this block, equal to 'lane_b_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_b_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase..

Line 264: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 265: `multiplier_right_r <= 32'sd0;` This makes multiplier_right_r, the value named 'multiplier right r' in this block, equal to '32'sd0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase..

Line 266: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 267: `state <= ST_MULTIPLY;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_MULTIPLY'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_MULTIPLY: the value named 'st multiply' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase..

Line 268: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLIER_LATCH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 270: `ST_MULTIPLY: begin` This is the ST_MULTIPLY case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 271: `multiplier_left_low_r <= $signed({1'b0, multiplier_left_r[15:0]});` This makes multiplier_left_low_r, the value named 'multiplier left low r' in this block, equal to '$signed({1'b0, multiplier_left_r[15:0]})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block; multiplier_left_r: the value named 'multiplier left r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 272: `multiplier_right_low_r <= $signed({1'b0, multiplier_right_r[15:0]});` This makes multiplier_right_low_r the value named 'multiplier right low r' in this block, equal to '$signed({1'b0, multiplier_right_r[15:0]})'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block; multiplier_right_r: the value named 'multiplier right r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 273: `multiplier_left_high_r <= multiplier_left_r[31:16];` This makes multiplier_left_high_r, the value named 'multiplier left high r' in this block, equal to 'multiplier_left_r[31:16]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_left_r: the value named 'multiplier left r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 274: `multiplier_right_high_r <= multiplier_right_r[31:16];` This makes multiplier_right_high_r, the value named 'multiplier right high r' in this block, equal to 'multiplier_right_r[31:16]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_right_r: the value named 'multiplier right r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 275: `state <= ST_PRODUCT_CROSS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_PRODUCT_CROSS'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PRODUCT_CROSS: the value named 'st product cross' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase..

Line 276: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_MULTIPLY, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 278: `ST_PRODUCT_CROSS: begin` This is the ST_PRODUCT_CROSS case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_CROSS, the machine performs the named control phase..

Line 279: `product_ll_r <= multiplier_product_ll;` This makes product_ll_r, the value named 'product ll r' in this block, equal to 'multiplier_product_ll'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_product_ll: the value named 'multiplier product ll' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_CROSS, the machine performs the named control phase..

Line 280: `product_lh_r <= multiplier_product_lh;` This makes product_lh_r, the value named 'product lh r' in this block, equal to 'multiplier_product_lh'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_product_lh: the value named 'multiplier product lh' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_CROSS, the machine performs the named control phase..

Line 281: `product_hl_r <= multiplier_product_hl;` This makes product_hl_r, the value named 'product hl r' in this block, equal to 'multiplier_product_hl'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_product_hl: the value named 'multiplier product hl' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_CROSS, the machine performs the named control phase..

Line 282: `product_hh_r <= multiplier_product_hh;` This makes product_hh_r, the value named 'product hh r' in this block, equal to 'multiplier_product_hh'. It updates after the active clock edge together with other nonblocking assignments. The important operands are multiplier_product_hh: the value named 'multiplier product hh' in this block.

Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_CROSS, the machine performs the named control phase..

Line 283: `state <= ST_PRODUCT_LOW;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_PRODUCT_LOW‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PRODUCT_LOW: the value named ’st product low’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_CROSS, the machine performs the named control phase..

Line 284: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_CROSS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 286: `ST_PRODUCT_LOW: begin` This is the ST_PRODUCT_LOW case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LOW, the machine performs the named control phase..

Line 287: `product_cross_r <= $signed({product_lh_r[32], product_lh_r})` This statement continues through source line 288; the explanation uses the complete expression rather than only the first visual fragment. This makes product_cross_r, the value named ’product cross r’ in this block, equal to ‘$signed({product_lh_r[32], product_lh_r}) + $signed({product_hl_r[32], product_hl_r})‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_lh_r: the value named ’product lh r’ in this block; product_hl_r: the value named ’product hl r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LOW, the machine performs the named control phase..

Line 288: `+ $signed({product_hl_r[32], product_hl_r});` This is continuation line 2 of the 2-line logical statement that begins at source line 287. Its exact fragment is ‘+ $signed({product_hl_r[32], product_hl_r});‘. The important operands are product_hl_r: the value named ’product hl r’ in this block. Read together, the complete operation is: This makes product_cross_r, the value named ’product cross r’ in this block, equal to ‘$signed({product_lh_r[32], product_lh_r}) + $signed({product_hl_r[32], product_hl_r})‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_lh_r: the value named ’product lh r’ in this block; product_hl_r: the value named ’product hl r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LOW, the machine performs the named control phase..

Line 289: `product_low_r <= $signed({{30{product_ll_r[33]}}, product_ll_r});` This makes product_low_r, the value named ’product low r’ in this block, equal to ‘$signed({{30{product_ll_r[33]}}, product_ll_r})‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_ll_r: the value named ’product ll r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LOW, the machine performs the named control phase..

Line 290: `state <= ST_PRODUCT_LATCH;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_PRODUCT_LATCH‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PRODUCT_LATCH: the value named ’st product latch’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LOW, the machine performs the named control phase..

Line 291: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LOW, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 293: `ST_PRODUCT_LATCH: begin` This is the ST_PRODUCT_LATCH case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LATCH, the machine performs the named control phase..

Line 294: `product_latched_r <= product_low_r` This statement continues through source line 296; the explanation uses the complete expression rather than only the first visual fragment. This makes product_latched_r, the value named 'product latched r' in this block, equal to 'product_low_r + ($signed({{30{product_cross_r[33]}}, product_cross_r}) <<< 16) + ($signed({{32{product_hh_r[31]}}, product_hh_r}) <<< 32)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_low_r: the value named 'product low r' in this block; product_cross_r: the value named 'product cross r' in this block; product_hh_r: the value named 'product hh r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LATCH, the machine performs the named control phase..

Line 295: `+ ($signed({{30{product_cross_r[33]}}, product_cross_r}) <<< 16)` This is continuation line 2 of the 3-line logical statement that begins at source line 294. Its exact fragment is '+ ($signed({{30{product_cross_r[33]}}, product_cross_r}) <<< 16)'. The important operands are product_cross_r: the value named 'product cross r' in this block. Read together, the complete operation is: This makes product_latched_r, the value named 'product latched r' in this block, equal to 'product_low_r + ($signed({{30{product_cross_r[33]}}, product_cross_r}) <<< 16) + ($signed({{32{product_hh_r[31]}}, product_hh_r}) <<< 32)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_low_r: the value named 'product low r' in this block; product_cross_r: the value named 'product cross r' in this block; product_hh_r: the value named 'product hh r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LATCH, the machine performs the named control phase..

Line 296: `+ ($signed({{32{product_hh_r[31]}}, product_hh_r}) <<< 32);` This is continuation line 3 of the 3-line logical statement that begins at source line 294. Its exact fragment is '+ ($signed({{32{product_hh_r[31]}}, product_hh_r}) <<< 32);'. The important operands are product_hh_r: the value named 'product hh r' in this block. Read together, the complete operation is: This makes product_latched_r, the value named 'product latched r' in this block, equal to 'product_low_r + ($signed({{30{product_cross_r[33]}}, product_cross_r}) <<< 16) + ($signed({{32{product_hh_r[31]}}, product_hh_r}) <<< 32)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_low_r: the value named 'product low r' in this block; product_cross_r: the value named 'product cross r' in this block; product_hh_r: the value named 'product hh r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LATCH, the machine performs the named control phase..

Line 297: `state <= ST_ABSOLUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ABSOLUTE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ABSOLUTE: the value named 'st absolute' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LATCH, the machine performs the named control phase..

Line 298: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PRODUCT_LATCH, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 300: `ST_ABSOLUTE: begin` This is the ST_ABSOLUTE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ABSOLUTE, the machine performs the named control phase..

Line 301: `product_negative_r <= product_latched_r[63];` This makes product_negative_r, the value named 'product negative r' in this block, equal to 'product_latched_r[63]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_latched_r: the value named 'product latched r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ABSOLUTE, the machine performs the named control phase..

Line 302: `magnitude_r <= product_latched_r[63]` This statement continues through source line 303; the explanation uses the complete expression rather than only the first visual fragment. This makes magnitude_r, the value named 'magnitude r' in this block, equal to 'product_latched_r[63] ? -product_latched_r : product_latched_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_latched_r:

the value named 'product latched r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ABSOLUTE, the machine performs the named control phase..

Line 303: `? -product_latched_r : product_latched_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 302. Its exact fragment is '? -product_latched_r : product_latched_r;'. The important operands are product_latched_r: the value named 'product latched r' in this block. Read together, the complete operation is: This makes magnitude_r, the value named 'magnitude r' in this block, equal to 'product_latched_r[63] ? -product_latched_r : product_latched_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_latched_r: the value named 'product latched r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ABSOLUTE, the machine performs the named control phase..

Line 304: `state <= ST_ROUND_BIAS;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_ROUND_BIAS'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_ROUND_BIAS: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ABSOLUTE, the machine performs the named control phase..

Line 305: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ABSOLUTE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 307: `ST_ROUND_BIAS: begin` This is the ST_ROUND_BIAS case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase..

Line 308: `biased_magnitude_r <= magnitude_r + rounding_bias_r;` This makes biased_magnitude_r, an additive learned offset applied after accumulation, equal to 'magnitude_r + rounding_bias_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are magnitude_r: the value named 'magnitude r' in this block; rounding_bias_r: an additive learned offset applied after accumulation. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase..

Line 309: `state <= ST_SHIFT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_SHIFT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_SHIFT: the value named 'st shift' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase..

Line 310: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_ROUND_BIAS, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 312: `ST_SHIFT: begin` This is the ST_SHIFT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase..

Line 313: `shifted_magnitude_r <= biased_magnitude_r >> shift_amount_r;` This makes shifted_magnitude_r, the value named 'shifted magnitude r' in this block, equal to 'biased_magnitude_r >> shift_amount_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are biased_magnitude_r: an additive learned offset applied after accumulation; shift_amount_r: the value named 'shift amount r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase..

Line 314: `state <= ST_RESTORE_SIGN;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_RESTORE_SIGN‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_RESTORE_SIGN: the value named ’st restore sign’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase..

Line 315: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_SHIFT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 317: `ST_RESTORE_SIGN: begin` This is the ST_RESTORE_SIGN case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_SIGN, the machine performs the named control phase..

Line 318: `rounded_signed_r <= product_negative_r` This statement continues through source line 320; the explanation uses the complete expression rather than only the first visual fragment. This makes rounded_signed_r, the value named ’rounded signed r’ in this block, equal to ‘product_negative_r ? -$signed(shifted_magnitude_r) : $signed(shifted_magnitude_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named ’product negative r’ in this block; shifted_magnitude_r: the value named ’shifted magnitude r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_SIGN, the machine performs the named control phase..

Line 319: `? -$signed(shifted_magnitude_r)` This is continuation line 2 of the 3-line logical statement that begins at source line 318. Its exact fragment is ‘? -$signed(shifted_magnitude_r)‘. The important operands are shifted_magnitude_r: the value named ’shifted magnitude r’ in this block. Read together, the complete operation is: This makes rounded_signed_r, the value named ’rounded signed r’ in this block, equal to ‘product_negative_r ? -$signed(shifted_magnitude_r) : $signed(shifted_magnitude_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named ’product negative r’ in this block; shifted_magnitude_r: the value named ’shifted magnitude r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_SIGN, the machine performs the named control phase..

Line 320: `: $signed(shifted_magnitude_r);` This is continuation line 3 of the 3-line logical statement that begins at source line 318. Its exact fragment is ‘: $signed(shifted_magnitude_r);‘. The important operands are shifted_magnitude_r: the value named ’shifted magnitude r’ in this block. Read together, the complete operation is: This makes rounded_signed_r, the value named ’rounded signed r’ in this block, equal to ‘product_negative_r ? -$signed(shifted_magnitude_r) : $signed(shifted_magnitude_r)‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are product_negative_r: the value named ’product negative r’ in this block; shifted_magnitude_r: the value named ’shifted magnitude r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_SIGN, the machine performs the named control phase..

Line 321: `state <= ST_OFFSET;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OFFSET‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OFFSET: the value named ’st offset’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_SIGN, the machine performs the named control phase..

Line 322: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_RESTORE_SIGN, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 324: `ST_OFFSET: begin` This is the ST_OFFSET case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OFFSET, the machine performs the named control phase..

Line 325: `unique case (operation_r)` This starts a multiplexer/state decode selected by ‘operation_r‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 326: `OP_ADD_SAT: begin` This is the OP_ADD_SAT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state OP_ADD_SAT, the machine performs the named control phase..

Line 327: `offset_left_r <= {lane_a_r[31], lane_a_r};` This makes offset_left_r, the value named ’offset left r’ in this block, equal to ‘{lane_a_r[31], lane_a_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state OP_ADD_SAT, the machine performs the named control phase..

Line 328: `offset_right_r <= {lane_b_r[31], lane_b_r};` This makes offset_right_r, the value named ’offset right r’ in this block, equal to ‘{lane_b_r[31], lane_b_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_b_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state OP_ADD_SAT, the machine performs the named control phase..

Line 329: `state <= ST_OFFSET_ADD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OFFSET_ADD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OFFSET_ADD: the value named ’st offset add’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_ADD_SAT, the machine performs the named control phase..

Line 330: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_ADD_SAT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 331: `OP_REQUANTIZE,` Within the clocked process that updates registers only on a rising clock edge Within state OP_ADD_SAT, the machine performs the named control phase., this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are OP_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code.

Line 332: `OP_FLEX_REQUANTIZE: begin` This is the OP_FLEX_REQUANTIZE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 333: `offset_left_r <= rounded_signed_r[32:0];` This makes offset_left_r, the value named ’offset left r’ in this block, equal to ‘rounded_signed_r[32:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounded_signed_r: the value named ’rounded signed r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 334: `offset_right_r <= {{17{output_zero_point_r[15]}},` This statement continues through source line 335; the explanation uses the complete expression rather than only the first visual fragment. This makes offset_right_r, the value named ’offset right r’ in this block, equal to ‘{{17{output_zero_point_r[15]}}, output_zero_point_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_zero_point_r: the integer code representing real numeric zero in affine quantization. Context: the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 335: `output_zero_point_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 334. Its exact fragment is ‘output_zero_point_r};‘. The important operands are output_zero_point_r: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This makes offset_right_r, the value named ’offset right r’ in this block, equal to ‘{{17{output_zero_point_r[15]}}, output_zero_point_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_zero_point_r: the integer code representing real numeric zero in affine quantization. Context: the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 336: `state <= ST_OFFSET_ADD;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OFFSET_ADD‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OFFSET_ADD: the value named ’st offset add’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 337: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 338: `OP_MULTIPLY_Q15: begin` This is the OP_MULTIPLY_Q15 case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase..

Line 339: `wide_result_r <= rounded_signed_r;` This makes wide_result_r, the value named ’wide result r’ in this block, equal to ‘rounded_signed_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are rounded_signed_r: the value named ’rounded signed r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase..

Line 340: `state <= ST_PRECOMPUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_PRECOMPUTE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PRECOMPUTE: the value named ’st precompute’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase..

Line 341: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 342: `default: begin` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase..

Line 343: `wide_result_r <= {{32{lane_a_r[31]}}, lane_a_r};` This makes wide_result_r, the value named ’wide result r’ in this block, equal to ‘{{32{lane_a_r[31]}}, lane_a_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase..

Line 344: `state <= ST_PRECOMPUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_PRECOMPUTE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PRECOMPUTE: the value named ’st precompute’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase..

Line 345: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 346: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 347: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 349: `ST_OFFSET_ADD: begin` This is the ST_OFFSET_ADD case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OFFSET_ADD, the machine performs the named control phase..

Line 350: `offset_sum_r <= offset_left_r + offset_right_r;` This makes offset_sum_r, the value named ’offset sum r’ in this block, equal to ‘offset_left_r + offset_right_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are offset_left_r: the value named ’offset left r’ in this block; offset_right_r: the value named ’offset right r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OFFSET_ADD, the machine performs the named control phase..

Line 351: `state <= ST_OFFSET_EXTEND;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OFFSET_EXTEND‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OFFSET_EXTEND: the value named ’st offset extend’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OFFSET_ADD, the machine performs the named control phase..

Line 352: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OFFSET_ADD, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 354: `ST_OFFSET_EXTEND: begin` This is the ST_OFFSET_EXTEND case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_OFFSET_EXTEND, the machine performs the named control phase..

Line 355: `wide_result_r <= {{31{offset_sum_r[32]}}, offset_sum_r};` This makes wide_result_r, the value named ’wide result r’ in this block, equal to ‘{{31{offset_sum_r[32]}}, offset_sum_r}‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are offset_sum_r: the value named ’offset sum r’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OFFSET_EXTEND, the machine performs the named control phase..

Line 356: `state <= ST_PRECOMPUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_PRECOMPUTE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_PRECOMPUTE: the value named ’st precompute’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_OFFSET_EXTEND, the machine performs the named control phase..

Line 357: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OFFSET_EXTEND, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 359: `ST_PRECOMPUTE: begin` This is the ST_PRECOMPUTE case branch, which issues arithmetic work using values already held on chip. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 360: `saturated_value_r <= saturate_s32(wide_result_r);` This makes saturated_value_r, the value named 'saturated value r' in this block, equal to 'saturate_s32(wide_result_r)'. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturate_s32: the value named 'saturate s32' in this block; wide_result_r: the value named 'wide result r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 361: `distance_difference_r <= {lane_a_r[31], lane_a_r}` This statement continues through source line 362; the explanation uses the complete expression rather than only the first visual fragment. This makes distance_difference_r, the value named 'distance difference r' in this block, equal to '{lane_a_r[31], lane_a_r} - {threshold_r[31], threshold_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath; threshold_r: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 362: `- {threshold_r[31], threshold_r};` This is continuation line 2 of the 2-line logical statement that begins at source line 361. Its exact fragment is '- {threshold_r[31], threshold_r};'. The important operands are threshold_r: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This makes distance_difference_r, the value named 'distance difference r' in this block, equal to '{lane_a_r[31], lane_a_r} - {threshold_r[31], threshold_r}'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath; threshold_r: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 363: `lane_a_negative_r <= lane_a_r < 0;` This makes lane_a_negative_r, one parallel arithmetic element within a vector or tensor datapath, equal to 'lane_a_r < 0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 364: `lane_a_ge_threshold_r <= lane_a_r >= threshold_r;` This makes lane_a_ge_threshold_r, the membrane level at which a neuron emits a spike, equal to 'lane_a_r >= threshold_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath; threshold_r: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 365: `lane_a_le_negative_threshold_r <= lane_a_r <= negative_threshold_r;` This makes lane_a_le_negative_thr the membrane level at which a neuron emits a spike, equal to 'lane_a_r <= negative_threshold_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath; negative_threshold_r: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 366: `lane_a_ge_b_r <= lane_a_r >= lane_b_r;` This makes lane_a_ge_b_r, one parallel arithmetic element within a vector or tensor datapath, equal to 'lane_a_r >= lane_b_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath; lane_b_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 367: `lane_a_zero_r <= lane_a_r == 0;` This makes lane_a_zero_r, one parallel arithmetic element within a vector or tensor datapath, equal to 'lane_a_r == 0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 368: `state <= ST_DISTANCE_ABSOLUTE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_DISTANCE_ABSOLUTE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_DISTANCE_ABSOLUTE: the value named ’st distance absolute’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip..

Line 369: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_PRECOMPUTE, the machine issues arithmetic work using values already held on chip.; it has no arithmetic value of its own.

Line 371: `ST_DISTANCE_ABSOLUTE: begin` This is the ST_DISTANCE_ABSOLUTE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_DISTANCE_ABSOLUTE, the machine performs the named control phase..

Line 372: `distance_r <= distance_difference_r[32]` This statement continues through source line 374; the explanation uses the complete expression rather than only the first visual fragment. This makes distance_r, the value named ’distance r’ in this block, equal to ‘distance_difference_r[32] ? (~distance_difference_r[31:0] + 1’b1) : distance_difference_r[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are distance_difference_r: the value named ’distance difference r’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DISTANCE_ABSOLUTE, the machine performs the named control phase..

Line 373: `? (~distance_difference_r[31:0] + 1’b1)` This is continuation line 2 of the 3-line logical statement that begins at source line 372. Its exact fragment is ‘? (~distance_difference_r[31:0] + 1’b1)‘. The important operands are distance_difference_r: the value named ’distance difference r’ in this block; b1: the value named ’b1’ in this block. Read together, the complete operation is: This makes distance_r, the value named ’distance r’ in this block, equal to ‘distance_difference_r[32] ? (~distance_difference_r[31:0] + 1’b1) : distance_difference_r[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are distance_difference_r: the value named ’distance difference r’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DISTANCE_ABSOLUTE, the machine performs the named control phase..

Line 374: `: distance_difference_r[31:0];` This is continuation line 3 of the 3-line logical statement that begins at source line 372. Its exact fragment is ‘: distance_difference_r[31:0];‘. The important operands are distance_difference_r: the value named ’distance difference r’ in this block. Read together, the complete operation is: This makes distance_r, the value named ’distance r’ in this block, equal to ‘distance_difference_r[32] ? (~distance_difference_r[31:0] + 1’b1) : distance_difference_r[31:0]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are distance_difference_r: the value named ’distance difference r’ in this block; b1: the value named ’b1’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DISTANCE_ABSOLUTE, the machine performs the named control phase..

Line 375: `state <= ST_COMPARE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_COMPARE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_COMPARE: the value named ’st compare’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_DISTANCE_ABSOLUTE, the machine performs the named control phase..

Line 376: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_DISTANCE_ABSOLUTE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 378: `ST_COMPARE: begin` This is the ST_COMPARE case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_COMPARE, the machine performs the named control phase..

Line 379: `inside_surrogate_window_r <= distance_r <= surrogate_window_r;` This makes inside_surrogate_window_r, the value named 'inside surrogate window r' in this block, equal to 'distance_r <= surrogate_window_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are distance_r: the value named 'distance r' in this block; surrogate_window_r: the value named 'surrogate window r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPARE, the machine performs the named control phase..

Line 380: `lane_a_below_clamp_r <= lane_a_r < clamp_minimum_r;` This makes lane_a_below_clamp_r, one parallel arithmetic element within a vector or tensor datapath, equal to 'lane_a_r < clamp_minimum_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath; clamp_minimum_r: the value named 'clamp minimum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPARE, the machine performs the named control phase..

Line 381: `lane_a_above_clamp_r <= lane_a_r > clamp_maximum_r;` This makes lane_a_above_clamp_r, one parallel arithmetic element within a vector or tensor datapath, equal to 'lane_a_r > clamp_maximum_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath; clamp_maximum_r: the value named 'clamp maximum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPARE, the machine performs the named control phase..

Line 382: `saturated_below_clamp_r <= saturated_value_r < clamp_minimum_r;` This makes saturated_below_clamp_r, the value named 'saturated below clamp r' in this block, equal to 'saturated_value_r < clamp_minimum_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturated_value_r: the value named 'saturated value r' in this block; clamp_minimum_r: the value named 'clamp minimum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPARE, the machine performs the named control phase..

Line 383: `saturated_above_clamp_r <= saturated_value_r > clamp_maximum_r;` This makes saturated_above_clamp_r, the value named 'saturated above clamp r' in this block, equal to 'saturated_value_r > clamp_maximum_r'. It updates after the active clock edge together with other nonblocking assignments. The important operands are saturated_value_r: the value named 'saturated value r' in this block; clamp_maximum_r: the value named 'clamp maximum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPARE, the machine performs the named control phase..

Line 384: `state <= ST_RESULT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_RESULT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_RESULT: the value named 'st result' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_COMPARE, the machine performs the named control phase..

Line 385: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_COMPARE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 387: `ST_RESULT: begin` This is the ST_RESULT case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state ST_RESULT, the machine performs the named control phase..

Line 388: `logic signed [31:0] lane_result;` This declares hardware signal storage or wiring in the clocked process that updates registers only on a rising clock edge Within state ST_RESULT, the machine performs the named control phase.. Specifically, lane_result is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 389: `lane_result = 32'sd0;` This makes lane_result, one parallel arithmetic element within a vector or tensor datapath, equal to '32'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_RESULT, the machine performs the named control phase..

Line 391: `unique case (operation_r)` This starts a multiplexer/state decode selected by 'operation_r'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 392: `OP_ADD_SAT: lane_result = saturated_value_r;` This is the OP_ADD_SAT case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = saturated_value_r;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_ADD_SAT, the machine performs the named control phase..

Line 393: `OP_RELU: lane_result = lane_a_negative_r ? 32'sd0 : lane_a_r;` This is the OP_RELU case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_negative_r ? 32'sd0 : lane_a_r;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_RELU, the machine performs the named control phase..

Line 394: `OP_REQUANTIZE,` Within the clocked process that updates registers only on a rising clock edge Within state OP_RELU, the machine performs the named control phase., this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are OP_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code.

Line 395: `OP_FLEX_REQUANTIZE: begin` This is the OP_FLEX_REQUANTIZE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 396: `if (saturated_below_clamp_r) begin` This condition selects the following hardware branch only when 'saturated_below_clamp_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are saturated_below_clamp_r: the value named 'saturated below clamp r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 397: `lane_result = clamp_minimum_r;` This makes lane_result, one parallel arithmetic element within a vector or tensor datapath, equal to 'clamp_minimum_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are clamp_minimum_r: the value named 'clamp minimum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 398: `end else if (saturated_above_clamp_r) begin` This condition selects the following hardware branch only when 'saturated_above_clamp_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are saturated_above_clamp_r: the value named 'saturated above clamp r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 399: `lane_result = clamp_maximum_r;` This makes lane_result, one parallel arithmetic element within a vector or tensor datapath, equal to 'clamp_maximum_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are clamp_maximum_r: the value named 'clamp maximum r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 400: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 401: `lane_result = saturated_value_r;` This makes lane_result, one parallel arithmetic element within a vector or tensor datapath, equal to 'saturated_value_r'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturated_value_r: the value named 'saturated value r' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase..

Line 402: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 403: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_FLEX_REQUANTIZE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 404: `OP_BINARY_SPIKE: lane_result = lane_a_ge_threshold_r` This statement continues through source line 405; the explanation uses the complete expression rather than only the first visual fragment. This is the OP_BINARY_SPIKE case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_ge_threshold_r ? 32'sd1 : 32'sd0;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_BINARY_SPIKE, the machine performs the named control phase..

Line 405: `? 32'sd1 : 32'sd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 404. Its exact fragment is '? 32'sd1 : 32'sd0;'. The important operands are sd1: the value named 'sd1' in this block; sd0: the value named 'sd0' in this block. Read together, the complete operation is: This is the OP_BINARY_SPIKE case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_ge_threshold_r ? 32'sd1 : 32'sd0;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_BINARY_SPIKE, the machine performs the named control phase..

Line 406: `OP_TERNARY_SPIKE: begin` This is the OP_TERNARY_SPIKE case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state OP_TERNARY_SPIKE, the machine performs the named control phase..

Line 407: `if (lane_a_ge_threshold_r) lane_result = 32'sd1;` This condition selects the following hardware branch only when 'lane_a_ge_threshold_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lane_a_ge_threshold_r: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state OP_TERNARY_SPIKE, the machine performs the named control phase..

Line 408: `else if (lane_a_le_negative_threshold_r) lane_result = -32'sd1;` This condition selects the following hardware branch only when 'lane_a_le_negative_threshold_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lane_a_le_negative_threshold_r: the membrane level at which a neuron emits a spike. Context: the clocked process that updates registers only on a rising clock edge Within state OP_TERNARY_SPIKE, the machine performs the named control phase..

Line 409: `else lane_result = 32'sd0;` Within the clocked process that updates registers only on a rising clock edge Within state OP_TERNARY_SPIKE, the machine performs the named control phase., this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are lane_result: one parallel arithmetic element within a vector or tensor datapath; sd0: the value named 'sd0' in this block.

Line 410: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_TERNARY_SPIKE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 411: `OP_SURROGATE_GRADIENT: lane_result =` This statement continues through source line 412; the explanation uses the complete expression rather than only the first visual fragment. This is the OP_SURROGATE_GRADIENT case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = inside_surrogate_windov ? lane_b_r : 32'sd0;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_SURROGATE_GRADIENT, the machine performs the named control phase..

Line 412: `inside_surrogate_window_r ? lane_b_r : 32'sd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 411. Its exact fragment is 'inside_surrogate_window_r ? lane_b_r : 32'sd0;'. The important operands are inside_surrogate_window_r: the value named 'inside surrogate window r' in this block; lane_b_r: one parallel arithmetic element within a vector or tensor datapath; sd0: the value named 'sd0' in this block. Read together, the complete operation is: This is the OP_SURROGATE_GRADIENT case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = inside_surrogate_window_r ? lane_b_r : 32'sd0;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_SURROGATE_GRADIENT, the machine performs the named control phase..

Line 413: `OP_MAXIMUM: lane_result = lane_a_ge_b_r` This statement continues through source line 414; the explanation uses the complete expression rather than only the first visual fragment. This is the OP_MAXIMUM case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_ge_b_r ? lane_a_r : lane_b_r;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_MAXIMUM, the machine performs the named control phase..

Line 414: `? lane_a_r : lane_b_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 413. Its exact fragment is '? lane_a_r : lane_b_r;'. The important operands are lane_a_r: one parallel arithmetic element within a vector or tensor datapath; lane_b_r: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This is the OP_MAXIMUM case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_ge_b_r ? lane_a_r : lane_b_r;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_MAXIMUM, the machine performs the named control phase..

Line 415: `OP_MULTIPLY_Q15: lane_result = saturated_value_r;` This is the OP_MULTIPLY_Q15 case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = saturated_value_r;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_MULTIPLY_Q15, the machine performs the named control phase..

Line 416: `OP_CLAMP: begin` This is the OP_CLAMP case branch, which performs this explicitly named phase. The same-cycle action begins with 'begin'; context is the clocked process that updates registers only on a rising clock edge Within state OP_CLAMP, the machine performs the named control phase..

Line 417: `if (lane_a_below_clamp_r) lane_result = clamp_minimum_r;` This condition selects the following hardware branch only when 'lane_a_below_clamp_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lane_a_below_clamp_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state OP_CLAMP, the machine performs the named control phase..

Line 418: `else if (lane_a_above_clamp_r) lane_result = clamp_maximum_r;` This condition selects the following hardware branch only when 'lane_a_above_clamp_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lane_a_above_clamp_r: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state OP_CLAMP, the machine performs the named control phase..

Line 419: `else lane_result = lane_a_r;` Within the clocked process that updates registers only on a rising clock edge Within state OP_CLAMP, the machine performs the named control phase., this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are lane_result: one parallel arithmetic element within a vector or tensor datapath; lane_a_r: one parallel arithmetic element within a vector or tensor datapath.

Line 420: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_CLAMP, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 421: `OP_SELECT_ZERO: lane_result = lane_a_zero_r` This statement continues through source line 422; the explanation uses the complete expression rather than only the first visual fragment. This is the OP_SELECT_ZERO case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_zero_r ? lane_b_r : 32'sd0;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_SELECT_ZERO, the machine performs the named control phase..

Line 422: `? lane_b_r : 32'sd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 421. Its exact fragment is '? lane_b_r : 32'sd0;'. The important operands are lane_b_r: one parallel arithmetic element within a vector or tensor datapath; sd0: the value named 'sd0' in this block. Read together, the complete operation is: This is the OP_SELECT_ZERO case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_zero_r ? lane_b_r : 32'sd0;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_SELECT_ZERO, the machine performs the named control phase..

Line 423: `OP_SELECT_NONZERO: lane_result = lane_a_zero_r` This statement continues through source line 424; the explanation uses the complete expression rather than only the first visual fragment. This is the OP_SELECT_NONZERO case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_zero_r ? 32'sd0 : lane_b_r;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_SELECT_NONZERO, the machine performs the named control phase..

Line 424: `? 32'sd0 : lane_b_r;` This is continuation line 2 of the 2-line logical statement that begins at source line 423. Its exact fragment is '? 32'sd0 : lane_b_r;'. The important operands are sd0: the value named 'sd0' in this block; lane_b_r: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This is the OP_SELECT_NONZERO case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_zero_r ? 32'sd0 : lane_b_r;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_SELECT_NONZERO, the machine performs the named control phase..

Line 425: `OP_COMPARE_GE: lane_result = lane_a_ge_b_r` This statement continues through source line 426; the explanation uses the complete expression rather than only the first visual fragment. This is the OP_COMPARE_GE case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_ge_b_r ? 32'sd1 : 32'sd0;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_COMPARE_GE, the machine performs the named control phase..

Line 426: `? 32'sd1 : 32'sd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 425. Its exact fragment is '? 32'sd1 : 32'sd0;'. The important operands are sd1: the value named 'sd1' in this block; sd0: the value named 'sd0' in this block. Read together, the complete operation is: This is the OP_COMPARE_GE case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = lane_a_ge_b_r ? 32'sd1 : 32'sd0;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_COMPARE_GE, the machine performs the named control phase..

Line 427: `default: lane_result = 32'sd0;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'lane_result = 32'sd0;'; context is the clocked process that updates registers only on a rising clock edge Within state OP_COMPARE_GE, the machine performs the named control phase..

Line 428: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state OP_COMPARE_GE, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 429: `// Keep the operation-select mux local to the scalar lane` Design intent: Keep the operation-select mux local to the scalar lane This comment anchors the surrounding logic in the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 430: `// pipeline. The following state moves this registered value` Design intent: pipeline. The following state moves this registered value This comment anchors the surrounding logic in the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 431: `// into the wide retained output vector, avoiding a routed` Design intent: into the wide retained output vector, avoiding a routed This comment anchors the surrounding logic in the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 432: `// comparison-to-distant-result path across the device.` Design intent: comparison-to-distant-result path across the device. This comment anchors the surrounding logic in the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

Line 433: `lane_result_r <= lane_result;` This makes lane_result_r, one parallel arithmetic element within a vector or tensor datapath, equal to ‘lane_result‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_result: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge.

Line 434: `state <= ST_STORE_RESULT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_STORE_RESULT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_STORE_RESULT: the value named ’st store result’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 435: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 437: `ST_STORE_RESULT: begin` This is the ST_STORE_RESULT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘begin‘; context is the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 438: `result[lane_index*DATA_BITS +: DATA_BITS] <= lane_result_r;` Within the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase., this exact statement contributes to the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives. The important operands are result: the value named ’result’ in this block; lane_index: an integer selecting one tensor element, lane, bank, or state entry; DATA_BITS: the value named ’data bits’ in this block.

Line 439: `if (lane_index == $clog2(LANES)'(LANES - 1)) begin` This condition selects the following hardware branch only when ‘lane_index == $clog2(LANES)’(LANES - 1)‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are lane_index: an integer selecting one tensor element, lane, bank, or state entry; clog2: the value named ’clog2’ in this block; LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 440: `state <= ST_OUTPUT;` This makes state, the finite-state-machine phase retained in flip-flops, equal to ‘ST_OUTPUT‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_OUTPUT: the value named ’st output’ in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 441: `end else begin` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 442: `lane_index <= lane_index + 1'b1;` This makes lane_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'lane_index + 1'b1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are lane_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named 'b1' in this block. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 443: `state <= ST_LOAD_LANE;` This makes state, the finite-state-machine phase retained in flip-flops, equal to 'ST_LOAD_LANE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are ST_LOAD_LANE: one parallel arithmetic element within a vector or tensor datapath. Context: the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase..

Line 444: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 445: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_STORE_RESULT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 447: `ST_OUTPUT: if (output_ready) state <= ST_IDLE;` This is the ST_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with 'if (output_ready) state <= ST_IDLE;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 448: `default: state <= ST_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'state <= ST_IDLE;'; context is the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 449: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge Within state ST_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 450: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 451: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 453: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 454: `if (LANES != 8) $fatal(1, "neuraedge_vector_alu currently requires LANES=8");` This condition selects the following hardware branch only when 'LANES != 8) $fatal(1, "neuraedge_vector_alu currently requires LANES=8"' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are LANES: one parallel arithmetic element within a vector or tensor datapath; fatal: the value named 'fatal' in this block; neuraedge_vector_alu: the value named 'neuraedge vector alu' in this block. Context: the module's structural declarations.

Line 455: `if (DATA_BITS != 32) $fatal(1, "neuraedge_vector_alu currently requires DATA_BITS=32");` This condition selects the following hardware branch only when 'DATA_BITS != 32) $fatal(1, "neuraedge_vector_alu currently requires DATA_BITS=32"' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are DATA_BITS: the value named 'data bits' in this block; fatal: the value named 'fatal' in this block; neuraedge_vector_alu: the value named 'neuraedge vector alu' in this block. Context: the module's structural declarations.

Line 456: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 457: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 459: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that executes lane-wise residual, activation, comparison, requantization, and pooling primitives.

23 rtl/neuraedge_weight_bank.sv

Role in the complete system: stores one 16-column by 2,048-entry weight tile in banked FPGA memory.

23.1 Beginner architecture map

Dataflow responsibility Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.

What becomes FPGA hardware Vivado elaborates module(s) neuraedge_weight_bank into physical nets and primitives. The source declares 6 input-port lines, 1 output-port lines, 1 clocked process(es), and 0 combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.

Cycle-level control This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls.

Connections to other blocks This source instantiates unique as case. Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection.

How to debug it In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.

23.2 Complete source

```
`timescale 1ns / 1ps
`default_nettype none

module neuraedge_weight_bank #(
    parameter int DEPTH = 256,
    parameter int ADDRESS_BITS = 8
) (
    input  wire clk,
    input  wire write_enable,
    input  wire [ADDRESS_BITS-1:0] write_address,
    input  wire [1:0] write_segment,
    input  wire [63:0] write_data,
    input  wire [ADDRESS_BITS-1:0] read_address,
    output logic [127:0] read_data
);
    // Four independent 64-bit memories map directly to BRAM primitives. The
    // segment write enable avoids the 3D partial-write pattern that Vivado
    // otherwise expands into half a million flip-flops.
    (* ram_style = "block" *) logic [63:0] memory0 [0:DEPTH-1];
    (* ram_style = "block" *) logic [63:0] memory1 [0:DEPTH-1];
    logic [127:0] memory_read_data_r;

    always_ff @(posedge clk) begin
        if (write_enable) begin
            unique case (write_segment)
                2'd0: memory0[write_address] <= write_data;
                2'd1: memory1[write_address] <= write_data;
                default: memory1[write_address] <= write_data;
            endcase
        end
        memory_read_data_r[63:0] <= memory0[read_address];
        memory_read_data_r[127:64] <= memory1[read_address];
        read_data <= memory_read_data_r;
    end

    initial begin
        if (DEPTH != (1 << ADDRESS_BITS)) $fatal(1, "weight bank depth/address mismatch");
    end
endmodule

`default_nettype wire
```

23.3 Line annotations

Line 1: `‘timescale 1ns / 1ps` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `‘default_nettype none` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module neuraedge_weight_bank #(` This opens neuraedge_weight_bank, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that stores one 16-column by 2,048-entry weight tile in banked FPGA memory.

Line 5: `parameter int DEPTH = 256,` This statement continues through source line 15; the explanation uses the complete expression rather than only the first visual fragment. This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 6: `parameter int ADDRESS_BITS = 8` This is continuation line 2 of the 11-line logical statement that begins at source line 5. Its exact fragment is ‘parameter int ADDRESS_BITS = 8‘. The important operands are parameter: the value named 'parameter' in this block; ADDRESS_BITS: the value named 'address bits' in this block. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 7: `) (` This is continuation line 3 of the 11-line logical statement that begins at source line 5. Its exact fragment is ‘) (‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 8: `input wire clk,` This is continuation line 4 of the 11-line logical statement that begins at source line 5. Its exact fragment is ‘input wire clk,‘. The important operands are input: the value named 'input' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 9: `input wire write_enable,` This is continuation line 5 of the 11-line logical statement that begins at source line 5. Its exact fragment is ‘input wire write_enable,‘. The important operands are input: the value named 'input' in this block; write_enable: a control bit that permits the associated state or storage update. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 10: `input wire [ADDRESS_BITS-1:0] write_address,` This is continuation line 6 of the 11-line logical statement that begins at source line 5. Its exact fragment is ‘input wire [ADDRESS_BITS-1:0] write_address,‘. The important operands are input: the value named 'input' in this block; ADDRESS_BITS: the value named 'address bits' in this block; write_address: the value named 'write address' in this block. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input

wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 11: `input wire [1:0] write_segment,` This is continuation line 7 of the 11-line logical statement that begins at source line 5. Its exact fragment is 'input wire [1:0] write_segment,'. The important operands are input: the value named 'input' in this block; write_segment: the value named 'write segment' in this block. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 12: `input wire [63:0] write_data,` This is continuation line 8 of the 11-line logical statement that begins at source line 5. Its exact fragment is 'input wire [63:0] write_data,'. The important operands are input: the value named 'input' in this block; write_data: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 13: `input wire [ADDRESS_BITS-1:0] read_address,` This is continuation line 9 of the 11-line logical statement that begins at source line 5. Its exact fragment is 'input wire [ADDRESS_BITS-1:0] read_address,'. The important operands are input: the value named 'input' in this block; ADDRESS_BITS: the value named 'address bits' in this block; read_address: the value named 'read address' in this block. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 14: `output logic [127:0] read_data` This is continuation line 10 of the 11-line logical statement that begins at source line 5. Its exact fragment is 'output logic [127:0] read_data'. The important operands are output: the value named 'output' in this block; read_data: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 15: `);` This is continuation line 11 of the 11-line logical statement that begins at source line 5. Its exact fragment is ');'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This fixes DEPTH to 256, parameter int ADDRESS_BITS = 8) (input wire clk, input wire write_enable, input wire [ADDRESS_BITS-1:0] write_address, input wire [1:0] write_segment, input wire [63:0] write_data, input wire [ADDRESS_BITS-1:0] read_address, output logic [127:0] read_data) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; DEPTH denotes the value named 'depth' in this block.

Line 16: `// Four independent 64-bit memories map directly to BRAM primitives. The` Design intent: Four independent 64-bit memories map directly to BRAM primitives. The This comment anchors the surrounding logic in the block that stores one 16-column by 2,048-entry weight tile in banked FPGA memory.

Line 17: `// segment write enable avoids the 3D partial-write pattern that Vivado` Design intent: segment write enable avoids the 3D partial-write pattern that Vivado This comment anchors the surrounding logic in the block that stores one 16-column by 2,048-entry weight tile in banked FPGA memory.

Line 18: `// otherwise expands into half a million flip-flops.` Design intent: otherwise expands into half a million flip-flops. This comment anchors the surrounding logic in the block that stores one 16-column by 2,048-entry weight tile in banked FPGA memory.

Line 19: `(* ram_style = "block" *) logic [63:0] memory0 [0:DEPTH-1];` Within the module's structural declarations, this exact statement contributes to the block that stores one 16-column by 2,048-entry weight tile in banked FPGA memory. The important operands are ram_style: the value named 'ram style' in this block; block: the value named 'block' in this block; memory0: local FPGA storage addressed by surrounding index logic.

Line 20: `(* ram_style = "block" *) logic [63:0] memory1 [0:DEPTH-1];` Within the module's structural declarations, this exact statement contributes to the block that stores one 16-column by 2,048-entry weight tile in banked FPGA memory. The important operands are ram_style: the value named 'ram style' in this block; block: the value named 'block' in this block; memory1: local FPGA storage addressed by surrounding index logic.

Line 21: `logic [127:0] memory_read_data_r;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, memory_read_data_r is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `always_ff @(posedge clk) begin` This opens clocked sequential logic. Everything assigned with '<=' inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state.

Line 24: `if (write_enable) begin` This condition selects the following hardware branch only when 'write_enable' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are write_enable: a control bit that permits the associated state or storage update. Context: the clocked process that updates registers only on a rising clock edge.

Line 25: `unique case (write_segment)` This starts a multiplexer/state decode selected by 'write_segment'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 26: `2'd0: memory0[write_address] <= write_data;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that stores one 16-column by 2,048-entry weight tile in banked FPGA memory. The important operands are d0: the value named 'd0' in this block; memory0: local FPGA storage addressed by surrounding index logic; write_address: the value named 'write address' in this block.

Line 27: `2'd1: memory1[write_address] <= write_data;` Within the clocked process that updates registers only on a rising clock edge, this exact statement contributes to the block that stores one 16-column by 2,048-entry weight tile in banked FPGA memory. The important operands are d1: the value named 'd1' in this block; memory1: local FPGA storage addressed by surrounding index logic; write_address: the value named 'write address' in this block.

Line 28: `default: memory1[write_address] <= write_data;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'memory1[write_address] <= write_data;'; context is the clocked process that updates registers only on a rising clock edge.

Line 29: `endcase` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 30: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 31: `memory_read_data_r[63:0] <= memory0[read_address];` This makes memory_read_data_r[63:0], local FPGA storage addressed by surrounding index logic, equal to 'memory0[read_address]'. It updates after the active clock edge together with other nonblocking assignments. The important operands are memory0: local FPGA storage addressed by surrounding index logic; read_address: the value named 'read address' in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 32: `memory_read_data_r[127:64] <= memory1[read_address];` This makes memory_read_data_r[127:64], local FPGA storage addressed by surrounding index logic, equal to ‘memory1[read_address]‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are memory1: local FPGA storage addressed by surrounding index logic; read_address: the value named ’read address’ in this block. Context: the clocked process that updates registers only on a rising clock edge.

Line 33: `read_data <= memory_read_data_r;` This makes read_data, the payload carried with the surrounding valid/ready handshake, equal to ‘memory_read_data_r‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are memory_read_data_r: local FPGA storage addressed by surrounding index logic. Context: the clocked process that updates registers only on a rising clock edge.

Line 34: `end` This structural keyword opens or closes the branch/process scope described as the clocked process that updates registers only on a rising clock edge; it has no arithmetic value of its own.

Line 36: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 37: `if (DEPTH != (1 << ADDRESS_BITS)) $fatal(1, "weight bank depth/address mismatch");` This condition selects the following hardware branch only when ‘DEPTH != (1 << ADDRESS_BITS)) $fatal(1, "weight bank depth/address mismatch"‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are DEPTH: the value named ’depth’ in this block; ADDRESS_BITS: the value named ’address bits’ in this block; fatal: the value named ’fatal’ in this block. Context: the module’s structural declarations.

Line 38: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 39: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 41: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that stores one 16-column by 2,048-entry weight tile in banked FPGA memory.

24 software/benchmark_compiler_memory.py

Role in the complete system: measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

24.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: none. Functions: benchmark, parse_arguments. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

24.2 Complete source

```
"""Measure compilation time and Python memory for one real exported projection."""

from __future__ import annotations

import argparse
import json
import platform
import time
import tracemalloc
from datetime import datetime, timezone
from pathlib import Path

import numpy as np

from neuraedge_compiler import compile_spike_gemm
from spikingformer_parameters import SpikingformerParameterArchive


def benchmark(manifest: Path, projection_name: str, rows: int) -> dict[str, object]:
    if rows <= 0:
        raise ValueError("rows must be positive")
    archive = SpikingformerParameterArchive(manifest)
    projection = archive.projection(projection_name)
    spikes = np.zeros((rows, projection.weight_k_by_n.shape[0]), dtype=np.uint8)
    tracemalloc.start()
    started = time.perf_counter()
    program = compile_spike_gemm(spikes, projection.weight_k_by_n)
    elapsed = time.perf_counter() - started
    current_bytes, peak_bytes = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    output_beats = sum(instruction.output_beats for instruction in program.instructions)
    descriptor_bytes = 24 * len(program.instructions)
    return {
        "format": "neuraedge-compiler-memory-benchmark-v1",
        "completed_utc": datetime.now(timezone.utc).isoformat(),
        "python": platform.python_version(),
        "numpy": np.__version__,
        "manifest": str(manifest.resolve()),
        "projection": projection_name,
        "shape": list(program.shape),
        "instructions": len(program.instructions),
        "payload_input_bytes": program.estimated_input_bytes,
        "descriptor_bytes": descriptor_bytes,
        "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes,
        "dma_output_bytes": output_beats * 8,
        "compile_seconds": elapsed,
        "tracemalloc_current_bytes": current_bytes,
        "tracemalloc_peak_bytes": peak_bytes,
    }


def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument("--manifest", required=True, type=Path)
    parser.add_argument("--projection", default="patch_embed.block3")
    parser.add_argument("--rows", type=int, default=1024)
    parser.add_argument("--output", type=Path)
    return parser.parse_args()


if __name__ == "__main__":
    arguments = parse_arguments()
    result = benchmark(arguments.manifest, arguments.projection, arguments.rows)
    payload = json.dumps(result, indent=2) + "\n"
    if arguments.output is not None:
        arguments.output.parent.mkdir(parents=True, exist_ok=True)
        arguments.output.write_text(payload, encoding="ascii")
    print(payload, end="")
```

24.3 Line annotations

Line 1: `"""Measure compilation time and Python memory for one real exported projection."""` This is executable documentation attached to the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import json` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `import platform` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `import time` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import tracemalloc` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `from datetime import datetime, timezone` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `from pathlib import Path` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `import numpy as np` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `from neuraedge_compiler import compile_spike_gemm` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `from spikingformer_parameters import SpikingformerParameterArchive` This imports the named Python API so the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `def benchmark(manifest: Path, projection_name: str, rows: int) -> dict[str, object]:` This starts function benchmark. Its indented body performs benchmark as part of the file that measures time, memory, command count, and byte traffic for the largest real Spikingformer projection; it runs only when called.

Line 20: `if rows <= 0:` This executes the following indented host-code branch only when ‘rows <= 0‘ is true. The important operands are rows: one output-row position processed by the tensor tile. Context: the function benchmark, which handles benchmark.

Line 21: `raise ValueError("rows must be positive")` This rejects an invalid state by raising ‘ValueError("rows must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 22: `archive = SpikingformerParameterArchive(manifest)` This assigns ‘SpikingformerParameterArchive(manifest)‘ to ‘archive ‘ in host memory. Here archive means the value named ’archive’ in this block. The important operands are SpikingformerParameterArchive: the value named ’spikingformer parameter archive’ in this block; manifest: the value named ’manifest’ in this block. Context: the function benchmark, which handles benchmark.

Line 23: `projection = archive.projection(projection_name)` This assigns ‘archive.projection(projection_name)‘ to ‘projection ‘ in host memory. Here projection means the value named ’projection’ in this block. The important operands are archive: the value named ’archive’ in this block; projection: the value named ’projection’ in this block; projection_name: the value named ’projection name’ in this block. Context: the function benchmark, which handles benchmark.

Line 24: `spikes = np.zeros((rows, projection.weight_k_by_n.shape[0]), dtype=np.uint8)` This assigns ‘np.zeros((rows projection.weight_k_by_n.shape[0]), dtype=np.uint8)‘ to ‘spikes ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are zeros: the value named ’zeros’ in this block; rows: one output-row position processed by the tensor tile; projection: the value named ’projection’ in this block. Context: the function benchmark, which handles benchmark.

Line 25: `tracemalloc.start()` This calls ‘tracemalloc.start‘ with “ inside the function benchmark, which handles benchmark. The call advances the host-side job that measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

Line 26: `started = time.perf_counter()` This assigns ‘time.perf_counter()‘ to ‘started ‘ in host memory. Here started means the value named ’started’ in this block. The important operands are time: the value named ’time’ in this block; perf_counter: a hardware telemetry register that accumulates measured events. Context: the function benchmark, which handles benchmark.

Line 27: `program = compile_spike_gemm(spikes, projection.weight_k_by_n)` This assigns ‘compile_spike_gemm(spikes, projection.weight_k_by_n)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold; spikes: a binary event produced when a neuron crosses its threshold; projection: the value named ’projection’ in this block. Context: the function benchmark, which handles benchmark.

Line 28: `elapsed = time.perf_counter() - started` This assigns ‘time.perf_counter() - started‘ to ‘elapsed ‘ in host memory. Here elapsed means the value named ’elapsed’ in this block. The important operands are time: the value named ’time’ in this block; perf_counter: a hardware telemetry register that accumulates measured events; started: the value named ’started’ in this block. Context: the function benchmark, which handles benchmark.

Line 29: `current_bytes, peak_bytes = tracemalloc.get_traced_memory()` This assigns ‘tracemalloc.get_traced_memory()
to ‘current_bytes, peak_bytes ‘ in host memory. Here current_bytes means the value named ’current bytes’ in this block.
The important operands are tracemalloc: the value named ’tracemalloc’ in this block; get_traced_memory: local FPGA
storage addressed by surrounding index logic. Context: the function benchmark, which handles benchmark.

Line 30: `tracemalloc.stop()` This calls ‘tracemalloc.stop‘ with “ inside the function benchmark, which handles
benchmark. The call advances the host-side job that measures time, memory, command count, and byte traffic for the
largest real Spikingformer projection.

Line 31: `output_beats = sum(instruction.output_beats for instruction in program.instructions)` This assigns ‘sum(instruction.output_beats for instruction in program.instructions)‘ to ‘output_beats ‘ in host memory. Here
output_beats means the value named ’output beats’ in this block. The important operands are sum: the value named
’sum’ in this block; instruction: the value named ’instruction’ in this block; output_beats: the value named ’output
beats’ in this block. Context: the function benchmark, which handles benchmark.

Line 32: `descriptor_bytes = 24 * len(program.instructions)` This assigns ‘24 * len(program.instructions)‘ to
‘descriptor_bytes ‘ in host memory. Here descriptor_bytes means a DMA record header plus one command and its
declared payload. The important operands are len: the value named ’len’ in this block; program: the value named
’program’ in this block; instructions: the value named ’instructions’ in this block. Context: the function benchmark,
which handles benchmark.

Line 33: `return {` This statement continues through source line 49; the explanation uses the complete expression
rather than only the first visual fragment. This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__,
"manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instruc
"payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes":
program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed,
"tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function
benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named
’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in
this block.

Line 34: `"format": "neuraedge-compiler-memory-benchmark-v1",` This is continuation line 2 of the 17-line logical statement that begins at source line 33. Its exact fragment is ‘"format": "neuraedge-compiler-memory-benchmark-v1",‘.
The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in
this block; compiler: the value named ’compiler’ in this block. Read together, the complete operation is: This returns
‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(),
"python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes
+ descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes":
current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge:
the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 35: `"completed_utc": datetime.now(timezone.utc).isoformat(),` This is continuation line 3 of the 17-line
logical statement that begins at source line 33. Its exact fragment is ‘"completed_utc": datetime.now(timezone.utc).isoformat(),‘.
The important operands are completed_utc: the value named ’completed utc’ in this block; datetime: the value named
’datetime’ in this block; now: the value named ’now’ in this block. Read together, the complete operation is: This returns
‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(),
"python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes
+ descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes":
current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles bench-

mark, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; compiler: the value named 'compiler' in this block.

Line 36: `"python": platform.python_version(),` This is continuation line 4 of the 17-line logical statement that begins at source line 33. Its exact fragment is '"python": platform.python_version(),'. The important operands are python: the value named 'python' in this block; platform: the value named 'platform' in this block; python_version: the value named 'python version' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_by output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }' to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; compiler: the value named 'compiler' in this block.

Line 37: `"numpy": np.__version__,` This is continuation line 5 of the 17-line logical statement that begins at source line 33. Its exact fragment is '"numpy": np.__version__,'. The important operands are numpy: the value named 'numpy' in this block; __version__: the value named 'version' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }' to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; compiler: the value named 'compiler' in this block.

Line 38: `"manifest": str(manifest.resolve()),` This is continuation line 6 of the 17-line logical statement that begins at source line 33. Its exact fragment is '"manifest": str(manifest.resolve()),'. The important operands are manifest: the value named 'manifest' in this block; str: the value named 'str' in this block; resolve: the value named 'resolve' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instruc "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }' to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; compiler: the value named 'compiler' in this block.

Line 39: `"projection": projection_name,` This is continuation line 7 of the 17-line logical statement that begins at source line 33. Its exact fragment is '"projection": projection_name,'. The important operands are projection: the value named 'projection' in this block; projection_name: the value named 'projection name' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }' to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; compiler: the value named 'compiler' in this block.

Line 40: `"shape": list(program.shape),` This is continuation line 8 of the 17-line logical statement that begins at source line 33. Its exact fragment is '"shape": list(program.shape),'. The important operands are shape: the value named 'shape' in this block; list: the value named 'list' in this block; program: the value named 'program' in this block.

Read together, the complete operation is: This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instruc "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 41: `"instructions": len(program.instructions),` This is continuation line 9 of the 17-line logical statement that begins at source line 33. Its exact fragment is ‘"instructions": len(program.instructions),‘. The important operands are instructions: the value named ’instructions’ in this block; len: the value named ’len’ in this block; program: the value named ’program’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 42: `"payload_input_bytes": program.estimated_input_bytes,` This is continuation line 10 of the 17-line logical statement that begins at source line 33. Its exact fragment is ‘"payload_input_bytes": program.estimated_input_bytes,‘. The important operands are payload_input_bytes: stream data belonging to the current command; program: the value named ’program’ in this block; estimated_input_bytes: the value named ’estimated input bytes’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instruc "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 43: `"descriptor_bytes": descriptor_bytes,` This is continuation line 11 of the 17-line logical statement that begins at source line 33. Its exact fragment is ‘"descriptor_bytes": descriptor_bytes,‘. The important operands are descriptor_bytes: a DMA record header plus one command and its declared payload. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.reso "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 44: `"dma_input_bytes": program.estimated_input_bytes + descriptor_bytes,` This is continuation line 12 of the 17-line logical statement that begins at source line 33. Its exact fragment is ‘"dma_input_bytes": program.estimated_input_bytes + descriptor_bytes,‘. The important operands are dma_input_bytes: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; program: the value named ’program’ in this block; estimated_input_bytes: the value named ’estimated input bytes’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.reso "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes":

program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 45: `"dma_output_bytes": output_beats * 8,` This is continuation line 13 of the 17-line logical statement that begins at source line 33. Its exact fragment is ‘"dma_output_bytes": output_beats * 8,‘. The important operands are dma_output_bytes: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; output_beats: the value named ’output beats’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 46: `"compile_seconds": elapsed,` This is continuation line 14 of the 17-line logical statement that begins at source line 33. Its exact fragment is ‘"compile_seconds": elapsed,‘. The important operands are compile_seconds: the value named ’compile seconds’ in this block; elapsed: the value named ’elapsed’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 47: `"tracemalloc_current_bytes": current_bytes,` This is continuation line 15 of the 17-line logical statement that begins at source line 33. Its exact fragment is ‘"tracemalloc_current_bytes": current_bytes,‘. The important operands are tracemalloc_current_bytes: the value named ’tracemalloc current bytes’ in this block; current_bytes: the value named ’current bytes’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 48: `"tracemalloc_peak_bytes": peak_bytes,` This is continuation line 16 of the 17-line logical statement that begins at source line 33. Its exact fragment is ‘"tracemalloc_peak_bytes": peak_bytes,‘. The important operands are tracemalloc_peak_bytes: the value named ’tracemalloc peak bytes’ in this block; peak_bytes: the value named ’peak bytes’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.__version__, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }‘ to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; compiler: the value named ’compiler’ in this block.

Line 49: `}` This is continuation line 17 of the 17-line logical statement that begins at source line 33. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns '{ "format": "neuraedge-compiler-memory-benchmark-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "python": platform.python_version(), "numpy": np.___version___, "manifest": str(manifest.resolve()), "projection": projection_name, "shape": list(program.shape), "instructions": len(program.instructions), "payload_input_bytes": program.estimated_input_bytes, "descriptor_bytes": descriptor_bytes, "dma_input_bytes": program.estimated_input_bytes + descriptor_bytes, "dma_output_bytes": output_beats * 8, "compile_seconds": elapsed, "tracemalloc_current_bytes": current_bytes, "tracemalloc_peak_bytes": peak_bytes, }' to the caller of the function benchmark, which handles benchmark, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; compiler: the value named 'compiler' in this block.

Line 52: `def parse_arguments() -> argparse.Namespace:` This starts function parse_arguments. Its indented body performs parse arguments as part of the file that measures time, memory, command count, and byte traffic for the largest real Spikingformer projection; it runs only when called.

Line 53: `parser = argparse.ArgumentParser()` This assigns 'argparse.ArgumentParser()' to 'parser ' in host memory. Here parser means the value named 'parser' in this block. The important operands are argparse: the value named 'argparse' in this block; ArgumentParser: the value named 'argument parser' in this block. Context: the function parse_arguments, which handles parse arguments.

Line 54: `parser.add_argument("-manifest", required=True, type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 55: `parser.add_argument("-projection", default="patch_embed.block3")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 56: `parser.add_argument("-rows", type=int, default=1024)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 57: `parser.add_argument("-output", type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 58: `return parser.parse_args()` This returns 'parser.parse_args()' to the caller of the function parse_arguments, which handles parse arguments, ending this invocation. The important operands are parser: the value named 'parser' in this block; parse_args: the value named 'parse args' in this block.

Line 61: `if __name__ == "__main__":` This executes the following indented host-code branch only when '___name___ == "___main___"' is true. The important operands are ___name___: the value named 'name' in this block; ___main___: the value named 'main' in this block. Context: the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

Line 62: `arguments = parse_arguments()` This assigns 'parse_arguments()' to 'arguments ' in host memory. Here arguments means the value named 'arguments' in this block. The important operands are parse_arguments: the value named 'parse arguments' in this block. Context: the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

Line 63: `result = benchmark(arguments.manifest, arguments.projection, arguments.rows)` This assigns 'benchmark(arguments.manifest, arguments.projection, arguments.rows)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are benchmark: the value named 'benchmark' in this block;

arguments: the value named 'arguments' in this block; manifest: the value named 'manifest' in this block. Context: the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

Line 64: `payload = json.dumps(result, indent=2) + "\n"` This assigns ‘json.dumps(result, indent=2) + "\n"‘ to ‘payload ‘ in host memory. Here payload means stream data belonging to the current command. The important operands are json: the value named 'json' in this block; dumps: the value named 'dumps' in this block; result: the value named 'result' in this block. Context: the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

Line 65: `if arguments.output is not None:` This executes the following indented host-code branch only when ‘arguments.output is not None‘ is true. The important operands are arguments: the value named 'arguments' in this block; output: the value named 'output' in this block; is: the value named 'is' in this block. Context: the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

Line 66: `arguments.output.parent.mkdir(parents=True, exist_ok=True)` This calls ‘arguments.output.parent.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection. The call advances the host-side job that measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

Line 67: `arguments.output.write_text(payload, encoding="ascii")` This calls ‘arguments.output.write_text‘ with ‘payload, encoding="ascii"‘ inside the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection. The call advances the host-side job that measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

Line 68: `print(payload, end="")` This calls ‘print‘ with ‘payload, end=""‘ inside the file whose role is to measures time, memory, command count, and byte traffic for the largest real Spikingformer projection. The call advances the host-side job that measures time, memory, command count, and byte traffic for the largest real Spikingformer projection.

25 software/evaluate_spikingformer.py

Role in the complete system: evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

25.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: RunningTensorStats, CalibrationObserver. Functions: evaluate, parse_arguments, from_tensor, update, as_dict, ___init___, ___call___, as_dict. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

25.2 Complete source

```
"""Evaluate the trained Spikingformer and capture hardware calibration data."""

from __future__ import annotations

import argparse
import csv
import json
import time
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import torch
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

from spikingformer_reference import load_checkpoint
from spikingformer_quantization import (
    prepare_hardware_emulation,
    quantize_cifar_input_u8,
)


CIFAR10_CLASSES = (
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck",
)
CIFAR10_MEAN = (0.4914, 0.4822, 0.4465)
CIFAR10_STD = (0.2470, 0.2435, 0.2616)


@dataclass
class RunningTensorStats:
    """Numerically stable aggregate statistics retained on the source device."""

    elements: int
    batches: int
    minimum: torch.Tensor
    maximum: torch.Tensor
    total: torch.Tensor
    total_squares: torch.Tensor
    nonzero: torch.Tensor
    is_spike: bool

    @classmethod
    def from_tensor(cls, value: torch.Tensor, is_spike: bool) -> "RunningTensorStats":
        detached = value.detach().float()
        return cls(
            elements=detached.numel(),
            batches=1,
            minimum=detached.amin(),
            maximum=detached.amax(),
```

```
            total=detached.sum(dtype=torch.float64),
            total_squares=detached.square().sum(dtype=torch.float64),
            nonzero=torch.count_nonzero(detached),
            is_spike=is_spike,
        )

    def update(self, value: torch.Tensor, is_spike: bool) -> None:
        if self.is_spike != is_spike:
            raise ValueError("a tensor changed between analog and spike semantics")
        detached = value.detach().float()
        self.elements += detached.numel()
        self.batches += 1
        self.minimum = torch.minimum(self.minimum, detached.amin())
        self.maximum = torch.maximum(self.maximum, detached.amax())
        self.total += detached.sum(dtype=torch.float64)
        self.total_squares += detached.square().sum(dtype=torch.float64)
        self.nonzero += torch.count_nonzero(detached)

    def as_dict(self) -> dict[str, int | float | bool]:
        mean = float((self.total / self.elements).cpu())
        second_moment = float((self.total_squares / self.elements).cpu())
        variance = max(0.0, second_moment - mean * mean)
        return {
            "elements": self.elements,
            "batches": self.batches,
            "minimum": float(self.minimum.cpu()),
            "maximum": float(self.maximum.cpu()),
            "mean": mean,
            "standard_deviation": variance ** 0.5,
            "nonzero_fraction": float((self.nonzero / self.elements).cpu()),
            "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()),
            "is_spike": self.is_spike,
        }


class CalibrationObserver:
    """Collect aggregate ranges and sparsity without retaining feature maps."""

    def __init__(self) -> None:
        self.stats: dict[str, RunningTensorStats] = {}

    def __call__(self, name: str, value: torch.Tensor, is_spike: bool) -> None:
        if name in self.stats:
            self.stats[name].update(value, is_spike)
        else:
            self.stats[name] = RunningTensorStats.from_tensor(value, is_spike)

    def as_dict(self) -> dict[str, dict[str, int | float | bool]]:
        return {name: item.as_dict() for name, item in self.stats.items()}


def evaluate(arguments: argparse.Namespace) -> dict[str, object]:
    output_dir = Path(arguments.output_dir).resolve()
    output_dir.mkdir(parents=True, exist_ok=True)
    device = torch.device(arguments.device)
    loaded = load_checkpoint(arguments.checkpoint, device=device)
    model = loaded.model
    quantization_report: dict[str, object] | None = None
    if arguments.hardware_emulation:
        model, hardware_report = prepare_hardware_emulation(
            model,
            fractional_bits=arguments.fractional_bits,
            weight_granularity=arguments.weight_granularity,
        )
        model = model.to(device)
        quantization_report = hardware_report.as_dict()
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),
    ])
    dataset = datasets.CIFAR10(
        root=str(Path(arguments.data_root).resolve()),
        train=False,
        transform=transform,
        download=arguments.download,
    )
    generator = torch.Generator().manual_seed(arguments.seed)
    loader = DataLoader(
        dataset,
        batch_size=arguments.batch_size,
        shuffle=False,
        num_workers=arguments.workers,
        pin_memory=device.type == "cuda",
        persistent_workers=arguments.workers > 0,
        generator=generator,
    )
    observer = CalibrationObserver() if arguments.collect_stats else None
```

```
    confusion = np.zeros((10, 10), dtype=np.int64)
    all_logits: list[np.ndarray] = []
    all_labels: list[np.ndarray] = []
    correct = 0
    samples = 0
    started = time.perf_counter()

    with torch.inference_mode():
        for batch_index, (image, label) in enumerate(loader):
            image = image.to(device, non_blocking=True)
            label = label.to(device, non_blocking=True)
            if arguments.hardware_emulation:
                image, _, _ = quantize_cifar_input_u8(image)
            logits = model(image, observer=observer)
            prediction = logits.argmax(dim=1)
            labels_cpu = label.cpu().numpy()
            predictions_cpu = prediction.cpu().numpy()
            np.add.at(confusion, (labels_cpu, predictions_cpu), 1)
            all_logits.append(logits.float().cpu().numpy())
            all_labels.append(labels_cpu)
            correct += int((prediction == label).sum().item())
            samples += label.numel()
            if arguments.log_interval and (batch_index + 1) % arguments.log_interval == 0:
                print(
                    f"batch={batch_index + 1}/{len(loader)} "
                    f"samples={samples} accuracy={100.0 * correct / samples:.4f}%",
                    flush=True,
                )

    elapsed = time.perf_counter() - started
    logits_array = np.concatenate(all_logits, axis=0)
    labels_array = np.concatenate(all_labels, axis=0)
    predictions_array = logits_array.argmax(axis=1)
    class_totals = confusion.sum(axis=1)
    class_correct = np.diag(confusion)
    class_accuracy = np.divide(
        class_correct,
        class_totals,
        out=np.zeros(10, dtype=np.float64),
        where=class_totals != 0,
    )
    accuracy = 100.0 * correct / samples

    np.savez_compressed(
        output_dir / "spikingformer_predictions.npz",
        logits=logits_array,
        labels=labels_array,
        predictions=predictions_array,
        confusion_matrix=confusion,
    )
    with (output_dir / "spikingformer_predictions.csv").open(
        "w", newline="", encoding="ascii") as handle:
        writer = csv.writer(handle)
        writer.writerow(("sample", "label", "prediction", "correct", *(
            f"logit_{name}" for name in CIFAR10_CLASSES)))
        for index, (label, prediction, logits) in enumerate(zip(
            labels_array, predictions_array, logits_array)):
            writer.writerow((
                index,
                int(label),
                int(prediction),
                int(label == prediction),
                *(f"{float(item):.9g}" for item in logits),
            ))

    report: dict[str, object] = {
        "checkpoint": str(loaded.checkpoint_path),
        "architecture": loaded.architecture,
        "checkpoint_epoch": loaded.epoch,
        "checkpoint_recorded_metric_percent": loaded.metric,
        "dataset": "CIFAR-10 official test split",
        "samples": samples,
        "correct": correct,
        "top1_accuracy_percent": accuracy,
        "batch_size": arguments.batch_size,
        "device": str(device),
        "elapsed_seconds": elapsed,
        "images_per_second": samples / elapsed,
        "normalization_mean": CIFAR10_MEAN,
        "normalization_std": CIFAR10_STD,
        "class_accuracy_percent": {
            name: 100.0 * float(class_accuracy[index])
            for index, name in enumerate(CIFAR10_CLASSES)
        },
        "confusion_matrix_rows_true_columns_predicted": confusion.tolist(),
        "activation_statistics": observer.as_dict() if observer else {},
        "hardware_emulation": quantization_report,
```

```
    }
    (output_dir / "spikingformer_evaluation.json").write_text(
        json.dumps(report, indent=2) + "\n", encoding="ascii")
    print(json.dumps({
        "samples": samples,
        "correct": correct,
        "top1_accuracy_percent": accuracy,
        "elapsed_seconds": elapsed,
        "images_per_second": samples / elapsed,
        "output_dir": str(output_dir),
    }, indent=2), flush=True)
    return report


def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument("--checkpoint", required=True)
    parser.add_argument("--data-root", required=True)
    parser.add_argument("--output-dir", required=True)
    parser.add_argument("--device", default="cuda")
    parser.add_argument("--batch-size", type=int, default=32)
    parser.add_argument("--workers", type=int, default=2)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--log-interval", type=int, default=20)
    parser.add_argument("--collect-stats", action="store_true")
    parser.add_argument("--hardware-emulation", action="store_true")
    parser.add_argument("--fractional-bits", type=int, default=10)
    parser.add_argument(
        "--weight-granularity", choices=("tensor", "channel"), default="tensor")
    parser.add_argument("--download", action="store_true")
    return parser.parse_args()


if __name__ == "__main__":
    evaluate(parse_arguments())
```

25.3 Line annotations

Line 1: `"""Evaluate the trained Spikingformer and capture hardware calibration data."""` This is executable documentation attached to the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import csv` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `import json` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `import time` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `from pathlib import Path` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `import numpy as np` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `import torch` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `from torch.utils.data import DataLoader` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `from torchvision import datasets, transforms` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `from spikingformer_reference import load_checkpoint` This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `from spikingformer_quantization import (` This statement continues through source line 21; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `prepare_hardware_emulation,` This is continuation line 2 of the 4-line logical statement that begins at source line 18. Its exact fragment is ‘prepare_hardware_emulation,‘. The important operands are prepare_hardware_emulation: the value named ’prepare hardware emulation’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `quantize_cifar_input_u8,` This is continuation line 3 of the 4-line logical statement that begins at source line 18. Its exact fragment is ‘quantize_cifar_input_u8,‘. The important operands are quantize_cifar_input_u8: the value named ’quantize cifar input u8’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 18. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `CIFAR10_CLASSES = (` This statement continues through source line 27; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘("airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck",)‘ to ‘CIFAR10_CLASSES ‘ in host memory. Here CIFAR10_CLASSES means the value named ’cifar10 classes’ in this block. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named ’automobile’ in this block; bird: the value named ’bird’ in this block. Context: the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 25: `"airplane", "automobile", "bird", "cat", "deer",` This is continuation line 2 of the 4-line logical statement that begins at source line 24. Its exact fragment is ‘"airplane", "automobile", "bird", "cat", "deer",‘. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named 'automobile' in this block; bird: the value named 'bird' in this block. Read together, the complete operation is: This assigns ‘("airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck",)‘ to ‘CIFAR10_CLASSES ‘ in host memory. Here CIFAR10_CLASSES means the value named 'cifar10 classes' in this block. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named 'automobile' in this block; bird: the value named 'bird' in this block. Context: the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 26: `"dog", "frog", "horse", "ship", "truck",` This is continuation line 3 of the 4-line logical statement that begins at source line 24. Its exact fragment is ‘"dog", "frog", "horse", "ship", "truck",‘. The important operands are dog: the value named 'dog' in this block; frog: the value named 'frog' in this block; horse: the value named 'horse' in this block. Read together, the complete operation is: This assigns ‘("airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck",)‘ to ‘CIFAR10_CLASSES ‘ in host memory. Here CIFAR10_CLASSES means the value named 'cifar10 classes' in this block. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named 'automobile' in this block; bird: the value named 'bird' in this block. Context: the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 27: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 24. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘("airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck",)‘ to ‘CIFAR10_CLASSES ‘ in host memory. Here CIFAR10_CLASSES means the value named 'cifar10 classes' in this block. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named 'automobile' in this block; bird: the value named 'bird' in this block. Context: the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 28: `CIFAR10_MEAN = (0.4914, 0.4822, 0.4465)` This assigns ‘(0.4914, 0.4822, 0.4465)‘ to ‘CIFAR10_MEAN ‘ in host memory. Here CIFAR10_MEAN means the arithmetic average used by normalization. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 29: `CIFAR10_STD = (0.2470, 0.2435, 0.2616)` This assigns ‘(0.2470, 0.2435, 0.2616)‘ to ‘CIFAR10_STD ‘ in host memory. Here CIFAR10_STD means the value named 'cifar10 std' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 32: `@dataclass` This decorator changes how the following class or function is constructed. ‘@dataclass‘ is applied at import time on the host and is part of the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set; it creates no FPGA circuit by itself.

Line 33: `class RunningTensorStats:` This defines class RunningTensorStats, which groups data and methods for running tensor stats. Instances exist in host memory and organize the flow that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 34: `"""Numerically stable aggregate statistics retained on the source device."""` This is executable documentation attached to the class RunningTensorStats, which handles running tensor stats. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 36: `elements: int` Inside the class RunningTensorStats, which handles running tensor stats, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are elements: the value named 'elements' in this block.

Line 37: `batches: int` Inside the class RunningTensorStats, which handles running tensor stats, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are batches: the value named 'batches' in this block.

Line 38: `minimum: torch.Tensor` Inside the class RunningTensorStats, which handles running tensor stats, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are minimum: the value named 'minimum' in this block; torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 39: `maximum: torch.Tensor` Inside the class RunningTensorStats, which handles running tensor stats, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are maximum: the value named 'maximum' in this block; torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 40: `total: torch.Tensor` Inside the class RunningTensorStats, which handles running tensor stats, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are total: the value named 'total' in this block; torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 41: `total_squares: torch.Tensor` Inside the class RunningTensorStats, which handles running tensor stats, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are total_squares: the value named 'total squares' in this block; torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 42: `nonzero: torch.Tensor` Inside the class RunningTensorStats, which handles running tensor stats, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are nonzero: the value named 'nonzero' in this block; torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 43: `is_spike: bool` Inside the class RunningTensorStats, which handles running tensor stats, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are is_spike: a binary event produced when a neuron crosses its threshold; bool: the value named 'bool' in this block.

Line 45: `@classmethod` This decorator changes how the following class or function is constructed. '@classmethod' is applied at import time on the host and is part of the class RunningTensorStats, which handles running tensor stats; it creates no FPGA circuit by itself.

Line 46: `def from_tensor(cls, value: torch.Tensor, is_spike: bool) -> "RunningTensorStats":` This starts function from_tensor. Its indented body performs from tensor as part of the file that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set; it runs only when called.

Line 47: `detached = value.detach().float()` This assigns 'value.detach().float()' to 'detached ' in host memory. Here detached means the value named 'detached' in this block. The important operands are value: the value named 'value' in this block; detach: the value named 'detach' in this block; float: the value named 'float' in this block. Context: the function from_tensor, which handles from tensor.

Line 48: `return cls(` This statement continues through source line 57; the explanation uses the complete expression rather than only the first visual fragment. This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.float64), total_squares=detached.square(). nonzero=torch.count_nonzero(detached), is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 49: `elements=detached.numel(),` This is continuation line 2 of the 10-line logical statement that begins at source line 48. Its exact fragment is 'elements=detached.numel(),'. The important operands are elements: the value named 'elements' in this block; detached: the value named 'detached' in this block; numel: the value named 'numel' in this block. Read together, the complete operation is: This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.float64), total_squares=detached.square(). nonzero=torch.count_nonzero(detached), is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 50: `batches=1,` This is continuation line 3 of the 10-line logical statement that begins at source line 48. Its exact fragment is 'batches=1,'. The important operands are batches: the value named 'batches' in this block. Read together, the complete operation is: This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.float64), total_squares=detached.square().sum(dtype=torch.float64), nonzero=torch.count_nonzero(detached), is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 51: `minimum=detached.amin(),` This is continuation line 4 of the 10-line logical statement that begins at source line 48. Its exact fragment is 'minimum=detached.amin(),'. The important operands are minimum: the value named 'minimum' in this block; detached: the value named 'detached' in this block; amin: the value named 'amin' in this block. Read together, the complete operation is: This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.float64), total_squares=detached.square(). nonzero=torch.count_nonzero(detached), is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 52: `maximum=detached.amax(),` This is continuation line 5 of the 10-line logical statement that begins at source line 48. Its exact fragment is 'maximum=detached.amax(),'. The important operands are maximum: the value named 'maximum' in this block; detached: the value named 'detached' in this block; amax: the value named 'amax' in this block. Read together, the complete operation is: This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.float64), total_squares=detached.square(). nonzero=torch.count_nonzero(detached), is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 53: `total=detached.sum(dtype=torch.float64),` This is continuation line 6 of the 10-line logical statement that begins at source line 48. Its exact fragment is 'total=detached.sum(dtype=torch.float64),'. The important operands are total: the value named 'total' in this block; detached: the value named 'detached' in this block; sum: the value named 'sum' in this block. Read together, the complete operation is: This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.float64), total_squares=detached.squa nonzero=torch.count_nonzero(detached), is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 54: `total_squares=detached.square().sum(dtype=torch.float64),` This is continuation line 7 of the 10-line logical statement that begins at source line 48. Its exact fragment is 'total_squares=detached.square().sum(dtype=torch.float64),'. The important operands are total_squares: the value named 'total squares' in this block; detached: the value named 'detached' in this block; square: the value named 'square' in this block. Read together, the complete operation is: This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.float64), total_squares=detached.square().sum(dtype=torch.float64), nonzero=torch.count_nonze is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 55: `nonzero=torch.count_nonzero(detached),` This is continuation line 8 of the 10-line logical statement that begins at source line 48. Its exact fragment is 'nonzero=torch.count_nonzero(detached),'. The important operands

are nonzero: the value named 'nonzero' in this block; torch: the value named 'torch' in this block; count_nonzero: the value named 'count nonzero' in this block. Read together, the complete operation is: This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.f total_squares=detached.square().sum(dtype=torch.float64), nonzero=torch.count_nonzero(detached), is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 56: `is_spike=is_spike,` This is continuation line 9 of the 10-line logical statement that begins at source line 48. Its exact fragment is 'is_spike=is_spike,'. The important operands are is_spike: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.float64), total_squares=detached.squa nonzero=torch.count_nonzero(detached), is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 57: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 48. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'cls(elements=detached.numel(), batches=1, minimum=detached.amin(), maximum=detached.amax(), total=detached.sum(dtype=torch.float64), total_squares=detached.square().sum(dtype=torch.float64), nonzero=torch.count_nonzero(detached), is_spike=is_spike,)' to the caller of the function from_tensor, which handles from tensor, ending this invocation. The important operands are cls: the value named 'cls' in this block; elements: the value named 'elements' in this block; detached: the value named 'detached' in this block.

Line 59: `def update(self, value: torch.Tensor, is_spike: bool) -> None:` This starts function update. Its indented body performs update as part of the file that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set; it runs only when called.

Line 60: `if self.is_spike != is_spike:` This executes the following indented host-code branch only when 'self.is_spike != is_spike' is true. The important operands are is_spike: a binary event produced when a neuron crosses its threshold. Context: the function update, which handles update.

Line 61: `raise ValueError("a tensor changed between analog and spike semantics")` This rejects an invalid state by raising 'ValueError("a tensor changed between analog and spike semantics")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 62: `detached = value.detach().float()` This assigns 'value.detach().float()' to 'detached ' in host memory. Here detached means the value named 'detached' in this block. The important operands are value: the value named 'value' in this block; detach: the value named 'detach' in this block; float: the value named 'float' in this block. Context: the function update, which handles update.

Line 63: `self.elements += detached.numel()` Inside the function update, which handles update, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are elements: the value named 'elements' in this block; detached: the value named 'detached' in this block; numel: the value named 'numel' in this block.

Line 64: `self.batches += 1` Inside the function update, which handles update, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are batches: the value named 'batches' in this block.

Line 65: `self.minimum = torch.minimum(self.minimum, detached.amin())` This assigns 'torch.minimum(self.minimum, detached.amin())' to 'self.minimum ' in host memory. Here minimum means the value named 'minimum' in this block. The important operands are torch: the value named 'torch' in this block; minimum: the value named 'minimum' in this block; detached: the value named 'detached' in this block. Context: the function update, which handles update.

Line 66: `self.maximum = torch.maximum(self.maximum, detached.amax())` This assigns ‘torch.maximum(self.maximum, detached.amax())‘ to ‘self.maximum ‘ in host memory. Here maximum means the value named ’maximum’ in this block. The important operands are torch: the value named ’torch’ in this block; maximum: the value named ’maximum’ in this block; detached: the value named ’detached’ in this block. Context: the function update, which handles update.

Line 67: `self.total += detached.sum(dtype=torch.float64)` Inside the function update, which handles update, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are total: the value named ’total’ in this block; detached: the value named ’detached’ in this block; sum: the value named ’sum’ in this block.

Line 68: `self.total_squares += detached.square().sum(dtype=torch.float64)` Inside the function update, which handles update, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are total_squares: the value named ’total squares’ in this block; detached: the value named ’detached’ in this block; square: the value named ’square’ in this block.

Line 69: `self.nonzero += torch.count_nonzero(detached)` Inside the function update, which handles update, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are nonzero: the value named ’nonzero’ in this block; torch: the value named ’torch’ in this block; count_nonzero: the value named ’count nonzero’ in this block.

Line 71: `def as_dict(self) -> dict[str, int | float | bool]:` This starts function as_dict. Its indented body performs as dict as part of the file that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set; it runs only when called.

Line 72: `mean = float((self.total / self.elements).cpu())` This assigns ‘float((self.total / self.elements).cpu())‘ to ‘mean ‘ in host memory. Here mean means the arithmetic average used by normalization. The important operands are float: the value named ’float’ in this block; total: the value named ’total’ in this block; elements: the value named ’elements’ in this block. Context: the function as_dict, which handles as dict.

Line 73: `second_moment = float((self.total_squares / self.elements).cpu())` This assigns ‘float((self.total_squares / self.elements).cpu())‘ to ‘second_moment ‘ in host memory. Here second_moment means the value named ’second moment’ in this block. The important operands are float: the value named ’float’ in this block; total_squares: the value named ’total squares’ in this block; elements: the value named ’elements’ in this block. Context: the function as_dict, which handles as dict.

Line 74: `variance = max(0.0, second_moment - mean * mean)` This assigns ‘max(0.0, second_moment - mean * mean)‘ to ‘variance ‘ in host memory. Here variance means the average squared distance from the mean. The important operands are max: the value named ’max’ in this block; second_moment: the value named ’second moment’ in this block; mean: the arithmetic average used by normalization. Context: the function as_dict, which handles as dict.

Line 75: `return {` This statement continues through source line 85; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }‘ to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named ’elements’ in this block; batches: the value named ’batches’ in this block; minimum: the value named ’minimum’ in this block.

Line 76: `"elements": self.elements,` This is continuation line 2 of the 11-line logical statement that begins at source line 75. Its exact fragment is ‘"elements": self.elements,‘. The important operands are elements: the value named ’elements’ in this block. Read together, the complete operation is: This returns ‘{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }‘ to the caller of the function as_dict, which

handles as dict, ending this invocation. The important operands are elements: the value named 'elements' in this block; batches: the value named 'batches' in this block; minimum: the value named 'minimum' in this block.

Line 77: `"batches": self.batches,` This is continuation line 3 of the 11-line logical statement that begins at source line 75. Its exact fragment is '"batches": self.batches,'. The important operands are batches: the value named 'batches' in this block. Read together, the complete operation is: This returns '{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }' to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named 'elements' in this block; batches: the value named 'batches' in this block; minimum: the value named 'minimum' in this block.

Line 78: `"minimum": float(self.minimum.cpu()),` This is continuation line 4 of the 11-line logical statement that begins at source line 75. Its exact fragment is '"minimum": float(self.minimum.cpu()),'. The important operands are minimum: the value named 'minimum' in this block; float: the value named 'float' in this block; cpu: the value named 'cpu' in this block. Read together, the complete operation is: This returns '{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }' to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named 'elements' in this block; batches: the value named 'batches' in this block; minimum: the value named 'minimum' in this block.

Line 79: `"maximum": float(self.maximum.cpu()),` This is continuation line 5 of the 11-line logical statement that begins at source line 75. Its exact fragment is '"maximum": float(self.maximum.cpu()),'. The important operands are maximum: the value named 'maximum' in this block; float: the value named 'float' in this block; cpu: the value named 'cpu' in this block. Read together, the complete operation is: This returns '{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }' to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named 'elements' in this block; batches: the value named 'batches' in this block; minimum: the value named 'minimum' in this block.

Line 80: `"mean": mean,` This is continuation line 6 of the 11-line logical statement that begins at source line 75. Its exact fragment is '"mean": mean,'. The important operands are mean: the arithmetic average used by normalization. Read together, the complete operation is: This returns '{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }' to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named 'elements' in this block; batches: the value named 'batches' in this block; minimum: the value named 'minimum' in this block.

Line 81: `"standard_deviation": variance ** 0.5,` This is continuation line 7 of the 11-line logical statement that begins at source line 75. Its exact fragment is '"standard_deviation": variance ** 0.5,'. The important operands are standard_deviation: the value named 'standard deviation' in this block; variance: the average squared distance from the mean. Read together, the complete operation is: This returns '{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }' to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named 'elements' in this block; batches: the value named 'batches' in this block; minimum: the value named 'minimum' in this block.

Line 82: `"nonzero_fraction": float((self.nonzero / self.elements).cpu()),` This is continuation line 8 of the 11-line logical statement that begins at source line 75. Its exact fragment is '"nonzero_fraction": float((self.nonzero / self.elements).cpu()),'. The important operands are nonzero_fraction: the value named 'nonzero fraction' in this block; float: the value named 'float' in this block; nonzero: the value named 'nonzero' in this block. Read together, the complete operation is: This returns '{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()),

"maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }‘ to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named ’elements’ in this block; batches: the value named ’batches’ in this block; minimum: the value named ’minimum’ in this block.

Line 83: `"zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()),` This is continuation line 9 of the 11-line logical statement that begins at source line 75. Its exact fragment is ‘"zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()),‘. The important operands are zero_fraction: the value named ’zero fraction’ in this block; float: the value named ’float’ in this block; nonzero: the value named ’nonzero’ in this block. Read together, the complete operation is: This returns ‘{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }‘ to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named ’elements’ in this block; batches: the value named ’batches’ in this block; minimum: the value named ’minimum’ in this block.

Line 84: `"is_spike": self.is_spike,` This is continuation line 10 of the 11-line logical statement that begins at source line 75. Its exact fragment is ‘"is_spike": self.is_spike,‘. The important operands are is_spike: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This returns ‘{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }‘ to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named ’elements’ in this block; batches: the value named ’batches’ in this block; minimum: the value named ’minimum’ in this block.

Line 85: `}` This is continuation line 11 of the 11-line logical statement that begins at source line 75. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘{ "elements": self.elements, "batches": self.batches, "minimum": float(self.minimum.cpu()), "maximum": float(self.maximum.cpu()), "mean": mean, "standard_deviation": variance ** 0.5, "nonzero_fraction": float((self.nonzero / self.elements).cpu()), "zero_fraction": 1.0 - float((self.nonzero / self.elements).cpu()), "is_spike": self.is_spike, }‘ to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are elements: the value named ’elements’ in this block; batches: the value named ’batches’ in this block; minimum: the value named ’minimum’ in this block.

Line 88: `class CalibrationObserver:` This defines class CalibrationObserver, which groups data and methods for calibration observer. Instances exist in host memory and organize the flow that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 89: `"""Collect aggregate ranges and sparsity without retaining feature maps."""` This is executable documentation attached to the class CalibrationObserver, which handles calibration observer. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 91: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set; it runs only when called.

Line 92: `self.stats: dict[str, RunningTensorStats] = {}` This assigns ‘{}‘ to ‘self.stats‘ in host memory. Here stats means the value named ’stats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 94: `def __call__(self, name: str, value: torch.Tensor, is_spike: bool) -> None:` This starts function ___call___. Its indented body performs call as part of the file that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set; it runs only when called.

Line 95: `if name in self.stats:` This executes the following indented host-code branch only when ‘name in self.stats‘ is true. The important operands are name: the value named ’name’ in this block; in: the value named ’in’ in this block; stats: the value named ’stats’ in this block. Context: the function ___call___, which handles call.

Line 96: `self.stats[name].update(value, is_spike)` Inside the function ___call___, which handles call, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are stats: the value named ’stats’ in this block; name: the value named ’name’ in this block; update: the value named ’update’ in this block.

Line 97: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function ___call___, which handles call.

Line 98: `self.stats[name] = RunningTensorStats.from_tensor(value, is_spike)` This assigns ‘RunningTensorStats.from_tensor(value, is_spike)‘ to ‘self.stats[name] ‘ in host memory. Here stats means the value named ’stats’ in this block. The important operands are RunningTensorStats: the value named ’running tensor stats’ in this block; from_tensor: the value named ’from tensor’ in this block; value: the value named ’value’ in this block. Context: the function ___call___, which handles call.

Line 100: `def as_dict(self) -> dict[str, dict[str, int | float | bool]]:` This starts function as_dict. Its indented body performs as dict as part of the file that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set; it runs only when called.

Line 101: `return {name: item.as_dict() for name, item in self.stats.items()}` This returns ‘{name: item.as_dict() for name, item in self.stats.items()}‘ to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are name: the value named ’name’ in this block; item: the value named ’item’ in this block; as_dict: the value named ’as dict’ in this block.

Line 104: `def evaluate(arguments: argparse.Namespace) -> dict[str, object]:` This starts function evaluate. Its indented body performs evaluate as part of the file that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set; it runs only when called.

Line 105: `output_dir = Path(arguments.output_dir).resolve()` This assigns ‘Path(arguments.output_dir).resolve()‘ to ‘output_dir ‘ in host memory. Here output_dir means the value named ’output dir’ in this block. The important operands are Path: the value named ’path’ in this block; arguments: the value named ’arguments’ in this block; output_dir: the value named ’output dir’ in this block. Context: the function evaluate, which handles evaluate.

Line 106: `output_dir.mkdir(parents=True, exist_ok=True)` This calls ‘output_dir.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 107: `device = torch.device(arguments.device)` This assigns ‘torch.device(arguments.device)‘ to ‘device ‘ in host memory. Here device means the value named ’device’ in this block. The important operands are torch: the value named ’torch’ in this block; device: the value named ’device’ in this block; arguments: the value named ’arguments’ in this block. Context: the function evaluate, which handles evaluate.

Line 108: `loaded = load_checkpoint(arguments.checkpoint, device=device)` This assigns ‘load_checkpoint(arguments.checkpoint, device=device)‘ to ‘loaded ‘ in host memory. Here loaded means the value named ’loaded’ in this block. The important operands are load_checkpoint: the value named ’load checkpoint’ in this block; arguments: the value named ’arguments’ in this block; checkpoint: the value named ’checkpoint’ in this block. Context: the function evaluate, which handles evaluate.

Line 109: `model = loaded.model` This assigns ‘loaded.model‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are loaded: the value named ’loaded’ in this block; model: the value named ’model’ in this block. Context: the function evaluate, which handles evaluate.

Line 110: `quantization_report: dict[str, object] | None = None` This assigns ‘None‘ to ‘quantization_report‘ in host memory. Here quantization_report means the value named ’quantization report’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function evaluate, which handles evaluate.

Line 111: `if arguments.hardware_emulation:` This executes the following indented host-code branch only when ‘arguments.hardware_emulation‘ is true. The important operands are arguments: the value named ’arguments’ in this block; hardware_emulation: the value named ’hardware emulation’ in this block. Context: the function evaluate, which handles evaluate.

Line 112: `model, hardware_report = prepare_hardware_emulation(` This statement continues through source line 116; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘prepare_hardware_emulation(model, fractional_bits=arguments.fractional_bits, weight_granularity=arguments.weight_granularity)‘ to ‘model, hardware_report ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are prepare_hardware_emulation: the value named ’prepare hardware emulation’ in this block; model: the value named ’model’ in this block; fractional_bits: the value named ’fractional bits’ in this block. Context: the function evaluate, which handles evaluate.

Line 113: `model,` This is continuation line 2 of the 5-line logical statement that begins at source line 112. Its exact fragment is ‘model,‘. The important operands are model: the value named ’model’ in this block. Read together, the complete operation is: This assigns ‘prepare_hardware_emulation(model, fractional_bits=arguments.fractional_bits, weight_granularity=arguments.weight_granularity,)‘ to ‘model, hardware_report ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are prepare_hardware_emulation: the value named ’prepare hardware emulation’ in this block; model: the value named ’model’ in this block; fractional_bits: the value named ’fractional bits’ in this block. Context: the function evaluate, which handles evaluate.

Line 114: `fractional_bits=arguments.fractional_bits,` This is continuation line 3 of the 5-line logical statement that begins at source line 112. Its exact fragment is ‘fractional_bits=arguments.fractional_bits,‘. The important operands are fractional_bits: the value named ’fractional bits’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This assigns ‘prepare_hardware_emulation(model, fractional_bits=arguments.fractional_bits, weight_granularity=arguments.weight_granularity,)‘ to ‘model, hardware_report ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are prepare_hardware_emulation: the value named ’prepare hardware emulation’ in this block; model: the value named ’model’ in this block; fractional_bits: the value named ’fractional bits’ in this block. Context: the function evaluate, which handles evaluate.

Line 115: `weight_granularity=arguments.weight_granularity,` This is continuation line 4 of the 5-line logical statement that begins at source line 112. Its exact fragment is ‘weight_granularity=arguments.weight_granularity,‘. The important operands are weight_granularity: a learned coefficient multiplied by an activation; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This assigns ‘prepare_hardware_emulation(model, fractional_bits=arguments.fractional_bits, weight_granularity=arguments.weight_granularity,)‘ to ‘model, hardware_report ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are prepare_hardware_emulation: the value named ’prepare hardware emulation’ in this block; model: the value named ’model’ in this block; fractional_bits: the value named ’fractional bits’ in this block. Context: the function evaluate, which handles evaluate.

Line 116: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 112. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘prepare_hardware_emulation(model, fractional_bits=arguments.fractional_bits, weight_granularity=arguments.weight_granularity,)‘ to ‘model, hardware_report ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are prepare_hardware_emulation: the value named ’prepare hardware emulation’ in this block; model: the value named ’model’ in this block; fractional_bits: the value named ’fractional bits’ in this block. Context: the function evaluate, which handles evaluate.

Line 117: `model = model.to(device)` This assigns ‘model.to(device)‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are model: the value named ’model’ in this block; to:

the value named 'to' in this block; device: the value named 'device' in this block. Context: the function evaluate, which handles evaluate.

Line 118: `quantization_report = hardware_report.as_dict()` This assigns 'hardware_report.as_dict()' to 'quantization_report ' in host memory. Here quantization_report means the value named 'quantization report' in this block. The important operands are hardware_report: the value named 'hardware report' in this block; as_dict: the value named 'as dict' in this block. Context: the function evaluate, which handles evaluate.

Line 119: `transform = transforms.Compose([` This statement continues through source line 122; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'transforms.Compose([transforms.ToTensor(), transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),])' to 'transform ' in host memory. Here transform means the value named 'transform' in this block. The important operands are transforms: the value named 'transforms' in this block; Compose: the value named 'compose' in this block; ToTensor: the value named 'to tensor' in this block. Context: the function evaluate, which handles evaluate.

Line 120: `transforms.ToTensor(),` This is continuation line 2 of the 4-line logical statement that begins at source line 119. Its exact fragment is 'transforms.ToTensor(),'. The important operands are transforms: the value named 'transforms' in this block; ToTensor: the value named 'to tensor' in this block. Read together, the complete operation is: This assigns 'transforms.Compose([transforms.ToTensor(), transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),])' to 'transform ' in host memory. Here transform means the value named 'transform' in this block. The important operands are transforms: the value named 'transforms' in this block; Compose: the value named 'compose' in this block; ToTensor: the value named 'to tensor' in this block. Context: the function evaluate, which handles evaluate.

Line 121: `transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),` This is continuation line 3 of the 4-line logical statement that begins at source line 119. Its exact fragment is 'transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),'. The important operands are transforms: the value named 'transforms' in this block; Normalize: the value named 'normalize' in this block; CIFAR10_MEAN: the arithmetic average used by normalization. Read together, the complete operation is: This assigns 'transforms.Compose([transforms.ToTensor(), transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),])' to 'transform ' in host memory. Here transform means the value named 'transform' in this block. The important operands are transforms: the value named 'transforms' in this block; Compose: the value named 'compose' in this block; ToTensor: the value named 'to tensor' in this block. Context: the function evaluate, which handles evaluate.

Line 122: `])` This is continuation line 4 of the 4-line logical statement that begins at source line 119. Its exact fragment is '])'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'transforms.Compose([transforms.ToTensor(), transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),])' to 'transform ' in host memory. Here transform means the value named 'transform' in this block. The important operands are transforms: the value named 'transforms' in this block; Compose: the value named 'compose' in this block; ToTensor: the value named 'to tensor' in this block. Context: the function evaluate, which handles evaluate.

Line 123: `dataset = datasets.CIFAR10(` This statement continues through source line 128; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'datasets.CIFAR10(root=str(Path(arguments.data_root train=False, transform=transform, download=arguments.download,)' to 'dataset ' in host memory. Here dataset means the value named 'dataset' in this block. The important operands are datasets: the value named 'datasets' in this block; CIFAR10: the value named 'cifar10' in this block; root: the value named 'root' in this block. Context: the function evaluate, which handles evaluate.

Line 124: `root=str(Path(arguments.data_root).resolve()),` This is continuation line 2 of the 6-line logical statement that begins at source line 123. Its exact fragment is 'root=str(Path(arguments.data_root).resolve()),'. The important operands are root: the value named 'root' in this block; str: the value named 'str' in this block; Path: the value named 'path' in this block. Read together, the complete operation is: This assigns 'datasets.CIFAR10(root=str(Path(arguments.data_root).resolve()), train=False, transform=transform, download=arguments.download,)' to 'dataset ' in host memory. Here dataset means the value named 'dataset' in this block. The important operands are datasets: the value named 'datasets' in this block; CIFAR10: the value named 'cifar10' in this block; root: the value named 'root' in this block. Context: the function evaluate, which handles evaluate.

Line 125: `train=False,` This is continuation line 3 of the 6-line logical statement that begins at source line 123. Its exact fragment is 'train=False,'. The important operands are train: the value named 'train' in this block. Read together, the complete operation is: This assigns 'datasets.CIFAR10(root=str(Path(arguments.data_root).resolve()), train=False, transform=transform, download=arguments.download,)' to 'dataset ' in host memory. Here dataset means the value named 'dataset' in this block. The important operands are datasets: the value named 'datasets' in this block; CIFAR10: the value named 'cifar10' in this block; root: the value named 'root' in this block. Context: the function evaluate, which handles evaluate.

Line 126: `transform=transform,` This is continuation line 4 of the 6-line logical statement that begins at source line 123. Its exact fragment is 'transform=transform,'. The important operands are transform: the value named 'transform' in this block. Read together, the complete operation is: This assigns 'datasets.CIFAR10(root=str(Path(arguments.data_root).resolve(train=False, transform=transform, download=arguments.download,)' to 'dataset ' in host memory. Here dataset means the value named 'dataset' in this block. The important operands are datasets: the value named 'datasets' in this block; CIFAR10: the value named 'cifar10' in this block; root: the value named 'root' in this block. Context: the function evaluate, which handles evaluate.

Line 127: `download=arguments.download,` This is continuation line 5 of the 6-line logical statement that begins at source line 123. Its exact fragment is 'download=arguments.download,'. The important operands are download: the value named 'download' in this block; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns 'datasets.CIFAR10(root=str(Path(arguments.data_root).resolve()), train=False, transform=transform, download=arguments.download,)' to 'dataset ' in host memory. Here dataset means the value named 'dataset' in this block. The important operands are datasets: the value named 'datasets' in this block; CIFAR10: the value named 'cifar10' in this block; root: the value named 'root' in this block. Context: the function evaluate, which handles evaluate.

Line 128: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 123. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'datasets.CIFAR10(root=str(Path(arguments.data_root).resolve()), train=False, transform=transform, download=arguments.download,)' to 'dataset ' in host memory. Here dataset means the value named 'dataset' in this block. The important operands are datasets: the value named 'datasets' in this block; CIFAR10: the value named 'cifar10' in this block; root: the value named 'root' in this block. Context: the function evaluate, which handles evaluate.

Line 129: `generator = torch.Generator().manual_seed(arguments.seed)` This assigns 'torch.Generator().manual_seed(arg to 'generator ' in host memory. Here generator means the value named 'generator' in this block. The important operands are torch: the value named 'torch' in this block; Generator: the value named 'generator' in this block; manual_seed: the value named 'manual seed' in this block. Context: the function evaluate, which handles evaluate.

Line 130: `loader = DataLoader(` This statement continues through source line 138; the explanation uses the complete expression rather than only the first visual fragment. Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are loader: the value named 'loader' in this block; DataLoader: the value named 'data loader' in this block; dataset: the value named 'dataset' in this block.

Line 131: `dataset,` This is continuation line 2 of the 9-line logical statement that begins at source line 130. Its exact fragment is 'dataset,'. The important operands are dataset: the value named 'dataset' in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are loader: the value named 'loader' in this block; DataLoader: the value named 'data loader' in this block; dataset: the value named 'dataset' in this block.

Line 132: `batch_size=arguments.batch_size,` This is continuation line 3 of the 9-line logical statement that begins at source line 130. Its exact fragment is 'batch_size=arguments.batch_size,'. The important operands are batch_size: the value named 'batch size' in this block; arguments: the value named 'arguments' in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important

operands are loader: the value named 'loader' in this block; DataLoader: the value named 'data loader' in this block; dataset: the value named 'dataset' in this block.

Line 133: `shuffle=False,` This is continuation line 4 of the 9-line logical statement that begins at source line 130. Its exact fragment is 'shuffle=False,'. The important operands are shuffle: the value named 'shuffle' in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are loader: the value named 'loader' in this block; DataLoader: the value named 'data loader' in this block; dataset: the value named 'dataset' in this block.

Line 134: `num_workers=arguments.workers,` This is continuation line 5 of the 9-line logical statement that begins at source line 130. Its exact fragment is 'num_workers=arguments.workers,'. The important operands are num_workers: the value named 'num workers' in this block; arguments: the value named 'arguments' in this block; workers: the value named 'workers' in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are loader: the value named 'loader' in this block; DataLoader: the value named 'data loader' in this block; dataset: the value named 'dataset' in this block.

Line 135: `pin_memory=device.type == "cuda",` This is continuation line 6 of the 9-line logical statement that begins at source line 130. Its exact fragment is 'pin_memory=device.type == "cuda",'. The important operands are pin_memory: local FPGA storage addressed by surrounding index logic; device: the value named 'device' in this block; type: the value named 'type' in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are loader: the value named 'loader' in this block; DataLoader: the value named 'data loader' in this block; dataset: the value named 'dataset' in this block.

Line 136: `persistent_workers=arguments.workers > 0,` This is continuation line 7 of the 9-line logical statement that begins at source line 130. Its exact fragment is 'persistent_workers=arguments.workers > 0,'. The important operands are persistent_workers: the value named 'persistent workers' in this block; arguments: the value named 'arguments' in this block; workers: the value named 'workers' in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are loader: the value named 'loader' in this block; DataLoader: the value named 'data loader' in this block; dataset: the value named 'dataset' in this block.

Line 137: `generator=generator,` This is continuation line 8 of the 9-line logical statement that begins at source line 130. Its exact fragment is 'generator=generator,'. The important operands are generator: the value named 'generator' in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are loader: the value named 'loader' in this block; DataLoader: the value named 'data loader' in this block; dataset: the value named 'dataset' in this block.

Line 138: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 130. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are loader: the value named 'loader' in this block; DataLoader: the value named 'data loader' in this block; dataset: the value named 'dataset' in this block.

Line 139: `observer = CalibrationObserver() if arguments.collect_stats else None` This assigns 'CalibrationObserver() if arguments.collect_stats else None' to 'observer ' in host memory. Here observer means the value named 'observer' in this block. The important operands are CalibrationObserver: the value named 'calibration observer' in this block; arguments: the value named 'arguments' in this block; collect_stats: the value named 'collect stats' in this block. Context: the function evaluate, which handles evaluate.

Line 140: `confusion = np.zeros((10, 10), dtype=np.int64)` This assigns ‘np.zeros((10, 10), dtype=np.int64)‘ to ‘confusion ‘ in host memory. Here confusion means the value named ’confusion’ in this block. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function evaluate, which handles evaluate.

Line 141: `all_logits: list[np.ndarray] = []` This assigns ‘[]‘ to ‘all_logits‘ in host memory. Here all_logits means the ten unnormalized classifier scores used for argmax. The expression is a literal or structural operator and has no hidden software state. Context: the function evaluate, which handles evaluate.

Line 142: `all_labels: list[np.ndarray] = []` This assigns ‘[]‘ to ‘all_labels‘ in host memory. Here all_labels means the value named ’all labels’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function evaluate, which handles evaluate.

Line 143: `correct = 0` This assigns ‘0‘ to ‘correct ‘ in host memory. Here correct means the value named ’correct’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function evaluate, which handles evaluate.

Line 144: `samples = 0` This assigns ‘0‘ to ‘samples ‘ in host memory. Here samples means the value named ’samples’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function evaluate, which handles evaluate.

Line 145: `started = time.perf_counter()` This assigns ‘time.perf_counter()‘ to ‘started ‘ in host memory. Here started means the value named ’started’ in this block. The important operands are time: the value named ’time’ in this block; perf_counter: a hardware telemetry register that accumulates measured events. Context: the function evaluate, which handles evaluate.

Line 147: `with torch.inference_mode():` This opens a managed resource scope for ‘torch.inference_mode()‘ so files, tracing, or temporary state are finalized even if the function evaluate, which handles evaluate raises an error.

Line 148: `for batch_index, (image, label) in enumerate(loader):` This iterates batch_index, (image, label) over ‘enumerate(loader)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function evaluate, which handles evaluate.

Line 149: `image = image.to(device, non_blocking=True)` This assigns ‘image.to(device, non_blocking=True)‘ to ‘image ‘ in host memory. Here image means the value named ’image’ in this block. The important operands are image: the value named ’image’ in this block; to: the value named ’to’ in this block; device: the value named ’device’ in this block. Context: the function evaluate, which handles evaluate.

Line 150: `label = label.to(device, non_blocking=True)` This assigns ‘label.to(device, non_blocking=True)‘ to ‘label ‘ in host memory. Here label means the value named ’label’ in this block. The important operands are label: the value named ’label’ in this block; to: the value named ’to’ in this block; device: the value named ’device’ in this block. Context: the function evaluate, which handles evaluate.

Line 151: `if arguments.hardware_emulation:` This executes the following indented host-code branch only when ‘arguments.hardware_emulation‘ is true. The important operands are arguments: the value named ’arguments’ in this block; hardware_emulation: the value named ’hardware emulation’ in this block. Context: the function evaluate, which handles evaluate.

Line 152: `image, _, _ = quantize_cifar_input_u8(image)` This assigns ‘quantize_cifar_input_u8(image)‘ to ‘image, _, _ ‘ in host memory. Here image means the value named ’image’ in this block. The important operands are quantize_cifar_input_u8: the value named ’quantize cifar input u8’ in this block; image: the value named ’image’ in this block. Context: the function evaluate, which handles evaluate.

Line 153: `logits = model(image, observer=observer)` This assigns ‘model(image, observer=observer)‘ to ‘logits ‘ in host memory. Here logits means the ten unnormalized classifier scores used for argmax. The important operands are model: the value named ’model’ in this block; image: the value named ’image’ in this block; observer: the value named ’observer’ in this block. Context: the function evaluate, which handles evaluate.

Line 154: `prediction = logits.argmax(dim=1)` This assigns ‘logits.argmax(dim=1)‘ to ‘prediction ‘ in host memory. Here prediction means the class index with the greatest classifier score. The important operands are logits: the ten unnormalized classifier scores used for argmax; argmax: the value named ’argmax’ in this block; dim: the value named ’dim’ in this block. Context: the function evaluate, which handles evaluate.

Line 155: `labels_cpu = label.cpu().numpy()` This assigns ‘label.cpu().numpy()‘ to ‘labels_cpu ‘ in host memory. Here labels_cpu means the value named ’labels cpu’ in this block. The important operands are label: the value named ’label’ in this block; cpu: the value named ’cpu’ in this block; numpy: the value named ’numpy’ in this block. Context: the function evaluate, which handles evaluate.

Line 156: `predictions_cpu = prediction.cpu().numpy()` This assigns ‘prediction.cpu().numpy()‘ to ‘predictions_cpu ‘ in host memory. Here predictions_cpu means the class index with the greatest classifier score. The important operands are prediction: the class index with the greatest classifier score; cpu: the value named ’cpu’ in this block; numpy: the value named ’numpy’ in this block. Context: the function evaluate, which handles evaluate.

Line 157: `np.add.at(confusion, (labels_cpu, predictions_cpu), 1)` This calls ‘np.add.at‘ with ‘confusion, (labels_cpu, predictions_cpu), 1‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 158: `all_logits.append(logits.float().cpu().numpy())` This calls ‘all_logits.append‘ with ‘logits.float().cpu().numpy inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 159: `all_labels.append(labels_cpu)` This calls ‘all_labels.append‘ with ‘labels_cpu‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 160: `correct += int((prediction == label).sum().item())` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are correct: the value named ’correct’ in this block; prediction: the class index with the greatest classifier score; label: the value named ’label’ in this block.

Line 161: `samples += label.numel()` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are samples: the value named ’samples’ in this block; label: the value named ’label’ in this block; numel: the value named ’numel’ in this block.

Line 162: `if arguments.log_interval and (batch_index + 1) % arguments.log_interval == 0:` This executes the following indented host-code branch only when ‘arguments.log_interval and (batch_index + 1) % arguments.log_interval == 0‘ is true. The important operands are arguments: the value named ’arguments’ in this block; log_interval: the value named ’log interval’ in this block; and: the value named ’and’ in this block. Context: the function evaluate, which handles evaluate.

Line 163: `print(` This statement continues through source line 167; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘print‘ with ‘ f"batch={batch_index + 1}/{len(loader)} " f"samples={samples} accuracy={100.0 * correct / samples:.4f}%", flush=True, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 164: `f"batch={batch_index + 1}/{len(loader)} "` This is continuation line 2 of the 5-line logical statement that begins at source line 163. Its exact fragment is ‘f"batch={batch_index + 1}/{len(loader)} "‘. The important operands are f: the value named ’f’ in this block; batch: the value named ’batch’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘print‘ with ‘ f"batch={batch_index + 1}/{len(loader)} " f"samples={samples} accuracy={100.0 * correct / samples:.4f}%", flush=True, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 165: `f"samples={samples} accuracy={100.0 * correct / samples:.4f}%",` This is continuation line 3 of the 5-line logical statement that begins at source line 163. Its exact fragment is ‘f"samples={samples} accuracy={100.0 * correct / samples:.4f}%",‘. The important operands are f: the value named ’f’ in this block; samples: the value named ’samples’ in this block; accuracy: the value named ’accuracy’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘ f"batch={batch_index + 1}/{len(loader)} " f"samples={samples} accuracy={100.0 * correct / samples:.4f}%", flush=True, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 166: `flush=True,` This is continuation line 4 of the 5-line logical statement that begins at source line 163. Its exact fragment is ‘flush=True,‘. The important operands are flush: the value named ’flush’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘ f"batch={batch_index + 1}/{len(loader)} " f"samples={samples} accuracy={100.0 * correct / samples:.4f}%", flush=True, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 167: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 163. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘print‘ with ‘ f"batch={batch_index + 1}/{len(loader)} " f"samples={samples} accuracy={100.0 * correct / samples:.4f}%", flush=True, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 169: `elapsed = time.perf_counter() - started` This assigns ‘time.perf_counter() - started‘ to ‘elapsed ‘ in host memory. Here elapsed means the value named ’elapsed’ in this block. The important operands are time: the value named ’time’ in this block; perf_counter: a hardware telemetry register that accumulates measured events; started: the value named ’started’ in this block. Context: the function evaluate, which handles evaluate.

Line 170: `logits_array = np.concatenate(all_logits, axis=0)` This assigns ‘np.concatenate(all_logits, axis=0)‘ to ‘logits_array ‘ in host memory. Here logits_array means the ten unnormalized classifier scores used for argmax. The important operands are concatenate: the value named ’concatenate’ in this block; all_logits: the ten unnormalized classifier scores used for argmax; axis: the value named ’axis’ in this block. Context: the function evaluate, which handles evaluate.

Line 171: `labels_array = np.concatenate(all_labels, axis=0)` This assigns ‘np.concatenate(all_labels, axis=0)‘ to ‘labels_array ‘ in host memory. Here labels_array means the value named ’labels array’ in this block. The important operands are concatenate: the value named ’concatenate’ in this block; all_labels: the value named ’all labels’ in this block; axis: the value named ’axis’ in this block. Context: the function evaluate, which handles evaluate.

Line 172: `predictions_array = logits_array.argmax(axis=1)` This assigns ‘logits_array.argmax(axis=1)‘ to ‘predictions_array ‘ in host memory. Here predictions_array means the class index with the greatest classifier score. The important operands are logits_array: the ten unnormalized classifier scores used for argmax; argmax: the value named ’argmax’ in this block; axis: the value named ’axis’ in this block. Context: the function evaluate, which handles evaluate.

Line 173: `class_totals = confusion.sum(axis=1)` This assigns ‘confusion.sum(axis=1)‘ to ‘class_totals ‘ in host memory. Here class_totals means the value named ’class totals’ in this block. The important operands are confusion: the value named ’confusion’ in this block; sum: the value named ’sum’ in this block; axis: the value named ’axis’ in this block. Context: the function evaluate, which handles evaluate.

Line 174: `class_correct = np.diag(confusion)` This assigns ‘np.diag(confusion)‘ to ‘class_correct ‘ in host memory. Here class_correct means the value named ’class correct’ in this block. The important operands are diag: the value named ’diag’ in this block; confusion: the value named ’confusion’ in this block. Context: the function evaluate, which handles evaluate.

Line 175: `class_accuracy = np.divide(` This statement continues through source line 180; the explanation uses the complete expression rather than only the first visual fragment. Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are class_accuracy: the value named ’class accuracy’ in this block; divide: the value named ’divide’ in this block; class_correct: the value named ’class correct’ in this block.

Line 176: `class_correct,` This is continuation line 2 of the 6-line logical statement that begins at source line 175. Its exact fragment is ‘class_correct,‘. The important operands are class_correct: the value named ’class correct’ in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are class_accuracy: the value named ’class accuracy’ in this block; divide: the value named ’divide’ in this block; class_correct: the value named ’class correct’ in this block.

Line 177: `class_totals,` This is continuation line 3 of the 6-line logical statement that begins at source line 175. Its exact fragment is ‘class_totals,‘. The important operands are class_totals: the value named ’class totals’ in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are class_accuracy: the value named ’class accuracy’ in this block; divide: the value named ’divide’ in this block; class_correct: the value named ’class correct’ in this block.

Line 178: `out=np.zeros(10, dtype=np.float64),` This is continuation line 4 of the 6-line logical statement that begins at source line 175. Its exact fragment is ‘out=np.zeros(10, dtype=np.float64),‘. The important operands are out: the value named ’out’ in this block; zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are class_accuracy: the value named ’class accuracy’ in this block; divide: the value named ’divide’ in this block; class_correct: the value named ’class correct’ in this block.

Line 179: `where=class_totals != 0,` This is continuation line 5 of the 6-line logical statement that begins at source line 175. Its exact fragment is ‘where=class_totals != 0,‘. The important operands are where: the value named ’where’ in this block; class_totals: the value named ’class totals’ in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are class_accuracy: the value named ’class accuracy’ in this block; divide: the value named ’divide’ in this block; class_correct: the value named ’class correct’ in this block.

Line 180: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 175. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are class_accuracy: the value named ’class accuracy’ in this block; divide: the value named ’divide’ in this block; class_correct: the value named ’class correct’ in this block.

Line 181: `accuracy = 100.0 * correct / samples` This assigns ‘100.0 * correct / samples‘ to ‘accuracy ‘ in host memory. Here accuracy means the value named ’accuracy’ in this block. The important operands are correct: the value named ’correct’ in this block; samples: the value named ’samples’ in this block. Context: the function evaluate, which handles evaluate.

Line 183: `np.savez_compressed(` This statement continues through source line 189; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.savez_compressed‘ with ‘ output_dir / "spikingformer_predictions.npz", logits=logits_array, labels=labels_array, predictions=predictions_array, confusion_matrix=confusion,

‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 184: `output_dir / "spikingformer_predictions.npz",` This is continuation line 2 of the 7-line logical statement that begins at source line 183. Its exact fragment is ‘output_dir / "spikingformer_predictions.npz",‘. The important operands are output_dir: the value named ’output dir’ in this block; spikingformer_predictions: the class index with the greatest classifier score; npz: the value named ’npz’ in this block. Read together, the complete operation is: This calls ‘np.savez_compressed‘ with ‘ output_dir / "spikingformer_predictions.npz", logits=logits_array, labels=labels_array, predictions=predictions_array, confusion_matrix=confusion, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 185: `logits=logits_array,` This is continuation line 3 of the 7-line logical statement that begins at source line 183. Its exact fragment is ‘logits=logits_array,‘. The important operands are logits: the ten unnormalized classifier scores used for argmax; logits_array: the ten unnormalized classifier scores used for argmax. Read together, the complete operation is: This calls ‘np.savez_compressed‘ with ‘ output_dir / "spikingformer_predictions.npz", logits=logits_array, labels=labels_array, predictions=predictions_array, confusion_matrix=confusion, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 186: `labels=labels_array,` This is continuation line 4 of the 7-line logical statement that begins at source line 183. Its exact fragment is ‘labels=labels_array,‘. The important operands are labels: the value named ’labels’ in this block; labels_array: the value named ’labels array’ in this block. Read together, the complete operation is: This calls ‘np.savez_compressed‘ with ‘ output_dir / "spikingformer_predictions.npz", logits=logits_array, labels=labels_array, predictions=predictions_array, confusion_matrix=confusion, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 187: `predictions=predictions_array,` This is continuation line 5 of the 7-line logical statement that begins at source line 183. Its exact fragment is ‘predictions=predictions_array,‘. The important operands are predictions: the class index with the greatest classifier score; predictions_array: the class index with the greatest classifier score. Read together, the complete operation is: This calls ‘np.savez_compressed‘ with ‘ output_dir / "spikingformer_predictions.npz", logits=logits_array, labels=labels_array, predictions=predictions_array, confusion_matrix=confusion, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 188: `confusion_matrix=confusion,` This is continuation line 6 of the 7-line logical statement that begins at source line 183. Its exact fragment is ‘confusion_matrix=confusion,‘. The important operands are confusion_matrix: the value named ’confusion matrix’ in this block; confusion: the value named ’confusion’ in this block. Read together, the complete operation is: This calls ‘np.savez_compressed‘ with ‘ output_dir / "spikingformer_predictions.npz", logits=logits_array, labels=labels_array, predictions=predictions_array, confusion_matrix=confusion, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 189: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 183. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘np.savez_compressed‘ with ‘ output_dir / "spikingformer_predictions.npz", logits=logits_array, labels=labels_array, predictions=predictions_array, confusion_matrix=confusion, ‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 190: `with (output_dir / "spikingformer_predictions.csv").open(` This opens a managed resource scope for ‘(output_dir / "spikingformer_predictions.csv").open‘ so files, tracing, or temporary state are finalized even if the function evaluate, which handles evaluate raises an error.

Line 191: `"w", newline="", encoding="ascii") as handle:` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are w: the value named 'w' in this block; newline: the value named 'newline' in this block; encoding: the value named 'encoding' in this block.

Line 192: `writer = csv.writer(handle)` This assigns ‘csv.writer(handle)‘ to ‘writer ‘ in host memory. Here writer means the value named 'writer' in this block. The important operands are csv: the value named 'csv' in this block; writer: the value named 'writer' in this block; handle: the value named 'handle' in this block. Context: the function evaluate, which handles evaluate.

Line 193: `writer.writerow(("sample", "label", "prediction", "correct", *(` This statement continues through source line 194; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘writer.writerow‘ with ‘("sample", "label", "prediction", "correct", *(f"logit_{name}" for name in CIFAR10_CLASSES))‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 194: `f"logit_{name}" for name in CIFAR10_CLASSES)))` This is continuation line 2 of the 2-line logical statement that begins at source line 193. Its exact fragment is ‘f"logit_{name}" for name in CIFAR10_CLASSES)))‘. The important operands are f: the value named 'f' in this block; logit_: the value named 'logit' in this block; name: the value named 'name' in this block. Read together, the complete operation is: This calls ‘writer.writerow‘ with ‘("sample", "label", "prediction", "correct", *(f"logit_{name}" for name in CIFAR10_CLASSES))‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 195: `for index, (label, prediction, logits) in enumerate(zip(` This line opens or closes a multi-line Python expression inside the function evaluate, which handles evaluate; the neighboring lines supply its arguments or elements.

Line 196: `labels_array, predictions_array, logits_array)):` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are labels_array: the value named 'labels array' in this block; predictions_array: the class index with the greatest classifier score; logits_array: the ten unnormalized classifier scores used for argmax.

Line 197: `writer.writerow((` This statement continues through source line 203; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘writer.writerow‘ with ‘(index, int(label), int(prediction), int(label == prediction), *(f"{float(item):.9g}" for item in logits),)‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 198: `index,` This is continuation line 2 of the 7-line logical statement that begins at source line 197. Its exact fragment is ‘index,‘. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘writer.writerow‘ with ‘(index, int(label), int(prediction), int(label == prediction), *(f"{float(item):.9g}" for item in logits),)‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 199: `int(label),` This is continuation line 3 of the 7-line logical statement that begins at source line 197. Its exact fragment is ‘int(label),‘. The important operands are label: the value named 'label' in this block. Read together, the complete operation is: This calls ‘writer.writerow‘ with ‘(index, int(label), int(prediction), int(label == prediction), *(f"{float(item):.9g}" for item in logits),)‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 200: `int(prediction),` This is continuation line 4 of the 7-line logical statement that begins at source line 197. Its exact fragment is ‘int(prediction),‘. The important operands are prediction: the class index with the greatest classifier score. Read together, the complete operation is: This calls ‘writer.writerow‘ with ‘(index, int(label), int(prediction), int(label == prediction), *(f"{float(item):.9g}" for item in logits),)‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 201: `int(label == prediction),` This is continuation line 5 of the 7-line logical statement that begins at source line 197. Its exact fragment is ‘int(label == prediction),‘. The important operands are label: the value named ’label’ in this block; prediction: the class index with the greatest classifier score. Read together, the complete operation is: This calls ‘writer.writerow‘ with ‘(index, int(label), int(prediction), int(label == prediction), *(f"{float(item):.9g}" for item in logits),)‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 202: `*(f"{float(item):.9g}" for item in logits),` This is continuation line 6 of the 7-line logical statement that begins at source line 197. Its exact fragment is ‘*(f"{float(item):.9g}" for item in logits),‘. The important operands are f: the value named ’f’ in this block; float: the value named ’float’ in this block; item: the value named ’item’ in this block. Read together, the complete operation is: This calls ‘writer.writerow‘ with ‘(index, int(label), int(prediction), int(label == prediction), *(f"{float(item):.9g}" for item in logits),)‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 203: `))` This is continuation line 7 of the 7-line logical statement that begins at source line 197. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘writer.writerow‘ with ‘(index, int(label), int(prediction), int(label == prediction), *(f"{float(item):.9g}" for item in logits),)‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 205: `report: dict[str, object] = {` This statement continues through source line 227; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_c confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 206: `"checkpoint": str(loaded.checkpoint_path),` This is continuation line 2 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"checkpoint": str(loaded.checkpoint_path),‘. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 207: `"architecture": loaded.architecture,` This is continuation line 3 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"architecture": loaded.architecture,‘. The important operands are architecture: the value named ’architecture’ in this block; loaded: the value named ’loaded’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 208: `"checkpoint_epoch": loaded.epoch,` This is continuation line 4 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"checkpoint_epoch": loaded.epoch,‘. The important operands are checkpoint_epoch: the value named ’checkpoint epoch’ in this block; loaded: the value named ’loaded’ in this block; epoch: the value named ’epoch’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 209: `"checkpoint_recorded_metric_percent": loaded.metric,` This is continuation line 5 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"checkpoint_recorded_metric_percent": loaded.metric,‘. The important operands are checkpoint_recorded_metric_percent: the value named ’checkpoint recorded metric percent’ in this block; loaded: the value named ’loaded’ in this block; metric: the value named ’metric’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 210: `"dataset": "CIFAR-10 official test split",` This is continuation line 6 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"dataset": "CIFAR-10 official test split",‘. The important operands are dataset: the value named ’dataset’ in this block; CIFAR: the value named ’cifar’ in this block; official: the value named ’official’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the

function evaluate, which handles evaluate.

Line 211: `"samples": samples,` This is continuation line 7 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"samples": samples,‘. The important operands are samples: the value named ’samples’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_c confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 212: `"correct": correct,` This is continuation line 8 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"correct": correct,‘. The important operands are correct: the value named ’correct’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_c confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 213: `"top1_accuracy_percent": accuracy,` This is continuation line 9 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"top1_accuracy_percent": accuracy,‘. The important operands are top1_accuracy_percent: the value named ’top1 accuracy percent’ in this block; accuracy: the value named ’accuracy’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_c confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 214: `"batch_size": arguments.batch_size,` This is continuation line 10 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"batch_size": arguments.batch_size,‘. The important operands are batch_size: the value named ’batch size’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_pred confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 215: `"device": str(device),` This is continuation line 11 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"device": str(device),‘. The important operands are device: the value named ’device’ in this block; str: the value named ’str’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 216: `"elapsed_seconds": elapsed,` This is continuation line 12 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"elapsed_seconds": elapsed,‘. The important operands are elapsed_seconds: the value named ’elapsed seconds’ in this block; elapsed: the value named ’elapsed’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 217: `"images_per_second": samples / elapsed,` This is continuation line 13 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"images_per_second": samples / elapsed,‘. The important operands are images_per_second: the value named ’images per second’ in this block; samples: the value named ’samples’ in this block; elapsed: the value named ’elapsed’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 218: `"normalization_mean": CIFAR10_MEAN,` This is continuation line 14 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"normalization_mean": CIFAR10_MEAN,‘. The important operands are normalization_mean: the arithmetic average used by normalization; CIFAR10_MEAN: the arithmetic average used by normalization. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_c confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 219: `"normalization_std": CIFAR10_STD,` This is continuation line 15 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"normalization_std": CIFAR10_STD,‘. The important operands are normalization_std: the value named ’normalization std’ in this block; CIFAR10_STD: the value named ’cifar10 std’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_c confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 220: `"class_accuracy_percent": {` This is continuation line 16 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘"class_accuracy_percent": {‘. The important operands are class_accuracy_percent: the value named ’class accuracy percent’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 221: `name: 100.0 * float(class_accuracy[index])` This is continuation line 17 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘name: 100.0 * float(class_accuracy[index])‘. The important operands are name: the value named ’name’ in this block; float: the value named ’float’ in this block; class_accuracy: the value named ’class accuracy’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 222: `for index, name in enumerate(CIFAR10_CLASSES)` This is continuation line 18 of the 23-line logical statement that begins at source line 205. Its exact fragment is ‘for index, name in enumerate(CIFAR10_CLASSES)‘. The important operands are for: the value named ’for’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; name: the value named ’name’ in this block. Read together, the complete operation is: This assigns ‘{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; loaded: the value named ’loaded’ in this block. Context: the function evaluate, which handles evaluate.

Line 223: `},` This is continuation line 19 of the 23-line logical statement that begins at source line 205. Its exact fragment is '},'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are checkpoint: the value named 'checkpoint' in this block; str: the value named 'str' in this block; loaded: the value named 'loaded' in this block. Context: the function evaluate, which handles evaluate.

Line 224: `"confusion_matrix_rows_true_columns_predicted": confusion.tolist(),` This is continuation line 20 of the 23-line logical statement that begins at source line 205. Its exact fragment is '"confusion_matrix_rows_true_columns_pred confusion.tolist(),'. The important operands are confusion_matrix_rows_true_columns_predicted: one output-row position processed by the tensor tile; confusion: the value named 'confusion' in this block; tolist: the value named 'tolist' in this block. Read together, the complete operation is: This assigns '{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_c confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are checkpoint: the value named 'checkpoint' in this block; str: the value named 'str' in this block; loaded: the value named 'loaded' in this block. Context: the function evaluate, which handles evaluate.

Line 225: `"activation_statistics": observer.as_dict() if observer else {},` This is continuation line 21 of the 23-line logical statement that begins at source line 205. Its exact fragment is '"activation_statistics": observer.as_dict() if observer else {},'. The important operands are activation_statistics: an input feature value consumed by a neuron or multiply-accumulate operation; observer: the value named 'observer' in this block; as_dict: the value named 'as dict' in this block. Read together, the complete operation is: This assigns '{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_ confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are checkpoint: the value named 'checkpoint' in this block; str: the value named 'str' in this block; loaded: the value named 'loaded' in this block. Context: the function evaluate, which handles evaluate.

Line 226: `"hardware_emulation": quantization_report,` This is continuation line 22 of the 23-line logical statement that begins at source line 205. Its exact fragment is '"hardware_emulation": quantization_report,'. The important operands are hardware_emulation: the value named 'hardware emulation' in this block; quantization_report: the value named 'quantization report' in this block. Read together, the complete operation is: This assigns '{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are checkpoint: the value named 'checkpoint' in this block; str: the value named 'str' in this block; loaded: the value named 'loaded' in this block. Context: the function evaluate, which handles evaluate.

Line 227: `}` This is continuation line 23 of the 23-line logical statement that begins at source line 205. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "checkpoint": str(loaded.checkpoint_path), "architecture": loaded.architecture, "checkpoint_epoch": loaded.epoch, "checkpoint_recorded_metric_percent": loaded.metric, "dataset": "CIFAR-10 official test split", "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "batch_size": arguments.batch_size, "device": str(device), "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "normalization_mean": CIFAR10_MEAN, "normalization_std": CIFAR10_STD, "class_accuracy_percent": { name: 100.0 * float(class_accuracy[index]) for index, name in enumerate(CIFAR10_CLASSES) }, "confusion_matrix_rows_true_columns_predicted": confusion.tolist(), "activation_statistics": observer.as_dict() if observer else {}, "hardware_emulation": quantization_report, }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are checkpoint: the value named 'checkpoint' in this block; str: the value named 'str' in this block; loaded: the value named 'loaded' in this block. Context: the function evaluate, which handles evaluate.

Line 228: `(output_dir / "spikingformer_evaluation.json").write_text(` This statement continues through source line 229; the explanation uses the complete expression rather than only the first visual fragment. Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are output_dir: the value named 'output dir' in this block; spikingformer_evaluation: the value named 'spikingformer evaluation' in this block; json: the value named 'json' in this block.

Line 229: `json.dumps(report, indent=2) + "\n", encoding="ascii")` This is continuation line 2 of the 2-line logical statement that begins at source line 228. Its exact fragment is 'json.dumps(report, indent=2) + "\n", encoding="ascii")'. The important operands are json: the value named 'json' in this block; dumps: the value named 'dumps' in this block; report: the value named 'report' in this block. Read together, the complete operation is: Inside the function evaluate, which handles evaluate, this statement advances the host implementation that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The important operands are output_dir: the value named 'output dir' in this block; spikingformer_evaluation: the value named 'spikingformer evaluation' in this block; json: the value named 'json' in this block.

Line 230: `print(json.dumps({` This statement continues through source line 237; the explanation uses the complete expression rather than only the first visual fragment. This calls 'print' with 'json.dumps({ "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "output_dir": str(output_dir), }, indent=2), flush=True' inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 231: `"samples": samples,` This is continuation line 2 of the 8-line logical statement that begins at source line 230. Its exact fragment is '"samples": samples,'. The important operands are samples: the value named 'samples' in this block. Read together, the complete operation is: This calls 'print' with 'json.dumps({ "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "output_dir": str(output_dir), }, indent=2), flush=True' inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 232: `"correct": correct,` This is continuation line 3 of the 8-line logical statement that begins at source line 230. Its exact fragment is '"correct": correct,'. The important operands are correct: the value named 'correct' in this block. Read together, the complete operation is: This calls 'print' with 'json.dumps({ "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "output_dir": str(output_dir), }, indent=2), flush=True' inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 233: `"top1_accuracy_percent": accuracy,` This is continuation line 4 of the 8-line logical statement that begins at source line 230. Its exact fragment is '"top1_accuracy_percent": accuracy,'. The important operands are top1_accuracy_percent: the value named 'top1 accuracy percent' in this block; accuracy: the value named 'accuracy' in this block. Read together, the complete operation is: This calls 'print' with 'json.dumps({ "samples": samples, "correct":

correct, "top1_accuracy_percent": accuracy, "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "output_dir": str(output_dir), }, indent=2), flush=True‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 234: `"elapsed_seconds": elapsed,` This is continuation line 5 of the 8-line logical statement that begins at source line 230. Its exact fragment is ‘"elapsed_seconds": elapsed,‘. The important operands are elapsed_seconds: the value named ’elapsed seconds’ in this block; elapsed: the value named ’elapsed’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "output_dir": str(output_dir), }, indent=2), flush=True‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 235: `"images_per_second": samples / elapsed,` This is continuation line 6 of the 8-line logical statement that begins at source line 230. Its exact fragment is ‘"images_per_second": samples / elapsed,‘. The important operands are images_per_second: the value named ’images per second’ in this block; samples: the value named ’samples’ in this block; elapsed: the value named ’elapsed’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "output_dir": str(output_dir), }, indent=2), flush=True‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 236: `"output_dir": str(output_dir),` This is continuation line 7 of the 8-line logical statement that begins at source line 230. Its exact fragment is ‘"output_dir": str(output_dir),‘. The important operands are output_dir: the value named ’output dir’ in this block; str: the value named ’str’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "output_dir": str(output_dir), }, indent=2), flush=True‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 237: `}, indent=2), flush=True)` This is continuation line 8 of the 8-line logical statement that begins at source line 230. Its exact fragment is ‘}, indent=2), flush=True)‘. The important operands are indent: the value named ’indent’ in this block; flush: the value named ’flush’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "samples": samples, "correct": correct, "top1_accuracy_percent": accuracy, "elapsed_seconds": elapsed, "images_per_second": samples / elapsed, "output_dir": str(output_dir), }, indent=2), flush=True‘ inside the function evaluate, which handles evaluate. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 238: `return report` This returns ‘report‘ to the caller of the function evaluate, which handles evaluate, ending this invocation. The important operands are report: the value named ’report’ in this block.

Line 241: `def parse_arguments() -> argparse.Namespace:` This starts function parse_arguments. Its indented body performs parse arguments as part of the file that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set; it runs only when called.

Line 242: `parser = argparse.ArgumentParser()` This assigns ‘argparse.ArgumentParser()‘ to ‘parser ‘ in host memory. Here parser means the value named ’parser’ in this block. The important operands are argparse: the value named ’argparse’ in this block; ArgumentParser: the value named ’argument parser’ in this block. Context: the function parse_arguments, which handles parse arguments.

Line 243: `parser.add_argument("-checkpoint", required=True)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 244: `parser.add_argument("-data-root", required=True)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 245: `parser.add_argument("-output-dir", required=True)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 246: `parser.add_argument("-device", default="cuda")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 247: `parser.add_argument("-batch-size", type=int, default=32)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 248: `parser.add_argument("-workers", type=int, default=2)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 249: `parser.add_argument("-seed", type=int, default=42)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 250: `parser.add_argument("-log-interval", type=int, default=20)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 251: `parser.add_argument("-collect-stats", action="store_true")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 252: `parser.add_argument("-hardware-emulation", action="store_true")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 253: `parser.add_argument("-fractional-bits", type=int, default=10)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 254: `parser.add_argument(` This statement continues through source line 255; the explanation uses the complete expression rather than only the first visual fragment. This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 255: `"-weight-granularity", choices=("tensor", "channel"), default="tensor")` This is continuation line 2 of the 2-line logical statement that begins at source line 254. Its exact fragment is ‘"–weight-granularity", choices=("tensor", "channel"), default="tensor")‘. The important operands are weight: a learned coefficient multiplied by an activation; granularity: the value named 'granularity' in this block; choices: the value named 'choices' in this block. Read together, the complete operation is: This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 256: `parser.add_argument("-download", action="store_true")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 257: `return parser.parse_args()` This returns ‘parser.parse_args()‘ to the caller of the function parse_arguments, which handles parse arguments, ending this invocation. The important operands are parser: the value named ’parser’ in this block; parse_args: the value named ’parse args’ in this block.

Line 260: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

Line 261: `evaluate(parse_arguments())` This calls ‘evaluate‘ with ‘parse_arguments()‘ inside the file whose role is to evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set. The call advances the host-side job that evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set.

26 software/export_cifar10_shards.py

Role in the complete system: writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

26.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: none. Functions: parse_arguments, sha256, write_split. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

26.2 Complete source

```
"""Download the official CIFAR-10 split and export board-friendly NPZ shards."""

from __future__ import annotations

import argparse
import hashlib
import json
import os
from datetime import datetime, timezone
from pathlib import Path

import numpy as np


def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument("--dataset-root", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--shard-size", type=int, default=1000)
    parser.add_argument("--no-download", action="store_true")
    return parser.parse_args()


def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


def write_split(
    dataset,
    split: str,
    output_dir: Path,
    shard_size: int,
) -> list[dict[str, object]]:
    images = np.asarray(dataset.data, dtype=np.uint8)
    labels = np.asarray(dataset.targets, dtype=np.uint8)
    if images.shape != (len(labels), 32, 32, 3):
        raise ValueError(f"unexpected official CIFAR-10 {split} shape {images.shape}")
    records: list[dict[str, object]] = []
    for shard_index, start in enumerate(range(0, len(labels), shard_size)):
        stop = min(start + shard_size, len(labels))
        target = output_dir / f"cifar10_{split}_{shard_index:03d}.npz"
        temporary = target.with_name(target.name + ".tmp")
        with temporary.open("wb") as stream:
            np.savez_compressed(stream, images=images[start:stop], labels=labels[start:stop])
            stream.flush()
            os.fsync(stream.fileno())
        os.replace(temporary, target)
        records.append({
```

```
            "file": target.name,
            "samples": stop - start,
            "first_index": start,
            "last_index_exclusive": stop,
            "bytes": target.stat().st_size,
            "sha256": sha256(target),
        })
    return records


if __name__ == "__main__":
    arguments = parse_arguments()
    if arguments.shard_size <= 0:
        raise ValueError("shard-size must be positive")
    try:
        from torchvision.datasets import CIFAR10
    except ImportError as error:
        raise RuntimeError("torchvision is required to export official CIFAR-10") from error
    output_dir = arguments.output_dir.expanduser().resolve()
    output_dir.mkdir(parents=True, exist_ok=True)
    dataset_root = arguments.dataset_root.expanduser().resolve()
    train = CIFAR10(
        root=str(dataset_root), train=True, download=not arguments.no_download)
    test = CIFAR10(
        root=str(dataset_root), train=False, download=not arguments.no_download)
    manifest = {
        "format": "neuraedge-official-cifar10-shards-v1",
        "created_utc": datetime.now(timezone.utc).isoformat(),
        "source": "torchvision.datasets.CIFAR10 official Python archive",
        "image_encoding": "uint8 NHWC RGB",
        "label_encoding": "uint8 class index 0..9",
        "shard_size": arguments.shard_size,
        "splits": {
            "train": write_split(train, "train", output_dir, arguments.shard_size),
            "test": write_split(test, "test", output_dir, arguments.shard_size),
        },
    }
    manifest_path = output_dir / "cifar10_shards_manifest.json"
    temporary_manifest = manifest_path.with_name(manifest_path.name + ".tmp")
    temporary_manifest.write_text(
        json.dumps(manifest, indent=2) + "\n", encoding="ascii")
    os.replace(temporary_manifest, manifest_path)
    print(json.dumps({
        "manifest": str(manifest_path),
        "train_samples": sum(item["samples"] for item in manifest["splits"]["train"]),
        "test_samples": sum(item["samples"] for item in manifest["splits"]["test"]),
    }, indent=2))
```

26.3 Line annotations

Line 1: `"""Download the official CIFAR-10 split and export board-friendly NPZ shards."""` This is executable documentation attached to the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import hashlib` This imports the named Python API so the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `import json` This imports the named Python API so the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `import os` This imports the named Python API so the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `from datetime import datetime, timezone` This imports the named Python API so the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `from pathlib import Path` This imports the named Python API so the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `import numpy as np` This imports the named Python API so the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `def parse_arguments() -> argparse.Namespace:` This starts function parse_arguments. Its indented body performs parse arguments as part of the file that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning; it runs only when called.

Line 16: `parser = argparse.ArgumentParser()` This assigns ‘argparse.ArgumentParser()‘ to ‘parser ‘ in host memory. Here parser means the value named ’parser’ in this block. The important operands are argparse: the value named ’argparse’ in this block; ArgumentParser: the value named ’argument parser’ in this block. Context: the function parse_arguments, which handles parse arguments.

Line 17: `parser.add_argument("-dataset-root", type=Path, required=True)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 18: `parser.add_argument("-output-dir", type=Path, required=True)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 19: `parser.add_argument("-shard-size", type=int, default=1000)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 20: `parser.add_argument("-no-download", action="store_true")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 21: `return parser.parse_args()` This returns ‘parser.parse_args()‘ to the caller of the function parse_arguments, which handles parse arguments, ending this invocation. The important operands are parser: the value named ’parser’ in this block; parse_args: the value named ’parse args’ in this block.

Line 24: `def sha256(path: Path) -> str:` This starts function sha256. Its indented body performs sha256 as part of the file that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning; it runs only when called.

Line 25: `digest = hashlib.sha256()` This assigns ‘hashlib.sha256()‘ to ‘digest ‘ in host memory. Here digest means the value named ’digest’ in this block. The important operands are hashlib: the value named ’hashlib’ in this block; sha256: the value named ’sha256’ in this block. Context: the function sha256, which handles sha256.

Line 26: `with path.open("rb") as stream:` This opens a managed resource scope for ‘path.open("rb") as stream‘ so files, tracing, or temporary state are finalized even if the function sha256, which handles sha256 raises an error.

Line 27: `for chunk in iter(lambda: stream.read(1 << 20), b""):` This iterates chunk over ‘iter(lambda: stream.read(1 << 20), b"")‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function sha256, which handles sha256.

Line 28: `digest.update(chunk)` This calls ‘digest.update‘ with ‘chunk‘ inside the function sha256, which handles sha256. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 29: `return digest.hexdigest()` This returns ‘digest.hexdigest()‘ to the caller of the function sha256, which handles sha256, ending this invocation. The important operands are digest: the value named ’digest’ in this block; hexdigest: the value named ’hexdigest’ in this block.

Line 32: `def write_split(` This starts function write_split. Its indented body performs write split as part of the file that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning; it runs only when called.

Line 33: `dataset,` Inside the function write_split, which handles write split, this statement advances the host implementation that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The important operands are dataset: the value named ’dataset’ in this block.

Line 34: `split: str,` Inside the function write_split, which handles write split, this statement advances the host implementation that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The important operands are split: the value named ’split’ in this block; str: the value named ’str’ in this block.

Line 35: `output_dir: Path,` Inside the function write_split, which handles write split, this statement advances the host implementation that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The important operands are output_dir: the value named ’output dir’ in this block; Path: the value named ’path’ in this block.

Line 36: `shard_size: int,` Inside the function write_split, which handles write split, this statement advances the host implementation that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The important operands are shard_size: the value named ’shard size’ in this block.

Line 37: `) -> list[dict[str, object]]:` Inside the function write_split, which handles write split, this statement advances the host implementation that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The important operands are list: the value named ’list’ in this block; dict: the value named ’dict’ in this block; str: the value named ’str’ in this block.

Line 38: `images = np.asarray(dataset.data, dtype=np.uint8)` This assigns ‘np.asarray(dataset.data, dtype=np.uint8)‘ to ‘images ‘ in host memory. Here images means the value named ’images’ in this block. The important operands are asarray: the value named ’asarray’ in this block; dataset: the value named ’dataset’ in this block; data: the value named ’data’ in this block. Context: the function write_split, which handles write split.

Line 39: `labels = np.asarray(dataset.targets, dtype=np.uint8)` This assigns ‘np.asarray(dataset.targets, dtype=np.uint8 to ‘labels ‘ in host memory. Here labels means the value named ’labels’ in this block. The important operands are asarray: the value named ’asarray’ in this block; dataset: the value named ’dataset’ in this block; targets: the value named ’targets’ in this block. Context: the function write_split, which handles write split.

Line 40: `if images.shape != (len(labels), 32, 32, 3):` This executes the following indented host-code branch only when ‘images.shape != (len(labels), 32, 32, 3)‘ is true. The important operands are images: the value named ’images’ in this block; shape: the value named ’shape’ in this block; len: the value named ’len’ in this block. Context: the function write_split, which handles write split.

Line 41: `raise ValueError(f"unexpected official CIFAR-10 {split} shape {images.shape}")` This rejects an invalid state by raising ‘ValueError(f"unexpected official CIFAR-10 {split} shape {images.shape}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 42: `records: list[dict[str, object]] = []` This assigns ‘[]‘ to ‘records‘ in host memory. Here records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the function write_split, which handles write split.

Line 43: `for shard_index, start in enumerate(range(0, len(labels), shard_size)):` This iterates shard_index, start over ‘enumerate(range(0, len(labels), shard_size))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function write_split, which handles write split.

Line 44: `stop = min(start + shard_size, len(labels))` This assigns ‘min(start + shard_size, len(labels))‘ to ‘stop ‘ in host memory. Here stop means the value named ’stop’ in this block. The important operands are min: the value named ’min’ in this block; start: the value named ’start’ in this block; shard_size: the value named ’shard size’ in this block. Context: the function write_split, which handles write split.

Line 45: `target = output_dir / f"cifar10_{split}_{shard_index:03d}.npz"` This assigns ‘output_dir / f"cifar10_{split}_ to ‘target ‘ in host memory. Here target means the value named ’target’ in this block. The important operands are output_dir: the value named ’output dir’ in this block; f: the value named ’f’ in this block; cifar10_: the value named ’cifar10’ in this block. Context: the function write_split, which handles write split.

Line 46: `temporary = target.with_name(target.name + ".tmp")` This assigns ‘target.with_name(target.name + ".tmp")‘ to ‘temporary ‘ in host memory. Here temporary means the value named ’temporary’ in this block. The important operands are target: the value named ’target’ in this block; with_name: the value named ’with name’ in this block; name: the value named ’name’ in this block. Context: the function write_split, which handles write split.

Line 47: `with temporary.open("wb") as stream:` This opens a managed resource scope for ‘temporary.open("wb") as stream‘ so files, tracing, or temporary state are finalized even if the function write_split, which handles write split raises an error.

Line 48: `np.savez_compressed(stream, images=images[start:stop], labels=labels[start:stop])` This calls ‘np.savez_compressed‘ with ‘stream, images=images[start:stop], labels=labels[start:stop]‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 49: `stream.flush()` This calls ‘stream.flush‘ with “ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 50: `os.fsync(stream.fileno())` This calls ‘os.fsync‘ with ‘stream.fileno()‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 51: `os.replace(temporary, target)` This calls ‘os.replace‘ with ‘temporary, target‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 52: `records.append({` This statement continues through source line 59; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘records.append‘ with ‘{ "file": target.name, "samples": stop - start, "first_index": start, "last_index_exclusive": stop, "bytes": target.stat().st_size, "sha256": sha256(target), }‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 53: `"file": target.name,` This is continuation line 2 of the 8-line logical statement that begins at source line 52. Its exact fragment is ‘"file": target.name,‘. The important operands are file: the value named ’file’ in this block; target: the value named ’target’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘{ "file": target.name, "samples": stop - start, "first_index": start, "last_index_exclusive": stop, "bytes": target.stat().st_size, "sha256": sha256(target), }‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 54: `"samples": stop - start,` This is continuation line 3 of the 8-line logical statement that begins at source line 52. Its exact fragment is ‘"samples": stop - start,‘. The important operands are samples: the value named ’samples’ in this block; stop: the value named ’stop’ in this block; start: the value named ’start’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘{ "file": target.name, "samples": stop - start, "first_index": start, "last_index_exclusive": stop, "bytes": target.stat().st_size, "sha256": sha256(target), }‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 55: `"first_index": start,` This is continuation line 4 of the 8-line logical statement that begins at source line 52. Its exact fragment is ‘"first_index": start,‘. The important operands are first_index: an integer selecting one tensor element, lane, bank, or state entry; start: the value named ’start’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘{ "file": target.name, "samples": stop - start, "first_index": start, "last_index_exclusive": stop, "bytes": target.stat().st_size, "sha256": sha256(target), }‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 56: `"last_index_exclusive": stop,` This is continuation line 5 of the 8-line logical statement that begins at source line 52. Its exact fragment is ‘"last_index_exclusive": stop,‘. The important operands are last_index_exclusive: the AXI-Stream marker for the final beat of a packet or command payload; stop: the value named ’stop’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘{ "file": target.name, "samples": stop - start, "first_index": start, "last_index_exclusive": stop, "bytes": target.stat().st_size, "sha256": sha256(target), }‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 57: `"bytes": target.stat().st_size,` This is continuation line 6 of the 8-line logical statement that begins at source line 52. Its exact fragment is ‘"bytes": target.stat().st_size,‘. The important operands are bytes: the value named ’bytes’ in this block; target: the value named ’target’ in this block; stat: the value named ’stat’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘{ "file": target.name, "samples": stop - start, "first_index": start, "last_index_exclusive": stop, "bytes": target.stat().st_size, "sha256": sha256(target), }‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 58: `"sha256": sha256(target),` This is continuation line 7 of the 8-line logical statement that begins at source line 52. Its exact fragment is ‘"sha256": sha256(target),‘. The important operands are sha256: the value named ’sha256’ in this block; target: the value named ’target’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘{ "file": target.name, "samples": stop - start, "first_index": start, "last_index_exclusive": stop, "bytes": target.stat().st_size, "sha256": sha256(target), }‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 59: `})` This is continuation line 8 of the 8-line logical statement that begins at source line 52. Its exact fragment is ‘})‘. The expression is a literal or structural operator and has no hidden software state. Read together, the com-

plete operation is: This calls ‘records.append‘ with ‘{ "file": target.name, "samples": stop - start, "first_index": start, "last_index_exclusive": stop, "bytes": target.stat().st_size, "sha256": sha256(target), }‘ inside the function write_split, which handles write split. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 60: `return records` This returns ‘records‘ to the caller of the function write_split, which handles write split, ending this invocation. The important operands are records: the number of descriptor records accepted since program start.

Line 63: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 64: `arguments = parse_arguments()` This assigns ‘parse_arguments()‘ to ‘arguments ‘ in host memory. Here arguments means the value named ’arguments’ in this block. The important operands are parse_arguments: the value named ’parse arguments’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 65: `if arguments.shard_size <= 0:` This executes the following indented host-code branch only when ‘arguments.shard_size <= 0‘ is true. The important operands are arguments: the value named ’arguments’ in this block; shard_size: the value named ’shard size’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 66: `raise ValueError("shard-size must be positive")` This rejects an invalid state by raising ‘ValueError("shard-size must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 67: `try:` This is explicit host-side failure handling inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 68: `from torchvision.datasets import CIFAR10` This imports the named Python API so the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 69: `except ImportError as error:` This is explicit host-side failure handling inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 70: `raise RuntimeError("torchvision is required to export official CIFAR-10") from error` This rejects an invalid state by raising ‘RuntimeError("torchvision is required to export official CIFAR-10") from error‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 71: `output_dir = arguments.output_dir.expanduser().resolve()` This assigns ‘arguments.output_dir.expanduser().re to ‘output_dir ‘ in host memory. Here output_dir means the value named ’output dir’ in this block. The important operands are arguments: the value named ’arguments’ in this block; output_dir: the value named ’output dir’ in this block; expanduser: the value named ’expanduser’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 72: `output_dir.mkdir(parents=True, exist_ok=True)` This calls ‘output_dir.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 73: `dataset_root = arguments.dataset_root.expanduser().resolve()` This assigns ‘arguments.dataset_root.expandu to ‘dataset_root ‘ in host memory. Here dataset_root means the value named ’dataset root’ in this block. The important operands are arguments: the value named ’arguments’ in this block; dataset_root: the value named ’dataset root’ in this block; expanduser: the value named ’expanduser’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 74: `train = CIFAR10(` This statement continues through source line 75; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘CIFAR10(root=str(dataset_root), train=True, download=not arguments.no_download)‘ to ‘train ‘ in host memory. Here train means the value named ’train’ in this block. The important operands are CIFAR10: the value named ’cifar10’ in this block; root: the value named ’root’ in this block; str: the value named ’str’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 75: `root=str(dataset_root), train=True, download=not arguments.no_download)` This is continuation line 2 of the 2-line logical statement that begins at source line 74. Its exact fragment is ‘root=str(dataset_root), train=True, download=not arguments.no_download)‘. The important operands are root: the value named ’root’ in this block; str: the value named ’str’ in this block; dataset_root: the value named ’dataset root’ in this block. Read together, the complete operation is: This assigns ‘CIFAR10(root=str(dataset_root), train=True, download=not arguments.no_download)‘ to ‘train ‘ in host memory. Here train means the value named ’train’ in this block. The important operands are CIFAR10: the value named ’cifar10’ in this block; root: the value named ’root’ in this block; str: the value named ’str’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 76: `test = CIFAR10(` This statement continues through source line 77; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘CIFAR10(root=str(dataset_root), train=False, download=not arguments.no_download)‘ to ‘test ‘ in host memory. Here test means the value named ’test’ in this block. The important operands are CIFAR10: the value named ’cifar10’ in this block; root: the value named ’root’ in this block; str: the value named ’str’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 77: `root=str(dataset_root), train=False, download=not arguments.no_download)` This is continuation line 2 of the 2-line logical statement that begins at source line 76. Its exact fragment is ‘root=str(dataset_root), train=False, download=not arguments.no_download)‘. The important operands are root: the value named ’root’ in this block; str: the value named ’str’ in this block; dataset_root: the value named ’dataset root’ in this block. Read together, the complete operation is: This assigns ‘CIFAR10(root=str(dataset_root), train=False, download=not arguments.no_download)‘ to ‘test ‘ in host memory. Here test means the value named ’test’ in this block. The important operands are CIFAR10: the value named ’cifar10’ in this block; root: the value named ’root’ in this block; str: the value named ’str’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 78: `manifest = {` This statement continues through source line 89; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 79: `"format": "neuraedge-official-cifar10-shards-v1",` This is continuation line 2 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘"format": "neuraedge-official-cifar10-shards-v1",‘. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8

class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 80: `"created_utc": datetime.now(timezone.utc).isoformat(),` This is continuation line 3 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘"created_utc": datetime.now(timezone.utc).isoformat(),‘. The important operands are created_utc: the value named ’created utc’ in this block; datetime: the value named ’datetime’ in this block; now: the value named ’now’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 81: `"source": "torchvision.datasets.CIFAR10 official Python archive",` This is continuation line 4 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘"source": "torchvision.datasets.CIFAR10 official Python archive",‘. The important operands are source: the value named ’source’ in this block; torchvision: the value named ’torchvision’ in this block; datasets: the value named ’datasets’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 82: `"image_encoding": "uint8 NHWC RGB",` This is continuation line 5 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘"image_encoding": "uint8 NHWC RGB",‘. The important operands are image_encoding: the value named ’image encoding’ in this block; uint8: the value named ’uint8’ in this block; NHWC: the value named ’nhwc’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 83: `"label_encoding": "uint8 class index 0..9",` This is continuation line 6 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘"label_encoding": "uint8 class index 0..9",‘. The important operands are label_encoding: the value named ’label encoding’ in this block; uint8: the value named ’uint8’ in this block; class: the value named ’class’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 84: `"shard_size": arguments.shard_size,` This is continuation line 7 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘"shard_size": arguments.shard_size,‘. The important operands are shard_size: the value named ’shard size’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 85: `"splits": {` This is continuation line 8 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘"splits": {‘. The important operands are splits: the value named ’splits’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 86: `"train": write_split(train, "train", output_dir, arguments.shard_size),` This is continuation line 9 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘"train": write_split(train, "train", output_dir, arguments.shard_size),‘. The important operands are train: the value named ’train’ in this block; write_split: the value named ’write split’ in this block; output_dir: the value named ’output dir’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 87: `"test": write_split(test, "test", output_dir, arguments.shard_size),` This is continuation line 10 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘"test": write_split(test, "test", output_dir, arguments.shard_size),‘. The important operands are test: the value named ’test’ in this block; write_split: the value named ’write split’ in this block; output_dir: the value named ’output dir’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat() "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; official: the value named ’official’ in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 88: `},` This is continuation line 11 of the 12-line logical statement that begins at source line 78. Its exact fragment is ‘},‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block;

official: the value named 'official' in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 89: `}` This is continuation line 12 of the 12-line logical statement that begins at source line 78. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "format": "neuraedge-official-cifar10-shards-v1", "created_utc": datetime.now(timezone.utc).isoformat(), "source": "torchvision.datasets.CIFAR10 official Python archive", "image_encoding": "uint8 NHWC RGB", "label_encoding": "uint8 class index 0..9", "shard_size": arguments.shard_size, "splits": { "train": write_split(train, "train", output_dir, arguments.shard_size), "test": write_split(test, "test", output_dir, arguments.shard_size), }, }' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; official: the value named 'official' in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 90: `manifest_path = output_dir / "cifar10_shards_manifest.json"` This assigns 'output_dir / "cifar10_shards_ma
to 'manifest_path ' in host memory. Here manifest_path means the value named 'manifest path' in this block. The important operands are output_dir: the value named 'output dir' in this block; cifar10_shards_manifest: the value named 'cifar10 shards manifest' in this block; json: the value named 'json' in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 91: `temporary_manifest = manifest_path.with_name(manifest_path.name + ".tmp")` This assigns 'manifest_path.with_name(manifest_path.name + ".tmp")' to 'temporary_manifest ' in host memory. Here temporary_manifest means the value named 'temporary manifest' in this block. The important operands are manifest_path: the value named 'manifest path' in this block; with_name: the value named 'with name' in this block; name: the value named 'name' in this block. Context: the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 92: `temporary_manifest.write_text(` This statement continues through source line 93; the explanation uses the complete expression rather than only the first visual fragment. This calls 'temporary_manifest.write_text' with ' json.dumps(manifest, indent=2) + "\n", encoding="ascii"' inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 93: `json.dumps(manifest, indent=2) + "\n", encoding="ascii")` This is continuation line 2 of the 2-line logical statement that begins at source line 92. Its exact fragment is 'json.dumps(manifest, indent=2) + "\n", encoding="ascii")'. The important operands are json: the value named 'json' in this block; dumps: the value named 'dumps' in this block; manifest: the value named 'manifest' in this block. Read together, the complete operation is: This calls 'temporary_manifest.write_text' with ' json.dumps(manifest, indent=2) + "\n", encoding="ascii"' inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 94: `os.replace(temporary_manifest, manifest_path)` This calls 'os.replace' with 'temporary_manifest, manifest_path' inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 95: `print(json.dumps({` This statement continues through source line 99; the explanation uses the complete expression rather than only the first visual fragment. This calls 'print' with 'json.dumps({ "manifest": str(manifest_path), "train_samples": sum(item["samples"] for item in manifest["splits"]["train"]), "test_samples": sum(item["samples"] for item in manifest["splits"]["test"]), }, indent=2)' inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 96: `"manifest": str(manifest_path),` This is continuation line 2 of the 5-line logical statement that begins at source line 95. Its exact fragment is ‘"manifest": str(manifest_path),‘. The important operands are manifest: the value named 'manifest' in this block; str: the value named 'str' in this block; manifest_path: the value named 'manifest path' in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "manifest": str(manifest_path), "train_samples": sum(item["samples"] for item in manifest["splits"]["train"]), "test_samples": sum(item["samples"] for item in manifest["splits"]["test"]), }, indent=2)‘ inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 97: `"train_samples": sum(item["samples"] for item in manifest["splits"]["train"]),` This is continuation line 3 of the 5-line logical statement that begins at source line 95. Its exact fragment is ‘"train_samples": sum(item["samples"] for item in manifest["splits"]["train"]),‘. The important operands are train_samples: the value named 'train samples' in this block; sum: the value named 'sum' in this block; item: the value named 'item' in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "manifest": str(manifest_path), "train_samples": sum(item["samples"] for item in manifest["splits"]["train"]), "test_samples": sum(item["samples"] for item in manifest["splits"]["test"]), }, indent=2)‘ inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 98: `"test_samples": sum(item["samples"] for item in manifest["splits"]["test"]),` This is continuation line 4 of the 5-line logical statement that begins at source line 95. Its exact fragment is ‘"test_samples": sum(item["samples"] for item in manifest["splits"]["test"]),‘. The important operands are test_samples: the value named 'test samples' in this block; sum: the value named 'sum' in this block; item: the value named 'item' in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "manifest": str(manifest_path), "train_samples": sum(item["samples"] for item in manifest["splits"]["train"]), "test_samples": sum(item["samples"] for item in manifest["splits"]["test"]), }, indent=2)‘ inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

Line 99: `}, indent=2))` This is continuation line 5 of the 5-line logical statement that begins at source line 95. Its exact fragment is ‘}, indent=2))‘. The important operands are indent: the value named 'indent' in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "manifest": str(manifest_path), "train_samples": sum(item["samples"] for item in manifest["splits"]["train"]), "test_samples": sum(item["samples"] for item in manifest["splits"]["test"]), }, indent=2)‘ inside the file whose role is to writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning. The call advances the host-side job that writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning.

27 software/export_spikingformer_hardware.py

Role in the complete system: folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

27.1 Beginner architecture map

Dataflow responsibility Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: BinarySection, ProjectionExport, AlignedBinaryWriter. Functions: quantize_weight_tensor, export_projection, projection_pairs, export_model, parse_arguments, ___init___, append. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

27.2 Complete source

```
"""Export the trained Spikingformer into the NeuraEdge binary parameter ABI."""

from __future__ import annotations

import argparse
import hashlib
import json
from dataclasses import asdict, dataclass
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn

from neuraedge_golden import quantize_flexible_scale
from spikingformer_quantization import quantize_cifar_input_u8
from spikingformer_reference import Spikingformer, load_checkpoint


ALIGNMENT = 64


@dataclass(frozen=True)
class BinarySection:
    name: str
    offset: int
    length: int
    dtype: str
    shape: tuple[int, ...]
    sha256: str


@dataclass(frozen=True)
class ProjectionExport:
    name: str
    source_weight_shape: tuple[int, ...]
    hardware_matrix_shape: tuple[int, int]
    input_encoding: str
    compute_opcode: str
    output_fractional_bits: int
    weight_scale: float
    accumulator_factor_minimum: float
    accumulator_factor_maximum: float
    represented_factor_maximum_error: float
    folded_bias_minimum: int
    folded_bias_maximum: int
    sections: tuple[BinarySection, ...]


```

```
class AlignedBinaryWriter:
    """Append typed arrays while retaining deterministic aligned offsets."""

    def __init__(self) -> None:
        self.data = bytearray()
        self.sections: list[BinarySection] = []

    def append(self, name: str, value: np.ndarray) -> BinarySection:
        padding = (-len(self.data)) % ALIGNMENT
        self.data.extend(b"\x00" * padding)
        contiguous = np.ascontiguousarray(value)
        payload = contiguous.tobytes(order="C")
        section = BinarySection(
            name=name,
            offset=len(self.data),
            length=len(payload),
            dtype=contiguous.dtype.str,
            shape=tuple(contiguous.shape),
            sha256=hashlib.sha256(payload).hexdigest(),
        )
        self.data.extend(payload)
        self.sections.append(section)
        return section


def quantize_weight_tensor(weight: torch.Tensor) -> tuple[torch.Tensor, float]:
    """Symmetric signed-int8 quantization used by the measured 95.4% model."""

    maximum = float(weight.detach().abs().amax().item())
    scale = maximum / 127.0 if maximum > 0.0 else 1.0
    integer = torch.round(weight.detach() / scale).clamp(-127, 127).to(torch.int8)
    return integer, scale


def export_projection(
    writer: AlignedBinaryWriter,
    *,
    name: str,
    convolution: nn.Conv1d | nn.Conv2d,
    batch_norm: nn.BatchNorm1d | nn.BatchNorm2d,
    input_scale: float,
    fractional_bits: int,
    input_encoding: str,
    compute_opcode: str,
) -> ProjectionExport:
    """Quantize one Conv-BN pair and generate its exact post-accumulator fields."""

    integer_weight, weight_scale = quantize_weight_tensor(convolution.weight)
    output_channels = convolution.out_channels
    matrix = integer_weight.reshape(output_channels, -1).transpose(0, 1).contiguous()
    batch_norm_scale = batch_norm.weight.detach() / torch.sqrt(
        batch_norm.running_var.detach() + batch_norm.eps)
    if convolution.bias is None:
        convolution_bias = torch.zeros_like(batch_norm.running_mean)
    else:
        convolution_bias = convolution.bias.detach()
    folded_bias = (
        (convolution_bias - batch_norm.running_mean.detach()) * batch_norm_scale
        + batch_norm.bias.detach()
    )
    output_scale = 1 << fractional_bits
    accumulator_factor = (
        input_scale * weight_scale * batch_norm_scale * output_scale)
    multiplier: list[int] = []
    binary_shift: list[int] = []
    represented: list[float] = []
    for value in accumulator_factor.tolist():
        q15, shift, represented_value = quantize_flexible_scale(float(value))
        multiplier.append(q15)
        binary_shift.append(shift)
        represented.append(represented_value)
    bias_integer = torch.round(folded_bias * output_scale).to(torch.int32)
    factor_error = torch.tensor(represented) - accumulator_factor.cpu()

    sections = (
        writer.append(f"{name}.weight_k_by_n_int8", matrix.cpu().numpy()),
        writer.append(f"{name}.bias_q{fractional_bits}", bias_integer.cpu().numpy()),
        writer.append(f"{name}.multiplier_q15", np.asarray(multiplier, dtype="<i2")),
        writer.append(f"{name}.binary_point_shift", np.asarray(binary_shift, dtype=np.int8)),
    )
    return ProjectionExport(
        name=name,
        source_weight_shape=tuple(convolution.weight.shape),
        hardware_matrix_shape=tuple(matrix.shape),
        input_encoding=input_encoding,
        compute_opcode=compute_opcode,
        output_fractional_bits=fractional_bits,
```

```
        weight_scale=weight_scale,
        accumulator_factor_minimum=float(accumulator_factor.amin().item()),
        accumulator_factor_maximum=float(accumulator_factor.amax().item()),
        represented_factor_maximum_error=float(factor_error.abs().amax().item()),
        folded_bias_minimum=int(bias_integer.amin().item()),
        folded_bias_maximum=int(bias_integer.amax().item()),
        sections=sections,
    )


def projection_pairs(model: Spikingformer) -> list[tuple[str, nn.Module, nn.Module]]:
    pairs: list[tuple[str, nn.Module, nn.Module]] = [
        (f"patch_embed.block{index}",
         getattr(model.patch_embed, f"block{index}_conv"),
         getattr(model.patch_embed, f"block{index}_bn"))
        for index in range(5)
    ]
    for index, block in enumerate(model.block):
        for name in ("q", "k", "v", "proj"):
            pairs.append((
                f"block.{index}.attn.{name}",
                getattr(block.attn, f"{name}_conv"),
                getattr(block.attn, f"{name}_bn"),
            ))
        for layer in (1, 2):
            pairs.append((
                f"block.{index}.mlp.fc{layer}",
                getattr(block.mlp, f"mlp{layer}_conv"),
                getattr(block.mlp, f"mlp{layer}_bn"),
            ))
    return pairs


def export_model(
    checkpoint_path: Path,
    output_dir: Path,
    *,
    fractional_bits: int = 10,
    head_activation_scale: float = 0.125,
) -> dict[str, object]:
    loaded = load_checkpoint(checkpoint_path, device="cpu")
    model = loaded.model
    writer = AlignedBinaryWriter()
    input_probe = torch.zeros(1, 3, 1, 1)
    _, image_scale, image_zero_point = quantize_cifar_input_u8(input_probe)
    projections: list[ProjectionExport] = []
    for name, convolution, batch_norm in projection_pairs(model):
        first_layer = name == "patch_embed.block0"
        projections.append(export_projection(
            writer,
            name=name,
            convolution=convolution,
            batch_norm=batch_norm,
            input_scale=image_scale if first_layer else 1.0,
            fractional_bits=fractional_bits,
            input_encoding=(
                f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}"
                if first_layer else "binary spike 0/1"),
            compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),
        ))

    head_weight, head_weight_scale = quantize_weight_tensor(model.head.weight)
    head_matrix = head_weight.transpose(0, 1).contiguous().cpu().numpy()
    head_accumulator_scale = head_activation_scale * head_weight_scale
    head_bias = torch.round(
        model.head.bias.detach() / head_accumulator_scale).to(torch.int32).cpu().numpy()
    head_sections = (
        writer.append("head.weight_k_by_n_int8", head_matrix),
        writer.append("head.bias_accumulator_units", head_bias),
    )
    checkpoint_hash = hashlib.sha256(checkpoint_path.read_bytes()).hexdigest()
    output_dir.mkdir(parents=True, exist_ok=True)
    binary_path = output_dir / "spikingformer_neuraedge_weights.bin"
    binary_path.write_bytes(writer.data)
    binary_hash = hashlib.sha256(writer.data).hexdigest()
    manifest: dict[str, object] = {
        "format": "neuraedge-spikingformer-parameters-v1",
        "checkpoint": str(checkpoint_path.resolve()),
        "checkpoint_sha256": checkpoint_hash,
        "checkpoint_epoch": loaded.epoch,
        "checkpoint_metric_percent": loaded.metric,
        "binary_file": binary_path.name,
        "binary_bytes": len(writer.data),
        "binary_sha256": binary_hash,
        "alignment_bytes": ALIGNMENT,
        "fixed_point": {
            "activation_fractional_bits": fractional_bits,
```

```
            "activation_scale": 1 << fractional_bits,
            "lif_membrane_bits": 24,
            "lif_current_bits": 16,
            "image_input_scale": image_scale,
            "image_input_zero_point": image_zero_point,
            "head_activation_scale": head_activation_scale,
        },
        "projections": [asdict(item) for item in projections],
        "head": {
            "source_weight_shape": list(model.head.weight.shape),
            "hardware_matrix_shape": list(head_matrix.shape),
            "activation_encoding": "signed int8 encoded as uint8 with zero point 128",
            "weight_encoding": "signed int8 encoded as uint8 with zero point 128",
            "compute_opcode": "COMPUTE_DENSE_DOT8",
            "weight_scale": head_weight_scale,
            "accumulator_scale": head_accumulator_scale,
            "bias_minimum": int(head_bias.min()),
            "bias_maximum": int(head_bias.max()),
            "sections": [asdict(item) for item in head_sections],
        },
        "sections": [asdict(item) for item in writer.sections],
    }
    manifest_path = output_dir / "spikingformer_neuraedge_weights.json"
    manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="ascii")
    return manifest


def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument("--checkpoint", required=True, type=Path)
    parser.add_argument("--output-dir", required=True, type=Path)
    parser.add_argument("--fractional-bits", type=int, default=10)
    parser.add_argument("--head-activation-scale", type=float, default=0.125)
    return parser.parse_args()


if __name__ == "__main__":
    arguments = parse_arguments()
    exported = export_model(
        arguments.checkpoint.resolve(),
        arguments.output_dir.resolve(),
        fractional_bits=arguments.fractional_bits,
        head_activation_scale=arguments.head_activation_scale,
    )
    print(json.dumps({
        "binary_file": exported["binary_file"],
        "binary_bytes": exported["binary_bytes"],
        "binary_sha256": exported["binary_sha256"],
        "projection_count": len(exported["projections"]),
    }, indent=2))
```

27.3 Line annotations

Line 1: `"""Export the trained Spikingformer into the NeuraEdge binary parameter ABI."""` This is executable documentation attached to the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import hashlib` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `import json` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `from dataclasses import asdict, dataclass` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `from pathlib import Path` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `import numpy as np` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `import torch` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `import torch.nn as nn` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `from neuraedge_golden import quantize_flexible_scale` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `from spikingformer_quantization import quantize_cifar_input_u8` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `from spikingformer_reference import Spikingformer, load_checkpoint` This imports the named Python API so the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `ALIGNMENT = 64` This assigns ‘64‘ to ‘ALIGNMENT ‘ in host memory. Here ALIGNMENT means the value named ’alignment’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 23: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections; it creates no FPGA circuit by itself.

Line 24: `class BinarySection:` This defines class BinarySection, which groups data and methods for binary section. Instances exist in host memory and organize the flow that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 25: `name: str` Inside the class BinarySection, which handles binary section, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are name: the value named ’name’ in this block; str: the value named ’str’ in this block.

Line 26: `offset: int` Inside the class BinarySection, which handles binary section, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are offset: the value named ’offset’ in this block.

Line 27: `length: int` Inside the class BinarySection, which handles binary section, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are length: the value named 'length' in this block.

Line 28: `dtype: str` Inside the class BinarySection, which handles binary section, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are dtype: the value named 'dtype' in this block; str: the value named 'str' in this block.

Line 29: `shape: tuple[int, ...]` Inside the class BinarySection, which handles binary section, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are shape: the value named 'shape' in this block; tuple: the value named 'tuple' in this block.

Line 30: `sha256: str` Inside the class BinarySection, which handles binary section, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are sha256: the value named 'sha256' in this block; str: the value named 'str' in this block.

Line 33: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections; it creates no FPGA circuit by itself.

Line 34: `class ProjectionExport:` This defines class ProjectionExport, which groups data and methods for projection export. Instances exist in host memory and organize the flow that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 35: `name: str` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are name: the value named 'name' in this block; str: the value named 'str' in this block.

Line 36: `source_weight_shape: tuple[int, ...]` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are source_weight_shape: a learned coefficient multiplied by an activation; tuple: the value named 'tuple' in this block.

Line 37: `hardware_matrix_shape: tuple[int, int]` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are hardware_matrix_shape: the value named 'hardware matrix shape' in this block; tuple: the value named 'tuple' in this block.

Line 38: `input_encoding: str` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are input_encoding: the value named 'input encoding' in this block; str: the value named 'str' in this block.

Line 39: `compute_opcode: str` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are compute_opcode: the value named 'compute opcode' in this block; str: the value named 'str' in this block.

Line 40: `output_fractional_bits: int` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are output_fractional_bits: the value named 'output fractional bits' in this block.

Line 41: `weight_scale: float` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are weight_scale: a learned coefficient multiplied by an activation; float: the value named 'float' in this block.

Line 42: `accumulator_factor_minimum: float` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are accumulator_factor_minimum: a widened running sum that prevents intermediate overflow; float: the value named 'float' in this block.

Line 43: `accumulator_factor_maximum: float` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are accumulator_factor_maximum: a widened running sum that prevents intermediate overflow; float: the value named 'float' in this block.

Line 44: `represented_factor_maximum_error: float` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are represented_factor_maximum_error: the value named 'represented factor maximum error' in this block; float: the value named 'float' in this block.

Line 45: `folded_bias_minimum: int` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are folded_bias_minimum: an additive learned offset applied after accumulation.

Line 46: `folded_bias_maximum: int` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are folded_bias_maximum: an additive learned offset applied after accumulation.

Line 47: `sections: tuple[BinarySection, ...]` Inside the class ProjectionExport, which handles projection export, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are sections: the value named 'sections' in this block; tuple: the value named 'tuple' in this block; BinarySection: the value named 'binary section' in this block.

Line 50: `class AlignedBinaryWriter:` This defines class AlignedBinaryWriter, which groups data and methods for aligned binary writer. Instances exist in host memory and organize the flow that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 51: `"""Append typed arrays while retaining deterministic aligned offsets."""` This is executable documentation attached to the class AlignedBinaryWriter, which handles aligned binary writer. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 53: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections; it runs only when called.

Line 54: `self.data = bytearray()` This assigns 'bytearray()' to 'self.data ' in host memory. Here data means the value named 'data' in this block. The important operands are bytearray: the value named 'bytearray' in this block. Context: the function ___init___, which handles init.

Line 55: `self.sections: list[BinarySection] = []` This assigns '[]' to 'self.sections' in host memory. Here sections means the value named 'sections' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 57: `def append(self, name: str, value: np.ndarray) -> BinarySection:` This starts function append. Its indented body performs append as part of the file that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections; it runs only when called.

Line 58: `padding = (-len(self.data)) % ALIGNMENT` This assigns '(-len(self.data)) % ALIGNMENT' to 'padding ' in host memory. Here padding means the value named 'padding' in this block. The important operands are len: the value named 'len' in this block; data: the value named 'data' in this block; ALIGNMENT: the value named 'alignment' in this block. Context: the function append, which handles append.

Line 59: `self.data.extend(b"\x00" * padding)` This calls 'self.data.extend' with 'b"\x00" * padding' inside the function append, which handles append. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 60: `contiguous = np.ascontiguousarray(value)` This assigns 'np.ascontiguousarray(value)' to 'contiguous ' in host memory. Here contiguous means the value named 'contiguous' in this block. The important operands are ascontiguousarray: the value named 'ascontiguousarray' in this block; value: the value named 'value' in this block. Context: the function append, which handles append.

Line 61: `payload = contiguous.tobytes(order="C")` This assigns 'contiguous.tobytes(order="C")' to 'payload ' in host memory. Here payload means stream data belonging to the current command. The important operands are contiguous: the value named 'contiguous' in this block; tobytes: the value named 'tobytes' in this block; order: the value named 'order' in this block. Context: the function append, which handles append.

Line 62: `section = BinarySection(` This statement continues through source line 69; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'BinarySection(name=name, offset=len(self.data), length=len(payload), dtype=contiguous.dtype.str, shape=tuple(contiguous.shape), sha256=hashlib.sha256(payload).hexdigest(),)' to 'section ' in host memory. Here section means the value named 'section' in this block. The important operands are BinarySection: the value named 'binary section' in this block; name: the value named 'name' in this block; offset: the value named 'offset' in this block. Context: the function append, which handles append.

Line 63: `name=name,` This is continuation line 2 of the 8-line logical statement that begins at source line 62. Its exact fragment is 'name=name,'. The important operands are name: the value named 'name' in this block. Read together, the complete operation is: This assigns 'BinarySection(name=name, offset=len(self.data), length=len(payload), dtype=contiguous.dtype.str, shape=tuple(contiguous.shape), sha256=hashlib.sha256(payload).hexdigest(),)' to 'section ' in host memory. Here section means the value named 'section' in this block. The important operands are BinarySection: the value named 'binary section' in this block; name: the value named 'name' in this block; offset: the value named 'offset' in this block. Context: the function append, which handles append.

Line 64: `offset=len(self.data),` This is continuation line 3 of the 8-line logical statement that begins at source line 62. Its exact fragment is 'offset=len(self.data),'. The important operands are offset: the value named 'offset' in this block; len: the value named 'len' in this block; data: the value named 'data' in this block. Read together, the complete operation is: This assigns 'BinarySection(name=name, offset=len(self.data), length=len(payload), dtype=contiguous.dtype.str, shape=tuple(contiguous.shape), sha256=hashlib.sha256(payload).hexdigest(),)' to 'section ' in host memory. Here section means the value named 'section' in this block. The important operands are BinarySection: the value named 'binary section' in this block; name: the value named 'name' in this block; offset: the value named 'offset' in this block. Context: the function append, which handles append.

Line 65: `length=len(payload),` This is continuation line 4 of the 8-line logical statement that begins at source line 62. Its exact fragment is 'length=len(payload),'. The important operands are length: the value named 'length' in this block; len: the value named 'len' in this block; payload: stream data belonging to the current command. Read together, the complete operation is: This assigns 'BinarySection(name=name, offset=len(self.data), length=len(payload), dtype=contiguous.dtype.str, shape=tuple(contiguous.shape), sha256=hashlib.sha256(payload).hexdigest(),)' to 'section ' in host memory. Here section means the value named 'section' in this block. The important operands are BinarySection: the value named 'binary section' in this block; name: the value named 'name' in this block; offset: the value named 'offset' in this block. Context: the function append, which handles append.

Line 66: `dtype=contiguous.dtype.str,` This is continuation line 5 of the 8-line logical statement that begins at source line 62. Its exact fragment is ‘dtype=contiguous.dtype.str,‘. The important operands are dtype: the value named ’dtype’ in this block; contiguous: the value named ’contiguous’ in this block; str: the value named ’str’ in this block. Read together, the complete operation is: This assigns ‘BinarySection(name=name, offset=len(self.data), length=len(payload), dtype=contiguous.dtype.str, shape=tuple(contiguous.shape), sha256=hashlib.sha256(payload).hexdigest(),)‘ to ‘section ‘ in host memory. Here section means the value named ’section’ in this block. The important operands are BinarySection: the value named ’binary section’ in this block; name: the value named ’name’ in this block; offset: the value named ’offset’ in this block. Context: the function append, which handles append.

Line 67: `shape=tuple(contiguous.shape),` This is continuation line 6 of the 8-line logical statement that begins at source line 62. Its exact fragment is ‘shape=tuple(contiguous.shape),‘. The important operands are shape: the value named ’shape’ in this block; tuple: the value named ’tuple’ in this block; contiguous: the value named ’contiguous’ in this block. Read together, the complete operation is: This assigns ‘BinarySection(name=name, offset=len(self.data), length=len(payload), dtype=contiguous.dtype.str, shape=tuple(contiguous.shape), sha256=hashlib.sha256(payload).hexdigest(),)‘ to ‘section ‘ in host memory. Here section means the value named ’section’ in this block. The important operands are BinarySection: the value named ’binary section’ in this block; name: the value named ’name’ in this block; offset: the value named ’offset’ in this block. Context: the function append, which handles append.

Line 68: `sha256=hashlib.sha256(payload).hexdigest(),` This is continuation line 7 of the 8-line logical statement that begins at source line 62. Its exact fragment is ‘sha256=hashlib.sha256(payload).hexdigest(),‘. The important operands are sha256: the value named ’sha256’ in this block; hashlib: the value named ’hashlib’ in this block; payload: stream data belonging to the current command. Read together, the complete operation is: This assigns ‘BinarySection(name=name, offset=len(self.data), length=len(payload), dtype=contiguous.dtype.str, shape=tuple(contiguous.shape), sha256=hashlib.sha256(payload).hexdigest(),)‘ to ‘section ‘ in host memory. Here section means the value named ’section’ in this block. The important operands are BinarySection: the value named ’binary section’ in this block; name: the value named ’name’ in this block; offset: the value named ’offset’ in this block. Context: the function append, which handles append.

Line 69: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 62. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘BinarySection(name=name, offset=len(self.data), length=len(payload), dtype=contiguous.dtype.str, shape=tuple(contiguous.shape), sha256=hashlib.sha256(payload).hexdigest(),)‘ to ‘section ‘ in host memory. Here section means the value named ’section’ in this block. The important operands are BinarySection: the value named ’binary section’ in this block; name: the value named ’name’ in this block; offset: the value named ’offset’ in this block. Context: the function append, which handles append.

Line 70: `self.data.extend(payload)` This calls ‘self.data.extend‘ with ‘payload‘ inside the function append, which handles append. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 71: `self.sections.append(section)` This calls ‘self.sections.append‘ with ‘section‘ inside the function append, which handles append. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 72: `return section` This returns ‘section‘ to the caller of the function append, which handles append, ending this invocation. The important operands are section: the value named ’section’ in this block.

Line 75: `def quantize_weight_tensor(weight: torch.Tensor) -> tuple[torch.Tensor, float]:` This starts function quantize_weight_tensor. Its indented body performs quantize weight tensor as part of the file that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections; it runs only when called.

Line 76: `"""Symmetric signed-int8 quantization used by the measured 95.4% model."""` This is executable documentation attached to the function quantize_weight_tensor, which handles quantize weight tensor. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 78: `maximum = float(weight.detach().abs().amax().item())` This assigns ‘float(weight.detach().abs().amax().item())‘ to ‘maximum ‘ in host memory. Here maximum means the value named ’maximum’ in this block. The important operands are float: the value named ’float’ in this block; weight: a learned coefficient multiplied by an activation; detach: the value named ’detach’ in this block. Context: the function quantize_weight_tensor, which handles quantize weight tensor.

Line 79: `scale = maximum / 127.0 if maximum > 0.0 else 1.0` This assigns ‘maximum / 127.0 if maximum > 0.0 else 1.0‘ to ‘scale ‘ in host memory. Here scale means the real-value spacing represented by one quantized integer step. The important operands are maximum: the value named ’maximum’ in this block. Context: the function quantize_weight_tensor, which handles quantize weight tensor.

Line 80: `integer = torch.round(weight.detach() / scale).clamp(-127, 127).to(torch.int8)` This assigns ‘torch.round(weight.detach() / scale).clamp(-127, 127).to(torch.int8)‘ to ‘integer ‘ in host memory. Here integer means the value named ’integer’ in this block. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function quantize_weight_tensor, which handles quantize weight tensor.

Line 81: `return integer, scale` This returns ‘integer, scale‘ to the caller of the function quantize_weight_tensor, which handles quantize weight tensor, ending this invocation. The important operands are integer: the value named ’integer’ in this block; scale: the real-value spacing represented by one quantized integer step.

Line 84: `def export_projection(` This starts function export_projection. Its indented body performs export projection as part of the file that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections; it runs only when called.

Line 85: `writer: AlignedBinaryWriter,` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are writer: the value named ’writer’ in this block; AlignedBinaryWriter: the value named ’aligned binary writer’ in this block.

Line 86: `*,` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The expression is a literal or structural operator and has no hidden software state.

Line 87: `name: str,` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are name: the value named ’name’ in this block; str: the value named ’str’ in this block.

Line 88: `convolution: nn.Conv1d | nn.Conv2d,` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are convolution: the value named ’convolution’ in this block; nn: the value named ’nn’ in this block; Conv1d: the value named ’conv1d’ in this block.

Line 89: `batch_norm: nn.BatchNorm1d | nn.BatchNorm2d,` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are batch_norm: the value named ’batch norm’ in this block; nn: the value named ’nn’ in this block; BatchNorm1d: the value named ’batch norm1d’ in this block.

Line 90: `input_scale: float,` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are input_scale: the real-value spacing represented by one quantized integer step; float: the value named ’float’ in this block.

Line 91: `fractional_bits: int,` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are fractional_bits: the value named 'fractional bits' in this block.

Line 92: `input_encoding: str,` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are input_encoding: the value named 'input encoding' in this block; str: the value named 'str' in this block.

Line 93: `compute_opcode: str,` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are compute_opcode: the value named 'compute opcode' in this block; str: the value named 'str' in this block.

Line 94: `) -> ProjectionExport:` Inside the function export_projection, which handles export projection, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are ProjectionExport: the value named 'projection export' in this block.

Line 95: `"""Quantize one Conv-BN pair and generate its exact post-accumulator fields."""` This is executable documentation attached to the function export_projection, which handles export projection. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 97: `integer_weight, weight_scale = quantize_weight_tensor(convolution.weight)` This assigns 'quantize_weight_tensor(convolution.weight)' to 'integer_weight, weight_scale ' in host memory. Here integer_weight means a learned coefficient multiplied by an activation. The important operands are quantize_weight_tensor: a learned coefficient multiplied by an activation; convolution: the value named 'convolution' in this block; weight: a learned coefficient multiplied by an activation. Context: the function export_projection, which handles export projection.

Line 98: `output_channels = convolution.out_channels` This assigns 'convolution.out_channels' to 'output_channels ' in host memory. Here output_channels means the value named 'output channels' in this block. The important operands are convolution: the value named 'convolution' in this block; out_channels: the value named 'out channels' in this block. Context: the function export_projection, which handles export projection.

Line 99: `matrix = integer_weight.reshape(output_channels, -1).transpose(0, 1).contiguous()` This assigns 'integer_weight.reshape(output_channels, -1).transpose(0, 1).contiguous()' to 'matrix ' in host memory. Here matrix means the value named 'matrix' in this block. The important operands are integer_weight: a learned coefficient multiplied by an activation; reshape: the value named 'reshape' in this block; output_channels: the value named 'output channels' in this block. Context: the function export_projection, which handles export projection.

Line 100: `batch_norm_scale = batch_norm.weight.detach() / torch.sqrt(` This statement continues through source line 101; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'batch_norm.weight.detach() / torch.sqrt(batch_norm.running_var.detach() + batch_norm.eps)' to 'batch_norm_scale ' in host memory. Here batch_norm_scale means the real-value spacing represented by one quantized integer step. The important operands are batch_norm: the value named 'batch norm' in this block; weight: a learned coefficient multiplied by an activation; detach: the value named 'detach' in this block. Context: the function export_projection, which handles export projection.

Line 101: `batch_norm.running_var.detach() + batch_norm.eps)` This is continuation line 2 of the 2-line logical statement that begins at source line 100. Its exact fragment is 'batch_norm.running_var.detach() + batch_norm.eps)'. The important operands are batch_norm: the value named 'batch norm' in this block; running_var: the value named 'running var' in this block; detach: the value named 'detach' in this block. Read together, the complete operation is: This assigns 'batch_norm.weight.detach() / torch.sqrt(batch_norm.running_var.detach() + batch_norm.eps)' to

‘batch_norm_scale ‘ in host memory. Here batch_norm_scale means the real-value spacing represented by one quantized integer step. The important operands are batch_norm: the value named ’batch norm’ in this block; weight: a learned coefficient multiplied by an activation; detach: the value named ’detach’ in this block. Context: the function export_projection, which handles export projection.

Line 102: `if convolution.bias is None:` This executes the following indented host-code branch only when ‘convolution.bias is None‘ is true. The important operands are convolution: the value named ’convolution’ in this block; bias: an additive learned offset applied after accumulation; is: the value named ’is’ in this block. Context: the function export_projection, which handles export projection.

Line 103: `convolution_bias = torch.zeros_like(batch_norm.running_mean)` This assigns ‘torch.zeros_like(batch_norm.ru to ‘convolution_bias ‘ in host memory. Here convolution_bias means an additive learned offset applied after accumulation. The important operands are torch: the value named ’torch’ in this block; zeros_like: the value named ’zeros like’ in this block; batch_norm: the value named ’batch norm’ in this block. Context: the function export_projection, which handles export projection.

Line 104: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function export_projection, which handles export projection.

Line 105: `convolution_bias = convolution.bias.detach()` This assigns ‘convolution.bias.detach()‘ to ‘convolution_bias ‘ in host memory. Here convolution_bias means an additive learned offset applied after accumulation. The important operands are convolution: the value named ’convolution’ in this block; bias: an additive learned offset applied after accumulation; detach: the value named ’detach’ in this block. Context: the function export_projection, which handles export projection.

Line 106: `folded_bias = (` This statement continues through source line 109; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘((convolution_bias - batch_norm.running_mean.detach()) * batch_norm_scale + batch_norm.bias.detach())‘ to ‘folded_bias ‘ in host memory. Here folded_bias means an additive learned offset applied after accumulation. The important operands are convolution_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Context: the function export_projection, which handles export projection.

Line 107: `(convolution_bias - batch_norm.running_mean.detach()) * batch_norm_scale` This is continuation line 2 of the 4-line logical statement that begins at source line 106. Its exact fragment is ‘(convolution_bias - batch_norm.running_m * batch_norm_scale‘. The important operands are convolution_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Read together, the complete operation is: This assigns ‘((convolution_bias - batch_norm.running_mean.detach()) * batch_norm_scale + batch_norm.bias.detach())‘ to ‘folded_bias ‘ in host memory. Here folded_bias means an additive learned offset applied after accumulation. The important operands are convolution_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Context: the function export_projection, which handles export projection.

Line 108: `+ batch_norm.bias.detach()` This is continuation line 3 of the 4-line logical statement that begins at source line 106. Its exact fragment is ‘+ batch_norm.bias.detach()‘. The important operands are batch_norm: the value named ’batch norm’ in this block; bias: an additive learned offset applied after accumulation; detach: the value named ’detach’ in this block. Read together, the complete operation is: This assigns ‘((convolution_bias - batch_norm.running_mean.detach()) * batch_norm_scale + batch_norm.bias.detach())‘ to ‘folded_bias ‘ in host memory. Here folded_bias means an additive learned offset applied after accumulation. The important operands are convolution_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Context: the function export_projection, which handles export projection.

Line 109: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 106. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘((convolution_bias - batch_norm.running_mean.detach()) * batch_norm_scale +

batch_norm.bias.detach())‘ to ‘folded_bias ‘ in host memory. Here folded_bias means an additive learned offset applied after accumulation. The important operands are convolution_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Context: the function export_projection, which handles export projection.

Line 110: `output_scale = 1 << fractional_bits` This assigns ‘1 << fractional_bits‘ to ‘output_scale ‘ in host memory. Here output_scale means the real-value spacing represented by one quantized integer step. The important operands are fractional_bits: the value named ’fractional bits’ in this block. Context: the function export_projection, which handles export projection.

Line 111: `accumulator_factor = (` This statement continues through source line 112; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(input_scale * weight_scale * batch_norm_scale * output_scale)‘ to ‘accumulator_factor ‘ in host memory. Here accumulator_factor means a widened running sum that prevents intermediate overflow. The important operands are input_scale: the real-value spacing represented by one quantized integer step; weight_scale: a learned coefficient multiplied by an activation; batch_norm_scale: the real-value spacing represented by one quantized integer step. Context: the function export_projection, which handles export projection.

Line 112: `input_scale * weight_scale * batch_norm_scale * output_scale)` This is continuation line 2 of the 2-line logical statement that begins at source line 111. Its exact fragment is ‘input_scale * weight_scale * batch_norm_scale * output_scale)‘. The important operands are input_scale: the real-value spacing represented by one quantized integer step; weight_scale: a learned coefficient multiplied by an activation; batch_norm_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns ‘(input_scale * weight_scale * batch_norm_scale * output_scale)‘ to ‘accumulator_factor ‘ in host memory. Here accumulator_factor means a widened running sum that prevents intermediate overflow. The important operands are input_scale: the real-value spacing represented by one quantized integer step; weight_scale: a learned coefficient multiplied by an activation; batch_norm_scale: the real-value spacing represented by one quantized integer step. Context: the function export_projection, which handles export projection.

Line 113: `multiplier: list[int] = []` This assigns ‘[]‘ to ‘multiplier‘ in host memory. Here multiplier means the value named ’multiplier’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function export_projection, which handles export projection.

Line 114: `binary_shift: list[int] = []` This assigns ‘[]‘ to ‘binary_shift‘ in host memory. Here binary_shift means the value named ’binary shift’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function export_projection, which handles export projection.

Line 115: `represented: list[float] = []` This assigns ‘[]‘ to ‘represented‘ in host memory. Here represented means the value named ’represented’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function export_projection, which handles export projection.

Line 116: `for value in accumulator_factor.tolist():` This iterates value over ‘accumulator_factor.tolist()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function export_projection, which handles export projection.

Line 117: `q15, shift, represented_value = quantize_flexible_scale(float(value))` This assigns ‘quantize_flexible_sc to ‘q15, shift, represented_value ‘ in host memory. Here q15 means the value named ’q15’ in this block. The important operands are quantize_flexible_scale: the real-value spacing represented by one quantized integer step; float: the value named ’float’ in this block; value: the value named ’value’ in this block. Context: the function export_projection, which handles export projection.

Line 118: `multiplier.append(q15)` This calls ‘multiplier.append‘ with ‘q15‘ inside the function export_projection, which handles export projection. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 119: `binary_shift.append(shift)` This calls ‘binary_shift.append‘ with ‘shift‘ inside the function export_projection, which handles export projection. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 120: `represented.append(represented_value)` This calls ‘represented.append‘ with ‘represented_value‘ inside the function export_projection, which handles export projection. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 121: `bias_integer = torch.round(folded_bias * output_scale).to(torch.int32)` This assigns ‘torch.round(folded_ * output_scale).to(torch.int32)‘ to ‘bias_integer ‘ in host memory. Here bias_integer means an additive learned offset applied after accumulation. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; folded_bias: an additive learned offset applied after accumulation. Context: the function export_projection, which handles export projection.

Line 122: `factor_error = torch.tensor(represented) - accumulator_factor.cpu()` This assigns ‘torch.tensor(represente - accumulator_factor.cpu()‘ to ‘factor_error ‘ in host memory. Here factor_error means the value named ’factor error’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block; represented: the value named ’represented’ in this block. Context: the function export_projection, which handles export projection.

Line 124: `sections = (` This statement continues through source line 129; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(writer.append(f"{name}.weight_k_by_n_int8", matrix.cpu().numpy()), writer.append(f"{name}.bias_q{fractional_bits}", bias_integer.cpu().numpy()), writer.append(f"{name}.mult np.asarray(multiplier, dtype="<i2")), writer.append(f"{name}.binary_point_shift", np.asarray(binary_shift, dtype=np.int8)),)‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Context: the function export_projection, which handles export projection.

Line 125: `writer.append(f"{name}.weight_k_by_n_int8", matrix.cpu().numpy()),` This is continuation line 2 of the 6-line logical statement that begins at source line 124. Its exact fragment is ‘writer.append(f"{name}.weight_k_by_n_int8", matrix.cpu().numpy()),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Read together, the complete operation is: This assigns ‘(writer.append(f"{name}.weight_k_by_n_int8", matrix.cpu().numpy()), writer.append(f"{name}.bias_q{fractional_bits}", bias_integer.cpu().numpy()), writer.append(f"{name}.multiplier_q15", np.asarray(multiplier, dtype="<i2")), writer.append(f"{nam np.asarray(binary_shift, dtype=np.int8)),)‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Context: the function export_projection, which handles export projection.

Line 126: `writer.append(f"{name}.bias_q{fractional_bits}", bias_integer.cpu().numpy()),` This is continuation line 3 of the 6-line logical statement that begins at source line 124. Its exact fragment is ‘writer.append(f"{name}.bias_q{fracti bias_integer.cpu().numpy()),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Read together, the complete operation is: This assigns ‘(writer.append(f"{name}.weight_k_by_n_int8", matrix.cpu().numpy()), writer.append(f"{name}.bias_q{fractional_bits}", bias_integer.cpu().numpy()), writer.append(f"{name}.multiplier_q15", np.asarray(multiplier, dtype="<i2")), writer.append(f"{nam np.asarray(binary_shift, dtype=np.int8)),)‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Context: the function export_projection, which handles export projection.

Line 127: `writer.append(f"{name}.multiplier_q15", np.asarray(multiplier, dtype="<i2")),` This is continuation line 4 of the 6-line logical statement that begins at source line 124. Its exact fragment is ‘writer.append(f"{name}.multiplier_q np.asarray(multiplier, dtype="<i2")),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Read together, the complete operation is: This assigns ‘(writer.append(f"{name}.weight_k_by_n_int8", matrix.cpu().numpy()), writer.append(f"{name}.bias_q{fraction bias_integer.cpu().numpy()), writer.append(f"{name}.multiplier_q15", np.asarray(multiplier, dtype="<i2")), writer.append(f"{nam

np.asarray(binary_shift, dtype=np.int8)),)‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Context: the function export_projection, which handles export projection.

Line 128: `writer.append(f"{name}.binary_point_shift", np.asarray(binary_shift, dtype=np.int8)),` This is continuation line 5 of the 6-line logical statement that begins at source line 124. Its exact fragment is ‘writer.append(f"{name}.binar
np.asarray(binary_shift, dtype=np.int8)),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Read together, the complete operation is: This assigns ‘(writer.append(f"{name}.weight_k_by_n_int8", matrix.cpu().numpy()), writer.append(f"{name}.bias_q{fraction
bias_integer.cpu().numpy()), writer.append(f"{name}.multiplier_q15", np.asarray(multiplier, dtype="<i2")), writer.append(f"{nam
np.asarray(binary_shift, dtype=np.int8)),)‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Context: the function export_projection, which handles export projection.

Line 129: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 124. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(writer.append(f"{name}.weight_k_by_n_int8", matrix.cpu().numpy()), writer.append(f"{name}.bias_q{frac
bias_integer.cpu().numpy()), writer.append(f"{name}.multiplier_q15", np.asarray(multiplier, dtype="<i2")), writer.append(f"{nam
np.asarray(binary_shift, dtype=np.int8)),)‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; f: the value named ’f’ in this block. Context: the function export_projection, which handles export projection.

Line 130: `return ProjectionExport(` This statement continues through source line 144; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_e
compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_m
accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro
folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)‘ to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named ’projection export’ in this block; name: the value named ’name’ in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 131: `name=name,` This is continuation line 2 of the 15-line logical statement that begins at source line 130. Its exact fragment is ‘name=name,‘. The important operands are name: the value named ’name’ in this block. Read together, the complete operation is: This returns ‘ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.ar
accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro
folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)‘ to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named ’projection export’ in this block; name: the value named ’name’ in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 132: `source_weight_shape=tuple(convolution.weight.shape),` This is continuation line 3 of the 15-line logical statement that begins at source line 130. Its exact fragment is ‘source_weight_shape=tuple(convolution.weight.shape),‘. The important operands are source_weight_shape: a learned coefficient multiplied by an activation; tuple: the value named ’tuple’ in this block; convolution: the value named ’convolution’ in this block. Read together, the complete operation is: This returns ‘ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.amin
accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro
folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)‘ to the caller of the function export_projection, which handles export projection, ending this invocation.

The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 133: `hardware_matrix_shape=tuple(matrix.shape),` This is continuation line 4 of the 15-line logical statement that begins at source line 130. Its exact fragment is 'hardware_matrix_shape=tuple(matrix.shape),'. The important operands are hardware_matrix_shape: the value named 'hardware matrix shape' in this block; tuple: the value named 'tuple' in this block; matrix: the value named 'matrix' in this block. Read together, the complete operation is: This returns 'ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.amin(
accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro
folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)' to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 134: `input_encoding=input_encoding,` This is continuation line 5 of the 15-line logical statement that begins at source line 130. Its exact fragment is 'input_encoding=input_encoding,'. The important operands are input_encoding: the value named 'input encoding' in this block. Read together, the complete operation is: This returns 'ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weig
accumulator_factor_minimum=float(accumulator_factor.amin().item()), accumulator_factor_maximum=float(accumulator_fact
represented_factor_maximum_error=float(factor_error.abs().amax().item()), folded_bias_minimum=int(bias_integer.amin().ite
folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)' to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 135: `compute_opcode=compute_opcode,` This is continuation line 6 of the 15-line logical statement that begins at source line 130. Its exact fragment is 'compute_opcode=compute_opcode,'. The important operands are compute_opcode: the value named 'compute opcode' in this block. Read together, the complete operation is: This returns 'ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.s
input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=we
accumulator_factor_minimum=float(accumulator_factor.amin().item()), accumulator_factor_maximum=float(accumulator_fact
represented_factor_maximum_error=float(factor_error.abs().amax().item()), folded_bias_minimum=int(bias_integer.amin().ite
folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)' to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 136: `output_fractional_bits=fractional_bits,` This is continuation line 7 of the 15-line logical statement that begins at source line 130. Its exact fragment is 'output_fractional_bits=fractional_bits,'. The important operands are output_fractional_bits: the value named 'output fractional bits' in this block; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This returns 'ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_e
compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_m
accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro
folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)' to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 137: `weight_scale=weight_scale,` This is continuation line 8 of the 15-line logical statement that begins at source line 130. Its exact fragment is 'weight_scale=weight_scale,'. The important operands are weight_scale: a learned coefficient multiplied by an activation. Read together, the complete operation is: This returns 'ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weig

accumulator_factor_minimum=float(accumulator_factor.amin().item()), accumulator_factor_maximum=float(accumulator_fact represented_factor_maximum_error=float(factor_error.abs().amax().item()), folded_bias_minimum=int(bias_integer.amin().ite folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)‘ to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 138: `accumulator_factor_minimum=float(accumulator_factor.amin().item()),` This is continuation line 9 of the 15-line logical statement that begins at source line 130. Its exact fragment is ‘accumulator_factor_minimum=float(accumulato The important operands are accumulator_factor_minimum: a widened running sum that prevents intermediate overflow; float: the value named 'float' in this block; accumulator_factor: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This returns ‘ProjectionExport(name=name, source_weight_shape=tuple(convolut hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.ar accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)‘ to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 139: `accumulator_factor_maximum=float(accumulator_factor.amax().item()),` This is continuation line 10 of the 15-line logical statement that begins at source line 130. Its exact fragment is ‘accumulator_factor_maximum=float(accumul The important operands are accumulator_factor_maximum: a widened running sum that prevents intermediate overflow; float: the value named 'float' in this block; accumulator_factor: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This returns ‘ProjectionExport(name=name, source_weight_shape=tuple(convolut hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.ar accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)‘ to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 140: `represented_factor_maximum_error=float(factor_error.abs().amax().item()),` This is continuation line 11 of the 15-line logical statement that begins at source line 130. Its exact fragment is ‘represented_factor_maximum_error= The important operands are represented_factor_maximum_error: the value named 'represented factor maximum error' in this block; float: the value named 'float' in this block; factor_error: the value named 'factor error' in this block. Read together, the complete operation is: This returns ‘ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.s hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.ar accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)‘ to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 141: `folded_bias_minimum=int(bias_integer.amin().item()),` This is continuation line 12 of the 15-line logical statement that begins at source line 130. Its exact fragment is ‘folded_bias_minimum=int(bias_integer.amin().item()),‘. The important operands are folded_bias_minimum: an additive learned offset applied after accumulation; bias_integer: an additive learned offset applied after accumulation; amin: the value named 'amin' in this block. Read together, the complete operation is: This returns ‘ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.ar accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)‘ to the caller of the function export_projection, which handles export projection, ending this invocation.

The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 142: `folded_bias_maximum=int(bias_integer.amax().item()),` This is continuation line 13 of the 15-line logical statement that begins at source line 130. Its exact fragment is 'folded_bias_maximum=int(bias_integer.amax().item()),'. The important operands are folded_bias_maximum: an additive learned offset applied after accumulation; bias_integer: an additive learned offset applied after accumulation; amax: the value named 'amax' in this block. Read together, the complete operation is: This returns 'ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.an accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)' to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 143: `sections=sections,` This is continuation line 14 of the 15-line logical statement that begins at source line 130. Its exact fragment is 'sections=sections,'. The important operands are sections: the value named 'sections' in this block. Read together, the complete operation is: This returns 'ProjectionExport(name=name, source_weight_shape=tuple(convolu hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.an accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)' to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 144: `)` This is continuation line 15 of the 15-line logical statement that begins at source line 130. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'ProjectionExport(name=name, source_weight_shape=tuple(convolution.weight.shape), hardware_matrix_shape=tuple(matrix.shape), input_encoding=input_encoding, compute_opcode=compute_opcode, output_fractional_bits=fractional_bits, weight_scale=weight_scale, accumulator_factor_minimum=float(accumulator_factor.amin accumulator_factor_maximum=float(accumulator_factor.amax().item()), represented_factor_maximum_error=float(factor_erro folded_bias_minimum=int(bias_integer.amin().item()), folded_bias_maximum=int(bias_integer.amax().item()), sections=sections,)' to the caller of the function export_projection, which handles export projection, ending this invocation. The important operands are ProjectionExport: the value named 'projection export' in this block; name: the value named 'name' in this block; source_weight_shape: a learned coefficient multiplied by an activation.

Line 147: `def projection_pairs(model: Spikingformer) -> list[tuple[str, nn.Module, nn.Module]]:` This starts function projection_pairs. Its indented body performs projection pairs as part of the file that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections; it runs only when called.

Line 148: `pairs: list[tuple[str, nn.Module, nn.Module]] = [` This statement continues through source line 153; the explanation uses the complete expression rather than only the first visual fragment. This assigns '[(f"patch_embed.block{ind getattr(model.patch_embed, f"block{index}_conv"), getattr(model.patch_embed, f"block{index}_bn")) for index in range(5)]' to 'pairs' in host memory. Here pairs means the value named 'pairs' in this block. The important operands are f: the value named 'f' in this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this block. Context: the function projection_pairs, which handles projection pairs.

Line 149: `(f"patch_embed.block{index}",` This is continuation line 2 of the 6-line logical statement that begins at source line 148. Its exact fragment is '(f"patch_embed.block{index}",'. The important operands are f: the value named 'f' in this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this block. Read together, the complete operation is: This assigns '[(f"patch_embed.block{index}", getattr(model.patch_embed, f"block{index}_conv"), getattr(model.patch_embed, f"block{index}_bn")) for index in range(5)]' to 'pairs' in host memory. Here pairs means the value named 'pairs' in this block. The important operands are f: the value named 'f' in

this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this block. Context: the function projection_pairs, which handles projection pairs.

Line 150: `getattr(model.patch_embed, f"block{index}_conv"),` This is continuation line 3 of the 6-line logical statement that begins at source line 148. Its exact fragment is 'getattr(model.patch_embed, f"block{index}_conv"),'. The important operands are getattr: the value named 'getattr' in this block; model: the value named 'model' in this block; patch_embed: the value named 'patch embed' in this block. Read together, the complete operation is: This assigns '[(f"patch_embed.block{index}", getattr(model.patch_embed, f"block{index}_conv"), getattr(model.patch_embed, f"block{index}_ for index in range(5)]' to 'pairs' in host memory. Here pairs means the value named 'pairs' in this block. The important operands are f: the value named 'f' in this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this block. Context: the function projection_pairs, which handles projection pairs.

Line 151: `getattr(model.patch_embed, f"block{index}_bn"))` This is continuation line 4 of the 6-line logical statement that begins at source line 148. Its exact fragment is 'getattr(model.patch_embed, f"block{index}_bn"))'. The important operands are getattr: the value named 'getattr' in this block; model: the value named 'model' in this block; patch_embed: the value named 'patch embed' in this block. Read together, the complete operation is: This assigns '[(f"patch_embed.block{index}", getattr(model.patch_embed, f"block{index}_conv"), getattr(model.patch_embed, f"block{index}_ for index in range(5)]' to 'pairs' in host memory. Here pairs means the value named 'pairs' in this block. The important operands are f: the value named 'f' in this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this block. Context: the function projection_pairs, which handles projection pairs.

Line 152: `for index in range(5)` This is continuation line 5 of the 6-line logical statement that begins at source line 148. Its exact fragment is 'for index in range(5)'. The important operands are for: the value named 'for' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; in: the value named 'in' in this block. Read together, the complete operation is: This assigns '[(f"patch_embed.block{index}", getattr(model.patch_embed, f"block{index}_conv"), getattr(model.patch_embed, f"block{index}_bn")) for index in range(5)]' to 'pairs' in host memory. Here pairs means the value named 'pairs' in this block. The important operands are f: the value named 'f' in this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this block. Context: the function projection_pairs, which handles projection pairs.

Line 153: `]` This is continuation line 6 of the 6-line logical statement that begins at source line 148. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '[(f"patch_embed.block{index}", getattr(model.patch_embed, f"block{index}_conv"), getattr(model.patch_embed, f"block{index}_bn")) for index in range(5)]' to 'pairs' in host memory. Here pairs means the value named 'pairs' in this block. The important operands are f: the value named 'f' in this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this block. Context: the function projection_pairs, which handles projection pairs.

Line 154: `for index, block in enumerate(model.block):` This iterates index, block over 'enumerate(model.block)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function projection_pairs, which handles projection pairs.

Line 155: `for name in ("q", "k", "v", "proj"):` This iterates name over '("q", "k", "v", "proj")'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function projection_pairs, which handles projection pairs.

Line 156: `pairs.append((` This statement continues through source line 160; the explanation uses the complete expression rather than only the first visual fragment. This calls 'pairs.append' with '(f"block.{index}.attn.{name}", getattr(block.attn, f"{name}_conv"), getattr(block.attn, f"{name}_bn"),)' inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 157: `f"block.{index}.attn.{name}",` This is continuation line 2 of the 5-line logical statement that begins at source line 156. Its exact fragment is 'f"block.{index}.attn.{name}",'. The important operands are f: the value named 'f' in this block; block: the value named 'block' in this block; index: an integer selecting one tensor element, lane, bank,

or state entry. Read together, the complete operation is: This calls ‘pairs.append‘ with ‘(f"block.{index}.attn.{name}", getattr(block.attn, f"{name}_conv"), getattr(block.attn, f"{name}_bn"),)‘ inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 158: `getattr(block.attn, f"{name}_conv"),` This is continuation line 3 of the 5-line logical statement that begins at source line 156. Its exact fragment is ‘getattr(block.attn, f"{name}_conv"),‘. The important operands are getattr: the value named ’getattr’ in this block; block: the value named ’block’ in this block; attn: the value named ’attn’ in this block. Read together, the complete operation is: This calls ‘pairs.append‘ with ‘(f"block.{index}.attn.{name}", getattr(block.attn, f"{name}_conv"), getattr(block.attn, f"{name}_bn"),)‘ inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 159: `getattr(block.attn, f"{name}_bn"),` This is continuation line 4 of the 5-line logical statement that begins at source line 156. Its exact fragment is ‘getattr(block.attn, f"{name}_bn"),‘. The important operands are getattr: the value named ’getattr’ in this block; block: the value named ’block’ in this block; attn: the value named ’attn’ in this block. Read together, the complete operation is: This calls ‘pairs.append‘ with ‘(f"block.{index}.attn.{name}", getattr(block.attn, f"{name}_conv"), getattr(block.attn, f"{name}_bn"),)‘ inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 160: `))` This is continuation line 5 of the 5-line logical statement that begins at source line 156. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘pairs.append‘ with ‘(f"block.{index}.attn.{name}", getattr(block.attn, f"{name}_conv"), getattr(block.attn, f"{name}_bn"),)‘ inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 161: `for layer in (1, 2):` This iterates layer over ‘(1, 2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function projection_pairs, which handles projection pairs.

Line 162: `pairs.append((` This statement continues through source line 166; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘pairs.append‘ with ‘(f"block.{index}.mlp.fc{layer}", getattr(block.mlp, f"mlp{layer}_conv"), getattr(block.mlp, f"mlp{layer}_bn"),)‘ inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 163: `f"block.{index}.mlp.fc{layer}",` This is continuation line 2 of the 5-line logical statement that begins at source line 162. Its exact fragment is ‘f"block.{index}.mlp.fc{layer}",‘. The important operands are f: the value named ’f’ in this block; block: the value named ’block’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘pairs.append‘ with ‘(f"block.{index}.mlp.fc{layer}", getattr(block.mlp, f"mlp{layer}_conv"), getattr(block.mlp, f"mlp{layer}_bn"),)‘ inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 164: `getattr(block.mlp, f"mlp{layer}_conv"),` This is continuation line 3 of the 5-line logical statement that begins at source line 162. Its exact fragment is ‘getattr(block.mlp, f"mlp{layer}_conv"),‘. The important operands are getattr: the value named ’getattr’ in this block; block: the value named ’block’ in this block; mlp: the value named ’mlp’ in this block. Read together, the complete operation is: This calls ‘pairs.append‘ with ‘(f"block.{index}.mlp.fc{layer}", getattr(block.mlp, f"mlp{layer}_conv"), getattr(block.mlp, f"mlp{layer}_bn"),)‘ inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 165: `getattr(block.mlp, f"mlp{layer}_bn"),` This is continuation line 4 of the 5-line logical statement that begins at source line 162. Its exact fragment is ‘getattr(block.mlp, f"mlp{layer}_bn"),‘. The important operands are

getattr: the value named 'getattr' in this block; block: the value named 'block' in this block; mlp: the value named 'mlp' in this block. Read together, the complete operation is: This calls ‘pairs.append‘ with ‘(f"block.{index}.mlp.fc{layer}", getattr(block.mlp, f"mlp{layer}_conv"), getattr(block.mlp, f"mlp{layer}_bn"),)‘ inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 166: `))` This is continuation line 5 of the 5-line logical statement that begins at source line 162. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘pairs.append‘ with ‘(f"block.{index}.mlp.fc{layer}", getattr(block.mlp, f"mlp{layer}_conv"), getattr(block.mlp, f"mlp{layer}_bn"),)‘ inside the function projection_pairs, which handles projection pairs. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 167: `return pairs` This returns ‘pairs‘ to the caller of the function projection_pairs, which handles projection pairs, ending this invocation. The important operands are pairs: the value named 'pairs' in this block.

Line 170: `def export_model(` This starts function export_model. Its indented body performs export model as part of the file that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections; it runs only when called.

Line 171: `checkpoint_path: Path,` Inside the function export_model, which handles export model, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are checkpoint_path: the value named 'checkpoint path' in this block; Path: the value named 'path' in this block.

Line 172: `output_dir: Path,` Inside the function export_model, which handles export model, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are output_dir: the value named 'output dir' in this block; Path: the value named 'path' in this block.

Line 173: `*,` Inside the function export_model, which handles export model, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The expression is a literal or structural operator and has no hidden software state.

Line 174: `fractional_bits: int = 10,` This assigns ‘10,‘ to ‘fractional_bits‘ in host memory. Here fractional_bits means the value named 'fractional bits' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function export_model, which handles export model.

Line 175: `head_activation_scale: float = 0.125,` This assigns ‘0.125,‘ to ‘head_activation_scale‘ in host memory. Here head_activation_scale means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the function export_model, which handles export model.

Line 176: `) -> dict[str, object]:` Inside the function export_model, which handles export model, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are dict: the value named 'dict' in this block; str: the value named 'str' in this block; object: the value named 'object' in this block.

Line 177: `loaded = load_checkpoint(checkpoint_path, device="cpu")` This assigns ‘load_checkpoint(checkpoint_path, device="cpu")‘ to ‘loaded ‘ in host memory. Here loaded means the value named 'loaded' in this block. The important operands are load_checkpoint: the value named 'load checkpoint' in this block; checkpoint_path: the value named 'checkpoint path' in this block; device: the value named 'device' in this block. Context: the function export_model, which handles export model.

Line 178: `model = loaded.model` This assigns ‘loaded.model‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are loaded: the value named ’loaded’ in this block; model: the value named ’model’ in this block. Context: the function export_model, which handles export model.

Line 179: `writer = AlignedBinaryWriter()` This assigns ‘AlignedBinaryWriter()‘ to ‘writer ‘ in host memory. Here writer means the value named ’writer’ in this block. The important operands are AlignedBinaryWriter: the value named ’aligned binary writer’ in this block. Context: the function export_model, which handles export model.

Line 180: `input_probe = torch.zeros(1, 3, 1, 1)` This assigns ‘torch.zeros(1, 3, 1, 1)‘ to ‘input_probe ‘ in host memory. Here input_probe means the value named ’input probe’ in this block. The important operands are torch: the value named ’torch’ in this block; zeros: the value named ’zeros’ in this block. Context: the function export_model, which handles export model.

Line 181: `_, image_scale, image_zero_point = quantize_cifar_input_u8(input_probe)` This assigns ‘quantize_cifar_input_u8(input_probe)‘ to ‘_, image_scale, image_zero_point ‘ in host memory. Here _ means the value named ” in this block. The important operands are quantize_cifar_input_u8: the value named ’quantize cifar input u8’ in this block; input_probe: the value named ’input probe’ in this block. Context: the function export_model, which handles export model.

Line 182: `projections: list[ProjectionExport] = []` This assigns ‘[]‘ to ‘projections‘ in host memory. Here projections means the value named ’projections’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function export_model, which handles export model.

Line 183: `for name, convolution, batch_norm in projection_pairs(model):` This iterates name, convolution, batch_norm over ‘projection_pairs(model)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function export_model, which handles export model.

Line 184: `first_layer = name == "patch_embed.block0"` Inside the function export_model, which handles export model, this statement advances the host implementation that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The important operands are first_layer: the value named ’first layer’ in this block; name: the value named ’name’ in this block; patch_embed: the value named ’patch embed’ in this block.

Line 185: `projections.append(export_projection(` This statement continues through source line 196; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 186: `writer,` This is continuation line 2 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘writer,‘. The important operands are writer: the value named ’writer’ in this block. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 187: `name=name,` This is continuation line 3 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘name=name,‘. The important operands are name: the value named ’name’ in this block. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else

"binary spike 0/1"), compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 188: `convolution=convolution,` This is continuation line 4 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘convolution=convolution,‘. The important operands are convolution: the value named ’convolution’ in this block. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:. if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 189: `batch_norm=batch_norm,` This is continuation line 5 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘batch_norm=batch_norm,‘. The important operands are batch_norm: the value named ’batch norm’ in this block. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 190: `input_scale=image_scale if first_layer else 1.0,` This is continuation line 6 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘input_scale=image_scale if first_layer else 1.0,‘. The important operands are input_scale: the real-value spacing represented by one quantized integer step; image_scale: the real-value spacing represented by one quantized integer step; first_layer: the value named ’first layer’ in this block. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 191: `fractional_bits=fractional_bits,` This is continuation line 7 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘fractional_bits=fractional_bits,‘. The important operands are fractional_bits: the value named ’fractional bits’ in this block. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 192: `input_encoding=(` This is continuation line 8 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘input_encoding=(‘. The important operands are input_encoding: the value named ’input encoding’ in this block. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 193: `f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}"` This is continuation line 9 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}"‘. The important operands are f: the value named ’f’ in this

block; asymmetric: the value named 'asymmetric' in this block; uint8: the value named 'uint8' in this block. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 194: `if first_layer else "binary spike 0/1"),` This is continuation line 10 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘if first_layer else "binary spike 0/1"),‘. The important operands are first_layer: the value named 'first layer' in this block; binary: the value named 'binary' in this block; spike: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 195: `compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),` This is continuation line 11 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘compute_opcode=("COMPUTE_DI if first_layer else "COMPUTE_BINARY"),‘. The important operands are compute_opcode: the value named 'compute opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block; first_layer: the value named 'first layer' in this block. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 196: `))` This is continuation line 12 of the 12-line logical statement that begins at source line 185. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘projections.append‘ with ‘export_projection(writer, name=name, convolution=convolution, batch_norm=batch_norm, input_scale=image_scale if first_layer else 1.0, fractional_bits=fractional_bits, input_encoding=(f"asymmetric uint8 zero_point={image_zero_point} scale={image_scale:.12g}" if first_layer else "binary spike 0/1"), compute_opcode=("COMPUTE_DENSE_OUTER" if first_layer else "COMPUTE_BINARY"),)‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 198: `head_weight, head_weight_scale = quantize_weight_tensor(model.head.weight)` This assigns ‘quantize_weight_tensor(model.head.weight)‘ to ‘head_weight, head_weight_scale ‘ in host memory. Here head_weight means a learned coefficient multiplied by an activation. The important operands are quantize_weight_tensor: a learned coefficient multiplied by an activation; model: the value named 'model' in this block; head: the value named 'head' in this block. Context: the function export_model, which handles export model.

Line 199: `head_matrix = head_weight.transpose(0, 1).contiguous().cpu().numpy()` This assigns ‘head_weight.transpos 1).contiguous().cpu().numpy()‘ to ‘head_matrix ‘ in host memory. Here head_matrix means the value named 'head matrix' in this block. The important operands are head_weight: a learned coefficient multiplied by an activation; transpose: the value named 'transpose' in this block; contiguous: the value named 'contiguous' in this block. Context: the function export_model, which handles export model.

Line 200: `head_accumulator_scale = head_activation_scale * head_weight_scale` This assigns ‘head_activation_scale * head_weight_scale‘ to ‘head_accumulator_scale ‘ in host memory. Here head_accumulator_scale means a widened running sum that prevents intermediate overflow. The important operands are head_activation_scale: an input feature

value consumed by a neuron or multiply-accumulate operation; head_weight_scale: a learned coefficient multiplied by an activation. Context: the function export_model, which handles export model.

Line 201: `head_bias = torch.round(` This statement continues through source line 202; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘torch.round(model.head.bias.detach() / head_accumulator_scale).to(torch.int32).cpu().numpy()‘ to ‘head_bias ‘ in host memory. Here head_bias means an additive learned offset applied after accumulation. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; model: the value named ’model’ in this block. Context: the function export_model, which handles export model.

Line 202: `model.head.bias.detach() / head_accumulator_scale).to(torch.int32).cpu().numpy()` This is continuation line 2 of the 2-line logical statement that begins at source line 201. Its exact fragment is ‘model.head.bias.detach() / head_accumulator_scale).to(torch.int32).cpu().numpy()‘. The important operands are model: the value named ’model’ in this block; head: the value named ’head’ in this block; bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘torch.round(model.head.bias.detach() / head_accumulator_scale).to(torch.int32).cpu().numpy() to ‘head_bias ‘ in host memory. Here head_bias means an additive learned offset applied after accumulation. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; model: the value named ’model’ in this block. Context: the function export_model, which handles export model.

Line 203: `head_sections = (` This statement continues through source line 206; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(writer.append("head.weight_k_by_n_int8", head_matrix), writer.append("head.bias_accumulator_units", head_bias),)‘ to ‘head_sections ‘ in host memory. Here head_sections means the value named ’head sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; head: the value named ’head’ in this block. Context: the function export_model, which handles export model.

Line 204: `writer.append("head.weight_k_by_n_int8", head_matrix),` This is continuation line 2 of the 4-line logical statement that begins at source line 203. Its exact fragment is ‘writer.append("head.weight_k_by_n_int8", head_matrix),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; head: the value named ’head’ in this block. Read together, the complete operation is: This assigns ‘(writer.append("head.weight_k_by_n_int8", head_matrix), writer.append("head.bias_accumulator_units", head_bias),)‘ to ‘head_sections ‘ in host memory. Here head_sections means the value named ’head sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; head: the value named ’head’ in this block. Context: the function export_model, which handles export model.

Line 205: `writer.append("head.bias_accumulator_units", head_bias),` This is continuation line 3 of the 4-line logical statement that begins at source line 203. Its exact fragment is ‘writer.append("head.bias_accumulator_units", head_bias),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; head: the value named ’head’ in this block. Read together, the complete operation is: This assigns ‘(writer.append("head.weight_k_by_n_int8", head_matrix), writer.append("head.bias_accumulator_units", head_bias),)‘ to ‘head_sections ‘ in host memory. Here head_sections means the value named ’head sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; head: the value named ’head’ in this block. Context: the function export_model, which handles export model.

Line 206: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 203. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(writer.append("head.weight_k_by_n_int8", head_matrix), writer.append("head.bias_accumulator_units", head_bias),)‘ to ‘head_sections ‘ in host memory. Here head_sections means the value named ’head sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; head: the value named ’head’ in this block. Context: the function export_model, which handles export model.

Line 207: `checkpoint_hash = hashlib.sha256(checkpoint_path.read_bytes()).hexdigest()` This assigns ‘hashlib.sha256(checkpoint_path.read_bytes()).hexdigest()‘ to ‘checkpoint_hash ‘ in host memory. Here checkpoint_hash means the value named ’checkpoint hash’ in this block. The important operands are hashlib: the value named ’hashlib’

in this block; sha256: the value named 'sha256' in this block; checkpoint_path: the value named 'checkpoint path' in this block. Context: the function export_model, which handles export model.

Line 208: `output_dir.mkdir(parents=True, exist_ok=True)` This calls 'output_dir.mkdir' with 'parents=True, exist_ok=True' inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 209: `binary_path = output_dir / "spikingformer_neuraedge_weights.bin"` This assigns 'output_dir / "spikingformer_neuraedge_weights.bin"' to 'binary_path ' in host memory. Here binary_path means the value named 'binary path' in this block. The important operands are output_dir: the value named 'output dir' in this block; spikingformer_neuraedge_weights: a learned coefficient multiplied by an activation; bin: the value named 'bin' in this block. Context: the function export_model, which handles export model.

Line 210: `binary_path.write_bytes(writer.data)` This calls 'binary_path.write_bytes' with 'writer.data' inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 211: `binary_hash = hashlib.sha256(writer.data).hexdigest()` This assigns 'hashlib.sha256(writer.data).hexdigest()' to 'binary_hash ' in host memory. Here binary_hash means the value named 'binary hash' in this block. The important operands are hashlib: the value named 'hashlib' in this block; sha256: the value named 'sha256' in this block; writer: the value named 'writer' in this block. Context: the function export_model, which handles export model.

Line 212: `manifest: dict[str, object] = {` This statement continues through source line 245; the explanation uses the complete expression rather than only the first visual fragment. This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 213: `"format": "neuraedge-spikingformer-parameters-v1",` This is continuation line 2 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"format": "neuraedge-spikingformer-parameters-v1",'. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format:

the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 214: `"checkpoint": str(checkpoint_path.resolve()),` This is continuation line 3 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"checkpoint": str(checkpoint_path.resolve()),'. The important operands are checkpoint: the value named 'checkpoint' in this block; str: the value named 'str' in this block; checkpoint_path: the value named 'checkpoint path' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 215: `"checkpoint_sha256": checkpoint_hash,` This is continuation line 4 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"checkpoint_sha256": checkpoint_hash,'. The important operands are checkpoint_sha256: the value named 'checkpoint sha256' in this block; checkpoint_hash: the value named 'checkpoint hash' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 216: `"checkpoint_epoch": loaded.epoch,` This is continuation line 5 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"checkpoint_epoch": loaded.epoch,'. The important operands are checkpoint_epoch: the value named 'checkpoint epoch' in this block; loaded: the value named 'loaded' in this block; epoch: the value named 'epoch' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scal
head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.we
"hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero
point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT
"weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()),
"bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item)
for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block.
The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in

this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 217: `"checkpoint_metric_percent": loaded.metric,` This is continuation line 6 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"checkpoint_metric_percent": loaded.metric,'. The important operands are checkpoint_metric_percent: the value named 'checkpoint metric percent' in this block; loaded: the value named 'loaded' in this block; metric: the value named 'metric' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 218: `"binary_file": binary_path.name,` This is continuation line 7 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"binary_file": binary_path.name,'. The important operands are binary_file: the value named 'binary file' in this block; binary_path: the value named 'binary path' in this block; name: the value named 'name' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scal
head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.we
"hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero
point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT
"weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()),
"bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item)
for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block.
The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in
this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which
handles export model.

Line 219: `"binary_bytes": len(writer.data),` This is continuation line 8 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"binary_bytes": len(writer.data),'. The important operands are binary_bytes: the value named 'binary bytes' in this block; len: the value named 'len' in this block; writer: the value named 'writer' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in

this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 220: `"binary_sha256": binary_hash,` This is continuation line 9 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"binary_sha256": binary_hash,'. The important operands are binary_sha256: the value named 'binary sha256' in this block; binary_hash: the value named 'binary hash' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_p loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 221: `"alignment_bytes": ALIGNMENT,` This is continuation line 10 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"alignment_bytes": ALIGNMENT,'. The important operands are alignment_bytes: the value named 'alignment bytes' in this block; ALIGNMENT: the value named 'alignment' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 222: `"fixed_point": {` This is continuation line 11 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"fixed_point": {'. The important operands are fixed_point: the value named 'fixed point' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 223: `"activation_fractional_bits": fractional_bits,` This is continuation line 12 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"activation_fractional_bits": fractional_bits,'. The important operands are activation_fractional_bits: an input feature value consumed by a neuron or multiply-accumulate operation; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 224: `"activation_scale": 1 << fractional_bits,` This is continuation line 13 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"activation_scale": 1 << fractional_bits,'. The important operands are activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 225: `"lif_membrane_bits": 24,` This is continuation line 14 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"lif_membrane_bits": 24,'. The important operands are lif_membrane_bits: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_s
head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.we
"hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero
point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT
"weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()),
"bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item)
for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block.
The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in
this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which
handles export model.

Line 226: `"lif_current_bits": 16,` This is continuation line 15 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"lif_current_bits": 16,‘. The important operands are lif_current_bits: the value named ’lif current bits’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scal head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.we "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function export_model, which handles export model.

Line 227: `"image_input_scale": image_scale,` This is continuation line 16 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"image_input_scale": image_scale,‘. The important operands are image_input_scale: the real-value spacing represented by one quantized integer step; image_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scal head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.we "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function export_model, which handles export model.

Line 228: `"image_input_zero_point": image_zero_point,` This is continuation line 17 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"image_input_zero_point": image_zero_point,‘. The important operands are image_input_zero_point: the integer code representing real numeric zero in affine quantization; image_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function export_model, which handles export model.

Line 229: `"head_activation_scale": head_activation_scale,` This is continuation line 18 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"head_activation_scale": head_activation_scale,'. The important operands are head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 230: `},` This is continuation line 19 of the 34-line logical statement that begins at source line 212. Its exact fragment is '},'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_p
loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 231: `"projections": [asdict(item) for item in projections],` This is continuation line 20 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"projections": [asdict(item) for item in projections],'. The important operands are projections: the value named 'projections' in this block; asdict: the value named 'asdict' in this block; item: the value named 'item' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 232: `"head": {` This is continuation line 21 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"head": {'. The important operands are head: the value named 'head' in this block. Read

together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_p loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function export_model, which handles export model.

Line 233: `"source_weight_shape": list(model.head.weight.shape),` This is continuation line 22 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"source_weight_shape": list(model.head.weight.shape),‘. The important operands are source_weight_shape: a learned coefficient multiplied by an activation; list: the value named ’list’ in this block; model: the value named ’model’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function export_model, which handles export model.

Line 234: `"hardware_matrix_shape": list(head_matrix.shape),` This is continuation line 23 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"hardware_matrix_shape": list(head_matrix.shape),‘. The important operands are hardware_matrix_shape: the value named ’hardware matrix shape’ in this block; list: the value named ’list’ in this block; head_matrix: the value named ’head matrix’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function export_model, which handles export model.

Line 235: `"activation_encoding": "signed int8 encoded as uint8 with zero point 128",` This is continuation line 24 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"activation_encoding": "signed int8 encoded as uint8 with zero point 128",‘. The important operands are activation_encoding: an input feature value consumed by a neuron or multiply-accumulate operation; int8: the value named ’int8’ in this block; en-

coded: the value named 'encoded' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scal head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.we "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 236: `"weight_encoding": "signed int8 encoded as uint8 with zero point 128",` This is continuation line 25 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"weight_encoding": "signed int8 encoded as uint8 with zero point 128",'. The important operands are weight_encoding: a learned coefficient multiplied by an activation; int8: the value named 'int8' in this block; encoded: the value named 'encoded' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_p loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 237: `"compute_opcode": "COMPUTE_DENSE_DOT8",` This is continuation line 26 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"compute_opcode": "COMPUTE_DENSE_DOT8",'. The important operands are compute_opcode: the value named 'compute opcode' in this block; COMPUTE_DENSE_DOT8: the value named 'compute dense dot8' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_s head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.we "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }' to 'manifest' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 238: `"weight_scale": head_weight_scale,` This is continuation line 27 of the 34-line logical statement that begins at source line 212. Its exact fragment is '"weight_scale": head_weight_scale,'. The important operands are weight_scale: a learned coefficient multiplied by an activation; head_weight_scale: a learned coefficient multiplied by an

activation. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function export_model, which handles export model.

Line 239: `"accumulator_scale": head_accumulator_scale,` This is continuation line 28 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"accumulator_scale": head_accumulator_scale,‘. The important operands are accumulator_scale: a widened running sum that prevents intermediate overflow; head_accumulator_scale: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_s
head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.we
"hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero
point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT
"weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()),
"bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item)
for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block.
The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in
this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function export_model, which
handles export model.

Line 240: `"bias_minimum": int(head_bias.min()),` This is continuation line 29 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"bias_minimum": int(head_bias.min()),‘. The important operands are bias_minimum: an additive learned offset applied after accumulation; head_bias: an additive learned offset applied after accumulation; min: the value named ’min’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function export_model, which handles export model.

Line 241: `"bias_maximum": int(head_bias.max()),` This is continuation line 30 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"bias_maximum": int(head_bias.max()),‘. The important operands are bias_maximum: an additive learned offset applied after accumulation; head_bias: an additive learned offset ap-

plied after accumulation; max: the value named 'max' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 242: `"sections": [asdict(item) for item in head_sections],` This is continuation line 31 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"sections": [asdict(item) for item in head_sections],‘. The important operands are sections: the value named 'sections' in this block; asdict: the value named 'asdict' in this block; item: the value named 'item' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 243: `},` This is continuation line 32 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘},‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_p
loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 244: `"sections": [asdict(item) for item in writer.sections],` This is continuation line 33 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘"sections": [asdict(item) for item in writer.sections],‘. The important operands are sections: the value named 'sections' in this block; asdict: the value named 'asdict' in this block; item: the value named 'item' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "check-

point_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_percent": loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 245: `}` This is continuation line 34 of the 34-line logical statement that begins at source line 212. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-parameters-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_sha256": checkpoint_hash, "checkpoint_epoch": loaded.epoch, "checkpoint_metric_p
loaded.metric, "binary_file": binary_path.name, "binary_bytes": len(writer.data), "binary_sha256": binary_hash, "alignment_bytes": ALIGNMENT, "fixed_point": { "activation_fractional_bits": fractional_bits, "activation_scale": 1 << fractional_bits, "lif_membrane_bits": 24, "lif_current_bits": 16, "image_input_scale": image_scale, "image_input_zero_point": image_zero_point, "head_activation_scale": head_activation_scale, }, "projections": [asdict(item) for item in projections], "head": { "source_weight_shape": list(model.head.weight.shape), "hardware_matrix_shape": list(head_matrix.shape), "activation_encoding": "signed int8 encoded as uint8 with zero point 128", "weight_encoding": "signed int8 encoded as uint8 with zero point 128", "compute_opcode": "COMPUTE_DENSE_DOT8", "weight_scale": head_weight_scale, "accumulator_scale": head_accumulator_scale, "bias_minimum": int(head_bias.min()), "bias_maximum": int(head_bias.max()), "sections": [asdict(item) for item in head_sections], }, "sections": [asdict(item) for item in writer.sections], }‘ to ‘manifest‘ in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function export_model, which handles export model.

Line 246: `manifest_path = output_dir / "spikingformer_neuraedge_weights.json"` This assigns ‘output_dir / "spikingformer_neuraedge_weights.json"‘ to ‘manifest_path ‘ in host memory. Here manifest_path means the value named 'manifest path' in this block. The important operands are output_dir: the value named 'output dir' in this block; spikingformer_neuraedge_weights: a learned coefficient multiplied by an activation; json: the value named 'json' in this block. Context: the function export_model, which handles export model.

Line 247: `manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="ascii")` This calls ‘manifest_path.write_text‘ with ‘json.dumps(manifest, indent=2) + "\n", encoding="ascii"‘ inside the function export_model, which handles export model. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 248: `return manifest` This returns ‘manifest‘ to the caller of the function export_model, which handles export model, ending this invocation. The important operands are manifest: the value named 'manifest' in this block.

Line 251: `def parse_arguments() -> argparse.Namespace:` This starts function parse_arguments. Its indented body performs parse arguments as part of the file that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections; it runs only when called.

Line 252: `parser = argparse.ArgumentParser()` This assigns ‘argparse.ArgumentParser()‘ to ‘parser ‘ in host memory. Here parser means the value named 'parser' in this block. The important operands are argparse: the value named 'argparse' in this block; ArgumentParser: the value named 'argument parser' in this block. Context: the function parse_arguments, which handles parse arguments.

Line 253: `parser.add_argument("-checkpoint", required=True, type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 254: `parser.add_argument("-output-dir", required=True, type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 255: `parser.add_argument("-fractional-bits", type=int, default=10)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 256: `parser.add_argument("-head-activation-scale", type=float, default=0.125)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 257: `return parser.parse_args()` This returns ‘parser.parse_args()‘ to the caller of the function parse_arguments, which handles parse arguments, ending this invocation. The important operands are parser: the value named ’parser’ in this block; parse_args: the value named ’parse args’ in this block.

Line 260: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 261: `arguments = parse_arguments()` This assigns ‘parse_arguments()‘ to ‘arguments ‘ in host memory. Here arguments means the value named ’arguments’ in this block. The important operands are parse_arguments: the value named ’parse arguments’ in this block. Context: the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 262: `exported = export_model(` This statement continues through source line 267; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘export_model(arguments.checkpoint.resolve(), arguments.output_dir.resolve(), fractional_bits=arguments.fractional_bits, head_activation_scale=arguments.head_activation_s)‘ to ‘exported ‘ in host memory. Here exported means the value named ’exported’ in this block. The important operands are export_model: the value named ’export model’ in this block; arguments: the value named ’arguments’ in this block; checkpoint: the value named ’checkpoint’ in this block. Context: the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 263: `arguments.checkpoint.resolve(),` This is continuation line 2 of the 6-line logical statement that begins at source line 262. Its exact fragment is ‘arguments.checkpoint.resolve(),‘. The important operands are arguments: the value named ’arguments’ in this block; checkpoint: the value named ’checkpoint’ in this block; resolve: the value named ’resolve’ in this block. Read together, the complete operation is: This assigns ‘export_model(arguments.checkpoint.resolve(), arguments.output_dir.resolve(), fractional_bits=arguments.fractional_bits, head_activation_scale=arguments.head_activation_sca)‘ to ‘exported ‘ in host memory. Here exported means the value named ’exported’ in this block. The important operands are export_model: the value named ’export model’ in this block; arguments: the value named ’arguments’ in this block; checkpoint: the value named ’checkpoint’ in this block. Context: the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 264: `arguments.output_dir.resolve(),` This is continuation line 3 of the 6-line logical statement that begins at source line 262. Its exact fragment is ‘arguments.output_dir.resolve(),‘. The important operands are arguments: the value named ’arguments’ in this block; output_dir: the value named ’output dir’ in this block; resolve: the value named ’resolve’ in this block. Read together, the complete operation is: This assigns ‘export_model(arguments.checkpoint.resolve(), arguments.output_dir.resolve(), fractional_bits=arguments.fractional_bits, head_activation_scale=arguments.head_activation_sca)‘ to ‘exported ‘ in host memory. Here exported means the value named ’exported’ in this block. The important operands are export_model: the value named ’export model’ in this block; arguments: the value named ’arguments’ in this block;

checkpoint: the value named 'checkpoint' in this block. Context: the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 265: `fractional_bits=arguments.fractional_bits,` This is continuation line 4 of the 6-line logical statement that begins at source line 262. Its exact fragment is 'fractional_bits=arguments.fractional_bits,'. The important operands are fractional_bits: the value named 'fractional bits' in this block; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns 'export_model(arguments.checkpoint.resolve(), arguments.output_dir.resolve(), fractional_bits=arguments.fractional_bits, head_activation_scale=arguments.head_activation_scale,)' to 'exported ' in host memory. Here exported means the value named 'exported' in this block. The important operands are export_model: the value named 'export model' in this block; arguments: the value named 'arguments' in this block; checkpoint: the value named 'checkpoint' in this block. Context: the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 266: `head_activation_scale=arguments.head_activation_scale,` This is continuation line 5 of the 6-line logical statement that begins at source line 262. Its exact fragment is 'head_activation_scale=arguments.head_activation_scale,'. The important operands are head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns 'export_model(arguments.checkpoint.resolve(), arguments.output_dir.resolve(), fractional_bits=arguments.fractional_bits, head_activation_scale=arguments.head_activation_scale,)' to 'exported ' in host memory. Here exported means the value named 'exported' in this block. The important operands are export_model: the value named 'export model' in this block; arguments: the value named 'arguments' in this block; checkpoint: the value named 'checkpoint' in this block. Context: the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 267: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 262. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'export_model(arguments.checkpoint.resolve(), arguments.output_dir.resolve(), fractional_bits=arguments.fractional_bits, head_activation_scale=arguments.head_activation_scale,)' to 'exported ' in host memory. Here exported means the value named 'exported' in this block. The important operands are export_model: the value named 'export model' in this block; arguments: the value named 'arguments' in this block; checkpoint: the value named 'checkpoint' in this block. Context: the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 268: `print(json.dumps({` This statement continues through source line 273; the explanation uses the complete expression rather than only the first visual fragment. This calls 'print' with 'json.dumps({ "binary_file": exported["binary_file"], "binary_bytes": exported["binary_bytes"], "binary_sha256": exported["binary_sha256"], "projection_count": len(exported["projections"]), }, indent=2)' inside the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 269: `"binary_file": exported["binary_file"],` This is continuation line 2 of the 6-line logical statement that begins at source line 268. Its exact fragment is '"binary_file": exported["binary_file"],'. The important operands are binary_file: the value named 'binary file' in this block; exported: the value named 'exported' in this block. Read together, the complete operation is: This calls 'print' with 'json.dumps({ "binary_file": exported["binary_file"], "binary_bytes": exported["binary_bytes"], "binary_sha256": exported["binary_sha256"], "projection_count": len(exported["projections"]), }, indent=2)' inside the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 270: `"binary_bytes": exported["binary_bytes"],` This is continuation line 3 of the 6-line logical statement that begins at source line 268. Its exact fragment is '"binary_bytes": exported["binary_bytes"],'. The important operands are binary_bytes: the value named 'binary bytes' in this block; exported: the value named 'exported' in this block. Read together, the complete operation is: This calls 'print' with 'json.dumps({ "binary_file": exported["binary_file"], "binary_bytes": exported["binary_bytes"], "binary_sha256": exported["binary_sha256"], "projection_count": len(exported["projections"]), }, indent=2)' inside the file whose role is to folds BatchNorm, calibrates

quantizers, and writes aligned hashed hardware parameter sections. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 271: `"binary_sha256": exported["binary_sha256"],` This is continuation line 4 of the 6-line logical statement that begins at source line 268. Its exact fragment is ‘"binary_sha256": exported["binary_sha256"],‘. The important operands are binary_sha256: the value named ’binary sha256’ in this block; exported: the value named ’exported’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "binary_file": exported["binary_file"], "binary_bytes": exported["binary_bytes"], "binary_sha256": exported["binary_sha256"], "projection_count": len(exported["projections"]), }, indent=2)‘ inside the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 272: `"projection_count": len(exported["projections"]),` This is continuation line 5 of the 6-line logical statement that begins at source line 268. Its exact fragment is ‘"projection_count": len(exported["projections"]),‘. The important operands are projection_count: an integer count retained for control or measurable telemetry; len: the value named ’len’ in this block; exported: the value named ’exported’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "binary_file": exported["binary_file"], "binary_bytes": exported["binary_bytes"], "binary_sha256": exported["binary_sha256"], "projection_count": len(exported["projections"]), }, indent=2)‘ inside the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

Line 273: `}, indent=2))` This is continuation line 6 of the 6-line logical statement that begins at source line 268. Its exact fragment is ‘}, indent=2))‘. The important operands are indent: the value named ’indent’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘json.dumps({ "binary_file": exported["binary_file"], "binary_bytes": exported["binary_bytes"], "binary_sha256": exported["binary_sha256"], "projection_count": len(exported["projection
}, indent=2)‘ inside the file whose role is to folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections. The call advances the host-side job that folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections.

28 software/generate_debug_report.py

Role in the complete system: turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

28.1 Beginner architecture map

Dataflow responsibility This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: PlotRecord, VcdSignal. Functions: root_dir, load_json, style_axes, save_figure, text_page, confusion_figure, parse_vcd, choose_wave_signals, waveform_figure, arithmetic_groups, parse_timing_slacks, parse_implementation_summary, parse_clock_timing, parse_utilization_hierarchy, parse_drc_summary, parse_cdc_summ parse_power_report, file_evidence, latex_escape, latex_breakable, write_report_tex, generate, compile_tex, main. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

28.2 Complete source

```
"""Generate model, compiler, waveform, timing, utilization, and power plots."""

from __future__ import annotations

import argparse
import hashlib
import json
import math
import re
import shutil
import subprocess
from collections import Counter, defaultdict
from dataclasses import dataclass
from pathlib import Path
from typing import Callable, Iterable, Sequence

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.backends.backend_pdf import PdfPages


CLASSES = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck",
]
COLORS = ["#155e75", "#166534", "#991b1b", "#7c3e00", "#4f46e5", "#374151"]


@dataclass
class PlotRecord:
    filename: str
    title: str
    evidence: str


@dataclass
class VcdSignal:
    name: str
    width: int
    transitions: list[tuple[int, float]]


def root_dir() -> Path:
    return Path(__file__).resolve().parents[1]


```

```
def load_json(path: Path) -> dict:
    return json.loads(path.read_text(encoding="utf-8-sig"))


def style_axes(axis: plt.Axes, title: str, xlabel: str = "", ylabel: str = "") -> None:
    axis.set_title(title, fontsize=12, fontweight="bold", loc="left")
    axis.set_xlabel(xlabel)
    axis.set_ylabel(ylabel)
    axis.grid(True, axis="y", alpha=0.22, linewidth=0.7)
    axis.spines[["top", "right"]].set_visible(False)


def save_figure(
    figure: plt.Figure,
    output_dir: Path,
    pdf: PdfPages,
    records: list[PlotRecord],
    index: int,
    slug: str,
    title: str,
    evidence: str,
) -> int:
    figure.tight_layout()
    filename = f"{index:02d}_{slug}.png"
    figure.savefig(output_dir / filename, dpi=190, bbox_inches="tight")
    pdf.savefig(figure, bbox_inches="tight")
    plt.close(figure)
    records.append(PlotRecord(filename, title, evidence))
    return index + 1


def text_page(title: str, lines: Sequence[str]) -> plt.Figure:
    figure = plt.figure(figsize=(11.69, 8.27))
    figure.patch.set_facecolor("white")
    figure.text(0.055, 0.93, title, fontsize=20, fontweight="bold", color="#111827")
    y = 0.86
    for line in lines:
        wrapped = [line[start : start + 125] for start in range(0, len(line), 125)] or [""]
        for segment in wrapped:
            figure.text(0.065, y, segment, fontsize=10.5, family="monospace", color="#1f2937")
            y -= 0.038
        y -= 0.008
    return figure


def confusion_figure(matrix: np.ndarray, title: str, signed: bool = False) -> plt.Figure:
    figure, axis = plt.subplots(figsize=(9.5, 8.2))
    if signed:
        limit = max(1, int(np.abs(matrix).max()))
        image = axis.imshow(matrix, cmap="RdBu_r", vmin=-limit, vmax=limit)
    else:
        image = axis.imshow(matrix, cmap="YlGnBu", vmin=0)
    figure.colorbar(image, ax=axis, fraction=0.046, pad=0.04)
    axis.set_xticks(range(10), CLASSES, rotation=45, ha="right")
    axis.set_yticks(range(10), CLASSES)
    axis.set_xlabel("Predicted class")
    axis.set_ylabel("True class")
    axis.set_title(title, fontsize=14, fontweight="bold", loc="left")
    threshold = np.max(np.abs(matrix)) * 0.55
    for row in range(10):
        for column in range(10):
            value = int(matrix[row, column])
            if value != 0 or not signed:
                axis.text(column, row, str(value), ha="center", va="center", fontsize=8,
                          color="white" if abs(value) > threshold else "black")
    return figure


def parse_vcd(path: Path) -> tuple[str, dict[str, VcdSignal]]:
    scopes: list[str] = []
    code_definitions: dict[str, list[tuple[str, int]]] = defaultdict(list)
    timescale = "1ps"
    in_definitions = True
    selected_codes: set[str] = set()
    transitions: dict[str, list[tuple[int, float]]] = defaultdict(list)
    current_time = 0

    with path.open("r", encoding="utf-8", errors="replace") as handle:
        for raw in handle:
            line = raw.strip()
            if in_definitions:
                if line.startswith("$timescale"):
                    pieces = line.split()
                    if len(pieces) >= 2 and pieces[1] != "$end":
                        timescale = pieces[1]
                elif line.startswith("$scope"):
                    pieces = line.split()
```

```
                    scopes.append(pieces[2])
                elif line.startswith("$upscope"):
                    if scopes:
                        scopes.pop()
                elif line.startswith("$var"):
                    pieces = line.split()
                    if len(pieces) >= 6:
                        width = int(pieces[2])
                        code = pieces[3]
                        reference = pieces[4]
                        name = "/" + "/".join([*scopes, reference])
                        code_definitions[code].append((name, width))
                elif line.startswith("$enddefinitions"):
                    in_definitions = False
                    selected_codes = set(code_definitions)
                continue
            if not line or line.startswith("$"):
                continue
            if line.startswith("#"):
                current_time = int(line[1:])
                continue
            if line[0] in "01xXzZ":
                code = line[1:]
                if code in selected_codes:
                    value = float(line[0]) if line[0] in "01" else math.nan
                    transitions[code].append((current_time, value))
            elif line[0] in "bBrR":
                pieces = line.split(maxsplit=1)
                if len(pieces) != 2:
                    continue
                encoded, code = pieces
                if code not in selected_codes:
                    continue
                digits = encoded[1:]
                if any(character.lower() in "xz" for character in digits):
                    value = math.nan
                elif encoded[0].lower() == "b":
                    value = float(int(digits, 2))
                else:
                    try:
                        value = float(digits)
                    except ValueError:
                        value = math.nan
                transitions[code].append((current_time, value))

    signals: dict[str, VcdSignal] = {}
    for code, definitions in code_definitions.items():
        values = transitions.get(code, [])
        for name, width in definitions:
            signals[name] = VcdSignal(name, width, values)
    return timescale, signals


def choose_wave_signals(signals: dict[str, VcdSignal], limit: int = 10) -> list[VcdSignal]:
    priority = (
        "program_active", "program_done", "program_error", "state", "command_valid",
        "input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid",
        "core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready",
        "output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds",
        "update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product",
        "activation", "weight",
    )
    candidates: list[tuple[int, int, str, VcdSignal]] = []
    for signal in signals.values():
        lowered = signal.name.lower()
        leaf = lowered.rsplit("/", 1)[-1]
        if lowered.endswith("/clk") or not signal.transitions:
            continue
        if leaf in {"clear_state", "lif_clear_state", "clear_counters"}:
            continue
        matches = [
            index
            for index, token in enumerate(priority)
            if leaf == token or leaf == token + "_r" or (token != "state" and token in leaf)
        ]
        if not matches:
            continue
        # Prefer DUT registers and shallow hierarchy so plots remain readable.
        hierarchy_penalty = signal.name.count("/")
        dut_penalty = 0 if "/dut/" in lowered else 2
        candidates.append((min(matches), hierarchy_penalty + dut_penalty, signal.name, signal))
    chosen: list[VcdSignal] = []
    used_leaf_names: set[str] = set()
    for _, _, _, signal in sorted(candidates):
        leaf = signal.name.rsplit("/", 1)[-1]
        if leaf in used_leaf_names:
            continue
```

```
        used_leaf_names.add(leaf)
        chosen.append(signal)
        if len(chosen) == limit:
            break
    return chosen


def waveform_figure(vcd_path: Path, title: str) -> tuple[plt.Figure, list[str]]:
    timescale, signals = parse_vcd(vcd_path)
    selected = choose_wave_signals(signals)
    if not selected:
        raise RuntimeError(f"no debug signals selected from {vcd_path}")
    figure, axes = plt.subplots(len(selected), 1, figsize=(12.5, 1.25 * len(selected) + 1.2), sharex=True)
    axes_array = np.atleast_1d(axes)
    unit_scale = {"1ps": 1e-3, "1ns": 1.0}.get(timescale, 1.0)
    for axis, signal, color in zip(axes_array, selected, COLORS * 3):
        data = signal.transitions
        if len(data) > 2500:
            stride = math.ceil(len(data) / 2500)
            data = data[::stride]
        x = np.asarray([item[0] * unit_scale for item in data], dtype=np.float64)
        y = np.asarray([item[1] for item in data], dtype=np.float64)
        if signal.width == 1:
            axis.step(x, y, where="post", color=color, linewidth=1.2)
            axis.set_ylim(-0.2, 1.2)
            axis.set_yticks([0, 1])
        else:
            axis.step(x, y, where="post", color=color, linewidth=1.05)
        axis.set_ylabel(signal.name.rsplit("/", 1)[-1], rotation=0, ha="right", va="center", fontsize=8)
        axis.grid(True, axis="x", alpha=0.18)
        axis.spines[["top", "right"]].set_visible(False)
    axes_array[0].set_title(title, fontsize=13, fontweight="bold", loc="left")
    axes_array[-1].set_xlabel(f"Simulation time ({'ns' if unit_scale == 1 else 'ns; VCD timescale ' + timescale})")
    return figure, [signal.name for signal in selected]


def arithmetic_groups(operators: Sequence[dict]) -> dict[str, int]:
    totals: dict[str, int] = defaultdict(int)
    for operator in operators:
        matrix = operator.get("matrix")
        if matrix:
            totals[matrix["arithmetic"]] += int(matrix["mac_equivalents"])
    return dict(totals)


def parse_timing_slacks(path: Path) -> np.ndarray:
    text = path.read_text(encoding="utf-8", errors="ignore")
    values = [float(value) for value in re.findall(r"Slack \((?:MET|VIOLATED)\)\s*:\s*(-?\d+\.\d+)ns", text)]
    return np.asarray(values, dtype=np.float64)


def parse_implementation_summary(path: Path) -> dict[str, float]:
    result: dict[str, float] = {}
    if not path.exists():
        return result
    for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():
        if "=" not in raw:
            continue
        key, value = [part.strip() for part in raw.split("=", 1)]
        try:
            result[key] = float(value)
        except ValueError:
            continue
    return result


def parse_clock_timing(path: Path) -> dict[str, tuple[float, float]]:
    """Return routed setup/hold slack for each FPGA clock-domain row."""
    clocks: dict[str, tuple[float, float]] = {}
    if not path.exists():
        return clocks
    for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():
        fields = raw.split()
        if len(fields) < 10 or not re.fullmatch(r"clk_fpga_\d+", fields[0]):
            continue
        try:
            setup_slack = float(fields[1])
            hold_slack = float(fields[5])
        except ValueError:
            continue
        clocks.setdefault(fields[0], (setup_slack, hold_slack))
    return clocks


def parse_utilization_hierarchy(path: Path) -> dict[str, dict[str, int]]:
    """Parse named rows from Vivado's hierarchical utilization table."""
    wanted = {
```

```
        "system_wrapper", "accelerator", "accelerator_control_cdc",
        "control_interconnect", "dma", "dma_control_cdc",
        "memory_interconnect", "mm2s_cdc", "s2mm_cdc",
    }
    rows: dict[str, dict[str, int]] = {}
    if not path.exists():
        return rows
    for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():
        if not raw.startswith("|"):
            continue
        fields = [field.strip() for field in raw.split("|")]
        if len(fields) < 12 or fields[1] not in wanted:
            continue
        try:
            rows[fields[1]] = {
                "lut": int(fields[3]), "ff": int(fields[7]),
                "bram36": int(fields[8]), "bram18": int(fields[9]),
                "dsp": int(fields[10]),
            }
        except ValueError:
            continue
    return rows


def parse_drc_summary(path: Path) -> list[tuple[str, str, str, int]]:
    """Parse DRC rule, severity, description, and count from the summary table."""
    if not path.exists():
        return []
    pattern = re.compile(
        r"^\|\s*([A-Z]+-\d+)\s*\|\s*(\w+)\s*\|\s*(.*?)\s*\|\s*(\d+)\s*\|$"
    )
    rows: list[tuple[str, str, str, int]] = []
    for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():
        match = pattern.match(raw)
        if match:
            rows.append((match.group(1), match.group(2), match.group(3), int(match.group(4))))
    return rows


def parse_cdc_summary(path: Path) -> list[tuple[str, str, int, str]]:
    """Parse the compact CDC classification table before detailed paths."""
    if not path.exists():
        return []
    pattern = re.compile(r"^(CDC-\d+)\s+(\w+)\s+(\d+)\s+(.+)$")
    rows: list[tuple[str, str, int, str]] = []
    for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():
        match = pattern.match(raw.strip())
        if match:
            rows.append((match.group(1), match.group(2), int(match.group(3)), match.group(4)))
        elif rows:
            break
    return rows


def parse_power_report(path: Path) -> dict[str, object]:
    """Extract explicit Vivado power totals, resources, hierarchy, and confidence."""
    if not path.exists():
        return {}
    text = path.read_text(encoding="utf-8", errors="ignore")

    def number(label: str) -> float | None:
        match = re.search(rf"^\|\s*{re.escape(label)}\s*\|\s*([0-9.]+)", text, re.MULTILINE)
        return float(match.group(1)) if match else None

    confidence_match = re.search(
        r"^\|\s*Confidence Level\s*\|\s*(\w+)", text, re.MULTILINE
    )
    matched_match = re.search(
        r"^\|\s*Design Nets Matched\s*\|\s*([^|]+)", text, re.MULTILINE
    )
    resources: dict[str, float] = {}
    for label in ("Clocks", "Slice Logic", "Signals", "Block RAM", "DSPs", "PS7", "Static Power", "Total"):
        match = re.search(
            rf"^\|\s*{re.escape(label)}\s*\|\s*<?([0-9.]+)", text, re.MULTILINE
        )
        if match:
            resources[label] = float(match.group(1))
    hierarchy: dict[str, float] = {}
    hierarchy_names = (
        "system_wrapper", "system_i", "accelerator", "accelerator_control_cdc",
        "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect",
        "mm2s_cdc", "ps7", "reset_100", "reset_125", "reset_200", "s2mm_cdc",
    )
    for name in hierarchy_names:
        match = re.search(
            rf"^\|\s*{re.escape(name)}\s*\|\s*([0-9.]+)\s*\|", text, re.MULTILINE
        )
```

```
        if match:
            hierarchy[name] = float(match.group(1))
    return {
        "total": number("Total On-Chip Power (W)"),
        "dynamic": number("Dynamic (W)"),
        "static": number("Device Static (W)"),
        "confidence": confidence_match.group(1) if confidence_match else "unknown",
        "matched": matched_match.group(1).strip() if matched_match else "not reported",
        "resources": resources,
        "hierarchy": hierarchy,
    }


def file_evidence(path: Path) -> tuple[int, str]:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for chunk in iter(lambda: handle.read(1024 * 1024), b""):
            digest.update(chunk)
    return path.stat().st_size, digest.hexdigest()


def latex_escape(value: str) -> str:
    replacements = {
        "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$",
        "#": r"\#", "_": r"\_", "{": r"\{", "}": r"\}",
        "~": r"\textasciitilde{}", "^": r"\textasciicircum{}",
    }
    return "".join(replacements.get(character, character) for character in value)


def latex_breakable(value: str) -> str:
    """Escape provenance text while adding legal line-break opportunities."""
    escaped = latex_escape(value)
    escaped = escaped.replace(r"\textbackslash{}", r"\textbackslash{}\allowbreak{}")
    escaped = escaped.replace(r"\_", r"\_\allowbreak{}")
    escaped = escaped.replace("/", r"/\allowbreak{}")
    escaped = escaped.replace(";", r";\allowbreak{}")
    escaped = escaped.replace(", ", r",\allowbreak{} ")
    return escaped


def write_report_tex(
    output_dir: Path,
    records: Sequence[PlotRecord],
    block_design_filename: str | None,
) -> Path:
    tex_path = output_dir / "neuraedge_deep_debug_report.tex"
    lines = [
        r"\documentclass[10pt]{article}",
        r"\usepackage[margin=0.65in]{geometry}",
        r"\usepackage[T1]{fontenc}",
        r"\usepackage{lmodern}",
        r"\usepackage{graphicx}",
        r"\usepackage{pdfpages}",
        r"\usepackage[hidelinks]{hyperref}",
        r"\setlength{\parindent}{0pt}",
        r"\setlength{\parskip}{5pt}",
        r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}",
        r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}",
        r"\date{\today}",
        r"\begin{document}",
        r"\maketitle",
        r"\tableofcontents",
        r"\clearpage",
        r"\section{Evidence rule}",
        "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",
    ]
    if block_design_filename is not None:
        lines.extend(
            [
                r"\clearpage",
                r"\section{Native Vivado PYNQ block design}",
                "This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.",
                rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filename}}}",
            ]
        )
    for index, record in enumerate(records, start=1):
        lines.extend(
            [
                r"\clearpage",
                rf"\section{{{latex_escape(record.title)}}}",
                rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}",
                r"\begin{center}",
                rf"\includegraphics[width=0.98\textwidth,height=0.78\textheight,keepaspectratio]{{plots/{record.filename}}}",
```

```
                r"\end{center}",
            ]
        )
    lines.append(r"\end{document}")
    tex_path.write_text("\n".join(lines) + "\n", encoding="utf-8")
    return tex_path


def generate(
    output_dir: Path,
    timing_dir: Path | None,
    waveform_dir: Path | None = None,
    saif_power_report: Path | None = None,
    block_design: Path | None = None,
) -> tuple[Path, Path, Path]:
    root = root_dir()
    timing_dir = timing_dir or root / "outputs/pynq_full"
    waveform_dir = waveform_dir or root / "outputs/waveforms_final"
    saif_power_report = saif_power_report or (
        root / "outputs/power_activity_full/vivado_reports/routed_saif_power.rpt"
    )
    block_design = block_design or root / "outputs/pynq_full/neuraedge_pynq_block_design.pdf"
    output_dir.mkdir(parents=True, exist_ok=True)
    plot_dir = output_dir / "plots"
    plot_dir.mkdir(parents=True, exist_ok=True)
    pdf_path = output_dir / "neuraedge_deep_debug_plots.pdf"
    records: list[PlotRecord] = []
    index = 1

    reference = load_json(root / "outputs/spikingformer_reference/spikingformer_evaluation.json")
    quantized = load_json(root / "outputs/spikingformer_hw_q10_tensor_exact/spikingformer_evaluation.json")
    execution = load_json(root / "outputs/spikingformer_reference/spikingformer_execution_manifest.json")
    parameters = load_json(root / "outputs/spikingformer_hardware_parameters/spikingformer_neuraedge_weights.json")
    compiler_benchmark = load_json(root / "outputs/benchmarks/compiler_block3_memory.json")
    training_coverage_path = root / "outputs/training/spikingformer_training_coverage.json"
    training_coverage = (
        load_json(training_coverage_path) if training_coverage_path.exists() else None
    )
    forward_validation_path = root / "outputs/regression/real_full_graph_forward_validation.json"
    forward_validation = (
        load_json(forward_validation_path) if forward_validation_path.exists() else None
    )
    shard_manifest_path = root / "outputs/cifar10_shards/cifar10_shards_manifest.json"
    shard_manifest = load_json(shard_manifest_path) if shard_manifest_path.exists() else None
    float_npz = np.load(root / "outputs/spikingformer_reference/spikingformer_predictions.npz")
    quant_npz = np.load(root / "outputs/spikingformer_hw_q10_tensor_exact/spikingformer_predictions.npz")
    summary_path = timing_dir / "full_implementation_summary.txt"
    if not summary_path.exists():
        summary_path = timing_dir / "implementation_summary.txt"
    implementation = parse_implementation_summary(summary_path)
    vectorless_power_path = timing_dir / "full_vectorless_power.rpt"
    if not vectorless_power_path.exists():
        vectorless_power_path = timing_dir / "routed_vectorless_power.rpt"
    vectorless_power = parse_power_report(vectorless_power_path)
    saif_power = parse_power_report(saif_power_report)
    block_design_filename: str | None = None
    if block_design.exists():
        copied_block_design = output_dir / "pynq-block-design.pdf"
        shutil.copy2(block_design, copied_block_design)
        block_design_filename = copied_block_design.name

    with PdfPages(pdf_path) as pdf:
        summary_lines = [
            f"Dataset: {reference['dataset']} ({reference['samples']:,} samples)",
            f"Checkpoint: {reference['correct']:,}/{reference['samples']:,} = {reference['top1_accuracy_percent']:.2f}%",
            f"Exported integer arithmetic: {quantized['correct']:,}/{quantized['samples']:,} = {quantized['top1_accuracy_percent']:.2f}%",
            f"Prediction disagreements: {int(np.count_nonzero(float_npz['predictions'] != quant_npz['predictions'])):,}",
            f"Model MAC equivalents: {execution['per_image_totals']['mac_equivalents_after_exact_temporal_hoist']:,}",
            f"Tile commands: {execution['per_image_totals']['tensor_commands']:,}",
            f"Hardware parameter archive: {parameters['binary_bytes']:,} bytes, SHA-256 {parameters['binary_sha256']}",
            "Power pages use Vivado report data only; no physical-board value is inferred.",
        ]
        if implementation:
            summary_lines.extend(
                [
                    f"Full PYNQ routed setup/hold slack: {implementation.get('setup_wns_ns', float('nan')):+.3f} / {implementation.get('
     hold_whs_ns', float('nan')):+.3f} ns",
                    f"Clock domains: accelerator {implementation.get('accelerator_clock_mhz', float('nan')):.0f} MHz, DMA {implementation.get
     ('dma_clock_mhz', float('nan')):.0f} MHz, control {implementation.get('control_clock_mhz', float('nan')):.0f} MHz",
                ]
            )
        if saif_power:
            summary_lines.append(
                f"Full-system SAIF estimate: {saif_power['total']:.3f} W total, {saif_power['dynamic']:.3f} W dynamic, confidence {saif_power
     ['confidence']}, nets matched {saif_power['matched']}"
            )
        figure = text_page("Measured Evidence Summary", summary_lines)
```

```
    index = save_figure(figure, plot_dir, pdf, records, index, "evidence_summary", "Measured evidence summary",
                        "evaluation JSON, execution manifest, parameter manifest")

    if forward_validation:
        validation_lines = [
            f"Result: {forward_validation['result']}",
            f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}",
            f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}",
            f"Complete graph commands: {forward_validation['command_count']:,}",
            f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}",
            f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['
maximum_absolute_logit_delta_from_archived_10000_image_evaluation']:.3e}",
            f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha256
']}",
            f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",
        ]
        figure = text_page("Complete Exported-Graph Trace Validation", validation_lines)
        index = save_figure(
            figure, plot_dir, pdf, records, index, "full_graph_validation",
            "Complete exported-graph trace validation", str(forward_validation_path),
        )

    artifact_lines: list[str] = []
    for artifact_name in (
        "neuraedge_pynq_full.bit", "neuraedge_pynq_full.hwh",
        "neuraedge_pynq_full.xsa", "neuraedge_pynq_full_routed.dcp",
    ):
        artifact_path = timing_dir / artifact_name
        if artifact_path.exists():
            size, digest = file_evidence(artifact_path)
            artifact_lines.append(f"{artifact_name}: {size:,} bytes")
            artifact_lines.append(f"  SHA-256 {digest}")
    if artifact_lines:
        figure = text_page("Final PYNQ Deliverable Integrity", artifact_lines)
        index = save_figure(
            figure, plot_dir, pdf, records, index, "artifact_integrity",
            "Final PYNQ artifact integrity", str(timing_dir),
        )

    float_cm = np.asarray(reference["confusion_matrix_rows_true_columns_predicted"], dtype=np.int64)
    quant_cm = np.asarray(quantized["confusion_matrix_rows_true_columns_predicted"], dtype=np.int64)
    for slug, title, matrix, signed in (
        ("float_confusion", "Checkpoint confusion matrix: 10,000 images", float_cm, False),
        ("quant_confusion", "Exported integer confusion matrix: 10,000 images", quant_cm, False),
        ("confusion_delta", "Integer minus checkpoint confusion counts", quant_cm - float_cm, True),
    ):
        index = save_figure(confusion_figure(matrix, title, signed), plot_dir, pdf, records, index, slug, title,
                            "full CIFAR-10 prediction NPZ/JSON")

    figure, axis = plt.subplots(figsize=(11, 5.8))
    x = np.arange(10)
    width = 0.39
    float_acc = np.diag(float_cm) / float_cm.sum(axis=1) * 100
    quant_acc = np.diag(quant_cm) / quant_cm.sum(axis=1) * 100
    axis.bar(x - width / 2, float_acc, width, label="checkpoint", color=COLORS[0])
    axis.bar(x + width / 2, quant_acc, width, label="integer export", color=COLORS[1])
    axis.set_xticks(x, CLASSES, rotation=32, ha="right")
    axis.set_ylim(80, 100)
    axis.legend(frameon=False)
    style_axes(axis, "Per-class top-1 accuracy", ylabel="Accuracy (%)")
    index = save_figure(figure, plot_dir, pdf, records, index, "class_accuracy", "Per-class top-1 accuracy",
                        "full CIFAR-10 confusion matrices")

    labels = float_npz["labels"].astype(np.int64)
    float_logits = float_npz["logits"].astype(np.float64)
    quant_logits = quant_npz["logits"].astype(np.float64)
    float_sorted = np.sort(float_logits, axis=1)
    quant_sorted = np.sort(quant_logits, axis=1)
    float_margin = float_sorted[:, -1] - float_sorted[:, -2]
    quant_margin = quant_sorted[:, -1] - quant_sorted[:, -2]
    figure, axis = plt.subplots(figsize=(10.8, 5.8))
    axis.hist(float_margin, bins=100, density=True, alpha=0.58, label="checkpoint", color=COLORS[0])
    axis.hist(quant_margin, bins=100, density=True, alpha=0.52, label="integer export", color=COLORS[2])
    axis.set_xlim(0, np.percentile(np.concatenate([float_margin, quant_margin]), 99.5))
    axis.legend(frameon=False)
    style_axes(axis, "Top-1 versus runner-up confidence margin", "Logit margin", "Probability density")
    index = save_figure(figure, plot_dir, pdf, records, index, "margin_distribution", "Classifier margin distribution",
                        "all 10,000 checkpoint and integer logits")

    logit_error = quant_logits - float_logits
    figure, axes = plt.subplots(1, 2, figsize=(12, 5))
    axes[0].hist(logit_error.ravel(), bins=150, color=COLORS[4], alpha=0.8)
    style_axes(axes[0], "All logit errors", "Integer - checkpoint logit", "Count")
    axes[1].boxplot([logit_error[:, index] for index in range(10)], tick_labels=CLASSES, showfliers=False)
    axes[1].tick_params(axis="x", rotation=40)
    style_axes(axes[1], "Error by output class", ylabel="Logit error")
    index = save_figure(figure, plot_dir, pdf, records, index, "logit_error", "Integer-export logit error",
```

```
                        "paired 10,000 x 10 logit arrays")

    disagreements = float_npz["predictions"] != quant_npz["predictions"]
    transition = np.zeros((10, 10), dtype=np.int64)
    for before, after in zip(float_npz["predictions"][disagreements], quant_npz["predictions"][disagreements]):
        transition[int(before), int(after)] += 1
    index = save_figure(confusion_figure(transition, "Prediction changes: checkpoint class to integer class"),
                        plot_dir, pdf, records, index, "prediction_changes", "Prediction disagreement transitions",
                        f"{int(disagreements.sum())} changed predictions from paired NPZ files")

    activation_stats = reference["activation_statistics"]
    spike_items = [(name, values) for name, values in activation_stats.items() if values["is_spike"]]
    spike_items.sort(key=lambda item: item[1]["nonzero_fraction"], reverse=True)
    figure, axis = plt.subplots(figsize=(12, 8))
    names = [item[0] for item in spike_items]
    rates = [item[1]["nonzero_fraction"] * 100 for item in spike_items]
    axis.barh(np.arange(len(names)), rates, color=COLORS[0])
    axis.set_yticks(np.arange(len(names)), names, fontsize=7)
    axis.invert_yaxis()
    style_axes(axis, "Measured spike density by activation", "Nonzero spike fraction (%)")
    index = save_figure(figure, plot_dir, pdf, records, index, "spike_density", "Measured spike density",
                        "streaming activation statistics over all CIFAR-10 test images")

    figure, axis = plt.subplots(figsize=(11, 6.2))
    qkv_names = [name for name in activation_stats if re.search(r"block\.\d+\.attn\.[qkv]_lif$", name)]
    x = np.arange(4)
    for offset, suffix, color in ((-0.25, "q_lif", COLORS[0]), (0, "k_lif", COLORS[1]), (0.25, "v_lif", COLORS[2])):
        values = [activation_stats[f"block.{block}.attn.{suffix}"]["nonzero_fraction"] * 100 for block in range(4)]
        axis.bar(x + offset, values, 0.24, label=suffix[0].upper(), color=color)
    axis.set_xticks(x, [f"Transformer block {block}" for block in range(4)])
    axis.legend(frameon=False)
    style_axes(axis, "Q/K/V spike density across depth", ylabel="Spike density (%)")
    index = save_figure(figure, plot_dir, pdf, records, index, "qkv_density", "Q/K/V spike density",
                        "full-test streaming activation statistics")

    nonspike_items = [(name, values) for name, values in activation_stats.items() if not values["is_spike"]]
    figure, axis = plt.subplots(figsize=(12, 7))
    names = [item[0] for item in nonspike_items]
    means = np.asarray([item[1]["mean"] for item in nonspike_items])
    stds = np.asarray([item[1]["standard_deviation"] for item in nonspike_items])
    axis.errorbar(np.arange(len(names)), means, yerr=stds, fmt="o", markersize=3, capsize=2, color=COLORS[4])
    axis.axhline(0, color="#111827", linewidth=0.8)
    axis.set_xticks(np.arange(len(names)), names, rotation=75, ha="right", fontsize=6.5)
    style_axes(axis, "Non-spiking activation mean and one standard deviation", ylabel="Activation value")
    index = save_figure(figure, plot_dir, pdf, records, index, "activation_moments", "Activation mean and spread",
                        "full-test streaming activation moments")

    operators = execution["operators"]
    operator_names = [operator["name"] for operator in operators if operator.get("matrix")]
    macs = np.asarray([operator["matrix"]["mac_equivalents"] for operator in operators if operator.get("matrix")])
    commands = np.asarray([operator["matrix"]["commands"] for operator in operators if operator.get("matrix")])
    input_bytes = np.asarray([operator["matrix"]["input_stream_bytes"] for operator in operators if operator.get("matrix")])
    output_bytes = np.asarray([operator["matrix"]["output_stream_bytes"] for operator in operators if operator.get("matrix")])
    for values, slug, title, ylabel, log_scale in (
        (macs, "operator_macs", "Model work by learned operator", "MAC equivalents", True),
        (commands, "operator_commands", "Tile commands by learned operator", "Commands", True),
    ):
        figure, axis = plt.subplots(figsize=(12, 7))
        order = np.argsort(values)
        axis.barh(np.arange(len(order)), values[order], color=COLORS[0 if slug.endswith("macs") else 1])
        axis.set_yticks(np.arange(len(order)), [operator_names[item] for item in order], fontsize=7)
        if log_scale:
            axis.set_xscale("log")
        style_axes(axis, title, ylabel)
        index = save_figure(figure, plot_dir, pdf, records, index, slug, title,
                            "execution manifest generated from exported tensor shapes")

    figure, axis = plt.subplots(figsize=(12, 7))
    order = np.argsort(input_bytes + output_bytes)
    axis.barh(np.arange(len(order)), input_bytes[order] / 1e6, label="input", color=COLORS[0])
    axis.barh(np.arange(len(order)), output_bytes[order] / 1e6, left=input_bytes[order] / 1e6,
              label="output", color=COLORS[3])
    axis.set_yticks(np.arange(len(order)), [operator_names[item] for item in order], fontsize=7)
    axis.legend(frameon=False)
    style_axes(axis, "Raw tile-stream traffic by learned operator", "Traffic (MB per image)")
    index = save_figure(figure, plot_dir, pdf, records, index, "operator_traffic", "Operator stream traffic",
                        "execution manifest; excludes inter-layer DDR spill")

    arithmetic = arithmetic_groups(operators)
    figure, axis = plt.subplots(figsize=(8, 6))
    axis.pie(list(arithmetic.values()), labels=list(arithmetic), autopct="%1.1f%%", colors=COLORS[: len(arithmetic)],
             startangle=90, wedgeprops={"linewidth": 0.7, "edgecolor": "white"})
    axis.set_title("MAC-equivalent share by arithmetic mode", fontsize=13, fontweight="bold")
    index = save_figure(figure, plot_dir, pdf, records, index, "arithmetic_share", "Arithmetic mode share",
                        "execution manifest model-equivalent operation counts")

    matrix_records = [(operator["name"], operator["matrix"]) for operator in operators if operator.get("matrix")]
```

```
    figure, axis = plt.subplots(figsize=(9.5, 6.5))
    for arithmetic_name in sorted({item[1]["arithmetic"] for item in matrix_records}):
        subset = [item for item in matrix_records if item[1]["arithmetic"] == arithmetic_name]
        axis.scatter([item[1]["reduction"] for item in subset], [item[1]["columns"] for item in subset],
                     s=[max(25, math.sqrt(item[1]["rows"])) for item in subset], label=arithmetic_name, alpha=0.75)
    axis.set_xscale("log")
    axis.set_yscale("log")
    axis.legend(frameon=False)
    style_axes(axis, "Matrix shape landscape", "Reduction K", "Output columns N")
    index = save_figure(figure, plot_dir, pdf, records, index, "matrix_shapes", "Matrix shape landscape",
                        "execution manifest; marker area tracks output-row count")

    projection_names = [projection["name"] for projection in parameters["projections"]]
    weight_scales = [projection["weight_scale"] for projection in parameters["projections"]]
    factor_errors = [projection["represented_factor_maximum_error"] for projection in parameters["projections"]]
    figure, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
    axes[0].plot(weight_scales, marker="o", markersize=3, color=COLORS[0])
    axes[0].set_yscale("log")
    style_axes(axes[0], "Per-projection INT8 weight scale", ylabel="Scale")
    axes[1].plot(factor_errors, marker="o", markersize=3, color=COLORS[2])
    axes[1].set_yscale("log")
    axes[1].set_xticks(range(len(projection_names)), projection_names, rotation=75, ha="right", fontsize=6.5)
    style_axes(axes[1], "Maximum represented requantization-factor error", ylabel="Absolute factor error")
    index = save_figure(figure, plot_dir, pdf, records, index, "projection_quantization", "Projection quantization diagnostics",
                        "hashed hardware parameter manifest")

    binary_path = root / "outputs/spikingformer_hardware_parameters" / parameters["binary_file"]
    binary = binary_path.read_bytes()
    all_weights: list[np.ndarray] = []
    for projection in parameters["projections"]:
        section = projection["sections"][0]
        if "weight" in section["name"] and section["dtype"] == "|i1":
            all_weights.append(np.frombuffer(binary, dtype=np.int8, count=section["length"], offset=section["offset"]))
    head_section = parameters["head"]["sections"][0]
    all_weights.append(np.frombuffer(binary, dtype=np.int8, count=head_section["length"], offset=head_section["offset"]))
    weight_values = np.concatenate(all_weights)
    figure, axes = plt.subplots(1, 2, figsize=(12, 5.2))
    axes[0].hist(weight_values, bins=np.arange(-128.5, 128.5), color=COLORS[0])
    axes[0].set_yscale("log")
    style_axes(axes[0], "All exported INT8 weights", "Integer weight", "Count (log)")
    unique, counts = np.unique(weight_values, return_counts=True)
    top = np.argsort(counts)[-20:]
    axes[1].bar(unique[top].astype(str), counts[top], color=COLORS[1])
    axes[1].tick_params(axis="x", rotation=60)
    style_axes(axes[1], "Twenty most frequent codes", "Integer weight", "Count")
    index = save_figure(figure, plot_dir, pdf, records, index, "weight_distribution", "Exported weight distribution",
                        "direct section views from the SHA-256-checked parameter binary")

    figure, axes = plt.subplots(1, 2, figsize=(11, 5))
    names = ["scalar packer", "vectorized packer"]
    seconds = [95.601, compiler_benchmark["compile_seconds"]]
    peaks = [71.468169, compiler_benchmark["tracemalloc_peak_bytes"] / 1e6]
    axes[0].bar(names, seconds, color=[COLORS[2], COLORS[1]])
    axes[0].set_yscale("log")
    style_axes(axes[0], "Block-3 compile time", ylabel="Seconds (log)")
    axes[1].bar(names, peaks, color=[COLORS[2], COLORS[1]])
    style_axes(axes[1], "Peak traced memory", ylabel="MB")
    index = save_figure(figure, plot_dir, pdf, records, index, "compiler_benchmark", "Largest-layer compiler benchmark",
                        "recorded scalar baseline and outputs/benchmarks/compiler_block3_memory.json")

    if training_coverage:
        training_summary = training_coverage["summary"]
        semantics = training_coverage["training_semantics"]
        training_lines = [
            f"Target: {semantics['target_graph']}",
            f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['
execution_operator_count']}; uncovered {training_summary['uncovered_operator_count']}",
            f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['
dynamic_attention_matmul_count']}",
            f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}",
            f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary
['learned_backward_multiplications'] + training_summary['dynamic_backward_multiplications']):,}",
            f"Fabric SGDW update commands: {training_summary['sgdw_update_commands']:,}",
            f"Quantizer derivative: {semantics['quantizer_gradient']}",
            f"LIF derivative: {semantics['lif_gradient']}",
            f"Temporal schedule: {semantics['temporal_method']}",
            f"Optimizer: {semantics['optimizer']}",
            f"Parameter residency: {semantics['parameter_residency']}",
        ]
        figure = text_page("Full-Backbone Training Schedule", training_lines)
        index = save_figure(
            figure, plot_dir, pdf, records, index, "training_schedule",
            "Full-backbone training schedule and numeric contract", str(training_coverage_path),
        )

        learned = training_coverage["learned_affine_coverage"]
        learned_names = [item["name"] for item in learned]
```

```
            weight_elements = np.asarray([item["weight_elements"] for item in learned], dtype=np.int64)
            bias_elements = np.asarray([item["bias_elements"] for item in learned], dtype=np.int64)
            order = np.argsort(weight_elements + bias_elements)
            figure, axis = plt.subplots(figsize=(12, 8.5))
            axis.barh(np.arange(len(order)), weight_elements[order], label="weight", color=COLORS[0])
            axis.barh(
                np.arange(len(order)), bias_elements[order], left=weight_elements[order],
                label="bias", color=COLORS[3],
            )
            axis.set_yticks(np.arange(len(order)), [learned_names[item] for item in order], fontsize=7)
            axis.set_xscale("log")
            axis.legend(frameon=False)
            style_axes(axis, "Trainable deployed parameters by affine group", "Parameter elements (log)")
            index = save_figure(
                figure, plot_dir, pdf, records, index, "training_parameters",
                "Trainable deployed parameter distribution", str(training_coverage_path),
            )

            backward_tiles = np.asarray(
                [item["backward_partial_tiles"] for item in learned], dtype=np.int64
            )
            backward_multiplies = np.asarray(
                [item["backward_multiplications"] for item in learned], dtype=np.int64
            )
            order = np.argsort(backward_multiplies)
            figure, axes = plt.subplots(1, 2, figsize=(14, 9), sharey=True)
            y = np.arange(len(order))
            axes[0].barh(y, backward_tiles[order], color=COLORS[1])
            axes[0].set_yticks(y, [learned_names[item] for item in order], fontsize=7)
            axes[0].set_xscale("log")
            style_axes(axes[0], "Backward partial tiles", "Tiles per sample (log)")
            axes[1].barh(y, backward_multiplies[order], color=COLORS[2])
            axes[1].set_xscale("log")
            style_axes(axes[1], "Signed backward multiplications", "Multiplications per sample (log)")
            index = save_figure(
                figure, plot_dir, pdf, records, index, "training_backward_work",
                "Exact learned-layer backward work", str(training_coverage_path),
            )

            backward_sites = {
                "affine dX/dW/dB": training_summary["learned_affine_group_count"],
                "dynamic QK/AV": training_summary["dynamic_attention_matmul_count"],
                "LIF surrogate": training_summary["lif_node_count"],
                "max-pool scatter": training_summary["maxpool_backward_count"],
                "residual fan-out": training_summary["residual_gradient_fanout_count"],
                "global pool": 1,
            }
            figure, axes = plt.subplots(1, 2, figsize=(12.5, 5.8))
            axes[0].bar(
                np.arange(len(backward_sites)), list(backward_sites.values()),
                color=COLORS[:len(backward_sites)],
            )
            axes[0].set_xticks(
                np.arange(len(backward_sites)), list(backward_sites), rotation=35, ha="right"
            )
            style_axes(axes[0], "Implemented backward sites", ylabel="Site count")
            work_labels = ["learned affine", "dynamic attention"]
            work_values = [
                training_summary["learned_backward_multiplications"],
                training_summary["dynamic_backward_multiplications"],
            ]
            axes[1].pie(
                work_values, labels=work_labels, autopct="%1.2f%%", colors=[COLORS[0], COLORS[3]],
                startangle=90, wedgeprops={"linewidth": 0.7, "edgecolor": "white"},
            )
            axes[1].set_title("Backward multiplication share", fontsize=13, fontweight="bold")
            index = save_figure(
                figure, plot_dir, pdf, records, index, "training_operator_coverage",
                "Backward operator coverage and work composition", str(training_coverage_path),
            )

        if shard_manifest:
            split_names: list[str] = []
            shard_indices: list[int] = []
            shard_sizes: list[float] = []
            for split_name, shards in shard_manifest["splits"].items():
                for shard_index, shard in enumerate(shards):
                    split_names.append(split_name)
                    shard_indices.append(shard_index)
                    shard_sizes.append(shard["bytes"] / 1e6)
            figure, axis = plt.subplots(figsize=(11.5, 5.8))
            for color_index, split_name in enumerate(sorted(set(split_names))):
                selected = [index for index, name in enumerate(split_names) if name == split_name]
                axis.plot(
                    [shard_indices[index] for index in selected],
                    [shard_sizes[index] for index in selected],
                    marker="o", markersize=3, label=split_name, color=COLORS[color_index],
```

```
            )
        axis.legend(frameon=False)
        style_axes(axis, "Official CIFAR-10 shard sizes", "Shard index", "Compressed NPZ size (MB)")
        index = save_figure(
            figure, plot_dir, pdf, records, index, "training_dataset_shards",
            "Deterministic CIFAR-10 board-training shards", str(shard_manifest_path),
        )

    waveform_manifest_path = waveform_dir / "waveform_manifest.json"
    if waveform_manifest_path.exists():
        waveform_manifest = load_json(waveform_manifest_path)
        for capture in waveform_manifest["captures"]:
            vcd_path = waveform_dir / capture["vcd"]
            figure, selected_names = waveform_figure(vcd_path, f"XSim: {capture['testbench']}")
            evidence = (
                f"Vivado 2026.1 XSim VCD, requested window {capture['duration']} {capture['unit']}; "
                f"signals: {', '.join(name.rsplit('/', 1)[-1] for name in selected_names)}"
            )
            index = save_figure(figure, plot_dir, pdf, records, index,
                                f"wave_{capture['testbench'].removeprefix('tb_neuraedge_')}",
                                f"Waveform: {capture['testbench']}", evidence)

    if timing_dir is not None and timing_dir.exists():
        timing_summary_path = timing_dir / "full_routed_timing_summary.rpt"
        if not timing_summary_path.exists():
            timing_summary_path = timing_dir / "routed_timing_summary.rpt"
        clock_timing = parse_clock_timing(timing_summary_path)
        if clock_timing:
            names = sorted(clock_timing)
            setup = [clock_timing[name][0] for name in names]
            hold = [clock_timing[name][1] for name in names]
            figure, axes = plt.subplots(1, 2, figsize=(11.5, 5.2))
            axes[0].bar(names, setup, color=COLORS[: len(names)])
            axes[0].axhline(0, color=COLORS[2], linewidth=1.0)
            style_axes(axes[0], "Setup slack by clock domain", ylabel="WNS (ns)")
            axes[1].bar(names, hold, color=COLORS[: len(names)])
            axes[1].axhline(0, color=COLORS[2], linewidth=1.0)
            style_axes(axes[1], "Hold slack by clock domain", ylabel="WHS (ns)")
            index = save_figure(
                figure, plot_dir, pdf, records, index, "clock_domain_timing",
                "Routed timing by clock domain", str(timing_summary_path),
            )

        utilization_path = timing_dir / "full_routed_utilization_hierarchical.rpt"
        if not utilization_path.exists():
            utilization_path = timing_dir / "routed_utilization_hierarchical.rpt"
        utilization = parse_utilization_hierarchy(utilization_path)
        if "system_wrapper" in utilization:
            top = utilization["system_wrapper"]
            labels_resource = ["LUT", "FF", "DSP48", "BRAM tile"]
            counts_resource = [top["lut"], top["ff"], top["dsp"], top["bram36"] + top["bram18"] / 2]
            capacities = [53_200, 106_400, 220, 140]
            percents = [100 * count / capacity for count, capacity in zip(counts_resource, capacities)]
            figure, axes = plt.subplots(1, 2, figsize=(11.5, 5.2))
            axes[0].bar(labels_resource, counts_resource, color=COLORS[:4])
            axes[0].set_yscale("log")
            style_axes(axes[0], "Implemented primitive use", ylabel="Count (log scale)")
            axes[1].bar(labels_resource, percents, color=COLORS[:4])
            axes[1].set_ylim(0, 100)
            for position, value in enumerate(percents):
                axes[1].text(position, value + 1.5, f"{value:.1f}%", ha="center", fontsize=9)
            style_axes(axes[1], "PYNQ-Z2 device utilization", ylabel="Available resources used (%)")
            index = save_figure(
                figure, plot_dir, pdf, records, index, "full_routed_resources",
                "Full PYNQ routed resource utilization", str(utilization_path),
            )

            hierarchy_order = [
                name for name in (
                    "accelerator", "dma", "control_interconnect", "memory_interconnect",
                    "accelerator_control_cdc", "dma_control_cdc", "mm2s_cdc", "s2mm_cdc",
                ) if name in utilization
            ]
            figure, axes = plt.subplots(2, 2, figsize=(12, 8.2))
            for axis, key, title in zip(
                axes.ravel(), ("lut", "ff", "dsp", "bram36"),
                ("LUTs", "Flip-flops", "DSP48 blocks", "RAMB36 blocks"),
            ):
                values = [utilization[name][key] for name in hierarchy_order]
                axis.barh(np.arange(len(hierarchy_order)), values, color=COLORS[0])
                axis.set_yticks(np.arange(len(hierarchy_order)), hierarchy_order, fontsize=8)
                axis.invert_yaxis()
                style_axes(axis, title, xlabel="Primitive count")
            index = save_figure(
                figure, plot_dir, pdf, records, index, "resource_hierarchy",
                "Resource use by full-system hierarchy", str(utilization_path),
            )
```

```
993
994             drc_path = timing_dir / "full_drc.rpt"
995             drc_rows = parse_drc_summary(drc_path)
996             if drc_rows:
997                 figure, axis = plt.subplots(figsize=(11, 6.2))
998                 labels_drc = [f"{rule}: {description}" for rule, _, description, _ in drc_rows]
999                 counts_drc = [count for _, _, _, count in drc_rows]
1000                axis.barh(np.arange(len(drc_rows)), counts_drc, color=COLORS[3])
1001                axis.set_yticks(np.arange(len(drc_rows)), labels_drc, fontsize=8)
1002                axis.invert_yaxis()
1003                axis.set_xscale("log")
1004                style_axes(axis, "Vivado DRC advisory classification", "Reported checks (log scale)")
1005                index = save_figure(
1006                    figure, plot_dir, pdf, records, index, "drc_classification",
1007                    "DRC warning classification", str(drc_path),
1008                )
1009
1010            cdc_path = timing_dir / "full_cdc.rpt"
1011            cdc_rows = parse_cdc_summary(cdc_path)
1012            if cdc_rows:
1013                figure, axis = plt.subplots(figsize=(10.5, 5.4))
1014                labels_cdc = [f"{rule} {severity}\n{description}" for rule, severity, _, description in cdc_rows]
1015                counts_cdc = [count for _, _, count, _ in cdc_rows]
1016                axis.bar(labels_cdc, counts_cdc, color=[COLORS[1], COLORS[3]][: len(cdc_rows)])
1017                style_axes(axis, "Vivado clock-domain crossing classification", ylabel="Reported crossings")
1018                index = save_figure(
1019                    figure, plot_dir, pdf, records, index, "cdc_classification",
1020                    "Clock-domain crossing classification", str(cdc_path),
1021                )
1022
1023            if vectorless_power and saif_power:
1024                labels_power = ["Total", "Dynamic", "Static"]
1025                vector_values = [vectorless_power[key] for key in ("total", "dynamic", "static")]
1026                saif_values = [saif_power[key] for key in ("total", "dynamic", "static")]
1027                x = np.arange(len(labels_power))
1028                figure, axes = plt.subplots(1, 2, figsize=(12, 5.5))
1029                axes[0].bar(x - 0.2, vector_values, 0.4, label="vectorless", color=COLORS[3])
1030                axes[0].bar(x + 0.2, saif_values, 0.4, label="SAIF annotated", color=COLORS[1])
1031                axes[0].set_xticks(x, labels_power)
1032                axes[0].legend(frameon=False)
1033                style_axes(axes[0], "Full-system Vivado power estimates", ylabel="Power (W)")
1034                hierarchy_order = [
1035                    name for name in ("accelerator", "dma", "mm2s_cdc", "s2mm_cdc")
1036                    if name in vectorless_power["hierarchy"] and name in saif_power["hierarchy"]
1037                ]
1038                hierarchy_x = np.arange(len(hierarchy_order))
1039                axes[1].bar(
1040                    hierarchy_x - 0.2,
1041                    [vectorless_power["hierarchy"][name] for name in hierarchy_order],
1042                    0.4, label="vectorless", color=COLORS[3],
1043                )
1044                axes[1].bar(
1045                    hierarchy_x + 0.2,
1046                    [saif_power["hierarchy"][name] for name in hierarchy_order],
1047                    0.4, label="SAIF annotated", color=COLORS[1],
1048                )
1049                axes[1].set_xticks(hierarchy_x, hierarchy_order, rotation=25, ha="right")
1050                axes[1].legend(frameon=False)
1051                style_axes(axes[1], "Dynamic power by hierarchy", ylabel="Dynamic power (W)")
1052                index = save_figure(
1053                    figure, plot_dir, pdf, records, index, "power_comparison",
1054                    "Vectorless and SAIF-annotated power estimates",
1055                    f"{vectorless_power_path}; {saif_power_report}; SAIF match {saif_power['matched']}",
1056                )
1057
1058                power_lines = [
1059                    f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confidence {
    vectorless_power['confidence']}",
1060                    f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['confidence']}",
1061                    f"SAIF implementation nets matched: {saif_power['matched']}",
1062                    "Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched storage.",
1063                    "These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",
1064                ]
1065                figure = text_page("Power Evidence and Confidence Boundary", power_lines)
1066                index = save_figure(
1067                    figure, plot_dir, pdf, records, index, "power_confidence",
1068                    "Power confidence and annotation coverage", str(saif_power_report),
1069                )
1070
1071            timing_path = timing_dir / "full_routed_timing_paths_250.rpt"
1072            if not timing_path.exists():
1073                timing_path = timing_dir / "routed_timing_paths_250.rpt"
1074            if timing_path.exists():
1075                slacks = parse_timing_slacks(timing_path)
1076                if slacks.size:
1077                    figure, axes = plt.subplots(1, 2, figsize=(11, 5))
1078                    axes[0].hist(slacks, bins=min(40, max(8, slacks.size // 5)), color=COLORS[0])
```

```
                    axes[0].axvline(0, color=COLORS[2], linewidth=1.2)
                    style_axes(axes[0], "Reported routed path slack", "Setup slack (ns)", "Path count")
                    axes[1].plot(np.sort(slacks), color=COLORS[4])
                    axes[1].axhline(0, color=COLORS[2], linewidth=1.2)
                    style_axes(axes[1], "Sorted critical-path slack", "Path rank", "Setup slack (ns)")
                    index = save_figure(figure, plot_dir, pdf, records, index, "routed_timing", "Routed timing path distribution",
                                        str(timing_path))

    tex_path = write_report_tex(output_dir, records, block_design_filename)
    manifest_path = output_dir / "neuraedge_debug_plot_manifest.json"
    manifest_path.write_text(
        json.dumps(
            {
                "format": "neuraedge-deep-debug-v1",
                "plot_count": len(records),
                "plots": [record.__dict__ for record in records],
                "multipage_pdf": pdf_path.name,
                "latex": tex_path.name,
                "native_vivado_block_design": block_design_filename,
            },
            indent=2,
        )
        + "\n",
        encoding="utf-8",
    )
    return pdf_path, tex_path, manifest_path


def compile_tex(tex_path: Path, pdflatex: str) -> Path:
    for _ in range(2):
        subprocess.run([pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name],
                       cwd=tex_path.parent, check=True)
    return tex_path.with_suffix(".pdf")


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output-dir", type=Path, default=root_dir() / "outputs/deep_debug")
    parser.add_argument("--timing-dir", type=Path)
    parser.add_argument("--waveform-dir", type=Path)
    parser.add_argument("--saif-power-report", type=Path)
    parser.add_argument("--block-design", type=Path)
    parser.add_argument("--compile", action="store_true")
    parser.add_argument("--pdflatex", default="pdflatex")
    args = parser.parse_args()
    pdf_path, tex_path, manifest_path = generate(
        args.output_dir.resolve(),
        args.timing_dir.resolve() if args.timing_dir else None,
        args.waveform_dir.resolve() if args.waveform_dir else None,
        args.saif_power_report.resolve() if args.saif_power_report else None,
        args.block_design.resolve() if args.block_design else None,
    )
    print(f"plots_pdf={pdf_path}")
    print(f"latex={tex_path}")
    print(f"manifest={manifest_path}")
    if args.compile:
        print(f"report_pdf={compile_tex(tex_path, args.pdflatex)}")


if __name__ == "__main__":
    main()
```

28.3 Line annotations

Line 1: `"""Generate model, compiler, waveform, timing, utilization, and power plots."""` This is executable documentation attached to the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import hashlib` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `import json` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `import math` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import re` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `import shutil` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `import subprocess` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `from collections import Counter, defaultdict` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `from pathlib import Path` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `from typing import Callable, Iterable, Sequence` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `import matplotlib.pyplot as plt` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `import numpy as np` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `from matplotlib.backends.backend_pdf import PdfPages` This imports the named Python API so the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `CLASSES = [` This statement continues through source line 25; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck",]‘ to ‘CLASSES ‘ in host memory. Here CLASSES means the value named ’classes’ in this block. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named ’automobile’ in this block; bird: the value named ’bird’ in this block. Context: the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 23: `"airplane", "automobile", "bird", "cat", "deer",` This is continuation line 2 of the 4-line logical statement that begins at source line 22. Its exact fragment is ‘"airplane", "automobile", "bird", "cat", "deer",‘. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named ’automobile’ in this block; bird: the value named ’bird’ in this block. Read together, the complete operation is: This assigns ‘["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck",]‘ to ‘CLASSES ‘ in host memory. Here CLASSES means the value named ’classes’ in this block. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named ’automobile’ in this block; bird: the value named ’bird’ in this block. Context: the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 24: `"dog", "frog", "horse", "ship", "truck",` This is continuation line 3 of the 4-line logical statement that begins at source line 22. Its exact fragment is ‘"dog", "frog", "horse", "ship", "truck",‘. The important operands are dog: the value named ’dog’ in this block; frog: the value named ’frog’ in this block; horse: the value named ’horse’ in this block. Read together, the complete operation is: This assigns ‘["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck",]‘ to ‘CLASSES ‘ in host memory. Here CLASSES means the value named ’classes’ in this block. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named ’automobile’ in this block; bird: the value named ’bird’ in this block. Context: the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 25: `]` This is continuation line 4 of the 4-line logical statement that begins at source line 22. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘["airplane", "automobile", "bird", "cat", "deer", "dog", "frog", "horse", "ship", "truck",]‘ to ‘CLASSES ‘ in host memory. Here CLASSES means the value named ’classes’ in this block. The important operands are airplane: one parallel arithmetic element within a vector or tensor datapath; automobile: the value named ’automobile’ in this block; bird: the value named ’bird’ in this block. Context: the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 26: `COLORS = ["#155e75", "#166534", "#991b1b", "#7c3e00", "#4f46e5", "#374151"]` This assigns ‘["#155e75", "#166534", "#991b1b", "#7c3e00", "#4f46e5", "#374151"]‘ to ‘COLORS ‘ in host memory. Here COLORS means the value named ’colors’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 29: `@dataclass` This decorator changes how the following class or function is constructed. ‘@dataclass‘ is applied at import time on the host and is part of the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it creates no FPGA circuit by itself.

Line 30: `class PlotRecord:` This defines class PlotRecord, which groups data and methods for plot record. Instances exist in host memory and organize the flow that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 31: `filename: str` Inside the class PlotRecord, which handles plot record, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are filename: the value named ’filename’ in this block; str: the value named ’str’ in this block.

Line 32: `title: str` Inside the class PlotRecord, which handles plot record, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are title: the value named 'title' in this block; str: the value named 'str' in this block.

Line 33: `evidence: str` Inside the class PlotRecord, which handles plot record, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are evidence: the value named 'evidence' in this block; str: the value named 'str' in this block.

Line 36: `@dataclass` This decorator changes how the following class or function is constructed. '@dataclass' is applied at import time on the host and is part of the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it creates no FPGA circuit by itself.

Line 37: `class VcdSignal:` This defines class VcdSignal, which groups data and methods for vcd signal. Instances exist in host memory and organize the flow that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 38: `name: str` Inside the class VcdSignal, which handles vcd signal, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are name: the value named 'name' in this block; str: the value named 'str' in this block.

Line 39: `width: int` Inside the class VcdSignal, which handles vcd signal, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are width: the value named 'width' in this block.

Line 40: `transitions: list[tuple[int, float]]` Inside the class VcdSignal, which handles vcd signal, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are transitions: the value named 'transitions' in this block; list: the value named 'list' in this block; tuple: the value named 'tuple' in this block.

Line 43: `def root_dir() -> Path:` This starts function root_dir. Its indented body performs root dir as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 44: `return Path(__file__).resolve().parents[1]` This returns 'Path(___file___).resolve().parents[1]' to the caller of the function root_dir, which handles root dir, ending this invocation. The important operands are Path: the value named 'path' in this block; ___file___: the value named 'file' in this block; resolve: the value named 'resolve' in this block.

Line 47: `def load_json(path: Path) -> dict:` This starts function load_json. Its indented body performs load json as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 48: `return json.loads(path.read_text(encoding="utf-8-sig"))` This returns 'json.loads(path.read_text(encoding="u
8-sig"))' to the caller of the function load_json, which handles load json, ending this invocation. The important operands are json: the value named 'json' in this block; loads: the value named 'loads' in this block; path: the value named 'path' in this block.

Line 51: `def style_axes(axis: plt.Axes, title: str, xlabel: str = "", ylabel: str = "") -> None:` This starts function style_axes. Its indented body performs style axes as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 52: `axis.set_title(title, fontsize=12, fontweight="bold", loc="left")` This calls ‘axis.set_title‘ with ‘title, fontsize=12, fontweight="bold", loc="left"‘ inside the function style_axes, which handles style axes. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 53: `axis.set_xlabel(xlabel)` This calls ‘axis.set_xlabel‘ with ‘xlabel‘ inside the function style_axes, which handles style axes. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 54: `axis.set_ylabel(ylabel)` This calls ‘axis.set_ylabel‘ with ‘ylabel‘ inside the function style_axes, which handles style axes. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 55: `axis.grid(True, axis="y", alpha=0.22, linewidth=0.7)` This calls ‘axis.grid‘ with ‘True, axis="y", alpha=0.22, linewidth=0.7‘ inside the function style_axes, which handles style axes. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 56: `axis.spines[["top", "right"]].set_visible(False)` Inside the function style_axes, which handles style axes, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axis: the value named ’axis’ in this block; spines: the value named ’spines’ in this block; top: the value named ’top’ in this block.

Line 59: `def save_figure(` This starts function save_figure. Its indented body performs save figure as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 60: `figure: plt.Figure,` Inside the function save_figure, which handles save figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are figure: the value named ’figure’ in this block; plt: the value named ’plt’ in this block; Figure: the value named ’figure’ in this block.

Line 61: `output_dir: Path,` Inside the function save_figure, which handles save figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are output_dir: the value named ’output dir’ in this block; Path: the value named ’path’ in this block.

Line 62: `pdf: PdfPages,` Inside the function save_figure, which handles save figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are pdf: the value named ’pdf’ in this block; PdfPages: the value named ’pdf pages’ in this block.

Line 63: `records: list[PlotRecord],` Inside the function save_figure, which handles save figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are records: the number of descriptor records accepted since program start; list: the value named ’list’ in this block; PlotRecord: the value named ’plot record’ in this block.

Line 64: `index: int,` Inside the function save_figure, which handles save figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry.

Line 65: `slug: str,` Inside the function save_figure, which handles save figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are slug: the value named 'slug' in this block; str: the value named 'str' in this block.

Line 66: `title: str,` Inside the function save_figure, which handles save figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are title: the value named 'title' in this block; str: the value named 'str' in this block.

Line 67: `evidence: str,` Inside the function save_figure, which handles save figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are evidence: the value named 'evidence' in this block; str: the value named 'str' in this block.

Line 68: `) -> int:` Inside the function save_figure, which handles save figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The expression is a literal or structural operator and has no hidden software state.

Line 69: `figure.tight_layout()` This calls 'figure.tight_layout' with " inside the function save_figure, which handles save figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 70: `filename = f"{index:02d}_{slug}.png"` This assigns 'f"{index:02d}_{slug}.png"' to 'filename ' in host memory. Here filename means the value named 'filename' in this block. The important operands are f: the value named 'f' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; _: the value named " in this block. Context: the function save_figure, which handles save figure.

Line 71: `figure.savefig(output_dir / filename, dpi=190, bbox_inches="tight")` This calls 'figure.savefig' with 'output_dir / filename, dpi=190, bbox_inches="tight"' inside the function save_figure, which handles save figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 72: `pdf.savefig(figure, bbox_inches="tight")` This calls 'pdf.savefig' with 'figure, bbox_inches="tight"' inside the function save_figure, which handles save figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 73: `plt.close(figure)` This calls 'plt.close' with 'figure' inside the function save_figure, which handles save figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 74: `records.append(PlotRecord(filename, title, evidence))` This calls 'records.append' with 'PlotRecord(filename, title, evidence)' inside the function save_figure, which handles save figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 75: `return index + 1` This returns 'index + 1' to the caller of the function save_figure, which handles save figure, ending this invocation. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry.

Line 78: `def text_page(title: str, lines: Sequence[str]) -> plt.Figure:` This starts function text_page. Its indented body performs text page as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 79: `figure = plt.figure(figsize=(11.69, 8.27))` This assigns ‘plt.figure(figsize=(11.69, 8.27))‘ to ‘figure ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; figure: the value named ’figure’ in this block; figsize: the value named ’figsize’ in this block. Context: the function text_page, which handles text page.

Line 80: `figure.patch.set_facecolor("white")` This calls ‘figure.patch.set_facecolor‘ with ‘"white"‘ inside the function text_page, which handles text page. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 81: `figure.text(0.055, 0.93, title, fontsize=20, fontweight="bold", color="#111827")` This calls ‘figure.text‘ with ‘0.055, 0.93, title, fontsize=20, fontweight="bold", color="#111827"‘ inside the function text_page, which handles text page. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 82: `y = 0.86` This assigns ‘0.86‘ to ‘y ‘ in host memory. Here y means the value named ’y’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function text_page, which handles text page.

Line 83: `for line in lines:` This iterates line over ‘lines‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function text_page, which handles text page.

Line 84: `wrapped = [line[start : start + 125] for start in range(0, len(line), 125)] or [""]` This assigns ‘[line[start : start + 125] for start in range(0, len(line), 125)] or [""]‘ to ‘wrapped ‘ in host memory. Here wrapped means the value named ’wrapped’ in this block. The important operands are line: the value named ’line’ in this block; start: the value named ’start’ in this block; for: the value named ’for’ in this block. Context: the function text_page, which handles text page.

Line 85: `for segment in wrapped:` This iterates segment over ‘wrapped‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function text_page, which handles text page.

Line 86: `figure.text(0.065, y, segment, fontsize=10.5, family="monospace", color="#1f2937")` This calls ‘figure.text‘ with ‘0.065, y, segment, fontsize=10.5, family="monospace", color="#1f2937"‘ inside the function text_page, which handles text page. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 87: `y -= 0.038` Inside the function text_page, which handles text page, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are y: the value named ’y’ in this block.

Line 88: `y -= 0.008` Inside the function text_page, which handles text page, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are y: the value named ’y’ in this block.

Line 89: `return figure` This returns ‘figure‘ to the caller of the function text_page, which handles text page, ending this invocation. The important operands are figure: the value named ’figure’ in this block.

Line 92: `def confusion_figure(matrix: np.ndarray, title: str, signed: bool = False) -> plt.Figure:` This starts function confusion_figure. Its indented body performs confusion figure as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 93: `figure, axis = plt.subplots(figsize=(9.5, 8.2))` This assigns ‘plt.subplots(figsize=(9.5, 8.2))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function confusion_figure, which handles confusion figure.

Line 94: `if signed:` This executes the following indented host-code branch only when ‘signed‘ is true. The expression is a literal or structural operator and has no hidden software state. Context: the function confusion_figure, which handles confusion figure.

Line 95: `limit = max(1, int(np.abs(matrix).max()))` This assigns ‘max(1, int(np.abs(matrix).max()))‘ to ‘limit ‘ in host memory. Here limit means the value named ’limit’ in this block. The important operands are max: the value named ’max’ in this block; abs: the value named ’abs’ in this block; matrix: the value named ’matrix’ in this block. Context: the function confusion_figure, which handles confusion figure.

Line 96: `image = axis.imshow(matrix, cmap="RdBu_r", vmin=-limit, vmax=limit)` This assigns ‘axis.imshow(matrix, cmap="RdBu_r", vmin=-limit, vmax=limit)‘ to ‘image ‘ in host memory. Here image means the value named ’image’ in this block. The important operands are axis: the value named ’axis’ in this block; imshow: the value named ’imshow’ in this block; matrix: the value named ’matrix’ in this block. Context: the function confusion_figure, which handles confusion figure.

Line 97: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function confusion_figure, which handles confusion figure.

Line 98: `image = axis.imshow(matrix, cmap="YlGnBu", vmin=0)` This assigns ‘axis.imshow(matrix, cmap="YlGnBu", vmin=0)‘ to ‘image ‘ in host memory. Here image means the value named ’image’ in this block. The important operands are axis: the value named ’axis’ in this block; imshow: the value named ’imshow’ in this block; matrix: the value named ’matrix’ in this block. Context: the function confusion_figure, which handles confusion figure.

Line 99: `figure.colorbar(image, ax=axis, fraction=0.046, pad=0.04)` This calls ‘figure.colorbar‘ with ‘image, ax=axis, fraction=0.046, pad=0.04‘ inside the function confusion_figure, which handles confusion figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 100: `axis.set_xticks(range(10), CLASSES, rotation=45, ha="right")` This calls ‘axis.set_xticks‘ with ‘range(10), CLASSES, rotation=45, ha="right"‘ inside the function confusion_figure, which handles confusion figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 101: `axis.set_yticks(range(10), CLASSES)` This calls ‘axis.set_yticks‘ with ‘range(10), CLASSES‘ inside the function confusion_figure, which handles confusion figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 102: `axis.set_xlabel("Predicted class")` This calls ‘axis.set_xlabel‘ with ‘"Predicted class"‘ inside the function confusion_figure, which handles confusion figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 103: `axis.set_ylabel("True class")` This calls ‘axis.set_ylabel‘ with ‘"True class"‘ inside the function confusion_figure, which handles confusion figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 104: `axis.set_title(title, fontsize=14, fontweight="bold", loc="left")` This calls ‘axis.set_title‘ with ‘title, fontsize=14, fontweight="bold", loc="left"‘ inside the function confusion_figure, which handles confusion figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 105: `threshold = np.max(np.abs(matrix)) * 0.55` This assigns ‘np.max(np.abs(matrix)) * 0.55‘ to ‘threshold ‘ in host memory. Here threshold means the membrane level at which a neuron emits a spike. The important operands are max: the value named ’max’ in this block; abs: the value named ’abs’ in this block; matrix: the value named ’matrix’ in this block. Context: the function confusion_figure, which handles confusion figure.

Line 106: `for row in range(10):` This iterates row over ‘range(10)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function confusion_figure, which handles confusion figure.

Line 107: `for column in range(10):` This iterates column over ‘range(10)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function confusion_figure, which handles confusion figure.

Line 108: `value = int(matrix[row, column])` This assigns ‘int(matrix[row, column])‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are matrix: the value named ’matrix’ in this block; row: one output-row position processed by the tensor tile; column: one output-feature position processed in parallel. Context: the function confusion_figure, which handles confusion figure.

Line 109: `if value != 0 or not signed:` This executes the following indented host-code branch only when ‘value != 0 or not signed‘ is true. The important operands are value: the value named ’value’ in this block; or: the value named ’or’ in this block; not: the value named ’not’ in this block. Context: the function confusion_figure, which handles confusion figure.

Line 110: `axis.text(column, row, str(value), ha="center", va="center", fontsize=8,` This statement continues through source line 111; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘axis.text‘ with ‘column, row, str(value), ha="center", va="center", fontsize=8, color="white" if abs(value) > threshold else "black"‘ inside the function confusion_figure, which handles confusion figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 111: `color="white" if abs(value) > threshold else "black")` This is continuation line 2 of the 2-line logical statement that begins at source line 110. Its exact fragment is ‘color="white" if abs(value) > threshold else "black")‘. The important operands are color: the value named ’color’ in this block; white: the value named ’white’ in this block; abs: the value named ’abs’ in this block. Read together, the complete operation is: This calls ‘axis.text‘ with ‘column, row, str(value), ha="center", va="center", fontsize=8, color="white" if abs(value) > threshold else "black"‘ inside the function confusion_figure, which handles confusion figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 112: `return figure` This returns ‘figure‘ to the caller of the function confusion_figure, which handles confusion figure, ending this invocation. The important operands are figure: the value named ’figure’ in this block.

Line 115: `def parse_vcd(path: Path) -> tuple[str, dict[str, VcdSignal]]:` This starts function parse_vcd. Its indented body performs parse vcd as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 116: `scopes: list[str] = []` This assigns ‘[]‘ to ‘scopes‘ in host memory. Here scopes means the value named ’scopes’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_vcd, which handles parse vcd.

Line 117: `code_definitions: dict[str, list[tuple[str, int]]] = defaultdict(list)` This assigns ‘defaultdict(list)‘ to ‘code_definitions‘ in host memory. Here code_definitions means the value named ’code definitions’ in this block. The important operands are defaultdict: the value named ’defaultdict’ in this block; list: the value named ’list’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 118: `timescale = "1ps"` This assigns '"1ps"' to 'timescale ' in host memory. Here timescale means the real-value spacing represented by one quantized integer step. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_vcd, which handles parse vcd.

Line 119: `in_definitions = True` This assigns 'True' to 'in_definitions ' in host memory. Here in_definitions means the value named 'in definitions' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_vcd, which handles parse vcd.

Line 120: `selected_codes: set[str] = set()` This assigns 'set()' to 'selected_codes' in host memory. Here selected_codes means the value named 'selected codes' in this block. The important operands are set: the value named 'set' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 121: `transitions: dict[str, list[tuple[int, float]]] = defaultdict(list)` This assigns 'defaultdict(list)' to 'transitions' in host memory. Here transitions means the value named 'transitions' in this block. The important operands are defaultdict: the value named 'defaultdict' in this block; list: the value named 'list' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 122: `current_time = 0` This assigns '0' to 'current_time ' in host memory. Here current_time means the value named 'current time' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_vcd, which handles parse vcd.

Line 124: `with path.open("r", encoding="utf-8", errors="replace") as handle:` This opens a managed resource scope for 'path.open("r", encoding="utf-8", errors="replace") as handle' so files, tracing, or temporary state are finalized even if the function parse_vcd, which handles parse vcd raises an error.

Line 125: `for raw in handle:` This iterates raw over 'handle'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_vcd, which handles parse vcd.

Line 126: `line = raw.strip()` This assigns 'raw.strip()' to 'line ' in host memory. Here line means the value named 'line' in this block. The important operands are raw: the value named 'raw' in this block; strip: the value named 'strip' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 127: `if in_definitions:` This executes the following indented host-code branch only when 'in_definitions' is true. The important operands are in_definitions: the value named 'in definitions' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 128: `if line.startswith("$timescale"):` This executes the following indented host-code branch only when 'line.startswith("$timescale")' is true. The important operands are line: the value named 'line' in this block; startswith: the value named 'startswith' in this block; timescale: the real-value spacing represented by one quantized integer step. Context: the function parse_vcd, which handles parse vcd.

Line 129: `pieces = line.split()` This assigns 'line.split()' to 'pieces ' in host memory. Here pieces means the value named 'pieces' in this block. The important operands are line: the value named 'line' in this block; split: the value named 'split' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 130: `if len(pieces) >= 2 and pieces[1] != "$end":` This executes the following indented host-code branch only when 'len(pieces) >= 2 and pieces[1] != "$end"' is true. The important operands are len: the value named 'len' in this block; pieces: the value named 'pieces' in this block; and: the value named 'and' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 131: `timescale = pieces[1]` This assigns 'pieces[1]' to 'timescale ' in host memory. Here timescale means the real-value spacing represented by one quantized integer step. The important operands are pieces: the value named 'pieces' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 132: `elif line.startswith("$scope"):` This executes the following indented host-code branch only when ‘line.startswith("$scope")‘ is true. The important operands are line: the value named ’line’ in this block; startswith: the value named ’startswith’ in this block; scope: the value named ’scope’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 133: `pieces = line.split()` This assigns ‘line.split()‘ to ‘pieces ‘ in host memory. Here pieces means the value named ’pieces’ in this block. The important operands are line: the value named ’line’ in this block; split: the value named ’split’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 134: `scopes.append(pieces[2])` This calls ‘scopes.append‘ with ‘pieces[2]‘ inside the function parse_vcd, which handles parse vcd. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 135: `elif line.startswith("$upscope"):` This executes the following indented host-code branch only when ‘line.startswith("$upscope")‘ is true. The important operands are line: the value named ’line’ in this block; startswith: the value named ’startswith’ in this block; upscope: the value named ’upscope’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 136: `if scopes:` This executes the following indented host-code branch only when ‘scopes‘ is true. The important operands are scopes: the value named ’scopes’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 137: `scopes.pop()` This calls ‘scopes.pop‘ with “ inside the function parse_vcd, which handles parse vcd. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 138: `elif line.startswith("$var"):` This executes the following indented host-code branch only when ‘line.startswith("$v is true. The important operands are line: the value named ’line’ in this block; startswith: the value named ’startswith’ in this block; var: the value named ’var’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 139: `pieces = line.split()` This assigns ‘line.split()‘ to ‘pieces ‘ in host memory. Here pieces means the value named ’pieces’ in this block. The important operands are line: the value named ’line’ in this block; split: the value named ’split’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 140: `if len(pieces) >= 6:` This executes the following indented host-code branch only when ‘len(pieces) >= 6‘ is true. The important operands are len: the value named ’len’ in this block; pieces: the value named ’pieces’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 141: `width = int(pieces[2])` This assigns ‘int(pieces[2])‘ to ‘width ‘ in host memory. Here width means the value named ’width’ in this block. The important operands are pieces: the value named ’pieces’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 142: `code = pieces[3]` This assigns ‘pieces[3]‘ to ‘code ‘ in host memory. Here code means the value named ’code’ in this block. The important operands are pieces: the value named ’pieces’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 143: `reference = pieces[4]` This assigns ‘pieces[4]‘ to ‘reference ‘ in host memory. Here reference means the value named ’reference’ in this block. The important operands are pieces: the value named ’pieces’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 144: `name = "/" + "/".join([*scopes, reference])` This assigns ‘"/" + "/".join([*scopes, reference])‘ to ‘name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are join: the value named ’join’ in this block; scopes: the value named ’scopes’ in this block; reference: the value named ’reference’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 145: `code_definitions[code].append((name, width))` Inside the function parse_vcd, which handles parse vcd, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are code_definitions: the value named 'code definitions' in this block; code: the value named 'code' in this block; append: the value named 'append' in this block.

Line 146: `elif line.startswith("$enddefinitions"):` This executes the following indented host-code branch only when 'line.startswith("$enddefinitions")' is true. The important operands are line: the value named 'line' in this block; startswith: the value named 'startswith' in this block; enddefinitions: the value named 'enddefinitions' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 147: `in_definitions = False` This assigns 'False' to 'in_definitions ' in host memory. Here in_definitions means the value named 'in definitions' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_vcd, which handles parse vcd.

Line 148: `selected_codes = set(code_definitions)` This assigns 'set(code_definitions)' to 'selected_codes ' in host memory. Here selected_codes means the value named 'selected codes' in this block. The important operands are set: the value named 'set' in this block; code_definitions: the value named 'code definitions' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 149: `continue` Inside the function parse_vcd, which handles parse vcd, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named 'continue' in this block.

Line 150: `if not line or line.startswith("$"):` This executes the following indented host-code branch only when 'not line or line.startswith("$")' is true. The important operands are not: the value named 'not' in this block; line: the value named 'line' in this block; or: the value named 'or' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 151: `continue` Inside the function parse_vcd, which handles parse vcd, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named 'continue' in this block.

Line 152: `if line.startswith("#"):` This executes the following indented host-code branch only when 'line.startswith("#")' is true. The important operands are line: the value named 'line' in this block; startswith: the value named 'startswith' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 153: `current_time = int(line[1:])` This assigns 'int(line[1:])' to 'current_time ' in host memory. Here current_time means the value named 'current time' in this block. The important operands are line: the value named 'line' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 154: `continue` Inside the function parse_vcd, which handles parse vcd, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named 'continue' in this block.

Line 155: `if line[0] in "01xXzZ":` This executes the following indented host-code branch only when 'line[0] in "01xXzZ"' is true. The important operands are line: the value named 'line' in this block; in: the value named 'in' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 156: `code = line[1:]` This assigns 'line[1:]' to 'code ' in host memory. Here code means the value named 'code' in this block. The important operands are line: the value named 'line' in this block. Context: the function parse_vcd, which handles parse vcd.

Line 157: `if code in selected_codes:` This executes the following indented host-code branch only when ‘code in selected_codes‘ is true. The important operands are code: the value named ’code’ in this block; in: the value named ’in’ in this block; selected_codes: the value named ’selected codes’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 158: `value = float(line[0]) if line[0] in "01" else math.nan` This assigns ‘float(line[0]) if line[0] in "01" else math.nan‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are float: the value named ’float’ in this block; line: the value named ’line’ in this block; in: the value named ’in’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 159: `transitions[code].append((current_time, value))` Inside the function parse_vcd, which handles parse vcd, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are transitions: the value named ’transitions’ in this block; code: the value named ’code’ in this block; append: the value named ’append’ in this block.

Line 160: `elif line[0] in "bBrR":` This executes the following indented host-code branch only when ‘line[0] in "bBrR"‘ is true. The important operands are line: the value named ’line’ in this block; in: the value named ’in’ in this block; bBrR: the value named ’b br r’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 161: `pieces = line.split(maxsplit=1)` This assigns ‘line.split(maxsplit=1)‘ to ‘pieces ‘ in host memory. Here pieces means the value named ’pieces’ in this block. The important operands are line: the value named ’line’ in this block; split: the value named ’split’ in this block; maxsplit: the value named ’maxsplit’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 162: `if len(pieces) != 2:` This executes the following indented host-code branch only when ‘len(pieces) != 2‘ is true. The important operands are len: the value named ’len’ in this block; pieces: the value named ’pieces’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 163: `continue` Inside the function parse_vcd, which handles parse vcd, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named ’continue’ in this block.

Line 164: `encoded, code = pieces` This assigns ‘pieces‘ to ‘encoded, code ‘ in host memory. Here encoded means the value named ’encoded’ in this block. The important operands are pieces: the value named ’pieces’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 165: `if code not in selected_codes:` This executes the following indented host-code branch only when ‘code not in selected_codes‘ is true. The important operands are code: the value named ’code’ in this block; not: the value named ’not’ in this block; in: the value named ’in’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 166: `continue` Inside the function parse_vcd, which handles parse vcd, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named ’continue’ in this block.

Line 167: `digits = encoded[1:]` This assigns ‘encoded[1:]‘ to ‘digits ‘ in host memory. Here digits means the value named ’digits’ in this block. The important operands are encoded: the value named ’encoded’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 168: `if any(character.lower() in "xz" for character in digits):` This executes the following indented host-code branch only when ‘any(character.lower() in "xz" for character in digits)‘ is true. The important operands are any: the value named ’any’ in this block; character: the value named ’character’ in this block; lower: the value named ’lower’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 169: `value = math.nan` This assigns ‘math.nan‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are math: the value named ’math’ in this block; nan: the value named ’nan’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 170: `elif encoded[0].lower() == "b":` This executes the following indented host-code branch only when ‘encoded[0].lower() == "b"‘ is true. The important operands are encoded: the value named ’encoded’ in this block; lower: the value named ’lower’ in this block; b: the value named ’b’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 171: `value = float(int(digits, 2))` This assigns ‘float(int(digits, 2))‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are float: the value named ’float’ in this block; digits: the value named ’digits’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 172: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function parse_vcd, which handles parse vcd.

Line 173: `try:` This is explicit host-side failure handling inside the function parse_vcd, which handles parse vcd; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 174: `value = float(digits)` This assigns ‘float(digits)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are float: the value named ’float’ in this block; digits: the value named ’digits’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 175: `except ValueError:` This is explicit host-side failure handling inside the function parse_vcd, which handles parse vcd; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 176: `value = math.nan` This assigns ‘math.nan‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are math: the value named ’math’ in this block; nan: the value named ’nan’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 177: `transitions[code].append((current_time, value))` Inside the function parse_vcd, which handles parse vcd, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are transitions: the value named ’transitions’ in this block; code: the value named ’code’ in this block; append: the value named ’append’ in this block.

Line 179: `signals: dict[str, VcdSignal] = {}` This assigns ‘{}‘ to ‘signals‘ in host memory. Here signals means the value named ’signals’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_vcd, which handles parse vcd.

Line 180: `for code, definitions in code_definitions.items():` This iterates code, definitions over ‘code_definitions.items
Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_vcd, which handles parse vcd.

Line 181: `values = transitions.get(code, [])` This assigns ‘transitions.get(code, [])‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are transitions: the value named ’transitions’ in this block; get: the value named ’get’ in this block; code: the value named ’code’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 182: `for name, width in definitions:` This iterates name, width over ‘definitions‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_vcd, which handles parse vcd.

Line 183: `signals[name] = VcdSignal(name, width, values)` This assigns ‘VcdSignal(name, width, values)‘ to ‘signals[name] ‘ in host memory. Here signals means the value named ’signals’ in this block. The important operands are VcdSignal: the value named ’vcd signal’ in this block; name: the value named ’name’ in this block; width: the value named ’width’ in this block. Context: the function parse_vcd, which handles parse vcd.

Line 184: `return timescale, signals` This returns ‘timescale, signals‘ to the caller of the function parse_vcd, which handles parse vcd, ending this invocation. The important operands are timescale: the real-value spacing represented by one quantized integer step; signals: the value named ’signals’ in this block.

Line 187: `def choose_wave_signals(signals: dict[str, VcdSignal], limit: int = 10) -> list[VcdSignal]:` This starts function choose_wave_signals. Its indented body performs choose wave signals as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 188: `priority = (` This statement continues through source line 195; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘("program_active", "program_done", "program_error", "state", "command_valid", "input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid", "core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready", "output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds", "update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product", "activation", "weight",)‘ to ‘priority ‘ in host memory. Here priority means the value named ’priority’ in this block. The important operands are program_active: the value named ’program active’ in this block; program_done: the value named ’program done’ in this block; program_error: the value named ’program error’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 189: `"program_active", "program_done", "program_error", "state", "command_valid",` This is continuation line 2 of the 8-line logical statement that begins at source line 188. Its exact fragment is ‘"program_active", "program_done", "program_error", "state", "command_valid",‘. The important operands are program_active: the value named ’program active’ in this block; program_done: the value named ’program done’ in this block; program_error: the value named ’program error’ in this block. Read together, the complete operation is: This assigns ‘("program_active", "program_done", "program_error", "state", "command_valid", "input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid", "core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready", "output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds", "update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product", "activation", "weight",)‘ to ‘priority ‘ in host memory. Here priority means the value named ’priority’ in this block. The important operands are program_active: the value named ’program active’ in this block; program_done: the value named ’program done’ in this block; program_error: the value named ’program error’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 190: `"input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid",` This is continuation line 3 of the 8-line logical statement that begins at source line 188. Its exact fragment is ‘"input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid",‘. The important operands are input_fifo_count: an integer count retained for control or measurable telemetry; s_axis_tvalid: the producer’s statement that its current payload is meaningful; s_axis_tready: the consumer’s statement that it can accept a payload this cycle. Read together, the complete operation is: This assigns ‘("program_active", "program_done", "program_error", "state", "command_valid", "input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid", "core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready", "output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds", "update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product", "activation", "weight",)‘ to ‘priority ‘ in host memory. Here priority means the value named ’priority’ in this block. The important operands are program_active: the value named ’program active’ in this block; program_done: the value named ’program done’ in this block; program_error: the value named ’program error’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 191: `"core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready",` This is continuation line 4 of the 8-line logical statement that begins at source line 188. Its exact fragment is ‘"core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready",‘. The important operands are core_s_ready: the consumer’s statement that it can accept a payload this cycle; issue_valid: the producer’s statement that its current payload is meaningful; issue_ready: the consumer’s statement that it can accept a payload this cycle. Read together, the complete operation is: This assigns ‘("program_active", "program_done", "program_error", "state", "command_valid",

"input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid", "core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready", "output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds", "update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product", "activation", "weight",)‘ to ‘priority ‘ in host memory. Here priority means the value named ’priority’ in this block. The important operands are program_active: the value named ’program active’ in this block; program_done: the value named ’program done’ in this block; program_error: the value named ’program error’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 192: `"output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds",` This is continuation line 5 of the 8-line logical statement that begins at source line 188. Its exact fragment is ‘"output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds",‘. The important operands are output_valid: the producer’s statement that its current payload is meaningful; output_ready: the consumer’s statement that it can accept a payload this cycle; spike: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns ‘("program_active", "program_done", "program_error", "state", "command_valid", "input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid", "core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready", "output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds", "update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product", "activation", "weight",)‘ to ‘priority ‘ in host memory. Here priority means the value named ’priority’ in this block. The important operands are program_active: the value named ’program active’ in this block; program_done: the value named ’program done’ in this block; program_error: the value named ’program error’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 193: `"update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product",` This is continuation line 6 of the 8-line logical statement that begins at source line 188. Its exact fragment is ‘"update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product",‘. The important operands are update_count: an integer count retained for control or measurable telemetry; result_fifo_count: an integer count retained for control or measurable telemetry; m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This assigns ‘("program_active", "program_done", "program_error", "state", "command_valid", "input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid", "core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready", "output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds", "update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product", "activation", "weight",)‘ to ‘priority ‘ in host memory. Here priority means the value named ’priority’ in this block. The important operands are program_active: the value named ’program active’ in this block; program_done: the value named ’program done’ in this block; program_error: the value named ’program error’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 194: `"activation", "weight",` This is continuation line 7 of the 8-line logical statement that begins at source line 188. Its exact fragment is ‘"activation", "weight",‘. The important operands are activation: an input feature value consumed by a neuron or multiply-accumulate operation; weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘("program_active", "program_done", "program_error", "state", "command_valid", "input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid", "core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready", "output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds", "update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product", "activation", "weight",)‘ to ‘priority ‘ in host memory. Here priority means the value named ’priority’ in this block. The important operands are program_active: the value named ’program active’ in this block; program_done: the value named ’program done’ in this block; program_error: the value named ’program error’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 195: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 188. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘("program_active", "program_done", "program_error", "state", "command_valid", "input_fifo_count", "s_axis_tvalid", "s_axis_tready", "core_s_valid", "core_s_ready", "issue_valid", "issue_ready", "result_valid", "result_ready", "output_valid", "output_ready", "spike", "membrane", "dense_macs", "spike_adds", "update_count", "result_fifo_count", "m_axis_tlast", "interrupt", "product", "activation", "weight",)‘ to ‘priority ‘ in host memory. Here priority means the value named ’priority’ in this block. The important operands are program_active: the value named ’program active’ in this block; program_done: the value named ’program done’ in this block; program_error: the value named ’program error’ in this block. Context: the function choose_wave_signals, which handles

choose wave signals.

Line 196: `candidates: list[tuple[int, int, str, VcdSignal]] = []` This assigns '[]' to 'candidates' in host memory. Here candidates means the value named 'candidates' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function choose_wave_signals, which handles choose wave signals.

Line 197: `for signal in signals.values():` This iterates signal over 'signals.values()'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function choose_wave_signals, which handles choose wave signals.

Line 198: `lowered = signal.name.lower()` This assigns 'signal.name.lower()' to 'lowered ' in host memory. Here lowered means the value named 'lowered' in this block. The important operands are signal: the value named 'signal' in this block; name: the value named 'name' in this block; lower: the value named 'lower' in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 199: `leaf = lowered.rsplit("/", 1)[-1]` This assigns 'lowered.rsplit("/", 1)[-1]' to 'leaf ' in host memory. Here leaf means the value named 'leaf' in this block. The important operands are lowered: the value named 'lowered' in this block; rsplit: the value named 'rsplit' in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 200: `if lowered.endswith("/clk") or not signal.transitions:` This executes the following indented host-code branch only when 'lowered.endswith("/clk") or not signal.transitions' is true. The important operands are lowered: the value named 'lowered' in this block; endswith: the value named 'endswith' in this block; clk: the clock that makes all registers advance together. Context: the function choose_wave_signals, which handles choose wave signals.

Line 201: `continue` Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named 'continue' in this block.

Line 202: `if leaf in {"clear_state", "lif_clear_state", "clear_counters"}:` This executes the following indented host-code branch only when 'leaf in {"clear_state", "lif_clear_state", "clear_counters"}' is true. The important operands are leaf: the value named 'leaf' in this block; in: the value named 'in' in this block; clear_state: the finite-state-machine phase retained in flip-flops. Context: the function choose_wave_signals, which handles choose wave signals.

Line 203: `continue` Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named 'continue' in this block.

Line 204: `matches = [` This statement continues through source line 208; the explanation uses the complete expression rather than only the first visual fragment. Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are matches: the value named 'matches' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block.

Line 205: `index` This is continuation line 2 of the 5-line logical statement that begins at source line 204. Its exact fragment is 'index'. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are matches: the value named 'matches' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block.

Line 206: `for index, token in enumerate(priority)` This is continuation line 3 of the 5-line logical statement that begins at source line 204. Its exact fragment is 'for index, token in enumerate(priority)'. The important operands are for: the value named 'for' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; token: the value named 'token' in this block. Read together, the complete operation is: Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are matches: the value named 'matches' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block.

Line 207: `if leaf == token or leaf == token + "_r" or (token != "state" and token in leaf)` This is continuation line 4 of the 5-line logical statement that begins at source line 204. Its exact fragment is 'if leaf == token or leaf == token + "_r" or (token != "state" and token in leaf)'. The important operands are leaf: the value named 'leaf' in this block; token: the value named 'token' in this block; or: the value named 'or' in this block. Read together, the complete operation is: Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are matches: the value named 'matches' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block.

Line 208: `]` This is continuation line 5 of the 5-line logical statement that begins at source line 204. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are matches: the value named 'matches' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block.

Line 209: `if not matches:` This executes the following indented host-code branch only when 'not matches' is true. The important operands are not: the value named 'not' in this block; matches: the value named 'matches' in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 210: `continue` Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named 'continue' in this block.

Line 211: `# Prefer DUT registers and shallow hierarchy so plots remain readable.` The comment records this intent: Prefer DUT registers and shallow hierarchy so plots remain readable. It applies inside the function choose_wave_signals, which handles choose wave signals.

Line 212: `hierarchy_penalty = signal.name.count("/")` This assigns 'signal.name.count("/")' to 'hierarchy_penalty ' in host memory. Here hierarchy_penalty means the value named 'hierarchy penalty' in this block. The important operands are signal: the value named 'signal' in this block; name: the value named 'name' in this block; count: the value named 'count' in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 213: `dut_penalty = 0 if "/dut/" in lowered else 2` This assigns '0 if "/dut/" in lowered else 2' to 'dut_penalty ' in host memory. Here dut_penalty means the value named 'dut penalty' in this block. The important operands are dut: the value named 'dut' in this block; in: the value named 'in' in this block; lowered: the value named 'lowered' in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 214: `candidates.append((min(matches), hierarchy_penalty + dut_penalty, signal.name, signal))` This calls 'candidates.append' with '(min(matches), hierarchy_penalty + dut_penalty, signal.name, signal)' inside the function choose_wave_signals, which handles choose wave signals. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 215: `chosen: list[VcdSignal] = []` This assigns '[]' to 'chosen' in host memory. Here chosen means the value named 'chosen' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function choose_wave_signals, which handles choose wave signals.

Line 216: `used_leaf_names: set[str] = set()` This assigns ‘set()‘ to ‘used_leaf_names‘ in host memory. Here used_leaf_names means the value named ’used leaf names’ in this block. The important operands are set: the value named ’set’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 217: `for _, _, _, signal in sorted(candidates):` This iterates _, _, _, signal over ‘sorted(candidates)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function choose_wave_signals, which handles choose wave signals.

Line 218: `leaf = signal.name.rsplit("/", 1)[-1]` This assigns ‘signal.name.rsplit("/", 1)[-1]‘ to ‘leaf ‘ in host memory. Here leaf means the value named ’leaf’ in this block. The important operands are signal: the value named ’signal’ in this block; name: the value named ’name’ in this block; rsplit: the value named ’rsplit’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 219: `if leaf in used_leaf_names:` This executes the following indented host-code branch only when ‘leaf in used_leaf_names‘ is true. The important operands are leaf: the value named ’leaf’ in this block; in: the value named ’in’ in this block; used_leaf_names: the value named ’used leaf names’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 220: `continue` Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named ’continue’ in this block.

Line 221: `used_leaf_names.add(leaf)` This calls ‘used_leaf_names.add‘ with ‘leaf‘ inside the function choose_wave_signals, which handles choose wave signals. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 222: `chosen.append(signal)` This calls ‘chosen.append‘ with ‘signal‘ inside the function choose_wave_signals, which handles choose wave signals. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 223: `if len(chosen) == limit:` This executes the following indented host-code branch only when ‘len(chosen) == limit‘ is true. The important operands are len: the value named ’len’ in this block; chosen: the value named ’chosen’ in this block; limit: the value named ’limit’ in this block. Context: the function choose_wave_signals, which handles choose wave signals.

Line 224: `break` Inside the function choose_wave_signals, which handles choose wave signals, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are break: the value named ’break’ in this block.

Line 225: `return chosen` This returns ‘chosen‘ to the caller of the function choose_wave_signals, which handles choose wave signals, ending this invocation. The important operands are chosen: the value named ’chosen’ in this block.

Line 228: `def waveform_figure(vcd_path: Path, title: str) -> tuple[plt.Figure, list[str]]:` This starts function waveform_figure. Its indented body performs waveform figure as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 229: `timescale, signals = parse_vcd(vcd_path)` This assigns ‘parse_vcd(vcd_path)‘ to ‘timescale, signals ‘ in host memory. Here timescale means the real-value spacing represented by one quantized integer step. The important operands are parse_vcd: the value named ’parse vcd’ in this block; vcd_path: the value named ’vcd path’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 230: `selected = choose_wave_signals(signals)` This assigns ‘choose_wave_signals(signals)‘ to ‘selected ‘ in host memory. Here selected means the value named ’selected’ in this block. The important operands are choose_wave_signals: the value named ’choose wave signals’ in this block; signals: the value named ’signals’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 231: `if not selected:` This executes the following indented host-code branch only when ‘not selected‘ is true. The important operands are not: the value named ’not’ in this block; selected: the value named ’selected’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 232: `raise RuntimeError(f"no debug signals selected from {vcd_path}")` This rejects an invalid state by raising ‘RuntimeError(f"no debug signals selected from {vcd_path}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 233: `figure, axes = plt.subplots(len(selected), 1, figsize=(12.5, 1.25 * len(selected) + 1.2), sharex=True)` This assigns ‘plt.subplots(len(selected), 1, figsize=(12.5, 1.25 * len(selected) + 1.2), sharex=True)‘ to ‘figure, axes ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; len: the value named ’len’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 234: `axes_array = np.atleast_1d(axes)` This assigns ‘np.atleast_1d(axes)‘ to ‘axes_array ‘ in host memory. Here axes_array means the value named ’axes array’ in this block. The important operands are atleast_1d: the value named ’atleast 1d’ in this block; axes: the value named ’axes’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 235: `unit_scale = {"1ps": 1e-3, "1ns": 1.0}.get(timescale, 1.0)` This assigns ‘{"1ps": 1e-3, "1ns": 1.0}.get(timescale, 1.0)‘ to ‘unit_scale ‘ in host memory. Here unit_scale means the real-value spacing represented by one quantized integer step. The important operands are get: the value named ’get’ in this block; timescale: the real-value spacing represented by one quantized integer step. Context: the function waveform_figure, which handles waveform figure.

Line 236: `for axis, signal, color in zip(axes_array, selected, COLORS * 3):` This iterates axis, signal, color over ‘zip(axes_array, selected, COLORS * 3)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function waveform_figure, which handles waveform figure.

Line 237: `data = signal.transitions` This assigns ‘signal.transitions‘ to ‘data ‘ in host memory. Here data means the value named ’data’ in this block. The important operands are signal: the value named ’signal’ in this block; transitions: the value named ’transitions’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 238: `if len(data) > 2500:` This executes the following indented host-code branch only when ‘len(data) > 2500‘ is true. The important operands are len: the value named ’len’ in this block; data: the value named ’data’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 239: `stride = math.ceil(len(data) / 2500)` This assigns ‘math.ceil(len(data) / 2500)‘ to ‘stride ‘ in host memory. Here stride means the value named ’stride’ in this block. The important operands are math: the value named ’math’ in this block; ceil: the value named ’ceil’ in this block; len: the value named ’len’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 240: `data = data[::stride]` This assigns ‘data[::stride]‘ to ‘data ‘ in host memory. Here data means the value named ’data’ in this block. The important operands are data: the value named ’data’ in this block; stride: the value named ’stride’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 241: `x = np.asarray([item[0] * unit_scale for item in data], dtype=np.float64)` This assigns ‘np.asarray([item * unit_scale for item in data], dtype=np.float64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; unit_scale: the real-value spacing represented by one quantized integer step. Context: the function waveform_figure, which handles waveform figure.

Line 242: `y = np.asarray([item[1] for item in data], dtype=np.float64)` This assigns ‘np.asarray([item[1] for item in data], dtype=np.float64)‘ to ‘y ‘ in host memory. Here y means the value named ’y’ in this block. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; for: the value named ’for’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 243: `if signal.width == 1:` This executes the following indented host-code branch only when ‘signal.width == 1‘ is true. The important operands are signal: the value named ’signal’ in this block; width: the value named ’width’ in this block. Context: the function waveform_figure, which handles waveform figure.

Line 244: `axis.step(x, y, where="post", color=color, linewidth=1.2)` This calls ‘axis.step‘ with ‘x, y, where="post", color=color, linewidth=1.2‘ inside the function waveform_figure, which handles waveform figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 245: `axis.set_ylim(-0.2, 1.2)` This calls ‘axis.set_ylim‘ with ‘-0.2, 1.2‘ inside the function waveform_figure, which handles waveform figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 246: `axis.set_yticks([0, 1])` This calls ‘axis.set_yticks‘ with ‘[0, 1]‘ inside the function waveform_figure, which handles waveform figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 247: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function waveform_figure, which handles waveform figure.

Line 248: `axis.step(x, y, where="post", color=color, linewidth=1.05)` This calls ‘axis.step‘ with ‘x, y, where="post", color=color, linewidth=1.05‘ inside the function waveform_figure, which handles waveform figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 249: `axis.set_ylabel(signal.name.rsplit("/", 1)[-1], rotation=0, ha="right", va="center", fontsize=8)` This calls ‘axis.set_ylabel‘ with ‘signal.name.rsplit("/", 1)[-1], rotation=0, ha="right", va="center", fontsize=8‘ inside the function waveform_figure, which handles waveform figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 250: `axis.grid(True, axis="x", alpha=0.18)` This calls ‘axis.grid‘ with ‘True, axis="x", alpha=0.18‘ inside the function waveform_figure, which handles waveform figure. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 251: `axis.spines[["top", "right"]].set_visible(False)` Inside the function waveform_figure, which handles waveform figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axis: the value named ’axis’ in this block; spines: the value named ’spines’ in this block; top: the value named ’top’ in this block.

Line 252: `axes_array[0].set_title(title, fontsize=13, fontweight="bold", loc="left")` Inside the function waveform_figure, which handles waveform figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes_array: the value named ’axes array’ in this block; set_title: the value named ’set title’ in this block; title: the value named ’title’ in this block.

Line 253: `axes_array[-1].set_xlabel(f"Simulation time ({'ns' if unit_scale == 1 else 'ns; VCD timescale ' + timescale...` Inside the function waveform_figure, which handles waveform figure, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes_array: the value named 'axes array' in this block; set_xlabel: the value named 'set xlabel' in this block; f: the value named 'f' in this block.

Line 254: `return figure, [signal.name for signal in selected]` This returns 'figure, [signal.name for signal in selected]' to the caller of the function waveform_figure, which handles waveform figure, ending this invocation. The important operands are figure: the value named 'figure' in this block; signal: the value named 'signal' in this block; name: the value named 'name' in this block.

Line 257: `def arithmetic_groups(operators: Sequence[dict]) -> dict[str, int]:` This starts function arithmetic_groups. Its indented body performs arithmetic groups as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 258: `totals: dict[str, int] = defaultdict(int)` This assigns 'defaultdict(int)' to 'totals' in host memory. Here totals means the value named 'totals' in this block. The important operands are defaultdict: the value named 'defaultdict' in this block. Context: the function arithmetic_groups, which handles arithmetic groups.

Line 259: `for operator in operators:` This iterates operator over 'operators'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function arithmetic_groups, which handles arithmetic groups.

Line 260: `matrix = operator.get("matrix")` This assigns 'operator.get("matrix")' to 'matrix ' in host memory. Here matrix means the value named 'matrix' in this block. The important operands are operator: the value named 'operator' in this block; get: the value named 'get' in this block; matrix: the value named 'matrix' in this block. Context: the function arithmetic_groups, which handles arithmetic groups.

Line 261: `if matrix:` This executes the following indented host-code branch only when 'matrix' is true. The important operands are matrix: the value named 'matrix' in this block. Context: the function arithmetic_groups, which handles arithmetic groups.

Line 262: `totals[matrix["arithmetic"]] += int(matrix["mac_equivalents"])` Inside the function arithmetic_groups, which handles arithmetic groups, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are totals: the value named 'totals' in this block; matrix: the value named 'matrix' in this block; arithmetic: the value named 'arithmetic' in this block.

Line 263: `return dict(totals)` This returns 'dict(totals)' to the caller of the function arithmetic_groups, which handles arithmetic groups, ending this invocation. The important operands are dict: the value named 'dict' in this block; totals: the value named 'totals' in this block.

Line 266: `def parse_timing_slacks(path: Path) -> np.ndarray:` This starts function parse_timing_slacks. Its indented body performs parse timing slacks as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 267: `text = path.read_text(encoding="utf-8", errors="ignore")` This assigns 'path.read_text(encoding="utf-8", errors="ignore")' to 'text ' in host memory. Here text means the value named 'text' in this block. The important operands are path: the value named 'path' in this block; read_text: the value named 'read text' in this block; encoding: the value named 'encoding' in this block. Context: the function parse_timing_slacks, which handles parse timing slacks.

Line 268: `values = [float(value) for value in re.findall(r"Slack \((?:MET|VIOLATED)\)\s*:\s*(-?\d+\.\d+)ns", text)]` This assigns ‘[float(value) for value in re.findall(r"Slack \((?:MET|VIOLATED)\)\s*:\s*(-?\d+\.\d+)ns", text)]‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are float: the value named ’float’ in this block; value: the value named ’value’ in this block; for: the value named ’for’ in this block. Context: the function parse_timing_slacks, which handles parse timing slacks.

Line 269: `return np.asarray(values, dtype=np.float64)` This returns ‘np.asarray(values, dtype=np.float64)‘ to the caller of the function parse_timing_slacks, which handles parse timing slacks, ending this invocation. The important operands are asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block; dtype: the value named ’dtype’ in this block.

Line 272: `def parse_implementation_summary(path: Path) -> dict[str, float]:` This starts function parse_implementa Its indented body performs parse implementation summary as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 273: `result: dict[str, float] = {}` This assigns ‘{}‘ to ‘result‘ in host memory. Here result means the value named ’result’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_implementation_summary, which handles parse implementation summary.

Line 274: `if not path.exists():` This executes the following indented host-code branch only when ‘not path.exists()‘ is true. The important operands are not: the value named ’not’ in this block; path: the value named ’path’ in this block; exists: the value named ’exists’ in this block. Context: the function parse_implementation_summary, which handles parse implementation summary.

Line 275: `return result` This returns ‘result‘ to the caller of the function parse_implementation_summary, which handles parse implementation summary, ending this invocation. The important operands are result: the value named ’result’ in this block.

Line 276: `for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():` This iterates raw over ‘path.read_text(encoding="utf-8", errors="ignore").splitlines()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_implementation_summary, which handles parse implementation summary.

Line 277: `if "=" not in raw:` This executes the following indented host-code branch only when ‘"=" not in raw‘ is true. The important operands are not: the value named ’not’ in this block; in: the value named ’in’ in this block; raw: the value named ’raw’ in this block. Context: the function parse_implementation_summary, which handles parse implementation summary.

Line 278: `continue` Inside the function parse_implementation_summary, which handles parse implementation summary, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named ’continue’ in this block.

Line 279: `key, value = [part.strip() for part in raw.split("=", 1)]` This assigns ‘[part.strip() for part in raw.split("=", 1)]‘ to ‘key, value ‘ in host memory. Here key means the value named ’key’ in this block. The important operands are part: the value named ’part’ in this block; strip: the value named ’strip’ in this block; for: the value named ’for’ in this block. Context: the function parse_implementation_summary, which handles parse implementation summary.

Line 280: `try:` This is explicit host-side failure handling inside the function parse_implementation_summary, which handles parse implementation summary; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 281: `result[key] = float(value)` This assigns ‘float(value)‘ to ‘result[key] ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are float: the value named ’float’ in this block; value: the value named ’value’ in this block. Context: the function parse_implementation_summary, which handles parse implementation summary.

Line 282: `except ValueError:` This is explicit host-side failure handling inside the function parse_implementation_summary, which handles parse implementation summary; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 283: `continue` Inside the function parse_implementation_summary, which handles parse implementation summary, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named ’continue’ in this block.

Line 284: `return result` This returns ‘result‘ to the caller of the function parse_implementation_summary, which handles parse implementation summary, ending this invocation. The important operands are result: the value named ’result’ in this block.

Line 287: `def parse_clock_timing(path: Path) -> dict[str, tuple[float, float]]:` This starts function parse_clock_timing. Its indented body performs parse clock timing as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 288: `"""Return routed setup/hold slack for each FPGA clock-domain row."""` This is executable documentation attached to the function parse_clock_timing, which handles parse clock timing. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 289: `clocks: dict[str, tuple[float, float]] = {}` This assigns ‘{}‘ to ‘clocks‘ in host memory. Here clocks means the value named ’clocks’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_clock_timing, which handles parse clock timing.

Line 290: `if not path.exists():` This executes the following indented host-code branch only when ‘not path.exists()‘ is true. The important operands are not: the value named ’not’ in this block; path: the value named ’path’ in this block; exists: the value named ’exists’ in this block. Context: the function parse_clock_timing, which handles parse clock timing.

Line 291: `return clocks` This returns ‘clocks‘ to the caller of the function parse_clock_timing, which handles parse clock timing, ending this invocation. The important operands are clocks: the value named ’clocks’ in this block.

Line 292: `for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():` This iterates raw over ‘path.read_text(encoding="utf-8", errors="ignore").splitlines()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_clock_timing, which handles parse clock timing.

Line 293: `fields = raw.split()` This assigns ‘raw.split()‘ to ‘fields ‘ in host memory. Here fields means the value named ’fields’ in this block. The important operands are raw: the value named ’raw’ in this block; split: the value named ’split’ in this block. Context: the function parse_clock_timing, which handles parse clock timing.

Line 294: `if len(fields) < 10 or not re.fullmatch(r"clk_fpga_\d+", fields[0]):` This executes the following indented host-code branch only when ‘len(fields) $<$ 10 or not re.fullmatch(r"clk_fpga_\d+", fields[0])‘ is true. The important operands are len: the value named ’len’ in this block; fields: the value named ’fields’ in this block; or: the value named ’or’ in this block. Context: the function parse_clock_timing, which handles parse clock timing.

Line 295: `continue` Inside the function parse_clock_timing, which handles parse clock timing, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named 'continue' in this block.

Line 296: `try:` This is explicit host-side failure handling inside the function parse_clock_timing, which handles parse clock timing; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 297: `setup_slack = float(fields[1])` This assigns ‘float(fields[1])‘ to ‘setup_slack ‘ in host memory. Here setup_slack means the value named 'setup slack' in this block. The important operands are float: the value named 'float' in this block; fields: the value named 'fields' in this block. Context: the function parse_clock_timing, which handles parse clock timing.

Line 298: `hold_slack = float(fields[5])` This assigns ‘float(fields[5])‘ to ‘hold_slack ‘ in host memory. Here hold_slack means the value named 'hold slack' in this block. The important operands are float: the value named 'float' in this block; fields: the value named 'fields' in this block. Context: the function parse_clock_timing, which handles parse clock timing.

Line 299: `except ValueError:` This is explicit host-side failure handling inside the function parse_clock_timing, which handles parse clock timing; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 300: `continue` Inside the function parse_clock_timing, which handles parse clock timing, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named 'continue' in this block.

Line 301: `clocks.setdefault(fields[0], (setup_slack, hold_slack))` This calls ‘clocks.setdefault‘ with ‘fields[0], (setup_slack, hold_slack)‘ inside the function parse_clock_timing, which handles parse clock timing. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 302: `return clocks` This returns ‘clocks‘ to the caller of the function parse_clock_timing, which handles parse clock timing, ending this invocation. The important operands are clocks: the value named 'clocks' in this block.

Line 305: `def parse_utilization_hierarchy(path: Path) -> dict[str, dict[str, int]]:` This starts function parse_utilization_hierarchy. Its indented body performs parse utilization hierarchy as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 306: `"""Parse named rows from Vivado’s hierarchical utilization table."""` This is executable documentation attached to the function parse_utilization_hierarchy, which handles parse utilization hierarchy. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 307: `wanted = {` This statement continues through source line 311; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "system_wrapper", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "s2mm_cdc", }‘ to ‘wanted ‘ in host memory. Here wanted means the value named 'wanted' in this block. The important operands are system_wrapper: the value named 'system wrapper' in this block; accelerator: the value named 'accelerator' in this block; accelerator_control_cdc: the value named 'accelerator control cdc' in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 308: `"system_wrapper", "accelerator", "accelerator_control_cdc",` This is continuation line 2 of the 5-line logical statement that begins at source line 307. Its exact fragment is ‘"system_wrapper", "accelerator", "accelerator_control_cdc",‘. The important operands are system_wrapper: the value named ’system wrapper’ in this block; accelerator: the value named ’accelerator’ in this block; accelerator_control_cdc: the value named ’accelerator control cdc’ in this block. Read together, the complete operation is: This assigns ‘{ "system_wrapper", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "s2mm_cdc", }‘ to ‘wanted ‘ in host memory. Here wanted means the value named ’wanted’ in this block. The important operands are system_wrapper: the value named ’system wrapper’ in this block; accelerator: the value named ’accelerator’ in this block; accelerator_control_cdc: the value named ’accelerator control cdc’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 309: `"control_interconnect", "dma", "dma_control_cdc",` This is continuation line 3 of the 5-line logical statement that begins at source line 307. Its exact fragment is ‘"control_interconnect", "dma", "dma_control_cdc",‘. The important operands are control_interconnect: the value named ’control interconnect’ in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; dma_control_cdc: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. Read together, the complete operation is: This assigns ‘{ "system_wrapper", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "s2mm_cdc", }‘ to ‘wanted ‘ in host memory. Here wanted means the value named ’wanted’ in this block. The important operands are system_wrapper: the value named ’system wrapper’ in this block; accelerator: the value named ’accelerator’ in this block; accelerator_control_cdc: the value named ’accelerator control cdc’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 310: `"memory_interconnect", "mm2s_cdc", "s2mm_cdc",` This is continuation line 4 of the 5-line logical statement that begins at source line 307. Its exact fragment is ‘"memory_interconnect", "mm2s_cdc", "s2mm_cdc",‘. The important operands are memory_interconnect: local FPGA storage addressed by surrounding index logic; mm2s_cdc: the value named ’mm2s cdc’ in this block; s2mm_cdc: the value named ’s2mm cdc’ in this block. Read together, the complete operation is: This assigns ‘{ "system_wrapper", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "s2mm_cdc", }‘ to ‘wanted ‘ in host memory. Here wanted means the value named ’wanted’ in this block. The important operands are system_wrapper: the value named ’system wrapper’ in this block; accelerator: the value named ’accelerator’ in this block; accelerator_control_cdc: the value named ’accelerator control cdc’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 311: `}` This is continuation line 5 of the 5-line logical statement that begins at source line 307. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "system_wrapper", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "s2mm_cdc", }‘ to ‘wanted ‘ in host memory. Here wanted means the value named ’wanted’ in this block. The important operands are system_wrapper: the value named ’system wrapper’ in this block; accelerator: the value named ’accelerator’ in this block; accelerator_control_cdc: the value named ’accelerator control cdc’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 312: `rows: dict[str, dict[str, int]] = {}` This assigns ‘{}‘ to ‘rows‘ in host memory. Here rows means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 313: `if not path.exists():` This executes the following indented host-code branch only when ‘not path.exists()‘ is true. The important operands are not: the value named ’not’ in this block; path: the value named ’path’ in this block; exists: the value named ’exists’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 314: `return rows` This returns ‘rows‘ to the caller of the function parse_utilization_hierarchy, which handles parse utilization hierarchy, ending this invocation. The important operands are rows: one output-row position processed by the tensor tile.

Line 315: `for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():` This iterates raw over ‘path.read_text(encoding="utf-8", errors="ignore").splitlines()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 316: `if not raw.startswith("|"):` This executes the following indented host-code branch only when ‘not raw.startswith("|")‘ is true. The important operands are not: the value named ’not’ in this block; raw: the value named ’raw’ in this block; startswith: the value named ’startswith’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 317: `continue` Inside the function parse_utilization_hierarchy, which handles parse utilization hierarchy, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named ’continue’ in this block.

Line 318: `fields = [field.strip() for field in raw.split("|")]` This assigns ‘[field.strip() for field in raw.split("|")]‘ to ‘fields ‘ in host memory. Here fields means the value named ’fields’ in this block. The important operands are field: the value named ’field’ in this block; strip: the value named ’strip’ in this block; for: the value named ’for’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 319: `if len(fields) < 12 or fields[1] not in wanted:` This executes the following indented host-code branch only when ‘len(fields) < 12 or fields[1] not in wanted‘ is true. The important operands are len: the value named ’len’ in this block; fields: the value named ’fields’ in this block; or: the value named ’or’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 320: `continue` Inside the function parse_utilization_hierarchy, which handles parse utilization hierarchy, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named ’continue’ in this block.

Line 321: `try:` This is explicit host-side failure handling inside the function parse_utilization_hierarchy, which handles parse utilization hierarchy; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 322: `rows[fields[1]] = {` This statement continues through source line 326; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "lut": int(fields[3]), "ff": int(fields[7]), "bram36": int(fields[8]), "bram18": int(fields[9]), "dsp": int(fields[10]), }‘ to ‘rows[fields[1]] ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are lut: the value named ’lut’ in this block; fields: the value named ’fields’ in this block; ff: the value named ’ff’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 323: `"lut": int(fields[3]), "ff": int(fields[7]),` This is continuation line 2 of the 5-line logical statement that begins at source line 322. Its exact fragment is ‘"lut": int(fields[3]), "ff": int(fields[7]),‘. The important operands are lut: the value named ’lut’ in this block; fields: the value named ’fields’ in this block; ff: the value named ’ff’ in this block. Read together, the complete operation is: This assigns ‘{ "lut": int(fields[3]), "ff": int(fields[7]), "bram36": int(fields[8]), "bram18": int(fields[9]), "dsp": int(fields[10]), }‘ to ‘rows[fields[1]] ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are lut: the value named ’lut’ in this block; fields: the value named ’fields’ in this block; ff: the value named ’ff’ in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 324: `"bram36": int(fields[8]), "bram18": int(fields[9]),` This is continuation line 3 of the 5-line logical statement that begins at source line 322. Its exact fragment is ‘"bram36": int(fields[8]), "bram18": int(fields[9]),‘. The important operands are bram36: the value named ’bram36’ in this block; fields: the value named ’fields’ in this block; bram18: the value named ’bram18’ in this block. Read together, the complete operation is: This assigns ‘{ "lut": int(fields[3]), "ff": int(fields[7]), "bram36": int(fields[8]), "bram18": int(fields[9]), "dsp": int(fields[10]), }‘ to ‘rows[fields[1]]

‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are lut: the value named 'lut' in this block; fields: the value named 'fields' in this block; ff: the value named 'ff' in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 325: `"dsp": int(fields[10]),` This is continuation line 4 of the 5-line logical statement that begins at source line 322. Its exact fragment is ‘"dsp": int(fields[10]),‘. The important operands are dsp: the value named 'dsp' in this block; fields: the value named 'fields' in this block. Read together, the complete operation is: This assigns ‘{ "lut": int(fields[3]), "ff": int(fields[7]), "bram36": int(fields[8]), "bram18": int(fields[9]), "dsp": int(fields[10]), }‘ to ‘rows[fields[1]] ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are lut: the value named 'lut' in this block; fields: the value named 'fields' in this block; ff: the value named 'ff' in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 326: `}` This is continuation line 5 of the 5-line logical statement that begins at source line 322. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "lut": int(fields[3]), "ff": int(fields[7]), "bram36": int(fields[8]), "bram18": int(fields[9]), "dsp": int(fields[10]), }‘ to ‘rows[fields[1]] ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are lut: the value named 'lut' in this block; fields: the value named 'fields' in this block; ff: the value named 'ff' in this block. Context: the function parse_utilization_hierarchy, which handles parse utilization hierarchy.

Line 327: `except ValueError:` This is explicit host-side failure handling inside the function parse_utilization_hierarchy, which handles parse utilization hierarchy; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 328: `continue` Inside the function parse_utilization_hierarchy, which handles parse utilization hierarchy, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are continue: the value named 'continue' in this block.

Line 329: `return rows` This returns ‘rows‘ to the caller of the function parse_utilization_hierarchy, which handles parse utilization hierarchy, ending this invocation. The important operands are rows: one output-row position processed by the tensor tile.

Line 332: `def parse_drc_summary(path: Path) -> list[tuple[str, str, str, int]]:` This starts function parse_drc_summary. Its indented body performs parse drc summary as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 333: `"""Parse DRC rule, severity, description, and count from the summary table."""` This is executable documentation attached to the function parse_drc_summary, which handles parse drc summary. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 334: `if not path.exists():` This executes the following indented host-code branch only when ‘not path.exists()‘ is true. The important operands are not: the value named 'not' in this block; path: the value named 'path' in this block; exists: the value named 'exists' in this block. Context: the function parse_drc_summary, which handles parse drc summary.

Line 335: `return []` This returns ‘[]‘ to the caller of the function parse_drc_summary, which handles parse drc summary, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 336: `pattern = re.compile(` This statement continues through source line 338; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘re.compile(r"^\|\s*([A-Z]+-\d+)\s*\|\s*(\w+)\s*\|\s*(.*?)
)‘ to ‘pattern ‘ in host memory. Here pattern means the value named 'pattern' in this block. The important operands

are re: the value named 're' in this block; compile: the value named 'compile' in this block; r: the value named 'r' in this block. Context: the function parse_drc_summary, which handles parse drc summary.

Line 337: `r"^\|\s*([A-Z]+-\d+)\s*\|\s*(\w+)\s*\|\s*(.*?)\s*\|\s*(\d+)\s*\|$"` This is continuation line 2 of the 3-line logical statement that begins at source line 336. Its exact fragment is 'r"^\|\s*([A-Z]+-\d+)\s*\|\s*(\w+)\s*\|\s*(.*?)\s
The important operands are r: the value named 'r' in this block; s: the value named 's' in this block; A: the value named 'a' in this block. Read together, the complete operation is: This assigns 're.compile(r"^\|\s*([A-Z]+-\d+)\s*\|\s*(\w+)\s*\|\s*(.*?)\s
)' to 'pattern ' in host memory. Here pattern means the value named 'pattern' in this block. The important operands are re: the value named 're' in this block; compile: the value named 'compile' in this block; r: the value named 'r' in this block. Context: the function parse_drc_summary, which handles parse drc summary.

Line 338: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 336. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 're.compile(r"^\|\s*([A-Z]+-\d+)\s*\|\s*(\w+)\s*\|\s*(.*?)\s*\|\s*(\d+)\s*\|$")' to 'pattern ' in host memory. Here pattern means the value named 'pattern' in this block. The important operands are re: the value named 're' in this block; compile: the value named 'compile' in this block; r: the value named 'r' in this block. Context: the function parse_drc_summary, which handles parse drc summary.

Line 339: `rows: list[tuple[str, str, str, int]] = []` This assigns '[]' to 'rows' in host memory. Here rows means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_drc_summary, which handles parse drc summary.

Line 340: `for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():` This iterates raw over 'path.read_text(encoding="utf-8", errors="ignore").splitlines()'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_drc_summary, which handles parse drc summary.

Line 341: `match = pattern.match(raw)` This assigns 'pattern.match(raw)' to 'match ' in host memory. Here match means the value named 'match' in this block. The important operands are pattern: the value named 'pattern' in this block; match: the value named 'match' in this block; raw: the value named 'raw' in this block. Context: the function parse_drc_summary, which handles parse drc summary.

Line 342: `if match:` This executes the following indented host-code branch only when 'match' is true. The important operands are match: the value named 'match' in this block. Context: the function parse_drc_summary, which handles parse drc summary.

Line 343: `rows.append((match.group(1), match.group(2), match.group(3), int(match.group(4))))` This calls 'rows.append' with '(match.group(1), match.group(2), match.group(3), int(match.group(4)))' inside the function parse_drc_summa
which handles parse drc summary. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 344: `return rows` This returns 'rows' to the caller of the function parse_drc_summary, which handles parse drc summary, ending this invocation. The important operands are rows: one output-row position processed by the tensor tile.

Line 347: `def parse_cdc_summary(path: Path) -> list[tuple[str, str, int, str]]:` This starts function parse_cdc_summary. Its indented body performs parse cdc summary as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 348: `"""Parse the compact CDC classification table before detailed paths."""` This is executable documentation attached to the function parse_cdc_summary, which handles parse cdc summary. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 349: `if not path.exists():` This executes the following indented host-code branch only when ‘not path.exists()‘ is true. The important operands are not: the value named ’not’ in this block; path: the value named ’path’ in this block; exists: the value named ’exists’ in this block. Context: the function parse_cdc_summary, which handles parse cdc summary.

Line 350: `return []` This returns ‘[]‘ to the caller of the function parse_cdc_summary, which handles parse cdc summary, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 351: `pattern = re.compile(r"^(CDC-\d+)\s+(\w+)\s+(\d+)\s+(.+)$")` This assigns ‘re.compile(r"^(CDC-\d+)\s+(\w+)\s+(\d+)\s+(.+)$")‘ to ‘pattern ‘ in host memory. Here pattern means the value named ’pattern’ in this block. The important operands are re: the value named ’re’ in this block; compile: the value named ’compile’ in this block; r: the value named ’r’ in this block. Context: the function parse_cdc_summary, which handles parse cdc summary.

Line 352: `rows: list[tuple[str, str, int, str]] = []` This assigns ‘[]‘ to ‘rows‘ in host memory. Here rows means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_cdc_summary, which handles parse cdc summary.

Line 353: `for raw in path.read_text(encoding="utf-8", errors="ignore").splitlines():` This iterates raw over ‘path.read_text(encoding="utf-8", errors="ignore").splitlines()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_cdc_summary, which handles parse cdc summary.

Line 354: `match = pattern.match(raw.strip())` This assigns ‘pattern.match(raw.strip())‘ to ‘match ‘ in host memory. Here match means the value named ’match’ in this block. The important operands are pattern: the value named ’pattern’ in this block; match: the value named ’match’ in this block; raw: the value named ’raw’ in this block. Context: the function parse_cdc_summary, which handles parse cdc summary.

Line 355: `if match:` This executes the following indented host-code branch only when ‘match‘ is true. The important operands are match: the value named ’match’ in this block. Context: the function parse_cdc_summary, which handles parse cdc summary.

Line 356: `rows.append((match.group(1), match.group(2), int(match.group(3)), match.group(4)))` This calls ‘rows.append‘ with ‘(match.group(1), match.group(2), int(match.group(3)), match.group(4))‘ inside the function parse_cdc_summa which handles parse cdc summary. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 357: `elif rows:` This executes the following indented host-code branch only when ‘rows‘ is true. The important operands are rows: one output-row position processed by the tensor tile. Context: the function parse_cdc_summary, which handles parse cdc summary.

Line 358: `break` Inside the function parse_cdc_summary, which handles parse cdc summary, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are break: the value named ’break’ in this block.

Line 359: `return rows` This returns ‘rows‘ to the caller of the function parse_cdc_summary, which handles parse cdc summary, ending this invocation. The important operands are rows: one output-row position processed by the tensor tile.

Line 362: `def parse_power_report(path: Path) -> dict[str, object]:` This starts function parse_power_report. Its indented body performs parse power report as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 363: `"""Extract explicit Vivado power totals, resources, hierarchy, and confidence."""` This is executable documentation attached to the function parse_power_report, which handles parse power report. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 364: `if not path.exists():` This executes the following indented host-code branch only when ‘not path.exists()‘ is true. The important operands are not: the value named ’not’ in this block; path: the value named ’path’ in this block; exists: the value named ’exists’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 365: `return {}` This returns ‘{}‘ to the caller of the function parse_power_report, which handles parse power report, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 366: `text = path.read_text(encoding="utf-8", errors="ignore")` This assigns ‘path.read_text(encoding="utf-8", errors="ignore")‘ to ‘text ‘ in host memory. Here text means the value named ’text’ in this block. The important operands are path: the value named ’path’ in this block; read_text: the value named ’read text’ in this block; encoding: the value named ’encoding’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 368: `def number(label: str) -> float | None:` This starts function number. Its indented body performs number as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 369: `match = re.search(rf"^\|\s*{re.escape(label)}\s*\|\s*([0-9.]+)", text, re.MULTILINE)` This assigns ‘re.search(rf"^\|\s*{re.escape(label)}\s*\|\s*([0-9.]+)", text, re.MULTILINE)‘ to ‘match ‘ in host memory. Here match means the value named ’match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; rf: the value named ’rf’ in this block. Context: the function number, which handles number.

Line 370: `return float(match.group(1)) if match else None` This returns ‘float(match.group(1)) if match else None‘ to the caller of the function number, which handles number, ending this invocation. The important operands are float: the value named ’float’ in this block; match: the value named ’match’ in this block; group: the value named ’group’ in this block.

Line 372: `confidence_match = re.search(` This statement continues through source line 374; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘re.search(r"^\|\s*Confidence Level\s*\|\s*(\w+)", text, re.MULTILINE)‘ to ‘confidence_match ‘ in host memory. Here confidence_match means the value named ’confidence match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; r: the value named ’r’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 373: `r"^\|\s*Confidence Level\s*\|\s*(\w+)", text, re.MULTILINE` This is continuation line 2 of the 3-line logical statement that begins at source line 372. Its exact fragment is ‘r"^\|\s*Confidence Level\s*\|\s*(\w+)", text, re.MULTILINE‘. The important operands are r: the value named ’r’ in this block; s: the value named ’s’ in this block; Confidence: the value named ’confidence’ in this block. Read together, the complete operation is: This assigns ‘re.search(r"^\|\s*Confidence Level\s*\|\s*(\w+)", text, re.MULTILINE)‘ to ‘confidence_match ‘ in host memory. Here confidence_match means the value named ’confidence match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; r: the value named ’r’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 374: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 372. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘re.search(r"^\|\s*Confidence Level\s*\|\s*(\w+)", text, re.MULTILINE)‘ to ‘confidence_match ‘ in host memory. Here confidence_match means the value named ’confidence match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; r: the value named ’r’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 375: `matched_match = re.search(` This statement continues through source line 377; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘re.search(r"^\|\s*Design Nets Matched\s*\|\s*([^|]+)", text, re.MULTILINE)‘ to ‘matched_match ‘ in host memory. Here matched_match means the value named ’matched match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; r: the value named ’r’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 376: `r"^\|\s*Design Nets Matched\s*\|\s*([^|]+)", text, re.MULTILINE` This is continuation line 2 of the 3-line logical statement that begins at source line 375. Its exact fragment is ‘r"^\|\s*Design Nets Matched\s*\|\s*([^|]+)", text, re.MULTILINE‘. The important operands are r: the value named ’r’ in this block; s: the value named ’s’ in this block; Design: the value named ’design’ in this block. Read together, the complete operation is: This assigns ‘re.search(r"^\|\s*Design Nets Matched\s*\|\s*([^|]+)", text, re.MULTILINE)‘ to ‘matched_match ‘ in host memory. Here matched_match means the value named ’matched match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; r: the value named ’r’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 377: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 375. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘re.search(r"^\|\s*Design Nets Matched\s*\|\s*([^|]+)", text, re.MULTILINE)‘ to ‘matched_match ‘ in host memory. Here matched_match means the value named ’matched match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; r: the value named ’r’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 378: `resources: dict[str, float] = {}` This assigns ‘{}‘ to ‘resources‘ in host memory. Here resources means the value named ’resources’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_power_report, which handles parse power report.

Line 379: `for label in ("Clocks", "Slice Logic", "Signals", "Block RAM", "DSPs", "PS7", "Static Power", "Total"):` This iterates label over ‘("Clocks", "Slice Logic", "Signals", "Block RAM", "DSPs", "PS7", "Static Power", "Total")‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_power_report, which handles parse power report.

Line 380: `match = re.search(` This statement continues through source line 382; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘re.search(rf"^\|\s*{re.escape(label)}\s*\|\s*<?([0-9.]+)", text, re.MULTILINE)‘ to ‘match ‘ in host memory. Here match means the value named ’match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; rf: the value named ’rf’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 381: `rf"^\|\s*{re.escape(label)}\s*\|\s*<?([0-9.]+)", text, re.MULTILINE` This is continuation line 2 of the 3-line logical statement that begins at source line 380. Its exact fragment is ‘rf"^\|\s*{re.escape(label)}\s*\|\s*<?([0-9.]+)", text, re.MULTILINE‘. The important operands are rf: the value named ’rf’ in this block; s: the value named ’s’ in this block; re: the value named ’re’ in this block. Read together, the complete operation is: This assigns ‘re.search(rf"^\|\s*{re.escape(label)}\s*\|\s*<?([0-9.]+)", text, re.MULTILINE)‘ to ‘match ‘ in host memory. Here match means the value named ’match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; rf: the value named ’rf’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 382: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 380. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘re.search(rf"^\|\s*{re.escape(label)}\s*\|\s*<?([0-9.]+)", text, re.MULTILINE)‘ to ‘match ‘ in host memory. Here match means the value named ’match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; rf: the value named ’rf’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 383: `if match:` This executes the following indented host-code branch only when ‘match‘ is true. The important operands are match: the value named ’match’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 384: `resources[label] = float(match.group(1))` This assigns ‘float(match.group(1))‘ to ‘resources[label] ‘ in host memory. Here resources means the value named ’resources’ in this block. The important operands are float: the value named ’float’ in this block; match: the value named ’match’ in this block; group: the value named ’group’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 385: `hierarchy: dict[str, float] = {}` This assigns ‘{}‘ to ‘hierarchy‘ in host memory. Here hierarchy means the value named ’hierarchy’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function parse_power_report, which handles parse power report.

Line 386: `hierarchy_names = (` This statement continues through source line 390; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘("system_wrapper", "system_i", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "ps7", "reset_100", "reset_125", "reset_200", "s2mm_cdc",)‘ to ‘hierarchy_names ‘ in host memory. Here hierarchy_names means the value named ’hierarchy names’ in this block. The important operands are system_wrapper: the value named ’system wrapper’ in this block; system_i: the value named ’system i’ in this block; accelerator: the value named ’accelerator’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 387: `"system_wrapper", "system_i", "accelerator", "accelerator_control_cdc",` This is continuation line 2 of the 5-line logical statement that begins at source line 386. Its exact fragment is ‘"system_wrapper", "system_i", "accelerator", "accelerator_control_cdc",‘. The important operands are system_wrapper: the value named ’system wrapper’ in this block; system_i: the value named ’system i’ in this block; accelerator: the value named ’accelerator’ in this block. Read together, the complete operation is: This assigns ‘("system_wrapper", "system_i", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "ps7", "reset_100", "reset_125", "reset_200", "s2mm_cdc",)‘ to ‘hierarchy_names ‘ in host memory. Here hierarchy_names means the value named ’hierarchy names’ in this block. The important operands are system_wrapper: the value named ’system wrapper’ in this block; system_i: the value named ’system i’ in this block; accelerator: the value named ’accelerator’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 388: `"control_interconnect", "dma", "dma_control_cdc", "memory_interconnect",` This is continuation line 3 of the 5-line logical statement that begins at source line 386. Its exact fragment is ‘"control_interconnect", "dma", "dma_control_cdc", "memory_interconnect",‘. The important operands are control_interconnect: the value named ’control interconnect’ in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; dma_control_cdc: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. Read together, the complete operation is: This assigns ‘("system_wrapper", "system_i", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "ps7", "reset_100", "reset_125", "reset_200", "s2mm_cdc",)‘ to ‘hierarchy_names ‘ in host memory. Here hierarchy_names means the value named ’hierarchy names’ in this block. The important operands are system_wrapper: the value named ’system wrapper’ in this block; system_i: the value named ’system i’ in this block; accelerator: the value named ’accelerator’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 389: `"mm2s_cdc", "ps7", "reset_100", "reset_125", "reset_200", "s2mm_cdc",` This is continuation line 4 of the 5-line logical statement that begins at source line 386. Its exact fragment is ‘"mm2s_cdc", "ps7", "reset_100", "reset_125", "reset_200", "s2mm_cdc",‘. The important operands are mm2s_cdc: the value named ’mm2s cdc’ in this block; ps7: the value named ’ps7’ in this block; reset_100: the value named ’reset 100’ in this block. Read together, the complete operation is: This assigns ‘("system_wrapper", "system_i", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "ps7", "reset_100", "reset_125", "reset_200", "s2mm_cdc",)‘ to ‘hierarchy_names ‘ in host memory. Here hierarchy_names means the value named ’hierarchy names’ in this block. The important operands are system_wrapper: the value named ’system wrapper’ in this block; system_i: the value named ’system i’ in this block; accelerator: the value named ’accelerator’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 390: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 386. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘("system_wrapper", "system_i", "accelerator", "accelerator_control_cdc", "control_interconnect", "dma", "dma_control_cdc", "memory_interconnect", "mm2s_cdc", "ps7", "reset_100", "reset_125", "reset_200", "s2mm_cdc",)‘ to ‘hierarchy_names ‘ in host memory. Here hierarchy_names means the value named ’hierarchy names’ in this block. The important operands are system_wrapper: the value named ’system wrapper’ in this block; system_i: the value named ’system i’ in this block; accelerator: the value named ’accelerator’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 391: `for name in hierarchy_names:` This iterates name over ‘hierarchy_names‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function parse_power_report, which handles parse power report.

Line 392: `match = re.search(` This statement continues through source line 394; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘re.search(rf"^\|\s*{re.escape(name)}\s*\|\s*([0-9.]+)\s*\|", text, re.MULTILINE)‘ to ‘match ‘ in host memory. Here match means the value named ’match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; rf: the value named ’rf’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 393: `rf"^\|\s*{re.escape(name)}\s*\|\s*([0-9.]+)\s*\|", text, re.MULTILINE` This is continuation line 2 of the 3-line logical statement that begins at source line 392. Its exact fragment is ‘rf"^\|\s*{re.escape(name)}\s*\|\s*([0-9.]+)\s*\|", text, re.MULTILINE‘. The important operands are rf: the value named ’rf’ in this block; s: the value named ’s’ in this block; re: the value named ’re’ in this block. Read together, the complete operation is: This assigns ‘re.search(rf"^\|\s*{re.escape(name)}\s*\|\s*([0-9.]+)\s*\|", text, re.MULTILINE)‘ to ‘match ‘ in host memory. Here match means the value named ’match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; rf: the value named ’rf’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 394: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 392. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘re.search(rf"^\|\s*{re.escape(name)}\s*\|\s*([0-9.]+)\s*\|", text, re.MULTILINE)‘ to ‘match ‘ in host memory. Here match means the value named ’match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; rf: the value named ’rf’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 395: `if match:` This executes the following indented host-code branch only when ‘match‘ is true. The important operands are match: the value named ’match’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 396: `hierarchy[name] = float(match.group(1))` This assigns ‘float(match.group(1))‘ to ‘hierarchy[name] ‘ in host memory. Here hierarchy means the value named ’hierarchy’ in this block. The important operands are float: the value named ’float’ in this block; match: the value named ’match’ in this block; group: the value named ’group’ in this block. Context: the function parse_power_report, which handles parse power report.

Line 397: `return {` This statement continues through source line 405; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘{ "total": number("Total On-Chip Power (W)"), "dynamic": number("Dynamic (W)"), "static": number("Device Static (W)"), "confidence": confidence_match.group(1) if confidence_match else "unknown", "matched": matched_match.group(1).strip() if matched_match else "not reported", "resources": resources, "hierarchy": hierarchy, }‘ to the caller of the function parse_power_report, which handles parse power report, ending this invocation. The important operands are total: the value named ’total’ in this block; number: the value named ’number’ in this block; Total: the value named ’total’ in this block.

Line 398: `"total": number("Total On-Chip Power (W)"),` This is continuation line 2 of the 9-line logical statement that begins at source line 397. Its exact fragment is ‘"total": number("Total On-Chip Power (W)"),‘. The important operands are total: the value named ’total’ in this block; number: the value named ’number’ in this block;

Total: the value named 'total' in this block. Read together, the complete operation is: This returns ‘{ "total": number("Total On-Chip Power (W)"), "dynamic": number("Dynamic (W)"), "static": number("Device Static (W)"), "confidence": confidence_match.group(1) if confidence_match else "unknown", "matched": matched_match.group(1).strip() if matched_match else "not reported", "resources": resources, "hierarchy": hierarchy, }‘ to the caller of the function parse_power_report, which handles parse power report, ending this invocation. The important operands are total: the value named 'total' in this block; number: the value named 'number' in this block; Total: the value named 'total' in this block.

Line 399: `"dynamic": number("Dynamic (W)"),` This is continuation line 3 of the 9-line logical statement that begins at source line 397. Its exact fragment is ‘"dynamic": number("Dynamic (W)"),‘. The important operands are dynamic: the value named 'dynamic' in this block; number: the value named 'number' in this block; Dynamic: the value named 'dynamic' in this block. Read together, the complete operation is: This returns ‘{ "total": number("Total On-Chip Power (W)"), "dynamic": number("Dynamic (W)"), "static": number("Device Static (W)"), "confidence": confidence_match.group(1) if confidence_match else "unknown", "matched": matched_match.group(1).strip() if matched_match else "not reported", "resources": resources, "hierarchy": hierarchy, }‘ to the caller of the function parse_power_report, which handles parse power report, ending this invocation. The important operands are total: the value named 'total' in this block; number: the value named 'number' in this block; Total: the value named 'total' in this block.

Line 400: `"static": number("Device Static (W)"),` This is continuation line 4 of the 9-line logical statement that begins at source line 397. Its exact fragment is ‘"static": number("Device Static (W)"),‘. The important operands are static: the value named 'static' in this block; number: the value named 'number' in this block; Device: the value named 'device' in this block. Read together, the complete operation is: This returns ‘{ "total": number("Total On-Chip Power (W)"), "dynamic": number("Dynamic (W)"), "static": number("Device Static (W)"), "confidence": confidence_match.group(1) if confidence_match else "unknown", "matched": matched_match.group(1).strip() if matched_match else "not reported", "resources": resources, "hierarchy": hierarchy, }‘ to the caller of the function parse_power_report, which handles parse power report, ending this invocation. The important operands are total: the value named 'total' in this block; number: the value named 'number' in this block; Total: the value named 'total' in this block.

Line 401: `"confidence": confidence_match.group(1) if confidence_match else "unknown",` This is continuation line 5 of the 9-line logical statement that begins at source line 397. Its exact fragment is ‘"confidence": confidence_match.group(1) if confidence_match else "unknown",‘. The important operands are confidence: the value named 'confidence' in this block; confidence_match: the value named 'confidence match' in this block; group: the value named 'group' in this block. Read together, the complete operation is: This returns ‘{ "total": number("Total On-Chip Power (W)"), "dynamic": number("Dynamic (W)"), "static": number("Device Static (W)"), "confidence": confidence_match.group(1) if confidence_match else "unknown", "matched": matched_match.group(1).strip() if matched_match else "not reported", "resources": resources, "hierarchy": hierarchy, }‘ to the caller of the function parse_power_report, which handles parse power report, ending this invocation. The important operands are total: the value named 'total' in this block; number: the value named 'number' in this block; Total: the value named 'total' in this block.

Line 402: `"matched": matched_match.group(1).strip() if matched_match else "not reported",` This is continuation line 6 of the 9-line logical statement that begins at source line 397. Its exact fragment is ‘"matched": matched_match.group(1).strip() if matched_match else "not reported",‘. The important operands are matched: the value named 'matched' in this block; matched_match: the value named 'matched match' in this block; group: the value named 'group' in this block. Read together, the complete operation is: This returns ‘{ "total": number("Total On-Chip Power (W)"), "dynamic": number("Dynamic (W)"), "static": number("Device Static (W)"), "confidence": confidence_match.group(1) if confidence_match else "unknown", "matched": matched_match.group(1).strip() if matched_match else "not reported", "resources": resources, "hierarchy": hierarchy, }‘ to the caller of the function parse_power_report, which handles parse power report, ending this invocation. The important operands are total: the value named 'total' in this block; number: the value named 'number' in this block; Total: the value named 'total' in this block.

Line 403: `"resources": resources,` This is continuation line 7 of the 9-line logical statement that begins at source line 397. Its exact fragment is ‘"resources": resources,‘. The important operands are resources: the value named 'resources' in this block. Read together, the complete operation is: This returns ‘{ "total": number("Total On-Chip Power (W)"), "dynamic": number("Dynamic (W)"), "static": number("Device Static (W)"), "confidence": confidence_match.group(1) if confidence_match else "unknown", "matched": matched_match.group(1).strip() if matched_match else "not reported", "resources": resources, "hierarchy": hierarchy, }‘ to the caller of the function parse_power_report, which handles parse

power report, ending this invocation. The important operands are total: the value named 'total' in this block; number: the value named 'number' in this block; Total: the value named 'total' in this block.

Line 404: `"hierarchy": hierarchy,` This is continuation line 8 of the 9-line logical statement that begins at source line 397. Its exact fragment is ‘"hierarchy": hierarchy,‘. The important operands are hierarchy: the value named 'hierarchy' in this block. Read together, the complete operation is: This returns ‘{ "total": number("Total On-Chip Power (W)"), "dynamic": number("Dynamic (W)"), "static": number("Device Static (W)"), "confidence": confidence_match.group(1) if confidence_match else "unknown", "matched": matched_match.group(1).strip() if matched_match else "not reported", "resources": resources, "hierarchy": hierarchy, }‘ to the caller of the function parse_power_report, which handles parse power report, ending this invocation. The important operands are total: the value named 'total' in this block; number: the value named 'number' in this block; Total: the value named 'total' in this block.

Line 405: `}` This is continuation line 9 of the 9-line logical statement that begins at source line 397. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘{ "total": number("Total On-Chip Power (W)"), "dynamic": number("Dynamic (W)"), "static": number("Device Static (W)"), "confidence": confidence_match.group(1) if confidence_match else "unknown", "matched": matched_match.group(1).strip() if matched_match else "not reported", "resources": resources, "hierarchy": hierarchy, }‘ to the caller of the function parse_power_report, which handles parse power report, ending this invocation. The important operands are total: the value named 'total' in this block; number: the value named 'number' in this block; Total: the value named 'total' in this block.

Line 408: `def file_evidence(path: Path) -> tuple[int, str]:` This starts function file_evidence. Its indented body performs file evidence as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 409: `digest = hashlib.sha256()` This assigns ‘hashlib.sha256()‘ to ‘digest ‘ in host memory. Here digest means the value named 'digest' in this block. The important operands are hashlib: the value named 'hashlib' in this block; sha256: the value named 'sha256' in this block. Context: the function file_evidence, which handles file evidence.

Line 410: `with path.open("rb") as handle:` This opens a managed resource scope for ‘path.open("rb") as handle‘ so files, tracing, or temporary state are finalized even if the function file_evidence, which handles file evidence raises an error.

Line 411: `for chunk in iter(lambda: handle.read(1024 * 1024), b""):` This iterates chunk over ‘iter(lambda: handle.read(1024 * 1024), b"")‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function file_evidence, which handles file evidence.

Line 412: `digest.update(chunk)` This calls ‘digest.update‘ with ‘chunk‘ inside the function file_evidence, which handles file evidence. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 413: `return path.stat().st_size, digest.hexdigest()` This returns ‘path.stat().st_size, digest.hexdigest()‘ to the caller of the function file_evidence, which handles file evidence, ending this invocation. The important operands are path: the value named 'path' in this block; stat: the value named 'stat' in this block; st_size: the value named 'st size' in this block.

Line 416: `def latex_escape(value: str) -> str:` This starts function latex_escape. Its indented body performs latex escape as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 417: `replacements = {` This statement continues through source line 421; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", }‘ to ‘replacements ‘ in host memory. Here replacements means the value named 'replacements' in this block. The important

operands are r: the value named 'r' in this block; textbackslash: the value named 'textbackslash' in this block; _: the value named '' in this block. Context: the function latex_escape, which handles latex escape.

Line 418: `"\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$",` This is continuation line 2 of the 5-line logical statement that begins at source line 417. Its exact fragment is `'"\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$",'`. The important operands are r: the value named 'r' in this block; textbackslash: the value named 'textbackslash' in this block. Read together, the complete operation is: This assigns `'{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"\_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", }'` to 'replacements ' in host memory. Here replacements means the value named 'replacements' in this block. The important operands are r: the value named 'r' in this block; textbackslash: the value named 'textbackslash' in this block; _: the value named '' in this block. Context: the function latex_escape, which handles latex escape.

Line 419: `"#": r"\#", "_": r"\_", "{": r"\{", "}": r"\},` This is continuation line 3 of the 5-line logical statement that begins at source line 417. Its exact fragment is `'"#": r"\#", "_": r"\_", "{": r"\{", "}": r"\},'`. The important operands are r: the value named 'r' in this block; _: the value named '' in this block. Read together, the complete operation is: This assigns `'{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"\_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", }'` to 'replacements ' in host memory. Here replacements means the value named 'replacements' in this block. The important operands are r: the value named 'r' in this block; textbackslash: the value named 'textbackslash' in this block; _: the value named '' in this block. Context: the function latex_escape, which handles latex escape.

Line 420: `"~": r"\textasciitilde{}", "^": r"\textasciicircum{}",` This is continuation line 4 of the 5-line logical statement that begins at source line 417. Its exact fragment is `'"~": r"\textasciitilde{}", "^": r"\textasciicircum{}",'`. The important operands are r: the value named 'r' in this block; textasciitilde: the value named 'textasciitilde' in this block; textasciicircum: the value named 'textasciicircum' in this block. Read together, the complete operation is: This assigns `'{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"\_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", }'` to 'replacements ' in host memory. Here replacements means the value named 'replacements' in this block. The important operands are r: the value named 'r' in this block; textbackslash: the value named 'textbackslash' in this block; _: the value named '' in this block. Context: the function latex_escape, which handles latex escape.

Line 421: `}` This is continuation line 5 of the 5-line logical statement that begins at source line 417. Its exact fragment is `'}'`. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns `'{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"\_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", }'` to 'replacements ' in host memory. Here replacements means the value named 'replacements' in this block. The important operands are r: the value named 'r' in this block; textbackslash: the value named 'textbackslash' in this block; _: the value named '' in this block. Context: the function latex_escape, which handles latex escape.

Line 422: `return "".join(replacements.get(character, character) for character in value)` This returns `'"".join(replacements.get(character, character) for character in value)'` to the caller of the function latex_escape, which handles latex escape, ending this invocation. The important operands are join: the value named 'join' in this block; replacements: the value named 'replacements' in this block; get: the value named 'get' in this block.

Line 425: `def latex_breakable(value: str) -> str:` This starts function latex_breakable. Its indented body performs latex breakable as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 426: `"""Escape provenance text while adding legal line-break opportunities."""` This is executable documentation attached to the function latex_breakable, which handles latex breakable. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 427: `escaped = latex_escape(value)` This assigns `'latex_escape(value)'` to 'escaped ' in host memory. Here escaped means the value named 'escaped' in this block. The important operands are latex_escape: the value named

'latex escape' in this block; value: the value named 'value' in this block. Context: the function latex_breakable, which handles latex breakable.

Line 428: `escaped = escaped.replace(r"\textbackslash{}", r"\textbackslash{}\allowbreak{}")` This assigns 'escaped.replace(r"\textbackslash{}", r"\textbackslash{}\allowbreak{}")' to 'escaped ' in host memory. Here escaped means the value named 'escaped' in this block. The important operands are escaped: the value named 'escaped' in this block; replace: the value named 'replace' in this block; r: the value named 'r' in this block. Context: the function latex_breakable, which handles latex breakable.

Line 429: `escaped = escaped.replace(r"\_", r"\_\allowbreak{}")` This assigns 'escaped.replace(r"_", r"_\allowbreak{}
to 'escaped ' in host memory. Here escaped means the value named 'escaped' in this block. The important operands are escaped: the value named 'escaped' in this block; replace: the value named 'replace' in this block; r: the value named 'r' in this block. Context: the function latex_breakable, which handles latex breakable.

Line 430: `escaped = escaped.replace("/", r"/\allowbreak{}")` This assigns 'escaped.replace("/", r"/\allowbreak{}")' to 'escaped ' in host memory. Here escaped means the value named 'escaped' in this block. The important operands are escaped: the value named 'escaped' in this block; replace: the value named 'replace' in this block; r: the value named 'r' in this block. Context: the function latex_breakable, which handles latex breakable.

Line 431: `escaped = escaped.replace(";", r";\allowbreak{}")` This assigns 'escaped.replace(";", r";\allowbreak{}")' to 'escaped ' in host memory. Here escaped means the value named 'escaped' in this block. The important operands are escaped: the value named 'escaped' in this block; replace: the value named 'replace' in this block; r: the value named 'r' in this block. Context: the function latex_breakable, which handles latex breakable.

Line 432: `escaped = escaped.replace(", ", r",\allowbreak{} ")` This assigns 'escaped.replace(", ", r",\allowbreak{}
")' to 'escaped ' in host memory. Here escaped means the value named 'escaped' in this block. The important operands are escaped: the value named 'escaped' in this block; replace: the value named 'replace' in this block; r: the value named 'r' in this block. Context: the function latex_breakable, which handles latex breakable.

Line 433: `return escaped` This returns 'escaped' to the caller of the function latex_breakable, which handles latex breakable, ending this invocation. The important operands are escaped: the value named 'escaped' in this block.

Line 436: `def write_report_tex(` This starts function write_report_tex. Its indented body performs write report tex as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 437: `output_dir: Path,` Inside the function write_report_tex, which handles write report tex, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are output_dir: the value named 'output dir' in this block; Path: the value named 'path' in this block.

Line 438: `records: Sequence[PlotRecord],` Inside the function write_report_tex, which handles write report tex, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are records: the number of descriptor records accepted since program start; Sequence: the value named 'sequence' in this block; PlotRecord: the value named 'plot record' in this block.

Line 439: `block_design_filename: str | None,` Inside the function write_report_tex, which handles write report tex, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are block_design_filename: the value named 'block design filename' in this block; str: the value named 'str' in this block; None: the value named 'none' in this block.

Line 440: `) -> Path:` Inside the function write_report_tex, which handles write report tex, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are Path: the value named 'path' in this block.

Line 441: `tex_path = output_dir / "neuraedge_deep_debug_report.tex"` This assigns 'output_dir / "neuraedge_deep_deb to 'tex_path ' in host memory. Here tex_path means the value named 'tex path' in this block. The important operands are output_dir: the value named 'output dir' in this block; neuraedge_deep_debug_report: the value named 'neuraedge deep debug report' in this block; tex: the value named 'tex' in this block. Context: the function write_report_tex, which handles write report tex.

Line 442: `lines = [` This statement continues through source line 461; the explanation uses the complete expression rather than only the first visual fragment. This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry} r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document} r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named 'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex, which handles write report tex.

Line 443: `r"\documentclass[10pt]{article}",` This is continuation line 2 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\documentclass[10pt]{article}",'. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named 'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex, which handles write report tex.

Line 444: `r"\usepackage[margin=0.65in]{geometry}",` This is continuation line 3 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\usepackage[margin=0.65in]{geometry}",'. The important operands are r: the value named 'r' in this block; usepackage: the value named 'usepackage' in this block; margin: the value named 'margin' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named 'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex, which handles write report tex.

Line 445: `r"\usepackage[T1]{fontenc}",` This is continuation line 4 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\usepackage[T1]{fontenc}",'. The important operands are r: the value named 'r' in this block; usepackage: the value named 'usepackage' in this block; T1: the value named 't1' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}",

r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli
r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and
Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}
r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact.
CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository
testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate
is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named
’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named
’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex,
which handles write report tex.

Line 446: `r"\usepackage{lmodern}",` This is continuation line 5 of the 20-line logical statement that begins at source line 442. Its exact fragment is ‘r"\usepackage{lmodern}",‘. The important operands are r: the value named ’r’ in this block; usepackage: the value named ’usepackage’ in this block; lmodern: the value named ’lmodern’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}",
r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli
r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and
Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}
r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact.
CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository
testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate
is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named
’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named
’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex,
which handles write report tex.

Line 447: `r"\usepackage{graphicx}",` This is continuation line 6 of the 20-line logical statement that begins at source line 442. Its exact fragment is ‘r"\usepackage{graphicx}",‘. The important operands are r: the value named ’r’ in this block; usepackage: the value named ’usepackage’ in this block; graphicx: the value named ’graphicx’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}",
r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli
r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and
Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}
r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact.
CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository
testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate
is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named
’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named
’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex,
which handles write report tex.

Line 448: `r"\usepackage{pdfpages}",` This is continuation line 7 of the 20-line logical statement that begins at source line 442. Its exact fragment is ‘r"\usepackage{pdfpages}",‘. The important operands are r: the value named ’r’ in this block; usepackage: the value named ’usepackage’ in this block; pdfpages: the value named ’pdfpages’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}",
r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli
r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and
Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}
r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact.
CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository
testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate
is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named
’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named
’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex,
which handles write report tex.

Line 449: `r"\usepackage[hidelinks]{hyperref}",` This is continuation line 8 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\usepackage[hidelinks]{hyperref}",'. The important operands are r: the value named 'r' in this block; usepackage: the value named 'usepackage' in this block; hidelinks: the value named 'hidelinks' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named 'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex, which handles write report tex.

Line 450: `r"\setlength{\parindent}{0pt}",` This is continuation line 9 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\setlength{\parindent}{0pt}",'. The important operands are r: the value named 'r' in this block; setlength: the value named 'setlength' in this block; parindent: the value named 'parindent' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named 'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex, which handles write report tex.

Line 451: `r"\setlength{\parskip}{5pt}",` This is continuation line 10 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\setlength{\parskip}{5pt}",'. The important operands are r: the value named 'r' in this block; setlength: the value named 'setlength' in this block; parskip: the value named 'parskip' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli
r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and
Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}
r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact.
CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository
testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate
is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named
'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named
'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex,
which handles write report tex.

Line 452: `r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}",` This is continuation line 11 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}",'. The important operands are r: the value named 'r' in this block; title: the value named 'title' in this block; NeuraEdge: the value named 'neura edge' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli
r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and
Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}
r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact.
CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository
testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate
is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named

'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex, which handles write report tex.

Line 453: `r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}",` This is continuation line 12 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}",'. The important operands are r: the value named 'r' in this block; author: the value named 'author' in this block; Generated: the value named 'generated' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document} r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named 'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex, which handles write report tex.

Line 454: `r"\date{\today}",` This is continuation line 13 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\date{\today}",'. The important operands are r: the value named 'r' in this block; date: the value named 'date' in this block; today: the value named 'today' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{ r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named 'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex, which handles write report tex.

Line 455: `r"\begin{document}",` This is continuation line 14 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\begin{document}",'. The important operands are r: the value named 'r' in this block; document: the value named 'document' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]' to 'lines ' in host memory. Here lines means the value named 'lines' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function write_report_tex, which handles write report tex.

Line 456: `r"\maketitle",` This is continuation line 15 of the 20-line logical statement that begins at source line 442. Its exact fragment is 'r"\maketitle",'. The important operands are r: the value named 'r' in this block; maketitle: the value named 'maketitle' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included

only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named ’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex, which handles write report tex.

Line 457: `r"\tableofcontents",` This is continuation line 16 of the 20-line logical statement that begins at source line 442. Its exact fragment is ‘r"\tableofcontents",‘. The important operands are r: the value named ’r’ in this block; tableofcontents: the value named ’tableofcontents’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named ’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex, which handles write report tex.

Line 458: `r"\clearpage",` This is continuation line 17 of the 20-line logical statement that begins at source line 442. Its exact fragment is ‘r"\clearpage",‘. The important operands are r: the value named ’r’ in this block; clearpage: the value named ’clearpage’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named ’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex, which handles write report tex.

Line 459: `r"\section{Evidence rule}",` This is continuation line 18 of the 20-line logical statement that begins at source line 442. Its exact fragment is ‘r"\section{Evidence rule}",‘. The important operands are r: the value named ’r’ in this block; section: the value named ’section’ in this block; Evidence: the value named ’evidence’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document} r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named ’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex, which handles write report tex.

Line 460: `"Every page names its source artifact.  CIFAR-10 plots use all 10,000 official test samples.  Waveforms are g...` This is continuation line 19 of the 20-line logical statement that begins at source line 442. Its exact fragment is ‘"Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",‘. The important operands are Every: the value named ’every’ in this block; page: the value named ’page’ in this block; names: the value named ’names’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}",

r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named ’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex, which handles write report tex.

Line 461: `]` This is continuation line 20 of the 20-line logical statement that begins at source line 442. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘[r"\documentclass[10pt]{article}", r"\usepackage[margin=0.65in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{graphicx}", r"\usepackage{pdfpages}", r"\usepackage[hideli r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{5pt}", r"\title{NeuraEdge Universal: Deep Debug Plots and Waveforms}", r"\author{Generated from CIFAR-10, XSim, compiler, and Vivado artifacts}", r"\date{\today}", r"\begin{document} r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{Evidence rule}", "Every page names its source artifact. CIFAR-10 plots use all 10,000 official test samples. Waveforms are generated by Vivado 2026.1 XSim from repository testbenches. Timing and power pages are included only when the corresponding routed report exists. A Vivado estimate is not relabeled as a physical board measurement.",]‘ to ‘lines ‘ in host memory. Here lines means the value named ’lines’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function write_report_tex, which handles write report tex.

Line 462: `if block_design_filename is not None:` This executes the following indented host-code branch only when ‘block_design_filename is not None‘ is true. The important operands are block_design_filename: the value named ’block design filename’ in this block; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function write_report_tex, which handles write report tex.

Line 463: `lines.extend(` This statement continues through source line 470; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘lines.extend‘ with ‘ [r"\clearpage", r"\section{Native Vivado PYNQ block design}", "This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.", rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filenar] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 464: `[` This is continuation line 2 of the 8-line logical statement that begins at source line 463. Its exact fragment is ‘[‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", r"\section{Native Vivado PYNQ block design}", "This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.", rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filename}}}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 465: `r"\clearpage",` This is continuation line 3 of the 8-line logical statement that begins at source line 463. Its exact fragment is ‘r"\clearpage",‘. The important operands are r: the value named ’r’ in this block; clearpage: the value named ’clearpage’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", r"\section{Native Vivado PYNQ block design}", "This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.", rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 466: `r"\section{Native Vivado PYNQ block design}",` This is continuation line 4 of the 8-line logical statement that begins at source line 463. Its exact fragment is ‘r"\section{Native Vivado PYNQ block design}",‘. The important operands are r: the value named ’r’ in this block; section: the value named ’section’ in this block; Native: the value named ’native’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘

with ‘ [r"\clearpage", r"\section{Native Vivado PYNQ block design}", "This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.", rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filename}}}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 467: `"This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a red...` This is continuation line 5 of the 8-line logical statement that begins at source line 463. Its exact fragment is ‘"This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.",‘. The important operands are This: the value named ’this’ in this block; page: the value named ’page’ in this block; is: the value named ’is’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", r"\section{Native Vivado PYNQ block design}", "This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.", rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filename}}}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 468: `rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filename}}}",` This is continuation line 6 of the 8-line logical statement that begins at source line 463. Its exact fragment is ‘rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filename}}}",‘. The important operands are rf: the value named ’rf’ in this block; includepdf: the value named ’includepdf’ in this block; pages: the value named ’pages’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", r"\section{Native Vivado PYNQ block design}", "This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.", rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filenar] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 469: `]` This is continuation line 7 of the 8-line logical statement that begins at source line 463. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", r"\section{Native Vivado PYNQ block design}", "This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.", rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filename}}}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 470: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 463. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", r"\section{Native Vivado PYNQ block design}", "This page is exported directly from the validated system.bd block design by Vivado 2026.1. It is not a redrawn or inferred schematic.", rf"\includepdf[pages=-,fitpaper=true,angle=90,pagecommand={{}}]{{{block_design_filename}}}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 471: `for index, record in enumerate(records, start=1):` This iterates index, record over ‘enumerate(records, start=1)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function write_report_tex, which handles write report tex.

Line 472: `lines.extend(` This statement continues through source line 481; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title) rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\textheight,keepaspectratio]{{plots/{record.filename}}}", r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 473: `[` This is continuation line 2 of the 10-line logical statement that begins at source line 472. Its exact fragment is ‘[‘. The expression is a literal or structural operator and has no hidden software state. Read together,

the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title)}}}", rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\textheight,keepaspectratio]{{plots/{record.filename}}}", r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 474: `r"\clearpage",` This is continuation line 3 of the 10-line logical statement that begins at source line 472. Its exact fragment is ‘r"\clearpage",‘. The important operands are r: the value named ’r’ in this block; clearpage: the value named ’clearpage’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title)}}}", rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\tex r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 475: `rf"\section{{{latex_escape(record.title)}}}",` This is continuation line 4 of the 10-line logical statement that begins at source line 472. Its exact fragment is ‘rf"\section{{{latex_escape(record.title)}}}",‘. The important operands are rf: the value named ’rf’ in this block; section: the value named ’section’ in this block; latex_escape: the value named ’latex escape’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title)}}}", rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\tex r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 476: `rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}`
This is continuation line 5 of the 10-line logical statement that begins at source line 472. Its exact fragment is ‘rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}",‘. The important operands are rf: the value named ’rf’ in this block; sloppypar: the value named ’sloppypar’ in this block; small: the value named ’small’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title)}}}", rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\tex r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 477: `r"\begin{center}",` This is continuation line 6 of the 10-line logical statement that begins at source line 472. Its exact fragment is ‘r"\begin{center}",‘. The important operands are r: the value named ’r’ in this block; center: the value named ’center’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title)}}}", rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\tex r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 478: `rf"\includegraphics[width=0.98\textwidth,height=0.78\textheight,keepaspectratio]{{plots/{record.fil`
This is continuation line 7 of the 10-line logical statement that begins at source line 472. Its exact fragment is ‘rf"\includegraphics[width=0.98\textwidth,height=0.78\textheight,keepaspectratio]{{plots/{record.filename}}}",‘. The important operands are rf: the value named ’rf’ in this block; includegraphics: the value named ’includegraphics’ in this block; width: the value named ’width’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title)}}}", rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\ r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 479: `r"\end{center}",` This is continuation line 8 of the 10-line logical statement that begins at source line 472. Its exact fragment is ‘r"\end{center}",‘. The important operands are r: the value named ’r’ in this block; center: the value named ’center’ in this block. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title)}}}", rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\te
r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 480: `]` This is continuation line 9 of the 10-line logical statement that begins at source line 472. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title)}}}", rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\textheight,keepaspectratio]{{plots/{record.filename}}}", r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 481: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 472. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘lines.extend‘ with ‘ [r"\clearpage", rf"\section{{{latex_escape(record.title)}}}", rf"\begin{{sloppypar}}\small\textbf{{Evidence:}} {latex_breakable(record.evidence)}\end{{sloppypar}}", r"\begin{center}", rf"\includegraphics[width=0.98\textwidth,height=0.78\textheight,keepaspectratio]{{plots/{record.filename}}}", r"\end{center}",] ‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 482: `lines.append(r"\end{document}")` This calls ‘lines.append‘ with ‘r"\end{document}"‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 483: `tex_path.write_text("\n".join(lines) + "\n", encoding="utf-8")` This calls ‘tex_path.write_text‘ with ‘"\n".join(lines) + "\n", encoding="utf-8"‘ inside the function write_report_tex, which handles write report tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 484: `return tex_path` This returns ‘tex_path‘ to the caller of the function write_report_tex, which handles write report tex, ending this invocation. The important operands are tex_path: the value named ’tex path’ in this block.

Line 487: `def generate(` This starts function generate. Its indented body performs generate as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 488: `output_dir: Path,` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are output_dir: the value named ’output dir’ in this block; Path: the value named ’path’ in this block.

Line 489: `timing_dir: Path | None,` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are timing_dir: the value named ’timing dir’ in this block; Path: the value named ’path’ in this block; None: the value named ’none’ in this block.

Line 490: `waveform_dir: Path | None = None,` This assigns ‘None,‘ to ‘waveform_dir‘ in host memory. Here waveform_dir means the value named ’waveform dir’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function generate, which handles generate.

Line 491: `saif_power_report: Path | None = None,` This assigns 'None,' to 'saif_power_report' in host memory. Here saif_power_report means the value named 'saif power report' in this block. The important operands are None: the value named 'none' in this block. Context: the function generate, which handles generate.

Line 492: `block_design: Path | None = None,` This assigns 'None,' to 'block_design' in host memory. Here block_design means the value named 'block design' in this block. The important operands are None: the value named 'none' in this block. Context: the function generate, which handles generate.

Line 493: `) -> tuple[Path, Path, Path]:` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are tuple: the value named 'tuple' in this block; Path: the value named 'path' in this block.

Line 494: `root = root_dir()` This assigns 'root_dir()' to 'root ' in host memory. Here root means the value named 'root' in this block. The important operands are root_dir: the value named 'root dir' in this block. Context: the function generate, which handles generate.

Line 495: `timing_dir = timing_dir or root / "outputs/pynq_full"` This assigns 'timing_dir or root / "outputs/pynq_full"' to 'timing_dir ' in host memory. Here timing_dir means the value named 'timing dir' in this block. The important operands are timing_dir: the value named 'timing dir' in this block; or: the value named 'or' in this block; root: the value named 'root' in this block. Context: the function generate, which handles generate.

Line 496: `waveform_dir = waveform_dir or root / "outputs/waveforms_final"` This assigns 'waveform_dir or root / "outputs/waveforms_final"' to 'waveform_dir ' in host memory. Here waveform_dir means the value named 'waveform dir' in this block. The important operands are waveform_dir: the value named 'waveform dir' in this block; or: the value named 'or' in this block; root: the value named 'root' in this block. Context: the function generate, which handles generate.

Line 497: `saif_power_report = saif_power_report or (` This statement continues through source line 499; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'saif_power_report or (root / "outputs/power_activity_full/vivado_reports/routed_saif_power.rpt")' to 'saif_power_report ' in host memory. Here saif_power_report means the value named 'saif power report' in this block. The important operands are saif_power_report: the value named 'saif power report' in this block; or: the value named 'or' in this block; root: the value named 'root' in this block. Context: the function generate, which handles generate.

Line 498: `root / "outputs/power_activity_full/vivado_reports/routed_saif_power.rpt"` This is continuation line 2 of the 3-line logical statement that begins at source line 497. Its exact fragment is 'root / "outputs/power_activity_full/viv The important operands are root: the value named 'root' in this block; outputs: the value named 'outputs' in this block; power_activity_full: the value named 'power activity full' in this block. Read together, the complete operation is: This assigns 'saif_power_report or (root / "outputs/power_activity_full/vivado_reports/routed_saif_power.rpt")' to 'saif_power_report ' in host memory. Here saif_power_report means the value named 'saif power report' in this block. The important operands are saif_power_report: the value named 'saif power report' in this block; or: the value named 'or' in this block; root: the value named 'root' in this block. Context: the function generate, which handles generate.

Line 499: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 497. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'saif_power_report or (root / "outputs/power_activity_full/vivado_reports/routed_saif_power.rpt")' to 'saif_power_report ' in host memory. Here saif_power_report means the value named 'saif power report' in this block. The important operands are saif_power_report: the value named 'saif power report' in this block; or: the value named 'or' in this block; root: the value named 'root' in this block. Context: the function generate, which handles generate.

Line 500: `block_design = block_design or root / "outputs/pynq_full/neuraedge_pynq_block_design.pdf"` This assigns 'block_design or root / "outputs/pynq_full/neuraedge_pynq_block_design.pdf"' to 'block_design ' in host memory. Here block_design means the value named 'block design' in this block. The important operands are

block_design: the value named 'block design' in this block; or: the value named 'or' in this block; root: the value named 'root' in this block. Context: the function generate, which handles generate.

Line 501: `output_dir.mkdir(parents=True, exist_ok=True)` This calls ‘output_dir.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 502: `plot_dir = output_dir / "plots"` This assigns ‘output_dir / "plots"‘ to ‘plot_dir ‘ in host memory. Here plot_dir means the value named 'plot dir' in this block. The important operands are output_dir: the value named 'output dir' in this block; plots: the value named 'plots' in this block. Context: the function generate, which handles generate.

Line 503: `plot_dir.mkdir(parents=True, exist_ok=True)` This calls ‘plot_dir.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 504: `pdf_path = output_dir / "neuraedge_deep_debug_plots.pdf"` This assigns ‘output_dir / "neuraedge_deep_debu
to ‘pdf_path ‘ in host memory. Here pdf_path means the value named 'pdf path' in this block. The important operands are output_dir: the value named 'output dir' in this block; neuraedge_deep_debug_plots: the value named 'neuraedge deep debug plots' in this block; pdf: the value named 'pdf' in this block. Context: the function generate, which handles generate.

Line 505: `records: list[PlotRecord] = []` This assigns ‘[]‘ to ‘records‘ in host memory. Here records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 506: `index = 1` This assigns ‘1‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 508: `reference = load_json(root / "outputs/spikingformer_reference/spikingformer_evaluation.json")`
This assigns ‘load_json(root / "outputs/spikingformer_reference/spikingformer_evaluation.json")‘ to ‘reference ‘ in host memory. Here reference means the value named 'reference' in this block. The important operands are load_json: the value named 'load json' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function generate, which handles generate.

Line 509: `quantized = load_json(root / "outputs/spikingformer_hw_q10_tensor_exact/spikingformer_evaluation.js`
This assigns ‘load_json(root / "outputs/spikingformer_hw_q10_tensor_exact/spikingformer_evaluation.json")‘ to ‘quantized ‘ in host memory. Here quantized means the value named 'quantized' in this block. The important operands are load_json: the value named 'load json' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function generate, which handles generate.

Line 510: `execution = load_json(root / "outputs/spikingformer_reference/spikingformer_execution_manifest.json`
This assigns ‘load_json(root / "outputs/spikingformer_reference/spikingformer_execution_manifest.json")‘ to ‘execution ‘ in host memory. Here execution means the value named 'execution' in this block. The important operands are load_json: the value named 'load json' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function generate, which handles generate.

Line 511: `parameters = load_json(root / "outputs/spikingformer_hardware_parameters/spikingformer_neuraedge_we`
This assigns ‘load_json(root / "outputs/spikingformer_hardware_parameters/spikingformer_neuraedge_weights.json")‘ to ‘parameters ‘ in host memory. Here parameters means the value named 'parameters' in this block. The important operands are load_json: the value named 'load json' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function generate, which handles generate.

Line 512: `compiler_benchmark = load_json(root / "outputs/benchmarks/compiler_block3_memory.json")` This assigns ‘load_json(root / "outputs/benchmarks/compiler_block3_memory.json")‘ to ‘compiler_benchmark ‘ in host memory. Here compiler_benchmark means the value named ’compiler benchmark’ in this block. The important operands are load_json: the value named ’load json’ in this block; root: the value named ’root’ in this block; outputs: the value named ’outputs’ in this block. Context: the function generate, which handles generate.

Line 513: `training_coverage_path = root / "outputs/training/spikingformer_training_coverage.json"` This assigns ‘root / "outputs/training/spikingformer_training_coverage.json"‘ to ‘training_coverage_path ‘ in host memory. Here training_coverage_path means the value named ’training coverage path’ in this block. The important operands are root: the value named ’root’ in this block; outputs: the value named ’outputs’ in this block; training: the value named ’training’ in this block. Context: the function generate, which handles generate.

Line 514: `training_coverage = (` This statement continues through source line 516; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(load_json(training_coverage_path) if training_coverage_path.exists() else None)‘ to ‘training_coverage ‘ in host memory. Here training_coverage means the value named ’training coverage’ in this block. The important operands are load_json: the value named ’load json’ in this block; training_coverage_path: the value named ’training coverage path’ in this block; exists: the value named ’exists’ in this block. Context: the function generate, which handles generate.

Line 515: `load_json(training_coverage_path) if training_coverage_path.exists() else None` This is continuation line 2 of the 3-line logical statement that begins at source line 514. Its exact fragment is ‘load_json(training_coverage_path if training_coverage_path.exists() else None‘. The important operands are load_json: the value named ’load json’ in this block; training_coverage_path: the value named ’training coverage path’ in this block; exists: the value named ’exists’ in this block. Read together, the complete operation is: This assigns ‘(load_json(training_coverage_path) if training_coverage_path.exists() else None)‘ to ‘training_coverage ‘ in host memory. Here training_coverage means the value named ’training coverage’ in this block. The important operands are load_json: the value named ’load json’ in this block; training_coverage_path: the value named ’training coverage path’ in this block; exists: the value named ’exists’ in this block. Context: the function generate, which handles generate.

Line 516: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 514. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(load_json(training_coverage_path) if training_coverage_path.exists() else None)‘ to ‘training_coverage ‘ in host memory. Here training_coverage means the value named ’training coverage’ in this block. The important operands are load_json: the value named ’load json’ in this block; training_coverage_path: the value named ’training coverage path’ in this block; exists: the value named ’exists’ in this block. Context: the function generate, which handles generate.

Line 517: `forward_validation_path = root / "outputs/regression/real_full_graph_forward_validation.json"` This assigns ‘root / "outputs/regression/real_full_graph_forward_validation.json"‘ to ‘forward_validation_path ‘ in host memory. Here forward_validation_path means the producer’s statement that its current payload is meaningful. The important operands are root: the value named ’root’ in this block; outputs: the value named ’outputs’ in this block; regression: the value named ’regression’ in this block. Context: the function generate, which handles generate.

Line 518: `forward_validation = (` This statement continues through source line 520; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(load_json(forward_validation_path) if forward_validation_path.exists() else None)‘ to ‘forward_validation ‘ in host memory. Here forward_validation means the producer’s statement that its current payload is meaningful. The important operands are load_json: the value named ’load json’ in this block; forward_validation_path: the producer’s statement that its current payload is meaningful; exists: the value named ’exists’ in this block. Context: the function generate, which handles generate.

Line 519: `load_json(forward_validation_path) if forward_validation_path.exists() else None` This is continuation line 2 of the 3-line logical statement that begins at source line 518. Its exact fragment is ‘load_json(forward_validation_pat if forward_validation_path.exists() else None‘. The important operands are load_json: the value named ’load json’ in this block; forward_validation_path: the producer’s statement that its current payload is meaningful; exists: the value named ’exists’ in this block. Read together, the complete operation is: This assigns ‘(load_json(forward_validation_path) if forward_validation_path.exists() else None)‘ to ‘forward_validation ‘ in host memory. Here forward_validation means

the producer's statement that its current payload is meaningful. The important operands are load_json: the value named 'load json' in this block; forward_validation_path: the producer's statement that its current payload is meaningful; exists: the value named 'exists' in this block. Context: the function generate, which handles generate.

Line 520: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 518. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(load_json(forward_validation_path) if forward_validation_path.exists() else None)' to 'forward_validation ' in host memory. Here forward_validation means the producer's statement that its current payload is meaningful. The important operands are load_json: the value named 'load json' in this block; forward_validation_path: the producer's statement that its current payload is meaningful; exists: the value named 'exists' in this block. Context: the function generate, which handles generate.

Line 521: `shard_manifest_path = root / "outputs/cifar10_shards/cifar10_shards_manifest.json"` This assigns 'root / "outputs/cifar10_shards/cifar10_shards_manifest.json"' to 'shard_manifest_path ' in host memory. Here shard_manifest_path means the value named 'shard manifest path' in this block. The important operands are root: the value named 'root' in this block; outputs: the value named 'outputs' in this block; cifar10_shards: the value named 'cifar10 shards' in this block. Context: the function generate, which handles generate.

Line 522: `shard_manifest = load_json(shard_manifest_path) if shard_manifest_path.exists() else None` This assigns 'load_json(shard_manifest_path) if shard_manifest_path.exists() else None' to 'shard_manifest ' in host memory. Here shard_manifest means the value named 'shard manifest' in this block. The important operands are load_json: the value named 'load json' in this block; shard_manifest_path: the value named 'shard manifest path' in this block; exists: the value named 'exists' in this block. Context: the function generate, which handles generate.

Line 523: `float_npz = np.load(root / "outputs/spikingformer_reference/spikingformer_predictions.npz")` This assigns 'np.load(root / "outputs/spikingformer_reference/spikingformer_predictions.npz")' to 'float_npz ' in host memory. Here float_npz means the value named 'float npz' in this block. The important operands are load: the value named 'load' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function generate, which handles generate.

Line 524: `quant_npz = np.load(root / "outputs/spikingformer_hw_q10_tensor_exact/spikingformer_predictions.npz` This assigns 'np.load(root / "outputs/spikingformer_hw_q10_tensor_exact/spikingformer_predictions.npz")' to 'quant_npz ' in host memory. Here quant_npz means the value named 'quant npz' in this block. The important operands are load: the value named 'load' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function generate, which handles generate.

Line 525: `summary_path = timing_dir / "full_implementation_summary.txt"` This assigns 'timing_dir / "full_implementa to 'summary_path ' in host memory. Here summary_path means the value named 'summary path' in this block. The important operands are timing_dir: the value named 'timing dir' in this block; full_implementation_summary: the value named 'full implementation summary' in this block; txt: the value named 'txt' in this block. Context: the function generate, which handles generate.

Line 526: `if not summary_path.exists():` This executes the following indented host-code branch only when 'not summary_path.exists()' is true. The important operands are not: the value named 'not' in this block; summary_path: the value named 'summary path' in this block; exists: the value named 'exists' in this block. Context: the function generate, which handles generate.

Line 527: `summary_path = timing_dir / "implementation_summary.txt"` This assigns 'timing_dir / "implementation_summary.txt"' to 'summary_path ' in host memory. Here summary_path means the value named 'summary path' in this block. The important operands are timing_dir: the value named 'timing dir' in this block; implementation_summary: the value named 'implementation summary' in this block; txt: the value named 'txt' in this block. Context: the function generate, which handles generate.

Line 528: `implementation = parse_implementation_summary(summary_path)` This assigns ‘parse_implementation_summary to ‘implementation ‘ in host memory. Here implementation means the value named ’implementation’ in this block. The important operands are parse_implementation_summary: the value named ’parse implementation summary’ in this block; summary_path: the value named ’summary path’ in this block. Context: the function generate, which handles generate.

Line 529: `vectorless_power_path = timing_dir / "full_vectorless_power.rpt"` This assigns ‘timing_dir / "full_vectorless_power.rpt"‘ to ‘vectorless_power_path ‘ in host memory. Here vectorless_power_path means the value named ’vectorless power path’ in this block. The important operands are timing_dir: the value named ’timing dir’ in this block; full_vectorless_power: the value named ’full vectorless power’ in this block; rpt: the value named ’rpt’ in this block. Context: the function generate, which handles generate.

Line 530: `if not vectorless_power_path.exists():` This executes the following indented host-code branch only when ‘not vectorless_power_path.exists()‘ is true. The important operands are not: the value named ’not’ in this block; vectorless_power_path: the value named ’vectorless power path’ in this block; exists: the value named ’exists’ in this block. Context: the function generate, which handles generate.

Line 531: `vectorless_power_path = timing_dir / "routed_vectorless_power.rpt"` This assigns ‘timing_dir / "routed_vectorless_power.rpt"‘ to ‘vectorless_power_path ‘ in host memory. Here vectorless_power_path means the value named ’vectorless power path’ in this block. The important operands are timing_dir: the value named ’timing dir’ in this block; routed_vectorless_power: the value named ’routed vectorless power’ in this block; rpt: the value named ’rpt’ in this block. Context: the function generate, which handles generate.

Line 532: `vectorless_power = parse_power_report(vectorless_power_path)` This assigns ‘parse_power_report(vectorless_ to ‘vectorless_power ‘ in host memory. Here vectorless_power means the value named ’vectorless power’ in this block. The important operands are parse_power_report: the value named ’parse power report’ in this block; vectorless_power_path: the value named ’vectorless power path’ in this block. Context: the function generate, which handles generate.

Line 533: `saif_power = parse_power_report(saif_power_report)` This assigns ‘parse_power_report(saif_power_report)‘ to ‘saif_power ‘ in host memory. Here saif_power means the value named ’saif power’ in this block. The important operands are parse_power_report: the value named ’parse power report’ in this block; saif_power_report: the value named ’saif power report’ in this block. Context: the function generate, which handles generate.

Line 534: `block_design_filename: str | None = None` This assigns ‘None‘ to ‘block_design_filename‘ in host memory. Here block_design_filename means the value named ’block design filename’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function generate, which handles generate.

Line 535: `if block_design.exists():` This executes the following indented host-code branch only when ‘block_design.exists()‘ is true. The important operands are block_design: the value named ’block design’ in this block; exists: the value named ’exists’ in this block. Context: the function generate, which handles generate.

Line 536: `copied_block_design = output_dir / "pynq-block-design.pdf"` This assigns ‘output_dir / "pynq-block-design.pdf"‘ to ‘copied_block_design ‘ in host memory. Here copied_block_design means the value named ’copied block design’ in this block. The important operands are output_dir: the value named ’output dir’ in this block; pynq: the value named ’pynq’ in this block; block: the value named ’block’ in this block. Context: the function generate, which handles generate.

Line 537: `shutil.copy2(block_design, copied_block_design)` This calls ‘shutil.copy2‘ with ‘block_design, copied_block_de inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 538: `block_design_filename = copied_block_design.name` This assigns ‘copied_block_design.name‘ to ‘block_design_ ‘ in host memory. Here block_design_filename means the value named ’block design filename’ in this block. The important operands are copied_block_design: the value named ’copied block design’ in this block; name: the value named ’name’ in this block. Context: the function generate, which handles generate.

Line 540: `with PdfPages(pdf_path) as pdf:` This opens a managed resource scope for 'PdfPages(pdf_path) as pdf' so files, tracing, or temporary state are finalized even if the function generate, which handles generate raises an error.

Line 541: `summary_lines = [` This statement continues through source line 550; the explanation uses the complete expression rather than only the first visual fragment. Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named 'summary lines' in this block; f: the value named 'f' in this block; Dataset: the value named 'dataset' in this block.

Line 542: `f"Dataset: {reference['dataset']} ({reference['samples']:,} samples)",` This is continuation line 2 of the 10-line logical statement that begins at source line 541. Its exact fragment is 'f"Dataset: {reference['dataset']} ({reference['samples']:,} samples)",'. The important operands are f: the value named 'f' in this block; Dataset: the value named 'dataset' in this block; reference: the value named 'reference' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named 'summary lines' in this block; f: the value named 'f' in this block; Dataset: the value named 'dataset' in this block.

Line 543: `f"Checkpoint: {reference['correct']:,}/{reference['samples']:,} = {reference['top1_accuracy_percen`
This is continuation line 3 of the 10-line logical statement that begins at source line 541. Its exact fragment is 'f"Checkpoint: {reference['correct']:,}/{reference['samples']:,} = {reference['top1_accuracy_percent']:.2f}%",'. The important operands are f: the value named 'f' in this block; Checkpoint: the value named 'checkpoint' in this block; reference: the value named 'reference' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named 'summary lines' in this block; f: the value named 'f' in this block; Dataset: the value named 'dataset' in this block.

Line 544: `f"Exported integer arithmetic: {quantized['correct']:,}/{quantized['samples']:,} = {quantized['top`
This is continuation line 4 of the 10-line logical statement that begins at source line 541. Its exact fragment is 'f"Exported integer arithmetic: {quantized['correct']:,}/{quantized['samples']:,} = {quantized['top1_accuracy_percent']:.2f}%",'. The important operands are f: the value named 'f' in this block; Exported: the value named 'exported' in this block; integer: the value named 'integer' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named 'summary lines' in this block; f: the value named 'f' in this block; Dataset: the value named 'dataset' in this block.

Line 545: `f"Prediction disagreements: {int(np.count_nonzero(float_npz['predictions'] != quant_npz['predictio`
This is continuation line 5 of the 10-line logical statement that begins at source line 541. Its exact fragment is 'f"Prediction disagreements: {int(np.count_nonzero(float_npz['predictions'] != quant_npz['predictions'])):,}",'. The important operands are f: the value named 'f' in this block; Prediction: the class index with the greatest classifier score; disagreements: the value named 'disagreements' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named 'summary lines' in this block; f: the value named 'f' in this block; Dataset: the value named 'dataset' in this block.

Line 546: `f"Model MAC equivalents: {execution['per_image_totals']['mac_equivalents_after_exact_temporal_hois`
This is continuation line 6 of the 10-line logical statement that begins at source line 541. Its exact fragment is 'f"Model MAC equivalents: {execution['per_image_totals']['mac_equivalents_after_exact_temporal_hoist']:,}",'. The important operands are f: the value named 'f' in this block; Model: the value named 'model' in this block; MAC: the value named 'mac' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named 'summary lines' in this block; f: the value named 'f' in this block; Dataset: the value named 'dataset' in this block.

Line 547: `f"Tile commands: {execution['per_image_totals']['tensor_commands']:,}",` This is continuation line 7 of the 10-line logical statement that begins at source line 541. Its exact fragment is ‘f"Tile commands: {execution['per_image_totals']['tensor_commands']:,}",‘. The important operands are f: the value named ’f’ in this block; Tile: the value named ’tile’ in this block; commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named ’summary lines’ in this block; f: the value named ’f’ in this block; Dataset: the value named ’dataset’ in this block.

Line 548: `f"Hardware parameter archive: {parameters['binary_bytes']:,} bytes, SHA-256 {parameters['binary_sh` This is continuation line 8 of the 10-line logical statement that begins at source line 541. Its exact fragment is ‘f"Hardware parameter archive: {parameters['binary_bytes']:,} bytes, SHA-256 {parameters['binary_sha256']}",‘. The important operands are f: the value named ’f’ in this block; Hardware: the value named ’hardware’ in this block; parameter: the value named ’parameter’ in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named ’summary lines’ in this block; f: the value named ’f’ in this block; Dataset: the value named ’dataset’ in this block.

Line 549: `"Power pages use Vivado report data only; no physical-board value is inferred.",` This is continuation line 9 of the 10-line logical statement that begins at source line 541. Its exact fragment is ‘"Power pages use Vivado report data only; no physical-board value is inferred.",‘. The important operands are Power: the value named ’power’ in this block; pages: the value named ’pages’ in this block; use: the value named ’use’ in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named ’summary lines’ in this block; f: the value named ’f’ in this block; Dataset: the value named ’dataset’ in this block.

Line 550: `]` This is continuation line 10 of the 10-line logical statement that begins at source line 541. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are summary_lines: the value named ’summary lines’ in this block; f: the value named ’f’ in this block; Dataset: the value named ’dataset’ in this block.

Line 551: `if implementation:` This executes the following indented host-code branch only when ‘implementation‘ is true. The important operands are implementation: the value named ’implementation’ in this block. Context: the function generate, which handles generate.

Line 552: `summary_lines.extend(` This statement continues through source line 557; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘summary_lines.extend‘ with ‘ [f"Full PYNQ routed setup/hold slack: {implementation.get('setup_wns_ns', float('nan')):+.3f} / {implementation.get('hold_whs_ns', float('nan')):+.3 ns", f"Clock domains: accelerator {implementation.get('accelerator_clock_mhz', float('nan')):.0f} MHz, DMA {implementation.get('dma_clock_mhz', float('nan')):.0f} MHz, control {implementation.get('control_clock_mhz', float('nan')):.0f} MHz",] ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 553: `[` This is continuation line 2 of the 6-line logical statement that begins at source line 552. Its exact fragment is ‘[‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘summary_lines.extend‘ with ‘ [f"Full PYNQ routed setup/hold slack: {implementation.get('setup_wns_ns', float('nan')):+.3f} / {implementation.get('hold_whs_ns', float('nan')):+.3f} ns", f"Clock domains: accelerator {implementation.get('accelerator_clock_mhz', float('nan')):.0f} MHz, DMA {implementation.get('dma_clock_mhz', float('nan')):.0f} MHz, control {implementation.get('control_clock_mhz', float('nan')):.0f} MHz",] ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 554: `f"Full PYNQ routed setup/hold slack: {implementation.get('setup_wns_ns', float('nan')):+.3f} / {implementat...` This is continuation line 3 of the 6-line logical statement that begins at source line 552. Its exact fragment is 'f"Full PYNQ routed setup/hold slack: {implementation.get('setup_wns_ns', float('nan')):+.3f} / {implementation.get('hold_whs_ns', float('nan')):+.3f} ns",'. The important operands are f: the value named 'f' in this block; Full: the value named 'full' in this block; PYNQ: the value named 'pynq' in this block. Read together, the complete operation is: This calls 'summary_lines.extend' with ' [f"Full PYNQ routed setup/hold slack: {implementation.get('setup_wns_ns', float('nan')):+.3f} / {implementation.get('hold_whs_ns', float('nan')):+.3f} ns", f"Clock domains: accelerator {implementation.get('accelerator_clock_mhz', float('nan')):.0f} MHz, DMA {implementation.get('dma_clock_m
float('nan')):.0f} MHz, control {implementation.get('control_clock_mhz', float('nan')):.0f} MHz",] ' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 555: `f"Clock domains: accelerator {implementation.get('accelerator_clock_mhz', float('nan')):.0f} MHz, DMA {impl...` This is continuation line 4 of the 6-line logical statement that begins at source line 552. Its exact fragment is 'f"Clock domains: accelerator {implementation.get('accelerator_clock_mhz', float('nan')):.0f} MHz, DMA {implementation.get('dma_clock_mhz', float('nan')):.0f} MHz, control {implementation.get('control_clock_mhz', float('nan')):.0f} MHz",'. The important operands are f: the value named 'f' in this block; Clock: the value named 'clock' in this block; domains: the value named 'domains' in this block. Read together, the complete operation is: This calls 'summary_lines.extend' with ' [f"Full PYNQ routed setup/hold slack: {implementation.get('setup_wns_ns', float('nan')):+.3f} / {implementation.get('hold_whs_ns', float('nan')):+.3f} ns", f"Clock domains: accelerator {implementation.get('accelerator_clock_mhz', float('nan')):.0f} MHz, DMA {implementation.get('dma_clock_mhz', float('nan')):.0f} MHz, control {implementation.get('control_clock_mhz', float('nan')):.0f} MHz",] ' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 556: `]` This is continuation line 5 of the 6-line logical statement that begins at source line 552. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'summary_lines.extend' with ' [f"Full PYNQ routed setup/hold slack: {implementation.get('setup_wns_ns', float('nan')):+.3f} / {implementation.get('hold_whs_ns', float('nan')):+.3f} ns", f"Clock domains: accelerator {implementation.get('accelerator_clock_mhz', float('nan')):.0f} MHz, DMA {implementation.get('dma_clock_mhz', float('nan')):.0f} MHz, control {implementation.get('control_clock_mhz', float('nan')):.0f} MHz",] ' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 557: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 552. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'summary_lines.extend' with ' [f"Full PYNQ routed setup/hold slack: {implementation.get('setup_wns_ns', float('nan')):+.3f} / {implementation.get('hold_whs_ns', float('nan')):+.3f} ns", f"Clock domains: accelerator {implementation.get('accelerator_clock_mhz', float('nan')):.0f} MHz, DMA {implementation.get('dma_clock_mhz', float('nan')):.0f} MHz, control {implementation.get('control_clock_mhz', float('nan')):.0f} MHz",] ' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 558: `if saif_power:` This executes the following indented host-code branch only when 'saif_power' is true. The important operands are saif_power: the value named 'saif power' in this block. Context: the function generate, which handles generate.

Line 559: `summary_lines.append(` This statement continues through source line 561; the explanation uses the complete expression rather than only the first visual fragment. This calls 'summary_lines.append' with ' f"Full-system SAIF estimate: {saif_power['total']:.3f} W total, {saif_power['dynamic']:.3f} W dynamic, confidence {saif_power['confidence']}, nets matched {saif_power['matched']}" ' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 560: `f"Full-system SAIF estimate: {saif_power['total']:.3f} W total, {saif_power['dynamic']:.3f} W dynamic, conf...` This is continuation line 2 of the 3-line logical statement that begins at source line 559. Its

exact fragment is ‘f"Full-system SAIF estimate: {saif_power['total']:.3f} W total, {saif_power['dynamic']:.3f} W dynamic, confidence {saif_power['confidence']}, nets matched {saif_power['matched']}"‘. The important operands are f: the value named ’f’ in this block; Full: the value named ’full’ in this block; system: the value named ’system’ in this block. Read together, the complete operation is: This calls ‘summary_lines.append‘ with ‘ f"Full-system SAIF estimate: {saif_power['total']:.3f} W total, {saif_power['dynamic']:.3f} W dynamic, confidence {saif_power['confidence']}, nets matched {saif_power['matched']}" ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 561: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 559. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘summary_lines.append‘ with ‘ f"Full-system SAIF estimate: {saif_power['total']:.3f} W total, {saif_power['dynamic']:.3f} W dynamic, confidence {saif_power['confidence']}, nets matched {saif_power['matched']}" ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 562: `figure = text_page("Measured Evidence Summary", summary_lines)` This assigns ‘text_page("Measured Evidence Summary", summary_lines)‘ to ‘figure ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are text_page: the value named ’text page’ in this block; Measured: the value named ’measured’ in this block; Evidence: the value named ’evidence’ in this block. Context: the function generate, which handles generate.

Line 563: `index = save_figure(figure, plot_dir, pdf, records, index, "evidence_summary", "Measured evidence summary",` This statement continues through source line 564; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "evidence_summary", "Measured evidence summary", "evaluation JSON, execution manifest, parameter manifest")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 564: `"evaluation JSON, execution manifest, parameter manifest")` This is continuation line 2 of the 2-line logical statement that begins at source line 563. Its exact fragment is ‘"evaluation JSON, execution manifest, parameter manifest")‘. The important operands are evaluation: the value named ’evaluation’ in this block; JSON: the value named ’json’ in this block; execution: the value named ’execution’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "evidence_summary", "Measured evidence summary", "evaluation JSON, execution manifest, parameter manifest")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 566: `if forward_validation:` This executes the following indented host-code branch only when ‘forward_validation‘ is true. The important operands are forward_validation: the producer’s statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 567: `validation_lines = [` This statement continues through source line 576; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}", f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['maximum_absolute_logit_delta_from_archived_10000_i f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25 f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]‘ to ‘validation_lines ‘ in host memory. Here validation_lines means the producer’s statement that its current payload is meaningful. The important operands are f: the value named ’f’ in this block; Result: the value named ’result’ in this block; forward_validation: the producer’s statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 568: `f"Result: {forward_validation['result']}",` This is continuation line 2 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'f"Result: {forward_validation['result']}",'. The important operands are f: the value named 'f' in this block; Result: the value named 'result' in this block; forward_validation: the producer's statement that its current payload is meaningful. Read together, the complete operation is: This assigns '[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}", f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['maximum f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25 f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]' to 'validation_lines ' in host memory. Here validation_lines means the producer's statement that its current payload is meaningful. The important operands are f: the value named 'f' in this block; Result: the value named 'result' in this block; forward_validation: the producer's statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 569: `f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}",` This is continuation line 3 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}",'. The important operands are f: the value named 'f' in this block; Official: the value named 'official' in this block; CIFAR: the value named 'cifar' in this block. Read together, the complete operation is: This assigns '[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}", f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['max f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25 f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]' to 'validation_lines ' in host memory. Here validation_lines means the producer's statement that its current payload is meaningful. The important operands are f: the value named 'f' in this block; Result: the value named 'result' in this block; forward_validation: the producer's statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 570: `f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}` This is continuation line 4 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}",'. The important operands are f: the value named 'f' in this block; Label: the value named 'label' in this block; predicted: the value named 'predicted' in this block. Read together, the complete operation is: This assigns '[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}", f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['maximum_absolute_logit_delta_from_archived_10000_i f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25 f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]' to 'validation_lines ' in host memory. Here validation_lines means the producer's statement that its current payload is meaningful. The important operands are f: the value named 'f' in this block; Result: the value named 'result' in this block; forward_validation: the producer's statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 571: `f"Complete graph commands: {forward_validation['command_count']:,}",` This is continuation line 5 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'f"Complete graph commands: {forward_validation['command_count']:,}",'. The important operands are f: the value named 'f' in this block; Complete: the value named 'complete' in this block; graph: the value named 'graph' in this block. Read together, the complete operation is: This assigns '[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}", f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['max f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25 f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]' to 'validation_lines ' in host memory. Here validation_lines means the producer's statement that its current payload is meaningful. The important operands are f: the value named 'f' in this block; Result: the value named 'result' in this block; forward_validation: the producer's statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 572: `f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output`
This is continuation line 6 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}",'. The important operands are f: the value named 'f' in this block; Input: the value named 'input' in this block; output: the value named 'output' in this block. Read together, the complete operation is: This assigns '[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}", f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['maximum
f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25
f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]' to 'validation_lines ' in host memory. Here validation_lines means the producer's statement that its current payload is meaningful. The important operands are f: the value named 'f' in this block; Result: the value named 'result' in this block; forward_validation: the producer's statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 573: `f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['maximum_absolute`
This is continuation line 7 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['maximum_absolute_logit_delta_from_archived_10000_i
The important operands are f: the value named 'f' in this block; Maximum: the value named 'maximum' in this block; logit: the value named 'logit' in this block. Read together, the complete operation is: This assigns '[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}", f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['maximum
f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25
f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]' to 'validation_lines ' in host memory. Here validation_lines means the producer's statement that its current payload is meaningful. The important operands are f: the value named 'f' in this block; Result: the value named 'result' in this block; forward_validation: the producer's statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 574: `f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validatio`
This is continuation line 8 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha256']}",'. The important operands are f: the value named 'f' in this block; Parameter: the value named 'parameter' in this block; bytes: the value named 'bytes' in this block. Read together, the complete operation is: This assigns '[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_class']}", f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['input_beats']:,} / {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['maximum
f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25
f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]' to 'validation_lines ' in host memory. Here validation_lines means the producer's statement that its current payload is meaningful. The important operands are f: the value named 'f' in this block; Result: the value named 'result' in this block; forward_validation: the producer's statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 575: `f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",` This is continuation line 9 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",'. The important operands are f: the value named 'f' in this block; Host: the value named 'host' in this block; trace: the value named 'trace' in this block. Read together, the complete operation is: This assigns '[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_clas
f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['inpu
/ {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['max
f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25
f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]' to 'validation_lines ' in host memory. Here validation_lines means the producer's statement that its current payload is meaningful. The important operands are f: the value named 'f' in this block; Result: the value named 'result' in this block; forward_validation: the producer's

statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 576: `]` This is continuation line 10 of the 10-line logical statement that begins at source line 567. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘[f"Result: {forward_validation['result']}", f"Official CIFAR-10 fixture: {forward_validation['dataset_sample']}", f"Label / predicted class: {forward_validation['label']} / {forward_validation['predicted_clas f"Complete graph commands: {forward_validation['command_count']:,}", f"Input / output stream beats: {forward_validation['inpu / {forward_validation['output_beats']:,}", f"Maximum logit delta from archived 10,000-image evaluation: {forward_validation['max f"Parameter bytes / SHA-256: {forward_validation['parameter_binary_bytes']:,} / {forward_validation['parameter_binary_sha25 f"Host trace elapsed time: {forward_validation['elapsed_seconds']:.3f} s",]‘ to ‘validation_lines ‘ in host memory. Here validation_lines means the producer’s statement that its current payload is meaningful. The important operands are f: the value named ’f’ in this block; Result: the value named ’result’ in this block; forward_validation: the producer’s statement that its current payload is meaningful. Context: the function generate, which handles generate.

Line 577: `figure = text_page("Complete Exported-Graph Trace Validation", validation_lines)` This assigns ‘text_page("Complete Exported-Graph Trace Validation", validation_lines)‘ to ‘figure ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are text_page: the value named ’text page’ in this block; Complete: the value named ’complete’ in this block; Exported: the value named ’exported’ in this block. Context: the function generate, which handles generate.

Line 578: `index = save_figure(` This statement continues through source line 581; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "full_graph_validation", "Complete exported-graph trace validation", str(forward_validation_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 579: `figure, plot_dir, pdf, records, index, "full_graph_validation",` This is continuation line 2 of the 4-line logical statement that begins at source line 578. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "full_graph_validation",‘. The important operands are figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "full_graph_validation", "Complete exported-graph trace validation", str(forward_validation_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 580: `"Complete exported-graph trace validation", str(forward_validation_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 578. Its exact fragment is ‘"Complete exported-graph trace validation", str(forward_validation_path),‘. The important operands are Complete: the value named ’complete’ in this block; exported: the value named ’exported’ in this block; graph: the value named ’graph’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "full_graph_validation", "Complete exported-graph trace validation", str(forward_validation_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 581: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 578. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "full_graph_validation", "Complete exported-graph trace validation", str(forward_validation_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 583: `artifact_lines: list[str] = []` This assigns '[]' to 'artifact_lines' in host memory. Here artifact_lines means the value named 'artifact lines' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 584: `for artifact_name in (` This line opens or closes a multi-line Python expression inside the function generate, which handles generate; the neighboring lines supply its arguments or elements.

Line 585: `"neuraedge_pynq_full.bit", "neuraedge_pynq_full.hwh",` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are neuraedge_pynq_full: the value named 'neuraedge pynq full' in this block; bit: the value named 'bit' in this block; hwh: the value named 'hwh' in this block.

Line 586: `"neuraedge_pynq_full.xsa", "neuraedge_pynq_full_routed.dcp",` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are neuraedge_pynq_full: the value named 'neuraedge pynq full' in this block; xsa: the value named 'xsa' in this block; neuraedge_pynq_full_routed: the value named 'neuraedge pynq full routed' in this block.

Line 587: `):` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The expression is a literal or structural operator and has no hidden software state.

Line 588: `artifact_path = timing_dir / artifact_name` This assigns 'timing_dir / artifact_name' to 'artifact_path ' in host memory. Here artifact_path means the value named 'artifact path' in this block. The important operands are timing_dir: the value named 'timing dir' in this block; artifact_name: the value named 'artifact name' in this block. Context: the function generate, which handles generate.

Line 589: `if artifact_path.exists():` This executes the following indented host-code branch only when 'artifact_path.exists()' is true. The important operands are artifact_path: the value named 'artifact path' in this block; exists: the value named 'exists' in this block. Context: the function generate, which handles generate.

Line 590: `size, digest = file_evidence(artifact_path)` This assigns 'file_evidence(artifact_path)' to 'size, digest ' in host memory. Here size means the value named 'size' in this block. The important operands are file_evidence: the value named 'file evidence' in this block; artifact_path: the value named 'artifact path' in this block. Context: the function generate, which handles generate.

Line 591: `artifact_lines.append(f"{artifact_name}: {size:,} bytes")` This calls 'artifact_lines.append' with 'f"{artifact_name}: {size:,} bytes"' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 592: `artifact_lines.append(f" SHA-256 {digest}")` This calls 'artifact_lines.append' with 'f" SHA-256 {digest}"' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 593: `if artifact_lines:` This executes the following indented host-code branch only when 'artifact_lines' is true. The important operands are artifact_lines: the value named 'artifact lines' in this block. Context: the function generate, which handles generate.

Line 594: `figure = text_page("Final PYNQ Deliverable Integrity", artifact_lines)` This assigns 'text_page("Final PYNQ Deliverable Integrity", artifact_lines)' to 'figure ' in host memory. Here figure means the value named 'figure' in this block. The important operands are text_page: the value named 'text page' in this block; Final: the value named 'final' in this block; PYNQ: the value named 'pynq' in this block. Context: the function generate, which handles generate.

Line 595: `index = save_figure(` This statement continues through source line 598; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "artifact_integrity", "Final PYNQ artifact integrity", str(timing_dir),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 596: `figure, plot_dir, pdf, records, index, "artifact_integrity",` This is continuation line 2 of the 4-line logical statement that begins at source line 595. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "artifact_integrity",‘. The important operands are figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "artifact_integrity", "Final PYNQ artifact integrity", str(timing_dir),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 597: `"Final PYNQ artifact integrity", str(timing_dir),` This is continuation line 3 of the 4-line logical statement that begins at source line 595. Its exact fragment is ‘"Final PYNQ artifact integrity", str(timing_dir),‘. The important operands are Final: the value named ’final’ in this block; PYNQ: the value named ’pynq’ in this block; artifact: the value named ’artifact’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "artifact_integrity", "Final PYNQ artifact integrity", str(timing_dir),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 598: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 595. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "artifact_integrity", "Final PYNQ artifact integrity", str(timing_dir),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 600: `float_cm = np.asarray(reference["confusion_matrix_rows_true_columns_predicted"], dtype=np.int64)` This assigns ‘np.asarray(reference["confusion_matrix_rows_true_columns_predicted"], dtype=np.int64)‘ to ‘float_cm ‘ in host memory. Here float_cm means the value named ’float cm’ in this block. The important operands are asarray: the value named ’asarray’ in this block; reference: the value named ’reference’ in this block; confusion_matrix_rows_true_columns_pre one output-row position processed by the tensor tile. Context: the function generate, which handles generate.

Line 601: `quant_cm = np.asarray(quantized["confusion_matrix_rows_true_columns_predicted"], dtype=np.int64)` This assigns ‘np.asarray(quantized["confusion_matrix_rows_true_columns_predicted"], dtype=np.int64)‘ to ‘quant_cm ‘ in host memory. Here quant_cm means the value named ’quant cm’ in this block. The important operands are asarray: the value named ’asarray’ in this block; quantized: the value named ’quantized’ in this block; confusion_matrix_rows_true_columns one output-row position processed by the tensor tile. Context: the function generate, which handles generate.

Line 602: `for slug, title, matrix, signed in (` This line opens or closes a multi-line Python expression inside the function generate, which handles generate; the neighboring lines supply its arguments or elements.

Line 603: `("float_confusion", "Checkpoint confusion matrix: 10,000 images", float_cm, False),` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are float_confusion: the value named ’float confusion’ in this block; Checkpoint: the value named ’checkpoint’ in this block; confusion: the value named ’confusion’ in this block.

Line 604: `("quant_confusion", "Exported integer confusion matrix:  10,000 images", quant_cm, False),` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are quant_confusion: the value named 'quant confusion' in this block; Exported: the value named 'exported' in this block; integer: the value named 'integer' in this block.

Line 605: `("confusion_delta", "Integer minus checkpoint confusion counts", quant_cm - float_cm, True),` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are confusion_delta: the value named 'confusion delta' in this block; Integer: the value named 'integer' in this block; minus: the value named 'minus' in this block.

Line 606: `):` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The expression is a literal or structural operator and has no hidden software state.

Line 607: `index = save_figure(confusion_figure(matrix, title, signed), plot_dir, pdf, records, index, slug, title,` This statement continues through source line 608; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'save_figure(confusion_figure(matrix, title, signed), plot_dir, pdf, records, index, slug, title, "full CIFAR-10 prediction NPZ/JSON")' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; confusion_figure: the value named 'confusion figure' in this block; matrix: the value named 'matrix' in this block. Context: the function generate, which handles generate.

Line 608: `"full CIFAR-10 prediction NPZ/JSON")` This is continuation line 2 of the 2-line logical statement that begins at source line 607. Its exact fragment is '"full CIFAR-10 prediction NPZ/JSON")'. The important operands are full: the value named 'full' in this block; CIFAR: the value named 'cifar' in this block; prediction: the class index with the greatest classifier score. Read together, the complete operation is: This assigns 'save_figure(confusion_figure(matrix, title, signed), plot_dir, pdf, records, index, slug, title, "full CIFAR-10 prediction NPZ/JSON")' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; confusion_figure: the value named 'confusion figure' in this block; matrix: the value named 'matrix' in this block. Context: the function generate, which handles generate.

Line 610: `figure, axis = plt.subplots(figsize=(11, 5.8))` This assigns 'plt.subplots(figsize=(11, 5.8))' to 'figure, axis ' in host memory. Here figure means the value named 'figure' in this block. The important operands are plt: the value named 'plt' in this block; subplots: the value named 'subplots' in this block; figsize: the value named 'figsize' in this block. Context: the function generate, which handles generate.

Line 611: `x = np.arange(10)` This assigns 'np.arange(10)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are arange: the value named 'arange' in this block. Context: the function generate, which handles generate.

Line 612: `width = 0.39` This assigns '0.39' to 'width ' in host memory. Here width means the value named 'width' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 613: `float_acc = np.diag(float_cm) / float_cm.sum(axis=1) * 100` This assigns 'np.diag(float_cm) / float_cm.sum(axis=1) * 100' to 'float_acc ' in host memory. Here float_acc means the value named 'float acc' in this block. The important operands are diag: the value named 'diag' in this block; float_cm: the value named 'float cm' in this block; sum: the value named 'sum' in this block. Context: the function generate, which handles generate.

Line 614: `quant_acc = np.diag(quant_cm) / quant_cm.sum(axis=1) * 100` This assigns 'np.diag(quant_cm) / quant_cm.sum(axis=1) * 100' to 'quant_acc ' in host memory. Here quant_acc means the value named 'quant acc' in this block. The important operands are diag: the value named 'diag' in this block; quant_cm: the value named 'quant cm' in this block; sum: the value named 'sum' in this block. Context: the function generate, which handles generate.

Line 615: `axis.bar(x - width / 2, float_acc, width, label="checkpoint", color=COLORS[0])` This calls ‘axis.bar‘ with ‘x - width / 2, float_acc, width, label="checkpoint", color=COLORS[0]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 616: `axis.bar(x + width / 2, quant_acc, width, label="integer export", color=COLORS[1])` This calls ‘axis.bar‘ with ‘x + width / 2, quant_acc, width, label="integer export", color=COLORS[1]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 617: `axis.set_xticks(x, CLASSES, rotation=32, ha="right")` This calls ‘axis.set_xticks‘ with ‘x, CLASSES, rotation=32, ha="right"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 618: `axis.set_ylim(80, 100)` This calls ‘axis.set_ylim‘ with ‘80, 100‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 619: `axis.legend(frameon=False)` This calls ‘axis.legend‘ with ‘frameon=False‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 620: `style_axes(axis, "Per-class top-1 accuracy", ylabel="Accuracy (%)")` This calls ‘style_axes‘ with ‘axis, "Per-class top-1 accuracy", ylabel="Accuracy (%)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 621: `index = save_figure(figure, plot_dir, pdf, records, index, "class_accuracy", "Per-class top-1 accuracy",` This statement continues through source line 622; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "class_accuracy", "Per-class top-1 accuracy", "full CIFAR-10 confusion matrices")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 622: `"full CIFAR-10 confusion matrices")` This is continuation line 2 of the 2-line logical statement that begins at source line 621. Its exact fragment is ‘"full CIFAR-10 confusion matrices")‘. The important operands are full: the value named ’full’ in this block; CIFAR: the value named ’cifar’ in this block; confusion: the value named ’confusion’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "class_accuracy", "Per-class top-1 accuracy", "full CIFAR-10 confusion matrices")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 624: `labels = float_npz["labels"].astype(np.int64)` This assigns ‘float_npz["labels"].astype(np.int64)‘ to ‘labels ‘ in host memory. Here labels means the value named ’labels’ in this block. The important operands are float_npz: the value named ’float npz’ in this block; labels: the value named ’labels’ in this block; astype: the value named ’astype’ in this block. Context: the function generate, which handles generate.

Line 625: `float_logits = float_npz["logits"].astype(np.float64)` This assigns ‘float_npz["logits"].astype(np.float64)‘ to ‘float_logits ‘ in host memory. Here float_logits means the ten unnormalized classifier scores used for argmax. The important operands are float_npz: the value named ’float npz’ in this block; logits: the ten unnormalized classifier scores used for argmax; astype: the value named ’astype’ in this block. Context: the function generate, which handles generate.

Line 626: `quant_logits = quant_npz["logits"].astype(np.float64)` This assigns ‘quant_npz["logits"].astype(np.float64)‘ to ‘quant_logits ‘ in host memory. Here quant_logits means the ten unnormalized classifier scores used for argmax. The important operands are quant_npz: the value named ’quant npz’ in this block; logits: the ten unnormalized classifier scores used for argmax; astype: the value named ’astype’ in this block. Context: the function generate, which handles generate.

Line 627: `float_sorted = np.sort(float_logits, axis=1)` This assigns ‘np.sort(float_logits, axis=1)‘ to ‘float_sorted ‘ in host memory. Here float_sorted means the value named ’float sorted’ in this block. The important operands are sort: the value named ’sort’ in this block; float_logits: the ten unnormalized classifier scores used for argmax; axis: the value named ’axis’ in this block. Context: the function generate, which handles generate.

Line 628: `quant_sorted = np.sort(quant_logits, axis=1)` This assigns ‘np.sort(quant_logits, axis=1)‘ to ‘quant_sorted ‘ in host memory. Here quant_sorted means the value named ’quant sorted’ in this block. The important operands are sort: the value named ’sort’ in this block; quant_logits: the ten unnormalized classifier scores used for argmax; axis: the value named ’axis’ in this block. Context: the function generate, which handles generate.

Line 629: `float_margin = float_sorted[:, -1] - float_sorted[:, -2]` This assigns ‘float_sorted[:, -1] - float_sorted[:, -2]‘ to ‘float_margin ‘ in host memory. Here float_margin means the value named ’float margin’ in this block. The important operands are float_sorted: the value named ’float sorted’ in this block. Context: the function generate, which handles generate.

Line 630: `quant_margin = quant_sorted[:, -1] - quant_sorted[:, -2]` This assigns ‘quant_sorted[:, -1] - quant_sorted[:, -2]‘ to ‘quant_margin ‘ in host memory. Here quant_margin means the value named ’quant margin’ in this block. The important operands are quant_sorted: the value named ’quant sorted’ in this block. Context: the function generate, which handles generate.

Line 631: `figure, axis = plt.subplots(figsize=(10.8, 5.8))` This assigns ‘plt.subplots(figsize=(10.8, 5.8))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 632: `axis.hist(float_margin, bins=100, density=True, alpha=0.58, label="checkpoint", color=COLORS[0])` This calls ‘axis.hist‘ with ‘float_margin, bins=100, density=True, alpha=0.58, label="checkpoint", color=COLORS[0]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 633: `axis.hist(quant_margin, bins=100, density=True, alpha=0.52, label="integer export", color=COLORS[2]` This calls ‘axis.hist‘ with ‘quant_margin, bins=100, density=True, alpha=0.52, label="integer export", color=COLORS[2]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 634: `axis.set_xlim(0, np.percentile(np.concatenate([float_margin, quant_margin]), 99.5))` This calls ‘axis.set_xlim‘ with ‘0, np.percentile(np.concatenate([float_margin, quant_margin]), 99.5)‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 635: `axis.legend(frameon=False)` This calls ‘axis.legend‘ with ‘frameon=False‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 636: `style_axes(axis, "Top-1 versus runner-up confidence margin", "Logit margin", "Probability density")` This calls ‘style_axes‘ with ‘axis, "Top-1 versus runner-up confidence margin", "Logit margin", "Probability density"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 637: `index = save_figure(figure, plot_dir, pdf, records, index, "margin_distribution", "Classifier margin distri...` This statement continues through source line 638; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "margin_distribution", "Classifier margin distribution", "all 10,000 checkpoint and integer logits")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 638: `"all 10,000 checkpoint and integer logits")` This is continuation line 2 of the 2-line logical statement that begins at source line 637. Its exact fragment is ‘"all 10,000 checkpoint and integer logits")‘. The important operands are all: the value named ’all’ in this block; checkpoint: the value named ’checkpoint’ in this block; and: the value named ’and’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "margin_distribution", "Classifier margin distribution", "all 10,000 checkpoint and integer logits")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 640: `logit_error = quant_logits - float_logits` This assigns ‘quant_logits - float_logits‘ to ‘logit_error ‘ in host memory. Here logit_error means the value named ’logit error’ in this block. The important operands are quant_logits: the ten unnormalized classifier scores used for argmax; float_logits: the ten unnormalized classifier scores used for argmax. Context: the function generate, which handles generate.

Line 641: `figure, axes = plt.subplots(1, 2, figsize=(12, 5))` This assigns ‘plt.subplots(1, 2, figsize=(12, 5))‘ to ‘figure, axes ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 642: `axes[0].hist(logit_error.ravel(), bins=150, color=COLORS[4], alpha=0.8)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; hist: the value named ’hist’ in this block; logit_error: the value named ’logit error’ in this block.

Line 643: `style_axes(axes[0], "All logit errors", "Integer - checkpoint logit", "Count")` This calls ‘style_axes‘ with ‘axes[0], "All logit errors", "Integer - checkpoint logit", "Count"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 644: `axes[1].boxplot([logit_error[:, index] for index in range(10)], tick_labels=CLASSES, showfliers=Fal` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; boxplot: the value named ’boxplot’ in this block; logit_error: the value named ’logit error’ in this block.

Line 645: `axes[1].tick_params(axis="x", rotation=40)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; tick_params: the value named ’tick params’ in this block; axis: the value named ’axis’ in this block.

Line 646: `style_axes(axes[1], "Error by output class", ylabel="Logit error")` This calls ‘style_axes‘ with ‘axes[1], "Error by output class", ylabel="Logit error"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 647: `index = save_figure(figure, plot_dir, pdf, records, index, "logit_error", "Integer-export logit error",` This statement continues through source line 648; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "logit_error", "Integer-export logit error", "paired 10,000 x 10 logit arrays")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 648: `"paired 10,000 x 10 logit arrays")` This is continuation line 2 of the 2-line logical statement that begins at source line 647. Its exact fragment is ‘"paired 10,000 x 10 logit arrays")‘. The important operands are paired: the value named ’paired’ in this block; x: the value named ’x’ in this block; logit: the value named ’logit’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "logit_error", "Integer-export logit error", "paired 10,000 x 10 logit arrays")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 650: `disagreements = float_npz["predictions"] != quant_npz["predictions"]` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are disagreements: the value named ’disagreements’ in this block; float_npz: the value named ’float npz’ in this block; predictions: the class index with the greatest classifier score.

Line 651: `transition = np.zeros((10, 10), dtype=np.int64)` This assigns ‘np.zeros((10, 10), dtype=np.int64)‘ to ‘transition ‘ in host memory. Here transition means the value named ’transition’ in this block. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function generate, which handles generate.

Line 652: `for before, after in zip(float_npz["predictions"][disagreements], quant_npz["predictions"][disagree` This iterates before, after over ‘zip(float_npz["predictions"][disagreements], quant_npz["predictions"][disagreements])‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 653: `transition[int(before), int(after)] += 1` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are transition: the value named ’transition’ in this block; before: the value named ’before’ in this block; after: the value named ’after’ in this block.

Line 654: `index = save_figure(confusion_figure(transition, "Prediction changes: checkpoint class to integer class"),` This statement continues through source line 656; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(confusion_figure(transition, "Prediction changes: checkpoint class to integer class"), plot_dir, pdf, records, index, "prediction_changes", "Prediction disagreement transitions", f"{int(disagreements.sum())} changed predictions from paired NPZ files")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; confusion_figure: the value named ’confusion figure’ in this block; transition: the value named ’transition’ in this block. Context: the function generate, which handles generate.

Line 655: `plot_dir, pdf, records, index, "prediction_changes", "Prediction disagreement transitions",` This is continuation line 2 of the 3-line logical statement that begins at source line 654. Its exact fragment is ‘plot_dir, pdf, records, index, "prediction_changes", "Prediction disagreement transitions",‘. The important operands are plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block; records: the number of descriptor records accepted since program start. Read together, the complete operation is: This assigns ‘save_figure(confusion_figure(transition, "Prediction changes: checkpoint class to integer class"), plot_dir, pdf, records, index, "prediction_changes", "Prediction disagreement transitions", f"{int(disagreements.sum())} changed predictions from paired NPZ files")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands

are save_figure: the value named 'save figure' in this block; confusion_figure: the value named 'confusion figure' in this block; transition: the value named 'transition' in this block. Context: the function generate, which handles generate.

Line 656: `f"{int(disagreements.sum())} changed predictions from paired NPZ files")` This is continuation line 3 of the 3-line logical statement that begins at source line 654. Its exact fragment is 'f"{int(disagreements.sum())} changed predictions from paired NPZ files")'. The important operands are f: the value named 'f' in this block; disagreements: the value named 'disagreements' in this block; sum: the value named 'sum' in this block. Read together, the complete operation is: This assigns 'save_figure(confusion_figure(transition, "Prediction changes: checkpoint class to integer class"), plot_dir, pdf, records, index, "prediction_changes", "Prediction disagreement transitions", f"{int(disagreements.sum())} changed predictions from paired NPZ files")' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; confusion_figure: the value named 'confusion figure' in this block; transition: the value named 'transition' in this block. Context: the function generate, which handles generate.

Line 658: `activation_stats = reference["activation_statistics"]` This assigns 'reference["activation_statistics"]' to 'activation_stats ' in host memory. Here activation_stats means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are reference: the value named 'reference' in this block; activation_statistics: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function generate, which handles generate.

Line 659: `spike_items = [(name, values) for name, values in activation_stats.items() if values["is_spike"]]` This assigns '[(name, values) for name, values in activation_stats.items() if values["is_spike"]]' to 'spike_items ' in host memory. Here spike_items means a binary event produced when a neuron crosses its threshold. The important operands are name: the value named 'name' in this block; values: the value named 'values' in this block; for: the value named 'for' in this block. Context: the function generate, which handles generate.

Line 660: `spike_items.sort(key=lambda item: item[1]["nonzero_fraction"], reverse=True)` This calls 'spike_items.so with 'key=lambda item: item[1]["nonzero_fraction"], reverse=True' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 661: `figure, axis = plt.subplots(figsize=(12, 8))` This assigns 'plt.subplots(figsize=(12, 8))' to 'figure, axis ' in host memory. Here figure means the value named 'figure' in this block. The important operands are plt: the value named 'plt' in this block; subplots: the value named 'subplots' in this block; figsize: the value named 'figsize' in this block. Context: the function generate, which handles generate.

Line 662: `names = [item[0] for item in spike_items]` This assigns '[item[0] for item in spike_items]' to 'names ' in host memory. Here names means the value named 'names' in this block. The important operands are item: the value named 'item' in this block; for: the value named 'for' in this block; in: the value named 'in' in this block. Context: the function generate, which handles generate.

Line 663: `rates = [item[1]["nonzero_fraction"] * 100 for item in spike_items]` This assigns '[item[1]["nonzero_fracti * 100 for item in spike_items]' to 'rates ' in host memory. Here rates means the value named 'rates' in this block. The important operands are item: the value named 'item' in this block; nonzero_fraction: the value named 'nonzero fraction' in this block; for: the value named 'for' in this block. Context: the function generate, which handles generate.

Line 664: `axis.barh(np.arange(len(names)), rates, color=COLORS[0])` This calls 'axis.barh' with 'np.arange(len(names)), rates, color=COLORS[0]' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 665: `axis.set_yticks(np.arange(len(names)), names, fontsize=7)` This calls 'axis.set_yticks' with 'np.arange(len(na names, fontsize=7' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 666: `axis.invert_yaxis()` This calls ‘axis.invert_yaxis‘ with “ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 667: `style_axes(axis, "Measured spike density by activation", "Nonzero spike fraction (%)")` This calls ‘style_axes‘ with ‘axis, "Measured spike density by activation", "Nonzero spike fraction (%)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 668: `index = save_figure(figure, plot_dir, pdf, records, index, "spike_density", "Measured spike density",` This statement continues through source line 669; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "spike_density", "Measured spike density", "streaming activation statistics over all CIFAR-10 test images")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 669: `"streaming activation statistics over all CIFAR-10 test images")` This is continuation line 2 of the 2-line logical statement that begins at source line 668. Its exact fragment is ‘"streaming activation statistics over all CIFAR-10 test images")‘. The important operands are streaming: the value named ’streaming’ in this block; activation: an input feature value consumed by a neuron or multiply-accumulate operation; statistics: the value named ’statistics’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "spike_density", "Measured spike density", "streaming activation statistics over all CIFAR-10 test images")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 671: `figure, axis = plt.subplots(figsize=(11, 6.2))` This assigns ‘plt.subplots(figsize=(11, 6.2))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 672: `qkv_names = [name for name in activation_stats if re.search(r"block\.\d+\.attn\.[qkv]_lif$", name)]` This assigns ‘[name for name in activation_stats if re.search(r"block\.\d+\.attn\.[qkv]_lif$", name)]‘ to ‘qkv_names ‘ in host memory. Here qkv_names means the value named ’qkv names’ in this block. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function generate, which handles generate.

Line 673: `x = np.arange(4)` This assigns ‘np.arange(4)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are arange: the value named ’arange’ in this block. Context: the function generate, which handles generate.

Line 674: `for offset, suffix, color in ((-0.25, "q_lif", COLORS[0]), (0, "k_lif", COLORS[1]), (0.25, "v_lif", COLORS[...` This iterates offset, suffix, color over ‘((-0.25, "q_lif", COLORS[0]), (0, "k_lif", COLORS[1]), (0.25, "v_lif", COLORS[2]))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 675: `values = [activation_stats[f"block.{block}.attn.{suffix}"]["nonzero_fraction"] * 100 for block in range(4)]` This assigns ‘[activation_stats[f"block.{block}.attn.{suffix}"]["nonzero_fraction"] * 100 for block in range(4)]‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are activation_stats: an input feature value consumed by a neuron or multiply-accumulate operation; f: the value named ’f’ in this block; block: the value named ’block’ in this block. Context: the function generate, which handles generate.

Line 676: `axis.bar(x + offset, values, 0.24, label=suffix[0].upper(), color=color)` This calls ‘axis.bar‘ with ‘x + offset, values, 0.24, label=suffix[0].upper(), color=color‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 677: `axis.set_xticks(x, [f"Transformer block {block}" for block in range(4)])` This calls ‘axis.set_xticks‘ with ‘x, [f"Transformer block {block}" for block in range(4)]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 678: `axis.legend(frameon=False)` This calls ‘axis.legend‘ with ‘frameon=False‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 679: `style_axes(axis, "Q/K/V spike density across depth", ylabel="Spike density (%)")` This calls ‘style_axes‘ with ‘axis, "Q/K/V spike density across depth", ylabel="Spike density (%)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 680: `index = save_figure(figure, plot_dir, pdf, records, index, "qkv_density", "Q/K/V spike density",` This statement continues through source line 681; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "qkv_density", "Q/K/V spike density", "full-test streaming activation statistics")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 681: `"full-test streaming activation statistics")` This is continuation line 2 of the 2-line logical statement that begins at source line 680. Its exact fragment is ‘"full-test streaming activation statistics")‘. The important operands are full: the value named ’full’ in this block; test: the value named ’test’ in this block; streaming: the value named ’streaming’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "qkv_density", "Q/K/V spike density", "full-test streaming activation statistics")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 683: `nonspike_items = [(name, values) for name, values in activation_stats.items() if not values["is_spi` This assigns ‘[(name, values) for name, values in activation_stats.items() if not values["is_spike"]]‘ to ‘nonspike_items ‘ in host memory. Here nonspike_items means a binary event produced when a neuron crosses its threshold. The important operands are name: the value named ’name’ in this block; values: the value named ’values’ in this block; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 684: `figure, axis = plt.subplots(figsize=(12, 7))` This assigns ‘plt.subplots(figsize=(12, 7))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 685: `names = [item[0] for item in nonspike_items]` This assigns ‘[item[0] for item in nonspike_items]‘ to ‘names ‘ in host memory. Here names means the value named ’names’ in this block. The important operands are item: the value named ’item’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function generate, which handles generate.

Line 686: `means = np.asarray([item[1]["mean"] for item in nonspike_items])` This assigns ‘np.asarray([item[1]["mean"] for item in nonspike_items])‘ to ‘means ‘ in host memory. Here means means the arithmetic average used by normaliza-

tion. The important operands are asarray: the value named 'asarray' in this block; item: the value named 'item' in this block; mean: the arithmetic average used by normalization. Context: the function generate, which handles generate.

Line 687: `stds = np.asarray([item[1]["standard_deviation"] for item in nonspike_items])` This assigns 'np.asarray([item[1]["standard_deviation"] for item in nonspike_items])' to 'stds ' in host memory. Here stds means the value named 'stds' in this block. The important operands are asarray: the value named 'asarray' in this block; item: the value named 'item' in this block; standard_deviation: the value named 'standard deviation' in this block. Context: the function generate, which handles generate.

Line 688: `axis.errorbar(np.arange(len(names)), means, yerr=stds, fmt="o", markersize=3, capsize=2, color=COLO` This calls 'axis.errorbar' with 'np.arange(len(names)), means, yerr=stds, fmt="o", markersize=3, capsize=2, color=COLORS[4]' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 689: `axis.axhline(0, color="#111827", linewidth=0.8)` This calls 'axis.axhline' with '0, color="#111827", linewidth=0.8' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 690: `axis.set_xticks(np.arange(len(names)), names, rotation=75, ha="right", fontsize=6.5)` This calls 'axis.set_xticks' with 'np.arange(len(names)), names, rotation=75, ha="right", fontsize=6.5' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 691: `style_axes(axis, "Non-spiking activation mean and one standard deviation", ylabel="Activation value")` This calls 'style_axes' with 'axis, "Non-spiking activation mean and one standard deviation", ylabel="Activation value"' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 692: `index = save_figure(figure, plot_dir, pdf, records, index, "activation_moments", "Activation mean and spread",` This statement continues through source line 693; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'save_figure(figure, plot_dir, pdf, records, index, "activation_moments", "Activation mean and spread", "full-test streaming activation moments")' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 693: `"full-test streaming activation moments")` This is continuation line 2 of the 2-line logical statement that begins at source line 692. Its exact fragment is '"full-test streaming activation moments")'. The important operands are full: the value named 'full' in this block; test: the value named 'test' in this block; streaming: the value named 'streaming' in this block. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "activation_moments", "Activation mean and spread", "full-test streaming activation moments")' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 695: `operators = execution["operators"]` This assigns 'execution["operators"]' to 'operators ' in host memory. Here operators means the value named 'operators' in this block. The important operands are execution: the value named 'execution' in this block; operators: the value named 'operators' in this block. Context: the function generate, which handles generate.

Line 696: `operator_names = [operator["name"] for operator in operators if operator.get("matrix")]` This assigns '[operator["name"] for operator in operators if operator.get("matrix")]' to 'operator_names ' in host memory. Here operator_names means the value named 'operator names' in this block. The important operands are operator: the value named 'operator' in this block; name: the value named 'name' in this block; for: the value named 'for' in this block. Context: the function generate, which handles generate.

Line 697: `macs = np.asarray([operator["matrix"]["mac_equivalents"] for operator in operators if operator.get(` This assigns ‘np.asarray([operator["matrix"]["mac_equivalents"] for operator in operators if operator.get("matrix")])‘ to ‘macs ‘ in host memory. Here macs means the value named ’macs’ in this block. The important operands are asarray: the value named ’asarray’ in this block; operator: the value named ’operator’ in this block; matrix: the value named ’matrix’ in this block. Context: the function generate, which handles generate.

Line 698: `commands = np.asarray([operator["matrix"]["commands"] for operator in operators if operator.get("ma` This assigns ‘np.asarray([operator["matrix"]["commands"] for operator in operators if operator.get("matrix")])‘ to ‘commands ‘ in host memory. Here commands means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are asarray: the value named ’asarray’ in this block; operator: the value named ’operator’ in this block; matrix: the value named ’matrix’ in this block. Context: the function generate, which handles generate.

Line 699: `input_bytes = np.asarray([operator["matrix"]["input_stream_bytes"] for operator in operators if operator.ge...` This assigns ‘np.asarray([operator["matrix"]["input_stream_bytes"] for operator in operators if operator.get("matrix")])‘ to ‘input_bytes ‘ in host memory. Here input_bytes means the value named ’input bytes’ in this block. The important operands are asarray: the value named ’asarray’ in this block; operator: the value named ’operator’ in this block; matrix: the value named ’matrix’ in this block. Context: the function generate, which handles generate.

Line 700: `output_bytes = np.asarray([operator["matrix"]["output_stream_bytes"] for operator in operators if operator....` This assigns ‘np.asarray([operator["matrix"]["output_stream_bytes"] for operator in operators if operator.get("matrix")])‘ to ‘output_bytes ‘ in host memory. Here output_bytes means the value named ’output bytes’ in this block. The important operands are asarray: the value named ’asarray’ in this block; operator: the value named ’operator’ in this block; matrix: the value named ’matrix’ in this block. Context: the function generate, which handles generate.

Line 701: `for values, slug, title, ylabel, log_scale in (` This line opens or closes a multi-line Python expression inside the function generate, which handles generate; the neighboring lines supply its arguments or elements.

Line 702: `(macs, "operator_macs", "Model work by learned operator", "MAC equivalents", True),` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are macs: the value named ’macs’ in this block; operator_macs: the value named ’operator macs’ in this block; Model: the value named ’model’ in this block.

Line 703: `(commands, "operator_commands", "Tile commands by learned operator", "Commands", True),` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operator_commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Tile: the value named ’tile’ in this block.

Line 704: `):` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The expression is a literal or structural operator and has no hidden software state.

Line 705: `figure, axis = plt.subplots(figsize=(12, 7))` This assigns ‘plt.subplots(figsize=(12, 7))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 706: `order = np.argsort(values)` This assigns ‘np.argsort(values)‘ to ‘order ‘ in host memory. Here order means the value named ’order’ in this block. The important operands are argsort: the value named ’argsort’ in this block; values: the value named ’values’ in this block. Context: the function generate, which handles generate.

Line 707: `axis.barh(np.arange(len(order)), values[order], color=COLORS[0 if slug.endswith("macs") else 1])` This calls ‘axis.barh‘ with ‘np.arange(len(order)), values[order], color=COLORS[0 if slug.endswith("macs") else 1]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 708: `axis.set_yticks(np.arange(len(order)), [operator_names[item] for item in order], fontsize=7)` This calls ‘axis.set_yticks‘ with ‘np.arange(len(order)), [operator_names[item] for item in order], fontsize=7‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 709: `if log_scale:` This executes the following indented host-code branch only when ‘log_scale‘ is true. The important operands are log_scale: the real-value spacing represented by one quantized integer step. Context: the function generate, which handles generate.

Line 710: `axis.set_xscale("log")` This calls ‘axis.set_xscale‘ with ‘"log"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 711: `style_axes(axis, title, ylabel)` This calls ‘style_axes‘ with ‘axis, title, ylabel‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 712: `index = save_figure(figure, plot_dir, pdf, records, index, slug, title,` This statement continues through source line 713; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, slug, title, "execution manifest generated from exported tensor shapes")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 713: `"execution manifest generated from exported tensor shapes")` This is continuation line 2 of the 2-line logical statement that begins at source line 712. Its exact fragment is ‘"execution manifest generated from exported tensor shapes")‘. The important operands are execution: the value named ’execution’ in this block; manifest: the value named ’manifest’ in this block; generated: the value named ’generated’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, slug, title, "execution manifest generated from exported tensor shapes")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 715: `figure, axis = plt.subplots(figsize=(12, 7))` This assigns ‘plt.subplots(figsize=(12, 7))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 716: `order = np.argsort(input_bytes + output_bytes)` This assigns ‘np.argsort(input_bytes + output_bytes)‘ to ‘order ‘ in host memory. Here order means the value named ’order’ in this block. The important operands are argsort: the value named ’argsort’ in this block; input_bytes: the value named ’input bytes’ in this block; output_bytes: the value named ’output bytes’ in this block. Context: the function generate, which handles generate.

Line 717: `axis.barh(np.arange(len(order)), input_bytes[order] / 1e6, label="input", color=COLORS[0])` This calls ‘axis.barh‘ with ‘np.arange(len(order)), input_bytes[order] / 1e6, label="input", color=COLORS[0]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 718: `axis.barh(np.arange(len(order)), output_bytes[order] / 1e6, left=input_bytes[order] / 1e6,` This statement continues through source line 719; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘axis.barh‘ with ‘np.arange(len(order)), output_bytes[order] / 1e6, left=input_bytes[order] / 1e6, label="output", color=COLORS[3]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 719: `label="output", color=COLORS[3])` This is continuation line 2 of the 2-line logical statement that begins at source line 718. Its exact fragment is ‘label="output", color=COLORS[3])‘. The important operands are label: the value named ’label’ in this block; output: the value named ’output’ in this block; color: the value named ’color’ in this block. Read together, the complete operation is: This calls ‘axis.barh‘ with ‘np.arange(len(order)), output_bytes[order] / 1e6, left=input_bytes[order] / 1e6, label="output", color=COLORS[3]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 720: `axis.set_yticks(np.arange(len(order)), [operator_names[item] for item in order], fontsize=7)` This calls ‘axis.set_yticks‘ with ‘np.arange(len(order)), [operator_names[item] for item in order], fontsize=7‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 721: `axis.legend(frameon=False)` This calls ‘axis.legend‘ with ‘frameon=False‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 722: `style_axes(axis, "Raw tile-stream traffic by learned operator", "Traffic (MB per image)")` This calls ‘style_axes‘ with ‘axis, "Raw tile-stream traffic by learned operator", "Traffic (MB per image)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 723: `index = save_figure(figure, plot_dir, pdf, records, index, "operator_traffic", "Operator stream traffic",` This statement continues through source line 724; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "operator_traffic", "Operator stream traffic", "execution manifest; excludes inter-layer DDR spill")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 724: `"execution manifest; excludes inter-layer DDR spill")` This is continuation line 2 of the 2-line logical statement that begins at source line 723. Its exact fragment is ‘"execution manifest; excludes inter-layer DDR spill")‘. The important operands are execution: the value named ’execution’ in this block; manifest: the value named ’manifest’ in this block; excludes: the value named ’excludes’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "operator_traffic", "Operator stream traffic", "execution manifest; excludes inter-layer DDR spill")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 726: `arithmetic = arithmetic_groups(operators)` This assigns ‘arithmetic_groups(operators)‘ to ‘arithmetic ‘ in host memory. Here arithmetic means the value named ’arithmetic’ in this block. The important operands are arithmetic_groups: the value named ’arithmetic groups’ in this block; operators: the value named ’operators’ in this block. Context: the function generate, which handles generate.

Line 727: `figure, axis = plt.subplots(figsize=(8, 6))` This assigns ‘plt.subplots(figsize=(8, 6))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 728: `axis.pie(list(arithmetic.values()), labels=list(arithmetic), autopct="%1.1f%%", colors=COLORS[: len(arithme...` This statement continues through source line 729; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘axis.pie‘ with ‘list(arithmetic.values()), labels=list(arithmetic), autopct="%1.1f%%", colors=COLORS[: len(arithmetic)], startangle=90, wedgeprops={"linewidth": 0.7, "edgecolor": "white"}‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 729: `startangle=90, wedgeprops={"linewidth": 0.7, "edgecolor": "white"})` This is continuation line 2 of the 2-line logical statement that begins at source line 728. Its exact fragment is ‘startangle=90, wedgeprops={"linewidth": 0.7, "edgecolor": "white"})‘. The important operands are startangle: the value named ’startangle’ in this block; wedgeprops: the value named ’wedgeprops’ in this block; linewidth: the value named ’linewidth’ in this block. Read together, the complete operation is: This calls ‘axis.pie‘ with ‘list(arithmetic.values()), labels=list(arithmetic), autopct="%1.1f%%", colors=COLORS[: len(arithmetic)], startangle=90, wedgeprops={"linewidth": 0.7, "edgecolor": "white"}‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 730: `axis.set_title("MAC-equivalent share by arithmetic mode", fontsize=13, fontweight="bold")` This calls ‘axis.set_title‘ with ‘"MAC-equivalent share by arithmetic mode", fontsize=13, fontweight="bold"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 731: `index = save_figure(figure, plot_dir, pdf, records, index, "arithmetic_share", "Arithmetic mode share",` This statement continues through source line 732; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "arithmetic_share", "Arithmetic mode share", "execution manifest model-equivalent operation counts")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 732: `"execution manifest model-equivalent operation counts")` This is continuation line 2 of the 2-line logical statement that begins at source line 731. Its exact fragment is ‘"execution manifest model-equivalent operation counts")‘. The important operands are execution: the value named ’execution’ in this block; manifest: the value named ’manifest’ in this block; model: the value named ’model’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "arithmetic_share", "Arithmetic mode share", "execution manifest model-equivalent operation counts")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 734: `matrix_records = [(operator["name"], operator["matrix"]) for operator in operators if operator.get(` This assigns ‘[(operator["name"], operator["matrix"]) for operator in operators if operator.get("matrix")]‘ to ‘matrix_records ‘ in host memory. Here matrix_records means the number of descriptor records accepted since program start. The important operands are operator: the value named ’operator’ in this block; name: the value named ’name’ in this block; matrix: the value named ’matrix’ in this block. Context: the function generate, which handles generate.

Line 735: `figure, axis = plt.subplots(figsize=(9.5, 6.5))` This assigns ‘plt.subplots(figsize=(9.5, 6.5))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 736: `for arithmetic_name in sorted({item[1]["arithmetic"] for item in matrix_records}):` This iterates arithmetic_name over ‘sorted({item[1]["arithmetic"] for item in matrix_records})‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 737: `subset = [item for item in matrix_records if item[1]["arithmetic"] == arithmetic_name]` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are subset: the value named 'subset' in this block; item: the value named 'item' in this block; for: the value named 'for' in this block.

Line 738: `axis.scatter([item[1]["reduction"] for item in subset], [item[1]["columns"] for item in subset],` This statement continues through source line 739; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘axis.scatter‘ with ‘[item[1]["reduction"] for item in subset], [item[1]["columns"] for item in subset], s=[max(25, math.sqrt(item[1]["rows"])) for item in subset], label=arithmetic_name, alpha=0.75‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 739: `s=[max(25, math.sqrt(item[1]["rows"])) for item in subset], label=arithmetic_name, alpha=0.75)` This is continuation line 2 of the 2-line logical statement that begins at source line 738. Its exact fragment is ‘s=[max(25, math.sqrt(item[1]["rows"])) for item in subset], label=arithmetic_name, alpha=0.75)‘. The important operands are s: the value named 's' in this block; max: the value named 'max' in this block; math: the value named 'math' in this block. Read together, the complete operation is: This calls ‘axis.scatter‘ with ‘[item[1]["reduction"] for item in subset], [item[1]["columns"] for item in subset], s=[max(25, math.sqrt(item[1]["rows"])) for item in subset], label=arithmetic_name, alpha=0.75‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 740: `axis.set_xscale("log")` This calls ‘axis.set_xscale‘ with ‘"log"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 741: `axis.set_yscale("log")` This calls ‘axis.set_yscale‘ with ‘"log"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 742: `axis.legend(frameon=False)` This calls ‘axis.legend‘ with ‘frameon=False‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 743: `style_axes(axis, "Matrix shape landscape", "Reduction K", "Output columns N")` This calls ‘style_axes‘ with ‘axis, "Matrix shape landscape", "Reduction K", "Output columns N"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 744: `index = save_figure(figure, plot_dir, pdf, records, index, "matrix_shapes", "Matrix shape landscape",` This statement continues through source line 745; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "matrix_shapes", "Matrix shape landscape", "execution manifest; marker area tracks output-row count")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 745: `"execution manifest; marker area tracks output-row count")` This is continuation line 2 of the 2-line logical statement that begins at source line 744. Its exact fragment is ‘"execution manifest; marker area tracks output-row count")‘. The important operands are execution: the value named 'execution' in this block; manifest: the value named 'manifest' in this block; marker: the value named 'marker' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "matrix_shapes", "Matrix shape landscape", "execution manifest; marker area tracks output-row count")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 747: `projection_names = [projection["name"] for projection in parameters["projections"]]` This assigns ‘[projection["name"] for projection in parameters["projections"]]‘ to ‘projection_names ‘ in host memory. Here projection_names means the value named ’projection names’ in this block. The important operands are projection: the value named ’projection’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 748: `weight_scales = [projection["weight_scale"] for projection in parameters["projections"]]` This assigns ‘[projection["weight_scale"] for projection in parameters["projections"]]‘ to ‘weight_scales ‘ in host memory. Here weight_scales means a learned coefficient multiplied by an activation. The important operands are projection: the value named ’projection’ in this block; weight_scale: a learned coefficient multiplied by an activation; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 749: `factor_errors = [projection["represented_factor_maximum_error"] for projection in parameters["proje` This assigns ‘[projection["represented_factor_maximum_error"] for projection in parameters["projections"]]‘ to ‘factor_errors ‘ in host memory. Here factor_errors means the value named ’factor errors’ in this block. The important operands are projection: the value named ’projection’ in this block; represented_factor_maximum_error: the value named ’represented factor maximum error’ in this block; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 750: `figure, axes = plt.subplots(2, 1, figsize=(12, 8), sharex=True)` This assigns ‘plt.subplots(2, 1, figsize=(12, 8), sharex=True)‘ to ‘figure, axes ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 751: `axes[0].plot(weight_scales, marker="o", markersize=3, color=COLORS[0])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; plot: the value named ’plot’ in this block; weight_scales: a learned coefficient multiplied by an activation.

Line 752: `axes[0].set_yscale("log")` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; set_yscale: the real-value spacing represented by one quantized integer step; log: the value named ’log’ in this block.

Line 753: `style_axes(axes[0], "Per-projection INT8 weight scale", ylabel="Scale")` This calls ‘style_axes‘ with ‘axes[0], "Per-projection INT8 weight scale", ylabel="Scale"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 754: `axes[1].plot(factor_errors, marker="o", markersize=3, color=COLORS[2])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; plot: the value named ’plot’ in this block; factor_errors: the value named ’factor errors’ in this block.

Line 755: `axes[1].set_yscale("log")` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; set_yscale: the real-value spacing represented by one quantized integer step; log: the value named ’log’ in this block.

Line 756: `axes[1].set_xticks(range(len(projection_names)), projection_names, rotation=75, ha="right", fontsize=6.5)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and

PDFs. The important operands are axes: the value named 'axes' in this block; set_xticks: the value named 'set xticks' in this block; range: the value named 'range' in this block.

Line 757: `style_axes(axes[1], "Maximum represented requantization-factor error", ylabel="Absolute factor error")` This calls ‘style_axes‘ with ‘axes[1], "Maximum represented requantization-factor error", ylabel="Absolute factor error"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 758: `index = save_figure(figure, plot_dir, pdf, records, index, "projection_quantization", "Projection quantizat...` This statement continues through source line 759; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "projection_quantization", "Projection quantization diagnostics", "hashed hardware parameter manifest")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 759: `"hashed hardware parameter manifest")` This is continuation line 2 of the 2-line logical statement that begins at source line 758. Its exact fragment is ‘"hashed hardware parameter manifest")‘. The important operands are hashed: the value named 'hashed' in this block; hardware: the value named 'hardware' in this block; parameter: the value named 'parameter' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "projection_quantization", "Projection quantization diagnostics", "hashed hardware parameter manifest")‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 761: `binary_path = root / "outputs/spikingformer_hardware_parameters" / parameters["binary_file"]` This assigns ‘root / "outputs/spikingformer_hardware_parameters" / parameters["binary_file"]‘ to ‘binary_path ‘ in host memory. Here binary_path means the value named 'binary path' in this block. The important operands are root: the value named 'root' in this block; outputs: the value named 'outputs' in this block; spikingformer_hardware_parameters: the value named 'spikingformer hardware parameters' in this block. Context: the function generate, which handles generate.

Line 762: `binary = binary_path.read_bytes()` This assigns ‘binary_path.read_bytes()‘ to ‘binary ‘ in host memory. Here binary means the value named 'binary' in this block. The important operands are binary_path: the value named 'binary path' in this block; read_bytes: the value named 'read bytes' in this block. Context: the function generate, which handles generate.

Line 763: `all_weights: list[np.ndarray] = []` This assigns ‘[]‘ to ‘all_weights‘ in host memory. Here all_weights means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 764: `for projection in parameters["projections"]:` This iterates projection over ‘parameters["projections"]‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 765: `section = projection["sections"][0]` This assigns ‘projection["sections"][0]‘ to ‘section ‘ in host memory. Here section means the value named 'section' in this block. The important operands are projection: the value named 'projection' in this block; sections: the value named 'sections' in this block. Context: the function generate, which handles generate.

Line 766: `if "weight" in section["name"] and section["dtype"] == "|i1":` This executes the following indented host-code branch only when ‘"weight" in section["name"] and section["dtype"] == "|i1"‘ is true. The important operands are weight: a learned coefficient multiplied by an activation; in: the value named 'in' in this block; section: the value named 'section' in this block. Context: the function generate, which handles generate.

Line 767: `all_weights.append(np.frombuffer(binary, dtype=np.int8, count=section["length"], offset=section["of` This calls ‘all_weights.append‘ with ‘np.frombuffer(binary, dtype=np.int8, count=section["length"], offset=section["offset"])‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 768: `head_section = parameters["head"]["sections"][0]` This assigns ‘parameters["head"]["sections"][0]‘ to ‘head_section ‘ in host memory. Here head_section means the value named ’head section’ in this block. The important operands are parameters: the value named ’parameters’ in this block; head: the value named ’head’ in this block; sections: the value named ’sections’ in this block. Context: the function generate, which handles generate.

Line 769: `all_weights.append(np.frombuffer(binary, dtype=np.int8, count=head_section["length"], offset=head_s` This calls ‘all_weights.append‘ with ‘np.frombuffer(binary, dtype=np.int8, count=head_section["length"], offset=head_section["offs inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 770: `weight_values = np.concatenate(all_weights)` This assigns ‘np.concatenate(all_weights)‘ to ‘weight_values ‘ in host memory. Here weight_values means a learned coefficient multiplied by an activation. The important operands are concatenate: the value named ’concatenate’ in this block; all_weights: a learned coefficient multiplied by an activation. Context: the function generate, which handles generate.

Line 771: `figure, axes = plt.subplots(1, 2, figsize=(12, 5.2))` This assigns ‘plt.subplots(1, 2, figsize=(12, 5.2))‘ to ‘figure, axes ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 772: `axes[0].hist(weight_values, bins=np.arange(-128.5, 128.5), color=COLORS[0])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; hist: the value named ’hist’ in this block; weight_values: a learned coefficient multiplied by an activation.

Line 773: `axes[0].set_yscale("log")` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; set_yscale: the real-value spacing represented by one quantized integer step; log: the value named ’log’ in this block.

Line 774: `style_axes(axes[0], "All exported INT8 weights", "Integer weight", "Count (log)")` This calls ‘style_axes‘ with ‘axes[0], "All exported INT8 weights", "Integer weight", "Count (log)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 775: `unique, counts = np.unique(weight_values, return_counts=True)` This assigns ‘np.unique(weight_values, return_counts=True)‘ to ‘unique, counts ‘ in host memory. Here unique means the value named ’unique’ in this block. The important operands are unique: the value named ’unique’ in this block; weight_values: a learned coefficient multiplied by an activation; return_counts: the value named ’return counts’ in this block. Context: the function generate, which handles generate.

Line 776: `top = np.argsort(counts)[-20:]` This assigns ‘np.argsort(counts)[-20:]‘ to ‘top ‘ in host memory. Here top means the value named ’top’ in this block. The important operands are argsort: the value named ’argsort’ in this block; counts: the value named ’counts’ in this block. Context: the function generate, which handles generate.

Line 777: `axes[1].bar(unique[top].astype(str), counts[top], color=COLORS[1])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes:

the value named 'axes' in this block; bar: the value named 'bar' in this block; unique: the value named 'unique' in this block.

Line 778: `axes[1].tick_params(axis="x", rotation=60)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; tick_params: the value named 'tick params' in this block; axis: the value named 'axis' in this block.

Line 779: `style_axes(axes[1], "Twenty most frequent codes", "Integer weight", "Count")` This calls 'style_axes' with 'axes[1], "Twenty most frequent codes", "Integer weight", "Count"' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 780: `index = save_figure(figure, plot_dir, pdf, records, index, "weight_distribution", "Exported weight distribu...` This statement continues through source line 781; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'save_figure(figure, plot_dir, pdf, records, index, "weight_distribution", "Exported weight distribution", "direct section views from the SHA-256-checked parameter binary")' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 781: `"direct section views from the SHA-256-checked parameter binary")` This is continuation line 2 of the 2-line logical statement that begins at source line 780. Its exact fragment is '"direct section views from the SHA-256-checked parameter binary")'. The important operands are direct: the value named 'direct' in this block; section: the value named 'section' in this block; views: the value named 'views' in this block. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "weight_distribution", "Exported weight distribution", "direct section views from the SHA-256-checked parameter binary")' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 783: `figure, axes = plt.subplots(1, 2, figsize=(11, 5))` This assigns 'plt.subplots(1, 2, figsize=(11, 5))' to 'figure, axes ' in host memory. Here figure means the value named 'figure' in this block. The important operands are plt: the value named 'plt' in this block; subplots: the value named 'subplots' in this block; figsize: the value named 'figsize' in this block. Context: the function generate, which handles generate.

Line 784: `names = ["scalar packer", "vectorized packer"]` This assigns '["scalar packer", "vectorized packer"]' to 'names ' in host memory. Here names means the value named 'names' in this block. The important operands are scalar: the value named 'scalar' in this block; packer: the value named 'packer' in this block; vectorized: the value named 'vectorized' in this block. Context: the function generate, which handles generate.

Line 785: `seconds = [95.601, compiler_benchmark["compile_seconds"]]` This assigns '[95.601, compiler_benchmark["comp to 'seconds ' in host memory. Here seconds means the value named 'seconds' in this block. The important operands are compiler_benchmark: the value named 'compiler benchmark' in this block; compile_seconds: the value named 'compile seconds' in this block. Context: the function generate, which handles generate.

Line 786: `peaks = [71.468169, compiler_benchmark["tracemalloc_peak_bytes"] / 1e6]` This assigns '[71.468169, compiler_benchmark["tracemalloc_peak_bytes"] / 1e6]' to 'peaks ' in host memory. Here peaks means the value named 'peaks' in this block. The important operands are compiler_benchmark: the value named 'compiler benchmark' in this block; tracemalloc_peak_bytes: the value named 'tracemalloc peak bytes' in this block. Context: the function generate, which handles generate.

Line 787: `axes[0].bar(names, seconds, color=[COLORS[2], COLORS[1]])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests,

XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; names: the value named 'names' in this block.

Line 788: `axes[0].set_yscale("log")` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_yscale: the real-value spacing represented by one quantized integer step; log: the value named 'log' in this block.

Line 789: `style_axes(axes[0], "Block-3 compile time", ylabel="Seconds (log)")` This calls 'style_axes' with 'axes[0], "Block-3 compile time", ylabel="Seconds (log)"' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 790: `axes[1].bar(names, peaks, color=[COLORS[2], COLORS[1]])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; names: the value named 'names' in this block.

Line 791: `style_axes(axes[1], "Peak traced memory", ylabel="MB")` This calls 'style_axes' with 'axes[1], "Peak traced memory", ylabel="MB"' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 792: `index = save_figure(figure, plot_dir, pdf, records, index, "compiler_benchmark", "Largest-layer compiler be...` This statement continues through source line 793; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'save_figure(figure, plot_dir, pdf, records, index, "compiler_benchmark", "Largest-layer compiler benchmark", "recorded scalar baseline and outputs/benchmarks/compiler_block3_memory.json")' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 793: `"recorded scalar baseline and outputs/benchmarks/compiler_block3_memory.json")` This is continuation line 2 of the 2-line logical statement that begins at source line 792. Its exact fragment is '"recorded scalar baseline and outputs/benchmarks/compiler_block3_memory.json")'. The important operands are recorded: the value named 'recorded' in this block; scalar: the value named 'scalar' in this block; baseline: the value named 'baseline' in this block. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "compiler_benchmark", "Largest-layer compiler benchmark", "recorded scalar baseline and outputs/benchmarks/compiler_block3_memo to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 795: `if training_coverage:` This executes the following indented host-code branch only when 'training_coverage' is true. The important operands are training_coverage: the value named 'training coverage' in this block. Context: the function generate, which handles generate.

Line 796: `training_summary = training_coverage["summary"]` This assigns 'training_coverage["summary"]' to 'training_summary ' in host memory. Here training_summary means the value named 'training summary' in this block. The important operands are training_coverage: the value named 'training coverage' in this block; summary: the value named 'summary' in this block. Context: the function generate, which handles generate.

Line 797: `semantics = training_coverage["training_semantics"]` This assigns 'training_coverage["training_semantics"]' to 'semantics ' in host memory. Here semantics means the value named 'semantics' in this block. The important operands are training_coverage: the value named 'training coverage' in this block; training_semantics: the value named 'training semantics' in this block. Context: the function generate, which handles generate.

Line 798: `training_lines = [` This statement continues through source line 810; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operator_count']} uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 799: `f"Target: {semantics['target_graph']}",` This is continuation line 2 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Target: {semantics['target_graph']}",‘. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operator_count']} uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 800: `f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_su`
This is continuation line 3 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operator_count']} uncovered {training_summary['uncovered_operator_count']}",‘. The important operands are f: the value named ’f’ in this block; Execution: the value named ’execution’ in this block; operators: the value named ’operators’ in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operator_count']}; uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 801: `f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_coun / {t...` This is continuation line 4 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}",‘. The important operands are f: the value named ’f’ in this block; Learned: the value named ’learned’ in this block; affine: the value named ’affine’ in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operator_count']}; uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable

deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named 'training lines' in this block. The important operands are f: the value named 'f' in this block; Target: the value named 'target' in this block; semantics: the value named 'semantics' in this block. Context: the function generate, which handles generate.

Line 802: `f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements'`
This is continuation line 5 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}",‘. The important operands are f: the value named 'f' in this block; Trainable: the value named 'trainable' in this block; deployed: the value named 'deployed' in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summ uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named 'training lines' in this block. The important operands are f: the value named 'f' in this block; Target: the value named 'target' in this block; semantics: the value named 'semantics' in this block. Context: the function generate, which handles generate.

Line 803: `f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles / {...`
This is continuation line 6 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_multiplications'] + training_summary['dynamic_backward_multiplications']):,}",‘. The important operands are f: the value named 'f' in this block; Backward: the value named 'backward' in this block; partial: the value named 'partial' in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summ uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named 'training lines' in this block. The important operands are f: the value named 'f' in this block; Target: the value named 'target' in this block; semantics: the value named 'semantics' in this block. Context: the function generate, which handles generate.

Line 804: `f"Fabric SGDW update commands: {training_summary['sgdw_update_commands']:,}",` This is continuation line 7 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Fabric SGDW update commands: {training_summary['sgdw_update_commands']:,}",‘. The important operands are f: the value named 'f' in this block; Fabric: the value named 'fabric' in this block; SGDW: the value named 'sgdw' in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operator_count']}; uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {seman-

tics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 805: `f"Quantizer derivative: {semantics['quantizer_gradient']}",` This is continuation line 8 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Quantizer derivative: {semantics['quantizer_gradient’] The important operands are f: the value named ’f’ in this block; Quantizer: the value named ’quantizer’ in this block; derivative: the value named ’derivative’ in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 806: `f"LIF derivative: {semantics['lif_gradient']}",` This is continuation line 9 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"LIF derivative: {semantics['lif_gradient']}",‘. The important operands are f: the value named ’f’ in this block; LIF: the value named ’lif’ in this block; derivative: the value named ’derivative’ in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operat uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 807: `f"Temporal schedule: {semantics['temporal_method']}",` This is continuation line 10 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Temporal schedule: {semantics['temporal_method']}",‘. The important operands are f: the value named ’f’ in this block; Temporal: the value named ’temporal’ in this block; schedule: the value named ’schedule’ in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summ uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 808: `f"Optimizer: {semantics['optimizer']}",` This is continuation line 11 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Optimizer: {semantics['optimizer']}",‘. The important operands are f: the value named ’f’ in this block; Optimizer: the value named ’optimizer’ in this block; semantics: the value named ’semantics’ in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operat

uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 809: `f"Parameter residency: {semantics['parameter_residency']}",` This is continuation line 12 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘f"Parameter residency: {semantics['parameter_residency']}",‘. The important operands are f: the value named ’f’ in this block; Parameter: the value named ’parameter’ in this block; residency: the value named ’residency’ in this block. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operator_count']}; uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_ / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['train f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['lea + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 810: `]` This is continuation line 13 of the 13-line logical statement that begins at source line 798. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘[f"Target: {semantics['target_graph']}", f"Execution operators covered: {training_summary['covered_execution_operator_count']}/{training_summary['execution_operator_count']}; uncovered {training_summary['uncovered_operator_count']}", f"Learned affine groups / dynamic attention products: {training_summary['learned_affine_group_count']} / {training_summary['dynamic_attention_matmul_count']}", f"Trainable deployed parameter elements: {training_summary['trainable_deployed_parameter_elements']:,}", f"Backward partial tiles / signed multiplications: {training_summary['total_backward_partial_tiles']:,} / {(training_summary['learned_backward_m + training_summary['dynamic_backward_multiplications']):,}", f"Fabric SGDW update commands: {training_summary['sgdw_up f"Quantizer derivative: {semantics['quantizer_gradient']}", f"LIF derivative: {semantics['lif_gradient']}", f"Temporal schedule: {semantics['temporal_method']}", f"Optimizer: {semantics['optimizer']}", f"Parameter residency: {semantics['parameter_residency']}",]‘ to ‘training_lines ‘ in host memory. Here training_lines means the value named ’training lines’ in this block. The important operands are f: the value named ’f’ in this block; Target: the value named ’target’ in this block; semantics: the value named ’semantics’ in this block. Context: the function generate, which handles generate.

Line 811: `figure = text_page("Full-Backbone Training Schedule", training_lines)` This assigns ‘text_page("Full-Backbone Training Schedule", training_lines)‘ to ‘figure ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are text_page: the value named ’text page’ in this block; Full: the value named ’full’ in this block; Backbone: the value named ’backbone’ in this block. Context: the function generate, which handles generate.

Line 812: `index = save_figure(` This statement continues through source line 815; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_schedule", "Full-backbone training schedule and numeric contract", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 813: `figure, plot_dir, pdf, records, index, "training_schedule",` This is continuation line 2 of the 4-line logical statement that begins at source line 812. Its exact fragment is ‘figure, plot_dir, pdf, records, index,

"training_schedule",‘. The important operands are figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_schedule", "Full-backbone training schedule and numeric contract", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 814: `"Full-backbone training schedule and numeric contract", str(training_coverage_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 812. Its exact fragment is ‘"Full-backbone training schedule and numeric contract", str(training_coverage_path),‘. The important operands are Full: the value named ’full’ in this block; backbone: the value named ’backbone’ in this block; training: the value named ’training’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_schedule", "Full-backbone training schedule and numeric contract", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 815: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 812. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_schedule", "Full-backbone training schedule and numeric contract", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 817: `learned = training_coverage["learned_affine_coverage"]` This assigns ‘training_coverage["learned_affine_cov to ‘learned ‘ in host memory. Here learned means the value named ’learned’ in this block. The important operands are training_coverage: the value named ’training coverage’ in this block; learned_affine_coverage: the value named ’learned affine coverage’ in this block. Context: the function generate, which handles generate.

Line 818: `learned_names = [item["name"] for item in learned]` This assigns ‘[item["name"] for item in learned]‘ to ‘learned_names ‘ in host memory. Here learned_names means the value named ’learned names’ in this block. The important operands are item: the value named ’item’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 819: `weight_elements = np.asarray([item["weight_elements"] for item in learned], dtype=np.int64)` This assigns ‘np.asarray([item["weight_elements"] for item in learned], dtype=np.int64)‘ to ‘weight_elements ‘ in host memory. Here weight_elements means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; weight_elements: a learned coefficient multiplied by an activation. Context: the function generate, which handles generate.

Line 820: `bias_elements = np.asarray([item["bias_elements"] for item in learned], dtype=np.int64)` This assigns ‘np.asarray([item["bias_elements"] for item in learned], dtype=np.int64)‘ to ‘bias_elements ‘ in host memory. Here bias_elements means an additive learned offset applied after accumulation. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; bias_elements: an additive learned offset applied after accumulation. Context: the function generate, which handles generate.

Line 821: `order = np.argsort(weight_elements + bias_elements)` This assigns ‘np.argsort(weight_elements + bias_elements)‘ to ‘order ‘ in host memory. Here order means the value named ’order’ in this block. The important operands are argsort: the value named ’argsort’ in this block; weight_elements: a learned coefficient multiplied by an activation; bias_elements: an additive learned offset applied after accumulation. Context: the function generate, which handles generate.

Line 822: `figure, axis = plt.subplots(figsize=(12, 8.5))` This assigns ‘plt.subplots(figsize=(12, 8.5))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 823: `axis.barh(np.arange(len(order)), weight_elements[order], label="weight", color=COLORS[0])` This calls ‘axis.barh‘ with ‘np.arange(len(order)), weight_elements[order], label="weight", color=COLORS[0]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 824: `axis.barh(` This statement continues through source line 827; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘axis.barh‘ with ‘ np.arange(len(order)), bias_elements[order], left=weight_elements[order], label="bias", color=COLORS[3], ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 825: `np.arange(len(order)), bias_elements[order], left=weight_elements[order],` This is continuation line 2 of the 4-line logical statement that begins at source line 824. Its exact fragment is ‘np.arange(len(order)), bias_elements[order], left=weight_elements[order],‘. The important operands are arange: the value named ’arange’ in this block; len: the value named ’len’ in this block; order: the value named ’order’ in this block. Read together, the complete operation is: This calls ‘axis.barh‘ with ‘ np.arange(len(order)), bias_elements[order], left=weight_elements[order], label="bias", color=COLORS[3], ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 826: `label="bias", color=COLORS[3],` This is continuation line 3 of the 4-line logical statement that begins at source line 824. Its exact fragment is ‘label="bias", color=COLORS[3],‘. The important operands are label: the value named ’label’ in this block; bias: an additive learned offset applied after accumulation; color: the value named ’color’ in this block. Read together, the complete operation is: This calls ‘axis.barh‘ with ‘ np.arange(len(order)), bias_elements[order], left=weight_elements[order], label="bias", color=COLORS[3], ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 827: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 824. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘axis.barh‘ with ‘ np.arange(len(order)), bias_elements[order], left=weight_elements[order], label="bias", color=COLORS[3], ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 828: `axis.set_yticks(np.arange(len(order)), [learned_names[item] for item in order], fontsize=7)` This calls ‘axis.set_yticks‘ with ‘np.arange(len(order)), [learned_names[item] for item in order], fontsize=7‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 829: `axis.set_xscale("log")` This calls ‘axis.set_xscale‘ with ‘"log"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 830: `axis.legend(frameon=False)` This calls ‘axis.legend‘ with ‘frameon=False‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 831: `style_axes(axis, "Trainable deployed parameters by affine group", "Parameter elements (log)")` This calls ‘style_axes‘ with ‘axis, "Trainable deployed parameters by affine group", "Parameter elements (log)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 832: `index = save_figure(` This statement continues through source line 835; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_parameters", "Trainable deployed parameter distribution", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 833: `figure, plot_dir, pdf, records, index, "training_parameters",` This is continuation line 2 of the 4-line logical statement that begins at source line 832. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "training_parameters",‘. The important operands are figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_parameters", "Trainable deployed parameter distribution", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 834: `"Trainable deployed parameter distribution", str(training_coverage_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 832. Its exact fragment is ‘"Trainable deployed parameter distribution", str(training_coverage_path),‘. The important operands are Trainable: the value named ’trainable’ in this block; deployed: the value named ’deployed’ in this block; parameter: the value named ’parameter’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_parameters", "Trainable deployed parameter distribution", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 835: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 832. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_parameters", "Trainable deployed parameter distribution", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 837: `backward_tiles = np.asarray(` This statement continues through source line 839; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.asarray([item["backward_partial_tiles"] for item in learned], dtype=np.int64)‘ to ‘backward_tiles ‘ in host memory. Here backward_tiles means the value named ’backward tiles’ in this block. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; backward_partial_tiles: the value named ’backward partial tiles’ in this block. Context: the function generate, which handles generate.

Line 838: `[item["backward_partial_tiles"] for item in learned], dtype=np.int64` This is continuation line 2 of the 3-line logical statement that begins at source line 837. Its exact fragment is ‘[item["backward_partial_tiles"] for item in learned], dtype=np.int64‘. The important operands are item: the value named ’item’ in this block; backward_partial_tiles: the value named ’backward partial tiles’ in this block; for: the value named ’for’ in this block. Read together, the complete operation is: This assigns ‘np.asarray([item["backward_partial_tiles"] for item in learned], dtype=np.int64)‘ to ‘backward_tiles ‘ in host memory. Here backward_tiles means the value named ’backward tiles’ in this block. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; backward_partial_tiles: the value named ’backward partial tiles’ in this block. Context: the function generate, which handles generate.

Line 839: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 837. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.asarray([item["backward_partial_tiles"] for item in learned], dtype=np.int64)‘ to ‘backward_tiles ‘ in host memory. Here backward_tiles means the value named ’backward tiles’ in this block. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; backward_partial_tiles: the value named ’backward partial tiles’ in this block. Context: the function generate, which handles generate.

Line 840: `backward_multiplies = np.asarray(` This statement continues through source line 842; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.asarray([item["backward_multiplications"] for item in learned], dtype=np.int64)‘ to ‘backward_multiplies ‘ in host memory. Here backward_multiplies means the value named ’backward multiplies’ in this block. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; backward_multiplications: the value named ’backward multiplications’ in this block. Context: the function generate, which handles generate.

Line 841: `[item["backward_multiplications"] for item in learned], dtype=np.int64` This is continuation line 2 of the 3-line logical statement that begins at source line 840. Its exact fragment is ‘[item["backward_multiplications"] for item in learned], dtype=np.int64‘. The important operands are item: the value named ’item’ in this block; backward_multiplications: the value named ’backward multiplications’ in this block; for: the value named ’for’ in this block. Read together, the complete operation is: This assigns ‘np.asarray([item["backward_multiplications"] for item in learned], dtype=np.int64)‘ to ‘backward_multiplies ‘ in host memory. Here backward_multiplies means the value named ’backward multiplies’ in this block. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; backward_multiplications: the value named ’backward multiplications’ in this block. Context: the function generate, which handles generate.

Line 842: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 840. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.asarray([item["backward_multiplications"] for item in learned], dtype=np.int64)‘ to ‘backward_multiplies ‘ in host memory. Here backward_multiplies means the value named ’backward multiplies’ in this block. The important operands are asarray: the value named ’asarray’ in this block; item: the value named ’item’ in this block; backward_multiplications: the value named ’backward multiplications’ in this block. Context: the function generate, which handles generate.

Line 843: `order = np.argsort(backward_multiplies)` This assigns ‘np.argsort(backward_multiplies)‘ to ‘order ‘ in host memory. Here order means the value named ’order’ in this block. The important operands are argsort: the value named ’argsort’ in this block; backward_multiplies: the value named ’backward multiplies’ in this block. Context: the function generate, which handles generate.

Line 844: `figure, axes = plt.subplots(1, 2, figsize=(14, 9), sharey=True)` This assigns ‘plt.subplots(1, 2, figsize=(14, 9), sharey=True)‘ to ‘figure, axes ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 845: `y = np.arange(len(order))` This assigns ‘np.arange(len(order))‘ to ‘y ‘ in host memory. Here y means the value named ’y’ in this block. The important operands are arange: the value named ’arange’ in this block; len: the value named ’len’ in this block; order: the value named ’order’ in this block. Context: the function generate, which handles generate.

Line 846: `axes[0].barh(y, backward_tiles[order], color=COLORS[1])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; barh: the value named ’barh’ in this block; y: the value named ’y’ in this block.

Line 847: `axes[0].set_yticks(y, [learned_names[item] for item in order], fontsize=7)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions,

compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_yticks: the value named 'set yticks' in this block; y: the value named 'y' in this block.

Line 848: `axes[0].set_xscale("log")` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_xscale: the real-value spacing represented by one quantized integer step; log: the value named 'log' in this block.

Line 849: `style_axes(axes[0], "Backward partial tiles", "Tiles per sample (log)")` This calls ‘style_axes‘ with ‘axes[0], "Backward partial tiles", "Tiles per sample (log)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 850: `axes[1].barh(y, backward_multiplies[order], color=COLORS[2])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; barh: the value named 'barh' in this block; y: the value named 'y' in this block.

Line 851: `axes[1].set_xscale("log")` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_xscale: the real-value spacing represented by one quantized integer step; log: the value named 'log' in this block.

Line 852: `style_axes(axes[1], "Signed backward multiplications", "Multiplications per sample (log)")` This calls ‘style_axes‘ with ‘axes[1], "Signed backward multiplications", "Multiplications per sample (log)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 853: `index = save_figure(` This statement continues through source line 856; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_backward_work", "Exact learned-layer backward work", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 854: `figure, plot_dir, pdf, records, index, "training_backward_work",` This is continuation line 2 of the 4-line logical statement that begins at source line 853. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "training_backward_work",‘. The important operands are figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block; pdf: the value named 'pdf' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_backward_work", "Exact learned-layer backward work", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 855: `"Exact learned-layer backward work", str(training_coverage_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 853. Its exact fragment is ‘"Exact learned-layer backward work", str(training_coverage_path),‘. The important operands are Exact: the value named 'exact' in this block; learned: the value named 'learned' in this block; layer: the value named 'layer' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_backward_work", "Exact learned-layer backward work", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 856: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 853. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_backward_work", "Exact learned-layer backward work", str(training_coverage_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 858: `backward_sites = {` This statement continues through source line 865; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "affine dX/dW/dB": training_summary["learned_affine_group_
"dynamic QK/AV": training_summary["dynamic_attention_matmul_count"], "LIF surrogate": training_summary["lif_node_coun
"max-pool scatter": training_summary["maxpool_backward_count"], "residual fan-out": training_summary["residual_gradient_fa
"global pool": 1, }‘ to ‘backward_sites ‘ in host memory. Here backward_sites means the value named ’backward sites’ in this block. The important operands are affine: the value named ’affine’ in this block; dX: the value named ’d x’ in this block; dW: the value named ’d w’ in this block. Context: the function generate, which handles generate.

Line 859: `"affine dX/dW/dB": training_summary["learned_affine_group_count"],` This is continuation line 2 of the 8-line logical statement that begins at source line 858. Its exact fragment is ‘"affine dX/dW/dB": training_summary["learned_affine_group_count"],‘. The important operands are affine: the value named ’affine’ in this block; dX: the value named ’d x’ in this block; dW: the value named ’d w’ in this block. Read together, the complete operation is: This assigns ‘{ "affine dX/dW/dB": training_summary["learned_affine_group_count"], "dynamic QK/AV": training_summary["dynamic_attention_matmul_count"], "LIF surrogate": training_summary["lif_node_count"], "max-pool scatter": training_summary["maxpool_backward_count"], "residual fan-out": training_summary["residual_gradient_fanout_coun
"global pool": 1, }‘ to ‘backward_sites ‘ in host memory. Here backward_sites means the value named ’backward sites’ in this block. The important operands are affine: the value named ’affine’ in this block; dX: the value named ’d x’ in this block; dW: the value named ’d w’ in this block. Context: the function generate, which handles generate.

Line 860: `"dynamic QK/AV": training_summary["dynamic_attention_matmul_count"],` This is continuation line 3 of the 8-line logical statement that begins at source line 858. Its exact fragment is ‘"dynamic QK/AV": training_summary["dynamic_attention_matmul_count"],‘. The important operands are dynamic: the value named ’dynamic’ in this block; QK: the value named ’qk’ in this block; AV: the value named ’av’ in this block. Read together, the complete operation is: This assigns ‘{ "affine dX/dW/dB": training_summary["learned_affine_group_count"], "dynamic QK/AV": training_summary["dynamic_attention_matmul_count"], "LIF surrogate": training_summary["lif_node_count"], "max-pool scatter": training_summary["maxpool_backward_count"], "residual fan-out": training_summary["residual_gradient_fanout_
"global pool": 1, }‘ to ‘backward_sites ‘ in host memory. Here backward_sites means the value named ’backward sites’ in this block. The important operands are affine: the value named ’affine’ in this block; dX: the value named ’d x’ in this block; dW: the value named ’d w’ in this block. Context: the function generate, which handles generate.

Line 861: `"LIF surrogate": training_summary["lif_node_count"],` This is continuation line 4 of the 8-line logical statement that begins at source line 858. Its exact fragment is ‘"LIF surrogate": training_summary["lif_node_count"],‘. The important operands are LIF: the value named ’lif’ in this block; surrogate: the value named ’surrogate’ in this block; training_summary: the value named ’training summary’ in this block. Read together, the complete operation is: This assigns ‘{ "affine dX/dW/dB": training_summary["learned_affine_group_count"], "dynamic QK/AV": training_summary["dynamic_attention_matmul_count"], "LIF surrogate": training_summary["lif_node_count"], "max-pool scatter": training_summary["maxpool_backward_count"], "residual fan-out": training_summary["residual_gradient_fanout_coun
"global pool": 1, }‘ to ‘backward_sites ‘ in host memory. Here backward_sites means the value named ’backward sites’ in this block. The important operands are affine: the value named ’affine’ in this block; dX: the value named ’d x’ in this block; dW: the value named ’d w’ in this block. Context: the function generate, which handles generate.

Line 862: `"max-pool scatter": training_summary["maxpool_backward_count"],` This is continuation line 5 of the 8-line logical statement that begins at source line 858. Its exact fragment is ‘"max-pool scatter": training_summary["maxpool_ba
The important operands are max: the value named ’max’ in this block; pool: the value named ’pool’ in this block; scatter: the value named ’scatter’ in this block. Read together, the complete operation is: This assigns ‘{ "affine dX/dW/dB": training_summary["learned_affine_group_count"], "dynamic QK/AV": training_summary["dynamic_attention_matmul_count"], "LIF surrogate": training_summary["lif_node_count"], "max-pool scatter": training_summary["maxpool_backward_count"], "residual fan-out": training_summary["residual_gradient_fanout_count"], "global pool": 1, }‘ to ‘backward_sites ‘ in host memory. Here backward_sites means the value named ’backward sites’ in this block. The important operands are

affine: the value named 'affine' in this block; dX: the value named 'd x' in this block; dW: the value named 'd w' in this block. Context: the function generate, which handles generate.

Line 863: `"residual fan-out": training_summary["residual_gradient_fanout_count"],` This is continuation line 6 of the 8-line logical statement that begins at source line 858. Its exact fragment is '"residual fan-out": training_summary["residual_gradient_fanout_count"],'. The important operands are residual: the value named 'residual' in this block; fan: the value named 'fan' in this block; out: the value named 'out' in this block. Read together, the complete operation is: This assigns '{ "affine dX/dW/dB": training_summary["learned_affine_group_count"], "dynamic QK/AV": training_summary["dynamic_attention_matmul_count"], "LIF surrogate": training_summary["lif_node_count"], "max-pool scatter": training_summary["maxpool_backward_count"], "residual fan-out": training_summary["residual_gradient_fanout_ "global pool": 1, }' to 'backward_sites ' in host memory. Here backward_sites means the value named 'backward sites' in this block. The important operands are affine: the value named 'affine' in this block; dX: the value named 'd x' in this block; dW: the value named 'd w' in this block. Context: the function generate, which handles generate.

Line 864: `"global pool": 1,` This is continuation line 7 of the 8-line logical statement that begins at source line 858. Its exact fragment is '"global pool": 1,'. The important operands are global: the value named 'global' in this block; pool: the value named 'pool' in this block. Read together, the complete operation is: This assigns '{ "affine dX/dW/dB": training_summary["learned_affine_group_count"], "dynamic QK/AV": training_summary["dynamic_attention_matmul_count"], "LIF surrogate": training_summary["lif_node_count"], "max-pool scatter": training_summary["maxpool_backward_count"], "residual fan-out": training_summary["residual_gradient_fanout_count"], "global pool": 1, }' to 'backward_sites ' in host memory. Here backward_sites means the value named 'backward sites' in this block. The important operands are affine: the value named 'affine' in this block; dX: the value named 'd x' in this block; dW: the value named 'd w' in this block. Context: the function generate, which handles generate.

Line 865: `}` This is continuation line 8 of the 8-line logical statement that begins at source line 858. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "affine dX/dW/dB": training_summary["learned_affine_group_count"], "dynamic QK/AV": training_summary["dynamic_attention_matmul_count"], "LIF surrogate": training_summary["lif_node_count"], "max-pool scatter": training_summary["maxpool_backward_count"], "residual fan-out": training_summary["residual_gradient_fanout_ "global pool": 1, }' to 'backward_sites ' in host memory. Here backward_sites means the value named 'backward sites' in this block. The important operands are affine: the value named 'affine' in this block; dX: the value named 'd x' in this block; dW: the value named 'd w' in this block. Context: the function generate, which handles generate.

Line 866: `figure, axes = plt.subplots(1, 2, figsize=(12.5, 5.8))` This assigns 'plt.subplots(1, 2, figsize=(12.5, 5.8))' to 'figure, axes ' in host memory. Here figure means the value named 'figure' in this block. The important operands are plt: the value named 'plt' in this block; subplots: the value named 'subplots' in this block; figsize: the value named 'figsize' in this block. Context: the function generate, which handles generate.

Line 867: `axes[0].bar(` This statement continues through source line 870; the explanation uses the complete expression rather than only the first visual fragment. Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; arange: the value named 'arange' in this block.

Line 868: `np.arange(len(backward_sites)), list(backward_sites.values()),` This is continuation line 2 of the 4-line logical statement that begins at source line 867. Its exact fragment is 'np.arange(len(backward_sites)), list(backward_sites.values()),'. The important operands are arange: the value named 'arange' in this block; len: the value named 'len' in this block; backward_sites: the value named 'backward sites' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; arange: the value named 'arange' in this block.

Line 869: `color=COLORS[:len(backward_sites)],` This is continuation line 3 of the 4-line logical statement that begins at source line 867. Its exact fragment is 'color=COLORS[:len(backward_sites)],'. The important operands are color: the value named 'color' in this block; COLORS: the value named 'colors' in this block; len: the value named

'len' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; arange: the value named 'arange' in this block.

Line 870: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 867. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; arange: the value named 'arange' in this block.

Line 871: `axes[0].set_xticks(` This statement continues through source line 873; the explanation uses the complete expression rather than only the first visual fragment. Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_xticks: the value named 'set xticks' in this block; arange: the value named 'arange' in this block.

Line 872: `np.arange(len(backward_sites)), list(backward_sites), rotation=35, ha="right"` This is continuation line 2 of the 3-line logical statement that begins at source line 871. Its exact fragment is 'np.arange(len(backward_sites)), list(backward_sites), rotation=35, ha="right"'. The important operands are arange: the value named 'arange' in this block; len: the value named 'len' in this block; backward_sites: the value named 'backward sites' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_xticks: the value named 'set xticks' in this block; arange: the value named 'arange' in this block.

Line 873: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 871. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_xticks: the value named 'set xticks' in this block; arange: the value named 'arange' in this block.

Line 874: `style_axes(axes[0], "Implemented backward sites", ylabel="Site count")` This calls 'style_axes' with 'axes[0], "Implemented backward sites", ylabel="Site count"' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 875: `work_labels = ["learned affine", "dynamic attention"]` This assigns '["learned affine", "dynamic attention"]' to 'work_labels ' in host memory. Here work_labels means the value named 'work labels' in this block. The important operands are learned: the value named 'learned' in this block; affine: the value named 'affine' in this block; dynamic: the value named 'dynamic' in this block. Context: the function generate, which handles generate.

Line 876: `work_values = [` This statement continues through source line 879; the explanation uses the complete expression rather than only the first visual fragment. This assigns '[training_summary["learned_backward_multiplications"], training_summary["dynamic_backward_multiplications"],]' to 'work_values ' in host memory. Here work_values means the value named 'work values' in this block. The important operands are training_summary: the value named 'training summary' in this block; learned_backward_multiplications: the value named 'learned backward multiplications' in this block; dynamic_backward_multiplications: the value named 'dynamic backward multiplications' in this block. Context: the function generate, which handles generate.

Line 877: `training_summary["learned_backward_multiplications"],` This is continuation line 2 of the 4-line logical statement that begins at source line 876. Its exact fragment is 'training_summary["learned_backward_multiplications"],'. The important operands are training_summary: the value named 'training summary' in this block; learned_backward_multiplicatio

the value named 'learned backward multiplications' in this block. Read together, the complete operation is: This assigns '[training_summary["learned_backward_multiplications"], training_summary["dynamic_backward_multiplications"],]' to 'work_values ' in host memory. Here work_values means the value named 'work values' in this block. The important operands are training_summary: the value named 'training summary' in this block; learned_backward_multiplications: the value named 'learned backward multiplications' in this block; dynamic_backward_multiplications: the value named 'dynamic backward multiplications' in this block. Context: the function generate, which handles generate.

Line 878: `training_summary["dynamic_backward_multiplications"],` This is continuation line 3 of the 4-line logical statement that begins at source line 876. Its exact fragment is 'training_summary["dynamic_backward_multiplications"],'. The important operands are training_summary: the value named 'training summary' in this block; dynamic_backward_multiplicati the value named 'dynamic backward multiplications' in this block. Read together, the complete operation is: This assigns '[training_summary["learned_backward_multiplications"], training_summary["dynamic_backward_multiplications"],]' to 'work_values ' in host memory. Here work_values means the value named 'work values' in this block. The important operands are training_summary: the value named 'training summary' in this block; learned_backward_multiplications: the value named 'learned backward multiplications' in this block; dynamic_backward_multiplications: the value named 'dynamic backward multiplications' in this block. Context: the function generate, which handles generate.

Line 879: `]` This is continuation line 4 of the 4-line logical statement that begins at source line 876. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '[training_summary["learned_backward_multiplications"], training_summary["dynamic_backward_multiplica]' to 'work_values ' in host memory. Here work_values means the value named 'work values' in this block. The important operands are training_summary: the value named 'training summary' in this block; learned_backward_multiplications: the value named 'learned backward multiplications' in this block; dynamic_backward_multiplications: the value named 'dynamic backward multiplications' in this block. Context: the function generate, which handles generate.

Line 880: `axes[1].pie(` This statement continues through source line 883; the explanation uses the complete expression rather than only the first visual fragment. Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; pie: the value named 'pie' in this block; work_values: the value named 'work values' in this block.

Line 881: `work_values, labels=work_labels, autopct="%1.2f%%", colors=[COLORS[0], COLORS[3]],` This is continuation line 2 of the 4-line logical statement that begins at source line 880. Its exact fragment is 'work_values, labels=work_labels, autopct="%1.2f%%", colors=[COLORS[0], COLORS[3]],'. The important operands are work_values: the value named 'work values' in this block; labels: the value named 'labels' in this block; work_labels: the value named 'work labels' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; pie: the value named 'pie' in this block; work_values: the value named 'work values' in this block.

Line 882: `startangle=90, wedgeprops={"linewidth": 0.7, "edgecolor": "white"},` This is continuation line 3 of the 4-line logical statement that begins at source line 880. Its exact fragment is 'startangle=90, wedgeprops={"linewidth": 0.7, "edgecolor": "white"},'. The important operands are startangle: the value named 'startangle' in this block; wedgeprops: the value named 'wedgeprops' in this block; linewidth: the value named 'linewidth' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; pie: the value named 'pie' in this block; work_values: the value named 'work values' in this block.

Line 883: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 880. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; pie: the value named 'pie' in this block; work_values: the value named 'work values' in this block.

Line 884: `axes[1].set_title("Backward multiplication share", fontsize=13, fontweight="bold")` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_title: the value named 'set title' in this block; Backward: the value named 'backward' in this block.

Line 885: `index = save_figure(` This statement continues through source line 888; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'save_figure(figure, plot_dir, pdf, records, index, "training_operator_coverage", "Backward operator coverage and work composition", str(training_coverage_path),)' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 886: `figure, plot_dir, pdf, records, index, "training_operator_coverage",` This is continuation line 2 of the 4-line logical statement that begins at source line 885. Its exact fragment is 'figure, plot_dir, pdf, records, index, "training_operator_coverage",'. The important operands are figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block; pdf: the value named 'pdf' in this block. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "training_operator_coverage", "Backward operator coverage and work composition", str(training_coverage_path),)' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 887: `"Backward operator coverage and work composition", str(training_coverage_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 885. Its exact fragment is '"Backward operator coverage and work composition", str(training_coverage_path),'. The important operands are Backward: the value named 'backward' in this block; operator: the value named 'operator' in this block; coverage: the value named 'coverage' in this block. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "training_operator_coverage", "Backward operator coverage and work composition", str(training_coverage_path),)' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 888: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 885. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "training_operator_coverage", "Backward operator coverage and work composition", str(training_coverage_path),)' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 890: `if shard_manifest:` This executes the following indented host-code branch only when 'shard_manifest' is true. The important operands are shard_manifest: the value named 'shard manifest' in this block. Context: the function generate, which handles generate.

Line 891: `split_names: list[str] = []` This assigns '[]' to 'split_names' in host memory. Here split_names means the value named 'split names' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 892: `shard_indices: list[int] = []` This assigns '[]' to 'shard_indices' in host memory. Here shard_indices means the value named 'shard indices' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 893: `shard_sizes: list[float] = []` This assigns '[]' to 'shard_sizes' in host memory. Here shard_sizes means the value named 'shard sizes' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 894: `for split_name, shards in shard_manifest["splits"].items():` This iterates split_name, shards over ‘shard_manifest["splits"].items()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 895: `for shard_index, shard in enumerate(shards):` This iterates shard_index, shard over ‘enumerate(shards)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 896: `split_names.append(split_name)` This calls ‘split_names.append‘ with ‘split_name‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 897: `shard_indices.append(shard_index)` This calls ‘shard_indices.append‘ with ‘shard_index‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 898: `shard_sizes.append(shard["bytes"] / 1e6)` This calls ‘shard_sizes.append‘ with ‘shard["bytes"] / 1e6‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 899: `figure, axis = plt.subplots(figsize=(11.5, 5.8))` This assigns ‘plt.subplots(figsize=(11.5, 5.8))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 900: `for color_index, split_name in enumerate(sorted(set(split_names))):` This iterates color_index, split_name over ‘enumerate(sorted(set(split_names)))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 901: `selected = [index for index, name in enumerate(split_names) if name == split_name]` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are selected: the value named ’selected’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block.

Line 902: `axis.plot(` This statement continues through source line 906; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘axis.plot‘ with ‘ [shard_indices[index] for index in selected], [shard_sizes[index] for index in selected], marker="o", markersize=3, label=split_name, color=COLORS[color_index], ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 903: `[shard_indices[index] for index in selected],` This is continuation line 2 of the 5-line logical statement that begins at source line 902. Its exact fragment is ‘[shard_indices[index] for index in selected],‘. The important operands are shard_indices: the value named ’shard indices’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block. Read together, the complete operation is: This calls ‘axis.plot‘ with ‘ [shard_indices[index] for index in selected], [shard_sizes[index] for index in selected], marker="o", markersize=3, label=split_name, color=COLORS[color_index], ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 904: `[shard_sizes[index] for index in selected],` This is continuation line 3 of the 5-line logical statement that begins at source line 902. Its exact fragment is ‘[shard_sizes[index] for index in selected],‘. The important operands are shard_sizes: the value named ’shard sizes’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block. Read together, the complete operation is: This calls

‘axis.plot‘ with ‘ [shard_indices[index] for index in selected], [shard_sizes[index] for index in selected], marker="o", markersize=3, label=split_name, color=COLORS[color_index], ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 905: `marker="o", markersize=3, label=split_name, color=COLORS[color_index],` This is continuation line 4 of the 5-line logical statement that begins at source line 902. Its exact fragment is ‘marker="o", markersize=3, label=split_name, color=COLORS[color_index],‘. The important operands are marker: the value named ’marker’ in this block; o: the value named ’o’ in this block; markersize: the value named ’markersize’ in this block. Read together, the complete operation is: This calls ‘axis.plot‘ with ‘ [shard_indices[index] for index in selected], [shard_sizes[index] for index in selected], marker="o", markersize=3, label=split_name, color=COLORS[color_index], ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 906: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 902. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘axis.plot‘ with ‘ [shard_indices[index] for index in selected], [shard_sizes[index] for index in selected], marker="o", markersize=3, label=split_name, color=COLORS[color_index], ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 907: `axis.legend(frameon=False)` This calls ‘axis.legend‘ with ‘frameon=False‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 908: `style_axes(axis, "Official CIFAR-10 shard sizes", "Shard index", "Compressed NPZ size (MB)")` This calls ‘style_axes‘ with ‘axis, "Official CIFAR-10 shard sizes", "Shard index", "Compressed NPZ size (MB)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 909: `index = save_figure(` This statement continues through source line 912; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_dataset_shards", "Deterministic CIFAR-10 board-training shards", str(shard_manifest_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 910: `figure, plot_dir, pdf, records, index, "training_dataset_shards",` This is continuation line 2 of the 4-line logical statement that begins at source line 909. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "training_dataset_shards",‘. The important operands are figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_dataset_shards", "Deterministic CIFAR-10 board-training shards", str(shard_manifest_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 911: `"Deterministic CIFAR-10 board-training shards", str(shard_manifest_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 909. Its exact fragment is ‘"Deterministic CIFAR-10 board-training shards", str(shard_manifest_path),‘. The important operands are Deterministic: the value named ’deterministic’ in this block; CIFAR: the value named ’cifar’ in this block; board: the value named ’board’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_dataset_shards", "Deterministic CIFAR-10 board-training shards", str(shard_manifest_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 912: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 909. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "training_dataset_shards", "Deterministic CIFAR-10 board-training shards", str(shard_manifest_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 914: `waveform_manifest_path = waveform_dir / "waveform_manifest.json"` This assigns ‘waveform_dir / "waveform_manifest.json"‘ to ‘waveform_manifest_path ‘ in host memory. Here waveform_manifest_path means the value named ’waveform manifest path’ in this block. The important operands are waveform_dir: the value named ’waveform dir’ in this block; waveform_manifest: the value named ’waveform manifest’ in this block; json: the value named ’json’ in this block. Context: the function generate, which handles generate.

Line 915: `if waveform_manifest_path.exists():` This executes the following indented host-code branch only when ‘waveform_manifest_path.exists()‘ is true. The important operands are waveform_manifest_path: the value named ’waveform manifest path’ in this block; exists: the value named ’exists’ in this block. Context: the function generate, which handles generate.

Line 916: `waveform_manifest = load_json(waveform_manifest_path)` This assigns ‘load_json(waveform_manifest_path)‘ to ‘waveform_manifest ‘ in host memory. Here waveform_manifest means the value named ’waveform manifest’ in this block. The important operands are load_json: the value named ’load json’ in this block; waveform_manifest_path: the value named ’waveform manifest path’ in this block. Context: the function generate, which handles generate.

Line 917: `for capture in waveform_manifest["captures"]:` This iterates capture over ‘waveform_manifest["captures"]‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 918: `vcd_path = waveform_dir / capture["vcd"]` This assigns ‘waveform_dir / capture["vcd"]‘ to ‘vcd_path ‘ in host memory. Here vcd_path means the value named ’vcd path’ in this block. The important operands are waveform_dir: the value named ’waveform dir’ in this block; capture: the value named ’capture’ in this block; vcd: the value named ’vcd’ in this block. Context: the function generate, which handles generate.

Line 919: `figure, selected_names = waveform_figure(vcd_path, f"XSim: {capture['testbench']}")` This assigns ‘waveform_figure(vcd_path, f"XSim: {capture['testbench']}")‘ to ‘figure, selected_names ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are waveform_figure: the value named ’waveform figure’ in this block; vcd_path: the value named ’vcd path’ in this block; f: the value named ’f’ in this block. Context: the function generate, which handles generate.

Line 920: `evidence = (` This statement continues through source line 923; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(f"Vivado 2026.1 XSim VCD, requested window {capture['duration']} {capture['unit']}; " f"signals: {', '.join(name.rsplit('/', 1)[-1] for name in selected_names)}")‘ to ‘evidence ‘ in host memory. Here evidence means the value named ’evidence’ in this block. The important operands are f: the value named ’f’ in this block; Vivado: the value named ’vivado’ in this block; XSim: the value named ’xsim’ in this block. Context: the function generate, which handles generate.

Line 921: `f"Vivado 2026.1 XSim VCD, requested window {capture['duration']} {capture['unit']}; "` This is continuation line 2 of the 4-line logical statement that begins at source line 920. Its exact fragment is ‘f"Vivado 2026.1 XSim VCD, requested window {capture['duration']} {capture['unit']}; "‘. The important operands are f: the value named ’f’ in this block; Vivado: the value named ’vivado’ in this block; XSim: the value named ’xsim’ in this block. Read together, the complete operation is: This assigns ‘(f"Vivado 2026.1 XSim VCD, requested window {capture['duration']} {capture['unit']}; " f"signals: {', '.join(name.rsplit('/', 1)[-1] for name in selected_names)}")‘ to ‘evidence ‘ in host memory. Here evidence means the value named ’evidence’ in this block. The important operands are f: the value named ’f’ in this block; Vivado: the value named ’vivado’ in this block; XSim: the value named ’xsim’ in this block. Context: the function generate, which handles generate.

Line 922: `f"signals: {', '.join(name.rsplit('/', 1)[-1] for name in selected_names)}"` This is continuation line 3 of the 4-line logical statement that begins at source line 920. Its exact fragment is ‘f"signals: {', '.join(name.rsplit('/', 1)[-1] for name in selected_names)}"‘. The important operands are f: the value named ’f’ in this block; signals: the value named ’signals’ in this block; join: the value named ’join’ in this block. Read together, the complete operation is: This assigns ‘(f"Vivado 2026.1 XSim VCD, requested window {capture['duration']} {capture['unit']}; " f"signals: {', '.join(name.rsplit('/', 1)[-1] for name in selected_names)}")‘ to ‘evidence ‘ in host memory. Here evidence means the value named ’evidence’ in this block. The important operands are f: the value named ’f’ in this block; Vivado: the value named ’vivado’ in this block; XSim: the value named ’xsim’ in this block. Context: the function generate, which handles generate.

Line 923: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 920. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(f"Vivado 2026.1 XSim VCD, requested window {capture['duration']} {capture['unit']}; " f"signals: {', '.join(name.rsplit('/', 1)[-1] for name in selected_names)}")‘ to ‘evidence ‘ in host memory. Here evidence means the value named ’evidence’ in this block. The important operands are f: the value named ’f’ in this block; Vivado: the value named ’vivado’ in this block; XSim: the value named ’xsim’ in this block. Context: the function generate, which handles generate.

Line 924: `index = save_figure(figure, plot_dir, pdf, records, index,` This statement continues through source line 926; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, f"wave_{capture['testbench'].removeprefix('tb_neuraedge_')}", f"Waveform: {capture['testbench']}", evidence)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 925: `f"wave_{capture['testbench'].removeprefix('tb_neuraedge_')}",` This is continuation line 2 of the 3-line logical statement that begins at source line 924. Its exact fragment is ‘f"wave_{capture['testbench'].removeprefix('tb_neuraedg
The important operands are f: the value named ’f’ in this block; wave_: the value named ’wave’ in this block; capture: the value named ’capture’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, f"wave_{capture['testbench'].removeprefix('tb_neuraedge_')}", f"Waveform: {capture['testbench']}", evidence)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 926: `f"Waveform: {capture['testbench']}", evidence)` This is continuation line 3 of the 3-line logical statement that begins at source line 924. Its exact fragment is ‘f"Waveform: {capture['testbench']}", evidence)‘. The important operands are f: the value named ’f’ in this block; Waveform: the value named ’waveform’ in this block; capture: the value named ’capture’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, f"wave_{capture['testbench'].removeprefix('tb_neuraedge_')}", f"Waveform: {capture['testbench']}", evidence)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 928: `if timing_dir is not None and timing_dir.exists():` This executes the following indented host-code branch only when ‘timing_dir is not None and timing_dir.exists()‘ is true. The important operands are timing_dir: the value named ’timing dir’ in this block; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function generate, which handles generate.

Line 929: `timing_summary_path = timing_dir / "full_routed_timing_summary.rpt"` This assigns ‘timing_dir / "full_routed_timing_summary.rpt"‘ to ‘timing_summary_path ‘ in host memory. Here timing_summary_path means the value named ’timing summary path’ in this block. The important operands are timing_dir: the value named ’timing dir’ in this block; full_routed_timing_summary: the value named ’full routed timing summary’ in this block; rpt: the value named ’rpt’ in this block. Context: the function generate, which handles generate.

Line 930: `if not timing_summary_path.exists():` This executes the following indented host-code branch only when ‘not timing_summary_path.exists()‘ is true. The important operands are not: the value named ’not’ in this block; timing_summary_path: the value named ’timing summary path’ in this block; exists: the value named ’exists’ in this block. Context: the function generate, which handles generate.

Line 931: `timing_summary_path = timing_dir / "routed_timing_summary.rpt"` This assigns ‘timing_dir / "routed_timing_ to ‘timing_summary_path ‘ in host memory. Here timing_summary_path means the value named ’timing summary path’ in this block. The important operands are timing_dir: the value named ’timing dir’ in this block; routed_timing_summary: the value named ’routed timing summary’ in this block; rpt: the value named ’rpt’ in this block. Context: the function generate, which handles generate.

Line 932: `clock_timing = parse_clock_timing(timing_summary_path)` This assigns ‘parse_clock_timing(timing_summary_ to ‘clock_timing ‘ in host memory. Here clock_timing means the value named ’clock timing’ in this block. The important operands are parse_clock_timing: the value named ’parse clock timing’ in this block; timing_summary_path: the value named ’timing summary path’ in this block. Context: the function generate, which handles generate.

Line 933: `if clock_timing:` This executes the following indented host-code branch only when ‘clock_timing‘ is true. The important operands are clock_timing: the value named ’clock timing’ in this block. Context: the function generate, which handles generate.

Line 934: `names = sorted(clock_timing)` This assigns ‘sorted(clock_timing)‘ to ‘names ‘ in host memory. Here names means the value named ’names’ in this block. The important operands are sorted: the value named ’sorted’ in this block; clock_timing: the value named ’clock timing’ in this block. Context: the function generate, which handles generate.

Line 935: `setup = [clock_timing[name][0] for name in names]` This assigns ‘[clock_timing[name][0] for name in names]‘ to ‘setup ‘ in host memory. Here setup means the value named ’setup’ in this block. The important operands are clock_timing: the value named ’clock timing’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 936: `hold = [clock_timing[name][1] for name in names]` This assigns ‘[clock_timing[name][1] for name in names]‘ to ‘hold ‘ in host memory. Here hold means the value named ’hold’ in this block. The important operands are clock_timing: the value named ’clock timing’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 937: `figure, axes = plt.subplots(1, 2, figsize=(11.5, 5.2))` This assigns ‘plt.subplots(1, 2, figsize=(11.5, 5.2))‘ to ‘figure, axes ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 938: `axes[0].bar(names, setup, color=COLORS[: len(names)])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; bar: the value named ’bar’ in this block; names: the value named ’names’ in this block.

Line 939: `axes[0].axhline(0, color=COLORS[2], linewidth=1.0)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; axhline: the value named ’axhline’ in this block; color: the value named ’color’ in this block.

Line 940: `style_axes(axes[0], "Setup slack by clock domain", ylabel="WNS (ns)")` This calls ‘style_axes‘ with ‘axes[0], "Setup slack by clock domain", ylabel="WNS (ns)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 941: `axes[1].bar(names, hold, color=COLORS[: len(names)])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; names: the value named 'names' in this block.

Line 942: `axes[1].axhline(0, color=COLORS[2], linewidth=1.0)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; axhline: the value named 'axhline' in this block; color: the value named 'color' in this block.

Line 943: `style_axes(axes[1], "Hold slack by clock domain", ylabel="WHS (ns)")` This calls ‘style_axes‘ with ‘axes[1], "Hold slack by clock domain", ylabel="WHS (ns)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 944: `index = save_figure(` This statement continues through source line 947; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "clock_domain_timing", "Routed timing by clock domain", str(timing_summary_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 945: `figure, plot_dir, pdf, records, index, "clock_domain_timing",` This is continuation line 2 of the 4-line logical statement that begins at source line 944. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "clock_domain_timing",‘. The important operands are figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block; pdf: the value named 'pdf' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "clock_domain_timing", "Routed timing by clock domain", str(timing_summary_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 946: `"Routed timing by clock domain", str(timing_summary_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 944. Its exact fragment is ‘"Routed timing by clock domain", str(timing_summary_path),‘. The important operands are Routed: the value named 'routed' in this block; timing: the value named 'timing' in this block; by: the value named 'by' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "clock_domain_timing", "Routed timing by clock domain", str(timing_summary_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 947: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 944. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "clock_domain_timing", "Routed timing by clock domain", str(timing_summary_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 949: `utilization_path = timing_dir / "full_routed_utilization_hierarchical.rpt"` This assigns ‘timing_dir / "full_routed_utilization_hierarchical.rpt"‘ to ‘utilization_path ‘ in host memory. Here utilization_path means the value named 'utilization path' in this block. The important operands are timing_dir: the value named 'timing dir' in this block; full_routed_utilization_hierarchical: the value named 'full routed utilization hierarchical' in this block; rpt: the value named 'rpt' in this block. Context: the function generate, which handles generate.

Line 950: `if not utilization_path.exists():` This executes the following indented host-code branch only when ‘not utilization_path.exists()‘ is true. The important operands are not: the value named ’not’ in this block; utilization_path: the value named ’utilization path’ in this block; exists: the value named ’exists’ in this block. Context: the function generate, which handles generate.

Line 951: `utilization_path = timing_dir / "routed_utilization_hierarchical.rpt"` This assigns ‘timing_dir / "routed_utilization_hierarchical.rpt"‘ to ‘utilization_path ‘ in host memory. Here utilization_path means the value named ’utilization path’ in this block. The important operands are timing_dir: the value named ’timing dir’ in this block; routed_utilization_hierarchical: the value named ’routed utilization hierarchical’ in this block; rpt: the value named ’rpt’ in this block. Context: the function generate, which handles generate.

Line 952: `utilization = parse_utilization_hierarchy(utilization_path)` This assigns ‘parse_utilization_hierarchy(utili to ‘utilization ‘ in host memory. Here utilization means the value named ’utilization’ in this block. The important operands are parse_utilization_hierarchy: the value named ’parse utilization hierarchy’ in this block; utilization_path: the value named ’utilization path’ in this block. Context: the function generate, which handles generate.

Line 953: `if "system_wrapper" in utilization:` This executes the following indented host-code branch only when ‘"system_wrapper" in utilization‘ is true. The important operands are system_wrapper: the value named ’system wrapper’ in this block; in: the value named ’in’ in this block; utilization: the value named ’utilization’ in this block. Context: the function generate, which handles generate.

Line 954: `top = utilization["system_wrapper"]` This assigns ‘utilization["system_wrapper"]‘ to ‘top ‘ in host memory. Here top means the value named ’top’ in this block. The important operands are utilization: the value named ’utilization’ in this block; system_wrapper: the value named ’system wrapper’ in this block. Context: the function generate, which handles generate.

Line 955: `labels_resource = ["LUT", "FF", "DSP48", "BRAM tile"]` This assigns ‘["LUT", "FF", "DSP48", "BRAM tile"]‘ to ‘labels_resource ‘ in host memory. Here labels_resource means the value named ’labels resource’ in this block. The important operands are LUT: the value named ’lut’ in this block; FF: the value named ’ff’ in this block; DSP48: the value named ’dsp48’ in this block. Context: the function generate, which handles generate.

Line 956: `counts_resource = [top["lut"], top["ff"], top["dsp"], top["bram36"] + top["bram18"] / 2]` This assigns ‘[top["lut"], top["ff"], top["dsp"], top["bram36"] + top["bram18"] / 2]‘ to ‘counts_resource ‘ in host memory. Here counts_resource means the value named ’counts resource’ in this block. The important operands are top: the value named ’top’ in this block; lut: the value named ’lut’ in this block; ff: the value named ’ff’ in this block. Context: the function generate, which handles generate.

Line 957: `capacities = [53_200, 106_400, 220, 140]` This assigns ‘[53_200, 106_400, 220, 140]‘ to ‘capacities ‘ in host memory. Here capacities means the value named ’capacities’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 958: `percents = [100 * count / capacity for count, capacity in zip(counts_resource, capacities)]` This assigns ‘[100 * count / capacity for count, capacity in zip(counts_resource, capacities)]‘ to ‘percents ‘ in host memory. Here percents means the value named ’percents’ in this block. The important operands are count: the value named ’count’ in this block; capacity: the value named ’capacity’ in this block; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 959: `figure, axes = plt.subplots(1, 2, figsize=(11.5, 5.2))` This assigns ‘plt.subplots(1, 2, figsize=(11.5, 5.2))‘ to ‘figure, axes ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 960: `axes[0].bar(labels_resource, counts_resource, color=COLORS[:4])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; labels_resource: the value named 'labels resource' in this block.

Line 961: `axes[0].set_yscale("log")` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_yscale: the real-value spacing represented by one quantized integer step; log: the value named 'log' in this block.

Line 962: `style_axes(axes[0], "Implemented primitive use", ylabel="Count (log scale)")` This calls 'style_axes' with 'axes[0], "Implemented primitive use", ylabel="Count (log scale)"' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 963: `axes[1].bar(labels_resource, percents, color=COLORS[:4])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; labels_resource: the value named 'labels resource' in this block.

Line 964: `axes[1].set_ylim(0, 100)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_ylim: the value named 'set ylim' in this block.

Line 965: `for position, value in enumerate(percents):` This iterates position, value over 'enumerate(percents)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 966: `axes[1].text(position, value + 1.5, f"{value:.1f}%", ha="center", fontsize=9)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; text: the value named 'text' in this block; position: the value named 'position' in this block.

Line 967: `style_axes(axes[1], "PYNQ-Z2 device utilization", ylabel="Available resources used (%)")` This calls 'style_axes' with 'axes[1], "PYNQ-Z2 device utilization", ylabel="Available resources used (%)"' inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 968: `index = save_figure(` This statement continues through source line 971; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'save_figure(figure, plot_dir, pdf, records, index, "full_routed_resources", "Full PYNQ routed resource utilization", str(utilization_path),)' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 969: `figure, plot_dir, pdf, records, index, "full_routed_resources",` This is continuation line 2 of the 4-line logical statement that begins at source line 968. Its exact fragment is 'figure, plot_dir, pdf, records, index, "full_routed_resources",'. The important operands are figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block; pdf: the value named 'pdf' in this block. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "full_routed_resources", "Full PYNQ routed resource utilization", str(utilization_path),)' to 'index ' in host memory. Here index means an integer selecting one tensor element,

lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 970: `"Full PYNQ routed resource utilization", str(utilization_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 968. Its exact fragment is '"Full PYNQ routed resource utilization", str(utilization_path),'. The important operands are Full: the value named 'full' in this block; PYNQ: the value named 'pynq' in this block; routed: the value named 'routed' in this block. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "full_routed_resources", "Full PYNQ routed resource utilization", str(utilization_path),)' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 971: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 968. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "full_routed_resources", "Full PYNQ routed resource utilization", str(utilization_path),)' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 973: `hierarchy_order = [` This statement continues through source line 978; the explanation uses the complete expression rather than only the first visual fragment. This assigns '[name for name in ("accelerator", "dma", "control_interconnect", "memory_interconnect", "accelerator_control_cdc", "dma_control_cdc", "mm2s_cdc", "s2mm_cdc",) if name in utilization]' to 'hierarchy_order ' in host memory. Here hierarchy_order means the value named 'hierarchy order' in this block. The important operands are name: the value named 'name' in this block; for: the value named 'for' in this block; in: the value named 'in' in this block. Context: the function generate, which handles generate.

Line 974: `name for name in (` This is continuation line 2 of the 6-line logical statement that begins at source line 973. Its exact fragment is 'name for name in ('. The important operands are name: the value named 'name' in this block; for: the value named 'for' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns '[name for name in ("accelerator", "dma", "control_interconnect", "memory_interconnect", "accelerator_control_cdc", "dma_control_cdc", "mm2s_cdc", "s2mm_cdc",) if name in utilization]' to 'hierarchy_order ' in host memory. Here hierarchy_order means the value named 'hierarchy order' in this block. The important operands are name: the value named 'name' in this block; for: the value named 'for' in this block; in: the value named 'in' in this block. Context: the function generate, which handles generate.

Line 975: `"accelerator", "dma", "control_interconnect", "memory_interconnect",` This is continuation line 3 of the 6-line logical statement that begins at source line 973. Its exact fragment is '"accelerator", "dma", "control_interconnect", "memory_interconnect",'. The important operands are accelerator: the value named 'accelerator' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; control_interconnect: the value named 'control interconnect' in this block. Read together, the complete operation is: This assigns '[name for name in ("accelerator", "dma", "control_interconnect", "memory_interconnect", "accelerator_control_cdc", "dma_control_cdc", "mm2s_cdc", "s2mm_cdc",) if name in utilization]' to 'hierarchy_order ' in host memory. Here hierarchy_order means the value named 'hierarchy order' in this block. The important operands are name: the value named 'name' in this block; for: the value named 'for' in this block; in: the value named 'in' in this block. Context: the function generate, which handles generate.

Line 976: `"accelerator_control_cdc", "dma_control_cdc", "mm2s_cdc", "s2mm_cdc",` This is continuation line 4 of the 6-line logical statement that begins at source line 973. Its exact fragment is '"accelerator_control_cdc", "dma_control_cdc", "mm2s_cdc", "s2mm_cdc",'. The important operands are accelerator_control_cdc: the value named 'accelerator control cdc' in this block; dma_control_cdc: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; mm2s_cdc: the value named 'mm2s cdc' in this block. Read together, the complete operation is: This assigns '[name for name in ("accelerator", "dma", "control_interconnect", "memory_interconnect", "accelerator_control_cdc", "dma_control_cdc", "mm2s_cdc", "s2mm_cdc",) if name in utilization]' to 'hierarchy_order

‘ in host memory. Here hierarchy_order means the value named ’hierarchy order’ in this block. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function generate, which handles generate.

Line 977: `) if name in utilization` This is continuation line 5 of the 6-line logical statement that begins at source line 973. Its exact fragment is ‘) if name in utilization‘. The important operands are name: the value named ’name’ in this block; in: the value named ’in’ in this block; utilization: the value named ’utilization’ in this block. Read together, the complete operation is: This assigns ‘[name for name in ("accelerator", "dma", "control_interconnect", "memory_interconnect", "accelerator_control_cdc", "dma_control_cdc", "mm2s_cdc", "s2mm_cdc",) if name in utilization]‘ to ‘hierarchy_order ‘ in host memory. Here hierarchy_order means the value named ’hierarchy order’ in this block. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function generate, which handles generate.

Line 978: `]` This is continuation line 6 of the 6-line logical statement that begins at source line 973. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘[name for name in ("accelerator", "dma", "control_interconnect", "memory_interconnect", "accelerator_control_cdc", "dma_control_cdc", "mm2s_cdc", "s2mm_cdc",) if name in utilization]‘ to ‘hierarchy_order ‘ in host memory. Here hierarchy_order means the value named ’hierarchy order’ in this block. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function generate, which handles generate.

Line 979: `figure, axes = plt.subplots(2, 2, figsize=(12, 8.2))` This assigns ‘plt.subplots(2, 2, figsize=(12, 8.2))‘ to ‘figure, axes ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 980: `for axis, key, title in zip(` This line opens or closes a multi-line Python expression inside the function generate, which handles generate; the neighboring lines supply its arguments or elements.

Line 981: `axes.ravel(), ("lut", "ff", "dsp", "bram36"),` This calls ‘axes.ravel‘ with ‘), ("lut", "ff", "dsp", "bram36"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 982: `("LUTs", "Flip-flops", "DSP48 blocks", "RAMB36 blocks"),` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are LUTs: the value named ’luts’ in this block; Flip: the value named ’flip’ in this block; flops: the value named ’flops’ in this block.

Line 983: `):` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The expression is a literal or structural operator and has no hidden software state.

Line 984: `values = [utilization[name][key] for name in hierarchy_order]` This assigns ‘[utilization[name][key] for name in hierarchy_order]‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are utilization: the value named ’utilization’ in this block; name: the value named ’name’ in this block; key: the value named ’key’ in this block. Context: the function generate, which handles generate.

Line 985: `axis.barh(np.arange(len(hierarchy_order)), values, color=COLORS[0])` This calls ‘axis.barh‘ with ‘np.arange(len(hierarchy_order)), values, color=COLORS[0]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 986: `axis.set_yticks(np.arange(len(hierarchy_order)), hierarchy_order, fontsize=8)` This calls ‘axis.set_ytick with ‘np.arange(len(hierarchy_order)), hierarchy_order, fontsize=8‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 987: `axis.invert_yaxis()` This calls ‘axis.invert_yaxis‘ with “ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 988: `style_axes(axis, title, xlabel="Primitive count")` This calls ‘style_axes‘ with ‘axis, title, xlabel="Primitive count"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 989: `index = save_figure(` This statement continues through source line 992; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "resource_hierarchy", "Resource use by full-system hierarchy", str(utilization_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 990: `figure, plot_dir, pdf, records, index, "resource_hierarchy",` This is continuation line 2 of the 4-line logical statement that begins at source line 989. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "resource_hierarchy",‘. The important operands are figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "resource_hierarchy", "Resource use by full-system hierarchy", str(utilization_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 991: `"Resource use by full-system hierarchy", str(utilization_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 989. Its exact fragment is ‘"Resource use by full-system hierarchy", str(utilization_path),‘. The important operands are Resource: the value named ’resource’ in this block; use: the value named ’use’ in this block; by: the value named ’by’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "resource_hierarchy", "Resource use by full-system hierarchy", str(utilization_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 992: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 989. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "resource_hierarchy", "Resource use by full-system hierarchy", str(utilization_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 994: `drc_path = timing_dir / "full_drc.rpt"` This assigns ‘timing_dir / "full_drc.rpt"‘ to ‘drc_path ‘ in host memory. Here drc_path means the value named ’drc path’ in this block. The important operands are timing_dir: the value named ’timing dir’ in this block; full_drc: the value named ’full drc’ in this block; rpt: the value named ’rpt’ in this block. Context: the function generate, which handles generate.

Line 995: `drc_rows = parse_drc_summary(drc_path)` This assigns ‘parse_drc_summary(drc_path)‘ to ‘drc_rows ‘ in host memory. Here drc_rows means one output-row position processed by the tensor tile. The important operands

are parse_drc_summary: the value named 'parse drc summary' in this block; drc_path: the value named 'drc path' in this block. Context: the function generate, which handles generate.

Line 996: `if drc_rows:` This executes the following indented host-code branch only when ‘drc_rows‘ is true. The important operands are drc_rows: one output-row position processed by the tensor tile. Context: the function generate, which handles generate.

Line 997: `figure, axis = plt.subplots(figsize=(11, 6.2))` This assigns ‘plt.subplots(figsize=(11, 6.2))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named 'figure' in this block. The important operands are plt: the value named 'plt' in this block; subplots: the value named 'subplots' in this block; figsize: the value named 'figsize' in this block. Context: the function generate, which handles generate.

Line 998: `labels_drc = [f"{rule}: {description}" for rule, _, description, _ in drc_rows]` This assigns ‘[f"{rule}: {description}" for rule, _, description, _ in drc_rows]‘ to ‘labels_drc ‘ in host memory. Here labels_drc means the value named 'labels drc' in this block. The important operands are f: the value named 'f' in this block; rule: the value named 'rule' in this block; description: the value named 'description' in this block. Context: the function generate, which handles generate.

Line 999: `counts_drc = [count for _, _, _, count in drc_rows]` This assigns ‘[count for _, _, _, count in drc_rows]‘ to ‘counts_drc ‘ in host memory. Here counts_drc means the value named 'counts drc' in this block. The important operands are count: the value named 'count' in this block; for: the value named 'for' in this block; _: the value named '' in this block. Context: the function generate, which handles generate.

Line 1000: `axis.barh(np.arange(len(drc_rows)), counts_drc, color=COLORS[3])` This calls ‘axis.barh‘ with ‘np.arange(len(drc_rows)), counts_drc, color=COLORS[3]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1001: `axis.set_yticks(np.arange(len(drc_rows)), labels_drc, fontsize=8)` This calls ‘axis.set_yticks‘ with ‘np.arange(len(drc_rows)), labels_drc, fontsize=8‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1002: `axis.invert_yaxis()` This calls ‘axis.invert_yaxis‘ with ‘‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1003: `axis.set_xscale("log")` This calls ‘axis.set_xscale‘ with ‘"log"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1004: `style_axes(axis, "Vivado DRC advisory classification", "Reported checks (log scale)")` This calls ‘style_axes‘ with ‘axis, "Vivado DRC advisory classification", "Reported checks (log scale)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1005: `index = save_figure(` This statement continues through source line 1008; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "drc_classification", "DRC warning classification", str(drc_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1006: `figure, plot_dir, pdf, records, index, "drc_classification",` This is continuation line 2 of the 4-line logical statement that begins at source line 1005. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "drc_classification",‘. The important operands are figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "drc_classification", "DRC warning classification", str(drc_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 1007: `"DRC warning classification", str(drc_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 1005. Its exact fragment is ‘"DRC warning classification", str(drc_path),‘. The important operands are DRC: the value named ’drc’ in this block; warning: the value named ’warning’ in this block; classification: the value named ’classification’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "drc_classification", "DRC warning classification", str(drc_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 1008: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1005. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "drc_classification", "DRC warning classification", str(drc_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 1010: `cdc_path = timing_dir / "full_cdc.rpt"` This assigns ‘timing_dir / "full_cdc.rpt"‘ to ‘cdc_path ‘ in host memory. Here cdc_path means the value named ’cdc path’ in this block. The important operands are timing_dir: the value named ’timing dir’ in this block; full_cdc: the value named ’full cdc’ in this block; rpt: the value named ’rpt’ in this block. Context: the function generate, which handles generate.

Line 1011: `cdc_rows = parse_cdc_summary(cdc_path)` This assigns ‘parse_cdc_summary(cdc_path)‘ to ‘cdc_rows ‘ in host memory. Here cdc_rows means one output-row position processed by the tensor tile. The important operands are parse_cdc_summary: the value named ’parse cdc summary’ in this block; cdc_path: the value named ’cdc path’ in this block. Context: the function generate, which handles generate.

Line 1012: `if cdc_rows:` This executes the following indented host-code branch only when ‘cdc_rows‘ is true. The important operands are cdc_rows: one output-row position processed by the tensor tile. Context: the function generate, which handles generate.

Line 1013: `figure, axis = plt.subplots(figsize=(10.5, 5.4))` This assigns ‘plt.subplots(figsize=(10.5, 5.4))‘ to ‘figure, axis ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 1014: `labels_cdc = [f"{rule} {severity}\n{description}" for rule, severity, _, description in cdc_rows]` This assigns ‘[f"{rule} {severity}\n{description}" for rule, severity, _, description in cdc_rows]‘ to ‘labels_cdc ‘ in host memory. Here labels_cdc means the value named ’labels cdc’ in this block. The important operands are f: the value named ’f’ in this block; rule: the value named ’rule’ in this block; severity: the value named ’severity’ in this block. Context: the function generate, which handles generate.

Line 1015: `counts_cdc = [count for _, _, count, _ in cdc_rows]` This assigns ‘[count for _, _, count, _ in cdc_rows]‘ to ‘counts_cdc ‘ in host memory. Here counts_cdc means the value named ’counts cdc’ in this block. The important operands are count: the value named ’count’ in this block; for: the value named ’for’ in this block; _: the value named ” in this block. Context: the function generate, which handles generate.

Line 1016: `axis.bar(labels_cdc, counts_cdc, color=[COLORS[1], COLORS[3]][: len(cdc_rows)])` This calls ‘axis.bar‘ with ‘labels_cdc, counts_cdc, color=[COLORS[1], COLORS[3]][: len(cdc_rows)]‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1017: `style_axes(axis, "Vivado clock-domain crossing classification", ylabel="Reported crossings")` This calls ‘style_axes‘ with ‘axis, "Vivado clock-domain crossing classification", ylabel="Reported crossings"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1018: `index = save_figure(` This statement continues through source line 1021; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "cdc_classification", "Clock-domain crossing classification", str(cdc_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 1019: `figure, plot_dir, pdf, records, index, "cdc_classification",` This is continuation line 2 of the 4-line logical statement that begins at source line 1018. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "cdc_classification",‘. The important operands are figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "cdc_classification", "Clock-domain crossing classification", str(cdc_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 1020: `"Clock-domain crossing classification", str(cdc_path),` This is continuation line 3 of the 4-line logical statement that begins at source line 1018. Its exact fragment is ‘"Clock-domain crossing classification", str(cdc_path),‘. The important operands are Clock: the value named ’clock’ in this block; domain: the value named ’domain’ in this block; crossing: the value named ’crossing’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "cdc_classification", "Clock-domain crossing classification", str(cdc_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 1021: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1018. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "cdc_classification", "Clock-domain crossing classification", str(cdc_path),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 1023: `if vectorless_power and saif_power:` This executes the following indented host-code branch only when ‘vectorless_power and saif_power‘ is true. The important operands are vectorless_power: the value named ’vectorless power’ in this block; and: the value named ’and’ in this block; saif_power: the value named ’saif power’ in this block. Context: the function generate, which handles generate.

Line 1024: `labels_power = ["Total", "Dynamic", "Static"]` This assigns ‘["Total", "Dynamic", "Static"]‘ to ‘labels_power ‘ in host memory. Here labels_power means the value named ’labels power’ in this block. The important operands are Total: the value named ’total’ in this block; Dynamic: the value named ’dynamic’ in this block; Static: the value named ’static’ in this block. Context: the function generate, which handles generate.

Line 1025: `vector_values = [vectorless_power[key] for key in ("total", "dynamic", "static")]` This assigns ‘[vectorless_power[key] for key in ("total", "dynamic", "static")]‘ to ‘vector_values ‘ in host memory. Here vector_values means the value named ’vector values’ in this block. The important operands are vectorless_power: the value named ’vectorless power’ in this block; key: the value named ’key’ in this block; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 1026: `saif_values = [saif_power[key] for key in ("total", "dynamic", "static")]` This assigns ‘[saif_power[ke
for key in ("total", "dynamic", "static")]‘ to ‘saif_values ‘ in host memory. Here saif_values means the value named ’saif values’ in this block. The important operands are saif_power: the value named ’saif power’ in this block; key: the value named ’key’ in this block; for: the value named ’for’ in this block. Context: the function generate, which handles generate.

Line 1027: `x = np.arange(len(labels_power))` This assigns ‘np.arange(len(labels_power))‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are arange: the value named ’arange’ in this block; len: the value named ’len’ in this block; labels_power: the value named ’labels power’ in this block. Context: the function generate, which handles generate.

Line 1028: `figure, axes = plt.subplots(1, 2, figsize=(12, 5.5))` This assigns ‘plt.subplots(1, 2, figsize=(12, 5.5))‘ to ‘figure, axes ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are plt: the value named ’plt’ in this block; subplots: the value named ’subplots’ in this block; figsize: the value named ’figsize’ in this block. Context: the function generate, which handles generate.

Line 1029: `axes[0].bar(x - 0.2, vector_values, 0.4, label="vectorless", color=COLORS[3])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; bar: the value named ’bar’ in this block; x: the value named ’x’ in this block.

Line 1030: `axes[0].bar(x + 0.2, saif_values, 0.4, label="SAIF annotated", color=COLORS[1])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; bar: the value named ’bar’ in this block; x: the value named ’x’ in this block.

Line 1031: `axes[0].set_xticks(x, labels_power)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; set_xticks: the value named ’set xticks’ in this block; x: the value named ’x’ in this block.

Line 1032: `axes[0].legend(frameon=False)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; legend: the value named ’legend’ in this block; frameon: the value named ’frameon’ in this block.

Line 1033: `style_axes(axes[0], "Full-system Vivado power estimates", ylabel="Power (W)")` This calls ‘style_axes‘ with ‘axes[0], "Full-system Vivado power estimates", ylabel="Power (W)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1034: `hierarchy_order = [` This statement continues through source line 1037; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘[name for name in ("accelerator", "dma", "mm2s_cdc", "s2mm_cdc") if name in vectorless_power["hierarchy"] and name in saif_power["hierarchy"]]‘ to ‘hierarchy_order ‘ in host memory. Here hierarchy_order means the value named ’hierarchy order’ in this block. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function generate, which handles generate.

Line 1035: `name for name in ("accelerator", "dma", "mm2s_cdc", "s2mm_cdc")` This is continuation line 2 of the 4-line logical statement that begins at source line 1034. Its exact fragment is ‘name for name in ("accelerator", "dma", "mm2s_cdc", "s2mm_cdc")‘. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘[name for name in ("accelerator", "dma", "mm2s_cdc", "s2mm_cdc") if name in vectorless_power["hierarchy"] and name in saif_power["hierarchy"]]‘ to ‘hierarchy_order ‘ in host memory. Here hierarchy_order means the value named ’hierarchy order’ in this block. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function generate, which handles generate.

Line 1036: `if name in vectorless_power["hierarchy"] and name in saif_power["hierarchy"]` This is continuation line 3 of the 4-line logical statement that begins at source line 1034. Its exact fragment is ‘if name in vectorless_power["hierarchy"] and name in saif_power["hierarchy"]‘. The important operands are name: the value named ’name’ in this block; in: the value named ’in’ in this block; vectorless_power: the value named ’vectorless power’ in this block. Read together, the complete operation is: This assigns ‘[name for name in ("accelerator", "dma", "mm2s_cdc", "s2mm_cdc") if name in vectorless_power["hierarchy"] and name in saif_power["hierarchy"]]‘ to ‘hierarchy_order ‘ in host memory. Here hierarchy_order means the value named ’hierarchy order’ in this block. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function generate, which handles generate.

Line 1037: `]` This is continuation line 4 of the 4-line logical statement that begins at source line 1034. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘[name for name in ("accelerator", "dma", "mm2s_cdc", "s2mm_cdc") if name in vectorless_power["hierarchy"] and name in saif_power["hierarchy"]]‘ to ‘hierarchy_order ‘ in host memory. Here hierarchy_order means the value named ’hierarchy order’ in this block. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function generate, which handles generate.

Line 1038: `hierarchy_x = np.arange(len(hierarchy_order))` This assigns ‘np.arange(len(hierarchy_order))‘ to ‘hierarchy_x ‘ in host memory. Here hierarchy_x means the value named ’hierarchy x’ in this block. The important operands are arange: the value named ’arange’ in this block; len: the value named ’len’ in this block; hierarchy_order: the value named ’hierarchy order’ in this block. Context: the function generate, which handles generate.

Line 1039: `axes[1].bar(` This statement continues through source line 1043; the explanation uses the complete expression rather than only the first visual fragment. Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; bar: the value named ’bar’ in this block; hierarchy_x: the value named ’hierarchy x’ in this block.

Line 1040: `hierarchy_x - 0.2,` This is continuation line 2 of the 5-line logical statement that begins at source line 1039. Its exact fragment is ‘hierarchy_x - 0.2,‘. The important operands are hierarchy_x: the value named ’hierarchy x’ in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; bar: the value named ’bar’ in this block; hierarchy_x: the value named ’hierarchy x’ in this block.

Line 1041: `[vectorless_power["hierarchy"][name] for name in hierarchy_order],` This is continuation line 3 of the 5-line logical statement that begins at source line 1039. Its exact fragment is ‘[vectorless_power["hierarchy"][name] for name in hierarchy_order],‘. The important operands are vectorless_power: the value named ’vectorless power’ in this block; hierarchy: the value named ’hierarchy’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named ’axes’ in this block; bar: the value named ’bar’ in this block; hierarchy_x: the value named ’hierarchy x’ in this block.

Line 1042: `0.4, label="vectorless", color=COLORS[3],` This is continuation line 4 of the 5-line logical statement that begins at source line 1039. Its exact fragment is ‘0.4, label="vectorless", color=COLORS[3],‘. The important

operands are label: the value named 'label' in this block; vectorless: the value named 'vectorless' in this block; color: the value named 'color' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; hierarchy_x: the value named 'hierarchy x' in this block.

Line 1043: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1039. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; hierarchy_x: the value named 'hierarchy x' in this block.

Line 1044: `axes[1].bar(` This statement continues through source line 1048; the explanation uses the complete expression rather than only the first visual fragment. Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; hierarchy_x: the value named 'hierarchy x' in this block.

Line 1045: `hierarchy_x + 0.2,` This is continuation line 2 of the 5-line logical statement that begins at source line 1044. Its exact fragment is 'hierarchy_x + 0.2,'. The important operands are hierarchy_x: the value named 'hierarchy x' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; hierarchy_x: the value named 'hierarchy x' in this block.

Line 1046: `[saif_power["hierarchy"][name] for name in hierarchy_order],` This is continuation line 3 of the 5-line logical statement that begins at source line 1044. Its exact fragment is '[saif_power["hierarchy"][name] for name in hierarchy_order],'. The important operands are saif_power: the value named 'saif power' in this block; hierarchy: the value named 'hierarchy' in this block; name: the value named 'name' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; hierarchy_x: the value named 'hierarchy x' in this block.

Line 1047: `0.4, label="SAIF annotated", color=COLORS[1],` This is continuation line 4 of the 5-line logical statement that begins at source line 1044. Its exact fragment is '0.4, label="SAIF annotated", color=COLORS[1],'. The important operands are label: the value named 'label' in this block; SAIF: the value named 'saif' in this block; annotated: the value named 'annotated' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; hierarchy_x: the value named 'hierarchy x' in this block.

Line 1048: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1044. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; bar: the value named 'bar' in this block; hierarchy_x: the value named 'hierarchy x' in this block.

Line 1049: `axes[1].set_xticks(hierarchy_x, hierarchy_order, rotation=25, ha="right")` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; set_xticks: the value named 'set xticks' in this block; hierarchy_x: the value named 'hierarchy x' in this block.

Line 1050: `axes[1].legend(frameon=False)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; legend: the value named 'legend' in this block; frameon: the value named 'frameon' in this block.

Line 1051: `style_axes(axes[1], "Dynamic power by hierarchy", ylabel="Dynamic power (W)")` This calls ‘style_axes‘ with ‘axes[1], "Dynamic power by hierarchy", ylabel="Dynamic power (W)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1052: `index = save_figure(` This statement continues through source line 1056; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "power_comparison", "Vectorless and SAIF-annotated power estimates", f"{vectorless_power_path}; {saif_power_report}; SAIF match {saif_power['matched']}",)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1053: `figure, plot_dir, pdf, records, index, "power_comparison",` This is continuation line 2 of the 5-line logical statement that begins at source line 1052. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "power_comparison",‘. The important operands are figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block; pdf: the value named 'pdf' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "power_comparison", "Vectorless and SAIF-annotated power estimates", f"{vectorless_power_path}; {saif_power_report}; SAIF match {saif_power['matched']}",)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1054: `"Vectorless and SAIF-annotated power estimates",` This is continuation line 3 of the 5-line logical statement that begins at source line 1052. Its exact fragment is ‘"Vectorless and SAIF-annotated power estimates",‘. The important operands are Vectorless: the value named 'vectorless' in this block; and: the value named 'and' in this block; SAIF: the value named 'saif' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "power_comparison", "Vectorless and SAIF-annotated power estimates", f"{vectorless_power_path}; {saif_power_report}; SAIF match {saif_power['matched']}",)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1055: `f"{vectorless_power_path}; {saif_power_report}; SAIF match {saif_power['matched']}",` This is continuation line 4 of the 5-line logical statement that begins at source line 1052. Its exact fragment is ‘f"{vectorless_power_path}; {saif_power_report}; SAIF match {saif_power['matched']}",‘. The important operands are f: the value named 'f' in this block; vectorless_power_path: the value named 'vectorless power path' in this block; saif_power_report: the value named 'saif power report' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "power_comparison", "Vectorless and SAIF-annotated power estimates", f"{vectorless_power_path}; {saif_power_report}; SAIF match {saif_power['matched']}",)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1056: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1052. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "power_comparison", "Vectorless and SAIF-annotated power estimates", f"{vectorless_power_path}; {saif_power_report}; SAIF match {saif_power['matched']}",)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1058: `power_lines = [` This statement continues through source line 1064; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘[f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confidence {vectorless_power['confidence']}", f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['confidence']}", f"SAIF implementation nets matched: {saif_power['matched']}", "Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched storage.", "These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",]‘ to ‘power_lines ‘ in host memory. Here power_lines means the value named ’power lines’ in this block. The important operands are f: the value named ’f’ in this block; Vectorless: the value named ’vectorless’ in this block; total: the value named ’total’ in this block. Context: the function generate, which handles generate.

Line 1059: `f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confide...` This is continuation line 2 of the 7-line logical statement that begins at source line 1058. Its exact fragment is ‘f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confidence {vectorless_power['confidence']}",‘. The important operands are f: the value named ’f’ in this block; Vectorless: the value named ’vectorless’ in this block; total: the value named ’total’ in this block. Read together, the complete operation is: This assigns ‘[f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confidence {vectorless_power['confidence']}", f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['confidence']}", f"SAIF implementation nets matched: {saif_power['matched']}", "Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched storage.", "These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",]‘ to ‘power_lines ‘ in host memory. Here power_lines means the value named ’power lines’ in this block. The important operands are f: the value named ’f’ in this block; Vectorless: the value named ’vectorless’ in this block; total: the value named ’total’ in this block. Context: the function generate, which handles generate.

Line 1060: `f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['c...` This is continuation line 3 of the 7-line logical statement that begins at source line 1058. Its exact fragment is ‘f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['confidence']} The important operands are f: the value named ’f’ in this block; SAIF: the value named ’saif’ in this block; total: the value named ’total’ in this block. Read together, the complete operation is: This assigns ‘[f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confidence {vectorless_power['confidence']}", f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['confidence']}", f"SAIF implementation nets matched: {saif_power['matched']}", "Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched storage.", "These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",]‘ to ‘power_lines ‘ in host memory. Here power_lines means the value named ’power lines’ in this block. The important operands are f: the value named ’f’ in this block; Vectorless: the value named ’vectorless’ in this block; total: the value named ’total’ in this block. Context: the function generate, which handles generate.

Line 1061: `f"SAIF implementation nets matched: {saif_power['matched']}",` This is continuation line 4 of the 7-line logical statement that begins at source line 1058. Its exact fragment is ‘f"SAIF implementation nets matched: {saif_power['matched']}",‘. The important operands are f: the value named ’f’ in this block; SAIF: the value named ’saif’ in this block; implementation: the value named ’implementation’ in this block. Read together, the complete operation is: This assigns ‘[f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confidence {vectorless_power['confidence']}", f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['confidence']}", f"SAIF implementation nets matched: {saif_power['matched']}", "Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched storage.", "These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",]‘ to ‘power_lines ‘ in host memory. Here power_lines means the value named ’power lines’ in this block. The important operands are f: the value named ’f’ in this block; Vectorless: the value named ’vectorless’ in this block; total: the value named ’total’ in this block. Context: the function generate, which handles generate.

Line 1062: `"Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched stor...` This is continuation line 5 of the 7-line logical statement that begins at source line 1058. Its exact fragment is ‘"Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched storage.",‘. The important operands are Captured: the value named ’captured’ in this block; tests: the value named ’tests’ in this block; do: the value named ’do’ in this block. Read together, the complete operation is: This assigns

‘[f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confidence {vectorless_power['confidence']}", f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['confidence']}", f"SAIF implementation nets matched: {saif_power['matched']}", "Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched storage.", "These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",]‘ to ‘power_lines ‘ in host memory. Here power_lines means the value named ’power lines’ in this block. The important operands are f: the value named ’f’ in this block; Vectorless: the value named ’vectorless’ in this block; total: the value named ’total’ in this block. Context: the function generate, which handles generate.

Line 1063: `"These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",` This is continuation line 6 of the 7-line logical statement that begins at source line 1058. Its exact fragment is ‘"These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",‘. The important operands are These: the value named ’these’ in this block; are: the value named ’are’ in this block; tool: the value named ’tool’ in this block. Read together, the complete operation is: This assigns ‘[f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confidence {vectorless_power['confidence']}", f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['confidence']}", f"SAIF implementation nets matched: {saif_power['matched']}", "Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched storage.", "These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",]‘ to ‘power_lines ‘ in host memory. Here power_lines means the value named ’power lines’ in this block. The important operands are f: the value named ’f’ in this block; Vectorless: the value named ’vectorless’ in this block; total: the value named ’total’ in this block. Context: the function generate, which handles generate.

Line 1064: `]` This is continuation line 7 of the 7-line logical statement that begins at source line 1058. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘[f"Vectorless: total {vectorless_power['total']:.3f} W, dynamic {vectorless_power['dynamic']:.3f} W, confidence {vectorless_power['confidence']}", f"SAIF: total {saif_power['total']:.3f} W, dynamic {saif_power['dynamic']:.3f} W, confidence {saif_power['confidence']}", f"SAIF implementation nets matched: {saif_power['matched']}", "Captured tests do not initialize every memory location; Vivado reports X-state warnings for untouched storage.", "These are tool estimates for the routed design, not measurements from a PYNQ-Z2 power rail.",]‘ to ‘power_lines ‘ in host memory. Here power_lines means the value named ’power lines’ in this block. The important operands are f: the value named ’f’ in this block; Vectorless: the value named ’vectorless’ in this block; total: the value named ’total’ in this block. Context: the function generate, which handles generate.

Line 1065: `figure = text_page("Power Evidence and Confidence Boundary", power_lines)` This assigns ‘text_page("Po Evidence and Confidence Boundary", power_lines)‘ to ‘figure ‘ in host memory. Here figure means the value named ’figure’ in this block. The important operands are text_page: the value named ’text page’ in this block; Power: the value named ’power’ in this block; Evidence: the value named ’evidence’ in this block. Context: the function generate, which handles generate.

Line 1066: `index = save_figure(` This statement continues through source line 1069; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "power_confidence", "Power confidence and annotation coverage", str(saif_power_report),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 1067: `figure, plot_dir, pdf, records, index, "power_confidence",` This is continuation line 2 of the 4-line logical statement that begins at source line 1066. Its exact fragment is ‘figure, plot_dir, pdf, records, index, "power_confidence",‘. The important operands are figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block; pdf: the value named ’pdf’ in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "power_confidence", "Power confidence and annotation coverage", str(saif_power_report),)‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named ’save figure’ in this block; figure: the value named ’figure’ in this block; plot_dir: the value named ’plot dir’ in this block. Context: the function generate, which handles generate.

Line 1068: `"Power confidence and annotation coverage", str(saif_power_report),` This is continuation line 3 of the 4-line logical statement that begins at source line 1066. Its exact fragment is '"Power confidence and annotation coverage", str(saif_power_report),'. The important operands are Power: the value named 'power' in this block; confidence: the value named 'confidence' in this block; and: the value named 'and' in this block. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "power_confidence", "Power confidence and annotation coverage", str(saif_power_report),)' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1069: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1066. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'save_figure(figure, plot_dir, pdf, records, index, "power_confidence", "Power confidence and annotation coverage", str(saif_power_report),)' to 'index ' in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1071: `timing_path = timing_dir / "full_routed_timing_paths_250.rpt"` This assigns 'timing_dir / "full_routed_ti
to 'timing_path ' in host memory. Here timing_path means the value named 'timing path' in this block. The important operands are timing_dir: the value named 'timing dir' in this block; full_routed_timing_paths_250: the value named 'full routed timing paths 250' in this block; rpt: the value named 'rpt' in this block. Context: the function generate, which handles generate.

Line 1072: `if not timing_path.exists():` This executes the following indented host-code branch only when 'not timing_path.exists()' is true. The important operands are not: the value named 'not' in this block; timing_path: the value named 'timing path' in this block; exists: the value named 'exists' in this block. Context: the function generate, which handles generate.

Line 1073: `timing_path = timing_dir / "routed_timing_paths_250.rpt"` This assigns 'timing_dir / "routed_timing_paths
to 'timing_path ' in host memory. Here timing_path means the value named 'timing path' in this block. The important operands are timing_dir: the value named 'timing dir' in this block; routed_timing_paths_250: the value named 'routed timing paths 250' in this block; rpt: the value named 'rpt' in this block. Context: the function generate, which handles generate.

Line 1074: `if timing_path.exists():` This executes the following indented host-code branch only when 'timing_path.exists()' is true. The important operands are timing_path: the value named 'timing path' in this block; exists: the value named 'exists' in this block. Context: the function generate, which handles generate.

Line 1075: `slacks = parse_timing_slacks(timing_path)` This assigns 'parse_timing_slacks(timing_path)' to 'slacks ' in host memory. Here slacks means the value named 'slacks' in this block. The important operands are parse_timing_slacks: the value named 'parse timing slacks' in this block; timing_path: the value named 'timing path' in this block. Context: the function generate, which handles generate.

Line 1076: `if slacks.size:` This executes the following indented host-code branch only when 'slacks.size' is true. The important operands are slacks: the value named 'slacks' in this block; size: the value named 'size' in this block. Context: the function generate, which handles generate.

Line 1077: `figure, axes = plt.subplots(1, 2, figsize=(11, 5))` This assigns 'plt.subplots(1, 2, figsize=(11, 5))' to 'figure, axes ' in host memory. Here figure means the value named 'figure' in this block. The important operands are plt: the value named 'plt' in this block; subplots: the value named 'subplots' in this block; figsize: the value named 'figsize' in this block. Context: the function generate, which handles generate.

Line 1078: `axes[0].hist(slacks, bins=min(40, max(8, slacks.size // 5)), color=COLORS[0])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; hist: the value named 'hist' in this block; slacks: the value named 'slacks' in this block.

Line 1079: `axes[0].axvline(0, color=COLORS[2], linewidth=1.2)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; axvline: the value named 'axvline' in this block; color: the value named 'color' in this block.

Line 1080: `style_axes(axes[0], "Reported routed path slack", "Setup slack (ns)", "Path count")` This calls ‘style_axes‘ with ‘axes[0], "Reported routed path slack", "Setup slack (ns)", "Path count"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1081: `axes[1].plot(np.sort(slacks), color=COLORS[4])` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; plot: the value named 'plot' in this block; sort: the value named 'sort' in this block.

Line 1082: `axes[1].axhline(0, color=COLORS[2], linewidth=1.2)` Inside the function generate, which handles generate, this statement advances the host implementation that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The important operands are axes: the value named 'axes' in this block; axhline: the value named 'axhline' in this block; color: the value named 'color' in this block.

Line 1083: `style_axes(axes[1], "Sorted critical-path slack", "Path rank", "Setup slack (ns)")` This calls ‘style_axes‘ with ‘axes[1], "Sorted critical-path slack", "Path rank", "Setup slack (ns)"‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1084: `index = save_figure(figure, plot_dir, pdf, records, index, "routed_timing", "Routed timing path distribution",` This statement continues through source line 1085; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "routed_timing", "Routed timing path distribution", str(timing_path))‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1085: `str(timing_path))` This is continuation line 2 of the 2-line logical statement that begins at source line 1084. Its exact fragment is ‘str(timing_path))‘. The important operands are str: the value named 'str' in this block; timing_path: the value named 'timing path' in this block. Read together, the complete operation is: This assigns ‘save_figure(figure, plot_dir, pdf, records, index, "routed_timing", "Routed timing path distribution", str(timing_path))‘ to ‘index ‘ in host memory. Here index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are save_figure: the value named 'save figure' in this block; figure: the value named 'figure' in this block; plot_dir: the value named 'plot dir' in this block. Context: the function generate, which handles generate.

Line 1087: `tex_path = write_report_tex(output_dir, records, block_design_filename)` This assigns ‘write_report_tex
records, block_design_filename)‘ to ‘tex_path ‘ in host memory. Here tex_path means the value named 'tex path' in this block. The important operands are write_report_tex: the value named 'write report tex' in this block; output_dir: the value named 'output dir' in this block; records: the number of descriptor records accepted since program start. Context: the function generate, which handles generate.

Line 1088: `manifest_path = output_dir / "neuraedge_debug_plot_manifest.json"` This assigns ‘output_dir / "neuraedge_debug_plot_manifest.json"‘ to ‘manifest_path ‘ in host memory. Here manifest_path means the value named ’manifest path’ in this block. The important operands are output_dir: the value named ’output dir’ in this block; neuraedge_debug_plot_manifest: the value named ’neuraedge debug plot manifest’ in this block; json: the value named ’json’ in this block. Context: the function generate, which handles generate.

Line 1089: `manifest_path.write_text(` This statement continues through source line 1103; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1090: `json.dumps(` This is continuation line 2 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘json.dumps(‘. The important operands are json: the value named ’json’ in this block; dumps: the value named ’dumps’ in this block. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1091: `{` This is continuation line 3 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘{‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1092: `"format": "neuraedge-deep-debug-v1",` This is continuation line 4 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘"format": "neuraedge-deep-debug-v1",‘. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; deep: the value named ’deep’ in this block. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1093: `"plot_count": len(records),` This is continuation line 5 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘"plot_count": len(records),‘. The important operands are plot_count: an integer count retained for control or measurable telemetry; len: the value named ’len’ in this block; records: the number of descriptor records accepted since program start. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filena }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1094: `"plots": [record.__dict__ for record in records],` This is continuation line 6 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘"plots": [record.___dict___ for record in records],‘. The important operands are plots: the value named ’plots’ in this block; record: the value named ’record’ in this block; ___dict___: the value named ’dict’ in this block. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records),

"plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1095: `"multipage_pdf": pdf_path.name,` This is continuation line 7 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘"multipage_pdf": pdf_path.name,‘. The important operands are multipage_pdf: the value named ’multipage pdf’ in this block; pdf_path: the value named ’pdf path’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1096: `"latex": tex_path.name,` This is continuation line 8 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘"latex": tex_path.name,‘. The important operands are latex: the value named ’latex’ in this block; tex_path: the value named ’tex path’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1097: `"native_vivado_block_design": block_design_filename,` This is continuation line 9 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘"native_vivado_block_design": block_design_filename,‘. The important operands are native_vivado_block_design: the value named ’native vivado block design’ in this block; block_design_filename: the value named ’block design filename’ in this block. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1098: `},` This is continuation line 10 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘},‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1099: `indent=2,` This is continuation line 11 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘indent=2,‘. The important operands are indent: the value named ’indent’ in this block. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1100: `)` This is continuation line 12 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘

inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1101: `+ "\n",` This is continuation line 13 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘+ "\n",‘. The important operands are n: the value named ’n’ in this block. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1102: `encoding="utf-8",` This is continuation line 14 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘encoding="utf-8",‘. The important operands are encoding: the value named ’encoding’ in this block; utf: the value named ’utf’ in this block. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1103: `)` This is continuation line 15 of the 15-line logical statement that begins at source line 1089. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘manifest_path.write_text‘ with ‘ json.dumps({ "format": "neuraedge-deep-debug-v1", "plot_count": len(records), "plots": [record.___dict___ for record in records], "multipage_pdf": pdf_path.name, "latex": tex_path.name, "native_vivado_block_design": block_design_filename, }, indent=2,) + "\n", encoding="utf-8", ‘ inside the function generate, which handles generate. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1104: `return pdf_path, tex_path, manifest_path` This returns ‘pdf_path, tex_path, manifest_path‘ to the caller of the function generate, which handles generate, ending this invocation. The important operands are pdf_path: the value named ’pdf path’ in this block; tex_path: the value named ’tex path’ in this block; manifest_path: the value named ’manifest path’ in this block.

Line 1107: `def compile_tex(tex_path: Path, pdflatex: str) -> Path:` This starts function compile_tex. Its indented body performs compile tex as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 1108: `for _ in range(2):` This iterates _ over ‘range(2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_tex, which handles compile tex.

Line 1109: `subprocess.run([pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name],` This statement continues through source line 1110; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘subprocess.run‘ with ‘[pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name], cwd=tex_path.parent, check=True‘ inside the function compile_tex, which handles compile tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1110: `cwd=tex_path.parent, check=True)` This is continuation line 2 of the 2-line logical statement that begins at source line 1109. Its exact fragment is ‘cwd=tex_path.parent, check=True)‘. The important operands are cwd: the value named ’cwd’ in this block; tex_path: the value named ’tex path’ in this block; parent: the value named ’parent’ in this block. Read together, the complete operation is: This calls ‘subprocess.run‘ with ‘[pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name], cwd=tex_path.parent, check=True‘ inside the function compile_tex, which handles compile tex. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1111: `return tex_path.with_suffix(".pdf")` This returns ‘tex_path.with_suffix(".pdf")‘ to the caller of the function compile_tex, which handles compile tex, ending this invocation. The important operands are tex_path: the value named ’tex path’ in this block; with_suffix: the value named ’with suffix’ in this block; pdf: the value named ’pdf’ in this block.

Line 1114: `def main() -> None:` This starts function main. Its indented body performs main as part of the file that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs; it runs only when called.

Line 1115: `parser = argparse.ArgumentParser(description=__doc__)` This assigns ‘argparse.ArgumentParser(description= to ‘parser ‘ in host memory. Here parser means the value named ’parser’ in this block. The important operands are argparse: the value named ’argparse’ in this block; ArgumentParser: the value named ’argument parser’ in this block; description: the value named ’description’ in this block. Context: the function main, which handles main.

Line 1116: `parser.add_argument("-output-dir", type=Path, default=root_dir() / "outputs/deep_debug")` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 1117: `parser.add_argument("-timing-dir", type=Path)` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 1118: `parser.add_argument("-waveform-dir", type=Path)` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 1119: `parser.add_argument("-saif-power-report", type=Path)` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 1120: `parser.add_argument("-block-design", type=Path)` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 1121: `parser.add_argument("-compile", action="store_true")` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 1122: `parser.add_argument("-pdflatex", default="pdflatex")` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 1123: `args = parser.parse_args()` This assigns ‘parser.parse_args()‘ to ‘args ‘ in host memory. Here args means the value named ’args’ in this block. The important operands are parser: the value named ’parser’ in this block; parse_args: the value named ’parse args’ in this block. Context: the function main, which handles main.

Line 1124: `pdf_path, tex_path, manifest_path = generate(` This statement continues through source line 1130; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘generate(args.output_dir.resolv args.timing_dir.resolve() if args.timing_dir else None, args.waveform_dir.resolve() if args.waveform_dir else None, args.saif_power_ if args.saif_power_report else None, args.block_design.resolve() if args.block_design else None,)‘ to ‘pdf_path, tex_path, manifest_path ‘ in host memory. Here pdf_path means the value named ’pdf path’ in this block. The important operands are generate: the value named ’generate’ in this block; args: the value named ’args’ in this block; output_dir: the value named ’output dir’ in this block. Context: the function main, which handles main.

Line 1125: `args.output_dir.resolve(),` This is continuation line 2 of the 7-line logical statement that begins at source line 1124. Its exact fragment is 'args.output_dir.resolve(),'. The important operands are args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block; resolve: the value named 'resolve' in this block. Read together, the complete operation is: This assigns 'generate(args.output_dir.resolve(), args.timing_dir.resolve() if args.timing_dir else None, args.waveform_dir.resolve() if args.waveform_dir else None, args.saif_power_report.resolve() if args.saif_power_report else None, args.block_design.resolve() if args.block_design else None,)' to 'pdf_path, tex_path, manifest_path ' in host memory. Here pdf_path means the value named 'pdf path' in this block. The important operands are generate: the value named 'generate' in this block; args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block. Context: the function main, which handles main.

Line 1126: `args.timing_dir.resolve() if args.timing_dir else None,` This is continuation line 3 of the 7-line logical statement that begins at source line 1124. Its exact fragment is 'args.timing_dir.resolve() if args.timing_dir else None,'. The important operands are args: the value named 'args' in this block; timing_dir: the value named 'timing dir' in this block; resolve: the value named 'resolve' in this block. Read together, the complete operation is: This assigns 'generate(args.output_dir.resolve(), args.timing_dir.resolve() if args.timing_dir else None, args.waveform_dir.resolve() if args.waveform_dir else None, args.saif_power_report.resolve() if args.saif_power_report else None, args.block_design.resolve() if args.block_design else None,)' to 'pdf_path, tex_path, manifest_path ' in host memory. Here pdf_path means the value named 'pdf path' in this block. The important operands are generate: the value named 'generate' in this block; args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block. Context: the function main, which handles main.

Line 1127: `args.waveform_dir.resolve() if args.waveform_dir else None,` This is continuation line 4 of the 7-line logical statement that begins at source line 1124. Its exact fragment is 'args.waveform_dir.resolve() if args.waveform_dir else None,'. The important operands are args: the value named 'args' in this block; waveform_dir: the value named 'waveform dir' in this block; resolve: the value named 'resolve' in this block. Read together, the complete operation is: This assigns 'generate(args.output_dir.resolve(), args.timing_dir.resolve() if args.timing_dir else None, args.waveform_dir.resolve() if args.waveform_dir else None, args.saif_power_report.resolve() if args.saif_power_report else None, args.block_design.resolve() if args.block_design else None,)' to 'pdf_path, tex_path, manifest_path ' in host memory. Here pdf_path means the value named 'pdf path' in this block. The important operands are generate: the value named 'generate' in this block; args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block. Context: the function main, which handles main.

Line 1128: `args.saif_power_report.resolve() if args.saif_power_report else None,` This is continuation line 5 of the 7-line logical statement that begins at source line 1124. Its exact fragment is 'args.saif_power_report.resolve() if args.saif_power_report else None,'. The important operands are args: the value named 'args' in this block; saif_power_report: the value named 'saif power report' in this block; resolve: the value named 'resolve' in this block. Read together, the complete operation is: This assigns 'generate(args.output_dir.resolve(), args.timing_dir.resolve() if args.timing_dir else None, args.waveform_dir.resolve() if args.waveform_dir else None, args.saif_power_report.resolve() if args.saif_power_report else None, args.block_design.resolve() if args.block_design else None,)' to 'pdf_path, tex_path, manifest_path ' in host memory. Here pdf_path means the value named 'pdf path' in this block. The important operands are generate: the value named 'generate' in this block; args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block. Context: the function main, which handles main.

Line 1129: `args.block_design.resolve() if args.block_design else None,` This is continuation line 6 of the 7-line logical statement that begins at source line 1124. Its exact fragment is 'args.block_design.resolve() if args.block_design else None,'. The important operands are args: the value named 'args' in this block; block_design: the value named 'block design' in this block; resolve: the value named 'resolve' in this block. Read together, the complete operation is: This assigns 'generate(args.output_dir.resolve(), args.timing_dir.resolve() if args.timing_dir else None, args.waveform_dir.resolve() if args.waveform_dir else None, args.saif_power_report.resolve() if args.saif_power_report else None, args.block_design.resolve() if args.block_design else None,)' to 'pdf_path, tex_path, manifest_path ' in host memory. Here pdf_path means the value named 'pdf path' in this block. The important operands are generate: the value named 'generate' in this block; args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block. Context: the function main, which handles main.

Line 1130: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1124. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'generate(args.output_dir.resolve(), args.timing_dir.resolve() if args.timing_dir else None,

args.waveform_dir.resolve() if args.waveform_dir else None, args.saif_power_report.resolve() if args.saif_power_report else None, args.block_design.resolve() if args.block_design else None,)‘ to ‘pdf_path, tex_path, manifest_path ‘ in host memory. Here pdf_path means the value named ’pdf path’ in this block. The important operands are generate: the value named ’generate’ in this block; args: the value named ’args’ in this block; output_dir: the value named ’output dir’ in this block. Context: the function main, which handles main.

Line 1131: `print(f"plots_pdf={pdf_path}")` This calls ‘print‘ with ‘f"plots_pdf={pdf_path}"‘ inside the function main, which handles main. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1132: `print(f"latex={tex_path}")` This calls ‘print‘ with ‘f"latex={tex_path}"‘ inside the function main, which handles main. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1133: `print(f"manifest={manifest_path}")` This calls ‘print‘ with ‘f"manifest={manifest_path}"‘ inside the function main, which handles main. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1134: `if args.compile:` This executes the following indented host-code branch only when ‘args.compile‘ is true. The important operands are args: the value named ’args’ in this block; compile: the value named ’compile’ in this block. Context: the function main, which handles main.

Line 1135: `print(f"report_pdf={compile_tex(tex_path, args.pdflatex)}")` This calls ‘print‘ with ‘f"report_pdf={compile
args.pdflatex)}"‘ inside the function main, which handles main. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1138: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

Line 1139: `main()` This calls ‘main‘ with “ inside the file whose role is to turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs. The call advances the host-side job that turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs.

29 software/generate_operator_fixtures.py

Role in the complete system: builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

29.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: none. Functions: pack_signed, write_hex, generate_vector_cases, generate_optimizer_cases, generate_layernorm_cases, generate_softmax_cases, main. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

29.2 Complete source

```
"""Generate deterministic cross-language fixtures for vector and optimizer RTL."""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import numpy as np

from math import isqrt

from neuraedge_golden import (
    VectorOperation,
    integer_layer_norm,
    log_int_softmax_q15,
    sgd_momentum_step,
    vector_alu,
)


def pack_signed(values: np.ndarray, bits: int) -> int:
    packed = 0
    mask = (1 << bits) - 1
    for lane, value in enumerate(np.asarray(values, dtype=np.int64).reshape(-1)):
        packed |= (int(value) & mask) << (lane * bits)
    return packed


def write_hex(path: Path, values: list[int], width_bits: int) -> None:
    digits = (width_bits + 3) // 4
    path.write_text("".join(f"{value & ((1 << width_bits) - 1):0{digits}x}\n" for value in values))


def generate_vector_cases(rng: np.random.Generator, count_per_operation: int) -> dict[str, list[int]]:
    commands: list[int] = []
    operands_a: list[int] = []
    operands_b: list[int] = []
    expected: list[int] = []
    boundaries = np.array(
        [-(1 << 31), -32768, -129, -1, 0, 1, 32767, (1 << 31) - 1], dtype=np.int64
    )

    for operation in VectorOperation:
        for case_index in range(count_per_operation):
            if case_index == 0:
                a = boundaries.copy()
                b = boundaries[::-1].copy()
            else:
                a = rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)
                b = rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)

```

```
            multiplier = int(rng.integers(-32768, 32768))
            if operation == VectorOperation.FLEX_REQUANTIZE:
                right_shift = int(rng.integers(-15, 16))
            else:
                right_shift = int(rng.integers(0, 8))
            zero_point = int(rng.integers(-128, 128))
            threshold = int(rng.integers(1, 4097))
            window = int(rng.integers(0, 4097))
            clamp_minimum = int(rng.integers(-32768, 0))
            clamp_maximum = int(rng.integers(0, 32768))
            result = vector_alu(
                operation,
                a,
                b,
                multiplier_q15=multiplier,
                right_shift=right_shift,
                output_zero_point=zero_point,
                threshold=threshold,
                surrogate_window=window,
                clamp_minimum=clamp_minimum,
                clamp_maximum=clamp_maximum,
            )

            command = 0
            command |= int(operation)
            command |= (multiplier & 0xFFFF) << 4
            command |= (right_shift & 0x1F) << 20
            command |= (zero_point & 0xFFFF) << 25
            command |= (threshold & 0xFFFFFFFF) << 41
            command |= window << 73
            command |= (clamp_minimum & 0xFFFFFFFF) << 105
            command |= (clamp_maximum & 0xFFFFFFFF) << 137
            commands.append(command)
            operands_a.append(pack_signed(a, 32))
            operands_b.append(pack_signed(b, 32))
            expected.append(pack_signed(result, 32))
    return {"commands": commands, "a": operands_a, "b": operands_b, "expected": expected}


def generate_optimizer_cases(rng: np.random.Generator, case_count: int) -> dict[str, list[int]]:
    parameters: list[int] = []
    weights: list[int] = []
    gradients: list[int] = []
    velocities: list[int] = []
    expected_weights: list[int] = []
    expected_velocities: list[int] = []

    for case_index in range(case_count):
        if case_index == 0:
            weight = np.array([-32768, -32767, -1, 0, 1, 32766, 32767, 123], dtype=np.int64)
            gradient = np.array([-(1 << 23), -(1 << 20), -1, 0, 1, 1 << 20, (1 << 23) - 1, -999])
            velocity = gradient[::-1].copy()
        elif case_index == 1:
            # This case distinguishes SGDW from coupled L2: decay changes the
            # weight direction but must not be retained in the velocity.
            weight = np.full(8, 1000, dtype=np.int64)
            gradient = np.zeros(8, dtype=np.int64)
            velocity = np.full(8, 1000, dtype=np.int64)
        else:
            weight = rng.integers(-32768, 32768, size=8, dtype=np.int64)
            gradient = rng.integers(-(1 << 23), 1 << 23, size=8, dtype=np.int64)
            velocity = rng.integers(-(1 << 23), 1 << 23, size=8, dtype=np.int64)
        if case_index == 1:
            learning_rate = 65536
            momentum = 65536
            weight_decay = 65536
            update_right_shift = 16
        else:
            learning_rate = int(rng.integers(1, 131072))
            momentum = int(rng.integers(0, 65537))
            weight_decay = int(rng.integers(0, 65537))
            update_right_shift = 16 if case_index == 0 else int(rng.integers(0, 32))
        next_weight, next_velocity = sgd_momentum_step(
            weight,
            gradient,
            velocity,
            learning_rate,
            momentum,
            weight_decay,
            update_right_shift,
        )
        parameters.append(
            learning_rate
            | (momentum << 18)
            | (weight_decay << 35)
            | (update_right_shift << 52)
        )
```

```
        weights.append(pack_signed(weight, 16))
        gradients.append(pack_signed(gradient, 24))
        velocities.append(pack_signed(velocity, 24))
        expected_weights.append(pack_signed(next_weight, 16))
        expected_velocities.append(pack_signed(next_velocity, 24))
    return {
        "parameters": parameters,
        "weights": weights,
        "gradients": gradients,
        "velocities": velocities,
        "expected_weights": expected_weights,
        "expected_velocities": expected_velocities,
    }


def generate_layernorm_cases(rng: np.random.Generator) -> dict[str, list[int]]:
    descriptors: list[int] = []
    values: list[int] = []
    gammas: list[int] = []
    betas: list[int] = []
    expected: list[int] = []
    debug: list[int] = []
    lengths = [1, 4, 7, 32, 64, 127, 384, 1536]

    for case_index, length in enumerate(lengths):
        epsilon = 1 + case_index * 3
        if case_index == 0:
            x = np.array([17], dtype=np.int64)
        elif case_index == 1:
            x = np.full(length, -91, dtype=np.int64)
        else:
            x = rng.integers(-8192, 8193, size=length, dtype=np.int64)
        gamma = rng.integers(8192, 49153, size=length, dtype=np.int64)
        beta = rng.integers(-2048, 2049, size=length, dtype=np.int64)
        output = integer_layer_norm(x, gamma, beta, epsilon)
        mean = int(x.sum() // length)
        centered = x - mean
        variance = int((centered * centered).sum() // length)
        sigma = max(1, isqrt(variance + epsilon))
        reciprocal = ((1 << 31) - 1) // sigma

        descriptors.append(length | (epsilon << 12))
        values.extend(int(value) for value in x)
        gammas.extend(int(value) for value in gamma)
        betas.extend(int(value) for value in beta)
        expected.extend(int(value) for value in output)
        debug.append(
            (mean & 0xFFFFFFFF)
            | (variance << 32)
            | (sigma << 64)
            | (reciprocal << 96)
        )
    return {
        "descriptors": descriptors,
        "values": values,
        "gammas": gammas,
        "betas": betas,
        "expected": expected,
        "debug": debug,
        "lengths": lengths,
    }


def generate_softmax_cases(rng: np.random.Generator) -> dict[str, list[int]]:
    descriptors: list[int] = []
    scores: list[int] = []
    expected: list[int] = []
    debug: list[int] = []
    lengths = [1, 4, 7, 32, 64, 127, 256]

    for case_index, length in enumerate(lengths):
        difference_shift = case_index % 8
        if case_index == 0:
            score = np.array([123], dtype=np.int64)
        else:
            score = rng.integers(-(1 << 18), 1 << 18, size=length, dtype=np.int64)
        probability = log_int_softmax_q15(score, difference_shift)
        difference = score.max() - score
        exponent = np.minimum(difference >> difference_shift, 15)
        unnormalized = 1 << (15 - exponent)
        denominator = int(unnormalized.sum())
        reciprocal = ((1 << 31) - 1) // denominator

        descriptors.append(length | (difference_shift << 9))
        scores.extend(int(value) for value in score)
        expected.extend(int(value) for value in probability)
        debug.append((int(score.max()) & 0xFFFFFF) | (denominator << 24) | (reciprocal << 48))
```

```
    return {
        "descriptors": descriptors,
        "scores": scores,
        "expected": expected,
        "debug": debug,
        "lengths": lengths,
    }


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--output-dir", type=Path, default=Path("tb/fixtures"))
    parser.add_argument("--vector-count-per-operation", type=int, default=24)
    parser.add_argument("--optimizer-cases", type=int, default=128)
    args = parser.parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(20260711)

    vector = generate_vector_cases(rng, args.vector_count_per_operation)
    optimizer = generate_optimizer_cases(rng, args.optimizer_cases)
    layernorm = generate_layernorm_cases(rng)
    softmax = generate_softmax_cases(rng)
    write_hex(args.output_dir / "vector_commands.mem", vector["commands"], 192)
    write_hex(args.output_dir / "vector_a.mem", vector["a"], 256)
    write_hex(args.output_dir / "vector_b.mem", vector["b"], 256)
    write_hex(args.output_dir / "vector_expected.mem", vector["expected"], 256)
    write_hex(args.output_dir / "optimizer_parameters.mem", optimizer["parameters"], 64)
    write_hex(args.output_dir / "optimizer_weights.mem", optimizer["weights"], 128)
    write_hex(args.output_dir / "optimizer_gradients.mem", optimizer["gradients"], 192)
    write_hex(args.output_dir / "optimizer_velocities.mem", optimizer["velocities"], 192)
    write_hex(args.output_dir / "optimizer_expected_weights.mem", optimizer["expected_weights"], 128)
    write_hex(args.output_dir / "optimizer_expected_velocities.mem", optimizer["expected_velocities"], 192)
    write_hex(args.output_dir / "layernorm_cases.mem", layernorm["descriptors"], 64)
    write_hex(args.output_dir / "layernorm_values.mem", layernorm["values"], 24)
    write_hex(args.output_dir / "layernorm_gammas.mem", layernorm["gammas"], 24)
    write_hex(args.output_dir / "layernorm_betas.mem", layernorm["betas"], 24)
    write_hex(args.output_dir / "layernorm_expected.mem", layernorm["expected"], 32)
    write_hex(args.output_dir / "layernorm_debug.mem", layernorm["debug"], 128)
    write_hex(args.output_dir / "softmax_cases.mem", softmax["descriptors"], 32)
    write_hex(args.output_dir / "softmax_scores.mem", softmax["scores"], 24)
    write_hex(args.output_dir / "softmax_expected.mem", softmax["expected"], 16)
    write_hex(args.output_dir / "softmax_debug.mem", softmax["debug"], 80)

    manifest = {
        "seed": 20260711,
        "vector_cases": len(vector["commands"]),
        "vector_cases_per_operation": args.vector_count_per_operation,
        "optimizer_cases": len(optimizer["parameters"]),
        "layernorm_cases": len(layernorm["lengths"]),
        "layernorm_samples": len(layernorm["values"]),
        "softmax_cases": len(softmax["lengths"]),
        "softmax_samples": len(softmax["scores"]),
        "rounding": "nearest, ties away from zero",
    }
    (args.output_dir / "operator_fixture_manifest.json").write_text(
        json.dumps(manifest, indent=2) + "\n"
    )
    (args.output_dir / "operator_fixture_params.svh").write_text(
        f"`define NEURAEDGE_VECTOR_CASES {manifest['vector_cases']}\n"
        f"`define NEURAEDGE_OPTIMIZER_CASES {manifest['optimizer_cases']}\n"
        f"`define NEURAEDGE_LAYERNORM_CASES {manifest['layernorm_cases']}\n"
        f"`define NEURAEDGE_LAYERNORM_SAMPLES {manifest['layernorm_samples']}\n"
        f"`define NEURAEDGE_SOFTMAX_CASES {manifest['softmax_cases']}\n"
        f"`define NEURAEDGE_SOFTMAX_SAMPLES {manifest['softmax_samples']}\n"
    )


if __name__ == "__main__":
    main()
```

29.3 Line annotations

Line 1: `"""Generate deterministic cross-language fixtures for vector and optimizer RTL."""` This is executable documentation attached to the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import json` This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from pathlib import Path` This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import numpy as np` This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `from math import isqrt` This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `from neuraedge_golden import (` This statement continues through source line 19; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `VectorOperation,` This is continuation line 2 of the 7-line logical statement that begins at source line 13. Its exact fragment is 'VectorOperation,'. The important operands are VectorOperation: the value named 'vector operation' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `integer_layer_norm,` This is continuation line 3 of the 7-line logical statement that begins at source line 13. Its exact fragment is 'integer_layer_norm,'. The important operands are integer_layer_norm: the value named 'integer layer norm' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `log_int_softmax_q15,` This is continuation line 4 of the 7-line logical statement that begins at source line 13. Its exact fragment is 'log_int_softmax_q15,'. The important operands are log_int_softmax_q15: the value named 'log int softmax q15' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `sgd_momentum_step,` This is continuation line 5 of the 7-line logical statement that begins at source line 13. Its exact fragment is 'sgd_momentum_step,'. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `vector_alu,` This is continuation line 6 of the 7-line logical statement that begins at source line 13. Its exact fragment is 'vector_alu,'. The important operands are vector_alu: the value named 'vector alu' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 13. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `def pack_signed(values: np.ndarray, bits: int) -> int:` This starts function pack_signed. Its indented body performs pack signed as part of the file that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests; it runs only when called.

Line 23: `packed = 0` This assigns '0' to 'packed ' in host memory. Here packed means the value named 'packed' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_signed, which handles pack signed.

Line 24: `mask = (1 << bits) - 1` This assigns '(1 << bits) - 1' to 'mask ' in host memory. Here mask means the value named 'mask' in this block. The important operands are bits: the value named 'bits' in this block. Context: the function pack_signed, which handles pack signed.

Line 25: `for lane, value in enumerate(np.asarray(values, dtype=np.int64).reshape(-1)):` This iterates lane, value over 'enumerate(np.asarray(values, dtype=np.int64).reshape(-1))'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_signed, which handles pack signed.

Line 26: `packed |= (int(value) & mask) << (lane * bits)` Inside the function pack_signed, which handles pack signed, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are packed: the value named 'packed' in this block; value: the value named 'value' in this block; mask: the value named 'mask' in this block.

Line 27: `return packed` This returns 'packed' to the caller of the function pack_signed, which handles pack signed, ending this invocation. The important operands are packed: the value named 'packed' in this block.

Line 30: `def write_hex(path: Path, values: list[int], width_bits: int) -> None:` This starts function write_hex. Its indented body performs write hex as part of the file that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests; it runs only when called.

Line 31: `digits = (width_bits + 3) // 4` This assigns '(width_bits + 3) // 4' to 'digits ' in host memory. Here digits means the value named 'digits' in this block. The important operands are width_bits: the value named 'width bits' in this block. Context: the function write_hex, which handles write hex.

Line 32: `path.write_text("".join(f"{value & ((1 << width_bits) - 1):0{digits}x}\n" for value in values))` This calls 'path.write_text' with '"".join(f"{value & ((1 << width_bits) - 1):0{digits}x}\n" for value in values)' inside the function write_hex, which handles write hex. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 35: `def generate_vector_cases(rng: np.random.Generator, count_per_operation: int) -> dict[str, list[int]]:` This starts function generate_vector_cases. Its indented body performs generate vector cases as part of the file that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests; it runs only when called.

Line 36: `commands: list[int] = []` This assigns '[]' to 'commands' in host memory. Here commands means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_vector_cases, which handles generate vector cases.

Line 37: `operands_a: list[int] = []` This assigns '[]' to 'operands_a' in host memory. Here operands_a means the value named 'operands a' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_vector_cases, which handles generate vector cases.

Line 38: `operands_b: list[int] = []` This assigns '[]' to 'operands_b' in host memory. Here operands_b means the value named 'operands b' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_vector_cases, which handles generate vector cases.

Line 39: `expected: list[int] = []` This assigns '[]' to 'expected' in host memory. Here expected means the value named 'expected' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_vector_cases, which handles generate vector cases.

Line 40: `boundaries = np.array(` This statement continues through source line 42; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.array([-(1 << 31), -32768, -129, -1, 0, 1, 32767, (1 << 31) - 1], dtype=np.int64)' to 'boundaries ' in host memory. Here boundaries means the value named 'boundaries' in this block. The important operands are array: the value named 'array' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 41: `[-(1 << 31), -32768, -129, -1, 0, 1, 32767, (1 << 31) - 1], dtype=np.int64` This is continuation line 2 of the 3-line logical statement that begins at source line 40. Its exact fragment is '[-(1 << 31), -32768, -129, -1, 0, 1, 32767, (1 << 31) - 1], dtype=np.int64'. The important operands are dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'np.array([-(1 << 31), -32768, -129, -1, 0, 1, 32767, (1 << 31) - 1], dtype=np.int64)' to 'boundaries ' in host memory. Here boundaries means the value named 'boundaries' in this block. The important operands are array: the value named 'array' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 42: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 40. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.array([-(1 << 31), -32768, -129, -1, 0, 1, 32767, (1 << 31) - 1], dtype=np.int64)' to 'boundaries ' in host memory. Here boundaries means the value named 'boundaries' in this block. The important operands are array: the value named 'array' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 44: `for operation in VectorOperation:` This iterates operation over 'VectorOperation'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate_vector_cases, which handles generate vector cases.

Line 45: `for case_index in range(count_per_operation):` This iterates case_index over 'range(count_per_operation)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate_vector_cases, which handles generate vector cases.

Line 46: `if case_index == 0:` This executes the following indented host-code branch only when 'case_index == 0' is true. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function generate_vector_cases, which handles generate vector cases.

Line 47: `a = boundaries.copy()` This assigns 'boundaries.copy()' to 'a ' in host memory. Here a means the value named 'a' in this block. The important operands are boundaries: the value named 'boundaries' in this block; copy: the value named 'copy' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 48: `b = boundaries[::-1].copy()` This assigns 'boundaries[::-1].copy()' to 'b ' in host memory. Here b means the value named 'b' in this block. The important operands are boundaries: the value named 'boundaries' in this block; copy: the value named 'copy' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 49: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function generate_vector_cases, which handles generate vector cases.

Line 50: `a = rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)` This assigns ‘rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)‘ to ‘a ‘ in host memory. Here a means the value named ’a’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 51: `b = rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)` This assigns ‘rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)‘ to ‘b ‘ in host memory. Here b means the value named ’b’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 53: `multiplier = int(rng.integers(-32768, 32768))` This assigns ‘int(rng.integers(-32768, 32768))‘ to ‘multiplier ‘ in host memory. Here multiplier means the value named ’multiplier’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 54: `if operation == VectorOperation.FLEX_REQUANTIZE:` This executes the following indented host-code branch only when ‘operation == VectorOperation.FLEX_REQUANTIZE‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function generate_vector_cases, which handles generate vector cases.

Line 55: `right_shift = int(rng.integers(-15, 16))` This assigns ‘int(rng.integers(-15, 16))‘ to ‘right_shift ‘ in host memory. Here right_shift means the value named ’right shift’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 56: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function generate_vector_cases, which handles generate vector cases.

Line 57: `right_shift = int(rng.integers(0, 8))` This assigns ‘int(rng.integers(0, 8))‘ to ‘right_shift ‘ in host memory. Here right_shift means the value named ’right shift’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 58: `zero_point = int(rng.integers(-128, 128))` This assigns ‘int(rng.integers(-128, 128))‘ to ‘zero_point ‘ in host memory. Here zero_point means the integer code representing real numeric zero in affine quantization. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 59: `threshold = int(rng.integers(1, 4097))` This assigns ‘int(rng.integers(1, 4097))‘ to ‘threshold ‘ in host memory. Here threshold means the membrane level at which a neuron emits a spike. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 60: `window = int(rng.integers(0, 4097))` This assigns ‘int(rng.integers(0, 4097))‘ to ‘window ‘ in host memory. Here window means the value named ’window’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 61: `clamp_minimum = int(rng.integers(-32768, 0))` This assigns ‘int(rng.integers(-32768, 0))‘ to ‘clamp_minimum ‘ in host memory. Here clamp_minimum means the value named ’clamp minimum’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 62: `clamp_maximum = int(rng.integers(0, 32768))` This assigns ‘int(rng.integers(0, 32768))‘ to ‘clamp_maximum ‘ in host memory. Here clamp_maximum means the value named ’clamp maximum’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 63: `result = vector_alu(` This statement continues through source line 74; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=cla clamp_maximum=clamp_maximum,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; a: the value named ’a’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 64: `operation,` This is continuation line 2 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘operation,‘. The important operands are operation: the value named ’operation’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; a: the value named ’a’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 65: `a,` This is continuation line 3 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘a,‘. The important operands are a: the value named ’a’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; a: the value named ’a’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 66: `b,` This is continuation line 4 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘b,‘. The important operands are b: the value named ’b’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; a: the value named ’a’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 67: `multiplier_q15=multiplier,` This is continuation line 5 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘multiplier_q15=multiplier,‘. The important operands are multiplier_q15: the value named ’multiplier q15’ in this block; multiplier: the value named ’multiplier’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maxi)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; a: the value named ’a’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 68: `right_shift=right_shift,` This is continuation line 6 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘right_shift=right_shift,‘. The important operands are right_shift: the value named ’right shift’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation,

a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘result ‘ in host memory. Here result means the value named 'result' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; a: the value named 'a' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 69: `output_zero_point=zero_point,` This is continuation line 7 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘output_zero_point=zero_point,‘. The important operands are output_zero_point: the integer code representing real numeric zero in affine quantization; zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘result ‘ in host memory. Here result means the value named 'result' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; a: the value named 'a' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 70: `threshold=threshold,` This is continuation line 8 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘threshold=threshold,‘. The important operands are threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘result ‘ in host memory. Here result means the value named 'result' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; a: the value named 'a' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 71: `surrogate_window=window,` This is continuation line 9 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘surrogate_window=window,‘. The important operands are surrogate_window: the value named 'surrogate window' in this block; window: the value named 'window' in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maxi
)‘ to ‘result ‘ in host memory. Here result means the value named 'result' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; a: the value named 'a' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 72: `clamp_minimum=clamp_minimum,` This is continuation line 10 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘clamp_minimum=clamp_minimum,‘. The important operands are clamp_minimum: the value named 'clamp minimum' in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘result ‘ in host memory. Here result means the value named 'result' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; a: the value named 'a' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 73: `clamp_maximum=clamp_maximum,` This is continuation line 11 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘clamp_maximum=clamp_maximum,‘. The important operands are clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘result ‘ in host memory. Here result means the value named 'result' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; a: the value named 'a' in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 74: `)` This is continuation line 12 of the 12-line logical statement that begins at source line 63. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘vector_alu(operation, a, b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=zero_point, threshold=threshold, surrogate_window=window, clamp_minimum=clamp_minimum, clamp_maxi

)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; a: the value named ’a’ in this block. Context: the function generate_vector_cases, which handles generate vector cases.

Line 76: `command = 0` This assigns ‘0‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_vector_cases, which handles generate vector cases.

Line 77: `command |= int(operation)` Inside the function generate_vector_cases, which handles generate vector cases, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named ’operation’ in this block.

Line 78: `command |= (multiplier & 0xFFFF) << 4` Inside the function generate_vector_cases, which handles generate vector cases, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; multiplier: the value named ’multiplier’ in this block.

Line 79: `command |= (right_shift & 0x1F) << 20` Inside the function generate_vector_cases, which handles generate vector cases, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; right_shift: the value named ’right shift’ in this block.

Line 80: `command |= (zero_point & 0xFFFF) << 25` Inside the function generate_vector_cases, which handles generate vector cases, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; zero_point: the integer code representing real numeric zero in affine quantization.

Line 81: `command |= (threshold & 0xFFFFFFFF) << 41` Inside the function generate_vector_cases, which handles generate vector cases, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; threshold: the membrane level at which a neuron emits a spike.

Line 82: `command |= window << 73` Inside the function generate_vector_cases, which handles generate vector cases, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; window: the value named ’window’ in this block.

Line 83: `command |= (clamp_minimum & 0xFFFFFFFF) << 105` Inside the function generate_vector_cases, which handles generate vector cases, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; clamp_minimum: the value named ’clamp minimum’ in this block.

Line 84: `command |= (clamp_maximum & 0xFFFFFFFF) << 137` Inside the function generate_vector_cases, which handles generate vector cases, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; clamp_maximum: the value named ’clamp maximum’ in this block.

Line 85: `commands.append(command)` This calls ‘commands.append‘ with ‘command‘ inside the function generate_vector_cases, which handles generate vector cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 86: `operands_a.append(pack_signed(a, 32))` This calls ‘operands_a.append‘ with ‘pack_signed(a, 32)‘ inside the function generate_vector_cases, which handles generate vector cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 87: `operands_b.append(pack_signed(b, 32))` This calls ‘operands_b.append‘ with ‘pack_signed(b, 32)‘ inside the function generate_vector_cases, which handles generate vector cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 88: `expected.append(pack_signed(result, 32))` This calls ‘expected.append‘ with ‘pack_signed(result, 32)‘ inside the function generate_vector_cases, which handles generate vector cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 89: `return {"commands": commands, "a": operands_a, "b": operands_b, "expected": expected}` This returns ‘{"commands": commands, "a": operands_a, "b": operands_b, "expected": expected}‘ to the caller of the function generate_vector_cases, which handles generate vector cases, ending this invocation. The important operands are commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; a: the value named ’a’ in this block; operands_a: the value named ’operands a’ in this block.

Line 92: `def generate_optimizer_cases(rng: np.random.Generator, case_count: int) -> dict[str, list[int]]:` This starts function generate_optimizer_cases. Its indented body performs generate optimizer cases as part of the file that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests; it runs only when called.

Line 93: `parameters: list[int] = []` This assigns ‘[]‘ to ‘parameters‘ in host memory. Here parameters means the value named ’parameters’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 94: `weights: list[int] = []` This assigns ‘[]‘ to ‘weights‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 95: `gradients: list[int] = []` This assigns ‘[]‘ to ‘gradients‘ in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 96: `velocities: list[int] = []` This assigns ‘[]‘ to ‘velocities‘ in host memory. Here velocities means the value named ’velocities’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 97: `expected_weights: list[int] = []` This assigns ‘[]‘ to ‘expected_weights‘ in host memory. Here expected_weights means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 98: `expected_velocities: list[int] = []` This assigns ‘[]‘ to ‘expected_velocities‘ in host memory. Here expected_velocities means the value named ’expected velocities’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 100: `for case_index in range(case_count):` This iterates case_index over ‘range(case_count)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate_optimizer_cases, which handles generate optimizer cases.

Line 101: `if case_index == 0:` This executes the following indented host-code branch only when ‘case_index == 0‘ is true. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 102: `weight = np.array([-32768, -32767, -1, 0, 1, 32766, 32767, 123], dtype=np.int64)` This assigns ‘np.array([-32768, -32767, -1, 0, 1, 32766, 32767, 123], dtype=np.int64)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 103: `gradient = np.array([-(1 << 23), -(1 << 20), -1, 0, 1, 1 << 20, (1 << 23) - 1, -999])` This assigns ‘np.array([-(1 << 23), -(1 << 20), -1, 0, 1, 1 << 20, (1 << 23) - 1, -999])‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are array: the value named ’array’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 104: `velocity = gradient[::-1].copy()` This assigns ‘gradient[::-1].copy()‘ to ‘velocity ‘ in host memory. Here velocity means the momentum history retained by the SGD optimizer. The important operands are gradient: the signed loss derivative used to update a learned parameter; copy: the value named ’copy’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 105: `elif case_index == 1:` This executes the following indented host-code branch only when ‘case_index == 1‘ is true. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 106: `# This case distinguishes SGDW from coupled L2: decay changes the` The comment records this intent: This case distinguishes SGDW from coupled L2: decay changes the It applies inside the function generate_optimizer_cases, which handles generate optimizer cases.

Line 107: `# weight direction but must not be retained in the velocity.` The comment records this intent: weight direction but must not be retained in the velocity. It applies inside the function generate_optimizer_cases, which handles generate optimizer cases.

Line 108: `weight = np.full(8, 1000, dtype=np.int64)` This assigns ‘np.full(8, 1000, dtype=np.int64)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are full: the value named ’full’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 109: `gradient = np.zeros(8, dtype=np.int64)` This assigns ‘np.zeros(8, dtype=np.int64)‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 110: `velocity = np.full(8, 1000, dtype=np.int64)` This assigns ‘np.full(8, 1000, dtype=np.int64)‘ to ‘velocity ‘ in host memory. Here velocity means the momentum history retained by the SGD optimizer. The important operands are full: the value named ’full’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 111: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function generate_optimizer_cases, which handles generate optimizer cases.

Line 112: `weight = rng.integers(-32768, 32768, size=8, dtype=np.int64)` This assigns ‘rng.integers(-32768, 32768, size=8, dtype=np.int64)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 113: `gradient = rng.integers(-(1 << 23), 1 << 23, size=8, dtype=np.int64)` This assigns ‘rng.integers(-(1 << 23), 1 << 23, size=8, dtype=np.int64)‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 114: `velocity = rng.integers(-(1 << 23), 1 << 23, size=8, dtype=np.int64)` This assigns ‘rng.integers(-(1 << 23), 1 << 23, size=8, dtype=np.int64)‘ to ‘velocity ‘ in host memory. Here velocity means the momentum history retained by the SGD optimizer. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 115: `if case_index == 1:` This executes the following indented host-code branch only when ‘case_index == 1‘ is true. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 116: `learning_rate = 65536` This assigns ‘65536‘ to ‘learning_rate ‘ in host memory. Here learning_rate means the fixed-point multiplier controlling update magnitude. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 117: `momentum = 65536` This assigns ‘65536‘ to ‘momentum ‘ in host memory. Here momentum means the value named ’momentum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 118: `weight_decay = 65536` This assigns ‘65536‘ to ‘weight_decay ‘ in host memory. Here weight_decay means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 119: `update_right_shift = 16` This assigns ‘16‘ to ‘update_right_shift ‘ in host memory. Here update_right_shift means the value named ’update right shift’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 120: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function generate_optimizer_cases, which handles generate optimizer cases.

Line 121: `learning_rate = int(rng.integers(1, 131072))` This assigns ‘int(rng.integers(1, 131072))‘ to ‘learning_rate ‘ in host memory. Here learning_rate means the fixed-point multiplier controlling update magnitude. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 122: `momentum = int(rng.integers(0, 65537))` This assigns ‘int(rng.integers(0, 65537))‘ to ‘momentum ‘ in host memory. Here momentum means the value named ’momentum’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 123: `weight_decay = int(rng.integers(0, 65537))` This assigns ‘int(rng.integers(0, 65537))‘ to ‘weight_decay ‘ in host memory. Here weight_decay means a learned coefficient multiplied by an activation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 124: `update_right_shift = 16 if case_index == 0 else int(rng.integers(0, 32))` Inside the function generate_optimizer_cases, which handles generate optimizer cases, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are update_right_shift: the value named 'update right shift' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry; rng: the value named 'rng' in this block.

Line 125: `next_weight, next_velocity = sgd_momentum_step(` This statement continues through source line 133; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sgd_momentum_step(weight, gradient, velocity, learning_rate, momentum, weight_decay, update_right_shift,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weight: a learned coefficient multiplied by an activation; gradient: the signed loss derivative used to update a learned parameter. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 126: `weight,` This is continuation line 2 of the 9-line logical statement that begins at source line 125. Its exact fragment is 'weight,'. The important operands are weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'sgd_momentum_step(weight, gradient, velocity, learning_rate, momentum, weight_decay, update_right_shift,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weight: a learned coefficient multiplied by an activation; gradient: the signed loss derivative used to update a learned parameter. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 127: `gradient,` This is continuation line 3 of the 9-line logical statement that begins at source line 125. Its exact fragment is 'gradient,'. The important operands are gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'sgd_momentum_step(weight, gradient, velocity, learning_rate, momentum, weight_decay, update_right_shift,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weight: a learned coefficient multiplied by an activation; gradient: the signed loss derivative used to update a learned parameter. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 128: `velocity,` This is continuation line 4 of the 9-line logical statement that begins at source line 125. Its exact fragment is 'velocity,'. The important operands are velocity: the momentum history retained by the SGD optimizer. Read together, the complete operation is: This assigns 'sgd_momentum_step(weight, gradient, velocity, learning_rate, momentum, weight_decay, update_right_shift,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weight: a learned coefficient multiplied by an activation; gradient: the signed loss derivative used to update a learned parameter. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 129: `learning_rate,` This is continuation line 5 of the 9-line logical statement that begins at source line 125. Its exact fragment is 'learning_rate,'. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns 'sgd_momentum_step(weight, gradient, velocity, learning_rate, momentum, weight_decay, update_right_shift,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weight: a learned coefficient multiplied by an activation; gradient: the signed loss derivative used to update a learned parameter. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 130: `momentum,` This is continuation line 6 of the 9-line logical statement that begins at source line 125. Its exact fragment is 'momentum,'. The important operands are momentum: the value named 'momentum' in this block. Read together, the complete operation is: This assigns 'sgd_momentum_step(weight, gradient, velocity, learning_rate, momentum, weight_decay, update_right_shift,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weight: a learned coefficient multiplied by an activation; gradient: the signed

loss derivative used to update a learned parameter. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 131: `weight_decay,` This is continuation line 7 of the 9-line logical statement that begins at source line 125. Its exact fragment is 'weight_decay,'. The important operands are weight_decay: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'sgd_momentum_step(weight, gradient, velocity, learning_rate, momentum, weight_decay, update_right_shift,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weight: a learned coefficient multiplied by an activation; gradient: the signed loss derivative used to update a learned parameter. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 132: `update_right_shift,` This is continuation line 8 of the 9-line logical statement that begins at source line 125. Its exact fragment is 'update_right_shift,'. The important operands are update_right_shift: the value named 'update right shift' in this block. Read together, the complete operation is: This assigns 'sgd_momentum_step(weight, gradient, velocity, learning_rate, momentum, weight_decay, update_right_shift,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weight: a learned coefficient multiplied by an activation; gradient: the signed loss derivative used to update a learned parameter. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 133: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 125. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(weight, gradient, velocity, learning_rate, momentum, weight_decay, update_right_shift,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weight: a learned coefficient multiplied by an activation; gradient: the signed loss derivative used to update a learned parameter. Context: the function generate_optimizer_cases, which handles generate optimizer cases.

Line 134: `parameters.append(` This statement continues through source line 139; the explanation uses the complete expression rather than only the first visual fragment. This calls 'parameters.append' with ' learning_rate | (momentum << 18) | (weight_decay << 35) | (update_right_shift << 52) ' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 135: `learning_rate` This is continuation line 2 of the 6-line logical statement that begins at source line 134. Its exact fragment is 'learning_rate'. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This calls 'parameters.append' with ' learning_rate | (momentum << 18) | (weight_decay << 35) | (update_right_shift << 52) ' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 136: `| (momentum << 18)` This is continuation line 3 of the 6-line logical statement that begins at source line 134. Its exact fragment is '| (momentum << 18)'. The important operands are momentum: the value named 'momentum' in this block. Read together, the complete operation is: This calls 'parameters.append' with ' learning_rate | (momentum << 18) | (weight_decay << 35) | (update_right_shift << 52) ' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 137: `| (weight_decay << 35)` This is continuation line 4 of the 6-line logical statement that begins at source line 134. Its exact fragment is '| (weight_decay << 35)'. The important operands are weight_decay: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls 'parameters.append' with ' learning_rate | (momentum << 18) | (weight_decay << 35) | (update_right_shift << 52) ' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 138: `| (update_right_shift << 52)` This is continuation line 5 of the 6-line logical statement that begins at source line 134. Its exact fragment is '| (update_right_shift << 52)'. The important operands are update_right_shift: the value named 'update right shift' in this block. Read together, the complete operation is: This calls 'parameters.append' with ' learning_rate | (momentum << 18) | (weight_decay << 35) | (update_right_shift << 52) ' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 139: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 134. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'parameters.append' with ' learning_rate | (momentum << 18) | (weight_decay << 35) | (update_right_shift << 52) ' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 140: `weights.append(pack_signed(weight, 16))` This calls 'weights.append' with 'pack_signed(weight, 16)' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 141: `gradients.append(pack_signed(gradient, 24))` This calls 'gradients.append' with 'pack_signed(gradient, 24)' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 142: `velocities.append(pack_signed(velocity, 24))` This calls 'velocities.append' with 'pack_signed(velocity, 24)' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 143: `expected_weights.append(pack_signed(next_weight, 16))` This calls 'expected_weights.append' with 'pack_signed(next_weight, 16)' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 144: `expected_velocities.append(pack_signed(next_velocity, 24))` This calls 'expected_velocities.append' with 'pack_signed(next_velocity, 24)' inside the function generate_optimizer_cases, which handles generate optimizer cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 145: `return {` This statement continues through source line 152; the explanation uses the complete expression rather than only the first visual fragment. This returns '{ "parameters": parameters, "weights": weights, "gradients": gradients, "velocities": velocities, "expected_weights": expected_weights, "expected_velocities": expected_velocities, }' to the caller of the function generate_optimizer_cases, which handles generate optimizer cases, ending this invocation. The important operands are parameters: the value named 'parameters' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter.

Line 146: `"parameters": parameters,` This is continuation line 2 of the 8-line logical statement that begins at source line 145. Its exact fragment is '"parameters": parameters,'. The important operands are parameters: the value named 'parameters' in this block. Read together, the complete operation is: This returns '{ "parameters": parameters, "weights": weights, "gradients": gradients, "velocities": velocities, "expected_weights": expected_weights, "expected_velocities": expected_velocities, }' to the caller of the function generate_optimizer_cases, which handles generate optimizer cases, ending this invocation. The important operands are parameters: the value named 'parameters' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter.

Line 147: `"weights": weights,` This is continuation line 3 of the 8-line logical statement that begins at source line 145. Its exact fragment is '"weights": weights,'. The important operands are weights: a learned coefficient multiplied by an activation. Read together, the complete operation is: This returns '{ "parameters": parameters, "weights": weights, "gradients": gradients, "velocities": velocities, "expected_weights": expected_weights, "expected_velocities":

expected_velocities, }‘ to the caller of the function generate_optimizer_cases, which handles generate optimizer cases, ending this invocation. The important operands are parameters: the value named ’parameters’ in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter.

Line 148: `"gradients": gradients,` This is continuation line 4 of the 8-line logical statement that begins at source line 145. Its exact fragment is ‘"gradients": gradients,‘. The important operands are gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This returns ‘{ "parameters": parameters, "weights": weights, "gradients": gradients, "velocities": velocities, "expected_weights": expected_weights, "expected_velocities": expected_velocities, }‘ to the caller of the function generate_optimizer_cases, which handles generate optimizer cases, ending this invocation. The important operands are parameters: the value named ’parameters’ in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter.

Line 149: `"velocities": velocities,` This is continuation line 5 of the 8-line logical statement that begins at source line 145. Its exact fragment is ‘"velocities": velocities,‘. The important operands are velocities: the value named ’velocities’ in this block. Read together, the complete operation is: This returns ‘{ "parameters": parameters, "weights": weights, "gradients": gradients, "velocities": velocities, "expected_weights": expected_weights, "expected_velocities": expected_velocities, }‘ to the caller of the function generate_optimizer_cases, which handles generate optimizer cases, ending this invocation. The important operands are parameters: the value named ’parameters’ in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter.

Line 150: `"expected_weights": expected_weights,` This is continuation line 6 of the 8-line logical statement that begins at source line 145. Its exact fragment is ‘"expected_weights": expected_weights,‘. The important operands are expected_weights: a learned coefficient multiplied by an activation. Read together, the complete operation is: This returns ‘{ "parameters": parameters, "weights": weights, "gradients": gradients, "velocities": velocities, "expected_weights": expected_weights, "expected_velocities": expected_velocities, }‘ to the caller of the function generate_optimizer_cases, which handles generate optimizer cases, ending this invocation. The important operands are parameters: the value named ’parameters’ in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter.

Line 151: `"expected_velocities": expected_velocities,` This is continuation line 7 of the 8-line logical statement that begins at source line 145. Its exact fragment is ‘"expected_velocities": expected_velocities,‘. The important operands are expected_velocities: the value named ’expected velocities’ in this block. Read together, the complete operation is: This returns ‘{ "parameters": parameters, "weights": weights, "gradients": gradients, "velocities": velocities, "expected_weights": expected_weights, "expected_velocities": expected_velocities, }‘ to the caller of the function generate_optimizer_cases, which handles generate optimizer cases, ending this invocation. The important operands are parameters: the value named ’parameters’ in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter.

Line 152: `}` This is continuation line 8 of the 8-line logical statement that begins at source line 145. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘{ "parameters": parameters, "weights": weights, "gradients": gradients, "velocities": velocities, "expected_weights": expected_weights, "expected_velocities": expected_velocities, }‘ to the caller of the function generate_optimizer_cases, which handles generate optimizer cases, ending this invocation. The important operands are parameters: the value named ’parameters’ in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter.

Line 155: `def generate_layernorm_cases(rng: np.random.Generator) -> dict[str, list[int]]:` This starts function generate_layernorm_cases. Its indented body performs generate layernorm cases as part of the file that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests; it runs only when called.

Line 156: `descriptors: list[int] = []` This assigns ‘[]‘ to ‘descriptors‘ in host memory. Here descriptors means a DMA record header plus one command and its declared payload. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 157: `values: list[int] = []` This assigns '[]' to 'values' in host memory. Here values means the value named 'values' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 158: `gammas: list[int] = []` This assigns '[]' to 'gammas' in host memory. Here gammas means the learned multiplicative LayerNorm parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 159: `betas: list[int] = []` This assigns '[]' to 'betas' in host memory. Here betas means the learned additive LayerNorm parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 160: `expected: list[int] = []` This assigns '[]' to 'expected' in host memory. Here expected means the value named 'expected' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 161: `debug: list[int] = []` This assigns '[]' to 'debug' in host memory. Here debug means the value named 'debug' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 162: `lengths = [1, 4, 7, 32, 64, 127, 384, 1536]` This assigns '[1, 4, 7, 32, 64, 127, 384, 1536]' to 'lengths ' in host memory. Here lengths means the value named 'lengths' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 164: `for case_index, length in enumerate(lengths):` This iterates case_index, length over 'enumerate(lengths)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate_layernorm_cases, which handles generate layernorm cases.

Line 165: `epsilon = 1 + case_index * 3` This assigns '1 + case_index * 3' to 'epsilon ' in host memory. Here epsilon means the value named 'epsilon' in this block. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 166: `if case_index == 0:` This executes the following indented host-code branch only when 'case_index == 0' is true. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 167: `x = np.array([17], dtype=np.int64)` This assigns 'np.array([17], dtype=np.int64)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are array: the value named 'array' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 168: `elif case_index == 1:` This executes the following indented host-code branch only when 'case_index == 1' is true. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 169: `x = np.full(length, -91, dtype=np.int64)` This assigns 'np.full(length, -91, dtype=np.int64)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are full: the value named 'full' in this block; length: the value named 'length' in this block; dtype: the value named 'dtype' in this block. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 170: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function generate_layernorm_cases, which handles generate layernorm cases.

Line 171: `x = rng.integers(-8192, 8193, size=length, dtype=np.int64)` This assigns ‘rng.integers(-8192, 8193, size=length, dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 172: `gamma = rng.integers(8192, 49153, size=length, dtype=np.int64)` This assigns ‘rng.integers(8192, 49153, size=length, dtype=np.int64)‘ to ‘gamma ‘ in host memory. Here gamma means the learned multiplicative LayerNorm parameter. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 173: `beta = rng.integers(-2048, 2049, size=length, dtype=np.int64)` This assigns ‘rng.integers(-2048, 2049, size=length, dtype=np.int64)‘ to ‘beta ‘ in host memory. Here beta means the learned additive LayerNorm parameter. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 174: `output = integer_layer_norm(x, gamma, beta, epsilon)` This assigns ‘integer_layer_norm(x, gamma, beta, epsilon)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are integer_layer_norm: the value named ’integer layer norm’ in this block; x: the value named ’x’ in this block; gamma: the learned multiplicative LayerNorm parameter. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 175: `mean = int(x.sum() // length)` This assigns ‘int(x.sum() // length)‘ to ‘mean ‘ in host memory. Here mean means the arithmetic average used by normalization. The important operands are x: the value named ’x’ in this block; sum: the value named ’sum’ in this block; length: the value named ’length’ in this block. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 176: `centered = x - mean` This assigns ‘x - mean‘ to ‘centered ‘ in host memory. Here centered means the value named ’centered’ in this block. The important operands are x: the value named ’x’ in this block; mean: the arithmetic average used by normalization. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 177: `variance = int((centered * centered).sum() // length)` This assigns ‘int((centered * centered).sum() // length)‘ to ‘variance ‘ in host memory. Here variance means the average squared distance from the mean. The important operands are centered: the value named ’centered’ in this block; sum: the value named ’sum’ in this block; length: the value named ’length’ in this block. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 178: `sigma = max(1, isqrt(variance + epsilon))` This assigns ‘max(1, isqrt(variance + epsilon))‘ to ‘sigma ‘ in host memory. Here sigma means the value named ’sigma’ in this block. The important operands are max: the value named ’max’ in this block; isqrt: the value named ’isqrt’ in this block; variance: the average squared distance from the mean. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 179: `reciprocal = ((1 << 31) - 1) // sigma` This assigns ‘((1 << 31) - 1) // sigma‘ to ‘reciprocal ‘ in host memory. Here reciprocal means the value named ’reciprocal’ in this block. The important operands are sigma: the value named ’sigma’ in this block. Context: the function generate_layernorm_cases, which handles generate layernorm cases.

Line 181: `descriptors.append(length | (epsilon << 12))` This calls ‘descriptors.append‘ with ‘length | (epsilon << 12)‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 182: `values.extend(int(value) for value in x)` This calls ‘values.extend‘ with ‘int(value) for value in x‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 183: `gammas.extend(int(value) for value in gamma)` This calls ‘gammas.extend‘ with ‘int(value) for value in gamma‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 184: `betas.extend(int(value) for value in beta)` This calls ‘betas.extend‘ with ‘int(value) for value in beta‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 185: `expected.extend(int(value) for value in output)` This calls ‘expected.extend‘ with ‘int(value) for value in output‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 186: `debug.append(` This statement continues through source line 191; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘debug.append‘ with ‘ (mean & 0xFFFFFFFF) | (variance << 32) | (sigma << 64) | (reciprocal << 96) ‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 187: `(mean & 0xFFFFFFFF)` This is continuation line 2 of the 6-line logical statement that begins at source line 186. Its exact fragment is ‘(mean & 0xFFFFFFFF)‘. The important operands are mean: the arithmetic average used by normalization. Read together, the complete operation is: This calls ‘debug.append‘ with ‘ (mean & 0xFFFFFFFF) | (variance << 32) | (sigma << 64) | (reciprocal << 96) ‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 188: `| (variance << 32)` This is continuation line 3 of the 6-line logical statement that begins at source line 186. Its exact fragment is ‘| (variance << 32)‘. The important operands are variance: the average squared distance from the mean. Read together, the complete operation is: This calls ‘debug.append‘ with ‘ (mean & 0xFFFFFFFF) | (variance << 32) | (sigma << 64) | (reciprocal << 96) ‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 189: `| (sigma << 64)` This is continuation line 4 of the 6-line logical statement that begins at source line 186. Its exact fragment is ‘| (sigma << 64)‘. The important operands are sigma: the value named ’sigma’ in this block. Read together, the complete operation is: This calls ‘debug.append‘ with ‘ (mean & 0xFFFFFFFF) | (variance << 32) | (sigma << 64) | (reciprocal << 96) ‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 190: `| (reciprocal << 96)` This is continuation line 5 of the 6-line logical statement that begins at source line 186. Its exact fragment is ‘| (reciprocal << 96)‘. The important operands are reciprocal: the value named ’reciprocal’ in this block. Read together, the complete operation is: This calls ‘debug.append‘ with ‘ (mean & 0xFFFFFFFF) | (variance << 32) | (sigma << 64) | (reciprocal << 96) ‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 191: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 186. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘debug.append‘ with ‘ (mean & 0xFFFFFFFF) | (variance << 32) | (sigma << 64) | (reciprocal << 96) ‘ inside the function generate_layernorm_cases, which handles generate layernorm cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 192: `return {` This statement continues through source line 200; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘{ "descriptors": descriptors, "values": values, "gammas": gammas, "betas": betas, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_layernorm_cases, which handles generate layernorm cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; values: the value named ’values’ in this block; gammas: the learned multiplicative LayerNorm parameter.

Line 193: `"descriptors": descriptors,` This is continuation line 2 of the 9-line logical statement that begins at source line 192. Its exact fragment is ‘"descriptors": descriptors,‘. The important operands are descriptors: a DMA record header plus one command and its declared payload. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "values": values, "gammas": gammas, "betas": betas, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_layernorm_cases, which handles generate layernorm cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; values: the value named ’values’ in this block; gammas: the learned multiplicative LayerNorm parameter.

Line 194: `"values": values,` This is continuation line 3 of the 9-line logical statement that begins at source line 192. Its exact fragment is ‘"values": values,‘. The important operands are values: the value named ’values’ in this block. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "values": values, "gammas": gammas, "betas": betas, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_layernorm_cases, which handles generate layernorm cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; values: the value named ’values’ in this block; gammas: the learned multiplicative LayerNorm parameter.

Line 195: `"gammas": gammas,` This is continuation line 4 of the 9-line logical statement that begins at source line 192. Its exact fragment is ‘"gammas": gammas,‘. The important operands are gammas: the learned multiplicative LayerNorm parameter. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "values": values, "gammas": gammas, "betas": betas, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_layernorm_cases, which handles generate layernorm cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; values: the value named ’values’ in this block; gammas: the learned multiplicative LayerNorm parameter.

Line 196: `"betas": betas,` This is continuation line 5 of the 9-line logical statement that begins at source line 192. Its exact fragment is ‘"betas": betas,‘. The important operands are betas: the learned additive LayerNorm parameter. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "values": values, "gammas": gammas, "betas": betas, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_layernorm_cases, which handles generate layernorm cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; values: the value named ’values’ in this block; gammas: the learned multiplicative LayerNorm parameter.

Line 197: `"expected": expected,` This is continuation line 6 of the 9-line logical statement that begins at source line 192. Its exact fragment is ‘"expected": expected,‘. The important operands are expected: the value named ’expected’ in this block. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "values": values, "gammas": gammas, "betas": betas, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_layernorm_cases, which handles generate layernorm cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; values: the value named ’values’ in this block; gammas: the learned multiplicative LayerNorm parameter.

Line 198: `"debug": debug,` This is continuation line 7 of the 9-line logical statement that begins at source line 192. Its exact fragment is ‘"debug": debug,‘. The important operands are debug: the value named ’debug’ in this block. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "values": values, "gammas": gammas, "betas": betas, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_layernorm_cases, which handles generate layernorm cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; values: the value named ’values’ in this block; gammas: the learned multiplicative LayerNorm parameter.

Line 199: `"lengths": lengths,` This is continuation line 8 of the 9-line logical statement that begins at source line 192. Its exact fragment is ‘"lengths": lengths,‘. The important operands are lengths: the value named ’lengths’ in this block. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "values": values, "gammas": gammas, "betas": betas, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_layernorm_cases, which handles generate layernorm cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; values: the value named ’values’ in this block; gammas: the learned multiplicative LayerNorm parameter.

Line 200: `}` This is continuation line 9 of the 9-line logical statement that begins at source line 192. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "values": values, "gammas": gammas, "betas": betas, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_layernorm_cases, which handles generate layernorm cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; values: the value named ’values’ in this block; gammas: the learned multiplicative LayerNorm parameter.

Line 203: `def generate_softmax_cases(rng: np.random.Generator) -> dict[str, list[int]]:` This starts function generate_softmax_cases. Its indented body performs generate softmax cases as part of the file that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests; it runs only when called.

Line 204: `descriptors: list[int] = []` This assigns ‘[]‘ to ‘descriptors‘ in host memory. Here descriptors means a DMA record header plus one command and its declared payload. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 205: `scores: list[int] = []` This assigns ‘[]‘ to ‘scores‘ in host memory. Here scores means the value named ’scores’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 206: `expected: list[int] = []` This assigns ‘[]‘ to ‘expected‘ in host memory. Here expected means the value named ’expected’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 207: `debug: list[int] = []` This assigns ‘[]‘ to ‘debug‘ in host memory. Here debug means the value named ’debug’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 208: `lengths = [1, 4, 7, 32, 64, 127, 256]` This assigns ‘[1, 4, 7, 32, 64, 127, 256]‘ to ‘lengths ‘ in host memory. Here lengths means the value named ’lengths’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 210: `for case_index, length in enumerate(lengths):` This iterates case_index, length over ‘enumerate(lengths)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate_softmax_cases, which handles generate softmax cases.

Line 211: `difference_shift = case_index % 8` This assigns ‘case_index % 8‘ to ‘difference_shift ‘ in host memory. Here difference_shift means the value named ’difference shift’ in this block. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 212: `if case_index == 0:` This executes the following indented host-code branch only when ‘case_index == 0‘ is true. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 213: `score = np.array([123], dtype=np.int64)` This assigns ‘np.array([123], dtype=np.int64)‘ to ‘score ‘ in host memory. Here score means the value named ’score’ in this block. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 214: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function generate_softmax_cases, which handles generate softmax cases.

Line 215: `score = rng.integers(-(1 << 18), 1 << 18, size=length, dtype=np.int64)` This assigns ‘rng.integers(-(1 << 18), 1 << 18, size=length, dtype=np.int64)‘ to ‘score ‘ in host memory. Here score means the value named ’score’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 216: `probability = log_int_softmax_q15(score, difference_shift)` This assigns ‘log_int_softmax_q15(score, difference_shift)‘ to ‘probability ‘ in host memory. Here probability means the value named ’probability’ in this block. The important operands are log_int_softmax_q15: the value named ’log int softmax q15’ in this block; score: the value named ’score’ in this block; difference_shift: the value named ’difference shift’ in this block. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 217: `difference = score.max() - score` This assigns ‘score.max() - score‘ to ‘difference ‘ in host memory. Here difference means the value named ’difference’ in this block. The important operands are score: the value named ’score’ in this block; max: the value named ’max’ in this block. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 218: `exponent = np.minimum(difference >> difference_shift, 15)` This assigns ‘np.minimum(difference >> difference_shift, 15)‘ to ‘exponent ‘ in host memory. Here exponent means the value named ’exponent’ in this block. The important operands are minimum: the value named ’minimum’ in this block; difference: the value named ’difference’ in this block; difference_shift: the value named ’difference shift’ in this block. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 219: `unnormalized = 1 << (15 - exponent)` This assigns ‘1 << (15 - exponent)‘ to ‘unnormalized ‘ in host memory. Here unnormalized means the value named ’unnormalized’ in this block. The important operands are exponent: the value named ’exponent’ in this block. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 220: `denominator = int(unnormalized.sum())` This assigns ‘int(unnormalized.sum())‘ to ‘denominator ‘ in host memory. Here denominator means the value named ’denominator’ in this block. The important operands are unnormalized: the value named ’unnormalized’ in this block; sum: the value named ’sum’ in this block. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 221: `reciprocal = ((1 << 31) - 1) // denominator` This assigns ‘((1 << 31) - 1) // denominator‘ to ‘reciprocal ‘ in host memory. Here reciprocal means the value named ’reciprocal’ in this block. The important operands are denominator: the value named ’denominator’ in this block. Context: the function generate_softmax_cases, which handles generate softmax cases.

Line 223: `descriptors.append(length | (difference_shift << 9))` This calls ‘descriptors.append‘ with ‘length | (difference_shift << 9)‘ inside the function generate_softmax_cases, which handles generate softmax cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 224: `scores.extend(int(value) for value in score)` This calls ‘scores.extend‘ with ‘int(value) for value in score‘ inside the function generate_softmax_cases, which handles generate softmax cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 225: `expected.extend(int(value) for value in probability)` This calls ‘expected.extend‘ with ‘int(value) for value in probability‘ inside the function generate_softmax_cases, which handles generate softmax cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 226: `debug.append((int(score.max()) & 0xFFFFFF) | (denominator << 24) | (reciprocal << 48))` This calls ‘debug.append‘ with ‘(int(score.max()) & 0xFFFFFF) | (denominator << 24) | (reciprocal << 48)‘ inside the function generate_softmax_cases, which handles generate softmax cases. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 227: `return {` This statement continues through source line 233; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘{ "descriptors": descriptors, "scores": scores, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_softmax_cases, which handles generate softmax cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; scores: the value named ’scores’ in this block; expected: the value named ’expected’ in this block.

Line 228: `"descriptors": descriptors,` This is continuation line 2 of the 7-line logical statement that begins at source line 227. Its exact fragment is ‘"descriptors": descriptors,‘. The important operands are descriptors: a DMA record header plus one command and its declared payload. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "scores": scores, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_softmax_cases, which handles generate softmax cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; scores: the value named ’scores’ in this block; expected: the value named ’expected’ in this block.

Line 229: `"scores": scores,` This is continuation line 3 of the 7-line logical statement that begins at source line 227. Its exact fragment is ‘"scores": scores,‘. The important operands are scores: the value named ’scores’ in this block. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "scores": scores, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_softmax_cases, which handles generate softmax cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; scores: the value named ’scores’ in this block; expected: the value named ’expected’ in this block.

Line 230: `"expected": expected,` This is continuation line 4 of the 7-line logical statement that begins at source line 227. Its exact fragment is ‘"expected": expected,‘. The important operands are expected: the value named ’expected’ in this block. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "scores": scores, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_softmax_cases, which handles generate softmax cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; scores: the value named ’scores’ in this block; expected: the value named ’expected’ in this block.

Line 231: `"debug": debug,` This is continuation line 5 of the 7-line logical statement that begins at source line 227. Its exact fragment is ‘"debug": debug,‘. The important operands are debug: the value named ’debug’ in this block. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "scores": scores, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_softmax_cases, which handles generate softmax cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; scores: the value named ’scores’ in this block; expected: the value named ’expected’ in this block.

Line 232: `"lengths": lengths,` This is continuation line 6 of the 7-line logical statement that begins at source line 227. Its exact fragment is ‘"lengths": lengths,‘. The important operands are lengths: the value named ’lengths’ in this block. Read together, the complete operation is: This returns ‘{ "descriptors": descriptors, "scores": scores, "expected": expected, "debug": debug, "lengths": lengths, }‘ to the caller of the function generate_softmax_cases, which handles generate softmax cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; scores: the value named ’scores’ in this block; expected: the value named ’expected’ in this block.

Line 233: `}` This is continuation line 7 of the 7-line logical statement that begins at source line 227. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns '{ "descriptors": descriptors, "scores": scores, "expected": expected, "debug": debug, "lengths": lengths, }' to the caller of the function generate_softmax_cases, which handles generate softmax cases, ending this invocation. The important operands are descriptors: a DMA record header plus one command and its declared payload; scores: the value named 'scores' in this block; expected: the value named 'expected' in this block.

Line 236: `def main() -> None:` This starts function main. Its indented body performs main as part of the file that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests; it runs only when called.

Line 237: `parser = argparse.ArgumentParser()` This assigns 'argparse.ArgumentParser()' to 'parser ' in host memory. Here parser means the value named 'parser' in this block. The important operands are argparse: the value named 'argparse' in this block; ArgumentParser: the value named 'argument parser' in this block. Context: the function main, which handles main.

Line 238: `parser.add_argument("-output-dir", type=Path, default=Path("tb/fixtures"))` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 239: `parser.add_argument("-vector-count-per-operation", type=int, default=24)` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 240: `parser.add_argument("-optimizer-cases", type=int, default=128)` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 241: `args = parser.parse_args()` This assigns 'parser.parse_args()' to 'args ' in host memory. Here args means the value named 'args' in this block. The important operands are parser: the value named 'parser' in this block; parse_args: the value named 'parse args' in this block. Context: the function main, which handles main.

Line 242: `args.output_dir.mkdir(parents=True, exist_ok=True)` This calls 'args.output_dir.mkdir' with 'parents=True, exist_ok=True' inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 243: `rng = np.random.default_rng(20260711)` This assigns 'np.random.default_rng(20260711)' to 'rng ' in host memory. Here rng means the value named 'rng' in this block. The important operands are random: the value named 'random' in this block; default_rng: the value named 'default rng' in this block. Context: the function main, which handles main.

Line 245: `vector = generate_vector_cases(rng, args.vector_count_per_operation)` This assigns 'generate_vector_case
args.vector_count_per_operation)' to 'vector ' in host memory. Here vector means the value named 'vector' in this block. The important operands are generate_vector_cases: the value named 'generate vector cases' in this block; rng: the value named 'rng' in this block; args: the value named 'args' in this block. Context: the function main, which handles main.

Line 246: `optimizer = generate_optimizer_cases(rng, args.optimizer_cases)` This assigns 'generate_optimizer_cases(r
args.optimizer_cases)' to 'optimizer ' in host memory. Here optimizer means the value named 'optimizer' in this block. The important operands are generate_optimizer_cases: the value named 'generate optimizer cases' in this block; rng: the value named 'rng' in this block; args: the value named 'args' in this block. Context: the function main, which handles main.

Line 247: `layernorm = generate_layernorm_cases(rng)` This assigns 'generate_layernorm_cases(rng)' to 'layernorm ' in host memory. Here layernorm means the value named 'layernorm' in this block. The important operands are generate_layernorm_cases: the value named 'generate layernorm cases' in this block; rng: the value named 'rng' in this block. Context: the function main, which handles main.

Line 248: `softmax = generate_softmax_cases(rng)` This assigns ‘generate_softmax_cases(rng)‘ to ‘softmax ‘ in host memory. Here softmax means the value named ’softmax’ in this block. The important operands are generate_softmax_cases: the value named ’generate softmax cases’ in this block; rng: the value named ’rng’ in this block. Context: the function main, which handles main.

Line 249: `write_hex(args.output_dir / "vector_commands.mem", vector["commands"], 192)` This calls ‘write_hex‘ with ‘args.output_dir / "vector_commands.mem", vector["commands"], 192‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 250: `write_hex(args.output_dir / "vector_a.mem", vector["a"], 256)` This calls ‘write_hex‘ with ‘args.output_dir / "vector_a.mem", vector["a"], 256‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 251: `write_hex(args.output_dir / "vector_b.mem", vector["b"], 256)` This calls ‘write_hex‘ with ‘args.output_dir / "vector_b.mem", vector["b"], 256‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 252: `write_hex(args.output_dir / "vector_expected.mem", vector["expected"], 256)` This calls ‘write_hex‘ with ‘args.output_dir / "vector_expected.mem", vector["expected"], 256‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 253: `write_hex(args.output_dir / "optimizer_parameters.mem", optimizer["parameters"], 64)` This calls ‘write_hex‘ with ‘args.output_dir / "optimizer_parameters.mem", optimizer["parameters"], 64‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 254: `write_hex(args.output_dir / "optimizer_weights.mem", optimizer["weights"], 128)` This calls ‘write_hex‘ with ‘args.output_dir / "optimizer_weights.mem", optimizer["weights"], 128‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 255: `write_hex(args.output_dir / "optimizer_gradients.mem", optimizer["gradients"], 192)` This calls ‘write_hex‘ with ‘args.output_dir / "optimizer_gradients.mem", optimizer["gradients"], 192‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 256: `write_hex(args.output_dir / "optimizer_velocities.mem", optimizer["velocities"], 192)` This calls ‘write_hex‘ with ‘args.output_dir / "optimizer_velocities.mem", optimizer["velocities"], 192‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 257: `write_hex(args.output_dir / "optimizer_expected_weights.mem", optimizer["expected_weights"], 128)` This calls ‘write_hex‘ with ‘args.output_dir / "optimizer_expected_weights.mem", optimizer["expected_weights"], 128‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 258: `write_hex(args.output_dir / "optimizer_expected_velocities.mem", optimizer["expected_velocities"], 192)` This calls ‘write_hex‘ with ‘args.output_dir / "optimizer_expected_velocities.mem", optimizer["expected_velocities"], 192‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 259: `write_hex(args.output_dir / "layernorm_cases.mem", layernorm["descriptors"], 64)` This calls ‘write_hex‘ with ‘args.output_dir / "layernorm_cases.mem", layernorm["descriptors"], 64‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 260: `write_hex(args.output_dir / "layernorm_values.mem", layernorm["values"], 24)` This calls ‘write_hex‘ with ‘args.output_dir / "layernorm_values.mem", layernorm["values"], 24‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 261: `write_hex(args.output_dir / "layernorm_gammas.mem", layernorm["gammas"], 24)` This calls ‘write_hex‘ with ‘args.output_dir / "layernorm_gammas.mem", layernorm["gammas"], 24‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 262: `write_hex(args.output_dir / "layernorm_betas.mem", layernorm["betas"], 24)` This calls ‘write_hex‘ with ‘args.output_dir / "layernorm_betas.mem", layernorm["betas"], 24‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 263: `write_hex(args.output_dir / "layernorm_expected.mem", layernorm["expected"], 32)` This calls ‘write_hex‘ with ‘args.output_dir / "layernorm_expected.mem", layernorm["expected"], 32‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 264: `write_hex(args.output_dir / "layernorm_debug.mem", layernorm["debug"], 128)` This calls ‘write_hex‘ with ‘args.output_dir / "layernorm_debug.mem", layernorm["debug"], 128‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 265: `write_hex(args.output_dir / "softmax_cases.mem", softmax["descriptors"], 32)` This calls ‘write_hex‘ with ‘args.output_dir / "softmax_cases.mem", softmax["descriptors"], 32‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 266: `write_hex(args.output_dir / "softmax_scores.mem", softmax["scores"], 24)` This calls ‘write_hex‘ with ‘args.output_dir / "softmax_scores.mem", softmax["scores"], 24‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 267: `write_hex(args.output_dir / "softmax_expected.mem", softmax["expected"], 16)` This calls ‘write_hex‘ with ‘args.output_dir / "softmax_expected.mem", softmax["expected"], 16‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 268: `write_hex(args.output_dir / "softmax_debug.mem", softmax["debug"], 80)` This calls ‘write_hex‘ with ‘args.output_dir / "softmax_debug.mem", softmax["debug"], 80‘ inside the function main, which handles main. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 270: `manifest = {` This statement continues through source line 280; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"])

"softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are seed: the value named ’seed’ in this block; vector_cases: the value named ’vector cases’ in this block; len: the value named ’len’ in this block. Context: the function main, which handles main.

Line 271: `"seed": 20260711,` This is continuation line 2 of the 11-line logical statement that begins at source line 270. Its exact fragment is ‘"seed": 20260711,‘. The important operands are seed: the value named ’seed’ in this block. Read together, the complete operation is: This assigns ‘{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are seed: the value named ’seed’ in this block; vector_cases: the value named ’vector cases’ in this block; len: the value named ’len’ in this block. Context: the function main, which handles main.

Line 272: `"vector_cases": len(vector["commands"]),` This is continuation line 3 of the 11-line logical statement that begins at source line 270. Its exact fragment is ‘"vector_cases": len(vector["commands"]),‘. The important operands are vector_cases: the value named ’vector cases’ in this block; len: the value named ’len’ in this block; vector: the value named ’vector’ in this block. Read together, the complete operation is: This assigns ‘{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are seed: the value named ’seed’ in this block; vector_cases: the value named ’vector cases’ in this block; len: the value named ’len’ in this block. Context: the function main, which handles main.

Line 273: `"vector_cases_per_operation": args.vector_count_per_operation,` This is continuation line 4 of the 11-line logical statement that begins at source line 270. Its exact fragment is ‘"vector_cases_per_operation": args.vector_count_per_operation,‘. The important operands are vector_cases_per_operation: the value named ’vector cases per operation’ in this block; args: the value named ’args’ in this block; vector_count_per_operation: the value named ’vector count per operation’ in this block. Read together, the complete operation is: This assigns ‘{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are seed: the value named ’seed’ in this block; vector_cases: the value named ’vector cases’ in this block; len: the value named ’len’ in this block. Context: the function main, which handles main.

Line 274: `"optimizer_cases": len(optimizer["parameters"]),` This is continuation line 5 of the 11-line logical statement that begins at source line 270. Its exact fragment is ‘"optimizer_cases": len(optimizer["parameters"]),‘. The important operands are optimizer_cases: the value named ’optimizer cases’ in this block; len: the value named ’len’ in this block; optimizer: the value named ’optimizer’ in this block. Read together, the complete operation is: This assigns ‘{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are seed: the value named ’seed’ in this block; vector_cases: the value named ’vector cases’ in this block; len: the value named ’len’ in this block. Context: the function main, which handles main.

Line 275: `"layernorm_cases": len(layernorm["lengths"]),` This is continuation line 6 of the 11-line logical statement that begins at source line 270. Its exact fragment is ‘"layernorm_cases": len(layernorm["lengths"]),‘. The important operands are layernorm_cases: the value named ’layernorm cases’ in this block; len: the value named ’len’ in this block; layernorm: the value named ’layernorm’ in this block. Read together, the complete operation is: This assigns ‘{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’

in this block. The important operands are seed: the value named 'seed' in this block; vector_cases: the value named 'vector cases' in this block; len: the value named 'len' in this block. Context: the function main, which handles main.

Line 276: `"layernorm_samples": len(layernorm["values"]),` This is continuation line 7 of the 11-line logical statement that begins at source line 270. Its exact fragment is '"layernorm_samples": len(layernorm["values"]),'. The important operands are layernorm_samples: the value named 'layernorm samples' in this block; len: the value named 'len' in this block; layernorm: the value named 'layernorm' in this block. Read together, the complete operation is: This assigns '{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"] "rounding": "nearest, ties away from zero", }' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are seed: the value named 'seed' in this block; vector_cases: the value named 'vector cases' in this block; len: the value named 'len' in this block. Context: the function main, which handles main.

Line 277: `"softmax_cases": len(softmax["lengths"]),` This is continuation line 8 of the 11-line logical statement that begins at source line 270. Its exact fragment is '"softmax_cases": len(softmax["lengths"]),'. The important operands are softmax_cases: the value named 'softmax cases' in this block; len: the value named 'len' in this block; softmax: the value named 'softmax' in this block. Read together, the complete operation is: This assigns '{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are seed: the value named 'seed' in this block; vector_cases: the value named 'vector cases' in this block; len: the value named 'len' in this block. Context: the function main, which handles main.

Line 278: `"softmax_samples": len(softmax["scores"]),` This is continuation line 9 of the 11-line logical statement that begins at source line 270. Its exact fragment is '"softmax_samples": len(softmax["scores"]),'. The important operands are softmax_samples: the value named 'softmax samples' in this block; len: the value named 'len' in this block; softmax: the value named 'softmax' in this block. Read together, the complete operation is: This assigns '{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are seed: the value named 'seed' in this block; vector_cases: the value named 'vector cases' in this block; len: the value named 'len' in this block. Context: the function main, which handles main.

Line 279: `"rounding": "nearest, ties away from zero",` This is continuation line 10 of the 11-line logical statement that begins at source line 270. Its exact fragment is '"rounding": "nearest, ties away from zero",'. The important operands are rounding: the value named 'rounding' in this block; nearest: the value named 'nearest' in this block; ties: the value named 'ties' in this block. Read together, the complete operation is: This assigns '{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are seed: the value named 'seed' in this block; vector_cases: the value named 'vector cases' in this block; len: the value named 'len' in this block. Context: the function main, which handles main.

Line 280: `}` This is continuation line 11 of the 11-line logical statement that begins at source line 270. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "seed": 20260711, "vector_cases": len(vector["commands"]), "vector_cases_per_operation": args.vector_count_per_operation, "optimizer_cases": len(optimizer["parameters"]), "layernorm_cases": len(layernorm["lengths"]), "layernorm_samples": len(layernorm["values"]), "softmax_cases": len(softmax["lengths"]), "softmax_samples": len(softmax["scores"]), "rounding": "nearest, ties away from zero", }' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are seed: the value named 'seed' in this block; vector_cases: the value named 'vector cases' in this block; len: the value named 'len' in this block. Context: the function main, which handles main.

Line 281: `(args.output_dir / "operator_fixture_manifest.json").write_text(` This statement continues through source line 283; the explanation uses the complete expression rather than only the first visual fragment. Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block; operator_fixture_manifest: the value named 'operator fixture manifest' in this block.

Line 282: `json.dumps(manifest, indent=2) + "\n"` This is continuation line 2 of the 3-line logical statement that begins at source line 281. Its exact fragment is 'json.dumps(manifest, indent=2) + "\n"'. The important operands are json: the value named 'json' in this block; dumps: the value named 'dumps' in this block; manifest: the value named 'manifest' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block; operator_fixture_manifest: the value named 'operator fixture manifest' in this block.

Line 283: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 281. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block; operator_fixture_manifest: the value named 'operator fixture manifest' in this block.

Line 284: `(args.output_dir / "operator_fixture_params.svh").write_text(` This statement continues through source line 291; the explanation uses the complete expression rather than only the first visual fragment. Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block; operator_fixture_params: the value named 'operator fixture params' in this block.

Line 285: `f"'define NEURAEDGE_VECTOR_CASES {manifest['vector_cases']}\n"` This is continuation line 2 of the 8-line logical statement that begins at source line 284. Its exact fragment is 'f"'define NEURAEDGE_VECTOR_CASES {manifest['vector_cases']}\n"'. The important operands are f: the value named 'f' in this block; define: the value named 'define' in this block; NEURAEDGE_VECTOR_CASES: the value named 'neuraedge vector cases' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block; operator_fixture_params: the value named 'operator fixture params' in this block.

Line 286: `f"'define NEURAEDGE_OPTIMIZER_CASES {manifest['optimizer_cases']}\n"` This is continuation line 3 of the 8-line logical statement that begins at source line 284. Its exact fragment is 'f"'define NEURAEDGE_OPTIMIZER_CASES {manifest['optimizer_cases']}\n"'. The important operands are f: the value named 'f' in this block; define: the value named 'define' in this block; NEURAEDGE_OPTIMIZER_CASES: the value named 'neuraedge optimizer cases' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block; operator_fixture_params: the value named 'operator fixture params' in this block.

Line 287: `f"'define NEURAEDGE_LAYERNORM_CASES {manifest['layernorm_cases']}\n"` This is continuation line 4 of the 8-line logical statement that begins at source line 284. Its exact fragment is 'f"'define NEURAEDGE_LAYERNORM_CASES {manifest['layernorm_cases']}\n"'. The important operands are f: the value named 'f' in this block; define: the value named 'define' in this block; NEURAEDGE_LAYERNORM_CASES: the value named 'neuraedge layernorm cases' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named 'args' in this block; output_dir: the value named 'output dir' in this block; operator_fixture_params: the value named 'operator fixture params' in this block.

Line 288: `f"‘define NEURAEDGE_LAYERNORM_SAMPLES {manifest['layernorm_samples']}\n"` This is continuation line 5 of the 8-line logical statement that begins at source line 284. Its exact fragment is ‘f"‘define NEURAEDGE_LAYERNORM_SA {manifest['layernorm_samples']}\n"‘. The important operands are f: the value named ’f’ in this block; define: the value named ’define’ in this block; NEURAEDGE_LAYERNORM_SAMPLES: the value named ’neuraedge layernorm samples’ in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named ’args’ in this block; output_dir: the value named ’output dir’ in this block; operator_fixture_params: the value named ’operator fixture params’ in this block.

Line 289: `f"‘define NEURAEDGE_SOFTMAX_CASES {manifest['softmax_cases']}\n"` This is continuation line 6 of the 8-line logical statement that begins at source line 284. Its exact fragment is ‘f"‘define NEURAEDGE_SOFTMAX_CASES {manifest['softmax_cases']}\n"‘. The important operands are f: the value named ’f’ in this block; define: the value named ’define’ in this block; NEURAEDGE_SOFTMAX_CASES: the value named ’neuraedge softmax cases’ in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named ’args’ in this block; output_dir: the value named ’output dir’ in this block; operator_fixture_params: the value named ’operator fixture params’ in this block.

Line 290: `f"‘define NEURAEDGE_SOFTMAX_SAMPLES {manifest['softmax_samples']}\n"` This is continuation line 7 of the 8-line logical statement that begins at source line 284. Its exact fragment is ‘f"‘define NEURAEDGE_SOFTMAX_SAMPLES {manifest['softmax_samples']}\n"‘. The important operands are f: the value named ’f’ in this block; define: the value named ’define’ in this block; NEURAEDGE_SOFTMAX_SAMPLES: the value named ’neuraedge softmax samples’ in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named ’args’ in this block; output_dir: the value named ’output dir’ in this block; operator_fixture_params: the value named ’operator fixture params’ in this block.

Line 291: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 284. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The important operands are args: the value named ’args’ in this block; output_dir: the value named ’output dir’ in this block; operator_fixture_params: the value named ’operator fixture params’ in this block.

Line 294: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

Line 295: `main()` This calls ‘main‘ with “ inside the file whose role is to builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests. The call advances the host-side job that builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests.

30 software/generate_source_reference.py

Role in the complete system: generates this source-reference volume and its source-integrity manifest.

30.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to generates this source-reference volume and its source-integrity manifest.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: SourceEntry, StatementSpan. Functions: project_root, source_entries, inferred_file_purpose, humanize, identifier_meaning, summarize_expression, python_contexts, _joined_statement, python_statement_spans, sv_statement_spans, sv_contexts, explain_sv_line, explain_python_line, explain_script_line, latex_escape, source_archite annotation_tex, generate, compile_pdf, main. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

30.2 Complete source

```
"""Generate an auditable, context-aware line-by-line source reference.

The generated TeX contains the complete source text and one explanation for
every nonblank line.  Explanations use the enclosing Python function or RTL
process, known accelerator signal meanings, and the exact left/right sides of
assignments.  A JSON manifest records every included file and SHA-256 digest so
the PDF can be checked against the source tree that produced it.
"""

from __future__ import annotations

import argparse
import ast
import hashlib
import json
import os
import re
import subprocess
from dataclasses import dataclass
from pathlib import Path
from typing import Iterable


FILE_PURPOSES = {
    "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads",
    "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes",
    "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted",
    "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores",
    "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO",
    "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software",
    "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams",
    "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory",
    "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating",
    "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm",
    "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting",
    "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array",
    "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware",
    "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior",
    "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication",
    "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ",
    "rtl/neuraedge_sgd_optimizer8.sv": "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state",
    "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array",
    "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle",
    "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path",
```

```
    "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives",
    "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory",
    "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection",
    "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set",
    "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning",
    "software/export_spikingformer_hardware.py": "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections",
    "software/generate_debug_report.py": "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",
    "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests",
    "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest",
    "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total",
    "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy",
    "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison",
    "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads",
    "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive",
    "software/neuraedge_protocol.py": "defines command bit fields, descriptor records, packed streams, and result decoders",
    "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry",
    "software/neuraedge_training.py": "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates",
    "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay",
    "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations",
    "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive",
    "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math",
    "software/spikingformer_reference.py": "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison",
    "software/spikingformer_training_runtime.py": "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",
    "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports",
}


FILE_DATAFLOWS = {
    "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.",
    "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.",
    "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.",
    "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.",
    "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.",
    "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.",
    "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.",
    "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.",
    "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.",
    "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.",
    "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.",
    "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.",
    "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.",
    "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and
```

```
        streams the result.",
    "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.",
    "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.",
    "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.",
    "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.",
    "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.",
    "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.",
    "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.",
    "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.",
    "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.",
    "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.",
    "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.",
    "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.",
    "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.",
    "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.",
    "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.",
}


IDENTIFIER_MEANINGS = {
    "clk": "the clock that makes all registers advance together",
    "aclk": "the AXI and accelerator clock",
    "rst_n": "the active-low synchronous reset",
    "aresetn": "the active-low AXI reset",
    "valid": "the producer's statement that its current payload is meaningful",
    "ready": "the consumer's statement that it can accept a payload this cycle",
    "last": "the AXI-Stream marker for the final beat of a packet or command payload",
    "state": "the finite-state-machine phase retained in flip-flops",
    "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters",
    "descriptor": "a DMA record header plus one command and its declared payload",
    "input_remaining": "the number of command payload beats still expected",
    "output_remaining": "the number of command result beats still expected",
    "program_output_remaining": "the number of result beats left in the complete descriptor program",
    "activation": "an input feature value consumed by a neuron or multiply-accumulate operation",
    "weight": "a learned coefficient multiplied by an activation",
    "bias": "an additive learned offset applied after accumulation",
    "accumulator": "a widened running sum that prevents intermediate overflow",
    "membrane": "the persistent LIF neuron state carrying charge between timesteps",
    "threshold": "the membrane level at which a neuron emits a spike",
    "leak": "the retained fraction of prior membrane state",
    "spike": "a binary event produced when a neuron crosses its threshold",
    "dense_mac": "a count of ordinary activation-times-weight operations actually issued",
    "spike_add": "a count of event-driven conditional weight additions actually issued",
    "zero_point": "the integer code representing real numeric zero in affine quantization",
    "scale": "the real-value spacing represented by one quantized integer step",
    "requant": "the conversion of a wide accumulated result back into a bounded activation code",
    "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention",
    "gamma": "the learned multiplicative LayerNorm parameter",
    "beta": "the learned additive LayerNorm parameter",
    "mean": "the arithmetic average used by normalization",
    "variance": "the average squared distance from the mean",
    "gradient": "the signed loss derivative used to update a learned parameter",
    "velocity": "the momentum history retained by the SGD optimizer",
    "learning_rate": "the fixed-point multiplier controlling update magnitude",
    "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic",
    "interrupt": "the hardware notification that wakes software after completion or error",
    "counter": "a hardware telemetry register that accumulates measured events",
```

```
    "records": "the number of descriptor records accepted since program start",
    "payload": "stream data belonging to the current command",
    "buffer": "local storage that decouples producer and consumer timing",
    "index": "an integer selecting one tensor element, lane, bank, or state entry",
    "lane": "one parallel arithmetic element within a vector or tensor datapath",
    "row": "one output-row position processed by the tensor tile",
    "column": "one output-feature position processed in parallel",
    "timestep": "one discrete SNN update instant",
    "logits": "the ten unnormalized classifier scores used for argmax",
    "prediction": "the class index with the greatest classifier score",
}


STATE_MEANINGS = {
    "IDLE": "waits for a new operation and does not consume payload data",
    "HEADER": "accepts and validates the descriptor record header",
    "COMMAND_LOW": "captures command bits 63 through 0",
    "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST",
    "SUBMIT": "holds command-valid until the universal core accepts the instruction",
    "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine",
    "WAIT": "waits for both command completion and the declared output count",
    "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned",
    "LOAD": "stores incoming operands into local FPGA memory",
    "COMPUTE": "issues arithmetic work using values already held on chip",
    "OUTPUT": "presents computed results under valid/ready backpressure",
    "COUNT": "updates a segmented telemetry counter without a long carry chain",
    "DONE": "raises completion for one cycle and returns control to idle",
}


@dataclass(frozen=True)
class SourceEntry:
    path: Path
    relative: str
    purpose: str


@dataclass(frozen=True)
class StatementSpan:
    start: int
    end: int
    text: str


def project_root() -> Path:
    return Path(__file__).resolve().parents[1]


def source_entries(root: Path) -> list[SourceEntry]:
    patterns = (
        ("rtl", "*.sv"),
        ("software", "*.py"),
        ("tb", "*.sv"),
        ("tests", "*.py"),
        ("scripts", "*.ps1"),
        ("scripts", "*.tcl"),
    )
    entries: list[SourceEntry] = []
    for directory, pattern in patterns:
        for path in sorted((root / directory).glob(pattern)):
            relative = path.relative_to(root).as_posix()
            purpose = FILE_PURPOSES.get(relative, inferred_file_purpose(relative))
            entries.append(SourceEntry(path, relative, purpose))
    return entries


def inferred_file_purpose(relative: str) -> str:
    stem = Path(relative).stem
    words = humanize(stem.removeprefix("tb_").removeprefix("test_"))
    if relative.startswith("tb/"):
        return f"drives, backpressures, and checks the {words} RTL block in simulation"
    if relative.startswith("tests/"):
        return f"checks {words} behavior independently with Python assertions"
    if relative.endswith(".tcl"):
        return f"runs the reproducible Vivado {words} hardware flow"
    if relative.endswith(".ps1"):
        return f"orchestrates the repeatable Windows {words} command-line flow"
    return f"implements the {words} part of the project"


def humanize(identifier: str) -> str:
    text = re.sub(r"([a-z0-9])([A-Z])", r"\1 \2", identifier)
    return text.replace("_", " ").replace("-", " ").strip().lower()


def identifier_meaning(identifier: str) -> str:
    lowered = identifier.lower().removesuffix("_r").removesuffix("_i").removesuffix("_o")
```

```
    if lowered in IDENTIFIER_MEANINGS:
        return IDENTIFIER_MEANINGS[lowered]
    matches = [meaning for key, meaning in IDENTIFIER_MEANINGS.items() if key in lowered]
    if matches:
        return matches[0]
    if lowered.endswith("_mem") or "memory" in lowered:
        return "local FPGA storage addressed by surrounding index logic"
    if lowered.endswith("_count") or lowered.endswith("_counter"):
        return "an integer count retained for control or measurable telemetry"
    if lowered.endswith("_valid"):
        return "a handshake flag saying the associated payload is meaningful"
    if lowered.endswith("_ready"):
        return "a handshake flag saying the associated consumer can accept data"
    if lowered.endswith("_data"):
        return "the payload carried with the surrounding valid/ready handshake"
    if lowered.endswith("_enable") or lowered.endswith("_en"):
        return "a control bit that permits the associated state or storage update"
    return f"the value named '{humanize(identifier)}' in this block"


def summarize_expression(expression: str) -> str:
    identifiers = re.findall(r"\b[A-Za-z_][A-Za-z0-9_$]*\b", expression)
    ignored = {
        "if", "else", "begin", "end", "logic", "wire", "signed", "unsigned",
        "true", "false", "int", "return", "self", "np", "Path",
    }
    selected: list[str] = []
    for identifier in identifiers:
        if identifier.lower() in ignored or identifier in selected:
            continue
        selected.append(identifier)
        if len(selected) == 3:
            break
    if not selected:
        return "The expression is a literal or structural operator and has no hidden software state."
    meanings = "; ".join(f"{name}: {identifier_meaning(name)}" for name in selected)
    return f"The important operands are {meanings}."


def python_contexts(text: str) -> dict[int, str]:
    contexts: dict[int, tuple[int, str]] = {}
    try:
        tree = ast.parse(text)
    except SyntaxError:
        return {}
    for node in ast.walk(tree):
        if isinstance(node, (ast.ClassDef, ast.FunctionDef, ast.AsyncFunctionDef)):
            end = getattr(node, "end_lineno", node.lineno)
            kind = "class" if isinstance(node, ast.ClassDef) else "function"
            description = f"the {kind} {node.name}, which handles {humanize(node.name)}"
            span = end - node.lineno
            for line_number in range(node.lineno, end + 1):
                previous = contexts.get(line_number)
                if previous is None or span < previous[0]:
                    contexts[line_number] = (span, description)
    return {line: value[1] for line, value in contexts.items()}


def _joined_statement(lines: list[str], start: int, end: int) -> str:
    return " ".join(
        fragment for fragment in (line.strip() for line in lines[start - 1:end])
        if fragment
    )


def python_statement_spans(text: str) -> dict[int, StatementSpan]:
    """Map multiline host statements to their complete source expression."""
    lines = text.splitlines()
    try:
        tree = ast.parse(text)
    except SyntaxError:
        return {}
    candidates: list[ast.AST] = []
    simple_nodes = (
        ast.Assign, ast.AnnAssign, ast.AugAssign, ast.Return, ast.Raise,
        ast.Assert, ast.Import, ast.ImportFrom,
    )
    for node in ast.walk(tree):
        if isinstance(node, simple_nodes):
            candidates.append(node)
        elif isinstance(node, ast.Expr) and isinstance(node.value, ast.Call):
            candidates.append(node)
    result: dict[int, StatementSpan] = {}
    for node in candidates:
        start = int(getattr(node, "lineno", 0))
        end = int(getattr(node, "end_lineno", start))
        if start <= 0 or end <= start:
```

```
            continue
        span = StatementSpan(start, end, _joined_statement(lines, start, end))
        for line_number in range(start, end + 1):
            previous = result.get(line_number)
            if previous is None or (end - start) < (previous.end - previous.start):
                result[line_number] = span
    return result


def sv_statement_spans(lines: list[str]) -> dict[int, StatementSpan]:
    """Recover multiline RTL assignments without pretending fragments are complete."""
    result: dict[int, StatementSpan] = {}
    assignment_start = re.compile(
        r"^(?:assign\s+)?[A-Za-z_][A-Za-z0-9_$\[\].:+* /()'-]*\s*(?:<=|=(?!=))"
    )
    state_encoding = re.compile(r"^[A-Z][A-Z0-9_]*\s*=.*,$")
    line_index = 0
    while line_index < len(lines):
        stripped = lines[line_index].strip()
        if (not stripped or stripped.startswith("//") or ";" in stripped
                or state_encoding.match(stripped) or not assignment_start.match(stripped)):
            line_index += 1
            continue
        end_index = line_index + 1
        while end_index < len(lines) and ";" not in lines[end_index]:
            end_index += 1
        if end_index >= len(lines) or end_index - line_index > 64:
            line_index += 1
            continue
        start = line_index + 1
        end = end_index + 1
        span = StatementSpan(start, end, _joined_statement(lines, start, end))
        for line_number in range(start, end + 1):
            result[line_number] = span
        line_index = end_index + 1
    return result


def sv_contexts(lines: list[str]) -> dict[int, str]:
    result: dict[int, str] = {}
    process = "the module's structural declarations"
    process_depth = 0
    process_until: str | None = None
    state = ""
    for line_number, raw in enumerate(lines, start=1):
        stripped = raw.strip()
        starts_process = False
        if re.search(r"\balways_ff\b", stripped):
            process = "the clocked process that updates registers only on a rising clock edge"
            process_depth = max(1, len(re.findall(r"\bbegin\b", stripped)))
            process_until = None
            starts_process = True
        elif re.search(r"\balways_comb\b", stripped):
            process = "the combinational process whose outputs respond without waiting for a clock edge"
            process_depth = max(1, len(re.findall(r"\bbegin\b", stripped)))
            process_until = None
            starts_process = True
        elif stripped.startswith("function"):
            names = re.findall(r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", stripped)
            name = names[-1] if names else "function"
            process = f"the combinational helper {humanize(name)}"
            process_until = "endfunction"
            process_depth = 0
            starts_process = True
        elif stripped.startswith("task"):
            names = re.findall(r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", stripped)
            name = names[-1] if names else "task"
            process = f"the simulation task {humanize(name)}"
            process_until = "endtask"
            process_depth = 0
            starts_process = True
        state_match = re.match(r"(?:ST_|CORE_|WR_|RD_)?([A-Z][A-Z0-9_]*)\s*:", stripped)
        if state_match:
            state_name = state_match.group(1)
            meaning = next((value for key, value in STATE_MEANINGS.items() if key in state_name), "performs the named control phase")
            state = f" Within state {state_match.group(0)[:-1]}, the machine {meaning}."
        result[line_number] = process + state
        if stripped == "endcase":
            state = ""
        if process_until is not None and stripped.startswith(process_until):
            process = "the module's structural declarations"
            process_until = None
            process_depth = 0
            state = ""
        elif process_until is None and process != "the module's structural declarations":
            if not starts_process:
                process_depth += len(re.findall(r"\bbegin\b", stripped))
```

```
                process_depth -= len(re.findall(r"\bend\b", stripped))
            if process_depth <= 0 or (starts_process and "begin" not in stripped):
                process = "the module's structural declarations"
                process_depth = 0
                state = ""
    return result


def explain_sv_line(line: str, context: str, purpose: str) -> str:
    stripped = line.strip()
    if stripped.startswith("//"):
        comment = stripped[2:].strip()
        return f"Design intent: {comment} This comment anchors the surrounding logic in the block that {purpose}."
    if stripped.startswith("`"):
        if "default_nettype none" in stripped:
            return "This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware."
        if "timescale" in stripped:
            return "This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic."
        return f"This SystemVerilog preprocessor line controls compilation of the block that {purpose}."
    if re.match(r"(?:module|package)\b", stripped):
        name = re.split(r"[\s(#]", stripped, maxsplit=2)[1]
        return f"This opens {name}, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that {purpose}."
    if stripped.startswith(("endmodule", "endpackage")):
        return "This closes the current hardware definition; no circuit behavior is attached to the keyword itself."
    if stripped.startswith("import "):
        return f"This imports shared package names into the current module so opcodes and mode constants have exactly one definition. The import changes compile-time name visibility; it does not create registers or consume a clock cycle."
    port = re.match(r"(input|output|inout)\s+(?:wire|logic)?\s*(?:signed\s*)?(\[[^]]+\])?\s*([A-Za-z_][A-Za-z0-9_$]*)", stripped)
    if port:
        direction, width, name = port.groups()
        width_text = width or "one bit"
        return f"This declares {name} as a {direction} port with width {width_text}. It carries {identifier_meaning(name)} across the module boundary; a port is a bundle of physical FPGA nets, not a software argument."
    parameter = re.match(r"(?:parameter|localparam)\s+(?:(?:logic|integer|int|bit)\s+)?(?:signed\s*)?(?:\[[^]]+\]\s*)?([A-Za-z_][A-Za-z0-9_$]*)\s*=\s*(.*?)[,;]?$", stripped)
    if parameter:
        name, value = parameter.groups()
        return f"This fixes {name} to {value} during elaboration. Vivado uses that constant to shape or simplify the generated circuit; {name} denotes {identifier_meaning(name)}."
    if "typedef enum" in stripped:
        return "This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable."
    state_match = re.match(r"([A-Z][A-Z0-9_]*)\s*=\s*[^,}]+", stripped)
    if state_match:
        name = state_match.group(1)
        meaning = next((value for key, value in STATE_MEANINGS.items() if key in name), "identifies one explicit control phase")
        return f"This assigns a fixed binary encoding to state {name}, which {meaning}. The encoding is visible in waveforms even though the source uses the readable name."
    declaration = re.match(r"(?:logic|wire|reg|integer|genvar)\s+(?:signed\s*)?(?:\[[^]]+\]\s*)?(.+?);$", stripped)
    if declaration:
        names = [re.split(r"[\s[=]", item.strip(), maxsplit=1)[0] for item in declaration.group(1).split(",")]
        descriptions = "; ".join(f"{name} is {identifier_meaning(name)}" for name in names[:4])
        return f"This declares hardware signal storage or wiring in {context}. Specifically, {descriptions}. Widths in square brackets count from the stated most-significant bit down to the least-significant bit."
    assignment = re.match(r"(?:assign\s+)?([A-Za-z_][A-Za-z0-9_$\[\].:]*)\s*(<=|=)\s*(.+?);?$", stripped)
    if assignment:
        lhs, operator, rhs = assignment.groups()
        timing = "updates after the active clock edge together with other nonblocking assignments" if operator == "<=" else "is driven by combinational or procedural logic in the current evaluation"
        return f"This makes {lhs}, {identifier_meaning(lhs.split('[')[0])}, equal to `{rhs.rstrip(';')}`. It {timing}. {summarize_expression(rhs)} Context: {context}."
    if stripped.startswith("always_ff"):
        return "This opens clocked sequential logic. Everything assigned with `<=` inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator's notion of cycles and persistent state."
    if stripped.startswith("always_comb"):
        return "This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path."
    if stripped.startswith("initial"):
        return "This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior."
    reset_match = re.match(r"if\s*\(\s*!?(?:rst_n|aresetn)\s*\)", stripped)
    if reset_match:
        return f"This is the reset branch for `{reset_match.group(0)[3:-1].strip()}`. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: {context}."
    if_match = re.match(r"(?:end\s+)?(?:else\s+)?if\s*\((.*)\)", stripped)
    if if_match:
        condition = if_match.group(1)
        return f"This condition selects the following hardware branch only when `{condition}` is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. {summarize_expression(condition)} Context: {context}."
    if re.search(r"\b(?:unique\s+)?case\s*\((.*)\)", stripped):
        selector = re.search(r"\((.*)\)", stripped).group(1)
        return f"This starts a multiplexer/state decode selected by `{selector}`. `unique` allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches."
```

```
    label = re.match(r"([A-Za-z_][A-Za-z0-9_$]*)\s*:\s*(.*)", stripped)
    if label:
        name, action = label.groups()
        meaning = next((value for key, value in STATE_MEANINGS.items() if key in name), "performs this explicitly named phase")
        return f"This is the {name} case branch, which {meaning}. The same-cycle action begins with `{action}`; context is {context}."
    if stripped.startswith("default:"):
        return f"This is the default decode branch. It supplies deterministic behavior for every unlisted bit pattern, which prevents inferred latches in combinational logic and gives corrupted state/opcode values an explicit recovery or error path. Context: {context}."
    port_map = re.match(r"\.([A-Za-z_][A-Za-z0-9_$]*)\s*\((.*?)\)\s*,?$", stripped)
    if port_map:
        port_name, signal = port_map.groups()
        return f"This connects the child instance's named port `{port_name}` to parent expression `{signal}`. The connection is a physical net bundle after synthesis; direction is declared in the child module, and {identifier_meaning(signal.split('[')[0] or port_name)}."
    instance = re.match(r"([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?([A-Za-z_][A-Za-z0-9_$]*)\s*\(", stripped)
    if instance and instance.group(1) not in {"if", "for", "while", "case"}:
        return f"This instantiates module {instance.group(1)} as {instance.group(2)}. Synthesis creates that subordinate circuit and connects its ports in the lines that follow."
    if stripped.startswith("for ") or stripped.startswith("for("):
        return f"This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: {context}."
    if stripped in {"begin", "end", "endcase", "endgenerate", "generate"} or stripped.startswith(("end else", "else begin")):
        return f"This structural keyword opens or closes the branch/process scope described as {context}; it has no arithmetic value of its own."
    if stripped.startswith(("$display", "$fatal", "$finish", "assert")):
        return "This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit."
    return f"Within {context}, this exact statement contributes to the block that {purpose}. {summarize_expression(stripped)}"


def explain_python_line(line: str, context: str, purpose: str) -> str:
    stripped = line.strip()
    if stripped.startswith("#"):
        return f"The comment records this intent: {stripped[1:].strip()} It applies inside {context}."
    if stripped.startswith(("import ", "from ")):
        return f"This imports the named Python API so {context} can use it explicitly. Imports execute on the host processor and create no FPGA hardware."
    if stripped.startswith("class "):
        name = stripped.split()[1].split("(")[0].rstrip(":")
        return f"This defines class {name}, which groups data and methods for {humanize(name)}. Instances exist in host memory and organize the flow that {purpose}."
    if stripped.startswith(("def ", "async def ")):
        name = stripped.split("def ", 1)[1].split("(", 1)[0]
        return f"This starts function {name}. Its indented body performs {humanize(name)} as part of the file that {purpose}; it runs only when called."
    if stripped.startswith("return"):
        value = stripped.removeprefix("return").strip()
        return f"This returns `{value}` to the caller of {context}, ending this invocation. {summarize_expression(value)}"
    if stripped.startswith("raise "):
        return f"This rejects an invalid state by raising `{stripped[6:]}`. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results."
    if stripped.startswith("assert "):
        return f"This test requires `{stripped[7:]}` to be true. A false result fails verification instead of allowing an unchecked numerical or protocol mismatch."
    if_match = re.match(r"(?:el)?if\s+(.+):$", stripped)
    if if_match:
        condition = if_match.group(1)
        return f"This executes the following indented host-code branch only when `{condition}` is true. {summarize_expression(condition)} Context: {context}."
    if stripped == "else:":
        return f"This begins the alternative host-code branch when the preceding condition was false, still inside {context}."
    loop = re.match(r"for\s+(.+?)\s+in\s+(.+):$", stripped)
    if loop:
        target, source = loop.groups()
        return f"This iterates {target} over `{source}`. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in {context}."
    if stripped.startswith("while "):
        return f"This repeats the following host-code block while `{stripped[6:-1]}` remains true; surrounding timeout/error logic prevents an unbounded hardware wait."
    if stripped.startswith("with "):
        return f"This opens a managed resource scope for `{stripped[5:-1]}` so files, tracing, or temporary state are finalized even if {context} raises an error."
    assignment = re.match(r"([A-Za-z_][A-Za-z0-9_.\[\], ]*)\s*(?::[^=]+)?\s*=\s*(.+)", stripped)
    if assignment and "==" not in stripped and ">=" not in stripped and "<=" not in stripped and "!=" not in stripped:
        lhs, rhs = assignment.groups()
        name = lhs.split(",", 1)[0].strip().split("[")[0].split(".")[-1]
        return f"This assigns `{rhs}` to `{lhs}` in host memory. Here {name} means {identifier_meaning(name)}. {summarize_expression(rhs)} Context: {context}."
    if stripped.startswith(("try:", "except ", "finally:")):
        return f"This is explicit host-side failure handling inside {context}; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically."
    if stripped.startswith("if __name__"):
        return "This standard entry-point guard runs the command-line main function only when this file is executed directly, while keeping its definitions importable by tests."
    if stripped in {"(", ")", "[", "]", "{", "}"} or stripped.endswith(("(", "[", "{")):
        return f"This line opens or closes a multi-line Python expression inside {context}; the neighboring lines supply its arguments or elements."
```

```
    if stripped.startswith("@"):
        return f"This decorator changes how the following class or function is constructed. ‘{stripped}‘ is applied at import time on the host and is part of {context}; it creates no FPGA circuit by itself."
    if stripped.startswith(('"""', "'''")):
        return f"This is executable documentation attached to {context}. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work."
    if ".add_argument(" in stripped:
        return f"This declares a command-line option for {context}. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting."
    call = re.match(r"([A-Za-z_][A-Za-z0-9_.]*)\((.*)\),?$", stripped)
    if call:
        return f"This calls ‘{call.group(1)}‘ with ‘{call.group(2)}‘ inside {context}. The call advances the host-side job that {purpose}."
    return f"Inside {context}, this statement advances the host implementation that {purpose}. {summarize_expression(stripped)}"


def explain_script_line(line: str, suffix: str, purpose: str) -> str:
    stripped = line.strip()
    if stripped.startswith("#"):
        return f"This comment documents the batch-flow intent: {stripped[1:].strip()}"
    if suffix == ".tcl":
        if stripped.startswith("read_verilog"):
            return "This loads the listed RTL into Vivado’s in-memory design so the subsequent synthesis uses the repository source."
        if stripped.startswith("synth_design"):
            return "This invokes Vivado synthesis for the named top and FPGA part, mapping behavioral RTL into LUT, flip-flop, BRAM, and DSP primitives."
        if stripped.startswith(("opt_design", "place_design", "phys_opt_design", "route_design")):
            return f"This runs Vivado implementation command ‘{stripped}‘, changing the physical netlist or placement/routing needed for real timing."
        if stripped.startswith("report_") or stripped.startswith("write_"):
            return f"This writes an auditable Vivado artifact using ‘{stripped}‘; later documentation reads the resulting report rather than inventing a metric."
        if stripped.startswith("create_clock"):
            return "This declares the clock period against which Vivado checks every synchronous setup and hold path."
        if stripped.startswith("create_bd_cell"):
            return f"This adds the named IP or module-reference cell to the Vivado block design. The cell becomes a concrete PS, DMA, converter, interconnect, reset, or accelerator instance after generation: ‘{stripped}‘."
        if stripped.startswith("connect_bd_intf_net"):
            return f"This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for ‘{stripped}‘."
        if stripped.startswith("connect_bd_net"):
            return f"This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: ‘{stripped}‘."
        if stripped.startswith("assign_bd_address"):
            return "This asks Vivado to establish an AXI address map; following explicit offset/range settings make the software-visible register addresses stable across rebuilds."
        if stripped.startswith(("launch_runs", "wait_on_run", "open_run")):
            return f"This controls a named Vivado synthesis or implementation run with ‘{stripped}‘. Waiting and opening the completed run prevents reports from reading an earlier or unfinished netlist."
        if stripped.startswith("read_saif"):
            return f"This annotates implemented nets with measured XSim switching activity from the named SAIF file. The retained match report quantifies which simulated names mapped to routed nets: ‘{stripped}‘."
    else:
        if "$ErrorActionPreference" in stripped:
            return "This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands."
        if stripped.startswith("param") or re.match(r"\[[A-Za-z]+\]\$", stripped):
            return "This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options."
        if "LASTEXITCODE" in stripped:
            return "This checks the external program’s process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline."
        if re.search(r"vivado|xsim|xelab|xvlog|iverilog|verilator|python", stripped, re.IGNORECASE):
            return f"This locates or invokes a required tool as part of the script that {purpose}; the exact command and arguments remain visible here for reproducibility."
        if stripped.startswith(("Resolve-Path", "Join-Path", "$ProjectRoot")) or "[System.IO.Path]::GetFullPath" in stripped:
            return f"This resolves a deterministic filesystem location for the flow that {purpose}. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output."
        if stripped.startswith("New-Item"):
            return "This creates the declared output directory before a tool writes into it. ‘-Force‘ permits an existing directory but does not make an external tool failure succeed."
        if stripped.startswith("Remove-Item"):
            return "This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file."
    return f"This batch-control line contributes to the flow that {purpose}. {summarize_expression(stripped)}"


def latex_escape(text: str) -> str:
    replacements = {
        "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$",
        "#": r"\#", "_": r"\_", "{": r"\{", "}": r"\}",
        "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", "<": r"\textless{}",
        ">": r"\textgreater{}",
    }
    return "".join(replacements.get(character, character) for character in text)


def source_architecture_notes(entry: SourceEntry, text: str) -> list[tuple[str, str]]:
    """Build a file-level map so line annotations have an understandable frame."""
    suffix = entry.path.suffix.lower()
```

```
    dataflow = FILE_DATAFLOWS.get(
        entry.relative,
        f"This file's end-to-end responsibility is to {entry.purpose}.",
    )
    notes: list[tuple[str, str]] = [("Dataflow responsibility", dataflow)]
    if suffix in {".sv", ".v"} and not entry.relative.startswith("tb/"):
        modules = re.findall(r"^\s*module\s+([A-Za-z_][A-Za-z0-9_$]*)", text, re.MULTILINE)
        inputs = len(re.findall(r"^\s*input\b", text, re.MULTILINE))
        outputs = len(re.findall(r"^\s*output\b", text, re.MULTILINE))
        clocked = len(re.findall(r"\balways_ff\b", text))
        combinational = len(re.findall(r"\balways_comb\b", text))
        states = sorted(set(re.findall(r"\bST_[A-Z0-9_]+\b", text)))
        instances: list[str] = []
        instance_pattern = re.compile(
            r"^\s*([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?"
            r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", re.MULTILINE,
        )
        for module_name, instance_name in instance_pattern.findall(text):
            if module_name in {"module", "function", "task", "if", "for", "while", "case"}:
                continue
            label = f"{module_name} as {instance_name}"
            if label not in instances:
                instances.append(label)
        notes.extend(
            [
                (
                    "What becomes FPGA hardware",
                    f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",
                ),
                (
                    "Cycle-level control",
                    (
                        f"The named control phases include {', '.join(states[:18])}"
                        + (" and additional states" if len(states) > 18 else "")
                        + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update."
                        if states
                        else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."
                    ),
                ),
                (
                    "Connections to other blocks",
                    (
                        "This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection."
                        if instances
                        else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."
                    ),
                ),
                (
                    "How to debug it",
                    "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",
                ),
            ]
        )
    elif entry.relative.startswith("tb/"):
        tops = re.findall(r"^\s*module\s+([A-Za-z_][A-Za-z0-9_$]*)", text, re.MULTILINE)
        checks = len(re.findall(r"\$(?:fatal|error)|\bassert\b|FAILED|PASSED", text))
        notes.extend(
            [
                ("Simulation-only status", f"{', '.join(tops) or 'This testbench'} is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker."),
                ("Evidence produced", f"The file contains approximately {checks} explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient."),
            ]
        )
    elif suffix == ".py":
        try:
            tree = ast.parse(text)
            classes = [node.name for node in ast.walk(tree) if isinstance(node, ast.ClassDef)]
            functions = [node.name for node in ast.walk(tree) if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef))]
        except SyntaxError:
            classes, functions = [], []
        notes.extend(
            [
                ("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite."),
                ("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(functions[:24]) or 'none'}. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it."),
```

```
                ("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact."),
            ]
        )
    else:
        tool = "Vivado Tcl" if suffix == ".tcl" else "PowerShell"
        notes.extend(
            [
                ("Where it runs", f"This is {tool} orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable."),
                ("Reproducibility contract", "Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate."),
            ]
        )
    return notes


def annotation_tex(entry: SourceEntry, text: str) -> tuple[list[str], int]:
    lines = text.splitlines()
    suffix = entry.path.suffix.lower()
    contexts = python_contexts(text) if suffix == ".py" else sv_contexts(lines) if suffix in {".sv", ".v"} else {}
    statement_spans = (
        python_statement_spans(text) if suffix == ".py"
        else sv_statement_spans(lines) if suffix in {".sv", ".v"}
        else {}
    )
    result: list[str] = []
    explained = 0
    for line_number, raw in enumerate(lines, start=1):
        if not raw.strip():
            continue
        context = contexts.get(line_number, f"the file whose role is to {entry.purpose}")
        span = statement_spans.get(line_number)
        if suffix in {".sv", ".v"}:
            complete_explanation = explain_sv_line(
                span.text if span else raw, context, entry.purpose)
        elif suffix == ".py":
            complete_explanation = explain_python_line(
                span.text if span else raw, context, entry.purpose)
        else:
            complete_explanation = explain_script_line(raw, suffix, entry.purpose)
        if span and line_number != span.start:
            position = line_number - span.start + 1
            length = span.end - span.start + 1
            explanation = (
                f"This is continuation line {position} of the {length}-line logical statement "
                f"that begins at source line {span.start}. Its exact fragment is ‘{raw.strip()}‘. "
                f"{summarize_expression(raw)} Read together, the complete operation is: "
                f"{complete_explanation}"
            )
        elif span:
            explanation = (
                f"This statement continues through source line {span.end}; the explanation uses "
                f"the complete expression rather than only the first visual fragment. "
                f"{complete_explanation}"
            )
        else:
            explanation = complete_explanation
        preview = raw.strip()
        if len(preview) > 110:
            preview = preview[:107] + "..."
        result.extend(
            [
                rf"\paragraph{{Line {line_number}: \texttt{{{latex_escape(preview)}}}}}",
                latex_escape(explanation),
            ]
        )
        explained += 1
    return result, explained


def generate(output_dir: Path) -> tuple[Path, Path]:
    root = project_root()
    output_dir = output_dir.resolve()
    output_dir.mkdir(parents=True, exist_ok=True)
    entries = source_entries(root)
    tex_path = output_dir / "neuraedge_line_by_line_source_reference.tex"
    manifest_path = output_dir / "neuraedge_line_by_line_source_manifest.json"
    tex = [
        r"\documentclass[9pt]{article}",
        r"\usepackage[margin=0.62in]{geometry}",
        r"\usepackage[T1]{fontenc}",
        r"\usepackage{lmodern}",
        r"\usepackage{xcolor}",
        r"\usepackage{listings}",
```

```
        r"\usepackage[hidelinks]{hyperref}",
        r"\setlength{\parindent}{0pt}",
        r"\setlength{\parskip}{3pt}",
        r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,showstringspaces=false,numbers=left,numberstyle=\tiny,frame=single}",
        r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}",
        r"\author{Generated from hashed repository source}",
        r"\date{\today}",
        r"\begin{document}",
        r"\maketitle",
        r"\tableofcontents",
        r"\clearpage",
        r"\section{How to read this volume}",
        "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",
    ]
    manifest: dict[str, object] = {"format": "neuraedge-source-reference-v1", "files": []}
    total_lines = 0
    total_nonblank = 0
    for entry in entries:
        raw_bytes = entry.path.read_bytes()
        text = raw_bytes.decode("utf-8", errors="strict")
        line_count = len(text.splitlines())
        nonblank_count = sum(bool(line.strip()) for line in text.splitlines())
        annotations, explained = annotation_tex(entry, text)
        if explained != nonblank_count:
            raise RuntimeError(f"annotation coverage mismatch for {entry.relative}: {explained} != {nonblank_count}")
        total_lines += line_count
        total_nonblank += nonblank_count
        listing_path = os.path.relpath(entry.path, output_dir).replace("\\", "/")
        manifest["files"].append(
            {
                "path": entry.relative,
                "sha256": hashlib.sha256(raw_bytes).hexdigest(),
                "lines": line_count,
                "nonblank_lines": nonblank_count,
                "explained_nonblank_lines": explained,
                "purpose": entry.purpose,
            }
        )
        tex.extend(
            [
                r"\clearpage",
                rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}",
                rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.",
                r"\subsection{Beginner architecture map}",
                r"\begin{description}",
                *[
                    rf"\item[{latex_escape(label)}] {latex_escape(note)}"
                    for label, note in source_architecture_notes(entry, text)
                ],
                r"\end{description}",
                r"\subsection{Complete source}",
                rf"\lstinputlisting{{{listing_path}}}",
                r"\subsection{Line annotations}",
                *annotations,
            ]
        )
    manifest["summary"] = {
        "file_count": len(entries),
        "source_lines": total_lines,
        "nonblank_lines": total_nonblank,
        "explained_nonblank_lines": total_nonblank,
    }
    tex.extend(
        [
            r"\clearpage",
            r"\section{Coverage statement}",
            f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank lines. The companion JSON manifest records each source SHA-256 digest.",
            r"\end{document}",
        ]
    )
    tex_path.write_text("\n".join(tex) + "\n", encoding="utf-8")
    manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")
    return tex_path, manifest_path


def compile_pdf(tex_path: Path, pdflatex: str) -> Path:
    for _ in range(2):
        subprocess.run(
            [pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name],
            cwd=tex_path.parent,
            check=True,
```

```
        )
    return tex_path.with_suffix(".pdf")


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output-dir", type=Path, default=project_root() / "docs")
    parser.add_argument("--compile", action="store_true")
    parser.add_argument("--pdflatex", default="pdflatex")
    args = parser.parse_args()
    tex_path, manifest_path = generate(args.output_dir.resolve())
    print(f"tex={tex_path}")
    print(f"manifest={manifest_path}")
    if args.compile:
        print(f"pdf={compile_pdf(tex_path, args.pdflatex)}")


if __name__ == "__main__":
    main()
```

30.3 Line annotations

Line 1: `"""Generate an auditable, context-aware line-by-line source reference.` This is executable documentation attached to the file whose role is to generates this source-reference volume and its source-integrity manifest. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `The generated TeX contains the complete source text and one explanation for` Inside the file whose role is to generates this source-reference volume and its source-integrity manifest, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are The: the value named 'the' in this block; generated: the value named 'generated' in this block; TeX: the value named 'te x' in this block.

Line 4: `every nonblank line.  Explanations use the enclosing Python function or RTL` Inside the file whose role is to generates this source-reference volume and its source-integrity manifest, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are every: the value named 'every' in this block; nonblank: the value named 'nonblank' in this block; line: the value named 'line' in this block.

Line 5: `process, known accelerator signal meanings, and the exact left/right sides of` Inside the file whose role is to generates this source-reference volume and its source-integrity manifest, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are process: the value named 'process' in this block; known: the value named 'known' in this block; accelerator: the value named 'accelerator' in this block.

Line 6: `assignments.  A JSON manifest records every included file and SHA-256 digest so` Inside the file whose role is to generates this source-reference volume and its source-integrity manifest, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are assignments: the value named 'assignments' in this block; A: the value named 'a' in this block; JSON: the value named 'json' in this block.

Line 7: `the PDF can be checked against the source tree that produced it.` Inside the file whose role is to generates this source-reference volume and its source-integrity manifest, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are the: the value named 'the' in this block; PDF: the value named 'pdf' in this block; can: the value named 'can' in this block.

Line 8: `"""` This is executable documentation attached to the file whose role is to generates this source-reference volume and its source-integrity manifest. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 10: `from __future__ import annotations` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `import argparse` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `import ast` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `import hashlib` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `import json` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `import os` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `import re` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `import subprocess` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `from pathlib import Path` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `from typing import Iterable` This imports the named Python API so the file whose role is to generates this source-reference volume and its source-integrity manifest can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `FILE_PURPOSES = {` This statement continues through source line 69; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv":

"stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv":
"evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating",
"rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring
algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold,
reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gra-
dients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer
softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates
many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv":
"packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv":
"provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv":
"implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv":
"performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "de-
codes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines
through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, re-
quantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile
in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and
byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source
checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py":
"writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",
"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors
for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity
manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command
count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer pa-
rameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py":
"builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py":
"tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py":
"implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py":
"controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised up-
dates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spiking-
former graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP,
residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and
exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines cali-
bration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",
"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordi-
nates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means
the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neu-
raedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value
named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity
manifest.

Line 25: `"rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated...` This is continuation line 2 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates re-

sult backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd_ "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 26: `"rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table o...` This is continuation line 3 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_lut8: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the re-

duction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 27: `"rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation...` This is continuation line 4 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_active_index_buffer: local storage that decouples producer and consumer timing; sv: the value named ’sv’ in this block. Read together, the

complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 28: `"rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated...` This is continuation line 5 of the 46-line logical statement that begins at source line 24. Its ex-

act fragment is ‘"rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_alpha_xnor: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 29: `"rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction posi...` This is continuation line 6 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_buffered_tensor_core: local storage that decouples producer and consumer timing; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neu-

raedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 30: `"rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL a...` This is continuation line 7 of the 46-line logical statement that begins at source line 24. Its exact fragment is '"rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software",'. The important operands are rtl: the value named 'rtl' in this block; neuraedge_command_pkg: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns '{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",

"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 31: `"rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and p...` This is continuation line 8 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_descriptor_sequencer: a DMA record header plus one command and its declared payload; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines cali-

bration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",
"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 32: `"rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in...` This is continuation line 9 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_gradient_accumulator_bank: a widened running sum that prevents intermediate overflow; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spiking-

former graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 33: `"rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integ...` This is continuation line 10 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_integer_layernorm: the value named 'neuraedge integer layernorm' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py":

"controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 34: `"rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an ite...` This is continuation line 11 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_isqrt64: the value named ’neuraedge isqrt64’ in this block; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py":

"tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py":
"implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py":
"controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",
"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 35: `"rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold...` This is continuation line 12 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_lif_bank: the value named 'neuraedge lif bank' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv":
"implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv":
"performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py":
"writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",
"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command

count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 36: `"rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a r...` This is continuation line 13 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_linear_backward8: the value named 'neuraedge linear backward8' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",

"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 37: `"rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-poi...` This is continuation line 14 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_log_softmax: the value named ’neuraedge log softmax’ in this block; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py":

"writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",
"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors
for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity
manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command
count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer pa-
rameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py":
"builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py":
"tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py":
"implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py":
"controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised up-
dates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spiking-
former graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP,
residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and
exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines cali-
bration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",
"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordi-
nates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means
the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neu-
raedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value
named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity
manifest.

Line 38: `"rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FP...` This is continuation line 15 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_mac2_array_probe: the value named 'neuraedge mac2 array probe' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile

in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 39: `"rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplica...` This is continuation line 16 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_mac2_u8: the value named ’neuraedge mac2 u8’ in this block; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "de-

codes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 40: `"rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, a...` This is continuation line 17 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_pynq_axi_top: the value named ’neuraedge pynq axi top’ in this block; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd

"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 41: `"rtl/neuraedge_sgd_optimizer8.sv": "applies signed fixed-point SGD with momentum to eight weight lanes and ...` This is continuation line 18 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_sgd_optimizer8.sv": "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_sgd_optimizer8: the value named 'neuraedge sgd optimizer8' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates

many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 42: `"rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arit...` This is continuation line 19 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_tensor_tile: the value named ’neuraedge tensor tile’ in this block; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold,

reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 43: `"rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle",` This is continuation line 20 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_udivider: the value named 'neuraedge udivider' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "eval-

uates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 44: `"rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalizati LIF,...` This is continuation line 21 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_universal_core: the value named 'neuraedge universal core' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes

and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 45: `"rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization and po...` This is continuation line 22 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_vector_alu: the value named 'neuraedge vector alu' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR

spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 46: `"rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory",` This is continuation line 23 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_weight_bank: a learned coefficient multiplied by an activation; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for

repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 47: `"software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the la...` This is continuation line 24 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the

largest real Spikingformer projection",‘. The important operands are software: the value named ’software’ in this block; benchmark_compiler_memory: local FPGA storage addressed by surrounding index logic; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 48: `"software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over...` This is continuation line 25 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set",‘. The important operands are software: the value named ’software’ in this block; evaluate_spikingformer: the value named ’evaluate spikingformer’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value

named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 49: `"software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for boun...` This is continuation line 26 of the 46-line logical statement that begins at source line 24. Its exact fragment is '"software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning",'. The important operands are software: the value named 'software' in this block; export_cifar10_shards: the value named 'export cifar10 shards' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns '{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordi-

nates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 50: `"software/export_spikingformer_hardware.py": "folds BatchNorm, calibrates quantizers, and writes aligned ha...` This is continuation line 27 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/export_spikingformer_hardware.py": "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections",‘. The important operands are software: the value named ’software’ in this block; export_spikingformer_hardware: the value named ’export spikingformer hardware’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p

"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",
"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 51: `"software/generate_debug_report.py": "turns full-test predictions, compiler manifests, XSim waveforms, and ...` This is continuation line 28 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/generate_debug_report.py": "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",‘. The important operands are software: the value named ’software’ in this block; generate_debug_report: the value named ’generate debug report’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",
"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP,

residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 52: `"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LU...` This is continuation line 29 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests",‘. The important operands are software: the value named ’software’ in this block; generate_operator_fixtures: the value named ’generate operator fixtures’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p

"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",
"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

**Line 53: `"software/generate_source_reference.py": "generates this source-reference volume and its source-integ
m...`** This is continuation line 30 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest",‘. The important operands are software: the value named 'software' in this block; generate_source_reference: the value named 'generate source reference' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",
"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py":

"implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py":
"controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",
"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 54: `"software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command co...` This is continuation line 31 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total",‘. The important operands are software: the value named ’software’ in this block; generate_spikingformer_manifest: the value named ’generate spikingformer manifest’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv":
"implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv":
"performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo
"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",
"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer pa-

rameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 55: `"software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradient...` This is continuation line 32 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy",‘. The important operands are software: the value named 'software' in this block; generate_training_coverage: the value named 'generate training coverage' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors

for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 56: `"software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors...` This is continuation line 33 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison",‘. The important operands are software: the value named ’software’ in this block; generate_tensor_tile_fixture: the value named ’generate tensor tile fixture’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo

"folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report
"turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs",
"software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors
for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity
manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command
count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer pa-
rameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py":
"builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py":
"tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py":
"implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py
"defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py":
"controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p
"builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised up-
dates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spiking-
former graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP,
residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and
exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines cali-
bration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p
"reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr
"runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",
"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordi-
nates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means
the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neu-
raedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value
named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity
manifest.

Line 57: `"software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and ...` This is continuation line 34 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads",‘. The important operands are software: the value named 'software' in this block; neu-
raedge_compiler: the value named 'neuraedge compiler' in this block; py: the value named 'py' in this block. Read
together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of un-
signed eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a
programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the re-
duction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes
alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv":
"loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates re-
sult backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes
and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-
record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv":
"stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv":
"evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating",
"rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring
algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold,
reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gra-
dients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer
softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates
many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv":
"packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv":
"provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd
"applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv":
"implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv":
"performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "de-
codes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines
through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, re-
quantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile
in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and

byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 58: `"software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware ar...` This is continuation line 35 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive",‘. The important operands are software: the value named ’software’ in this block; neuraedge_golden: the value named ’neuraedge golden’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines

through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 59: `"software/neuraedge_protocol.py": "defines command bit fields, descriptor records, packed streams, and resu...` This is continuation line 36 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/neuraedge_protocol.py": "defines command bit fields, descriptor records, packed streams, and result decoders",‘. The important operands are software: the value named 'software' in this block; neuraedge_protocol: the value named 'neuraedge protocol' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv":

"implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 60: `"software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitionin...` This is continuation line 37 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry",‘. The important operands are software: the value named ’software’ in this block; neuraedge_pynq: the value named ’neuraedge pynq’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv":

"packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 61: `"software/neuraedge_training.py": "builds quantized forward, backward, surrogate-gradient, and optimizer pr...` This is continuation line 38 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/neuraedge_training.py": "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates",‘. The important operands are software: the value named ’software’ in this block; neuraedge_training: the value named ’neuraedge training’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gra-

dients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 62: `"software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingf...` This is continuation line 39 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay",‘. The important operands are software: the value named 'software' in this block; run_spikingformer_pynq: the value named 'run spikingformer pynq' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating",

"rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_t "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 63: `"software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, ...` This is continuation line 40 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations",‘. The important operands are software: the value named ’software’ in this block; spikingformer_accelerator: the value named ’spikingformer accelerator’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-

record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 64: `"software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned paramet...` This is continuation line 41 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive",‘. The important operands are software: the value named ’software’ in this block; spikingformer_parameters: the value named ’spikingformer parameters’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activa-

tion and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd_ "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 65: `"software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization roundi...` This is continuation line 42 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math",‘. The important operands are software: the value named ’software’ in this block; spikingformer_quantization: the value named ’spikingformer quantization’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and eval-

uates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named ’sv’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 66: `"software/spikingformer_reference.py": "reconstructs the trained source graph and its integer hardware-emul...` This is continuation line 43 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/spikingformer_reference.py": "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison",‘. The important operands are software: the value named ’software’ in this block;

spikingformer_reference: the value named 'spikingformer reference' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 67: `"software/spikingformer_training_runtime.py": "runs resumable CIFAR-10 training through the accelerator com...` This is continuation line 44 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘"software/spikingformer_training_runtime.py": "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",‘. The important operands are software: the value named ’software’ in this block; spikingformer_training_runtime: the value named ’spikingformer training runtime’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd_ "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation", "software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named ’file purposes’ in this block. The important operands are rtl: the value named ’rtl’ in this block; neu-

raedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 68: `"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and...` This is continuation line 45 of the 46-line logical statement that begins at source line 24. Its exact fragment is '"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports",'. The important operands are software: the value named 'software' in this block; train_spikingformer_pynq: the value named 'train spikingformer pynq' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns '{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",

"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 69: } This is continuation line 46 of the 46-line logical statement that begins at source line 24. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "stores one local tile of unsigned eight-bit activations for repeated tensor-array reads", "rtl/neuraedge_activation_lut8.sv": "loads and evaluates a programmable 256-entry activation lookup table on eight lanes", "rtl/neuraedge_active_index_buffer.sv": "stores the reduction indices that remain after all-zero activation positions are compacted", "rtl/neuraedge_alpha_xnor.sv": "computes alpha-weighted XNOR spiking similarity and streams its accumulated scores", "rtl/neuraedge_buffered_tensor_core.sv": "loads activation and weight tiles, compacts sparse reduction positions, schedules the physical tensor tile, and isolates result backpressure with a registered skid FIFO", "rtl/neuraedge_command_pkg.sv": "defines the shared command opcodes and tensor-mode encodings used by RTL and host software", "rtl/neuraedge_descriptor_sequencer.sv": "turns a multi-record DMA program into checked core commands and payload/output streams", "rtl/neuraedge_gradient_accumulator_bank.sv": "stores one lane of exact cross-command backward gradients in inferred distributed memory", "rtl/neuraedge_integer_layernorm.sv": "evaluates integer LayerNorm by accumulating moments, taking an integer square root, dividing, scaling, and saturating", "rtl/neuraedge_isqrt64.sv": "computes an exact floor square root of an unsigned 64-bit radicand with an iterative restoring algorithm", "rtl/neuraedge_lif_bank.sv": "stores 32 persistent membrane values and applies leak, integration, threshold, reset, and spike counting", "rtl/neuraedge_linear_backward8.sv": "computes exact tiled linear-layer input and weight gradients with a reusable four-by-four signed DSP array", "rtl/neuraedge_log_softmax.sv": "implements power-of-two integer softmax normalization without floating-point exponent hardware", "rtl/neuraedge_mac2_array_probe.sv": "instantiates many packed MAC blocks solely to measure their routed FPGA resource and timing behavior", "rtl/neuraedge_mac2_u8.sv": "packs two exact unsigned eight-bit multiplications into one DSP48E1 multiplication", "rtl/neuraedge_pynq_axi_top.sv": "provides AXI-Lite control, AXI-Stream DMA, interrupts, descriptor mode, and hardware telemetry for PYNQ", "rtl/neuraedge_sgd "applies signed fixed-point SGD with momentum to eight weight lanes and stores updated state", "rtl/neuraedge_tensor_tile.sv": "implements the eight-row by sixteen-column dense, binary, and ternary arithmetic array", "rtl/neuraedge_udivider.sv": "performs deterministic unsigned integer division one quotient bit per cycle", "rtl/neuraedge_universal_core.sv": "decodes commands and arbitrates the tensor, vector, normalization, LIF, LUT, attention, backward, and optimizer engines through a staged result path", "rtl/neuraedge_vector_alu.sv": "executes lane-wise residual, activation, comparison, requantization, and pooling primitives", "rtl/neuraedge_weight_bank.sv": "stores one 16-column by 2,048-entry weight tile in banked FPGA memory", "software/benchmark_compiler_memory.py": "measures time, memory, command count, and byte traffic for the largest real Spikingformer projection", "software/evaluate_spikingformer.py": "evaluates the source checkpoint and exported integer arithmetic over the complete CIFAR-10 test set", "software/export_cifar10_shards.py": "writes deterministic CIFAR-10 training and validation shards for bounded-memory board-side learning", "software/export_spikingfo "folds BatchNorm, calibrates quantizers, and writes aligned hashed hardware parameter sections", "software/generate_debug_report "turns full-test predictions, compiler manifests, XSim waveforms, and final Vivado reports into auditable plots and PDFs", "software/generate_operator_fixtures.py": "builds independent LayerNorm, softmax, vector, optimizer, and LUT vectors for RTL tests", "software/generate_source_reference.py": "generates this source-reference volume and its source-integrity manifest", "software/generate_spikingformer_manifest.py": "derives every model tensor shape, tile schedule, command count, operation count, and traffic total", "software/generate_training_coverage.py": "records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy", "software/generate_tensor_tile_fixture.py": "builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison", "software/neuraedge_compiler.py": "tiles matrices and convolutions into validated accelerator commands and packed stream payloads", "software/neuraedge_golden.py": "implements independent mathematical reference models for every hardware arithmetic primitive", "software/neuraedge_protocol.py "defines command bit fields, descriptor records, packed streams, and result decoders", "software/neuraedge_pynq.py": "controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry", "software/neuraedge_training.p "builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates", "software/run_spikingformer_pynq.py": "loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay", "software/spikingformer_accelerator.py": "maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations", "software/spikingformer_parameters.py": "validates and exposes typed read-only views of the aligned parameter archive", "software/spikingformer_quantization.py": "defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math", "software/spikingformer_reference.p "reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison", "software/spikingformer_tr "runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation",

"software/train_spikingformer_pynq.py": "provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports", }‘ to ‘FILE_PURPOSES ‘ in host memory. Here FILE_PURPOSES means the value named 'file purposes' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 72: `FILE_DATAFLOWS = {` This statement continues through source line 102; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv":

"Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 73: `"rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. Durin...` This is continuation line 2 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories

and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Check-

point weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 74: `"rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A...` This is continuation line 3 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_lut8: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv":

"The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 75: `"rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction ...` This is continuation line 4 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.",‘. The important operands are rtl: the value named ’rtl’ in this block; neuraedge_active_index_buffer: local storage that decouples producer and consumer timing; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile’s 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command’s payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP’s signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine’s counters from entering another engine’s timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or

clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 76: `"rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block count...` This is continuation line 5 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_alpha_xnor: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves

one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both

emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 77: `"rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physica...` This is continuation line 6 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_buffered_tensor_core: local storage that decouples producer and consumer timing; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv":

"The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 78: `"rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result len...` This is continuation line 7 of the 31-line logical statement that begins at source line 72. Its exact fragment is '"rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.",'. The important operands are rtl: the value named 'rtl' in this block; neuraedge_descriptor_sequencer: a DMA record header plus one command and its declared payload; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns '{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or

clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 79: `"rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squar...` This is continuation line 8 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_integer_layernorm: the value named 'neuraedge integer layernorm' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves

one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both

emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 80: `"rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainde...` This is continuation line 9 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_isqrt64: the value named 'neuraedge isqrt64' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned

byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 81: `"rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, add...` This is continuation line 10 of the 31-line logical statement that begins at source line 72. Its

exact fragment is ‘"rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_lif_bank: the value named 'neuraedge lif bank' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks

feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 82: `"rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output featu...` This is continuation line 11 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_linear_backward8: the value named 'neuraedge linear backward8' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves

one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both

emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 83: `"rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative ...` This is continuation line 12 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_log_softmax: the value named 'neuraedge log softmax' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned

byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 84: `"rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one a...` This is continuation line 13 of the 31-line logical statement that begins at source line 72. Its exact frag-

ment is ‘"rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_mac2_u8: the value named 'neuraedge mac2 u8' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to

support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 85: `"rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and...` This is continuation line 14 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_pynq_axi_top: the value named 'neuraedge pynq axi top' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and

result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native

block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 86: `"rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum veloci...` This is continuation line 15 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_sgd_optimizer8: the value named 'neuraedge sgd optimizer8' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices

recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 87: `"rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lane...` This is continuation line 16 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse

popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_tensor_tile: the value named 'neuraedge tensor tile' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-

sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 88: `"rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, co...` This is continuation line 17 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_udivider: the value named 'neuraedge udivider' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.",

"rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported

integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 89: `"rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared s...` This is continuation line 18 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_universal_core: the value named 'neuraedge universal core' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is

the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 90: `"rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conver...` This is continuation line 19 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.",‘. The important operands are rtl: the value named 'rtl'

in this block; neuraedge_vector_alu: the value named 'neuraedge vector alu' in this block; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns '{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimen-

sions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 91: `"rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight ...` This is continuation line 20 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.",‘. The important operands are rtl: the value named 'rtl' in this block; neuraedge_weight_bank: a learned coefficient multiplied by an activation; sv: the value named 'sv' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It

derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "soft-

ware/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 92: `"software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and wei...` This is continuation line 21 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.",‘. The important operands are software: the value named 'software' in this block; neuraedge_compiler: the value named 'neuraedge compiler' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and

telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 93: `"software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-b...` This is continuation line 22 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.",‘. The important operands are software: the value named 'software' in this block; neuraedge_protocol: the value named 'neuraedge protocol' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load

command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and

decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 94: `"software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous ...` This is continuation line 23 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.",‘. The important operands are software: the value named 'software' in this block; neuraedge_pynq: the value named 'neuraedge pynq' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One

iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor pro-

grams. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 95: `"software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss ...` This is continuation line 24 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.",‘. The important operands are software: the value named 'software' in this block; neuraedge_training: the value named 'neuraedge training' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum

velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 96: `"software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint ...` This is continuation line 25 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.",‘. The important operands are software: the value named 'software' in this block; spikingformer_accelerator: the value named 'spikingformer accelerator' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries.

During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a

software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 97: `"software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into a...` This is continuation line 26 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.",‘. The important operands are software: the value named 'software' in this block; export_spikingformer_hardware: the value named 'export spikingformer hardware' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand

bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are check-

pointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 98: `"software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rer...` This is continuation line 27 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.",‘. The important operands are software: the value named 'software' in this block; generate_debug_report: the value named 'generate debug report' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16

tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 99: `"software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep a...` This is continuation line 28 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.",‘. The important operands are software: the value named 'software' in this block; spikingformer_reference: the value named 'spikingformer reference' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table.

A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.",

"software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 100: `"software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the...` This is continuation line 29 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.",‘. The important operands are software: the value named 'software' in this block; spikingformer_training_runtime: the value named 'spikingformer training runtime' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor,

and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state,

hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 101: `"software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter arch...` This is continuation line 30 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘"software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.",‘. The important operands are software: the value named 'software' in this block; train_spikingformer_pynq: the value named 'train spikingformer pynq' in this block; py: the value named 'py' in this block. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.",

"rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 102: } This is continuation line 31 of the 31-line logical statement that begins at source line 72. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "rtl/neuraedge_activation_buffer.sv": "A load command writes activation bytes into addressed entries. During compute, the scheduler presents reduction indices and this memory returns one activation vector per selected row; keeping the tile local avoids rereading DDR for every output column.", "rtl/neuraedge_activation_lut8.sv": "Software first streams address/value pairs into the 256-entry table. A later execute command reads eight addresses in parallel, returns eight programmed nonlinear values, and counts only newly initialized addresses so duplicate writes cannot falsely report a complete table.", "rtl/neuraedge_active_index_buffer.sv": "While activations load, the tensor core tests whether a reduction position is all zero across active rows. Only nonzero positions are written here; compute

later walks this compact list, so sparse positions remove issue cycles rather than performing useless zero arithmetic.", "rtl/neuraedge_alpha_xnor.sv": "The input stream supplies packed query and key spike masks. The block counts matching one bits and matching zero bits, weights zero matches by programmable alpha, accumulates across chunks, and emits one fixed-point attention score under valid/ready backpressure.", "rtl/neuraedge_buffered_tensor_core.sv": "This is the tile-level controller around memories and the physical array. Commands are predecoded, validated, and then routed through activation load, weight load, sparse-index formation, issue FIFO, accumulation, and ordered result output states. A two-entry skid FIFO reserves one beat for registered READY latency, decouples the tile's 128-entry result store from parent backpressure, and delays command completion until the buffered final result is actually transferred.", "rtl/neuraedge_descriptor_sequencer.sv": "A DMA record header declares one command's payload and result lengths. The sequencer captures both halves of the 128-bit command, forwards exactly the declared payload, counts accepted results, regenerates final TLAST, and drains malformed packets so the next descriptor starts aligned.", "rtl/neuraedge_integer_layernorm.sv": "The block first stores a vector while accumulating its sum and squared sum. It derives integer mean and variance, invokes the iterative square-root and divider, applies per-element gamma and beta, saturates the result, and preserves output data whenever downstream ready is low.", "rtl/neuraedge_isqrt64.sv": "One iteration examines the next two radicand bits, shifts the partial remainder, conditionally subtracts the trial divisor, and appends one root bit. After 32 iterations the 32-bit result is exactly floor(sqrt(radicand)); a widened subtraction bit makes borrow detection explicit.", "rtl/neuraedge_lif_bank.sv": "Each of 32 lanes reads persistent membrane state, applies the leak shift, adds signed input current, compares against threshold, emits a spike, and writes either reset or integrated membrane back. Segmented counters avoid a long telemetry carry chain on the 200 MHz datapath.", "rtl/neuraedge_linear_backward8.sv": "A command declares up to eight rows, input features, and output features. Streamed activations, signed weights, and output gradients are buffered in four-lane halves; sixteen signed DSP products are accumulated per cycle. The controller first reduces dY times transposed W into dX, then transposed X times dY into dW, retaining 47-bit partial sums across tiled commands when requested. Separate magnitude, rounding-bias, shift, sign, and saturation states preserve the reference round-once result while breaking the long FPGA timing path.", "rtl/neuraedge_log_softmax.sv": "The engine buffers logits, finds their maximum, converts bounded negative differences to power-of-two terms, sums the denominator, divides each term into a fixed probability scale, and streams normalized outputs without a floating-point exponential unit.", "rtl/neuraedge_mac2_u8.sv": "Two unsigned byte weights are separated by a guard bit and multiplied by one activation in a DSP48E1. Registered correction handles the DSP's signed interpretation, then slices recover two independent exact 16-bit products without cross-field carry.", "rtl/neuraedge_pynq_axi_top.sv": "This is the software-visible shell. AXI-Lite writes configure command and descriptor registers, AXI-Stream carries DMA data, registered credit boundaries break READY timing paths, and status/interrupt registers expose completion, errors, and telemetry.", "rtl/neuraedge_sgd_optimizer8.sv": "For eight lanes, the engine reads weight, gradient, and momentum velocity, computes the fixed-point momentum update, rounds the learning-rate product symmetrically, saturates the new signed weight, stores new velocity/weight state, and streams the result.", "rtl/neuraedge_tensor_tile.sv": "The 8-by-16 tile schedules 64 packed DSP blocks as 128 dense byte MAC lanes, or bypasses multipliers for binary/ternary conditional additions. Per-row active masks, registered sparse popcount, issue/result FIFOs, and deterministic row/column output order make backpressure safe.", "rtl/neuraedge_udivider.sv": "A restoring divider shifts one dividend bit into the remainder each cycle, compares/subtracts the divisor, and writes one quotient bit. Divide-by-zero is reported explicitly instead of producing an accidental numeric value.", "rtl/neuraedge_universal_core.sv": "This command arbiter gives exactly one engine ownership of the shared streams. It stages LUT loads, LIF input, and other multibeat commands, then captures each engine into a source-local result bank before a registered shared output stage. Completion waits for the final AXI transfer, so stalls cannot acknowledge a command early; local accept signals prevent an inactive engine's counters from entering another engine's timing cone.", "rtl/neuraedge_vector_alu.sv": "Eight signed lanes execute residual add, ReLU, requantization, spike conversion, surrogate-gradient gating, maximum, Q15 multiply, or clamp. Wide intermediate products are registered before rounding and saturation so fixed-point behavior is deterministic.", "rtl/neuraedge_weight_bank.sv": "Each bank stores sixteen weight bytes for every reduction position. Eight banks feed outer mode rows or adjacent reduction groups, allowing the same memory organization to support matrix tiles and narrow dot-product schedules.", "software/neuraedge_compiler.py": "Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.", "software/neuraedge_protocol.py": "This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.", "software/neuraedge_pynq.py": "The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.", "software/neuraedge_training.py": "The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload

length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.", "software/spikingformer_accelerator.py": "The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.", "software/export_spikingformer_hardware.py": "Checkpoint weights and BatchNorm statistics are folded into affine projections, calibrated, quantized, aligned to 64-byte sections, and hashed. The JSON manifest records offsets, shapes, dtypes, scales, and section digests consumed by both emulator and board runtime.", "software/generate_debug_report.py": "This evidence builder reads immutable result artifacts instead of rerunning or inventing measurements. It parses VCD transitions and Vivado text tables, plots all standard-test predictions and hardware reports, embeds the native block design, and records every plot's source artifact.", "software/spikingformer_reference.py": "The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.", "software/spikingformer_training_runtime.py": "Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.", "software/train_spikingformer_pynq.py": "The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.", }‘ to ‘FILE_DATAFLOWS ‘ in host memory. Here FILE_DATAFLOWS means the value named 'file dataflows' in this block. The important operands are rtl: the value named 'rtl' in this block; neuraedge_activation_buffer: an input feature value consumed by a neuron or multiply-accumulate operation; sv: the value named 'sv' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 105: `IDENTIFIER_MEANINGS = {` This statement continues through source line 153; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file

whose role is to generates this source-reference volume and its source-integrity manifest.

Line 106: `"clk": "the clock that makes all registers advance together",` This is continuation line 2 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"clk": "the clock that makes all registers advance together",‘. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 107: `"aclk": "the AXI and accelerator clock",` This is continuation line 3 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"aclk": "the AXI and accelerator clock",‘. The important operands are aclk: the AXI and accelerator clock; the: the value named ’the’ in this block; AXI: the value named ’axi’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code

representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 108: `"rst_n": "the active-low synchronous reset",` This is continuation line 4 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"rst_n": "the active-low synchronous reset",‘. The important operands are rst_n: the active-low synchronous reset; the: the value named ’the’ in this block; active: the value named ’active’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 109: `"aresetn": "the active-low AXI reset",` This is continuation line 5 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"aresetn": "the active-low AXI reset",‘. The important operands

are aresetn: the active-low AXI reset; the: the value named 'the' in this block; active: the value named 'active' in this block. Read together, the complete operation is: This assigns '{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }' to 'IDENTIFIER_MEANINGS ' in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 110: `"valid": "the producer's statement that its current payload is meaningful",` This is continuation line 6 of the 49-line logical statement that begins at source line 105. Its exact fragment is '"valid": "the producer's statement that its current payload is meaningful",'. The important operands are valid: the producer's statement that its current payload is meaningful; the: the value named 'the' in this block; producer: the value named 'producer' in this block. Read together, the complete operation is: This assigns '{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normaliza-

tion", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 111: `"ready": "the consumer's statement that it can accept a payload this cycle",` This is continuation line 7 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"ready": "the consumer's statement that it can accept a payload this cycle",‘. The important operands are ready: the consumer's statement that it can accept a payload this cycle; the: the value named 'the' in this block; consumer: the value named 'consumer' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 112: `"last": "the AXI-Stream marker for the final beat of a packet or command payload",` This is continuation line 8 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"last": "the AXI-Stream marker for the final beat of a packet or command payload",‘. The important operands are last: the AXI-Stream marker for the final beat of a packet or command payload; the: the value named 'the' in this block; AXI: the value named 'axi' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes

all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 113: `"state": "the finite-state-machine phase retained in flip-flops",` This is continuation line 9 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"state": "the finite-state-machine phase retained in flip-flops",‘. The important operands are state: the finite-state-machine phase retained in flip-flops; the: the value named 'the' in this block; finite: the value named 'finite' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the

direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 114: `"command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters",` This is continuation line 10 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters",‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; the: the value named 'the' in this block; bit: the value named 'bit' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 115: `"descriptor": "a DMA record header plus one command and its declared payload",` This is continuation line 11 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"descriptor": "a DMA record header plus one command and its declared payload",‘. The important operands are descriptor: a DMA record header plus one command and its declared payload; a: the value named 'a' in this block; DMA: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that

its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 116: `"input_remaining": "the number of command payload beats still expected",` This is continuation line 12 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"input_remaining": "the number of command payload beats still expected",‘. The important operands are input_remaining: the number of command payload beats still expected; the: the value named 'the' in this block; number: the value named 'number' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between

DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 117: `"output_remaining": "the number of command result beats still expected",` This is continuation line 13 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"output_remaining": "the number of command result beats still expected",‘. The important operands are output_remaining: the number of command result beats still expected; the: the value named 'the' in this block; number: the value named 'number' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 118: `"program_output_remaining": "the number of result beats left in the complete descriptor program",` This is continuation line 14 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"program_output_remaining": "the number of result beats left in the complete descriptor program",‘. The important operands are program_output_remaining: the number of result beats left in the complete descriptor program; the: the value named 'the' in this block; number: the value named 'number' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the

AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 119: `"activation": "an input feature value consumed by a neuron or multiply-accumulate operation",` This is continuation line 15 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"activation": "an input feature value consumed by a neuron or multiply-accumulate operation",‘. The important operands are activation: an input feature value consumed by a neuron or multiply-accumulate operation; an: the value named 'an' in this block; input: the value named 'input' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or

error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 120: `"weight": "a learned coefficient multiplied by an activation",` This is continuation line 16 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"weight": "a learned coefficient multiplied by an activation",‘. The important operands are weight: a learned coefficient multiplied by an activation; a: the value named 'a' in this block; learned: the value named 'learned' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 121: `"bias": "an additive learned offset applied after accumulation",` This is continuation line 17 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"bias": "an additive learned offset applied after accumulation",‘. The important operands are bias: an additive learned offset applied after accumulation; an: the value named 'an' in this block; additive: the value named 'additive' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parame-

ters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 122: `"accumulator": "a widened running sum that prevents intermediate overflow",` This is continuation line 18 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"accumulator": "a widened running sum that prevents intermediate overflow",‘. The important operands are accumulator: a widened running sum that prevents intermediate overflow; a: the value named ’a’ in this block; widened: the value named ’widened’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples

producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 123: `"membrane": "the persistent LIF neuron state carrying charge between timesteps",` This is continuation line 19 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"membrane": "the persistent LIF neuron state carrying charge between timesteps",‘. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; the: the value named 'the' in this block; persistent: the value named 'persistent' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 124: `"threshold": "the membrane level at which a neuron emits a spike",` This is continuation line 20 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"threshold": "the membrane level at which a neuron emits a spike",‘. The important operands are threshold: the membrane level at which a neuron emits a spike; the: the value named 'the' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining":

"the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 125: `"leak": "the retained fraction of prior membrane state",` This is continuation line 21 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"leak": "the retained fraction of prior membrane state",‘. The important operands are leak: the retained fraction of prior membrane state; the: the value named ’the’ in this block; retained: the value named ’retained’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position

processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 126: `"spike": "a binary event produced when a neuron crosses its threshold",` This is continuation line 22 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"spike": "a binary event produced when a neuron crosses its threshold",‘. The important operands are spike: a binary event produced when a neuron crosses its threshold; a: the value named ’a’ in this block; binary: the value named ’binary’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 127: `"dense_mac": "a count of ordinary activation-times-weight operations actually issued",` This is continuation line 23 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"dense_mac": "a count of ordinary activation-times-weight operations actually issued",‘. The important operands are dense_mac: a count of ordinary activation-times-weight operations actually issued; a: the value named ’a’ in this block; count: the value named ’count’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron

or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 128: `"spike_add": "a count of event-driven conditional weight additions actually issued",` This is continuation line 24 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"spike_add": "a count of event-driven conditional weight additions actually issued",‘. The important operands are spike_add: a count of event-driven conditional weight additions actually issued; a: the value named ’a’ in this block; count: the value named ’count’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the

greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 129: `"zero_point": "the integer code representing real numeric zero in affine quantization",` This is continuation line 25 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"zero_point": "the integer code representing real numeric zero in affine quantization",‘. The important operands are zero_point: the integer code representing real numeric zero in affine quantization; the: the value named ’the’ in this block; integer: the value named ’integer’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 130: `"scale": "the real-value spacing represented by one quantized integer step",` This is continuation line 26 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"scale": "the real-value spacing represented by one quantized integer step",‘. The important operands are scale: the real-value spacing represented by one quantized integer step; the: the value named ’the’ in this block; real: the value named ’real’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persis-

tent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 131: `"requant":  "the conversion of a wide accumulated result back into a bounded activation code",`
This is continuation line 27 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"requant": "the conversion of a wide accumulated result back into a bounded activation code",‘. The important operands are requant: the conversion of a wide accumulated result back into a bounded activation code; the: the value named ’the’ in this block; conversion: the value named ’conversion’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance

together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 132: `"alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention",` This is continuation line 28 of the 49-line logical statement that begins at source line 105. Its exact fragment is '"alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention",'. The important operands are alpha: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; the: the value named 'the' in this block; programmable: the value named 'programmable' in this block. Read together, the complete operation is: This assigns '{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }' to 'IDENTIFIER_MEANINGS ' in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 133: `"gamma": "the learned multiplicative LayerNorm parameter",` This is continuation line 29 of the 49-line logical statement that begins at source line 105. Its exact fragment is '"gamma": "the learned multiplicative LayerNorm parameter",'. The important operands are gamma: the learned multiplicative LayerNorm parameter; the: the value named 'the' in this block; learned: the value named 'learned' in this block. Read together, the complete operation is: This assigns '{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane

state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 134: `"beta": "the learned additive LayerNorm parameter",` This is continuation line 30 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"beta": "the learned additive LayerNorm parameter",‘. The important operands are beta: the learned additive LayerNorm parameter; the: the value named ’the’ in this block; learned: the value named ’learned’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 135: `"mean": "the arithmetic average used by normalization",` This is continuation line 31 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"mean": "the arithmetic average used by normalization",‘. The important operands are mean: the arithmetic average used by normalization; the: the value named ’the’ in this block; arithmetic: the value named ’arithmetic’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 136: `"variance": "the average squared distance from the mean",` This is continuation line 32 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"variance": "the average squared distance from the mean",‘. The important operands are variance: the average squared distance from the mean; the: the value named ’the’ in this block; average: the value named ’average’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result

back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 137: `"gradient": "the signed loss derivative used to update a learned parameter",` This is continuation line 33 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"gradient": "the signed loss derivative used to update a learned parameter",‘. The important operands are gradient: the signed loss derivative used to update a learned parameter; the: the value named ’the’ in this block; loss: the value named ’loss’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 138: `"velocity": "the momentum history retained by the SGD optimizer",` This is continuation line 34 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"velocity": "the momentum history

retained by the SGD optimizer",‘. The important operands are velocity: the momentum history retained by the SGD optimizer; the: the value named 'the' in this block; momentum: the value named 'momentum' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 139: `"learning_rate": "the fixed-point multiplier controlling update magnitude",` This is continuation line 35 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"learning_rate": "the fixed-point multiplier controlling update magnitude",‘. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude; the: the value named 'the' in this block; fixed: the value named 'fixed' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm

parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 140: `"dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic",` This is continuation line 36 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic",‘. The important operands are dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; the: the value named 'the' in this block; direct: the value named 'direct' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 141: `"interrupt": "the hardware notification that wakes software after completion or error",` This is continuation line 37 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"interrupt": "the hardware notification that wakes software after completion or error",‘. The important operands are interrupt: the

hardware notification that wakes software after completion or error; the: the value named 'the' in this block; hardware: the value named 'hardware' in this block. Read together, the complete operation is: This assigns '{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }' to 'IDENTIFIER_MEANINGS ' in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 142: `"counter": "a hardware telemetry register that accumulates measured events",` This is continuation line 38 of the 49-line logical statement that begins at source line 105. Its exact fragment is '"counter": "a hardware telemetry register that accumulates measured events",'. The important operands are counter: a hardware telemetry register that accumulates measured events; a: the value named 'a' in this block; hardware: the value named 'hardware' in this block. Read together, the complete operation is: This assigns '{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by

normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 143: `"records": "the number of descriptor records accepted since program start",` This is continuation line 39 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"records": "the number of descriptor records accepted since program start",‘. The important operands are records: the number of descriptor records accepted since program start; the: the value named 'the' in this block; number: the value named 'number' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 144: `"payload": "stream data belonging to the current command",` This is continuation line 40 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"payload": "stream data belonging to the current command",‘. The important operands are payload: stream data belonging to the current command; stream: the value named 'stream' in this block; data: the value named 'data' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock",

"rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 145: `"buffer": "local storage that decouples producer and consumer timing",` This is continuation line 41 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"buffer": "local storage that decouples producer and consumer timing",‘. The important operands are buffer: local storage that decouples producer and consumer timing; local: the value named 'local' in this block; storage: the value named 'storage' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate":

"the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 146: `"index": "an integer selecting one tensor element, lane, bank, or state entry",` This is continuation line 42 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"index": "an integer selecting one tensor element, lane, bank, or state entry",‘. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry; an: the value named 'an' in this block; integer: the value named 'integer' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 147: `"lane": "one parallel arithmetic element within a vector or tensor datapath",` This is continuation line 43 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"lane": "one parallel arithmetic element within a vector or tensor datapath",‘. The important operands are lane: one parallel arithmetic element within a vector or tensor datapath; one: the value named 'one' in this block; parallel: the value named 'parallel' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's

statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 148: `"row": "one output-row position processed by the tensor tile",` This is continuation line 44 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"row": "one output-row position processed by the tensor tile",‘. The important operands are row: one output-row position processed by the tensor tile; one: the value named 'one' in this block; output: the value named 'output' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates

measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 149: `"column": "one output-feature position processed in parallel",` This is continuation line 45 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"column": "one output-feature position processed in parallel",‘. The important operands are column: one output-feature position processed in parallel; one: the value named 'one' in this block; output: the value named 'output' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 150: `"timestep": "one discrete SNN update instant",` This is continuation line 46 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"timestep": "one discrete SNN update instant",‘. The important operands are timestep: one discrete SNN update instant; one: the value named 'one' in this block; discrete: the value named 'discrete' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus

one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named ’the’ in this block; clock: the value named ’clock’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 151: `"logits": "the ten unnormalized classifier scores used for argmax",` This is continuation line 47 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"logits": "the ten unnormalized classifier scores used for argmax",‘. The important operands are logits: the ten unnormalized classifier scores used for argmax; the: the value named ’the’ in this block; ten: the value named ’ten’ in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer’s statement that its current payload is meaningful", "ready": "the consumer’s statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector

or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 152: `"prediction": "the class index with the greatest classifier score",` This is continuation line 48 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘"prediction": "the class index with the greatest classifier score",‘. The important operands are prediction: the class index with the greatest classifier score; the: the value named 'the' in this block; class: the value named 'class' in this block. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 153: `}` This is continuation line 49 of the 49-line logical statement that begins at source line 105. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "clk": "the clock that makes all registers advance together", "aclk": "the AXI and accelerator clock", "rst_n": "the active-low synchronous reset", "aresetn": "the active-low AXI reset", "valid": "the producer's statement that its current payload is meaningful", "ready": "the consumer's statement that it can accept a payload this cycle", "last": "the AXI-Stream marker for the final beat of a packet or command payload", "state": "the finite-state-machine phase retained in flip-flops", "command": "the 128-bit instruction selecting an engine, shape, mode, and numeric parameters", "descriptor": "a DMA record header plus one command and its declared payload", "input_remaining": "the number of command payload beats still expected", "output_remaining": "the number of command result beats still expected", "program_output_remaining": "the number of result beats left in the complete descriptor program", "activation": "an input feature value consumed by a neuron or multiply-accumulate operation", "weight": "a learned coefficient multiplied by an activation", "bias": "an additive learned offset applied after accumulation", "accumulator": "a widened

running sum that prevents intermediate overflow", "membrane": "the persistent LIF neuron state carrying charge between timesteps", "threshold": "the membrane level at which a neuron emits a spike", "leak": "the retained fraction of prior membrane state", "spike": "a binary event produced when a neuron crosses its threshold", "dense_mac": "a count of ordinary activation-times-weight operations actually issued", "spike_add": "a count of event-driven conditional weight additions actually issued", "zero_point": "the integer code representing real numeric zero in affine quantization", "scale": "the real-value spacing represented by one quantized integer step", "requant": "the conversion of a wide accumulated result back into a bounded activation code", "alpha": "the programmable weight assigned to matching zero spikes in alpha-XNOR attention", "gamma": "the learned multiplicative LayerNorm parameter", "beta": "the learned additive LayerNorm parameter", "mean": "the arithmetic average used by normalization", "variance": "the average squared distance from the mean", "gradient": "the signed loss derivative used to update a learned parameter", "velocity": "the momentum history retained by the SGD optimizer", "learning_rate": "the fixed-point multiplier controlling update magnitude", "dma": "the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic", "interrupt": "the hardware notification that wakes software after completion or error", "counter": "a hardware telemetry register that accumulates measured events", "records": "the number of descriptor records accepted since program start", "payload": "stream data belonging to the current command", "buffer": "local storage that decouples producer and consumer timing", "index": "an integer selecting one tensor element, lane, bank, or state entry", "lane": "one parallel arithmetic element within a vector or tensor datapath", "row": "one output-row position processed by the tensor tile", "column": "one output-feature position processed in parallel", "timestep": "one discrete SNN update instant", "logits": "the ten unnormalized classifier scores used for argmax", "prediction": "the class index with the greatest classifier score", }‘ to ‘IDENTIFIER_MEANINGS ‘ in host memory. Here IDENTIFIER_MEANINGS means the arithmetic average used by normalization. The important operands are clk: the clock that makes all registers advance together; the: the value named 'the' in this block; clock: the value named 'clock' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 156: `STATE_MEANINGS = {` This statement continues through source line 170; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named 'idle' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 157: `"IDLE": "waits for a new operation and does not consume payload data",` This is continuation line 2 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"IDLE": "waits for a new operation and does not consume payload data",‘. The important operands are IDLE: the value named 'idle' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named 'idle' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 158: `"HEADER": "accepts and validates the descriptor record header",` This is continuation line 3 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"HEADER": "accepts and validates the descriptor record header",‘. The important operands are HEADER: the value named ’header’ in this block; accepts: the value named ’accepts’ in this block; and: the value named ’and’ in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named ’idle’ in this block; waits: the value named ’waits’ in this block; for: the value named ’for’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 159: `"COMMAND_LOW": "captures command bits 63 through 0",` This is continuation line 4 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"COMMAND_LOW": "captures command bits 63 through 0",‘. The important operands are COMMAND_LOW: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; captures: the value named ’captures’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named ’idle’ in this block; waits: the value named ’waits’ in this block; for: the value named ’for’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 160: `"COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST",` This is continuation line 5 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST",‘. The important operands are COMMAND_HIGH: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; captures: the value named ’captures’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named ’idle’ in this block; waits: the value named ’waits’ in this block; for: the value named ’for’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 161: `"SUBMIT": "holds command-valid until the universal core accepts the instruction",` This is continuation line 6 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"SUBMIT": "holds command-valid until the universal core accepts the instruction",‘. The important operands are SUBMIT: the

value named 'submit' in this block; holds: the value named 'holds' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named 'idle' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 162: `"PAYLOAD": "forwards exactly the declared number of input beats to the selected engine",` This is continuation line 7 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"PAYLOAD": "forwards exactly the declared number of input beats to the selected engine",‘. The important operands are PAYLOAD: stream data belonging to the current command; forwards: the value named 'forwards' in this block; exactly: the value named 'exactly' in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named 'idle' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 163: `"WAIT": "waits for both command completion and the declared output count",` This is continuation line 8 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"WAIT": "waits for both command completion and the declared output count",‘. The important operands are WAIT: the value named 'wait' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named 'idle' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 164: `"ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned",` This is continuation line 9 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned",‘. The important operands are ERROR_DRAIN: the value named 'error drain' in this block; discards: the value named 'discards' in this block; remaining: the value named 'remaining' in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the de-

scriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named 'idle' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 165: `"LOAD": "stores incoming operands into local FPGA memory",` This is continuation line 10 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"LOAD": "stores incoming operands into local FPGA memory",‘. The important operands are LOAD: the value named 'load' in this block; stores: the value named 'stores' in this block; incoming: the value named 'incoming' in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named 'idle' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 166: `"COMPUTE": "issues arithmetic work using values already held on chip",` This is continuation line 11 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"COMPUTE": "issues arithmetic work using values already held on chip",‘. The important operands are COMPUTE: the value named 'compute' in this block; issues: the value named 'issues' in this block; arithmetic: the value named 'arithmetic' in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named 'idle' in this block; waits: the value named 'waits' in this block; for: the value named 'for' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 167: `"OUTPUT": "presents computed results under valid/ready backpressure",` This is continuation line 12 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"OUTPUT": "presents computed results under valid/ready backpressure",‘. The important operands are OUTPUT: the value named 'output' in this block; presents: the value named 'presents' in this block; computed: the value named 'computed' in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD":

"stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named ’idle’ in this block; waits: the value named ’waits’ in this block; for: the value named ’for’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 168: `"COUNT": "updates a segmented telemetry counter without a long carry chain",` This is continuation line 13 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"COUNT": "updates a segmented telemetry counter without a long carry chain",‘. The important operands are COUNT: the value named ’count’ in this block; updates: the value named ’updates’ in this block; a: the value named ’a’ in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named ’idle’ in this block; waits: the value named ’waits’ in this block; for: the value named ’for’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 169: `"DONE": "raises completion for one cycle and returns control to idle",` This is continuation line 14 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘"DONE": "raises completion for one cycle and returns control to idle",‘. The important operands are DONE: the value named ’done’ in this block; raises: the value named ’raises’ in this block; completion: the value named ’completion’ in this block. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named ’idle’ in this block; waits: the value named ’waits’ in this block; for: the value named ’for’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 170: `}` This is continuation line 15 of the 15-line logical statement that begins at source line 156. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "IDLE": "waits for a new operation and does not consume payload data", "HEADER": "accepts and validates the descriptor record header", "COMMAND_LOW": "captures command bits 63 through 0", "COMMAND_HIGH": "captures command bits 127 through 64 and checks input TLAST", "SUBMIT": "holds command-valid until the universal core accepts the instruction", "PAYLOAD": "forwards exactly the declared number of input beats to the selected engine", "WAIT": "waits for both command completion and the declared output count", "ERROR_DRAIN": "discards remaining DMA input until TLAST so the next transfer starts aligned", "LOAD": "stores incoming operands into local FPGA memory", "COMPUTE": "issues arithmetic work using values already held on chip", "OUTPUT": "presents computed results under valid/ready backpressure", "COUNT": "updates a segmented telemetry counter without a long carry chain", "DONE": "raises completion for one cycle and returns control to idle", }‘ to ‘STATE_MEANINGS ‘ in host memory. Here STATE_MEANINGS means the finite-state-machine phase retained in flip-flops. The important operands are IDLE: the value named ’idle’ in this block; waits: the value named ’waits’ in this

block; for: the value named 'for' in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 173: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to generates this source-reference volume and its source-integrity manifest; it creates no FPGA circuit by itself.

Line 174: `class SourceEntry:` This defines class SourceEntry, which groups data and methods for source entry. Instances exist in host memory and organize the flow that generates this source-reference volume and its source-integrity manifest.

Line 175: `path: Path` Inside the class SourceEntry, which handles source entry, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are path: the value named 'path' in this block; Path: the value named 'path' in this block.

Line 176: `relative: str` Inside the class SourceEntry, which handles source entry, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are relative: the value named 'relative' in this block; str: the value named 'str' in this block.

Line 177: `purpose: str` Inside the class SourceEntry, which handles source entry, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are purpose: the value named 'purpose' in this block; str: the value named 'str' in this block.

Line 180: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to generates this source-reference volume and its source-integrity manifest; it creates no FPGA circuit by itself.

Line 181: `class StatementSpan:` This defines class StatementSpan, which groups data and methods for statement span. Instances exist in host memory and organize the flow that generates this source-reference volume and its source-integrity manifest.

Line 182: `start: int` Inside the class StatementSpan, which handles statement span, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are start: the value named 'start' in this block.

Line 183: `end: int` Inside the class StatementSpan, which handles statement span, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The expression is a literal or structural operator and has no hidden software state.

Line 184: `text: str` Inside the class StatementSpan, which handles statement span, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are text: the value named 'text' in this block; str: the value named 'str' in this block.

Line 187: `def project_root() -> Path:` This starts function project_root. Its indented body performs project root as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 188: `return Path(__file__).resolve().parents[1]` This returns 'Path(___file___).resolve().parents[1]' to the caller of the function project_root, which handles project root, ending this invocation. The important operands are Path: the value named 'path' in this block; ___file___: the value named 'file' in this block; resolve: the value named 'resolve' in this block.

Line 191: `def source_entries(root: Path) -> list[SourceEntry]:` This starts function source_entries. Its indented body performs source entries as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 192: `patterns = (` This statement continues through source line 199; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(("rtl", "*.sv"), ("software", "*.py"), ("tb", "*.sv"), ("tests", "*.py"), ("scripts", "*.ps1"), ("scripts", "*.tcl"),)‘ to ‘patterns ‘ in host memory. Here patterns means the value named ’patterns’ in this block. The important operands are rtl: the value named ’rtl’ in this block; sv: the value named ’sv’ in this block; software: the value named ’software’ in this block. Context: the function source_entries, which handles source entries.

Line 193: `("rtl", "*.sv"),` This is continuation line 2 of the 8-line logical statement that begins at source line 192. Its exact fragment is ‘("rtl", "*.sv"),‘. The important operands are rtl: the value named ’rtl’ in this block; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘(("rtl", "*.sv"), ("software", "*.py"), ("tb", "*.sv"), ("tests", "*.py"), ("scripts", "*.ps1"), ("scripts", "*.tcl"),)‘ to ‘patterns ‘ in host memory. Here patterns means the value named ’patterns’ in this block. The important operands are rtl: the value named ’rtl’ in this block; sv: the value named ’sv’ in this block; software: the value named ’software’ in this block. Context: the function source_entries, which handles source entries.

Line 194: `("software", "*.py"),` This is continuation line 3 of the 8-line logical statement that begins at source line 192. Its exact fragment is ‘("software", "*.py"),‘. The important operands are software: the value named ’software’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘(("rtl", "*.sv"), ("software", "*.py"), ("tb", "*.sv"), ("tests", "*.py"), ("scripts", "*.ps1"), ("scripts", "*.tcl"),)‘ to ‘patterns ‘ in host memory. Here patterns means the value named ’patterns’ in this block. The important operands are rtl: the value named ’rtl’ in this block; sv: the value named ’sv’ in this block; software: the value named ’software’ in this block. Context: the function source_entries, which handles source entries.

Line 195: `("tb", "*.sv"),` This is continuation line 4 of the 8-line logical statement that begins at source line 192. Its exact fragment is ‘("tb", "*.sv"),‘. The important operands are tb: the value named ’tb’ in this block; sv: the value named ’sv’ in this block. Read together, the complete operation is: This assigns ‘(("rtl", "*.sv"), ("software", "*.py"), ("tb", "*.sv"), ("tests", "*.py"), ("scripts", "*.ps1"), ("scripts", "*.tcl"),)‘ to ‘patterns ‘ in host memory. Here patterns means the value named ’patterns’ in this block. The important operands are rtl: the value named ’rtl’ in this block; sv: the value named ’sv’ in this block; software: the value named ’software’ in this block. Context: the function source_entries, which handles source entries.

Line 196: `("tests", "*.py"),` This is continuation line 5 of the 8-line logical statement that begins at source line 192. Its exact fragment is ‘("tests", "*.py"),‘. The important operands are tests: the value named ’tests’ in this block; py: the value named ’py’ in this block. Read together, the complete operation is: This assigns ‘(("rtl", "*.sv"), ("software", "*.py"), ("tb", "*.sv"), ("tests", "*.py"), ("scripts", "*.ps1"), ("scripts", "*.tcl"),)‘ to ‘patterns ‘ in host memory. Here patterns means the value named ’patterns’ in this block. The important operands are rtl: the value named ’rtl’ in this block; sv: the value named ’sv’ in this block; software: the value named ’software’ in this block. Context: the function source_entries, which handles source entries.

Line 197: `("scripts", "*.ps1"),` This is continuation line 6 of the 8-line logical statement that begins at source line 192. Its exact fragment is ‘("scripts", "*.ps1"),‘. The important operands are scripts: the value named ’scripts’ in this block; ps1: the value named ’ps1’ in this block. Read together, the complete operation is: This assigns ‘(("rtl", "*.sv"), ("software", "*.py"), ("tb", "*.sv"), ("tests", "*.py"), ("scripts", "*.ps1"), ("scripts", "*.tcl"),)‘ to ‘patterns ‘ in host memory. Here patterns means the value named ’patterns’ in this block. The important operands are rtl: the value named ’rtl’ in this block; sv: the value named ’sv’ in this block; software: the value named ’software’ in this block. Context: the function source_entries, which handles source entries.

Line 198: `("scripts", "*.tcl"),` This is continuation line 7 of the 8-line logical statement that begins at source line 192. Its exact fragment is ‘("scripts", "*.tcl"),‘. The important operands are scripts: the value named ’scripts’ in this block; tcl: the value named ’tcl’ in this block. Read together, the complete operation is: This assigns ‘(("rtl", "*.sv"), ("software", "*.py"), ("tb", "*.sv"), ("tests", "*.py"), ("scripts", "*.ps1"), ("scripts", "*.tcl"),)‘ to ‘patterns ‘ in host memory. Here patterns means the value named ’patterns’ in this block. The important operands are rtl: the value named ’rtl’ in this block; sv: the value named ’sv’ in this block; software: the value named ’software’ in this block. Context: the function source_entries, which handles source entries.

Line 199: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 192. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(("rtl", "*.sv"), ("software", "*.py"), ("tb", "*.sv"), ("tests", "*.py"), ("scripts", "*.ps1"), ("scripts", "*.tcl"),)‘ to ‘patterns ‘ in host memory. Here patterns means the value named ’patterns’ in this block. The important operands are rtl: the value named ’rtl’ in this block; sv: the value named ’sv’ in this block; software: the value named ’software’ in this block. Context: the function source_entries, which handles source entries.

Line 200: `entries: list[SourceEntry] = []` This assigns ‘[]‘ to ‘entries‘ in host memory. Here entries means the value named ’entries’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function source_entries, which handles source entries.

Line 201: `for directory, pattern in patterns:` This iterates directory, pattern over ‘patterns‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function source_entries, which handles source entries.

Line 202: `for path in sorted((root / directory).glob(pattern)):` This iterates path over ‘sorted((root / directory).glob(pattern))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function source_entries, which handles source entries.

Line 203: `relative = path.relative_to(root).as_posix()` This assigns ‘path.relative_to(root).as_posix()‘ to ‘relative ‘ in host memory. Here relative means the value named ’relative’ in this block. The important operands are path: the value named ’path’ in this block; relative_to: the value named ’relative to’ in this block; root: the value named ’root’ in this block. Context: the function source_entries, which handles source entries.

Line 204: `purpose = FILE_PURPOSES.get(relative, inferred_file_purpose(relative))` This assigns ‘FILE_PURPOSES.
inferred_file_purpose(relative))‘ to ‘purpose ‘ in host memory. Here purpose means the value named ’purpose’ in this block. The important operands are FILE_PURPOSES: the value named ’file purposes’ in this block; get: the value named ’get’ in this block; relative: the value named ’relative’ in this block. Context: the function source_entries, which handles source entries.

Line 205: `entries.append(SourceEntry(path, relative, purpose))` This calls ‘entries.append‘ with ‘SourceEntry(path, relative, purpose)‘ inside the function source_entries, which handles source entries. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 206: `return entries` This returns ‘entries‘ to the caller of the function source_entries, which handles source entries, ending this invocation. The important operands are entries: the value named ’entries’ in this block.

Line 209: `def inferred_file_purpose(relative: str) -> str:` This starts function inferred_file_purpose. Its indented body performs inferred file purpose as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 210: `stem = Path(relative).stem` This assigns ‘Path(relative).stem‘ to ‘stem ‘ in host memory. Here stem means the value named ’stem’ in this block. The important operands are Path: the value named ’path’ in this block; relative: the value named ’relative’ in this block; stem: the value named ’stem’ in this block. Context: the function inferred_file_purpose, which handles inferred file purpose.

Line 211: `words = humanize(stem.removeprefix("tb_").removeprefix("test_"))` This assigns ‘humanize(stem.removeprefi
to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are humanize: the value named ’humanize’ in this block; stem: the value named ’stem’ in this block; removeprefix: the value named ’removeprefix’ in this block. Context: the function inferred_file_purpose, which handles inferred file purpose.

Line 212: `if relative.startswith("tb/"):` This executes the following indented host-code branch only when ‘relative.startswith("tb/")‘ is true. The important operands are relative: the value named ’relative’ in this block; startswith: the value named ’startswith’ in this block; tb: the value named ’tb’ in this block. Context: the function inferred_file_purpose, which handles inferred file purpose.

Line 213: `return f"drives, backpressures, and checks the {words} RTL block in simulation"` This returns ‘f"drives, backpressures, and checks the {words} RTL block in simulation"‘ to the caller of the function inferred_file_purpose, which handles inferred file purpose, ending this invocation. The important operands are f: the value named ’f’ in this block; drives: the value named ’drives’ in this block; backpressures: the value named ’backpressures’ in this block.

Line 214: `if relative.startswith("tests/"):` This executes the following indented host-code branch only when ‘relative.startswith("tests/")‘ is true. The important operands are relative: the value named ’relative’ in this block; startswith: the value named ’startswith’ in this block; tests: the value named ’tests’ in this block. Context: the function inferred_file_purpose, which handles inferred file purpose.

Line 215: `return f"checks {words} behavior independently with Python assertions"` This returns ‘f"checks {words} behavior independently with Python assertions"‘ to the caller of the function inferred_file_purpose, which handles inferred file purpose, ending this invocation. The important operands are f: the value named ’f’ in this block; checks: the value named ’checks’ in this block; words: the value named ’words’ in this block.

Line 216: `if relative.endswith(".tcl"):` This executes the following indented host-code branch only when ‘relative.endswith(".tcl")‘ is true. The important operands are relative: the value named ’relative’ in this block; endswith: the value named ’endswith’ in this block; tcl: the value named ’tcl’ in this block. Context: the function inferred_file_purpose, which handles inferred file purpose.

Line 217: `return f"runs the reproducible Vivado {words} hardware flow"` This returns ‘f"runs the reproducible Vivado {words} hardware flow"‘ to the caller of the function inferred_file_purpose, which handles inferred file purpose, ending this invocation. The important operands are f: the value named ’f’ in this block; runs: the value named ’runs’ in this block; the: the value named ’the’ in this block.

Line 218: `if relative.endswith(".ps1"):` This executes the following indented host-code branch only when ‘relative.endswith(".ps1")‘ is true. The important operands are relative: the value named ’relative’ in this block; endswith: the value named ’endswith’ in this block; ps1: the value named ’ps1’ in this block. Context: the function inferred_file_purpose, which handles inferred file purpose.

Line 219: `return f"orchestrates the repeatable Windows {words} command-line flow"` This returns ‘f"orchestrates the repeatable Windows {words} command-line flow"‘ to the caller of the function inferred_file_purpose, which handles inferred file purpose, ending this invocation. The important operands are f: the value named ’f’ in this block; orchestrates: the value named ’orchestrates’ in this block; the: the value named ’the’ in this block.

Line 220: `return f"implements the {words} part of the project"` This returns ‘f"implements the {words} part of the project"‘ to the caller of the function inferred_file_purpose, which handles inferred file purpose, ending this invocation. The important operands are f: the value named ’f’ in this block; implements: the value named ’implements’ in this block; the: the value named ’the’ in this block.

Line 223: `def humanize(identifier: str) -> str:` This starts function humanize. Its indented body performs humanize as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 224: `text = re.sub(r"([a-z0-9])([A-Z])", r"\1 \2", identifier)` This assigns ‘re.sub(r"([a-z0-9])([A-Z])", r"\1 \2", identifier)‘ to ‘text ‘ in host memory. Here text means the value named ’text’ in this block. The important operands are re: the value named ’re’ in this block; sub: the value named ’sub’ in this block; r: the value named ’r’ in this block. Context: the function humanize, which handles humanize.

Line 225: `return text.replace("_", " ").replace("-", " ").strip().lower()` This returns ‘text.replace("_", " ").replace("-", " ").strip().lower()‘ to the caller of the function humanize, which handles humanize, ending this invocation. The important operands are text: the value named ’text’ in this block; replace: the value named ’replace’ in this block; _: the value named ” in this block.

Line 228: `def identifier_meaning(identifier: str) -> str:` This starts function identifier_meaning. Its indented body performs identifier meaning as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 229: `lowered = identifier.lower().removesuffix("_r").removesuffix("_i").removesuffix("_o")` This assigns ‘identifier.lower().removesuffix("_r").removesuffix("_i").removesuffix("_o")‘ to ‘lowered ‘ in host memory. Here lowered means the value named ’lowered’ in this block. The important operands are identifier: the value named ’identifier’ in this block; lower: the value named ’lower’ in this block; removesuffix: the value named ’removesuffix’ in this block. Context: the function identifier_meaning, which handles identifier meaning.

Line 230: `if lowered in IDENTIFIER_MEANINGS:` This executes the following indented host-code branch only when ‘lowered in IDENTIFIER_MEANINGS‘ is true. The important operands are lowered: the value named ’lowered’ in this block; in: the value named ’in’ in this block; IDENTIFIER_MEANINGS: the arithmetic average used by normalization. Context: the function identifier_meaning, which handles identifier meaning.

Line 231: `return IDENTIFIER_MEANINGS[lowered]` This returns ‘IDENTIFIER_MEANINGS[lowered]‘ to the caller of the function identifier_meaning, which handles identifier meaning, ending this invocation. The important operands are IDENTIFIER_MEANINGS: the arithmetic average used by normalization; lowered: the value named ’lowered’ in this block.

Line 232: `matches = [meaning for key, meaning in IDENTIFIER_MEANINGS.items() if key in lowered]` This assigns ‘[meaning for key, meaning in IDENTIFIER_MEANINGS.items() if key in lowered]‘ to ‘matches ‘ in host memory. Here matches means the value named ’matches’ in this block. The important operands are meaning: the arithmetic average used by normalization; for: the value named ’for’ in this block; key: the value named ’key’ in this block. Context: the function identifier_meaning, which handles identifier meaning.

Line 233: `if matches:` This executes the following indented host-code branch only when ‘matches‘ is true. The important operands are matches: the value named ’matches’ in this block. Context: the function identifier_meaning, which handles identifier meaning.

Line 234: `return matches[0]` This returns ‘matches[0]‘ to the caller of the function identifier_meaning, which handles identifier meaning, ending this invocation. The important operands are matches: the value named ’matches’ in this block.

Line 235: `if lowered.endswith("_mem") or "memory" in lowered:` This executes the following indented host-code branch only when ‘lowered.endswith("_mem") or "memory" in lowered‘ is true. The important operands are lowered: the value named ’lowered’ in this block; endswith: the value named ’endswith’ in this block; _mem: local FPGA storage addressed by surrounding index logic. Context: the function identifier_meaning, which handles identifier meaning.

Line 236: `return "local FPGA storage addressed by surrounding index logic"` This returns ‘"local FPGA storage addressed by surrounding index logic"‘ to the caller of the function identifier_meaning, which handles identifier meaning, ending this invocation. The important operands are local: the value named ’local’ in this block; FPGA: the value named ’fpga’ in this block; storage: the value named ’storage’ in this block.

Line 237: `if lowered.endswith("_count") or lowered.endswith("_counter"):` This executes the following indented host-code branch only when ‘lowered.endswith("_count") or lowered.endswith("_counter")‘ is true. The important operands are lowered: the value named ’lowered’ in this block; endswith: the value named ’endswith’ in this block; _count: an integer count retained for control or measurable telemetry. Context: the function identifier_meaning, which handles identifier meaning.

Line 238: `return "an integer count retained for control or measurable telemetry"` This returns ‘"an integer count retained for control or measurable telemetry"‘ to the caller of the function identifier_meaning, which handles identifier meaning, ending this invocation. The important operands are an: the value named ’an’ in this block; integer: the value named ’integer’ in this block; count: the value named ’count’ in this block.

Line 239: `if lowered.endswith("_valid"):` This executes the following indented host-code branch only when ‘lowered.endswith("_valid")‘ is true. The important operands are lowered: the value named ’lowered’ in this block; endswith: the value named ’endswith’ in this block; _valid: the producer’s statement that its current payload is meaningful. Context: the function identifier_meaning, which handles identifier meaning.

Line 240: `return "a handshake flag saying the associated payload is meaningful"` This returns ‘"a handshake flag saying the associated payload is meaningful"‘ to the caller of the function identifier_meaning, which handles identifier meaning, ending this invocation. The important operands are a: the value named ’a’ in this block; handshake: the value named ’handshake’ in this block; flag: the value named ’flag’ in this block.

Line 241: `if lowered.endswith("_ready"):` This executes the following indented host-code branch only when ‘lowered.endswith("_ready")‘ is true. The important operands are lowered: the value named ’lowered’ in this block; endswith: the value named ’endswith’ in this block; _ready: the consumer’s statement that it can accept a payload this cycle. Context: the function identifier_meaning, which handles identifier meaning.

Line 242: `return "a handshake flag saying the associated consumer can accept data"` This returns ‘"a handshake flag saying the associated consumer can accept data"‘ to the caller of the function identifier_meaning, which handles identifier meaning, ending this invocation. The important operands are a: the value named ’a’ in this block; handshake: the value named ’handshake’ in this block; flag: the value named ’flag’ in this block.

Line 243: `if lowered.endswith("_data"):` This executes the following indented host-code branch only when ‘lowered.endswith("_data")‘ is true. The important operands are lowered: the value named ’lowered’ in this block; endswith: the value named ’endswith’ in this block; _data: the payload carried with the surrounding valid/ready handshake. Context: the function identifier_meaning, which handles identifier meaning.

Line 244: `return "the payload carried with the surrounding valid/ready handshake"` This returns ‘"the payload carried with the surrounding valid/ready handshake"‘ to the caller of the function identifier_meaning, which handles identifier meaning, ending this invocation. The important operands are the: the value named ’the’ in this block; payload: stream data belonging to the current command; carried: the value named ’carried’ in this block.

Line 245: `if lowered.endswith("_enable") or lowered.endswith("_en"):` This executes the following indented host-code branch only when ‘lowered.endswith("_enable") or lowered.endswith("_en")‘ is true. The important operands are lowered: the value named ’lowered’ in this block; endswith: the value named ’endswith’ in this block; _enable: a control bit that permits the associated state or storage update. Context: the function identifier_meaning, which handles identifier meaning.

Line 246: `return "a control bit that permits the associated state or storage update"` This returns ‘"a control bit that permits the associated state or storage update"‘ to the caller of the function identifier_meaning, which handles identifier meaning, ending this invocation. The important operands are a: the value named ’a’ in this block; control: the value named ’control’ in this block; bit: the value named ’bit’ in this block.

Line 247: `return f"the value named '{humanize(identifier)}' in this block"` This returns ‘f"the value named ’{humanize(identifier)}’ in this block"‘ to the caller of the function identifier_meaning, which handles identifier meaning, ending this invocation. The important operands are f: the value named ’f’ in this block; the: the value named ’the’ in this block; value: the value named ’value’ in this block.

Line 250: `def summarize_expression(expression: str) -> str:` This starts function summarize_expression. Its indented body performs summarize expression as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 251: `identifiers = re.findall(r"\b[A-Za-z_][A-Za-z0-9_$]*\b", expression)` This assigns ‘re.findall(r"\b[A-Za-z_][A-Za-z0-9_$]*\b", expression)‘ to ‘identifiers ‘ in host memory. Here identifiers means the value named ’identifiers’ in this block. The important operands are re: the value named ’re’ in this block; findall: the value named ’findall’ in this block; r: the value named ’r’ in this block. Context: the function summarize_expression, which handles summarize expression.

Line 252: `ignored = {` This statement continues through source line 255; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "if", "else", "begin", "end", "logic", "wire", "signed", "unsigned", "true", "false", "int", "return", "self", "np", "Path", }‘ to ‘ignored ‘ in host memory. Here ignored means the value named ’ignored’ in this block. The important operands are Path: the value named ’path’ in this block. Context: the function summarize_expression, which handles summarize expression.

Line 253: `"if", "else", "begin", "end", "logic", "wire", "signed", "unsigned",` This is continuation line 2 of the 4-line logical statement that begins at source line 252. Its exact fragment is ‘"if", "else", "begin", "end", "logic", "wire", "signed", "unsigned",‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "if", "else", "begin", "end", "logic", "wire", "signed", "unsigned", "true", "false", "int", "return", "self", "np", "Path", }‘ to ‘ignored ‘ in host memory. Here ignored means the value named ’ignored’ in this block. The important operands are Path: the value named ’path’ in this block. Context: the function summarize_expression, which handles summarize expression.

Line 254: `"true", "false", "int", "return", "self", "np", "Path",` This is continuation line 3 of the 4-line logical statement that begins at source line 252. Its exact fragment is ‘"true", "false", "int", "return", "self", "np", "Path",‘. The important operands are Path: the value named ’path’ in this block. Read together, the complete operation is: This assigns ‘{ "if", "else", "begin", "end", "logic", "wire", "signed", "unsigned", "true", "false", "int", "return", "self", "np", "Path", }‘ to ‘ignored ‘ in host memory. Here ignored means the value named ’ignored’ in this block. The important operands are Path: the value named ’path’ in this block. Context: the function summarize_expression, which handles summarize expression.

Line 255: `}` This is continuation line 4 of the 4-line logical statement that begins at source line 252. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "if", "else", "begin", "end", "logic", "wire", "signed", "unsigned", "true", "false", "int", "return", "self", "np", "Path", }‘ to ‘ignored ‘ in host memory. Here ignored means the value named ’ignored’ in this block. The important operands are Path: the value named ’path’ in this block. Context: the function summarize_expression, which handles summarize expression.

Line 256: `selected: list[str] = []` This assigns ‘[]‘ to ‘selected‘ in host memory. Here selected means the value named ’selected’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function summarize_expression, which handles summarize expression.

Line 257: `for identifier in identifiers:` This iterates identifier over ‘identifiers‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function summarize_expression, which handles summarize expression.

Line 258: `if identifier.lower() in ignored or identifier in selected:` This executes the following indented host-code branch only when ‘identifier.lower() in ignored or identifier in selected‘ is true. The important operands are identifier: the value named ’identifier’ in this block; lower: the value named ’lower’ in this block; in: the value named ’in’ in this block. Context: the function summarize_expression, which handles summarize expression.

Line 259: `continue` Inside the function summarize_expression, which handles summarize expression, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are continue: the value named ’continue’ in this block.

Line 260: `selected.append(identifier)` This calls ‘selected.append‘ with ‘identifier‘ inside the function summarize_expression, which handles summarize expression. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 261: `if len(selected) == 3:` This executes the following indented host-code branch only when ‘len(selected) == 3‘ is true. The important operands are len: the value named ’len’ in this block; selected: the value named ’selected’ in this block. Context: the function summarize_expression, which handles summarize expression.

Line 262: `break` Inside the function summarize_expression, which handles summarize expression, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are break: the value named 'break' in this block.

Line 263: `if not selected:` This executes the following indented host-code branch only when ‘not selected‘ is true. The important operands are not: the value named 'not' in this block; selected: the value named 'selected' in this block. Context: the function summarize_expression, which handles summarize expression.

Line 264: `return "The expression is a literal or structural operator and has no hidden software state."` This returns ‘"The expression is a literal or structural operator and has no hidden software state."‘ to the caller of the function summarize_expression, which handles summarize expression, ending this invocation. The important operands are The: the value named 'the' in this block; expression: the value named 'expression' in this block; is: the value named 'is' in this block.

Line 265: `meanings = "; ".join(f"{name}: {identifier_meaning(name)}" for name in selected)` This assigns ‘"; ".join(f"{name}: {identifier_meaning(name)}" for name in selected)‘ to ‘meanings ‘ in host memory. Here meanings means the arithmetic average used by normalization. The important operands are join: the value named 'join' in this block; f: the value named 'f' in this block; name: the value named 'name' in this block. Context: the function summarize_expression, which handles summarize expression.

Line 266: `return f"The important operands are {meanings}."` This returns ‘f"The important operands are {meanings}."‘ to the caller of the function summarize_expression, which handles summarize expression, ending this invocation. The important operands are f: the value named 'f' in this block; The: the value named 'the' in this block; important: the value named 'important' in this block.

Line 269: `def python_contexts(text: str) -> dict[int, str]:` This starts function python_contexts. Its indented body performs python contexts as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 270: `contexts: dict[int, tuple[int, str]] = {}` This assigns ‘{}‘ to ‘contexts‘ in host memory. Here contexts means the value named 'contexts' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function python_contexts, which handles python contexts.

Line 271: `try:` This is explicit host-side failure handling inside the function python_contexts, which handles python contexts; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 272: `tree = ast.parse(text)` This assigns ‘ast.parse(text)‘ to ‘tree ‘ in host memory. Here tree means the value named 'tree' in this block. The important operands are ast: the value named 'ast' in this block; parse: the value named 'parse' in this block; text: the value named 'text' in this block. Context: the function python_contexts, which handles python contexts.

Line 273: `except SyntaxError:` This is explicit host-side failure handling inside the function python_contexts, which handles python contexts; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 274: `return {}` This returns ‘{}‘ to the caller of the function python_contexts, which handles python contexts, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 275: `for node in ast.walk(tree):` This iterates node over ‘ast.walk(tree)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function python_contexts, which handles python contexts.

Line 276: `if isinstance(node, (ast.ClassDef, ast.FunctionDef, ast.AsyncFunctionDef)):` This executes the following indented host-code branch only when ‘isinstance(node, (ast.ClassDef, ast.FunctionDef, ast.AsyncFunctionDef))‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; node: the value named ’node’ in this block; ast: the value named ’ast’ in this block. Context: the function python_contexts, which handles python contexts.

Line 277: `end = getattr(node, "end_lineno", node.lineno)` This assigns ‘getattr(node, "end_lineno", node.lineno)‘ to ‘end ‘ in host memory. Here end means the value named ’end’ in this block. The important operands are getattr: the value named ’getattr’ in this block; node: the value named ’node’ in this block; end_lineno: the value named ’end lineno’ in this block. Context: the function python_contexts, which handles python contexts.

Line 278: `kind = "class" if isinstance(node, ast.ClassDef) else "function"` This assigns ‘"class" if isinstance(node, ast.ClassDef) else "function"‘ to ‘kind ‘ in host memory. Here kind means the value named ’kind’ in this block. The important operands are class: the value named ’class’ in this block; isinstance: the value named ’isinstance’ in this block; node: the value named ’node’ in this block. Context: the function python_contexts, which handles python contexts.

Line 279: `description = f"the {kind} {node.name}, which handles {humanize(node.name)}"` This assigns ‘f"the {kind} {node.name}, which handles {humanize(node.name)}"‘ to ‘description ‘ in host memory. Here description means the value named ’description’ in this block. The important operands are f: the value named ’f’ in this block; the: the value named ’the’ in this block; kind: the value named ’kind’ in this block. Context: the function python_contexts, which handles python contexts.

Line 280: `span = end - node.lineno` This assigns ‘end - node.lineno‘ to ‘span ‘ in host memory. Here span means the value named ’span’ in this block. The important operands are node: the value named ’node’ in this block; lineno: the value named ’lineno’ in this block. Context: the function python_contexts, which handles python contexts.

Line 281: `for line_number in range(node.lineno, end + 1):` This iterates line_number over ‘range(node.lineno, end + 1)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function python_contexts, which handles python contexts.

Line 282: `previous = contexts.get(line_number)` This assigns ‘contexts.get(line_number)‘ to ‘previous ‘ in host memory. Here previous means the value named ’previous’ in this block. The important operands are contexts: the value named ’contexts’ in this block; get: the value named ’get’ in this block; line_number: the value named ’line number’ in this block. Context: the function python_contexts, which handles python contexts.

Line 283: `if previous is None or span < previous[0]:` This executes the following indented host-code branch only when ‘previous is None or span < previous[0]‘ is true. The important operands are previous: the value named ’previous’ in this block; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function python_contexts, which handles python contexts.

Line 284: `contexts[line_number] = (span, description)` This assigns ‘(span, description)‘ to ‘contexts[line_number] ‘ in host memory. Here contexts means the value named ’contexts’ in this block. The important operands are span: the value named ’span’ in this block; description: the value named ’description’ in this block. Context: the function python_contexts, which handles python contexts.

Line 285: `return {line: value[1] for line, value in contexts.items()}` This returns ‘{line: value[1] for line, value in contexts.items()}‘ to the caller of the function python_contexts, which handles python contexts, ending this invocation. The important operands are line: the value named ’line’ in this block; value: the value named ’value’ in this block; for: the value named ’for’ in this block.

Line 288: `def _joined_statement(lines: list[str], start: int, end: int) -> str:` This starts function _joined_statement. Its indented body performs joined statement as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 289: `return " ".join(` This statement continues through source line 292; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘" ".join(fragment for fragment in (line.strip() for line in lines[start - 1:end]) if fragment)‘ to the caller of the function _joined_statement, which handles joined statement, ending this invocation. The important operands are join: the value named ’join’ in this block; fragment: the value named ’fragment’ in this block; for: the value named ’for’ in this block.

Line 290: `fragment for fragment in (line.strip() for line in lines[start - 1:end])` This is continuation line 2 of the 4-line logical statement that begins at source line 289. Its exact fragment is ‘fragment for fragment in (line.strip() for line in lines[start - 1:end])‘. The important operands are fragment: the value named ’fragment’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This returns ‘" ".join(fragment for fragment in (line.strip() for line in lines[start - 1:end]) if fragment)‘ to the caller of the function _joined_statement, which handles joined statement, ending this invocation. The important operands are join: the value named ’join’ in this block; fragment: the value named ’fragment’ in this block; for: the value named ’for’ in this block.

Line 291: `if fragment` This is continuation line 3 of the 4-line logical statement that begins at source line 289. Its exact fragment is ‘if fragment‘. The important operands are fragment: the value named ’fragment’ in this block. Read together, the complete operation is: This returns ‘" ".join(fragment for fragment in (line.strip() for line in lines[start - 1:end]) if fragment)‘ to the caller of the function _joined_statement, which handles joined statement, ending this invocation. The important operands are join: the value named ’join’ in this block; fragment: the value named ’fragment’ in this block; for: the value named ’for’ in this block.

Line 292: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 289. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘" ".join(fragment for fragment in (line.strip() for line in lines[start - 1:end]) if fragment)‘ to the caller of the function _joined_statement, which handles joined statement, ending this invocation. The important operands are join: the value named ’join’ in this block; fragment: the value named ’fragment’ in this block; for: the value named ’for’ in this block.

Line 295: `def python_statement_spans(text: str) -> dict[int, StatementSpan]:` This starts function python_stateme Its indented body performs python statement spans as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 296: `"""Map multiline host statements to their complete source expression."""` This is executable documentation attached to the function python_statement_spans, which handles python statement spans. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 297: `lines = text.splitlines()` This assigns ‘text.splitlines()‘ to ‘lines ‘ in host memory. Here lines means the value named ’lines’ in this block. The important operands are text: the value named ’text’ in this block; splitlines: the value named ’splitlines’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 298: `try:` This is explicit host-side failure handling inside the function python_statement_spans, which handles python statement spans; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 299: `tree = ast.parse(text)` This assigns ‘ast.parse(text)‘ to ‘tree ‘ in host memory. Here tree means the value named ’tree’ in this block. The important operands are ast: the value named ’ast’ in this block; parse: the value named ’parse’ in this block; text: the value named ’text’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 300: `except SyntaxError:` This is explicit host-side failure handling inside the function python_statement_spans, which handles python statement spans; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 301: `return {}` This returns ‘{}‘ to the caller of the function python_statement_spans, which handles python statement spans, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 302: `candidates: list[ast.AST] = []` This assigns ‘[]‘ to ‘candidates‘ in host memory. Here candidates means the value named ’candidates’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function python_statement_spans, which handles python statement spans.

Line 303: `simple_nodes = (` This statement continues through source line 306; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(ast.Assign, ast.AnnAssign, ast.AugAssign, ast.Return, ast.Raise, ast.Assert, ast.Import, ast.ImportFrom,)‘ to ‘simple_nodes ‘ in host memory. Here simple_nodes means the value named ’simple nodes’ in this block. The important operands are ast: the value named ’ast’ in this block; Assign: the value named ’assign’ in this block; AnnAssign: the value named ’ann assign’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 304: `ast.Assign, ast.AnnAssign, ast.AugAssign, ast.Return, ast.Raise,` This is continuation line 2 of the 4-line logical statement that begins at source line 303. Its exact fragment is ‘ast.Assign, ast.AnnAssign, ast.AugAssign, ast.Return, ast.Raise,‘. The important operands are ast: the value named ’ast’ in this block; Assign: the value named ’assign’ in this block; AnnAssign: the value named ’ann assign’ in this block. Read together, the complete operation is: This assigns ‘(ast.Assign, ast.AnnAssign, ast.AugAssign, ast.Return, ast.Raise, ast.Assert, ast.Import, ast.ImportFrom,)‘ to ‘simple_nodes ‘ in host memory. Here simple_nodes means the value named ’simple nodes’ in this block. The important operands are ast: the value named ’ast’ in this block; Assign: the value named ’assign’ in this block; AnnAssign: the value named ’ann assign’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 305: `ast.Assert, ast.Import, ast.ImportFrom,` This is continuation line 3 of the 4-line logical statement that begins at source line 303. Its exact fragment is ‘ast.Assert, ast.Import, ast.ImportFrom,‘. The important operands are ast: the value named ’ast’ in this block; Assert: the value named ’assert’ in this block; Import: the value named ’import’ in this block. Read together, the complete operation is: This assigns ‘(ast.Assign, ast.AnnAssign, ast.AugAssign, ast.Return, ast.Raise, ast.Assert, ast.Import, ast.ImportFrom,)‘ to ‘simple_nodes ‘ in host memory. Here simple_nodes means the value named ’simple nodes’ in this block. The important operands are ast: the value named ’ast’ in this block; Assign: the value named ’assign’ in this block; AnnAssign: the value named ’ann assign’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 306: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 303. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(ast.Assign, ast.AnnAssign, ast.AugAssign, ast.Return, ast.Raise, ast.Assert, ast.Import, ast.ImportFrom,)‘ to ‘simple_nodes ‘ in host memory. Here simple_nodes means the value named ’simple nodes’ in this block. The important operands are ast: the value named ’ast’ in this block; Assign: the value named ’assign’ in this block; AnnAssign: the value named ’ann assign’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 307: `for node in ast.walk(tree):` This iterates node over ‘ast.walk(tree)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function python_statement_spans, which handles python statement spans.

Line 308: `if isinstance(node, simple_nodes):` This executes the following indented host-code branch only when ‘isinstance(node, simple_nodes)‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; node: the value named ’node’ in this block; simple_nodes: the value named ’simple nodes’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 309: `candidates.append(node)` This calls ‘candidates.append‘ with ‘node‘ inside the function python_statement_spans, which handles python statement spans. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 310: `elif isinstance(node, ast.Expr) and isinstance(node.value, ast.Call):` This executes the following indented host-code branch only when ‘isinstance(node, ast.Expr) and isinstance(node.value, ast.Call)‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; node: the value named ’node’ in this block; ast: the value named ’ast’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 311: `candidates.append(node)` This calls ‘candidates.append‘ with ‘node‘ inside the function python_statement_spans, which handles python statement spans. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 312: `result: dict[int, StatementSpan] = {}` This assigns ‘{}‘ to ‘result‘ in host memory. Here result means the value named ’result’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function python_statement_spans, which handles python statement spans.

Line 313: `for node in candidates:` This iterates node over ‘candidates‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function python_statement_spans, which handles python statement spans.

Line 314: `start = int(getattr(node, "lineno", 0))` This assigns ‘int(getattr(node, "lineno", 0))‘ to ‘start ‘ in host memory. Here start means the value named ’start’ in this block. The important operands are getattr: the value named ’getattr’ in this block; node: the value named ’node’ in this block; lineno: the value named ’lineno’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 315: `end = int(getattr(node, "end_lineno", start))` This assigns ‘int(getattr(node, "end_lineno", start))‘ to ‘end ‘ in host memory. Here end means the value named ’end’ in this block. The important operands are getattr: the value named ’getattr’ in this block; node: the value named ’node’ in this block; end_lineno: the value named ’end lineno’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 316: `if start <= 0 or end <= start:` This executes the following indented host-code branch only when ‘start <= 0 or end <= start‘ is true. The important operands are start: the value named ’start’ in this block; or: the value named ’or’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 317: `continue` Inside the function python_statement_spans, which handles python statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are continue: the value named ’continue’ in this block.

Line 318: `span = StatementSpan(start, end, _joined_statement(lines, start, end))` This assigns ‘StatementSpan(start, end, _joined_statement(lines, start, end))‘ to ‘span ‘ in host memory. Here span means the value named ’span’ in this block. The important operands are StatementSpan: the finite-state-machine phase retained in flip-flops; start: the value named ’start’ in this block; _joined_statement: the finite-state-machine phase retained in flip-flops. Context: the function python_statement_spans, which handles python statement spans.

Line 319: `for line_number in range(start, end + 1):` This iterates line_number over ‘range(start, end + 1)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function python_statement_spans, which handles python statement spans.

Line 320: `previous = result.get(line_number)` This assigns ‘result.get(line_number)‘ to ‘previous ‘ in host memory. Here previous means the value named ’previous’ in this block. The important operands are result: the value named ’result’ in this block; get: the value named ’get’ in this block; line_number: the value named ’line number’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 321: `if previous is None or (end - start) < (previous.end - previous.start):` This executes the following indented host-code branch only when ‘previous is None or (end - start) < (previous.end - previous.start)‘ is true. The important operands are previous: the value named ’previous’ in this block; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 322: `result[line_number] = span` This assigns ‘span‘ to ‘result[line_number] ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are span: the value named ’span’ in this block. Context: the function python_statement_spans, which handles python statement spans.

Line 323: `return result` This returns ‘result‘ to the caller of the function python_statement_spans, which handles python statement spans, ending this invocation. The important operands are result: the value named ’result’ in this block.

Line 326: `def sv_statement_spans(lines: list[str]) -> dict[int, StatementSpan]:` This starts function sv_statement_spans. Its indented body performs sv statement spans as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 327: `"""Recover multiline RTL assignments without pretending fragments are complete."""` This is executable documentation attached to the function sv_statement_spans, which handles sv statement spans. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 328: `result: dict[int, StatementSpan] = {}` This assigns ‘{}‘ to ‘result‘ in host memory. Here result means the value named ’result’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_statement_spans, which handles sv statement spans.

Line 329: `assignment_start = re.compile(` This statement continues through source line 331; the explanation uses the complete expression rather than only the first visual fragment. Inside the function sv_statement_spans, which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are assignment_start: the value named ’assignment start’ in this block; re: the value named ’re’ in this block; compile: the value named ’compile’ in this block.

Line 330: `r"^(?:assign\s+)?[A-Za-z_][A-Za-z0-9_$\[\].:+* /()’-]*\s*(?:<=|=(?!=))"` This is continuation line 2 of the 3-line logical statement that begins at source line 329. Its exact fragment is ‘r"^(?:assign\s+)?[A-Za-z_][A-Za-z0-9_$\[\].:+* /()’-]*\s*(?:<=|=(?!=))"‘. The important operands are r: the value named ’r’ in this block; assign: the value named ’assign’ in this block; s: the value named ’s’ in this block. Read together, the complete operation is: Inside the function sv_statement_spans, which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are assignment_start: the value named ’assignment start’ in this block; re: the value named ’re’ in this block; compile: the value named ’compile’ in this block.

Line 331: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 329. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function sv_statement_spans, which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are assignment_start: the value named ’assignment start’ in this block; re: the value named ’re’ in this block; compile: the value named ’compile’ in this block.

Line 332: `state_encoding = re.compile(r"^[A-Z][A-Z0-9_]*\s*=.*,$")` This assigns ‘re.compile(r"^[A-Z][A-Z0-9_]*\s*=.*,$")‘ to ‘state_encoding ‘ in host memory. Here state_encoding means the finite-state-machine phase retained in flip-flops. The important operands are re: the value named ’re’ in this block; compile: the value named ’compile’ in this block; r: the value named ’r’ in this block. Context: the function sv_statement_spans, which handles sv statement spans.

Line 333: `line_index = 0` This assigns ‘0‘ to ‘line_index ‘ in host memory. Here line_index means an integer selecting one tensor element, lane, bank, or state entry. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_statement_spans, which handles sv statement spans.

Line 334: `while line_index < len(lines):` This repeats the following host-code block while ‘line_index < len(lines)‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 335: `stripped = lines[line_index].strip()` This assigns ‘lines[line_index].strip()‘ to ‘stripped ‘ in host memory. Here stripped means the value named ’stripped’ in this block. The important operands are lines: the value named ’lines’ in this block; line_index: an integer selecting one tensor element, lane, bank, or state entry; strip: the value named ’strip’ in this block. Context: the function sv_statement_spans, which handles sv statement spans.

Line 336: `if (not stripped or stripped.startswith("//") or ";" in stripped` Inside the function sv_statement_spans which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are not: the value named ’not’ in this block; stripped: the value named ’stripped’ in this block; or: the value named ’or’ in this block.

Line 337: `or state_encoding.match(stripped) or not assignment_start.match(stripped)):` Inside the function sv_statement_spans, which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are or: the value named ’or’ in this block; state_encoding: the finite-state-machine phase retained in flip-flops; match: the value named ’match’ in this block.

Line 338: `line_index += 1` Inside the function sv_statement_spans, which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are line_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 339: `continue` Inside the function sv_statement_spans, which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are continue: the value named ’continue’ in this block.

Line 340: `end_index = line_index + 1` This assigns ‘line_index + 1‘ to ‘end_index ‘ in host memory. Here end_index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are line_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function sv_statement_spans, which handles sv statement spans.

Line 341: `while end_index < len(lines) and ";" not in lines[end_index]:` This repeats the following host-code block while ‘end_index < len(lines) and ";" not in lines[end_index]‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 342: `end_index += 1` Inside the function sv_statement_spans, which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are end_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 343: `if end_index >= len(lines) or end_index - line_index > 64:` This executes the following indented host-code branch only when ‘end_index >= len(lines) or end_index - line_index > 64‘ is true. The important operands are end_index: an integer selecting one tensor element, lane, bank, or state entry; len: the value named ’len’ in this block; lines: the value named ’lines’ in this block. Context: the function sv_statement_spans, which handles sv statement spans.

Line 344: `line_index += 1` Inside the function sv_statement_spans, which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are line_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 345: `continue` Inside the function sv_statement_spans, which handles sv statement spans, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are continue: the value named 'continue' in this block.

Line 346: `start = line_index + 1` This assigns ‘line_index + 1‘ to ‘start ‘ in host memory. Here start means the value named 'start' in this block. The important operands are line_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function sv_statement_spans, which handles sv statement spans.

Line 347: `end = end_index + 1` This assigns ‘end_index + 1‘ to ‘end ‘ in host memory. Here end means the value named 'end' in this block. The important operands are end_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function sv_statement_spans, which handles sv statement spans.

Line 348: `span = StatementSpan(start, end, _joined_statement(lines, start, end))` This assigns ‘StatementSpan(start, end, _joined_statement(lines, start, end))‘ to ‘span ‘ in host memory. Here span means the value named 'span' in this block. The important operands are StatementSpan: the finite-state-machine phase retained in flip-flops; start: the value named 'start' in this block; _joined_statement: the finite-state-machine phase retained in flip-flops. Context: the function sv_statement_spans, which handles sv statement spans.

Line 349: `for line_number in range(start, end + 1):` This iterates line_number over ‘range(start, end + 1)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function sv_statement_spans, which handles sv statement spans.

Line 350: `result[line_number] = span` This assigns ‘span‘ to ‘result[line_number] ‘ in host memory. Here result means the value named 'result' in this block. The important operands are span: the value named 'span' in this block. Context: the function sv_statement_spans, which handles sv statement spans.

Line 351: `line_index = end_index + 1` This assigns ‘end_index + 1‘ to ‘line_index ‘ in host memory. Here line_index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are end_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function sv_statement_spans, which handles sv statement spans.

Line 352: `return result` This returns ‘result‘ to the caller of the function sv_statement_spans, which handles sv statement spans, ending this invocation. The important operands are result: the value named 'result' in this block.

Line 355: `def sv_contexts(lines: list[str]) -> dict[int, str]:` This starts function sv_contexts. Its indented body performs sv contexts as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 356: `result: dict[int, str] = {}` This assigns ‘{}‘ to ‘result‘ in host memory. Here result means the value named 'result' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 357: `process = "the module's structural declarations"` This assigns ‘"the module's structural declarations"‘ to ‘process ‘ in host memory. Here process means the value named 'process' in this block. The important operands are the: the value named 'the' in this block; module: the value named 'module' in this block; s: the value named 's' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 358: `process_depth = 0` This assigns ‘0‘ to ‘process_depth ‘ in host memory. Here process_depth means the value named 'process depth' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 359: `process_until: str | None = None` This assigns ‘None‘ to ‘process_until‘ in host memory. Here process_until means the value named 'process until' in this block. The important operands are None: the value named 'none' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 360: `state = ""` This assigns '""' to 'state ' in host memory. Here state means the finite-state-machine phase retained in flip-flops. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 361: `for line_number, raw in enumerate(lines, start=1):` This iterates line_number, raw over 'enumerate(lines, start=1)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function sv_contexts, which handles sv contexts.

Line 362: `stripped = raw.strip()` This assigns 'raw.strip()' to 'stripped ' in host memory. Here stripped means the value named 'stripped' in this block. The important operands are raw: the value named 'raw' in this block; strip: the value named 'strip' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 363: `starts_process = False` This assigns 'False' to 'starts_process ' in host memory. Here starts_process means the value named 'starts process' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 364: `if re.search(r"\balways_ff\b", stripped):` This executes the following indented host-code branch only when 're.search(r"\balways_ff\b", stripped)' is true. The important operands are re: the value named 're' in this block; search: the value named 'search' in this block; r: the value named 'r' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 365: `process = "the clocked process that updates registers only on a rising clock edge"` This assigns '"the clocked process that updates registers only on a rising clock edge"' to 'process ' in host memory. Here process means the value named 'process' in this block. The important operands are the: the value named 'the' in this block; clocked: the value named 'clocked' in this block; process: the value named 'process' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 366: `process_depth = max(1, len(re.findall(r"\bbegin\b", stripped)))` This assigns 'max(1, len(re.findall(r"\bbeg stripped)))' to 'process_depth ' in host memory. Here process_depth means the value named 'process depth' in this block. The important operands are max: the value named 'max' in this block; len: the value named 'len' in this block; re: the value named 're' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 367: `process_until = None` This assigns 'None' to 'process_until ' in host memory. Here process_until means the value named 'process until' in this block. The important operands are None: the value named 'none' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 368: `starts_process = True` This assigns 'True' to 'starts_process ' in host memory. Here starts_process means the value named 'starts process' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 369: `elif re.search(r"\balways_comb\b", stripped):` This executes the following indented host-code branch only when 're.search(r"\balways_comb\b", stripped)' is true. The important operands are re: the value named 're' in this block; search: the value named 'search' in this block; r: the value named 'r' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 370: `process = "the combinational process whose outputs respond without waiting for a clock edge"` This assigns '"the combinational process whose outputs respond without waiting for a clock edge"' to 'process ' in host memory. Here process means the value named 'process' in this block. The important operands are the: the value named 'the' in this block; combinational: the value named 'combinational' in this block; process: the value named 'process' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 371: `process_depth = max(1, len(re.findall(r"\bbegin\b", stripped)))` This assigns 'max(1, len(re.findall(r"\bbeg stripped)))' to 'process_depth ' in host memory. Here process_depth means the value named 'process depth' in this block. The important operands are max: the value named 'max' in this block; len: the value named 'len' in this block; re: the value named 're' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 372: `process_until = None` This assigns ‘None‘ to ‘process_until ‘ in host memory. Here process_until means the value named ’process until’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 373: `starts_process = True` This assigns ‘True‘ to ‘starts_process ‘ in host memory. Here starts_process means the value named ’starts process’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 374: `elif stripped.startswith("function"):` This executes the following indented host-code branch only when ‘stripped.startswith("function")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; function: the value named ’function’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 375: `names = re.findall(r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", stripped)` This assigns ‘re.findall(r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", stripped)‘ to ‘names ‘ in host memory. Here names means the value named ’names’ in this block. The important operands are re: the value named ’re’ in this block; findall: the value named ’findall’ in this block; r: the value named ’r’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 376: `name = names[-1] if names else "function"` This assigns ‘names[-1] if names else "function"‘ to ‘name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are names: the value named ’names’ in this block; function: the value named ’function’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 377: `process = f"the combinational helper {humanize(name)}"` This assigns ‘f"the combinational helper {humanize(name)}"‘ to ‘process ‘ in host memory. Here process means the value named ’process’ in this block. The important operands are f: the value named ’f’ in this block; the: the value named ’the’ in this block; combinational: the value named ’combinational’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 378: `process_until = "endfunction"` This assigns ‘"endfunction"‘ to ‘process_until ‘ in host memory. Here process_until means the value named ’process until’ in this block. The important operands are endfunction: the value named ’endfunction’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 379: `process_depth = 0` This assigns ‘0‘ to ‘process_depth ‘ in host memory. Here process_depth means the value named ’process depth’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 380: `starts_process = True` This assigns ‘True‘ to ‘starts_process ‘ in host memory. Here starts_process means the value named ’starts process’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 381: `elif stripped.startswith("task"):` This executes the following indented host-code branch only when ‘stripped.startswith("task")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; task: the value named ’task’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 382: `names = re.findall(r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", stripped)` This assigns ‘re.findall(r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", stripped)‘ to ‘names ‘ in host memory. Here names means the value named ’names’ in this block. The important operands are re: the value named ’re’ in this block; findall: the value named ’findall’ in this block; r: the value named ’r’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 383: `name = names[-1] if names else "task"` This assigns ‘names[-1] if names else "task"‘ to ‘name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are names: the value named ’names’ in this block; task: the value named ’task’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 384: `process = f"the simulation task {humanize(name)}"` This assigns ‘f"the simulation task {humanize(name)}"‘ to ‘process ‘ in host memory. Here process means the value named ’process’ in this block. The important operands are f: the value named ’f’ in this block; the: the value named ’the’ in this block; simulation: the value named ’simulation’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 385: `process_until = "endtask"` This assigns ‘"endtask"‘ to ‘process_until ‘ in host memory. Here process_until means the value named ’process until’ in this block. The important operands are endtask: the value named ’endtask’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 386: `process_depth = 0` This assigns ‘0‘ to ‘process_depth ‘ in host memory. Here process_depth means the value named ’process depth’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 387: `starts_process = True` This assigns ‘True‘ to ‘starts_process ‘ in host memory. Here starts_process means the value named ’starts process’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 388: `state_match = re.match(r"(?:ST_|CORE_|WR_|RD_)?([A-Z][A-Z0-9_]*)\s*:", stripped)` This assigns ‘re.match(r"(?:ST_|CORE_|WR_|RD_)?([A-Z][A-Z0-9_]*)\s*:", stripped)‘ to ‘state_match ‘ in host memory. Here state_match means the finite-state-machine phase retained in flip-flops. The important operands are re: the value named ’re’ in this block; match: the value named ’match’ in this block; r: the value named ’r’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 389: `if state_match:` This executes the following indented host-code branch only when ‘state_match‘ is true. The important operands are state_match: the finite-state-machine phase retained in flip-flops. Context: the function sv_contexts, which handles sv contexts.

Line 390: `state_name = state_match.group(1)` This assigns ‘state_match.group(1)‘ to ‘state_name ‘ in host memory. Here state_name means the finite-state-machine phase retained in flip-flops. The important operands are state_match: the finite-state-machine phase retained in flip-flops; group: the value named ’group’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 391: `meaning = next((value for key, value in STATE_MEANINGS.items() if key in state_name), "performs the named c...` This assigns ‘next((value for key, value in STATE_MEANINGS.items() if key in state_name), "performs the named control phase")‘ to ‘meaning ‘ in host memory. Here meaning means the arithmetic average used by normalization. The important operands are next: the value named ’next’ in this block; value: the value named ’value’ in this block; for: the value named ’for’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 392: `state = f" Within state {state_match.group(0)[:-1]}, the machine {meaning}."` This assigns ‘f" Within state {state_match.group(0)[:-1]}, the machine {meaning}."‘ to ‘state ‘ in host memory. Here state means the finite-state-machine phase retained in flip-flops. The important operands are f: the value named ’f’ in this block; Within: the value named ’within’ in this block; state: the finite-state-machine phase retained in flip-flops. Context: the function sv_contexts, which handles sv contexts.

Line 393: `result[line_number] = process + state` This assigns ‘process + state‘ to ‘result[line_number] ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are process: the value named ’process’ in this block; state: the finite-state-machine phase retained in flip-flops. Context: the function sv_contexts, which handles sv contexts.

Line 394: `if stripped == "endcase":` This executes the following indented host-code branch only when ‘stripped == "endcase"‘ is true. The important operands are stripped: the value named ’stripped’ in this block; endcase: the value named ’endcase’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 395: `state = ""` This assigns ‘""‘ to ‘state ‘ in host memory. Here state means the finite-state-machine phase retained in flip-flops. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 396: `if process_until is not None and stripped.startswith(process_until):` This executes the following indented host-code branch only when ‘process_until is not None and stripped.startswith(process_until)‘ is true. The important operands are process_until: the value named ’process until’ in this block; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 397: `process = "the module’s structural declarations"` This assigns ‘"the module’s structural declarations"‘ to ‘process ‘ in host memory. Here process means the value named ’process’ in this block. The important operands are the: the value named ’the’ in this block; module: the value named ’module’ in this block; s: the value named ’s’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 398: `process_until = None` This assigns ‘None‘ to ‘process_until ‘ in host memory. Here process_until means the value named ’process until’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 399: `process_depth = 0` This assigns ‘0‘ to ‘process_depth ‘ in host memory. Here process_depth means the value named ’process depth’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 400: `state = ""` This assigns ‘""‘ to ‘state ‘ in host memory. Here state means the finite-state-machine phase retained in flip-flops. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 401: `elif process_until is None and process != "the module’s structural declarations":` This executes the following indented host-code branch only when ‘process_until is None and process != "the module’s structural declarations"‘ is true. The important operands are process_until: the value named ’process until’ in this block; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 402: `if not starts_process:` This executes the following indented host-code branch only when ‘not starts_process‘ is true. The important operands are not: the value named ’not’ in this block; starts_process: the value named ’starts process’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 403: `process_depth += len(re.findall(r"\bbegin\b", stripped))` Inside the function sv_contexts, which handles sv contexts, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are process_depth: the value named ’process depth’ in this block; len: the value named ’len’ in this block; re: the value named ’re’ in this block.

Line 404: `process_depth -= len(re.findall(r"\bend\b", stripped))` Inside the function sv_contexts, which handles sv contexts, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are process_depth: the value named ’process depth’ in this block; len: the value named ’len’ in this block; re: the value named ’re’ in this block.

Line 405: `if process_depth <= 0 or (starts_process and "begin" not in stripped):` This executes the following indented host-code branch only when ‘process_depth <= 0 or (starts_process and "begin" not in stripped)‘ is true. The important operands are process_depth: the value named ’process depth’ in this block; or: the value named ’or’ in this block; starts_process: the value named ’starts process’ in this block. Context: the function sv_contexts, which handles sv contexts.

Line 406: `process = "the module's structural declarations"` This assigns ‘"the module's structural declarations"‘ to ‘process ‘ in host memory. Here process means the value named 'process' in this block. The important operands are the: the value named 'the' in this block; module: the value named 'module' in this block; s: the value named 's' in this block. Context: the function sv_contexts, which handles sv contexts.

Line 407: `process_depth = 0` This assigns ‘0‘ to ‘process_depth ‘ in host memory. Here process_depth means the value named 'process depth' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 408: `state = ""` This assigns ‘""‘ to ‘state ‘ in host memory. Here state means the finite-state-machine phase retained in flip-flops. The expression is a literal or structural operator and has no hidden software state. Context: the function sv_contexts, which handles sv contexts.

Line 409: `return result` This returns ‘result‘ to the caller of the function sv_contexts, which handles sv contexts, ending this invocation. The important operands are result: the value named 'result' in this block.

Line 412: `def explain_sv_line(line: str, context: str, purpose: str) -> str:` This starts function explain_sv_line. Its indented body performs explain sv line as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 413: `stripped = line.strip()` This assigns ‘line.strip()‘ to ‘stripped ‘ in host memory. Here stripped means the value named 'stripped' in this block. The important operands are line: the value named 'line' in this block; strip: the value named 'strip' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 414: `if stripped.startswith("//"):` This executes the following indented host-code branch only when ‘stripped.startswit is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 415: `comment = stripped[2:].strip()` This assigns ‘stripped[2:].strip()‘ to ‘comment ‘ in host memory. Here comment means the value named 'comment' in this block. The important operands are stripped: the value named 'stripped' in this block; strip: the value named 'strip' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 416: `return f"Design intent: {comment} This comment anchors the surrounding logic in the block that {purpose}."` This returns ‘f"Design intent: {comment} This comment anchors the surrounding logic in the block that {purpose}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named 'f' in this block; Design: the value named 'design' in this block; intent: the value named 'intent' in this block.

Line 417: `if stripped.startswith("‘"):` This executes the following indented host-code branch only when ‘stripped.startswith is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 418: `if "default_nettype none" in stripped:` This executes the following indented host-code branch only when ‘"default_nettype none" in stripped‘ is true. The important operands are default_nettype: the value named 'default nettype' in this block; none: the value named 'none' in this block; in: the value named 'in' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 419: `return "This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names ...` This returns ‘"This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are This: the value named 'this' in this block; compiler: the value named 'compiler' in this block; directive: the value named 'directive' in this block.

Line 420: `if "timescale" in stripped:` This executes the following indented host-code branch only when ‘"timescale" in stripped‘ is true. The important operands are timescale: the real-value spacing represented by one quantized integer step; in: the value named ’in’ in this block; stripped: the value named ’stripped’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 421: `return "This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp pre...` This returns ‘"This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are This: the value named ’this’ in this block; simulation: the value named ’simulation’ in this block; directive: the value named ’directive’ in this block.

Line 422: `return f"This SystemVerilog preprocessor line controls compilation of the block that {purpose}."` This returns ‘f"This SystemVerilog preprocessor line controls compilation of the block that {purpose}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; SystemVerilog: the value named ’system verilog’ in this block.

Line 423: `if re.match(r"(?:module|package)\b", stripped):` This executes the following indented host-code branch only when ‘re.match(r"(?:module|package)\b", stripped)‘ is true. The important operands are re: the value named ’re’ in this block; match: the value named ’match’ in this block; r: the value named ’r’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 424: `name = re.split(r"[\s(#]", stripped, maxsplit=2)[1]` This assigns ‘re.split(r"[\s(#]", stripped, maxsplit=2)[1]‘ to ‘name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are re: the value named ’re’ in this block; split: the value named ’split’ in this block; r: the value named ’r’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 425: `return f"This opens {name}, an independently compiled hardware definition. Its ports and statements collect...` This returns ‘f"This opens {name}, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that {purpose}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; opens: the value named ’opens’ in this block.

Line 426: `if stripped.startswith(("endmodule", "endpackage")):` This executes the following indented host-code branch only when ‘stripped.startswith(("endmodule", "endpackage"))‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; endmodule: the value named ’endmodule’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 427: `return "This closes the current hardware definition; no circuit behavior is attached to the keyword itself."` This returns ‘"This closes the current hardware definition; no circuit behavior is attached to the keyword itself."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are This: the value named ’this’ in this block; closes: the value named ’closes’ in this block; the: the value named ’the’ in this block.

Line 428: `if stripped.startswith("import "):` This executes the following indented host-code branch only when ‘stripped.startswith("import ")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; import: the value named ’import’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 429: `return f"This imports shared package names into the current module so opcodes and mode constants have exact...` This returns ‘f"This imports shared package names into the current module so opcodes and mode constants have exactly one definition. The import changes compile-time name visibility; it does not create registers or consume a clock cycle."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; imports: the value named ’imports’ in this block.

Line 430: `port = re.match(r"(input|output|inout)\s+(?:wire|logic)?\s*(?:signed\s*)?(\[[^]]+\])?\s*([A-Za-z_][` This assigns ‘re.match(r"(input|output|inout)\s+(?:wire|logic)?\s*(?:signed\s*)?(\[[^]]+\])?\s*([A-Za-z_][A-Za-z0-9_$]*)", stripped)‘ to ‘port ‘ in host memory. Here port means the value named ’port’ in this block. The important operands are re: the value named ’re’ in this block; match: the value named ’match’ in this block; r: the value named ’r’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 431: `if port:` This executes the following indented host-code branch only when ‘port‘ is true. The important operands are port: the value named ’port’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 432: `direction, width, name = port.groups()` This assigns ‘port.groups()‘ to ‘direction, width, name ‘ in host memory. Here direction means the value named ’direction’ in this block. The important operands are port: the value named ’port’ in this block; groups: the value named ’groups’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 433: `width_text = width or "one bit"` This assigns ‘width or "one bit"‘ to ‘width_text ‘ in host memory. Here width_text means the value named ’width text’ in this block. The important operands are width: the value named ’width’ in this block; or: the value named ’or’ in this block; one: the value named ’one’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 434: `return f"This declares {name} as a {direction} port with width {width_text}.  It carries {identifie` This returns ‘f"This declares {name} as a {direction} port with width {width_text}. It carries {identifier_meaning(name)} across the module boundary; a port is a bundle of physical FPGA nets, not a software argument."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; declares: the value named ’declares’ in this block.

Line 435: `parameter = re.match(r"(?:parameter|localparam)\s+(?:(?:logic|integer|int|bit)\s+)?(?:signed\s*)?(?` This assigns ‘re.match(r"(?:parameter|localparam)\s+(?:(?:logic|integer|int|bit)\s+)?(?:signed\s*)?(?:\[[^]]+\]\s*)?([A-Za-z_][A-Za-z0-9_$]*)\s*=\s*(.*?)[,;]?$", stripped)‘ to ‘parameter ‘ in host memory. Here parameter means the value named ’parameter’ in this block. The important operands are re: the value named ’re’ in this block; match: the value named ’match’ in this block; r: the value named ’r’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 436: `if parameter:` This executes the following indented host-code branch only when ‘parameter‘ is true. The important operands are parameter: the value named ’parameter’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 437: `name, value = parameter.groups()` This assigns ‘parameter.groups()‘ to ‘name, value ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are parameter: the value named ’parameter’ in this block; groups: the value named ’groups’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 438: `return f"This fixes {name} to {value} during elaboration.  Vivado uses that constant to shape or simplify th...` This returns ‘f"This fixes {name} to {value} during elaboration. Vivado uses that constant to shape or simplify the generated circuit; {name} denotes {identifier_meaning(name)}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; fixes: the value named ’fixes’ in this block.

Line 439: `if "typedef enum" in stripped:` This executes the following indented host-code branch only when ‘"typedef enum" in stripped‘ is true. The important operands are typedef: the value named ’typedef’ in this block; enum: the value named ’enum’ in this block; in: the value named ’in’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 440: `return "This begins a named finite-state-machine type.  The encoded bits are stored in flip-flops, while sym...` This returns '"This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable."' to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are This: the value named 'this' in this block; begins: the value named 'begins' in this block; a: the value named 'a' in this block.

Line 441: `state_match = re.match(r"([A-Z][A-Z0-9_]*)\s*=\s*[^,}]+", stripped)` This assigns 're.match(r"([A-Z][A-Z0-9_]*)\s*=\s*[^,}]+", stripped)' to 'state_match ' in host memory. Here state_match means the finite-state-machine phase retained in flip-flops. The important operands are re: the value named 're' in this block; match: the value named 'match' in this block; r: the value named 'r' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 442: `if state_match:` This executes the following indented host-code branch only when 'state_match' is true. The important operands are state_match: the finite-state-machine phase retained in flip-flops. Context: the function explain_sv_line, which handles explain sv line.

Line 443: `name = state_match.group(1)` This assigns 'state_match.group(1)' to 'name ' in host memory. Here name means the value named 'name' in this block. The important operands are state_match: the finite-state-machine phase retained in flip-flops; group: the value named 'group' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 444: `meaning = next((value for key, value in STATE_MEANINGS.items() if key in name), "identifies one explicit co...` This assigns 'next((value for key, value in STATE_MEANINGS.items() if key in name), "identifies one explicit control phase")' to 'meaning ' in host memory. Here meaning means the arithmetic average used by normalization. The important operands are next: the value named 'next' in this block; value: the value named 'value' in this block; for: the value named 'for' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 445: `return f"This assigns a fixed binary encoding to state {name}, which {meaning}.  The encoding is visible in ...` This returns 'f"This assigns a fixed binary encoding to state {name}, which {meaning}. The encoding is visible in waveforms even though the source uses the readable name."' to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; assigns: the value named 'assigns' in this block.

Line 446: `declaration = re.match(r"(?:logic|wire|reg|integer|genvar)\s+(?:signed\s*)?(?:\[[^]]+\]\s*)?(.+?);$", strip...` This assigns 're.match(r"(?:logic|wire|reg|integer|genvar)\s+(?:signed\s*)?(?:\[[^]]+\]\s*)?(.+?);$", stripped)' to 'declaration ' in host memory. Here declaration means the value named 'declaration' in this block. The important operands are re: the value named 're' in this block; match: the value named 'match' in this block; r: the value named 'r' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 447: `if declaration:` This executes the following indented host-code branch only when 'declaration' is true. The important operands are declaration: the value named 'declaration' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 448: `names = [re.split(r"[\s[=]", item.strip(), maxsplit=1)[0] for item in declaration.group(1).split(",")]` This assigns '[re.split(r"[\s[=]", item.strip(), maxsplit=1)[0] for item in declaration.group(1).split(",")]' to 'names ' in host memory. Here names means the value named 'names' in this block. The important operands are re: the value named 're' in this block; split: the value named 'split' in this block; r: the value named 'r' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 449: `descriptions = "; ".join(f"{name} is {identifier_meaning(name)}" for name in names[:4])` This assigns '"; ".join(f"{name} is {identifier_meaning(name)}" for name in names[:4])' to 'descriptions ' in host memory. Here descriptions means the value named 'descriptions' in this block. The important operands are join: the value named 'join' in this block; f: the value named 'f' in this block; name: the value named 'name' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 450: `return f"This declares hardware signal storage or wiring in {context}. Specifically, {descriptions Widths...` This returns ‘f"This declares hardware signal storage or wiring in {context}. Specifically, {descriptions}. Widths in square brackets count from the stated most-significant bit down to the least-significant bit."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; declares: the value named ’declares’ in this block.

Line 451: `assignment = re.match(r"(?:assign\s+)?([A-Za-z_][A-Za-z0-9_$\[\].:]*)\s*(<=|=)\s*(.+?);?$", stripped)` Inside the function explain_sv_line, which handles explain sv line, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are assignment: the value named ’assignment’ in this block; re: the value named ’re’ in this block; match: the value named ’match’ in this block.

Line 452: `if assignment:` This executes the following indented host-code branch only when ‘assignment‘ is true. The important operands are assignment: the value named ’assignment’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 453: `lhs, operator, rhs = assignment.groups()` This assigns ‘assignment.groups()‘ to ‘lhs, operator, rhs ‘ in host memory. Here lhs means the value named ’lhs’ in this block. The important operands are assignment: the value named ’assignment’ in this block; groups: the value named ’groups’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 454: `timing = "updates after the active clock edge together with other nonblocking assignments" if operator == "...` Inside the function explain_sv_line, which handles explain sv line, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are timing: the value named ’timing’ in this block; updates: the value named ’updates’ in this block; after: the value named ’after’ in this block.

Line 455: `return f"This makes {lhs}, {identifier_meaning(lhs.split('[')[0])}, equal to ‘{rhs.rstrip(';')}‘. It {timin...` This returns ‘f"This makes {lhs}, {identifier_meaning(lhs.split('[')[0])}, equal to ‘{rhs.rstrip(';')}‘. It {timing}. {summarize_expression(rhs)} Context: {context}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; makes: the value named ’makes’ in this block.

Line 456: `if stripped.startswith("always_ff"):` This executes the following indented host-code branch only when ‘stripped.startswith("always_ff")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; always_ff: the value named ’always ff’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 457: `return "This opens clocked sequential logic. Everything assigned with ‘<=‘ inside becomes a flip-flop or me...` This returns ‘"This opens clocked sequential logic. Everything assigned with ‘<=‘ inside becomes a flip-flop or memory write and changes at a clock edge, which creates the accelerator’s notion of cycles and persistent state."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are This: the value named ’this’ in this block; opens: the value named ’opens’ in this block; clocked: the value named ’clocked’ in this block.

Line 458: `if stripped.startswith("always_comb"):` This executes the following indented host-code branch only when ‘stripped.startswith("always_comb")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; always_comb: the value named ’always comb’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 459: `return "This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously fol...` This returns ‘"This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands

are This: the value named 'this' in this block; opens: the value named 'opens' in this block; combinational: the value named 'combinational' in this block.

Line 460: `if stripped.startswith("initial"):` This executes the following indented host-code branch only when ‘stripped.startswith("initial")‘ is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block; initial: the value named 'initial' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 461: `return "This opens simulation initialization. In a testbench it drives starting values and stimulus; projec...` This returns ‘"This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are This: the value named 'this' in this block; opens: the value named 'opens' in this block; simulation: the value named 'simulation' in this block.

Line 462: `reset_match = re.match(r"if\s*\(\s*!?(?:rst_n|aresetn)\s*\)", stripped)` This assigns ‘re.match(r"if\s*\(\ stripped)‘ to ‘reset_match ‘ in host memory. Here reset_match means the value named 'reset match' in this block. The important operands are re: the value named 're' in this block; match: the value named 'match' in this block; r: the value named 'r' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 463: `if reset_match:` This executes the following indented host-code branch only when ‘reset_match‘ is true. The important operands are reset_match: the value named 'reset match' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 464: `return f"This is the reset branch for ‘{reset_match.group(0)[3:-1].strip()}‘. While reset is asserted, the ...` This returns ‘f"This is the reset branch for ‘{reset_match.group(0)[3:-1].strip()}‘. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: {context}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; is: the value named 'is' in this block.

Line 465: `if_match = re.match(r"(?:end\s+)?(?:else\s+)?if\s*\((.*)\)", stripped)` This assigns ‘re.match(r"(?:end\s stripped)‘ to ‘if_match ‘ in host memory. Here if_match means the value named 'if match' in this block. The important operands are re: the value named 're' in this block; match: the value named 'match' in this block; r: the value named 'r' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 466: `if if_match:` This executes the following indented host-code branch only when ‘if_match‘ is true. The important operands are if_match: the value named 'if match' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 467: `condition = if_match.group(1)` This assigns ‘if_match.group(1)‘ to ‘condition ‘ in host memory. Here condition means the value named 'condition' in this block. The important operands are if_match: the value named 'if match' in this block; group: the value named 'group' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 468: `return f"This condition selects the following hardware branch only when ‘{condition}‘ is true. Both branche...` This returns ‘f"This condition selects the following hardware branch only when ‘{condition}‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. {summarize_expression(condition)} Context: {context}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; condition: the value named 'condition' in this block.

Line 469: `if re.search(r"\b(?:unique\s+)?case\s*\((.*)\)", stripped):` This executes the following indented host-code branch only when ‘re.search(r"\b(?:unique\s+)?case\s*\((.*)\)", stripped)‘ is true. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; r: the value named ’r’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 470: `selector = re.search(r"\((.*)\)", stripped).group(1)` This assigns ‘re.search(r"\((.*)\)", stripped).group(1)‘ to ‘selector ‘ in host memory. Here selector means the value named ’selector’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; r: the value named ’r’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 471: `return f"This starts a multiplexer/state decode selected by ‘{selector}‘. ‘unique‘ allows tools to diagnose...` This returns ‘f"This starts a multiplexer/state decode selected by ‘{selector}‘. ‘unique‘ allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; starts: the value named ’starts’ in this block.

Line 472: `label = re.match(r"([A-Za-z_][A-Za-z0-9_$]*)\s*:\s*(.*)", stripped)` This assigns ‘re.match(r"([A-Za-z_][A-Za-z0-9_$]*)\s*:\s*(.*)", stripped)‘ to ‘label ‘ in host memory. Here label means the value named ’label’ in this block. The important operands are re: the value named ’re’ in this block; match: the value named ’match’ in this block; r: the value named ’r’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 473: `if label:` This executes the following indented host-code branch only when ‘label‘ is true. The important operands are label: the value named ’label’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 474: `name, action = label.groups()` This assigns ‘label.groups()‘ to ‘name, action ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are label: the value named ’label’ in this block; groups: the value named ’groups’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 475: `meaning = next((value for key, value in STATE_MEANINGS.items() if key in name), "performs this explicitly n...` This assigns ‘next((value for key, value in STATE_MEANINGS.items() if key in name), "performs this explicitly named phase")‘ to ‘meaning ‘ in host memory. Here meaning means the arithmetic average used by normalization. The important operands are next: the value named ’next’ in this block; value: the value named ’value’ in this block; for: the value named ’for’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 476: `return f"This is the {name} case branch, which {meaning}. The same-cycle action begins with ‘{action}‘; con...` This returns ‘f"This is the {name} case branch, which {meaning}. The same-cycle action begins with ‘{action}‘; context is {context}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; is: the value named ’is’ in this block.

Line 477: `if stripped.startswith("default:"):` This executes the following indented host-code branch only when ‘stripped.startswith("default:")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; default: the value named ’default’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 478: `return f"This is the default decode branch. It supplies deterministic behavior for every unlisted bit patte...` This returns ‘f"This is the default decode branch. It supplies deterministic behavior for every unlisted bit pattern, which prevents inferred latches in combinational logic and gives corrupted state/opcode values an explicit recovery or error path. Context: {context}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; is: the value named ’is’ in this block.

Line 479: `port_map = re.match(r"\.([A-Za-z_][A-Za-z0-9_$]*)\s*\((.*?)\)\s*,?$", stripped)` This assigns ‘re.match(r"\.([A-Za-z_][A-Za-z0-9_$]*)\s*\((.*?)\)\s*,?$", stripped)‘ to ‘port_map ‘ in host memory. Here port_map means the value named ’port map’ in this block. The important operands are re: the value named ’re’ in this block; match: the value named ’match’ in this block; r: the value named ’r’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 480: `if port_map:` This executes the following indented host-code branch only when ‘port_map‘ is true. The important operands are port_map: the value named ’port map’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 481: `port_name, signal = port_map.groups()` This assigns ‘port_map.groups()‘ to ‘port_name, signal ‘ in host memory. Here port_name means the value named ’port name’ in this block. The important operands are port_map: the value named ’port map’ in this block; groups: the value named ’groups’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 482: `return f"This connects the child instance's named port '{port_name}' to parent expression '{signal}'. The c...` This returns ‘f"This connects the child instance's named port ‘{port_name}‘ to parent expression ‘{signal}‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and {identifier_meaning(signal.split('[')[0] or port_name)}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; connects: the value named ’connects’ in this block.

Line 483: `instance = re.match(r"([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?([A-Za-z_][A-Za-z0-9_$]*)\s*str...` This assigns ‘re.match(r"([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?([A-Za-z_][A-Za-z0-9_$]*)\s*\(", stripped)‘ to ‘instance ‘ in host memory. Here instance means the value named ’instance’ in this block. The important operands are re: the value named ’re’ in this block; match: the value named ’match’ in this block; r: the value named ’r’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 484: `if instance and instance.group(1) not in {"if", "for", "while", "case"}:` This executes the following indented host-code branch only when ‘instance and instance.group(1) not in {"if", "for", "while", "case"}‘ is true. The important operands are instance: the value named ’instance’ in this block; and: the value named ’and’ in this block; group: the value named ’group’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 485: `return f"This instantiates module {instance.group(1)} as {instance.group(2)}. Synthesis creates that subord...` This returns ‘f"This instantiates module {instance.group(1)} as {instance.group(2)}. Synthesis creates that subordinate circuit and connects its ports in the lines that follow."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; instantiates: the value named ’instantiates’ in this block.

Line 486: `if stripped.startswith("for ") or stripped.startswith("for("):` This executes the following indented host-code branch only when ‘stripped.startswith("for ") or stripped.startswith("for(")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; for: the value named ’for’ in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 487: `return f"This generate/procedural loop repeats the following operation across lanes or entries. In synthesi...` This returns ‘f"This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: {context}."‘ to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; generate: the value named ’generate’ in this block.

Line 488: `if stripped in {"begin", "end", "endcase", "endgenerate", "generate"} or stripped.startswith(("end else", "...` This executes the following indented host-code branch only when ‘stripped in {"begin", "end", "endcase", "endgenerate", "generate"} or stripped.startswith(("end else", "else begin"))‘ is true. The important operands are stripped:

the value named 'stripped' in this block; in: the value named 'in' in this block; endcase: the value named 'endcase' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 489: `return f"This structural keyword opens or closes the branch/process scope described as {context}; it has no...` This returns 'f"This structural keyword opens or closes the branch/process scope described as {context}; it has no arithmetic value of its own."' to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; structural: the value named 'structural' in this block.

Line 490: `if stripped.startswith(("$display", "$fatal", "$finish", "assert")):` This executes the following indented host-code branch only when 'stripped.startswith(("$display", "$fatal", "$finish", "assert"))' is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block; display: the value named 'display' in this block. Context: the function explain_sv_line, which handles explain sv line.

Line 491: `return "This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is ...` This returns '"This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit."' to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are This: the value named 'this' in this block; is: the value named 'is' in this block; a: the value named 'a' in this block.

Line 492: `return f"Within {context}, this exact statement contributes to the block that {purpose}. {summariz` This returns 'f"Within {context}, this exact statement contributes to the block that {purpose}. {summarize_expression(stripped)}"' to the caller of the function explain_sv_line, which handles explain sv line, ending this invocation. The important operands are f: the value named 'f' in this block; Within: the value named 'within' in this block; context: the value named 'context' in this block.

Line 495: `def explain_python_line(line: str, context: str, purpose: str) -> str:` This starts function explain_python_line. Its indented body performs explain python line as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 496: `stripped = line.strip()` This assigns 'line.strip()' to 'stripped ' in host memory. Here stripped means the value named 'stripped' in this block. The important operands are line: the value named 'line' in this block; strip: the value named 'strip' in this block. Context: the function explain_python_line, which handles explain python line.

Line 497: `if stripped.startswith("#"):` This executes the following indented host-code branch only when 'stripped.startswith is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block. Context: the function explain_python_line, which handles explain python line.

Line 498: `return f"The comment records this intent: {stripped[1:].strip()} It applies inside {context}."` This returns 'f"The comment records this intent: {stripped[1:].strip()} It applies inside {context}."' to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; The: the value named 'the' in this block; comment: the value named 'comment' in this block.

Line 499: `if stripped.startswith(("import ", "from ")):` This executes the following indented host-code branch only when 'stripped.startswith(("import ", "from "))' is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block; import: the value named 'import' in this block. Context: the function explain_python_line, which handles explain python line.

Line 500: `return f"This imports the named Python API so {context} can use it explicitly. Imports execute on the host ...` This returns 'f"This imports the named Python API so {context} can use it explicitly. Imports execute on the host processor and create no FPGA hardware."' to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; imports: the value named 'imports' in this block.

Line 501: `if stripped.startswith("class "):` This executes the following indented host-code branch only when ‘stripped.startswith("class ")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; class: the value named ’class’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 502: `name = stripped.split()[1].split("(")[0].rstrip(":")` This assigns ‘stripped.split()[1].split("(")[0].rstrip(":")‘ to ‘name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are stripped: the value named ’stripped’ in this block; split: the value named ’split’ in this block; rstrip: the value named ’rstrip’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 503: `return f"This defines class {name}, which groups data and methods for {humanize(name)}. Instances exist in ...` This returns ‘f"This defines class {name}, which groups data and methods for {humanize(name)}. Instances exist in host memory and organize the flow that {purpose}."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; defines: the value named ’defines’ in this block.

Line 504: `if stripped.startswith(("def ", "async def ")):` This executes the following indented host-code branch only when ‘stripped.startswith(("def ", "async def "))‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; def: the value named ’def’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 505: `name = stripped.split("def ", 1)[1].split("(", 1)[0]` This assigns ‘stripped.split("def ", 1)[1].split("(", 1)[0]‘ to ‘name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are stripped: the value named ’stripped’ in this block; split: the value named ’split’ in this block; def: the value named ’def’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 506: `return f"This starts function {name}. Its indented body performs {humanize(name)} as part of the file that ...` This returns ‘f"This starts function {name}. Its indented body performs {humanize(name)} as part of the file that {purpose}; it runs only when called."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; starts: the value named ’starts’ in this block.

Line 507: `if stripped.startswith("return"):` This executes the following indented host-code branch only when ‘stripped.startswith("return")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 508: `value = stripped.removeprefix("return").strip()` This assigns ‘stripped.removeprefix("return").strip()‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are stripped: the value named ’stripped’ in this block; removeprefix: the value named ’removeprefix’ in this block; strip: the value named ’strip’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 509: `return f"This returns ‘{value}‘ to the caller of {context}, ending this invocation. {summarize_exp` This returns ‘f"This returns ‘{value}‘ to the caller of {context}, ending this invocation. {summarize_expression(value)}"‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; returns: the value named ’returns’ in this block.

Line 510: `if stripped.startswith("raise "):` This executes the following indented host-code branch only when ‘stripped.startswith("raise ")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; raise: the value named ’raise’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 511: `return f"This rejects an invalid state by raising `{stripped[6:]}`. Failing before DMA prevents malformed s...` This returns `f"This rejects an invalid state by raising `{stripped[6:]}`. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results."` to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; rejects: the value named 'rejects' in this block.

Line 512: `if stripped.startswith("assert "):` This executes the following indented host-code branch only when `stripped.startswith("assert ")` is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block; assert: the value named 'assert' in this block. Context: the function explain_python_line, which handles explain python line.

Line 513: `return f"This test requires `{stripped[7:]}` to be true. A false result fails verification instead of allow...` This returns `f"This test requires `{stripped[7:]}` to be true. A false result fails verification instead of allowing an unchecked numerical or protocol mismatch."` to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; test: the value named 'test' in this block.

Line 514: `if_match = re.match(r"(?:el)?if\s+(.+):$", stripped)` This assigns `re.match(r"(?:el)?if\s+(.+):$", stripped)` to `if_match ` in host memory. Here if_match means the value named 'if match' in this block. The important operands are re: the value named 're' in this block; match: the value named 'match' in this block; r: the value named 'r' in this block. Context: the function explain_python_line, which handles explain python line.

Line 515: `if if_match:` This executes the following indented host-code branch only when `if_match` is true. The important operands are if_match: the value named 'if match' in this block. Context: the function explain_python_line, which handles explain python line.

Line 516: `condition = if_match.group(1)` This assigns `if_match.group(1)` to `condition ` in host memory. Here condition means the value named 'condition' in this block. The important operands are if_match: the value named 'if match' in this block; group: the value named 'group' in this block. Context: the function explain_python_line, which handles explain python line.

Line 517: `return f"This executes the following indented host-code branch only when `{condition}` is true. {summarize_...` This returns `f"This executes the following indented host-code branch only when `{condition}` is true. {summarize_expression(condition)} Context: {context}."` to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; executes: the value named 'executes' in this block.

Line 518: `if stripped == "else":` This executes the following indented host-code branch only when `stripped == "else:"` is true. The important operands are stripped: the value named 'stripped' in this block. Context: the function explain_python_line, which handles explain python line.

Line 519: `return f"This begins the alternative host-code branch when the preceding condition was false, still inside ...` This returns `f"This begins the alternative host-code branch when the preceding condition was false, still inside {context}."` to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; begins: the value named 'begins' in this block.

Line 520: `loop = re.match(r"for\s+(.+?)\s+in\s+(.+):$", stripped)` This assigns `re.match(r"for\s+(.+?)\s+in\s+(.+):$", stripped)` to `loop ` in host memory. Here loop means the value named 'loop' in this block. The important operands are re: the value named 're' in this block; match: the value named 'match' in this block; r: the value named 'r' in this block. Context: the function explain_python_line, which handles explain python line.

Line 521: `if loop:` This executes the following indented host-code branch only when `loop` is true. The important operands are loop: the value named 'loop' in this block. Context: the function explain_python_line, which handles explain python line.

Line 522: `target, source = loop.groups()` This assigns ‘loop.groups()‘ to ‘target, source ‘ in host memory. Here target means the value named ’target’ in this block. The important operands are loop: the value named ’loop’ in this block; groups: the value named ’groups’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 523: `return f"This iterates {target} over ‘{source}‘. Each iteration repeats the indented host operation, common...` This returns ‘f"This iterates {target} over ‘{source}‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in {context}."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; iterates: the value named ’iterates’ in this block.

Line 524: `if stripped.startswith("while "):` This executes the following indented host-code branch only when ‘stripped.startswith("while ")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; while: the value named ’while’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 525: `return f"This repeats the following host-code block while ‘{stripped[6:-1]}‘ remains true; surrounding time...` This returns ‘f"This repeats the following host-code block while ‘{stripped[6:-1]}‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; repeats: the value named ’repeats’ in this block.

Line 526: `if stripped.startswith("with "):` This executes the following indented host-code branch only when ‘stripped.startswith("with ")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; with: the value named ’with’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 527: `return f"This opens a managed resource scope for ‘{stripped[5:-1]}‘ so files, tracing, or temporary state a...` This returns ‘f"This opens a managed resource scope for ‘{stripped[5:-1]}‘ so files, tracing, or temporary state are finalized even if {context} raises an error."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; opens: the value named ’opens’ in this block.

Line 528: `assignment = re.match(r"([A-Za-z_][A-Za-z0-9_.\[\], ]*)\s*(?::[^=]+)?\s*=\s*(.+)", stripped)` This assigns ‘re.match(r"([A-Za-z_][A-Za-z0-9_.\[\],]*)\s*(?::[^=]+)?\s*=\s*(.+)", stripped)‘ to ‘assignment ‘ in host memory. Here assignment means the value named ’assignment’ in this block. The important operands are re: the value named ’re’ in this block; match: the value named ’match’ in this block; r: the value named ’r’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 529: `if assignment and "==" not in stripped and ">=" not in stripped and "<=" not in stripped and "!=" not in st...` This executes the following indented host-code branch only when ‘assignment and "==" not in stripped and ">=" not in stripped and "<=" not in stripped and "!=" not in stripped‘ is true. The important operands are assignment: the value named ’assignment’ in this block; and: the value named ’and’ in this block; not: the value named ’not’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 530: `lhs, rhs = assignment.groups()` This assigns ‘assignment.groups()‘ to ‘lhs, rhs ‘ in host memory. Here lhs means the value named ’lhs’ in this block. The important operands are assignment: the value named ’assignment’ in this block; groups: the value named ’groups’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 531: `name = lhs.split(",", 1)[0].strip().split("[")[0].split(".")[-1]` This assigns ‘lhs.split(",", 1)[0].strip().spl
1]‘ to ‘name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are lhs: the value named ’lhs’ in this block; split: the value named ’split’ in this block; strip: the value named ’strip’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 532: `return f"This assigns ‘{rhs}‘ to ‘{lhs}‘ in host memory. Here {name} means {identifier_meaning(nam {sum...` This returns ‘f"This assigns ‘{rhs}‘ to ‘{lhs}‘ in host memory. Here {name} means {identifier_meaning(name)}. {summarize_expression(rhs)} Context: {context}."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; assigns: the value named ’assigns’ in this block.

Line 533: `if stripped.startswith(("try:", "except ", "finally:")):` This executes the following indented host-code branch only when ‘stripped.startswith(("try:", "except ", "finally:"))‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; try: the value named ’try’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 534: `return f"This is explicit host-side failure handling inside {context}; it ensures malformed artifacts, DMA ...` This returns ‘f"This is explicit host-side failure handling inside {context}; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; is: the value named ’is’ in this block.

Line 535: `if stripped.startswith("if __name__"):` This executes the following indented host-code branch only when ‘stripped.startswith("if ___name___")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; ___name___: the value named ’name’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 536: `return "This standard entry-point guard runs the command-line main function only when this file is executed...` This returns ‘"This standard entry-point guard runs the command-line main function only when this file is executed directly, while keeping its definitions importable by tests."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are This: the value named ’this’ in this block; standard: the value named ’standard’ in this block; entry: the value named ’entry’ in this block.

Line 537: `if stripped in {"(", ")", "[", "]", "{", "}"} or stripped.endswith(("(", "[", "{")):` This executes the following indented host-code branch only when ‘stripped in {"(", ")", "[", "]", "{", "}"} or stripped.endswith(("(", "[", "{"))‘ is true. The important operands are stripped: the value named ’stripped’ in this block; in: the value named ’in’ in this block; or: the value named ’or’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 538: `return f"This line opens or closes a multi-line Python expression inside {context}; the neighboring lines s...` This returns ‘f"This line opens or closes a multi-line Python expression inside {context}; the neighboring lines supply its arguments or elements."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; line: the value named ’line’ in this block.

Line 539: `if stripped.startswith("@"):` This executes the following indented host-code branch only when ‘stripped.startswith is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block. Context: the function explain_python_line, which handles explain python line.

Line 540: `return f"This decorator changes how the following class or function is constructed. ‘{stripped}‘ is applied...` This returns ‘f"This decorator changes how the following class or function is constructed. ‘{stripped}‘ is applied at import time on the host and is part of {context}; it creates no FPGA circuit by itself."‘ to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; decorator: the value named ’decorator’ in this block.

Line 541: `if stripped.startswith(('"""', ""'")):` This executes the following indented host-code branch only when ‘stripped.startswith(('"""', ""'"))‘ is true. The important operands are stripped: the value named ’stripped’ in this

block; startswith: the value named 'startswith' in this block. Context: the function explain_python_line, which handles explain python line.

Line 542: `return f"This is executable documentation attached to {context}.  It states the intended contract that tests...` This returns 'f"This is executable documentation attached to {context}. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work."' to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; is: the value named 'is' in this block.

Line 543: `if ".add_argument(" in stripped:` This executes the following indented host-code branch only when '".add_argument(" in stripped' is true. The important operands are add_argument: the value named 'add argument' in this block; in: the value named 'in' in this block; stripped: the value named 'stripped' in this block. Context: the function explain_python_line, which handles explain python line.

Line 544: `return f"This declares a command-line option for {context}.  Its name, type, default, and required status ma...` This returns 'f"This declares a command-line option for {context}. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting."' to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; declares: the value named 'declares' in this block.

Line 545: `call = re.match(r"([A-Za-z_][A-Za-z0-9_.]*)\((.*)\),?$", stripped)` This assigns 're.match(r"([A-Za-z_][A-Za-z0-9_.]*)\((.*)\),?$", stripped)' to 'call ' in host memory. Here call means the value named 'call' in this block. The important operands are re: the value named 're' in this block; match: the value named 'match' in this block; r: the value named 'r' in this block. Context: the function explain_python_line, which handles explain python line.

Line 546: `if call:` This executes the following indented host-code branch only when 'call' is true. The important operands are call: the value named 'call' in this block. Context: the function explain_python_line, which handles explain python line.

Line 547: `return f"This calls '{call.group(1)}' with '{call.group(2)}' inside {context}.  The call advances the host-s...` This returns 'f"This calls '{call.group(1)}' with '{call.group(2)}' inside {context}. The call advances the host-side job that {purpose}."' to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; calls: the value named 'calls' in this block.

Line 548: `return f"Inside {context}, this statement advances the host implementation that {purpose}. {summarize_expre...` This returns 'f"Inside {context}, this statement advances the host implementation that {purpose}. {summarize_expression(stripped)}"' to the caller of the function explain_python_line, which handles explain python line, ending this invocation. The important operands are f: the value named 'f' in this block; Inside: the value named 'inside' in this block; context: the value named 'context' in this block.

Line 551: `def explain_script_line(line:  str, suffix:  str, purpose:  str) -> str:` This starts function explain_script_line. Its indented body performs explain script line as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 552: `stripped = line.strip()` This assigns 'line.strip()' to 'stripped ' in host memory. Here stripped means the value named 'stripped' in this block. The important operands are line: the value named 'line' in this block; strip: the value named 'strip' in this block. Context: the function explain_script_line, which handles explain script line.

Line 553: `if stripped.startswith("#"):` This executes the following indented host-code branch only when 'stripped.startswith is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block. Context: the function explain_script_line, which handles explain script line.

Line 554: `return f"This comment documents the batch-flow intent: {stripped[1:].strip()}"` This returns ‘f"This comment documents the batch-flow intent: {stripped[1:].strip()}"‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; comment: the value named ’comment’ in this block.

Line 555: `if suffix == ".tcl":` This executes the following indented host-code branch only when ‘suffix == ".tcl"‘ is true. The important operands are suffix: the value named ’suffix’ in this block; tcl: the value named ’tcl’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 556: `if stripped.startswith("read_verilog"):` This executes the following indented host-code branch only when ‘stripped.startswith("read_verilog")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; read_verilog: the value named ’read verilog’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 557: `return "This loads the listed RTL into Vivado’s in-memory design so the subsequent synthesis uses the repos...` This returns ‘"This loads the listed RTL into Vivado’s in-memory design so the subsequent synthesis uses the repository source."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are This: the value named ’this’ in this block; loads: the value named ’loads’ in this block; the: the value named ’the’ in this block.

Line 558: `if stripped.startswith("synth_design"):` This executes the following indented host-code branch only when ‘stripped.startswith("synth_design")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; synth_design: the value named ’synth design’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 559: `return "This invokes Vivado synthesis for the named top and FPGA part, mapping behavioral RTL into LUT, fli...` This returns ‘"This invokes Vivado synthesis for the named top and FPGA part, mapping behavioral RTL into LUT, flip-flop, BRAM, and DSP primitives."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are This: the value named ’this’ in this block; invokes: the value named ’invokes’ in this block; Vivado: the value named ’vivado’ in this block.

Line 560: `if stripped.startswith(("opt_design", "place_design", "phys_opt_design", "route_design")):` This executes the following indented host-code branch only when ‘stripped.startswith(("opt_design", "place_design", "phys_opt_design", "route_design"))‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; opt_design: the value named ’opt design’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 561: `return f"This runs Vivado implementation command ‘{stripped}‘, changing the physical netlist or placement/r...` This returns ‘f"This runs Vivado implementation command ‘{stripped}‘, changing the physical netlist or placement/routing needed for real timing."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; runs: the value named ’runs’ in this block.

Line 562: `if stripped.startswith("report_") or stripped.startswith("write_"):` This executes the following indented host-code branch only when ‘stripped.startswith("report_") or stripped.startswith("write_")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; report_: the value named ’report’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 563: `return f"This writes an auditable Vivado artifact using ‘{stripped}‘; later documentation reads the resulti...` This returns ‘f"This writes an auditable Vivado artifact using ‘{stripped}‘; later documentation reads the resulting report rather than inventing a metric."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; writes: the value named ’writes’ in this block.

Line 564: `if stripped.startswith("create_clock"):` This executes the following indented host-code branch only when ‘stripped.startswith("create_clock")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; create_clock: the value named ’create clock’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 565: `return "This declares the clock period against which Vivado checks every synchronous setup and hold path."` This returns ‘"This declares the clock period against which Vivado checks every synchronous setup and hold path."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are This: the value named ’this’ in this block; declares: the value named ’declares’ in this block; the: the value named ’the’ in this block.

Line 566: `if stripped.startswith("create_bd_cell"):` This executes the following indented host-code branch only when ‘stripped.startswith("create_bd_cell")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; create_bd_cell: the value named ’create bd cell’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 567: `return f"This adds the named IP or module-reference cell to the Vivado block design. The cell becomes a con...` This returns ‘f"This adds the named IP or module-reference cell to the Vivado block design. The cell becomes a concrete PS, DMA, converter, interconnect, reset, or accelerator instance after generation: ‘{stripped}‘."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; adds: the value named ’adds’ in this block.

Line 568: `if stripped.startswith("connect_bd_intf_net"):` This executes the following indented host-code branch only when ‘stripped.startswith("connect_bd_intf_net")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; connect_bd_intf_net: the value named ’connect bd intf net’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 569: `return f"This connects two complete block-design interfaces, carrying all AXI channel signals defined by th...` This returns ‘f"This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for ‘{stripped}‘."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; connects: the value named ’connects’ in this block.

Line 570: `if stripped.startswith("connect_bd_net"):` This executes the following indented host-code branch only when ‘stripped.startswith("connect_bd_net")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; connect_bd_net: the value named ’connect bd net’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 571: `return f"This connects scalar or bus pins in the block design. For clocks and resets, this exact connection...` This returns ‘f"This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: ‘{stripped}‘."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; connects: the value named ’connects’ in this block.

Line 572: `if stripped.startswith("assign_bd_address"):` This executes the following indented host-code branch only when ‘stripped.startswith("assign_bd_address")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; assign_bd_address: the value named ’assign bd address’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 573: `return "This asks Vivado to establish an AXI address map; following explicit offset/range settings make the...` This returns ‘"This asks Vivado to establish an AXI address map; following explicit offset/range settings make the software-visible register addresses stable across rebuilds."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are This: the value named ’this’ in this block; asks: the value named ’asks’ in this block; Vivado: the value named ’vivado’ in this block.

Line 574: `if stripped.startswith(("launch_runs", "wait_on_run", "open_run")):` This executes the following indented host-code branch only when ‘stripped.startswith(("launch_runs", "wait_on_run", "open_run"))‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; launch_runs: the value named ’launch runs’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 575: `return f"This controls a named Vivado synthesis or implementation run with ‘{stripped}‘. Waiting and openin...` This returns ‘f"This controls a named Vivado synthesis or implementation run with ‘{stripped}‘. Waiting and opening the completed run prevents reports from reading an earlier or unfinished netlist."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; controls: the value named ’controls’ in this block.

Line 576: `if stripped.startswith("read_saif"):` This executes the following indented host-code branch only when ‘stripped.startswith("read_saif")‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; read_saif: the value named ’read saif’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 577: `return f"This annotates implemented nets with measured XSim switching activity from the named SAIF file. Th...` This returns ‘f"This annotates implemented nets with measured XSim switching activity from the named SAIF file. The retained match report quantifies which simulated names mapped to routed nets: ‘{stripped}‘."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; annotates: the value named ’annotates’ in this block.

Line 578: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function explain_script_line, which handles explain script line.

Line 579: `if "$ErrorActionPreference" in stripped:` This executes the following indented host-code branch only when ‘"$ErrorActionPreference" in stripped‘ is true. The important operands are ErrorActionPreference: the value named ’error action preference’ in this block; in: the value named ’in’ in this block; stripped: the value named ’stripped’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 580: `return "This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful ...` This returns ‘"This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are This: the value named ’this’ in this block; makes: the value named ’makes’ in this block; PowerShell: the value named ’power shell’ in this block.

Line 581: `if stripped.startswith("param") or re.match(r"\[[A-Za-z]+\]\$", stripped):` This executes the following indented host-code branch only when ‘stripped.startswith("param") or re.match(r"\[[A-Za-z]+\]\$", stripped)‘ is true. The important operands are stripped: the value named ’stripped’ in this block; startswith: the value named ’startswith’ in this block; param: the value named ’param’ in this block. Context: the function explain_script_line, which handles explain script line.

Line 582: `return "This declares a typed command-line parameter so the flow can be rerun with explicit, validated path...` This returns ‘"This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options."‘ to the caller of the function explain_script_line, which handles explain script line,

ending this invocation. The important operands are This: the value named 'this' in this block; declares: the value named 'declares' in this block; a: the value named 'a' in this block.

Line 583: `if "LASTEXITCODE" in stripped:` This executes the following indented host-code branch only when ‘"LASTEXITCODE" in stripped‘ is true. The important operands are LASTEXITCODE: the AXI-Stream marker for the final beat of a packet or command payload; in: the value named 'in' in this block; stripped: the value named 'stripped' in this block. Context: the function explain_script_line, which handles explain script line.

Line 584: `return "This checks the external program's process exit code and turns a failed compiler, simulator, or Viv...` This returns ‘"This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are This: the value named 'this' in this block; checks: the value named 'checks' in this block; the: the value named 'the' in this block.

Line 585: `if re.search(r"vivado|xsim|xelab|xvlog|iverilog|verilator|python", stripped, re.IGNORECASE):` This executes the following indented host-code branch only when ‘re.search(r"vivado|xsim|xelab|xvlog|iverilog|verilator|python", stripped, re.IGNORECASE)‘ is true. The important operands are re: the value named 're' in this block; search: the value named 'search' in this block; r: the value named 'r' in this block. Context: the function explain_script_line, which handles explain script line.

Line 586: `return f"This locates or invokes a required tool as part of the script that {purpose}; the exact command an...` This returns ‘f"This locates or invokes a required tool as part of the script that {purpose}; the exact command and arguments remain visible here for reproducibility."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; locates: the value named 'locates' in this block.

Line 587: `if stripped.startswith(("Resolve-Path", "Join-Path", "$ProjectRoot")) or "[System.IO.Path]::GetFull in...` This executes the following indented host-code branch only when ‘stripped.startswith(("Resolve-Path", "Join-Path", "$ProjectRoot")) or "[System.IO.Path]::GetFullPath" in stripped‘ is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block; Resolve: the value named 'resolve' in this block. Context: the function explain_script_line, which handles explain script line.

Line 588: `return f"This resolves a deterministic filesystem location for the flow that {purpose}. Literal/fu paths ...` This returns ‘f"This resolves a deterministic filesystem location for the flow that {purpose}. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; resolves: the value named 'resolves' in this block.

Line 589: `if stripped.startswith("New-Item"):` This executes the following indented host-code branch only when ‘stripped.startswith("New-Item")‘ is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block; New: the value named 'new' in this block. Context: the function explain_script_line, which handles explain script line.

Line 590: `return "This creates the declared output directory before a tool writes into it. '-Force' permits an existi...` This returns ‘"This creates the declared output directory before a tool writes into it. '-Force' permits an existing directory but does not make an external tool failure succeed."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are This: the value named 'this' in this block; creates: the value named 'creates' in this block; the: the value named 'the' in this block.

Line 591: `if stripped.startswith("Remove-Item"):` This executes the following indented host-code branch only when ‘stripped.startswith("Remove-Item")‘ is true. The important operands are stripped: the value named 'stripped' in this block; startswith: the value named 'startswith' in this block; Remove: the value named 'remove' in this block. Context: the function explain_script_line, which handles explain script line.

Line 592: `return "This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard ...` This returns ‘"This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file."‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are This: the value named ’this’ in this block; removes: the value named ’removes’ in this block; the: the value named ’the’ in this block.

Line 593: `return f"This batch-control line contributes to the flow that {purpose}. {summarize_expression(str` This returns ‘f"This batch-control line contributes to the flow that {purpose}. {summarize_expression(stripped)}"‘ to the caller of the function explain_script_line, which handles explain script line, ending this invocation. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; batch: the value named ’batch’ in this block.

Line 596: `def latex_escape(text: str) -> str:` This starts function latex_escape. Its indented body performs latex escape as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 597: `replacements = {` This statement continues through source line 602; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", "<": r"\textless{}", ">": r"\textgreater{}", }‘ to ‘replacements ‘ in host memory. Here replacements means the value named ’replacements’ in this block. The important operands are r: the value named ’r’ in this block; textbackslash: the value named ’textbackslash’ in this block; _: the value named ” in this block. Context: the function latex_escape, which handles latex escape.

Line 598: `"\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$",` This is continuation line 2 of the 6-line logical statement that begins at source line 597. Its exact fragment is ‘"\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$",‘. The important operands are r: the value named ’r’ in this block; textbackslash: the value named ’textbackslash’ in this block. Read together, the complete operation is: This assigns ‘{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", "<": r"\textless{}", ">": r"\textgreater{}", }‘ to ‘replacements ‘ in host memory. Here replacements means the value named ’replacements’ in this block. The important operands are r: the value named ’r’ in this block; textbackslash: the value named ’textbackslash’ in this block; _: the value named ” in this block. Context: the function latex_escape, which handles latex escape.

Line 599: `"#": r"\#", "_": r"\_", "{": r"\{", "}": r"\}",` This is continuation line 3 of the 6-line logical statement that begins at source line 597. Its exact fragment is ‘"#": r"\#", "_": r"_", "{": r"\{", "}": r"\}",‘. The important operands are r: the value named ’r’ in this block; _: the value named ” in this block. Read together, the complete operation is: This assigns ‘{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", "<": r"\textless{}", ">": r"\textgreater{}", }‘ to ‘replacements ‘ in host memory. Here replacements means the value named ’replacements’ in this block. The important operands are r: the value named ’r’ in this block; textbackslash: the value named ’textbackslash’ in this block; _: the value named ” in this block. Context: the function latex_escape, which handles latex escape.

Line 600: `"~": r"\textasciitilde{}", "^": r"\textasciicircum{}", "<": r"\textless{}",` This is continuation line 4 of the 6-line logical statement that begins at source line 597. Its exact fragment is ‘"~": r"\textasciitilde{}", "^": r"\textasciicircum{}", "<": r"\textless{}",‘. The important operands are r: the value named ’r’ in this block; textasciitilde: the value named ’textasciitilde’ in this block; textasciicircum: the value named ’textasciicircum’ in this block. Read together, the complete operation is: This assigns ‘{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", "<": r"\textless{}", ">": r"\textgreater{}", }‘ to ‘replacements ‘ in host memory. Here replacements means the value named ’replacements’ in this block. The important operands are r: the value named ’r’ in this block; textbackslash: the value named ’textbackslash’ in this block; _: the value named ” in this block. Context: the function latex_escape, which handles latex escape.

Line 601: `">": r"\textgreater{}",` This is continuation line 5 of the 6-line logical statement that begins at source line 597. Its exact fragment is ‘">": r"\textgreater{}",‘. The important operands are r: the value named ’r’ in this

block; textgreater: the value named 'textgreater' in this block. Read together, the complete operation is: This assigns '{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", "<": r"\textless{}", ">": r"\textgreater{}", }' to 'replacements ' in host memory. Here replacements means the value named 'replacements' in this block. The important operands are r: the value named 'r' in this block; textbackslash: the value named 'textbackslash' in this block; _: the value named '' in this block. Context: the function latex_escape, which handles latex escape.

Line 602: `}` This is continuation line 6 of the 6-line logical statement that begins at source line 597. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "\\": r"\textbackslash{}", "&": r"\&", "%": r"\%", "$": r"\$", "#": r"\#", "_": r"_", "{": r"\{", "}": r"\}", "~": r"\textasciitilde{}", "^": r"\textasciicircum{}", "<": r"\textless{}", ">": r"\textgreater{}", }' to 'replacements ' in host memory. Here replacements means the value named 'replacements' in this block. The important operands are r: the value named 'r' in this block; textbackslash: the value named 'textbackslash' in this block; _: the value named '' in this block. Context: the function latex_escape, which handles latex escape.

Line 603: `return "".join(replacements.get(character, character) for character in text)` This returns '"".join(replacements.get(character, character) for character in text)' to the caller of the function latex_escape, which handles latex escape, ending this invocation. The important operands are join: the value named 'join' in this block; replacements: the value named 'replacements' in this block; get: the value named 'get' in this block.

Line 606: `def source_architecture_notes(entry: SourceEntry, text: str) -> list[tuple[str, str]]:` This starts function source_architecture_notes. Its indented body performs source architecture notes as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 607: `"""Build a file-level map so line annotations have an understandable frame."""` This is executable documentation attached to the function source_architecture_notes, which handles source architecture notes. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 608: `suffix = entry.path.suffix.lower()` This assigns 'entry.path.suffix.lower()' to 'suffix ' in host memory. Here suffix means the value named 'suffix' in this block. The important operands are entry: the value named 'entry' in this block; path: the value named 'path' in this block; suffix: the value named 'suffix' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 609: `dataflow = FILE_DATAFLOWS.get(` This statement continues through source line 612; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'FILE_DATAFLOWS.get(entry.relative, f"This file's end-to-end responsibility is to {entry.purpose}.",)' to 'dataflow ' in host memory. Here dataflow means the value named 'dataflow' in this block. The important operands are FILE_DATAFLOWS: the value named 'file dataflows' in this block; get: the value named 'get' in this block; entry: the value named 'entry' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 610: `entry.relative,` This is continuation line 2 of the 4-line logical statement that begins at source line 609. Its exact fragment is 'entry.relative,'. The important operands are entry: the value named 'entry' in this block; relative: the value named 'relative' in this block. Read together, the complete operation is: This assigns 'FILE_DATAFLOWS.get(entry.relative, f"This file's end-to-end responsibility is to {entry.purpose}.",)' to 'dataflow ' in host memory. Here dataflow means the value named 'dataflow' in this block. The important operands are FILE_DATAFLOWS: the value named 'file dataflows' in this block; get: the value named 'get' in this block; entry: the value named 'entry' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 611: `f"This file's end-to-end responsibility is to {entry.purpose}.",` This is continuation line 3 of the 4-line logical statement that begins at source line 609. Its exact fragment is 'f"This file's end-to-end responsibility is to {entry.purpose}.",'. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; file: the value named 'file' in this block. Read together, the complete operation is: This assigns 'FILE_DATAFLOWS.get(entry.relative, f"This file's end-to-end responsibility is to {entry.purpose}.",)' to 'dataflow ' in host memory. Here dataflow means the value named 'dataflow' in this block. The important operands are

FILE_DATAFLOWS: the value named 'file dataflows' in this block; get: the value named 'get' in this block; entry: the value named 'entry' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 612: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 609. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'FILE_DATAFLOWS.get(entry.relative, f"This file's end-to-end responsibility is to {entry.purpose}.",)' to 'dataflow ' in host memory. Here dataflow means the value named 'dataflow' in this block. The important operands are FILE_DATAFLOWS: the value named 'file dataflows' in this block; get: the value named 'get' in this block; entry: the value named 'entry' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 613: `notes: list[tuple[str, str]] = [("Dataflow responsibility", dataflow)]` This assigns '[("Dataflow responsibility", dataflow)]' to 'notes' in host memory. Here notes means the value named 'notes' in this block. The important operands are Dataflow: the value named 'dataflow' in this block; responsibility: the value named 'responsibility' in this block; dataflow: the value named 'dataflow' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 614: `if suffix in {".sv", ".v"} and not entry.relative.startswith("tb/"):` This executes the following indented host-code branch only when 'suffix in {".sv", ".v"} and not entry.relative.startswith("tb/")' is true. The important operands are suffix: the value named 'suffix' in this block; in: the value named 'in' in this block; sv: the value named 'sv' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 615: `modules = re.findall(r"^\s*module\s+([A-Za-z_][A-Za-z0-9_$]*)", text, re.MULTILINE)` This assigns 're.findall(r"^\s*module\s+([A-Za-z_][A-Za-z0-9_$]*)", text, re.MULTILINE)' to 'modules ' in host memory. Here modules means the value named 'modules' in this block. The important operands are re: the value named 're' in this block; findall: the value named 'findall' in this block; r: the value named 'r' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 616: `inputs = len(re.findall(r"^\s*input\b", text, re.MULTILINE))` This assigns 'len(re.findall(r"^\s*input\b", text, re.MULTILINE))' to 'inputs ' in host memory. Here inputs means the value named 'inputs' in this block. The important operands are len: the value named 'len' in this block; re: the value named 're' in this block; findall: the value named 'findall' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 617: `outputs = len(re.findall(r"^\s*output\b", text, re.MULTILINE))` This assigns 'len(re.findall(r"^\s*output\b' text, re.MULTILINE))' to 'outputs ' in host memory. Here outputs means the value named 'outputs' in this block. The important operands are len: the value named 'len' in this block; re: the value named 're' in this block; findall: the value named 'findall' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 618: `clocked = len(re.findall(r"\balways_ff\b", text))` This assigns 'len(re.findall(r"\balways_ff\b", text))' to 'clocked ' in host memory. Here clocked means the value named 'clocked' in this block. The important operands are len: the value named 'len' in this block; re: the value named 're' in this block; findall: the value named 'findall' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 619: `combinational = len(re.findall(r"\balways_comb\b", text))` This assigns 'len(re.findall(r"\balways_comb\b", text))' to 'combinational ' in host memory. Here combinational means the value named 'combinational' in this block. The important operands are len: the value named 'len' in this block; re: the value named 're' in this block; findall: the value named 'findall' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 620: `states = sorted(set(re.findall(r"\bST_[A-Z0-9_]+\b", text)))` This assigns 'sorted(set(re.findall(r"\bST_[A-Z0-9_]+\b", text)))' to 'states ' in host memory. Here states means the finite-state-machine phase retained in flip-flops. The important operands are sorted: the value named 'sorted' in this block; set: the value named 'set' in this block; re: the value named 're' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 621: `instances: list[str] = []` This assigns '[]' to 'instances' in host memory. Here instances means the value named 'instances' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function source_architecture_notes, which handles source architecture notes.

Line 622: `instance_pattern = re.compile(` This statement continues through source line 625; the explanation uses the complete expression rather than only the first visual fragment. This assigns 're.compile(r"^\s*([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?" r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", re.MULTILINE,)' to 'instance_pattern ' in host memory. Here instance_pattern means the value named 'instance pattern' in this block. The important operands are re: the value named 're' in this block; compile: the value named 'compile' in this block; r: the value named 'r' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 623: `r"^\s*([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?"` This is continuation line 2 of the 4-line logical statement that begins at source line 622. Its exact fragment is 'r"^\s*([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?"'. The important operands are r: the value named 'r' in this block; s: the value named 's' in this block; A: the value named 'a' in this block. Read together, the complete operation is: This assigns 're.compile(r"^\s*([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?" r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", re.MULTILINE,)' to 'instance_pattern ' in host memory. Here instance_pattern means the value named 'instance pattern' in this block. The important operands are re: the value named 're' in this block; compile: the value named 'compile' in this block; r: the value named 'r' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 624: `r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", re.MULTILINE,` This is continuation line 3 of the 4-line logical statement that begins at source line 622. Its exact fragment is 'r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", re.MULTILINE,'. The important operands are r: the value named 'r' in this block; A: the value named 'a' in this block; Za: the value named 'za' in this block. Read together, the complete operation is: This assigns 're.compile(r"^\s*([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?" r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", re.MULTILINE,)' to 'instance_pattern ' in host memory. Here instance_pattern means the value named 'instance pattern' in this block. The important operands are re: the value named 're' in this block; compile: the value named 'compile' in this block; r: the value named 'r' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 625: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 622. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 're.compile(r"^\s*([A-Za-z_][A-Za-z0-9_$]*)\s+(?:#\s*\([^;]*\)\s*)?" r"([A-Za-z_][A-Za-z0-9_$]*)\s*\(", re.MULTILINE,)' to 'instance_pattern ' in host memory. Here instance_pattern means the value named 'instance pattern' in this block. The important operands are re: the value named 're' in this block; compile: the value named 'compile' in this block; r: the value named 'r' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 626: `for module_name, instance_name in instance_pattern.findall(text):` This iterates module_name, instance_name over 'instance_pattern.findall(text)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function source_architecture_notes, which handles source architecture notes.

Line 627: `if module_name in {"module", "function", "task", "if", "for", "while", "case"}:` This executes the following indented host-code branch only when 'module_name in {"module", "function", "task", "if", "for", "while", "case"}' is true. The important operands are module_name: the value named 'module name' in this block; in: the value named 'in' in this block; module: the value named 'module' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 628: `continue` Inside the function source_architecture_notes, which handles source architecture notes, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are continue: the value named 'continue' in this block.

Line 629: `label = f"{module_name} as {instance_name}"` This assigns 'f"{module_name} as {instance_name}"' to 'label ' in host memory. Here label means the value named 'label' in this block. The important operands are f: the value named 'f' in this block; module_name: the value named 'module name' in this block; as: the value named 'as' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 630: `if label not in instances:` This executes the following indented host-code branch only when ‘label not in instances‘ is true. The important operands are label: the value named ’label’ in this block; not: the value named ’not’ in this block; in: the value named ’in’ in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 631: `instances.append(label)` This calls ‘instances.append‘ with ‘label‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 632: `notes.extend(` This statement continues through source line 661; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child’s named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface’s valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 633: `[` This is continuation line 2 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘[‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child’s named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface’s valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 634: `(` This is continuation line 3 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘(‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is

primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 635: `"What becomes FPGA hardware",` This is continuation line 4 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘"What becomes FPGA hardware",‘. The important operands are What: the value named 'what' in this block; becomes: the value named 'becomes' in this block; FPGA: the value named 'fpga' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 636: `f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. Th...` This is continuation line 5 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",‘. The important operands are f: the value named 'f' in this block; Vivado: the value named 'vivado' in this block; elaborates: the value named 'elaborates' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 637: `),` This is continuation line 6 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 638: `(` This is continuation line 7 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘(‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 639: `"Cycle-level control",` This is continuation line 8 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘"Cycle-level control",‘. The important operands are Cycle: the value named 'cycle' in this block; level: the value named 'level' in this block; control: the value named 'control' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function

source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 640: `(` This is continuation line 9 of the 30-line logical statement that begins at source line 632. Its exact fragment is '('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'notes.extend' with ' [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 641: `f"The named control phases include {', '.join(states[:18])}"` This is continuation line 10 of the 30-line logical statement that begins at source line 632. Its exact fragment is 'f"The named control phases include {', '.join(states[:18])}"'. The important operands are f: the value named 'f' in this block; The: the value named 'the' in this block; named: the value named 'named' in this block. Read together, the complete operation is: This calls 'notes.extend' with ' [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 642: `+ (" and additional states" if len(states) > 18 else "")` This is continuation line 11 of the 30-line logical statement that begins at source line 632. Its exact fragment is '+ (" and additional states" if len(states) > 18 else "")'. The important operands are and: the value named 'and' in this block; additional: the value named 'additional' in this block; states: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This calls 'notes.extend' with ' [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ",

".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 643: `+ ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and...` This is continuation line 12 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘+ ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update."‘. The important operands are A: the value named 'a' in this block; state: the finite-state-machine phase retained in flip-flops; is: the value named 'is' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 644: `if states` This is continuation line 13 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘if states‘. The important operands are states: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 645: `else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers ...` This is continuation line 14 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."‘. The important operands are This: the value named 'this' in this block; block: the value named 'block' in this block; is: the value named 'is' in

this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 646:), This is continuation line 15 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 647:), This is continuation line 16 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 648: `(` This is continuation line 17 of the 30-line logical statement that begins at source line 632. Its exact fragment is '('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'notes.extend' with ' [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 649: `"Connections to other blocks",` This is continuation line 18 of the 30-line logical statement that begins at source line 632. Its exact fragment is '"Connections to other blocks",'. The important operands are Connections: the value named 'connections' in this block; to: the value named 'to' in this block; other: the value named 'other' in this block. Read together, the complete operation is: This calls 'notes.extend' with ' [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 650: `(` This is continuation line 19 of the 30-line logical statement that begins at source line 632. Its exact fragment is '('. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'notes.extend' with ' [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ' inside the function

source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 651: `"This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to ...` This is continuation line 20 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘"This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection."‘. The important operands are This: the value named 'this' in this block; source: the value named 'source' in this block; instantiates: the value named 'instantiates' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 652: `if instances` This is continuation line 21 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘if instances‘. The important operands are instances: the value named 'instances' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 653: `else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another...` This is continuation line 22 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."‘. The important operands are This: the value named 'this' in this block; is: the value named 'is' in this block; a: the value named 'a' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted

valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 654:), This is continuation line 23 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 655:), This is continuation line 24 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 656: (This is continuation line 25 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘(‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es).

Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 657: `"How to debug it",` This is continuation line 26 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘"How to debug it",‘. The important operands are How: the value named 'how' in this block; to: the value named 'to' in this block; debug: the value named 'debug' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 658: `"In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals tog...` This is continuation line 27 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘"In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",‘. The important operands are In: the value named 'in' in this block; a: the value named 'a' in this block; waveform: the value named 'waveform' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates

this source-reference volume and its source-integrity manifest.

Line 659:), This is continuation line 28 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 660:] This is continuation line 29 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid, ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 661:) This is continuation line 30 of the 30-line logical statement that begins at source line 632. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("What becomes FPGA hardware", f"Vivado elaborates module(s) {', '.join(modules) or 'declared here'} into physical nets and primitives. The source declares {inputs} input-port lines, {outputs} output-port lines, {clocked} clocked process(es), and {combinational} combinational process(es). Clocked assignments become retained state; combinational assignments become gates, arithmetic, and multiplexers.",), ("Cycle-level control", (f"The named control phases include {', '.join(states[:18])}" + (" and additional states" if len(states) > 18 else "") + ". A state is a binary value held in flip-flops. On each rising edge, accepted valid/ready handshakes and current state determine the next state and which memories or output registers update." if states else "This block is primarily a datapath or storage primitive rather than a large state machine. Registers still divide combinational work into cycles and hold valid/data stable during downstream stalls."),), ("Connections to other blocks", ("This source instantiates " + ", ".join(instances[:12]) + ". Port-map lines connect actual parent wires to each child's named ports; no data is copied merely by naming a connection." if instances else "This is a leaf block: parent modules create it and connect its ports. It does not instantiate another project RTL module, so its arithmetic or storage maps directly into FPGA primitives."),), ("How to debug it", "In a waveform, place clock and reset first, then each interface's valid,

ready, data, and last signals together. A transfer exists only on an edge where valid and ready are both one. Add state, address/index, accumulator, and error signals to explain why the interface advanced or stalled.",),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 662: `elif entry.relative.startswith("tb/"):` This executes the following indented host-code branch only when ‘entry.relative.startswith("tb/")‘ is true. The important operands are entry: the value named ’entry’ in this block; relative: the value named ’relative’ in this block; startswith: the value named ’startswith’ in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 663: `tops = re.findall(r"^\s*module\s+([A-Za-z_][A-Za-z0-9_$]*)", text, re.MULTILINE)` This assigns ‘re.findall(r"^\s*module\s+([A-Za-z_][A-Za-z0-9_$]*)", text, re.MULTILINE)‘ to ‘tops ‘ in host memory. Here tops means the value named ’tops’ in this block. The important operands are re: the value named ’re’ in this block; findall: the value named ’findall’ in this block; r: the value named ’r’ in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 664: `checks = len(re.findall(r"\$(?:fatal|error)|\bassert\b|FAILED|PASSED", text))` This assigns ‘len(re.findall(r"\$(?:fatal|error)|\bassert\b|FAILED|PASSED", text))‘ to ‘checks ‘ in host memory. Here checks means the value named ’checks’ in this block. The important operands are len: the value named ’len’ in this block; re: the value named ’re’ in this block; findall: the value named ’findall’ in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 665: `notes.extend(` This statement continues through source line 670; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘notes.extend‘ with ‘ [("Simulation-only status", f"{’, ’.join(tops) or ’This testbench’} is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker."), ("Evidence produced", f"The file contains approximately {checks} explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 666: `[` This is continuation line 2 of the 6-line logical statement that begins at source line 665. Its exact fragment is ‘[‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Simulation-only status", f"{’, ’.join(tops) or ’This testbench’} is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker."), ("Evidence produced", f"The file contains approximately {checks} explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 667: `("Simulation-only status", f"{’, ’.join(tops) or ’This testbench’} is not synthesized into the FPGA. It gen...` This is continuation line 3 of the 6-line logical statement that begins at source line 665. Its exact fragment is ‘("Simulation-only status", f"{’, ’.join(tops) or ’This testbench’} is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker."),‘. The important operands are Simulation: the value named ’simulation’ in this block; only: the value named ’only’ in this block; status: the value named ’status’ in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Simulation-only status", f"{’, ’.join(tops) or ’This testbench’} is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker."), ("Evidence produced", f"The file contains approximately {checks} explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 668: `("Evidence produced", f"The file contains approximately {checks} explicit assertions, failure diagnostics, ...` This is continuation line 4 of the 6-line logical statement that begins at source line 665. Its exact fragment is '("Evidence produced", f"The file contains approximately {checks} explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient."),'. The important operands are Evidence: the value named 'evidence' in this block; produced: the value named 'produced' in this block; f: the value named 'f' in this block. Read together, the complete operation is: This calls 'notes.extend' with ' [("Simulation-only status", f"{', '.join(tops) or 'This testbench'} is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker."), ("Evidence produced", f"The file contains approximately {checks} explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient."),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 669: `]` This is continuation line 5 of the 6-line logical statement that begins at source line 665. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'notes.extend' with ' [("Simulation-only status", f"{', '.join(tops) or 'This testbench'} is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker."), ("Evidence produced", f"The file contains approximately {checks} explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient."),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 670: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 665. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'notes.extend' with ' [("Simulation-only status", f"{', '.join(tops) or 'This testbench'} is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker."), ("Evidence produced", f"The file contains approximately {checks} explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient."),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 671: `elif suffix == ".py":` This executes the following indented host-code branch only when 'suffix == ".py"' is true. The important operands are suffix: the value named 'suffix' in this block; py: the value named 'py' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 672: `try:` This is explicit host-side failure handling inside the function source_architecture_notes, which handles source architecture notes; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 673: `tree = ast.parse(text)` This assigns 'ast.parse(text)' to 'tree ' in host memory. Here tree means the value named 'tree' in this block. The important operands are ast: the value named 'ast' in this block; parse: the value named 'parse' in this block; text: the value named 'text' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 674: `classes = [node.name for node in ast.walk(tree) if isinstance(node, ast.ClassDef)]` This assigns '[node.name for node in ast.walk(tree) if isinstance(node, ast.ClassDef)]' to 'classes ' in host memory. Here classes means the value named 'classes' in this block. The important operands are node: the value named 'node' in this block; name: the value named 'name' in this block; for: the value named 'for' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 675: `functions = [node.name for node in ast.walk(tree) if isinstance(node, (ast.FunctionDef, ast.AsyncFu`
This assigns '[node.name for node in ast.walk(tree) if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef))]' to 'functions ' in host memory. Here functions means the value named 'functions' in this block. The important operands

are node: the value named 'node' in this block; name: the value named 'name' in this block; for: the value named 'for' in this block. Context: the function source_architecture_notes, which handles source architecture notes.

Line 676: `except SyntaxError:` This is explicit host-side failure handling inside the function source_architecture_notes, which handles source architecture notes; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 677: `classes, functions = [], []` This assigns '[], []' to 'classes, functions ' in host memory. Here classes means the value named 'classes' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function source_architecture_notes, which handles source architecture notes.

Line 678: `notes.extend(` This statement continues through source line 684; the explanation uses the complete expression rather than only the first visual fragment. This calls 'notes.extend' with ' [("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite."), ("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(functions[:24]) or 'none'}. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it."), ("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact."),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 679: `[` This is continuation line 2 of the 7-line logical statement that begins at source line 678. Its exact fragment is '['. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'notes.extend' with ' [("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite."), ("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(functions[:24]) or 'none'}. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it."), ("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact."),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 680: `("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA wo...` This is continuation line 3 of the 7-line logical statement that begins at source line 678. Its exact fragment is '("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite."),'. The important operands are Where: the value named 'where' in this block; it: the value named 'it' in this block; runs: the value named 'runs' in this block. Read together, the complete operation is: This calls 'notes.extend' with ' [("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite."), ("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(functions[:24]) or 'none'}. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it."), ("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact."),] ' inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 681: `("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(funct...` This is continuation line 4 of the 7-line logical statement that begins at source line 678. Its exact fragment is '("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(functions[:24]) or 'none'}. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it."),'. The important operands are Public: the value named 'public' in this block; structure: the value

named 'structure' in this block; f: the value named 'f' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite."), ("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(functions[:24]) or 'none'}. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it."), ("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 682: `("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the h...` This is continuation line 5 of the 7-line logical statement that begins at source line 678. Its exact fragment is ‘("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact."),‘. The important operands are Numeric: the value named 'numeric' in this block; and: the value named 'and' in this block; failure: the value named 'failure' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite."), ("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(functions[:24]) or 'none'}. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it."), ("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 683: `]` This is continuation line 6 of the 7-line logical statement that begins at source line 678. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite."), ("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(functions[:24]) or 'none'}. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it."), ("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 684: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 678. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Where it runs", "This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite."), ("Public structure", f"Classes: {', '.join(classes[:16]) or 'none'}. Functions: {', '.join(functions[:24]) or 'none'}. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it."), ("Numeric and failure boundary", "Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 685: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function source_architecture_notes, which handles source architecture notes.

Line 686: `tool = "Vivado Tcl" if suffix == ".tcl" else "PowerShell"` Inside the function source_architecture_notes, which handles source architecture notes, this statement advances the host implementation that generates this source-

reference volume and its source-integrity manifest. The important operands are tool: the value named 'tool' in this block; Vivado: the value named 'vivado' in this block; Tcl: the value named 'tcl' in this block.

Line 687: `notes.extend(` This statement continues through source line 692; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘notes.extend‘ with ‘ [("Where it runs", f"This is {tool} orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable."), ("Reproducibility contract", "Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 688: `[` This is continuation line 2 of the 6-line logical statement that begins at source line 687. Its exact fragment is ‘[‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Where it runs", f"This is {tool} orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable."), ("Reproducibility contract", "Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 689: `("Where it runs", f"This is {tool} orchestration, not accelerator datapath hardware. It makes tool inputs, ...` This is continuation line 3 of the 6-line logical statement that begins at source line 687. Its exact fragment is ‘("Where it runs", f"This is {tool} orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable."),‘. The important operands are Where: the value named 'where' in this block; it: the value named 'it' in this block; runs: the value named 'runs' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Where it runs", f"This is {tool} orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable."), ("Reproducibility contract", "Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 690: `("Reproducibility contract", "Every external command must be checked, every required artifact must exist an...` This is continuation line 4 of the 6-line logical statement that begins at source line 687. Its exact fragment is ‘("Reproducibility contract", "Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate."),‘. The important operands are Reproducibility: the value named 'reproducibility' in this block; contract: the value named 'contract' in this block; Every: the value named 'every' in this block. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Where it runs", f"This is {tool} orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable."), ("Reproducibility contract", "Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 691: `]` This is continuation line 5 of the 6-line logical statement that begins at source line 687. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Where it runs", f"This is {tool} orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable."), ("Reproducibility contract", "Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 692: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 687. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘notes.extend‘ with ‘ [("Where it runs", f"This is {tool} orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable."), ("Reproducibility contract", "Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate."),] ‘ inside the function source_architecture_notes, which handles source architecture notes. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 693: `return notes` This returns ‘notes‘ to the caller of the function source_architecture_notes, which handles source architecture notes, ending this invocation. The important operands are notes: the value named ’notes’ in this block.

Line 696: `def annotation_tex(entry: SourceEntry, text: str) -> tuple[list[str], int]:` This starts function annotation_tex. Its indented body performs annotation tex as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 697: `lines = text.splitlines()` This assigns ‘text.splitlines()‘ to ‘lines ‘ in host memory. Here lines means the value named ’lines’ in this block. The important operands are text: the value named ’text’ in this block; splitlines: the value named ’splitlines’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 698: `suffix = entry.path.suffix.lower()` This assigns ‘entry.path.suffix.lower()‘ to ‘suffix ‘ in host memory. Here suffix means the value named ’suffix’ in this block. The important operands are entry: the value named ’entry’ in this block; path: the value named ’path’ in this block; suffix: the value named ’suffix’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 699: `contexts = python_contexts(text) if suffix == ".py" else sv_contexts(lines) if suffix in {".sv", ".v"} else {}` Inside the function annotation_tex, which handles annotation tex, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are contexts: the value named ’contexts’ in this block; python_contexts: the value named ’python contexts’ in this block; text: the value named ’text’ in this block.

Line 700: `statement_spans = (` This statement continues through source line 704; the explanation uses the complete expression rather than only the first visual fragment. Inside the function annotation_tex, which handles annotation tex, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are statement_spans: the finite-state-machine phase retained in flip-flops; python_statement_spans: the finite-state-machine phase retained in flip-flops; text: the value named ’text’ in this block.

Line 701: `python_statement_spans(text) if suffix == ".py"` This is continuation line 2 of the 5-line logical statement that begins at source line 700. Its exact fragment is ‘python_statement_spans(text) if suffix == ".py"‘. The important operands are python_statement_spans: the finite-state-machine phase retained in flip-flops; text: the value named ’text’ in this block; suffix: the value named ’suffix’ in this block. Read together, the complete operation is: Inside the function annotation_tex, which handles annotation tex, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are statement_spans: the finite-state-machine phase retained in flip-flops; python_statement_spans: the finite-state-machine phase retained in flip-flops; text: the value named ’text’ in this block.

Line 702: `else sv_statement_spans(lines) if suffix in {".sv", ".v"}` This is continuation line 3 of the 5-line logical statement that begins at source line 700. Its exact fragment is ‘else sv_statement_spans(lines) if suffix in {".sv", ".v"}‘. The important operands are sv_statement_spans: the finite-state-machine phase retained in flip-flops; lines: the value named ’lines’ in this block; suffix: the value named ’suffix’ in this block. Read together, the complete operation is: Inside the function annotation_tex, which handles annotation tex, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are statement_spans: the finite-state-machine phase retained in flip-flops; python_statement_spans: the finite-state-machine phase retained in flip-flops; text: the value named ’text’ in this block.

Line 703: `else {}` This is continuation line 4 of the 5-line logical statement that begins at source line 700. Its exact fragment is ‘else {}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function annotation_tex, which handles annotation tex, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are statement_spans: the finite-state-machine phase retained in flip-flops; python_statement_spans: the finite-state-machine phase retained in flip-flops; text: the value named ’text’ in this block.

Line 704: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 700. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function annotation_tex, which handles annotation tex, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are statement_spans: the finite-state-machine phase retained in flip-flops; python_statement_spans: the finite-state-machine phase retained in flip-flops; text: the value named ’text’ in this block.

Line 705: `result: list[str] = []` This assigns ‘[]‘ to ‘result‘ in host memory. Here result means the value named ’result’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function annotation_tex, which handles annotation tex.

Line 706: `explained = 0` This assigns ‘0‘ to ‘explained ‘ in host memory. Here explained means the value named ’explained’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function annotation_tex, which handles annotation tex.

Line 707: `for line_number, raw in enumerate(lines, start=1):` This iterates line_number, raw over ‘enumerate(lines, start=1)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function annotation_tex, which handles annotation tex.

Line 708: `if not raw.strip():` This executes the following indented host-code branch only when ‘not raw.strip()‘ is true. The important operands are not: the value named ’not’ in this block; raw: the value named ’raw’ in this block; strip: the value named ’strip’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 709: `continue` Inside the function annotation_tex, which handles annotation tex, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are continue: the value named ’continue’ in this block.

Line 710: `context = contexts.get(line_number, f"the file whose role is to {entry.purpose}")` This assigns ‘contexts.get(line_number, f"the file whose role is to {entry.purpose}")‘ to ‘context ‘ in host memory. Here context means the value named ’context’ in this block. The important operands are contexts: the value named ’contexts’ in this block; get: the value named ’get’ in this block; line_number: the value named ’line number’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 711: `span = statement_spans.get(line_number)` This assigns ‘statement_spans.get(line_number)‘ to ‘span ‘ in host memory. Here span means the value named ’span’ in this block. The important operands are statement_spans: the finite-state-machine phase retained in flip-flops; get: the value named ’get’ in this block; line_number: the value named ’line number’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 712: `if suffix in {".sv", ".v"}:` This executes the following indented host-code branch only when ‘suffix in {".sv", ".v"}‘ is true. The important operands are suffix: the value named ’suffix’ in this block; in: the value named ’in’ in this block; sv: the value named ’sv’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 713: `complete_explanation = explain_sv_line(` This statement continues through source line 714; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘explain_sv_line(span.text if span else raw, context, entry.purpose)‘ to ‘complete_explanation ‘ in host memory. Here complete_explanation means the value named ’complete explanation’ in this block. The important operands are explain_sv_line: the value named

'explain sv line' in this block; span: the value named 'span' in this block; text: the value named 'text' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 714: `span.text if span else raw, context, entry.purpose)` This is continuation line 2 of the 2-line logical statement that begins at source line 713. Its exact fragment is 'span.text if span else raw, context, entry.purpose)'. The important operands are span: the value named 'span' in this block; text: the value named 'text' in this block; raw: the value named 'raw' in this block. Read together, the complete operation is: This assigns 'explain_sv_line(span.text if span else raw, context, entry.purpose)' to 'complete_explanation ' in host memory. Here complete_explanation means the value named 'complete explanation' in this block. The important operands are explain_sv_line: the value named 'explain sv line' in this block; span: the value named 'span' in this block; text: the value named 'text' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 715: `elif suffix == ".py":` This executes the following indented host-code branch only when 'suffix == ".py"' is true. The important operands are suffix: the value named 'suffix' in this block; py: the value named 'py' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 716: `complete_explanation = explain_python_line(` This statement continues through source line 717; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'explain_python_line(span.text if span else raw, context, entry.purpose)' to 'complete_explanation ' in host memory. Here complete_explanation means the value named 'complete explanation' in this block. The important operands are explain_python_line: the value named 'explain python line' in this block; span: the value named 'span' in this block; text: the value named 'text' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 717: `span.text if span else raw, context, entry.purpose)` This is continuation line 2 of the 2-line logical statement that begins at source line 716. Its exact fragment is 'span.text if span else raw, context, entry.purpose)'. The important operands are span: the value named 'span' in this block; text: the value named 'text' in this block; raw: the value named 'raw' in this block. Read together, the complete operation is: This assigns 'explain_python_line(span.text if span else raw, context, entry.purpose)' to 'complete_explanation ' in host memory. Here complete_explanation means the value named 'complete explanation' in this block. The important operands are explain_python_line: the value named 'explain python line' in this block; span: the value named 'span' in this block; text: the value named 'text' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 718: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function annotation_tex, which handles annotation tex.

Line 719: `complete_explanation = explain_script_line(raw, suffix, entry.purpose)` This assigns 'explain_script_line suffix, entry.purpose)' to 'complete_explanation ' in host memory. Here complete_explanation means the value named 'complete explanation' in this block. The important operands are explain_script_line: the value named 'explain script line' in this block; raw: the value named 'raw' in this block; suffix: the value named 'suffix' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 720: `if span and line_number != span.start:` This executes the following indented host-code branch only when 'span and line_number != span.start' is true. The important operands are span: the value named 'span' in this block; and: the value named 'and' in this block; line_number: the value named 'line number' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 721: `position = line_number - span.start + 1` This assigns 'line_number - span.start + 1' to 'position ' in host memory. Here position means the value named 'position' in this block. The important operands are line_number: the value named 'line number' in this block; span: the value named 'span' in this block; start: the value named 'start' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 722: `length = span.end - span.start + 1` This assigns 'span.end - span.start + 1' to 'length ' in host memory. Here length means the value named 'length' in this block. The important operands are span: the value named 'span' in this block; start: the value named 'start' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 723: `explanation = (` This statement continues through source line 728; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(f"This is continuation line {position} of the {length}-line logical statement " f"that begins at source line {span.start}. Its exact fragment is ‘{raw.strip()}‘. " f"{summarize_expression(raw)} Read together, the complete operation is: " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named ’explanation’ in this block. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; is: the value named ’is’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 724: `f"This is continuation line {position} of the {length}-line logical statement "` This is continuation line 2 of the 6-line logical statement that begins at source line 723. Its exact fragment is ‘f"This is continuation line {position} of the {length}-line logical statement "‘. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; is: the value named ’is’ in this block. Read together, the complete operation is: This assigns ‘(f"This is continuation line {position} of the {length}-line logical statement " f"that begins at source line {span.start}. Its exact fragment is ‘{raw.strip()}‘. " f"{summarize_expression(raw)} Read together, the complete operation is: " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named ’explanation’ in this block. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; is: the value named ’is’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 725: `f"that begins at source line {span.start}. Its exact fragment is ‘{raw.strip()}‘. "` This is continuation line 3 of the 6-line logical statement that begins at source line 723. Its exact fragment is ‘f"that begins at source line {span.start}. Its exact fragment is ‘{raw.strip()}‘. "‘. The important operands are f: the value named ’f’ in this block; that: the value named ’that’ in this block; begins: the value named ’begins’ in this block. Read together, the complete operation is: This assigns ‘(f"This is continuation line {position} of the {length}-line logical statement " f"that begins at source line {span.start}. Its exact fragment is ‘{raw.strip()}‘. " f"{summarize_expression(raw)} Read together, the complete operation is: " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named ’explanation’ in this block. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; is: the value named ’is’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 726: `f"{summarize_expression(raw)} Read together, the complete operation is: "` This is continuation line 4 of the 6-line logical statement that begins at source line 723. Its exact fragment is ‘f"{summarize_expression(raw)} Read together, the complete operation is: "‘. The important operands are f: the value named ’f’ in this block; summarize_expression: the value named ’summarize expression’ in this block; raw: the value named ’raw’ in this block. Read together, the complete operation is: This assigns ‘(f"This is continuation line {position} of the {length}-line logical statement " f"that begins at source line {span.start}. Its exact fragment is ‘{raw.strip()}‘. " f"{summarize_expression(raw)} Read together, the complete operation is: " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named ’explanation’ in this block. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; is: the value named ’is’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 727: `f"{complete_explanation}"` This is continuation line 5 of the 6-line logical statement that begins at source line 723. Its exact fragment is ‘f"{complete_explanation}"‘. The important operands are f: the value named ’f’ in this block; complete_explanation: the value named ’complete explanation’ in this block. Read together, the complete operation is: This assigns ‘(f"This is continuation line {position} of the {length}-line logical statement " f"that begins at source line {span.start}. Its exact fragment is ‘{raw.strip()}‘. " f"{summarize_expression(raw)} Read together, the complete operation is: " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named ’explanation’ in this block. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; is: the value named ’is’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 728: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 723. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(f"This is continuation line {position} of the {length}-line logical statement " f"that begins at source line {span.start}. Its exact fragment is ‘{raw.strip()}‘. " f"{summarize_expression(raw)} Read together, the complete operation is: " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named ’explanation’ in this block. The important operands are f: the value named ’f’ in this block; This: the value

named 'this' in this block; is: the value named 'is' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 729: `elif span:` This executes the following indented host-code branch only when ‘span‘ is true. The important operands are span: the value named 'span' in this block. Context: the function annotation_tex, which handles annotation tex.

Line 730: `explanation = (` This statement continues through source line 734; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(f"This statement continues through source line {span.end}; the explanation uses " f"the complete expression rather than only the first visual fragment. " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named 'explanation' in this block. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; statement: the finite-state-machine phase retained in flip-flops. Context: the function annotation_tex, which handles annotation tex.

Line 731: `f"This statement continues through source line {span.end}; the explanation uses "` This is continuation line 2 of the 5-line logical statement that begins at source line 730. Its exact fragment is ‘f"This statement continues through source line {span.end}; the explanation uses "‘. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; statement: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns ‘(f"This statement continues through source line {span.end}; the explanation uses " f"the complete expression rather than only the first visual fragment. " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named 'explanation' in this block. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; statement: the finite-state-machine phase retained in flip-flops. Context: the function annotation_tex, which handles annotation tex.

Line 732: `f"the complete expression rather than only the first visual fragment. "` This is continuation line 3 of the 5-line logical statement that begins at source line 730. Its exact fragment is ‘f"the complete expression rather than only the first visual fragment. "‘. The important operands are f: the value named 'f' in this block; the: the value named 'the' in this block; complete: the value named 'complete' in this block. Read together, the complete operation is: This assigns ‘(f"This statement continues through source line {span.end}; the explanation uses " f"the complete expression rather than only the first visual fragment. " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named 'explanation' in this block. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; statement: the finite-state-machine phase retained in flip-flops. Context: the function annotation_tex, which handles annotation tex.

Line 733: `f"{complete_explanation}"` This is continuation line 4 of the 5-line logical statement that begins at source line 730. Its exact fragment is ‘f"{complete_explanation}"‘. The important operands are f: the value named 'f' in this block; complete_explanation: the value named 'complete explanation' in this block. Read together, the complete operation is: This assigns ‘(f"This statement continues through source line {span.end}; the explanation uses " f"the complete expression rather than only the first visual fragment. " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named 'explanation' in this block. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; statement: the finite-state-machine phase retained in flip-flops. Context: the function annotation_tex, which handles annotation tex.

Line 734: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 730. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(f"This statement continues through source line {span.end}; the explanation uses " f"the complete expression rather than only the first visual fragment. " f"{complete_explanation}")‘ to ‘explanation ‘ in host memory. Here explanation means the value named 'explanation' in this block. The important operands are f: the value named 'f' in this block; This: the value named 'this' in this block; statement: the finite-state-machine phase retained in flip-flops. Context: the function annotation_tex, which handles annotation tex.

Line 735: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function annotation_tex, which handles annotation tex.

Line 736: `explanation = complete_explanation` This assigns ‘complete_explanation‘ to ‘explanation ‘ in host memory. Here explanation means the value named ’explanation’ in this block. The important operands are complete_explanation: the value named ’complete explanation’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 737: `preview = raw.strip()` This assigns ‘raw.strip()‘ to ‘preview ‘ in host memory. Here preview means the value named ’preview’ in this block. The important operands are raw: the value named ’raw’ in this block; strip: the value named ’strip’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 738: `if len(preview) > 110:` This executes the following indented host-code branch only when ‘len(preview) > 110‘ is true. The important operands are len: the value named ’len’ in this block; preview: the value named ’preview’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 739: `preview = preview[:107] + "..."` This assigns ‘preview[:107] + "..."‘ to ‘preview ‘ in host memory. Here preview means the value named ’preview’ in this block. The important operands are preview: the value named ’preview’ in this block. Context: the function annotation_tex, which handles annotation tex.

Line 740: `result.extend(` This statement continues through source line 745; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘result.extend‘ with ‘ [rf"\paragraph{{Line {line_number}: \texttt{{{latex_escape(preview)}}}}}", latex_escape(explanation),] ‘ inside the function annotation_tex, which handles annotation tex. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 741: `[` This is continuation line 2 of the 6-line logical statement that begins at source line 740. Its exact fragment is ‘[‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘result.extend‘ with ‘ [rf"\paragraph{{Line {line_number}: \texttt{{{latex_escape(preview)}}}}}", latex_escape(explanation),] ‘ inside the function annotation_tex, which handles annotation tex. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 742: `rf"\paragraph{{Line {line_number}: \texttt{{{latex_escape(preview)}}}}}",` This is continuation line 3 of the 6-line logical statement that begins at source line 740. Its exact fragment is ‘rf"\paragraph{{Line {line_number}: \texttt{{{latex_escape(preview)}}}}}",‘. The important operands are rf: the value named ’rf’ in this block; paragraph: the value named ’paragraph’ in this block; Line: the value named ’line’ in this block. Read together, the complete operation is: This calls ‘result.extend‘ with ‘ [rf"\paragraph{{Line {line_number}: \texttt{{{latex_escape(preview)}}}}} latex_escape(explanation),] ‘ inside the function annotation_tex, which handles annotation tex. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 743: `latex_escape(explanation),` This is continuation line 4 of the 6-line logical statement that begins at source line 740. Its exact fragment is ‘latex_escape(explanation),‘. The important operands are latex_escape: the value named ’latex escape’ in this block; explanation: the value named ’explanation’ in this block. Read together, the complete operation is: This calls ‘result.extend‘ with ‘ [rf"\paragraph{{Line {line_number}: \texttt{{{latex_escape(preview)}}}}}", latex_escape(explanation),] ‘ inside the function annotation_tex, which handles annotation tex. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 744: `]` This is continuation line 5 of the 6-line logical statement that begins at source line 740. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘result.extend‘ with ‘ [rf"\paragraph{{Line {line_number}: \texttt{{{latex_escape(preview)}}}}}", latex_escape(explanation),] ‘ inside the function annotation_tex, which handles annotation tex. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 745: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 740. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘result.extend‘ with ‘ [rf"\paragraph{{Line {line_number}: \texttt{{{latex_escape(preview)}}}}}", latex_escape(explanation),] ‘ inside the function annotation_tex, which handles annotation tex. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 746: `explained += 1` Inside the function annotation_tex, which handles annotation tex, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are explained: the value named 'explained' in this block.

Line 747: `return result, explained` This returns ‘result, explained‘ to the caller of the function annotation_tex, which handles annotation tex, ending this invocation. The important operands are result: the value named 'result' in this block; explained: the value named 'explained' in this block.

Line 750: `def generate(output_dir: Path) -> tuple[Path, Path]:` This starts function generate. Its indented body performs generate as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 751: `root = project_root()` This assigns ‘project_root()‘ to ‘root ‘ in host memory. Here root means the value named 'root' in this block. The important operands are project_root: the value named 'project root' in this block. Context: the function generate, which handles generate.

Line 752: `output_dir = output_dir.resolve()` This assigns ‘output_dir.resolve()‘ to ‘output_dir ‘ in host memory. Here output_dir means the value named 'output dir' in this block. The important operands are output_dir: the value named 'output dir' in this block; resolve: the value named 'resolve' in this block. Context: the function generate, which handles generate.

Line 753: `output_dir.mkdir(parents=True, exist_ok=True)` This calls ‘output_dir.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 754: `entries = source_entries(root)` This assigns ‘source_entries(root)‘ to ‘entries ‘ in host memory. Here entries means the value named 'entries' in this block. The important operands are source_entries: the value named 'source entries' in this block; root: the value named 'root' in this block. Context: the function generate, which handles generate.

Line 755: `tex_path = output_dir / "neuraedge_line_by_line_source_reference.tex"` This assigns ‘output_dir / "neuraedge_line_by_line_source_reference.tex"‘ to ‘tex_path ‘ in host memory. Here tex_path means the value named 'tex path' in this block. The important operands are output_dir: the value named 'output dir' in this block; neuraedge_line_by_line_source_reference: the value named 'neuraedge line by line source reference' in this block; tex: the value named 'tex' in this block. Context: the function generate, which handles generate.

Line 756: `manifest_path = output_dir / "neuraedge_line_by_line_source_manifest.json"` This assigns ‘output_dir / "neuraedge_line_by_line_source_manifest.json"‘ to ‘manifest_path ‘ in host memory. Here manifest_path means the value named 'manifest path' in this block. The important operands are output_dir: the value named 'output dir' in this block; neuraedge_line_by_line_source_manifest: the value named 'neuraedge line by line source manifest' in this block; json: the value named 'json' in this block. Context: the function generate, which handles generate.

Line 757: `tex = [` This statement continues through source line 777; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}" r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{ r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=f r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named 'tex' in this block. The important operands are r: the value named 'r' in this block; documentclass:

the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function generate, which handles generate.

Line 758: `r"\documentclass[9pt]{article}",` This is continuation line 2 of the 21-line logical statement that begins at source line 757. Its exact fragment is 'r"\documentclass[9pt]{article}",'. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,showstringspaces=false,numbers=le r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]' to 'tex ' in host memory. Here tex means the value named 'tex' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function generate, which handles generate.

Line 759: `r"\usepackage[margin=0.62in]{geometry}",` This is continuation line 3 of the 21-line logical statement that begins at source line 757. Its exact fragment is 'r"\usepackage[margin=0.62in]{geometry}",'. The important operands are r: the value named 'r' in this block; usepackage: the value named 'usepackage' in this block; margin: the value named 'margin' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,showstringspaces=false,numbers=le r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]' to 'tex ' in host memory. Here tex means the value named 'tex' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function generate, which handles generate.

Line 760: `r"\usepackage[T1]{fontenc}",` This is continuation line 4 of the 21-line logical statement that begins at source line 757. Its exact fragment is 'r"\usepackage[T1]{fontenc}",'. The important operands are r: the value named 'r' in this block; usepackage: the value named 'usepackage' in this block; T1: the value named 't1' in this block. Read together, the complete operation is: This assigns '[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks] r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns= r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]' to 'tex ' in host memory. Here tex means the value named 'tex' in this block. The important operands are r: the value named 'r' in this block; documentclass: the value named 'documentclass' in this block; article: the value named 'article' in this block. Context: the function generate, which handles generate.

Line 761: `r"\usepackage{lmodern}",` This is continuation line 5 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\usepackage{lmodern}",‘. The important operands are r: the value named ’r’ in this block; usepackage: the value named ’usepackage’ in this block; lmodern: the value named ’lmodern’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{ r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=f r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 762: `r"\usepackage{xcolor}",` This is continuation line 6 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\usepackage{xcolor}",‘. The important operands are r: the value named ’r’ in this block; usepackage: the value named ’usepackage’ in this block; xcolor: the value named ’xcolor’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{ r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=f r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 763: `r"\usepackage{listings}",` This is continuation line 7 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\usepackage{listings}",‘. The important operands are r: the value named ’r’ in this block; usepackage: the value named ’usepackage’ in this block; listings: the value named ’listings’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{ r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=f r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 764: `r"\usepackage[hidelinks]{hyperref}",` This is continuation line 8 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\usepackage[hidelinks]{hyperref}",‘. The important operands are r: the value named ’r’ in this block; usepackage: the value named ’usepackage’ in this block; hidelinks: the value named ’hidelinks’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}",

r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,showstringspaces=false,numbers=le
r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 765: `r"\setlength{\parindent}{0pt}",` This is continuation line 9 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\setlength{\parindent}{0pt}",‘. The important operands are r: the value named ’r’ in this block; setlength: the value named ’setlength’ in this block; parindent: the value named ’parindent’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,showstringspaces=false,numbers=le
r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 766: `r"\setlength{\parskip}{3pt}",` This is continuation line 10 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\setlength{\parskip}{3pt}",‘. The important operands are r: the value named ’r’ in this block; setlength: the value named ’setlength’ in this block; parskip: the value named ’parskip’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{
r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=f
r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 767: `r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,show`
This is continuation line 11 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,showstringspaces=false,numbers=l
The important operands are r: the value named ’r’ in this block; lstset: the value named ’lstset’ in this block; basicstyle: the value named ’basicstyle’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}"
r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,show
r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to

read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 768: `r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}",` This is continuation line 12 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}",‘. The important operands are r: the value named ’r’ in this block; title: the value named ’title’ in this block; NeuraEdge: the value named ’neura edge’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{
r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=
r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 769: `r"\author{Generated from hashed repository source}",` This is continuation line 13 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\author{Generated from hashed repository source}",‘. The important operands are r: the value named ’r’ in this block; author: the value named ’author’ in this block; Generated: the value named ’generated’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\par
r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,show
r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 770: `r"\date{\today}",` This is continuation line 14 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\date{\today}",‘. The important operands are r: the value named ’r’ in this block; date: the value named ’date’ in this block; today: the value named ’today’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\par
r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,show
r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge.

Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 771: `r"\begin{document}",` This is continuation line 15 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\begin{document}",‘. The important operands are r: the value named ’r’ in this block; document: the value named ’document’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}" r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,show r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 772: `r"\maketitle",` This is continuation line 16 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\maketitle",‘. The important operands are r: the value named ’r’ in this block; maketitle: the value named ’maketitle’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,showstringspaces=false,numbers=le r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 773: `r"\tableofcontents",` This is continuation line 17 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\tableofcontents",‘. The important operands are r: the value named ’r’ in this block; tableofcontents: the value named ’tableofcontents’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\par r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,show r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 774: `r"\clearpage",` This is continuation line 18 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\clearpage",‘. The important operands are r: the value named ’r’ in this block; clearpage: the value named ’clearpage’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{hyperref}", r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,showstringspaces=false,numbers=le r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 775: `r"\section{How to read this volume}",` This is continuation line 19 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘r"\section{How to read this volume}",‘. The important operands are r: the value named ’r’ in this block; section: the value named ’section’ in this block; How: the value named ’how’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}" r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{ r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=f r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 776: `"Each file begins with its system-level responsibility and a complete numbered source listing. The followin...` This is continuation line 20 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘"Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",‘. The important operands are Each: the value named ’each’ in this block; file: the value named ’file’ in this block; begins: the value named ’begins’ in this block. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{ r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=f r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 777: `]` This is continuation line 21 of the 21-line logical statement that begins at source line 757. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘[r"\documentclass[9pt]{article}", r"\usepackage[margin=0.62in]{geometry}", r"\usepackage[T1]{fontenc}", r"\usepackage{lmodern}", r"\usepackage{xcolor}", r"\usepackage{listings}", r"\usepackage[hidelinks]{ r"\setlength{\parindent}{0pt}", r"\setlength{\parskip}{3pt}", r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=f r"\title{NeuraEdge Universal: Complete Line-by-Line Source Reference}", r"\author{Generated from hashed repository source}", r"\date{\today}", r"\begin{document}", r"\maketitle", r"\tableofcontents", r"\clearpage", r"\section{How to read this volume}", "Each file begins with its system-level responsibility and a complete numbered source listing. The following annotations explain every nonblank line using its enclosing function, RTL process, or finite-state-machine phase. Blank lines are visible in the listing but have no executable or synthesized behavior. SystemVerilog describes parallel hardware: an if statement controls a multiplexer and a nonblocking assignment updates a register on a clock edge. Python, PowerShell, and Tcl run sequentially on a processor or inside a tool.",]‘ to ‘tex ‘ in host memory. Here tex means the value named ’tex’ in this block. The important operands are r: the value named ’r’ in this block; documentclass: the value named ’documentclass’ in this block; article: the value named ’article’ in this block. Context: the function generate, which handles generate.

Line 778: `manifest: dict[str, object] = {"format": "neuraedge-source-reference-v1", "files": []}` This assigns ‘{"format": "neuraedge-source-reference-v1", "files": []}‘ to ‘manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; source: the value named ’source’ in this block. Context: the function generate, which handles generate.

Line 779: `total_lines = 0` This assigns ‘0‘ to ‘total_lines ‘ in host memory. Here total_lines means the value named ’total lines’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 780: `total_nonblank = 0` This assigns ‘0‘ to ‘total_nonblank ‘ in host memory. Here total_nonblank means the value named ’total nonblank’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function generate, which handles generate.

Line 781: `for entry in entries:` This iterates entry over ‘entries‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function generate, which handles generate.

Line 782: `raw_bytes = entry.path.read_bytes()` This assigns ‘entry.path.read_bytes()‘ to ‘raw_bytes ‘ in host memory. Here raw_bytes means the value named ’raw bytes’ in this block. The important operands are entry: the value named ’entry’ in this block; path: the value named ’path’ in this block; read_bytes: the value named ’read bytes’ in this block. Context: the function generate, which handles generate.

Line 783: `text = raw_bytes.decode("utf-8", errors="strict")` This assigns ‘raw_bytes.decode("utf-8", errors="strict")‘ to ‘text ‘ in host memory. Here text means the value named ’text’ in this block. The important operands are raw_bytes: the value named ’raw bytes’ in this block; decode: the value named ’decode’ in this block; utf: the value named ’utf’ in this block. Context: the function generate, which handles generate.

Line 784: `line_count = len(text.splitlines())` This assigns ‘len(text.splitlines())‘ to ‘line_count ‘ in host memory. Here line_count means an integer count retained for control or measurable telemetry. The important operands are len: the value named ’len’ in this block; text: the value named ’text’ in this block; splitlines: the value named ’splitlines’ in this block. Context: the function generate, which handles generate.

Line 785: `nonblank_count = sum(bool(line.strip()) for line in text.splitlines())` This assigns ‘sum(bool(line.strip(for line in text.splitlines()))‘ to ‘nonblank_count ‘ in host memory. Here nonblank_count means an integer count retained for control or measurable telemetry. The important operands are sum: the value named ’sum’ in this block; bool: the value named ’bool’ in this block; line: the value named ’line’ in this block. Context: the function generate, which handles generate.

Line 786: `annotations, explained = annotation_tex(entry, text)` This assigns ‘annotation_tex(entry, text)‘ to ‘annotations, explained ‘ in host memory. Here annotations means the value named ’annotations’ in this block. The important operands are annotation_tex: the value named ’annotation tex’ in this block; entry: the value named ’entry’ in this block; text: the value named ’text’ in this block. Context: the function generate, which handles generate.

Line 787: `if explained != nonblank_count:` This executes the following indented host-code branch only when ‘explained != nonblank_count‘ is true. The important operands are explained: the value named ’explained’ in this block; nonblank_count: an integer count retained for control or measurable telemetry. Context: the function generate, which handles generate.

Line 788: `raise RuntimeError(f"annotation coverage mismatch for {entry.relative}: {explained} != {nonblank_c` This rejects an invalid state by raising ‘RuntimeError(f"annotation coverage mismatch for {entry.relative}: {explained} != {nonblank_count}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 789: `total_lines += line_count` Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are total_lines: the value named ’total lines’ in this block; line_count: an integer count retained for control or measurable telemetry.

Line 790: `total_nonblank += nonblank_count` Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are total_nonblank: the value named ’total nonblank’ in this block; nonblank_count: an integer count retained for control or measurable telemetry.

Line 791: `listing_path = os.path.relpath(entry.path, output_dir).replace("\\", "/")` This assigns ‘os.path.relpath(output_dir).replace("\\", "/")‘ to ‘listing_path ‘ in host memory. Here listing_path means the value named ’listing path’ in this block. The important operands are os: the value named ’os’ in this block; path: the value named ’path’ in this block; relpath: the value named ’relpath’ in this block. Context: the function generate, which handles generate.

Line 792: `manifest["files"].append(` This statement continues through source line 801; the explanation uses the complete expression rather than only the first visual fragment. Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named ’manifest’ in this block; files: the value named ’files’ in this block; append: the value named ’append’ in this block.

Line 793: `{` This is continuation line 2 of the 10-line logical statement that begins at source line 792. Its exact fragment is ‘{‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named ’manifest’ in this block; files: the value named ’files’ in this block; append: the value named ’append’ in this block.

Line 794: `"path": entry.relative,` This is continuation line 3 of the 10-line logical statement that begins at source line 792. Its exact fragment is ‘"path": entry.relative,‘. The important operands are path: the value named ’path’ in this block; entry: the value named ’entry’ in this block; relative: the value named ’relative’ in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named ’manifest’ in this block; files: the value named ’files’ in this block; append: the value named ’append’ in this block.

Line 795: `"sha256": hashlib.sha256(raw_bytes).hexdigest(),` This is continuation line 4 of the 10-line logical statement that begins at source line 792. Its exact fragment is ‘"sha256": hashlib.sha256(raw_bytes).hexdigest(),‘. The important operands are sha256: the value named ’sha256’ in this block; hashlib: the value named ’hashlib’ in this block; raw_bytes: the value named ’raw bytes’ in this block. Read together, the complete operation is: Inside the function

generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; files: the value named 'files' in this block; append: the value named 'append' in this block.

Line 796: `"lines": line_count,` This is continuation line 5 of the 10-line logical statement that begins at source line 792. Its exact fragment is ‘"lines": line_count,‘. The important operands are lines: the value named 'lines' in this block; line_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; files: the value named 'files' in this block; append: the value named 'append' in this block.

Line 797: `"nonblank_lines": nonblank_count,` This is continuation line 6 of the 10-line logical statement that begins at source line 792. Its exact fragment is ‘"nonblank_lines": nonblank_count,‘. The important operands are nonblank_lines: the value named 'nonblank lines' in this block; nonblank_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; files: the value named 'files' in this block; append: the value named 'append' in this block.

Line 798: `"explained_nonblank_lines": explained,` This is continuation line 7 of the 10-line logical statement that begins at source line 792. Its exact fragment is ‘"explained_nonblank_lines": explained,‘. The important operands are explained_nonblank_lines: the value named 'explained nonblank lines' in this block; explained: the value named 'explained' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; files: the value named 'files' in this block; append: the value named 'append' in this block.

Line 799: `"purpose": entry.purpose,` This is continuation line 8 of the 10-line logical statement that begins at source line 792. Its exact fragment is ‘"purpose": entry.purpose,‘. The important operands are purpose: the value named 'purpose' in this block; entry: the value named 'entry' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; files: the value named 'files' in this block; append: the value named 'append' in this block.

Line 800: `}` This is continuation line 9 of the 10-line logical statement that begins at source line 792. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; files: the value named 'files' in this block; append: the value named 'append' in this block.

Line 801: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 792. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; files: the value named 'files' in this block; append: the value named 'append' in this block.

Line 802: `tex.extend(` This statement continues through source line 819; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.rela rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 803: `[` This is continuation line 2 of the 18-line logical statement that begins at source line 802. Its exact fragment is '['. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'tex.extend' with ' [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ' inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 804: `r"\clearpage",` This is continuation line 3 of the 18-line logical statement that begins at source line 802. Its exact fragment is 'r"\clearpage",'. The important operands are r: the value named 'r' in this block; clearpage: the value named 'clearpage' in this block. Read together, the complete operation is: This calls 'tex.extend' with ' [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ' inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 805: `rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}",` This is continuation line 4 of the 18-line logical statement that begins at source line 802. Its exact fragment is 'rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}",'. The important operands are rf: the value named 'rf' in this block; section: the value named 'section' in this block; texttt: the value named 'texttt' in this block. Read together, the complete operation is: This calls 'tex.extend' with ' [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ' inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 806: `rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.",` This is continuation line 5 of the 18-line logical statement that begins at source line 802. Its exact fragment is 'rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.",'. The important operands are rf: the value named 'rf' in this block; textbf: the value named 'textbf' in this block; Role: the value named 'role' in this block. Read together, the complete operation is: This calls 'tex.extend' with ' [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ' inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 807: `r"\subsection{Beginner architecture map}",` This is continuation line 6 of the 18-line logical statement that begins at source line 802. Its exact fragment is 'r"\subsection{Beginner architecture map}",'. The important operands are r: the value named 'r' in this block; subsection: the value named 'subsection' in this block; Beginner: the value named 'beginner' in this block. Read together, the complete operation is: This calls 'tex.extend' with ' [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ' inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 808: `r"\begin{description}",` This is continuation line 7 of the 18-line logical statement that begins at source line 802. Its exact fragment is 'r"\begin{description}",'. The important operands are r: the value named 'r' in this block; description: the value named 'description' in this block. Read together, the complete operation is: This calls 'tex.extend' with ' [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}",

*[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 809: `*[` This is continuation line 8 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘*[‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 810: `rf"\item[{latex_escape(label)}] {latex_escape(note)}"` This is continuation line 9 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘rf"\item[{latex_escape(label)}] {latex_escape(note)}"‘. The important operands are rf: the value named ’rf’ in this block; item: the value named ’item’ in this block; latex_escape: the value named ’latex escape’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 811: `for label, note in source_architecture_notes(entry, text)` This is continuation line 10 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘for label, note in source_architecture_notes(entry, text)‘. The important operands are for: the value named ’for’ in this block; label: the value named ’label’ in this block; note: the value named ’note’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 812: `],` This is continuation line 11 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘],‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 813: `r"\end{description}",` This is continuation line 12 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘r"\end{description}",‘. The important operands are r: the value named ’r’ in this block; description: the value named ’description’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 814: `r"\subsection{Complete source}",` This is continuation line 13 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘r"\subsection{Complete source}",‘. The important operands are r: the value named ’r’ in this block; subsection: the value named ’subsection’ in this block; Complete: the value named ’complete’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)
r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}"
for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting
r"\subsection{Line annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 815: `rf"\lstinputlisting{{{listing_path}}}",` This is continuation line 14 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘rf"\lstinputlisting{{{listing_path}}}",‘. The important operands are rf: the value named ’rf’ in this block; lstinputlisting: the value named ’lstinputlisting’ in this block; listing_path: the value named ’listing path’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}", rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}", r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry, text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 816: `r"\subsection{Line annotations}",` This is continuation line 15 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘r"\subsection{Line annotations}",‘. The important operands are r: the value named ’r’ in this block; subsection: the value named ’subsection’ in this block; Line: the value named ’line’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.rela
rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}",
r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry,
text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line
annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job
that generates this source-reference volume and its source-integrity manifest.

Line 817: `*annotations,` This is continuation line 16 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘*annotations,‘. The important operands are annotations: the value named ’annotations’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.rela
rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}",
r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry,
text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line
annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job
that generates this source-reference volume and its source-integrity manifest.

Line 818: `]` This is continuation line 17 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}",
rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}",
r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry,
text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line
annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job
that generates this source-reference volume and its source-integrity manifest.

Line 819: `)` This is continuation line 18 of the 18-line logical statement that begins at source line 802. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", rf"\section{{\texttt{{{latex_escape(entry.relative)}}}}}",
rf"\textbf{{Role in the complete system:}} {latex_escape(entry.purpose)}.", r"\subsection{Beginner architecture map}",
r"\begin{description}", *[rf"\item[{latex_escape(label)}] {latex_escape(note)}" for label, note in source_architecture_notes(entry,
text)], r"\end{description}", r"\subsection{Complete source}", rf"\lstinputlisting{{{listing_path}}}", r"\subsection{Line
annotations}", *annotations,] ‘ inside the function generate, which handles generate. The call advances the host-side job
that generates this source-reference volume and its source-integrity manifest.

Line 820: `manifest["summary"] = {` This statement continues through source line 825; the explanation uses the complete expression rather than only the first visual fragment. Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; summary: the value named 'summary' in this block; file_count: an integer count retained for control or measurable telemetry.

Line 821: `"file_count": len(entries),` This is continuation line 2 of the 6-line logical statement that begins at source line 820. Its exact fragment is ‘"file_count": len(entries),‘. The important operands are file_count: an integer count retained for control or measurable telemetry; len: the value named 'len' in this block; entries: the value named 'entries' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; summary: the value named 'summary' in this block; file_count: an integer count retained for control or measurable telemetry.

Line 822: `"source_lines": total_lines,` This is continuation line 3 of the 6-line logical statement that begins at source line 820. Its exact fragment is ‘"source_lines": total_lines,‘. The important operands are source_lines: the value named 'source lines' in this block; total_lines: the value named 'total lines' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; summary: the value named 'summary' in this block; file_count: an integer count retained for control or measurable telemetry.

Line 823: `"nonblank_lines": total_nonblank,` This is continuation line 4 of the 6-line logical statement that begins at source line 820. Its exact fragment is ‘"nonblank_lines": total_nonblank,‘. The important operands are nonblank_lines: the value named 'nonblank lines' in this block; total_nonblank: the value named 'total nonblank' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; summary: the value named 'summary' in this block; file_count: an integer count retained for control or measurable telemetry.

Line 824: `"explained_nonblank_lines": total_nonblank,` This is continuation line 5 of the 6-line logical statement that begins at source line 820. Its exact fragment is ‘"explained_nonblank_lines": total_nonblank,‘. The important operands are explained_nonblank_lines: the value named 'explained nonblank lines' in this block; total_nonblank: the value named 'total nonblank' in this block. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; summary: the value named 'summary' in this block; file_count: an integer count retained for control or measurable telemetry.

Line 825: `}` This is continuation line 6 of the 6-line logical statement that begins at source line 820. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function generate, which handles generate, this statement advances the host implementation that generates this source-reference volume and its source-integrity manifest. The important operands are manifest: the value named 'manifest' in this block; summary: the value named 'summary' in this block; file_count: an integer count retained for control or measurable telemetry.

Line 826: `tex.extend(` This statement continues through source line 833; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘tex.extend‘ with ‘ [r"\clearpage", r"\section{Coverage statement}", f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank lines. The companion JSON manifest records each source SHA-256 digest.", r"\end{document}",] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 827: `[` This is continuation line 2 of the 8-line logical statement that begins at source line 826. Its exact fragment is ‘[‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", r"\section{Coverage statement}", f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank

lines. The companion JSON manifest records each source SHA-256 digest.", r"\end{document}",] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 828: `r"\clearpage",` This is continuation line 3 of the 8-line logical statement that begins at source line 826. Its exact fragment is ‘r"\clearpage",‘. The important operands are r: the value named ’r’ in this block; clearpage: the value named ’clearpage’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", r"\section{Coverage statement}", f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank lines. The companion JSON manifest records each source SHA-256 digest.", r"\end{document}",] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 829: `r"\section{Coverage statement}",` This is continuation line 4 of the 8-line logical statement that begins at source line 826. Its exact fragment is ‘r"\section{Coverage statement}",‘. The important operands are r: the value named ’r’ in this block; section: the value named ’section’ in this block; Coverage: the value named ’coverage’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", r"\section{Coverage statement}", f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank lines. The companion JSON manifest records each source SHA-256 digest.", r"\end{document}",] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 830: `f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanati...` This is continuation line 5 of the 8-line logical statement that begins at source line 826. Its exact fragment is ‘f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank lines. The companion JSON manifest records each source SHA-256 digest.",‘. The important operands are f: the value named ’f’ in this block; This: the value named ’this’ in this block; volume: the value named ’volume’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", r"\section{Coverage statement}", f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank lines. The companion JSON manifest records each source SHA-256 digest.", r"\end{document}",] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 831: `r"\end{document}",` This is continuation line 6 of the 8-line logical statement that begins at source line 826. Its exact fragment is ‘r"\end{document}",‘. The important operands are r: the value named ’r’ in this block; document: the value named ’document’ in this block. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", r"\section{Coverage statement}", f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank lines. The companion JSON manifest records each source SHA-256 digest.", r"\end{document}",] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 832: `]` This is continuation line 7 of the 8-line logical statement that begins at source line 826. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", r"\section{Coverage statement}", f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank lines. The companion JSON manifest records each source SHA-256 digest.", r"\end{document}",] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 833: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 826. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘tex.extend‘ with ‘ [r"\clearpage", r"\section{Coverage statement}", f"This volume contains {len(entries)} source files, {total_lines:,} physical source lines, and an explanation for all {total_nonblank:,} nonblank lines. The companion JSON manifest records each source SHA-256 digest.", r"\end{document}",] ‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 834: `tex_path.write_text("\n".join(tex) + "\n", encoding="utf-8")` This calls ‘tex_path.write_text‘ with ‘"\n".join(tex) + "\n", encoding="utf-8"‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 835: `manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")` This calls ‘manifest_path.write_text‘ with ‘json.dumps(manifest, indent=2) + "\n", encoding="utf-8"‘ inside the function generate, which handles generate. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 836: `return tex_path, manifest_path` This returns ‘tex_path, manifest_path‘ to the caller of the function generate, which handles generate, ending this invocation. The important operands are tex_path: the value named ’tex path’ in this block; manifest_path: the value named ’manifest path’ in this block.

Line 839: `def compile_pdf(tex_path: Path, pdflatex: str) -> Path:` This starts function compile_pdf. Its indented body performs compile pdf as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 840: `for _ in range(2):` This iterates _ over ‘range(2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_pdf, which handles compile pdf.

Line 841: `subprocess.run(` This statement continues through source line 845; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘subprocess.run‘ with ‘ [pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name], cwd=tex_path.parent, check=True, ‘ inside the function compile_pdf, which handles compile pdf. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 842: `[pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name],` This is continuation line 2 of the 5-line logical statement that begins at source line 841. Its exact fragment is ‘[pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name],‘. The important operands are pdflatex: the value named ’pdflatex’ in this block; interaction: the value named ’interaction’ in this block; nonstopmode: the value named ’nonstopmode’ in this block. Read together, the complete operation is: This calls ‘subprocess.run‘ with ‘ [pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name], cwd=tex_path.parent, check=True, ‘ inside the function compile_pdf, which handles compile pdf. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 843: `cwd=tex_path.parent,` This is continuation line 3 of the 5-line logical statement that begins at source line 841. Its exact fragment is ‘cwd=tex_path.parent,‘. The important operands are cwd: the value named ’cwd’ in this block; tex_path: the value named ’tex path’ in this block; parent: the value named ’parent’ in this block. Read together, the complete operation is: This calls ‘subprocess.run‘ with ‘ [pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name], cwd=tex_path.parent, check=True, ‘ inside the function compile_pdf, which handles compile pdf. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 844: `check=True,` This is continuation line 4 of the 5-line logical statement that begins at source line 841. Its exact fragment is ‘check=True,‘. The important operands are check: the value named ’check’ in this block. Read together, the complete operation is: This calls ‘subprocess.run‘ with ‘ [pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name], cwd=tex_path.parent, check=True, ‘ inside the function compile_pdf, which handles compile pdf. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 845: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 841. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘subprocess.run‘ with ‘ [pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name], cwd=tex_path.parent, check=True, ‘ inside the function compile_pdf, which handles compile pdf. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 846: `return tex_path.with_suffix(".pdf")` This returns ‘tex_path.with_suffix(".pdf")‘ to the caller of the function compile_pdf, which handles compile pdf, ending this invocation. The important operands are tex_path: the value named ’tex path’ in this block; with_suffix: the value named ’with suffix’ in this block; pdf: the value named ’pdf’ in this block.

Line 849: `def main() -> None:` This starts function main. Its indented body performs main as part of the file that generates this source-reference volume and its source-integrity manifest; it runs only when called.

Line 850: `parser = argparse.ArgumentParser(description=__doc__)` This assigns ‘argparse.ArgumentParser(description=_ to ‘parser ‘ in host memory. Here parser means the value named ’parser’ in this block. The important operands are argparse: the value named ’argparse’ in this block; ArgumentParser: the value named ’argument parser’ in this block; description: the value named ’description’ in this block. Context: the function main, which handles main.

Line 851: `parser.add_argument("-output-dir", type=Path, default=project_root() / "docs")` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 852: `parser.add_argument("-compile", action="store_true")` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 853: `parser.add_argument("-pdflatex", default="pdflatex")` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 854: `args = parser.parse_args()` This assigns ‘parser.parse_args()‘ to ‘args ‘ in host memory. Here args means the value named ’args’ in this block. The important operands are parser: the value named ’parser’ in this block; parse_args: the value named ’parse args’ in this block. Context: the function main, which handles main.

Line 855: `tex_path, manifest_path = generate(args.output_dir.resolve())` This assigns ‘generate(args.output_dir.resol to ‘tex_path, manifest_path ‘ in host memory. Here tex_path means the value named ’tex path’ in this block. The important operands are generate: the value named ’generate’ in this block; args: the value named ’args’ in this block; output_dir: the value named ’output dir’ in this block. Context: the function main, which handles main.

Line 856: `print(f"tex={tex_path}")` This calls ‘print‘ with ‘f"tex={tex_path}"‘ inside the function main, which handles main. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 857: `print(f"manifest={manifest_path}")` This calls ‘print‘ with ‘f"manifest={manifest_path}"‘ inside the function main, which handles main. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 858: `if args.compile:` This executes the following indented host-code branch only when ‘args.compile‘ is true. The important operands are args: the value named ’args’ in this block; compile: the value named ’compile’ in this block. Context: the function main, which handles main.

Line 859: `print(f"pdf={compile_pdf(tex_path, args.pdflatex)}")` This calls ‘print‘ with ‘f"pdf={compile_pdf(tex_path, args.pdflatex)}"‘ inside the function main, which handles main. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

Line 862: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to generates this source-reference volume and its source-integrity manifest.

Line 863: `main()` This calls ‘main‘ with “ inside the file whose role is to generates this source-reference volume and its source-integrity manifest. The call advances the host-side job that generates this source-reference volume and its source-integrity manifest.

31 software/generate_spikingformer_manifest.py

Role in the complete system: derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

31.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: MatrixSchedule, OperatorMapping. Functions: matrix_schedule, build_operator_manifest, generate, parse_arguments, matrix_operator. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

31.2 Complete source

```
"""Generate the exact Spikingformer-to-NeuraEdge execution manifest."""

from __future__ import annotations

import argparse
import json
from dataclasses import asdict, dataclass
from math import ceil
from pathlib import Path
from typing import Literal

import torch


ROW_TILE = 8
COLUMN_TILE = 16
MAX_REDUCTION = 2048


@dataclass(frozen=True)
class MatrixSchedule:
    rows: int
    reduction: int
    columns: int
    repetitions: int
    arithmetic: Literal["u8_mac", "binary_spike_add", "dynamic_u8_mac"]
    mac_equivalents: int
    reduction_chunks: int
    compute_tiles: int
    commands: int
    input_stream_bytes: int
    output_stream_bytes: int


@dataclass(frozen=True)
class OperatorMapping:
    name: str
    source_operation: str
    input_shape_per_image: tuple[int, ...]
    output_shape_per_image: tuple[int, ...]
    hardware_primitive: str
    matrix: MatrixSchedule | None
    exact_optimization: str | None = None


def matrix_schedule(
    rows: int,
    reduction: int,
    columns: int,
    *,
    repetitions: int = 1,
    arithmetic: Literal["u8_mac", "binary_spike_add", "dynamic_u8_mac"],
```

```
) -> MatrixSchedule:
    """Count commands and stream bytes using the compiler's public tiling ABI."""

    if min(rows, reduction, columns, repetitions) <= 0:
        raise ValueError("all matrix dimensions and repetitions must be positive")
    total_commands = 0
    total_compute_tiles = 0
    total_input_bytes = 0
    total_output_bytes = 0
    chunks = 0
    for reduction_start in range(0, reduction, MAX_REDUCTION):
        chunk = min(MAX_REDUCTION, reduction - reduction_start)
        chunks += 1
        row_tiles = ceil(rows / ROW_TILE)
        column_tiles = ceil(columns / COLUMN_TILE)
        compute_tiles = row_tiles * column_tiles
        if rows == 1:
            issues = ceil(chunk / 8)
            activation_tile_bytes = issues * 8
            weight_tile_bytes = issues * 8 * 16
        else:
            activation_tile_bytes = chunk * ROW_TILE
            weight_tile_bytes = chunk * COLUMN_TILE
        input_broadcast_bytes = (
            row_tiles * activation_tile_bytes
            + compute_tiles * weight_tile_bytes
        )
        weight_broadcast_bytes = (
            column_tiles * weight_tile_bytes
            + compute_tiles * activation_tile_bytes
        )
        if input_broadcast_bytes <= weight_broadcast_bytes:
            load_commands = row_tiles + compute_tiles
            stream_bytes = input_broadcast_bytes
        else:
            load_commands = column_tiles + compute_tiles
            stream_bytes = weight_broadcast_bytes
        total_commands += load_commands + compute_tiles
        total_compute_tiles += compute_tiles
        total_input_bytes += stream_bytes
        # The current output ABI emits one 32-bit result in each 64-bit beat.
        total_output_bytes += rows * columns * 8
    return MatrixSchedule(
        rows=rows,
        reduction=reduction,
        columns=columns,
        repetitions=repetitions,
        arithmetic=arithmetic,
        mac_equivalents=rows * reduction * columns * repetitions,
        reduction_chunks=chunks * repetitions,
        compute_tiles=total_compute_tiles * repetitions,
        commands=total_commands * repetitions,
        input_stream_bytes=total_input_bytes * repetitions,
        output_stream_bytes=total_output_bytes * repetitions,
    )


def build_operator_manifest() -> list[OperatorMapping]:
    """Describe every learned and data-dependent operator for one image."""

    time_steps = 4
    channels = 384
    tokens = 64
    heads = 12
    head_dim = channels // heads
    operators: list[OperatorMapping] = []

    def matrix_operator(
        name: str,
        source: str,
        input_shape: tuple[int, ...],
        output_shape: tuple[int, ...],
        rows: int,
        reduction: int,
        columns: int,
        arithmetic: Literal["u8_mac", "binary_spike_add", "dynamic_u8_mac"],
        primitive: str,
        *,
        repetitions: int = 1,
        optimization: str | None = None,
    ) -> None:
        operators.append(OperatorMapping(
            name=name,
            source_operation=source,
            input_shape_per_image=input_shape,
            output_shape_per_image=output_shape,
            hardware_primitive=primitive,
```

```
            matrix=matrix_schedule(
                rows, reduction, columns,
                repetitions=repetitions,
                arithmetic=arithmetic,
            ),
            exact_optimization=optimization,
        ))

    matrix_operator(
        "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm",
        (3, 32, 32), (time_steps, 48, 32, 32),
        32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile",
        optimization=(
            "Compute once and broadcast to four timesteps. The source repeats "
            "one unchanged image before this stateless operator, so this removes "
            "three of four executions with exact output equivalence."),
    )
    matrix_operator(
        "patch_embed.block1", "LIF + Conv2d 3x3 + folded BatchNorm",
        (time_steps, 48, 32, 32), (time_steps, 96, 32, 32),
        time_steps * 32 * 32, 48 * 3 * 3, 96,
        "binary_spike_add", "active-index sparse tensor tile")
    matrix_operator(
        "patch_embed.block2", "LIF + Conv2d 3x3 + folded BatchNorm",
        (time_steps, 96, 32, 32), (time_steps, 192, 32, 32),
        time_steps * 32 * 32, 96 * 3 * 3, 192,
        "binary_spike_add", "active-index sparse tensor tile")
    matrix_operator(
        "patch_embed.block3", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm",
        (time_steps, 192, 32, 32), (time_steps, 384, 16, 16),
        time_steps * 16 * 16, 192 * 3 * 3, 384,
        "binary_spike_add", "vector maximum + active-index sparse tensor tile")
    matrix_operator(
        "patch_embed.block4", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm",
        (time_steps, 384, 16, 16), (time_steps, 384, 8, 8),
        time_steps * 8 * 8, 384 * 3 * 3, 384,
        "binary_spike_add", "vector maximum + active-index sparse tensor tile")

    for block in range(4):
        prefix = f"block.{block}"
        for projection in ("q", "k", "v"):
            matrix_operator(
                f"{prefix}.attn.{projection}",
                f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm",
                (time_steps, channels, 8, 8),
                (time_steps, channels, tokens),
                time_steps * tokens, channels, channels,
                "binary_spike_add", "active-index sparse tensor tile")
        matrix_operator(
            f"{prefix}.attn.qk", "binary query times transposed binary key",
            (time_steps, heads, tokens, head_dim),
            (time_steps, heads, tokens, tokens),
            tokens, head_dim, tokens,
            "dynamic_u8_mac", "dense tensor tile",
            repetitions=time_steps * heads)
        matrix_operator(
            f"{prefix}.attn.av", "attention counts times binary value",
            (time_steps, heads, tokens, tokens),
            (time_steps, heads, tokens, head_dim),
            tokens, tokens, head_dim,
            "dynamic_u8_mac", "dense tensor tile",
            repetitions=time_steps * heads)
        matrix_operator(
            f"{prefix}.attn.proj", "LIF + projection Conv1d + folded BatchNorm",
            (time_steps, channels, tokens),
            (time_steps, channels, 8, 8),
            time_steps * tokens, channels, channels,
            "binary_spike_add", "active-index sparse tensor tile")
        matrix_operator(
            f"{prefix}.mlp.fc1", "LIF + 1x1 Conv2d + folded BatchNorm",
            (time_steps, channels, 8, 8),
            (time_steps, channels * 4, 8, 8),
            time_steps * tokens, channels, channels * 4,
            "binary_spike_add", "active-index sparse tensor tile")
        matrix_operator(
            f"{prefix}.mlp.fc2", "LIF + 1x1 Conv2d + folded BatchNorm",
            (time_steps, channels * 4, 8, 8),
            (time_steps, channels, 8, 8),
            time_steps * tokens, channels * 4, channels,
            "binary_spike_add", "active-index sparse tensor tile")
        operators.append(OperatorMapping(
            name=f"{prefix}.residuals",
            source_operation="two elementwise residual additions",
            input_shape_per_image=(time_steps, channels, 8, 8),
            output_shape_per_image=(time_steps, channels, 8, 8),
            hardware_primitive="saturating vector add",
            matrix=None,
```

```
        ))

    operators.append(OperatorMapping(
        name="global_average_pool",
        source_operation="mean over 64 tokens then mean over four timesteps",
        input_shape_per_image=(time_steps, channels, 8, 8),
        output_shape_per_image=(channels,),
        hardware_primitive="vector add followed by exact power-of-two shifts",
        matrix=None,
    ))
    matrix_operator(
        "head", "Linear classifier",
        (channels,), (10,), 1, channels, 10,
        "u8_mac", "dense Dot8 tensor tile")
    return operators


def generate(checkpoint_path: Path, output_path: Path) -> dict[str, object]:
    checkpoint = torch.load(checkpoint_path, map_location="cpu")
    state = checkpoint["state_dict"]
    parameter_tensors = {
        name: {
            "shape": list(value.shape),
            "elements": value.numel(),
            "source_dtype": str(value.dtype).replace("torch.", ""),
        }
        for name, value in state.items()
        if name.endswith(("weight", "bias"))
    }
    operators = build_operator_manifest()
    matrix_operators = [item.matrix for item in operators if item.matrix is not None]
    total_macs = sum(item.mac_equivalents for item in matrix_operators)
    total_commands = sum(item.commands for item in matrix_operators)
    total_input_bytes = sum(item.input_stream_bytes for item in matrix_operators)
    total_output_bytes = sum(item.output_stream_bytes for item in matrix_operators)
    trainable_parameters = sum(
        value.numel() for name, value in state.items()
        if name.endswith(("weight", "bias"))
    )
    int8_weight_elements = sum(
        value.numel() for name, value in state.items()
        if name.endswith("weight")
        and not ("norm1" in name or "norm2" in name or "_bn." in name)
    )
    folded_bias_channels = sum(
        value.shape[0] for name, value in state.items()
        if name.endswith("running_mean")
    ) + state["head.bias"].numel()
    report: dict[str, object] = {
        "format": "neuraedge-spikingformer-execution-manifest-v1",
        "checkpoint": str(checkpoint_path.resolve()),
        "checkpoint_epoch": int(checkpoint.get("epoch", -1)),
        "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))),
        "model": {
            "time_steps": 4,
            "embed_dim": 384,
            "heads": 12,
            "head_dim": 32,
            "transformer_blocks": 4,
            "tokens": 64,
            "classes": 10,
            "trainable_parameter_elements": trainable_parameters,
            "source_parameter_bytes_fp32": trainable_parameters * 4,
            "mapped_int8_weight_bytes": int8_weight_elements,
            "mapped_int32_folded_bias_bytes": folded_bias_channels * 4,
        },
        "accelerator": {
            "row_tile": ROW_TILE,
            "column_tile": COLUMN_TILE,
            "maximum_reduction_per_command": MAX_REDUCTION,
            "packed_u8_macs_per_cycle": 128,
            "target_clock_hz": 200_000_000,
            "peak_dense_gmac_per_second": 25.6,
        },
        "per_image_totals": {
            "mac_equivalents_after_exact_temporal_hoist": total_macs,
            "tensor_commands": total_commands,
            "input_stream_bytes_without_inter_layer_spill": total_input_bytes,
            "output_stream_bytes_without_inter_layer_spill": total_output_bytes,
        },
        "operators": [asdict(item) for item in operators],
        "parameter_tensors": parameter_tensors,
        "semantic_notes": [
            "BatchNorm is folded into the preceding learned projection in evaluation mode.",
            "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.",
            "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with
     dense execution.",
```

```
            "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.",
            "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",
        ],
    }
    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_text(json.dumps(report, indent=2) + "\n", encoding="ascii")
    return report


def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument("--checkpoint", required=True, type=Path)
    parser.add_argument("--output", required=True, type=Path)
    return parser.parse_args()


if __name__ == "__main__":
    arguments = parse_arguments()
    result = generate(arguments.checkpoint.resolve(), arguments.output.resolve())
    print(json.dumps(result["per_image_totals"], indent=2))
```

31.3 Line annotations

Line 1: `"""Generate the exact Spikingformer-to-NeuraEdge execution manifest."""` This is executable documentation attached to the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import json` This imports the named Python API so the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from dataclasses import asdict, dataclass` This imports the named Python API so the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `from math import ceil` This imports the named Python API so the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `from pathlib import Path` This imports the named Python API so the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `from typing import Literal` This imports the named Python API so the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `import torch` This imports the named Python API so the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `ROW_TILE = 8` This assigns ‘8‘ to ‘ROW_TILE ‘ in host memory. Here ROW_TILE means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 16: `COLUMN_TILE = 16` This assigns ‘16‘ to ‘COLUMN_TILE ‘ in host memory. Here COLUMN_TILE means one output-feature position processed in parallel. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 17: `MAX_REDUCTION = 2048` This assigns ‘2048‘ to ‘MAX_REDUCTION ‘ in host memory. Here MAX_REDUCTION means the value named ’max reduction’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 20: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total; it creates no FPGA circuit by itself.

Line 21: `class MatrixSchedule:` This defines class MatrixSchedule, which groups data and methods for matrix schedule. Instances exist in host memory and organize the flow that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 22: `rows: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are rows: one output-row position processed by the tensor tile.

Line 23: `reduction: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are reduction: the value named ’reduction’ in this block.

Line 24: `columns: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are columns: one output-feature position processed in parallel.

Line 25: `repetitions: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are repetitions: the value named ’repetitions’ in this block.

Line 26: `arithmetic: Literal["u8_mac", "binary_spike_add", "dynamic_u8_mac"]` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are arithmetic: the value named ’arithmetic’ in this block; Literal: the value named ’literal’ in this block; u8_mac: the value named ’u8 mac’ in this block.

Line 27: `mac_equivalents: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are mac_equivalents: the value named ’mac equivalents’ in this block.

Line 28: `reduction_chunks: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are reduction_chunks: the value named ’reduction chunks’ in this block.

Line 29: `compute_tiles: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are compute_tiles: the value named 'compute tiles' in this block.

Line 30: `commands: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 31: `input_stream_bytes: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are input_stream_bytes: the value named 'input stream bytes' in this block.

Line 32: `output_stream_bytes: int` Inside the class MatrixSchedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are output_stream_bytes: the value named 'output stream bytes' in this block.

Line 35: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total; it creates no FPGA circuit by itself.

Line 36: `class OperatorMapping:` This defines class OperatorMapping, which groups data and methods for operator mapping. Instances exist in host memory and organize the flow that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 37: `name: str` Inside the class OperatorMapping, which handles operator mapping, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are name: the value named 'name' in this block; str: the value named 'str' in this block.

Line 38: `source_operation: str` Inside the class OperatorMapping, which handles operator mapping, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are source_operation: the value named 'source operation' in this block; str: the value named 'str' in this block.

Line 39: `input_shape_per_image: tuple[int, ...]` Inside the class OperatorMapping, which handles operator mapping, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are input_shape_per_image: the value named 'input shape per image' in this block; tuple: the value named 'tuple' in this block.

Line 40: `output_shape_per_image: tuple[int, ...]` Inside the class OperatorMapping, which handles operator mapping, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are output_shape_per_image: the value named 'output shape per image' in this block; tuple: the value named 'tuple' in this block.

Line 41: `hardware_primitive: str` Inside the class OperatorMapping, which handles operator mapping, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are hardware_primitive: the value named 'hardware primitive' in this block; str: the value named 'str' in this block.

Line 42: `matrix: MatrixSchedule | None` Inside the class OperatorMapping, which handles operator mapping, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are matrix: the value named 'matrix' in this block; MatrixSchedule: the value named 'matrix schedule' in this block; None: the value named 'none' in this block.

Line 43: `exact_optimization: str | None = None` This assigns ‘None‘ to ‘exact_optimization‘ in host memory. Here exact_optimization means the value named ’exact optimization’ in this block. The important operands are None: the value named ’none’ in this block. Context: the class OperatorMapping, which handles operator mapping.

Line 46: `def matrix_schedule(` This starts function matrix_schedule. Its indented body performs matrix schedule as part of the file that derives every model tensor shape, tile schedule, command count, operation count, and traffic total; it runs only when called.

Line 47: `rows: int,` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are rows: one output-row position processed by the tensor tile.

Line 48: `reduction: int,` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are reduction: the value named ’reduction’ in this block.

Line 49: `columns: int,` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are columns: one output-feature position processed in parallel.

Line 50: `*,` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The expression is a literal or structural operator and has no hidden software state.

Line 51: `repetitions: int = 1,` This assigns ‘1,‘ to ‘repetitions‘ in host memory. Here repetitions means the value named ’repetitions’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function matrix_schedule, which handles matrix schedule.

Line 52: `arithmetic: Literal["u8_mac", "binary_spike_add", "dynamic_u8_mac"],` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are arithmetic: the value named ’arithmetic’ in this block; Literal: the value named ’literal’ in this block; u8_mac: the value named ’u8 mac’ in this block.

Line 53: `) -> MatrixSchedule:` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are MatrixSchedule: the value named ’matrix schedule’ in this block.

Line 54: `"""Count commands and stream bytes using the compiler's public tiling ABI."""` This is executable documentation attached to the function matrix_schedule, which handles matrix schedule. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 56: `if min(rows, reduction, columns, repetitions) <= 0:` This executes the following indented host-code branch only when ‘min(rows, reduction, columns, repetitions) <= 0‘ is true. The important operands are min: the value named ’min’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 57: `raise ValueError("all matrix dimensions and repetitions must be positive")` This rejects an invalid state by raising ‘ValueError("all matrix dimensions and repetitions must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 58: `total_commands = 0` This assigns ‘0‘ to ‘total_commands ‘ in host memory. Here total_commands means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The expression is a literal or structural operator and has no hidden software state. Context: the function matrix_schedule, which handles matrix schedule.

Line 59: `total_compute_tiles = 0` This assigns ‘0‘ to ‘total_compute_tiles ‘ in host memory. Here total_compute_tiles means the value named ’total compute tiles’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function matrix_schedule, which handles matrix schedule.

Line 60: `total_input_bytes = 0` This assigns ‘0‘ to ‘total_input_bytes ‘ in host memory. Here total_input_bytes means the value named ’total input bytes’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function matrix_schedule, which handles matrix schedule.

Line 61: `total_output_bytes = 0` This assigns ‘0‘ to ‘total_output_bytes ‘ in host memory. Here total_output_bytes means the value named ’total output bytes’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function matrix_schedule, which handles matrix schedule.

Line 62: `chunks = 0` This assigns ‘0‘ to ‘chunks ‘ in host memory. Here chunks means the value named ’chunks’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function matrix_schedule, which handles matrix schedule.

Line 63: `for reduction_start in range(0, reduction, MAX_REDUCTION):` This iterates reduction_start over ‘range(0, reduction, MAX_REDUCTION)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function matrix_schedule, which handles matrix schedule.

Line 64: `chunk = min(MAX_REDUCTION, reduction - reduction_start)` This assigns ‘min(MAX_REDUCTION, reduction - reduction_start)‘ to ‘chunk ‘ in host memory. Here chunk means the value named ’chunk’ in this block. The important operands are min: the value named ’min’ in this block; MAX_REDUCTION: the value named ’max reduction’ in this block; reduction: the value named ’reduction’ in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 65: `chunks += 1` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are chunks: the value named ’chunks’ in this block.

Line 66: `row_tiles = ceil(rows / ROW_TILE)` This assigns ‘ceil(rows / ROW_TILE)‘ to ‘row_tiles ‘ in host memory. Here row_tiles means one output-row position processed by the tensor tile. The important operands are ceil: the value named ’ceil’ in this block; rows: one output-row position processed by the tensor tile; ROW_TILE: one output-row position processed by the tensor tile. Context: the function matrix_schedule, which handles matrix schedule.

Line 67: `column_tiles = ceil(columns / COLUMN_TILE)` This assigns ‘ceil(columns / COLUMN_TILE)‘ to ‘column_tiles ‘ in host memory. Here column_tiles means one output-feature position processed in parallel. The important operands are ceil: the value named ’ceil’ in this block; columns: one output-feature position processed in parallel; COLUMN_TILE: one output-feature position processed in parallel. Context: the function matrix_schedule, which handles matrix schedule.

Line 68: `compute_tiles = row_tiles * column_tiles` This assigns ‘row_tiles * column_tiles‘ to ‘compute_tiles ‘ in host memory. Here compute_tiles means the value named ’compute tiles’ in this block. The important operands are row_tiles: one output-row position processed by the tensor tile; column_tiles: one output-feature position processed in parallel. Context: the function matrix_schedule, which handles matrix schedule.

Line 69: `if rows == 1:` This executes the following indented host-code branch only when ‘rows == 1‘ is true. The important operands are rows: one output-row position processed by the tensor tile. Context: the function matrix_schedule, which handles matrix schedule.

Line 70: `issues = ceil(chunk / 8)` This assigns ‘ceil(chunk / 8)‘ to ‘issues ‘ in host memory. Here issues means the value named ’issues’ in this block. The important operands are ceil: the value named ’ceil’ in this block; chunk: the value named ’chunk’ in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 71: `activation_tile_bytes = issues * 8` This assigns ‘issues * 8‘ to ‘activation_tile_bytes ‘ in host memory. Here activation_tile_bytes means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are issues: the value named ’issues’ in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 72: `weight_tile_bytes = issues * 8 * 16` This assigns ‘issues * 8 * 16‘ to ‘weight_tile_bytes ‘ in host memory. Here weight_tile_bytes means a learned coefficient multiplied by an activation. The important operands are issues: the value named ’issues’ in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 73: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function matrix_schedule, which handles matrix schedule.

Line 74: `activation_tile_bytes = chunk * ROW_TILE` This assigns ‘chunk * ROW_TILE‘ to ‘activation_tile_bytes ‘ in host memory. Here activation_tile_bytes means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are chunk: the value named ’chunk’ in this block; ROW_TILE: one output-row position processed by the tensor tile. Context: the function matrix_schedule, which handles matrix schedule.

Line 75: `weight_tile_bytes = chunk * COLUMN_TILE` This assigns ‘chunk * COLUMN_TILE‘ to ‘weight_tile_bytes ‘ in host memory. Here weight_tile_bytes means a learned coefficient multiplied by an activation. The important operands are chunk: the value named ’chunk’ in this block; COLUMN_TILE: one output-feature position processed in parallel. Context: the function matrix_schedule, which handles matrix schedule.

Line 76: `input_broadcast_bytes = (` This statement continues through source line 79; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(row_tiles * activation_tile_bytes + compute_tiles * weight_tile_bytes)‘ to ‘input_broadcast_bytes ‘ in host memory. Here input_broadcast_bytes means the value named ’input broadcast bytes’ in this block. The important operands are row_tiles: one output-row position processed by the tensor tile; activation_tile_bytes: an input feature value consumed by a neuron or multiply-accumulate operation; compute_tiles: the value named ’compute tiles’ in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 77: `row_tiles * activation_tile_bytes` This is continuation line 2 of the 4-line logical statement that begins at source line 76. Its exact fragment is ‘row_tiles * activation_tile_bytes‘. The important operands are row_tiles: one output-row position processed by the tensor tile; activation_tile_bytes: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘(row_tiles * activation_tile_bytes + compute_tiles * weight_tile_bytes)‘ to ‘input_broadcast_bytes ‘ in host memory. Here input_broadcast_bytes means the value named ’input broadcast bytes’ in this block. The important operands are row_tiles: one output-row position processed by the tensor tile; activation_tile_bytes: an input feature value consumed by a neuron or multiply-accumulate operation; compute_tiles: the value named ’compute tiles’ in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 78: `+ compute_tiles * weight_tile_bytes` This is continuation line 3 of the 4-line logical statement that begins at source line 76. Its exact fragment is ‘+ compute_tiles * weight_tile_bytes‘. The important operands are compute_tiles: the value named ’compute tiles’ in this block; weight_tile_bytes: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘(row_tiles * activation_tile_bytes + compute_tiles * weight_tile_bytes)‘ to ‘input_broadcast_bytes ‘ in host memory. Here input_broadcast_bytes means the value named ’input broadcast bytes’ in this block. The important operands are row_tiles: one output-row position processed by the tensor tile; activation_tile_bytes: an input feature value consumed by a neuron or multiply-accumulate operation; compute_tiles: the value named ’compute tiles’ in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 79: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 76. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(row_tiles * activation_tile_bytes + compute_tiles * weight_tile_bytes)‘ to ‘input_broadcast_bytes ‘ in host memory. Here input_broadcast_bytes means the value named ’input broadcast bytes’ in this block. The important operands are row_tiles: one output-row position processed by the tensor tile; activation_tile_bytes: an input

feature value consumed by a neuron or multiply-accumulate operation; compute_tiles: the value named 'compute tiles' in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 80: `weight_broadcast_bytes = (` This statement continues through source line 83; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(column_tiles * weight_tile_bytes + compute_tiles * activation_tile_bytes)' to 'weight_broadcast_bytes ' in host memory. Here weight_broadcast_bytes means a learned coefficient multiplied by an activation. The important operands are column_tiles: one output-feature position processed in parallel; weight_tile_bytes: a learned coefficient multiplied by an activation; compute_tiles: the value named 'compute tiles' in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 81: `column_tiles * weight_tile_bytes` This is continuation line 2 of the 4-line logical statement that begins at source line 80. Its exact fragment is 'column_tiles * weight_tile_bytes'. The important operands are column_tiles: one output-feature position processed in parallel; weight_tile_bytes: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns '(column_tiles * weight_tile_bytes + compute_tiles * activation_tile_bytes)' to 'weight_broadcast_bytes ' in host memory. Here weight_broadcast_bytes means a learned coefficient multiplied by an activation. The important operands are column_tiles: one output-feature position processed in parallel; weight_tile_bytes: a learned coefficient multiplied by an activation; compute_tiles: the value named 'compute tiles' in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 82: `+ compute_tiles * activation_tile_bytes` This is continuation line 3 of the 4-line logical statement that begins at source line 80. Its exact fragment is '+ compute_tiles * activation_tile_bytes'. The important operands are compute_tiles: the value named 'compute tiles' in this block; activation_tile_bytes: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns '(column_tiles * weight_tile_bytes + compute_tiles * activation_tile_bytes)' to 'weight_broadcast_bytes ' in host memory. Here weight_broadcast_bytes means a learned coefficient multiplied by an activation. The important operands are column_tiles: one output-feature position processed in parallel; weight_tile_bytes: a learned coefficient multiplied by an activation; compute_tiles: the value named 'compute tiles' in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 83: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 80. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(column_tiles * weight_tile_bytes + compute_tiles * activation_tile_bytes)' to 'weight_broadcast_bytes ' in host memory. Here weight_broadcast_bytes means a learned coefficient multiplied by an activation. The important operands are column_tiles: one output-feature position processed in parallel; weight_tile_bytes: a learned coefficient multiplied by an activation; compute_tiles: the value named 'compute tiles' in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 84: `if input_broadcast_bytes <= weight_broadcast_bytes:` This executes the following indented host-code branch only when 'input_broadcast_bytes <= weight_broadcast_bytes' is true. The important operands are input_broadcast_bytes: the value named 'input broadcast bytes' in this block; weight_broadcast_bytes: a learned coefficient multiplied by an activation. Context: the function matrix_schedule, which handles matrix schedule.

Line 85: `load_commands = row_tiles + compute_tiles` This assigns 'row_tiles + compute_tiles' to 'load_commands ' in host memory. Here load_commands means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are row_tiles: one output-row position processed by the tensor tile; compute_tiles: the value named 'compute tiles' in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 86: `stream_bytes = input_broadcast_bytes` This assigns 'input_broadcast_bytes' to 'stream_bytes ' in host memory. Here stream_bytes means the value named 'stream bytes' in this block. The important operands are input_broadcast_bytes: the value named 'input broadcast bytes' in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 87: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function matrix_schedule, which handles matrix schedule.

Line 88: `load_commands = column_tiles + compute_tiles` This assigns ‘column_tiles + compute_tiles‘ to ‘load_commands ‘ in host memory. Here load_commands means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are column_tiles: one output-feature position processed in parallel; compute_tiles: the value named ’compute tiles’ in this block. Context: the function matrix_schedule, which handles matrix schedule.

Line 89: `stream_bytes = weight_broadcast_bytes` This assigns ‘weight_broadcast_bytes‘ to ‘stream_bytes ‘ in host memory. Here stream_bytes means the value named ’stream bytes’ in this block. The important operands are weight_broadcast_bytes: a learned coefficient multiplied by an activation. Context: the function matrix_schedule, which handles matrix schedule.

Line 90: `total_commands += load_commands + compute_tiles` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are total_commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; load_commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_tiles: the value named ’compute tiles’ in this block.

Line 91: `total_compute_tiles += compute_tiles` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are total_compute_tiles: the value named ’total compute tiles’ in this block; compute_tiles: the value named ’compute tiles’ in this block.

Line 92: `total_input_bytes += stream_bytes` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are total_input_bytes: the value named ’total input bytes’ in this block; stream_bytes: the value named ’stream bytes’ in this block.

Line 93: `# The current output ABI emits one 32-bit result in each 64-bit beat.` The comment records this intent: The current output ABI emits one 32-bit result in each 64-bit beat. It applies inside the function matrix_schedule, which handles matrix schedule.

Line 94: `total_output_bytes += rows * columns * 8` Inside the function matrix_schedule, which handles matrix schedule, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are total_output_bytes: the value named ’total output bytes’ in this block; rows: one output-row position processed by the tensor tile; columns: one output-feature position processed in parallel.

Line 95: `return MatrixSchedule(` This statement continues through source line 107; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_outpu * repetitions,)‘ to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named ’matrix schedule’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 96: `rows=rows,` This is continuation line 2 of the 13-line logical statement that begins at source line 95. Its exact fragment is ‘rows=rows,‘. The important operands are rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This returns ‘MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_output_bytes * repetitions,)‘ to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named ’matrix schedule’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 97: `reduction=reduction,` This is continuation line 3 of the 13-line logical statement that begins at source line 95. Its exact fragment is 'reduction=reduction,'. The important operands are reduction: the value named 'reduction' in this block. Read together, the complete operation is: This returns 'MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_outpu * repetitions,)' to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 98: `columns=columns,` This is continuation line 4 of the 13-line logical statement that begins at source line 95. Its exact fragment is 'columns=columns,'. The important operands are columns: one output-feature position processed in parallel. Read together, the complete operation is: This returns 'MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_outpu * repetitions,)' to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 99: `repetitions=repetitions,` This is continuation line 5 of the 13-line logical statement that begins at source line 95. Its exact fragment is 'repetitions=repetitions,'. The important operands are repetitions: the value named 'repetitions' in this block. Read together, the complete operation is: This returns 'MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_outpu * repetitions,)' to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 100: `arithmetic=arithmetic,` This is continuation line 6 of the 13-line logical statement that begins at source line 95. Its exact fragment is 'arithmetic=arithmetic,'. The important operands are arithmetic: the value named 'arithmetic' in this block. Read together, the complete operation is: This returns 'MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_outpu * repetitions,)' to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 101: `mac_equivalents=rows * reduction * columns * repetitions,` This is continuation line 7 of the 13-line logical statement that begins at source line 95. Its exact fragment is 'mac_equivalents=rows * reduction * columns * repetitions,'. The important operands are mac_equivalents: the value named 'mac equivalents' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block. Read together, the complete operation is: This returns 'MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=t * repetitions, output_stream_bytes=total_output_bytes * repetitions,)' to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 102: `reduction_chunks=chunks * repetitions,` This is continuation line 8 of the 13-line logical statement that begins at source line 95. Its exact fragment is 'reduction_chunks=chunks * repetitions,'. The important operands are reduction_chunks: the value named 'reduction chunks' in this block; chunks: the value named 'chunks' in this block; repetitions: the value named 'repetitions' in this block. Read together, the complete operation is: This returns 'MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic,

mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_output_bytes * repetitions,)‘ to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 103: `compute_tiles=total_compute_tiles * repetitions,` This is continuation line 9 of the 13-line logical statement that begins at source line 95. Its exact fragment is ‘compute_tiles=total_compute_tiles * repetitions,‘. The important operands are compute_tiles: the value named 'compute tiles' in this block; total_compute_tiles: the value named 'total compute tiles' in this block; repetitions: the value named 'repetitions' in this block. Read together, the complete operation is: This returns ‘MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input * repetitions, output_stream_bytes=total_output_bytes * repetitions,)‘ to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 104: `commands=total_commands * repetitions,` This is continuation line 10 of the 13-line logical statement that begins at source line 95. Its exact fragment is ‘commands=total_commands * repetitions,‘. The important operands are commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; total_commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; repetitions: the value named 'repetitions' in this block. Read together, the complete operation is: This returns ‘MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_output * repetitions,)‘ to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 105: `input_stream_bytes=total_input_bytes * repetitions,` This is continuation line 11 of the 13-line logical statement that begins at source line 95. Its exact fragment is ‘input_stream_bytes=total_input_bytes * repetitions,‘. The important operands are input_stream_bytes: the value named 'input stream bytes' in this block; total_input_bytes: the value named 'total input bytes' in this block; repetitions: the value named 'repetitions' in this block. Read together, the complete operation is: This returns ‘MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_output_bytes * repetitions,)‘ to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 106: `output_stream_bytes=total_output_bytes * repetitions,` This is continuation line 12 of the 13-line logical statement that begins at source line 95. Its exact fragment is ‘output_stream_bytes=total_output_bytes * repetitions,‘. The important operands are output_stream_bytes: the value named 'output stream bytes' in this block; total_output_bytes: the value named 'total output bytes' in this block; repetitions: the value named 'repetitions' in this block. Read together, the complete operation is: This returns ‘MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_input_bytes * repetitions, output_stream_bytes=total_output * repetitions,)‘ to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named 'matrix schedule' in this block; rows: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block.

Line 107: `)` This is continuation line 13 of the 13-line logical statement that begins at source line 95. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete op-

eration is: This returns ‘MatrixSchedule(rows=rows, reduction=reduction, columns=columns, repetitions=repetitions, arithmetic=arithmetic, mac_equivalents=rows * reduction * columns * repetitions, reduction_chunks=chunks * repetitions, compute_tiles=total_compute_tiles * repetitions, commands=total_commands * repetitions, input_stream_bytes=total_in * repetitions, output_stream_bytes=total_output_bytes * repetitions,)‘ to the caller of the function matrix_schedule, which handles matrix schedule, ending this invocation. The important operands are MatrixSchedule: the value named ’matrix schedule’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 110: `def build_operator_manifest() -> list[OperatorMapping]:` This starts function build_operator_manifest. Its indented body performs build operator manifest as part of the file that derives every model tensor shape, tile schedule, command count, operation count, and traffic total; it runs only when called.

Line 111: `"""Describe every learned and data-dependent operator for one image."""` This is executable documentation attached to the function build_operator_manifest, which handles build operator manifest. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 113: `time_steps = 4` This assigns ‘4‘ to ‘time_steps ‘ in host memory. Here time_steps means the value named ’time steps’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_operator_manifest, which handles build operator manifest.

Line 114: `channels = 384` This assigns ‘384‘ to ‘channels ‘ in host memory. Here channels means the value named ’channels’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_operator_manifest, which handles build operator manifest.

Line 115: `tokens = 64` This assigns ‘64‘ to ‘tokens ‘ in host memory. Here tokens means the value named ’tokens’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_operator_manifest, which handles build operator manifest.

Line 116: `heads = 12` This assigns ‘12‘ to ‘heads ‘ in host memory. Here heads means the value named ’heads’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_operator_manifest, which handles build operator manifest.

Line 117: `head_dim = channels // heads` This assigns ‘channels // heads‘ to ‘head_dim ‘ in host memory. Here head_dim means the value named ’head dim’ in this block. The important operands are channels: the value named ’channels’ in this block; heads: the value named ’heads’ in this block. Context: the function build_operator_manifest, which handles build operator manifest.

Line 118: `operators: list[OperatorMapping] = []` This assigns ‘[]‘ to ‘operators‘ in host memory. Here operators means the value named ’operators’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_operator_manifest, which handles build operator manifest.

Line 120: `def matrix_operator(` This starts function matrix_operator. Its indented body performs matrix operator as part of the file that derives every model tensor shape, tile schedule, command count, operation count, and traffic total; it runs only when called.

Line 121: `name: str,` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are name: the value named ’name’ in this block; str: the value named ’str’ in this block.

Line 122: `source: str,` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are source: the value named ’source’ in this block; str: the value named ’str’ in this block.

Line 123: `input_shape: tuple[int, ...],` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are input_shape: the value named 'input shape' in this block; tuple: the value named 'tuple' in this block.

Line 124: `output_shape: tuple[int, ...],` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are output_shape: the value named 'output shape' in this block; tuple: the value named 'tuple' in this block.

Line 125: `rows: int,` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are rows: one output-row position processed by the tensor tile.

Line 126: `reduction: int,` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are reduction: the value named 'reduction' in this block.

Line 127: `columns: int,` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are columns: one output-feature position processed in parallel.

Line 128: `arithmetic: Literal["u8_mac", "binary_spike_add", "dynamic_u8_mac"],` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are arithmetic: the value named 'arithmetic' in this block; Literal: the value named 'literal' in this block; u8_mac: the value named 'u8 mac' in this block.

Line 129: `primitive: str,` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are primitive: the value named 'primitive' in this block; str: the value named 'str' in this block.

Line 130: `*,` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The expression is a literal or structural operator and has no hidden software state.

Line 131: `repetitions: int = 1,` This assigns '1,' to 'repetitions' in host memory. Here repetitions means the value named 'repetitions' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function matrix_operator, which handles matrix operator.

Line 132: `optimization: str | None = None,` This assigns 'None,' to 'optimization' in host memory. Here optimization means the value named 'optimization' in this block. The important operands are None: the value named 'none' in this block. Context: the function matrix_operator, which handles matrix operator.

Line 133: `) -> None:` Inside the function matrix_operator, which handles matrix operator, this statement advances the host implementation that derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The important operands are None: the value named 'none' in this block.

Line 134: `operators.append(OperatorMapping(` This statement continues through source line 146; the explanation uses the complete expression rather than only the first visual fragment. This calls 'operators.append' with 'OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic), exact_optimization=optimization,)' inside the function matrix_operator, which handles matrix operator. The call

advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 135: `name=name,` This is continuation line 2 of the 13-line logical statement that begins at source line 134. Its exact fragment is 'name=name,'. The important operands are name: the value named 'name' in this block. Read together, the complete operation is: This calls 'operators.append' with 'OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic,), exact_optimization=optimizati)' inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 136: `source_operation=source,` This is continuation line 3 of the 13-line logical statement that begins at source line 134. Its exact fragment is 'source_operation=source,'. The important operands are source_operation: the value named 'source operation' in this block; source: the value named 'source' in this block. Read together, the complete operation is: This calls 'operators.append' with 'OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic,), exact_optimization=optimizati)' inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 137: `input_shape_per_image=input_shape,` This is continuation line 4 of the 13-line logical statement that begins at source line 134. Its exact fragment is 'input_shape_per_image=input_shape,'. The important operands are input_shape_per_image: the value named 'input shape per image' in this block; input_shape: the value named 'input shape' in this block. Read together, the complete operation is: This calls 'operators.append' with 'OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic), exact_optimization=optimization,)' inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 138: `output_shape_per_image=output_shape,` This is continuation line 5 of the 13-line logical statement that begins at source line 134. Its exact fragment is 'output_shape_per_image=output_shape,'. The important operands are output_shape_per_image: the value named 'output shape per image' in this block; output_shape: the value named 'output shape' in this block. Read together, the complete operation is: This calls 'operators.append' with 'OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic), exact_optimization=optimization,)' inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 139: `hardware_primitive=primitive,` This is continuation line 6 of the 13-line logical statement that begins at source line 134. Its exact fragment is 'hardware_primitive=primitive,'. The important operands are hardware_primitive: the value named 'hardware primitive' in this block; primitive: the value named 'primitive' in this block. Read together, the complete operation is: This calls 'operators.append' with 'OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic,), exact_optimization=optimizati)' inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 140: `matrix=matrix_schedule(` This is continuation line 7 of the 13-line logical statement that begins at source line 134. Its exact fragment is 'matrix=matrix_schedule('. The important operands are matrix: the value named 'matrix' in this block; matrix_schedule: the value named 'matrix schedule' in this block. Read together, the complete operation is: This calls 'operators.append' with 'OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic,), exact_optimization=optimizati

)‘ inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 141: `rows, reduction, columns,` This is continuation line 8 of the 13-line logical statement that begins at source line 134. Its exact fragment is ‘rows, reduction, columns,‘. The important operands are rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block; columns: one output-feature position processed in parallel. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic), exact_optimization=optimization,)‘ inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 142: `repetitions=repetitions,` This is continuation line 9 of the 13-line logical statement that begins at source line 134. Its exact fragment is ‘repetitions=repetitions,‘. The important operands are repetitions: the value named ’repetitions’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic), exact_optimization=optimization,)‘ inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 143: `arithmetic=arithmetic,` This is continuation line 10 of the 13-line logical statement that begins at source line 134. Its exact fragment is ‘arithmetic=arithmetic,‘. The important operands are arithmetic: the value named ’arithmetic’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic), exact_optimization=optimization,)‘ inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 144: `),` This is continuation line 11 of the 13-line logical statement that begins at source line 134. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic,), exact_optimization=optimizati)‘ inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 145: `exact_optimization=optimization,` This is continuation line 12 of the 13-line logical statement that begins at source line 134. Its exact fragment is ‘exact_optimization=optimization,‘. The important operands are exact_optimization: the value named ’exact optimization’ in this block; optimization: the value named ’optimization’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic,), exact_optimization=optimiz)‘ inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 146: `))` This is continuation line 13 of the 13-line logical statement that begins at source line 134. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=name, source_operation=source, input_shape_per_image=input_shape, output_shape_per_image=output_shape, hardware_primitive=primitive, matrix=matrix_schedule(rows, reduction, columns, repetitions=repetitions, arithmetic=arithmetic,), exact_optimization=optimizati)‘ inside the function matrix_operator, which handles matrix operator. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 148: `matrix_operator(` This statement continues through source line 156; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm", (3, 32, 32), (time_steps, 48, 32, 32), 32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile", optimization=("Compute once and broadcast to four timesteps. The source repeats " "one unchanged image before this stateless operator, so this removes " "three of four executions with exact output equivalence."), ‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 149: `"patch_embed.block0", "Conv2d 3x3 + folded BatchNorm",` This is continuation line 2 of the 9-line logical statement that begins at source line 148. Its exact fragment is ‘"patch_embed.block0", "Conv2d 3x3 + folded BatchNorm",‘. The important operands are patch_embed: the value named ’patch embed’ in this block; block0: the value named ’block0’ in this block; Conv2d: the value named ’conv2d’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm", (3, 32, 32), (time_steps, 48, 32, 32), 32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile", optimization=("Compute once and broadcast to four timesteps. The source repeats " "one unchanged image before this stateless operator, so this removes " "three of four executions with exact output equivalence."), ‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 150: `(3, 32, 32), (time_steps, 48, 32, 32),` This is continuation line 3 of the 9-line logical statement that begins at source line 148. Its exact fragment is ‘(3, 32, 32), (time_steps, 48, 32, 32),‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm", (3, 32, 32), (time_steps, 48, 32, 32), 32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile", optimization=("Compute once and broadcast to four timesteps. The source repeats " "one unchanged image before this stateless operator, so this removes " "three of four executions with exact output equivalence."), ‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 151: `32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile",` This is continuation line 4 of the 9-line logical statement that begins at source line 148. Its exact fragment is ‘32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile",‘. The important operands are u8_mac: the value named ’u8 mac’ in this block; dense: the value named ’dense’ in this block; tensor: the value named ’tensor’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm", (3, 32, 32), (time_steps, 48, 32, 32), 32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile", optimization=("Compute once and broadcast to four timesteps. The source repeats " "one unchanged image before this stateless operator, so this removes " "three of four executions with exact output equivalence."), ‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 152: `optimization=(` This is continuation line 5 of the 9-line logical statement that begins at source line 148. Its exact fragment is ‘optimization=(‘. The important operands are optimization: the value named ’optimization’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm", (3, 32, 32), (time_steps, 48, 32, 32), 32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile", optimization=("Compute once and broadcast to four timesteps. The source repeats " "one unchanged image before this stateless operator, so this removes " "three of four executions with exact output equivalence."), ‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 153: `"Compute once and broadcast to four timesteps. The source repeats "` This is continuation line 6 of the 9-line logical statement that begins at source line 148. Its exact fragment is ‘"Compute once and broadcast to four timesteps. The source repeats "‘. The important operands are Compute: the value named ’compute’ in this block; once: the value named ’once’ in this block; and: the value named ’and’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm", (3, 32, 32), (time_steps, 48, 32, 32), 32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile", optimization=("Compute once and broadcast to four timesteps. The source repeats " "one unchanged image before this stateless operator, so this removes " "three of four executions with exact output equivalence."), ‘ inside the function build_operator_manifest, which handles build

operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 154: `"one unchanged image before this stateless operator, so this removes "` This is continuation line 7 of the 9-line logical statement that begins at source line 148. Its exact fragment is ‘"one unchanged image before this stateless operator, so this removes "‘. The important operands are one: the value named ’one’ in this block; unchanged: the value named ’unchanged’ in this block; image: the value named ’image’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm", (3, 32, 32), (time_steps, 48, 32, 32), 32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile", optimization=("Compute once and broadcast to four timesteps. The source repeats " "one unchanged image before this stateless operator, so this removes " "three of four executions with exact output equivalence."), ‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 155: `"three of four executions with exact output equivalence."),` This is continuation line 8 of the 9-line logical statement that begins at source line 148. Its exact fragment is ‘"three of four executions with exact output equivalence."),‘. The important operands are three: the value named ’three’ in this block; of: the value named ’of’ in this block; four: the value named ’four’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm", (3, 32, 32), (time_steps, 48, 32, 32), 32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile", optimization=("Compute once and broadcast to four timesteps. The source repeats " "one unchanged image before this stateless operator, so this removes " "three of four executions with exact output equivalence."), ‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 156: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 148. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block0", "Conv2d 3x3 + folded BatchNorm", (3, 32, 32), (time_steps, 48, 32, 32), 32 * 32, 3 * 3 * 3, 48, "u8_mac", "dense tensor tile", optimization=("Compute once and broadcast to four timesteps. The source repeats " "one unchanged image before this stateless operator, so this removes " "three of four executions with exact output equivalence."), ‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 157: `matrix_operator(` This statement continues through source line 161; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ "patch_embed.block1", "LIF + Conv2d 3x3 + folded BatchNorm", (time_steps, 48, 32, 32), (time_steps, 96, 32, 32), time_steps * 32 * 32, 48 * 3 * 3, 96, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 158: `"patch_embed.block1", "LIF + Conv2d 3x3 + folded BatchNorm",` This is continuation line 2 of the 5-line logical statement that begins at source line 157. Its exact fragment is ‘"patch_embed.block1", "LIF + Conv2d 3x3 + folded BatchNorm",‘. The important operands are patch_embed: the value named ’patch embed’ in this block; block1: the value named ’block1’ in this block; LIF: the value named ’lif’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block1", "LIF + Conv2d 3x3 + folded BatchNorm", (time_steps, 48, 32, 32), (time_steps, 96, 32, 32), time_steps * 32 * 32, 48 * 3 * 3, 96, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 159: `(time_steps, 48, 32, 32), (time_steps, 96, 32, 32),` This is continuation line 3 of the 5-line logical statement that begins at source line 157. Its exact fragment is ‘(time_steps, 48, 32, 32), (time_steps, 96, 32, 32),‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block1", "LIF + Conv2d 3x3 + folded BatchNorm", (time_steps, 48, 32, 32), (time_steps, 96, 32, 32), time_steps * 32 * 32, 48 * 3 * 3, 96, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 160: `time_steps * 32 * 32, 48 * 3 * 3, 96,` This is continuation line 4 of the 5-line logical statement that begins at source line 157. Its exact fragment is ‘time_steps * 32 * 32, 48 * 3 * 3, 96,‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block1", "LIF + Conv2d 3x3 + folded BatchNorm", (time_steps, 48, 32, 32), (time_steps, 96, 32, 32), time_steps * 32 * 32, 48 * 3 * 3, 96, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 161: `"binary_spike_add", "active-index sparse tensor tile")` This is continuation line 5 of the 5-line logical statement that begins at source line 157. Its exact fragment is ‘"binary_spike_add", "active-index sparse tensor tile")‘. The important operands are binary_spike_add: a binary event produced when a neuron crosses its threshold; active: the value named ’active’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block1", "LIF + Conv2d 3x3 + folded BatchNorm", (time_steps, 48, 32, 32), (time_steps, 96, 32, 32), time_steps * 32 * 32, 48 * 3 * 3, 96, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 162: `matrix_operator(` This statement continues through source line 166; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ "patch_embed.block2", "LIF + Conv2d 3x3 + folded BatchNorm", (time_steps, 96, 32, 32), (time_steps, 192, 32, 32), time_steps * 32 * 32, 96 * 3 * 3, 192, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 163: `"patch_embed.block2", "LIF + Conv2d 3x3 + folded BatchNorm",` This is continuation line 2 of the 5-line logical statement that begins at source line 162. Its exact fragment is ‘"patch_embed.block2", "LIF + Conv2d 3x3 + folded BatchNorm",‘. The important operands are patch_embed: the value named ’patch embed’ in this block; block2: the value named ’block2’ in this block; LIF: the value named ’lif’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block2", "LIF + Conv2d 3x3 + folded BatchNorm", (time_steps, 96, 32, 32), (time_steps, 192, 32, 32), time_steps * 32 * 32, 96 * 3 * 3, 192, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 164: `(time_steps, 96, 32, 32), (time_steps, 192, 32, 32),` This is continuation line 3 of the 5-line logical statement that begins at source line 162. Its exact fragment is ‘(time_steps, 96, 32, 32), (time_steps, 192, 32, 32),‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block2", "LIF + Conv2d 3x3 + folded BatchNorm", (time_steps, 96, 32, 32), (time_steps, 192, 32, 32), time_steps * 32 * 32, 96 * 3 * 3, 192, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 165: `time_steps * 32 * 32, 96 * 3 * 3, 192,` This is continuation line 4 of the 5-line logical statement that begins at source line 162. Its exact fragment is ‘time_steps * 32 * 32, 96 * 3 * 3, 192,‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block2", "LIF + Conv2d 3x3 + folded BatchNorm", (time_steps, 96, 32, 32), (time_steps, 192, 32, 32), time_steps * 32 * 32, 96 * 3 * 3, 192, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 166: `"binary_spike_add", "active-index sparse tensor tile")` This is continuation line 5 of the 5-line logical statement that begins at source line 162. Its exact fragment is ‘"binary_spike_add", "active-index sparse tensor tile")‘. The important operands are binary_spike_add: a binary event produced when a neuron crosses its threshold; active: the value named ’active’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block2", "LIF + Conv2d

3x3 + folded BatchNorm", (time_steps, 96, 32, 32), (time_steps, 192, 32, 32), time_steps * 32 * 32, 96 * 3 * 3, 192, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 167: `matrix_operator(` This statement continues through source line 171; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ "patch_embed.block3", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 192, 32, 32), (time_steps, 384, 16, 16), time_steps * 16 * 16, 192 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 168: `"patch_embed.block3", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm",` This is continuation line 2 of the 5-line logical statement that begins at source line 167. Its exact fragment is ‘"patch_embed.block3", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm",‘. The important operands are patch_embed: the value named ’patch embed’ in this block; block3: the value named ’block3’ in this block; LIF: the value named ’lif’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block3", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 192, 32, 32), (time_steps, 384, 16, 16), time_steps * 16 * 16, 192 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 169: `(time_steps, 192, 32, 32), (time_steps, 384, 16, 16),` This is continuation line 3 of the 5-line logical statement that begins at source line 167. Its exact fragment is ‘(time_steps, 192, 32, 32), (time_steps, 384, 16, 16),‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block3", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 192, 32, 32), (time_steps, 384, 16, 16), time_steps * 16 * 16, 192 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 170: `time_steps * 16 * 16, 192 * 3 * 3, 384,` This is continuation line 4 of the 5-line logical statement that begins at source line 167. Its exact fragment is ‘time_steps * 16 * 16, 192 * 3 * 3, 384,‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block3", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 192, 32, 32), (time_steps, 384, 16, 16), time_steps * 16 * 16, 192 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 171: `"binary_spike_add", "vector maximum + active-index sparse tensor tile")` This is continuation line 5 of the 5-line logical statement that begins at source line 167. Its exact fragment is ‘"binary_spike_add", "vector maximum + active-index sparse tensor tile")‘. The important operands are binary_spike_add: a binary event produced when a neuron crosses its threshold; vector: the value named ’vector’ in this block; maximum: the value named ’maximum’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block3", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 192, 32, 32), (time_steps, 384, 16, 16), time_steps * 16 * 16, 192 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 172: `matrix_operator(` This statement continues through source line 176; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ "patch_embed.block4", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 384, 16, 16), (time_steps, 384, 8, 8), time_steps * 8 * 8, 384 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 173: `"patch_embed.block4", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm",` This is continuation line 2 of the 5-line logical statement that begins at source line 172. Its exact fragment is ‘"patch_embed.block4", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm",‘. The important operands are patch_embed: the value named ’patch embed’ in this block; block4: the value named ’block4’ in this block; LIF: the value named ’lif’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block4", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 384, 16, 16), (time_steps, 384, 8, 8), time_steps * 8 * 8, 384 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 174: `(time_steps, 384, 16, 16), (time_steps, 384, 8, 8),` This is continuation line 3 of the 5-line logical statement that begins at source line 172. Its exact fragment is ‘(time_steps, 384, 16, 16), (time_steps, 384, 8, 8),‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block4", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 384, 16, 16), (time_steps, 384, 8, 8), time_steps * 8 * 8, 384 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 175: `time_steps * 8 * 8, 384 * 3 * 3, 384,` This is continuation line 4 of the 5-line logical statement that begins at source line 172. Its exact fragment is ‘time_steps * 8 * 8, 384 * 3 * 3, 384,‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block4", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 384, 16, 16), (time_steps, 384, 8, 8), time_steps * 8 * 8, 384 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 176: `"binary_spike_add", "vector maximum + active-index sparse tensor tile")` This is continuation line 5 of the 5-line logical statement that begins at source line 172. Its exact fragment is ‘"binary_spike_add", "vector maximum + active-index sparse tensor tile")‘. The important operands are binary_spike_add: a binary event produced when a neuron crosses its threshold; vector: the value named ’vector’ in this block; maximum: the value named ’maximum’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "patch_embed.block4", "LIF + MaxPool2d + Conv2d 3x3 + folded BatchNorm", (time_steps, 384, 16, 16), (time_steps, 384, 8, 8), time_steps * 8 * 8, 384 * 3 * 3, 384, "binary_spike_add", "vector maximum + active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 178: `for block in range(4):` This iterates block over ‘range(4)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function build_operator_manifest, which handles build operator manifest.

Line 179: `prefix = f"block.{block}"` This assigns ‘f"block.{block}"‘ to ‘prefix ‘ in host memory. Here prefix means the value named ’prefix’ in this block. The important operands are f: the value named ’f’ in this block; block: the value named ’block’ in this block. Context: the function build_operator_manifest, which handles build operator manifest.

Line 180: `for projection in ("q", "k", "v"):` This iterates projection over ‘("q", "k", "v")‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function build_operator_manifest, which handles build operator manifest.

Line 181: `matrix_operator(` This statement continues through source line 187; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.{projection}", f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels, tokens), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 182: `f"{prefix}.attn.{projection}",` This is continuation line 2 of the 7-line logical statement that begins at source line 181. Its exact fragment is ‘f"{prefix}.attn.{projection}",‘. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; attn: the value named ’attn’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.{projection}", f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels, tokens), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 183: `f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm",` This is continuation line 3 of the 7-line logical statement that begins at source line 181. Its exact fragment is ‘f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm",‘. The important operands are f: the value named ’f’ in this block; LIF: the value named ’lif’ in this block; projection: the value named ’projection’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.{projection}", f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels, tokens), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 184: `(time_steps, channels, 8, 8),` This is continuation line 4 of the 7-line logical statement that begins at source line 181. Its exact fragment is ‘(time_steps, channels, 8, 8),‘. The important operands are time_steps: the value named ’time steps’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.{projection}", f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels, tokens), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 185: `(time_steps, channels, tokens),` This is continuation line 5 of the 7-line logical statement that begins at source line 181. Its exact fragment is ‘(time_steps, channels, tokens),‘. The important operands are time_steps: the value named ’time steps’ in this block; channels: the value named ’channels’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.{projection}", f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels, tokens), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 186: `time_steps * tokens, channels, channels,` This is continuation line 6 of the 7-line logical statement that begins at source line 181. Its exact fragment is ‘time_steps * tokens, channels, channels,‘. The important operands are time_steps: the value named ’time steps’ in this block; tokens: the value named ’tokens’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.{projection}", f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels, tokens), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 187: `"binary_spike_add", "active-index sparse tensor tile")` This is continuation line 7 of the 7-line logical statement that begins at source line 181. Its exact fragment is ‘"binary_spike_add", "active-index sparse tensor tile")‘. The important operands are binary_spike_add: a binary event produced when a neuron crosses its threshold; active: the value named ’active’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.{projection}", f"LIF + {projection.upper()} Conv1d 1x1 + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels, tokens), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 188: `matrix_operator(` This statement continues through source line 194; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.qk", "binary query times transposed binary key", (time_steps, heads, tokens, head_dim), (time_steps, heads, tokens, tokens), tokens, head_dim, tokens, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 189: `f"{prefix}.attn.qk", "binary query times transposed binary key",` This is continuation line 2 of the 7-line logical statement that begins at source line 188. Its exact fragment is ‘f"{prefix}.attn.qk", "binary query times transposed binary key",‘. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; attn: the value named ’attn’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.qk", "binary query times transposed binary key", (time_steps, heads, tokens, head_dim), (time_steps, heads, tokens, tokens), tokens, head_dim, tokens, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 190: `(time_steps, heads, tokens, head_dim),` This is continuation line 3 of the 7-line logical statement that begins at source line 188. Its exact fragment is ‘(time_steps, heads, tokens, head_dim),‘. The important operands are time_steps: the value named ’time steps’ in this block; heads: the value named ’heads’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.qk", "binary query times transposed binary key", (time_steps, heads, tokens, head_dim), (time_steps, heads, tokens, tokens), tokens, head_dim, tokens, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 191: `(time_steps, heads, tokens, tokens),` This is continuation line 4 of the 7-line logical statement that begins at source line 188. Its exact fragment is ‘(time_steps, heads, tokens, tokens),‘. The important operands are time_steps: the value named ’time steps’ in this block; heads: the value named ’heads’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.qk", "binary query times transposed binary key", (time_steps, heads, tokens, head_dim), (time_steps, heads, tokens, tokens), tokens, head_dim, tokens, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 192: `tokens, head_dim, tokens,` This is continuation line 5 of the 7-line logical statement that begins at source line 188. Its exact fragment is ‘tokens, head_dim, tokens,‘. The important operands are tokens: the value named ’tokens’ in this block; head_dim: the value named ’head dim’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.qk", "binary query times transposed binary key", (time_steps, heads, tokens, head_dim), (time_steps, heads, tokens, tokens), tokens, head_dim, tokens, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 193: `"dynamic_u8_mac", "dense tensor tile",` This is continuation line 6 of the 7-line logical statement that begins at source line 188. Its exact fragment is ‘"dynamic_u8_mac", "dense tensor tile",‘. The important operands are dynamic_u8_mac: the value named ’dynamic u8 mac’ in this block; dense: the value named ’dense’ in this block; tensor: the value named ’tensor’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.qk", "binary query times transposed binary key", (time_steps, heads, tokens, head_dim), (time_steps, heads, tokens, tokens), tokens, head_dim, tokens, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 194: `repetitions=time_steps * heads)` This is continuation line 7 of the 7-line logical statement that begins at source line 188. Its exact fragment is ‘repetitions=time_steps * heads)‘. The important operands are repetitions: the value named ’repetitions’ in this block; time_steps: the value named ’time steps’ in this block; heads: the value named

'heads' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.qk", "binary query times transposed binary key", (time_steps, heads, tokens, head_dim), (time_steps, heads, tokens, tokens), tokens, head_dim, tokens, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 195: `matrix_operator(` This statement continues through source line 201; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.av", "attention counts times binary value", (time_steps, heads, tokens, tokens), (time_steps, heads, tokens, head_dim), tokens, tokens, head_dim, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 196: `f"{prefix}.attn.av", "attention counts times binary value",` This is continuation line 2 of the 7-line logical statement that begins at source line 195. Its exact fragment is ‘f"{prefix}.attn.av", "attention counts times binary value",‘. The important operands are f: the value named 'f' in this block; prefix: the value named 'prefix' in this block; attn: the value named 'attn' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.av", "attention counts times binary value", (time_steps, heads, tokens, tokens), (time_steps, heads, tokens, head_dim), tokens, tokens, head_dim, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 197: `(time_steps, heads, tokens, tokens),` This is continuation line 3 of the 7-line logical statement that begins at source line 195. Its exact fragment is ‘(time_steps, heads, tokens, tokens),‘. The important operands are time_steps: the value named 'time steps' in this block; heads: the value named 'heads' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.av", "attention counts times binary value", (time_steps, heads, tokens, tokens), (time_steps, heads, tokens, head_dim), tokens, tokens, head_dim, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 198: `(time_steps, heads, tokens, head_dim),` This is continuation line 4 of the 7-line logical statement that begins at source line 195. Its exact fragment is ‘(time_steps, heads, tokens, head_dim),‘. The important operands are time_steps: the value named 'time steps' in this block; heads: the value named 'heads' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.av", "attention counts times binary value", (time_steps, heads, tokens, tokens), (time_steps, heads, tokens, head_dim), tokens, tokens, head_dim, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 199: `tokens, tokens, head_dim,` This is continuation line 5 of the 7-line logical statement that begins at source line 195. Its exact fragment is ‘tokens, tokens, head_dim,‘. The important operands are tokens: the value named 'tokens' in this block; head_dim: the value named 'head dim' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.av", "attention counts times binary value", (time_steps, heads, tokens, tokens), (time_steps, heads, tokens, head_dim), tokens, tokens, head_dim, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 200: `"dynamic_u8_mac", "dense tensor tile",` This is continuation line 6 of the 7-line logical statement that begins at source line 195. Its exact fragment is ‘"dynamic_u8_mac", "dense tensor tile",‘. The important operands are dynamic_u8_mac: the value named 'dynamic u8 mac' in this block; dense: the value named 'dense' in this block; tensor: the value named 'tensor' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.av", "attention counts times binary value", (time_steps, heads, tokens, tokens), (time_steps, heads, tokens, head_dim), tokens, tokens, head_dim, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads‘

inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 201: `repetitions=time_steps * heads)` This is continuation line 7 of the 7-line logical statement that begins at source line 195. Its exact fragment is 'repetitions=time_steps * heads)'. The important operands are repetitions: the value named 'repetitions' in this block; time_steps: the value named 'time steps' in this block; heads: the value named 'heads' in this block. Read together, the complete operation is: This calls 'matrix_operator' with ' f"{prefix}.attn.av", "attention counts times binary value", (time_steps, heads, tokens, tokens), (time_steps, heads, tokens, head_dim), tokens, tokens, head_dim, "dynamic_u8_mac", "dense tensor tile", repetitions=time_steps * heads' inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 202: `matrix_operator(` This statement continues through source line 207; the explanation uses the complete expression rather than only the first visual fragment. This calls 'matrix_operator' with ' f"{prefix}.attn.proj", "LIF + projection Conv1d + folded BatchNorm", (time_steps, channels, tokens), (time_steps, channels, 8, 8), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"' inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 203: `f"{prefix}.attn.proj", "LIF + projection Conv1d + folded BatchNorm",` This is continuation line 2 of the 6-line logical statement that begins at source line 202. Its exact fragment is 'f"{prefix}.attn.proj", "LIF + projection Conv1d + folded BatchNorm",'. The important operands are f: the value named 'f' in this block; prefix: the value named 'prefix' in this block; attn: the value named 'attn' in this block. Read together, the complete operation is: This calls 'matrix_operator' with ' f"{prefix}.attn.proj", "LIF + projection Conv1d + folded BatchNorm", (time_steps, channels, tokens), (time_steps, channels, 8, 8), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"' inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 204: `(time_steps, channels, tokens),` This is continuation line 3 of the 6-line logical statement that begins at source line 202. Its exact fragment is '(time_steps, channels, tokens),'. The important operands are time_steps: the value named 'time steps' in this block; channels: the value named 'channels' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This calls 'matrix_operator' with ' f"{prefix}.attn.proj", "LIF + projection Conv1d + folded BatchNorm", (time_steps, channels, tokens), (time_steps, channels, 8, 8), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"' inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 205: `(time_steps, channels, 8, 8),` This is continuation line 4 of the 6-line logical statement that begins at source line 202. Its exact fragment is '(time_steps, channels, 8, 8),'. The important operands are time_steps: the value named 'time steps' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This calls 'matrix_operator' with ' f"{prefix}.attn.proj", "LIF + projection Conv1d + folded BatchNorm", (time_steps, channels, tokens), (time_steps, channels, 8, 8), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"' inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 206: `time_steps * tokens, channels, channels,` This is continuation line 5 of the 6-line logical statement that begins at source line 202. Its exact fragment is 'time_steps * tokens, channels, channels,'. The important operands are time_steps: the value named 'time steps' in this block; tokens: the value named 'tokens' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This calls 'matrix_operator' with ' f"{prefix}.attn.proj", "LIF + projection Conv1d + folded BatchNorm", (time_steps, channels, tokens), (time_steps, channels, 8, 8), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"' inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 207: `"binary_spike_add", "active-index sparse tensor tile")` This is continuation line 6 of the 6-line logical statement that begins at source line 202. Its exact fragment is ‘"binary_spike_add", "active-index sparse tensor tile")‘. The important operands are binary_spike_add: a binary event produced when a neuron crosses its threshold; active: the value named ’active’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.attn.proj", "LIF + projection Conv1d + folded BatchNorm", (time_steps, channels, tokens), (time_steps, channels, 8, 8), time_steps * tokens, channels, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 208: `matrix_operator(` This statement continues through source line 213; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc1", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels * 4, 8, 8), time_steps * tokens, channels, channels * 4, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 209: `f"{prefix}.mlp.fc1", "LIF + 1x1 Conv2d + folded BatchNorm",` This is continuation line 2 of the 6-line logical statement that begins at source line 208. Its exact fragment is ‘f"{prefix}.mlp.fc1", "LIF + 1x1 Conv2d + folded BatchNorm",‘. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; mlp: the value named ’mlp’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc1", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels * 4, 8, 8), time_steps * tokens, channels, channels * 4, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 210: `(time_steps, channels, 8, 8),` This is continuation line 3 of the 6-line logical statement that begins at source line 208. Its exact fragment is ‘(time_steps, channels, 8, 8),‘. The important operands are time_steps: the value named ’time steps’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc1", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels * 4, 8, 8), time_steps * tokens, channels, channels * 4, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 211: `(time_steps, channels * 4, 8, 8),` This is continuation line 4 of the 6-line logical statement that begins at source line 208. Its exact fragment is ‘(time_steps, channels * 4, 8, 8),‘. The important operands are time_steps: the value named ’time steps’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc1", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels * 4, 8, 8), time_steps * tokens, channels, channels * 4, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 212: `time_steps * tokens, channels, channels * 4,` This is continuation line 5 of the 6-line logical statement that begins at source line 208. Its exact fragment is ‘time_steps * tokens, channels, channels * 4,‘. The important operands are time_steps: the value named ’time steps’ in this block; tokens: the value named ’tokens’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc1", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels * 4, 8, 8), time_steps * tokens, channels, channels * 4, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 213: `"binary_spike_add", "active-index sparse tensor tile")` This is continuation line 6 of the 6-line logical statement that begins at source line 208. Its exact fragment is ‘"binary_spike_add", "active-index sparse tensor tile")‘. The important operands are binary_spike_add: a binary event produced when a neuron crosses its threshold;

active: the value named 'active' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc1", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels, 8, 8), (time_steps, channels * 4, 8, 8), time_steps * tokens, channels, channels * 4, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 214: `matrix_operator(` This statement continues through source line 219; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc2", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels * 4, 8, 8), (time_steps, channels, 8, 8), time_steps * tokens, channels * 4, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 215: `f"{prefix}.mlp.fc2", "LIF + 1x1 Conv2d + folded BatchNorm",` This is continuation line 2 of the 6-line logical statement that begins at source line 214. Its exact fragment is ‘f"{prefix}.mlp.fc2", "LIF + 1x1 Conv2d + folded BatchNorm",‘. The important operands are f: the value named 'f' in this block; prefix: the value named 'prefix' in this block; mlp: the value named 'mlp' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc2", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels * 4, 8, 8), (time_steps, channels, 8, 8), time_steps * tokens, channels * 4, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 216: `(time_steps, channels * 4, 8, 8),` This is continuation line 3 of the 6-line logical statement that begins at source line 214. Its exact fragment is ‘(time_steps, channels * 4, 8, 8),‘. The important operands are time_steps: the value named 'time steps' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc2", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels * 4, 8, 8), (time_steps, channels, 8, 8), time_steps * tokens, channels * 4, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 217: `(time_steps, channels, 8, 8),` This is continuation line 4 of the 6-line logical statement that begins at source line 214. Its exact fragment is ‘(time_steps, channels, 8, 8),‘. The important operands are time_steps: the value named 'time steps' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc2", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels * 4, 8, 8), (time_steps, channels, 8, 8), time_steps * tokens, channels * 4, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 218: `time_steps * tokens, channels * 4, channels,` This is continuation line 5 of the 6-line logical statement that begins at source line 214. Its exact fragment is ‘time_steps * tokens, channels * 4, channels,‘. The important operands are time_steps: the value named 'time steps' in this block; tokens: the value named 'tokens' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc2", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels * 4, 8, 8), (time_steps, channels, 8, 8), time_steps * tokens, channels * 4, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 219: `"binary_spike_add", "active-index sparse tensor tile")` This is continuation line 6 of the 6-line logical statement that begins at source line 214. Its exact fragment is ‘"binary_spike_add", "active-index sparse tensor tile")‘. The important operands are binary_spike_add: a binary event produced when a neuron crosses its threshold; active: the value named 'active' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ f"{prefix}.mlp.fc2", "LIF + 1x1 Conv2d + folded BatchNorm", (time_steps, channels * 4, 8, 8), (time_steps, channels, 8, 8), time_steps * tokens, channels *

4, channels, "binary_spike_add", "active-index sparse tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 220: `operators.append(OperatorMapping(` This statement continues through source line 227; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘operators.append‘ with ‘OperatorMapping(name=f"{prefix}.residuals", source_operation="two elementwise residual additions", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(time_steps, channels, 8, 8), hardware_primitive="saturating vector add", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 221: `name=f"{prefix}.residuals",` This is continuation line 2 of the 8-line logical statement that begins at source line 220. Its exact fragment is ‘name=f"{prefix}.residuals",‘. The important operands are name: the value named ’name’ in this block; f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=f"{prefix}.residuals", source_operation="two elementwise residual additions", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(time_steps, channels, 8, 8), hardware_primitive="saturating vector add", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 222: `source_operation="two elementwise residual additions",` This is continuation line 3 of the 8-line logical statement that begins at source line 220. Its exact fragment is ‘source_operation="two elementwise residual additions",‘. The important operands are source_operation: the value named ’source operation’ in this block; two: the value named ’two’ in this block; elementwise: the value named ’elementwise’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=f"{prefix}.residuals", source_operation="two elementwise residual additions", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(time_steps, channels, 8, 8), hardware_primitive="saturating vector add", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 223: `input_shape_per_image=(time_steps, channels, 8, 8),` This is continuation line 4 of the 8-line logical statement that begins at source line 220. Its exact fragment is ‘input_shape_per_image=(time_steps, channels, 8, 8),‘. The important operands are input_shape_per_image: the value named ’input shape per image’ in this block; time_steps: the value named ’time steps’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=f"{prefix}.residuals", source_operation="two elementwise residual additions", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(time_steps, channels, 8, 8), hardware_primitive="saturating vector add", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 224: `output_shape_per_image=(time_steps, channels, 8, 8),` This is continuation line 5 of the 8-line logical statement that begins at source line 220. Its exact fragment is ‘output_shape_per_image=(time_steps, channels, 8, 8),‘. The important operands are output_shape_per_image: the value named ’output shape per image’ in this block; time_steps: the value named ’time steps’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=f"{prefix}.residuals", source_operation="two elementwise residual additions", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(time_steps, channels, 8, 8), hardware_primitive="saturating vector add", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 225: `hardware_primitive="saturating vector add",` This is continuation line 6 of the 8-line logical statement that begins at source line 220. Its exact fragment is ‘hardware_primitive="saturating vector add",‘. The important operands are hardware_primitive: the value named ’hardware primitive’ in this block; saturating: the value named ’saturating’ in this block; vector: the value named ’vector’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=f"{prefix}.residuals", source_operation="two elementwise residual additions", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(time_steps, channels, 8,

8), hardware_primitive="saturating vector add", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 226: `matrix=None,` This is continuation line 7 of the 8-line logical statement that begins at source line 220. Its exact fragment is ‘matrix=None,‘. The important operands are matrix: the value named ’matrix’ in this block; None: the value named ’none’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=f"{prefix}.residuals", source_operation="two elementwise residual additions", input_shape_per_image=(time_ channels, 8, 8), output_shape_per_image=(time_steps, channels, 8, 8), hardware_primitive="saturating vector add", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 227: `))` This is continuation line 8 of the 8-line logical statement that begins at source line 220. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name=f"{prefix}.residuals", source_operation="two elementwise residual additions", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(time_steps, channels, 8, 8), hardware_primitive="saturating vector add", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 229: `operators.append(OperatorMapping(` This statement continues through source line 236; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘operators.append‘ with ‘OperatorMapping(name="global_average_pool", source_operation="mean over 64 tokens then mean over four timesteps", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(channels,), hardware_primitive="vector add followed by exact power-of-two shifts", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 230: `name="global_average_pool",` This is continuation line 2 of the 8-line logical statement that begins at source line 229. Its exact fragment is ‘name="global_average_pool",‘. The important operands are name: the value named ’name’ in this block; global_average_pool: the value named ’global average pool’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name="global_average_pool", source_operation="mean over 64 tokens then mean over four timesteps", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(channels,), hardware_primitive="vector add followed by exact power-of-two shifts", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 231: `source_operation="mean over 64 tokens then mean over four timesteps",` This is continuation line 3 of the 8-line logical statement that begins at source line 229. Its exact fragment is ‘source_operation="mean over 64 tokens then mean over four timesteps",‘. The important operands are source_operation: the value named ’source operation’ in this block; mean: the arithmetic average used by normalization; over: the value named ’over’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name="global_average_pool", source_operation="mean over 64 tokens then mean over four timesteps", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(channels,), hardware_primitive="vector add followed by exact power-of-two shifts", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 232: `input_shape_per_image=(time_steps, channels, 8, 8),` This is continuation line 4 of the 8-line logical statement that begins at source line 229. Its exact fragment is ‘input_shape_per_image=(time_steps, channels, 8, 8),‘. The important operands are input_shape_per_image: the value named ’input shape per image’ in this block; time_steps: the value named ’time steps’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name="global_average_pool", source_operation="mean over 64 tokens then mean over four timesteps", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(channels,), hardware_primitive="vector add followed by exact power-of-two

shifts", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 233: `output_shape_per_image=(channels,),` This is continuation line 5 of the 8-line logical statement that begins at source line 229. Its exact fragment is ‘output_shape_per_image=(channels,),‘. The important operands are output_shape_per_image: the value named ’output shape per image’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name="global_average_pool", source_operation="mean over 64 tokens then mean over four timesteps", input_shape_per_image=(channels, 8, 8), output_shape_per_image=(channels,), hardware_primitive="vector add followed by exact power-of-two shifts", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 234: `hardware_primitive="vector add followed by exact power-of-two shifts",` This is continuation line 6 of the 8-line logical statement that begins at source line 229. Its exact fragment is ‘hardware_primitive="vector add followed by exact power-of-two shifts",‘. The important operands are hardware_primitive: the value named ’hardware primitive’ in this block; vector: the value named ’vector’ in this block; add: the value named ’add’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name="global_average_pool", source_operation="mean over 64 tokens then mean over four timesteps", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(channels,), hardware_primitive="vector add followed by exact power-of-two shifts", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 235: `matrix=None,` This is continuation line 7 of the 8-line logical statement that begins at source line 229. Its exact fragment is ‘matrix=None,‘. The important operands are matrix: the value named ’matrix’ in this block; None: the value named ’none’ in this block. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name="global_average_pool", source_operation="mean over 64 tokens then mean over four timesteps", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(channels,), hardware_primitive="vector add followed by exact power-of-two shifts", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 236: `))` This is continuation line 8 of the 8-line logical statement that begins at source line 229. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘operators.append‘ with ‘OperatorMapping(name="global_average_pool", source_operation="mean over 64 tokens then mean over four timesteps", input_shape_per_image=(time_steps, channels, 8, 8), output_shape_per_image=(channels,), hardware_primitive="vector add followed by exact power-of-two shifts", matrix=None,)‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 237: `matrix_operator(` This statement continues through source line 240; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘matrix_operator‘ with ‘ "head", "Linear classifier", (channels,), (10,), 1, channels, 10, "u8_mac", "dense Dot8 tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 238: `"head", "Linear classifier",` This is continuation line 2 of the 4-line logical statement that begins at source line 237. Its exact fragment is ‘"head", "Linear classifier",‘. The important operands are head: the value named ’head’ in this block; Linear: the value named ’linear’ in this block; classifier: the value named ’classifier’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "head", "Linear classifier", (channels,), (10,), 1, channels, 10, "u8_mac", "dense Dot8 tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 239: `(channels,), (10,), 1, channels, 10,` This is continuation line 3 of the 4-line logical statement that begins at source line 237. Its exact fragment is ‘(channels,), (10,), 1, channels, 10,‘. The important operands are channels: the value named ’channels’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "head", "Linear classifier", (channels,), (10,), 1, channels, 10, "u8_mac", "dense Dot8 tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 240: `"u8_mac", "dense Dot8 tensor tile")` This is continuation line 4 of the 4-line logical statement that begins at source line 237. Its exact fragment is ‘"u8_mac", "dense Dot8 tensor tile")‘. The important operands are u8_mac: the value named ’u8 mac’ in this block; dense: the value named ’dense’ in this block; Dot8: the value named ’dot8’ in this block. Read together, the complete operation is: This calls ‘matrix_operator‘ with ‘ "head", "Linear classifier", (channels,), (10,), 1, channels, 10, "u8_mac", "dense Dot8 tensor tile"‘ inside the function build_operator_manifest, which handles build operator manifest. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 241: `return operators` This returns ‘operators‘ to the caller of the function build_operator_manifest, which handles build operator manifest, ending this invocation. The important operands are operators: the value named ’operators’ in this block.

Line 244: `def generate(checkpoint_path: Path, output_path: Path) -> dict[str, object]:` This starts function generate. Its indented body performs generate as part of the file that derives every model tensor shape, tile schedule, command count, operation count, and traffic total; it runs only when called.

Line 245: `checkpoint = torch.load(checkpoint_path, map_location="cpu")` This assigns ‘torch.load(checkpoint_path, map_location="cpu")‘ to ‘checkpoint ‘ in host memory. Here checkpoint means the value named ’checkpoint’ in this block. The important operands are torch: the value named ’torch’ in this block; load: the value named ’load’ in this block; checkpoint_path: the value named ’checkpoint path’ in this block. Context: the function generate, which handles generate.

Line 246: `state = checkpoint["state_dict"]` This assigns ‘checkpoint["state_dict"]‘ to ‘state ‘ in host memory. Here state means the finite-state-machine phase retained in flip-flops. The important operands are checkpoint: the value named ’checkpoint’ in this block; state_dict: the finite-state-machine phase retained in flip-flops. Context: the function generate, which handles generate.

Line 247: `parameter_tensors = {` This statement continues through source line 255; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ name: { "shape": list(value.shape), "elements": value.numel(), "source_dtype": str(value.dtype).replace("torch.", ""), } for name, value in state.items() if name.endswith(("weight", "bias")) }‘ to ‘parameter_tensors ‘ in host memory. Here parameter_tensors means the value named ’parameter tensors’ in this block. The important operands are name: the value named ’name’ in this block; shape: the value named ’shape’ in this block; list: the value named ’list’ in this block. Context: the function generate, which handles generate.

Line 248: `name: {` This is continuation line 2 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘name: {‘. The important operands are name: the value named ’name’ in this block. Read together, the complete operation is: This assigns ‘{ name: { "shape": list(value.shape), "elements": value.numel(), "source_dtype": str(value.dtype).replace("torch.", ""), } for name, value in state.items() if name.endswith(("weight", "bias")) }‘ to ‘parameter_tensors ‘ in host memory. Here parameter_tensors means the value named ’parameter tensors’ in this block. The important operands are name: the value named ’name’ in this block; shape: the value named ’shape’ in this block; list: the value named ’list’ in this block. Context: the function generate, which handles generate.

Line 249: `"shape": list(value.shape),` This is continuation line 3 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘"shape": list(value.shape),‘. The important operands are shape: the value named ’shape’ in this block; list: the value named ’list’ in this block; value: the value named ’value’ in this block. Read together, the complete operation is: This assigns ‘{ name: { "shape": list(value.shape), "elements": value.numel(), "source_dtype": str(value.dtype).replace("torch.", ""), } for name, value in state.items() if name.endswith(("weight",

"bias")) }‘ to ‘parameter_tensors ‘ in host memory. Here parameter_tensors means the value named ’parameter tensors’ in this block. The important operands are name: the value named ’name’ in this block; shape: the value named ’shape’ in this block; list: the value named ’list’ in this block. Context: the function generate, which handles generate.

Line 250: `"elements": value.numel(),` This is continuation line 4 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘"elements": value.numel(),‘. The important operands are elements: the value named ’elements’ in this block; value: the value named ’value’ in this block; numel: the value named ’numel’ in this block. Read together, the complete operation is: This assigns ‘{ name: { "shape": list(value.shape), "elements": value.numel(), "source_dtype": str(value.dtype).replace("torch.", ""), } for name, value in state.items() if name.endswith(("weight", "bias")) }‘ to ‘parameter_tensors ‘ in host memory. Here parameter_tensors means the value named ’parameter tensors’ in this block. The important operands are name: the value named ’name’ in this block; shape: the value named ’shape’ in this block; list: the value named ’list’ in this block. Context: the function generate, which handles generate.

Line 251: `"source_dtype": str(value.dtype).replace("torch.", ""),` This is continuation line 5 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘"source_dtype": str(value.dtype).replace("torch.", ""),‘. The important operands are source_dtype: the value named ’source dtype’ in this block; str: the value named ’str’ in this block; value: the value named ’value’ in this block. Read together, the complete operation is: This assigns ‘{ name: { "shape": list(value.shape), "elements": value.numel(), "source_dtype": str(value.dtype).replace("torch.", ""), } for name, value in state.items() if name.endswith(("weight", "bias")) }‘ to ‘parameter_tensors ‘ in host memory. Here parameter_tensors means the value named ’parameter tensors’ in this block. The important operands are name: the value named ’name’ in this block; shape: the value named ’shape’ in this block; list: the value named ’list’ in this block. Context: the function generate, which handles generate.

Line 252: `}` This is continuation line 6 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ name: { "shape": list(value.shape), "elements": value.numel(), "source_dtype": str(value.dtype).replace("torch.", ""), } for name, value in state.items() if name.endswith(("weight", "bias")) }‘ to ‘parameter_tensors ‘ in host memory. Here parameter_tensors means the value named ’parameter tensors’ in this block. The important operands are name: the value named ’name’ in this block; shape: the value named ’shape’ in this block; list: the value named ’list’ in this block. Context: the function generate, which handles generate.

Line 253: `for name, value in state.items()` This is continuation line 7 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘for name, value in state.items()‘. The important operands are for: the value named ’for’ in this block; name: the value named ’name’ in this block; value: the value named ’value’ in this block. Read together, the complete operation is: This assigns ‘{ name: { "shape": list(value.shape), "elements": value.numel(), "source_dtype": str(value.dtype).replace("torch.", ""), } for name, value in state.items() if name.endswith(("weight", "bias")) }‘ to ‘parameter_tensors ‘ in host memory. Here parameter_tensors means the value named ’parameter tensors’ in this block. The important operands are name: the value named ’name’ in this block; shape: the value named ’shape’ in this block; list: the value named ’list’ in this block. Context: the function generate, which handles generate.

Line 254: `if name.endswith(("weight", "bias"))` This is continuation line 8 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘if name.endswith(("weight", "bias"))‘. The important operands are name: the value named ’name’ in this block; endswith: the value named ’endswith’ in this block; weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘{ name: { "shape": list(value.shape), "elements": value.numel(), "source_dtype": str(value.dtype).replace("torch.", ""), } for name, value in state.items() if name.endswith(("weight", "bias")) }‘ to ‘parameter_tensors ‘ in host memory. Here parameter_tensors means the value named ’parameter tensors’ in this block. The important operands are name: the value named ’name’ in this block; shape: the value named ’shape’ in this block; list: the value named ’list’ in this block. Context: the function generate, which handles generate.

Line 255: `}` This is continuation line 9 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ name: { "shape": list(value.shape), "elements": value.numel(), "source_dtype": str(value.dtype).replace("torch.", ""), } for name, value in state.items() if name.endswith(("weight", "bias")) }‘ to ‘parameter_tensors ‘ in host memory. Here parameter_tensors means the value named ’parameter tensors’ in this block. The

important operands are name: the value named 'name' in this block; shape: the value named 'shape' in this block; list: the value named 'list' in this block. Context: the function generate, which handles generate.

Line 256: `operators = build_operator_manifest()` This assigns 'build_operator_manifest()' to 'operators ' in host memory. Here operators means the value named 'operators' in this block. The important operands are build_operator_manifest the value named 'build operator manifest' in this block. Context: the function generate, which handles generate.

Line 257: `matrix_operators = [item.matrix for item in operators if item.matrix is not None]` This assigns '[item.matrix for item in operators if item.matrix is not None]' to 'matrix_operators ' in host memory. Here matrix_operators means the value named 'matrix operators' in this block. The important operands are item: the value named 'item' in this block; matrix: the value named 'matrix' in this block; for: the value named 'for' in this block. Context: the function generate, which handles generate.

Line 258: `total_macs = sum(item.mac_equivalents for item in matrix_operators)` This assigns 'sum(item.mac_equivale for item in matrix_operators)' to 'total_macs ' in host memory. Here total_macs means the value named 'total macs' in this block. The important operands are sum: the value named 'sum' in this block; item: the value named 'item' in this block; mac_equivalents: the value named 'mac equivalents' in this block. Context: the function generate, which handles generate.

Line 259: `total_commands = sum(item.commands for item in matrix_operators)` This assigns 'sum(item.commands for item in matrix_operators)' to 'total_commands ' in host memory. Here total_commands means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are sum: the value named 'sum' in this block; item: the value named 'item' in this block; commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function generate, which handles generate.

Line 260: `total_input_bytes = sum(item.input_stream_bytes for item in matrix_operators)` This assigns 'sum(item.input_stream_bytes for item in matrix_operators)' to 'total_input_bytes ' in host memory. Here total_input_bytes means the value named 'total input bytes' in this block. The important operands are sum: the value named 'sum' in this block; item: the value named 'item' in this block; input_stream_bytes: the value named 'input stream bytes' in this block. Context: the function generate, which handles generate.

Line 261: `total_output_bytes = sum(item.output_stream_bytes for item in matrix_operators)` This assigns 'sum(item.output_stream_bytes for item in matrix_operators)' to 'total_output_bytes ' in host memory. Here total_output_bytes means the value named 'total output bytes' in this block. The important operands are sum: the value named 'sum' in this block; item: the value named 'item' in this block; output_stream_bytes: the value named 'output stream bytes' in this block. Context: the function generate, which handles generate.

Line 262: `trainable_parameters = sum(` This statement continues through source line 265; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sum(value.numel() for name, value in state.items() if name.endswith(("weight", "bias")))' to 'trainable_parameters ' in host memory. Here trainable_parameters means the value named 'trainable parameters' in this block. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; numel: the value named 'numel' in this block. Context: the function generate, which handles generate.

Line 263: `value.numel() for name, value in state.items()` This is continuation line 2 of the 4-line logical statement that begins at source line 262. Its exact fragment is 'value.numel() for name, value in state.items()'. The important operands are value: the value named 'value' in this block; numel: the value named 'numel' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This assigns 'sum(value.numel() for name, value in state.items() if name.endswith(("weight", "bias")))' to 'trainable_parameters ' in host memory. Here trainable_parameters means the value named 'trainable parameters' in this block. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; numel: the value named 'numel' in this block. Context: the function generate, which handles generate.

Line 264: `if name.endswith(("weight", "bias"))` This is continuation line 3 of the 4-line logical statement that begins at source line 262. Its exact fragment is ‘if name.endswith(("weight", "bias"))‘. The important operands are name: the value named ’name’ in this block; endswith: the value named ’endswith’ in this block; weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘sum(value.numel() for name, value in state.items() if name.endswith(("weight", "bias")))‘ to ‘trainable_parameters ‘ in host memory. Here trainable_parameters means the value named ’trainable parameters’ in this block. The important operands are sum: the value named ’sum’ in this block; value: the value named ’value’ in this block; numel: the value named ’numel’ in this block. Context: the function generate, which handles generate.

Line 265: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 262. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘sum(value.numel() for name, value in state.items() if name.endswith(("weight", "bias")))‘ to ‘trainable_parameters ‘ in host memory. Here trainable_parameters means the value named ’trainable parameters’ in this block. The important operands are sum: the value named ’sum’ in this block; value: the value named ’value’ in this block; numel: the value named ’numel’ in this block. Context: the function generate, which handles generate.

Line 266: `int8_weight_elements = sum(` This statement continues through source line 270; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sum(value.numel() for name, value in state.items() if name.endswith("weight") and not ("norm1" in name or "norm2" in name or "_bn." in name))‘ to ‘int8_weight_elements ‘ in host memory. Here int8_weight_elements means a learned coefficient multiplied by an activation. The important operands are sum: the value named ’sum’ in this block; value: the value named ’value’ in this block; numel: the value named ’numel’ in this block. Context: the function generate, which handles generate.

Line 267: `value.numel() for name, value in state.items()` This is continuation line 2 of the 5-line logical statement that begins at source line 266. Its exact fragment is ‘value.numel() for name, value in state.items()‘. The important operands are value: the value named ’value’ in this block; numel: the value named ’numel’ in this block; for: the value named ’for’ in this block. Read together, the complete operation is: This assigns ‘sum(value.numel() for name, value in state.items() if name.endswith("weight") and not ("norm1" in name or "norm2" in name or "_bn." in name))‘ to ‘int8_weight_elements ‘ in host memory. Here int8_weight_elements means a learned coefficient multiplied by an activation. The important operands are sum: the value named ’sum’ in this block; value: the value named ’value’ in this block; numel: the value named ’numel’ in this block. Context: the function generate, which handles generate.

Line 268: `if name.endswith("weight")` This is continuation line 3 of the 5-line logical statement that begins at source line 266. Its exact fragment is ‘if name.endswith("weight")‘. The important operands are name: the value named ’name’ in this block; endswith: the value named ’endswith’ in this block; weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘sum(value.numel() for name, value in state.items() if name.endswith("weight") and not ("norm1" in name or "norm2" in name or "_bn." in name))‘ to ‘int8_weight_elements ‘ in host memory. Here int8_weight_elements means a learned coefficient multiplied by an activation. The important operands are sum: the value named ’sum’ in this block; value: the value named ’value’ in this block; numel: the value named ’numel’ in this block. Context: the function generate, which handles generate.

Line 269: `and not ("norm1" in name or "norm2" in name or "_bn." in name)` This is continuation line 4 of the 5-line logical statement that begins at source line 266. Its exact fragment is ‘and not ("norm1" in name or "norm2" in name or "_bn." in name)‘. The important operands are and: the value named ’and’ in this block; not: the value named ’not’ in this block; norm1: the value named ’norm1’ in this block. Read together, the complete operation is: This assigns ‘sum(value.numel() for name, value in state.items() if name.endswith("weight") and not ("norm1" in name or "norm2" in name or "_bn." in name))‘ to ‘int8_weight_elements ‘ in host memory. Here int8_weight_elements means a learned coefficient multiplied by an activation. The important operands are sum: the value named ’sum’ in this block; value: the value named ’value’ in this block; numel: the value named ’numel’ in this block. Context: the function generate, which handles generate.

Line 270: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 266. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘sum(value.numel() for name, value in state.items() if name.endswith("weight") and not ("norm1" in name or "norm2" in name or "_bn." in name))‘ to ‘int8_weight_elements ‘ in host memory. Here int8_weight_elements means a learned coefficient multiplied by an activation. The important operands are sum: the

value named 'sum' in this block; value: the value named 'value' in this block; numel: the value named 'numel' in this block. Context: the function generate, which handles generate.

Line 271: `folded_bias_channels = sum(` This statement continues through source line 274; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sum(value.shape[0] for name, value in state.items() if name.endswith("running_mean")) + state["head.bias"].numel()' to 'folded_bias_channels ' in host memory. Here folded_bias_channels means an additive learned offset applied after accumulation. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function generate, which handles generate.

Line 272: `value.shape[0] for name, value in state.items()` This is continuation line 2 of the 4-line logical statement that begins at source line 271. Its exact fragment is 'value.shape[0] for name, value in state.items()'. The important operands are value: the value named 'value' in this block; shape: the value named 'shape' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This assigns 'sum(value.shape[0] for name, value in state.items() if name.endswith("running_mean")) + state["head.bias"].numel()' to 'folded_bias_channels ' in host memory. Here folded_bias_channels means an additive learned offset applied after accumulation. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function generate, which handles generate.

Line 273: `if name.endswith("running_mean")` This is continuation line 3 of the 4-line logical statement that begins at source line 271. Its exact fragment is 'if name.endswith("running_mean")'. The important operands are name: the value named 'name' in this block; endswith: the value named 'endswith' in this block; running_mean: the arithmetic average used by normalization. Read together, the complete operation is: This assigns 'sum(value.shape[0] for name, value in state.items() if name.endswith("running_mean")) + state["head.bias"].numel()' to 'folded_bias_channels ' in host memory. Here folded_bias_channels means an additive learned offset applied after accumulation. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function generate, which handles generate.

Line 274: `) + state["head.bias"].numel()` This is continuation line 4 of the 4-line logical statement that begins at source line 271. Its exact fragment is ') + state["head.bias"].numel()'. The important operands are state: the finite-state-machine phase retained in flip-flops; head: the value named 'head' in this block; bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'sum(value.shape[0] for name, value in state.items() if name.endswith("running_mean")) + state["head.bias"].numel()' to 'folded_bias_channels ' in host memory. Here folded_bias_channels means an additive learned offset applied after accumulation. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function generate, which handles generate.

Line 275: `report: dict[str, object] = {` This statement continues through source line 316; the explanation uses the complete expression rather than only the first visual fragment. This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 276: `"format": "neuraedge-spikingformer-execution-manifest-v1",` This is continuation line 2 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"format": "neuraedge-spikingformer-execution-manifest-v1",‘. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint’s LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function generate, which handles generate.

Line 277: `"checkpoint": str(checkpoint_path.resolve()),` This is continuation line 3 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"checkpoint": str(checkpoint_path.resolve()),‘. The important operands are checkpoint: the value named ’checkpoint’ in this block; str: the value named ’str’ in this block; checkpoint_path: the value named ’checkpoint path’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint’s LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named ’report’ in this block. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block. Context: the function generate, which handles generate.

Line 278: `"checkpoint_epoch": int(checkpoint.get("epoch", -1)),` This is continuation line 4 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"checkpoint_epoch": int(checkpoint.get("epoch", -1)),‘. The important operands are checkpoint_epoch: the value named ’checkpoint epoch’ in this block; checkpoint: the value named ’checkpoint’ in this block; get: the value named ’get’ in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command":

MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 279: `"checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))),` This is continuation line 5 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))),‘. The important operands are checkpoint_metric_percent: the value named 'checkpoint metric percent' in this block; float: the value named 'float' in this block; checkpoint: the value named 'checkpoint' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 280: `"model": {` This is continuation line 6 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"model": {‘. The important operands are model: the value named 'model' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function

generate, which handles generate.

Line 281: `"time_steps": 4,` This is continuation line 7 of the 42-line logical statement that begins at source line 275. Its exact fragment is '"time_steps": 4,'. The important operands are time_steps: the value named 'time steps' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 282: `"embed_dim": 384,` This is continuation line 8 of the 42-line logical statement that begins at source line 275. Its exact fragment is '"embed_dim": 384,'. The important operands are embed_dim: the value named 'embed dim' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 283: `"heads": 12,` This is continuation line 9 of the 42-line logical statement that begins at source line 275. Its exact fragment is '"heads": 12,'. The important operands are heads: the value named 'heads' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la

total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 284: `"head_dim": 32,` This is continuation line 10 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"head_dim": 32,‘. The important operands are head_dim: the value named 'head dim' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 285: `"transformer_blocks": 4,` This is continuation line 11 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"transformer_blocks": 4,‘. The important operands are transformer_blocks: the value named 'transformer blocks' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 286: `"tokens": 64,` This is continuation line 12 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"tokens": 64,‘. The important operands are tokens: the value named 'tokens' in this block. Read

together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter
trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per
MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la
total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 287: `"classes": 10,` This is continuation line 13 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"classes": 10,‘. The important operands are classes: the value named 'classes' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el
"mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_
total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 288: `"trainable_parameter_elements": trainable_parameters,` This is continuation line 14 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"trainable_parameter_elements": trainable_parameters,‘. The important operands are trainable_parameter_elements: the value named 'trainable parameter elements' in this block; trainable_parameters: the value named 'trainable parameters' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi
float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per
MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la
total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike

layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 289: `"source_parameter_bytes_fp32": trainable_parameters * 4,` This is continuation line 15 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"source_parameter_bytes_fp32": trainable_parameters * 4,‘. The important operands are source_parameter_bytes_fp32: the value named 'source parameter bytes fp32' in this block; trainable_parameters: the value named 'trainable parameters' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 290: `"mapped_int8_weight_bytes": int8_weight_elements,` This is continuation line 16 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"mapped_int8_weight_bytes": int8_weight_elements,‘. The important operands are mapped_int8_weight_bytes: a learned coefficient multiplied by an activation; int8_weight_elements: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 291: `"mapped_int32_folded_bias_bytes": folded_bias_channels * 4,` This is continuation line 17 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"mapped_int32_folded_bias_bytes":

folded_bias_channels * 4,‘. The important operands are mapped_int32_folded_bias_bytes: an additive learned offset applied after accumulation; folded_bias_channels: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 292: `},` This is continuation line 18 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘},‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 293: `"accelerator": {` This is continuation line 19 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"accelerator": {‘. The important operands are accelerator: the value named 'accelerator' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called

by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 294: `"row_tile": ROW_TILE,` This is continuation line 20 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"row_tile": ROW_TILE,‘. The important operands are row_tile: one output-row position processed by the tensor tile; ROW_TILE: one output-row position processed by the tensor tile. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 295: `"column_tile": COLUMN_TILE,` This is continuation line 21 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"column_tile": COLUMN_TILE,‘. The important operands are column_tile: one output-feature position processed in parallel; COLUMN_TILE: one output-feature position processed in parallel. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 296: `"maximum_reduction_per_command": MAX_REDUCTION,` This is continuation line 22 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"maximum_reduction_per_command": MAX_REDUCTION,‘. The important operands are maximum_reduction_per_command: the 128-bit instruction selecting an engine, shape,

mode, and numeric parameters; MAX_REDUCTION: the value named 'max reduction' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 297: `"packed_u8_macs_per_cycle": 128,` This is continuation line 23 of the 42-line logical statement that begins at source line 275. Its exact fragment is '"packed_u8_macs_per_cycle": 128,'. The important operands are packed_u8_macs_per_cycle: the value named 'packed u8 macs per cycle' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 298: `"target_clock_hz": 200_000_000,` This is continuation line 24 of the 42-line logical statement that begins at source line 275. Its exact fragment is '"target_clock_hz": 200_000_000,'. The important operands are target_clock_hz: the value named 'target clock hz' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's

LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 299: `"peak_dense_gmac_per_second": 25.6,` This is continuation line 25 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"peak_dense_gmac_per_second": 25.6,‘. The important operands are peak_dense_gmac_per_second: the value named 'peak dense gmac per second' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 300: `},` This is continuation line 26 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘},‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 301: `"per_image_totals": {` This is continuation line 27 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"per_image_totals": {‘. The important operands are per_image_totals: the value named 'per image totals' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-

spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 302: `"mac_equivalents_after_exact_temporal_hoist": total_macs,` This is continuation line 28 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"mac_equivalents_after_exact_temporal_hoist": total_macs,‘. The important operands are mac_equivalents_after_exact_temporal_hoist: the value named 'mac equivalents after exact temporal hoist' in this block; total_macs: the value named 'total macs' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 303: `"tensor_commands": total_commands,` This is continuation line 29 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"tensor_commands": total_commands,‘. The important operands are tensor_commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; total_commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's

LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 304: `"input_stream_bytes_without_inter_layer_spill": total_input_bytes,` This is continuation line 30 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"input_stream_bytes_without_inter_layer_s total_input_bytes,‘. The important operands are input_stream_bytes_without_inter_layer_spill: the value named 'input stream bytes without inter layer spill' in this block; total_input_bytes: the value named 'total input bytes' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 305: `"output_stream_bytes_without_inter_layer_spill": total_output_bytes,` This is continuation line 31 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"output_stream_bytes_without_inter_layer_ total_output_bytes,‘. The important operands are output_stream_bytes_without_inter_layer_spill: the value named 'output stream bytes without inter layer spill' in this block; total_output_bytes: the value named 'total output bytes' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 306: `},` This is continuation line 32 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘},‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 307: `"operators": [asdict(item) for item in operators],` This is continuation line 33 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"operators": [asdict(item) for item in operators],‘. The important operands are operators: the value named 'operators' in this block; asdict: the value named 'asdict' in this block; item: the value named 'item' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 308: `"parameter_tensors": parameter_tensors,` This is continuation line 34 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"parameter_tensors": parameter_tensors,‘. The important operands are parameter_tensors: the value named 'parameter tensors' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "se-

mantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 309: `"semantic_notes": [` This is continuation line 35 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"semantic_notes": [‘. The important operands are semantic_notes: the value named 'semantic notes' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 310: `"BatchNorm is folded into the preceding learned projection in evaluation mode.",` This is continuation line 36 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"BatchNorm is folded into the preceding learned projection in evaluation mode.",‘. The important operands are BatchNorm: the value named 'batch norm' in this block; is: the value named 'is' in this block; folded: the value named 'folded' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 311: `"The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream f...` This is continuation line 37 of the 42-line logical statement that begins at source line 275. Its exact fragment is '"The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.",'. The important operands are The: the value named 'the' in this block; checkpoint: the value named 'checkpoint' in this block; s: the value named 's' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 312: `"Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only fo...` This is continuation line 38 of the 42-line logical statement that begins at source line 275. Its exact fragment is '"Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.",'. The important operands are Binary: the value named 'binary' in this block; spike: a binary event produced when a neuron crosses its threshold; layers: the value named 'layers' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_el "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_ total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_layer_spill": total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }' to 'report' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 313: `"QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in t...` This is continuation line 39 of the 42-line logical statement that begins at source line 275. Its exact fragment is '"QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.",'. The important operands are QK: the value named 'qk' in this block; and: the value named 'and' in this block; attention: the value named 'attention' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoint.get("metric", float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64,

"classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_per_command": MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 314: `"DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determi...` This is continuation line 40 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘"DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",‘. The important operands are DDR: the value named 'ddr' in this block; traffic: the value named 'traffic' in this block; is: the value named 'is' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 315: `],` This is continuation line 41 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘],‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path

in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 316: `}` This is continuation line 42 of the 42-line logical statement that begins at source line 275. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-spikingformer-execution-manifest-v1", "checkpoint": str(checkpoint_path.resolve()), "checkpoint_epoch": int(checkpoint.get("epoch", -1)), "checkpoint_metric_percent": float(checkpoi float("nan"))), "model": { "time_steps": 4, "embed_dim": 384, "heads": 12, "head_dim": 32, "transformer_blocks": 4, "tokens": 64, "classes": 10, "trainable_parameter_elements": trainable_parameters, "source_parameter_bytes_fp32": trainable_parameters * 4, "mapped_int8_weight_bytes": int8_weight_elements, "mapped_int32_folded_bias_bytes": folded_bias_channels * 4, }, "accelerator": { "row_tile": ROW_TILE, "column_tile": COLUMN_TILE, "maximum_reduction_pe MAX_REDUCTION, "packed_u8_macs_per_cycle": 128, "target_clock_hz": 200_000_000, "peak_dense_gmac_per_second": 25.6, }, "per_image_totals": { "mac_equivalents_after_exact_temporal_hoist": total_macs, "tensor_commands": total_commands, "input_stream_bytes_without_inter_layer_spill": total_input_bytes, "output_stream_bytes_without_inter_la total_output_bytes, }, "operators": [asdict(item) for item in operators], "parameter_tensors": parameter_tensors, "semantic_notes": ["BatchNorm is folded into the preceding learned projection in evaluation mode.", "The checkpoint's LayerNorm parameters are retained for strict loading but are not called by its upstream forward graph.", "Binary-spike layers use signed int8 weights and addition/subtraction; MAC-equivalents are reported only for comparison with dense execution.", "QK and attention-value products have two dynamic operands and therefore use the exact dynamic-u8 path in this manifest.", "DDR traffic is not inferred from the summed tile streams because the final on-chip buffer schedule determines which operands are retained or spilled.",], }‘ to ‘report‘ in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Context: the function generate, which handles generate.

Line 317: `output_path.parent.mkdir(parents=True, exist_ok=True)` This calls ‘output_path.parent.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the function generate, which handles generate. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 318: `output_path.write_text(json.dumps(report, indent=2) + "\n", encoding="ascii")` This calls ‘output_path.write_text‘ with ‘json.dumps(report, indent=2) + "\n", encoding="ascii"‘ inside the function generate, which handles generate. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 319: `return report` This returns ‘report‘ to the caller of the function generate, which handles generate, ending this invocation. The important operands are report: the value named 'report' in this block.

Line 322: `def parse_arguments() -> argparse.Namespace:` This starts function parse_arguments. Its indented body performs parse arguments as part of the file that derives every model tensor shape, tile schedule, command count, operation count, and traffic total; it runs only when called.

Line 323: `parser = argparse.ArgumentParser()` This assigns ‘argparse.ArgumentParser()‘ to ‘parser ‘ in host memory. Here parser means the value named 'parser' in this block. The important operands are argparse: the value named 'argparse' in this block; ArgumentParser: the value named 'argument parser' in this block. Context: the function parse_arguments, which handles parse arguments.

Line 324: `parser.add_argument("-checkpoint", required=True, type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 325: `parser.add_argument("-output", required=True, type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 326: `return parser.parse_args()` This returns ‘parser.parse_args()‘ to the caller of the function parse_arguments, which handles parse arguments, ending this invocation. The important operands are parser: the value named ’parser’ in this block; parse_args: the value named ’parse args’ in this block.

Line 329: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 330: `arguments = parse_arguments()` This assigns ‘parse_arguments()‘ to ‘arguments ‘ in host memory. Here arguments means the value named ’arguments’ in this block. The important operands are parse_arguments: the value named ’parse arguments’ in this block. Context: the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 331: `result = generate(arguments.checkpoint.resolve(), arguments.output.resolve())` This assigns ‘generate(arguments.checkpoint.resolve(), arguments.output.resolve())‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are generate: the value named ’generate’ in this block; arguments: the value named ’arguments’ in this block; checkpoint: the value named ’checkpoint’ in this block. Context: the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

Line 332: `print(json.dumps(result["per_image_totals"], indent=2))` This calls ‘print‘ with ‘json.dumps(result["per_imag indent=2)‘ inside the file whose role is to derives every model tensor shape, tile schedule, command count, operation count, and traffic total. The call advances the host-side job that derives every model tensor shape, tile schedule, command count, operation count, and traffic total.

32 software/generate_tensor_tile_fixture.py

Role in the complete system: builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

32.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: FixtureCase. Functions: pack_bytes, make_dense_outer, make_dense_dot8, make_binary, make_ternary, command_word, write_hex, main. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

32.2 Complete source

```
"""Generate deterministic, independently computed tensor-tile RTL fixtures."""

from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path

import numpy as np

from neuraedge_golden import (
    ComputeMode,
    binary_weighted_gemm,
    gemm_u8_zero_point,
    ternary_weighted_gemm,
)


ROW_LANES = 8
COLUMN_LANES = 16


@dataclass
class FixtureCase:
    name: str
    mode: ComputeMode
    rows: int
    columns: int
    activation_zero_point: int
    weight_zero_point: int
    activation_words: list[int]
    weight_words: list[int]
    expected: np.ndarray


def pack_bytes(values: np.ndarray | list[int]) -> int:
    packed = 0
    for index, value in enumerate(np.asarray(values).reshape(-1)):
        packed |= (int(value) & 0xFF) << (8 * index)
    return packed


def make_dense_outer(rng: np.random.Generator) -> FixtureCase:
    rows, columns, reduction = 8, 16, 19
    za, zb = 117, 139
    a = rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)
    b = rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)
    a[:, [2, 7, 14]] = za
    expected = gemm_u8_zero_point(a, b, za, zb)
    a_words = [pack_bytes(a[:, k]) for k in range(reduction)]
    b_words = [pack_bytes(b[k]) for k in range(reduction)]
    return FixtureCase("dense_outer_8x16x19", ComputeMode.DENSE_OUTER, rows, columns, za, zb, a_words, b_words, expected)
```

```

def make_dense_dot8(rng: np.random.Generator) -> FixtureCase:
    rows, columns, reduction = 1, 13, 21
    padded_reduction = 24
    za, zb = 128, 128
    a = np.full((rows, padded_reduction), za, dtype=np.uint8)
    b = np.full((padded_reduction, columns), zb, dtype=np.uint8)
    a[:, :reduction] = rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)
    b[:reduction] = rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)
    a[:, 8:16] = za
    expected = gemm_u8_zero_point(a, b, za, zb)
    a_words: list[int] = []
    b_words: list[int] = []
    for chunk_start in range(0, padded_reduction, ROW_LANES):
        a_words.append(pack_bytes(a[0, chunk_start : chunk_start + ROW_LANES]))
        packed_weights = 0
        for column in range(COLUMN_LANES):
            lane_values = (
                b[chunk_start : chunk_start + ROW_LANES, column]
                if column < columns
                else np.full(ROW_LANES, zb, dtype=np.uint8)
            )
            packed_weights |= pack_bytes(lane_values) << (64 * column)
        b_words.append(packed_weights)
    return FixtureCase("dense_dot8_1x13x24", ComputeMode.DENSE_DOT8, rows, columns, za, zb, a_words, b_words, expected)


def make_binary(rng: np.random.Generator) -> FixtureCase:
    rows, columns, reduction = 5, 11, 17
    spikes = rng.integers(0, 2, size=(rows, reduction), dtype=np.int8)
    weights = rng.integers(-128, 128, size=(reduction, columns), dtype=np.int16)
    spikes[:, [4, 12]] = 0
    expected = binary_weighted_gemm(spikes, weights)
    a_words: list[int] = []
    b_words: list[int] = []
    for k in range(reduction):
        a_padded = np.zeros(ROW_LANES, dtype=np.int16)
        b_padded = np.zeros(COLUMN_LANES, dtype=np.int16)
        a_padded[:rows] = spikes[:, k]
        b_padded[:columns] = weights[k]
        a_words.append(pack_bytes(a_padded))
        b_words.append(pack_bytes(b_padded))
    return FixtureCase("binary_weighted_5x11x17", ComputeMode.BINARY_WEIGHTED, rows, columns, 0, 0, a_words, b_words, expected)


def make_ternary(rng: np.random.Generator) -> FixtureCase:
    rows, columns, reduction = 7, 9, 15
    spikes = rng.integers(-1, 2, size=(rows, reduction), dtype=np.int8)
    weights = rng.integers(-128, 128, size=(reduction, columns), dtype=np.int16)
    spikes[:, [1, 10]] = 0
    expected = ternary_weighted_gemm(spikes, weights)
    a_words: list[int] = []
    b_words: list[int] = []
    for k in range(reduction):
        a_padded = np.zeros(ROW_LANES, dtype=np.int16)
        b_padded = np.zeros(COLUMN_LANES, dtype=np.int16)
        a_padded[:rows] = spikes[:, k]
        b_padded[:columns] = weights[k]
        a_words.append(pack_bytes(a_padded))
        b_words.append(pack_bytes(b_padded))
    return FixtureCase("ternary_weighted_7x9x15", ComputeMode.TERNARY_WEIGHTED, rows, columns, 0, 0, a_words, b_words, expected)


def command_word(case: FixtureCase) -> int:
    issues = len(case.activation_words)
    return (
        int(case.mode)
        | (case.rows << 4)
        | (case.columns << 8)
        | (issues << 14)
        | (case.activation_zero_point << 26)
        | (case.weight_zero_point << 34)
    )


def write_hex(path: Path, values: list[int], width_bits: int) -> None:
    digits = (width_bits + 3) // 4
    mask = (1 << width_bits) - 1
    path.write_text("".join(f"{value & mask:0{digits}x}\n" for value in values), encoding="ascii")


def main() -> None:
    fixture_dir = Path(__file__).resolve().parents[1] / "tb" / "fixtures"
    fixture_dir.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(0x4E455552)
    cases = [make_dense_outer(rng), make_dense_dot8(rng), make_binary(rng), make_ternary(rng)]
```

```
140
141     commands = [command_word(case) for case in cases]
142     activation_words = [word for case in cases for word in case.activation_words]
143     weight_words = [word for case in cases for word in case.weight_words]
144     expected_values = [int(value) for case in cases for value in case.expected.reshape(-1)]
145
146     write_hex(fixture_dir / "tensor_cases.mem", commands, 64)
147     write_hex(fixture_dir / "tensor_activations.mem", activation_words, 64)
148     write_hex(fixture_dir / "tensor_weights.mem", weight_words, COLUMN_LANES * ROW_LANES * 8)
149     write_hex(fixture_dir / "tensor_expected.mem", expected_values, 64)
150     (fixture_dir / "tensor_tile_params.svh").write_text(
151         "`define NEURAEDGE_TENSOR_CASES %d\n"
152         "`define NEURAEDGE_TENSOR_ISSUES %d\n"
153         "`define NEURAEDGE_TENSOR_RESULTS %d\n"
154         "`define NEURAEDGE_COLUMN_LANES %d\n"
155         % (len(cases), len(activation_words), len(expected_values), COLUMN_LANES),
156         encoding="ascii",
157     )
158
159     manifest = {
160         "seed": "0x4E455552",
161         "case_count": len(cases),
162         "issue_count": len(activation_words),
163         "result_count": len(expected_values),
164         "cases": [
165             {
166                 "name": case.name,
167                 "mode": case.mode.name,
168                 "rows": case.rows,
169                 "columns": case.columns,
170                 "issues": len(case.activation_words),
171                 "activation_zero_point": case.activation_zero_point,
172                 "weight_zero_point": case.weight_zero_point,
173                 "skipped_zero_issues": sum(word == pack_bytes([case.activation_zero_point] * ROW_LANES) for word in case.activation_words)
174                 if case.mode in (ComputeMode.DENSE_OUTER, ComputeMode.DENSE_DOT8)
175                 else sum(word == 0 for word in case.activation_words),
176             }
177             for case in cases
178         ],
179     }
180     (fixture_dir / "tensor_fixture_manifest.json").write_text(json.dumps(manifest, indent=2), encoding="utf-8")
181     print(json.dumps(manifest, indent=2))
182
183
184 if __name__ == "__main__":
185     main()
```

32.3 Line annotations

Line 1: `"""Generate deterministic, independently computed tensor-tile RTL fixtures."""` This is executable documentation attached to the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import json` This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from pathlib import Path` This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import numpy as np` This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `from neuraedge_golden import (` This statement continues through source line 16; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `ComputeMode,` This is continuation line 2 of the 6-line logical statement that begins at source line 11. Its exact fragment is 'ComputeMode,'. The important operands are ComputeMode: the value named 'compute mode' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `binary_weighted_gemm,` This is continuation line 3 of the 6-line logical statement that begins at source line 11. Its exact fragment is 'binary_weighted_gemm,'. The important operands are binary_weighted_gemm: a learned coefficient multiplied by an activation. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `gemm_u8_zero_point,` This is continuation line 4 of the 6-line logical statement that begins at source line 11. Its exact fragment is 'gemm_u8_zero_point,'. The important operands are gemm_u8_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `ternary_weighted_gemm,` This is continuation line 5 of the 6-line logical statement that begins at source line 11. Its exact fragment is 'ternary_weighted_gemm,'. The important operands are ternary_weighted_gemm: a learned coefficient multiplied by an activation. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 11. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `ROW_LANES = 8` This assigns '8' to 'ROW_LANES ' in host memory. Here ROW_LANES means one parallel arithmetic element within a vector or tensor datapath. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 20: `COLUMN_LANES = 16` This assigns '16' to 'COLUMN_LANES ' in host memory. Here COLUMN_LANES means one parallel arithmetic element within a vector or tensor datapath. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 23: `@dataclass` This decorator changes how the following class or function is constructed. '@dataclass' is applied at import time on the host and is part of the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison; it creates no FPGA circuit by itself.

Line 24: `class FixtureCase:` This defines class FixtureCase, which groups data and methods for fixture case. Instances exist in host memory and organize the flow that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 25: `name: str` Inside the class FixtureCase, which handles fixture case, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are name: the value named 'name' in this block; str: the value named 'str' in this block.

Line 26: `mode: ComputeMode` Inside the class FixtureCase, which handles fixture case, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are mode: the value named 'mode' in this block; ComputeMode: the value named 'compute mode' in this block.

Line 27: `rows: int` Inside the class FixtureCase, which handles fixture case, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are rows: one output-row position processed by the tensor tile.

Line 28: `columns: int` Inside the class FixtureCase, which handles fixture case, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are columns: one output-feature position processed in parallel.

Line 29: `activation_zero_point: int` Inside the class FixtureCase, which handles fixture case, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 30: `weight_zero_point: int` Inside the class FixtureCase, which handles fixture case, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are weight_zero_point: a learned coefficient multiplied by an activation.

Line 31: `activation_words: list[int]` Inside the class FixtureCase, which handles fixture case, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are activation_words: an input feature value consumed by a neuron or multiply-accumulate operation; list: the value named 'list' in this block.

Line 32: `weight_words: list[int]` Inside the class FixtureCase, which handles fixture case, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are weight_words: a learned coefficient multiplied by an activation; list: the value named 'list' in this block.

Line 33: `expected: np.ndarray` Inside the class FixtureCase, which handles fixture case, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are expected: the value named 'expected' in this block; ndarray: the value named 'ndarray' in this block.

Line 36: `def pack_bytes(values: np.ndarray | list[int]) -> int:` This starts function pack_bytes. Its indented body performs pack bytes as part of the file that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison; it runs only when called.

Line 37: `packed = 0` This assigns '0' to 'packed ' in host memory. Here packed means the value named 'packed' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_bytes, which handles pack bytes.

Line 38: `for index, value in enumerate(np.asarray(values).reshape(-1)):` This iterates index, value over ‘enumerate(np.asarray(values).reshape(-1))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_bytes, which handles pack bytes.

Line 39: `packed |= (int(value) & 0xFF) << (8 * index)` Inside the function pack_bytes, which handles pack bytes, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are packed: the value named ’packed’ in this block; value: the value named ’value’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry.

Line 40: `return packed` This returns ‘packed‘ to the caller of the function pack_bytes, which handles pack bytes, ending this invocation. The important operands are packed: the value named ’packed’ in this block.

Line 43: `def make_dense_outer(rng: np.random.Generator) -> FixtureCase:` This starts function make_dense_outer. Its indented body performs make dense outer as part of the file that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison; it runs only when called.

Line 44: `rows, columns, reduction = 8, 16, 19` This assigns ‘8, 16, 19‘ to ‘rows, columns, reduction ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function make_dense_outer, which handles make dense outer.

Line 45: `za, zb = 117, 139` This assigns ‘117, 139‘ to ‘za, zb ‘ in host memory. Here za means the value named ’za’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_dense_outer, which handles make dense outer.

Line 46: `a = rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)` This assigns ‘rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)‘ to ‘a ‘ in host memory. Here a means the value named ’a’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function make_dense_outer, which handles make dense outer.

Line 47: `b = rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)` This assigns ‘rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)‘ to ‘b ‘ in host memory. Here b means the value named ’b’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function make_dense_outer, which handles make dense outer.

Line 48: `a[:, [2, 7, 14]] = za` This assigns ‘za‘ to ‘a[‘ in host memory. Here a means the value named ’a’ in this block. The important operands are za: the value named ’za’ in this block. Context: the function make_dense_outer, which handles make dense outer.

Line 49: `expected = gemm_u8_zero_point(a, b, za, zb)` This assigns ‘gemm_u8_zero_point(a, b, za, zb)‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are gemm_u8_zero_point: the integer code representing real numeric zero in affine quantization; a: the value named ’a’ in this block; b: the value named ’b’ in this block. Context: the function make_dense_outer, which handles make dense outer.

Line 50: `a_words = [pack_bytes(a[:, k]) for k in range(reduction)]` This assigns ‘[pack_bytes(a[:, k]) for k in range(reduction)]‘ to ‘a_words ‘ in host memory. Here a_words means the value named ’a words’ in this block. The important operands are pack_bytes: the value named ’pack bytes’ in this block; a: the value named ’a’ in this block; k: the value named ’k’ in this block. Context: the function make_dense_outer, which handles make dense outer.

Line 51: `b_words = [pack_bytes(b[k]) for k in range(reduction)]` This assigns ‘[pack_bytes(b[k]) for k in range(reduction)]‘ to ‘b_words ‘ in host memory. Here b_words means the value named ’b words’ in this block. The important operands are pack_bytes: the value named ’pack bytes’ in this block; b: the value named ’b’ in this block; k: the value named ’k’ in this block. Context: the function make_dense_outer, which handles make dense outer.

Line 52: `return FixtureCase("dense_outer_8x16x19", ComputeMode.DENSE_OUTER, rows, columns, za, zb, a_words, b_words,...` This returns 'FixtureCase("dense_outer_8x16x19", ComputeMode.DENSE_OUTER, rows, columns, za, zb, a_words, b_words, expected)' to the caller of the function make_dense_outer, which handles make dense outer, ending this invocation. The important operands are FixtureCase: the value named 'fixture case' in this block; dense_outer_8x16x19: the value named 'dense outer 8x16x19' in this block; ComputeMode: the value named 'compute mode' in this block.

Line 55: `def make_dense_dot8(rng: np.random.Generator) -> FixtureCase:` This starts function make_dense_dot8. Its indented body performs make dense dot8 as part of the file that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison; it runs only when called.

Line 56: `rows, columns, reduction = 1, 13, 21` This assigns '1, 13, 21' to 'rows, columns, reduction ' in host memory. Here rows means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function make_dense_dot8, which handles make dense dot8.

Line 57: `padded_reduction = 24` This assigns '24' to 'padded_reduction ' in host memory. Here padded_reduction means the value named 'padded reduction' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_dense_dot8, which handles make dense dot8.

Line 58: `za, zb = 128, 128` This assigns '128, 128' to 'za, zb ' in host memory. Here za means the value named 'za' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_dense_dot8, which handles make dense dot8.

Line 59: `a = np.full((rows, padded_reduction), za, dtype=np.uint8)` This assigns 'np.full((rows, padded_reduction), za, dtype=np.uint8)' to 'a ' in host memory. Here a means the value named 'a' in this block. The important operands are full: the value named 'full' in this block; rows: one output-row position processed by the tensor tile; padded_reduction: the value named 'padded reduction' in this block. Context: the function make_dense_dot8, which handles make dense dot8.

Line 60: `b = np.full((padded_reduction, columns), zb, dtype=np.uint8)` This assigns 'np.full((padded_reduction, columns), zb, dtype=np.uint8)' to 'b ' in host memory. Here b means the value named 'b' in this block. The important operands are full: the value named 'full' in this block; padded_reduction: the value named 'padded reduction' in this block; columns: one output-feature position processed in parallel. Context: the function make_dense_dot8, which handles make dense dot8.

Line 61: `a[:, :reduction] = rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)` This assigns 'rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)' to 'a[' in host memory. Here a means the value named 'a' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function make_dense_dot8, which handles make dense dot8.

Line 62: `b[:reduction] = rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)` This assigns 'rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)' to 'b[' in host memory. Here b means the value named 'b' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function make_dense_dot8, which handles make dense dot8.

Line 63: `a[:, 8:16] = za` This assigns 'za' to 'a[' in host memory. Here a means the value named 'a' in this block. The important operands are za: the value named 'za' in this block. Context: the function make_dense_dot8, which handles make dense dot8.

Line 64: `expected = gemm_u8_zero_point(a, b, za, zb)` This assigns 'gemm_u8_zero_point(a, b, za, zb)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are gemm_u8_zero_point: the integer code representing real numeric zero in affine quantization; a: the value named 'a' in this block; b: the value named 'b' in this block. Context: the function make_dense_dot8, which handles make dense dot8.

Line 65: `a_words: list[int] = []` This assigns '[]' to 'a_words' in host memory. Here a_words means the value named 'a words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_dense_dot8, which handles make dense dot8.

Line 66: `b_words: list[int] = []` This assigns '[]' to 'b_words' in host memory. Here b_words means the value named 'b words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_dense_dot8, which handles make dense dot8.

Line 67: `for chunk_start in range(0, padded_reduction, ROW_LANES):` This iterates chunk_start over 'range(0, padded_reduction, ROW_LANES)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function make_dense_dot8, which handles make dense dot8.

Line 68: `a_words.append(pack_bytes(a[0, chunk_start : chunk_start + ROW_LANES]))` This calls 'a_words.append' with 'pack_bytes(a[0, chunk_start : chunk_start + ROW_LANES])' inside the function make_dense_dot8, which handles make dense dot8. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 69: `packed_weights = 0` This assigns '0' to 'packed_weights ' in host memory. Here packed_weights means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function make_dense_dot8, which handles make dense dot8.

Line 70: `for column in range(COLUMN_LANES):` This iterates column over 'range(COLUMN_LANES)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function make_dense_dot8, which handles make dense dot8.

Line 71: `lane_values = (` This statement continues through source line 75; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(b[chunk_start : chunk_start + ROW_LANES, column] if column < columns else np.full(ROW_LANES, zb, dtype=np.uint8))' to 'lane_values ' in host memory. Here lane_values means one parallel arithmetic element within a vector or tensor datapath. The important operands are b: the value named 'b' in this block; chunk_start: the value named 'chunk start' in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function make_dense_dot8, which handles make dense dot8.

Line 72: `b[chunk_start : chunk_start + ROW_LANES, column]` This is continuation line 2 of the 5-line logical statement that begins at source line 71. Its exact fragment is 'b[chunk_start : chunk_start + ROW_LANES, column]'. The important operands are b: the value named 'b' in this block; chunk_start: the value named 'chunk start' in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns '(b[chunk_start : chunk_start + ROW_LANES, column] if column < columns else np.full(ROW_LANES, zb, dtype=np.uint8))' to 'lane_values ' in host memory. Here lane_values means one parallel arithmetic element within a vector or tensor datapath. The important operands are b: the value named 'b' in this block; chunk_start: the value named 'chunk start' in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function make_dense_dot8, which handles make dense dot8.

Line 73: `if column < columns` This is continuation line 3 of the 5-line logical statement that begins at source line 71. Its exact fragment is 'if column < columns'. The important operands are column: one output-feature position processed in parallel; columns: one output-feature position processed in parallel. Read together, the complete operation is: This assigns '(b[chunk_start : chunk_start + ROW_LANES, column] if column < columns else np.full(ROW_LANES, zb, dtype=np.uint8))' to 'lane_values ' in host memory. Here lane_values means one parallel arithmetic element within a vector or tensor datapath. The important operands are b: the value named 'b' in this block; chunk_start: the value named 'chunk start' in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function make_dense_dot8, which handles make dense dot8.

Line 74: `else np.full(ROW_LANES, zb, dtype=np.uint8)` This is continuation line 4 of the 5-line logical statement that begins at source line 71. Its exact fragment is 'else np.full(ROW_LANES, zb, dtype=np.uint8)'. The important operands are full: the value named 'full' in this block; ROW_LANES: one parallel arithmetic element within a

vector or tensor datapath; zb: the value named 'zb' in this block. Read together, the complete operation is: This assigns '(b[chunk_start : chunk_start + ROW_LANES, column] if column < columns else np.full(ROW_LANES, zb, dtype=np.uint8))' to 'lane_values ' in host memory. Here lane_values means one parallel arithmetic element within a vector or tensor datapath. The important operands are b: the value named 'b' in this block; chunk_start: the value named 'chunk start' in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function make_dense_dot8, which handles make dense dot8.

Line 75: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 71. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(b[chunk_start : chunk_start + ROW_LANES, column] if column < columns else np.full(ROW_LANES, zb, dtype=np.uint8))' to 'lane_values ' in host memory. Here lane_values means one parallel arithmetic element within a vector or tensor datapath. The important operands are b: the value named 'b' in this block; chunk_start: the value named 'chunk start' in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function make_dense_dot8, which handles make dense dot8.

Line 76: `packed_weights |= pack_bytes(lane_values) << (64 * column)` Inside the function make_dense_dot8, which handles make dense dot8, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are packed_weights: a learned coefficient multiplied by an activation; pack_bytes: the value named 'pack bytes' in this block; lane_values: one parallel arithmetic element within a vector or tensor datapath.

Line 77: `b_words.append(packed_weights)` This calls 'b_words.append' with 'packed_weights' inside the function make_dense_dot8, which handles make dense dot8. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 78: `return FixtureCase("dense_dot8_1x13x24", ComputeMode.DENSE_DOT8, rows, columns, za, zb, a_words, b_words, e...` This returns 'FixtureCase("dense_dot8_1x13x24", ComputeMode.DENSE_DOT8, rows, columns, za, zb, a_words, b_words, expected)' to the caller of the function make_dense_dot8, which handles make dense dot8, ending this invocation. The important operands are FixtureCase: the value named 'fixture case' in this block; dense_dot8_1x13x24: the value named 'dense dot8 1x13x24' in this block; ComputeMode: the value named 'compute mode' in this block.

Line 81: `def make_binary(rng: np.random.Generator) -> FixtureCase:` This starts function make_binary. Its indented body performs make binary as part of the file that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison; it runs only when called.

Line 82: `rows, columns, reduction = 5, 11, 17` This assigns '5, 11, 17' to 'rows, columns, reduction ' in host memory. Here rows means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function make_binary, which handles make binary.

Line 83: `spikes = rng.integers(0, 2, size=(rows, reduction), dtype=np.int8)` This assigns 'rng.integers(0, 2, size=(rows, reduction), dtype=np.int8)' to 'spikes ' in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function make_binary, which handles make binary.

Line 84: `weights = rng.integers(-128, 128, size=(reduction, columns), dtype=np.int16)` This assigns 'rng.integers(-128, 128, size=(reduction, columns), dtype=np.int16)' to 'weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function make_binary, which handles make binary.

Line 85: `spikes[:, [4, 12]] = 0` This assigns '0' to 'spikes[' in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the function make_binary, which handles make binary.

Line 86: `expected = binary_weighted_gemm(spikes, weights)` This assigns ‘binary_weighted_gemm(spikes, weights)‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are binary_weighted_gemm: a learned coefficient multiplied by an activation; spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation. Context: the function make_binary, which handles make binary.

Line 87: `a_words: list[int] = []` This assigns ‘[]‘ to ‘a_words‘ in host memory. Here a_words means the value named ’a words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_binary, which handles make binary.

Line 88: `b_words: list[int] = []` This assigns ‘[]‘ to ‘b_words‘ in host memory. Here b_words means the value named ’b words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_binary, which handles make binary.

Line 89: `for k in range(reduction):` This iterates k over ‘range(reduction)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function make_binary, which handles make binary.

Line 90: `a_padded = np.zeros(ROW_LANES, dtype=np.int16)` This assigns ‘np.zeros(ROW_LANES, dtype=np.int16)‘ to ‘a_padded ‘ in host memory. Here a_padded means the value named ’a padded’ in this block. The important operands are zeros: the value named ’zeros’ in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; dtype: the value named ’dtype’ in this block. Context: the function make_binary, which handles make binary.

Line 91: `b_padded = np.zeros(COLUMN_LANES, dtype=np.int16)` This assigns ‘np.zeros(COLUMN_LANES, dtype=np.int16) to ‘b_padded ‘ in host memory. Here b_padded means the value named ’b padded’ in this block. The important operands are zeros: the value named ’zeros’ in this block; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath; dtype: the value named ’dtype’ in this block. Context: the function make_binary, which handles make binary.

Line 92: `a_padded[:rows] = spikes[:, k]` This assigns ‘spikes[:, k]‘ to ‘a_padded[‘ in host memory. Here a_padded means the value named ’a padded’ in this block. The important operands are spikes: a binary event produced when a neuron crosses its threshold; k: the value named ’k’ in this block. Context: the function make_binary, which handles make binary.

Line 93: `b_padded[:columns] = weights[k]` This assigns ‘weights[k]‘ to ‘b_padded[‘ in host memory. Here b_padded means the value named ’b padded’ in this block. The important operands are weights: a learned coefficient multiplied by an activation; k: the value named ’k’ in this block. Context: the function make_binary, which handles make binary.

Line 94: `a_words.append(pack_bytes(a_padded))` This calls ‘a_words.append‘ with ‘pack_bytes(a_padded)‘ inside the function make_binary, which handles make binary. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 95: `b_words.append(pack_bytes(b_padded))` This calls ‘b_words.append‘ with ‘pack_bytes(b_padded)‘ inside the function make_binary, which handles make binary. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 96: `return FixtureCase("binary_weighted_5x11x17", ComputeMode.BINARY_WEIGHTED, rows, columns, 0, 0, a_words, b_...` This returns ‘FixtureCase("binary_weighted_5x11x17", ComputeMode.BINARY_WEIGHTED, rows, columns, 0, 0, a_words, b_words, expected)‘ to the caller of the function make_binary, which handles make binary, ending this invocation. The important operands are FixtureCase: the value named ’fixture case’ in this block; binary_weighted_5x11x17: a learned coefficient multiplied by an activation; ComputeMode: the value named ’compute mode’ in this block.

Line 99: `def make_ternary(rng: np.random.Generator) -> FixtureCase:` This starts function make_ternary. Its indented body performs make ternary as part of the file that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison; it runs only when called.

Line 100: `rows, columns, reduction = 7, 9, 15` This assigns ‘7, 9, 15‘ to ‘rows, columns, reduction ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function make_ternary, which handles make ternary.

Line 101: `spikes = rng.integers(-1, 2, size=(rows, reduction), dtype=np.int8)` This assigns ‘rng.integers(-1, 2, size=(rows, reduction), dtype=np.int8)‘ to ‘spikes ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function make_ternary, which handles make ternary.

Line 102: `weights = rng.integers(-128, 128, size=(reduction, columns), dtype=np.int16)` This assigns ‘rng.integers(128, 128, size=(reduction, columns), dtype=np.int16)‘ to ‘weights ‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function make_ternary, which handles make ternary.

Line 103: `spikes[:, [1, 10]] = 0` This assigns ‘0‘ to ‘spikes[‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the function make_ternary, which handles make ternary.

Line 104: `expected = ternary_weighted_gemm(spikes, weights)` This assigns ‘ternary_weighted_gemm(spikes, weights)‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are ternary_weighted_gemm: a learned coefficient multiplied by an activation; spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation. Context: the function make_ternary, which handles make ternary.

Line 105: `a_words: list[int] = []` This assigns ‘[]‘ to ‘a_words‘ in host memory. Here a_words means the value named ’a words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_ternary, which handles make ternary.

Line 106: `b_words: list[int] = []` This assigns ‘[]‘ to ‘b_words‘ in host memory. Here b_words means the value named ’b words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_ternary, which handles make ternary.

Line 107: `for k in range(reduction):` This iterates k over ‘range(reduction)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function make_ternary, which handles make ternary.

Line 108: `a_padded = np.zeros(ROW_LANES, dtype=np.int16)` This assigns ‘np.zeros(ROW_LANES, dtype=np.int16)‘ to ‘a_padded ‘ in host memory. Here a_padded means the value named ’a padded’ in this block. The important operands are zeros: the value named ’zeros’ in this block; ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; dtype: the value named ’dtype’ in this block. Context: the function make_ternary, which handles make ternary.

Line 109: `b_padded = np.zeros(COLUMN_LANES, dtype=np.int16)` This assigns ‘np.zeros(COLUMN_LANES, dtype=np.int16‘ to ‘b_padded ‘ in host memory. Here b_padded means the value named ’b padded’ in this block. The important operands are zeros: the value named ’zeros’ in this block; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath; dtype: the value named ’dtype’ in this block. Context: the function make_ternary, which handles make ternary.

Line 110: `a_padded[:rows] = spikes[:, k]` This assigns ‘spikes[:, k]‘ to ‘a_padded[‘ in host memory. Here a_padded means the value named ’a padded’ in this block. The important operands are spikes: a binary event produced when a neuron crosses its threshold; k: the value named ’k’ in this block. Context: the function make_ternary, which handles make ternary.

Line 111: `b_padded[:columns] = weights[k]` This assigns ‘weights[k]‘ to ‘b_padded[‘ in host memory. Here b_padded means the value named ’b padded’ in this block. The important operands are weights: a learned coefficient multiplied by an activation; k: the value named ’k’ in this block. Context: the function make_ternary, which handles make ternary.

Line 112: `a_words.append(pack_bytes(a_padded))` This calls ‘a_words.append‘ with ‘pack_bytes(a_padded)‘ inside the function make_ternary, which handles make ternary. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 113: `b_words.append(pack_bytes(b_padded))` This calls ‘b_words.append‘ with ‘pack_bytes(b_padded)‘ inside the function make_ternary, which handles make ternary. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 114: `return FixtureCase("ternary_weighted_7x9x15", ComputeMode.TERNARY_WEIGHTED, rows, columns, 0, 0, a_words, b...` This returns ‘FixtureCase("ternary_weighted_7x9x15", ComputeMode.TERNARY_WEIGHTED, rows, columns, 0, 0, a_words, b_words, expected)‘ to the caller of the function make_ternary, which handles make ternary, ending this invocation. The important operands are FixtureCase: the value named ’fixture case’ in this block; ternary_weighted_7x9x15: a learned coefficient multiplied by an activation; ComputeMode: the value named ’compute mode’ in this block.

Line 117: `def command_word(case: FixtureCase) -> int:` This starts function command_word. Its indented body performs command word as part of the file that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison; it runs only when called.

Line 118: `issues = len(case.activation_words)` This assigns ‘len(case.activation_words)‘ to ‘issues ‘ in host memory. Here issues means the value named ’issues’ in this block. The important operands are len: the value named ’len’ in this block; case: the value named ’case’ in this block; activation_words: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function command_word, which handles command word.

Line 119: `return (` This statement continues through source line 126; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘(int(case.mode) | (case.rows << 4) | (case.columns << 8) | (issues << 14) | (case.activation_zero_point << 26) | (case.weight_zero_point << 34))‘ to the caller of the function command_word, which handles command word, ending this invocation. The important operands are case: the value named ’case’ in this block; mode: the value named ’mode’ in this block; rows: one output-row position processed by the tensor tile.

Line 120: `int(case.mode)` This is continuation line 2 of the 8-line logical statement that begins at source line 119. Its exact fragment is ‘int(case.mode)‘. The important operands are case: the value named ’case’ in this block; mode: the value named ’mode’ in this block. Read together, the complete operation is: This returns ‘(int(case.mode) | (case.rows << 4) | (case.columns << 8) | (issues << 14) | (case.activation_zero_point << 26) | (case.weight_zero_point << 34))‘ to the caller of the function command_word, which handles command word, ending this invocation. The important operands are case: the value named ’case’ in this block; mode: the value named ’mode’ in this block; rows: one output-row position processed by the tensor tile.

Line 121: `| (case.rows << 4)` This is continuation line 3 of the 8-line logical statement that begins at source line 119. Its exact fragment is ‘| (case.rows << 4)‘. The important operands are case: the value named ’case’ in this block; rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This returns ‘(int(case.mode) | (case.rows << 4) | (case.columns << 8) | (issues << 14) | (case.activation_zero_point << 26) | (case.weight_zero_point << 34))‘ to the caller of the function command_word, which handles command word, ending this invocation. The important operands are case: the value named ’case’ in this block; mode: the value named ’mode’ in this block; rows: one output-row position processed by the tensor tile.

Line 122: `| (case.columns << 8)` This is continuation line 4 of the 8-line logical statement that begins at source line 119. Its exact fragment is '| (case.columns << 8)'. The important operands are case: the value named 'case' in this block; columns: one output-feature position processed in parallel. Read together, the complete operation is: This returns '(int(case.mode) | (case.rows << 4) | (case.columns << 8) | (issues << 14) | (case.activation_zero_point << 26) | (case.weight_zero_point << 34))' to the caller of the function command_word, which handles command word, ending this invocation. The important operands are case: the value named 'case' in this block; mode: the value named 'mode' in this block; rows: one output-row position processed by the tensor tile.

Line 123: `| (issues << 14)` This is continuation line 5 of the 8-line logical statement that begins at source line 119. Its exact fragment is '| (issues << 14)'. The important operands are issues: the value named 'issues' in this block. Read together, the complete operation is: This returns '(int(case.mode) | (case.rows << 4) | (case.columns << 8) | (issues << 14) | (case.activation_zero_point << 26) | (case.weight_zero_point << 34))' to the caller of the function command_word, which handles command word, ending this invocation. The important operands are case: the value named 'case' in this block; mode: the value named 'mode' in this block; rows: one output-row position processed by the tensor tile.

Line 124: `| (case.activation_zero_point << 26)` This is continuation line 6 of the 8-line logical statement that begins at source line 119. Its exact fragment is '| (case.activation_zero_point << 26)'. The important operands are case: the value named 'case' in this block; activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This returns '(int(case.mode) | (case.rows << 4) | (case.columns << 8) | (issues << 14) | (case.activation_zero_point << 26) | (case.weight_zero_point << 34))' to the caller of the function command_word, which handles command word, ending this invocation. The important operands are case: the value named 'case' in this block; mode: the value named 'mode' in this block; rows: one output-row position processed by the tensor tile.

Line 125: `| (case.weight_zero_point << 34)` This is continuation line 7 of the 8-line logical statement that begins at source line 119. Its exact fragment is '| (case.weight_zero_point << 34)'. The important operands are case: the value named 'case' in this block; weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This returns '(int(case.mode) | (case.rows << 4) | (case.columns << 8) | (issues << 14) | (case.activation_zero_point << 26) | (case.weight_zero_point << 34))' to the caller of the function command_word, which handles command word, ending this invocation. The important operands are case: the value named 'case' in this block; mode: the value named 'mode' in this block; rows: one output-row position processed by the tensor tile.

Line 126: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 119. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns '(int(case.mode) | (case.rows << 4) | (case.columns << 8) | (issues << 14) | (case.activation_zero_point << 26) | (case.weight_zero_point << 34))' to the caller of the function command_word, which handles command word, ending this invocation. The important operands are case: the value named 'case' in this block; mode: the value named 'mode' in this block; rows: one output-row position processed by the tensor tile.

Line 129: `def write_hex(path: Path, values: list[int], width_bits: int) -> None:` This starts function write_hex. Its indented body performs write hex as part of the file that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison; it runs only when called.

Line 130: `digits = (width_bits + 3) // 4` This assigns '(width_bits + 3) // 4' to 'digits ' in host memory. Here digits means the value named 'digits' in this block. The important operands are width_bits: the value named 'width bits' in this block. Context: the function write_hex, which handles write hex.

Line 131: `mask = (1 << width_bits) - 1` This assigns '(1 << width_bits) - 1' to 'mask ' in host memory. Here mask means the value named 'mask' in this block. The important operands are width_bits: the value named 'width bits' in this block. Context: the function write_hex, which handles write hex.

Line 132: `path.write_text("".join(f"{value & mask:0{digits}x}\n" for value in values), encoding="ascii")` This calls 'path.write_text' with '"".join(f"{value & mask:0{digits}x}\n" for value in values), encoding="ascii"' inside the function write_hex, which handles write hex. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 135: `def main() -> None:` This starts function main. Its indented body performs main as part of the file that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison; it runs only when called.

Line 136: `fixture_dir = Path(__file__).resolve().parents[1] / "tb" / "fixtures"` This assigns ‘Path(___file___).reso / "tb" / "fixtures"‘ to ‘fixture_dir ‘ in host memory. Here fixture_dir means the value named ’fixture dir’ in this block. The important operands are Path: the value named ’path’ in this block; ___file___: the value named ’file’ in this block; resolve: the value named ’resolve’ in this block. Context: the function main, which handles main.

Line 137: `fixture_dir.mkdir(parents=True, exist_ok=True)` This calls ‘fixture_dir.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the function main, which handles main. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 138: `rng = np.random.default_rng(0x4E455552)` This assigns ‘np.random.default_rng(0x4E455552)‘ to ‘rng ‘ in host memory. Here rng means the value named ’rng’ in this block. The important operands are random: the value named ’random’ in this block; default_rng: the value named ’default rng’ in this block. Context: the function main, which handles main.

Line 139: `cases = [make_dense_outer(rng), make_dense_dot8(rng), make_binary(rng), make_ternary(rng)]` This assigns ‘[make_dense_outer(rng), make_dense_dot8(rng), make_binary(rng), make_ternary(rng)]‘ to ‘cases ‘ in host memory. Here cases means the value named ’cases’ in this block. The important operands are make_dense_outer: the value named ’make dense outer’ in this block; rng: the value named ’rng’ in this block; make_dense_dot8: the value named ’make dense dot8’ in this block. Context: the function main, which handles main.

Line 141: `commands = [command_word(case) for case in cases]` This assigns ‘[command_word(case) for case in cases]‘ to ‘commands ‘ in host memory. Here commands means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are command_word: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; case: the value named ’case’ in this block; for: the value named ’for’ in this block. Context: the function main, which handles main.

Line 142: `activation_words = [word for case in cases for word in case.activation_words]` This assigns ‘[word for case in cases for word in case.activation_words]‘ to ‘activation_words ‘ in host memory. Here activation_words means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are word: the value named ’word’ in this block; for: the value named ’for’ in this block; case: the value named ’case’ in this block. Context: the function main, which handles main.

Line 143: `weight_words = [word for case in cases for word in case.weight_words]` This assigns ‘[word for case in cases for word in case.weight_words]‘ to ‘weight_words ‘ in host memory. Here weight_words means a learned coefficient multiplied by an activation. The important operands are word: the value named ’word’ in this block; for: the value named ’for’ in this block; case: the value named ’case’ in this block. Context: the function main, which handles main.

Line 144: `expected_values = [int(value) for case in cases for value in case.expected.reshape(-1)]` This assigns ‘[int(value) for case in cases for value in case.expected.reshape(-1)]‘ to ‘expected_values ‘ in host memory. Here expected_values means the value named ’expected values’ in this block. The important operands are value: the value named ’value’ in this block; for: the value named ’for’ in this block; case: the value named ’case’ in this block. Context: the function main, which handles main.

Line 146: `write_hex(fixture_dir / "tensor_cases.mem", commands, 64)` This calls ‘write_hex‘ with ‘fixture_dir / "tensor_cases.mem", commands, 64‘ inside the function main, which handles main. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 147: `write_hex(fixture_dir / "tensor_activations.mem", activation_words, 64)` This calls ‘write_hex‘ with ‘fixture_dir / "tensor_activations.mem", activation_words, 64‘ inside the function main, which handles main. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 148: `write_hex(fixture_dir / "tensor_weights.mem", weight_words, COLUMN_LANES * ROW_LANES * 8)` This calls ‘write_hex‘ with ‘fixture_dir / "tensor_weights.mem", weight_words, COLUMN_LANES * ROW_LANES * 8‘ inside the function main, which handles main. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 149: `write_hex(fixture_dir / "tensor_expected.mem", expected_values, 64)` This calls ‘write_hex‘ with ‘fixture_dir / "tensor_expected.mem", expected_values, 64‘ inside the function main, which handles main. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 150: `(fixture_dir / "tensor_tile_params.svh").write_text(` This statement continues through source line 157; the explanation uses the complete expression rather than only the first visual fragment. Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are fixture_dir: the value named ’fixture dir’ in this block; tensor_tile_params: the value named ’tensor tile params’ in this block; svh: the value named ’svh’ in this block.

Line 151: `"‘define NEURAEDGE_TENSOR_CASES %d\n"` This is continuation line 2 of the 8-line logical statement that begins at source line 150. Its exact fragment is ‘"‘define NEURAEDGE_TENSOR_CASES %d\n"‘. The important operands are define: the value named ’define’ in this block; NEURAEDGE_TENSOR_CASES: the value named ’neuraedge tensor cases’ in this block; d: the value named ’d’ in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are fixture_dir: the value named ’fixture dir’ in this block; tensor_tile_params: the value named ’tensor tile params’ in this block; svh: the value named ’svh’ in this block.

Line 152: `"‘define NEURAEDGE_TENSOR_ISSUES %d\n"` This is continuation line 3 of the 8-line logical statement that begins at source line 150. Its exact fragment is ‘"‘define NEURAEDGE_TENSOR_ISSUES %d\n"‘. The important operands are define: the value named ’define’ in this block; NEURAEDGE_TENSOR_ISSUES: the value named ’neuraedge tensor issues’ in this block; d: the value named ’d’ in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are fixture_dir: the value named ’fixture dir’ in this block; tensor_tile_params: the value named ’tensor tile params’ in this block; svh: the value named ’svh’ in this block.

Line 153: `"‘define NEURAEDGE_TENSOR_RESULTS %d\n"` This is continuation line 4 of the 8-line logical statement that begins at source line 150. Its exact fragment is ‘"‘define NEURAEDGE_TENSOR_RESULTS %d\n"‘. The important operands are define: the value named ’define’ in this block; NEURAEDGE_TENSOR_RESULTS: the value named ’neuraedge tensor results’ in this block; d: the value named ’d’ in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are fixture_dir: the value named ’fixture dir’ in this block; tensor_tile_params: the value named ’tensor tile params’ in this block; svh: the value named ’svh’ in this block.

Line 154: `"‘define NEURAEDGE_COLUMN_LANES %d\n"` This is continuation line 5 of the 8-line logical statement that begins at source line 150. Its exact fragment is ‘"‘define NEURAEDGE_COLUMN_LANES %d\n"‘. The important operands are define: the value named ’define’ in this block; NEURAEDGE_COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath; d: the value named ’d’ in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are fixture_dir: the value named ’fixture dir’ in this block; tensor_tile_params: the value named ’tensor tile params’ in this block; svh: the value named ’svh’ in this block.

Line 155: `% (len(cases), len(activation_words), len(expected_values), COLUMN_LANES),` This is continuation line 6 of the 8-line logical statement that begins at source line 150. Its exact fragment is ‘% (len(cases), len(activation_words), len(expected_values), COLUMN_LANES),‘. The important operands are len: the value named ’len’ in this block; cases:

the value named 'cases' in this block; activation_words: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are fixture_dir: the value named 'fixture dir' in this block; tensor_tile_params: the value named 'tensor tile params' in this block; svh: the value named 'svh' in this block.

Line 156: `encoding="ascii",` This is continuation line 7 of the 8-line logical statement that begins at source line 150. Its exact fragment is 'encoding="ascii",'. The important operands are encoding: the value named 'encoding' in this block; ascii: the value named 'ascii' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are fixture_dir: the value named 'fixture dir' in this block; tensor_tile_params: the value named 'tensor tile params' in this block; svh: the value named 'svh' in this block.

Line 157: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 150. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are fixture_dir: the value named 'fixture dir' in this block; tensor_tile_params: the value named 'tensor tile params' in this block; svh: the value named 'svh' in this block.

Line 159: `manifest = {` This statement continues through source line 179; the explanation uses the complete expression rather than only the first visual fragment. Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 160: `"seed": "0x4E455552",` This is continuation line 2 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"seed": "0x4E455552",'. The important operands are seed: the value named 'seed' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 161: `"case_count": len(cases),` This is continuation line 3 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"case_count": len(cases),'. The important operands are case_count: an integer count retained for control or measurable telemetry; len: the value named 'len' in this block; cases: the value named 'cases' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 162: `"issue_count": len(activation_words),` This is continuation line 4 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"issue_count": len(activation_words),'. The important operands are issue_count: an integer count retained for control or measurable telemetry; len: the value named 'len' in this block; activation_words: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 163: `"result_count": len(expected_values),` This is continuation line 5 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"result_count": len(expected_values),'. The important operands are result_count: an integer count retained for control or measurable telemetry; len: the value named 'len' in this block; expected_values: the value named 'expected values' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense,

binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 164: `"cases": [` This is continuation line 6 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"cases": [‘. The important operands are cases: the value named 'cases' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 165: `{` This is continuation line 7 of the 21-line logical statement that begins at source line 159. Its exact fragment is '{‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 166: `"name": case.name,` This is continuation line 8 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"name": case.name,‘. The important operands are name: the value named 'name' in this block; case: the value named 'case' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 167: `"mode": case.mode.name,` This is continuation line 9 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"mode": case.mode.name,‘. The important operands are mode: the value named 'mode' in this block; case: the value named 'case' in this block; name: the value named 'name' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 168: `"rows": case.rows,` This is continuation line 10 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"rows": case.rows,‘. The important operands are rows: one output-row position processed by the tensor tile; case: the value named 'case' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 169: `"columns": case.columns,` This is continuation line 11 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"columns": case.columns,‘. The important operands are columns: one output-feature position processed in parallel; case: the value named 'case' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 170: `"issues": len(case.activation_words),` This is continuation line 12 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"issues": len(case.activation_words),‘. The important operands are issues: the value named 'issues' in this block; len: the value named 'len' in this block; case: the value named 'case' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL

comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 171: `"activation_zero_point": case.activation_zero_point,` This is continuation line 13 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"activation_zero_point": case.activation_zero_point,'. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; case: the value named 'case' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 172: `"weight_zero_point": case.weight_zero_point,` This is continuation line 14 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"weight_zero_point": case.weight_zero_point,'. The important operands are weight_zero_point: a learned coefficient multiplied by an activation; case: the value named 'case' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 173: `"skipped_zero_issues": sum(word == pack_bytes([case.activation_zero_point] * ROW_LANES) for word in case.ac...` This is continuation line 15 of the 21-line logical statement that begins at source line 159. Its exact fragment is '"skipped_zero_issues": sum(word == pack_bytes([case.activation_zero_point] * ROW_LANES) for word in case.activation_words)'. The important operands are skipped_zero_issues: the value named 'skipped zero issues' in this block; sum: the value named 'sum' in this block; word: the value named 'word' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 174: `if case.mode in (ComputeMode.DENSE_OUTER, ComputeMode.DENSE_DOT8)` This is continuation line 16 of the 21-line logical statement that begins at source line 159. Its exact fragment is 'if case.mode in (ComputeMode.DENSE_OUTER, ComputeMode.DENSE_DOT8)'. The important operands are case: the value named 'case' in this block; mode: the value named 'mode' in this block; in: the value named 'in' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 175: `else sum(word == 0 for word in case.activation_words),` This is continuation line 17 of the 21-line logical statement that begins at source line 159. Its exact fragment is 'else sum(word == 0 for word in case.activation_words),'. The important operands are sum: the value named 'sum' in this block; word: the value named 'word' in this block; for: the value named 'for' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 176: `}` This is continuation line 18 of the 21-line logical statement that begins at source line 159. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 177: `for case in cases` This is continuation line 19 of the 21-line logical statement that begins at source line 159. Its exact fragment is 'for case in cases'. The important operands are for: the value named 'for' in this block; case:

the value named 'case' in this block; in: the value named 'in' in this block. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 178: `],` This is continuation line 20 of the 21-line logical statement that begins at source line 159. Its exact fragment is '],'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 179: `}` This is continuation line 21 of the 21-line logical statement that begins at source line 159. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are manifest: the value named 'manifest' in this block; seed: the value named 'seed' in this block; case_count: an integer count retained for control or measurable telemetry.

Line 180: `(fixture_dir / "tensor_fixture_manifest.json").write_text(json.dumps(manifest, indent=2), encoding="utf-8")` Inside the function main, which handles main, this statement advances the host implementation that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The important operands are fixture_dir: the value named 'fixture dir' in this block; tensor_fixture_manifest: the value named 'tensor fixture manifest' in this block; json: the value named 'json' in this block.

Line 181: `print(json.dumps(manifest, indent=2))` This calls 'print' with 'json.dumps(manifest, indent=2)' inside the function main, which handles main. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 184: `if __name__ == "__main__":` This executes the following indented host-code branch only when '___name___ == "___main___"' is true. The important operands are ___name___: the value named 'name' in this block; ___main___: the value named 'main' in this block. Context: the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

Line 185: `main()` This calls 'main' with " inside the file whose role is to builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison. The call advances the host-side job that builds deterministic dense, binary, and ternary tensor vectors for bit-exact RTL comparison.

33 software/generate_training_coverage.py

Role in the complete system: records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

33.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: none. Functions: main. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

33.2 Complete source

```
"""Generate the audited full-graph NeuraEdge training coverage manifest."""

from __future__ import annotations

import argparse
import json
from pathlib import Path

from neuraedge_training import build_spikingformer_training_coverage


def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--execution-manifest", required=True, type=Path)
    parser.add_argument("--parameter-manifest", required=True, type=Path)
    parser.add_argument("--output", required=True, type=Path)
    arguments = parser.parse_args()
    result = build_spikingformer_training_coverage(
        arguments.execution_manifest,
        arguments.parameter_manifest,
    )
    output = arguments.output.expanduser().resolve()
    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_text(json.dumps(result, indent=2) + "\n", encoding="ascii")
    summary = result["summary"]
    print(
        "NEURAEDGE_TRAINING_COVERAGE_PASSED "
        f"operators={summary['covered_execution_operator_count']} "
        f"learned_groups={summary['learned_affine_group_count']} "
        f"parameters={summary['trainable_deployed_parameter_elements']} "
        f"output={output}"
    )


if __name__ == "__main__":
    main()
```

33.3 Line annotations

Line 1: `"""Generate the audited full-graph NeuraEdge training coverage manifest."""` This is executable documentation attached to the file whose role is to records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import json` This imports the named Python API so the file whose role is to records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from pathlib import Path` This imports the named Python API so the file whose role is to records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `from neuraedge_training import build_spikingformer_training_coverage` This imports the named Python API so the file whose role is to records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `def main() -> None:` This starts function main. Its indented body performs main as part of the file that records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy; it runs only when called.

Line 13: `parser = argparse.ArgumentParser()` This assigns ‘argparse.ArgumentParser()‘ to ‘parser ‘ in host memory. Here parser means the value named ’parser’ in this block. The important operands are argparse: the value named ’argparse’ in this block; ArgumentParser: the value named ’argument parser’ in this block. Context: the function main, which handles main.

Line 14: `parser.add_argument("-execution-manifest", required=True, type=Path)` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 15: `parser.add_argument("-parameter-manifest", required=True, type=Path)` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 16: `parser.add_argument("-output", required=True, type=Path)` This declares a command-line option for the function main, which handles main. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 17: `arguments = parser.parse_args()` This assigns ‘parser.parse_args()‘ to ‘arguments ‘ in host memory. Here arguments means the value named ’arguments’ in this block. The important operands are parser: the value named ’parser’ in this block; parse_args: the value named ’parse args’ in this block. Context: the function main, which handles main.

Line 18: `result = build_spikingformer_training_coverage(` This statement continues through source line 21; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘build_spikingformer_training_cove arguments.execution_manifest, arguments.parameter_manifest,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are build_spikingformer_training_coverage: the value named ’build spikingformer training coverage’ in this block; arguments: the value named ’arguments’ in this block; execution_manifest: the value named ’execution manifest’ in this block. Context: the function main, which handles main.

Line 19: `arguments.execution_manifest,` This is continuation line 2 of the 4-line logical statement that begins at source line 18. Its exact fragment is ‘arguments.execution_manifest,‘. The important operands are arguments: the value named ’arguments’ in this block; execution_manifest: the value named ’execution manifest’ in this block. Read together, the complete operation is: This assigns ‘build_spikingformer_training_coverage(arguments.execution_manifest, arguments.parameter_manifest,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are build_spikingformer_training_coverage: the value named ’build spikingformer training coverage’ in this block; arguments: the value named ’arguments’ in this block; execution_manifest: the value named ’execution manifest’ in this block. Context: the function main, which handles main.

Line 20: `arguments.parameter_manifest,` This is continuation line 3 of the 4-line logical statement that begins at source line 18. Its exact fragment is ‘arguments.parameter_manifest,‘. The important operands are arguments: the value named ’arguments’ in this block; parameter_manifest: the value named ’parameter manifest’ in this block. Read together, the complete operation is: This assigns ‘build_spikingformer_training_coverage(arguments.execution_manifest, arguments.parameter_manifest,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are build_spikingformer_training_coverage: the value named ’build spikingformer training coverage’ in this block; arguments: the value named ’arguments’ in this block; execution_manifest: the value named ’execution manifest’ in this block. Context: the function main, which handles main.

Line 21: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 18. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘build_spikingformer_training_coverage(arguments.execution_manifest, arguments.parameter_manifest,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are build_spikingformer_training_coverage: the value named ’build spikingformer training coverage’ in this block; arguments: the value named ’arguments’ in this block; execution_manifest: the value named ’execution manifest’ in this block. Context: the function main, which handles main.

Line 22: `output = arguments.output.expanduser().resolve()` This assigns ‘arguments.output.expanduser().resolve()‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are arguments: the value named ’arguments’ in this block; output: the value named ’output’ in this block; expanduser: the value named ’expanduser’ in this block. Context: the function main, which handles main.

Line 23: `output.parent.mkdir(parents=True, exist_ok=True)` This calls ‘output.parent.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the function main, which handles main. The call advances the host-side job that records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

Line 24: `output.write_text(json.dumps(result, indent=2) + "\n", encoding="ascii")` This calls ‘output.write_text‘ with ‘json.dumps(result, indent=2) + "\n", encoding="ascii"‘ inside the function main, which handles main. The call advances the host-side job that records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

Line 25: `summary = result["summary"]` This assigns ‘result["summary"]‘ to ‘summary ‘ in host memory. Here summary means the value named ’summary’ in this block. The important operands are result: the value named ’result’ in this block; summary: the value named ’summary’ in this block. Context: the function main, which handles main.

Line 26: `print(` This statement continues through source line 32; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘print‘ with ‘ "NEURAEDGE_TRAINING_COVERAGE_PASSED " f"operators={summary['covered_execution_operator_count']} " f"learned_groups={summary['learned_affine_group_count']} " f"parameters={summary['trainable_deployed_parameter_elements']} " f"output={output}" ‘ inside the function main, which handles main. The call advances the host-side job that records which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

Line 27: `"NEURAEDGE_TRAINING_COVERAGE_PASSED "` This is continuation line 2 of the 7-line logical statement that begins at source line 26. Its exact fragment is ‘"NEURAEDGE_TRAINING_COVERAGE_PASSED "‘. The important operands are NEURAEDGE_TRAINING_COVERAGE_PASSED: the value named ’neuraedge training coverage passed’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘ "NEURAEDGE_TRAINING_COVERAGE_

" f"operators={summary['covered_execution_operator_count']} " f"learned_groups={summary['learned_affine_group_count']}
" f"parameters={summary['trainable_deployed_parameter_elements']} " f"output={output}" ‘ inside the function main,
which handles main. The call advances the host-side job that records which Spikingformer parameters receive hardware
gradients and optimizer updates in each training policy.

Line 28: `f"operators={summary['covered_execution_operator_count']} "` This is continuation line 3 of the 7-
line logical statement that begins at source line 26. Its exact fragment is ‘f"operators={summary['covered_execution_operator_coun
"‘. The important operands are f: the value named ’f’ in this block; operators: the value named ’operators’ in this block;
summary: the value named ’summary’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘
"NEURAEDGE_TRAINING_COVERAGE_PASSED " f"operators={summary['covered_execution_operator_count']}
" f"learned_groups={summary['learned_affine_group_count']} " f"parameters={summary['trainable_deployed_parameter_elemer
" f"output={output}" ‘ inside the function main, which handles main. The call advances the host-side job that records
which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

Line 29: `f"learned_groups={summary['learned_affine_group_count']} "` This is continuation line 4 of the 7-line
logical statement that begins at source line 26. Its exact fragment is ‘f"learned_groups={summary['learned_affine_group_count']}
"‘. The important operands are f: the value named ’f’ in this block; learned_groups: the value named ’learned groups’ in
this block; summary: the value named ’summary’ in this block. Read together, the complete operation is: This calls ‘print‘
with ‘ "NEURAEDGE_TRAINING_COVERAGE_PASSED " f"operators={summary['covered_execution_operator_count']}
" f"learned_groups={summary['learned_affine_group_count']} " f"parameters={summary['trainable_deployed_parameter_elemer
" f"output={output}" ‘ inside the function main, which handles main. The call advances the host-side job that records
which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

Line 30: `f"parameters={summary['trainable_deployed_parameter_elements']} "` This is continuation line 5 of
the 7-line logical statement that begins at source line 26. Its exact fragment is ‘f"parameters={summary['trainable_deployed_parame
"‘. The important operands are f: the value named ’f’ in this block; parameters: the value named ’parameters’ in this block;
summary: the value named ’summary’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘
"NEURAEDGE_TRAINING_COVERAGE_PASSED " f"operators={summary['covered_execution_operator_count']}
" f"learned_groups={summary['learned_affine_group_count']} " f"parameters={summary['trainable_deployed_parameter_elemer
" f"output={output}" ‘ inside the function main, which handles main. The call advances the host-side job that records
which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

Line 31: `f"output={output}"` This is continuation line 6 of the 7-line logical statement that begins at source line
26. Its exact fragment is ‘f"output={output}"‘. The important operands are f: the value named ’f’ in this block;
output: the value named ’output’ in this block. Read together, the complete operation is: This calls ‘print‘ with ‘
"NEURAEDGE_TRAINING_COVERAGE_PASSED " f"operators={summary['covered_execution_operator_count']}
" f"learned_groups={summary['learned_affine_group_count']} " f"parameters={summary['trainable_deployed_parameter_elemer
" f"output={output}" ‘ inside the function main, which handles main. The call advances the host-side job that records
which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

Line 32: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 26. Its exact fragment is ‘)‘.
The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation
is: This calls ‘print‘ with ‘ "NEURAEDGE_TRAINING_COVERAGE_PASSED " f"operators={summary['covered_execution_ope
" f"learned_groups={summary['learned_affine_group_count']} " f"parameters={summary['trainable_deployed_parameter_elemer
" f"output={output}" ‘ inside the function main, which handles main. The call advances the host-side job that records
which Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

Line 35: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___
== "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___:
the value named ’main’ in this block. Context: the file whose role is to records which Spikingformer parameters receive
hardware gradients and optimizer updates in each training policy.

Line 36: `main()` This calls ‘main‘ with “ inside the file whose role is to records which Spikingformer parameters receive
hardware gradients and optimizer updates in each training policy. The call advances the host-side job that records which
Spikingformer parameters receive hardware gradients and optimizer updates in each training policy.

34 software/neuraedge_compiler.py

Role in the complete system: tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

34.1 Beginner architecture map

Dataflow responsibility Validated NumPy tensors are tiled into hardware-sized activation and weight loads followed by compute records. The compiler chooses outer or dot8 reuse from traffic, lowers grouped convolution through im2col, packs little-endian stream beats, and rejects dimensions the RTL cannot represent.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: CommandExecutor, ProgramExecutor, TensorInstruction, ResultBinding, CompiledGemm, ConvGroupProgram, CompiledConv2d. Functions: _pack_compiler_u8_rows, _accumulate_tensor_tile, _as_u8_matrix, _u8, _pair, _im2col_nchw, compile_u8_gemm, compile_spike_gemm, compile_u8_conv2d, compile_spike_conv2d, execute, execute_program, execute, execute_batched, execute, execute_batched, mathematical_macs, estimated_input_byte estimated_spike_adds, append_load, append_compute, append_load, activation_load, weight_load. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

34.2 Complete source

```
"""Tiled uint8 GEMM compiler for the NeuraEdge tensor engine.

The compiler turns a mathematical ``(A - za) @ (B - zb) + bias`` operation
into the exact load/compute transactions accepted by the RTL.  It deliberately
keeps compilation separate from execution: plans can be inspected, tested,
serialized, or sent to a PYNQ board through :class:`NeuraEdgeDriver`.
"""

from __future__ import annotations

from dataclasses import dataclass
from typing import Protocol, Sequence

import numpy as np

from neuraedge_golden import Dataflow, choose_dataflow
from neuraedge_protocol import (
    Command128,
    Opcode,
    PackedStreamBeats,
    ProgramRecord,
    StreamBeat,
    tensor_command,
    unpack_tensor_result,
)


ROW_TILE = 8
COLUMN_TILE = 16
MAX_REDUCTION = 2048


def _pack_compiler_u8_rows(values: np.ndarray, row_bytes: int) -> PackedStreamBeats:
    """Pack validated uint8 rows into 64-bit stream words without Python lane loops."""

    array = np.asarray(values)
    if array.dtype != np.uint8 or array.ndim < 2 or array.shape[-1] != row_bytes:
        raise ValueError(f"compiler stream tensor must be uint8 with final dimension {row_bytes}")
    if row_bytes % 8:
        raise ValueError("compiler stream row width must be a multiple of eight")
    lanes = np.ascontiguousarray(array).reshape(-1, 8).astype(np.uint64)
    shifts = (np.arange(8, dtype=np.uint64) * np.uint64(8)).reshape(1, 8)
    words = np.bitwise_or.reduce(lanes << shifts, axis=1).astype("<u8", copy=False)
    return PackedStreamBeats.from_little_endian_bytes(words.tobytes(order="C"))
```

```

class CommandExecutor(Protocol):
    """Smallest driver surface required to run a compiled program."""

    def execute(
        self,
        command: Command128,
        input_beats: Sequence[StreamBeat] = (),
        *,
        output_beats: int = 0,
        timeout_seconds: float = 5.0,
    ) -> tuple[int, ...]: ...


class ProgramExecutor(CommandExecutor, Protocol):
    def execute_program(
        self,
        records: Sequence[ProgramRecord],
        *,
        timeout_seconds: float = 30.0,
    ) -> tuple[int, ...]: ...


@dataclass(frozen=True)
class TensorInstruction:
    """One atomic command, its input stream, and expected output length."""

    command: Command128
    input_beats: Sequence[StreamBeat] = ()
    output_beats: int = 0
    description: str = ""


@dataclass(frozen=True)
class ResultBinding:
    """Maps local row/column result tags back to the destination matrix."""

    instruction_index: int
    row_start: int
    column_start: int
    rows: int
    columns: int
    reduction_start: int
    reduction_count: int


@dataclass(frozen=True)
class CompiledGemm:
    """Executable, inspectable transaction program for one uint8 GEMM."""

    shape: tuple[int, int, int]
    activation_zero_point: int
    weight_zero_point: int
    instructions: tuple[TensorInstruction, ...]
    result_bindings: tuple[ResultBinding, ...]
    bias: tuple[int, ...] | None
    estimated_input_bytes: int
    mathematical_macs: int
    estimated_spike_adds: int = 0

    def execute(self, driver: CommandExecutor, *, timeout_seconds: float = 5.0) -> np.ndarray:
        """Run every transaction and reconstruct the signed int64 matrix."""
        rows, _, columns = self.shape
        output = np.zeros((rows, columns), dtype=np.int64)
        bindings = {binding.instruction_index: binding for binding in self.result_bindings}

        for instruction_index, instruction in enumerate(self.instructions):
            words = driver.execute(
                instruction.command,
                instruction.input_beats,
                output_beats=instruction.output_beats,
                timeout_seconds=timeout_seconds,
            )
            if len(words) != instruction.output_beats:
                raise RuntimeError(
                    f"instruction {instruction_index} returned {len(words)} beats; "
                    f"expected {instruction.output_beats}"
                )
            binding = bindings.get(instruction_index)
            if binding is None:
                continue

            _accumulate_tensor_tile(output, binding, words)

        if self.bias is not None:
            output += np.asarray(self.bias, dtype=np.int64).reshape(1, columns)
```

```
        return output

    def execute_batched(
        self,
        driver: ProgramExecutor,
        *,
        timeout_seconds: float = 30.0,
    ) -> np.ndarray:
        """Run every tile through one descriptor-program DMA transaction."""

        rows, _, columns = self.shape
        records = tuple(ProgramRecord(
            instruction.command,
            instruction.input_beats,
            instruction.output_beats,
        ) for instruction in self.instructions)
        flat_words = driver.execute_program(records, timeout_seconds=timeout_seconds)
        expected_words = sum(instruction.output_beats for instruction in self.instructions)
        if len(flat_words) != expected_words:
            raise RuntimeError(
                f"descriptor program returned {len(flat_words)} beats; expected {expected_words}")
        output = np.zeros((rows, columns), dtype=np.int64)
        bindings = {binding.instruction_index: binding for binding in self.result_bindings}
        word_offset = 0
        for instruction_index, instruction in enumerate(self.instructions):
            word_end = word_offset + instruction.output_beats
            words = flat_words[word_offset:word_end]
            binding = bindings.get(instruction_index)
            if binding is not None:
                _accumulate_tensor_tile(output, binding, words)
            word_offset = word_end
        if self.bias is not None:
            output += np.asarray(self.bias, dtype=np.int64).reshape(1, columns)
        return output


def _accumulate_tensor_tile(
    output: np.ndarray,
    binding: ResultBinding,
    words: Sequence[int],
) -> None:
    seen: set[tuple[int, int]] = set()
    for beat_index, word in enumerate(words):
        value, local_row, local_column, last = unpack_tensor_result(word)
        if local_row >= binding.rows or local_column >= binding.columns:
            raise RuntimeError(
                f"out-of-range tensor tag ({local_row},{local_column}) "
                f"for {binding.rows}x{binding.columns} result tile")
        tag = (local_row, local_column)
        if tag in seen:
            raise RuntimeError(f"duplicate tensor result tag {tag}")
        if last != (beat_index == len(words) - 1):
            raise RuntimeError("tensor TLAST metadata does not match the final result beat")
        seen.add(tag)
        output[binding.row_start + local_row,
               binding.column_start + local_column] += value
    if len(seen) != binding.rows * binding.columns:
        raise RuntimeError("tensor result tile did not contain every row/column tag")


@dataclass(frozen=True)
class ConvGroupProgram:
    output_channel_start: int
    output_channel_count: int
    gemm: CompiledGemm


@dataclass(frozen=True)
class CompiledConv2d:
    """One or more GEMM programs plus the exact NCHW reconstruction map."""

    input_shape: tuple[int, int, int, int]
    output_shape: tuple[int, int, int, int]
    kernel: tuple[int, int]
    stride: tuple[int, int]
    padding: tuple[int, int]
    dilation: tuple[int, int]
    groups: int
    group_programs: tuple[ConvGroupProgram, ...]

    def execute(self, driver: CommandExecutor, *, timeout_seconds: float = 5.0) -> np.ndarray:
        batch, output_channels, output_height, output_width = self.output_shape
        spatial_rows = batch * output_height * output_width
        flat = np.zeros((spatial_rows, output_channels), dtype=np.int64)
        for group in self.group_programs:
            values = group.gemm.execute(driver, timeout_seconds=timeout_seconds)
            start = group.output_channel_start
```

```
            flat[:, start:start + group.output_channel_count] = values
        return flat.reshape(batch, output_height, output_width, output_channels).transpose(0, 3, 1, 2)

    def execute_batched(
        self,
        driver: ProgramExecutor,
        *,
        timeout_seconds: float = 30.0,
    ) -> np.ndarray:
        batch, output_channels, output_height, output_width = self.output_shape
        spatial_rows = batch * output_height * output_width
        flat = np.zeros((spatial_rows, output_channels), dtype=np.int64)
        for group in self.group_programs:
            values = group.gemm.execute_batched(
                driver, timeout_seconds=timeout_seconds)
            start = group.output_channel_start
            flat[:, start:start + group.output_channel_count] = values
        return flat.reshape(batch, output_height, output_width, output_channels).transpose(0, 3, 1, 2)

    @property
    def mathematical_macs(self) -> int:
        return sum(group.gemm.mathematical_macs for group in self.group_programs)

    @property
    def estimated_input_bytes(self) -> int:
        return sum(group.gemm.estimated_input_bytes for group in self.group_programs)

    @property
    def estimated_spike_adds(self) -> int:
        return sum(group.gemm.estimated_spike_adds for group in self.group_programs)


def _as_u8_matrix(name: str, value: np.ndarray | Sequence[Sequence[int]]) -> np.ndarray:
    array = np.asarray(value)
    if array.ndim != 2:
        raise ValueError(f"{name} must be a two-dimensional matrix")
    if array.size == 0:
        raise ValueError(f"{name} must not be empty")
    if not np.issubdtype(array.dtype, np.integer):
        raise ValueError(f"{name} must contain integers")
    as_i64 = array.astype(np.int64)
    if np.any(as_i64 < 0) or np.any(as_i64 > 255):
        raise ValueError(f"{name} entries must be in [0,255]")
    return as_i64.astype(np.uint8)


def _u8(name: str, value: int) -> int:
    integer = int(value)
    if integer < 0 or integer > 255:
        raise ValueError(f"{name} must be in [0,255]")
    return integer


def _pair(name: str, value: int | Sequence[int], *, allow_zero: bool = False) -> tuple[int, int]:
    if isinstance(value, Sequence) and not isinstance(value, (str, bytes)):
        if len(value) != 2:
            raise ValueError(f"{name} must be an integer or a two-element sequence")
        pair = (int(value[0]), int(value[1]))
    else:
        pair = (int(value), int(value))
    minimum = 0 if allow_zero else 1
    if pair[0] < minimum or pair[1] < minimum:
        raise ValueError(f"{name} values must be at least {minimum}")
    return pair


def _im2col_nchw(
    activations: np.ndarray,
    kernel: tuple[int, int],
    stride: tuple[int, int],
    padding: tuple[int, int],
    dilation: tuple[int, int],
    pad_value: int,
) -> tuple[np.ndarray, int, int]:
    batch, channels, height, width = activations.shape
    kernel_height, kernel_width = kernel
    effective_height = dilation[0] * (kernel_height - 1) + 1
    effective_width = dilation[1] * (kernel_width - 1) + 1
    numerator_height = height + 2 * padding[0] - effective_height
    numerator_width = width + 2 * padding[1] - effective_width
    if numerator_height < 0 or numerator_width < 0:
        raise ValueError("effective convolution kernel is larger than the padded input")
    output_height = numerator_height // stride[0] + 1
    output_width = numerator_width // stride[1] + 1
    padded = np.full(
        (batch, channels, height + 2 * padding[0], width + 2 * padding[1]),
        pad_value,
```

```
        dtype=activations.dtype,
    )
    padded[:, :, padding[0]:padding[0] + height, padding[1]:padding[1] + width] = activations
    columns = np.empty(
        (batch * output_height * output_width, channels * kernel_height * kernel_width),
        dtype=activations.dtype,
    )
    row = 0
    for batch_index in range(batch):
        for output_y in range(output_height):
            input_y = output_y * stride[0]
            for output_x in range(output_width):
                input_x = output_x * stride[1]
                column = 0
                for channel in range(channels):
                    for kernel_y in range(kernel_height):
                        y = input_y + kernel_y * dilation[0]
                        for kernel_x in range(kernel_width):
                            x = input_x + kernel_x * dilation[1]
                            columns[row, column] = padded[batch_index, channel, y, x]
                            column += 1
                row += 1
    return columns, output_height, output_width


def compile_u8_gemm(
    activations: np.ndarray | Sequence[Sequence[int]],
    weights: np.ndarray | Sequence[Sequence[int]],
    *,
    activation_zero_point: int = 0,
    weight_zero_point: int = 0,
    bias: Sequence[int] | np.ndarray | None = None,
    dataflow: Dataflow | None = None,
) -> CompiledGemm:
    """Compile an exact asymmetric-uint8 matrix multiplication.

    Reduction dimensions above 2,048 are split into independently corrected
    chunks and accumulated by the runtime.  A single-row matrix uses Dot8 mode;
    all other shapes use the 8x16 outer-product mode.
    """
    a = _as_u8_matrix("activations", activations)
    b = _as_u8_matrix("weights", weights)
    rows, reduction = a.shape
    weight_reduction, columns = b.shape
    if reduction != weight_reduction:
        raise ValueError("activation columns must equal weight rows")
    za = _u8("activation_zero_point", activation_zero_point)
    zb = _u8("weight_zero_point", weight_zero_point)

    bias_tuple: tuple[int, ...] | None = None
    if bias is not None:
        bias_array = np.asarray(bias)
        if bias_array.ndim != 1 or len(bias_array) != columns:
            raise ValueError("bias must be a one-dimensional vector with one entry per output column")
        if not np.issubdtype(bias_array.dtype, np.integer):
            raise ValueError("bias must contain integers")
        bias_tuple = tuple(int(value) for value in bias_array)

    instructions: list[TensorInstruction] = []
    bindings: list[ResultBinding] = []
    estimated_bytes = 0

    def append_load(command: Command128, beats: Sequence[StreamBeat], description: str) -> None:
        nonlocal estimated_bytes
        instructions.append(TensorInstruction(command, beats, 0, description))
        estimated_bytes += 8 * len(beats)

    def append_compute(
        command: Command128,
        row_start: int,
        column_start: int,
        tile_rows: int,
        tile_columns: int,
        reduction_start: int,
        reduction_count: int,
    ) -> None:
        instruction_index = len(instructions)
        instructions.append(TensorInstruction(
            command, (), tile_rows * tile_columns,
            f"compute C[{row_start}:{row_start + tile_rows},"
            f"{column_start}:{column_start + tile_columns}] "
            f"from K[{reduction_start}:{reduction_start + reduction_count}]",
        ))
        bindings.append(ResultBinding(
            instruction_index, row_start, column_start, tile_rows, tile_columns,
            reduction_start, reduction_count,
        ))
```

```
393
394     for reduction_start in range(0, reduction, MAX_REDUCTION):
395         reduction_count = min(MAX_REDUCTION, reduction - reduction_start)
396         selected = dataflow or choose_dataflow(rows, columns, reduction_count).mode
397         if selected not in {Dataflow.INPUT_BROADCAST, Dataflow.WEIGHT_BROADCAST}:
398             raise ValueError(f"unsupported dataflow {selected}")
399         use_dot8 = rows == 1
400         issue_count = (reduction_count + 7) // 8 if use_dot8 else reduction_count
401         load_weight_opcode = Opcode.LOAD_WEIGHTS_DOT8 if use_dot8 else Opcode.LOAD_WEIGHTS_OUTER
402         compute_opcode = Opcode.COMPUTE_DENSE_DOT8 if use_dot8 else Opcode.COMPUTE_DENSE_OUTER
403
404         def activation_load(row_start: int) -> tuple[Command128, PackedStreamBeats]:
405             tile_rows = min(ROW_TILE, rows - row_start)
406             if use_dot8:
407                 padded = np.full(issue_count * 8, za, dtype=np.uint8)
408                 padded[:reduction_count] = a[row_start, reduction_start:reduction_start + reduction_count]
409                 vectors = padded.reshape(issue_count, 8)
410             else:
411                 vectors = np.full((reduction_count, ROW_TILE), za, dtype=np.uint8)
412                 vectors[:, :tile_rows] = a[
413                     row_start:row_start + tile_rows,
414                     reduction_start:reduction_start + reduction_count,
415                 ].T
416             command = tensor_command(
417                 Opcode.LOAD_ACTIVATIONS,
418                 issue_count=issue_count,
419                 activation_zero_point=za,
420             )
421             return command, _pack_compiler_u8_rows(vectors, 8)
422
423         def weight_load(column_start: int) -> tuple[Command128, PackedStreamBeats]:
424             tile_columns = min(COLUMN_TILE, columns - column_start)
425             if use_dot8:
426                 tensor = np.full((issue_count, 8, COLUMN_TILE), zb, dtype=np.uint8)
427                 flat = tensor.reshape(issue_count * 8, COLUMN_TILE)
428                 flat[:reduction_count, :tile_columns] = b[
429                     reduction_start:reduction_start + reduction_count,
430                     column_start:column_start + tile_columns,
431                 ]
432                 beats = _pack_compiler_u8_rows(tensor, 16)
433             else:
434                 matrix = np.full((reduction_count, COLUMN_TILE), zb, dtype=np.uint8)
435                 matrix[:, :tile_columns] = b[
436                     reduction_start:reduction_start + reduction_count,
437                     column_start:column_start + tile_columns,
438                 ]
439                 beats = _pack_compiler_u8_rows(matrix, 16)
440             command = tensor_command(load_weight_opcode, issue_count=issue_count)
441             return command, beats
442
443         def compute(row_start: int, column_start: int) -> None:
444             tile_rows = min(ROW_TILE, rows - row_start)
445             tile_columns = min(COLUMN_TILE, columns - column_start)
446             command = tensor_command(
447                 compute_opcode,
448                 rows=tile_rows,
449                 columns=tile_columns,
450                 issue_count=issue_count,
451                 activation_zero_point=za,
452                 weight_zero_point=zb,
453             )
454             append_compute(
455                 command, row_start, column_start, tile_rows, tile_columns,
456                 reduction_start, reduction_count,
457             )
458
459         if selected == Dataflow.INPUT_BROADCAST:
460             for row_start in range(0, rows, ROW_TILE):
461                 command, beats = activation_load(row_start)
462                 append_load(command, beats, f"load activation tile at row {row_start}")
463                 for column_start in range(0, columns, COLUMN_TILE):
464                     command, beats = weight_load(column_start)
465                     append_load(command, beats, f"load weight tile at column {column_start}")
466                     compute(row_start, column_start)
467         else:
468             for column_start in range(0, columns, COLUMN_TILE):
469                 command, beats = weight_load(column_start)
470                 append_load(command, beats, f"load weight tile at column {column_start}")
471                 for row_start in range(0, rows, ROW_TILE):
472                     command, beats = activation_load(row_start)
473                     append_load(command, beats, f"load activation tile at row {row_start}")
474                     compute(row_start, column_start)
475
476     return CompiledGemm(
477         (rows, reduction, columns), za, zb, tuple(instructions), tuple(bindings),
478         bias_tuple, estimated_bytes, rows * reduction * columns,
479     )
```

```

def compile_spike_gemm(
    spikes: np.ndarray | Sequence[Sequence[int]],
    weights: np.ndarray | Sequence[Sequence[int]],
    *,
    ternary: bool = False,
    bias: Sequence[int] | np.ndarray | None = None,
    dataflow: Dataflow | None = None,
) -> CompiledGemm:
    """Compile binary or ternary spikes times signed INT8 weights.

    The RTL performs one signed addition for each non-zero spike/output-column
    pair and performs no multiplication.  Ternary ``-1`` spikes select
    subtraction.  Zero-spike K positions are compacted by the activation-load
    pipeline and do not reach the tensor tile.
    """
    spike_array = np.asarray(spikes)
    weight_array = np.asarray(weights)
    if spike_array.ndim != 2 or weight_array.ndim != 2:
        raise ValueError("spikes and weights must be two-dimensional matrices")
    if spike_array.size == 0 or weight_array.size == 0:
        raise ValueError("spikes and weights must not be empty")
    if not np.issubdtype(spike_array.dtype, np.integer) or not np.issubdtype(weight_array.dtype, np.integer):
        raise ValueError("spikes and weights must contain integers")
    spike_i64 = spike_array.astype(np.int64)
    weight_i64 = weight_array.astype(np.int64)
    allowed = np.isin(spike_i64, (-1, 0, 1) if ternary else (0, 1))
    if not np.all(allowed):
        expected = "{-1,0,1}" if ternary else "{0,1}"
        raise ValueError(f"spikes must be in {expected}")
    if np.any(weight_i64 < -128) or np.any(weight_i64 > 127):
        raise ValueError("sparse-mode weights must be signed INT8 values")

    rows, reduction = spike_i64.shape
    weight_reduction, columns = weight_i64.shape
    if reduction != weight_reduction:
        raise ValueError("spike columns must equal weight rows")
    bias_tuple: tuple[int, ...] | None = None
    if bias is not None:
        bias_array = np.asarray(bias)
        if bias_array.ndim != 1 or len(bias_array) != columns:
            raise ValueError("bias must have one entry per output column")
        if not np.issubdtype(bias_array.dtype, np.integer):
            raise ValueError("bias must contain integers")
        bias_tuple = tuple(int(value) for value in bias_array)

    encoded_spikes = (spike_i64 & 0xFF).astype(np.uint8)
    encoded_weights = (weight_i64 & 0xFF).astype(np.uint8)
    instructions: list[TensorInstruction] = []
    bindings: list[ResultBinding] = []
    estimated_bytes = 0
    compute_opcode = Opcode.COMPUTE_TERNARY if ternary else Opcode.COMPUTE_BINARY

    def append_load(command: Command128, beats: Sequence[StreamBeat], description: str) -> None:
        nonlocal estimated_bytes
        instructions.append(TensorInstruction(command, beats, 0, description))
        estimated_bytes += 8 * len(beats)

    for reduction_start in range(0, reduction, MAX_REDUCTION):
        reduction_count = min(MAX_REDUCTION, reduction - reduction_start)
        selected = dataflow or choose_dataflow(rows, columns, reduction_count).mode
        if selected not in {Dataflow.INPUT_BROADCAST, Dataflow.WEIGHT_BROADCAST}:
            raise ValueError(f"unsupported dataflow {selected}")

        def load_activations(row_start: int) -> None:
            tile_rows = min(ROW_TILE, rows - row_start)
            vectors = np.zeros((reduction_count, ROW_TILE), dtype=np.uint8)
            vectors[:, :tile_rows] = encoded_spikes[
                row_start:row_start + tile_rows,
                reduction_start:reduction_start + reduction_count,
            ].T
            append_load(
                tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=reduction_count),
                _pack_compiler_u8_rows(vectors, 8),
                f"load {'ternary' if ternary else 'binary'} spike tile at row {row_start}",
            )

        def load_weights(column_start: int) -> None:
            tile_columns = min(COLUMN_TILE, columns - column_start)
            matrix = np.zeros((reduction_count, COLUMN_TILE), dtype=np.uint8)
            matrix[:, :tile_columns] = encoded_weights[
                reduction_start:reduction_start + reduction_count,
                column_start:column_start + tile_columns,
            ]
            append_load(
                tensor_command(Opcode.LOAD_WEIGHTS_OUTER, issue_count=reduction_count),
```

```
                _pack_compiler_u8_rows(matrix, 16),
                f"load signed weight tile at column {column_start}",
            )

        def append_compute(row_start: int, column_start: int) -> None:
            tile_rows = min(ROW_TILE, rows - row_start)
            tile_columns = min(COLUMN_TILE, columns - column_start)
            instruction_index = len(instructions)
            instructions.append(TensorInstruction(
                tensor_command(
                    compute_opcode,
                    rows=tile_rows,
                    columns=tile_columns,
                    issue_count=reduction_count,
                ),
                (),
                tile_rows * tile_columns,
                f"compute sparse C tile at ({row_start},{column_start})",
            ))
            bindings.append(ResultBinding(
                instruction_index, row_start, column_start, tile_rows, tile_columns,
                reduction_start, reduction_count,
            ))

        if selected == Dataflow.INPUT_BROADCAST:
            for row_start in range(0, rows, ROW_TILE):
                load_activations(row_start)
                for column_start in range(0, columns, COLUMN_TILE):
                    load_weights(column_start)
                    append_compute(row_start, column_start)
        else:
            for column_start in range(0, columns, COLUMN_TILE):
                load_weights(column_start)
                for row_start in range(0, rows, ROW_TILE):
                    load_activations(row_start)
                    append_compute(row_start, column_start)

    spike_adds = int(np.count_nonzero(spike_i64)) * columns
    return CompiledGemm(
        (rows, reduction, columns), 0, 0, tuple(instructions), tuple(bindings),
        bias_tuple, estimated_bytes, rows * reduction * columns, spike_adds,
    )


def compile_u8_conv2d(
    activations: np.ndarray | Sequence,
    weights: np.ndarray | Sequence,
    *,
    activation_zero_point: int = 0,
    weight_zero_point: int = 0,
    bias: Sequence[int] | np.ndarray | None = None,
    stride: int | Sequence[int] = 1,
    padding: int | Sequence[int] = 0,
    dilation: int | Sequence[int] = 1,
    groups: int = 1,
    dataflow: Dataflow | None = None,
) -> CompiledConv2d:
    """Lower an NCHW/OIHW asymmetric-uint8 convolution to tensor commands."""
    a = np.asarray(activations)
    w = np.asarray(weights)
    if a.ndim != 4 or w.ndim != 4:
        raise ValueError("convolution activations and weights must be NCHW and OIHW tensors")
    if not np.issubdtype(a.dtype, np.integer) or not np.issubdtype(w.dtype, np.integer):
        raise ValueError("convolution activations and weights must contain integers")
    a_i64 = a.astype(np.int64)
    w_i64 = w.astype(np.int64)
    if np.any(a_i64 < 0) or np.any(a_i64 > 255):
        raise ValueError("convolution activations must be in [0,255]")
    if np.any(w_i64 < 0) or np.any(w_i64 > 255):
        raise ValueError("convolution weights must be in [0,255]")
    a_u8 = a_i64.astype(np.uint8)
    w_u8 = w_i64.astype(np.uint8)
    za = _u8("activation_zero_point", activation_zero_point)
    zb = _u8("weight_zero_point", weight_zero_point)
    stride_pair = _pair("stride", stride)
    padding_pair = _pair("padding", padding, allow_zero=True)
    dilation_pair = _pair("dilation", dilation)

    batch, input_channels, height, width = a_u8.shape
    output_channels, weight_channels, kernel_height, kernel_width = w_u8.shape
    group_count = int(groups)
    if group_count <= 0 or input_channels % group_count or output_channels % group_count:
        raise ValueError("groups must divide both input and output channels")
    channels_per_group = input_channels // group_count
    outputs_per_group = output_channels // group_count
    if weight_channels != channels_per_group:
        raise ValueError("OIHW weight input channels must equal input_channels/groups")
```

```
    if kernel_height == 0 or kernel_width == 0:
        raise ValueError("convolution kernel dimensions must be non-zero")
    if bias is None:
        bias_values = None
    else:
        bias_array = np.asarray(bias)
        if bias_array.ndim != 1 or len(bias_array) != output_channels:
            raise ValueError("convolution bias must have one entry per output channel")
        if not np.issubdtype(bias_array.dtype, np.integer):
            raise ValueError("convolution bias must contain integers")
        bias_values = bias_array.astype(np.int64)

    programs: list[ConvGroupProgram] = []
    output_height = output_width = 0
    for group in range(group_count):
        input_start = group * channels_per_group
        output_start = group * outputs_per_group
        columns, output_height, output_width = _im2col_nchw(
            a_u8[:, input_start:input_start + channels_per_group],
            (kernel_height, kernel_width), stride_pair, padding_pair,
            dilation_pair, za,
        )
        weight_matrix = w_u8[
            output_start:output_start + outputs_per_group
        ].reshape(outputs_per_group, -1).T
        group_bias = None if bias_values is None else bias_values[
            output_start:output_start + outputs_per_group
        ]
        programs.append(ConvGroupProgram(
            output_start,
            outputs_per_group,
            compile_u8_gemm(
                columns,
                weight_matrix,
                activation_zero_point=za,
                weight_zero_point=zb,
                bias=group_bias,
                dataflow=dataflow,
            ),
        ))
    return CompiledConv2d(
        (batch, input_channels, height, width),
        (batch, output_channels, output_height, output_width),
        (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair,
        group_count, tuple(programs),
    )


def compile_spike_conv2d(
    spikes: np.ndarray | Sequence,
    weights: np.ndarray | Sequence,
    *,
    ternary: bool = False,
    bias: Sequence[int] | np.ndarray | None = None,
    stride: int | Sequence[int] = 1,
    padding: int | Sequence[int] = 0,
    dilation: int | Sequence[int] = 1,
    groups: int = 1,
    dataflow: Dataflow | None = None,
) -> CompiledConv2d:
    """Lower grouped binary/ternary spike convolution to addition-only GEMMs."""
    spike_array = np.asarray(spikes)
    weight_array = np.asarray(weights)
    if spike_array.ndim != 4 or weight_array.ndim != 4:
        raise ValueError("spike convolution requires NCHW activations and OIHW weights")
    if not np.issubdtype(spike_array.dtype, np.integer) or not np.issubdtype(weight_array.dtype, np.integer):
        raise ValueError("spike convolution tensors must contain integers")
    spike_i64 = spike_array.astype(np.int64)
    weight_i64 = weight_array.astype(np.int64)
    if not np.all(np.isin(spike_i64, (-1, 0, 1) if ternary else (0, 1))):
        raise ValueError("spike values do not match the selected binary/ternary mode")
    if np.any(weight_i64 < -128) or np.any(weight_i64 > 127):
        raise ValueError("spike convolution weights must be signed INT8")
    stride_pair = _pair("stride", stride)
    padding_pair = _pair("padding", padding, allow_zero=True)
    dilation_pair = _pair("dilation", dilation)
    batch, input_channels, height, width = spike_i64.shape
    output_channels, weight_channels, kernel_height, kernel_width = weight_i64.shape
    group_count = int(groups)
    if group_count <= 0 or input_channels % group_count or output_channels % group_count:
        raise ValueError("groups must divide both input and output channels")
    channels_per_group = input_channels // group_count
    outputs_per_group = output_channels // group_count
    if weight_channels != channels_per_group:
        raise ValueError("OIHW weight input channels must equal input_channels/groups")
    if bias is None:
        bias_values = None
```

```
    else:
        bias_array = np.asarray(bias)
        if bias_array.ndim != 1 or len(bias_array) != output_channels:
            raise ValueError("spike convolution bias must have one entry per output channel")
        if not np.issubdtype(bias_array.dtype, np.integer):
            raise ValueError("spike convolution bias must contain integers")
        bias_values = bias_array.astype(np.int64)

    programs: list[ConvGroupProgram] = []
    output_height = output_width = 0
    for group in range(group_count):
        input_start = group * channels_per_group
        output_start = group * outputs_per_group
        columns, output_height, output_width = _im2col_nchw(
            spike_i64[:, input_start:input_start + channels_per_group],
            (kernel_height, kernel_width), stride_pair, padding_pair,
            dilation_pair, 0,
        )
        weight_matrix = weight_i64[
            output_start:output_start + outputs_per_group
        ].reshape(outputs_per_group, -1).T
        group_bias = None if bias_values is None else bias_values[
            output_start:output_start + outputs_per_group
        ]
        programs.append(ConvGroupProgram(
            output_start,
            outputs_per_group,
            compile_spike_gemm(
                columns,
                weight_matrix,
                ternary=ternary,
                bias=group_bias,
                dataflow=dataflow,
            ),
        ))
    return CompiledConv2d(
        (batch, input_channels, height, width),
        (batch, output_channels, output_height, output_width),
        (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair,
        group_count, tuple(programs),
    )
```

34.3 Line annotations

Line 1: `"""Tiled uint8 GEMM compiler for the NeuraEdge tensor engine.` This is executable documentation attached to the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `The compiler turns a mathematical "(A - za) @ (B - zb) + bias" operation` Inside the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are The: the value named 'the' in this block; compiler: the value named 'compiler' in this block; turns: the value named 'turns' in this block.

Line 4: `into the exact load/compute transactions accepted by the RTL. It deliberately` Inside the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are into: the value named 'into' in this block; the: the value named 'the' in this block; exact: the value named 'exact' in this block.

Line 5: `keeps compilation separate from execution: plans can be inspected, tested,` Inside the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are keeps: the value named 'keeps' in this block; compilation: the value named 'compilation' in this block; separate: the value named 'separate' in this block.

Line 6: `serialized, or sent to a PYNQ board through :class:‘NeuraEdgeDriver‘.` Inside the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads, this statement

advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are serialized: the value named 'serialized' in this block; or: the value named 'or' in this block; sent: the value named 'sent' in this block.

Line 7: `"""` This is executable documentation attached to the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 9: `from __future__ import annotations` This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `from typing import Protocol, Sequence` This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `import numpy as np` This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `from neuraedge_golden import Dataflow, choose_dataflow` This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `from neuraedge_protocol import (` This statement continues through source line 25; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `Command128,` This is continuation line 2 of the 9-line logical statement that begins at source line 17. Its exact fragment is 'Command128,'. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `Opcode,` This is continuation line 3 of the 9-line logical statement that begins at source line 17. Its exact fragment is 'Opcode,'. The important operands are Opcode: the value named 'opcode' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `PackedStreamBeats,` This is continuation line 4 of the 9-line logical statement that begins at source line 17. Its exact fragment is 'PackedStreamBeats,'. The important operands are PackedStreamBeats: the value named 'packed stream beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `ProgramRecord,` This is continuation line 5 of the 9-line logical statement that begins at source line 17. Its exact fragment is 'ProgramRecord,'. The important operands are ProgramRecord: the value named 'program record' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `StreamBeat,` This is continuation line 6 of the 9-line logical statement that begins at source line 17. Its exact fragment is 'StreamBeat,'. The important operands are StreamBeat: the value named 'stream beat' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `tensor_command,` This is continuation line 7 of the 9-line logical statement that begins at source line 17. Its exact fragment is 'tensor_command,'. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `unpack_tensor_result,` This is continuation line 8 of the 9-line logical statement that begins at source line 17. Its exact fragment is 'unpack_tensor_result,'. The important operands are unpack_tensor_result: the value named 'unpack tensor result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 25: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 17. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 28: `ROW_TILE = 8` This assigns '8' to 'ROW_TILE ' in host memory. Here ROW_TILE means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 29: `COLUMN_TILE = 16` This assigns '16' to 'COLUMN_TILE ' in host memory. Here COLUMN_TILE means one output-feature position processed in parallel. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 30: `MAX_REDUCTION = 2048` This assigns '2048' to 'MAX_REDUCTION ' in host memory. Here MAX_REDUCTION means the value named 'max reduction' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 33: `def _pack_compiler_u8_rows(values: np.ndarray, row_bytes: int) -> PackedStreamBeats:` This starts function _pack_compiler_u8_rows. Its indented body performs pack compiler u8 rows as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 34: `"""Pack validated uint8 rows into 64-bit stream words without Python lane loops."""` This is executable documentation attached to the function _pack_compiler_u8_rows, which handles pack compiler u8 rows. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 36: `array = np.asarray(values)` This assigns ‘np.asarray(values)‘ to ‘array ‘ in host memory. Here array means the value named ’array’ in this block. The important operands are asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block. Context: the function _pack_compiler_u8_rows, which handles pack compiler u8 rows.

Line 37: `if array.dtype != np.uint8 or array.ndim < 2 or array.shape[-1] != row_bytes:` This executes the following indented host-code branch only when ‘array.dtype != np.uint8 or array.ndim < 2 or array.shape[-1] != row_bytes‘ is true. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function _pack_compiler_u8_rows, which handles pack compiler u8 rows.

Line 38: `raise ValueError(f"compiler stream tensor must be uint8 with final dimension {row_bytes}")` This rejects an invalid state by raising ‘ValueError(f"compiler stream tensor must be uint8 with final dimension {row_bytes}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 39: `if row_bytes % 8:` This executes the following indented host-code branch only when ‘row_bytes % 8‘ is true. The important operands are row_bytes: one output-row position processed by the tensor tile. Context: the function _pack_compiler_u8_rows, which handles pack compiler u8 rows.

Line 40: `raise ValueError("compiler stream row width must be a multiple of eight")` This rejects an invalid state by raising ‘ValueError("compiler stream row width must be a multiple of eight")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 41: `lanes = np.ascontiguousarray(array).reshape(-1, 8).astype(np.uint64)` This assigns ‘np.ascontiguousarray(ar 1, 8).astype(np.uint64)‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are ascontiguousarray: the value named ’ascontiguousarray’ in this block; array: the value named ’array’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _pack_compiler_u8_rows, which handles pack compiler u8 rows.

Line 42: `shifts = (np.arange(8, dtype=np.uint64) * np.uint64(8)).reshape(1, 8)` This assigns ‘(np.arange(8, dtype=np.uint64) * np.uint64(8)).reshape(1, 8)‘ to ‘shifts ‘ in host memory. Here shifts means the value named ’shifts’ in this block. The important operands are arange: the value named ’arange’ in this block; dtype: the value named ’dtype’ in this block; uint64: the value named ’uint64’ in this block. Context: the function _pack_compiler_u8_rows, which handles pack compiler u8 rows.

Line 43: `words = np.bitwise_or.reduce(lanes << shifts, axis=1).astype("<u8", copy=False)` This assigns ‘np.bitwise_or.reduce(lanes << shifts, axis=1).astype("<u8", copy=False)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are bitwise_or: the value named ’bitwise or’ in this block; reduce: the value named ’reduce’ in this block; lanes: one parallel arithmetic element within a vector or tensor datapath. Context: the function _pack_compiler_u8_rows, which handles pack compiler u8 rows.

Line 44: `return PackedStreamBeats.from_little_endian_bytes(words.tobytes(order="C"))` This returns ‘PackedStreamBeats.from_little_endian_bytes(words.tobytes(order="C"))‘ to the caller of the function _pack_compiler_u8_rows, which handles pack compiler u8 rows, ending this invocation. The important operands are PackedStreamBeats: the value named ’packed stream beats’ in this block; from_little_endian_bytes: the value named ’from little endian bytes’ in this block; words: the value named ’words’ in this block.

Line 47: `class CommandExecutor(Protocol):` This defines class CommandExecutor, which groups data and methods for command executor. Instances exist in host memory and organize the flow that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 48: `"""Smallest driver surface required to run a compiled program."""` This is executable documentation attached to the class CommandExecutor, which handles command executor. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 50: `def execute(` This starts function execute. Its indented body performs execute as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 51: `self,` Inside the function execute, which handles execute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 52: `command: Command128,` Inside the function execute, which handles execute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 53: `input_beats: Sequence[StreamBeat] = (),` This assigns '(),' to 'input_beats' in host memory. Here input_beats means the value named 'input beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 54: `*,` Inside the function execute, which handles execute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 55: `output_beats: int = 0,` This assigns '0,' to 'output_beats' in host memory. Here output_beats means the value named 'output beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 56: `timeout_seconds: float = 5.0,` This assigns '5.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 57: `) -> tuple[int, ...]: ...` Inside the function execute, which handles execute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are tuple: the value named 'tuple' in this block.

Line 60: `class ProgramExecutor(CommandExecutor, Protocol):` This defines class ProgramExecutor, which groups data and methods for program executor. Instances exist in host memory and organize the flow that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 61: `def execute_program(` This starts function execute_program. Its indented body performs execute program as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 62: `self,` Inside the function execute_program, which handles execute program, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 63: `records: Sequence[ProgramRecord],` Inside the function execute_program, which handles execute program, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are records: the number of descriptor records accepted since program start; Sequence: the value named 'sequence' in this block; ProgramRecord: the value named 'program record' in this block.

Line 64: `*,` Inside the function execute_program, which handles execute program, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 65: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 66: `) -> tuple[int, ...]: ...` Inside the function execute_program, which handles execute program, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are tuple: the value named ’tuple’ in this block.

Line 69: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it creates no FPGA circuit by itself.

Line 70: `class TensorInstruction:` This defines class TensorInstruction, which groups data and methods for tensor instruction. Instances exist in host memory and organize the flow that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 71: `"""One atomic command, its input stream, and expected output length."""` This is executable documentation attached to the class TensorInstruction, which handles tensor instruction. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 73: `command: Command128` Inside the class TensorInstruction, which handles tensor instruction, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 74: `input_beats: Sequence[StreamBeat] = ()` This assigns ‘()‘ to ‘input_beats‘ in host memory. Here input_beats means the value named ’input beats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class TensorInstruction, which handles tensor instruction.

Line 75: `output_beats: int = 0` This assigns ‘0‘ to ‘output_beats‘ in host memory. Here output_beats means the value named ’output beats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class TensorInstruction, which handles tensor instruction.

Line 76: `description: str = ""` This assigns ‘""‘ to ‘description‘ in host memory. Here description means the value named ’description’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class TensorInstruction, which handles tensor instruction.

Line 79: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it creates no FPGA circuit by itself.

Line 80: `class ResultBinding:` This defines class ResultBinding, which groups data and methods for result binding. Instances exist in host memory and organize the flow that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 81: `"""Maps local row/column result tags back to the destination matrix."""` This is executable documentation attached to the class ResultBinding, which handles result binding. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 83: `instruction_index: int` Inside the class ResultBinding, which handles result binding, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are instruction_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 84: `row_start: int` Inside the class ResultBinding, which handles result binding, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are row_start: one output-row position processed by the tensor tile.

Line 85: `column_start: int` Inside the class ResultBinding, which handles result binding, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are column_start: one output-feature position processed in parallel.

Line 86: `rows: int` Inside the class ResultBinding, which handles result binding, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are rows: one output-row position processed by the tensor tile.

Line 87: `columns: int` Inside the class ResultBinding, which handles result binding, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are columns: one output-feature position processed in parallel.

Line 88: `reduction_start: int` Inside the class ResultBinding, which handles result binding, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are reduction_start: the value named 'reduction start' in this block.

Line 89: `reduction_count: int` Inside the class ResultBinding, which handles result binding, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are reduction_count: an integer count retained for control or measurable telemetry.

Line 92: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it creates no FPGA circuit by itself.

Line 93: `class CompiledGemm:` This defines class CompiledGemm, which groups data and methods for compiled gemm. Instances exist in host memory and organize the flow that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 94: `"""Executable, inspectable transaction program for one uint8 GEMM."""` This is executable documentation attached to the class CompiledGemm, which handles compiled gemm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 96: `shape: tuple[int, int, int]` Inside the class CompiledGemm, which handles compiled gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are shape: the value named 'shape' in this block; tuple: the value named 'tuple' in this block.

Line 97: `activation_zero_point: int` Inside the class CompiledGemm, which handles compiled gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 98: `weight_zero_point: int` Inside the class CompiledGemm, which handles compiled gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are weight_zero_point: a learned coefficient multiplied by an activation.

Line 99: `instructions: tuple[TensorInstruction, ...]` Inside the class CompiledGemm, which handles compiled gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are instructions: the value named 'instructions' in this block; tuple: the value named 'tuple' in this block; TensorInstruction: the value named 'tensor instruction' in this block.

Line 100: `result_bindings: tuple[ResultBinding, ...]` Inside the class CompiledGemm, which handles compiled gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are result_bindings: the value named 'result bindings' in this block; tuple: the value named 'tuple' in this block; ResultBinding: the value named 'result binding' in this block.

Line 101: `bias: tuple[int, ...] | None` Inside the class CompiledGemm, which handles compiled gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are bias: an additive learned offset applied after accumulation; tuple: the value named 'tuple' in this block; None: the value named 'none' in this block.

Line 102: `estimated_input_bytes: int` Inside the class CompiledGemm, which handles compiled gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are estimated_input_bytes: the value named 'estimated input bytes' in this block.

Line 103: `mathematical_macs: int` Inside the class CompiledGemm, which handles compiled gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are mathematical_macs: the value named 'mathematical macs' in this block.

Line 104: `estimated_spike_adds: int = 0` This assigns ‘0‘ to ‘estimated_spike_adds‘ in host memory. Here estimated_spike_adds means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the class CompiledGemm, which handles compiled gemm.

Line 106: `def execute(self, driver: CommandExecutor, *, timeout_seconds: float = 5.0) -> np.ndarray:` This starts function execute. Its indented body performs execute as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 107: `"""Run every transaction and reconstruct the signed int64 matrix."""` This is executable documentation attached to the function execute, which handles execute. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 108: `rows, _, columns = self.shape` This assigns ‘self.shape‘ to ‘rows, _, columns ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are shape: the value named 'shape' in this block. Context: the function execute, which handles execute.

Line 109: `output = np.zeros((rows, columns), dtype=np.int64)` This assigns ‘np.zeros((rows, columns), dtype=np.int64)‘ to ‘output ‘ in host memory. Here output means the value named 'output' in this block. The important operands are zeros: the value named 'zeros' in this block; rows: one output-row position processed by the tensor tile; columns: one output-feature position processed in parallel. Context: the function execute, which handles execute.

Line 110: `bindings = {binding.instruction_index: binding for binding in self.result_bindings}` This assigns ‘{binding.instruction_index: binding for binding in self.result_bindings}‘ to ‘bindings ‘ in host memory. Here bindings means the value named 'bindings' in this block. The important operands are binding: the value named 'binding' in this block; instruction_index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block. Context: the function execute, which handles execute.

Line 112: `for instruction_index, instruction in enumerate(self.instructions):` This iterates instruction_index, instruction over ‘enumerate(self.instructions)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 113: `words = driver.execute(` This statement continues through source line 118; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘driver.execute(instruction.command, instruction.input_beats, output_beats=instruction.output_beats, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute, which handles execute.

Line 114: `instruction.command,` This is continuation line 2 of the 6-line logical statement that begins at source line 113. Its exact fragment is ‘instruction.command,‘. The important operands are instruction: the value named ’instruction’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘driver.execute(instruction.command, instruction.input_beats, output_beats=instruction.output_beats, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute, which handles execute.

Line 115: `instruction.input_beats,` This is continuation line 3 of the 6-line logical statement that begins at source line 113. Its exact fragment is ‘instruction.input_beats,‘. The important operands are instruction: the value named ’instruction’ in this block; input_beats: the value named ’input beats’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(instruction.command, instruction.input_beats, output_beats=instruction.output_beats, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute, which handles execute.

Line 116: `output_beats=instruction.output_beats,` This is continuation line 4 of the 6-line logical statement that begins at source line 113. Its exact fragment is ‘output_beats=instruction.output_beats,‘. The important operands are output_beats: the value named ’output beats’ in this block; instruction: the value named ’instruction’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(instruction.command, instruction.input_beats, output_beats=instruction.output_beats, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute, which handles execute.

Line 117: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 113. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(instruction.command, instruction.input_beats, output_beats=instruction.output_beats, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute, which handles execute.

Line 118: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 113. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘driver.execute(instruction.command, instruction.input_beats, output_beats=instruction.output_beats, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute, which handles execute.

Line 119: `if len(words) != instruction.output_beats:` This executes the following indented host-code branch only when ‘len(words) != instruction.output_beats‘ is true. The important operands are len: the value named ’len’ in

this block; words: the value named 'words' in this block; instruction: the value named 'instruction' in this block. Context: the function execute, which handles execute.

Line 120: `raise RuntimeError(` This statement continues through source line 123; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘RuntimeError(f"instruction {instruction_index} returned {len(words)} beats; " f"expected {instruction.output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 121: `f"instruction {instruction_index} returned {len(words)} beats; "` This is continuation line 2 of the 4-line logical statement that begins at source line 120. Its exact fragment is ‘f"instruction {instruction_index} returned {len(words)} beats; "‘. The important operands are f: the value named 'f' in this block; instruction: the value named 'instruction' in this block; instruction_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"instruction {instruction_index} returned {len(words)} beats; " f"expected {instruction.output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 122: `f"expected {instruction.output_beats}"` This is continuation line 3 of the 4-line logical statement that begins at source line 120. Its exact fragment is ‘f"expected {instruction.output_beats}"‘. The important operands are f: the value named 'f' in this block; expected: the value named 'expected' in this block; instruction: the value named 'instruction' in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"instruction {instruction_index} returned {len(words)} beats; " f"expected {instruction.output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 123: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 120. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"instruction {instruction_index} returned {len(words)} beats; " f"expected {instruction.output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 124: `binding = bindings.get(instruction_index)` This assigns ‘bindings.get(instruction_index)‘ to ‘binding ‘ in host memory. Here binding means the value named 'binding' in this block. The important operands are bindings: the value named 'bindings' in this block; get: the value named 'get' in this block; instruction_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function execute, which handles execute.

Line 125: `if binding is None:` This executes the following indented host-code branch only when ‘binding is None‘ is true. The important operands are binding: the value named 'binding' in this block; is: the value named 'is' in this block; None: the value named 'none' in this block. Context: the function execute, which handles execute.

Line 126: `continue` Inside the function execute, which handles execute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are continue: the value named 'continue' in this block.

Line 128: `_accumulate_tensor_tile(output, binding, words)` This calls ‘_accumulate_tensor_tile‘ with ‘output, binding, words‘ inside the function execute, which handles execute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 130: `if self.bias is not None:` This executes the following indented host-code branch only when ‘self.bias is not None‘ is true. The important operands are bias: an additive learned offset applied after accumulation; is: the value named 'is' in this block; not: the value named 'not' in this block. Context: the function execute, which handles execute.

Line 131: `output += np.asarray(self.bias, dtype=np.int64).reshape(1, columns)` Inside the function execute, which handles execute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are output: the value named 'output' in this block; asarray: the value named 'asarray' in this block; bias: an additive learned offset applied after accumulation.

Line 132: `return output` This returns ‘output‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are output: the value named ’output’ in this block.

Line 134: `def execute_batched(` This starts function execute_batched. Its indented body performs execute batched as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 135: `self,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 136: `driver: ProgramExecutor,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are driver: the value named ’driver’ in this block; ProgramExecutor: the value named ’program executor’ in this block.

Line 137: `*,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 138: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 139: `) -> np.ndarray:` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are ndarray: the value named ’ndarray’ in this block.

Line 140: `"""Run every tile through one descriptor-program DMA transaction."""` This is executable documentation attached to the function execute_batched, which handles execute batched. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 142: `rows, _, columns = self.shape` This assigns ‘self.shape‘ to ‘rows, _, columns ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are shape: the value named ’shape’ in this block. Context: the function execute_batched, which handles execute batched.

Line 143: `records = tuple(ProgramRecord(` This statement continues through source line 147; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘tuple(ProgramRecord(instruction.command, instruction.input_beats, instruction.output_beats,) for instruction in self.instructions)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named ’tuple’ in this block; ProgramRecord: the value named ’program record’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute_batched, which handles execute batched.

Line 144: `instruction.command,` This is continuation line 2 of the 5-line logical statement that begins at source line 143. Its exact fragment is ‘instruction.command,‘. The important operands are instruction: the value named ’instruction’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘tuple(ProgramRecord(instruction.command, instruction.input_beats, instruction.output_beats,) for instruction in self.instructions)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named ’tuple’ in this block; ProgramRecord: the value named ’program record’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute_batched, which handles execute batched.

Line 145: `instruction.input_beats,` This is continuation line 3 of the 5-line logical statement that begins at source line 143. Its exact fragment is ‘instruction.input_beats,‘. The important operands are instruction: the value named ’instruction’ in this block; input_beats: the value named ’input beats’ in this block. Read together, the complete operation is: This assigns ‘tuple(ProgramRecord(instruction.command, instruction.input_beats, instruction.output_beats,) for instruction in self.instructions)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named ’tuple’ in this block; ProgramRecord: the value named ’program record’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute_batched, which handles execute batched.

Line 146: `instruction.output_beats,` This is continuation line 4 of the 5-line logical statement that begins at source line 143. Its exact fragment is ‘instruction.output_beats,‘. The important operands are instruction: the value named ’instruction’ in this block; output_beats: the value named ’output beats’ in this block. Read together, the complete operation is: This assigns ‘tuple(ProgramRecord(instruction.command, instruction.input_beats, instruction.output_beats,) for instruction in self.instructions)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named ’tuple’ in this block; ProgramRecord: the value named ’program record’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute_batched, which handles execute batched.

Line 147: `)  for instruction in self.instructions)` This is continuation line 5 of the 5-line logical statement that begins at source line 143. Its exact fragment is ‘) for instruction in self.instructions)‘. The important operands are for: the value named ’for’ in this block; instruction: the value named ’instruction’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘tuple(ProgramRecord(instruction.command, instruction.input_beats, instruction.output_beats,) for instruction in self.instructions)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named ’tuple’ in this block; ProgramRecord: the value named ’program record’ in this block; instruction: the value named ’instruction’ in this block. Context: the function execute_batched, which handles execute batched.

Line 148: `flat_words = driver.execute_program(records, timeout_seconds=timeout_seconds)` This assigns ‘driver.execute_program(records, timeout_seconds=timeout_seconds)‘ to ‘flat_words ‘ in host memory. Here flat_words means the value named ’flat words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start. Context: the function execute_batched, which handles execute batched.

Line 149: `expected_words = sum(instruction.output_beats for instruction in self.instructions)` This assigns ‘sum(instruction.output_beats for instruction in self.instructions)‘ to ‘expected_words ‘ in host memory. Here expected_words means the value named ’expected words’ in this block. The important operands are sum: the value named ’sum’ in this block; instruction: the value named ’instruction’ in this block; output_beats: the value named ’output beats’ in this block. Context: the function execute_batched, which handles execute batched.

Line 150: `if len(flat_words) != expected_words:` This executes the following indented host-code branch only when ‘len(flat_words) != expected_words‘ is true. The important operands are len: the value named ’len’ in this block; flat_words: the value named ’flat words’ in this block; expected_words: the value named ’expected words’ in this block. Context: the function execute_batched, which handles execute batched.

Line 151: `raise RuntimeError(` This statement continues through source line 152; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘RuntimeError(f"descriptor program returned {len(flat_words)} beats; expected {expected_words}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 152: `f"descriptor program returned {len(flat_words)} beats; expected {expected_words}")` This is continuation line 2 of the 2-line logical statement that begins at source line 151. Its exact fragment is ‘f"descriptor program returned {len(flat_words)} beats; expected {expected_words}")‘. The important operands are f: the value named ’f’ in this block; descriptor: a DMA record header plus one command and its declared payload; program: the value named ’program’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"descriptor program returned {len(flat_words)} beats; expected {expected_words}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 153: `output = np.zeros((rows, columns), dtype=np.int64)` This assigns ‘np.zeros((rows, columns), dtype=np.int64)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are zeros: the value named ’zeros’ in this block; rows: one output-row position processed by the tensor tile; columns: one output-feature position processed in parallel. Context: the function execute_batched, which handles execute batched.

Line 154: `bindings = {binding.instruction_index: binding for binding in self.result_bindings}` This assigns ‘{binding.instruction_index: binding for binding in self.result_bindings}‘ to ‘bindings ‘ in host memory. Here bindings means the value named ’bindings’ in this block. The important operands are binding: the value named ’binding’ in this block; instruction_index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block. Context: the function execute_batched, which handles execute batched.

Line 155: `word_offset = 0` This assigns ‘0‘ to ‘word_offset ‘ in host memory. Here word_offset means the value named ’word offset’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 156: `for instruction_index, instruction in enumerate(self.instructions):` This iterates instruction_index, instruction over ‘enumerate(self.instructions)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_batched, which handles execute batched.

Line 157: `word_end = word_offset + instruction.output_beats` This assigns ‘word_offset + instruction.output_beats‘ to ‘word_end ‘ in host memory. Here word_end means the value named ’word end’ in this block. The important operands are word_offset: the value named ’word offset’ in this block; instruction: the value named ’instruction’ in this block; output_beats: the value named ’output beats’ in this block. Context: the function execute_batched, which handles execute batched.

Line 158: `words = flat_words[word_offset:word_end]` This assigns ‘flat_words[word_offset:word_end]‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are flat_words: the value named ’flat words’ in this block; word_offset: the value named ’word offset’ in this block; word_end: the value named ’word end’ in this block. Context: the function execute_batched, which handles execute batched.

Line 159: `binding = bindings.get(instruction_index)` This assigns ‘bindings.get(instruction_index)‘ to ‘binding ‘ in host memory. Here binding means the value named ’binding’ in this block. The important operands are bindings: the value named ’bindings’ in this block; get: the value named ’get’ in this block; instruction_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function execute_batched, which handles execute batched.

Line 160: `if binding is not None:` This executes the following indented host-code branch only when ‘binding is not None‘ is true. The important operands are binding: the value named ’binding’ in this block; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function execute_batched, which handles execute batched.

Line 161: `_accumulate_tensor_tile(output, binding, words)` This calls ‘_accumulate_tensor_tile‘ with ‘output, binding, words‘ inside the function execute_batched, which handles execute batched. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 162: `word_offset = word_end` This assigns ‘word_end‘ to ‘word_offset ‘ in host memory. Here word_offset means the value named ’word offset’ in this block. The important operands are word_end: the value named ’word end’ in this block. Context: the function execute_batched, which handles execute batched.

Line 163: `if self.bias is not None:` This executes the following indented host-code branch only when ‘self.bias is not None‘ is true. The important operands are bias: an additive learned offset applied after accumulation; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function execute_batched, which handles execute batched.

Line 164: `output += np.asarray(self.bias, dtype=np.int64).reshape(1, columns)` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are output: the value named 'output' in this block; asarray: the value named 'asarray' in this block; bias: an additive learned offset applied after accumulation.

Line 165: `return output` This returns ‘output‘ to the caller of the function execute_batched, which handles execute batched, ending this invocation. The important operands are output: the value named 'output' in this block.

Line 168: `def _accumulate_tensor_tile(` This starts function _accumulate_tensor_tile. Its indented body performs accumulate tensor tile as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 169: `output: np.ndarray,` Inside the function _accumulate_tensor_tile, which handles accumulate tensor tile, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are output: the value named 'output' in this block; ndarray: the value named 'ndarray' in this block.

Line 170: `binding: ResultBinding,` Inside the function _accumulate_tensor_tile, which handles accumulate tensor tile, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are binding: the value named 'binding' in this block; ResultBinding: the value named 'result binding' in this block.

Line 171: `words: Sequence[int],` Inside the function _accumulate_tensor_tile, which handles accumulate tensor tile, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are words: the value named 'words' in this block; Sequence: the value named 'sequence' in this block.

Line 172: `) -> None:` Inside the function _accumulate_tensor_tile, which handles accumulate tensor tile, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are None: the value named 'none' in this block.

Line 173: `seen: set[tuple[int, int]] = set()` This assigns ‘set()‘ to ‘seen‘ in host memory. Here seen means the value named 'seen' in this block. The important operands are set: the value named 'set' in this block. Context: the function _accumulate_tensor_tile, which handles accumulate tensor tile.

Line 174: `for beat_index, word in enumerate(words):` This iterates beat_index, word over ‘enumerate(words)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _accumulate_tensor_tile, which handles accumulate tensor tile.

Line 175: `value, local_row, local_column, last = unpack_tensor_result(word)` This assigns ‘unpack_tensor_result(wo to ‘value, local_row, local_column, last ‘ in host memory. Here value means the value named 'value' in this block. The important operands are unpack_tensor_result: the value named 'unpack tensor result' in this block; word: the value named 'word' in this block. Context: the function _accumulate_tensor_tile, which handles accumulate tensor tile.

Line 176: `if local_row >= binding.rows or local_column >= binding.columns:` This executes the following indented host-code branch only when ‘local_row >= binding.rows or local_column >= binding.columns‘ is true. The important operands are local_row: one output-row position processed by the tensor tile; binding: the value named 'binding' in this block; rows: one output-row position processed by the tensor tile. Context: the function _accumulate_tensor_tile, which handles accumulate tensor tile.

Line 177: `raise RuntimeError(` This statement continues through source line 179; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘RuntimeError(f"out-of-range tensor tag ({local_row},{local_column}) " f"for {binding.rows}x{binding.columns} result tile")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 178: `f"out-of-range tensor tag ({local_row},{local_column}) "` This is continuation line 2 of the 3-line logical statement that begins at source line 177. Its exact fragment is ‘f"out-of-range tensor tag ({local_row},{local_column}) "‘. The important operands are f: the value named ’f’ in this block; out: the value named ’out’ in this block; of: the value named ’of’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"out-of-range tensor tag ({local_row},{local_column}) " f"for {binding.rows}x{binding.columns} result tile")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 179: `f"for {binding.rows}x{binding.columns} result tile")` This is continuation line 3 of the 3-line logical statement that begins at source line 177. Its exact fragment is ‘f"for {binding.rows}x{binding.columns} result tile")‘. The important operands are f: the value named ’f’ in this block; for: the value named ’for’ in this block; binding: the value named ’binding’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"out-of-range tensor tag ({local_row},{local_column}) " f"for {binding.rows}x{binding.columns} result tile")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 180: `tag = (local_row, local_column)` This assigns ‘(local_row, local_column)‘ to ‘tag ‘ in host memory. Here tag means the value named ’tag’ in this block. The important operands are local_row: one output-row position processed by the tensor tile; local_column: one output-feature position processed in parallel. Context: the function _accumulate_tensor_tile, which handles accumulate tensor tile.

Line 181: `if tag in seen:` This executes the following indented host-code branch only when ‘tag in seen‘ is true. The important operands are tag: the value named ’tag’ in this block; in: the value named ’in’ in this block; seen: the value named ’seen’ in this block. Context: the function _accumulate_tensor_tile, which handles accumulate tensor tile.

Line 182: `raise RuntimeError(f"duplicate tensor result tag {tag}")` This rejects an invalid state by raising ‘RuntimeError(f"duplicate tensor result tag {tag}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 183: `if last != (beat_index == len(words) - 1):` This executes the following indented host-code branch only when ‘last != (beat_index == len(words) - 1)‘ is true. The important operands are last: the AXI-Stream marker for the final beat of a packet or command payload; beat_index: an integer selecting one tensor element, lane, bank, or state entry; len: the value named ’len’ in this block. Context: the function _accumulate_tensor_tile, which handles accumulate tensor tile.

Line 184: `raise RuntimeError("tensor TLAST metadata does not match the final result beat")` This rejects an invalid state by raising ‘RuntimeError("tensor TLAST metadata does not match the final result beat")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 185: `seen.add(tag)` This calls ‘seen.add‘ with ‘tag‘ inside the function _accumulate_tensor_tile, which handles accumulate tensor tile. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 186: `output[binding.row_start + local_row,` This statement continues through source line 187; the explanation uses the complete expression rather than only the first visual fragment. Inside the function _accumulate_tensor_tile, which handles accumulate tensor tile, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are output: the value named ’output’ in this block; binding: the value named ’binding’ in this block; row_start: one output-row position processed by the tensor tile.

Line 187: `binding.column_start + local_column] += value` This is continuation line 2 of the 2-line logical statement that begins at source line 186. Its exact fragment is ‘binding.column_start + local_column] += value‘. The important operands are binding: the value named ’binding’ in this block; column_start: one output-feature position processed in parallel; local_column: one output-feature position processed in parallel. Read together, the complete operation is: Inside the function _accumulate_tensor_tile, which handles accumulate tensor tile, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

The important operands are output: the value named 'output' in this block; binding: the value named 'binding' in this block; row_start: one output-row position processed by the tensor tile.

Line 188: `if len(seen) != binding.rows * binding.columns:` This executes the following indented host-code branch only when 'len(seen) != binding.rows * binding.columns' is true. The important operands are len: the value named 'len' in this block; seen: the value named 'seen' in this block; binding: the value named 'binding' in this block. Context: the function _accumulate_tensor_tile, which handles accumulate tensor tile.

Line 189: `raise RuntimeError("tensor result tile did not contain every row/column tag")` This rejects an invalid state by raising 'RuntimeError("tensor result tile did not contain every row/column tag")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 192: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it creates no FPGA circuit by itself.

Line 193: `class ConvGroupProgram:` This defines class ConvGroupProgram, which groups data and methods for conv group program. Instances exist in host memory and organize the flow that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 194: `output_channel_start: int` Inside the class ConvGroupProgram, which handles conv group program, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are output_channel_start: the value named 'output channel start' in this block.

Line 195: `output_channel_count: int` Inside the class ConvGroupProgram, which handles conv group program, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are output_channel_count: an integer count retained for control or measurable telemetry.

Line 196: `gemm: CompiledGemm` Inside the class ConvGroupProgram, which handles conv group program, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are gemm: the value named 'gemm' in this block; CompiledGemm: the value named 'compiled gemm' in this block.

Line 199: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it creates no FPGA circuit by itself.

Line 200: `class CompiledConv2d:` This defines class CompiledConv2d, which groups data and methods for compiled conv2d. Instances exist in host memory and organize the flow that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 201: `"""One or more GEMM programs plus the exact NCHW reconstruction map."""` This is executable documentation attached to the class CompiledConv2d, which handles compiled conv2d. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 203: `input_shape: tuple[int, int, int, int]` Inside the class CompiledConv2d, which handles compiled conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are input_shape: the value named 'input shape' in this block; tuple: the value named 'tuple' in this block.

Line 204: `output_shape: tuple[int, int, int, int]` Inside the class CompiledConv2d, which handles compiled conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are output_shape: the value named 'output shape' in this block; tuple: the value named 'tuple' in this block.

Line 205: `kernel: tuple[int, int]` Inside the class CompiledConv2d, which handles compiled conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are kernel: the value named 'kernel' in this block; tuple: the value named 'tuple' in this block.

Line 206: `stride: tuple[int, int]` Inside the class CompiledConv2d, which handles compiled conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are stride: the value named 'stride' in this block; tuple: the value named 'tuple' in this block.

Line 207: `padding: tuple[int, int]` Inside the class CompiledConv2d, which handles compiled conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are padding: the value named 'padding' in this block; tuple: the value named 'tuple' in this block.

Line 208: `dilation: tuple[int, int]` Inside the class CompiledConv2d, which handles compiled conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are dilation: the value named 'dilation' in this block; tuple: the value named 'tuple' in this block.

Line 209: `groups: int` Inside the class CompiledConv2d, which handles compiled conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are groups: the value named 'groups' in this block.

Line 210: `group_programs: tuple[ConvGroupProgram, ...]` Inside the class CompiledConv2d, which handles compiled conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are group_programs: the value named 'group programs' in this block; tuple: the value named 'tuple' in this block; ConvGroupProgram: the value named 'conv group program' in this block.

Line 212: `def execute(self, driver: CommandExecutor, *, timeout_seconds: float = 5.0) -> np.ndarray:` This starts function execute. Its indented body performs execute as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 213: `batch, output_channels, output_height, output_width = self.output_shape` This assigns 'self.output_shape' to 'batch, output_channels, output_height, output_width ' in host memory. Here batch means the value named 'batch' in this block. The important operands are output_shape: the value named 'output shape' in this block. Context: the function execute, which handles execute.

Line 214: `spatial_rows = batch * output_height * output_width` This assigns 'batch * output_height * output_width' to 'spatial_rows ' in host memory. Here spatial_rows means one output-row position processed by the tensor tile. The important operands are batch: the value named 'batch' in this block; output_height: the value named 'output height' in this block; output_width: the value named 'output width' in this block. Context: the function execute, which handles execute.

Line 215: `flat = np.zeros((spatial_rows, output_channels), dtype=np.int64)` This assigns 'np.zeros((spatial_rows, output_channels), dtype=np.int64)' to 'flat ' in host memory. Here flat means the value named 'flat' in this block. The important operands are zeros: the value named 'zeros' in this block; spatial_rows: one output-row position processed by the tensor tile; output_channels: the value named 'output channels' in this block. Context: the function execute, which handles execute.

Line 216: `for group in self.group_programs:` This iterates group over ‘self.group_programs‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 217: `values = group.gemm.execute(driver, timeout_seconds=timeout_seconds)` This assigns ‘group.gemm.execute(d timeout_seconds=timeout_seconds)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are group: the value named ’group’ in this block; gemm: the value named ’gemm’ in this block; execute: the value named ’execute’ in this block. Context: the function execute, which handles execute.

Line 218: `start = group.output_channel_start` This assigns ‘group.output_channel_start‘ to ‘start ‘ in host memory. Here start means the value named ’start’ in this block. The important operands are group: the value named ’group’ in this block; output_channel_start: the value named ’output channel start’ in this block. Context: the function execute, which handles execute.

Line 219: `flat[:, start:start + group.output_channel_count] = values` This assigns ‘values‘ to ‘flat[‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are values: the value named ’values’ in this block. Context: the function execute, which handles execute.

Line 220: `return flat.reshape(batch, output_height, output_width, output_channels).transpose(0, 3, 1, 2)` This returns ‘flat.reshape(batch, output_height, output_width, output_channels).transpose(0, 3, 1, 2)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are flat: the value named ’flat’ in this block; reshape: the value named ’reshape’ in this block; batch: the value named ’batch’ in this block.

Line 222: `def execute_batched(` This starts function execute_batched. Its indented body performs execute batched as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 223: `self,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 224: `driver: ProgramExecutor,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are driver: the value named ’driver’ in this block; ProgramExecutor: the value named ’program executor’ in this block.

Line 225: `*,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 226: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 227: `) -> np.ndarray:` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are ndarray: the value named ’ndarray’ in this block.

Line 228: `batch, output_channels, output_height, output_width = self.output_shape` This assigns ‘self.output_shape‘ to ‘batch, output_channels, output_height, output_width ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are output_shape: the value named ’output shape’ in this block. Context: the function execute_batched, which handles execute batched.

Line 229: `spatial_rows = batch * output_height * output_width` This assigns ‘batch * output_height * output_width‘ to ‘spatial_rows ‘ in host memory. Here spatial_rows means one output-row position processed by the tensor tile. The important operands are batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block; output_width: the value named ’output width’ in this block. Context: the function execute_batched, which handles execute batched.

Line 230: `flat = np.zeros((spatial_rows, output_channels), dtype=np.int64)` This assigns ‘np.zeros((spatial_rows, output_channels), dtype=np.int64)‘ to ‘flat ‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are zeros: the value named ’zeros’ in this block; spatial_rows: one output-row position processed by the tensor tile; output_channels: the value named ’output channels’ in this block. Context: the function execute_batched, which handles execute batched.

Line 231: `for group in self.group_programs:` This iterates group over ‘self.group_programs‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_batched, which handles execute batched.

Line 232: `values = group.gemm.execute_batched(` This statement continues through source line 233; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘group.gemm.execute_batched(driver, timeout_seconds=timeout_seconds)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are group: the value named ’group’ in this block; gemm: the value named ’gemm’ in this block; execute_batched: the value named ’execute batched’ in this block. Context: the function execute_batched, which handles execute batched.

Line 233: `driver, timeout_seconds=timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 232. Its exact fragment is ‘driver, timeout_seconds=timeout_seconds)‘. The important operands are driver: the value named ’driver’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘group.gemm.execute_batched(driver, timeout_seconds=timeout_seconds)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are group: the value named ’group’ in this block; gemm: the value named ’gemm’ in this block; execute_batched: the value named ’execute batched’ in this block. Context: the function execute_batched, which handles execute batched.

Line 234: `start = group.output_channel_start` This assigns ‘group.output_channel_start‘ to ‘start ‘ in host memory. Here start means the value named ’start’ in this block. The important operands are group: the value named ’group’ in this block; output_channel_start: the value named ’output channel start’ in this block. Context: the function execute_batched, which handles execute batched.

Line 235: `flat[:, start:start + group.output_channel_count] = values` This assigns ‘values‘ to ‘flat[‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are values: the value named ’values’ in this block. Context: the function execute_batched, which handles execute batched.

Line 236: `return flat.reshape(batch, output_height, output_width, output_channels).transpose(0, 3, 1, 2)` This returns ‘flat.reshape(batch, output_height, output_width, output_channels).transpose(0, 3, 1, 2)‘ to the caller of the function execute_batched, which handles execute batched, ending this invocation. The important operands are flat: the value named ’flat’ in this block; reshape: the value named ’reshape’ in this block; batch: the value named ’batch’ in this block.

Line 238: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class CompiledConv2d, which handles compiled conv2d; it creates no FPGA circuit by itself.

Line 239: `def mathematical_macs(self) -> int:` This starts function mathematical_macs. Its indented body performs mathematical macs as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 240: `return sum(group.gemm.mathematical_macs for group in self.group_programs)` This returns ‘sum(group.gemm for group in self.group_programs)‘ to the caller of the function mathematical_macs, which handles mathematical macs, ending this invocation. The important operands are sum: the value named ’sum’ in this block; group: the value named ’group’ in this block; gemm: the value named ’gemm’ in this block.

Line 242: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class CompiledConv2d, which handles compiled conv2d; it creates no FPGA circuit by itself.

Line 243: `def estimated_input_bytes(self) -> int:` This starts function estimated_input_bytes. Its indented body performs estimated input bytes as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 244: `return sum(group.gemm.estimated_input_bytes for group in self.group_programs)` This returns ‘sum(group.gemm.estimated_input_bytes for group in self.group_programs)‘ to the caller of the function estimated_input_bytes, which handles estimated input bytes, ending this invocation. The important operands are sum: the value named ’sum’ in this block; group: the value named ’group’ in this block; gemm: the value named ’gemm’ in this block.

Line 246: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class CompiledConv2d, which handles compiled conv2d; it creates no FPGA circuit by itself.

Line 247: `def estimated_spike_adds(self) -> int:` This starts function estimated_spike_adds. Its indented body performs estimated spike adds as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 248: `return sum(group.gemm.estimated_spike_adds for group in self.group_programs)` This returns ‘sum(group.gemm for group in self.group_programs)‘ to the caller of the function estimated_spike_adds, which handles estimated spike adds, ending this invocation. The important operands are sum: the value named ’sum’ in this block; group: the value named ’group’ in this block; gemm: the value named ’gemm’ in this block.

Line 251: `def _as_u8_matrix(name: str, value: np.ndarray | Sequence[Sequence[int]]) -> np.ndarray:` This starts function _as_u8_matrix. Its indented body performs as u8 matrix as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 252: `array = np.asarray(value)` This assigns ‘np.asarray(value)‘ to ‘array ‘ in host memory. Here array means the value named ’array’ in this block. The important operands are asarray: the value named ’asarray’ in this block; value: the value named ’value’ in this block. Context: the function _as_u8_matrix, which handles as u8 matrix.

Line 253: `if array.ndim != 2:` This executes the following indented host-code branch only when ‘array.ndim != 2‘ is true. The important operands are array: the value named ’array’ in this block; ndim: the value named ’ndim’ in this block. Context: the function _as_u8_matrix, which handles as u8 matrix.

Line 254: `raise ValueError(f"{name} must be a two-dimensional matrix")` This rejects an invalid state by raising ‘ValueError(f"{name} must be a two-dimensional matrix")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 255: `if array.size == 0:` This executes the following indented host-code branch only when ‘array.size == 0‘ is true. The important operands are array: the value named ’array’ in this block; size: the value named ’size’ in this block. Context: the function _as_u8_matrix, which handles as u8 matrix.

Line 256: `raise ValueError(f"{name} must not be empty")` This rejects an invalid state by raising ‘ValueError(f"{name} must not be empty")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 257: `if not np.issubdtype(array.dtype, np.integer):` This executes the following indented host-code branch only when ‘not np.issubdtype(array.dtype, np.integer)‘ is true. The important operands are not: the value named ’not’ in this block; issubdtype: the value named ’issubdtype’ in this block; array: the value named ’array’ in this block. Context: the function _as_u8_matrix, which handles as u8 matrix.

Line 258: `raise ValueError(f"{name} must contain integers")` This rejects an invalid state by raising ‘ValueError(f"{name} must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 259: `as_i64 = array.astype(np.int64)` This assigns ‘array.astype(np.int64)‘ to ‘as_i64 ‘ in host memory. Here as_i64 means the value named ’as i64’ in this block. The important operands are array: the value named ’array’ in this block; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function _as_u8_matrix, which handles as u8 matrix.

Line 260: `if np.any(as_i64 < 0) or np.any(as_i64 > 255):` This executes the following indented host-code branch only when ‘np.any(as_i64 $<$ 0) or np.any(as_i64 $>$ 255)‘ is true. The important operands are any: the value named ’any’ in this block; as_i64: the value named ’as i64’ in this block; or: the value named ’or’ in this block. Context: the function _as_u8_matrix, which handles as u8 matrix.

Line 261: `raise ValueError(f"{name} entries must be in [0,255]")` This rejects an invalid state by raising ‘ValueError(f"{name} entries must be in [0,255]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 262: `return as_i64.astype(np.uint8)` This returns ‘as_i64.astype(np.uint8)‘ to the caller of the function _as_u8_matrix, which handles as u8 matrix, ending this invocation. The important operands are as_i64: the value named ’as i64’ in this block; astype: the value named ’astype’ in this block; uint8: the value named ’uint8’ in this block.

Line 265: `def _u8(name: str, value: int) -> int:` This starts function _u8. Its indented body performs u8 as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 266: `integer = int(value)` This assigns ‘int(value)‘ to ‘integer ‘ in host memory. Here integer means the value named ’integer’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function _u8, which handles u8.

Line 267: `if integer < 0 or integer > 255:` This executes the following indented host-code branch only when ‘integer $<$ 0 or integer $>$ 255‘ is true. The important operands are integer: the value named ’integer’ in this block; or: the value named ’or’ in this block. Context: the function _u8, which handles u8.

Line 268: `raise ValueError(f"{name} must be in [0,255]")` This rejects an invalid state by raising ‘ValueError(f"{name} must be in [0,255]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 269: `return integer` This returns ‘integer‘ to the caller of the function _u8, which handles u8, ending this invocation. The important operands are integer: the value named ’integer’ in this block.

Line 272: `def _pair(name: str, value: int | Sequence[int], *, allow_zero: bool = False) -> tuple[int, int]:` This starts function _pair. Its indented body performs pair as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 273: `if isinstance(value, Sequence) and not isinstance(value, (str, bytes)):` This executes the following indented host-code branch only when ‘isinstance(value, Sequence) and not isinstance(value, (str, bytes))‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; value: the value named ’value’ in this block; Sequence: the value named ’sequence’ in this block. Context: the function _pair, which handles pair.

Line 274: `if len(value) != 2:` This executes the following indented host-code branch only when ‘len(value) != 2‘ is true. The important operands are len: the value named ’len’ in this block; value: the value named ’value’ in this block. Context: the function _pair, which handles pair.

Line 275: `raise ValueError(f"{name} must be an integer or a two-element sequence")` This rejects an invalid state by raising ‘ValueError(f"{name} must be an integer or a two-element sequence")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 276: `pair = (int(value[0]), int(value[1]))` This assigns ‘(int(value[0]), int(value[1]))‘ to ‘pair ‘ in host memory. Here pair means the value named ’pair’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function _pair, which handles pair.

Line 277: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function _pair, which handles pair.

Line 278: `pair = (int(value), int(value))` This assigns ‘(int(value), int(value))‘ to ‘pair ‘ in host memory. Here pair means the value named ’pair’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function _pair, which handles pair.

Line 279: `minimum = 0 if allow_zero else 1` This assigns ‘0 if allow_zero else 1‘ to ‘minimum ‘ in host memory. Here minimum means the value named ’minimum’ in this block. The important operands are allow_zero: the value named ’allow zero’ in this block. Context: the function _pair, which handles pair.

Line 280: `if pair[0] < minimum or pair[1] < minimum:` This executes the following indented host-code branch only when ‘pair[0] < minimum or pair[1] < minimum‘ is true. The important operands are pair: the value named ’pair’ in this block; minimum: the value named ’minimum’ in this block; or: the value named ’or’ in this block. Context: the function _pair, which handles pair.

Line 281: `raise ValueError(f"{name} values must be at least {minimum}")` This rejects an invalid state by raising ‘ValueError(f"{name} values must be at least {minimum}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 282: `return pair` This returns ‘pair‘ to the caller of the function _pair, which handles pair, ending this invocation. The important operands are pair: the value named ’pair’ in this block.

Line 285: `def _im2col_nchw(` This starts function _im2col_nchw. Its indented body performs im2col nchw as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 286: `activations: np.ndarray,` Inside the function _im2col_nchw, which handles im2col nchw, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named ’ndarray’ in this block.

Line 287: `kernel: tuple[int, int],` Inside the function _im2col_nchw, which handles im2col nchw, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are kernel: the value named ’kernel’ in this block; tuple: the value named ’tuple’ in this block.

Line 288: `stride: tuple[int, int],` Inside the function _im2col_nchw, which handles im2col nchw, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are stride: the value named ’stride’ in this block; tuple: the value named ’tuple’ in this block.

Line 289: `padding: tuple[int, int],` Inside the function _im2col_nchw, which handles im2col nchw, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are padding: the value named 'padding' in this block; tuple: the value named 'tuple' in this block.

Line 290: `dilation: tuple[int, int],` Inside the function _im2col_nchw, which handles im2col nchw, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are dilation: the value named 'dilation' in this block; tuple: the value named 'tuple' in this block.

Line 291: `pad_value: int,` Inside the function _im2col_nchw, which handles im2col nchw, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are pad_value: the value named 'pad value' in this block.

Line 292: `) -> tuple[np.ndarray, int, int]:` Inside the function _im2col_nchw, which handles im2col nchw, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 293: `batch, channels, height, width = activations.shape` This assigns 'activations.shape' to 'batch, channels, height, width ' in host memory. Here batch means the value named 'batch' in this block. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; shape: the value named 'shape' in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 294: `kernel_height, kernel_width = kernel` This assigns 'kernel' to 'kernel_height, kernel_width ' in host memory. Here kernel_height means the value named 'kernel height' in this block. The important operands are kernel: the value named 'kernel' in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 295: `effective_height = dilation[0] * (kernel_height - 1) + 1` This assigns 'dilation[0] * (kernel_height - 1) + 1' to 'effective_height ' in host memory. Here effective_height means the value named 'effective height' in this block. The important operands are dilation: the value named 'dilation' in this block; kernel_height: the value named 'kernel height' in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 296: `effective_width = dilation[1] * (kernel_width - 1) + 1` This assigns 'dilation[1] * (kernel_width - 1) + 1' to 'effective_width ' in host memory. Here effective_width means the value named 'effective width' in this block. The important operands are dilation: the value named 'dilation' in this block; kernel_width: the value named 'kernel width' in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 297: `numerator_height = height + 2 * padding[0] - effective_height` This assigns 'height + 2 * padding[0] - effective_height' to 'numerator_height ' in host memory. Here numerator_height means the value named 'numerator height' in this block. The important operands are height: the value named 'height' in this block; padding: the value named 'padding' in this block; effective_height: the value named 'effective height' in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 298: `numerator_width = width + 2 * padding[1] - effective_width` This assigns 'width + 2 * padding[1] - effective_width' to 'numerator_width ' in host memory. Here numerator_width means the value named 'numerator width' in this block. The important operands are width: the value named 'width' in this block; padding: the value named 'padding' in this block; effective_width: the value named 'effective width' in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 299: `if numerator_height < 0 or numerator_width < 0:` This executes the following indented host-code branch only when 'numerator_height $<$ 0 or numerator_width $<$ 0' is true. The important operands are numerator_height: the value named 'numerator height' in this block; or: the value named 'or' in this block; numerator_width: the value named 'numerator width' in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 300: `raise ValueError("effective convolution kernel is larger than the padded input")` This rejects an invalid state by raising ‘ValueError("effective convolution kernel is larger than the padded input")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 301: `output_height = numerator_height // stride[0] + 1` This assigns ‘numerator_height // stride[0] + 1‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are numerator_height: the value named ’numerator height’ in this block; stride: the value named ’stride’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 302: `output_width = numerator_width // stride[1] + 1` This assigns ‘numerator_width // stride[1] + 1‘ to ‘output_width ‘ in host memory. Here output_width means the value named ’output width’ in this block. The important operands are numerator_width: the value named ’numerator width’ in this block; stride: the value named ’stride’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 303: `padded = np.full(` This statement continues through source line 307; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.full((batch, channels, height + 2 * padding[0], width + 2 * padding[1]), pad_value, dtype=activations.dtype,)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are full: the value named ’full’ in this block; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 304: `(batch, channels, height + 2 * padding[0], width + 2 * padding[1]),` This is continuation line 2 of the 5-line logical statement that begins at source line 303. Its exact fragment is ‘(batch, channels, height + 2 * padding[0], width + 2 * padding[1]),‘. The important operands are batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block; height: the value named ’height’ in this block. Read together, the complete operation is: This assigns ‘np.full((batch, channels, height + 2 * padding[0], width + 2 * padding[1]), pad_value, dtype=activations.dtype,)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are full: the value named ’full’ in this block; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 305: `pad_value,` This is continuation line 3 of the 5-line logical statement that begins at source line 303. Its exact fragment is ‘pad_value,‘. The important operands are pad_value: the value named ’pad value’ in this block. Read together, the complete operation is: This assigns ‘np.full((batch, channels, height + 2 * padding[0], width + 2 * padding[1]), pad_value, dtype=activations.dtype,)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are full: the value named ’full’ in this block; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 306: `dtype=activations.dtype,` This is continuation line 4 of the 5-line logical statement that begins at source line 303. Its exact fragment is ‘dtype=activations.dtype,‘. The important operands are dtype: the value named ’dtype’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘np.full((batch, channels, height + 2 * padding[0], width + 2 * padding[1]), pad_value, dtype=activations.dtype,)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are full: the value named ’full’ in this block; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 307: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 303. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.full((batch, channels, height + 2 * padding[0], width + 2 * padding[1]), pad_value, dtype=activations.dtype,)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are full: the value named ’full’ in this block; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 308: `padded[:, :, padding[0]:padding[0] + height, padding[1]:padding[1] + width] = activations` This assigns ‘activations‘ to ‘padded[‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _im2col_nchw, which handles im2col nchw.

Line 309: `columns = np.empty(` This statement continues through source line 312; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.empty((batch * output_height * output_width, channels * kernel_height * kernel_width), dtype=activations.dtype,)‘ to ‘columns ‘ in host memory. Here columns means one output-feature position processed in parallel. The important operands are empty: the value named ’empty’ in this block; batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 310: `(batch * output_height * output_width, channels * kernel_height * kernel_width),` This is continuation line 2 of the 4-line logical statement that begins at source line 309. Its exact fragment is ‘(batch * output_height * output_width, channels * kernel_height * kernel_width),‘. The important operands are batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block; output_width: the value named ’output width’ in this block. Read together, the complete operation is: This assigns ‘np.empty((batch * output_height * output_width, channels * kernel_height * kernel_width), dtype=activations.dtype,)‘ to ‘columns ‘ in host memory. Here columns means one output-feature position processed in parallel. The important operands are empty: the value named ’empty’ in this block; batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 311: `dtype=activations.dtype,` This is continuation line 3 of the 4-line logical statement that begins at source line 309. Its exact fragment is ‘dtype=activations.dtype,‘. The important operands are dtype: the value named ’dtype’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘np.empty((batch * output_height * output_width, channels * kernel_height * kernel_width), dtype=activations.dtype,)‘ to ‘columns ‘ in host memory. Here columns means one output-feature position processed in parallel. The important operands are empty: the value named ’empty’ in this block; batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 312: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 309. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.empty((batch * output_height * output_width, channels * kernel_height * kernel_width), dtype=activations.dtype,)‘ to ‘columns ‘ in host memory. Here columns means one output-feature position processed in parallel. The important operands are empty: the value named ’empty’ in this block; batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 313: `row = 0` This assigns ‘0‘ to ‘row ‘ in host memory. Here row means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function _im2col_nchw, which handles im2col nchw.

Line 314: `for batch_index in range(batch):` This iterates batch_index over ‘range(batch)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _im2col_nchw, which handles im2col nchw.

Line 315: `for output_y in range(output_height):` This iterates output_y over ‘range(output_height)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _im2col_nchw, which handles im2col nchw.

Line 316: `input_y = output_y * stride[0]` This assigns ‘output_y * stride[0]‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride: the value named ’stride’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 317: `for output_x in range(output_width):` This iterates output_x over ‘range(output_width)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _im2col_nchw, which handles im2col nchw.

Line 318: `input_x = output_x * stride[1]` This assigns ‘output_x * stride[1]‘ to ‘input_x ‘ in host memory. Here input_x means the value named ’input x’ in this block. The important operands are output_x: the value named ’output x’ in this block; stride: the value named ’stride’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 319: `column = 0` This assigns ‘0‘ to ‘column ‘ in host memory. Here column means one output-feature position processed in parallel. The expression is a literal or structural operator and has no hidden software state. Context: the function _im2col_nchw, which handles im2col nchw.

Line 320: `for channel in range(channels):` This iterates channel over ‘range(channels)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _im2col_nchw, which handles im2col nchw.

Line 321: `for kernel_y in range(kernel_height):` This iterates kernel_y over ‘range(kernel_height)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _im2col_nchw, which handles im2col nchw.

Line 322: `y = input_y + kernel_y * dilation[0]` This assigns ‘input_y + kernel_y * dilation[0]‘ to ‘y ‘ in host memory. Here y means the value named ’y’ in this block. The important operands are input_y: the value named ’input y’ in this block; kernel_y: the value named ’kernel y’ in this block; dilation: the value named ’dilation’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 323: `for kernel_x in range(kernel_width):` This iterates kernel_x over ‘range(kernel_width)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _im2col_nchw, which handles im2col nchw.

Line 324: `x = input_x + kernel_x * dilation[1]` This assigns ‘input_x + kernel_x * dilation[1]‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are input_x: the value named ’input x’ in this block; kernel_x: the value named ’kernel x’ in this block; dilation: the value named ’dilation’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 325: `columns[row, column] = padded[batch_index, channel, y, x]` This assigns ‘padded[batch_index, channel, y, x]‘ to ‘columns[row, column] ‘ in host memory. Here columns means one output-feature position processed in parallel. The important operands are padded: the value named ’padded’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named ’channel’ in this block. Context: the function _im2col_nchw, which handles im2col nchw.

Line 326: `column += 1` Inside the function _im2col_nchw, which handles im2col nchw, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are column: one output-feature position processed in parallel.

Line 327: `row += 1` Inside the function _im2col_nchw, which handles im2col nchw, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are row: one output-row position processed by the tensor tile.

Line 328: `return columns, output_height, output_width` This returns ‘columns, output_height, output_width‘ to the caller of the function _im2col_nchw, which handles im2col nchw, ending this invocation. The important operands are columns: one output-feature position processed in parallel; output_height: the value named ’output height’ in this block; output_width: the value named ’output width’ in this block.

Line 331: `def compile_u8_gemm(` This starts function compile_u8_gemm. Its indented body performs compile u8 gemm as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 332: `activations: np.ndarray | Sequence[Sequence[int]],` Inside the function compile_u8_gemm, which handles compile u8 gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named 'ndarray' in this block; Sequence: the value named 'sequence' in this block.

Line 333: `weights: np.ndarray | Sequence[Sequence[int]],` Inside the function compile_u8_gemm, which handles compile u8 gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are weights: a learned coefficient multiplied by an activation; ndarray: the value named 'ndarray' in this block; Sequence: the value named 'sequence' in this block.

Line 334: `*,` Inside the function compile_u8_gemm, which handles compile u8 gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 335: `activation_zero_point: int = 0,` This assigns '0,' to 'activation_zero_point' in host memory. Here activation_zero_point means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 336: `weight_zero_point: int = 0,` This assigns '0,' to 'weight_zero_point' in host memory. Here weight_zero_point means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 337: `bias: Sequence[int] | np.ndarray | None = None,` This assigns 'None,' to 'bias' in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 338: `dataflow: Dataflow | None = None,` This assigns 'None,' to 'dataflow' in host memory. Here dataflow means the value named 'dataflow' in this block. The important operands are None: the value named 'none' in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 339: `) -> CompiledGemm:` Inside the function compile_u8_gemm, which handles compile u8 gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are CompiledGemm: the value named 'compiled gemm' in this block.

Line 340: `"""Compile an exact asymmetric-uint8 matrix multiplication.` This is executable documentation attached to the function compile_u8_gemm, which handles compile u8 gemm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 342: `Reduction dimensions above 2,048 are split into independently corrected` Inside the function compile_u8_gemm, which handles compile u8 gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are Reduction: the value named 'reduction' in this block; dimensions: the value named 'dimensions' in this block; above: the value named 'above' in this block.

Line 343: `chunks and accumulated by the runtime.  A single-row matrix uses Dot8 mode;` Inside the function compile_u8_gemm, which handles compile u8 gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are chunks: the value named 'chunks' in this block; and: the value named 'and' in this block; accumulated: the value named 'accumulated' in this block.

Line 344: `all other shapes use the 8x16 outer-product mode.` Inside the function compile_u8_gemm, which handles compile u8 gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are all: the value named 'all' in this block; other: the value named 'other' in this block; shapes: the value named 'shapes' in this block.

Line 345: `"""` This is executable documentation attached to the function compile_u8_gemm, which handles compile u8 gemm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 346: `a = _as_u8_matrix("activations", activations)` This assigns '_as_u8_matrix("activations", activations)' to 'a ' in host memory. Here a means the value named 'a' in this block. The important operands are _as_u8_matrix: the value named 'as u8 matrix' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 347: `b = _as_u8_matrix("weights", weights)` This assigns '_as_u8_matrix("weights", weights)' to 'b ' in host memory. Here b means the value named 'b' in this block. The important operands are _as_u8_matrix: the value named 'as u8 matrix' in this block; weights: a learned coefficient multiplied by an activation. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 348: `rows, reduction = a.shape` This assigns 'a.shape' to 'rows, reduction ' in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are a: the value named 'a' in this block; shape: the value named 'shape' in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 349: `weight_reduction, columns = b.shape` This assigns 'b.shape' to 'weight_reduction, columns ' in host memory. Here weight_reduction means a learned coefficient multiplied by an activation. The important operands are b: the value named 'b' in this block; shape: the value named 'shape' in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 350: `if reduction != weight_reduction:` This executes the following indented host-code branch only when 'reduction != weight_reduction' is true. The important operands are reduction: the value named 'reduction' in this block; weight_reduction: a learned coefficient multiplied by an activation. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 351: `raise ValueError("activation columns must equal weight rows")` This rejects an invalid state by raising 'ValueError("activation columns must equal weight rows")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 352: `za = _u8("activation_zero_point", activation_zero_point)` This assigns '_u8("activation_zero_point", activation_zero_point)' to 'za ' in host memory. Here za means the value named 'za' in this block. The important operands are _u8: the value named 'u8' in this block; activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 353: `zb = _u8("weight_zero_point", weight_zero_point)` This assigns '_u8("weight_zero_point", weight_zero_point to 'zb ' in host memory. Here zb means the value named 'zb' in this block. The important operands are _u8: the value named 'u8' in this block; weight_zero_point: a learned coefficient multiplied by an activation. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 355: `bias_tuple: tuple[int, ...] | None = None` This assigns ‘None‘ to ‘bias_tuple‘ in host memory. Here bias_tuple means an additive learned offset applied after accumulation. The important operands are None: the value named ’none’ in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 356: `if bias is not None:` This executes the following indented host-code branch only when ‘bias is not None‘ is true. The important operands are bias: an additive learned offset applied after accumulation; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 357: `bias_array = np.asarray(bias)` This assigns ‘np.asarray(bias)‘ to ‘bias_array ‘ in host memory. Here bias_array means an additive learned offset applied after accumulation. The important operands are asarray: the value named ’asarray’ in this block; bias: an additive learned offset applied after accumulation. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 358: `if bias_array.ndim != 1 or len(bias_array) != columns:` This executes the following indented host-code branch only when ‘bias_array.ndim != 1 or len(bias_array) != columns‘ is true. The important operands are bias_array: an additive learned offset applied after accumulation; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 359: `raise ValueError("bias must be a one-dimensional vector with one entry per output column")` This rejects an invalid state by raising ‘ValueError("bias must be a one-dimensional vector with one entry per output column")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 360: `if not np.issubdtype(bias_array.dtype, np.integer):` This executes the following indented host-code branch only when ‘not np.issubdtype(bias_array.dtype, np.integer)‘ is true. The important operands are not: the value named ’not’ in this block; issubdtype: the value named ’issubdtype’ in this block; bias_array: an additive learned offset applied after accumulation. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 361: `raise ValueError("bias must contain integers")` This rejects an invalid state by raising ‘ValueError("bias must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 362: `bias_tuple = tuple(int(value) for value in bias_array)` This assigns ‘tuple(int(value) for value in bias_array)‘ to ‘bias_tuple ‘ in host memory. Here bias_tuple means an additive learned offset applied after accumulation. The important operands are tuple: the value named ’tuple’ in this block; value: the value named ’value’ in this block; for: the value named ’for’ in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 364: `instructions: list[TensorInstruction] = []` This assigns ‘[]‘ to ‘instructions‘ in host memory. Here instructions means the value named ’instructions’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 365: `bindings: list[ResultBinding] = []` This assigns ‘[]‘ to ‘bindings‘ in host memory. Here bindings means the value named ’bindings’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 366: `estimated_bytes = 0` This assigns ‘0‘ to ‘estimated_bytes ‘ in host memory. Here estimated_bytes means the value named ’estimated bytes’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 368: `def append_load(command: Command128, beats: Sequence[StreamBeat], description: str) -> None:` This starts function append_load. Its indented body performs append load as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 369: `nonlocal estimated_bytes` Inside the function append_load, which handles append load, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are nonlocal: the value named 'nonlocal' in this block; estimated_bytes: the value named 'estimated bytes' in this block.

Line 370: `instructions.append(TensorInstruction(command, beats, 0, description))` This calls 'instructions.append' with 'TensorInstruction(command, beats, 0, description)' inside the function append_load, which handles append load. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 371: `estimated_bytes += 8 * len(beats)` Inside the function append_load, which handles append load, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are estimated_bytes: the value named 'estimated bytes' in this block; len: the value named 'len' in this block; beats: the value named 'beats' in this block.

Line 373: `def append_compute(` This starts function append_compute. Its indented body performs append compute as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 374: `command: Command128,` Inside the function append_compute, which handles append compute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 375: `row_start: int,` Inside the function append_compute, which handles append compute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are row_start: one output-row position processed by the tensor tile.

Line 376: `column_start: int,` Inside the function append_compute, which handles append compute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are column_start: one output-feature position processed in parallel.

Line 377: `tile_rows: int,` Inside the function append_compute, which handles append compute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are tile_rows: one output-row position processed by the tensor tile.

Line 378: `tile_columns: int,` Inside the function append_compute, which handles append compute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are tile_columns: one output-feature position processed in parallel.

Line 379: `reduction_start: int,` Inside the function append_compute, which handles append compute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are reduction_start: the value named 'reduction start' in this block.

Line 380: `reduction_count: int,` Inside the function append_compute, which handles append compute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are reduction_count: an integer count retained for control or measurable telemetry.

Line 381: `) -> None:` Inside the function append_compute, which handles append compute, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are None: the value named 'none' in this block.

Line 382: `instruction_index = len(instructions)` This assigns ‘len(instructions)‘ to ‘instruction_index ‘ in host memory. Here instruction_index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are len: the value named ’len’ in this block; instructions: the value named ’instructions’ in this block. Context: the function append_compute, which handles append compute.

Line 383: `instructions.append(TensorInstruction(` This statement continues through source line 388; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘instructions.append‘ with ‘TensorInstruction(command, (), tile_rows * tile_columns, f"compute C[{row_start}:{row_start + tile_rows}," f"{column_start}:{column_start + tile_columns}] " f"from K[{reduction_start}:{reduction_start + reduction_count}]",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 384: `command, (), tile_rows * tile_columns,` This is continuation line 2 of the 6-line logical statement that begins at source line 383. Its exact fragment is ‘command, (), tile_rows * tile_columns,‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; tile_rows: one output-row position processed by the tensor tile; tile_columns: one output-feature position processed in parallel. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(command, (), tile_rows * tile_columns, f"compute C[{row_start}:{row_start + tile_rows}," f"{column_start}:{column_start + tile_columns}] " f"from K[{reduction_start}:{reduction_start + reduction_count}]",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 385: `f"compute C[{row_start}:{row_start + tile_rows},"` This is continuation line 3 of the 6-line logical statement that begins at source line 383. Its exact fragment is ‘f"compute C[{row_start}:{row_start + tile_rows},"‘. The important operands are f: the value named ’f’ in this block; compute: the value named ’compute’ in this block; C: the value named ’c’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(command, (), tile_rows * tile_columns, f"compute C[{row_start}:{row_start + tile_rows}," f"{column_start}:{column_start + tile_columns}] " f"from K[{reduction_start}:{reduction_start + reduction_count}]",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 386: `f"{column_start}:{column_start + tile_columns}] "` This is continuation line 4 of the 6-line logical statement that begins at source line 383. Its exact fragment is ‘f"{column_start}:{column_start + tile_columns}] "‘. The important operands are f: the value named ’f’ in this block; column_start: one output-feature position processed in parallel; tile_columns: one output-feature position processed in parallel. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(command, (), tile_rows * tile_columns, f"compute C[{row_start}:{row_start + tile_rows}," f"{column_start}:{column_start + tile_columns}] " f"from K[{reduction_start}:{reducti + reduction_count}]",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 387: `f"from K[{reduction_start}:{reduction_start + reduction_count}]",` This is continuation line 5 of the 6-line logical statement that begins at source line 383. Its exact fragment is ‘f"from K[{reduction_start}:{reduction_start + reduction_count}]",‘. The important operands are f: the value named ’f’ in this block; from: the value named ’from’ in this block; K: the value named ’k’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(command, (), tile_rows * tile_columns, f"compute C[{row_start}:{row_start + tile_rows}," f"{column_start}:{column_start + tile_columns}] " f"from K[{reduction_start}:{reduction_start + reduction_count}]",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 388: `))` This is continuation line 6 of the 6-line logical statement that begins at source line 383. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(command, (), tile_rows * tile_columns, f"compute C[{row_start}:{row_start + tile_rows}," f"{column_start}:{column_start + tile_columns}] " f"from K[{reduction_start}:{reduction_start + reduction_count}]",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 389: `bindings.append(ResultBinding(` This statement continues through source line 392; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘bindings.append‘ with ‘ResultBinding(instruction_index, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count,)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 390: `instruction_index, row_start, column_start, tile_rows, tile_columns,` This is continuation line 2 of the 4-line logical statement that begins at source line 389. Its exact fragment is ‘instruction_index, row_start, column_start, tile_rows, tile_columns,‘. The important operands are instruction_index: an integer selecting one tensor element, lane, bank, or state entry; row_start: one output-row position processed by the tensor tile; column_start: one output-feature position processed in parallel. Read together, the complete operation is: This calls ‘bindings.append‘ with ‘ResultBinding(instruction_index, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count,)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 391: `reduction_start, reduction_count,` This is continuation line 3 of the 4-line logical statement that begins at source line 389. Its exact fragment is ‘reduction_start, reduction_count,‘. The important operands are reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This calls ‘bindings.append‘ with ‘ResultBinding(instruction_index, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count,)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 392: `))` This is continuation line 4 of the 4-line logical statement that begins at source line 389. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘bindings.append‘ with ‘ResultBinding(instruction_index, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count,)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 394: `for reduction_start in range(0, reduction, MAX_REDUCTION):` This iterates reduction_start over ‘range(0, reduction, MAX_REDUCTION)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_u8_gemm, which handles compile u8 gemm.

Line 395: `reduction_count = min(MAX_REDUCTION, reduction - reduction_start)` This assigns ‘min(MAX_REDUCTION reduction - reduction_start)‘ to ‘reduction_count ‘ in host memory. Here reduction_count means an integer count retained for control or measurable telemetry. The important operands are min: the value named ’min’ in this block; MAX_REDUCTION: the value named ’max reduction’ in this block; reduction: the value named ’reduction’ in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 396: `selected = dataflow or choose_dataflow(rows, columns, reduction_count).mode` This assigns ‘dataflow or choose_dataflow(rows, columns, reduction_count).mode‘ to ‘selected ‘ in host memory. Here selected means the value named ’selected’ in this block. The important operands are dataflow: the value named ’dataflow’ in this block; or: the value named ’or’ in this block; choose_dataflow: the value named ’choose dataflow’ in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 397: `if selected not in {Dataflow.INPUT_BROADCAST, Dataflow.WEIGHT_BROADCAST}:` This executes the following indented host-code branch only when ‘selected not in {Dataflow.INPUT_BROADCAST, Dataflow.WEIGHT_BROADCAS is true. The important operands are selected: the value named ’selected’ in this block; not: the value named ’not’ in this block; in: the value named ’in’ in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 398: `raise ValueError(f"unsupported dataflow {selected}")` This rejects an invalid state by raising ‘ValueError(f"unsupported dataflow {selected}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 399: `use_dot8 = rows == 1` Inside the function compile_u8_gemm, which handles compile u8 gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are use_dot8: the value named 'use dot8' in this block; rows: one output-row position processed by the tensor tile.

Line 400: `issue_count = (reduction_count + 7) // 8 if use_dot8 else reduction_count` This assigns '(reduction_count + 7) // 8 if use_dot8 else reduction_count' to 'issue_count ' in host memory. Here issue_count means an integer count retained for control or measurable telemetry. The important operands are reduction_count: an integer count retained for control or measurable telemetry; use_dot8: the value named 'use dot8' in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 401: `load_weight_opcode = Opcode.LOAD_WEIGHTS_DOT8 if use_dot8 else Opcode.LOAD_WEIGHTS_OUTER` This assigns 'Opcode.LOAD_WEIGHTS_DOT8 if use_dot8 else Opcode.LOAD_WEIGHTS_OUTER' to 'load_weight_opcode ' in host memory. Here load_weight_opcode means a learned coefficient multiplied by an activation. The important operands are Opcode: the value named 'opcode' in this block; LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation; use_dot8: the value named 'use dot8' in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 402: `compute_opcode = Opcode.COMPUTE_DENSE_DOT8 if use_dot8 else Opcode.COMPUTE_DENSE_OUTER` This assigns 'Opcode.COMPUTE_DENSE_DOT8 if use_dot8 else Opcode.COMPUTE_DENSE_OUTER' to 'compute_opcode ' in host memory. Here compute_opcode means the value named 'compute opcode' in this block. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_DOT8: the value named 'compute dense dot8' in this block; use_dot8: the value named 'use dot8' in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 404: `def activation_load(row_start: int) -> tuple[Command128, PackedStreamBeats]:` This starts function activation_load. Its indented body performs activation load as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 405: `tile_rows = min(ROW_TILE, rows - row_start)` This assigns 'min(ROW_TILE, rows - row_start)' to 'tile_rows ' in host memory. Here tile_rows means one output-row position processed by the tensor tile. The important operands are min: the value named 'min' in this block; ROW_TILE: one output-row position processed by the tensor tile; rows: one output-row position processed by the tensor tile. Context: the function activation_load, which handles activation load.

Line 406: `if use_dot8:` This executes the following indented host-code branch only when 'use_dot8' is true. The important operands are use_dot8: the value named 'use dot8' in this block. Context: the function activation_load, which handles activation load.

Line 407: `padded = np.full(issue_count * 8, za, dtype=np.uint8)` This assigns 'np.full(issue_count * 8, za, dtype=np.uint8)' to 'padded ' in host memory. Here padded means the value named 'padded' in this block. The important operands are full: the value named 'full' in this block; issue_count: an integer count retained for control or measurable telemetry; za: the value named 'za' in this block. Context: the function activation_load, which handles activation load.

Line 408: `padded[:reduction_count] = a[row_start, reduction_start:reduction_start + reduction_count]` This assigns 'a[row_start, reduction_start:reduction_start + reduction_count]' to 'padded[' in host memory. Here padded means the value named 'padded' in this block. The important operands are a: the value named 'a' in this block; row_start: one output-row position processed by the tensor tile; reduction_start: the value named 'reduction start' in this block. Context: the function activation_load, which handles activation load.

Line 409: `vectors = padded.reshape(issue_count, 8)` This assigns 'padded.reshape(issue_count, 8)' to 'vectors ' in host memory. Here vectors means the value named 'vectors' in this block. The important operands are padded: the value named 'padded' in this block; reshape: the value named 'reshape' in this block; issue_count: an integer count retained for control or measurable telemetry. Context: the function activation_load, which handles activation load.

Line 410: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function activation_load, which handles activation load.

Line 411: `vectors = np.full((reduction_count, ROW_TILE), za, dtype=np.uint8)` This assigns ‘np.full((reduction_count ROW_TILE), za, dtype=np.uint8)‘ to ‘vectors ‘ in host memory. Here vectors means the value named ’vectors’ in this block. The important operands are full: the value named ’full’ in this block; reduction_count: an integer count retained for control or measurable telemetry; ROW_TILE: one output-row position processed by the tensor tile. Context: the function activation_load, which handles activation load.

Line 412: `vectors[:, :tile_rows] = a[` This statement continues through source line 415; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘a[row_start:row_start + tile_rows, reduction_start:reduction_start + reduction_count,].T‘ to ‘vectors[‘ in host memory. Here vectors means the value named ’vectors’ in this block. The important operands are a: the value named ’a’ in this block; row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Context: the function activation_load, which handles activation load.

Line 413: `row_start:row_start + tile_rows,` This is continuation line 2 of the 4-line logical statement that begins at source line 412. Its exact fragment is ‘row_start:row_start + tile_rows,‘. The important operands are row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This assigns ‘a[row_start:row_start + tile_rows, reduction_start:reduction_start + reduction_count,].T‘ to ‘vectors[‘ in host memory. Here vectors means the value named ’vectors’ in this block. The important operands are a: the value named ’a’ in this block; row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Context: the function activation_load, which handles activation load.

Line 414: `reduction_start:reduction_start + reduction_count,` This is continuation line 3 of the 4-line logical statement that begins at source line 412. Its exact fragment is ‘reduction_start:reduction_start + reduction_count,‘. The important operands are reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This assigns ‘a[row_start:row_start + tile_rows, reduction_start:reduction_start + reduction_count,].T‘ to ‘vectors[‘ in host memory. Here vectors means the value named ’vectors’ in this block. The important operands are a: the value named ’a’ in this block; row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Context: the function activation_load, which handles activation load.

Line 415: `].T` This is continuation line 4 of the 4-line logical statement that begins at source line 412. Its exact fragment is ‘].T‘. The important operands are T: the value named ’t’ in this block. Read together, the complete operation is: This assigns ‘a[row_start:row_start + tile_rows, reduction_start:reduction_start + reduction_count,].T‘ to ‘vectors[‘ in host memory. Here vectors means the value named ’vectors’ in this block. The important operands are a: the value named ’a’ in this block; row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Context: the function activation_load, which handles activation load.

Line 416: `command = tensor_command(` This statement continues through source line 420; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=issue_count, activation_zero_point=za,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named ’opcode’ in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function activation_load, which handles activation load.

Line 417: `Opcode.LOAD_ACTIVATIONS,` This is continuation line 2 of the 5-line logical statement that begins at source line 416. Its exact fragment is ‘Opcode.LOAD_ACTIVATIONS,‘. The important operands are Opcode: the value named ’opcode’ in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=issue_count, activation_zero_point=za,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named ’opcode’

in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function activation_load, which handles activation load.

Line 418: `issue_count=issue_count,` This is continuation line 3 of the 5-line logical statement that begins at source line 416. Its exact fragment is 'issue_count=issue_count,'. The important operands are issue_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This assigns 'tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=issue_count, activation_zero_point=za,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function activation_load, which handles activation load.

Line 419: `activation_zero_point=za,` This is continuation line 4 of the 5-line logical statement that begins at source line 416. Its exact fragment is 'activation_zero_point=za,'. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; za: the value named 'za' in this block. Read together, the complete operation is: This assigns 'tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=issue_count, activation_zero_point=za,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function activation_load, which handles activation load.

Line 420: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 416. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=issue_count, activation_zero_point=za,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function activation_load, which handles activation load.

Line 421: `return command, _pack_compiler_u8_rows(vectors, 8)` This returns 'command, _pack_compiler_u8_rows(vecto 8)' to the caller of the function activation_load, which handles activation load, ending this invocation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _pack_compiler_u8_rows: one output-row position processed by the tensor tile; vectors: the value named 'vectors' in this block.

Line 423: `def weight_load(column_start: int) -> tuple[Command128, PackedStreamBeats]:` This starts function weight_load. Its indented body performs weight load as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 424: `tile_columns = min(COLUMN_TILE, columns - column_start)` This assigns 'min(COLUMN_TILE, columns - column_start)' to 'tile_columns ' in host memory. Here tile_columns means one output-feature position processed in parallel. The important operands are min: the value named 'min' in this block; COLUMN_TILE: one output-feature position processed in parallel; columns: one output-feature position processed in parallel. Context: the function weight_load, which handles weight load.

Line 425: `if use_dot8:` This executes the following indented host-code branch only when 'use_dot8' is true. The important operands are use_dot8: the value named 'use dot8' in this block. Context: the function weight_load, which handles weight load.

Line 426: `tensor = np.full((issue_count, 8, COLUMN_TILE), zb, dtype=np.uint8)` This assigns 'np.full((issue_count, 8, COLUMN_TILE), zb, dtype=np.uint8)' to 'tensor ' in host memory. Here tensor means the value named 'tensor' in this block. The important operands are full: the value named 'full' in this block; issue_count: an integer count retained for control or measurable telemetry; COLUMN_TILE: one output-feature position processed in parallel. Context: the function weight_load, which handles weight load.

Line 427: `flat = tensor.reshape(issue_count * 8, COLUMN_TILE)` This assigns ‘tensor.reshape(issue_count * 8, COLUMN_TILE)‘ to ‘flat ‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are tensor: the value named ’tensor’ in this block; reshape: the value named ’reshape’ in this block; issue_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 428: `flat[:reduction_count, :tile_columns] = b[` This statement continues through source line 431; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘b[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘flat[‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are b: the value named ’b’ in this block; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 429: `reduction_start:reduction_start + reduction_count,` This is continuation line 2 of the 4-line logical statement that begins at source line 428. Its exact fragment is ‘reduction_start:reduction_start + reduction_count,‘. The important operands are reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This assigns ‘b[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘flat[‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are b: the value named ’b’ in this block; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 430: `column_start:column_start + tile_columns,` This is continuation line 3 of the 4-line logical statement that begins at source line 428. Its exact fragment is ‘column_start:column_start + tile_columns,‘. The important operands are column_start: one output-feature position processed in parallel; tile_columns: one output-feature position processed in parallel. Read together, the complete operation is: This assigns ‘b[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘flat[‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are b: the value named ’b’ in this block; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 431: `]` This is continuation line 4 of the 4-line logical statement that begins at source line 428. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘b[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘flat[‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are b: the value named ’b’ in this block; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 432: `beats = _pack_compiler_u8_rows(tensor, 16)` This assigns ‘_pack_compiler_u8_rows(tensor, 16)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are _pack_compiler_u8_rows: one output-row position processed by the tensor tile; tensor: the value named ’tensor’ in this block. Context: the function weight_load, which handles weight load.

Line 433: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function weight_load, which handles weight load.

Line 434: `matrix = np.full((reduction_count, COLUMN_TILE), zb, dtype=np.uint8)` This assigns ‘np.full((reduction_cou COLUMN_TILE), zb, dtype=np.uint8)‘ to ‘matrix ‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are full: the value named ’full’ in this block; reduction_count: an integer count retained for control or measurable telemetry; COLUMN_TILE: one output-feature position processed in parallel. Context: the function weight_load, which handles weight load.

Line 435: `matrix[:, :tile_columns] = b[` This statement continues through source line 438; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘b[reduction_start:reduction_start +

reduction_count, column_start:column_start + tile_columns,]‘ to ‘matrix[‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are b: the value named ’b’ in this block; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 436: `reduction_start:reduction_start + reduction_count,` This is continuation line 2 of the 4-line logical statement that begins at source line 435. Its exact fragment is ‘reduction_start:reduction_start + reduction_count,‘. The important operands are reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This assigns ‘b[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘matrix[‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are b: the value named ’b’ in this block; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 437: `column_start:column_start + tile_columns,` This is continuation line 3 of the 4-line logical statement that begins at source line 435. Its exact fragment is ‘column_start:column_start + tile_columns,‘. The important operands are column_start: one output-feature position processed in parallel; tile_columns: one output-feature position processed in parallel. Read together, the complete operation is: This assigns ‘b[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘matrix[‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are b: the value named ’b’ in this block; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 438: `]` This is continuation line 4 of the 4-line logical statement that begins at source line 435. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘b[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘matrix[‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are b: the value named ’b’ in this block; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 439: `beats = _pack_compiler_u8_rows(matrix, 16)` This assigns ‘_pack_compiler_u8_rows(matrix, 16)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are _pack_compiler_u8_rows: one output-row position processed by the tensor tile; matrix: the value named ’matrix’ in this block. Context: the function weight_load, which handles weight load.

Line 440: `command = tensor_command(load_weight_opcode, issue_count=issue_count)` This assigns ‘tensor_command(loa issue_count=issue_count)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; load_weight_opcode: a learned coefficient multiplied by an activation; issue_count: an integer count retained for control or measurable telemetry. Context: the function weight_load, which handles weight load.

Line 441: `return command, beats` This returns ‘command, beats‘ to the caller of the function weight_load, which handles weight load, ending this invocation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; beats: the value named ’beats’ in this block.

Line 443: `def compute(row_start: int, column_start: int) -> None:` This starts function compute. Its indented body performs compute as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 444: `tile_rows = min(ROW_TILE, rows - row_start)` This assigns ‘min(ROW_TILE, rows - row_start)‘ to ‘tile_rows ‘ in host memory. Here tile_rows means one output-row position processed by the tensor tile. The important operands are min: the value named ’min’ in this block; ROW_TILE: one output-row position processed by the tensor tile; rows: one output-row position processed by the tensor tile. Context: the function compute, which handles compute.

Line 445: `tile_columns = min(COLUMN_TILE, columns - column_start)` This assigns ‘min(COLUMN_TILE, columns - column_start)‘ to ‘tile_columns ‘ in host memory. Here tile_columns means one output-feature position processed in parallel. The important operands are min: the value named ’min’ in this block; COLUMN_TILE: one output-feature position processed in parallel; columns: one output-feature position processed in parallel. Context: the function compute, which handles compute.

Line 446: `command = tensor_command(` This statement continues through source line 453; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=issue_count, activation_zero_point=za, weight_zero_point=zb,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_opcode: the value named ’compute opcode’ in this block; rows: one output-row position processed by the tensor tile. Context: the function compute, which handles compute.

Line 447: `compute_opcode,` This is continuation line 2 of the 8-line logical statement that begins at source line 446. Its exact fragment is ‘compute_opcode,‘. The important operands are compute_opcode: the value named ’compute opcode’ in this block. Read together, the complete operation is: This assigns ‘tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=issue_count, activation_zero_point=za, weight_zero_point=zb,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_opcode: the value named ’compute opcode’ in this block; rows: one output-row position processed by the tensor tile. Context: the function compute, which handles compute.

Line 448: `rows=tile_rows,` This is continuation line 3 of the 8-line logical statement that begins at source line 446. Its exact fragment is ‘rows=tile_rows,‘. The important operands are rows: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This assigns ‘tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=issue_count, activation_zero_point=za, weight_zero_point=zb,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_opcode: the value named ’compute opcode’ in this block; rows: one output-row position processed by the tensor tile. Context: the function compute, which handles compute.

Line 449: `columns=tile_columns,` This is continuation line 4 of the 8-line logical statement that begins at source line 446. Its exact fragment is ‘columns=tile_columns,‘. The important operands are columns: one output-feature position processed in parallel; tile_columns: one output-feature position processed in parallel. Read together, the complete operation is: This assigns ‘tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=issue_count, activation_zero_point=za, weight_zero_point=zb,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_opcode: the value named ’compute opcode’ in this block; rows: one output-row position processed by the tensor tile. Context: the function compute, which handles compute.

Line 450: `issue_count=issue_count,` This is continuation line 5 of the 8-line logical statement that begins at source line 446. Its exact fragment is ‘issue_count=issue_count,‘. The important operands are issue_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This assigns ‘tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=issue_count, activation_zero_point=za, weight_zero_po)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_opcode: the value named ’compute opcode’ in this block; rows: one output-row position processed by the tensor tile. Context: the function compute, which handles compute.

Line 451: `activation_zero_point=za,` This is continuation line 6 of the 8-line logical statement that begins at source line 446. Its exact fragment is ‘activation_zero_point=za,‘. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; za: the value named ’za’ in this block. Read together, the complete operation is: This assigns ‘tensor_command(compute_opcode, rows=tile_rows,

columns=tile_columns, issue_count=issue_count, activation_zero_point=za, weight_zero_point=zb,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_opcode: the value named 'compute opcode' in this block; rows: one output-row position processed by the tensor tile. Context: the function compute, which handles compute.

Line 452: `weight_zero_point=zb,` This is continuation line 7 of the 8-line logical statement that begins at source line 446. Its exact fragment is ‘weight_zero_point=zb,‘. The important operands are weight_zero_point: a learned coefficient multiplied by an activation; zb: the value named 'zb' in this block. Read together, the complete operation is: This assigns ‘tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=issue_count, activation_zero_point=za, weight_zero_point=zb,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_opcode: the value named 'compute opcode' in this block; rows: one output-row position processed by the tensor tile. Context: the function compute, which handles compute.

Line 453: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 446. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=issue_count, activation_zero_point=za, weight_zero_point=zb,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_opcode: the value named 'compute opcode' in this block; rows: one output-row position processed by the tensor tile. Context: the function compute, which handles compute.

Line 454: `append_compute(` This statement continues through source line 457; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘append_compute‘ with ‘ command, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count, ‘ inside the function compute, which handles compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 455: `command, row_start, column_start, tile_rows, tile_columns,` This is continuation line 2 of the 4-line logical statement that begins at source line 454. Its exact fragment is ‘command, row_start, column_start, tile_rows, tile_columns,‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; row_start: one output-row position processed by the tensor tile; column_start: one output-feature position processed in parallel. Read together, the complete operation is: This calls ‘append_compute‘ with ‘ command, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count, ‘ inside the function compute, which handles compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 456: `reduction_start, reduction_count,` This is continuation line 3 of the 4-line logical statement that begins at source line 454. Its exact fragment is ‘reduction_start, reduction_count,‘. The important operands are reduction_start: the value named 'reduction start' in this block; reduction_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This calls ‘append_compute‘ with ‘ command, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count, ‘ inside the function compute, which handles compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 457: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 454. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘append_compute‘ with ‘ command, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count, ‘ inside the function compute, which handles compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 459: `if selected == Dataflow.INPUT_BROADCAST:` This executes the following indented host-code branch only when ‘selected == Dataflow.INPUT_BROADCAST‘ is true. The important operands are selected: the value named

'selected' in this block; Dataflow: the value named 'dataflow' in this block; INPUT_BROADCAST: the value named 'input broadcast' in this block. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 460: `for row_start in range(0, rows, ROW_TILE):` This iterates row_start over ‘range(0, rows, ROW_TILE)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_u8_gemm, which handles compile u8 gemm.

Line 461: `command, beats = activation_load(row_start)` This assigns ‘activation_load(row_start)‘ to ‘command, beats ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are activation_load: an input feature value consumed by a neuron or multiply-accumulate operation; row_start: one output-row position processed by the tensor tile. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 462: `append_load(command, beats, f"load activation tile at row {row_start}")` This calls ‘append_load‘ with ‘command, beats, f"load activation tile at row {row_start}"‘ inside the function compile_u8_gemm, which handles compile u8 gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 463: `for column_start in range(0, columns, COLUMN_TILE):` This iterates column_start over ‘range(0, columns, COLUMN_TILE)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_u8_gemm, which handles compile u8 gemm.

Line 464: `command, beats = weight_load(column_start)` This assigns ‘weight_load(column_start)‘ to ‘command, beats ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are weight_load: a learned coefficient multiplied by an activation; column_start: one output-feature position processed in parallel. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 465: `append_load(command, beats, f"load weight tile at column {column_start}")` This calls ‘append_load‘ with ‘command, beats, f"load weight tile at column {column_start}"‘ inside the function compile_u8_gemm, which handles compile u8 gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 466: `compute(row_start, column_start)` This calls ‘compute‘ with ‘row_start, column_start‘ inside the function compile_u8_gemm, which handles compile u8 gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 467: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function compile_u8_gemm, which handles compile u8 gemm.

Line 468: `for column_start in range(0, columns, COLUMN_TILE):` This iterates column_start over ‘range(0, columns, COLUMN_TILE)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_u8_gemm, which handles compile u8 gemm.

Line 469: `command, beats = weight_load(column_start)` This assigns ‘weight_load(column_start)‘ to ‘command, beats ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are weight_load: a learned coefficient multiplied by an activation; column_start: one output-feature position processed in parallel. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 470: `append_load(command, beats, f"load weight tile at column {column_start}")` This calls ‘append_load‘ with ‘command, beats, f"load weight tile at column {column_start}"‘ inside the function compile_u8_gemm, which handles compile u8 gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 471: `for row_start in range(0, rows, ROW_TILE):` This iterates row_start over ‘range(0, rows, ROW_TILE)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_u8_gemm, which handles compile u8 gemm.

Line 472: `command, beats = activation_load(row_start)` This assigns ‘activation_load(row_start)‘ to ‘command, beats ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are activation_load: an input feature value consumed by a neuron or multiply-accumulate operation; row_start: one output-row position processed by the tensor tile. Context: the function compile_u8_gemm, which handles compile u8 gemm.

Line 473: `append_load(command, beats, f"load activation tile at row {row_start}")` This calls ‘append_load‘ with ‘command, beats, f"load activation tile at row {row_start}"‘ inside the function compile_u8_gemm, which handles compile u8 gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 474: `compute(row_start, column_start)` This calls ‘compute‘ with ‘row_start, column_start‘ inside the function compile_u8_gemm, which handles compile u8 gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 476: `return CompiledGemm(` This statement continues through source line 479; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘CompiledGemm((rows, reduction, columns), za, zb, tuple(instructions), tuple(bindings), bias_tuple, estimated_bytes, rows * reduction * columns,)‘ to the caller of the function compile_u8_gemm, which handles compile u8 gemm, ending this invocation. The important operands are CompiledGemm: the value named ’compiled gemm’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 477: `(rows, reduction, columns), za, zb, tuple(instructions), tuple(bindings),` This is continuation line 2 of the 4-line logical statement that begins at source line 476. Its exact fragment is ‘(rows, reduction, columns), za, zb, tuple(instructions), tuple(bindings),‘. The important operands are rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block; columns: one output-feature position processed in parallel. Read together, the complete operation is: This returns ‘CompiledGemm((rows, reduction, columns), za, zb, tuple(instructions), tuple(bindings), bias_tuple, estimated_bytes, rows * reduction * columns,)‘ to the caller of the function compile_u8_gemm, which handles compile u8 gemm, ending this invocation. The important operands are CompiledGemm: the value named ’compiled gemm’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 478: `bias_tuple, estimated_bytes, rows * reduction * columns,` This is continuation line 3 of the 4-line logical statement that begins at source line 476. Its exact fragment is ‘bias_tuple, estimated_bytes, rows * reduction * columns,‘. The important operands are bias_tuple: an additive learned offset applied after accumulation; estimated_bytes: the value named ’estimated bytes’ in this block; rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This returns ‘CompiledGemm((rows, reduction, columns), za, zb, tuple(instructions), tuple(bindings), bias_tuple, estimated_bytes, rows * reduction * columns,)‘ to the caller of the function compile_u8_gemm, which handles compile u8 gemm, ending this invocation. The important operands are CompiledGemm: the value named ’compiled gemm’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 479: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 476. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘CompiledGemm((rows, reduction, columns), za, zb, tuple(instructions), tuple(bindings), bias_tuple, estimated_bytes, rows * reduction * columns,)‘ to the caller of the function compile_u8_gemm, which handles compile u8 gemm, ending this invocation. The important operands are CompiledGemm: the value named ’compiled gemm’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 482: `def compile_spike_gemm(` This starts function compile_spike_gemm. Its indented body performs compile spike gemm as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 483: `spikes: np.ndarray | Sequence[Sequence[int]],` Inside the function compile_spike_gemm, which handles compile spike gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are spikes: a binary event produced when a neuron crosses its threshold; ndarray: the value named 'ndarray' in this block; Sequence: the value named 'sequence' in this block.

Line 484: `weights: np.ndarray | Sequence[Sequence[int]],` Inside the function compile_spike_gemm, which handles compile spike gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are weights: a learned coefficient multiplied by an activation; ndarray: the value named 'ndarray' in this block; Sequence: the value named 'sequence' in this block.

Line 485: `*,` Inside the function compile_spike_gemm, which handles compile spike gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 486: `ternary: bool = False,` This assigns 'False,' to 'ternary' in host memory. Here ternary means the value named 'ternary' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 487: `bias: Sequence[int] | np.ndarray | None = None,` This assigns 'None,' to 'bias' in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 488: `dataflow: Dataflow | None = None,` This assigns 'None,' to 'dataflow' in host memory. Here dataflow means the value named 'dataflow' in this block. The important operands are None: the value named 'none' in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 489: `) -> CompiledGemm:` Inside the function compile_spike_gemm, which handles compile spike gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are CompiledGemm: the value named 'compiled gemm' in this block.

Line 490: `"""Compile binary or ternary spikes times signed INT8 weights.` This is executable documentation attached to the function compile_spike_gemm, which handles compile spike gemm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 492: `The RTL performs one signed addition for each non-zero spike/output-column` Inside the function compile_spike_gemm, which handles compile spike gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are The: the value named 'the' in this block; RTL: the value named 'rtl' in this block; performs: the value named 'performs' in this block.

Line 493: `pair and performs no multiplication. Ternary "-1" spikes select` Inside the function compile_spike_gemm which handles compile spike gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are pair: the value named 'pair' in this block; and: the value named 'and' in this block; performs: the value named 'performs' in this block.

Line 494: `subtraction.  Zero-spike K positions are compacted by the activation-load` Inside the function compile_spike_gemm, which handles compile spike gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are subtraction: the value named 'subtraction' in this block; Zero: the value named 'zero' in this block; spike: a binary event produced when a neuron crosses its threshold.

Line 495: `pipeline and do not reach the tensor tile.` Inside the function compile_spike_gemm, which handles compile spike gemm, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are pipeline: the value named 'pipeline' in this block; and: the value named 'and' in this block; do: the value named 'do' in this block.

Line 496: `"""` This is executable documentation attached to the function compile_spike_gemm, which handles compile spike gemm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 497: `spike_array = np.asarray(spikes)` This assigns ‘np.asarray(spikes)‘ to ‘spike_array ‘ in host memory. Here spike_array means a binary event produced when a neuron crosses its threshold. The important operands are asarray: the value named 'asarray' in this block; spikes: a binary event produced when a neuron crosses its threshold. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 498: `weight_array = np.asarray(weights)` This assigns ‘np.asarray(weights)‘ to ‘weight_array ‘ in host memory. Here weight_array means a learned coefficient multiplied by an activation. The important operands are asarray: the value named 'asarray' in this block; weights: a learned coefficient multiplied by an activation. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 499: `if spike_array.ndim != 2 or weight_array.ndim != 2:` This executes the following indented host-code branch only when ‘spike_array.ndim != 2 or weight_array.ndim != 2‘ is true. The important operands are spike_array: a binary event produced when a neuron crosses its threshold; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 500: `raise ValueError("spikes and weights must be two-dimensional matrices")` This rejects an invalid state by raising ‘ValueError("spikes and weights must be two-dimensional matrices")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 501: `if spike_array.size == 0 or weight_array.size == 0:` This executes the following indented host-code branch only when ‘spike_array.size == 0 or weight_array.size == 0‘ is true. The important operands are spike_array: a binary event produced when a neuron crosses its threshold; size: the value named 'size' in this block; or: the value named 'or' in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 502: `raise ValueError("spikes and weights must not be empty")` This rejects an invalid state by raising ‘ValueError("spikes and weights must not be empty")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 503: `if not np.issubdtype(spike_array.dtype, np.integer) or not np.issubdtype(weight_array.dtype, np.integer):` This executes the following indented host-code branch only when ‘not np.issubdtype(spike_array.dtype, np.integer) or not np.issubdtype(weight_array.dtype, np.integer)‘ is true. The important operands are not: the value named 'not' in this block; issubdtype: the value named 'issubdtype' in this block; spike_array: a binary event produced when a neuron crosses its threshold. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 504: `raise ValueError("spikes and weights must contain integers")` This rejects an invalid state by raising ‘ValueError("spikes and weights must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 505: `spike_i64 = spike_array.astype(np.int64)` This assigns ‘spike_array.astype(np.int64)‘ to ‘spike_i64 ‘ in host memory. Here spike_i64 means a binary event produced when a neuron crosses its threshold. The important operands are spike_array: a binary event produced when a neuron crosses its threshold; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 506: `weight_i64 = weight_array.astype(np.int64)` This assigns ‘weight_array.astype(np.int64)‘ to ‘weight_i64 ‘ in host memory. Here weight_i64 means a learned coefficient multiplied by an activation. The important operands are weight_array: a learned coefficient multiplied by an activation; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 507: `allowed = np.isin(spike_i64, (-1, 0, 1) if ternary else (0, 1))` This assigns ‘np.isin(spike_i64, (-1, 0, 1) if ternary else (0, 1))‘ to ‘allowed ‘ in host memory. Here allowed means the value named ’allowed’ in this block. The important operands are isin: the value named ’isin’ in this block; spike_i64: a binary event produced when a neuron crosses its threshold; ternary: the value named ’ternary’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 508: `if not np.all(allowed):` This executes the following indented host-code branch only when ‘not np.all(allowed)‘ is true. The important operands are not: the value named ’not’ in this block; all: the value named ’all’ in this block; allowed: the value named ’allowed’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 509: `expected = "{-1,0,1}" if ternary else "{0,1}"` This assigns ‘"{-1,0,1}" if ternary else "{0,1}"‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are ternary: the value named ’ternary’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 510: `raise ValueError(f"spikes must be in {expected}")` This rejects an invalid state by raising ‘ValueError(f"spikes must be in {expected}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 511: `if np.any(weight_i64 < -128) or np.any(weight_i64 > 127):` This executes the following indented host-code branch only when ‘np.any(weight_i64 < -128) or np.any(weight_i64 > 127)‘ is true. The important operands are any: the value named ’any’ in this block; weight_i64: a learned coefficient multiplied by an activation; or: the value named ’or’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 512: `raise ValueError("sparse-mode weights must be signed INT8 values")` This rejects an invalid state by raising ‘ValueError("sparse-mode weights must be signed INT8 values")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 514: `rows, reduction = spike_i64.shape` This assigns ‘spike_i64.shape‘ to ‘rows, reduction ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are spike_i64: a binary event produced when a neuron crosses its threshold; shape: the value named ’shape’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 515: `weight_reduction, columns = weight_i64.shape` This assigns ‘weight_i64.shape‘ to ‘weight_reduction, columns ‘ in host memory. Here weight_reduction means a learned coefficient multiplied by an activation. The important operands are weight_i64: a learned coefficient multiplied by an activation; shape: the value named ’shape’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 516: `if reduction != weight_reduction:` This executes the following indented host-code branch only when ‘reduction != weight_reduction‘ is true. The important operands are reduction: the value named ’reduction’ in this block; weight_reduction: a learned coefficient multiplied by an activation. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 517: `raise ValueError("spike columns must equal weight rows")` This rejects an invalid state by raising ‘ValueError("spike columns must equal weight rows")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 518: `bias_tuple: tuple[int, ...] | None = None` This assigns ‘None‘ to ‘bias_tuple‘ in host memory. Here bias_tuple means an additive learned offset applied after accumulation. The important operands are None: the value named ’none’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 519: `if bias is not None:` This executes the following indented host-code branch only when ‘bias is not None‘ is true. The important operands are bias: an additive learned offset applied after accumulation; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 520: `bias_array = np.asarray(bias)` This assigns ‘np.asarray(bias)‘ to ‘bias_array ‘ in host memory. Here bias_array means an additive learned offset applied after accumulation. The important operands are asarray: the value named ’asarray’ in this block; bias: an additive learned offset applied after accumulation. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 521: `if bias_array.ndim != 1 or len(bias_array) != columns:` This executes the following indented host-code branch only when ‘bias_array.ndim != 1 or len(bias_array) != columns‘ is true. The important operands are bias_array: an additive learned offset applied after accumulation; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 522: `raise ValueError("bias must have one entry per output column")` This rejects an invalid state by raising ‘ValueError("bias must have one entry per output column")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 523: `if not np.issubdtype(bias_array.dtype, np.integer):` This executes the following indented host-code branch only when ‘not np.issubdtype(bias_array.dtype, np.integer)‘ is true. The important operands are not: the value named ’not’ in this block; issubdtype: the value named ’issubdtype’ in this block; bias_array: an additive learned offset applied after accumulation. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 524: `raise ValueError("bias must contain integers")` This rejects an invalid state by raising ‘ValueError("bias must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 525: `bias_tuple = tuple(int(value) for value in bias_array)` This assigns ‘tuple(int(value) for value in bias_array)‘ to ‘bias_tuple ‘ in host memory. Here bias_tuple means an additive learned offset applied after accumulation. The important operands are tuple: the value named ’tuple’ in this block; value: the value named ’value’ in this block; for: the value named ’for’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 527: `encoded_spikes = (spike_i64 & 0xFF).astype(np.uint8)` This assigns ‘(spike_i64 & 0xFF).astype(np.uint8)‘ to ‘encoded_spikes ‘ in host memory. Here encoded_spikes means a binary event produced when a neuron crosses its threshold. The important operands are spike_i64: a binary event produced when a neuron crosses its threshold; astype: the value named ’astype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 528: `encoded_weights = (weight_i64 & 0xFF).astype(np.uint8)` This assigns ‘(weight_i64 & 0xFF).astype(np.uint8)‘ to ‘encoded_weights ‘ in host memory. Here encoded_weights means a learned coefficient multiplied by an activation. The important operands are weight_i64: a learned coefficient multiplied by an activation; astype: the value named ’astype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 529: `instructions: list[TensorInstruction] = []` This assigns '[]' to 'instructions' in host memory. Here instructions means the value named 'instructions' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 530: `bindings: list[ResultBinding] = []` This assigns '[]' to 'bindings' in host memory. Here bindings means the value named 'bindings' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 531: `estimated_bytes = 0` This assigns '0' to 'estimated_bytes ' in host memory. Here estimated_bytes means the value named 'estimated bytes' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 532: `compute_opcode = Opcode.COMPUTE_TERNARY if ternary else Opcode.COMPUTE_BINARY` This assigns 'Opcode.COMPUTE_TERNARY if ternary else Opcode.COMPUTE_BINARY' to 'compute_opcode ' in host memory. Here compute_opcode means the value named 'compute opcode' in this block. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_TERNARY: the value named 'compute ternary' in this block; ternary: the value named 'ternary' in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 534: `def append_load(command: Command128, beats: Sequence[StreamBeat], description: str) -> None:` This starts function append_load. Its indented body performs append load as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 535: `nonlocal estimated_bytes` Inside the function append_load, which handles append load, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are nonlocal: the value named 'nonlocal' in this block; estimated_bytes: the value named 'estimated bytes' in this block.

Line 536: `instructions.append(TensorInstruction(command, beats, 0, description))` This calls 'instructions.append' with 'TensorInstruction(command, beats, 0, description)' inside the function append_load, which handles append load. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 537: `estimated_bytes += 8 * len(beats)` Inside the function append_load, which handles append load, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are estimated_bytes: the value named 'estimated bytes' in this block; len: the value named 'len' in this block; beats: the value named 'beats' in this block.

Line 539: `for reduction_start in range(0, reduction, MAX_REDUCTION):` This iterates reduction_start over 'range(0, reduction, MAX_REDUCTION)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_spike_gemm, which handles compile spike gemm.

Line 540: `reduction_count = min(MAX_REDUCTION, reduction - reduction_start)` This assigns 'min(MAX_REDUCTION reduction - reduction_start)' to 'reduction_count ' in host memory. Here reduction_count means an integer count retained for control or measurable telemetry. The important operands are min: the value named 'min' in this block; MAX_REDUCTION: the value named 'max reduction' in this block; reduction: the value named 'reduction' in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 541: `selected = dataflow or choose_dataflow(rows, columns, reduction_count).mode` This assigns 'dataflow or choose_dataflow(rows, columns, reduction_count).mode' to 'selected ' in host memory. Here selected means the value named 'selected' in this block. The important operands are dataflow: the value named 'dataflow' in this block; or: the value named 'or' in this block; choose_dataflow: the value named 'choose dataflow' in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 542: `if selected not in {Dataflow.INPUT_BROADCAST, Dataflow.WEIGHT_BROADCAST}:` This executes the following indented host-code branch only when 'selected not in {Dataflow.INPUT_BROADCAST, Dataflow.WEIGHT_BROADCAS is true. The important operands are selected: the value named 'selected' in this block; not: the value named 'not' in this block; in: the value named 'in' in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 543: `raise ValueError(f"unsupported dataflow {selected}")` This rejects an invalid state by raising 'ValueError(f"unsupported dataflow {selected}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 545: `def load_activations(row_start: int) -> None:` This starts function load_activations. Its indented body performs load activations as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 546: `tile_rows = min(ROW_TILE, rows - row_start)` This assigns 'min(ROW_TILE, rows - row_start)' to 'tile_rows ' in host memory. Here tile_rows means one output-row position processed by the tensor tile. The important operands are min: the value named 'min' in this block; ROW_TILE: one output-row position processed by the tensor tile; rows: one output-row position processed by the tensor tile. Context: the function load_activations, which handles load activations.

Line 547: `vectors = np.zeros((reduction_count, ROW_TILE), dtype=np.uint8)` This assigns 'np.zeros((reduction_count, ROW_TILE), dtype=np.uint8)' to 'vectors ' in host memory. Here vectors means the value named 'vectors' in this block. The important operands are zeros: the value named 'zeros' in this block; reduction_count: an integer count retained for control or measurable telemetry; ROW_TILE: one output-row position processed by the tensor tile. Context: the function load_activations, which handles load activations.

Line 548: `vectors[:, :tile_rows] = encoded_spikes[` This statement continues through source line 551; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'encoded_spikes[row_start:row_start + tile_rows, reduction_start:reduction_start + reduction_count,].T' to 'vectors[' in host memory. Here vectors means the value named 'vectors' in this block. The important operands are encoded_spikes: a binary event produced when a neuron crosses its threshold; row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Context: the function load_activations, which handles load activations.

Line 549: `row_start:row_start + tile_rows,` This is continuation line 2 of the 4-line logical statement that begins at source line 548. Its exact fragment is 'row_start:row_start + tile_rows,'. The important operands are row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This assigns 'encoded_spikes[row_start:row_start + tile_rows, reduction_start:reduction_start + reduction_count,].T' to 'vectors[' in host memory. Here vectors means the value named 'vectors' in this block. The important operands are encoded_spikes: a binary event produced when a neuron crosses its threshold; row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Context: the function load_activations, which handles load activations.

Line 550: `reduction_start:reduction_start + reduction_count,` This is continuation line 3 of the 4-line logical statement that begins at source line 548. Its exact fragment is 'reduction_start:reduction_start + reduction_count,'. The important operands are reduction_start: the value named 'reduction start' in this block; reduction_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This assigns 'encoded_spikes[row_start:row_start + tile_rows, reduction_start:reduction_start + reduction_count,].T' to 'vectors[' in host memory. Here vectors means the value named 'vectors' in this block. The important operands are encoded_spikes: a binary event produced when a neuron crosses its threshold; row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Context: the function load_activations, which handles load activations.

Line 551: `].T` This is continuation line 4 of the 4-line logical statement that begins at source line 548. Its exact fragment is '].T'. The important operands are T: the value named 't' in this block. Read together, the complete operation is:

This assigns ‘encoded_spikes[row_start:row_start + tile_rows, reduction_start:reduction_start + reduction_count,].T‘ to ‘vectors[‘ in host memory. Here vectors means the value named ’vectors’ in this block. The important operands are encoded_spikes: a binary event produced when a neuron crosses its threshold; row_start: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Context: the function load_activations, which handles load activations.

Line 552: `append_load(` This statement continues through source line 556; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘append_load‘ with ‘ tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=reduction_count), _pack_compiler_u8_rows(vectors, 8), f"load {’ternary’ if ternary else ’binary’} spike tile at row {row_start}", ‘ inside the function load_activations, which handles load activations. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 553: `tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=reduction_count),` This is continuation line 2 of the 5-line logical statement that begins at source line 552. Its exact fragment is ‘tensor_command(Opcode.LOAD_ACTIVATIO issue_count=reduction_count),‘. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named ’opcode’ in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This calls ‘append_load‘ with ‘ tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=reduction_count), _pack_compiler_u8_rows(vectors, 8), f"load {’ternary’ if ternary else ’binary’} spike tile at row {row_start}", ‘ inside the function load_activations, which handles load activations. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 554: `_pack_compiler_u8_rows(vectors, 8),` This is continuation line 3 of the 5-line logical statement that begins at source line 552. Its exact fragment is ‘_pack_compiler_u8_rows(vectors, 8),‘. The important operands are _pack_compiler_u8_rows: one output-row position processed by the tensor tile; vectors: the value named ’vectors’ in this block. Read together, the complete operation is: This calls ‘append_load‘ with ‘ tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=reduction_count), _pack_compiler_u8_rows(vectors, 8), f"load {’ternary’ if ternary else ’binary’} spike tile at row {row_start}", ‘ inside the function load_activations, which handles load activations. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 555: `f"load {’ternary’ if ternary else ’binary’} spike tile at row {row_start}",` This is continuation line 4 of the 5-line logical statement that begins at source line 552. Its exact fragment is ‘f"load {’ternary’ if ternary else ’binary’} spike tile at row {row_start}",‘. The important operands are f: the value named ’f’ in this block; load: the value named ’load’ in this block; ternary: the value named ’ternary’ in this block. Read together, the complete operation is: This calls ‘append_load‘ with ‘ tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=reduction_count), _pack_compiler_u8_rows(vectors, 8), f"load {’ternary’ if ternary else ’binary’} spike tile at row {row_start}", ‘ inside the function load_activations, which handles load activations. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 556: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 552. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘append_load‘ with ‘ tensor_command(Opcode.LOAD_ACTIVATIONS, issue_count=reduction_count), _pack_compiler_u8_rows(vectors, 8), f"load {’ternary’ if ternary else ’binary’} spike tile at row {row_start}", ‘ inside the function load_activations, which handles load activations. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 558: `def load_weights(column_start: int) -> None:` This starts function load_weights. Its indented body performs load weights as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 559: `tile_columns = min(COLUMN_TILE, columns - column_start)` This assigns ‘min(COLUMN_TILE, columns - column_start)‘ to ‘tile_columns ‘ in host memory. Here tile_columns means one output-feature position processed in parallel. The important operands are min: the value named ’min’ in this block; COLUMN_TILE: one output-feature position processed in parallel; columns: one output-feature position processed in parallel. Context: the function load_weights, which handles load weights.

Line 560: `matrix = np.zeros((reduction_count, COLUMN_TILE), dtype=np.uint8)` This assigns ‘np.zeros((reduction_coun COLUMN_TILE), dtype=np.uint8)‘ to ‘matrix ‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are zeros: the value named ’zeros’ in this block; reduction_count: an integer count retained for control or measurable telemetry; COLUMN_TILE: one output-feature position processed in parallel. Context: the function load_weights, which handles load weights.

Line 561: `matrix[:, :tile_columns] = encoded_weights[` This statement continues through source line 564; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘encoded_weights[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘matrix[‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are encoded_weights: a learned coefficient multiplied by an activation; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function load_weights, which handles load weights.

Line 562: `reduction_start:reduction_start + reduction_count,` This is continuation line 2 of the 4-line logical statement that begins at source line 561. Its exact fragment is ‘reduction_start:reduction_start + reduction_count,‘. The important operands are reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This assigns ‘encoded_weights[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘matrix[‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are encoded_weights: a learned coefficient multiplied by an activation; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function load_weights, which handles load weights.

Line 563: `column_start:column_start + tile_columns,` This is continuation line 3 of the 4-line logical statement that begins at source line 561. Its exact fragment is ‘column_start:column_start + tile_columns,‘. The important operands are column_start: one output-feature position processed in parallel; tile_columns: one output-feature position processed in parallel. Read together, the complete operation is: This assigns ‘encoded_weights[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘matrix[‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are encoded_weights: a learned coefficient multiplied by an activation; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function load_weights, which handles load weights.

Line 564: `]` This is continuation line 4 of the 4-line logical statement that begins at source line 561. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘encoded_weights[reduction_start:reduction_start + reduction_count, column_start:column_start + tile_columns,]‘ to ‘matrix[‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are encoded_weights: a learned coefficient multiplied by an activation; reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Context: the function load_weights, which handles load weights.

Line 565: `append_load(` This statement continues through source line 569; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘append_load‘ with ‘ tensor_command(Opcode.LOAD_WEIGHTS_OUTE issue_count=reduction_count), _pack_compiler_u8_rows(matrix, 16), f"load signed weight tile at column {column_start}", ‘ inside the function load_weights, which handles load weights. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 566: `tensor_command(Opcode.LOAD_WEIGHTS_OUTER, issue_count=reduction_count),` This is continuation line 2 of the 5-line logical statement that begins at source line 565. Its exact fragment is ‘tensor_command(Opcode.LOAD_WEIGHT issue_count=reduction_count),‘. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named ’opcode’ in this block; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls ‘append_load‘ with ‘ tensor_command(Opcode.LOAD_WEIGHTS_OUTER, issue_count=reduction_count), _pack_compiler_u8_rows(matrix, 16), f"load signed weight tile at column {column_start}", ‘ inside the function load_weights, which handles load weights.

The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 567: `_pack_compiler_u8_rows(matrix, 16),` This is continuation line 3 of the 5-line logical statement that begins at source line 565. Its exact fragment is '_pack_compiler_u8_rows(matrix, 16),'. The important operands are _pack_compiler_u8_rows: one output-row position processed by the tensor tile; matrix: the value named 'matrix' in this block. Read together, the complete operation is: This calls 'append_load' with ' tensor_command(Opcode.LOAD_WEIGHTS_OU issue_count=reduction_count), _pack_compiler_u8_rows(matrix, 16), f"load signed weight tile at column {column_start}", ' inside the function load_weights, which handles load weights. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 568: `f"load signed weight tile at column {column_start}",` This is continuation line 4 of the 5-line logical statement that begins at source line 565. Its exact fragment is 'f"load signed weight tile at column {column_start}",'. The important operands are f: the value named 'f' in this block; load: the value named 'load' in this block; weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls 'append_load' with ' tensor_command(Opcode.LOAD_WEIGHTS_OUTER, issue_count=reduction_count), _pack_compiler_u8_rows(matrix, 16), f"load signed weight tile at column {column_start}", ' inside the function load_weights, which handles load weights. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 569: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 565. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'append_load' with ' tensor_command(Opcode.LOAD_WEIGHTS_OUTER, issue_count=reduction_count), _pack_compiler_u8_rows(matrix, 16), f"load signed weight tile at column {column_start}", ' inside the function load_weights, which handles load weights. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 571: `def append_compute(row_start: int, column_start: int) -> None:` This starts function append_compute. Its indented body performs append compute as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 572: `tile_rows = min(ROW_TILE, rows - row_start)` This assigns 'min(ROW_TILE, rows - row_start)' to 'tile_rows ' in host memory. Here tile_rows means one output-row position processed by the tensor tile. The important operands are min: the value named 'min' in this block; ROW_TILE: one output-row position processed by the tensor tile; rows: one output-row position processed by the tensor tile. Context: the function append_compute, which handles append compute.

Line 573: `tile_columns = min(COLUMN_TILE, columns - column_start)` This assigns 'min(COLUMN_TILE, columns - column_start)' to 'tile_columns ' in host memory. Here tile_columns means one output-feature position processed in parallel. The important operands are min: the value named 'min' in this block; COLUMN_TILE: one output-feature position processed in parallel; columns: one output-feature position processed in parallel. Context: the function append_compute, which handles append compute.

Line 574: `instruction_index = len(instructions)` This assigns 'len(instructions)' to 'instruction_index ' in host memory. Here instruction_index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are len: the value named 'len' in this block; instructions: the value named 'instructions' in this block. Context: the function append_compute, which handles append compute.

Line 575: `instructions.append(TensorInstruction(` This statement continues through source line 585; the explanation uses the complete expression rather than only the first visual fragment. This calls 'instructions.append' with 'TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)' inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 576: `tensor_command(` This is continuation line 2 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘tensor_command(‘. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 577: `compute_opcode,` This is continuation line 3 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘compute_opcode,‘. The important operands are compute_opcode: the value named ’compute opcode’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 578: `rows=tile_rows,` This is continuation line 4 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘rows=tile_rows,‘. The important operands are rows: one output-row position processed by the tensor tile; tile_rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 579: `columns=tile_columns,` This is continuation line 5 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘columns=tile_columns,‘. The important operands are columns: one output-feature position processed in parallel; tile_columns: one output-feature position processed in parallel. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 580: `issue_count=reduction_count,` This is continuation line 6 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘issue_count=reduction_count,‘. The important operands are issue_count: an integer count retained for control or measurable telemetry; reduction_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 581: `),` This is continuation line 7 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 582: `(),` This is continuation line 8 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘(),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append

compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 583: `tile_rows * tile_columns,` This is continuation line 9 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘tile_rows * tile_columns,‘. The important operands are tile_rows: one output-row position processed by the tensor tile; tile_columns: one output-feature position processed in parallel. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 584: `f"compute sparse C tile at ({row_start},{column_start})",` This is continuation line 10 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘f"compute sparse C tile at ({row_start},{column_start})",‘. The important operands are f: the value named ’f’ in this block; compute: the value named ’compute’ in this block; sparse: the value named ’sparse’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 585: `))` This is continuation line 11 of the 11-line logical statement that begins at source line 575. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘TensorInstruction(tensor_command(compute_opcode, rows=tile_rows, columns=tile_columns, issue_count=reduction_count,), (), tile_rows * tile_columns, f"compute sparse C tile at ({row_start},{column_start})",)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 586: `bindings.append(ResultBinding(` This statement continues through source line 589; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘bindings.append‘ with ‘ResultBinding(instruction_index, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count,)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 587: `instruction_index, row_start, column_start, tile_rows, tile_columns,` This is continuation line 2 of the 4-line logical statement that begins at source line 586. Its exact fragment is ‘instruction_index, row_start, column_start, tile_rows, tile_columns,‘. The important operands are instruction_index: an integer selecting one tensor element, lane, bank, or state entry; row_start: one output-row position processed by the tensor tile; column_start: one output-feature position processed in parallel. Read together, the complete operation is: This calls ‘bindings.append‘ with ‘ResultBinding(instruction_index, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count,)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 588: `reduction_start, reduction_count,` This is continuation line 3 of the 4-line logical statement that begins at source line 586. Its exact fragment is ‘reduction_start, reduction_count,‘. The important operands are reduction_start: the value named ’reduction start’ in this block; reduction_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This calls ‘bindings.append‘ with ‘ResultBinding(instruction_index, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count,)‘ inside the function append_compute, which handles append compute. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 589: `))` This is continuation line 4 of the 4-line logical statement that begins at source line 586. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘bindings.append‘ with ‘ResultBinding(instruction_index, row_start, column_start, tile_rows, tile_columns, reduction_start, reduction_count,)‘ inside the function append_compute, which handles append compute.

The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 591: `if selected == Dataflow.INPUT_BROADCAST:` This executes the following indented host-code branch only when 'selected == Dataflow.INPUT_BROADCAST' is true. The important operands are selected: the value named 'selected' in this block; Dataflow: the value named 'dataflow' in this block; INPUT_BROADCAST: the value named 'input broadcast' in this block. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 592: `for row_start in range(0, rows, ROW_TILE):` This iterates row_start over 'range(0, rows, ROW_TILE)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_spike_gemm, which handles compile spike gemm.

Line 593: `load_activations(row_start)` This calls 'load_activations' with 'row_start' inside the function compile_spike_gemm, which handles compile spike gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 594: `for column_start in range(0, columns, COLUMN_TILE):` This iterates column_start over 'range(0, columns, COLUMN_TILE)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_spike_gemm, which handles compile spike gemm.

Line 595: `load_weights(column_start)` This calls 'load_weights' with 'column_start' inside the function compile_spike_gemm, which handles compile spike gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 596: `append_compute(row_start, column_start)` This calls 'append_compute' with 'row_start, column_start' inside the function compile_spike_gemm, which handles compile spike gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 597: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function compile_spike_gemm, which handles compile spike gemm.

Line 598: `for column_start in range(0, columns, COLUMN_TILE):` This iterates column_start over 'range(0, columns, COLUMN_TILE)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_spike_gemm, which handles compile spike gemm.

Line 599: `load_weights(column_start)` This calls 'load_weights' with 'column_start' inside the function compile_spike_gemm, which handles compile spike gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 600: `for row_start in range(0, rows, ROW_TILE):` This iterates row_start over 'range(0, rows, ROW_TILE)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_spike_gemm, which handles compile spike gemm.

Line 601: `load_activations(row_start)` This calls 'load_activations' with 'row_start' inside the function compile_spike_gemm, which handles compile spike gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 602: `append_compute(row_start, column_start)` This calls 'append_compute' with 'row_start, column_start' inside the function compile_spike_gemm, which handles compile spike gemm. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 604: `spike_adds = int(np.count_nonzero(spike_i64)) * columns` This assigns ‘int(np.count_nonzero(spike_i64)) * columns‘ to ‘spike_adds ‘ in host memory. Here spike_adds means a binary event produced when a neuron crosses its threshold. The important operands are count_nonzero: the value named ’count nonzero’ in this block; spike_i64: a binary event produced when a neuron crosses its threshold; columns: one output-feature position processed in parallel. Context: the function compile_spike_gemm, which handles compile spike gemm.

Line 605: `return CompiledGemm(` This statement continues through source line 608; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘CompiledGemm((rows, reduction, columns), 0, 0, tuple(instructions), tuple(bindings), bias_tuple, estimated_bytes, rows * reduction * columns, spike_adds,)‘ to the caller of the function compile_spike_gemm, which handles compile spike gemm, ending this invocation. The important operands are CompiledGemm: the value named ’compiled gemm’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 606: `(rows, reduction, columns), 0, 0, tuple(instructions), tuple(bindings),` This is continuation line 2 of the 4-line logical statement that begins at source line 605. Its exact fragment is ‘(rows, reduction, columns), 0, 0, tuple(instructions), tuple(bindings),‘. The important operands are rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block; columns: one output-feature position processed in parallel. Read together, the complete operation is: This returns ‘CompiledGemm((rows, reduction, columns), 0, 0, tuple(instructions), tuple(bindings), bias_tuple, estimated_bytes, rows * reduction * columns, spike_adds,)‘ to the caller of the function compile_spike_gemm, which handles compile spike gemm, ending this invocation. The important operands are CompiledGemm: the value named ’compiled gemm’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 607: `bias_tuple, estimated_bytes, rows * reduction * columns, spike_adds,` This is continuation line 3 of the 4-line logical statement that begins at source line 605. Its exact fragment is ‘bias_tuple, estimated_bytes, rows * reduction * columns, spike_adds,‘. The important operands are bias_tuple: an additive learned offset applied after accumulation; estimated_bytes: the value named ’estimated bytes’ in this block; rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This returns ‘CompiledGemm((rows, reduction, columns), 0, 0, tuple(instructions), tuple(bindings), bias_tuple, estimated_bytes, rows * reduction * columns, spike_adds,)‘ to the caller of the function compile_spike_gemm, which handles compile spike gemm, ending this invocation. The important operands are CompiledGemm: the value named ’compiled gemm’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 608: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 605. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘CompiledGemm((rows, reduction, columns), 0, 0, tuple(instructions), tuple(bindings), bias_tuple, estimated_bytes, rows * reduction * columns, spike_adds,)‘ to the caller of the function compile_spike_gemm, which handles compile spike gemm, ending this invocation. The important operands are CompiledGemm: the value named ’compiled gemm’ in this block; rows: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block.

Line 611: `def compile_u8_conv2d(` This starts function compile_u8_conv2d. Its indented body performs compile u8 conv2d as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 612: `activations: np.ndarray | Sequence,` Inside the function compile_u8_conv2d, which handles compile u8 conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named ’ndarray’ in this block; Sequence: the value named ’sequence’ in this block.

Line 613: `weights: np.ndarray | Sequence,` Inside the function compile_u8_conv2d, which handles compile u8 conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are weights: a learned coefficient multiplied by an activation; ndarray: the value named ’ndarray’ in this block; Sequence: the value named ’sequence’ in this block.

Line 614: `*,` Inside the function compile_u8_conv2d, which handles compile u8 conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 615: `activation_zero_point: int = 0,` This assigns ‘0,‘ to ‘activation_zero_point‘ in host memory. Here activation_zero_point means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 616: `weight_zero_point: int = 0,` This assigns ‘0,‘ to ‘weight_zero_point‘ in host memory. Here weight_zero_point means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 617: `bias: Sequence[int] | np.ndarray | None = None,` This assigns ‘None,‘ to ‘bias‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 618: `stride: int | Sequence[int] = 1,` This assigns ‘1,‘ to ‘stride‘ in host memory. Here stride means the value named 'stride' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 619: `padding: int | Sequence[int] = 0,` This assigns ‘0,‘ to ‘padding‘ in host memory. Here padding means the value named 'padding' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 620: `dilation: int | Sequence[int] = 1,` This assigns ‘1,‘ to ‘dilation‘ in host memory. Here dilation means the value named 'dilation' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 621: `groups: int = 1,` This assigns ‘1,‘ to ‘groups‘ in host memory. Here groups means the value named 'groups' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 622: `dataflow: Dataflow | None = None,` This assigns ‘None,‘ to ‘dataflow‘ in host memory. Here dataflow means the value named 'dataflow' in this block. The important operands are None: the value named 'none' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 623: `) -> CompiledConv2d:` Inside the function compile_u8_conv2d, which handles compile u8 conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are CompiledConv2d: the value named 'compiled conv2d' in this block.

Line 624: `"""Lower an NCHW/OIHW asymmetric-uint8 convolution to tensor commands."""` This is executable documentation attached to the function compile_u8_conv2d, which handles compile u8 conv2d. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 625: `a = np.asarray(activations)` This assigns ‘np.asarray(activations)‘ to ‘a ‘ in host memory. Here a means the value named 'a' in this block. The important operands are asarray: the value named 'asarray' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 626: `w = np.asarray(weights)` This assigns ‘np.asarray(weights)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are asarray: the value named ’asarray’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 627: `if a.ndim != 4 or w.ndim != 4:` This executes the following indented host-code branch only when ‘a.ndim != 4 or w.ndim != 4‘ is true. The important operands are a: the value named ’a’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 628: `raise ValueError("convolution activations and weights must be NCHW and OIHW tensors")` This rejects an invalid state by raising ‘ValueError("convolution activations and weights must be NCHW and OIHW tensors")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 629: `if not np.issubdtype(a.dtype, np.integer) or not np.issubdtype(w.dtype, np.integer):` This executes the following indented host-code branch only when ‘not np.issubdtype(a.dtype, np.integer) or not np.issubdtype(w.dtype, np.integer)‘ is true. The important operands are not: the value named ’not’ in this block; issubdtype: the value named ’issubdtype’ in this block; a: the value named ’a’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 630: `raise ValueError("convolution activations and weights must contain integers")` This rejects an invalid state by raising ‘ValueError("convolution activations and weights must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 631: `a_i64 = a.astype(np.int64)` This assigns ‘a.astype(np.int64)‘ to ‘a_i64 ‘ in host memory. Here a_i64 means the value named ’a i64’ in this block. The important operands are a: the value named ’a’ in this block; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 632: `w_i64 = w.astype(np.int64)` This assigns ‘w.astype(np.int64)‘ to ‘w_i64 ‘ in host memory. Here w_i64 means the value named ’w i64’ in this block. The important operands are w: the value named ’w’ in this block; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 633: `if np.any(a_i64 < 0) or np.any(a_i64 > 255):` This executes the following indented host-code branch only when ‘np.any(a_i64 $<$ 0) or np.any(a_i64 $>$ 255)‘ is true. The important operands are any: the value named ’any’ in this block; a_i64: the value named ’a i64’ in this block; or: the value named ’or’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 634: `raise ValueError("convolution activations must be in [0,255]")` This rejects an invalid state by raising ‘ValueError("convolution activations must be in [0,255]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 635: `if np.any(w_i64 < 0) or np.any(w_i64 > 255):` This executes the following indented host-code branch only when ‘np.any(w_i64 $<$ 0) or np.any(w_i64 $>$ 255)‘ is true. The important operands are any: the value named ’any’ in this block; w_i64: the value named ’w i64’ in this block; or: the value named ’or’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 636: `raise ValueError("convolution weights must be in [0,255]")` This rejects an invalid state by raising ‘ValueError("convolution weights must be in [0,255]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 637: `a_u8 = a_i64.astype(np.uint8)` This assigns ‘a_i64.astype(np.uint8)‘ to ‘a_u8 ‘ in host memory. Here a_u8 means the value named ’a u8’ in this block. The important operands are a_i64: the value named ’a i64’ in this block; astype: the value named ’astype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 638: `w_u8 = w_i64.astype(np.uint8)` This assigns ‘w_i64.astype(np.uint8)‘ to ‘w_u8 ‘ in host memory. Here w_u8 means the value named ’w u8’ in this block. The important operands are w_i64: the value named ’w i64’ in this block; astype: the value named ’astype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 639: `za = _u8("activation_zero_point", activation_zero_point)` This assigns ‘_u8("activation_zero_point", activation_zero_point)‘ to ‘za ‘ in host memory. Here za means the value named ’za’ in this block. The important operands are _u8: the value named ’u8’ in this block; activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 640: `zb = _u8("weight_zero_point", weight_zero_point)` This assigns ‘_u8("weight_zero_point", weight_zero_point to ‘zb ‘ in host memory. Here zb means the value named ’zb’ in this block. The important operands are _u8: the value named ’u8’ in this block; weight_zero_point: a learned coefficient multiplied by an activation. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 641: `stride_pair = _pair("stride", stride)` This assigns ‘_pair("stride", stride)‘ to ‘stride_pair ‘ in host memory. Here stride_pair means the value named ’stride pair’ in this block. The important operands are _pair: the value named ’pair’ in this block; stride: the value named ’stride’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 642: `padding_pair = _pair("padding", padding, allow_zero=True)` This assigns ‘_pair("padding", padding, allow_zero=True)‘ to ‘padding_pair ‘ in host memory. Here padding_pair means the value named ’padding pair’ in this block. The important operands are _pair: the value named ’pair’ in this block; padding: the value named ’padding’ in this block; allow_zero: the value named ’allow zero’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 643: `dilation_pair = _pair("dilation", dilation)` This assigns ‘_pair("dilation", dilation)‘ to ‘dilation_pair ‘ in host memory. Here dilation_pair means the value named ’dilation pair’ in this block. The important operands are _pair: the value named ’pair’ in this block; dilation: the value named ’dilation’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 645: `batch, input_channels, height, width = a_u8.shape` This assigns ‘a_u8.shape‘ to ‘batch, input_channels, height, width ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are a_u8: the value named ’a u8’ in this block; shape: the value named ’shape’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 646: `output_channels, weight_channels, kernel_height, kernel_width = w_u8.shape` This assigns ‘w_u8.shape‘ to ‘output_channels, weight_channels, kernel_height, kernel_width ‘ in host memory. Here output_channels means the value named ’output channels’ in this block. The important operands are w_u8: the value named ’w u8’ in this block; shape: the value named ’shape’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 647: `group_count = int(groups)` This assigns ‘int(groups)‘ to ‘group_count ‘ in host memory. Here group_count means an integer count retained for control or measurable telemetry. The important operands are groups: the value named ’groups’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 648: `if group_count <= 0 or input_channels % group_count or output_channels % group_count:` This executes the following indented host-code branch only when ‘group_count <= 0 or input_channels % group_count or

output_channels % group_count‘ is true. The important operands are group_count: an integer count retained for control or measurable telemetry; or: the value named 'or' in this block; input_channels: the value named 'input channels' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 649: `raise ValueError("groups must divide both input and output channels")` This rejects an invalid state by raising ‘ValueError("groups must divide both input and output channels")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 650: `channels_per_group = input_channels // group_count` This assigns ‘input_channels // group_count‘ to ‘channels_per_group ‘ in host memory. Here channels_per_group means the value named 'channels per group' in this block. The important operands are input_channels: the value named 'input channels' in this block; group_count: an integer count retained for control or measurable telemetry. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 651: `outputs_per_group = output_channels // group_count` This assigns ‘output_channels // group_count‘ to ‘outputs_per_group ‘ in host memory. Here outputs_per_group means the value named 'outputs per group' in this block. The important operands are output_channels: the value named 'output channels' in this block; group_count: an integer count retained for control or measurable telemetry. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 652: `if weight_channels != channels_per_group:` This executes the following indented host-code branch only when ‘weight_channels != channels_per_group‘ is true. The important operands are weight_channels: a learned coefficient multiplied by an activation; channels_per_group: the value named 'channels per group' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 653: `raise ValueError("OIHW weight input channels must equal input_channels/groups")` This rejects an invalid state by raising ‘ValueError("OIHW weight input channels must equal input_channels/groups")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 654: `if kernel_height == 0 or kernel_width == 0:` This executes the following indented host-code branch only when ‘kernel_height == 0 or kernel_width == 0‘ is true. The important operands are kernel_height: the value named 'kernel height' in this block; or: the value named 'or' in this block; kernel_width: the value named 'kernel width' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 655: `raise ValueError("convolution kernel dimensions must be non-zero")` This rejects an invalid state by raising ‘ValueError("convolution kernel dimensions must be non-zero")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 656: `if bias is None:` This executes the following indented host-code branch only when ‘bias is None‘ is true. The important operands are bias: an additive learned offset applied after accumulation; is: the value named 'is' in this block; None: the value named 'none' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 657: `bias_values = None` This assigns ‘None‘ to ‘bias_values ‘ in host memory. Here bias_values means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 658: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 659: `bias_array = np.asarray(bias)` This assigns ‘np.asarray(bias)‘ to ‘bias_array ‘ in host memory. Here bias_array means an additive learned offset applied after accumulation. The important operands are asarray: the value named 'asarray' in this block; bias: an additive learned offset applied after accumulation. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 660: `if bias_array.ndim != 1 or len(bias_array) != output_channels:` This executes the following indented host-code branch only when ‘bias_array.ndim != 1 or len(bias_array) != output_channels‘ is true. The important operands are bias_array: an additive learned offset applied after accumulation; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 661: `raise ValueError("convolution bias must have one entry per output channel")` This rejects an invalid state by raising ‘ValueError("convolution bias must have one entry per output channel")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 662: `if not np.issubdtype(bias_array.dtype, np.integer):` This executes the following indented host-code branch only when ‘not np.issubdtype(bias_array.dtype, np.integer)‘ is true. The important operands are not: the value named ’not’ in this block; issubdtype: the value named ’issubdtype’ in this block; bias_array: an additive learned offset applied after accumulation. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 663: `raise ValueError("convolution bias must contain integers")` This rejects an invalid state by raising ‘ValueError("convolution bias must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 664: `bias_values = bias_array.astype(np.int64)` This assigns ‘bias_array.astype(np.int64)‘ to ‘bias_values ‘ in host memory. Here bias_values means an additive learned offset applied after accumulation. The important operands are bias_array: an additive learned offset applied after accumulation; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 666: `programs: list[ConvGroupProgram] = []` This assigns ‘[]‘ to ‘programs‘ in host memory. Here programs means the value named ’programs’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 667: `output_height = output_width = 0` This assigns ‘output_width = 0‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are output_width: the value named ’output width’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 668: `for group in range(group_count):` This iterates group over ‘range(group_count)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 669: `input_start = group * channels_per_group` This assigns ‘group * channels_per_group‘ to ‘input_start ‘ in host memory. Here input_start means the value named ’input start’ in this block. The important operands are group: the value named ’group’ in this block; channels_per_group: the value named ’channels per group’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 670: `output_start = group * outputs_per_group` This assigns ‘group * outputs_per_group‘ to ‘output_start ‘ in host memory. Here output_start means the value named ’output start’ in this block. The important operands are group: the value named ’group’ in this block; outputs_per_group: the value named ’outputs per group’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 671: `columns, output_height, output_width = _im2col_nchw(` This statement continues through source line 675; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_im2col_nchw(a_u8[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, za,)‘ to ‘columns, output_height, output_width ‘ in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named ’im2col nchw’ in this block; a_u8: the value named ’a u8’ in this block; input_start: the value named ’input start’ in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 672: `a_u8[:, input_start:input_start + channels_per_group],` This is continuation line 2 of the 5-line logical statement that begins at source line 671. Its exact fragment is 'a_u8[:, input_start:input_start + channels_per_group],'. The important operands are a_u8: the value named 'a u8' in this block; input_start: the value named 'input start' in this block; channels_per_group: the value named 'channels per group' in this block. Read together, the complete operation is: This assigns '_im2col_nchw(a_u8[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, za,)' to 'columns, output_height, output_width ' in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named 'im2col nchw' in this block; a_u8: the value named 'a u8' in this block; input_start: the value named 'input start' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 673: `(kernel_height, kernel_width), stride_pair, padding_pair,` This is continuation line 3 of the 5-line logical statement that begins at source line 671. Its exact fragment is '(kernel_height, kernel_width), stride_pair, padding_pair,'. The important operands are kernel_height: the value named 'kernel height' in this block; kernel_width: the value named 'kernel width' in this block; stride_pair: the value named 'stride pair' in this block. Read together, the complete operation is: This assigns '_im2col_nchw(a_u8[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, za,)' to 'columns, output_height, output_width ' in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named 'im2col nchw' in this block; a_u8: the value named 'a u8' in this block; input_start: the value named 'input start' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 674: `dilation_pair, za,` This is continuation line 4 of the 5-line logical statement that begins at source line 671. Its exact fragment is 'dilation_pair, za,'. The important operands are dilation_pair: the value named 'dilation pair' in this block; za: the value named 'za' in this block. Read together, the complete operation is: This assigns '_im2col_nchw(a_u8[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, za,)' to 'columns, output_height, output_width ' in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named 'im2col nchw' in this block; a_u8: the value named 'a u8' in this block; input_start: the value named 'input start' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 675: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 671. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_im2col_nchw(a_u8[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, za,)' to 'columns, output_height, output_width ' in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named 'im2col nchw' in this block; a_u8: the value named 'a u8' in this block; input_start: the value named 'input start' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 676: `weight_matrix = w_u8[` This statement continues through source line 678; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'w_u8[output_start:output_start + outputs_per_group].reshape(outputs_per_group, -1).T' to 'weight_matrix ' in host memory. Here weight_matrix means a learned coefficient multiplied by an activation. The important operands are w_u8: the value named 'w u8' in this block; output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 677: `output_start:output_start + outputs_per_group` This is continuation line 2 of the 3-line logical statement that begins at source line 676. Its exact fragment is 'output_start:output_start + outputs_per_group'. The important operands are output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Read together, the complete operation is: This assigns 'w_u8[output_start:output_start + outputs_per_group].reshape(outputs_per_group, -1).T' to 'weight_matrix ' in host memory. Here weight_matrix means a learned coefficient multiplied by an activation. The important operands are w_u8: the value named 'w u8' in this block; output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 678: `].reshape(outputs_per_group, -1).T` This is continuation line 3 of the 3-line logical statement that begins at source line 676. Its exact fragment is '].reshape(outputs_per_group, -1).T'. The important operands are reshape: the value named 'reshape' in this block; outputs_per_group: the value named 'outputs per group' in this block; T: the

value named 't' in this block. Read together, the complete operation is: This assigns ‘w_u8[output_start:output_start + outputs_per_group].reshape(outputs_per_group, -1).T‘ to ‘weight_matrix ‘ in host memory. Here weight_matrix means a learned coefficient multiplied by an activation. The important operands are w_u8: the value named 'w u8' in this block; output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 679: `group_bias = None if bias_values is None else bias_values[` This statement continues through source line 681; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘None if bias_values is None else bias_values[output_start:output_start + outputs_per_group]‘ to ‘group_bias ‘ in host memory. Here group_bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block; bias_values: an additive learned offset applied after accumulation; is: the value named 'is' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 680: `output_start:output_start + outputs_per_group` This is continuation line 2 of the 3-line logical statement that begins at source line 679. Its exact fragment is ‘output_start:output_start + outputs_per_group‘. The important operands are output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Read together, the complete operation is: This assigns ‘None if bias_values is None else bias_values[output_start:output_start + outputs_per_group]‘ to ‘group_bias ‘ in host memory. Here group_bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block; bias_values: an additive learned offset applied after accumulation; is: the value named 'is' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 681: `]` This is continuation line 3 of the 3-line logical statement that begins at source line 679. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘None if bias_values is None else bias_values[output_start:output_start + outputs_per_group]‘ to ‘group_bias ‘ in host memory. Here group_bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block; bias_values: an additive learned offset applied after accumulation; is: the value named 'is' in this block. Context: the function compile_u8_conv2d, which handles compile u8 conv2d.

Line 682: `programs.append(ConvGroupProgram(` This statement continues through source line 693; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘programs.append‘ with ‘ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)‘ inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 683: `output_start,` This is continuation line 2 of the 12-line logical statement that begins at source line 682. Its exact fragment is ‘output_start,‘. The important operands are output_start: the value named 'output start' in this block. Read together, the complete operation is: This calls ‘programs.append‘ with ‘ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)‘ inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 684: `outputs_per_group,` This is continuation line 3 of the 12-line logical statement that begins at source line 682. Its exact fragment is ‘outputs_per_group,‘. The important operands are outputs_per_group: the value named 'outputs per group' in this block. Read together, the complete operation is: This calls ‘programs.append‘ with ‘ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)‘ inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 685: `compile_u8_gemm(` This is continuation line 4 of the 12-line logical statement that begins at source line 682. Its exact fragment is ‘compile_u8_gemm(‘. The important operands are compile_u8_gemm: the value named 'compile

u8 gemm' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)' inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 686: `columns,` This is continuation line 5 of the 12-line logical statement that begins at source line 682. Its exact fragment is 'columns,'. The important operands are columns: one output-feature position processed in parallel. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)' inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 687: `weight_matrix,` This is continuation line 6 of the 12-line logical statement that begins at source line 682. Its exact fragment is 'weight_matrix,'. The important operands are weight_matrix: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)' inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 688: `activation_zero_point=za,` This is continuation line 7 of the 12-line logical statement that begins at source line 682. Its exact fragment is 'activation_zero_point=za,'. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; za: the value named 'za' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)' inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 689: `weight_zero_point=zb,` This is continuation line 8 of the 12-line logical statement that begins at source line 682. Its exact fragment is 'weight_zero_point=zb,'. The important operands are weight_zero_point: a learned coefficient multiplied by an activation; zb: the value named 'zb' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)' inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 690: `bias=group_bias,` This is continuation line 9 of the 12-line logical statement that begins at source line 682. Its exact fragment is 'bias=group_bias,'. The important operands are bias: an additive learned offset applied after accumulation; group_bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)' inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 691: `dataflow=dataflow,` This is continuation line 10 of the 12-line logical statement that begins at source line 682. Its exact fragment is 'dataflow=dataflow,'. The important operands are dataflow: the value named 'dataflow' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)' inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 692: `),` This is continuation line 11 of the 12-line logical statement that begins at source line 682. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘programs.append‘ with ‘ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)‘ inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 693: `))` This is continuation line 12 of the 12-line logical statement that begins at source line 682. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘programs.append‘ with ‘ConvGroupProgram(output_start, outputs_per_group, compile_u8_gemm(columns, weight_matrix, activation_zero_point=za, weight_zero_point=zb, bias=group_bias, dataflow=dataflow,),)‘ inside the function compile_u8_conv2d, which handles compile u8 conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 694: `return CompiledConv2d(` This statement continues through source line 699; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_u8_conv2d, which handles compile u8 conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

Line 695: `(batch, input_channels, height, width),` This is continuation line 2 of the 6-line logical statement that begins at source line 694. Its exact fragment is ‘(batch, input_channels, height, width),‘. The important operands are batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block; height: the value named ’height’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_u8_conv2d, which handles compile u8 conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

Line 696: `(batch, output_channels, output_height, output_width),` This is continuation line 3 of the 6-line logical statement that begins at source line 694. Its exact fragment is ‘(batch, output_channels, output_height, output_width),‘. The important operands are batch: the value named ’batch’ in this block; output_channels: the value named ’output channels’ in this block; output_height: the value named ’output height’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_u8_conv2d, which handles compile u8 conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

Line 697: `(kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair,` This is continuation line 4 of the 6-line logical statement that begins at source line 694. Its exact fragment is ‘(kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair,‘. The important operands are kernel_height: the value named ’kernel height’ in this block; kernel_width: the value named ’kernel width’ in this block; stride_pair: the value named ’stride pair’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_u8_conv2d, which handles compile u8 conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

Line 698: `group_count, tuple(programs),` This is continuation line 5 of the 6-line logical statement that begins at source line 694. Its exact fragment is ‘group_count, tuple(programs),‘. The important operands are group_count: an integer count retained for control or measurable telemetry; tuple: the value named ’tuple’ in this block; programs: the value named ’programs’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2d(

(batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_u8_conv2d, which handles compile u8 conv2d, ending this invocation. The important operands are CompiledConv2d: the value named 'compiled conv2d' in this block; batch: the value named 'batch' in this block; input_channels: the value named 'input channels' in this block.

Line 699: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 694. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_u8_conv2d, which handles compile u8 conv2d, ending this invocation. The important operands are CompiledConv2d: the value named 'compiled conv2d' in this block; batch: the value named 'batch' in this block; input_channels: the value named 'input channels' in this block.

Line 702: `def compile_spike_conv2d(` This starts function compile_spike_conv2d. Its indented body performs compile spike conv2d as part of the file that tiles matrices and convolutions into validated accelerator commands and packed stream payloads; it runs only when called.

Line 703: `spikes: np.ndarray | Sequence,` Inside the function compile_spike_conv2d, which handles compile spike conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are spikes: a binary event produced when a neuron crosses its threshold; ndarray: the value named 'ndarray' in this block; Sequence: the value named 'sequence' in this block.

Line 704: `weights: np.ndarray | Sequence,` Inside the function compile_spike_conv2d, which handles compile spike conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are weights: a learned coefficient multiplied by an activation; ndarray: the value named 'ndarray' in this block; Sequence: the value named 'sequence' in this block.

Line 705: `*,` Inside the function compile_spike_conv2d, which handles compile spike conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The expression is a literal or structural operator and has no hidden software state.

Line 706: `ternary: bool = False,` This assigns ‘False,‘ to ‘ternary‘ in host memory. Here ternary means the value named 'ternary' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 707: `bias: Sequence[int] | np.ndarray | None = None,` This assigns ‘None,‘ to ‘bias‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 708: `stride: int | Sequence[int] = 1,` This assigns ‘1,‘ to ‘stride‘ in host memory. Here stride means the value named 'stride' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 709: `padding: int | Sequence[int] = 0,` This assigns ‘0,‘ to ‘padding‘ in host memory. Here padding means the value named 'padding' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 710: `dilation: int | Sequence[int] = 1,` This assigns ‘1,‘ to ‘dilation‘ in host memory. Here dilation means the value named 'dilation' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 711: `groups: int = 1,` This assigns ‘1,‘ to ‘groups‘ in host memory. Here groups means the value named ’groups’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 712: `dataflow: Dataflow | None = None,` This assigns ‘None,‘ to ‘dataflow‘ in host memory. Here dataflow means the value named ’dataflow’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 713: `) -> CompiledConv2d:` Inside the function compile_spike_conv2d, which handles compile spike conv2d, this statement advances the host implementation that tiles matrices and convolutions into validated accelerator commands and packed stream payloads. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block.

Line 714: `"""Lower grouped binary/ternary spike convolution to addition-only GEMMs."""` This is executable documentation attached to the function compile_spike_conv2d, which handles compile spike conv2d. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 715: `spike_array = np.asarray(spikes)` This assigns ‘np.asarray(spikes)‘ to ‘spike_array ‘ in host memory. Here spike_array means a binary event produced when a neuron crosses its threshold. The important operands are asarray: the value named ’asarray’ in this block; spikes: a binary event produced when a neuron crosses its threshold. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 716: `weight_array = np.asarray(weights)` This assigns ‘np.asarray(weights)‘ to ‘weight_array ‘ in host memory. Here weight_array means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 717: `if spike_array.ndim != 4 or weight_array.ndim != 4:` This executes the following indented host-code branch only when ‘spike_array.ndim != 4 or weight_array.ndim != 4‘ is true. The important operands are spike_array: a binary event produced when a neuron crosses its threshold; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 718: `raise ValueError("spike convolution requires NCHW activations and OIHW weights")` This rejects an invalid state by raising ‘ValueError("spike convolution requires NCHW activations and OIHW weights")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 719: `if not np.issubdtype(spike_array.dtype, np.integer) or not np.issubdtype(weight_array.dtype, np.integer):` This executes the following indented host-code branch only when ‘not np.issubdtype(spike_array.dtype, np.integer) or not np.issubdtype(weight_array.dtype, np.integer)‘ is true. The important operands are not: the value named ’not’ in this block; issubdtype: the value named ’issubdtype’ in this block; spike_array: a binary event produced when a neuron crosses its threshold. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 720: `raise ValueError("spike convolution tensors must contain integers")` This rejects an invalid state by raising ‘ValueError("spike convolution tensors must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 721: `spike_i64 = spike_array.astype(np.int64)` This assigns ‘spike_array.astype(np.int64)‘ to ‘spike_i64 ‘ in host memory. Here spike_i64 means a binary event produced when a neuron crosses its threshold. The important operands are spike_array: a binary event produced when a neuron crosses its threshold; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 722: `weight_i64 = weight_array.astype(np.int64)` This assigns ‘weight_array.astype(np.int64)‘ to ‘weight_i64 ‘ in host memory. Here weight_i64 means a learned coefficient multiplied by an activation. The important operands are weight_array: a learned coefficient multiplied by an activation; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 723: `if not np.all(np.isin(spike_i64, (-1, 0, 1) if ternary else (0, 1))):` This executes the following indented host-code branch only when ‘not np.all(np.isin(spike_i64, (-1, 0, 1) if ternary else (0, 1)))‘ is true. The important operands are not: the value named ’not’ in this block; all: the value named ’all’ in this block; isin: the value named ’isin’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 724: `raise ValueError("spike values do not match the selected binary/ternary mode")` This rejects an invalid state by raising ‘ValueError("spike values do not match the selected binary/ternary mode")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 725: `if np.any(weight_i64 < -128) or np.any(weight_i64 > 127):` This executes the following indented host-code branch only when ‘np.any(weight_i64 < -128) or np.any(weight_i64 > 127)‘ is true. The important operands are any: the value named ’any’ in this block; weight_i64: a learned coefficient multiplied by an activation; or: the value named ’or’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 726: `raise ValueError("spike convolution weights must be signed INT8")` This rejects an invalid state by raising ‘ValueError("spike convolution weights must be signed INT8")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 727: `stride_pair = _pair("stride", stride)` This assigns ‘_pair("stride", stride)‘ to ‘stride_pair ‘ in host memory. Here stride_pair means the value named ’stride pair’ in this block. The important operands are _pair: the value named ’pair’ in this block; stride: the value named ’stride’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 728: `padding_pair = _pair("padding", padding, allow_zero=True)` This assigns ‘_pair("padding", padding, allow_zero=True)‘ to ‘padding_pair ‘ in host memory. Here padding_pair means the value named ’padding pair’ in this block. The important operands are _pair: the value named ’pair’ in this block; padding: the value named ’padding’ in this block; allow_zero: the value named ’allow zero’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 729: `dilation_pair = _pair("dilation", dilation)` This assigns ‘_pair("dilation", dilation)‘ to ‘dilation_pair ‘ in host memory. Here dilation_pair means the value named ’dilation pair’ in this block. The important operands are _pair: the value named ’pair’ in this block; dilation: the value named ’dilation’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 730: `batch, input_channels, height, width = spike_i64.shape` This assigns ‘spike_i64.shape‘ to ‘batch, input_channels, height, width ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are spike_i64: a binary event produced when a neuron crosses its threshold; shape: the value named ’shape’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 731: `output_channels, weight_channels, kernel_height, kernel_width = weight_i64.shape` This assigns ‘weight_i64.shape‘ to ‘output_channels, weight_channels, kernel_height, kernel_width ‘ in host memory. Here output_channels means the value named ’output channels’ in this block. The important operands are weight_i64: a learned coefficient multiplied by an activation; shape: the value named ’shape’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 732: `group_count = int(groups)` This assigns ‘int(groups)‘ to ‘group_count ‘ in host memory. Here group_count means an integer count retained for control or measurable telemetry. The important operands are groups: the value named ’groups’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 733: `if group_count <= 0 or input_channels % group_count or output_channels % group_count:` This executes the following indented host-code branch only when ‘group_count <= 0 or input_channels % group_count or output_channels % group_count‘ is true. The important operands are group_count: an integer count retained for control or measurable telemetry; or: the value named ’or’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 734: `raise ValueError("groups must divide both input and output channels")` This rejects an invalid state by raising ‘ValueError("groups must divide both input and output channels")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 735: `channels_per_group = input_channels // group_count` This assigns ‘input_channels // group_count‘ to ‘channels_per_group ‘ in host memory. Here channels_per_group means the value named ’channels per group’ in this block. The important operands are input_channels: the value named ’input channels’ in this block; group_count: an integer count retained for control or measurable telemetry. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 736: `outputs_per_group = output_channels // group_count` This assigns ‘output_channels // group_count‘ to ‘outputs_per_group ‘ in host memory. Here outputs_per_group means the value named ’outputs per group’ in this block. The important operands are output_channels: the value named ’output channels’ in this block; group_count: an integer count retained for control or measurable telemetry. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 737: `if weight_channels != channels_per_group:` This executes the following indented host-code branch only when ‘weight_channels != channels_per_group‘ is true. The important operands are weight_channels: a learned coefficient multiplied by an activation; channels_per_group: the value named ’channels per group’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 738: `raise ValueError("OIHW weight input channels must equal input_channels/groups")` This rejects an invalid state by raising ‘ValueError("OIHW weight input channels must equal input_channels/groups")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 739: `if bias is None:` This executes the following indented host-code branch only when ‘bias is None‘ is true. The important operands are bias: an additive learned offset applied after accumulation; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 740: `bias_values = None` This assigns ‘None‘ to ‘bias_values ‘ in host memory. Here bias_values means an additive learned offset applied after accumulation. The important operands are None: the value named ’none’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 741: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function compile_spike_conv2d, which handles compile spike conv2d.

Line 742: `bias_array = np.asarray(bias)` This assigns ‘np.asarray(bias)‘ to ‘bias_array ‘ in host memory. Here bias_array means an additive learned offset applied after accumulation. The important operands are asarray: the value named ’asarray’ in this block; bias: an additive learned offset applied after accumulation. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 743: `if bias_array.ndim != 1 or len(bias_array) != output_channels:` This executes the following indented host-code branch only when ‘bias_array.ndim != 1 or len(bias_array) != output_channels‘ is true. The important operands are bias_array: an additive learned offset applied after accumulation; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 744: `raise ValueError("spike convolution bias must have one entry per output channel")` This rejects an invalid state by raising ‘ValueError("spike convolution bias must have one entry per output channel")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 745: `if not np.issubdtype(bias_array.dtype, np.integer):` This executes the following indented host-code branch only when ‘not np.issubdtype(bias_array.dtype, np.integer)‘ is true. The important operands are not: the value named ’not’ in this block; issubdtype: the value named ’issubdtype’ in this block; bias_array: an additive learned offset applied after accumulation. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 746: `raise ValueError("spike convolution bias must contain integers")` This rejects an invalid state by raising ‘ValueError("spike convolution bias must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 747: `bias_values = bias_array.astype(np.int64)` This assigns ‘bias_array.astype(np.int64)‘ to ‘bias_values ‘ in host memory. Here bias_values means an additive learned offset applied after accumulation. The important operands are bias_array: an additive learned offset applied after accumulation; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 749: `programs: list[ConvGroupProgram] = []` This assigns ‘[]‘ to ‘programs‘ in host memory. Here programs means the value named ’programs’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 750: `output_height = output_width = 0` This assigns ‘output_width = 0‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are output_width: the value named ’output width’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 751: `for group in range(group_count):` This iterates group over ‘range(group_count)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_spike_conv2d, which handles compile spike conv2d.

Line 752: `input_start = group * channels_per_group` This assigns ‘group * channels_per_group‘ to ‘input_start ‘ in host memory. Here input_start means the value named ’input start’ in this block. The important operands are group: the value named ’group’ in this block; channels_per_group: the value named ’channels per group’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 753: `output_start = group * outputs_per_group` This assigns ‘group * outputs_per_group‘ to ‘output_start ‘ in host memory. Here output_start means the value named ’output start’ in this block. The important operands are group: the value named ’group’ in this block; outputs_per_group: the value named ’outputs per group’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 754: `columns, output_height, output_width = _im2col_nchw(` This statement continues through source line 758; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_im2col_nchw(spike_i64[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, 0,)‘ to ‘columns, output_height, output_width ‘ in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named ’im2col nchw’ in this block; spike_i64: a binary event produced when a neuron crosses its threshold; input_start: the value named ’input start’ in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 755: `spike_i64[:, input_start:input_start + channels_per_group],` This is continuation line 2 of the 5-line logical statement that begins at source line 754. Its exact fragment is ‘spike_i64[:, input_start:input_start + channels_per_group],‘. The important operands are spike_i64: a binary event produced when a neuron crosses its threshold; input_start: the value named ’input start’ in this block; channels_per_group: the value named ’channels per group’ in

this block. Read together, the complete operation is: This assigns '_im2col_nchw(spike_i64[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, 0,)' to 'columns, output_height, output_width ' in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named 'im2col nchw' in this block; spike_i64: a binary event produced when a neuron crosses its threshold; input_start: the value named 'input start' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 756: `(kernel_height, kernel_width), stride_pair, padding_pair,` This is continuation line 3 of the 5-line logical statement that begins at source line 754. Its exact fragment is '(kernel_height, kernel_width), stride_pair, padding_pair,'. The important operands are kernel_height: the value named 'kernel height' in this block; kernel_width: the value named 'kernel width' in this block; stride_pair: the value named 'stride pair' in this block. Read together, the complete operation is: This assigns '_im2col_nchw(spike_i64[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, 0,)' to 'columns, output_height, output_width ' in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named 'im2col nchw' in this block; spike_i64: a binary event produced when a neuron crosses its threshold; input_start: the value named 'input start' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 757: `dilation_pair, 0,` This is continuation line 4 of the 5-line logical statement that begins at source line 754. Its exact fragment is 'dilation_pair, 0,'. The important operands are dilation_pair: the value named 'dilation pair' in this block. Read together, the complete operation is: This assigns '_im2col_nchw(spike_i64[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, 0,)' to 'columns, output_height, output_width ' in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named 'im2col nchw' in this block; spike_i64: a binary event produced when a neuron crosses its threshold; input_start: the value named 'input start' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 758: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 754. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_im2col_nchw(spike_i64[:, input_start:input_start + channels_per_group], (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, 0,)' to 'columns, output_height, output_width ' in host memory. Here columns means one output-feature position processed in parallel. The important operands are _im2col_nchw: the value named 'im2col nchw' in this block; spike_i64: a binary event produced when a neuron crosses its threshold; input_start: the value named 'input start' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 759: `weight_matrix = weight_i64[` This statement continues through source line 761; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'weight_i64[output_start:output_start + outputs_per_group].reshape(outputs_per_group, -1).T' to 'weight_matrix ' in host memory. Here weight_matrix means a learned coefficient multiplied by an activation. The important operands are weight_i64: a learned coefficient multiplied by an activation; output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 760: `output_start:output_start + outputs_per_group` This is continuation line 2 of the 3-line logical statement that begins at source line 759. Its exact fragment is 'output_start:output_start + outputs_per_group'. The important operands are output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Read together, the complete operation is: This assigns 'weight_i64[output_start:output_start + outputs_per_group].reshape(outputs_per_group, -1).T' to 'weight_matrix ' in host memory. Here weight_matrix means a learned coefficient multiplied by an activation. The important operands are weight_i64: a learned coefficient multiplied by an activation; output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 761: `].reshape(outputs_per_group, -1).T` This is continuation line 3 of the 3-line logical statement that begins at source line 759. Its exact fragment is '].reshape(outputs_per_group, -1).T'. The important operands are

reshape: the value named 'reshape' in this block; outputs_per_group: the value named 'outputs per group' in this block; T: the value named 't' in this block. Read together, the complete operation is: This assigns ‘weight_i64[output_start:output_start + outputs_per_group].reshape(outputs_per_group, -1).T‘ to ‘weight_matrix ‘ in host memory. Here weight_matrix means a learned coefficient multiplied by an activation. The important operands are weight_i64: a learned coefficient multiplied by an activation; output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 762: `group_bias = None if bias_values is None else bias_values[` This statement continues through source line 764; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘None if bias_values is None else bias_values[output_start:output_start + outputs_per_group]‘ to ‘group_bias ‘ in host memory. Here group_bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block; bias_values: an additive learned offset applied after accumulation; is: the value named 'is' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 763: `output_start:output_start + outputs_per_group` This is continuation line 2 of the 3-line logical statement that begins at source line 762. Its exact fragment is ‘output_start:output_start + outputs_per_group‘. The important operands are output_start: the value named 'output start' in this block; outputs_per_group: the value named 'outputs per group' in this block. Read together, the complete operation is: This assigns ‘None if bias_values is None else bias_values[output_start:output_start + outputs_per_group]‘ to ‘group_bias ‘ in host memory. Here group_bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block; bias_values: an additive learned offset applied after accumulation; is: the value named 'is' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 764: `]` This is continuation line 3 of the 3-line logical statement that begins at source line 762. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘None if bias_values is None else bias_values[output_start:output_start + outputs_per_group]‘ to ‘group_bias ‘ in host memory. Here group_bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block; bias_values: an additive learned offset applied after accumulation; is: the value named 'is' in this block. Context: the function compile_spike_conv2d, which handles compile spike conv2d.

Line 765: `programs.append(ConvGroupProgram(` This statement continues through source line 775; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘programs.append‘ with ‘ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)‘ inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 766: `output_start,` This is continuation line 2 of the 11-line logical statement that begins at source line 765. Its exact fragment is ‘output_start,‘. The important operands are output_start: the value named 'output start' in this block. Read together, the complete operation is: This calls ‘programs.append‘ with ‘ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)‘ inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 767: `outputs_per_group,` This is continuation line 3 of the 11-line logical statement that begins at source line 765. Its exact fragment is ‘outputs_per_group,‘. The important operands are outputs_per_group: the value named 'outputs per group' in this block. Read together, the complete operation is: This calls ‘programs.append‘ with ‘ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)‘ inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 768: `compile_spike_gemm(` This is continuation line 4 of the 11-line logical statement that begins at source line 765. Its exact fragment is 'compile_spike_gemm('. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)' inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 769: `columns,` This is continuation line 5 of the 11-line logical statement that begins at source line 765. Its exact fragment is 'columns,'. The important operands are columns: one output-feature position processed in parallel. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)' inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 770: `weight_matrix,` This is continuation line 6 of the 11-line logical statement that begins at source line 765. Its exact fragment is 'weight_matrix,'. The important operands are weight_matrix: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)' inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 771: `ternary=ternary,` This is continuation line 7 of the 11-line logical statement that begins at source line 765. Its exact fragment is 'ternary=ternary,'. The important operands are ternary: the value named 'ternary' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)' inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 772: `bias=group_bias,` This is continuation line 8 of the 11-line logical statement that begins at source line 765. Its exact fragment is 'bias=group_bias,'. The important operands are bias: an additive learned offset applied after accumulation; group_bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)' inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 773: `dataflow=dataflow,` This is continuation line 9 of the 11-line logical statement that begins at source line 765. Its exact fragment is 'dataflow=dataflow,'. The important operands are dataflow: the value named 'dataflow' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)' inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 774: `),` This is continuation line 10 of the 11-line logical statement that begins at source line 765. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'programs.append' with 'ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)' inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 775: `))` This is continuation line 11 of the 11-line logical statement that begins at source line 765. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘programs.append‘ with ‘ConvGroupProgram(output_start, outputs_per_group, compile_spike_gemm(columns, weight_matrix, ternary=ternary, bias=group_bias, dataflow=dataflow,),)‘ inside the function compile_spike_conv2d, which handles compile spike conv2d. The call advances the host-side job that tiles matrices and convolutions into validated accelerator commands and packed stream payloads.

Line 776: `return CompiledConv2d(` This statement continues through source line 781; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_spike_conv2d, which handles compile spike conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

Line 777: `(batch, input_channels, height, width),` This is continuation line 2 of the 6-line logical statement that begins at source line 776. Its exact fragment is ‘(batch, input_channels, height, width),‘. The important operands are batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block; height: the value named ’height’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_spike_conv2d, which handles compile spike conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

Line 778: `(batch, output_channels, output_height, output_width),` This is continuation line 3 of the 6-line logical statement that begins at source line 776. Its exact fragment is ‘(batch, output_channels, output_height, output_width),‘. The important operands are batch: the value named ’batch’ in this block; output_channels: the value named ’output channels’ in this block; output_height: the value named ’output height’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_spike_conv2d, which handles compile spike conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

Line 779: `(kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair,` This is continuation line 4 of the 6-line logical statement that begins at source line 776. Its exact fragment is ‘(kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair,‘. The important operands are kernel_height: the value named ’kernel height’ in this block; kernel_width: the value named ’kernel width’ in this block; stride_pair: the value named ’stride pair’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_spike_conv2d, which handles compile spike conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

Line 780: `group_count, tuple(programs),` This is continuation line 5 of the 6-line logical statement that begins at source line 776. Its exact fragment is ‘group_count, tuple(programs),‘. The important operands are group_count: an integer count retained for control or measurable telemetry; tuple: the value named ’tuple’ in this block; programs: the value named ’programs’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_spike_conv2d, which handles compile spike conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

Line 781:) This is continuation line 6 of the 6-line logical statement that begins at source line 776. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘CompiledConv2d((batch, input_channels, height, width), (batch, output_channels, output_height, output_width), (kernel_height, kernel_width), stride_pair, padding_pair, dilation_pair, group_count, tuple(programs),)‘ to the caller of the function compile_spike_conv2d, which handles compile spike conv2d, ending this invocation. The important operands are CompiledConv2d: the value named ’compiled conv2d’ in this block; batch: the value named ’batch’ in this block; input_channels: the value named ’input channels’ in this block.

35 software/neuraedge_golden.py

Role in the complete system: implements independent mathematical reference models for every hardware arithmetic primitive.

35.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to implements independent mathematical reference models for every hardware arithmetic primitive.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: Dataflow, ComputeMode, VectorOperation, DataflowDecision. Functions: encode_int8_as_uint8, decode_uint8_as_int8, packed_mac2_u8, gemm_u8_zero_point, direct_centered_gemm, active_outer_indices, active_dot8_indices, binary_weighted_gemm, ternary_weighted_gemm, alpha_xnor_similarity, log_int_softmax_q15, integer_layer_norm, lif_step, ternary_spike, saturate_signed, round_shift_signed, linear_backward_tile, quantize_flexible_scale, sgd_momentum_step, vector_alu, choose_dataflow, peak_dense_gops. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

35.2 Complete source

```
"""Independent integer reference model for the NeuraEdge universal accelerator."""

from __future__ import annotations

from dataclasses import dataclass
from enum import IntEnum
from math import ceil, frexp, isqrt

import numpy as np


class Dataflow(IntEnum):
    INPUT_BROADCAST = 0
    WEIGHT_BROADCAST = 1


class ComputeMode(IntEnum):
    DENSE_OUTER = 0
    DENSE_DOT8 = 1
    BINARY_WEIGHTED = 2
    TERNARY_WEIGHTED = 3


class VectorOperation(IntEnum):
    ADD_SAT = 0
    RELU = 1
    REQUANTIZE = 2
    BINARY_SPIKE = 3
    TERNARY_SPIKE = 4
    SURROGATE_GRADIENT = 5
    MAXIMUM = 6
    MULTIPLY_Q15 = 7
    CLAMP = 8
    FLEX_REQUANTIZE = 9
    SELECT_ZERO = 10
    SELECT_NONZERO = 11
    COMPARE_GE = 12


@dataclass(frozen=True)
class DataflowDecision:
    mode: Dataflow
    input_broadcast_bytes: int
    weight_broadcast_bytes: int


```

```
def encode_int8_as_uint8(values: np.ndarray) -> np.ndarray:
    """Map signed INT8 bit patterns to the accelerator's unsigned arithmetic domain."""
    signed = np.asarray(values, dtype=np.int16)
    if np.any(signed < -128) or np.any(signed > 127):
        raise ValueError("signed INT8 values must be in [-128, 127]")
    return (signed + 128).astype(np.uint8)


def decode_uint8_as_int8(values: np.ndarray) -> np.ndarray:
    """Undo encode_int8_as_uint8 without relying on implementation-defined casts."""
    unsigned = np.asarray(values, dtype=np.int16)
    if np.any(unsigned < 0) or np.any(unsigned > 255):
        raise ValueError("encoded values must be in [0, 255]")
    return (unsigned - 128).astype(np.int16)


def packed_mac2_u8(activation: int, weight_lo: int, weight_hi: int) -> tuple[int, int]:
    """Bit-accurate model of neuraedge_mac2_u8, including its guard-bit packing."""
    for name, value in (("activation", activation), ("weight_lo", weight_lo), ("weight_hi", weight_hi)):
        if not 0 <= value <= 255:
            raise ValueError(f"{name} must be an unsigned byte")
    packed_weight = weight_lo | (weight_hi << 17)
    packed_product = packed_weight * activation
    return packed_product & 0xFFFF, (packed_product >> 17) & 0xFFFF


def gemm_u8_zero_point(
    activations: np.ndarray,
    weights: np.ndarray,
    activation_zero_point: int,
    weight_zero_point: int,
    bias: np.ndarray | None = None,
) -> np.ndarray:
    """Compute (A-Za)(B-Zb)+bias using the accelerator's raw-sum correction."""
    a = np.asarray(activations, dtype=np.int64)
    b = np.asarray(weights, dtype=np.int64)
    if a.ndim != 2 or b.ndim != 2 or a.shape[1] != b.shape[0]:
        raise ValueError("expected A[M,K] and B[K,N]")
    if np.any(a < 0) or np.any(a > 255) or np.any(b < 0) or np.any(b > 255):
        raise ValueError("accelerator GEMM operands must be unsigned bytes")
    if not 0 <= activation_zero_point <= 255 or not 0 <= weight_zero_point <= 255:
        raise ValueError("zero points must be unsigned bytes")

    reduction = a.shape[1]
    raw = a @ b
    row_sum = a.sum(axis=1, keepdims=True)
    column_sum = b.sum(axis=0, keepdims=True)
    corrected = (
        raw
        - weight_zero_point * row_sum
        - activation_zero_point * column_sum
        + reduction * activation_zero_point * weight_zero_point
    )
    if bias is not None:
        bias_array = np.asarray(bias, dtype=np.int64).reshape(1, -1)
        if bias_array.shape[1] != b.shape[1]:
            raise ValueError("bias length must equal output columns")
        corrected = corrected + bias_array
    return corrected


def direct_centered_gemm(
    activations: np.ndarray,
    weights: np.ndarray,
    activation_zero_point: int,
    weight_zero_point: int,
    bias: np.ndarray | None = None,
) -> np.ndarray:
    """Simple centered expression used only as an independent algebra oracle."""
    a = np.asarray(activations, dtype=np.int64) - activation_zero_point
    b = np.asarray(weights, dtype=np.int64) - weight_zero_point
    result = a @ b
    if bias is not None:
        result = result + np.asarray(bias, dtype=np.int64).reshape(1, -1)
    return result


def active_outer_indices(activations: np.ndarray, zero_point: int) -> np.ndarray:
    """Return K indices where at least one row contains a non-zero activation."""
    a = np.asarray(activations)
    if a.ndim != 2:
        raise ValueError("outer-mode activations must have shape [M,K]")
    return np.flatnonzero(np.any(a != zero_point, axis=0)).astype(np.int64)


def active_dot8_indices(activation_row: np.ndarray, zero_point: int) -> np.ndarray:
    """Return eight-element K chunks containing at least one non-zero activation."""
```

```
    row = np.asarray(activation_row).reshape(-1)
    padded_length = ceil(row.size / 8) * 8
    padded = np.full(padded_length, zero_point, dtype=row.dtype)
    padded[: row.size] = row
    chunks = padded.reshape(-1, 8)
    return np.flatnonzero(np.any(chunks != zero_point, axis=1)).astype(np.int64)


def binary_weighted_gemm(spikes: np.ndarray, weights: np.ndarray) -> np.ndarray:
    """Addition-only matrix product for binary activations in {0,1}."""
    x = np.asarray(spikes, dtype=np.int64)
    w = np.asarray(weights, dtype=np.int64)
    if np.any((x != 0) & (x != 1)):
        raise ValueError("binary activations must contain only 0 and 1")
    return x @ w


def ternary_weighted_gemm(spikes: np.ndarray, weights: np.ndarray) -> np.ndarray:
    """Addition/subtraction-only matrix product for activations in {-1,0,1}."""
    x = np.asarray(spikes, dtype=np.int64)
    w = np.asarray(weights, dtype=np.int64)
    if np.any((x < -1) | (x > 1)):
        raise ValueError("ternary activations must be in {-1,0,1}")
    return x @ w


def alpha_xnor_similarity(query: np.ndarray, key: np.ndarray, alpha_q8: int) -> np.ndarray:
    """Equation 18 of Xiao et al. CVPR 2025, represented in unsigned Q8."""
    q = np.asarray(query, dtype=np.uint8)
    k = np.asarray(key, dtype=np.uint8)
    if q.ndim != 2 or k.ndim != 2 or q.shape[1] != k.shape[1]:
        raise ValueError("expected query[M,D] and key[N,D]")
    if np.any(q > 1) or np.any(k > 1):
        raise ValueError("alpha-XNOR inputs must be binary")
    if not 0 <= alpha_q8 <= 256:
        raise ValueError("alpha_q8 must represent a value in [0,1]")
    spike_matches = q.astype(np.int64) @ k.astype(np.int64).T
    zero_matches = (1 - q.astype(np.int64)) @ (1 - k.astype(np.int64)).T
    return spike_matches * 256 + zero_matches * alpha_q8


def log_int_softmax_q15(scores: np.ndarray, difference_shift: int) -> np.ndarray:
    """Power-of-two softmax with the Q31 reciprocal used by the RTL datapath."""
    x = np.asarray(scores, dtype=np.int64)
    if x.ndim != 1 or x.size == 0:
        raise ValueError("scores must be a non-empty vector")
    if not 0 <= difference_shift <= 31:
        raise ValueError("difference_shift must be in [0,31]")
    differences = x.max() - x
    exponents = np.minimum(differences >> difference_shift, 15).astype(np.int64)
    unnormalized = (1 << (15 - exponents)).astype(np.int64)
    denominator = int(unnormalized.sum())
    reciprocal_q31 = ((1 << 31) - 1) // denominator
    return round_shift_signed(unnormalized * reciprocal_q31, 16)


def integer_layer_norm(
    values: np.ndarray,
    gamma_q15: np.ndarray,
    beta: np.ndarray,
    epsilon: int = 1,
) -> np.ndarray:
    """I-ViT-style integer LayerNorm using sqrt and one shared Q31 reciprocal."""
    x = np.asarray(values, dtype=np.int64).reshape(-1)
    gamma = np.asarray(gamma_q15, dtype=np.int64).reshape(-1)
    offset = np.asarray(beta, dtype=np.int64).reshape(-1)
    if x.size == 0 or gamma.size != x.size or offset.size != x.size:
        raise ValueError("values, gamma, and beta must be equally sized non-empty vectors")
    mean = int(x.sum() // x.size)
    centered = x - mean
    variance = int((centered * centered).sum() // x.size)
    sigma = max(1, isqrt(variance + epsilon))
    reciprocal_q31 = ((1 << 31) - 1) // sigma
    normalized = round_shift_signed(centered * reciprocal_q31, 31)
    return round_shift_signed(normalized * gamma, 15) + offset


def lif_step(
    membrane: np.ndarray,
    current: np.ndarray,
    leak_shift: int,
    threshold: int,
    reset_value: int = 0,
) -> tuple[np.ndarray, np.ndarray]:
    """One deterministic leaky-integrate-and-fire step."""
    v = np.asarray(membrane, dtype=np.int64)
    i = np.asarray(current, dtype=np.int64)
```

```
    integrated = v - (v >> leak_shift) + i
    spikes = integrated >= threshold
    next_membrane = np.where(spikes, reset_value, integrated)
    return next_membrane.astype(np.int64), spikes.astype(np.uint8)


def ternary_spike(values: np.ndarray, threshold: int) -> np.ndarray:
    """Symmetric ternary firing rule used by the addition-only datapath."""
    x = np.asarray(values, dtype=np.int64)
    return np.where(x >= threshold, 1, np.where(x <= -threshold, -1, 0)).astype(np.int8)


def saturate_signed(values: np.ndarray | int, bits: int) -> np.ndarray:
    """Clamp integers to the exact range of a signed two's-complement field."""
    if bits < 2 or bits > 63:
        raise ValueError("bits must be in [2,63]")
    array = np.asarray(values, dtype=np.int64)
    minimum = -(1 << (bits - 1))
    maximum = (1 << (bits - 1)) - 1
    return np.clip(array, minimum, maximum).astype(np.int64)


def round_shift_signed(values: np.ndarray | int, shift: int) -> np.ndarray:
    """Round signed integers to nearest, with exact half cases away from zero."""
    if not 0 <= shift <= 62:
        raise ValueError("shift must be in [0,62]")
    array = np.asarray(values, dtype=np.int64)
    if shift == 0:
        return array.copy()
    magnitude = np.abs(array)
    rounded = (magnitude + (1 << (shift - 1))) >> shift
    return np.where(array < 0, -rounded, rounded).astype(np.int64)


def linear_backward_tile(
    activations: np.ndarray,
    weights: np.ndarray,
    output_gradients: np.ndarray,
    *,
    input_gradient_shift: int = 0,
    weight_gradient_shift: int = 0,
) -> tuple[np.ndarray, np.ndarray]:
    """Bit-exact reference for the signed RTL linear backward tile."""

    x = np.asarray(activations, dtype=np.int64)
    w = np.asarray(weights, dtype=np.int64)
    dy = np.asarray(output_gradients, dtype=np.int64)
    if x.ndim != 2 or w.ndim != 2 or dy.ndim != 2:
        raise ValueError("linear backward operands must be two-dimensional")
    rows, input_features = x.shape
    weight_inputs, output_features = w.shape
    if weight_inputs != input_features or dy.shape != (rows, output_features):
        raise ValueError("linear backward matrix dimensions do not align")
    if not 1 <= rows <= 8 or not 1 <= input_features <= 8 or not 1 <= output_features <= 8:
        raise ValueError("linear backward tile dimensions must be in [1,8]")
    if np.any(x < -(1 << 15)) or np.any(x > (1 << 15) - 1):
        raise ValueError("linear backward activations must fit signed 16-bit")
    if np.any(w < -(1 << 15)) or np.any(w > (1 << 15) - 1):
        raise ValueError("linear backward weights must fit signed 16-bit")
    if np.any(dy < -(1 << 15)) or np.any(dy > (1 << 15) - 1):
        raise ValueError("linear backward output gradients must fit signed 16-bit")
    if not 0 <= input_gradient_shift <= 31 or not 0 <= weight_gradient_shift <= 31:
        raise ValueError("linear backward shifts must be in [0,31]")

    input_gradient = round_shift_signed(dy @ w.T, input_gradient_shift)
    weight_gradient = round_shift_signed(x.T @ dy, weight_gradient_shift)
    return saturate_signed(input_gradient, 32), saturate_signed(weight_gradient, 24)


def quantize_flexible_scale(scale: float) -> tuple[int, int, float]:
    """Encode a signed scale as Q15 mantissa and signed binary-point shift.

    The flexible vector operation evaluates ``x * multiplier / 2**(15+shift)``.
    Its signed five-bit shift covers [-15, 15], giving useful non-zero scale
    magnitudes from approximately 2^-16 through 2^15.
    """

    if not np.isfinite(scale):
        raise ValueError("scale must be finite")
    if scale == 0.0:
        return 0, 0, 0.0
    mantissa, exponent = frexp(scale)
    multiplier = int(round(mantissa * (1 << 15)))
    if multiplier == (1 << 15):
        multiplier //= 2
        exponent += 1
    if multiplier < -(1 << 15) or multiplier > (1 << 15) - 1:
```

```
        raise ValueError("scale mantissa does not fit signed Q15")
    shift = -exponent
    if not -15 <= shift <= 15:
        raise ValueError("scale is outside the flexible requantizer range")
    represented = multiplier / float(1 << (15 + shift))
    return multiplier, shift, represented


def sgd_momentum_step(
    weights: np.ndarray,
    gradients: np.ndarray,
    velocities: np.ndarray,
    learning_rate_q16: int,
    momentum_q16: int,
    weight_decay_q16: int,
    update_right_shift: int = 16,
) -> tuple[np.ndarray, np.ndarray]:
    """Bit-exact SGDW update used by the fabric optimizer lanes."""
    w = np.asarray(weights, dtype=np.int64)
    g = np.asarray(gradients, dtype=np.int64)
    v = np.asarray(velocities, dtype=np.int64)
    if w.shape != g.shape or w.shape != v.shape:
        raise ValueError("weights, gradients, and velocities must have equal shapes")
    if not 0 <= learning_rate_q16 <= 131071:
        raise ValueError("learning_rate_q16 must be in [0,131071]")
    if not 0 <= momentum_q16 <= 65536 or not 0 <= weight_decay_q16 <= 65536:
        raise ValueError("momentum and weight decay must be unsigned Q16 values")
    if not 0 <= update_right_shift <= 31:
        raise ValueError("update_right_shift must be in [0,31]")
    momentum_term = round_shift_signed(v * momentum_q16, 16)
    decay_term = round_shift_signed(w * weight_decay_q16, 16)
    next_velocity = saturate_signed(momentum_term + g, 24)
    update_direction = saturate_signed(next_velocity + decay_term, 24)
    update = round_shift_signed(
        update_direction * learning_rate_q16, update_right_shift)
    next_weight = saturate_signed(w - update, 16)
    return next_weight, next_velocity


def vector_alu(
    operation: VectorOperation,
    operand_a: np.ndarray,
    operand_b: np.ndarray,
    *,
    multiplier_q15: int = 0,
    right_shift: int = 0,
    output_zero_point: int = 0,
    threshold: int = 0,
    surrogate_window: int = 0,
    clamp_minimum: int = -(1 << 31),
    clamp_maximum: int = (1 << 31) - 1,
) -> np.ndarray:
    """Reference for every operation in the eight-lane integer vector ALU."""
    a = np.asarray(operand_a, dtype=np.int64)
    b = np.asarray(operand_b, dtype=np.int64)
    if a.shape != b.shape:
        raise ValueError("operand vectors must have equal shapes")
    if operation == VectorOperation.FLEX_REQUANTIZE:
        if not -15 <= right_shift <= 15:
            raise ValueError("flexible binary-point shift must be in [-15,15]")
    elif not 0 <= right_shift <= 31:
        raise ValueError("right_shift must be in [0,31]")
    if clamp_minimum > clamp_maximum:
        raise ValueError("clamp_minimum must not exceed clamp_maximum")

    if operation == VectorOperation.ADD_SAT:
        output = saturate_signed(a + b, 32)
    elif operation == VectorOperation.RELU:
        output = np.maximum(a, 0)
    elif operation == VectorOperation.REQUANTIZE:
        output = round_shift_signed(a * multiplier_q15, 15 + right_shift)
        output = np.clip(output + output_zero_point, clamp_minimum, clamp_maximum)
    elif operation == VectorOperation.FLEX_REQUANTIZE:
        output = round_shift_signed(a * multiplier_q15, 15 + right_shift)
        output = np.clip(output + output_zero_point, clamp_minimum, clamp_maximum)
    elif operation == VectorOperation.BINARY_SPIKE:
        output = (a >= threshold).astype(np.int64)
    elif operation == VectorOperation.TERNARY_SPIKE:
        output = np.where(a >= threshold, 1, np.where(a <= -threshold, -1, 0))
    elif operation == VectorOperation.SURROGATE_GRADIENT:
        output = np.where(np.abs(a - threshold) <= surrogate_window, b, 0)
    elif operation == VectorOperation.MAXIMUM:
        output = np.maximum(a, b)
    elif operation == VectorOperation.MULTIPLY_Q15:
        output = saturate_signed(round_shift_signed(a * b, 15), 32)
    elif operation == VectorOperation.CLAMP:
        output = np.clip(a, clamp_minimum, clamp_maximum)
```

```
    elif operation == VectorOperation.SELECT_ZERO:
        output = np.where(a == 0, b, 0)
    elif operation == VectorOperation.SELECT_NONZERO:
        output = np.where(a != 0, b, 0)
    elif operation == VectorOperation.COMPARE_GE:
        output = (a >= b).astype(np.int64)
    else:
        raise ValueError(f"unsupported vector operation {operation}")
    return np.asarray(output, dtype=np.int64)


def choose_dataflow(
    rows: int,
    columns: int,
    reduction: int,
    row_tile: int = 8,
    column_tile: int = 16,
) -> DataflowDecision:
    """Choose the traversal with fewer estimated operand bytes."""
    if min(rows, columns, reduction, row_tile, column_tile) <= 0:
        raise ValueError("matrix and tile dimensions must be positive")
    row_tiles = ceil(rows / row_tile)
    column_tiles = ceil(columns / column_tile)
    input_tile_bytes = row_tile * reduction
    weight_tile_bytes = column_tile * reduction
    input_broadcast_bytes = row_tiles * input_tile_bytes + row_tiles * column_tiles * weight_tile_bytes
    weight_broadcast_bytes = column_tiles * weight_tile_bytes + column_tiles * row_tiles * input_tile_bytes
    mode = (
        Dataflow.INPUT_BROADCAST
        if input_broadcast_bytes <= weight_broadcast_bytes
        else Dataflow.WEIGHT_BROADCAST
    )
    return DataflowDecision(mode, input_broadcast_bytes, weight_broadcast_bytes)


def peak_dense_gops(clock_hz: float, packed_dsp_pairs: int) -> float:
    """Two operations per MAC and two MACs per packed DSP pair."""
    return clock_hz * packed_dsp_pairs * 2 * 2 / 1e9
```

35.3 Line annotations

Line 1: `"""Independent integer reference model for the NeuraEdge universal accelerator."""` This is executable documentation attached to the file whose role is to implements independent mathematical reference models for every hardware arithmetic primitive. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to implements independent mathematical reference models for every hardware arithmetic primitive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to implements independent mathematical reference models for every hardware arithmetic primitive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `from enum import IntEnum` This imports the named Python API so the file whose role is to implements independent mathematical reference models for every hardware arithmetic primitive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from math import ceil, frexp, isqrt` This imports the named Python API so the file whose role is to implements independent mathematical reference models for every hardware arithmetic primitive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import numpy as np` This imports the named Python API so the file whose role is to implements independent mathematical reference models for every hardware arithmetic primitive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `class Dataflow(IntEnum):` This defines class Dataflow, which groups data and methods for dataflow. Instances exist in host memory and organize the flow that implements independent mathematical reference models for every hardware arithmetic primitive.

Line 13: `INPUT_BROADCAST = 0` This assigns ‘0‘ to ‘INPUT_BROADCAST ‘ in host memory. Here INPUT_BROADCAST means the value named ’input broadcast’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Dataflow, which handles dataflow.

Line 14: `WEIGHT_BROADCAST = 1` This assigns ‘1‘ to ‘WEIGHT_BROADCAST ‘ in host memory. Here WEIGHT_BROADCAST means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the class Dataflow, which handles dataflow.

Line 17: `class ComputeMode(IntEnum):` This defines class ComputeMode, which groups data and methods for compute mode. Instances exist in host memory and organize the flow that implements independent mathematical reference models for every hardware arithmetic primitive.

Line 18: `DENSE_OUTER = 0` This assigns ‘0‘ to ‘DENSE_OUTER ‘ in host memory. Here DENSE_OUTER means the value named ’dense outer’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class ComputeMode, which handles compute mode.

Line 19: `DENSE_DOT8 = 1` This assigns ‘1‘ to ‘DENSE_DOT8 ‘ in host memory. Here DENSE_DOT8 means the value named ’dense dot8’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class ComputeMode, which handles compute mode.

Line 20: `BINARY_WEIGHTED = 2` This assigns ‘2‘ to ‘BINARY_WEIGHTED ‘ in host memory. Here BINARY_WEIGHTED means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the class ComputeMode, which handles compute mode.

Line 21: `TERNARY_WEIGHTED = 3` This assigns ‘3‘ to ‘TERNARY_WEIGHTED ‘ in host memory. Here TERNARY_WEIGHTED means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the class ComputeMode, which handles compute mode.

Line 24: `class VectorOperation(IntEnum):` This defines class VectorOperation, which groups data and methods for vector operation. Instances exist in host memory and organize the flow that implements independent mathematical reference models for every hardware arithmetic primitive.

Line 25: `ADD_SAT = 0` This assigns ‘0‘ to ‘ADD_SAT ‘ in host memory. Here ADD_SAT means the value named ’add sat’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 26: `RELU = 1` This assigns ‘1‘ to ‘RELU ‘ in host memory. Here RELU means the value named ’relu’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 27: `REQUANTIZE = 2` This assigns ‘2‘ to ‘REQUANTIZE ‘ in host memory. Here REQUANTIZE means the conversion of a wide accumulated result back into a bounded activation code. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 28: `BINARY_SPIKE = 3` This assigns ‘3‘ to ‘BINARY_SPIKE ‘ in host memory. Here BINARY_SPIKE means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 29: `TERNARY_SPIKE = 4` This assigns ‘4‘ to ‘TERNARY_SPIKE ‘ in host memory. Here TERNARY_SPIKE means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 30: `SURROGATE_GRADIENT = 5` This assigns ‘5‘ to ‘SURROGATE_GRADIENT ‘ in host memory. Here SURROGATE_GRADIENT means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 31: `MAXIMUM = 6` This assigns ‘6‘ to ‘MAXIMUM ‘ in host memory. Here MAXIMUM means the value named ’maximum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 32: `MULTIPLY_Q15 = 7` This assigns ‘7‘ to ‘MULTIPLY_Q15 ‘ in host memory. Here MULTIPLY_Q15 means the value named ’multiply q15’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 33: `CLAMP = 8` This assigns ‘8‘ to ‘CLAMP ‘ in host memory. Here CLAMP means the value named ’clamp’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 34: `FLEX_REQUANTIZE = 9` This assigns ‘9‘ to ‘FLEX_REQUANTIZE ‘ in host memory. Here FLEX_REQUANTIZE means the conversion of a wide accumulated result back into a bounded activation code. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 35: `SELECT_ZERO = 10` This assigns ‘10‘ to ‘SELECT_ZERO ‘ in host memory. Here SELECT_ZERO means the value named ’select zero’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 36: `SELECT_NONZERO = 11` This assigns ‘11‘ to ‘SELECT_NONZERO ‘ in host memory. Here SELECT_NONZERO means the value named ’select nonzero’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 37: `COMPARE_GE = 12` This assigns ‘12‘ to ‘COMPARE_GE ‘ in host memory. Here COMPARE_GE means the value named ’compare ge’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class VectorOperation, which handles vector operation.

Line 40: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to implements independent mathematical reference models for every hardware arithmetic primitive; it creates no FPGA circuit by itself.

Line 41: `class DataflowDecision:` This defines class DataflowDecision, which groups data and methods for dataflow decision. Instances exist in host memory and organize the flow that implements independent mathematical reference models for every hardware arithmetic primitive.

Line 42: `mode: Dataflow` Inside the class DataflowDecision, which handles dataflow decision, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are mode: the value named ’mode’ in this block; Dataflow: the value named ’dataflow’ in this block.

Line 43: `input_broadcast_bytes: int` Inside the class DataflowDecision, which handles dataflow decision, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are input_broadcast_bytes: the value named 'input broadcast bytes' in this block.

Line 44: `weight_broadcast_bytes: int` Inside the class DataflowDecision, which handles dataflow decision, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are weight_broadcast_bytes: a learned coefficient multiplied by an activation.

Line 47: `def encode_int8_as_uint8(values: np.ndarray) -> np.ndarray:` This starts function encode_int8_as_uint8. Its indented body performs encode int8 as uint8 as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 48: `"""Map signed INT8 bit patterns to the accelerator's unsigned arithmetic domain."""` This is executable documentation attached to the function encode_int8_as_uint8, which handles encode int8 as uint8. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 49: `signed = np.asarray(values, dtype=np.int16)` This assigns 'np.asarray(values, dtype=np.int16)' to 'signed ' in host memory. Here signed means the value named 'signed' in this block. The important operands are asarray: the value named 'asarray' in this block; values: the value named 'values' in this block; dtype: the value named 'dtype' in this block. Context: the function encode_int8_as_uint8, which handles encode int8 as uint8.

Line 50: `if np.any(signed < -128) or np.any(signed > 127):` This executes the following indented host-code branch only when 'np.any(signed < -128) or np.any(signed > 127)' is true. The important operands are any: the value named 'any' in this block; or: the value named 'or' in this block. Context: the function encode_int8_as_uint8, which handles encode int8 as uint8.

Line 51: `raise ValueError("signed INT8 values must be in [-128, 127]")` This rejects an invalid state by raising 'ValueError("signed INT8 values must be in [-128, 127]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 52: `return (signed + 128).astype(np.uint8)` This returns '(signed + 128).astype(np.uint8)' to the caller of the function encode_int8_as_uint8, which handles encode int8 as uint8, ending this invocation. The important operands are astype: the value named 'astype' in this block; uint8: the value named 'uint8' in this block.

Line 55: `def decode_uint8_as_int8(values: np.ndarray) -> np.ndarray:` This starts function decode_uint8_as_int8. Its indented body performs decode uint8 as int8 as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 56: `"""Undo encode_int8_as_uint8 without relying on implementation-defined casts."""` This is executable documentation attached to the function decode_uint8_as_int8, which handles decode uint8 as int8. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 57: `unsigned = np.asarray(values, dtype=np.int16)` This assigns 'np.asarray(values, dtype=np.int16)' to 'unsigned ' in host memory. Here unsigned means the value named 'unsigned' in this block. The important operands are asarray: the value named 'asarray' in this block; values: the value named 'values' in this block; dtype: the value named 'dtype' in this block. Context: the function decode_uint8_as_int8, which handles decode uint8 as int8.

Line 58: `if np.any(unsigned < 0) or np.any(unsigned > 255):` This executes the following indented host-code branch only when 'np.any(unsigned < 0) or np.any(unsigned > 255)' is true. The important operands are any: the value named 'any' in this block; or: the value named 'or' in this block. Context: the function decode_uint8_as_int8, which handles decode uint8 as int8.

Line 59: `raise ValueError("encoded values must be in [0, 255]")` This rejects an invalid state by raising ‘ValueError("encoded values must be in [0, 255]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 60: `return (unsigned - 128).astype(np.int16)` This returns ‘(unsigned - 128).astype(np.int16)‘ to the caller of the function decode_uint8_as_int8, which handles decode uint8 as int8, ending this invocation. The important operands are astype: the value named ’astype’ in this block; int16: the value named ’int16’ in this block.

Line 63: `def packed_mac2_u8(activation: int, weight_lo: int, weight_hi: int) -> tuple[int, int]:` This starts function packed_mac2_u8. Its indented body performs packed mac2 u8 as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 64: `"""Bit-accurate model of neuraedge_mac2_u8, including its guard-bit packing."""` This is executable documentation attached to the function packed_mac2_u8, which handles packed mac2 u8. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 65: `for name, value in (("activation", activation), ("weight_lo", weight_lo), ("weight_hi", weight_hi)):` This iterates name, value over ‘(("activation", activation), ("weight_lo", weight_lo), ("weight_hi", weight_hi))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function packed_mac2_u8, which handles packed mac2 u8.

Line 66: `if not 0 <= value <= 255:` This executes the following indented host-code branch only when ‘not 0 <= value <= 255‘ is true. The important operands are not: the value named ’not’ in this block; value: the value named ’value’ in this block. Context: the function packed_mac2_u8, which handles packed mac2 u8.

Line 67: `raise ValueError(f"{name} must be an unsigned byte")` This rejects an invalid state by raising ‘ValueError(f"{name} must be an unsigned byte")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 68: `packed_weight = weight_lo | (weight_hi << 17)` This assigns ‘weight_lo | (weight_hi << 17)‘ to ‘packed_weight ‘ in host memory. Here packed_weight means a learned coefficient multiplied by an activation. The important operands are weight_lo: a learned coefficient multiplied by an activation; weight_hi: a learned coefficient multiplied by an activation. Context: the function packed_mac2_u8, which handles packed mac2 u8.

Line 69: `packed_product = packed_weight * activation` This assigns ‘packed_weight * activation‘ to ‘packed_product ‘ in host memory. Here packed_product means the value named ’packed product’ in this block. The important operands are packed_weight: a learned coefficient multiplied by an activation; activation: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function packed_mac2_u8, which handles packed mac2 u8.

Line 70: `return packed_product & 0xFFFF, (packed_product >> 17) & 0xFFFF` This returns ‘packed_product & 0xFFFF, (packed_product >> 17) & 0xFFFF‘ to the caller of the function packed_mac2_u8, which handles packed mac2 u8, ending this invocation. The important operands are packed_product: the value named ’packed product’ in this block.

Line 73: `def gemm_u8_zero_point(` This starts function gemm_u8_zero_point. Its indented body performs gemm u8 zero point as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 74: `activations: np.ndarray,` Inside the function gemm_u8_zero_point, which handles gemm u8 zero point, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named ’ndarray’ in this block.

Line 75: `weights: np.ndarray,` Inside the function gemm_u8_zero_point, which handles gemm u8 zero point, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are weights: a learned coefficient multiplied by an activation; ndarray: the value named 'ndarray' in this block.

Line 76: `activation_zero_point: int,` Inside the function gemm_u8_zero_point, which handles gemm u8 zero point, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 77: `weight_zero_point: int,` Inside the function gemm_u8_zero_point, which handles gemm u8 zero point, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are weight_zero_point: a learned coefficient multiplied by an activation.

Line 78: `bias: np.ndarray | None = None,` This assigns 'None,' to 'bias' in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 79: `) -> np.ndarray:` Inside the function gemm_u8_zero_point, which handles gemm u8 zero point, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are ndarray: the value named 'ndarray' in this block.

Line 80: `"""Compute (A-Za)(B-Zb)+bias using the accelerator's raw-sum correction."""` This is executable documentation attached to the function gemm_u8_zero_point, which handles gemm u8 zero point. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 81: `a = np.asarray(activations, dtype=np.int64)` This assigns 'np.asarray(activations, dtype=np.int64)' to 'a ' in host memory. Here a means the value named 'a' in this block. The important operands are asarray: the value named 'asarray' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; dtype: the value named 'dtype' in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 82: `b = np.asarray(weights, dtype=np.int64)` This assigns 'np.asarray(weights, dtype=np.int64)' to 'b ' in host memory. Here b means the value named 'b' in this block. The important operands are asarray: the value named 'asarray' in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named 'dtype' in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 83: `if a.ndim != 2 or b.ndim != 2 or a.shape[1] != b.shape[0]:` This executes the following indented host-code branch only when 'a.ndim != 2 or b.ndim != 2 or a.shape[1] != b.shape[0]' is true. The important operands are a: the value named 'a' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 84: `raise ValueError("expected A[M,K] and B[K,N]")` This rejects an invalid state by raising 'ValueError("expected A[M,K] and B[K,N]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 85: `if np.any(a < 0) or np.any(a > 255) or np.any(b < 0) or np.any(b > 255):` This executes the following indented host-code branch only when 'np.any(a $<$ 0) or np.any(a $>$ 255) or np.any(b $<$ 0) or np.any(b $>$ 255)' is true. The important operands are any: the value named 'any' in this block; a: the value named 'a' in this block; or: the value named 'or' in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 86: `raise ValueError("accelerator GEMM operands must be unsigned bytes")` This rejects an invalid state by raising 'ValueError("accelerator GEMM operands must be unsigned bytes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 87: `if not 0 <= activation_zero_point <= 255 or not 0 <= weight_zero_point <= 255:` This executes the following indented host-code branch only when ‘not 0 <= activation_zero_point <= 255 or not 0 <= weight_zero_point <= 255‘ is true. The important operands are not: the value named ’not’ in this block; activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; or: the value named ’or’ in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 88: `raise ValueError("zero points must be unsigned bytes")` This rejects an invalid state by raising ‘ValueError("zero points must be unsigned bytes")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 90: `reduction = a.shape[1]` This assigns ‘a.shape[1]‘ to ‘reduction ‘ in host memory. Here reduction means the value named ’reduction’ in this block. The important operands are a: the value named ’a’ in this block; shape: the value named ’shape’ in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 91: `raw = a @ b` This assigns ‘a @ b‘ to ‘raw ‘ in host memory. Here raw means the value named ’raw’ in this block. The important operands are a: the value named ’a’ in this block; b: the value named ’b’ in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 92: `row_sum = a.sum(axis=1, keepdims=True)` This assigns ‘a.sum(axis=1, keepdims=True)‘ to ‘row_sum ‘ in host memory. Here row_sum means one output-row position processed by the tensor tile. The important operands are a: the value named ’a’ in this block; sum: the value named ’sum’ in this block; axis: the value named ’axis’ in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 93: `column_sum = b.sum(axis=0, keepdims=True)` This assigns ‘b.sum(axis=0, keepdims=True)‘ to ‘column_sum ‘ in host memory. Here column_sum means one output-feature position processed in parallel. The important operands are b: the value named ’b’ in this block; sum: the value named ’sum’ in this block; axis: the value named ’axis’ in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 94: `corrected = (` This statement continues through source line 99; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(raw - weight_zero_point * row_sum - activation_zero_point * column_sum + reduction * activation_zero_point * weight_zero_point)‘ to ‘corrected ‘ in host memory. Here corrected means the value named ’corrected’ in this block. The important operands are raw: the value named ’raw’ in this block; weight_zero_point: a learned coefficient multiplied by an activation; row_sum: one output-row position processed by the tensor tile. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 95: `raw` This is continuation line 2 of the 6-line logical statement that begins at source line 94. Its exact fragment is ‘raw‘. The important operands are raw: the value named ’raw’ in this block. Read together, the complete operation is: This assigns ‘(raw - weight_zero_point * row_sum - activation_zero_point * column_sum + reduction * activation_zero_point * weight_zero_point)‘ to ‘corrected ‘ in host memory. Here corrected means the value named ’corrected’ in this block. The important operands are raw: the value named ’raw’ in this block; weight_zero_point: a learned coefficient multiplied by an activation; row_sum: one output-row position processed by the tensor tile. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 96: `- weight_zero_point * row_sum` This is continuation line 3 of the 6-line logical statement that begins at source line 94. Its exact fragment is ‘- weight_zero_point * row_sum‘. The important operands are weight_zero_point: a learned coefficient multiplied by an activation; row_sum: one output-row position processed by the tensor tile. Read together, the complete operation is: This assigns ‘(raw - weight_zero_point * row_sum - activation_zero_point * column_sum + reduction * activation_zero_point * weight_zero_point)‘ to ‘corrected ‘ in host memory. Here corrected means the value named ’corrected’ in this block. The important operands are raw: the value named ’raw’ in this block; weight_zero_point: a learned coefficient multiplied by an activation; row_sum: one output-row position processed by the tensor tile. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 97: `- activation_zero_point * column_sum` This is continuation line 4 of the 6-line logical statement that begins at source line 94. Its exact fragment is ‘- activation_zero_point * column_sum‘. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; column_sum:

one output-feature position processed in parallel. Read together, the complete operation is: This assigns ‘(raw - weight_zero_point * row_sum - activation_zero_point * column_sum + reduction * activation_zero_point * weight_zero_point)‘ to ‘corrected ‘ in host memory. Here corrected means the value named ’corrected’ in this block. The important operands are raw: the value named ’raw’ in this block; weight_zero_point: a learned coefficient multiplied by an activation; row_sum: one output-row position processed by the tensor tile. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 98: `+ reduction * activation_zero_point * weight_zero_point` This is continuation line 5 of the 6-line logical statement that begins at source line 94. Its exact fragment is ‘+ reduction * activation_zero_point * weight_zero_point‘. The important operands are reduction: the value named ’reduction’ in this block; activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘(raw - weight_zero_point * row_sum - activation_zero_point * column_sum + reduction * activation_zero_point * weight_zero_point)‘ to ‘corrected ‘ in host memory. Here corrected means the value named ’corrected’ in this block. The important operands are raw: the value named ’raw’ in this block; weight_zero_point: a learned coefficient multiplied by an activation; row_sum: one output-row position processed by the tensor tile. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 99: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 94. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(raw - weight_zero_point * row_sum - activation_zero_point * column_sum + reduction * activation_zero_point * weight_zero_point)‘ to ‘corrected ‘ in host memory. Here corrected means the value named ’corrected’ in this block. The important operands are raw: the value named ’raw’ in this block; weight_zero_point: a learned coefficient multiplied by an activation; row_sum: one output-row position processed by the tensor tile. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 100: `if bias is not None:` This executes the following indented host-code branch only when ‘bias is not None‘ is true. The important operands are bias: an additive learned offset applied after accumulation; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 101: `bias_array = np.asarray(bias, dtype=np.int64).reshape(1, -1)` This assigns ‘np.asarray(bias, dtype=np.int64 -1)‘ to ‘bias_array ‘ in host memory. Here bias_array means an additive learned offset applied after accumulation. The important operands are asarray: the value named ’asarray’ in this block; bias: an additive learned offset applied after accumulation; dtype: the value named ’dtype’ in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 102: `if bias_array.shape[1] != b.shape[1]:` This executes the following indented host-code branch only when ‘bias_array.shape[1] != b.shape[1]‘ is true. The important operands are bias_array: an additive learned offset applied after accumulation; shape: the value named ’shape’ in this block; b: the value named ’b’ in this block. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 103: `raise ValueError("bias length must equal output columns")` This rejects an invalid state by raising ‘ValueError("bias length must equal output columns")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 104: `corrected = corrected + bias_array` This assigns ‘corrected + bias_array‘ to ‘corrected ‘ in host memory. Here corrected means the value named ’corrected’ in this block. The important operands are corrected: the value named ’corrected’ in this block; bias_array: an additive learned offset applied after accumulation. Context: the function gemm_u8_zero_point, which handles gemm u8 zero point.

Line 105: `return corrected` This returns ‘corrected‘ to the caller of the function gemm_u8_zero_point, which handles gemm u8 zero point, ending this invocation. The important operands are corrected: the value named ’corrected’ in this block.

Line 108: `def direct_centered_gemm(` This starts function direct_centered_gemm. Its indented body performs direct centered gemm as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 109: `activations: np.ndarray,` Inside the function direct_centered_gemm, which handles direct centered gemm, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named 'ndarray' in this block.

Line 110: `weights: np.ndarray,` Inside the function direct_centered_gemm, which handles direct centered gemm, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are weights: a learned coefficient multiplied by an activation; ndarray: the value named 'ndarray' in this block.

Line 111: `activation_zero_point: int,` Inside the function direct_centered_gemm, which handles direct centered gemm, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 112: `weight_zero_point: int,` Inside the function direct_centered_gemm, which handles direct centered gemm, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are weight_zero_point: a learned coefficient multiplied by an activation.

Line 113: `bias: np.ndarray | None = None,` This assigns 'None,' to 'bias' in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are None: the value named 'none' in this block. Context: the function direct_centered_gemm, which handles direct centered gemm.

Line 114: `) -> np.ndarray:` Inside the function direct_centered_gemm, which handles direct centered gemm, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are ndarray: the value named 'ndarray' in this block.

Line 115: `"""Simple centered expression used only as an independent algebra oracle."""` This is executable documentation attached to the function direct_centered_gemm, which handles direct centered gemm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 116: `a = np.asarray(activations, dtype=np.int64) - activation_zero_point` This assigns 'np.asarray(activations, dtype=np.int64) - activation_zero_point' to 'a ' in host memory. Here a means the value named 'a' in this block. The important operands are asarray: the value named 'asarray' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; dtype: the value named 'dtype' in this block. Context: the function direct_centered_gemm, which handles direct centered gemm.

Line 117: `b = np.asarray(weights, dtype=np.int64) - weight_zero_point` This assigns 'np.asarray(weights, dtype=np.int64) - weight_zero_point' to 'b ' in host memory. Here b means the value named 'b' in this block. The important operands are asarray: the value named 'asarray' in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named 'dtype' in this block. Context: the function direct_centered_gemm, which handles direct centered gemm.

Line 118: `result = a @ b` This assigns 'a @ b' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are a: the value named 'a' in this block; b: the value named 'b' in this block. Context: the function direct_centered_gemm, which handles direct centered gemm.

Line 119: `if bias is not None:` This executes the following indented host-code branch only when ‘bias is not None‘ is true. The important operands are bias: an additive learned offset applied after accumulation; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function direct_centered_gemm, which handles direct centered gemm.

Line 120: `result = result + np.asarray(bias, dtype=np.int64).reshape(1, -1)` This assigns ‘result + np.asarray(bias, dtype=np.int64).reshape(1, -1)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are result: the value named ’result’ in this block; asarray: the value named ’asarray’ in this block; bias: an additive learned offset applied after accumulation. Context: the function direct_centered_gemm, which handles direct centered gemm.

Line 121: `return result` This returns ‘result‘ to the caller of the function direct_centered_gemm, which handles direct centered gemm, ending this invocation. The important operands are result: the value named ’result’ in this block.

Line 124: `def active_outer_indices(activations: np.ndarray, zero_point: int) -> np.ndarray:` This starts function active_outer_indices. Its indented body performs active outer indices as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 125: `"""Return K indices where at least one row contains a non-zero activation."""` This is executable documentation attached to the function active_outer_indices, which handles active outer indices. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 126: `a = np.asarray(activations)` This assigns ‘np.asarray(activations)‘ to ‘a ‘ in host memory. Here a means the value named ’a’ in this block. The important operands are asarray: the value named ’asarray’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function active_outer_indices, which handles active outer indices.

Line 127: `if a.ndim != 2:` This executes the following indented host-code branch only when ‘a.ndim != 2‘ is true. The important operands are a: the value named ’a’ in this block; ndim: the value named ’ndim’ in this block. Context: the function active_outer_indices, which handles active outer indices.

Line 128: `raise ValueError("outer-mode activations must have shape [M,K]")` This rejects an invalid state by raising ‘ValueError("outer-mode activations must have shape [M,K]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 129: `return np.flatnonzero(np.any(a != zero_point, axis=0)).astype(np.int64)` This returns ‘np.flatnonzero(np.any(a != zero_point, axis=0)).astype(np.int64)‘ to the caller of the function active_outer_indices, which handles active outer indices, ending this invocation. The important operands are flatnonzero: the value named ’flatnonzero’ in this block; any: the value named ’any’ in this block; a: the value named ’a’ in this block.

Line 132: `def active_dot8_indices(activation_row: np.ndarray, zero_point: int) -> np.ndarray:` This starts function active_dot8_indices. Its indented body performs active dot8 indices as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 133: `"""Return eight-element K chunks containing at least one non-zero activation."""` This is executable documentation attached to the function active_dot8_indices, which handles active dot8 indices. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 134: `row = np.asarray(activation_row).reshape(-1)` This assigns ‘np.asarray(activation_row).reshape(-1)‘ to ‘row ‘ in host memory. Here row means one output-row position processed by the tensor tile. The important operands are asarray: the value named ’asarray’ in this block; activation_row: an input feature value consumed by a neuron or multiply-accumulate operation; reshape: the value named ’reshape’ in this block. Context: the function active_dot8_indices, which handles active dot8 indices.

Line 135: `padded_length = ceil(row.size / 8) * 8` This assigns ‘ceil(row.size / 8) * 8‘ to ‘padded_length ‘ in host memory. Here padded_length means the value named ’padded length’ in this block. The important operands are ceil: the value named ’ceil’ in this block; row: one output-row position processed by the tensor tile; size: the value named ’size’ in this block. Context: the function active_dot8_indices, which handles active dot8 indices.

Line 136: `padded = np.full(padded_length, zero_point, dtype=row.dtype)` This assigns ‘np.full(padded_length, zero_point, dtype=row.dtype)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are full: the value named ’full’ in this block; padded_length: the value named ’padded length’ in this block; zero_point: the integer code representing real numeric zero in affine quantization. Context: the function active_dot8_indices, which handles active dot8 indices.

Line 137: `padded[: row.size] = row` This assigns ‘row‘ to ‘padded[‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are row: one output-row position processed by the tensor tile. Context: the function active_dot8_indices, which handles active dot8 indices.

Line 138: `chunks = padded.reshape(-1, 8)` This assigns ‘padded.reshape(-1, 8)‘ to ‘chunks ‘ in host memory. Here chunks means the value named ’chunks’ in this block. The important operands are padded: the value named ’padded’ in this block; reshape: the value named ’reshape’ in this block. Context: the function active_dot8_indices, which handles active dot8 indices.

Line 139: `return np.flatnonzero(np.any(chunks != zero_point, axis=1)).astype(np.int64)` This returns ‘np.flatnonze != zero_point, axis=1)).astype(np.int64)‘ to the caller of the function active_dot8_indices, which handles active dot8 indices, ending this invocation. The important operands are flatnonzero: the value named ’flatnonzero’ in this block; any: the value named ’any’ in this block; chunks: the value named ’chunks’ in this block.

Line 142: `def binary_weighted_gemm(spikes: np.ndarray, weights: np.ndarray) -> np.ndarray:` This starts function binary_weighted_gemm. Its indented body performs binary weighted gemm as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 143: `"""Addition-only matrix product for binary activations in {0,1}."""` This is executable documentation attached to the function binary_weighted_gemm, which handles binary weighted gemm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 144: `x = np.asarray(spikes, dtype=np.int64)` This assigns ‘np.asarray(spikes, dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are asarray: the value named ’asarray’ in this block; spikes: a binary event produced when a neuron crosses its threshold; dtype: the value named ’dtype’ in this block. Context: the function binary_weighted_gemm, which handles binary weighted gemm.

Line 145: `w = np.asarray(weights, dtype=np.int64)` This assigns ‘np.asarray(weights, dtype=np.int64)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are asarray: the value named ’asarray’ in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named ’dtype’ in this block. Context: the function binary_weighted_gemm, which handles binary weighted gemm.

Line 146: `if np.any((x != 0) & (x != 1)):` This executes the following indented host-code branch only when ‘np.any((x != 0) & (x != 1))‘ is true. The important operands are any: the value named ’any’ in this block; x: the value named ’x’ in this block. Context: the function binary_weighted_gemm, which handles binary weighted gemm.

Line 147: `raise ValueError("binary activations must contain only 0 and 1")` This rejects an invalid state by raising ‘ValueError("binary activations must contain only 0 and 1")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 148: `return x @ w` This returns ‘x @ w‘ to the caller of the function binary_weighted_gemm, which handles binary weighted gemm, ending this invocation. The important operands are x: the value named ’x’ in this block; w: the value named ’w’ in this block.

Line 151: `def ternary_weighted_gemm(spikes: np.ndarray, weights: np.ndarray) -> np.ndarray:` This starts function ternary_weighted_gemm. Its indented body performs ternary weighted gemm as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 152: `"""Addition/subtraction-only matrix product for activations in {-1,0,1}."""` This is executable documentation attached to the function ternary_weighted_gemm, which handles ternary weighted gemm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 153: `x = np.asarray(spikes, dtype=np.int64)` This assigns 'np.asarray(spikes, dtype=np.int64)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are asarray: the value named 'asarray' in this block; spikes: a binary event produced when a neuron crosses its threshold; dtype: the value named 'dtype' in this block. Context: the function ternary_weighted_gemm, which handles ternary weighted gemm.

Line 154: `w = np.asarray(weights, dtype=np.int64)` This assigns 'np.asarray(weights, dtype=np.int64)' to 'w ' in host memory. Here w means the value named 'w' in this block. The important operands are asarray: the value named 'asarray' in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named 'dtype' in this block. Context: the function ternary_weighted_gemm, which handles ternary weighted gemm.

Line 155: `if np.any((x < -1) | (x > 1)):` This executes the following indented host-code branch only when 'np.any((x < -1) | (x > 1))' is true. The important operands are any: the value named 'any' in this block; x: the value named 'x' in this block. Context: the function ternary_weighted_gemm, which handles ternary weighted gemm.

Line 156: `raise ValueError("ternary activations must be in {-1,0,1}")` This rejects an invalid state by raising 'ValueError("ternary activations must be in {-1,0,1}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 157: `return x @ w` This returns 'x @ w' to the caller of the function ternary_weighted_gemm, which handles ternary weighted gemm, ending this invocation. The important operands are x: the value named 'x' in this block; w: the value named 'w' in this block.

Line 160: `def alpha_xnor_similarity(query: np.ndarray, key: np.ndarray, alpha_q8: int) -> np.ndarray:` This starts function alpha_xnor_similarity. Its indented body performs alpha xnor similarity as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 161: `"""Equation 18 of Xiao et al. CVPR 2025, represented in unsigned Q8."""` This is executable documentation attached to the function alpha_xnor_similarity, which handles alpha xnor similarity. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 162: `q = np.asarray(query, dtype=np.uint8)` This assigns 'np.asarray(query, dtype=np.uint8)' to 'q ' in host memory. Here q means the value named 'q' in this block. The important operands are asarray: the value named 'asarray' in this block; query: the value named 'query' in this block; dtype: the value named 'dtype' in this block. Context: the function alpha_xnor_similarity, which handles alpha xnor similarity.

Line 163: `k = np.asarray(key, dtype=np.uint8)` This assigns 'np.asarray(key, dtype=np.uint8)' to 'k ' in host memory. Here k means the value named 'k' in this block. The important operands are asarray: the value named 'asarray' in this block; key: the value named 'key' in this block; dtype: the value named 'dtype' in this block. Context: the function alpha_xnor_similarity, which handles alpha xnor similarity.

Line 164: `if q.ndim != 2 or k.ndim != 2 or q.shape[1] != k.shape[1]:` This executes the following indented host-code branch only when 'q.ndim != 2 or k.ndim != 2 or q.shape[1] != k.shape[1]' is true. The important operands are q: the value named 'q' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function alpha_xnor_similarity, which handles alpha xnor similarity.

Line 165: `raise ValueError("expected query[M,D] and key[N,D]")` This rejects an invalid state by raising ‘ValueError("expected query[M,D] and key[N,D]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 166: `if np.any(q > 1) or np.any(k > 1):` This executes the following indented host-code branch only when ‘np.any(q > 1) or np.any(k > 1)‘ is true. The important operands are any: the value named ’any’ in this block; q: the value named ’q’ in this block; or: the value named ’or’ in this block. Context: the function alpha_xnor_similarity, which handles alpha xnor similarity.

Line 167: `raise ValueError("alpha-XNOR inputs must be binary")` This rejects an invalid state by raising ‘ValueError("alph XNOR inputs must be binary")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 168: `if not 0 <= alpha_q8 <= 256:` This executes the following indented host-code branch only when ‘not 0 <= alpha_q8 <= 256‘ is true. The important operands are not: the value named ’not’ in this block; alpha_q8: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the function alpha_xnor_similarity, which handles alpha xnor similarity.

Line 169: `raise ValueError("alpha_q8 must represent a value in [0,1]")` This rejects an invalid state by raising ‘ValueError("alpha_q8 must represent a value in [0,1]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 170: `spike_matches = q.astype(np.int64) @ k.astype(np.int64).T` This assigns ‘q.astype(np.int64) @ k.astype(np.in to ‘spike_matches ‘ in host memory. Here spike_matches means a binary event produced when a neuron crosses its threshold. The important operands are q: the value named ’q’ in this block; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function alpha_xnor_similarity, which handles alpha xnor similarity.

Line 171: `zero_matches = (1 - q.astype(np.int64)) @ (1 - k.astype(np.int64)).T` This assigns ‘(1 - q.astype(np.int64) @ (1 - k.astype(np.int64)).T‘ to ‘zero_matches ‘ in host memory. Here zero_matches means the value named ’zero matches’ in this block. The important operands are q: the value named ’q’ in this block; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function alpha_xnor_similarity, which handles alpha xnor similarity.

Line 172: `return spike_matches * 256 + zero_matches * alpha_q8` This returns ‘spike_matches * 256 + zero_matches * alpha_q8‘ to the caller of the function alpha_xnor_similarity, which handles alpha xnor similarity, ending this invocation. The important operands are spike_matches: a binary event produced when a neuron crosses its threshold; zero_matches: the value named ’zero matches’ in this block; alpha_q8: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 175: `def log_int_softmax_q15(scores: np.ndarray, difference_shift: int) -> np.ndarray:` This starts function log_int_softmax_q15. Its indented body performs log int softmax q15 as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 176: `"""Power-of-two softmax with the Q31 reciprocal used by the RTL datapath."""` This is executable documentation attached to the function log_int_softmax_q15, which handles log int softmax q15. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 177: `x = np.asarray(scores, dtype=np.int64)` This assigns ‘np.asarray(scores, dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are asarray: the value named ’asarray’ in this block; scores: the value named ’scores’ in this block; dtype: the value named ’dtype’ in this block. Context: the function log_int_softmax_q15, which handles log int softmax q15.

Line 178: `if x.ndim != 1 or x.size == 0:` This executes the following indented host-code branch only when ‘x.ndim != 1 or x.size == 0‘ is true. The important operands are x: the value named ’x’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function log_int_softmax_q15, which handles log int softmax q15.

Line 179: `raise ValueError("scores must be a non-empty vector")` This rejects an invalid state by raising ‘ValueError("scores must be a non-empty vector")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 180: `if not 0 <= difference_shift <= 31:` This executes the following indented host-code branch only when ‘not 0 <= difference_shift <= 31‘ is true. The important operands are not: the value named ’not’ in this block; difference_shift: the value named ’difference shift’ in this block. Context: the function log_int_softmax_q15, which handles log int softmax q15.

Line 181: `raise ValueError("difference_shift must be in [0,31]")` This rejects an invalid state by raising ‘ValueError("difference_shift must be in [0,31]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 182: `differences = x.max() - x` This assigns ‘x.max() - x‘ to ‘differences ‘ in host memory. Here differences means the value named ’differences’ in this block. The important operands are x: the value named ’x’ in this block; max: the value named ’max’ in this block. Context: the function log_int_softmax_q15, which handles log int softmax q15.

Line 183: `exponents = np.minimum(differences >> difference_shift, 15).astype(np.int64)` This assigns ‘np.minimum >> difference_shift, 15).astype(np.int64)‘ to ‘exponents ‘ in host memory. Here exponents means the value named ’exponents’ in this block. The important operands are minimum: the value named ’minimum’ in this block; differences: the value named ’differences’ in this block; difference_shift: the value named ’difference shift’ in this block. Context: the function log_int_softmax_q15, which handles log int softmax q15.

Line 184: `unnormalized = (1 << (15 - exponents)).astype(np.int64)` This assigns ‘(1 << (15 - exponents)).astype(np.int6 to ‘unnormalized ‘ in host memory. Here unnormalized means the value named ’unnormalized’ in this block. The important operands are exponents: the value named ’exponents’ in this block; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function log_int_softmax_q15, which handles log int softmax q15.

Line 185: `denominator = int(unnormalized.sum())` This assigns ‘int(unnormalized.sum())‘ to ‘denominator ‘ in host memory. Here denominator means the value named ’denominator’ in this block. The important operands are unnormalized: the value named ’unnormalized’ in this block; sum: the value named ’sum’ in this block. Context: the function log_int_softmax_q15, which handles log int softmax q15.

Line 186: `reciprocal_q31 = ((1 << 31) - 1) // denominator` This assigns ‘((1 << 31) - 1) // denominator‘ to ‘reciprocal_q31 ‘ in host memory. Here reciprocal_q31 means the value named ’reciprocal q31’ in this block. The important operands are denominator: the value named ’denominator’ in this block. Context: the function log_int_softmax_q15, which handles log int softmax q15.

Line 187: `return round_shift_signed(unnormalized * reciprocal_q31, 16)` This returns ‘round_shift_signed(unnormaliz * reciprocal_q31, 16)‘ to the caller of the function log_int_softmax_q15, which handles log int softmax q15, ending this invocation. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; unnormalized: the value named ’unnormalized’ in this block; reciprocal_q31: the value named ’reciprocal q31’ in this block.

Line 190: `def integer_layer_norm(` This starts function integer_layer_norm. Its indented body performs integer layer norm as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 191: `values: np.ndarray,` Inside the function integer_layer_norm, which handles integer layer norm, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are values: the value named 'values' in this block; ndarray: the value named 'ndarray' in this block.

Line 192: `gamma_q15: np.ndarray,` Inside the function integer_layer_norm, which handles integer layer norm, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are gamma_q15: the learned multiplicative LayerNorm parameter; ndarray: the value named 'ndarray' in this block.

Line 193: `beta: np.ndarray,` Inside the function integer_layer_norm, which handles integer layer norm, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are beta: the learned additive LayerNorm parameter; ndarray: the value named 'ndarray' in this block.

Line 194: `epsilon: int = 1,` This assigns '1,' to 'epsilon' in host memory. Here epsilon means the value named 'epsilon' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function integer_layer_norm, which handles integer layer norm.

Line 195: `) -> np.ndarray:` Inside the function integer_layer_norm, which handles integer layer norm, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are ndarray: the value named 'ndarray' in this block.

Line 196: `"""I-ViT-style integer LayerNorm using sqrt and one shared Q31 reciprocal."""` This is executable documentation attached to the function integer_layer_norm, which handles integer layer norm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 197: `x = np.asarray(values, dtype=np.int64).reshape(-1)` This assigns 'np.asarray(values, dtype=np.int64).reshape(-1)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are asarray: the value named 'asarray' in this block; values: the value named 'values' in this block; dtype: the value named 'dtype' in this block. Context: the function integer_layer_norm, which handles integer layer norm.

Line 198: `gamma = np.asarray(gamma_q15, dtype=np.int64).reshape(-1)` This assigns 'np.asarray(gamma_q15, dtype=np.int64).reshape(-1)' to 'gamma ' in host memory. Here gamma means the learned multiplicative LayerNorm parameter. The important operands are asarray: the value named 'asarray' in this block; gamma_q15: the learned multiplicative LayerNorm parameter; dtype: the value named 'dtype' in this block. Context: the function integer_layer_norm, which handles integer layer norm.

Line 199: `offset = np.asarray(beta, dtype=np.int64).reshape(-1)` This assigns 'np.asarray(beta, dtype=np.int64).reshape(-1)' to 'offset ' in host memory. Here offset means the value named 'offset' in this block. The important operands are asarray: the value named 'asarray' in this block; beta: the learned additive LayerNorm parameter; dtype: the value named 'dtype' in this block. Context: the function integer_layer_norm, which handles integer layer norm.

Line 200: `if x.size == 0 or gamma.size != x.size or offset.size != x.size:` This executes the following indented host-code branch only when 'x.size == 0 or gamma.size != x.size or offset.size != x.size' is true. The important operands are x: the value named 'x' in this block; size: the value named 'size' in this block; or: the value named 'or' in this block. Context: the function integer_layer_norm, which handles integer layer norm.

Line 201: `raise ValueError("values, gamma, and beta must be equally sized non-empty vectors")` This rejects an invalid state by raising 'ValueError("values, gamma, and beta must be equally sized non-empty vectors")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 202: `mean = int(x.sum() // x.size)` This assigns ‘int(x.sum() // x.size)‘ to ‘mean ‘ in host memory. Here mean means the arithmetic average used by normalization. The important operands are x: the value named ’x’ in this block; sum: the value named ’sum’ in this block; size: the value named ’size’ in this block. Context: the function integer_layer_norm, which handles integer layer norm.

Line 203: `centered = x - mean` This assigns ‘x - mean‘ to ‘centered ‘ in host memory. Here centered means the value named ’centered’ in this block. The important operands are x: the value named ’x’ in this block; mean: the arithmetic average used by normalization. Context: the function integer_layer_norm, which handles integer layer norm.

Line 204: `variance = int((centered * centered).sum() // x.size)` This assigns ‘int((centered * centered).sum() // x.size)‘ to ‘variance ‘ in host memory. Here variance means the average squared distance from the mean. The important operands are centered: the value named ’centered’ in this block; sum: the value named ’sum’ in this block; x: the value named ’x’ in this block. Context: the function integer_layer_norm, which handles integer layer norm.

Line 205: `sigma = max(1, isqrt(variance + epsilon))` This assigns ‘max(1, isqrt(variance + epsilon))‘ to ‘sigma ‘ in host memory. Here sigma means the value named ’sigma’ in this block. The important operands are max: the value named ’max’ in this block; isqrt: the value named ’isqrt’ in this block; variance: the average squared distance from the mean. Context: the function integer_layer_norm, which handles integer layer norm.

Line 206: `reciprocal_q31 = ((1 << 31) - 1) // sigma` This assigns ‘((1 << 31) - 1) // sigma‘ to ‘reciprocal_q31 ‘ in host memory. Here reciprocal_q31 means the value named ’reciprocal q31’ in this block. The important operands are sigma: the value named ’sigma’ in this block. Context: the function integer_layer_norm, which handles integer layer norm.

Line 207: `normalized = round_shift_signed(centered * reciprocal_q31, 31)` This assigns ‘round_shift_signed(centered * reciprocal_q31, 31)‘ to ‘normalized ‘ in host memory. Here normalized means the value named ’normalized’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; centered: the value named ’centered’ in this block; reciprocal_q31: the value named ’reciprocal q31’ in this block. Context: the function integer_layer_norm, which handles integer layer norm.

Line 208: `return round_shift_signed(normalized * gamma, 15) + offset` This returns ‘round_shift_signed(normalized * gamma, 15) + offset‘ to the caller of the function integer_layer_norm, which handles integer layer norm, ending this invocation. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; normalized: the value named ’normalized’ in this block; gamma: the learned multiplicative LayerNorm parameter.

Line 211: `def lif_step(` This starts function lif_step. Its indented body performs lif step as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 212: `membrane: np.ndarray,` Inside the function lif_step, which handles lif step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; ndarray: the value named ’ndarray’ in this block.

Line 213: `current: np.ndarray,` Inside the function lif_step, which handles lif step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are current: the value named ’current’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 214: `leak_shift: int,` Inside the function lif_step, which handles lif step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are leak_shift: the retained fraction of prior membrane state.

Line 215: `threshold: int,` Inside the function lif_step, which handles lif step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are threshold: the membrane level at which a neuron emits a spike.

Line 216: `reset_value:  int = 0,` This assigns '0,' to 'reset_value' in host memory. Here reset_value means the value named 'reset value' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_step, which handles lif step.

Line 217: `)  -> tuple[np.ndarray, np.ndarray]:` Inside the function lif_step, which handles lif step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 218: `"""One deterministic leaky-integrate-and-fire step."""` This is executable documentation attached to the function lif_step, which handles lif step. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 219: `v = np.asarray(membrane, dtype=np.int64)` This assigns 'np.asarray(membrane, dtype=np.int64)' to 'v ' in host memory. Here v means the value named 'v' in this block. The important operands are asarray: the value named 'asarray' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; dtype: the value named 'dtype' in this block. Context: the function lif_step, which handles lif step.

Line 220: `i = np.asarray(current, dtype=np.int64)` This assigns 'np.asarray(current, dtype=np.int64)' to 'i ' in host memory. Here i means the value named 'i' in this block. The important operands are asarray: the value named 'asarray' in this block; current: the value named 'current' in this block; dtype: the value named 'dtype' in this block. Context: the function lif_step, which handles lif step.

Line 221: `integrated = v - (v >> leak_shift) + i` This assigns 'v - (v >> leak_shift) + i' to 'integrated ' in host memory. Here integrated means the value named 'integrated' in this block. The important operands are v: the value named 'v' in this block; leak_shift: the retained fraction of prior membrane state; i: the value named 'i' in this block. Context: the function lif_step, which handles lif step.

Line 222: `spikes = integrated >= threshold` Inside the function lif_step, which handles lif step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are spikes: a binary event produced when a neuron crosses its threshold; integrated: the value named 'integrated' in this block; threshold: the membrane level at which a neuron emits a spike.

Line 223: `next_membrane = np.where(spikes, reset_value, integrated)` This assigns 'np.where(spikes, reset_value, integrated)' to 'next_membrane ' in host memory. Here next_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are where: the value named 'where' in this block; spikes: a binary event produced when a neuron crosses its threshold; reset_value: the value named 'reset value' in this block. Context: the function lif_step, which handles lif step.

Line 224: `return next_membrane.astype(np.int64), spikes.astype(np.uint8)` This returns 'next_membrane.astype(np.in spikes.astype(np.uint8)' to the caller of the function lif_step, which handles lif step, ending this invocation. The important operands are next_membrane: the persistent LIF neuron state carrying charge between timesteps; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block.

Line 227: `def ternary_spike(values:  np.ndarray, threshold:  int) -> np.ndarray:` This starts function ternary_spike. Its indented body performs ternary spike as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 228: `"""Symmetric ternary firing rule used by the addition-only datapath."""` This is executable documentation attached to the function ternary_spike, which handles ternary spike. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 229: `x = np.asarray(values, dtype=np.int64)` This assigns ‘np.asarray(values, dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block; dtype: the value named ’dtype’ in this block. Context: the function ternary_spike, which handles ternary spike.

Line 230: `return np.where(x >= threshold, 1, np.where(x <= -threshold, -1, 0)).astype(np.int8)` This returns ‘np.where(x >= threshold, 1, np.where(x <= -threshold, -1, 0)).astype(np.int8)‘ to the caller of the function ternary_spike, which handles ternary spike, ending this invocation. The important operands are where: the value named ’where’ in this block; x: the value named ’x’ in this block; threshold: the membrane level at which a neuron emits a spike.

Line 233: `def saturate_signed(values: np.ndarray | int, bits: int) -> np.ndarray:` This starts function saturate_signed. Its indented body performs saturate signed as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 234: `"""Clamp integers to the exact range of a signed two's-complement field."""` This is executable documentation attached to the function saturate_signed, which handles saturate signed. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 235: `if bits < 2 or bits > 63:` This executes the following indented host-code branch only when ‘bits < 2 or bits > 63‘ is true. The important operands are bits: the value named ’bits’ in this block; or: the value named ’or’ in this block. Context: the function saturate_signed, which handles saturate signed.

Line 236: `raise ValueError("bits must be in [2,63]")` This rejects an invalid state by raising ‘ValueError("bits must be in [2,63]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 237: `array = np.asarray(values, dtype=np.int64)` This assigns ‘np.asarray(values, dtype=np.int64)‘ to ‘array ‘ in host memory. Here array means the value named ’array’ in this block. The important operands are asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block; dtype: the value named ’dtype’ in this block. Context: the function saturate_signed, which handles saturate signed.

Line 238: `minimum = -(1 << (bits - 1))` This assigns ‘-(1 << (bits - 1))‘ to ‘minimum ‘ in host memory. Here minimum means the value named ’minimum’ in this block. The important operands are bits: the value named ’bits’ in this block. Context: the function saturate_signed, which handles saturate signed.

Line 239: `maximum = (1 << (bits - 1)) - 1` This assigns ‘(1 << (bits - 1)) - 1‘ to ‘maximum ‘ in host memory. Here maximum means the value named ’maximum’ in this block. The important operands are bits: the value named ’bits’ in this block. Context: the function saturate_signed, which handles saturate signed.

Line 240: `return np.clip(array, minimum, maximum).astype(np.int64)` This returns ‘np.clip(array, minimum, maximum).astype(np.int64)‘ to the caller of the function saturate_signed, which handles saturate signed, ending this invocation. The important operands are clip: the value named ’clip’ in this block; array: the value named ’array’ in this block; minimum: the value named ’minimum’ in this block.

Line 243: `def round_shift_signed(values: np.ndarray | int, shift: int) -> np.ndarray:` This starts function round_shift_signed. Its indented body performs round shift signed as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 244: `"""Round signed integers to nearest, with exact half cases away from zero."""` This is executable documentation attached to the function round_shift_signed, which handles round shift signed. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 245: `if not 0 <= shift <= 62:` This executes the following indented host-code branch only when ‘not 0 <= shift <= 62‘ is true. The important operands are not: the value named ’not’ in this block; shift: the value named ’shift’ in this block. Context: the function round_shift_signed, which handles round shift signed.

Line 246: `raise ValueError("shift must be in [0,62]")` This rejects an invalid state by raising ‘ValueError("shift must be in [0,62]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 247: `array = np.asarray(values, dtype=np.int64)` This assigns ‘np.asarray(values, dtype=np.int64)‘ to ‘array ‘ in host memory. Here array means the value named ’array’ in this block. The important operands are asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block; dtype: the value named ’dtype’ in this block. Context: the function round_shift_signed, which handles round shift signed.

Line 248: `if shift == 0:` This executes the following indented host-code branch only when ‘shift == 0‘ is true. The important operands are shift: the value named ’shift’ in this block. Context: the function round_shift_signed, which handles round shift signed.

Line 249: `return array.copy()` This returns ‘array.copy()‘ to the caller of the function round_shift_signed, which handles round shift signed, ending this invocation. The important operands are array: the value named ’array’ in this block; copy: the value named ’copy’ in this block.

Line 250: `magnitude = np.abs(array)` This assigns ‘np.abs(array)‘ to ‘magnitude ‘ in host memory. Here magnitude means the value named ’magnitude’ in this block. The important operands are abs: the value named ’abs’ in this block; array: the value named ’array’ in this block. Context: the function round_shift_signed, which handles round shift signed.

Line 251: `rounded = (magnitude + (1 << (shift - 1))) >> shift` This assigns ‘(magnitude + (1 << (shift - 1))) >> shift‘ to ‘rounded ‘ in host memory. Here rounded means the value named ’rounded’ in this block. The important operands are magnitude: the value named ’magnitude’ in this block; shift: the value named ’shift’ in this block. Context: the function round_shift_signed, which handles round shift signed.

Line 252: `return np.where(array < 0, -rounded, rounded).astype(np.int64)` This returns ‘np.where(array < 0, -rounded, rounded).astype(np.int64)‘ to the caller of the function round_shift_signed, which handles round shift signed, ending this invocation. The important operands are where: the value named ’where’ in this block; array: the value named ’array’ in this block; rounded: the value named ’rounded’ in this block.

Line 255: `def linear_backward_tile(` This starts function linear_backward_tile. Its indented body performs linear backward tile as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 256: `activations: np.ndarray,` Inside the function linear_backward_tile, which handles linear backward tile, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named ’ndarray’ in this block.

Line 257: `weights: np.ndarray,` Inside the function linear_backward_tile, which handles linear backward tile, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are weights: a learned coefficient multiplied by an activation; ndarray: the value named ’ndarray’ in this block.

Line 258: `output_gradients: np.ndarray,` Inside the function linear_backward_tile, which handles linear backward tile, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are output_gradients: the signed loss derivative used to update a learned parameter; ndarray: the value named ’ndarray’ in this block.

Line 259: `*,` Inside the function linear_backward_tile, which handles linear backward tile, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The expression is a literal or structural operator and has no hidden software state.

Line 260: `input_gradient_shift: int = 0,` This assigns ‘0,‘ to ‘input_gradient_shift‘ in host memory. Here input_gradient_shift means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function linear_backward_tile, which handles linear backward tile.

Line 261: `weight_gradient_shift: int = 0,` This assigns ‘0,‘ to ‘weight_gradient_shift‘ in host memory. Here weight_gradient_shift means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function linear_backward_tile, which handles linear backward tile.

Line 262: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function linear_backward_tile, which handles linear backward tile, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are tuple: the value named ’tuple’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 263: `"""Bit-exact reference for the signed RTL linear backward tile."""` This is executable documentation attached to the function linear_backward_tile, which handles linear backward tile. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 265: `x = np.asarray(activations, dtype=np.int64)` This assigns ‘np.asarray(activations, dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are asarray: the value named ’asarray’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; dtype: the value named ’dtype’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 266: `w = np.asarray(weights, dtype=np.int64)` This assigns ‘np.asarray(weights, dtype=np.int64)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are asarray: the value named ’asarray’ in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named ’dtype’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 267: `dy = np.asarray(output_gradients, dtype=np.int64)` This assigns ‘np.asarray(output_gradients, dtype=np.int64
to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are asarray: the value named ’asarray’ in this block; output_gradients: the signed loss derivative used to update a learned parameter; dtype: the value named ’dtype’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 268: `if x.ndim != 2 or w.ndim != 2 or dy.ndim != 2:` This executes the following indented host-code branch only when ‘x.ndim != 2 or w.ndim != 2 or dy.ndim != 2‘ is true. The important operands are x: the value named ’x’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 269: `raise ValueError("linear backward operands must be two-dimensional")` This rejects an invalid state by raising ‘ValueError("linear backward operands must be two-dimensional")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 270: `rows, input_features = x.shape` This assigns ‘x.shape‘ to ‘rows, input_features ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are x: the value named ’x’ in this block; shape: the value named ’shape’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 271: `weight_inputs, output_features = w.shape` This assigns ‘w.shape‘ to ‘weight_inputs, output_features ‘ in host memory. Here weight_inputs means a learned coefficient multiplied by an activation. The important operands are w: the value named ’w’ in this block; shape: the value named ’shape’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 272: `if weight_inputs != input_features or dy.shape != (rows, output_features):` This executes the following indented host-code branch only when ‘weight_inputs != input_features or dy.shape != (rows, output_features)‘ is true. The important operands are weight_inputs: a learned coefficient multiplied by an activation; input_features: the value named ’input features’ in this block; or: the value named ’or’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 273: `raise ValueError("linear backward matrix dimensions do not align")` This rejects an invalid state by raising ‘ValueError("linear backward matrix dimensions do not align")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 274: `if not 1 <= rows <= 8 or not 1 <= input_features <= 8 or not 1 <= output_features <= 8:` This executes the following indented host-code branch only when ‘not 1 <= rows <= 8 or not 1 <= input_features <= 8 or not 1 <= output_features <= 8‘ is true. The important operands are not: the value named ’not’ in this block; rows: one output-row position processed by the tensor tile; or: the value named ’or’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 275: `raise ValueError("linear backward tile dimensions must be in [1,8]")` This rejects an invalid state by raising ‘ValueError("linear backward tile dimensions must be in [1,8]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 276: `if np.any(x < -(1 << 15)) or np.any(x > (1 << 15) - 1):` This executes the following indented host-code branch only when ‘np.any(x < -(1 << 15)) or np.any(x > (1 << 15) - 1)‘ is true. The important operands are any: the value named ’any’ in this block; x: the value named ’x’ in this block; or: the value named ’or’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 277: `raise ValueError("linear backward activations must fit signed 16-bit")` This rejects an invalid state by raising ‘ValueError("linear backward activations must fit signed 16-bit")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 278: `if np.any(w < -(1 << 15)) or np.any(w > (1 << 15) - 1):` This executes the following indented host-code branch only when ‘np.any(w < -(1 << 15)) or np.any(w > (1 << 15) - 1)‘ is true. The important operands are any: the value named ’any’ in this block; w: the value named ’w’ in this block; or: the value named ’or’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 279: `raise ValueError("linear backward weights must fit signed 16-bit")` This rejects an invalid state by raising ‘ValueError("linear backward weights must fit signed 16-bit")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 280: `if np.any(dy < -(1 << 15)) or np.any(dy > (1 << 15) - 1):` This executes the following indented host-code branch only when ‘np.any(dy < -(1 << 15)) or np.any(dy > (1 << 15) - 1)‘ is true. The important operands are any: the value named ’any’ in this block; dy: the value named ’dy’ in this block; or: the value named ’or’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 281: `raise ValueError("linear backward output gradients must fit signed 16-bit")` This rejects an invalid state by raising ‘ValueError("linear backward output gradients must fit signed 16-bit")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 282: `if not 0 <= input_gradient_shift <= 31 or not 0 <= weight_gradient_shift <= 31:` This executes the following indented host-code branch only when ‘not 0 <= input_gradient_shift <= 31 or not 0 <= weight_gradient_shift <= 31‘ is true. The important operands are not: the value named ’not’ in this block; input_gradient_shift: the signed loss derivative used to update a learned parameter; or: the value named ’or’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 283: `raise ValueError("linear backward shifts must be in [0,31]")` This rejects an invalid state by raising ‘ValueError("linear backward shifts must be in [0,31]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 285: `input_gradient = round_shift_signed(dy @ w.T, input_gradient_shift)` This assigns ‘round_shift_signed(dy @ w.T, input_gradient_shift)‘ to ‘input_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; dy: the value named ’dy’ in this block; w: the value named ’w’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 286: `weight_gradient = round_shift_signed(x.T @ dy, weight_gradient_shift)` This assigns ‘round_shift_signed(@ dy, weight_gradient_shift)‘ to ‘weight_gradient ‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; x: the value named ’x’ in this block; T: the value named ’t’ in this block. Context: the function linear_backward_tile, which handles linear backward tile.

Line 287: `return saturate_signed(input_gradient, 32), saturate_signed(weight_gradient, 24)` This returns ‘saturate_signed(input_gradient, 32), saturate_signed(weight_gradient, 24)‘ to the caller of the function linear_backward_tile, which handles linear backward tile, ending this invocation. The important operands are saturate_signed: the value named ’saturate signed’ in this block; input_gradient: the signed loss derivative used to update a learned parameter; weight_gradient: a learned coefficient multiplied by an activation.

Line 290: `def quantize_flexible_scale(scale: float) -> tuple[int, int, float]:` This starts function quantize_flexible_scale. Its indented body performs quantize flexible scale as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 291: `"""Encode a signed scale as Q15 mantissa and signed binary-point shift.` This is executable documentation attached to the function quantize_flexible_scale, which handles quantize flexible scale. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 293: `The flexible vector operation evaluates "x * multiplier / 2**(15+shift)".` Inside the function quantize_flexible_scale, which handles quantize flexible scale, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are The: the value named ’the’ in this block; flexible: the value named ’flexible’ in this block; vector: the value named ’vector’ in this block.

Line 294: `Its signed five-bit shift covers [-15, 15], giving useful non-zero scale` Inside the function quantize_flexible_scale, which handles quantize flexible scale, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are Its: the value named ’its’ in this block; five: the value named ’five’ in this block; bit: the value named ’bit’ in this block.

Line 295: `magnitudes from approximately 2^-16 through 2^15.` Inside the function quantize_flexible_scale, which handles quantize flexible scale, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are magnitudes: the value named ’magnitudes’ in this block; from: the value named ’from’ in this block; approximately: the value named ’approximately’ in this block.

Line 296: `"""` This is executable documentation attached to the function quantize_flexible_scale, which handles quantize flexible scale. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 298: `if not np.isfinite(scale):` This executes the following indented host-code branch only when 'not np.isfinite(scale)' is true. The important operands are not: the value named 'not' in this block; isfinite: the value named 'isfinite' in this block; scale: the real-value spacing represented by one quantized integer step. Context: the function quantize_flexible_scale, which handles quantize flexible scale.

Line 299: `raise ValueError("scale must be finite")` This rejects an invalid state by raising 'ValueError("scale must be finite")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 300: `if scale == 0.0:` This executes the following indented host-code branch only when 'scale == 0.0' is true. The important operands are scale: the real-value spacing represented by one quantized integer step. Context: the function quantize_flexible_scale, which handles quantize flexible scale.

Line 301: `return 0, 0, 0.0` This returns '0, 0, 0.0' to the caller of the function quantize_flexible_scale, which handles quantize flexible scale, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 302: `mantissa, exponent = frexp(scale)` This assigns 'frexp(scale)' to 'mantissa, exponent ' in host memory. Here mantissa means the value named 'mantissa' in this block. The important operands are frexp: the value named 'frexp' in this block; scale: the real-value spacing represented by one quantized integer step. Context: the function quantize_flexible_scale, which handles quantize flexible scale.

Line 303: `multiplier = int(round(mantissa * (1 << 15)))` This assigns 'int(round(mantissa * (1 << 15)))' to 'multiplier ' in host memory. Here multiplier means the value named 'multiplier' in this block. The important operands are round: the value named 'round' in this block; mantissa: the value named 'mantissa' in this block. Context: the function quantize_flexible_scale, which handles quantize flexible scale.

Line 304: `if multiplier == (1 << 15):` This executes the following indented host-code branch only when 'multiplier == (1 << 15)' is true. The important operands are multiplier: the value named 'multiplier' in this block. Context: the function quantize_flexible_scale, which handles quantize flexible scale.

Line 305: `multiplier //= 2` Inside the function quantize_flexible_scale, which handles quantize flexible scale, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are multiplier: the value named 'multiplier' in this block.

Line 306: `exponent += 1` Inside the function quantize_flexible_scale, which handles quantize flexible scale, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are exponent: the value named 'exponent' in this block.

Line 307: `if multiplier < -(1 << 15) or multiplier > (1 << 15) - 1:` This executes the following indented host-code branch only when 'multiplier < -(1 << 15) or multiplier > (1 << 15) - 1' is true. The important operands are multiplier: the value named 'multiplier' in this block; or: the value named 'or' in this block. Context: the function quantize_flexible_scale, which handles quantize flexible scale.

Line 308: `raise ValueError("scale mantissa does not fit signed Q15")` This rejects an invalid state by raising 'ValueError("scale mantissa does not fit signed Q15")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 309: `shift = -exponent` This assigns '-exponent' to 'shift ' in host memory. Here shift means the value named 'shift' in this block. The important operands are exponent: the value named 'exponent' in this block. Context: the function quantize_flexible_scale, which handles quantize flexible scale.

Line 310: `if not -15 <= shift <= 15:` This executes the following indented host-code branch only when ‘not -15 <= shift <= 15‘ is true. The important operands are not: the value named ’not’ in this block; shift: the value named ’shift’ in this block. Context: the function quantize_flexible_scale, which handles quantize flexible scale.

Line 311: `raise ValueError("scale is outside the flexible requantizer range")` This rejects an invalid state by raising ‘ValueError("scale is outside the flexible requantizer range")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 312: `represented = multiplier / float(1 << (15 + shift))` This assigns ‘multiplier / float(1 << (15 + shift))‘ to ‘represented ‘ in host memory. Here represented means the value named ’represented’ in this block. The important operands are multiplier: the value named ’multiplier’ in this block; float: the value named ’float’ in this block; shift: the value named ’shift’ in this block. Context: the function quantize_flexible_scale, which handles quantize flexible scale.

Line 313: `return multiplier, shift, represented` This returns ‘multiplier, shift, represented‘ to the caller of the function quantize_flexible_scale, which handles quantize flexible scale, ending this invocation. The important operands are multiplier: the value named ’multiplier’ in this block; shift: the value named ’shift’ in this block; represented: the value named ’represented’ in this block.

Line 316: `def sgd_momentum_step(` This starts function sgd_momentum_step. Its indented body performs sgd momentum step as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 317: `weights: np.ndarray,` Inside the function sgd_momentum_step, which handles sgd momentum step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are weights: a learned coefficient multiplied by an activation; ndarray: the value named ’ndarray’ in this block.

Line 318: `gradients: np.ndarray,` Inside the function sgd_momentum_step, which handles sgd momentum step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are gradients: the signed loss derivative used to update a learned parameter; ndarray: the value named ’ndarray’ in this block.

Line 319: `velocities: np.ndarray,` Inside the function sgd_momentum_step, which handles sgd momentum step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are velocities: the value named ’velocities’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 320: `learning_rate_q16: int,` Inside the function sgd_momentum_step, which handles sgd momentum step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude.

Line 321: `momentum_q16: int,` Inside the function sgd_momentum_step, which handles sgd momentum step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are momentum_q16: the value named ’momentum q16’ in this block.

Line 322: `weight_decay_q16: int,` Inside the function sgd_momentum_step, which handles sgd momentum step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are weight_decay_q16: a learned coefficient multiplied by an activation.

Line 323: `update_right_shift:  int = 16,` This assigns '16,' to 'update_right_shift' in host memory. Here update_right_shift means the value named 'update right shift' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 324: `)  -> tuple[np.ndarray, np.ndarray]:` Inside the function sgd_momentum_step, which handles sgd momentum step, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 325: `"""Bit-exact SGDW update used by the fabric optimizer lanes."""` This is executable documentation attached to the function sgd_momentum_step, which handles sgd momentum step. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 326: `w = np.asarray(weights, dtype=np.int64)` This assigns 'np.asarray(weights, dtype=np.int64)' to 'w ' in host memory. Here w means the value named 'w' in this block. The important operands are asarray: the value named 'asarray' in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named 'dtype' in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 327: `g = np.asarray(gradients, dtype=np.int64)` This assigns 'np.asarray(gradients, dtype=np.int64)' to 'g ' in host memory. Here g means the value named 'g' in this block. The important operands are asarray: the value named 'asarray' in this block; gradients: the signed loss derivative used to update a learned parameter; dtype: the value named 'dtype' in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 328: `v = np.asarray(velocities, dtype=np.int64)` This assigns 'np.asarray(velocities, dtype=np.int64)' to 'v ' in host memory. Here v means the value named 'v' in this block. The important operands are asarray: the value named 'asarray' in this block; velocities: the value named 'velocities' in this block; dtype: the value named 'dtype' in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 329: `if w.shape != g.shape or w.shape != v.shape:` This executes the following indented host-code branch only when 'w.shape != g.shape or w.shape != v.shape' is true. The important operands are w: the value named 'w' in this block; shape: the value named 'shape' in this block; g: the value named 'g' in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 330: `raise ValueError("weights, gradients, and velocities must have equal shapes")` This rejects an invalid state by raising 'ValueError("weights, gradients, and velocities must have equal shapes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 331: `if not 0 <= learning_rate_q16 <= 131071:` This executes the following indented host-code branch only when 'not 0 <= learning_rate_q16 <= 131071' is true. The important operands are not: the value named 'not' in this block; learning_rate_q16: the fixed-point multiplier controlling update magnitude. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 332: `raise ValueError("learning_rate_q16 must be in [0,131071]")` This rejects an invalid state by raising 'ValueError("learning_rate_q16 must be in [0,131071]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 333: `if not 0 <= momentum_q16 <= 65536 or not 0 <= weight_decay_q16 <= 65536:` This executes the following indented host-code branch only when 'not 0 <= momentum_q16 <= 65536 or not 0 <= weight_decay_q16 <= 65536' is true. The important operands are not: the value named 'not' in this block; momentum_q16: the value named 'momentum q16' in this block; or: the value named 'or' in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 334: `raise ValueError("momentum and weight decay must be unsigned Q16 values")` This rejects an invalid state by raising ‘ValueError("momentum and weight decay must be unsigned Q16 values")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 335: `if not 0 <= update_right_shift <= 31:` This executes the following indented host-code branch only when ‘not 0 <= update_right_shift <= 31‘ is true. The important operands are not: the value named ’not’ in this block; update_right_shift: the value named ’update right shift’ in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 336: `raise ValueError("update_right_shift must be in [0,31]")` This rejects an invalid state by raising ‘ValueError("update_right_shift must be in [0,31]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 337: `momentum_term = round_shift_signed(v * momentum_q16, 16)` This assigns ‘round_shift_signed(v * momentum_q16, 16)‘ to ‘momentum_term ‘ in host memory. Here momentum_term means the value named ’momentum term’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; v: the value named ’v’ in this block; momentum_q16: the value named ’momentum q16’ in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 338: `decay_term = round_shift_signed(w * weight_decay_q16, 16)` This assigns ‘round_shift_signed(w * weight_decay_q16, 16)‘ to ‘decay_term ‘ in host memory. Here decay_term means the value named ’decay term’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; w: the value named ’w’ in this block; weight_decay_q16: a learned coefficient multiplied by an activation. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 339: `next_velocity = saturate_signed(momentum_term + g, 24)` This assigns ‘saturate_signed(momentum_term + g, 24)‘ to ‘next_velocity ‘ in host memory. Here next_velocity means the momentum history retained by the SGD optimizer. The important operands are saturate_signed: the value named ’saturate signed’ in this block; momentum_term: the value named ’momentum term’ in this block; g: the value named ’g’ in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 340: `update_direction = saturate_signed(next_velocity + decay_term, 24)` This assigns ‘saturate_signed(next_v + decay_term, 24)‘ to ‘update_direction ‘ in host memory. Here update_direction means the value named ’update direction’ in this block. The important operands are saturate_signed: the value named ’saturate signed’ in this block; next_velocity: the momentum history retained by the SGD optimizer; decay_term: the value named ’decay term’ in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 341: `update = round_shift_signed(` This statement continues through source line 342; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘round_shift_signed(update_direction * learning_rate_q16, update_right_shift)‘ to ‘update ‘ in host memory. Here update means the value named ’update’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; update_direction: the value named ’update direction’ in this block; learning_rate_q16: the fixed-point multiplier controlling update magnitude. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 342: `update_direction * learning_rate_q16, update_right_shift)` This is continuation line 2 of the 2-line logical statement that begins at source line 341. Its exact fragment is ‘update_direction * learning_rate_q16, update_right_shift)‘. The important operands are update_direction: the value named ’update direction’ in this block; learning_rate_q16: the fixed-point multiplier controlling update magnitude; update_right_shift: the value named ’update right shift’ in this block. Read together, the complete operation is: This assigns ‘round_shift_signed(update_direction * learning_rate_q16, update_right_shift)‘ to ‘update ‘ in host memory. Here update means the value named ’update’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; update_direction: the value named ’update direction’ in this block; learning_rate_q16: the fixed-point multiplier controlling update magnitude. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 343: `next_weight = saturate_signed(w - update, 16)` This assigns ‘saturate_signed(w - update, 16)‘ to ‘next_weight ‘ in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are saturate_signed: the value named ’saturate signed’ in this block; w: the value named ’w’ in this block; update: the value named ’update’ in this block. Context: the function sgd_momentum_step, which handles sgd momentum step.

Line 344: `return next_weight, next_velocity` This returns ‘next_weight, next_velocity‘ to the caller of the function sgd_momentum_step, which handles sgd momentum step, ending this invocation. The important operands are next_weight: a learned coefficient multiplied by an activation; next_velocity: the momentum history retained by the SGD optimizer.

Line 347: `def vector_alu(` This starts function vector_alu. Its indented body performs vector alu as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 348: `operation: VectorOperation,` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block.

Line 349: `operand_a: np.ndarray,` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are operand_a: the value named ’operand a’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 350: `operand_b: np.ndarray,` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are operand_b: the value named ’operand b’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 351: `*,` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The expression is a literal or structural operator and has no hidden software state.

Line 352: `multiplier_q15: int = 0,` This assigns ‘0,‘ to ‘multiplier_q15‘ in host memory. Here multiplier_q15 means the value named ’multiplier q15’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_alu, which handles vector alu.

Line 353: `right_shift: int = 0,` This assigns ‘0,‘ to ‘right_shift‘ in host memory. Here right_shift means the value named ’right shift’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_alu, which handles vector alu.

Line 354: `output_zero_point: int = 0,` This assigns ‘0,‘ to ‘output_zero_point‘ in host memory. Here output_zero_point means the integer code representing real numeric zero in affine quantization. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_alu, which handles vector alu.

Line 355: `threshold: int = 0,` This assigns ‘0,‘ to ‘threshold‘ in host memory. Here threshold means the membrane level at which a neuron emits a spike. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_alu, which handles vector alu.

Line 356: `surrogate_window: int = 0,` This assigns ‘0,‘ to ‘surrogate_window‘ in host memory. Here surrogate_window means the value named ’surrogate window’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_alu, which handles vector alu.

Line 357: `clamp_minimum: int = -(1 << 31),` This assigns '-(1 << 31),' to 'clamp_minimum' in host memory. Here clamp_minimum means the value named 'clamp minimum' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_alu, which handles vector alu.

Line 358: `clamp_maximum: int = (1 << 31) - 1,` This assigns '(1 << 31) - 1,' to 'clamp_maximum' in host memory. Here clamp_maximum means the value named 'clamp maximum' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_alu, which handles vector alu.

Line 359: `) -> np.ndarray:` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are ndarray: the value named 'ndarray' in this block.

Line 360: `"""Reference for every operation in the eight-lane integer vector ALU."""` This is executable documentation attached to the function vector_alu, which handles vector alu. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 361: `a = np.asarray(operand_a, dtype=np.int64)` This assigns 'np.asarray(operand_a, dtype=np.int64)' to 'a ' in host memory. Here a means the value named 'a' in this block. The important operands are asarray: the value named 'asarray' in this block; operand_a: the value named 'operand a' in this block; dtype: the value named 'dtype' in this block. Context: the function vector_alu, which handles vector alu.

Line 362: `b = np.asarray(operand_b, dtype=np.int64)` This assigns 'np.asarray(operand_b, dtype=np.int64)' to 'b ' in host memory. Here b means the value named 'b' in this block. The important operands are asarray: the value named 'asarray' in this block; operand_b: the value named 'operand b' in this block; dtype: the value named 'dtype' in this block. Context: the function vector_alu, which handles vector alu.

Line 363: `if a.shape != b.shape:` This executes the following indented host-code branch only when 'a.shape != b.shape' is true. The important operands are a: the value named 'a' in this block; shape: the value named 'shape' in this block; b: the value named 'b' in this block. Context: the function vector_alu, which handles vector alu.

Line 364: `raise ValueError("operand vectors must have equal shapes")` This rejects an invalid state by raising 'ValueError("operand vectors must have equal shapes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 365: `if operation == VectorOperation.FLEX_REQUANTIZE:` This executes the following indented host-code branch only when 'operation == VectorOperation.FLEX_REQUANTIZE' is true. The important operands are operation: the value named 'operation' in this block; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function vector_alu, which handles vector alu.

Line 366: `if not -15 <= right_shift <= 15:` This executes the following indented host-code branch only when 'not -15 <= right_shift <= 15' is true. The important operands are not: the value named 'not' in this block; right_shift: the value named 'right shift' in this block. Context: the function vector_alu, which handles vector alu.

Line 367: `raise ValueError("flexible binary-point shift must be in [-15,15]")` This rejects an invalid state by raising 'ValueError("flexible binary-point shift must be in [-15,15]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 368: `elif not 0 <= right_shift <= 31:` This executes the following indented host-code branch only when 'not 0 <= right_shift <= 31' is true. The important operands are not: the value named 'not' in this block; right_shift: the value named 'right shift' in this block. Context: the function vector_alu, which handles vector alu.

Line 369: `raise ValueError("right_shift must be in [0,31]")` This rejects an invalid state by raising ‘ValueError("right_shift must be in [0,31]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 370: `if clamp_minimum > clamp_maximum:` This executes the following indented host-code branch only when ‘clamp_minimum > clamp_maximum‘ is true. The important operands are clamp_minimum: the value named ’clamp minimum’ in this block; clamp_maximum: the value named ’clamp maximum’ in this block. Context: the function vector_alu, which handles vector alu.

Line 371: `raise ValueError("clamp_minimum must not exceed clamp_maximum")` This rejects an invalid state by raising ‘ValueError("clamp_minimum must not exceed clamp_maximum")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 373: `if operation == VectorOperation.ADD_SAT:` This executes the following indented host-code branch only when ‘operation == VectorOperation.ADD_SAT‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; ADD_SAT: the value named ’add sat’ in this block. Context: the function vector_alu, which handles vector alu.

Line 374: `output = saturate_signed(a + b, 32)` This assigns ‘saturate_signed(a + b, 32)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are saturate_signed: the value named ’saturate signed’ in this block; a: the value named ’a’ in this block; b: the value named ’b’ in this block. Context: the function vector_alu, which handles vector alu.

Line 375: `elif operation == VectorOperation.RELU:` This executes the following indented host-code branch only when ‘operation == VectorOperation.RELU‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; RELU: the value named ’relu’ in this block. Context: the function vector_alu, which handles vector alu.

Line 376: `output = np.maximum(a, 0)` This assigns ‘np.maximum(a, 0)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are maximum: the value named ’maximum’ in this block; a: the value named ’a’ in this block. Context: the function vector_alu, which handles vector alu.

Line 377: `elif operation == VectorOperation.REQUANTIZE:` This executes the following indented host-code branch only when ‘operation == VectorOperation.REQUANTIZE‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function vector_alu, which handles vector alu.

Line 378: `output = round_shift_signed(a * multiplier_q15, 15 + right_shift)` This assigns ‘round_shift_signed(a * multiplier_q15, 15 + right_shift)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; a: the value named ’a’ in this block; multiplier_q15: the value named ’multiplier q15’ in this block. Context: the function vector_alu, which handles vector alu.

Line 379: `output = np.clip(output + output_zero_point, clamp_minimum, clamp_maximum)` This assigns ‘np.clip(output + output_zero_point, clamp_minimum, clamp_maximum)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are clip: the value named ’clip’ in this block; output: the value named ’output’ in this block; output_zero_point: the integer code representing real numeric zero in affine quantization. Context: the function vector_alu, which handles vector alu.

Line 380: `elif operation == VectorOperation.FLEX_REQUANTIZE:` This executes the following indented host-code branch only when ‘operation == VectorOperation.FLEX_REQUANTIZE‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function vector_alu, which handles vector alu.

Line 381: `output = round_shift_signed(a * multiplier_q15, 15 + right_shift)` This assigns ‘round_shift_signed(a * multiplier_q15, 15 + right_shift)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; a: the value named ’a’ in this block; multiplier_q15: the value named ’multiplier q15’ in this block. Context: the function vector_alu, which handles vector alu.

Line 382: `output = np.clip(output + output_zero_point, clamp_minimum, clamp_maximum)` This assigns ‘np.clip(output + output_zero_point, clamp_minimum, clamp_maximum)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are clip: the value named ’clip’ in this block; output: the value named ’output’ in this block; output_zero_point: the integer code representing real numeric zero in affine quantization. Context: the function vector_alu, which handles vector alu.

Line 383: `elif operation == VectorOperation.BINARY_SPIKE:` This executes the following indented host-code branch only when ‘operation == VectorOperation.BINARY_SPIKE‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; BINARY_SPIKE: a binary event produced when a neuron crosses its threshold. Context: the function vector_alu, which handles vector alu.

Line 384: `output = (a >= threshold).astype(np.int64)` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are output: the value named ’output’ in this block; a: the value named ’a’ in this block; threshold: the membrane level at which a neuron emits a spike.

Line 385: `elif operation == VectorOperation.TERNARY_SPIKE:` This executes the following indented host-code branch only when ‘operation == VectorOperation.TERNARY_SPIKE‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; TERNARY_SPIKE: a binary event produced when a neuron crosses its threshold. Context: the function vector_alu, which handles vector alu.

Line 386: `output = np.where(a >= threshold, 1, np.where(a <= -threshold, -1, 0))` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are output: the value named ’output’ in this block; where: the value named ’where’ in this block; a: the value named ’a’ in this block.

Line 387: `elif operation == VectorOperation.SURROGATE_GRADIENT:` This executes the following indented host-code branch only when ‘operation == VectorOperation.SURROGATE_GRADIENT‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; SURROGATE_GRADIENT: the signed loss derivative used to update a learned parameter. Context: the function vector_alu, which handles vector alu.

Line 388: `output = np.where(np.abs(a - threshold) <= surrogate_window, b, 0)` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are output: the value named ’output’ in this block; where: the value named ’where’ in this block; abs: the value named ’abs’ in this block.

Line 389: `elif operation == VectorOperation.MAXIMUM:` This executes the following indented host-code branch only when ‘operation == VectorOperation.MAXIMUM‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; MAXIMUM: the value named ’maximum’ in this block. Context: the function vector_alu, which handles vector alu.

Line 390: `output = np.maximum(a, b)` This assigns ‘np.maximum(a, b)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are maximum: the value named ’maximum’ in this block; a: the value named ’a’ in this block; b: the value named ’b’ in this block. Context: the function vector_alu, which handles vector alu.

Line 391: `elif operation == VectorOperation.MULTIPLY_Q15:` This executes the following indented host-code branch only when ‘operation == VectorOperation.MULTIPLY_Q15‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; MULTIPLY_Q15: the value named ’multiply q15’ in this block. Context: the function vector_alu, which handles vector alu.

Line 392: `output = saturate_signed(round_shift_signed(a * b, 15), 32)` This assigns ‘saturate_signed(round_shift_sig * b, 15), 32)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are saturate_signed: the value named ’saturate signed’ in this block; round_shift_signed: the value named ’round shift signed’ in this block; a: the value named ’a’ in this block. Context: the function vector_alu, which handles vector alu.

Line 393: `elif operation == VectorOperation.CLAMP:` This executes the following indented host-code branch only when ‘operation == VectorOperation.CLAMP‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; CLAMP: the value named ’clamp’ in this block. Context: the function vector_alu, which handles vector alu.

Line 394: `output = np.clip(a, clamp_minimum, clamp_maximum)` This assigns ‘np.clip(a, clamp_minimum, clamp_maximum to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are clip: the value named ’clip’ in this block; a: the value named ’a’ in this block; clamp_minimum: the value named ’clamp minimum’ in this block. Context: the function vector_alu, which handles vector alu.

Line 395: `elif operation == VectorOperation.SELECT_ZERO:` This executes the following indented host-code branch only when ‘operation == VectorOperation.SELECT_ZERO‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; SELECT_ZERO: the value named ’select zero’ in this block. Context: the function vector_alu, which handles vector alu.

Line 396: `output = np.where(a == 0, b, 0)` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are output: the value named ’output’ in this block; where: the value named ’where’ in this block; a: the value named ’a’ in this block.

Line 397: `elif operation == VectorOperation.SELECT_NONZERO:` This executes the following indented host-code branch only when ‘operation == VectorOperation.SELECT_NONZERO‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; SELECT_NONZERO: the value named ’select nonzero’ in this block. Context: the function vector_alu, which handles vector alu.

Line 398: `output = np.where(a != 0, b, 0)` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are output: the value named ’output’ in this block; where: the value named ’where’ in this block; a: the value named ’a’ in this block.

Line 399: `elif operation == VectorOperation.COMPARE_GE:` This executes the following indented host-code branch only when ‘operation == VectorOperation.COMPARE_GE‘ is true. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block; COMPARE_GE: the value named ’compare ge’ in this block. Context: the function vector_alu, which handles vector alu.

Line 400: `output = (a >= b).astype(np.int64)` Inside the function vector_alu, which handles vector alu, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are output: the value named ’output’ in this block; a: the value named ’a’ in this block; b: the value named ’b’ in this block.

Line 401: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function vector_alu, which handles vector alu.

Line 402: `raise ValueError(f"unsupported vector operation {operation}")` This rejects an invalid state by raising ‘ValueError(f"unsupported vector operation {operation}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 403: `return np.asarray(output, dtype=np.int64)` This returns ‘np.asarray(output, dtype=np.int64)‘ to the caller of the function vector_alu, which handles vector alu, ending this invocation. The important operands are asarray: the value named ’asarray’ in this block; output: the value named ’output’ in this block; dtype: the value named ’dtype’ in this block.

Line 406: `def choose_dataflow(` This starts function choose_dataflow. Its indented body performs choose dataflow as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 407: `rows: int,` Inside the function choose_dataflow, which handles choose dataflow, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are rows: one output-row position processed by the tensor tile.

Line 408: `columns: int,` Inside the function choose_dataflow, which handles choose dataflow, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are columns: one output-feature position processed in parallel.

Line 409: `reduction: int,` Inside the function choose_dataflow, which handles choose dataflow, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are reduction: the value named ’reduction’ in this block.

Line 410: `row_tile: int = 8,` This assigns ‘8,‘ to ‘row_tile‘ in host memory. Here row_tile means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function choose_dataflow, which handles choose dataflow.

Line 411: `column_tile: int = 16,` This assigns ‘16,‘ to ‘column_tile‘ in host memory. Here column_tile means one output-feature position processed in parallel. The expression is a literal or structural operator and has no hidden software state. Context: the function choose_dataflow, which handles choose dataflow.

Line 412: `) -> DataflowDecision:` Inside the function choose_dataflow, which handles choose dataflow, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are DataflowDecision: the value named ’dataflow decision’ in this block.

Line 413: `"""Choose the traversal with fewer estimated operand bytes."""` This is executable documentation attached to the function choose_dataflow, which handles choose dataflow. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 414: `if min(rows, columns, reduction, row_tile, column_tile) <= 0:` This executes the following indented host-code branch only when ‘min(rows, columns, reduction, row_tile, column_tile) <= 0‘ is true. The important operands are min: the value named ’min’ in this block; rows: one output-row position processed by the tensor tile; columns: one output-feature position processed in parallel. Context: the function choose_dataflow, which handles choose dataflow.

Line 415: `raise ValueError("matrix and tile dimensions must be positive")` This rejects an invalid state by raising ‘ValueError("matrix and tile dimensions must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 416: `row_tiles = ceil(rows / row_tile)` This assigns ‘ceil(rows / row_tile)‘ to ‘row_tiles ‘ in host memory. Here row_tiles means one output-row position processed by the tensor tile. The important operands are ceil: the value named ’ceil’ in this block; rows: one output-row position processed by the tensor tile; row_tile: one output-row position processed by the tensor tile. Context: the function choose_dataflow, which handles choose dataflow.

Line 417: `column_tiles = ceil(columns / column_tile)` This assigns ‘ceil(columns / column_tile)‘ to ‘column_tiles ‘ in host memory. Here column_tiles means one output-feature position processed in parallel. The important operands are ceil: the value named ’ceil’ in this block; columns: one output-feature position processed in parallel; column_tile: one output-feature position processed in parallel. Context: the function choose_dataflow, which handles choose dataflow.

Line 418: `input_tile_bytes = row_tile * reduction` This assigns ‘row_tile * reduction‘ to ‘input_tile_bytes ‘ in host memory. Here input_tile_bytes means the value named ’input tile bytes’ in this block. The important operands are row_tile: one output-row position processed by the tensor tile; reduction: the value named ’reduction’ in this block. Context: the function choose_dataflow, which handles choose dataflow.

Line 419: `weight_tile_bytes = column_tile * reduction` This assigns ‘column_tile * reduction‘ to ‘weight_tile_bytes ‘ in host memory. Here weight_tile_bytes means a learned coefficient multiplied by an activation. The important operands are column_tile: one output-feature position processed in parallel; reduction: the value named ’reduction’ in this block. Context: the function choose_dataflow, which handles choose dataflow.

Line 420: `input_broadcast_bytes = row_tiles * input_tile_bytes + row_tiles * column_tiles * weight_tile_bytes` This assigns ‘row_tiles * input_tile_bytes + row_tiles * column_tiles * weight_tile_bytes‘ to ‘input_broadcast_bytes ‘ in host memory. Here input_broadcast_bytes means the value named ’input broadcast bytes’ in this block. The important operands are row_tiles: one output-row position processed by the tensor tile; input_tile_bytes: the value named ’input tile bytes’ in this block; column_tiles: one output-feature position processed in parallel. Context: the function choose_dataflow, which handles choose dataflow.

Line 421: `weight_broadcast_bytes = column_tiles * weight_tile_bytes + column_tiles * row_tiles * input_tile_b` This assigns ‘column_tiles * weight_tile_bytes + column_tiles * row_tiles * input_tile_bytes‘ to ‘weight_broadcast_bytes ‘ in host memory. Here weight_broadcast_bytes means a learned coefficient multiplied by an activation. The important operands are column_tiles: one output-feature position processed in parallel; weight_tile_bytes: a learned coefficient multiplied by an activation; row_tiles: one output-row position processed by the tensor tile. Context: the function choose_dataflow, which handles choose dataflow.

Line 422: `mode = (` This statement continues through source line 426; the explanation uses the complete expression rather than only the first visual fragment. Inside the function choose_dataflow, which handles choose dataflow, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are mode: the value named ’mode’ in this block; Dataflow: the value named ’dataflow’ in this block; INPUT_BROADCAST: the value named ’input broadcast’ in this block.

Line 423: `Dataflow.INPUT_BROADCAST` This is continuation line 2 of the 5-line logical statement that begins at source line 422. Its exact fragment is ‘Dataflow.INPUT_BROADCAST‘. The important operands are Dataflow: the value named ’dataflow’ in this block; INPUT_BROADCAST: the value named ’input broadcast’ in this block. Read together, the complete operation is: Inside the function choose_dataflow, which handles choose dataflow, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are mode: the value named ’mode’ in this block; Dataflow: the value named ’dataflow’ in this block; INPUT_BROADCAST: the value named ’input broadcast’ in this block.

Line 424: `if input_broadcast_bytes <= weight_broadcast_bytes` This is continuation line 3 of the 5-line logical statement that begins at source line 422. Its exact fragment is ‘if input_broadcast_bytes <= weight_broadcast_bytes‘. The important operands are input_broadcast_bytes: the value named ’input broadcast bytes’ in this block; weight_broadcast_bytes a learned coefficient multiplied by an activation. Read together, the complete operation is: Inside the function choose_dataflow, which handles choose dataflow, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are mode: the value named ’mode’ in this block; Dataflow: the value named ’dataflow’ in this block; INPUT_BROADCAST: the value named ’input broadcast’ in this block.

Line 425: `else Dataflow.WEIGHT_BROADCAST` This is continuation line 4 of the 5-line logical statement that begins at source line 422. Its exact fragment is ‘else Dataflow.WEIGHT_BROADCAST‘. The important operands are Dataflow: the value named ’dataflow’ in this block; WEIGHT_BROADCAST: a learned coefficient multiplied by an activation. Read together, the complete operation is: Inside the function choose_dataflow, which handles choose dataflow, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are mode: the value named ’mode’ in this block; Dataflow: the value named ’dataflow’ in this block; INPUT_BROADCAST: the value named ’input broadcast’ in this block.

Line 426: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 422. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function choose_dataflow, which handles choose dataflow, this statement advances the host implementation that implements independent mathematical reference models for every hardware arithmetic primitive. The important operands are mode: the value named ’mode’ in this block; Dataflow: the value named ’dataflow’ in this block; INPUT_BROADCAST: the value named ’input broadcast’ in this block.

Line 427: `return DataflowDecision(mode, input_broadcast_bytes, weight_broadcast_bytes)` This returns ‘DataflowDecision(mode, input_broadcast_bytes, weight_broadcast_bytes)‘ to the caller of the function choose_dataflow, which handles choose dataflow, ending this invocation. The important operands are DataflowDecision: the value named ’dataflow decision’ in this block; mode: the value named ’mode’ in this block; input_broadcast_bytes: the value named ’input broadcast bytes’ in this block.

Line 430: `def peak_dense_gops(clock_hz: float, packed_dsp_pairs: int) -> float:` This starts function peak_dense_gops. Its indented body performs peak dense gops as part of the file that implements independent mathematical reference models for every hardware arithmetic primitive; it runs only when called.

Line 431: `"""Two operations per MAC and two MACs per packed DSP pair."""` This is executable documentation attached to the function peak_dense_gops, which handles peak dense gops. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 432: `return clock_hz * packed_dsp_pairs * 2 * 2 / 1e9` This returns ‘clock_hz * packed_dsp_pairs * 2 * 2 / 1e9‘ to the caller of the function peak_dense_gops, which handles peak dense gops, ending this invocation. The important operands are clock_hz: the value named ’clock hz’ in this block; packed_dsp_pairs: the value named ’packed dsp pairs’ in this block.

36 software/neuraedge_protocol.py

Role in the complete system: defines command bit fields, descriptor records, packed streams, and result decoders.

36.1 Beginner architecture map

Dataflow responsibility This file is the bit-level contract shared with RTL. It packs each 128-bit command field, declares exact payload/result lengths, wraps records in descriptor headers, and decodes signed results so a software integer has the same bit meaning as FPGA wires.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: Opcode, LutSuboperation, LinearGradientMode, StreamBeat, PackedStreamBeats, ProgramRecord, Command128. Functions: _unsigned, _signed_bits, _decode_signed, pack_descriptor_program, _put, clear_status_command, linear_backward_command, tensor_command, alpha_xnor_command, vector_command, layernorm_command, log_softmax_command, lif_command, sgd_command, lut_command, _finish_beats, pack_activation pack_outer_weight_beats, pack_dot8_weight_beats, pack_alpha_beats, pack_vector_beats, pack_layernorm_beats, pack_softmax_beats, pack_lif_beats. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

36.2 Complete source

```
"""Typed command and stream ABI for the NeuraEdge PYNQ accelerator.

The RTL command port is 128 bits wide and the data path is AXI4-Stream with
64-bit beats.  This module is deliberately independent of PYNQ and NumPy so it
can be used by fixture generators, desktop tests, and the board driver.
"""

from __future__ import annotations

from array import array
from dataclasses import dataclass
from enum import IntEnum
from sys import byteorder
from typing import Iterable, Iterator, Sequence, overload


class Opcode(IntEnum):
    TRAIN_LINEAR_BACKWARD = 0x0
    LOAD_ACTIVATIONS = 0x1
    LOAD_WEIGHTS_OUTER = 0x2
    LOAD_WEIGHTS_DOT8 = 0x3
    COMPUTE_DENSE_OUTER = 0x4
    COMPUTE_DENSE_DOT8 = 0x5
    COMPUTE_BINARY = 0x6
    COMPUTE_TERNARY = 0x7
    ALPHA_XNOR = 0x8
    VECTOR_ALU = 0x9
    LAYERNORM = 0xA
    LOG_SOFTMAX = 0xB
    LIF_STEP = 0xC
    SGD_UPDATE = 0xD
    ACTIVATION_LUT = 0xE
    CLEAR_STATUS = 0xF


class LutSuboperation(IntEnum):
    CLEAR = 0
    LOAD = 1
    EVALUATE = 2


class LinearGradientMode(IntEnum):
    BOTH = 0
    INPUT = 1
    WEIGHT = 2

```

```
@dataclass(frozen=True)
class StreamBeat:
    data: int
    last: bool = False

    def __post_init__(self) -> None:
        _unsigned("stream data", self.data, 64)


class PackedStreamBeats(Sequence[StreamBeat]):
    """Compact 64-bit payload storage with the ordinary StreamBeat interface."""

    def __init__(self, words: Iterable[int] = ()) -> None:
        self.words = array("Q", (_unsigned("stream word", word, 64) for word in words))
        if self.words.itemsize != 8:
            raise RuntimeError("platform unsigned long long is not 64 bits")

    @classmethod
    def from_little_endian_bytes(cls, payload: bytes) -> "PackedStreamBeats":
        if byteorder != "little":
            raise RuntimeError("compact stream payloads require a little-endian host")
        if len(payload) % 8:
            raise ValueError("packed stream byte length must be a multiple of eight")
        result = cls()
        result.words.frombytes(payload)
        return result

    def __len__(self) -> int:
        return len(self.words)

    @overload
    def __getitem__(self, index: int) -> StreamBeat: ...

    @overload
    def __getitem__(self, index: slice) -> tuple[StreamBeat, ...]: ...

    def __getitem__(self, index: int | slice) -> StreamBeat | tuple[StreamBeat, ...]:
        if isinstance(index, slice):
            indices = range(*index.indices(len(self)))
            return tuple(StreamBeat(self.words[item], item == len(self) - 1) for item in indices)
        normalized = index + len(self) if index < 0 else index
        if not 0 <= normalized < len(self):
            raise IndexError(index)
        return StreamBeat(self.words[normalized], normalized == len(self) - 1)

    def __iter__(self) -> Iterator[StreamBeat]:
        final = len(self) - 1
        return (StreamBeat(word, index == final) for index, word in enumerate(self.words))


@dataclass(frozen=True)
class ProgramRecord:
    """One descriptor-program command and its stream contract."""

    command: Command128
    input_beats: Sequence[StreamBeat] = ()
    output_beats: int = 0

    def __post_init__(self) -> None:
        _unsigned("program output beat count", self.output_beats, 31)
        _unsigned("program input beat count", len(self.input_beats), 32)
        if not isinstance(self.input_beats, PackedStreamBeats):
            for index, beat in enumerate(self.input_beats):
                if beat.last != (index == len(self.input_beats) - 1):
                    raise ValueError("record input TLAST must mark only its final payload beat")


@dataclass(frozen=True)
class Command128:
    value: int

    def __post_init__(self) -> None:
        _unsigned("command", self.value, 128)

    @property
    def opcode(self) -> Opcode:
        return Opcode(self.value & 0xF)

    @property
    def words_le(self) -> tuple[int, int, int, int]:
        return tuple((self.value >> (32 * index)) & 0xFFFFFFFF for index in range(4))

    @classmethod
    def from_words_le(cls, words: Sequence[int]) -> "Command128":
        if len(words) != 4:
            raise ValueError("a command requires exactly four 32-bit words")
```

```
        value = 0
        for index, word in enumerate(words):
            value |= _unsigned(f"command word {index}", word, 32) << (32 * index)
        return cls(value)


def _unsigned(name: str, value: int, bits: int, maximum: int | None = None) -> int:
    integer = int(value)
    limit = (1 << bits) - 1 if maximum is None else maximum
    if integer < 0 or integer > limit:
        raise ValueError(f"{name} must be in [0,{limit}]")
    return integer


def _signed_bits(name: str, value: int, bits: int) -> int:
    integer = int(value)
    minimum = -(1 << (bits - 1))
    maximum = (1 << (bits - 1)) - 1
    if integer < minimum or integer > maximum:
        raise ValueError(f"{name} must be in [{minimum},{maximum}]")
    return integer & ((1 << bits) - 1)


def _decode_signed(value: int, bits: int) -> int:
    masked = value & ((1 << bits) - 1)
    return masked - (1 << bits) if masked & (1 << (bits - 1)) else masked


def pack_descriptor_program(
    records: Sequence[ProgramRecord],
) -> tuple[tuple[StreamBeat, ...], int]:
    """Pack records into the descriptor sequencer's single-DMA stream."""

    if not records:
        raise ValueError("a descriptor program must contain at least one record")
    words: list[int] = []
    total_outputs = 0
    for index, record in enumerate(records):
        final = index == len(records) - 1
        header = len(record.input_beats)
        header |= record.output_beats << 32
        header |= int(final) << 63
        words.extend((
            header,
            record.command.value & ((1 << 64) - 1),
            record.command.value >> 64,
        ))
        words.extend(beat.data for beat in record.input_beats)
        total_outputs += record.output_beats
        _unsigned("program total output beat count", total_outputs, 32)
    beats = tuple(StreamBeat(word, item == len(words) - 1)
                  for item, word in enumerate(words))
    return beats, total_outputs


def _put(command: int, value: int, offset: int, bits: int) -> int:
    return command | ((value & ((1 << bits) - 1)) << offset)


def clear_status_command() -> Command128:
    return Command128(int(Opcode.CLEAR_STATUS))


def linear_backward_command(
    rows: int,
    input_features: int,
    output_features: int,
    *,
    input_gradient_shift: int = 0,
    weight_gradient_shift: int = 0,
    mode: LinearGradientMode = LinearGradientMode.BOTH,
    accumulate: bool = False,
    suppress_output: bool = False,
) -> Command128:
    """Configure one signed 8x8 linear-layer backward tile."""

    row_count = _unsigned("rows", rows, 4, 8)
    input_count = _unsigned("input_features", input_features, 4, 8)
    output_count = _unsigned("output_features", output_features, 4, 8)
    if row_count == 0 or input_count == 0 or output_count == 0:
        raise ValueError("linear backward dimensions must be non-zero")
    command = _put(int(Opcode.TRAIN_LINEAR_BACKWARD), row_count, 4, 4)
    command = _put(command, input_count, 8, 4)
    command = _put(command, output_count, 12, 4)
    command = _put(
        command,
        _unsigned("input_gradient_shift", input_gradient_shift, 5),
```

```
        16,
        5,
    )
    command = _put(
        command,
        _unsigned("weight_gradient_shift", weight_gradient_shift, 5),
        21,
        5,
    )
    try:
        selected_mode = LinearGradientMode(mode)
    except ValueError as error:
        raise ValueError("linear backward mode must be BOTH, INPUT, or WEIGHT") from error
    command = _put(command, int(selected_mode), 26, 2)
    command = _put(command, int(bool(accumulate)), 28, 1)
    command = _put(command, int(bool(suppress_output)), 29, 1)
    return Command128(command)


def tensor_command(
    opcode: Opcode,
    *,
    rows: int = 0,
    columns: int = 0,
    issue_count: int,
    activation_zero_point: int = 0,
    weight_zero_point: int = 0,
) -> Command128:
    tensor_opcodes = {
        Opcode.LOAD_ACTIVATIONS,
        Opcode.LOAD_WEIGHTS_OUTER,
        Opcode.LOAD_WEIGHTS_DOT8,
        Opcode.COMPUTE_DENSE_OUTER,
        Opcode.COMPUTE_DENSE_DOT8,
        Opcode.COMPUTE_BINARY,
        Opcode.COMPUTE_TERNARY,
    }
    if opcode not in tensor_opcodes:
        raise ValueError("tensor_command requires a tensor opcode")
    maximum_issues = 256 if opcode in {Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_DOT8} else 2048
    count = _unsigned("issue_count", issue_count, 12, maximum_issues)
    if count == 0:
        raise ValueError("issue_count must be non-zero")

    compute = opcode in {
        Opcode.COMPUTE_DENSE_OUTER,
        Opcode.COMPUTE_DENSE_DOT8,
        Opcode.COMPUTE_BINARY,
        Opcode.COMPUTE_TERNARY,
    }
    if compute:
        row_count = _unsigned("rows", rows, 4, 8)
        column_count = _unsigned("columns", columns, 6, 16)
        if row_count == 0 or column_count == 0:
            raise ValueError("compute rows and columns must be non-zero")
        if opcode == Opcode.COMPUTE_DENSE_DOT8 and row_count != 1:
            raise ValueError("dense Dot8 mode requires exactly one output row")
    else:
        row_count = _unsigned("rows", rows, 4, 8)
        column_count = _unsigned("columns", columns, 6, 16)

    command = int(opcode)
    command = _put(command, row_count, 4, 4)
    command = _put(command, column_count, 8, 6)
    command = _put(command, count, 14, 12)
    command = _put(command, _unsigned("activation_zero_point", activation_zero_point, 8), 26, 8)
    command = _put(command, _unsigned("weight_zero_point", weight_zero_point, 8), 34, 8)
    return Command128(command)


def alpha_xnor_command(active_bits: int, alpha_q8: int) -> Command128:
    active = _unsigned("active_bits", active_bits, 9, 256)
    alpha = _unsigned("alpha_q8", alpha_q8, 9, 256)
    if active == 0:
        raise ValueError("active_bits must be non-zero")
    command = _put(int(Opcode.ALPHA_XNOR), active, 42, 9)
    command = _put(command, alpha, 51, 9)
    return Command128(command)


def vector_command(
    operation: int,
    *,
    multiplier_q15: int = 0,
    right_shift: int = 0,
    output_zero_point: int = 0,
    threshold: int = 0,
```

```
    surrogate_window: int = 0,
) -> Command128:
    command = _put(int(Opcode.VECTOR_ALU), _unsigned("operation", operation, 4, 12), 4, 4)
    command = _put(command, _signed_bits("multiplier_q15", multiplier_q15, 16), 8, 16)
    if operation == 9:
        if right_shift == -16:
            raise ValueError("binary_point_shift must be in [-15,15]")
        shift_bits = _signed_bits("binary_point_shift", right_shift, 5)
    else:
        shift_bits = _unsigned("right_shift", right_shift, 5)
    command = _put(command, shift_bits, 24, 5)
    command = _put(command, _signed_bits("output_zero_point", output_zero_point, 16), 29, 16)
    command = _put(command, _signed_bits("threshold", threshold, 32), 45, 32)
    command = _put(command, _unsigned("surrogate_window", surrogate_window, 32), 77, 32)
    return Command128(command)


def layernorm_command(feature_count: int, epsilon: int = 1) -> Command128:
    count = _unsigned("feature_count", feature_count, 12, 2048)
    if count == 0:
        raise ValueError("feature_count must be non-zero")
    command = _put(int(Opcode.LAYERNORM), count, 4, 12)
    command = _put(command, _unsigned("epsilon", epsilon, 32), 16, 32)
    return Command128(command)


def log_softmax_command(item_count: int, difference_shift: int) -> Command128:
    count = _unsigned("item_count", item_count, 9, 256)
    if count == 0:
        raise ValueError("item_count must be non-zero")
    command = _put(int(Opcode.LOG_SOFTMAX), count, 4, 9)
    command = _put(command, _unsigned("difference_shift", difference_shift, 5), 13, 5)
    return Command128(command)


def lif_command(
    leak_shift: int,
    threshold: int,
    reset_value: int,
    *,
    clear_state: bool = False,
    trace_state: bool = False,
) -> Command128:
    command = _put(int(Opcode.LIF_STEP), _unsigned("leak_shift", leak_shift, 5), 4, 5)
    command = _put(command, _signed_bits("threshold", threshold, 24), 9, 24)
    command = _put(command, _signed_bits("reset_value", reset_value, 24), 33, 24)
    command = _put(command, int(bool(clear_state)), 57, 1)
    command = _put(command, int(bool(trace_state)), 58, 1)
    return Command128(command)


def sgd_command(
    learning_rate_q16: int,
    momentum_q16: int,
    weight_decay_q16: int,
    *,
    update_right_shift: int = 16,
) -> Command128:
    command = _put(int(Opcode.SGD_UPDATE), _unsigned("learning_rate_q16", learning_rate_q16, 18, 131071), 4, 18)
    command = _put(command, _unsigned("momentum_q16", momentum_q16, 17, 65536), 22, 17)
    command = _put(command, _unsigned("weight_decay_q16", weight_decay_q16, 17, 65536), 39, 17)
    command = _put(command, _unsigned("update_right_shift", update_right_shift, 5), 56, 5)
    return Command128(command)


def lut_command(suboperation: LutSuboperation, item_count: int = 0) -> Command128:
    count = _unsigned("item_count", item_count, 12, 256)
    if suboperation == LutSuboperation.LOAD and count == 0:
        raise ValueError("LUT load requires at least one entry")
    if suboperation == LutSuboperation.EVALUATE and count not in {0, 8}:
        raise ValueError("LUT evaluate processes exactly eight indices")
    if suboperation == LutSuboperation.CLEAR and count != 0:
        raise ValueError("LUT clear does not accept stream items")
    command = _put(int(Opcode.ACTIVATION_LUT), int(suboperation), 4, 4)
    command = _put(command, count, 8, 12)
    return Command128(command)


def _finish_beats(words: Iterable[int]) -> PackedStreamBeats:
    return PackedStreamBeats(words)


def pack_activation_beats(vectors: Sequence[Sequence[int]]) -> PackedStreamBeats:
    words = []
    for issue, vector in enumerate(vectors):
        if len(vector) != 8:
            raise ValueError(f"activation issue {issue} must contain eight bytes")
```

```
        word = sum(_unsigned(f"activation[{issue},{lane}]", value, 8) << (8 * lane)
                   for lane, value in enumerate(vector))
        words.append(word)
    return _finish_beats(words)


def pack_outer_weight_beats(weights: Sequence[Sequence[int]]) -> PackedStreamBeats:
    words = []
    for issue, vector in enumerate(weights):
        if len(vector) != 16:
            raise ValueError(f"outer weight issue {issue} must contain sixteen bytes")
        for half in range(2):
            words.append(sum(_unsigned(f"weight[{issue},{8 * half + lane}]", vector[8 * half + lane], 8)
                             << (8 * lane) for lane in range(8)))
    return _finish_beats(words)


def pack_dot8_weight_beats(weights: Sequence[Sequence[Sequence[int]]]) -> PackedStreamBeats:
    words = []
    for issue, matrix in enumerate(weights):
        if len(matrix) != 8 or any(len(row) != 16 for row in matrix):
            raise ValueError(f"Dot8 issue {issue} must have shape [8,16]")
        for reduction_lane, row in enumerate(matrix):
            for half in range(2):
                words.append(sum(_unsigned(
                    f"dot8_weight[{issue},{reduction_lane},{8 * half + lane}]",
                    row[8 * half + lane], 8) << (8 * lane) for lane in range(8)))
    return _finish_beats(words)


def pack_alpha_beats(query: Sequence[int], key: Sequence[int]) -> PackedStreamBeats:
    if len(query) != 256 or len(key) != 256:
        raise ValueError("alpha-XNOR query and key must each contain 256 bits")
    words = []
    for vector_name, vector in (("query", query), ("key", key)):
        for word_index in range(4):
            word = 0
            for bit_index in range(64):
                bit = _unsigned(f"{vector_name} bit", vector[word_index * 64 + bit_index], 1)
                word |= bit << bit_index
            words.append(word)
    return _finish_beats(words)


def pack_vector_beats(
    operand_a: Sequence[int],
    operand_b: Sequence[int],
    *,
    clamp_minimum: int = -(1 << 31),
    clamp_maximum: int = (1 << 31) - 1,
) -> PackedStreamBeats:
    if len(operand_a) != 8 or len(operand_b) != 8:
        raise ValueError("vector operands must each contain eight signed 32-bit lanes")
    words = [_signed_bits("clamp_minimum", clamp_minimum, 32)
             | (_signed_bits("clamp_maximum", clamp_maximum, 32) << 32)]
    for name, operand in (("operand_a", operand_a), ("operand_b", operand_b)):
        for lane in range(0, 8, 2):
            words.append(_signed_bits(f"{name}[{lane}]", operand[lane], 32)
                         | (_signed_bits(f"{name}[{lane + 1}]", operand[lane + 1], 32) << 32))
    return _finish_beats(words)


def pack_layernorm_beats(
    values: Sequence[int], gamma_q15: Sequence[int], beta: Sequence[int]
) -> PackedStreamBeats:
    if not (len(values) == len(gamma_q15) == len(beta)) or len(values) == 0:
        raise ValueError("LayerNorm value, gamma, and beta arrays must have equal non-zero length")
    words = []
    for index, (value, gamma, offset) in enumerate(zip(values, gamma_q15, beta)):
        beta_bits = _signed_bits(f"beta[{index}]", offset, 24)
        word0 = _signed_bits(f"value[{index}]", value, 24)
        word0 |= _signed_bits(f"gamma[{index}]", gamma, 24) << 24
        word0 |= (beta_bits & 0xFFFF) << 48
        words.extend((word0, beta_bits >> 16))
    return _finish_beats(words)


def pack_softmax_beats(scores: Sequence[int]) -> PackedStreamBeats:
    if not scores:
        raise ValueError("softmax requires at least one score")
    return _finish_beats(_signed_bits(f"score[{index}]", score, 24)
                         for index, score in enumerate(scores))


def pack_lif_beats(currents: Sequence[int]) -> PackedStreamBeats:
    if len(currents) != 32:
        raise ValueError("LIF current vector must contain 32 signed 16-bit lanes")
```

```
    words = []
    for lane in range(0, 32, 4):
        words.append(sum(_signed_bits(f"current[{lane + offset}]", currents[lane + offset], 16)
                         << (16 * offset) for offset in range(4)))
    return _finish_beats(words)


def pack_optimizer_beats(
    weights: Sequence[int], gradients: Sequence[int], velocities: Sequence[int]
) -> PackedStreamBeats:
    if not (len(weights) == len(gradients) == len(velocities) == 8):
        raise ValueError("optimizer vectors must each contain eight lanes")
    words = []
    for lane in range(8):
        word = _signed_bits(f"weight[{lane}]", weights[lane], 16)
        word |= _signed_bits(f"gradient[{lane}]", gradients[lane], 24) << 16
        word |= _signed_bits(f"velocity[{lane}]", velocities[lane], 24) << 40
        words.append(word)
    return _finish_beats(words)


def pack_linear_backward_beats(
    activations: Sequence[Sequence[int]],
    weights: Sequence[Sequence[int]],
    output_gradients: Sequence[Sequence[int]],
    *,
    mode: LinearGradientMode = LinearGradientMode.BOTH,
) -> PackedStreamBeats:
    """Pack signed X, W, and dY matrices for one linear backward tile."""

    rows = len(activations)
    input_features = len(weights)
    output_features = len(weights[0]) if input_features else 0
    if not 1 <= rows <= 8 or not 1 <= input_features <= 8 or not 1 <= output_features <= 8:
        raise ValueError("linear backward tile dimensions must be in [1,8]")
    if any(len(row) != input_features for row in activations):
        raise ValueError("activation rows must match the weight input dimension")
    if any(len(row) != output_features for row in weights):
        raise ValueError("weight rows must have one common output dimension")
    if len(output_gradients) != rows or any(
        len(row) != output_features for row in output_gradients
    ):
        raise ValueError("output gradients must have shape [rows, output_features]")

    try:
        selected_mode = LinearGradientMode(mode)
    except ValueError as error:
        raise ValueError("linear backward mode must be BOTH, INPUT, or WEIGHT") from error
    words: list[int] = []
    if selected_mode != LinearGradientMode.INPUT:
        for row_index, row in enumerate(activations):
            padded = list(row) + [0] * (8 - input_features)
            for half in range(2):
                words.append(sum(
                    _signed_bits(
                        f"activation[{row_index},{half * 4 + lane}]",
                        padded[half * 4 + lane],
                        16,
                    ) << (16 * lane)
                    for lane in range(4)
                ))
    if selected_mode != LinearGradientMode.WEIGHT:
        for row_index, row in enumerate(weights):
            padded = list(row) + [0] * (8 - output_features)
            for half in range(2):
                words.append(sum(
                    _signed_bits(
                        f"weight[{row_index},{half * 4 + lane}]",
                        padded[half * 4 + lane],
                        16,
                    ) << (16 * lane)
                    for lane in range(4)
                ))
    for row_index, row in enumerate(output_gradients):
        padded = list(row) + [0] * (8 - output_features)
        for half in range(2):
            words.append(sum(
                _signed_bits(
                    f"output_gradient[{row_index},{half * 4 + lane}]",
                    padded[half * 4 + lane],
                    16,
                ) << (16 * lane)
                for lane in range(4)
            ))
    return _finish_beats(words)


```

```
def pack_lut_load_beats(entries: Sequence[tuple[int, int]]) -> PackedStreamBeats:
    if not entries:
        raise ValueError("LUT load requires at least one entry")
    words = []
    for index, (address, value) in enumerate(entries):
        words.append(_unsigned(f"LUT address {index}", address, 8)
                     | (_signed_bits(f"LUT value {index}", value, 16) << 8))
    return _finish_beats(words)


def pack_lut_eval_beat(indices: Sequence[int]) -> PackedStreamBeats:
    if len(indices) != 8:
        raise ValueError("LUT evaluate requires exactly eight indices")
    word = sum(_unsigned(f"LUT index {lane}", value, 8) << (8 * lane)
               for lane, value in enumerate(indices))
    return [StreamBeat(word, True)]


def unpack_tensor_result(word: int) -> tuple[int, int, int, bool]:
    value = _unsigned("tensor result word", word, 64)
    return (_decode_signed(value, 48), (value >> 48) & 0xF,
            (value >> 52) & 0x3F, bool((value >> 58) & 1))


def unpack_alpha_result(word: int) -> tuple[int, int, int]:
    value = _unsigned("alpha result word", word, 64)
    return value & 0x3FFFF, (value >> 18) & 0x1FF, (value >> 27) & 0x1FF


def unpack_vector_result(words: Sequence[int]) -> tuple[int, ...]:
    if len(words) != 4:
        raise ValueError("vector result requires four words")
    return tuple(_decode_signed(word >> shift, 32)
                 for word in words for shift in (0, 32))


def unpack_layernorm_result(word: int) -> tuple[int, int, bool]:
    value = _unsigned("LayerNorm result word", word, 64)
    return _decode_signed(value, 32), (value >> 32) & 0xFFF, bool((value >> 44) & 1)


def unpack_softmax_result(word: int) -> tuple[int, int, bool]:
    value = _unsigned("softmax result word", word, 64)
    return value & 0xFFFF, (value >> 16) & 0x1FF, bool((value >> 25) & 1)


def unpack_lif_result(word: int) -> tuple[int, int]:
    value = _unsigned("LIF result word", word, 64)
    return value & 0xFFFFFFFF, value >> 32


def unpack_lif_trace(words: Sequence[int]) -> tuple[int, int, tuple[int, ...]]:
    """Decode one traced LIF response: metadata followed by 32 s24 membranes."""

    if len(words) != 13:
        raise ValueError("a traced LIF result requires exactly thirteen words")
    spikes, timestep = unpack_lif_result(words[0])
    packed_membranes = 0
    for index, word in enumerate(words[1:]):
        packed_membranes |= _unsigned(f"LIF trace word {index}", word, 64) << (64 * index)
    membranes = tuple(
        _decode_signed(packed_membranes >> (24 * lane), 24)
        for lane in range(32)
    )
    return spikes, timestep, membranes


def unpack_optimizer_result(words: Sequence[int]) -> tuple[tuple[int, ...], tuple[int, ...]]:
    if len(words) != 8:
        raise ValueError("optimizer result requires eight words")
    weights = tuple(_decode_signed(word, 16) for word in words)
    velocities = tuple(_decode_signed(word >> 16, 24) for word in words)
    return weights, velocities


def unpack_linear_backward_result(
    words: Sequence[int],
    rows: int,
    input_features: int,
    output_features: int,
    *,
    mode: LinearGradientMode = LinearGradientMode.BOTH,
) -> tuple[tuple[tuple[int, ...], ...], tuple[tuple[int, ...], ...]]:
    """Decode padded s32 dX rows and packed s24 dW rows from the RTL."""

    if not 1 <= rows <= 8 or not 1 <= input_features <= 8 or not 1 <= output_features <= 8:
        raise ValueError("linear backward tile dimensions must be in [1,8]")
```

```
    try:
        selected_mode = LinearGradientMode(mode)
    except ValueError as error:
        raise ValueError("linear backward mode must be BOTH, INPUT, or WEIGHT") from error
    expected_words = 0
    if selected_mode != LinearGradientMode.WEIGHT:
        expected_words += 4 * rows
    if selected_mode != LinearGradientMode.INPUT:
        expected_words += 3 * input_features
    if len(words) != expected_words:
        raise ValueError(
            f"linear backward result requires {expected_words} words; received {len(words)}"
        )
    input_gradients: list[tuple[int, ...]] = []
    cursor = 0
    if selected_mode != LinearGradientMode.WEIGHT:
        for _ in range(rows):
            lanes = tuple(
                _decode_signed(words[cursor + lane // 2] >> (32 * (lane % 2)), 32)
                for lane in range(input_features)
            )
            input_gradients.append(lanes)
            cursor += 4
    weight_gradients: list[tuple[int, ...]] = []
    if selected_mode != LinearGradientMode.INPUT:
        for _ in range(input_features):
            packed = sum(words[cursor + index] << (64 * index) for index in range(3))
            weight_gradients.append(tuple(
                _decode_signed(packed >> (24 * lane), 24)
                for lane in range(output_features)
            ))
            cursor += 3
    return tuple(input_gradients), tuple(weight_gradients)


def unpack_lut_result(words: Sequence[int]) -> tuple[int, ...]:
    if len(words) != 2:
        raise ValueError("LUT result requires two words")
    return tuple(_decode_signed(word >> (16 * lane), 16)
                 for word in words for lane in range(4))
```

36.3 Line annotations

Line 1: `"""Typed command and stream ABI for the NeuraEdge PYNQ accelerator.` This is executable documentation attached to the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `The RTL command port is 128 bits wide and the data path is AXI4-Stream with` Inside the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are The: the value named 'the' in this block; RTL: the value named 'rtl' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 4: `64-bit beats. This module is deliberately independent of PYNQ and NumPy so it` Inside the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are bit: the value named 'bit' in this block; beats: the value named 'beats' in this block; This: the value named 'this' in this block.

Line 5: `can be used by fixture generators, desktop tests, and the board driver.` Inside the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are can: the value named 'can' in this block; be: the value named 'be' in this block; used: the value named 'used' in this block.

Line 6: `"""` This is executable documentation attached to the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 8: `from __future__ import annotations` This imports the named Python API so the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `from array import array` This imports the named Python API so the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `from enum import IntEnum` This imports the named Python API so the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `from sys import byteorder` This imports the named Python API so the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `from typing import Iterable, Iterator, Sequence, overload` This imports the named Python API so the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `class Opcode(IntEnum):` This defines class Opcode, which groups data and methods for opcode. Instances exist in host memory and organize the flow that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 18: `TRAIN_LINEAR_BACKWARD = 0x0` This assigns ‘0x0‘ to ‘TRAIN_LINEAR_BACKWARD ‘ in host memory. Here TRAIN_LINEAR_BACKWARD means the value named ’train linear backward’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 19: `LOAD_ACTIVATIONS = 0x1` This assigns ‘0x1‘ to ‘LOAD_ACTIVATIONS ‘ in host memory. Here LOAD_ACTIVATION means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 20: `LOAD_WEIGHTS_OUTER = 0x2` This assigns ‘0x2‘ to ‘LOAD_WEIGHTS_OUTER ‘ in host memory. Here LOAD_WEIGHTS_OUTER means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 21: `LOAD_WEIGHTS_DOT8 = 0x3` This assigns ‘0x3‘ to ‘LOAD_WEIGHTS_DOT8 ‘ in host memory. Here LOAD_WEIGHTS_DOT8 means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 22: `COMPUTE_DENSE_OUTER = 0x4` This assigns ‘0x4‘ to ‘COMPUTE_DENSE_OUTER ‘ in host memory. Here COMPUTE_DENSE_OUTER means the value named ’compute dense outer’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 23: `COMPUTE_DENSE_DOT8 = 0x5` This assigns ‘0x5‘ to ‘COMPUTE_DENSE_DOT8 ‘ in host memory. Here COMPUTE_DENSE_DOT8 means the value named ’compute dense dot8’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 24: `COMPUTE_BINARY = 0x6` This assigns ‘0x6‘ to ‘COMPUTE_BINARY ‘ in host memory. Here COMPUTE_BINARY means the value named ’compute binary’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 25: `COMPUTE_TERNARY = 0x7` This assigns ‘0x7‘ to ‘COMPUTE_TERNARY ‘ in host memory. Here COMPUTE_TERNARY means the value named ’compute ternary’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 26: `ALPHA_XNOR = 0x8` This assigns ‘0x8‘ to ‘ALPHA_XNOR ‘ in host memory. Here ALPHA_XNOR means the programmable weight assigned to matching zero spikes in alpha-XNOR attention. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 27: `VECTOR_ALU = 0x9` This assigns ‘0x9‘ to ‘VECTOR_ALU ‘ in host memory. Here VECTOR_ALU means the value named ’vector alu’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 28: `LAYERNORM = 0xA` This assigns ‘0xA‘ to ‘LAYERNORM ‘ in host memory. Here LAYERNORM means the value named ’layernorm’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 29: `LOG_SOFTMAX = 0xB` This assigns ‘0xB‘ to ‘LOG_SOFTMAX ‘ in host memory. Here LOG_SOFTMAX means the value named ’log softmax’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 30: `LIF_STEP = 0xC` This assigns ‘0xC‘ to ‘LIF_STEP ‘ in host memory. Here LIF_STEP means the value named ’lif step’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 31: `SGD_UPDATE = 0xD` This assigns ‘0xD‘ to ‘SGD_UPDATE ‘ in host memory. Here SGD_UPDATE means the value named ’sgd update’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 32: `ACTIVATION_LUT = 0xE` This assigns ‘0xE‘ to ‘ACTIVATION_LUT ‘ in host memory. Here ACTIVATION_LUT means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 33: `CLEAR_STATUS = 0xF` This assigns ‘0xF‘ to ‘CLEAR_STATUS ‘ in host memory. Here CLEAR_STATUS means the value named ’clear status’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class Opcode, which handles opcode.

Line 36: `class LutSuboperation(IntEnum):` This defines class LutSuboperation, which groups data and methods for lut suboperation. Instances exist in host memory and organize the flow that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 37: `CLEAR = 0` This assigns ‘0‘ to ‘CLEAR ‘ in host memory. Here CLEAR means the value named ’clear’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class LutSuboperation, which handles lut suboperation.

Line 38: `LOAD = 1` This assigns ‘1‘ to ‘LOAD ‘ in host memory. Here LOAD means the value named ’load’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class LutSuboperation, which handles lut suboperation.

Line 39: `EVALUATE = 2` This assigns ‘2‘ to ‘EVALUATE ‘ in host memory. Here EVALUATE means the value named ’evaluate’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class LutSuboperation, which handles lut suboperation.

Line 42: `class LinearGradientMode(IntEnum):` This defines class LinearGradientMode, which groups data and methods for linear gradient mode. Instances exist in host memory and organize the flow that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 43: `BOTH = 0` This assigns ‘0‘ to ‘BOTH ‘ in host memory. Here BOTH means the value named ’both’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class LinearGradientMode, which handles linear gradient mode.

Line 44: `INPUT = 1` This assigns ‘1‘ to ‘INPUT ‘ in host memory. Here INPUT means the value named ’input’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class LinearGradientMode, which handles linear gradient mode.

Line 45: `WEIGHT = 2` This assigns ‘2‘ to ‘WEIGHT ‘ in host memory. Here WEIGHT means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the class LinearGradientMode, which handles linear gradient mode.

Line 48: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders; it creates no FPGA circuit by itself.

Line 49: `class StreamBeat:` This defines class StreamBeat, which groups data and methods for stream beat. Instances exist in host memory and organize the flow that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 50: `data: int` Inside the class StreamBeat, which handles stream beat, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are data: the value named ’data’ in this block.

Line 51: `last: bool = False` This assigns ‘False‘ to ‘last‘ in host memory. Here last means the AXI-Stream marker for the final beat of a packet or command payload. The expression is a literal or structural operator and has no hidden software state. Context: the class StreamBeat, which handles stream beat.

Line 53: `def __post_init__(self) -> None:` This starts function ___post_init___. Its indented body performs post init as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 54: `_unsigned("stream data", self.data, 64)` This calls ‘_unsigned‘ with ‘"stream data", self.data, 64‘ inside the function ___post_init___, which handles post init. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 57: `class PackedStreamBeats(Sequence[StreamBeat]):` This defines class PackedStreamBeats, which groups data and methods for packed stream beats. Instances exist in host memory and organize the flow that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 58: `"""Compact 64-bit payload storage with the ordinary StreamBeat interface."""` This is executable documentation attached to the class PackedStreamBeats, which handles packed stream beats. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 60: `def __init__(self, words: Iterable[int] = ()) -> None:` This starts function ___init___. Its indented body performs init as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 61: `self.words = array("Q", (_unsigned("stream word", word, 64) for word in words))` This assigns ‘array("Q", (_unsigned("stream word", word, 64) for word in words))‘ to ‘self.words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are array: the value named ’array’ in this block; Q: the value named ’q’ in this block; _unsigned: the value named ’unsigned’ in this block. Context: the function ___init___, which handles init.

Line 62: `if self.words.itemsize != 8:` This executes the following indented host-code branch only when ‘self.words.itemsize != 8‘ is true. The important operands are words: the value named ’words’ in this block; itemsize: the value named ’itemsize’ in this block. Context: the function ___init___, which handles init.

Line 63: `raise RuntimeError("platform unsigned long long is not 64 bits")` This rejects an invalid state by raising ‘RuntimeError("platform unsigned long long is not 64 bits")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 65: `@classmethod` This decorator changes how the following class or function is constructed. ‘@classmethod‘ is applied at import time on the host and is part of the class PackedStreamBeats, which handles packed stream beats; it creates no FPGA circuit by itself.

Line 66: `def from_little_endian_bytes(cls, payload: bytes) -> "PackedStreamBeats":` This starts function from_little_endian_bytes. Its indented body performs from little endian bytes as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 67: `if byteorder != "little":` This executes the following indented host-code branch only when ‘byteorder != "little"‘ is true. The important operands are byteorder: the value named ’byteorder’ in this block; little: the value named ’little’ in this block. Context: the function from_little_endian_bytes, which handles from little endian bytes.

Line 68: `raise RuntimeError("compact stream payloads require a little-endian host")` This rejects an invalid state by raising ‘RuntimeError("compact stream payloads require a little-endian host")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 69: `if len(payload) % 8:` This executes the following indented host-code branch only when ‘len(payload) % 8‘ is true. The important operands are len: the value named ’len’ in this block; payload: stream data belonging to the current command. Context: the function from_little_endian_bytes, which handles from little endian bytes.

Line 70: `raise ValueError("packed stream byte length must be a multiple of eight")` This rejects an invalid state by raising ‘ValueError("packed stream byte length must be a multiple of eight")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 71: `result = cls()` This assigns ‘cls()‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are cls: the value named ’cls’ in this block. Context: the function from_little_endian_bytes, which handles from little endian bytes.

Line 72: `result.words.frombytes(payload)` This calls ‘result.words.frombytes‘ with ‘payload‘ inside the function from_little_endian_bytes, which handles from little endian bytes. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 73: `return result` This returns ‘result‘ to the caller of the function from_little_endian_bytes, which handles from little endian bytes, ending this invocation. The important operands are result: the value named ’result’ in this block.

Line 75: `def __len__(self) -> int:` This starts function ___len___. Its indented body performs len as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 76: `return len(self.words)` This returns ‘len(self.words)‘ to the caller of the function ___len___, which handles len, ending this invocation. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block.

Line 78: `@overload` This decorator changes how the following class or function is constructed. ‘@overload‘ is applied at import time on the host and is part of the class PackedStreamBeats, which handles packed stream beats; it creates no FPGA circuit by itself.

Line 79: `def __getitem__(self, index: int) -> StreamBeat: ...` This starts function ___getitem___. Its indented body performs getitem as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 81: `@overload` This decorator changes how the following class or function is constructed. ‘@overload‘ is applied at import time on the host and is part of the class PackedStreamBeats, which handles packed stream beats; it creates no FPGA circuit by itself.

Line 82: `def __getitem__(self, index: slice) -> tuple[StreamBeat, ...]: ...` This starts function ___getitem___. Its indented body performs getitem as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 84: `def __getitem__(self, index: int | slice) -> StreamBeat | tuple[StreamBeat, ...]:` This starts function ___getitem___. Its indented body performs getitem as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 85: `if isinstance(index, slice):` This executes the following indented host-code branch only when ‘isinstance(index, slice)‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; slice: the value named ’slice’ in this block. Context: the function ___getitem___, which handles getitem.

Line 86: `indices = range(*index.indices(len(self)))` This assigns ‘range(*index.indices(len(self)))‘ to ‘indices ‘ in host memory. Here indices means the value named ’indices’ in this block. The important operands are range: the value named ’range’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; indices: the value named ’indices’ in this block. Context: the function ___getitem___, which handles getitem.

Line 87: `return tuple(StreamBeat(self.words[item], item == len(self) - 1) for item in indices)` This returns ‘tuple(StreamBeat(self.words[item], item == len(self) - 1) for item in indices)‘ to the caller of the function ___getitem___, which handles getitem, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; StreamBeat: the value named ’stream beat’ in this block; words: the value named ’words’ in this block.

Line 88: `normalized = index + len(self) if index < 0 else index` This assigns ‘index + len(self) if index $<$ 0 else index‘ to ‘normalized ‘ in host memory. Here normalized means the value named ’normalized’ in this block. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry; len: the value named ’len’ in this block. Context: the function ___getitem___, which handles getitem.

Line 89: `if not 0 <= normalized < len(self):` This executes the following indented host-code branch only when ‘not 0 $<=$ normalized $<$ len(self)‘ is true. The important operands are not: the value named ’not’ in this block; normalized: the value named ’normalized’ in this block; len: the value named ’len’ in this block. Context: the function ___getitem___, which handles getitem.

Line 90: `raise IndexError(index)` This rejects an invalid state by raising ‘IndexError(index)‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 91: `return StreamBeat(self.words[normalized], normalized == len(self) - 1)` This returns ‘StreamBeat(self.words[normalized], normalized == len(self) - 1)‘ to the caller of the function ___getitem___, which handles getitem, ending this invocation. The important operands are StreamBeat: the value named ’stream beat’ in this block; words: the value named ’words’ in this block; normalized: the value named ’normalized’ in this block.

Line 93: `def __iter__(self) -> Iterator[StreamBeat]:` This starts function ___iter___. Its indented body performs iter as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 94: `final = len(self) - 1` This assigns ‘len(self) - 1‘ to ‘final ‘ in host memory. Here final means the value named ’final’ in this block. The important operands are len: the value named ’len’ in this block. Context: the function ___iter___, which handles iter.

Line 95: `return (StreamBeat(word, index == final) for index, word in enumerate(self.words))` This returns ‘(StreamBeat(word, index == final) for index, word in enumerate(self.words))‘ to the caller of the function ___iter___, which handles iter, ending this invocation. The important operands are StreamBeat: the value named ’stream beat’ in this block; word: the value named ’word’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry.

Line 98: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders; it creates no FPGA circuit by itself.

Line 99: `class ProgramRecord:` This defines class ProgramRecord, which groups data and methods for program record. Instances exist in host memory and organize the flow that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 100: `"""One descriptor-program command and its stream contract."""` This is executable documentation attached to the class ProgramRecord, which handles program record. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 102: `command: Command128` Inside the class ProgramRecord, which handles program record, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 103: `input_beats: Sequence[StreamBeat] = ()` This assigns ‘()‘ to ‘input_beats‘ in host memory. Here input_beats means the value named ’input beats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class ProgramRecord, which handles program record.

Line 104: `output_beats: int = 0` This assigns ‘0‘ to ‘output_beats‘ in host memory. Here output_beats means the value named ’output beats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class ProgramRecord, which handles program record.

Line 106: `def __post_init__(self) -> None:` This starts function ___post_init___. Its indented body performs post init as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 107: `_unsigned("program output beat count", self.output_beats, 31)` This calls ‘_unsigned‘ with ‘"program output beat count", self.output_beats, 31‘ inside the function ___post_init___, which handles post init. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 108: `_unsigned("program input beat count", len(self.input_beats), 32)` This calls '_unsigned' with '"program input beat count", len(self.input_beats), 32' inside the function ___post_init___, which handles post init. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 109: `if not isinstance(self.input_beats, PackedStreamBeats):` This executes the following indented host-code branch only when 'not isinstance(self.input_beats, PackedStreamBeats)' is true. The important operands are not: the value named 'not' in this block; isinstance: the value named 'isinstance' in this block; input_beats: the value named 'input beats' in this block. Context: the function ___post_init___, which handles post init.

Line 110: `for index, beat in enumerate(self.input_beats):` This iterates index, beat over 'enumerate(self.input_beats)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function ___post_init___, which handles post init.

Line 111: `if beat.last != (index == len(self.input_beats) - 1):` This executes the following indented host-code branch only when 'beat.last != (index == len(self.input_beats) - 1)' is true. The important operands are beat: the value named 'beat' in this block; last: the AXI-Stream marker for the final beat of a packet or command payload; index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function ___post_init___, which handles post init.

Line 112: `raise ValueError("record input TLAST must mark only its final payload beat")` This rejects an invalid state by raising 'ValueError("record input TLAST must mark only its final payload beat")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 115: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to defines command bit fields, descriptor records, packed streams, and result decoders; it creates no FPGA circuit by itself.

Line 116: `class Command128:` This defines class Command128, which groups data and methods for command128. Instances exist in host memory and organize the flow that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 117: `value: int` Inside the class Command128, which handles command128, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are value: the value named 'value' in this block.

Line 119: `def __post_init__(self) -> None:` This starts function ___post_init___. Its indented body performs post init as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 120: `_unsigned("command", self.value, 128)` This calls '_unsigned' with '"command", self.value, 128' inside the function ___post_init___, which handles post init. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 122: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class Command128, which handles command128; it creates no FPGA circuit by itself.

Line 123: `def opcode(self) -> Opcode:` This starts function opcode. Its indented body performs opcode as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 124: `return Opcode(self.value & 0xF)` This returns 'Opcode(self.value & 0xF)' to the caller of the function opcode, which handles opcode, ending this invocation. The important operands are Opcode: the value named 'opcode' in this block; value: the value named 'value' in this block.

Line 126: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class Command128, which handles command128; it creates no FPGA circuit by itself.

Line 127: `def words_le(self) -> tuple[int, int, int, int]:` This starts function words_le. Its indented body performs words le as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 128: `return tuple((self.value >> (32 * index)) & 0xFFFFFFFF for index in range(4))` This returns ‘tuple((self.value >> (32 * index)) & 0xFFFFFFFF for index in range(4))‘ to the caller of the function words_le, which handles words le, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; value: the value named ’value’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry.

Line 130: `@classmethod` This decorator changes how the following class or function is constructed. ‘@classmethod‘ is applied at import time on the host and is part of the class Command128, which handles command128; it creates no FPGA circuit by itself.

Line 131: `def from_words_le(cls, words: Sequence[int]) -> "Command128":` This starts function from_words_le. Its indented body performs from words le as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 132: `if len(words) != 4:` This executes the following indented host-code branch only when ‘len(words) != 4‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block. Context: the function from_words_le, which handles from words le.

Line 133: `raise ValueError("a command requires exactly four 32-bit words")` This rejects an invalid state by raising ‘ValueError("a command requires exactly four 32-bit words")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 134: `value = 0` This assigns ‘0‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function from_words_le, which handles from words le.

Line 135: `for index, word in enumerate(words):` This iterates index, word over ‘enumerate(words)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function from_words_le, which handles from words le.

Line 136: `value |= _unsigned(f"command word {index}", word, 32) << (32 * index)` Inside the function from_words_le which handles from words le, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are value: the value named ’value’ in this block; _unsigned: the value named ’unsigned’ in this block; f: the value named ’f’ in this block.

Line 137: `return cls(value)` This returns ‘cls(value)‘ to the caller of the function from_words_le, which handles from words le, ending this invocation. The important operands are cls: the value named ’cls’ in this block; value: the value named ’value’ in this block.

Line 140: `def _unsigned(name: str, value: int, bits: int, maximum: int | None = None) -> int:` This starts function _unsigned. Its indented body performs unsigned as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 141: `integer = int(value)` This assigns ‘int(value)‘ to ‘integer ‘ in host memory. Here integer means the value named ’integer’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function _unsigned, which handles unsigned.

Line 142: `limit = (1 << bits) - 1 if maximum is None else maximum` This assigns ‘(1 << bits) - 1 if maximum is None else maximum‘ to ‘limit ‘ in host memory. Here limit means the value named ’limit’ in this block. The important operands are bits: the value named ’bits’ in this block; maximum: the value named ’maximum’ in this block; is: the value named ’is’ in this block. Context: the function _unsigned, which handles unsigned.

Line 143: `if integer < 0 or integer > limit:` This executes the following indented host-code branch only when ‘integer < 0 or integer > limit‘ is true. The important operands are integer: the value named ’integer’ in this block; or: the value named ’or’ in this block; limit: the value named ’limit’ in this block. Context: the function _unsigned, which handles unsigned.

Line 144: `raise ValueError(f"{name} must be in [0,{limit}]")` This rejects an invalid state by raising ‘ValueError(f"{name} must be in [0,{limit}]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 145: `return integer` This returns ‘integer‘ to the caller of the function _unsigned, which handles unsigned, ending this invocation. The important operands are integer: the value named ’integer’ in this block.

Line 148: `def _signed_bits(name: str, value: int, bits: int) -> int:` This starts function _signed_bits. Its indented body performs signed bits as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 149: `integer = int(value)` This assigns ‘int(value)‘ to ‘integer ‘ in host memory. Here integer means the value named ’integer’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function _signed_bits, which handles signed bits.

Line 150: `minimum = -(1 << (bits - 1))` This assigns ‘-(1 << (bits - 1))‘ to ‘minimum ‘ in host memory. Here minimum means the value named ’minimum’ in this block. The important operands are bits: the value named ’bits’ in this block. Context: the function _signed_bits, which handles signed bits.

Line 151: `maximum = (1 << (bits - 1)) - 1` This assigns ‘(1 << (bits - 1)) - 1‘ to ‘maximum ‘ in host memory. Here maximum means the value named ’maximum’ in this block. The important operands are bits: the value named ’bits’ in this block. Context: the function _signed_bits, which handles signed bits.

Line 152: `if integer < minimum or integer > maximum:` This executes the following indented host-code branch only when ‘integer < minimum or integer > maximum‘ is true. The important operands are integer: the value named ’integer’ in this block; minimum: the value named ’minimum’ in this block; or: the value named ’or’ in this block. Context: the function _signed_bits, which handles signed bits.

Line 153: `raise ValueError(f"{name} must be in [{minimum},{maximum}]")` This rejects an invalid state by raising ‘ValueError(f"{name} must be in [{minimum},{maximum}]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 154: `return integer & ((1 << bits) - 1)` This returns ‘integer & ((1 << bits) - 1)‘ to the caller of the function _signed_bits, which handles signed bits, ending this invocation. The important operands are integer: the value named ’integer’ in this block; bits: the value named ’bits’ in this block.

Line 157: `def _decode_signed(value: int, bits: int) -> int:` This starts function _decode_signed. Its indented body performs decode signed as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 158: `masked = value & ((1 << bits) - 1)` This assigns ‘value & ((1 << bits) - 1)‘ to ‘masked ‘ in host memory. Here masked means the value named ’masked’ in this block. The important operands are value: the value named ’value’ in this block; bits: the value named ’bits’ in this block. Context: the function _decode_signed, which handles decode signed.

Line 159: `return masked - (1 << bits) if masked & (1 << (bits - 1)) else masked` This returns ‘masked - (1 << bits) if masked & (1 << (bits - 1)) else masked‘ to the caller of the function _decode_signed, which handles decode signed, ending this invocation. The important operands are masked: the value named ’masked’ in this block; bits: the value named ’bits’ in this block.

Line 162: `def pack_descriptor_program(` This starts function pack_descriptor_program. Its indented body performs pack descriptor program as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 163: `records: Sequence[ProgramRecord],` Inside the function pack_descriptor_program, which handles pack descriptor program, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are records: the number of descriptor records accepted since program start; Sequence: the value named ’sequence’ in this block; ProgramRecord: the value named ’program record’ in this block.

Line 164: `) -> tuple[tuple[StreamBeat, ...], int]:` Inside the function pack_descriptor_program, which handles pack descriptor program, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are tuple: the value named ’tuple’ in this block; StreamBeat: the value named ’stream beat’ in this block.

Line 165: `"""Pack records into the descriptor sequencer’s single-DMA stream."""` This is executable documentation attached to the function pack_descriptor_program, which handles pack descriptor program. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 167: `if not records:` This executes the following indented host-code branch only when ‘not records‘ is true. The important operands are not: the value named ’not’ in this block; records: the number of descriptor records accepted since program start. Context: the function pack_descriptor_program, which handles pack descriptor program.

Line 168: `raise ValueError("a descriptor program must contain at least one record")` This rejects an invalid state by raising ‘ValueError("a descriptor program must contain at least one record")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 169: `words: list[int] = []` This assigns ‘[]‘ to ‘words‘ in host memory. Here words means the value named ’words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_descriptor_program, which handles pack descriptor program.

Line 170: `total_outputs = 0` This assigns ‘0‘ to ‘total_outputs ‘ in host memory. Here total_outputs means the value named ’total outputs’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_descriptor_program, which handles pack descriptor program.

Line 171: `for index, record in enumerate(records):` This iterates index, record over ‘enumerate(records)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_descriptor_program, which handles pack descriptor program.

Line 172: `final = index == len(records) - 1` Inside the function pack_descriptor_program, which handles pack descriptor program, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are final: the value named ’final’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; len: the value named ’len’ in this block.

Line 173: `header = len(record.input_beats)` This assigns ‘len(record.input_beats)‘ to ‘header ‘ in host memory. Here header means the value named ’header’ in this block. The important operands are len: the value named ’len’ in this block; record: the value named ’record’ in this block; input_beats: the value named ’input beats’ in this block. Context: the function pack_descriptor_program, which handles pack descriptor program.

Line 174: `header |= record.output_beats << 32` Inside the function pack_descriptor_program, which handles pack descriptor program, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are header: the value named 'header' in this block; record: the value named 'record' in this block; output_beats: the value named 'output beats' in this block.

Line 175: `header |= int(final) << 63` Inside the function pack_descriptor_program, which handles pack descriptor program, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are header: the value named 'header' in this block; final: the value named 'final' in this block.

Line 176: `words.extend((` This statement continues through source line 180; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘words.extend‘ with ‘(header, record.command.value & ((1 << 64) - 1), record.command.value >> 64,)‘ inside the function pack_descriptor_program, which handles pack descriptor program. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 177: `header,` This is continuation line 2 of the 5-line logical statement that begins at source line 176. Its exact fragment is ‘header,‘. The important operands are header: the value named 'header' in this block. Read together, the complete operation is: This calls ‘words.extend‘ with ‘(header, record.command.value & ((1 << 64) - 1), record.command.value >> 64,)‘ inside the function pack_descriptor_program, which handles pack descriptor program. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 178: `record.command.value & ((1 << 64) - 1),` This is continuation line 3 of the 5-line logical statement that begins at source line 176. Its exact fragment is ‘record.command.value & ((1 << 64) - 1),‘. The important operands are record: the value named 'record' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Read together, the complete operation is: This calls ‘words.extend‘ with ‘(header, record.command.value & ((1 << 64) - 1), record.command.value >> 64,)‘ inside the function pack_descriptor_program, which handles pack descriptor program. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 179: `record.command.value >> 64,` This is continuation line 4 of the 5-line logical statement that begins at source line 176. Its exact fragment is ‘record.command.value >> 64,‘. The important operands are record: the value named 'record' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Read together, the complete operation is: This calls ‘words.extend‘ with ‘(header, record.command.value & ((1 << 64) - 1), record.command.value >> 64,)‘ inside the function pack_descriptor_program, which handles pack descriptor program. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 180: `))` This is continuation line 5 of the 5-line logical statement that begins at source line 176. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘words.extend‘ with ‘(header, record.command.value & ((1 << 64) - 1), record.command.value >> 64,)‘ inside the function pack_descriptor_program, which handles pack descriptor program. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 181: `words.extend(beat.data for beat in record.input_beats)` This calls ‘words.extend‘ with ‘beat.data for beat in record.input_beats‘ inside the function pack_descriptor_program, which handles pack descriptor program. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 182: `total_outputs += record.output_beats` Inside the function pack_descriptor_program, which handles pack descriptor program, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are total_outputs: the value named 'total outputs' in this block; record: the value named 'record' in this block; output_beats: the value named 'output beats' in this block.

Line 183: `_unsigned("program total output beat count", total_outputs, 32)` This calls ‘_unsigned‘ with ‘"program total output beat count", total_outputs, 32‘ inside the function pack_descriptor_program, which handles pack descriptor program. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 184: `beats = tuple(StreamBeat(word, item == len(words) - 1)` This statement continues through source line 185; the explanation uses the complete expression rather than only the first visual fragment. Inside the function pack_descriptor_program, which handles pack descriptor program, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are beats: the value named ’beats’ in this block; tuple: the value named ’tuple’ in this block; StreamBeat: the value named ’stream beat’ in this block.

Line 185: `for item, word in enumerate(words))` This is continuation line 2 of the 2-line logical statement that begins at source line 184. Its exact fragment is ‘for item, word in enumerate(words))‘. The important operands are for: the value named ’for’ in this block; item: the value named ’item’ in this block; word: the value named ’word’ in this block. Read together, the complete operation is: Inside the function pack_descriptor_program, which handles pack descriptor program, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are beats: the value named ’beats’ in this block; tuple: the value named ’tuple’ in this block; StreamBeat: the value named ’stream beat’ in this block.

Line 186: `return beats, total_outputs` This returns ‘beats, total_outputs‘ to the caller of the function pack_descriptor_prog which handles pack descriptor program, ending this invocation. The important operands are beats: the value named ’beats’ in this block; total_outputs: the value named ’total outputs’ in this block.

Line 189: `def _put(command: int, value: int, offset: int, bits: int) -> int:` This starts function _put. Its indented body performs put as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 190: `return command | ((value & ((1 << bits) - 1)) << offset)` This returns ‘command | ((value & ((1 << bits) - 1)) << offset)‘ to the caller of the function _put, which handles put, ending this invocation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block; bits: the value named ’bits’ in this block.

Line 193: `def clear_status_command() -> Command128:` This starts function clear_status_command. Its indented body performs clear status command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 194: `return Command128(int(Opcode.CLEAR_STATUS))` This returns ‘Command128(int(Opcode.CLEAR_STATUS))‘ to the caller of the function clear_status_command, which handles clear status command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named ’opcode’ in this block; CLEAR_STATUS: the value named ’clear status’ in this block.

Line 197: `def linear_backward_command(` This starts function linear_backward_command. Its indented body performs linear backward command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 198: `rows: int,` Inside the function linear_backward_command, which handles linear backward command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are rows: one output-row position processed by the tensor tile.

Line 199: `input_features: int,` Inside the function linear_backward_command, which handles linear backward command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are input_features: the value named ’input features’ in this block.

Line 200: `output_features: int,` Inside the function linear_backward_command, which handles linear backward command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are output_features: the value named 'output features' in this block.

Line 201: `*,` Inside the function linear_backward_command, which handles linear backward command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The expression is a literal or structural operator and has no hidden software state.

Line 202: `input_gradient_shift: int = 0,` This assigns '0,' to 'input_gradient_shift' in host memory. Here input_gradient_shift means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function linear_backward_command, which handles linear backward command.

Line 203: `weight_gradient_shift: int = 0,` This assigns '0,' to 'weight_gradient_shift' in host memory. Here weight_gradient_shift means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function linear_backward_command, which handles linear backward command.

Line 204: `mode: LinearGradientMode = LinearGradientMode.BOTH,` This assigns 'LinearGradientMode.BOTH,' to 'mode' in host memory. Here mode means the value named 'mode' in this block. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter; BOTH: the value named 'both' in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 205: `accumulate: bool = False,` This assigns 'False,' to 'accumulate' in host memory. Here accumulate means the value named 'accumulate' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function linear_backward_command, which handles linear backward command.

Line 206: `suppress_output: bool = False,` This assigns 'False,' to 'suppress_output' in host memory. Here suppress_output means the value named 'suppress output' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function linear_backward_command, which handles linear backward command.

Line 207: `) -> Command128:` Inside the function linear_backward_command, which handles linear backward command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 208: `"""Configure one signed 8x8 linear-layer backward tile."""` This is executable documentation attached to the function linear_backward_command, which handles linear backward command. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 210: `row_count = _unsigned("rows", rows, 4, 8)` This assigns '_unsigned("rows", rows, 4, 8)' to 'row_count ' in host memory. Here row_count means one output-row position processed by the tensor tile. The important operands are _unsigned: the value named 'unsigned' in this block; rows: one output-row position processed by the tensor tile. Context: the function linear_backward_command, which handles linear backward command.

Line 211: `input_count = _unsigned("input_features", input_features, 4, 8)` This assigns '_unsigned("input_features" input_features, 4, 8)' to 'input_count ' in host memory. Here input_count means an integer count retained for control or measurable telemetry. The important operands are _unsigned: the value named 'unsigned' in this block; input_features: the value named 'input features' in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 212: `output_count = _unsigned("output_features", output_features, 4, 8)` This assigns '_unsigned("output_feat output_features, 4, 8)' to 'output_count ' in host memory. Here output_count means an integer count retained for control or measurable telemetry. The important operands are _unsigned: the value named 'unsigned' in this block; output_features: the value named 'output features' in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 213: `if row_count == 0 or input_count == 0 or output_count == 0:` This executes the following indented host-code branch only when 'row_count == 0 or input_count == 0 or output_count == 0' is true. The important operands are row_count: one output-row position processed by the tensor tile; or: the value named 'or' in this block; input_count: an integer count retained for control or measurable telemetry. Context: the function linear_backward_command, which handles linear backward command.

Line 214: `raise ValueError("linear backward dimensions must be non-zero")` This rejects an invalid state by raising 'ValueError("linear backward dimensions must be non-zero")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 215: `command = _put(int(Opcode.TRAIN_LINEAR_BACKWARD), row_count, 4, 4)` This assigns '_put(int(Opcode.TRAI row_count, 4, 4)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; Opcode: the value named 'opcode' in this block; TRAIN_LINEAR_BACKWARD: the value named 'train linear backward' in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 216: `command = _put(command, input_count, 8, 4)` This assigns '_put(command, input_count, 8, 4)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_count: an integer count retained for control or measurable telemetry. Context: the function linear_backward_command, which handles linear backward command.

Line 217: `command = _put(command, output_count, 12, 4)` This assigns '_put(command, output_count, 12, 4)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; output_count: an integer count retained for control or measurable telemetry. Context: the function linear_backward_command, which handles linear backward command.

Line 218: `command = _put(` This statement continues through source line 223; the explanation uses the complete expression rather than only the first visual fragment. This assigns '_put(command, _unsigned("input_gradient_shift", input_gradient_shift, 5), 16, 5,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named 'unsigned' in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 219: `command,` This is continuation line 2 of the 6-line logical statement that begins at source line 218. Its exact fragment is 'command,'. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns '_put(command, _unsigned("input_gradient_shift", input_gradient_shift, 5), 16, 5,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named 'unsigned' in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 220: `_unsigned("input_gradient_shift", input_gradient_shift, 5),` This is continuation line 3 of the 6-line logical statement that begins at source line 218. Its exact fragment is '_unsigned("input_gradient_shift", input_gradient_shift, 5),'. The important operands are _unsigned: the value named 'unsigned' in this block; input_gradient_shift:

the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘_put(command, _unsigned("input_gradient_shift", input_gradient_shift, 5), 16, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 221: `16,` This is continuation line 4 of the 6-line logical statement that begins at source line 218. Its exact fragment is ‘16,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_put(command, _unsigned("input_gradient_shift", input_gradient_shift, 5), 16, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 222: `5,` This is continuation line 5 of the 6-line logical statement that begins at source line 218. Its exact fragment is ‘5,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_put(command, _unsigned("input_gradient_shift", input_gradient_shift, 5), 16, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 223: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 218. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_put(command, _unsigned("input_gradient_shift", input_gradient_shift, 5), 16, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 224: `command = _put(` This statement continues through source line 229; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_put(command, _unsigned("weight_gradient_shift", weight_gradient_shift, 5), 21, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 225: `command,` This is continuation line 2 of the 6-line logical statement that begins at source line 224. Its exact fragment is ‘command,‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘_put(command, _unsigned("weight_gradient_shift", weight_gradient_shift, 5), 21, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 226: `_unsigned("weight_gradient_shift", weight_gradient_shift, 5),` This is continuation line 3 of the 6-line logical statement that begins at source line 224. Its exact fragment is ‘_unsigned("weight_gradient_shift", weight_gradient_shift, 5),‘. The important operands are _unsigned: the value named ’unsigned’ in this block; weight_gradient_shift a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘_put(command, _unsigned("weight_gradient_shift", weight_gradient_shift, 5), 21, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 227: `21,` This is continuation line 4 of the 6-line logical statement that begins at source line 224. Its exact fragment is ‘21,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_put(command, _unsigned("weight_gradient_shift", weight_gradient_shift, 5), 21, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 228: `5,` This is continuation line 5 of the 6-line logical statement that begins at source line 224. Its exact fragment is ‘5,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_put(command, _unsigned("weight_gradient_shift", weight_gradient_shift, 5), 21, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 229: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 224. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_put(command, _unsigned("weight_gradient_shift", weight_gradient_shift, 5), 21, 5,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 230: `try:` This is explicit host-side failure handling inside the function linear_backward_command, which handles linear backward command; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 231: `selected_mode = LinearGradientMode(mode)` This assigns ‘LinearGradientMode(mode)‘ to ‘selected_mode ‘ in host memory. Here selected_mode means the value named ’selected mode’ in this block. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter; mode: the value named ’mode’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 232: `except ValueError as error:` This is explicit host-side failure handling inside the function linear_backward_comma which handles linear backward command; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 233: `raise ValueError("linear backward mode must be BOTH, INPUT, or WEIGHT") from error` This rejects an invalid state by raising ‘ValueError("linear backward mode must be BOTH, INPUT, or WEIGHT") from error‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 234: `command = _put(command, int(selected_mode), 26, 2)` This assigns ‘_put(command, int(selected_mode), 26, 2)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; selected_mode: the value named ’selected mode’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 235: `command = _put(command, int(bool(accumulate)), 28, 1)` This assigns ‘_put(command, int(bool(accumulate)), 28, 1)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; bool: the value named ’bool’ in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 236: `command = _put(command, int(bool(suppress_output)), 29, 1)` This assigns '_put(command, int(bool(suppress 29, 1)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; bool: the value named 'bool' in this block. Context: the function linear_backward_command, which handles linear backward command.

Line 237: `return Command128(command)` This returns 'Command128(command)' to the caller of the function linear_backward_command, which handles linear backward command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 240: `def tensor_command(` This starts function tensor_command. Its indented body performs tensor command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 241: `opcode: Opcode,` Inside the function tensor_command, which handles tensor command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are opcode: the value named 'opcode' in this block; Opcode: the value named 'opcode' in this block.

Line 242: `*,` Inside the function tensor_command, which handles tensor command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The expression is a literal or structural operator and has no hidden software state.

Line 243: `rows: int = 0,` This assigns '0,' to 'rows' in host memory. Here rows means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function tensor_command, which handles tensor command.

Line 244: `columns: int = 0,` This assigns '0,' to 'columns' in host memory. Here columns means one output-feature position processed in parallel. The expression is a literal or structural operator and has no hidden software state. Context: the function tensor_command, which handles tensor command.

Line 245: `issue_count: int,` Inside the function tensor_command, which handles tensor command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are issue_count: an integer count retained for control or measurable telemetry.

Line 246: `activation_zero_point: int = 0,` This assigns '0,' to 'activation_zero_point' in host memory. Here activation_zero_point means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the function tensor_command, which handles tensor command.

Line 247: `weight_zero_point: int = 0,` This assigns '0,' to 'weight_zero_point' in host memory. Here weight_zero_point means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function tensor_command, which handles tensor command.

Line 248: `) -> Command128:` Inside the function tensor_command, which handles tensor command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 249: `tensor_opcodes = {` This statement continues through source line 257; the explanation uses the complete expression rather than only the first visual fragment. This assigns '{ Opcode.LOAD_ACTIVATIONS, Opcode.LOAD_WEIGHTS_OUTER, Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }' to 'tensor_opcodes

‘ in host memory. Here tensor_opcodes means the value named 'tensor opcodes' in this block. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function tensor_command, which handles tensor command.

Line 250: `Opcode.LOAD_ACTIVATIONS,` This is continuation line 2 of the 9-line logical statement that begins at source line 249. Its exact fragment is ‘Opcode.LOAD_ACTIVATIONS,‘. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘{ Opcode.LOAD_ACTIVATIONS, Opcode.LOAD_WEIGHTS_OUTER, Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘tensor_opcodes ‘ in host memory. Here tensor_opcodes means the value named 'tensor opcodes' in this block. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function tensor_command, which handles tensor command.

Line 251: `Opcode.LOAD_WEIGHTS_OUTER,` This is continuation line 3 of the 9-line logical statement that begins at source line 249. Its exact fragment is ‘Opcode.LOAD_WEIGHTS_OUTER,‘. The important operands are Opcode: the value named 'opcode' in this block; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘{ Opcode.LOAD_ACTIVATIONS, Opcode.LOAD_WEIGHTS_OUTER, Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘tensor_opcodes ‘ in host memory. Here tensor_opcodes means the value named 'tensor opcodes' in this block. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function tensor_command, which handles tensor command.

Line 252: `Opcode.LOAD_WEIGHTS_DOT8,` This is continuation line 4 of the 9-line logical statement that begins at source line 249. Its exact fragment is ‘Opcode.LOAD_WEIGHTS_DOT8,‘. The important operands are Opcode: the value named 'opcode' in this block; LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘{ Opcode.LOAD_ACTIVATIONS, Opcode.LOAD_WEIGHTS_OUTER, Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘tensor_opcodes ‘ in host memory. Here tensor_opcodes means the value named 'tensor opcodes' in this block. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function tensor_command, which handles tensor command.

Line 253: `Opcode.COMPUTE_DENSE_OUTER,` This is continuation line 5 of the 9-line logical statement that begins at source line 249. Its exact fragment is ‘Opcode.COMPUTE_DENSE_OUTER,‘. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block. Read together, the complete operation is: This assigns ‘{ Opcode.LOAD_ACTIVATIONS, Opcode.LOAD_WEIGHTS_OUTER, Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘tensor_opcodes ‘ in host memory. Here tensor_opcodes means the value named 'tensor opcodes' in this block. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function tensor_command, which handles tensor command.

Line 254: `Opcode.COMPUTE_DENSE_DOT8,` This is continuation line 6 of the 9-line logical statement that begins at source line 249. Its exact fragment is ‘Opcode.COMPUTE_DENSE_DOT8,‘. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_DOT8: the value named 'compute dense dot8' in this block. Read together, the complete operation is: This assigns ‘{ Opcode.LOAD_ACTIVATIONS, Opcode.LOAD_WEIGHTS_OUTER, Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘tensor_opcodes ‘ in host memory. Here tensor_opcodes means the value named 'tensor opcodes' in this block. The important operands are Opcode: the value

named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function tensor_command, which handles tensor command.

Line 255: `Opcode.COMPUTE_BINARY,` This is continuation line 7 of the 9-line logical statement that begins at source line 249. Its exact fragment is 'Opcode.COMPUTE_BINARY,'. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_BINARY: the value named 'compute binary' in this block. Read together, the complete operation is: This assigns '{ Opcode.LOAD_ACTIVATIONS, Opcode.LOAD_WEIGHTS_OUTER, Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }' to 'tensor_opcodes ' in host memory. Here tensor_opcodes means the value named 'tensor opcodes' in this block. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function tensor_command, which handles tensor command.

Line 256: `Opcode.COMPUTE_TERNARY,` This is continuation line 8 of the 9-line logical statement that begins at source line 249. Its exact fragment is 'Opcode.COMPUTE_TERNARY,'. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_TERNARY: the value named 'compute ternary' in this block. Read together, the complete operation is: This assigns '{ Opcode.LOAD_ACTIVATIONS, Opcode.LOAD_WEIGHTS_OUTER, Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }' to 'tensor_opcodes ' in host memory. Here tensor_opcodes means the value named 'tensor opcodes' in this block. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function tensor_command, which handles tensor command.

Line 257: `}` This is continuation line 9 of the 9-line logical statement that begins at source line 249. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ Opcode.LOAD_ACTIVATIONS, Opcode.LOAD_WEIGHTS_OUTER, Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }' to 'tensor_opcodes ' in host memory. Here tensor_opcodes means the value named 'tensor opcodes' in this block. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function tensor_command, which handles tensor command.

Line 258: `if opcode not in tensor_opcodes:` This executes the following indented host-code branch only when 'opcode not in tensor_opcodes' is true. The important operands are opcode: the value named 'opcode' in this block; not: the value named 'not' in this block; in: the value named 'in' in this block. Context: the function tensor_command, which handles tensor command.

Line 259: `raise ValueError("tensor_command requires a tensor opcode")` This rejects an invalid state by raising 'ValueError("tensor_command requires a tensor opcode")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 260: `maximum_issues = 256 if opcode in {Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_DOT8} else 2048` This assigns '256 if opcode in {Opcode.LOAD_WEIGHTS_DOT8, Opcode.COMPUTE_DENSE_DOT8} else 2048' to 'maximum_issues ' in host memory. Here maximum_issues means the value named 'maximum issues' in this block. The important operands are opcode: the value named 'opcode' in this block; in: the value named 'in' in this block; Opcode: the value named 'opcode' in this block. Context: the function tensor_command, which handles tensor command.

Line 261: `count = _unsigned("issue_count", issue_count, 12, maximum_issues)` This assigns '_unsigned("issue_count", issue_count, 12, maximum_issues)' to 'count ' in host memory. Here count means the value named 'count' in this block. The important operands are _unsigned: the value named 'unsigned' in this block; issue_count: an integer count retained

for control or measurable telemetry; maximum_issues: the value named 'maximum issues' in this block. Context: the function tensor_command, which handles tensor command.

Line 262: `if count == 0:` This executes the following indented host-code branch only when ‘count == 0‘ is true. The important operands are count: the value named 'count' in this block. Context: the function tensor_command, which handles tensor command.

Line 263: `raise ValueError("issue_count must be non-zero")` This rejects an invalid state by raising ‘ValueError("issue_count must be non-zero")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 265: `compute = opcode in {` This statement continues through source line 270; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘opcode in { Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘compute ‘ in host memory. Here compute means the value named 'compute' in this block. The important operands are opcode: the value named 'opcode' in this block; in: the value named 'in' in this block; Opcode: the value named 'opcode' in this block. Context: the function tensor_command, which handles tensor command.

Line 266: `Opcode.COMPUTE_DENSE_OUTER,` This is continuation line 2 of the 6-line logical statement that begins at source line 265. Its exact fragment is ‘Opcode.COMPUTE_DENSE_OUTER,‘. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block. Read together, the complete operation is: This assigns ‘opcode in { Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘compute ‘ in host memory. Here compute means the value named 'compute' in this block. The important operands are opcode: the value named 'opcode' in this block; in: the value named 'in' in this block; Opcode: the value named 'opcode' in this block. Context: the function tensor_command, which handles tensor command.

Line 267: `Opcode.COMPUTE_DENSE_DOT8,` This is continuation line 3 of the 6-line logical statement that begins at source line 265. Its exact fragment is ‘Opcode.COMPUTE_DENSE_DOT8,‘. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_DOT8: the value named 'compute dense dot8' in this block. Read together, the complete operation is: This assigns ‘opcode in { Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘compute ‘ in host memory. Here compute means the value named 'compute' in this block. The important operands are opcode: the value named 'opcode' in this block; in: the value named 'in' in this block; Opcode: the value named 'opcode' in this block. Context: the function tensor_command, which handles tensor command.

Line 268: `Opcode.COMPUTE_BINARY,` This is continuation line 4 of the 6-line logical statement that begins at source line 265. Its exact fragment is ‘Opcode.COMPUTE_BINARY,‘. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_BINARY: the value named 'compute binary' in this block. Read together, the complete operation is: This assigns ‘opcode in { Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘compute ‘ in host memory. Here compute means the value named 'compute' in this block. The important operands are opcode: the value named 'opcode' in this block; in: the value named 'in' in this block; Opcode: the value named 'opcode' in this block. Context: the function tensor_command, which handles tensor command.

Line 269: `Opcode.COMPUTE_TERNARY,` This is continuation line 5 of the 6-line logical statement that begins at source line 265. Its exact fragment is ‘Opcode.COMPUTE_TERNARY,‘. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_TERNARY: the value named 'compute ternary' in this block. Read together, the complete operation is: This assigns ‘opcode in { Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }‘ to ‘compute ‘ in host memory. Here compute means the value named 'compute' in this block. The important operands are opcode: the value named 'opcode' in this block; in: the value named 'in' in this block; Opcode: the value named 'opcode' in this block. Context: the function tensor_command, which handles tensor command.

Line 270: `}` This is continuation line 6 of the 6-line logical statement that begins at source line 265. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'opcode in { Opcode.COMPUTE_DENSE_OUTER, Opcode.COMPUTE_DENSE_DOT8, Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY, }' to 'compute ' in host memory. Here compute means the value named 'compute' in this block. The important operands are opcode: the value named 'opcode' in this block; in: the value named 'in' in this block; Opcode: the value named 'opcode' in this block. Context: the function tensor_command, which handles tensor command.

Line 271: `if compute:` This executes the following indented host-code branch only when 'compute' is true. The important operands are compute: the value named 'compute' in this block. Context: the function tensor_command, which handles tensor command.

Line 272: `row_count = _unsigned("rows", rows, 4, 8)` This assigns '_unsigned("rows", rows, 4, 8)' to 'row_count ' in host memory. Here row_count means one output-row position processed by the tensor tile. The important operands are _unsigned: the value named 'unsigned' in this block; rows: one output-row position processed by the tensor tile. Context: the function tensor_command, which handles tensor command.

Line 273: `column_count = _unsigned("columns", columns, 6, 16)` This assigns '_unsigned("columns", columns, 6, 16)' to 'column_count ' in host memory. Here column_count means one output-feature position processed in parallel. The important operands are _unsigned: the value named 'unsigned' in this block; columns: one output-feature position processed in parallel. Context: the function tensor_command, which handles tensor command.

Line 274: `if row_count == 0 or column_count == 0:` This executes the following indented host-code branch only when 'row_count == 0 or column_count == 0' is true. The important operands are row_count: one output-row position processed by the tensor tile; or: the value named 'or' in this block; column_count: one output-feature position processed in parallel. Context: the function tensor_command, which handles tensor command.

Line 275: `raise ValueError("compute rows and columns must be non-zero")` This rejects an invalid state by raising 'ValueError("compute rows and columns must be non-zero")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 276: `if opcode == Opcode.COMPUTE_DENSE_DOT8 and row_count != 1:` This executes the following indented host-code branch only when 'opcode == Opcode.COMPUTE_DENSE_DOT8 and row_count != 1' is true. The important operands are opcode: the value named 'opcode' in this block; Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_DOT8: the value named 'compute dense dot8' in this block. Context: the function tensor_command, which handles tensor command.

Line 277: `raise ValueError("dense Dot8 mode requires exactly one output row")` This rejects an invalid state by raising 'ValueError("dense Dot8 mode requires exactly one output row")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 278: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function tensor_command, which handles tensor command.

Line 279: `row_count = _unsigned("rows", rows, 4, 8)` This assigns '_unsigned("rows", rows, 4, 8)' to 'row_count ' in host memory. Here row_count means one output-row position processed by the tensor tile. The important operands are _unsigned: the value named 'unsigned' in this block; rows: one output-row position processed by the tensor tile. Context: the function tensor_command, which handles tensor command.

Line 280: `column_count = _unsigned("columns", columns, 6, 16)` This assigns '_unsigned("columns", columns, 6, 16)' to 'column_count ' in host memory. Here column_count means one output-feature position processed in parallel. The important operands are _unsigned: the value named 'unsigned' in this block; columns: one output-feature position processed in parallel. Context: the function tensor_command, which handles tensor command.

Line 282: `command = int(opcode)` This assigns ‘int(opcode)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are opcode: the value named ’opcode’ in this block. Context: the function tensor_command, which handles tensor command.

Line 283: `command = _put(command, row_count, 4, 4)` This assigns ‘_put(command, row_count, 4, 4)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; row_count: one output-row position processed by the tensor tile. Context: the function tensor_command, which handles tensor command.

Line 284: `command = _put(command, column_count, 8, 6)` This assigns ‘_put(command, column_count, 8, 6)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; column_count: one output-feature position processed in parallel. Context: the function tensor_command, which handles tensor command.

Line 285: `command = _put(command, count, 14, 12)` This assigns ‘_put(command, count, 14, 12)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; count: the value named ’count’ in this block. Context: the function tensor_command, which handles tensor command.

Line 286: `command = _put(command, _unsigned("activation_zero_point", activation_zero_point, 8), 26, 8)` This assigns ‘_put(command, _unsigned("activation_zero_point", activation_zero_point, 8), 26, 8)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function tensor_command, which handles tensor command.

Line 287: `command = _put(command, _unsigned("weight_zero_point", weight_zero_point, 8), 34, 8)` This assigns ‘_put(command, _unsigned("weight_zero_point", weight_zero_point, 8), 34, 8)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function tensor_command, which handles tensor command.

Line 288: `return Command128(command)` This returns ‘Command128(command)‘ to the caller of the function tensor_command, which handles tensor command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 291: `def alpha_xnor_command(active_bits: int, alpha_q8: int) -> Command128:` This starts function alpha_xnor_command. Its indented body performs alpha xnor command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 292: `active = _unsigned("active_bits", active_bits, 9, 256)` This assigns ‘_unsigned("active_bits", active_bits, 9, 256)‘ to ‘active ‘ in host memory. Here active means the value named ’active’ in this block. The important operands are _unsigned: the value named ’unsigned’ in this block; active_bits: the value named ’active bits’ in this block. Context: the function alpha_xnor_command, which handles alpha xnor command.

Line 293: `alpha = _unsigned("alpha_q8", alpha_q8, 9, 256)` This assigns ‘_unsigned("alpha_q8", alpha_q8, 9, 256)‘ to ‘alpha ‘ in host memory. Here alpha means the programmable weight assigned to matching zero spikes in alpha-XNOR attention. The important operands are _unsigned: the value named ’unsigned’ in this block; alpha_q8: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the function alpha_xnor_command, which handles alpha xnor command.

Line 294: `if active == 0:` This executes the following indented host-code branch only when ‘active == 0‘ is true. The important operands are active: the value named ’active’ in this block. Context: the function alpha_xnor_command, which handles alpha xnor command.

Line 295: `raise ValueError("active_bits must be non-zero")` This rejects an invalid state by raising ‘ValueError("active_bits must be non-zero")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 296: `command = _put(int(Opcode.ALPHA_XNOR), active, 42, 9)` This assigns ‘_put(int(Opcode.ALPHA_XNOR), active, 42, 9)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; Opcode: the value named ’opcode’ in this block; ALPHA_XNOR: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the function alpha_xnor_command, which handles alpha xnor command.

Line 297: `command = _put(command, alpha, 51, 9)` This assigns ‘_put(command, alpha, 51, 9)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; alpha: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the function alpha_xnor_command, which handles alpha xnor command.

Line 298: `return Command128(command)` This returns ‘Command128(command)‘ to the caller of the function alpha_xnor_command, which handles alpha xnor command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 301: `def vector_command(` This starts function vector_command. Its indented body performs vector command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 302: `operation: int,` Inside the function vector_command, which handles vector command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are operation: the value named ’operation’ in this block.

Line 303: `*,` Inside the function vector_command, which handles vector command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The expression is a literal or structural operator and has no hidden software state.

Line 304: `multiplier_q15: int = 0,` This assigns ‘0,‘ to ‘multiplier_q15‘ in host memory. Here multiplier_q15 means the value named ’multiplier q15’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_command, which handles vector command.

Line 305: `right_shift: int = 0,` This assigns ‘0,‘ to ‘right_shift‘ in host memory. Here right_shift means the value named ’right shift’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_command, which handles vector command.

Line 306: `output_zero_point: int = 0,` This assigns ‘0,‘ to ‘output_zero_point‘ in host memory. Here output_zero_point means the integer code representing real numeric zero in affine quantization. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_command, which handles vector command.

Line 307: `threshold: int = 0,` This assigns ‘0,‘ to ‘threshold‘ in host memory. Here threshold means the membrane level at which a neuron emits a spike. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_command, which handles vector command.

Line 308: `surrogate_window: int = 0,` This assigns ‘0,‘ to ‘surrogate_window‘ in host memory. Here surrogate_window means the value named ’surrogate window’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_command, which handles vector command.

Line 309: `) -> Command128:` Inside the function vector_command, which handles vector command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 310: `command = _put(int(Opcode.VECTOR_ALU), _unsigned("operation", operation, 4, 12), 4, 4)` This assigns ‘_put(int(Opcode.VECTOR_ALU), _unsigned("operation", operation, 4, 12), 4, 4)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; Opcode: the value named ’opcode’ in this block; VECTOR_ALU: the value named ’vector alu’ in this block. Context: the function vector_command, which handles vector command.

Line 311: `command = _put(command, _signed_bits("multiplier_q15", multiplier_q15, 16), 8, 16)` This assigns ‘_put(command, _signed_bits("multiplier_q15", multiplier_q15, 16), 8, 16)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _signed_bits: the value named ’signed bits’ in this block. Context: the function vector_command, which handles vector command.

Line 312: `if operation == 9:` This executes the following indented host-code branch only when ‘operation == 9‘ is true. The important operands are operation: the value named ’operation’ in this block. Context: the function vector_command, which handles vector command.

Line 313: `if right_shift == -16:` This executes the following indented host-code branch only when ‘right_shift == -16‘ is true. The important operands are right_shift: the value named ’right shift’ in this block. Context: the function vector_command, which handles vector command.

Line 314: `raise ValueError("binary_point_shift must be in [-15,15]")` This rejects an invalid state by raising ‘ValueError("binary_point_shift must be in [-15,15]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 315: `shift_bits = _signed_bits("binary_point_shift", right_shift, 5)` This assigns ‘_signed_bits("binary_point right_shift, 5)‘ to ‘shift_bits ‘ in host memory. Here shift_bits means the value named ’shift bits’ in this block. The important operands are _signed_bits: the value named ’signed bits’ in this block; binary_point_shift: the value named ’binary point shift’ in this block; right_shift: the value named ’right shift’ in this block. Context: the function vector_command, which handles vector command.

Line 316: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function vector_command, which handles vector command.

Line 317: `shift_bits = _unsigned("right_shift", right_shift, 5)` This assigns ‘_unsigned("right_shift", right_shift, 5)‘ to ‘shift_bits ‘ in host memory. Here shift_bits means the value named ’shift bits’ in this block. The important operands are _unsigned: the value named ’unsigned’ in this block; right_shift: the value named ’right shift’ in this block. Context: the function vector_command, which handles vector command.

Line 318: `command = _put(command, shift_bits, 24, 5)` This assigns ‘_put(command, shift_bits, 24, 5)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; shift_bits: the value named ’shift bits’ in this block. Context: the function vector_command, which handles vector command.

Line 319: `command = _put(command, _signed_bits("output_zero_point", output_zero_point, 16), 29, 16)` This assigns ‘_put(command, _signed_bits("output_zero_point", output_zero_point, 16), 29, 16)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _signed_bits: the value named ’signed bits’ in this block. Context: the function vector_command, which handles vector command.

Line 320: `command = _put(command, _signed_bits("threshold", threshold, 32), 45, 32)` This assigns ‘_put(command, _signed_bits("threshold", threshold, 32), 45, 32)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _signed_bits: the value named ’signed bits’ in this block. Context: the function vector_command, which handles vector command.

Line 321: `command = _put(command, _unsigned("surrogate_window", surrogate_window, 32), 77, 32)` This assigns ‘_put(command, _unsigned("surrogate_window", surrogate_window, 32), 77, 32)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function vector_command, which handles vector command.

Line 322: `return Command128(command)` This returns ‘Command128(command)‘ to the caller of the function vector_command, which handles vector command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 325: `def layernorm_command(feature_count: int, epsilon: int = 1) -> Command128:` This starts function layernorm_command. Its indented body performs layernorm command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 326: `count = _unsigned("feature_count", feature_count, 12, 2048)` This assigns ‘_unsigned("feature_count", feature_count, 12, 2048)‘ to ‘count ‘ in host memory. Here count means the value named ’count’ in this block. The important operands are _unsigned: the value named ’unsigned’ in this block; feature_count: an integer count retained for control or measurable telemetry. Context: the function layernorm_command, which handles layernorm command.

Line 327: `if count == 0:` This executes the following indented host-code branch only when ‘count == 0‘ is true. The important operands are count: the value named ’count’ in this block. Context: the function layernorm_command, which handles layernorm command.

Line 328: `raise ValueError("feature_count must be non-zero")` This rejects an invalid state by raising ‘ValueError("feature_count must be non-zero")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 329: `command = _put(int(Opcode.LAYERNORM), count, 4, 12)` This assigns ‘_put(int(Opcode.LAYERNORM), count, 4, 12)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; Opcode: the value named ’opcode’ in this block; LAYERNORM: the value named ’layernorm’ in this block. Context: the function layernorm_command, which handles layernorm command.

Line 330: `command = _put(command, _unsigned("epsilon", epsilon, 32), 16, 32)` This assigns ‘_put(command, _unsigned("epsilon", epsilon, 32), 16, 32)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function layernorm_command, which handles layernorm command.

Line 331: `return Command128(command)` This returns ‘Command128(command)‘ to the caller of the function layernorm_command, which handles layernorm command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 334: `def log_softmax_command(item_count: int, difference_shift: int) -> Command128:` This starts function log_softmax_command. Its indented body performs log softmax command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 335: `count = _unsigned("item_count", item_count, 9, 256)` This assigns ‘_unsigned("item_count", item_count, 9, 256)‘ to ‘count ‘ in host memory. Here count means the value named ’count’ in this block. The important operands are _unsigned: the value named ’unsigned’ in this block; item_count: an integer count retained for control or measurable telemetry. Context: the function log_softmax_command, which handles log softmax command.

Line 336: `if count == 0:` This executes the following indented host-code branch only when ‘count == 0‘ is true. The important operands are count: the value named ’count’ in this block. Context: the function log_softmax_command, which handles log softmax command.

Line 337: `raise ValueError("item_count must be non-zero")` This rejects an invalid state by raising ‘ValueError("item_count must be non-zero")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 338: `command = _put(int(Opcode.LOG_SOFTMAX), count, 4, 9)` This assigns ‘_put(int(Opcode.LOG_SOFTMAX), count, 4, 9)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; Opcode: the value named ’opcode’ in this block; LOG_SOFTMAX: the value named ’log softmax’ in this block. Context: the function log_softmax_command, which handles log softmax command.

Line 339: `command = _put(command, _unsigned("difference_shift", difference_shift, 5), 13, 5)` This assigns ‘_put(command, _unsigned("difference_shift", difference_shift, 5), 13, 5)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named ’unsigned’ in this block. Context: the function log_softmax_command, which handles log softmax command.

Line 340: `return Command128(command)` This returns ‘Command128(command)‘ to the caller of the function log_softmax_comm which handles log softmax command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 343: `def lif_command(` This starts function lif_command. Its indented body performs lif command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 344: `leak_shift: int,` Inside the function lif_command, which handles lif command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are leak_shift: the retained fraction of prior membrane state.

Line 345: `threshold: int,` Inside the function lif_command, which handles lif command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are threshold: the membrane level at which a neuron emits a spike.

Line 346: `reset_value: int,` Inside the function lif_command, which handles lif command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are reset_value: the value named ’reset value’ in this block.

Line 347: `*,` Inside the function lif_command, which handles lif command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The expression is a literal or structural operator and has no hidden software state.

Line 348: `clear_state: bool = False,` This assigns ‘False,‘ to ‘clear_state‘ in host memory. Here clear_state means the finite-state-machine phase retained in flip-flops. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_command, which handles lif command.

Line 349: `trace_state: bool = False,` This assigns ‘False,‘ to ‘trace_state‘ in host memory. Here trace_state means the finite-state-machine phase retained in flip-flops. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_command, which handles lif command.

Line 350: `) -> Command128:` Inside the function lif_command, which handles lif command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 351: `command = _put(int(Opcode.LIF_STEP), _unsigned("leak_shift", leak_shift, 5), 4, 5)` This assigns ‘_put(int(Opcode.LIF_STEP), _unsigned("leak_shift", leak_shift, 5), 4, 5)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; Opcode: the value named ’opcode’ in this block; LIF_STEP: the value named ’lif step’ in this block. Context: the function lif_command, which handles lif command.

Line 352: `command = _put(command, _signed_bits("threshold", threshold, 24), 9, 24)` This assigns ‘_put(command, _signed_bits("threshold", threshold, 24), 9, 24)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _signed_bits: the value named ’signed bits’ in this block. Context: the function lif_command, which handles lif command.

Line 353: `command = _put(command, _signed_bits("reset_value", reset_value, 24), 33, 24)` This assigns ‘_put(command, _signed_bits("reset_value", reset_value, 24), 33, 24)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _signed_bits: the value named ’signed bits’ in this block. Context: the function lif_command, which handles lif command.

Line 354: `command = _put(command, int(bool(clear_state)), 57, 1)` This assigns ‘_put(command, int(bool(clear_state)) 57, 1)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; bool: the value named ’bool’ in this block. Context: the function lif_command, which handles lif command.

Line 355: `command = _put(command, int(bool(trace_state)), 58, 1)` This assigns ‘_put(command, int(bool(trace_state)) 58, 1)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named ’put’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; bool: the value named ’bool’ in this block. Context: the function lif_command, which handles lif command.

Line 356: `return Command128(command)` This returns ‘Command128(command)‘ to the caller of the function lif_command, which handles lif command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 359: `def sgd_command(` This starts function sgd_command. Its indented body performs sgd command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 360: `learning_rate_q16: int,` Inside the function sgd_command, which handles sgd command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude.

Line 361: `momentum_q16: int,` Inside the function sgd_command, which handles sgd command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are momentum_q16: the value named 'momentum q16' in this block.

Line 362: `weight_decay_q16: int,` Inside the function sgd_command, which handles sgd command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are weight_decay_q16: a learned coefficient multiplied by an activation.

Line 363: `*,` Inside the function sgd_command, which handles sgd command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The expression is a literal or structural operator and has no hidden software state.

Line 364: `update_right_shift: int = 16,` This assigns '16,' to 'update_right_shift' in host memory. Here update_right_shift means the value named 'update right shift' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function sgd_command, which handles sgd command.

Line 365: `) -> Command128:` Inside the function sgd_command, which handles sgd command, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 366: `command = _put(int(Opcode.SGD_UPDATE), _unsigned("learning_rate_q16", learning_rate_q16, 18, 131071), 4, 18)` This assigns '_put(int(Opcode.SGD_UPDATE), _unsigned("learning_rate_q16", learning_rate_q16, 18, 131071), 4, 18)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; Opcode: the value named 'opcode' in this block; SGD_UPDATE: the value named 'sgd update' in this block. Context: the function sgd_command, which handles sgd command.

Line 367: `command = _put(command, _unsigned("momentum_q16", momentum_q16, 17, 65536), 22, 17)` This assigns '_put(command, _unsigned("momentum_q16", momentum_q16, 17, 65536), 22, 17)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named 'unsigned' in this block. Context: the function sgd_command, which handles sgd command.

Line 368: `command = _put(command, _unsigned("weight_decay_q16", weight_decay_q16, 17, 65536), 39, 17)` This assigns '_put(command, _unsigned("weight_decay_q16", weight_decay_q16, 17, 65536), 39, 17)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; _unsigned: the value named 'unsigned' in this block. Context: the function sgd_command, which handles sgd command.

Line 369: `command = _put(command, _unsigned("update_right_shift", update_right_shift, 5), 56, 5)` This assigns '_put(command, _unsigned("update_right_shift", update_right_shift, 5), 56, 5)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; command: the 128-bit instruction selecting an engine,

shape, mode, and numeric parameters; _unsigned: the value named 'unsigned' in this block. Context: the function sgd_command, which handles sgd command.

Line 370: `return Command128(command)` This returns 'Command128(command)' to the caller of the function sgd_command, which handles sgd command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 373: `def lut_command(suboperation: LutSuboperation, item_count: int = 0) -> Command128:` This starts function lut_command. Its indented body performs lut command as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 374: `count = _unsigned("item_count", item_count, 12, 256)` This assigns '_unsigned("item_count", item_count, 12, 256)' to 'count ' in host memory. Here count means the value named 'count' in this block. The important operands are _unsigned: the value named 'unsigned' in this block; item_count: an integer count retained for control or measurable telemetry. Context: the function lut_command, which handles lut command.

Line 375: `if suboperation == LutSuboperation.LOAD and count == 0:` This executes the following indented host-code branch only when 'suboperation == LutSuboperation.LOAD and count == 0' is true. The important operands are suboperation: the value named 'suboperation' in this block; LutSuboperation: the value named 'lut suboperation' in this block; LOAD: the value named 'load' in this block. Context: the function lut_command, which handles lut command.

Line 376: `raise ValueError("LUT load requires at least one entry")` This rejects an invalid state by raising 'ValueError("LUT load requires at least one entry")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 377: `if suboperation == LutSuboperation.EVALUATE and count not in {0, 8}:` This executes the following indented host-code branch only when 'suboperation == LutSuboperation.EVALUATE and count not in {0, 8}' is true. The important operands are suboperation: the value named 'suboperation' in this block; LutSuboperation: the value named 'lut suboperation' in this block; EVALUATE: the value named 'evaluate' in this block. Context: the function lut_command, which handles lut command.

Line 378: `raise ValueError("LUT evaluate processes exactly eight indices")` This rejects an invalid state by raising 'ValueError("LUT evaluate processes exactly eight indices")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 379: `if suboperation == LutSuboperation.CLEAR and count != 0:` This executes the following indented host-code branch only when 'suboperation == LutSuboperation.CLEAR and count != 0' is true. The important operands are suboperation: the value named 'suboperation' in this block; LutSuboperation: the value named 'lut suboperation' in this block; CLEAR: the value named 'clear' in this block. Context: the function lut_command, which handles lut command.

Line 380: `raise ValueError("LUT clear does not accept stream items")` This rejects an invalid state by raising 'ValueError("LUT clear does not accept stream items")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 381: `command = _put(int(Opcode.ACTIVATION_LUT), int(suboperation), 4, 4)` This assigns '_put(int(Opcode.ACTI int(suboperation), 4, 4)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; Opcode: the value named 'opcode' in this block; ACTIVATION_LUT: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function lut_command, which handles lut command.

Line 382: `command = _put(command, count, 8, 12)` This assigns '_put(command, count, 8, 12)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are _put: the value named 'put' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; count: the value named 'count' in this block. Context: the function lut_command, which handles lut command.

Line 383: `return Command128(command)` This returns 'Command128(command)' to the caller of the function lut_command, which handles lut command, ending this invocation. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 386: `def _finish_beats(words: Iterable[int]) -> PackedStreamBeats:` This starts function _finish_beats. Its indented body performs finish beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 387: `return PackedStreamBeats(words)` This returns 'PackedStreamBeats(words)' to the caller of the function _finish_beats, which handles finish beats, ending this invocation. The important operands are PackedStreamBeats: the value named 'packed stream beats' in this block; words: the value named 'words' in this block.

Line 390: `def pack_activation_beats(vectors: Sequence[Sequence[int]]) -> PackedStreamBeats:` This starts function pack_activation_beats. Its indented body performs pack activation beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 391: `words = []` This assigns '[]' to 'words ' in host memory. Here words means the value named 'words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_activation_beats, which handles pack activation beats.

Line 392: `for issue, vector in enumerate(vectors):` This iterates issue, vector over 'enumerate(vectors)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_activation_beats, which handles pack activation beats.

Line 393: `if len(vector) != 8:` This executes the following indented host-code branch only when 'len(vector) != 8' is true. The important operands are len: the value named 'len' in this block; vector: the value named 'vector' in this block. Context: the function pack_activation_beats, which handles pack activation beats.

Line 394: `raise ValueError(f"activation issue {issue} must contain eight bytes")` This rejects an invalid state by raising 'ValueError(f"activation issue {issue} must contain eight bytes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 395: `word = sum(_unsigned(f"activation[{issue},{lane}]", value, 8) << (8 * lane)` This statement continues through source line 396; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sum(_unsigned(f"activation[{issue},{lane}]", value, 8) << (8 * lane) for lane, value in enumerate(vector))' to 'word ' in host memory. Here word means the value named 'word' in this block. The important operands are sum: the value named 'sum' in this block; _unsigned: the value named 'unsigned' in this block; f: the value named 'f' in this block. Context: the function pack_activation_beats, which handles pack activation beats.

Line 396: `for lane, value in enumerate(vector))` This is continuation line 2 of the 2-line logical statement that begins at source line 395. Its exact fragment is 'for lane, value in enumerate(vector))'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; value: the value named 'value' in this block. Read together, the complete operation is: This assigns 'sum(_unsigned(f"activation[{issue},{lane}]", value, 8) << (8 * lane) for lane, value in enumerate(vector))' to 'word ' in host memory. Here word means the value named 'word' in this block. The important operands are sum: the value named 'sum' in this block; _unsigned: the value named 'unsigned' in this block; f: the value named 'f' in this block. Context: the function pack_activation_beats, which handles pack activation beats.

Line 397: `words.append(word)` This calls ‘words.append‘ with ‘word‘ inside the function pack_activation_beats, which handles pack activation beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 398: `return _finish_beats(words)` This returns ‘_finish_beats(words)‘ to the caller of the function pack_activation_bea which handles pack activation beats, ending this invocation. The important operands are _finish_beats: the value named 'finish beats' in this block; words: the value named 'words' in this block.

Line 401: `def pack_outer_weight_beats(weights: Sequence[Sequence[int]]) -> PackedStreamBeats:` This starts function pack_outer_weight_beats. Its indented body performs pack outer weight beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 402: `words = []` This assigns ‘[]‘ to ‘words ‘ in host memory. Here words means the value named 'words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_outer_weight_beats, which handles pack outer weight beats.

Line 403: `for issue, vector in enumerate(weights):` This iterates issue, vector over ‘enumerate(weights)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_outer_weight_beats, which handles pack outer weight beats.

Line 404: `if len(vector) != 16:` This executes the following indented host-code branch only when ‘len(vector) != 16‘ is true. The important operands are len: the value named 'len' in this block; vector: the value named 'vector' in this block. Context: the function pack_outer_weight_beats, which handles pack outer weight beats.

Line 405: `raise ValueError(f"outer weight issue {issue} must contain sixteen bytes")` This rejects an invalid state by raising ‘ValueError(f"outer weight issue {issue} must contain sixteen bytes")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 406: `for half in range(2):` This iterates half over ‘range(2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_outer_weight_beats, which handles pack outer weight beats.

Line 407: `words.append(sum(_unsigned(f"weight[{issue},{8 * half + lane}]", vector[8 * half + lane], 8)` This statement continues through source line 408; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘words.append‘ with ‘sum(_unsigned(f"weight[{issue},{8 * half + lane}]", vector[8 * half + lane], 8) << (8 * lane) for lane in range(8))‘ inside the function pack_outer_weight_beats, which handles pack outer weight beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 408: `<< (8 * lane) for lane in range(8)))` This is continuation line 2 of the 2-line logical statement that begins at source line 407. Its exact fragment is ‘<< (8 * lane) for lane in range(8)))‘. The important operands are lane: one parallel arithmetic element within a vector or tensor datapath; for: the value named 'for' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_unsigned(f"weight[{issue},{8 * half + lane}]", vector[8 * half + lane], 8) << (8 * lane) for lane in range(8))‘ inside the function pack_outer_weight_beats, which handles pack outer weight beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 409: `return _finish_beats(words)` This returns ‘_finish_beats(words)‘ to the caller of the function pack_outer_weight_ which handles pack outer weight beats, ending this invocation. The important operands are _finish_beats: the value named 'finish beats' in this block; words: the value named 'words' in this block.

Line 412: `def pack_dot8_weight_beats(weights: Sequence[Sequence[Sequence[int]]]) -> PackedStreamBeats:` This starts function pack_dot8_weight_beats. Its indented body performs pack dot8 weight beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 413: `words = []` This assigns '[]' to 'words ' in host memory. Here words means the value named 'words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_dot8_weight_beats, which handles pack dot8 weight beats.

Line 414: `for issue, matrix in enumerate(weights):` This iterates issue, matrix over 'enumerate(weights)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_dot8_weight_beats, which handles pack dot8 weight beats.

Line 415: `if len(matrix) != 8 or any(len(row) != 16 for row in matrix):` This executes the following indented host-code branch only when 'len(matrix) != 8 or any(len(row) != 16 for row in matrix)' is true. The important operands are len: the value named 'len' in this block; matrix: the value named 'matrix' in this block; or: the value named 'or' in this block. Context: the function pack_dot8_weight_beats, which handles pack dot8 weight beats.

Line 416: `raise ValueError(f"Dot8 issue {issue} must have shape [8,16]")` This rejects an invalid state by raising 'ValueError(f"Dot8 issue {issue} must have shape [8,16]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 417: `for reduction_lane, row in enumerate(matrix):` This iterates reduction_lane, row over 'enumerate(matrix)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_dot8_weight_beats, which handles pack dot8 weight beats.

Line 418: `for half in range(2):` This iterates half over 'range(2)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_dot8_weight_beats, which handles pack dot8 weight beats.

Line 419: `words.append(sum(_unsigned(` This statement continues through source line 421; the explanation uses the complete expression rather than only the first visual fragment. This calls 'words.append' with 'sum(_unsigned(f"dot8_weight[{issue},{reduction_lane},{8 * half + lane}]", row[8 * half + lane], 8) << (8 * lane) for lane in range(8))' inside the function pack_dot8_weight_beats, which handles pack dot8 weight beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 420: `f"dot8_weight[{issue},{reduction_lane},{8 * half + lane}]",` This is continuation line 2 of the 3-line logical statement that begins at source line 419. Its exact fragment is 'f"dot8_weight[{issue},{reduction_lane},{8 * half + lane}]",'. The important operands are f: the value named 'f' in this block; dot8_weight: a learned coefficient multiplied by an activation; issue: the value named 'issue' in this block. Read together, the complete operation is: This calls 'words.append' with 'sum(_unsigned(f"dot8_weight[{issue},{reduction_lane},{8 * half + lane}]", row[8 * half + lane], 8) << (8 * lane) for lane in range(8))' inside the function pack_dot8_weight_beats, which handles pack dot8 weight beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 421: `row[8 * half + lane], 8) << (8 * lane) for lane in range(8)))` This is continuation line 3 of the 3-line logical statement that begins at source line 419. Its exact fragment is 'row[8 * half + lane], 8) << (8 * lane) for lane in range(8)))'. The important operands are row: one output-row position processed by the tensor tile; half: the value named 'half' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls 'words.append' with 'sum(_unsigned(f"dot8_weight[{issue},{reduction_lane},{8 * half + lane}]", row[8 * half + lane], 8) << (8 * lane) for lane in range(8))' inside the function pack_dot8_weight_beats, which handles pack dot8 weight beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 422: `return _finish_beats(words)` This returns '_finish_beats(words)' to the caller of the function pack_dot8_weight_b which handles pack dot8 weight beats, ending this invocation. The important operands are _finish_beats: the value named 'finish beats' in this block; words: the value named 'words' in this block.

Line 425: `def pack_alpha_beats(query: Sequence[int], key: Sequence[int]) -> PackedStreamBeats:` This starts function pack_alpha_beats. Its indented body performs pack alpha beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 426: `if len(query) != 256 or len(key) != 256:` This executes the following indented host-code branch only when ‘len(query) != 256 or len(key) != 256‘ is true. The important operands are len: the value named ’len’ in this block; query: the value named ’query’ in this block; or: the value named ’or’ in this block. Context: the function pack_alpha_beats, which handles pack alpha beats.

Line 427: `raise ValueError("alpha-XNOR query and key must each contain 256 bits")` This rejects an invalid state by raising ‘ValueError("alpha-XNOR query and key must each contain 256 bits")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 428: `words = []` This assigns ‘[]‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_alpha_beats, which handles pack alpha beats.

Line 429: `for vector_name, vector in (("query", query), ("key", key)):` This iterates vector_name, vector over ‘(("query", query), ("key", key))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_alpha_beats, which handles pack alpha beats.

Line 430: `for word_index in range(4):` This iterates word_index over ‘range(4)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_alpha_beats, which handles pack alpha beats.

Line 431: `word = 0` This assigns ‘0‘ to ‘word ‘ in host memory. Here word means the value named ’word’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_alpha_beats, which handles pack alpha beats.

Line 432: `for bit_index in range(64):` This iterates bit_index over ‘range(64)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_alpha_beats, which handles pack alpha beats.

Line 433: `bit = _unsigned(f"{vector_name} bit", vector[word_index * 64 + bit_index], 1)` This assigns ‘_unsigned(f"{vector_name} bit", vector[word_index * 64 + bit_index], 1)‘ to ‘bit ‘ in host memory. Here bit means the value named ’bit’ in this block. The important operands are _unsigned: the value named ’unsigned’ in this block; f: the value named ’f’ in this block; vector_name: the value named ’vector name’ in this block. Context: the function pack_alpha_beats, which handles pack alpha beats.

Line 434: `word |= bit << bit_index` Inside the function pack_alpha_beats, which handles pack alpha beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are word: the value named ’word’ in this block; bit: the value named ’bit’ in this block; bit_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 435: `words.append(word)` This calls ‘words.append‘ with ‘word‘ inside the function pack_alpha_beats, which handles pack alpha beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 436: `return _finish_beats(words)` This returns ‘_finish_beats(words)‘ to the caller of the function pack_alpha_beats, which handles pack alpha beats, ending this invocation. The important operands are _finish_beats: the value named ’finish beats’ in this block; words: the value named ’words’ in this block.

Line 439: `def pack_vector_beats(` This starts function pack_vector_beats. Its indented body performs pack vector beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 440: `operand_a: Sequence[int],` Inside the function pack_vector_beats, which handles pack vector beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are operand_a: the value named 'operand a' in this block; Sequence: the value named 'sequence' in this block.

Line 441: `operand_b: Sequence[int],` Inside the function pack_vector_beats, which handles pack vector beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are operand_b: the value named 'operand b' in this block; Sequence: the value named 'sequence' in this block.

Line 442: `*,` Inside the function pack_vector_beats, which handles pack vector beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The expression is a literal or structural operator and has no hidden software state.

Line 443: `clamp_minimum: int = -(1 << 31),` This assigns '-(1 << 31),' to 'clamp_minimum' in host memory. Here clamp_minimum means the value named 'clamp minimum' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_vector_beats, which handles pack vector beats.

Line 444: `clamp_maximum: int = (1 << 31) - 1,` This assigns '(1 << 31) - 1,' to 'clamp_maximum' in host memory. Here clamp_maximum means the value named 'clamp maximum' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_vector_beats, which handles pack vector beats.

Line 445: `) -> PackedStreamBeats:` Inside the function pack_vector_beats, which handles pack vector beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are PackedStreamBeats: the value named 'packed stream beats' in this block.

Line 446: `if len(operand_a) != 8 or len(operand_b) != 8:` This executes the following indented host-code branch only when 'len(operand_a) != 8 or len(operand_b) != 8' is true. The important operands are len: the value named 'len' in this block; operand_a: the value named 'operand a' in this block; or: the value named 'or' in this block. Context: the function pack_vector_beats, which handles pack vector beats.

Line 447: `raise ValueError("vector operands must each contain eight signed 32-bit lanes")` This rejects an invalid state by raising 'ValueError("vector operands must each contain eight signed 32-bit lanes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 448: `words = [_signed_bits("clamp_minimum", clamp_minimum, 32)` This statement continues through source line 449; the explanation uses the complete expression rather than only the first visual fragment. This assigns '[_signed_bits("clamp_ clamp_minimum, 32) | (_signed_bits("clamp_maximum", clamp_maximum, 32) << 32)]' to 'words ' in host memory. Here words means the value named 'words' in this block. The important operands are _signed_bits: the value named 'signed bits' in this block; clamp_minimum: the value named 'clamp minimum' in this block; clamp_maximum: the value named 'clamp maximum' in this block. Context: the function pack_vector_beats, which handles pack vector beats.

Line 449: `| (_signed_bits("clamp_maximum", clamp_maximum, 32) << 32)]` This is continuation line 2 of the 2-line logical statement that begins at source line 448. Its exact fragment is '| (_signed_bits("clamp_maximum", clamp_maximum, 32) << 32)]'. The important operands are _signed_bits: the value named 'signed bits' in this block; clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This assigns '[_signed_bits("clamp_minimum", clamp_minimum, 32) | (_signed_bits("clamp_maximum", clamp_maximum, 32) << 32)]' to 'words ' in host memory. Here words means the value named 'words' in this block. The important operands are _signed_bits: the value named 'signed bits' in this block; clamp_minimum: the value named 'clamp

minimum' in this block; clamp_maximum: the value named 'clamp maximum' in this block. Context: the function pack_vector_beats, which handles pack vector beats.

Line 450: `for name, operand in (("operand_a", operand_a), ("operand_b", operand_b)):` This iterates name, operand over '(("operand_a", operand_a), ("operand_b", operand_b))'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_vector_beats, which handles pack vector beats.

Line 451: `for lane in range(0, 8, 2):` This iterates lane over 'range(0, 8, 2)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_vector_beats, which handles pack vector beats.

Line 452: `words.append(_signed_bits(f"{name}[{lane}]", operand[lane], 32)` This statement continues through source line 453; the explanation uses the complete expression rather than only the first visual fragment. This calls 'words.append' with '_signed_bits(f"{name}[{lane}]", operand[lane], 32) | (_signed_bits(f"{name}[{lane + 1}]", operand[lane + 1], 32) << 32)' inside the function pack_vector_beats, which handles pack vector beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 453: `| (_signed_bits(f"{name}[{lane + 1}]", operand[lane + 1], 32) << 32))` This is continuation line 2 of the 2-line logical statement that begins at source line 452. Its exact fragment is '| (_signed_bits(f"{name}[{lane + 1}]", operand[lane + 1], 32) << 32))'. The important operands are _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block; name: the value named 'name' in this block. Read together, the complete operation is: This calls 'words.append' with '_signed_bits(f"{name}[{lane}]", operand[lane], 32) | (_signed_bits(f"{name}[{lane + 1}]", operand[lane + 1], 32) << 32)' inside the function pack_vector_beats, which handles pack vector beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 454: `return _finish_beats(words)` This returns '_finish_beats(words)' to the caller of the function pack_vector_beats, which handles pack vector beats, ending this invocation. The important operands are _finish_beats: the value named 'finish beats' in this block; words: the value named 'words' in this block.

Line 457: `def pack_layernorm_beats(` This starts function pack_layernorm_beats. Its indented body performs pack layernorm beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 458: `values: Sequence[int], gamma_q15: Sequence[int], beta: Sequence[int]` Inside the function pack_layernorm_beats, which handles pack layernorm beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are values: the value named 'values' in this block; Sequence: the value named 'sequence' in this block; gamma_q15: the learned multiplicative LayerNorm parameter.

Line 459: `) -> PackedStreamBeats:` Inside the function pack_layernorm_beats, which handles pack layernorm beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are PackedStreamBeats: the value named 'packed stream beats' in this block.

Line 460: `if not (len(values) == len(gamma_q15) == len(beta)) or len(values) == 0:` This executes the following indented host-code branch only when 'not (len(values) == len(gamma_q15) == len(beta)) or len(values) == 0' is true. The important operands are not: the value named 'not' in this block; len: the value named 'len' in this block; values: the value named 'values' in this block. Context: the function pack_layernorm_beats, which handles pack layernorm beats.

Line 461: `raise ValueError("LayerNorm value, gamma, and beta arrays must have equal non-zero length")` This rejects an invalid state by raising 'ValueError("LayerNorm value, gamma, and beta arrays must have equal non-zero length")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 462: `words = []` This assigns '[]' to 'words ' in host memory. Here words means the value named 'words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_layernorm_beats, which handles pack layernorm beats.

Line 463: `for index, (value, gamma, offset) in enumerate(zip(values, gamma_q15, beta)):` This iterates index, (value, gamma, offset) over 'enumerate(zip(values, gamma_q15, beta))'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_layernorm_beats, which handles pack layernorm beats.

Line 464: `beta_bits = _signed_bits(f"beta[{index}]", offset, 24)` This assigns '_signed_bits(f"beta[{index}]", offset, 24)' to 'beta_bits ' in host memory. Here beta_bits means the learned additive LayerNorm parameter. The important operands are _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block; beta: the learned additive LayerNorm parameter. Context: the function pack_layernorm_beats, which handles pack layernorm beats.

Line 465: `word0 = _signed_bits(f"value[{index}]", value, 24)` This assigns '_signed_bits(f"value[{index}]", value, 24)' to 'word0 ' in host memory. Here word0 means the value named 'word0' in this block. The important operands are _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block; value: the value named 'value' in this block. Context: the function pack_layernorm_beats, which handles pack layernorm beats.

Line 466: `word0 |= _signed_bits(f"gamma[{index}]", gamma, 24) << 24` Inside the function pack_layernorm_beats, which handles pack layernorm beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are word0: the value named 'word0' in this block; _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block.

Line 467: `word0 |= (beta_bits & 0xFFFF) << 48` Inside the function pack_layernorm_beats, which handles pack layernorm beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are word0: the value named 'word0' in this block; beta_bits: the learned additive LayerNorm parameter.

Line 468: `words.extend((word0, beta_bits >> 16))` This calls 'words.extend' with '(word0, beta_bits >> 16)' inside the function pack_layernorm_beats, which handles pack layernorm beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 469: `return _finish_beats(words)` This returns '_finish_beats(words)' to the caller of the function pack_layernorm_bea which handles pack layernorm beats, ending this invocation. The important operands are _finish_beats: the value named 'finish beats' in this block; words: the value named 'words' in this block.

Line 472: `def pack_softmax_beats(scores: Sequence[int]) -> PackedStreamBeats:` This starts function pack_softmax_ Its indented body performs pack softmax beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 473: `if not scores:` This executes the following indented host-code branch only when 'not scores' is true. The important operands are not: the value named 'not' in this block; scores: the value named 'scores' in this block. Context: the function pack_softmax_beats, which handles pack softmax beats.

Line 474: `raise ValueError("softmax requires at least one score")` This rejects an invalid state by raising 'ValueError("softmax requires at least one score")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 475: `return _finish_beats(_signed_bits(f"score[{index}]", score, 24)` This statement continues through source line 476; the explanation uses the complete expression rather than only the first visual fragment. This returns '_finish_beats(_signed_bits(f"score[{index}]", score, 24) for index, score in enumerate(scores))' to the caller of the function pack_softmax_beats, which handles pack softmax beats, ending this invocation. The important operands are

_finish_beats: the value named 'finish beats' in this block; _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block.

Line 476: `for index, score in enumerate(scores))` This is continuation line 2 of the 2-line logical statement that begins at source line 475. Its exact fragment is 'for index, score in enumerate(scores))'. The important operands are for: the value named 'for' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; score: the value named 'score' in this block. Read together, the complete operation is: This returns '_finish_beats(_signed_bits(f"score[{index}]", score, 24) for index, score in enumerate(scores))' to the caller of the function pack_softmax_beats, which handles pack softmax beats, ending this invocation. The important operands are _finish_beats: the value named 'finish beats' in this block; _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block.

Line 479: `def pack_lif_beats(currents: Sequence[int]) -> PackedStreamBeats:` This starts function pack_lif_beats. Its indented body performs pack lif beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 480: `if len(currents) != 32:` This executes the following indented host-code branch only when 'len(currents) != 32' is true. The important operands are len: the value named 'len' in this block; currents: the value named 'currents' in this block. Context: the function pack_lif_beats, which handles pack lif beats.

Line 481: `raise ValueError("LIF current vector must contain 32 signed 16-bit lanes")` This rejects an invalid state by raising 'ValueError("LIF current vector must contain 32 signed 16-bit lanes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 482: `words = []` This assigns '[]' to 'words ' in host memory. Here words means the value named 'words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_lif_beats, which handles pack lif beats.

Line 483: `for lane in range(0, 32, 4):` This iterates lane over 'range(0, 32, 4)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_lif_beats, which handles pack lif beats.

Line 484: `words.append(sum(_signed_bits(f"current[{lane + offset}]", currents[lane + offset], 16)` This statement continues through source line 485; the explanation uses the complete expression rather than only the first visual fragment. This calls 'words.append' with 'sum(_signed_bits(f"current[{lane + offset}]", currents[lane + offset], 16) << (16 * offset) for offset in range(4))' inside the function pack_lif_beats, which handles pack lif beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 485: `<< (16 * offset) for offset in range(4)))` This is continuation line 2 of the 2-line logical statement that begins at source line 484. Its exact fragment is '<< (16 * offset) for offset in range(4)))'. The important operands are offset: the value named 'offset' in this block; for: the value named 'for' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This calls 'words.append' with 'sum(_signed_bits(f"current[{lane + offset}]", currents[lane + offset], 16) << (16 * offset) for offset in range(4))' inside the function pack_lif_beats, which handles pack lif beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 486: `return _finish_beats(words)` This returns '_finish_beats(words)' to the caller of the function pack_lif_beats, which handles pack lif beats, ending this invocation. The important operands are _finish_beats: the value named 'finish beats' in this block; words: the value named 'words' in this block.

Line 489: `def pack_optimizer_beats(` This starts function pack_optimizer_beats. Its indented body performs pack optimizer beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 490: `weights: Sequence[int], gradients: Sequence[int], velocities: Sequence[int]` Inside the function pack_optimizer_beats, which handles pack optimizer beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are weights: a learned coefficient multiplied by an activation; Sequence: the value named 'sequence' in this block; gradients: the signed loss derivative used to update a learned parameter.

Line 491: `) -> PackedStreamBeats:` Inside the function pack_optimizer_beats, which handles pack optimizer beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are PackedStreamBeats: the value named 'packed stream beats' in this block.

Line 492: `if not (len(weights) == len(gradients) == len(velocities) == 8):` This executes the following indented host-code branch only when 'not (len(weights) == len(gradients) == len(velocities) == 8)' is true. The important operands are not: the value named 'not' in this block; len: the value named 'len' in this block; weights: a learned coefficient multiplied by an activation. Context: the function pack_optimizer_beats, which handles pack optimizer beats.

Line 493: `raise ValueError("optimizer vectors must each contain eight lanes")` This rejects an invalid state by raising 'ValueError("optimizer vectors must each contain eight lanes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 494: `words = []` This assigns '[]' to 'words ' in host memory. Here words means the value named 'words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_optimizer_beats, which handles pack optimizer beats.

Line 495: `for lane in range(8):` This iterates lane over 'range(8)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_optimizer_beats, which handles pack optimizer beats.

Line 496: `word = _signed_bits(f"weight[{lane}]", weights[lane], 16)` This assigns '_signed_bits(f"weight[{lane}]", weights[lane], 16)' to 'word ' in host memory. Here word means the value named 'word' in this block. The important operands are _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block; weight: a learned coefficient multiplied by an activation. Context: the function pack_optimizer_beats, which handles pack optimizer beats.

Line 497: `word |= _signed_bits(f"gradient[{lane}]", gradients[lane], 24) << 16` Inside the function pack_optimizer_
which handles pack optimizer beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are word: the value named 'word' in this block; _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block.

Line 498: `word |= _signed_bits(f"velocity[{lane}]", velocities[lane], 24) << 40` Inside the function pack_optimize
which handles pack optimizer beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are word: the value named 'word' in this block; _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block.

Line 499: `words.append(word)` This calls 'words.append' with 'word' inside the function pack_optimizer_beats, which handles pack optimizer beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 500: `return _finish_beats(words)` This returns '_finish_beats(words)' to the caller of the function pack_optimizer_beat
which handles pack optimizer beats, ending this invocation. The important operands are _finish_beats: the value named 'finish beats' in this block; words: the value named 'words' in this block.

Line 503: `def pack_linear_backward_beats(` This starts function pack_linear_backward_beats. Its indented body performs pack linear backward beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 504: `activations: Sequence[Sequence[int]],` Inside the function pack_linear_backward_beats, which handles pack linear backward beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; Sequence: the value named 'sequence' in this block.

Line 505: `weights: Sequence[Sequence[int]],` Inside the function pack_linear_backward_beats, which handles pack linear backward beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are weights: a learned coefficient multiplied by an activation; Sequence: the value named 'sequence' in this block.

Line 506: `output_gradients: Sequence[Sequence[int]],` Inside the function pack_linear_backward_beats, which handles pack linear backward beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are output_gradients: the signed loss derivative used to update a learned parameter; Sequence: the value named 'sequence' in this block.

Line 507: `*,` Inside the function pack_linear_backward_beats, which handles pack linear backward beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The expression is a literal or structural operator and has no hidden software state.

Line 508: `mode: LinearGradientMode = LinearGradientMode.BOTH,` This assigns 'LinearGradientMode.BOTH,' to 'mode' in host memory. Here mode means the value named 'mode' in this block. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter; BOTH: the value named 'both' in this block. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 509: `) -> PackedStreamBeats:` Inside the function pack_linear_backward_beats, which handles pack linear backward beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are PackedStreamBeats: the value named 'packed stream beats' in this block.

Line 510: `"""Pack signed X, W, and dY matrices for one linear backward tile."""` This is executable documentation attached to the function pack_linear_backward_beats, which handles pack linear backward beats. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 512: `rows = len(activations)` This assigns 'len(activations)' to 'rows ' in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are len: the value named 'len' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 513: `input_features = len(weights)` This assigns 'len(weights)' to 'input_features ' in host memory. Here input_features means the value named 'input features' in this block. The important operands are len: the value named 'len' in this block; weights: a learned coefficient multiplied by an activation. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 514: `output_features = len(weights[0]) if input_features else 0` This assigns 'len(weights[0]) if input_features else 0' to 'output_features ' in host memory. Here output_features means the value named 'output features' in this block. The important operands are len: the value named 'len' in this block; weights: a learned coefficient multiplied by an activation; input_features: the value named 'input features' in this block. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 515: `if not 1 <= rows <= 8 or not 1 <= input_features <= 8 or not 1 <= output_features <= 8:` This executes the following indented host-code branch only when ‘not 1 <= rows <= 8 or not 1 <= input_features <= 8 or not 1 <= output_features <= 8‘ is true. The important operands are not: the value named ’not’ in this block; rows: one output-row position processed by the tensor tile; or: the value named ’or’ in this block. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 516: `raise ValueError("linear backward tile dimensions must be in [1,8]")` This rejects an invalid state by raising ‘ValueError("linear backward tile dimensions must be in [1,8]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 517: `if any(len(row) != input_features for row in activations):` This executes the following indented host-code branch only when ‘any(len(row) != input_features for row in activations)‘ is true. The important operands are any: the value named ’any’ in this block; len: the value named ’len’ in this block; row: one output-row position processed by the tensor tile. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 518: `raise ValueError("activation rows must match the weight input dimension")` This rejects an invalid state by raising ‘ValueError("activation rows must match the weight input dimension")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 519: `if any(len(row) != output_features for row in weights):` This executes the following indented host-code branch only when ‘any(len(row) != output_features for row in weights)‘ is true. The important operands are any: the value named ’any’ in this block; len: the value named ’len’ in this block; row: one output-row position processed by the tensor tile. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 520: `raise ValueError("weight rows must have one common output dimension")` This rejects an invalid state by raising ‘ValueError("weight rows must have one common output dimension")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 521: `if len(output_gradients) != rows or any(` This line opens or closes a multi-line Python expression inside the function pack_linear_backward_beats, which handles pack linear backward beats; the neighboring lines supply its arguments or elements.

Line 522: `len(row) != output_features for row in output_gradients` Inside the function pack_linear_backward_beats, which handles pack linear backward beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are len: the value named ’len’ in this block; row: one output-row position processed by the tensor tile; output_features: the value named ’output features’ in this block.

Line 523: `):` Inside the function pack_linear_backward_beats, which handles pack linear backward beats, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The expression is a literal or structural operator and has no hidden software state.

Line 524: `raise ValueError("output gradients must have shape [rows, output_features]")` This rejects an invalid state by raising ‘ValueError("output gradients must have shape [rows, output_features]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 526: `try:` This is explicit host-side failure handling inside the function pack_linear_backward_beats, which handles pack linear backward beats; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 527: `selected_mode = LinearGradientMode(mode)` This assigns ‘LinearGradientMode(mode)‘ to ‘selected_mode ‘ in host memory. Here selected_mode means the value named ’selected mode’ in this block. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter; mode: the value named ’mode’ in this block. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 528: `except ValueError as error:` This is explicit host-side failure handling inside the function pack_linear_backward_b which handles pack linear backward beats; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 529: `raise ValueError("linear backward mode must be BOTH, INPUT, or WEIGHT") from error` This rejects an invalid state by raising 'ValueError("linear backward mode must be BOTH, INPUT, or WEIGHT") from error'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 530: `words: list[int] = []` This assigns '[]' to 'words' in host memory. Here words means the value named 'words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 531: `if selected_mode != LinearGradientMode.INPUT:` This executes the following indented host-code branch only when 'selected_mode != LinearGradientMode.INPUT' is true. The important operands are selected_mode: the value named 'selected mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; INPUT: the value named 'input' in this block. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 532: `for row_index, row in enumerate(activations):` This iterates row_index, row over 'enumerate(activations)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 533: `padded = list(row) + [0] * (8 - input_features)` This assigns 'list(row) + [0] * (8 - input_features)' to 'padded ' in host memory. Here padded means the value named 'padded' in this block. The important operands are list: the value named 'list' in this block; row: one output-row position processed by the tensor tile; input_features: the value named 'input features' in this block. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 534: `for half in range(2):` This iterates half over 'range(2)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 535: `words.append(sum(` This statement continues through source line 542; the explanation uses the complete expression rather than only the first visual fragment. This calls 'words.append' with 'sum(_signed_bits(f"activation[{row_index},{ha * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))' inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 536: `_signed_bits(` This is continuation line 2 of the 8-line logical statement that begins at source line 535. Its exact fragment is '_signed_bits('. The important operands are _signed_bits: the value named 'signed bits' in this block. Read together, the complete operation is: This calls 'words.append' with 'sum(_signed_bits(f"activation[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))' inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 537: `f"activation[{row_index},{half * 4 + lane}]",` This is continuation line 3 of the 8-line logical statement that begins at source line 535. Its exact fragment is 'f"activation[{row_index},{half * 4 + lane}]",'. The important operands are f: the value named 'f' in this block; activation: an input feature value consumed by a neuron or multiply-accumulate operation; row_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'words.append' with 'sum(_signed_bits(f"activation[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))' inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 538: `padded[half * 4 + lane],` This is continuation line 4 of the 8-line logical statement that begins at source line 535. Its exact fragment is ‘padded[half * 4 + lane],‘. The important operands are padded: the value named ’padded’ in this block; half: the value named ’half’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"activation[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 539: `16,` This is continuation line 5 of the 8-line logical statement that begins at source line 535. Its exact fragment is ‘16,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"activation[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 540: `) << (16 * lane)` This is continuation line 6 of the 8-line logical statement that begins at source line 535. Its exact fragment is ‘) << (16 * lane)‘. The important operands are lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"activation[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 541: `for lane in range(4)` This is continuation line 7 of the 8-line logical statement that begins at source line 535. Its exact fragment is ‘for lane in range(4)‘. The important operands are for: the value named ’for’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named ’in’ in this block. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"activation[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 542: `))` This is continuation line 8 of the 8-line logical statement that begins at source line 535. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"activation[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 543: `if selected_mode != LinearGradientMode.WEIGHT:` This executes the following indented host-code branch only when ‘selected_mode != LinearGradientMode.WEIGHT‘ is true. The important operands are selected_mode: the value named ’selected mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 544: `for row_index, row in enumerate(weights):` This iterates row_index, row over ‘enumerate(weights)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 545: `padded = list(row) + [0] * (8 - output_features)` This assigns ‘list(row) + [0] * (8 - output_features)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are list: the value named ’list’ in this block; row: one output-row position processed by the tensor tile; output_features: the value named ’output features’ in this block. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 546: `for half in range(2):` This iterates half over ‘range(2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 547: `words.append(sum(` This statement continues through source line 554; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘words.append‘ with ‘sum(_signed_bits(f"weight[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 548: `_signed_bits(` This is continuation line 2 of the 8-line logical statement that begins at source line 547. Its exact fragment is ‘_signed_bits(‘. The important operands are _signed_bits: the value named ’signed bits’ in this block. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"weight[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 549: `f"weight[{row_index},{half * 4 + lane}]",` This is continuation line 3 of the 8-line logical statement that begins at source line 547. Its exact fragment is ‘f"weight[{row_index},{half * 4 + lane}]",‘. The important operands are f: the value named ’f’ in this block; weight: a learned coefficient multiplied by an activation; row_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"weight[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 550: `padded[half * 4 + lane],` This is continuation line 4 of the 8-line logical statement that begins at source line 547. Its exact fragment is ‘padded[half * 4 + lane],‘. The important operands are padded: the value named ’padded’ in this block; half: the value named ’half’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"weight[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 551: `16,` This is continuation line 5 of the 8-line logical statement that begins at source line 547. Its exact fragment is ‘16,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"weight[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 552: `) << (16 * lane)` This is continuation line 6 of the 8-line logical statement that begins at source line 547. Its exact fragment is ‘) << (16 * lane)‘. The important operands are lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"weight[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 553: `for lane in range(4)` This is continuation line 7 of the 8-line logical statement that begins at source line 547. Its exact fragment is ‘for lane in range(4)‘. The important operands are for: the value named ’for’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named ’in’ in this block. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"weight[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 554: `))` This is continuation line 8 of the 8-line logical statement that begins at source line 547. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"weight[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles

pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 555: `for row_index, row in enumerate(output_gradients):` This iterates row_index, row over ‘enumerate(output_gradients)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 556: `padded = list(row) + [0] * (8 - output_features)` This assigns ‘list(row) + [0] * (8 - output_features)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are list: the value named ’list’ in this block; row: one output-row position processed by the tensor tile; output_features: the value named ’output features’ in this block. Context: the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 557: `for half in range(2):` This iterates half over ‘range(2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_linear_backward_beats, which handles pack linear backward beats.

Line 558: `words.append(sum(` This statement continues through source line 565; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘words.append‘ with ‘sum(_signed_bits(f"output_gradient[{row_inde
* 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 559: `_signed_bits(` This is continuation line 2 of the 8-line logical statement that begins at source line 558. Its exact fragment is ‘_signed_bits(‘. The important operands are _signed_bits: the value named ’signed bits’ in this block. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"output_gradient[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 560: `f"output_gradient[{row_index},{half * 4 + lane}]",` This is continuation line 3 of the 8-line logical statement that begins at source line 558. Its exact fragment is ‘f"output_gradient[{row_index},{half * 4 + lane}]",‘. The important operands are f: the value named ’f’ in this block; output_gradient: the signed loss derivative used to update a learned parameter; row_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"output_gradient[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 561: `padded[half * 4 + lane],` This is continuation line 4 of the 8-line logical statement that begins at source line 558. Its exact fragment is ‘padded[half * 4 + lane],‘. The important operands are padded: the value named ’padded’ in this block; half: the value named ’half’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"output_gradient[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 562: `16,` This is continuation line 5 of the 8-line logical statement that begins at source line 558. Its exact fragment is ‘16,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"output_gradient[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 563: `) << (16 * lane)` This is continuation line 6 of the 8-line logical statement that begins at source line 558. Its exact fragment is ‘) << (16 * lane)‘. The important operands are lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"output_gradient[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 564: `for lane in range(4)` This is continuation line 7 of the 8-line logical statement that begins at source line 558. Its exact fragment is ‘for lane in range(4)‘. The important operands are for: the value named ’for’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named ’in’ in this block. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"output_gradient[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 565: `))` This is continuation line 8 of the 8-line logical statement that begins at source line 558. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘words.append‘ with ‘sum(_signed_bits(f"output_gradient[{row_index},{half * 4 + lane}]", padded[half * 4 + lane], 16,) << (16 * lane) for lane in range(4))‘ inside the function pack_linear_backward_beats, which handles pack linear backward beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 566: `return _finish_beats(words)` This returns ‘_finish_beats(words)‘ to the caller of the function pack_linear_backwar which handles pack linear backward beats, ending this invocation. The important operands are _finish_beats: the value named ’finish beats’ in this block; words: the value named ’words’ in this block.

Line 569: `def pack_lut_load_beats(entries: Sequence[tuple[int, int]]) -> PackedStreamBeats:` This starts function pack_lut_load_beats. Its indented body performs pack lut load beats as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 570: `if not entries:` This executes the following indented host-code branch only when ‘not entries‘ is true. The important operands are not: the value named ’not’ in this block; entries: the value named ’entries’ in this block. Context: the function pack_lut_load_beats, which handles pack lut load beats.

Line 571: `raise ValueError("LUT load requires at least one entry")` This rejects an invalid state by raising ‘ValueError("LUT load requires at least one entry")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 572: `words = []` This assigns ‘[]‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function pack_lut_load_beats, which handles pack lut load beats.

Line 573: `for index, (address, value) in enumerate(entries):` This iterates index, (address, value) over ‘enumerate(entries)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function pack_lut_load_beats, which handles pack lut load beats.

Line 574: `words.append(_unsigned(f"LUT address {index}", address, 8)` This statement continues through source line 575; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘words.append‘ with ‘_unsigned(f"LUT address {index}", address, 8) | (_signed_bits(f"LUT value {index}", value, 16) << 8)‘ inside the function pack_lut_load_beats, which handles pack lut load beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 575: `| (_signed_bits(f"LUT value {index}", value, 16) << 8))` This is continuation line 2 of the 2-line logical statement that begins at source line 574. Its exact fragment is '| (_signed_bits(f"LUT value {index}", value, 16) << 8))'. The important operands are _signed_bits: the value named 'signed bits' in this block; f: the value named 'f' in this block; LUT: the value named 'lut' in this block. Read together, the complete operation is: This calls 'words.append' with '_unsigned(f"LUT address {index}", address, 8) | (_signed_bits(f"LUT value {index}", value, 16) << 8)' inside the function pack_lut_load_beats, which handles pack lut load beats. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 576: `return _finish_beats(words)` This returns '_finish_beats(words)' to the caller of the function pack_lut_load_beats which handles pack lut load beats, ending this invocation. The important operands are _finish_beats: the value named 'finish beats' in this block; words: the value named 'words' in this block.

Line 579: `def pack_lut_eval_beat(indices: Sequence[int]) -> PackedStreamBeats:` This starts function pack_lut_eval_beat. Its indented body performs pack lut eval beat as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 580: `if len(indices) != 8:` This executes the following indented host-code branch only when 'len(indices) != 8' is true. The important operands are len: the value named 'len' in this block; indices: the value named 'indices' in this block. Context: the function pack_lut_eval_beat, which handles pack lut eval beat.

Line 581: `raise ValueError("LUT evaluate requires exactly eight indices")` This rejects an invalid state by raising 'ValueError("LUT evaluate requires exactly eight indices")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 582: `word = sum(_unsigned(f"LUT index {lane}", value, 8) << (8 * lane)` This statement continues through source line 583; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sum(_unsigned(f"LUT index {lane}", value, 8) << (8 * lane) for lane, value in enumerate(indices))' to 'word ' in host memory. Here word means the value named 'word' in this block. The important operands are sum: the value named 'sum' in this block; _unsigned: the value named 'unsigned' in this block; f: the value named 'f' in this block. Context: the function pack_lut_eval_beat, which handles pack lut eval beat.

Line 583: `for lane, value in enumerate(indices))` This is continuation line 2 of the 2-line logical statement that begins at source line 582. Its exact fragment is 'for lane, value in enumerate(indices))'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; value: the value named 'value' in this block. Read together, the complete operation is: This assigns 'sum(_unsigned(f"LUT index {lane}", value, 8) << (8 * lane) for lane, value in enumerate(indices))' to 'word ' in host memory. Here word means the value named 'word' in this block. The important operands are sum: the value named 'sum' in this block; _unsigned: the value named 'unsigned' in this block; f: the value named 'f' in this block. Context: the function pack_lut_eval_beat, which handles pack lut eval beat.

Line 584: `return [StreamBeat(word, True)]` This returns '[StreamBeat(word, True)]' to the caller of the function pack_lut_eval_beat, which handles pack lut eval beat, ending this invocation. The important operands are StreamBeat: the value named 'stream beat' in this block; word: the value named 'word' in this block.

Line 587: `def unpack_tensor_result(word: int) -> tuple[int, int, int, bool]:` This starts function unpack_tensor_result. Its indented body performs unpack tensor result as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 588: `value = _unsigned("tensor result word", word, 64)` This assigns '_unsigned("tensor result word", word, 64)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are _unsigned: the value named 'unsigned' in this block; tensor: the value named 'tensor' in this block; result: the value named 'result' in this block. Context: the function unpack_tensor_result, which handles unpack tensor result.

Line 589: `return (_decode_signed(value, 48), (value >> 48) & 0xF,` This statement continues through source line 590; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘(_decode_signed(value, 48), (value >> 48) & 0xF, (value >> 52) & 0x3F, bool((value >> 58) & 1))‘ to the caller of the function unpack_tensor_result, which handles unpack tensor result, ending this invocation. The important operands are _decode_signed: the value named ’decode signed’ in this block; value: the value named ’value’ in this block; bool: the value named ’bool’ in this block.

Line 590: `(value >> 52) & 0x3F, bool((value >> 58) & 1))` This is continuation line 2 of the 2-line logical statement that begins at source line 589. Its exact fragment is ‘(value >> 52) & 0x3F, bool((value >> 58) & 1))‘. The important operands are value: the value named ’value’ in this block; bool: the value named ’bool’ in this block. Read together, the complete operation is: This returns ‘(_decode_signed(value, 48), (value >> 48) & 0xF, (value >> 52) & 0x3F, bool((value >> 58) & 1))‘ to the caller of the function unpack_tensor_result, which handles unpack tensor result, ending this invocation. The important operands are _decode_signed: the value named ’decode signed’ in this block; value: the value named ’value’ in this block; bool: the value named ’bool’ in this block.

Line 593: `def unpack_alpha_result(word: int) -> tuple[int, int, int]:` This starts function unpack_alpha_result. Its indented body performs unpack alpha result as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 594: `value = _unsigned("alpha result word", word, 64)` This assigns ‘_unsigned("alpha result word", word, 64)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are _unsigned: the value named ’unsigned’ in this block; alpha: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; result: the value named ’result’ in this block. Context: the function unpack_alpha_result, which handles unpack alpha result.

Line 595: `return value & 0x3FFFF, (value >> 18) & 0x1FF, (value >> 27) & 0x1FF` This returns ‘value & 0x3FFFF, (value >> 18) & 0x1FF, (value >> 27) & 0x1FF‘ to the caller of the function unpack_alpha_result, which handles unpack alpha result, ending this invocation. The important operands are value: the value named ’value’ in this block.

Line 598: `def unpack_vector_result(words: Sequence[int]) -> tuple[int, ...]:` This starts function unpack_vector_result. Its indented body performs unpack vector result as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 599: `if len(words) != 4:` This executes the following indented host-code branch only when ‘len(words) != 4‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block. Context: the function unpack_vector_result, which handles unpack vector result.

Line 600: `raise ValueError("vector result requires four words")` This rejects an invalid state by raising ‘ValueError("vector result requires four words")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 601: `return tuple(_decode_signed(word >> shift, 32)` This statement continues through source line 602; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘tuple(_decode_signed(word >> shift, 32) for word in words for shift in (0, 32))‘ to the caller of the function unpack_vector_result, which handles unpack vector result, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; word: the value named ’word’ in this block.

Line 602: `for word in words for shift in (0, 32))` This is continuation line 2 of the 2-line logical statement that begins at source line 601. Its exact fragment is ‘for word in words for shift in (0, 32))‘. The important operands are for: the value named ’for’ in this block; word: the value named ’word’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This returns ‘tuple(_decode_signed(word >> shift, 32) for word in words for shift in (0, 32))‘ to the caller of the function unpack_vector_result, which handles unpack vector result, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; word: the value named ’word’ in this block.

Line 605: `def unpack_layernorm_result(word:  int) -> tuple[int, int, bool]:` This starts function unpack_layernorm
Its indented body performs unpack layernorm result as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 606: `value = _unsigned("LayerNorm result word", word, 64)` This assigns '_unsigned("LayerNorm result word", word, 64)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are _unsigned: the value named 'unsigned' in this block; LayerNorm: the value named 'layer norm' in this block; result: the value named 'result' in this block. Context: the function unpack_layernorm_result, which handles unpack layernorm result.

Line 607: `return _decode_signed(value, 32), (value >> 32) & 0xFFF, bool((value >> 44) & 1)` This returns '_decode_signed(value, 32), (value >> 32) & 0xFFF, bool((value >> 44) & 1)' to the caller of the function unpack_layernorm_result, which handles unpack layernorm result, ending this invocation. The important operands are _decode_signed: the value named 'decode signed' in this block; value: the value named 'value' in this block; bool: the value named 'bool' in this block.

Line 610: `def unpack_softmax_result(word:  int) -> tuple[int, int, bool]:` This starts function unpack_softmax_res
Its indented body performs unpack softmax result as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 611: `value = _unsigned("softmax result word", word, 64)` This assigns '_unsigned("softmax result word", word, 64)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are _unsigned: the value named 'unsigned' in this block; softmax: the value named 'softmax' in this block; result: the value named 'result' in this block. Context: the function unpack_softmax_result, which handles unpack softmax result.

Line 612: `return value & 0xFFFF, (value >> 16) & 0x1FF, bool((value >> 25) & 1)` This returns 'value & 0xFFFF, (value >> 16) & 0x1FF, bool((value >> 25) & 1)' to the caller of the function unpack_softmax_result, which handles unpack softmax result, ending this invocation. The important operands are value: the value named 'value' in this block; bool: the value named 'bool' in this block.

Line 615: `def unpack_lif_result(word:  int) -> tuple[int, int]:` This starts function unpack_lif_result. Its indented body performs unpack lif result as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 616: `value = _unsigned("LIF result word", word, 64)` This assigns '_unsigned("LIF result word", word, 64)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are _unsigned: the value named 'unsigned' in this block; LIF: the value named 'lif' in this block; result: the value named 'result' in this block. Context: the function unpack_lif_result, which handles unpack lif result.

Line 617: `return value & 0xFFFFFFFF, value >> 32` This returns 'value & 0xFFFFFFFF, value >> 32' to the caller of the function unpack_lif_result, which handles unpack lif result, ending this invocation. The important operands are value: the value named 'value' in this block.

Line 620: `def unpack_lif_trace(words:  Sequence[int]) -> tuple[int, int, tuple[int, ...]]:` This starts function unpack_lif_trace. Its indented body performs unpack lif trace as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 621: `"""Decode one traced LIF response:  metadata followed by 32 s24 membranes."""` This is executable documentation attached to the function unpack_lif_trace, which handles unpack lif trace. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 623: `if len(words) != 13:` This executes the following indented host-code branch only when 'len(words) != 13' is true. The important operands are len: the value named 'len' in this block; words: the value named 'words' in this block. Context: the function unpack_lif_trace, which handles unpack lif trace.

Line 624: `raise ValueError("a traced LIF result requires exactly thirteen words")` This rejects an invalid state by raising 'ValueError("a traced LIF result requires exactly thirteen words")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 625: `spikes, timestep = unpack_lif_result(words[0])` This assigns 'unpack_lif_result(words[0])' to 'spikes, timestep ' in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are unpack_lif_result: the value named 'unpack lif result' in this block; words: the value named 'words' in this block. Context: the function unpack_lif_trace, which handles unpack lif trace.

Line 626: `packed_membranes = 0` This assigns '0' to 'packed_membranes ' in host memory. Here packed_membranes means the persistent LIF neuron state carrying charge between timesteps. The expression is a literal or structural operator and has no hidden software state. Context: the function unpack_lif_trace, which handles unpack lif trace.

Line 627: `for index, word in enumerate(words[1:]):` This iterates index, word over 'enumerate(words[1:])'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function unpack_lif_trace, which handles unpack lif trace.

Line 628: `packed_membranes |= _unsigned(f"LIF trace word {index}", word, 64) << (64 * index)` Inside the function unpack_lif_trace, which handles unpack lif trace, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are packed_membranes: the persistent LIF neuron state carrying charge between timesteps; _unsigned: the value named 'unsigned' in this block; f: the value named 'f' in this block.

Line 629: `membranes = tuple(` This statement continues through source line 632; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'tuple(_decode_signed(packed_membranes >> (24 * lane), 24) for lane in range(32))' to 'membranes ' in host memory. Here membranes means the persistent LIF neuron state carrying charge between timesteps. The important operands are tuple: the value named 'tuple' in this block; _decode_signed: the value named 'decode signed' in this block; packed_membranes: the persistent LIF neuron state carrying charge between timesteps. Context: the function unpack_lif_trace, which handles unpack lif trace.

Line 630: `_decode_signed(packed_membranes >> (24 * lane), 24)` This is continuation line 2 of the 4-line logical statement that begins at source line 629. Its exact fragment is '_decode_signed(packed_membranes >> (24 * lane), 24)'. The important operands are _decode_signed: the value named 'decode signed' in this block; packed_membranes: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'tuple(_decode_signed(packed_membranes >> (24 * lane), 24) for lane in range(32))' to 'membranes ' in host memory. Here membranes means the persistent LIF neuron state carrying charge between timesteps. The important operands are tuple: the value named 'tuple' in this block; _decode_signed: the value named 'decode signed' in this block; packed_membranes: the persistent LIF neuron state carrying charge between timesteps. Context: the function unpack_lif_trace, which handles unpack lif trace.

Line 631: `for lane in range(32)` This is continuation line 3 of the 4-line logical statement that begins at source line 629. Its exact fragment is 'for lane in range(32)'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'tuple(_decode_signed(packed_membranes >> (24 * lane), 24) for lane in range(32))' to 'membranes ' in host memory. Here membranes means the persistent LIF neuron state carrying charge between timesteps. The important operands are tuple: the value named 'tuple' in this block; _decode_signed: the value named 'decode signed' in this block; packed_membranes: the persistent LIF neuron state carrying charge between timesteps. Context: the function unpack_lif_trace, which handles unpack lif trace.

Line 632: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 629. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'tuple(_decode_signed(packed_membranes >> (24 * lane), 24) for lane in range(32))' to 'membranes ' in host memory. Here membranes means the persistent LIF neuron state carrying charge between timesteps. The important operands are tuple: the value named 'tuple' in this block; _decode_signed: the value named 'decode signed' in this block; packed_membranes: the persistent LIF neuron state carrying charge between timesteps. Context: the function unpack_lif_trace, which handles unpack lif trace.

Line 633: `return spikes, timestep, membranes` This returns ‘spikes, timestep, membranes‘ to the caller of the function unpack_lif_trace, which handles unpack lif trace, ending this invocation. The important operands are spikes: a binary event produced when a neuron crosses its threshold; timestep: one discrete SNN update instant; membranes: the persistent LIF neuron state carrying charge between timesteps.

Line 636: `def unpack_optimizer_result(words: Sequence[int]) -> tuple[tuple[int, ...], tuple[int, ...]]:` This starts function unpack_optimizer_result. Its indented body performs unpack optimizer result as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 637: `if len(words) != 8:` This executes the following indented host-code branch only when ‘len(words) != 8‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block. Context: the function unpack_optimizer_result, which handles unpack optimizer result.

Line 638: `raise ValueError("optimizer result requires eight words")` This rejects an invalid state by raising ‘ValueError("optimizer result requires eight words")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 639: `weights = tuple(_decode_signed(word, 16) for word in words)` This assigns ‘tuple(_decode_signed(word, 16) for word in words)‘ to ‘weights ‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; word: the value named ’word’ in this block. Context: the function unpack_optimizer_result, which handles unpack optimizer result.

Line 640: `velocities = tuple(_decode_signed(word >> 16, 24) for word in words)` This assigns ‘tuple(_decode_signed >> 16, 24) for word in words)‘ to ‘velocities ‘ in host memory. Here velocities means the value named ’velocities’ in this block. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; word: the value named ’word’ in this block. Context: the function unpack_optimizer_result, which handles unpack optimizer result.

Line 641: `return weights, velocities` This returns ‘weights, velocities‘ to the caller of the function unpack_optimizer_result, which handles unpack optimizer result, ending this invocation. The important operands are weights: a learned coefficient multiplied by an activation; velocities: the value named ’velocities’ in this block.

Line 644: `def unpack_linear_backward_result(` This starts function unpack_linear_backward_result. Its indented body performs unpack linear backward result as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 645: `words: Sequence[int],` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are words: the value named ’words’ in this block; Sequence: the value named ’sequence’ in this block.

Line 646: `rows: int,` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are rows: one output-row position processed by the tensor tile.

Line 647: `input_features: int,` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are input_features: the value named ’input features’ in this block.

Line 648: `output_features: int,` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are output_features: the value named ’output features’ in this block.

Line 649: `*,` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The expression is a literal or structural operator and has no hidden software state.

Line 650: `mode: LinearGradientMode = LinearGradientMode.BOTH,` This assigns ‘LinearGradientMode.BOTH,‘ to ‘mode‘ in host memory. Here mode means the value named ’mode’ in this block. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter; BOTH: the value named ’both’ in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 651: `) -> tuple[tuple[tuple[int, ...], ...], tuple[tuple[int, ...], ...]]:` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are tuple: the value named ’tuple’ in this block.

Line 652: `"""Decode padded s32 dX rows and packed s24 dW rows from the RTL."""` This is executable documentation attached to the function unpack_linear_backward_result, which handles unpack linear backward result. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 654: `if not 1 <= rows <= 8 or not 1 <= input_features <= 8 or not 1 <= output_features <= 8:` This executes the following indented host-code branch only when ‘not 1 <= rows <= 8 or not 1 <= input_features <= 8 or not 1 <= output_features <= 8‘ is true. The important operands are not: the value named ’not’ in this block; rows: one output-row position processed by the tensor tile; or: the value named ’or’ in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 655: `raise ValueError("linear backward tile dimensions must be in [1,8]")` This rejects an invalid state by raising ‘ValueError("linear backward tile dimensions must be in [1,8]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 656: `try:` This is explicit host-side failure handling inside the function unpack_linear_backward_result, which handles unpack linear backward result; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 657: `selected_mode = LinearGradientMode(mode)` This assigns ‘LinearGradientMode(mode)‘ to ‘selected_mode ‘ in host memory. Here selected_mode means the value named ’selected mode’ in this block. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter; mode: the value named ’mode’ in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 658: `except ValueError as error:` This is explicit host-side failure handling inside the function unpack_linear_backward which handles unpack linear backward result; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 659: `raise ValueError("linear backward mode must be BOTH, INPUT, or WEIGHT") from error` This rejects an invalid state by raising ‘ValueError("linear backward mode must be BOTH, INPUT, or WEIGHT") from error‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 660: `expected_words = 0` This assigns ‘0‘ to ‘expected_words ‘ in host memory. Here expected_words means the value named ’expected words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 661: `if selected_mode != LinearGradientMode.WEIGHT:` This executes the following indented host-code branch only when ‘selected_mode != LinearGradientMode.WEIGHT‘ is true. The important operands are selected_mode: the value named ’selected mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 662: `expected_words += 4 * rows` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are expected_words: the value named 'expected words' in this block; rows: one output-row position processed by the tensor tile.

Line 663: `if selected_mode != LinearGradientMode.INPUT:` This executes the following indented host-code branch only when ‘selected_mode != LinearGradientMode.INPUT‘ is true. The important operands are selected_mode: the value named 'selected mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; INPUT: the value named 'input' in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 664: `expected_words += 3 * input_features` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are expected_words: the value named 'expected words' in this block; input_features: the value named 'input features' in this block.

Line 665: `if len(words) != expected_words:` This executes the following indented host-code branch only when ‘len(words) != expected_words‘ is true. The important operands are len: the value named 'len' in this block; words: the value named 'words' in this block; expected_words: the value named 'expected words' in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 666: `raise ValueError(` This statement continues through source line 668; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘ValueError(f"linear backward result requires {expected_words} words; received {len(words)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 667: `f"linear backward result requires {expected_words} words; received {len(words)}"` This is continuation line 2 of the 3-line logical statement that begins at source line 666. Its exact fragment is ‘f"linear backward result requires {expected_words} words; received {len(words)}"‘. The important operands are f: the value named 'f' in this block; linear: the value named 'linear' in this block; backward: the value named 'backward' in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘ValueError(f"linear backward result requires {expected_words} words; received {len(words)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 668: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 666. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This rejects an invalid state by raising ‘ValueError(f"linear backward result requires {expected_words} words; received {len(words)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 669: `input_gradients: list[tuple[int, ...]] = []` This assigns ‘[]‘ to ‘input_gradients‘ in host memory. Here input_gradients means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 670: `cursor = 0` This assigns ‘0‘ to ‘cursor ‘ in host memory. Here cursor means the value named 'cursor' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 671: `if selected_mode != LinearGradientMode.WEIGHT:` This executes the following indented host-code branch only when ‘selected_mode != LinearGradientMode.WEIGHT‘ is true. The important operands are selected_mode: the value named 'selected mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 672: `for _ in range(rows):` This iterates _ over ‘range(rows)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 673: `lanes = tuple(` This statement continues through source line 676; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘tuple(_decode_signed(words[cursor + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(input_features))‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; words: the value named ’words’ in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 674: `_decode_signed(words[cursor + lane // 2] >> (32 * (lane % 2)), 32)` This is continuation line 2 of the 4-line logical statement that begins at source line 673. Its exact fragment is ‘_decode_signed(words[cursor + lane // 2] >> (32 * (lane % 2)), 32)‘. The important operands are _decode_signed: the value named ’decode signed’ in this block; words: the value named ’words’ in this block; cursor: the value named ’cursor’ in this block. Read together, the complete operation is: This assigns ‘tuple(_decode_signed(words[cursor + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(input_features))‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; words: the value named ’words’ in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 675: `for lane in range(input_features)` This is continuation line 3 of the 4-line logical statement that begins at source line 673. Its exact fragment is ‘for lane in range(input_features)‘. The important operands are for: the value named ’for’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘tuple(_decode_signed(words[cursor + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(input_features))‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; words: the value named ’words’ in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 676: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 673. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘tuple(_decode_signed(words[cursor + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(input_features))‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; words: the value named ’words’ in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 677: `input_gradients.append(lanes)` This calls ‘input_gradients.append‘ with ‘lanes‘ inside the function unpack_linear_backward_result, which handles unpack linear backward result. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 678: `cursor += 4` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are cursor: the value named ’cursor’ in this block.

Line 679: `weight_gradients: list[tuple[int, ...]] = []` This assigns ‘[]‘ to ‘weight_gradients‘ in host memory. Here weight_gradients means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 680: `if selected_mode != LinearGradientMode.INPUT:` This executes the following indented host-code branch only when ‘selected_mode != LinearGradientMode.INPUT‘ is true. The important operands are selected_mode: the value named ’selected mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; INPUT: the value named ’input’ in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 681: `for _ in range(input_features):` This iterates _ over ‘range(input_features)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 682: `packed = sum(words[cursor + index] << (64 * index) for index in range(3))` This assigns ‘sum(words[curso + index] << (64 * index) for index in range(3))‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are sum: the value named ’sum’ in this block; words: the value named ’words’ in this block; cursor: the value named ’cursor’ in this block. Context: the function unpack_linear_backward_result, which handles unpack linear backward result.

Line 683: `weight_gradients.append(tuple(` This statement continues through source line 686; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘weight_gradients.append‘ with ‘tuple(_decode_signed(packed >> (24 * lane), 24) for lane in range(output_features))‘ inside the function unpack_linear_backward_result, which handles unpack linear backward result. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 684: `_decode_signed(packed >> (24 * lane), 24)` This is continuation line 2 of the 4-line logical statement that begins at source line 683. Its exact fragment is ‘_decode_signed(packed >> (24 * lane), 24)‘. The important operands are _decode_signed: the value named ’decode signed’ in this block; packed: the value named ’packed’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘weight_gradients.append‘ with ‘tuple(_decode_signed(packed >> (24 * lane), 24) for lane in range(output_features))‘ inside the function unpack_linear_backward_result, which handles unpack linear backward result. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 685: `for lane in range(output_features)` This is continuation line 3 of the 4-line logical statement that begins at source line 683. Its exact fragment is ‘for lane in range(output_features)‘. The important operands are for: the value named ’for’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named ’in’ in this block. Read together, the complete operation is: This calls ‘weight_gradients.append‘ with ‘tuple(_decode_signed(packed >> (24 * lane), 24) for lane in range(output_features))‘ inside the function unpack_linear_backward_result, which handles unpack linear backward result. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 686: `))` This is continuation line 4 of the 4-line logical statement that begins at source line 683. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘weight_gradients.append‘ with ‘tuple(_decode_signed(packed >> (24 * lane), 24) for lane in range(output_features))‘ inside the function unpack_linear_backward_result, which handles unpack linear backward result. The call advances the host-side job that defines command bit fields, descriptor records, packed streams, and result decoders.

Line 687: `cursor += 3` Inside the function unpack_linear_backward_result, which handles unpack linear backward result, this statement advances the host implementation that defines command bit fields, descriptor records, packed streams, and result decoders. The important operands are cursor: the value named ’cursor’ in this block.

Line 688: `return tuple(input_gradients), tuple(weight_gradients)` This returns ‘tuple(input_gradients), tuple(weight_gradients)‘ to the caller of the function unpack_linear_backward_result, which handles unpack linear backward result, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; input_gradients: the signed loss derivative used to update a learned parameter; weight_gradients: a learned coefficient multiplied by an activation.

Line 691: `def unpack_lut_result(words: Sequence[int]) -> tuple[int, ...]:` This starts function unpack_lut_result. Its indented body performs unpack lut result as part of the file that defines command bit fields, descriptor records, packed streams, and result decoders; it runs only when called.

Line 692: `if len(words) != 2:` This executes the following indented host-code branch only when ‘len(words) != 2‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block. Context: the function unpack_lut_result, which handles unpack lut result.

Line 693: `raise ValueError("LUT result requires two words")` This rejects an invalid state by raising ‘ValueError("LUT result requires two words")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 694: `return tuple(_decode_signed(word >> (16 * lane), 16)` This statement continues through source line 695; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘tuple(_decode_signed(word >> (16 * lane), 16) for word in words for lane in range(4))‘ to the caller of the function unpack_lut_result, which handles unpack lut result, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; word: the value named ’word’ in this block.

Line 695: `for word in words for lane in range(4))` This is continuation line 2 of the 2-line logical statement that begins at source line 694. Its exact fragment is ‘for word in words for lane in range(4))‘. The important operands are for: the value named ’for’ in this block; word: the value named ’word’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This returns ‘tuple(_decode_signed(word >> (16 * lane), 16) for word in words for lane in range(4))‘ to the caller of the function unpack_lut_result, which handles unpack lut result, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; _decode_signed: the value named ’decode signed’ in this block; word: the value named ’word’ in this block.

37 software/neuraedge_pynq.py

Role in the complete system: controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

37.1 Beginner architecture map

Dataflow responsibility The board driver loads the BIT/HWH overlay, allocates physically contiguous DMA buffers, writes AXI-Lite registers, arms receive before send, waits with timeouts, checks hardware errors, and returns telemetry beside result words.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: RegisterMap, PerformanceSnapshot, NeuraEdgeHardwareError, NeuraEdgeTimeout, NeuraEdgeDriver. Functions: ___init___, from_bitfile, _check_identity, target_clock_hz, capabilities, status, _signed32, performance_snapshot, _control_value, clear_wrapper_status, _validate_input_beats, _release_buffer, _wait_dma_chan __wait_idle, execute, execute_program, _partition_program, _program_sizes, _fill_program_buffer, _execute_program_pa Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

37.2 Complete source

```
"""PYNQ DMA/MMIO driver for the complete NeuraEdge overlay."""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from time import monotonic, sleep
from typing import Callable, Sequence

import numpy as np

from neuraedge_protocol import (
    Command128,
    PackedStreamBeats,
    ProgramRecord,
    StreamBeat,
)


@dataclass(frozen=True)
class RegisterMap:
    control: int = 0x00
    status: int = 0x04
    active_opcode: int = 0x08
    command_count: int = 0x0C
    command0: int = 0x10
    input_beats: int = 0x20
    output_beats: int = 0x24
    target_clock: int = 0x28
    capabilities: int = 0x2C
    version: int = 0x30
    identity: int = 0x34
    tensor_loaded: int = 0x38
    tensor_active: int = 0x3C
    tensor_cycles: int = 0x40
    tensor_skipped: int = 0x44
    dense_macs_low: int = 0x48
    dense_macs_high: int = 0x4C
    spike_adds_low: int = 0x50
    spike_adds_high: int = 0x54
    layernorm_cycles: int = 0x58
    layernorm_mean: int = 0x5C
    layernorm_variance: int = 0x60
    layernorm_sigma: int = 0x64
    softmax_cycles: int = 0x68
    softmax_maximum: int = 0x6C
```

```
    softmax_denominator: int = 0x70
    lif_spikes_low: int = 0x74
    lif_spikes_high: int = 0x78
    optimizer_updates: int = 0x7C
    lut_entries: int = 0x80
    layernorm_reciprocal: int = 0x84
    softmax_reciprocal: int = 0x88
    program_records: int = 0x8C
    program_output_beats: int = 0x90
    training_tiles: int = 0x94
    training_cycles: int = 0x98


@dataclass(frozen=True)
class PerformanceSnapshot:
    tensor_loaded_activations: int
    tensor_loaded_weights: int
    tensor_active_issues: int
    tensor_compute_cycles: int
    tensor_skipped_issues: int
    dense_macs: int
    spike_adds: int
    layernorm_cycles: int
    layernorm_mean: int
    layernorm_variance: int
    layernorm_sigma: int
    layernorm_reciprocal_q31: int
    softmax_cycles: int
    softmax_maximum: int
    softmax_denominator: int
    softmax_reciprocal_q31: int
    lif_total_spikes: int
    optimizer_updates: int
    lut_loaded_entries: int
    training_tiles: int
    training_cycles: int


class NeuraEdgeHardwareError(RuntimeError):
    """Raised when the RTL reports a sticky protocol or configuration error."""


class NeuraEdgeTimeout(TimeoutError):
    """Raised when a DMA channel or accelerator command exceeds its deadline."""


class NeuraEdgeDriver:
    STATUS_BUSY = 1 << 0
    STATUS_ERROR = 1 << 1
    STATUS_DONE = 1 << 2
    STATUS_PENDING = 1 << 3
    STATUS_PROGRAM_ACTIVE = 1 << 5
    EXPECTED_ID = 0x4E455552
    EXPECTED_ABI_MAJOR = 1
    MAX_DMA_TRANSFER_BYTES = (1 << 26) - 1
    PACK_COPY_WORDS = 1 << 16

    def __init__(self, mmio, dma, allocator: Callable[..., object], *, interrupt_enable: bool = True):
        self.mmio = mmio
        self.dma = dma
        self.allocator = allocator
        self.registers = RegisterMap()
        self.interrupt_enable = bool(interrupt_enable)
        self._check_identity()

    @classmethod
    def from_bitfile(
        cls,
        bitfile: str | Path,
        *,
        download: bool = True,
        accelerator_name: str = "accelerator",
        dma_name: str = "dma",
    ) -> "NeuraEdgeDriver":
        try:
            from pynq import Overlay, allocate
        except ImportError as error:
            raise RuntimeError("the pynq package is required on the PYNQ-Z2 board") from error

        overlay = Overlay(str(Path(bitfile)), download=download)
        accelerator = getattr(overlay, accelerator_name)
        dma = getattr(overlay, dma_name)
        driver = cls(accelerator.mmio, dma, allocate)
        driver.overlay = overlay
        return driver

    def _check_identity(self) -> None:
```

```
        identity = int(self.mmio.read(self.registers.identity))
        version = int(self.mmio.read(self.registers.version))
        if identity != self.EXPECTED_ID:
            raise NeuraEdgeHardwareError(
                f"unexpected accelerator identity 0x{identity:08x}; expected 0x{self.EXPECTED_ID:08x}"
            )
        if version >> 16 != self.EXPECTED_ABI_MAJOR:
            raise NeuraEdgeHardwareError(
                f"unsupported accelerator ABI 0x{version:08x}; expected major {self.EXPECTED_ABI_MAJOR}"
            )

    @property
    def target_clock_hz(self) -> int:
        return int(self.mmio.read(self.registers.target_clock))

    @property
    def capabilities(self) -> int:
        return int(self.mmio.read(self.registers.capabilities))

    def status(self) -> int:
        return int(self.mmio.read(self.registers.status))

    @staticmethod
    def _signed32(value: int) -> int:
        masked = int(value) & 0xFFFFFFFF
        return masked - (1 << 32) if masked & (1 << 31) else masked

    def performance_snapshot(self) -> PerformanceSnapshot:
        """Read one coherent-enough diagnostic snapshot while the core is idle."""
        if self.status() & (self.STATUS_BUSY | self.STATUS_PENDING):
            raise NeuraEdgeHardwareError("performance counters must be sampled while the accelerator is idle")
        registers = self.registers
        loaded = int(self.mmio.read(registers.tensor_loaded))
        dense_macs = int(self.mmio.read(registers.dense_macs_low))
        dense_macs |= int(self.mmio.read(registers.dense_macs_high)) << 32
        spike_adds = int(self.mmio.read(registers.spike_adds_low))
        spike_adds |= int(self.mmio.read(registers.spike_adds_high)) << 32
        lif_spikes = int(self.mmio.read(registers.lif_spikes_low))
        lif_spikes |= int(self.mmio.read(registers.lif_spikes_high)) << 32
        return PerformanceSnapshot(
            loaded & 0xFFF,
            (loaded >> 16) & 0xFFF,
            int(self.mmio.read(registers.tensor_active)) & 0xFFF,
            int(self.mmio.read(registers.tensor_cycles)),
            int(self.mmio.read(registers.tensor_skipped)),
            dense_macs,
            spike_adds,
            int(self.mmio.read(registers.layernorm_cycles)),
            self._signed32(self.mmio.read(registers.layernorm_mean)),
            int(self.mmio.read(registers.layernorm_variance)),
            int(self.mmio.read(registers.layernorm_sigma)),
            int(self.mmio.read(registers.layernorm_reciprocal)),
            int(self.mmio.read(registers.softmax_cycles)),
            self._signed32(self.mmio.read(registers.softmax_maximum)),
            int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF,
            int(self.mmio.read(registers.softmax_reciprocal)),
            lif_spikes,
            int(self.mmio.read(registers.optimizer_updates)),
            int(self.mmio.read(registers.lut_entries)) & 0x1FF,
            int(self.mmio.read(registers.training_tiles)),
            int(self.mmio.read(registers.training_cycles)),
        )

    def _control_value(self, transient_bits: int = 0) -> int:
        return ((1 << 8) if self.interrupt_enable else 0) | transient_bits

    def clear_wrapper_status(self) -> None:
        self.mmio.write(self.registers.control, self._control_value(1 << 1))

    @staticmethod
    def _validate_input_beats(beats: Sequence[StreamBeat]) -> None:
        if isinstance(beats, PackedStreamBeats):
            return
        for index, beat in enumerate(beats):
            if beat.last != (index == len(beats) - 1):
                raise ValueError("input StreamBeat.last must be true only on the final beat")

    @staticmethod
    def _release_buffer(buffer) -> None:
        if buffer is None:
            return
        if hasattr(buffer, "freebuffer"):
            buffer.freebuffer()
        elif hasattr(buffer, "close"):
            buffer.close()

    @staticmethod
```

```
    def _wait_dma_channel(channel, deadline: float, name: str) -> None:
        while not bool(channel.idle):
            if monotonic() >= deadline:
                if hasattr(channel, "stop"):
                    channel.stop()
                raise NeuraEdgeTimeout(f"{name} DMA channel timed out")
            sleep(0.0005)
        channel.wait()

    def _wait_idle(self, deadline: float) -> None:
        while self.status() & (self.STATUS_BUSY | self.STATUS_PENDING):
            if monotonic() >= deadline:
                raise NeuraEdgeTimeout("accelerator did not become idle")
            sleep(0.0005)

    def execute(
        self,
        command: Command128,
        input_beats: Sequence[StreamBeat] = (),
        *,
        output_beats: int = 0,
        timeout_seconds: float = 5.0,
    ) -> tuple[int, ...]:
        if timeout_seconds <= 0:
            raise ValueError("timeout_seconds must be positive")
        if output_beats < 0:
            raise ValueError("output_beats must be non-negative")
        self._validate_input_beats(input_beats)
        deadline = monotonic() + timeout_seconds
        self._wait_idle(deadline)

        input_buffer = None
        output_buffer = None
        try:
            if input_beats:
                input_buffer = self.allocator(shape=(len(input_beats),), dtype=np.uint64)
                if isinstance(input_beats, PackedStreamBeats):
                    input_buffer[:] = np.frombuffer(input_beats.words, dtype=np.uint64)
                else:
                    input_buffer[:] = np.fromiter(
                        (beat.data for beat in input_beats),
                        dtype=np.uint64,
                        count=len(input_beats),
                    )
                if hasattr(input_buffer, "flush"):
                    input_buffer.flush()
            if output_beats:
                output_buffer = self.allocator(shape=(output_beats,), dtype=np.uint64)
                self.dma.recvchannel.transfer(output_buffer, nbytes=output_beats * 8)

            self.clear_wrapper_status()
            for index, word in enumerate(command.words_le):
                self.mmio.write(self.registers.command0 + 4 * index, word)
            self.mmio.write(self.registers.control, self._control_value(1))

            if input_buffer is not None:
                self.dma.sendchannel.transfer(input_buffer, nbytes=len(input_beats) * 8)
                self._wait_dma_channel(self.dma.sendchannel, deadline, "MM2S")
            if output_buffer is not None:
                self._wait_dma_channel(self.dma.recvchannel, deadline, "S2MM")

            while True:
                status = self.status()
                if status & self.STATUS_ERROR:
                    raise NeuraEdgeHardwareError(
                        f"accelerator rejected opcode {int(command.opcode):x}; status=0x{status:08x}"
                    )
                if status & self.STATUS_DONE:
                    break
                if monotonic() >= deadline:
                    raise NeuraEdgeTimeout(f"opcode {int(command.opcode):x} did not complete")
                sleep(0.0005)

            if output_buffer is None:
                return ()
            if hasattr(output_buffer, "invalidate"):
                output_buffer.invalidate()
            return tuple(int(value) for value in output_buffer[:output_beats])
        finally:
            self._release_buffer(input_buffer)
            self._release_buffer(output_buffer)

    def execute_program(
        self,
        records: Sequence[ProgramRecord],
        *,
        timeout_seconds: float = 30.0,
```

```
    ) -> tuple[int, ...]:
        """Execute records in the minimum number of legal descriptor packets."""

        if timeout_seconds <= 0:
            raise ValueError("timeout_seconds must be positive")
        if not records:
            raise ValueError("a descriptor program must contain at least one record")
        deadline = monotonic() + timeout_seconds
        output: list[int] = []
        for packet in self._partition_program(records):
            output.extend(self._execute_program_packet(packet, deadline))
        return tuple(output)

    def _partition_program(
        self,
        records: Sequence[ProgramRecord],
    ) -> tuple[tuple[ProgramRecord, ...], ...]:
        packets: list[tuple[ProgramRecord, ...]] = []
        current: list[ProgramRecord] = []
        current_input_bytes = 0
        current_output_bytes = 0
        for record in records:
            record_input_bytes = (3 + len(record.input_beats)) * 8
            record_output_bytes = record.output_beats * 8
            if (record_input_bytes > self.MAX_DMA_TRANSFER_BYTES
                    or record_output_bytes > self.MAX_DMA_TRANSFER_BYTES):
                raise ValueError("one descriptor record exceeds the configured AXI DMA length")
            if current and (
                    current_input_bytes + record_input_bytes > self.MAX_DMA_TRANSFER_BYTES
                    or current_output_bytes + record_output_bytes > self.MAX_DMA_TRANSFER_BYTES):
                packets.append(tuple(current))
                current = []
                current_input_bytes = 0
                current_output_bytes = 0
            current.append(record)
            current_input_bytes += record_input_bytes
            current_output_bytes += record_output_bytes
        packets.append(tuple(current))
        return tuple(packets)

    @staticmethod
    def _program_sizes(records: Sequence[ProgramRecord]) -> tuple[int, int]:
        input_words = sum(3 + len(record.input_beats) for record in records)
        output_words = sum(record.output_beats for record in records)
        return input_words, output_words

    def _fill_program_buffer(self, buffer, records: Sequence[ProgramRecord]) -> None:
        cursor = 0
        for record_index, record in enumerate(records):
            header = len(record.input_beats)
            header |= record.output_beats << 32
            header |= int(record_index == len(records) - 1) << 63
            buffer[cursor] = header
            buffer[cursor + 1] = record.command.value & ((1 << 64) - 1)
            buffer[cursor + 2] = record.command.value >> 64
            cursor += 3
            if isinstance(record.input_beats, PackedStreamBeats):
                words = np.frombuffer(record.input_beats.words, dtype=np.uint64)
                buffer[cursor:cursor + len(words)] = words
                cursor += len(words)
                continue
            for start in range(0, len(record.input_beats), self.PACK_COPY_WORDS):
                beats = record.input_beats[start:start + self.PACK_COPY_WORDS]
                words = np.fromiter(
                    (beat.data for beat in beats), dtype=np.uint64, count=len(beats))
                buffer[cursor:cursor + len(beats)] = words
                cursor += len(beats)
        if cursor != len(buffer):
            raise RuntimeError(f"packed {cursor} descriptor words into a {len(buffer)}-word buffer")

    def _execute_program_packet(
        self,
        records: Sequence[ProgramRecord],
        deadline: float,
    ) -> tuple[int, ...]:
        input_words, output_beats = self._program_sizes(records)
        self._wait_idle(deadline)
        input_buffer = None
        output_buffer = None
        try:
            input_buffer = self.allocator(shape=(input_words,), dtype=np.uint64)
            self._fill_program_buffer(input_buffer, records)
            if hasattr(input_buffer, "flush"):
                input_buffer.flush()
            if output_beats:
                output_buffer = self.allocator(shape=(output_beats,), dtype=np.uint64)
                self.dma.recvchannel.transfer(output_buffer, nbytes=output_beats * 8)
```

```
            self.clear_wrapper_status()
            self.mmio.write(self.registers.program_output_beats, output_beats)
            self.mmio.write(self.registers.control, self._control_value(1 << 2))
            self.dma.sendchannel.transfer(input_buffer, nbytes=input_words * 8)
            self._wait_dma_channel(self.dma.sendchannel, deadline, "program MM2S")
            if output_buffer is not None:
                self._wait_dma_channel(self.dma.recvchannel, deadline, "program S2MM")

            while True:
                status = self.status()
                if status & self.STATUS_ERROR:
                    raise NeuraEdgeHardwareError(
                        f"descriptor program failed; status=0x{status:08x}")
                if status & self.STATUS_DONE:
                    break
                if monotonic() >= deadline:
                    raise NeuraEdgeTimeout("descriptor program did not complete")
                sleep(0.0005)
            completed = int(self.mmio.read(self.registers.program_records))
            if completed != len(records):
                raise NeuraEdgeHardwareError(
                    f"descriptor program completed {completed} records; expected {len(records)}")
            if output_buffer is None:
                return ()
            if hasattr(output_buffer, "invalidate"):
                output_buffer.invalidate()
            return tuple(int(value) for value in output_buffer[:output_beats])
        finally:
            self._release_buffer(input_buffer)
            self._release_buffer(output_buffer)
```

37.3 Line annotations

Line 1: `"""PYNQ DMA/MMIO driver for the complete NeuraEdge overlay."""` This is executable documentation attached to the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `from pathlib import Path` This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from time import monotonic, sleep` This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `from typing import Callable, Sequence` This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `import numpy as np` This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `from neuraedge_protocol import (` This statement continues through source line 17; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `Command128,` This is continuation line 2 of the 6-line logical statement that begins at source line 12. Its exact fragment is 'Command128,'. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `PackedStreamBeats,` This is continuation line 3 of the 6-line logical statement that begins at source line 12. Its exact fragment is 'PackedStreamBeats,'. The important operands are PackedStreamBeats: the value named 'packed stream beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `ProgramRecord,` This is continuation line 4 of the 6-line logical statement that begins at source line 12. Its exact fragment is 'ProgramRecord,'. The important operands are ProgramRecord: the value named 'program record' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `StreamBeat,` This is continuation line 5 of the 6-line logical statement that begins at source line 12. Its exact fragment is 'StreamBeat,'. The important operands are StreamBeat: the value named 'stream beat' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 12. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it creates no FPGA circuit by itself.

Line 21: `class RegisterMap:` This defines class RegisterMap, which groups data and methods for register map. Instances exist in host memory and organize the flow that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 22: `control: int = 0x00` This assigns '0x00' to 'control' in host memory. Here control means the value named 'control' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 23: `status: int = 0x04` This assigns '0x04' to 'status' in host memory. Here status means the value named 'status' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 24: `active_opcode: int = 0x08` This assigns '0x08' to 'active_opcode' in host memory. Here active_opcode means the value named 'active opcode' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 25: `command_count: int = 0x0C` This assigns ‘0x0C‘ to ‘command_count‘ in host memory. Here command_count means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 26: `command0: int = 0x10` This assigns ‘0x10‘ to ‘command0‘ in host memory. Here command0 means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 27: `input_beats: int = 0x20` This assigns ‘0x20‘ to ‘input_beats‘ in host memory. Here input_beats means the value named 'input beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 28: `output_beats: int = 0x24` This assigns ‘0x24‘ to ‘output_beats‘ in host memory. Here output_beats means the value named 'output beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 29: `target_clock: int = 0x28` This assigns ‘0x28‘ to ‘target_clock‘ in host memory. Here target_clock means the value named 'target clock' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 30: `capabilities: int = 0x2C` This assigns ‘0x2C‘ to ‘capabilities‘ in host memory. Here capabilities means the value named 'capabilities' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 31: `version: int = 0x30` This assigns ‘0x30‘ to ‘version‘ in host memory. Here version means the value named 'version' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 32: `identity: int = 0x34` This assigns ‘0x34‘ to ‘identity‘ in host memory. Here identity means the value named 'identity' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 33: `tensor_loaded: int = 0x38` This assigns ‘0x38‘ to ‘tensor_loaded‘ in host memory. Here tensor_loaded means the value named 'tensor loaded' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 34: `tensor_active: int = 0x3C` This assigns ‘0x3C‘ to ‘tensor_active‘ in host memory. Here tensor_active means the value named 'tensor active' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 35: `tensor_cycles: int = 0x40` This assigns ‘0x40‘ to ‘tensor_cycles‘ in host memory. Here tensor_cycles means the value named 'tensor cycles' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 36: `tensor_skipped: int = 0x44` This assigns ‘0x44‘ to ‘tensor_skipped‘ in host memory. Here tensor_skipped means the value named 'tensor skipped' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 37: `dense_macs_low: int = 0x48` This assigns ‘0x48‘ to ‘dense_macs_low‘ in host memory. Here dense_macs_low means a count of ordinary activation-times-weight operations actually issued. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 38: `dense_macs_high: int = 0x4C` This assigns ‘0x4C‘ to ‘dense_macs_high‘ in host memory. Here dense_macs_high means a count of ordinary activation-times-weight operations actually issued. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 39: `spike_adds_low: int = 0x50` This assigns ‘0x50‘ to ‘spike_adds_low‘ in host memory. Here spike_adds_low means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 40: `spike_adds_high: int = 0x54` This assigns ‘0x54‘ to ‘spike_adds_high‘ in host memory. Here spike_adds_high means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 41: `layernorm_cycles: int = 0x58` This assigns ‘0x58‘ to ‘layernorm_cycles‘ in host memory. Here layernorm_cycles means the value named ’layernorm cycles’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 42: `layernorm_mean: int = 0x5C` This assigns ‘0x5C‘ to ‘layernorm_mean‘ in host memory. Here layernorm_mean means the arithmetic average used by normalization. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 43: `layernorm_variance: int = 0x60` This assigns ‘0x60‘ to ‘layernorm_variance‘ in host memory. Here layernorm_variance means the average squared distance from the mean. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 44: `layernorm_sigma: int = 0x64` This assigns ‘0x64‘ to ‘layernorm_sigma‘ in host memory. Here layernorm_sigma means the value named ’layernorm sigma’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 45: `softmax_cycles: int = 0x68` This assigns ‘0x68‘ to ‘softmax_cycles‘ in host memory. Here softmax_cycles means the value named ’softmax cycles’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 46: `softmax_maximum: int = 0x6C` This assigns ‘0x6C‘ to ‘softmax_maximum‘ in host memory. Here softmax_maximum means the value named ’softmax maximum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 47: `softmax_denominator: int = 0x70` This assigns ‘0x70‘ to ‘softmax_denominator‘ in host memory. Here softmax_denominator means the value named ’softmax denominator’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 48: `lif_spikes_low: int = 0x74` This assigns ‘0x74‘ to ‘lif_spikes_low‘ in host memory. Here lif_spikes_low means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 49: `lif_spikes_high: int = 0x78` This assigns ‘0x78‘ to ‘lif_spikes_high‘ in host memory. Here lif_spikes_high means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 50: `optimizer_updates: int = 0x7C` This assigns ‘0x7C‘ to ‘optimizer_updates‘ in host memory. Here optimizer_updates means the value named ’optimizer updates’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 51: `lut_entries: int = 0x80` This assigns ‘0x80‘ to ‘lut_entries‘ in host memory. Here lut_entries means the value named ’lut entries’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 52: `layernorm_reciprocal: int = 0x84` This assigns ‘0x84‘ to ‘layernorm_reciprocal‘ in host memory. Here layernorm_reciprocal means the value named ’layernorm reciprocal’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 53: `softmax_reciprocal: int = 0x88` This assigns ‘0x88‘ to ‘softmax_reciprocal‘ in host memory. Here softmax_reciprocal means the value named ’softmax reciprocal’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 54: `program_records: int = 0x8C` This assigns ‘0x8C‘ to ‘program_records‘ in host memory. Here program_records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 55: `program_output_beats: int = 0x90` This assigns ‘0x90‘ to ‘program_output_beats‘ in host memory. Here program_output_beats means the value named ’program output beats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 56: `training_tiles: int = 0x94` This assigns ‘0x94‘ to ‘training_tiles‘ in host memory. Here training_tiles means the value named ’training tiles’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 57: `training_cycles: int = 0x98` This assigns ‘0x98‘ to ‘training_cycles‘ in host memory. Here training_cycles means the value named ’training cycles’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class RegisterMap, which handles register map.

Line 60: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it creates no FPGA circuit by itself.

Line 61: `class PerformanceSnapshot:` This defines class PerformanceSnapshot, which groups data and methods for performance snapshot. Instances exist in host memory and organize the flow that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 62: `tensor_loaded_activations: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are tensor_loaded_activations: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 63: `tensor_loaded_weights: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are tensor_loaded_weights: a learned coefficient multiplied by an activation.

Line 64: `tensor_active_issues: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are tensor_active_issues: the value named ’tensor active issues’ in this block.

Line 65: `tensor_compute_cycles: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are tensor_compute_cycles: the value named 'tensor compute cycles' in this block.

Line 66: `tensor_skipped_issues: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are tensor_skipped_issues: the value named 'tensor skipped issues' in this block.

Line 67: `dense_macs: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are dense_macs: a count of ordinary activation-times-weight operations actually issued.

Line 68: `spike_adds: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are spike_adds: a binary event produced when a neuron crosses its threshold.

Line 69: `layernorm_cycles: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are layernorm_cycles: the value named 'layernorm cycles' in this block.

Line 70: `layernorm_mean: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are layernorm_mean: the arithmetic average used by normalization.

Line 71: `layernorm_variance: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are layernorm_variance: the average squared distance from the mean.

Line 72: `layernorm_sigma: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are layernorm_sigma: the value named 'layernorm sigma' in this block.

Line 73: `layernorm_reciprocal_q31: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are layernorm_reciprocal_q31: the value named 'layernorm reciprocal q31' in this block.

Line 74: `softmax_cycles: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are softmax_cycles: the value named 'softmax cycles' in this block.

Line 75: `softmax_maximum: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are softmax_maximum: the value named 'softmax maximum' in this block.

Line 76: `softmax_denominator: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are softmax_denominator: the value named 'softmax denominator' in this block.

Line 77: `softmax_reciprocal_q31: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are softmax_reciprocal_q31: the value named 'softmax reciprocal q31' in this block.

Line 78: `lif_total_spikes: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are lif_total_spikes: a binary event produced when a neuron crosses its threshold.

Line 79: `optimizer_updates: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are optimizer_updates: the value named 'optimizer updates' in this block.

Line 80: `lut_loaded_entries: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are lut_loaded_entries: the value named 'lut loaded entries' in this block.

Line 81: `training_tiles: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are training_tiles: the value named 'training tiles' in this block.

Line 82: `training_cycles: int` Inside the class PerformanceSnapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are training_cycles: the value named 'training cycles' in this block.

Line 85: `class NeuraEdgeHardwareError(RuntimeError):` This defines class NeuraEdgeHardwareError, which groups data and methods for neura edge hardware error. Instances exist in host memory and organize the flow that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 86: `"""Raised when the RTL reports a sticky protocol or configuration error."""` This is executable documentation attached to the class NeuraEdgeHardwareError, which handles neura edge hardware error. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 89: `class NeuraEdgeTimeout(TimeoutError):` This defines class NeuraEdgeTimeout, which groups data and methods for neura edge timeout. Instances exist in host memory and organize the flow that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 90: `"""Raised when a DMA channel or accelerator command exceeds its deadline."""` This is executable documentation attached to the class NeuraEdgeTimeout, which handles neura edge timeout. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 93: `class NeuraEdgeDriver:` This defines class NeuraEdgeDriver, which groups data and methods for neura edge driver. Instances exist in host memory and organize the flow that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 94: `STATUS_BUSY = 1 << 0` This assigns ‘1 << 0‘ to ‘STATUS_BUSY ‘ in host memory. Here STATUS_BUSY means the value named ’status busy’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class NeuraEdgeDriver, which handles neura edge driver.

Line 95: `STATUS_ERROR = 1 << 1` This assigns ‘1 << 1‘ to ‘STATUS_ERROR ‘ in host memory. Here STATUS_ERROR means the value named ’status error’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class NeuraEdgeDriver, which handles neura edge driver.

Line 96: `STATUS_DONE = 1 << 2` This assigns ‘1 << 2‘ to ‘STATUS_DONE ‘ in host memory. Here STATUS_DONE means the value named ’status done’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class NeuraEdgeDriver, which handles neura edge driver.

Line 97: `STATUS_PENDING = 1 << 3` This assigns ‘1 << 3‘ to ‘STATUS_PENDING ‘ in host memory. Here STATUS_PENDING means the value named ’status pending’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class NeuraEdgeDriver, which handles neura edge driver.

Line 98: `STATUS_PROGRAM_ACTIVE = 1 << 5` This assigns ‘1 << 5‘ to ‘STATUS_PROGRAM_ACTIVE ‘ in host memory. Here STATUS_PROGRAM_ACTIVE means the value named ’status program active’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class NeuraEdgeDriver, which handles neura edge driver.

Line 99: `EXPECTED_ID = 0x4E455552` This assigns ‘0x4E455552‘ to ‘EXPECTED_ID ‘ in host memory. Here EXPECTED_ID means the value named ’expected id’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class NeuraEdgeDriver, which handles neura edge driver.

Line 100: `EXPECTED_ABI_MAJOR = 1` This assigns ‘1‘ to ‘EXPECTED_ABI_MAJOR ‘ in host memory. Here EXPECTED_ABI_MAJOR means the value named ’expected abi major’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class NeuraEdgeDriver, which handles neura edge driver.

Line 101: `MAX_DMA_TRANSFER_BYTES = (1 << 26) - 1` This assigns ‘(1 << 26) - 1‘ to ‘MAX_DMA_TRANSFER_BYTES ‘ in host memory. Here MAX_DMA_TRANSFER_BYTES means the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. The expression is a literal or structural operator and has no hidden software state. Context: the class NeuraEdgeDriver, which handles neura edge driver.

Line 102: `PACK_COPY_WORDS = 1 << 16` This assigns ‘1 << 16‘ to ‘PACK_COPY_WORDS ‘ in host memory. Here PACK_COPY_WORDS means the value named ’pack copy words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the class NeuraEdgeDriver, which handles neura edge driver.

Line 104: `def __init__(self, mmio, dma, allocator: Callable[..., object], *, interrupt_enable: bool = True):` This starts function ___init___. Its indented body performs init as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 105: `self.mmio = mmio` This assigns ‘mmio‘ to ‘self.mmio ‘ in host memory. Here mmio means the value named ’mmio’ in this block. The important operands are mmio: the value named ’mmio’ in this block. Context: the function ___init___, which handles init.

Line 106: `self.dma = dma` This assigns ‘dma‘ to ‘self.dma ‘ in host memory. Here dma means the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. The important operands are dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. Context: the function ___init___, which handles init.

Line 107: `self.allocator = allocator` This assigns ‘allocator‘ to ‘self.allocator ‘ in host memory. Here allocator means the value named ’allocator’ in this block. The important operands are allocator: the value named ’allocator’ in this block. Context: the function ___init___, which handles init.

Line 108: `self.registers = RegisterMap()` This assigns ‘RegisterMap()‘ to ‘self.registers ‘ in host memory. Here registers means the value named ’registers’ in this block. The important operands are RegisterMap: the value named ’register map’ in this block. Context: the function ___init___, which handles init.

Line 109: `self.interrupt_enable = bool(interrupt_enable)` This assigns ‘bool(interrupt_enable)‘ to ‘self.interrupt_enable ‘ in host memory. Here interrupt_enable means the hardware notification that wakes software after completion or error. The important operands are bool: the value named ’bool’ in this block; interrupt_enable: the hardware notification that wakes software after completion or error. Context: the function ___init___, which handles init.

Line 110: `self._check_identity()` This calls ‘self._check_identity‘ with “ inside the function ___init___, which handles init. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 112: `@classmethod` This decorator changes how the following class or function is constructed. ‘@classmethod‘ is applied at import time on the host and is part of the class NeuraEdgeDriver, which handles neura edge driver; it creates no FPGA circuit by itself.

Line 113: `def from_bitfile(` This starts function from_bitfile. Its indented body performs from bitfile as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 114: `cls,` Inside the function from_bitfile, which handles from bitfile, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are cls: the value named ’cls’ in this block.

Line 115: `bitfile: str | Path,` Inside the function from_bitfile, which handles from bitfile, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are bitfile: the value named ’bitfile’ in this block; str: the value named ’str’ in this block; Path: the value named ’path’ in this block.

Line 116: `*,` Inside the function from_bitfile, which handles from bitfile, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The expression is a literal or structural operator and has no hidden software state.

Line 117: `download: bool = True,` This assigns ‘True,‘ to ‘download‘ in host memory. Here download means the value named ’download’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function from_bitfile, which handles from bitfile.

Line 118: `accelerator_name: str = "accelerator",` This assigns ‘"accelerator",‘ to ‘accelerator_name‘ in host memory. Here accelerator_name means the value named ’accelerator name’ in this block. The important operands are accelerator: the value named ’accelerator’ in this block. Context: the function from_bitfile, which handles from bitfile.

Line 119: `dma_name: str = "dma",` This assigns ‘"dma",‘ to ‘dma_name‘ in host memory. Here dma_name means the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. The important operands are dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. Context: the function from_bitfile, which handles from bitfile.

Line 120: `) -> "NeuraEdgeDriver":` Inside the function from_bitfile, which handles from bitfile, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are NeuraEdgeDriver: the value named ’neura edge driver’ in this block.

Line 121: `try:` This is explicit host-side failure handling inside the function from_bitfile, which handles from bitfile; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 122: `from pynq import Overlay, allocate` This imports the named Python API so the function from_bitfile, which handles from bitfile can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 123: `except ImportError as error:` This is explicit host-side failure handling inside the function from_bitfile, which handles from bitfile; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 124: `raise RuntimeError("the pynq package is required on the PYNQ-Z2 board") from error` This rejects an invalid state by raising ‘RuntimeError("the pynq package is required on the PYNQ-Z2 board") from error‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 126: `overlay = Overlay(str(Path(bitfile)), download=download)` This assigns ‘Overlay(str(Path(bitfile)), download=download)‘ to ‘overlay ‘ in host memory. Here overlay means the value named ’overlay’ in this block. The important operands are Overlay: the value named ’overlay’ in this block; str: the value named ’str’ in this block; Path: the value named ’path’ in this block. Context: the function from_bitfile, which handles from bitfile.

Line 127: `accelerator = getattr(overlay, accelerator_name)` This assigns ‘getattr(overlay, accelerator_name)‘ to ‘accelerator ‘ in host memory. Here accelerator means the value named ’accelerator’ in this block. The important operands are getattr: the value named ’getattr’ in this block; overlay: the value named ’overlay’ in this block; accelerator_name: the value named ’accelerator name’ in this block. Context: the function from_bitfile, which handles from bitfile.

Line 128: `dma = getattr(overlay, dma_name)` This assigns ‘getattr(overlay, dma_name)‘ to ‘dma ‘ in host memory. Here dma means the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. The important operands are getattr: the value named ’getattr’ in this block; overlay: the value named ’overlay’ in this block; dma_name: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. Context: the function from_bitfile, which handles from bitfile.

Line 129: `driver = cls(accelerator.mmio, dma, allocate)` This assigns ‘cls(accelerator.mmio, dma, allocate)‘ to ‘driver ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are cls: the value named ’cls’ in this block; accelerator: the value named ’accelerator’ in this block; mmio: the value named ’mmio’ in this block. Context: the function from_bitfile, which handles from bitfile.

Line 130: `driver.overlay = overlay` This assigns ‘overlay‘ to ‘driver.overlay ‘ in host memory. Here overlay means the value named ’overlay’ in this block. The important operands are overlay: the value named ’overlay’ in this block. Context: the function from_bitfile, which handles from bitfile.

Line 131: `return driver` This returns ‘driver‘ to the caller of the function from_bitfile, which handles from bitfile, ending this invocation. The important operands are driver: the value named ’driver’ in this block.

Line 133: `def _check_identity(self) -> None:` This starts function _check_identity. Its indented body performs check identity as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 134: `identity = int(self.mmio.read(self.registers.identity))` This assigns ‘int(self.mmio.read(self.registers.identit
to ‘identity ‘ in host memory. Here identity means the value named ’identity’ in this block. The important operands are mmio: the value named ’mmio’ in this block; read: the value named ’read’ in this block; registers: the value named ’registers’ in this block. Context: the function _check_identity, which handles check identity.

Line 135: `version = int(self.mmio.read(self.registers.version))` This assigns ‘int(self.mmio.read(self.registers.version))’ to ‘version ‘ in host memory. Here version means the value named ’version’ in this block. The important operands are mmio: the value named ’mmio’ in this block; read: the value named ’read’ in this block; registers: the value named ’registers’ in this block. Context: the function _check_identity, which handles check identity.

Line 136: `if identity != self.EXPECTED_ID:` This executes the following indented host-code branch only when ‘identity != self.EXPECTED_ID‘ is true. The important operands are identity: the value named ’identity’ in this block; EXPECTED_ID: the value named ’expected id’ in this block. Context: the function _check_identity, which handles check identity.

Line 137: `raise NeuraEdgeHardwareError(` This statement continues through source line 139; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"unexpected accelerator identity 0x{identity:08x}; expected 0x{self.EXPECTED_ID:08x}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 138: `f"unexpected accelerator identity 0x{identity:08x}; expected 0x{self.EXPECTED_ID:08x}"` This is continuation line 2 of the 3-line logical statement that begins at source line 137. Its exact fragment is ‘f"unexpected accelerator identity 0x{identity:08x}; expected 0x{self.EXPECTED_ID:08x}"‘. The important operands are f: the value named ’f’ in this block; unexpected: the value named ’unexpected’ in this block; accelerator: the value named ’accelerator’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"unexpected accelerator identity 0x{identity:08x}; expected 0x{self.EXPECTED_ID:08x}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 139: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 137. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"unexpected accelerator identity 0x{identity:08x}; expected 0x{self.EXPECTED_ID:08x}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 140: `if version >> 16 != self.EXPECTED_ABI_MAJOR:` This executes the following indented host-code branch only when ‘version >> 16 != self.EXPECTED_ABI_MAJOR‘ is true. The important operands are version: the value named ’version’ in this block; EXPECTED_ABI_MAJOR: the value named ’expected abi major’ in this block. Context: the function _check_identity, which handles check identity.

Line 141: `raise NeuraEdgeHardwareError(` This statement continues through source line 143; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"unsupported accelerator ABI 0x{version:08x}; expected major {self.EXPECTED_ABI_MAJOR}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 142: `f"unsupported accelerator ABI 0x{version:08x}; expected major {self.EXPECTED_ABI_MAJOR}"` This is continuation line 2 of the 3-line logical statement that begins at source line 141. Its exact fragment is ‘f"unsupported accelerator ABI 0x{version:08x}; expected major {self.EXPECTED_ABI_MAJOR}"‘. The important operands are f: the value named ’f’ in this block; unsupported: the value named ’unsupported’ in this block; accelerator: the value named ’accelerator’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"unsupported accelerator ABI 0x{version:08x}; expected major {self.EXPECTED_ABI_MAJOR}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 143: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 141. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"unsupported accelerator ABI 0x{version:08x}; expected major {self.EXPECTED_ABI_MAJOR}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 145: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class NeuraEdgeDriver, which handles neura edge driver; it creates no FPGA circuit by itself.

Line 146: `def target_clock_hz(self) -> int:` This starts function target_clock_hz. Its indented body performs target clock hz as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 147: `return int(self.mmio.read(self.registers.target_clock))` This returns ‘int(self.mmio.read(self.registers.target to the caller of the function target_clock_hz, which handles target clock hz, ending this invocation. The important operands are mmio: the value named ’mmio’ in this block; read: the value named ’read’ in this block; registers: the value named ’registers’ in this block.

Line 149: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class NeuraEdgeDriver, which handles neura edge driver; it creates no FPGA circuit by itself.

Line 150: `def capabilities(self) -> int:` This starts function capabilities. Its indented body performs capabilities as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 151: `return int(self.mmio.read(self.registers.capabilities))` This returns ‘int(self.mmio.read(self.registers.capab to the caller of the function capabilities, which handles capabilities, ending this invocation. The important operands are mmio: the value named ’mmio’ in this block; read: the value named ’read’ in this block; registers: the value named ’registers’ in this block.

Line 153: `def status(self) -> int:` This starts function status. Its indented body performs status as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 154: `return int(self.mmio.read(self.registers.status))` This returns ‘int(self.mmio.read(self.registers.status))‘ to the caller of the function status, which handles status, ending this invocation. The important operands are mmio: the value named ’mmio’ in this block; read: the value named ’read’ in this block; registers: the value named ’registers’ in this block.

Line 156: `@staticmethod` This decorator changes how the following class or function is constructed. ‘@staticmethod‘ is applied at import time on the host and is part of the class NeuraEdgeDriver, which handles neura edge driver; it creates no FPGA circuit by itself.

Line 157: `def _signed32(value: int) -> int:` This starts function _signed32. Its indented body performs signed32 as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 158: `masked = int(value) & 0xFFFFFFFF` This assigns ‘int(value) & 0xFFFFFFFF‘ to ‘masked ‘ in host memory. Here masked means the value named ’masked’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function _signed32, which handles signed32.

Line 159: `return masked - (1 << 32) if masked & (1 << 31) else masked` This returns ‘masked - (1 << 32) if masked & (1 << 31) else masked‘ to the caller of the function _signed32, which handles signed32, ending this invocation. The important operands are masked: the value named ’masked’ in this block.

Line 161: `def performance_snapshot(self) -> PerformanceSnapshot:` This starts function performance_snapshot. Its indented body performs performance snapshot as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 162: `"""Read one coherent-enough diagnostic snapshot while the core is idle."""` This is executable documentation attached to the function performance_snapshot, which handles performance snapshot. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 163: `if self.status() & (self.STATUS_BUSY | self.STATUS_PENDING):` This executes the following indented host-code branch only when 'self.status() & (self.STATUS_BUSY | self.STATUS_PENDING)' is true. The important operands are status: the value named 'status' in this block; STATUS_BUSY: the value named 'status busy' in this block; STATUS_PENDING: the value named 'status pending' in this block. Context: the function performance_snapshot, which handles performance snapshot.

Line 164: `raise NeuraEdgeHardwareError("performance counters must be sampled while the accelerator is idle")` This rejects an invalid state by raising 'NeuraEdgeHardwareError("performance counters must be sampled while the accelerator is idle")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 165: `registers = self.registers` This assigns 'self.registers' to 'registers ' in host memory. Here registers means the value named 'registers' in this block. The important operands are registers: the value named 'registers' in this block. Context: the function performance_snapshot, which handles performance snapshot.

Line 166: `loaded = int(self.mmio.read(registers.tensor_loaded))` This assigns 'int(self.mmio.read(registers.tensor_loade to 'loaded ' in host memory. Here loaded means the value named 'loaded' in this block. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Context: the function performance_snapshot, which handles performance snapshot.

Line 167: `dense_macs = int(self.mmio.read(registers.dense_macs_low))` This assigns 'int(self.mmio.read(registers.dense_ to 'dense_macs ' in host memory. Here dense_macs means a count of ordinary activation-times-weight operations actually issued. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Context: the function performance_snapshot, which handles performance snapshot.

Line 168: `dense_macs |= int(self.mmio.read(registers.dense_macs_high)) << 32` Inside the function performance_snapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are dense_macs: a count of ordinary activation-times-weight operations actually issued; mmio: the value named 'mmio' in this block; read: the value named 'read' in this block.

Line 169: `spike_adds = int(self.mmio.read(registers.spike_adds_low))` This assigns 'int(self.mmio.read(registers.spike_ to 'spike_adds ' in host memory. Here spike_adds means a binary event produced when a neuron crosses its threshold. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Context: the function performance_snapshot, which handles performance snapshot.

Line 170: `spike_adds |= int(self.mmio.read(registers.spike_adds_high)) << 32` Inside the function performance_snapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are spike_adds: a binary event produced when a neuron crosses its threshold; mmio: the value named 'mmio' in this block; read: the value named 'read' in this block.

Line 171: `lif_spikes = int(self.mmio.read(registers.lif_spikes_low))` This assigns 'int(self.mmio.read(registers.lif_spi to 'lif_spikes ' in host memory. Here lif_spikes means a binary event produced when a neuron crosses its threshold. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Context: the function performance_snapshot, which handles performance snapshot.

Line 172: `lif_spikes |= int(self.mmio.read(registers.lif_spikes_high)) << 32` Inside the function performance_snapshot, which handles performance snapshot, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are lif_spikes: a binary event produced when a neuron crosses its threshold; mmio: the value named 'mmio' in this block; read: the value named 'read' in this block.

Line 173: `return PerformanceSnapshot(` This statement continues through source line 195; the explanation uses the complete expression rather than only the first visual fragment. This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.read(registers.layernorm_cycles)), self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes, int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 174: `loaded & 0xFFF,` This is continuation line 2 of the 23-line logical statement that begins at source line 173. Its exact fragment is 'loaded & 0xFFF,'. The important operands are loaded: the value named 'loaded' in this block. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.read(registers.layernorm_cycles)), self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes, int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 175: `(loaded >> 16) & 0xFFF,` This is continuation line 3 of the 23-line logical statement that begins at source line 173. Its exact fragment is '(loaded >> 16) & 0xFFF,'. The important operands are loaded: the value named 'loaded' in this block. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.read(registers.layernorm_cycles)), self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes, int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 176: `int(self.mmio.read(registers.tensor_active)) & 0xFFF,` This is continuation line 4 of the 23-line logical statement that begins at source line 173. Its exact fragment is 'int(self.mmio.read(registers.tensor_active)) & 0xFFF,'. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio. self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes, int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr

int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 177: `int(self.mmio.read(registers.tensor_cycles)),` This is continuation line 5 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘int(self.mmio.read(registers.tensor_cycles)),‘. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 178: `int(self.mmio.read(registers.tensor_skipped)),` This is continuation line 6 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘int(self.mmio.read(registers.tensor_skipped)),‘. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 179: `dense_macs,` This is continuation line 7 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘dense_macs,‘. The important operands are dense_macs: a count of ordinary activation-times-weight operations actually issued. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.read(registers.layernorm_cycles)), self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 180: `spike_adds,` This is continuation line 8 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘spike_adds,‘. The important operands are spike_adds: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.read(registers.layernorm_cycles)), self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,

int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 181: `int(self.mmio.read(registers.layernorm_cycles)),` This is continuation line 9 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘int(self.mmio.read(registers.layernorm_cycles)),‘. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 182: `self._signed32(self.mmio.read(registers.layernorm_mean)),` This is continuation line 10 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘self._signed32(self.mmio.read(registers.layernorm_mean))
The important operands are _signed32: the value named 'signed32' in this block; mmio: the value named 'mmio' in this block; read: the value named 'read' in this block. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 183: `int(self.mmio.read(registers.layernorm_variance)),` This is continuation line 11 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘int(self.mmio.read(registers.layernorm_variance)),‘. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 184: `int(self.mmio.read(registers.layernorm_sigma)),` This is continuation line 12 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘int(self.mmio.read(registers.layernorm_sigma)),‘. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.

self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 185: `int(self.mmio.read(registers.layernorm_reciprocal)),` This is continuation line 13 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘int(self.mmio.read(registers.layernorm_reciprocal)),‘. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 186: `int(self.mmio.read(registers.softmax_cycles)),` This is continuation line 14 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘int(self.mmio.read(registers.softmax_cycles)),‘. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 187: `self._signed32(self.mmio.read(registers.softmax_maximum)),` This is continuation line 15 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘self._signed32(self.mmio.read(registers.softmax_maximum
The important operands are _signed32: the value named 'signed32' in this block; mmio: the value named 'mmio' in this block; read: the value named 'read' in this block. Read together, the complete operation is: This returns ‘PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)‘ to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 188: `int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF,` This is continuation line 16 of the 23-line logical statement that begins at source line 173. Its exact fragment is ‘int(self.mmio.read(registers.softmax_denominator))
& 0xFFFFFF,‘. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in

this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 189: `int(self.mmio.read(registers.softmax_reciprocal)),` This is continuation line 17 of the 23-line logical statement that begins at source line 173. Its exact fragment is 'int(self.mmio.read(registers.softmax_reciprocal)),'. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 190: `lif_spikes,` This is continuation line 18 of the 23-line logical statement that begins at source line 173. Its exact fragment is 'lif_spikes,'. The important operands are lif_spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.read(registers.layernorm_cycles)), self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 191: `int(self.mmio.read(registers.optimizer_updates)),` This is continuation line 19 of the 23-line logical statement that begins at source line 173. Its exact fragment is 'int(self.mmio.read(registers.optimizer_updates)),'. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 192: `int(self.mmio.read(registers.lut_entries)) & 0x1FF,` This is continuation line 20 of the 23-line logical statement that begins at source line 173. Its exact fragment is 'int(self.mmio.read(registers.lut_entries)) & 0x1FF,'. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 193: `int(self.mmio.read(registers.training_tiles)),` This is continuation line 21 of the 23-line logical statement that begins at source line 173. Its exact fragment is 'int(self.mmio.read(registers.training_tiles)),'. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 194: `int(self.mmio.read(registers.training_cycles)),` This is continuation line 22 of the 23-line logical statement that begins at source line 173. Its exact fragment is 'int(self.mmio.read(registers.training_cycles)),'. The important operands are mmio: the value named 'mmio' in this block; read: the value named 'read' in this block; registers: the value named 'registers' in this block. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tensor_skipped)), dense_macs, spike_adds, int(self.mmio.
self._signed32(self.mmio.read(registers.layernorm_mean)), int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(re
int(self.mmio.read(registers.layernorm_reciprocal)), int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(re
int(self.mmio.read(registers.softmax_denominator)) & 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes,
int(self.mmio.read(registers.optimizer_updates)), int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.tr
int(self.mmio.read(registers.training_cycles)),)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 195: `)` This is continuation line 23 of the 23-line logical statement that begins at source line 173. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'PerformanceSnapshot(loaded & 0xFFF, (loaded >> 16) & 0xFFF, int(self.mmio.read(registers.tensor_active)) & 0xFFF, int(self.mmio.read(registers.tensor_cycles)), int(self.mmio.read(registers.tens
dense_macs, spike_adds, int(self.mmio.read(registers.layernorm_cycles)), self._signed32(self.mmio.read(registers.layernorm_mean
int(self.mmio.read(registers.layernorm_variance)), int(self.mmio.read(registers.layernorm_sigma)), int(self.mmio.read(registers.laye
int(self.mmio.read(registers.softmax_cycles)), self._signed32(self.mmio.read(registers.softmax_maximum)), int(self.mmio.read(regi
& 0xFFFFFF, int(self.mmio.read(registers.softmax_reciprocal)), lif_spikes, int(self.mmio.read(registers.optimizer_updates)),
int(self.mmio.read(registers.lut_entries)) & 0x1FF, int(self.mmio.read(registers.training_tiles)), int(self.mmio.read(registers.trainin
)' to the caller of the function performance_snapshot, which handles performance snapshot, ending this invocation. The important operands are PerformanceSnapshot: the value named 'performance snapshot' in this block; loaded: the value named 'loaded' in this block; mmio: the value named 'mmio' in this block.

Line 197: `def _control_value(self, transient_bits: int = 0) -> int:` This starts function _control_value. Its indented body performs control value as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 198: `return ((1 << 8) if self.interrupt_enable else 0) | transient_bits` This returns '((1 << 8) if self.interrupt_enable else 0) | transient_bits' to the caller of the function _control_value, which handles control value, ending this invocation. The important operands are interrupt_enable: the hardware notification that wakes software after completion or error; transient_bits: the value named 'transient bits' in this block.

Line 200: `def clear_wrapper_status(self) -> None:` This starts function clear_wrapper_status. Its indented body performs clear wrapper status as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 201: `self.mmio.write(self.registers.control, self._control_value(1 << 1))` This calls 'self.mmio.write' with 'self.registers.control, self._control_value(1 << 1)' inside the function clear_wrapper_status, which handles clear wrapper status. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 203: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class NeuraEdgeDriver, which handles neura edge driver; it creates no FPGA circuit by itself.

Line 204: `def _validate_input_beats(beats: Sequence[StreamBeat]) -> None:` This starts function _validate_input_beats. Its indented body performs validate input beats as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 205: `if isinstance(beats, PackedStreamBeats):` This executes the following indented host-code branch only when 'isinstance(beats, PackedStreamBeats)' is true. The important operands are isinstance: the value named 'isinstance' in this block; beats: the value named 'beats' in this block; PackedStreamBeats: the value named 'packed stream beats' in this block. Context: the function _validate_input_beats, which handles validate input beats.

Line 206: `return` This returns " to the caller of the function _validate_input_beats, which handles validate input beats, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 207: `for index, beat in enumerate(beats):` This iterates index, beat over 'enumerate(beats)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _validate_input_beats, which handles validate input beats.

Line 208: `if beat.last != (index == len(beats) - 1):` This executes the following indented host-code branch only when 'beat.last != (index == len(beats) - 1)' is true. The important operands are beat: the value named 'beat' in this block; last: the AXI-Stream marker for the final beat of a packet or command payload; index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function _validate_input_beats, which handles validate input beats.

Line 209: `raise ValueError("input StreamBeat.last must be true only on the final beat")` This rejects an invalid state by raising 'ValueError("input StreamBeat.last must be true only on the final beat")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 211: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class NeuraEdgeDriver, which handles neura edge driver; it creates no FPGA circuit by itself.

Line 212: `def _release_buffer(buffer) -> None:` This starts function _release_buffer. Its indented body performs release buffer as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 213: `if buffer is None:` This executes the following indented host-code branch only when ‘buffer is None‘ is true. The important operands are buffer: local storage that decouples producer and consumer timing; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function _release_buffer, which handles release buffer.

Line 214: `return` This returns “ to the caller of the function _release_buffer, which handles release buffer, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 215: `if hasattr(buffer, "freebuffer"):` This executes the following indented host-code branch only when ‘hasattr(buffer, "freebuffer")‘ is true. The important operands are hasattr: the value named ’hasattr’ in this block; buffer: local storage that decouples producer and consumer timing; freebuffer: local storage that decouples producer and consumer timing. Context: the function _release_buffer, which handles release buffer.

Line 216: `buffer.freebuffer()` This calls ‘buffer.freebuffer‘ with “ inside the function _release_buffer, which handles release buffer. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 217: `elif hasattr(buffer, "close"):` This executes the following indented host-code branch only when ‘hasattr(buffer, "close")‘ is true. The important operands are hasattr: the value named ’hasattr’ in this block; buffer: local storage that decouples producer and consumer timing; close: the value named ’close’ in this block. Context: the function _release_buffer, which handles release buffer.

Line 218: `buffer.close()` This calls ‘buffer.close‘ with “ inside the function _release_buffer, which handles release buffer. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 220: `@staticmethod` This decorator changes how the following class or function is constructed. ‘@staticmethod‘ is applied at import time on the host and is part of the class NeuraEdgeDriver, which handles neura edge driver; it creates no FPGA circuit by itself.

Line 221: `def _wait_dma_channel(channel, deadline: float, name: str) -> None:` This starts function _wait_dma_c
Its indented body performs wait dma channel as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 222: `while not bool(channel.idle):` This repeats the following host-code block while ‘not bool(channel.idle)‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 223: `if monotonic() >= deadline:` This executes the following indented host-code branch only when ‘monotonic() >= deadline‘ is true. The important operands are monotonic: the value named ’monotonic’ in this block; deadline: the value named ’deadline’ in this block. Context: the function _wait_dma_channel, which handles wait dma channel.

Line 224: `if hasattr(channel, "stop"):` This executes the following indented host-code branch only when ‘hasattr(channel, "stop")‘ is true. The important operands are hasattr: the value named ’hasattr’ in this block; channel: the value named ’channel’ in this block; stop: the value named ’stop’ in this block. Context: the function _wait_dma_channel, which handles wait dma channel.

Line 225: `channel.stop()` This calls ‘channel.stop‘ with “ inside the function _wait_dma_channel, which handles wait dma channel. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 226: `raise NeuraEdgeTimeout(f"{name} DMA channel timed out")` This rejects an invalid state by raising ‘NeuraEdgeTimeout(f"{name} DMA channel timed out")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 227: `sleep(0.0005)` This calls ‘sleep‘ with ‘0.0005‘ inside the function _wait_dma_channel, which handles wait dma channel. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 228: `channel.wait()` This calls ‘channel.wait‘ with “ inside the function _wait_dma_channel, which handles wait dma channel. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 230: `def _wait_idle(self, deadline: float) -> None:` This starts function _wait_idle. Its indented body performs wait idle as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 231: `while self.status() & (self.STATUS_BUSY | self.STATUS_PENDING):` This repeats the following host-code block while ‘self.status() & (self.STATUS_BUSY | self.STATUS_PENDING)‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 232: `if monotonic() >= deadline:` This executes the following indented host-code branch only when ‘monotonic() >= deadline‘ is true. The important operands are monotonic: the value named ’monotonic’ in this block; deadline: the value named ’deadline’ in this block. Context: the function _wait_idle, which handles wait idle.

Line 233: `raise NeuraEdgeTimeout("accelerator did not become idle")` This rejects an invalid state by raising ‘NeuraEdgeTimeout("accelerator did not become idle")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 234: `sleep(0.0005)` This calls ‘sleep‘ with ‘0.0005‘ inside the function _wait_idle, which handles wait idle. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 236: `def execute(` This starts function execute. Its indented body performs execute as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 237: `self,` Inside the function execute, which handles execute, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The expression is a literal or structural operator and has no hidden software state.

Line 238: `command: Command128,` Inside the function execute, which handles execute, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 239: `input_beats: Sequence[StreamBeat] = (),` This assigns ‘(),‘ to ‘input_beats‘ in host memory. Here input_beats means the value named ’input beats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 240: `*,` Inside the function execute, which handles execute, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The expression is a literal or structural operator and has no hidden software state.

Line 241: `output_beats: int = 0,` This assigns '0,' to 'output_beats' in host memory. Here output_beats means the value named 'output beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 242: `timeout_seconds: float = 5.0,` This assigns '5.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 243: `) -> tuple[int, ...]:` Inside the function execute, which handles execute, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are tuple: the value named 'tuple' in this block.

Line 244: `if timeout_seconds <= 0:` This executes the following indented host-code branch only when 'timeout_seconds <= 0' is true. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Context: the function execute, which handles execute.

Line 245: `raise ValueError("timeout_seconds must be positive")` This rejects an invalid state by raising 'ValueError("timeout_seconds must be positive")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 246: `if output_beats < 0:` This executes the following indented host-code branch only when 'output_beats < 0' is true. The important operands are output_beats: the value named 'output beats' in this block. Context: the function execute, which handles execute.

Line 247: `raise ValueError("output_beats must be non-negative")` This rejects an invalid state by raising 'ValueError("output_beats must be non-negative")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 248: `self._validate_input_beats(input_beats)` This calls 'self._validate_input_beats' with 'input_beats' inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 249: `deadline = monotonic() + timeout_seconds` This assigns 'monotonic() + timeout_seconds' to 'deadline ' in host memory. Here deadline means the value named 'deadline' in this block. The important operands are monotonic: the value named 'monotonic' in this block; timeout_seconds: the value named 'timeout seconds' in this block. Context: the function execute, which handles execute.

Line 250: `self._wait_idle(deadline)` This calls 'self._wait_idle' with 'deadline' inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 252: `input_buffer = None` This assigns 'None' to 'input_buffer ' in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are None: the value named 'none' in this block. Context: the function execute, which handles execute.

Line 253: `output_buffer = None` This assigns 'None' to 'output_buffer ' in host memory. Here output_buffer means local storage that decouples producer and consumer timing. The important operands are None: the value named 'none' in this block. Context: the function execute, which handles execute.

Line 254: `try:` This is explicit host-side failure handling inside the function execute, which handles execute; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 255: `if input_beats:` This executes the following indented host-code branch only when ‘input_beats‘ is true. The important operands are input_beats: the value named ’input beats’ in this block. Context: the function execute, which handles execute.

Line 256: `input_buffer = self.allocator(shape=(len(input_beats),), dtype=np.uint64)` This assigns ‘self.allocator(sha dtype=np.uint64)‘ to ‘input_buffer ‘ in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are allocator: the value named ’allocator’ in this block; shape: the value named ’shape’ in this block; len: the value named ’len’ in this block. Context: the function execute, which handles execute.

Line 257: `if isinstance(input_beats, PackedStreamBeats):` This executes the following indented host-code branch only when ‘isinstance(input_beats, PackedStreamBeats)‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; input_beats: the value named ’input beats’ in this block; PackedStreamBeats: the value named ’packed stream beats’ in this block. Context: the function execute, which handles execute.

Line 258: `input_buffer[:] = np.frombuffer(input_beats.words, dtype=np.uint64)` This assigns ‘np.frombuffer(input_b dtype=np.uint64)‘ to ‘input_buffer[‘ in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are frombuffer: local storage that decouples producer and consumer timing; input_beats: the value named ’input beats’ in this block; words: the value named ’words’ in this block. Context: the function execute, which handles execute.

Line 259: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute, which handles execute.

Line 260: `input_buffer[:] = np.fromiter(` This statement continues through source line 264; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.fromiter((beat.data for beat in input_beats), dtype=np.uint64, count=len(input_beats),)‘ to ‘input_buffer[‘ in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are fromiter: the value named ’fromiter’ in this block; beat: the value named ’beat’ in this block; data: the value named ’data’ in this block. Context: the function execute, which handles execute.

Line 261: `(beat.data for beat in input_beats),` This is continuation line 2 of the 5-line logical statement that begins at source line 260. Its exact fragment is ‘(beat.data for beat in input_beats),‘. The important operands are beat: the value named ’beat’ in this block; data: the value named ’data’ in this block; for: the value named ’for’ in this block. Read together, the complete operation is: This assigns ‘np.fromiter((beat.data for beat in input_beats), dtype=np.uint64, count=len(input_beats),)‘ to ‘input_buffer[‘ in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are fromiter: the value named ’fromiter’ in this block; beat: the value named ’beat’ in this block; data: the value named ’data’ in this block. Context: the function execute, which handles execute.

Line 262: `dtype=np.uint64,` This is continuation line 3 of the 5-line logical statement that begins at source line 260. Its exact fragment is ‘dtype=np.uint64,‘. The important operands are dtype: the value named ’dtype’ in this block; uint64: the value named ’uint64’ in this block. Read together, the complete operation is: This assigns ‘np.fromiter((beat.data for beat in input_beats), dtype=np.uint64, count=len(input_beats),)‘ to ‘input_buffer[‘ in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are fromiter: the value named ’fromiter’ in this block; beat: the value named ’beat’ in this block; data: the value named ’data’ in this block. Context: the function execute, which handles execute.

Line 263: `count=len(input_beats),` This is continuation line 4 of the 5-line logical statement that begins at source line 260. Its exact fragment is ‘count=len(input_beats),‘. The important operands are count: the value named ’count’ in this block; len: the value named ’len’ in this block; input_beats: the value named ’input beats’ in this block. Read together, the complete operation is: This assigns ‘np.fromiter((beat.data for beat in input_beats), dtype=np.uint64, count=len(input_beats),)‘ to ‘input_buffer[‘ in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are fromiter: the value named ’fromiter’ in this block; beat: the value named ’beat’ in this block; data: the value named ’data’ in this block. Context: the function execute, which handles execute.

Line 264: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 260. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.fromiter((beat.data for beat in input_beats), dtype=np.uint64, count=len(input_beats),)' to 'input_buffer[' in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are fromiter: the value named 'fromiter' in this block; beat: the value named 'beat' in this block; data: the value named 'data' in this block. Context: the function execute, which handles execute.

Line 265: `if hasattr(input_buffer, "flush"):` This executes the following indented host-code branch only when 'hasattr(input_buffer, "flush")' is true. The important operands are hasattr: the value named 'hasattr' in this block; input_buffer: local storage that decouples producer and consumer timing; flush: the value named 'flush' in this block. Context: the function execute, which handles execute.

Line 266: `input_buffer.flush()` This calls 'input_buffer.flush' with " inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 267: `if output_beats:` This executes the following indented host-code branch only when 'output_beats' is true. The important operands are output_beats: the value named 'output beats' in this block. Context: the function execute, which handles execute.

Line 268: `output_buffer = self.allocator(shape=(output_beats,), dtype=np.uint64)` This assigns 'self.allocator(shape= dtype=np.uint64)' to 'output_buffer ' in host memory. Here output_buffer means local storage that decouples producer and consumer timing. The important operands are allocator: the value named 'allocator' in this block; shape: the value named 'shape' in this block; output_beats: the value named 'output beats' in this block. Context: the function execute, which handles execute.

Line 269: `self.dma.recvchannel.transfer(output_buffer, nbytes=output_beats * 8)` This calls 'self.dma.recvchannel.tra with 'output_buffer, nbytes=output_beats * 8' inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 271: `self.clear_wrapper_status()` This calls 'self.clear_wrapper_status' with " inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 272: `for index, word in enumerate(command.words_le):` This iterates index, word over 'enumerate(command.words_le Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 273: `self.mmio.write(self.registers.command0 + 4 * index, word)` This calls 'self.mmio.write' with 'self.registers.co + 4 * index, word' inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 274: `self.mmio.write(self.registers.control, self._control_value(1))` This calls 'self.mmio.write' with 'self.registers.control, self._control_value(1)' inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 276: `if input_buffer is not None:` This executes the following indented host-code branch only when 'input_buffer is not None' is true. The important operands are input_buffer: local storage that decouples producer and consumer timing; is: the value named 'is' in this block; not: the value named 'not' in this block. Context: the function execute, which handles execute.

Line 277: `self.dma.sendchannel.transfer(input_buffer, nbytes=len(input_beats) * 8)` This calls 'self.dma.sendchanne with 'input_buffer, nbytes=len(input_beats) * 8' inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 278: `self._wait_dma_channel(self.dma.sendchannel, deadline, "MM2S")` This calls ‘self._wait_dma_channel‘ with ‘self.dma.sendchannel, deadline, "MM2S"‘ inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 279: `if output_buffer is not None:` This executes the following indented host-code branch only when ‘output_buffer is not None‘ is true. The important operands are output_buffer: local storage that decouples producer and consumer timing; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function execute, which handles execute.

Line 280: `self._wait_dma_channel(self.dma.recvchannel, deadline, "S2MM")` This calls ‘self._wait_dma_channel‘ with ‘self.dma.recvchannel, deadline, "S2MM"‘ inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 282: `while True:` This repeats the following host-code block while ‘True‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 283: `status = self.status()` This assigns ‘self.status()‘ to ‘status ‘ in host memory. Here status means the value named ’status’ in this block. The important operands are status: the value named ’status’ in this block. Context: the function execute, which handles execute.

Line 284: `if status & self.STATUS_ERROR:` This executes the following indented host-code branch only when ‘status & self.STATUS_ERROR‘ is true. The important operands are status: the value named ’status’ in this block; STATUS_ERROR: the value named ’status error’ in this block. Context: the function execute, which handles execute.

Line 285: `raise NeuraEdgeHardwareError(` This statement continues through source line 287; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"accelerator rejected opcode {int(command.opcode):x}; status=0x{status:08x}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 286: `f"accelerator rejected opcode {int(command.opcode):x}; status=0x{status:08x}"` This is continuation line 2 of the 3-line logical statement that begins at source line 285. Its exact fragment is ‘f"accelerator rejected opcode {int(command.opcode):x}; status=0x{status:08x}"‘. The important operands are f: the value named ’f’ in this block; accelerator: the value named ’accelerator’ in this block; rejected: the value named ’rejected’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"accelerator rejected opcode {int(command.opcode):x}; status=0x{status:08x}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 287: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 285. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"accelerator rejected opcode {int(command.opcode):x}; status=0x{status:08x}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 288: `if status & self.STATUS_DONE:` This executes the following indented host-code branch only when ‘status & self.STATUS_DONE‘ is true. The important operands are status: the value named ’status’ in this block; STATUS_DONE: the value named ’status done’ in this block. Context: the function execute, which handles execute.

Line 289: `break` Inside the function execute, which handles execute, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are break: the value named ’break’ in this block.

Line 290: `if monotonic() >= deadline:` This executes the following indented host-code branch only when ‘monotonic() >= deadline‘ is true. The important operands are monotonic: the value named ’monotonic’ in this block; deadline: the value named ’deadline’ in this block. Context: the function execute, which handles execute.

Line 291: `raise NeuraEdgeTimeout(f"opcode {int(command.opcode):x} did not complete")` This rejects an invalid state by raising ‘NeuraEdgeTimeout(f"opcode {int(command.opcode):x} did not complete")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 292: `sleep(0.0005)` This calls ‘sleep‘ with ‘0.0005‘ inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 294: `if output_buffer is None:` This executes the following indented host-code branch only when ‘output_buffer is None‘ is true. The important operands are output_buffer: local storage that decouples producer and consumer timing; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function execute, which handles execute.

Line 295: `return ()` This returns ‘()‘ to the caller of the function execute, which handles execute, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 296: `if hasattr(output_buffer, "invalidate"):` This executes the following indented host-code branch only when ‘hasattr(output_buffer, "invalidate")‘ is true. The important operands are hasattr: the value named ’hasattr’ in this block; output_buffer: local storage that decouples producer and consumer timing; invalidate: the producer’s statement that its current payload is meaningful. Context: the function execute, which handles execute.

Line 297: `output_buffer.invalidate()` This calls ‘output_buffer.invalidate‘ with “ inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 298: `return tuple(int(value) for value in output_buffer[:output_beats])` This returns ‘tuple(int(value) for value in output_buffer[:output_beats])‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; value: the value named ’value’ in this block; for: the value named ’for’ in this block.

Line 299: `finally:` This is explicit host-side failure handling inside the function execute, which handles execute; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 300: `self._release_buffer(input_buffer)` This calls ‘self._release_buffer‘ with ‘input_buffer‘ inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 301: `self._release_buffer(output_buffer)` This calls ‘self._release_buffer‘ with ‘output_buffer‘ inside the function execute, which handles execute. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 303: `def execute_program(` This starts function execute_program. Its indented body performs execute program as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 304: `self,` Inside the function execute_program, which handles execute program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The expression is a literal or structural operator and has no hidden software state.

Line 305: `records: Sequence[ProgramRecord],` Inside the function execute_program, which handles execute program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are records: the number of descriptor records accepted since program start; Sequence: the value named ’sequence’ in this block; ProgramRecord: the value named ’program record’ in this block.

Line 306: `*,` Inside the function execute_program, which handles execute program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The expression is a literal or structural operator and has no hidden software state.

Line 307: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 308: `) -> tuple[int, ...]:` Inside the function execute_program, which handles execute program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are tuple: the value named ’tuple’ in this block.

Line 309: `"""Execute records in the minimum number of legal descriptor packets."""` This is executable documentation attached to the function execute_program, which handles execute program. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 311: `if timeout_seconds <= 0:` This executes the following indented host-code branch only when ‘timeout_seconds <= 0‘ is true. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Context: the function execute_program, which handles execute program.

Line 312: `raise ValueError("timeout_seconds must be positive")` This rejects an invalid state by raising ‘ValueError("timeout_seconds must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 313: `if not records:` This executes the following indented host-code branch only when ‘not records‘ is true. The important operands are not: the value named ’not’ in this block; records: the number of descriptor records accepted since program start. Context: the function execute_program, which handles execute program.

Line 314: `raise ValueError("a descriptor program must contain at least one record")` This rejects an invalid state by raising ‘ValueError("a descriptor program must contain at least one record")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 315: `deadline = monotonic() + timeout_seconds` This assigns ‘monotonic() + timeout_seconds‘ to ‘deadline ‘ in host memory. Here deadline means the value named ’deadline’ in this block. The important operands are monotonic: the value named ’monotonic’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block. Context: the function execute_program, which handles execute program.

Line 316: `output: list[int] = []` This assigns ‘[]‘ to ‘output‘ in host memory. Here output means the value named ’output’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 317: `for packet in self._partition_program(records):` This iterates packet over ‘self._partition_program(records)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_program, which handles execute program.

Line 318: `output.extend(self._execute_program_packet(packet, deadline))` This calls ‘output.extend‘ with ‘self._execute_program_packet(packet, deadline)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 319: `return tuple(output)` This returns ‘tuple(output)‘ to the caller of the function execute_program, which handles execute program, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; output: the value named ’output’ in this block.

Line 321: `def _partition_program(` This starts function _partition_program. Its indented body performs partition program as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 322: `self,` Inside the function _partition_program, which handles partition program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The expression is a literal or structural operator and has no hidden software state.

Line 323: `records: Sequence[ProgramRecord],` Inside the function _partition_program, which handles partition program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are records: the number of descriptor records accepted since program start; Sequence: the value named 'sequence' in this block; ProgramRecord: the value named 'program record' in this block.

Line 324: `) -> tuple[tuple[ProgramRecord, ...], ...]:` Inside the function _partition_program, which handles partition program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are tuple: the value named 'tuple' in this block; ProgramRecord: the value named 'program record' in this block.

Line 325: `packets: list[tuple[ProgramRecord, ...]] = []` This assigns '[]' to 'packets' in host memory. Here packets means the value named 'packets' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _partition_program, which handles partition program.

Line 326: `current: list[ProgramRecord] = []` This assigns '[]' to 'current' in host memory. Here current means the value named 'current' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _partition_program, which handles partition program.

Line 327: `current_input_bytes = 0` This assigns '0' to 'current_input_bytes ' in host memory. Here current_input_bytes means the value named 'current input bytes' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _partition_program, which handles partition program.

Line 328: `current_output_bytes = 0` This assigns '0' to 'current_output_bytes ' in host memory. Here current_output_bytes means the value named 'current output bytes' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _partition_program, which handles partition program.

Line 329: `for record in records:` This iterates record over 'records'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _partition_program, which handles partition program.

Line 330: `record_input_bytes = (3 + len(record.input_beats)) * 8` This assigns '(3 + len(record.input_beats)) * 8' to 'record_input_bytes ' in host memory. Here record_input_bytes means the value named 'record input bytes' in this block. The important operands are len: the value named 'len' in this block; record: the value named 'record' in this block; input_beats: the value named 'input beats' in this block. Context: the function _partition_program, which handles partition program.

Line 331: `record_output_bytes = record.output_beats * 8` This assigns 'record.output_beats * 8' to 'record_output_bytes ' in host memory. Here record_output_bytes means the value named 'record output bytes' in this block. The important operands are record: the value named 'record' in this block; output_beats: the value named 'output beats' in this block. Context: the function _partition_program, which handles partition program.

Line 332: `if (record_input_bytes > self.MAX_DMA_TRANSFER_BYTES` Inside the function _partition_program, which handles partition program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are record_input_bytes: the value named 'record input bytes' in this block; MAX_DMA_TRANSFER_BYTES: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic.

Line 333: `or record_output_bytes > self.MAX_DMA_TRANSFER_BYTES):` Inside the function _partition_program, which handles partition program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are or: the value named 'or' in this block; record_output_bytes: the value named 'record output bytes' in this block; MAX_DMA_TRANSFER_BYTES: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic.

Line 334: `raise ValueError("one descriptor record exceeds the configured AXI DMA length")` This rejects an invalid state by raising ‘ValueError("one descriptor record exceeds the configured AXI DMA length")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 335: `if current and (` This line opens or closes a multi-line Python expression inside the function _partition_program, which handles partition program; the neighboring lines supply its arguments or elements.

Line 336: `current_input_bytes + record_input_bytes > self.MAX_DMA_TRANSFER_BYTES` Inside the function _partition_program, which handles partition program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are current_input_bytes: the value named 'current input bytes' in this block; record_input_bytes: the value named 'record input bytes' in this block; MAX_DMA_TRANSFER_BYTES: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic.

Line 337: `or current_output_bytes + record_output_bytes > self.MAX_DMA_TRANSFER_BYTES):` Inside the function _partition_program, which handles partition program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are or: the value named 'or' in this block; current_output_bytes: the value named 'current output bytes' in this block; record_output_bytes: the value named 'record output bytes' in this block.

Line 338: `packets.append(tuple(current))` This calls ‘packets.append‘ with ‘tuple(current)‘ inside the function _partition_program, which handles partition program. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 339: `current = []` This assigns ‘[]‘ to ‘current ‘ in host memory. Here current means the value named 'current' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _partition_program, which handles partition program.

Line 340: `current_input_bytes = 0` This assigns ‘0‘ to ‘current_input_bytes ‘ in host memory. Here current_input_bytes means the value named 'current input bytes' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _partition_program, which handles partition program.

Line 341: `current_output_bytes = 0` This assigns ‘0‘ to ‘current_output_bytes ‘ in host memory. Here current_output_bytes means the value named 'current output bytes' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _partition_program, which handles partition program.

Line 342: `current.append(record)` This calls ‘current.append‘ with ‘record‘ inside the function _partition_program, which handles partition program. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 343: `current_input_bytes += record_input_bytes` Inside the function _partition_program, which handles partition program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are current_input_bytes: the value named 'current input bytes' in this block; record_input_bytes: the value named 'record input bytes' in this block.

Line 344: `current_output_bytes += record_output_bytes` Inside the function _partition_program, which handles partition program, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are current_output_bytes: the value named 'current output bytes' in this block; record_output_bytes: the value named 'record output bytes' in this block.

Line 345: `packets.append(tuple(current))` This calls ‘packets.append‘ with ‘tuple(current)‘ inside the function _partition_program, which handles partition program. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 346: `return tuple(packets)` This returns ‘tuple(packets)‘ to the caller of the function _partition_program, which handles partition program, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; packets: the value named ’packets’ in this block.

Line 348: `@staticmethod` This decorator changes how the following class or function is constructed. ‘@staticmethod‘ is applied at import time on the host and is part of the class NeuraEdgeDriver, which handles neura edge driver; it creates no FPGA circuit by itself.

Line 349: `def _program_sizes(records: Sequence[ProgramRecord]) -> tuple[int, int]:` This starts function _program_sizes. Its indented body performs program sizes as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 350: `input_words = sum(3 + len(record.input_beats) for record in records)` This assigns ‘sum(3 + len(record.input_beats) for record in records)‘ to ‘input_words ‘ in host memory. Here input_words means the value named ’input words’ in this block. The important operands are sum: the value named ’sum’ in this block; len: the value named ’len’ in this block; record: the value named ’record’ in this block. Context: the function _program_sizes, which handles program sizes.

Line 351: `output_words = sum(record.output_beats for record in records)` This assigns ‘sum(record.output_beats for record in records)‘ to ‘output_words ‘ in host memory. Here output_words means the value named ’output words’ in this block. The important operands are sum: the value named ’sum’ in this block; record: the value named ’record’ in this block; output_beats: the value named ’output beats’ in this block. Context: the function _program_sizes, which handles program sizes.

Line 352: `return input_words, output_words` This returns ‘input_words, output_words‘ to the caller of the function _program_sizes, which handles program sizes, ending this invocation. The important operands are input_words: the value named ’input words’ in this block; output_words: the value named ’output words’ in this block.

Line 354: `def _fill_program_buffer(self, buffer, records: Sequence[ProgramRecord]) -> None:` This starts function _fill_program_buffer. Its indented body performs fill program buffer as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 355: `cursor = 0` This assigns ‘0‘ to ‘cursor ‘ in host memory. Here cursor means the value named ’cursor’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 356: `for record_index, record in enumerate(records):` This iterates record_index, record over ‘enumerate(records)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _fill_program_buffer, which handles fill program buffer.

Line 357: `header = len(record.input_beats)` This assigns ‘len(record.input_beats)‘ to ‘header ‘ in host memory. Here header means the value named ’header’ in this block. The important operands are len: the value named ’len’ in this block; record: the value named ’record’ in this block; input_beats: the value named ’input beats’ in this block. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 358: `header |= record.output_beats << 32` Inside the function _fill_program_buffer, which handles fill program buffer, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are header: the value named ’header’ in this block; record: the value named ’record’ in this block; output_beats: the value named ’output beats’ in this block.

Line 359: `header |= int(record_index == len(records) - 1) << 63` Inside the function _fill_program_buffer, which handles fill program buffer, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are header: the value named 'header' in this block; record_index: an integer selecting one tensor element, lane, bank, or state entry; len: the value named 'len' in this block.

Line 360: `buffer[cursor] = header` This assigns 'header' to 'buffer[cursor] ' in host memory. Here buffer means local storage that decouples producer and consumer timing. The important operands are header: the value named 'header' in this block. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 361: `buffer[cursor + 1] = record.command.value & ((1 << 64) - 1)` Inside the function _fill_program_buffer, which handles fill program buffer, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are buffer: local storage that decouples producer and consumer timing; cursor: the value named 'cursor' in this block; record: the value named 'record' in this block.

Line 362: `buffer[cursor + 2] = record.command.value >> 64` Inside the function _fill_program_buffer, which handles fill program buffer, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are buffer: local storage that decouples producer and consumer timing; cursor: the value named 'cursor' in this block; record: the value named 'record' in this block.

Line 363: `cursor += 3` Inside the function _fill_program_buffer, which handles fill program buffer, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are cursor: the value named 'cursor' in this block.

Line 364: `if isinstance(record.input_beats, PackedStreamBeats):` This executes the following indented host-code branch only when 'isinstance(record.input_beats, PackedStreamBeats)' is true. The important operands are isinstance: the value named 'isinstance' in this block; record: the value named 'record' in this block; input_beats: the value named 'input beats' in this block. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 365: `words = np.frombuffer(record.input_beats.words, dtype=np.uint64)` This assigns 'np.frombuffer(record.input_ dtype=np.uint64)' to 'words ' in host memory. Here words means the value named 'words' in this block. The important operands are frombuffer: local storage that decouples producer and consumer timing; record: the value named 'record' in this block; input_beats: the value named 'input beats' in this block. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 366: `buffer[cursor:cursor + len(words)] = words` This assigns 'words' to 'buffer[cursor' in host memory. Here buffer means local storage that decouples producer and consumer timing. The important operands are words: the value named 'words' in this block. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 367: `cursor += len(words)` Inside the function _fill_program_buffer, which handles fill program buffer, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are cursor: the value named 'cursor' in this block; len: the value named 'len' in this block; words: the value named 'words' in this block.

Line 368: `continue` Inside the function _fill_program_buffer, which handles fill program buffer, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are continue: the value named 'continue' in this block.

Line 369: `for start in range(0, len(record.input_beats), self.PACK_COPY_WORDS):` This iterates start over 'range(0, len(record.input_beats), self.PACK_COPY_WORDS)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _fill_program_buffer, which handles fill program buffer.

Line 370: `beats = record.input_beats[start:start + self.PACK_COPY_WORDS]` This assigns ‘record.input_beats[start:start + self.PACK_COPY_WORDS]‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are record: the value named ’record’ in this block; input_beats: the value named ’input beats’ in this block; start: the value named ’start’ in this block. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 371: `words = np.fromiter(` This statement continues through source line 372; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.fromiter((beat.data for beat in beats), dtype=np.uint64, count=len(beats))‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are fromiter: the value named ’fromiter’ in this block; beat: the value named ’beat’ in this block; data: the value named ’data’ in this block. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 372: `(beat.data for beat in beats), dtype=np.uint64, count=len(beats))` This is continuation line 2 of the 2-line logical statement that begins at source line 371. Its exact fragment is ‘(beat.data for beat in beats), dtype=np.uint64, count=len(beats))‘. The important operands are beat: the value named ’beat’ in this block; data: the value named ’data’ in this block; for: the value named ’for’ in this block. Read together, the complete operation is: This assigns ‘np.fromiter((beat.data for beat in beats), dtype=np.uint64, count=len(beats))‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are fromiter: the value named ’fromiter’ in this block; beat: the value named ’beat’ in this block; data: the value named ’data’ in this block. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 373: `buffer[cursor:cursor + len(beats)] = words` This assigns ‘words‘ to ‘buffer[cursor‘ in host memory. Here buffer means local storage that decouples producer and consumer timing. The important operands are words: the value named ’words’ in this block. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 374: `cursor += len(beats)` Inside the function _fill_program_buffer, which handles fill program buffer, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are cursor: the value named ’cursor’ in this block; len: the value named ’len’ in this block; beats: the value named ’beats’ in this block.

Line 375: `if cursor != len(buffer):` This executes the following indented host-code branch only when ‘cursor != len(buffer)‘ is true. The important operands are cursor: the value named ’cursor’ in this block; len: the value named ’len’ in this block; buffer: local storage that decouples producer and consumer timing. Context: the function _fill_program_buffer, which handles fill program buffer.

Line 376: `raise RuntimeError(f"packed {cursor} descriptor words into a {len(buffer)}-word buffer")` This rejects an invalid state by raising ‘RuntimeError(f"packed {cursor} descriptor words into a {len(buffer)}-word buffer")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 378: `def _execute_program_packet(` This starts function _execute_program_packet. Its indented body performs execute program packet as part of the file that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry; it runs only when called.

Line 379: `self,` Inside the function _execute_program_packet, which handles execute program packet, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The expression is a literal or structural operator and has no hidden software state.

Line 380: `records: Sequence[ProgramRecord],` Inside the function _execute_program_packet, which handles execute program packet, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are records: the number of descriptor records accepted since program start; Sequence: the value named ’sequence’ in this block; ProgramRecord: the value named ’program record’ in this block.

Line 381: `deadline: float,` Inside the function _execute_program_packet, which handles execute program packet, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are deadline: the value named 'deadline' in this block; float: the value named 'float' in this block.

Line 382: `) -> tuple[int, ...]:` Inside the function _execute_program_packet, which handles execute program packet, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are tuple: the value named 'tuple' in this block.

Line 383: `input_words, output_beats = self._program_sizes(records)` This assigns 'self._program_sizes(records)' to 'input_words, output_beats ' in host memory. Here input_words means the value named 'input words' in this block. The important operands are _program_sizes: the value named 'program sizes' in this block; records: the number of descriptor records accepted since program start. Context: the function _execute_program_packet, which handles execute program packet.

Line 384: `self._wait_idle(deadline)` This calls 'self._wait_idle' with 'deadline' inside the function _execute_program_packe which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 385: `input_buffer = None` This assigns 'None' to 'input_buffer ' in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are None: the value named 'none' in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 386: `output_buffer = None` This assigns 'None' to 'output_buffer ' in host memory. Here output_buffer means local storage that decouples producer and consumer timing. The important operands are None: the value named 'none' in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 387: `try:` This is explicit host-side failure handling inside the function _execute_program_packet, which handles execute program packet; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 388: `input_buffer = self.allocator(shape=(input_words,), dtype=np.uint64)` This assigns 'self.allocator(shape=(dtype=np.uint64)' to 'input_buffer ' in host memory. Here input_buffer means local storage that decouples producer and consumer timing. The important operands are allocator: the value named 'allocator' in this block; shape: the value named 'shape' in this block; input_words: the value named 'input words' in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 389: `self._fill_program_buffer(input_buffer, records)` This calls 'self._fill_program_buffer' with 'input_buffer, records' inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 390: `if hasattr(input_buffer, "flush"):` This executes the following indented host-code branch only when 'hasattr(input_buffer, "flush")' is true. The important operands are hasattr: the value named 'hasattr' in this block; input_buffer: local storage that decouples producer and consumer timing; flush: the value named 'flush' in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 391: `input_buffer.flush()` This calls 'input_buffer.flush' with " inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 392: `if output_beats:` This executes the following indented host-code branch only when 'output_beats' is true. The important operands are output_beats: the value named 'output beats' in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 393: `output_buffer = self.allocator(shape=(output_beats,), dtype=np.uint64)` This assigns ‘self.allocator(shape= dtype=np.uint64)‘ to ‘output_buffer ‘ in host memory. Here output_buffer means local storage that decouples producer and consumer timing. The important operands are allocator: the value named ’allocator’ in this block; shape: the value named ’shape’ in this block; output_beats: the value named ’output beats’ in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 394: `self.dma.recvchannel.transfer(output_buffer, nbytes=output_beats * 8)` This calls ‘self.dma.recvchannel.tra with ‘output_buffer, nbytes=output_beats * 8‘ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 396: `self.clear_wrapper_status()` This calls ‘self.clear_wrapper_status‘ with “ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 397: `self.mmio.write(self.registers.program_output_beats, output_beats)` This calls ‘self.mmio.write‘ with ‘self.registers.program_output_beats, output_beats‘ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 398: `self.mmio.write(self.registers.control, self._control_value(1 << 2))` This calls ‘self.mmio.write‘ with ‘self.registers.control, self._control_value(1 << 2)‘ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 399: `self.dma.sendchannel.transfer(input_buffer, nbytes=input_words * 8)` This calls ‘self.dma.sendchannel.trans with ‘input_buffer, nbytes=input_words * 8‘ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 400: `self._wait_dma_channel(self.dma.sendchannel, deadline, "program MM2S")` This calls ‘self._wait_dma_chan with ‘self.dma.sendchannel, deadline, "program MM2S"‘ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 401: `if output_buffer is not None:` This executes the following indented host-code branch only when ‘output_buffer is not None‘ is true. The important operands are output_buffer: local storage that decouples producer and consumer timing; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 402: `self._wait_dma_channel(self.dma.recvchannel, deadline, "program S2MM")` This calls ‘self._wait_dma_chan with ‘self.dma.recvchannel, deadline, "program S2MM"‘ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 404: `while True:` This repeats the following host-code block while ‘True‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 405: `status = self.status()` This assigns ‘self.status()‘ to ‘status ‘ in host memory. Here status means the value named ’status’ in this block. The important operands are status: the value named ’status’ in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 406: `if status & self.STATUS_ERROR:` This executes the following indented host-code branch only when ‘status & self.STATUS_ERROR‘ is true. The important operands are status: the value named ’status’ in this block; STATUS_ERROR: the value named ’status error’ in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 407: `raise NeuraEdgeHardwareError(` This statement continues through source line 408; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"descriptor program failed; status=0x{status:08x}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 408: `f"descriptor program failed; status=0x{status:08x}")` This is continuation line 2 of the 2-line logical statement that begins at source line 407. Its exact fragment is ‘f"descriptor program failed; status=0x{status:08x}")‘. The important operands are f: the value named ’f’ in this block; descriptor: a DMA record header plus one command and its declared payload; program: the value named ’program’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"descriptor program failed; status=0x{status:08x}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 409: `if status & self.STATUS_DONE:` This executes the following indented host-code branch only when ‘status & self.STATUS_DONE‘ is true. The important operands are status: the value named ’status’ in this block; STATUS_DONE: the value named ’status done’ in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 410: `break` Inside the function _execute_program_packet, which handles execute program packet, this statement advances the host implementation that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry. The important operands are break: the value named ’break’ in this block.

Line 411: `if monotonic() >= deadline:` This executes the following indented host-code branch only when ‘monotonic() >= deadline‘ is true. The important operands are monotonic: the value named ’monotonic’ in this block; deadline: the value named ’deadline’ in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 412: `raise NeuraEdgeTimeout("descriptor program did not complete")` This rejects an invalid state by raising ‘NeuraEdgeTimeout("descriptor program did not complete")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 413: `sleep(0.0005)` This calls ‘sleep‘ with ‘0.0005‘ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 414: `completed = int(self.mmio.read(self.registers.program_records))` This assigns ‘int(self.mmio.read(self.regist to ‘completed ‘ in host memory. Here completed means the value named ’completed’ in this block. The important operands are mmio: the value named ’mmio’ in this block; read: the value named ’read’ in this block; registers: the value named ’registers’ in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 415: `if completed != len(records):` This executes the following indented host-code branch only when ‘completed != len(records)‘ is true. The important operands are completed: the value named ’completed’ in this block; len: the value named ’len’ in this block; records: the number of descriptor records accepted since program start. Context: the function _execute_program_packet, which handles execute program packet.

Line 416: `raise NeuraEdgeHardwareError(` This statement continues through source line 417; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"descriptor program completed {completed} records; expected {len(records)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 417: `f"descriptor program completed {completed} records; expected {len(records)}")` This is continuation line 2 of the 2-line logical statement that begins at source line 416. Its exact fragment is ‘f"descriptor program completed {completed} records; expected {len(records)}")‘. The important operands are f: the value named ’f’ in this block; descriptor: a DMA record header plus one command and its declared payload; program: the value named ’program’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘NeuraEdgeHardwareError(f"descriptor program completed {completed} records; expected {len(records)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 418: `if output_buffer is None:` This executes the following indented host-code branch only when ‘output_buffer is None‘ is true. The important operands are output_buffer: local storage that decouples producer and consumer timing; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function _execute_program_packet, which handles execute program packet.

Line 419: `return ()` This returns ‘()‘ to the caller of the function _execute_program_packet, which handles execute program packet, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 420: `if hasattr(output_buffer, "invalidate"):` This executes the following indented host-code branch only when ‘hasattr(output_buffer, "invalidate")‘ is true. The important operands are hasattr: the value named ’hasattr’ in this block; output_buffer: local storage that decouples producer and consumer timing; invalidate: the producer’s statement that its current payload is meaningful. Context: the function _execute_program_packet, which handles execute program packet.

Line 421: `output_buffer.invalidate()` This calls ‘output_buffer.invalidate‘ with “ inside the function _execute_program_pac which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 422: `return tuple(int(value) for value in output_buffer[:output_beats])` This returns ‘tuple(int(value) for value in output_buffer[:output_beats])‘ to the caller of the function _execute_program_packet, which handles execute program packet, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; value: the value named ’value’ in this block; for: the value named ’for’ in this block.

Line 423: `finally:` This is explicit host-side failure handling inside the function _execute_program_packet, which handles execute program packet; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 424: `self._release_buffer(input_buffer)` This calls ‘self._release_buffer‘ with ‘input_buffer‘ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

Line 425: `self._release_buffer(output_buffer)` This calls ‘self._release_buffer‘ with ‘output_buffer‘ inside the function _execute_program_packet, which handles execute program packet. The call advances the host-side job that controls the overlay, AXI-Lite registers, DMA allocation, packet partitioning, errors, and telemetry.

38 software/neuraedge_training.py

Role in the complete system: builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

38.1 Beginner architecture map

Dataflow responsibility The training compiler converts saved forward activations and class loss derivatives into descriptor programs. It emits surrogate-gradient gates for spike nonlinearities, exact tiled linear-backward commands for dX and dW, and Q16 optimizer commands for persistent weight and momentum state. Every payload length, gradient shape, numeric shift, and returned update is checked against the same protocol helpers used by RTL tests.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: TrainingCommandExecutor, TrainingProgramExecutor, BackwardInstruction, CompiledLinearBackward, CompiledAffineBackward, ConvBackwardGroup, CompiledConv2dBackward. Functions: _signed_matrix, _conv_output_size, im2col_signed_nchw, col2im_accumulate_fixed, execute_col2im_accumulate, _tile_starts, compile_linear_backward, compile_affine_backward, compile_conv2d_backward, sigmoid_surrogate_lut_q15, surrogate_lut_addresses, lif_bptt_fixed, _execute_vector8, execute_sgdw_update, execute_int8_weight_projection, execute_lif_bptt, _pair, maxpool2d_backward_fixed, execute_maxpool2d_backward, _shape_elements, linear_backward_ build_spikingformer_training_coverage, execute, execute_program. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

38.2 Complete source

```
"""Exact tiled backward compiler for the NeuraEdge training datapath.

For ``Y = XW`` the accelerator evaluates both matrix derivatives:
``dX = dY W^T`` and ``dW = X^T dY``.  Hardware retains 47-bit unrounded
partial sums between commands, so this compiler may tile every dimension
without changing the integer equation.
"""

from __future__ import annotations

import json
from dataclasses import dataclass
from math import ceil
from pathlib import Path
from typing import Protocol, Sequence

import numpy as np

from neuraedge_protocol import (
    LinearGradientMode,
    LutSuboperation,
    ProgramRecord,
    StreamBeat,
    linear_backward_command,
    lut_command,
    pack_linear_backward_beats,
    pack_lut_eval_beat,
    pack_lut_load_beats,
    pack_optimizer_beats,
    pack_vector_beats,
    sgd_command,
    unpack_linear_backward_result,
    unpack_lut_result,
    unpack_optimizer_result,
    unpack_vector_result,
    vector_command,
)
from neuraedge_golden import VectorOperation, round_shift_signed, saturate_signed


TILE = 8
```

```
class TrainingCommandExecutor(Protocol):
    def execute(
        self,
        command,
        input_beats: Sequence[StreamBeat] = (),
        *,
        output_beats: int = 0,
        timeout_seconds: float = 5.0,
    ) -> tuple[int, ...]: ...


class TrainingProgramExecutor(TrainingCommandExecutor, Protocol):
    def execute_program(
        self,
        records: Sequence[ProgramRecord],
        *,
        timeout_seconds: float = 30.0,
    ) -> tuple[int, ...]: ...


@dataclass(frozen=True)
class BackwardInstruction:
    """One partial tile and the global matrix region to which it belongs."""

    record: ProgramRecord
    mode: LinearGradientMode
    row_start: int
    input_start: int
    output_start: int
    rows: int
    inputs: int
    outputs: int
    accumulate: bool
    suppress_output: bool


@dataclass(frozen=True)
class CompiledLinearBackward:
    """Executable descriptor program for one arbitrary signed linear layer."""

    shape: tuple[int, int, int]
    input_gradient_shift: int
    weight_gradient_shift: int
    instructions: tuple[BackwardInstruction, ...]
    estimated_input_bytes: int
    mathematical_multiplications: int

    @property
    def records(self) -> tuple[ProgramRecord, ...]:
        return tuple(instruction.record for instruction in self.instructions)

    @property
    def output_beats(self) -> int:
        return sum(instruction.record.output_beats for instruction in self.instructions)

    @property
    def partial_tile_count(self) -> int:
        return len(self.instructions)

    def _decode_outputs(self, instruction_words: Sequence[Sequence[int]]) -> tuple[np.ndarray, np.ndarray]:
        rows, inputs, outputs = self.shape
        input_gradient = np.zeros((rows, inputs), dtype=np.int64)
        weight_gradient = np.zeros((inputs, outputs), dtype=np.int64)
        if len(instruction_words) != len(self.instructions):
            raise RuntimeError("backward result partition does not match the instruction count")
        for instruction, words in zip(self.instructions, instruction_words):
            if len(words) != instruction.record.output_beats:
                raise RuntimeError(
                    f"backward tile returned {len(words)} beats; "
                    f"expected {instruction.record.output_beats}"
                )
            if instruction.suppress_output:
                continue
            dx_tile, dw_tile = unpack_linear_backward_result(
                words,
                instruction.rows,
                instruction.inputs,
                instruction.outputs,
                mode=instruction.mode,
            )
            if instruction.mode == LinearGradientMode.INPUT:
                input_gradient[
                    instruction.row_start:instruction.row_start + instruction.rows,
                    instruction.input_start:instruction.input_start + instruction.inputs,
                ] = np.asarray(dx_tile, dtype=np.int64)
```

```
            else:
                weight_gradient[
                    instruction.input_start:instruction.input_start + instruction.inputs,
                    instruction.output_start:instruction.output_start + instruction.outputs,
                ] = np.asarray(dw_tile, dtype=np.int64)
        return input_gradient, weight_gradient

    def execute(
        self,
        driver: TrainingCommandExecutor,
        *,
        timeout_seconds: float = 5.0,
    ) -> tuple[np.ndarray, np.ndarray]:
        partitions: list[tuple[int, ...]] = []
        for instruction in self.instructions:
            partitions.append(driver.execute(
                instruction.record.command,
                instruction.record.input_beats,
                output_beats=instruction.record.output_beats,
                timeout_seconds=timeout_seconds,
            ))
        return self._decode_outputs(partitions)

    def execute_batched(
        self,
        driver: TrainingProgramExecutor,
        *,
        timeout_seconds: float = 30.0,
    ) -> tuple[np.ndarray, np.ndarray]:
        words = driver.execute_program(self.records, timeout_seconds=timeout_seconds)
        if len(words) != self.output_beats:
            raise RuntimeError(
                f"backward program returned {len(words)} beats; expected {self.output_beats}"
            )
        partitions: list[tuple[int, ...]] = []
        cursor = 0
        for instruction in self.instructions:
            end = cursor + instruction.record.output_beats
            partitions.append(tuple(words[cursor:end]))
            cursor = end
        return self._decode_outputs(partitions)


@dataclass(frozen=True)
class CompiledAffineBackward:
    """Linear backward plan whose final augmented row represents the bias."""

    linear: CompiledLinearBackward
    input_features: int

    def split(
        self,
        augmented_input_gradient: np.ndarray,
        augmented_weight_gradient: np.ndarray,
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        return (
            augmented_input_gradient[:, :self.input_features],
            augmented_weight_gradient[:self.input_features, :],
            augmented_weight_gradient[self.input_features, :],
        )

    def execute(
        self,
        driver: TrainingCommandExecutor,
        *,
        timeout_seconds: float = 5.0,
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        return self.split(*self.linear.execute(driver, timeout_seconds=timeout_seconds))

    def execute_batched(
        self,
        driver: TrainingProgramExecutor,
        *,
        timeout_seconds: float = 30.0,
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        return self.split(*self.linear.execute_batched(driver, timeout_seconds=timeout_seconds))


@dataclass(frozen=True)
class ConvBackwardGroup:
    input_channel_start: int
    output_channel_start: int
    input_channels: int
    output_channels: int
    affine: CompiledAffineBackward


```

```
@dataclass(frozen=True)
class CompiledConv2dBackward:
    input_shape: tuple[int, int, int, int]
    weight_shape: tuple[int, int, int, int]
    output_shape: tuple[int, int, int, int]
    kernel: tuple[int, int]
    stride: tuple[int, int]
    padding: tuple[int, int]
    dilation: tuple[int, int]
    groups: int
    group_programs: tuple[ConvBackwardGroup, ...]

    @property
    def mathematical_multiplications(self) -> int:
        return sum(
            group.affine.linear.mathematical_multiplications
            for group in self.group_programs
        )

    @property
    def partial_tile_count(self) -> int:
        return sum(
            group.affine.linear.partial_tile_count for group in self.group_programs
        )

    def execute_batched(
        self,
        driver: TrainingProgramExecutor,
        *,
        timeout_seconds: float = 30.0,
        fabric_col2im: bool = True,
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        input_gradient = np.zeros(self.input_shape, dtype=np.int64)
        weight_gradient = np.zeros(self.weight_shape, dtype=np.int64)
        bias_gradient = np.zeros(self.output_shape[1], dtype=np.int64)
        output_height, output_width = self.output_shape[2:]
        kernel_h, kernel_w = self.kernel
        for group in self.group_programs:
            patch_gradient, matrix_gradient, group_bias = group.affine.execute_batched(
                driver, timeout_seconds=timeout_seconds
            )
            if fabric_col2im:
                group_input_gradient = execute_col2im_accumulate(
                    driver,
                    patch_gradient,
                    batch=self.input_shape[0],
                    channels=group.input_channels,
                    input_height=self.input_shape[2],
                    input_width=self.input_shape[3],
                    output_height=output_height,
                    output_width=output_width,
                    kernel=self.kernel,
                    stride=self.stride,
                    padding=self.padding,
                    dilation=self.dilation,
                    timeout_seconds=timeout_seconds,
                )
            else:
                group_input_gradient = col2im_accumulate_fixed(
                    patch_gradient,
                    batch=self.input_shape[0],
                    channels=group.input_channels,
                    input_height=self.input_shape[2],
                    input_width=self.input_shape[3],
                    output_height=output_height,
                    output_width=output_width,
                    kernel=self.kernel,
                    stride=self.stride,
                    padding=self.padding,
                    dilation=self.dilation,
                )
            input_start = group.input_channel_start
            output_start = group.output_channel_start
            input_gradient[:, input_start:input_start + group.input_channels] = group_input_gradient
            matrix = matrix_gradient.T.reshape(
                group.output_channels,
                group.input_channels,
                kernel_h,
                kernel_w,
            )
            weight_gradient[
                output_start:output_start + group.output_channels
            ] = matrix
            bias_gradient[
                output_start:output_start + group.output_channels
            ] = group_bias
        return input_gradient, weight_gradient, bias_gradient
```

```

def _signed_matrix(name: str, value, bits: int) -> np.ndarray:
    array = np.asarray(value)
    if array.ndim != 2 or array.size == 0:
        raise ValueError(f"{name} must be a non-empty two-dimensional matrix")
    if not np.issubdtype(array.dtype, np.integer):
        raise ValueError(f"{name} must contain integers")
    result = array.astype(np.int64)
    minimum = -(1 << (bits - 1))
    maximum = (1 << (bits - 1)) - 1
    if np.any(result < minimum) or np.any(result > maximum):
        raise ValueError(f"{name} must fit signed {bits}-bit values")
    return result


def _conv_output_size(
    input_size: int,
    kernel: int,
    stride: int,
    padding: int,
    dilation: int,
) -> int:
    return (input_size + 2 * padding - dilation * (kernel - 1) - 1) // stride + 1


def im2col_signed_nchw(
    values,
    *,
    kernel,
    stride,
    padding,
    dilation,
) -> np.ndarray:
    """Flatten NCHW convolution patches in batch/output-y/output-x order."""

    source = np.asarray(values, dtype=np.int64)
    if source.ndim != 4:
        raise ValueError("im2col source must be an NCHW tensor")
    kernel_h, kernel_w = _pair("kernel", kernel)
    stride_h, stride_w = _pair("stride", stride)
    dilation_h, dilation_w = _pair("dilation", dilation)
    pad_h, pad_w = (padding, padding) if isinstance(padding, int) else tuple(padding)
    batch, channels, height, width = source.shape
    output_height = _conv_output_size(height, kernel_h, stride_h, pad_h, dilation_h)
    output_width = _conv_output_size(width, kernel_w, stride_w, pad_w, dilation_w)
    patches = np.zeros(
        (batch * output_height * output_width, channels * kernel_h * kernel_w),
        dtype=np.int64,
    )
    row = 0
    for batch_index in range(batch):
        for output_y in range(output_height):
            for output_x in range(output_width):
                column = 0
                for channel in range(channels):
                    for kernel_y in range(kernel_h):
                        input_y = output_y * stride_h - pad_h + kernel_y * dilation_h
                        for kernel_x in range(kernel_w):
                            input_x = output_x * stride_w - pad_w + kernel_x * dilation_w
                            if 0 <= input_y < height and 0 <= input_x < width:
                                patches[row, column] = source[
                                    batch_index, channel, input_y, input_x
                                ]
                            column += 1
                row += 1
    return patches


def col2im_accumulate_fixed(
    patch_gradients,
    *,
    batch: int,
    channels: int,
    input_height: int,
    input_width: int,
    output_height: int,
    output_width: int,
    kernel,
    stride,
    padding,
    dilation,
) -> np.ndarray:
    """Scatter patch gradients with one signed-s32 saturation per addition."""

    patches = np.asarray(patch_gradients, dtype=np.int64)
    kernel_h, kernel_w = _pair("kernel", kernel)
```

```
    stride_h, stride_w = _pair("stride", stride)
    dilation_h, dilation_w = _pair("dilation", dilation)
    pad_h, pad_w = (padding, padding) if isinstance(padding, int) else tuple(padding)
    expected_shape = (
        batch * output_height * output_width,
        channels * kernel_h * kernel_w,
    )
    if patches.shape != expected_shape:
        raise ValueError(f"patch gradient shape must be {expected_shape}")
    output = np.zeros((batch, channels, input_height, input_width), dtype=np.int64)
    row = 0
    for batch_index in range(batch):
        for output_y in range(output_height):
            for output_x in range(output_width):
                column = 0
                for channel in range(channels):
                    for kernel_y in range(kernel_h):
                        input_y = output_y * stride_h - pad_h + kernel_y * dilation_h
                        for kernel_x in range(kernel_w):
                            input_x = output_x * stride_w - pad_w + kernel_x * dilation_w
                            if 0 <= input_y < input_height and 0 <= input_x < input_width:
                                value = output[batch_index, channel, input_y, input_x]
                                output[batch_index, channel, input_y, input_x] = saturate_signed(
                                    np.asarray(value + patches[row, column], dtype=np.int64), 32
                                )
                            column += 1
                row += 1
    return output


def execute_col2im_accumulate(
    driver: TrainingCommandExecutor,
    patch_gradients,
    *,
    batch: int,
    channels: int,
    input_height: int,
    input_width: int,
    output_height: int,
    output_width: int,
    kernel,
    stride,
    padding,
    dilation,
    timeout_seconds: float = 5.0,
) -> np.ndarray:
    """Scatter every overlapping convolution gradient with fabric vector adds."""

    patches = np.asarray(patch_gradients, dtype=np.int64)
    kernel_h, kernel_w = _pair("kernel", kernel)
    stride_h, stride_w = _pair("stride", stride)
    dilation_h, dilation_w = _pair("dilation", dilation)
    pad_h, pad_w = (padding, padding) if isinstance(padding, int) else tuple(padding)
    expected_shape = (
        batch * output_height * output_width,
        channels * kernel_h * kernel_w,
    )
    if patches.shape != expected_shape:
        raise ValueError(f"patch gradient shape must be {expected_shape}")
    output = np.zeros((batch, channels, input_height, input_width), dtype=np.int64)
    row = 0
    pending: list[tuple[tuple[int, int, int, int], int]] = []

    def flush() -> None:
        nonlocal pending
        if not pending:
            return
        left = [int(output[address]) for address, _ in pending]
        right = [value for _, value in pending]
        left.extend([0] * (8 - len(pending)))
        right.extend([0] * (8 - len(pending)))
        summed = _execute_vector8(
            driver,
            VectorOperation.ADD_SAT,
            left,
            right,
            timeout_seconds=timeout_seconds,
        )
        for lane, (address, _) in enumerate(pending):
            output[address] = summed[lane]
        pending = []

    for batch_index in range(batch):
        for output_y in range(output_height):
            for output_x in range(output_width):
                column = 0
                for channel in range(channels):
```

```
                    for kernel_y in range(kernel_h):
                        input_y = output_y * stride_h - pad_h + kernel_y * dilation_h
                        for kernel_x in range(kernel_w):
                            input_x = output_x * stride_w - pad_w + kernel_x * dilation_w
                            if 0 <= input_y < input_height and 0 <= input_x < input_width:
                                address = (batch_index, channel, input_y, input_x)
                                if len(pending) == 8 or any(item[0] == address for item in pending):
                                    flush()
                                pending.append((address, int(patches[row, column])))
                            column += 1
                row += 1
    flush()
    return output


def _tile_starts(size: int) -> range:
    return range(0, size, TILE)


def compile_linear_backward(
    activations,
    weights,
    output_gradients,
    *,
    input_gradient_shift: int = 0,
    weight_gradient_shift: int = 0,
) -> CompiledLinearBackward:
    """Compile exact dX and dW passes for matrices of any positive shape."""

    x = _signed_matrix("activations", activations, 16)
    w = _signed_matrix("weights", weights, 16)
    dy = _signed_matrix("output_gradients", output_gradients, 16)
    rows, inputs = x.shape
    weight_inputs, outputs = w.shape
    if weight_inputs != inputs or dy.shape != (rows, outputs):
        raise ValueError("linear backward matrix dimensions do not align")
    if not 0 <= input_gradient_shift <= 31:
        raise ValueError("input_gradient_shift must be in [0,31]")
    if not 0 <= weight_gradient_shift <= 31:
        raise ValueError("weight_gradient_shift must be in [0,31]")

    instructions: list[BackwardInstruction] = []
    input_bytes = 0

    # dX tiles retain one MxK accumulator while traversing every N chunk.
    for row_start in _tile_starts(rows):
        row_end = min(row_start + TILE, rows)
        for input_start in _tile_starts(inputs):
            input_end = min(input_start + TILE, inputs)
            output_starts = tuple(_tile_starts(outputs))
            for chunk_index, output_start in enumerate(output_starts):
                output_end = min(output_start + TILE, outputs)
                final = chunk_index == len(output_starts) - 1
                x_tile = x[row_start:row_end, input_start:input_end]
                w_tile = w[input_start:input_end, output_start:output_end]
                dy_tile = dy[row_start:row_end, output_start:output_end]
                beats = pack_linear_backward_beats(
                    x_tile.tolist(),
                    w_tile.tolist(),
                    dy_tile.tolist(),
                    mode=LinearGradientMode.INPUT,
                )
                record = ProgramRecord(
                    linear_backward_command(
                        row_end - row_start,
                        input_end - input_start,
                        output_end - output_start,
                        input_gradient_shift=input_gradient_shift,
                        weight_gradient_shift=weight_gradient_shift,
                        mode=LinearGradientMode.INPUT,
                        accumulate=chunk_index != 0,
                        suppress_output=not final,
                    ),
                    beats,
                    4 * (row_end - row_start) if final else 0,
                )
                instructions.append(BackwardInstruction(
                    record,
                    LinearGradientMode.INPUT,
                    row_start,
                    input_start,
                    output_start,
                    row_end - row_start,
                    input_end - input_start,
                    output_end - output_start,
                    chunk_index != 0,
                    not final,
```

```
                ))
                input_bytes += len(beats) * 8

    # dW tiles retain one KxN accumulator while traversing every M chunk.
    for input_start in _tile_starts(inputs):
        input_end = min(input_start + TILE, inputs)
        for output_start in _tile_starts(outputs):
            output_end = min(output_start + TILE, outputs)
            row_starts = tuple(_tile_starts(rows))
            for chunk_index, row_start in enumerate(row_starts):
                row_end = min(row_start + TILE, rows)
                final = chunk_index == len(row_starts) - 1
                x_tile = x[row_start:row_end, input_start:input_end]
                w_tile = w[input_start:input_end, output_start:output_end]
                dy_tile = dy[row_start:row_end, output_start:output_end]
                beats = pack_linear_backward_beats(
                    x_tile.tolist(),
                    w_tile.tolist(),
                    dy_tile.tolist(),
                    mode=LinearGradientMode.WEIGHT,
                )
                record = ProgramRecord(
                    linear_backward_command(
                        row_end - row_start,
                        input_end - input_start,
                        output_end - output_start,
                        input_gradient_shift=input_gradient_shift,
                        weight_gradient_shift=weight_gradient_shift,
                        mode=LinearGradientMode.WEIGHT,
                        accumulate=chunk_index != 0,
                        suppress_output=not final,
                    ),
                    beats,
                    3 * (input_end - input_start) if final else 0,
                )
                instructions.append(BackwardInstruction(
                    record,
                    LinearGradientMode.WEIGHT,
                    row_start,
                    input_start,
                    output_start,
                    row_end - row_start,
                    input_end - input_start,
                    output_end - output_start,
                    chunk_index != 0,
                    not final,
                ))
                input_bytes += len(beats) * 8

    return CompiledLinearBackward(
        (rows, inputs, outputs),
        input_gradient_shift,
        weight_gradient_shift,
        tuple(instructions),
        input_bytes,
        2 * rows * inputs * outputs,
    )


def compile_affine_backward(
    activations,
    weights,
    bias,
    output_gradients,
    *,
    input_gradient_shift: int = 0,
    weight_gradient_shift: int = 0,
) -> CompiledAffineBackward:
    """Compile weight, bias, and input gradients for ``Y = XW + b``."""

    x = _signed_matrix("activations", activations, 16)
    w = _signed_matrix("weights", weights, 16)
    bias_array = np.asarray(bias)
    if bias_array.ndim != 1 or bias_array.shape[0] != w.shape[1]:
        raise ValueError("bias must be one-dimensional with one value per output")
    bias_matrix = _signed_matrix("bias", bias_array.reshape(1, -1), 16)
    if x.shape[1] != w.shape[0]:
        raise ValueError("affine activation and weight dimensions do not align")
    augmented_x = np.concatenate(
        (x, np.ones((x.shape[0], 1), dtype=np.int64)), axis=1
    )
    augmented_w = np.concatenate((w, bias_matrix), axis=0)
    return CompiledAffineBackward(
        compile_linear_backward(
            augmented_x,
            augmented_w,
            output_gradients,
```

```
            input_gradient_shift=input_gradient_shift,
            weight_gradient_shift=weight_gradient_shift,
        ),
        x.shape[1],
    )


def compile_conv2d_backward(
    activations,
    weights,
    bias,
    output_gradients,
    *,
    stride=1,
    padding=0,
    dilation=1,
    groups: int = 1,
    input_gradient_shift: int = 0,
    weight_gradient_shift: int = 0,
) -> CompiledConv2dBackward:
    """Lower a signed NCHW convolution backward pass to exact affine tiles."""

    x = np.asarray(activations)
    w = np.asarray(weights)
    dy = np.asarray(output_gradients)
    bias_array = np.asarray(bias)
    for name, array, bits in (
        ("activations", x, 16),
        ("weights", w, 16),
        ("output_gradients", dy, 16),
        ("bias", bias_array, 16),
    ):
        if not np.issubdtype(array.dtype, np.integer):
            raise ValueError(f"{name} must contain integers")
        minimum = -(1 << (bits - 1))
        maximum = (1 << (bits - 1)) - 1
        if np.any(array < minimum) or np.any(array > maximum):
            raise ValueError(f"{name} must fit signed {bits}-bit values")
    if x.ndim != 4 or w.ndim != 4 or dy.ndim != 4 or bias_array.ndim != 1:
        raise ValueError("convolution operands must use NCHW/OIHW tensors and a bias vector")
    if groups <= 0:
        raise ValueError("groups must be positive")
    batch, input_channels, input_height, input_width = x.shape
    output_channels, weight_inputs, kernel_h, kernel_w = w.shape
    if input_channels % groups or output_channels % groups:
        raise ValueError("input and output channels must be divisible by groups")
    input_channels_per_group = input_channels // groups
    output_channels_per_group = output_channels // groups
    if weight_inputs != input_channels_per_group:
        raise ValueError("weight input channels do not match the grouped convolution")
    if bias_array.shape != (output_channels,):
        raise ValueError("bias must contain one value per output channel")
    stride_pair = _pair("stride", stride)
    dilation_pair = _pair("dilation", dilation)
    if isinstance(padding, int):
        padding_pair = (int(padding), int(padding))
    else:
        padding_pair = tuple(map(int, padding))
        if len(padding_pair) != 2:
            raise ValueError("padding must be an integer or a pair")
    if min(padding_pair) < 0:
        raise ValueError("padding must be nonnegative")
    output_height = _conv_output_size(
        input_height, kernel_h, stride_pair[0], padding_pair[0], dilation_pair[0]
    )
    output_width = _conv_output_size(
        input_width, kernel_w, stride_pair[1], padding_pair[1], dilation_pair[1]
    )
    expected_output_shape = (batch, output_channels, output_height, output_width)
    if dy.shape != expected_output_shape:
        raise ValueError(
            f"output_gradients must have shape {expected_output_shape}; received {dy.shape}"
        )
    programs: list[ConvBackwardGroup] = []
    for group in range(groups):
        input_start = group * input_channels_per_group
        output_start = group * output_channels_per_group
        patches = im2col_signed_nchw(
            x[:, input_start:input_start + input_channels_per_group],
            kernel=(kernel_h, kernel_w),
            stride=stride_pair,
            padding=padding_pair,
            dilation=dilation_pair,
        )
        weight_matrix = w[
            output_start:output_start + output_channels_per_group
        ].reshape(output_channels_per_group, -1).T
```

```
        gradient_matrix = dy[
            :, output_start:output_start + output_channels_per_group
        ].transpose(0, 2, 3, 1).reshape(-1, output_channels_per_group)
        programs.append(ConvBackwardGroup(
            input_start,
            output_start,
            input_channels_per_group,
            output_channels_per_group,
            compile_affine_backward(
                patches,
                weight_matrix,
                bias_array[output_start:output_start + output_channels_per_group],
                gradient_matrix,
                input_gradient_shift=input_gradient_shift,
                weight_gradient_shift=weight_gradient_shift,
            ),
        ))
    return CompiledConv2dBackward(
        tuple(map(int, x.shape)),
        tuple(map(int, w.shape)),
        tuple(map(int, expected_output_shape)),
        (kernel_h, kernel_w),
        stride_pair,
        padding_pair,
        dilation_pair,
        groups,
        tuple(programs),
    )


def sigmoid_surrogate_lut_q15(
    *,
    alpha: float = 4.0,
    membrane_fractional_bits: int = 10,
    address_shift: int | None = None,
) -> tuple[np.ndarray, int]:
    """Quantize SpikingJelly's sigmoid surrogate derivative into 256 Q15 entries."""

    if not np.isfinite(alpha) or alpha <= 0:
        raise ValueError("surrogate alpha must be finite and positive")
    if not 0 <= membrane_fractional_bits <= 22:
        raise ValueError("membrane_fractional_bits must be in [0,22]")
    selected_shift = (
        max(membrane_fractional_bits - 5, 0)
        if address_shift is None
        else int(address_shift)
    )
    if not 0 <= selected_shift <= membrane_fractional_bits:
        raise ValueError("address_shift must be in [0, membrane_fractional_bits]")
    addresses = np.arange(256, dtype=np.float64)
    delta = (addresses - 128.0) * (2.0 ** selected_shift)
    delta /= 2.0 ** membrane_fractional_bits
    sigmoid = 1.0 / (1.0 + np.exp(-alpha * delta))
    derivative = alpha * sigmoid * (1.0 - sigmoid)
    table = np.rint(derivative * (1 << 15))
    table = np.clip(table, 0, (1 << 15) - 1).astype(np.int64)
    return table, selected_shift


def surrogate_lut_addresses(
    pre_reset_membrane,
    threshold: int,
    address_shift: int,
) -> np.ndarray:
    """Map signed fixed-point membrane distance to a saturating LUT address."""

    if not 0 <= address_shift <= 31:
        raise ValueError("address_shift must be in [0,31]")
    membrane = np.asarray(pre_reset_membrane, dtype=np.int64)
    distance = membrane - int(threshold)
    quantized = round_shift_signed(distance, address_shift)
    return np.clip(quantized + 128, 0, 255).astype(np.uint8)


def lif_bptt_fixed(
    pre_reset_membrane,
    spikes,
    spike_gradients,
    *,
    threshold: int,
    membrane_gradients=None,
    reciprocal_tau_q15: int = 1 << 14,
    surrogate_table_q15=None,
    surrogate_address_shift: int = 5,
) -> tuple[np.ndarray, np.ndarray]:
    """Bit-exact detached-reset hard-LIF BPTT using signed Q15 products."""

```

```
    membrane = np.asarray(pre_reset_membrane, dtype=np.int64)
    spike = np.asarray(spikes, dtype=np.int64)
    grad_spike = np.asarray(spike_gradients, dtype=np.int64)
    if membrane.ndim != 2 or spike.shape != membrane.shape or grad_spike.shape != membrane.shape:
        raise ValueError("LIF trace, spike, and gradient arrays must share shape [time, neurons]")
    if np.any((spike != 0) & (spike != 1)):
        raise ValueError("LIF spikes must be binary")
    if membrane_gradients is None:
        grad_v = np.zeros_like(membrane)
    else:
        grad_v = np.asarray(membrane_gradients, dtype=np.int64)
        if grad_v.shape != membrane.shape:
            raise ValueError("membrane gradients must match the LIF trace shape")
    if not 0 <= reciprocal_tau_q15 <= (1 << 15):
        raise ValueError("reciprocal_tau_q15 must be in [0,32768]")
    if surrogate_table_q15 is None:
        surrogate_table_q15, _ = sigmoid_surrogate_lut_q15(
            address_shift=surrogate_address_shift
        )
    table = np.asarray(surrogate_table_q15, dtype=np.int64)
    if table.shape != (256,) or np.any(table < 0) or np.any(table > 32767):
        raise ValueError("surrogate table must contain 256 nonnegative signed-Q15 values")
    addresses = surrogate_lut_addresses(membrane, threshold, surrogate_address_shift)
    derivative = table[addresses]
    grad_input = np.zeros_like(membrane)
    grad_h_trace = np.zeros_like(membrane)
    grad_h_next = np.zeros(membrane.shape[1], dtype=np.int64)
    leak_q15 = (1 << 15) - reciprocal_tau_q15
    for timestep in range(membrane.shape[0] - 1, -1, -1):
        temporal = saturate_signed(
            round_shift_signed(grad_h_next * leak_q15, 15), 32
        )
        membrane_path = saturate_signed(grad_v[timestep] + temporal, 32)
        membrane_path = np.where(spike[timestep] == 0, membrane_path, 0)
        spike_path = saturate_signed(
            round_shift_signed(grad_spike[timestep] * derivative[timestep], 15), 32
        )
        grad_h = saturate_signed(membrane_path + spike_path, 32)
        grad_input[timestep] = saturate_signed(
            round_shift_signed(grad_h * reciprocal_tau_q15, 15), 32
        )
        grad_h_trace[timestep] = grad_h
        grad_h_next = grad_h
    return grad_input, grad_h_trace


def _execute_vector8(
    driver: TrainingCommandExecutor,
    operation: VectorOperation,
    operand_a: Sequence[int],
    operand_b: Sequence[int],
    *,
    timeout_seconds: float,
    clamp_minimum: int = -(1 << 31),
    clamp_maximum: int = (1 << 31) - 1,
) -> np.ndarray:
    words = driver.execute(
        vector_command(int(operation)),
        pack_vector_beats(
            operand_a,
            operand_b,
            clamp_minimum=clamp_minimum,
            clamp_maximum=clamp_maximum,
        ),
        output_beats=4,
        timeout_seconds=timeout_seconds,
    )
    if len(words) != 4:
        raise RuntimeError(f"vector operation returned {len(words)} beats; expected four")
    return np.asarray(unpack_vector_result(words), dtype=np.int64)


def execute_sgdw_update(
    driver: TrainingCommandExecutor,
    weights,
    gradients,
    velocities,
    *,
    learning_rate_q16: int,
    momentum_q16: int,
    weight_decay_q16: int,
    update_right_shift: int = 16,
    timeout_seconds: float = 5.0,
) -> tuple[np.ndarray, np.ndarray]:
    """Update an arbitrary tensor through the fabric's eight SGDW lanes."""

    weight = np.asarray(weights, dtype=np.int64)
```

```
    gradient = np.asarray(gradients, dtype=np.int64)
    velocity = np.asarray(velocities, dtype=np.int64)
    if weight.shape != gradient.shape or weight.shape != velocity.shape or weight.size == 0:
        raise ValueError("weights, gradients, and velocities must share a non-empty shape")
    if np.any(weight < -(1 << 15)) or np.any(weight > (1 << 15) - 1):
        raise ValueError("optimizer weights must fit signed 16-bit")
    for name, value in (("gradients", gradient), ("velocities", velocity)):
        if np.any(value < -(1 << 23)) or np.any(value > (1 << 23) - 1):
            raise ValueError(f"optimizer {name} must fit signed 24-bit")
    flat_weight = weight.reshape(-1)
    flat_gradient = gradient.reshape(-1)
    flat_velocity = velocity.reshape(-1)
    next_weight = np.zeros_like(flat_weight)
    next_velocity = np.zeros_like(flat_velocity)
    command = sgd_command(
        learning_rate_q16,
        momentum_q16,
        weight_decay_q16,
        update_right_shift=update_right_shift,
    )
    for start in range(0, flat_weight.size, 8):
        stop = min(start + 8, flat_weight.size)
        active = stop - start

        def padded(values):
            return tuple(map(int, values)) + (0,) * (8 - active)

        words = driver.execute(
            command,
            pack_optimizer_beats(
                padded(flat_weight[start:stop]),
                padded(flat_gradient[start:stop]),
                padded(flat_velocity[start:stop]),
            ),
            output_beats=8,
            timeout_seconds=timeout_seconds,
        )
        updated_weight, updated_velocity = unpack_optimizer_result(words)
        next_weight[start:stop] = updated_weight[:active]
        next_velocity[start:stop] = updated_velocity[:active]
    return next_weight.reshape(weight.shape), next_velocity.reshape(weight.shape)


def execute_int8_weight_projection(
    driver: TrainingCommandExecutor,
    master_weights,
    *,
    timeout_seconds: float = 5.0,
) -> np.ndarray:
    """Clamp signed-s16 master weights to the deployed signed-int8 tensor."""

    master = np.asarray(master_weights, dtype=np.int64)
    if master.size == 0 or np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1):
        raise ValueError("master_weights must be a non-empty signed-16 tensor")
    flat = master.reshape(-1)
    result = np.zeros_like(flat)
    for start in range(0, flat.size, 8):
        stop = min(start + 8, flat.size)
        active = stop - start
        operand = tuple(map(int, flat[start:stop])) + (0,) * (8 - active)
        clamped = _execute_vector8(
            driver,
            VectorOperation.CLAMP,
            operand,
            (0,) * 8,
            timeout_seconds=timeout_seconds,
            clamp_minimum=-128,
            clamp_maximum=127,
        )
        result[start:stop] = clamped[:active]
    return result.reshape(master.shape).astype(np.int8)


def execute_lif_bptt(
    driver: TrainingCommandExecutor,
    pre_reset_membrane,
    spikes,
    spike_gradients,
    *,
    threshold: int,
    membrane_gradients=None,
    reciprocal_tau_q15: int = 1 << 14,
    surrogate_table_q15=None,
    surrogate_address_shift: int = 5,
    load_surrogate_table: bool = True,
    timeout_seconds: float = 5.0,
) -> tuple[np.ndarray, np.ndarray]:
```

```
    """Execute detached-reset hard-LIF BPTT entirely through fabric operators."""

    membrane = np.asarray(pre_reset_membrane, dtype=np.int64)
    spike = np.asarray(spikes, dtype=np.int64)
    grad_spike = np.asarray(spike_gradients, dtype=np.int64)
    if membrane.ndim != 2 or spike.shape != membrane.shape or grad_spike.shape != membrane.shape:
        raise ValueError("LIF trace, spike, and gradient arrays must share shape [time, neurons]")
    if membrane_gradients is None:
        grad_v = np.zeros_like(membrane)
    else:
        grad_v = np.asarray(membrane_gradients, dtype=np.int64)
        if grad_v.shape != membrane.shape:
            raise ValueError("membrane gradients must match the LIF trace shape")
    if surrogate_table_q15 is None:
        surrogate_table_q15, _ = sigmoid_surrogate_lut_q15(
            address_shift=surrogate_address_shift
        )
    table = np.asarray(surrogate_table_q15, dtype=np.int64)
    if table.shape != (256,):
        raise ValueError("surrogate table must contain exactly 256 entries")
    if load_surrogate_table:
        load_words = driver.execute(
            lut_command(LutSuboperation.LOAD, 256),
            pack_lut_load_beats(
                tuple((address, int(table[address])) for address in range(256))),
            output_beats=0,
            timeout_seconds=timeout_seconds,
        )
        if load_words:
            raise RuntimeError("LUT load unexpectedly returned stream data")

    addresses = surrogate_lut_addresses(membrane, threshold, surrogate_address_shift)
    time_steps, neurons = membrane.shape
    grad_input = np.zeros_like(membrane)
    grad_h_trace = np.zeros_like(membrane)
    grad_h_next = np.zeros(neurons, dtype=np.int64)
    leak_q15 = (1 << 15) - int(reciprocal_tau_q15)
    for timestep in range(time_steps - 1, -1, -1):
        for start in range(0, neurons, 8):
            stop = min(start + 8, neurons)
            active = stop - start

            def padded(values, fill=0):
                return tuple(map(int, values)) + (fill,) * (8 - active)

            temporal = _execute_vector8(
                driver,
                VectorOperation.MULTIPLY_Q15,
                padded(grad_h_next[start:stop]),
                (leak_q15,) * 8,
                timeout_seconds=timeout_seconds,
            )
            membrane_path = _execute_vector8(
                driver,
                VectorOperation.ADD_SAT,
                padded(grad_v[timestep, start:stop]),
                temporal,
                timeout_seconds=timeout_seconds,
            )
            membrane_path = _execute_vector8(
                driver,
                VectorOperation.SELECT_ZERO,
                padded(spike[timestep, start:stop]),
                membrane_path,
                timeout_seconds=timeout_seconds,
            )
            lut_words = driver.execute(
                lut_command(LutSuboperation.EVALUATE, 8),
                pack_lut_eval_beat(padded(addresses[timestep, start:stop], 128)),
                output_beats=2,
                timeout_seconds=timeout_seconds,
            )
            derivative = np.asarray(unpack_lut_result(lut_words), dtype=np.int64)
            spike_path = _execute_vector8(
                driver,
                VectorOperation.MULTIPLY_Q15,
                padded(grad_spike[timestep, start:stop]),
                derivative,
                timeout_seconds=timeout_seconds,
            )
            grad_h = _execute_vector8(
                driver,
                VectorOperation.ADD_SAT,
                membrane_path,
                spike_path,
                timeout_seconds=timeout_seconds,
            )
```

```
            input_chunk = _execute_vector8(
                driver,
                VectorOperation.MULTIPLY_Q15,
                grad_h,
                (int(reciprocal_tau_q15),) * 8,
                timeout_seconds=timeout_seconds,
            )
            grad_h_trace[timestep, start:stop] = grad_h[:active]
            grad_input[timestep, start:stop] = input_chunk[:active]
        grad_h_next = grad_h_trace[timestep].copy()
    return grad_input, grad_h_trace


def _pair(name: str, value) -> tuple[int, int]:
    if isinstance(value, int):
        result = (int(value), int(value))
    else:
        result = tuple(map(int, value))
        if len(result) != 2:
            raise ValueError(f"{name} must be an integer or a pair")
    if result[0] <= 0 or result[1] <= 0:
        raise ValueError(f"{name} values must be positive")
    return result


def maxpool2d_backward_fixed(
    values,
    output_gradients,
    *,
    kernel_size=3,
    stride=2,
    padding=1,
    dilation=1,
) -> tuple[np.ndarray, np.ndarray]:
    """Fixed-point MaxPool2d backward with PyTorch's first-maximum tie rule."""

    source = np.asarray(values, dtype=np.int64)
    gradient = np.asarray(output_gradients, dtype=np.int64)
    if source.ndim != 4 or gradient.ndim != 4:
        raise ValueError("max-pool values and gradients must be NCHW tensors")
    kernel_h, kernel_w = _pair("kernel_size", kernel_size)
    stride_h, stride_w = _pair("stride", stride)
    dilation_h, dilation_w = _pair("dilation", dilation)
    if isinstance(padding, int):
        pad_h, pad_w = int(padding), int(padding)
    else:
        pad_h, pad_w = tuple(map(int, padding))
    if pad_h < 0 or pad_w < 0:
        raise ValueError("padding must be nonnegative")
    batch, channels, height, width = source.shape
    output_height = (
        height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1
    ) // stride_h + 1
    output_width = (
        width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1
    ) // stride_w + 1
    if gradient.shape != (batch, channels, output_height, output_width):
        raise ValueError("max-pool output gradient shape does not match its geometry")
    input_gradient = np.zeros(source.shape, dtype=np.int64)
    winners = np.zeros(gradient.shape, dtype=np.int64)
    for batch_index in range(batch):
        for channel in range(channels):
            for output_y in range(output_height):
                for output_x in range(output_width):
                    winner_value: int | None = None
                    winner_index = -1
                    for kernel_y in range(kernel_h):
                        input_y = output_y * stride_h - pad_h + kernel_y * dilation_h
                        if not 0 <= input_y < height:
                            continue
                        for kernel_x in range(kernel_w):
                            input_x = output_x * stride_w - pad_w + kernel_x * dilation_w
                            if not 0 <= input_x < width:
                                continue
                            candidate = int(source[batch_index, channel, input_y, input_x])
                            if winner_value is None or candidate > winner_value:
                                winner_value = candidate
                                winner_index = kernel_y * kernel_w + kernel_x
                    if winner_index < 0:
                        raise ValueError("a max-pool window contains no source elements")
                    winners[batch_index, channel, output_y, output_x] = winner_index
                    input_y = (
                        output_y * stride_h - pad_h
                        + (winner_index // kernel_w) * dilation_h
                    )
                    input_x = (
                        output_x * stride_w - pad_w
```

```
                        + (winner_index % kernel_w) * dilation_w
                    )
                    prior = input_gradient[batch_index, channel, input_y, input_x]
                    update = gradient[batch_index, channel, output_y, output_x]
                    input_gradient[batch_index, channel, input_y, input_x] = saturate_signed(
                        np.asarray(prior + update, dtype=np.int64), 32
                    )
    return input_gradient, winners


def execute_maxpool2d_backward(
    driver: TrainingCommandExecutor,
    values,
    output_gradients,
    *,
    kernel_size=3,
    stride=2,
    padding=1,
    dilation=1,
    timeout_seconds: float = 5.0,
) -> tuple[np.ndarray, np.ndarray]:
    """Run max-pool winner selection and gradient scatter through vector RTL."""

    source = np.asarray(values, dtype=np.int64)
    gradient = np.asarray(output_gradients, dtype=np.int64)
    if source.ndim != 4 or gradient.ndim != 4:
        raise ValueError("max-pool values and gradients must be NCHW tensors")
    kernel_h, kernel_w = _pair("kernel_size", kernel_size)
    stride_h, stride_w = _pair("stride", stride)
    dilation_h, dilation_w = _pair("dilation", dilation)
    if isinstance(padding, int):
        pad_h, pad_w = int(padding), int(padding)
    else:
        pad_h, pad_w = tuple(map(int, padding))
    batch, channels, height, width = source.shape
    output_height = (
        height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1
    ) // stride_h + 1
    output_width = (
        width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1
    ) // stride_w + 1
    if gradient.shape != (batch, channels, output_height, output_width):
        raise ValueError("max-pool output gradient shape does not match its geometry")
    windows = tuple(
        (batch_index, channel, output_y, output_x)
        for batch_index in range(batch)
        for channel in range(channels)
        for output_y in range(output_height)
        for output_x in range(output_width)
    )
    input_gradient = np.zeros(source.shape, dtype=np.int64)
    winners = np.zeros(gradient.shape, dtype=np.int64)

    for window_start in range(0, len(windows), 8):
        active_windows = windows[window_start:window_start + 8]
        active = len(active_windows)
        first_indices: list[int] = []
        current_values: list[int] = []
        for batch_index, channel, output_y, output_x in active_windows:
            found = False
            for kernel_y in range(kernel_h):
                input_y = output_y * stride_h - pad_h + kernel_y * dilation_h
                for kernel_x in range(kernel_w):
                    input_x = output_x * stride_w - pad_w + kernel_x * dilation_w
                    if 0 <= input_y < height and 0 <= input_x < width:
                        first_indices.append(kernel_y * kernel_w + kernel_x)
                        current_values.append(int(source[batch_index, channel, input_y, input_x]))
                        found = True
                        break
                if found:
                    break
            if not found:
                raise ValueError("a max-pool window contains no source elements")
        current = np.asarray(current_values + [0] * (8 - active), dtype=np.int64)
        winner = np.asarray(first_indices + [0] * (8 - active), dtype=np.int64)
        for candidate_index in range(kernel_h * kernel_w):
            candidates = current.copy()
            for lane, (batch_index, channel, output_y, output_x) in enumerate(active_windows):
                kernel_y, kernel_x = divmod(candidate_index, kernel_w)
                input_y = output_y * stride_h - pad_h + kernel_y * dilation_h
                input_x = output_x * stride_w - pad_w + kernel_x * dilation_w
                if (candidate_index > first_indices[lane]
                        and 0 <= input_y < height and 0 <= input_x < width):
                    candidates[lane] = source[batch_index, channel, input_y, input_x]
            keep_mask = _execute_vector8(
                driver,
                VectorOperation.COMPARE_GE,
```

```
                current,
                candidates,
                timeout_seconds=timeout_seconds,
            )
            current = _execute_vector8(
                driver,
                VectorOperation.MAXIMUM,
                current,
                candidates,
                timeout_seconds=timeout_seconds,
            )
            kept_winner = _execute_vector8(
                driver,
                VectorOperation.SELECT_NONZERO,
                keep_mask,
                winner,
                timeout_seconds=timeout_seconds,
            )
            replacement = _execute_vector8(
                driver,
                VectorOperation.SELECT_ZERO,
                keep_mask,
                (candidate_index,) * 8,
                timeout_seconds=timeout_seconds,
            )
            winner = _execute_vector8(
                driver,
                VectorOperation.ADD_SAT,
                kept_winner,
                replacement,
                timeout_seconds=timeout_seconds,
            )

        updates: list[tuple[tuple[int, int, int, int], int]] = []
        for lane, window in enumerate(active_windows):
            batch_index, channel, output_y, output_x = window
            winner_index = int(winner[lane])
            winners[window] = winner_index
            input_y = (
                output_y * stride_h - pad_h
                + (winner_index // kernel_w) * dilation_h
            )
            input_x = (
                output_x * stride_w - pad_w
                + (winner_index % kernel_w) * dilation_w
            )
            updates.append(
                ((batch_index, channel, input_y, input_x), int(gradient[window]))
            )

        # Each vector command updates distinct addresses. Repeated winners are
        # deferred to a later round, preserving output-window order per address.
        pending = list(updates)
        while pending:
            selected: list[tuple[tuple[int, int, int, int], int]] = []
            deferred: list[tuple[tuple[int, int, int, int], int]] = []
            seen: set[tuple[int, int, int, int]] = set()
            for update in pending:
                if update[0] in seen or len(selected) == 8:
                    deferred.append(update)
                else:
                    selected.append(update)
                    seen.add(update[0])
            left = [int(input_gradient[address]) for address, _ in selected]
            right = [value for _, value in selected]
            left.extend([0] * (8 - len(selected)))
            right.extend([0] * (8 - len(selected)))
            summed = _execute_vector8(
                driver,
                VectorOperation.ADD_SAT,
                left,
                right,
                timeout_seconds=timeout_seconds,
            )
            for lane, (address, _) in enumerate(selected):
                input_gradient[address] = summed[lane]
            pending = deferred
    return input_gradient, winners


def _shape_elements(shape: Sequence[int]) -> int:
    result = 1
    for dimension in shape:
        result *= int(dimension)
    return result


```

```
def linear_backward_partial_tiles(
    rows: int,
    inputs: int,
    outputs: int,
    *,
    repetitions: int = 1,
) -> int:
    """Count dX plus dW partial commands without materializing operands."""

    if min(rows, inputs, outputs, repetitions) <= 0:
        raise ValueError("linear backward dimensions and repetitions must be positive")
    return (
        2
        * ceil(rows / TILE)
        * ceil(inputs / TILE)
        * ceil(outputs / TILE)
        * repetitions
    )


def build_spikingformer_training_coverage(
    execution_manifest_path: str | Path,
    parameter_manifest_path: str | Path,
) -> dict[str, object]:
    """Prove hardware backward/update coverage for the exported deployed graph."""

    execution_path = Path(execution_manifest_path).expanduser().resolve()
    parameter_path = Path(parameter_manifest_path).expanduser().resolve()
    execution = json.loads(execution_path.read_text(encoding="ascii"))
    parameters = json.loads(parameter_path.read_text(encoding="ascii"))
    operators = {item["name"]: item for item in execution["operators"]}
    projection_entries = {item["name"]: item for item in parameters["projections"]}
    learned_names = tuple(projection_entries) + ("head",)
    missing_operators = sorted(set(learned_names) - set(operators))
    if missing_operators:
        raise ValueError(f"learned parameter groups lack graph operators: {missing_operators}")

    learned_coverage: list[dict[str, object]] = []
    learned_parameter_elements = 0
    learned_backward_tiles = 0
    learned_backward_multiplications = 0
    for name in learned_names:
        operator = operators[name]
        matrix = operator["matrix"]
        rows = int(matrix["rows"])
        inputs = int(matrix["reduction"])
        outputs = int(matrix["columns"])
        repetitions = int(matrix.get("repetitions", 1))
        augmented_inputs = inputs + 1
        partial_tiles = linear_backward_partial_tiles(
            rows,
            augmented_inputs,
            outputs,
            repetitions=repetitions,
        )
        multiplications = 2 * rows * augmented_inputs * outputs * repetitions
        if name == "head":
            entry = parameters["head"]
            weight_elements = _shape_elements(entry["hardware_matrix_shape"])
            bias_elements = int(entry["hardware_matrix_shape"][1])
        else:
            entry = projection_entries[name]
            weight_elements = _shape_elements(entry["hardware_matrix_shape"])
            bias_elements = int(entry["hardware_matrix_shape"][1])
        parameter_elements = weight_elements + bias_elements
        learned_parameter_elements += parameter_elements
        learned_backward_tiles += partial_tiles
        learned_backward_multiplications += multiplications
        learned_coverage.append({
            "name": name,
            "rows": rows,
            "inputs": inputs,
            "outputs": outputs,
            "repetitions": repetitions,
            "weight_elements": weight_elements,
            "bias_elements": bias_elements,
            "trainable_deployed_elements": parameter_elements,
            "backward_partial_tiles": partial_tiles,
            "backward_multiplications": multiplications,
            "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row",
            "update_method": "eight-lane signed fixed-point SGDW in programmable logic",
        })

    dynamic_names = tuple(
        name for name in operators if name.endswith(".attn.qk") or name.endswith(".attn.av")
    )
    dynamic_coverage: list[dict[str, object]] = []
```

```
    dynamic_tiles = 0
    dynamic_multiplications = 0
    for name in dynamic_names:
        matrix = operators[name]["matrix"]
        rows = int(matrix["rows"])
        inputs = int(matrix["reduction"])
        outputs = int(matrix["columns"])
        repetitions = int(matrix.get("repetitions", 1))
        partial_tiles = linear_backward_partial_tiles(
            rows, inputs, outputs, repetitions=repetitions
        )
        multiplications = 2 * rows * inputs * outputs * repetitions
        dynamic_tiles += partial_tiles
        dynamic_multiplications += multiplications
        dynamic_coverage.append({
            "name": name,
            "rows": rows,
            "inputs": inputs,
            "outputs": outputs,
            "repetitions": repetitions,
            "backward_partial_tiles": partial_tiles,
            "backward_multiplications": multiplications,
            "gradient_method": "two signed raw-accumulator matrix passes",
        })

    operator_coverage: list[dict[str, str]] = []
    uncovered: list[str] = []
    for name, operator in operators.items():
        source = str(operator["source_operation"])
        if name in learned_names:
            method = "affine dX/dW/dB plus SGDW"
            if "MaxPool2d" in source:
                method += "; first-winner max-pool backward and fabric scatter-add"
            if "LIF" in source:
                method += "; traced sigmoid-surrogate hard-LIF BPTT"
        elif name in dynamic_names:
            method = "dynamic matrix dX/dW; 0.125 attention gradient uses Q15 multiply"
        elif name.endswith(".residuals"):
            method = "identity gradient fan-out; converging branches use saturating vector add"
        elif name == "global_average_pool":
            method = "gradient broadcast with exact power-of-two division by 256"
        else:
            uncovered.append(name)
            method = "UNMAPPED"
        operator_coverage.append({"name": name, "backward_method": method})
    if uncovered:
        raise ValueError(f"execution operators lack backward methods: {uncovered}")

    depth = int(execution["model"]["transformer_blocks"])
    lif_nodes = 4 + 7 * depth
    optimizer_commands = ceil(learned_parameter_elements / 8)
    return {
        "format": "neuraedge-spikingformer-training-coverage-v1",
        "execution_manifest": str(execution_path),
        "parameter_manifest": str(parameter_path),
        "training_semantics": {
            "target_graph": "complete exported integer inference graph",
            "quantizer_gradient": "straight-through estimator with fixed exported scales",
            "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True",
            "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces",
            "optimizer": (
                "signed fixed-point SGDW with momentum; weight decay is applied "
                "to the update direction and excluded from retained velocity"
            ),
            "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic",
        },
        "summary": {
            "execution_operator_count": len(operators),
            "covered_execution_operator_count": len(operator_coverage),
            "learned_affine_group_count": len(learned_names),
            "backbone_projection_count": len(projection_entries),
            "dynamic_attention_matmul_count": len(dynamic_names),
            "lif_node_count": lif_nodes,
            "maxpool_backward_count": 2,
            "residual_gradient_fanout_count": 2 * depth,
            "trainable_deployed_parameter_elements": learned_parameter_elements,
            "sgdw_update_commands": optimizer_commands,
            "learned_backward_partial_tiles": learned_backward_tiles,
            "dynamic_backward_partial_tiles": dynamic_tiles,
            "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles,
            "learned_backward_multiplications": learned_backward_multiplications,
            "dynamic_backward_multiplications": dynamic_multiplications,
            "full_deployed_backbone_covered": True,
            "uncovered_operator_count": 0,
        },
        "learned_affine_coverage": learned_coverage,
        "dynamic_attention_coverage": dynamic_coverage,
```

```
        "operator_coverage": operator_coverage,
    }
```

38.3 Line annotations

Line 1: `"""Exact tiled backward compiler for the NeuraEdge training datapath.` This is executable documentation attached to the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `For "Y = XW" the accelerator evaluates both matrix derivatives:` Inside the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are For: the value named 'for' in this block; Y: the value named 'y' in this block; XW: the value named 'xw' in this block.

Line 4: `"dX = dY W^T" and "dW = X^T dY". Hardware retains 47-bit unrounded` Inside the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are dX: the value named 'd x' in this block; dY: the value named 'd y' in this block; W: the value named 'w' in this block.

Line 5: `partial sums between commands, so this compiler may tile every dimension` Inside the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are partial: the value named 'partial' in this block; sums: the value named 'sums' in this block; between: the value named 'between' in this block.

Line 6: `without changing the integer equation.` Inside the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are without: the value named 'without' in this block; changing: the value named 'changing' in this block; the: the value named 'the' in this block.

Line 7: `"""` This is executable documentation attached to the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 9: `from __future__ import annotations` This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `import json` This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `from math import ceil` This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `from pathlib import Path` This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `from typing import Protocol, Sequence` This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `import numpy as np` This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `from neuraedge_protocol import (` This statement continues through source line 37; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `LinearGradientMode,` This is continuation line 2 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'LinearGradientMode,'. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `LutSuboperation,` This is continuation line 3 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'LutSuboperation,'. The important operands are LutSuboperation: the value named 'lut suboperation' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `ProgramRecord,` This is continuation line 4 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'ProgramRecord,'. The important operands are ProgramRecord: the value named 'program record' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `StreamBeat,` This is continuation line 5 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'StreamBeat,'. The important operands are StreamBeat: the value named 'stream beat' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `linear_backward_command,` This is continuation line 6 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'linear_backward_command,'. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 25: `lut_command,` This is continuation line 7 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'lut_command,'. The important operands are lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 26: `pack_linear_backward_beats,` This is continuation line 8 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'pack_linear_backward_beats,'. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 27: `pack_lut_eval_beat,` This is continuation line 9 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'pack_lut_eval_beat,'. The important operands are pack_lut_eval_beat: the value named 'pack lut eval beat' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 28: `pack_lut_load_beats,` This is continuation line 10 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'pack_lut_load_beats,'. The important operands are pack_lut_load_beats: the value named 'pack lut load beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 29: `pack_optimizer_beats,` This is continuation line 11 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'pack_optimizer_beats,'. The important operands are pack_optimizer_beats: the value named 'pack optimizer beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 30: `pack_vector_beats,` This is continuation line 12 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'pack_vector_beats,'. The important operands are pack_vector_beats: the value named 'pack vector beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 31: `sgd_command,` This is continuation line 13 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'sgd_command,'. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 32: `unpack_linear_backward_result,` This is continuation line 14 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'unpack_linear_backward_result,'. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 33: `unpack_lut_result,` This is continuation line 15 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'unpack_lut_result,'. The important operands are unpack_lut_result: the value named 'unpack lut result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 34: `unpack_optimizer_result,` This is continuation line 16 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'unpack_optimizer_result,'. The important operands are unpack_optimizer_result: the

value named 'unpack optimizer result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 35: `unpack_vector_result,` This is continuation line 17 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'unpack_vector_result,'. The important operands are unpack_vector_result: the value named 'unpack vector result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 36: `vector_command,` This is continuation line 18 of the 19-line logical statement that begins at source line 19. Its exact fragment is 'vector_command,'. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 37: `)` This is continuation line 19 of the 19-line logical statement that begins at source line 19. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 38: `from neuraedge_golden import VectorOperation, round_shift_signed, saturate_signed` This imports the named Python API so the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 41: `TILE = 8` This assigns '8' to 'TILE ' in host memory. Here TILE means the value named 'tile' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 44: `class TrainingCommandExecutor(Protocol):` This defines class TrainingCommandExecutor, which groups data and methods for training command executor. Instances exist in host memory and organize the flow that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 45: `def execute(` This starts function execute. Its indented body performs execute as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 46: `self,` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 47: `command,` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 48: `input_beats: Sequence[StreamBeat] = (),` This assigns '(),' to 'input_beats' in host memory. Here input_beats means the value named 'input beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 49: `*,` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 50: `output_beats: int = 0,` This assigns ‘0,‘ to ‘output_beats‘ in host memory. Here output_beats means the value named ’output beats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 51: `timeout_seconds: float = 5.0,` This assigns ‘5.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 52: `) -> tuple[int, ...]: ...` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named ’tuple’ in this block.

Line 55: `class TrainingProgramExecutor(TrainingCommandExecutor, Protocol):` This defines class TrainingProgramExecutor, which groups data and methods for training program executor. Instances exist in host memory and organize the flow that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 56: `def execute_program(` This starts function execute_program. Its indented body performs execute program as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 57: `self,` Inside the function execute_program, which handles execute program, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 58: `records: Sequence[ProgramRecord],` Inside the function execute_program, which handles execute program, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are records: the number of descriptor records accepted since program start; Sequence: the value named ’sequence’ in this block; ProgramRecord: the value named ’program record’ in this block.

Line 59: `*,` Inside the function execute_program, which handles execute program, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 60: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 61: `) -> tuple[int, ...]: ...` Inside the function execute_program, which handles execute program, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named ’tuple’ in this block.

Line 64: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it creates no FPGA circuit by itself.

Line 65: `class BackwardInstruction:` This defines class BackwardInstruction, which groups data and methods for backward instruction. Instances exist in host memory and organize the flow that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 66: `"""One partial tile and the global matrix region to which it belongs."""` This is executable documentation attached to the class BackwardInstruction, which handles backward instruction. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 68: `record: ProgramRecord` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block.

Line 69: `mode: LinearGradientMode` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter.

Line 70: `row_start: int` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are row_start: one output-row position processed by the tensor tile.

Line 71: `input_start: int` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_start: the value named 'input start' in this block.

Line 72: `output_start: int` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_start: the value named 'output start' in this block.

Line 73: `rows: int` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are rows: one output-row position processed by the tensor tile.

Line 74: `inputs: int` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are inputs: the value named 'inputs' in this block.

Line 75: `outputs: int` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are outputs: the value named 'outputs' in this block.

Line 76: `accumulate: bool` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are accumulate: the value named 'accumulate' in this block; bool: the value named 'bool' in this block.

Line 77: `suppress_output: bool` Inside the class BackwardInstruction, which handles backward instruction, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are suppress_output: the value named 'suppress output' in this block; bool: the value named 'bool' in this block.

Line 80: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it creates no FPGA circuit by itself.

Line 81: `class CompiledLinearBackward:` This defines class CompiledLinearBackward, which groups data and methods for compiled linear backward. Instances exist in host memory and organize the flow that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 82: `"""Executable descriptor program for one arbitrary signed linear layer."""` This is executable documentation attached to the class CompiledLinearBackward, which handles compiled linear backward. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 84: `shape: tuple[int, int, int]` Inside the class CompiledLinearBackward, which handles compiled linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are shape: the value named 'shape' in this block; tuple: the value named 'tuple' in this block.

Line 85: `input_gradient_shift: int` Inside the class CompiledLinearBackward, which handles compiled linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter.

Line 86: `weight_gradient_shift: int` Inside the class CompiledLinearBackward, which handles compiled linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation.

Line 87: `instructions: tuple[BackwardInstruction, ...]` Inside the class CompiledLinearBackward, which handles compiled linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are instructions: the value named 'instructions' in this block; tuple: the value named 'tuple' in this block; BackwardInstruction: the value named 'backward instruction' in this block.

Line 88: `estimated_input_bytes: int` Inside the class CompiledLinearBackward, which handles compiled linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are estimated_input_bytes: the value named 'estimated input bytes' in this block.

Line 89: `mathematical_multiplications: int` Inside the class CompiledLinearBackward, which handles compiled linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are mathematical_multiplications: the value named 'mathematical multiplications' in this block.

Line 91: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class CompiledLinearBackward, which handles compiled linear backward; it creates no FPGA circuit by itself.

Line 92: `def records(self) -> tuple[ProgramRecord, ...]:` This starts function records. Its indented body performs records as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 93: `return tuple(instruction.record for instruction in self.instructions)` This returns ‘tuple(instruction.recor for instruction in self.instructions)‘ to the caller of the function records, which handles records, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; instruction: the value named ’instruction’ in this block; record: the value named ’record’ in this block.

Line 95: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class CompiledLinearBackward, which handles compiled linear backward; it creates no FPGA circuit by itself.

Line 96: `def output_beats(self) -> int:` This starts function output_beats. Its indented body performs output beats as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 97: `return sum(instruction.record.output_beats for instruction in self.instructions)` This returns ‘sum(instruction.record.output_beats for instruction in self.instructions)‘ to the caller of the function output_beats, which handles output beats, ending this invocation. The important operands are sum: the value named ’sum’ in this block; instruction: the value named ’instruction’ in this block; record: the value named ’record’ in this block.

Line 99: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class CompiledLinearBackward, which handles compiled linear backward; it creates no FPGA circuit by itself.

Line 100: `def partial_tile_count(self) -> int:` This starts function partial_tile_count. Its indented body performs partial tile count as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 101: `return len(self.instructions)` This returns ‘len(self.instructions)‘ to the caller of the function partial_tile_count, which handles partial tile count, ending this invocation. The important operands are len: the value named ’len’ in this block; instructions: the value named ’instructions’ in this block.

Line 103: `def _decode_outputs(self, instruction_words: Sequence[Sequence[int]]) -> tuple[np.ndarray, np.ndarray]:` This starts function _decode_outputs. Its indented body performs decode outputs as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 104: `rows, inputs, outputs = self.shape` This assigns ‘self.shape‘ to ‘rows, inputs, outputs ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are shape: the value named ’shape’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 105: `input_gradient = np.zeros((rows, inputs), dtype=np.int64)` This assigns ‘np.zeros((rows, inputs), dtype=np.int64)‘ to ‘input_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are zeros: the value named ’zeros’ in this block; rows: one output-row position processed by the tensor tile; inputs: the value named ’inputs’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 106: `weight_gradient = np.zeros((inputs, outputs), dtype=np.int64)` This assigns ‘np.zeros((inputs, outputs), dtype=np.int64)‘ to ‘weight_gradient ‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are zeros: the value named ’zeros’ in this block; inputs: the value named ’inputs’ in this block; outputs: the value named ’outputs’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 107: `if len(instruction_words) != len(self.instructions):` This executes the following indented host-code branch only when ‘len(instruction_words) != len(self.instructions)‘ is true. The important operands are len: the value named ’len’ in this block; instruction_words: the value named ’instruction words’ in this block; instructions: the value named ’instructions’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 108: `raise RuntimeError("backward result partition does not match the instruction count")` This rejects an invalid state by raising ‘RuntimeError("backward result partition does not match the instruction count")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 109: `for instruction, words in zip(self.instructions, instruction_words):` This iterates instruction, words over ‘zip(self.instructions, instruction_words)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _decode_outputs, which handles decode outputs.

Line 110: `if len(words) != instruction.record.output_beats:` This executes the following indented host-code branch only when ‘len(words) != instruction.record.output_beats‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block; instruction: the value named ’instruction’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 111: `raise RuntimeError(` This statement continues through source line 114; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘RuntimeError(f"backward tile returned {len(words)} beats; " f"expected {instruction.record.output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 112: `f"backward tile returned {len(words)} beats; "` This is continuation line 2 of the 4-line logical statement that begins at source line 111. Its exact fragment is ‘f"backward tile returned {len(words)} beats; "‘. The important operands are f: the value named ’f’ in this block; backward: the value named ’backward’ in this block; tile: the value named ’tile’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"backward tile returned {len(words)} beats; " f"expected {instruction.record.output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 113: `f"expected {instruction.record.output_beats}"` This is continuation line 3 of the 4-line logical statement that begins at source line 111. Its exact fragment is ‘f"expected {instruction.record.output_beats}"‘. The important operands are f: the value named ’f’ in this block; expected: the value named ’expected’ in this block; instruction: the value named ’instruction’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"backward tile returned {len(words)} beats; " f"expected {instruction.record.output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 114: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 111. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"backward tile returned {len(words)} beats; " f"expected {instruction.record.output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 115: `if instruction.suppress_output:` This executes the following indented host-code branch only when ‘instruction.suppress_output‘ is true. The important operands are instruction: the value named ’instruction’ in this block; suppress_output: the value named ’suppress output’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 116: `continue` Inside the function _decode_outputs, which handles decode outputs, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are continue: the value named ’continue’ in this block.

Line 117: `dx_tile, dw_tile = unpack_linear_backward_result(` This statement continues through source line 123; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘unpack_linear_backward_ words, instruction.rows, instruction.inputs, instruction.outputs, mode=instruction.mode,)‘ to ‘dx_tile, dw_tile ‘ in host memory. Here dx_tile means the value named ’dx tile’ in this block. The important operands are unpack_linear_backward_result: the value named ’unpack linear backward result’ in this block; words: the value named ’words’ in this block; instruction: the value named ’instruction’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 118: `words,` This is continuation line 2 of the 7-line logical statement that begins at source line 117. Its exact fragment is 'words,'. The important operands are words: the value named 'words' in this block. Read together, the complete operation is: This assigns 'unpack_linear_backward_result(words, instruction.rows, instruction.inputs, instruction.outputs, mode=instruction.mode,)' to 'dx_tile, dw_tile ' in host memory. Here dx_tile means the value named 'dx tile' in this block. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block; words: the value named 'words' in this block; instruction: the value named 'instruction' in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 119: `instruction.rows,` This is continuation line 3 of the 7-line logical statement that begins at source line 117. Its exact fragment is 'instruction.rows,'. The important operands are instruction: the value named 'instruction' in this block; rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This assigns 'unpack_linear_backward_result(words, instruction.rows, instruction.inputs, instruction.outputs, mode=instruction.mode,)' to 'dx_tile, dw_tile ' in host memory. Here dx_tile means the value named 'dx tile' in this block. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block; words: the value named 'words' in this block; instruction: the value named 'instruction' in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 120: `instruction.inputs,` This is continuation line 4 of the 7-line logical statement that begins at source line 117. Its exact fragment is 'instruction.inputs,'. The important operands are instruction: the value named 'instruction' in this block; inputs: the value named 'inputs' in this block. Read together, the complete operation is: This assigns 'unpack_linear_backward_result(words, instruction.rows, instruction.inputs, instruction.outputs, mode=instruction.mode,)' to 'dx_tile, dw_tile ' in host memory. Here dx_tile means the value named 'dx tile' in this block. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block; words: the value named 'words' in this block; instruction: the value named 'instruction' in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 121: `instruction.outputs,` This is continuation line 5 of the 7-line logical statement that begins at source line 117. Its exact fragment is 'instruction.outputs,'. The important operands are instruction: the value named 'instruction' in this block; outputs: the value named 'outputs' in this block. Read together, the complete operation is: This assigns 'unpack_linear_backward_result(words, instruction.rows, instruction.inputs, instruction.outputs, mode=instruction.mode,)' to 'dx_tile, dw_tile ' in host memory. Here dx_tile means the value named 'dx tile' in this block. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block; words: the value named 'words' in this block; instruction: the value named 'instruction' in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 122: `mode=instruction.mode,` This is continuation line 6 of the 7-line logical statement that begins at source line 117. Its exact fragment is 'mode=instruction.mode,'. The important operands are mode: the value named 'mode' in this block; instruction: the value named 'instruction' in this block. Read together, the complete operation is: This assigns 'unpack_linear_backward_result(words, instruction.rows, instruction.inputs, instruction.outputs, mode=instruction.mode,)' to 'dx_tile, dw_tile ' in host memory. Here dx_tile means the value named 'dx tile' in this block. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block; words: the value named 'words' in this block; instruction: the value named 'instruction' in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 123: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 117. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'unpack_linear_backward_result(words, instruction.rows, instruction.inputs, instruction.outputs, mode=instruction.mode,)' to 'dx_tile, dw_tile ' in host memory. Here dx_tile means the value named 'dx tile' in this block. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block; words: the value named 'words' in this block; instruction: the value named 'instruction' in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 124: `if instruction.mode == LinearGradientMode.INPUT:` This executes the following indented host-code branch only when 'instruction.mode == LinearGradientMode.INPUT' is true. The important operands are instruction: the value named 'instruction' in this block; mode: the value named 'mode' in this block; LinearGradientMode: the

signed loss derivative used to update a learned parameter. Context: the function _decode_outputs, which handles decode outputs.

Line 125: `input_gradient[` This statement continues through source line 128; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.asarray(dx_tile, dtype=np.int64)‘ to ‘input_gradient[instruction.row_start‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are asarray: the value named ’asarray’ in this block; dx_tile: the value named ’dx tile’ in this block; dtype: the value named ’dtype’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 126: `instruction.row_start:instruction.row_start + instruction.rows,` This is continuation line 2 of the 4-line logical statement that begins at source line 125. Its exact fragment is ‘instruction.row_start:instruction.row_start + instruction.rows,‘. The important operands are instruction: the value named ’instruction’ in this block; row_start: one output-row position processed by the tensor tile; rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This assigns ‘np.asarray(dx_tile, dtype=np.int64)‘ to ‘input_gradient[instruction.row_start‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are asarray: the value named ’asarray’ in this block; dx_tile: the value named ’dx tile’ in this block; dtype: the value named ’dtype’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 127: `instruction.input_start:instruction.input_start + instruction.inputs,` This is continuation line 3 of the 4-line logical statement that begins at source line 125. Its exact fragment is ‘instruction.input_start:instruction.input_start + instruction.inputs,‘. The important operands are instruction: the value named ’instruction’ in this block; input_start: the value named ’input start’ in this block; inputs: the value named ’inputs’ in this block. Read together, the complete operation is: This assigns ‘np.asarray(dx_tile, dtype=np.int64)‘ to ‘input_gradient[instruction.row_start‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are asarray: the value named ’asarray’ in this block; dx_tile: the value named ’dx tile’ in this block; dtype: the value named ’dtype’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 128: `] = np.asarray(dx_tile, dtype=np.int64)` This is continuation line 4 of the 4-line logical statement that begins at source line 125. Its exact fragment is ‘] = np.asarray(dx_tile, dtype=np.int64)‘. The important operands are asarray: the value named ’asarray’ in this block; dx_tile: the value named ’dx tile’ in this block; dtype: the value named ’dtype’ in this block. Read together, the complete operation is: This assigns ‘np.asarray(dx_tile, dtype=np.int64)‘ to ‘input_gradient[instruction.row_start‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are asarray: the value named ’asarray’ in this block; dx_tile: the value named ’dx tile’ in this block; dtype: the value named ’dtype’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 129: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function _decode_outputs, which handles decode outputs.

Line 130: `weight_gradient[` This statement continues through source line 133; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.asarray(dw_tile, dtype=np.int64)‘ to ‘weight_gradient[instruction.input_start‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; dw_tile: the value named ’dw tile’ in this block; dtype: the value named ’dtype’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 131: `instruction.input_start:instruction.input_start + instruction.inputs,` This is continuation line 2 of the 4-line logical statement that begins at source line 130. Its exact fragment is ‘instruction.input_start:instruction.input_start + instruction.inputs,‘. The important operands are instruction: the value named ’instruction’ in this block; input_start: the value named ’input start’ in this block; inputs: the value named ’inputs’ in this block. Read together, the complete operation is: This assigns ‘np.asarray(dw_tile, dtype=np.int64)‘ to ‘weight_gradient[instruction.input_start‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; dw_tile: the value named ’dw tile’ in this block; dtype: the value named ’dtype’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 132: `instruction.output_start:instruction.output_start + instruction.outputs,` This is continuation line 3 of the 4-line logical statement that begins at source line 130. Its exact fragment is ‘instruction.output_start:instruction.output_ + instruction.outputs,‘. The important operands are instruction: the value named ’instruction’ in this block; output_start: the value named ’output start’ in this block; outputs: the value named ’outputs’ in this block. Read together, the complete operation is: This assigns ‘np.asarray(dw_tile, dtype=np.int64)‘ to ‘weight_gradient[instruction.input_start‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; dw_tile: the value named ’dw tile’ in this block; dtype: the value named ’dtype’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 133: `] = np.asarray(dw_tile, dtype=np.int64)` This is continuation line 4 of the 4-line logical statement that begins at source line 130. Its exact fragment is ‘] = np.asarray(dw_tile, dtype=np.int64)‘. The important operands are asarray: the value named ’asarray’ in this block; dw_tile: the value named ’dw tile’ in this block; dtype: the value named ’dtype’ in this block. Read together, the complete operation is: This assigns ‘np.asarray(dw_tile, dtype=np.int64)‘ to ‘weight_gradient[instruction.input_start‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; dw_tile: the value named ’dw tile’ in this block; dtype: the value named ’dtype’ in this block. Context: the function _decode_outputs, which handles decode outputs.

Line 134: `return input_gradient, weight_gradient` This returns ‘input_gradient, weight_gradient‘ to the caller of the function _decode_outputs, which handles decode outputs, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; weight_gradient: a learned coefficient multiplied by an activation.

Line 136: `def execute(` This starts function execute. Its indented body performs execute as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 137: `self,` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 138: `driver: TrainingCommandExecutor,` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named ’driver’ in this block; TrainingCommandExecutor: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 139: `*,` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 140: `timeout_seconds: float = 5.0,` This assigns ‘5.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 141: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named ’tuple’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 142: `partitions: list[tuple[int, ...]] = []` This assigns ‘[]‘ to ‘partitions‘ in host memory. Here partitions means the value named ’partitions’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 143: `for instruction in self.instructions:` This iterates instruction over ‘self.instructions‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 144: `partitions.append(driver.execute(` This statement continues through source line 149; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘partitions.append‘ with ‘driver.execute(instruction.record.command, instruction.record.input_beats, output_beats=instruction.record.output_beats, timeout_seconds=ti)‘ inside the function execute, which handles execute. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 145: `instruction.record.command,` This is continuation line 2 of the 6-line logical statement that begins at source line 144. Its exact fragment is ‘instruction.record.command,‘. The important operands are instruction: the value named ’instruction’ in this block; record: the value named ’record’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘partitions.append‘ with ‘driver.execute(instruction.record.command, instruction.record.input_beats, output_beats=instruction.record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute, which handles execute. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 146: `instruction.record.input_beats,` This is continuation line 3 of the 6-line logical statement that begins at source line 144. Its exact fragment is ‘instruction.record.input_beats,‘. The important operands are instruction: the value named ’instruction’ in this block; record: the value named ’record’ in this block; input_beats: the value named ’input beats’ in this block. Read together, the complete operation is: This calls ‘partitions.append‘ with ‘driver.execute(instruction.record.command, instruction.record.input_beats, output_beats=instruction.record.output_beats, timeout_seconds=timeout_)‘ inside the function execute, which handles execute. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 147: `output_beats=instruction.record.output_beats,` This is continuation line 4 of the 6-line logical statement that begins at source line 144. Its exact fragment is ‘output_beats=instruction.record.output_beats,‘. The important operands are output_beats: the value named ’output beats’ in this block; instruction: the value named ’instruction’ in this block; record: the value named ’record’ in this block. Read together, the complete operation is: This calls ‘partitions.append‘ with ‘driver.execute(instruction.record.command, instruction.record.input_beats, output_beats=instruction.record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute, which handles execute. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 148: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 144. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This calls ‘partitions.append‘ with ‘driver.execute(instruction.record.command, instruction.record.input_beats, output_beats=instruction.record.o timeout_seconds=timeout_seconds,)‘ inside the function execute, which handles execute. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 149: `))` This is continuation line 6 of the 6-line logical statement that begins at source line 144. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘partitions.append‘ with ‘driver.execute(instruction.record.command, instruction.record.input_beats, output_beats=instruction.record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute, which handles execute. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 150: `return self._decode_outputs(partitions)` This returns ‘self._decode_outputs(partitions)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are _decode_outputs: the value named ’decode outputs’ in this block; partitions: the value named ’partitions’ in this block.

Line 152: `def execute_batched(` This starts function execute_batched. Its indented body performs execute batched as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 153: `self,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 154: `driver: TrainingProgramExecutor,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named 'driver' in this block; TrainingProgramExecutor: the value named 'training program executor' in this block.

Line 155: `*,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 156: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 157: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 158: `words = driver.execute_program(self.records, timeout_seconds=timeout_seconds)` This assigns 'driver.execute_program(self.records, timeout_seconds=timeout_seconds)' to 'words ' in host memory. Here words means the value named 'words' in this block. The important operands are driver: the value named 'driver' in this block; execute_program: the value named 'execute program' in this block; records: the number of descriptor records accepted since program start. Context: the function execute_batched, which handles execute batched.

Line 159: `if len(words) != self.output_beats:` This executes the following indented host-code branch only when 'len(words) != self.output_beats' is true. The important operands are len: the value named 'len' in this block; words: the value named 'words' in this block; output_beats: the value named 'output beats' in this block. Context: the function execute_batched, which handles execute batched.

Line 160: `raise RuntimeError(` This statement continues through source line 162; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising 'RuntimeError(f"backward program returned {len(words)} beats; expected {self.output_beats}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 161: `f"backward program returned {len(words)} beats; expected {self.output_beats}"` This is continuation line 2 of the 3-line logical statement that begins at source line 160. Its exact fragment is 'f"backward program returned {len(words)} beats; expected {self.output_beats}"'. The important operands are f: the value named 'f' in this block; backward: the value named 'backward' in this block; program: the value named 'program' in this block. Read together, the complete operation is: This rejects an invalid state by raising 'RuntimeError(f"backward program returned {len(words)} beats; expected {self.output_beats}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 162: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 160. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This rejects an invalid state by raising 'RuntimeError(f"backward program returned {len(words)} beats; expected {self.output_beats}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 163: `partitions: list[tuple[int, ...]] = []` This assigns '[]' to 'partitions' in host memory. Here partitions means the value named 'partitions' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 164: `cursor = 0` This assigns '0' to 'cursor ' in host memory. Here cursor means the value named 'cursor' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 165: `for instruction in self.instructions:` This iterates instruction over 'self.instructions'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_batched, which handles execute batched.

Line 166: `end = cursor + instruction.record.output_beats` This assigns 'cursor + instruction.record.output_beats' to 'end ' in host memory. Here end means the value named 'end' in this block. The important operands are cursor: the value named 'cursor' in this block; instruction: the value named 'instruction' in this block; record: the value named 'record' in this block. Context: the function execute_batched, which handles execute batched.

Line 167: `partitions.append(tuple(words[cursor:end]))` This calls 'partitions.append' with 'tuple(words[cursor:end])' inside the function execute_batched, which handles execute batched. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 168: `cursor = end` This assigns 'end' to 'cursor ' in host memory. Here cursor means the value named 'cursor' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 169: `return self._decode_outputs(partitions)` This returns 'self._decode_outputs(partitions)' to the caller of the function execute_batched, which handles execute batched, ending this invocation. The important operands are _decode_outputs: the value named 'decode outputs' in this block; partitions: the value named 'partitions' in this block.

Line 172: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it creates no FPGA circuit by itself.

Line 173: `class CompiledAffineBackward:` This defines class CompiledAffineBackward, which groups data and methods for compiled affine backward. Instances exist in host memory and organize the flow that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 174: `"""Linear backward plan whose final augmented row represents the bias."""` This is executable documentation attached to the class CompiledAffineBackward, which handles compiled affine backward. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 176: `linear: CompiledLinearBackward` Inside the class CompiledAffineBackward, which handles compiled affine backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are linear: the value named 'linear' in this block; CompiledLinearBackward: the value named 'compiled linear backward' in this block.

Line 177: `input_features: int` Inside the class CompiledAffineBackward, which handles compiled affine backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_features: the value named 'input features' in this block.

Line 179: `def split(` This starts function split. Its indented body performs split as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 180: `self,` Inside the function split, which handles split, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 181: `augmented_input_gradient: np.ndarray,` Inside the function split, which handles split, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are augmented_input_gradient: the signed loss derivative used to update a learned parameter; ndarray: the value named 'ndarray' in this block.

Line 182: `augmented_weight_gradient: np.ndarray,` Inside the function split, which handles split, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are augmented_weight_gradient: a learned coefficient multiplied by an activation; ndarray: the value named 'ndarray' in this block.

Line 183: `) -> tuple[np.ndarray, np.ndarray, np.ndarray]:` Inside the function split, which handles split, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 184: `return (` This statement continues through source line 188; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘(augmented_input_gradient[:, :self.input_features], augmented_weight_gradient[:self.input_features, :], augmented_weight_gradient[self.input_features, :],)‘ to the caller of the function split, which handles split, ending this invocation. The important operands are augmented_input_gradient: the signed loss derivative used to update a learned parameter; input_features: the value named 'input features' in this block; augmented_weight_gradient: a learned coefficient multiplied by an activation.

Line 185: `augmented_input_gradient[:, :self.input_features],` This is continuation line 2 of the 5-line logical statement that begins at source line 184. Its exact fragment is ‘augmented_input_gradient[:, :self.input_features],‘. The important operands are augmented_input_gradient: the signed loss derivative used to update a learned parameter; input_features: the value named 'input features' in this block. Read together, the complete operation is: This returns ‘(augmented_input_gradient[:, :self.input_features], augmented_weight_gradient[:self.input_features, :], augmented_weight_gradient[self.input_features, :],)‘ to the caller of the function split, which handles split, ending this invocation. The important operands are augmented_input_gradient: the signed loss derivative used to update a learned parameter; input_features: the value named 'input features' in this block; augmented_weight_gradient: a learned coefficient multiplied by an activation.

Line 186: `augmented_weight_gradient[:self.input_features, :],` This is continuation line 3 of the 5-line logical statement that begins at source line 184. Its exact fragment is ‘augmented_weight_gradient[:self.input_features, :],‘. The important operands are augmented_weight_gradient: a learned coefficient multiplied by an activation; input_features: the value named 'input features' in this block. Read together, the complete operation is: This returns ‘(augmented_input_gradient[:, :self.input_features], augmented_weight_gradient[:self.input_features, :], augmented_weight_gradient[self.input_features, :],)‘ to the caller of the function split, which handles split, ending this invocation. The important operands are augmented_input_gradient: the signed loss derivative used to update a learned parameter; input_features: the value named 'input features' in this block; augmented_weight_gradient: a learned coefficient multiplied by an activation.

Line 187: `augmented_weight_gradient[self.input_features, :],` This is continuation line 4 of the 5-line logical statement that begins at source line 184. Its exact fragment is ‘augmented_weight_gradient[self.input_features, :],‘. The important operands are augmented_weight_gradient: a learned coefficient multiplied by an activation; input_features: the value named 'input features' in this block. Read together, the complete operation is: This returns ‘(augmented_input_gradient[:, :self.input_features], augmented_weight_gradient[:self.input_features, :], augmented_weight_gradient[self.input_features, :],)‘ to the caller of the function split, which handles split, ending this

invocation. The important operands are augmented_input_gradient: the signed loss derivative used to update a learned parameter; input_features: the value named 'input features' in this block; augmented_weight_gradient: a learned coefficient multiplied by an activation.

Line 188: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 184. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘(augmented_input_gradient[:, :self.input_features], augmented_weight_gradient[:self.input_features, :], augmented_weight_gradient[self.input_features, :],)‘ to the caller of the function split, which handles split, ending this invocation. The important operands are augmented_input_gradient: the signed loss derivative used to update a learned parameter; input_features: the value named 'input features' in this block; augmented_weight_gradient: a learned coefficient multiplied by an activation.

Line 190: `def execute(` This starts function execute. Its indented body performs execute as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 191: `self,` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 192: `driver: TrainingCommandExecutor,` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named 'driver' in this block; TrainingCommandExecutor: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 193: `*,` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 194: `timeout_seconds: float = 5.0,` This assigns ‘5.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 195: `) -> tuple[np.ndarray, np.ndarray, np.ndarray]:` Inside the function execute, which handles execute, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 196: `return self.split(*self.linear.execute(driver, timeout_seconds=timeout_seconds))` This returns ‘self.split(*self.linear.execute(driver, timeout_seconds=timeout_seconds))‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are split: the value named 'split' in this block; linear: the value named 'linear' in this block; execute: the value named 'execute' in this block.

Line 198: `def execute_batched(` This starts function execute_batched. Its indented body performs execute batched as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 199: `self,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 200: `driver: TrainingProgramExecutor,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named 'driver' in this block; TrainingProgramExecutor: the value named 'training program executor' in this block.

Line 201: `*,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 202: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 203: `) -> tuple[np.ndarray, np.ndarray, np.ndarray]:` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 204: `return self.split(*self.linear.execute_batched(driver, timeout_seconds=timeout_seconds))` This returns 'self.split(*self.linear.execute_batched(driver, timeout_seconds=timeout_seconds))' to the caller of the function execute_batched, which handles execute batched, ending this invocation. The important operands are split: the value named 'split' in this block; linear: the value named 'linear' in this block; execute_batched: the value named 'execute batched' in this block.

Line 207: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it creates no FPGA circuit by itself.

Line 208: `class ConvBackwardGroup:` This defines class ConvBackwardGroup, which groups data and methods for conv backward group. Instances exist in host memory and organize the flow that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 209: `input_channel_start: int` Inside the class ConvBackwardGroup, which handles conv backward group, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_channel_start: the value named 'input channel start' in this block.

Line 210: `output_channel_start: int` Inside the class ConvBackwardGroup, which handles conv backward group, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_channel_start: the value named 'output channel start' in this block.

Line 211: `input_channels: int` Inside the class ConvBackwardGroup, which handles conv backward group, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_channels: the value named 'input channels' in this block.

Line 212: `output_channels: int` Inside the class ConvBackwardGroup, which handles conv backward group, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_channels: the value named 'output channels' in this block.

Line 213: `affine: CompiledAffineBackward` Inside the class ConvBackwardGroup, which handles conv backward group, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are affine: the value named 'affine' in this block; CompiledAffineBackward: the value named 'compiled affine backward' in this block.

Line 216: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it creates no FPGA circuit by itself.

Line 217: `class CompiledConv2dBackward:` This defines class CompiledConv2dBackward, which groups data and methods for compiled conv2d backward. Instances exist in host memory and organize the flow that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 218: `input_shape: tuple[int, int, int, int]` Inside the class CompiledConv2dBackward, which handles compiled conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_shape: the value named 'input shape' in this block; tuple: the value named 'tuple' in this block.

Line 219: `weight_shape: tuple[int, int, int, int]` Inside the class CompiledConv2dBackward, which handles compiled conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are weight_shape: a learned coefficient multiplied by an activation; tuple: the value named 'tuple' in this block.

Line 220: `output_shape: tuple[int, int, int, int]` Inside the class CompiledConv2dBackward, which handles compiled conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_shape: the value named 'output shape' in this block; tuple: the value named 'tuple' in this block.

Line 221: `kernel: tuple[int, int]` Inside the class CompiledConv2dBackward, which handles compiled conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are kernel: the value named 'kernel' in this block; tuple: the value named 'tuple' in this block.

Line 222: `stride: tuple[int, int]` Inside the class CompiledConv2dBackward, which handles compiled conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are stride: the value named 'stride' in this block; tuple: the value named 'tuple' in this block.

Line 223: `padding: tuple[int, int]` Inside the class CompiledConv2dBackward, which handles compiled conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are padding: the value named 'padding' in this block; tuple: the value named 'tuple' in this block.

Line 224: `dilation: tuple[int, int]` Inside the class CompiledConv2dBackward, which handles compiled conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are dilation: the value named 'dilation' in this block; tuple: the value named 'tuple' in this block.

Line 225: `groups: int` Inside the class CompiledConv2dBackward, which handles compiled conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are groups: the value named 'groups' in this block.

Line 226: `group_programs: tuple[ConvBackwardGroup, ...]` Inside the class CompiledConv2dBackward, which handles compiled conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are group_programs: the value named 'group programs' in this block; tuple: the value named 'tuple' in this block; ConvBackwardGroup: the value named 'conv backward group' in this block.

Line 228: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class CompiledConv2dBackward, which handles compiled conv2d backward; it creates no FPGA circuit by itself.

Line 229: `def mathematical_multiplications(self) -> int:` This starts function mathematical_multiplications. Its indented body performs mathematical multiplications as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 230: `return sum(` This statement continues through source line 233; the explanation uses the complete expression rather than only the first visual fragment. This returns 'sum(group.affine.linear.mathematical_multiplications for group in self.group_programs)' to the caller of the function mathematical_multiplications, which handles mathematical multiplications, ending this invocation. The important operands are sum: the value named 'sum' in this block; group: the value named 'group' in this block; affine: the value named 'affine' in this block.

Line 231: `group.affine.linear.mathematical_multiplications` This is continuation line 2 of the 4-line logical statement that begins at source line 230. Its exact fragment is 'group.affine.linear.mathematical_multiplications'. The important operands are group: the value named 'group' in this block; affine: the value named 'affine' in this block; linear: the value named 'linear' in this block. Read together, the complete operation is: This returns 'sum(group.affine.linear.mathematical_multiplications for group in self.group_programs)' to the caller of the function mathematical_multiplications, which handles mathematical multiplications, ending this invocation. The important operands are sum: the value named 'sum' in this block; group: the value named 'group' in this block; affine: the value named 'affine' in this block.

Line 232: `for group in self.group_programs` This is continuation line 3 of the 4-line logical statement that begins at source line 230. Its exact fragment is 'for group in self.group_programs'. The important operands are for: the value named 'for' in this block; group: the value named 'group' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This returns 'sum(group.affine.linear.mathematical_multiplications for group in self.group_programs)' to the caller of the function mathematical_multiplications, which handles mathematical multiplications, ending this invocation. The important operands are sum: the value named 'sum' in this block; group: the value named 'group' in this block; affine: the value named 'affine' in this block.

Line 233: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 230. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'sum(group.affine.linear.mathematical_multiplications for group in self.group_programs)' to the caller of the function mathematical_multiplications, which handles mathematical multiplications, ending this invocation. The important operands are sum: the value named 'sum' in this block; group: the value named 'group' in this block; affine: the value named 'affine' in this block.

Line 235: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class CompiledConv2dBackward, which handles compiled conv2d backward; it creates no FPGA circuit by itself.

Line 236: `def partial_tile_count(self) -> int:` This starts function partial_tile_count. Its indented body performs partial tile count as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 237: `return sum(` This statement continues through source line 239; the explanation uses the complete expression rather than only the first visual fragment. This returns 'sum(group.affine.linear.partial_tile_count for group in self.group_programs)' to the caller of the function partial_tile_count, which handles partial tile count, ending this invocation. The important operands are sum: the value named 'sum' in this block; group: the value named 'group' in this block; affine: the value named 'affine' in this block.

Line 238: `group.affine.linear.partial_tile_count for group in self.group_programs` This is continuation line 2 of the 3-line logical statement that begins at source line 237. Its exact fragment is 'group.affine.linear.partial_tile_count for group in self.group_programs'. The important operands are group: the value named 'group' in this block; affine: the

value named 'affine' in this block; linear: the value named 'linear' in this block. Read together, the complete operation is: This returns ‘sum(group.affine.linear.partial_tile_count for group in self.group_programs)‘ to the caller of the function partial_tile_count, which handles partial tile count, ending this invocation. The important operands are sum: the value named 'sum' in this block; group: the value named 'group' in this block; affine: the value named 'affine' in this block.

Line 239: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 237. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘sum(group.affine.linear.partial_tile_count for group in self.group_programs)‘ to the caller of the function partial_tile_count, which handles partial tile count, ending this invocation. The important operands are sum: the value named 'sum' in this block; group: the value named 'group' in this block; affine: the value named 'affine' in this block.

Line 241: `def execute_batched(` This starts function execute_batched. Its indented body performs execute batched as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 242: `self,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 243: `driver: TrainingProgramExecutor,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named 'driver' in this block; TrainingProgramExecutor: the value named 'training program executor' in this block.

Line 244: `*,` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 245: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 246: `fabric_col2im: bool = True,` This assigns ‘True,‘ to ‘fabric_col2im‘ in host memory. Here fabric_col2im means the value named 'fabric col2im' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_batched, which handles execute batched.

Line 247: `) -> tuple[np.ndarray, np.ndarray, np.ndarray]:` Inside the function execute_batched, which handles execute batched, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 248: `input_gradient = np.zeros(self.input_shape, dtype=np.int64)` This assigns ‘np.zeros(self.input_shape, dtype=np.int64)‘ to ‘input_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are zeros: the value named 'zeros' in this block; input_shape: the value named 'input shape' in this block; dtype: the value named 'dtype' in this block. Context: the function execute_batched, which handles execute batched.

Line 249: `weight_gradient = np.zeros(self.weight_shape, dtype=np.int64)` This assigns ‘np.zeros(self.weight_shape, dtype=np.int64)‘ to ‘weight_gradient ‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are zeros: the value named 'zeros' in this block; weight_shape: a learned coefficient multiplied by an activation; dtype: the value named 'dtype' in this block. Context: the function execute_batched, which handles execute batched.

Line 250: `bias_gradient = np.zeros(self.output_shape[1], dtype=np.int64)` This assigns ‘np.zeros(self.output_shape[1] dtype=np.int64)‘ to ‘bias_gradient ‘ in host memory. Here bias_gradient means an additive learned offset applied after accumulation. The important operands are zeros: the value named ’zeros’ in this block; output_shape: the value named ’output shape’ in this block; dtype: the value named ’dtype’ in this block. Context: the function execute_batched, which handles execute batched.

Line 251: `output_height, output_width = self.output_shape[2:]` This assigns ‘self.output_shape[2:]‘ to ‘output_height, output_width ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are output_shape: the value named ’output shape’ in this block. Context: the function execute_batched, which handles execute batched.

Line 252: `kernel_h, kernel_w = self.kernel` This assigns ‘self.kernel‘ to ‘kernel_h, kernel_w ‘ in host memory. Here kernel_h means the value named ’kernel h’ in this block. The important operands are kernel: the value named ’kernel’ in this block. Context: the function execute_batched, which handles execute batched.

Line 253: `for group in self.group_programs:` This iterates group over ‘self.group_programs‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_batched, which handles execute batched.

Line 254: `patch_gradient, matrix_gradient, group_bias = group.affine.execute_batched(` This statement continues through source line 256; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘group.affine.execute_batched(driver, timeout_seconds=timeout_seconds)‘ to ‘patch_gradient, matrix_gradient, group_bias ‘ in host memory. Here patch_gradient means the signed loss derivative used to update a learned parameter. The important operands are group: the value named ’group’ in this block; affine: the value named ’affine’ in this block; execute_batched: the value named ’execute batched’ in this block. Context: the function execute_batched, which handles execute batched.

Line 255: `driver, timeout_seconds=timeout_seconds` This is continuation line 2 of the 3-line logical statement that begins at source line 254. Its exact fragment is ‘driver, timeout_seconds=timeout_seconds‘. The important operands are driver: the value named ’driver’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘group.affine.execute_batched(driver, timeout_seconds=timeout_seconds)‘ to ‘patch_gradient, matrix_gradient, group_bias ‘ in host memory. Here patch_gradient means the signed loss derivative used to update a learned parameter. The important operands are group: the value named ’group’ in this block; affine: the value named ’affine’ in this block; execute_batched: the value named ’execute batched’ in this block. Context: the function execute_batched, which handles execute batched.

Line 256: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 254. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘group.affine.execute_batched(driver, timeout_seconds=timeout_seconds)‘ to ‘patch_gradient, matrix_gradient, group_bias ‘ in host memory. Here patch_gradient means the signed loss derivative used to update a learned parameter. The important operands are group: the value named ’group’ in this block; affine: the value named ’affine’ in this block; execute_batched: the value named ’execute batched’ in this block. Context: the function execute_batched, which handles execute batched.

Line 257: `if fabric_col2im:` This executes the following indented host-code branch only when ‘fabric_col2im‘ is true. The important operands are fabric_col2im: the value named ’fabric col2im’ in this block. Context: the function execute_batched, which handles execute batched.

Line 258: `group_input_gradient = execute_col2im_accumulate(` This statement continues through source line 272; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.i input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.st padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)‘ to ‘group_input_gradient ‘ in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named ’execute col2im accumulate’ in this block; driver:

the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 259: `driver,` This is continuation line 2 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 260: `patch_gradient,` This is continuation line 3 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'patch_gradient,'. The important operands are patch_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.st
padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 261: `batch=self.input_shape[0],` This is continuation line 4 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'batch=self.input_shape[0],'. The important operands are batch: the value named 'batch' in this block; input_shape: the value named 'input shape' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 262: `channels=group.input_channels,` This is continuation line 5 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'channels=group.input_channels,'. The important operands are channels: the value named 'channels' in this block; group: the value named 'group' in this block; input_channels: the value named 'input channels' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.st
padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 263: `input_height=self.input_shape[2],` This is continuation line 6 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'input_height=self.input_shape[2],'. The important operands are input_height: the value named 'input height' in this block; input_shape: the value named 'input shape' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradie means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate:

the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 264: `input_width=self.input_shape[3],` This is continuation line 7 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'input_width=self.input_shape[3],'. The important operands are input_width: the value named 'input width' in this block; input_shape: the value named 'input shape' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_he output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 265: `output_height=output_height,` This is continuation line 8 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'output_height=output_height,'. The important operands are output_height: the value named 'output height' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 266: `output_width=output_width,` This is continuation line 9 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'output_width=output_width,'. The important operands are output_width: the value named 'output width' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.st padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 267: `kernel=self.kernel,` This is continuation line 10 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'kernel=self.kernel,'. The important operands are kernel: the value named 'kernel' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradie means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 268: `stride=self.stride,` This is continuation line 11 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'stride=self.stride,'. The important operands are stride: the value named 'stride' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradie means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient:

the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 269: `padding=self.padding,` This is continuation line 12 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'padding=self.padding,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradie means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 270: `dilation=self.dilation,` This is continuation line 13 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'dilation=self.dilation,'. The important operands are dilation: the value named 'dilation' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradie means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 271: `timeout_seconds=timeout_seconds,` This is continuation line 14 of the 15-line logical statement that begins at source line 258. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 272: `)` This is continuation line 15 of the 15-line logical statement that begins at source line 258. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_col2im_accumulate(driver, patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation, timeout_seconds=timeout_seconds,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_col2im_accumulate: the value named 'execute col2im accumulate' in this block; driver: the value named 'driver' in this block; patch_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 273: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute_batched, which handles execute batched.

Line 274: `group_input_gradient = col2im_accumulate_fixed(` This statement continues through source line 286; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named

'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 275: `patch_gradient,` This is continuation line 2 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'patch_gradient,'. The important operands are patch_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 276: `batch=self.input_shape[0],` This is continuation line 3 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'batch=self.input_shape[0],'. The important operands are batch: the value named 'batch' in this block; input_shape: the value named 'input shape' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_wid kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 277: `channels=group.input_channels,` This is continuation line 4 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'channels=group.input_channels,'. The important operands are channels: the value named 'channels' in this block; group: the value named 'group' in this block; input_channels: the value named 'input channels' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 278: `input_height=self.input_shape[2],` This is continuation line 5 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'input_height=self.input_shape[2],'. The important operands are input_height: the value named 'input height' in this block; input_shape: the value named 'input shape' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_he output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 279: `input_width=self.input_shape[3],` This is continuation line 6 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'input_width=self.input_shape[3],'. The important operands are input_width: the value named 'input width' in this block; input_shape: the value named 'input shape' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 280: `output_height=output_height,` This is continuation line 7 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'output_height=output_height,'. The important operands are output_height: the value named 'output height' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_ input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.st padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 281: `output_width=output_width,` This is continuation line 8 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'output_width=output_width,'. The important operands are output_width: the value named 'output width' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 282: `kernel=self.kernel,` This is continuation line 9 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'kernel=self.kernel,'. The important operands are kernel: the value named 'kernel' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 283: `stride=self.stride,` This is continuation line 10 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'stride=self.stride,'. The important operands are stride: the value named 'stride' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 284: `padding=self.padding,` This is continuation line 11 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'padding=self.padding,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 285: `dilation=self.dilation,` This is continuation line 12 of the 13-line logical statement that begins at source line 274. Its exact fragment is 'dilation=self.dilation,'. The important operands are dilation: the value named 'dilation' in this block. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss

derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 286: `)` This is continuation line 13 of the 13-line logical statement that begins at source line 274. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'col2im_accumulate_fixed(patch_gradient, batch=self.input_shape[0], channels=group.input_channels, input_height=self.input_shape[2], input_width=self.input_shape[3], output_height=output_height, output_width=output_width, kernel=self.kernel, stride=self.stride, padding=self.padding, dilation=self.dilation,)' to 'group_input_gradient ' in host memory. Here group_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are col2im_accumulate_fixed: the value named 'col2im accumulate fixed' in this block; patch_gradient: the signed loss derivative used to update a learned parameter; batch: the value named 'batch' in this block. Context: the function execute_batched, which handles execute batched.

Line 287: `input_start = group.input_channel_start` This assigns 'group.input_channel_start' to 'input_start ' in host memory. Here input_start means the value named 'input start' in this block. The important operands are group: the value named 'group' in this block; input_channel_start: the value named 'input channel start' in this block. Context: the function execute_batched, which handles execute batched.

Line 288: `output_start = group.output_channel_start` This assigns 'group.output_channel_start' to 'output_start ' in host memory. Here output_start means the value named 'output start' in this block. The important operands are group: the value named 'group' in this block; output_channel_start: the value named 'output channel start' in this block. Context: the function execute_batched, which handles execute batched.

Line 289: `input_gradient[:, input_start:input_start + group.input_channels] = group_input_gradient` This assigns 'group_input_gradient' to 'input_gradient[' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are group_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_batched, which handles execute batched.

Line 290: `matrix = matrix_gradient.T.reshape(` This statement continues through source line 295; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'matrix_gradient.T.reshape(group.output_channels, group.input_channels, kernel_h, kernel_w,)' to 'matrix ' in host memory. Here matrix means the value named 'matrix' in this block. The important operands are matrix_gradient: the signed loss derivative used to update a learned parameter; T: the value named 't' in this block; reshape: the value named 'reshape' in this block. Context: the function execute_batched, which handles execute batched.

Line 291: `group.output_channels,` This is continuation line 2 of the 6-line logical statement that begins at source line 290. Its exact fragment is 'group.output_channels,'. The important operands are group: the value named 'group' in this block; output_channels: the value named 'output channels' in this block. Read together, the complete operation is: This assigns 'matrix_gradient.T.reshape(group.output_channels, group.input_channels, kernel_h, kernel_w,)' to 'matrix ' in host memory. Here matrix means the value named 'matrix' in this block. The important operands are matrix_gradient: the signed loss derivative used to update a learned parameter; T: the value named 't' in this block; reshape: the value named 'reshape' in this block. Context: the function execute_batched, which handles execute batched.

Line 292: `group.input_channels,` This is continuation line 3 of the 6-line logical statement that begins at source line 290. Its exact fragment is 'group.input_channels,'. The important operands are group: the value named 'group' in this block; input_channels: the value named 'input channels' in this block. Read together, the complete operation is: This assigns 'matrix_gradient.T.reshape(group.output_channels, group.input_channels, kernel_h, kernel_w,)' to 'matrix ' in host memory. Here matrix means the value named 'matrix' in this block. The important operands are matrix_gradient: the signed loss derivative used to update a learned parameter; T: the value named 't' in this block; reshape: the value named 'reshape' in this block. Context: the function execute_batched, which handles execute batched.

Line 293: `kernel_h,` This is continuation line 4 of the 6-line logical statement that begins at source line 290. Its exact fragment is 'kernel_h,'. The important operands are kernel_h: the value named 'kernel h' in this block. Read together, the complete operation is: This assigns 'matrix_gradient.T.reshape(group.output_channels, group.input_channels,

kernel_h, kernel_w,)‘ to ‘matrix ‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are matrix_gradient: the signed loss derivative used to update a learned parameter; T: the value named ’t’ in this block; reshape: the value named ’reshape’ in this block. Context: the function execute_batched, which handles execute batched.

Line 294: `kernel_w,` This is continuation line 5 of the 6-line logical statement that begins at source line 290. Its exact fragment is ‘kernel_w,‘. The important operands are kernel_w: the value named ’kernel w’ in this block. Read together, the complete operation is: This assigns ‘matrix_gradient.T.reshape(group.output_channels, group.input_channels, kernel_h, kernel_w,)‘ to ‘matrix ‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are matrix_gradient: the signed loss derivative used to update a learned parameter; T: the value named ’t’ in this block; reshape: the value named ’reshape’ in this block. Context: the function execute_batched, which handles execute batched.

Line 295: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 290. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘matrix_gradient.T.reshape(group.output_channels, group.input_channels, kernel_h, kernel_w,)‘ to ‘matrix ‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are matrix_gradient: the signed loss derivative used to update a learned parameter; T: the value named ’t’ in this block; reshape: the value named ’reshape’ in this block. Context: the function execute_batched, which handles execute batched.

Line 296: `weight_gradient[` This statement continues through source line 298; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘matrix‘ to ‘weight_gradient[output_start‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are matrix: the value named ’matrix’ in this block. Context: the function execute_batched, which handles execute batched.

Line 297: `output_start:output_start + group.output_channels` This is continuation line 2 of the 3-line logical statement that begins at source line 296. Its exact fragment is ‘output_start:output_start + group.output_channels‘. The important operands are output_start: the value named ’output start’ in this block; group: the value named ’group’ in this block; output_channels: the value named ’output channels’ in this block. Read together, the complete operation is: This assigns ‘matrix‘ to ‘weight_gradient[output_start‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are matrix: the value named ’matrix’ in this block. Context: the function execute_batched, which handles execute batched.

Line 298: `] = matrix` This is continuation line 3 of the 3-line logical statement that begins at source line 296. Its exact fragment is ‘] = matrix‘. The important operands are matrix: the value named ’matrix’ in this block. Read together, the complete operation is: This assigns ‘matrix‘ to ‘weight_gradient[output_start‘ in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are matrix: the value named ’matrix’ in this block. Context: the function execute_batched, which handles execute batched.

Line 299: `bias_gradient[` This statement continues through source line 301; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘group_bias‘ to ‘bias_gradient[output_start‘ in host memory. Here bias_gradient means an additive learned offset applied after accumulation. The important operands are group_bias: an additive learned offset applied after accumulation. Context: the function execute_batched, which handles execute batched.

Line 300: `output_start:output_start + group.output_channels` This is continuation line 2 of the 3-line logical statement that begins at source line 299. Its exact fragment is ‘output_start:output_start + group.output_channels‘. The important operands are output_start: the value named ’output start’ in this block; group: the value named ’group’ in this block; output_channels: the value named ’output channels’ in this block. Read together, the complete operation is: This assigns ‘group_bias‘ to ‘bias_gradient[output_start‘ in host memory. Here bias_gradient means an additive learned offset applied after accumulation. The important operands are group_bias: an additive learned offset applied after accumulation. Context: the function execute_batched, which handles execute batched.

Line 301: `] = group_bias` This is continuation line 3 of the 3-line logical statement that begins at source line 299. Its exact fragment is '] = group_bias'. The important operands are group_bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'group_bias' to 'bias_gradient[output_start' in host memory. Here bias_gradient means an additive learned offset applied after accumulation. The important operands are group_bias: an additive learned offset applied after accumulation. Context: the function execute_batched, which handles execute batched.

Line 302: `return input_gradient, weight_gradient, bias_gradient` This returns 'input_gradient, weight_gradient, bias_gradient' to the caller of the function execute_batched, which handles execute batched, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; weight_gradient: a learned coefficient multiplied by an activation; bias_gradient: an additive learned offset applied after accumulation.

Line 305: `def _signed_matrix(name: str, value, bits: int) -> np.ndarray:` This starts function _signed_matrix. Its indented body performs signed matrix as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 306: `array = np.asarray(value)` This assigns 'np.asarray(value)' to 'array ' in host memory. Here array means the value named 'array' in this block. The important operands are asarray: the value named 'asarray' in this block; value: the value named 'value' in this block. Context: the function _signed_matrix, which handles signed matrix.

Line 307: `if array.ndim != 2 or array.size == 0:` This executes the following indented host-code branch only when 'array.ndim != 2 or array.size == 0' is true. The important operands are array: the value named 'array' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function _signed_matrix, which handles signed matrix.

Line 308: `raise ValueError(f"{name} must be a non-empty two-dimensional matrix")` This rejects an invalid state by raising 'ValueError(f"{name} must be a non-empty two-dimensional matrix")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 309: `if not np.issubdtype(array.dtype, np.integer):` This executes the following indented host-code branch only when 'not np.issubdtype(array.dtype, np.integer)' is true. The important operands are not: the value named 'not' in this block; issubdtype: the value named 'issubdtype' in this block; array: the value named 'array' in this block. Context: the function _signed_matrix, which handles signed matrix.

Line 310: `raise ValueError(f"{name} must contain integers")` This rejects an invalid state by raising 'ValueError(f"{name} must contain integers")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 311: `result = array.astype(np.int64)` This assigns 'array.astype(np.int64)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are array: the value named 'array' in this block; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Context: the function _signed_matrix, which handles signed matrix.

Line 312: `minimum = -(1 << (bits - 1))` This assigns '-(1 << (bits - 1))' to 'minimum ' in host memory. Here minimum means the value named 'minimum' in this block. The important operands are bits: the value named 'bits' in this block. Context: the function _signed_matrix, which handles signed matrix.

Line 313: `maximum = (1 << (bits - 1)) - 1` This assigns '(1 << (bits - 1)) - 1' to 'maximum ' in host memory. Here maximum means the value named 'maximum' in this block. The important operands are bits: the value named 'bits' in this block. Context: the function _signed_matrix, which handles signed matrix.

Line 314: `if np.any(result < minimum) or np.any(result > maximum):` This executes the following indented host-code branch only when 'np.any(result < minimum) or np.any(result > maximum)' is true. The important operands are any: the value named 'any' in this block; result: the value named 'result' in this block; minimum: the value named 'minimum' in this block. Context: the function _signed_matrix, which handles signed matrix.

Line 315: `raise ValueError(f"{name} must fit signed {bits}-bit values")` This rejects an invalid state by raising ‘ValueError(f"{name} must fit signed {bits}-bit values")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 316: `return result` This returns ‘result‘ to the caller of the function _signed_matrix, which handles signed matrix, ending this invocation. The important operands are result: the value named 'result' in this block.

Line 319: `def _conv_output_size(` This starts function _conv_output_size. Its indented body performs conv output size as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 320: `input_size: int,` Inside the function _conv_output_size, which handles conv output size, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_size: the value named 'input size' in this block.

Line 321: `kernel: int,` Inside the function _conv_output_size, which handles conv output size, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are kernel: the value named 'kernel' in this block.

Line 322: `stride: int,` Inside the function _conv_output_size, which handles conv output size, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are stride: the value named 'stride' in this block.

Line 323: `padding: int,` Inside the function _conv_output_size, which handles conv output size, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are padding: the value named 'padding' in this block.

Line 324: `dilation: int,` Inside the function _conv_output_size, which handles conv output size, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are dilation: the value named 'dilation' in this block.

Line 325: `) -> int:` Inside the function _conv_output_size, which handles conv output size, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 326: `return (input_size + 2 * padding - dilation * (kernel - 1) - 1) // stride + 1` This returns ‘(input_size + 2 * padding - dilation * (kernel - 1) - 1) // stride + 1‘ to the caller of the function _conv_output_size, which handles conv output size, ending this invocation. The important operands are input_size: the value named 'input size' in this block; padding: the value named 'padding' in this block; dilation: the value named 'dilation' in this block.

Line 329: `def im2col_signed_nchw(` This starts function im2col_signed_nchw. Its indented body performs im2col signed nchw as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 330: `values,` Inside the function im2col_signed_nchw, which handles im2col signed nchw, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are values: the value named 'values' in this block.

Line 331: `*,` Inside the function im2col_signed_nchw, which handles im2col signed nchw, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 332: `kernel,` Inside the function im2col_signed_nchw, which handles im2col signed nchw, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are kernel: the value named 'kernel' in this block.

Line 333: `stride,` Inside the function im2col_signed_nchw, which handles im2col signed nchw, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are stride: the value named 'stride' in this block.

Line 334: `padding,` Inside the function im2col_signed_nchw, which handles im2col signed nchw, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are padding: the value named 'padding' in this block.

Line 335: `dilation,` Inside the function im2col_signed_nchw, which handles im2col signed nchw, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are dilation: the value named 'dilation' in this block.

Line 336: `) -> np.ndarray:` Inside the function im2col_signed_nchw, which handles im2col signed nchw, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are ndarray: the value named 'ndarray' in this block.

Line 337: `"""Flatten NCHW convolution patches in batch/output-y/output-x order."""` This is executable documentation attached to the function im2col_signed_nchw, which handles im2col signed nchw. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 339: `source = np.asarray(values, dtype=np.int64)` This assigns ‘np.asarray(values, dtype=np.int64)‘ to ‘source ‘ in host memory. Here source means the value named 'source' in this block. The important operands are asarray: the value named 'asarray' in this block; values: the value named 'values' in this block; dtype: the value named 'dtype' in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 340: `if source.ndim != 4:` This executes the following indented host-code branch only when ‘source.ndim != 4‘ is true. The important operands are source: the value named 'source' in this block; ndim: the value named 'ndim' in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 341: `raise ValueError("im2col source must be an NCHW tensor")` This rejects an invalid state by raising ‘ValueError("im2col source must be an NCHW tensor")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 342: `kernel_h, kernel_w = _pair("kernel", kernel)` This assigns ‘_pair("kernel", kernel)‘ to ‘kernel_h, kernel_w ‘ in host memory. Here kernel_h means the value named 'kernel h' in this block. The important operands are _pair: the value named 'pair' in this block; kernel: the value named 'kernel' in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 343: `stride_h, stride_w = _pair("stride", stride)` This assigns ‘_pair("stride", stride)‘ to ‘stride_h, stride_w ‘ in host memory. Here stride_h means the value named 'stride h' in this block. The important operands are _pair: the value named 'pair' in this block; stride: the value named 'stride' in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 344: `dilation_h, dilation_w = _pair("dilation", dilation)` This assigns ‘_pair("dilation", dilation)‘ to ‘dilation_h, dilation_w ‘ in host memory. Here dilation_h means the value named 'dilation h' in this block. The important operands are _pair: the value named 'pair' in this block; dilation: the value named 'dilation' in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 345: `pad_h, pad_w = (padding, padding) if isinstance(padding, int) else tuple(padding)` This assigns ‘(padding, padding) if isinstance(padding, int) else tuple(padding)‘ to ‘pad_h, pad_w ‘ in host memory. Here pad_h means the value named ’pad h’ in this block. The important operands are padding: the value named ’padding’ in this block; isinstance: the value named ’isinstance’ in this block; tuple: the value named ’tuple’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 346: `batch, channels, height, width = source.shape` This assigns ‘source.shape‘ to ‘batch, channels, height, width ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are source: the value named ’source’ in this block; shape: the value named ’shape’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 347: `output_height = _conv_output_size(height, kernel_h, stride_h, pad_h, dilation_h)` This assigns ‘_conv_output_size(height, kernel_h, stride_h, pad_h, dilation_h)‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are _conv_output_size: the value named ’conv output size’ in this block; height: the value named ’height’ in this block; kernel_h: the value named ’kernel h’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 348: `output_width = _conv_output_size(width, kernel_w, stride_w, pad_w, dilation_w)` This assigns ‘_conv_output_size(width, kernel_w, stride_w, pad_w, dilation_w)‘ to ‘output_width ‘ in host memory. Here output_width means the value named ’output width’ in this block. The important operands are _conv_output_size: the value named ’conv output size’ in this block; width: the value named ’width’ in this block; kernel_w: the value named ’kernel w’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 349: `patches = np.zeros(` This statement continues through source line 352; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.zeros((batch * output_height * output_width, channels * kernel_h * kernel_w), dtype=np.int64,)‘ to ‘patches ‘ in host memory. Here patches means the value named ’patches’ in this block. The important operands are zeros: the value named ’zeros’ in this block; batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 350: `(batch * output_height * output_width, channels * kernel_h * kernel_w),` This is continuation line 2 of the 4-line logical statement that begins at source line 349. Its exact fragment is ‘(batch * output_height * output_width, channels * kernel_h * kernel_w),‘. The important operands are batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block; output_width: the value named ’output width’ in this block. Read together, the complete operation is: This assigns ‘np.zeros((batch * output_height * output_width, channels * kernel_h * kernel_w), dtype=np.int64,)‘ to ‘patches ‘ in host memory. Here patches means the value named ’patches’ in this block. The important operands are zeros: the value named ’zeros’ in this block; batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 351: `dtype=np.int64,` This is continuation line 3 of the 4-line logical statement that begins at source line 349. Its exact fragment is ‘dtype=np.int64,‘. The important operands are dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This assigns ‘np.zeros((batch * output_height * output_width, channels * kernel_h * kernel_w), dtype=np.int64,)‘ to ‘patches ‘ in host memory. Here patches means the value named ’patches’ in this block. The important operands are zeros: the value named ’zeros’ in this block; batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 352: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 349. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.zeros((batch * output_height * output_width, channels * kernel_h * kernel_w), dtype=np.int64,)‘ to ‘patches ‘ in host memory. Here patches means the value named ’patches’ in this block. The important operands are zeros: the value named ’zeros’ in this block; batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 353: `row = 0` This assigns ‘0‘ to ‘row ‘ in host memory. Here row means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 354: `for batch_index in range(batch):` This iterates batch_index over ‘range(batch)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function im2col_signed_nchw, which handles im2col signed nchw.

Line 355: `for output_y in range(output_height):` This iterates output_y over ‘range(output_height)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function im2col_signed_nchw, which handles im2col signed nchw.

Line 356: `for output_x in range(output_width):` This iterates output_x over ‘range(output_width)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function im2col_signed_nchw, which handles im2col signed nchw.

Line 357: `column = 0` This assigns ‘0‘ to ‘column ‘ in host memory. Here column means one output-feature position processed in parallel. The expression is a literal or structural operator and has no hidden software state. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 358: `for channel in range(channels):` This iterates channel over ‘range(channels)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function im2col_signed_nchw, which handles im2col signed nchw.

Line 359: `for kernel_y in range(kernel_h):` This iterates kernel_y over ‘range(kernel_h)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function im2col_signed_nchw, which handles im2col signed nchw.

Line 360: `input_y = output_y * stride_h - pad_h + kernel_y * dilation_h` This assigns ‘output_y * stride_h - pad_h + kernel_y * dilation_h‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride_h: the value named ’stride h’ in this block; pad_h: the value named ’pad h’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 361: `for kernel_x in range(kernel_w):` This iterates kernel_x over ‘range(kernel_w)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function im2col_signed_nchw, which handles im2col signed nchw.

Line 362: `input_x = output_x * stride_w - pad_w + kernel_x * dilation_w` This assigns ‘output_x * stride_w - pad_w + kernel_x * dilation_w‘ to ‘input_x ‘ in host memory. Here input_x means the value named ’input x’ in this block. The important operands are output_x: the value named ’output x’ in this block; stride_w: the value named ’stride w’ in this block; pad_w: the value named ’pad w’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 363: `if 0 <= input_y < height and 0 <= input_x < width:` This executes the following indented host-code branch only when ‘0 <= input_y < height and 0 <= input_x < width‘ is true. The important operands are input_y: the value named ’input y’ in this block; height: the value named ’height’ in this block; and: the value named ’and’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 364: `patches[row, column] = source[` This statement continues through source line 366; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘source[batch_index, channel, input_y, input_x]‘ to ‘patches[row, column] ‘ in host memory. Here patches means the value named ’patches’ in this block. The important operands are source: the value named ’source’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named ’channel’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 365: `batch_index, channel, input_y, input_x` This is continuation line 2 of the 3-line logical statement that begins at source line 364. Its exact fragment is ‘batch_index, channel, input_y, input_x‘. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named ’channel’ in this block; input_y: the value named ’input y’ in this block. Read together, the complete operation is: This assigns ‘source[batch_index, channel, input_y, input_x]‘ to ‘patches[row, column] ‘ in host memory. Here patches means the value named ’patches’ in this block. The important operands are source: the value named ’source’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named ’channel’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 366: `]` This is continuation line 3 of the 3-line logical statement that begins at source line 364. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘source[batch_index, channel, input_y, input_x]‘ to ‘patches[row, column] ‘ in host memory. Here patches means the value named ’patches’ in this block. The important operands are source: the value named ’source’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named ’channel’ in this block. Context: the function im2col_signed_nchw, which handles im2col signed nchw.

Line 367: `column += 1` Inside the function im2col_signed_nchw, which handles im2col signed nchw, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are column: one output-feature position processed in parallel.

Line 368: `row += 1` Inside the function im2col_signed_nchw, which handles im2col signed nchw, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are row: one output-row position processed by the tensor tile.

Line 369: `return patches` This returns ‘patches‘ to the caller of the function im2col_signed_nchw, which handles im2col signed nchw, ending this invocation. The important operands are patches: the value named ’patches’ in this block.

Line 372: `def col2im_accumulate_fixed(` This starts function col2im_accumulate_fixed. Its indented body performs col2im accumulate fixed as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 373: `patch_gradients,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are patch_gradients: the signed loss derivative used to update a learned parameter.

Line 374: `*,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 375: `batch: int,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are batch: the value named ’batch’ in this block.

Line 376: `channels: int,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are channels: the value named ’channels’ in this block.

Line 377: `input_height: int,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_height: the value named 'input height' in this block.

Line 378: `input_width: int,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_width: the value named 'input width' in this block.

Line 379: `output_height: int,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_height: the value named 'output height' in this block.

Line 380: `output_width: int,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_width: the value named 'output width' in this block.

Line 381: `kernel,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are kernel: the value named 'kernel' in this block.

Line 382: `stride,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are stride: the value named 'stride' in this block.

Line 383: `padding,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are padding: the value named 'padding' in this block.

Line 384: `dilation,` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are dilation: the value named 'dilation' in this block.

Line 385: `) -> np.ndarray:` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are ndarray: the value named 'ndarray' in this block.

Line 386: `"""Scatter patch gradients with one signed-s32 saturation per addition."""` This is executable documentation attached to the function col2im_accumulate_fixed, which handles col2im accumulate fixed. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 388: `patches = np.asarray(patch_gradients, dtype=np.int64)` This assigns 'np.asarray(patch_gradients, dtype=np.int64)' to 'patches ' in host memory. Here patches means the value named 'patches' in this block. The important operands are asarray: the value named 'asarray' in this block; patch_gradients: the signed loss derivative used to update a learned parameter; dtype: the value named 'dtype' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 389: `kernel_h, kernel_w = _pair("kernel", kernel)` This assigns '_pair("kernel", kernel)' to 'kernel_h, kernel_w ' in host memory. Here kernel_h means the value named 'kernel h' in this block. The important operands are _pair: the value named 'pair' in this block; kernel: the value named 'kernel' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 390: `stride_h, stride_w = _pair("stride", stride)` This assigns '_pair("stride", stride)' to 'stride_h, stride_w ' in host memory. Here stride_h means the value named 'stride h' in this block. The important operands are _pair: the value named 'pair' in this block; stride: the value named 'stride' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 391: `dilation_h, dilation_w = _pair("dilation", dilation)` This assigns '_pair("dilation", dilation)' to 'dilation_h, dilation_w ' in host memory. Here dilation_h means the value named 'dilation h' in this block. The important operands are _pair: the value named 'pair' in this block; dilation: the value named 'dilation' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 392: `pad_h, pad_w = (padding, padding) if isinstance(padding, int) else tuple(padding)` This assigns '(padding, padding) if isinstance(padding, int) else tuple(padding)' to 'pad_h, pad_w ' in host memory. Here pad_h means the value named 'pad h' in this block. The important operands are padding: the value named 'padding' in this block; isinstance: the value named 'isinstance' in this block; tuple: the value named 'tuple' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 393: `expected_shape = (` This statement continues through source line 396; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(batch * output_height * output_width, channels * kernel_h * kernel_w,)' to 'expected_shape ' in host memory. Here expected_shape means the value named 'expected shape' in this block. The important operands are batch: the value named 'batch' in this block; output_height: the value named 'output height' in this block; output_width: the value named 'output width' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 394: `batch * output_height * output_width,` This is continuation line 2 of the 4-line logical statement that begins at source line 393. Its exact fragment is 'batch * output_height * output_width,'. The important operands are batch: the value named 'batch' in this block; output_height: the value named 'output height' in this block; output_width: the value named 'output width' in this block. Read together, the complete operation is: This assigns '(batch * output_height * output_width, channels * kernel_h * kernel_w,)' to 'expected_shape ' in host memory. Here expected_shape means the value named 'expected shape' in this block. The important operands are batch: the value named 'batch' in this block; output_height: the value named 'output height' in this block; output_width: the value named 'output width' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 395: `channels * kernel_h * kernel_w,` This is continuation line 3 of the 4-line logical statement that begins at source line 393. Its exact fragment is 'channels * kernel_h * kernel_w,'. The important operands are channels: the value named 'channels' in this block; kernel_h: the value named 'kernel h' in this block; kernel_w: the value named 'kernel w' in this block. Read together, the complete operation is: This assigns '(batch * output_height * output_width, channels * kernel_h * kernel_w,)' to 'expected_shape ' in host memory. Here expected_shape means the value named 'expected shape' in this block. The important operands are batch: the value named 'batch' in this block; output_height: the value named 'output height' in this block; output_width: the value named 'output width' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 396: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 393. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(batch * output_height * output_width, channels * kernel_h * kernel_w,)' to 'expected_shape ' in host memory. Here expected_shape means the value named 'expected shape' in this block. The important operands are batch: the value named 'batch' in this block; output_height: the value named 'output height' in this block; output_width: the value named 'output width' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 397: `if patches.shape != expected_shape:` This executes the following indented host-code branch only when 'patches.shape != expected_shape' is true. The important operands are patches: the value named 'patches' in this block; shape: the value named 'shape' in this block; expected_shape: the value named 'expected shape' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 398: `raise ValueError(f"patch gradient shape must be {expected_shape}")` This rejects an invalid state by raising 'ValueError(f"patch gradient shape must be {expected_shape}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 399: `output = np.zeros((batch, channels, input_height, input_width), dtype=np.int64)` This assigns 'np.zeros((batch, channels, input_height, input_width), dtype=np.int64)' to 'output ' in host memory. Here output means the value named 'output' in this block. The important operands are zeros: the value named 'zeros' in this block; batch: the value named 'batch' in this block; channels: the value named 'channels' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 400: `row = 0` This assigns '0' to 'row ' in host memory. Here row means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 401: `for batch_index in range(batch):` This iterates batch_index over 'range(batch)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 402: `for output_y in range(output_height):` This iterates output_y over 'range(output_height)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 403: `for output_x in range(output_width):` This iterates output_x over 'range(output_width)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 404: `column = 0` This assigns '0' to 'column ' in host memory. Here column means one output-feature position processed in parallel. The expression is a literal or structural operator and has no hidden software state. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 405: `for channel in range(channels):` This iterates channel over 'range(channels)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 406: `for kernel_y in range(kernel_h):` This iterates kernel_y over 'range(kernel_h)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 407: `input_y = output_y * stride_h - pad_h + kernel_y * dilation_h` This assigns 'output_y * stride_h - pad_h + kernel_y * dilation_h' to 'input_y ' in host memory. Here input_y means the value named 'input y' in this block. The important operands are output_y: the value named 'output y' in this block; stride_h: the value named 'stride h' in this block; pad_h: the value named 'pad h' in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 408: `for kernel_x in range(kernel_w):` This iterates kernel_x over 'range(kernel_w)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 409: `input_x = output_x * stride_w - pad_w + kernel_x * dilation_w` This assigns ‘output_x * stride_w - pad_w + kernel_x * dilation_w‘ to ‘input_x ‘ in host memory. Here input_x means the value named ’input x’ in this block. The important operands are output_x: the value named ’output x’ in this block; stride_w: the value named ’stride w’ in this block; pad_w: the value named ’pad w’ in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 410: `if 0 <= input_y < input_height and 0 <= input_x < input_width:` This executes the following indented host-code branch only when ‘0 <= input_y < input_height and 0 <= input_x < input_width‘ is true. The important operands are input_y: the value named ’input y’ in this block; input_height: the value named ’input height’ in this block; and: the value named ’and’ in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 411: `value = output[batch_index, channel, input_y, input_x]` This assigns ‘output[batch_index, channel, input_y, input_x]‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are output: the value named ’output’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named ’channel’ in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 412: `output[batch_index, channel, input_y, input_x] = saturate_signed(` This statement continues through source line 414; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘saturate_signed(np.asarray(value + patches[row, column], dtype=np.int64), 32)‘ to ‘output[batch_index, channel, input_y, input_x] ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are saturate_signed: the value named ’saturate signed’ in this block; asarray: the value named ’asarray’ in this block; value: the value named ’value’ in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 413: `np.asarray(value + patches[row, column], dtype=np.int64), 32` This is continuation line 2 of the 3-line logical statement that begins at source line 412. Its exact fragment is ‘np.asarray(value + patches[row, column], dtype=np.int64), 32‘. The important operands are asarray: the value named ’asarray’ in this block; value: the value named ’value’ in this block; patches: the value named ’patches’ in this block. Read together, the complete operation is: This assigns ‘saturate_signed(np.asarray(value + patches[row, column], dtype=np.int64), 32)‘ to ‘output[batch_index, channel, input_y, input_x] ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are saturate_signed: the value named ’saturate signed’ in this block; asarray: the value named ’asarray’ in this block; value: the value named ’value’ in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 414: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 412. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘saturate_signed(np.asarray(value + patches[row, column], dtype=np.int64), 32)‘ to ‘output[batch_index, channel, input_y, input_x] ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are saturate_signed: the value named ’saturate signed’ in this block; asarray: the value named ’asarray’ in this block; value: the value named ’value’ in this block. Context: the function col2im_accumulate_fixed, which handles col2im accumulate fixed.

Line 415: `column += 1` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are column: one output-feature position processed in parallel.

Line 416: `row += 1` Inside the function col2im_accumulate_fixed, which handles col2im accumulate fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are row: one output-row position processed by the tensor tile.

Line 417: `return output` This returns ‘output‘ to the caller of the function col2im_accumulate_fixed, which handles col2im accumulate fixed, ending this invocation. The important operands are output: the value named 'output' in this block.

Line 420: `def execute_col2im_accumulate(` This starts function execute_col2im_accumulate. Its indented body performs execute col2im accumulate as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 421: `driver: TrainingCommandExecutor,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named 'driver' in this block; TrainingCommandExecutor: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 422: `patch_gradients,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are patch_gradients: the signed loss derivative used to update a learned parameter.

Line 423: `*,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 424: `batch: int,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are batch: the value named 'batch' in this block.

Line 425: `channels: int,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are channels: the value named 'channels' in this block.

Line 426: `input_height: int,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_height: the value named 'input height' in this block.

Line 427: `input_width: int,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_width: the value named 'input width' in this block.

Line 428: `output_height: int,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_height: the value named 'output height' in this block.

Line 429: `output_width: int,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_width: the value named 'output width' in this block.

Line 430: `kernel,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are kernel: the value named 'kernel' in this block.

Line 431: `stride,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are stride: the value named 'stride' in this block.

Line 432: `padding,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are padding: the value named 'padding' in this block.

Line 433: `dilation,` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are dilation: the value named 'dilation' in this block.

Line 434: `timeout_seconds: float = 5.0,` This assigns '5.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 435: `) -> np.ndarray:` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are ndarray: the value named 'ndarray' in this block.

Line 436: `"""Scatter every overlapping convolution gradient with fabric vector adds."""` This is executable documentation attached to the function execute_col2im_accumulate, which handles execute col2im accumulate. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 438: `patches = np.asarray(patch_gradients, dtype=np.int64)` This assigns 'np.asarray(patch_gradients, dtype=np.int64)' to 'patches ' in host memory. Here patches means the value named 'patches' in this block. The important operands are asarray: the value named 'asarray' in this block; patch_gradients: the signed loss derivative used to update a learned parameter; dtype: the value named 'dtype' in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 439: `kernel_h, kernel_w = _pair("kernel", kernel)` This assigns '_pair("kernel", kernel)' to 'kernel_h, kernel_w ' in host memory. Here kernel_h means the value named 'kernel h' in this block. The important operands are _pair: the value named 'pair' in this block; kernel: the value named 'kernel' in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 440: `stride_h, stride_w = _pair("stride", stride)` This assigns '_pair("stride", stride)' to 'stride_h, stride_w ' in host memory. Here stride_h means the value named 'stride h' in this block. The important operands are _pair: the value named 'pair' in this block; stride: the value named 'stride' in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 441: `dilation_h, dilation_w = _pair("dilation", dilation)` This assigns '_pair("dilation", dilation)' to 'dilation_h, dilation_w ' in host memory. Here dilation_h means the value named 'dilation h' in this block. The important operands are _pair: the value named 'pair' in this block; dilation: the value named 'dilation' in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 442: `pad_h, pad_w = (padding, padding) if isinstance(padding, int) else tuple(padding)` This assigns ‘(padding, padding) if isinstance(padding, int) else tuple(padding)‘ to ‘pad_h, pad_w ‘ in host memory. Here pad_h means the value named ’pad h’ in this block. The important operands are padding: the value named ’padding’ in this block; isinstance: the value named ’isinstance’ in this block; tuple: the value named ’tuple’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 443: `expected_shape = (` This statement continues through source line 446; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(batch * output_height * output_width, channels * kernel_h * kernel_w,)‘ to ‘expected_shape ‘ in host memory. Here expected_shape means the value named ’expected shape’ in this block. The important operands are batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block; output_width: the value named ’output width’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 444: `batch * output_height * output_width,` This is continuation line 2 of the 4-line logical statement that begins at source line 443. Its exact fragment is ‘batch * output_height * output_width,‘. The important operands are batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block; output_width: the value named ’output width’ in this block. Read together, the complete operation is: This assigns ‘(batch * output_height * output_width, channels * kernel_h * kernel_w,)‘ to ‘expected_shape ‘ in host memory. Here expected_shape means the value named ’expected shape’ in this block. The important operands are batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block; output_width: the value named ’output width’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 445: `channels * kernel_h * kernel_w,` This is continuation line 3 of the 4-line logical statement that begins at source line 443. Its exact fragment is ‘channels * kernel_h * kernel_w,‘. The important operands are channels: the value named ’channels’ in this block; kernel_h: the value named ’kernel h’ in this block; kernel_w: the value named ’kernel w’ in this block. Read together, the complete operation is: This assigns ‘(batch * output_height * output_width, channels * kernel_h * kernel_w,)‘ to ‘expected_shape ‘ in host memory. Here expected_shape means the value named ’expected shape’ in this block. The important operands are batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block; output_width: the value named ’output width’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 446: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 443. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(batch * output_height * output_width, channels * kernel_h * kernel_w,)‘ to ‘expected_shape ‘ in host memory. Here expected_shape means the value named ’expected shape’ in this block. The important operands are batch: the value named ’batch’ in this block; output_height: the value named ’output height’ in this block; output_width: the value named ’output width’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 447: `if patches.shape != expected_shape:` This executes the following indented host-code branch only when ‘patches.shape != expected_shape‘ is true. The important operands are patches: the value named ’patches’ in this block; shape: the value named ’shape’ in this block; expected_shape: the value named ’expected shape’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 448: `raise ValueError(f"patch gradient shape must be {expected_shape}")` This rejects an invalid state by raising ‘ValueError(f"patch gradient shape must be {expected_shape}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 449: `output = np.zeros((batch, channels, input_height, input_width), dtype=np.int64)` This assigns ‘np.zeros((batch, channels, input_height, input_width), dtype=np.int64)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are zeros: the value named ’zeros’ in this block; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 450: `row = 0` This assigns ‘0‘ to ‘row ‘ in host memory. Here row means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 451: `pending: list[tuple[tuple[int, int, int, int], int]] = []` This assigns ‘[]‘ to ‘pending‘ in host memory. Here pending means the value named ’pending’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 453: `def flush() -> None:` This starts function flush. Its indented body performs flush as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 454: `nonlocal pending` Inside the function flush, which handles flush, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are nonlocal: the value named ’nonlocal’ in this block; pending: the value named ’pending’ in this block.

Line 455: `if not pending:` This executes the following indented host-code branch only when ‘not pending‘ is true. The important operands are not: the value named ’not’ in this block; pending: the value named ’pending’ in this block. Context: the function flush, which handles flush.

Line 456: `return` This returns “ to the caller of the function flush, which handles flush, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 457: `left = [int(output[address]) for address, _ in pending]` This assigns ‘[int(output[address]) for address, _ in pending]‘ to ‘left ‘ in host memory. Here left means the value named ’left’ in this block. The important operands are output: the value named ’output’ in this block; address: the value named ’address’ in this block; for: the value named ’for’ in this block. Context: the function flush, which handles flush.

Line 458: `right = [value for _, value in pending]` This assigns ‘[value for _, value in pending]‘ to ‘right ‘ in host memory. Here right means the value named ’right’ in this block. The important operands are value: the value named ’value’ in this block; for: the value named ’for’ in this block; _: the value named ” in this block. Context: the function flush, which handles flush.

Line 459: `left.extend([0] * (8 - len(pending)))` This calls ‘left.extend‘ with ‘[0] * (8 - len(pending))‘ inside the function flush, which handles flush. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 460: `right.extend([0] * (8 - len(pending)))` This calls ‘right.extend‘ with ‘[0] * (8 - len(pending))‘ inside the function flush, which handles flush. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 461: `summed = _execute_vector8(` This statement continues through source line 467; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)‘ to ‘summed ‘ in host memory. Here summed means the value named ’summed’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function flush, which handles flush.

Line 462: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 461. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)‘ to ‘summed ‘ in host memory. Here summed means the value named ’summed’

in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function flush, which handles flush.

Line 463: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 461. Its exact fragment is 'VectorOperation.ADD_SAT,'. The important operands are VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named 'add sat' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function flush, which handles flush.

Line 464: `left,` This is continuation line 4 of the 7-line logical statement that begins at source line 461. Its exact fragment is 'left,'. The important operands are left: the value named 'left' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function flush, which handles flush.

Line 465: `right,` This is continuation line 5 of the 7-line logical statement that begins at source line 461. Its exact fragment is 'right,'. The important operands are right: the value named 'right' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function flush, which handles flush.

Line 466: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 461. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function flush, which handles flush.

Line 467: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 461. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function flush, which handles flush.

Line 468: `for lane, (address, _) in enumerate(pending):` This iterates lane, (address, _) over 'enumerate(pending)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function flush, which handles flush.

Line 469: `output[address] = summed[lane]` This assigns 'summed[lane]' to 'output[address] ' in host memory. Here output means the value named 'output' in this block. The important operands are summed: the value named 'summed' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function flush, which handles flush.

Line 470: `pending = []` This assigns '[]' to 'pending ' in host memory. Here pending means the value named 'pending' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function flush, which handles flush.

Line 472: `for batch_index in range(batch):` This iterates batch_index over ‘range(batch)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 473: `for output_y in range(output_height):` This iterates output_y over ‘range(output_height)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 474: `for output_x in range(output_width):` This iterates output_x over ‘range(output_width)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 475: `column = 0` This assigns ‘0‘ to ‘column ‘ in host memory. Here column means one output-feature position processed in parallel. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 476: `for channel in range(channels):` This iterates channel over ‘range(channels)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 477: `for kernel_y in range(kernel_h):` This iterates kernel_y over ‘range(kernel_h)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 478: `input_y = output_y * stride_h - pad_h + kernel_y * dilation_h` This assigns ‘output_y * stride_h - pad_h + kernel_y * dilation_h‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride_h: the value named ’stride h’ in this block; pad_h: the value named ’pad h’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 479: `for kernel_x in range(kernel_w):` This iterates kernel_x over ‘range(kernel_w)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 480: `input_x = output_x * stride_w - pad_w + kernel_x * dilation_w` This assigns ‘output_x * stride_w - pad_w + kernel_x * dilation_w‘ to ‘input_x ‘ in host memory. Here input_x means the value named ’input x’ in this block. The important operands are output_x: the value named ’output x’ in this block; stride_w: the value named ’stride w’ in this block; pad_w: the value named ’pad w’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 481: `if 0 <= input_y < input_height and 0 <= input_x < input_width:` This executes the following indented host-code branch only when ‘0 <= input_y < input_height and 0 <= input_x < input_width‘ is true. The important operands are input_y: the value named ’input y’ in this block; input_height: the value named ’input height’ in this block; and: the value named ’and’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 482: `address = (batch_index, channel, input_y, input_x)` This assigns ‘(batch_index, channel, input_y, input_x)‘ to ‘address ‘ in host memory. Here address means the value named ’address’ in this block. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named ’channel’ in this block; input_y: the value named ’input y’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 483: `if len(pending) == 8 or any(item[0] == address for item in pending):` This executes the following indented host-code branch only when ‘len(pending) == 8 or any(item[0] == address for item in pending)‘ is true. The important operands are len: the value named ’len’ in this block; pending: the value named ’pending’ in this block; or: the value named ’or’ in this block. Context: the function execute_col2im_accumulate, which handles execute col2im accumulate.

Line 484: `flush()` This calls ‘flush‘ with “ inside the function execute_col2im_accumulate, which handles execute col2im accumulate. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 485: `pending.append((address, int(patches[row, column])))` This calls ‘pending.append‘ with ‘(address, int(patches[row, column]))‘ inside the function execute_col2im_accumulate, which handles execute col2im accumulate. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 486: `column += 1` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are column: one output-feature position processed in parallel.

Line 487: `row += 1` Inside the function execute_col2im_accumulate, which handles execute col2im accumulate, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are row: one output-row position processed by the tensor tile.

Line 488: `flush()` This calls ‘flush‘ with “ inside the function execute_col2im_accumulate, which handles execute col2im accumulate. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 489: `return output` This returns ‘output‘ to the caller of the function execute_col2im_accumulate, which handles execute col2im accumulate, ending this invocation. The important operands are output: the value named ’output’ in this block.

Line 492: `def _tile_starts(size: int) -> range:` This starts function _tile_starts. Its indented body performs tile starts as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 493: `return range(0, size, TILE)` This returns ‘range(0, size, TILE)‘ to the caller of the function _tile_starts, which handles tile starts, ending this invocation. The important operands are range: the value named ’range’ in this block; size: the value named ’size’ in this block; TILE: the value named ’tile’ in this block.

Line 496: `def compile_linear_backward(` This starts function compile_linear_backward. Its indented body performs compile linear backward as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 497: `activations,` Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 498: `weights,` Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are weights: a learned coefficient multiplied by an activation.

Line 499: `output_gradients,` Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_gradients: the signed loss derivative used to update a learned parameter.

Line 500: `*,` Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 501: `input_gradient_shift: int = 0,` This assigns ‘0,‘ to ‘input_gradient_shift‘ in host memory. Here input_gradient_shift means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_linear_backward, which handles compile linear backward.

Line 502: `weight_gradient_shift: int = 0,` This assigns ‘0,‘ to ‘weight_gradient_shift‘ in host memory. Here weight_gradient_shift means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_linear_backward, which handles compile linear backward.

Line 503: `) -> CompiledLinearBackward:` Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are CompiledLinearBackward: the value named ’compiled linear backward’ in this block.

Line 504: `"""Compile exact dX and dW passes for matrices of any positive shape."""` This is executable documentation attached to the function compile_linear_backward, which handles compile linear backward. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 506: `x = _signed_matrix("activations", activations, 16)` This assigns ‘_signed_matrix("activations", activations, 16)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are _signed_matrix: the value named ’signed matrix’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function compile_linear_backward, which handles compile linear backward.

Line 507: `w = _signed_matrix("weights", weights, 16)` This assigns ‘_signed_matrix("weights", weights, 16)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are _signed_matrix: the value named ’signed matrix’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function compile_linear_backward, which handles compile linear backward.

Line 508: `dy = _signed_matrix("output_gradients", output_gradients, 16)` This assigns ‘_signed_matrix("output_grad output_gradients, 16)‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are _signed_matrix: the value named ’signed matrix’ in this block; output_gradients: the signed loss derivative used to update a learned parameter. Context: the function compile_linear_backward, which handles compile linear backward.

Line 509: `rows, inputs = x.shape` This assigns ‘x.shape‘ to ‘rows, inputs ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are x: the value named ’x’ in this block; shape: the value named ’shape’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 510: `weight_inputs, outputs = w.shape` This assigns ‘w.shape‘ to ‘weight_inputs, outputs ‘ in host memory. Here weight_inputs means a learned coefficient multiplied by an activation. The important operands are w: the value named ’w’ in this block; shape: the value named ’shape’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 511: `if weight_inputs != inputs or dy.shape != (rows, outputs):` This executes the following indented host-code branch only when ‘weight_inputs != inputs or dy.shape != (rows, outputs)‘ is true. The important operands are weight_inputs: a learned coefficient multiplied by an activation; inputs: the value named ’inputs’ in this block; or: the value named ’or’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 512: `raise ValueError("linear backward matrix dimensions do not align")` This rejects an invalid state by raising ‘ValueError("linear backward matrix dimensions do not align")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 513: `if not 0 <= input_gradient_shift <= 31:` This executes the following indented host-code branch only when ‘not 0 <= input_gradient_shift <= 31‘ is true. The important operands are not: the value named ’not’ in this block; input_gradient_shift: the signed loss derivative used to update a learned parameter. Context: the function compile_linear_backward, which handles compile linear backward.

Line 514: `raise ValueError("input_gradient_shift must be in [0,31]")` This rejects an invalid state by raising ‘ValueError("input_gradient_shift must be in [0,31]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 515: `if not 0 <= weight_gradient_shift <= 31:` This executes the following indented host-code branch only when ‘not 0 <= weight_gradient_shift <= 31‘ is true. The important operands are not: the value named ’not’ in this block; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function compile_linear_backward, which handles compile linear backward.

Line 516: `raise ValueError("weight_gradient_shift must be in [0,31]")` This rejects an invalid state by raising ‘ValueError("weight_gradient_shift must be in [0,31]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 518: `instructions: list[BackwardInstruction] = []` This assigns ‘[]‘ to ‘instructions‘ in host memory. Here instructions means the value named ’instructions’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_linear_backward, which handles compile linear backward.

Line 519: `input_bytes = 0` This assigns ‘0‘ to ‘input_bytes ‘ in host memory. Here input_bytes means the value named ’input bytes’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_linear_backward, which handles compile linear backward.

Line 521: `# dX tiles retain one MxK accumulator while traversing every N chunk.` The comment records this intent: dX tiles retain one MxK accumulator while traversing every N chunk. It applies inside the function compile_linear_backward, which handles compile linear backward.

Line 522: `for row_start in _tile_starts(rows):` This iterates row_start over ‘_tile_starts(rows)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_linear_backward, which handles compile linear backward.

Line 523: `row_end = min(row_start + TILE, rows)` This assigns ‘min(row_start + TILE, rows)‘ to ‘row_end ‘ in host memory. Here row_end means one output-row position processed by the tensor tile. The important operands are min: the value named ’min’ in this block; row_start: one output-row position processed by the tensor tile; TILE: the value named ’tile’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 524: `for input_start in _tile_starts(inputs):` This iterates input_start over ‘_tile_starts(inputs)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_linear_backward, which handles compile linear backward.

Line 525: `input_end = min(input_start + TILE, inputs)` This assigns ‘min(input_start + TILE, inputs)‘ to ‘input_end ‘ in host memory. Here input_end means the value named ’input end’ in this block. The important operands are min: the value named ’min’ in this block; input_start: the value named ’input start’ in this block; TILE: the value named ’tile’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 526: `output_starts = tuple(_tile_starts(outputs))` This assigns ‘tuple(_tile_starts(outputs))‘ to ‘output_starts ‘ in host memory. Here output_starts means the value named ’output starts’ in this block. The important operands are tuple: the value named ’tuple’ in this block; _tile_starts: the value named ’tile starts’ in this block; outputs: the value named ’outputs’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 527: `for chunk_index, output_start in enumerate(output_starts):` This iterates chunk_index, output_start over ‘enumerate(output_starts)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_linear_backward, which handles compile linear backward.

Line 528: `output_end = min(output_start + TILE, outputs)` This assigns ‘min(output_start + TILE, outputs)‘ to ‘output_end ‘ in host memory. Here output_end means the value named ’output end’ in this block. The important operands are min: the value named ’min’ in this block; output_start: the value named ’output start’ in this block; TILE: the value named ’tile’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 529: `final = chunk_index == len(output_starts) - 1` Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are final: the value named ’final’ in this block; chunk_index: an integer selecting one tensor element, lane, bank, or state entry; len: the value named ’len’ in this block.

Line 530: `x_tile = x[row_start:row_end, input_start:input_end]` This assigns ‘x[row_start:row_end, input_start:input_ to ‘x_tile ‘ in host memory. Here x_tile means the value named ’x tile’ in this block. The important operands are x: the value named ’x’ in this block; row_start: one output-row position processed by the tensor tile; row_end: one output-row position processed by the tensor tile. Context: the function compile_linear_backward, which handles compile linear backward.

Line 531: `w_tile = w[input_start:input_end, output_start:output_end]` This assigns ‘w[input_start:input_end, output_start:output_end]‘ to ‘w_tile ‘ in host memory. Here w_tile means the value named ’w tile’ in this block. The important operands are w: the value named ’w’ in this block; input_start: the value named ’input start’ in this block; input_end: the value named ’input end’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 532: `dy_tile = dy[row_start:row_end, output_start:output_end]` This assigns ‘dy[row_start:row_end, output_start:output_end]‘ to ‘dy_tile ‘ in host memory. Here dy_tile means the value named ’dy tile’ in this block. The important operands are dy: the value named ’dy’ in this block; row_start: one output-row position processed by the tensor tile; row_end: one output-row position processed by the tensor tile. Context: the function compile_linear_backward, which handles compile linear backward.

Line 533: `beats = pack_linear_backward_beats(` This statement continues through source line 538; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.INPUT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 534: `x_tile.tolist(),` This is continuation line 2 of the 6-line logical statement that begins at source line 533. Its exact fragment is ‘x_tile.tolist(),‘. The important operands are x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(

x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.INPUT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 535: `w_tile.tolist(),` This is continuation line 3 of the 6-line logical statement that begins at source line 533. Its exact fragment is ‘w_tile.tolist(),‘. The important operands are w_tile: the value named ’w tile’ in this block; tolist: the value named ’tolist’ in this block. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.INPUT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 536: `dy_tile.tolist(),` This is continuation line 4 of the 6-line logical statement that begins at source line 533. Its exact fragment is ‘dy_tile.tolist(),‘. The important operands are dy_tile: the value named ’dy tile’ in this block; tolist: the value named ’tolist’ in this block. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.INPUT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 537: `mode=LinearGradientMode.INPUT,` This is continuation line 5 of the 6-line logical statement that begins at source line 533. Its exact fragment is ‘mode=LinearGradientMode.INPUT,‘. The important operands are mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; INPUT: the value named ’input’ in this block. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.INPUT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 538: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 533. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.INPUT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 539: `record = ProgramRecord(` This statement continues through source line 552; the explanation uses the complete expression rather than only the first visual fragment. Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named ’record’ in this block; ProgramRecord: the value named ’program record’ in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 540: `linear_backward_command(` This is continuation line 2 of the 14-line logical statement that begins at source line 539. Its exact fragment is ‘linear_backward_command(‘. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named ’record’ in this block; ProgramRecord: the value named ’program record’ in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 541: `row_end - row_start,` This is continuation line 3 of the 14-line logical statement that begins at source line 539. Its exact fragment is 'row_end - row_start,'. The important operands are row_end: one output-row position processed by the tensor tile; row_start: one output-row position processed by the tensor tile. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 542: `input_end - input_start,` This is continuation line 4 of the 14-line logical statement that begins at source line 539. Its exact fragment is 'input_end - input_start,'. The important operands are input_end: the value named 'input end' in this block; input_start: the value named 'input start' in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 543: `output_end - output_start,` This is continuation line 5 of the 14-line logical statement that begins at source line 539. Its exact fragment is 'output_end - output_start,'. The important operands are output_end: the value named 'output end' in this block; output_start: the value named 'output start' in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 544: `input_gradient_shift=input_gradient_shift,` This is continuation line 6 of the 14-line logical statement that begins at source line 539. Its exact fragment is 'input_gradient_shift=input_gradient_shift,'. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 545: `weight_gradient_shift=weight_gradient_shift,` This is continuation line 7 of the 14-line logical statement that begins at source line 539. Its exact fragment is 'weight_gradient_shift=weight_gradient_shift,'. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 546: `mode=LinearGradientMode.INPUT,` This is continuation line 8 of the 14-line logical statement that begins at source line 539. Its exact fragment is 'mode=LinearGradientMode.INPUT,'. The important operands are mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; INPUT: the value named 'input' in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 547: `accumulate=chunk_index != 0,` This is continuation line 9 of the 14-line logical statement that begins at source line 539. Its exact fragment is ‘accumulate=chunk_index != 0,‘. The important operands are accumulate: the value named ’accumulate’ in this block; chunk_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named ’record’ in this block; ProgramRecord: the value named ’program record’ in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 548: `suppress_output=not final,` This is continuation line 10 of the 14-line logical statement that begins at source line 539. Its exact fragment is ‘suppress_output=not final,‘. The important operands are suppress_output: the value named ’suppress output’ in this block; not: the value named ’not’ in this block; final: the value named ’final’ in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named ’record’ in this block; ProgramRecord: the value named ’program record’ in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 549: `),` This is continuation line 11 of the 14-line logical statement that begins at source line 539. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named ’record’ in this block; ProgramRecord: the value named ’program record’ in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 550: `beats,` This is continuation line 12 of the 14-line logical statement that begins at source line 539. Its exact fragment is ‘beats,‘. The important operands are beats: the value named ’beats’ in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named ’record’ in this block; ProgramRecord: the value named ’program record’ in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 551: `4 * (row_end - row_start) if final else 0,` This is continuation line 13 of the 14-line logical statement that begins at source line 539. Its exact fragment is ‘4 * (row_end - row_start) if final else 0,‘. The important operands are row_end: one output-row position processed by the tensor tile; row_start: one output-row position processed by the tensor tile; final: the value named ’final’ in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named ’record’ in this block; ProgramRecord: the value named ’program record’ in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 552: `)` This is continuation line 14 of the 14-line logical statement that begins at source line 539. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named ’record’ in this block; ProgramRecord: the value named ’program record’ in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 553: `instructions.append(BackwardInstruction(` This statement continues through source line 564; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function

compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 554: `record,` This is continuation line 2 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘record,‘. The important operands are record: the value named ’record’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 555: `LinearGradientMode.INPUT,` This is continuation line 3 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘LinearGradientMode.INPUT,‘. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter; INPUT: the value named ’input’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 556: `row_start,` This is continuation line 4 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘row_start,‘. The important operands are row_start: one output-row position processed by the tensor tile. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 557: `input_start,` This is continuation line 5 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘input_start,‘. The important operands are input_start: the value named ’input start’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 558: `output_start,` This is continuation line 6 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘output_start,‘. The important operands are output_start: the value named ’output start’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 559: `row_end - row_start,` This is continuation line 7 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘row_end - row_start,‘. The important operands are row_end: one output-row position processed by the tensor tile; row_start: one output-row position processed by the tensor tile. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 560: `input_end - input_start,` This is continuation line 8 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘input_end - input_start,‘. The important operands are input_end: the value named

'input end' in this block; input_start: the value named 'input start' in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 561: `output_end - output_start,` This is continuation line 9 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘output_end - output_start,‘. The important operands are output_end: the value named 'output end' in this block; output_start: the value named 'output start' in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 562: `chunk_index != 0,` This is continuation line 10 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘chunk_index != 0,‘. The important operands are chunk_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 563: `not final,` This is continuation line 11 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘not final,‘. The important operands are not: the value named 'not' in this block; final: the value named 'final' in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 564: `))` This is continuation line 12 of the 12-line logical statement that begins at source line 553. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.INPUT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 565: `input_bytes += len(beats) * 8` Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_bytes: the value named 'input bytes' in this block; len: the value named 'len' in this block; beats: the value named 'beats' in this block.

Line 567: `# dW tiles retain one KxN accumulator while traversing every M chunk.` The comment records this intent: dW tiles retain one KxN accumulator while traversing every M chunk. It applies inside the function compile_linear_backward, which handles compile linear backward.

Line 568: `for input_start in _tile_starts(inputs):` This iterates input_start over ‘_tile_starts(inputs)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_linear_backward, which handles compile linear backward.

Line 569: `input_end = min(input_start + TILE, inputs)` This assigns ‘min(input_start + TILE, inputs)‘ to ‘input_end ‘ in host memory. Here input_end means the value named ’input end’ in this block. The important operands are min: the value named ’min’ in this block; input_start: the value named ’input start’ in this block; TILE: the value named ’tile’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 570: `for output_start in _tile_starts(outputs):` This iterates output_start over ‘_tile_starts(outputs)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_linear_backward, which handles compile linear backward.

Line 571: `output_end = min(output_start + TILE, outputs)` This assigns ‘min(output_start + TILE, outputs)‘ to ‘output_end ‘ in host memory. Here output_end means the value named ’output end’ in this block. The important operands are min: the value named ’min’ in this block; output_start: the value named ’output start’ in this block; TILE: the value named ’tile’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 572: `row_starts = tuple(_tile_starts(rows))` This assigns ‘tuple(_tile_starts(rows))‘ to ‘row_starts ‘ in host memory. Here row_starts means one output-row position processed by the tensor tile. The important operands are tuple: the value named ’tuple’ in this block; _tile_starts: the value named ’tile starts’ in this block; rows: one output-row position processed by the tensor tile. Context: the function compile_linear_backward, which handles compile linear backward.

Line 573: `for chunk_index, row_start in enumerate(row_starts):` This iterates chunk_index, row_start over ‘enumerate(row_starts)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_linear_backward, which handles compile linear backward.

Line 574: `row_end = min(row_start + TILE, rows)` This assigns ‘min(row_start + TILE, rows)‘ to ‘row_end ‘ in host memory. Here row_end means one output-row position processed by the tensor tile. The important operands are min: the value named ’min’ in this block; row_start: one output-row position processed by the tensor tile; TILE: the value named ’tile’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 575: `final = chunk_index == len(row_starts) - 1` Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are final: the value named ’final’ in this block; chunk_index: an integer selecting one tensor element, lane, bank, or state entry; len: the value named ’len’ in this block.

Line 576: `x_tile = x[row_start:row_end, input_start:input_end]` This assigns ‘x[row_start:row_end, input_start:input_ to ‘x_tile ‘ in host memory. Here x_tile means the value named ’x tile’ in this block. The important operands are x: the value named ’x’ in this block; row_start: one output-row position processed by the tensor tile; row_end: one output-row position processed by the tensor tile. Context: the function compile_linear_backward, which handles compile linear backward.

Line 577: `w_tile = w[input_start:input_end, output_start:output_end]` This assigns ‘w[input_start:input_end, output_start:output_end]‘ to ‘w_tile ‘ in host memory. Here w_tile means the value named ’w tile’ in this block. The important operands are w: the value named ’w’ in this block; input_start: the value named ’input start’ in this block; input_end: the value named ’input end’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 578: `dy_tile = dy[row_start:row_end, output_start:output_end]` This assigns ‘dy[row_start:row_end, output_start:output_end]‘ to ‘dy_tile ‘ in host memory. Here dy_tile means the value named ’dy tile’ in this block. The important operands are dy: the value named ’dy’ in this block; row_start: one output-row position processed by the tensor tile; row_end: one output-row position processed by the tensor tile. Context: the function compile_linear_backward, which handles compile linear backward.

Line 579: `beats = pack_linear_backward_beats(` This statement continues through source line 584; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.WEIGHT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 580: `x_tile.tolist(),` This is continuation line 2 of the 6-line logical statement that begins at source line 579. Its exact fragment is ‘x_tile.tolist(),‘. The important operands are x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.WEIGHT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 581: `w_tile.tolist(),` This is continuation line 3 of the 6-line logical statement that begins at source line 579. Its exact fragment is ‘w_tile.tolist(),‘. The important operands are w_tile: the value named ’w tile’ in this block; tolist: the value named ’tolist’ in this block. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.WEIGHT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 582: `dy_tile.tolist(),` This is continuation line 4 of the 6-line logical statement that begins at source line 579. Its exact fragment is ‘dy_tile.tolist(),‘. The important operands are dy_tile: the value named ’dy tile’ in this block; tolist: the value named ’tolist’ in this block. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.WEIGHT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 583: `mode=LinearGradientMode.WEIGHT,` This is continuation line 5 of the 6-line logical statement that begins at source line 579. Its exact fragment is ‘mode=LinearGradientMode.WEIGHT,‘. The important operands are mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.WEIGHT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 584: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 579. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats(x_tile.tolist(), w_tile.tolist(), dy_tile.tolist(), mode=LinearGradientMode.WEIGHT,)‘ to ‘beats ‘ in host memory. Here beats means the value named ’beats’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; x_tile: the value named ’x tile’ in this block; tolist: the value named ’tolist’ in this block. Context: the function compile_linear_backward, which handles compile linear backward.

Line 585: `record = ProgramRecord(` This statement continues through source line 598; the explanation uses the complete expression rather than only the first visual fragment. Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named ’record’ in this block; ProgramRecord: the value named ’program record’ in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 586: `linear_backward_command(` This is continuation line 2 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'linear_backward_command('. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 587: `row_end - row_start,` This is continuation line 3 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'row_end - row_start,'. The important operands are row_end: one output-row position processed by the tensor tile; row_start: one output-row position processed by the tensor tile. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 588: `input_end - input_start,` This is continuation line 4 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'input_end - input_start,'. The important operands are input_end: the value named 'input end' in this block; input_start: the value named 'input start' in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 589: `output_end - output_start,` This is continuation line 5 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'output_end - output_start,'. The important operands are output_end: the value named 'output end' in this block; output_start: the value named 'output start' in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 590: `input_gradient_shift=input_gradient_shift,` This is continuation line 6 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'input_gradient_shift=input_gradient_shift,'. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 591: `weight_gradient_shift=weight_gradient_shift,` This is continuation line 7 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'weight_gradient_shift=weight_gradient_shift,'. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 592: `mode=LinearGradientMode.WEIGHT,` This is continuation line 8 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'mode=LinearGradientMode.WEIGHT,'. The important operands are mode: the

value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 593: `accumulate=chunk_index != 0,` This is continuation line 9 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'accumulate=chunk_index != 0,'. The important operands are accumulate: the value named 'accumulate' in this block; chunk_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 594: `suppress_output=not final,` This is continuation line 10 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'suppress_output=not final,'. The important operands are suppress_output: the value named 'suppress output' in this block; not: the value named 'not' in this block; final: the value named 'final' in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 595: `),` This is continuation line 11 of the 14-line logical statement that begins at source line 585. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 596: `beats,` This is continuation line 12 of the 14-line logical statement that begins at source line 585. Its exact fragment is 'beats,'. The important operands are beats: the value named 'beats' in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 597: `3 * (input_end - input_start) if final else 0,` This is continuation line 13 of the 14-line logical statement that begins at source line 585. Its exact fragment is '3 * (input_end - input_start) if final else 0,'. The important operands are input_end: the value named 'input end' in this block; input_start: the value named 'input start' in this block; final: the value named 'final' in this block. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 598: `)` This is continuation line 14 of the 14-line logical statement that begins at source line 585. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer

programs for hardware-resident supervised updates. The important operands are record: the value named 'record' in this block; ProgramRecord: the value named 'program record' in this block; linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 599: `instructions.append(BackwardInstruction(` This statement continues through source line 610; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 600: `record,` This is continuation line 2 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘record,‘. The important operands are record: the value named 'record' in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 601: `LinearGradientMode.WEIGHT,` This is continuation line 3 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘LinearGradientMode.WEIGHT,‘. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 602: `row_start,` This is continuation line 4 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘row_start,‘. The important operands are row_start: one output-row position processed by the tensor tile. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 603: `input_start,` This is continuation line 5 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘input_start,‘. The important operands are input_start: the value named 'input start' in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 604: `output_start,` This is continuation line 6 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘output_start,‘. The important operands are output_start: the value named 'output start' in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 605: `row_end - row_start,` This is continuation line 7 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘row_end - row_start,‘. The important operands are row_end: one output-row position processed by the tensor tile; row_start: one output-row position processed by the tensor tile. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT,

row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 606: `input_end - input_start,` This is continuation line 8 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘input_end - input_start,‘. The important operands are input_end: the value named ’input end’ in this block; input_start: the value named ’input start’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 607: `output_end - output_start,` This is continuation line 9 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘output_end - output_start,‘. The important operands are output_end: the value named ’output end’ in this block; output_start: the value named ’output start’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 608: `chunk_index != 0,` This is continuation line 10 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘chunk_index != 0,‘. The important operands are chunk_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 609: `not final,` This is continuation line 11 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘not final,‘. The important operands are not: the value named ’not’ in this block; final: the value named ’final’ in this block. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 610: `))` This is continuation line 12 of the 12-line logical statement that begins at source line 599. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘instructions.append‘ with ‘BackwardInstruction(record, LinearGradientMode.WEIGHT, row_start, input_start, output_start, row_end - row_start, input_end - input_start, output_end - output_start, chunk_index != 0, not final,)‘ inside the function compile_linear_backward, which handles compile linear backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 611: `input_bytes += len(beats) * 8` Inside the function compile_linear_backward, which handles compile linear backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are input_bytes: the value named ’input bytes’ in this block; len: the value named ’len’ in this block; beats: the value named ’beats’ in this block.

Line 613: `return CompiledLinearBackward(` This statement continues through source line 620; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘CompiledLinearBackward((rows, inputs, outputs), input_gradient_shift, weight_gradient_shift, tuple(instructions), input_bytes, 2 * rows * inputs * outputs,)‘

to the caller of the function compile_linear_backward, which handles compile linear backward, ending this invocation. The important operands are CompiledLinearBackward: the value named 'compiled linear backward' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 614: `(rows, inputs, outputs),` This is continuation line 2 of the 8-line logical statement that begins at source line 613. Its exact fragment is '(rows, inputs, outputs),'. The important operands are rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block; outputs: the value named 'outputs' in this block. Read together, the complete operation is: This returns 'CompiledLinearBackward((rows, inputs, outputs), input_gradient_shift, weight_gradient_shift, tuple(instructions), input_bytes, 2 * rows * inputs * outputs,)' to the caller of the function compile_linear_backward, which handles compile linear backward, ending this invocation. The important operands are CompiledLinearBackward: the value named 'compiled linear backward' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 615: `input_gradient_shift,` This is continuation line 3 of the 8-line logical statement that begins at source line 613. Its exact fragment is 'input_gradient_shift,'. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This returns 'CompiledLinearBackward((rows, inputs, outputs), input_gradient_shift, weight_gradient_shift, tuple(instructions), input_bytes, 2 * rows * inputs * outputs,)' to the caller of the function compile_linear_backward, which handles compile linear backward, ending this invocation. The important operands are CompiledLinearBackward: the value named 'compiled linear backward' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 616: `weight_gradient_shift,` This is continuation line 4 of the 8-line logical statement that begins at source line 613. Its exact fragment is 'weight_gradient_shift,'. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Read together, the complete operation is: This returns 'CompiledLinearBackward((rows, inputs, outputs), input_gradient_shift, weight_gradient_shift, tuple(instructions), input_bytes, 2 * rows * inputs * outputs,)' to the caller of the function compile_linear_backward, which handles compile linear backward, ending this invocation. The important operands are CompiledLinearBackward: the value named 'compiled linear backward' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 617: `tuple(instructions),` This is continuation line 5 of the 8-line logical statement that begins at source line 613. Its exact fragment is 'tuple(instructions),'. The important operands are tuple: the value named 'tuple' in this block; instructions: the value named 'instructions' in this block. Read together, the complete operation is: This returns 'CompiledLinearBackward((rows, inputs, outputs), input_gradient_shift, weight_gradient_shift, tuple(instructions), input_bytes, 2 * rows * inputs * outputs,)' to the caller of the function compile_linear_backward, which handles compile linear backward, ending this invocation. The important operands are CompiledLinearBackward: the value named 'compiled linear backward' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 618: `input_bytes,` This is continuation line 6 of the 8-line logical statement that begins at source line 613. Its exact fragment is 'input_bytes,'. The important operands are input_bytes: the value named 'input bytes' in this block. Read together, the complete operation is: This returns 'CompiledLinearBackward((rows, inputs, outputs), input_gradient_shift, weight_gradient_shift, tuple(instructions), input_bytes, 2 * rows * inputs * outputs,)' to the caller of the function compile_linear_backward, which handles compile linear backward, ending this invocation. The important operands are CompiledLinearBackward: the value named 'compiled linear backward' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 619: `2 * rows * inputs * outputs,` This is continuation line 7 of the 8-line logical statement that begins at source line 613. Its exact fragment is '2 * rows * inputs * outputs,'. The important operands are rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block; outputs: the value named 'outputs' in this block. Read together, the complete operation is: This returns 'CompiledLinearBackward((rows, inputs, outputs), input_gradient_shift, weight_gradient_shift, tuple(instructions), input_bytes, 2 * rows * inputs * outputs,)' to the caller of the function compile_linear_backward, which handles compile linear backward, ending this invocation. The important operands are CompiledLinearBackward: the value named 'compiled linear backward' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 620: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 613. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘CompiledLinearBackward((rows, inputs, outputs), input_gradient_shift, weight_gradient_shift, tuple(instructions), input_bytes, 2 * rows * inputs * outputs,)‘ to the caller of the function compile_linear_backward, which handles compile linear backward, ending this invocation. The important operands are CompiledLinearBackward: the value named ’compiled linear backward’ in this block; rows: one output-row position processed by the tensor tile; inputs: the value named ’inputs’ in this block.

Line 623: `def compile_affine_backward(` This starts function compile_affine_backward. Its indented body performs compile affine backward as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 624: `activations,` Inside the function compile_affine_backward, which handles compile affine backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 625: `weights,` Inside the function compile_affine_backward, which handles compile affine backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are weights: a learned coefficient multiplied by an activation.

Line 626: `bias,` Inside the function compile_affine_backward, which handles compile affine backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are bias: an additive learned offset applied after accumulation.

Line 627: `output_gradients,` Inside the function compile_affine_backward, which handles compile affine backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_gradients: the signed loss derivative used to update a learned parameter.

Line 628: `*,` Inside the function compile_affine_backward, which handles compile affine backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 629: `input_gradient_shift: int = 0,` This assigns ‘0,‘ to ‘input_gradient_shift‘ in host memory. Here input_gradient_shift means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_affine_backward, which handles compile affine backward.

Line 630: `weight_gradient_shift: int = 0,` This assigns ‘0,‘ to ‘weight_gradient_shift‘ in host memory. Here weight_gradient_shift means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_affine_backward, which handles compile affine backward.

Line 631: `) -> CompiledAffineBackward:` Inside the function compile_affine_backward, which handles compile affine backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are CompiledAffineBackward: the value named ’compiled affine backward’ in this block.

Line 632: `"""Compile weight, bias, and input gradients for "Y = XW + b"."""` This is executable documentation attached to the function compile_affine_backward, which handles compile affine backward. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 634: `x = _signed_matrix("activations", activations, 16)` This assigns '_signed_matrix("activations", activations, 16)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are _signed_matrix: the value named 'signed matrix' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function compile_affine_backward, which handles compile affine backward.

Line 635: `w = _signed_matrix("weights", weights, 16)` This assigns '_signed_matrix("weights", weights, 16)' to 'w ' in host memory. Here w means the value named 'w' in this block. The important operands are _signed_matrix: the value named 'signed matrix' in this block; weights: a learned coefficient multiplied by an activation. Context: the function compile_affine_backward, which handles compile affine backward.

Line 636: `bias_array = np.asarray(bias)` This assigns 'np.asarray(bias)' to 'bias_array ' in host memory. Here bias_array means an additive learned offset applied after accumulation. The important operands are asarray: the value named 'asarray' in this block; bias: an additive learned offset applied after accumulation. Context: the function compile_affine_backward, which handles compile affine backward.

Line 637: `if bias_array.ndim != 1 or bias_array.shape[0] != w.shape[1]:` This executes the following indented host-code branch only when 'bias_array.ndim != 1 or bias_array.shape[0] != w.shape[1]' is true. The important operands are bias_array: an additive learned offset applied after accumulation; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function compile_affine_backward, which handles compile affine backward.

Line 638: `raise ValueError("bias must be one-dimensional with one value per output")` This rejects an invalid state by raising 'ValueError("bias must be one-dimensional with one value per output")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 639: `bias_matrix = _signed_matrix("bias", bias_array.reshape(1, -1), 16)` This assigns '_signed_matrix("bias", bias_array.reshape(1, -1), 16)' to 'bias_matrix ' in host memory. Here bias_matrix means an additive learned offset applied after accumulation. The important operands are _signed_matrix: the value named 'signed matrix' in this block; bias: an additive learned offset applied after accumulation; bias_array: an additive learned offset applied after accumulation. Context: the function compile_affine_backward, which handles compile affine backward.

Line 640: `if x.shape[1] != w.shape[0]:` This executes the following indented host-code branch only when 'x.shape[1] != w.shape[0]' is true. The important operands are x: the value named 'x' in this block; shape: the value named 'shape' in this block; w: the value named 'w' in this block. Context: the function compile_affine_backward, which handles compile affine backward.

Line 641: `raise ValueError("affine activation and weight dimensions do not align")` This rejects an invalid state by raising 'ValueError("affine activation and weight dimensions do not align")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 642: `augmented_x = np.concatenate(` This statement continues through source line 644; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.concatenate((x, np.ones((x.shape[0], 1), dtype=np.int64)), axis=1)' to 'augmented_x ' in host memory. Here augmented_x means the value named 'augmented x' in this block. The important operands are concatenate: the value named 'concatenate' in this block; x: the value named 'x' in this block; ones: the value named 'ones' in this block. Context: the function compile_affine_backward, which handles compile affine backward.

Line 643: `(x, np.ones((x.shape[0], 1), dtype=np.int64)), axis=1` This is continuation line 2 of the 3-line logical statement that begins at source line 642. Its exact fragment is '(x, np.ones((x.shape[0], 1), dtype=np.int64)), axis=1'. The important operands are x: the value named 'x' in this block; ones: the value named 'ones' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns 'np.concatenate((x, np.ones((x.shape[0], 1), dtype=np.int64)), axis=1)' to 'augmented_x ' in host memory. Here augmented_x means the value named 'augmented x' in this block. The important operands are concatenate: the value named 'concatenate'

in this block; x: the value named 'x' in this block; ones: the value named 'ones' in this block. Context: the function compile_affine_backward, which handles compile affine backward.

Line 644: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 642. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.concatenate((x, np.ones((x.shape[0], 1), dtype=np.int64)), axis=1)' to 'augmented_x ' in host memory. Here augmented_x means the value named 'augmented x' in this block. The important operands are concatenate: the value named 'concatenate' in this block; x: the value named 'x' in this block; ones: the value named 'ones' in this block. Context: the function compile_affine_backward, which handles compile affine backward.

Line 645: `augmented_w = np.concatenate((w, bias_matrix), axis=0)` This assigns 'np.concatenate((w, bias_matrix), axis=0)' to 'augmented_w ' in host memory. Here augmented_w means the value named 'augmented w' in this block. The important operands are concatenate: the value named 'concatenate' in this block; w: the value named 'w' in this block; bias_matrix: an additive learned offset applied after accumulation. Context: the function compile_affine_backward, which handles compile affine backward.

Line 646: `return CompiledAffineBackward(` This statement continues through source line 655; the explanation uses the complete expression rather than only the first visual fragment. This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,), x.shape[1],)' to the caller of the function compile_affine_backward, which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 647: `compile_linear_backward(` This is continuation line 2 of the 10-line logical statement that begins at source line 646. Its exact fragment is 'compile_linear_backward('. The important operands are compile_linear_backward: the value named 'compile linear backward' in this block. Read together, the complete operation is: This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gr
weight_gradient_shift=weight_gradient_shift,), x.shape[1],)' to the caller of the function compile_affine_backward, which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 648: `augmented_x,` This is continuation line 3 of the 10-line logical statement that begins at source line 646. Its exact fragment is 'augmented_x,'. The important operands are augmented_x: the value named 'augmented x' in this block. Read together, the complete operation is: This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradie
), x.shape[1],)' to the caller of the function compile_affine_backward, which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 649: `augmented_w,` This is continuation line 4 of the 10-line logical statement that begins at source line 646. Its exact fragment is 'augmented_w,'. The important operands are augmented_w: the value named 'augmented w' in this block. Read together, the complete operation is: This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,), x.shape[1],)' to the caller of the function compile_affine_backward, which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 650: `output_gradients,` This is continuation line 5 of the 10-line logical statement that begins at source line 646. Its exact fragment is 'output_gradients,'. The important operands are output_gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,), x.shape[1],)' to the caller of the function compile_affine_backward,

which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 651: `input_gradient_shift=input_gradient_shift,` This is continuation line 6 of the 10-line logical statement that begins at source line 646. Its exact fragment is 'input_gradient_shift=input_gradient_shift,'. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,), x.shape[1],)' to the caller of the function compile_affine_backward, which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 652: `weight_gradient_shift=weight_gradient_shift,` This is continuation line 7 of the 10-line logical statement that begins at source line 646. Its exact fragment is 'weight_gradient_shift=weight_gradient_shift,'. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Read together, the complete operation is: This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,), x.shape[1],)' to the caller of the function compile_affine_backward, which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 653: `),` This is continuation line 8 of the 10-line logical statement that begins at source line 646. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,), x.shape[1],)' to the caller of the function compile_affine_backward, which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 654: `x.shape[1],` This is continuation line 9 of the 10-line logical statement that begins at source line 646. Its exact fragment is 'x.shape[1],'. The important operands are x: the value named 'x' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,), x.shape[1],)' to the caller of the function compile_affine_backward, which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 655: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 646. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'CompiledAffineBackward(compile_linear_backward(augmented_x, augmented_w, output_gradients, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,), x.shape[1],)' to the caller of the function compile_affine_backward, which handles compile affine backward, ending this invocation. The important operands are CompiledAffineBackward: the value named 'compiled affine backward' in this block; compile_linear_backward: the value named 'compile linear backward' in this block; augmented_x: the value named 'augmented x' in this block.

Line 658: `def compile_conv2d_backward(` This starts function compile_conv2d_backward. Its indented body performs compile conv2d backward as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 659: `activations,` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 660: `weights,` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are weights: a learned coefficient multiplied by an activation.

Line 661: `bias,` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are bias: an additive learned offset applied after accumulation.

Line 662: `output_gradients,` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_gradients: the signed loss derivative used to update a learned parameter.

Line 663: `*,` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 664: `stride=1,` This assigns ‘1,‘ to ‘stride‘ in host memory. Here stride means the value named ’stride’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 665: `padding=0,` This assigns ‘0,‘ to ‘padding‘ in host memory. Here padding means the value named ’padding’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 666: `dilation=1,` This assigns ‘1,‘ to ‘dilation‘ in host memory. Here dilation means the value named ’dilation’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 667: `groups: int = 1,` This assigns ‘1,‘ to ‘groups‘ in host memory. Here groups means the value named ’groups’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 668: `input_gradient_shift: int = 0,` This assigns ‘0,‘ to ‘input_gradient_shift‘ in host memory. Here input_gradient_shift means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 669: `weight_gradient_shift: int = 0,` This assigns ‘0,‘ to ‘weight_gradient_shift‘ in host memory. Here weight_gradient_shift means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 670: `) -> CompiledConv2dBackward:` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are CompiledConv2dBackward: the value named ’compiled conv2d backward’ in this block.

Line 671: `"""Lower a signed NCHW convolution backward pass to exact affine tiles."""` This is executable documentation attached to the function compile_conv2d_backward, which handles compile conv2d backward. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 673: `x = np.asarray(activations)` This assigns ‘np.asarray(activations)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are asarray: the value named ’asarray’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 674: `w = np.asarray(weights)` This assigns ‘np.asarray(weights)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are asarray: the value named ’asarray’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 675: `dy = np.asarray(output_gradients)` This assigns ‘np.asarray(output_gradients)‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are asarray: the value named ’asarray’ in this block; output_gradients: the signed loss derivative used to update a learned parameter. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 676: `bias_array = np.asarray(bias)` This assigns ‘np.asarray(bias)‘ to ‘bias_array ‘ in host memory. Here bias_array means an additive learned offset applied after accumulation. The important operands are asarray: the value named ’asarray’ in this block; bias: an additive learned offset applied after accumulation. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 677: `for name, array, bits in (` This line opens or closes a multi-line Python expression inside the function compile_conv2d_backward, which handles compile conv2d backward; the neighboring lines supply its arguments or elements.

Line 678: `("activations", x, 16),` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; x: the value named ’x’ in this block.

Line 679: `("weights", w, 16),` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are weights: a learned coefficient multiplied by an activation; w: the value named ’w’ in this block.

Line 680: `("output_gradients", dy, 16),` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_gradients: the signed loss derivative used to update a learned parameter; dy: the value named ’dy’ in this block.

Line 681: `("bias", bias_array, 16),` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are bias: an additive learned offset applied after accumulation; bias_array: an additive learned offset applied after accumulation.

Line 682: `):` Inside the function compile_conv2d_backward, which handles compile conv2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 683: `if not np.issubdtype(array.dtype, np.integer):` This executes the following indented host-code branch only when ‘not np.issubdtype(array.dtype, np.integer)‘ is true. The important operands are not: the value named ’not’ in this block; issubdtype: the value named ’issubdtype’ in this block; array: the value named ’array’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 684: `raise ValueError(f"{name} must contain integers")` This rejects an invalid state by raising ‘ValueError(f"{name} must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 685: `minimum = -(1 << (bits - 1))` This assigns ‘-(1 << (bits - 1))‘ to ‘minimum ‘ in host memory. Here minimum means the value named ’minimum’ in this block. The important operands are bits: the value named ’bits’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 686: `maximum = (1 << (bits - 1)) - 1` This assigns ‘(1 << (bits - 1)) - 1‘ to ‘maximum ‘ in host memory. Here maximum means the value named ’maximum’ in this block. The important operands are bits: the value named ’bits’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 687: `if np.any(array < minimum) or np.any(array > maximum):` This executes the following indented host-code branch only when ‘np.any(array < minimum) or np.any(array > maximum)‘ is true. The important operands are any: the value named ’any’ in this block; array: the value named ’array’ in this block; minimum: the value named ’minimum’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 688: `raise ValueError(f"{name} must fit signed {bits}-bit values")` This rejects an invalid state by raising ‘ValueError(f"{name} must fit signed {bits}-bit values")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 689: `if x.ndim != 4 or w.ndim != 4 or dy.ndim != 4 or bias_array.ndim != 1:` This executes the following indented host-code branch only when ‘x.ndim != 4 or w.ndim != 4 or dy.ndim != 4 or bias_array.ndim != 1‘ is true. The important operands are x: the value named ’x’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 690: `raise ValueError("convolution operands must use NCHW/OIHW tensors and a bias vector")` This rejects an invalid state by raising ‘ValueError("convolution operands must use NCHW/OIHW tensors and a bias vector")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 691: `if groups <= 0:` This executes the following indented host-code branch only when ‘groups <= 0‘ is true. The important operands are groups: the value named ’groups’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 692: `raise ValueError("groups must be positive")` This rejects an invalid state by raising ‘ValueError("groups must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 693: `batch, input_channels, input_height, input_width = x.shape` This assigns ‘x.shape‘ to ‘batch, input_channels, input_height, input_width ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are x: the value named ’x’ in this block; shape: the value named ’shape’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 694: `output_channels, weight_inputs, kernel_h, kernel_w = w.shape` This assigns ‘w.shape‘ to ‘output_channels, weight_inputs, kernel_h, kernel_w ‘ in host memory. Here output_channels means the value named ’output channels’ in this block. The important operands are w: the value named ’w’ in this block; shape: the value named ’shape’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 695: `if input_channels % groups or output_channels % groups:` This executes the following indented host-code branch only when ‘input_channels % groups or output_channels % groups‘ is true. The important operands are input_channels: the value named ’input channels’ in this block; groups: the value named ’groups’ in this block; or: the value named ’or’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 696: `raise ValueError("input and output channels must be divisible by groups")` This rejects an invalid state by raising ‘ValueError("input and output channels must be divisible by groups")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 697: `input_channels_per_group = input_channels // groups` This assigns ‘input_channels // groups‘ to ‘input_channels_per_group ‘ in host memory. Here input_channels_per_group means the value named ’input channels per group’ in this block. The important operands are input_channels: the value named ’input channels’ in this block; groups: the value named ’groups’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 698: `output_channels_per_group = output_channels // groups` This assigns ‘output_channels // groups‘ to ‘output_channels_per_group ‘ in host memory. Here output_channels_per_group means the value named ’output channels per group’ in this block. The important operands are output_channels: the value named ’output channels’ in this block; groups: the value named ’groups’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 699: `if weight_inputs != input_channels_per_group:` This executes the following indented host-code branch only when ‘weight_inputs != input_channels_per_group‘ is true. The important operands are weight_inputs: a learned coefficient multiplied by an activation; input_channels_per_group: the value named ’input channels per group’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 700: `raise ValueError("weight input channels do not match the grouped convolution")` This rejects an invalid state by raising ‘ValueError("weight input channels do not match the grouped convolution")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 701: `if bias_array.shape != (output_channels,):` This executes the following indented host-code branch only when ‘bias_array.shape != (output_channels,)‘ is true. The important operands are bias_array: an additive learned offset applied after accumulation; shape: the value named ’shape’ in this block; output_channels: the value named ’output channels’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 702: `raise ValueError("bias must contain one value per output channel")` This rejects an invalid state by raising ‘ValueError("bias must contain one value per output channel")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 703: `stride_pair = _pair("stride", stride)` This assigns ‘_pair("stride", stride)‘ to ‘stride_pair ‘ in host memory. Here stride_pair means the value named ’stride pair’ in this block. The important operands are _pair: the value named ’pair’ in this block; stride: the value named ’stride’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 704: `dilation_pair = _pair("dilation", dilation)` This assigns ‘_pair("dilation", dilation)‘ to ‘dilation_pair ‘ in host memory. Here dilation_pair means the value named ’dilation pair’ in this block. The important operands are _pair: the value named ’pair’ in this block; dilation: the value named ’dilation’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 705: `if isinstance(padding, int):` This executes the following indented host-code branch only when ‘isinstance(padding, int)‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; padding: the value named ’padding’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 706: `padding_pair = (int(padding), int(padding))` This assigns ‘(int(padding), int(padding))‘ to ‘padding_pair ‘ in host memory. Here padding_pair means the value named ’padding pair’ in this block. The important operands are padding: the value named ’padding’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 707: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function compile_conv2d_backward, which handles compile conv2d backward.

Line 708: `padding_pair = tuple(map(int, padding))` This assigns ‘tuple(map(int, padding))‘ to ‘padding_pair ‘ in host memory. Here padding_pair means the value named ’padding pair’ in this block. The important operands are tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block; padding: the value named ’padding’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 709: `if len(padding_pair) != 2:` This executes the following indented host-code branch only when ‘len(padding_pair) != 2‘ is true. The important operands are len: the value named ’len’ in this block; padding_pair: the value named ’padding pair’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 710: `raise ValueError("padding must be an integer or a pair")` This rejects an invalid state by raising ‘ValueError("padding must be an integer or a pair")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 711: `if min(padding_pair) < 0:` This executes the following indented host-code branch only when ‘min(padding_pair) < 0‘ is true. The important operands are min: the value named ’min’ in this block; padding_pair: the value named ’padding pair’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 712: `raise ValueError("padding must be nonnegative")` This rejects an invalid state by raising ‘ValueError("padding must be nonnegative")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 713: `output_height = _conv_output_size(` This statement continues through source line 715; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_conv_output_size(input_height, kernel_h, stride_pair[0], padding_pair[0], dilation_pair[0])‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are _conv_output_size: the value named ’conv output size’ in this block; input_height: the value named ’input height’ in this block; kernel_h: the value named ’kernel h’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 714: `input_height, kernel_h, stride_pair[0], padding_pair[0], dilation_pair[0]` This is continuation line 2 of the 3-line logical statement that begins at source line 713. Its exact fragment is ‘input_height, kernel_h, stride_pair[0], padding_pair[0], dilation_pair[0]‘. The important operands are input_height: the value named ’input height’ in this block; kernel_h: the value named ’kernel h’ in this block; stride_pair: the value named ’stride pair’ in this block. Read together, the complete operation is: This assigns ‘_conv_output_size(input_height, kernel_h, stride_pair[0], padding_pair[0], dilation_pair[0])‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are _conv_output_size: the value named ’conv output size’ in this block; input_height: the value named ’input height’ in this block; kernel_h: the value named ’kernel h’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 715: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 713. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_conv_output_size(input_height, kernel_h, stride_pair[0], padding_pair[0], dilation_pair[0])‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are _conv_output_size: the value named ’conv output size’ in this block; input_height: the value named ’input height’ in this block; kernel_h: the value named ’kernel h’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 716: `output_width = _conv_output_size(` This statement continues through source line 718; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_conv_output_size(input_width, kernel_w, stride_pair[1], padding_pair[1], dilation_pair[1])‘ to ‘output_width ‘ in host memory. Here output_width means the value named ’output width’ in this block. The important operands are _conv_output_size: the value named ’conv output size’ in this block; input_width: the value named ’input width’ in this block; kernel_w: the value named ’kernel w’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 717: `input_width, kernel_w, stride_pair[1], padding_pair[1], dilation_pair[1]` This is continuation line 2 of the 3-line logical statement that begins at source line 716. Its exact fragment is ‘input_width, kernel_w, stride_pair[1], padding_pair[1], dilation_pair[1]‘. The important operands are input_width: the value named ’input width’ in this block; kernel_w: the value named ’kernel w’ in this block; stride_pair: the value named ’stride pair’ in this block. Read together, the complete operation is: This assigns ‘_conv_output_size(input_width, kernel_w, stride_pair[1], padding_pair[1], dilation_pair[1])‘ to ‘output_width ‘ in host memory. Here output_width means the value named ’output width’ in this block. The important operands are _conv_output_size: the value named ’conv output size’ in this block; input_width: the value named ’input width’ in this block; kernel_w: the value named ’kernel w’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 718: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 716. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_conv_output_size(input_width, kernel_w, stride_pair[1], padding_pair[1], dilation_pair[1])‘ to ‘output_width ‘ in host memory. Here output_width means the value named ’output width’ in this block. The important operands are _conv_output_size: the value named ’conv output size’ in this block; input_width: the value named ’input width’ in this block; kernel_w: the value named ’kernel w’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 719: `expected_output_shape = (batch, output_channels, output_height, output_width)` This assigns ‘(batch, output_channels, output_height, output_width)‘ to ‘expected_output_shape ‘ in host memory. Here expected_output_shape means the value named ’expected output shape’ in this block. The important operands are batch: the value named ’batch’ in this block; output_channels: the value named ’output channels’ in this block; output_height: the value named ’output height’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 720: `if dy.shape != expected_output_shape:` This executes the following indented host-code branch only when ‘dy.shape != expected_output_shape‘ is true. The important operands are dy: the value named ’dy’ in this block; shape: the value named ’shape’ in this block; expected_output_shape: the value named ’expected output shape’ in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 721: `raise ValueError(` This statement continues through source line 723; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘ValueError(f"output_gradients must have shape {expected_output_shape}; received {dy.shape}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 722: `f"output_gradients must have shape {expected_output_shape}; received {dy.shape}"` This is continuation line 2 of the 3-line logical statement that begins at source line 721. Its exact fragment is ‘f"output_gradients must have shape {expected_output_shape}; received {dy.shape}"‘. The important operands are f: the value named ’f’ in this block; output_gradients: the signed loss derivative used to update a learned parameter; must: the value named ’must’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘ValueError(f"output_gradients must have shape {expected_output_shape}; received {dy.shape}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 723: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 721. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This rejects an invalid state by raising ‘ValueError(f"output_gradients must have shape {expected_output_shape}; received {dy.shape}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 724: `programs: list[ConvBackwardGroup] = []` This assigns '[]' to 'programs' in host memory. Here programs means the value named 'programs' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 725: `for group in range(groups):` This iterates group over 'range(groups)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function compile_conv2d_backward, which handles compile conv2d backward.

Line 726: `input_start = group * input_channels_per_group` This assigns 'group * input_channels_per_group' to 'input_start ' in host memory. Here input_start means the value named 'input start' in this block. The important operands are group: the value named 'group' in this block; input_channels_per_group: the value named 'input channels per group' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 727: `output_start = group * output_channels_per_group` This assigns 'group * output_channels_per_group' to 'output_start ' in host memory. Here output_start means the value named 'output start' in this block. The important operands are group: the value named 'group' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 728: `patches = im2col_signed_nchw(` This statement continues through source line 734; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'im2col_signed_nchw(x[:, input_start:input_start + input_channels_per_group], kernel=(kernel_h, kernel_w), stride=stride_pair, padding=padding_pair, dilation=dilation_pair,)' to 'patches ' in host memory. Here patches means the value named 'patches' in this block. The important operands are im2col_signed_nchw: the value named 'im2col signed nchw' in this block; x: the value named 'x' in this block; input_start: the value named 'input start' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 729: `x[:, input_start:input_start + input_channels_per_group],` This is continuation line 2 of the 7-line logical statement that begins at source line 728. Its exact fragment is 'x[:, input_start:input_start + input_channels_per_group],'. The important operands are x: the value named 'x' in this block; input_start: the value named 'input start' in this block; input_channels_per_group: the value named 'input channels per group' in this block. Read together, the complete operation is: This assigns 'im2col_signed_nchw(x[:, input_start:input_start + input_channels_per_group], kernel=(kernel_h, kernel_w), stride=stride_pair, padding=padding_pair, dilation=dilation_pair,)' to 'patches ' in host memory. Here patches means the value named 'patches' in this block. The important operands are im2col_signed_nchw: the value named 'im2col signed nchw' in this block; x: the value named 'x' in this block; input_start: the value named 'input start' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 730: `kernel=(kernel_h, kernel_w),` This is continuation line 3 of the 7-line logical statement that begins at source line 728. Its exact fragment is 'kernel=(kernel_h, kernel_w),'. The important operands are kernel: the value named 'kernel' in this block; kernel_h: the value named 'kernel h' in this block; kernel_w: the value named 'kernel w' in this block. Read together, the complete operation is: This assigns 'im2col_signed_nchw(x[:, input_start:input_start + input_channels_per_group], kernel=(kernel_h, kernel_w), stride=stride_pair, padding=padding_pair, dilation=dilation_pair,)' to 'patches ' in host memory. Here patches means the value named 'patches' in this block. The important operands are im2col_signed_nchw: the value named 'im2col signed nchw' in this block; x: the value named 'x' in this block; input_start: the value named 'input start' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 731: `stride=stride_pair,` This is continuation line 4 of the 7-line logical statement that begins at source line 728. Its exact fragment is 'stride=stride_pair,'. The important operands are stride: the value named 'stride' in this block; stride_pair: the value named 'stride pair' in this block. Read together, the complete operation is: This assigns 'im2col_signed_nchw(x[:, input_start:input_start + input_channels_per_group], kernel=(kernel_h, kernel_w), stride=stride_pair, padding=padding_pair, dilation=dilation_pair,)' to 'patches ' in host memory. Here patches means the value named 'patches' in this block. The important operands are im2col_signed_nchw: the value named 'im2col signed nchw' in this block; x: the value named 'x' in this block; input_start: the value named 'input start' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 732: `padding=padding_pair,` This is continuation line 5 of the 7-line logical statement that begins at source line 728. Its exact fragment is 'padding=padding_pair,'. The important operands are padding: the value named 'padding' in this block; padding_pair: the value named 'padding pair' in this block. Read together, the complete operation is: This assigns 'im2col_signed_nchw(x[:, input_start:input_start + input_channels_per_group], kernel=(kernel_h, kernel_w), stride=stride_pair, padding=padding_pair, dilation=dilation_pair,)' to 'patches ' in host memory. Here patches means the value named 'patches' in this block. The important operands are im2col_signed_nchw: the value named 'im2col signed nchw' in this block; x: the value named 'x' in this block; input_start: the value named 'input start' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 733: `dilation=dilation_pair,` This is continuation line 6 of the 7-line logical statement that begins at source line 728. Its exact fragment is 'dilation=dilation_pair,'. The important operands are dilation: the value named 'dilation' in this block; dilation_pair: the value named 'dilation pair' in this block. Read together, the complete operation is: This assigns 'im2col_signed_nchw(x[:, input_start:input_start + input_channels_per_group], kernel=(kernel_h, kernel_w), stride=stride_pair, padding=padding_pair, dilation=dilation_pair,)' to 'patches ' in host memory. Here patches means the value named 'patches' in this block. The important operands are im2col_signed_nchw: the value named 'im2col signed nchw' in this block; x: the value named 'x' in this block; input_start: the value named 'input start' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 734: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 728. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'im2col_signed_nchw(x[:, input_start:input_start + input_channels_per_group], kernel=(kernel_h, kernel_w), stride=stride_pair, padding=padding_pair, dilation=dilation_pair,)' to 'patches ' in host memory. Here patches means the value named 'patches' in this block. The important operands are im2col_signed_nchw: the value named 'im2col signed nchw' in this block; x: the value named 'x' in this block; input_start: the value named 'input start' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 735: `weight_matrix = w[` This statement continues through source line 737; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'w[output_start:output_start + output_channels_per_group].reshape(output_channels_per_group, -1).T' to 'weight_matrix ' in host memory. Here weight_matrix means a learned coefficient multiplied by an activation. The important operands are w: the value named 'w' in this block; output_start: the value named 'output start' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 736: `output_start:output_start + output_channels_per_group` This is continuation line 2 of the 3-line logical statement that begins at source line 735. Its exact fragment is 'output_start:output_start + output_channels_per_group'. The important operands are output_start: the value named 'output start' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Read together, the complete operation is: This assigns 'w[output_start:output_start + output_channels_per_group].reshape(output_channels_per_group, -1).T' to 'weight_matrix ' in host memory. Here weight_matrix means a learned coefficient multiplied by an activation. The important operands are w: the value named 'w' in this block; output_start: the value named 'output start' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 737: `].reshape(output_channels_per_group, -1).T` This is continuation line 3 of the 3-line logical statement that begins at source line 735. Its exact fragment is '].reshape(output_channels_per_group, -1).T'. The important operands are reshape: the value named 'reshape' in this block; output_channels_per_group: the value named 'output channels per group' in this block; T: the value named 't' in this block. Read together, the complete operation is: This assigns 'w[output_start:output_start + output_channels_per_group].reshape(output_channels_per_group, -1).T' to 'weight_matrix ' in host memory. Here weight_matrix means a learned coefficient multiplied by an activation. The important operands are w: the value named 'w' in this block; output_start: the value named 'output start' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 738: `gradient_matrix = dy[` This statement continues through source line 740; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'dy[:, output_start:output_start + output_channels_per_group].transpose(0, 2, 3, 1).reshape(-1, output_channels_per_group)' to 'gradient_matrix ' in

host memory. Here gradient_matrix means the signed loss derivative used to update a learned parameter. The important operands are dy: the value named 'dy' in this block; output_start: the value named 'output start' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 739: `:, output_start:output_start + output_channels_per_group` This is continuation line 2 of the 3-line logical statement that begins at source line 738. Its exact fragment is ':, output_start:output_start + output_channels_per_group'. The important operands are output_start: the value named 'output start' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Read together, the complete operation is: This assigns 'dy[:, output_start:output_start + output_channels_per_group].transpose(0, 2, 3, 1).reshape(-1, output_channels_per_group)' to 'gradient_matrix ' in host memory. Here gradient_matrix means the signed loss derivative used to update a learned parameter. The important operands are dy: the value named 'dy' in this block; output_start: the value named 'output start' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 740: `].transpose(0, 2, 3, 1).reshape(-1, output_channels_per_group)` This is continuation line 3 of the 3-line logical statement that begins at source line 738. Its exact fragment is '].transpose(0, 2, 3, 1).reshape(-1, output_channels_per_group)'. The important operands are transpose: the value named 'transpose' in this block; reshape: the value named 'reshape' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Read together, the complete operation is: This assigns 'dy[:, output_start:output_start + output_channels_per_group].transpose(0, 2, 3, 1).reshape(-1, output_channels_per_group)' to 'gradient_matrix ' in host memory. Here gradient_matrix means the signed loss derivative used to update a learned parameter. The important operands are dy: the value named 'dy' in this block; output_start: the value named 'output start' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Context: the function compile_conv2d_backward, which handles compile conv2d backward.

Line 741: `programs.append(ConvBackwardGroup(` This statement continues through source line 754; the explanation uses the complete expression rather than only the first visual fragment. This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 742: `input_start,` This is continuation line 2 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'input_start,'. The important operands are input_start: the value named 'input start' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_
weight_gradient_shift=weight_gradient_shift,),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 743: `output_start,` This is continuation line 3 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'output_start,'. The important operands are output_start: the value named 'output start' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_
weight_gradient_shift=weight_gradient_shift,),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 744: `input_channels_per_group,` This is continuation line 4 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'input_channels_per_group,'. The important operands are input_channels_per_group:

the value named 'input channels per group' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 745: `output_channels_per_group,` This is continuation line 5 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'output_channels_per_group,'. The important operands are output_channels_per_group: the value named 'output channels per group' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 746: `compile_affine_backward(` This is continuation line 6 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'compile_affine_backward('. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 747: `patches,` This is continuation line 7 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'patches,'. The important operands are patches: the value named 'patches' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_s + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_g),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 748: `weight_matrix,` This is continuation line 8 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'weight_matrix,'. The important operands are weight_matrix: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=i weight_gradient_shift=weight_gradient_shift,),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 749: `bias_array[output_start:output_start + output_channels_per_group],` This is continuation line 9 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'bias_array[output_start:output_start + output_channels_per_group],'. The important operands are bias_array: an additive learned offset applied after accumulation; output_start: the value named 'output start' in this block; output_channels_per_group: the value named 'output channels per group' in this block. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_bac patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 750: `gradient_matrix,` This is continuation line 10 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'gradient_matrix,'. The important operands are gradient_matrix: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_gradient_shift,),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 751: `input_gradient_shift=input_gradient_shift,` This is continuation line 11 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'input_gradient_shift=input_gradient_shift,'. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_s + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_g),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 752: `weight_gradient_shift=weight_gradient_shift,` This is continuation line 12 of the 14-line logical statement that begins at source line 741. Its exact fragment is 'weight_gradient_shift=weight_gradient_shift,'. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_start:output_start + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_g),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 753: `),` This is continuation line 13 of the 14-line logical statement that begins at source line 741. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_s + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_g),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 754: `))` This is continuation line 14 of the 14-line logical statement that begins at source line 741. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'programs.append' with 'ConvBackwardGroup(input_start, output_start, input_channels_per_group, output_channels_per_group, compile_affine_backward(patches, weight_matrix, bias_array[output_s + output_channels_per_group], gradient_matrix, input_gradient_shift=input_gradient_shift, weight_gradient_shift=weight_g),)' inside the function compile_conv2d_backward, which handles compile conv2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 755: `return CompiledConv2dBackward(` This statement continues through source line 765; the explanation uses the complete expression rather than only the first visual fragment. This returns 'CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)' to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named 'compiled conv2d backward' in this block; tuple: the value named 'tuple' in this block; map: the value named 'map' in this block.

Line 756: `tuple(map(int, x.shape)),` This is continuation line 2 of the 11-line logical statement that begins at source line 755. Its exact fragment is 'tuple(map(int, x.shape)),'. The important operands are tuple: the value named 'tuple' in this block; map: the value named 'map' in this block; x: the value named 'x' in this block. Read together, the complete operation is: This returns 'CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)' to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named 'compiled conv2d backward' in this block; tuple: the value named 'tuple' in this block; map: the value named 'map' in this block.

Line 757: `tuple(map(int, w.shape)),` This is continuation line 3 of the 11-line logical statement that begins at source line 755. Its exact fragment is 'tuple(map(int, w.shape)),'. The important operands are tuple: the value named 'tuple' in this block; map: the value named 'map' in this block; w: the value named 'w' in this block. Read together, the complete operation is: This returns 'CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)' to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named 'compiled conv2d backward' in this block; tuple: the value named 'tuple' in this block; map: the value named 'map' in this block.

Line 758: `tuple(map(int, expected_output_shape)),` This is continuation line 4 of the 11-line logical statement that begins at source line 755. Its exact fragment is 'tuple(map(int, expected_output_shape)),'. The important operands are tuple: the value named 'tuple' in this block; map: the value named 'map' in this block; expected_output_shape: the value named 'expected output shape' in this block. Read together, the complete operation is: This returns 'CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)' to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named 'compiled conv2d backward' in this block; tuple: the value named 'tuple' in this block; map: the value named 'map' in this block.

Line 759: `(kernel_h, kernel_w),` This is continuation line 5 of the 11-line logical statement that begins at source line 755. Its exact fragment is '(kernel_h, kernel_w),'. The important operands are kernel_h: the value named 'kernel h' in this block; kernel_w: the value named 'kernel w' in this block. Read together, the complete operation is: This returns 'CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)' to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named 'compiled conv2d backward' in this block; tuple: the value named 'tuple' in this block; map: the value named 'map' in this block.

Line 760: `stride_pair,` This is continuation line 6 of the 11-line logical statement that begins at source line 755. Its exact fragment is 'stride_pair,'. The important operands are stride_pair: the value named 'stride pair' in this block. Read together, the complete operation is: This returns 'CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)' to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named 'compiled conv2d backward' in this block; tuple: the value named 'tuple' in this block; map: the value named 'map' in this block.

Line 761: `padding_pair,` This is continuation line 7 of the 11-line logical statement that begins at source line 755. Its exact fragment is 'padding_pair,'. The important operands are padding_pair: the value named 'padding pair' in this block. Read together, the complete operation is: This returns 'CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)' to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named 'compiled conv2d backward' in this block; tuple: the value named 'tuple' in this block; map: the value named 'map' in this block.

Line 762: `dilation_pair,` This is continuation line 8 of the 11-line logical statement that begins at source line 755. Its exact fragment is 'dilation_pair,'. The important operands are dilation_pair: the value named 'dilation pair' in this

block. Read together, the complete operation is: This returns ‘CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)‘ to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named ’compiled conv2d backward’ in this block; tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block.

Line 763: `groups,` This is continuation line 9 of the 11-line logical statement that begins at source line 755. Its exact fragment is ‘groups,‘. The important operands are groups: the value named ’groups’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)‘ to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named ’compiled conv2d backward’ in this block; tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block.

Line 764: `tuple(programs),` This is continuation line 10 of the 11-line logical statement that begins at source line 755. Its exact fragment is ‘tuple(programs),‘. The important operands are tuple: the value named ’tuple’ in this block; programs: the value named ’programs’ in this block. Read together, the complete operation is: This returns ‘CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)‘ to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named ’compiled conv2d backward’ in this block; tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block.

Line 765: `)` This is continuation line 11 of the 11-line logical statement that begins at source line 755. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘CompiledConv2dBackward(tuple(map(int, x.shape)), tuple(map(int, w.shape)), tuple(map(int, expected_output_shape)), (kernel_h, kernel_w), stride_pair, padding_pair, dilation_pair, groups, tuple(programs),)‘ to the caller of the function compile_conv2d_backward, which handles compile conv2d backward, ending this invocation. The important operands are CompiledConv2dBackward: the value named ’compiled conv2d backward’ in this block; tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block.

Line 768: `def sigmoid_surrogate_lut_q15(` This starts function sigmoid_surrogate_lut_q15. Its indented body performs sigmoid surrogate lut q15 as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 769: `*,` Inside the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 770: `alpha: float = 4.0,` This assigns ‘4.0,‘ to ‘alpha‘ in host memory. Here alpha means the programmable weight assigned to matching zero spikes in alpha-XNOR attention. The expression is a literal or structural operator and has no hidden software state. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 771: `membrane_fractional_bits: int = 10,` This assigns ‘10,‘ to ‘membrane_fractional_bits‘ in host memory. Here membrane_fractional_bits means the persistent LIF neuron state carrying charge between timesteps. The expression is a literal or structural operator and has no hidden software state. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 772: `address_shift: int | None = None,` This assigns ‘None,‘ to ‘address_shift‘ in host memory. Here address_shift means the value named ’address shift’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 773: `) -> tuple[np.ndarray, int]:` Inside the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 774: `"""Quantize SpikingJelly's sigmoid surrogate derivative into 256 Q15 entries."""` This is executable documentation attached to the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 776: `if not np.isfinite(alpha) or alpha <= 0:` This executes the following indented host-code branch only when 'not np.isfinite(alpha) or alpha <= 0' is true. The important operands are not: the value named 'not' in this block; isfinite: the value named 'isfinite' in this block; alpha: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 777: `raise ValueError("surrogate alpha must be finite and positive")` This rejects an invalid state by raising 'ValueError("surrogate alpha must be finite and positive")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 778: `if not 0 <= membrane_fractional_bits <= 22:` This executes the following indented host-code branch only when 'not 0 <= membrane_fractional_bits <= 22' is true. The important operands are not: the value named 'not' in this block; membrane_fractional_bits: the persistent LIF neuron state carrying charge between timesteps. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 779: `raise ValueError("membrane_fractional_bits must be in [0,22]")` This rejects an invalid state by raising 'ValueError("membrane_fractional_bits must be in [0,22]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 780: `selected_shift = (` This statement continues through source line 784; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(max(membrane_fractional_bits - 5, 0) if address_shift is None else int(address_shift))' to 'selected_shift ' in host memory. Here selected_shift means the value named 'selected shift' in this block. The important operands are max: the value named 'max' in this block; membrane_fractional_bits: the persistent LIF neuron state carrying charge between timesteps; address_shift: the value named 'address shift' in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 781: `max(membrane_fractional_bits - 5, 0)` This is continuation line 2 of the 5-line logical statement that begins at source line 780. Its exact fragment is 'max(membrane_fractional_bits - 5, 0)'. The important operands are max: the value named 'max' in this block; membrane_fractional_bits: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns '(max(membrane_fractional_bits - 5, 0) if address_shift is None else int(address_shift))' to 'selected_shift ' in host memory. Here selected_shift means the value named 'selected shift' in this block. The important operands are max: the value named 'max' in this block; membrane_fractional_bits: the persistent LIF neuron state carrying charge between timesteps; address_shift: the value named 'address shift' in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 782: `if address_shift is None` This is continuation line 3 of the 5-line logical statement that begins at source line 780. Its exact fragment is 'if address_shift is None'. The important operands are address_shift: the value named 'address shift' in this block; is: the value named 'is' in this block; None: the value named 'none' in this block. Read together, the complete operation is: This assigns '(max(membrane_fractional_bits - 5, 0) if address_shift is None else int(address_shift))' to 'selected_shift ' in host memory. Here selected_shift means the value named 'selected shift' in this block. The important operands are max: the value named 'max' in this block; membrane_fractional_bits: the persistent LIF neuron state carrying charge between timesteps; address_shift: the value named 'address shift' in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 783: `else int(address_shift)` This is continuation line 4 of the 5-line logical statement that begins at source line 780. Its exact fragment is ‘else int(address_shift)‘. The important operands are address_shift: the value named ’address shift’ in this block. Read together, the complete operation is: This assigns ‘(max(membrane_fractional_bits - 5, 0) if address_shift is None else int(address_shift))‘ to ‘selected_shift ‘ in host memory. Here selected_shift means the value named ’selected shift’ in this block. The important operands are max: the value named ’max’ in this block; membrane_fractional_bits: the persistent LIF neuron state carrying charge between timesteps; address_shift: the value named ’address shift’ in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 784: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 780. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(max(membrane_fractional_bits - 5, 0) if address_shift is None else int(address_shift))‘ to ‘selected_shift ‘ in host memory. Here selected_shift means the value named ’selected shift’ in this block. The important operands are max: the value named ’max’ in this block; membrane_fractional_bits: the persistent LIF neuron state carrying charge between timesteps; address_shift: the value named ’address shift’ in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 785: `if not 0 <= selected_shift <= membrane_fractional_bits:` This executes the following indented host-code branch only when ‘not 0 <= selected_shift <= membrane_fractional_bits‘ is true. The important operands are not: the value named ’not’ in this block; selected_shift: the value named ’selected shift’ in this block; membrane_fractional_bits: the persistent LIF neuron state carrying charge between timesteps. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 786: `raise ValueError("address_shift must be in [0, membrane_fractional_bits]")` This rejects an invalid state by raising ‘ValueError("address_shift must be in [0, membrane_fractional_bits]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 787: `addresses = np.arange(256, dtype=np.float64)` This assigns ‘np.arange(256, dtype=np.float64)‘ to ‘addresses ‘ in host memory. Here addresses means the value named ’addresses’ in this block. The important operands are arange: the value named ’arange’ in this block; dtype: the value named ’dtype’ in this block; float64: the value named ’float64’ in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 788: `delta = (addresses - 128.0) * (2.0 ** selected_shift)` This assigns ‘(addresses - 128.0) * (2.0 ** selected_shift)‘ to ‘delta ‘ in host memory. Here delta means the value named ’delta’ in this block. The important operands are addresses: the value named ’addresses’ in this block; selected_shift: the value named ’selected shift’ in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 789: `delta /= 2.0 ** membrane_fractional_bits` Inside the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are delta: the value named ’delta’ in this block; membrane_fractional_bits: the persistent LIF neuron state carrying charge between timesteps.

Line 790: `sigmoid = 1.0 / (1.0 + np.exp(-alpha * delta))` This assigns ‘1.0 / (1.0 + np.exp(-alpha * delta))‘ to ‘sigmoid ‘ in host memory. Here sigmoid means the value named ’sigmoid’ in this block. The important operands are exp: the value named ’exp’ in this block; alpha: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; delta: the value named ’delta’ in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 791: `derivative = alpha * sigmoid * (1.0 - sigmoid)` This assigns ‘alpha * sigmoid * (1.0 - sigmoid)‘ to ‘derivative ‘ in host memory. Here derivative means the value named ’derivative’ in this block. The important operands are alpha: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; sigmoid: the value named ’sigmoid’ in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 792: `table = np.rint(derivative * (1 << 15))` This assigns ‘np.rint(derivative * (1 << 15))‘ to ‘table ‘ in host memory. Here table means the value named ’table’ in this block. The important operands are rint: the value named ’rint’ in this block; derivative: the value named ’derivative’ in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 793: `table = np.clip(table, 0, (1 << 15) - 1).astype(np.int64)` This assigns ‘np.clip(table, 0, (1 << 15) - 1).astype(np.int64)‘ to ‘table ‘ in host memory. Here table means the value named ’table’ in this block. The important operands are clip: the value named ’clip’ in this block; table: the value named ’table’ in this block; astype: the value named ’astype’ in this block. Context: the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15.

Line 794: `return table, selected_shift` This returns ‘table, selected_shift‘ to the caller of the function sigmoid_surrogate_lut_q15, which handles sigmoid surrogate lut q15, ending this invocation. The important operands are table: the value named ’table’ in this block; selected_shift: the value named ’selected shift’ in this block.

Line 797: `def surrogate_lut_addresses(` This starts function surrogate_lut_addresses. Its indented body performs surrogate lut addresses as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 798: `pre_reset_membrane,` Inside the function surrogate_lut_addresses, which handles surrogate lut addresses, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps.

Line 799: `threshold: int,` Inside the function surrogate_lut_addresses, which handles surrogate lut addresses, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are threshold: the membrane level at which a neuron emits a spike.

Line 800: `address_shift: int,` Inside the function surrogate_lut_addresses, which handles surrogate lut addresses, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are address_shift: the value named ’address shift’ in this block.

Line 801: `) -> np.ndarray:` Inside the function surrogate_lut_addresses, which handles surrogate lut addresses, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are ndarray: the value named ’ndarray’ in this block.

Line 802: `"""Map signed fixed-point membrane distance to a saturating LUT address."""` This is executable documentation attached to the function surrogate_lut_addresses, which handles surrogate lut addresses. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 804: `if not 0 <= address_shift <= 31:` This executes the following indented host-code branch only when ‘not 0 <= address_shift <= 31‘ is true. The important operands are not: the value named ’not’ in this block; address_shift: the value named ’address shift’ in this block. Context: the function surrogate_lut_addresses, which handles surrogate lut addresses.

Line 805: `raise ValueError("address_shift must be in [0,31]")` This rejects an invalid state by raising ‘ValueError("address_shift must be in [0,31]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 806: `membrane = np.asarray(pre_reset_membrane, dtype=np.int64)` This assigns ‘np.asarray(pre_reset_membrane, dtype=np.int64)‘ to ‘membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are asarray: the value named ’asarray’ in this block; pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; dtype: the value named ’dtype’ in this block. Context: the function surrogate_lut_addresses, which handles surrogate lut addresses.

Line 807: `distance = membrane - int(threshold)` This assigns ‘membrane - int(threshold)‘ to ‘distance ‘ in host memory. Here distance means the value named ’distance’ in this block. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; threshold: the membrane level at which a neuron emits a spike. Context: the function surrogate_lut_addresses, which handles surrogate lut addresses.

Line 808: `quantized = round_shift_signed(distance, address_shift)` This assigns ‘round_shift_signed(distance, address_shift)‘ to ‘quantized ‘ in host memory. Here quantized means the value named ’quantized’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; distance: the value named ’distance’ in this block; address_shift: the value named ’address shift’ in this block. Context: the function surrogate_lut_addresses, which handles surrogate lut addresses.

Line 809: `return np.clip(quantized + 128, 0, 255).astype(np.uint8)` This returns ‘np.clip(quantized + 128, 0, 255).astype(np.uint8)‘ to the caller of the function surrogate_lut_addresses, which handles surrogate lut addresses, ending this invocation. The important operands are clip: the value named ’clip’ in this block; quantized: the value named ’quantized’ in this block; astype: the value named ’astype’ in this block.

Line 812: `def lif_bptt_fixed(` This starts function lif_bptt_fixed. Its indented body performs lif bptt fixed as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 813: `pre_reset_membrane,` Inside the function lif_bptt_fixed, which handles lif bptt fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps.

Line 814: `spikes,` Inside the function lif_bptt_fixed, which handles lif bptt fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are spikes: a binary event produced when a neuron crosses its threshold.

Line 815: `spike_gradients,` Inside the function lif_bptt_fixed, which handles lif bptt fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are spike_gradients: a binary event produced when a neuron crosses its threshold.

Line 816: `*,` Inside the function lif_bptt_fixed, which handles lif bptt fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 817: `threshold: int,` Inside the function lif_bptt_fixed, which handles lif bptt fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are threshold: the membrane level at which a neuron emits a spike.

Line 818: `membrane_gradients=None,` This assigns ‘None,‘ to ‘membrane_gradients‘ in host memory. Here membrane_gradients means the persistent LIF neuron state carrying charge between timesteps. The important operands are None: the value named ’none’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 819: `reciprocal_tau_q15: int = 1 << 14,` This assigns '1 << 14,' to 'reciprocal_tau_q15' in host memory. Here reciprocal_tau_q15 means the value named 'reciprocal tau q15' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 820: `surrogate_table_q15=None,` This assigns 'None,' to 'surrogate_table_q15' in host memory. Here surrogate_table_q15 means the value named 'surrogate table q15' in this block. The important operands are None: the value named 'none' in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 821: `surrogate_address_shift: int = 5,` This assigns '5,' to 'surrogate_address_shift' in host memory. Here surrogate_address_shift means the value named 'surrogate address shift' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 822: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function lif_bptt_fixed, which handles lif bptt fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 823: `"""Bit-exact detached-reset hard-LIF BPTT using signed Q15 products."""` This is executable documentation attached to the function lif_bptt_fixed, which handles lif bptt fixed. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 825: `membrane = np.asarray(pre_reset_membrane, dtype=np.int64)` This assigns 'np.asarray(pre_reset_membrane, dtype=np.int64)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are asarray: the value named 'asarray' in this block; pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; dtype: the value named 'dtype' in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 826: `spike = np.asarray(spikes, dtype=np.int64)` This assigns 'np.asarray(spikes, dtype=np.int64)' to 'spike ' in host memory. Here spike means a binary event produced when a neuron crosses its threshold. The important operands are asarray: the value named 'asarray' in this block; spikes: a binary event produced when a neuron crosses its threshold; dtype: the value named 'dtype' in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 827: `grad_spike = np.asarray(spike_gradients, dtype=np.int64)` This assigns 'np.asarray(spike_gradients, dtype=np.int64)' to 'grad_spike ' in host memory. Here grad_spike means a binary event produced when a neuron crosses its threshold. The important operands are asarray: the value named 'asarray' in this block; spike_gradients: a binary event produced when a neuron crosses its threshold; dtype: the value named 'dtype' in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 828: `if membrane.ndim != 2 or spike.shape != membrane.shape or grad_spike.shape != membrane.shape:` This executes the following indented host-code branch only when 'membrane.ndim != 2 or spike.shape != membrane.shape or grad_spike.shape != membrane.shape' is true. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 829: `raise ValueError("LIF trace, spike, and gradient arrays must share shape [time, neurons]")` This rejects an invalid state by raising 'ValueError("LIF trace, spike, and gradient arrays must share shape [time, neurons]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 830: `if np.any((spike != 0) & (spike != 1)):` This executes the following indented host-code branch only when 'np.any((spike != 0) & (spike != 1))' is true. The important operands are any: the value named 'any' in this block; spike: a binary event produced when a neuron crosses its threshold. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 831: `raise ValueError("LIF spikes must be binary")` This rejects an invalid state by raising ‘ValueError("LIF spikes must be binary")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 832: `if membrane_gradients is None:` This executes the following indented host-code branch only when ‘membrane_gradients is None‘ is true. The important operands are membrane_gradients: the persistent LIF neuron state carrying charge between timesteps; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 833: `grad_v = np.zeros_like(membrane)` This assigns ‘np.zeros_like(membrane)‘ to ‘grad_v ‘ in host memory. Here grad_v means the value named ’grad v’ in this block. The important operands are zeros_like: the value named ’zeros like’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 834: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function lif_bptt_fixed, which handles lif bptt fixed.

Line 835: `grad_v = np.asarray(membrane_gradients, dtype=np.int64)` This assigns ‘np.asarray(membrane_gradients, dtype=np.int64)‘ to ‘grad_v ‘ in host memory. Here grad_v means the value named ’grad v’ in this block. The important operands are asarray: the value named ’asarray’ in this block; membrane_gradients: the persistent LIF neuron state carrying charge between timesteps; dtype: the value named ’dtype’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 836: `if grad_v.shape != membrane.shape:` This executes the following indented host-code branch only when ‘grad_v.shape != membrane.shape‘ is true. The important operands are grad_v: the value named ’grad v’ in this block; shape: the value named ’shape’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 837: `raise ValueError("membrane gradients must match the LIF trace shape")` This rejects an invalid state by raising ‘ValueError("membrane gradients must match the LIF trace shape")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 838: `if not 0 <= reciprocal_tau_q15 <= (1 << 15):` This executes the following indented host-code branch only when ‘not 0 <= reciprocal_tau_q15 <= (1 << 15)‘ is true. The important operands are not: the value named ’not’ in this block; reciprocal_tau_q15: the value named ’reciprocal tau q15’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 839: `raise ValueError("reciprocal_tau_q15 must be in [0,32768]")` This rejects an invalid state by raising ‘ValueError("reciprocal_tau_q15 must be in [0,32768]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 840: `if surrogate_table_q15 is None:` This executes the following indented host-code branch only when ‘surrogate_table_q15 is None‘ is true. The important operands are surrogate_table_q15: the value named ’surrogate table q15’ in this block; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 841: `surrogate_table_q15, _ = sigmoid_surrogate_lut_q15(` This statement continues through source line 843; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sigmoid_surrogate_lut_q1 address_shift=surrogate_address_shift)‘ to ‘surrogate_table_q15, _ ‘ in host memory. Here surrogate_table_q15 means the value named ’surrogate table q15’ in this block. The important operands are sigmoid_surrogate_lut_q15: the value named ’sigmoid surrogate lut q15’ in this block; address_shift: the value named ’address shift’ in this block; surrogate_address_shift: the value named ’surrogate address shift’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 842: `address_shift=surrogate_address_shift` This is continuation line 2 of the 3-line logical statement that begins at source line 841. Its exact fragment is ‘address_shift=surrogate_address_shift‘. The important operands are address_shift: the value named ’address shift’ in this block; surrogate_address_shift: the value named ’surrogate address shift’ in this block. Read together, the complete operation is: This assigns ‘sigmoid_surrogate_lut_q15(address_shift=surrogate_address_shift)‘ to ‘surrogate_table_q15, _ ‘ in host memory. Here surrogate_table_q15 means the value named ’surrogate table q15’ in this block. The important operands are sigmoid_surrogate_lut_q15: the value named ’sigmoid surrogate lut q15’ in this block; address_shift: the value named ’address shift’ in this block; surrogate_address_shift: the value named ’surrogate address shift’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 843: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 841. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘sigmoid_surrogate_lut_q15(address_shift=surrogate_address_shift)‘ to ‘surrogate_table_q15, _ ‘ in host memory. Here surrogate_table_q15 means the value named ’surrogate table q15’ in this block. The important operands are sigmoid_surrogate_lut_q15: the value named ’sigmoid surrogate lut q15’ in this block; address_shift: the value named ’address shift’ in this block; surrogate_address_shift: the value named ’surrogate address shift’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 844: `table = np.asarray(surrogate_table_q15, dtype=np.int64)` This assigns ‘np.asarray(surrogate_table_q15, dtype=np.int64)‘ to ‘table ‘ in host memory. Here table means the value named ’table’ in this block. The important operands are asarray: the value named ’asarray’ in this block; surrogate_table_q15: the value named ’surrogate table q15’ in this block; dtype: the value named ’dtype’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 845: `if table.shape != (256,) or np.any(table < 0) or np.any(table > 32767):` This executes the following indented host-code branch only when ‘table.shape != (256,) or np.any(table < 0) or np.any(table > 32767)‘ is true. The important operands are table: the value named ’table’ in this block; shape: the value named ’shape’ in this block; or: the value named ’or’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 846: `raise ValueError("surrogate table must contain 256 nonnegative signed-Q15 values")` This rejects an invalid state by raising ‘ValueError("surrogate table must contain 256 nonnegative signed-Q15 values")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 847: `addresses = surrogate_lut_addresses(membrane, threshold, surrogate_address_shift)` This assigns ‘surrogate_lut_addresses(membrane, threshold, surrogate_address_shift)‘ to ‘addresses ‘ in host memory. Here addresses means the value named ’addresses’ in this block. The important operands are surrogate_lut_addresses: the value named ’surrogate lut addresses’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; threshold: the membrane level at which a neuron emits a spike. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 848: `derivative = table[addresses]` This assigns ‘table[addresses]‘ to ‘derivative ‘ in host memory. Here derivative means the value named ’derivative’ in this block. The important operands are table: the value named ’table’ in this block; addresses: the value named ’addresses’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 849: `grad_input = np.zeros_like(membrane)` This assigns ‘np.zeros_like(membrane)‘ to ‘grad_input ‘ in host memory. Here grad_input means the value named ’grad input’ in this block. The important operands are zeros_like: the value named ’zeros like’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 850: `grad_h_trace = np.zeros_like(membrane)` This assigns ‘np.zeros_like(membrane)‘ to ‘grad_h_trace ‘ in host memory. Here grad_h_trace means the value named ’grad h trace’ in this block. The important operands are zeros_like: the value named ’zeros like’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 851: `grad_h_next = np.zeros(membrane.shape[1], dtype=np.int64)` This assigns ‘np.zeros(membrane.shape[1], dtype=np.int64)‘ to ‘grad_h_next ‘ in host memory. Here grad_h_next means the value named ’grad h next’ in this block. The important operands are zeros: the value named ’zeros’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; shape: the value named ’shape’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 852: `leak_q15 = (1 << 15) - reciprocal_tau_q15` This assigns ‘(1 << 15) - reciprocal_tau_q15‘ to ‘leak_q15 ‘ in host memory. Here leak_q15 means the retained fraction of prior membrane state. The important operands are reciprocal_tau_q15: the value named ’reciprocal tau q15’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 853: `for timestep in range(membrane.shape[0] - 1, -1, -1):` This iterates timestep over ‘range(membrane.shape[0] - 1, -1, -1)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_bptt_fixed, which handles lif bptt fixed.

Line 854: `temporal = saturate_signed(` This statement continues through source line 856; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘saturate_signed(round_shift_signed(grad_h_next * leak_q15, 15), 32)‘ to ‘temporal ‘ in host memory. Here temporal means the value named ’temporal’ in this block. The important operands are saturate_signed: the value named ’saturate signed’ in this block; round_shift_signed: the value named ’round shift signed’ in this block; grad_h_next: the value named ’grad h next’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 855: `round_shift_signed(grad_h_next * leak_q15, 15), 32` This is continuation line 2 of the 3-line logical statement that begins at source line 854. Its exact fragment is ‘round_shift_signed(grad_h_next * leak_q15, 15), 32‘. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; grad_h_next: the value named ’grad h next’ in this block; leak_q15: the retained fraction of prior membrane state. Read together, the complete operation is: This assigns ‘saturate_signed(round_shift_signed(grad_h_next * leak_q15, 15), 32)‘ to ‘temporal ‘ in host memory. Here temporal means the value named ’temporal’ in this block. The important operands are saturate_signed: the value named ’saturate signed’ in this block; round_shift_signed: the value named ’round shift signed’ in this block; grad_h_next: the value named ’grad h next’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 856: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 854. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘saturate_signed(round_shift_signed(grad_h_next * leak_q15, 15), 32)‘ to ‘temporal ‘ in host memory. Here temporal means the value named ’temporal’ in this block. The important operands are saturate_signed: the value named ’saturate signed’ in this block; round_shift_signed: the value named ’round shift signed’ in this block; grad_h_next: the value named ’grad h next’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 857: `membrane_path = saturate_signed(grad_v[timestep] + temporal, 32)` This assigns ‘saturate_signed(grad_v[ti + temporal, 32)‘ to ‘membrane_path ‘ in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are saturate_signed: the value named ’saturate signed’ in this block; grad_v: the value named ’grad v’ in this block; timestep: one discrete SNN update instant. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 858: `membrane_path = np.where(spike[timestep] == 0, membrane_path, 0)` Inside the function lif_bptt_fixed, which handles lif bptt fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are membrane_path: the persistent LIF neuron state carrying charge between timesteps; where: the value named ’where’ in this block; spike: a binary event produced when a neuron crosses its threshold.

Line 859: `spike_path = saturate_signed(` This statement continues through source line 861; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘saturate_signed(round_shift_signed(grad_spike[ti * derivative[timestep], 15), 32)‘ to ‘spike_path ‘ in host memory. Here spike_path means a binary event produced when

a neuron crosses its threshold. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; grad_spike: a binary event produced when a neuron crosses its threshold. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 860: `round_shift_signed(grad_spike[timestep] * derivative[timestep], 15), 32` This is continuation line 2 of the 3-line logical statement that begins at source line 859. Its exact fragment is 'round_shift_signed(grad_spike[timestep] * derivative[timestep], 15), 32'. The important operands are round_shift_signed: the value named 'round shift signed' in this block; grad_spike: a binary event produced when a neuron crosses its threshold; timestep: one discrete SNN update instant. Read together, the complete operation is: This assigns 'saturate_signed(round_shift_signed(grad_spike[timestep] * derivative[timestep], 15), 32)' to 'spike_path ' in host memory. Here spike_path means a binary event produced when a neuron crosses its threshold. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; grad_spike: a binary event produced when a neuron crosses its threshold. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 861: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 859. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'saturate_signed(round_shift_signed(grad_spike[timestep] * derivative[timestep], 15), 32)' to 'spike_path ' in host memory. Here spike_path means a binary event produced when a neuron crosses its threshold. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; grad_spike: a binary event produced when a neuron crosses its threshold. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 862: `grad_h = saturate_signed(membrane_path + spike_path, 32)` This assigns 'saturate_signed(membrane_path + spike_path, 32)' to 'grad_h ' in host memory. Here grad_h means the value named 'grad h' in this block. The important operands are saturate_signed: the value named 'saturate signed' in this block; membrane_path: the persistent LIF neuron state carrying charge between timesteps; spike_path: a binary event produced when a neuron crosses its threshold. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 863: `grad_input[timestep] = saturate_signed(` This statement continues through source line 865; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'saturate_signed(round_shift_signed(grad_h * reciprocal_tau_q15, 15), 32)' to 'grad_input[timestep] ' in host memory. Here grad_input means the value named 'grad input' in this block. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; grad_h: the value named 'grad h' in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 864: `round_shift_signed(grad_h * reciprocal_tau_q15, 15), 32` This is continuation line 2 of the 3-line logical statement that begins at source line 863. Its exact fragment is 'round_shift_signed(grad_h * reciprocal_tau_q15, 15), 32'. The important operands are round_shift_signed: the value named 'round shift signed' in this block; grad_h: the value named 'grad h' in this block; reciprocal_tau_q15: the value named 'reciprocal tau q15' in this block. Read together, the complete operation is: This assigns 'saturate_signed(round_shift_signed(grad_h * reciprocal_tau_q15, 15), 32)' to 'grad_input[timestep] ' in host memory. Here grad_input means the value named 'grad input' in this block. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; grad_h: the value named 'grad h' in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 865: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 863. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'saturate_signed(round_shift_signed(grad_h * reciprocal_tau_q15, 15), 32)' to 'grad_input[timestep] ' in host memory. Here grad_input means the value named 'grad input' in this block. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; grad_h: the value named 'grad h' in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 866: `grad_h_trace[timestep] = grad_h` This assigns 'grad_h' to 'grad_h_trace[timestep] ' in host memory. Here grad_h_trace means the value named 'grad h trace' in this block. The important operands are grad_h: the value named 'grad h' in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 867: `grad_h_next = grad_h` This assigns ‘grad_h‘ to ‘grad_h_next ‘ in host memory. Here grad_h_next means the value named ’grad h next’ in this block. The important operands are grad_h: the value named ’grad h’ in this block. Context: the function lif_bptt_fixed, which handles lif bptt fixed.

Line 868: `return grad_input, grad_h_trace` This returns ‘grad_input, grad_h_trace‘ to the caller of the function lif_bptt_fixed, which handles lif bptt fixed, ending this invocation. The important operands are grad_input: the value named ’grad input’ in this block; grad_h_trace: the value named ’grad h trace’ in this block.

Line 871: `def _execute_vector8(` This starts function _execute_vector8. Its indented body performs execute vector8 as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 872: `driver: TrainingCommandExecutor,` Inside the function _execute_vector8, which handles execute vector8, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named ’driver’ in this block; TrainingCommandExecutor: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 873: `operation: VectorOperation,` Inside the function _execute_vector8, which handles execute vector8, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block.

Line 874: `operand_a: Sequence[int],` Inside the function _execute_vector8, which handles execute vector8, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are operand_a: the value named ’operand a’ in this block; Sequence: the value named ’sequence’ in this block.

Line 875: `operand_b: Sequence[int],` Inside the function _execute_vector8, which handles execute vector8, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are operand_b: the value named ’operand b’ in this block; Sequence: the value named ’sequence’ in this block.

Line 876: `*,` Inside the function _execute_vector8, which handles execute vector8, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 877: `timeout_seconds: float,` Inside the function _execute_vector8, which handles execute vector8, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; float: the value named ’float’ in this block.

Line 878: `clamp_minimum: int = -(1 << 31),` This assigns ‘-(1 << 31),‘ to ‘clamp_minimum‘ in host memory. Here clamp_minimum means the value named ’clamp minimum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _execute_vector8, which handles execute vector8.

Line 879: `clamp_maximum: int = (1 << 31) - 1,` This assigns ‘(1 << 31) - 1,‘ to ‘clamp_maximum‘ in host memory. Here clamp_maximum means the value named ’clamp maximum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _execute_vector8, which handles execute vector8.

Line 880: `) -> np.ndarray:` Inside the function _execute_vector8, which handles execute vector8, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are ndarray: the value named ’ndarray’ in this block.

Line 881: `words = driver.execute(` This statement continues through source line 891; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 882: `vector_command(int(operation)),` This is continuation line 2 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘vector_command(int(operation)),‘. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named ’operation’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 883: `pack_vector_beats(` This is continuation line 3 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘pack_vector_beats(‘. The important operands are pack_vector_beats: the value named ’pack vector beats’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 884: `operand_a,` This is continuation line 4 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘operand_a,‘. The important operands are operand_a: the value named ’operand a’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 885: `operand_b,` This is continuation line 5 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘operand_b,‘. The important operands are operand_b: the value named ’operand b’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 886: `clamp_minimum=clamp_minimum,` This is continuation line 6 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘clamp_minimum=clamp_minimum,‘. The important operands are clamp_minimum: the value named ’clamp minimum’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 887: `clamp_maximum=clamp_maximum,` This is continuation line 7 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘clamp_maximum=clamp_maximum,‘. The important operands are clamp_maximum: the value named ’clamp maximum’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 888: `),` This is continuation line 8 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 889: `output_beats=4,` This is continuation line 9 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘output_beats=4,‘. The important operands are output_beats: the value named ’output beats’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 890: `timeout_seconds=timeout_seconds,` This is continuation line 10 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 891: `)` This is continuation line 11 of the 11-line logical statement that begins at source line 881. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘driver.execute(vector_command(int(operation)), pack_vector_beats(operand_a, operand_b, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), output_beats=4, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function _execute_vector8, which handles execute vector8.

Line 892: `if len(words) != 4:` This executes the following indented host-code branch only when ‘len(words) != 4‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block. Context: the function _execute_vector8, which handles execute vector8.

Line 893: `raise RuntimeError(f"vector operation returned {len(words)} beats; expected four")` This rejects an invalid state by raising ‘RuntimeError(f"vector operation returned {len(words)} beats; expected four")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 894: `return np.asarray(unpack_vector_result(words), dtype=np.int64)` This returns ‘np.asarray(unpack_vector_r dtype=np.int64)‘ to the caller of the function _execute_vector8, which handles execute vector8, ending this invocation. The important operands are asarray: the value named ’asarray’ in this block; unpack_vector_result: the value named ’unpack vector result’ in this block; words: the value named ’words’ in this block.

Line 897: `def execute_sgdw_update(` This starts function execute_sgdw_update. Its indented body performs execute sgdw update as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 898: `driver: TrainingCommandExecutor,` Inside the function execute_sgdw_update, which handles execute sgdw update, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named ’driver’ in this block; TrainingCommandExecutor: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 899: `weights,` Inside the function execute_sgdw_update, which handles execute sgdw update, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are weights: a learned coefficient multiplied by an activation.

Line 900: `gradients,` Inside the function execute_sgdw_update, which handles execute sgdw update, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are gradients: the signed loss derivative used to update a learned parameter.

Line 901: `velocities,` Inside the function execute_sgdw_update, which handles execute sgdw update, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are velocities: the value named ’velocities’ in this block.

Line 902: `*,` Inside the function execute_sgdw_update, which handles execute sgdw update, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 903: `learning_rate_q16: int,` Inside the function execute_sgdw_update, which handles execute sgdw update, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude.

Line 904: `momentum_q16: int,` Inside the function execute_sgdw_update, which handles execute sgdw update, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are momentum_q16: the value named ’momentum q16’ in this block.

Line 905: `weight_decay_q16: int,` Inside the function execute_sgdw_update, which handles execute sgdw update, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are weight_decay_q16: a learned coefficient multiplied by an activation.

Line 906: `update_right_shift: int = 16,` This assigns ‘16,‘ to ‘update_right_shift‘ in host memory. Here update_right_shift means the value named ’update right shift’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 907: `timeout_seconds: float = 5.0,` This assigns ‘5.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 908: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function execute_sgdw_update, which handles execute sgdw update, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named ’tuple’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 909: `"""Update an arbitrary tensor through the fabric's eight SGDW lanes."""` This is executable documentation attached to the function execute_sgdw_update, which handles execute sgdw update. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 911: `weight = np.asarray(weights, dtype=np.int64)` This assigns ‘np.asarray(weights, dtype=np.int64)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named ’dtype’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 912: `gradient = np.asarray(gradients, dtype=np.int64)` This assigns ‘np.asarray(gradients, dtype=np.int64)‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are asarray: the value named ’asarray’ in this block; gradients: the signed loss derivative used to update a learned parameter; dtype: the value named ’dtype’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 913: `velocity = np.asarray(velocities, dtype=np.int64)` This assigns ‘np.asarray(velocities, dtype=np.int64)‘ to ‘velocity ‘ in host memory. Here velocity means the momentum history retained by the SGD optimizer. The important operands are asarray: the value named ’asarray’ in this block; velocities: the value named ’velocities’ in this block; dtype: the value named ’dtype’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 914: `if weight.shape != gradient.shape or weight.shape != velocity.shape or weight.size == 0:` This executes the following indented host-code branch only when ‘weight.shape != gradient.shape or weight.shape != velocity.shape or weight.size == 0‘ is true. The important operands are weight: a learned coefficient multiplied by an activation; shape: the value named ’shape’ in this block; gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 915: `raise ValueError("weights, gradients, and velocities must share a non-empty shape")` This rejects an invalid state by raising ‘ValueError("weights, gradients, and velocities must share a non-empty shape")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 916: `if np.any(weight < -(1 << 15)) or np.any(weight > (1 << 15) - 1):` This executes the following indented host-code branch only when ‘np.any(weight < -(1 << 15)) or np.any(weight > (1 << 15) - 1)‘ is true. The important operands are any: the value named ’any’ in this block; weight: a learned coefficient multiplied by an activation; or: the value named ’or’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 917: `raise ValueError("optimizer weights must fit signed 16-bit")` This rejects an invalid state by raising ‘ValueError("optimizer weights must fit signed 16-bit")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 918: `for name, value in (("gradients", gradient), ("velocities", velocity)):` This iterates name, value over ‘(("gradients", gradient), ("velocities", velocity))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_sgdw_update, which handles execute sgdw update.

Line 919: `if np.any(value < -(1 << 23)) or np.any(value > (1 << 23) - 1):` This executes the following indented host-code branch only when ‘np.any(value < -(1 << 23)) or np.any(value > (1 << 23) - 1)‘ is true. The important operands are any: the value named ’any’ in this block; value: the value named ’value’ in this block; or: the value named ’or’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 920: `raise ValueError(f"optimizer {name} must fit signed 24-bit")` This rejects an invalid state by raising ‘ValueError(f"optimizer {name} must fit signed 24-bit")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 921: `flat_weight = weight.reshape(-1)` This assigns ‘weight.reshape(-1)‘ to ‘flat_weight ‘ in host memory. Here flat_weight means a learned coefficient multiplied by an activation. The important operands are weight: a learned coefficient multiplied by an activation; reshape: the value named ’reshape’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 922: `flat_gradient = gradient.reshape(-1)` This assigns ‘gradient.reshape(-1)‘ to ‘flat_gradient ‘ in host memory. Here flat_gradient means the signed loss derivative used to update a learned parameter. The important operands are gradient: the signed loss derivative used to update a learned parameter; reshape: the value named ’reshape’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 923: `flat_velocity = velocity.reshape(-1)` This assigns ‘velocity.reshape(-1)‘ to ‘flat_velocity ‘ in host memory. Here flat_velocity means the momentum history retained by the SGD optimizer. The important operands are velocity: the momentum history retained by the SGD optimizer; reshape: the value named ’reshape’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 924: `next_weight = np.zeros_like(flat_weight)` This assigns ‘np.zeros_like(flat_weight)‘ to ‘next_weight ‘ in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are zeros_like: the value named ’zeros like’ in this block; flat_weight: a learned coefficient multiplied by an activation. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 925: `next_velocity = np.zeros_like(flat_velocity)` This assigns ‘np.zeros_like(flat_velocity)‘ to ‘next_velocity ‘ in host memory. Here next_velocity means the momentum history retained by the SGD optimizer. The important operands are zeros_like: the value named ’zeros like’ in this block; flat_velocity: the momentum history retained by the SGD optimizer. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 926: `command = sgd_command(` This statement continues through source line 931; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sgd_command(learning_rate_q16, momentum_q16, weight_decay_q16, update_right_shift=update_right_shift,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; learning_rate_q16: the fixed-point multiplier controlling update magnitude; momentum_q16: the value named ’momentum q16’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 927: `learning_rate_q16,` This is continuation line 2 of the 6-line logical statement that begins at source line 926. Its exact fragment is ‘learning_rate_q16,‘. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns ‘sgd_command(learning_rate_q16, momentum_q16, weight_decay_q16, update_right_shift=update_right_shift,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; learning_rate_q16: the fixed-point multiplier controlling update magnitude; momentum_q16: the value named ’momentum q16’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 928: `momentum_q16,` This is continuation line 3 of the 6-line logical statement that begins at source line 926. Its exact fragment is ‘momentum_q16,‘. The important operands are momentum_q16: the value named ’momentum q16’ in this block. Read together, the complete operation is: This assigns ‘sgd_command(learning_rate_q16, momentum_q16, weight_decay_q16, update_right_shift=update_right_shift,)‘ to ‘command ‘ in host memory. Here

command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; learning_rate_q16: the fixed-point multiplier controlling update magnitude; momentum_q16: the value named 'momentum q16' in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 929: `weight_decay_q16,` This is continuation line 4 of the 6-line logical statement that begins at source line 926. Its exact fragment is 'weight_decay_q16,'. The important operands are weight_decay_q16: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'sgd_command(learning_rate_q16, momentum_q16, weight_decay_q16, update_right_shift=update_right_shift,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; learning_rate_q16: the fixed-point multiplier controlling update magnitude; momentum_q16: the value named 'momentum q16' in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 930: `update_right_shift=update_right_shift,` This is continuation line 5 of the 6-line logical statement that begins at source line 926. Its exact fragment is 'update_right_shift=update_right_shift,'. The important operands are update_right_shift: the value named 'update right shift' in this block. Read together, the complete operation is: This assigns 'sgd_command(learning_rate_q16, momentum_q16, weight_decay_q16, update_right_shift=update_right_shift,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; learning_rate_q16: the fixed-point multiplier controlling update magnitude; momentum_q16: the value named 'momentum q16' in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 931: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 926. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_command(learning_rate_q16, momentum_q16, weight_decay_q16, update_right_shift=update_right_shift,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; learning_rate_q16: the fixed-point multiplier controlling update magnitude; momentum_q16: the value named 'momentum q16' in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 932: `for start in range(0, flat_weight.size, 8):` This iterates start over 'range(0, flat_weight.size, 8)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_sgdw_update, which handles execute sgdw update.

Line 933: `stop = min(start + 8, flat_weight.size)` This assigns 'min(start + 8, flat_weight.size)' to 'stop ' in host memory. Here stop means the value named 'stop' in this block. The important operands are min: the value named 'min' in this block; start: the value named 'start' in this block; flat_weight: a learned coefficient multiplied by an activation. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 934: `active = stop - start` This assigns 'stop - start' to 'active ' in host memory. Here active means the value named 'active' in this block. The important operands are stop: the value named 'stop' in this block; start: the value named 'start' in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 936: `def padded(values):` This starts function padded. Its indented body performs padded as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 937: `return tuple(map(int, values)) + (0,) * (8 - active)` This returns 'tuple(map(int, values)) + (0,) * (8 - active)' to the caller of the function padded, which handles padded, ending this invocation. The important operands are tuple: the value named 'tuple' in this block; map: the value named 'map' in this block; values: the value named 'values' in this block.

Line 939: `words = driver.execute(` This statement continues through source line 948; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 940: `command,` This is continuation line 2 of the 10-line logical statement that begins at source line 939. Its exact fragment is ‘command,‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 941: `pack_optimizer_beats(` This is continuation line 3 of the 10-line logical statement that begins at source line 939. Its exact fragment is ‘pack_optimizer_beats(‘. The important operands are pack_optimizer_beats: the value named ’pack optimizer beats’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 942: `padded(flat_weight[start:stop]),` This is continuation line 4 of the 10-line logical statement that begins at source line 939. Its exact fragment is ‘padded(flat_weight[start:stop]),‘. The important operands are padded: the value named ’padded’ in this block; flat_weight: a learned coefficient multiplied by an activation; start: the value named ’start’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 943: `padded(flat_gradient[start:stop]),` This is continuation line 5 of the 10-line logical statement that begins at source line 939. Its exact fragment is ‘padded(flat_gradient[start:stop]),‘. The important operands are padded: the value named ’padded’ in this block; flat_gradient: the signed loss derivative used to update a learned parameter; start: the value named ’start’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 944: `padded(flat_velocity[start:stop]),` This is continuation line 6 of the 10-line logical statement that begins at source line 939. Its exact fragment is ‘padded(flat_velocity[start:stop]),‘. The important operands are padded: the value named ’padded’ in this block; flat_velocity: the momentum history retained by the SGD optimizer; start: the value named ’start’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 945: `),` This is continuation line 7 of the 10-line logical statement that begins at source line 939. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 946: `output_beats=8,` This is continuation line 8 of the 10-line logical statement that begins at source line 939. Its exact fragment is ‘output_beats=8,‘. The important operands are output_beats: the value named ’output beats’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 947: `timeout_seconds=timeout_seconds,` This is continuation line 9 of the 10-line logical statement that begins at source line 939. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 948: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 939. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘driver.execute(command, pack_optimizer_beats(padded(flat_weight[start:stop]), padded(flat_gradient[start:stop]), padded(flat_velocity[start:stop]),), output_beats=8, timeout_seconds=timeout_seconds,)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 949: `updated_weight, updated_velocity = unpack_optimizer_result(words)` This assigns ‘unpack_optimizer_result to ‘updated_weight, updated_velocity ‘ in host memory. Here updated_weight means a learned coefficient multiplied by an activation. The important operands are unpack_optimizer_result: the value named ’unpack optimizer result’ in this block; words: the value named ’words’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 950: `next_weight[start:stop] = updated_weight[:active]` This assigns ‘updated_weight[:active]‘ to ‘next_weight[sta in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are updated_weight: a learned coefficient multiplied by an activation; active: the value named ’active’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 951: `next_velocity[start:stop] = updated_velocity[:active]` This assigns ‘updated_velocity[:active]‘ to ‘next_velocity[start‘ in host memory. Here next_velocity means the momentum history retained by the SGD optimizer. The important operands are updated_velocity: the momentum history retained by the SGD optimizer; active: the value named ’active’ in this block. Context: the function execute_sgdw_update, which handles execute sgdw update.

Line 952: `return next_weight.reshape(weight.shape), next_velocity.reshape(weight.shape)` This returns ‘next_weight.reshape(weight.shape), next_velocity.reshape(weight.shape)‘ to the caller of the function execute_sgdw_update,

which handles execute sgdw update, ending this invocation. The important operands are next_weight: a learned coefficient multiplied by an activation; reshape: the value named 'reshape' in this block; weight: a learned coefficient multiplied by an activation.

Line 955: `def execute_int8_weight_projection(` This starts function execute_int8_weight_projection. Its indented body performs execute int8 weight projection as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 956: `driver: TrainingCommandExecutor,` Inside the function execute_int8_weight_projection, which handles execute int8 weight projection, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named 'driver' in this block; TrainingCommandExecutor: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 957: `master_weights,` Inside the function execute_int8_weight_projection, which handles execute int8 weight projection, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are master_weights: a learned coefficient multiplied by an activation.

Line 958: `*,` Inside the function execute_int8_weight_projection, which handles execute int8 weight projection, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 959: `timeout_seconds: float = 5.0,` This assigns ‘5.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 960: `) -> np.ndarray:` Inside the function execute_int8_weight_projection, which handles execute int8 weight projection, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are ndarray: the value named 'ndarray' in this block.

Line 961: `"""Clamp signed-s16 master weights to the deployed signed-int8 tensor."""` This is executable documentation attached to the function execute_int8_weight_projection, which handles execute int8 weight projection. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 963: `master = np.asarray(master_weights, dtype=np.int64)` This assigns ‘np.asarray(master_weights, dtype=np.int6
to ‘master ‘ in host memory. Here master means the value named 'master' in this block. The important operands are asarray: the value named 'asarray' in this block; master_weights: a learned coefficient multiplied by an activation; dtype: the value named 'dtype' in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 964: `if master.size == 0 or np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1):` This executes the following indented host-code branch only when ‘master.size == 0 or np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1)‘ is true. The important operands are master: the value named 'master' in this block; size: the value named 'size' in this block; or: the value named 'or' in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 965: `raise ValueError("master_weights must be a non-empty signed-16 tensor")` This rejects an invalid state by raising ‘ValueError("master_weights must be a non-empty signed-16 tensor")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 966: `flat = master.reshape(-1)` This assigns ‘master.reshape(-1)‘ to ‘flat ‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are master: the value named ’master’ in this block; reshape: the value named ’reshape’ in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 967: `result = np.zeros_like(flat)` This assigns ‘np.zeros_like(flat)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are zeros_like: the value named ’zeros like’ in this block; flat: the value named ’flat’ in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 968: `for start in range(0, flat.size, 8):` This iterates start over ‘range(0, flat.size, 8)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 969: `stop = min(start + 8, flat.size)` This assigns ‘min(start + 8, flat.size)‘ to ‘stop ‘ in host memory. Here stop means the value named ’stop’ in this block. The important operands are min: the value named ’min’ in this block; start: the value named ’start’ in this block; flat: the value named ’flat’ in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 970: `active = stop - start` This assigns ‘stop - start‘ to ‘active ‘ in host memory. Here active means the value named ’active’ in this block. The important operands are stop: the value named ’stop’ in this block; start: the value named ’start’ in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 971: `operand = tuple(map(int, flat[start:stop])) + (0,) * (8 - active)` This assigns ‘tuple(map(int, flat[start:stop])) + (0,) * (8 - active)‘ to ‘operand ‘ in host memory. Here operand means the value named ’operand’ in this block. The important operands are tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block; flat: the value named ’flat’ in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 972: `clamped = _execute_vector8(` This statement continues through source line 980; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_execute_vector8(driver, VectorOperation.CLAMP, operand, (0,) * 8, timeout_seconds=timeout_seconds, clamp_minimum=-128, clamp_maximum=127,)‘ to ‘clamped ‘ in host memory. Here clamped means the value named ’clamped’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 973: `driver,` This is continuation line 2 of the 9-line logical statement that begins at source line 972. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.CLAMP, operand, (0,) * 8, timeout_seconds=timeout_seconds, clamp_minimum=-128, clamp_maximum=127,)‘ to ‘clamped ‘ in host memory. Here clamped means the value named ’clamped’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 974: `VectorOperation.CLAMP,` This is continuation line 3 of the 9-line logical statement that begins at source line 972. Its exact fragment is ‘VectorOperation.CLAMP,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; CLAMP: the value named ’clamp’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.CLAMP, operand, (0,) * 8, timeout_seconds=timeout_seconds, clamp_minimum=-128, clamp_maximum=127,)‘ to ‘clamped ‘ in host memory. Here clamped means the value named ’clamped’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 975: `operand,` This is continuation line 4 of the 9-line logical statement that begins at source line 972. Its exact fragment is 'operand,'. The important operands are operand: the value named 'operand' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.CLAMP, operand, (0,) * 8, timeout_seconds=timeout_seconds, clamp_minimum=-128, clamp_maximum=127,)' to 'clamped ' in host memory. Here clamped means the value named 'clamped' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 976: `(0,) * 8,` This is continuation line 5 of the 9-line logical statement that begins at source line 972. Its exact fragment is '(0,) * 8,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.CLAMP, operand, (0,) * 8, timeout_seconds=timeout_seconds, clamp_minimum=-128, clamp_maximum=127,)' to 'clamped ' in host memory. Here clamped means the value named 'clamped' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 977: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 9-line logical statement that begins at source line 972. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.CLAMP, operand, (0,) * 8, timeout_seconds=timeout_seconds, clamp_minimum=-128, clamp_maximum=127,)' to 'clamped ' in host memory. Here clamped means the value named 'clamped' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 978: `clamp_minimum=-128,` This is continuation line 7 of the 9-line logical statement that begins at source line 972. Its exact fragment is 'clamp_minimum=-128,'. The important operands are clamp_minimum: the value named 'clamp minimum' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.CLAMP, operand, (0,) * 8, timeout_seconds=timeout_seconds, clamp_minimum=-128, clamp_maximum=127,)' to 'clamped ' in host memory. Here clamped means the value named 'clamped' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 979: `clamp_maximum=127,` This is continuation line 8 of the 9-line logical statement that begins at source line 972. Its exact fragment is 'clamp_maximum=127,'. The important operands are clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.CLAMP, operand, (0,) * 8, timeout_seconds=timeout_seconds, clamp_minimum=-128, clamp_maximum=127,)' to 'clamped ' in host memory. Here clamped means the value named 'clamped' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 980: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 972. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.CLAMP, operand, (0,) * 8, timeout_seconds=timeout_seconds, clamp_minimum=-128, clamp_maximum=127,)' to 'clamped ' in host memory. Here clamped means the value named 'clamped' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 981: `result[start:stop] = clamped[:active]` This assigns 'clamped[:active]' to 'result[start' in host memory. Here result means the value named 'result' in this block. The important operands are clamped: the value named 'clamped'

in this block; active: the value named 'active' in this block. Context: the function execute_int8_weight_projection, which handles execute int8 weight projection.

Line 982: `return result.reshape(master.shape).astype(np.int8)` This returns 'result.reshape(master.shape).astype(np.int to the caller of the function execute_int8_weight_projection, which handles execute int8 weight projection, ending this invocation. The important operands are result: the value named 'result' in this block; reshape: the value named 'reshape' in this block; master: the value named 'master' in this block.

Line 985: `def execute_lif_bptt(` This starts function execute_lif_bptt. Its indented body performs execute lif bptt as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 986: `driver: TrainingCommandExecutor,` Inside the function execute_lif_bptt, which handles execute lif bptt, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named 'driver' in this block; TrainingCommandExecutor: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 987: `pre_reset_membrane,` Inside the function execute_lif_bptt, which handles execute lif bptt, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps.

Line 988: `spikes,` Inside the function execute_lif_bptt, which handles execute lif bptt, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are spikes: a binary event produced when a neuron crosses its threshold.

Line 989: `spike_gradients,` Inside the function execute_lif_bptt, which handles execute lif bptt, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are spike_gradients: a binary event produced when a neuron crosses its threshold.

Line 990: `*,` Inside the function execute_lif_bptt, which handles execute lif bptt, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 991: `threshold: int,` Inside the function execute_lif_bptt, which handles execute lif bptt, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are threshold: the membrane level at which a neuron emits a spike.

Line 992: `membrane_gradients=None,` This assigns 'None,' to 'membrane_gradients' in host memory. Here membrane_gradients means the persistent LIF neuron state carrying charge between timesteps. The important operands are None: the value named 'none' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 993: `reciprocal_tau_q15: int = 1 << 14,` This assigns '1 << 14,' to 'reciprocal_tau_q15' in host memory. Here reciprocal_tau_q15 means the value named 'reciprocal tau q15' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 994: `surrogate_table_q15=None,` This assigns 'None,' to 'surrogate_table_q15' in host memory. Here surrogate_table_q15 means the value named 'surrogate table q15' in this block. The important operands are None: the value named 'none' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 995: `surrogate_address_shift: int = 5,` This assigns ‘5,‘ to ‘surrogate_address_shift‘ in host memory. Here surrogate_address_shift means the value named ’surrogate address shift’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 996: `load_surrogate_table: bool = True,` This assigns ‘True,‘ to ‘load_surrogate_table‘ in host memory. Here load_surrogate_table means the value named ’load surrogate table’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 997: `timeout_seconds: float = 5.0,` This assigns ‘5.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 998: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function execute_lif_bptt, which handles execute lif bptt, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named ’tuple’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 999: `"""Execute detached-reset hard-LIF BPTT entirely through fabric operators."""` This is executable documentation attached to the function execute_lif_bptt, which handles execute lif bptt. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1001: `membrane = np.asarray(pre_reset_membrane, dtype=np.int64)` This assigns ‘np.asarray(pre_reset_membrane, dtype=np.int64)‘ to ‘membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are asarray: the value named ’asarray’ in this block; pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; dtype: the value named ’dtype’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1002: `spike = np.asarray(spikes, dtype=np.int64)` This assigns ‘np.asarray(spikes, dtype=np.int64)‘ to ‘spike ‘ in host memory. Here spike means a binary event produced when a neuron crosses its threshold. The important operands are asarray: the value named ’asarray’ in this block; spikes: a binary event produced when a neuron crosses its threshold; dtype: the value named ’dtype’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1003: `grad_spike = np.asarray(spike_gradients, dtype=np.int64)` This assigns ‘np.asarray(spike_gradients, dtype=np.int64)‘ to ‘grad_spike ‘ in host memory. Here grad_spike means a binary event produced when a neuron crosses its threshold. The important operands are asarray: the value named ’asarray’ in this block; spike_gradients: a binary event produced when a neuron crosses its threshold; dtype: the value named ’dtype’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1004: `if membrane.ndim != 2 or spike.shape != membrane.shape or grad_spike.shape != membrane.shape:` This executes the following indented host-code branch only when ‘membrane.ndim != 2 or spike.shape != membrane.shape or grad_spike.shape != membrane.shape‘ is true. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1005: `raise ValueError("LIF trace, spike, and gradient arrays must share shape [time, neurons]")` This rejects an invalid state by raising ‘ValueError("LIF trace, spike, and gradient arrays must share shape [time, neurons]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1006: `if membrane_gradients is None:` This executes the following indented host-code branch only when ‘membrane_gradients is None‘ is true. The important operands are membrane_gradients: the persistent LIF neuron state carrying charge between timesteps; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1007: `grad_v = np.zeros_like(membrane)` This assigns ‘np.zeros_like(membrane)‘ to ‘grad_v ‘ in host memory. Here grad_v means the value named ’grad v’ in this block. The important operands are zeros_like: the value named ’zeros like’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1008: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute_lif_bptt, which handles execute lif bptt.

Line 1009: `grad_v = np.asarray(membrane_gradients, dtype=np.int64)` This assigns ‘np.asarray(membrane_gradients, dtype=np.int64)‘ to ‘grad_v ‘ in host memory. Here grad_v means the value named ’grad v’ in this block. The important operands are asarray: the value named ’asarray’ in this block; membrane_gradients: the persistent LIF neuron state carrying charge between timesteps; dtype: the value named ’dtype’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1010: `if grad_v.shape != membrane.shape:` This executes the following indented host-code branch only when ‘grad_v.shape != membrane.shape‘ is true. The important operands are grad_v: the value named ’grad v’ in this block; shape: the value named ’shape’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1011: `raise ValueError("membrane gradients must match the LIF trace shape")` This rejects an invalid state by raising ‘ValueError("membrane gradients must match the LIF trace shape")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1012: `if surrogate_table_q15 is None:` This executes the following indented host-code branch only when ‘surrogate_table_q15 is None‘ is true. The important operands are surrogate_table_q15: the value named ’surrogate table q15’ in this block; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1013: `surrogate_table_q15, _ = sigmoid_surrogate_lut_q15(` This statement continues through source line 1015; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sigmoid_surrogate_lut_q15(address_shift=surrogate_address_shift)‘ to ‘surrogate_table_q15, _ ‘ in host memory. Here surrogate_table_q15 means the value named ’surrogate table q15’ in this block. The important operands are sigmoid_surrogate_lut_q15: the value named ’sigmoid surrogate lut q15’ in this block; address_shift: the value named ’address shift’ in this block; surrogate_address_shift: the value named ’surrogate address shift’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1014: `address_shift=surrogate_address_shift` This is continuation line 2 of the 3-line logical statement that begins at source line 1013. Its exact fragment is ‘address_shift=surrogate_address_shift‘. The important operands are address_shift: the value named ’address shift’ in this block; surrogate_address_shift: the value named ’surrogate address shift’ in this block. Read together, the complete operation is: This assigns ‘sigmoid_surrogate_lut_q15(address_shift=surrogate_address_shift)‘ to ‘surrogate_table_q15, _ ‘ in host memory. Here surrogate_table_q15 means the value named ’surrogate table q15’ in this block. The important operands are sigmoid_surrogate_lut_q15: the value named ’sigmoid surrogate lut q15’ in this block; address_shift: the value named ’address shift’ in this block; surrogate_address_shift: the value named ’surrogate address shift’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1015: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 1013. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘sigmoid_surrogate_lut_q15(address_shift=surrogate_address_shift)‘ to ‘surrogate_table_q15, _ ‘ in host memory. Here surrogate_table_q15 means the value named ’surrogate table q15’ in this block. The important operands are sigmoid_surrogate_lut_q15: the value named ’sigmoid surrogate lut q15’ in this block; address_shift: the value named ’address shift’ in this block; surrogate_address_shift: the value named ’surrogate address shift’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1016: `table = np.asarray(surrogate_table_q15, dtype=np.int64)` This assigns ‘np.asarray(surrogate_table_q15, dtype=np.int64)‘ to ‘table ‘ in host memory. Here table means the value named ’table’ in this block. The important operands are asarray: the value named ’asarray’ in this block; surrogate_table_q15: the value named ’surrogate table q15’ in this block; dtype: the value named ’dtype’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1017: `if table.shape != (256,):` This executes the following indented host-code branch only when ‘table.shape != (256,)‘ is true. The important operands are table: the value named ’table’ in this block; shape: the value named ’shape’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1018: `raise ValueError("surrogate table must contain exactly 256 entries")` This rejects an invalid state by raising ‘ValueError("surrogate table must contain exactly 256 entries")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1019: `if load_surrogate_table:` This executes the following indented host-code branch only when ‘load_surrogate_table‘ is true. The important operands are load_surrogate_table: the value named ’load surrogate table’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1020: `load_words = driver.execute(` This statement continues through source line 1026; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘driver.execute(lut_command(LutSuboperation.LOA 256), pack_lut_load_beats(tuple((address, int(table[address])) for address in range(256))), output_beats=0, timeout_seconds=timeout_seconds,)‘ to ‘load_words ‘ in host memory. Here load_words means the value named ’load words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1021: `lut_command(LutSuboperation.LOAD, 256),` This is continuation line 2 of the 7-line logical statement that begins at source line 1020. Its exact fragment is ‘lut_command(LutSuboperation.LOAD, 256),‘. The important operands are lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; LutSuboperation: the value named ’lut suboperation’ in this block; LOAD: the value named ’load’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(lut_command(LutSuboperation.LOAD, 256), pack_lut_load_beats(tuple((address, int(table[address])) for address in range(256))), output_beats=0, timeout_seconds=timeout_seconds,)‘ to ‘load_words ‘ in host memory. Here load_words means the value named ’load words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1022: `pack_lut_load_beats(` This is continuation line 3 of the 7-line logical statement that begins at source line 1020. Its exact fragment is ‘pack_lut_load_beats(‘. The important operands are pack_lut_load_beats: the value named ’pack lut load beats’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(lut_command(LutSuboperation.LOAD, 256), pack_lut_load_beats(tuple((address, int(table[address])) for address in range(256))), output_beats=0, timeout_seconds=timeout_seconds,)‘ to ‘load_words ‘ in host memory. Here load_words means the value named ’load words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1023: `tuple((address, int(table[address])) for address in range(256))),` This is continuation line 4 of the 7-line logical statement that begins at source line 1020. Its exact fragment is ‘tuple((address, int(table[address])) for address in range(256))),‘. The important operands are tuple: the value named ’tuple’ in this block; address: the value named ’address’ in this block; table: the value named ’table’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(lut_command(LutSuboperation.LOAD, 256), pack_lut_load_beats(tuple((address, int(table[address])) for address in range(256))), output_beats=0, timeout_seconds=timeout_seconds,)‘ to ‘load_words ‘ in host memory. Here load_words means the value named ’load words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1024: `output_beats=0,` This is continuation line 5 of the 7-line logical statement that begins at source line 1020. Its exact fragment is 'output_beats=0,'. The important operands are output_beats: the value named 'output beats' in this block. Read together, the complete operation is: This assigns 'driver.execute(lut_command(LutSuboperation.LOAD, 256), pack_lut_load_beats(tuple((address, int(table[address])) for address in range(256))), output_beats=0, timeout_seconds=timeout_seconds,)' to 'load_words ' in host memory. Here load_words means the value named 'load words' in this block. The important operands are driver: the value named 'driver' in this block; execute: the value named 'execute' in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1025: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1020. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'driver.execute(lut_command(LutSuboperation.LOAD, 256), pack_lut_load_beats(tuple((address, int(table[address])) for address in range(256))), output_beats=0, timeout_seconds=timeout_seconds,)' to 'load_words ' in host memory. Here load_words means the value named 'load words' in this block. The important operands are driver: the value named 'driver' in this block; execute: the value named 'execute' in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1026: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1020. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'driver.execute(lut_command(LutSuboperation.LOAD, 256), pack_lut_load_beats(tuple((address, int(table[address])) for address in range(256))), output_beats=0, timeout_seconds=timeout_seconds,)' to 'load_words ' in host memory. Here load_words means the value named 'load words' in this block. The important operands are driver: the value named 'driver' in this block; execute: the value named 'execute' in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1027: `if load_words:` This executes the following indented host-code branch only when 'load_words' is true. The important operands are load_words: the value named 'load words' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1028: `raise RuntimeError("LUT load unexpectedly returned stream data")` This rejects an invalid state by raising 'RuntimeError("LUT load unexpectedly returned stream data")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1030: `addresses = surrogate_lut_addresses(membrane, threshold, surrogate_address_shift)` This assigns 'surrogate_lut_addresses(membrane, threshold, surrogate_address_shift)' to 'addresses ' in host memory. Here addresses means the value named 'addresses' in this block. The important operands are surrogate_lut_addresses: the value named 'surrogate lut addresses' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; threshold: the membrane level at which a neuron emits a spike. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1031: `time_steps, neurons = membrane.shape` This assigns 'membrane.shape' to 'time_steps, neurons ' in host memory. Here time_steps means the value named 'time steps' in this block. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; shape: the value named 'shape' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1032: `grad_input = np.zeros_like(membrane)` This assigns 'np.zeros_like(membrane)' to 'grad_input ' in host memory. Here grad_input means the value named 'grad input' in this block. The important operands are zeros_like: the value named 'zeros like' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1033: `grad_h_trace = np.zeros_like(membrane)` This assigns 'np.zeros_like(membrane)' to 'grad_h_trace ' in host memory. Here grad_h_trace means the value named 'grad h trace' in this block. The important operands

are zeros_like: the value named 'zeros like' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1034: `grad_h_next = np.zeros(neurons, dtype=np.int64)` This assigns 'np.zeros(neurons, dtype=np.int64)' to 'grad_h_next ' in host memory. Here grad_h_next means the value named 'grad h next' in this block. The important operands are zeros: the value named 'zeros' in this block; neurons: the value named 'neurons' in this block; dtype: the value named 'dtype' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1035: `leak_q15 = (1 << 15) - int(reciprocal_tau_q15)` This assigns '(1 << 15) - int(reciprocal_tau_q15)' to 'leak_q15 ' in host memory. Here leak_q15 means the retained fraction of prior membrane state. The important operands are reciprocal_tau_q15: the value named 'reciprocal tau q15' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1036: `for timestep in range(time_steps - 1, -1, -1):` This iterates timestep over 'range(time_steps - 1, -1, -1)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_lif_bptt, which handles execute lif bptt.

Line 1037: `for start in range(0, neurons, 8):` This iterates start over 'range(0, neurons, 8)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_lif_bptt, which handles execute lif bptt.

Line 1038: `stop = min(start + 8, neurons)` This assigns 'min(start + 8, neurons)' to 'stop ' in host memory. Here stop means the value named 'stop' in this block. The important operands are min: the value named 'min' in this block; start: the value named 'start' in this block; neurons: the value named 'neurons' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1039: `active = stop - start` This assigns 'stop - start' to 'active ' in host memory. Here active means the value named 'active' in this block. The important operands are stop: the value named 'stop' in this block; start: the value named 'start' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1041: `def padded(values, fill=0):` This starts function padded. Its indented body performs padded as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 1042: `return tuple(map(int, values)) + (fill,) * (8 - active)` This returns 'tuple(map(int, values)) + (fill,) * (8 - active)' to the caller of the function padded, which handles padded, ending this invocation. The important operands are tuple: the value named 'tuple' in this block; map: the value named 'map' in this block; values: the value named 'values' in this block.

Line 1044: `temporal = _execute_vector8(` This statement continues through source line 1050; the explanation uses the complete expression rather than only the first visual fragment. This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_h_next[start:stop]), (leak_q15,) * 8, timeout_seconds=timeout_seconds,)' to 'temporal ' in host memory. Here temporal means the value named 'temporal' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1045: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1044. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_h_next[start:stop]), (leak_q15,) * 8, timeout_seconds=timeout_seconds,)' to 'temporal ' in host memory. Here temporal means the value named 'temporal' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1046: `VectorOperation.MULTIPLY_Q15,` This is continuation line 3 of the 7-line logical statement that begins at source line 1044. Its exact fragment is 'VectorOperation.MULTIPLY_Q15,'. The important operands are VectorOperation: the value named 'vector operation' in this block; MULTIPLY_Q15: the value named 'multiply q15' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_h_next[start:stop]), (leak_q15,) * 8, timeout_seconds=timeout_seconds,)' to 'temporal ' in host memory. Here temporal means the value named 'temporal' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1047: `padded(grad_h_next[start:stop]),` This is continuation line 4 of the 7-line logical statement that begins at source line 1044. Its exact fragment is 'padded(grad_h_next[start:stop]),'. The important operands are padded: the value named 'padded' in this block; grad_h_next: the value named 'grad h next' in this block; start: the value named 'start' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_h_next[start:stop]), (leak_q15,) * 8, timeout_seconds=timeout_seconds,)' to 'temporal ' in host memory. Here temporal means the value named 'temporal' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1048: `(leak_q15,) * 8,` This is continuation line 5 of the 7-line logical statement that begins at source line 1044. Its exact fragment is '(leak_q15,) * 8,'. The important operands are leak_q15: the retained fraction of prior membrane state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_h_next[start:stop]), (leak_q15,) * 8, timeout_seconds=timeout_seconds,)' to 'temporal ' in host memory. Here temporal means the value named 'temporal' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1049: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1044. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_h_next[start:stop]), (leak_q15,) * 8, timeout_seconds=timeout_seconds,)' to 'temporal ' in host memory. Here temporal means the value named 'temporal' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1050: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1044. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_h_next[start:stop]), (leak_q15,) * 8, timeout_seconds=timeout_seconds,)' to 'temporal ' in host memory. Here temporal means the value named 'temporal' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1051: `membrane_path = _execute_vector8(` This statement continues through source line 1057; the explanation uses the complete expression rather than only the first visual fragment. This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, padded(grad_v[timestep, start:stop]), temporal, timeout_seconds=timeout_seconds,)' to 'membrane_path ' in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1052: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1051. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, padded(grad_v[timestep,

start:stop]), temporal, timeout_seconds=timeout_seconds,)‘ to ‘membrane_path ‘ in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1053: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 1051. Its exact fragment is ‘VectorOperation.ADD_SAT,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; ADD_SAT: the value named ’add sat’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, padded(grad_v[timestep, start:stop]), temporal, timeout_seconds=timeout_seconds,)‘ to ‘membrane_path ‘ in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1054: `padded(grad_v[timestep, start:stop]),` This is continuation line 4 of the 7-line logical statement that begins at source line 1051. Its exact fragment is ‘padded(grad_v[timestep, start:stop]),‘. The important operands are padded: the value named ’padded’ in this block; grad_v: the value named ’grad v’ in this block; timestep: one discrete SNN update instant. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, padded(grad_v[timestep, start:stop]), temporal, timeout_seconds=timeout_seconds,)‘ to ‘membrane_path ‘ in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1055: `temporal,` This is continuation line 5 of the 7-line logical statement that begins at source line 1051. Its exact fragment is ‘temporal,‘. The important operands are temporal: the value named ’temporal’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, padded(grad_v[timestep, start:stop]), temporal, timeout_seconds=timeout_seconds,)‘ to ‘membrane_path ‘ in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1056: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1051. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, padded(grad_v[timestep, start:stop]), temporal, timeout_seconds=timeout_seconds,)‘ to ‘membrane_path ‘ in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1057: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1051. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, padded(grad_v[timestep, start:stop]), temporal, timeout_seconds=timeout_seconds,)‘ to ‘membrane_path ‘ in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1058: `membrane_path = _execute_vector8(` This statement continues through source line 1064; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_execute_vector8(driver, VectorOperation.SELECT_ZERO, padded(spike[timestep, start:stop]), membrane_path, timeout_seconds=timeout_seconds,)‘ to ‘membrane_path ‘ in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1059: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1058. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_ZERO, padded(spike[timestep, start:stop]), membrane_path, timeout_seconds=timeout_seconds,)' to 'membrane_path ' in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1060: `VectorOperation.SELECT_ZERO,` This is continuation line 3 of the 7-line logical statement that begins at source line 1058. Its exact fragment is 'VectorOperation.SELECT_ZERO,'. The important operands are VectorOperation: the value named 'vector operation' in this block; SELECT_ZERO: the value named 'select zero' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_ZERO, padded(spike[timestep, start:stop]), membrane_path, timeout_seconds=timeout_seconds,)' to 'membrane_path ' in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1061: `padded(spike[timestep, start:stop]),` This is continuation line 4 of the 7-line logical statement that begins at source line 1058. Its exact fragment is 'padded(spike[timestep, start:stop]),'. The important operands are padded: the value named 'padded' in this block; spike: a binary event produced when a neuron crosses its threshold; timestep: one discrete SNN update instant. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_ZERO, padded(spike[timestep, start:stop]), membrane_path, timeout_seconds=timeout_seconds,)' to 'membrane_path ' in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1062: `membrane_path,` This is continuation line 5 of the 7-line logical statement that begins at source line 1058. Its exact fragment is 'membrane_path,'. The important operands are membrane_path: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_ZERO, padded(spike[timestep, start:stop]), membrane_path, timeout_seconds=timeout_seconds,)' to 'membrane_path ' in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1063: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1058. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_ZERO, padded(spike[timestep, start:stop]), membrane_path, timeout_seconds=timeout_seconds,)' to 'membrane_path ' in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1064: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1058. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_ZERO, padded(spike[timestep, start:stop]), membrane_path, timeout_seconds=timeout_seconds,)' to 'membrane_path ' in host memory. Here membrane_path means the persistent LIF neuron state carrying charge between timesteps. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1065: `lut_words = driver.execute(` This statement continues through source line 1070; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘driver.execute(lut_command(LutSuboperation.EVA 8), pack_lut_eval_beat(padded(addresses[timestep, start:stop], 128)), output_beats=2, timeout_seconds=timeout_seconds,)‘ to ‘lut_words ‘ in host memory. Here lut_words means the value named ’lut words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1066: `lut_command(LutSuboperation.EVALUATE, 8),` This is continuation line 2 of the 6-line logical statement that begins at source line 1065. Its exact fragment is ‘lut_command(LutSuboperation.EVALUATE, 8),‘. The important operands are lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; LutSuboperation: the value named ’lut suboperation’ in this block; EVALUATE: the value named ’evaluate’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(lut_command(LutSuboperation.EVALUATE, 8), pack_lut_eval_beat(padded(addresses[timestep, start:stop], 128)), output_beats=2, timeout_seconds=timeout_seconds,)‘ to ‘lut_words ‘ in host memory. Here lut_words means the value named ’lut words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1067: `pack_lut_eval_beat(padded(addresses[timestep, start:stop], 128)),` This is continuation line 3 of the 6-line logical statement that begins at source line 1065. Its exact fragment is ‘pack_lut_eval_beat(padded(addresses[timestep, start:stop], 128)),‘. The important operands are pack_lut_eval_beat: the value named ’pack lut eval beat’ in this block; padded: the value named ’padded’ in this block; addresses: the value named ’addresses’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(lut_command(LutSuboperation.EVALUATE, 8), pack_lut_eval_beat(padded(addresses[timestep, start:stop], 128)), output_beats=2, timeout_seconds=timeout_seconds,)‘ to ‘lut_words ‘ in host memory. Here lut_words means the value named ’lut words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1068: `output_beats=2,` This is continuation line 4 of the 6-line logical statement that begins at source line 1065. Its exact fragment is ‘output_beats=2,‘. The important operands are output_beats: the value named ’output beats’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(lut_command(LutSuboperation.EVALUATE, 8), pack_lut_eval_beat(padded(addresses[timestep, start:stop], 128)), output_beats=2, timeout_seconds=timeout_seconds,)‘ to ‘lut_words ‘ in host memory. Here lut_words means the value named ’lut words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1069: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1065. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(lut_command(LutSuboperation.EVALUATE, 8), pack_lut_eval_beat(padded(addresses[timestep, start:stop], 128)), output_beats=2, timeout_seconds=timeout_seconds,)‘ to ‘lut_words ‘ in host memory. Here lut_words means the value named ’lut words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1070: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1065. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘driver.execute(lut_command(LutSuboperation.EVALUATE, 8), pack_lut_eval_beat(padded(addresses[time start:stop], 128)), output_beats=2, timeout_seconds=timeout_seconds,)‘ to ‘lut_words ‘ in host memory. Here lut_words means the value named ’lut words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1071: `derivative = np.asarray(unpack_lut_result(lut_words), dtype=np.int64)` This assigns ‘np.asarray(unpack_ dtype=np.int64)‘ to ‘derivative ‘ in host memory. Here derivative means the value named ’derivative’ in this block. The important operands are asarray: the value named ’asarray’ in this block; unpack_lut_result: the value named ’unpack lut result’ in this block; lut_words: the value named ’lut words’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1072: `spike_path = _execute_vector8(` This statement continues through source line 1078; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_spike[timestep, start:stop]), derivative, timeout_seconds=timeout_seconds,)‘ to ‘spike_path ‘ in host memory. Here spike_path means a binary event produced when a neuron crosses its threshold. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1073: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1072. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_spike[timestep, start:stop]), derivative, timeout_seconds=timeout_seconds,)‘ to ‘spike_path ‘ in host memory. Here spike_path means a binary event produced when a neuron crosses its threshold. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1074: `VectorOperation.MULTIPLY_Q15,` This is continuation line 3 of the 7-line logical statement that begins at source line 1072. Its exact fragment is ‘VectorOperation.MULTIPLY_Q15,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; MULTIPLY_Q15: the value named ’multiply q15’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_spike[timestep, start:stop]), derivative, timeout_seconds=timeout_seconds,)‘ to ‘spike_path ‘ in host memory. Here spike_path means a binary event produced when a neuron crosses its threshold. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1075: `padded(grad_spike[timestep, start:stop]),` This is continuation line 4 of the 7-line logical statement that begins at source line 1072. Its exact fragment is ‘padded(grad_spike[timestep, start:stop]),‘. The important operands are padded: the value named ’padded’ in this block; grad_spike: a binary event produced when a neuron crosses its threshold; timestep: one discrete SNN update instant. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_spike[timestep, start:stop]), derivative, timeout_seconds=timeout_seconds,)‘ to ‘spike_path ‘ in host memory. Here spike_path means a binary event produced when a neuron crosses its threshold. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1076: `derivative,` This is continuation line 5 of the 7-line logical statement that begins at source line 1072. Its exact fragment is ‘derivative,‘. The important operands are derivative: the value named ’derivative’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_spike[timestep, start:stop]), derivative, timeout_seconds=timeout_seconds,)‘ to ‘spike_path ‘ in host memory. Here spike_path means a binary event produced when a neuron crosses its threshold. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1077: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1072. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_spike[timestep, start:stop]), derivative, timeout_seconds=timeout_seconds,)‘ to ‘spike_path ‘ in host memory. Here spike_path means a binary event

produced when a neuron crosses its threshold. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1078: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1072. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, padded(grad_spike[timestep, start:stop]), derivative, timeout_seconds=timeout_seconds,)' to 'spike_path ' in host memory. Here spike_path means a binary event produced when a neuron crosses its threshold. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1079: `grad_h = _execute_vector8(` This statement continues through source line 1085; the explanation uses the complete expression rather than only the first visual fragment. This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, membrane_path, spike_path, timeout_seconds=timeout_seconds,)' to 'grad_h ' in host memory. Here grad_h means the value named 'grad h' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1080: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1079. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, membrane_path, spike_path, timeout_seconds=timeout_seconds,)' to 'grad_h ' in host memory. Here grad_h means the value named 'grad h' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1081: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 1079. Its exact fragment is 'VectorOperation.ADD_SAT,'. The important operands are VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named 'add sat' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, membrane_path, spike_path, timeout_seconds=timeout_seconds,)' to 'grad_h ' in host memory. Here grad_h means the value named 'grad h' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1082: `membrane_path,` This is continuation line 4 of the 7-line logical statement that begins at source line 1079. Its exact fragment is 'membrane_path,'. The important operands are membrane_path: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, membrane_path, spike_path, timeout_seconds=timeout_seconds,)' to 'grad_h ' in host memory. Here grad_h means the value named 'grad h' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1083: `spike_path,` This is continuation line 5 of the 7-line logical statement that begins at source line 1079. Its exact fragment is 'spike_path,'. The important operands are spike_path: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, membrane_path, spike_path, timeout_seconds=timeout_seconds,)' to 'grad_h ' in host memory. Here grad_h means the value named 'grad h' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1084: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1079. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, membrane_path, spike_path, timeout_seconds=timeout_seconds,

)‘ to ‘grad_h ‘ in host memory. Here grad_h means the value named ’grad h’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1085: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1079. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, membrane_path, spike_path, timeout_seconds=timeout_seconds,)‘ to ‘grad_h ‘ in host memory. Here grad_h means the value named ’grad h’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1086: `input_chunk = _execute_vector8(` This statement continues through source line 1092; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, grad_h, (int(reciprocal_tau_q15),) * 8, timeout_seconds=timeout_seconds,)‘ to ‘input_chunk ‘ in host memory. Here input_chunk means the value named ’input chunk’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1087: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1086. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, grad_h, (int(reciprocal_tau_q15),) * 8, timeout_seconds=timeout_seconds,)‘ to ‘input_chunk ‘ in host memory. Here input_chunk means the value named ’input chunk’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1088: `VectorOperation.MULTIPLY_Q15,` This is continuation line 3 of the 7-line logical statement that begins at source line 1086. Its exact fragment is ‘VectorOperation.MULTIPLY_Q15,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; MULTIPLY_Q15: the value named ’multiply q15’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, grad_h, (int(reciprocal_tau_q15),) * 8, timeout_seconds=timeout_seconds,)‘ to ‘input_chunk ‘ in host memory. Here input_chunk means the value named ’input chunk’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1089: `grad_h,` This is continuation line 4 of the 7-line logical statement that begins at source line 1086. Its exact fragment is ‘grad_h,‘. The important operands are grad_h: the value named ’grad h’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, grad_h, (int(reciprocal_tau_q15),) * 8, timeout_seconds=timeout_seconds,)‘ to ‘input_chunk ‘ in host memory. Here input_chunk means the value named ’input chunk’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1090: `(int(reciprocal_tau_q15),) * 8,` This is continuation line 5 of the 7-line logical statement that begins at source line 1086. Its exact fragment is ‘(int(reciprocal_tau_q15),) * 8,‘. The important operands are reciprocal_tau_q15: the value named ’reciprocal tau q15’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, grad_h, (int(reciprocal_tau_q15),) * 8, timeout_seconds=timeout_seconds,)‘ to ‘input_chunk ‘ in host memory. Here input_chunk means the value named ’input chunk’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1091: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1086. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, grad_h, (int(reciprocal_tau_q15),) * 8, timeout_seconds=timeout_seconds,)' to 'input_chunk ' in host memory. Here input_chunk means the value named 'input chunk' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1092: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1086. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MULTIPLY_Q15, grad_h, (int(reciprocal_tau_q15),) * 8, timeout_seconds=timeout_seconds,)' to 'input_chunk ' in host memory. Here input_chunk means the value named 'input chunk' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1093: `grad_h_trace[timestep, start:stop] = grad_h[:active]` This assigns 'grad_h[:active]' to 'grad_h_trace[timest start' in host memory. Here grad_h_trace means the value named 'grad h trace' in this block. The important operands are grad_h: the value named 'grad h' in this block; active: the value named 'active' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1094: `grad_input[timestep, start:stop] = input_chunk[:active]` This assigns 'input_chunk[:active]' to 'grad_input[timestep, start' in host memory. Here grad_input means the value named 'grad input' in this block. The important operands are input_chunk: the value named 'input chunk' in this block; active: the value named 'active' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1095: `grad_h_next = grad_h_trace[timestep].copy()` This assigns 'grad_h_trace[timestep].copy()' to 'grad_h_next ' in host memory. Here grad_h_next means the value named 'grad h next' in this block. The important operands are grad_h_trace: the value named 'grad h trace' in this block; timestep: one discrete SNN update instant; copy: the value named 'copy' in this block. Context: the function execute_lif_bptt, which handles execute lif bptt.

Line 1096: `return grad_input, grad_h_trace` This returns 'grad_input, grad_h_trace' to the caller of the function execute_lif_bptt, which handles execute lif bptt, ending this invocation. The important operands are grad_input: the value named 'grad input' in this block; grad_h_trace: the value named 'grad h trace' in this block.

Line 1099: `def _pair(name:  str, value) -> tuple[int, int]:` This starts function _pair. Its indented body performs pair as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 1100: `if isinstance(value, int):` This executes the following indented host-code branch only when 'isinstance(value, int)' is true. The important operands are isinstance: the value named 'isinstance' in this block; value: the value named 'value' in this block. Context: the function _pair, which handles pair.

Line 1101: `result = (int(value), int(value))` This assigns '(int(value), int(value))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are value: the value named 'value' in this block. Context: the function _pair, which handles pair.

Line 1102: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function _pair, which handles pair.

Line 1103: `result = tuple(map(int, value))` This assigns 'tuple(map(int, value))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; map: the value named 'map' in this block; value: the value named 'value' in this block. Context: the function _pair, which handles pair.

Line 1104: `if len(result) != 2:` This executes the following indented host-code branch only when ‘len(result) != 2‘ is true. The important operands are len: the value named ’len’ in this block; result: the value named ’result’ in this block. Context: the function _pair, which handles pair.

Line 1105: `raise ValueError(f"{name} must be an integer or a pair")` This rejects an invalid state by raising ‘ValueError(f"{name} must be an integer or a pair")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1106: `if result[0] <= 0 or result[1] <= 0:` This executes the following indented host-code branch only when ‘result[0] <= 0 or result[1] <= 0‘ is true. The important operands are result: the value named ’result’ in this block; or: the value named ’or’ in this block. Context: the function _pair, which handles pair.

Line 1107: `raise ValueError(f"{name} values must be positive")` This rejects an invalid state by raising ‘ValueError(f"{name} values must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1108: `return result` This returns ‘result‘ to the caller of the function _pair, which handles pair, ending this invocation. The important operands are result: the value named ’result’ in this block.

Line 1111: `def maxpool2d_backward_fixed(` This starts function maxpool2d_backward_fixed. Its indented body performs maxpool2d backward fixed as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 1112: `values,` Inside the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are values: the value named ’values’ in this block.

Line 1113: `output_gradients,` Inside the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_gradients: the signed loss derivative used to update a learned parameter.

Line 1114: `*,` Inside the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 1115: `kernel_size=3,` This assigns ‘3,‘ to ‘kernel_size‘ in host memory. Here kernel_size means the value named ’kernel size’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1116: `stride=2,` This assigns ‘2,‘ to ‘stride‘ in host memory. Here stride means the value named ’stride’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1117: `padding=1,` This assigns ‘1,‘ to ‘padding‘ in host memory. Here padding means the value named ’padding’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1118: `dilation=1,` This assigns ‘1,‘ to ‘dilation‘ in host memory. Here dilation means the value named ’dilation’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1119: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 1120: `"""Fixed-point MaxPool2d backward with PyTorch's first-maximum tie rule."""` This is executable documentation attached to the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1122: `source = np.asarray(values, dtype=np.int64)` This assigns 'np.asarray(values, dtype=np.int64)' to 'source ' in host memory. Here source means the value named 'source' in this block. The important operands are asarray: the value named 'asarray' in this block; values: the value named 'values' in this block; dtype: the value named 'dtype' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1123: `gradient = np.asarray(output_gradients, dtype=np.int64)` This assigns 'np.asarray(output_gradients, dtype=np.int64)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are asarray: the value named 'asarray' in this block; output_gradients: the signed loss derivative used to update a learned parameter; dtype: the value named 'dtype' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1124: `if source.ndim != 4 or gradient.ndim != 4:` This executes the following indented host-code branch only when 'source.ndim != 4 or gradient.ndim != 4' is true. The important operands are source: the value named 'source' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1125: `raise ValueError("max-pool values and gradients must be NCHW tensors")` This rejects an invalid state by raising 'ValueError("max-pool values and gradients must be NCHW tensors")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1126: `kernel_h, kernel_w = _pair("kernel_size", kernel_size)` This assigns '_pair("kernel_size", kernel_size)' to 'kernel_h, kernel_w ' in host memory. Here kernel_h means the value named 'kernel h' in this block. The important operands are _pair: the value named 'pair' in this block; kernel_size: the value named 'kernel size' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1127: `stride_h, stride_w = _pair("stride", stride)` This assigns '_pair("stride", stride)' to 'stride_h, stride_w ' in host memory. Here stride_h means the value named 'stride h' in this block. The important operands are _pair: the value named 'pair' in this block; stride: the value named 'stride' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1128: `dilation_h, dilation_w = _pair("dilation", dilation)` This assigns '_pair("dilation", dilation)' to 'dilation_h, dilation_w ' in host memory. Here dilation_h means the value named 'dilation h' in this block. The important operands are _pair: the value named 'pair' in this block; dilation: the value named 'dilation' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1129: `if isinstance(padding, int):` This executes the following indented host-code branch only when 'isinstance(padding, int)' is true. The important operands are isinstance: the value named 'isinstance' in this block; padding: the value named 'padding' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1130: `pad_h, pad_w = int(padding), int(padding)` This assigns 'int(padding), int(padding)' to 'pad_h, pad_w ' in host memory. Here pad_h means the value named 'pad h' in this block. The important operands are padding: the value named 'padding' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1131: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1132: `pad_h, pad_w = tuple(map(int, padding))` This assigns ‘tuple(map(int, padding))‘ to ‘pad_h, pad_w ‘ in host memory. Here pad_h means the value named ’pad h’ in this block. The important operands are tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block; padding: the value named ’padding’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1133: `if pad_h < 0 or pad_w < 0:` This executes the following indented host-code branch only when ‘pad_h < 0 or pad_w < 0‘ is true. The important operands are pad_h: the value named ’pad h’ in this block; or: the value named ’or’ in this block; pad_w: the value named ’pad w’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1134: `raise ValueError("padding must be nonnegative")` This rejects an invalid state by raising ‘ValueError("padding must be nonnegative")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1135: `batch, channels, height, width = source.shape` This assigns ‘source.shape‘ to ‘batch, channels, height, width ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are source: the value named ’source’ in this block; shape: the value named ’shape’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1136: `output_height = (` This statement continues through source line 1138; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1) // stride_h + 1‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are height: the value named ’height’ in this block; pad_h: the value named ’pad h’ in this block; dilation_h: the value named ’dilation h’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1137: `height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1` This is continuation line 2 of the 3-line logical statement that begins at source line 1136. Its exact fragment is ‘height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1‘. The important operands are height: the value named ’height’ in this block; pad_h: the value named ’pad h’ in this block; dilation_h: the value named ’dilation h’ in this block. Read together, the complete operation is: This assigns ‘(height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1) // stride_h + 1‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are height: the value named ’height’ in this block; pad_h: the value named ’pad h’ in this block; dilation_h: the value named ’dilation h’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1138: `) // stride_h + 1` This is continuation line 3 of the 3-line logical statement that begins at source line 1136. Its exact fragment is ‘) // stride_h + 1‘. The important operands are stride_h: the value named ’stride h’ in this block. Read together, the complete operation is: This assigns ‘(height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1) // stride_h + 1‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are height: the value named ’height’ in this block; pad_h: the value named ’pad h’ in this block; dilation_h: the value named ’dilation h’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1139: `output_width = (` This statement continues through source line 1141; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1) // stride_w + 1‘ to ‘output_width ‘ in host memory. Here output_width means the value named ’output width’ in this block. The important operands are width: the value named ’width’ in this block; pad_w: the value named ’pad w’ in this block; dilation_w: the value named ’dilation w’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1140: `width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1` This is continuation line 2 of the 3-line logical statement that begins at source line 1139. Its exact fragment is ‘width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1‘. The important operands are width: the value named ’width’ in this block; pad_w: the value named ’pad w’ in this block; dilation_w: the value named ’dilation w’ in this block. Read together, the complete operation is: This assigns ‘(width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1) // stride_w + 1‘ to ‘output_width ‘ in host memory. Here output_width means the value named ’output width’ in this block. The important operands are width: the value named ’width’ in this block; pad_w: the value named ’pad w’ in this block; dilation_w: the value named ’dilation w’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1141: `) // stride_w + 1` This is continuation line 3 of the 3-line logical statement that begins at source line 1139. Its exact fragment is ‘) // stride_w + 1‘. The important operands are stride_w: the value named ’stride w’ in this block. Read together, the complete operation is: This assigns ‘(width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1) // stride_w + 1‘ to ‘output_width ‘ in host memory. Here output_width means the value named ’output width’ in this block. The important operands are width: the value named ’width’ in this block; pad_w: the value named ’pad w’ in this block; dilation_w: the value named ’dilation w’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1142: `if gradient.shape != (batch, channels, output_height, output_width):` This executes the following indented host-code branch only when ‘gradient.shape != (batch, channels, output_height, output_width)‘ is true. The important operands are gradient: the signed loss derivative used to update a learned parameter; shape: the value named ’shape’ in this block; batch: the value named ’batch’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1143: `raise ValueError("max-pool output gradient shape does not match its geometry")` This rejects an invalid state by raising ‘ValueError("max-pool output gradient shape does not match its geometry")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1144: `input_gradient = np.zeros(source.shape, dtype=np.int64)` This assigns ‘np.zeros(source.shape, dtype=np.int
to ‘input_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are zeros: the value named ’zeros’ in this block; source: the value named ’source’ in this block; shape: the value named ’shape’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1145: `winners = np.zeros(gradient.shape, dtype=np.int64)` This assigns ‘np.zeros(gradient.shape, dtype=np.int64)‘ to ‘winners ‘ in host memory. Here winners means the value named ’winners’ in this block. The important operands are zeros: the value named ’zeros’ in this block; gradient: the signed loss derivative used to update a learned parameter; shape: the value named ’shape’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1146: `for batch_index in range(batch):` This iterates batch_index over ‘range(batch)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1147: `for channel in range(channels):` This iterates channel over ‘range(channels)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1148: `for output_y in range(output_height):` This iterates output_y over ‘range(output_height)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1149: `for output_x in range(output_width):` This iterates output_x over ‘range(output_width)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1150: `winner_value:  int | None = None` This assigns ‘None‘ to ‘winner_value‘ in host memory. Here winner_value means the value named ’winner value’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1151: `winner_index = -1` This assigns ‘-1‘ to ‘winner_index ‘ in host memory. Here winner_index means an integer selecting one tensor element, lane, bank, or state entry. The expression is a literal or structural operator and has no hidden software state. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1152: `for kernel_y in range(kernel_h):` This iterates kernel_y over ‘range(kernel_h)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1153: `input_y = output_y * stride_h - pad_h + kernel_y * dilation_h` This assigns ‘output_y * stride_h - pad_h + kernel_y * dilation_h‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride_h: the value named ’stride h’ in this block; pad_h: the value named ’pad h’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1154: `if not 0 <= input_y < height:` This executes the following indented host-code branch only when ‘not 0 <= input_y < height‘ is true. The important operands are not: the value named ’not’ in this block; input_y: the value named ’input y’ in this block; height: the value named ’height’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1155: `continue` Inside the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are continue: the value named ’continue’ in this block.

Line 1156: `for kernel_x in range(kernel_w):` This iterates kernel_x over ‘range(kernel_w)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1157: `input_x = output_x * stride_w - pad_w + kernel_x * dilation_w` This assigns ‘output_x * stride_w - pad_w + kernel_x * dilation_w‘ to ‘input_x ‘ in host memory. Here input_x means the value named ’input x’ in this block. The important operands are output_x: the value named ’output x’ in this block; stride_w: the value named ’stride w’ in this block; pad_w: the value named ’pad w’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1158: `if not 0 <= input_x < width:` This executes the following indented host-code branch only when ‘not 0 <= input_x < width‘ is true. The important operands are not: the value named ’not’ in this block; input_x: the value named ’input x’ in this block; width: the value named ’width’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1159: `continue` Inside the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are continue: the value named ’continue’ in this block.

Line 1160: `candidate = int(source[batch_index, channel, input_y, input_x])` This assigns ‘int(source[batch_index, channel, input_y, input_x])‘ to ‘candidate ‘ in host memory. Here candidate means the value named ’candidate’ in this block. The important operands are source: the value named ’source’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named ’channel’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1161: `if winner_value is None or candidate > winner_value:` This executes the following indented host-code branch only when ‘winner_value is None or candidate > winner_value‘ is true. The important operands are winner_value: the value named ’winner value’ in this block; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1162: `winner_value = candidate` This assigns ‘candidate‘ to ‘winner_value ‘ in host memory. Here winner_value means the value named ’winner value’ in this block. The important operands are candidate: the value named ’candidate’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1163: `winner_index = kernel_y * kernel_w + kernel_x` This assigns ‘kernel_y * kernel_w + kernel_x‘ to ‘winner_index ‘ in host memory. Here winner_index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are kernel_y: the value named ’kernel y’ in this block; kernel_w: the value named ’kernel w’ in this block; kernel_x: the value named ’kernel x’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1164: `if winner_index < 0:` This executes the following indented host-code branch only when ‘winner_index < 0‘ is true. The important operands are winner_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1165: `raise ValueError("a max-pool window contains no source elements")` This rejects an invalid state by raising ‘ValueError("a max-pool window contains no source elements")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1166: `winners[batch_index, channel, output_y, output_x] = winner_index` This assigns ‘winner_index‘ to ‘winners[batch_index, channel, output_y, output_x] ‘ in host memory. Here winners means the value named ’winners’ in this block. The important operands are winner_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1167: `input_y = (` This statement continues through source line 1170; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(output_y * stride_h - pad_h + (winner_index // kernel_w) * dilation_h)‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride_h: the value named ’stride h’ in this block; pad_h: the value named ’pad h’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1168: `output_y * stride_h - pad_h` This is continuation line 2 of the 4-line logical statement that begins at source line 1167. Its exact fragment is ‘output_y * stride_h - pad_h‘. The important operands are output_y: the value named ’output y’ in this block; stride_h: the value named ’stride h’ in this block; pad_h: the value named ’pad h’ in this block. Read together, the complete operation is: This assigns ‘(output_y * stride_h - pad_h + (winner_index // kernel_w) * dilation_h)‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride_h: the value named ’stride h’ in this block; pad_h: the value named ’pad h’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1169: `+ (winner_index // kernel_w) * dilation_h` This is continuation line 3 of the 4-line logical statement that begins at source line 1167. Its exact fragment is ‘+ (winner_index // kernel_w) * dilation_h‘. The important operands are winner_index: an integer selecting one tensor element, lane, bank, or state entry; kernel_w: the value named ’kernel w’ in this block; dilation_h: the value named ’dilation h’ in this block. Read together, the complete operation is: This assigns ‘(output_y * stride_h - pad_h + (winner_index // kernel_w) * dilation_h)‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride_h: the value named ’stride h’ in this block; pad_h: the value named ’pad h’ in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1170: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1167. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(output_y * stride_h - pad_h + (winner_index // kernel_w) * dilation_h)' to 'input_y ' in host memory. Here input_y means the value named 'input y' in this block. The important operands are output_y: the value named 'output y' in this block; stride_h: the value named 'stride h' in this block; pad_h: the value named 'pad h' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1171: `input_x = (` This statement continues through source line 1174; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(output_x * stride_w - pad_w + (winner_index % kernel_w) * dilation_w)' to 'input_x ' in host memory. Here input_x means the value named 'input x' in this block. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1172: `output_x * stride_w - pad_w` This is continuation line 2 of the 4-line logical statement that begins at source line 1171. Its exact fragment is 'output_x * stride_w - pad_w'. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Read together, the complete operation is: This assigns '(output_x * stride_w - pad_w + (winner_index % kernel_w) * dilation_w)' to 'input_x ' in host memory. Here input_x means the value named 'input x' in this block. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1173: `+ (winner_index % kernel_w) * dilation_w` This is continuation line 3 of the 4-line logical statement that begins at source line 1171. Its exact fragment is '+ (winner_index % kernel_w) * dilation_w'. The important operands are winner_index: an integer selecting one tensor element, lane, bank, or state entry; kernel_w: the value named 'kernel w' in this block; dilation_w: the value named 'dilation w' in this block. Read together, the complete operation is: This assigns '(output_x * stride_w - pad_w + (winner_index % kernel_w) * dilation_w)' to 'input_x ' in host memory. Here input_x means the value named 'input x' in this block. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1174: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1171. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(output_x * stride_w - pad_w + (winner_index % kernel_w) * dilation_w)' to 'input_x ' in host memory. Here input_x means the value named 'input x' in this block. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1175: `prior = input_gradient[batch_index, channel, input_y, input_x]` This assigns 'input_gradient[batch_index, channel, input_y, input_x]' to 'prior ' in host memory. Here prior means the value named 'prior' in this block. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1176: `update = gradient[batch_index, channel, output_y, output_x]` This assigns 'gradient[batch_index, channel, output_y, output_x]' to 'update ' in host memory. Here update means the value named 'update' in this block. The important operands are gradient: the signed loss derivative used to update a learned parameter; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1177: `input_gradient[batch_index, channel, input_y, input_x] = saturate_signed(` This statement continues through source line 1179; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'saturate_signed(np.asarray(prior + update, dtype=np.int64), 32)' to 'input_gradient[batch_index,

channel, input_y, input_x] ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are saturate_signed: the value named 'saturate signed' in this block; asarray: the value named 'asarray' in this block; prior: the value named 'prior' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1178: `np.asarray(prior + update, dtype=np.int64), 32` This is continuation line 2 of the 3-line logical statement that begins at source line 1177. Its exact fragment is ‘np.asarray(prior + update, dtype=np.int64), 32‘. The important operands are asarray: the value named 'asarray' in this block; prior: the value named 'prior' in this block; update: the value named 'update' in this block. Read together, the complete operation is: This assigns ‘saturate_signed(np.asarray(prior + update, dtype=np.int64), 32)‘ to ‘input_gradient[batch_index, channel, input_y, input_x] ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are saturate_signed: the value named 'saturate signed' in this block; asarray: the value named 'asarray' in this block; prior: the value named 'prior' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1179: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 1177. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘saturate_signed(np.asarray(prior + update, dtype=np.int64), 32)‘ to ‘input_gradient[batch_index, channel, input_y, input_x] ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are saturate_signed: the value named 'saturate signed' in this block; asarray: the value named 'asarray' in this block; prior: the value named 'prior' in this block. Context: the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed.

Line 1180: `return input_gradient, winners` This returns ‘input_gradient, winners‘ to the caller of the function maxpool2d_backward_fixed, which handles maxpool2d backward fixed, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; winners: the value named 'winners' in this block.

Line 1183: `def execute_maxpool2d_backward(` This starts function execute_maxpool2d_backward. Its indented body performs execute maxpool2d backward as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 1184: `driver: TrainingCommandExecutor,` Inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are driver: the value named 'driver' in this block; TrainingCommandExecutor: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 1185: `values,` Inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are values: the value named 'values' in this block.

Line 1186: `output_gradients,` Inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are output_gradients: the signed loss derivative used to update a learned parameter.

Line 1187: `*,` Inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 1188: `kernel_size=3,` This assigns ‘3,‘ to ‘kernel_size‘ in host memory. Here kernel_size means the value named ’kernel size’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1189: `stride=2,` This assigns ‘2,‘ to ‘stride‘ in host memory. Here stride means the value named ’stride’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1190: `padding=1,` This assigns ‘1,‘ to ‘padding‘ in host memory. Here padding means the value named ’padding’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1191: `dilation=1,` This assigns ‘1,‘ to ‘dilation‘ in host memory. Here dilation means the value named ’dilation’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1192: `timeout_seconds: float = 5.0,` This assigns ‘5.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1193: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are tuple: the value named ’tuple’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 1194: `"""Run max-pool winner selection and gradient scatter through vector RTL."""` This is executable documentation attached to the function execute_maxpool2d_backward, which handles execute maxpool2d backward. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1196: `source = np.asarray(values, dtype=np.int64)` This assigns ‘np.asarray(values, dtype=np.int64)‘ to ‘source ‘ in host memory. Here source means the value named ’source’ in this block. The important operands are asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block; dtype: the value named ’dtype’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1197: `gradient = np.asarray(output_gradients, dtype=np.int64)` This assigns ‘np.asarray(output_gradients, dtype=np.int64)‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are asarray: the value named ’asarray’ in this block; output_gradients: the signed loss derivative used to update a learned parameter; dtype: the value named ’dtype’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1198: `if source.ndim != 4 or gradient.ndim != 4:` This executes the following indented host-code branch only when ‘source.ndim != 4 or gradient.ndim != 4‘ is true. The important operands are source: the value named ’source’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1199: `raise ValueError("max-pool values and gradients must be NCHW tensors")` This rejects an invalid state by raising ‘ValueError("max-pool values and gradients must be NCHW tensors")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1200: `kernel_h, kernel_w = _pair("kernel_size", kernel_size)` This assigns ‘_pair("kernel_size", kernel_size)‘ to ‘kernel_h, kernel_w ‘ in host memory. Here kernel_h means the value named ’kernel h’ in this block. The important operands are _pair: the value named ’pair’ in this block; kernel_size: the value named ’kernel size’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1201: `stride_h, stride_w = _pair("stride", stride)` This assigns ‘_pair("stride", stride)‘ to ‘stride_h, stride_w ‘ in host memory. Here stride_h means the value named ’stride h’ in this block. The important operands are _pair: the value named ’pair’ in this block; stride: the value named ’stride’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1202: `dilation_h, dilation_w = _pair("dilation", dilation)` This assigns ‘_pair("dilation", dilation)‘ to ‘dilation_h, dilation_w ‘ in host memory. Here dilation_h means the value named ’dilation h’ in this block. The important operands are _pair: the value named ’pair’ in this block; dilation: the value named ’dilation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1203: `if isinstance(padding, int):` This executes the following indented host-code branch only when ‘isinstance(padding, int)‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; padding: the value named ’padding’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1204: `pad_h, pad_w = int(padding), int(padding)` This assigns ‘int(padding), int(padding)‘ to ‘pad_h, pad_w ‘ in host memory. Here pad_h means the value named ’pad h’ in this block. The important operands are padding: the value named ’padding’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1205: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1206: `pad_h, pad_w = tuple(map(int, padding))` This assigns ‘tuple(map(int, padding))‘ to ‘pad_h, pad_w ‘ in host memory. Here pad_h means the value named ’pad h’ in this block. The important operands are tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block; padding: the value named ’padding’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1207: `batch, channels, height, width = source.shape` This assigns ‘source.shape‘ to ‘batch, channels, height, width ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are source: the value named ’source’ in this block; shape: the value named ’shape’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1208: `output_height = (` This statement continues through source line 1210; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1) // stride_h + 1‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are height: the value named ’height’ in this block; pad_h: the value named ’pad h’ in this block; dilation_h: the value named ’dilation h’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1209: `height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1` This is continuation line 2 of the 3-line logical statement that begins at source line 1208. Its exact fragment is ‘height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1‘. The important operands are height: the value named ’height’ in this block; pad_h: the value named ’pad h’ in this block; dilation_h: the value named ’dilation h’ in this block. Read together, the complete operation is: This assigns ‘(height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1) // stride_h + 1‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are height: the value named ’height’ in this block; pad_h: the value named ’pad h’ in this block; dilation_h: the value named ’dilation h’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1210: `) // stride_h + 1` This is continuation line 3 of the 3-line logical statement that begins at source line 1208. Its exact fragment is ‘) // stride_h + 1‘. The important operands are stride_h: the value named ’stride h’ in this block. Read together, the complete operation is: This assigns ‘(height + 2 * pad_h - dilation_h * (kernel_h - 1) - 1) // stride_h + 1‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are height: the value named ’height’ in this block; pad_h: the value named ’pad h’ in

this block; dilation_h: the value named 'dilation h' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1211: `output_width = (` This statement continues through source line 1213; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1) // stride_w + 1' to 'output_width ' in host memory. Here output_width means the value named 'output width' in this block. The important operands are width: the value named 'width' in this block; pad_w: the value named 'pad w' in this block; dilation_w: the value named 'dilation w' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1212: `width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1` This is continuation line 2 of the 3-line logical statement that begins at source line 1211. Its exact fragment is 'width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1'. The important operands are width: the value named 'width' in this block; pad_w: the value named 'pad w' in this block; dilation_w: the value named 'dilation w' in this block. Read together, the complete operation is: This assigns '(width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1) // stride_w + 1' to 'output_width ' in host memory. Here output_width means the value named 'output width' in this block. The important operands are width: the value named 'width' in this block; pad_w: the value named 'pad w' in this block; dilation_w: the value named 'dilation w' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1213: `) // stride_w + 1` This is continuation line 3 of the 3-line logical statement that begins at source line 1211. Its exact fragment is ') // stride_w + 1'. The important operands are stride_w: the value named 'stride w' in this block. Read together, the complete operation is: This assigns '(width + 2 * pad_w - dilation_w * (kernel_w - 1) - 1) // stride_w + 1' to 'output_width ' in host memory. Here output_width means the value named 'output width' in this block. The important operands are width: the value named 'width' in this block; pad_w: the value named 'pad w' in this block; dilation_w: the value named 'dilation w' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1214: `if gradient.shape != (batch, channels, output_height, output_width):` This executes the following indented host-code branch only when 'gradient.shape != (batch, channels, output_height, output_width)' is true. The important operands are gradient: the signed loss derivative used to update a learned parameter; shape: the value named 'shape' in this block; batch: the value named 'batch' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1215: `raise ValueError("max-pool output gradient shape does not match its geometry")` This rejects an invalid state by raising 'ValueError("max-pool output gradient shape does not match its geometry")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1216: `windows = tuple(` This statement continues through source line 1222; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'tuple((batch_index, channel, output_y, output_x) for batch_index in range(batch) for channel in range(channels) for output_y in range(output_height) for output_x in range(output_width))' to 'windows ' in host memory. Here windows means the value named 'windows' in this block. The important operands are tuple: the value named 'tuple' in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1217: `(batch_index, channel, output_y, output_x)` This is continuation line 2 of the 7-line logical statement that begins at source line 1216. Its exact fragment is '(batch_index, channel, output_y, output_x)'. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block; output_y: the value named 'output y' in this block. Read together, the complete operation is: This assigns 'tuple((batch_index, channel, output_y, output_x) for batch_index in range(batch) for channel in range(channels) for output_y in range(output_height) for output_x in range(output_width))' to 'windows ' in host memory. Here windows means the value named 'windows' in this block. The important operands are tuple: the value named 'tuple' in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1218: `for batch_index in range(batch)` This is continuation line 3 of the 7-line logical statement that begins at source line 1216. Its exact fragment is 'for batch_index in range(batch)'. The important operands are for: the value named 'for' in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'tuple((batch_index, channel, output_y, output_x) for batch_index in range(batch) for channel in range(channels) for output_y in range(output_height) for output_x in range(output_width))' to 'windows ' in host memory. Here windows means the value named 'windows' in this block. The important operands are tuple: the value named 'tuple' in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1219: `for channel in range(channels)` This is continuation line 4 of the 7-line logical statement that begins at source line 1216. Its exact fragment is 'for channel in range(channels)'. The important operands are for: the value named 'for' in this block; channel: the value named 'channel' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'tuple((batch_index, channel, output_y, output_x) for batch_index in range(batch) for channel in range(channels) for output_y in range(output_height) for output_x in range(output_width))' to 'windows ' in host memory. Here windows means the value named 'windows' in this block. The important operands are tuple: the value named 'tuple' in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1220: `for output_y in range(output_height)` This is continuation line 5 of the 7-line logical statement that begins at source line 1216. Its exact fragment is 'for output_y in range(output_height)'. The important operands are for: the value named 'for' in this block; output_y: the value named 'output y' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'tuple((batch_index, channel, output_y, output_x) for batch_index in range(batch) for channel in range(channels) for output_y in range(output_height) for output_x in range(output_width))' to 'windows ' in host memory. Here windows means the value named 'windows' in this block. The important operands are tuple: the value named 'tuple' in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1221: `for output_x in range(output_width)` This is continuation line 6 of the 7-line logical statement that begins at source line 1216. Its exact fragment is 'for output_x in range(output_width)'. The important operands are for: the value named 'for' in this block; output_x: the value named 'output x' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'tuple((batch_index, channel, output_y, output_x) for batch_index in range(batch) for channel in range(channels) for output_y in range(output_height) for output_x in range(output_width))' to 'windows ' in host memory. Here windows means the value named 'windows' in this block. The important operands are tuple: the value named 'tuple' in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1222: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1216. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'tuple((batch_index, channel, output_y, output_x) for batch_index in range(batch) for channel in range(channels) for output_y in range(output_height) for output_x in range(output_width))' to 'windows ' in host memory. Here windows means the value named 'windows' in this block. The important operands are tuple: the value named 'tuple' in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1223: `input_gradient = np.zeros(source.shape, dtype=np.int64)` This assigns 'np.zeros(source.shape, dtype=np.int
to 'input_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are zeros: the value named 'zeros' in this block; source: the value named 'source' in this block; shape: the value named 'shape' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1224: `winners = np.zeros(gradient.shape, dtype=np.int64)` This assigns ‘np.zeros(gradient.shape, dtype=np.int64)‘ to ‘winners ‘ in host memory. Here winners means the value named ’winners’ in this block. The important operands are zeros: the value named ’zeros’ in this block; gradient: the signed loss derivative used to update a learned parameter; shape: the value named ’shape’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1226: `for window_start in range(0, len(windows), 8):` This iterates window_start over ‘range(0, len(windows), 8)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1227: `active_windows = windows[window_start:window_start + 8]` This assigns ‘windows[window_start:window_star + 8]‘ to ‘active_windows ‘ in host memory. Here active_windows means the value named ’active windows’ in this block. The important operands are windows: the value named ’windows’ in this block; window_start: the value named ’window start’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1228: `active = len(active_windows)` This assigns ‘len(active_windows)‘ to ‘active ‘ in host memory. Here active means the value named ’active’ in this block. The important operands are len: the value named ’len’ in this block; active_windows: the value named ’active windows’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1229: `first_indices: list[int] = []` This assigns ‘[]‘ to ‘first_indices‘ in host memory. Here first_indices means the value named ’first indices’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1230: `current_values: list[int] = []` This assigns ‘[]‘ to ‘current_values‘ in host memory. Here current_values means the value named ’current values’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1231: `for batch_index, channel, output_y, output_x in active_windows:` This iterates batch_index, channel, output_y, output_x over ‘active_windows‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1232: `found = False` This assigns ‘False‘ to ‘found ‘ in host memory. Here found means the value named ’found’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1233: `for kernel_y in range(kernel_h):` This iterates kernel_y over ‘range(kernel_h)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_maxpool2d_backward which handles execute maxpool2d backward.

Line 1234: `input_y = output_y * stride_h - pad_h + kernel_y * dilation_h` This assigns ‘output_y * stride_h - pad_h + kernel_y * dilation_h‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride_h: the value named ’stride h’ in this block; pad_h: the value named ’pad h’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1235: `for kernel_x in range(kernel_w):` This iterates kernel_x over ‘range(kernel_w)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_maxpool2d_backward which handles execute maxpool2d backward.

Line 1236: `input_x = output_x * stride_w - pad_w + kernel_x * dilation_w` This assigns ‘output_x * stride_w - pad_w + kernel_x * dilation_w‘ to ‘input_x ‘ in host memory. Here input_x means the value named ’input x’ in this block. The important operands are output_x: the value named ’output x’ in this block; stride_w: the value named ’stride w’ in this block; pad_w: the value named ’pad w’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1237: `if 0 <= input_y < height and 0 <= input_x < width:` This executes the following indented host-code branch only when ‘0 <= input_y < height and 0 <= input_x < width‘ is true. The important operands are input_y: the value named ’input y’ in this block; height: the value named ’height’ in this block; and: the value named ’and’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1238: `first_indices.append(kernel_y * kernel_w + kernel_x)` This calls ‘first_indices.append‘ with ‘kernel_y * kernel_w + kernel_x‘ inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1239: `current_values.append(int(source[batch_index, channel, input_y, input_x]))` This calls ‘current_values.append‘ with ‘int(source[batch_index, channel, input_y, input_x])‘ inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1240: `found = True` This assigns ‘True‘ to ‘found ‘ in host memory. Here found means the value named ’found’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1241: `break` Inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are break: the value named ’break’ in this block.

Line 1242: `if found:` This executes the following indented host-code branch only when ‘found‘ is true. The important operands are found: the value named ’found’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1243: `break` Inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are break: the value named ’break’ in this block.

Line 1244: `if not found:` This executes the following indented host-code branch only when ‘not found‘ is true. The important operands are not: the value named ’not’ in this block; found: the value named ’found’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1245: `raise ValueError("a max-pool window contains no source elements")` This rejects an invalid state by raising ‘ValueError("a max-pool window contains no source elements")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1246: `current = np.asarray(current_values + [0] * (8 - active), dtype=np.int64)` This assigns ‘np.asarray(cur + [0] * (8 - active), dtype=np.int64)‘ to ‘current ‘ in host memory. Here current means the value named ’current’ in this block. The important operands are asarray: the value named ’asarray’ in this block; current_values: the value named ’current values’ in this block; active: the value named ’active’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1247: `winner = np.asarray(first_indices + [0] * (8 - active), dtype=np.int64)` This assigns ‘np.asarray(first_ + [0] * (8 - active), dtype=np.int64)‘ to ‘winner ‘ in host memory. Here winner means the value named ’winner’ in this block. The important operands are asarray: the value named ’asarray’ in this block; first_indices: the value named ’first indices’ in this block; active: the value named ’active’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1248: `for candidate_index in range(kernel_h * kernel_w):` This iterates candidate_index over ‘range(kernel_h * kernel_w)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1249: `candidates = current.copy()` This assigns ‘current.copy()‘ to ‘candidates ‘ in host memory. Here candidates means the value named ’candidates’ in this block. The important operands are current: the value named ’current’ in this block; copy: the value named ’copy’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1250: `for lane, (batch_index, channel, output_y, output_x) in enumerate(active_windows):` This iterates lane, (batch_index, channel, output_y, output_x) over ‘enumerate(active_windows)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1251: `kernel_y, kernel_x = divmod(candidate_index, kernel_w)` This assigns ‘divmod(candidate_index, kernel_w)‘ to ‘kernel_y, kernel_x ‘ in host memory. Here kernel_y means the value named ’kernel y’ in this block. The important operands are divmod: the value named ’divmod’ in this block; candidate_index: an integer selecting one tensor element, lane, bank, or state entry; kernel_w: the value named ’kernel w’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1252: `input_y = output_y * stride_h - pad_h + kernel_y * dilation_h` This assigns ‘output_y * stride_h - pad_h + kernel_y * dilation_h‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride_h: the value named ’stride h’ in this block; pad_h: the value named ’pad h’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1253: `input_x = output_x * stride_w - pad_w + kernel_x * dilation_w` This assigns ‘output_x * stride_w - pad_w + kernel_x * dilation_w‘ to ‘input_x ‘ in host memory. Here input_x means the value named ’input x’ in this block. The important operands are output_x: the value named ’output x’ in this block; stride_w: the value named ’stride w’ in this block; pad_w: the value named ’pad w’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1254: `if (candidate_index > first_indices[lane]` Inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are candidate_index: an integer selecting one tensor element, lane, bank, or state entry; first_indices: the value named ’first indices’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 1255: `and 0 <= input_y < height and 0 <= input_x < width):` Inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are and: the value named ’and’ in this block; input_y: the value named ’input y’ in this block; height: the value named ’height’ in this block.

Line 1256: `candidates[lane] = source[batch_index, channel, input_y, input_x]` This assigns ‘source[batch_index, channel, input_y, input_x]‘ to ‘candidates[lane] ‘ in host memory. Here candidates means the value named ’candidates’ in this block. The important operands are source: the value named ’source’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named ’channel’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1257: `keep_mask = _execute_vector8(` This statement continues through source line 1263; the explanation uses the complete expression rather than only the first visual fragment. This assigns '_execute_vector8(driver, VectorOperation.COMPARE_GE, current, candidates, timeout_seconds=timeout_seconds,)' to 'keep_mask ' in host memory. Here keep_mask means the value named 'keep mask' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1258: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1257. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.COMPARE_GE, current, candidates, timeout_seconds=timeout_seconds,)' to 'keep_mask ' in host memory. Here keep_mask means the value named 'keep mask' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1259: `VectorOperation.COMPARE_GE,` This is continuation line 3 of the 7-line logical statement that begins at source line 1257. Its exact fragment is 'VectorOperation.COMPARE_GE,'. The important operands are VectorOperation: the value named 'vector operation' in this block; COMPARE_GE: the value named 'compare ge' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.COMPARE_GE, current, candidates, timeout_seconds=timeout_seconds,)' to 'keep_mask ' in host memory. Here keep_mask means the value named 'keep mask' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1260: `current,` This is continuation line 4 of the 7-line logical statement that begins at source line 1257. Its exact fragment is 'current,'. The important operands are current: the value named 'current' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.COMPARE_GE, current, candidates, timeout_seconds=timeout_seconds,)' to 'keep_mask ' in host memory. Here keep_mask means the value named 'keep mask' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1261: `candidates,` This is continuation line 5 of the 7-line logical statement that begins at source line 1257. Its exact fragment is 'candidates,'. The important operands are candidates: the value named 'candidates' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.COMPARE_GE, current, candidates, timeout_seconds=timeout_seconds,)' to 'keep_mask ' in host memory. Here keep_mask means the value named 'keep mask' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1262: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1257. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.COMPARE_GE, current, candidates, timeout_seconds=timeout_seconds,)' to 'keep_mask ' in host memory. Here keep_mask means the value named 'keep mask' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1263: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1257. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.COMPARE_GE, current, candidates, timeout_seconds=timeout_seconds,)' to 'keep_mask ' in host memory. Here keep_mask means the value named 'keep mask' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block;

driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1264: `current = _execute_vector8(` This statement continues through source line 1270; the explanation uses the complete expression rather than only the first visual fragment. This assigns '_execute_vector8(driver, VectorOperation.MAXIMUM, current, candidates, timeout_seconds=timeout_seconds,)' to 'current ' in host memory. Here current means the value named 'current' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1265: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1264. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MAXIMUM, current, candidates, timeout_seconds=timeout_seconds,)' to 'current ' in host memory. Here current means the value named 'current' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1266: `VectorOperation.MAXIMUM,` This is continuation line 3 of the 7-line logical statement that begins at source line 1264. Its exact fragment is 'VectorOperation.MAXIMUM,'. The important operands are VectorOperation: the value named 'vector operation' in this block; MAXIMUM: the value named 'maximum' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MAXIMUM, current, candidates, timeout_seconds=timeout_seconds,)' to 'current ' in host memory. Here current means the value named 'current' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1267: `current,` This is continuation line 4 of the 7-line logical statement that begins at source line 1264. Its exact fragment is 'current,'. The important operands are current: the value named 'current' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MAXIMUM, current, candidates, timeout_seconds=timeout_seconds,)' to 'current ' in host memory. Here current means the value named 'current' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1268: `candidates,` This is continuation line 5 of the 7-line logical statement that begins at source line 1264. Its exact fragment is 'candidates,'. The important operands are candidates: the value named 'candidates' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MAXIMUM, current, candidates, timeout_seconds=timeout_seconds,)' to 'current ' in host memory. Here current means the value named 'current' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1269: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1264. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MAXIMUM, current, candidates, timeout_seconds=timeout_seconds,)' to 'current ' in host memory. Here current means the value named 'current' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1270: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1264. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.MAXIMUM, current, candidates, timeout_seconds=timeout_seconds,)' to 'current ' in host memory. Here current means the value named 'current' in

this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1271: `kept_winner = _execute_vector8(` This statement continues through source line 1277; the explanation uses the complete expression rather than only the first visual fragment. This assigns '_execute_vector8(driver, VectorOperation.SELECT_NONZERO, keep_mask, winner, timeout_seconds=timeout_seconds,)' to 'kept_winner ' in host memory. Here kept_winner means the value named 'kept winner' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1272: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1271. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_NONZERO, keep_mask, winner, timeout_seconds=timeout_seconds,)' to 'kept_winner ' in host memory. Here kept_winner means the value named 'kept winner' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1273: `VectorOperation.SELECT_NONZERO,` This is continuation line 3 of the 7-line logical statement that begins at source line 1271. Its exact fragment is 'VectorOperation.SELECT_NONZERO,'. The important operands are VectorOperation: the value named 'vector operation' in this block; SELECT_NONZERO: the value named 'select nonzero' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_NONZERO, keep_mask, winner, timeout_seconds=timeout_seconds,)' to 'kept_winner ' in host memory. Here kept_winner means the value named 'kept winner' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1274: `keep_mask,` This is continuation line 4 of the 7-line logical statement that begins at source line 1271. Its exact fragment is 'keep_mask,'. The important operands are keep_mask: the value named 'keep mask' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_NONZERO, keep_mask, winner, timeout_seconds=timeout_seconds,)' to 'kept_winner ' in host memory. Here kept_winner means the value named 'kept winner' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1275: `winner,` This is continuation line 5 of the 7-line logical statement that begins at source line 1271. Its exact fragment is 'winner,'. The important operands are winner: the value named 'winner' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_NONZERO, keep_mask, winner, timeout_seconds=timeout_seconds,)' to 'kept_winner ' in host memory. Here kept_winner means the value named 'kept winner' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1276: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1271. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.SELECT_NONZERO, keep_mask, winner, timeout_seconds=timeout_seconds,)' to 'kept_winner ' in host memory. Here kept_winner means the value named 'kept winner' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1277: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1271. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.SELECT_NONZERO, keep_mask, winner, timeout_seconds=timeout_seconds,)‘ to ‘kept_winner ‘ in host memory. Here kept_winner means the value named ’kept winner’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1278: `replacement = _execute_vector8(` This statement continues through source line 1284; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_execute_vector8(driver, VectorOperation.SELECT_ZERO, keep_mask, (candidate_index,) * 8, timeout_seconds=timeout_seconds,)‘ to ‘replacement ‘ in host memory. Here replacement means the value named ’replacement’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1279: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1278. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.SELECT_ZERO, keep_mask, (candidate_index,) * 8, timeout_seconds=timeout_seconds,)‘ to ‘replacement ‘ in host memory. Here replacement means the value named ’replacement’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1280: `VectorOperation.SELECT_ZERO,` This is continuation line 3 of the 7-line logical statement that begins at source line 1278. Its exact fragment is ‘VectorOperation.SELECT_ZERO,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; SELECT_ZERO: the value named ’select zero’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.SELECT_ZERO, keep_mask, (candidate_index,) * 8, timeout_seconds=timeout_seconds,)‘ to ‘replacement ‘ in host memory. Here replacement means the value named ’replacement’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1281: `keep_mask,` This is continuation line 4 of the 7-line logical statement that begins at source line 1278. Its exact fragment is ‘keep_mask,‘. The important operands are keep_mask: the value named ’keep mask’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.SELECT_ZERO, keep_mask, (candidate_index,) * 8, timeout_seconds=timeout_seconds,)‘ to ‘replacement ‘ in host memory. Here replacement means the value named ’replacement’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1282: `(candidate_index,) * 8,` This is continuation line 5 of the 7-line logical statement that begins at source line 1278. Its exact fragment is ‘(candidate_index,) * 8,‘. The important operands are candidate_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.SELECT_ZERO, keep_mask, (candidate_index,) * 8, timeout_seconds=timeout_seconds,)‘ to ‘replacement ‘ in host memory. Here replacement means the value named ’replacement’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1283: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1278. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.SELECT_ZERO, keep_mask, (candidate_index,) * 8, timeout_seconds=timeout_sec

)‘ to ‘replacement ‘ in host memory. Here replacement means the value named ’replacement’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1284: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1278. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.SELECT_ZERO, keep_mask, (candidate_index,) * 8, timeout_seconds=timeout_seconds,)‘ to ‘replacement ‘ in host memory. Here replacement means the value named ’replacement’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1285: `winner = _execute_vector8(` This statement continues through source line 1291; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, kept_winner, replacement, timeout_seconds=timeout_seconds,)‘ to ‘winner ‘ in host memory. Here winner means the value named ’winner’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1286: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1285. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, kept_winner, replacement, timeout_seconds=timeout_seconds,)‘ to ‘winner ‘ in host memory. Here winner means the value named ’winner’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1287: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 1285. Its exact fragment is ‘VectorOperation.ADD_SAT,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; ADD_SAT: the value named ’add sat’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, kept_winner, replacement, timeout_seconds=timeout_seconds,)‘ to ‘winner ‘ in host memory. Here winner means the value named ’winner’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1288: `kept_winner,` This is continuation line 4 of the 7-line logical statement that begins at source line 1285. Its exact fragment is ‘kept_winner,‘. The important operands are kept_winner: the value named ’kept winner’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, kept_winner, replacement, timeout_seconds=timeout_seconds,)‘ to ‘winner ‘ in host memory. Here winner means the value named ’winner’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1289: `replacement,` This is continuation line 5 of the 7-line logical statement that begins at source line 1285. Its exact fragment is ‘replacement,‘. The important operands are replacement: the value named ’replacement’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, kept_winner, replacement, timeout_seconds=timeout_seconds,)‘ to ‘winner ‘ in host memory. Here winner means the value named ’winner’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1290: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1285. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are

timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, kept_winner, replacement, timeout_seconds=timeout_seconds,)' to 'winner ' in host memory. Here winner means the value named 'winner' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1291: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1285. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, kept_winner, replacement, timeout_seconds=timeout_seconds,)' to 'winner ' in host memory. Here winner means the value named 'winner' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1293: `updates: list[tuple[tuple[int, int, int, int], int]] = []` This assigns '[]' to 'updates' in host memory. Here updates means the value named 'updates' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1294: `for lane, window in enumerate(active_windows):` This iterates lane, window over 'enumerate(active_windows)' Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1295: `batch_index, channel, output_y, output_x = window` This assigns 'window' to 'batch_index, channel, output_y, output_x ' in host memory. Here batch_index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are window: the value named 'window' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1296: `winner_index = int(winner[lane])` This assigns 'int(winner[lane])' to 'winner_index ' in host memory. Here winner_index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are winner: the value named 'winner' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1297: `winners[window] = winner_index` This assigns 'winner_index' to 'winners[window] ' in host memory. Here winners means the value named 'winners' in this block. The important operands are winner_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1298: `input_y = (` This statement continues through source line 1301; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(output_y * stride_h - pad_h + (winner_index // kernel_w) * dilation_h)' to 'input_y ' in host memory. Here input_y means the value named 'input y' in this block. The important operands are output_y: the value named 'output y' in this block; stride_h: the value named 'stride h' in this block; pad_h: the value named 'pad h' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1299: `output_y * stride_h - pad_h` This is continuation line 2 of the 4-line logical statement that begins at source line 1298. Its exact fragment is 'output_y * stride_h - pad_h'. The important operands are output_y: the value named 'output y' in this block; stride_h: the value named 'stride h' in this block; pad_h: the value named 'pad h' in this block. Read together, the complete operation is: This assigns '(output_y * stride_h - pad_h + (winner_index // kernel_w) * dilation_h)' to 'input_y ' in host memory. Here input_y means the value named 'input y' in this block. The important operands are output_y: the value named 'output y' in this block; stride_h: the value named 'stride h' in this block; pad_h: the value named 'pad h' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1300: `+ (winner_index // kernel_w) * dilation_h` This is continuation line 3 of the 4-line logical statement that begins at source line 1298. Its exact fragment is '+ (winner_index // kernel_w) * dilation_h'. The important operands are winner_index: an integer selecting one tensor element, lane, bank, or state entry; kernel_w: the value named 'kernel w' in this block; dilation_h: the value named 'dilation h' in this block. Read together, the complete operation is: This assigns '(output_y * stride_h - pad_h + (winner_index // kernel_w) * dilation_h)' to 'input_y ' in host memory. Here input_y means the value named 'input y' in this block. The important operands are output_y: the value named 'output y' in this block; stride_h: the value named 'stride h' in this block; pad_h: the value named 'pad h' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1301: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1298. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(output_y * stride_h - pad_h + (winner_index // kernel_w) * dilation_h)' to 'input_y ' in host memory. Here input_y means the value named 'input y' in this block. The important operands are output_y: the value named 'output y' in this block; stride_h: the value named 'stride h' in this block; pad_h: the value named 'pad h' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1302: `input_x = (` This statement continues through source line 1305; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(output_x * stride_w - pad_w + (winner_index % kernel_w) * dilation_w)' to 'input_x ' in host memory. Here input_x means the value named 'input x' in this block. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1303: `output_x * stride_w - pad_w` This is continuation line 2 of the 4-line logical statement that begins at source line 1302. Its exact fragment is 'output_x * stride_w - pad_w'. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Read together, the complete operation is: This assigns '(output_x * stride_w - pad_w + (winner_index % kernel_w) * dilation_w)' to 'input_x ' in host memory. Here input_x means the value named 'input x' in this block. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1304: `+ (winner_index % kernel_w) * dilation_w` This is continuation line 3 of the 4-line logical statement that begins at source line 1302. Its exact fragment is '+ (winner_index % kernel_w) * dilation_w'. The important operands are winner_index: an integer selecting one tensor element, lane, bank, or state entry; kernel_w: the value named 'kernel w' in this block; dilation_w: the value named 'dilation w' in this block. Read together, the complete operation is: This assigns '(output_x * stride_w - pad_w + (winner_index % kernel_w) * dilation_w)' to 'input_x ' in host memory. Here input_x means the value named 'input x' in this block. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1305: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1302. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(output_x * stride_w - pad_w + (winner_index % kernel_w) * dilation_w)' to 'input_x ' in host memory. Here input_x means the value named 'input x' in this block. The important operands are output_x: the value named 'output x' in this block; stride_w: the value named 'stride w' in this block; pad_w: the value named 'pad w' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1306: `updates.append(` This statement continues through source line 1308; the explanation uses the complete expression rather than only the first visual fragment. This calls 'updates.append' with ' ((batch_index, channel, input_y, input_x), int(gradient[window])) ' inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1307: `((batch_index, channel, input_y, input_x), int(gradient[window]))` This is continuation line 2 of the 3-line logical statement that begins at source line 1306. Its exact fragment is '((batch_index, channel, input_y, input_x), int(gradient[window]))'. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry; channel: the value named 'channel' in this block; input_y: the value named 'input y' in this block. Read together, the complete operation is: This calls 'updates.append' with ' ((batch_index, channel, input_y, input_x), int(gradient[window])) ' inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1308: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 1306. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'updates.append' with ' ((batch_index, channel, input_y, input_x), int(gradient[window])) ' inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1310: `# Each vector command updates distinct addresses. Repeated winners are` The comment records this intent: Each vector command updates distinct addresses. Repeated winners are It applies inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1311: `# deferred to a later round, preserving output-window order per address.` The comment records this intent: deferred to a later round, preserving output-window order per address. It applies inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1312: `pending = list(updates)` This assigns 'list(updates)' to 'pending ' in host memory. Here pending means the value named 'pending' in this block. The important operands are list: the value named 'list' in this block; updates: the value named 'updates' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1313: `while pending:` This repeats the following host-code block while 'pending' remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 1314: `selected: list[tuple[tuple[int, int, int, int], int]] = []` This assigns '[]' to 'selected' in host memory. Here selected means the value named 'selected' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1315: `deferred: list[tuple[tuple[int, int, int, int], int]] = []` This assigns '[]' to 'deferred' in host memory. Here deferred means the value named 'deferred' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1316: `seen: set[tuple[int, int, int, int]] = set()` This assigns 'set()' to 'seen' in host memory. Here seen means the value named 'seen' in this block. The important operands are set: the value named 'set' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1317: `for update in pending:` This iterates update over 'pending'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1318: `if update[0] in seen or len(selected) == 8:` This executes the following indented host-code branch only when 'update[0] in seen or len(selected) == 8' is true. The important operands are update: the value named 'update' in this block; in: the value named 'in' in this block; seen: the value named 'seen' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1319: `deferred.append(update)` This calls ‘deferred.append‘ with ‘update‘ inside the function execute_maxpool2d_backw which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1320: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1321: `selected.append(update)` This calls ‘selected.append‘ with ‘update‘ inside the function execute_maxpool2d_backwa which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1322: `seen.add(update[0])` This calls ‘seen.add‘ with ‘update[0]‘ inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1323: `left = [int(input_gradient[address]) for address, _ in selected]` This assigns ‘[int(input_gradient[addre for address, _ in selected]‘ to ‘left ‘ in host memory. Here left means the value named ’left’ in this block. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; address: the value named ’address’ in this block; for: the value named ’for’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1324: `right = [value for _, value in selected]` This assigns ‘[value for _, value in selected]‘ to ‘right ‘ in host memory. Here right means the value named ’right’ in this block. The important operands are value: the value named ’value’ in this block; for: the value named ’for’ in this block; _: the value named ” in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1325: `left.extend([0] * (8 - len(selected)))` This calls ‘left.extend‘ with ‘[0] * (8 - len(selected))‘ inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1326: `right.extend([0] * (8 - len(selected)))` This calls ‘right.extend‘ with ‘[0] * (8 - len(selected))‘ inside the function execute_maxpool2d_backward, which handles execute maxpool2d backward. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1327: `summed = _execute_vector8(` This statement continues through source line 1333; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)‘ to ‘summed ‘ in host memory. Here summed means the value named ’summed’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1328: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1327. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)‘ to ‘summed ‘ in host memory. Here summed means the value named ’summed’ in this block. The important operands are _execute_vector8: the value named ’execute vector8’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1329: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 1327. Its exact fragment is ‘VectorOperation.ADD_SAT,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; ADD_SAT: the value named ’add sat’ in this block. Read

together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1330: `left,` This is continuation line 4 of the 7-line logical statement that begins at source line 1327. Its exact fragment is 'left,'. The important operands are left: the value named 'left' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1331: `right,` This is continuation line 5 of the 7-line logical statement that begins at source line 1327. Its exact fragment is 'right,'. The important operands are right: the value named 'right' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1332: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1327. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1333: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1327. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '_execute_vector8(driver, VectorOperation.ADD_SAT, left, right, timeout_seconds=timeout_seconds,)' to 'summed ' in host memory. Here summed means the value named 'summed' in this block. The important operands are _execute_vector8: the value named 'execute vector8' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1334: `for lane, (address, _) in enumerate(selected):` This iterates lane, (address, _) over 'enumerate(selected)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1335: `input_gradient[address] = summed[lane]` This assigns 'summed[lane]' to 'input_gradient[address] ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are summed: the value named 'summed' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1336: `pending = deferred` This assigns 'deferred' to 'pending ' in host memory. Here pending means the value named 'pending' in this block. The important operands are deferred: the value named 'deferred' in this block. Context: the function execute_maxpool2d_backward, which handles execute maxpool2d backward.

Line 1337: `return input_gradient, winners` This returns 'input_gradient, winners' to the caller of the function execute_maxpool2d_backward, which handles execute maxpool2d backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; winners: the value named 'winners' in this block.

Line 1340: `def _shape_elements(shape: Sequence[int]) -> int:` This starts function _shape_elements. Its indented body performs shape elements as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 1341: `result = 1` This assigns ‘1‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _shape_elements, which handles shape elements.

Line 1342: `for dimension in shape:` This iterates dimension over ‘shape‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _shape_elements, which handles shape elements.

Line 1343: `result *= int(dimension)` Inside the function _shape_elements, which handles shape elements, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are result: the value named ’result’ in this block; dimension: the value named ’dimension’ in this block.

Line 1344: `return result` This returns ‘result‘ to the caller of the function _shape_elements, which handles shape elements, ending this invocation. The important operands are result: the value named ’result’ in this block.

Line 1347: `def linear_backward_partial_tiles(` This starts function linear_backward_partial_tiles. Its indented body performs linear backward partial tiles as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 1348: `rows: int,` Inside the function linear_backward_partial_tiles, which handles linear backward partial tiles, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are rows: one output-row position processed by the tensor tile.

Line 1349: `inputs: int,` Inside the function linear_backward_partial_tiles, which handles linear backward partial tiles, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are inputs: the value named ’inputs’ in this block.

Line 1350: `outputs: int,` Inside the function linear_backward_partial_tiles, which handles linear backward partial tiles, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are outputs: the value named ’outputs’ in this block.

Line 1351: `*,` Inside the function linear_backward_partial_tiles, which handles linear backward partial tiles, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 1352: `repetitions: int = 1,` This assigns ‘1,‘ to ‘repetitions‘ in host memory. Here repetitions means the value named ’repetitions’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function linear_backward_partial_tiles, which handles linear backward partial tiles.

Line 1353: `) -> int:` Inside the function linear_backward_partial_tiles, which handles linear backward partial tiles, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The expression is a literal or structural operator and has no hidden software state.

Line 1354: `"""Count dX plus dW partial commands without materializing operands."""` This is executable documentation attached to the function linear_backward_partial_tiles, which handles linear backward partial tiles. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1356: `if min(rows, inputs, outputs, repetitions) <= 0:` This executes the following indented host-code branch only when ‘min(rows, inputs, outputs, repetitions) <= 0‘ is true. The important operands are min: the value named ’min’ in this block; rows: one output-row position processed by the tensor tile; inputs: the value named ’inputs’ in this block. Context: the function linear_backward_partial_tiles, which handles linear backward partial tiles.

Line 1357: `raise ValueError("linear backward dimensions and repetitions must be positive")` This rejects an invalid state by raising ‘ValueError("linear backward dimensions and repetitions must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1358: `return (` This statement continues through source line 1364; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘(2 * ceil(rows / TILE) * ceil(inputs / TILE) * ceil(outputs / TILE) * repetitions)‘ to the caller of the function linear_backward_partial_tiles, which handles linear backward partial tiles, ending this invocation. The important operands are ceil: the value named ’ceil’ in this block; rows: one output-row position processed by the tensor tile; TILE: the value named ’tile’ in this block.

Line 1359: `2` This is continuation line 2 of the 7-line logical statement that begins at source line 1358. Its exact fragment is ‘2‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘(2 * ceil(rows / TILE) * ceil(inputs / TILE) * ceil(outputs / TILE) * repetitions)‘ to the caller of the function linear_backward_partial_tiles, which handles linear backward partial tiles, ending this invocation. The important operands are ceil: the value named ’ceil’ in this block; rows: one output-row position processed by the tensor tile; TILE: the value named ’tile’ in this block.

Line 1360: `* ceil(rows / TILE)` This is continuation line 3 of the 7-line logical statement that begins at source line 1358. Its exact fragment is ‘* ceil(rows / TILE)‘. The important operands are ceil: the value named ’ceil’ in this block; rows: one output-row position processed by the tensor tile; TILE: the value named ’tile’ in this block. Read together, the complete operation is: This returns ‘(2 * ceil(rows / TILE) * ceil(inputs / TILE) * ceil(outputs / TILE) * repetitions)‘ to the caller of the function linear_backward_partial_tiles, which handles linear backward partial tiles, ending this invocation. The important operands are ceil: the value named ’ceil’ in this block; rows: one output-row position processed by the tensor tile; TILE: the value named ’tile’ in this block.

Line 1361: `* ceil(inputs / TILE)` This is continuation line 4 of the 7-line logical statement that begins at source line 1358. Its exact fragment is ‘* ceil(inputs / TILE)‘. The important operands are ceil: the value named ’ceil’ in this block; inputs: the value named ’inputs’ in this block; TILE: the value named ’tile’ in this block. Read together, the complete operation is: This returns ‘(2 * ceil(rows / TILE) * ceil(inputs / TILE) * ceil(outputs / TILE) * repetitions)‘ to the caller of the function linear_backward_partial_tiles, which handles linear backward partial tiles, ending this invocation. The important operands are ceil: the value named ’ceil’ in this block; rows: one output-row position processed by the tensor tile; TILE: the value named ’tile’ in this block.

Line 1362: `* ceil(outputs / TILE)` This is continuation line 5 of the 7-line logical statement that begins at source line 1358. Its exact fragment is ‘* ceil(outputs / TILE)‘. The important operands are ceil: the value named ’ceil’ in this block; outputs: the value named ’outputs’ in this block; TILE: the value named ’tile’ in this block. Read together, the complete operation is: This returns ‘(2 * ceil(rows / TILE) * ceil(inputs / TILE) * ceil(outputs / TILE) * repetitions)‘ to the caller of the function linear_backward_partial_tiles, which handles linear backward partial tiles, ending this invocation. The important operands are ceil: the value named ’ceil’ in this block; rows: one output-row position processed by the tensor tile; TILE: the value named ’tile’ in this block.

Line 1363: `* repetitions` This is continuation line 6 of the 7-line logical statement that begins at source line 1358. Its exact fragment is ‘* repetitions‘. The important operands are repetitions: the value named ’repetitions’ in this block. Read together, the complete operation is: This returns ‘(2 * ceil(rows / TILE) * ceil(inputs / TILE) * ceil(outputs / TILE) * repetitions)‘ to the caller of the function linear_backward_partial_tiles, which handles linear backward partial

tiles, ending this invocation. The important operands are ceil: the value named 'ceil' in this block; rows: one output-row position processed by the tensor tile; TILE: the value named 'tile' in this block.

Line 1364: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1358. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns '(2 * ceil(rows / TILE) * ceil(inputs / TILE) * ceil(outputs / TILE) * repetitions)' to the caller of the function linear_backward_partial_tiles, which handles linear backward partial tiles, ending this invocation. The important operands are ceil: the value named 'ceil' in this block; rows: one output-row position processed by the tensor tile; TILE: the value named 'tile' in this block.

Line 1367: `def build_spikingformer_training_coverage(` This starts function build_spikingformer_training_coverage. Its indented body performs build spikingformer training coverage as part of the file that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates; it runs only when called.

Line 1368: `execution_manifest_path: str | Path,` Inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are execution_manifest_path: the value named 'execution manifest path' in this block; str: the value named 'str' in this block; Path: the value named 'path' in this block.

Line 1369: `parameter_manifest_path: str | Path,` Inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are parameter_manifest_path: the value named 'parameter manifest path' in this block; str: the value named 'str' in this block; Path: the value named 'path' in this block.

Line 1370: `) -> dict[str, object]:` Inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are dict: the value named 'dict' in this block; str: the value named 'str' in this block; object: the value named 'object' in this block.

Line 1371: `"""Prove hardware backward/update coverage for the exported deployed graph."""` This is executable documentation attached to the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1373: `execution_path = Path(execution_manifest_path).expanduser().resolve()` This assigns 'Path(execution_ma to 'execution_path ' in host memory. Here execution_path means the value named 'execution path' in this block. The important operands are Path: the value named 'path' in this block; execution_manifest_path: the value named 'execution manifest path' in this block; expanduser: the value named 'expanduser' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1374: `parameter_path = Path(parameter_manifest_path).expanduser().resolve()` This assigns 'Path(parameter_m to 'parameter_path ' in host memory. Here parameter_path means the value named 'parameter path' in this block. The important operands are Path: the value named 'path' in this block; parameter_manifest_path: the value named 'parameter manifest path' in this block; expanduser: the value named 'expanduser' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1375: `execution = json.loads(execution_path.read_text(encoding="ascii"))` This assigns 'json.loads(execution_p to 'execution ' in host memory. Here execution means the value named 'execution' in this block. The important operands are json: the value named 'json' in this block; loads: the value named 'loads' in this block; execution_path: the value named 'execution path' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1376: `parameters = json.loads(parameter_path.read_text(encoding="ascii"))` This assigns ‘json.loads(parameter_ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are json: the value named ’json’ in this block; loads: the value named ’loads’ in this block; parameter_path: the value named ’parameter path’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1377: `operators = {item["name"]: item for item in execution["operators"]}` This assigns ‘{item["name"]: item for item in execution["operators"]}‘ to ‘operators ‘ in host memory. Here operators means the value named ’operators’ in this block. The important operands are item: the value named ’item’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1378: `projection_entries = {item["name"]: item for item in parameters["projections"]}` This assigns ‘{item["name"]: item for item in parameters["projections"]}‘ to ‘projection_entries ‘ in host memory. Here projection_entries means the value named ’projection entries’ in this block. The important operands are item: the value named ’item’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1379: `learned_names = tuple(projection_entries) + ("head",)` This assigns ‘tuple(projection_entries) + ("head",)‘ to ‘learned_names ‘ in host memory. Here learned_names means the value named ’learned names’ in this block. The important operands are tuple: the value named ’tuple’ in this block; projection_entries: the value named ’projection entries’ in this block; head: the value named ’head’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1380: `missing_operators = sorted(set(learned_names) - set(operators))` This assigns ‘sorted(set(learned_names) - set(operators))‘ to ‘missing_operators ‘ in host memory. Here missing_operators means the value named ’missing operators’ in this block. The important operands are sorted: the value named ’sorted’ in this block; set: the value named ’set’ in this block; learned_names: the value named ’learned names’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1381: `if missing_operators:` This executes the following indented host-code branch only when ‘missing_operators‘ is true. The important operands are missing_operators: the value named ’missing operators’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1382: `raise ValueError(f"learned parameter groups lack graph operators: {missing_operators}")` This rejects an invalid state by raising ‘ValueError(f"learned parameter groups lack graph operators: {missing_operators}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1384: `learned_coverage: list[dict[str, object]] = []` This assigns ‘[]‘ to ‘learned_coverage‘ in host memory. Here learned_coverage means the value named ’learned coverage’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1385: `learned_parameter_elements = 0` This assigns ‘0‘ to ‘learned_parameter_elements ‘ in host memory. Here learned_parameter_elements means the value named ’learned parameter elements’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1386: `learned_backward_tiles = 0` This assigns ‘0‘ to ‘learned_backward_tiles ‘ in host memory. Here learned_backward_tiles means the value named ’learned backward tiles’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1387: `learned_backward_multiplications = 0` This assigns ‘0‘ to ‘learned_backward_multiplications ‘ in host memory. Here learned_backward_multiplications means the value named ’learned backward multiplications’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1388: `for name in learned_names:` This iterates name over ‘learned_names‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1389: `operator = operators[name]` This assigns ‘operators[name]‘ to ‘operator ‘ in host memory. Here operator means the value named ’operator’ in this block. The important operands are operators: the value named ’operators’ in this block; name: the value named ’name’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1390: `matrix = operator["matrix"]` This assigns ‘operator["matrix"]‘ to ‘matrix ‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are operator: the value named ’operator’ in this block; matrix: the value named ’matrix’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1391: `rows = int(matrix["rows"])` This assigns ‘int(matrix["rows"])‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are matrix: the value named ’matrix’ in this block; rows: one output-row position processed by the tensor tile. Context: the function build_spikingformer_training_covera which handles build spikingformer training coverage.

Line 1392: `inputs = int(matrix["reduction"])` This assigns ‘int(matrix["reduction"])‘ to ‘inputs ‘ in host memory. Here inputs means the value named ’inputs’ in this block. The important operands are matrix: the value named ’matrix’ in this block; reduction: the value named ’reduction’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1393: `outputs = int(matrix["columns"])` This assigns ‘int(matrix["columns"])‘ to ‘outputs ‘ in host memory. Here outputs means the value named ’outputs’ in this block. The important operands are matrix: the value named ’matrix’ in this block; columns: one output-feature position processed in parallel. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1394: `repetitions = int(matrix.get("repetitions", 1))` This assigns ‘int(matrix.get("repetitions", 1))‘ to ‘repetitions ‘ in host memory. Here repetitions means the value named ’repetitions’ in this block. The important operands are matrix: the value named ’matrix’ in this block; get: the value named ’get’ in this block; repetitions: the value named ’repetitions’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1395: `augmented_inputs = inputs + 1` This assigns ‘inputs + 1‘ to ‘augmented_inputs ‘ in host memory. Here augmented_inputs means the value named ’augmented inputs’ in this block. The important operands are inputs: the value named ’inputs’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1396: `partial_tiles = linear_backward_partial_tiles(` This statement continues through source line 1401; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘linear_backward_partial_tiles(rows, augmented_inputs, outputs, repetitions=repetitions,)‘ to ‘partial_tiles ‘ in host memory. Here partial_tiles means the value named ’partial tiles’ in this block. The important operands are linear_backward_partial_tiles: the value named ’linear backward partial tiles’ in this block; rows: one output-row position processed by the tensor tile; augmented_inputs: the value named ’augmented inputs’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1397: `rows,` This is continuation line 2 of the 6-line logical statement that begins at source line 1396. Its exact fragment is ‘rows,‘. The important operands are rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This assigns ‘linear_backward_partial_tiles(rows, augmented_inputs, outputs, repetitions=repetitions,)‘ to ‘partial_tiles ‘ in host memory. Here partial_tiles means the value named ’partial tiles’ in this block. The important operands are linear_backward_partial_tiles: the value named ’linear backward partial tiles’ in this block; rows: one output-row position processed by the tensor tile; augmented_inputs: the value named ’augmented inputs’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1398: `augmented_inputs,` This is continuation line 3 of the 6-line logical statement that begins at source line 1396. Its exact fragment is ‘augmented_inputs,‘. The important operands are augmented_inputs: the value named ’augmented inputs’ in this block. Read together, the complete operation is: This assigns ‘linear_backward_partial_tiles(rows, augmented_inputs, outputs, repetitions=repetitions,)‘ to ‘partial_tiles ‘ in host memory. Here partial_tiles means the value named ’partial tiles’ in this block. The important operands are linear_backward_partial_tiles: the value named ’linear backward partial tiles’ in this block; rows: one output-row position processed by the tensor tile; augmented_inputs: the value named ’augmented inputs’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1399: `outputs,` This is continuation line 4 of the 6-line logical statement that begins at source line 1396. Its exact fragment is ‘outputs,‘. The important operands are outputs: the value named ’outputs’ in this block. Read together, the complete operation is: This assigns ‘linear_backward_partial_tiles(rows, augmented_inputs, outputs, repetitions=repetitions,)‘ to ‘partial_tiles ‘ in host memory. Here partial_tiles means the value named ’partial tiles’ in this block. The important operands are linear_backward_partial_tiles: the value named ’linear backward partial tiles’ in this block; rows: one output-row position processed by the tensor tile; augmented_inputs: the value named ’augmented inputs’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1400: `repetitions=repetitions,` This is continuation line 5 of the 6-line logical statement that begins at source line 1396. Its exact fragment is ‘repetitions=repetitions,‘. The important operands are repetitions: the value named ’repetitions’ in this block. Read together, the complete operation is: This assigns ‘linear_backward_partial_tiles(rows, augmented_inputs, outputs, repetitions=repetitions,)‘ to ‘partial_tiles ‘ in host memory. Here partial_tiles means the value named ’partial tiles’ in this block. The important operands are linear_backward_partial_tiles: the value named ’linear backward partial tiles’ in this block; rows: one output-row position processed by the tensor tile; augmented_inputs: the value named ’augmented inputs’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1401: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1396. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘linear_backward_partial_tiles(rows, augmented_inputs, outputs, repetitions=repetitions,)‘ to ‘partial_tiles ‘ in host memory. Here partial_tiles means the value named ’partial tiles’ in this block. The important operands are linear_backward_partial_tiles: the value named ’linear backward partial tiles’ in this block; rows: one output-row position processed by the tensor tile; augmented_inputs: the value named ’augmented inputs’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1402: `multiplications = 2 * rows * augmented_inputs * outputs * repetitions` This assigns ‘2 * rows * augmented_inputs * outputs * repetitions‘ to ‘multiplications ‘ in host memory. Here multiplications means the value named ’multiplications’ in this block. The important operands are rows: one output-row position processed by the tensor tile; augmented_inputs: the value named ’augmented inputs’ in this block; outputs: the value named ’outputs’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1403: `if name == "head":` This executes the following indented host-code branch only when ‘name == "head"‘ is true. The important operands are name: the value named ’name’ in this block; head: the value named ’head’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1404: `entry = parameters["head"]` This assigns ‘parameters["head"]‘ to ‘entry ‘ in host memory. Here entry means the value named ’entry’ in this block. The important operands are parameters: the value named ’parameters’ in this block; head: the value named ’head’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1405: `weight_elements = _shape_elements(entry["hardware_matrix_shape"])` This assigns ‘_shape_elements(entry[to ‘weight_elements ‘ in host memory. Here weight_elements means a learned coefficient multiplied by an activation. The important operands are _shape_elements: the value named ’shape elements’ in this block; entry: the value named ’entry’ in this block; hardware_matrix_shape: the value named ’hardware matrix shape’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1406: `bias_elements = int(entry["hardware_matrix_shape"][1])` This assigns ‘int(entry["hardware_matrix_shape"][to ‘bias_elements ‘ in host memory. Here bias_elements means an additive learned offset applied after accumulation. The important operands are entry: the value named ’entry’ in this block; hardware_matrix_shape: the value named ’hardware matrix shape’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1407: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1408: `entry = projection_entries[name]` This assigns ‘projection_entries[name]‘ to ‘entry ‘ in host memory. Here entry means the value named ’entry’ in this block. The important operands are projection_entries: the value named ’projection entries’ in this block; name: the value named ’name’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1409: `weight_elements = _shape_elements(entry["hardware_matrix_shape"])` This assigns ‘_shape_elements(entry[to ‘weight_elements ‘ in host memory. Here weight_elements means a learned coefficient multiplied by an activation. The important operands are _shape_elements: the value named ’shape elements’ in this block; entry: the value named ’entry’ in this block; hardware_matrix_shape: the value named ’hardware matrix shape’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1410: `bias_elements = int(entry["hardware_matrix_shape"][1])` This assigns ‘int(entry["hardware_matrix_shape"][to ‘bias_elements ‘ in host memory. Here bias_elements means an additive learned offset applied after accumulation. The important operands are entry: the value named ’entry’ in this block; hardware_matrix_shape: the value named ’hardware matrix shape’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1411: `parameter_elements = weight_elements + bias_elements` This assigns ‘weight_elements + bias_elements‘ to ‘parameter_elements ‘ in host memory. Here parameter_elements means the value named ’parameter elements’ in this block. The important operands are weight_elements: a learned coefficient multiplied by an activation; bias_elements: an additive learned offset applied after accumulation. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1412: `learned_parameter_elements += parameter_elements` Inside the function build_spikingformer_training_coverag which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are learned_parameter_elements: the value named ’learned parameter elements’ in this block; parameter_elements: the value named ’parameter elements’ in this block.

Line 1413: `learned_backward_tiles += partial_tiles` Inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are learned_backward_tiles: the value named ’learned backward tiles’ in this block; partial_tiles: the value named ’partial tiles’ in this block.

Line 1414: `learned_backward_multiplications += multiplications` Inside the function build_spikingformer_training_cove which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are learned_backward_multiplications: the value named 'learned backward multiplications' in this block; multiplications: the value named 'multiplications' in this block.

Line 1415: `learned_coverage.append({` This statement continues through source line 1428; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1416: `"name": name,` This is continuation line 2 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"name": name,‘. The important operands are name: the value named 'name' in this block. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_tr which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1417: `"rows": rows,` This is continuation line 3 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"rows": rows,‘. The important operands are rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1418: `"inputs": inputs,` This is continuation line 4 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"inputs": inputs,‘. The important operands are inputs: the value named 'inputs' in this block. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1419: `"outputs": outputs,` This is continuation line 5 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"outputs": outputs,‘. The important operands are outputs: the value named 'outputs' in this block. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the

host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1420: `"repetitions": repetitions,` This is continuation line 6 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"repetitions": repetitions,‘. The important operands are repetitions: the value named 'repetitions' in this block. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1421: `"weight_elements": weight_elements,` This is continuation line 7 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"weight_elements": weight_elements,‘. The important operands are weight_elements: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1422: `"bias_elements": bias_elements,` This is continuation line 8 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"bias_elements": bias_elements,‘. The important operands are bias_elements: an additive learned offset applied after accumulation. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1423: `"trainable_deployed_elements": parameter_elements,` This is continuation line 9 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"trainable_deployed_elements": parameter_elements,‘. The important operands are trainable_deployed_elements: the value named 'trainable deployed elements' in this block; parameter_elements: the value named 'parameter elements' in this block. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1424: `"backward_partial_tiles": partial_tiles,` This is continuation line 10 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"backward_partial_tiles": partial_tiles,‘. The important operands are backward_partial_tiles: the value named 'backward partial tiles' in this block; partial_tiles: the value named 'partial tiles' in this block. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances

the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1425: `"backward_multiplications": multiplications,` This is continuation line 11 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"backward_multiplications": multiplications,‘. The important operands are backward_multiplications: the value named ’backward multiplications’ in this block; multiplications: the value named ’multiplications’ in this block. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1426: `"gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row",` This is continuation line 12 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row",‘. The important operands are gradient_method: the signed loss derivative used to update a learned parameter; bit: the value named ’bit’ in this block; raw: the value named ’raw’ in this block. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1427: `"update_method": "eight-lane signed fixed-point SGDW in programmable logic",` This is continuation line 13 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘"update_method": "eight-lane signed fixed-point SGDW in programmable logic",‘. The important operands are update_method: the value named ’update method’ in this block; eight: the value named ’eight’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1428: `})` This is continuation line 14 of the 14-line logical statement that begins at source line 1415. Its exact fragment is ‘})‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘learned_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "weight_elements": weight_elements, "bias_elements": bias_elements, "trainable_deployed_elements": parameter_elements, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "47-bit raw accumulation for dX/dW; homogeneous bias row", "update_method": "eight-lane signed fixed-point SGDW in programmable logic", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1430: `dynamic_names = tuple(` This statement continues through source line 1432; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘tuple(name for name in operators if name.endswith(".attn.qk") or name.endswith(".attn.av"))‘ to ‘dynamic_names ‘ in host memory. Here dynamic_names means the value named ’dynamic names’ in this block. The important operands are tuple: the value named ’tuple’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1431: `name for name in operators if name.endswith(".attn.qk") or name.endswith(".attn.av")` This is continuation line 2 of the 3-line logical statement that begins at source line 1430. Its exact fragment is ‘name for name in operators if name.endswith(".attn.qk") or name.endswith(".attn.av")‘. The important operands are name: the value named ’name’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘tuple(name for name in operators if name.endswith(".attn.qk") or name.endswith(".attn.av"))‘ to ‘dynamic_names ‘ in host memory. Here dynamic_names means the value named ’dynamic names’ in this block. The important operands are tuple: the value named ’tuple’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1432: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 1430. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘tuple(name for name in operators if name.endswith(".attn.qk") or name.endswith(".attn.av"))‘ to ‘dynamic_names ‘ in host memory. Here dynamic_names means the value named ’dynamic names’ in this block. The important operands are tuple: the value named ’tuple’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1433: `dynamic_coverage: list[dict[str, object]] = []` This assigns ‘[]‘ to ‘dynamic_coverage‘ in host memory. Here dynamic_coverage means the value named ’dynamic coverage’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1434: `dynamic_tiles = 0` This assigns ‘0‘ to ‘dynamic_tiles ‘ in host memory. Here dynamic_tiles means the value named ’dynamic tiles’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1435: `dynamic_multiplications = 0` This assigns ‘0‘ to ‘dynamic_multiplications ‘ in host memory. Here dynamic_multiplications means the value named ’dynamic multiplications’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1436: `for name in dynamic_names:` This iterates name over ‘dynamic_names‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1437: `matrix = operators[name]["matrix"]` This assigns ‘operators[name]["matrix"]‘ to ‘matrix ‘ in host memory. Here matrix means the value named ’matrix’ in this block. The important operands are operators: the value named ’operators’ in this block; name: the value named ’name’ in this block; matrix: the value named ’matrix’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1438: `rows = int(matrix["rows"])` This assigns ‘int(matrix["rows"])‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are matrix: the value named ’matrix’ in this block; rows: one output-row position processed by the tensor tile. Context: the function build_spikingformer_training_covera which handles build spikingformer training coverage.

Line 1439: `inputs = int(matrix["reduction"])` This assigns ‘int(matrix["reduction"])‘ to ‘inputs ‘ in host memory. Here inputs means the value named ’inputs’ in this block. The important operands are matrix: the value named ’matrix’ in this block; reduction: the value named ’reduction’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1440: `outputs = int(matrix["columns"])` This assigns ‘int(matrix["columns"])‘ to ‘outputs ‘ in host memory. Here outputs means the value named ’outputs’ in this block. The important operands are matrix: the value

named 'matrix' in this block; columns: one output-feature position processed in parallel. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1441: `repetitions = int(matrix.get("repetitions", 1))` This assigns ‘int(matrix.get("repetitions", 1))‘ to ‘repetitions ‘ in host memory. Here repetitions means the value named 'repetitions' in this block. The important operands are matrix: the value named 'matrix' in this block; get: the value named 'get' in this block; repetitions: the value named 'repetitions' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1442: `partial_tiles = linear_backward_partial_tiles(` This statement continues through source line 1444; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘linear_backward_partial_tiles(rows, inputs, outputs, repetitions=repetitions)‘ to ‘partial_tiles ‘ in host memory. Here partial_tiles means the value named 'partial tiles' in this block. The important operands are linear_backward_partial_tiles: the value named 'linear backward partial tiles' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1443: `rows, inputs, outputs, repetitions=repetitions` This is continuation line 2 of the 3-line logical statement that begins at source line 1442. Its exact fragment is ‘rows, inputs, outputs, repetitions=repetitions‘. The important operands are rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block; outputs: the value named 'outputs' in this block. Read together, the complete operation is: This assigns ‘linear_backward_partial_tiles(rows, inputs, outputs, repetitions=repetitions)‘ to ‘partial_tiles ‘ in host memory. Here partial_tiles means the value named 'partial tiles' in this block. The important operands are linear_backward_partial_tiles: the value named 'linear backward partial tiles' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1444: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 1442. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘linear_backward_partial_tiles(rows, inputs, outputs, repetitions=repetitions)‘ to ‘partial_tiles ‘ in host memory. Here partial_tiles means the value named 'partial tiles' in this block. The important operands are linear_backward_partial_tiles: the value named 'linear backward partial tiles' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1445: `multiplications = 2 * rows * inputs * outputs * repetitions` This assigns ‘2 * rows * inputs * outputs * repetitions‘ to ‘multiplications ‘ in host memory. Here multiplications means the value named 'multiplications' in this block. The important operands are rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block; outputs: the value named 'outputs' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1446: `dynamic_tiles += partial_tiles` Inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are dynamic_tiles: the value named 'dynamic tiles' in this block; partial_tiles: the value named 'partial tiles' in this block.

Line 1447: `dynamic_multiplications += multiplications` Inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are dynamic_multiplications: the value named 'dynamic multiplications' in this block; multiplications: the value named 'multiplications' in this block.

Line 1448: `dynamic_coverage.append({` This statement continues through source line 1457; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘dynamic_coverage.append‘ with ‘{ "name":

name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1449: `"name": name,` This is continuation line 2 of the 10-line logical statement that begins at source line 1448. Its exact fragment is ‘"name": name,‘. The important operands are name: the value named ’name’ in this block. Read together, the complete operation is: This calls ‘dynamic_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1450: `"rows": rows,` This is continuation line 3 of the 10-line logical statement that begins at source line 1448. Its exact fragment is ‘"rows": rows,‘. The important operands are rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This calls ‘dynamic_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1451: `"inputs": inputs,` This is continuation line 4 of the 10-line logical statement that begins at source line 1448. Its exact fragment is ‘"inputs": inputs,‘. The important operands are inputs: the value named ’inputs’ in this block. Read together, the complete operation is: This calls ‘dynamic_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1452: `"outputs": outputs,` This is continuation line 5 of the 10-line logical statement that begins at source line 1448. Its exact fragment is ‘"outputs": outputs,‘. The important operands are outputs: the value named ’outputs’ in this block. Read together, the complete operation is: This calls ‘dynamic_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1453: `"repetitions": repetitions,` This is continuation line 6 of the 10-line logical statement that begins at source line 1448. Its exact fragment is ‘"repetitions": repetitions,‘. The important operands are repetitions: the value named ’repetitions’ in this block. Read together, the complete operation is: This calls ‘dynamic_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1454: `"backward_partial_tiles": partial_tiles,` This is continuation line 7 of the 10-line logical statement that begins at source line 1448. Its exact fragment is ‘"backward_partial_tiles": partial_tiles,‘. The important operands are backward_partial_tiles: the value named ’backward partial tiles’ in this block; partial_tiles: the value named ’partial tiles’ in this block. Read together, the complete operation is: This calls ‘dynamic_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles,

"backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1455: `"backward_multiplications": multiplications,` This is continuation line 8 of the 10-line logical statement that begins at source line 1448. Its exact fragment is ‘"backward_multiplications": multiplications,‘. The important operands are backward_multiplications: the value named ’backward multiplications’ in this block; multiplications: the value named ’multiplications’ in this block. Read together, the complete operation is: This calls ‘dynamic_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1456: `"gradient_method": "two signed raw-accumulator matrix passes",` This is continuation line 9 of the 10-line logical statement that begins at source line 1448. Its exact fragment is ‘"gradient_method": "two signed raw-accumulator matrix passes",‘. The important operands are gradient_method: the signed loss derivative used to update a learned parameter; two: the value named ’two’ in this block; raw: the value named ’raw’ in this block. Read together, the complete operation is: This calls ‘dynamic_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1457: `})` This is continuation line 10 of the 10-line logical statement that begins at source line 1448. Its exact fragment is ‘})‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘dynamic_coverage.append‘ with ‘{ "name": name, "rows": rows, "inputs": inputs, "outputs": outputs, "repetitions": repetitions, "backward_partial_tiles": partial_tiles, "backward_multiplications": multiplications, "gradient_method": "two signed raw-accumulator matrix passes", }‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1459: `operator_coverage: list[dict[str, str]] = []` This assigns ‘[]‘ to ‘operator_coverage‘ in host memory. Here operator_coverage means the value named ’operator coverage’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1460: `uncovered: list[str] = []` This assigns ‘[]‘ to ‘uncovered‘ in host memory. Here uncovered means the value named ’uncovered’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1461: `for name, operator in operators.items():` This iterates name, operator over ‘operators.items()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1462: `source = str(operator["source_operation"])` This assigns ‘str(operator["source_operation"])‘ to ‘source ‘ in host memory. Here source means the value named ’source’ in this block. The important operands are str: the value named ’str’ in this block; operator: the value named ’operator’ in this block; source_operation: the value named ’source operation’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1463: `if name in learned_names:` This executes the following indented host-code branch only when ‘name in learned_names‘ is true. The important operands are name: the value named ’name’ in this block; in: the value

named 'in' in this block; learned_names: the value named 'learned names' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1464: `method = "affine dX/dW/dB plus SGDW"` This assigns ‘"affine dX/dW/dB plus SGDW"‘ to ‘method ‘ in host memory. Here method means the value named 'method' in this block. The important operands are affine: the value named 'affine' in this block; dX: the value named 'd x' in this block; dW: the value named 'd w' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1465: `if "MaxPool2d" in source:` This executes the following indented host-code branch only when ‘"MaxPool2d" in source‘ is true. The important operands are MaxPool2d: the value named 'max pool2d' in this block; in: the value named 'in' in this block; source: the value named 'source' in this block. Context: the function build_spikingformer_training_co which handles build spikingformer training coverage.

Line 1466: `method += "; first-winner max-pool backward and fabric scatter-add"` Inside the function build_spikingfor which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are method: the value named 'method' in this block; first: the value named 'first' in this block; winner: the value named 'winner' in this block.

Line 1467: `if "LIF" in source:` This executes the following indented host-code branch only when ‘"LIF" in source‘ is true. The important operands are LIF: the value named 'lif' in this block; in: the value named 'in' in this block; source: the value named 'source' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1468: `method += "; traced sigmoid-surrogate hard-LIF BPTT"` Inside the function build_spikingformer_training_co which handles build spikingformer training coverage, this statement advances the host implementation that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates. The important operands are method: the value named 'method' in this block; traced: the value named 'traced' in this block; sigmoid: the value named 'sigmoid' in this block.

Line 1469: `elif name in dynamic_names:` This executes the following indented host-code branch only when ‘name in dynamic_names‘ is true. The important operands are name: the value named 'name' in this block; in: the value named 'in' in this block; dynamic_names: the value named 'dynamic names' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1470: `method = "dynamic matrix dX/dW; 0.125 attention gradient uses Q15 multiply"` This assigns ‘"dynamic matrix dX/dW; 0.125 attention gradient uses Q15 multiply"‘ to ‘method ‘ in host memory. Here method means the value named 'method' in this block. The important operands are dynamic: the value named 'dynamic' in this block; matrix: the value named 'matrix' in this block; dX: the value named 'd x' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1471: `elif name.endswith(".residuals"):` This executes the following indented host-code branch only when ‘name.endswith(".residuals")‘ is true. The important operands are name: the value named 'name' in this block; endswith: the value named 'endswith' in this block; residuals: the value named 'residuals' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1472: `method = "identity gradient fan-out; converging branches use saturating vector add"` This assigns ‘"identity gradient fan-out; converging branches use saturating vector add"‘ to ‘method ‘ in host memory. Here method means the value named 'method' in this block. The important operands are identity: the value named 'identity' in this block; gradient: the signed loss derivative used to update a learned parameter; fan: the value named 'fan' in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1473: `elif name == "global_average_pool":` This executes the following indented host-code branch only when ‘name == "global_average_pool"‘ is true. The important operands are name: the value named ’name’ in this block; global_average_pool: the value named ’global average pool’ in this block. Context: the function build_spikingformer_training_cove which handles build spikingformer training coverage.

Line 1474: `method = "gradient broadcast with exact power-of-two division by 256"` This assigns ‘"gradient broadcast with exact power-of-two division by 256"‘ to ‘method ‘ in host memory. Here method means the value named ’method’ in this block. The important operands are gradient: the signed loss derivative used to update a learned parameter; broadcast: the value named ’broadcast’ in this block; with: the value named ’with’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1475: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1476: `uncovered.append(name)` This calls ‘uncovered.append‘ with ‘name‘ inside the function build_spikingformer_trainin which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1477: `method = "UNMAPPED"` This assigns ‘"UNMAPPED"‘ to ‘method ‘ in host memory. Here method means the value named ’method’ in this block. The important operands are UNMAPPED: the value named ’unmapped’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1478: `operator_coverage.append({"name": name, "backward_method": method})` This calls ‘operator_coverage.app with ‘{"name": name, "backward_method": method}‘ inside the function build_spikingformer_training_coverage, which handles build spikingformer training coverage. The call advances the host-side job that builds quantized forward, backward, surrogate-gradient, and optimizer programs for hardware-resident supervised updates.

Line 1479: `if uncovered:` This executes the following indented host-code branch only when ‘uncovered‘ is true. The important operands are uncovered: the value named ’uncovered’ in this block. Context: the function build_spikingformer_training_ which handles build spikingformer training coverage.

Line 1480: `raise ValueError(f"execution operators lack backward methods: {uncovered}")` This rejects an invalid state by raising ‘ValueError(f"execution operators lack backward methods: {uncovered}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1482: `depth = int(execution["model"]["transformer_blocks"])` This assigns ‘int(execution["model"]["transformer_bl to ‘depth ‘ in host memory. Here depth means the value named ’depth’ in this block. The important operands are execution: the value named ’execution’ in this block; model: the value named ’model’ in this block; transformer_blocks: the value named ’transformer blocks’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1483: `lif_nodes = 4 + 7 * depth` This assigns ‘4 + 7 * depth‘ to ‘lif_nodes ‘ in host memory. Here lif_nodes means the value named ’lif nodes’ in this block. The important operands are depth: the value named ’depth’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1484: `optimizer_commands = ceil(learned_parameter_elements / 8)` This assigns ‘ceil(learned_parameter_elements / 8)‘ to ‘optimizer_commands ‘ in host memory. Here optimizer_commands means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are ceil: the value named ’ceil’ in this block; learned_parameter_elements: the value named ’learned parameter elements’ in this block. Context: the function build_spikingformer_training_coverage, which handles build spikingformer training coverage.

Line 1485: `return {` This statement continues through source line 1522; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1486: `"format": "neuraedge-spikingformer-training-coverage-v1",` This is continuation line 2 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"format": "neuraedge-spikingformer-training-coverage-v1",‘. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_co lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1487: `"execution_manifest": str(execution_path),` This is continuation line 3 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"execution_manifest": str(execution_path),‘. The important operands are execution_manifest: the value named 'execution manifest' in this block; str: the value named 'str' in this block; execution_path: the value named 'execution path' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen

learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1488: `"parameter_manifest": str(parameter_path),` This is continuation line 4 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"parameter_manifest": str(parameter_path),‘. The important operands are parameter_manifest: the value named 'parameter manifest' in this block; str: the value named 'str' in this block; parameter_path: the value named 'parameter path' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1489: `"training_semantics": {` This is continuation line 5 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"training_semantics": {‘. The important operands are training_semantics: the value named 'training semantics' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_ "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entrie "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multip "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1490: `"target_graph": "complete exported integer inference graph",` This is continuation line 6 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"target_graph": "complete exported integer inference graph",‘. The important operands are target_graph: the value named 'target graph' in this

block; complete: the value named 'complete' in this block; exported: the value named 'exported' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_
len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count":
2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles":
dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications":
learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c
True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage":
dynamic_coverage, "operator_coverage": operator_coverage, }' to the caller of the function build_spikingformer_training_coverage
which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1491: `"quantizer_gradient": "straight-through estimator with fixed exported scales",` This is continuation line 7 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"quantizer_gradient": "straight-through estimator with fixed exported scales",'. The important operands are quantizer_gradient: the signed loss derivative used to update a learned parameter; straight: the value named 'straight' in this block; through: the value named 'through' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_
len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count":
2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles":
dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications":
learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c
True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage":
dynamic_coverage, "operator_coverage": operator_coverage, }' to the caller of the function build_spikingformer_training_coverage
which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1492: `"lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True",` This is continuation line 8 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True",'. The important operands are lif_gradient: the signed loss derivative used to update a learned parameter; SpikingJelly: the value named 'spiking jelly' in this block; sigmoid: the value named 'sigmoid' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "back-

bone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1493: `"temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces",`
This is continuation line 9 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces",‘. The important operands are temporal_method: the value named 'temporal method' in this block; four: the value named 'four' in this block; step: the value named 'step' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entrie "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multip "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1494: `"optimizer": (` This is continuation line 10 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"optimizer": (‘. The important operands are optimizer: the value named 'optimizer' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_co True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1495: `"signed fixed-point SGDW with momentum; weight decay is applied "` This is continuation line 11 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"signed fixed-point SGDW with momentum; weight decay is applied "‘. The important operands are fixed: the value named ’fixed’ in this block; point: the value named ’point’ in this block; SGDW: the value named ’sgdw’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block.

Line 1496: `"to the update direction and excluded from retained velocity"` This is continuation line 12 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"to the update direction and excluded from retained velocity"‘. The important operands are to: the value named ’to’ in this block; the: the value named ’the’ in this block; update: the value named ’update’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block.

Line 1497: `),` This is continuation line 13 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count":

2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_co True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1498: `"parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic",` This is continuation line 14 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic",‘. The important operands are parameter_residency: the value named 'parameter residency' in this block; PYNQ: the value named 'pynq' in this block; DDR: the value named 'ddr' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1499: `},` This is continuation line 15 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘},‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_co True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1500: `"summary": {` This is continuation line 16 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"summary": {'. The important operands are summary: the value named 'summary' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }' to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1501: `"execution_operator_count": len(operators),` This is continuation line 17 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"execution_operator_count": len(operators),'. The important operands are execution_operator_count: an integer count retained for control or measurable telemetry; len: the value named 'len' in this block; operators: the value named 'operators' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_co lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }' to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1502: `"covered_execution_operator_count": len(operator_coverage),` This is continuation line 18 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"covered_execution_operator_count": len(operator_coverage),'. The important operands are covered_execution_operator_count: an integer count retained for control or measurable telemetry; len: the value named 'len' in this block; operator_coverage: the value named 'operator coverage' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_

len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_co True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1503: `"learned_affine_group_count": len(learned_names),` This is continuation line 19 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"learned_affine_group_count": len(learned_names),‘. The important operands are learned_affine_group_count: an integer count retained for control or measurable telemetry; len: the value named 'len' in this block; learned_names: the value named 'learned names' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_co True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1504: `"backbone_projection_count": len(projection_entries),` This is continuation line 20 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"backbone_projection_count": len(projection_entries),‘. The important operands are backbone_projection_count: an integer count retained for control or measurable telemetry; len: the value named 'len' in this block; projection_entries: the value named 'projection entries' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_co True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1505: `"dynamic_attention_matmul_count": len(dynamic_names),` This is continuation line 21 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"dynamic_attention_matmul_count": len(dynamic_names),'. The important operands are dynamic_attention_matmul_count: an integer count retained for control or measurable telemetry; len: the value named 'len' in this block; dynamic_names: the value named 'dynamic names' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }' to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1506: `"lif_node_count": lif_nodes,` This is continuation line 22 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"lif_node_count": lif_nodes,'. The important operands are lif_node_count: an integer count retained for control or measurable telemetry; lif_nodes: the value named 'lif nodes' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }' to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1507: `"maxpool_backward_count": 2,` This is continuation line 23 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"maxpool_backward_count": 2,'. The important operands are maxpool_backward_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names

"backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_co
lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen
learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles":
learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_
+ dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications":
dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage"
learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to
the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage,
ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value
named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1508: `"residual_gradient_fanout_count": 2 * depth,` This is continuation line 24 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"residual_gradient_fanout_count": 2 * depth,‘. The important operands are residual_gradient_fanout_count: the signed loss derivative used to update a learned parameter; depth: the value named 'depth' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_
"training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entrie
"dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multip
"full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1509: `"trainable_deployed_parameter_elements": learned_parameter_elements,` This is continuation line 25 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"trainable_deployed_parameter_element
learned_parameter_elements,‘. The important operands are trainable_deployed_parameter_elements: the value named 'trainable deployed parameter elements' in this block; learned_parameter_elements: the value named 'learned parameter elements' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_
len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c
True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage
which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1510: `"sgdw_update_commands": optimizer_commands,` This is continuation line 26 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"sgdw_update_commands": optimizer_commands,‘. The important operands are sgdw_update_commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; optimizer_commands: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_co True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block.

Line 1511: `"learned_backward_partial_tiles": learned_backward_tiles,` This is continuation line 27 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"learned_backward_partial_tiles": learned_backward_tiles,‘. The important operands are learned_backward_partial_tiles: the value named ’learned backward partial tiles’ in this block; learned_backward_tiles: the value named ’learned backward tiles’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_co True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block.

Line 1512: `"dynamic_backward_partial_tiles": dynamic_tiles,` This is continuation line 28 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"dynamic_backward_partial_tiles": dynamic_tiles,‘. The important operands are dynamic_backward_partial_tiles: the value named ’dynamic backward partial tiles’ in this block; dynamic_tiles: the value named ’dynamic tiles’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors;

arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1513: `"total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles,` This is continuation line 29 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles,‘. The important operands are total_backward_partial_tiles: the value named 'total backward partial tiles' in this block; learned_backward_tiles: the value named 'learned backward tiles' in this block; dynamic_tiles: the value named 'dynamic tiles' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1514: `"learned_backward_multiplications": learned_backward_multiplications,` This is continuation line 30 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"learned_backward_multiplications": learned_backward_multiplications,‘. The important operands are learned_backward_multiplications: the value named 'learned backward multiplications' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_ "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entrie "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multip "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named

'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1515: `"dynamic_backward_multiplications": dynamic_multiplications,` This is continuation line 31 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"dynamic_backward_multiplications": dynamic_multiplications,'. The important operands are dynamic_backward_multiplications: the value named 'dynamic backward multiplications' in this block; dynamic_multiplications: the value named 'dynamic multiplications' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }' to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1516: `"full_deployed_backbone_covered": True,` This is continuation line 32 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"full_deployed_backbone_covered": True,'. The important operands are full_deployed_backbone_covered: the value named 'full deployed backbone covered' in this block. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }' to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1517: `"uncovered_operator_count": 0,` This is continuation line 33 of the 38-line logical statement that begins at source line 1485. Its exact fragment is '"uncovered_operator_count": 0,'. The important operands are uncovered_operator_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This returns '{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight

decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_co lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1518: `},` This is continuation line 34 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘},‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_c True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1519: `"learned_affine_coverage": learned_coverage,` This is continuation line 35 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"learned_affine_coverage": learned_coverage,‘. The important operands are learned_affine_coverage: the value named 'learned affine coverage' in this block; learned_coverage: the value named 'learned coverage' in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entrie "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multip "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

Line 1520: `"dynamic_attention_coverage": dynamic_coverage,` This is continuation line 36 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"dynamic_attention_coverage": dynamic_coverage,‘. The important operands are dynamic_attention_coverage: the value named ’dynamic attention coverage’ in this block; dynamic_coverage: the value named ’dynamic coverage’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block.

Line 1521: `"operator_coverage": operator_coverage,` This is continuation line 37 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘"operator_coverage": operator_coverage,‘. The important operands are operator_coverage: the value named ’operator coverage’ in this block. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_matmul_count": len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elemen learned_parameter_elements, "sgdw_update_commands": optimizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_ + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_covered": True, "uncovered_operator_count": 0, }, "learned_affine_coverage" learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage, which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named ’format’ in this block; neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block.

Line 1522: `}` This is continuation line 38 of the 38-line logical statement that begins at source line 1485. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘{ "format": "neuraedge-spikingformer-training-coverage-v1", "execution_manifest": str(execution_path), "parameter_manifest": str(parameter_path), "training_semantics": { "target_graph": "complete exported integer inference graph", "quantizer_gradient": "straight-through estimator with fixed exported scales", "lif_gradient": "SpikingJelly sigmoid surrogate alpha=4, hard reset, detach_reset=True", "temporal_method": "four-step BPTT in reverse timestep order through cached membrane traces", "optimizer": ("signed fixed-point SGDW with momentum; weight decay is applied " "to the update direction and excluded from retained velocity"), "parameter_residency": "PYNQ DDR master tensors; arithmetic and updates in programmable logic", }, "summary": { "execution_operator_count": len(operators), "covered_execution_operator_count": len(operator_coverage), "learned_affine_group_count": len(learned_names), "backbone_projection_count": len(projection_entries), "dynamic_attention_ len(dynamic_names), "lif_node_count": lif_nodes, "maxpool_backward_count": 2, "residual_gradient_fanout_count": 2 * depth, "trainable_deployed_parameter_elements": learned_parameter_elements, "sgdw_update_commands": op-

timizer_commands, "learned_backward_partial_tiles": learned_backward_tiles, "dynamic_backward_partial_tiles": dynamic_tiles, "total_backward_partial_tiles": learned_backward_tiles + dynamic_tiles, "learned_backward_multiplications": learned_backward_multiplications, "dynamic_backward_multiplications": dynamic_multiplications, "full_deployed_backbone_co True, "uncovered_operator_count": 0, }, "learned_affine_coverage": learned_coverage, "dynamic_attention_coverage": dynamic_coverage, "operator_coverage": operator_coverage, }‘ to the caller of the function build_spikingformer_training_coverage which handles build spikingformer training coverage, ending this invocation. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; spikingformer: the value named 'spikingformer' in this block.

39 software/run_spikingformer_pynq.py

Role in the complete system: loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

39.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: none. Functions: prepare_input, parse_arguments. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

39.2 Complete source

```
"""Run complete trained Spikingformer inference on a PYNQ-Z2 overlay."""

from __future__ import annotations

import argparse
import json
import time
from dataclasses import asdict
from datetime import datetime, timezone
from pathlib import Path

import numpy as np

from neuraedge_pynq import NeuraEdgeDriver
from spikingformer_accelerator import SpikingformerAcceleratorModel
from spikingformer_parameters import SpikingformerParameterArchive


CIFAR10_MEAN = np.asarray((0.4914, 0.4822, 0.4465), dtype=np.float32)
CIFAR10_STD = np.asarray((0.2470, 0.2435, 0.2616), dtype=np.float32)


def prepare_input(value: np.ndarray, input_format: str) -> np.ndarray:
    """Return normalized NCHW float32 data from raw-u8 or normalized arrays."""

    array = np.asarray(value)
    if array.ndim == 3:
        array = array[np.newaxis]
    if array.ndim != 4:
        raise ValueError("input array must have three or four dimensions")
    if array.shape[1:] == (3, 32, 32):
        nchw = array
    elif array.shape[1:] == (32, 32, 3):
        nchw = array.transpose(0, 3, 1, 2)
    else:
        raise ValueError("input array must be NCHW or NHWC CIFAR-10 shape")
    if input_format == "normalized":
        return nchw.astype(np.float32)
    if input_format != "raw-u8":
        raise ValueError("input_format must be normalized or raw-u8")
    if not np.issubdtype(nchw.dtype, np.integer) or np.any(nchw < 0) or np.any(nchw > 255):
        raise ValueError("raw-u8 input must contain integer pixels in [0,255]")
    unit = nchw.astype(np.float32) / np.float32(255.0)
    return (unit - CIFAR10_MEAN.reshape(1, 3, 1, 1)) / CIFAR10_STD.reshape(1, 3, 1, 1)


def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument("--bitfile", required=True, type=Path)
    parser.add_argument("--manifest", required=True, type=Path)
    parser.add_argument("--input", required=True, type=Path)
    parser.add_argument("--input-format", choices=("normalized", "raw-u8"), default="raw-u8")
    parser.add_argument("--output", required=True, type=Path)
```

```
    parser.add_argument("--layer-timeout-seconds", type=float, default=120.0)
    parser.add_argument("--no-download", action="store_true")
    return parser.parse_args()


if __name__ == "__main__":
    arguments = parse_arguments()
    archive = SpikingformerParameterArchive(arguments.manifest)
    image = prepare_input(np.load(arguments.input, allow_pickle=False), arguments.input_format)
    driver = NeuraEdgeDriver.from_bitfile(
        arguments.bitfile, download=not arguments.no_download)
    model = SpikingformerAcceleratorModel(
        archive, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)
    started = time.perf_counter()
    inference = model.infer(image)
    elapsed = time.perf_counter() - started
    performance = driver.performance_snapshot()
    report = {
        "format": "neuraedge-pynq-spikingformer-inference-v1",
        "completed_utc": datetime.now(timezone.utc).isoformat(),
        "bitfile": str(arguments.bitfile.resolve()),
        "parameter_manifest": str(arguments.manifest.resolve()),
        "parameter_binary_sha256": archive.manifest["binary_sha256"],
        "batch_size": int(image.shape[0]),
        "elapsed_seconds": elapsed,
        "images_per_second": image.shape[0] / elapsed,
        "predicted_class": inference.predicted_class.tolist(),
        "logits_accumulator": inference.logits_accumulator.tolist(),
        "logits": inference.logits.tolist(),
        "performance_counters": asdict(performance),
    }
    arguments.output.parent.mkdir(parents=True, exist_ok=True)
    arguments.output.write_text(json.dumps(report, indent=2) + "\n", encoding="ascii")
    print(json.dumps(report, indent=2))
```

39.3 Line annotations

Line 1: `"""Run complete trained Spikingformer inference on a PYNQ-Z2 overlay."""` This is executable documentation attached to the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import json` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `import time` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `from dataclasses import asdict` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `from datetime import datetime, timezone` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `from pathlib import Path` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `import numpy as np` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `from neuraedge_pynq import NeuraEdgeDriver` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `from spikingformer_accelerator import SpikingformerAcceleratorModel` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `from spikingformer_parameters import SpikingformerParameterArchive` This imports the named Python API so the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `CIFAR10_MEAN = np.asarray((0.4914, 0.4822, 0.4465), dtype=np.float32)` This assigns ‘np.asarray((0.4914, 0.4822, 0.4465), dtype=np.float32)‘ to ‘CIFAR10_MEAN ‘ in host memory. Here CIFAR10_MEAN means the arithmetic average used by normalization. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; float32: the value named ’float32’ in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 20: `CIFAR10_STD = np.asarray((0.2470, 0.2435, 0.2616), dtype=np.float32)` This assigns ‘np.asarray((0.2470, 0.2435, 0.2616), dtype=np.float32)‘ to ‘CIFAR10_STD ‘ in host memory. Here CIFAR10_STD means the value named ’cifar10 std’ in this block. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; float32: the value named ’float32’ in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 23: `def prepare_input(value: np.ndarray, input_format: str) -> np.ndarray:` This starts function prepare_input. Its indented body performs prepare input as part of the file that loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay; it runs only when called.

Line 24: `"""Return normalized NCHW float32 data from raw-u8 or normalized arrays."""` This is executable documentation attached to the function prepare_input, which handles prepare input. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 26: `array = np.asarray(value)` This assigns ‘np.asarray(value)‘ to ‘array ‘ in host memory. Here array means the value named ’array’ in this block. The important operands are asarray: the value named ’asarray’ in this block; value: the value named ’value’ in this block. Context: the function prepare_input, which handles prepare input.

Line 27: `if array.ndim == 3:` This executes the following indented host-code branch only when ‘array.ndim == 3‘ is true. The important operands are array: the value named ’array’ in this block; ndim: the value named ’ndim’ in this block. Context: the function prepare_input, which handles prepare input.

Line 28: `array = array[np.newaxis]` This assigns ‘array[np.newaxis]‘ to ‘array ‘ in host memory. Here array means the value named ’array’ in this block. The important operands are array: the value named ’array’ in this block; newaxis: the value named ’newaxis’ in this block. Context: the function prepare_input, which handles prepare input.

Line 29: `if array.ndim != 4:` This executes the following indented host-code branch only when ‘array.ndim != 4‘ is true. The important operands are array: the value named ’array’ in this block; ndim: the value named ’ndim’ in this block. Context: the function prepare_input, which handles prepare input.

Line 30: `raise ValueError("input array must have three or four dimensions")` This rejects an invalid state by raising ‘ValueError("input array must have three or four dimensions")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 31: `if array.shape[1:] == (3, 32, 32):` This executes the following indented host-code branch only when ‘array.shape[1:] == (3, 32, 32)‘ is true. The important operands are array: the value named ’array’ in this block; shape: the value named ’shape’ in this block. Context: the function prepare_input, which handles prepare input.

Line 32: `nchw = array` This assigns ‘array‘ to ‘nchw ‘ in host memory. Here nchw means the value named ’nchw’ in this block. The important operands are array: the value named ’array’ in this block. Context: the function prepare_input, which handles prepare input.

Line 33: `elif array.shape[1:] == (32, 32, 3):` This executes the following indented host-code branch only when ‘array.shape[1:] == (32, 32, 3)‘ is true. The important operands are array: the value named ’array’ in this block; shape: the value named ’shape’ in this block. Context: the function prepare_input, which handles prepare input.

Line 34: `nchw = array.transpose(0, 3, 1, 2)` This assigns ‘array.transpose(0, 3, 1, 2)‘ to ‘nchw ‘ in host memory. Here nchw means the value named ’nchw’ in this block. The important operands are array: the value named ’array’ in this block; transpose: the value named ’transpose’ in this block. Context: the function prepare_input, which handles prepare input.

Line 35: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function prepare_input, which handles prepare input.

Line 36: `raise ValueError("input array must be NCHW or NHWC CIFAR-10 shape")` This rejects an invalid state by raising ‘ValueError("input array must be NCHW or NHWC CIFAR-10 shape")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 37: `if input_format == "normalized":` This executes the following indented host-code branch only when ‘input_format == "normalized"‘ is true. The important operands are input_format: the value named ’input format’ in this block; normalized: the value named ’normalized’ in this block. Context: the function prepare_input, which handles prepare input.

Line 38: `return nchw.astype(np.float32)` This returns ‘nchw.astype(np.float32)‘ to the caller of the function prepare_input, which handles prepare input, ending this invocation. The important operands are nchw: the value named ’nchw’ in this block; astype: the value named ’astype’ in this block; float32: the value named ’float32’ in this block.

Line 39: `if input_format != "raw-u8":` This executes the following indented host-code branch only when ‘input_format != "raw-u8"‘ is true. The important operands are input_format: the value named ’input format’ in this block; raw: the value named ’raw’ in this block; u8: the value named ’u8’ in this block. Context: the function prepare_input, which handles prepare input.

Line 40: `raise ValueError("input_format must be normalized or raw-u8")` This rejects an invalid state by raising ‘ValueError("input_format must be normalized or raw-u8")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 41: `if not np.issubdtype(nchw.dtype, np.integer) or np.any(nchw < 0) or np.any(nchw > 255):` This executes the following indented host-code branch only when ‘not np.issubdtype(nchw.dtype, np.integer) or np.any(nchw < 0) or np.any(nchw > 255)‘ is true. The important operands are not: the value named ’not’ in this block; issubdtype: the value named ’issubdtype’ in this block; nchw: the value named ’nchw’ in this block. Context: the function prepare_input, which handles prepare input.

Line 42: `raise ValueError("raw-u8 input must contain integer pixels in [0,255]")` This rejects an invalid state by raising ‘ValueError("raw-u8 input must contain integer pixels in [0,255]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 43: `unit = nchw.astype(np.float32) / np.float32(255.0)` This assigns ‘nchw.astype(np.float32) / np.float32(255.0)‘ to ‘unit ‘ in host memory. Here unit means the value named ’unit’ in this block. The important operands are nchw: the value named ’nchw’ in this block; astype: the value named ’astype’ in this block; float32: the value named ’float32’ in this block. Context: the function prepare_input, which handles prepare input.

Line 44: `return (unit - CIFAR10_MEAN.reshape(1, 3, 1, 1)) / CIFAR10_STD.reshape(1, 3, 1, 1)` This returns ‘(unit - CIFAR10_MEAN.reshape(1, 3, 1, 1)) / CIFAR10_STD.reshape(1, 3, 1, 1)‘ to the caller of the function prepare_input, which handles prepare input, ending this invocation. The important operands are unit: the value named ’unit’ in this block; CIFAR10_MEAN: the arithmetic average used by normalization; reshape: the value named ’reshape’ in this block.

Line 47: `def parse_arguments() -> argparse.Namespace:` This starts function parse_arguments. Its indented body performs parse arguments as part of the file that loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay; it runs only when called.

Line 48: `parser = argparse.ArgumentParser()` This assigns ‘argparse.ArgumentParser()‘ to ‘parser ‘ in host memory. Here parser means the value named ’parser’ in this block. The important operands are argparse: the value named ’argparse’ in this block; ArgumentParser: the value named ’argument parser’ in this block. Context: the function parse_arguments, which handles parse arguments.

Line 49: `parser.add_argument("-bitfile", required=True, type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 50: `parser.add_argument("-manifest", required=True, type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 51: `parser.add_argument("-input", required=True, type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 52: `parser.add_argument("-input-format", choices=("normalized", "raw-u8"), default="raw-u8")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 53: `parser.add_argument("-output", required=True, type=Path)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 54: `parser.add_argument("-layer-timeout-seconds", type=float, default=120.0)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 55: `parser.add_argument("-no-download", action="store_true")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 56: `return parser.parse_args()` This returns ‘parser.parse_args()‘ to the caller of the function parse_arguments, which handles parse arguments, ending this invocation. The important operands are parser: the value named ’parser’ in this block; parse_args: the value named ’parse args’ in this block.

Line 59: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 60: `arguments = parse_arguments()` This assigns ‘parse_arguments()‘ to ‘arguments ‘ in host memory. Here arguments means the value named ’arguments’ in this block. The important operands are parse_arguments: the value named ’parse arguments’ in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 61: `archive = SpikingformerParameterArchive(arguments.manifest)` This assigns ‘SpikingformerParameterArchive(arguments.manifest)‘ to ‘archive ‘ in host memory. Here archive means the value named ’archive’ in this block. The important operands are SpikingformerParameterArchive: the value named ’spikingformer parameter archive’ in this block; arguments: the value named ’arguments’ in this block; manifest: the value named ’manifest’ in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 62: `image = prepare_input(np.load(arguments.input, allow_pickle=False), arguments.input_format)` This assigns ‘prepare_input(np.load(arguments.input, allow_pickle=False), arguments.input_format)‘ to ‘image ‘ in host memory. Here image means the value named ’image’ in this block. The important operands are prepare_input: the value named ’prepare input’ in this block; load: the value named ’load’ in this block; arguments: the value named ’arguments’ in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 63: `driver = NeuraEdgeDriver.from_bitfile(` This statement continues through source line 64; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘NeuraEdgeDriver.from_bitfile(arguments.bitfile, download=not arguments.no_download)‘ to ‘driver ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are NeuraEdgeDriver: the value named ’neura edge driver’ in this block; from_bitfile: the value named ’from bitfile’ in this block; arguments: the value named ’arguments’ in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 64: `arguments.bitfile, download=not arguments.no_download)` This is continuation line 2 of the 2-line logical statement that begins at source line 63. Its exact fragment is ‘arguments.bitfile, download=not arguments.no_download)‘. The important operands are arguments: the value named ’arguments’ in this block; bitfile: the value named ’bitfile’ in this block; download: the value named ’download’ in this block. Read together, the complete operation is: This assigns ‘NeuraEdgeDriver.from_bitfile(arguments.bitfile, download=not arguments.no_download)‘ to ‘driver ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are NeuraEdgeDriver: the value named ’neura edge driver’ in this block; from_bitfile: the value named ’from bitfile’ in this block; arguments: the value named ’arguments’ in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 65: `model = SpikingformerAcceleratorModel(` This statement continues through source line 66; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘SpikingformerAcceleratorModel(archive, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are SpikingformerAcceleratorModel: the value named ’spikingformer accelerator model’ in this block; archive: the value named ’archive’ in this block; driver: the value

named 'driver' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 66: `archive, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 65. Its exact fragment is 'archive, driver, layer_timeout_seconds=argur The important operands are archive: the value named 'archive' in this block; driver: the value named 'driver' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'SpikingformerAcceleratorModel(archive, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are SpikingformerAcceleratorModel: the value named 'spikingformer accelerator model' in this block; archive: the value named 'archive' in this block; driver: the value named 'driver' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 67: `started = time.perf_counter()` This assigns 'time.perf_counter()' to 'started ' in host memory. Here started means the value named 'started' in this block. The important operands are time: the value named 'time' in this block; perf_counter: a hardware telemetry register that accumulates measured events. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 68: `inference = model.infer(image)` This assigns 'model.infer(image)' to 'inference ' in host memory. Here inference means the value named 'inference' in this block. The important operands are model: the value named 'model' in this block; infer: the value named 'infer' in this block; image: the value named 'image' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 69: `elapsed = time.perf_counter() - started` This assigns 'time.perf_counter() - started' to 'elapsed ' in host memory. Here elapsed means the value named 'elapsed' in this block. The important operands are time: the value named 'time' in this block; perf_counter: a hardware telemetry register that accumulates measured events; started: the value named 'started' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 70: `performance = driver.performance_snapshot()` This assigns 'driver.performance_snapshot()' to 'performance ' in host memory. Here performance means the value named 'performance' in this block. The important operands are driver: the value named 'driver' in this block; performance_snapshot: the value named 'performance snapshot' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 71: `report = {` This statement continues through source line 84; the explanation uses the complete expression rather than only the first visual fragment. This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve()), "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[("elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.tolist(), "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 72: `"format": "neuraedge-pynq-spikingformer-inference-v1",` This is continuation line 2 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"format": "neuraedge-pynq-spikingformer-inference-v1",'. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve()), "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.t "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this

block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 73: `"completed_utc": datetime.now(timezone.utc).isoformat(),` This is continuation line 3 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"completed_utc": datetime.now(timezone.utc).isoformat(),'. The important operands are completed_utc: the value named 'completed utc' in this block; datetime: the value named 'datetime' in this block; now: the value named 'now' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve()), "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.t "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 74: `"bitfile": str(arguments.bitfile.resolve()),` This is continuation line 4 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"bitfile": str(arguments.bitfile.resolve()),'. The important operands are bitfile: the value named 'bitfile' in this block; str: the value named 'str' in this block; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve()), "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.t "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 75: `"parameter_manifest": str(arguments.manifest.resolve()),` This is continuation line 5 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"parameter_manifest": str(arguments.manifest.resolve()),'. The important operands are parameter_manifest: the value named 'parameter manifest' in this block; str: the value named 'str' in this block; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve()), "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.t "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 76: `"parameter_binary_sha256": archive.manifest["binary_sha256"],` This is continuation line 6 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"parameter_binary_sha256": archive.manifest["binary_s The important operands are parameter_binary_sha256: the value named 'parameter binary sha256' in this block; archive: the value named 'archive' in this block; manifest: the value named 'manifest' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.tolist(), "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this

block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 77: `"batch_size": int(image.shape[0]),` This is continuation line 7 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"batch_size": int(image.shape[0]),'. The important operands are batch_size: the value named 'batch size' in this block; image: the value named 'image' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve()), "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.tolist(), "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 78: `"elapsed_seconds": elapsed,` This is continuation line 8 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"elapsed_seconds": elapsed,'. The important operands are elapsed_seconds: the value named 'elapsed seconds' in this block; elapsed: the value named 'elapsed' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve
"parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.tolist(), "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 79: `"images_per_second": image.shape[0] / elapsed,` This is continuation line 9 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"images_per_second": image.shape[0] / elapsed,'. The important operands are images_per_second: the value named 'images per second' in this block; image: the value named 'image' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve()), "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.t
"logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 80: `"predicted_class": inference.predicted_class.tolist(),` This is continuation line 10 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"predicted_class": inference.predicted_class.tolist(),'. The important operands are predicted_class: the value named 'predicted class' in this block; inference: the value named 'inference' in this block; tolist: the value named 'tolist' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve()), "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.t
"logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 81: `"logits_accumulator": inference.logits_accumulator.tolist(),` This is continuation line 11 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"logits_accumulator": inference.logits_accumulator.tolis The important operands are logits_accumulator: a widened running sum that prevents intermediate overflow; inference: the value named 'inference' in this block; tolist: the value named 'tolist' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.tolist(), "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 82: `"logits": inference.logits.tolist(),` This is continuation line 12 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"logits": inference.logits.tolist(),'. The important operands are logits: the ten unnormalized classifier scores used for argmax; inference: the value named 'inference' in this block; tolist: the value named 'tolist' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve()), "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.tolist(), "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 83: `"performance_counters": asdict(performance),` This is continuation line 13 of the 14-line logical statement that begins at source line 71. Its exact fragment is '"performance_counters": asdict(performance),'. The important operands are performance_counters: a hardware telemetry register that accumulates measured events; asdict: the value named 'asdict' in this block; performance: the value named 'performance' in this block. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.tolist(), "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 84: `}` This is continuation line 14 of the 14-line logical statement that begins at source line 71. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "format": "neuraedge-pynq-spikingformer-inference-v1", "completed_utc": datetime.now(timezone.utc).isoformat(), "bitfile": str(arguments.bitfile.resolve()), "parameter_manifest": str(arguments.manifest.resolve "parameter_binary_sha256": archive.manifest["binary_sha256"], "batch_size": int(image.shape[0]), "elapsed_seconds": elapsed, "images_per_second": image.shape[0] / elapsed, "predicted_class": inference.predicted_class.tolist(), "logits_accumulator": inference.logits_accumulator.tolist(), "logits": inference.logits.tolist(), "performance_counters": asdict(performance), }' to 'report ' in host memory. Here report means the value named 'report' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; pynq: the value named 'pynq' in this block. Context: the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 85: `arguments.output.parent.mkdir(parents=True, exist_ok=True)` This calls 'arguments.output.parent.mkdir' with 'parents=True, exist_ok=True' inside the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay. The call advances the host-side job that loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 86: `arguments.output.write_text(json.dumps(report, indent=2) + "\n", encoding="ascii")` This calls ‘arguments.output.write_text‘ with ‘json.dumps(report, indent=2) + "\n", encoding="ascii"‘ inside the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay. The call advances the host-side job that loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

Line 87: `print(json.dumps(report, indent=2))` This calls ‘print‘ with ‘json.dumps(report, indent=2)‘ inside the file whose role is to loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay. The call advances the host-side job that loads one CIFAR-10 image and executes the complete exported Spikingformer graph on a PYNQ overlay.

40 software/spikingformer_accelerator.py

Role in the complete system: maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

40.1 Beginner architecture map

Dataflow responsibility The exported model graph calls compiler primitives in checkpoint order: tokenizer convolutions and LIF, four Q/K/V attention and MLP blocks, residuals, reduction, and classifier. Tensor shapes and quantization records, rather than a hard-coded class result, determine every hardware program.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: SpikingformerProjectionExecutor, SpikingformerInferenceResult, SpikingformerAcceleratorModel. Functions: vector_binary_on_accelerator, power_of_two_requantize_on_accelerator, global_spatiotemporal_sum_binary_max_pool2d_on_accelerator, lif_sequence_reference, lif_sequence_with_trace_reference, lif_sequence_on_accelerator, lif_sequence_with_trace_on_accelerator, requantize_projection_reference, requantize_projection_on_accelerator, _encoded_dense_weights, matrix, conv2d, ___init___, _lif, _project_tokens, _dynamic_binary_gemm, _dynamic_u8_gemm, _attention, _mlp, _tokenizer, infer. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

40.2 Complete source

```
"""Executable Spikingformer projection path for the NeuraEdge command ABI."""

from __future__ import annotations

import math
from dataclasses import dataclass
from typing import Sequence

import numpy as np

from neuraedge_compiler import (
    ProgramExecutor,
    compile_spike_conv2d,
    compile_spike_gemm,
    compile_u8_conv2d,
    compile_u8_gemm,
)
from neuraedge_golden import VectorOperation, round_shift_signed, saturate_signed
from neuraedge_protocol import (
    ProgramRecord,
    clear_status_command,
    pack_vector_beats,
    lif_command,
    pack_lif_beats,
    unpack_lif_result,
    unpack_lif_trace,
    unpack_vector_result,
    vector_command,
)
from spikingformer_parameters import ProjectionParameters, SpikingformerParameterArchive


VECTOR_LANES = 8
LIF_LANES = 32


def vector_binary_on_accelerator(
    driver: ProgramExecutor,
    operation: VectorOperation,
    operand_a: np.ndarray,
    operand_b: np.ndarray,
    *,
    timeout_seconds: float = 30.0,
) -> np.ndarray:
```

```
    """Apply a binary vector-ALU operation to two equally shaped tensors."""

    if operation not in {VectorOperation.ADD_SAT, VectorOperation.MAXIMUM}:
        raise ValueError("this helper supports saturating add and maximum")
    a = np.asarray(operand_a, dtype=np.int64)
    b = np.asarray(operand_b, dtype=np.int64)
    if a.shape != b.shape:
        raise ValueError("vector operands must have equal shapes")
    if a.size and (a.min() < -(1 << 31) or a.max() > (1 << 31) - 1
                   or b.min() < -(1 << 31) or b.max() > (1 << 31) - 1):
        raise OverflowError("vector operands do not fit signed 32-bit lanes")
    flat_a = a.reshape(-1)
    flat_b = b.reshape(-1)
    records: list[ProgramRecord] = []
    counts: list[int] = []
    command = vector_command(int(operation))
    for start in range(0, len(flat_a), VECTOR_LANES):
        count = min(VECTOR_LANES, len(flat_a) - start)
        lanes_a = flat_a[start:start + count].tolist()
        lanes_b = flat_b[start:start + count].tolist()
        lanes_a.extend([0] * (VECTOR_LANES - count))
        lanes_b.extend([0] * (VECTOR_LANES - count))
        records.append(ProgramRecord(
            command, pack_vector_beats(lanes_a, lanes_b), 4))
        counts.append(count)
    if not records:
        return np.empty(a.shape, dtype=np.int64)
    words = driver.execute_program(records, timeout_seconds=timeout_seconds)
    if len(words) != 4 * len(records):
        raise RuntimeError(f"vector program returned {len(words)} beats; expected {4 * len(records)}")
    result = np.empty(len(flat_a), dtype=np.int64)
    cursor = 0
    for record_index, count in enumerate(counts):
        lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])
        result[cursor:cursor + count] = lanes[:count]
        cursor += count
    return result.reshape(a.shape)


def power_of_two_requantize_on_accelerator(
    driver: ProgramExecutor,
    values: np.ndarray,
    *,
    division_shift: int,
    clamp_minimum: int = -(1 << 31),
    clamp_maximum: int = (1 << 31) - 1,
    timeout_seconds: float = 30.0,
) -> np.ndarray:
    """Round ``values / 2**division_shift`` in the flexible vector datapath."""

    source = np.asarray(values, dtype=np.int64)
    if not 0 <= division_shift <= 16:
        raise ValueError("division_shift must be in [0,16]")
    if source.size and (
        source.min() < -(1 << 31) or source.max() > (1 << 31) - 1
    ):
        raise OverflowError("requantizer input must fit signed 32-bit lanes")
    flat = source.reshape(-1)
    records: list[ProgramRecord] = []
    counts: list[int] = []
    command = vector_command(
        int(VectorOperation.FLEX_REQUANTIZE),
        multiplier_q15=1 << 14,
        right_shift=division_shift - 1,
    )
    for start in range(0, flat.size, VECTOR_LANES):
        stop = min(start + VECTOR_LANES, flat.size)
        count = stop - start
        lanes = flat[start:stop].tolist() + [0] * (VECTOR_LANES - count)
        records.append(ProgramRecord(
            command,
            pack_vector_beats(
                lanes,
                [0] * VECTOR_LANES,
                clamp_minimum=clamp_minimum,
                clamp_maximum=clamp_maximum,
            ),
            4,
        ))
        counts.append(count)
    if not records:
        return np.empty(source.shape, dtype=np.int64)
    words = driver.execute_program(records, timeout_seconds=timeout_seconds)
    if len(words) != 4 * len(records):
        raise RuntimeError("power-of-two requantizer returned an invalid stream length")
    result = np.empty(flat.size, dtype=np.int64)
    cursor = 0
```

```
    for record_index, count in enumerate(counts):
        lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])
        result[cursor:cursor + count] = lanes[:count]
        cursor += count
    return result.reshape(source.shape)


def global_spatiotemporal_sum_on_accelerator(
    driver: ProgramExecutor,
    values: np.ndarray,
    *,
    timeout_seconds: float = 30.0,
) -> np.ndarray:
    """Reduce a [T,B,C,H,W] tensor to [B,C] through pairwise vector adds."""

    source = np.asarray(values, dtype=np.int64)
    if source.ndim != 5:
        raise ValueError("global sum input must have shape [T,B,C,H,W]")
    timesteps, batch, channels, height, width = source.shape
    reduction = source.transpose(0, 3, 4, 1, 2).reshape(
        timesteps * height * width, batch, channels)
    if int(np.abs(reduction).max(initial=0)) * reduction.shape[0] > (1 << 31) - 1:
        raise OverflowError("global sum can exceed the signed-32 vector accumulator")
    candidates = [reduction[index] for index in range(reduction.shape[0])]
    while len(candidates) > 1:
        reduced: list[np.ndarray] = []
        for index in range(0, len(candidates) - 1, 2):
            reduced.append(vector_binary_on_accelerator(
                driver,
                VectorOperation.ADD_SAT,
                candidates[index],
                candidates[index + 1],
                timeout_seconds=timeout_seconds,
            ))
        if len(candidates) & 1:
            reduced.append(candidates[-1])
        candidates = reduced
    return candidates[0]


def binary_max_pool2d_on_accelerator(
    driver: ProgramExecutor,
    values: np.ndarray,
    *,
    kernel_size: int = 3,
    stride: int = 2,
    padding: int = 1,
    timeout_seconds: float = 30.0,
) -> np.ndarray:
    """Max-pool NCHW binary spikes with pairwise vector-maximum trees."""

    source = np.asarray(values)
    if source.ndim != 4 or np.any((source != 0) & (source != 1)):
        raise ValueError("binary max-pool input must be an NCHW tensor containing only 0/1")
    if kernel_size <= 0 or stride <= 0 or padding < 0:
        raise ValueError("invalid max-pool geometry")
    padded = np.pad(
        source.astype(np.int64),
        ((0, 0), (0, 0), (padding, padding), (padding, padding)),
        mode="constant",
    )
    output_height = (source.shape[2] + 2 * padding - kernel_size) // stride + 1
    output_width = (source.shape[3] + 2 * padding - kernel_size) // stride + 1
    candidates = [
        padded[:, :, kernel_y:kernel_y + output_height * stride:stride,
               kernel_x:kernel_x + output_width * stride:stride]
        for kernel_y in range(kernel_size)
        for kernel_x in range(kernel_size)
    ]
    while len(candidates) > 1:
        reduced: list[np.ndarray] = []
        for index in range(0, len(candidates) - 1, 2):
            reduced.append(vector_binary_on_accelerator(
                driver,
                VectorOperation.MAXIMUM,
                candidates[index],
                candidates[index + 1],
                timeout_seconds=timeout_seconds,
            ))
        if len(candidates) & 1:
            reduced.append(candidates[-1])
        candidates = reduced
    return candidates[0].astype(np.uint8)


def lif_sequence_reference(
    values_q10: np.ndarray,
```

```
    *,
    threshold_q10: int = 1 << 10,
    leak_shift: int = 1,
    reset_q10: int = 0,
) -> np.ndarray:
    """Bit-exact fixed-point LIF sequence for an arbitrary tensor shape."""

    values = np.asarray(values_q10, dtype=np.int64)
    if values.ndim < 2 or values.shape[0] == 0:
        raise ValueError("LIF input must have shape [timesteps, ...]")
    if not 0 <= leak_shift <= 31:
        raise ValueError("leak_shift must be in [0,31]")
    membrane = np.full(values.shape[1:], reset_q10, dtype=np.int64)
    spikes = np.zeros(values.shape, dtype=np.uint8)
    for timestep in range(values.shape[0]):
        current = np.clip(values[timestep] >> 1, -(1 << 15), (1 << 15) - 1)
        leaked = membrane - (membrane >> leak_shift)
        membrane = saturate_signed(leaked + current, 24)
        fired = membrane >= threshold_q10
        spikes[timestep] = fired
        membrane = np.where(fired, reset_q10, membrane)
    return spikes


def lif_sequence_with_trace_reference(
    values_q10: np.ndarray,
    *,
    threshold_q10: int = 1 << 10,
    leak_shift: int = 1,
    reset_q10: int = 0,
) -> tuple[np.ndarray, np.ndarray]:
    """Return spikes and the signed-24 membrane immediately before hard reset."""

    values = np.asarray(values_q10, dtype=np.int64)
    if values.ndim < 2 or values.shape[0] == 0:
        raise ValueError("LIF input must have shape [timesteps, ...]")
    if not 0 <= leak_shift <= 31:
        raise ValueError("leak_shift must be in [0,31]")
    membrane = np.full(values.shape[1:], reset_q10, dtype=np.int64)
    spikes = np.zeros(values.shape, dtype=np.uint8)
    membrane_trace = np.zeros(values.shape, dtype=np.int64)
    for timestep in range(values.shape[0]):
        current = np.clip(values[timestep] >> 1, -(1 << 15), (1 << 15) - 1)
        leaked = membrane - (membrane >> leak_shift)
        membrane = saturate_signed(leaked + current, 24)
        membrane_trace[timestep] = membrane
        fired = membrane >= threshold_q10
        spikes[timestep] = fired
        membrane = np.where(fired, reset_q10, membrane)
    return spikes, membrane_trace


def lif_sequence_on_accelerator(
    driver: ProgramExecutor,
    values_q10: np.ndarray,
    *,
    threshold_q10: int = 1 << 10,
    leak_shift: int = 1,
    reset_q10: int = 0,
    timeout_seconds: float = 30.0,
) -> np.ndarray:
    """Schedule every logical LIF state through the physical 32-lane bank."""

    values = np.asarray(values_q10, dtype=np.int64)
    if values.ndim < 2 or values.shape[0] == 0:
        raise ValueError("LIF input must have shape [timesteps, ...]")
    if values.size and (values.min() < -(1 << 63) or values.max() > (1 << 63) - 1):
        raise OverflowError("LIF input exceeds signed 64-bit preparation arithmetic")
    timesteps = values.shape[0]
    flat = values.reshape(timesteps, -1)
    records: list[ProgramRecord] = []
    bindings: list[tuple[int, int, int]] = []
    for element_start in range(0, flat.shape[1], LIF_LANES):
        element_count = min(LIF_LANES, flat.shape[1] - element_start)
        for timestep in range(timesteps):
            currents = np.clip(
                flat[timestep, element_start:element_start + element_count] >> 1,
                -(1 << 15),
                (1 << 15) - 1,
            ).astype(np.int64).tolist()
            currents.extend([0] * (LIF_LANES - element_count))
            records.append(ProgramRecord(
                lif_command(
                    leak_shift,
                    threshold_q10,
                    reset_q10,
                    clear_state=timestep == 0,
```

```
                ),
                pack_lif_beats(currents),
                1,
            ))
            bindings.append((timestep, element_start, element_count))
    words = driver.execute_program(records, timeout_seconds=timeout_seconds)
    if len(words) != len(records):
        raise RuntimeError(f"LIF program returned {len(words)} beats; expected {len(records)}")
    output = np.zeros(flat.shape, dtype=np.uint8)
    for word, (timestep, element_start, element_count) in zip(words, bindings):
        spike_word, _ = unpack_lif_result(word)
        for lane in range(element_count):
            output[timestep, element_start + lane] = (spike_word >> lane) & 1
    return output.reshape(values.shape)


def lif_sequence_with_trace_on_accelerator(
    driver: ProgramExecutor,
    values_q10: np.ndarray,
    *,
    threshold_q10: int = 1 << 10,
    leak_shift: int = 1,
    reset_q10: int = 0,
    timeout_seconds: float = 30.0,
) -> tuple[np.ndarray, np.ndarray]:
    """Run LIF in fabric and retain the pre-reset membrane required by BPTT."""

    values = np.asarray(values_q10, dtype=np.int64)
    if values.ndim < 2 or values.shape[0] == 0:
        raise ValueError("LIF input must have shape [timesteps, ...]")
    if values.size and (values.min() < -(1 << 63) or values.max() > (1 << 63) - 1):
        raise OverflowError("LIF input exceeds signed 64-bit preparation arithmetic")
    timesteps = values.shape[0]
    flat = values.reshape(timesteps, -1)
    records: list[ProgramRecord] = []
    bindings: list[tuple[int, int, int]] = []
    for element_start in range(0, flat.shape[1], LIF_LANES):
        element_count = min(LIF_LANES, flat.shape[1] - element_start)
        for timestep in range(timesteps):
            currents = np.clip(
                flat[timestep, element_start:element_start + element_count] >> 1,
                -(1 << 15),
                (1 << 15) - 1,
            ).astype(np.int64).tolist()
            currents.extend([0] * (LIF_LANES - element_count))
            records.append(ProgramRecord(
                lif_command(
                    leak_shift,
                    threshold_q10,
                    reset_q10,
                    clear_state=timestep == 0,
                    trace_state=True,
                ),
                pack_lif_beats(currents),
                13,
            ))
            bindings.append((timestep, element_start, element_count))
    words = driver.execute_program(records, timeout_seconds=timeout_seconds)
    expected_words = 13 * len(records)
    if len(words) != expected_words:
        raise RuntimeError(
            f"traced LIF program returned {len(words)} beats; expected {expected_words}")
    spikes = np.zeros(flat.shape, dtype=np.uint8)
    membrane_trace = np.zeros(flat.shape, dtype=np.int64)
    for record_index, (timestep, element_start, element_count) in enumerate(bindings):
        start = record_index * 13
        spike_word, _, membranes = unpack_lif_trace(words[start:start + 13])
        for lane in range(element_count):
            spikes[timestep, element_start + lane] = (spike_word >> lane) & 1
            membrane_trace[timestep, element_start + lane] = membranes[lane]
    return spikes.reshape(values.shape), membrane_trace.reshape(values.shape)


def requantize_projection_reference(
    accumulators: np.ndarray,
    parameters: ProjectionParameters,
) -> np.ndarray:
    """Apply the exported per-channel Q15 fields exactly like vector operation 9."""

    values = np.asarray(accumulators, dtype=np.int64)
    if values.ndim != 2 or values.shape[1] != parameters.weight_k_by_n.shape[1]:
        raise ValueError("accumulators must have shape [rows, projection output channels]")
    result = np.empty_like(values)
    for channel in range(values.shape[1]):
        effective_shift = 15 + int(parameters.binary_point_shift[channel])
        product = values[:, channel] * int(parameters.multiplier_q15[channel])
        requantized = round_shift_signed(product, effective_shift)
```

```
        requantized += int(parameters.bias_q10[channel])
        result[:, channel] = saturate_signed(requantized, 24)
    return result


def requantize_projection_on_accelerator(
    driver: ProgramExecutor,
    accumulators: np.ndarray,
    parameters: ProjectionParameters,
    *,
    timeout_seconds: float = 30.0,
) -> np.ndarray:
    """Execute per-channel folded BN scaling through the eight-lane vector RTL."""

    values = np.asarray(accumulators, dtype=np.int64)
    if values.ndim != 2 or values.shape[1] != parameters.weight_k_by_n.shape[1]:
        raise ValueError("accumulators must have shape [rows, projection output channels]")
    if values.size and (values.min() < -(1 << 31) or values.max() > (1 << 31) - 1):
        raise OverflowError("projection accumulators do not fit the vector ALU's signed 32-bit lanes")

    records: list[ProgramRecord] = []
    bindings: list[tuple[int, int, int]] = []
    zero = [0] * VECTOR_LANES
    rows, channels = values.shape
    for channel in range(channels):
        command = vector_command(
            int(VectorOperation.FLEX_REQUANTIZE),
            multiplier_q15=int(parameters.multiplier_q15[channel]),
            right_shift=int(parameters.binary_point_shift[channel]),
            output_zero_point=int(parameters.bias_q10[channel]),
        )
        for row_start in range(0, rows, VECTOR_LANES):
            row_count = min(VECTOR_LANES, rows - row_start)
            lanes = values[row_start:row_start + row_count, channel].tolist()
            lanes.extend([0] * (VECTOR_LANES - row_count))
            records.append(ProgramRecord(
                command,
                pack_vector_beats(
                    lanes,
                    zero,
                    clamp_minimum=-(1 << 23),
                    clamp_maximum=(1 << 23) - 1,
                ),
                4,
            ))
            bindings.append((row_start, row_count, channel))

    words = driver.execute_program(records, timeout_seconds=timeout_seconds)
    if len(words) != 4 * len(records):
        raise RuntimeError(
            f"requantizer returned {len(words)} beats; expected {4 * len(records)}")
    result = np.empty((rows, channels), dtype=np.int64)
    for record_index, (row_start, row_count, channel) in enumerate(bindings):
        lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])
        result[row_start:row_start + row_count, channel] = lanes[:row_count]
    return result


@dataclass
class SpikingformerProjectionExecutor:
    """Run exported Conv-BN projections through tensor and vector hardware."""

    archive: SpikingformerParameterArchive
    driver: ProgramExecutor

    @staticmethod
    def _encoded_dense_weights(weight: np.ndarray) -> np.ndarray:
        return (weight.astype(np.int16) + 128).astype(np.uint8)

    def matrix(
        self,
        name: str,
        activations: np.ndarray,
        *,
        activation_zero_point: int = 0,
        timeout_seconds: float = 30.0,
    ) -> np.ndarray:
        """Execute one exported Conv1d-style projection as a two-dimensional matrix."""

        parameters = self.archive.projection(name)
        values = np.asarray(activations)
        if values.ndim != 2 or values.shape[1] != parameters.weight_k_by_n.shape[0]:
            raise ValueError("matrix activations do not match the projection reduction dimension")
        if parameters.compute_opcode == "COMPUTE_BINARY":
            compiled = compile_spike_gemm(values, parameters.weight_k_by_n)
        elif parameters.compute_opcode == "COMPUTE_DENSE_OUTER":
            compiled = compile_u8_gemm(
```

```
                values,
                self._encoded_dense_weights(parameters.weight_k_by_n),
                activation_zero_point=activation_zero_point,
                weight_zero_point=128,
            )
        else:
            raise ValueError(f"unsupported projection opcode {parameters.compute_opcode!r}")
        accumulators = compiled.execute_batched(
            self.driver, timeout_seconds=timeout_seconds)
        return requantize_projection_on_accelerator(
            self.driver, accumulators, parameters, timeout_seconds=timeout_seconds)

    def conv2d(
        self,
        name: str,
        activations: np.ndarray,
        *,
        activation_zero_point: int = 0,
        stride: int | Sequence[int] = 1,
        padding: int | Sequence[int] = 0,
        dilation: int | Sequence[int] = 1,
        groups: int = 1,
        timeout_seconds: float = 30.0,
    ) -> np.ndarray:
        """Execute one exported Conv2d-BN pair and return signed Q10 NCHW data."""

        parameters = self.archive.projection(name)
        if len(parameters.source_weight_shape) != 4:
            raise ValueError(f"projection {name} is not Conv2d")
        weight = parameters.weight_k_by_n.T.reshape(parameters.source_weight_shape)
        values = np.asarray(activations)
        if parameters.compute_opcode == "COMPUTE_BINARY":
            compiled = compile_spike_conv2d(
                values, weight, stride=stride, padding=padding,
                dilation=dilation, groups=groups)
        elif parameters.compute_opcode == "COMPUTE_DENSE_OUTER":
            compiled = compile_u8_conv2d(
                values,
                self._encoded_dense_weights(weight),
                activation_zero_point=activation_zero_point,
                weight_zero_point=128,
                stride=stride,
                padding=padding,
                dilation=dilation,
                groups=groups,
            )
        else:
            raise ValueError(f"unsupported projection opcode {parameters.compute_opcode!r}")
        accumulator_nchw = compiled.execute_batched(
            self.driver, timeout_seconds=timeout_seconds)
        batch, channels, height, width = accumulator_nchw.shape
        matrix = accumulator_nchw.transpose(0, 2, 3, 1).reshape(-1, channels)
        quantized = requantize_projection_on_accelerator(
            self.driver, matrix, parameters, timeout_seconds=timeout_seconds)
        return quantized.reshape(batch, height, width, channels).transpose(0, 3, 1, 2)


@dataclass(frozen=True)
class SpikingformerInferenceResult:
    """Classifier result in both exact accumulator and represented float units."""

    logits_accumulator: np.ndarray
    logits: np.ndarray
    predicted_class: np.ndarray


class SpikingformerAcceleratorModel:
    """Full trained Spikingformer-4-384 inference graph on NeuraEdge primitives."""

    def __init__(
        self,
        archive: SpikingformerParameterArchive,
        driver: ProgramExecutor,
        *,
        layer_timeout_seconds: float = 120.0,
    ) -> None:
        if layer_timeout_seconds <= 0:
            raise ValueError("layer_timeout_seconds must be positive")
        self.archive = archive
        self.driver = driver
        self.projections = SpikingformerProjectionExecutor(archive, driver)
        self.layer_timeout_seconds = layer_timeout_seconds
        fixed = archive.manifest.get("fixed_point")
        if not isinstance(fixed, dict):
            raise ValueError("parameter archive has no fixed-point contract")
        self.fractional_bits = int(fixed["activation_fractional_bits"])
        self.fixed_scale = 1 << self.fractional_bits
```

```
        self.image_scale = float(fixed["image_input_scale"])
        self.image_zero_point = int(fixed["image_input_zero_point"])
        self.head_activation_scale = float(fixed["head_activation_scale"])
        required = {
            *(f"patch_embed.block{index}" for index in range(5)),
            *(f"block.{block}.attn.{name}"
              for block in range(4) for name in ("q", "k", "v", "proj")),
            *(f"block.{block}.mlp.fc{layer}"
              for block in range(4) for layer in (1, 2)),
        }
        missing = required - set(archive.projection_names)
        if missing:
            raise ValueError(f"parameter archive is missing projections: {sorted(missing)}")

    def _lif(self, value: np.ndarray, *, threshold: float = 1.0) -> np.ndarray:
        return lif_sequence_on_accelerator(
            self.driver,
            value,
            threshold_q10=int(round(threshold * self.fixed_scale)),
            leak_shift=1,
            reset_q10=0,
            timeout_seconds=self.layer_timeout_seconds,
        )

    def _project_tokens(self, name: str, spikes: np.ndarray) -> np.ndarray:
        if spikes.ndim != 4:
            raise ValueError("token projection input must have shape [T,B,N,C]")
        timesteps, batch, tokens, channels = spikes.shape
        rows = spikes.reshape(timesteps * batch * tokens, channels)
        result = self.projections.matrix(
            name, rows, timeout_seconds=self.layer_timeout_seconds)
        return result.reshape(timesteps, batch, tokens, -1)

    def _dynamic_binary_gemm(self, left: np.ndarray, right: np.ndarray) -> np.ndarray:
        program = compile_spike_gemm(left, right)
        return program.execute_batched(
            self.driver, timeout_seconds=self.layer_timeout_seconds)

    def _dynamic_u8_gemm(self, left: np.ndarray, right: np.ndarray) -> np.ndarray:
        program = compile_u8_gemm(left, right)
        return program.execute_batched(
            self.driver, timeout_seconds=self.layer_timeout_seconds)

    def _attention(self, block_index: int, value: np.ndarray) -> np.ndarray:
        timesteps, batch, channels, height, width = value.shape
        tokens = height * width
        projection_spikes = self._lif(value)
        token_spikes = projection_spikes.reshape(
            timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)
        prefix = f"block.{block_index}.attn"
        query = self._lif(self._project_tokens(f"{prefix}.q", token_spikes))
        key = self._lif(self._project_tokens(f"{prefix}.k", token_spikes))
        projected_value = self._lif(self._project_tokens(f"{prefix}.v", token_spikes))

        heads = 12
        if channels % heads:
            raise ValueError("attention channels must divide into 12 heads")
        head_dim = channels // heads
        query = query.reshape(timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)
        key = key.reshape(timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)
        projected_value = projected_value.reshape(
            timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)
        attention_value = np.empty(
            (timesteps, batch, heads, tokens, head_dim), dtype=np.int64)
        for timestep in range(timesteps):
            for batch_index in range(batch):
                for head in range(heads):
                    scores = self._dynamic_binary_gemm(
                        query[timestep, batch_index, head],
                        key[timestep, batch_index, head].T,
                    )
                    if scores.min() < 0 or scores.max() > 255:
                        raise OverflowError("binary QK score does not fit the dynamic-u8 AV path")
                    attention_value[timestep, batch_index, head] = self._dynamic_u8_gemm(
                        scores.astype(np.uint8),
                        projected_value[timestep, batch_index, head].astype(np.uint8),
                    )
        attention_q10 = attention_value * (self.fixed_scale // 8)
        merged = attention_q10.transpose(0, 1, 2, 4, 3).reshape(
            timesteps, batch, channels, tokens)
        attention_spikes = self._lif(merged, threshold=0.5)
        output_rows = attention_spikes.transpose(0, 1, 3, 2).reshape(
            timesteps * batch * tokens, channels)
        output = self.projections.matrix(
            f"{prefix}.proj", output_rows,
            timeout_seconds=self.layer_timeout_seconds)
        return output.reshape(timesteps, batch, tokens, channels).transpose(
```

```
            0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)

    def _mlp(self, block_index: int, value: np.ndarray) -> np.ndarray:
        timesteps, batch, channels, height, width = value.shape
        tokens = height * width
        first_spikes = self._lif(value)
        first_rows = first_spikes.reshape(
            timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(
                timesteps * batch * tokens, channels)
        hidden = self.projections.matrix(
            f"block.{block_index}.mlp.fc1", first_rows,
            timeout_seconds=self.layer_timeout_seconds)
        hidden_channels = hidden.shape[1]
        hidden = hidden.reshape(timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)
        hidden_spikes = self._lif(hidden)
        second_rows = hidden_spikes.transpose(0, 1, 3, 2).reshape(
            timesteps * batch * tokens, hidden_channels)
        output = self.projections.matrix(
            f"block.{block_index}.mlp.fc2", second_rows,
            timeout_seconds=self.layer_timeout_seconds)
        return output.reshape(timesteps, batch, tokens, channels).transpose(
            0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)

    def _tokenizer(self, image_u8: np.ndarray) -> np.ndarray:
        batch = image_u8.shape[0]
        first = self.projections.conv2d(
            "patch_embed.block0",
            image_u8,
            activation_zero_point=self.image_zero_point,
            padding=1,
            timeout_seconds=self.layer_timeout_seconds,
        )
        value = np.repeat(first[np.newaxis], 4, axis=0)
        for index in (1, 2):
            spikes = self._lif(value)
            flat = spikes.reshape(-1, *spikes.shape[2:])
            projected = self.projections.conv2d(
                f"patch_embed.block{index}",
                flat,
                padding=1,
                timeout_seconds=self.layer_timeout_seconds,
            )
            value = projected.reshape(4, batch, *projected.shape[1:])
        for index in (3, 4):
            spikes = self._lif(value)
            flat = spikes.reshape(-1, *spikes.shape[2:])
            pooled = binary_max_pool2d_on_accelerator(
                self.driver,
                flat,
                timeout_seconds=self.layer_timeout_seconds,
            )
            projected = self.projections.conv2d(
                f"patch_embed.block{index}",
                pooled,
                padding=1,
                timeout_seconds=self.layer_timeout_seconds,
            )
            value = projected.reshape(4, batch, *projected.shape[1:])
        return value

    def infer(self, normalized_image: np.ndarray) -> SpikingformerInferenceResult:
        """Run normalized CIFAR-10 NCHW images through the complete trained graph."""

        image = np.asarray(normalized_image, dtype=np.float32)
        if image.ndim != 4 or image.shape[1:] != (3, 32, 32) or image.shape[0] == 0:
            raise ValueError("input must have shape [batch,3,32,32]")
        image_u8 = np.rint(image / self.image_scale + self.image_zero_point)
        image_u8 = np.clip(image_u8, 0, 255).astype(np.uint8)
        self.driver.execute_program((ProgramRecord(clear_status_command()),),
                                    timeout_seconds=self.layer_timeout_seconds)
        value = self._tokenizer(image_u8)
        for block_index in range(4):
            attention = self._attention(block_index, value)
            value = vector_binary_on_accelerator(
                self.driver, VectorOperation.ADD_SAT, value, attention,
                timeout_seconds=self.layer_timeout_seconds)
            mlp = self._mlp(block_index, value)
            value = vector_binary_on_accelerator(
                self.driver, VectorOperation.ADD_SAT, value, mlp,
                timeout_seconds=self.layer_timeout_seconds)

        pooled_sum = global_spatiotemporal_sum_on_accelerator(
            self.driver,
            value,
            timeout_seconds=self.layer_timeout_seconds,
        )
        divisor = value.shape[0] * value.shape[3] * value.shape[4]
```

```
        denominator = divisor * self.fixed_scale * self.head_activation_scale
        integer_denominator = int(round(denominator))
        if (not math.isclose(denominator, integer_denominator, rel_tol=0.0, abs_tol=1e-9)
                or integer_denominator <= 0
                or integer_denominator & (integer_denominator - 1)):
            raise ValueError("head pooling denominator must be an exact positive power of two")
        head_integer = power_of_two_requantize_on_accelerator(
            self.driver,
            pooled_sum,
            division_shift=integer_denominator.bit_length() - 1,
            clamp_minimum=-128,
            clamp_maximum=127,
            timeout_seconds=self.layer_timeout_seconds,
        ).astype(np.int16)
        head = self.archive.head
        head_program = compile_u8_gemm(
            (head_integer + 128).astype(np.uint8),
            self.projections._encoded_dense_weights(head.weight_k_by_n),
            activation_zero_point=128,
            weight_zero_point=128,
            bias=head.bias_accumulator_units,
        )
        logits_accumulator = head_program.execute_batched(
            self.driver, timeout_seconds=self.layer_timeout_seconds)
        head_manifest = self.archive.manifest["head"]
        accumulator_scale = float(head_manifest["accumulator_scale"])
        logits = logits_accumulator.astype(np.float64) * accumulator_scale
        return SpikingformerInferenceResult(
            logits_accumulator=logits_accumulator,
            logits=logits,
            predicted_class=np.argmax(logits_accumulator, axis=1),
        )
```

40.3 Line annotations

Line 1: `"""Executable Spikingformer projection path for the NeuraEdge command ABI."""` This is executable documentation attached to the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import math` This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from typing import Sequence` This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import numpy as np` This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `from neuraedge_compiler import (` This statement continues through source line 17; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `ProgramExecutor,` This is continuation line 2 of the 7-line logical statement that begins at source line 11. Its exact fragment is 'ProgramExecutor,'. The important operands are ProgramExecutor: the value named 'program executor' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `compile_spike_conv2d,` This is continuation line 3 of the 7-line logical statement that begins at source line 11. Its exact fragment is 'compile_spike_conv2d,'. The important operands are compile_spike_conv2d: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `compile_spike_gemm,` This is continuation line 4 of the 7-line logical statement that begins at source line 11. Its exact fragment is 'compile_spike_gemm,'. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `compile_u8_conv2d,` This is continuation line 5 of the 7-line logical statement that begins at source line 11. Its exact fragment is 'compile_u8_conv2d,'. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `compile_u8_gemm,` This is continuation line 6 of the 7-line logical statement that begins at source line 11. Its exact fragment is 'compile_u8_gemm,'. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 11. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `from neuraedge_golden import VectorOperation, round_shift_signed, saturate_signed` This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `from neuraedge_protocol import (` This statement continues through source line 29; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `ProgramRecord,` This is continuation line 2 of the 11-line logical statement that begins at source line 19. Its exact fragment is 'ProgramRecord,'. The important operands are ProgramRecord: the value named 'program record' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `clear_status_command,` This is continuation line 3 of the 11-line logical statement that begins at source line 19. Its exact fragment is 'clear_status_command,'. The important operands are clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This

imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `pack_vector_beats,` This is continuation line 4 of the 11-line logical statement that begins at source line 19. Its exact fragment is 'pack_vector_beats,'. The important operands are pack_vector_beats: the value named 'pack vector beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `lif_command,` This is continuation line 5 of the 11-line logical statement that begins at source line 19. Its exact fragment is 'lif_command,'. The important operands are lif_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `pack_lif_beats,` This is continuation line 6 of the 11-line logical statement that begins at source line 19. Its exact fragment is 'pack_lif_beats,'. The important operands are pack_lif_beats: the value named 'pack lif beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 25: `unpack_lif_result,` This is continuation line 7 of the 11-line logical statement that begins at source line 19. Its exact fragment is 'unpack_lif_result,'. The important operands are unpack_lif_result: the value named 'unpack lif result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 26: `unpack_lif_trace,` This is continuation line 8 of the 11-line logical statement that begins at source line 19. Its exact fragment is 'unpack_lif_trace,'. The important operands are unpack_lif_trace: the value named 'unpack lif trace' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 27: `unpack_vector_result,` This is continuation line 9 of the 11-line logical statement that begins at source line 19. Its exact fragment is 'unpack_vector_result,'. The important operands are unpack_vector_result: the value named 'unpack vector result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 28: `vector_command,` This is continuation line 10 of the 11-line logical statement that begins at source line 19. Its exact fragment is 'vector_command,'. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 29: `)` This is continuation line 11 of the 11-line logical statement that begins at source line 19. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 30: `from spikingformer_parameters import ProjectionParameters, SpikingformerParameterArchive` This imports the named Python API so the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction,

and classifier stages to hardware operations can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 33: `VECTOR_LANES = 8` This assigns ‘8‘ to ‘VECTOR_LANES ‘ in host memory. Here VECTOR_LANES means one parallel arithmetic element within a vector or tensor datapath. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 34: `LIF_LANES = 32` This assigns ‘32‘ to ‘LIF_LANES ‘ in host memory. Here LIF_LANES means one parallel arithmetic element within a vector or tensor datapath. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 37: `def vector_binary_on_accelerator(` This starts function vector_binary_on_accelerator. Its indented body performs vector binary on accelerator as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 38: `driver: ProgramExecutor,` Inside the function vector_binary_on_accelerator, which handles vector binary on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are driver: the value named 'driver' in this block; ProgramExecutor: the value named 'program executor' in this block.

Line 39: `operation: VectorOperation,` Inside the function vector_binary_on_accelerator, which handles vector binary on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are operation: the value named 'operation' in this block; VectorOperation: the value named 'vector operation' in this block.

Line 40: `operand_a: np.ndarray,` Inside the function vector_binary_on_accelerator, which handles vector binary on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are operand_a: the value named 'operand a' in this block; ndarray: the value named 'ndarray' in this block.

Line 41: `operand_b: np.ndarray,` Inside the function vector_binary_on_accelerator, which handles vector binary on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are operand_b: the value named 'operand b' in this block; ndarray: the value named 'ndarray' in this block.

Line 42: `*,` Inside the function vector_binary_on_accelerator, which handles vector binary on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 43: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 44: `) -> np.ndarray:` Inside the function vector_binary_on_accelerator, which handles vector binary on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named 'ndarray' in this block.

Line 45: `"""Apply a binary vector-ALU operation to two equally shaped tensors."""` This is executable documentation attached to the function vector_binary_on_accelerator, which handles vector binary on accelerator. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 47: `if operation not in {VectorOperation.ADD_SAT, VectorOperation.MAXIMUM}:` This executes the following indented host-code branch only when ‘operation not in {VectorOperation.ADD_SAT, VectorOperation.MAXIMUM}‘ is true. The important operands are operation: the value named ’operation’ in this block; not: the value named ’not’ in this block; in: the value named ’in’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 48: `raise ValueError("this helper supports saturating add and maximum")` This rejects an invalid state by raising ‘ValueError("this helper supports saturating add and maximum")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 49: `a = np.asarray(operand_a, dtype=np.int64)` This assigns ‘np.asarray(operand_a, dtype=np.int64)‘ to ‘a ‘ in host memory. Here a means the value named ’a’ in this block. The important operands are asarray: the value named ’asarray’ in this block; operand_a: the value named ’operand a’ in this block; dtype: the value named ’dtype’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 50: `b = np.asarray(operand_b, dtype=np.int64)` This assigns ‘np.asarray(operand_b, dtype=np.int64)‘ to ‘b ‘ in host memory. Here b means the value named ’b’ in this block. The important operands are asarray: the value named ’asarray’ in this block; operand_b: the value named ’operand b’ in this block; dtype: the value named ’dtype’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 51: `if a.shape != b.shape:` This executes the following indented host-code branch only when ‘a.shape != b.shape‘ is true. The important operands are a: the value named ’a’ in this block; shape: the value named ’shape’ in this block; b: the value named ’b’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 52: `raise ValueError("vector operands must have equal shapes")` This rejects an invalid state by raising ‘ValueError("vector operands must have equal shapes")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 53: `if a.size and (a.min() < -(1 << 31) or a.max() > (1 << 31) - 1` Inside the function vector_binary_on_accel which handles vector binary on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are a: the value named ’a’ in this block; size: the value named ’size’ in this block; and: the value named ’and’ in this block.

Line 54: `or b.min() < -(1 << 31) or b.max() > (1 << 31) - 1):` Inside the function vector_binary_on_accelerator, which handles vector binary on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are or: the value named ’or’ in this block; b: the value named ’b’ in this block; min: the value named ’min’ in this block.

Line 55: `raise OverflowError("vector operands do not fit signed 32-bit lanes")` This rejects an invalid state by raising ‘OverflowError("vector operands do not fit signed 32-bit lanes")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 56: `flat_a = a.reshape(-1)` This assigns ‘a.reshape(-1)‘ to ‘flat_a ‘ in host memory. Here flat_a means the value named ’flat a’ in this block. The important operands are a: the value named ’a’ in this block; reshape: the value named ’reshape’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 57: `flat_b = b.reshape(-1)` This assigns ‘b.reshape(-1)‘ to ‘flat_b ‘ in host memory. Here flat_b means the value named ’flat b’ in this block. The important operands are b: the value named ’b’ in this block; reshape: the value named ’reshape’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 58: `records: list[ProgramRecord] = []` This assigns ‘[]‘ to ‘records‘ in host memory. Here records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 59: `counts: list[int] = []` This assigns ‘[]‘ to ‘counts‘ in host memory. Here counts means the value named ’counts’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 60: `command = vector_command(int(operation))` This assigns ‘vector_command(int(operation))‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named ’operation’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 61: `for start in range(0, len(flat_a), VECTOR_LANES):` This iterates start over ‘range(0, len(flat_a), VECTOR_LANES)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 62: `count = min(VECTOR_LANES, len(flat_a) - start)` This assigns ‘min(VECTOR_LANES, len(flat_a) - start)‘ to ‘count ‘ in host memory. Here count means the value named ’count’ in this block. The important operands are min: the value named ’min’ in this block; VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath; len: the value named ’len’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 63: `lanes_a = flat_a[start:start + count].tolist()` This assigns ‘flat_a[start:start + count].tolist()‘ to ‘lanes_a ‘ in host memory. Here lanes_a means one parallel arithmetic element within a vector or tensor datapath. The important operands are flat_a: the value named ’flat a’ in this block; start: the value named ’start’ in this block; count: the value named ’count’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 64: `lanes_b = flat_b[start:start + count].tolist()` This assigns ‘flat_b[start:start + count].tolist()‘ to ‘lanes_b ‘ in host memory. Here lanes_b means one parallel arithmetic element within a vector or tensor datapath. The important operands are flat_b: the value named ’flat b’ in this block; start: the value named ’start’ in this block; count: the value named ’count’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 65: `lanes_a.extend([0] * (VECTOR_LANES - count))` This calls ‘lanes_a.extend‘ with ‘[0] * (VECTOR_LANES - count)‘ inside the function vector_binary_on_accelerator, which handles vector binary on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 66: `lanes_b.extend([0] * (VECTOR_LANES - count))` This calls ‘lanes_b.extend‘ with ‘[0] * (VECTOR_LANES - count)‘ inside the function vector_binary_on_accelerator, which handles vector binary on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 67: `records.append(ProgramRecord(` This statement continues through source line 68; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes_a, lanes_b), 4)‘ inside the function vector_binary_on_accelerator, which handles vector binary on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 68: `command, pack_vector_beats(lanes_a, lanes_b), 4))` This is continuation line 2 of the 2-line logical statement that begins at source line 67. Its exact fragment is ‘command, pack_vector_beats(lanes_a, lanes_b), 4))‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; pack_vector_beats: the value named ’pack vector beats’ in this block; lanes_a: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes_a, lanes_b), 4)‘ inside the function vector_binary_on_accelerator, which handles vector binary on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 69: `counts.append(count)` This calls ‘counts.append‘ with ‘count‘ inside the function vector_binary_on_accelerator, which handles vector binary on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 70: `if not records:` This executes the following indented host-code branch only when ‘not records‘ is true. The important operands are not: the value named ’not’ in this block; records: the number of descriptor records accepted since program start. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 71: `return np.empty(a.shape, dtype=np.int64)` This returns ‘np.empty(a.shape, dtype=np.int64)‘ to the caller of the function vector_binary_on_accelerator, which handles vector binary on accelerator, ending this invocation. The important operands are empty: the value named ’empty’ in this block; a: the value named ’a’ in this block; shape: the value named ’shape’ in this block.

Line 72: `words = driver.execute_program(records, timeout_seconds=timeout_seconds)` This assigns ‘driver.execute_pro timeout_seconds=timeout_seconds)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 73: `if len(words) != 4 * len(records):` This executes the following indented host-code branch only when ‘len(words) != 4 * len(records)‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block; records: the number of descriptor records accepted since program start. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 74: `raise RuntimeError(f"vector program returned {len(words)} beats; expected {4 * len(records)}")` This rejects an invalid state by raising ‘RuntimeError(f"vector program returned {len(words)} beats; expected {4 * len(records)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 75: `result = np.empty(len(flat_a), dtype=np.int64)` This assigns ‘np.empty(len(flat_a), dtype=np.int64)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are empty: the value named ’empty’ in this block; len: the value named ’len’ in this block; flat_a: the value named ’flat a’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 76: `cursor = 0` This assigns ‘0‘ to ‘cursor ‘ in host memory. Here cursor means the value named ’cursor’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 77: `for record_index, count in enumerate(counts):` This iterates record_index, count over ‘enumerate(counts)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 78: `lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])` This assigns ‘unpack_vector_result(words[4 * record_index:4 * record_index + 4])‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are unpack_vector_result: the value named ’unpack vector result’ in this block; words: the value named ’words’ in this block; record_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 79: `result[cursor:cursor + count] = lanes[:count]` This assigns ‘lanes[:count]‘ to ‘result[cursor‘ in host memory. Here result means the value named ’result’ in this block. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath; count: the value named ’count’ in this block. Context: the function vector_binary_on_accelerator, which handles vector binary on accelerator.

Line 80: `cursor += count` Inside the function vector_binary_on_accelerator, which handles vector binary on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are cursor: the value named ’cursor’ in this block; count: the value named ’count’ in this block.

Line 81: `return result.reshape(a.shape)` This returns ‘result.reshape(a.shape)‘ to the caller of the function vector_binary_on_accelerator, which handles vector binary on accelerator, ending this invocation. The important operands are result: the value named ’result’ in this block; reshape: the value named ’reshape’ in this block; a: the value named ’a’ in this block.

Line 84: `def power_of_two_requantize_on_accelerator(` This starts function power_of_two_requantize_on_accelerator. Its indented body performs power of two requantize on accelerator as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 85: `driver: ProgramExecutor,` Inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are driver: the value named ’driver’ in this block; ProgramExecutor: the value named ’program executor’ in this block.

Line 86: `values: np.ndarray,` Inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are values: the value named ’values’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 87: `*,` Inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 88: `division_shift: int,` Inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are division_shift: the value named ’division shift’ in this block.

Line 89: `clamp_minimum: int = -(1 << 31),` This assigns ‘-(1 << 31),‘ to ‘clamp_minimum‘ in host memory. Here clamp_minimum means the value named ’clamp minimum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 90: `clamp_maximum: int = (1 << 31) - 1,` This assigns ‘(1 << 31) - 1,‘ to ‘clamp_maximum‘ in host memory. Here clamp_maximum means the value named ’clamp maximum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 91: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 92: `) -> np.ndarray:` Inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named 'ndarray' in this block.

Line 93: `"""Round "values / 2**division_shift" in the flexible vector datapath."""` This is executable documentation attached to the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 95: `source = np.asarray(values, dtype=np.int64)` This assigns 'np.asarray(values, dtype=np.int64)' to 'source ' in host memory. Here source means the value named 'source' in this block. The important operands are asarray: the value named 'asarray' in this block; values: the value named 'values' in this block; dtype: the value named 'dtype' in this block. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 96: `if not 0 <= division_shift <= 16:` This executes the following indented host-code branch only when 'not 0 <= division_shift <= 16' is true. The important operands are not: the value named 'not' in this block; division_shift: the value named 'division shift' in this block. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 97: `raise ValueError("division_shift must be in [0,16]")` This rejects an invalid state by raising 'ValueError("division_shift must be in [0,16]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 98: `if source.size and (` This line opens or closes a multi-line Python expression inside the function power_of_two_requa which handles power of two requantize on accelerator; the neighboring lines supply its arguments or elements.

Line 99: `source.min() < -(1 << 31) or source.max() > (1 << 31) - 1` Inside the function power_of_two_requantize_on which handles power of two requantize on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are source: the value named 'source' in this block; min: the value named 'min' in this block; or: the value named 'or' in this block.

Line 100: `):` Inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 101: `raise OverflowError("requantizer input must fit signed 32-bit lanes")` This rejects an invalid state by raising 'OverflowError("requantizer input must fit signed 32-bit lanes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 102: `flat = source.reshape(-1)` This assigns 'source.reshape(-1)' to 'flat ' in host memory. Here flat means the value named 'flat' in this block. The important operands are source: the value named 'source' in this block; reshape: the value named 'reshape' in this block. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 103: `records: list[ProgramRecord] = []` This assigns '[]' to 'records' in host memory. Here records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 104: `counts: list[int] = []` This assigns '[]' to 'counts' in host memory. Here counts means the value named 'counts' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 105: `command = vector_command(` This statement continues through source line 109; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'vector_command(int(VectorOperation.FLEX_REQUA multiplier_q15=1 << 14, right_shift=division_shift - 1,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 106: `int(VectorOperation.FLEX_REQUANTIZE),` This is continuation line 2 of the 5-line logical statement that begins at source line 105. Its exact fragment is 'int(VectorOperation.FLEX_REQUANTIZE),'. The important operands are VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=1 << 14, right_shift=division_shift - 1,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 107: `multiplier_q15=1 << 14,` This is continuation line 3 of the 5-line logical statement that begins at source line 105. Its exact fragment is 'multiplier_q15=1 << 14,'. The important operands are multiplier_q15: the value named 'multiplier q15' in this block. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=1 << 14, right_shift=division_shift - 1,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 108: `right_shift=division_shift - 1,` This is continuation line 4 of the 5-line logical statement that begins at source line 105. Its exact fragment is 'right_shift=division_shift - 1,'. The important operands are right_shift: the value named 'right shift' in this block; division_shift: the value named 'division shift' in this block. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=1 << 14, right_shift=division_shift - 1,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 109: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 105. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=1 << 14, right_shift=division_shift - 1,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in

this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 110: `for start in range(0, flat.size, VECTOR_LANES):` This iterates start over ‘range(0, flat.size, VECTOR_LANES)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 111: `stop = min(start + VECTOR_LANES, flat.size)` This assigns ‘min(start + VECTOR_LANES, flat.size)‘ to ‘stop ‘ in host memory. Here stop means the value named ’stop’ in this block. The important operands are min: the value named ’min’ in this block; start: the value named ’start’ in this block; VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 112: `count = stop - start` This assigns ‘stop - start‘ to ‘count ‘ in host memory. Here count means the value named ’count’ in this block. The important operands are stop: the value named ’stop’ in this block; start: the value named ’start’ in this block. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 113: `lanes = flat[start:stop].tolist() + [0] * (VECTOR_LANES - count)` This assigns ‘flat[start:stop].tolist() + [0] * (VECTOR_LANES - count)‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are flat: the value named ’flat’ in this block; start: the value named ’start’ in this block; stop: the value named ’stop’ in this block. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 114: `records.append(ProgramRecord(` This statement continues through source line 123; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximu), 4,)‘ inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 115: `command,` This is continuation line 2 of the 10-line logical statement that begins at source line 114. Its exact fragment is ‘command,‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)‘ inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 116: `pack_vector_beats(` This is continuation line 3 of the 10-line logical statement that begins at source line 114. Its exact fragment is ‘pack_vector_beats(‘. The important operands are pack_vector_beats: the value named ’pack vector beats’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)‘ inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 117: `lanes,` This is continuation line 4 of the 10-line logical statement that begins at source line 114. Its exact fragment is ‘lanes,‘. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)‘ inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 118: `[0] * VECTOR_LANES,` This is continuation line 5 of the 10-line logical statement that begins at source line 114. Its exact fragment is '[0] * VECTOR_LANES,'. The important operands are VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)' inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 119: `clamp_minimum=clamp_minimum,` This is continuation line 6 of the 10-line logical statement that begins at source line 114. Its exact fragment is 'clamp_minimum=clamp_minimum,'. The important operands are clamp_minimum: the value named 'clamp minimum' in this block. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_m
clamp_maximum=clamp_maximum,), 4,)' inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 120: `clamp_maximum=clamp_maximum,` This is continuation line 7 of the 10-line logical statement that begins at source line 114. Its exact fragment is 'clamp_maximum=clamp_maximum,'. The important operands are clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_m
clamp_maximum=clamp_maximum,), 4,)' inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 121: `),` This is continuation line 8 of the 10-line logical statement that begins at source line 114. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)' inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 122: `4,` This is continuation line 9 of the 10-line logical statement that begins at source line 114. Its exact fragment is '4,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)' inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 123: `))` This is continuation line 10 of the 10-line logical statement that begins at source line 114. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(lanes, [0] * VECTOR_LANES, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)' inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 124: `counts.append(count)` This calls 'counts.append' with 'count' inside the function power_of_two_requantize_on_acc
which handles power of two requantize on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 125: `if not records:` This executes the following indented host-code branch only when 'not records' is true. The important operands are not: the value named 'not' in this block; records: the number of descriptor records accepted since program start. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 126: `return np.empty(source.shape, dtype=np.int64)` This returns ‘np.empty(source.shape, dtype=np.int64)‘ to the caller of the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator, ending this invocation. The important operands are empty: the value named ’empty’ in this block; source: the value named ’source’ in this block; shape: the value named ’shape’ in this block.

Line 127: `words = driver.execute_program(records, timeout_seconds=timeout_seconds)` This assigns ‘driver.execute_pr timeout_seconds=timeout_seconds)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 128: `if len(words) != 4 * len(records):` This executes the following indented host-code branch only when ‘len(words) != 4 * len(records)‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block; records: the number of descriptor records accepted since program start. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 129: `raise RuntimeError("power-of-two requantizer returned an invalid stream length")` This rejects an invalid state by raising ‘RuntimeError("power-of-two requantizer returned an invalid stream length")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 130: `result = np.empty(flat.size, dtype=np.int64)` This assigns ‘np.empty(flat.size, dtype=np.int64)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are empty: the value named ’empty’ in this block; flat: the value named ’flat’ in this block; size: the value named ’size’ in this block. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 131: `cursor = 0` This assigns ‘0‘ to ‘cursor ‘ in host memory. Here cursor means the value named ’cursor’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 132: `for record_index, count in enumerate(counts):` This iterates record_index, count over ‘enumerate(counts)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 133: `lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])` This assigns ‘unpack_vector_result(words[4 * record_index:4 * record_index + 4])‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are unpack_vector_result: the value named ’unpack vector result’ in this block; words: the value named ’words’ in this block; record_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 134: `result[cursor:cursor + count] = lanes[:count]` This assigns ‘lanes[:count]‘ to ‘result[cursor‘ in host memory. Here result means the value named ’result’ in this block. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath; count: the value named ’count’ in this block. Context: the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator.

Line 135: `cursor += count` Inside the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are cursor: the value named ’cursor’ in this block; count: the value named ’count’ in this block.

Line 136: `return result.reshape(source.shape)` This returns ‘result.reshape(source.shape)‘ to the caller of the function power_of_two_requantize_on_accelerator, which handles power of two requantize on accelerator, ending this invocation. The important operands are result: the value named ’result’ in this block; reshape: the value named ’reshape’ in this block; source: the value named ’source’ in this block.

Line 139: `def global_spatiotemporal_sum_on_accelerator(` This starts function global_spatiotemporal_sum_on_accelerat Its indented body performs global spatiotemporal sum on accelerator as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 140: `driver: ProgramExecutor,` Inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are driver: the value named 'driver' in this block; ProgramExecutor: the value named 'program executor' in this block.

Line 141: `values: np.ndarray,` Inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are values: the value named 'values' in this block; ndarray: the value named 'ndarray' in this block.

Line 142: `*,` Inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 143: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 144: `) -> np.ndarray:` Inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named 'ndarray' in this block.

Line 145: `"""Reduce a [T,B,C,H,W] tensor to [B,C] through pairwise vector adds."""` This is executable documentation attached to the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 147: `source = np.asarray(values, dtype=np.int64)` This assigns 'np.asarray(values, dtype=np.int64)' to 'source ' in host memory. Here source means the value named 'source' in this block. The important operands are asarray: the value named 'asarray' in this block; values: the value named 'values' in this block; dtype: the value named 'dtype' in this block. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 148: `if source.ndim != 5:` This executes the following indented host-code branch only when 'source.ndim != 5' is true. The important operands are source: the value named 'source' in this block; ndim: the value named 'ndim' in this block. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 149: `raise ValueError("global sum input must have shape [T,B,C,H,W]")` This rejects an invalid state by raising 'ValueError("global sum input must have shape [T,B,C,H,W]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 150: `timesteps, batch, channels, height, width = source.shape` This assigns 'source.shape' to 'timesteps, batch, channels, height, width ' in host memory. Here timesteps means one discrete SNN update instant. The important operands are source: the value named 'source' in this block; shape: the value named 'shape' in this block. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 151: `reduction = source.transpose(0, 3, 4, 1, 2).reshape(` This statement continues through source line 152; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘source.transpose(0, 3, 4, 1, 2).reshape(timesteps * height * width, batch, channels)‘ to ‘reduction ‘ in host memory. Here reduction means the value named ’reduction’ in this block. The important operands are source: the value named ’source’ in this block; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 152: `timesteps * height * width, batch, channels)` This is continuation line 2 of the 2-line logical statement that begins at source line 151. Its exact fragment is ‘timesteps * height * width, batch, channels)‘. The important operands are timesteps: one discrete SNN update instant; height: the value named ’height’ in this block; width: the value named ’width’ in this block. Read together, the complete operation is: This assigns ‘source.transpose(0, 3, 4, 1, 2).reshape(timesteps * height * width, batch, channels)‘ to ‘reduction ‘ in host memory. Here reduction means the value named ’reduction’ in this block. The important operands are source: the value named ’source’ in this block; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 153: `if int(np.abs(reduction).max(initial=0)) * reduction.shape[0] > (1 << 31) - 1:` This executes the following indented host-code branch only when ‘int(np.abs(reduction).max(initial=0)) * reduction.shape[0] > (1 << 31) - 1‘ is true. The important operands are abs: the value named ’abs’ in this block; reduction: the value named ’reduction’ in this block; max: the value named ’max’ in this block. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 154: `raise OverflowError("global sum can exceed the signed-32 vector accumulator")` This rejects an invalid state by raising ‘OverflowError("global sum can exceed the signed-32 vector accumulator")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 155: `candidates = [reduction[index] for index in range(reduction.shape[0])]` This assigns ‘[reduction[index] for index in range(reduction.shape[0])]‘ to ‘candidates ‘ in host memory. Here candidates means the value named ’candidates’ in this block. The important operands are reduction: the value named ’reduction’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 156: `while len(candidates) > 1:` This repeats the following host-code block while ‘len(candidates) > 1‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 157: `reduced: list[np.ndarray] = []` This assigns ‘[]‘ to ‘reduced‘ in host memory. Here reduced means the value named ’reduced’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 158: `for index in range(0, len(candidates) - 1, 2):` This iterates index over ‘range(0, len(candidates) - 1, 2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 159: `reduced.append(vector_binary_on_accelerator(` This statement continues through source line 165; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘reduced.append‘ with ‘vector_binary_on_accelerator(driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)‘ inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 160: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 159. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This calls ‘reduced.append‘ with ‘vector_binary_on_accelerator(driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)‘ inside the function

global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 161: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 159. Its exact fragment is 'VectorOperation.ADD_SAT,'. The important operands are VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named 'add sat' in this block. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 162: `candidates[index],` This is continuation line 4 of the 7-line logical statement that begins at source line 159. Its exact fragment is 'candidates[index],'. The important operands are candidates: the value named 'candidates' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 163: `candidates[index + 1],` This is continuation line 5 of the 7-line logical statement that begins at source line 159. Its exact fragment is 'candidates[index + 1],'. The important operands are candidates: the value named 'candidates' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function global_spatiotemporal_sum_on_acc which handles global spatiotemporal sum on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 164: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 159. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 165: `))` This is continuation line 7 of the 7-line logical statement that begins at source line 159. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 166: `if len(candidates) & 1:` This executes the following indented host-code branch only when 'len(candidates) & 1' is true. The important operands are len: the value named 'len' in this block; candidates: the value named 'candidates' in this block. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 167: `reduced.append(candidates[-1])` This calls 'reduced.append' with 'candidates[-1]' inside the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 168: `candidates = reduced` This assigns ‘reduced‘ to ‘candidates ‘ in host memory. Here candidates means the value named ’candidates’ in this block. The important operands are reduced: the value named ’reduced’ in this block. Context: the function global_spatiotemporal_sum_on_accelerator, which handles global spatiotemporal sum on accelerator.

Line 169: `return candidates[0]` This returns ‘candidates[0]‘ to the caller of the function global_spatiotemporal_sum_on_acce which handles global spatiotemporal sum on accelerator, ending this invocation. The important operands are candidates: the value named ’candidates’ in this block.

Line 172: `def binary_max_pool2d_on_accelerator(` This starts function binary_max_pool2d_on_accelerator. Its indented body performs binary max pool2d on accelerator as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 173: `driver: ProgramExecutor,` Inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are driver: the value named ’driver’ in this block; ProgramExecutor: the value named ’program executor’ in this block.

Line 174: `values: np.ndarray,` Inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are values: the value named ’values’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 175: `*,` Inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 176: `kernel_size: int = 3,` This assigns ‘3,‘ to ‘kernel_size‘ in host memory. Here kernel_size means the value named ’kernel size’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 177: `stride: int = 2,` This assigns ‘2,‘ to ‘stride‘ in host memory. Here stride means the value named ’stride’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 178: `padding: int = 1,` This assigns ‘1,‘ to ‘padding‘ in host memory. Here padding means the value named ’padding’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 179: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 180: `) -> np.ndarray:` Inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named ’ndarray’ in this block.

Line 181: `"""Max-pool NCHW binary spikes with pairwise vector-maximum trees."""` This is executable documentation attached to the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 183: `source = np.asarray(values)` This assigns ‘np.asarray(values)‘ to ‘source ‘ in host memory. Here source means the value named ’source’ in this block. The important operands are asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 184: `if source.ndim != 4 or np.any((source != 0) & (source != 1)):` This executes the following indented host-code branch only when ‘source.ndim != 4 or np.any((source != 0) & (source != 1))‘ is true. The important operands are source: the value named ’source’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 185: `raise ValueError("binary max-pool input must be an NCHW tensor containing only 0/1")` This rejects an invalid state by raising ‘ValueError("binary max-pool input must be an NCHW tensor containing only 0/1")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 186: `if kernel_size <= 0 or stride <= 0 or padding < 0:` This executes the following indented host-code branch only when ‘kernel_size <= 0 or stride <= 0 or padding < 0‘ is true. The important operands are kernel_size: the value named ’kernel size’ in this block; or: the value named ’or’ in this block; stride: the value named ’stride’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 187: `raise ValueError("invalid max-pool geometry")` This rejects an invalid state by raising ‘ValueError("invalid max-pool geometry")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 188: `padded = np.pad(` This statement continues through source line 192; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.pad(source.astype(np.int64), ((0, 0), (0, 0), (padding, padding), (padding, padding)), mode="constant",)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are pad: the value named ’pad’ in this block; source: the value named ’source’ in this block; astype: the value named ’astype’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 189: `source.astype(np.int64),` This is continuation line 2 of the 5-line logical statement that begins at source line 188. Its exact fragment is ‘source.astype(np.int64),‘. The important operands are source: the value named ’source’ in this block; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This assigns ‘np.pad(source.astype(np.int64), ((0, 0), (0, 0), (padding, padding), (padding, padding)), mode="constant",)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are pad: the value named ’pad’ in this block; source: the value named ’source’ in this block; astype: the value named ’astype’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 190: `((0, 0), (0, 0), (padding, padding), (padding, padding)),` This is continuation line 3 of the 5-line logical statement that begins at source line 188. Its exact fragment is ‘((0, 0), (0, 0), (padding, padding), (padding, padding)),‘. The important operands are padding: the value named ’padding’ in this block. Read together, the complete operation is: This assigns ‘np.pad(source.astype(np.int64), ((0, 0), (0, 0), (padding, padding), (padding, padding)), mode="constant",)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are pad: the value named ’pad’ in this block; source: the value named ’source’ in this block; astype: the value named ’astype’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 191: `mode="constant",` This is continuation line 4 of the 5-line logical statement that begins at source line 188. Its exact fragment is ‘mode="constant",‘. The important operands are mode: the value named ’mode’ in this block; constant: the value named ’constant’ in this block. Read together, the complete operation is: This assigns ‘np.pad(source.astype(np.int64), ((0, 0), (0, 0), (padding, padding), (padding, padding)), mode="constant",)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are pad: the value named ’pad’ in this block; source: the value named ’source’ in this block; astype: the value named ’astype’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 192: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 188. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.pad(source.astype(np.int64), ((0, 0), (0, 0), (padding, padding), (padding, padding)), mode="constant",)‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are pad: the value named ’pad’ in this block; source: the value named ’source’ in this block; astype: the value named ’astype’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 193: `output_height = (source.shape[2] + 2 * padding - kernel_size) // stride + 1` This assigns ‘(source.shape + 2 * padding - kernel_size) // stride + 1‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are source: the value named ’source’ in this block; shape: the value named ’shape’ in this block; padding: the value named ’padding’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 194: `output_width = (source.shape[3] + 2 * padding - kernel_size) // stride + 1` This assigns ‘(source.shape[3 + 2 * padding - kernel_size) // stride + 1‘ to ‘output_width ‘ in host memory. Here output_width means the value named ’output width’ in this block. The important operands are source: the value named ’source’ in this block; shape: the value named ’shape’ in this block; padding: the value named ’padding’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 195: `candidates = [` This statement continues through source line 200; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘[padded[:, :, kernel_y:kernel_y + output_height * stride:stride, kernel_x:kernel_x + output_width * stride:stride] for kernel_y in range(kernel_size) for kernel_x in range(kernel_size)]‘ to ‘candidates ‘ in host memory. Here candidates means the value named ’candidates’ in this block. The important operands are padded: the value named ’padded’ in this block; kernel_y: the value named ’kernel y’ in this block; output_height: the value named ’output height’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 196: `padded[:, :, kernel_y:kernel_y + output_height * stride:stride,` This is continuation line 2 of the 6-line logical statement that begins at source line 195. Its exact fragment is ‘padded[:, :, kernel_y:kernel_y + output_height * stride:stride,‘. The important operands are padded: the value named ’padded’ in this block; kernel_y: the value named ’kernel y’ in this block; output_height: the value named ’output height’ in this block. Read together, the complete operation is: This assigns ‘[padded[:, :, kernel_y:kernel_y + output_height * stride:stride, kernel_x:kernel_x + output_width * stride:stride] for kernel_y in range(kernel_size) for kernel_x in range(kernel_size)]‘ to ‘candidates ‘ in host memory. Here candidates means the value named ’candidates’ in this block. The important operands are padded: the value named ’padded’ in this block; kernel_y: the value named ’kernel y’ in this block; output_height: the value named ’output height’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 197: `kernel_x:kernel_x + output_width * stride:stride]` This is continuation line 3 of the 6-line logical statement that begins at source line 195. Its exact fragment is ‘kernel_x:kernel_x + output_width * stride:stride]‘. The important operands are kernel_x: the value named ’kernel x’ in this block; output_width: the value named ’output width’ in this block; stride: the value named ’stride’ in this block. Read together, the complete operation is: This assigns ‘[padded[:, :, kernel_y:kernel_y + output_height * stride:stride, kernel_x:kernel_x + output_width * stride:stride] for kernel_y in range(kernel_size) for kernel_x in range(kernel_size)]‘ to ‘candidates ‘ in host memory. Here candidates means the value named ’candidates’ in this block. The important operands are padded: the value named ’padded’ in this block; kernel_y: the value named ’kernel y’ in this block; output_height: the value named ’output height’ in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 198: `for kernel_y in range(kernel_size)` This is continuation line 4 of the 6-line logical statement that begins at source line 195. Its exact fragment is ‘for kernel_y in range(kernel_size)‘. The important operands are for: the value named ’for’ in this block; kernel_y: the value named ’kernel y’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘[padded[:, :, kernel_y:kernel_y + output_height * stride:stride, kernel_x:kernel_x + output_width * stride:stride] for kernel_y in range(kernel_size) for kernel_x in range(kernel_size)]‘ to ‘candidates ‘ in host memory. Here candidates means the value named ’candidates’ in this block. The important

operands are padded: the value named 'padded' in this block; kernel_y: the value named 'kernel y' in this block; output_height: the value named 'output height' in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 199: `for kernel_x in range(kernel_size)` This is continuation line 5 of the 6-line logical statement that begins at source line 195. Its exact fragment is 'for kernel_x in range(kernel_size)'. The important operands are for: the value named 'for' in this block; kernel_x: the value named 'kernel x' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns '[padded[:, :, kernel_y:kernel_y + output_height * stride:stride, kernel_x:kernel_x + output_width * stride:stride] for kernel_y in range(kernel_size) for kernel_x in range(kernel_size)]' to 'candidates ' in host memory. Here candidates means the value named 'candidates' in this block. The important operands are padded: the value named 'padded' in this block; kernel_y: the value named 'kernel y' in this block; output_height: the value named 'output height' in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 200: `]` This is continuation line 6 of the 6-line logical statement that begins at source line 195. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '[padded[:, :, kernel_y:kernel_y + output_height * stride:stride, kernel_x:kernel_x + output_width * stride:stride] for kernel_y in range(kernel_size) for kernel_x in range(kernel_size)]' to 'candidates ' in host memory. Here candidates means the value named 'candidates' in this block. The important operands are padded: the value named 'padded' in this block; kernel_y: the value named 'kernel y' in this block; output_height: the value named 'output height' in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 201: `while len(candidates) > 1:` This repeats the following host-code block while 'len(candidates) > 1' remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 202: `reduced: list[np.ndarray] = []` This assigns '[]' to 'reduced' in host memory. Here reduced means the value named 'reduced' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 203: `for index in range(0, len(candidates) - 1, 2):` This iterates index over 'range(0, len(candidates) - 1, 2)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 204: `reduced.append(vector_binary_on_accelerator(` This statement continues through source line 210; the explanation uses the complete expression rather than only the first visual fragment. This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.MAXIMUM, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 205: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 204. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.MAXIMUM, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 206: `VectorOperation.MAXIMUM,` This is continuation line 3 of the 7-line logical statement that begins at source line 204. Its exact fragment is 'VectorOperation.MAXIMUM,'. The important operands are VectorOperation: the value named 'vector operation' in this block; MAXIMUM: the value named 'maximum' in this block. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.MAXIMUM, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 207: `candidates[index],` This is continuation line 4 of the 7-line logical statement that begins at source line 204. Its exact fragment is 'candidates[index],'. The important operands are candidates: the value named 'candidates' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.MAXIMUM, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function binary_max_pool2d_on_accelerator which handles binary max pool2d on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 208: `candidates[index + 1],` This is continuation line 5 of the 7-line logical statement that begins at source line 204. Its exact fragment is 'candidates[index + 1],'. The important operands are candidates: the value named 'candidates' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.MAXIMUM, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function binary_max_pool2d_on_accelerator which handles binary max pool2d on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 209: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 204. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.MAXIMUM, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 210: `))` This is continuation line 7 of the 7-line logical statement that begins at source line 204. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'reduced.append' with 'vector_binary_on_accelerator(driver, VectorOperation.MAXIMUM, candidates[index], candidates[index + 1], timeout_seconds=timeout_seconds,)' inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 211: `if len(candidates) & 1:` This executes the following indented host-code branch only when 'len(candidates) & 1' is true. The important operands are len: the value named 'len' in this block; candidates: the value named 'candidates' in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 212: `reduced.append(candidates[-1])` This calls 'reduced.append' with 'candidates[-1]' inside the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 213: `candidates = reduced` This assigns 'reduced' to 'candidates ' in host memory. Here candidates means the value named 'candidates' in this block. The important operands are reduced: the value named 'reduced' in this block. Context: the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator.

Line 214: `return candidates[0].astype(np.uint8)` This returns 'candidates[0].astype(np.uint8)' to the caller of the function binary_max_pool2d_on_accelerator, which handles binary max pool2d on accelerator, ending this invocation. The important operands are candidates: the value named 'candidates' in this block; astype: the value named 'astype' in this block; uint8: the value named 'uint8' in this block.

Line 217: `def lif_sequence_reference(` This starts function lif_sequence_reference. Its indented body performs lif sequence reference as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 218: `values_q10: np.ndarray,` Inside the function lif_sequence_reference, which handles lif sequence reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are values_q10: the value named 'values q10' in this block; ndarray: the value named 'ndarray' in this block.

Line 219: `*,` Inside the function lif_sequence_reference, which handles lif sequence reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 220: `threshold_q10: int = 1 << 10,` This assigns '1 << 10,' to 'threshold_q10' in host memory. Here threshold_q10 means the membrane level at which a neuron emits a spike. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 221: `leak_shift: int = 1,` This assigns '1,' to 'leak_shift' in host memory. Here leak_shift means the retained fraction of prior membrane state. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 222: `reset_q10: int = 0,` This assigns '0,' to 'reset_q10' in host memory. Here reset_q10 means the value named 'reset q10' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 223: `) -> np.ndarray:` Inside the function lif_sequence_reference, which handles lif sequence reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named 'ndarray' in this block.

Line 224: `"""Bit-exact fixed-point LIF sequence for an arbitrary tensor shape."""` This is executable documentation attached to the function lif_sequence_reference, which handles lif sequence reference. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 226: `values = np.asarray(values_q10, dtype=np.int64)` This assigns 'np.asarray(values_q10, dtype=np.int64)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are asarray: the value named 'asarray' in this block; values_q10: the value named 'values q10' in this block; dtype: the value named 'dtype' in this block. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 227: `if values.ndim < 2 or values.shape[0] == 0:` This executes the following indented host-code branch only when 'values.ndim < 2 or values.shape[0] == 0' is true. The important operands are values: the value named 'values' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 228: `raise ValueError("LIF input must have shape [timesteps, ...]")` This rejects an invalid state by raising 'ValueError("LIF input must have shape [timesteps, ...]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 229: `if not 0 <= leak_shift <= 31:` This executes the following indented host-code branch only when 'not 0 <= leak_shift <= 31' is true. The important operands are not: the value named 'not' in this block; leak_shift: the retained fraction of prior membrane state. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 230: `raise ValueError("leak_shift must be in [0,31]")` This rejects an invalid state by raising 'ValueError("leak_shift must be in [0,31]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 231: `membrane = np.full(values.shape[1:], reset_q10, dtype=np.int64)` This assigns ‘np.full(values.shape[1:], reset_q10, dtype=np.int64)‘ to ‘membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are full: the value named ’full’ in this block; values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 232: `spikes = np.zeros(values.shape, dtype=np.uint8)` This assigns ‘np.zeros(values.shape, dtype=np.uint8)‘ to ‘spikes ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are zeros: the value named ’zeros’ in this block; values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 233: `for timestep in range(values.shape[0]):` This iterates timestep over ‘range(values.shape[0])‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_reference, which handles lif sequence reference.

Line 234: `current = np.clip(values[timestep] >> 1, -(1 << 15), (1 << 15) - 1)` This assigns ‘np.clip(values[timestep] >> 1, -(1 << 15), (1 << 15) - 1)‘ to ‘current ‘ in host memory. Here current means the value named ’current’ in this block. The important operands are clip: the value named ’clip’ in this block; values: the value named ’values’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 235: `leaked = membrane - (membrane >> leak_shift)` This assigns ‘membrane - (membrane >> leak_shift)‘ to ‘leaked ‘ in host memory. Here leaked means the retained fraction of prior membrane state. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 236: `membrane = saturate_signed(leaked + current, 24)` This assigns ‘saturate_signed(leaked + current, 24)‘ to ‘membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are saturate_signed: the value named ’saturate signed’ in this block; leaked: the retained fraction of prior membrane state; current: the value named ’current’ in this block. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 237: `fired = membrane >= threshold_q10` Inside the function lif_sequence_reference, which handles lif sequence reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are fired: the value named ’fired’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; threshold_q10: the membrane level at which a neuron emits a spike.

Line 238: `spikes[timestep] = fired` This assigns ‘fired‘ to ‘spikes[timestep] ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are fired: the value named ’fired’ in this block. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 239: `membrane = np.where(fired, reset_q10, membrane)` This assigns ‘np.where(fired, reset_q10, membrane)‘ to ‘membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are where: the value named ’where’ in this block; fired: the value named ’fired’ in this block; reset_q10: the value named ’reset q10’ in this block. Context: the function lif_sequence_reference, which handles lif sequence reference.

Line 240: `return spikes` This returns ‘spikes‘ to the caller of the function lif_sequence_reference, which handles lif sequence reference, ending this invocation. The important operands are spikes: a binary event produced when a neuron crosses its threshold.

Line 243: `def lif_sequence_with_trace_reference(` This starts function lif_sequence_with_trace_reference. Its indented body performs lif sequence with trace reference as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 244: `values_q10: np.ndarray,` Inside the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are values_q10: the value named 'values q10' in this block; ndarray: the value named 'ndarray' in this block.

Line 245: `*,` Inside the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 246: `threshold_q10: int = 1 << 10,` This assigns '1 << 10,' to 'threshold_q10' in host memory. Here threshold_q10 means the membrane level at which a neuron emits a spike. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 247: `leak_shift: int = 1,` This assigns '1,' to 'leak_shift' in host memory. Here leak_shift means the retained fraction of prior membrane state. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 248: `reset_q10: int = 0,` This assigns '0,' to 'reset_q10' in host memory. Here reset_q10 means the value named 'reset q10' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 249: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 250: `"""Return spikes and the signed-24 membrane immediately before hard reset."""` This is executable documentation attached to the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 252: `values = np.asarray(values_q10, dtype=np.int64)` This assigns 'np.asarray(values_q10, dtype=np.int64)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are asarray: the value named 'asarray' in this block; values_q10: the value named 'values q10' in this block; dtype: the value named 'dtype' in this block. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 253: `if values.ndim < 2 or values.shape[0] == 0:` This executes the following indented host-code branch only when 'values.ndim < 2 or values.shape[0] == 0' is true. The important operands are values: the value named 'values' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 254: `raise ValueError("LIF input must have shape [timesteps, ...]")` This rejects an invalid state by raising 'ValueError("LIF input must have shape [timesteps, ...]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 255: `if not 0 <= leak_shift <= 31:` This executes the following indented host-code branch only when 'not 0 <= leak_shift <= 31' is true. The important operands are not: the value named 'not' in this block; leak_shift: the retained fraction of prior membrane state. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 256: `raise ValueError("leak_shift must be in [0,31]")` This rejects an invalid state by raising 'ValueError("leak_shift must be in [0,31]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 257: `membrane = np.full(values.shape[1:], reset_q10, dtype=np.int64)` This assigns 'np.full(values.shape[1:], reset_q10, dtype=np.int64)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are full: the value named 'full' in this block; values: the value named 'values' in this block; shape: the value named 'shape' in this block. Context: the function lif_sequence_with_trace_reference which handles lif sequence with trace reference.

Line 258: `spikes = np.zeros(values.shape, dtype=np.uint8)` This assigns 'np.zeros(values.shape, dtype=np.uint8)' to 'spikes ' in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are zeros: the value named 'zeros' in this block; values: the value named 'values' in this block; shape: the value named 'shape' in this block. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 259: `membrane_trace = np.zeros(values.shape, dtype=np.int64)` This assigns 'np.zeros(values.shape, dtype=np.int64 to 'membrane_trace ' in host memory. Here membrane_trace means the persistent LIF neuron state carrying charge between timesteps. The important operands are zeros: the value named 'zeros' in this block; values: the value named 'values' in this block; shape: the value named 'shape' in this block. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 260: `for timestep in range(values.shape[0]):` This iterates timestep over 'range(values.shape[0])'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 261: `current = np.clip(values[timestep] >> 1, -(1 << 15), (1 << 15) - 1)` This assigns 'np.clip(values[timestep] >> 1, -(1 << 15), (1 << 15) - 1)' to 'current ' in host memory. Here current means the value named 'current' in this block. The important operands are clip: the value named 'clip' in this block; values: the value named 'values' in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 262: `leaked = membrane - (membrane >> leak_shift)` This assigns 'membrane - (membrane >> leak_shift)' to 'leaked ' in host memory. Here leaked means the retained fraction of prior membrane state. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 263: `membrane = saturate_signed(leaked + current, 24)` This assigns 'saturate_signed(leaked + current, 24)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are saturate_signed: the value named 'saturate signed' in this block; leaked: the retained fraction of prior membrane state; current: the value named 'current' in this block. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 264: `membrane_trace[timestep] = membrane` This assigns 'membrane' to 'membrane_trace[timestep] ' in host memory. Here membrane_trace means the persistent LIF neuron state carrying charge between timesteps. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 265: `fired = membrane >= threshold_q10` Inside the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are fired: the value named 'fired' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; threshold_q10: the membrane level at which a neuron emits a spike.

Line 266: `spikes[timestep] = fired` This assigns 'fired' to 'spikes[timestep] ' in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are fired: the value named 'fired' in this block. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 267: `membrane = np.where(fired, reset_q10, membrane)` This assigns 'np.where(fired, reset_q10, membrane)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are where: the value named 'where' in this block; fired: the value named 'fired' in this block; reset_q10: the value named 'reset q10' in this block. Context: the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference.

Line 268: `return spikes, membrane_trace` This returns 'spikes, membrane_trace' to the caller of the function lif_sequence_with_trace_reference, which handles lif sequence with trace reference, ending this invocation. The important operands are spikes: a binary event produced when a neuron crosses its threshold; membrane_trace: the persistent LIF neuron state carrying charge between timesteps.

Line 271: `def lif_sequence_on_accelerator(` This starts function lif_sequence_on_accelerator. Its indented body performs lif sequence on accelerator as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 272: `driver: ProgramExecutor,` Inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are driver: the value named 'driver' in this block; ProgramExecutor: the value named 'program executor' in this block.

Line 273: `values_q10: np.ndarray,` Inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are values_q10: the value named 'values q10' in this block; ndarray: the value named 'ndarray' in this block.

Line 274: `*,` Inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 275: `threshold_q10: int = 1 << 10,` This assigns '1 << 10,' to 'threshold_q10' in host memory. Here threshold_q10 means the membrane level at which a neuron emits a spike. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 276: `leak_shift: int = 1,` This assigns '1,' to 'leak_shift' in host memory. Here leak_shift means the retained fraction of prior membrane state. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 277: `reset_q10: int = 0,` This assigns '0,' to 'reset_q10' in host memory. Here reset_q10 means the value named 'reset q10' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 278: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 279: `) -> np.ndarray:` Inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named 'ndarray' in this block.

Line 280: `"""Schedule every logical LIF state through the physical 32-lane bank."""` This is executable documentation attached to the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 282: `values = np.asarray(values_q10, dtype=np.int64)` This assigns ‘np.asarray(values_q10, dtype=np.int64)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are asarray: the value named ’asarray’ in this block; values_q10: the value named ’values q10’ in this block; dtype: the value named ’dtype’ in this block. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 283: `if values.ndim < 2 or values.shape[0] == 0:` This executes the following indented host-code branch only when ‘values.ndim < 2 or values.shape[0] == 0‘ is true. The important operands are values: the value named ’values’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 284: `raise ValueError("LIF input must have shape [timesteps, ...]")` This rejects an invalid state by raising ‘ValueError("LIF input must have shape [timesteps, ...]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 285: `if values.size and (values.min() < -(1 << 63) or values.max() > (1 << 63) - 1):` This executes the following indented host-code branch only when ‘values.size and (values.min() < -(1 << 63) or values.max() > (1 << 63) - 1)‘ is true. The important operands are values: the value named ’values’ in this block; size: the value named ’size’ in this block; and: the value named ’and’ in this block. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 286: `raise OverflowError("LIF input exceeds signed 64-bit preparation arithmetic")` This rejects an invalid state by raising ‘OverflowError("LIF input exceeds signed 64-bit preparation arithmetic")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 287: `timesteps = values.shape[0]` This assigns ‘values.shape[0]‘ to ‘timesteps ‘ in host memory. Here timesteps means one discrete SNN update instant. The important operands are values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 288: `flat = values.reshape(timesteps, -1)` This assigns ‘values.reshape(timesteps, -1)‘ to ‘flat ‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are values: the value named ’values’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 289: `records: list[ProgramRecord] = []` This assigns ‘[]‘ to ‘records‘ in host memory. Here records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 290: `bindings: list[tuple[int, int, int]] = []` This assigns ‘[]‘ to ‘bindings‘ in host memory. Here bindings means the value named ’bindings’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 291: `for element_start in range(0, flat.shape[1], LIF_LANES):` This iterates element_start over ‘range(0, flat.shape[1], LIF_LANES)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 292: `element_count = min(LIF_LANES, flat.shape[1] - element_start)` This assigns ‘min(LIF_LANES, flat.shape[1] - element_start)‘ to ‘element_count ‘ in host memory. Here element_count means an integer count retained for control or measurable telemetry. The important operands are min: the value named ’min’ in this block; LIF_LANES: one parallel arithmetic element within a vector or tensor datapath; flat: the value named ’flat’ in this block. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 293: `for timestep in range(timesteps):` This iterates timestep over ‘range(timesteps)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 294: `currents = np.clip(` This statement continues through source line 298; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 295: `flat[timestep, element_start:element_start + element_count] >> 1,` This is continuation line 2 of the 5-line logical statement that begins at source line 294. Its exact fragment is ‘flat[timestep, element_start:element_start + element_count] >> 1,‘. The important operands are flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant; element_start: the value named ’element start’ in this block. Read together, the complete operation is: This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 296: `-(1 << 15),` This is continuation line 3 of the 5-line logical statement that begins at source line 294. Its exact fragment is ‘-(1 << 15),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 297: `(1 << 15) - 1,` This is continuation line 4 of the 5-line logical statement that begins at source line 294. Its exact fragment is ‘(1 << 15) - 1,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 298: `).astype(np.int64).tolist()` This is continuation line 5 of the 5-line logical statement that begins at source line 294. Its exact fragment is ‘).astype(np.int64).tolist()‘. The important operands are astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block; tolist: the value named ’tolist’ in this block. Read together, the complete operation is: This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 299: `currents.extend([0] * (LIF_LANES - element_count))` This calls ‘currents.extend‘ with ‘[0] * (LIF_LANES - element_count)‘ inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 300: `records.append(ProgramRecord(` This statement continues through source line 309; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)‘ inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 301: `lif_command(` This is continuation line 2 of the 10-line logical statement that begins at source line 300. Its exact fragment is ‘lif_command(‘. The important operands are lif_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)‘ inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 302: `leak_shift,` This is continuation line 3 of the 10-line logical statement that begins at source line 300. Its exact fragment is ‘leak_shift,‘. The important operands are leak_shift: the retained fraction of prior membrane state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)‘ inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 303: `threshold_q10,` This is continuation line 4 of the 10-line logical statement that begins at source line 300. Its exact fragment is ‘threshold_q10,‘. The important operands are threshold_q10: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)‘ inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 304: `reset_q10,` This is continuation line 5 of the 10-line logical statement that begins at source line 300. Its exact fragment is ‘reset_q10,‘. The important operands are reset_q10: the value named ’reset q10’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)‘ inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 305: `clear_state=timestep == 0,` This is continuation line 6 of the 10-line logical statement that begins at source line 300. Its exact fragment is ‘clear_state=timestep == 0,‘. The important operands are clear_state: the finite-state-machine phase retained in flip-flops; timestep: one discrete SNN update instant. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)‘ inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 306: `),` This is continuation line 7 of the 10-line logical statement that begins at source line 300. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)‘ inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 307: `pack_lif_beats(currents),` This is continuation line 8 of the 10-line logical statement that begins at source line 300. Its exact fragment is ‘pack_lif_beats(currents),‘. The important operands are pack_lif_beats: the value named ’pack lif beats’ in this block; currents: the value named ’currents’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)‘ inside the function lif_sequence_on_accelerator, which

handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 308: `1,` This is continuation line 9 of the 10-line logical statement that begins at source line 300. Its exact fragment is '1,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)' inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 309: `))` This is continuation line 10 of the 10-line logical statement that begins at source line 300. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0,), pack_lif_beats(currents), 1,)' inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 310: `bindings.append((timestep, element_start, element_count))` This calls 'bindings.append' with '(timestep, element_start, element_count)' inside the function lif_sequence_on_accelerator, which handles lif sequence on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 311: `words = driver.execute_program(records, timeout_seconds=timeout_seconds)` This assigns 'driver.execute_pr timeout_seconds=timeout_seconds)' to 'words ' in host memory. Here words means the value named 'words' in this block. The important operands are driver: the value named 'driver' in this block; execute_program: the value named 'execute program' in this block; records: the number of descriptor records accepted since program start. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 312: `if len(words) != len(records):` This executes the following indented host-code branch only when 'len(words) != len(records)' is true. The important operands are len: the value named 'len' in this block; words: the value named 'words' in this block; records: the number of descriptor records accepted since program start. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 313: `raise RuntimeError(f"LIF program returned {len(words)} beats; expected {len(records)}")` This rejects an invalid state by raising 'RuntimeError(f"LIF program returned {len(words)} beats; expected {len(records)}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 314: `output = np.zeros(flat.shape, dtype=np.uint8)` This assigns 'np.zeros(flat.shape, dtype=np.uint8)' to 'output ' in host memory. Here output means the value named 'output' in this block. The important operands are zeros: the value named 'zeros' in this block; flat: the value named 'flat' in this block; shape: the value named 'shape' in this block. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 315: `for word, (timestep, element_start, element_count) in zip(words, bindings):` This iterates word, (timestep, element_start, element_count) over 'zip(words, bindings)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 316: `spike_word, _ = unpack_lif_result(word)` This assigns 'unpack_lif_result(word)' to 'spike_word, _ ' in host memory. Here spike_word means a binary event produced when a neuron crosses its threshold. The important operands are unpack_lif_result: the value named 'unpack lif result' in this block; word: the value named 'word' in this block. Context: the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 317: `for lane in range(element_count):` This iterates lane over 'range(element_count)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_on_accelerator, which handles lif sequence on accelerator.

Line 318: `output[timestep, element_start + lane] = (spike_word >> lane) & 1` Inside the function lif_sequence_on_a which handles lif sequence on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are output: the value named 'output' in this block; timestep: one discrete SNN update instant; element_start: the value named 'element start' in this block.

Line 319: `return output.reshape(values.shape)` This returns 'output.reshape(values.shape)' to the caller of the function lif_sequence_on_accelerator, which handles lif sequence on accelerator, ending this invocation. The important operands are output: the value named 'output' in this block; reshape: the value named 'reshape' in this block; values: the value named 'values' in this block.

Line 322: `def lif_sequence_with_trace_on_accelerator(` This starts function lif_sequence_with_trace_on_accelerator. Its indented body performs lif sequence with trace on accelerator as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 323: `driver: ProgramExecutor,` Inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are driver: the value named 'driver' in this block; ProgramExecutor: the value named 'program executor' in this block.

Line 324: `values_q10: np.ndarray,` Inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are values_q10: the value named 'values q10' in this block; ndarray: the value named 'ndarray' in this block.

Line 325: `*,` Inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 326: `threshold_q10: int = 1 << 10,` This assigns '1 << 10,' to 'threshold_q10' in host memory. Here threshold_q10 means the membrane level at which a neuron emits a spike. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 327: `leak_shift: int = 1,` This assigns '1,' to 'leak_shift' in host memory. Here leak_shift means the retained fraction of prior membrane state. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 328: `reset_q10: int = 0,` This assigns '0,' to 'reset_q10' in host memory. Here reset_q10 means the value named 'reset q10' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 329: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 330: `) -> tuple[np.ndarray, np.ndarray]:` Inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block.

Line 331: `"""Run LIF in fabric and retain the pre-reset membrane required by BPTT."""` This is executable documentation attached to the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 333: `values = np.asarray(values_q10, dtype=np.int64)` This assigns ‘np.asarray(values_q10, dtype=np.int64)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are asarray: the value named ’asarray’ in this block; values_q10: the value named ’values q10’ in this block; dtype: the value named ’dtype’ in this block. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 334: `if values.ndim < 2 or values.shape[0] == 0:` This executes the following indented host-code branch only when ‘values.ndim < 2 or values.shape[0] == 0‘ is true. The important operands are values: the value named ’values’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 335: `raise ValueError("LIF input must have shape [timesteps, ...]")` This rejects an invalid state by raising ‘ValueError("LIF input must have shape [timesteps, ...]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 336: `if values.size and (values.min() < -(1 << 63) or values.max() > (1 << 63) - 1):` This executes the following indented host-code branch only when ‘values.size and (values.min() < -(1 << 63) or values.max() > (1 << 63) - 1)‘ is true. The important operands are values: the value named ’values’ in this block; size: the value named ’size’ in this block; and: the value named ’and’ in this block. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 337: `raise OverflowError("LIF input exceeds signed 64-bit preparation arithmetic")` This rejects an invalid state by raising ‘OverflowError("LIF input exceeds signed 64-bit preparation arithmetic")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 338: `timesteps = values.shape[0]` This assigns ‘values.shape[0]‘ to ‘timesteps ‘ in host memory. Here timesteps means one discrete SNN update instant. The important operands are values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 339: `flat = values.reshape(timesteps, -1)` This assigns ‘values.reshape(timesteps, -1)‘ to ‘flat ‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are values: the value named ’values’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 340: `records: list[ProgramRecord] = []` This assigns ‘[]‘ to ‘records‘ in host memory. Here records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 341: `bindings: list[tuple[int, int, int]] = []` This assigns ‘[]‘ to ‘bindings‘ in host memory. Here bindings means the value named ’bindings’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 342: `for element_start in range(0, flat.shape[1], LIF_LANES):` This iterates element_start over ‘range(0, flat.shape[1], LIF_LANES)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 343: `element_count = min(LIF_LANES, flat.shape[1] - element_start)` This assigns ‘min(LIF_LANES, flat.shape[1] - element_start)‘ to ‘element_count ‘ in host memory. Here element_count means an integer count retained for control or measurable telemetry. The important operands are min: the value named ’min’ in this block; LIF_LANES: one parallel arithmetic element within a vector or tensor datapath; flat: the value named ’flat’ in this block. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 344: `for timestep in range(timesteps):` This iterates timestep over ‘range(timesteps)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_with_trace_on_ which handles lif sequence with trace on accelerator.

Line 345: `currents = np.clip(` This statement continues through source line 349; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 346: `flat[timestep, element_start:element_start + element_count] >> 1,` This is continuation line 2 of the 5-line logical statement that begins at source line 345. Its exact fragment is ‘flat[timestep, element_start:element_start + element_count] >> 1,‘. The important operands are flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant; element_start: the value named ’element start’ in this block. Read together, the complete operation is: This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 347: `-(1 << 15),` This is continuation line 3 of the 5-line logical statement that begins at source line 345. Its exact fragment is ‘-(1 << 15),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 348: `(1 << 15) - 1,` This is continuation line 4 of the 5-line logical statement that begins at source line 345. Its exact fragment is ‘(1 << 15) - 1,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 349: `).astype(np.int64).tolist()` This is continuation line 5 of the 5-line logical statement that begins at source line 345. Its exact fragment is ‘).astype(np.int64).tolist()‘. The important operands are astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block; tolist: the value named ’tolist’ in this block. Read together, the complete operation is: This assigns ‘np.clip(flat[timestep, element_start:element_start + element_count] >> 1, -(1 << 15), (1 << 15) - 1,).astype(np.int64).tolist()‘ to ‘currents ‘ in host memory. Here currents means the value named ’currents’ in this block. The important operands are clip: the value named ’clip’ in this block; flat: the value named ’flat’ in this block; timestep: one discrete SNN update instant. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 350: `currents.extend([0] * (LIF_LANES - element_count))` This calls ‘currents.extend‘ with ‘[0] * (LIF_LANES - element_count)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 351: `records.append(ProgramRecord(` This statement continues through source line 361; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 352: `lif_command(` This is continuation line 2 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘lif_command(‘. The important operands are lif_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 353: `leak_shift,` This is continuation line 3 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘leak_shift,‘. The important operands are leak_shift: the retained fraction of prior membrane state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 354: `threshold_q10,` This is continuation line 4 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘threshold_q10,‘. The important operands are threshold_q10: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 355: `reset_q10,` This is continuation line 5 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘reset_q10,‘. The important operands are reset_q10: the value named ’reset q10’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 356: `clear_state=timestep == 0,` This is continuation line 6 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘clear_state=timestep == 0,‘. The important operands are clear_state: the finite-state-machine phase retained in flip-flops; timestep: one discrete SNN update instant. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_ which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 357: `trace_state=True,` This is continuation line 7 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘trace_state=True,‘. The important operands are trace_state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 358: `),` This is continuation line 8 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 359: `pack_lif_beats(currents),` This is continuation line 9 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘pack_lif_beats(currents),‘. The important operands are pack_lif_beats: the value named ’pack lif beats’ in this block; currents: the value named ’currents’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_ which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 360: `13,` This is continuation line 10 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘13,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 361: `))` This is continuation line 11 of the 11-line logical statement that begins at source line 351. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(lif_command(leak_shift, threshold_q10, reset_q10, clear_state=timestep == 0, trace_state=True,), pack_lif_beats(currents), 13,)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 362: `bindings.append((timestep, element_start, element_count))` This calls ‘bindings.append‘ with ‘(timestep, element_start, element_count)‘ inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 363: `words = driver.execute_program(records, timeout_seconds=timeout_seconds)` This assigns ‘driver.execute_pr timeout_seconds=timeout_seconds)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 364: `expected_words = 13 * len(records)` This assigns ‘13 * len(records)‘ to ‘expected_words ‘ in host memory. Here expected_words means the value named ’expected words’ in this block. The important operands are len: the value named ’len’ in this block; records: the number of descriptor records accepted since program start. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 365: `if len(words) != expected_words:` This executes the following indented host-code branch only when ‘len(words) != expected_words‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block; expected_words: the value named ’expected words’ in this block. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 366: `raise RuntimeError(` This statement continues through source line 367; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘RuntimeError(f"traced LIF

program returned {len(words)} beats; expected {expected_words}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 367: `f"traced LIF program returned {len(words)} beats; expected {expected_words}")` This is continuation line 2 of the 2-line logical statement that begins at source line 366. Its exact fragment is ‘f"traced LIF program returned {len(words)} beats; expected {expected_words}")‘. The important operands are f: the value named ’f’ in this block; traced: the value named ’traced’ in this block; LIF: the value named ’lif’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"traced LIF program returned {len(words)} beats; expected {expected_words}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 368: `spikes = np.zeros(flat.shape, dtype=np.uint8)` This assigns ‘np.zeros(flat.shape, dtype=np.uint8)‘ to ‘spikes ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are zeros: the value named ’zeros’ in this block; flat: the value named ’flat’ in this block; shape: the value named ’shape’ in this block. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 369: `membrane_trace = np.zeros(flat.shape, dtype=np.int64)` This assigns ‘np.zeros(flat.shape, dtype=np.int64)‘ to ‘membrane_trace ‘ in host memory. Here membrane_trace means the persistent LIF neuron state carrying charge between timesteps. The important operands are zeros: the value named ’zeros’ in this block; flat: the value named ’flat’ in this block; shape: the value named ’shape’ in this block. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 370: `for record_index, (timestep, element_start, element_count) in enumerate(bindings):` This iterates record_index, (timestep, element_start, element_count) over ‘enumerate(bindings)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 371: `start = record_index * 13` This assigns ‘record_index * 13‘ to ‘start ‘ in host memory. Here start means the value named ’start’ in this block. The important operands are record_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 372: `spike_word, _, membranes = unpack_lif_trace(words[start:start + 13])` This assigns ‘unpack_lif_trace(words[start:start + 13])‘ to ‘spike_word, _, membranes ‘ in host memory. Here spike_word means a binary event produced when a neuron crosses its threshold. The important operands are unpack_lif_trace: the value named ’unpack lif trace’ in this block; words: the value named ’words’ in this block; start: the value named ’start’ in this block. Context: the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 373: `for lane in range(element_count):` This iterates lane over ‘range(element_count)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator.

Line 374: `spikes[timestep, element_start + lane] = (spike_word >> lane) & 1` Inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are spikes: a binary event produced when a neuron crosses its threshold; timestep: one discrete SNN update instant; element_start: the value named ’element start’ in this block.

Line 375: `membrane_trace[timestep, element_start + lane] = membranes[lane]` Inside the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are membrane_trace: the persistent LIF neuron state carrying charge between timesteps; timestep: one discrete SNN update instant; element_start: the value named ’element start’ in this block.

Line 376: `return spikes.reshape(values.shape), membrane_trace.reshape(values.shape)` This returns ‘spikes.reshape(v membrane_trace.reshape(values.shape)‘ to the caller of the function lif_sequence_with_trace_on_accelerator, which handles lif sequence with trace on accelerator, ending this invocation. The important operands are spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block; values: the value named ’values’ in this block.

Line 379: `def requantize_projection_reference(` This starts function requantize_projection_reference. Its indented body performs requantize projection reference as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 380: `accumulators: np.ndarray,` Inside the function requantize_projection_reference, which handles requantize projection reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are accumulators: a widened running sum that prevents intermediate overflow; ndarray: the value named ’ndarray’ in this block.

Line 381: `parameters: ProjectionParameters,` Inside the function requantize_projection_reference, which handles requantize projection reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are parameters: the value named ’parameters’ in this block; ProjectionParameters: the value named ’projection parameters’ in this block.

Line 382: `) -> np.ndarray:` Inside the function requantize_projection_reference, which handles requantize projection reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named ’ndarray’ in this block.

Line 383: `"""Apply the exported per-channel Q15 fields exactly like vector operation 9."""` This is executable documentation attached to the function requantize_projection_reference, which handles requantize projection reference. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 385: `values = np.asarray(accumulators, dtype=np.int64)` This assigns ‘np.asarray(accumulators, dtype=np.int64)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are asarray: the value named ’asarray’ in this block; accumulators: a widened running sum that prevents intermediate overflow; dtype: the value named ’dtype’ in this block. Context: the function requantize_projection_reference, which handles requantize projection reference.

Line 386: `if values.ndim != 2 or values.shape[1] != parameters.weight_k_by_n.shape[1]:` This executes the following indented host-code branch only when ‘values.ndim != 2 or values.shape[1] != parameters.weight_k_by_n.shape[1]‘ is true. The important operands are values: the value named ’values’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function requantize_projection_reference, which handles requantize projection reference.

Line 387: `raise ValueError("accumulators must have shape [rows, projection output channels]")` This rejects an invalid state by raising ‘ValueError("accumulators must have shape [rows, projection output channels]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 388: `result = np.empty_like(values)` This assigns ‘np.empty_like(values)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are empty_like: the value named ’empty like’ in this block; values: the value named ’values’ in this block. Context: the function requantize_projection_reference, which handles requantize projection reference.

Line 389: `for channel in range(values.shape[1]):` This iterates channel over ‘range(values.shape[1])‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function requantize_projection_reference, which handles requantize projection reference.

Line 390: `effective_shift = 15 + int(parameters.binary_point_shift[channel])` This assigns '15 + int(parameters.bina to 'effective_shift ' in host memory. Here effective_shift means the value named 'effective shift' in this block. The important operands are parameters: the value named 'parameters' in this block; binary_point_shift: the value named 'binary point shift' in this block; channel: the value named 'channel' in this block. Context: the function requantize_projection_reference, which handles requantize projection reference.

Line 391: `product = values[:, channel] * int(parameters.multiplier_q15[channel])` This assigns 'values[:, channel] * int(parameters.multiplier_q15[channel])' to 'product ' in host memory. Here product means the value named 'product' in this block. The important operands are values: the value named 'values' in this block; channel: the value named 'channel' in this block; parameters: the value named 'parameters' in this block. Context: the function requantize_projection_reference, which handles requantize projection reference.

Line 392: `requantized = round_shift_signed(product, effective_shift)` This assigns 'round_shift_signed(product, effective_shift)' to 'requantized ' in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are round_shift_signed: the value named 'round shift signed' in this block; product: the value named 'product' in this block; effective_shift: the value named 'effective shift' in this block. Context: the function requantize_projection_reference, which handles requantize projection reference.

Line 393: `requantized += int(parameters.bias_q10[channel])` Inside the function requantize_projection_reference, which handles requantize projection reference, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are requantized: the conversion of a wide accumulated result back into a bounded activation code; parameters: the value named 'parameters' in this block; bias_q10: an additive learned offset applied after accumulation.

Line 394: `result[:, channel] = saturate_signed(requantized, 24)` This assigns 'saturate_signed(requantized, 24)' to 'result[' in host memory. Here result means the value named 'result' in this block. The important operands are saturate_signed: the value named 'saturate signed' in this block; requantized: the conversion of a wide accumulated result back into a bounded activation code. Context: the function requantize_projection_reference, which handles requantize projection reference.

Line 395: `return result` This returns 'result' to the caller of the function requantize_projection_reference, which handles requantize projection reference, ending this invocation. The important operands are result: the value named 'result' in this block.

Line 398: `def requantize_projection_on_accelerator(` This starts function requantize_projection_on_accelerator. Its indented body performs requantize projection on accelerator as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 399: `driver: ProgramExecutor,` Inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are driver: the value named 'driver' in this block; ProgramExecutor: the value named 'program executor' in this block.

Line 400: `accumulators: np.ndarray,` Inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are accumulators: a widened running sum that prevents intermediate overflow; ndarray: the value named 'ndarray' in this block.

Line 401: `parameters: ProjectionParameters,` Inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are parameters: the value named 'parameters' in this block; ProjectionParameters: the value named 'projection parameters' in this block.

Line 402: `*,` Inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 403: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 404: `) -> np.ndarray:` Inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named 'ndarray' in this block.

Line 405: `"""Execute per-channel folded BN scaling through the eight-lane vector RTL."""` This is executable documentation attached to the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 407: `values = np.asarray(accumulators, dtype=np.int64)` This assigns 'np.asarray(accumulators, dtype=np.int64)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are asarray: the value named 'asarray' in this block; accumulators: a widened running sum that prevents intermediate overflow; dtype: the value named 'dtype' in this block. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 408: `if values.ndim != 2 or values.shape[1] != parameters.weight_k_by_n.shape[1]:` This executes the following indented host-code branch only when 'values.ndim != 2 or values.shape[1] != parameters.weight_k_by_n.shape[1]' is true. The important operands are values: the value named 'values' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 409: `raise ValueError("accumulators must have shape [rows, projection output channels]")` This rejects an invalid state by raising 'ValueError("accumulators must have shape [rows, projection output channels]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 410: `if values.size and (values.min() < -(1 << 31) or values.max() > (1 << 31) - 1):` This executes the following indented host-code branch only when 'values.size and (values.min() < -(1 << 31) or values.max() > (1 << 31) - 1)' is true. The important operands are values: the value named 'values' in this block; size: the value named 'size' in this block; and: the value named 'and' in this block. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 411: `raise OverflowError("projection accumulators do not fit the vector ALU's signed 32-bit lanes")` This rejects an invalid state by raising 'OverflowError("projection accumulators do not fit the vector ALU's signed 32-bit lanes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 413: `records: list[ProgramRecord] = []` This assigns '[]' to 'records' in host memory. Here records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 414: `bindings: list[tuple[int, int, int]] = []` This assigns '[]' to 'bindings' in host memory. Here bindings means the value named 'bindings' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 415: `zero = [0] * VECTOR_LANES` This assigns '[0] * VECTOR_LANES' to 'zero ' in host memory. Here zero means the value named 'zero' in this block. The important operands are VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 416: `rows, channels = values.shape` This assigns 'values.shape' to 'rows, channels ' in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are values: the value named 'values' in this block; shape: the value named 'shape' in this block. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 417: `for channel in range(channels):` This iterates channel over 'range(channels)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 418: `command = vector_command(` This statement continues through source line 423; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'vector_command(int(VectorOperation.FLEX_REQUA multiplier_q15=int(parameters.multiplier_q15[channel]), right_shift=int(parameters.binary_point_shift[channel]), output_zero_point=int(parameters.bias_q10[channel]),)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 419: `int(VectorOperation.FLEX_REQUANTIZE),` This is continuation line 2 of the 6-line logical statement that begins at source line 418. Its exact fragment is 'int(VectorOperation.FLEX_REQUANTIZE),'. The important operands are VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=int(parameters.multiplier_q15[channel]), right_shift=int(parameters.binary_point_shift[channel]), output_zero_point=int(parameters.bias_q10[channel]),)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 420: `multiplier_q15=int(parameters.multiplier_q15[channel]),` This is continuation line 3 of the 6-line logical statement that begins at source line 418. Its exact fragment is 'multiplier_q15=int(parameters.multiplier_q15[channel]),'. The important operands are multiplier_q15: the value named 'multiplier q15' in this block; parameters: the value named 'parameters' in this block; channel: the value named 'channel' in this block. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=int(parameters.multiplier_q15[channel]), right_shift=int(parameters.binary_point_shift[channel]), output_zero_point=int(parameters.bias_q10[channel]),)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 421: `right_shift=int(parameters.binary_point_shift[channel]),` This is continuation line 4 of the 6-line logical statement that begins at source line 418. Its exact fragment is 'right_shift=int(parameters.binary_point_shift[channel]),'. The important operands are right_shift: the value named 'right shift' in this block; parameters: the value named 'parameters' in this block; binary_point_shift: the value named 'binary point shift' in this block. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=int(parameters.multiplier right_shift=int(parameters.binary_point_shift[channel]), output_zero_point=int(parameters.bias_q10[channel]),)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode,

and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 422: `output_zero_point=int(parameters.bias_q10[channel]),` This is continuation line 5 of the 6-line logical statement that begins at source line 418. Its exact fragment is 'output_zero_point=int(parameters.bias_q10[channel]),'. The important operands are output_zero_point: the integer code representing real numeric zero in affine quantization; parameters: the value named 'parameters' in this block; bias_q10: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=int(parameters.multiplier_q15[channel]), right_shift=int(parameters.binary_point_shift[channel]), output_zero_point=int(parameters.bias_q10[channel]),)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 423: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 418. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=int(parameters.multiplier_q15[channel]), right_shift=int(parameters.binary_point_shift[channel]), output_zero_point)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 424: `for row_start in range(0, rows, VECTOR_LANES):` This iterates row_start over 'range(0, rows, VECTOR_LANES)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 425: `row_count = min(VECTOR_LANES, rows - row_start)` This assigns 'min(VECTOR_LANES, rows - row_start)' to 'row_count ' in host memory. Here row_count means one output-row position processed by the tensor tile. The important operands are min: the value named 'min' in this block; VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath; rows: one output-row position processed by the tensor tile. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 426: `lanes = values[row_start:row_start + row_count, channel].tolist()` This assigns 'values[row_start:row_start + row_count, channel].tolist()' to 'lanes ' in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are values: the value named 'values' in this block; row_start: one output-row position processed by the tensor tile; row_count: one output-row position processed by the tensor tile. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 427: `lanes.extend([0] * (VECTOR_LANES - row_count))` This calls 'lanes.extend' with '[0] * (VECTOR_LANES - row_count)' inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 428: `records.append(ProgramRecord(` This statement continues through source line 437; the explanation uses the complete expression rather than only the first visual fragment. This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)' inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 429: `command,` This is continuation line 2 of the 10-line logical statement that begins at source line 428. Its exact fragment is ‘command,‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 430: `pack_vector_beats(` This is continuation line 3 of the 10-line logical statement that begins at source line 428. Its exact fragment is ‘pack_vector_beats(‘. The important operands are pack_vector_beats: the value named ’pack vector beats’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 431: `lanes,` This is continuation line 4 of the 10-line logical statement that begins at source line 428. Its exact fragment is ‘lanes,‘. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 432: `zero,` This is continuation line 5 of the 10-line logical statement that begins at source line 428. Its exact fragment is ‘zero,‘. The important operands are zero: the value named ’zero’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 433: `clamp_minimum=-(1 << 23),` This is continuation line 6 of the 10-line logical statement that begins at source line 428. Its exact fragment is ‘clamp_minimum=-(1 << 23),‘. The important operands are clamp_minimum: the value named ’clamp minimum’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 434: `clamp_maximum=(1 << 23) - 1,` This is continuation line 7 of the 10-line logical statement that begins at source line 428. Its exact fragment is ‘clamp_maximum=(1 << 23) - 1,‘. The important operands are clamp_maximum: the value named ’clamp maximum’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 435: `),` This is continuation line 8 of the 10-line logical statement that begins at source line 428. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 436: `4,` This is continuation line 9 of the 10-line logical statement that begins at source line 428. Its exact fragment is ‘4,‘. The expression is a literal or structural operator and has no hidden software state. Read to-

gether, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 437: `))` This is continuation line 10 of the 10-line logical statement that begins at source line 428. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(lanes, zero, clamp_minimum=-(1 << 23), clamp_maximum=(1 << 23) - 1,), 4,)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 438: `bindings.append((row_start, row_count, channel))` This calls ‘bindings.append‘ with ‘(row_start, row_count, channel)‘ inside the function requantize_projection_on_accelerator, which handles requantize projection on accelerator. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 440: `words = driver.execute_program(records, timeout_seconds=timeout_seconds)` This assigns ‘driver.execute_pr timeout_seconds=timeout_seconds)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 441: `if len(words) != 4 * len(records):` This executes the following indented host-code branch only when ‘len(words) != 4 * len(records)‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block; records: the number of descriptor records accepted since program start. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 442: `raise RuntimeError(` This statement continues through source line 443; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘RuntimeError(f"requantizer returned {len(words)} beats; expected {4 * len(records)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 443: `f"requantizer returned {len(words)} beats; expected {4 * len(records)}")` This is continuation line 2 of the 2-line logical statement that begins at source line 442. Its exact fragment is ‘f"requantizer returned {len(words)} beats; expected {4 * len(records)}")‘. The important operands are f: the value named ’f’ in this block; requantizer: the conversion of a wide accumulated result back into a bounded activation code; returned: the value named ’returned’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"requantizer returned {len(words)} beats; expected {4 * len(records)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 444: `result = np.empty((rows, channels), dtype=np.int64)` This assigns ‘np.empty((rows, channels), dtype=np.int64 to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are empty: the value named ’empty’ in this block; rows: one output-row position processed by the tensor tile; channels: the value named ’channels’ in this block. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 445: `for record_index, (row_start, row_count, channel) in enumerate(bindings):` This iterates record_index, (row_start, row_count, channel) over ‘enumerate(bindings)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 446: `lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])` This assigns ‘unpack_vector_result(words[4 * record_index:4 * record_index + 4])‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are unpack_vector_result: the

value named 'unpack vector result' in this block; words: the value named 'words' in this block; record_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 447: `result[row_start:row_start + row_count, channel] = lanes[:row_count]` This assigns 'lanes[:row_count]' to 'result[row_start' in host memory. Here result means the value named 'result' in this block. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath; row_count: one output-row position processed by the tensor tile. Context: the function requantize_projection_on_accelerator, which handles requantize projection on accelerator.

Line 448: `return result` This returns 'result' to the caller of the function requantize_projection_on_accelerator, which handles requantize projection on accelerator, ending this invocation. The important operands are result: the value named 'result' in this block.

Line 451: `@dataclass` This decorator changes how the following class or function is constructed. '@dataclass' is applied at import time on the host and is part of the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it creates no FPGA circuit by itself.

Line 452: `class SpikingformerProjectionExecutor:` This defines class SpikingformerProjectionExecutor, which groups data and methods for spikingformer projection executor. Instances exist in host memory and organize the flow that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 453: `"""Run exported Conv-BN projections through tensor and vector hardware."""` This is executable documentation attached to the class SpikingformerProjectionExecutor, which handles spikingformer projection executor. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 455: `archive: SpikingformerParameterArchive` Inside the class SpikingformerProjectionExecutor, which handles spikingformer projection executor, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are archive: the value named 'archive' in this block; SpikingformerParameterArchive: the value named 'spikingformer parameter archive' in this block.

Line 456: `driver: ProgramExecutor` Inside the class SpikingformerProjectionExecutor, which handles spikingformer projection executor, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are driver: the value named 'driver' in this block; ProgramExecutor: the value named 'program executor' in this block.

Line 458: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class SpikingformerProjectionExecutor, which handles spikingformer projection executor; it creates no FPGA circuit by itself.

Line 459: `def _encoded_dense_weights(weight: np.ndarray) -> np.ndarray:` This starts function _encoded_dense_weig
Its indented body performs encoded dense weights as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 460: `return (weight.astype(np.int16) + 128).astype(np.uint8)` This returns '(weight.astype(np.int16) + 128).astype(np.uint8)' to the caller of the function _encoded_dense_weights, which handles encoded dense weights, ending this invocation. The important operands are weight: a learned coefficient multiplied by an activation; astype: the value named 'astype' in this block; int16: the value named 'int16' in this block.

Line 462: `def matrix(` This starts function matrix. Its indented body performs matrix as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 463: `self,` Inside the function matrix, which handles matrix, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 464: `name: str,` Inside the function matrix, which handles matrix, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are name: the value named 'name' in this block; str: the value named 'str' in this block.

Line 465: `activations: np.ndarray,` Inside the function matrix, which handles matrix, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named 'ndarray' in this block.

Line 466: `*,` Inside the function matrix, which handles matrix, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 467: `activation_zero_point: int = 0,` This assigns '0,' to 'activation_zero_point' in host memory. Here activation_zero_point means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the function matrix, which handles matrix.

Line 468: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function matrix, which handles matrix.

Line 469: `) -> np.ndarray:` Inside the function matrix, which handles matrix, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named 'ndarray' in this block.

Line 470: `"""Execute one exported Conv1d-style projection as a two-dimensional matrix."""` This is executable documentation attached to the function matrix, which handles matrix. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 472: `parameters = self.archive.projection(name)` This assigns 'self.archive.projection(name)' to 'parameters ' in host memory. Here parameters means the value named 'parameters' in this block. The important operands are archive: the value named 'archive' in this block; projection: the value named 'projection' in this block; name: the value named 'name' in this block. Context: the function matrix, which handles matrix.

Line 473: `values = np.asarray(activations)` This assigns 'np.asarray(activations)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are asarray: the value named 'asarray' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function matrix, which handles matrix.

Line 474: `if values.ndim != 2 or values.shape[1] != parameters.weight_k_by_n.shape[0]:` This executes the following indented host-code branch only when 'values.ndim != 2 or values.shape[1] != parameters.weight_k_by_n.shape[0]' is true. The important operands are values: the value named 'values' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function matrix, which handles matrix.

Line 475: `raise ValueError("matrix activations do not match the projection reduction dimension")` This rejects an invalid state by raising 'ValueError("matrix activations do not match the projection reduction dimension")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 476: `if parameters.compute_opcode == "COMPUTE_BINARY":` This executes the following indented host-code branch only when ‘parameters.compute_opcode == "COMPUTE_BINARY"‘ is true. The important operands are parameters: the value named ’parameters’ in this block; compute_opcode: the value named ’compute opcode’ in this block; COMPUTE_BINARY: the value named ’compute binary’ in this block. Context: the function matrix, which handles matrix.

Line 477: `compiled = compile_spike_gemm(values, parameters.weight_k_by_n)` This assigns ‘compile_spike_gemm(value parameters.weight_k_by_n)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold; values: the value named ’values’ in this block; parameters: the value named ’parameters’ in this block. Context: the function matrix, which handles matrix.

Line 478: `elif parameters.compute_opcode == "COMPUTE_DENSE_OUTER":` This executes the following indented host-code branch only when ‘parameters.compute_opcode == "COMPUTE_DENSE_OUTER"‘ is true. The important operands are parameters: the value named ’parameters’ in this block; compute_opcode: the value named ’compute opcode’ in this block; COMPUTE_DENSE_OUTER: the value named ’compute dense outer’ in this block. Context: the function matrix, which handles matrix.

Line 479: `compiled = compile_u8_gemm(` This statement continues through source line 484; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_u8_gemm(values, self._encoded_dense_weigh activation_zero_point=activation_zero_point, weight_zero_point=128,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function matrix, which handles matrix.

Line 480: `values,` This is continuation line 2 of the 6-line logical statement that begins at source line 479. Its exact fragment is ‘values,‘. The important operands are values: the value named ’values’ in this block. Read together, the complete operation is: This assigns ‘compile_u8_gemm(values, self._encoded_dense_weights(parameters.weight_k_by_n), activation_zero_point=activation_zero_point, weight_zero_point=128,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function matrix, which handles matrix.

Line 481: `self._encoded_dense_weights(parameters.weight_k_by_n),` This is continuation line 3 of the 6-line logical statement that begins at source line 479. Its exact fragment is ‘self._encoded_dense_weights(parameters.weight_k_by_n),‘. The important operands are _encoded_dense_weights: a learned coefficient multiplied by an activation; parameters: the value named ’parameters’ in this block; weight_k_by_n: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘compile_u8_gemm(values, self._encoded_dense_weights(parameters.weight_k_by_n), activation_zero_point=activation_zero_point, weight_zero_point=128,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function matrix, which handles matrix.

Line 482: `activation_zero_point=activation_zero_point,` This is continuation line 4 of the 6-line logical statement that begins at source line 479. Its exact fragment is ‘activation_zero_point=activation_zero_point,‘. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘compile_u8_gemm(values, self._encoded_dense_weights(parameters.weigh activation_zero_point=activation_zero_point, weight_zero_point=128,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function matrix, which handles matrix.

Line 483: `weight_zero_point=128,` This is continuation line 5 of the 6-line logical statement that begins at source line 479. Its exact fragment is ‘weight_zero_point=128,‘. The important operands are weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘compile_u8_gemm(values, self._encoded_dense_weights(parameters.weight_k_by_n), activation_zero_point=activation_zero_point, weight_zero_point=

)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function matrix, which handles matrix.

Line 484: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 479. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘compile_u8_gemm(values, self._encoded_dense_weights(parameters.weight_k_by_n), activation_zero_point=activation_zero_point, weight_zero_point=128,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function matrix, which handles matrix.

Line 485: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function matrix, which handles matrix.

Line 486: `raise ValueError(f"unsupported projection opcode {parameters.compute_opcode!r}")` This rejects an invalid state by raising ‘ValueError(f"unsupported projection opcode {parameters.compute_opcode!r}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 487: `accumulators = compiled.execute_batched(` This statement continues through source line 488; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compiled.execute_batched(self.driver, timeout_seconds=timeout_seconds)‘ to ‘accumulators ‘ in host memory. Here accumulators means a widened running sum that prevents intermediate overflow. The important operands are compiled: the value named ’compiled’ in this block; execute_batched: the value named ’execute batched’ in this block; driver: the value named ’driver’ in this block. Context: the function matrix, which handles matrix.

Line 488: `self.driver, timeout_seconds=timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 487. Its exact fragment is ‘self.driver, timeout_seconds=timeout_seconds)‘. The important operands are driver: the value named ’driver’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘compiled.execute_batched(self.driver, timeout_seconds=timeout_seconds)‘ to ‘accumulators ‘ in host memory. Here accumulators means a widened running sum that prevents intermediate overflow. The important operands are compiled: the value named ’compiled’ in this block; execute_batched: the value named ’execute batched’ in this block; driver: the value named ’driver’ in this block. Context: the function matrix, which handles matrix.

Line 489: `return requantize_projection_on_accelerator(` This statement continues through source line 490; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘requantize_projection_on_accelera
self.driver, accumulators, parameters, timeout_seconds=timeout_seconds)‘ to the caller of the function matrix, which handles matrix, ending this invocation. The important operands are requantize_projection_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named ’driver’ in this block; accumulators: a widened running sum that prevents intermediate overflow.

Line 490: `self.driver, accumulators, parameters, timeout_seconds=timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 489. Its exact fragment is ‘self.driver, accumulators, parameters, timeout_seconds=timeout_seconds)‘. The important operands are driver: the value named ’driver’ in this block; accumulators: a widened running sum that prevents intermediate overflow; parameters: the value named ’parameters’ in this block. Read together, the complete operation is: This returns ‘requantize_projection_on_accelerator(self.driver, accumulators, parameters, timeout_seconds=timeout_seconds)‘ to the caller of the function matrix, which handles matrix, ending this invocation. The important operands are requantize_projection_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named ’driver’ in this block; accumulators: a widened running sum that prevents intermediate overflow.

Line 492: `def conv2d(` This starts function conv2d. Its indented body performs conv2d as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 493: `self,` Inside the function conv2d, which handles conv2d, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 494: `name: str,` Inside the function conv2d, which handles conv2d, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are name: the value named 'name' in this block; str: the value named 'str' in this block.

Line 495: `activations: np.ndarray,` Inside the function conv2d, which handles conv2d, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named 'ndarray' in this block.

Line 496: `*,` Inside the function conv2d, which handles conv2d, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 497: `activation_zero_point: int = 0,` This assigns '0,' to 'activation_zero_point' in host memory. Here activation_zero_point means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the function conv2d, which handles conv2d.

Line 498: `stride: int | Sequence[int] = 1,` This assigns '1,' to 'stride' in host memory. Here stride means the value named 'stride' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function conv2d, which handles conv2d.

Line 499: `padding: int | Sequence[int] = 0,` This assigns '0,' to 'padding' in host memory. Here padding means the value named 'padding' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function conv2d, which handles conv2d.

Line 500: `dilation: int | Sequence[int] = 1,` This assigns '1,' to 'dilation' in host memory. Here dilation means the value named 'dilation' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function conv2d, which handles conv2d.

Line 501: `groups: int = 1,` This assigns '1,' to 'groups' in host memory. Here groups means the value named 'groups' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function conv2d, which handles conv2d.

Line 502: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function conv2d, which handles conv2d.

Line 503: `) -> np.ndarray:` Inside the function conv2d, which handles conv2d, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are ndarray: the value named 'ndarray' in this block.

Line 504: `"""Execute one exported Conv2d-BN pair and return signed Q10 NCHW data."""` This is executable documentation attached to the function conv2d, which handles conv2d. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 506: `parameters = self.archive.projection(name)` This assigns 'self.archive.projection(name)' to 'parameters ' in host memory. Here parameters means the value named 'parameters' in this block. The important operands are archive: the value named 'archive' in this block; projection: the value named 'projection' in this block; name: the value named 'name' in this block. Context: the function conv2d, which handles conv2d.

Line 507: `if len(parameters.source_weight_shape) != 4:` This executes the following indented host-code branch only when ‘len(parameters.source_weight_shape) != 4‘ is true. The important operands are len: the value named ’len’ in this block; parameters: the value named ’parameters’ in this block; source_weight_shape: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 508: `raise ValueError(f"projection {name} is not Conv2d")` This rejects an invalid state by raising ‘ValueError(f"projection {name} is not Conv2d")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 509: `weight = parameters.weight_k_by_n.T.reshape(parameters.source_weight_shape)` This assigns ‘parameters.weight_k_by_n.T.reshape(parameters.source_weight_shape)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are parameters: the value named ’parameters’ in this block; weight_k_by_n: a learned coefficient multiplied by an activation; T: the value named ’t’ in this block. Context: the function conv2d, which handles conv2d.

Line 510: `values = np.asarray(activations)` This assigns ‘np.asarray(activations)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are asarray: the value named ’asarray’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function conv2d, which handles conv2d.

Line 511: `if parameters.compute_opcode == "COMPUTE_BINARY":` This executes the following indented host-code branch only when ‘parameters.compute_opcode == "COMPUTE_BINARY"‘ is true. The important operands are parameters: the value named ’parameters’ in this block; compute_opcode: the value named ’compute opcode’ in this block; COMPUTE_BINARY: the value named ’compute binary’ in this block. Context: the function conv2d, which handles conv2d.

Line 512: `compiled = compile_spike_conv2d(` This statement continues through source line 514; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_spike_conv2d(values, weight, stride=stride, padding=padding, dilation=dilation, groups=groups)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_spike_conv2d: a binary event produced when a neuron crosses its threshold; values: the value named ’values’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 513: `values, weight, stride=stride, padding=padding,` This is continuation line 2 of the 3-line logical statement that begins at source line 512. Its exact fragment is ‘values, weight, stride=stride, padding=padding,‘. The important operands are values: the value named ’values’ in this block; weight: a learned coefficient multiplied by an activation; stride: the value named ’stride’ in this block. Read together, the complete operation is: This assigns ‘compile_spike_conv2d(values, weight, stride=stride, padding=padding, dilation=dilation, groups=groups)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_spike_conv2d: a binary event produced when a neuron crosses its threshold; values: the value named ’values’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 514: `dilation=dilation, groups=groups)` This is continuation line 3 of the 3-line logical statement that begins at source line 512. Its exact fragment is ‘dilation=dilation, groups=groups)‘. The important operands are dilation: the value named ’dilation’ in this block; groups: the value named ’groups’ in this block. Read together, the complete operation is: This assigns ‘compile_spike_conv2d(values, weight, stride=stride, padding=padding, dilation=dilation, groups=groups)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_spike_conv2d: a binary event produced when a neuron crosses its threshold; values: the value named ’values’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 515: `elif parameters.compute_opcode == "COMPUTE_DENSE_OUTER":` This executes the following indented host-code branch only when ‘parameters.compute_opcode == "COMPUTE_DENSE_OUTER"‘ is true. The important operands are parameters: the value named ’parameters’ in this block; compute_opcode: the value named ’compute opcode’ in this block; COMPUTE_DENSE_OUTER: the value named ’compute dense outer’ in this block. Context: the function conv2d, which handles conv2d.

Line 516: `compiled = compile_u8_conv2d(` This statement continues through source line 525; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 517: `values,` This is continuation line 2 of the 10-line logical statement that begins at source line 516. Its exact fragment is ‘values,‘. The important operands are values: the value named ’values’ in this block. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 518: `self._encoded_dense_weights(weight),` This is continuation line 3 of the 10-line logical statement that begins at source line 516. Its exact fragment is ‘self._encoded_dense_weights(weight),‘. The important operands are _encoded_dense_weights: a learned coefficient multiplied by an activation; weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 519: `activation_zero_point=activation_zero_point,` This is continuation line 4 of the 10-line logical statement that begins at source line 516. Its exact fragment is ‘activation_zero_point=activation_zero_point,‘. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 520: `weight_zero_point=128,` This is continuation line 5 of the 10-line logical statement that begins at source line 516. Its exact fragment is ‘weight_zero_point=128,‘. The important operands are weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 521: `stride=stride,` This is continuation line 6 of the 10-line logical statement that begins at source line 516. Its exact fragment is ‘stride=stride,‘. The important operands are stride: the value named ’stride’ in this block. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)‘ to ‘compiled ‘ in host memory. Here compiled means the value named ’compiled’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; values: the value named ’values’ in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 522: `padding=padding,` This is continuation line 7 of the 10-line logical statement that begins at source line 516. Its exact fragment is 'padding=padding,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)' to 'compiled ' in host memory. Here compiled means the value named 'compiled' in this block. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block; values: the value named 'values' in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 523: `dilation=dilation,` This is continuation line 8 of the 10-line logical statement that begins at source line 516. Its exact fragment is 'dilation=dilation,'. The important operands are dilation: the value named 'dilation' in this block. Read together, the complete operation is: This assigns 'compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)' to 'compiled ' in host memory. Here compiled means the value named 'compiled' in this block. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block; values: the value named 'values' in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 524: `groups=groups,` This is continuation line 9 of the 10-line logical statement that begins at source line 516. Its exact fragment is 'groups=groups,'. The important operands are groups: the value named 'groups' in this block. Read together, the complete operation is: This assigns 'compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)' to 'compiled ' in host memory. Here compiled means the value named 'compiled' in this block. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block; values: the value named 'values' in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 525: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 516. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'compile_u8_conv2d(values, self._encoded_dense_weights(weight), activation_zero_point=activation_zero_point, weight_zero_point=128, stride=stride, padding=padding, dilation=dilation, groups=groups,)' to 'compiled ' in host memory. Here compiled means the value named 'compiled' in this block. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block; values: the value named 'values' in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation. Context: the function conv2d, which handles conv2d.

Line 526: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function conv2d, which handles conv2d.

Line 527: `raise ValueError(f"unsupported projection opcode {parameters.compute_opcode!r}")` This rejects an invalid state by raising 'ValueError(f"unsupported projection opcode {parameters.compute_opcode!r}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 528: `accumulator_nchw = compiled.execute_batched(` This statement continues through source line 529; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'compiled.execute_batched(self.driver, timeout_seconds=timeout_seconds)' to 'accumulator_nchw ' in host memory. Here accumulator_nchw means a widened running sum that prevents intermediate overflow. The important operands are compiled: the value named 'compiled' in this block; execute_batched: the value named 'execute batched' in this block; driver: the value named 'driver' in this block. Context: the function conv2d, which handles conv2d.

Line 529: `self.driver, timeout_seconds=timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 528. Its exact fragment is 'self.driver, timeout_seconds=timeout_seconds)'. The important operands are driver: the value named 'driver' in this block; timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'compiled.execute_batched(self.driver, timeout_seconds=timeout_seconds)' to 'accumulator_nchw ' in host memory. Here accumulator_nchw means a widened running sum that prevents intermediate overflow. The important operands are compiled: the value named 'compiled' in

this block; execute_batched: the value named 'execute batched' in this block; driver: the value named 'driver' in this block. Context: the function conv2d, which handles conv2d.

Line 530: `batch, channels, height, width = accumulator_nchw.shape` This assigns 'accumulator_nchw.shape' to 'batch, channels, height, width ' in host memory. Here batch means the value named 'batch' in this block. The important operands are accumulator_nchw: a widened running sum that prevents intermediate overflow; shape: the value named 'shape' in this block. Context: the function conv2d, which handles conv2d.

Line 531: `matrix = accumulator_nchw.transpose(0, 2, 3, 1).reshape(-1, channels)` This assigns 'accumulator_nchw.transpose(0, 2, 3, 1).reshape(-1, channels)' to 'matrix ' in host memory. Here matrix means the value named 'matrix' in this block. The important operands are accumulator_nchw: a widened running sum that prevents intermediate overflow; transpose: the value named 'transpose' in this block; reshape: the value named 'reshape' in this block. Context: the function conv2d, which handles conv2d.

Line 532: `quantized = requantize_projection_on_accelerator(` This statement continues through source line 533; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'requantize_projection_on_accelerator(self.driver, matrix, parameters, timeout_seconds=timeout_seconds)' to 'quantized ' in host memory. Here quantized means the value named 'quantized' in this block. The important operands are requantize_projection_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; matrix: the value named 'matrix' in this block. Context: the function conv2d, which handles conv2d.

Line 533: `self.driver, matrix, parameters, timeout_seconds=timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 532. Its exact fragment is 'self.driver, matrix, parameters, timeout_seconds=timeout_seconds)'. The important operands are driver: the value named 'driver' in this block; matrix: the value named 'matrix' in this block; parameters: the value named 'parameters' in this block. Read together, the complete operation is: This assigns 'requantize_projection_on_accelerator(self.driver, matrix, parameters, timeout_seconds=timeout_seconds)' to 'quantized ' in host memory. Here quantized means the value named 'quantized' in this block. The important operands are requantize_projection_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; matrix: the value named 'matrix' in this block. Context: the function conv2d, which handles conv2d.

Line 534: `return quantized.reshape(batch, height, width, channels).transpose(0, 3, 1, 2)` This returns 'quantized.reshape(batch, height, width, channels).transpose(0, 3, 1, 2)' to the caller of the function conv2d, which handles conv2d, ending this invocation. The important operands are quantized: the value named 'quantized' in this block; reshape: the value named 'reshape' in this block; batch: the value named 'batch' in this block.

Line 537: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it creates no FPGA circuit by itself.

Line 538: `class SpikingformerInferenceResult:` This defines class SpikingformerInferenceResult, which groups data and methods for spikingformer inference result. Instances exist in host memory and organize the flow that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 539: `"""Classifier result in both exact accumulator and represented float units."""` This is executable documentation attached to the class SpikingformerInferenceResult, which handles spikingformer inference result. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 541: `logits_accumulator: np.ndarray` Inside the class SpikingformerInferenceResult, which handles spikingformer inference result, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are logits_accumulator: a widened running sum that prevents intermediate overflow; ndarray: the value named 'ndarray' in this block.

Line 542: `logits: np.ndarray` Inside the class SpikingformerInferenceResult, which handles spikingformer inference result, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are logits: the ten unnormalized classifier scores used for argmax; ndarray: the value named 'ndarray' in this block.

Line 543: `predicted_class: np.ndarray` Inside the class SpikingformerInferenceResult, which handles spikingformer inference result, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are predicted_class: the value named 'predicted class' in this block; ndarray: the value named 'ndarray' in this block.

Line 546: `class SpikingformerAcceleratorModel:` This defines class SpikingformerAcceleratorModel, which groups data and methods for spikingformer accelerator model. Instances exist in host memory and organize the flow that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 547: `"""Full trained Spikingformer-4-384 inference graph on NeuraEdge primitives."""` This is executable documentation attached to the class SpikingformerAcceleratorModel, which handles spikingformer accelerator model. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 549: `def __init__(` This starts function ___init___. Its indented body performs init as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 550: `self,` Inside the function ___init___, which handles init, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 551: `archive: SpikingformerParameterArchive,` Inside the function ___init___, which handles init, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are archive: the value named 'archive' in this block; SpikingformerParameterArchive: the value named 'spikingformer parameter archive' in this block.

Line 552: `driver: ProgramExecutor,` Inside the function ___init___, which handles init, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are driver: the value named 'driver' in this block; ProgramExecutor: the value named 'program executor' in this block.

Line 553: `*,` Inside the function ___init___, which handles init, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The expression is a literal or structural operator and has no hidden software state.

Line 554: `layer_timeout_seconds: float = 120.0,` This assigns '120.0,' to 'layer_timeout_seconds' in host memory. Here layer_timeout_seconds means the value named 'layer timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 555: `) -> None:` Inside the function ___init___, which handles init, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are None: the value named 'none' in this block.

Line 556: `if layer_timeout_seconds <= 0:` This executes the following indented host-code branch only when 'layer_timeout_seconds <= 0' is true. The important operands are layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Context: the function ___init___, which handles init.

Line 557: `raise ValueError("layer_timeout_seconds must be positive")` This rejects an invalid state by raising ‘ValueError("layer_timeout_seconds must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 558: `self.archive = archive` This assigns ‘archive‘ to ‘self.archive ‘ in host memory. Here archive means the value named ’archive’ in this block. The important operands are archive: the value named ’archive’ in this block. Context: the function ___init___, which handles init.

Line 559: `self.driver = driver` This assigns ‘driver‘ to ‘self.driver ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are driver: the value named ’driver’ in this block. Context: the function ___init___, which handles init.

Line 560: `self.projections = SpikingformerProjectionExecutor(archive, driver)` This assigns ‘SpikingformerProjectionExecutor(archive, driver)‘ to ‘self.projections ‘ in host memory. Here projections means the value named ’projections’ in this block. The important operands are SpikingformerProjectionExecutor: the value named ’spikingformer projection executor’ in this block; archive: the value named ’archive’ in this block; driver: the value named ’driver’ in this block. Context: the function ___init___, which handles init.

Line 561: `self.layer_timeout_seconds = layer_timeout_seconds` This assigns ‘layer_timeout_seconds‘ to ‘self.layer_timeo‘ in host memory. Here layer_timeout_seconds means the value named ’layer timeout seconds’ in this block. The important operands are layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Context: the function ___init___, which handles init.

Line 562: `fixed = archive.manifest.get("fixed_point")` This assigns ‘archive.manifest.get("fixed_point")‘ to ‘fixed ‘ in host memory. Here fixed means the value named ’fixed’ in this block. The important operands are archive: the value named ’archive’ in this block; manifest: the value named ’manifest’ in this block; get: the value named ’get’ in this block. Context: the function ___init___, which handles init.

Line 563: `if not isinstance(fixed, dict):` This executes the following indented host-code branch only when ‘not isinstance(fixed, dict)‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; fixed: the value named ’fixed’ in this block. Context: the function ___init___, which handles init.

Line 564: `raise ValueError("parameter archive has no fixed-point contract")` This rejects an invalid state by raising ‘ValueError("parameter archive has no fixed-point contract")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 565: `self.fractional_bits = int(fixed["activation_fractional_bits"])` This assigns ‘int(fixed["activation_fractio‘ to ‘self.fractional_bits ‘ in host memory. Here fractional_bits means the value named ’fractional bits’ in this block. The important operands are fixed: the value named ’fixed’ in this block; activation_fractional_bits: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function ___init___, which handles init.

Line 566: `self.fixed_scale = 1 << self.fractional_bits` This assigns ‘1 << self.fractional_bits‘ to ‘self.fixed_scale ‘ in host memory. Here fixed_scale means the real-value spacing represented by one quantized integer step. The important operands are fractional_bits: the value named ’fractional bits’ in this block. Context: the function ___init___, which handles init.

Line 567: `self.image_scale = float(fixed["image_input_scale"])` This assigns ‘float(fixed["image_input_scale"])‘ to ‘self.image_scale ‘ in host memory. Here image_scale means the real-value spacing represented by one quantized integer step. The important operands are float: the value named ’float’ in this block; fixed: the value named ’fixed’ in this block; image_input_scale: the real-value spacing represented by one quantized integer step. Context: the function ___init___, which handles init.

Line 568: `self.image_zero_point = int(fixed["image_input_zero_point"])` This assigns ‘int(fixed["image_input_zero_p to ‘self.image_zero_point ‘ in host memory. Here image_zero_point means the integer code representing real numeric zero in affine quantization. The important operands are fixed: the value named ’fixed’ in this block; image_input_zero_point: the integer code representing real numeric zero in affine quantization. Context: the function ___init___, which handles init.

Line 569: `self.head_activation_scale = float(fixed["head_activation_scale"])` This assigns ‘float(fixed["head_activat to ‘self.head_activation_scale ‘ in host memory. Here head_activation_scale means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are float: the value named ’float’ in this block; fixed: the value named ’fixed’ in this block; head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function ___init___, which handles init.

Line 570: `required = {` This statement continues through source line 576; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(4) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(4) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 571: `*(f"patch_embed.block{index}" for index in range(5)),` This is continuation line 2 of the 7-line logical statement that begins at source line 570. Its exact fragment is ‘*(f"patch_embed.block{index}" for index in range(5)),‘. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(4) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(4) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 572: `*(f"block.{block}.attn.{name}"` This is continuation line 3 of the 7-line logical statement that begins at source line 570. Its exact fragment is ‘*(f"block.{block}.attn.{name}"‘. The important operands are f: the value named ’f’ in this block; block: the value named ’block’ in this block; attn: the value named ’attn’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(4) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(4) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 573: `for block in range(4) for name in ("q", "k", "v", "proj")),` This is continuation line 4 of the 7-line logical statement that begins at source line 570. Its exact fragment is ‘for block in range(4) for name in ("q", "k", "v", "proj")),‘. The important operands are for: the value named ’for’ in this block; block: the value named ’block’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(4) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(4) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 574: `*(f"block.{block}.mlp.fc{layer}"` This is continuation line 5 of the 7-line logical statement that begins at source line 570. Its exact fragment is ‘*(f"block.{block}.mlp.fc{layer}"‘. The important operands are f: the value named ’f’ in this block; block: the value named ’block’ in this block; mlp: the value named ’mlp’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(4) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(4) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 575: `for block in range(4) for layer in (1, 2)),` This is continuation line 6 of the 7-line logical statement that begins at source line 570. Its exact fragment is 'for block in range(4) for layer in (1, 2)),'. The important operands are for: the value named 'for' in this block; block: the value named 'block' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns '{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(4) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(4) for layer in (1, 2)), }' to 'required ' in host memory. Here required means the value named 'required' in this block. The important operands are f: the value named 'f' in this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this block. Context: the function ___init___, which handles init.

Line 576: `}` This is continuation line 7 of the 7-line logical statement that begins at source line 570. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(4) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(4) for layer in (1, 2)), }' to 'required ' in host memory. Here required means the value named 'required' in this block. The important operands are f: the value named 'f' in this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this block. Context: the function ___init___, which handles init.

Line 577: `missing = required - set(archive.projection_names)` This assigns 'required - set(archive.projection_names)' to 'missing ' in host memory. Here missing means the value named 'missing' in this block. The important operands are required: the value named 'required' in this block; set: the value named 'set' in this block; archive: the value named 'archive' in this block. Context: the function ___init___, which handles init.

Line 578: `if missing:` This executes the following indented host-code branch only when 'missing' is true. The important operands are missing: the value named 'missing' in this block. Context: the function ___init___, which handles init.

Line 579: `raise ValueError(f"parameter archive is missing projections: {sorted(missing)}")` This rejects an invalid state by raising 'ValueError(f"parameter archive is missing projections: {sorted(missing)}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 581: `def _lif(self, value: np.ndarray, *, threshold: float = 1.0) -> np.ndarray:` This starts function _lif. Its indented body performs lif as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 582: `return lif_sequence_on_accelerator(` This statement continues through source line 589; the explanation uses the complete expression rather than only the first visual fragment. This returns 'lif_sequence_on_accelerator(self.driver, value, threshold_q10=int(round(threshold * self.fixed_scale)), leak_shift=1, reset_q10=0, timeout_seconds=self.layer_)' to the caller of the function _lif, which handles lif, ending this invocation. The important operands are lif_sequence_on_accelerato the value named 'lif sequence on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block.

Line 583: `self.driver,` This is continuation line 2 of the 8-line logical statement that begins at source line 582. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This returns 'lif_sequence_on_accelerator(self.driver, value, threshold_q10=int(round(threshold * self.fixed_scale)), leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)' to the caller of the function _lif, which handles lif, ending this invocation. The important operands are lif_sequence_on_accelerator: the value named 'lif sequence on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block.

Line 584: `value,` This is continuation line 3 of the 8-line logical statement that begins at source line 582. Its exact fragment is 'value,'. The important operands are value: the value named 'value' in this block. Read together, the complete operation is: This returns 'lif_sequence_on_accelerator(self.driver, value, threshold_q10=int(round(threshold * self.fixed_scale)), leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)' to the caller of the function _lif, which handles lif, ending this invocation. The important operands are lif_sequence_on_accelerator: the value named 'lif sequence on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block.

Line 585: `threshold_q10=int(round(threshold * self.fixed_scale)),` This is continuation line 4 of the 8-line logical statement that begins at source line 582. Its exact fragment is 'threshold_q10=int(round(threshold * self.fixed_scale)),'. The important operands are threshold_q10: the membrane level at which a neuron emits a spike; round: the value named 'round' in this block; threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This returns 'lif_sequence_on_accelerator(self.driver, value, threshold_q10=int(round(threshold * self.fixed_scale)), leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)' to the caller of the function _lif, which handles lif, ending this invocation. The important operands are lif_sequence_on_accelerator: the value named 'lif sequence on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block.

Line 586: `leak_shift=1,` This is continuation line 5 of the 8-line logical statement that begins at source line 582. Its exact fragment is 'leak_shift=1,'. The important operands are leak_shift: the retained fraction of prior membrane state. Read together, the complete operation is: This returns 'lif_sequence_on_accelerator(self.driver, value, threshold_q10=int(round(threshold * self.fixed_scale)), leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)' to the caller of the function _lif, which handles lif, ending this invocation. The important operands are lif_sequence_on_accelerato the value named 'lif sequence on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block.

Line 587: `reset_q10=0,` This is continuation line 6 of the 8-line logical statement that begins at source line 582. Its exact fragment is 'reset_q10=0,'. The important operands are reset_q10: the value named 'reset q10' in this block. Read together, the complete operation is: This returns 'lif_sequence_on_accelerator(self.driver, value, threshold_q10=int(round(threshold * self.fixed_scale)), leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)' to the caller of the function _lif, which handles lif, ending this invocation. The important operands are lif_sequence_on_accelerato the value named 'lif sequence on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block.

Line 588: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 7 of the 8-line logical statement that begins at source line 582. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This returns 'lif_sequence_on_accelerator(self.driver, value, threshold_q10=int(round(threshold * self.fixed_scale)), leak_shift=1, reset_q10=0, timeout_seconds=self.layer_)' to the caller of the function _lif, which handles lif, ending this invocation. The important operands are lif_sequence_on_accelerato the value named 'lif sequence on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block.

Line 589: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 582. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'lif_sequence_on_accelerator(self.driver, value, threshold_q10=int(round(threshold * self.fixed_scale)), leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)' to the caller of the function _lif, which handles lif, ending this invocation. The important operands are lif_sequence_on_accelerator: the value named 'lif sequence on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block.

Line 591: `def _project_tokens(self, name: str, spikes: np.ndarray) -> np.ndarray:` This starts function _project_tokens. Its indented body performs project tokens as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 592: `if spikes.ndim != 4:` This executes the following indented host-code branch only when 'spikes.ndim != 4' is true. The important operands are spikes: a binary event produced when a neuron crosses its threshold; ndim: the value named 'ndim' in this block. Context: the function _project_tokens, which handles project tokens.

Line 593: `raise ValueError("token projection input must have shape [T,B,N,C]")` This rejects an invalid state by raising 'ValueError("token projection input must have shape [T,B,N,C]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 594: `timesteps, batch, tokens, channels = spikes.shape` This assigns ‘spikes.shape‘ to ‘timesteps, batch, tokens, channels ‘ in host memory. Here timesteps means one discrete SNN update instant. The important operands are spikes: a binary event produced when a neuron crosses its threshold; shape: the value named ’shape’ in this block. Context: the function _project_tokens, which handles project tokens.

Line 595: `rows = spikes.reshape(timesteps * batch * tokens, channels)` This assigns ‘spikes.reshape(timesteps * batch * tokens, channels)‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _project_tokens, which handles project tokens.

Line 596: `result = self.projections.matrix(` This statement continues through source line 597; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.projections.matrix(name, rows, timeout_seconds=self.layer_timeout_seconds)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; name: the value named ’name’ in this block. Context: the function _project_tokens, which handles project tokens.

Line 597: `name, rows, timeout_seconds=self.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 596. Its exact fragment is ‘name, rows, timeout_seconds=self.layer_timeout_seconds)‘. The important operands are name: the value named ’name’ in this block; rows: one output-row position processed by the tensor tile; timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(name, rows, timeout_seconds=self.layer_timeout_seconds)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; name: the value named ’name’ in this block. Context: the function _project_tokens, which handles project tokens.

Line 598: `return result.reshape(timesteps, batch, tokens, -1)` This returns ‘result.reshape(timesteps, batch, tokens, -1)‘ to the caller of the function _project_tokens, which handles project tokens, ending this invocation. The important operands are result: the value named ’result’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant.

Line 600: `def _dynamic_binary_gemm(self, left: np.ndarray, right: np.ndarray) -> np.ndarray:` This starts function _dynamic_binary_gemm. Its indented body performs dynamic binary gemm as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 601: `program = compile_spike_gemm(left, right)` This assigns ‘compile_spike_gemm(left, right)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold; left: the value named ’left’ in this block; right: the value named ’right’ in this block. Context: the function _dynamic_binary_gemm, which handles dynamic binary gemm.

Line 602: `return program.execute_batched(` This statement continues through source line 603; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘program.execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)‘ to the caller of the function _dynamic_binary_gemm, which handles dynamic binary gemm, ending this invocation. The important operands are program: the value named ’program’ in this block; execute_batched: the value named ’execute batched’ in this block; driver: the value named ’driver’ in this block.

Line 603: `self.driver, timeout_seconds=self.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 602. Its exact fragment is ‘self.driver, timeout_seconds=self.layer_timeout_seconds)‘. The important operands are driver: the value named ’driver’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This returns ‘program.execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)‘ to the caller of the function _dynamic_binary_gemm, which handles dynamic binary gemm, ending this invocation. The

important operands are program: the value named 'program' in this block; execute_batched: the value named 'execute batched' in this block; driver: the value named 'driver' in this block.

Line 605: `def _dynamic_u8_gemm(self, left: np.ndarray, right: np.ndarray) -> np.ndarray:` This starts function _dynamic_u8_gemm. Its indented body performs dynamic u8 gemm as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 606: `program = compile_u8_gemm(left, right)` This assigns 'compile_u8_gemm(left, right)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; left: the value named 'left' in this block; right: the value named 'right' in this block. Context: the function _dynamic_u8_gemm, which handles dynamic u8 gemm.

Line 607: `return program.execute_batched(` This statement continues through source line 608; the explanation uses the complete expression rather than only the first visual fragment. This returns 'program.execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)' to the caller of the function _dynamic_u8_gemm, which handles dynamic u8 gemm, ending this invocation. The important operands are program: the value named 'program' in this block; execute_batched: the value named 'execute batched' in this block; driver: the value named 'driver' in this block.

Line 608: `self.driver, timeout_seconds=self.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 607. Its exact fragment is 'self.driver, timeout_seconds=self.layer_timeout_seconds)'. The important operands are driver: the value named 'driver' in this block; timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This returns 'program.execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)' to the caller of the function _dynamic_u8_gemm, which handles dynamic u8 gemm, ending this invocation. The important operands are program: the value named 'program' in this block; execute_batched: the value named 'execute batched' in this block; driver: the value named 'driver' in this block.

Line 610: `def _attention(self, block_index: int, value: np.ndarray) -> np.ndarray:` This starts function _attention. Its indented body performs attention as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 611: `timesteps, batch, channels, height, width = value.shape` This assigns 'value.shape' to 'timesteps, batch, channels, height, width ' in host memory. Here timesteps means one discrete SNN update instant. The important operands are value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function _attention, which handles attention.

Line 612: `tokens = height * width` This assigns 'height * width' to 'tokens ' in host memory. Here tokens means the value named 'tokens' in this block. The important operands are height: the value named 'height' in this block; width: the value named 'width' in this block. Context: the function _attention, which handles attention.

Line 613: `projection_spikes = self._lif(value)` This assigns 'self._lif(value)' to 'projection_spikes ' in host memory. Here projection_spikes means a binary event produced when a neuron crosses its threshold. The important operands are _lif: the value named 'lif' in this block; value: the value named 'value' in this block. Context: the function _attention, which handles attention.

Line 614: `token_spikes = projection_spikes.reshape(` This statement continues through source line 615; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'projection_spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)' to 'token_spikes ' in host memory. Here token_spikes means a binary event produced when a neuron crosses its threshold. The important operands are projection_spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 615: `timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)` This is continuation line 2 of the 2-line logical statement that begins at source line 614. Its exact fragment is ‘timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘projection_spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)‘ to ‘token_spikes ‘ in host memory. Here token_spikes means a binary event produced when a neuron crosses its threshold. The important operands are projection_spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 616: `prefix = f"block.{block_index}.attn"` This assigns ‘f"block.{block_index}.attn"‘ to ‘prefix ‘ in host memory. Here prefix means the value named ’prefix’ in this block. The important operands are f: the value named ’f’ in this block; block: the value named ’block’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function _attention, which handles attention.

Line 617: `query = self._lif(self._project_tokens(f"{prefix}.q", token_spikes))` This assigns ‘self._lif(self._project_ token_spikes))‘ to ‘query ‘ in host memory. Here query means the value named ’query’ in this block. The important operands are _lif: the value named ’lif’ in this block; _project_tokens: the value named ’project tokens’ in this block; f: the value named ’f’ in this block. Context: the function _attention, which handles attention.

Line 618: `key = self._lif(self._project_tokens(f"{prefix}.k", token_spikes))` This assigns ‘self._lif(self._project_to token_spikes))‘ to ‘key ‘ in host memory. Here key means the value named ’key’ in this block. The important operands are _lif: the value named ’lif’ in this block; _project_tokens: the value named ’project tokens’ in this block; f: the value named ’f’ in this block. Context: the function _attention, which handles attention.

Line 619: `projected_value = self._lif(self._project_tokens(f"{prefix}.v", token_spikes))` This assigns ‘self._lif(self._project_tokens(f"{prefix}.v", token_spikes))‘ to ‘projected_value ‘ in host memory. Here projected_value means the value named ’projected value’ in this block. The important operands are _lif: the value named ’lif’ in this block; _project_tokens: the value named ’project tokens’ in this block; f: the value named ’f’ in this block. Context: the function _attention, which handles attention.

Line 621: `heads = 12` This assigns ‘12‘ to ‘heads ‘ in host memory. Here heads means the value named ’heads’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _attention, which handles attention.

Line 622: `if channels % heads:` This executes the following indented host-code branch only when ‘channels % heads‘ is true. The important operands are channels: the value named ’channels’ in this block; heads: the value named ’heads’ in this block. Context: the function _attention, which handles attention.

Line 623: `raise ValueError("attention channels must divide into 12 heads")` This rejects an invalid state by raising ‘ValueError("attention channels must divide into 12 heads")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 624: `head_dim = channels // heads` This assigns ‘channels // heads‘ to ‘head_dim ‘ in host memory. Here head_dim means the value named ’head dim’ in this block. The important operands are channels: the value named ’channels’ in this block; heads: the value named ’heads’ in this block. Context: the function _attention, which handles attention.

Line 625: `query = query.reshape(timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)` This assigns ‘query.reshape(timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)‘ to ‘query ‘ in host memory. Here query means the value named ’query’ in this block. The important operands are query: the value named ’query’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 626: `key = key.reshape(timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)` This assigns ‘key.reshape(timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)‘ to ‘key ‘ in host memory. Here key means the value named ’key’ in this block. The important operands are key: the value named ’key’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 627: `projected_value = projected_value.reshape(` This statement continues through source line 628; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘projected_value.reshape(timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)‘ to ‘projected_value ‘ in host memory. Here projected_value means the value named ’projected value’ in this block. The important operands are projected_value: the value named ’projected value’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 628: `timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)` This is continuation line 2 of the 2-line logical statement that begins at source line 627. Its exact fragment is ‘timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This assigns ‘projected_value.reshape(timesteps, batch, tokens, heads, head_dim).transpose(0, 1, 3, 2, 4)‘ to ‘projected_value ‘ in host memory. Here projected_value means the value named ’projected value’ in this block. The important operands are projected_value: the value named ’projected value’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 629: `attention_value = np.empty(` This statement continues through source line 630; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.empty((timesteps, batch, heads, tokens, head_dim), dtype=np.int64)‘ to ‘attention_value ‘ in host memory. Here attention_value means the value named ’attention value’ in this block. The important operands are empty: the value named ’empty’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Context: the function _attention, which handles attention.

Line 630: `(timesteps, batch, heads, tokens, head_dim), dtype=np.int64)` This is continuation line 2 of the 2-line logical statement that begins at source line 629. Its exact fragment is ‘(timesteps, batch, heads, tokens, head_dim), dtype=np.int64)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; heads: the value named ’heads’ in this block. Read together, the complete operation is: This assigns ‘np.empty((timesteps, batch, heads, tokens, head_dim), dtype=np.int64)‘ to ‘attention_value ‘ in host memory. Here attention_value means the value named ’attention value’ in this block. The important operands are empty: the value named ’empty’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Context: the function _attention, which handles attention.

Line 631: `for timestep in range(timesteps):` This iterates timestep over ‘range(timesteps)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention, which handles attention.

Line 632: `for batch_index in range(batch):` This iterates batch_index over ‘range(batch)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention, which handles attention.

Line 633: `for head in range(heads):` This iterates head over ‘range(heads)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention, which handles attention.

Line 634: `scores = self._dynamic_binary_gemm(` This statement continues through source line 637; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._dynamic_binary_gemm(query[timestep, batch_index, head], key[timestep, batch_index, head].T,)‘ to ‘scores ‘ in host memory. Here scores means the value named ’scores’ in this block. The important operands are _dynamic_binary_gemm: the value named ’dynamic binary gemm’ in this block; query: the value named ’query’ in this block; timestep: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 635: `query[timestep, batch_index, head],` This is continuation line 2 of the 4-line logical statement that begins at source line 634. Its exact fragment is 'query[timestep, batch_index, head],'. The important operands are query: the value named 'query' in this block; timestep: one discrete SNN update instant; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'self._dynamic_binary_gemm(query[timestep, batch_index, head], key[timestep, batch_index, head].T,)' to 'scores ' in host memory. Here scores means the value named 'scores' in this block. The important operands are _dynamic_binary_gemm: the value named 'dynamic binary gemm' in this block; query: the value named 'query' in this block; timestep: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 636: `key[timestep, batch_index, head].T,` This is continuation line 3 of the 4-line logical statement that begins at source line 634. Its exact fragment is 'key[timestep, batch_index, head].T,'. The important operands are key: the value named 'key' in this block; timestep: one discrete SNN update instant; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'self._dynamic_binary_gemm(query[timestep, batch_index, head], key[timestep, batch_index, head].T,)' to 'scores ' in host memory. Here scores means the value named 'scores' in this block. The important operands are _dynamic_binary_gemm: the value named 'dynamic binary gemm' in this block; query: the value named 'query' in this block; timestep: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 637: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 634. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self._dynamic_binary_gemm(query[timestep, batch_index, head], key[timestep, batch_index, head].T,)' to 'scores ' in host memory. Here scores means the value named 'scores' in this block. The important operands are _dynamic_binary_gemm: the value named 'dynamic binary gemm' in this block; query: the value named 'query' in this block; timestep: one discrete SNN update instant. Context: the function _attention, which handles attention.

Line 638: `if scores.min() < 0 or scores.max() > 255:` This executes the following indented host-code branch only when 'scores.min() $<$ 0 or scores.max() $>$ 255' is true. The important operands are scores: the value named 'scores' in this block; min: the value named 'min' in this block; or: the value named 'or' in this block. Context: the function _attention, which handles attention.

Line 639: `raise OverflowError("binary QK score does not fit the dynamic-u8 AV path")` This rejects an invalid state by raising 'OverflowError("binary QK score does not fit the dynamic-u8 AV path")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 640: `attention_value[timestep, batch_index, head] = self._dynamic_u8_gemm(` This statement continues through source line 643; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._dynamic_u8_gemm(scores.astype(np.uint8), projected_value[timestep, batch_index, head].astype(np.uint8),)' to 'attention_value[timestep, batch_index, head] ' in host memory. Here attention_value means the value named 'attention value' in this block. The important operands are _dynamic_u8_gemm: the value named 'dynamic u8 gemm' in this block; scores: the value named 'scores' in this block; astype: the value named 'astype' in this block. Context: the function _attention, which handles attention.

Line 641: `scores.astype(np.uint8),` This is continuation line 2 of the 4-line logical statement that begins at source line 640. Its exact fragment is 'scores.astype(np.uint8),'. The important operands are scores: the value named 'scores' in this block; astype: the value named 'astype' in this block; uint8: the value named 'uint8' in this block. Read together, the complete operation is: This assigns 'self._dynamic_u8_gemm(scores.astype(np.uint8), projected_value[timestep, batch_index, head].astype(np.uint8),)' to 'attention_value[timestep, batch_index, head] ' in host memory. Here attention_value means the value named 'attention value' in this block. The important operands are _dynamic_u8_gemm: the value named 'dynamic u8 gemm' in this block; scores: the value named 'scores' in this block; astype: the value named 'astype' in this block. Context: the function _attention, which handles attention.

Line 642: `projected_value[timestep, batch_index, head].astype(np.uint8),` This is continuation line 3 of the 4-line logical statement that begins at source line 640. Its exact fragment is 'projected_value[timestep, batch_index, head].astype(np.uint8),'. The important operands are projected_value: the value named 'projected value' in this block; timestep: one discrete SNN update instant; batch_index: an integer selecting one tensor element, lane, bank, or state

entry. Read together, the complete operation is: This assigns ‘self._dynamic_u8_gemm(scores.astype(np.uint8), projected_value[timestep, batch_index, head].astype(np.uint8),)‘ to ‘attention_value[timestep, batch_index, head] ‘ in host memory. Here attention_value means the value named ’attention value’ in this block. The important operands are _dynamic_u8_gemm: the value named ’dynamic u8 gemm’ in this block; scores: the value named ’scores’ in this block; astype: the value named ’astype’ in this block. Context: the function _attention, which handles attention.

Line 643: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 640. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self._dynamic_u8_gemm(scores.astype(np.uint8), projected_value[timestep, batch_index, head].astype(np.uint8),)‘ to ‘attention_value[timestep, batch_index, head] ‘ in host memory. Here attention_value means the value named ’attention value’ in this block. The important operands are _dynamic_u8_gemm: the value named ’dynamic u8 gemm’ in this block; scores: the value named ’scores’ in this block; astype: the value named ’astype’ in this block. Context: the function _attention, which handles attention.

Line 644: `attention_q10 = attention_value * (self.fixed_scale // 8)` This assigns ‘attention_value * (self.fixed_scale // 8)‘ to ‘attention_q10 ‘ in host memory. Here attention_q10 means the value named ’attention q10’ in this block. The important operands are attention_value: the value named ’attention value’ in this block; fixed_scale: the real-value spacing represented by one quantized integer step. Context: the function _attention, which handles attention.

Line 645: `merged = attention_q10.transpose(0, 1, 2, 4, 3).reshape(` This statement continues through source line 646; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘attention_q10.transpose(0, 1, 2, 4, 3).reshape(timesteps, batch, channels, tokens)‘ to ‘merged ‘ in host memory. Here merged means the value named ’merged’ in this block. The important operands are attention_q10: the value named ’attention q10’ in this block; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _attention, which handles attention.

Line 646: `timesteps, batch, channels, tokens)` This is continuation line 2 of the 2-line logical statement that begins at source line 645. Its exact fragment is ‘timesteps, batch, channels, tokens)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘attention_q10.transpose(0, 1, 2, 4, 3).reshape(timesteps, batch, channels, tokens)‘ to ‘merged ‘ in host memory. Here merged means the value named ’merged’ in this block. The important operands are attention_q10: the value named ’attention q10’ in this block; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _attention, which handles attention.

Line 647: `attention_spikes = self._lif(merged, threshold=0.5)` This assigns ‘self._lif(merged, threshold=0.5)‘ to ‘attention_spikes ‘ in host memory. Here attention_spikes means a binary event produced when a neuron crosses its threshold. The important operands are _lif: the value named ’lif’ in this block; merged: the value named ’merged’ in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function _attention, which handles attention.

Line 648: `output_rows = attention_spikes.transpose(0, 1, 3, 2).reshape(` This statement continues through source line 649; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘attention_spikes.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are attention_spikes: a binary event produced when a neuron crosses its threshold; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _attention, which handles attention.

Line 649: `timesteps * batch * tokens, channels)` This is continuation line 2 of the 2-line logical statement that begins at source line 648. Its exact fragment is ‘timesteps * batch * tokens, channels)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This assigns ‘attention_spikes.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are attention_spikes: a binary event produced when a neuron crosses its threshold; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _attention, which handles attention.

Line 650: `output = self.projections.matrix(` This statement continues through source line 652; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.projections.matrix(f"{prefix}.proj", output_rows, timeout_seconds=self.layer_timeout_seconds)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _attention, which handles attention.

Line 651: `f"{prefix}.proj", output_rows,` This is continuation line 2 of the 3-line logical statement that begins at source line 650. Its exact fragment is ‘f"{prefix}.proj", output_rows,‘. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; proj: the value named ’proj’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"{prefix}.proj", output_rows, timeout_seconds=self.layer_timeout_seconds)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _attention, which handles attention.

Line 652: `timeout_seconds=self.layer_timeout_seconds)` This is continuation line 3 of the 3-line logical statement that begins at source line 650. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds)‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"{prefix}.proj", output_rows, timeout_seconds=self.layer_timeout_seconds)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _attention, which handles attention.

Line 653: `return output.reshape(timesteps, batch, tokens, channels).transpose(` This statement continues through source line 654; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘output.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to the caller of the function _attention, which handles attention, ending this invocation. The important operands are output: the value named ’output’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant.

Line 654: `0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)` This is continuation line 2 of the 2-line logical statement that begins at source line 653. Its exact fragment is ‘0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘. The important operands are reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Read together, the complete operation is: This returns ‘output.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to the caller of the function _attention, which handles attention, ending this invocation. The important operands are output: the value named ’output’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant.

Line 656: `def _mlp(self, block_index: int, value: np.ndarray) -> np.ndarray:` This starts function _mlp. Its indented body performs mlp as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 657: `timesteps, batch, channels, height, width = value.shape` This assigns ‘value.shape‘ to ‘timesteps, batch, channels, height, width ‘ in host memory. Here timesteps means one discrete SNN update instant. The important operands are value: the value named ’value’ in this block; shape: the value named ’shape’ in this block. Context: the function _mlp, which handles mlp.

Line 658: `tokens = height * width` This assigns ‘height * width‘ to ‘tokens ‘ in host memory. Here tokens means the value named ’tokens’ in this block. The important operands are height: the value named ’height’ in this block; width: the value named ’width’ in this block. Context: the function _mlp, which handles mlp.

Line 659: `first_spikes = self._lif(value)` This assigns ‘self._lif(value)‘ to ‘first_spikes ‘ in host memory. Here first_spikes means a binary event produced when a neuron crosses its threshold. The important operands are _lif: the

value named 'lif' in this block; value: the value named 'value' in this block. Context: the function _mlp, which handles mlp.

Line 660: `first_rows = first_spikes.reshape(` This statement continues through source line 662; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'first_spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)' to 'first_rows ' in host memory. Here first_rows means one output-row position processed by the tensor tile. The important operands are first_spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _mlp, which handles mlp.

Line 661: `timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(` This is continuation line 2 of the 3-line logical statement that begins at source line 660. Its exact fragment is 'timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape('. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This assigns 'first_spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)' to 'first_rows ' in host memory. Here first_rows means one output-row position processed by the tensor tile. The important operands are first_spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _mlp, which handles mlp.

Line 662: `timesteps * batch * tokens, channels)` This is continuation line 3 of the 3-line logical statement that begins at source line 660. Its exact fragment is 'timesteps * batch * tokens, channels)'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This assigns 'first_spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)' to 'first_rows ' in host memory. Here first_rows means one output-row position processed by the tensor tile. The important operands are first_spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _mlp, which handles mlp.

Line 663: `hidden = self.projections.matrix(` This statement continues through source line 665; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.projections.matrix(f"block.{block_index}.mlp.fc1", first_rows, timeout_seconds=self.layer_timeout_seconds)' to 'hidden ' in host memory. Here hidden means the value named 'hidden' in this block. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function _mlp, which handles mlp.

Line 664: `f"block.{block_index}.mlp.fc1", first_rows,` This is continuation line 2 of the 3-line logical statement that begins at source line 663. Its exact fragment is 'f"block.{block_index}.mlp.fc1", first_rows,'. The important operands are f: the value named 'f' in this block; block: the value named 'block' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'self.projections.matrix(f"block.{block_index}.mlp.fc1", first_rows, timeout_seconds=self.layer_timeout_seconds)' to 'hidden ' in host memory. Here hidden means the value named 'hidden' in this block. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function _mlp, which handles mlp.

Line 665: `timeout_seconds=self.layer_timeout_seconds)` This is continuation line 3 of the 3-line logical statement that begins at source line 663. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds)'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'self.projections.matrix(f"block.{block_index}.mlp.fc1", first_rows, timeout_seconds=self.layer_timeout_seconds)' to 'hidden ' in host memory. Here hidden means the value named 'hidden' in this block. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function _mlp, which handles mlp.

Line 666: `hidden_channels = hidden.shape[1]` This assigns 'hidden.shape[1]' to 'hidden_channels ' in host memory. Here hidden_channels means the value named 'hidden channels' in this block. The important operands are hidden:

the value named 'hidden' in this block; shape: the value named 'shape' in this block. Context: the function _mlp, which handles mlp.

Line 667: `hidden = hidden.reshape(timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)` This assigns 'hidden.reshape(timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)' to 'hidden ' in host memory. Here hidden means the value named 'hidden' in this block. The important operands are hidden: the value named 'hidden' in this block; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _mlp, which handles mlp.

Line 668: `hidden_spikes = self._lif(hidden)` This assigns 'self._lif(hidden)' to 'hidden_spikes ' in host memory. Here hidden_spikes means a binary event produced when a neuron crosses its threshold. The important operands are _lif: the value named 'lif' in this block; hidden: the value named 'hidden' in this block. Context: the function _mlp, which handles mlp.

Line 669: `second_rows = hidden_spikes.transpose(0, 1, 3, 2).reshape(` This statement continues through source line 670; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'hidden_spikes.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, hidden_channels)' to 'second_rows ' in host memory. Here second_rows means one output-row position processed by the tensor tile. The important operands are hidden_spikes: a binary event produced when a neuron crosses its threshold; transpose: the value named 'transpose' in this block; reshape: the value named 'reshape' in this block. Context: the function _mlp, which handles mlp.

Line 670: `timesteps * batch * tokens, hidden_channels)` This is continuation line 2 of the 2-line logical statement that begins at source line 669. Its exact fragment is 'timesteps * batch * tokens, hidden_channels)'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This assigns 'hidden_spikes.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, hidden_channels)' to 'second_rows ' in host memory. Here second_rows means one output-row position processed by the tensor tile. The important operands are hidden_spikes: a binary event produced when a neuron crosses its threshold; transpose: the value named 'transpose' in this block; reshape: the value named 'reshape' in this block. Context: the function _mlp, which handles mlp.

Line 671: `output = self.projections.matrix(` This statement continues through source line 673; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.projections.matrix(f"block.{block_index}.mlp.fc2", second_rows, timeout_seconds=self.layer_timeout_seconds)' to 'output ' in host memory. Here output means the value named 'output' in this block. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function _mlp, which handles mlp.

Line 672: `f"block.{block_index}.mlp.fc2", second_rows,` This is continuation line 2 of the 3-line logical statement that begins at source line 671. Its exact fragment is 'f"block.{block_index}.mlp.fc2", second_rows,'. The important operands are f: the value named 'f' in this block; block: the value named 'block' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'self.projections.matrix(f"block.{block_index}.mlp.fc2", second_rows, timeout_seconds=self.layer_timeout_seconds)' to 'output ' in host memory. Here output means the value named 'output' in this block. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function _mlp, which handles mlp.

Line 673: `timeout_seconds=self.layer_timeout_seconds)` This is continuation line 3 of the 3-line logical statement that begins at source line 671. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds)'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'self.projections.matrix(f"block.{block_index}.mlp.fc2", second_rows, timeout_seconds=self.layer_timeout_seconds)' to 'output ' in host memory. Here output means the value named 'output' in this block. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function _mlp, which handles mlp.

Line 674: `return output.reshape(timesteps, batch, tokens, channels).transpose(` This statement continues through source line 675; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘output.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to the caller of the function _mlp, which handles mlp, ending this invocation. The important operands are output: the value named ’output’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant.

Line 675: `0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)` This is continuation line 2 of the 2-line logical statement that begins at source line 674. Its exact fragment is ‘0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘. The important operands are reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Read together, the complete operation is: This returns ‘output.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to the caller of the function _mlp, which handles mlp, ending this invocation. The important operands are output: the value named ’output’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant.

Line 677: `def _tokenizer(self, image_u8: np.ndarray) -> np.ndarray:` This starts function _tokenizer. Its indented body performs tokenizer as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 678: `batch = image_u8.shape[0]` This assigns ‘image_u8.shape[0]‘ to ‘batch ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are image_u8: the value named ’image u8’ in this block; shape: the value named ’shape’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 679: `first = self.projections.conv2d(` This statement continues through source line 685; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeou)‘ to ‘first ‘ in host memory. Here first means the value named ’first’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; patch_embed: the value named ’patch embed’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 680: `"patch_embed.block0",` This is continuation line 2 of the 7-line logical statement that begins at source line 679. Its exact fragment is ‘"patch_embed.block0",‘. The important operands are patch_embed: the value named ’patch embed’ in this block; block0: the value named ’block0’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘first ‘ in host memory. Here first means the value named ’first’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; patch_embed: the value named ’patch embed’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 681: `image_u8,` This is continuation line 3 of the 7-line logical statement that begins at source line 679. Its exact fragment is ‘image_u8,‘. The important operands are image_u8: the value named ’image u8’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘first ‘ in host memory. Here first means the value named ’first’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; patch_embed: the value named ’patch embed’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 682: `activation_zero_point=self.image_zero_point,` This is continuation line 4 of the 7-line logical statement that begins at source line 679. Its exact fragment is ‘activation_zero_point=self.image_zero_point,‘. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; image_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns ‘self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘first ‘ in host memory. Here first means the value named ’first’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the

value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 683: `padding=1,` This is continuation line 5 of the 7-line logical statement that begins at source line 679. Its exact fragment is 'padding=1,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'first ' in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 684: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 679. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeou)' to 'first ' in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 685: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 679. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'first ' in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 686: `value = np.repeat(first[np.newaxis], 4, axis=0)` This assigns 'np.repeat(first[np.newaxis], 4, axis=0)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are repeat: the value named 'repeat' in this block; first: the value named 'first' in this block; newaxis: the value named 'newaxis' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 687: `for index in (1, 2):` This iterates index over '(1, 2)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _tokenizer, which handles tokenizer.

Line 688: `spikes = self._lif(value)` This assigns 'self._lif(value)' to 'spikes ' in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are _lif: the value named 'lif' in this block; value: the value named 'value' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 689: `flat = spikes.reshape(-1, *spikes.shape[2:])` This assigns 'spikes.reshape(-1, *spikes.shape[2:])' to 'flat ' in host memory. Here flat means the value named 'flat' in this block. The important operands are spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block; shape: the value named 'shape' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 690: `projected = self.projections.conv2d(` This statement continues through source line 695; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.projections.conv2d(f"patch_embed.block{index}", flat, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; f: the value named 'f' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 691: `f"patch_embed.block{index}",` This is continuation line 2 of the 6-line logical statement that begins at source line 690. Its exact fragment is 'f"patch_embed.block{index}",'. The important operands are f: the value named 'f' in this block; patch_embed: the value named 'patch embed' in this block; block: the value named 'block' in this

block. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", flat, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 692: `flat,` This is continuation line 3 of the 6-line logical statement that begins at source line 690. Its exact fragment is ‘flat,‘. The important operands are flat: the value named ’flat’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", flat, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 693: `padding=1,` This is continuation line 4 of the 6-line logical statement that begins at source line 690. Its exact fragment is ‘padding=1,‘. The important operands are padding: the value named ’padding’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", flat, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 694: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 690. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", flat, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 695: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 690. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", flat, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 696: `value = projected.reshape(4, batch, *projected.shape[1:])` This assigns ‘projected.reshape(4, batch, *projected.shape[1:])‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are projected: the value named ’projected’ in this block; reshape: the value named ’reshape’ in this block; batch: the value named ’batch’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 697: `for index in (3, 4):` This iterates index over ‘(3, 4)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _tokenizer, which handles tokenizer.

Line 698: `spikes = self._lif(value)` This assigns ‘self._lif(value)‘ to ‘spikes ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are _lif: the value named ’lif’ in this block; value: the value named ’value’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 699: `flat = spikes.reshape(-1, *spikes.shape[2:])` This assigns ‘spikes.reshape(-1, *spikes.shape[2:])‘ to ‘flat ‘ in host memory. Here flat means the value named ’flat’ in this block. The important operands are spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block; shape: the value named ’shape’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 700: `pooled = binary_max_pool2d_on_accelerator(` This statement continues through source line 704; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘binary_max_pool2d_on_accelerator self.driver, flat, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘pooled ‘ in host memory. Here pooled means the value named ’pooled’ in this block. The important operands are binary_max_pool2d_on_accelerator: the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat: the value named ’flat’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 701: `self.driver,` This is continuation line 2 of the 5-line logical statement that begins at source line 700. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘binary_max_pool2d_on_accelerator(self.driver, flat, timeout_seconds=self.layer_timeout_se)‘ to ‘pooled ‘ in host memory. Here pooled means the value named ’pooled’ in this block. The important operands are binary_max_pool2d_on_accelerator: the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat: the value named ’flat’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 702: `flat,` This is continuation line 3 of the 5-line logical statement that begins at source line 700. Its exact fragment is ‘flat,‘. The important operands are flat: the value named ’flat’ in this block. Read together, the complete operation is: This assigns ‘binary_max_pool2d_on_accelerator(self.driver, flat, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘pooled ‘ in host memory. Here pooled means the value named ’pooled’ in this block. The important operands are binary_max_pool2d_on_accelerator: the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat: the value named ’flat’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 703: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 4 of the 5-line logical statement that begins at source line 700. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘binary_max_pool2d_on_accelerator(self.driver, flat, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘pooled ‘ in host memory. Here pooled means the value named ’pooled’ in this block. The important operands are binary_max_pool2d_on_accelerat the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat: the value named ’flat’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 704: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 700. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘binary_max_pool2d_on_accelerator(self.driver, flat, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘pooled ‘ in host memory. Here pooled means the value named ’pooled’ in this block. The important operands are binary_max_pool2d_on_accelerator: the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat: the value named ’flat’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 705: `projected = self.projections.conv2d(` This statement continues through source line 710; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", pooled, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 706: `f"patch_embed.block{index}",` This is continuation line 2 of the 6-line logical statement that begins at source line 705. Its exact fragment is ‘f"patch_embed.block{index}",‘. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", pooled, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer, which handles tokenizer.

Line 707: `pooled,` This is continuation line 3 of the 6-line logical statement that begins at source line 705. Its exact fragment is 'pooled,'. The important operands are pooled: the value named 'pooled' in this block. Read together, the complete operation is: This assigns 'self.projections.conv2d(f"patch_embed.block{index}", pooled, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; f: the value named 'f' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 708: `padding=1,` This is continuation line 4 of the 6-line logical statement that begins at source line 705. Its exact fragment is 'padding=1,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'self.projections.conv2d(f"patch_embed.block{index}", pooled, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; f: the value named 'f' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 709: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 705. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'self.projections.conv2d(f"patch_embed.block{index}", pooled, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; f: the value named 'f' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 710: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 705. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self.projections.conv2d(f"patch_embed.block{index}", pooled, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; f: the value named 'f' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 711: `value = projected.reshape(4, batch, *projected.shape[1:])` This assigns 'projected.reshape(4, batch, *projected.shape[1:])' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are projected: the value named 'projected' in this block; reshape: the value named 'reshape' in this block; batch: the value named 'batch' in this block. Context: the function _tokenizer, which handles tokenizer.

Line 712: `return value` This returns 'value' to the caller of the function _tokenizer, which handles tokenizer, ending this invocation. The important operands are value: the value named 'value' in this block.

Line 714: `def infer(self, normalized_image: np.ndarray) -> SpikingformerInferenceResult:` This starts function infer. Its indented body performs infer as part of the file that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations; it runs only when called.

Line 715: `"""Run normalized CIFAR-10 NCHW images through the complete trained graph."""` This is executable documentation attached to the function infer, which handles infer. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 717: `image = np.asarray(normalized_image, dtype=np.float32)` This assigns 'np.asarray(normalized_image, dtype=np.float32)' to 'image ' in host memory. Here image means the value named 'image' in this block. The important operands are asarray: the value named 'asarray' in this block; normalized_image: the value named 'normalized image' in this block; dtype: the value named 'dtype' in this block. Context: the function infer, which handles infer.

Line 718: `if image.ndim != 4 or image.shape[1:] != (3, 32, 32) or image.shape[0] == 0:` This executes the following indented host-code branch only when ‘image.ndim != 4 or image.shape[1:] != (3, 32, 32) or image.shape[0] == 0‘ is true. The important operands are image: the value named ’image’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function infer, which handles infer.

Line 719: `raise ValueError("input must have shape [batch,3,32,32]")` This rejects an invalid state by raising ‘ValueError("input must have shape [batch,3,32,32]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 720: `image_u8 = np.rint(image / self.image_scale + self.image_zero_point)` This assigns ‘np.rint(image / self.image_scale + self.image_zero_point)‘ to ‘image_u8 ‘ in host memory. Here image_u8 means the value named ’image u8’ in this block. The important operands are rint: the value named ’rint’ in this block; image: the value named ’image’ in this block; image_scale: the real-value spacing represented by one quantized integer step. Context: the function infer, which handles infer.

Line 721: `image_u8 = np.clip(image_u8, 0, 255).astype(np.uint8)` This assigns ‘np.clip(image_u8, 0, 255).astype(np.uint to ‘image_u8 ‘ in host memory. Here image_u8 means the value named ’image u8’ in this block. The important operands are clip: the value named ’clip’ in this block; image_u8: the value named ’image u8’ in this block; astype: the value named ’astype’ in this block. Context: the function infer, which handles infer.

Line 722: `self.driver.execute_program((ProgramRecord(clear_status_command()),),` This statement continues through source line 723; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.driver.execute_program‘ with ‘(ProgramRecord(clear_status_command()),), timeout_seconds=self.layer_timeout_secon inside the function infer, which handles infer. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 723: `timeout_seconds=self.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 722. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds)‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This calls ‘self.driver.execute_program‘ with ‘(ProgramRecord(clear_status_command()),), timeout_seconds=self.layer_timeout_seconds‘ inside the function infer, which handles infer. The call advances the host-side job that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations.

Line 724: `value = self._tokenizer(image_u8)` This assigns ‘self._tokenizer(image_u8)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are _tokenizer: the value named ’tokenizer’ in this block; image_u8: the value named ’image u8’ in this block. Context: the function infer, which handles infer.

Line 725: `for block_index in range(4):` This iterates block_index over ‘range(4)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function infer, which handles infer.

Line 726: `attention = self._attention(block_index, value)` This assigns ‘self._attention(block_index, value)‘ to ‘attention ‘ in host memory. Here attention means the value named ’attention’ in this block. The important operands are _attention: the value named ’attention’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; value: the value named ’value’ in this block. Context: the function infer, which handles infer.

Line 727: `value = vector_binary_on_accelerator(` This statement continues through source line 729; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘vector_binary_on_accelerator(self.driver, VectorOperation.ADD_SAT, value, attention, timeout_seconds=self.layer_timeout_seconds)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are vector_binary_on_accelerator: the value named ’vector binary on accelerator’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function infer, which handles infer.

Line 728: `self.driver, VectorOperation.ADD_SAT, value, attention,` This is continuation line 2 of the 3-line logical statement that begins at source line 727. Its exact fragment is 'self.driver, VectorOperation.ADD_SAT, value, attention,'. The important operands are driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named 'add sat' in this block. Read together, the complete operation is: This assigns 'vector_binary_on_accelerator(self.driver, VectorOperation.ADD_SAT, value, attention, timeout_seconds=self.layer_timeout_seconds)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are vector_binary_on_accelerator: the value named 'vector binary on accelerator' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function infer, which handles infer.

Line 729: `timeout_seconds=self.layer_timeout_seconds)` This is continuation line 3 of the 3-line logical statement that begins at source line 727. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds)'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'vector_binary_on_accelerator(self.driver, VectorOperation.ADD_SAT, value, attention, timeout_seconds=self.layer_timeout_secon to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are vector_binary_on_accelerator: the value named 'vector binary on accelerator' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function infer, which handles infer.

Line 730: `mlp = self._mlp(block_index, value)` This assigns 'self._mlp(block_index, value)' to 'mlp ' in host memory. Here mlp means the value named 'mlp' in this block. The important operands are _mlp: the value named 'mlp' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; value: the value named 'value' in this block. Context: the function infer, which handles infer.

Line 731: `value = vector_binary_on_accelerator(` This statement continues through source line 733; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'vector_binary_on_accelerator(self.driver, VectorOperation.ADD_SAT, value, mlp, timeout_seconds=self.layer_timeout_seconds)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are vector_binary_on_accelerator: the value named 'vector binary on accelerator' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function infer, which handles infer.

Line 732: `self.driver, VectorOperation.ADD_SAT, value, mlp,` This is continuation line 2 of the 3-line logical statement that begins at source line 731. Its exact fragment is 'self.driver, VectorOperation.ADD_SAT, value, mlp,'. The important operands are driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named 'add sat' in this block. Read together, the complete operation is: This assigns 'vector_binary_on_accelerator(self.driver, VectorOperation.ADD_SAT, value, mlp, timeout_seconds=self.layer_timeout_seconds)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are vector_binary_on_accelerator: the value named 'vector binary on accelerator' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function infer, which handles infer.

Line 733: `timeout_seconds=self.layer_timeout_seconds)` This is continuation line 3 of the 3-line logical statement that begins at source line 731. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds)'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'vector_binary_on_accelerator(self.driver, VectorOperation.ADD_SAT, value, mlp, timeout_seconds=self.layer_timeout_seconds)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are vector_binary_on_accelerator: the value named 'vector binary on accelerator' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function infer, which handles infer.

Line 735: `pooled_sum = global_spatiotemporal_sum_on_accelerator(` This statement continues through source line 739; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'global_spatiotemporal_ self.driver, value, timeout_seconds=self.layer_timeout_seconds,)' to 'pooled_sum ' in host memory. Here pooled_sum means the value named 'pooled sum' in this block. The important operands are global_spatiotemporal_sum_on_accelerator:

the value named 'global spatiotemporal sum on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block. Context: the function infer, which handles infer.

Line 736: `self.driver,` This is continuation line 2 of the 5-line logical statement that begins at source line 735. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'global_spatiotemporal_sum_on_accelerator(self.driver, value, timeout_seconds=self.layer_timeout_seconds,)' to 'pooled_sum ' in host memory. Here pooled_sum means the value named 'pooled sum' in this block. The important operands are global_spatiotemporal_sum_on_accelerator: the value named 'global spatiotemporal sum on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block. Context: the function infer, which handles infer.

Line 737: `value,` This is continuation line 3 of the 5-line logical statement that begins at source line 735. Its exact fragment is 'value,'. The important operands are value: the value named 'value' in this block. Read together, the complete operation is: This assigns 'global_spatiotemporal_sum_on_accelerator(self.driver, value, timeout_seconds=self.layer_timeout_seco)' to 'pooled_sum ' in host memory. Here pooled_sum means the value named 'pooled sum' in this block. The important operands are global_spatiotemporal_sum_on_accelerator: the value named 'global spatiotemporal sum on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block. Context: the function infer, which handles infer.

Line 738: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 4 of the 5-line logical statement that begins at source line 735. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'global_spatiotemporal_sum_on_accele self.driver, value, timeout_seconds=self.layer_timeout_seconds,)' to 'pooled_sum ' in host memory. Here pooled_sum means the value named 'pooled sum' in this block. The important operands are global_spatiotemporal_sum_on_accelerator: the value named 'global spatiotemporal sum on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block. Context: the function infer, which handles infer.

Line 739: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 735. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'global_spatiotemporal_sum_on_accelerator(self.driver, value, timeout_seconds=self.layer_timeout_seconds,)' to 'pooled_sum ' in host memory. Here pooled_sum means the value named 'pooled sum' in this block. The important operands are global_spatiotemporal_sum_on_accelerator: the value named 'global spatiotemporal sum on accelerator' in this block; driver: the value named 'driver' in this block; value: the value named 'value' in this block. Context: the function infer, which handles infer.

Line 740: `divisor = value.shape[0] * value.shape[3] * value.shape[4]` This assigns 'value.shape[0] * value.shape[3] * value.shape[4]' to 'divisor ' in host memory. Here divisor means the value named 'divisor' in this block. The important operands are value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function infer, which handles infer.

Line 741: `denominator = divisor * self.fixed_scale * self.head_activation_scale` This assigns 'divisor * self.fixed_scale * self.head_activation_scale' to 'denominator ' in host memory. Here denominator means the value named 'denominator' in this block. The important operands are divisor: the value named 'divisor' in this block; fixed_scale: the real-value spacing represented by one quantized integer step; head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function infer, which handles infer.

Line 742: `integer_denominator = int(round(denominator))` This assigns 'int(round(denominator))' to 'integer_denominator ' in host memory. Here integer_denominator means the value named 'integer denominator' in this block. The important operands are round: the value named 'round' in this block; denominator: the value named 'denominator' in this block. Context: the function infer, which handles infer.

Line 743: `if (not math.isclose(denominator, integer_denominator, rel_tol=0.0, abs_tol=1e-9)` Inside the function infer, which handles infer, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are not: the

value named 'not' in this block; math: the value named 'math' in this block; isclose: the value named 'isclose' in this block.

Line 744: `or integer_denominator <= 0` Inside the function infer, which handles infer, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are or: the value named 'or' in this block; integer_denominator: the value named 'integer denominator' in this block.

Line 745: `or integer_denominator & (integer_denominator - 1)):` Inside the function infer, which handles infer, this statement advances the host implementation that maps all tokenizer, spiking attention, MLP, residual, reduction, and classifier stages to hardware operations. The important operands are or: the value named 'or' in this block; integer_denominator: the value named 'integer denominator' in this block.

Line 746: `raise ValueError("head pooling denominator must be an exact positive power of two")` This rejects an invalid state by raising 'ValueError("head pooling denominator must be an exact positive power of two")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 747: `head_integer = power_of_two_requantize_on_accelerator(` This statement continues through source line 754; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'power_of_two_requan self.driver, pooled_sum, division_shift=integer_denominator.bit_length() - 1, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,).astype(np.int16)' to 'head_integer ' in host memory. Here head_integer means the value named 'head integer' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function infer, which handles infer.

Line 748: `self.driver,` This is continuation line 2 of the 8-line logical statement that begins at source line 747. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(self.driver, pooled_sum, division_shift=integer_denominator.bit_length() - 1, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_tim).astype(np.int16)' to 'head_integer ' in host memory. Here head_integer means the value named 'head integer' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function infer, which handles infer.

Line 749: `pooled_sum,` This is continuation line 3 of the 8-line logical statement that begins at source line 747. Its exact fragment is 'pooled_sum,'. The important operands are pooled_sum: the value named 'pooled sum' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(self.driver, pooled_sum, division_shift=integer_denominator.bit_length() - 1, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_).astype(np.int16)' to 'head_integer ' in host memory. Here head_integer means the value named 'head integer' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function infer, which handles infer.

Line 750: `division_shift=integer_denominator.bit_length() - 1,` This is continuation line 4 of the 8-line logical statement that begins at source line 747. Its exact fragment is 'division_shift=integer_denominator.bit_length() - 1,'. The important operands are division_shift: the value named 'division shift' in this block; integer_denominator: the value named 'integer denominator' in this block; bit_length: the value named 'bit length' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(self.driver, pooled_sum, division_shift=integer_denominator.bit_length() - 1, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_tim).astype(np.int16)' to 'head_integer ' in host memory. Here head_integer means the value named 'head integer' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function infer, which handles infer.

Line 751: `clamp_minimum=-128,` This is continuation line 5 of the 8-line logical statement that begins at source line 747. Its exact fragment is 'clamp_minimum=-128,'. The important operands are clamp_minimum: the value named 'clamp minimum' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(self.driver, pooled_sum, division_shift=integer_denominator.bit_length() - 1, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,).astype(np.int16)' to 'head_integer ' in host memory. Here head_integer means the value named 'head integer' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function infer, which handles infer.

Line 752: `clamp_maximum=127,` This is continuation line 6 of the 8-line logical statement that begins at source line 747. Its exact fragment is 'clamp_maximum=127,'. The important operands are clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(self.driver, pooled_sum, division_shift=integer_denominator.bit_length() - 1, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,).astype(np.int16)' to 'head_integer ' in host memory. Here head_integer means the value named 'head integer' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function infer, which handles infer.

Line 753: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 7 of the 8-line logical statement that begins at source line 747. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelera self.driver, pooled_sum, division_shift=integer_denominator.bit_length() - 1, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,).astype(np.int16)' to 'head_integer ' in host memory. Here head_integer means the value named 'head integer' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function infer, which handles infer.

Line 754: `).astype(np.int16)` This is continuation line 8 of the 8-line logical statement that begins at source line 747. Its exact fragment is ').astype(np.int16)'. The important operands are astype: the value named 'astype' in this block; int16: the value named 'int16' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(self.driver, pooled_sum, division_shift=integer_denominator.bit_length() - 1, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,).astype(np.int16)' to 'head_integer ' in host memory. Here head_integer means the value named 'head integer' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function infer, which handles infer.

Line 755: `head = self.archive.head` This assigns 'self.archive.head' to 'head ' in host memory. Here head means the value named 'head' in this block. The important operands are archive: the value named 'archive' in this block; head: the value named 'head' in this block. Context: the function infer, which handles infer.

Line 756: `head_program = compile_u8_gemm(` This statement continues through source line 762; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'compile_u8_gemm((head_integer + 128).astype(np.uint8), self.projections._encoded_dense_weights(head.weight_k_by_n), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)' to 'head_program ' in host memory. Here head_program means the value named 'head program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; head_integer: the value named 'head integer' in this block; astype: the value named 'astype' in this block. Context: the function infer, which handles infer.

Line 757: `(head_integer + 128).astype(np.uint8),` This is continuation line 2 of the 7-line logical statement that begins at source line 756. Its exact fragment is '(head_integer + 128).astype(np.uint8),'. The important operands are head_integer: the value named 'head integer' in this block; astype: the value named 'astype' in this block; uint8: the value named 'uint8' in this block. Read together, the complete operation is: This assigns 'compile_u8_gemm((head_integer + 128).astype(np.uint8), self.projections._encoded_dense_weights(head.weight_k_by_n), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)' to 'head_program ' in host memory. Here head_program

means the value named 'head program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; head_integer: the value named 'head integer' in this block; astype: the value named 'astype' in this block. Context: the function infer, which handles infer.

Line 758: `self.projections._encoded_dense_weights(head.weight_k_by_n),` This is continuation line 3 of the 7-line logical statement that begins at source line 756. Its exact fragment is 'self.projections._encoded_dense_weights(head.weight_k_ The important operands are projections: the value named 'projections' in this block; _encoded_dense_weights: a learned coefficient multiplied by an activation; head: the value named 'head' in this block. Read together, the complete operation is: This assigns 'compile_u8_gemm((head_integer + 128).astype(np.uint8), self.projections._encoded_dense_weights(head.weight activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)' to 'head_program ' in host memory. Here head_program means the value named 'head program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; head_integer: the value named 'head integer' in this block; astype: the value named 'astype' in this block. Context: the function infer, which handles infer.

Line 759: `activation_zero_point=128,` This is continuation line 4 of the 7-line logical statement that begins at source line 756. Its exact fragment is 'activation_zero_point=128,'. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns 'compile_u8_gemm((head_integer + 128).astype(np.uint8), self.projections._encoded_dense_weights(head.weight_k_by_ activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)' to 'head_program ' in host memory. Here head_program means the value named 'head program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; head_integer: the value named 'head integer' in this block; astype: the value named 'astype' in this block. Context: the function infer, which handles infer.

Line 760: `weight_zero_point=128,` This is continuation line 5 of the 7-line logical statement that begins at source line 756. Its exact fragment is 'weight_zero_point=128,'. The important operands are weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'compile_u8_gemm((head_integer + 128).astype(np.uint8), self.projections._encoded_dense_weights(head.weight_k_by_n), activation_zero_point= weight_zero_point=128, bias=head.bias_accumulator_units,)' to 'head_program ' in host memory. Here head_program means the value named 'head program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; head_integer: the value named 'head integer' in this block; astype: the value named 'astype' in this block. Context: the function infer, which handles infer.

Line 761: `bias=head.bias_accumulator_units,` This is continuation line 6 of the 7-line logical statement that begins at source line 756. Its exact fragment is 'bias=head.bias_accumulator_units,'. The important operands are bias: an additive learned offset applied after accumulation; head: the value named 'head' in this block; bias_accumulator_units: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'compile_u8_gemm((head_integer + 128).astype(np.uint8), self.projections._encoded_dense_weights(head.weight_k_by_n), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)' to 'head_program ' in host memory. Here head_program means the value named 'head program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; head_integer: the value named 'head integer' in this block; astype: the value named 'astype' in this block. Context: the function infer, which handles infer.

Line 762: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 756. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'compile_u8_gemm((head_integer + 128).astype(np.uint8), self.projections._encoded_dense_weights(head.w activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)' to 'head_program ' in host memory. Here head_program means the value named 'head program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; head_integer: the value named 'head integer' in this block; astype: the value named 'astype' in this block. Context: the function infer, which handles infer.

Line 763: `logits_accumulator = head_program.execute_batched(` This statement continues through source line 764; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'head_program.execute_ba self.driver, timeout_seconds=self.layer_timeout_seconds)' to 'logits_accumulator ' in host memory. Here logits_accumulator means a widened running sum that prevents intermediate overflow. The important operands are head_program: the value named 'head program' in this block; execute_batched: the value named 'execute batched' in this block; driver: the value named 'driver' in this block. Context: the function infer, which handles infer.

Line 764: `self.driver, timeout_seconds=self.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 763. Its exact fragment is 'self.driver, timeout_seconds=self.layer_timeout_seconds)'. The important operands are driver: the value named 'driver' in this block; timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'head_program.execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)' to 'logits_accumulator ' in host memory. Here logits_accumulator means a widened running sum that prevents intermediate overflow. The important operands are head_program: the value named 'head program' in this block; execute_batched: the value named 'execute batched' in this block; driver: the value named 'driver' in this block. Context: the function infer, which handles infer.

Line 765: `head_manifest = self.archive.manifest["head"]` This assigns 'self.archive.manifest["head"]' to 'head_manifest ' in host memory. Here head_manifest means the value named 'head manifest' in this block. The important operands are archive: the value named 'archive' in this block; manifest: the value named 'manifest' in this block; head: the value named 'head' in this block. Context: the function infer, which handles infer.

Line 766: `accumulator_scale = float(head_manifest["accumulator_scale"])` This assigns 'float(head_manifest["accumula to 'accumulator_scale ' in host memory. Here accumulator_scale means a widened running sum that prevents intermediate overflow. The important operands are float: the value named 'float' in this block; head_manifest: the value named 'head manifest' in this block; accumulator_scale: a widened running sum that prevents intermediate overflow. Context: the function infer, which handles infer.

Line 767: `logits = logits_accumulator.astype(np.float64) * accumulator_scale` This assigns 'logits_accumulator.astyp * accumulator_scale' to 'logits ' in host memory. Here logits means the ten unnormalized classifier scores used for argmax. The important operands are logits_accumulator: a widened running sum that prevents intermediate overflow; astype: the value named 'astype' in this block; float64: the value named 'float64' in this block. Context: the function infer, which handles infer.

Line 768: `return SpikingformerInferenceResult(` This statement continues through source line 772; the explanation uses the complete expression rather than only the first visual fragment. This returns 'SpikingformerInferenceResult(logits_accumulator=logits_accumulator, logits=logits, predicted_class=np.argmax(logits_accumulator, axis=1),)' to the caller of the function infer, which handles infer, ending this invocation. The important operands are SpikingformerInferenceResult: the value named 'spikingformer inference result' in this block; logits_accumulator: a widened running sum that prevents intermediate overflow; logits: the ten unnormalized classifier scores used for argmax.

Line 769: `logits_accumulator=logits_accumulator,` This is continuation line 2 of the 5-line logical statement that begins at source line 768. Its exact fragment is 'logits_accumulator=logits_accumulator,'. The important operands are logits_accumulator: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This returns 'SpikingformerInferenceResult(logits_accumulator=logits_accumulator, logits=logits, predicted_class=np.argmax(logits_accumulator, axis=1),)' to the caller of the function infer, which handles infer, ending this invocation. The important operands are SpikingformerInferenceResult: the value named 'spikingformer inference result' in this block; logits_accumulator: a widened running sum that prevents intermediate overflow; logits: the ten unnormalized classifier scores used for argmax.

Line 770: `logits=logits,` This is continuation line 3 of the 5-line logical statement that begins at source line 768. Its exact fragment is 'logits=logits,'. The important operands are logits: the ten unnormalized classifier scores used for argmax. Read together, the complete operation is: This returns 'SpikingformerInferenceResult(logits_accumulator=logits_accumul logits=logits, predicted_class=np.argmax(logits_accumulator, axis=1),)' to the caller of the function infer, which handles infer, ending this invocation. The important operands are SpikingformerInferenceResult: the value named 'spikingformer inference result' in this block; logits_accumulator: a widened running sum that prevents intermediate overflow; logits: the ten unnormalized classifier scores used for argmax.

Line 771: `predicted_class=np.argmax(logits_accumulator, axis=1),` This is continuation line 4 of the 5-line logical statement that begins at source line 768. Its exact fragment is 'predicted_class=np.argmax(logits_accumulator, axis=1),'. The important operands are predicted_class: the value named 'predicted class' in this block; argmax: the value named 'argmax' in this block; logits_accumulator: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This returns 'SpikingformerInferenceResult(logits_accumulator=logits_accumulator,

logits=logits, predicted_class=np.argmax(logits_accumulator, axis=1),)‘ to the caller of the function infer, which handles infer, ending this invocation. The important operands are SpikingformerInferenceResult: the value named 'spikingformer inference result' in this block; logits_accumulator: a widened running sum that prevents intermediate overflow; logits: the ten unnormalized classifier scores used for argmax.

Line 772:) This is continuation line 5 of the 5-line logical statement that begins at source line 768. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘SpikingformerInferenceResult(logits_accumulator=logits_accumulator, logits=logits, predicted_class=np.argmax(logits_accumulator, axis=1),)‘ to the caller of the function infer, which handles infer, ending this invocation. The important operands are SpikingformerInferenceResult: the value named 'spikingformer inference result' in this block; logits_accumulator: a widened running sum that prevents intermediate overflow; logits: the ten unnormalized classifier scores used for argmax.

41 software/spikingformer_parameters.py

Role in the complete system: validates and exposes typed read-only views of the aligned parameter archive.

41.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to validates and exposes typed read-only views of the aligned parameter archive.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: ProjectionParameters, HeadParameters, SpikingformerParameterArchive. Functions: ___init___, _integer_field, _nonnegative_dimension, _verify_hash, _projection_objects, _section_names, section, projection_names, projection, head. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

41.2 Complete source

```
"""Strict reader for the exported NeuraEdge Spikingformer parameter archive."""

from __future__ import annotations

import hashlib
import json
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Mapping

import numpy as np


FORMAT = "neuraedge-spikingformer-parameters-v1"


@dataclass(frozen=True)
class ProjectionParameters:
    """Integer tensors consumed by one folded Conv-BN hardware projection."""

    name: str
    weight_k_by_n: np.ndarray
    bias_q10: np.ndarray
    multiplier_q15: np.ndarray
    binary_point_shift: np.ndarray
    compute_opcode: str
    source_weight_shape: tuple[int, ...]
    output_fractional_bits: int


@dataclass(frozen=True)
class HeadParameters:
    """Integer classifier matrix and bias in accumulator units."""

    weight_k_by_n: np.ndarray
    bias_accumulator_units: np.ndarray


class SpikingformerParameterArchive:
    """Validated, read-only view of one binary parameter package."""

    def __init__(self, manifest_path: str | Path, *, verify_hashes: bool = True) -> None:
        self.manifest_path = Path(manifest_path).expanduser().resolve()
        manifest_object = json.loads(self.manifest_path.read_text(encoding="ascii"))
        if not isinstance(manifest_object, Mapping):
            raise ValueError("parameter manifest root must be an object")
        self.manifest: Mapping[str, Any] = manifest_object
        if self.manifest.get("format") != FORMAT:
            raise ValueError(f"unsupported parameter format {self.manifest.get('format')!r}")

        binary_name = self.manifest.get("binary_file")
        if not isinstance(binary_name, str) or Path(binary_name).name != binary_name:
```

```
            raise ValueError("binary_file must be a file name beside the manifest")
        self.binary_path = self.manifest_path.parent / binary_name
        self._binary = self.binary_path.read_bytes()
        declared_bytes = self._integer_field(self.manifest, "binary_bytes")
        if len(self._binary) != declared_bytes:
            raise ValueError(
                f"parameter binary has {len(self._binary)} bytes; expected {declared_bytes}")
        if verify_hashes:
            self._verify_hash(self._binary, self.manifest.get("binary_sha256"), "binary")

        alignment = self._integer_field(self.manifest, "alignment_bytes")
        if alignment <= 0 or alignment & (alignment - 1):
            raise ValueError("alignment_bytes must be a positive power of two")
        section_objects = self.manifest.get("sections")
        if not isinstance(section_objects, list) or not section_objects:
            raise ValueError("manifest must contain a nonempty sections list")
        self._sections: dict[str, np.ndarray] = {}
        occupied_end = 0
        for section_object in section_objects:
            if not isinstance(section_object, Mapping):
                raise ValueError("each section must be an object")
            name = section_object.get("name")
            if not isinstance(name, str) or not name or name in self._sections:
                raise ValueError(f"invalid or duplicate section name {name!r}")
            offset = self._integer_field(section_object, "offset")
            length = self._integer_field(section_object, "length")
            if offset % alignment:
                raise ValueError(f"section {name} offset {offset} is not {alignment}-byte aligned")
            if offset < occupied_end:
                raise ValueError(f"section {name} overlaps a preceding section")
            if length < 0 or offset + length > len(self._binary):
                raise ValueError(f"section {name} is outside the parameter binary")
            dtype_value = section_object.get("dtype")
            shape_value = section_object.get("shape")
            if not isinstance(dtype_value, str) or not isinstance(shape_value, list):
                raise ValueError(f"section {name} has invalid dtype or shape metadata")
            dtype = np.dtype(dtype_value)
            shape = tuple(self._nonnegative_dimension(value, name) for value in shape_value)
            expected_length = int(np.prod(shape, dtype=np.int64)) * dtype.itemsize
            if length != expected_length:
                raise ValueError(
                    f"section {name} has {length} bytes; dtype/shape require {expected_length}")
            payload = self._binary[offset:offset + length]
            if verify_hashes:
                self._verify_hash(payload, section_object.get("sha256"), f"section {name}")
            array = np.frombuffer(payload, dtype=dtype).reshape(shape)
            array.flags.writeable = False
            self._sections[name] = array
            occupied_end = offset + length

        declared_section_names = set(self._sections)
        referenced_section_names: set[str] = set()
        for projection in self._projection_objects():
            referenced_section_names.update(self._section_names(projection, "projection"))
        head = self.manifest.get("head")
        if not isinstance(head, Mapping):
            raise ValueError("manifest head must be an object")
        referenced_section_names.update(self._section_names(head, "head"))
        if referenced_section_names != declared_section_names:
            missing = sorted(declared_section_names - referenced_section_names)
            unknown = sorted(referenced_section_names - declared_section_names)
            raise ValueError(f"section reference mismatch: unreferenced={missing}, unknown={unknown}")

    @staticmethod
    def _integer_field(mapping: Mapping[str, Any], name: str) -> int:
        value = mapping.get(name)
        if isinstance(value, bool) or not isinstance(value, int):
            raise ValueError(f"{name} must be an integer")
        return value

    @staticmethod
    def _nonnegative_dimension(value: Any, section_name: str) -> int:
        if isinstance(value, bool) or not isinstance(value, int) or value < 0:
            raise ValueError(f"section {section_name} has an invalid shape dimension")
        return value

    @staticmethod
    def _verify_hash(payload: bytes, expected: Any, description: str) -> None:
        if not isinstance(expected, str) or len(expected) != 64:
            raise ValueError(f"{description} SHA-256 metadata is invalid")
        actual = hashlib.sha256(payload).hexdigest()
        if actual != expected.lower():
            raise ValueError(f"{description} SHA-256 mismatch: {actual} != {expected}")

    def _projection_objects(self) -> list[Mapping[str, Any]]:
        projections = self.manifest.get("projections")
        if not isinstance(projections, list):
```

```
            raise ValueError("manifest projections must be a list")
        if not all(isinstance(item, Mapping) for item in projections):
            raise ValueError("each projection must be an object")
        return projections

    @staticmethod
    def _section_names(owner: Mapping[str, Any], owner_kind: str) -> list[str]:
        sections = owner.get("sections")
        if not isinstance(sections, list):
            raise ValueError(f"{owner_kind} sections must be a list")
        names: list[str] = []
        for section in sections:
            if not isinstance(section, Mapping) or not isinstance(section.get("name"), str):
                raise ValueError(f"{owner_kind} has an invalid section reference")
            names.append(section["name"])
        return names

    def section(self, name: str) -> np.ndarray:
        try:
            return self._sections[name]
        except KeyError as error:
            raise KeyError(f"parameter section {name!r} does not exist") from error

    @property
    def projection_names(self) -> tuple[str, ...]:
        return tuple(str(item.get("name")) for item in self._projection_objects())

    def projection(self, name: str) -> ProjectionParameters:
        matches = [item for item in self._projection_objects() if item.get("name") == name]
        if len(matches) != 1:
            raise KeyError(f"projection {name!r} does not exist uniquely")
        owner = matches[0]
        section_names = self._section_names(owner, f"projection {name}")
        expected_suffixes = (
            ".weight_k_by_n_int8", ".bias_q10", ".multiplier_q15",
            ".binary_point_shift",
        )
        selected: list[np.ndarray] = []
        for suffix in expected_suffixes:
            suffix_matches = [entry for entry in section_names if entry.endswith(suffix)]
            if len(suffix_matches) != 1:
                raise ValueError(f"projection {name} must reference one {suffix} section")
            selected.append(self.section(suffix_matches[0]))
        weight, bias, multiplier, shift = selected
        if weight.ndim != 2 or any(value.ndim != 1 for value in selected[1:]):
            raise ValueError(f"projection {name} section ranks are invalid")
        if not (weight.shape[1] == len(bias) == len(multiplier) == len(shift)):
            raise ValueError(f"projection {name} output-channel metadata is inconsistent")
        source_shape_object = owner.get("source_weight_shape")
        if not isinstance(source_shape_object, list):
            raise ValueError(f"projection {name} has no source weight shape")
        source_shape = tuple(
            self._nonnegative_dimension(value, f"projection {name}")
            for value in source_shape_object)
        if len(source_shape) not in (3, 4) or source_shape[0] != weight.shape[1]:
            raise ValueError(f"projection {name} source weight shape is inconsistent")
        fractional_bits = self._integer_field(owner, "output_fractional_bits")
        return ProjectionParameters(
            name, weight, bias, multiplier, shift, str(owner.get("compute_opcode")),
            source_shape, fractional_bits)

    @property
    def head(self) -> HeadParameters:
        owner = self.manifest["head"]
        names = self._section_names(owner, "head")
        weight_names = [name for name in names if name.endswith(".weight_k_by_n_int8")]
        bias_names = [name for name in names if name.endswith(".bias_accumulator_units")]
        if len(weight_names) != 1 or len(bias_names) != 1:
            raise ValueError("head must reference one weight and one bias section")
        weight = self.section(weight_names[0])
        bias = self.section(bias_names[0])
        if weight.ndim != 2 or bias.ndim != 1 or weight.shape[1] != len(bias):
            raise ValueError("head section shapes are inconsistent")
        return HeadParameters(weight, bias)
```

41.3 Line annotations

Line 1: `"""Strict reader for the exported NeuraEdge Spikingformer parameter archive."""` This is executable documentation attached to the file whose role is to validates and exposes typed read-only views of the aligned parameter archive. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to validates and exposes typed read-only views of the aligned parameter archive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import hashlib` This imports the named Python API so the file whose role is to validates and exposes typed read-only views of the aligned parameter archive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import json` This imports the named Python API so the file whose role is to validates and exposes typed read-only views of the aligned parameter archive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to validates and exposes typed read-only views of the aligned parameter archive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `from pathlib import Path` This imports the named Python API so the file whose role is to validates and exposes typed read-only views of the aligned parameter archive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `from typing import Any, Mapping` This imports the named Python API so the file whose role is to validates and exposes typed read-only views of the aligned parameter archive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `import numpy as np` This imports the named Python API so the file whose role is to validates and exposes typed read-only views of the aligned parameter archive can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `FORMAT = "neuraedge-spikingformer-parameters-v1"` This assigns ‘"neuraedge-spikingformer-parameters-v1"‘ to ‘FORMAT ‘ in host memory. Here FORMAT means the value named ’format’ in this block. The important operands are neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block; parameters: the value named ’parameters’ in this block. Context: the file whose role is to validates and exposes typed read-only views of the aligned parameter archive.

Line 17: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to validates and exposes typed read-only views of the aligned parameter archive; it creates no FPGA circuit by itself.

Line 18: `class ProjectionParameters:` This defines class ProjectionParameters, which groups data and methods for projection parameters. Instances exist in host memory and organize the flow that validates and exposes typed read-only views of the aligned parameter archive.

Line 19: `"""Integer tensors consumed by one folded Conv-BN hardware projection."""` This is executable documentation attached to the class ProjectionParameters, which handles projection parameters. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 21: `name: str` Inside the class ProjectionParameters, which handles projection parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are name: the value named ’name’ in this block; str: the value named ’str’ in this block.

Line 22: `weight_k_by_n: np.ndarray` Inside the class ProjectionParameters, which handles projection parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are weight_k_by_n: a learned coefficient multiplied by an activation; ndarray: the value named ’ndarray’ in this block.

Line 23: `bias_q10: np.ndarray` Inside the class ProjectionParameters, which handles projection parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are bias_q10: an additive learned offset applied after accumulation; ndarray: the value named 'ndarray' in this block.

Line 24: `multiplier_q15: np.ndarray` Inside the class ProjectionParameters, which handles projection parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are multiplier_q15: the value named 'multiplier q15' in this block; ndarray: the value named 'ndarray' in this block.

Line 25: `binary_point_shift: np.ndarray` Inside the class ProjectionParameters, which handles projection parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are binary_point_shift: the value named 'binary point shift' in this block; ndarray: the value named 'ndarray' in this block.

Line 26: `compute_opcode: str` Inside the class ProjectionParameters, which handles projection parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are compute_opcode: the value named 'compute opcode' in this block; str: the value named 'str' in this block.

Line 27: `source_weight_shape: tuple[int, ...]` Inside the class ProjectionParameters, which handles projection parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are source_weight_shape: a learned coefficient multiplied by an activation; tuple: the value named 'tuple' in this block.

Line 28: `output_fractional_bits: int` Inside the class ProjectionParameters, which handles projection parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are output_fractional_bits: the value named 'output fractional bits' in this block.

Line 31: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to validates and exposes typed read-only views of the aligned parameter archive; it creates no FPGA circuit by itself.

Line 32: `class HeadParameters:` This defines class HeadParameters, which groups data and methods for head parameters. Instances exist in host memory and organize the flow that validates and exposes typed read-only views of the aligned parameter archive.

Line 33: `"""Integer classifier matrix and bias in accumulator units."""` This is executable documentation attached to the class HeadParameters, which handles head parameters. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 35: `weight_k_by_n: np.ndarray` Inside the class HeadParameters, which handles head parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are weight_k_by_n: a learned coefficient multiplied by an activation; ndarray: the value named 'ndarray' in this block.

Line 36: `bias_accumulator_units: np.ndarray` Inside the class HeadParameters, which handles head parameters, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are bias_accumulator_units: an additive learned offset applied after accumulation; ndarray: the value named 'ndarray' in this block.

Line 39: `class SpikingformerParameterArchive:` This defines class SpikingformerParameterArchive, which groups data and methods for spikingformer parameter archive. Instances exist in host memory and organize the flow that validates and exposes typed read-only views of the aligned parameter archive.

Line 40: `"""Validated, read-only view of one binary parameter package."""` This is executable documentation attached to the class SpikingformerParameterArchive, which handles spikingformer parameter archive. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 42: `def __init__(self, manifest_path: str | Path, *, verify_hashes: bool = True) -> None:` This starts function ___init___. Its indented body performs init as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 43: `self.manifest_path = Path(manifest_path).expanduser().resolve()` This assigns ‘Path(manifest_path).expandu to ‘self.manifest_path ‘ in host memory. Here manifest_path means the value named ’manifest path’ in this block. The important operands are Path: the value named ’path’ in this block; manifest_path: the value named ’manifest path’ in this block; expanduser: the value named ’expanduser’ in this block. Context: the function ___init___, which handles init.

Line 44: `manifest_object = json.loads(self.manifest_path.read_text(encoding="ascii"))` This assigns ‘json.loads(self to ‘manifest_object ‘ in host memory. Here manifest_object means the value named ’manifest object’ in this block. The important operands are json: the value named ’json’ in this block; loads: the value named ’loads’ in this block; manifest_path: the value named ’manifest path’ in this block. Context: the function ___init___, which handles init.

Line 45: `if not isinstance(manifest_object, Mapping):` This executes the following indented host-code branch only when ‘not isinstance(manifest_object, Mapping)‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; manifest_object: the value named ’manifest object’ in this block. Context: the function ___init___, which handles init.

Line 46: `raise ValueError("parameter manifest root must be an object")` This rejects an invalid state by raising ‘ValueError("parameter manifest root must be an object")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 47: `self.manifest: Mapping[str, Any] = manifest_object` This assigns ‘manifest_object‘ to ‘self.manifest‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are manifest_object: the value named ’manifest object’ in this block. Context: the function ___init___, which handles init.

Line 48: `if self.manifest.get("format") != FORMAT:` This executes the following indented host-code branch only when ‘self.manifest.get("format") != FORMAT‘ is true. The important operands are manifest: the value named ’manifest’ in this block; get: the value named ’get’ in this block; format: the value named ’format’ in this block. Context: the function ___init___, which handles init.

Line 49: `raise ValueError(f"unsupported parameter format {self.manifest.get('format')!r}")` This rejects an invalid state by raising ‘ValueError(f"unsupported parameter format {self.manifest.get(’format’)!r}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 51: `binary_name = self.manifest.get("binary_file")` This assigns ‘self.manifest.get("binary_file")‘ to ‘binary_name ‘ in host memory. Here binary_name means the value named ’binary name’ in this block. The important operands are manifest: the value named ’manifest’ in this block; get: the value named ’get’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function ___init___, which handles init.

Line 52: `if not isinstance(binary_name, str) or Path(binary_name).name != binary_name:` This executes the following indented host-code branch only when ‘not isinstance(binary_name, str) or Path(binary_name).name != binary_name‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; binary_name: the value named ’binary name’ in this block. Context: the function ___init___, which handles init.

Line 53: `raise ValueError("binary_file must be a file name beside the manifest")` This rejects an invalid state by raising ‘ValueError("binary_file must be a file name beside the manifest")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 54: `self.binary_path = self.manifest_path.parent / binary_name` This assigns ‘self.manifest_path.parent / binary_name‘ to ‘self.binary_path ‘ in host memory. Here binary_path means the value named ’binary path’ in this block. The important operands are manifest_path: the value named ’manifest path’ in this block; parent: the value named ’parent’ in this block; binary_name: the value named ’binary name’ in this block. Context: the function ___init___, which handles init.

Line 55: `self._binary = self.binary_path.read_bytes()` This assigns ‘self.binary_path.read_bytes()‘ to ‘self._binary ‘ in host memory. Here _binary means the value named ’binary’ in this block. The important operands are binary_path: the value named ’binary path’ in this block; read_bytes: the value named ’read bytes’ in this block. Context: the function ___init___, which handles init.

Line 56: `declared_bytes = self._integer_field(self.manifest, "binary_bytes")` This assigns ‘self._integer_field(self.m "binary_bytes")‘ to ‘declared_bytes ‘ in host memory. Here declared_bytes means the value named ’declared bytes’ in this block. The important operands are _integer_field: the value named ’integer field’ in this block; manifest: the value named ’manifest’ in this block; binary_bytes: the value named ’binary bytes’ in this block. Context: the function ___init___, which handles init.

Line 57: `if len(self._binary) != declared_bytes:` This executes the following indented host-code branch only when ‘len(self._binary) != declared_bytes‘ is true. The important operands are len: the value named ’len’ in this block; _binary: the value named ’binary’ in this block; declared_bytes: the value named ’declared bytes’ in this block. Context: the function ___init___, which handles init.

Line 58: `raise ValueError(` This statement continues through source line 59; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘ValueError(f"parameter binary has {len(self._binary)} bytes; expected {declared_bytes}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 59: `f"parameter binary has {len(self._binary)} bytes; expected {declared_bytes}")` This is continuation line 2 of the 2-line logical statement that begins at source line 58. Its exact fragment is ‘f"parameter binary has {len(self._binary)} bytes; expected {declared_bytes}")‘. The important operands are f: the value named ’f’ in this block; parameter: the value named ’parameter’ in this block; binary: the value named ’binary’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘ValueError(f"parameter binary has {len(self._binary)} bytes; expected {declared_bytes}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 60: `if verify_hashes:` This executes the following indented host-code branch only when ‘verify_hashes‘ is true. The important operands are verify_hashes: the value named ’verify hashes’ in this block. Context: the function ___init___, which handles init.

Line 61: `self._verify_hash(self._binary, self.manifest.get("binary_sha256"), "binary")` This calls ‘self._verify_ha with ‘self._binary, self.manifest.get("binary_sha256"), "binary"‘ inside the function ___init___, which handles init. The call advances the host-side job that validates and exposes typed read-only views of the aligned parameter archive.

Line 63: `alignment = self._integer_field(self.manifest, "alignment_bytes")` This assigns ‘self._integer_field(self.man "alignment_bytes")‘ to ‘alignment ‘ in host memory. Here alignment means the value named ’alignment’ in this block. The important operands are _integer_field: the value named ’integer field’ in this block; manifest: the value named ’manifest’ in this block; alignment_bytes: the value named ’alignment bytes’ in this block. Context: the function ___init___, which handles init.

Line 64: `if alignment <= 0 or alignment & (alignment - 1):` This executes the following indented host-code branch only when ‘alignment <= 0 or alignment & (alignment - 1)‘ is true. The important operands are alignment: the value named ’alignment’ in this block; or: the value named ’or’ in this block. Context: the function ___init___, which handles init.

Line 65: `raise ValueError("alignment_bytes must be a positive power of two")` This rejects an invalid state by raising ‘ValueError("alignment_bytes must be a positive power of two")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 66: `section_objects = self.manifest.get("sections")` This assigns ‘self.manifest.get("sections")‘ to ‘section_objects ‘ in host memory. Here section_objects means the value named ’section objects’ in this block. The important operands are manifest: the value named ’manifest’ in this block; get: the value named ’get’ in this block; sections: the value named ’sections’ in this block. Context: the function ___init___, which handles init.

Line 67: `if not isinstance(section_objects, list) or not section_objects:` This executes the following indented host-code branch only when ‘not isinstance(section_objects, list) or not section_objects‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; section_objects: the value named ’section objects’ in this block. Context: the function ___init___, which handles init.

Line 68: `raise ValueError("manifest must contain a nonempty sections list")` This rejects an invalid state by raising ‘ValueError("manifest must contain a nonempty sections list")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 69: `self._sections: dict[str, np.ndarray] = {}` This assigns ‘{}‘ to ‘self._sections‘ in host memory. Here _sections means the value named ’sections’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 70: `occupied_end = 0` This assigns ‘0‘ to ‘occupied_end ‘ in host memory. Here occupied_end means the value named ’occupied end’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 71: `for section_object in section_objects:` This iterates section_object over ‘section_objects‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function ___init___, which handles init.

Line 72: `if not isinstance(section_object, Mapping):` This executes the following indented host-code branch only when ‘not isinstance(section_object, Mapping)‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; section_object: the value named ’section object’ in this block. Context: the function ___init___, which handles init.

Line 73: `raise ValueError("each section must be an object")` This rejects an invalid state by raising ‘ValueError("each section must be an object")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 74: `name = section_object.get("name")` This assigns ‘section_object.get("name")‘ to ‘name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are section_object: the value named ’section object’ in this block; get: the value named ’get’ in this block; name: the value named ’name’ in this block. Context: the function ___init___, which handles init.

Line 75: `if not isinstance(name, str) or not name or name in self._sections:` This executes the following indented host-code branch only when ‘not isinstance(name, str) or not name or name in self._sections‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; name: the value named ’name’ in this block. Context: the function ___init___, which handles init.

Line 76: `raise ValueError(f"invalid or duplicate section name {name!r}")` This rejects an invalid state by raising ‘ValueError(f"invalid or duplicate section name {name!r}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 77: `offset = self._integer_field(section_object, "offset")` This assigns ‘self._integer_field(section_object, "offset")‘ to ‘offset ‘ in host memory. Here offset means the value named ’offset’ in this block. The important operands are _integer_field: the value named ’integer field’ in this block; section_object: the value named ’section object’ in this block; offset: the value named ’offset’ in this block. Context: the function ___init___, which handles init.

Line 78: `length = self._integer_field(section_object, "length")` This assigns ‘self._integer_field(section_object, "length")‘ to ‘length ‘ in host memory. Here length means the value named ’length’ in this block. The important operands are _integer_field: the value named ’integer field’ in this block; section_object: the value named ’section object’ in this block; length: the value named ’length’ in this block. Context: the function ___init___, which handles init.

Line 79: `if offset % alignment:` This executes the following indented host-code branch only when ‘offset % alignment‘ is true. The important operands are offset: the value named ’offset’ in this block; alignment: the value named ’alignment’ in this block. Context: the function ___init___, which handles init.

Line 80: `raise ValueError(f"section {name} offset {offset} is not {alignment}-byte aligned")` This rejects an invalid state by raising ‘ValueError(f"section {name} offset {offset} is not {alignment}-byte aligned")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 81: `if offset < occupied_end:` This executes the following indented host-code branch only when ‘offset < occupied_end‘ is true. The important operands are offset: the value named ’offset’ in this block; occupied_end: the value named ’occupied end’ in this block. Context: the function ___init___, which handles init.

Line 82: `raise ValueError(f"section {name} overlaps a preceding section")` This rejects an invalid state by raising ‘ValueError(f"section {name} overlaps a preceding section")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 83: `if length < 0 or offset + length > len(self._binary):` This executes the following indented host-code branch only when ‘length < 0 or offset + length > len(self._binary)‘ is true. The important operands are length: the value named ’length’ in this block; or: the value named ’or’ in this block; offset: the value named ’offset’ in this block. Context: the function ___init___, which handles init.

Line 84: `raise ValueError(f"section {name} is outside the parameter binary")` This rejects an invalid state by raising ‘ValueError(f"section {name} is outside the parameter binary")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 85: `dtype_value = section_object.get("dtype")` This assigns ‘section_object.get("dtype")‘ to ‘dtype_value ‘ in host memory. Here dtype_value means the value named ’dtype value’ in this block. The important operands are section_object: the value named ’section object’ in this block; get: the value named ’get’ in this block; dtype: the value named ’dtype’ in this block. Context: the function ___init___, which handles init.

Line 86: `shape_value = section_object.get("shape")` This assigns ‘section_object.get("shape")‘ to ‘shape_value ‘ in host memory. Here shape_value means the value named ’shape value’ in this block. The important operands are section_object: the value named ’section object’ in this block; get: the value named ’get’ in this block; shape: the value named ’shape’ in this block. Context: the function ___init___, which handles init.

Line 87: `if not isinstance(dtype_value, str) or not isinstance(shape_value, list):` This executes the following indented host-code branch only when ‘not isinstance(dtype_value, str) or not isinstance(shape_value, list)‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; dtype_value: the value named ’dtype value’ in this block. Context: the function ___init___, which handles init.

Line 88: `raise ValueError(f"section {name} has invalid dtype or shape metadata")` This rejects an invalid state by raising ‘ValueError(f"section {name} has invalid dtype or shape metadata")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 89: `dtype = np.dtype(dtype_value)` This assigns ‘np.dtype(dtype_value)‘ to ‘dtype ‘ in host memory. Here dtype means the value named ’dtype’ in this block. The important operands are dtype: the value named ’dtype’ in this block; dtype_value: the value named ’dtype value’ in this block. Context: the function ___init___, which handles init.

Line 90: `shape = tuple(self._nonnegative_dimension(value, name) for value in shape_value)` This assigns ‘tuple(self._nonnegative_dimension(value, name) for value in shape_value)‘ to ‘shape ‘ in host memory. Here shape means the value named ’shape’ in this block. The important operands are tuple: the value named ’tuple’ in this block; _nonnegative_dimension: the value named ’nonnegative dimension’ in this block; value: the value named ’value’ in this block. Context: the function ___init___, which handles init.

Line 91: `expected_length = int(np.prod(shape, dtype=np.int64)) * dtype.itemsize` This assigns ‘int(np.prod(shape, dtype=np.int64)) * dtype.itemsize‘ to ‘expected_length ‘ in host memory. Here expected_length means the value named ’expected length’ in this block. The important operands are prod: the value named ’prod’ in this block; shape: the value named ’shape’ in this block; dtype: the value named ’dtype’ in this block. Context: the function ___init___, which handles init.

Line 92: `if length != expected_length:` This executes the following indented host-code branch only when ‘length != expected_length‘ is true. The important operands are length: the value named ’length’ in this block; expected_length: the value named ’expected length’ in this block. Context: the function ___init___, which handles init.

Line 93: `raise ValueError(` This statement continues through source line 94; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘ValueError(f"section {name} has {length} bytes; dtype/shape require {expected_length}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 94: `f"section {name} has {length} bytes; dtype/shape require {expected_length}")` This is continuation line 2 of the 2-line logical statement that begins at source line 93. Its exact fragment is ‘f"section {name} has {length} bytes; dtype/shape require {expected_length}")‘. The important operands are f: the value named ’f’ in this block; section: the value named ’section’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘ValueError(f"section {name} has {length} bytes; dtype/shape require {expected_length}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 95: `payload = self._binary[offset:offset + length]` This assigns ‘self._binary[offset:offset + length]‘ to ‘payload ‘ in host memory. Here payload means stream data belonging to the current command. The important operands are _binary: the value named ’binary’ in this block; offset: the value named ’offset’ in this block; length: the value named ’length’ in this block. Context: the function ___init___, which handles init.

Line 96: `if verify_hashes:` This executes the following indented host-code branch only when ‘verify_hashes‘ is true. The important operands are verify_hashes: the value named ’verify hashes’ in this block. Context: the function ___init___, which handles init.

Line 97: `self._verify_hash(payload, section_object.get("sha256"), f"section {name}")` This calls ‘self._verify_hash with ‘payload, section_object.get("sha256"), f"section {name}"‘ inside the function ___init___, which handles init. The call advances the host-side job that validates and exposes typed read-only views of the aligned parameter archive.

Line 98: `array = np.frombuffer(payload, dtype=dtype).reshape(shape)` This assigns ‘np.frombuffer(payload, dtype=dtype).reshape(shape)‘ to ‘array ‘ in host memory. Here array means the value named ’array’ in this block. The important operands are frombuffer: local storage that decouples producer and consumer timing; payload: stream data belonging to the current command; dtype: the value named ’dtype’ in this block. Context: the function ___init___, which handles init.

Line 99: `array.flags.writeable = False` This assigns ‘False‘ to ‘array.flags.writeable ‘ in host memory. Here writeable means the value named ’writeable’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 100: `self._sections[name] = array` This assigns ‘array‘ to ‘self._sections[name] ‘ in host memory. Here _sections means the value named ’sections’ in this block. The important operands are array: the value named ’array’ in this block. Context: the function ___init___, which handles init.

Line 101: `occupied_end = offset + length` This assigns ‘offset + length‘ to ‘occupied_end ‘ in host memory. Here occupied_end means the value named ’occupied end’ in this block. The important operands are offset: the value named ’offset’ in this block; length: the value named ’length’ in this block. Context: the function ___init___, which handles init.

Line 103: `declared_section_names = set(self._sections)` This assigns ‘set(self._sections)‘ to ‘declared_section_names ‘ in host memory. Here declared_section_names means the value named ’declared section names’ in this block. The important operands are set: the value named ’set’ in this block; _sections: the value named ’sections’ in this block. Context: the function ___init___, which handles init.

Line 104: `referenced_section_names: set[str] = set()` This assigns ‘set()‘ to ‘referenced_section_names‘ in host memory. Here referenced_section_names means the value named ’referenced section names’ in this block. The important operands are set: the value named ’set’ in this block. Context: the function ___init___, which handles init.

Line 105: `for projection in self._projection_objects():` This iterates projection over ‘self._projection_objects()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function ___init___, which handles init.

Line 106: `referenced_section_names.update(self._section_names(projection, "projection"))` This calls ‘referenced_section_names.update‘ with ‘self._section_names(projection, "projection")‘ inside the function ___init___, which handles init. The call advances the host-side job that validates and exposes typed read-only views of the aligned parameter archive.

Line 107: `head = self.manifest.get("head")` This assigns ‘self.manifest.get("head")‘ to ‘head ‘ in host memory. Here head means the value named ’head’ in this block. The important operands are manifest: the value named ’manifest’ in this block; get: the value named ’get’ in this block; head: the value named ’head’ in this block. Context: the function ___init___, which handles init.

Line 108: `if not isinstance(head, Mapping):` This executes the following indented host-code branch only when ‘not isinstance(head, Mapping)‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; head: the value named ’head’ in this block. Context: the function ___init___, which handles init.

Line 109: `raise ValueError("manifest head must be an object")` This rejects an invalid state by raising ‘ValueError("manifest head must be an object")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 110: `referenced_section_names.update(self._section_names(head, "head"))` This calls ‘referenced_section_names. with ‘self._section_names(head, "head")‘ inside the function ___init___, which handles init. The call advances the host-side job that validates and exposes typed read-only views of the aligned parameter archive.

Line 111: `if referenced_section_names != declared_section_names:` This executes the following indented host-code branch only when ‘referenced_section_names != declared_section_names‘ is true. The important operands are referenced_section_names: the value named ’referenced section names’ in this block; declared_section_names: the value named ’declared section names’ in this block. Context: the function ___init___, which handles init.

Line 112: `missing = sorted(declared_section_names - referenced_section_names)` This assigns ‘sorted(declared_sectior - referenced_section_names)‘ to ‘missing ‘ in host memory. Here missing means the value named ’missing’ in this block. The important operands are sorted: the value named ’sorted’ in this block; declared_section_names: the value named ’declared section names’ in this block; referenced_section_names: the value named ’referenced section names’ in this block. Context: the function ___init___, which handles init.

Line 113: `unknown = sorted(referenced_section_names - declared_section_names)` This assigns ‘sorted(referenced_secti - declared_section_names)‘ to ‘unknown ‘ in host memory. Here unknown means the value named ’unknown’ in this block. The important operands are sorted: the value named ’sorted’ in this block; referenced_section_names: the value named ’referenced section names’ in this block; declared_section_names: the value named ’declared section names’ in this block. Context: the function ___init___, which handles init.

Line 114: `raise ValueError(f"section reference mismatch: unreferenced={missing}, unknown={unknown}")` This rejects an invalid state by raising ‘ValueError(f"section reference mismatch: unreferenced={missing}, unknown={unknown}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 116: `@staticmethod` This decorator changes how the following class or function is constructed. ‘@staticmethod‘ is applied at import time on the host and is part of the class SpikingformerParameterArchive, which handles spikingformer parameter archive; it creates no FPGA circuit by itself.

Line 117: `def _integer_field(mapping: Mapping[str, Any], name: str) -> int:` This starts function _integer_field. Its indented body performs integer field as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 118: `value = mapping.get(name)` This assigns ‘mapping.get(name)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are mapping: the value named ’mapping’ in this block; get: the value named ’get’ in this block; name: the value named ’name’ in this block. Context: the function _integer_field, which handles integer field.

Line 119: `if isinstance(value, bool) or not isinstance(value, int):` This executes the following indented host-code branch only when ‘isinstance(value, bool) or not isinstance(value, int)‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; value: the value named ’value’ in this block; bool: the value named ’bool’ in this block. Context: the function _integer_field, which handles integer field.

Line 120: `raise ValueError(f"{name} must be an integer")` This rejects an invalid state by raising ‘ValueError(f"{name} must be an integer")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 121: `return value` This returns ‘value‘ to the caller of the function _integer_field, which handles integer field, ending this invocation. The important operands are value: the value named ’value’ in this block.

Line 123: `@staticmethod` This decorator changes how the following class or function is constructed. ‘@staticmethod‘ is applied at import time on the host and is part of the class SpikingformerParameterArchive, which handles spikingformer parameter archive; it creates no FPGA circuit by itself.

Line 124: `def _nonnegative_dimension(value: Any, section_name: str) -> int:` This starts function _nonnegative_dimension. Its indented body performs nonnegative dimension as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 125: `if isinstance(value, bool) or not isinstance(value, int) or value < 0:` This executes the following indented host-code branch only when ‘isinstance(value, bool) or not isinstance(value, int) or value < 0‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; value: the value named ’value’ in this block; bool: the value named ’bool’ in this block. Context: the function _nonnegative_dimension, which handles nonnegative dimension.

Line 126: `raise ValueError(f"section {section_name} has an invalid shape dimension")` This rejects an invalid state by raising 'ValueError(f"section {section_name} has an invalid shape dimension")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 127: `return value` This returns 'value' to the caller of the function _nonnegative_dimension, which handles nonnegative dimension, ending this invocation. The important operands are value: the value named 'value' in this block.

Line 129: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class SpikingformerParameterArchive, which handles spikingformer parameter archive; it creates no FPGA circuit by itself.

Line 130: `def _verify_hash(payload: bytes, expected: Any, description: str) -> None:` This starts function _verify_hash. Its indented body performs verify hash as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 131: `if not isinstance(expected, str) or len(expected) != 64:` This executes the following indented host-code branch only when 'not isinstance(expected, str) or len(expected) != 64' is true. The important operands are not: the value named 'not' in this block; isinstance: the value named 'isinstance' in this block; expected: the value named 'expected' in this block. Context: the function _verify_hash, which handles verify hash.

Line 132: `raise ValueError(f"{description} SHA-256 metadata is invalid")` This rejects an invalid state by raising 'ValueError(f"{description} SHA-256 metadata is invalid")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 133: `actual = hashlib.sha256(payload).hexdigest()` This assigns 'hashlib.sha256(payload).hexdigest()' to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are hashlib: the value named 'hashlib' in this block; sha256: the value named 'sha256' in this block; payload: stream data belonging to the current command. Context: the function _verify_hash, which handles verify hash.

Line 134: `if actual != expected.lower():` This executes the following indented host-code branch only when 'actual != expected.lower()' is true. The important operands are actual: the value named 'actual' in this block; expected: the value named 'expected' in this block; lower: the value named 'lower' in this block. Context: the function _verify_hash, which handles verify hash.

Line 135: `raise ValueError(f"{description} SHA-256 mismatch: {actual} != {expected}")` This rejects an invalid state by raising 'ValueError(f"{description} SHA-256 mismatch: {actual} != {expected}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 137: `def _projection_objects(self) -> list[Mapping[str, Any]]:` This starts function _projection_objects. Its indented body performs projection objects as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 138: `projections = self.manifest.get("projections")` This assigns 'self.manifest.get("projections")' to 'projections ' in host memory. Here projections means the value named 'projections' in this block. The important operands are manifest: the value named 'manifest' in this block; get: the value named 'get' in this block; projections: the value named 'projections' in this block. Context: the function _projection_objects, which handles projection objects.

Line 139: `if not isinstance(projections, list):` This executes the following indented host-code branch only when 'not isinstance(projections, list)' is true. The important operands are not: the value named 'not' in this block; isinstance: the value named 'isinstance' in this block; projections: the value named 'projections' in this block. Context: the function _projection_objects, which handles projection objects.

Line 140: `raise ValueError("manifest projections must be a list")` This rejects an invalid state by raising ‘ValueError("manifest projections must be a list")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 141: `if not all(isinstance(item, Mapping) for item in projections):` This executes the following indented host-code branch only when ‘not all(isinstance(item, Mapping) for item in projections)‘ is true. The important operands are not: the value named ’not’ in this block; all: the value named ’all’ in this block; isinstance: the value named ’isinstance’ in this block. Context: the function _projection_objects, which handles projection objects.

Line 142: `raise ValueError("each projection must be an object")` This rejects an invalid state by raising ‘ValueError("each projection must be an object")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 143: `return projections` This returns ‘projections‘ to the caller of the function _projection_objects, which handles projection objects, ending this invocation. The important operands are projections: the value named ’projections’ in this block.

Line 145: `@staticmethod` This decorator changes how the following class or function is constructed. ‘@staticmethod‘ is applied at import time on the host and is part of the class SpikingformerParameterArchive, which handles spikingformer parameter archive; it creates no FPGA circuit by itself.

Line 146: `def _section_names(owner: Mapping[str, Any], owner_kind: str) -> list[str]:` This starts function _section_names. Its indented body performs section names as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 147: `sections = owner.get("sections")` This assigns ‘owner.get("sections")‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are owner: the value named ’owner’ in this block; get: the value named ’get’ in this block; sections: the value named ’sections’ in this block. Context: the function _section_names, which handles section names.

Line 148: `if not isinstance(sections, list):` This executes the following indented host-code branch only when ‘not isinstance(sections, list)‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; sections: the value named ’sections’ in this block. Context: the function _section_names, which handles section names.

Line 149: `raise ValueError(f"{owner_kind} sections must be a list")` This rejects an invalid state by raising ‘ValueError(f"{owner_kind} sections must be a list")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 150: `names: list[str] = []` This assigns ‘[]‘ to ‘names‘ in host memory. Here names means the value named ’names’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _section_names, which handles section names.

Line 151: `for section in sections:` This iterates section over ‘sections‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _section_names, which handles section names.

Line 152: `if not isinstance(section, Mapping) or not isinstance(section.get("name"), str):` This executes the following indented host-code branch only when ‘not isinstance(section, Mapping) or not isinstance(section.get("name"), str)‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; section: the value named ’section’ in this block. Context: the function _section_names, which handles section names.

Line 153: `raise ValueError(f"{owner_kind} has an invalid section reference")` This rejects an invalid state by raising ‘ValueError(f"{owner_kind} has an invalid section reference")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 154: `names.append(section["name"])` This calls ‘names.append‘ with ‘section["name"]‘ inside the function _section_names, which handles section names. The call advances the host-side job that validates and exposes typed read-only views of the aligned parameter archive.

Line 155: `return names` This returns ‘names‘ to the caller of the function _section_names, which handles section names, ending this invocation. The important operands are names: the value named ’names’ in this block.

Line 157: `def section(self, name: str) -> np.ndarray:` This starts function section. Its indented body performs section as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 158: `try:` This is explicit host-side failure handling inside the function section, which handles section; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 159: `return self._sections[name]` This returns ‘self._sections[name]‘ to the caller of the function section, which handles section, ending this invocation. The important operands are _sections: the value named ’sections’ in this block; name: the value named ’name’ in this block.

Line 160: `except KeyError as error:` This is explicit host-side failure handling inside the function section, which handles section; it ensures malformed artifacts, DMA errors, and tool failures are surfaced or cleaned up deterministically.

Line 161: `raise KeyError(f"parameter section {name!r} does not exist") from error` This rejects an invalid state by raising ‘KeyError(f"parameter section {name!r} does not exist") from error‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 163: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class SpikingformerParameterArchive, which handles spikingformer parameter archive; it creates no FPGA circuit by itself.

Line 164: `def projection_names(self) -> tuple[str, ...]:` This starts function projection_names. Its indented body performs projection names as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 165: `return tuple(str(item.get("name")) for item in self._projection_objects())` This returns ‘tuple(str(item.get("name")) for item in self._projection_objects())‘ to the caller of the function projection_names, which handles projection names, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; str: the value named ’str’ in this block; item: the value named ’item’ in this block.

Line 167: `def projection(self, name: str) -> ProjectionParameters:` This starts function projection. Its indented body performs projection as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 168: `matches = [item for item in self._projection_objects() if item.get("name") == name]` Inside the function projection, which handles projection, this statement advances the host implementation that validates and exposes typed read-only views of the aligned parameter archive. The important operands are matches: the value named ’matches’ in this block; item: the value named ’item’ in this block; for: the value named ’for’ in this block.

Line 169: `if len(matches) != 1:` This executes the following indented host-code branch only when ‘len(matches) != 1‘ is true. The important operands are len: the value named ’len’ in this block; matches: the value named ’matches’ in this block. Context: the function projection, which handles projection.

Line 170: `raise KeyError(f"projection {name!r} does not exist uniquely")` This rejects an invalid state by raising ‘KeyError(f"projection {name!r} does not exist uniquely")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 171: `owner = matches[0]` This assigns ‘matches[0]‘ to ‘owner ‘ in host memory. Here owner means the value named ’owner’ in this block. The important operands are matches: the value named ’matches’ in this block. Context: the function projection, which handles projection.

Line 172: `section_names = self._section_names(owner, f"projection {name}")` This assigns ‘self._section_names(owner, f"projection {name}")‘ to ‘section_names ‘ in host memory. Here section_names means the value named ’section names’ in this block. The important operands are _section_names: the value named ’section names’ in this block; owner: the value named ’owner’ in this block; f: the value named ’f’ in this block. Context: the function projection, which handles projection.

Line 173: `expected_suffixes = (` This statement continues through source line 176; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(".weight_k_by_n_int8", ".bias_q10", ".multiplier_q15", ".binary_point_shift",)‘ to ‘expected_suffixes ‘ in host memory. Here expected_suffixes means the value named ’expected suffixes’ in this block. The important operands are weight_k_by_n_int8: a learned coefficient multiplied by an activation; bias_q10: an additive learned offset applied after accumulation; multiplier_q15: the value named ’multiplier q15’ in this block. Context: the function projection, which handles projection.

Line 174: `".weight_k_by_n_int8", ".bias_q10", ".multiplier_q15",` This is continuation line 2 of the 4-line logical statement that begins at source line 173. Its exact fragment is ‘".weight_k_by_n_int8", ".bias_q10", ".multiplier_q15",‘. The important operands are weight_k_by_n_int8: a learned coefficient multiplied by an activation; bias_q10: an additive learned offset applied after accumulation; multiplier_q15: the value named ’multiplier q15’ in this block. Read together, the complete operation is: This assigns ‘(".weight_k_by_n_int8", ".bias_q10", ".multiplier_q15", ".binary_point_shift",)‘ to ‘expected_suffixes ‘ in host memory. Here expected_suffixes means the value named ’expected suffixes’ in this block. The important operands are weight_k_by_n_int8: a learned coefficient multiplied by an activation; bias_q10: an additive learned offset applied after accumulation; multiplier_q15: the value named ’multiplier q15’ in this block. Context: the function projection, which handles projection.

Line 175: `".binary_point_shift",` This is continuation line 3 of the 4-line logical statement that begins at source line 173. Its exact fragment is ‘".binary_point_shift",‘. The important operands are binary_point_shift: the value named ’binary point shift’ in this block. Read together, the complete operation is: This assigns ‘(".weight_k_by_n_int8", ".bias_q10", ".multiplier_q15", ".binary_point_shift",)‘ to ‘expected_suffixes ‘ in host memory. Here expected_suffixes means the value named ’expected suffixes’ in this block. The important operands are weight_k_by_n_int8: a learned coefficient multiplied by an activation; bias_q10: an additive learned offset applied after accumulation; multiplier_q15: the value named ’multiplier q15’ in this block. Context: the function projection, which handles projection.

Line 176: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 173. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(".weight_k_by_n_int8", ".bias_q10", ".multiplier_q15", ".binary_point_shift",)‘ to ‘expected_suffixes ‘ in host memory. Here expected_suffixes means the value named ’expected suffixes’ in this block. The important operands are weight_k_by_n_int8: a learned coefficient multiplied by an activation; bias_q10: an additive learned offset applied after accumulation; multiplier_q15: the value named ’multiplier q15’ in this block. Context: the function projection, which handles projection.

Line 177: `selected: list[np.ndarray] = []` This assigns ‘[]‘ to ‘selected‘ in host memory. Here selected means the value named ’selected’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function projection, which handles projection.

Line 178: `for suffix in expected_suffixes:` This iterates suffix over ‘expected_suffixes‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function projection, which handles projection.

Line 179: `suffix_matches = [entry for entry in section_names if entry.endswith(suffix)]` This assigns '[entry for entry in section_names if entry.endswith(suffix)]' to 'suffix_matches ' in host memory. Here suffix_matches means the value named 'suffix matches' in this block. The important operands are entry: the value named 'entry' in this block; for: the value named 'for' in this block; in: the value named 'in' in this block. Context: the function projection, which handles projection.

Line 180: `if len(suffix_matches) != 1:` This executes the following indented host-code branch only when 'len(suffix_matches != 1' is true. The important operands are len: the value named 'len' in this block; suffix_matches: the value named 'suffix matches' in this block. Context: the function projection, which handles projection.

Line 181: `raise ValueError(f"projection {name} must reference one {suffix} section")` This rejects an invalid state by raising 'ValueError(f"projection {name} must reference one {suffix} section")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 182: `selected.append(self.section(suffix_matches[0]))` This calls 'selected.append' with 'self.section(suffix_matches inside the function projection, which handles projection. The call advances the host-side job that validates and exposes typed read-only views of the aligned parameter archive.

Line 183: `weight, bias, multiplier, shift = selected` This assigns 'selected' to 'weight, bias, multiplier, shift ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are selected: the value named 'selected' in this block. Context: the function projection, which handles projection.

Line 184: `if weight.ndim != 2 or any(value.ndim != 1 for value in selected[1:]):` This executes the following indented host-code branch only when 'weight.ndim != 2 or any(value.ndim != 1 for value in selected[1:])' is true. The important operands are weight: a learned coefficient multiplied by an activation; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function projection, which handles projection.

Line 185: `raise ValueError(f"projection {name} section ranks are invalid")` This rejects an invalid state by raising 'ValueError(f"projection {name} section ranks are invalid")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 186: `if not (weight.shape[1] == len(bias) == len(multiplier) == len(shift)):` This executes the following indented host-code branch only when 'not (weight.shape[1] == len(bias) == len(multiplier) == len(shift))' is true. The important operands are not: the value named 'not' in this block; weight: a learned coefficient multiplied by an activation; shape: the value named 'shape' in this block. Context: the function projection, which handles projection.

Line 187: `raise ValueError(f"projection {name} output-channel metadata is inconsistent")` This rejects an invalid state by raising 'ValueError(f"projection {name} output-channel metadata is inconsistent")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 188: `source_shape_object = owner.get("source_weight_shape")` This assigns 'owner.get("source_weight_shape")' to 'source_shape_object ' in host memory. Here source_shape_object means the value named 'source shape object' in this block. The important operands are owner: the value named 'owner' in this block; get: the value named 'get' in this block; source_weight_shape: a learned coefficient multiplied by an activation. Context: the function projection, which handles projection.

Line 189: `if not isinstance(source_shape_object, list):` This executes the following indented host-code branch only when 'not isinstance(source_shape_object, list)' is true. The important operands are not: the value named 'not' in this block; isinstance: the value named 'isinstance' in this block; source_shape_object: the value named 'source shape object' in this block. Context: the function projection, which handles projection.

Line 190: `raise ValueError(f"projection {name} has no source weight shape")` This rejects an invalid state by raising 'ValueError(f"projection {name} has no source weight shape")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 191: `source_shape = tuple(` This statement continues through source line 193; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘tuple(self._nonnegative_dimension(value, f"projection {name}") for value in source_shape_object)‘ to ‘source_shape ‘ in host memory. Here source_shape means the value named ’source shape’ in this block. The important operands are tuple: the value named ’tuple’ in this block; _nonnegative_dimension: the value named ’nonnegative dimension’ in this block; value: the value named ’value’ in this block. Context: the function projection, which handles projection.

Line 192: `self._nonnegative_dimension(value, f"projection {name}")` This is continuation line 2 of the 3-line logical statement that begins at source line 191. Its exact fragment is ‘self._nonnegative_dimension(value, f"projection {name}")‘. The important operands are _nonnegative_dimension: the value named ’nonnegative dimension’ in this block; value: the value named ’value’ in this block; f: the value named ’f’ in this block. Read together, the complete operation is: This assigns ‘tuple(self._nonnegative_dimension(value, f"projection {name}") for value in source_shape_object)‘ to ‘source_shape ‘ in host memory. Here source_shape means the value named ’source shape’ in this block. The important operands are tuple: the value named ’tuple’ in this block; _nonnegative_dimension: the value named ’nonnegative dimension’ in this block; value: the value named ’value’ in this block. Context: the function projection, which handles projection.

Line 193: `for value in source_shape_object)` This is continuation line 3 of the 3-line logical statement that begins at source line 191. Its exact fragment is ‘for value in source_shape_object)‘. The important operands are for: the value named ’for’ in this block; value: the value named ’value’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘tuple(self._nonnegative_dimension(value, f"projection {name}") for value in source_shape_object)‘ to ‘source_shape ‘ in host memory. Here source_shape means the value named ’source shape’ in this block. The important operands are tuple: the value named ’tuple’ in this block; _nonnegative_dimension: the value named ’nonnegative dimension’ in this block; value: the value named ’value’ in this block. Context: the function projection, which handles projection.

Line 194: `if len(source_shape) not in (3, 4) or source_shape[0] != weight.shape[1]:` This executes the following indented host-code branch only when ‘len(source_shape) not in (3, 4) or source_shape[0] != weight.shape[1]‘ is true. The important operands are len: the value named ’len’ in this block; source_shape: the value named ’source shape’ in this block; not: the value named ’not’ in this block. Context: the function projection, which handles projection.

Line 195: `raise ValueError(f"projection {name} source weight shape is inconsistent")` This rejects an invalid state by raising ‘ValueError(f"projection {name} source weight shape is inconsistent")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 196: `fractional_bits = self._integer_field(owner, "output_fractional_bits")` This assigns ‘self._integer_field(o "output_fractional_bits")‘ to ‘fractional_bits ‘ in host memory. Here fractional_bits means the value named ’fractional bits’ in this block. The important operands are _integer_field: the value named ’integer field’ in this block; owner: the value named ’owner’ in this block; output_fractional_bits: the value named ’output fractional bits’ in this block. Context: the function projection, which handles projection.

Line 197: `return ProjectionParameters(` This statement continues through source line 199; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘ProjectionParameters(name, weight, bias, multiplier, shift, str(owner.get("compute_opcode")), source_shape, fractional_bits)‘ to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; name: the value named ’name’ in this block; weight: a learned coefficient multiplied by an activation.

Line 198: `name, weight, bias, multiplier, shift, str(owner.get("compute_opcode")),` This is continuation line 2 of the 3-line logical statement that begins at source line 197. Its exact fragment is ‘name, weight, bias, multiplier, shift, str(owner.get("compute_opcode")),‘. The important operands are name: the value named ’name’ in this block; weight: a learned coefficient multiplied by an activation; bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This returns ‘ProjectionParameters(name, weight, bias, multiplier, shift, str(owner.get("compute_opcode")), source_shape, fractional_bits)‘ to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named ’projection

parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 199: `source_shape, fractional_bits)` This is continuation line 3 of the 3-line logical statement that begins at source line 197. Its exact fragment is 'source_shape, fractional_bits)'. The important operands are source_shape: the value named 'source shape' in this block; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This returns 'ProjectionParameters(name, weight, bias, multiplier, shift, str(owner.get("compute_opcode source_shape, fractional_bits)' to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 201: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class SpikingformerParameterArchive, which handles spikingformer parameter archive; it creates no FPGA circuit by itself.

Line 202: `def head(self) -> HeadParameters:` This starts function head. Its indented body performs head as part of the file that validates and exposes typed read-only views of the aligned parameter archive; it runs only when called.

Line 203: `owner = self.manifest["head"]` This assigns 'self.manifest["head"]' to 'owner ' in host memory. Here owner means the value named 'owner' in this block. The important operands are manifest: the value named 'manifest' in this block; head: the value named 'head' in this block. Context: the function head, which handles head.

Line 204: `names = self._section_names(owner, "head")` This assigns 'self._section_names(owner, "head")' to 'names ' in host memory. Here names means the value named 'names' in this block. The important operands are _section_names: the value named 'section names' in this block; owner: the value named 'owner' in this block; head: the value named 'head' in this block. Context: the function head, which handles head.

Line 205: `weight_names = [name for name in names if name.endswith(".weight_k_by_n_int8")]` This assigns '[name for name in names if name.endswith(".weight_k_by_n_int8")]' to 'weight_names ' in host memory. Here weight_names means a learned coefficient multiplied by an activation. The important operands are name: the value named 'name' in this block; for: the value named 'for' in this block; in: the value named 'in' in this block. Context: the function head, which handles head.

Line 206: `bias_names = [name for name in names if name.endswith(".bias_accumulator_units")]` This assigns '[name for name in names if name.endswith(".bias_accumulator_units")]' to 'bias_names ' in host memory. Here bias_names means an additive learned offset applied after accumulation. The important operands are name: the value named 'name' in this block; for: the value named 'for' in this block; in: the value named 'in' in this block. Context: the function head, which handles head.

Line 207: `if len(weight_names) != 1 or len(bias_names) != 1:` This executes the following indented host-code branch only when 'len(weight_names) != 1 or len(bias_names) != 1' is true. The important operands are len: the value named 'len' in this block; weight_names: a learned coefficient multiplied by an activation; or: the value named 'or' in this block. Context: the function head, which handles head.

Line 208: `raise ValueError("head must reference one weight and one bias section")` This rejects an invalid state by raising 'ValueError("head must reference one weight and one bias section")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 209: `weight = self.section(weight_names[0])` This assigns 'self.section(weight_names[0])' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are section: the value named 'section' in this block; weight_names: a learned coefficient multiplied by an activation. Context: the function head, which handles head.

Line 210: `bias = self.section(bias_names[0])` This assigns ‘self.section(bias_names[0])‘ to ‘bias ‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are section: the value named ’section’ in this block; bias_names: an additive learned offset applied after accumulation. Context: the function head, which handles head.

Line 211: `if weight.ndim != 2 or bias.ndim != 1 or weight.shape[1] != len(bias):` This executes the following indented host-code branch only when ‘weight.ndim != 2 or bias.ndim != 1 or weight.shape[1] != len(bias)‘ is true. The important operands are weight: a learned coefficient multiplied by an activation; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function head, which handles head.

Line 212: `raise ValueError("head section shapes are inconsistent")` This rejects an invalid state by raising ‘ValueError("head section shapes are inconsistent")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 213: `return HeadParameters(weight, bias)` This returns ‘HeadParameters(weight, bias)‘ to the caller of the function head, which handles head, ending this invocation. The important operands are HeadParameters: the value named ’head parameters’ in this block; weight: a learned coefficient multiplied by an activation; bias: an additive learned offset applied after accumulation.

42 software/spikingformer_quantization.py

Role in the complete system: defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

42.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: QuantizedWeight, HardwareEmulationReport, FixedPointHardResetLIF. Functions: quantize_cifar_input_u8, fake_fixed_point, prepare_hardware_emulation, _replace_lif_modules, input_scale_from_domain, _install_projection_requantizers, _projection_hook, as_dict, ___init___, forward, requantize. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

42.2 Complete source

```
"""Quantization functions that mirror the implemented NeuraEdge arithmetic."""

from __future__ import annotations

import copy
from dataclasses import asdict, dataclass
from typing import Literal

import torch
import torch.nn as nn

from neuraedge_golden import quantize_flexible_scale
from spikingformer_reference import MultiStepHardResetLIF, Spikingformer


WeightGranularity = Literal["tensor", "channel"]


@dataclass(frozen=True)
class QuantizedWeight:
    module: str
    shape: tuple[int, ...]
    granularity: WeightGranularity
    integer_minimum: int
    integer_maximum: int
    scale_minimum: float
    scale_maximum: float
    mean_absolute_error: float
    maximum_absolute_error: float


@dataclass(frozen=True)
class HardwareEmulationReport:
    fractional_bits: int
    fixed_point_scale: int
    input_scale: float
    input_zero_point: int
    head_activation_scale: float
    head_accumulator_scale: float
    weight_granularity: WeightGranularity
    quantized_weights: tuple[QuantizedWeight, ...]

    def as_dict(self) -> dict[str, object]:
        result = asdict(self)
        result["quantized_weights"] = [asdict(item) for item in self.quantized_weights]
        return result


class FixedPointHardResetLIF(MultiStepHardResetLIF):
```

```
    """Bit-exact model of ``neuraedge_lif_bank`` for one logical vector.

    The RTL first leaks ``v`` by ``v - (v >>> 1)`` and then adds a signed
    16-bit current.  Supplying ``x >>> 1`` as that current implements the
    checkpoint equation ``v/2 + x/2`` with deterministic integer rounding.
    Membranes saturate to signed 24-bit values before the threshold test.
    """

    def __init__(self, *, threshold: float, fractional_bits: int) -> None:
        super().__init__(tau=2.0, threshold=threshold)
        if not 0 <= fractional_bits <= 14:
            raise ValueError("fractional_bits must keep halved current in signed 16 bits")
        self.fractional_bits = fractional_bits
        self.fixed_scale = 1 << fractional_bits
        self.threshold_integer = int(round(threshold * self.fixed_scale))

    def forward(self, sequence: torch.Tensor) -> torch.Tensor:
        input_integer = torch.round(sequence * self.fixed_scale).to(torch.int64)
        voltage = torch.zeros_like(input_integer[0])
        spikes: list[torch.Tensor] = []
        minimum_membrane = -(1 << 23)
        maximum_membrane = (1 << 23) - 1
        for timestep in range(input_integer.shape[0]):
            current = torch.bitwise_right_shift(input_integer[timestep], 1)
            current = current.clamp(-(1 << 15), (1 << 15) - 1)
            leaked = voltage - torch.bitwise_right_shift(voltage, 1)
            voltage = (leaked + current).clamp(minimum_membrane, maximum_membrane)
            spike = voltage >= self.threshold_integer
            voltage = torch.where(spike, torch.zeros_like(voltage), voltage)
            spikes.append(spike.to(sequence.dtype))
        return torch.stack(spikes, dim=0)


def quantize_cifar_input_u8(image: torch.Tensor) -> tuple[torch.Tensor, float, int]:
    """Quantize the complete legal normalized CIFAR-10 input domain to uint8."""

    mean = torch.tensor((0.4914, 0.4822, 0.4465), device=image.device)
    standard_deviation = torch.tensor((0.2470, 0.2435, 0.2616), device=image.device)
    minimum = float(torch.min(-mean / standard_deviation).cpu())
    maximum = float(torch.max((1.0 - mean) / standard_deviation).cpu())
    scale = (maximum - minimum) / 255.0
    zero_point = int(round(-minimum / scale))
    integer = torch.round(image / scale + zero_point).clamp(0, 255)
    return (integer - zero_point) * scale, scale, zero_point


def fake_fixed_point(value: torch.Tensor, fractional_bits: int) -> torch.Tensor:
    """Round and saturate an analog activation to signed 24-bit fixed point."""

    scale = 1 << fractional_bits
    integer = torch.round(value * scale).clamp(-(1 << 23), (1 << 23) - 1)
    return integer / scale


def prepare_hardware_emulation(
    model: Spikingformer,
    *,
    fractional_bits: int = 10,
    weight_granularity: WeightGranularity = "tensor",
    head_activation_scale: float = 0.125,
) -> tuple[Spikingformer, HardwareEmulationReport]:
    """Clone a trained model and apply the arithmetic supported by the RTL.

    Conv/linear weights become signed int8 and are immediately dequantized for
    PyTorch execution.  This is algebraically equivalent to an int32 hardware
    accumulator followed by its recorded scale.  BatchNorm outputs are rounded
    to the common signed fixed-point format used by residuals and LIF neurons.
    """

    if weight_granularity not in {"tensor", "channel"}:
        raise ValueError("weight_granularity must be tensor or channel")
    if not 0 <= fractional_bits <= 14:
        raise ValueError("fractional_bits must be in [0, 14]")
    if head_activation_scale <= 0.0:
        raise ValueError("head_activation_scale must be positive")
    quantized_model = copy.deepcopy(model).eval()
    weight_reports: list[QuantizedWeight] = []
    weight_scales: dict[str, torch.Tensor] = {}

    for name, module in quantized_model.named_modules():
        if not isinstance(module, (nn.Conv1d, nn.Conv2d, nn.Linear)):
            continue
        weight = module.weight.detach()
        if weight_granularity == "channel":
            dimensions = tuple(range(1, weight.ndim))
            maximum = weight.abs().amax(dim=dimensions, keepdim=True)
        else:
```

```
            maximum = weight.abs().amax().reshape((1,) * weight.ndim)
        scale = torch.where(maximum > 0, maximum / 127.0, torch.ones_like(maximum))
        integer = torch.round(weight / scale).clamp(-127, 127)
        dequantized = integer * scale
        error = (dequantized - weight).abs()
        module.weight.data.copy_(dequantized)
        weight_scales[name] = scale.detach().reshape(-1)
        weight_reports.append(QuantizedWeight(
            module=name,
            shape=tuple(weight.shape),
            granularity=weight_granularity,
            integer_minimum=int(integer.amin().item()),
            integer_maximum=int(integer.amax().item()),
            scale_minimum=float(scale.amin().item()),
            scale_maximum=float(scale.amax().item()),
            mean_absolute_error=float(error.mean().item()),
            maximum_absolute_error=float(error.amax().item()),
        ))

    _replace_lif_modules(quantized_model, fractional_bits)
    _install_projection_requantizers(
        quantized_model,
        weight_scales,
        fractional_bits=fractional_bits,
        image_input_scale=input_scale_from_domain(),
    )
    quantized_model.head.register_forward_pre_hook(
        lambda _module, inputs, scale=head_activation_scale: (
            torch.round(inputs[0] / scale).clamp(-128, 127) * scale,
        ))
    head_weight_scale = float(quantized_model.head.weight.detach().abs().amax().item()) / 127.0
    head_accumulator_scale = head_activation_scale * head_weight_scale
    if quantized_model.head.bias is not None:
        quantized_model.head.bias.data.copy_(
            torch.round(quantized_model.head.bias.detach() / head_accumulator_scale)
            * head_accumulator_scale)

    input_probe = torch.zeros(1, 3, 1, 1)
    _, input_scale, input_zero_point = quantize_cifar_input_u8(input_probe)
    report = HardwareEmulationReport(
        fractional_bits=fractional_bits,
        fixed_point_scale=1 << fractional_bits,
        input_scale=input_scale,
        input_zero_point=input_zero_point,
        head_activation_scale=head_activation_scale,
        head_accumulator_scale=head_accumulator_scale,
        weight_granularity=weight_granularity,
        quantized_weights=tuple(weight_reports),
    )
    return quantized_model, report


def _replace_lif_modules(module: nn.Module, fractional_bits: int) -> None:
    for name, child in list(module.named_children()):
        if isinstance(child, MultiStepHardResetLIF):
            setattr(module, name, FixedPointHardResetLIF(
                threshold=child.threshold,
                fractional_bits=fractional_bits,
            ))
        else:
            _replace_lif_modules(child, fractional_bits)


def input_scale_from_domain() -> float:
    probe = torch.zeros(1, 3, 1, 1)
    _, scale, _ = quantize_cifar_input_u8(probe)
    return scale


def _install_projection_requantizers(
    model: Spikingformer,
    weight_scales: dict[str, torch.Tensor],
    *,
    fractional_bits: int,
    image_input_scale: float,
) -> None:
    modules = dict(model.named_modules())
    output_scale = 1 << fractional_bits
    for batch_norm_name, batch_norm in modules.items():
        if not isinstance(batch_norm, (nn.BatchNorm1d, nn.BatchNorm2d)):
            continue
        if not batch_norm_name.endswith("_bn"):
            raise ValueError(f"cannot locate projection for {batch_norm_name}")
        convolution_name = batch_norm_name[:-3] + "_conv"
        convolution = modules.get(convolution_name)
        if not isinstance(convolution, (nn.Conv1d, nn.Conv2d)):
            raise ValueError(f"cannot locate {convolution_name}")
```

```
        raw_weight_scale = weight_scales[convolution_name]
        if raw_weight_scale.numel() == 1:
            weight_scale = raw_weight_scale.repeat(convolution.out_channels)
        else:
            weight_scale = raw_weight_scale
        projection_input_scale = (
            image_input_scale if convolution_name == "patch_embed.block0_conv" else 1.0)
        bn_scale = batch_norm.weight.detach() / torch.sqrt(
            batch_norm.running_var.detach() + batch_norm.eps)
        factor = projection_input_scale * weight_scale * bn_scale * output_scale
        multiplier_and_shift = [quantize_flexible_scale(float(item)) for item in factor]
        multiplier = torch.tensor(
            [item[0] for item in multiplier_and_shift], dtype=torch.int64)
        effective_shift = torch.tensor(
            [15 + item[1] for item in multiplier_and_shift], dtype=torch.int64)
        if convolution.bias is None:
            convolution_bias = torch.zeros_like(batch_norm.running_mean)
        else:
            convolution_bias = convolution.bias.detach()
        folded_bias = (
            (convolution_bias - batch_norm.running_mean.detach()) * bn_scale
            + batch_norm.bias.detach())
        bias_integer = torch.round(folded_bias * output_scale).to(torch.int64)
        batch_norm.register_forward_hook(_projection_hook(
            convolution_bias=convolution_bias,
            projection_input_scale=projection_input_scale,
            weight_scale=weight_scale,
            multiplier=multiplier,
            effective_shift=effective_shift,
            bias_integer=bias_integer,
            output_scale=output_scale,
        ))


def _projection_hook(
    *,
    convolution_bias: torch.Tensor,
    projection_input_scale: float,
    weight_scale: torch.Tensor,
    multiplier: torch.Tensor,
    effective_shift: torch.Tensor,
    bias_integer: torch.Tensor,
    output_scale: int,
):
    def requantize(
        _module: nn.Module,
        inputs: tuple[torch.Tensor, ...],
        _output: torch.Tensor,
    ) -> torch.Tensor:
        convolution_output = inputs[0]
        channel_shape = (1, -1) + (1,) * (convolution_output.ndim - 2)
        device = convolution_output.device
        bias = convolution_bias.to(device).reshape(channel_shape)
        accumulator_scale = (
            weight_scale.to(device).reshape(channel_shape) * projection_input_scale)
        accumulator = torch.round(
            (convolution_output - bias) / accumulator_scale).to(torch.int64)
        q15 = multiplier.to(device).reshape(channel_shape)
        shifts = effective_shift.to(device).reshape(channel_shape)
        product = accumulator * q15
        magnitude = product.abs()
        rounding_bias = torch.where(
            shifts == 0,
            torch.zeros_like(shifts),
            torch.bitwise_left_shift(torch.ones_like(shifts), shifts - 1),
        )
        rounded_magnitude = torch.bitwise_right_shift(
            magnitude + rounding_bias, shifts)
        rounded = torch.where(product < 0, -rounded_magnitude, rounded_magnitude)
        quantized = rounded + bias_integer.to(device).reshape(channel_shape)
        quantized = quantized.clamp(-(1 << 23), (1 << 23) - 1)
        return quantized.to(convolution_output.dtype) / output_scale

    return requantize
```

42.3 Line annotations

Line 1: `"""Quantization functions that mirror the implemented NeuraEdge arithmetic."""` This is executable documentation attached to the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import copy` This imports the named Python API so the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `from dataclasses import asdict, dataclass` This imports the named Python API so the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from typing import Literal` This imports the named Python API so the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import torch` This imports the named Python API so the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `import torch.nn as nn` This imports the named Python API so the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `from neuraedge_golden import quantize_flexible_scale` This imports the named Python API so the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `from spikingformer_reference import MultiStepHardResetLIF, Spikingformer` This imports the named Python API so the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `WeightGranularity = Literal["tensor", "channel"]` This assigns ‘Literal["tensor", "channel"]‘ to ‘WeightGranularity ‘ in host memory. Here WeightGranularity means a learned coefficient multiplied by an activation. The important operands are Literal: the value named ’literal’ in this block; tensor: the value named ’tensor’ in this block; channel: the value named ’channel’ in this block. Context: the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 19: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it creates no FPGA circuit by itself.

Line 20: `class QuantizedWeight:` This defines class QuantizedWeight, which groups data and methods for quantized weight. Instances exist in host memory and organize the flow that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 21: `module: str` Inside the class QuantizedWeight, which handles quantized weight, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are module: the value named ’module’ in this block; str: the value named ’str’ in this block.

Line 22: `shape: tuple[int, ...]` Inside the class QuantizedWeight, which handles quantized weight, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are shape: the value named 'shape' in this block; tuple: the value named 'tuple' in this block.

Line 23: `granularity: WeightGranularity` Inside the class QuantizedWeight, which handles quantized weight, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are granularity: the value named 'granularity' in this block; WeightGranularity: a learned coefficient multiplied by an activation.

Line 24: `integer_minimum: int` Inside the class QuantizedWeight, which handles quantized weight, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are integer_minimum: the value named 'integer minimum' in this block.

Line 25: `integer_maximum: int` Inside the class QuantizedWeight, which handles quantized weight, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are integer_maximum: the value named 'integer maximum' in this block.

Line 26: `scale_minimum: float` Inside the class QuantizedWeight, which handles quantized weight, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are scale_minimum: the real-value spacing represented by one quantized integer step; float: the value named 'float' in this block.

Line 27: `scale_maximum: float` Inside the class QuantizedWeight, which handles quantized weight, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are scale_maximum: the real-value spacing represented by one quantized integer step; float: the value named 'float' in this block.

Line 28: `mean_absolute_error: float` Inside the class QuantizedWeight, which handles quantized weight, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are mean_absolute_error: the arithmetic average used by normalization; float: the value named 'float' in this block.

Line 29: `maximum_absolute_error: float` Inside the class QuantizedWeight, which handles quantized weight, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are maximum_absolute_error: the value named 'maximum absolute error' in this block; float: the value named 'float' in this block.

Line 32: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it creates no FPGA circuit by itself.

Line 33: `class HardwareEmulationReport:` This defines class HardwareEmulationReport, which groups data and methods for hardware emulation report. Instances exist in host memory and organize the flow that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 34: `fractional_bits: int` Inside the class HardwareEmulationReport, which handles hardware emulation report, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are fractional_bits: the value named 'fractional bits' in this block.

Line 35: `fixed_point_scale: int` Inside the class HardwareEmulationReport, which handles hardware emulation report, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are fixed_point_scale: the real-value spacing represented by one quantized integer step.

Line 36: `input_scale: float` Inside the class HardwareEmulationReport, which handles hardware emulation report, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are input_scale: the real-value spacing represented by one quantized integer step; float: the value named 'float' in this block.

Line 37: `input_zero_point: int` Inside the class HardwareEmulationReport, which handles hardware emulation report, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are input_zero_point: the integer code representing real numeric zero in affine quantization.

Line 38: `head_activation_scale: float` Inside the class HardwareEmulationReport, which handles hardware emulation report, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation; float: the value named 'float' in this block.

Line 39: `head_accumulator_scale: float` Inside the class HardwareEmulationReport, which handles hardware emulation report, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are head_accumulator_scale: a widened running sum that prevents intermediate overflow; float: the value named 'float' in this block.

Line 40: `weight_granularity: WeightGranularity` Inside the class HardwareEmulationReport, which handles hardware emulation report, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are weight_granularity: a learned coefficient multiplied by an activation; WeightGranularity: a learned coefficient multiplied by an activation.

Line 41: `quantized_weights: tuple[QuantizedWeight, ...]` Inside the class HardwareEmulationReport, which handles hardware emulation report, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are quantized_weights: a learned coefficient multiplied by an activation; tuple: the value named 'tuple' in this block; QuantizedWeight: a learned coefficient multiplied by an activation.

Line 43: `def as_dict(self) -> dict[str, object]:` This starts function as_dict. Its indented body performs as dict as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 44: `result = asdict(self)` This assigns ‘asdict(self)‘ to ‘result ‘ in host memory. Here result means the value named 'result' in this block. The important operands are asdict: the value named 'asdict' in this block. Context: the function as_dict, which handles as dict.

Line 45: `result["quantized_weights"] = [asdict(item) for item in self.quantized_weights]` Inside the function as_dict, which handles as dict, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are result: the value named 'result' in this block; quantized_weights: a learned coefficient multiplied by an activation; asdict: the value named 'asdict' in this block.

Line 46: `return result` This returns ‘result‘ to the caller of the function as_dict, which handles as dict, ending this invocation. The important operands are result: the value named 'result' in this block.

Line 49: `class FixedPointHardResetLIF(MultiStepHardResetLIF):` This defines class FixedPointHardResetLIF, which groups data and methods for fixed point hard reset lif. Instances exist in host memory and organize the flow that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 50: `"""Bit-exact model of "neuraedge_lif_bank" for one logical vector.` This is executable documentation attached to the class FixedPointHardResetLIF, which handles fixed point hard reset lif. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 52: `The RTL first leaks "v" by "v - (v >>> 1)" and then adds a signed` Inside the class FixedPointHardResetLIF which handles fixed point hard reset lif, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are The: the value named 'the' in this block; RTL: the value named 'rtl' in this block; first: the value named 'first' in this block.

Line 53: `16-bit current. Supplying "x >>> 1" as that current implements the` Inside the class FixedPointHardResetL which handles fixed point hard reset lif, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are bit: the value named 'bit' in this block; current: the value named 'current' in this block; Supplying: the value named 'supplying' in this block.

Line 54: `checkpoint equation "v/2 + x/2" with deterministic integer rounding.` Inside the class FixedPointHardReset which handles fixed point hard reset lif, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are checkpoint: the value named 'checkpoint' in this block; equation: the value named 'equation' in this block; v: the value named 'v' in this block.

Line 55: `Membranes saturate to signed 24-bit values before the threshold test.` Inside the class FixedPointHardResetLIF, which handles fixed point hard reset lif, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are Membranes: the persistent LIF neuron state carrying charge between timesteps; saturate: the value named 'saturate' in this block; to: the value named 'to' in this block.

Line 56: `"""` This is executable documentation attached to the class FixedPointHardResetLIF, which handles fixed point hard reset lif. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 58: `def __init__(self, *, threshold: float, fractional_bits: int) -> None:` This starts function ___init___. Its indented body performs init as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 59: `super().__init__(tau=2.0, threshold=threshold)` This calls 'super' with ').___init___(tau=2.0, threshold=threshold' inside the function ___init___, which handles init. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 60: `if not 0 <= fractional_bits <= 14:` This executes the following indented host-code branch only when 'not 0 <= fractional_bits <= 14' is true. The important operands are not: the value named 'not' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function ___init___, which handles init.

Line 61: `raise ValueError("fractional_bits must keep halved current in signed 16 bits")` This rejects an invalid state by raising 'ValueError("fractional_bits must keep halved current in signed 16 bits")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 62: `self.fractional_bits = fractional_bits` This assigns 'fractional_bits' to 'self.fractional_bits ' in host memory. Here fractional_bits means the value named 'fractional bits' in this block. The important operands are fractional_bits: the value named 'fractional bits' in this block. Context: the function ___init___, which handles init.

Line 63: `self.fixed_scale = 1 << fractional_bits` This assigns '1 << fractional_bits' to 'self.fixed_scale ' in host memory. Here fixed_scale means the real-value spacing represented by one quantized integer step. The important operands are fractional_bits: the value named 'fractional bits' in this block. Context: the function ___init___, which handles init.

Line 64: `self.threshold_integer = int(round(threshold * self.fixed_scale))` This assigns 'int(round(threshold * self.fixed_scale))' to 'self.threshold_integer ' in host memory. Here threshold_integer means the membrane level at which a neuron emits a spike. The important operands are round: the value named 'round' in this block; threshold: the membrane level at which a neuron emits a spike; fixed_scale: the real-value spacing represented by one quantized integer step. Context: the function ___init___, which handles init.

Line 66: `def forward(self, sequence: torch.Tensor) -> torch.Tensor:` This starts function forward. Its indented body performs forward as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 67: `input_integer = torch.round(sequence * self.fixed_scale).to(torch.int64)` This assigns 'torch.round(sequer * self.fixed_scale).to(torch.int64)' to 'input_integer ' in host memory. Here input_integer means the value named 'input integer' in this block. The important operands are torch: the value named 'torch' in this block; round: the value named 'round' in this block; sequence: the value named 'sequence' in this block. Context: the function forward, which handles forward.

Line 68: `voltage = torch.zeros_like(input_integer[0])` This assigns 'torch.zeros_like(input_integer[0])' to 'voltage ' in host memory. Here voltage means the value named 'voltage' in this block. The important operands are torch: the value named 'torch' in this block; zeros_like: the value named 'zeros like' in this block; input_integer: the value named 'input integer' in this block. Context: the function forward, which handles forward.

Line 69: `spikes: list[torch.Tensor] = []` This assigns '[]' to 'spikes' in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the function forward, which handles forward.

Line 70: `minimum_membrane = -(1 << 23)` This assigns '-(1 << 23)' to 'minimum_membrane ' in host memory. Here minimum_membrane means the persistent LIF neuron state carrying charge between timesteps. The expression is a literal or structural operator and has no hidden software state. Context: the function forward, which handles forward.

Line 71: `maximum_membrane = (1 << 23) - 1` This assigns '(1 << 23) - 1' to 'maximum_membrane ' in host memory. Here maximum_membrane means the persistent LIF neuron state carrying charge between timesteps. The expression is a literal or structural operator and has no hidden software state. Context: the function forward, which handles forward.

Line 72: `for timestep in range(input_integer.shape[0]):` This iterates timestep over 'range(input_integer.shape[0])'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function forward, which handles forward.

Line 73: `current = torch.bitwise_right_shift(input_integer[timestep], 1)` This assigns 'torch.bitwise_right_shift(inp 1)' to 'current ' in host memory. Here current means the value named 'current' in this block. The important operands are torch: the value named 'torch' in this block; bitwise_right_shift: the value named 'bitwise right shift' in this block; input_integer: the value named 'input integer' in this block. Context: the function forward, which handles forward.

Line 74: `current = current.clamp(-(1 << 15), (1 << 15) - 1)` This assigns 'current.clamp(-(1 << 15), (1 << 15) - 1)' to 'current ' in host memory. Here current means the value named 'current' in this block. The important operands are current: the value named 'current' in this block; clamp: the value named 'clamp' in this block. Context: the function forward, which handles forward.

Line 75: `leaked = voltage - torch.bitwise_right_shift(voltage, 1)` This assigns ‘voltage - torch.bitwise_right_shift(vol 1)‘ to ‘leaked ‘ in host memory. Here leaked means the retained fraction of prior membrane state. The important operands are voltage: the value named ’voltage’ in this block; torch: the value named ’torch’ in this block; bitwise_right_shift: the value named ’bitwise right shift’ in this block. Context: the function forward, which handles forward.

Line 76: `voltage = (leaked + current).clamp(minimum_membrane, maximum_membrane)` This assigns ‘(leaked + current).clamp(minimum_membrane, maximum_membrane)‘ to ‘voltage ‘ in host memory. Here voltage means the value named ’voltage’ in this block. The important operands are leaked: the retained fraction of prior membrane state; current: the value named ’current’ in this block; clamp: the value named ’clamp’ in this block. Context: the function forward, which handles forward.

Line 77: `spike = voltage >= self.threshold_integer` Inside the function forward, which handles forward, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are spike: a binary event produced when a neuron crosses its threshold; voltage: the value named ’voltage’ in this block; threshold_integer: the membrane level at which a neuron emits a spike.

Line 78: `voltage = torch.where(spike, torch.zeros_like(voltage), voltage)` This assigns ‘torch.where(spike, torch.zeros_like(voltage), voltage)‘ to ‘voltage ‘ in host memory. Here voltage means the value named ’voltage’ in this block. The important operands are torch: the value named ’torch’ in this block; where: the value named ’where’ in this block; spike: a binary event produced when a neuron crosses its threshold. Context: the function forward, which handles forward.

Line 79: `spikes.append(spike.to(sequence.dtype))` This calls ‘spikes.append‘ with ‘spike.to(sequence.dtype)‘ inside the function forward, which handles forward. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 80: `return torch.stack(spikes, dim=0)` This returns ‘torch.stack(spikes, dim=0)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are torch: the value named ’torch’ in this block; stack: the value named ’stack’ in this block; spikes: a binary event produced when a neuron crosses its threshold.

Line 83: `def quantize_cifar_input_u8(image: torch.Tensor) -> tuple[torch.Tensor, float, int]:` This starts function quantize_cifar_input_u8. Its indented body performs quantize cifar input u8 as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 84: `"""Quantize the complete legal normalized CIFAR-10 input domain to uint8."""` This is executable documentation attached to the function quantize_cifar_input_u8, which handles quantize cifar input u8. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 86: `mean = torch.tensor((0.4914, 0.4822, 0.4465), device=image.device)` This assigns ‘torch.tensor((0.4914, 0.4822, 0.4465), device=image.device)‘ to ‘mean ‘ in host memory. Here mean means the arithmetic average used by normalization. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block; device: the value named ’device’ in this block. Context: the function quantize_cifar_input_u8, which handles quantize cifar input u8.

Line 87: `standard_deviation = torch.tensor((0.2470, 0.2435, 0.2616), device=image.device)` This assigns ‘torch.tensor((0.2470, 0.2435, 0.2616), device=image.device)‘ to ‘standard_deviation ‘ in host memory. Here standard_deviation means the value named ’standard deviation’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block; device: the value named ’device’ in this block. Context: the function quantize_cifar_input_u8, which handles quantize cifar input u8.

Line 88: `minimum = float(torch.min(-mean / standard_deviation).cpu())` This assigns ‘float(torch.min(-mean / standard_deviation).cpu())‘ to ‘minimum ‘ in host memory. Here minimum means the value named ’minimum’ in this block. The important operands are float: the value named ’float’ in this block; torch: the value named ’torch’ in this block; min: the value named ’min’ in this block. Context: the function quantize_cifar_input_u8, which handles quantize cifar input u8.

Line 89: `maximum = float(torch.max((1.0 - mean) / standard_deviation).cpu())` This assigns ‘float(torch.max((1.0 - mean) / standard_deviation).cpu())‘ to ‘maximum ‘ in host memory. Here maximum means the value named ’maximum’ in this block. The important operands are float: the value named ’float’ in this block; torch: the value named ’torch’ in this block; max: the value named ’max’ in this block. Context: the function quantize_cifar_input_u8, which handles quantize cifar input u8.

Line 90: `scale = (maximum - minimum) / 255.0` This assigns ‘(maximum - minimum) / 255.0‘ to ‘scale ‘ in host memory. Here scale means the real-value spacing represented by one quantized integer step. The important operands are maximum: the value named ’maximum’ in this block; minimum: the value named ’minimum’ in this block. Context: the function quantize_cifar_input_u8, which handles quantize cifar input u8.

Line 91: `zero_point = int(round(-minimum / scale))` This assigns ‘int(round(-minimum / scale))‘ to ‘zero_point ‘ in host memory. Here zero_point means the integer code representing real numeric zero in affine quantization. The important operands are round: the value named ’round’ in this block; minimum: the value named ’minimum’ in this block; scale: the real-value spacing represented by one quantized integer step. Context: the function quantize_cifar_input_u8, which handles quantize cifar input u8.

Line 92: `integer = torch.round(image / scale + zero_point).clamp(0, 255)` This assigns ‘torch.round(image / scale + zero_point).clamp(0, 255)‘ to ‘integer ‘ in host memory. Here integer means the value named ’integer’ in this block. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; image: the value named ’image’ in this block. Context: the function quantize_cifar_input_u8, which handles quantize cifar input u8.

Line 93: `return (integer - zero_point) * scale, scale, zero_point` This returns ‘(integer - zero_point) * scale, scale, zero_point‘ to the caller of the function quantize_cifar_input_u8, which handles quantize cifar input u8, ending this invocation. The important operands are integer: the value named ’integer’ in this block; zero_point: the integer code representing real numeric zero in affine quantization; scale: the real-value spacing represented by one quantized integer step.

Line 96: `def fake_fixed_point(value: torch.Tensor, fractional_bits: int) -> torch.Tensor:` This starts function fake_fixed_point. Its indented body performs fake fixed point as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 97: `"""Round and saturate an analog activation to signed 24-bit fixed point."""` This is executable documentation attached to the function fake_fixed_point, which handles fake fixed point. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 99: `scale = 1 << fractional_bits` This assigns ‘1 << fractional_bits‘ to ‘scale ‘ in host memory. Here scale means the real-value spacing represented by one quantized integer step. The important operands are fractional_bits: the value named ’fractional bits’ in this block. Context: the function fake_fixed_point, which handles fake fixed point.

Line 100: `integer = torch.round(value * scale).clamp(-(1 << 23), (1 << 23) - 1)` This assigns ‘torch.round(value * scale).clamp(-(1 << 23), (1 << 23) - 1)‘ to ‘integer ‘ in host memory. Here integer means the value named ’integer’ in this block. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; value: the value named ’value’ in this block. Context: the function fake_fixed_point, which handles fake fixed point.

Line 101: `return integer / scale` This returns ‘integer / scale‘ to the caller of the function fake_fixed_point, which handles fake fixed point, ending this invocation. The important operands are integer: the value named ’integer’ in this block; scale: the real-value spacing represented by one quantized integer step.

Line 104: `def prepare_hardware_emulation(` This starts function prepare_hardware_emulation. Its indented body performs prepare hardware emulation as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 105: `model: Spikingformer,` Inside the function prepare_hardware_emulation, which handles prepare hardware emulation, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are model: the value named ’model’ in this block; Spikingformer: the value named ’spikingformer’ in this block.

Line 106: `*,` Inside the function prepare_hardware_emulation, which handles prepare hardware emulation, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The expression is a literal or structural operator and has no hidden software state.

Line 107: `fractional_bits: int = 10,` This assigns ‘10,‘ to ‘fractional_bits‘ in host memory. Here fractional_bits means the value named ’fractional bits’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 108: `weight_granularity: WeightGranularity = "tensor",` This assigns ‘"tensor",‘ to ‘weight_granularity‘ in host memory. Here weight_granularity means a learned coefficient multiplied by an activation. The important operands are tensor: the value named ’tensor’ in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 109: `head_activation_scale: float = 0.125,` This assigns ‘0.125,‘ to ‘head_activation_scale‘ in host memory. Here head_activation_scale means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 110: `) -> tuple[Spikingformer, HardwareEmulationReport]:` Inside the function prepare_hardware_emulation, which handles prepare hardware emulation, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are tuple: the value named ’tuple’ in this block; Spikingformer: the value named ’spikingformer’ in this block; HardwareEmulationReport: the value named ’hardware emulation report’ in this block.

Line 111: `"""Clone a trained model and apply the arithmetic supported by the RTL.` This is executable documentation attached to the function prepare_hardware_emulation, which handles prepare hardware emulation. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 113: `Conv/linear weights become signed int8 and are immediately dequantized for` Inside the function prepare_hardware_emulation, which handles prepare hardware emulation, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are Conv: the value named ’conv’ in this block; linear: the value named ’linear’ in this block; weights: a learned coefficient multiplied by an activation.

Line 114: `PyTorch execution. This is algebraically equivalent to an int32 hardware` Inside the function prepare_hardware_emulation, which handles prepare hardware emulation, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are PyTorch: the value named ’py torch’ in this block; execution: the value named ’execution’ in this block; This: the value named ’this’ in this block.

Line 115: `accumulator followed by its recorded scale. BatchNorm outputs are rounded` Inside the function prepare_hardware_emulation, which handles prepare hardware emulation, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are accumulator: a widened running sum that prevents intermediate overflow; followed: the value named 'followed' in this block; by: the value named 'by' in this block.

Line 116: `to the common signed fixed-point format used by residuals and LIF neurons.` Inside the function prepare_hardware_emulation, which handles prepare hardware emulation, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are to: the value named 'to' in this block; the: the value named 'the' in this block; common: the value named 'common' in this block.

Line 117: `"""` This is executable documentation attached to the function prepare_hardware_emulation, which handles prepare hardware emulation. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 119: `if weight_granularity not in {"tensor", "channel"}:` This executes the following indented host-code branch only when 'weight_granularity not in {"tensor", "channel"}' is true. The important operands are weight_granularity: a learned coefficient multiplied by an activation; not: the value named 'not' in this block; in: the value named 'in' in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 120: `raise ValueError("weight_granularity must be tensor or channel")` This rejects an invalid state by raising 'ValueError("weight_granularity must be tensor or channel")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 121: `if not 0 <= fractional_bits <= 14:` This executes the following indented host-code branch only when 'not 0 <= fractional_bits <= 14' is true. The important operands are not: the value named 'not' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 122: `raise ValueError("fractional_bits must be in [0, 14]")` This rejects an invalid state by raising 'ValueError("fractional_bits must be in [0, 14]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 123: `if head_activation_scale <= 0.0:` This executes the following indented host-code branch only when 'head_activation_scale <= 0.0' is true. The important operands are head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 124: `raise ValueError("head_activation_scale must be positive")` This rejects an invalid state by raising 'ValueError("head_activation_scale must be positive")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 125: `quantized_model = copy.deepcopy(model).eval()` This assigns 'copy.deepcopy(model).eval()' to 'quantized_model ' in host memory. Here quantized_model means the value named 'quantized model' in this block. The important operands are copy: the value named 'copy' in this block; deepcopy: the value named 'deepcopy' in this block; model: the value named 'model' in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 126: `weight_reports: list[QuantizedWeight] = []` This assigns '[]' to 'weight_reports' in host memory. Here weight_reports means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 127: `weight_scales: dict[str, torch.Tensor] = {}` This assigns '{}' to 'weight_scales' in host memory. Here weight_scales means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 129: `for name, module in quantized_model.named_modules():` This iterates name, module over 'quantized_model.nam Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 130: `if not isinstance(module, (nn.Conv1d, nn.Conv2d, nn.Linear)):` This executes the following indented host-code branch only when 'not isinstance(module, (nn.Conv1d, nn.Conv2d, nn.Linear))' is true. The important operands are not: the value named 'not' in this block; isinstance: the value named 'isinstance' in this block; module: the value named 'module' in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 131: `continue` Inside the function prepare_hardware_emulation, which handles prepare hardware emulation, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are continue: the value named 'continue' in this block.

Line 132: `weight = module.weight.detach()` This assigns 'module.weight.detach()' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are module: the value named 'module' in this block; weight: a learned coefficient multiplied by an activation; detach: the value named 'detach' in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 133: `if weight_granularity == "channel":` This executes the following indented host-code branch only when 'weight_granularity == "channel"' is true. The important operands are weight_granularity: a learned coefficient multiplied by an activation; channel: the value named 'channel' in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 134: `dimensions = tuple(range(1, weight.ndim))` This assigns 'tuple(range(1, weight.ndim))' to 'dimensions ' in host memory. Here dimensions means the value named 'dimensions' in this block. The important operands are tuple: the value named 'tuple' in this block; range: the value named 'range' in this block; weight: a learned coefficient multiplied by an activation. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 135: `maximum = weight.abs().amax(dim=dimensions, keepdim=True)` This assigns 'weight.abs().amax(dim=dimensions keepdim=True)' to 'maximum ' in host memory. Here maximum means the value named 'maximum' in this block. The important operands are weight: a learned coefficient multiplied by an activation; abs: the value named 'abs' in this block; amax: the value named 'amax' in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 136: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 137: `maximum = weight.abs().amax().reshape((1,) * weight.ndim)` This assigns 'weight.abs().amax().reshape((1,) * weight.ndim)' to 'maximum ' in host memory. Here maximum means the value named 'maximum' in this block. The important operands are weight: a learned coefficient multiplied by an activation; abs: the value named 'abs' in this block; amax: the value named 'amax' in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 138: `scale = torch.where(maximum > 0, maximum / 127.0, torch.ones_like(maximum))` This assigns 'torch.where(m > 0, maximum / 127.0, torch.ones_like(maximum))' to 'scale ' in host memory. Here scale means the real-value spacing represented by one quantized integer step. The important operands are torch: the value named 'torch' in this block; where: the value named 'where' in this block; maximum: the value named 'maximum' in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 139: `integer = torch.round(weight / scale).clamp(-127, 127)` This assigns ‘torch.round(weight / scale).clamp(-127, 127)‘ to ‘integer ‘ in host memory. Here integer means the value named ’integer’ in this block. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 140: `dequantized = integer * scale` This assigns ‘integer * scale‘ to ‘dequantized ‘ in host memory. Here dequantized means the value named ’dequantized’ in this block. The important operands are integer: the value named ’integer’ in this block; scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 141: `error = (dequantized - weight).abs()` This assigns ‘(dequantized - weight).abs()‘ to ‘error ‘ in host memory. Here error means the value named ’error’ in this block. The important operands are dequantized: the value named ’dequantized’ in this block; weight: a learned coefficient multiplied by an activation; abs: the value named ’abs’ in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 142: `module.weight.data.copy_(dequantized)` This calls ‘module.weight.data.copy_‘ with ‘dequantized‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 143: `weight_scales[name] = scale.detach().reshape(-1)` This assigns ‘scale.detach().reshape(-1)‘ to ‘weight_scales[na‘ in host memory. Here weight_scales means a learned coefficient multiplied by an activation. The important operands are scale: the real-value spacing represented by one quantized integer step; detach: the value named ’detach’ in this block; reshape: the value named ’reshape’ in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 144: `weight_reports.append(QuantizedWeight(` This statement continues through source line 154; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘weight_reports.append‘ with ‘QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.ami integer_maximum=int(integer.amax().item()), scale_minimum=float(scale.amin().item()), scale_maximum=float(scale.amax().ite mean_absolute_error=float(error.mean().item()), maximum_absolute_error=float(error.amax().item()),)‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 145: `module=name,` This is continuation line 2 of the 11-line logical statement that begins at source line 144. Its exact fragment is ‘module=name,‘. The important operands are module: the value named ’module’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: This calls ‘weight_reports.append‘ with ‘QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.ami integer_maximum=int(integer.amax().item()), scale_minimum=float(scale.amin().item()), scale_maximum=float(scale.amax().ite mean_absolute_error=float(error.mean().item()), maximum_absolute_error=float(error.amax().item()),)‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 146: `shape=tuple(weight.shape),` This is continuation line 3 of the 11-line logical statement that begins at source line 144. Its exact fragment is ‘shape=tuple(weight.shape),‘. The important operands are shape: the value named ’shape’ in this block; tuple: the value named ’tuple’ in this block; weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls ‘weight_reports.append‘ with ‘QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.amin().item()), integer_maximum=int(integer.amax().item()), scale_minimum=float(scale.amin().item()), scale_maximum=float(scale.amax().ite mean_absolute_error=float(error.mean().item()), maximum_absolute_error=float(error.amax().item()),)‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 147: `granularity=weight_granularity,` This is continuation line 4 of the 11-line logical statement that begins at source line 144. Its exact fragment is 'granularity=weight_granularity,'. The important operands are granularity: the value named 'granularity' in this block; weight_granularity: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls 'weight_reports.append' with 'QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.amin().item()), integer_maximum=int(integer.amax().item()), scale_minimum=float(scale.amin().item()), scale_maximum=float(scale.amax().ite mean_absolute_error=float(error.mean().item()), maximum_absolute_error=float(error.amax().item()),)' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 148: `integer_minimum=int(integer.amin().item()),` This is continuation line 5 of the 11-line logical statement that begins at source line 144. Its exact fragment is 'integer_minimum=int(integer.amin().item()),'. The important operands are integer_minimum: the value named 'integer minimum' in this block; integer: the value named 'integer' in this block; amin: the value named 'amin' in this block. Read together, the complete operation is: This calls 'weight_reports.append' with 'QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.amin().item()), integer_maximum=int(integer.amax().item()), scale_minimum=float(scale.amin().i scale_maximum=float(scale.amax().item()), mean_absolute_error=float(error.mean().item()), maximum_absolute_error=float(er)' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 149: `integer_maximum=int(integer.amax().item()),` This is continuation line 6 of the 11-line logical statement that begins at source line 144. Its exact fragment is 'integer_maximum=int(integer.amax().item()),'. The important operands are integer_maximum: the value named 'integer maximum' in this block; integer: the value named 'integer' in this block; amax: the value named 'amax' in this block. Read together, the complete operation is: This calls 'weight_reports.append' with 'QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.amin().item()), integer_maximum=int(integer.amax().item()), scale_minimum=float(scale.amin().i scale_maximum=float(scale.amax().item()), mean_absolute_error=float(error.mean().item()), maximum_absolute_error=float(er)' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 150: `scale_minimum=float(scale.amin().item()),` This is continuation line 7 of the 11-line logical statement that begins at source line 144. Its exact fragment is 'scale_minimum=float(scale.amin().item()),'. The important operands are scale_minimum: the real-value spacing represented by one quantized integer step; float: the value named 'float' in this block; scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This calls 'weight_reports.append' with 'QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.amin().item()), integer_maximum=int(integer.amax().item()), scale_minimum=float(scale.amin().item()), scale_maximum=float(scale.amax().item()), mean_absolute_error=float(error.mean() maximum_absolute_error=float(error.amax().item()),)' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 151: `scale_maximum=float(scale.amax().item()),` This is continuation line 8 of the 11-line logical statement that begins at source line 144. Its exact fragment is 'scale_maximum=float(scale.amax().item()),'. The important operands are scale_maximum: the real-value spacing represented by one quantized integer step; float: the value named 'float' in this block; scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This calls 'weight_reports.append' with 'QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.amin().item()), integer_maximum=int(integer.amax().item()), scale_minimum=float(scale.amin().item()), scale_maximum=float(scale.amax().item()), mean_absolute_error=float(error.mean() maximum_absolute_error=float(error.amax().item()),)' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 152: `mean_absolute_error=float(error.mean().item()),` This is continuation line 9 of the 11-line logical statement that begins at source line 144. Its exact fragment is 'mean_absolute_error=float(error.mean().item()),'. The important operands are mean_absolute_error: the arithmetic average used by normalization; float: the value

named 'float' in this block; error: the value named 'error' in this block. Read together, the complete operation is: This calls ‘weight_reports.append‘ with ‘QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.amin().item()), integer_maximum=int(integer.amax().item()), scale_minim scale_maximum=float(scale.amax().item()), mean_absolute_error=float(error.mean().item()), maximum_absolute_error=float(e)‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 153: `maximum_absolute_error=float(error.amax().item()),` This is continuation line 10 of the 11-line logical statement that begins at source line 144. Its exact fragment is ‘maximum_absolute_error=float(error.amax().item()),‘. The important operands are maximum_absolute_error: the value named 'maximum absolute error' in this block; float: the value named 'float' in this block; error: the value named 'error' in this block. Read together, the complete operation is: This calls ‘weight_reports.append‘ with ‘QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.amin().item()), integer_maximum=int(integer.amax().item()), scale_minim scale_maximum=float(scale.amax().item()), mean_absolute_error=float(error.mean().item()), maximum_absolute_error=float(e)‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 154: `))` This is continuation line 11 of the 11-line logical statement that begins at source line 144. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘weight_reports.append‘ with ‘QuantizedWeight(module=name, shape=tuple(weight.shape), granularity=weight_granularity, integer_minimum=int(integer.amin().item()), integer_maximum=int(integer.amax().item()), scale_minimum=float(scale.amin().item()), scale_maximum=float(scale.amax().item()), mean_absolute_error=float(error.mean() maximum_absolute_error=float(error.amax().item()),)‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 156: `_replace_lif_modules(quantized_model, fractional_bits)` This calls ‘_replace_lif_modules‘ with ‘quantized_model, fractional_bits‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 157: `_install_projection_requantizers(` This statement continues through source line 162; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘_install_projection_requantizers‘ with ‘ quantized_model, weight_scales, fractional_bits=fractional_bits, image_input_scale=input_scale_from_domain(), ‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 158: `quantized_model,` This is continuation line 2 of the 6-line logical statement that begins at source line 157. Its exact fragment is ‘quantized_model,‘. The important operands are quantized_model: the value named 'quantized model' in this block. Read together, the complete operation is: This calls ‘_install_projection_requantizers‘ with ‘ quantized_model, weight_scales, fractional_bits=fractional_bits, image_input_scale=input_scale_from_domain(), ‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 159: `weight_scales,` This is continuation line 3 of the 6-line logical statement that begins at source line 157. Its exact fragment is ‘weight_scales,‘. The important operands are weight_scales: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls ‘_install_projection_requantizers‘ with ‘ quantized_model, weight_scales, fractional_bits=fractional_bits, image_input_scale=input_scale_from_domain(), ‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 160: `fractional_bits=fractional_bits,` This is continuation line 4 of the 6-line logical statement that begins at source line 157. Its exact fragment is 'fractional_bits=fractional_bits,'. The important operands are fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This calls '_install_projection_requantizers' with ' quantized_model, weight_scales, fractional_bits=fractional_bits, image_input_scale=input_ ' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 161: `image_input_scale=input_scale_from_domain(),` This is continuation line 5 of the 6-line logical statement that begins at source line 157. Its exact fragment is 'image_input_scale=input_scale_from_domain(),'. The important operands are image_input_scale: the real-value spacing represented by one quantized integer step; input_scale_from_domain: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This calls '_install_projection_requantizers' with ' quantized_model, weight_scales, fractional_bits=fractional_bits, image_input_scale=input_scale_from_domain(), ' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 162: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 157. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls '_install_projection_requantizers' with ' quantized_model, weight_scales, fractional_bits=fractional_bits, image_input_scale=input_scale_from_domain(), ' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 163: `quantized_model.head.register_forward_pre_hook(` This statement continues through source line 166; the explanation uses the complete expression rather than only the first visual fragment. This calls 'quantized_model.head.register_fo with ' lambda _module, inputs, scale=head_activation_scale: (torch.round(inputs[0] / scale).clamp(-128, 127) * scale,)' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 164: `lambda _module, inputs, scale=head_activation_scale: (` This is continuation line 2 of the 4-line logical statement that begins at source line 163. Its exact fragment is 'lambda _module, inputs, scale=head_activation_scale: ('. The important operands are lambda: the value named 'lambda' in this block; _module: the value named 'module' in this block; inputs: the value named 'inputs' in this block. Read together, the complete operation is: This calls 'quantized_model.head.register_forward_pre_hook' with ' lambda _module, inputs, scale=head_activation_scale: (torch.round(inputs[0] / scale).clamp(-128, 127) * scale,)' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 165: `torch.round(inputs[0] / scale).clamp(-128, 127) * scale,` This is continuation line 3 of the 4-line logical statement that begins at source line 163. Its exact fragment is 'torch.round(inputs[0] / scale).clamp(-128, 127) * scale,'. The important operands are torch: the value named 'torch' in this block; round: the value named 'round' in this block; inputs: the value named 'inputs' in this block. Read together, the complete operation is: This calls 'quantized_model.head.register_forward_pre_hook' with ' lambda _module, inputs, scale=head_activation_scale: (torch.round(inputs[0] / scale).clamp(-128, 127) * scale,)' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 166: `))` This is continuation line 4 of the 4-line logical statement that begins at source line 163. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'quantized_model.head.register_forward_pre_hook' with ' lambda _module, inputs, scale=head_activation_scale: (torch.round(inputs[0] / scale).clamp(-128, 127) * scale,)' inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 167: `head_weight_scale = float(quantized_model.head.weight.detach().abs().amax().item()) / 127.0` This assigns ‘float(quantized_model.head.weight.detach().abs().amax().item()) / 127.0‘ to ‘head_weight_scale ‘ in host memory. Here head_weight_scale means a learned coefficient multiplied by an activation. The important operands are float: the value named ’float’ in this block; quantized_model: the value named ’quantized model’ in this block; head: the value named ’head’ in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 168: `head_accumulator_scale = head_activation_scale * head_weight_scale` This assigns ‘head_activation_scale * head_weight_scale‘ to ‘head_accumulator_scale ‘ in host memory. Here head_accumulator_scale means a widened running sum that prevents intermediate overflow. The important operands are head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation; head_weight_scale: a learned coefficient multiplied by an activation. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 169: `if quantized_model.head.bias is not None:` This executes the following indented host-code branch only when ‘quantized_model.head.bias is not None‘ is true. The important operands are quantized_model: the value named ’quantized model’ in this block; head: the value named ’head’ in this block; bias: an additive learned offset applied after accumulation. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 170: `quantized_model.head.bias.data.copy_(` This statement continues through source line 172; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘quantized_model.head.bias.data.copy_‘ with ‘ torch.round(quantized_model.head.bias.detach() / head_accumulator_scale) * head_accumulator_scale‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 171: `torch.round(quantized_model.head.bias.detach() / head_accumulator_scale)` This is continuation line 2 of the 3-line logical statement that begins at source line 170. Its exact fragment is ‘torch.round(quantized_model.head.bias.deta / head_accumulator_scale)‘. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; quantized_model: the value named ’quantized model’ in this block. Read together, the complete operation is: This calls ‘quantized_model.head.bias.data.copy_‘ with ‘ torch.round(quantized_model.head.bias.detach() / head_accumulator_scale) * head_accumulator_scale‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 172: `* head_accumulator_scale)` This is continuation line 3 of the 3-line logical statement that begins at source line 170. Its exact fragment is ‘* head_accumulator_scale)‘. The important operands are head_accumulator_scale: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This calls ‘quantized_model.head.bias.data.copy_‘ with ‘ torch.round(quantized_model.head.bias.detach() / head_accumulator_scale) * head_accumulator_scale‘ inside the function prepare_hardware_emulation, which handles prepare hardware emulation. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 174: `input_probe = torch.zeros(1, 3, 1, 1)` This assigns ‘torch.zeros(1, 3, 1, 1)‘ to ‘input_probe ‘ in host memory. Here input_probe means the value named ’input probe’ in this block. The important operands are torch: the value named ’torch’ in this block; zeros: the value named ’zeros’ in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 175: `_, input_scale, input_zero_point = quantize_cifar_input_u8(input_probe)` This assigns ‘quantize_cifar_input_u8(input_probe)‘ to ‘_, input_scale, input_zero_point ‘ in host memory. Here _ means the value named ” in this block. The important operands are quantize_cifar_input_u8: the value named ’quantize cifar input u8’ in this block; input_probe: the value named ’input probe’ in this block. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 176: `report = HardwareEmulationReport(` This statement continues through source line 185; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_p

head_activation_scale=head_activation_scale, head_accumulator_scale=head_accumulator_scale, weight_granularity=weight_
quantized_weights=tuple(weight_reports),)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 177: `fractional_bits=fractional_bits,` This is continuation line 2 of the 10-line logical statement that begins at source line 176. Its exact fragment is ‘fractional_bits=fractional_bits,‘. The important operands are fractional_bits: the value named ’fractional bits’ in this block. Read together, the complete operation is: This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_point, head_activation_scale=head_activation_scale, head_accumulator_scale=head_accumulato
weight_granularity=weight_granularity, quantized_weights=tuple(weight_reports),)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 178: `fixed_point_scale=1 << fractional_bits,` This is continuation line 3 of the 10-line logical statement that begins at source line 176. Its exact fragment is ‘fixed_point_scale=1 << fractional_bits,‘. The important operands are fixed_point_scale: the real-value spacing represented by one quantized integer step; fractional_bits: the value named ’fractional bits’ in this block. Read together, the complete operation is: This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_point, head_activation_scale=head_activation_scale, head_accumulator_scale=head_accumulato
weight_granularity=weight_granularity, quantized_weights=tuple(weight_reports),)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 179: `input_scale=input_scale,` This is continuation line 4 of the 10-line logical statement that begins at source line 176. Its exact fragment is ‘input_scale=input_scale,‘. The important operands are input_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_point, head_activation_scale=head_activation_scale, head_accumulator_scale=head_accumulato
weight_granularity=weight_granularity, quantized_weights=tuple(weight_reports),)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 180: `input_zero_point=input_zero_point,` This is continuation line 5 of the 10-line logical statement that begins at source line 176. Its exact fragment is ‘input_zero_point=input_zero_point,‘. The important operands are input_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_point, head_activation_scale=head_activation_scale, head_accumulator_scale=head_accumulator_scale, weight_granularity=weight_granularity, quantized_weights=tuple(weight_re
)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 181: `head_activation_scale=head_activation_scale,` This is continuation line 6 of the 10-line logical statement that begins at source line 176. Its exact fragment is ‘head_activation_scale=head_activation_scale,‘. The important operands are head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_point, head_activation_scale=

head_accumulator_scale=head_accumulator_scale, weight_granularity=weight_granularity, quantized_weights=tuple(weight_re)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 182: `head_accumulator_scale=head_accumulator_scale,` This is continuation line 7 of the 10-line logical statement that begins at source line 176. Its exact fragment is ‘head_accumulator_scale=head_accumulator_scale,‘. The important operands are head_accumulator_scale: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_point, head_activation_scale= head_accumulator_scale=head_accumulator_scale, weight_granularity=weight_granularity, quantized_weights=tuple(weight_re)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 183: `weight_granularity=weight_granularity,` This is continuation line 8 of the 10-line logical statement that begins at source line 176. Its exact fragment is ‘weight_granularity=weight_granularity,‘. The important operands are weight_granularity: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_point, head_activation_scale=head_activation_scale, head_accumulator weight_granularity=weight_granularity, quantized_weights=tuple(weight_reports),)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 184: `quantized_weights=tuple(weight_reports),` This is continuation line 9 of the 10-line logical statement that begins at source line 176. Its exact fragment is ‘quantized_weights=tuple(weight_reports),‘. The important operands are quantized_weights: a learned coefficient multiplied by an activation; tuple: the value named ’tuple’ in this block; weight_reports: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_point, head_activation_scale=head_activation_scale, head_accumulator weight_granularity=weight_granularity, quantized_weights=tuple(weight_reports),)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 185: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 176. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘HardwareEmulationReport(fractional_bits=fractional_bits, fixed_point_scale=1 << fractional_bits, input_scale=input_scale, input_zero_point=input_zero_point, head_activation_scale=head_activation_scale, head_accumulator_scale=head_accumulator_scale, weight_granularity=weight_granularity, quantized_weights=tuple(weight_re)‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are HardwareEmulationReport: the value named ’hardware emulation report’ in this block; fractional_bits: the value named ’fractional bits’ in this block; fixed_point_scale: the real-value spacing represented by one quantized integer step. Context: the function prepare_hardware_emulation, which handles prepare hardware emulation.

Line 186: `return quantized_model, report` This returns ‘quantized_model, report‘ to the caller of the function prepare_hardware_emulation, which handles prepare hardware emulation, ending this invocation. The important operands are quantized_model: the value named ’quantized model’ in this block; report: the value named ’report’ in this block.

Line 189: `def _replace_lif_modules(module: nn.Module, fractional_bits: int) -> None:` This starts function _replace_lif_modules. Its indented body performs replace lif modules as part of the file that defines calibration,

signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 190: `for name, child in list(module.named_children()):` This iterates name, child over ‘list(module.named_children Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _replace_lif_modules, which handles replace lif modules.

Line 191: `if isinstance(child, MultiStepHardResetLIF):` This executes the following indented host-code branch only when ‘isinstance(child, MultiStepHardResetLIF)‘ is true. The important operands are isinstance: the value named ’isinstance’ in this block; child: the value named ’child’ in this block; MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block. Context: the function _replace_lif_modules, which handles replace lif modules.

Line 192: `setattr(module, name, FixedPointHardResetLIF(` This statement continues through source line 195; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘setattr‘ with ‘module, name, FixedPointHardResetLIF(threshold=child.threshold, fractional_bits=fractional_bits,)‘ inside the function _replace_lif_modules, which handles replace lif modules. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 193: `threshold=child.threshold,` This is continuation line 2 of the 4-line logical statement that begins at source line 192. Its exact fragment is ‘threshold=child.threshold,‘. The important operands are threshold: the membrane level at which a neuron emits a spike; child: the value named ’child’ in this block. Read together, the complete operation is: This calls ‘setattr‘ with ‘module, name, FixedPointHardResetLIF(threshold=child.threshold, fractional_bits=fractional_bits,)‘ inside the function _replace_lif_modules, which handles replace lif modules. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 194: `fractional_bits=fractional_bits,` This is continuation line 3 of the 4-line logical statement that begins at source line 192. Its exact fragment is ‘fractional_bits=fractional_bits,‘. The important operands are fractional_bits: the value named ’fractional bits’ in this block. Read together, the complete operation is: This calls ‘setattr‘ with ‘module, name, FixedPointHardResetLIF(threshold=child.threshold, fractional_bits=fractional_bits,)‘ inside the function _replace_lif_modules, which handles replace lif modules. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 195: `))` This is continuation line 4 of the 4-line logical statement that begins at source line 192. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘setattr‘ with ‘module, name, FixedPointHardResetLIF(threshold=child.threshold, fractional_bits=fractional_bits,)‘ inside the function _replace_lif_modules, which handles replace lif modules. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 196: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function _replace_lif_modules, which handles replace lif modules.

Line 197: `_replace_lif_modules(child, fractional_bits)` This calls ‘_replace_lif_modules‘ with ‘child, fractional_bits‘ inside the function _replace_lif_modules, which handles replace lif modules. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 200: `def input_scale_from_domain() -> float:` This starts function input_scale_from_domain. Its indented body performs input scale from domain as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 201: `probe = torch.zeros(1, 3, 1, 1)` This assigns ‘torch.zeros(1, 3, 1, 1)‘ to ‘probe ‘ in host memory. Here probe means the value named ’probe’ in this block. The important operands are torch: the value named ’torch’ in this block; zeros: the value named ’zeros’ in this block. Context: the function input_scale_from_domain, which handles input scale from domain.

Line 202: `_, scale, _ = quantize_cifar_input_u8(probe)` This assigns ‘quantize_cifar_input_u8(probe)‘ to ‘_, scale, _ ‘ in host memory. Here _ means the value named ” in this block. The important operands are quantize_cifar_input_u8: the value named ’quantize cifar input u8’ in this block; probe: the value named ’probe’ in this block. Context: the function input_scale_from_domain, which handles input scale from domain.

Line 203: `return scale` This returns ‘scale‘ to the caller of the function input_scale_from_domain, which handles input scale from domain, ending this invocation. The important operands are scale: the real-value spacing represented by one quantized integer step.

Line 206: `def _install_projection_requantizers(` This starts function _install_projection_requantizers. Its indented body performs install projection requantizers as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 207: `model: Spikingformer,` Inside the function _install_projection_requantizers, which handles install projection requantizers, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are model: the value named ’model’ in this block; Spikingformer: the value named ’spikingformer’ in this block.

Line 208: `weight_scales: dict[str, torch.Tensor],` Inside the function _install_projection_requantizers, which handles install projection requantizers, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are weight_scales: a learned coefficient multiplied by an activation; dict: the value named ’dict’ in this block; str: the value named ’str’ in this block.

Line 209: `*,` Inside the function _install_projection_requantizers, which handles install projection requantizers, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The expression is a literal or structural operator and has no hidden software state.

Line 210: `fractional_bits: int,` Inside the function _install_projection_requantizers, which handles install projection requantizers, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are fractional_bits: the value named ’fractional bits’ in this block.

Line 211: `image_input_scale: float,` Inside the function _install_projection_requantizers, which handles install projection requantizers, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are image_input_scale: the real-value spacing represented by one quantized integer step; float: the value named ’float’ in this block.

Line 212: `) -> None:` Inside the function _install_projection_requantizers, which handles install projection requantizers, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are None: the value named ’none’ in this block.

Line 213: `modules = dict(model.named_modules())` This assigns ‘dict(model.named_modules())‘ to ‘modules ‘ in host memory. Here modules means the value named ’modules’ in this block. The important operands are dict: the value named ’dict’ in this block; model: the value named ’model’ in this block; named_modules: the value named ’named modules’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 214: `output_scale = 1 << fractional_bits` This assigns ‘1 << fractional_bits‘ to ‘output_scale ‘ in host memory. Here output_scale means the real-value spacing represented by one quantized integer step. The important operands are fractional_bits: the value named ’fractional bits’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 215: `for batch_norm_name, batch_norm in modules.items():` This iterates batch_norm_name, batch_norm over ‘modules.items()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _install_projection_requantizers, which handles install projection requantizers.

Line 216: `if not isinstance(batch_norm, (nn.BatchNorm1d, nn.BatchNorm2d)):` This executes the following indented host-code branch only when ‘not isinstance(batch_norm, (nn.BatchNorm1d, nn.BatchNorm2d))‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; batch_norm: the value named ’batch norm’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 217: `continue` Inside the function _install_projection_requantizers, which handles install projection requantizers, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are continue: the value named ’continue’ in this block.

Line 218: `if not batch_norm_name.endswith("_bn"):` This executes the following indented host-code branch only when ‘not batch_norm_name.endswith("_bn")‘ is true. The important operands are not: the value named ’not’ in this block; batch_norm_name: the value named ’batch norm name’ in this block; endswith: the value named ’endswith’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 219: `raise ValueError(f"cannot locate projection for {batch_norm_name}")` This rejects an invalid state by raising ‘ValueError(f"cannot locate projection for {batch_norm_name}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 220: `convolution_name = batch_norm_name[:-3] + "_conv"` This assigns ‘batch_norm_name[:-3] + "_conv"‘ to ‘convolution_name ‘ in host memory. Here convolution_name means the value named ’convolution name’ in this block. The important operands are batch_norm_name: the value named ’batch norm name’ in this block; _conv: the value named ’conv’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 221: `convolution = modules.get(convolution_name)` This assigns ‘modules.get(convolution_name)‘ to ‘convolution ‘ in host memory. Here convolution means the value named ’convolution’ in this block. The important operands are modules: the value named ’modules’ in this block; get: the value named ’get’ in this block; convolution_name: the value named ’convolution name’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 222: `if not isinstance(convolution, (nn.Conv1d, nn.Conv2d)):` This executes the following indented host-code branch only when ‘not isinstance(convolution, (nn.Conv1d, nn.Conv2d))‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; convolution: the value named ’convolution’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 223: `raise ValueError(f"cannot locate {convolution_name}")` This rejects an invalid state by raising ‘ValueError(f"cannot locate {convolution_name}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 224: `raw_weight_scale = weight_scales[convolution_name]` This assigns ‘weight_scales[convolution_name]‘ to ‘raw_weight_scale ‘ in host memory. Here raw_weight_scale means a learned coefficient multiplied by an activation. The important operands are weight_scales: a learned coefficient multiplied by an activation; convolution_name: the value named ’convolution name’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 225: `if raw_weight_scale.numel() == 1:` This executes the following indented host-code branch only when ‘raw_weight_scale.numel() == 1‘ is true. The important operands are raw_weight_scale: a learned coefficient multiplied by an activation; numel: the value named ’numel’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 226: `weight_scale = raw_weight_scale.repeat(convolution.out_channels)` This assigns ‘raw_weight_scale.repeat(c to ‘weight_scale ‘ in host memory. Here weight_scale means a learned coefficient multiplied by an activation. The important operands are raw_weight_scale: a learned coefficient multiplied by an activation; repeat: the value named ’repeat’ in this block; convolution: the value named ’convolution’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 227: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function _install_projection_requantizers, which handles install projection requantizers.

Line 228: `weight_scale = raw_weight_scale` This assigns ‘raw_weight_scale‘ to ‘weight_scale ‘ in host memory. Here weight_scale means a learned coefficient multiplied by an activation. The important operands are raw_weight_scale: a learned coefficient multiplied by an activation. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 229: `projection_input_scale = (` This statement continues through source line 230; the explanation uses the complete expression rather than only the first visual fragment. Inside the function _install_projection_requantizers, which handles install projection requantizers, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are projection_input_scale: the real-value spacing represented by one quantized integer step; image_input_scale: the real-value spacing represented by one quantized integer step; convolution_name: the value named ’convolution name’ in this block.

Line 230: `image_input_scale if convolution_name == "patch_embed.block0_conv" else 1.0)` This is continuation line 2 of the 2-line logical statement that begins at source line 229. Its exact fragment is ‘image_input_scale if convolution_name == "patch_embed.block0_conv" else 1.0)‘. The important operands are image_input_scale: the real-value spacing represented by one quantized integer step; convolution_name: the value named ’convolution name’ in this block; patch_embed: the value named ’patch embed’ in this block. Read together, the complete operation is: Inside the function _install_projection_requantizers, which handles install projection requantizers, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are projection_input_scale: the real-value spacing represented by one quantized integer step; image_input_scale: the real-value spacing represented by one quantized integer step; convolution_name: the value named ’convolution name’ in this block.

Line 231: `bn_scale = batch_norm.weight.detach() / torch.sqrt(` This statement continues through source line 232; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘batch_norm.weight.detach(/ torch.sqrt(batch_norm.running_var.detach() + batch_norm.eps)‘ to ‘bn_scale ‘ in host memory. Here bn_scale means the real-value spacing represented by one quantized integer step. The important operands are batch_norm: the value named ’batch norm’ in this block; weight: a learned coefficient multiplied by an activation; detach: the value named ’detach’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 232: `batch_norm.running_var.detach() + batch_norm.eps)` This is continuation line 2 of the 2-line logical statement that begins at source line 231. Its exact fragment is ‘batch_norm.running_var.detach() + batch_norm.eps)‘. The important operands are batch_norm: the value named ’batch norm’ in this block; running_var: the value named ’running var’ in this block; detach: the value named ’detach’ in this block. Read together, the complete operation is: This assigns ‘batch_norm.weight.detach() / torch.sqrt(batch_norm.running_var.detach() + batch_norm.eps)‘ to ‘bn_scale ‘ in host memory. Here bn_scale means the real-value spacing represented by one quantized integer step. The important operands are batch_norm: the value named ’batch norm’ in this block; weight: a learned coefficient multiplied by an activation; detach: the value named ’detach’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 233: `factor = projection_input_scale * weight_scale * bn_scale * output_scale` This assigns ‘projection_input_scale * weight_scale * bn_scale * output_scale‘ to ‘factor ‘ in host memory. Here factor means the value named ’factor’ in this block. The important operands are projection_input_scale: the real-value spacing represented by one quantized integer step; weight_scale: a learned coefficient multiplied by an activation; bn_scale: the real-value spacing represented by one quantized integer step. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 234: `multiplier_and_shift = [quantize_flexible_scale(float(item)) for item in factor]` This assigns ‘[quantize_flexible_scale(float(item)) for item in factor]‘ to ‘multiplier_and_shift ‘ in host memory. Here multiplier_and_shift means the value named ’multiplier and shift’ in this block. The important operands are quantize_flexible_scale: the real-value spacing represented by one quantized integer step; float: the value named ’float’ in this block; item: the value named ’item’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 235: `multiplier = torch.tensor(` This statement continues through source line 236; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘torch.tensor([item[0] for item in multiplier_and_shift], dtype=torch.int64)‘ to ‘multiplier ‘ in host memory. Here multiplier means the value named ’multiplier’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block; item: the value named ’item’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 236: `[item[0] for item in multiplier_and_shift], dtype=torch.int64)` This is continuation line 2 of the 2-line logical statement that begins at source line 235. Its exact fragment is ‘[item[0] for item in multiplier_and_shift], dtype=torch.int64)‘. The important operands are item: the value named ’item’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘torch.tensor([item[0] for item in multiplier_and_shift], dtype=torch.int64)‘ to ‘multiplier ‘ in host memory. Here multiplier means the value named ’multiplier’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block; item: the value named ’item’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 237: `effective_shift = torch.tensor(` This statement continues through source line 238; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘torch.tensor([15 + item[1] for item in multiplier_and_shift], dtype=torch.int64)‘ to ‘effective_shift ‘ in host memory. Here effective_shift means the value named ’effective shift’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block; item: the value named ’item’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 238: `[15 + item[1] for item in multiplier_and_shift], dtype=torch.int64)` This is continuation line 2 of the 2-line logical statement that begins at source line 237. Its exact fragment is ‘[15 + item[1] for item in multiplier_and_shift], dtype=torch.int64)‘. The important operands are item: the value named ’item’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘torch.tensor([15 + item[1] for item in multiplier_and_shift], dtype=torch.int64)‘ to ‘effective_shift ‘ in host memory. Here effective_shift means the value named ’effective shift’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block; item: the value named ’item’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 239: `if convolution.bias is None:` This executes the following indented host-code branch only when ‘convolution.bias is None‘ is true. The important operands are convolution: the value named ’convolution’ in this block; bias: an additive learned offset applied after accumulation; is: the value named ’is’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 240: `convolution_bias = torch.zeros_like(batch_norm.running_mean)` This assigns ‘torch.zeros_like(batch_norm.ru
to ‘convolution_bias ‘ in host memory. Here convolution_bias means an additive learned offset applied after accumulation. The important operands are torch: the value named ’torch’ in this block; zeros_like: the value named ’zeros like’ in this block; batch_norm: the value named ’batch norm’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 241: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function _install_projection_requantizers, which handles install projection requantizers.

Line 242: `convolution_bias = convolution.bias.detach()` This assigns ‘convolution.bias.detach()‘ to ‘convolution_bias ‘ in host memory. Here convolution_bias means an additive learned offset applied after accumulation. The important operands are convolution: the value named ’convolution’ in this block; bias: an additive learned offset applied after accumulation; detach: the value named ’detach’ in this block. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 243: `folded_bias = (` This statement continues through source line 245; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘((convolution_bias - batch_norm.running_mean.detach()) * bn_scale + batch_norm.bias.detach())‘ to ‘folded_bias ‘ in host memory. Here folded_bias means an additive learned offset applied after accumulation. The important operands are convolution_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 244: `(convolution_bias - batch_norm.running_mean.detach()) * bn_scale` This is continuation line 2 of the 3-line logical statement that begins at source line 243. Its exact fragment is ‘(convolution_bias - batch_norm.running_mean.deta * bn_scale‘. The important operands are convolution_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Read together, the complete operation is: This assigns ‘((convolution_bias - batch_norm.running_mean.detach()) * bn_scale + batch_norm.bias.detach())‘ to ‘folded_bias ‘ in host memory. Here folded_bias means an additive learned offset applied after accumulation. The important operands are convolution_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 245: `+ batch_norm.bias.detach())` This is continuation line 3 of the 3-line logical statement that begins at source line 243. Its exact fragment is ‘+ batch_norm.bias.detach())‘. The important operands are batch_norm: the value named ’batch norm’ in this block; bias: an additive learned offset applied after accumulation; detach: the value named ’detach’ in this block. Read together, the complete operation is: This assigns ‘((convolution_bias - batch_norm.running_mean.detach()) * bn_scale + batch_norm.bias.detach())‘ to ‘folded_bias ‘ in host memory. Here folded_bias means an additive learned offset applied after accumulation. The important operands are convolution_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 246: `bias_integer = torch.round(folded_bias * output_scale).to(torch.int64)` This assigns ‘torch.round(folded_ * output_scale).to(torch.int64)‘ to ‘bias_integer ‘ in host memory. Here bias_integer means an additive learned offset applied after accumulation. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; folded_bias: an additive learned offset applied after accumulation. Context: the function _install_projection_requantizers, which handles install projection requantizers.

Line 247: `batch_norm.register_forward_hook(_projection_hook(` This statement continues through source line 255; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘batch_norm.register_forward with ‘_projection_hook(convolution_bias=convolution_bias, projection_input_scale=projection_input_scale, weight_scale=wei multiplier=multiplier, effective_shift=effective_shift, bias_integer=bias_integer, output_scale=output_scale,)‘ inside the function _install_projection_requantizers, which handles install projection requantizers. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 248: `convolution_bias=convolution_bias,` This is continuation line 2 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘convolution_bias=convolution_bias,‘. The important operands are convolution_bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This calls ‘batch_norm.register_forward_hook‘ with ‘_projection_hook(convolution_bias=convolution_bias, projection_input_scale=projection_input_scale, weight_scale=weight_scale, multiplier=multiplier, effective_shift=effective_shift, bias_integer=bias_integer, output_scale=output_scale,)‘ inside the function _install_projection_requantizers, which

handles install projection requantizers. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 249: `projection_input_scale=projection_input_scale,` This is continuation line 3 of the 9-line logical statement that begins at source line 247. Its exact fragment is 'projection_input_scale=projection_input_scale,'. The important operands are projection_input_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This calls 'batch_norm.register_forward_hook' with '_projection_hook(convolution_bias=convolution_bias, projection_input_scale=projection_input_scale, weight_scale=weight_scale, multiplier=multiplier, effective_shift=effective_shift, bias_integer=bias_integer, output_scale=output_scale,)' inside the function _install_projection_requantizers, which handles install projection requantizers. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 250: `weight_scale=weight_scale,` This is continuation line 4 of the 9-line logical statement that begins at source line 247. Its exact fragment is 'weight_scale=weight_scale,'. The important operands are weight_scale: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls 'batch_norm.register_forward_hook' with '_projection_hook(convolution_bias=convolution_bias, projection_input_scale=projection_input_scale, weight_scale=wei multiplier=multiplier, effective_shift=effective_shift, bias_integer=bias_integer, output_scale=output_scale,)' inside the function _install_projection_requantizers, which handles install projection requantizers. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 251: `multiplier=multiplier,` This is continuation line 5 of the 9-line logical statement that begins at source line 247. Its exact fragment is 'multiplier=multiplier,'. The important operands are multiplier: the value named 'multiplier' in this block. Read together, the complete operation is: This calls 'batch_norm.register_forward_hook' with '_projection_hook(convolution_bias=convolution_bias, projection_input_scale=projection_input_scale, weight_scale=weight_scale, multiplier=multiplier, effective_shift=effective_shift, bias_integer=bias_integer, output_scale=output_scale,)' inside the function _install_projection_requantizers, which handles install projection requantizers. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 252: `effective_shift=effective_shift,` This is continuation line 6 of the 9-line logical statement that begins at source line 247. Its exact fragment is 'effective_shift=effective_shift,'. The important operands are effective_shift: the value named 'effective shift' in this block. Read together, the complete operation is: This calls 'batch_norm.register_forward_hook' with '_projection_hook(convolution_bias=convolution_bias, projection_input_scale=proj weight_scale=weight_scale, multiplier=multiplier, effective_shift=effective_shift, bias_integer=bias_integer, output_scale=outpu)' inside the function _install_projection_requantizers, which handles install projection requantizers. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 253: `bias_integer=bias_integer,` This is continuation line 7 of the 9-line logical statement that begins at source line 247. Its exact fragment is 'bias_integer=bias_integer,'. The important operands are bias_integer: an additive learned offset applied after accumulation. Read together, the complete operation is: This calls 'batch_norm.register_forward_hook' with '_projection_hook(convolution_bias=convolution_bias, projection_input_scale=projection_input_scale, weight_scale=wei multiplier=multiplier, effective_shift=effective_shift, bias_integer=bias_integer, output_scale=output_scale,)' inside the function _install_projection_requantizers, which handles install projection requantizers. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 254: `output_scale=output_scale,` This is continuation line 8 of the 9-line logical statement that begins at source line 247. Its exact fragment is 'output_scale=output_scale,'. The important operands are output_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This calls 'batch_norm.register_forward_hook' with '_projection_hook(convolution_bias=convolution_bias, projection_input_scale=proj weight_scale=weight_scale, multiplier=multiplier, effective_shift=effective_shift, bias_integer=bias_integer, output_scale=outpu)' inside the function _install_projection_requantizers, which handles install projection requantizers. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 255: `))` This is continuation line 9 of the 9-line logical statement that begins at source line 247. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘batch_norm.register_forward_hook‘ with ‘_projection_hook(convolution_bias=convolution_bias, projection_input_scale=projection_input_scale, weight_scale=weight_scale, multiplier=multiplier, effective_shift=effective_shift, bias_integer=bias_integer, output_scale=output_scale,)‘ inside the function _install_projection_ which handles install projection requantizers. The call advances the host-side job that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math.

Line 258: `def _projection_hook(` This starts function _projection_hook. Its indented body performs projection hook as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 259: `*,` Inside the function _projection_hook, which handles projection hook, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The expression is a literal or structural operator and has no hidden software state.

Line 260: `convolution_bias: torch.Tensor,` Inside the function _projection_hook, which handles projection hook, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are convolution_bias: an additive learned offset applied after accumulation; torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 261: `projection_input_scale: float,` Inside the function _projection_hook, which handles projection hook, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are projection_input_scale: the real-value spacing represented by one quantized integer step; float: the value named ’float’ in this block.

Line 262: `weight_scale: torch.Tensor,` Inside the function _projection_hook, which handles projection hook, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are weight_scale: a learned coefficient multiplied by an activation; torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 263: `multiplier: torch.Tensor,` Inside the function _projection_hook, which handles projection hook, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are multiplier: the value named ’multiplier’ in this block; torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 264: `effective_shift: torch.Tensor,` Inside the function _projection_hook, which handles projection hook, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are effective_shift: the value named ’effective shift’ in this block; torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 265: `bias_integer: torch.Tensor,` Inside the function _projection_hook, which handles projection hook, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are bias_integer: an additive learned offset applied after accumulation; torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 266: `output_scale: int,` Inside the function _projection_hook, which handles projection hook, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are output_scale: the real-value spacing represented by one quantized integer step.

Line 267: `):` Inside the function _projection_hook, which handles projection hook, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The expression is a literal or structural operator and has no hidden software state.

Line 268: `def requantize(` This starts function requantize. Its indented body performs requantize as part of the file that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math; it runs only when called.

Line 269: `_module: nn.Module,` Inside the function requantize, which handles requantize, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are _module: the value named 'module' in this block; nn: the value named 'nn' in this block; Module: the value named 'module' in this block.

Line 270: `inputs: tuple[torch.Tensor, ...],` Inside the function requantize, which handles requantize, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are inputs: the value named 'inputs' in this block; tuple: the value named 'tuple' in this block; torch: the value named 'torch' in this block.

Line 271: `_output: torch.Tensor,` Inside the function requantize, which handles requantize, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are _output: the value named 'output' in this block; torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 272: `) -> torch.Tensor:` Inside the function requantize, which handles requantize, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 273: `convolution_output = inputs[0]` This assigns ‘inputs[0]‘ to ‘convolution_output ‘ in host memory. Here convolution_output means the value named 'convolution output' in this block. The important operands are inputs: the value named 'inputs' in this block. Context: the function requantize, which handles requantize.

Line 274: `channel_shape = (1, -1) + (1,) * (convolution_output.ndim - 2)` This assigns ‘(1, -1) + (1,) * (convolution_output.ndim - 2)‘ to ‘channel_shape ‘ in host memory. Here channel_shape means the value named 'channel shape' in this block. The important operands are convolution_output: the value named 'convolution output' in this block; ndim: the value named 'ndim' in this block. Context: the function requantize, which handles requantize.

Line 275: `device = convolution_output.device` This assigns ‘convolution_output.device‘ to ‘device ‘ in host memory. Here device means the value named 'device' in this block. The important operands are convolution_output: the value named 'convolution output' in this block; device: the value named 'device' in this block. Context: the function requantize, which handles requantize.

Line 276: `bias = convolution_bias.to(device).reshape(channel_shape)` This assigns ‘convolution_bias.to(device).reshape
to ‘bias ‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are convolution_bias: an additive learned offset applied after accumulation; to: the value named 'to' in this block; device: the value named 'device' in this block. Context: the function requantize, which handles requantize.

Line 277: `accumulator_scale = (` This statement continues through source line 278; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(weight_scale.to(device).reshape(channel_shape) * projection_input_scale)‘ to ‘accumulator_scale ‘ in host memory. Here accumulator_scale means a widened running sum that prevents intermediate overflow. The important operands are weight_scale: a learned coefficient multiplied by an activation; to: the value named 'to' in this block; device: the value named 'device' in this block. Context: the function requantize, which handles requantize.

Line 278: `weight_scale.to(device).reshape(channel_shape) * projection_input_scale)` This is continuation line 2 of the 2-line logical statement that begins at source line 277. Its exact fragment is ‘weight_scale.to(device).reshape(channel_sha
* projection_input_scale)‘. The important operands are weight_scale: a learned coefficient multiplied by an activation; to: the value named 'to' in this block; device: the value named 'device' in this block. Read together, the complete

operation is: This assigns ‘(weight_scale.to(device).reshape(channel_shape) * projection_input_scale)‘ to ‘accumulator_scale ‘ in host memory. Here accumulator_scale means a widened running sum that prevents intermediate overflow. The important operands are weight_scale: a learned coefficient multiplied by an activation; to: the value named ’to’ in this block; device: the value named ’device’ in this block. Context: the function requantize, which handles requantize.

Line 279: `accumulator = torch.round(` This statement continues through source line 280; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘torch.round((convolution_output - bias) / accumulator_scale).to(torch.int64)‘ to ‘accumulator ‘ in host memory. Here accumulator means a widened running sum that prevents intermediate overflow. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; convolution_output: the value named ’convolution output’ in this block. Context: the function requantize, which handles requantize.

Line 280: `(convolution_output - bias) / accumulator_scale).to(torch.int64)` This is continuation line 2 of the 2-line logical statement that begins at source line 279. Its exact fragment is ‘(convolution_output - bias) / accumulator_scale).to(torch.int64)‘. The important operands are convolution_output: the value named ’convolution output’ in this block; bias: an additive learned offset applied after accumulation; accumulator_scale: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This assigns ‘torch.round((convolution_output - bias) / accumulator_scale).to(torch.int64)‘ to ‘accumulator ‘ in host memory. Here accumulator means a widened running sum that prevents intermediate overflow. The important operands are torch: the value named ’torch’ in this block; round: the value named ’round’ in this block; convolution_output: the value named ’convolution output’ in this block. Context: the function requantize, which handles requantize.

Line 281: `q15 = multiplier.to(device).reshape(channel_shape)` This assigns ‘multiplier.to(device).reshape(channel_shape to ‘q15 ‘ in host memory. Here q15 means the value named ’q15’ in this block. The important operands are multiplier: the value named ’multiplier’ in this block; to: the value named ’to’ in this block; device: the value named ’device’ in this block. Context: the function requantize, which handles requantize.

Line 282: `shifts = effective_shift.to(device).reshape(channel_shape)` This assigns ‘effective_shift.to(device).reshape(to ‘shifts ‘ in host memory. Here shifts means the value named ’shifts’ in this block. The important operands are effective_shift: the value named ’effective shift’ in this block; to: the value named ’to’ in this block; device: the value named ’device’ in this block. Context: the function requantize, which handles requantize.

Line 283: `product = accumulator * q15` This assigns ‘accumulator * q15‘ to ‘product ‘ in host memory. Here product means the value named ’product’ in this block. The important operands are accumulator: a widened running sum that prevents intermediate overflow; q15: the value named ’q15’ in this block. Context: the function requantize, which handles requantize.

Line 284: `magnitude = product.abs()` This assigns ‘product.abs()‘ to ‘magnitude ‘ in host memory. Here magnitude means the value named ’magnitude’ in this block. The important operands are product: the value named ’product’ in this block; abs: the value named ’abs’ in this block. Context: the function requantize, which handles requantize.

Line 285: `rounding_bias = torch.where(` This statement continues through source line 289; the explanation uses the complete expression rather than only the first visual fragment. Inside the function requantize, which handles requantize, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are rounding_bias: an additive learned offset applied after accumulation; torch: the value named ’torch’ in this block; where: the value named ’where’ in this block.

Line 286: `shifts == 0,` This is continuation line 2 of the 5-line logical statement that begins at source line 285. Its exact fragment is ‘shifts == 0,‘. The important operands are shifts: the value named ’shifts’ in this block. Read together, the complete operation is: Inside the function requantize, which handles requantize, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are rounding_bias: an additive learned offset applied after accumulation; torch: the value named ’torch’ in this block; where: the value named ’where’ in this block.

Line 287: `torch.zeros_like(shifts),` This is continuation line 3 of the 5-line logical statement that begins at source line 285. Its exact fragment is 'torch.zeros_like(shifts),'. The important operands are torch: the value named 'torch' in this block; zeros_like: the value named 'zeros like' in this block; shifts: the value named 'shifts' in this block. Read together, the complete operation is: Inside the function requantize, which handles requantize, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are rounding_bias: an additive learned offset applied after accumulation; torch: the value named 'torch' in this block; where: the value named 'where' in this block.

Line 288: `torch.bitwise_left_shift(torch.ones_like(shifts), shifts - 1),` This is continuation line 4 of the 5-line logical statement that begins at source line 285. Its exact fragment is 'torch.bitwise_left_shift(torch.ones_like(shifts), shifts - 1),'. The important operands are torch: the value named 'torch' in this block; bitwise_left_shift: the value named 'bitwise left shift' in this block; ones_like: the value named 'ones like' in this block. Read together, the complete operation is: Inside the function requantize, which handles requantize, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are rounding_bias: an additive learned offset applied after accumulation; torch: the value named 'torch' in this block; where: the value named 'where' in this block.

Line 289: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 285. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function requantize, which handles requantize, this statement advances the host implementation that defines calibration, signed/unsigned affine quantization, rounding, saturation, and folded projection math. The important operands are rounding_bias: an additive learned offset applied after accumulation; torch: the value named 'torch' in this block; where: the value named 'where' in this block.

Line 290: `rounded_magnitude = torch.bitwise_right_shift(` This statement continues through source line 291; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'torch.bitwise_right_shift(magnitude + rounding_bias, shifts)' to 'rounded_magnitude ' in host memory. Here rounded_magnitude means the value named 'rounded magnitude' in this block. The important operands are torch: the value named 'torch' in this block; bitwise_right_shift: the value named 'bitwise right shift' in this block; magnitude: the value named 'magnitude' in this block. Context: the function requantize, which handles requantize.

Line 291: `magnitude + rounding_bias, shifts)` This is continuation line 2 of the 2-line logical statement that begins at source line 290. Its exact fragment is 'magnitude + rounding_bias, shifts)'. The important operands are magnitude: the value named 'magnitude' in this block; rounding_bias: an additive learned offset applied after accumulation; shifts: the value named 'shifts' in this block. Read together, the complete operation is: This assigns 'torch.bitwise_right_shift(magnitude + rounding_bias, shifts)' to 'rounded_magnitude ' in host memory. Here rounded_magnitude means the value named 'rounded magnitude' in this block. The important operands are torch: the value named 'torch' in this block; bitwise_right_shift: the value named 'bitwise right shift' in this block; magnitude: the value named 'magnitude' in this block. Context: the function requantize, which handles requantize.

Line 292: `rounded = torch.where(product < 0, -rounded_magnitude, rounded_magnitude)` This assigns 'torch.where(pro
< 0, -rounded_magnitude, rounded_magnitude)' to 'rounded ' in host memory. Here rounded means the value named 'rounded' in this block. The important operands are torch: the value named 'torch' in this block; where: the value named 'where' in this block; product: the value named 'product' in this block. Context: the function requantize, which handles requantize.

Line 293: `quantized = rounded + bias_integer.to(device).reshape(channel_shape)` This assigns 'rounded + bias_integer.to(device).reshape(channel_shape)' to 'quantized ' in host memory. Here quantized means the value named 'quantized' in this block. The important operands are rounded: the value named 'rounded' in this block; bias_integer: an additive learned offset applied after accumulation; to: the value named 'to' in this block. Context: the function requantize, which handles requantize.

Line 294: `quantized = quantized.clamp(-(1 << 23), (1 << 23) - 1)` This assigns 'quantized.clamp(-(1 << 23), (1 << 23) - 1)' to 'quantized ' in host memory. Here quantized means the value named 'quantized' in this block. The important operands are quantized: the value named 'quantized' in this block; clamp: the value named 'clamp' in this block. Context: the function requantize, which handles requantize.

Line 295: `return quantized.to(convolution_output.dtype) / output_scale` This returns ‘quantized.to(convolution_outpu / output_scale‘ to the caller of the function requantize, which handles requantize, ending this invocation. The important operands are quantized: the value named ’quantized’ in this block; to: the value named ’to’ in this block; convolution_output: the value named ’convolution output’ in this block.

Line 297: `return requantize` This returns ‘requantize‘ to the caller of the function _projection_hook, which handles projection hook, ending this invocation. The important operands are requantize: the conversion of a wide accumulated result back into a bounded activation code.

43 software/spikingformer_reference.py

Role in the complete system: reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

43.1 Beginner architecture map

Dataflow responsibility The source checkpoint is reconstructed with its original timestep and spike behavior. A separate hardware-emulation path applies exported integer coefficients, allowing full-test accuracy and per-logit disagreement to be measured before board execution.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: MultiStepHardResetLIF, MLP, SpikingSelfAttention, SpikingTransformer, SpikingTokenizer, Spikingformer, LoadedCheckpoint. Functions: load_checkpoint, fold_batch_norm, _split_heads, _observe, ___init___, forward, ___init___, forward, ___init___, forward, ___init___, forward, ___init___, forward, ___init___, forward. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

43.2 Complete source

```
"""Checkpoint-faithful Spikingformer inference and hardware preparation.

The upstream CIFAR-10 implementation fixes every neuron to a four-step,
hard-reset leaky integrate-and-fire (LIF) model.  This module reproduces that
inference graph with ordinary PyTorch operators.  It deliberately avoids a
CuPy dependency so the reference remains executable on CPU and on any CUDA
device supported by PyTorch.

Module and parameter names match the upstream checkpoint exactly.  A strict
load therefore detects missing, unexpected, or shape-incompatible tensors
before any accuracy or hardware-equivalence result is accepted.
"""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any, Callable, Mapping

import torch
import torch.nn as nn


TensorObserver = Callable[[str, torch.Tensor, bool], None]


class MultiStepHardResetLIF(nn.Module):
    """Exact inference equation of the upstream ``MultiStepLIFNode``.

    With the checkpoint's ``tau=2``, ``decay_input=True`` and ``v_reset=0``,
    each timestep first computes ``v_pre = v + (x - v) / 2``.  A spike is one
    where ``v_pre >= threshold``.  Spiking lanes are then reset to zero while
    all other lanes retain ``v_pre`` for the next timestep.

    The state is intentionally local to one call.  This is equivalent to the
    upstream validation loop, which invokes ``functional.reset_net`` after
    every image batch.
    """

    def __init__(self, *, tau: float = 2.0, threshold: float = 1.0) -> None:
        super().__init__()
        if tau <= 1.0:
            raise ValueError("tau must be greater than one")
        self.tau = float(tau)
        self.threshold = float(threshold)

    def forward(self, sequence: torch.Tensor) -> torch.Tensor:
```

```
        if sequence.ndim < 2:
            raise ValueError("a multi-step LIF input must have shape [T, ...]")
        voltage = torch.zeros_like(sequence[0])
        spikes: list[torch.Tensor] = []
        for timestep in range(sequence.shape[0]):
            voltage = voltage + (sequence[timestep] - voltage) / self.tau
            spike = (voltage >= self.threshold).to(sequence.dtype)
            voltage = voltage * (1.0 - spike)
            spikes.append(spike)
        return torch.stack(spikes, dim=0)


class MLP(nn.Module):
    """Spikingformer's two 1x1-convolution feed-forward projections."""

    def __init__(self, channels: int, hidden_channels: int) -> None:
        super().__init__()
        self.mlp1_lif = MultiStepHardResetLIF()
        self.mlp1_conv = nn.Conv2d(channels, hidden_channels, kernel_size=1)
        self.mlp1_bn = nn.BatchNorm2d(hidden_channels)
        self.mlp2_lif = MultiStepHardResetLIF()
        self.mlp2_conv = nn.Conv2d(hidden_channels, channels, kernel_size=1)
        self.mlp2_bn = nn.BatchNorm2d(channels)
        self.hidden_channels = hidden_channels

    def forward(
        self,
        value: torch.Tensor,
        *,
        prefix: str,
        observer: TensorObserver | None,
    ) -> torch.Tensor:
        time_steps, batch, channels, height, width = value.shape
        value = self.mlp1_lif(value)
        _observe(observer, f"{prefix}.mlp1_lif", value, True)
        value = self.mlp1_conv(value.flatten(0, 1))
        value = self.mlp1_bn(value).reshape(
            time_steps, batch, self.hidden_channels, height, width)
        _observe(observer, f"{prefix}.mlp1_bn", value, False)
        value = self.mlp2_lif(value)
        _observe(observer, f"{prefix}.mlp2_lif", value, True)
        value = self.mlp2_conv(value.flatten(0, 1))
        value = self.mlp2_bn(value).reshape(
            time_steps, batch, channels, height, width)
        _observe(observer, f"{prefix}.mlp2_bn", value, False)
        return value


class SpikingSelfAttention(nn.Module):
    """Binary Q/K/V self-attention used by the trained checkpoint."""

    def __init__(self, channels: int, heads: int) -> None:
        super().__init__()
        if channels % heads:
            raise ValueError("channels must be divisible by heads")
        self.channels = channels
        self.heads = heads
        self.head_dim = channels // heads
        self.proj_lif = MultiStepHardResetLIF()
        self.q_conv = nn.Conv1d(channels, channels, kernel_size=1, bias=False)
        self.q_bn = nn.BatchNorm1d(channels)
        self.q_lif = MultiStepHardResetLIF()
        self.k_conv = nn.Conv1d(channels, channels, kernel_size=1, bias=False)
        self.k_bn = nn.BatchNorm1d(channels)
        self.k_lif = MultiStepHardResetLIF()
        self.v_conv = nn.Conv1d(channels, channels, kernel_size=1, bias=False)
        self.v_bn = nn.BatchNorm1d(channels)
        self.v_lif = MultiStepHardResetLIF()
        self.attn_lif = MultiStepHardResetLIF(threshold=0.5)
        self.proj_conv = nn.Conv1d(channels, channels, kernel_size=1)
        self.proj_bn = nn.BatchNorm1d(channels)

    def forward(
        self,
        value: torch.Tensor,
        *,
        prefix: str,
        observer: TensorObserver | None,
    ) -> torch.Tensor:
        time_steps, batch, channels, height, width = value.shape
        value = self.proj_lif(value)
        _observe(observer, f"{prefix}.proj_lif", value, True)
        value = value.flatten(3)
        tokens = value.shape[-1]
        projection_input = value.flatten(0, 1)

        query = self.q_bn(self.q_conv(projection_input)).reshape(
```

```
            time_steps, batch, channels, tokens)
        query = self.q_lif(query)
        _observe(observer, f"{prefix}.q_lif", query, True)
        key = self.k_bn(self.k_conv(projection_input)).reshape(
            time_steps, batch, channels, tokens)
        key = self.k_lif(key)
        _observe(observer, f"{prefix}.k_lif", key, True)
        value_projection = self.v_bn(self.v_conv(projection_input)).reshape(
            time_steps, batch, channels, tokens)
        value_projection = self.v_lif(value_projection)
        _observe(observer, f"{prefix}.v_lif", value_projection, True)

        query = _split_heads(query, self.heads)
        key = _split_heads(key, self.heads)
        value_projection = _split_heads(value_projection, self.heads)
        attention = query @ key.transpose(-2, -1)
        _observe(observer, f"{prefix}.qk", attention, False)
        value = (attention @ value_projection) * 0.125
        _observe(observer, f"{prefix}.attention_value", value, False)
        value = value.transpose(3, 4).reshape(
            time_steps, batch, channels, tokens)
        value = self.attn_lif(value)
        _observe(observer, f"{prefix}.attn_lif", value, True)
        value = self.proj_bn(self.proj_conv(value.flatten(0, 1)))
        value = value.reshape(time_steps, batch, channels, height, width)
        _observe(observer, f"{prefix}.proj_bn", value, False)
        return value


class SpikingTransformer(nn.Module):
    """One residual attention block followed by one residual spiking MLP."""

    def __init__(self, channels: int, heads: int, mlp_ratio: int) -> None:
        super().__init__()
        # The upstream model creates these LayerNorm parameters but does not
        # apply them in forward.  Keeping them is required for strict loading.
        self.norm1 = nn.LayerNorm(channels, eps=1e-6)
        self.attn = SpikingSelfAttention(channels, heads)
        self.norm2 = nn.LayerNorm(channels, eps=1e-6)
        self.mlp = MLP(channels, channels * mlp_ratio)

    def forward(
        self,
        value: torch.Tensor,
        *,
        prefix: str,
        observer: TensorObserver | None,
    ) -> torch.Tensor:
        value = value + self.attn(
            value, prefix=f"{prefix}.attn", observer=observer)
        _observe(observer, f"{prefix}.attention_residual", value, False)
        value = value + self.mlp(
            value, prefix=f"{prefix}.mlp", observer=observer)
        _observe(observer, f"{prefix}.mlp_residual", value, False)
        return value


class SpikingTokenizer(nn.Module):
    """Five 3x3 convolutions that convert 32x32 RGB frames to 8x8 tokens."""

    def __init__(self, input_channels: int, embed_dim: int) -> None:
        super().__init__()
        widths = (embed_dim // 8, embed_dim // 4, embed_dim // 2, embed_dim)
        self.block0_conv = nn.Conv2d(
            input_channels, widths[0], kernel_size=3, padding=1, bias=False)
        self.block0_bn = nn.BatchNorm2d(widths[0])
        self.block1_lif = MultiStepHardResetLIF()
        self.block1_conv = nn.Conv2d(
            widths[0], widths[1], kernel_size=3, padding=1, bias=False)
        self.block1_bn = nn.BatchNorm2d(widths[1])
        self.block2_lif = MultiStepHardResetLIF()
        self.block2_conv = nn.Conv2d(
            widths[1], widths[2], kernel_size=3, padding=1, bias=False)
        self.block2_bn = nn.BatchNorm2d(widths[2])
        self.block3_lif = MultiStepHardResetLIF()
        self.block3_mp = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        self.block3_conv = nn.Conv2d(
            widths[2], widths[3], kernel_size=3, padding=1, bias=False)
        self.block3_bn = nn.BatchNorm2d(widths[3])
        self.block4_lif = MultiStepHardResetLIF()
        self.block4_mp = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
        self.block4_conv = nn.Conv2d(
            widths[3], widths[3], kernel_size=3, padding=1, bias=False)
        self.block4_bn = nn.BatchNorm2d(widths[3])

    def forward(
        self,
```

```
        value: torch.Tensor,
        *,
        observer: TensorObserver | None,
    ) -> torch.Tensor:
        time_steps, batch, _, height, width = value.shape
        value = self.block0_bn(self.block0_conv(value.flatten(0, 1)))
        value = value.reshape(time_steps, batch, -1, height, width)
        _observe(observer, "patch_embed.block0_bn", value, False)

        value = self.block1_lif(value)
        _observe(observer, "patch_embed.block1_lif", value, True)
        value = self.block1_bn(self.block1_conv(value.flatten(0, 1)))
        value = value.reshape(time_steps, batch, -1, height, width)
        _observe(observer, "patch_embed.block1_bn", value, False)

        value = self.block2_lif(value)
        _observe(observer, "patch_embed.block2_lif", value, True)
        value = self.block2_bn(self.block2_conv(value.flatten(0, 1)))
        value = value.reshape(time_steps, batch, -1, height, width)
        _observe(observer, "patch_embed.block2_bn", value, False)

        value = self.block3_lif(value).flatten(0, 1)
        _observe(observer, "patch_embed.block3_lif", value, True)
        value = self.block3_mp(value)
        value = self.block3_bn(self.block3_conv(value))
        value = value.reshape(time_steps, batch, -1, height // 2, width // 2)
        _observe(observer, "patch_embed.block3_bn", value, False)

        value = self.block4_lif(value).flatten(0, 1)
        _observe(observer, "patch_embed.block4_lif", value, True)
        value = self.block4_mp(value)
        value = self.block4_bn(self.block4_conv(value))
        value = value.reshape(time_steps, batch, -1, height // 4, width // 4)
        _observe(observer, "patch_embed.block4_bn", value, False)
        return value


class Spikingformer(nn.Module):
    """The exact Spikingformer-4-384 graph used by the CIFAR-10 run."""

    def __init__(
        self,
        *,
        time_steps: int = 4,
        input_channels: int = 3,
        classes: int = 10,
        embed_dim: int = 384,
        heads: int = 12,
        depth: int = 4,
        mlp_ratio: int = 4,
    ) -> None:
        super().__init__()
        self.time_steps = time_steps
        self.patch_embed = SpikingTokenizer(input_channels, embed_dim)
        self.block = nn.ModuleList([
            SpikingTransformer(embed_dim, heads, mlp_ratio)
            for _ in range(depth)
        ])
        self.head = nn.Linear(embed_dim, classes)

    def forward(
        self,
        image: torch.Tensor,
        observer: TensorObserver | None = None,
    ) -> torch.Tensor:
        value = image.unsqueeze(0).repeat(self.time_steps, 1, 1, 1, 1)
        _observe(observer, "input_repeated", value, False)
        value = self.patch_embed(value, observer=observer)
        for index, transformer in enumerate(self.block):
            value = transformer(
                value, prefix=f"block.{index}", observer=observer)
        value = value.flatten(3).mean(3)
        _observe(observer, "global_average_pool", value, False)
        value = self.head(value.mean(0))
        _observe(observer, "head", value, False)
        return value


@dataclass(frozen=True)
class LoadedCheckpoint:
    """Strictly loaded model plus provenance stored by the training run."""

    model: Spikingformer
    epoch: int
    metric: float
    architecture: str
    checkpoint_path: Path
```

```

def load_checkpoint(
    checkpoint_path: str | Path,
    *,
    device: str | torch.device = "cpu",
) -> LoadedCheckpoint:
    """Load the trained checkpoint and reject every state-dictionary mismatch."""

    path = Path(checkpoint_path).expanduser().resolve()
    checkpoint: Mapping[str, Any] = torch.load(path, map_location="cpu")
    arguments = checkpoint.get("args")
    if arguments is None:
        raise ValueError("checkpoint does not contain its training arguments")
    model = Spikingformer(
        time_steps=int(getattr(arguments, "time_step")),
        input_channels=3,
        classes=int(getattr(arguments, "num_classes")),
        embed_dim=int(getattr(arguments, "dim")),
        heads=int(getattr(arguments, "num_heads")),
        depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))),
        mlp_ratio=int(getattr(arguments, "mlp_ratio")),
    )
    state = checkpoint.get("state_dict")
    if not isinstance(state, Mapping):
        raise ValueError("checkpoint does not contain a state_dict mapping")
    model.load_state_dict(state, strict=True)
    model.eval().to(device)
    return LoadedCheckpoint(
        model=model,
        epoch=int(checkpoint.get("epoch", -1)),
        metric=float(checkpoint.get("metric", float("nan"))),
        architecture=str(checkpoint.get("arch", "unknown")),
        checkpoint_path=path,
    )


def fold_batch_norm(
    convolution: nn.Conv1d | nn.Conv2d,
    batch_norm: nn.BatchNorm1d | nn.BatchNorm2d,
) -> tuple[torch.Tensor, torch.Tensor]:
    """Return convolution weight/bias tensors with evaluation BatchNorm folded.

    For each output channel ``o``, BatchNorm applies
    ``gamma[o] * (y - mean[o]) / sqrt(var[o] + eps) + beta[o]``.
    Multiplying the convolution kernel and shifting its bias by those constants
    produces exactly the same evaluation result up to floating-point rounding.
    """

    if convolution.out_channels != batch_norm.num_features:
        raise ValueError("convolution and BatchNorm output widths differ")
    weight = convolution.weight.detach()
    if convolution.bias is None:
        bias = torch.zeros(
            convolution.out_channels, dtype=weight.dtype, device=weight.device)
    else:
        bias = convolution.bias.detach()
    scale = batch_norm.weight.detach() / torch.sqrt(
        batch_norm.running_var.detach() + batch_norm.eps)
    reshape = (scale.shape[0],) + (1,) * (weight.ndim - 1)
    folded_weight = weight * scale.reshape(reshape)
    folded_bias = (bias - batch_norm.running_mean.detach()) * scale
    folded_bias = folded_bias + batch_norm.bias.detach()
    return folded_weight, folded_bias


def _split_heads(value: torch.Tensor, heads: int) -> torch.Tensor:
    time_steps, batch, channels, tokens = value.shape
    head_dim = channels // heads
    value = value.transpose(-1, -2)
    value = value.reshape(time_steps, batch, tokens, heads, head_dim)
    return value.permute(0, 1, 3, 2, 4)


def _observe(
    observer: TensorObserver | None,
    name: str,
    value: torch.Tensor,
    is_spike: bool,
) -> None:
    if observer is not None:
        observer(name, value, is_spike)
```

43.3 Line annotations

Line 1: `"""Checkpoint-faithful Spikingformer inference and hardware preparation.` This is executable documentation attached to the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `The upstream CIFAR-10 implementation fixes every neuron to a four-step,` Inside the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are The: the value named 'the' in this block; upstream: the value named 'upstream' in this block; CIFAR: the value named 'cifar' in this block.

Line 4: `hard-reset leaky integrate-and-fire (LIF) model. This module reproduces that` Inside the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are hard: the value named 'hard' in this block; reset: the value named 'reset' in this block; leaky: the retained fraction of prior membrane state.

Line 5: `inference graph with ordinary PyTorch operators. It deliberately avoids a` Inside the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are inference: the value named 'inference' in this block; graph: the value named 'graph' in this block; with: the value named 'with' in this block.

Line 6: `CuPy dependency so the reference remains executable on CPU and on any CUDA` Inside the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are CuPy: the value named 'cu py' in this block; dependency: the value named 'dependency' in this block; so: the value named 'so' in this block.

Line 7: `device supported by PyTorch.` Inside the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are device: the value named 'device' in this block; supported: the value named 'supported' in this block; by: the value named 'by' in this block.

Line 9: `Module and parameter names match the upstream checkpoint exactly. A strict` Inside the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are Module: the value named 'module' in this block; and: the value named 'and' in this block; parameter: the value named 'parameter' in this block.

Line 10: `load therefore detects missing, unexpected, or shape-incompatible tensors` Inside the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are load: the value named 'load' in this block; therefore: the value named 'therefore' in this block; detects: the value named 'detects' in this block.

Line 11: `before any accuracy or hardware-equivalence result is accepted.` Inside the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are before: the value named 'before' in this block; any: the value named 'any' in this block; accuracy: the value named 'accuracy' in this block.

Line 12: `"""` This is executable documentation attached to the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 14: `from __future__ import annotations` This imports the named Python API so the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `from pathlib import Path` This imports the named Python API so the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `from typing import Any, Callable, Mapping` This imports the named Python API so the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `import torch` This imports the named Python API so the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `import torch.nn as nn` This imports the named Python API so the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `TensorObserver = Callable[[str, torch.Tensor, bool], None]` This assigns ‘Callable[[str, torch.Tensor, bool], None]‘ to ‘TensorObserver ‘ in host memory. Here TensorObserver means the value named ’tensor observer’ in this block. The important operands are Callable: the value named ’callable’ in this block; str: the value named ’str’ in this block; torch: the value named ’torch’ in this block. Context: the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 27: `class MultiStepHardResetLIF(nn.Module):` This defines class MultiStepHardResetLIF, which groups data and methods for multi step hard reset lif. Instances exist in host memory and organize the flow that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 28: `"""Exact inference equation of the upstream "MultiStepLIFNode".` This is executable documentation attached to the class MultiStepHardResetLIF, which handles multi step hard reset lif. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 30: `With the checkpoint's "tau=2", "decay_input=True" and "v_reset=0",` Inside the class MultiStepHardResetLIF, which handles multi step hard reset lif, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are With: the value named ’with’ in this block; the: the value named ’the’ in this block; checkpoint: the value named ’checkpoint’ in this block.

Line 31: `each timestep first computes "v_pre = v + (x - v) / 2". A spike is one` Inside the class MultiStepHardResetLIF, which handles multi step hard reset lif, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are each: the value named ’each’ in this block; timestep: one discrete SNN update instant; first: the value named ’first’ in this block.

Line 32: `where "v_pre >= threshold". Spiking lanes are then reset to zero while` Inside the class MultiStepHardResetLIF, which handles multi step hard reset lif, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are where: the value named 'where' in this block; v_pre: the value named 'v pre' in this block; threshold: the membrane level at which a neuron emits a spike.

Line 33: `all other lanes retain "v_pre" for the next timestep.` Inside the class MultiStepHardResetLIF, which handles multi step hard reset lif, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are all: the value named 'all' in this block; other: the value named 'other' in this block; lanes: one parallel arithmetic element within a vector or tensor datapath.

Line 35: `The state is intentionally local to one call. This is equivalent to the` Inside the class MultiStepHardResetLIF, which handles multi step hard reset lif, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are The: the value named 'the' in this block; state: the finite-state-machine phase retained in flip-flops; is: the value named 'is' in this block.

Line 36: `upstream validation loop, which invokes "functional.reset_net" after` Inside the class MultiStepHardResetI which handles multi step hard reset lif, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are upstream: the value named 'upstream' in this block; validation: the producer's statement that its current payload is meaningful; loop: the value named 'loop' in this block.

Line 37: `every image batch.` Inside the class MultiStepHardResetLIF, which handles multi step hard reset lif, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are every: the value named 'every' in this block; image: the value named 'image' in this block; batch: the value named 'batch' in this block.

Line 38: `"""` This is executable documentation attached to the class MultiStepHardResetLIF, which handles multi step hard reset lif. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 40: `def __init__(self, *, tau: float = 2.0, threshold: float = 1.0) -> None:` This starts function ___init___. Its indented body performs init as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 41: `super().__init__()` This calls 'super' with ').___init___(' inside the function ___init___, which handles init. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 42: `if tau <= 1.0:` This executes the following indented host-code branch only when 'tau <= 1.0' is true. The important operands are tau: the value named 'tau' in this block. Context: the function ___init___, which handles init.

Line 43: `raise ValueError("tau must be greater than one")` This rejects an invalid state by raising 'ValueError("tau must be greater than one")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 44: `self.tau = float(tau)` This assigns 'float(tau)' to 'self.tau ' in host memory. Here tau means the value named 'tau' in this block. The important operands are float: the value named 'float' in this block; tau: the value named 'tau' in this block. Context: the function ___init___, which handles init.

Line 45: `self.threshold = float(threshold)` This assigns ‘float(threshold)‘ to ‘self.threshold ‘ in host memory. Here threshold means the membrane level at which a neuron emits a spike. The important operands are float: the value named ’float’ in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function ___init___, which handles init.

Line 47: `def forward(self, sequence: torch.Tensor) -> torch.Tensor:` This starts function forward. Its indented body performs forward as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 48: `if sequence.ndim < 2:` This executes the following indented host-code branch only when ‘sequence.ndim < 2‘ is true. The important operands are sequence: the value named ’sequence’ in this block; ndim: the value named ’ndim’ in this block. Context: the function forward, which handles forward.

Line 49: `raise ValueError("a multi-step LIF input must have shape [T, ...]")` This rejects an invalid state by raising ‘ValueError("a multi-step LIF input must have shape [T, ...]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 50: `voltage = torch.zeros_like(sequence[0])` This assigns ‘torch.zeros_like(sequence[0])‘ to ‘voltage ‘ in host memory. Here voltage means the value named ’voltage’ in this block. The important operands are torch: the value named ’torch’ in this block; zeros_like: the value named ’zeros like’ in this block; sequence: the value named ’sequence’ in this block. Context: the function forward, which handles forward.

Line 51: `spikes: list[torch.Tensor] = []` This assigns ‘[]‘ to ‘spikes‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the function forward, which handles forward.

Line 52: `for timestep in range(sequence.shape[0]):` This iterates timestep over ‘range(sequence.shape[0])‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function forward, which handles forward.

Line 53: `voltage = voltage + (sequence[timestep] - voltage) / self.tau` This assigns ‘voltage + (sequence[timestep] - voltage) / self.tau‘ to ‘voltage ‘ in host memory. Here voltage means the value named ’voltage’ in this block. The important operands are voltage: the value named ’voltage’ in this block; sequence: the value named ’sequence’ in this block; timestep: one discrete SNN update instant. Context: the function forward, which handles forward.

Line 54: `spike = (voltage >= self.threshold).to(sequence.dtype)` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are spike: a binary event produced when a neuron crosses its threshold; voltage: the value named ’voltage’ in this block; threshold: the membrane level at which a neuron emits a spike.

Line 55: `voltage = voltage * (1.0 - spike)` This assigns ‘voltage * (1.0 - spike)‘ to ‘voltage ‘ in host memory. Here voltage means the value named ’voltage’ in this block. The important operands are voltage: the value named ’voltage’ in this block; spike: a binary event produced when a neuron crosses its threshold. Context: the function forward, which handles forward.

Line 56: `spikes.append(spike)` This calls ‘spikes.append‘ with ‘spike‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 57: `return torch.stack(spikes, dim=0)` This returns ‘torch.stack(spikes, dim=0)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are torch: the value named ’torch’ in this block; stack: the value named ’stack’ in this block; spikes: a binary event produced when a neuron crosses its threshold.

Line 60: `class MLP(nn.Module):` This defines class MLP, which groups data and methods for mlp. Instances exist in host memory and organize the flow that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 61: `"""Spikingformer's two 1x1-convolution feed-forward projections."""` This is executable documentation attached to the class MLP, which handles mlp. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 63: `def __init__(self, channels: int, hidden_channels: int) -> None:` This starts function ___init___. Its indented body performs init as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 64: `super().__init__()` This calls 'super' with ').___init___(' inside the function ___init___, which handles init. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 65: `self.mlp1_lif = MultiStepHardResetLIF()` This assigns 'MultiStepHardResetLIF()' to 'self.mlp1_lif ' in host memory. Here mlp1_lif means the value named 'mlp1 lif' in this block. The important operands are MultiStepHardResetLIF: the value named 'multi step hard reset lif' in this block. Context: the function ___init___, which handles init.

Line 66: `self.mlp1_conv = nn.Conv2d(channels, hidden_channels, kernel_size=1)` This assigns 'nn.Conv2d(channels, hidden_channels, kernel_size=1)' to 'self.mlp1_conv ' in host memory. Here mlp1_conv means the value named 'mlp1 conv' in this block. The important operands are nn: the value named 'nn' in this block; Conv2d: the value named 'conv2d' in this block; channels: the value named 'channels' in this block. Context: the function ___init___, which handles init.

Line 67: `self.mlp1_bn = nn.BatchNorm2d(hidden_channels)` This assigns 'nn.BatchNorm2d(hidden_channels)' to 'self.mlp1_bn ' in host memory. Here mlp1_bn means the value named 'mlp1 bn' in this block. The important operands are nn: the value named 'nn' in this block; BatchNorm2d: the value named 'batch norm2d' in this block; hidden_channels: the value named 'hidden channels' in this block. Context: the function ___init___, which handles init.

Line 68: `self.mlp2_lif = MultiStepHardResetLIF()` This assigns 'MultiStepHardResetLIF()' to 'self.mlp2_lif ' in host memory. Here mlp2_lif means the value named 'mlp2 lif' in this block. The important operands are MultiStepHardResetLIF: the value named 'multi step hard reset lif' in this block. Context: the function ___init___, which handles init.

Line 69: `self.mlp2_conv = nn.Conv2d(hidden_channels, channels, kernel_size=1)` This assigns 'nn.Conv2d(hidden_cha channels, kernel_size=1)' to 'self.mlp2_conv ' in host memory. Here mlp2_conv means the value named 'mlp2 conv' in this block. The important operands are nn: the value named 'nn' in this block; Conv2d: the value named 'conv2d' in this block; hidden_channels: the value named 'hidden channels' in this block. Context: the function ___init___, which handles init.

Line 70: `self.mlp2_bn = nn.BatchNorm2d(channels)` This assigns 'nn.BatchNorm2d(channels)' to 'self.mlp2_bn ' in host memory. Here mlp2_bn means the value named 'mlp2 bn' in this block. The important operands are nn: the value named 'nn' in this block; BatchNorm2d: the value named 'batch norm2d' in this block; channels: the value named 'channels' in this block. Context: the function ___init___, which handles init.

Line 71: `self.hidden_channels = hidden_channels` This assigns 'hidden_channels' to 'self.hidden_channels ' in host memory. Here hidden_channels means the value named 'hidden channels' in this block. The important operands are hidden_channels: the value named 'hidden channels' in this block. Context: the function ___init___, which handles init.

Line 73: `def forward(` This starts function forward. Its indented body performs forward as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 74: `self,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 75: `value: torch.Tensor,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are value: the value named 'value' in this block; torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 76: `*,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 77: `prefix: str,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are prefix: the value named 'prefix' in this block; str: the value named 'str' in this block.

Line 78: `observer: TensorObserver | None,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are observer: the value named 'observer' in this block; TensorObserver: the value named 'tensor observer' in this block; None: the value named 'none' in this block.

Line 79: `) -> torch.Tensor:` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 80: `time_steps, batch, channels, height, width = value.shape` This assigns 'value.shape' to 'time_steps, batch, channels, height, width ' in host memory. Here time_steps means the value named 'time steps' in this block. The important operands are value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function forward, which handles forward.

Line 81: `value = self.mlp1_lif(value)` This assigns 'self.mlp1_lif(value)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are mlp1_lif: the value named 'mlp1 lif' in this block; value: the value named 'value' in this block. Context: the function forward, which handles forward.

Line 82: `_observe(observer, f"{prefix}.mlp1_lif", value, True)` This calls '_observe' with 'observer, f"{prefix}.mlp1_lif value, True' inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 83: `value = self.mlp1_conv(value.flatten(0, 1))` This assigns 'self.mlp1_conv(value.flatten(0, 1))' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are mlp1_conv: the value named 'mlp1 conv' in this block; value: the value named 'value' in this block; flatten: the value named 'flatten' in this block. Context: the function forward, which handles forward.

Line 84: `value = self.mlp1_bn(value).reshape(` This statement continues through source line 85; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.mlp1_bn(value).reshape(time_steps, batch, self.hidden_channels, height, width)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are mlp1_bn: the value named 'mlp1 bn' in this block; value: the value named 'value' in this block; reshape: the value named 'reshape' in this block. Context: the function forward, which handles forward.

Line 85: `time_steps, batch, self.hidden_channels, height, width)` This is continuation line 2 of the 2-line logical statement that begins at source line 84. Its exact fragment is 'time_steps, batch, self.hidden_channels, height, width)'. The important operands are time_steps: the value named 'time steps' in this block; batch: the value named 'batch' in this block; hidden_channels: the value named 'hidden channels' in this block. Read together, the complete operation is: This assigns 'self.mlp1_bn(value).reshape(time_steps, batch, self.hidden_channels, height, width)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are mlp1_bn: the value named 'mlp1 bn' in this block; value: the value named 'value' in this block; reshape: the value named 'reshape' in this block. Context: the function forward, which handles forward.

Line 86: `_observe(observer, f"{prefix}.mlp1_bn", value, False)` This calls '_observe' with 'observer, f"{prefix}.mlp1_bn value, False' inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 87: `value = self.mlp2_lif(value)` This assigns 'self.mlp2_lif(value)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are mlp2_lif: the value named 'mlp2 lif' in this block; value: the value named 'value' in this block. Context: the function forward, which handles forward.

Line 88: `_observe(observer, f"{prefix}.mlp2_lif", value, True)` This calls '_observe' with 'observer, f"{prefix}.mlp2_lif value, True' inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 89: `value = self.mlp2_conv(value.flatten(0, 1))` This assigns 'self.mlp2_conv(value.flatten(0, 1))' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are mlp2_conv: the value named 'mlp2 conv' in this block; value: the value named 'value' in this block; flatten: the value named 'flatten' in this block. Context: the function forward, which handles forward.

Line 90: `value = self.mlp2_bn(value).reshape(` This statement continues through source line 91; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.mlp2_bn(value).reshape(time_steps, batch, channels, height, width)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are mlp2_bn: the value named 'mlp2 bn' in this block; value: the value named 'value' in this block; reshape: the value named 'reshape' in this block. Context: the function forward, which handles forward.

Line 91: `time_steps, batch, channels, height, width)` This is continuation line 2 of the 2-line logical statement that begins at source line 90. Its exact fragment is 'time_steps, batch, channels, height, width)'. The important operands are time_steps: the value named 'time steps' in this block; batch: the value named 'batch' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This assigns 'self.mlp2_bn(value).reshape(time_steps, batch, channels, height, width)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are mlp2_bn: the value named 'mlp2 bn' in this block; value: the value named 'value' in this block; reshape: the value named 'reshape' in this block. Context: the function forward, which handles forward.

Line 92: `_observe(observer, f"{prefix}.mlp2_bn", value, False)` This calls '_observe' with 'observer, f"{prefix}.mlp2_bn value, False' inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 93: `return value` This returns 'value' to the caller of the function forward, which handles forward, ending this invocation. The important operands are value: the value named 'value' in this block.

Line 96: `class SpikingSelfAttention(nn.Module):` This defines class SpikingSelfAttention, which groups data and methods for spiking self attention. Instances exist in host memory and organize the flow that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 97: `"""Binary Q/K/V self-attention used by the trained checkpoint."""` This is executable documentation attached to the class SpikingSelfAttention, which handles spiking self attention. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 99: `def __init__(self, channels: int, heads: int) -> None:` This starts function ___init___. Its indented body performs init as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 100: `super().__init__()` This calls ‘super‘ with ‘).___init___(‘ inside the function ___init___, which handles init. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 101: `if channels % heads:` This executes the following indented host-code branch only when ‘channels % heads‘ is true. The important operands are channels: the value named ’channels’ in this block; heads: the value named ’heads’ in this block. Context: the function ___init___, which handles init.

Line 102: `raise ValueError("channels must be divisible by heads")` This rejects an invalid state by raising ‘ValueError("channels must be divisible by heads")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 103: `self.channels = channels` This assigns ‘channels‘ to ‘self.channels ‘ in host memory. Here channels means the value named ’channels’ in this block. The important operands are channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 104: `self.heads = heads` This assigns ‘heads‘ to ‘self.heads ‘ in host memory. Here heads means the value named ’heads’ in this block. The important operands are heads: the value named ’heads’ in this block. Context: the function ___init___, which handles init.

Line 105: `self.head_dim = channels // heads` This assigns ‘channels // heads‘ to ‘self.head_dim ‘ in host memory. Here head_dim means the value named ’head dim’ in this block. The important operands are channels: the value named ’channels’ in this block; heads: the value named ’heads’ in this block. Context: the function ___init___, which handles init.

Line 106: `self.proj_lif = MultiStepHardResetLIF()` This assigns ‘MultiStepHardResetLIF()‘ to ‘self.proj_lif ‘ in host memory. Here proj_lif means the value named ’proj lif’ in this block. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block. Context: the function ___init___, which handles init.

Line 107: `self.q_conv = nn.Conv1d(channels, channels, kernel_size=1, bias=False)` This assigns ‘nn.Conv1d(channels, channels, kernel_size=1, bias=False)‘ to ‘self.q_conv ‘ in host memory. Here q_conv means the value named ’q conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv1d: the value named ’conv1d’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 108: `self.q_bn = nn.BatchNorm1d(channels)` This assigns ‘nn.BatchNorm1d(channels)‘ to ‘self.q_bn ‘ in host memory. Here q_bn means the value named ’q bn’ in this block. The important operands are nn: the value named ’nn’ in this block; BatchNorm1d: the value named ’batch norm1d’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 109: `self.q_lif = MultiStepHardResetLIF()` This assigns ‘MultiStepHardResetLIF()‘ to ‘self.q_lif ‘ in host memory. Here q_lif means the value named ’q lif’ in this block. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block. Context: the function ___init___, which handles init.

Line 110: `self.k_conv = nn.Conv1d(channels, channels, kernel_size=1, bias=False)` This assigns ‘nn.Conv1d(channels, channels, kernel_size=1, bias=False)‘ to ‘self.k_conv ‘ in host memory. Here k_conv means the value named ’k conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv1d: the value named ’conv1d’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 111: `self.k_bn = nn.BatchNorm1d(channels)` This assigns ‘nn.BatchNorm1d(channels)‘ to ‘self.k_bn ‘ in host memory. Here k_bn means the value named ’k bn’ in this block. The important operands are nn: the value named ’nn’ in this block; BatchNorm1d: the value named ’batch norm1d’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 112: `self.k_lif = MultiStepHardResetLIF()` This assigns ‘MultiStepHardResetLIF()‘ to ‘self.k_lif ‘ in host memory. Here k_lif means the value named ’k lif’ in this block. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block. Context: the function ___init___, which handles init.

Line 113: `self.v_conv = nn.Conv1d(channels, channels, kernel_size=1, bias=False)` This assigns ‘nn.Conv1d(channels, channels, kernel_size=1, bias=False)‘ to ‘self.v_conv ‘ in host memory. Here v_conv means the value named ’v conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv1d: the value named ’conv1d’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 114: `self.v_bn = nn.BatchNorm1d(channels)` This assigns ‘nn.BatchNorm1d(channels)‘ to ‘self.v_bn ‘ in host memory. Here v_bn means the value named ’v bn’ in this block. The important operands are nn: the value named ’nn’ in this block; BatchNorm1d: the value named ’batch norm1d’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 115: `self.v_lif = MultiStepHardResetLIF()` This assigns ‘MultiStepHardResetLIF()‘ to ‘self.v_lif ‘ in host memory. Here v_lif means the value named ’v lif’ in this block. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block. Context: the function ___init___, which handles init.

Line 116: `self.attn_lif = MultiStepHardResetLIF(threshold=0.5)` This assigns ‘MultiStepHardResetLIF(threshold=0.5)‘ to ‘self.attn_lif ‘ in host memory. Here attn_lif means the value named ’attn lif’ in this block. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function ___init___, which handles init.

Line 117: `self.proj_conv = nn.Conv1d(channels, channels, kernel_size=1)` This assigns ‘nn.Conv1d(channels, channels, kernel_size=1)‘ to ‘self.proj_conv ‘ in host memory. Here proj_conv means the value named ’proj conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv1d: the value named ’conv1d’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 118: `self.proj_bn = nn.BatchNorm1d(channels)` This assigns ‘nn.BatchNorm1d(channels)‘ to ‘self.proj_bn ‘ in host memory. Here proj_bn means the value named ’proj bn’ in this block. The important operands are nn: the value named ’nn’ in this block; BatchNorm1d: the value named ’batch norm1d’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 120: `def forward(` This starts function forward. Its indented body performs forward as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 121: `self,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 122: `value: torch.Tensor,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are value: the value named ’value’ in this block; torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 123: `*,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 124: `prefix: str,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are prefix: the value named 'prefix' in this block; str: the value named 'str' in this block.

Line 125: `observer: TensorObserver | None,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are observer: the value named 'observer' in this block; TensorObserver: the value named 'tensor observer' in this block; None: the value named 'none' in this block.

Line 126: `) -> torch.Tensor:` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 127: `time_steps, batch, channels, height, width = value.shape` This assigns 'value.shape' to 'time_steps, batch, channels, height, width ' in host memory. Here time_steps means the value named 'time steps' in this block. The important operands are value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function forward, which handles forward.

Line 128: `value = self.proj_lif(value)` This assigns 'self.proj_lif(value)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are proj_lif: the value named 'proj lif' in this block; value: the value named 'value' in this block. Context: the function forward, which handles forward.

Line 129: `_observe(observer, f"{prefix}.proj_lif", value, True)` This calls '_observe' with 'observer, f"{prefix}.proj_li value, True' inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 130: `value = value.flatten(3)` This assigns 'value.flatten(3)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are value: the value named 'value' in this block; flatten: the value named 'flatten' in this block. Context: the function forward, which handles forward.

Line 131: `tokens = value.shape[-1]` This assigns 'value.shape[-1]' to 'tokens ' in host memory. Here tokens means the value named 'tokens' in this block. The important operands are value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function forward, which handles forward.

Line 132: `projection_input = value.flatten(0, 1)` This assigns 'value.flatten(0, 1)' to 'projection_input ' in host memory. Here projection_input means the value named 'projection input' in this block. The important operands are value: the value named 'value' in this block; flatten: the value named 'flatten' in this block. Context: the function forward, which handles forward.

Line 134: `query = self.q_bn(self.q_conv(projection_input)).reshape(` This statement continues through source line 135; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.q_bn(self.q_conv(time_steps, batch, channels, tokens)' to 'query ' in host memory. Here query means the value named 'query' in this block. The important operands are q_bn: the value named 'q bn' in this block; q_conv: the value named 'q conv' in this block; projection_input: the value named 'projection input' in this block. Context: the function forward, which handles forward.

Line 135: `time_steps, batch, channels, tokens)` This is continuation line 2 of the 2-line logical statement that begins at source line 134. Its exact fragment is 'time_steps, batch, channels, tokens)'. The important operands are time_steps: the value named 'time steps' in this block; batch: the value named 'batch' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This assigns 'self.q_bn(self.q_conv(projection_input)).resh time_steps, batch, channels, tokens)' to 'query ' in host memory. Here query means the value named 'query' in this block. The important operands are q_bn: the value named 'q bn' in this block; q_conv: the value named 'q conv' in this block; projection_input: the value named 'projection input' in this block. Context: the function forward, which handles forward.

Line 136: `query = self.q_lif(query)` This assigns ‘self.q_lif(query)‘ to ‘query ‘ in host memory. Here query means the value named ’query’ in this block. The important operands are q_lif: the value named ’q lif’ in this block; query: the value named ’query’ in this block. Context: the function forward, which handles forward.

Line 137: `_observe(observer, f"{prefix}.q_lif", query, True)` This calls ‘_observe‘ with ‘observer, f"{prefix}.q_lif", query, True‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 138: `key = self.k_bn(self.k_conv(projection_input)).reshape(` This statement continues through source line 139; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.k_bn(self.k_conv(p time_steps, batch, channels, tokens)‘ to ‘key ‘ in host memory. Here key means the value named ’key’ in this block. The important operands are k_bn: the value named ’k bn’ in this block; k_conv: the value named ’k conv’ in this block; projection_input: the value named ’projection input’ in this block. Context: the function forward, which handles forward.

Line 139: `time_steps, batch, channels, tokens)` This is continuation line 2 of the 2-line logical statement that begins at source line 138. Its exact fragment is ‘time_steps, batch, channels, tokens)‘. The important operands are time_steps: the value named ’time steps’ in this block; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘self.k_bn(self.k_conv(projection_input)).resh time_steps, batch, channels, tokens)‘ to ‘key ‘ in host memory. Here key means the value named ’key’ in this block. The important operands are k_bn: the value named ’k bn’ in this block; k_conv: the value named ’k conv’ in this block; projection_input: the value named ’projection input’ in this block. Context: the function forward, which handles forward.

Line 140: `key = self.k_lif(key)` This assigns ‘self.k_lif(key)‘ to ‘key ‘ in host memory. Here key means the value named ’key’ in this block. The important operands are k_lif: the value named ’k lif’ in this block; key: the value named ’key’ in this block. Context: the function forward, which handles forward.

Line 141: `_observe(observer, f"{prefix}.k_lif", key, True)` This calls ‘_observe‘ with ‘observer, f"{prefix}.k_lif", key, True‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 142: `value_projection = self.v_bn(self.v_conv(projection_input)).reshape(` This statement continues through source line 143; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.v_bn(self.v_conv(projection_input)).reshape(time_steps, batch, channels, tokens)‘ to ‘value_projection ‘ in host memory. Here value_projection means the value named ’value projection’ in this block. The important operands are v_bn: the value named ’v bn’ in this block; v_conv: the value named ’v conv’ in this block; projection_input: the value named ’projection input’ in this block. Context: the function forward, which handles forward.

Line 143: `time_steps, batch, channels, tokens)` This is continuation line 2 of the 2-line logical statement that begins at source line 142. Its exact fragment is ‘time_steps, batch, channels, tokens)‘. The important operands are time_steps: the value named ’time steps’ in this block; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘self.v_bn(self.v_conv(projection_input)).resh time_steps, batch, channels, tokens)‘ to ‘value_projection ‘ in host memory. Here value_projection means the value named ’value projection’ in this block. The important operands are v_bn: the value named ’v bn’ in this block; v_conv: the value named ’v conv’ in this block; projection_input: the value named ’projection input’ in this block. Context: the function forward, which handles forward.

Line 144: `value_projection = self.v_lif(value_projection)` This assigns ‘self.v_lif(value_projection)‘ to ‘value_projectio ‘ in host memory. Here value_projection means the value named ’value projection’ in this block. The important operands are v_lif: the value named ’v lif’ in this block; value_projection: the value named ’value projection’ in this block. Context: the function forward, which handles forward.

Line 145: `_observe(observer, f"{prefix}.v_lif", value_projection, True)` This calls '_observe' with 'observer, f"{prefix}.v_lif", value_projection, True' inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 147: `query = _split_heads(query, self.heads)` This assigns '_split_heads(query, self.heads)' to 'query ' in host memory. Here query means the value named 'query' in this block. The important operands are _split_heads: the value named 'split heads' in this block; query: the value named 'query' in this block; heads: the value named 'heads' in this block. Context: the function forward, which handles forward.

Line 148: `key = _split_heads(key, self.heads)` This assigns '_split_heads(key, self.heads)' to 'key ' in host memory. Here key means the value named 'key' in this block. The important operands are _split_heads: the value named 'split heads' in this block; key: the value named 'key' in this block; heads: the value named 'heads' in this block. Context: the function forward, which handles forward.

Line 149: `value_projection = _split_heads(value_projection, self.heads)` This assigns '_split_heads(value_projection self.heads)' to 'value_projection ' in host memory. Here value_projection means the value named 'value projection' in this block. The important operands are _split_heads: the value named 'split heads' in this block; value_projection: the value named 'value projection' in this block; heads: the value named 'heads' in this block. Context: the function forward, which handles forward.

Line 150: `attention = query @ key.transpose(-2, -1)` This assigns 'query @ key.transpose(-2, -1)' to 'attention ' in host memory. Here attention means the value named 'attention' in this block. The important operands are query: the value named 'query' in this block; key: the value named 'key' in this block; transpose: the value named 'transpose' in this block. Context: the function forward, which handles forward.

Line 151: `_observe(observer, f"{prefix}.qk", attention, False)` This calls '_observe' with 'observer, f"{prefix}.qk", attention, False' inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 152: `value = (attention @ value_projection) * 0.125` This assigns '(attention @ value_projection) * 0.125' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are attention: the value named 'attention' in this block; value_projection: the value named 'value projection' in this block. Context: the function forward, which handles forward.

Line 153: `_observe(observer, f"{prefix}.attention_value", value, False)` This calls '_observe' with 'observer, f"{prefix}.attention_value", value, False' inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 154: `value = value.transpose(3, 4).reshape(` This statement continues through source line 155; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'value.transpose(3, 4).reshape(time_steps, batch, channels, tokens)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are value: the value named 'value' in this block; transpose: the value named 'transpose' in this block; reshape: the value named 'reshape' in this block. Context: the function forward, which handles forward.

Line 155: `time_steps, batch, channels, tokens)` This is continuation line 2 of the 2-line logical statement that begins at source line 154. Its exact fragment is 'time_steps, batch, channels, tokens)'. The important operands are time_steps: the value named 'time steps' in this block; batch: the value named 'batch' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This assigns 'value.transpose(3, 4).reshape(time_steps, batch, channels, tokens)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are value: the value named 'value' in this block; transpose: the value named 'transpose' in this block; reshape: the value named 'reshape' in this block. Context: the function forward, which handles forward.

Line 156: `value = self.attn_lif(value)` This assigns ‘self.attn_lif(value)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are attn_lif: the value named ’attn lif’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 157: `_observe(observer, f"{prefix}.attn_lif", value, True)` This calls ‘_observe‘ with ‘observer, f"{prefix}.attn_li value, True‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 158: `value = self.proj_bn(self.proj_conv(value.flatten(0, 1)))` This assigns ‘self.proj_bn(self.proj_conv(value.fl 1)))‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are proj_bn: the value named ’proj bn’ in this block; proj_conv: the value named ’proj conv’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 159: `value = value.reshape(time_steps, batch, channels, height, width)` This assigns ‘value.reshape(time_steps, batch, channels, height, width)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; reshape: the value named ’reshape’ in this block; time_steps: the value named ’time steps’ in this block. Context: the function forward, which handles forward.

Line 160: `_observe(observer, f"{prefix}.proj_bn", value, False)` This calls ‘_observe‘ with ‘observer, f"{prefix}.proj_b value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 161: `return value` This returns ‘value‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are value: the value named ’value’ in this block.

Line 164: `class SpikingTransformer(nn.Module):` This defines class SpikingTransformer, which groups data and methods for spiking transformer. Instances exist in host memory and organize the flow that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 165: `"""One residual attention block followed by one residual spiking MLP."""` This is executable documentation attached to the class SpikingTransformer, which handles spiking transformer. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 167: `def __init__(self, channels: int, heads: int, mlp_ratio: int) -> None:` This starts function ___init___. Its indented body performs init as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 168: `super().__init__()` This calls ‘super‘ with ‘).___init___(‘ inside the function ___init___, which handles init. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 169: `# The upstream model creates these LayerNorm parameters but does not` The comment records this intent: The upstream model creates these LayerNorm parameters but does not It applies inside the function ___init___, which handles init.

Line 170: `# apply them in forward. Keeping them is required for strict loading.` The comment records this intent: apply them in forward. Keeping them is required for strict loading. It applies inside the function ___init___, which handles init.

Line 171: `self.norm1 = nn.LayerNorm(channels, eps=1e-6)` This assigns ‘nn.LayerNorm(channels, eps=1e-6)‘ to ‘self.norm1 ‘ in host memory. Here norm1 means the value named ’norm1’ in this block. The important operands are nn: the value named ’nn’ in this block; LayerNorm: the value named ’layer norm’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 172: `self.attn = SpikingSelfAttention(channels, heads)` This assigns ‘SpikingSelfAttention(channels, heads)‘ to ‘self.attn ‘ in host memory. Here attn means the value named ’attn’ in this block. The important operands are SpikingSelfAttention: the value named ’spiking self attention’ in this block; channels: the value named ’channels’ in this block; heads: the value named ’heads’ in this block. Context: the function ___init___, which handles init.

Line 173: `self.norm2 = nn.LayerNorm(channels, eps=1e-6)` This assigns ‘nn.LayerNorm(channels, eps=1e-6)‘ to ‘self.norm2 ‘ in host memory. Here norm2 means the value named ’norm2’ in this block. The important operands are nn: the value named ’nn’ in this block; LayerNorm: the value named ’layer norm’ in this block; channels: the value named ’channels’ in this block. Context: the function ___init___, which handles init.

Line 174: `self.mlp = MLP(channels, channels * mlp_ratio)` This assigns ‘MLP(channels, channels * mlp_ratio)‘ to ‘self.mlp ‘ in host memory. Here mlp means the value named ’mlp’ in this block. The important operands are MLP: the value named ’mlp’ in this block; channels: the value named ’channels’ in this block; mlp_ratio: the value named ’mlp ratio’ in this block. Context: the function ___init___, which handles init.

Line 176: `def forward(` This starts function forward. Its indented body performs forward as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 177: `self,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 178: `value: torch.Tensor,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are value: the value named ’value’ in this block; torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 179: `*,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 180: `prefix: str,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are prefix: the value named ’prefix’ in this block; str: the value named ’str’ in this block.

Line 181: `observer: TensorObserver | None,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are observer: the value named ’observer’ in this block; TensorObserver: the value named ’tensor observer’ in this block; None: the value named ’none’ in this block.

Line 182: `) -> torch.Tensor:` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 183: `value = value + self.attn(` This statement continues through source line 184; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘value + self.attn(value, prefix=f"{prefix}.attn", observer=observer)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; attn: the value named ’attn’ in this block; prefix: the value named ’prefix’ in this block. Context: the function forward, which handles forward.

Line 184: `value, prefix=f"{prefix}.attn", observer=observer)` This is continuation line 2 of the 2-line logical statement that begins at source line 183. Its exact fragment is ‘value, prefix=f"{prefix}.attn", observer=observer)‘. The important operands are value: the value named ’value’ in this block; prefix: the value named ’prefix’ in this block; f: the value named ’f’ in this block. Read together, the complete operation is: This assigns ‘value + self.attn(value, prefix=f"{prefix}.attn", observer=observer)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; attn: the value named ’attn’ in this block; prefix: the value named ’prefix’ in this block. Context: the function forward, which handles forward.

Line 185: `_observe(observer, f"{prefix}.attention_residual", value, False)` This calls ‘_observe‘ with ‘observer, f"{prefix}.attention_residual", value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 186: `value = value + self.mlp(` This statement continues through source line 187; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘value + self.mlp(value, prefix=f"{prefix}.mlp", observer=observer)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; mlp: the value named ’mlp’ in this block; prefix: the value named ’prefix’ in this block. Context: the function forward, which handles forward.

Line 187: `value, prefix=f"{prefix}.mlp", observer=observer)` This is continuation line 2 of the 2-line logical statement that begins at source line 186. Its exact fragment is ‘value, prefix=f"{prefix}.mlp", observer=observer)‘. The important operands are value: the value named ’value’ in this block; prefix: the value named ’prefix’ in this block; f: the value named ’f’ in this block. Read together, the complete operation is: This assigns ‘value + self.mlp(value, prefix=f"{prefix}.mlp", observer=observer)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; mlp: the value named ’mlp’ in this block; prefix: the value named ’prefix’ in this block. Context: the function forward, which handles forward.

Line 188: `_observe(observer, f"{prefix}.mlp_residual", value, False)` This calls ‘_observe‘ with ‘observer, f"{prefix}.mlp_residual", value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 189: `return value` This returns ‘value‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are value: the value named ’value’ in this block.

Line 192: `class SpikingTokenizer(nn.Module):` This defines class SpikingTokenizer, which groups data and methods for spiking tokenizer. Instances exist in host memory and organize the flow that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 193: `"""Five 3x3 convolutions that convert 32x32 RGB frames to 8x8 tokens."""` This is executable documentation attached to the class SpikingTokenizer, which handles spiking tokenizer. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 195: `def __init__(self, input_channels: int, embed_dim: int) -> None:` This starts function ___init___. Its indented body performs init as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 196: `super().__init__()` This calls ‘super‘ with ‘).___init___(‘ inside the function ___init___, which handles init. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 197: `widths = (embed_dim // 8, embed_dim // 4, embed_dim // 2, embed_dim)` This assigns ‘(embed_dim // 8, embed_dim // 4, embed_dim // 2, embed_dim)‘ to ‘widths ‘ in host memory. Here widths means the value named ’widths’ in this block. The important operands are embed_dim: the value named ’embed dim’ in this block. Context: the function ___init___, which handles init.

Line 198: `self.block0_conv = nn.Conv2d(` This statement continues through source line 199; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘nn.Conv2d(input_channels, widths[0], kernel_size=3, padding=1, bias=False)‘ to ‘self.block0_conv ‘ in host memory. Here block0_conv means the value named ’block0 conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv2d: the value named ’conv2d’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function ___init___, which handles init.

Line 199: `input_channels, widths[0], kernel_size=3, padding=1, bias=False)` This is continuation line 2 of the 2-line logical statement that begins at source line 198. Its exact fragment is ‘input_channels, widths[0], kernel_size=3, padding=1, bias=False)‘. The important operands are input_channels: the value named ’input channels’ in this block; widths: the value named ’widths’ in this block; kernel_size: the value named ’kernel size’ in this block. Read together, the complete operation is: This assigns ‘nn.Conv2d(input_channels, widths[0], kernel_size=3, padding=1, bias=False)‘ to ‘self.block0_conv ‘ in host memory. Here block0_conv means the value named ’block0 conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv2d: the value named ’conv2d’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function ___init___, which handles init.

Line 200: `self.block0_bn = nn.BatchNorm2d(widths[0])` This assigns ‘nn.BatchNorm2d(widths[0])‘ to ‘self.block0_bn ‘ in host memory. Here block0_bn means the value named ’block0 bn’ in this block. The important operands are nn: the value named ’nn’ in this block; BatchNorm2d: the value named ’batch norm2d’ in this block; widths: the value named ’widths’ in this block. Context: the function ___init___, which handles init.

Line 201: `self.block1_lif = MultiStepHardResetLIF()` This assigns ‘MultiStepHardResetLIF()‘ to ‘self.block1_lif ‘ in host memory. Here block1_lif means the value named ’block1 lif’ in this block. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block. Context: the function ___init___, which handles init.

Line 202: `self.block1_conv = nn.Conv2d(` This statement continues through source line 203; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘nn.Conv2d(widths[0], widths[1], kernel_size=3, padding=1, bias=False)‘ to ‘self.block1_conv ‘ in host memory. Here block1_conv means the value named ’block1 conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv2d: the value named ’conv2d’ in this block; widths: the value named ’widths’ in this block. Context: the function ___init___, which handles init.

Line 203: `widths[0], widths[1], kernel_size=3, padding=1, bias=False)` This is continuation line 2 of the 2-line logical statement that begins at source line 202. Its exact fragment is ‘widths[0], widths[1], kernel_size=3, padding=1, bias=False)‘. The important operands are widths: the value named ’widths’ in this block; kernel_size: the value named ’kernel size’ in this block; padding: the value named ’padding’ in this block. Read together, the complete operation is: This assigns ‘nn.Conv2d(widths[0], widths[1], kernel_size=3, padding=1, bias=False)‘ to ‘self.block1_conv ‘ in host memory. Here block1_conv means the value named ’block1 conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv2d: the value named ’conv2d’ in this block; widths: the value named ’widths’ in this block. Context: the function ___init___, which handles init.

Line 204: `self.block1_bn = nn.BatchNorm2d(widths[1])` This assigns ‘nn.BatchNorm2d(widths[1])‘ to ‘self.block1_bn ‘ in host memory. Here block1_bn means the value named ’block1 bn’ in this block. The important operands are nn: the value named ’nn’ in this block; BatchNorm2d: the value named ’batch norm2d’ in this block; widths: the value named ’widths’ in this block. Context: the function ___init___, which handles init.

Line 205: `self.block2_lif = MultiStepHardResetLIF()` This assigns ‘MultiStepHardResetLIF()‘ to ‘self.block2_lif ‘ in host memory. Here block2_lif means the value named ’block2 lif’ in this block. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block. Context: the function ___init___, which handles init.

Line 206: `self.block2_conv = nn.Conv2d(` This statement continues through source line 207; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘nn.Conv2d(widths[1], widths[2], kernel_size=3, padding=1, bias=False)‘ to ‘self.block2_conv ‘ in host memory. Here block2_conv means the value named

'block2 conv' in this block. The important operands are nn: the value named 'nn' in this block; Conv2d: the value named 'conv2d' in this block; widths: the value named 'widths' in this block. Context: the function ___init___, which handles init.

Line 207: `widths[1], widths[2], kernel_size=3, padding=1, bias=False)` This is continuation line 2 of the 2-line logical statement that begins at source line 206. Its exact fragment is 'widths[1], widths[2], kernel_size=3, padding=1, bias=False)'. The important operands are widths: the value named 'widths' in this block; kernel_size: the value named 'kernel size' in this block; padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'nn.Conv2d(widths[1], widths[2], kernel_size=3, padding=1, bias=False)' to 'self.block2_conv ' in host memory. Here block2_conv means the value named 'block2 conv' in this block. The important operands are nn: the value named 'nn' in this block; Conv2d: the value named 'conv2d' in this block; widths: the value named 'widths' in this block. Context: the function ___init___, which handles init.

Line 208: `self.block2_bn = nn.BatchNorm2d(widths[2])` This assigns 'nn.BatchNorm2d(widths[2])' to 'self.block2_bn ' in host memory. Here block2_bn means the value named 'block2 bn' in this block. The important operands are nn: the value named 'nn' in this block; BatchNorm2d: the value named 'batch norm2d' in this block; widths: the value named 'widths' in this block. Context: the function ___init___, which handles init.

Line 209: `self.block3_lif = MultiStepHardResetLIF()` This assigns 'MultiStepHardResetLIF()' to 'self.block3_lif ' in host memory. Here block3_lif means the value named 'block3 lif' in this block. The important operands are MultiStepHardResetLIF: the value named 'multi step hard reset lif' in this block. Context: the function ___init___, which handles init.

Line 210: `self.block3_mp = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)` This assigns 'nn.MaxPool2d(kernel_siz stride=2, padding=1)' to 'self.block3_mp ' in host memory. Here block3_mp means the value named 'block3 mp' in this block. The important operands are nn: the value named 'nn' in this block; MaxPool2d: the value named 'max pool2d' in this block; kernel_size: the value named 'kernel size' in this block. Context: the function ___init___, which handles init.

Line 211: `self.block3_conv = nn.Conv2d(` This statement continues through source line 212; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'nn.Conv2d(widths[2], widths[3], kernel_size=3, padding=1, bias=False)' to 'self.block3_conv ' in host memory. Here block3_conv means the value named 'block3 conv' in this block. The important operands are nn: the value named 'nn' in this block; Conv2d: the value named 'conv2d' in this block; widths: the value named 'widths' in this block. Context: the function ___init___, which handles init.

Line 212: `widths[2], widths[3], kernel_size=3, padding=1, bias=False)` This is continuation line 2 of the 2-line logical statement that begins at source line 211. Its exact fragment is 'widths[2], widths[3], kernel_size=3, padding=1, bias=False)'. The important operands are widths: the value named 'widths' in this block; kernel_size: the value named 'kernel size' in this block; padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'nn.Conv2d(widths[2], widths[3], kernel_size=3, padding=1, bias=False)' to 'self.block3_conv ' in host memory. Here block3_conv means the value named 'block3 conv' in this block. The important operands are nn: the value named 'nn' in this block; Conv2d: the value named 'conv2d' in this block; widths: the value named 'widths' in this block. Context: the function ___init___, which handles init.

Line 213: `self.block3_bn = nn.BatchNorm2d(widths[3])` This assigns 'nn.BatchNorm2d(widths[3])' to 'self.block3_bn ' in host memory. Here block3_bn means the value named 'block3 bn' in this block. The important operands are nn: the value named 'nn' in this block; BatchNorm2d: the value named 'batch norm2d' in this block; widths: the value named 'widths' in this block. Context: the function ___init___, which handles init.

Line 214: `self.block4_lif = MultiStepHardResetLIF()` This assigns 'MultiStepHardResetLIF()' to 'self.block4_lif ' in host memory. Here block4_lif means the value named 'block4 lif' in this block. The important operands are MultiStepHardResetLIF: the value named 'multi step hard reset lif' in this block. Context: the function ___init___, which handles init.

Line 215: `self.block4_mp = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)` This assigns ‘nn.MaxPool2d(kernel_siz stride=2, padding=1)‘ to ‘self.block4_mp ‘ in host memory. Here block4_mp means the value named ’block4 mp’ in this block. The important operands are nn: the value named ’nn’ in this block; MaxPool2d: the value named ’max pool2d’ in this block; kernel_size: the value named ’kernel size’ in this block. Context: the function ___init___, which handles init.

Line 216: `self.block4_conv = nn.Conv2d(` This statement continues through source line 217; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘nn.Conv2d(widths[3], widths[3], kernel_size=3, padding=1, bias=False)‘ to ‘self.block4_conv ‘ in host memory. Here block4_conv means the value named ’block4 conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv2d: the value named ’conv2d’ in this block; widths: the value named ’widths’ in this block. Context: the function ___init___, which handles init.

Line 217: `widths[3], widths[3], kernel_size=3, padding=1, bias=False)` This is continuation line 2 of the 2-line logical statement that begins at source line 216. Its exact fragment is ‘widths[3], widths[3], kernel_size=3, padding=1, bias=False)‘. The important operands are widths: the value named ’widths’ in this block; kernel_size: the value named ’kernel size’ in this block; padding: the value named ’padding’ in this block. Read together, the complete operation is: This assigns ‘nn.Conv2d(widths[3], widths[3], kernel_size=3, padding=1, bias=False)‘ to ‘self.block4_conv ‘ in host memory. Here block4_conv means the value named ’block4 conv’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv2d: the value named ’conv2d’ in this block; widths: the value named ’widths’ in this block. Context: the function ___init___, which handles init.

Line 218: `self.block4_bn = nn.BatchNorm2d(widths[3])` This assigns ‘nn.BatchNorm2d(widths[3])‘ to ‘self.block4_bn ‘ in host memory. Here block4_bn means the value named ’block4 bn’ in this block. The important operands are nn: the value named ’nn’ in this block; BatchNorm2d: the value named ’batch norm2d’ in this block; widths: the value named ’widths’ in this block. Context: the function ___init___, which handles init.

Line 220: `def forward(` This starts function forward. Its indented body performs forward as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 221: `self,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 222: `value: torch.Tensor,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are value: the value named ’value’ in this block; torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 223: `*,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 224: `observer: TensorObserver | None,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are observer: the value named ’observer’ in this block; TensorObserver: the value named ’tensor observer’ in this block; None: the value named ’none’ in this block.

Line 225: `) -> torch.Tensor:` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 226: `time_steps, batch, _, height, width = value.shape` This assigns ‘value.shape‘ to ‘time_steps, batch, _, height, width ‘ in host memory. Here time_steps means the value named ’time steps’ in this block. The important operands are value: the value named ’value’ in this block; shape: the value named ’shape’ in this block. Context: the function forward, which handles forward.

Line 227: `value = self.block0_bn(self.block0_conv(value.flatten(0, 1)))` This assigns ‘self.block0_bn(self.block0_con 1)))‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block0_bn: the value named ’block0 bn’ in this block; block0_conv: the value named ’block0 conv’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 228: `value = value.reshape(time_steps, batch, -1, height, width)` This assigns ‘value.reshape(time_steps, batch, -1, height, width)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; reshape: the value named ’reshape’ in this block; time_steps: the value named ’time steps’ in this block. Context: the function forward, which handles forward.

Line 229: `_observe(observer, "patch_embed.block0_bn", value, False)` This calls ‘_observe‘ with ‘observer, "patch_embed.block0_bn", value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 231: `value = self.block1_lif(value)` This assigns ‘self.block1_lif(value)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block1_lif: the value named ’block1 lif’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 232: `_observe(observer, "patch_embed.block1_lif", value, True)` This calls ‘_observe‘ with ‘observer, "patch_embed.block1_lif", value, True‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 233: `value = self.block1_bn(self.block1_conv(value.flatten(0, 1)))` This assigns ‘self.block1_bn(self.block1_con 1)))‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block1_bn: the value named ’block1 bn’ in this block; block1_conv: the value named ’block1 conv’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 234: `value = value.reshape(time_steps, batch, -1, height, width)` This assigns ‘value.reshape(time_steps, batch, -1, height, width)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; reshape: the value named ’reshape’ in this block; time_steps: the value named ’time steps’ in this block. Context: the function forward, which handles forward.

Line 235: `_observe(observer, "patch_embed.block1_bn", value, False)` This calls ‘_observe‘ with ‘observer, "patch_embed.block1_bn", value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 237: `value = self.block2_lif(value)` This assigns ‘self.block2_lif(value)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block2_lif: the value named ’block2 lif’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 238: `_observe(observer, "patch_embed.block2_lif", value, True)` This calls ‘_observe‘ with ‘observer, "patch_embed.block2_lif", value, True‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 239: `value = self.block2_bn(self.block2_conv(value.flatten(0, 1)))` This assigns ‘self.block2_bn(self.block2_con 1)))‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block2_bn: the value named ’block2 bn’ in this block; block2_conv: the value named ’block2 conv’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 240: `value = value.reshape(time_steps, batch, -1, height, width)` This assigns ‘value.reshape(time_steps, batch, -1, height, width)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; reshape: the value named ’reshape’ in this block; time_steps: the value named ’time steps’ in this block. Context: the function forward, which handles forward.

Line 241: `_observe(observer, "patch_embed.block2_bn", value, False)` This calls ‘_observe‘ with ‘observer, "patch_embed.block2_bn", value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 243: `value = self.block3_lif(value).flatten(0, 1)` This assigns ‘self.block3_lif(value).flatten(0, 1)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block3_lif: the value named ’block3 lif’ in this block; value: the value named ’value’ in this block; flatten: the value named ’flatten’ in this block. Context: the function forward, which handles forward.

Line 244: `_observe(observer, "patch_embed.block3_lif", value, True)` This calls ‘_observe‘ with ‘observer, "patch_embed.block3_lif", value, True‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 245: `value = self.block3_mp(value)` This assigns ‘self.block3_mp(value)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block3_mp: the value named ’block3 mp’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 246: `value = self.block3_bn(self.block3_conv(value))` This assigns ‘self.block3_bn(self.block3_conv(value))‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block3_bn: the value named ’block3 bn’ in this block; block3_conv: the value named ’block3 conv’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 247: `value = value.reshape(time_steps, batch, -1, height // 2, width // 2)` This assigns ‘value.reshape(time_s batch, -1, height // 2, width // 2)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; reshape: the value named ’reshape’ in this block; time_steps: the value named ’time steps’ in this block. Context: the function forward, which handles forward.

Line 248: `_observe(observer, "patch_embed.block3_bn", value, False)` This calls ‘_observe‘ with ‘observer, "patch_embed.block3_bn", value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 250: `value = self.block4_lif(value).flatten(0, 1)` This assigns ‘self.block4_lif(value).flatten(0, 1)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block4_lif: the value named ’block4 lif’ in this block; value: the value named ’value’ in this block; flatten: the value named ’flatten’ in this block. Context: the function forward, which handles forward.

Line 251: `_observe(observer, "patch_embed.block4_lif", value, True)` This calls ‘_observe‘ with ‘observer, "patch_embed.block4_lif", value, True‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 252: `value = self.block4_mp(value)` This assigns ‘self.block4_mp(value)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block4_mp: the value named ’block4 mp’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 253: `value = self.block4_bn(self.block4_conv(value))` This assigns ‘self.block4_bn(self.block4_conv(value))‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are block4_bn: the value named ’block4 bn’ in this block; block4_conv: the value named ’block4 conv’ in this block; value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 254: `value = value.reshape(time_steps, batch, -1, height // 4, width // 4)` This assigns ‘value.reshape(time_s batch, -1, height // 4, width // 4)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; reshape: the value named ’reshape’ in this block; time_steps: the value named ’time steps’ in this block. Context: the function forward, which handles forward.

Line 255: `_observe(observer, "patch_embed.block4_bn", value, False)` This calls ‘_observe‘ with ‘observer, "patch_embed.block4_bn", value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 256: `return value` This returns ‘value‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are value: the value named ’value’ in this block.

Line 259: `class Spikingformer(nn.Module):` This defines class Spikingformer, which groups data and methods for spikingformer. Instances exist in host memory and organize the flow that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 260: `"""The exact Spikingformer-4-384 graph used by the CIFAR-10 run."""` This is executable documentation attached to the class Spikingformer, which handles spikingformer. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 262: `def __init__(` This starts function ___init___. Its indented body performs init as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 263: `self,` Inside the function ___init___, which handles init, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 264: `*,` Inside the function ___init___, which handles init, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 265: `time_steps: int = 4,` This assigns ‘4,‘ to ‘time_steps‘ in host memory. Here time_steps means the value named ’time steps’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 266: `input_channels: int = 3,` This assigns ‘3,‘ to ‘input_channels‘ in host memory. Here input_channels means the value named ’input channels’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 267: `classes: int = 10,` This assigns ‘10,‘ to ‘classes‘ in host memory. Here classes means the value named ’classes’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 268: `embed_dim: int = 384,` This assigns ‘384,‘ to ‘embed_dim‘ in host memory. Here embed_dim means the value named ’embed dim’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 269: `heads: int = 12,` This assigns ‘12,‘ to ‘heads‘ in host memory. Here heads means the value named ’heads’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 270: `depth: int = 4,` This assigns '4,' to 'depth' in host memory. Here depth means the value named 'depth' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 271: `mlp_ratio: int = 4,` This assigns '4,' to 'mlp_ratio' in host memory. Here mlp_ratio means the value named 'mlp ratio' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 272: `) -> None:` Inside the function ___init___, which handles init, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are None: the value named 'none' in this block.

Line 273: `super().__init__()` This calls 'super' with ').___init___(' inside the function ___init___, which handles init. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 274: `self.time_steps = time_steps` This assigns 'time_steps' to 'self.time_steps ' in host memory. Here time_steps means the value named 'time steps' in this block. The important operands are time_steps: the value named 'time steps' in this block. Context: the function ___init___, which handles init.

Line 275: `self.patch_embed = SpikingTokenizer(input_channels, embed_dim)` This assigns 'SpikingTokenizer(input_chan
embed_dim)' to 'self.patch_embed ' in host memory. Here patch_embed means the value named 'patch embed' in this block. The important operands are SpikingTokenizer: the value named 'spiking tokenizer' in this block; input_channels: the value named 'input channels' in this block; embed_dim: the value named 'embed dim' in this block. Context: the function ___init___, which handles init.

Line 276: `self.block = nn.ModuleList([` This statement continues through source line 279; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'nn.ModuleList([SpikingTransformer(embed_dim, heads, mlp_ratio) for _ in range(depth)])' to 'self.block ' in host memory. Here block means the value named 'block' in this block. The important operands are nn: the value named 'nn' in this block; ModuleList: the value named 'module list' in this block; SpikingTransformer: the value named 'spiking transformer' in this block. Context: the function ___init___, which handles init.

Line 277: `SpikingTransformer(embed_dim, heads, mlp_ratio)` This is continuation line 2 of the 4-line logical statement that begins at source line 276. Its exact fragment is 'SpikingTransformer(embed_dim, heads, mlp_ratio)'. The important operands are SpikingTransformer: the value named 'spiking transformer' in this block; embed_dim: the value named 'embed dim' in this block; heads: the value named 'heads' in this block. Read together, the complete operation is: This assigns 'nn.ModuleList([SpikingTransformer(embed_dim, heads, mlp_ratio) for _ in range(depth)])' to 'self.block ' in host memory. Here block means the value named 'block' in this block. The important operands are nn: the value named 'nn' in this block; ModuleList: the value named 'module list' in this block; SpikingTransformer: the value named 'spiking transformer' in this block. Context: the function ___init___, which handles init.

Line 278: `for _ in range(depth)` This is continuation line 3 of the 4-line logical statement that begins at source line 276. Its exact fragment is 'for _ in range(depth)'. The important operands are for: the value named 'for' in this block; _: the value named '' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'nn.ModuleList([SpikingTransformer(embed_dim, heads, mlp_ratio) for _ in range(depth)])' to 'self.block ' in host memory. Here block means the value named 'block' in this block. The important operands are nn: the value named 'nn' in this block; ModuleList: the value named 'module list' in this block; SpikingTransformer: the value named 'spiking transformer' in this block. Context: the function ___init___, which handles init.

Line 279: `])` This is continuation line 4 of the 4-line logical statement that begins at source line 276. Its exact fragment is '])'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'nn.ModuleList([SpikingTransformer(embed_dim, heads, mlp_ratio) for _ in range(depth)])' to 'self.block ' in host memory. Here block means the value named 'block' in this block. The important operands are

nn: the value named 'nn' in this block; ModuleList: the value named 'module list' in this block; SpikingTransformer: the value named 'spiking transformer' in this block. Context: the function ___init___, which handles init.

Line 280: `self.head = nn.Linear(embed_dim, classes)` This assigns 'nn.Linear(embed_dim, classes)' to 'self.head ' in host memory. Here head means the value named 'head' in this block. The important operands are nn: the value named 'nn' in this block; Linear: the value named 'linear' in this block; embed_dim: the value named 'embed dim' in this block. Context: the function ___init___, which handles init.

Line 282: `def forward(` This starts function forward. Its indented body performs forward as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 283: `self,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 284: `image: torch.Tensor,` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are image: the value named 'image' in this block; torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 285: `observer: TensorObserver | None = None,` This assigns 'None,' to 'observer' in host memory. Here observer means the value named 'observer' in this block. The important operands are None: the value named 'none' in this block. Context: the function forward, which handles forward.

Line 286: `) -> torch.Tensor:` Inside the function forward, which handles forward, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 287: `value = image.unsqueeze(0).repeat(self.time_steps, 1, 1, 1, 1)` This assigns 'image.unsqueeze(0).repeat(se 1, 1, 1, 1)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are image: the value named 'image' in this block; unsqueeze: the value named 'unsqueeze' in this block; repeat: the value named 'repeat' in this block. Context: the function forward, which handles forward.

Line 288: `_observe(observer, "input_repeated", value, False)` This calls '_observe' with 'observer, "input_repeated", value, False' inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 289: `value = self.patch_embed(value, observer=observer)` This assigns 'self.patch_embed(value, observer=observer) to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are patch_embed: the value named 'patch embed' in this block; value: the value named 'value' in this block; observer: the value named 'observer' in this block. Context: the function forward, which handles forward.

Line 290: `for index, transformer in enumerate(self.block):` This iterates index, transformer over 'enumerate(self.block)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function forward, which handles forward.

Line 291: `value = transformer(` This statement continues through source line 292; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'transformer(value, prefix=f"block.{index}", observer=observer)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are transformer: the value named 'transformer' in this block; value: the value named 'value' in this block; prefix: the value named 'prefix' in this block. Context: the function forward, which handles forward.

Line 292: `value, prefix=f"block.{index}", observer=observer)` This is continuation line 2 of the 2-line logical statement that begins at source line 291. Its exact fragment is ‘value, prefix=f"block.{index}", observer=observer)‘. The important operands are value: the value named ’value’ in this block; prefix: the value named ’prefix’ in this block; f: the value named ’f’ in this block. Read together, the complete operation is: This assigns ‘transformer(value, prefix=f"block.{index}", observer=observer)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are transformer: the value named ’transformer’ in this block; value: the value named ’value’ in this block; prefix: the value named ’prefix’ in this block. Context: the function forward, which handles forward.

Line 293: `value = value.flatten(3).mean(3)` This assigns ‘value.flatten(3).mean(3)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; flatten: the value named ’flatten’ in this block; mean: the arithmetic average used by normalization. Context: the function forward, which handles forward.

Line 294: `_observe(observer, "global_average_pool", value, False)` This calls ‘_observe‘ with ‘observer, "global_averag value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 295: `value = self.head(value.mean(0))` This assigns ‘self.head(value.mean(0))‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are head: the value named ’head’ in this block; value: the value named ’value’ in this block; mean: the arithmetic average used by normalization. Context: the function forward, which handles forward.

Line 296: `_observe(observer, "head", value, False)` This calls ‘_observe‘ with ‘observer, "head", value, False‘ inside the function forward, which handles forward. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 297: `return value` This returns ‘value‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are value: the value named ’value’ in this block.

Line 300: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it creates no FPGA circuit by itself.

Line 301: `class LoadedCheckpoint:` This defines class LoadedCheckpoint, which groups data and methods for loaded checkpoint. Instances exist in host memory and organize the flow that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 302: `"""Strictly loaded model plus provenance stored by the training run."""` This is executable documentation attached to the class LoadedCheckpoint, which handles loaded checkpoint. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 304: `model: Spikingformer` Inside the class LoadedCheckpoint, which handles loaded checkpoint, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are model: the value named ’model’ in this block; Spikingformer: the value named ’spikingformer’ in this block.

Line 305: `epoch: int` Inside the class LoadedCheckpoint, which handles loaded checkpoint, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are epoch: the value named ’epoch’ in this block.

Line 306: `metric: float` Inside the class LoadedCheckpoint, which handles loaded checkpoint, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are metric: the value named ’metric’ in this block; float: the value named ’float’ in this block.

Line 307: `architecture: str` Inside the class LoadedCheckpoint, which handles loaded checkpoint, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are architecture: the value named 'architecture' in this block; str: the value named 'str' in this block.

Line 308: `checkpoint_path: Path` Inside the class LoadedCheckpoint, which handles loaded checkpoint, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are checkpoint_path: the value named 'checkpoint path' in this block; Path: the value named 'path' in this block.

Line 311: `def load_checkpoint(` This starts function load_checkpoint. Its indented body performs load checkpoint as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 312: `checkpoint_path: str | Path,` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are checkpoint_path: the value named 'checkpoint path' in this block; str: the value named 'str' in this block; Path: the value named 'path' in this block.

Line 313: `*,` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The expression is a literal or structural operator and has no hidden software state.

Line 314: `device: str | torch.device = "cpu",` This assigns ‘"cpu",‘ to ‘device‘ in host memory. Here device means the value named 'device' in this block. The important operands are cpu: the value named 'cpu' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 315: `) -> LoadedCheckpoint:` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are LoadedCheckpoint: the value named 'loaded checkpoint' in this block.

Line 316: `"""Load the trained checkpoint and reject every state-dictionary mismatch."""` This is executable documentation attached to the function load_checkpoint, which handles load checkpoint. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 318: `path = Path(checkpoint_path).expanduser().resolve()` This assigns ‘Path(checkpoint_path).expanduser().resol
to ‘path ‘ in host memory. Here path means the value named 'path' in this block. The important operands are Path: the value named 'path' in this block; checkpoint_path: the value named 'checkpoint path' in this block; expanduser: the value named 'expanduser' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 319: `checkpoint: Mapping[str, Any] = torch.load(path, map_location="cpu")` This assigns ‘torch.load(path, map_location="cpu")‘ to ‘checkpoint‘ in host memory. Here checkpoint means the value named 'checkpoint' in this block. The important operands are torch: the value named 'torch' in this block; load: the value named 'load' in this block; path: the value named 'path' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 320: `arguments = checkpoint.get("args")` This assigns ‘checkpoint.get("args")‘ to ‘arguments ‘ in host memory. Here arguments means the value named 'arguments' in this block. The important operands are checkpoint: the value named 'checkpoint' in this block; get: the value named 'get' in this block; args: the value named 'args' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 321: `if arguments is None:` This executes the following indented host-code branch only when ‘arguments is None‘ is true. The important operands are arguments: the value named 'arguments' in this block; is: the value named 'is' in this block; None: the value named 'none' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 322: `raise ValueError("checkpoint does not contain its training arguments")` This rejects an invalid state by raising ‘ValueError("checkpoint does not contain its training arguments")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 323: `model = Spikingformer(` This statement continues through source line 331; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘Spikingformer(time_steps=int(getattr(arguments, "time_step")), input_channels=3, classes=int(getattr(arguments, "num_classes")), embed_dim=int(getattr(arguments, "dim")), heads=int(getattr(arguments, "num_heads")), depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))), mlp_ratio=int(getattr(arguments, "mlp_ratio")),)‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; getattr: the value named ’getattr’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 324: `time_steps=int(getattr(arguments, "time_step")),` This is continuation line 2 of the 9-line logical statement that begins at source line 323. Its exact fragment is ‘time_steps=int(getattr(arguments, "time_step")),‘. The important operands are time_steps: the value named ’time steps’ in this block; getattr: the value named ’getattr’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=int(getattr(arguments, "time_step")), input_channels=3, classes=int(getattr(arguments, "num_classes")), embed_dim=int(getattr(arguments, "dim")), heads=int(getattr(arguments, "num_heads")), depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))), mlp_ratio=int(getattr(arguments, "mlp_ratio")),)‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; getattr: the value named ’getattr’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 325: `input_channels=3,` This is continuation line 3 of the 9-line logical statement that begins at source line 323. Its exact fragment is ‘input_channels=3,‘. The important operands are input_channels: the value named ’input channels’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=int(getattr(arguments, "time_step")), input_channels=3, classes=int(getattr(arguments, "num_classes")), embed_dim=int(getattr(arguments, "dim")), heads=int(getattr(arguments, "num_heads")), depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))), mlp_ratio=int(getattr(arguments, "mlp_ratio")),)‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; getattr: the value named ’getattr’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 326: `classes=int(getattr(arguments, "num_classes")),` This is continuation line 4 of the 9-line logical statement that begins at source line 323. Its exact fragment is ‘classes=int(getattr(arguments, "num_classes")),‘. The important operands are classes: the value named ’classes’ in this block; getattr: the value named ’getattr’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=int(getattr(arguments, "time_step")), input_channels=3, classes=int(getattr(arguments, "num_classes")), embed_dim=int(getattr(arguments, "dim")), heads=int(getattr(arguments, "num_heads")), depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))), mlp_ratio=int(getattr(arguments, "mlp_ratio")),)‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; getattr: the value named ’getattr’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 327: `embed_dim=int(getattr(arguments, "dim")),` This is continuation line 5 of the 9-line logical statement that begins at source line 323. Its exact fragment is ‘embed_dim=int(getattr(arguments, "dim")),‘. The important operands are embed_dim: the value named ’embed dim’ in this block; getattr: the value named ’getattr’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=int(getattr(arguments, "time_step")), input_channels=3, classes=int(getattr(arguments, "num_classes")), embed_dim=int(getattr(arguments, "dim")), heads=int(getattr(arguments, "num_heads")), depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))), mlp_ratio=int(getattr(arguments, "mlp_ratio")),)‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; getattr: the value named ’getattr’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 328: `heads=int(getattr(arguments, "num_heads")),` This is continuation line 6 of the 9-line logical statement that begins at source line 323. Its exact fragment is 'heads=int(getattr(arguments, "num_heads")),'. The important operands are heads: the value named 'heads' in this block; getattr: the value named 'getattr' in this block; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=int(getattr(arguments, "time_step")), input_channels=3, classes=int(getattr(arguments, "num_classes")), embed_dim=int(getattr(arguments, "dim")), heads=int(getattr(arguments, "num_heads")), depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))), mlp_ratio=int(getattr(arguments, "mlp_ratio")),)' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; getattr: the value named 'getattr' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 329: `depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))),` This is continuation line 7 of the 9-line logical statement that begins at source line 323. Its exact fragment is 'depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))),'. The important operands are depth: the value named 'depth' in this block; getattr: the value named 'getattr' in this block; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=int(getattr(arguments, "time_step")), input_channels=3, classes=int(getattr(arguments, "num_classes")), embed_dim=int(getattr(arguments, "dim")), heads=int(getattr(
"num_heads")), depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))), mlp_ratio=int(getattr(arguments, "mlp_ratio")),)' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; getattr: the value named 'getattr' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 330: `mlp_ratio=int(getattr(arguments, "mlp_ratio")),` This is continuation line 8 of the 9-line logical statement that begins at source line 323. Its exact fragment is 'mlp_ratio=int(getattr(arguments, "mlp_ratio")),'. The important operands are mlp_ratio: the value named 'mlp ratio' in this block; getattr: the value named 'getattr' in this block; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=int(getattr(arguments, "time_step")), input_channels=3, classes=int(getattr(arguments, "num_classes")), embed_dim=int(getattr(arguments, "dim")), heads=int(getattr(arguments, "num_heads")), depth=int(getattr(ar
"depths", getattr(arguments, "layer"))), mlp_ratio=int(getattr(arguments, "mlp_ratio")),)' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; getattr: the value named 'getattr' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 331: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 323. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=int(getattr(arguments, "time_step")), input_channels=3, classes=int(getattr(arguments, "num_classes")), embed_dim=int(getattr(arguments, "dim")), heads=int(getattr(arguments, "num_heads")), depth=int(getattr(arguments, "depths", getattr(arguments, "layer"))), mlp_ratio=int(getattr(arguments, "mlp_ratio")),)' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; getattr: the value named 'getattr' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 332: `state = checkpoint.get("state_dict")` This assigns 'checkpoint.get("state_dict")' to 'state ' in host memory. Here state means the finite-state-machine phase retained in flip-flops. The important operands are checkpoint: the value named 'checkpoint' in this block; get: the value named 'get' in this block; state_dict: the finite-state-machine phase retained in flip-flops. Context: the function load_checkpoint, which handles load checkpoint.

Line 333: `if not isinstance(state, Mapping):` This executes the following indented host-code branch only when 'not isinstance(state, Mapping)' is true. The important operands are not: the value named 'not' in this block; isinstance: the value named 'isinstance' in this block; state: the finite-state-machine phase retained in flip-flops. Context: the function load_checkpoint, which handles load checkpoint.

Line 334: `raise ValueError("checkpoint does not contain a state_dict mapping")` This rejects an invalid state by raising 'ValueError("checkpoint does not contain a state_dict mapping")'. Failing before DMA prevents mal-

formed shapes or artifacts from silently becoming hardware results.

Line 335: `model.load_state_dict(state, strict=True)` This calls ‘model.load_state_dict‘ with ‘state, strict=True‘ inside the function load_checkpoint, which handles load checkpoint. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 336: `model.eval().to(device)` This calls ‘model.eval‘ with ‘).to(device‘ inside the function load_checkpoint, which handles load checkpoint. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

Line 337: `return LoadedCheckpoint(` This statement continues through source line 343; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘LoadedCheckpoint(model=model, epoch=int(checkpoint.get("epoch", -1)), metric=float(checkpoint.get("metric", float("nan"))), architecture=str(checkpoint.get("arch" "unknown")), checkpoint_path=path,)‘ to the caller of the function load_checkpoint, which handles load checkpoint, ending this invocation. The important operands are LoadedCheckpoint: the value named ’loaded checkpoint’ in this block; model: the value named ’model’ in this block; epoch: the value named ’epoch’ in this block.

Line 338: `model=model,` This is continuation line 2 of the 7-line logical statement that begins at source line 337. Its exact fragment is ‘model=model,‘. The important operands are model: the value named ’model’ in this block. Read together, the complete operation is: This returns ‘LoadedCheckpoint(model=model, epoch=int(checkpoint.get("epoch", -1)), metric=float(checkpoint.get("metric", float("nan"))), architecture=str(checkpoint.get("arch", "unknown")), checkpoint_path=path,)‘ to the caller of the function load_checkpoint, which handles load checkpoint, ending this invocation. The important operands are LoadedCheckpoint: the value named ’loaded checkpoint’ in this block; model: the value named ’model’ in this block; epoch: the value named ’epoch’ in this block.

Line 339: `epoch=int(checkpoint.get("epoch", -1)),` This is continuation line 3 of the 7-line logical statement that begins at source line 337. Its exact fragment is ‘epoch=int(checkpoint.get("epoch", -1)),‘. The important operands are epoch: the value named ’epoch’ in this block; checkpoint: the value named ’checkpoint’ in this block; get: the value named ’get’ in this block. Read together, the complete operation is: This returns ‘LoadedCheckpoint(model=model, epoch=int(checkpoint.get("epoch", -1)), metric=float(checkpoint.get("metric", float("nan"))), architecture=str(checkpoint.get("arch" "unknown")), checkpoint_path=path,)‘ to the caller of the function load_checkpoint, which handles load checkpoint, ending this invocation. The important operands are LoadedCheckpoint: the value named ’loaded checkpoint’ in this block; model: the value named ’model’ in this block; epoch: the value named ’epoch’ in this block.

Line 340: `metric=float(checkpoint.get("metric", float("nan"))),` This is continuation line 4 of the 7-line logical statement that begins at source line 337. Its exact fragment is ‘metric=float(checkpoint.get("metric", float("nan"))),‘. The important operands are metric: the value named ’metric’ in this block; float: the value named ’float’ in this block; checkpoint: the value named ’checkpoint’ in this block. Read together, the complete operation is: This returns ‘LoadedCheckpoint(model=model, epoch=int(checkpoint.get("epoch", -1)), metric=float(checkpoint.get("metric", float("nan"))), architecture=str(checkpoint.get("arch", "unknown")), checkpoint_path=path,)‘ to the caller of the function load_checkpoint, which handles load checkpoint, ending this invocation. The important operands are LoadedCheckpoint: the value named ’loaded checkpoint’ in this block; model: the value named ’model’ in this block; epoch: the value named ’epoch’ in this block.

Line 341: `architecture=str(checkpoint.get("arch", "unknown")),` This is continuation line 5 of the 7-line logical statement that begins at source line 337. Its exact fragment is ‘architecture=str(checkpoint.get("arch", "unknown")),‘. The important operands are architecture: the value named ’architecture’ in this block; str: the value named ’str’ in this block; checkpoint: the value named ’checkpoint’ in this block. Read together, the complete operation is: This returns ‘LoadedCheckpoint(model=model, epoch=int(checkpoint.get("epoch", -1)), metric=float(checkpoint.get("metric", float("nan"))), architecture=str(checkpoint.get("arch", "unknown")), checkpoint_path=path,)‘ to the caller of the function load_checkpoint, which handles load checkpoint, ending this invocation. The important operands are LoadedCheckpoint: the value named ’loaded checkpoint’ in this block; model: the value named ’model’ in this block; epoch: the value named ’epoch’ in this block.

Line 342: `checkpoint_path=path,` This is continuation line 6 of the 7-line logical statement that begins at source line 337. Its exact fragment is 'checkpoint_path=path,'. The important operands are checkpoint_path: the value named 'checkpoint path' in this block; path: the value named 'path' in this block. Read together, the complete operation is: This returns 'LoadedCheckpoint(model=model, epoch=int(checkpoint.get("epoch", -1)), metric=float(checkpoint.get("metric", float("nan"))), architecture=str(checkpoint.get("arch", "unknown")), checkpoint_path=path,)' to the caller of the function load_checkpoint, which handles load checkpoint, ending this invocation. The important operands are LoadedCheckpoint: the value named 'loaded checkpoint' in this block; model: the value named 'model' in this block; epoch: the value named 'epoch' in this block.

Line 343: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 337. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'LoadedCheckpoint(model=model, epoch=int(checkpoint.get("epoch", -1)), metric=float(checkpoint.get("metric", float("nan"))), architecture=str(checkpoint.get("arch", "unknown")), checkpoint_path=path,)' to the caller of the function load_checkpoint, which handles load checkpoint, ending this invocation. The important operands are LoadedCheckpoint: the value named 'loaded checkpoint' in this block; model: the value named 'model' in this block; epoch: the value named 'epoch' in this block.

Line 346: `def fold_batch_norm(` This starts function fold_batch_norm. Its indented body performs fold batch norm as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 347: `convolution: nn.Conv1d | nn.Conv2d,` Inside the function fold_batch_norm, which handles fold batch norm, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are convolution: the value named 'convolution' in this block; nn: the value named 'nn' in this block; Conv1d: the value named 'conv1d' in this block.

Line 348: `batch_norm: nn.BatchNorm1d | nn.BatchNorm2d,` Inside the function fold_batch_norm, which handles fold batch norm, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are batch_norm: the value named 'batch norm' in this block; nn: the value named 'nn' in this block; BatchNorm1d: the value named 'batch norm1d' in this block.

Line 349: `) -> tuple[torch.Tensor, torch.Tensor]:` Inside the function fold_batch_norm, which handles fold batch norm, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are tuple: the value named 'tuple' in this block; torch: the value named 'torch' in this block; Tensor: the value named 'tensor' in this block.

Line 350: `"""Return convolution weight/bias tensors with evaluation BatchNorm folded.` This is executable documentation attached to the function fold_batch_norm, which handles fold batch norm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 352: `For each output channel "o", BatchNorm applies` Inside the function fold_batch_norm, which handles fold batch norm, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are For: the value named 'for' in this block; each: the value named 'each' in this block; output: the value named 'output' in this block.

Line 353: `"gamma[o] * (y - mean[o]) / sqrt(var[o] + eps) + beta[o]".` Inside the function fold_batch_norm, which handles fold batch norm, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are gamma: the learned multiplicative LayerNorm parameter; o: the value named 'o' in this block; y: the value named 'y' in this block.

Line 354: `Multiplying the convolution kernel and shifting its bias by those constants` Inside the function fold_batch_norm, which handles fold batch norm, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are

Multiplying: the value named 'multiplying' in this block; the: the value named 'the' in this block; convolution: the value named 'convolution' in this block.

Line 355: `produces exactly the same evaluation result up to floating-point rounding.` Inside the function fold_batch_norm, which handles fold batch norm, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are produces: the value named 'produces' in this block; exactly: the value named 'exactly' in this block; the: the value named 'the' in this block.

Line 356: `"""` This is executable documentation attached to the function fold_batch_norm, which handles fold batch norm. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 358: `if convolution.out_channels != batch_norm.num_features:` This executes the following indented host-code branch only when 'convolution.out_channels != batch_norm.num_features' is true. The important operands are convolution: the value named 'convolution' in this block; out_channels: the value named 'out channels' in this block; batch_norm: the value named 'batch norm' in this block. Context: the function fold_batch_norm, which handles fold batch norm.

Line 359: `raise ValueError("convolution and BatchNorm output widths differ")` This rejects an invalid state by raising 'ValueError("convolution and BatchNorm output widths differ")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 360: `weight = convolution.weight.detach()` This assigns 'convolution.weight.detach()' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are convolution: the value named 'convolution' in this block; weight: a learned coefficient multiplied by an activation; detach: the value named 'detach' in this block. Context: the function fold_batch_norm, which handles fold batch norm.

Line 361: `if convolution.bias is None:` This executes the following indented host-code branch only when 'convolution.bias is None' is true. The important operands are convolution: the value named 'convolution' in this block; bias: an additive learned offset applied after accumulation; is: the value named 'is' in this block. Context: the function fold_batch_norm, which handles fold batch norm.

Line 362: `bias = torch.zeros(` This statement continues through source line 363; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'torch.zeros(convolution.out_channels, dtype=weight.dtype, device=weight.device)' to 'bias ' in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are torch: the value named 'torch' in this block; zeros: the value named 'zeros' in this block; convolution: the value named 'convolution' in this block. Context: the function fold_batch_norm, which handles fold batch norm.

Line 363: `convolution.out_channels, dtype=weight.dtype, device=weight.device)` This is continuation line 2 of the 2-line logical statement that begins at source line 362. Its exact fragment is 'convolution.out_channels, dtype=weight.dtype, device=weight.device)'. The important operands are convolution: the value named 'convolution' in this block; out_channels: the value named 'out channels' in this block; dtype: the value named 'dtype' in this block. Read together, the complete operation is: This assigns 'torch.zeros(convolution.out_channels, dtype=weight.dtype, device=weight.device)' to 'bias ' in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are torch: the value named 'torch' in this block; zeros: the value named 'zeros' in this block; convolution: the value named 'convolution' in this block. Context: the function fold_batch_norm, which handles fold batch norm.

Line 364: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function fold_batch_norm, which handles fold batch norm.

Line 365: `bias = convolution.bias.detach()` This assigns ‘convolution.bias.detach()‘ to ‘bias ‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are convolution: the value named ’convolution’ in this block; bias: an additive learned offset applied after accumulation; detach: the value named ’detach’ in this block. Context: the function fold_batch_norm, which handles fold batch norm.

Line 366: `scale = batch_norm.weight.detach() / torch.sqrt(` This statement continues through source line 367; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘batch_norm.weight.detach() / torch.sqrt(batch_norm.running_var.detach() + batch_norm.eps)‘ to ‘scale ‘ in host memory. Here scale means the real-value spacing represented by one quantized integer step. The important operands are batch_norm: the value named ’batch norm’ in this block; weight: a learned coefficient multiplied by an activation; detach: the value named ’detach’ in this block. Context: the function fold_batch_norm, which handles fold batch norm.

Line 367: `batch_norm.running_var.detach() + batch_norm.eps)` This is continuation line 2 of the 2-line logical statement that begins at source line 366. Its exact fragment is ‘batch_norm.running_var.detach() + batch_norm.eps)‘. The important operands are batch_norm: the value named ’batch norm’ in this block; running_var: the value named ’running var’ in this block; detach: the value named ’detach’ in this block. Read together, the complete operation is: This assigns ‘batch_norm.weight.detach() / torch.sqrt(batch_norm.running_var.detach() + batch_norm.eps)‘ to ‘scale ‘ in host memory. Here scale means the real-value spacing represented by one quantized integer step. The important operands are batch_norm: the value named ’batch norm’ in this block; weight: a learned coefficient multiplied by an activation; detach: the value named ’detach’ in this block. Context: the function fold_batch_norm, which handles fold batch norm.

Line 368: `reshape = (scale.shape[0],) + (1,) * (weight.ndim - 1)` This assigns ‘(scale.shape[0],) + (1,) * (weight.ndim - 1)‘ to ‘reshape ‘ in host memory. Here reshape means the value named ’reshape’ in this block. The important operands are scale: the real-value spacing represented by one quantized integer step; shape: the value named ’shape’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function fold_batch_norm, which handles fold batch norm.

Line 369: `folded_weight = weight * scale.reshape(reshape)` This assigns ‘weight * scale.reshape(reshape)‘ to ‘folded_weight ‘ in host memory. Here folded_weight means a learned coefficient multiplied by an activation. The important operands are weight: a learned coefficient multiplied by an activation; scale: the real-value spacing represented by one quantized integer step; reshape: the value named ’reshape’ in this block. Context: the function fold_batch_norm, which handles fold batch norm.

Line 370: `folded_bias = (bias - batch_norm.running_mean.detach()) * scale` This assigns ‘(bias - batch_norm.running_ * scale‘ to ‘folded_bias ‘ in host memory. Here folded_bias means an additive learned offset applied after accumulation. The important operands are bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; running_mean: the arithmetic average used by normalization. Context: the function fold_batch_norm, which handles fold batch norm.

Line 371: `folded_bias = folded_bias + batch_norm.bias.detach()` This assigns ‘folded_bias + batch_norm.bias.detach()‘ to ‘folded_bias ‘ in host memory. Here folded_bias means an additive learned offset applied after accumulation. The important operands are folded_bias: an additive learned offset applied after accumulation; batch_norm: the value named ’batch norm’ in this block; bias: an additive learned offset applied after accumulation. Context: the function fold_batch_norm, which handles fold batch norm.

Line 372: `return folded_weight, folded_bias` This returns ‘folded_weight, folded_bias‘ to the caller of the function fold_batch_norm, which handles fold batch norm, ending this invocation. The important operands are folded_weight: a learned coefficient multiplied by an activation; folded_bias: an additive learned offset applied after accumulation.

Line 375: `def _split_heads(value: torch.Tensor, heads: int) -> torch.Tensor:` This starts function _split_heads. Its indented body performs split heads as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 376: `time_steps, batch, channels, tokens = value.shape` This assigns ‘value.shape‘ to ‘time_steps, batch, channels, tokens ‘ in host memory. Here time_steps means the value named ’time steps’ in this block. The important operands are value: the value named ’value’ in this block; shape: the value named ’shape’ in this block. Context: the function _split_heads, which handles split heads.

Line 377: `head_dim = channels // heads` This assigns ‘channels // heads‘ to ‘head_dim ‘ in host memory. Here head_dim means the value named ’head dim’ in this block. The important operands are channels: the value named ’channels’ in this block; heads: the value named ’heads’ in this block. Context: the function _split_heads, which handles split heads.

Line 378: `value = value.transpose(-1, -2)` This assigns ‘value.transpose(-1, -2)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; transpose: the value named ’transpose’ in this block. Context: the function _split_heads, which handles split heads.

Line 379: `value = value.reshape(time_steps, batch, tokens, heads, head_dim)` This assigns ‘value.reshape(time_steps, batch, tokens, heads, head_dim)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are value: the value named ’value’ in this block; reshape: the value named ’reshape’ in this block; time_steps: the value named ’time steps’ in this block. Context: the function _split_heads, which handles split heads.

Line 380: `return value.permute(0, 1, 3, 2, 4)` This returns ‘value.permute(0, 1, 3, 2, 4)‘ to the caller of the function _split_heads, which handles split heads, ending this invocation. The important operands are value: the value named ’value’ in this block; permute: the value named ’permute’ in this block.

Line 383: `def _observe(` This starts function _observe. Its indented body performs observe as part of the file that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison; it runs only when called.

Line 384: `observer: TensorObserver | None,` Inside the function _observe, which handles observe, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are observer: the value named ’observer’ in this block; TensorObserver: the value named ’tensor observer’ in this block; None: the value named ’none’ in this block.

Line 385: `name: str,` Inside the function _observe, which handles observe, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are name: the value named ’name’ in this block; str: the value named ’str’ in this block.

Line 386: `value: torch.Tensor,` Inside the function _observe, which handles observe, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are value: the value named ’value’ in this block; torch: the value named ’torch’ in this block; Tensor: the value named ’tensor’ in this block.

Line 387: `is_spike: bool,` Inside the function _observe, which handles observe, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are is_spike: a binary event produced when a neuron crosses its threshold; bool: the value named ’bool’ in this block.

Line 388: `) -> None:` Inside the function _observe, which handles observe, this statement advances the host implementation that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison. The important operands are None: the value named ’none’ in this block.

Line 389: `if observer is not None:` This executes the following indented host-code branch only when ‘observer is not None‘ is true. The important operands are observer: the value named ’observer’ in this block; is: the value named ’is’ in this block; not: the value named ’not’ in this block. Context: the function _observe, which handles observe.

Line 390: `observer(name, value, is_spike)` This calls ‘observer‘ with ‘name, value, is_spike‘ inside the function _observe, which handles observe. The call advances the host-side job that reconstructs the trained source graph and its integer hardware-emulation path for accuracy comparison.

44 software/spikingformer_training_runtime.py

Role in the complete system: runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

44.1 Beginner architecture map

Dataflow responsibility Each epoch loads deterministic CIFAR-10 shards, executes the quantized forward graph, forms the supervised loss gradient, and dispatches hardware backward and optimizer descriptor programs. Updated parameter and velocity arrays are checkpointed atomically with epoch, sample cursor, RNG state, hashes, and metrics, so pause or power loss resumes from a committed minibatch boundary instead of replaying an ambiguous update.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: FullTrainingDriver, BlockGradient, ParameterGradient, MasterTensor, TrainableSpikingformerState, LifTraceCache, TokenizerLayerCache, AttentionCache, MlpCache, TransformerBlockCache, SpikingformerTrainingCache, TrainingBackwardResult, TrainingStepResult, SpikingformerOnDeviceTrainer. Functions: __padded__lanes, execute__vector__tensor, execute__power__of__two__scale, normalize__gradient__s16, add__block__gradients, __required__right__shift, backward__requantize__projection, execute__linear__backward__block, execute__affine__backward__block execute__conv2d__backward__block, execute__softmax__q15, execute__cross__entropy__gradient__q15, __master__fractional__bits, execute__deploy__master, gradient__to__optimizer__q15, ___post__init___, reshape, transpose, ___post__init___, ___init___, projection__names, projection, head, _optimizer__coefficients. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

44.2 Complete source

```
"""Full fixed-point Spikingformer training runtime for the NeuraEdge fabric.

The processing system owns graph scheduling and DDR tensors. Matrix derivatives,
surrogate-gradient recurrence, pooling scatter, vector arithmetic, softmax, and
SGDW updates are submitted to programmable-logic engines through the command ABI.
"""

from __future__ import annotations

import hashlib
import json
import math
import os
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Mapping, Protocol, Sequence

import numpy as np

from neuraedge_compiler import compile_spike_gemm, compile_u8_gemm
from neuraedge_golden import VectorOperation, round_shift_signed
from neuraedge_protocol import (
    Opcode,
    ProgramRecord,
    clear_status_command,
    log_softmax_command,
    pack_softmax_beats,
    pack_vector_beats,
    unpack_softmax_result,
    unpack_vector_result,
    vector_command,
)
from neuraedge_training import (
    TrainingProgramExecutor,
    compile_affine_backward,
    compile_conv2d_backward,
    compile_linear_backward,
    execute_lif_bptt,
```

```
    execute_maxpool2d_backward,
    execute_sgdw_update,
)
from spikingformer_accelerator import (
    SpikingformerInferenceResult,
    SpikingformerProjectionExecutor,
    binary_max_pool2d_on_accelerator,
    lif_sequence_with_trace_on_accelerator,
)
from spikingformer_parameters import (
    HeadParameters,
    ProjectionParameters,
    SpikingformerParameterArchive,
)


VECTOR_LANES = 8
MASTER_FORMAT = "neuraedge-spikingformer-trainable-state-v1"
CHECKPOINT_FORMAT = "neuraedge-spikingformer-training-checkpoint-v1"


class FullTrainingDriver(TrainingProgramExecutor, Protocol):
    """Command interface required by both forward inference and backward update."""


@dataclass(frozen=True)
class BlockGradient:
    """Signed integer gradient plus its exact shared power-of-two exponent."""

    values: np.ndarray
    exponent: int

    def __post_init__(self) -> None:
        values = np.asarray(self.values, dtype=np.int64)
        if values.size and (
            values.min() < -(1 << 31) or values.max() > (1 << 31) - 1
        ):
            raise OverflowError("block-gradient lanes must fit signed 32-bit")
        object.__setattr__(self, "values", values)
        if not -127 <= int(self.exponent) <= 127:
            raise ValueError("block-gradient exponent must be in [-127,127]")

    def reshape(self, *shape: int) -> "BlockGradient":
        return BlockGradient(self.values.reshape(*shape), self.exponent)

    def transpose(self, *axes: int) -> "BlockGradient":
        return BlockGradient(self.values.transpose(*axes), self.exponent)


@dataclass(frozen=True)
class ParameterGradient:
    """Weight and bias derivatives emitted by one learned affine operator."""

    weight: BlockGradient
    bias: BlockGradient


@dataclass
class MasterTensor:
    """Signed-16 master value, signed-24 Q15 momentum, and deployment format."""

    master: np.ndarray
    velocity_q15: np.ndarray
    fractional_bits: int
    deploy_minimum: int
    deploy_maximum: int

    def __post_init__(self) -> None:
        master = np.asarray(self.master)
        velocity = np.asarray(self.velocity_q15)
        if not np.issubdtype(master.dtype, np.integer) or not np.issubdtype(
            velocity.dtype, np.integer
        ):
            raise ValueError("master and velocity tensors must contain integers")
        if master.shape != velocity.shape or master.size == 0:
            raise ValueError("master and velocity tensors must share a nonempty shape")
        if np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1):
            raise ValueError("master tensor must fit signed 16-bit")
        if np.any(velocity < -(1 << 23)) or np.any(velocity > (1 << 23) - 1):
            raise ValueError("master velocity must fit signed 24-bit")
        self.master = master.astype(np.int16, copy=False)
        self.velocity_q15 = velocity.astype(np.int32, copy=False)
        if not 0 <= self.fractional_bits <= 14:
            raise ValueError("master fractional bits must be in [0,14]")
        if self.deploy_minimum > self.deploy_maximum:
            raise ValueError("deployment clamp is inverted")

```

```

def _padded_lanes(values: Sequence[int], fill: int = 0) -> tuple[int, ...]:
    lanes = tuple(map(int, values))
    if len(lanes) > VECTOR_LANES:
        raise ValueError("a vector chunk cannot exceed eight lanes")
    return lanes + (int(fill),) * (VECTOR_LANES - len(lanes))


def execute_vector_tensor(
    driver: FullTrainingDriver,
    operation: VectorOperation,
    operand_a,
    operand_b=None,
    *,
    multiplier_q15: int = 0,
    right_shift: int = 0,
    output_zero_point: int = 0,
    threshold: int = 0,
    surrogate_window: int = 0,
    clamp_minimum: int = -(1 << 31),
    clamp_maximum: int = (1 << 31) - 1,
    timeout_seconds: float = 30.0,
) -> np.ndarray:
    """Apply one vector command to every element using descriptor batching."""

    a = np.asarray(operand_a, dtype=np.int64)
    b = np.zeros_like(a) if operand_b is None else np.asarray(operand_b, dtype=np.int64)
    if a.shape != b.shape:
        raise ValueError("vector operands must share a shape")
    for name, value in (("operand_a", a), ("operand_b", b)):
        if value.size and (
            value.min() < -(1 << 31) or value.max() > (1 << 31) - 1
        ):
            raise OverflowError(f"{name} does not fit signed 32-bit lanes")
    if a.size == 0:
        return np.empty(a.shape, dtype=np.int64)
    command = vector_command(
        int(operation),
        multiplier_q15=multiplier_q15,
        right_shift=right_shift,
        output_zero_point=output_zero_point,
        threshold=threshold,
        surrogate_window=surrogate_window,
    )
    flat_a = a.reshape(-1)
    flat_b = b.reshape(-1)
    records: list[ProgramRecord] = []
    counts: list[int] = []
    for start in range(0, flat_a.size, VECTOR_LANES):
        stop = min(start + VECTOR_LANES, flat_a.size)
        count = stop - start
        records.append(ProgramRecord(
            command,
            pack_vector_beats(
                _padded_lanes(flat_a[start:stop]),
                _padded_lanes(flat_b[start:stop]),
                clamp_minimum=clamp_minimum,
                clamp_maximum=clamp_maximum,
            ),
            4,
        ))
        counts.append(count)
    words = driver.execute_program(records, timeout_seconds=timeout_seconds)
    if len(words) != 4 * len(records):
        raise RuntimeError(
            f"vector program returned {len(words)} beats; expected {4 * len(records)}"
        )
    result = np.empty(flat_a.size, dtype=np.int64)
    cursor = 0
    for record_index, count in enumerate(counts):
        lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])
        result[cursor:cursor + count] = lanes[:count]
        cursor += count
    return result.reshape(a.shape)


def execute_power_of_two_scale(
    driver: FullTrainingDriver,
    values,
    power: int,
    *,
    clamp_minimum: int = -(1 << 31),
    clamp_maximum: int = (1 << 31) - 1,
    timeout_seconds: float = 30.0,
) -> np.ndarray:
    """Multiply by ``2**power`` with exact fabric round-to-nearest shifts."""

```

```
    result = np.asarray(values, dtype=np.int64)
    remaining = int(power)
    while remaining:
        step = min(remaining, 14) if remaining > 0 else max(remaining, -16)
        result = execute_vector_tensor(
            driver,
            VectorOperation.FLEX_REQUANTIZE,
            result,
            multiplier_q15=1 << 14,
            right_shift=-step - 1,
            clamp_minimum=clamp_minimum,
            clamp_maximum=clamp_maximum,
            timeout_seconds=timeout_seconds,
        )
        remaining -= step
    return result


def normalize_gradient_s16(
    driver: FullTrainingDriver,
    gradient: BlockGradient,
    *,
    timeout_seconds: float = 30.0,
) -> BlockGradient:
    """Right-shift a block only as much as required for signed-16 matrix input."""

    values = gradient.values
    maximum = int(np.abs(values).max(initial=0))
    shift = 0
    while maximum > 32767:
        maximum = (maximum + 1) // 2
        shift += 1
    if shift == 0:
        return BlockGradient(values.astype(np.int64, copy=False), gradient.exponent)
    shifted = execute_power_of_two_scale(
        driver,
        values,
        -shift,
        clamp_minimum=-32768,
        clamp_maximum=32767,
        timeout_seconds=timeout_seconds,
    )
    return BlockGradient(shifted, gradient.exponent + shift)


def add_block_gradients(
    driver: FullTrainingDriver,
    left: BlockGradient,
    right: BlockGradient,
    *,
    timeout_seconds: float = 30.0,
) -> BlockGradient:
    """Align two block exponents and add all lanes in programmable logic."""

    if left.values.shape != right.values.shape:
        raise ValueError("gradient addends must share a shape")
    exponent = max(left.exponent, right.exponent)
    left_values = left.values
    right_values = right.values
    if left.exponent != exponent:
        left_values = execute_power_of_two_scale(
            driver, left_values, left.exponent - exponent,
            timeout_seconds=timeout_seconds)
    if right.exponent != exponent:
        right_values = execute_power_of_two_scale(
            driver, right_values, right.exponent - exponent,
            timeout_seconds=timeout_seconds)
    result = execute_vector_tensor(
        driver,
        VectorOperation.ADD_SAT,
        left_values,
        right_values,
        timeout_seconds=timeout_seconds,
    )
    return BlockGradient(result, exponent)


def _required_right_shift(bound: int, maximum: int) -> int:
    if bound < 0 or maximum <= 0:
        raise ValueError("gradient bounds must be nonnegative with a positive limit")
    shift = 0
    while bound > maximum:
        bound = (bound + 1) // 2
        shift += 1
    if shift > 31:
        raise OverflowError("a gradient reduction requires more than 31 shift bits")
    return shift
```

```

def backward_requantize_projection(
    driver: FullTrainingDriver,
    output_gradient: BlockGradient,
    parameters: ProjectionParameters,
    *,
    timeout_seconds: float = 30.0,
) -> BlockGradient:
    """Differentiate the exported per-channel folded-BN requantizer with STE."""

    values = output_gradient.values
    channels = parameters.weight_k_by_n.shape[1]
    if values.ndim != 2 or values.shape[1] != channels:
        raise ValueError("projection output gradient must have shape [rows,channels]")
    shifts = parameters.binary_point_shift.astype(np.int64)
    multipliers = parameters.multiplier_q15.astype(np.int64)
    boost_minimum = max(int(shift) - 15 for shift in shifts)
    boost_maximum = min(int(shift) + 15 for shift in shifts)
    selected_boost: int | None = None
    for boost in range(boost_maximum, boost_minimum - 1, -1):
        represented_maximum = 0
        for channel in range(channels):
            product = values[:, channel] * int(multipliers[channel])
            represented = round_shift_signed(
                product, 15 + int(shifts[channel]) - boost)
            represented_maximum = max(
                represented_maximum, int(np.abs(represented).max(initial=0)))
        if represented_maximum <= 32767:
            selected_boost = boost
            break
    if selected_boost is None:
        raise OverflowError("projection derivative cannot be represented by signed-16 lanes")

    rows = values.shape[0]
    records: list[ProgramRecord] = []
    bindings: list[tuple[int, int, int]] = []
    zeros = (0,) * VECTOR_LANES
    for channel in range(channels):
        command = vector_command(
            int(VectorOperation.FLEX_REQUANTIZE),
            multiplier_q15=int(multipliers[channel]),
            right_shift=int(shifts[channel]) - selected_boost,
        )
        for row_start in range(0, rows, VECTOR_LANES):
            row_stop = min(row_start + VECTOR_LANES, rows)
            records.append(ProgramRecord(
                command,
                pack_vector_beats(
                    _padded_lanes(values[row_start:row_stop, channel]),
                    zeros,
                    clamp_minimum=-32768,
                    clamp_maximum=32767,
                ),
                4,
            ))
            bindings.append((row_start, row_stop, channel))
    words = driver.execute_program(records, timeout_seconds=timeout_seconds)
    if len(words) != 4 * len(records):
        raise RuntimeError("projection derivative returned an invalid stream length")
    result = np.zeros(values.shape, dtype=np.int64)
    for record_index, (row_start, row_stop, channel) in enumerate(bindings):
        lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])
        result[row_start:row_stop, channel] = lanes[:row_stop - row_start]
    return BlockGradient(result, output_gradient.exponent - selected_boost)


def execute_linear_backward_block(
    driver: FullTrainingDriver,
    activations,
    weights,
    output_gradient: BlockGradient,
    *,
    timeout_seconds: float = 120.0,
) -> tuple[BlockGradient, BlockGradient]:
    """Run a block-floating matrix backward pass with proven saturation bounds."""

    x = np.asarray(activations, dtype=np.int64)
    w = np.asarray(weights, dtype=np.int64)
    if x.ndim != 2 or w.ndim != 2 or x.shape[1] != w.shape[0]:
        raise ValueError("linear backward operands do not align")
    if output_gradient.values.shape != (x.shape[0], w.shape[1]):
        raise ValueError("linear output-gradient shape does not match Y=XW")
    dy = normalize_gradient_s16(
        driver, output_gradient, timeout_seconds=timeout_seconds)
    maximum_dy = int(np.abs(dy.values).max(initial=0))
    maximum_x = int(np.abs(x).max(initial=0))
```

```
    maximum_w = int(np.abs(w).max(initial=0))
    dx_bound = w.shape[1] * maximum_dy * maximum_w
    dw_bound = x.shape[0] * maximum_x * maximum_dy
    dx_shift = _required_right_shift(dx_bound, (1 << 31) - 1)
    dw_shift = _required_right_shift(dw_bound, (1 << 23) - 1)
    program = compile_linear_backward(
        x,
        w,
        dy.values,
        input_gradient_shift=dx_shift,
        weight_gradient_shift=dw_shift,
    )
    dx, dw = program.execute_batched(driver, timeout_seconds=timeout_seconds)
    return (
        BlockGradient(dx, dy.exponent + dx_shift),
        BlockGradient(dw, dy.exponent + dw_shift),
    )


def execute_affine_backward_block(
    driver: FullTrainingDriver,
    activations,
    weights,
    bias,
    output_gradient: BlockGradient,
    *,
    timeout_seconds: float = 120.0,
) -> tuple[BlockGradient, ParameterGradient]:
    """Run dX, dW, and dB for Y=XW+b through the matrix backward array."""

    x = np.asarray(activations, dtype=np.int64)
    w = np.asarray(weights, dtype=np.int64)
    b = np.asarray(bias, dtype=np.int64)
    if x.ndim != 2 or w.ndim != 2 or b.shape != (w.shape[1],):
        raise ValueError("affine operands have inconsistent shapes")
    if x.shape[1] != w.shape[0]:
        raise ValueError("affine activation and weight dimensions do not align")
    dy = normalize_gradient_s16(
        driver, output_gradient, timeout_seconds=timeout_seconds)
    maximum_dy = int(np.abs(dy.values).max(initial=0))
    maximum_x = max(int(np.abs(x).max(initial=0)), 1)
    maximum_w = max(int(np.abs(w).max(initial=0)), int(np.abs(b).max(initial=0)))
    dx_bound = w.shape[1] * maximum_dy * maximum_w
    dw_bound = x.shape[0] * maximum_x * maximum_dy
    dx_shift = _required_right_shift(dx_bound, (1 << 31) - 1)
    dw_shift = _required_right_shift(dw_bound, (1 << 23) - 1)
    program = compile_affine_backward(
        x,
        w,
        b,
        dy.values,
        input_gradient_shift=dx_shift,
        weight_gradient_shift=dw_shift,
    )
    dx, dw, db = program.execute_batched(driver, timeout_seconds=timeout_seconds)
    return (
        BlockGradient(dx, dy.exponent + dx_shift),
        ParameterGradient(
            BlockGradient(dw, dy.exponent + dw_shift),
            BlockGradient(db, dy.exponent + dw_shift),
        ),
    )


def execute_conv2d_backward_block(
    driver: FullTrainingDriver,
    activations,
    weights,
    bias,
    output_gradient: BlockGradient,
    *,
    stride=1,
    padding=0,
    dilation=1,
    groups: int = 1,
    timeout_seconds: float = 120.0,
) -> tuple[BlockGradient, ParameterGradient]:
    """Run grouped NCHW convolution dX/dW/dB with block-floating bounds."""

    x = np.asarray(activations, dtype=np.int64)
    w = np.asarray(weights, dtype=np.int64)
    b = np.asarray(bias, dtype=np.int64)
    dy = output_gradient.values
    if x.ndim != 4 or w.ndim != 4 or dy.ndim != 4:
        raise ValueError("convolution backward requires NCHW/OIHW tensors")
    normalized = normalize_gradient_s16(
        driver, output_gradient, timeout_seconds=timeout_seconds)
```

```
    maximum_dy = int(np.abs(normalized.values).max(initial=0))
    maximum_x = max(int(np.abs(x).max(initial=0)), 1)
    maximum_w = int(np.abs(w).max(initial=0))
    output_channels_per_group = w.shape[0] // groups
    kernel_elements = w.shape[2] * w.shape[3]
    dx_bound = output_channels_per_group * kernel_elements * maximum_dy * maximum_w
    reduction_rows = normalized.values.shape[0] * normalized.values.shape[2] * normalized.values.shape[3]
    dw_bound = reduction_rows * maximum_x * maximum_dy
    dx_shift = _required_right_shift(dx_bound, (1 << 31) - 1)
    dw_shift = _required_right_shift(dw_bound, (1 << 23) - 1)
    program = compile_conv2d_backward(
        x,
        w,
        b,
        normalized.values,
        stride=stride,
        padding=padding,
        dilation=dilation,
        groups=groups,
        input_gradient_shift=dx_shift,
        weight_gradient_shift=dw_shift,
    )
    dx, dw, db = program.execute_batched(driver, timeout_seconds=timeout_seconds)
    return (
        BlockGradient(dx, normalized.exponent + dx_shift),
        ParameterGradient(
            BlockGradient(dw, normalized.exponent + dw_shift),
            BlockGradient(db, normalized.exponent + dw_shift),
        ),
    )


def execute_softmax_q15(
    driver: FullTrainingDriver,
    logits,
    *,
    difference_shift: int,
    timeout_seconds: float = 30.0,
) -> np.ndarray:
    """Evaluate the hardware power-of-two softmax for each classifier row."""

    scores = np.asarray(logits, dtype=np.int64)
    if scores.ndim != 2 or not 1 <= scores.shape[1] <= 256:
        raise ValueError("logits must have shape [batch,classes] with 1..256 classes")
    if scores.size and (scores.min() < -(1 << 23) or scores.max() > (1 << 23) - 1):
        raise OverflowError("softmax logits must fit signed 24-bit")
    records = tuple(
        ProgramRecord(
            log_softmax_command(scores.shape[1], difference_shift),
            pack_softmax_beats(tuple(map(int, row))),
            scores.shape[1],
        )
        for row in scores
    )
    words = driver.execute_program(records, timeout_seconds=timeout_seconds)
    if len(words) != scores.size:
        raise RuntimeError("softmax returned an invalid stream length")
    probabilities = np.zeros(scores.shape, dtype=np.int64)
    for batch_index in range(scores.shape[0]):
        for class_index in range(scores.shape[1]):
            probability, returned_index, final = unpack_softmax_result(
                words[batch_index * scores.shape[1] + class_index]
            )
            if returned_index != class_index or final != (
                class_index == scores.shape[1] - 1
            ):
                raise RuntimeError("softmax result metadata is out of order")
            probabilities[batch_index, class_index] = probability
    return probabilities


def execute_cross_entropy_gradient_q15(
    driver: FullTrainingDriver,
    probabilities_q15,
    labels,
    *,
    timeout_seconds: float = 30.0,
) -> BlockGradient:
    """Form batch-mean ``probability - one_hot`` entirely with vector RTL."""

    probabilities = np.asarray(probabilities_q15, dtype=np.int64)
    targets = np.asarray(labels, dtype=np.int64)
    if probabilities.ndim != 2 or targets.shape != (probabilities.shape[0],):
        raise ValueError("labels must contain one class index per probability row")
    if np.any(targets < 0) or np.any(targets >= probabilities.shape[1]):
        raise ValueError("a target class is outside the classifier range")
    correction = np.zeros(probabilities.shape, dtype=np.int64)
```

```
    correction[np.arange(probabilities.shape[0]), targets] = -(1 << 15)
    gradient = execute_vector_tensor(
        driver,
        VectorOperation.ADD_SAT,
        probabilities,
        correction,
        timeout_seconds=timeout_seconds,
    )
    if probabilities.shape[0] > 1:
        reciprocal_q15 = int(round((1 << 15) / probabilities.shape[0]))
        gradient = execute_vector_tensor(
            driver,
            VectorOperation.MULTIPLY_Q15,
            gradient,
            np.full(gradient.shape, reciprocal_q15, dtype=np.int64),
            timeout_seconds=timeout_seconds,
        )
    return BlockGradient(gradient, -15)


def _master_fractional_bits(values: np.ndarray, maximum_bits: int) -> int:
    maximum = int(np.abs(np.asarray(values, dtype=np.int64)).max(initial=0))
    fractional = 0
    while fractional < maximum_bits and maximum <= (32767 >> (fractional + 1)):
        fractional += 1
    return fractional


def execute_deploy_master(
    driver: FullTrainingDriver,
    tensor: MasterTensor,
    *,
    timeout_seconds: float = 120.0,
) -> np.ndarray:
    """Round a signed-16 master tensor and clamp it to its deployment range."""

    shifted = execute_power_of_two_scale(
        driver,
        tensor.master,
        -tensor.fractional_bits,
        clamp_minimum=tensor.deploy_minimum,
        clamp_maximum=tensor.deploy_maximum,
        timeout_seconds=timeout_seconds,
    ) if tensor.fractional_bits else execute_vector_tensor(
        driver,
        VectorOperation.CLAMP,
        tensor.master,
        clamp_minimum=tensor.deploy_minimum,
        clamp_maximum=tensor.deploy_maximum,
        timeout_seconds=timeout_seconds,
    )
    return shifted.astype(np.int64)


def gradient_to_optimizer_q15(
    driver: FullTrainingDriver,
    gradient: BlockGradient,
    *,
    clip_q15: int,
    timeout_seconds: float = 120.0,
) -> np.ndarray:
    """Convert a block gradient to Q15 and apply explicit fabric clipping."""

    if not 1 <= clip_q15 <= (1 << 23) - 1:
        raise ValueError("clip_q15 must be in [1,8388607]")
    scaled = execute_power_of_two_scale(
        driver,
        gradient.values,
        gradient.exponent + 15,
        timeout_seconds=timeout_seconds,
    ) if gradient.exponent != -15 else gradient.values
    clipped = execute_vector_tensor(
        driver,
        VectorOperation.CLAMP,
        scaled,
        clamp_minimum=-clip_q15,
        clamp_maximum=clip_q15,
        timeout_seconds=timeout_seconds,
    )
    return clipped.astype(np.int64)


class TrainableSpikingformerState:
    """Mutable master parameters implementing the read-only archive interface."""

    def __init__(self, archive: SpikingformerParameterArchive) -> None:
        self.manifest = archive.manifest
```

```
        self.base_binary_sha256 = str(archive.manifest["binary_sha256"])
        self._projection_static: dict[str, ProjectionParameters] = {}
        self._projection_weight: dict[str, MasterTensor] = {}
        self._projection_bias: dict[str, MasterTensor] = {}
        self._deployed_projection_weight: dict[str, np.ndarray] = {}
        self._deployed_projection_bias: dict[str, np.ndarray] = {}
        for name in archive.projection_names:
            parameters = archive.projection(name)
            weight = parameters.weight_k_by_n.astype(np.int64)
            bias = parameters.bias_q10.astype(np.int64)
            weight_fractional_bits = 8
            bias_fractional_bits = _master_fractional_bits(bias, 8)
            self._projection_static[name] = parameters
            self._projection_weight[name] = MasterTensor(
                weight << weight_fractional_bits,
                np.zeros(weight.shape, dtype=np.int64),
                weight_fractional_bits,
                -128,
                127,
            )
            self._projection_bias[name] = MasterTensor(
                bias << bias_fractional_bits,
                np.zeros(bias.shape, dtype=np.int64),
                bias_fractional_bits,
                -(1 << 15),
                (1 << 15) - 1,
            )
            self._deployed_projection_weight[name] = weight.astype(np.int8)
            self._deployed_projection_bias[name] = bias.astype(np.int64)

        head = archive.head
        head_weight = head.weight_k_by_n.astype(np.int64)
        head_bias = head.bias_accumulator_units.astype(np.int64)
        head_bias_fractional_bits = _master_fractional_bits(head_bias, 8)
        self._head_weight = MasterTensor(
            head_weight << 8,
            np.zeros(head_weight.shape, dtype=np.int64),
            8,
            -128,
            127,
        )
        self._head_bias = MasterTensor(
            head_bias << head_bias_fractional_bits,
            np.zeros(head_bias.shape, dtype=np.int64),
            head_bias_fractional_bits,
            -(1 << 15),
            (1 << 15) - 1,
        )
        self._deployed_head_weight = head_weight.astype(np.int8)
        self._deployed_head_bias = head_bias.astype(np.int64)
        self.optimizer_step = 0

    @property
    def projection_names(self) -> tuple[str, ...]:
        return tuple(self._projection_static)

    def projection(self, name: str) -> ProjectionParameters:
        static = self._projection_static[name]
        return ProjectionParameters(
            static.name,
            self._deployed_projection_weight[name],
            self._deployed_projection_bias[name],
            static.multiplier_q15,
            static.binary_point_shift,
            static.compute_opcode,
            static.source_weight_shape,
            static.output_fractional_bits,
        )

    @property
    def head(self) -> HeadParameters:
        return HeadParameters(self._deployed_head_weight, self._deployed_head_bias)

    @staticmethod
    def _optimizer_coefficients(
        tensor: MasterTensor,
        learning_rate: float,
        momentum: float,
        weight_decay: float,
    ) -> tuple[int, int, int, int]:
        if not 0.0 <= learning_rate:
            raise ValueError("learning_rate must be nonnegative")
        if not 0.0 <= momentum <= 1.0 or not 0.0 <= weight_decay:
            raise ValueError("momentum must be in [0,1] and weight_decay nonnegative")
        learning_rate_q16 = int(round(learning_rate * (1 << 16)))
        momentum_q16 = int(round(momentum * (1 << 16)))
        decay_q16 = int(round(weight_decay * (1 << (31 - tensor.fractional_bits))))
```

```
        update_right_shift = 31 - tensor.fractional_bits
        if learning_rate > 0.0 and learning_rate_q16 == 0:
            raise ValueError("learning_rate is below the Q16 optimizer resolution")
        if learning_rate_q16 > 131071:
            raise OverflowError("Q16 learning rate exceeds the RTL field")
        if momentum_q16 > 65536 or decay_q16 > 65536:
            raise OverflowError("momentum or master-format weight decay exceeds the RTL field")
        return learning_rate_q16, momentum_q16, decay_q16, update_right_shift

    def _update_tensor(
        self,
        driver: FullTrainingDriver,
        tensor: MasterTensor,
        gradient: BlockGradient,
        *,
        learning_rate: float,
        momentum: float,
        weight_decay: float,
        gradient_clip_q15: int,
        timeout_seconds: float,
    ) -> np.ndarray:
        if gradient.values.shape != tensor.master.shape:
            raise ValueError("parameter gradient shape does not match its master tensor")
        gradient_q15 = gradient_to_optimizer_q15(
            driver,
            gradient,
            clip_q15=gradient_clip_q15,
            timeout_seconds=timeout_seconds,
        )
        coefficients = self._optimizer_coefficients(
            tensor, learning_rate, momentum, weight_decay)
        next_master, next_velocity = execute_sgdw_update(
            driver,
            tensor.master,
            gradient_q15,
            tensor.velocity_q15,
            learning_rate_q16=coefficients[0],
            momentum_q16=coefficients[1],
            weight_decay_q16=coefficients[2],
            update_right_shift=coefficients[3],
            timeout_seconds=timeout_seconds,
        )
        tensor.master = next_master.astype(np.int16)
        tensor.velocity_q15 = next_velocity.astype(np.int32)
        return execute_deploy_master(driver, tensor, timeout_seconds=timeout_seconds)

    def apply_gradients(
        self,
        driver: FullTrainingDriver,
        gradients: Mapping[str, ParameterGradient],
        *,
        learning_rate: float,
        momentum: float = 0.9,
        weight_decay: float = 5e-4,
        gradient_clip_q15: int = 4 << 15,
        timeout_seconds: float = 120.0,
    ) -> None:
        """Update every projection and classifier coefficient through RTL SGDW."""

        expected = set(self.projection_names) | {"head"}
        if set(gradients) != expected:
            missing = sorted(expected - set(gradients))
            extra = sorted(set(gradients) - expected)
            raise ValueError(f"parameter-gradient coverage mismatch: missing={missing}, extra={extra}")
        for name in self.projection_names:
            gradient = gradients[name]
            self._deployed_projection_weight[name] = self._update_tensor(
                driver,
                self._projection_weight[name],
                gradient.weight,
                learning_rate=learning_rate,
                momentum=momentum,
                weight_decay=weight_decay,
                gradient_clip_q15=gradient_clip_q15,
                timeout_seconds=timeout_seconds,
            ).astype(np.int8)
            self._deployed_projection_bias[name] = self._update_tensor(
                driver,
                self._projection_bias[name],
                gradient.bias,
                learning_rate=learning_rate,
                momentum=momentum,
                weight_decay=0.0,
                gradient_clip_q15=gradient_clip_q15,
                timeout_seconds=timeout_seconds,
            )
        head_gradient = gradients["head"]
```

```
        self._deployed_head_weight = self._update_tensor(
            driver,
            self._head_weight,
            head_gradient.weight,
            learning_rate=learning_rate,
            momentum=momentum,
            weight_decay=weight_decay,
            gradient_clip_q15=gradient_clip_q15,
            timeout_seconds=timeout_seconds,
        ).astype(np.int8)
        self._deployed_head_bias = self._update_tensor(
            driver,
            self._head_bias,
            head_gradient.bias,
            learning_rate=learning_rate,
            momentum=momentum,
            weight_decay=0.0,
            gradient_clip_q15=gradient_clip_q15,
            timeout_seconds=timeout_seconds,
        )
        self.optimizer_step += 1

    def _checkpoint_tensors(self) -> dict[str, np.ndarray]:
        tensors: dict[str, np.ndarray] = {}
        for index, name in enumerate(self.projection_names):
            for kind, tensor in (
                ("weight", self._projection_weight[name]),
                ("bias", self._projection_bias[name]),
            ):
                prefix = f"projection_{index}_{kind}"
                tensors[f"{prefix}_master"] = tensor.master
                tensors[f"{prefix}_velocity"] = tensor.velocity_q15
        tensors["head_weight_master"] = self._head_weight.master
        tensors["head_weight_velocity"] = self._head_weight.velocity_q15
        tensors["head_bias_master"] = self._head_bias.master
        tensors["head_bias_velocity"] = self._head_bias.velocity_q15
        return tensors

    def save_checkpoint(
        self,
        path: str | Path,
        *,
        epoch: int,
        batch_index: int,
        best_accuracy_percent: float,
        extra: Mapping[str, Any] | None = None,
    ) -> str:
        """Atomically save complete masters, momentum, and resume coordinates."""

        target = Path(path).expanduser().resolve()
        target.parent.mkdir(parents=True, exist_ok=True)
        metadata = {
            "format": CHECKPOINT_FORMAT,
            "state_format": MASTER_FORMAT,
            "base_binary_sha256": self.base_binary_sha256,
            "projection_names": list(self.projection_names),
            "projection_weight_fractional_bits": [
                self._projection_weight[name].fractional_bits for name in self.projection_names
            ],
            "projection_bias_fractional_bits": [
                self._projection_bias[name].fractional_bits for name in self.projection_names
            ],
            "head_weight_fractional_bits": self._head_weight.fractional_bits,
            "head_bias_fractional_bits": self._head_bias.fractional_bits,
            "optimizer_step": self.optimizer_step,
            "epoch": int(epoch),
            "batch_index": int(batch_index),
            "best_accuracy_percent": float(best_accuracy_percent),
            "extra": dict(extra or {}),
        }
        temporary = target.with_name(target.name + ".tmp")
        payload = self._checkpoint_tensors()
        payload["metadata_json"] = np.asarray(
            json.dumps(metadata, sort_keys=True, separators=(",", ":")))
        with temporary.open("wb") as stream:
            np.savez_compressed(stream, **payload)
            stream.flush()
            os.fsync(stream.fileno())
        os.replace(temporary, target)
        return hashlib.sha256(target.read_bytes()).hexdigest()

    def load_checkpoint(self, path: str | Path) -> Mapping[str, Any]:
        """Restore one complete checkpoint only after all metadata validates."""

        source = Path(path).expanduser().resolve()
        with np.load(source, allow_pickle=False) as archive:
            metadata = json.loads(str(archive["metadata_json"]))
```

```
            if metadata.get("format") != CHECKPOINT_FORMAT:
                raise ValueError("unsupported training checkpoint format")
            if metadata.get("base_binary_sha256") != self.base_binary_sha256:
                raise ValueError("checkpoint was initialized from a different parameter binary")
            if tuple(metadata.get("projection_names", ())) != self.projection_names:
                raise ValueError("checkpoint projection order does not match this graph")
            expected_weight_fractional = [
                self._projection_weight[name].fractional_bits for name in self.projection_names
            ]
            expected_bias_fractional = [
                self._projection_bias[name].fractional_bits for name in self.projection_names
            ]
            if metadata.get("projection_weight_fractional_bits") != expected_weight_fractional:
                raise ValueError("checkpoint projection weight Q formats do not match")
            if metadata.get("projection_bias_fractional_bits") != expected_bias_fractional:
                raise ValueError("checkpoint projection bias Q formats do not match")
            if int(metadata.get("head_weight_fractional_bits", -1)) != self._head_weight.fractional_bits:
                raise ValueError("checkpoint head weight Q format does not match")
            if int(metadata.get("head_bias_fractional_bits", -1)) != self._head_bias.fractional_bits:
                raise ValueError("checkpoint head bias Q format does not match")
            staged: list[tuple[MasterTensor, np.ndarray, np.ndarray]] = []
            for index, name in enumerate(self.projection_names):
                for kind, tensor in (
                    ("weight", self._projection_weight[name]),
                    ("bias", self._projection_bias[name]),
                ):
                    prefix = f"projection_{index}_{kind}"
                    master = np.asarray(archive[f"{prefix}_master"], dtype=np.int64)
                    velocity = np.asarray(archive[f"{prefix}_velocity"], dtype=np.int64)
                    if master.shape != tensor.master.shape or velocity.shape != tensor.master.shape:
                        raise ValueError(f"checkpoint tensor shape mismatch for {name} {kind}")
                    if (np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1)
                            or np.any(velocity < -(1 << 23))
                            or np.any(velocity > (1 << 23) - 1)):
                        raise ValueError(f"checkpoint tensor range mismatch for {name} {kind}")
                    staged.append((tensor, master.copy(), velocity.copy()))
            for prefix, tensor in (
                ("head_weight", self._head_weight),
                ("head_bias", self._head_bias),
            ):
                master = np.asarray(archive[f"{prefix}_master"], dtype=np.int64)
                velocity = np.asarray(archive[f"{prefix}_velocity"], dtype=np.int64)
                if master.shape != tensor.master.shape or velocity.shape != tensor.master.shape:
                    raise ValueError(f"checkpoint tensor shape mismatch for {prefix}")
                if (np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1)
                        or np.any(velocity < -(1 << 23))
                        or np.any(velocity > (1 << 23) - 1)):
                    raise ValueError(f"checkpoint tensor range mismatch for {prefix}")
                staged.append((tensor, master.copy(), velocity.copy()))
        for tensor, master, velocity in staged:
            tensor.master = master.astype(np.int16)
            tensor.velocity_q15 = velocity.astype(np.int32)
        self.optimizer_step = int(metadata["optimizer_step"])
        for name in self.projection_names:
            weight = self._projection_weight[name]
            bias = self._projection_bias[name]
            self._deployed_projection_weight[name] = np.clip(
                round_shift_signed(weight.master, weight.fractional_bits), -128, 127
            ).astype(np.int8)
            self._deployed_projection_bias[name] = np.clip(
                round_shift_signed(bias.master, bias.fractional_bits),
                -(1 << 15),
                (1 << 15) - 1,
            ).astype(np.int64)
        self._deployed_head_weight = np.clip(
            round_shift_signed(self._head_weight.master, self._head_weight.fractional_bits),
            -128,
            127,
        ).astype(np.int8)
        self._deployed_head_bias = np.clip(
            round_shift_signed(self._head_bias.master, self._head_bias.fractional_bits),
            -(1 << 15),
            (1 << 15) - 1,
        ).astype(np.int64)
        return metadata


@dataclass(frozen=True)
class LifTraceCache:
    """Forward values retained for one exact hard-reset LIF reverse pass."""

    input_q10: np.ndarray
    spikes: np.ndarray
    pre_reset_membrane: np.ndarray
    threshold_q10: int


```

```
@dataclass(frozen=True)
class TokenizerLayerCache:
    """One tokenizer projection and the LIF/pool operators that precede it."""

    index: int
    projection_input: np.ndarray
    projection_output: np.ndarray
    lif: LifTraceCache | None
    pooled: bool


@dataclass(frozen=True)
class AttentionCache:
    """All state needed to reverse one spike-driven self-attention branch."""

    block_index: int
    input_shape: tuple[int, ...]
    projection_lif: LifTraceCache
    token_spikes: np.ndarray
    query_lif: LifTraceCache
    key_lif: LifTraceCache
    value_lif: LifTraceCache
    query_heads: np.ndarray
    key_heads: np.ndarray
    value_heads: np.ndarray
    scores: np.ndarray
    attention_values: np.ndarray
    attention_lif: LifTraceCache
    projection_input: np.ndarray
    output: np.ndarray


@dataclass(frozen=True)
class MlpCache:
    """Forward state for the two learned MLP projections and their LIF nodes."""

    block_index: int
    input_shape: tuple[int, ...]
    input_lif: LifTraceCache
    first_projection_input: np.ndarray
    hidden_lif: LifTraceCache
    second_projection_input: np.ndarray
    output: np.ndarray


@dataclass(frozen=True)
class TransformerBlockCache:
    """Attention, MLP, and the two residual values of one transformer block."""

    input_value: np.ndarray
    attention: AttentionCache
    after_attention: np.ndarray
    mlp: MlpCache
    output: np.ndarray


@dataclass(frozen=True)
class SpikingformerTrainingCache:
    """Complete forward tape for one batch of the exported integer graph."""

    normalized_image: np.ndarray
    image_u8: np.ndarray
    image_centered: np.ndarray
    tokenizer: tuple[TokenizerLayerCache, ...]
    blocks: tuple[TransformerBlockCache, ...]
    final_value: np.ndarray
    head_preclip: np.ndarray
    head_input: np.ndarray
    logits_accumulator: np.ndarray
    logits: np.ndarray


@dataclass(frozen=True)
class TrainingBackwardResult:
    """Loss, classifier probabilities, all parameter gradients, and input dX."""

    loss: float
    probabilities_q15: np.ndarray
    predicted_class: np.ndarray
    parameter_gradients: Mapping[str, ParameterGradient]
    input_gradient: BlockGradient


@dataclass(frozen=True)
class TrainingStepResult:
    """Observable result of one complete forward/backward/SGDW transaction."""

```

```
    loss: float
    probabilities_q15: np.ndarray
    predicted_class: np.ndarray
    optimizer_step: int


class SpikingformerOnDeviceTrainer:
    """Execute and train the complete exported Spikingformer graph on NeuraEdge."""

    def __init__(
        self,
        state: TrainableSpikingformerState,
        driver: FullTrainingDriver,
        *,
        transformer_blocks: int = 4,
        attention_heads: int = 12,
        timesteps: int = 4,
        input_shape: tuple[int, int, int] = (3, 32, 32),
        layer_timeout_seconds: float = 120.0,
    ) -> None:
        if transformer_blocks <= 0 or attention_heads <= 0 or timesteps <= 0:
            raise ValueError("block, head, and timestep counts must be positive")
        if len(input_shape) != 3 or min(input_shape) <= 0:
            raise ValueError("input_shape must contain positive C,H,W dimensions")
        if layer_timeout_seconds <= 0:
            raise ValueError("layer_timeout_seconds must be positive")
        self.state = state
        self.driver = driver
        self.projections = SpikingformerProjectionExecutor(state, driver)
        self.transformer_blocks = int(transformer_blocks)
        self.attention_heads = int(attention_heads)
        self.timesteps = int(timesteps)
        self.input_shape = tuple(map(int, input_shape))
        self.layer_timeout_seconds = float(layer_timeout_seconds)
        self._surrogate_lut_loaded = False
        fixed = state.manifest.get("fixed_point")
        if not isinstance(fixed, Mapping):
            raise ValueError("trainable state has no fixed-point manifest")
        self.fractional_bits = int(fixed["activation_fractional_bits"])
        self.fixed_scale = 1 << self.fractional_bits
        if self.fixed_scale % 8:
            raise ValueError("attention scaling requires at least three activation fractional bits")
        attention_scale = self.fixed_scale // 8
        if attention_scale <= 0 or attention_scale & (attention_scale - 1):
            raise ValueError("attention scaling must be an exact power of two")
        self.attention_scale_shift = attention_scale.bit_length() - 1
        self.image_scale = float(fixed["image_input_scale"])
        self.image_zero_point = int(fixed["image_input_zero_point"])
        self.head_activation_scale = float(fixed["head_activation_scale"])
        required = {
            *(f"patch_embed.block{index}" for index in range(5)),
            *(f"block.{block}.attn.{name}"
              for block in range(self.transformer_blocks)
              for name in ("q", "k", "v", "proj")),
            *(f"block.{block}.mlp.fc{layer}"
              for block in range(self.transformer_blocks) for layer in (1, 2)),
        }
        missing = required - set(state.projection_names)
        if missing:
            raise ValueError(f"trainable state is missing projections: {sorted(missing)}")
        accumulator_scale = float(state.manifest["head"]["accumulator_scale"])
        ideal_difference = math.log(2.0) / accumulator_scale
        self.softmax_difference_shift = int(round(math.log2(ideal_difference)))
        if not 0 <= self.softmax_difference_shift <= 31:
            raise ValueError("head accumulator scale is outside the hardware softmax range")

    def _lif_trace(self, values: np.ndarray, *, threshold: float = 1.0) -> LifTraceCache:
        threshold_q10 = int(round(threshold * self.fixed_scale))
        spikes, membrane = lif_sequence_with_trace_on_accelerator(
            self.driver,
            values,
            threshold_q10=threshold_q10,
            leak_shift=1,
            reset_q10=0,
            timeout_seconds=self.layer_timeout_seconds,
        )
        return LifTraceCache(
            np.asarray(values, dtype=np.int64),
            spikes,
            membrane,
            threshold_q10,
        )

    def _dynamic_binary_gemm(self, left: np.ndarray, right: np.ndarray) -> np.ndarray:
        return compile_spike_gemm(left, right).execute_batched(
            self.driver, timeout_seconds=self.layer_timeout_seconds)

```

```
    def _dynamic_u8_gemm(self, left: np.ndarray, right: np.ndarray) -> np.ndarray:
        return compile_u8_gemm(left, right).execute_batched(
            self.driver, timeout_seconds=self.layer_timeout_seconds)

    def _global_sum(self, value: np.ndarray) -> np.ndarray:
        """Reduce time and space through a pairwise fabric ADD tree."""

        timesteps, batch, channels, height, width = value.shape
        reduction = value.transpose(0, 3, 4, 1, 2).reshape(
            timesteps * height * width, batch, channels)
        conservative_bound = int(np.abs(reduction).max(initial=0)) * reduction.shape[0]
        if conservative_bound > (1 << 31) - 1:
            raise OverflowError("global pooling could saturate the signed-32 vector tree")
        candidates = [reduction[index] for index in range(reduction.shape[0])]
        while len(candidates) > 1:
            reduced: list[np.ndarray] = []
            for index in range(0, len(candidates) - 1, 2):
                reduced.append(execute_vector_tensor(
                    self.driver,
                    VectorOperation.ADD_SAT,
                    candidates[index],
                    candidates[index + 1],
                    timeout_seconds=self.layer_timeout_seconds,
                ))
            if len(candidates) & 1:
                reduced.append(candidates[-1])
            candidates = reduced
        return candidates[0]

    def _tokenizer_forward(
        self, image_u8: np.ndarray, image_centered: np.ndarray
    ) -> tuple[np.ndarray, tuple[TokenizerLayerCache, ...]]:
        batch = image_u8.shape[0]
        first = self.projections.conv2d(
            "patch_embed.block0",
            image_u8,
            activation_zero_point=self.image_zero_point,
            padding=1,
            timeout_seconds=self.layer_timeout_seconds,
        )
        caches: list[TokenizerLayerCache] = [
            TokenizerLayerCache(0, image_centered, first, None, False)
        ]
        value = np.repeat(first[np.newaxis], self.timesteps, axis=0)
        for index in (1, 2, 3, 4):
            lif = self._lif_trace(value)
            flat_spikes = lif.spikes.reshape(-1, *lif.spikes.shape[2:])
            if index >= 3:
                projection_input = binary_max_pool2d_on_accelerator(
                    self.driver,
                    flat_spikes,
                    timeout_seconds=self.layer_timeout_seconds,
                )
            else:
                projection_input = flat_spikes
            projected = self.projections.conv2d(
                f"patch_embed.block{index}",
                projection_input,
                padding=1,
                timeout_seconds=self.layer_timeout_seconds,
            )
            caches.append(TokenizerLayerCache(
                index,
                projection_input,
                projected,
                lif,
                index >= 3,
            ))
            value = projected.reshape(
                self.timesteps, batch, *projected.shape[1:])
        return value, tuple(caches)

    def _attention_forward(self, block_index: int, value: np.ndarray) -> AttentionCache:
        timesteps, batch, channels, height, width = value.shape
        tokens = height * width
        projection_lif = self._lif_trace(value)
        token_spikes = projection_lif.spikes.reshape(
            timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)
        rows = token_spikes.reshape(timesteps * batch * tokens, channels)
        prefix = f"block.{block_index}.attn"

        def projected_lif(name: str) -> LifTraceCache:
            projected = self.projections.matrix(
                f"{prefix}.{name}", rows,
                timeout_seconds=self.layer_timeout_seconds)
            shaped = projected.reshape(timesteps, batch, tokens, channels)
            return self._lif_trace(shaped)
```

```
        query_lif = projected_lif("q")
        key_lif = projected_lif("k")
        value_lif = projected_lif("v")
        if channels % self.attention_heads:
            raise ValueError("attention channels must divide evenly into the head count")
        head_dim = channels // self.attention_heads

        def heads(spikes: np.ndarray) -> np.ndarray:
            return spikes.reshape(
                timesteps, batch, tokens, self.attention_heads, head_dim
            ).transpose(0, 1, 3, 2, 4)

        query_heads = heads(query_lif.spikes)
        key_heads = heads(key_lif.spikes)
        value_heads = heads(value_lif.spikes)
        scores = np.empty(
            (timesteps, batch, self.attention_heads, tokens, tokens),
            dtype=np.int64,
        )
        attention_values = np.empty(
            (timesteps, batch, self.attention_heads, tokens, head_dim),
            dtype=np.int64,
        )
        for timestep in range(timesteps):
            for batch_index in range(batch):
                for head in range(self.attention_heads):
                    score = self._dynamic_binary_gemm(
                        query_heads[timestep, batch_index, head],
                        key_heads[timestep, batch_index, head].T,
                    )
                    if score.min() < 0 or score.max() > 255:
                        raise OverflowError("binary QK score exceeds the unsigned AV contract")
                    scores[timestep, batch_index, head] = score
                    attention_values[timestep, batch_index, head] = self._dynamic_u8_gemm(
                        score.astype(np.uint8),
                        value_heads[timestep, batch_index, head].astype(np.uint8),
                    )
        attention_q10 = attention_values * (1 << self.attention_scale_shift)
        merged = attention_q10.transpose(0, 1, 2, 4, 3).reshape(
            timesteps, batch, channels, tokens)
        attention_lif = self._lif_trace(merged, threshold=0.5)
        projection_input = attention_lif.spikes.transpose(0, 1, 3, 2).reshape(
            timesteps * batch * tokens, channels)
        output_rows = self.projections.matrix(
            f"{prefix}.proj",
            projection_input,
            timeout_seconds=self.layer_timeout_seconds,
        )
        output = output_rows.reshape(
            timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(
                timesteps, batch, channels, height, width)
        return AttentionCache(
            block_index,
            tuple(value.shape),
            projection_lif,
            token_spikes,
            query_lif,
            key_lif,
            value_lif,
            query_heads,
            key_heads,
            value_heads,
            scores,
            attention_values,
            attention_lif,
            projection_input,
            output,
        )

    def _mlp_forward(self, block_index: int, value: np.ndarray) -> MlpCache:
        timesteps, batch, channels, height, width = value.shape
        tokens = height * width
        input_lif = self._lif_trace(value)
        first_input = input_lif.spikes.reshape(
            timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(
                timesteps * batch * tokens, channels)
        hidden_rows = self.projections.matrix(
            f"block.{block_index}.mlp.fc1",
            first_input,
            timeout_seconds=self.layer_timeout_seconds,
        )
        hidden_channels = hidden_rows.shape[1]
        hidden = hidden_rows.reshape(
            timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)
        hidden_lif = self._lif_trace(hidden)
        second_input = hidden_lif.spikes.transpose(0, 1, 3, 2).reshape(
```

```
            timesteps * batch * tokens, hidden_channels)
        output_rows = self.projections.matrix(
            f"block.{block_index}.mlp.fc2",
            second_input,
            timeout_seconds=self.layer_timeout_seconds,
        )
        output = output_rows.reshape(
            timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(
                timesteps, batch, channels, height, width)
        return MlpCache(
            block_index,
            tuple(value.shape),
            input_lif,
            first_input,
            hidden_lif,
            second_input,
            output,
        )

    def forward(self, normalized_image) -> SpikingformerTrainingCache:
        """Run the full graph and retain every state required by reverse mode."""

        image = np.asarray(normalized_image, dtype=np.float32)
        if image.ndim != 4 or tuple(image.shape[1:]) != self.input_shape or image.shape[0] == 0:
            raise ValueError(f"input must have shape [batch,{','.join(map(str, self.input_shape))}]")
        image_u8 = np.rint(image / self.image_scale + self.image_zero_point)
        image_u8 = np.clip(image_u8, 0, 255).astype(np.uint8)
        image_centered = image_u8.astype(np.int64) - self.image_zero_point
        self.driver.execute_program(
            (ProgramRecord(clear_status_command()),),
            timeout_seconds=self.layer_timeout_seconds,
        )
        value, tokenizer = self._tokenizer_forward(image_u8, image_centered)
        block_caches: list[TransformerBlockCache] = []
        for block_index in range(self.transformer_blocks):
            block_input = value
            attention = self._attention_forward(block_index, block_input)
            after_attention = execute_vector_tensor(
                self.driver,
                VectorOperation.ADD_SAT,
                block_input,
                attention.output,
                timeout_seconds=self.layer_timeout_seconds,
            )
            mlp = self._mlp_forward(block_index, after_attention)
            value = execute_vector_tensor(
                self.driver,
                VectorOperation.ADD_SAT,
                after_attention,
                mlp.output,
                timeout_seconds=self.layer_timeout_seconds,
            )
            block_caches.append(TransformerBlockCache(
                block_input, attention, after_attention, mlp, value))

        pooled_sum = self._global_sum(value.astype(np.int64))
        divisor = value.shape[0] * value.shape[3] * value.shape[4]
        denominator = divisor * self.fixed_scale * self.head_activation_scale
        rounded_denominator = int(round(denominator))
        if not math.isclose(denominator, rounded_denominator, rel_tol=0.0, abs_tol=1e-9):
            raise ValueError("head pooling denominator is not an exact integer")
        if rounded_denominator <= 0 or rounded_denominator & (rounded_denominator - 1):
            raise ValueError("head pooling denominator must be a positive power of two")
        head_preclip = execute_power_of_two_scale(
            self.driver,
            pooled_sum,
            -(rounded_denominator.bit_length() - 1),
            timeout_seconds=self.layer_timeout_seconds,
        )
        head_input = execute_vector_tensor(
            self.driver,
            VectorOperation.CLAMP,
            head_preclip,
            clamp_minimum=-128,
            clamp_maximum=127,
            timeout_seconds=self.layer_timeout_seconds,
        )
        head = self.state.head
        head_program = compile_u8_gemm(
            (head_input + 128).astype(np.uint8),
            (head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8),
            activation_zero_point=128,
            weight_zero_point=128,
            bias=head.bias_accumulator_units,
        )
        logits_accumulator = head_program.execute_batched(
            self.driver, timeout_seconds=self.layer_timeout_seconds)
```

```
        accumulator_scale = float(self.state.manifest["head"]["accumulator_scale"])
        logits = logits_accumulator.astype(np.float64) * accumulator_scale
        return SpikingformerTrainingCache(
            image,
            image_u8,
            image_centered,
            tokenizer,
            tuple(block_caches),
            value,
            head_preclip,
            head_input,
            logits_accumulator,
            logits,
        )

    def _lif_backward(
        self, cache: LifTraceCache, output_gradient: BlockGradient
    ) -> BlockGradient:
        if output_gradient.values.shape != cache.spikes.shape:
            raise ValueError("LIF output gradient does not match its forward trace")
        timesteps = cache.spikes.shape[0]
        membrane = cache.pre_reset_membrane.reshape(timesteps, -1)
        spikes = cache.spikes.reshape(timesteps, -1)
        gradient = output_gradient.values.reshape(timesteps, -1)
        input_gradient, _ = execute_lif_bptt(
            self.driver,
            membrane,
            spikes,
            gradient,
            threshold=cache.threshold_q10,
            reciprocal_tau_q15=1 << 14,
            surrogate_address_shift=max(self.fractional_bits - 5, 0),
            load_surrogate_table=not self._surrogate_lut_loaded,
            timeout_seconds=self.layer_timeout_seconds,
        )
        self._surrogate_lut_loaded = True
        return BlockGradient(
            input_gradient.reshape(cache.input_q10.shape), output_gradient.exponent)

    def _bias_gradient(self, output_gradient: BlockGradient) -> BlockGradient:
        if output_gradient.values.ndim != 2:
            raise ValueError("bias reduction expects a [rows,channels] gradient")
        rows, channels = output_gradient.values.shape
        ones = np.ones((rows, 1), dtype=np.int64)
        zeros = np.zeros((1, channels), dtype=np.int64)
        _, gradient = execute_linear_backward_block(
            self.driver,
            ones,
            zeros,
            output_gradient,
            timeout_seconds=self.layer_timeout_seconds,
        )
        return BlockGradient(gradient.values.reshape(channels), gradient.exponent)

    def _projection_matrix_backward(
        self,
        name: str,
        activations: np.ndarray,
        output_gradient: BlockGradient,
    ) -> tuple[BlockGradient, ParameterGradient]:
        parameters = self.state.projection(name)
        scaled_gradient = backward_requantize_projection(
            self.driver,
            output_gradient,
            parameters,
            timeout_seconds=self.layer_timeout_seconds,
        )
        input_gradient, weight_gradient = execute_linear_backward_block(
            self.driver,
            activations,
            parameters.weight_k_by_n,
            scaled_gradient,
            timeout_seconds=self.layer_timeout_seconds,
        )
        bias_gradient = self._bias_gradient(output_gradient)
        return input_gradient, ParameterGradient(weight_gradient, bias_gradient)

    def _projection_conv2d_backward(
        self,
        name: str,
        activations: np.ndarray,
        output_gradient: BlockGradient,
        *,
        stride=1,
        padding=0,
        dilation=1,
        groups: int = 1,
```

```
    ) -> tuple[BlockGradient, ParameterGradient]:
        parameters = self.state.projection(name)
        values = output_gradient.values
        if values.ndim != 4:
            raise ValueError("projection convolution gradient must be NCHW")
        batch, channels, height, width = values.shape
        matrix_gradient = output_gradient.transpose(0, 2, 3, 1).reshape(
            batch * height * width, channels)
        scaled_matrix = backward_requantize_projection(
            self.driver,
            matrix_gradient,
            parameters,
            timeout_seconds=self.layer_timeout_seconds,
        )
        scaled_nchw = scaled_matrix.reshape(
            batch, height, width, channels).transpose(0, 3, 1, 2)
        weight_oihw = parameters.weight_k_by_n.T.reshape(parameters.source_weight_shape)
        input_gradient, accumulator_parameter = execute_conv2d_backward_block(
            self.driver,
            activations,
            weight_oihw,
            parameters.bias_q10,
            scaled_nchw,
            stride=stride,
            padding=padding,
            dilation=dilation,
            groups=groups,
            timeout_seconds=self.layer_timeout_seconds,
        )
        weight_gradient = accumulator_parameter.weight.values.reshape(
            parameters.source_weight_shape[0], -1).T
        bias_gradient = self._bias_gradient(matrix_gradient)
        return input_gradient, ParameterGradient(
            BlockGradient(weight_gradient, accumulator_parameter.weight.exponent),
            bias_gradient,
        )

    def _token_rows_gradient(
        self, gradient: BlockGradient, channels: int
    ) -> BlockGradient:
        timesteps, batch, _, height, width = gradient.values.shape
        tokens = height * width
        values = gradient.values.reshape(
            timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(
                timesteps * batch * tokens, channels)
        return BlockGradient(values, gradient.exponent)

    @staticmethod
    def _rows_to_channel_first(
        gradient: BlockGradient,
        shape: tuple[int, ...],
    ) -> BlockGradient:
        timesteps, batch, channels, height, width = shape
        tokens = height * width
        values = gradient.values.reshape(
            timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(shape)
        return BlockGradient(values, gradient.exponent)

    def _stack_disjoint_gradients(
        self,
        gradients: Sequence[BlockGradient],
        shape: tuple[int, ...],
    ) -> BlockGradient:
        if not gradients:
            raise ValueError("at least one gradient is required for stacking")
        exponent = max(gradient.exponent for gradient in gradients)
        aligned: list[np.ndarray] = []
        for gradient in gradients:
            values = gradient.values
            if gradient.exponent != exponent:
                values = execute_power_of_two_scale(
                    self.driver,
                    values,
                    gradient.exponent - exponent,
                    timeout_seconds=self.layer_timeout_seconds,
                )
            aligned.append(values)
        return BlockGradient(np.stack(aligned).reshape(shape), exponent)

    def _gate_saturating_add_gradient(
        self,
        output_gradient: BlockGradient,
        left: np.ndarray,
        right: np.ndarray,
    ) -> BlockGradient:
        total = np.asarray(left, dtype=np.int64) + np.asarray(right, dtype=np.int64)
        mask = ((total >= -(1 << 31)) & (total <= (1 << 31) - 1)).astype(np.int64)
```

```
        gated = execute_vector_tensor(
            self.driver,
            VectorOperation.SELECT_NONZERO,
            mask,
            output_gradient.values,
            timeout_seconds=self.layer_timeout_seconds,
        )
        return BlockGradient(gated, output_gradient.exponent)

    def _mlp_backward(
        self,
        cache: MlpCache,
        output_gradient: BlockGradient,
    ) -> tuple[BlockGradient, dict[str, ParameterGradient]]:
        timesteps, batch, channels, height, width = cache.input_shape
        tokens = height * width
        output_rows = self._token_rows_gradient(output_gradient, channels)
        second_name = f"block.{cache.block_index}.mlp.fc2"
        hidden_spike_rows, second_parameter = self._projection_matrix_backward(
            second_name, cache.second_projection_input, output_rows)
        hidden_channels = cache.hidden_lif.spikes.shape[2]
        hidden_spike_gradient = hidden_spike_rows.reshape(
            timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)
        hidden_gradient = self._lif_backward(
            cache.hidden_lif,
            hidden_spike_gradient,
        )
        hidden_rows = BlockGradient(
            hidden_gradient.values.transpose(0, 1, 3, 2).reshape(
                timesteps * batch * tokens, hidden_channels),
            hidden_gradient.exponent,
        )
        first_name = f"block.{cache.block_index}.mlp.fc1"
        input_spike_rows, first_parameter = self._projection_matrix_backward(
            first_name, cache.first_projection_input, hidden_rows)
        input_spike_gradient = self._rows_to_channel_first(
            input_spike_rows, cache.input_shape)
        input_gradient = self._lif_backward(cache.input_lif, input_spike_gradient)
        return input_gradient, {
            first_name: first_parameter,
            second_name: second_parameter,
        }

    def _attention_backward(
        self,
        cache: AttentionCache,
        output_gradient: BlockGradient,
    ) -> tuple[BlockGradient, dict[str, ParameterGradient]]:
        timesteps, batch, channels, height, width = cache.input_shape
        tokens = height * width
        head_dim = channels // self.attention_heads
        prefix = f"block.{cache.block_index}.attn"
        output_rows = self._token_rows_gradient(output_gradient, channels)
        attention_spike_rows, projection_parameter = self._projection_matrix_backward(
            f"{prefix}.proj", cache.projection_input, output_rows)
        attention_spike_gradient = BlockGradient(
            attention_spike_rows.values.reshape(
                timesteps, batch, tokens, channels).transpose(0, 1, 3, 2),
            attention_spike_rows.exponent,
        )
        merged_gradient = self._lif_backward(
            cache.attention_lif, attention_spike_gradient)
        attention_value_gradient = BlockGradient(
            merged_gradient.values.reshape(
                timesteps, batch, self.attention_heads, head_dim, tokens
            ).transpose(0, 1, 2, 4, 3),
            merged_gradient.exponent + self.attention_scale_shift,
        )

        score_parts: list[BlockGradient] = []
        value_parts: list[BlockGradient] = []
        for timestep in range(timesteps):
            for batch_index in range(batch):
                for head in range(self.attention_heads):
                    score_gradient, value_gradient = execute_linear_backward_block(
                        self.driver,
                        cache.scores[timestep, batch_index, head],
                        cache.value_heads[timestep, batch_index, head],
                        BlockGradient(
                            attention_value_gradient.values[
                                timestep, batch_index, head],
                            attention_value_gradient.exponent,
                        ),
                        timeout_seconds=self.layer_timeout_seconds,
                    )
                    score_parts.append(score_gradient)
                    value_parts.append(value_gradient)
```

```
        score_gradient = self._stack_disjoint_gradients(
            score_parts,
            (timesteps, batch, self.attention_heads, tokens, tokens),
        )
        value_head_gradient = self._stack_disjoint_gradients(
            value_parts,
            (timesteps, batch, self.attention_heads, tokens, head_dim),
        )

        query_parts: list[BlockGradient] = []
        key_parts: list[BlockGradient] = []
        part_index = 0
        for timestep in range(timesteps):
            for batch_index in range(batch):
                for head in range(self.attention_heads):
                    query_gradient, transposed_key_gradient = execute_linear_backward_block(
                        self.driver,
                        cache.query_heads[timestep, batch_index, head],
                        cache.key_heads[timestep, batch_index, head].T,
                        BlockGradient(
                            score_gradient.values[timestep, batch_index, head],
                            score_gradient.exponent,
                        ),
                        timeout_seconds=self.layer_timeout_seconds,
                    )
                    query_parts.append(query_gradient)
                    key_parts.append(BlockGradient(
                        transposed_key_gradient.values.T,
                        transposed_key_gradient.exponent,
                    ))
                    part_index += 1
        query_head_gradient = self._stack_disjoint_gradients(
            query_parts,
            (timesteps, batch, self.attention_heads, tokens, head_dim),
        )
        key_head_gradient = self._stack_disjoint_gradients(
            key_parts,
            (timesteps, batch, self.attention_heads, tokens, head_dim),
        )

        def merge_heads(gradient: BlockGradient) -> BlockGradient:
            return BlockGradient(
                gradient.values.transpose(0, 1, 3, 2, 4).reshape(
                    timesteps, batch, tokens, channels),
                gradient.exponent,
            )

        query_gradient = self._lif_backward(cache.query_lif, merge_heads(query_head_gradient))
        key_gradient = self._lif_backward(cache.key_lif, merge_heads(key_head_gradient))
        value_gradient = self._lif_backward(cache.value_lif, merge_heads(value_head_gradient))
        token_input = cache.token_spikes.reshape(timesteps * batch * tokens, channels)
        branch_input_gradients: list[BlockGradient] = []
        parameters: dict[str, ParameterGradient] = {
            f"{prefix}.proj": projection_parameter,
        }
        for name, gradient in (
            ("q", query_gradient),
            ("k", key_gradient),
            ("v", value_gradient),
        ):
            rows = BlockGradient(
                gradient.values.reshape(timesteps * batch * tokens, channels),
                gradient.exponent,
            )
            input_rows, parameter = self._projection_matrix_backward(
                f"{prefix}.{name}", token_input, rows)
            branch_input_gradients.append(input_rows)
            parameters[f"{prefix}.{name}"] = parameter
        token_gradient = add_block_gradients(
            self.driver,
            branch_input_gradients[0],
            branch_input_gradients[1],
            timeout_seconds=self.layer_timeout_seconds,
        )
        token_gradient = add_block_gradients(
            self.driver,
            token_gradient,
            branch_input_gradients[2],
            timeout_seconds=self.layer_timeout_seconds,
        )
        projection_spike_gradient = self._rows_to_channel_first(
            token_gradient, cache.input_shape)
        input_gradient = self._lif_backward(
            cache.projection_lif, projection_spike_gradient)
        return input_gradient, parameters

    def _block_backward(
```

```
        self,
        cache: TransformerBlockCache,
        output_gradient: BlockGradient,
    ) -> tuple[BlockGradient, dict[str, ParameterGradient]]:
        residual_two_gradient = self._gate_saturating_add_gradient(
            output_gradient, cache.after_attention, cache.mlp.output)
        mlp_input_gradient, mlp_parameters = self._mlp_backward(
            cache.mlp, residual_two_gradient)
        after_attention_gradient = add_block_gradients(
            self.driver,
            residual_two_gradient,
            mlp_input_gradient,
            timeout_seconds=self.layer_timeout_seconds,
        )
        residual_one_gradient = self._gate_saturating_add_gradient(
            after_attention_gradient, cache.input_value, cache.attention.output)
        attention_input_gradient, attention_parameters = self._attention_backward(
            cache.attention, residual_one_gradient)
        input_gradient = add_block_gradients(
            self.driver,
            residual_one_gradient,
            attention_input_gradient,
            timeout_seconds=self.layer_timeout_seconds,
        )
        return input_gradient, {**attention_parameters, **mlp_parameters}

    def _sum_timestep_gradients(self, gradient: BlockGradient) -> BlockGradient:
        parts = [
            BlockGradient(gradient.values[timestep], gradient.exponent)
            for timestep in range(gradient.values.shape[0])
        ]
        while len(parts) > 1:
            reduced: list[BlockGradient] = []
            for index in range(0, len(parts) - 1, 2):
                reduced.append(add_block_gradients(
                    self.driver,
                    parts[index],
                    parts[index + 1],
                    timeout_seconds=self.layer_timeout_seconds,
                ))
            if len(parts) & 1:
                reduced.append(parts[-1])
            parts = reduced
        return parts[0]

    def _tokenizer_backward(
        self,
        caches: tuple[TokenizerLayerCache, ...],
        output_gradient: BlockGradient,
    ) -> tuple[BlockGradient, dict[str, ParameterGradient]]:
        gradient = output_gradient
        parameters: dict[str, ParameterGradient] = {}
        for index in range(4, 0, -1):
            cache = caches[index]
            if cache.lif is None:
                raise RuntimeError("nonzero tokenizer layer has no LIF trace")
            flat_gradient = BlockGradient(
                gradient.values.reshape(-1, *gradient.values.shape[2:]),
                gradient.exponent,
            )
            name = f"patch_embed.block{index}"
            projection_input_gradient, parameter = self._projection_conv2d_backward(
                name,
                cache.projection_input,
                flat_gradient,
                padding=1,
            )
            parameters[name] = parameter
            if cache.pooled:
                flat_spikes = cache.lif.spikes.reshape(-1, *cache.lif.spikes.shape[2:])
                spike_gradient, _ = execute_maxpool2d_backward(
                    self.driver,
                    flat_spikes,
                    projection_input_gradient.values,
                    kernel_size=3,
                    stride=2,
                    padding=1,
                    timeout_seconds=self.layer_timeout_seconds,
                )
                projection_input_gradient = BlockGradient(
                    spike_gradient, projection_input_gradient.exponent)
            lif_spike_gradient = BlockGradient(
                projection_input_gradient.values.reshape(cache.lif.spikes.shape),
                projection_input_gradient.exponent,
            )
            gradient = self._lif_backward(cache.lif, lif_spike_gradient)

```

```
        first_output_gradient = self._sum_timestep_gradients(gradient)
        first_cache = caches[0]
        image_gradient, first_parameter = self._projection_conv2d_backward(
            "patch_embed.block0",
            first_cache.projection_input,
            first_output_gradient,
            padding=1,
        )
        parameters["patch_embed.block0"] = first_parameter
        return image_gradient, parameters

    def backward(
        self,
        cache: SpikingformerTrainingCache,
        labels,
    ) -> TrainingBackwardResult:
        """Reverse every exported operator and produce all trainable gradients."""

        labels_array = np.asarray(labels, dtype=np.int64)
        probabilities = execute_softmax_q15(
            self.driver,
            cache.logits_accumulator,
            difference_shift=self.softmax_difference_shift,
            timeout_seconds=self.layer_timeout_seconds,
        )
        head_output_gradient = execute_cross_entropy_gradient_q15(
            self.driver,
            probabilities,
            labels_array,
            timeout_seconds=self.layer_timeout_seconds,
        )
        head = self.state.head
        head_input_gradient, head_parameter = execute_affine_backward_block(
            self.driver,
            cache.head_input,
            head.weight_k_by_n,
            head.bias_accumulator_units,
            head_output_gradient,
            timeout_seconds=self.layer_timeout_seconds,
        )
        clip_mask = (
            (cache.head_preclip >= -128) & (cache.head_preclip <= 127)
        ).astype(np.int64)
        head_input_values = execute_vector_tensor(
            self.driver,
            VectorOperation.SELECT_NONZERO,
            clip_mask,
            head_input_gradient.values,
            timeout_seconds=self.layer_timeout_seconds,
        )
        head_input_gradient = BlockGradient(
            head_input_values, head_input_gradient.exponent)
        divisor = (
            cache.final_value.shape[0]
            * cache.final_value.shape[3]
            * cache.final_value.shape[4]
        )
        denominator = int(round(divisor * self.fixed_scale * self.head_activation_scale))
        if denominator <= 0 or denominator & (denominator - 1):
            raise ValueError("the exported head pooling denominator must be a power of two")
        pooling_shift = denominator.bit_length() - 1
        final_gradient = BlockGradient(
            np.broadcast_to(
                head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis],
                cache.final_value.shape,
            ).copy(),
            head_input_gradient.exponent - pooling_shift,
        )

        parameter_gradients: dict[str, ParameterGradient] = {"head": head_parameter}
        gradient = final_gradient
        for block_cache in reversed(cache.blocks):
            gradient, block_parameters = self._block_backward(block_cache, gradient)
            overlap = set(parameter_gradients) & set(block_parameters)
            if overlap:
                raise RuntimeError(f"duplicate block parameter gradients: {sorted(overlap)}")
            parameter_gradients.update(block_parameters)
        image_gradient, tokenizer_parameters = self._tokenizer_backward(
            cache.tokenizer, gradient)
        overlap = set(parameter_gradients) & set(tokenizer_parameters)
        if overlap:
            raise RuntimeError(f"duplicate tokenizer parameter gradients: {sorted(overlap)}")
        parameter_gradients.update(tokenizer_parameters)
        expected = set(self.state.projection_names) | {"head"}
        if set(parameter_gradients) != expected:
            missing = sorted(expected - set(parameter_gradients))
            extra = sorted(set(parameter_gradients) - expected)
```

```
            raise RuntimeError(f"reverse graph coverage mismatch: missing={missing}, extra={extra}")

        selected = probabilities[np.arange(probabilities.shape[0]), labels_array]
        loss = float(-np.log(np.maximum(selected, 1) / float(1 << 15)).mean())
        return TrainingBackwardResult(
            loss,
            probabilities,
            np.argmax(cache.logits_accumulator, axis=1),
            parameter_gradients,
            image_gradient,
        )

    def train_step(
        self,
        normalized_image,
        labels,
        *,
        learning_rate: float,
        momentum: float = 0.9,
        weight_decay: float = 5e-4,
        gradient_clip_q15: int = 4 << 15,
    ) -> TrainingStepResult:
        """Execute one full forward, reverse, and all-parameter SGDW update."""

        cache = self.forward(normalized_image)
        backward = self.backward(cache, labels)
        self.state.apply_gradients(
            self.driver,
            backward.parameter_gradients,
            learning_rate=learning_rate,
            momentum=momentum,
            weight_decay=weight_decay,
            gradient_clip_q15=gradient_clip_q15,
            timeout_seconds=self.layer_timeout_seconds,
        )
        return TrainingStepResult(
            backward.loss,
            backward.probabilities_q15,
            backward.predicted_class,
            self.state.optimizer_step,
        )
```

44.3 Line annotations

Line 1: `"""Full fixed-point Spikingformer training runtime for the NeuraEdge fabric.` This is executable documentation attached to the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `The processing system owns graph scheduling and DDR tensors. Matrix derivatives,` Inside the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are The: the value named 'the' in this block; processing: the value named 'processing' in this block; system: the value named 'system' in this block.

Line 4: `surrogate-gradient recurrence, pooling scatter, vector arithmetic, softmax, and` Inside the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are surrogate: the value named 'surrogate' in this block; gradient: the signed loss derivative used to update a learned parameter; recurrence: the value named 'recurrence' in this block.

Line 5: `SGDW updates are submitted to programmable-logic engines through the command ABI.` Inside the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are SGDW: the value named 'sgdw' in this block; updates: the value named 'updates' in this block; are: the value named 'are' in this block.

Line 6: `"""` This is executable documentation attached to the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. It states the intended contract that tests

and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 8: `from __future__ import annotations` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `import hashlib` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `import json` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `import math` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `import os` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `from dataclasses import dataclass` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `from pathlib import Path` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `from typing import Any, Mapping, Protocol, Sequence` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `import numpy as np` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `from neuraedge_compiler import compile_spike_gemm, compile_u8_gemm` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `from neuraedge_golden import VectorOperation, round_shift_signed` This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `from neuraedge_protocol import (` This statement continues through source line 32; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `Opcode,` This is continuation line 2 of the 11-line logical statement that begins at source line 22. Its exact fragment is 'Opcode,'. The important operands are Opcode: the value named 'opcode' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `ProgramRecord,` This is continuation line 3 of the 11-line logical statement that begins at source line 22. Its exact fragment is 'ProgramRecord,'. The important operands are ProgramRecord: the value named 'program record' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 25: `clear_status_command,` This is continuation line 4 of the 11-line logical statement that begins at source line 22. Its exact fragment is 'clear_status_command,'. The important operands are clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 26: `log_softmax_command,` This is continuation line 5 of the 11-line logical statement that begins at source line 22. Its exact fragment is 'log_softmax_command,'. The important operands are log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 27: `pack_softmax_beats,` This is continuation line 6 of the 11-line logical statement that begins at source line 22. Its exact fragment is 'pack_softmax_beats,'. The important operands are pack_softmax_beats: the value named 'pack softmax beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 28: `pack_vector_beats,` This is continuation line 7 of the 11-line logical statement that begins at source line 22. Its exact fragment is 'pack_vector_beats,'. The important operands are pack_vector_beats: the value named 'pack vector beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 29: `unpack_softmax_result,` This is continuation line 8 of the 11-line logical statement that begins at source line 22. Its exact fragment is 'unpack_softmax_result,'. The important operands are unpack_softmax_result: the value named 'unpack softmax result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 30: `unpack_vector_result,` This is continuation line 9 of the 11-line logical statement that begins at source line 22. Its exact fragment is 'unpack_vector_result,'. The important operands are unpack_vector_result: the value named 'unpack vector result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 31: `vector_command,` This is continuation line 10 of the 11-line logical statement that begins at source line 22. Its exact fragment is 'vector_command,'. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 32: `)` This is continuation line 11 of the 11-line logical statement that begins at source line 22. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 33: `from neuraedge_training import (` This statement continues through source line 41; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 34: `TrainingProgramExecutor,` This is continuation line 2 of the 9-line logical statement that begins at source line 33. Its exact fragment is 'TrainingProgramExecutor,'. The important operands are TrainingProgramExecutor: the value named 'training program executor' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 35: `compile_affine_backward,` This is continuation line 3 of the 9-line logical statement that begins at source line 33. Its exact fragment is 'compile_affine_backward,'. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 36: `compile_conv2d_backward,` This is continuation line 4 of the 9-line logical statement that begins at source line 33. Its exact fragment is 'compile_conv2d_backward,'. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 37: `compile_linear_backward,` This is continuation line 5 of the 9-line logical statement that begins at source line 33. Its exact fragment is 'compile_linear_backward,'. The important operands are compile_linear_backward: the value named 'compile linear backward' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 38: `execute_lif_bptt,` This is continuation line 6 of the 9-line logical statement that begins at source line 33. Its exact fragment is 'execute_lif_bptt,'. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 39: `execute_maxpool2d_backward,` This is continuation line 7 of the 9-line logical statement that begins at source line 33. Its exact fragment is 'execute_maxpool2d_backward,'. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 40: `execute_sgdw_update,` This is continuation line 8 of the 9-line logical statement that begins at source line 33. Its exact fragment is 'execute_sgdw_update,'. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block. Read together, the complete operation is: This imports the named Python API so

the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 41: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 33. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 42: `from spikingformer_accelerator import (` This statement continues through source line 47; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 43: `SpikingformerInferenceResult,` This is continuation line 2 of the 6-line logical statement that begins at source line 42. Its exact fragment is ‘SpikingformerInferenceResult,‘. The important operands are SpikingformerInferenceResult: the value named ’spikingformer inference result’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 44: `SpikingformerProjectionExecutor,` This is continuation line 3 of the 6-line logical statement that begins at source line 42. Its exact fragment is ‘SpikingformerProjectionExecutor,‘. The important operands are SpikingformerProjectionExecutor: the value named ’spikingformer projection executor’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 45: `binary_max_pool2d_on_accelerator,` This is continuation line 4 of the 6-line logical statement that begins at source line 42. Its exact fragment is ‘binary_max_pool2d_on_accelerator,‘. The important operands are binary_max_pool2d_on_accelerator: the value named ’binary max pool2d on accelerator’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 46: `lif_sequence_with_trace_on_accelerator,` This is continuation line 5 of the 6-line logical statement that begins at source line 42. Its exact fragment is ‘lif_sequence_with_trace_on_accelerator,‘. The important operands are lif_sequence_with_trace_on_accelerator: the value named ’lif sequence with trace on accelerator’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 47: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 42. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 48: `from spikingformer_parameters import (` This statement continues through source line 52; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 49: `HeadParameters,` This is continuation line 2 of the 5-line logical statement that begins at source line 48. Its exact fragment is ‘HeadParameters,‘. The important operands are HeadParameters: the value named ’head parameters’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 50: `ProjectionParameters,` This is continuation line 3 of the 5-line logical statement that begins at source line 48. Its exact fragment is ‘ProjectionParameters,‘. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 51: `SpikingformerParameterArchive,` This is continuation line 4 of the 5-line logical statement that begins at source line 48. Its exact fragment is ‘SpikingformerParameterArchive,‘. The important operands are SpikingformerParameterArchive: the value named ’spikingformer parameter archive’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 52: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 48. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 55: `VECTOR_LANES = 8` This assigns ‘8‘ to ‘VECTOR_LANES ‘ in host memory. Here VECTOR_LANES means one parallel arithmetic element within a vector or tensor datapath. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 56: `MASTER_FORMAT = "neuraedge-spikingformer-trainable-state-v1"` This assigns ‘"neuraedge-spikingformer-trainable-state-v1"‘ to ‘MASTER_FORMAT ‘ in host memory. Here MASTER_FORMAT means the value named ’master format’ in this block. The important operands are neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block; trainable: the value named ’trainable’ in this block. Context: the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 57: `CHECKPOINT_FORMAT = "neuraedge-spikingformer-training-checkpoint-v1"` This assigns ‘"neuraedge-spikingformer-training-checkpoint-v1"‘ to ‘CHECKPOINT_FORMAT ‘ in host memory. Here CHECKPOINT_FORMAT means the value named ’checkpoint format’ in this block. The important operands are neuraedge: the value named ’neuraedge’ in this block; spikingformer: the value named ’spikingformer’ in this block; training: the value named ’training’ in this block. Context: the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 60: `class FullTrainingDriver(TrainingProgramExecutor, Protocol):` This defines class FullTrainingDriver, which groups data and methods for full training driver. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 61: `"""Command interface required by both forward inference and backward update."""` This is executable documentation attached to the class FullTrainingDriver, which handles full training driver. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 64: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10

training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 65: `class BlockGradient:` This defines class BlockGradient, which groups data and methods for block gradient. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 66: `"""Signed integer gradient plus its exact shared power-of-two exponent."""` This is executable documentation attached to the class BlockGradient, which handles block gradient. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 68: `values: np.ndarray` Inside the class BlockGradient, which handles block gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are values: the value named 'values' in this block; ndarray: the value named 'ndarray' in this block.

Line 69: `exponent: int` Inside the class BlockGradient, which handles block gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are exponent: the value named 'exponent' in this block.

Line 71: `def __post_init__(self) -> None:` This starts function ___post_init___. Its indented body performs post init as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 72: `values = np.asarray(self.values, dtype=np.int64)` This assigns ‘np.asarray(self.values, dtype=np.int64)‘ to ‘values ‘ in host memory. Here values means the value named 'values' in this block. The important operands are asarray: the value named 'asarray' in this block; values: the value named 'values' in this block; dtype: the value named 'dtype' in this block. Context: the function ___post_init___, which handles post init.

Line 73: `if values.size and (` This line opens or closes a multi-line Python expression inside the function ___post_init___, which handles post init; the neighboring lines supply its arguments or elements.

Line 74: `values.min() < -(1 << 31) or values.max() > (1 << 31) - 1` Inside the function ___post_init___, which handles post init, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are values: the value named 'values' in this block; min: the value named 'min' in this block; or: the value named 'or' in this block.

Line 75: `):` Inside the function ___post_init___, which handles post init, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 76: `raise OverflowError("block-gradient lanes must fit signed 32-bit")` This rejects an invalid state by raising ‘OverflowError("block-gradient lanes must fit signed 32-bit")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 77: `object.__setattr__(self, "values", values)` This calls ‘object.___setattr___‘ with ‘self, "values", values‘ inside the function ___post_init___, which handles post init. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 78: `if not -127 <= int(self.exponent) <= 127:` This executes the following indented host-code branch only when ‘not -127 <= int(self.exponent) <= 127‘ is true. The important operands are not: the value named 'not' in this block; exponent: the value named 'exponent' in this block. Context: the function ___post_init___, which handles post init.

Line 79: `raise ValueError("block-gradient exponent must be in [-127,127]")` This rejects an invalid state by raising ‘ValueError("block-gradient exponent must be in [-127,127]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 81: `def reshape(self, *shape: int) -> "BlockGradient":` This starts function reshape. Its indented body performs reshape as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 82: `return BlockGradient(self.values.reshape(*shape), self.exponent)` This returns ‘BlockGradient(self.values.re self.exponent)‘ to the caller of the function reshape, which handles reshape, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; reshape: the value named ’reshape’ in this block.

Line 84: `def transpose(self, *axes: int) -> "BlockGradient":` This starts function transpose. Its indented body performs transpose as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 85: `return BlockGradient(self.values.transpose(*axes), self.exponent)` This returns ‘BlockGradient(self.values.t self.exponent)‘ to the caller of the function transpose, which handles transpose, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; transpose: the value named ’transpose’ in this block.

Line 88: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. ‘@dataclass(frozen=True)‘ is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 89: `class ParameterGradient:` This defines class ParameterGradient, which groups data and methods for parameter gradient. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 90: `"""Weight and bias derivatives emitted by one learned affine operator."""` This is executable documentation attached to the class ParameterGradient, which handles parameter gradient. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 92: `weight: BlockGradient` Inside the class ParameterGradient, which handles parameter gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are weight: a learned coefficient multiplied by an activation; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 93: `bias: BlockGradient` Inside the class ParameterGradient, which handles parameter gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are bias: an additive learned offset applied after accumulation; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 96: `@dataclass` This decorator changes how the following class or function is constructed. ‘@dataclass‘ is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 97: `class MasterTensor:` This defines class MasterTensor, which groups data and methods for master tensor. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 98: `"""Signed-16 master value, signed-24 Q15 momentum, and deployment format."""` This is executable documentation attached to the class MasterTensor, which handles master tensor. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 100: `master: np.ndarray` Inside the class MasterTensor, which handles master tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are master: the value named 'master' in this block; ndarray: the value named 'ndarray' in this block.

Line 101: `velocity_q15: np.ndarray` Inside the class MasterTensor, which handles master tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are velocity_q15: the momentum history retained by the SGD optimizer; ndarray: the value named 'ndarray' in this block.

Line 102: `fractional_bits: int` Inside the class MasterTensor, which handles master tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are fractional_bits: the value named 'fractional bits' in this block.

Line 103: `deploy_minimum: int` Inside the class MasterTensor, which handles master tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are deploy_minimum: the value named 'deploy minimum' in this block.

Line 104: `deploy_maximum: int` Inside the class MasterTensor, which handles master tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are deploy_maximum: the value named 'deploy maximum' in this block.

Line 106: `def __post_init__(self) -> None:` This starts function ___post_init___. Its indented body performs post init as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 107: `master = np.asarray(self.master)` This assigns 'np.asarray(self.master)' to 'master ' in host memory. Here master means the value named 'master' in this block. The important operands are asarray: the value named 'asarray' in this block; master: the value named 'master' in this block. Context: the function ___post_init___, which handles post init.

Line 108: `velocity = np.asarray(self.velocity_q15)` This assigns 'np.asarray(self.velocity_q15)' to 'velocity ' in host memory. Here velocity means the momentum history retained by the SGD optimizer. The important operands are asarray: the value named 'asarray' in this block; velocity_q15: the momentum history retained by the SGD optimizer. Context: the function ___post_init___, which handles post init.

Line 109: `if not np.issubdtype(master.dtype, np.integer) or not np.issubdtype(` This line opens or closes a multi-line Python expression inside the function ___post_init___, which handles post init; the neighboring lines supply its arguments or elements.

Line 110: `velocity.dtype, np.integer` Inside the function ___post_init___, which handles post init, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are velocity: the momentum history retained by the SGD optimizer; dtype: the value named 'dtype' in this block; integer: the value named 'integer' in this block.

Line 111: `):` Inside the function ___post_init___, which handles post init, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 112: `raise ValueError("master and velocity tensors must contain integers")` This rejects an invalid state by raising ‘ValueError("master and velocity tensors must contain integers")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 113: `if master.shape != velocity.shape or master.size == 0:` This executes the following indented host-code branch only when ‘master.shape != velocity.shape or master.size == 0‘ is true. The important operands are master: the value named ’master’ in this block; shape: the value named ’shape’ in this block; velocity: the momentum history retained by the SGD optimizer. Context: the function ___post_init___, which handles post init.

Line 114: `raise ValueError("master and velocity tensors must share a nonempty shape")` This rejects an invalid state by raising ‘ValueError("master and velocity tensors must share a nonempty shape")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 115: `if np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1):` This executes the following indented host-code branch only when ‘np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1)‘ is true. The important operands are any: the value named ’any’ in this block; master: the value named ’master’ in this block; or: the value named ’or’ in this block. Context: the function ___post_init___, which handles post init.

Line 116: `raise ValueError("master tensor must fit signed 16-bit")` This rejects an invalid state by raising ‘ValueError("master tensor must fit signed 16-bit")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 117: `if np.any(velocity < -(1 << 23)) or np.any(velocity > (1 << 23) - 1):` This executes the following indented host-code branch only when ‘np.any(velocity < -(1 << 23)) or np.any(velocity > (1 << 23) - 1)‘ is true. The important operands are any: the value named ’any’ in this block; velocity: the momentum history retained by the SGD optimizer; or: the value named ’or’ in this block. Context: the function ___post_init___, which handles post init.

Line 118: `raise ValueError("master velocity must fit signed 24-bit")` This rejects an invalid state by raising ‘ValueError("master velocity must fit signed 24-bit")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 119: `self.master = master.astype(np.int16, copy=False)` This assigns ‘master.astype(np.int16, copy=False)‘ to ‘self.master ‘ in host memory. Here master means the value named ’master’ in this block. The important operands are master: the value named ’master’ in this block; astype: the value named ’astype’ in this block; int16: the value named ’int16’ in this block. Context: the function ___post_init___, which handles post init.

Line 120: `self.velocity_q15 = velocity.astype(np.int32, copy=False)` This assigns ‘velocity.astype(np.int32, copy=False)‘ to ‘self.velocity_q15 ‘ in host memory. Here velocity_q15 means the momentum history retained by the SGD optimizer. The important operands are velocity: the momentum history retained by the SGD optimizer; astype: the value named ’astype’ in this block; int32: the value named ’int32’ in this block. Context: the function ___post_init___, which handles post init.

Line 121: `if not 0 <= self.fractional_bits <= 14:` This executes the following indented host-code branch only when ‘not 0 <= self.fractional_bits <= 14‘ is true. The important operands are not: the value named ’not’ in this block; fractional_bits: the value named ’fractional bits’ in this block. Context: the function ___post_init___, which handles post init.

Line 122: `raise ValueError("master fractional bits must be in [0,14]")` This rejects an invalid state by raising ‘ValueError("master fractional bits must be in [0,14]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 123: `if self.deploy_minimum > self.deploy_maximum:` This executes the following indented host-code branch only when ‘self.deploy_minimum > self.deploy_maximum‘ is true. The important operands are deploy_minimum: the value named ’deploy minimum’ in this block; deploy_maximum: the value named ’deploy maximum’ in this block. Context: the function ___post_init___, which handles post init.

Line 124: `raise ValueError("deployment clamp is inverted")` This rejects an invalid state by raising ‘ValueError("deployment clamp is inverted")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 127: `def _padded_lanes(values: Sequence[int], fill: int = 0) -> tuple[int, ...]:` This starts function _padded_lanes. Its indented body performs padded lanes as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 128: `lanes = tuple(map(int, values))` This assigns ‘tuple(map(int, values))‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block; values: the value named ’values’ in this block. Context: the function _padded_lanes, which handles padded lanes.

Line 129: `if len(lanes) > VECTOR_LANES:` This executes the following indented host-code branch only when ‘len(lanes) > VECTOR_LANES‘ is true. The important operands are len: the value named ’len’ in this block; lanes: one parallel arithmetic element within a vector or tensor datapath; VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function _padded_lanes, which handles padded lanes.

Line 130: `raise ValueError("a vector chunk cannot exceed eight lanes")` This rejects an invalid state by raising ‘ValueError("a vector chunk cannot exceed eight lanes")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 131: `return lanes + (int(fill),) * (VECTOR_LANES - len(lanes))` This returns ‘lanes + (int(fill),) * (VECTOR_LANES - len(lanes))‘ to the caller of the function _padded_lanes, which handles padded lanes, ending this invocation. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath; fill: the value named ’fill’ in this block; VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath.

Line 134: `def execute_vector_tensor(` This starts function execute_vector_tensor. Its indented body performs execute vector tensor as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 135: `driver: FullTrainingDriver,` Inside the function execute_vector_tensor, which handles execute vector tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named ’driver’ in this block; FullTrainingDriver: the value named ’full training driver’ in this block.

Line 136: `operation: VectorOperation,` Inside the function execute_vector_tensor, which handles execute vector tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are operation: the value named ’operation’ in this block; VectorOperation: the value named ’vector operation’ in this block.

Line 137: `operand_a,` Inside the function execute_vector_tensor, which handles execute vector tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are operand_a: the value named ’operand a’ in this block.

Line 138: `operand_b=None,` This assigns ‘None,‘ to ‘operand_b‘ in host memory. Here operand_b means the value named ’operand b’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 139: `*,` Inside the function execute_vector_tensor, which handles execute vector tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 140: `multiplier_q15: int = 0,` This assigns ‘0,‘ to ‘multiplier_q15‘ in host memory. Here multiplier_q15 means the value named ’multiplier q15’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 141: `right_shift: int = 0,` This assigns ‘0,‘ to ‘right_shift‘ in host memory. Here right_shift means the value named ’right shift’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 142: `output_zero_point: int = 0,` This assigns ‘0,‘ to ‘output_zero_point‘ in host memory. Here output_zero_point means the integer code representing real numeric zero in affine quantization. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 143: `threshold: int = 0,` This assigns ‘0,‘ to ‘threshold‘ in host memory. Here threshold means the membrane level at which a neuron emits a spike. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 144: `surrogate_window: int = 0,` This assigns ‘0,‘ to ‘surrogate_window‘ in host memory. Here surrogate_window means the value named ’surrogate window’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 145: `clamp_minimum: int = -(1 << 31),` This assigns ‘-(1 << 31),‘ to ‘clamp_minimum‘ in host memory. Here clamp_minimum means the value named ’clamp minimum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 146: `clamp_maximum: int = (1 << 31) - 1,` This assigns ‘(1 << 31) - 1,‘ to ‘clamp_maximum‘ in host memory. Here clamp_maximum means the value named ’clamp maximum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 147: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 148: `) -> np.ndarray:` Inside the function execute_vector_tensor, which handles execute vector tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are ndarray: the value named ’ndarray’ in this block.

Line 149: `"""Apply one vector command to every element using descriptor batching."""` This is executable documentation attached to the function execute_vector_tensor, which handles execute vector tensor. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 151: `a = np.asarray(operand_a, dtype=np.int64)` This assigns ‘np.asarray(operand_a, dtype=np.int64)‘ to ‘a ‘ in host memory. Here a means the value named ’a’ in this block. The important operands are asarray: the value named ’asarray’ in this block; operand_a: the value named ’operand a’ in this block; dtype: the value named ’dtype’ in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 152: `b = np.zeros_like(a) if operand_b is None else np.asarray(operand_b, dtype=np.int64)` This assigns ‘np.zeros_like(a) if operand_b is None else np.asarray(operand_b, dtype=np.int64)‘ to ‘b ‘ in host memory. Here b means the value named ’b’ in this block. The important operands are zeros_like: the value named ’zeros like’ in this block; a: the value named ’a’ in this block; operand_b: the value named ’operand b’ in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 153: `if a.shape != b.shape:` This executes the following indented host-code branch only when ‘a.shape != b.shape‘ is true. The important operands are a: the value named ’a’ in this block; shape: the value named ’shape’ in this block; b: the value named ’b’ in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 154: `raise ValueError("vector operands must share a shape")` This rejects an invalid state by raising ‘ValueError("vector operands must share a shape")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 155: `for name, value in (("operand_a", a), ("operand_b", b)):` This iterates name, value over ‘(("operand_a", a), ("operand_b", b))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_vector_tensor, which handles execute vector tensor.

Line 156: `if value.size and (` This line opens or closes a multi-line Python expression inside the function execute_vector_tensor, which handles execute vector tensor; the neighboring lines supply its arguments or elements.

Line 157: `value.min() < -(1 << 31) or value.max() > (1 << 31) - 1` Inside the function execute_vector_tensor, which handles execute vector tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are value: the value named ’value’ in this block; min: the value named ’min’ in this block; or: the value named ’or’ in this block.

Line 158: `):` Inside the function execute_vector_tensor, which handles execute vector tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 159: `raise OverflowError(f"{name} does not fit signed 32-bit lanes")` This rejects an invalid state by raising ‘OverflowError(f"{name} does not fit signed 32-bit lanes")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 160: `if a.size == 0:` This executes the following indented host-code branch only when ‘a.size == 0‘ is true. The important operands are a: the value named ’a’ in this block; size: the value named ’size’ in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 161: `return np.empty(a.shape, dtype=np.int64)` This returns ‘np.empty(a.shape, dtype=np.int64)‘ to the caller of the function execute_vector_tensor, which handles execute vector tensor, ending this invocation. The important operands are empty: the value named ’empty’ in this block; a: the value named ’a’ in this block; shape: the value named ’shape’ in this block.

Line 162: `command = vector_command(` This statement continues through source line 169; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘vector_command(int(operation), multiplier_q15=multiplier_q15, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named ’operation’ in this block; multiplier_q15: the value named ’multiplier q15’ in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 163: `int(operation),` This is continuation line 2 of the 8-line logical statement that begins at source line 162. Its exact fragment is ‘int(operation),‘. The important operands are operation: the value named ’operation’ in this block. Read together, the complete operation is: This assigns ‘vector_command(int(operation), multiplier_q15=multiplier_q15, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape,

mode, and numeric parameters; operation: the value named 'operation' in this block; multiplier_q15: the value named 'multiplier q15' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 164: `multiplier_q15=multiplier_q15,` This is continuation line 3 of the 8-line logical statement that begins at source line 162. Its exact fragment is 'multiplier_q15=multiplier_q15,'. The important operands are multiplier_q15: the value named 'multiplier q15' in this block. Read together, the complete operation is: This assigns 'vector_command(int(operation), multiplier_q15=multiplier_q15, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named 'operation' in this block; multiplier_q15: the value named 'multiplier q15' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 165: `right_shift=right_shift,` This is continuation line 4 of the 8-line logical statement that begins at source line 162. Its exact fragment is 'right_shift=right_shift,'. The important operands are right_shift: the value named 'right shift' in this block. Read together, the complete operation is: This assigns 'vector_command(int(operation), multiplier_q15=multiplier_q15, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named 'operation' in this block; multiplier_q15: the value named 'multiplier q15' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 166: `output_zero_point=output_zero_point,` This is continuation line 5 of the 8-line logical statement that begins at source line 162. Its exact fragment is 'output_zero_point=output_zero_point,'. The important operands are output_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns 'vector_command(int(operation), multiplier_q15=multiplier_q15, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named 'operation' in this block; multiplier_q15: the value named 'multiplier q15' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 167: `threshold=threshold,` This is continuation line 6 of the 8-line logical statement that begins at source line 162. Its exact fragment is 'threshold=threshold,'. The important operands are threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns 'vector_command(int(operation), multiplier_q15=multiplier_q15, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named 'operation' in this block; multiplier_q15: the value named 'multiplier q15' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 168: `surrogate_window=surrogate_window,` This is continuation line 7 of the 8-line logical statement that begins at source line 162. Its exact fragment is 'surrogate_window=surrogate_window,'. The important operands are surrogate_window: the value named 'surrogate window' in this block. Read together, the complete operation is: This assigns 'vector_command(int(operation), multiplier_q15=multiplier_q15, right_shift=right_shift, output_zero_point=output_zero_po threshold=threshold, surrogate_window=surrogate_window,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named 'operation' in this block; multiplier_q15: the value named 'multiplier q15' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 169: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 162. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'vector_command(int(operation), multiplier_q15=multiplier_q15, right_shift=right_shift,

output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; operation: the value named 'operation' in this block; multiplier_q15: the value named 'multiplier q15' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 170: `flat_a = a.reshape(-1)` This assigns ‘a.reshape(-1)‘ to ‘flat_a ‘ in host memory. Here flat_a means the value named 'flat a' in this block. The important operands are a: the value named 'a' in this block; reshape: the value named 'reshape' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 171: `flat_b = b.reshape(-1)` This assigns ‘b.reshape(-1)‘ to ‘flat_b ‘ in host memory. Here flat_b means the value named 'flat b' in this block. The important operands are b: the value named 'b' in this block; reshape: the value named 'reshape' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 172: `records: list[ProgramRecord] = []` This assigns ‘[]‘ to ‘records‘ in host memory. Here records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 173: `counts: list[int] = []` This assigns ‘[]‘ to ‘counts‘ in host memory. Here counts means the value named 'counts' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 174: `for start in range(0, flat_a.size, VECTOR_LANES):` This iterates start over ‘range(0, flat_a.size, VECTOR_LANES)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_vector_tensor, which handles execute vector tensor.

Line 175: `stop = min(start + VECTOR_LANES, flat_a.size)` This assigns ‘min(start + VECTOR_LANES, flat_a.size)‘ to ‘stop ‘ in host memory. Here stop means the value named 'stop' in this block. The important operands are min: the value named 'min' in this block; start: the value named 'start' in this block; VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 176: `count = stop - start` This assigns ‘stop - start‘ to ‘count ‘ in host memory. Here count means the value named 'count' in this block. The important operands are stop: the value named 'stop' in this block; start: the value named 'start' in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 177: `records.append(ProgramRecord(` This statement continues through source line 186; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[start:stop]), clamp_minimum=clamp_mini clamp_maximum=clamp_maximum,), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 178: `command,` This is continuation line 2 of the 10-line logical statement that begins at source line 177. Its exact fragment is ‘command,‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[start:stop]), clamp_minimum=clamp_r clamp_maximum=clamp_maximum,), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 179: `pack_vector_beats(` This is continuation line 3 of the 10-line logical statement that begins at source line 177. Its exact fragment is ‘pack_vector_beats(‘. The important operands are pack_vector_beats: the value named 'pack vector beats' in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(

command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[start:stop]), clamp_minimum=clamp_m
clamp_maximum=clamp_maximum,), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 180: `_padded_lanes(flat_a[start:stop]),` This is continuation line 4 of the 10-line logical statement that begins at source line 177. Its exact fragment is ‘_padded_lanes(flat_a[start:stop]),‘. The important operands are _padded_lanes: one parallel arithmetic element within a vector or tensor datapath; flat_a: the value named ’flat a’ in this block; start: the value named ’start’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[start:stop]), clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 181: `_padded_lanes(flat_b[start:stop]),` This is continuation line 5 of the 10-line logical statement that begins at source line 177. Its exact fragment is ‘_padded_lanes(flat_b[start:stop]),‘. The important operands are _padded_lanes: one parallel arithmetic element within a vector or tensor datapath; flat_b: the value named ’flat b’ in this block; start: the value named ’start’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[start:stop]), clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 182: `clamp_minimum=clamp_minimum,` This is continuation line 6 of the 10-line logical statement that begins at source line 177. Its exact fragment is ‘clamp_minimum=clamp_minimum,‘. The important operands are clamp_minimum: the value named ’clamp minimum’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[
clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 183: `clamp_maximum=clamp_maximum,` This is continuation line 7 of the 10-line logical statement that begins at source line 177. Its exact fragment is ‘clamp_maximum=clamp_maximum,‘. The important operands are clamp_maximum: the value named ’clamp maximum’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[
clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 184: `),` This is continuation line 8 of the 10-line logical statement that begins at source line 177. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[start:stop]), clamp_minimum=clamp_minimum, clamp_maximum=cla
), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 185: `4,` This is continuation line 9 of the 10-line logical statement that begins at source line 177. Its exact fragment is ‘4,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[start:stop]), clamp_minimum=clamp_minimum, clamp_maximum=cla
), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 186: `))` This is continuation line 10 of the 10-line logical statement that begins at source line 177. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(flat_a[start:stop]), _padded_lanes(flat_b[start:stop]), clamp_minimum=clamp_minimum, clamp_maximum=cla), 4,)‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 187: `counts.append(count)` This calls ‘counts.append‘ with ‘count‘ inside the function execute_vector_tensor, which handles execute vector tensor. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 188: `words = driver.execute_program(records, timeout_seconds=timeout_seconds)` This assigns ‘driver.execute_pr timeout_seconds=timeout_seconds)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 189: `if len(words) != 4 * len(records):` This executes the following indented host-code branch only when ‘len(words) != 4 * len(records)‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block; records: the number of descriptor records accepted since program start. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 190: `raise RuntimeError(` This statement continues through source line 192; the explanation uses the complete expression rather than only the first visual fragment. This rejects an invalid state by raising ‘RuntimeError(f"vector program returned {len(words)} beats; expected {4 * len(records)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 191: `f"vector program returned {len(words)} beats; expected {4 * len(records)}"` This is continuation line 2 of the 3-line logical statement that begins at source line 190. Its exact fragment is ‘f"vector program returned {len(words)} beats; expected {4 * len(records)}"‘. The important operands are f: the value named ’f’ in this block; vector: the value named ’vector’ in this block; program: the value named ’program’ in this block. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"vector program returned {len(words)} beats; expected {4 * len(records)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 192: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 190. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This rejects an invalid state by raising ‘RuntimeError(f"vector program returned {len(words)} beats; expected {4 * len(records)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 193: `result = np.empty(flat_a.size, dtype=np.int64)` This assigns ‘np.empty(flat_a.size, dtype=np.int64)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are empty: the value named ’empty’ in this block; flat_a: the value named ’flat a’ in this block; size: the value named ’size’ in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 194: `cursor = 0` This assigns ‘0‘ to ‘cursor ‘ in host memory. Here cursor means the value named ’cursor’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 195: `for record_index, count in enumerate(counts):` This iterates record_index, count over ‘enumerate(counts)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_vector_tensor, which handles execute vector tensor.

Line 196: `lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])` This assigns ‘unpack_vector_result(words[4 * record_index:4 * record_index + 4])‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are unpack_vector_result: the value named ’unpack vector result’ in this block; words: the value named ’words’ in this block; record_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 197: `result[cursor:cursor + count] = lanes[:count]` This assigns ‘lanes[:count]‘ to ‘result[cursor‘ in host memory. Here result means the value named ’result’ in this block. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath; count: the value named ’count’ in this block. Context: the function execute_vector_tensor, which handles execute vector tensor.

Line 198: `cursor += count` Inside the function execute_vector_tensor, which handles execute vector tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are cursor: the value named ’cursor’ in this block; count: the value named ’count’ in this block.

Line 199: `return result.reshape(a.shape)` This returns ‘result.reshape(a.shape)‘ to the caller of the function execute_vector_tensor, which handles execute vector tensor, ending this invocation. The important operands are result: the value named ’result’ in this block; reshape: the value named ’reshape’ in this block; a: the value named ’a’ in this block.

Line 202: `def execute_power_of_two_scale(` This starts function execute_power_of_two_scale. Its indented body performs execute power of two scale as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 203: `driver: FullTrainingDriver,` Inside the function execute_power_of_two_scale, which handles execute power of two scale, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named ’driver’ in this block; FullTrainingDriver: the value named ’full training driver’ in this block.

Line 204: `values,` Inside the function execute_power_of_two_scale, which handles execute power of two scale, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are values: the value named ’values’ in this block.

Line 205: `power: int,` Inside the function execute_power_of_two_scale, which handles execute power of two scale, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are power: the value named ’power’ in this block.

Line 206: `*,` Inside the function execute_power_of_two_scale, which handles execute power of two scale, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 207: `clamp_minimum: int = -(1 << 31),` This assigns ‘-(1 << 31),‘ to ‘clamp_minimum‘ in host memory. Here clamp_minimum means the value named ’clamp minimum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 208: `clamp_maximum: int = (1 << 31) - 1,` This assigns ‘(1 << 31) - 1,‘ to ‘clamp_maximum‘ in host memory. Here clamp_maximum means the value named ’clamp maximum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 209: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 210: `) -> np.ndarray:` Inside the function execute_power_of_two_scale, which handles execute power of two scale, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are ndarray: the value named 'ndarray' in this block.

Line 211: `"""Multiply by "2**power" with exact fabric round-to-nearest shifts."""` This is executable documentation attached to the function execute_power_of_two_scale, which handles execute power of two scale. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 213: `result = np.asarray(values, dtype=np.int64)` This assigns 'np.asarray(values, dtype=np.int64)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are asarray: the value named 'asarray' in this block; values: the value named 'values' in this block; dtype: the value named 'dtype' in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 214: `remaining = int(power)` This assigns 'int(power)' to 'remaining ' in host memory. Here remaining means the value named 'remaining' in this block. The important operands are power: the value named 'power' in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 215: `while remaining:` This repeats the following host-code block while 'remaining' remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 216: `step = min(remaining, 14) if remaining > 0 else max(remaining, -16)` This assigns 'min(remaining, 14) if remaining > 0 else max(remaining, -16)' to 'step ' in host memory. Here step means the value named 'step' in this block. The important operands are min: the value named 'min' in this block; remaining: the value named 'remaining' in this block; max: the value named 'max' in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 217: `result = execute_vector_tensor(` This statement continues through source line 226; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 218: `driver,` This is continuation line 2 of the 10-line logical statement that begins at source line 217. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 219: `VectorOperation.FLEX_REQUANTIZE,` This is continuation line 3 of the 10-line logical statement that begins at source line 217. Its exact fragment is 'VectorOperation.FLEX_REQUANTIZE,'. The important operands are VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide

accumulated result back into a bounded activation code. Read together, the complete operation is: This assigns ‘execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 220: `result,` This is continuation line 4 of the 10-line logical statement that begins at source line 217. Its exact fragment is ‘result,‘. The important operands are result: the value named ’result’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 221: `multiplier_q15=1 << 14,` This is continuation line 5 of the 10-line logical statement that begins at source line 217. Its exact fragment is ‘multiplier_q15=1 << 14,‘. The important operands are multiplier_q15: the value named ’multiplier q15’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 222: `right_shift=-step - 1,` This is continuation line 6 of the 10-line logical statement that begins at source line 217. Its exact fragment is ‘right_shift=-step - 1,‘. The important operands are right_shift: the value named ’right shift’ in this block; step: the value named ’step’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 223: `clamp_minimum=clamp_minimum,` This is continuation line 7 of the 10-line logical statement that begins at source line 217. Its exact fragment is ‘clamp_minimum=clamp_minimum,‘. The important operands are clamp_minimum: the value named ’clamp minimum’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 224: `clamp_maximum=clamp_maximum,` This is continuation line 8 of the 10-line logical statement that begins at source line 217. Its exact fragment is ‘clamp_maximum=clamp_maximum,‘. The important operands are clamp_maximum: the value named ’clamp maximum’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 225: `timeout_seconds=timeout_seconds,` This is continuation line 9 of the 10-line logical statement that begins at source line 217. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 226: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 217. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.FLEX_REQUANTIZE, result, multiplier_q15=1 << 14, right_shift=-step - 1, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum, timeout_seconds=timeout_seconds,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_power_of_two_scale, which handles execute power of two scale.

Line 227: `remaining -= step` Inside the function execute_power_of_two_scale, which handles execute power of two scale, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are remaining: the value named 'remaining' in this block; step: the value named 'step' in this block.

Line 228: `return result` This returns 'result' to the caller of the function execute_power_of_two_scale, which handles execute power of two scale, ending this invocation. The important operands are result: the value named 'result' in this block.

Line 231: `def normalize_gradient_s16(` This starts function normalize_gradient_s16. Its indented body performs normalize gradient s16 as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 232: `driver: FullTrainingDriver,` Inside the function normalize_gradient_s16, which handles normalize gradient s16, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 233: `gradient: BlockGradient,` Inside the function normalize_gradient_s16, which handles normalize gradient s16, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 234: `*,` Inside the function normalize_gradient_s16, which handles normalize gradient s16, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 235: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 236: `) -> BlockGradient:` Inside the function normalize_gradient_s16, which handles normalize gradient s16, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter.

Line 237: `"""Right-shift a block only as much as required for signed-16 matrix input."""` This is executable documentation attached to the function normalize_gradient_s16, which handles normalize gradient s16. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 239: `values = gradient.values` This assigns ‘gradient.values‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 240: `maximum = int(np.abs(values).max(initial=0))` This assigns ‘int(np.abs(values).max(initial=0))‘ to ‘maximum ‘ in host memory. Here maximum means the value named ’maximum’ in this block. The important operands are abs: the value named ’abs’ in this block; values: the value named ’values’ in this block; max: the value named ’max’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 241: `shift = 0` This assigns ‘0‘ to ‘shift ‘ in host memory. Here shift means the value named ’shift’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 242: `while maximum > 32767:` This repeats the following host-code block while ‘maximum $>$ 32767‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 243: `maximum = (maximum + 1) // 2` This assigns ‘(maximum + 1) // 2‘ to ‘maximum ‘ in host memory. Here maximum means the value named ’maximum’ in this block. The important operands are maximum: the value named ’maximum’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 244: `shift += 1` Inside the function normalize_gradient_s16, which handles normalize gradient s16, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are shift: the value named ’shift’ in this block.

Line 245: `if shift == 0:` This executes the following indented host-code branch only when ‘shift == 0‘ is true. The important operands are shift: the value named ’shift’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 246: `return BlockGradient(values.astype(np.int64, copy=False), gradient.exponent)` This returns ‘BlockGradient(values.astype(np.int64, copy=False), gradient.exponent)‘ to the caller of the function normalize_gradient_s16, which handles normalize gradient s16, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; astype: the value named ’astype’ in this block.

Line 247: `shifted = execute_power_of_two_scale(` This statement continues through source line 254; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_power_of_two_scale(driver, values, -shift, clamp_minimum=-32768, clamp_maximum=32767, timeout_seconds=timeout_seconds,)‘ to ‘shifted ‘ in host memory. Here shifted means the value named ’shifted’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 248: `driver,` This is continuation line 2 of the 8-line logical statement that begins at source line 247. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, values, -shift, clamp_minimum=-32768, clamp_maximum=32767, timeout_seconds=timeout_seconds,)‘ to ‘shifted ‘ in host memory. Here shifted means the value named ’shifted’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 249: `values,` This is continuation line 3 of the 8-line logical statement that begins at source line 247. Its exact fragment is ‘values,‘. The important operands are values: the value named ’values’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, values, -shift, clamp_minimum=-32768, clamp_maximum=32767, timeout_seconds=timeout_seconds,)‘ to ‘shifted ‘ in host memory. Here shifted means the value named ’shifted’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 250: `-shift,` This is continuation line 4 of the 8-line logical statement that begins at source line 247. Its exact fragment is ‘-shift,‘. The important operands are shift: the value named ’shift’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, values, -shift, clamp_minimum=-32768, clamp_maximum=32767, timeout_seconds=timeout_seconds,)‘ to ‘shifted ‘ in host memory. Here shifted means the value named ’shifted’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 251: `clamp_minimum=-32768,` This is continuation line 5 of the 8-line logical statement that begins at source line 247. Its exact fragment is ‘clamp_minimum=-32768,‘. The important operands are clamp_minimum: the value named ’clamp minimum’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, values, -shift, clamp_minimum=-32768, clamp_maximum=32767, timeout_seconds=timeout_seconds,)‘ to ‘shifted ‘ in host memory. Here shifted means the value named ’shifted’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 252: `clamp_maximum=32767,` This is continuation line 6 of the 8-line logical statement that begins at source line 247. Its exact fragment is ‘clamp_maximum=32767,‘. The important operands are clamp_maximum: the value named ’clamp maximum’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, values, -shift, clamp_minimum=-32768, clamp_maximum=32767, timeout_seconds=timeout_seconds,)‘ to ‘shifted ‘ in host memory. Here shifted means the value named ’shifted’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 253: `timeout_seconds=timeout_seconds,` This is continuation line 7 of the 8-line logical statement that begins at source line 247. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, values, -shift, clamp_minimum=-32768, clamp_maximum=32767, timeout_seconds=timeout_seconds,)‘ to ‘shifted ‘ in host memory. Here shifted means the value named ’shifted’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 254: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 247. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, values, -shift, clamp_minimum=-32768, clamp_maximum=32767, timeout_seconds=timeout_seconds,)‘ to ‘shifted ‘ in host memory. Here shifted means the value named ’shifted’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function normalize_gradient_s16, which handles normalize gradient s16.

Line 255: `return BlockGradient(shifted, gradient.exponent + shift)` This returns ‘BlockGradient(shifted, gradient.exponent + shift)‘ to the caller of the function normalize_gradient_s16, which handles normalize gradient s16, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; shifted: the value named ’shifted’ in this block; gradient: the signed loss derivative used to update a learned parameter.

Line 258: `def add_block_gradients(` This starts function add_block_gradients. Its indented body performs add block gradients as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 259: `driver: FullTrainingDriver,` Inside the function add_block_gradients, which handles add block gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 260: `left: BlockGradient,` Inside the function add_block_gradients, which handles add block gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are left: the value named 'left' in this block; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 261: `right: BlockGradient,` Inside the function add_block_gradients, which handles add block gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are right: the value named 'right' in this block; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 262: `*,` Inside the function add_block_gradients, which handles add block gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 263: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function add_block_gradients, which handles add block gradients.

Line 264: `) -> BlockGradient:` Inside the function add_block_gradients, which handles add block gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter.

Line 265: `"""Align two block exponents and add all lanes in programmable logic."""` This is executable documentation attached to the function add_block_gradients, which handles add block gradients. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 267: `if left.values.shape != right.values.shape:` This executes the following indented host-code branch only when 'left.values.shape != right.values.shape' is true. The important operands are left: the value named 'left' in this block; values: the value named 'values' in this block; shape: the value named 'shape' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 268: `raise ValueError("gradient addends must share a shape")` This rejects an invalid state by raising 'ValueError("gradient addends must share a shape")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 269: `exponent = max(left.exponent, right.exponent)` This assigns 'max(left.exponent, right.exponent)' to 'exponent ' in host memory. Here exponent means the value named 'exponent' in this block. The important operands are max: the value named 'max' in this block; left: the value named 'left' in this block; exponent: the value named 'exponent' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 270: `left_values = left.values` This assigns 'left.values' to 'left_values ' in host memory. Here left_values means the value named 'left values' in this block. The important operands are left: the value named 'left' in this block; values: the value named 'values' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 271: `right_values = right.values` This assigns ‘right.values‘ to ‘right_values ‘ in host memory. Here right_values means the value named ’right values’ in this block. The important operands are right: the value named ’right’ in this block; values: the value named ’values’ in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 272: `if left.exponent != exponent:` This executes the following indented host-code branch only when ‘left.exponent != exponent‘ is true. The important operands are left: the value named ’left’ in this block; exponent: the value named ’exponent’ in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 273: `left_values = execute_power_of_two_scale(` This statement continues through source line 275; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_power_of_two_scale(driver, left_values, left.exponent - exponent, timeout_seconds=timeout_seconds)‘ to ‘left_values ‘ in host memory. Here left_values means the value named ’left values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; left_values: the value named ’left values’ in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 274: `driver, left_values, left.exponent - exponent,` This is continuation line 2 of the 3-line logical statement that begins at source line 273. Its exact fragment is ‘driver, left_values, left.exponent - exponent,‘. The important operands are driver: the value named ’driver’ in this block; left_values: the value named ’left values’ in this block; left: the value named ’left’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, left_values, left.exponent - exponent, timeout_seconds=timeout_seconds)‘ to ‘left_values ‘ in host memory. Here left_values means the value named ’left values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; left_values: the value named ’left values’ in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 275: `timeout_seconds=timeout_seconds)` This is continuation line 3 of the 3-line logical statement that begins at source line 273. Its exact fragment is ‘timeout_seconds=timeout_seconds)‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, left_values, left.exponent - exponent, timeout_seconds=timeout_seconds)‘ to ‘left_values ‘ in host memory. Here left_values means the value named ’left values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; left_values: the value named ’left values’ in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 276: `if right.exponent != exponent:` This executes the following indented host-code branch only when ‘right.exponent != exponent‘ is true. The important operands are right: the value named ’right’ in this block; exponent: the value named ’exponent’ in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 277: `right_values = execute_power_of_two_scale(` This statement continues through source line 279; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_power_of_two_scale(driver, right_values, right.exponent - exponent, timeout_seconds=timeout_seconds)‘ to ‘right_values ‘ in host memory. Here right_values means the value named ’right values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; right_values: the value named ’right values’ in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 278: `driver, right_values, right.exponent - exponent,` This is continuation line 2 of the 3-line logical statement that begins at source line 277. Its exact fragment is ‘driver, right_values, right.exponent - exponent,‘. The important operands are driver: the value named ’driver’ in this block; right_values: the value named ’right values’ in this block; right: the value named ’right’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(driver, right_values, right.exponent - exponent, timeout_seconds=timeout_seconds)‘ to ‘right_values ‘ in host memory. Here right_values means the value named ’right values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver:

the value named 'driver' in this block; right_values: the value named 'right values' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 279: `timeout_seconds=timeout_seconds)` This is continuation line 3 of the 3-line logical statement that begins at source line 277. Its exact fragment is 'timeout_seconds=timeout_seconds)'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, right_values, right.exponent - exponent, timeout_seconds=timeout_seconds)' to 'right_values ' in host memory. Here right_values means the value named 'right values' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; right_values: the value named 'right values' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 280: `result = execute_vector_tensor(` This statement continues through source line 286; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, left_values, right_values, timeout_seconds=timeout_seconds,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 281: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 280. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, left_values, right_values, timeout_seconds=timeout_seconds,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 282: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 280. Its exact fragment is 'VectorOperation.ADD_SAT,'. The important operands are VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named 'add sat' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, left_values, right_values, timeout_seconds=timeout_seconds,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 283: `left_values,` This is continuation line 4 of the 7-line logical statement that begins at source line 280. Its exact fragment is 'left_values,'. The important operands are left_values: the value named 'left values' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, left_values, right_values, timeout_seconds=timeout_seconds,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 284: `right_values,` This is continuation line 5 of the 7-line logical statement that begins at source line 280. Its exact fragment is 'right_values,'. The important operands are right_values: the value named 'right values' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, left_values, right_values, timeout_seconds=timeout_seconds,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 285: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 280. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'exe-

cute_vector_tensor(driver, VectorOperation.ADD_SAT, left_values, right_values, timeout_seconds=timeout_seconds,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 286: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 280. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_vector_tensor(driver, VectorOperation.ADD_SAT, left_values, right_values, timeout_seconds=timeout_seconds,)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function add_block_gradients, which handles add block gradients.

Line 287: `return BlockGradient(result, exponent)` This returns ‘BlockGradient(result, exponent)‘ to the caller of the function add_block_gradients, which handles add block gradients, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; result: the value named ’result’ in this block; exponent: the value named ’exponent’ in this block.

Line 290: `def _required_right_shift(bound: int, maximum: int) -> int:` This starts function _required_right_shift. Its indented body performs required right shift as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 291: `if bound < 0 or maximum <= 0:` This executes the following indented host-code branch only when ‘bound < 0 or maximum <= 0‘ is true. The important operands are bound: the value named ’bound’ in this block; or: the value named ’or’ in this block; maximum: the value named ’maximum’ in this block. Context: the function _required_right_shift, which handles required right shift.

Line 292: `raise ValueError("gradient bounds must be nonnegative with a positive limit")` This rejects an invalid state by raising ‘ValueError("gradient bounds must be nonnegative with a positive limit")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 293: `shift = 0` This assigns ‘0‘ to ‘shift ‘ in host memory. Here shift means the value named ’shift’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _required_right_shift, which handles required right shift.

Line 294: `while bound > maximum:` This repeats the following host-code block while ‘bound > maximum‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 295: `bound = (bound + 1) // 2` This assigns ‘(bound + 1) // 2‘ to ‘bound ‘ in host memory. Here bound means the value named ’bound’ in this block. The important operands are bound: the value named ’bound’ in this block. Context: the function _required_right_shift, which handles required right shift.

Line 296: `shift += 1` Inside the function _required_right_shift, which handles required right shift, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are shift: the value named ’shift’ in this block.

Line 297: `if shift > 31:` This executes the following indented host-code branch only when ‘shift > 31‘ is true. The important operands are shift: the value named ’shift’ in this block. Context: the function _required_right_shift, which handles required right shift.

Line 298: `raise OverflowError("a gradient reduction requires more than 31 shift bits")` This rejects an invalid state by raising ‘OverflowError("a gradient reduction requires more than 31 shift bits")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 299: `return shift` This returns ‘shift‘ to the caller of the function _required_right_shift, which handles required right shift, ending this invocation. The important operands are shift: the value named ’shift’ in this block.

Line 302: `def backward_requantize_projection(` This starts function backward_requantize_projection. Its indented body performs backward requantize projection as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 303: `driver: FullTrainingDriver,` Inside the function backward_requantize_projection, which handles backward requantize projection, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named ’driver’ in this block; FullTrainingDriver: the value named ’full training driver’ in this block.

Line 304: `output_gradient: BlockGradient,` Inside the function backward_requantize_projection, which handles backward requantize projection, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 305: `parameters: ProjectionParameters,` Inside the function backward_requantize_projection, which handles backward requantize projection, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are parameters: the value named ’parameters’ in this block; ProjectionParameters: the value named ’projection parameters’ in this block.

Line 306: `*,` Inside the function backward_requantize_projection, which handles backward requantize projection, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 307: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 308: `) -> BlockGradient:` Inside the function backward_requantize_projection, which handles backward requantize projection, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter.

Line 309: `"""Differentiate the exported per-channel folded-BN requantizer with STE."""` This is executable documentation attached to the function backward_requantize_projection, which handles backward requantize projection. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 311: `values = output_gradient.values` This assigns ‘output_gradient.values‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 312: `channels = parameters.weight_k_by_n.shape[1]` This assigns ‘parameters.weight_k_by_n.shape[1]‘ to ‘channels ‘ in host memory. Here channels means the value named ’channels’ in this block. The important operands are parameters: the value named ’parameters’ in this block; weight_k_by_n: a learned coefficient multiplied by an activation; shape: the value named ’shape’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 313: `if values.ndim != 2 or values.shape[1] != channels:` This executes the following indented host-code branch only when ‘values.ndim != 2 or values.shape[1] != channels‘ is true. The important operands are values: the value named ’values’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 314: `raise ValueError("projection output gradient must have shape [rows,channels]")` This rejects an invalid state by raising ‘ValueError("projection output gradient must have shape [rows,channels]")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 315: `shifts = parameters.binary_point_shift.astype(np.int64)` This assigns ‘parameters.binary_point_shift.astype to ‘shifts ‘ in host memory. Here shifts means the value named ’shifts’ in this block. The important operands are parameters: the value named ’parameters’ in this block; binary_point_shift: the value named ’binary point shift’ in this block; astype: the value named ’astype’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 316: `multipliers = parameters.multiplier_q15.astype(np.int64)` This assigns ‘parameters.multiplier_q15.astype(np to ‘multipliers ‘ in host memory. Here multipliers means the value named ’multipliers’ in this block. The important operands are parameters: the value named ’parameters’ in this block; multiplier_q15: the value named ’multiplier q15’ in this block; astype: the value named ’astype’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 317: `boost_minimum = max(int(shift) - 15 for shift in shifts)` This assigns ‘max(int(shift) - 15 for shift in shifts)‘ to ‘boost_minimum ‘ in host memory. Here boost_minimum means the value named ’boost minimum’ in this block. The important operands are max: the value named ’max’ in this block; shift: the value named ’shift’ in this block; for: the value named ’for’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 318: `boost_maximum = min(int(shift) + 15 for shift in shifts)` This assigns ‘min(int(shift) + 15 for shift in shifts)‘ to ‘boost_maximum ‘ in host memory. Here boost_maximum means the value named ’boost maximum’ in this block. The important operands are min: the value named ’min’ in this block; shift: the value named ’shift’ in this block; for: the value named ’for’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 319: `selected_boost: int | None = None` This assigns ‘None‘ to ‘selected_boost‘ in host memory. Here selected_boost means the value named ’selected boost’ in this block. The important operands are None: the value named ’none’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 320: `for boost in range(boost_maximum, boost_minimum - 1, -1):` This iterates boost over ‘range(boost_maximum, boost_minimum - 1, -1)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function backward_requantize_projection, which handles backward requantize projection.

Line 321: `represented_maximum = 0` This assigns ‘0‘ to ‘represented_maximum ‘ in host memory. Here represented_maximum means the value named ’represented maximum’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 322: `for channel in range(channels):` This iterates channel over ‘range(channels)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function backward_requantize_projection, which handles backward requantize projection.

Line 323: `product = values[:, channel] * int(multipliers[channel])` This assigns ‘values[:, channel] * int(multipliers[cha to ‘product ‘ in host memory. Here product means the value named ’product’ in this block. The important operands are values: the value named ’values’ in this block; channel: the value named ’channel’ in this block; multipliers: the

value named 'multipliers' in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 324: `represented = round_shift_signed(` This statement continues through source line 325; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'round_shift_signed(product, 15 + int(shifts[channel]) - boost)' to 'represented ' in host memory. Here represented means the value named 'represented' in this block. The important operands are round_shift_signed: the value named 'round shift signed' in this block; product: the value named 'product' in this block; shifts: the value named 'shifts' in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 325: `product, 15 + int(shifts[channel]) - boost)` This is continuation line 2 of the 2-line logical statement that begins at source line 324. Its exact fragment is 'product, 15 + int(shifts[channel]) - boost)'. The important operands are product: the value named 'product' in this block; shifts: the value named 'shifts' in this block; channel: the value named 'channel' in this block. Read together, the complete operation is: This assigns 'round_shift_signed(product, 15 + int(shifts[channel]) - boost)' to 'represented ' in host memory. Here represented means the value named 'represented' in this block. The important operands are round_shift_signed: the value named 'round shift signed' in this block; product: the value named 'product' in this block; shifts: the value named 'shifts' in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 326: `represented_maximum = max(` This statement continues through source line 327; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'max(represented_maximum, int(np.abs(represented).ma
to 'represented_maximum ' in host memory. Here represented_maximum means the value named 'represented maximum' in this block. The important operands are max: the value named 'max' in this block; represented_maximum: the value named 'represented maximum' in this block; abs: the value named 'abs' in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 327: `represented_maximum, int(np.abs(represented).max(initial=0)))` This is continuation line 2 of the 2-line logical statement that begins at source line 326. Its exact fragment is 'represented_maximum, int(np.abs(represented).max(init
The important operands are represented_maximum: the value named 'represented maximum' in this block; abs: the value named 'abs' in this block; represented: the value named 'represented' in this block. Read together, the complete operation is: This assigns 'max(represented_maximum, int(np.abs(represented).max(initial=0)))' to 'represented_maximum ' in host memory. Here represented_maximum means the value named 'represented maximum' in this block. The important operands are max: the value named 'max' in this block; represented_maximum: the value named 'represented maximum' in this block; abs: the value named 'abs' in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 328: `if represented_maximum <= 32767:` This executes the following indented host-code branch only when 'represented_maximum <= 32767' is true. The important operands are represented_maximum: the value named 'represented maximum' in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 329: `selected_boost = boost` This assigns 'boost' to 'selected_boost ' in host memory. Here selected_boost means the value named 'selected boost' in this block. The important operands are boost: the value named 'boost' in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 330: `break` Inside the function backward_requantize_projection, which handles backward requantize projection, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are break: the value named 'break' in this block.

Line 331: `if selected_boost is None:` This executes the following indented host-code branch only when 'selected_boost is None' is true. The important operands are selected_boost: the value named 'selected boost' in this block; is: the value named 'is' in this block; None: the value named 'none' in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 332: `raise OverflowError("projection derivative cannot be represented by signed-16 lanes")` This rejects an invalid state by raising ‘OverflowError("projection derivative cannot be represented by signed-16 lanes")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 334: `rows = values.shape[0]` This assigns ‘values.shape[0]‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 335: `records: list[ProgramRecord] = []` This assigns ‘[]‘ to ‘records‘ in host memory. Here records means the number of descriptor records accepted since program start. The expression is a literal or structural operator and has no hidden software state. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 336: `bindings: list[tuple[int, int, int]] = []` This assigns ‘[]‘ to ‘bindings‘ in host memory. Here bindings means the value named ’bindings’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 337: `zeros = (0,) * VECTOR_LANES` This assigns ‘(0,) * VECTOR_LANES‘ to ‘zeros ‘ in host memory. Here zeros means the value named ’zeros’ in this block. The important operands are VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 338: `for channel in range(channels):` This iterates channel over ‘range(channels)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function backward_requantize_projection, which handles backward requantize projection.

Line 339: `command = vector_command(` This statement continues through source line 343; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘vector_command(int(VectorOperation.FLEX_REQUA multiplier_q15=int(multipliers[channel]), right_shift=int(shifts[channel]) - selected_boost,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 340: `int(VectorOperation.FLEX_REQUANTIZE),` This is continuation line 2 of the 5-line logical statement that begins at source line 339. Its exact fragment is ‘int(VectorOperation.FLEX_REQUANTIZE),‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This assigns ‘vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=int(multipliers[channel]), right_shift=int(shifts[channel]) - selected_boost,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 341: `multiplier_q15=int(multipliers[channel]),` This is continuation line 3 of the 5-line logical statement that begins at source line 339. Its exact fragment is ‘multiplier_q15=int(multipliers[channel]),‘. The important operands are multiplier_q15: the value named ’multiplier q15’ in this block; multipliers: the value named ’multipliers’ in this block; channel: the value named ’channel’ in this block. Read together, the complete operation is: This assigns ‘vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=int(multipliers[channel]), right_shift=int(shifts[channel]) - selected_boost,)‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command:

the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 342: `right_shift=int(shifts[channel]) - selected_boost,` This is continuation line 4 of the 5-line logical statement that begins at source line 339. Its exact fragment is 'right_shift=int(shifts[channel]) - selected_boost,'. The important operands are right_shift: the value named 'right shift' in this block; shifts: the value named 'shifts' in this block; channel: the value named 'channel' in this block. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=int(multipliers[channel]), right_shift=int(shifts[chan - selected_boost,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 343: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 339. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'vector_command(int(VectorOperation.FLEX_REQUANTIZE), multiplier_q15=int(multipliers[channel]), right_shift=int(shifts[channel]) - selected_boost,)' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 344: `for row_start in range(0, rows, VECTOR_LANES):` This iterates row_start over 'range(0, rows, VECTOR_LANES)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function backward_requantize_projection, which handles backward requantize projection.

Line 345: `row_stop = min(row_start + VECTOR_LANES, rows)` This assigns 'min(row_start + VECTOR_LANES, rows)' to 'row_stop ' in host memory. Here row_stop means one output-row position processed by the tensor tile. The important operands are min: the value named 'min' in this block; row_start: one output-row position processed by the tensor tile; VECTOR_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 346: `records.append(ProgramRecord(` This statement continues through source line 355; the explanation uses the complete expression rather than only the first visual fragment. This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(_padded_lanes(values[row_start:row_stop, channel]), zeros, clamp_minimum=-32768, clamp_maximum=32767,), 4,)' inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 347: `command,` This is continuation line 2 of the 10-line logical statement that begins at source line 346. Its exact fragment is 'command,'. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(_padded_lanes(values[row_start:row_stop, channel]), zeros, clamp_minimum=-32768, clamp_maximum=32767,), 4,)' inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 348: `pack_vector_beats(` This is continuation line 3 of the 10-line logical statement that begins at source line 346. Its exact fragment is 'pack_vector_beats('. The important operands are pack_vector_beats: the value named 'pack vector beats' in this block. Read together, the complete operation is: This calls 'records.append' with 'ProgramRecord(command, pack_vector_beats(_padded_lanes(values[row_start:row_stop, channel]), zeros, clamp_minimum=-32768,

clamp_maximum=32767,), 4,)‘ inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 349: `_padded_lanes(values[row_start:row_stop, channel]),` This is continuation line 4 of the 10-line logical statement that begins at source line 346. Its exact fragment is ‘_padded_lanes(values[row_start:row_stop, channel]),‘. The important operands are _padded_lanes: one parallel arithmetic element within a vector or tensor datapath; values: the value named ’values’ in this block; row_start: one output-row position processed by the tensor tile. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(values[row_start:row_stop, channel]), zeros, clamp_minimum=-32768, clamp_maximum=32767,), 4,)‘ inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 350: `zeros,` This is continuation line 5 of the 10-line logical statement that begins at source line 346. Its exact fragment is ‘zeros,‘. The important operands are zeros: the value named ’zeros’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(values[row_start:row channel]), zeros, clamp_minimum=-32768, clamp_maximum=32767,), 4,)‘ inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 351: `clamp_minimum=-32768,` This is continuation line 6 of the 10-line logical statement that begins at source line 346. Its exact fragment is ‘clamp_minimum=-32768,‘. The important operands are clamp_minimum: the value named ’clamp minimum’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(values[row_start:row_stop, channel]), zeros, clamp_minimum=-32768, clamp_maximum=32767,), 4,)‘ inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 352: `clamp_maximum=32767,` This is continuation line 7 of the 10-line logical statement that begins at source line 346. Its exact fragment is ‘clamp_maximum=32767,‘. The important operands are clamp_maximum: the value named ’clamp maximum’ in this block. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(values[row_start:row_stop, channel]), zeros, clamp_minimum=-32768, clamp_maximum=32767,), 4,)‘ inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 353: `),` This is continuation line 8 of the 10-line logical statement that begins at source line 346. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(values[row_start:row_stop, channel]), zeros, clamp_minimum=-32768, clamp_maximum=32767,), 4,)‘ inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 354: `4,` This is continuation line 9 of the 10-line logical statement that begins at source line 346. Its exact fragment is ‘4,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(_padded_lanes(values[row_start:row_stop, channel]), zeros, clamp_minimum=-32768, clamp_maximum=32767,), 4,)‘ inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 355: `))` This is continuation line 10 of the 10-line logical statement that begins at source line 346. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘records.append‘ with ‘ProgramRecord(command, pack_vector_beats(

_padded_lanes(values[row_start:row_stop, channel]), zeros, clamp_minimum=-32768, clamp_maximum=32767,), 4,)‘ inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 356: `bindings.append((row_start, row_stop, channel))` This calls ‘bindings.append‘ with ‘(row_start, row_stop, channel)‘ inside the function backward_requantize_projection, which handles backward requantize projection. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 357: `words = driver.execute_program(records, timeout_seconds=timeout_seconds)` This assigns ‘driver.execute_pr timeout_seconds=timeout_seconds)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 358: `if len(words) != 4 * len(records):` This executes the following indented host-code branch only when ‘len(words) != 4 * len(records)‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block; records: the number of descriptor records accepted since program start. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 359: `raise RuntimeError("projection derivative returned an invalid stream length")` This rejects an invalid state by raising ‘RuntimeError("projection derivative returned an invalid stream length")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 360: `result = np.zeros(values.shape, dtype=np.int64)` This assigns ‘np.zeros(values.shape, dtype=np.int64)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are zeros: the value named ’zeros’ in this block; values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 361: `for record_index, (row_start, row_stop, channel) in enumerate(bindings):` This iterates record_index, (row_start, row_stop, channel) over ‘enumerate(bindings)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function backward_requantize_projection, which handles backward requantize projection.

Line 362: `lanes = unpack_vector_result(words[4 * record_index:4 * record_index + 4])` This assigns ‘unpack_vector_result(words[4 * record_index:4 * record_index + 4])‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are unpack_vector_result: the value named ’unpack vector result’ in this block; words: the value named ’words’ in this block; record_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 363: `result[row_start:row_stop, channel] = lanes[:row_stop - row_start]` This assigns ‘lanes[:row_stop - row_start]‘ to ‘result[row_start‘ in host memory. Here result means the value named ’result’ in this block. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath; row_stop: one output-row position processed by the tensor tile; row_start: one output-row position processed by the tensor tile. Context: the function backward_requantize_projection, which handles backward requantize projection.

Line 364: `return BlockGradient(result, output_gradient.exponent - selected_boost)` This returns ‘BlockGradient(result, output_gradient.exponent - selected_boost)‘ to the caller of the function backward_requantize_projection, which handles backward requantize projection, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; result: the value named ’result’ in this block; output_gradient: the signed loss derivative used to update a learned parameter.

Line 367: `def execute_linear_backward_block(` This starts function execute_linear_backward_block. Its indented body performs execute linear backward block as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 368: `driver: FullTrainingDriver,` Inside the function execute_linear_backward_block, which handles execute linear backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 369: `activations,` Inside the function execute_linear_backward_block, which handles execute linear backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 370: `weights,` Inside the function execute_linear_backward_block, which handles execute linear backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are weights: a learned coefficient multiplied by an activation.

Line 371: `output_gradient: BlockGradient,` Inside the function execute_linear_backward_block, which handles execute linear backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 372: `*,` Inside the function execute_linear_backward_block, which handles execute linear backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 373: `timeout_seconds: float = 120.0,` This assigns '120.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 374: `) -> tuple[BlockGradient, BlockGradient]:` Inside the function execute_linear_backward_block, which handles execute linear backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named 'tuple' in this block; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 375: `"""Run a block-floating matrix backward pass with proven saturation bounds."""` This is executable documentation attached to the function execute_linear_backward_block, which handles execute linear backward block. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 377: `x = np.asarray(activations, dtype=np.int64)` This assigns 'np.asarray(activations, dtype=np.int64)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are asarray: the value named 'asarray' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; dtype: the value named 'dtype' in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 378: `w = np.asarray(weights, dtype=np.int64)` This assigns ‘np.asarray(weights, dtype=np.int64)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are asarray: the value named ’asarray’ in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named ’dtype’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 379: `if x.ndim != 2 or w.ndim != 2 or x.shape[1] != w.shape[0]:` This executes the following indented host-code branch only when ‘x.ndim != 2 or w.ndim != 2 or x.shape[1] != w.shape[0]‘ is true. The important operands are x: the value named ’x’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 380: `raise ValueError("linear backward operands do not align")` This rejects an invalid state by raising ‘ValueError("linear backward operands do not align")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 381: `if output_gradient.values.shape != (x.shape[0], w.shape[1]):` This executes the following indented host-code branch only when ‘output_gradient.values.shape != (x.shape[0], w.shape[1])‘ is true. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 382: `raise ValueError("linear output-gradient shape does not match Y=XW")` This rejects an invalid state by raising ‘ValueError("linear output-gradient shape does not match Y=XW")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 383: `dy = normalize_gradient_s16(` This statement continues through source line 384; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘normalize_gradient_s16(driver, output_gradient, timeout_seconds=timeout_seconds)‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are normalize_gradient_s16: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 384: `driver, output_gradient, timeout_seconds=timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 383. Its exact fragment is ‘driver, output_gradient, timeout_seconds=timeout_seconds)‘. The important operands are driver: the value named ’driver’ in this block; output_gradient: the signed loss derivative used to update a learned parameter; timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘normalize_gradient_s16(driver, output_gradient, timeout_seconds=timeout_seconds)‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are normalize_gradient_s16: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 385: `maximum_dy = int(np.abs(dy.values).max(initial=0))` This assigns ‘int(np.abs(dy.values).max(initial=0))‘ to ‘maximum_dy ‘ in host memory. Here maximum_dy means the value named ’maximum dy’ in this block. The important operands are abs: the value named ’abs’ in this block; dy: the value named ’dy’ in this block; values: the value named ’values’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 386: `maximum_x = int(np.abs(x).max(initial=0))` This assigns ‘int(np.abs(x).max(initial=0))‘ to ‘maximum_x ‘ in host memory. Here maximum_x means the value named ’maximum x’ in this block. The important operands are abs: the value named ’abs’ in this block; x: the value named ’x’ in this block; max: the value named ’max’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 387: `maximum_w = int(np.abs(w).max(initial=0))` This assigns ‘int(np.abs(w).max(initial=0))‘ to ‘maximum_w ‘ in host memory. Here maximum_w means the value named ’maximum w’ in this block. The important operands are abs: the value named ’abs’ in this block; w: the value named ’w’ in this block; max: the value named ’max’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 388: `dx_bound = w.shape[1] * maximum_dy * maximum_w` This assigns ‘w.shape[1] * maximum_dy * maximum_w‘ to ‘dx_bound ‘ in host memory. Here dx_bound means the value named ’dx bound’ in this block. The important operands are w: the value named ’w’ in this block; shape: the value named ’shape’ in this block; maximum_dy: the value named ’maximum dy’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 389: `dw_bound = x.shape[0] * maximum_x * maximum_dy` This assigns ‘x.shape[0] * maximum_x * maximum_dy‘ to ‘dw_bound ‘ in host memory. Here dw_bound means the value named ’dw bound’ in this block. The important operands are x: the value named ’x’ in this block; shape: the value named ’shape’ in this block; maximum_x: the value named ’maximum x’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 390: `dx_shift = _required_right_shift(dx_bound, (1 << 31) - 1)` This assigns ‘_required_right_shift(dx_bound, (1 << 31) - 1)‘ to ‘dx_shift ‘ in host memory. Here dx_shift means the value named ’dx shift’ in this block. The important operands are _required_right_shift: the value named ’required right shift’ in this block; dx_bound: the value named ’dx bound’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 391: `dw_shift = _required_right_shift(dw_bound, (1 << 23) - 1)` This assigns ‘_required_right_shift(dw_bound, (1 << 23) - 1)‘ to ‘dw_shift ‘ in host memory. Here dw_shift means the value named ’dw shift’ in this block. The important operands are _required_right_shift: the value named ’required right shift’ in this block; dw_bound: the value named ’dw bound’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 392: `program = compile_linear_backward(` This statement continues through source line 398; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_linear_backward(x, w, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 393: `x,` This is continuation line 2 of the 7-line logical statement that begins at source line 392. Its exact fragment is ‘x,‘. The important operands are x: the value named ’x’ in this block. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 394: `w,` This is continuation line 3 of the 7-line logical statement that begins at source line 392. Its exact fragment is ‘w,‘. The important operands are w: the value named ’w’ in this block. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 395: `dy.values,` This is continuation line 4 of the 7-line logical statement that begins at source line 392. Its exact fragment is ‘dy.values,‘. The important operands are dy: the value named ’dy’ in this block; values: the value named ’values’ in this block. Read together, the complete operation is: This assigns ‘compile_linear_backward(x,

w, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 396: `input_gradient_shift=dx_shift,` This is continuation line 5 of the 7-line logical statement that begins at source line 392. Its exact fragment is ‘input_gradient_shift=dx_shift,‘. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter; dx_shift: the value named ’dx shift’ in this block. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 397: `weight_gradient_shift=dw_shift,` This is continuation line 6 of the 7-line logical statement that begins at source line 392. Its exact fragment is ‘weight_gradient_shift=dw_shift,‘. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation; dw_shift: the value named ’dw shift’ in this block. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 398: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 392. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 399: `dx, dw = program.execute_batched(driver, timeout_seconds=timeout_seconds)` This assigns ‘program.execute_batched(driver, timeout_seconds=timeout_seconds)‘ to ‘dx, dw ‘ in host memory. Here dx means the value named ’dx’ in this block. The important operands are program: the value named ’program’ in this block; execute_batched: the value named ’execute batched’ in this block; driver: the value named ’driver’ in this block. Context: the function execute_linear_backward_block, which handles execute linear backward block.

Line 400: `return (` This statement continues through source line 403; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), BlockGradient(dw, dy.exponent + dw_shift),)‘ to the caller of the function execute_linear_backward_block, which handles execute linear backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block.

Line 401: `BlockGradient(dx, dy.exponent + dx_shift),` This is continuation line 2 of the 4-line logical statement that begins at source line 400. Its exact fragment is ‘BlockGradient(dx, dy.exponent + dx_shift),‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block. Read together, the complete operation is: This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), BlockGradient(dw, dy.exponent + dw_shift),)‘ to the caller of the function execute_linear_backward_block, which handles execute linear backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block.

Line 402: `BlockGradient(dw, dy.exponent + dw_shift),` This is continuation line 3 of the 4-line logical statement that begins at source line 400. Its exact fragment is ‘BlockGradient(dw, dy.exponent + dw_shift),‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dw: the value named ’dw’ in this block; dy: the value named ’dy’ in this block. Read together, the complete operation is: This returns ‘(

BlockGradient(dx, dy.exponent + dx_shift), BlockGradient(dw, dy.exponent + dw_shift),)‘ to the caller of the function execute_linear_backward_block, which handles execute linear backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; dy: the value named 'dy' in this block.

Line 403: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 400. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), BlockGradient(dw, dy.exponent + dw_shift),)‘ to the caller of the function execute_linear_backward_block, which handles execute linear backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; dy: the value named 'dy' in this block.

Line 406: `def execute_affine_backward_block(` This starts function execute_affine_backward_block. Its indented body performs execute affine backward block as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 407: `driver: FullTrainingDriver,` Inside the function execute_affine_backward_block, which handles execute affine backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 408: `activations,` Inside the function execute_affine_backward_block, which handles execute affine backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 409: `weights,` Inside the function execute_affine_backward_block, which handles execute affine backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are weights: a learned coefficient multiplied by an activation.

Line 410: `bias,` Inside the function execute_affine_backward_block, which handles execute affine backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are bias: an additive learned offset applied after accumulation.

Line 411: `output_gradient: BlockGradient,` Inside the function execute_affine_backward_block, which handles execute affine backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 412: `*,` Inside the function execute_affine_backward_block, which handles execute affine backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 413: `timeout_seconds: float = 120.0,` This assigns ‘120.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 414: `) -> tuple[BlockGradient, ParameterGradient]:` Inside the function execute_affine_backward_block, which handles execute affine backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named 'tuple' in this block; BlockGradient: the signed loss derivative used to update a learned parameter; ParameterGradient: the signed loss derivative used to update a learned parameter.

Line 415: `"""Run dX, dW, and dB for Y=XW+b through the matrix backward array."""` This is executable documentation attached to the function execute_affine_backward_block, which handles execute affine backward block. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 417: `x = np.asarray(activations, dtype=np.int64)` This assigns 'np.asarray(activations, dtype=np.int64)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are asarray: the value named 'asarray' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; dtype: the value named 'dtype' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 418: `w = np.asarray(weights, dtype=np.int64)` This assigns 'np.asarray(weights, dtype=np.int64)' to 'w ' in host memory. Here w means the value named 'w' in this block. The important operands are asarray: the value named 'asarray' in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named 'dtype' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 419: `b = np.asarray(bias, dtype=np.int64)` This assigns 'np.asarray(bias, dtype=np.int64)' to 'b ' in host memory. Here b means the value named 'b' in this block. The important operands are asarray: the value named 'asarray' in this block; bias: an additive learned offset applied after accumulation; dtype: the value named 'dtype' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 420: `if x.ndim != 2 or w.ndim != 2 or b.shape != (w.shape[1],):` This executes the following indented host-code branch only when 'x.ndim != 2 or w.ndim != 2 or b.shape != (w.shape[1],)' is true. The important operands are x: the value named 'x' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 421: `raise ValueError("affine operands have inconsistent shapes")` This rejects an invalid state by raising 'ValueError("affine operands have inconsistent shapes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 422: `if x.shape[1] != w.shape[0]:` This executes the following indented host-code branch only when 'x.shape[1] != w.shape[0]' is true. The important operands are x: the value named 'x' in this block; shape: the value named 'shape' in this block; w: the value named 'w' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 423: `raise ValueError("affine activation and weight dimensions do not align")` This rejects an invalid state by raising 'ValueError("affine activation and weight dimensions do not align")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 424: `dy = normalize_gradient_s16(` This statement continues through source line 425; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'normalize_gradient_s16(driver, output_gradient, timeout_seconds=timeout_seconds)' to 'dy ' in host memory. Here dy means the value named 'dy' in this block. The important operands are normalize_gradient_s16: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 425: `driver, output_gradient, timeout_seconds=timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 424. Its exact fragment is 'driver, output_gradient, timeout_seconds=timeout_seconds)'. The important operands are driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter; timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'normalize_gradient_s16(driver, output_gradient, timeout_seconds=timeout_seconds)' to 'dy ' in host memory. Here dy means the value named 'dy' in this block. The important operands are normalize_gradient_s16: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 426: `maximum_dy = int(np.abs(dy.values).max(initial=0))` This assigns 'int(np.abs(dy.values).max(initial=0))' to 'maximum_dy ' in host memory. Here maximum_dy means the value named 'maximum dy' in this block. The important operands are abs: the value named 'abs' in this block; dy: the value named 'dy' in this block; values: the value named 'values' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 427: `maximum_x = max(int(np.abs(x).max(initial=0)), 1)` This assigns 'max(int(np.abs(x).max(initial=0)), 1)' to 'maximum_x ' in host memory. Here maximum_x means the value named 'maximum x' in this block. The important operands are max: the value named 'max' in this block; abs: the value named 'abs' in this block; x: the value named 'x' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 428: `maximum_w = max(int(np.abs(w).max(initial=0)), int(np.abs(b).max(initial=0)))` This assigns 'max(int(np.abs(w).max(initial=0)), int(np.abs(b).max(initial=0)))' to 'maximum_w ' in host memory. Here maximum_w means the value named 'maximum w' in this block. The important operands are max: the value named 'max' in this block; abs: the value named 'abs' in this block; w: the value named 'w' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 429: `dx_bound = w.shape[1] * maximum_dy * maximum_w` This assigns 'w.shape[1] * maximum_dy * maximum_w' to 'dx_bound ' in host memory. Here dx_bound means the value named 'dx bound' in this block. The important operands are w: the value named 'w' in this block; shape: the value named 'shape' in this block; maximum_dy: the value named 'maximum dy' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 430: `dw_bound = x.shape[0] * maximum_x * maximum_dy` This assigns 'x.shape[0] * maximum_x * maximum_dy' to 'dw_bound ' in host memory. Here dw_bound means the value named 'dw bound' in this block. The important operands are x: the value named 'x' in this block; shape: the value named 'shape' in this block; maximum_x: the value named 'maximum x' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 431: `dx_shift = _required_right_shift(dx_bound, (1 << 31) - 1)` This assigns '_required_right_shift(dx_bound, (1 << 31) - 1)' to 'dx_shift ' in host memory. Here dx_shift means the value named 'dx shift' in this block. The important operands are _required_right_shift: the value named 'required right shift' in this block; dx_bound: the value named 'dx bound' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 432: `dw_shift = _required_right_shift(dw_bound, (1 << 23) - 1)` This assigns '_required_right_shift(dw_bound, (1 << 23) - 1)' to 'dw_shift ' in host memory. Here dw_shift means the value named 'dw shift' in this block. The important operands are _required_right_shift: the value named 'required right shift' in this block; dw_bound: the value named 'dw bound' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 433: `program = compile_affine_backward(` This statement continues through source line 440; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'compile_affine_backward(x, w, b, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here

program means the value named 'program' in this block. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 434: `x,` This is continuation line 2 of the 8-line logical statement that begins at source line 433. Its exact fragment is 'x,'. The important operands are x: the value named 'x' in this block. Read together, the complete operation is: This assigns 'compile_affine_backward(x, w, b, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 435: `w,` This is continuation line 3 of the 8-line logical statement that begins at source line 433. Its exact fragment is 'w,'. The important operands are w: the value named 'w' in this block. Read together, the complete operation is: This assigns 'compile_affine_backward(x, w, b, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 436: `b,` This is continuation line 4 of the 8-line logical statement that begins at source line 433. Its exact fragment is 'b,'. The important operands are b: the value named 'b' in this block. Read together, the complete operation is: This assigns 'compile_affine_backward(x, w, b, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 437: `dy.values,` This is continuation line 5 of the 8-line logical statement that begins at source line 433. Its exact fragment is 'dy.values,'. The important operands are dy: the value named 'dy' in this block; values: the value named 'values' in this block. Read together, the complete operation is: This assigns 'compile_affine_backward(x, w, b, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 438: `input_gradient_shift=dx_shift,` This is continuation line 6 of the 8-line logical statement that begins at source line 433. Its exact fragment is 'input_gradient_shift=dx_shift,'. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter; dx_shift: the value named 'dx shift' in this block. Read together, the complete operation is: This assigns 'compile_affine_backward(x, w, b, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 439: `weight_gradient_shift=dw_shift,` This is continuation line 7 of the 8-line logical statement that begins at source line 433. Its exact fragment is 'weight_gradient_shift=dw_shift,'. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation; dw_shift: the value named 'dw shift' in this block. Read together, the complete operation is: This assigns 'compile_affine_backward(x, w, b, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 440: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 433. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together,

the complete operation is: This assigns ‘compile_affine_backward(x, w, b, dy.values, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_affine_backward: the value named ’compile affine backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 441: `dx, dw, db = program.execute_batched(driver, timeout_seconds=timeout_seconds)` This assigns ‘program.execute_batched(driver, timeout_seconds=timeout_seconds)‘ to ‘dx, dw, db ‘ in host memory. Here dx means the value named ’dx’ in this block. The important operands are program: the value named ’program’ in this block; execute_batched: the value named ’execute batched’ in this block; driver: the value named ’driver’ in this block. Context: the function execute_affine_backward_block, which handles execute affine backward block.

Line 442: `return (` This statement continues through source line 448; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), ParameterGradient(BlockGradient(dw, dy.exponent + dw_shift), BlockGradient(db, dy.exponent + dw_shift),),)‘ to the caller of the function execute_affine_backward_block, which handles execute affine backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block.

Line 443: `BlockGradient(dx, dy.exponent + dx_shift),` This is continuation line 2 of the 7-line logical statement that begins at source line 442. Its exact fragment is ‘BlockGradient(dx, dy.exponent + dx_shift),‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block. Read together, the complete operation is: This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), ParameterGradient(BlockGradient(dw, dy.exponent + dw_shift), BlockGradient(db, dy.exponent + dw_shift),),)‘ to the caller of the function execute_affine_backward_block, which handles execute affine backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block.

Line 444: `ParameterGradient(` This is continuation line 3 of the 7-line logical statement that begins at source line 442. Its exact fragment is ‘ParameterGradient(‘. The important operands are ParameterGradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), ParameterGradient(BlockGradient(dw, dy.exponent + dw_shift), BlockGradient(db, dy.exponent + dw_shift),),)‘ to the caller of the function execute_affine_backward_block, which handles execute affine backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block.

Line 445: `BlockGradient(dw, dy.exponent + dw_shift),` This is continuation line 4 of the 7-line logical statement that begins at source line 442. Its exact fragment is ‘BlockGradient(dw, dy.exponent + dw_shift),‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dw: the value named ’dw’ in this block; dy: the value named ’dy’ in this block. Read together, the complete operation is: This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), ParameterGradient(BlockGradient(dw, dy.exponent + dw_shift), BlockGradient(db, dy.exponent + dw_shift),),)‘ to the caller of the function execute_affine_backward_block, which handles execute affine backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block.

Line 446: `BlockGradient(db, dy.exponent + dw_shift),` This is continuation line 5 of the 7-line logical statement that begins at source line 442. Its exact fragment is ‘BlockGradient(db, dy.exponent + dw_shift),‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; db: the value named ’db’ in this block; dy: the value named ’dy’ in this block. Read together, the complete operation is: This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), ParameterGradient(BlockGradient(dw, dy.exponent + dw_shift), BlockGradient(db, dy.exponent + dw_shift),),)‘ to the caller of the function execute_affine_backward_block, which handles execute affine backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block.

Line 447: `),` This is continuation line 6 of the 7-line logical statement that begins at source line 442. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together,

the complete operation is: This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), ParameterGradient(BlockGradient(dw, dy.exponent + dw_shift), BlockGradient(db, dy.exponent + dw_shift),),)‘ to the caller of the function execute_affine_backward_block, which handles execute affine backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block.

Line 448: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 442. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘(BlockGradient(dx, dy.exponent + dx_shift), ParameterGradient(BlockGradient(dw, dy.exponent + dw_shift), BlockGradient(db, dy.exponent + dw_shift),),)‘ to the caller of the function execute_affine_backward_block, which handles execute affine backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named ’dx’ in this block; dy: the value named ’dy’ in this block.

Line 451: `def execute_conv2d_backward_block(` This starts function execute_conv2d_backward_block. Its indented body performs execute conv2d backward block as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 452: `driver: FullTrainingDriver,` Inside the function execute_conv2d_backward_block, which handles execute conv2d backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named ’driver’ in this block; FullTrainingDriver: the value named ’full training driver’ in this block.

Line 453: `activations,` Inside the function execute_conv2d_backward_block, which handles execute conv2d backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 454: `weights,` Inside the function execute_conv2d_backward_block, which handles execute conv2d backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are weights: a learned coefficient multiplied by an activation.

Line 455: `bias,` Inside the function execute_conv2d_backward_block, which handles execute conv2d backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are bias: an additive learned offset applied after accumulation.

Line 456: `output_gradient: BlockGradient,` Inside the function execute_conv2d_backward_block, which handles execute conv2d backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 457: `*,` Inside the function execute_conv2d_backward_block, which handles execute conv2d backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 458: `stride=1,` This assigns ‘1,‘ to ‘stride‘ in host memory. Here stride means the value named ’stride’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 459: `padding=0,` This assigns ‘0,‘ to ‘padding‘ in host memory. Here padding means the value named ’padding’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 460: `dilation=1,` This assigns ‘1,‘ to ‘dilation‘ in host memory. Here dilation means the value named ’dilation’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 461: `groups: int = 1,` This assigns ‘1,‘ to ‘groups‘ in host memory. Here groups means the value named ’groups’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 462: `timeout_seconds: float = 120.0,` This assigns ‘120.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 463: `) -> tuple[BlockGradient, ParameterGradient]:` Inside the function execute_conv2d_backward_block, which handles execute conv2d backward block, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named ’tuple’ in this block; BlockGradient: the signed loss derivative used to update a learned parameter; ParameterGradient: the signed loss derivative used to update a learned parameter.

Line 464: `"""Run grouped NCHW convolution dX/dW/dB with block-floating bounds."""` This is executable documentation attached to the function execute_conv2d_backward_block, which handles execute conv2d backward block. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 466: `x = np.asarray(activations, dtype=np.int64)` This assigns ‘np.asarray(activations, dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are asarray: the value named ’asarray’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; dtype: the value named ’dtype’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 467: `w = np.asarray(weights, dtype=np.int64)` This assigns ‘np.asarray(weights, dtype=np.int64)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are asarray: the value named ’asarray’ in this block; weights: a learned coefficient multiplied by an activation; dtype: the value named ’dtype’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 468: `b = np.asarray(bias, dtype=np.int64)` This assigns ‘np.asarray(bias, dtype=np.int64)‘ to ‘b ‘ in host memory. Here b means the value named ’b’ in this block. The important operands are asarray: the value named ’asarray’ in this block; bias: an additive learned offset applied after accumulation; dtype: the value named ’dtype’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 469: `dy = output_gradient.values` This assigns ‘output_gradient.values‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 470: `if x.ndim != 4 or w.ndim != 4 or dy.ndim != 4:` This executes the following indented host-code branch only when ‘x.ndim != 4 or w.ndim != 4 or dy.ndim != 4‘ is true. The important operands are x: the value named ’x’ in this block; ndim: the value named ’ndim’ in this block; or: the value named ’or’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 471: `raise ValueError("convolution backward requires NCHW/OIHW tensors")` This rejects an invalid state by raising 'ValueError("convolution backward requires NCHW/OIHW tensors")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 472: `normalized = normalize_gradient_s16(` This statement continues through source line 473; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'normalize_gradient_s16(driver, output_gradient, timeout_seconds=timeout_seconds)' to 'normalized ' in host memory. Here normalized means the value named 'normalized' in this block. The important operands are normalize_gradient_s16: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 473: `driver, output_gradient, timeout_seconds=timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 472. Its exact fragment is 'driver, output_gradient, timeout_seconds=timeout_seconds)'. The important operands are driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter; timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'normalize_gradient_s16(driver, output_gradient, timeout_seconds=timeout_seconds)' to 'normalized ' in host memory. Here normalized means the value named 'normalized' in this block. The important operands are normalize_gradient_s16: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 474: `maximum_dy = int(np.abs(normalized.values).max(initial=0))` This assigns 'int(np.abs(normalized.values).max to 'maximum_dy ' in host memory. Here maximum_dy means the value named 'maximum dy' in this block. The important operands are abs: the value named 'abs' in this block; normalized: the value named 'normalized' in this block; values: the value named 'values' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 475: `maximum_x = max(int(np.abs(x).max(initial=0)), 1)` This assigns 'max(int(np.abs(x).max(initial=0)), 1)' to 'maximum_x ' in host memory. Here maximum_x means the value named 'maximum x' in this block. The important operands are max: the value named 'max' in this block; abs: the value named 'abs' in this block; x: the value named 'x' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 476: `maximum_w = int(np.abs(w).max(initial=0))` This assigns 'int(np.abs(w).max(initial=0))' to 'maximum_w ' in host memory. Here maximum_w means the value named 'maximum w' in this block. The important operands are abs: the value named 'abs' in this block; w: the value named 'w' in this block; max: the value named 'max' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 477: `output_channels_per_group = w.shape[0] // groups` This assigns 'w.shape[0] // groups' to 'output_channels_per ' in host memory. Here output_channels_per_group means the value named 'output channels per group' in this block. The important operands are w: the value named 'w' in this block; shape: the value named 'shape' in this block; groups: the value named 'groups' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 478: `kernel_elements = w.shape[2] * w.shape[3]` This assigns 'w.shape[2] * w.shape[3]' to 'kernel_elements ' in host memory. Here kernel_elements means the value named 'kernel elements' in this block. The important operands are w: the value named 'w' in this block; shape: the value named 'shape' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 479: `dx_bound = output_channels_per_group * kernel_elements * maximum_dy * maximum_w` This assigns 'output_channels_per_group * kernel_elements * maximum_dy * maximum_w' to 'dx_bound ' in host memory. Here dx_bound means the value named 'dx bound' in this block. The important operands are output_channels_per_group: the value named 'output channels per group' in this block; kernel_elements: the value named 'kernel elements' in this block; maximum_dy: the value named 'maximum dy' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 480: `reduction_rows = normalized.values.shape[0] * normalized.values.shape[2] * normalized.values.shape[` This assigns ‘normalized.values.shape[0] * normalized.values.shape[2] * normalized.values.shape[3]‘ to ‘reduction_rows ‘ in host memory. Here reduction_rows means one output-row position processed by the tensor tile. The important operands are normalized: the value named ’normalized’ in this block; values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 481: `dw_bound = reduction_rows * maximum_x * maximum_dy` This assigns ‘reduction_rows * maximum_x * maximum_dy‘ to ‘dw_bound ‘ in host memory. Here dw_bound means the value named ’dw bound’ in this block. The important operands are reduction_rows: one output-row position processed by the tensor tile; maximum_x: the value named ’maximum x’ in this block; maximum_dy: the value named ’maximum dy’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 482: `dx_shift = _required_right_shift(dx_bound, (1 << 31) - 1)` This assigns ‘_required_right_shift(dx_bound, (1 << 31) - 1)‘ to ‘dx_shift ‘ in host memory. Here dx_shift means the value named ’dx shift’ in this block. The important operands are _required_right_shift: the value named ’required right shift’ in this block; dx_bound: the value named ’dx bound’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 483: `dw_shift = _required_right_shift(dw_bound, (1 << 23) - 1)` This assigns ‘_required_right_shift(dw_bound, (1 << 23) - 1)‘ to ‘dw_shift ‘ in host memory. Here dw_shift means the value named ’dw shift’ in this block. The important operands are _required_right_shift: the value named ’required right shift’ in this block; dw_bound: the value named ’dw bound’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 484: `program = compile_conv2d_backward(` This statement continues through source line 495; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_conv2d_backward: the value named ’compile conv2d backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 485: `x,` This is continuation line 2 of the 12-line logical statement that begins at source line 484. Its exact fragment is ‘x,‘. The important operands are x: the value named ’x’ in this block. Read together, the complete operation is: This assigns ‘compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_conv2d_backward: the value named ’compile conv2d backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 486: `w,` This is continuation line 3 of the 12-line logical statement that begins at source line 484. Its exact fragment is ‘w,‘. The important operands are w: the value named ’w’ in this block. Read together, the complete operation is: This assigns ‘compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_conv2d_backward: the value named ’compile conv2d backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 487: `b,` This is continuation line 4 of the 12-line logical statement that begins at source line 484. Its exact fragment is ‘b,‘. The important operands are b: the value named ’b’ in this block. Read together, the complete operation is: This assigns ‘compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_conv2d_backward: the value named ’compile conv2d backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 488: `normalized.values,` This is continuation line 5 of the 12-line logical statement that begins at source line 484. Its exact fragment is 'normalized.values,'. The important operands are normalized: the value named 'normalized' in this block; values: the value named 'values' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 489: `stride=stride,` This is continuation line 6 of the 12-line logical statement that begins at source line 484. Its exact fragment is 'stride=stride,'. The important operands are stride: the value named 'stride' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 490: `padding=padding,` This is continuation line 7 of the 12-line logical statement that begins at source line 484. Its exact fragment is 'padding=padding,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_s)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 491: `dilation=dilation,` This is continuation line 8 of the 12-line logical statement that begins at source line 484. Its exact fragment is 'dilation=dilation,'. The important operands are dilation: the value named 'dilation' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 492: `groups=groups,` This is continuation line 9 of the 12-line logical statement that begins at source line 484. Its exact fragment is 'groups=groups,'. The important operands are groups: the value named 'groups' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 493: `input_gradient_shift=dx_shift,` This is continuation line 10 of the 12-line logical statement that begins at source line 484. Its exact fragment is 'input_gradient_shift=dx_shift,'. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter; dx_shift: the value named 'dx shift' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 494: `weight_gradient_shift=dw_shift,` This is continuation line 11 of the 12-line logical statement that begins at source line 484. Its exact fragment is 'weight_gradient_shift=dw_shift,'. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation; dw_shift: the value named 'dw shift' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_s)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 495: `)` This is continuation line 12 of the 12-line logical statement that begins at source line 484. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, b, normalized.values, stride=stride, padding=padding, dilation=dilation, groups=groups, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 496: `dx, dw, db = program.execute_batched(driver, timeout_seconds=timeout_seconds)` This assigns 'program.execute_batched(driver, timeout_seconds=timeout_seconds)' to 'dx, dw, db ' in host memory. Here dx means the value named 'dx' in this block. The important operands are program: the value named 'program' in this block; execute_batched: the value named 'execute batched' in this block; driver: the value named 'driver' in this block. Context: the function execute_conv2d_backward_block, which handles execute conv2d backward block.

Line 497: `return (` This statement continues through source line 503; the explanation uses the complete expression rather than only the first visual fragment. This returns '(BlockGradient(dx, normalized.exponent + dx_shift), ParameterGradient(BlockGradient(dw, normalized.exponent + dw_shift), BlockGradient(db, normalized.exponent + dw_shift),),)' to the caller of the function execute_conv2d_backward_block, which handles execute conv2d backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; normalized: the value named 'normalized' in this block.

Line 498: `BlockGradient(dx, normalized.exponent + dx_shift),` This is continuation line 2 of the 7-line logical statement that begins at source line 497. Its exact fragment is 'BlockGradient(dx, normalized.exponent + dx_shift),'. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; normalized: the value named 'normalized' in this block. Read together, the complete operation is: This returns '(BlockGradient(dx, normalized.exponent + dx_shift), ParameterGradient(BlockGradient(dw, normalized.exponent + dw_shift), BlockGradient(db, normalized.exponent + dw_shift),),)' to the caller of the function execute_conv2d_backward_block, which handles execute conv2d backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; normalized: the value named 'normalized' in this block.

Line 499: `ParameterGradient(` This is continuation line 3 of the 7-line logical statement that begins at source line 497. Its exact fragment is 'ParameterGradient('. The important operands are ParameterGradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This returns '(BlockGradient(dx, normalized.exponent + dx_shift), ParameterGradient(BlockGradient(dw, normalized.exponent + dw_shift), BlockGradient(db, normalized.exponent + dw_shift),),)' to the caller of the function execute_conv2d_backward_block, which handles execute conv2d backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; normalized: the value named 'normalized' in this block.

Line 500: `BlockGradient(dw, normalized.exponent + dw_shift),` This is continuation line 4 of the 7-line logical statement that begins at source line 497. Its exact fragment is 'BlockGradient(dw, normalized.exponent + dw_shift),'. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dw: the value named 'dw' in this block; normalized: the value named 'normalized' in this block. Read together, the complete operation

is: This returns ‘(BlockGradient(dx, normalized.exponent + dx_shift), ParameterGradient(BlockGradient(dw, normalized.exponent + dw_shift), BlockGradient(db, normalized.exponent + dw_shift),),)‘ to the caller of the function execute_conv2d_backward_block, which handles execute conv2d backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; normalized: the value named 'normalized' in this block.

Line 501: `BlockGradient(db, normalized.exponent + dw_shift),` This is continuation line 5 of the 7-line logical statement that begins at source line 497. Its exact fragment is ‘BlockGradient(db, normalized.exponent + dw_shift),‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; db: the value named 'db' in this block; normalized: the value named 'normalized' in this block. Read together, the complete operation is: This returns ‘(BlockGradient(dx, normalized.exponent + dx_shift), ParameterGradient(BlockGradient(dw, normalized.exponent + dw_shift), BlockGradient(db, normalized.exponent + dw_shift),),)‘ to the caller of the function execute_conv2d_backward_block, which handles execute conv2d backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; normalized: the value named 'normalized' in this block.

Line 502: `),` This is continuation line 6 of the 7-line logical statement that begins at source line 497. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘(BlockGradient(dx, normalized.exponent + dx_shift), ParameterGradient(BlockGradient(dw, normalized.exponent + dw_shift), BlockGradient(db, normalized.exponent + dw_shift),),)‘ to the caller of the function execute_conv2d_backward_block, which handles execute conv2d backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; normalized: the value named 'normalized' in this block.

Line 503: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 497. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘(BlockGradient(dx, normalized.exponent + dx_shift), ParameterGradient(BlockGradient(dw, normalized.exponent + dw_shift), BlockGradient(db, normalized.exponent + dw_shift),),)‘ to the caller of the function execute_conv2d_backward_block, which handles execute conv2d backward block, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; dx: the value named 'dx' in this block; normalized: the value named 'normalized' in this block.

Line 506: `def execute_softmax_q15(` This starts function execute_softmax_q15. Its indented body performs execute softmax q15 as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 507: `driver: FullTrainingDriver,` Inside the function execute_softmax_q15, which handles execute softmax q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 508: `logits,` Inside the function execute_softmax_q15, which handles execute softmax q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are logits: the ten unnormalized classifier scores used for argmax.

Line 509: `*,` Inside the function execute_softmax_q15, which handles execute softmax q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 510: `difference_shift: int,` Inside the function execute_softmax_q15, which handles execute softmax q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are difference_shift: the value named 'difference shift' in this block.

Line 511: `timeout_seconds: float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 512: `) -> np.ndarray:` Inside the function execute_softmax_q15, which handles execute softmax q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are ndarray: the value named 'ndarray' in this block.

Line 513: `"""Evaluate the hardware power-of-two softmax for each classifier row."""` This is executable documentation attached to the function execute_softmax_q15, which handles execute softmax q15. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 515: `scores = np.asarray(logits, dtype=np.int64)` This assigns 'np.asarray(logits, dtype=np.int64)' to 'scores ' in host memory. Here scores means the value named 'scores' in this block. The important operands are asarray: the value named 'asarray' in this block; logits: the ten unnormalized classifier scores used for argmax; dtype: the value named 'dtype' in this block. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 516: `if scores.ndim != 2 or not 1 <= scores.shape[1] <= 256:` This executes the following indented host-code branch only when 'scores.ndim != 2 or not 1 <= scores.shape[1] <= 256' is true. The important operands are scores: the value named 'scores' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 517: `raise ValueError("logits must have shape [batch,classes] with 1..256 classes")` This rejects an invalid state by raising 'ValueError("logits must have shape [batch,classes] with 1..256 classes")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 518: `if scores.size and (scores.min() < -(1 << 23) or scores.max() > (1 << 23) - 1):` This executes the following indented host-code branch only when 'scores.size and (scores.min() < -(1 << 23) or scores.max() > (1 << 23) - 1)' is true. The important operands are scores: the value named 'scores' in this block; size: the value named 'size' in this block; and: the value named 'and' in this block. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 519: `raise OverflowError("softmax logits must fit signed 24-bit")` This rejects an invalid state by raising 'OverflowError("softmax logits must fit signed 24-bit")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 520: `records = tuple(` This statement continues through source line 527; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'tuple(ProgramRecord(log_softmax_command(scores.shape[1], difference_shift), pack_softmax_beats(tuple(map(int, row))), scores.shape[1],) for row in scores)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named 'tuple' in this block; ProgramRecord: the value named 'program record' in this block; log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 521: `ProgramRecord(` This is continuation line 2 of the 8-line logical statement that begins at source line 520. Its exact fragment is 'ProgramRecord('. The important operands are ProgramRecord: the value named 'program record' in this block. Read together, the complete operation is: This assigns 'tuple(ProgramRecord(log_softmax_command(scores.shape[1], difference_shift), pack_softmax_beats(tuple(map(int, row))), scores.shape[1],) for row in scores)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named 'tuple' in this block; ProgramRecord: the value named 'program record' in this block; log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 522: `log_softmax_command(scores.shape[1], difference_shift),` This is continuation line 3 of the 8-line logical statement that begins at source line 520. Its exact fragment is 'log_softmax_command(scores.shape[1], difference_shift),'. The important operands are log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; scores: the value named 'scores' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns 'tuple(ProgramRecord(log_softmax_command(scores.shape[1], difference_shift), pack_softmax_beats(tuple(map(int, row))), scores.shape[1],) for row in scores)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named 'tuple' in this block; ProgramRecord: the value named 'program record' in this block; log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 523: `pack_softmax_beats(tuple(map(int, row))),` This is continuation line 4 of the 8-line logical statement that begins at source line 520. Its exact fragment is 'pack_softmax_beats(tuple(map(int, row))),'. The important operands are pack_softmax_beats: the value named 'pack softmax beats' in this block; tuple: the value named 'tuple' in this block; map: the value named 'map' in this block. Read together, the complete operation is: This assigns 'tuple(ProgramRecord(log_softmax_command(scores.shape[1], difference_shift), pack_softmax_beats(tuple(map(int, row))), scores.shape[1],) for row in scores)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named 'tuple' in this block; ProgramRecord: the value named 'program record' in this block; log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 524: `scores.shape[1],` This is continuation line 5 of the 8-line logical statement that begins at source line 520. Its exact fragment is 'scores.shape[1],'. The important operands are scores: the value named 'scores' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns 'tuple(ProgramRecord(log_softmax_command(scores.shape[1], difference_shift), pack_softmax_beats(tuple(map(int, row))), scores.shape[1],) for row in scores)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named 'tuple' in this block; ProgramRecord: the value named 'program record' in this block; log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 525: `)` This is continuation line 6 of the 8-line logical statement that begins at source line 520. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'tuple(ProgramRecord(log_softmax_command(scores.shape[1], difference_shift), pack_softmax_beats(tuple(map(int, row))), scores.shape[1],) for row in scores)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named 'tuple' in this block; ProgramRecord: the value named 'program record' in this block; log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 526: `for row in scores` This is continuation line 7 of the 8-line logical statement that begins at source line 520. Its exact fragment is 'for row in scores'. The important operands are for: the value named 'for' in this block; row: one output-row position processed by the tensor tile; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'tuple(ProgramRecord(log_softmax_command(scores.shape[1], difference_shift), pack_softmax_beats(tuple(map(int, row))), scores.shape[1],) for row in scores)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named 'tuple' in this block; ProgramRecord: the value named 'program record' in this block; log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 527: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 520. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'tuple(ProgramRecord(log_softmax_command(scores.shape[1], difference_shift), pack_softmax_beats(tuple(map(int, row))), scores.shape[1],) for row in scores)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are tuple: the value named 'tuple' in this block; ProgramRecord: the value named 'program record' in this block;

log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 528: `words = driver.execute_program(records, timeout_seconds=timeout_seconds)` This assigns ‘driver.execute_pr timeout_seconds=timeout_seconds)‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are driver: the value named ’driver’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 529: `if len(words) != scores.size:` This executes the following indented host-code branch only when ‘len(words) != scores.size‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block; scores: the value named ’scores’ in this block. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 530: `raise RuntimeError("softmax returned an invalid stream length")` This rejects an invalid state by raising ‘RuntimeError("softmax returned an invalid stream length")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 531: `probabilities = np.zeros(scores.shape, dtype=np.int64)` This assigns ‘np.zeros(scores.shape, dtype=np.int64) to ‘probabilities ‘ in host memory. Here probabilities means the value named ’probabilities’ in this block. The important operands are zeros: the value named ’zeros’ in this block; scores: the value named ’scores’ in this block; shape: the value named ’shape’ in this block. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 532: `for batch_index in range(scores.shape[0]):` This iterates batch_index over ‘range(scores.shape[0])‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_softmax_q15, which handles execute softmax q15.

Line 533: `for class_index in range(scores.shape[1]):` This iterates class_index over ‘range(scores.shape[1])‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_softmax_q15, which handles execute softmax q15.

Line 534: `probability, returned_index, final = unpack_softmax_result(` This statement continues through source line 536; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘unpack_softmax_result(words[batch_index * scores.shape[1] + class_index])‘ to ‘probability, returned_index, final ‘ in host memory. Here probability means the value named ’probability’ in this block. The important operands are unpack_softmax_result: the value named ’unpack softmax result’ in this block; words: the value named ’words’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 535: `words[batch_index * scores.shape[1] + class_index]` This is continuation line 2 of the 3-line logical statement that begins at source line 534. Its exact fragment is ‘words[batch_index * scores.shape[1] + class_index]‘. The important operands are words: the value named ’words’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry; scores: the value named ’scores’ in this block. Read together, the complete operation is: This assigns ‘unpack_softmax_result(words[batch_index * scores.shape[1] + class_index])‘ to ‘probability, returned_index, final ‘ in host memory. Here probability means the value named ’probability’ in this block. The important operands are unpack_softmax_result: the value named ’unpack softmax result’ in this block; words: the value named ’words’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 536: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 534. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘unpack_softmax_result(words[batch_index * scores.shape[1] + class_index])‘ to ‘probability, returned_index, final ‘ in host memory. Here probability means the value named ’probability’ in this block. The important operands are unpack_softmax_result: the value named ’unpack softmax result’ in this block; words:

the value named 'words' in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 537: `if returned_index != class_index or final != (` This line opens or closes a multi-line Python expression inside the function execute_softmax_q15, which handles execute softmax q15; the neighboring lines supply its arguments or elements.

Line 538: `class_index == scores.shape[1] - 1` Inside the function execute_softmax_q15, which handles execute softmax q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are class_index: an integer selecting one tensor element, lane, bank, or state entry; scores: the value named 'scores' in this block; shape: the value named 'shape' in this block.

Line 539: `):` Inside the function execute_softmax_q15, which handles execute softmax q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 540: `raise RuntimeError("softmax result metadata is out of order")` This rejects an invalid state by raising 'RuntimeError("softmax result metadata is out of order")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 541: `probabilities[batch_index, class_index] = probability` This assigns 'probability' to 'probabilities[batch_index, class_index] ' in host memory. Here probabilities means the value named 'probabilities' in this block. The important operands are probability: the value named 'probability' in this block. Context: the function execute_softmax_q15, which handles execute softmax q15.

Line 542: `return probabilities` This returns 'probabilities' to the caller of the function execute_softmax_q15, which handles execute softmax q15, ending this invocation. The important operands are probabilities: the value named 'probabilities' in this block.

Line 545: `def execute_cross_entropy_gradient_q15(` This starts function execute_cross_entropy_gradient_q15. Its indented body performs execute cross entropy gradient q15 as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 546: `driver: FullTrainingDriver,` Inside the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 547: `probabilities_q15,` Inside the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are probabilities_q15: the value named 'probabilities q15' in this block.

Line 548: `labels,` Inside the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are labels: the value named 'labels' in this block.

Line 549: `*,` Inside the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 550: `timeout_seconds:  float = 30.0,` This assigns '30.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 551: `)  -> BlockGradient:` Inside the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter.

Line 552: `"""Form batch-mean "probability - one_hot" entirely with vector RTL."""` This is executable documentation attached to the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 554: `probabilities = np.asarray(probabilities_q15, dtype=np.int64)` This assigns 'np.asarray(probabilities_q15, dtype=np.int64)' to 'probabilities ' in host memory. Here probabilities means the value named 'probabilities' in this block. The important operands are asarray: the value named 'asarray' in this block; probabilities_q15: the value named 'probabilities q15' in this block; dtype: the value named 'dtype' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 555: `targets = np.asarray(labels, dtype=np.int64)` This assigns 'np.asarray(labels, dtype=np.int64)' to 'targets ' in host memory. Here targets means the value named 'targets' in this block. The important operands are asarray: the value named 'asarray' in this block; labels: the value named 'labels' in this block; dtype: the value named 'dtype' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 556: `if probabilities.ndim != 2 or targets.shape != (probabilities.shape[0],):` This executes the following indented host-code branch only when 'probabilities.ndim != 2 or targets.shape != (probabilities.shape[0],)' is true. The important operands are probabilities: the value named 'probabilities' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 557: `raise ValueError("labels must contain one class index per probability row")` This rejects an invalid state by raising 'ValueError("labels must contain one class index per probability row")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 558: `if np.any(targets < 0) or np.any(targets >= probabilities.shape[1]):` This executes the following indented host-code branch only when 'np.any(targets < 0) or np.any(targets >= probabilities.shape[1])' is true. The important operands are any: the value named 'any' in this block; targets: the value named 'targets' in this block; or: the value named 'or' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 559: `raise ValueError("a target class is outside the classifier range")` This rejects an invalid state by raising 'ValueError("a target class is outside the classifier range")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 560: `correction = np.zeros(probabilities.shape, dtype=np.int64)` This assigns 'np.zeros(probabilities.shape, dtype=np.int64)' to 'correction ' in host memory. Here correction means the value named 'correction' in this block. The important operands are zeros: the value named 'zeros' in this block; probabilities: the value named 'probabilities' in this block; shape: the value named 'shape' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 561: `correction[np.arange(probabilities.shape[0]), targets] = -(1 << 15)` Inside the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are correction: the value named 'correction' in this block; arange: the value named 'arange' in this block; probabilities: the value named 'probabilities' in this block.

Line 562: `gradient = execute_vector_tensor(` This statement continues through source line 568; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, probabilities, correction, timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 563: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 562. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, probabilities, correction, timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 564: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 562. Its exact fragment is 'VectorOperation.ADD_SAT,'. The important operands are VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named 'add sat' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, probabilities, correction, timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 565: `probabilities,` This is continuation line 4 of the 7-line logical statement that begins at source line 562. Its exact fragment is 'probabilities,'. The important operands are probabilities: the value named 'probabilities' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, probabilities, correction, timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 566: `correction,` This is continuation line 5 of the 7-line logical statement that begins at source line 562. Its exact fragment is 'correction,'. The important operands are correction: the value named 'correction' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, probabilities, correction, timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 567: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 562. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, probabilities, correction, timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The

important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 568: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 562. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.ADD_SAT, probabilities, correction, timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 569: `if probabilities.shape[0] > 1:` This executes the following indented host-code branch only when 'probabilities.shape[0] > 1' is true. The important operands are probabilities: the value named 'probabilities' in this block; shape: the value named 'shape' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 570: `reciprocal_q15 = int(round((1 << 15) / probabilities.shape[0]))` This assigns 'int(round((1 << 15) / probabilities.shape[0]))' to 'reciprocal_q15 ' in host memory. Here reciprocal_q15 means the value named 'reciprocal q15' in this block. The important operands are round: the value named 'round' in this block; probabilities: the value named 'probabilities' in this block; shape: the value named 'shape' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 571: `gradient = execute_vector_tensor(` This statement continues through source line 577; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_vector_tensor(driver, VectorOperation.MULTIPLY_Q15, gradient, np.full(gradient.shape, reciprocal_q15, dtype=np.int64), timeout_seconds=timeout_sec)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 572: `driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 571. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.MULTIPLY_Q15, gradient, np.full(gradient.shape, reciprocal_q15, dtype=np.int64), timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 573: `VectorOperation.MULTIPLY_Q15,` This is continuation line 3 of the 7-line logical statement that begins at source line 571. Its exact fragment is 'VectorOperation.MULTIPLY_Q15,'. The important operands are VectorOperation: the value named 'vector operation' in this block; MULTIPLY_Q15: the value named 'multiply q15' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.MULTIPLY_Q15, gradient, np.full(gradient.shape, reciprocal_q15, dtype=np.int64), timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 574: `gradient,` This is continuation line 4 of the 7-line logical statement that begins at source line 571. Its exact fragment is 'gradient,'. The important operands are gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.MULTIPLY_Q15, gradient, np.full(gradient.shape, reciprocal_q15, dtype=np.int64), timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The

important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 575: `np.full(gradient.shape, reciprocal_q15, dtype=np.int64),` This is continuation line 5 of the 7-line logical statement that begins at source line 571. Its exact fragment is 'np.full(gradient.shape, reciprocal_q15, dtype=np.int64),'. The important operands are full: the value named 'full' in this block; gradient: the signed loss derivative used to update a learned parameter; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.MULTIPLY_Q15, gradient, np.full(gradient.shape, reciprocal_q15, dtype=np.int64), timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 576: `timeout_seconds=timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 571. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.MULTIPLY_Q15, gradient, np.full(gradient.shape, reciprocal_q15, dtype=np.int64), timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 577: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 571. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.MULTIPLY_Q15, gradient, np.full(gradient.shape, reciprocal_q15, dtype=np.int64), timeout_seconds=timeout_seconds,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15.

Line 578: `return BlockGradient(gradient, -15)` This returns 'BlockGradient(gradient, -15)' to the caller of the function execute_cross_entropy_gradient_q15, which handles execute cross entropy gradient q15, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter.

Line 581: `def _master_fractional_bits(values: np.ndarray, maximum_bits: int) -> int:` This starts function _master_fractional_bits. Its indented body performs master fractional bits as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 582: `maximum = int(np.abs(np.asarray(values, dtype=np.int64)).max(initial=0))` This assigns 'int(np.abs(np.asa dtype=np.int64)).max(initial=0))' to 'maximum ' in host memory. Here maximum means the value named 'maximum' in this block. The important operands are abs: the value named 'abs' in this block; asarray: the value named 'asarray' in this block; values: the value named 'values' in this block. Context: the function _master_fractional_bits, which handles master fractional bits.

Line 583: `fractional = 0` This assigns '0' to 'fractional ' in host memory. Here fractional means the value named 'fractional' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _master_fractional_bits, which handles master fractional bits.

Line 584: `while fractional < maximum_bits and maximum <= (32767 >> (fractional + 1)):` This repeats the following host-code block while ‘fractional < maximum_bits and maximum <= (32767 >> (fractional + 1))‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 585: `fractional += 1` Inside the function _master_fractional_bits, which handles master fractional bits, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are fractional: the value named 'fractional' in this block.

Line 586: `return fractional` This returns ‘fractional‘ to the caller of the function _master_fractional_bits, which handles master fractional bits, ending this invocation. The important operands are fractional: the value named 'fractional' in this block.

Line 589: `def execute_deploy_master(` This starts function execute_deploy_master. Its indented body performs execute deploy master as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 590: `driver: FullTrainingDriver,` Inside the function execute_deploy_master, which handles execute deploy master, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 591: `tensor: MasterTensor,` Inside the function execute_deploy_master, which handles execute deploy master, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tensor: the value named 'tensor' in this block; MasterTensor: the value named 'master tensor' in this block.

Line 592: `*,` Inside the function execute_deploy_master, which handles execute deploy master, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 593: `timeout_seconds: float = 120.0,` This assigns ‘120.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_deploy_master, which handles execute deploy master.

Line 594: `) -> np.ndarray:` Inside the function execute_deploy_master, which handles execute deploy master, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are ndarray: the value named 'ndarray' in this block.

Line 595: `"""Round a signed-16 master tensor and clamp it to its deployment range."""` This is executable documentation attached to the function execute_deploy_master, which handles execute deploy master. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 597: `shifted = execute_power_of_two_scale(` This statement continues through source line 611; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maxim timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeou)‘ to ‘shifted ‘ in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 598: `driver,` This is continuation line 2 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.de clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 599: `tensor.master,` This is continuation line 3 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'tensor.master,'. The important operands are tensor: the value named 'tensor' in this block; master: the value named 'master' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximu timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 600: `-tensor.fractional_bits,` This is continuation line 4 of the 15-line logical statement that begins at source line 597. Its exact fragment is '-tensor.fractional_bits,'. The important operands are tensor: the value named 'tensor' in this block; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximu timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 601: `clamp_minimum=tensor.deploy_minimum,` This is continuation line 5 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'clamp_minimum=tensor.deploy_minimum,'. The important operands are clamp_minimum: the value named 'clamp minimum' in this block; tensor: the value named 'tensor' in this block; deploy_minimum: the value named 'deploy minimum' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximu timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 602: `clamp_maximum=tensor.deploy_maximum,` This is continuation line 6 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'clamp_maximum=tensor.deploy_maximum,'. The important operands are clamp_maximum: the value named 'clamp maximum' in this block; tensor: the value named 'tensor' in this block; deploy_maximum: the value named 'deploy maximum' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximu timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 603: `timeout_seconds=timeout_seconds,` This is continuation line 7 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximu timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 604: `) if tensor.fractional_bits else execute_vector_tensor(` This is continuation line 8 of the 15-line logical statement that begins at source line 597. Its exact fragment is ') if tensor.fractional_bits else execute_vector_tensor('. The important operands are tensor: the value named 'tensor' in this block; fractional_bits: the value named 'fractional bits' in this block; execute_vector_tensor: the value named 'execute vector tensor' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.de clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 605: `driver,` This is continuation line 9 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.de clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 606: `VectorOperation.CLAMP,` This is continuation line 10 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'VectorOperation.CLAMP,'. The important operands are VectorOperation: the value named 'vector operation' in this block; CLAMP: the value named 'clamp' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.de clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 607: `tensor.master,` This is continuation line 11 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'tensor.master,'. The important operands are tensor: the value named 'tensor' in this block; master: the value named 'master' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximu timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 608: `clamp_minimum=tensor.deploy_minimum,` This is continuation line 12 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'clamp_minimum=tensor.deploy_minimum,'. The important operands

are clamp_minimum: the value named 'clamp minimum' in this block; tensor: the value named 'tensor' in this block; deploy_minimum: the value named 'deploy minimum' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximu timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 609: `clamp_maximum=tensor.deploy_maximum,` This is continuation line 13 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'clamp_maximum=tensor.deploy_maximum,'. The important operands are clamp_maximum: the value named 'clamp maximum' in this block; tensor: the value named 'tensor' in this block; deploy_maximum: the value named 'deploy maximum' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximu timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 610: `timeout_seconds=timeout_seconds,` This is continuation line 14 of the 15-line logical statement that begins at source line 597. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.deploy_minimum, clamp_maximu timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 611: `)` This is continuation line 15 of the 15-line logical statement that begins at source line 597. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(driver, tensor.master, -tensor.fractional_bits, clamp_minimum=tensor.deploy_minimum, clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,) if tensor.fractional_bits else execute_vector_tensor(driver, VectorOperation.CLAMP, tensor.master, clamp_minimum=tensor.de clamp_maximum=tensor.deploy_maximum, timeout_seconds=timeout_seconds,)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function execute_deploy_master, which handles execute deploy master.

Line 612: `return shifted.astype(np.int64)` This returns 'shifted.astype(np.int64)' to the caller of the function execute_deploy_master, which handles execute deploy master, ending this invocation. The important operands are shifted: the value named 'shifted' in this block; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block.

Line 615: `def gradient_to_optimizer_q15(` This starts function gradient_to_optimizer_q15. Its indented body performs gradient to optimizer q15 as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 616: `driver: FullTrainingDriver,` Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 617: `gradient: BlockGradient,` Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 618: `*,` Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 619: `clip_q15: int,` Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are clip_q15: the value named 'clip q15' in this block.

Line 620: `timeout_seconds: float = 120.0,` This assigns '120.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 621: `) -> np.ndarray:` Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are ndarray: the value named 'ndarray' in this block.

Line 622: `"""Convert a block gradient to Q15 and apply explicit fabric clipping."""` This is executable documentation attached to the function gradient_to_optimizer_q15, which handles gradient to optimizer q15. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 624: `if not 1 <= clip_q15 <= (1 << 23) - 1:` This executes the following indented host-code branch only when 'not 1 <= clip_q15 <= (1 << 23) - 1' is true. The important operands are not: the value named 'not' in this block; clip_q15: the value named 'clip q15' in this block. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 625: `raise ValueError("clip_q15 must be in [1,8388607]")` This rejects an invalid state by raising 'ValueError("clip_q15 must be in [1,8388607]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 626: `scaled = execute_power_of_two_scale(` This statement continues through source line 631; the explanation uses the complete expression rather than only the first visual fragment. Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are scaled: the real-value spacing represented by one quantized integer step; execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block.

Line 627: `driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 626. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are scaled: the real-value spacing represented by one quantized integer step; execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block.

Line 628: `gradient.values,` This is continuation line 3 of the 6-line logical statement that begins at source line 626. Its exact fragment is 'gradient.values,'. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Read together, the complete operation is: Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are scaled: the real-value spacing represented by one quantized integer step; execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block.

Line 629: `gradient.exponent + 15,` This is continuation line 4 of the 6-line logical statement that begins at source line 626. Its exact fragment is 'gradient.exponent + 15,'. The important operands are gradient: the signed loss derivative used to update a learned parameter; exponent: the value named 'exponent' in this block. Read together, the complete operation is: Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are scaled: the real-value spacing represented by one quantized integer step; execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block.

Line 630: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 626. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are scaled: the real-value spacing represented by one quantized integer step; execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block.

Line 631: `) if gradient.exponent != -15 else gradient.values` This is continuation line 6 of the 6-line logical statement that begins at source line 626. Its exact fragment is ') if gradient.exponent != -15 else gradient.values'. The important operands are gradient: the signed loss derivative used to update a learned parameter; exponent: the value named 'exponent' in this block; values: the value named 'values' in this block. Read together, the complete operation is: Inside the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are scaled: the real-value spacing represented by one quantized integer step; execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block.

Line 632: `clipped = execute_vector_tensor(` This statement continues through source line 639; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_vector_tensor(driver, VectorOperation.CLAMP, scaled, clamp_minimum=-clip_q15, clamp_maximum=clip_q15, timeout_seconds=timeout_seconds,)' to 'clipped ' in host memory. Here clipped means the value named 'clipped' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 633: `driver,` This is continuation line 2 of the 8-line logical statement that begins at source line 632. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.CLAMP, scaled, clamp_minimum=-clip_q15, clamp_maximum=clip_q15, timeout_seconds=timeout_seconds,)' to 'clipped ' in host memory. Here clipped means the value named 'clipped' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 634: `VectorOperation.CLAMP,` This is continuation line 3 of the 8-line logical statement that begins at source line 632. Its exact fragment is 'VectorOperation.CLAMP,'. The important operands are VectorOperation: the value

named 'vector operation' in this block; CLAMP: the value named 'clamp' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.CLAMP, scaled, clamp_minimum=-clip_q15, clamp_maximum=clip_q15, timeout_seconds=timeout_seconds,)' to 'clipped ' in host memory. Here clipped means the value named 'clipped' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 635: `scaled,` This is continuation line 4 of the 8-line logical statement that begins at source line 632. Its exact fragment is 'scaled,'. The important operands are scaled: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.CLAMP, scaled, clamp_minimum=-clip_q15, clamp_maximum=clip_q15, timeout_seconds=timeout_seconds,)' to 'clipped ' in host memory. Here clipped means the value named 'clipped' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 636: `clamp_minimum=-clip_q15,` This is continuation line 5 of the 8-line logical statement that begins at source line 632. Its exact fragment is 'clamp_minimum=-clip_q15,'. The important operands are clamp_minimum: the value named 'clamp minimum' in this block; clip_q15: the value named 'clip q15' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.CLAMP, scaled, clamp_minimum=-clip_q15, clamp_maximum=clip_q15, timeout_seconds=timeout_seconds,)' to 'clipped ' in host memory. Here clipped means the value named 'clipped' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 637: `clamp_maximum=clip_q15,` This is continuation line 6 of the 8-line logical statement that begins at source line 632. Its exact fragment is 'clamp_maximum=clip_q15,'. The important operands are clamp_maximum: the value named 'clamp maximum' in this block; clip_q15: the value named 'clip q15' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.CLAMP, scaled, clamp_minimum=-clip_q15, clamp_maximum=clip_q15, timeout_seconds=timeout_seconds,)' to 'clipped ' in host memory. Here clipped means the value named 'clipped' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 638: `timeout_seconds=timeout_seconds,` This is continuation line 7 of the 8-line logical statement that begins at source line 632. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.CLAMP, scaled, clamp_minimum=-clip_q15, clamp_maximum=clip_q15, timeout_seconds=timeout_seconds,)' to 'clipped ' in host memory. Here clipped means the value named 'clipped' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 639: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 632. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_vector_tensor(driver, VectorOperation.CLAMP, scaled, clamp_minimum=-clip_q15, clamp_maximum=clip_q15, timeout_seconds=timeout_seconds,)' to 'clipped ' in host memory. Here clipped means the value named 'clipped' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function gradient_to_optimizer_q15, which handles gradient to optimizer q15.

Line 640: `return clipped.astype(np.int64)` This returns 'clipped.astype(np.int64)' to the caller of the function gradient_to_optimizer_q15, which handles gradient to optimizer q15, ending this invocation. The important operands are clipped: the value named 'clipped' in this block; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block.

Line 643: `class TrainableSpikingformerState:` This defines class TrainableSpikingformerState, which groups data and methods for trainable spikingformer state. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 644: `"""Mutable master parameters implementing the read-only archive interface."""` This is executable documentation attached to the class TrainableSpikingformerState, which handles trainable spikingformer state. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 646: `def __init__(self, archive: SpikingformerParameterArchive) -> None:` This starts function ___init___. Its indented body performs init as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 647: `self.manifest = archive.manifest` This assigns ‘archive.manifest‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named 'manifest' in this block. The important operands are archive: the value named 'archive' in this block; manifest: the value named 'manifest' in this block. Context: the function ___init___, which handles init.

Line 648: `self.base_binary_sha256 = str(archive.manifest["binary_sha256"])` This assigns ‘str(archive.manifest["binary to ‘self.base_binary_sha256 ‘ in host memory. Here base_binary_sha256 means the value named 'base binary sha256' in this block. The important operands are str: the value named 'str' in this block; archive: the value named 'archive' in this block; manifest: the value named 'manifest' in this block. Context: the function ___init___, which handles init.

Line 649: `self._projection_static: dict[str, ProjectionParameters] = {}` This assigns ‘{}‘ to ‘self._projection_static in host memory. Here _projection_static means the value named 'projection static' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 650: `self._projection_weight: dict[str, MasterTensor] = {}` This assigns ‘{}‘ to ‘self._projection_weight‘ in host memory. Here _projection_weight means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 651: `self._projection_bias: dict[str, MasterTensor] = {}` This assigns ‘{}‘ to ‘self._projection_bias‘ in host memory. Here _projection_bias means an additive learned offset applied after accumulation. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 652: `self._deployed_projection_weight: dict[str, np.ndarray] = {}` This assigns ‘{}‘ to ‘self._deployed_projecti in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 653: `self._deployed_projection_bias: dict[str, np.ndarray] = {}` This assigns ‘{}‘ to ‘self._deployed_projection in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 654: `for name in archive.projection_names:` This iterates name over ‘archive.projection_names‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function ___init___, which handles init.

Line 655: `parameters = archive.projection(name)` This assigns ‘archive.projection(name)‘ to ‘parameters ‘ in host memory. Here parameters means the value named 'parameters' in this block. The important operands are archive: the value named 'archive' in this block; projection: the value named 'projection' in this block; name: the value named 'name' in this block. Context: the function ___init___, which handles init.

Line 656: `weight = parameters.weight_k_by_n.astype(np.int64)` This assigns ‘parameters.weight_k_by_n.astype(np.int64‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are parameters: the value named ’parameters’ in this block; weight_k_by_n: a learned coefficient multiplied by an activation; astype: the value named ’astype’ in this block. Context: the function ___init___, which handles init.

Line 657: `bias = parameters.bias_q10.astype(np.int64)` This assigns ‘parameters.bias_q10.astype(np.int64)‘ to ‘bias ‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are parameters: the value named ’parameters’ in this block; bias_q10: an additive learned offset applied after accumulation; astype: the value named ’astype’ in this block. Context: the function ___init___, which handles init.

Line 658: `weight_fractional_bits = 8` This assigns ‘8‘ to ‘weight_fractional_bits ‘ in host memory. Here weight_fractional_b means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 659: `bias_fractional_bits = _master_fractional_bits(bias, 8)` This assigns ‘_master_fractional_bits(bias, 8)‘ to ‘bias_fractional_bits ‘ in host memory. Here bias_fractional_bits means an additive learned offset applied after accumulation. The important operands are _master_fractional_bits: the value named ’master fractional bits’ in this block; bias: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 660: `self._projection_static[name] = parameters` This assigns ‘parameters‘ to ‘self._projection_static[name] ‘ in host memory. Here _projection_static means the value named ’projection static’ in this block. The important operands are parameters: the value named ’parameters’ in this block. Context: the function ___init___, which handles init.

Line 661: `self._projection_weight[name] = MasterTensor(` This statement continues through source line 667; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘MasterTensor(weight << weight_fractional_bits, np.zeros(weight.shape, dtype=np.int64), weight_fractional_bits, -128, 127,)‘ to ‘self._projection_weight[name] ‘ in host memory. Here _projection_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; weight: a learned coefficient multiplied by an activation; weight_fractional_bits: a learned coefficient multiplied by an activation. Context: the function ___init___, which handles init.

Line 662: `weight << weight_fractional_bits,` This is continuation line 2 of the 7-line logical statement that begins at source line 661. Its exact fragment is ‘weight << weight_fractional_bits,‘. The important operands are weight: a learned coefficient multiplied by an activation; weight_fractional_bits: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘MasterTensor(weight << weight_fractional_bits, np.zeros(weight.shape, dtype=np.int64), weight_fractional_bits, -128, 127,)‘ to ‘self._projection_weight[name] ‘ in host memory. Here _projection_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; weight: a learned coefficient multiplied by an activation; weight_fractional_bits: a learned coefficient multiplied by an activation. Context: the function ___init___, which handles init.

Line 663: `np.zeros(weight.shape, dtype=np.int64),` This is continuation line 3 of the 7-line logical statement that begins at source line 661. Its exact fragment is ‘np.zeros(weight.shape, dtype=np.int64),‘. The important operands are zeros: the value named ’zeros’ in this block; weight: a learned coefficient multiplied by an activation; shape: the value named ’shape’ in this block. Read together, the complete operation is: This assigns ‘MasterTensor(weight << weight_fractional_bits, np.zeros(weight.shape, dtype=np.int64), weight_fractional_bits, -128, 127,)‘ to ‘self._projection_weight[n ‘ in host memory. Here _projection_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; weight: a learned coefficient multiplied by an activation; weight_fractional_bits: a learned coefficient multiplied by an activation. Context: the function ___init___, which handles init.

Line 664: `weight_fractional_bits,` This is continuation line 4 of the 7-line logical statement that begins at source line 661. Its exact fragment is ‘weight_fractional_bits,‘. The important operands are weight_fractional_bits:

a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘MasterTensor(weight << weight_fractional_bits, np.zeros(weight.shape, dtype=np.int64), weight_fractional_bits, -128, 127,)‘ to ‘self._projection_weight[name] ‘ in host memory. Here _projection_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; weight: a learned coefficient multiplied by an activation; weight_fractional_bits: a learned coefficient multiplied by an activation. Context: the function ___init___, which handles init.

Line 665: `-128,` This is continuation line 5 of the 7-line logical statement that begins at source line 661. Its exact fragment is ‘-128,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(weight << weight_fractional_bits, np.zeros(weight.shape, dtype=np.int64), weight_fractional_bits, -128, 127,)‘ to ‘self._projection_weight[name] ‘ in host memory. Here _projection_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; weight: a learned coefficient multiplied by an activation; weight_fractional_bits: a learned coefficient multiplied by an activation. Context: the function ___init___, which handles init.

Line 666: `127,` This is continuation line 6 of the 7-line logical statement that begins at source line 661. Its exact fragment is ‘127,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(weight << weight_fractional_bits, np.zeros(weight.shape, dtype=np.int64), weight_fractional_bits, -128, 127,)‘ to ‘self._projection_weight[name] ‘ in host memory. Here _projection_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; weight: a learned coefficient multiplied by an activation; weight_fractional_bits: a learned coefficient multiplied by an activation. Context: the function ___init___, which handles init.

Line 667: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 661. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(weight << weight_fractional_bits, np.zeros(weight.shape, dtype=np.int64), weight_fractional_bits, -128, 127,)‘ to ‘self._projection_weight[name] ‘ in host memory. Here _projection_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; weight: a learned coefficient multiplied by an activation; weight_fractional_bits: a learned coefficient multiplied by an activation. Context: the function ___init___, which handles init.

Line 668: `self._projection_bias[name] = MasterTensor(` This statement continues through source line 674; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘MasterTensor(bias << bias_fractional_bits, np.zeros(bias.shape, dtype=np.int64), bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)‘ to ‘self._projection_bias[name] ‘ in host memory. Here _projection_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named ’master tensor’ in this block; bias: an additive learned offset applied after accumulation; bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 669: `bias << bias_fractional_bits,` This is continuation line 2 of the 7-line logical statement that begins at source line 668. Its exact fragment is ‘bias << bias_fractional_bits,‘. The important operands are bias: an additive learned offset applied after accumulation; bias_fractional_bits: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘MasterTensor(bias << bias_fractional_bits, np.zeros(bias.shape, dtype=np.int64), bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)‘ to ‘self._projection_bias[name] ‘ in host memory. Here _projection_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named ’master tensor’ in this block; bias: an additive learned offset applied after accumulation; bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 670: `np.zeros(bias.shape, dtype=np.int64),` This is continuation line 3 of the 7-line logical statement that begins at source line 668. Its exact fragment is ‘np.zeros(bias.shape, dtype=np.int64),‘. The important operands are zeros: the value named ’zeros’ in this block; bias: an additive learned offset applied after accumulation; shape: the value named ’shape’ in this block. Read together, the complete operation is: This assigns ‘MasterTensor(bias << bias_fractional_bits, np.zeros(bias.shape, dtype=np.int64), bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)‘ to ‘self._projection_bias[name] ‘ in host memory. Here _projection_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named ’master tensor’ in this block; bias: an additive learned offset applied

after accumulation; bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 671: `bias_fractional_bits,` This is continuation line 4 of the 7-line logical statement that begins at source line 668. Its exact fragment is 'bias_fractional_bits,'. The important operands are bias_fractional_bits: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'MasterTensor(bias << bias_fractional_bits, np.zeros(bias.shape, dtype=np.int64), bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)' to 'self._projection_bias[name] ' in host memory. Here _projection_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named 'master tensor' in this block; bias: an additive learned offset applied after accumulation; bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 672: `-(1 << 15),` This is continuation line 5 of the 7-line logical statement that begins at source line 668. Its exact fragment is '-(1 << 15),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'MasterTensor(bias << bias_fractional_bits, np.zeros(bias.shape, dtype=np.int64), bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)' to 'self._projection_bias[name] ' in host memory. Here _projection_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named 'master tensor' in this block; bias: an additive learned offset applied after accumulation; bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 673: `(1 << 15) - 1,` This is continuation line 6 of the 7-line logical statement that begins at source line 668. Its exact fragment is '(1 << 15) - 1,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'MasterTensor(bias << bias_fractional_bits, np.zeros(bias.shape, dtype=np.int64), bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)' to 'self._projection_bias[name] ' in host memory. Here _projection_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named 'master tensor' in this block; bias: an additive learned offset applied after accumulation; bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 674: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 668. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'MasterTensor(bias << bias_fractional_bits, np.zeros(bias.shape, dtype=np.int64), bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)' to 'self._projection_bias[name] ' in host memory. Here _projection_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named 'master tensor' in this block; bias: an additive learned offset applied after accumulation; bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 675: `self._deployed_projection_weight[name] = weight.astype(np.int8)` This assigns 'weight.astype(np.int8)' to 'self._deployed_projection_weight[name] ' in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are weight: a learned coefficient multiplied by an activation; astype: the value named 'astype' in this block; int8: the value named 'int8' in this block. Context: the function ___init___, which handles init.

Line 676: `self._deployed_projection_bias[name] = bias.astype(np.int64)` This assigns 'bias.astype(np.int64)' to 'self._deployed_projection_bias[name] ' in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are bias: an additive learned offset applied after accumulation; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Context: the function ___init___, which handles init.

Line 678: `head = archive.head` This assigns 'archive.head' to 'head ' in host memory. Here head means the value named 'head' in this block. The important operands are archive: the value named 'archive' in this block; head: the value named 'head' in this block. Context: the function ___init___, which handles init.

Line 679: `head_weight = head.weight_k_by_n.astype(np.int64)` This assigns ‘head.weight_k_by_n.astype(np.int64)‘ to ‘head_weight ‘ in host memory. Here head_weight means a learned coefficient multiplied by an activation. The important operands are head: the value named ’head’ in this block; weight_k_by_n: a learned coefficient multiplied by an activation; astype: the value named ’astype’ in this block. Context: the function ___init___, which handles init.

Line 680: `head_bias = head.bias_accumulator_units.astype(np.int64)` This assigns ‘head.bias_accumulator_units.astype to ‘head_bias ‘ in host memory. Here head_bias means an additive learned offset applied after accumulation. The important operands are head: the value named ’head’ in this block; bias_accumulator_units: an additive learned offset applied after accumulation; astype: the value named ’astype’ in this block. Context: the function ___init___, which handles init.

Line 681: `head_bias_fractional_bits = _master_fractional_bits(head_bias, 8)` This assigns ‘_master_fractional_bits(8)‘ to ‘head_bias_fractional_bits ‘ in host memory. Here head_bias_fractional_bits means an additive learned offset applied after accumulation. The important operands are _master_fractional_bits: the value named ’master fractional bits’ in this block; head_bias: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 682: `self._head_weight = MasterTensor(` This statement continues through source line 688; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘MasterTensor(head_weight << 8, np.zeros(head_weight.shape, dtype=np.int64), 8, -128, 127,)‘ to ‘self._head_weight ‘ in host memory. Here _head_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; head_weight: a learned coefficient multiplied by an activation; zeros: the value named ’zeros’ in this block. Context: the function ___init___, which handles init.

Line 683: `head_weight << 8,` This is continuation line 2 of the 7-line logical statement that begins at source line 682. Its exact fragment is ‘head_weight << 8,‘. The important operands are head_weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘MasterTensor(head_weight << 8, np.zeros(head_weight.shape, dtype=np.int64), 8, -128, 127,)‘ to ‘self._head_weight ‘ in host memory. Here _head_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; head_weight: a learned coefficient multiplied by an activation; zeros: the value named ’zeros’ in this block. Context: the function ___init___, which handles init.

Line 684: `np.zeros(head_weight.shape, dtype=np.int64),` This is continuation line 3 of the 7-line logical statement that begins at source line 682. Its exact fragment is ‘np.zeros(head_weight.shape, dtype=np.int64),‘. The important operands are zeros: the value named ’zeros’ in this block; head_weight: a learned coefficient multiplied by an activation; shape: the value named ’shape’ in this block. Read together, the complete operation is: This assigns ‘MasterTensor(head_weight << 8, np.zeros(head_weight.shape, dtype=np.int64), 8, -128, 127,)‘ to ‘self._head_weight ‘ in host memory. Here _head_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; head_weight: a learned coefficient multiplied by an activation; zeros: the value named ’zeros’ in this block. Context: the function ___init___, which handles init.

Line 685: `8,` This is continuation line 4 of the 7-line logical statement that begins at source line 682. Its exact fragment is ‘8,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(head_weight << 8, np.zeros(head_weight.shape, dtype=np.int64), 8, -128, 127,)‘ to ‘self._head_weight ‘ in host memory. Here _head_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; head_weight: a learned coefficient multiplied by an activation; zeros: the value named ’zeros’ in this block. Context: the function ___init___, which handles init.

Line 686: `-128,` This is continuation line 5 of the 7-line logical statement that begins at source line 682. Its exact fragment is ‘-128,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(head_weight << 8, np.zeros(head_weight.shape, dtype=np.int64), 8, -128, 127,)‘ to ‘self._head_weight ‘ in host memory. Here _head_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named ’master tensor’ in this block; head_weight: a learned coefficient multiplied by an activation; zeros: the value named ’zeros’ in this block. Context: the function ___init___, which handles init.

Line 687: `127,` This is continuation line 6 of the 7-line logical statement that begins at source line 682. Its exact fragment is '127,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'MasterTensor(head_weight << 8, np.zeros(head_weight.shape, dtype=np.int64), 8, -128, 127,)' to 'self._head_weight ' in host memory. Here _head_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named 'master tensor' in this block; head_weight: a learned coefficient multiplied by an activation; zeros: the value named 'zeros' in this block. Context: the function ___init___, which handles init.

Line 688: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 682. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'MasterTensor(head_weight << 8, np.zeros(head_weight.shape, dtype=np.int64), 8, -128, 127,)' to 'self._head_weight ' in host memory. Here _head_weight means a learned coefficient multiplied by an activation. The important operands are MasterTensor: the value named 'master tensor' in this block; head_weight: a learned coefficient multiplied by an activation; zeros: the value named 'zeros' in this block. Context: the function ___init___, which handles init.

Line 689: `self._head_bias = MasterTensor(` This statement continues through source line 695; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'MasterTensor(head_bias << head_bias_fractional_bits, np.zeros(head_bias.shape, dtype=np.int64), head_bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)' to 'self._head_bias ' in host memory. Here _head_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named 'master tensor' in this block; head_bias: an additive learned offset applied after accumulation; head_bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 690: `head_bias << head_bias_fractional_bits,` This is continuation line 2 of the 7-line logical statement that begins at source line 689. Its exact fragment is 'head_bias << head_bias_fractional_bits,'. The important operands are head_bias: an additive learned offset applied after accumulation; head_bias_fractional_bits: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'MasterTensor(head_bias << head_bias_fractional_bits, np.zeros(head_bias.shape, dtype=np.int64), head_bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)' to 'self._head_bias ' in host memory. Here _head_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named 'master tensor' in this block; head_bias: an additive learned offset applied after accumulation; head_bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 691: `np.zeros(head_bias.shape, dtype=np.int64),` This is continuation line 3 of the 7-line logical statement that begins at source line 689. Its exact fragment is 'np.zeros(head_bias.shape, dtype=np.int64),'. The important operands are zeros: the value named 'zeros' in this block; head_bias: an additive learned offset applied after accumulation; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns 'MasterTensor(head_bias << head_bias_fractional_bits, np.zeros(head_bias.shape, dtype=np.int64), head_bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)' to 'self._head_bias ' in host memory. Here _head_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named 'master tensor' in this block; head_bias: an additive learned offset applied after accumulation; head_bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 692: `head_bias_fractional_bits,` This is continuation line 4 of the 7-line logical statement that begins at source line 689. Its exact fragment is 'head_bias_fractional_bits,'. The important operands are head_bias_fractional_bits: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'MasterTensor(head_bias << head_bias_fractional_bits, np.zeros(head_bias.shape, dtype=np.int64), head_bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)' to 'self._head_bias ' in host memory. Here _head_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named 'master tensor' in this block; head_bias: an additive learned offset applied after accumulation; head_bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 693: `-(1 << 15),` This is continuation line 5 of the 7-line logical statement that begins at source line 689. Its exact fragment is '-(1 << 15),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'MasterTensor(head_bias << head_bias_fractional_bits,

np.zeros(head_bias.shape, dtype=np.int64), head_bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)‘ to ‘self._head_bias ‘ in host memory. Here _head_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named ’master tensor’ in this block; head_bias: an additive learned offset applied after accumulation; head_bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 694: `(1 << 15) - 1,` This is continuation line 6 of the 7-line logical statement that begins at source line 689. Its exact fragment is ‘(1 << 15) - 1,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(head_bias << head_bias_fractional_bits, np.zeros(head_bias.shape, dtype=np.int64), head_bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)‘ to ‘self._head_bias ‘ in host memory. Here _head_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named ’master tensor’ in this block; head_bias: an additive learned offset applied after accumulation; head_bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 695: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 689. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(head_bias << head_bias_fractional_bits, np.zeros(head_bias.shape, dtype=np.int64), head_bias_fractional_bits, -(1 << 15), (1 << 15) - 1,)‘ to ‘self._head_bias ‘ in host memory. Here _head_bias means an additive learned offset applied after accumulation. The important operands are MasterTensor: the value named ’master tensor’ in this block; head_bias: an additive learned offset applied after accumulation; head_bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function ___init___, which handles init.

Line 696: `self._deployed_head_weight = head_weight.astype(np.int8)` This assigns ‘head_weight.astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are head_weight: a learned coefficient multiplied by an activation; astype: the value named ’astype’ in this block; int8: the value named ’int8’ in this block. Context: the function ___init___, which handles init.

Line 697: `self._deployed_head_bias = head_bias.astype(np.int64)` This assigns ‘head_bias.astype(np.int64)‘ to ‘self._deployed_head_bias ‘ in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are head_bias: an additive learned offset applied after accumulation; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function ___init___, which handles init.

Line 698: `self.optimizer_step = 0` This assigns ‘0‘ to ‘self.optimizer_step ‘ in host memory. Here optimizer_step means the value named ’optimizer step’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 700: `@property` This decorator changes how the following class or function is constructed. ‘@property‘ is applied at import time on the host and is part of the class TrainableSpikingformerState, which handles trainable spikingformer state; it creates no FPGA circuit by itself.

Line 701: `def projection_names(self) -> tuple[str, ...]:` This starts function projection_names. Its indented body performs projection names as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 702: `return tuple(self._projection_static)` This returns ‘tuple(self._projection_static)‘ to the caller of the function projection_names, which handles projection names, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; _projection_static: the value named ’projection static’ in this block.

Line 704: `def projection(self, name: str) -> ProjectionParameters:` This starts function projection. Its indented body performs projection as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 705: `static = self._projection_static[name]` This assigns ‘self._projection_static[name]‘ to ‘static ‘ in host memory. Here static means the value named ’static’ in this block. The important operands are _projection_static: the value named ’projection static’ in this block; name: the value named ’name’ in this block. Context: the function projection, which handles projection.

Line 706: `return ProjectionParameters(` This statement continues through source line 715; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘ProjectionParameters(static.name, self._deployed_projection_weight[name], self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_weight_shape, static.output_fractional_bits,)‘ to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; static: the value named ’static’ in this block; name: the value named ’name’ in this block.

Line 707: `static.name,` This is continuation line 2 of the 10-line logical statement that begins at source line 706. Its exact fragment is ‘static.name,‘. The important operands are static: the value named ’static’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: This returns ‘ProjectionParameters(static.name, self._deployed_projection_weight[name], self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_weight_shape, static.output_fractional_bits,)‘ to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; static: the value named ’static’ in this block; name: the value named ’name’ in this block.

Line 708: `self._deployed_projection_weight[name],` This is continuation line 3 of the 10-line logical statement that begins at source line 706. Its exact fragment is ‘self._deployed_projection_weight[name],‘. The important operands are _deployed_projection_weight: a learned coefficient multiplied by an activation; name: the value named ’name’ in this block. Read together, the complete operation is: This returns ‘ProjectionParameters(static.name, self._deployed_projection_weight[name], self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_weight_shape, static.output_fractional_bits,)‘ to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; static: the value named ’static’ in this block; name: the value named ’name’ in this block.

Line 709: `self._deployed_projection_bias[name],` This is continuation line 4 of the 10-line logical statement that begins at source line 706. Its exact fragment is ‘self._deployed_projection_bias[name],‘. The important operands are _deployed_projection_bias: an additive learned offset applied after accumulation; name: the value named ’name’ in this block. Read together, the complete operation is: This returns ‘ProjectionParameters(static.name, self._deployed_projection_weigh self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_wei static.output_fractional_bits,)‘ to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; static: the value named ’static’ in this block; name: the value named ’name’ in this block.

Line 710: `static.multiplier_q15,` This is continuation line 5 of the 10-line logical statement that begins at source line 706. Its exact fragment is ‘static.multiplier_q15,‘. The important operands are static: the value named ’static’ in this block; multiplier_q15: the value named ’multiplier q15’ in this block. Read together, the complete operation is: This returns ‘ProjectionParameters(static.name, self._deployed_projection_weight[name], self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_weight_shape, static.output_fractional_bi)‘ to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; static: the value named ’static’ in this block; name: the value named ’name’ in this block.

Line 711: `static.binary_point_shift,` This is continuation line 6 of the 10-line logical statement that begins at source line 706. Its exact fragment is ‘static.binary_point_shift,‘. The important operands are static: the value named ’static’ in this block; binary_point_shift: the value named ’binary point shift’ in this block. Read together, the complete operation is: This returns ‘ProjectionParameters(static.name, self._deployed_projection_weight[name], self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_wei static.output_fractional_bits,)‘ to the caller of the function projection, which handles projection, ending this invocation.

The important operands are ProjectionParameters: the value named 'projection parameters' in this block; static: the value named 'static' in this block; name: the value named 'name' in this block.

Line 712: `static.compute_opcode,` This is continuation line 7 of the 10-line logical statement that begins at source line 706. Its exact fragment is 'static.compute_opcode,'. The important operands are static: the value named 'static' in this block; compute_opcode: the value named 'compute opcode' in this block. Read together, the complete operation is: This returns 'ProjectionParameters(static.name, self._deployed_projection_weight[name], self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_weight_shape, static.output_fractional_bi)' to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; static: the value named 'static' in this block; name: the value named 'name' in this block.

Line 713: `static.source_weight_shape,` This is continuation line 8 of the 10-line logical statement that begins at source line 706. Its exact fragment is 'static.source_weight_shape,'. The important operands are static: the value named 'static' in this block; source_weight_shape: a learned coefficient multiplied by an activation. Read together, the complete operation is: This returns 'ProjectionParameters(static.name, self._deployed_projection_weight[name], self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_wei static.output_fractional_bits,)' to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; static: the value named 'static' in this block; name: the value named 'name' in this block.

Line 714: `static.output_fractional_bits,` This is continuation line 9 of the 10-line logical statement that begins at source line 706. Its exact fragment is 'static.output_fractional_bits,'. The important operands are static: the value named 'static' in this block; output_fractional_bits: the value named 'output fractional bits' in this block. Read together, the complete operation is: This returns 'ProjectionParameters(static.name, self._deployed_projection_weight[name], self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_wei static.output_fractional_bits,)' to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; static: the value named 'static' in this block; name: the value named 'name' in this block.

Line 715: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 706. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'ProjectionParameters(static.name, self._deployed_projection_weight[name], self._deployed_projection_bias[name], static.multiplier_q15, static.binary_point_shift, static.compute_opcode, static.source_wei static.output_fractional_bits,)' to the caller of the function projection, which handles projection, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; static: the value named 'static' in this block; name: the value named 'name' in this block.

Line 717: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class TrainableSpikingformerState, which handles trainable spikingformer state; it creates no FPGA circuit by itself.

Line 718: `def head(self) -> HeadParameters:` This starts function head. Its indented body performs head as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 719: `return HeadParameters(self._deployed_head_weight, self._deployed_head_bias)` This returns 'HeadParameters(self._deployed_head_weight, self._deployed_head_bias)' to the caller of the function head, which handles head, ending this invocation. The important operands are HeadParameters: the value named 'head parameters' in this block; _deployed_head_weight: a learned coefficient multiplied by an activation; _deployed_head_bias: an additive learned offset applied after accumulation.

Line 721: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class TrainableSpikingformerState, which handles trainable spikingformer state; it creates no FPGA circuit by itself.

Line 722: `def _optimizer_coefficients(` This starts function _optimizer_coefficients. Its indented body performs optimizer coefficients as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 723: `tensor: MasterTensor,` Inside the function _optimizer_coefficients, which handles optimizer coefficients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tensor: the value named 'tensor' in this block; MasterTensor: the value named 'master tensor' in this block.

Line 724: `learning_rate: float,` Inside the function _optimizer_coefficients, which handles optimizer coefficients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude; float: the value named 'float' in this block.

Line 725: `momentum: float,` Inside the function _optimizer_coefficients, which handles optimizer coefficients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are momentum: the value named 'momentum' in this block; float: the value named 'float' in this block.

Line 726: `weight_decay: float,` Inside the function _optimizer_coefficients, which handles optimizer coefficients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are weight_decay: a learned coefficient multiplied by an activation; float: the value named 'float' in this block.

Line 727: `) -> tuple[int, int, int, int]:` Inside the function _optimizer_coefficients, which handles optimizer coefficients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named 'tuple' in this block.

Line 728: `if not 0.0 <= learning_rate:` This executes the following indented host-code branch only when ‘not 0.0 <= learning_rate‘ is true. The important operands are not: the value named 'not' in this block; learning_rate: the fixed-point multiplier controlling update magnitude. Context: the function _optimizer_coefficients, which handles optimizer coefficients.

Line 729: `raise ValueError("learning_rate must be nonnegative")` This rejects an invalid state by raising ‘ValueError("learning_rate must be nonnegative")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 730: `if not 0.0 <= momentum <= 1.0 or not 0.0 <= weight_decay:` This executes the following indented host-code branch only when ‘not 0.0 <= momentum <= 1.0 or not 0.0 <= weight_decay‘ is true. The important operands are not: the value named 'not' in this block; momentum: the value named 'momentum' in this block; or: the value named 'or' in this block. Context: the function _optimizer_coefficients, which handles optimizer coefficients.

Line 731: `raise ValueError("momentum must be in [0,1] and weight_decay nonnegative")` This rejects an invalid state by raising ‘ValueError("momentum must be in [0,1] and weight_decay nonnegative")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 732: `learning_rate_q16 = int(round(learning_rate * (1 << 16)))` This assigns ‘int(round(learning_rate * (1 << 16)))‘ to ‘learning_rate_q16 ‘ in host memory. Here learning_rate_q16 means the fixed-point multiplier controlling update magnitude. The important operands are round: the value named 'round' in this block; learning_rate: the fixed-point multiplier controlling update magnitude. Context: the function _optimizer_coefficients, which handles optimizer coefficients.

Line 733: `momentum_q16 = int(round(momentum * (1 << 16)))` This assigns ‘int(round(momentum * (1 << 16)))‘ to ‘momentum_q16 ‘ in host memory. Here momentum_q16 means the value named ’momentum q16’ in this block. The important operands are round: the value named ’round’ in this block; momentum: the value named ’momentum’ in this block. Context: the function _optimizer_coefficients, which handles optimizer coefficients.

Line 734: `decay_q16 = int(round(weight_decay * (1 << (31 - tensor.fractional_bits))))` This assigns ‘int(round(wei * (1 << (31 - tensor.fractional_bits))))‘ to ‘decay_q16 ‘ in host memory. Here decay_q16 means the value named ’decay q16’ in this block. The important operands are round: the value named ’round’ in this block; weight_decay: a learned coefficient multiplied by an activation; tensor: the value named ’tensor’ in this block. Context: the function _optimizer_coefficients, which handles optimizer coefficients.

Line 735: `update_right_shift = 31 - tensor.fractional_bits` This assigns ‘31 - tensor.fractional_bits‘ to ‘update_right_shift ‘ in host memory. Here update_right_shift means the value named ’update right shift’ in this block. The important operands are tensor: the value named ’tensor’ in this block; fractional_bits: the value named ’fractional bits’ in this block. Context: the function _optimizer_coefficients, which handles optimizer coefficients.

Line 736: `if learning_rate > 0.0 and learning_rate_q16 == 0:` This executes the following indented host-code branch only when ‘learning_rate > 0.0 and learning_rate_q16 == 0‘ is true. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude; and: the value named ’and’ in this block; learning_rate_q16: the fixed-point multiplier controlling update magnitude. Context: the function _optimizer_coefficients, which handles optimizer coefficients.

Line 737: `raise ValueError("learning_rate is below the Q16 optimizer resolution")` This rejects an invalid state by raising ‘ValueError("learning_rate is below the Q16 optimizer resolution")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 738: `if learning_rate_q16 > 131071:` This executes the following indented host-code branch only when ‘learning_rate_q16 > 131071‘ is true. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude. Context: the function _optimizer_coefficients, which handles optimizer coefficients.

Line 739: `raise OverflowError("Q16 learning rate exceeds the RTL field")` This rejects an invalid state by raising ‘OverflowError("Q16 learning rate exceeds the RTL field")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 740: `if momentum_q16 > 65536 or decay_q16 > 65536:` This executes the following indented host-code branch only when ‘momentum_q16 > 65536 or decay_q16 > 65536‘ is true. The important operands are momentum_q16: the value named ’momentum q16’ in this block; or: the value named ’or’ in this block; decay_q16: the value named ’decay q16’ in this block. Context: the function _optimizer_coefficients, which handles optimizer coefficients.

Line 741: `raise OverflowError("momentum or master-format weight decay exceeds the RTL field")` This rejects an invalid state by raising ‘OverflowError("momentum or master-format weight decay exceeds the RTL field")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 742: `return learning_rate_q16, momentum_q16, decay_q16, update_right_shift` This returns ‘learning_rate_q16, momentum_q16, decay_q16, update_right_shift‘ to the caller of the function _optimizer_coefficients, which handles optimizer coefficients, ending this invocation. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude; momentum_q16: the value named ’momentum q16’ in this block; decay_q16: the value named ’decay q16’ in this block.

Line 744: `def _update_tensor(` This starts function _update_tensor. Its indented body performs update tensor as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 745: `self,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 746: `driver: FullTrainingDriver,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 747: `tensor: MasterTensor,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tensor: the value named 'tensor' in this block; MasterTensor: the value named 'master tensor' in this block.

Line 748: `gradient: BlockGradient,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 749: `*,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 750: `learning_rate: float,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude; float: the value named 'float' in this block.

Line 751: `momentum: float,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are momentum: the value named 'momentum' in this block; float: the value named 'float' in this block.

Line 752: `weight_decay: float,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are weight_decay: a learned coefficient multiplied by an activation; float: the value named 'float' in this block.

Line 753: `gradient_clip_q15: int,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are gradient_clip_q15: the signed loss derivative used to update a learned parameter.

Line 754: `timeout_seconds: float,` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; float: the value named 'float' in this block.

Line 755: `) -> np.ndarray:` Inside the function _update_tensor, which handles update tensor, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are ndarray: the value named 'ndarray' in this block.

Line 756: `if gradient.values.shape != tensor.master.shape:` This executes the following indented host-code branch only when ‘gradient.values.shape != tensor.master.shape‘ is true. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function _update_tensor, which handles update tensor.

Line 757: `raise ValueError("parameter gradient shape does not match its master tensor")` This rejects an invalid state by raising ‘ValueError("parameter gradient shape does not match its master tensor")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 758: `gradient_q15 = gradient_to_optimizer_q15(` This statement continues through source line 763; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘gradient_to_optimizer_q15(driver, gradient, clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘gradient_q15 ‘ in host memory. Here gradient_q15 means the signed loss derivative used to update a learned parameter. The important operands are gradient_to_optimizer_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; gradient: the signed loss derivative used to update a learned parameter. Context: the function _update_tensor, which handles update tensor.

Line 759: `driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 758. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘gradient_to_optimizer_q15(driver, gradient, clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘gradient_q15 ‘ in host memory. Here gradient_q15 means the signed loss derivative used to update a learned parameter. The important operands are gradient_to_optimizer_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; gradient: the signed loss derivative used to update a learned parameter. Context: the function _update_tensor, which handles update tensor.

Line 760: `gradient,` This is continuation line 3 of the 6-line logical statement that begins at source line 758. Its exact fragment is ‘gradient,‘. The important operands are gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘gradient_to_optimizer_q15(driver, gradient, clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘gradient_q15 ‘ in host memory. Here gradient_q15 means the signed loss derivative used to update a learned parameter. The important operands are gradient_to_optimizer_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; gradient: the signed loss derivative used to update a learned parameter. Context: the function _update_tensor, which handles update tensor.

Line 761: `clip_q15=gradient_clip_q15,` This is continuation line 4 of the 6-line logical statement that begins at source line 758. Its exact fragment is ‘clip_q15=gradient_clip_q15,‘. The important operands are clip_q15: the value named ’clip q15’ in this block; gradient_clip_q15: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘gradient_to_optimizer_q15(driver, gradient, clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘gradient_q15 ‘ in host memory. Here gradient_q15 means the signed loss derivative used to update a learned parameter. The important operands are gradient_to_optimizer_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; gradient: the signed loss derivative used to update a learned parameter. Context: the function _update_tensor, which handles update tensor.

Line 762: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 758. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘gradient_to_optimizer_q15(driver, gradient, clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘gradient_q15 ‘ in host memory. Here gradient_q15 means the signed loss derivative used to update a learned parameter. The important operands are gradient_to_optimizer_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; gradient: the signed loss derivative used to update a learned parameter. Context: the function _update_tensor, which handles update tensor.

Line 763: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 758. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘gradient_to_optimizer_q15(driver, gradient, clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘gradient_q15 ‘ in host memory. Here gradient_q15 means the signed loss

derivative used to update a learned parameter. The important operands are gradient_to_optimizer_q15: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; gradient: the signed loss derivative used to update a learned parameter. Context: the function _update_tensor, which handles update tensor.

Line 764: `coefficients = self._optimizer_coefficients(` This statement continues through source line 765; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._optimizer_coefficients(tensor, learning_rate, momentum, weight_decay)' to 'coefficients ' in host memory. Here coefficients means the value named 'coefficients' in this block. The important operands are _optimizer_coefficients: the value named 'optimizer coefficients' in this block; tensor: the value named 'tensor' in this block; learning_rate: the fixed-point multiplier controlling update magnitude. Context: the function _update_tensor, which handles update tensor.

Line 765: `tensor, learning_rate, momentum, weight_decay)` This is continuation line 2 of the 2-line logical statement that begins at source line 764. Its exact fragment is 'tensor, learning_rate, momentum, weight_decay)'. The important operands are tensor: the value named 'tensor' in this block; learning_rate: the fixed-point multiplier controlling update magnitude; momentum: the value named 'momentum' in this block. Read together, the complete operation is: This assigns 'self._optimizer_coefficients(tensor, learning_rate, momentum, weight_decay)' to 'coefficients ' in host memory. Here coefficients means the value named 'coefficients' in this block. The important operands are _optimizer_coefficients: the value named 'optimizer coefficients' in this block; tensor: the value named 'tensor' in this block; learning_rate: the fixed-point multiplier controlling update magnitude. Context: the function _update_tensor, which handles update tensor.

Line 766: `next_master, next_velocity = execute_sgdw_update(` This statement continues through source line 776; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[2], update_right_shift=coefficients[3], timeout_seconds=timeout_se)' to 'next_master, next_velocity ' in host memory. Here next_master means the value named 'next master' in this block. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function _update_tensor, which handles update tensor.

Line 767: `driver,` This is continuation line 2 of the 11-line logical statement that begins at source line 766. Its exact fragment is 'driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[2], update_right_shift=coefficients timeout_seconds=timeout_seconds,)' to 'next_master, next_velocity ' in host memory. Here next_master means the value named 'next master' in this block. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function _update_tensor, which handles update tensor.

Line 768: `tensor.master,` This is continuation line 3 of the 11-line logical statement that begins at source line 766. Its exact fragment is 'tensor.master,'. The important operands are tensor: the value named 'tensor' in this block; master: the value named 'master' in this block. Read together, the complete operation is: This assigns 'execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[2], update_right_shift=coefficients[3], timeout_seconds=timeout_seconds,)' to 'next_master, next_velocity ' in host memory. Here next_master means the value named 'next master' in this block. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function _update_tensor, which handles update tensor.

Line 769: `gradient_q15,` This is continuation line 4 of the 11-line logical statement that begins at source line 766. Its exact fragment is 'gradient_q15,'. The important operands are gradient_q15: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[2], update_right_shift=coefficients[3], timeout_seconds=timeout_seconds,)' to 'next_master, next_velocity ' in host memory. Here next_master means the value named 'next master' in this block. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; driver: the value named

'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function _update_tensor, which handles update tensor.

Line 770: `tensor.velocity_q15,` This is continuation line 5 of the 11-line logical statement that begins at source line 766. Its exact fragment is 'tensor.velocity_q15,'. The important operands are tensor: the value named 'tensor' in this block; velocity_q15: the momentum history retained by the SGD optimizer. Read together, the complete operation is: This assigns 'execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[2], update_right_shift=coefficients[timeout_seconds=timeout_seconds,)' to 'next_master, next_velocity ' in host memory. Here next_master means the value named 'next master' in this block. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function _update_tensor, which handles update tensor.

Line 771: `learning_rate_q16=coefficients[0],` This is continuation line 6 of the 11-line logical statement that begins at source line 766. Its exact fragment is 'learning_rate_q16=coefficients[0],'. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude; coefficients: the value named 'coefficients' in this block. Read together, the complete operation is: This assigns 'execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[update_right_shift=coefficients[3], timeout_seconds=timeout_seconds,)' to 'next_master, next_velocity ' in host memory. Here next_master means the value named 'next master' in this block. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function _update_tensor, which handles update tensor.

Line 772: `momentum_q16=coefficients[1],` This is continuation line 7 of the 11-line logical statement that begins at source line 766. Its exact fragment is 'momentum_q16=coefficients[1],'. The important operands are momentum_q16: the value named 'momentum q16' in this block; coefficients: the value named 'coefficients' in this block. Read together, the complete operation is: This assigns 'execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[2], update_right_shift=coeffic timeout_seconds=timeout_seconds,)' to 'next_master, next_velocity ' in host memory. Here next_master means the value named 'next master' in this block. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function _update_tensor, which handles update tensor.

Line 773: `weight_decay_q16=coefficients[2],` This is continuation line 8 of the 11-line logical statement that begins at source line 766. Its exact fragment is 'weight_decay_q16=coefficients[2],'. The important operands are weight_decay_q16: a learned coefficient multiplied by an activation; coefficients: the value named 'coefficients' in this block. Read together, the complete operation is: This assigns 'execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[update_right_shift=coefficients[3], timeout_seconds=timeout_seconds,)' to 'next_master, next_velocity ' in host memory. Here next_master means the value named 'next master' in this block. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function _update_tensor, which handles update tensor.

Line 774: `update_right_shift=coefficients[3],` This is continuation line 9 of the 11-line logical statement that begins at source line 766. Its exact fragment is 'update_right_shift=coefficients[3],'. The important operands are update_right_shift: the value named 'update right shift' in this block; coefficients: the value named 'coefficients' in this block. Read together, the complete operation is: This assigns 'execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[update_right_shift=coefficients[3], timeout_seconds=timeout_seconds,)' to 'next_master, next_velocity ' in host memory. Here next_master means the value named 'next master' in this block. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; driver: the value named 'driver' in this block; tensor: the value named 'tensor' in this block. Context: the function _update_tensor, which handles update tensor.

Line 775: `timeout_seconds=timeout_seconds,` This is continuation line 10 of the 11-line logical statement that begins at source line 766. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'ex-

ecute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[2], update_right_shift=coefficients[3], timeout_seconds=timeout_se)‘ to ‘next_master, next_velocity ‘ in host memory. Here next_master means the value named ’next master’ in this block. The important operands are execute_sgdw_update: the value named ’execute sgdw update’ in this block; driver: the value named ’driver’ in this block; tensor: the value named ’tensor’ in this block. Context: the function _update_tensor, which handles update tensor.

Line 776: `)` This is continuation line 11 of the 11-line logical statement that begins at source line 766. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_sgdw_update(driver, tensor.master, gradient_q15, tensor.velocity_q15, learning_rate_q16=coefficients[0], momentum_q16=coefficients[1], weight_decay_q16=coefficients[2], update_right_shift=coefficients[timeout_seconds=timeout_seconds,)‘ to ‘next_master, next_velocity ‘ in host memory. Here next_master means the value named ’next master’ in this block. The important operands are execute_sgdw_update: the value named ’execute sgdw update’ in this block; driver: the value named ’driver’ in this block; tensor: the value named ’tensor’ in this block. Context: the function _update_tensor, which handles update tensor.

Line 777: `tensor.master = next_master.astype(np.int16)` This assigns ‘next_master.astype(np.int16)‘ to ‘tensor.master ‘ in host memory. Here master means the value named ’master’ in this block. The important operands are next_master: the value named ’next master’ in this block; astype: the value named ’astype’ in this block; int16: the value named ’int16’ in this block. Context: the function _update_tensor, which handles update tensor.

Line 778: `tensor.velocity_q15 = next_velocity.astype(np.int32)` This assigns ‘next_velocity.astype(np.int32)‘ to ‘tensor.velocity_q15 ‘ in host memory. Here velocity_q15 means the momentum history retained by the SGD optimizer. The important operands are next_velocity: the momentum history retained by the SGD optimizer; astype: the value named ’astype’ in this block; int32: the value named ’int32’ in this block. Context: the function _update_tensor, which handles update tensor.

Line 779: `return execute_deploy_master(driver, tensor, timeout_seconds=timeout_seconds)` This returns ‘execute_deploy_master(driver, tensor, timeout_seconds=timeout_seconds)‘ to the caller of the function _update_tensor, which handles update tensor, ending this invocation. The important operands are execute_deploy_master: the value named ’execute deploy master’ in this block; driver: the value named ’driver’ in this block; tensor: the value named ’tensor’ in this block.

Line 781: `def apply_gradients(` This starts function apply_gradients. Its indented body performs apply gradients as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 782: `self,` Inside the function apply_gradients, which handles apply gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 783: `driver: FullTrainingDriver,` Inside the function apply_gradients, which handles apply gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named ’driver’ in this block; FullTrainingDriver: the value named ’full training driver’ in this block.

Line 784: `gradients: Mapping[str, ParameterGradient],` Inside the function apply_gradients, which handles apply gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are gradients: the signed loss derivative used to update a learned parameter; Mapping: the value named ’mapping’ in this block; str: the value named ’str’ in this block.

Line 785: `*,` Inside the function apply_gradients, which handles apply gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 786: `learning_rate: float,` Inside the function apply_gradients, which handles apply gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude; float: the value named 'float' in this block.

Line 787: `momentum: float = 0.9,` This assigns '0.9,' to 'momentum' in host memory. Here momentum means the value named 'momentum' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function apply_gradients, which handles apply gradients.

Line 788: `weight_decay: float = 5e-4,` This assigns '5e-4,' to 'weight_decay' in host memory. Here weight_decay means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function apply_gradients, which handles apply gradients.

Line 789: `gradient_clip_q15: int = 4 << 15,` This assigns '4 << 15,' to 'gradient_clip_q15' in host memory. Here gradient_clip_q15 means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function apply_gradients, which handles apply gradients.

Line 790: `timeout_seconds: float = 120.0,` This assigns '120.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function apply_gradients, which handles apply gradients.

Line 791: `) -> None:` Inside the function apply_gradients, which handles apply gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are None: the value named 'none' in this block.

Line 792: `"""Update every projection and classifier coefficient through RTL SGDW."""` This is executable documentation attached to the function apply_gradients, which handles apply gradients. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 794: `expected = set(self.projection_names) | {"head"}` This assigns 'set(self.projection_names) | {"head"}' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are set: the value named 'set' in this block; projection_names: the value named 'projection names' in this block; head: the value named 'head' in this block. Context: the function apply_gradients, which handles apply gradients.

Line 795: `if set(gradients) != expected:` This executes the following indented host-code branch only when 'set(gradients) != expected' is true. The important operands are set: the value named 'set' in this block; gradients: the signed loss derivative used to update a learned parameter; expected: the value named 'expected' in this block. Context: the function apply_gradients, which handles apply gradients.

Line 796: `missing = sorted(expected - set(gradients))` This assigns 'sorted(expected - set(gradients))' to 'missing ' in host memory. Here missing means the value named 'missing' in this block. The important operands are sorted: the value named 'sorted' in this block; expected: the value named 'expected' in this block; set: the value named 'set' in this block. Context: the function apply_gradients, which handles apply gradients.

Line 797: `extra = sorted(set(gradients) - expected)` This assigns 'sorted(set(gradients) - expected)' to 'extra ' in host memory. Here extra means the value named 'extra' in this block. The important operands are sorted: the value named 'sorted' in this block; set: the value named 'set' in this block; gradients: the signed loss derivative used to update a learned parameter. Context: the function apply_gradients, which handles apply gradients.

Line 798: `raise ValueError(f"parameter-gradient coverage mismatch: missing={missing}, extra={extra}")` This rejects an invalid state by raising 'ValueError(f"parameter-gradient coverage mismatch: missing={missing}, extra={extra}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 799: `for name in self.projection_names:` This iterates name over ‘self.projection_names‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function apply_gradients, which handles apply gradients.

Line 800: `gradient = gradients[name]` This assigns ‘gradients[name]‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are gradients: the signed loss derivative used to update a learned parameter; name: the value named ’name’ in this block. Context: the function apply_gradients, which handles apply gradients.

Line 801: `self._deployed_projection_weight[name] = self._update_tensor(` This statement continues through source line 810; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._update_tensor(driver, self._projection_weight[name], gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_projection_weight[name] ‘ in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 802: `driver,` This is continuation line 2 of the 10-line logical statement that begins at source line 801. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_weight[name], gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_projection_weight[name] ‘ in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 803: `self._projection_weight[name],` This is continuation line 3 of the 10-line logical statement that begins at source line 801. Its exact fragment is ‘self._projection_weight[name],‘. The important operands are _projection_weight: a learned coefficient multiplied by an activation; name: the value named ’name’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_weight[name], gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_projection_weight[name] ‘ in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 804: `gradient.weight,` This is continuation line 4 of the 10-line logical statement that begins at source line 801. Its exact fragment is ‘gradient.weight,‘. The important operands are gradient: the signed loss derivative used to update a learned parameter; weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_weight[name], gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconc).astype(np.int8)‘ to ‘self._deployed_projection_weight[name] ‘ in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 805: `learning_rate=learning_rate,` This is continuation line 5 of the 10-line logical statement that begins at source line 801. Its exact fragment is ‘learning_rate=learning_rate,‘. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_weight[name], gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_projection_weight[name] ‘ in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named

'update tensor' in this block; driver: the value named 'driver' in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 806: `momentum=momentum,` This is continuation line 6 of the 10-line logical statement that begins at source line 801. Its exact fragment is 'momentum=momentum,'. The important operands are momentum: the value named 'momentum' in this block. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._projection_weight[nam gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradien timeout_seconds=timeout_seconds,).astype(np.int8)' to 'self._deployed_projection_weight[name] ' in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 807: `weight_decay=weight_decay,` This is continuation line 7 of the 10-line logical statement that begins at source line 801. Its exact fragment is 'weight_decay=weight_decay,'. The important operands are weight_decay: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._projection_weight[name], gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weig gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)' to 'self._deployed_projection_weig ' in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 808: `gradient_clip_q15=gradient_clip_q15,` This is continuation line 8 of the 10-line logical statement that begins at source line 801. Its exact fragment is 'gradient_clip_q15=gradient_clip_q15,'. The important operands are gradient_clip_q15: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._projection_weight[name], gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_secon).astype(np.int8)' to 'self._deployed_projection_weight[name] ' in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 809: `timeout_seconds=timeout_seconds,` This is continuation line 9 of the 10-line logical statement that begins at source line 801. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._projection_weight[name], gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)' to 'self._deployed_projection_weight[name] ' in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 810: `).astype(np.int8)` This is continuation line 10 of the 10-line logical statement that begins at source line 801. Its exact fragment is ').astype(np.int8)'. The important operands are astype: the value named 'astype' in this block; int8: the value named 'int8' in this block. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._projection_weight[name], gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weig gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)' to 'self._deployed_projection_weig ' in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _projection_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 811: `self._deployed_projection_bias[name] = self._update_tensor(` This statement continues through source line 820; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momen-

tum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 812: `driver,` This is continuation line 2 of the 10-line logical statement that begins at source line 811. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=t)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 813: `self._projection_bias[name],` This is continuation line 3 of the 10-line logical statement that begins at source line 811. Its exact fragment is ‘self._projection_bias[name],‘. The important operands are _projection_bias: an additive learned offset applied after accumulation; name: the value named ’name’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=t)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 814: `gradient.bias,` This is continuation line 4 of the 10-line logical statement that begins at source line 811. Its exact fragment is ‘gradient.bias,‘. The important operands are gradient: the signed loss derivative used to update a learned parameter; bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 815: `learning_rate=learning_rate,` This is continuation line 5 of the 10-line logical statement that begins at source line 811. Its exact fragment is ‘learning_rate=learning_rate,‘. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 816: `momentum=momentum,` This is continuation line 6 of the 10-line logical statement that begins at source line 811. Its exact fragment is ‘momentum=momentum,‘. The important operands are momentum: the value named ’momentum’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bia means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 817: `weight_decay=0.0,` This is continuation line 7 of the 10-line logical statement that begins at source line 811. Its exact fragment is ‘weight_decay=0.0,‘. The important operands are weight_decay: a learned coeffi-

cient multiplied by an activation. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 818: `gradient_clip_q15=gradient_clip_q15,` This is continuation line 8 of the 10-line logical statement that begins at source line 811. Its exact fragment is ‘gradient_clip_q15=gradient_clip_q15,‘. The important operands are gradient_clip_q15: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 819: `timeout_seconds=timeout_seconds,` This is continuation line 9 of the 10-line logical statement that begins at source line 811. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 820: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 811. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._projection_bias[name], gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=t)‘ to ‘self._deployed_projection_bias[name] ‘ in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _projection_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 821: `head_gradient = gradients["head"]` This assigns ‘gradients["head"]‘ to ‘head_gradient ‘ in host memory. Here head_gradient means the signed loss derivative used to update a learned parameter. The important operands are gradients: the signed loss derivative used to update a learned parameter; head: the value named ’head’ in this block. Context: the function apply_gradients, which handles apply gradients.

Line 822: `self._deployed_head_weight = self._update_tensor(` This statement continues through source line 831; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_d gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 823: `driver,` This is continuation line 2 of the 10-line logical statement that begins at source line 822. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15,

timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 824: `self._head_weight,` This is continuation line 3 of the 10-line logical statement that begins at source line 822. Its exact fragment is ‘self._head_weight,‘. The important operands are _head_weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 825: `head_gradient.weight,` This is continuation line 4 of the 10-line logical statement that begins at source line 822. Its exact fragment is ‘head_gradient.weight,‘. The important operands are head_gradient: the signed loss derivative used to update a learned parameter; weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 826: `learning_rate=learning_rate,` This is continuation line 5 of the 10-line logical statement that begins at source line 822. Its exact fragment is ‘learning_rate=learning_rate,‘. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 827: `momentum=momentum,` This is continuation line 6 of the 10-line logical statement that begins at source line 822. Its exact fragment is ‘momentum=momentum,‘. The important operands are momentum: the value named 'momentum' in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=g timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 828: `weight_decay=weight_decay,` This is continuation line 7 of the 10-line logical statement that begins at source line 822. Its exact fragment is ‘weight_decay=weight_decay,‘. The important operands are weight_decay: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_d gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 829: `gradient_clip_q15=gradient_clip_q15,` This is continuation line 8 of the 10-line logical statement that begins at source line 822. Its exact fragment is ‘gradient_clip_q15=gradient_clip_q15,‘. The important operands are gra-

dient_clip_q15: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 830: `timeout_seconds=timeout_seconds,` This is continuation line 9 of the 10-line logical statement that begins at source line 822. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 831: `).astype(np.int8)` This is continuation line 10 of the 10-line logical statement that begins at source line 822. Its exact fragment is ‘).astype(np.int8)‘. The important operands are astype: the value named ’astype’ in this block; int8: the value named ’int8’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_weight, head_gradient.weight, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_d gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,).astype(np.int8)‘ to ‘self._deployed_head_weight ‘ in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function apply_gradients, which handles apply gradients.

Line 832: `self._deployed_head_bias = self._update_tensor(` This statement continues through source line 841; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘self._deployed_head_bias ‘ in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 833: `driver,` This is continuation line 2 of the 10-line logical statement that begins at source line 832. Its exact fragment is ‘driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=t)‘ to ‘self._deployed_head_bias ‘ in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 834: `self._head_bias,` This is continuation line 3 of the 10-line logical statement that begins at source line 832. Its exact fragment is ‘self._head_bias,‘. The important operands are _head_bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)‘ to ‘self._deployed_head_bias ‘ in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named ’update tensor’ in this block; driver: the value named ’driver’ in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 835: `head_gradient.bias,` This is continuation line 4 of the 10-line logical statement that begins at source line 832. Its exact fragment is 'head_gradient.bias,'. The important operands are head_gradient: the signed loss derivative used to update a learned parameter; bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=t)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 836: `learning_rate=learning_rate,` This is continuation line 5 of the 10-line logical statement that begins at source line 832. Its exact fragment is 'learning_rate=learning_rate,'. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 837: `momentum=momentum,` This is continuation line 6 of the 10-line logical statement that begins at source line 832. Its exact fragment is 'momentum=momentum,'. The important operands are momentum: the value named 'momentum' in this block. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_ timeout_seconds=timeout_seconds,)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 838: `weight_decay=0.0,` This is continuation line 7 of the 10-line logical statement that begins at source line 832. Its exact fragment is 'weight_decay=0.0,'. The important operands are weight_decay: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 839: `gradient_clip_q15=gradient_clip_q15,` This is continuation line 8 of the 10-line logical statement that begins at source line 832. Its exact fragment is 'gradient_clip_q15=gradient_clip_q15,'. The important operands are gradient_clip_q15: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 840: `timeout_seconds=timeout_seconds,` This is continuation line 9 of the 10-line logical statement that begins at source line 832. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in

this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 841: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 832. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self._update_tensor(driver, self._head_bias, head_gradient.bias, learning_rate=learning_rate, momentum=momentum, weight_decay=0.0, gradient_clip_q15=gradient_clip_q15, timeout_seconds=timeout_seconds,)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are _update_tensor: the value named 'update tensor' in this block; driver: the value named 'driver' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function apply_gradients, which handles apply gradients.

Line 842: `self.optimizer_step += 1` Inside the function apply_gradients, which handles apply gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are optimizer_step: the value named 'optimizer step' in this block.

Line 844: `def _checkpoint_tensors(self) -> dict[str, np.ndarray]:` This starts function _checkpoint_tensors. Its indented body performs checkpoint tensors as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 845: `tensors: dict[str, np.ndarray] = {}` This assigns '{}' to 'tensors' in host memory. Here tensors means the value named 'tensors' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _checkpoint_tensors, which handles checkpoint tensors.

Line 846: `for index, name in enumerate(self.projection_names):` This iterates index, name over 'enumerate(self.projection_names)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _checkpoint_tensors, which handles checkpoint tensors.

Line 847: `for kind, tensor in (` This line opens or closes a multi-line Python expression inside the function _checkpoint_tensors, which handles checkpoint tensors; the neighboring lines supply its arguments or elements.

Line 848: `("weight", self._projection_weight[name]),` Inside the function _checkpoint_tensors, which handles checkpoint tensors, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are weight: a learned coefficient multiplied by an activation; _projection_weight: a learned coefficient multiplied by an activation; name: the value named 'name' in this block.

Line 849: `("bias", self._projection_bias[name]),` Inside the function _checkpoint_tensors, which handles checkpoint tensors, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are bias: an additive learned offset applied after accumulation; _projection_bias: an additive learned offset applied after accumulation; name: the value named 'name' in this block.

Line 850: `):` Inside the function _checkpoint_tensors, which handles checkpoint tensors, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 851: `prefix = f"projection_{index}_{kind}"` This assigns 'f"projection_{index}_{kind}"' to 'prefix ' in host memory. Here prefix means the value named 'prefix' in this block. The important operands are f: the value named 'f' in this block; projection_: the value named 'projection' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function _checkpoint_tensors, which handles checkpoint tensors.

Line 852: `tensors[f"{prefix}_master"] = tensor.master` Inside the function _checkpoint_tensors, which handles checkpoint tensors, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tensors: the value named 'tensors' in this block; f: the value named 'f' in this block; prefix: the value named 'prefix' in this block.

Line 853: `tensors[f"{prefix}_velocity"] = tensor.velocity_q15` Inside the function _checkpoint_tensors, which handles checkpoint tensors, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tensors: the value named 'tensors' in this block; f: the value named 'f' in this block; prefix: the value named 'prefix' in this block.

Line 854: `tensors["head_weight_master"] = self._head_weight.master` Inside the function _checkpoint_tensors, which handles checkpoint tensors, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tensors: the value named 'tensors' in this block; head_weight_master: a learned coefficient multiplied by an activation; _head_weight: a learned coefficient multiplied by an activation.

Line 855: `tensors["head_weight_velocity"] = self._head_weight.velocity_q15` Inside the function _checkpoint_tensors, which handles checkpoint tensors, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tensors: the value named 'tensors' in this block; head_weight_velocity: a learned coefficient multiplied by an activation; _head_weight: a learned coefficient multiplied by an activation.

Line 856: `tensors["head_bias_master"] = self._head_bias.master` Inside the function _checkpoint_tensors, which handles checkpoint tensors, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tensors: the value named 'tensors' in this block; head_bias_master: an additive learned offset applied after accumulation; _head_bias: an additive learned offset applied after accumulation.

Line 857: `tensors["head_bias_velocity"] = self._head_bias.velocity_q15` Inside the function _checkpoint_tensors, which handles checkpoint tensors, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tensors: the value named 'tensors' in this block; head_bias_velocity: an additive learned offset applied after accumulation; _head_bias: an additive learned offset applied after accumulation.

Line 858: `return tensors` This returns ‘tensors‘ to the caller of the function _checkpoint_tensors, which handles checkpoint tensors, ending this invocation. The important operands are tensors: the value named 'tensors' in this block.

Line 860: `def save_checkpoint(` This starts function save_checkpoint. Its indented body performs save checkpoint as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 861: `self,` Inside the function save_checkpoint, which handles save checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 862: `path: str | Path,` Inside the function save_checkpoint, which handles save checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are path: the value named 'path' in this block; str: the value named 'str' in this block; Path: the value named 'path' in this block.

Line 863: `*,` Inside the function save_checkpoint, which handles save checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 864: `epoch: int,` Inside the function save_checkpoint, which handles save checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are epoch: the value named 'epoch' in this block.

Line 865: `batch_index: int,` Inside the function save_checkpoint, which handles save checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 866: `best_accuracy_percent: float,` Inside the function save_checkpoint, which handles save checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are best_accuracy_percent: the value named 'best accuracy percent' in this block; float: the value named 'float' in this block.

Line 867: `extra: Mapping[str, Any] | None = None,` This assigns 'None,' to 'extra' in host memory. Here extra means the value named 'extra' in this block. The important operands are None: the value named 'none' in this block. Context: the function save_checkpoint, which handles save checkpoint.

Line 868: `) -> str:` Inside the function save_checkpoint, which handles save checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are str: the value named 'str' in this block.

Line 869: `"""Atomically save complete masters, momentum, and resume coordinates."""` This is executable documentation attached to the function save_checkpoint, which handles save checkpoint. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 871: `target = Path(path).expanduser().resolve()` This assigns 'Path(path).expanduser().resolve()' to 'target ' in host memory. Here target means the value named 'target' in this block. The important operands are Path: the value named 'path' in this block; path: the value named 'path' in this block; expanduser: the value named 'expanduser' in this block. Context: the function save_checkpoint, which handles save checkpoint.

Line 872: `target.parent.mkdir(parents=True, exist_ok=True)` This calls 'target.parent.mkdir' with 'parents=True, exist_ok=True' inside the function save_checkpoint, which handles save checkpoint. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 873: `metadata = {` This statement continues through source line 891; the explanation uses the complete expression rather than only the first visual fragment. This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 874: `"format": CHECKPOINT_FORMAT,` This is continuation line 2 of the 19-line logical statement that begins at source line 873. Its exact fragment is '"format": CHECKPOINT_FORMAT,'. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits,

"optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }‘ to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are format: the value named ’format’ in this block; CHECKPOINT_FORMAT: the value named ’checkpoint format’ in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 875: `"state_format": MASTER_FORMAT,` This is continuation line 3 of the 19-line logical statement that begins at source line 873. Its exact fragment is ‘"state_format": MASTER_FORMAT,‘. The important operands are state_format: the finite-state-machine phase retained in flip-flops; MASTER_FORMAT: the value named ’master format’ in this block. Read together, the complete operation is: This assigns ‘{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_ "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }‘ to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are format: the value named ’format’ in this block; CHECKPOINT_FORMAT: the value named ’checkpoint format’ in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 876: `"base_binary_sha256": self.base_binary_sha256,` This is continuation line 4 of the 19-line logical statement that begins at source line 873. Its exact fragment is ‘"base_binary_sha256": self.base_binary_sha256,‘. The important operands are base_binary_sha256: the value named ’base binary sha256’ in this block. Read together, the complete operation is: This assigns ‘{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_ [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }‘ to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are format: the value named ’format’ in this block; CHECKPOINT_FORMAT: the value named ’checkpoint format’ in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 877: `"projection_names": list(self.projection_names),` This is continuation line 5 of the 19-line logical statement that begins at source line 873. Its exact fragment is ‘"projection_names": list(self.projection_names),‘. The important operands are projection_names: the value named ’projection names’ in this block; list: the value named ’list’ in this block. Read together, the complete operation is: This assigns ‘{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_ "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }‘ to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are format: the value named ’format’ in this block; CHECKPOINT_FORMAT: the value named ’checkpoint format’ in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 878: `"projection_weight_fractional_bits": [` This is continuation line 6 of the 19-line logical statement that begins at source line 873. Its exact fragment is ‘"projection_weight_fractional_bits": [‘. The important operands are projection_weight_fractional_bits: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_ [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }‘

to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are format: the value named ’format’ in this block; CHECKPOINT_FORMAT: the value named ’checkpoint format’ in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 879: `self._projection_weight[name].fractional_bits for name in self.projection_names` This is continuation line 7 of the 19-line logical statement that begins at source line 873. Its exact fragment is ‘self._projection_weight[name].fra for name in self.projection_names‘. The important operands are _projection_weight: a learned coefficient multiplied by an activation; name: the value named ’name’ in this block; fractional_bits: the value named ’fractional bits’ in this block. Read together, the complete operation is: This assigns ‘{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }‘ to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are format: the value named ’format’ in this block; CHECKPOINT_FORMAT: the value named ’checkpoint format’ in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 880: `],` This is continuation line 8 of the 19-line logical statement that begins at source line 873. Its exact fragment is ‘],‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_ [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }‘ to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are format: the value named ’format’ in this block; CHECKPOINT_FORMAT: the value named ’checkpoint format’ in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 881: `"projection_bias_fractional_bits": [` This is continuation line 9 of the 19-line logical statement that begins at source line 873. Its exact fragment is ‘"projection_bias_fractional_bits": [‘. The important operands are projection_bias_fractional_bits: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }‘ to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are format: the value named ’format’ in this block; CHECKPOINT_FORMAT: the value named ’checkpoint format’ in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 882: `self._projection_bias[name].fractional_bits for name in self.projection_names` This is continuation line 10 of the 19-line logical statement that begins at source line 873. Its exact fragment is ‘self._projection_bias[name].fraction for name in self.projection_names‘. The important operands are _projection_bias: an additive learned offset applied after accumulation; name: the value named ’name’ in this block; fractional_bits: the value named ’fractional bits’ in this block. Read together, the complete operation is: This assigns ‘{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }‘ to ‘metadata ‘ in host memory. Here metadata means the

value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 883: `],` This is continuation line 11 of the 19-line logical statement that begins at source line 873. Its exact fragment is '],'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 884: `"head_weight_fractional_bits": self._head_weight.fractional_bits,` This is continuation line 12 of the 19-line logical statement that begins at source line 873. Its exact fragment is '"head_weight_fractional_bits": self._head_weight.fractional_bits,'. The important operands are head_weight_fractional_bits: a learned coefficient multiplied by an activation; _head_weight: a learned coefficient multiplied by an activation; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 885: `"head_bias_fractional_bits": self._head_bias.fractional_bits,` This is continuation line 13 of the 19-line logical statement that begins at source line 873. Its exact fragment is '"head_bias_fractional_bits": self._head_bias.fract The important operands are head_bias_fractional_bits: an additive learned offset applied after accumulation; _head_bias: an additive learned offset applied after accumulation; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 886: `"optimizer_step": self.optimizer_step,` This is continuation line 14 of the 19-line logical statement that begins at source line 873. Its exact fragment is '"optimizer_step": self.optimizer_step,'. The important operands are optimizer_step: the value named 'optimizer step' in this block. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named

'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 887: `"epoch": int(epoch),` This is continuation line 15 of the 19-line logical statement that begins at source line 873. Its exact fragment is '"epoch": int(epoch),'. The important operands are epoch: the value named 'epoch' in this block. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 888: `"batch_index": int(batch_index),` This is continuation line 16 of the 19-line logical statement that begins at source line 873. Its exact fragment is '"batch_index": int(batch_index),'. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 889: `"best_accuracy_percent": float(best_accuracy_percent),` This is continuation line 17 of the 19-line logical statement that begins at source line 873. Its exact fragment is '"best_accuracy_percent": float(best_accuracy_percent),'. The important operands are best_accuracy_percent: the value named 'best accuracy percent' in this block; float: the value named 'float' in this block. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 890: `"extra": dict(extra or {}),` This is continuation line 18 of the 19-line logical statement that begins at source line 873. Its exact fragment is '"extra": dict(extra or {}),'. The important operands are extra: the value named 'extra' in this block; dict: the value named 'dict' in this block; or: the value named 'or' in this block. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_bits": [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight.fractional_bits, "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 891: `}` This is continuation line 19 of the 19-line logical statement that begins at source line 873. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "format": CHECKPOINT_FORMAT, "state_format": MASTER_FORMAT, "base_binary_sha256": self.base_binary_sha256, "projection_names": list(self.projection_names), "projection_weight_fractional_ [self._projection_weight[name].fractional_bits for name in self.projection_names], "projection_bias_fractional_bits": [self._projection_bias[name].fractional_bits for name in self.projection_names], "head_weight_fractional_bits": self._head_weight "head_bias_fractional_bits": self._head_bias.fractional_bits, "optimizer_step": self.optimizer_step, "epoch": int(epoch), "batch_index": int(batch_index), "best_accuracy_percent": float(best_accuracy_percent), "extra": dict(extra or {}), }' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are format: the value named 'format' in this block; CHECKPOINT_FORMAT: the value named 'checkpoint format' in this block; state_format: the finite-state-machine phase retained in flip-flops. Context: the function save_checkpoint, which handles save checkpoint.

Line 892: `temporary = target.with_name(target.name + ".tmp")` This assigns 'target.with_name(target.name + ".tmp")' to 'temporary ' in host memory. Here temporary means the value named 'temporary' in this block. The important operands are target: the value named 'target' in this block; with_name: the value named 'with name' in this block; name: the value named 'name' in this block. Context: the function save_checkpoint, which handles save checkpoint.

Line 893: `payload = self._checkpoint_tensors()` This assigns 'self._checkpoint_tensors()' to 'payload ' in host memory. Here payload means stream data belonging to the current command. The important operands are _checkpoint_tensors: the value named 'checkpoint tensors' in this block. Context: the function save_checkpoint, which handles save checkpoint.

Line 894: `payload["metadata_json"] = np.asarray(` This statement continues through source line 895; the explanation uses the complete expression rather than only the first visual fragment. Inside the function save_checkpoint, which handles save checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are payload: stream data belonging to the current command; metadata_json: the value named 'metadata json' in this block; asarray: the value named 'asarray' in this block.

Line 895: `json.dumps(metadata, sort_keys=True, separators=(",", ":")))` This is continuation line 2 of the 2-line logical statement that begins at source line 894. Its exact fragment is 'json.dumps(metadata, sort_keys=True, separators=(",", ":")))'. The important operands are json: the value named 'json' in this block; dumps: the value named 'dumps' in this block; metadata: the value named 'metadata' in this block. Read together, the complete operation is: Inside the function save_checkpoint, which handles save checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are payload: stream data belonging to the current command; metadata_json: the value named 'metadata json' in this block; asarray: the value named 'asarray' in this block.

Line 896: `with temporary.open("wb") as stream:` This opens a managed resource scope for 'temporary.open("wb") as stream' so files, tracing, or temporary state are finalized even if the function save_checkpoint, which handles save checkpoint raises an error.

Line 897: `np.savez_compressed(stream, **payload)` This calls 'np.savez_compressed' with 'stream, **payload' inside the function save_checkpoint, which handles save checkpoint. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 898: `stream.flush()` This calls 'stream.flush' with " inside the function save_checkpoint, which handles save checkpoint. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 899: `os.fsync(stream.fileno())` This calls 'os.fsync' with 'stream.fileno()' inside the function save_checkpoint, which handles save checkpoint. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 900: `os.replace(temporary, target)` This calls ‘os.replace‘ with ‘temporary, target‘ inside the function save_checkpoint, which handles save checkpoint. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 901: `return hashlib.sha256(target.read_bytes()).hexdigest()` This returns ‘hashlib.sha256(target.read_bytes()).he to the caller of the function save_checkpoint, which handles save checkpoint, ending this invocation. The important operands are hashlib: the value named ’hashlib’ in this block; sha256: the value named ’sha256’ in this block; target: the value named ’target’ in this block.

Line 903: `def load_checkpoint(self, path: str | Path) -> Mapping[str, Any]:` This starts function load_checkpoint. Its indented body performs load checkpoint as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 904: `"""Restore one complete checkpoint only after all metadata validates."""` This is executable documentation attached to the function load_checkpoint, which handles load checkpoint. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 906: `source = Path(path).expanduser().resolve()` This assigns ‘Path(path).expanduser().resolve()‘ to ‘source ‘ in host memory. Here source means the value named ’source’ in this block. The important operands are Path: the value named ’path’ in this block; path: the value named ’path’ in this block; expanduser: the value named ’expanduser’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 907: `with np.load(source, allow_pickle=False) as archive:` This opens a managed resource scope for ‘np.load(source, allow_pickle=False) as archive‘ so files, tracing, or temporary state are finalized even if the function load_checkpoint, which handles load checkpoint raises an error.

Line 908: `metadata = json.loads(str(archive["metadata_json"]))` This assigns ‘json.loads(str(archive["metadata_json"])) to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are json: the value named ’json’ in this block; loads: the value named ’loads’ in this block; str: the value named ’str’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 909: `if metadata.get("format") != CHECKPOINT_FORMAT:` This executes the following indented host-code branch only when ‘metadata.get("format") != CHECKPOINT_FORMAT‘ is true. The important operands are metadata: the value named ’metadata’ in this block; get: the value named ’get’ in this block; format: the value named ’format’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 910: `raise ValueError("unsupported training checkpoint format")` This rejects an invalid state by raising ‘ValueError("unsupported training checkpoint format")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 911: `if metadata.get("base_binary_sha256") != self.base_binary_sha256:` This executes the following indented host-code branch only when ‘metadata.get("base_binary_sha256") != self.base_binary_sha256‘ is true. The important operands are metadata: the value named ’metadata’ in this block; get: the value named ’get’ in this block; base_binary_sha256: the value named ’base binary sha256’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 912: `raise ValueError("checkpoint was initialized from a different parameter binary")` This rejects an invalid state by raising ‘ValueError("checkpoint was initialized from a different parameter binary")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 913: `if tuple(metadata.get("projection_names", ())) != self.projection_names:` This executes the following indented host-code branch only when ‘tuple(metadata.get("projection_names", ())) != self.projection_names‘ is true. The important operands are tuple: the value named ’tuple’ in this block; metadata: the value named ’metadata’ in this block; get: the value named ’get’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 914: `raise ValueError("checkpoint projection order does not match this graph")` This rejects an invalid state by raising 'ValueError("checkpoint projection order does not match this graph")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 915: `expected_weight_fractional = [` This statement continues through source line 917; the explanation uses the complete expression rather than only the first visual fragment. This assigns '[self._projection_weight[name].fractional_bits for name in self.projection_names]' to 'expected_weight_fractional ' in host memory. Here expected_weight_fractional means a learned coefficient multiplied by an activation. The important operands are _projection_weight: a learned coefficient multiplied by an activation; name: the value named 'name' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 916: `self._projection_weight[name].fractional_bits for name in self.projection_names` This is continuation line 2 of the 3-line logical statement that begins at source line 915. Its exact fragment is 'self._projection_weight[name].frac for name in self.projection_names'. The important operands are _projection_weight: a learned coefficient multiplied by an activation; name: the value named 'name' in this block; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This assigns '[self._projection_weight[name].fractional_bits for name in self.projection_names]' to 'expected_weight_fractional ' in host memory. Here expected_weight_fractional means a learned coefficient multiplied by an activation. The important operands are _projection_weight: a learned coefficient multiplied by an activation; name: the value named 'name' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 917: `]` This is continuation line 3 of the 3-line logical statement that begins at source line 915. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '[self._projection_weight[name].fractional_bits for name in self.projection_names]' to 'expected_weight_fractional ' in host memory. Here expected_weight_fractional means a learned coefficient multiplied by an activation. The important operands are _projection_weight: a learned coefficient multiplied by an activation; name: the value named 'name' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 918: `expected_bias_fractional = [` This statement continues through source line 920; the explanation uses the complete expression rather than only the first visual fragment. This assigns '[self._projection_bias[name].fractional_bits for name in self.projection_names]' to 'expected_bias_fractional ' in host memory. Here expected_bias_fractional means an additive learned offset applied after accumulation. The important operands are _projection_bias: an additive learned offset applied after accumulation; name: the value named 'name' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 919: `self._projection_bias[name].fractional_bits for name in self.projection_names` This is continuation line 2 of the 3-line logical statement that begins at source line 918. Its exact fragment is 'self._projection_bias[name].fractional for name in self.projection_names'. The important operands are _projection_bias: an additive learned offset applied after accumulation; name: the value named 'name' in this block; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This assigns '[self._projection_bias[name].fractional_bits for name in self.projection_names]' to 'expected_bias_fractional ' in host memory. Here expected_bias_fractional means an additive learned offset applied after accumulation. The important operands are _projection_bias: an additive learned offset applied after accumulation; name: the value named 'name' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 920: `]` This is continuation line 3 of the 3-line logical statement that begins at source line 918. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '[self._projection_bias[name].fractional_bits for name in self.projection_names]' to 'expected_bias_fractional ' in host memory. Here expected_bias_fractional means an additive learned offset applied after accumulation. The important operands are _projection_bias: an additive learned offset applied after accumulation; name: the value named 'name' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 921: `if metadata.get("projection_weight_fractional_bits") != expected_weight_fractional:` This executes the following indented host-code branch only when 'metadata.get("projection_weight_fractional_bits") != ex-

pected_weight_fractional‘ is true. The important operands are metadata: the value named 'metadata' in this block; get: the value named 'get' in this block; projection_weight_fractional_bits: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 922: `raise ValueError("checkpoint projection weight Q formats do not match")` This rejects an invalid state by raising ‘ValueError("checkpoint projection weight Q formats do not match")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 923: `if metadata.get("projection_bias_fractional_bits") != expected_bias_fractional:` This executes the following indented host-code branch only when ‘metadata.get("projection_bias_fractional_bits") != expected_bias_fracti is true. The important operands are metadata: the value named 'metadata' in this block; get: the value named 'get' in this block; projection_bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 924: `raise ValueError("checkpoint projection bias Q formats do not match")` This rejects an invalid state by raising ‘ValueError("checkpoint projection bias Q formats do not match")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 925: `if int(metadata.get("head_weight_fractional_bits", -1)) != self._head_weight.fractional_bits:` This executes the following indented host-code branch only when ‘int(metadata.get("head_weight_fractional_bits", -1)) != self._head_weight.fractional_bits‘ is true. The important operands are metadata: the value named 'metadata' in this block; get: the value named 'get' in this block; head_weight_fractional_bits: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 926: `raise ValueError("checkpoint head weight Q format does not match")` This rejects an invalid state by raising ‘ValueError("checkpoint head weight Q format does not match")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 927: `if int(metadata.get("head_bias_fractional_bits", -1)) != self._head_bias.fractional_bits:` This executes the following indented host-code branch only when ‘int(metadata.get("head_bias_fractional_bits", -1)) != self._head_bias.fractional_bits‘ is true. The important operands are metadata: the value named 'metadata' in this block; get: the value named 'get' in this block; head_bias_fractional_bits: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 928: `raise ValueError("checkpoint head bias Q format does not match")` This rejects an invalid state by raising ‘ValueError("checkpoint head bias Q format does not match")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 929: `staged: list[tuple[MasterTensor, np.ndarray, np.ndarray]] = []` This assigns ‘[]‘ to ‘staged‘ in host memory. Here staged means the value named 'staged' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function load_checkpoint, which handles load checkpoint.

Line 930: `for index, name in enumerate(self.projection_names):` This iterates index, name over ‘enumerate(self.projection_names)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function load_checkpoint, which handles load checkpoint.

Line 931: `for kind, tensor in (` This line opens or closes a multi-line Python expression inside the function load_checkpoint, which handles load checkpoint; the neighboring lines supply its arguments or elements.

Line 932: `("weight", self._projection_weight[name]),` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are weight: a learned coefficient multiplied by an activation; _projection_weight: a learned coefficient multiplied by an activation; name: the value named 'name' in this block.

Line 933: `("bias", self._projection_bias[name]),` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are bias: an additive learned offset applied after accumulation; _projection_bias: an additive learned offset applied after accumulation; name: the value named 'name' in this block.

Line 934: `):` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 935: `prefix = f"projection_{index}_{kind}"` This assigns ‘f"projection_{index}_{kind}"‘ to ‘prefix ‘ in host memory. Here prefix means the value named 'prefix' in this block. The important operands are f: the value named 'f' in this block; projection_: the value named 'projection' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function load_checkpoint, which handles load checkpoint.

Line 936: `master = np.asarray(archive[f"{prefix}_master"], dtype=np.int64)` This assigns ‘np.asarray(archive[f"{prefix dtype=np.int64)‘ to ‘master ‘ in host memory. Here master means the value named 'master' in this block. The important operands are asarray: the value named 'asarray' in this block; archive: the value named 'archive' in this block; f: the value named 'f' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 937: `velocity = np.asarray(archive[f"{prefix}_velocity"], dtype=np.int64)` This assigns ‘np.asarray(archive[f"{ dtype=np.int64)‘ to ‘velocity ‘ in host memory. Here velocity means the momentum history retained by the SGD optimizer. The important operands are asarray: the value named 'asarray' in this block; archive: the value named 'archive' in this block; f: the value named 'f' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 938: `if master.shape != tensor.master.shape or velocity.shape != tensor.master.shape:` This executes the following indented host-code branch only when ‘master.shape != tensor.master.shape or velocity.shape != tensor.master.shape‘ is true. The important operands are master: the value named 'master' in this block; shape: the value named 'shape' in this block; tensor: the value named 'tensor' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 939: `raise ValueError(f"checkpoint tensor shape mismatch for {name} {kind}")` This rejects an invalid state by raising ‘ValueError(f"checkpoint tensor shape mismatch for {name} {kind}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 940: `if (np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1)` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are any: the value named 'any' in this block; master: the value named 'master' in this block; or: the value named 'or' in this block.

Line 941: `or np.any(velocity < -(1 << 23))` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are or: the value named 'or' in this block; any: the value named 'any' in this block; velocity: the momentum history retained by the SGD optimizer.

Line 942: `or np.any(velocity > (1 << 23) - 1)):` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are or: the value named 'or' in this block; any: the value named 'any' in this block; velocity: the momentum history retained by the SGD optimizer.

Line 943: `raise ValueError(f"checkpoint tensor range mismatch for {name} {kind}")` This rejects an invalid state by raising ‘ValueError(f"checkpoint tensor range mismatch for {name} {kind}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 944: `staged.append((tensor, master.copy(), velocity.copy()))` This calls ‘staged.append‘ with ‘(tensor, master.copy(), velocity.copy())‘ inside the function load_checkpoint, which handles load checkpoint. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 945: `for prefix, tensor in (` This line opens or closes a multi-line Python expression inside the function load_checkpoint, which handles load checkpoint; the neighboring lines supply its arguments or elements.

Line 946: `("head_weight", self._head_weight),` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are head_weight: a learned coefficient multiplied by an activation; _head_weight: a learned coefficient multiplied by an activation.

Line 947: `("head_bias", self._head_bias),` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are head_bias: an additive learned offset applied after accumulation; _head_bias: an additive learned offset applied after accumulation.

Line 948: `):` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 949: `master = np.asarray(archive[f"{prefix}_master"], dtype=np.int64)` This assigns ‘np.asarray(archive[f"{prefix dtype=np.int64)‘ to ‘master ‘ in host memory. Here master means the value named ’master’ in this block. The important operands are asarray: the value named ’asarray’ in this block; archive: the value named ’archive’ in this block; f: the value named ’f’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 950: `velocity = np.asarray(archive[f"{prefix}_velocity"], dtype=np.int64)` This assigns ‘np.asarray(archive[f"{ dtype=np.int64)‘ to ‘velocity ‘ in host memory. Here velocity means the momentum history retained by the SGD optimizer. The important operands are asarray: the value named ’asarray’ in this block; archive: the value named ’archive’ in this block; f: the value named ’f’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 951: `if master.shape != tensor.master.shape or velocity.shape != tensor.master.shape:` This executes the following indented host-code branch only when ‘master.shape != tensor.master.shape or velocity.shape != tensor.master.shape‘ is true. The important operands are master: the value named ’master’ in this block; shape: the value named ’shape’ in this block; tensor: the value named ’tensor’ in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 952: `raise ValueError(f"checkpoint tensor shape mismatch for {prefix}")` This rejects an invalid state by raising ‘ValueError(f"checkpoint tensor shape mismatch for {prefix}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 953: `if (np.any(master < -(1 << 15)) or np.any(master > (1 << 15) - 1)` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are any: the value named ’any’ in this block; master: the value named ’master’ in this block; or: the value named ’or’ in this block.

Line 954: `or np.any(velocity < -(1 << 23))` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are or: the value named ’or’ in this block; any: the value named ’any’ in this block; velocity: the momentum history retained by the SGD optimizer.

Line 955: `or np.any(velocity > (1 << 23) - 1)):` Inside the function load_checkpoint, which handles load checkpoint, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are or: the value named 'or' in this block; any: the value named 'any' in this block; velocity: the momentum history retained by the SGD optimizer.

Line 956: `raise ValueError(f"checkpoint tensor range mismatch for {prefix}")` This rejects an invalid state by raising 'ValueError(f"checkpoint tensor range mismatch for {prefix}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 957: `staged.append((tensor, master.copy(), velocity.copy()))` This calls 'staged.append' with '(tensor, master.copy(), velocity.copy())' inside the function load_checkpoint, which handles load checkpoint. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 958: `for tensor, master, velocity in staged:` This iterates tensor, master, velocity over 'staged'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function load_checkpoint, which handles load checkpoint.

Line 959: `tensor.master = master.astype(np.int16)` This assigns 'master.astype(np.int16)' to 'tensor.master ' in host memory. Here master means the value named 'master' in this block. The important operands are master: the value named 'master' in this block; astype: the value named 'astype' in this block; int16: the value named 'int16' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 960: `tensor.velocity_q15 = velocity.astype(np.int32)` This assigns 'velocity.astype(np.int32)' to 'tensor.velocity_q15 ' in host memory. Here velocity_q15 means the momentum history retained by the SGD optimizer. The important operands are velocity: the momentum history retained by the SGD optimizer; astype: the value named 'astype' in this block; int32: the value named 'int32' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 961: `self.optimizer_step = int(metadata["optimizer_step"])` This assigns 'int(metadata["optimizer_step"])' to 'self.optimizer_step ' in host memory. Here optimizer_step means the value named 'optimizer step' in this block. The important operands are metadata: the value named 'metadata' in this block; optimizer_step: the value named 'optimizer step' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 962: `for name in self.projection_names:` This iterates name over 'self.projection_names'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function load_checkpoint, which handles load checkpoint.

Line 963: `weight = self._projection_weight[name]` This assigns 'self._projection_weight[name]' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are _projection_weight: a learned coefficient multiplied by an activation; name: the value named 'name' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 964: `bias = self._projection_bias[name]` This assigns 'self._projection_bias[name]' to 'bias ' in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are _projection_bias: an additive learned offset applied after accumulation; name: the value named 'name' in this block. Context: the function load_checkpoint, which handles load checkpoint.

Line 965: `self._deployed_projection_weight[name] = np.clip(` This statement continues through source line 967; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.clip(round_shift_signed(weight.master, weight.fractional_bits), -128, 127).astype(np.int8)' to 'self._deployed_projection_weight[name ' in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; weight: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 966: `round_shift_signed(weight.master, weight.fractional_bits), -128, 127` This is continuation line 2 of the 3-line logical statement that begins at source line 965. Its exact fragment is 'round_shift_signed(weight.master, weight.fractional_bits), -128, 127'. The important operands are round_shift_signed: the value named 'round shift signed' in this block; weight: a learned coefficient multiplied by an activation; master: the value named 'master' in this block. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(weight.master, weight.fractional_bits), -128, 127).astype(np.int8)' to 'self._deployed_projection_weight[name] ' in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; weight: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 967: `).astype(np.int8)` This is continuation line 3 of the 3-line logical statement that begins at source line 965. Its exact fragment is ').astype(np.int8)'. The important operands are astype: the value named 'astype' in this block; int8: the value named 'int8' in this block. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(weight.master, weight.fractional_bits), -128, 127).astype(np.int8)' to 'self._deployed_projection_weight[name ' in host memory. Here _deployed_projection_weight means a learned coefficient multiplied by an activation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; weight: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 968: `self._deployed_projection_bias[name] = np.clip(` This statement continues through source line 972; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.clip(round_shift_signed(bia bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_projection_bias[name] ' in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 969: `round_shift_signed(bias.master, bias.fractional_bits),` This is continuation line 2 of the 5-line logical statement that begins at source line 968. Its exact fragment is 'round_shift_signed(bias.master, bias.fractional_bits),'. The important operands are round_shift_signed: the value named 'round shift signed' in this block; bias: an additive learned offset applied after accumulation; master: the value named 'master' in this block. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(bias.master, bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_projection_bias[name] ' in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 970: `-(1 << 15),` This is continuation line 3 of the 5-line logical statement that begins at source line 968. Its exact fragment is '-(1 << 15),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(bias.master, bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_projection_bias[name] ' in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 971: `(1 << 15) - 1,` This is continuation line 4 of the 5-line logical statement that begins at source line 968. Its exact fragment is '(1 << 15) - 1,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(bias.master, bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_projection_bias[name] ' in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 972: `).astype(np.int64)` This is continuation line 5 of the 5-line logical statement that begins at source line 968. Its exact fragment is ').astype(np.int64)'. The important operands are astype: the value named 'astype'

in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(bias.master, bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_projection_bias[name] ' in host memory. Here _deployed_projection_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 973: `self._deployed_head_weight = np.clip(` This statement continues through source line 977; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.clip(round_shift_signed(self._head_we self._head_weight.fractional_bits), -128, 127,).astype(np.int8)' to 'self._deployed_head_weight ' in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 974: `round_shift_signed(self._head_weight.master, self._head_weight.fractional_bits),` This is continuation line 2 of the 5-line logical statement that begins at source line 973. Its exact fragment is 'round_shift_signed(self._head_we self._head_weight.fractional_bits),'. The important operands are round_shift_signed: the value named 'round shift signed' in this block; _head_weight: a learned coefficient multiplied by an activation; master: the value named 'master' in this block. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(self._head_weight.master, self._head_weight.fractional_bits), -128, 127,).astype(np.int8)' to 'self._deployed_head_weight ' in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 975: `-128,` This is continuation line 3 of the 5-line logical statement that begins at source line 973. Its exact fragment is '-128,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(self._head_weight.master, self._head_weight.fractional_bits), -128, 127,).astype(np.int8)' to 'self._deployed_head_weight ' in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 976: `127,` This is continuation line 4 of the 5-line logical statement that begins at source line 973. Its exact fragment is '127,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(self._head_weight.master, self._head_weight.fractional_bits), -128, 127,).astype(np.int8)' to 'self._deployed_head_weight ' in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 977: `).astype(np.int8)` This is continuation line 5 of the 5-line logical statement that begins at source line 973. Its exact fragment is ').astype(np.int8)'. The important operands are astype: the value named 'astype' in this block; int8: the value named 'int8' in this block. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(self._head_weight.master, self._head_weight.fractional_bits), -128, 127,).astype(np.int8)' to 'self._deployed_ ' in host memory. Here _deployed_head_weight means a learned coefficient multiplied by an activation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_weight: a learned coefficient multiplied by an activation. Context: the function load_checkpoint, which handles load checkpoint.

Line 978: `self._deployed_head_bias = np.clip(` This statement continues through source line 982; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.clip(round_shift_signed(self._head_bias.m self._head_bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 979: `round_shift_signed(self._head_bias.master, self._head_bias.fractional_bits),` This is continuation line 2 of the 5-line logical statement that begins at source line 978. Its exact fragment is 'round_shift_signed(self._head_bias.m self._head_bias.fractional_bits),'. The important operands are round_shift_signed: the value named 'round shift signed' in this block; _head_bias: an additive learned offset applied after accumulation; master: the value named 'master' in this block. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(self._head_bias.master, self._head_bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 980: `-(1 << 15),` This is continuation line 3 of the 5-line logical statement that begins at source line 978. Its exact fragment is '-(1 << 15),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(self._head_bias.master, self._head_bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 981: `(1 << 15) - 1,` This is continuation line 4 of the 5-line logical statement that begins at source line 978. Its exact fragment is '(1 << 15) - 1,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(self._head_bias.master, self._head_bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 982: `).astype(np.int64)` This is continuation line 5 of the 5-line logical statement that begins at source line 978. Its exact fragment is ').astype(np.int64)'. The important operands are astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'np.clip(round_shift_signed(self._head_bias.master, self._head_bias.fractional_bits), -(1 << 15), (1 << 15) - 1,).astype(np.int64)' to 'self._deployed_head_bias ' in host memory. Here _deployed_head_bias means an additive learned offset applied after accumulation. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; _head_bias: an additive learned offset applied after accumulation. Context: the function load_checkpoint, which handles load checkpoint.

Line 983: `return metadata` This returns 'metadata' to the caller of the function load_checkpoint, which handles load checkpoint, ending this invocation. The important operands are metadata: the value named 'metadata' in this block.

Line 986: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 987: `class LifTraceCache:` This defines class LifTraceCache, which groups data and methods for lif trace cache. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 988: `"""Forward values retained for one exact hard-reset LIF reverse pass."""` This is executable documentation attached to the class LifTraceCache, which handles lif trace cache. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 990: `input_q10: np.ndarray` Inside the class LifTraceCache, which handles lif trace cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are input_q10: the value named 'input q10' in this block; ndarray: the value named 'ndarray' in this block.

Line 991: `spikes: np.ndarray` Inside the class LifTraceCache, which handles lif trace cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are spikes: a binary event produced when a neuron crosses its threshold; ndarray: the value named 'ndarray' in this block.

Line 992: `pre_reset_membrane: np.ndarray` Inside the class LifTraceCache, which handles lif trace cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; ndarray: the value named 'ndarray' in this block.

Line 993: `threshold_q10: int` Inside the class LifTraceCache, which handles lif trace cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are threshold_q10: the membrane level at which a neuron emits a spike.

Line 996: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 997: `class TokenizerLayerCache:` This defines class TokenizerLayerCache, which groups data and methods for tokenizer layer cache. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 998: `"""One tokenizer projection and the LIF/pool operators that precede it."""` This is executable documentation attached to the class TokenizerLayerCache, which handles tokenizer layer cache. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1000: `index: int` Inside the class TokenizerLayerCache, which handles tokenizer layer cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry.

Line 1001: `projection_input: np.ndarray` Inside the class TokenizerLayerCache, which handles tokenizer layer cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are projection_input: the value named 'projection input' in this block; ndarray: the value named 'ndarray' in this block.

Line 1002: `projection_output: np.ndarray` Inside the class TokenizerLayerCache, which handles tokenizer layer cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are projection_output: the value named 'projection output' in this block; ndarray: the value named 'ndarray' in this block.

Line 1003: `lif: LifTraceCache | None` Inside the class TokenizerLayerCache, which handles tokenizer layer cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are lif: the value named 'lif' in this block; LifTraceCache: the value named 'lif trace cache' in this block; None: the value named 'none' in this block.

Line 1004: `pooled: bool` Inside the class TokenizerLayerCache, which handles tokenizer layer cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are pooled: the value named 'pooled' in this block; bool: the value named 'bool' in this block.

Line 1007: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 1008: `class AttentionCache:` This defines class AttentionCache, which groups data and methods for attention cache. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1009: `"""All state needed to reverse one spike-driven self-attention branch."""` This is executable documentation attached to the class AttentionCache, which handles attention cache. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1011: `block_index: int` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are block_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 1012: `input_shape: tuple[int, ...]` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are input_shape: the value named 'input shape' in this block; tuple: the value named 'tuple' in this block.

Line 1013: `projection_lif: LifTraceCache` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are projection_lif: the value named 'projection lif' in this block; LifTraceCache: the value named 'lif trace cache' in this block.

Line 1014: `token_spikes: np.ndarray` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are token_spikes: a binary event produced when a neuron crosses its threshold; ndarray: the value named 'ndarray' in this block.

Line 1015: `query_lif: LifTraceCache` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are query_lif: the value named 'query lif' in this block; LifTraceCache: the value named 'lif trace cache' in this block.

Line 1016: `key_lif: LifTraceCache` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are key_lif: the value named 'key lif' in this block; LifTraceCache: the value named 'lif trace cache' in this block.

Line 1017: `value_lif: LifTraceCache` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are value_lif: the value named 'value lif' in this block; LifTraceCache: the value named 'lif trace cache' in this block.

Line 1018: `query_heads: np.ndarray` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are query_heads: the value named 'query heads' in this block; ndarray: the value named 'ndarray' in this block.

Line 1019: `key_heads: np.ndarray` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are key_heads: the value named 'key heads' in this block; ndarray: the value named 'ndarray' in this block.

Line 1020: `value_heads: np.ndarray` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are value_heads: the value named 'value heads' in this block; ndarray: the value named 'ndarray' in this block.

Line 1021: `scores: np.ndarray` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are scores: the value named 'scores' in this block; ndarray: the value named 'ndarray' in this block.

Line 1022: `attention_values: np.ndarray` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are attention_values: the value named 'attention values' in this block; ndarray: the value named 'ndarray' in this block.

Line 1023: `attention_lif: LifTraceCache` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are attention_lif: the value named 'attention lif' in this block; LifTraceCache: the value named 'lif trace cache' in this block.

Line 1024: `projection_input: np.ndarray` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are projection_input: the value named 'projection input' in this block; ndarray: the value named 'ndarray' in this block.

Line 1025: `output: np.ndarray` Inside the class AttentionCache, which handles attention cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output: the value named 'output' in this block; ndarray: the value named 'ndarray' in this block.

Line 1028: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 1029: `class MlpCache:` This defines class MlpCache, which groups data and methods for mlp cache. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1030: `"""Forward state for the two learned MLP projections and their LIF nodes."""` This is executable documentation attached to the class MlpCache, which handles mlp cache. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1032: `block_index: int` Inside the class MlpCache, which handles mlp cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are block_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 1033: `input_shape: tuple[int, ...]` Inside the class MlpCache, which handles mlp cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are input_shape: the value named 'input shape' in this block; tuple: the value named 'tuple' in this block.

Line 1034: `input_lif: LifTraceCache` Inside the class MlpCache, which handles mlp cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are input_lif: the value named 'input lif' in this block; LifTraceCache: the value named 'lif trace cache' in this block.

Line 1035: `first_projection_input: np.ndarray` Inside the class MlpCache, which handles mlp cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are first_projection_input: the value named 'first projection input' in this block; ndarray: the value named 'ndarray' in this block.

Line 1036: `hidden_lif: LifTraceCache` Inside the class MlpCache, which handles mlp cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are hidden_lif: the value named 'hidden lif' in this block; LifTraceCache: the value named 'lif trace cache' in this block.

Line 1037: `second_projection_input: np.ndarray` Inside the class MlpCache, which handles mlp cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are second_projection_input: the value named 'second projection input' in this block; ndarray: the value named 'ndarray' in this block.

Line 1038: `output: np.ndarray` Inside the class MlpCache, which handles mlp cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output: the value named 'output' in this block; ndarray: the value named 'ndarray' in this block.

Line 1041: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 1042: `class TransformerBlockCache:` This defines class TransformerBlockCache, which groups data and methods for transformer block cache. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1043: `"""Attention, MLP, and the two residual values of one transformer block."""` This is executable documentation attached to the class TransformerBlockCache, which handles transformer block cache. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1045: `input_value: np.ndarray` Inside the class TransformerBlockCache, which handles transformer block cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are input_value: the value named 'input value' in this block; ndarray: the value named 'ndarray' in this block.

Line 1046: `attention: AttentionCache` Inside the class TransformerBlockCache, which handles transformer block cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are attention: the value named 'attention' in this block; AttentionCache: the value named 'attention cache' in this block.

Line 1047: `after_attention: np.ndarray` Inside the class TransformerBlockCache, which handles transformer block cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are after_attention: the value named 'after attention' in this block; ndarray: the value named 'ndarray' in this block.

Line 1048: `mlp: MlpCache` Inside the class TransformerBlockCache, which handles transformer block cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are mlp: the value named 'mlp' in this block; MlpCache: the value named 'mlp cache' in this block.

Line 1049: `output: np.ndarray` Inside the class TransformerBlockCache, which handles transformer block cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output: the value named 'output' in this block; ndarray: the value named 'ndarray' in this block.

Line 1052: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 1053: `class SpikingformerTrainingCache:` This defines class SpikingformerTrainingCache, which groups data and methods for spikingformer training cache. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1054: `"""Complete forward tape for one batch of the exported integer graph."""` This is executable documentation attached to the class SpikingformerTrainingCache, which handles spikingformer training cache. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1056: `normalized_image: np.ndarray` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are normalized_image: the value named 'normalized image' in this block; ndarray: the value named 'ndarray' in this block.

Line 1057: `image_u8: np.ndarray` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are image_u8: the value named 'image u8' in this block; ndarray: the value named 'ndarray' in this block.

Line 1058: `image_centered: np.ndarray` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are image_centered: the value named 'image centered' in this block; ndarray: the value named 'ndarray' in this block.

Line 1059: `tokenizer: tuple[TokenizerLayerCache, ...]` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tokenizer: the value named 'tokenizer' in this block; tuple: the value named 'tuple' in this block; TokenizerLayerCache: the value named 'tokenizer layer cache' in this block.

Line 1060: `blocks: tuple[TransformerBlockCache, ...]` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are blocks: the value named 'blocks' in this block; tuple: the value named 'tuple' in this block; TransformerBlockCache: the value named 'transformer block cache' in this block.

Line 1061: `final_value: np.ndarray` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are final_value: the value named 'final value' in this block; ndarray: the value named 'ndarray' in this block.

Line 1062: `head_preclip: np.ndarray` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are head_preclip: the value named 'head preclip' in this block; ndarray: the value named 'ndarray' in this block.

Line 1063: `head_input: np.ndarray` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are head_input: the value named 'head input' in this block; ndarray: the value named 'ndarray' in this block.

Line 1064: `logits_accumulator: np.ndarray` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are logits_accumulator: a widened running sum that prevents intermediate overflow; ndarray: the value named 'ndarray' in this block.

Line 1065: `logits: np.ndarray` Inside the class SpikingformerTrainingCache, which handles spikingformer training cache, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are logits: the ten unnormalized classifier scores used for argmax; ndarray: the value named 'ndarray' in this block.

Line 1068: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 1069: `class TrainingBackwardResult:` This defines class TrainingBackwardResult, which groups data and methods for training backward result. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1070: `"""Loss, classifier probabilities, all parameter gradients, and input dX."""` This is executable documentation attached to the class TrainingBackwardResult, which handles training backward result. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1072: `loss: float` Inside the class TrainingBackwardResult, which handles training backward result, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are loss: the value named 'loss' in this block; float: the value named 'float' in this block.

Line 1073: `probabilities_q15: np.ndarray` Inside the class TrainingBackwardResult, which handles training backward result, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are probabilities_q15: the value named 'probabilities q15' in this block; ndarray: the value named 'ndarray' in this block.

Line 1074: `predicted_class: np.ndarray` Inside the class TrainingBackwardResult, which handles training backward result, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are predicted_class: the value named 'predicted class' in this block; ndarray: the value named 'ndarray' in this block.

Line 1075: `parameter_gradients: Mapping[str, ParameterGradient]` Inside the class TrainingBackwardResult, which handles training backward result, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are parameter_gradients: the signed loss derivative used to update a learned parameter; Mapping: the value named 'mapping' in this block; str: the value named 'str' in this block.

Line 1076: `input_gradient: BlockGradient` Inside the class TrainingBackwardResult, which handles training backward result, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1079: `@dataclass(frozen=True)` This decorator changes how the following class or function is constructed. '@dataclass(frozen=True)' is applied at import time on the host and is part of the file whose role is to runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it creates no FPGA circuit by itself.

Line 1080: `class TrainingStepResult:` This defines class TrainingStepResult, which groups data and methods for training step result. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1081: `"""Observable result of one complete forward/backward/SGDW transaction."""` This is executable documentation attached to the class TrainingStepResult, which handles training step result. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1083: `loss: float` Inside the class TrainingStepResult, which handles training step result, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are loss: the value named 'loss' in this block; float: the value named 'float' in this block.

Line 1084: `probabilities_q15: np.ndarray` Inside the class TrainingStepResult, which handles training step result, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are probabilities_q15: the value named 'probabilities q15' in this block; ndarray: the value named 'ndarray' in this block.

Line 1085: `predicted_class: np.ndarray` Inside the class TrainingStepResult, which handles training step result, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are predicted_class: the value named 'predicted class' in this block; ndarray: the value named 'ndarray' in this block.

Line 1086: `optimizer_step: int` Inside the class TrainingStepResult, which handles training step result, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are optimizer_step: the value named 'optimizer step' in this block.

Line 1089: `class SpikingformerOnDeviceTrainer:` This defines class SpikingformerOnDeviceTrainer, which groups data and methods for spikingformer on device trainer. Instances exist in host memory and organize the flow that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1090: `"""Execute and train the complete exported Spikingformer graph on NeuraEdge."""` This is executable documentation attached to the class SpikingformerOnDeviceTrainer, which handles spikingformer on device trainer. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1092: `def __init__(` This starts function ___init___. Its indented body performs init as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1093: `self,` Inside the function ___init___, which handles init, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1094: `state: TrainableSpikingformerState,` Inside the function ___init___, which handles init, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are state: the finite-state-machine phase retained in flip-flops; TrainableSpikingformerState: the finite-state-machine phase retained in flip-flops.

Line 1095: `driver: FullTrainingDriver,` Inside the function ___init___, which handles init, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are driver: the value named 'driver' in this block; FullTrainingDriver: the value named 'full training driver' in this block.

Line 1096: `*,` Inside the function ___init___, which handles init, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1097: `transformer_blocks: int = 4,` This assigns '4,' to 'transformer_blocks' in host memory. Here transformer_blocks means the value named 'transformer blocks' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 1098: `attention_heads: int = 12,` This assigns '12,' to 'attention_heads' in host memory. Here attention_heads means the value named 'attention heads' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 1099: `timesteps: int = 4,` This assigns '4,' to 'timesteps' in host memory. Here timesteps means one discrete SNN update instant. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 1100: `input_shape: tuple[int, int, int] = (3, 32, 32),` This assigns '(3, 32, 32),' to 'input_shape' in host memory. Here input_shape means the value named 'input shape' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 1101: `layer_timeout_seconds: float = 120.0,` This assigns '120.0,' to 'layer_timeout_seconds' in host memory. Here layer_timeout_seconds means the value named 'layer timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 1102: `) -> None:` Inside the function ___init___, which handles init, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are None: the value named 'none' in this block.

Line 1103: `if transformer_blocks <= 0 or attention_heads <= 0 or timesteps <= 0:` This executes the following indented host-code branch only when ‘transformer_blocks <= 0 or attention_heads <= 0 or timesteps <= 0‘ is true. The important operands are transformer_blocks: the value named ’transformer blocks’ in this block; or: the value named ’or’ in this block; attention_heads: the value named ’attention heads’ in this block. Context: the function ___init___, which handles init.

Line 1104: `raise ValueError("block, head, and timestep counts must be positive")` This rejects an invalid state by raising ‘ValueError("block, head, and timestep counts must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1105: `if len(input_shape) != 3 or min(input_shape) <= 0:` This executes the following indented host-code branch only when ‘len(input_shape) != 3 or min(input_shape) <= 0‘ is true. The important operands are len: the value named ’len’ in this block; input_shape: the value named ’input shape’ in this block; or: the value named ’or’ in this block. Context: the function ___init___, which handles init.

Line 1106: `raise ValueError("input_shape must contain positive C,H,W dimensions")` This rejects an invalid state by raising ‘ValueError("input_shape must contain positive C,H,W dimensions")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1107: `if layer_timeout_seconds <= 0:` This executes the following indented host-code branch only when ‘layer_timeout_seconds <= 0‘ is true. The important operands are layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Context: the function ___init___, which handles init.

Line 1108: `raise ValueError("layer_timeout_seconds must be positive")` This rejects an invalid state by raising ‘ValueError("layer_timeout_seconds must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1109: `self.state = state` This assigns ‘state‘ to ‘self.state ‘ in host memory. Here state means the finite-state-machine phase retained in flip-flops. The important operands are state: the finite-state-machine phase retained in flip-flops. Context: the function ___init___, which handles init.

Line 1110: `self.driver = driver` This assigns ‘driver‘ to ‘self.driver ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are driver: the value named ’driver’ in this block. Context: the function ___init___, which handles init.

Line 1111: `self.projections = SpikingformerProjectionExecutor(state, driver)` This assigns ‘SpikingformerProjectionExecutor(state, driver)‘ to ‘self.projections ‘ in host memory. Here projections means the value named ’projections’ in this block. The important operands are SpikingformerProjectionExecutor: the value named ’spikingformer projection executor’ in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named ’driver’ in this block. Context: the function ___init___, which handles init.

Line 1112: `self.transformer_blocks = int(transformer_blocks)` This assigns ‘int(transformer_blocks)‘ to ‘self.transformer ‘ in host memory. Here transformer_blocks means the value named ’transformer blocks’ in this block. The important operands are transformer_blocks: the value named ’transformer blocks’ in this block. Context: the function ___init___, which handles init.

Line 1113: `self.attention_heads = int(attention_heads)` This assigns ‘int(attention_heads)‘ to ‘self.attention_heads ‘ in host memory. Here attention_heads means the value named ’attention heads’ in this block. The important operands are attention_heads: the value named ’attention heads’ in this block. Context: the function ___init___, which handles init.

Line 1114: `self.timesteps = int(timesteps)` This assigns ‘int(timesteps)‘ to ‘self.timesteps ‘ in host memory. Here timesteps means one discrete SNN update instant. The important operands are timesteps: one discrete SNN update instant. Context: the function ___init___, which handles init.

Line 1115: `self.input_shape = tuple(map(int, input_shape))` This assigns ‘tuple(map(int, input_shape))‘ to ‘self.input_shape ‘ in host memory. Here input_shape means the value named ’input shape’ in this block. The important operands are tuple: the value named ’tuple’ in this block; map: the value named ’map’ in this block; input_shape: the value named ’input shape’ in this block. Context: the function ___init___, which handles init.

Line 1116: `self.layer_timeout_seconds = float(layer_timeout_seconds)` This assigns ‘float(layer_timeout_seconds)‘ to ‘self.layer_timeout_seconds ‘ in host memory. Here layer_timeout_seconds means the value named ’layer timeout seconds’ in this block. The important operands are float: the value named ’float’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Context: the function ___init___, which handles init.

Line 1117: `self._surrogate_lut_loaded = False` This assigns ‘False‘ to ‘self._surrogate_lut_loaded ‘ in host memory. Here _surrogate_lut_loaded means the value named ’surrogate lut loaded’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 1118: `fixed = state.manifest.get("fixed_point")` This assigns ‘state.manifest.get("fixed_point")‘ to ‘fixed ‘ in host memory. Here fixed means the value named ’fixed’ in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; manifest: the value named ’manifest’ in this block; get: the value named ’get’ in this block. Context: the function ___init___, which handles init.

Line 1119: `if not isinstance(fixed, Mapping):` This executes the following indented host-code branch only when ‘not isinstance(fixed, Mapping)‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; fixed: the value named ’fixed’ in this block. Context: the function ___init___, which handles init.

Line 1120: `raise ValueError("trainable state has no fixed-point manifest")` This rejects an invalid state by raising ‘ValueError("trainable state has no fixed-point manifest")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1121: `self.fractional_bits = int(fixed["activation_fractional_bits"])` This assigns ‘int(fixed["activation_fracti
to ‘self.fractional_bits ‘ in host memory. Here fractional_bits means the value named ’fractional bits’ in this block. The important operands are fixed: the value named ’fixed’ in this block; activation_fractional_bits: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function ___init___, which handles init.

Line 1122: `self.fixed_scale = 1 << self.fractional_bits` This assigns ‘1 << self.fractional_bits‘ to ‘self.fixed_scale ‘ in host memory. Here fixed_scale means the real-value spacing represented by one quantized integer step. The important operands are fractional_bits: the value named ’fractional bits’ in this block. Context: the function ___init___, which handles init.

Line 1123: `if self.fixed_scale % 8:` This executes the following indented host-code branch only when ‘self.fixed_scale % 8‘ is true. The important operands are fixed_scale: the real-value spacing represented by one quantized integer step. Context: the function ___init___, which handles init.

Line 1124: `raise ValueError("attention scaling requires at least three activation fractional bits")` This rejects an invalid state by raising ‘ValueError("attention scaling requires at least three activation fractional bits")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1125: `attention_scale = self.fixed_scale // 8` This assigns ‘self.fixed_scale // 8‘ to ‘attention_scale ‘ in host memory. Here attention_scale means the real-value spacing represented by one quantized integer step. The important operands are fixed_scale: the real-value spacing represented by one quantized integer step. Context: the function ___init___, which handles init.

Line 1126: `if attention_scale <= 0 or attention_scale & (attention_scale - 1):` This executes the following indented host-code branch only when ‘attention_scale <= 0 or attention_scale & (attention_scale - 1)‘ is true. The important operands are attention_scale: the real-value spacing represented by one quantized integer step; or: the value named ’or’ in this block. Context: the function ___init___, which handles init.

Line 1127: `raise ValueError("attention scaling must be an exact power of two")` This rejects an invalid state by raising ‘ValueError("attention scaling must be an exact power of two")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1128: `self.attention_scale_shift = attention_scale.bit_length() - 1` This assigns ‘attention_scale.bit_length() - 1‘ to ‘self.attention_scale_shift ‘ in host memory. Here attention_scale_shift means the real-value spacing represented by one quantized integer step. The important operands are attention_scale: the real-value spacing represented by one quantized integer step; bit_length: the value named ’bit length’ in this block. Context: the function ___init___, which handles init.

Line 1129: `self.image_scale = float(fixed["image_input_scale"])` This assigns ‘float(fixed["image_input_scale"])‘ to ‘self.image_scale ‘ in host memory. Here image_scale means the real-value spacing represented by one quantized integer step. The important operands are float: the value named ’float’ in this block; fixed: the value named ’fixed’ in this block; image_input_scale: the real-value spacing represented by one quantized integer step. Context: the function ___init___, which handles init.

Line 1130: `self.image_zero_point = int(fixed["image_input_zero_point"])` This assigns ‘int(fixed["image_input_zero_
to ‘self.image_zero_point ‘ in host memory. Here image_zero_point means the integer code representing real numeric zero in affine quantization. The important operands are fixed: the value named ’fixed’ in this block; image_input_zero_point: the integer code representing real numeric zero in affine quantization. Context: the function ___init___, which handles init.

Line 1131: `self.head_activation_scale = float(fixed["head_activation_scale"])` This assigns ‘float(fixed["head_activa
to ‘self.head_activation_scale ‘ in host memory. Here head_activation_scale means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are float: the value named ’float’ in this block; fixed: the value named ’fixed’ in this block; head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function ___init___, which handles init.

Line 1132: `required = {` This statement continues through source line 1139; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(self.transformer_blocks) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(self.transformer_blocks) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 1133: `*(f"patch_embed.block{index}" for index in range(5)),` This is continuation line 2 of the 8-line logical statement that begins at source line 1132. Its exact fragment is ‘*(f"patch_embed.block{index}" for index in range(5)),‘. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(self.transformer_blocks) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(self.transformer
for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 1134: `*(f"block.{block}.attn.{name}"` This is continuation line 3 of the 8-line logical statement that begins at source line 1132. Its exact fragment is ‘*(f"block.{block}.attn.{name}"‘. The important operands are f: the value named ’f’ in this block; block: the value named ’block’ in this block; attn: the value named ’attn’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(self.transformer_blocks) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.f
for block in range(self.transformer_blocks) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 1135: `for block in range(self.transformer_blocks)` This is continuation line 4 of the 8-line logical statement that begins at source line 1132. Its exact fragment is ‘for block in range(self.transformer_blocks)‘. The important operands are for: the value named ’for’ in this block; block: the value named ’block’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(self.transformer_blocks) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(self.transformer_blocks) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 1136: `for name in ("q", "k", "v", "proj")),` This is continuation line 5 of the 8-line logical statement that begins at source line 1132. Its exact fragment is ‘for name in ("q", "k", "v", "proj")),‘. The important operands are for: the value named ’for’ in this block; name: the value named ’name’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(self.transformer_blocks) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(self.transformer_blocks) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 1137: `*(f"block.{block}.mlp.fc{layer}"` This is continuation line 6 of the 8-line logical statement that begins at source line 1132. Its exact fragment is ‘*(f"block.{block}.mlp.fc{layer}"‘. The important operands are f: the value named ’f’ in this block; block: the value named ’block’ in this block; mlp: the value named ’mlp’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(self.transformer_blocks) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(self.transformer_blocks) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 1138: `for block in range(self.transformer_blocks) for layer in (1, 2)),` This is continuation line 7 of the 8-line logical statement that begins at source line 1132. Its exact fragment is ‘for block in range(self.transformer_blocks) for layer in (1, 2)),‘. The important operands are for: the value named ’for’ in this block; block: the value named ’block’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(self.transformer_blocks) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(self.transformer_blocks) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 1139: `}` This is continuation line 8 of the 8-line logical statement that begins at source line 1132. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ *(f"patch_embed.block{index}" for index in range(5)), *(f"block.{block}.attn.{name}" for block in range(self.transformer_blocks) for name in ("q", "k", "v", "proj")), *(f"block.{block}.mlp.fc{layer}" for block in range(self.transformer_blocks) for layer in (1, 2)), }‘ to ‘required ‘ in host memory. Here required means the value named ’required’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function ___init___, which handles init.

Line 1140: `missing = required - set(state.projection_names)` This assigns ‘required - set(state.projection_names)‘ to ‘missing ‘ in host memory. Here missing means the value named ’missing’ in this block. The important operands are required: the value named ’required’ in this block; set: the value named ’set’ in this block; state: the finite-state-machine phase retained in flip-flops. Context: the function ___init___, which handles init.

Line 1141: `if missing:` This executes the following indented host-code branch only when ‘missing‘ is true. The important operands are missing: the value named ’missing’ in this block. Context: the function ___init___, which

handles init.

Line 1142: `raise ValueError(f"trainable state is missing projections: {sorted(missing)}")` This rejects an invalid state by raising ‘ValueError(f"trainable state is missing projections: {sorted(missing)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1143: `accumulator_scale = float(state.manifest["head"]["accumulator_scale"])` This assigns ‘float(state.manifes to ‘accumulator_scale ‘ in host memory. Here accumulator_scale means a widened running sum that prevents intermediate overflow. The important operands are float: the value named ’float’ in this block; state: the finite-state-machine phase retained in flip-flops; manifest: the value named ’manifest’ in this block. Context: the function ___init___, which handles init.

Line 1144: `ideal_difference = math.log(2.0) / accumulator_scale` This assigns ‘math.log(2.0) / accumulator_scale‘ to ‘ideal_difference ‘ in host memory. Here ideal_difference means the value named ’ideal difference’ in this block. The important operands are math: the value named ’math’ in this block; log: the value named ’log’ in this block; accumulator_scale: a widened running sum that prevents intermediate overflow. Context: the function ___init___, which handles init.

Line 1145: `self.softmax_difference_shift = int(round(math.log2(ideal_difference)))` This assigns ‘int(round(math.l to ‘self.softmax_difference_shift ‘ in host memory. Here softmax_difference_shift means the value named ’softmax difference shift’ in this block. The important operands are round: the value named ’round’ in this block; math: the value named ’math’ in this block; log2: the value named ’log2’ in this block. Context: the function ___init___, which handles init.

Line 1146: `if not 0 <= self.softmax_difference_shift <= 31:` This executes the following indented host-code branch only when ‘not 0 <= self.softmax_difference_shift <= 31‘ is true. The important operands are not: the value named ’not’ in this block; softmax_difference_shift: the value named ’softmax difference shift’ in this block. Context: the function ___init___, which handles init.

Line 1147: `raise ValueError("head accumulator scale is outside the hardware softmax range")` This rejects an invalid state by raising ‘ValueError("head accumulator scale is outside the hardware softmax range")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1149: `def _lif_trace(self, values: np.ndarray, *, threshold: float = 1.0) -> LifTraceCache:` This starts function _lif_trace. Its indented body performs lif trace as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1150: `threshold_q10 = int(round(threshold * self.fixed_scale))` This assigns ‘int(round(threshold * self.fixed_scale))‘ to ‘threshold_q10 ‘ in host memory. Here threshold_q10 means the membrane level at which a neuron emits a spike. The important operands are round: the value named ’round’ in this block; threshold: the membrane level at which a neuron emits a spike; fixed_scale: the real-value spacing represented by one quantized integer step. Context: the function _lif_trace, which handles lif trace.

Line 1151: `spikes, membrane = lif_sequence_with_trace_on_accelerator(` This statement continues through source line 1158; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘lif_sequence_with_trace_on_accelerator(self.driver, values, threshold_q10=threshold_q10, leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spikes, membrane ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named ’lif sequence with trace on accelerator’ in this block; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _lif_trace, which handles lif trace.

Line 1152: `self.driver,` This is continuation line 2 of the 8-line logical statement that begins at source line 1151. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read

together, the complete operation is: This assigns ‘lif_sequence_with_trace_on_accelerator(self.driver, values, threshold_q10=threshold_q10, leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spikes, membrane ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named ’lif sequence with trace on accelerator’ in this block; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _lif_trace, which handles lif trace.

Line 1153: `values,` This is continuation line 3 of the 8-line logical statement that begins at source line 1151. Its exact fragment is ‘values,‘. The important operands are values: the value named ’values’ in this block. Read together, the complete operation is: This assigns ‘lif_sequence_with_trace_on_accelerator(self.driver, values, threshold_q10=threshold_q10, leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spikes, membrane ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named ’lif sequence with trace on accelerator’ in this block; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _lif_trace, which handles lif trace.

Line 1154: `threshold_q10=threshold_q10,` This is continuation line 4 of the 8-line logical statement that begins at source line 1151. Its exact fragment is ‘threshold_q10=threshold_q10,‘. The important operands are threshold_q10: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns ‘lif_sequence_with_trace_on_accelerator(self.driver, values, threshold_q10=threshold_q10, leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spikes, membrane ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named ’lif sequence with trace on accelerator’ in this block; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _lif_trace, which handles lif trace.

Line 1155: `leak_shift=1,` This is continuation line 5 of the 8-line logical statement that begins at source line 1151. Its exact fragment is ‘leak_shift=1,‘. The important operands are leak_shift: the retained fraction of prior membrane state. Read together, the complete operation is: This assigns ‘lif_sequence_with_trace_on_accelerator(self.driver, values, threshold_q10=threshold_q10, leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spikes, membrane ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named ’lif sequence with trace on accelerator’ in this block; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _lif_trace, which handles lif trace.

Line 1156: `reset_q10=0,` This is continuation line 6 of the 8-line logical statement that begins at source line 1151. Its exact fragment is ‘reset_q10=0,‘. The important operands are reset_q10: the value named ’reset q10’ in this block. Read together, the complete operation is: This assigns ‘lif_sequence_with_trace_on_accelerator(self.driver, values, threshold_q10=threshold_q10, leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spikes, membrane ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named ’lif sequence with trace on accelerator’ in this block; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _lif_trace, which handles lif trace.

Line 1157: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 7 of the 8-line logical statement that begins at source line 1151. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘lif_sequence_with_trace_on_ac self.driver, values, threshold_q10=threshold_q10, leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spikes, membrane ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named ’lif sequence with trace on accelerator’ in this block; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _lif_trace, which handles lif trace.

Line 1158: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 1151. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘lif_sequence_with_trace_on_accelerator(self.driver, values, threshold_q10=threshold_q10,

leak_shift=1, reset_q10=0, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spikes, membrane ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named ’lif sequence with trace on accelerator’ in this block; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _lif_trace, which handles lif trace.

Line 1159: `return LifTraceCache(` This statement continues through source line 1164; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘LifTraceCache(np.asarray(values, dtype=np.int64), spikes, membrane, threshold_q10,)‘ to the caller of the function _lif_trace, which handles lif trace, ending this invocation. The important operands are LifTraceCache: the value named ’lif trace cache’ in this block; asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block.

Line 1160: `np.asarray(values, dtype=np.int64),` This is continuation line 2 of the 6-line logical statement that begins at source line 1159. Its exact fragment is ‘np.asarray(values, dtype=np.int64),‘. The important operands are asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block; dtype: the value named ’dtype’ in this block. Read together, the complete operation is: This returns ‘LifTraceCache(np.asarray(values, dtype=np.int64), spikes, membrane, threshold_q10,)‘ to the caller of the function _lif_trace, which handles lif trace, ending this invocation. The important operands are LifTraceCache: the value named ’lif trace cache’ in this block; asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block.

Line 1161: `spikes,` This is continuation line 3 of the 6-line logical statement that begins at source line 1159. Its exact fragment is ‘spikes,‘. The important operands are spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This returns ‘LifTraceCache(np.asarray(values, dtype=np.int64), spikes, membrane, threshold_q10,)‘ to the caller of the function _lif_trace, which handles lif trace, ending this invocation. The important operands are LifTraceCache: the value named ’lif trace cache’ in this block; asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block.

Line 1162: `membrane,` This is continuation line 4 of the 6-line logical statement that begins at source line 1159. Its exact fragment is ‘membrane,‘. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This returns ‘LifTraceCache(np.asarray(values, dtype=np.int64), spikes, membrane, threshold_q10,)‘ to the caller of the function _lif_trace, which handles lif trace, ending this invocation. The important operands are LifTraceCache: the value named ’lif trace cache’ in this block; asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block.

Line 1163: `threshold_q10,` This is continuation line 5 of the 6-line logical statement that begins at source line 1159. Its exact fragment is ‘threshold_q10,‘. The important operands are threshold_q10: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This returns ‘LifTraceCache(np.asarray(values, dtype=np.int64), spikes, membrane, threshold_q10,)‘ to the caller of the function _lif_trace, which handles lif trace, ending this invocation. The important operands are LifTraceCache: the value named ’lif trace cache’ in this block; asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block.

Line 1164: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1159. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘LifTraceCache(np.asarray(values, dtype=np.int64), spikes, membrane, threshold_q10,)‘ to the caller of the function _lif_trace, which handles lif trace, ending this invocation. The important operands are LifTraceCache: the value named ’lif trace cache’ in this block; asarray: the value named ’asarray’ in this block; values: the value named ’values’ in this block.

Line 1166: `def _dynamic_binary_gemm(self, left: np.ndarray, right: np.ndarray) -> np.ndarray:` This starts function _dynamic_binary_gemm. Its indented body performs dynamic binary gemm as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1167: `return compile_spike_gemm(left, right).execute_batched(` This statement continues through source line 1168; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘compile_spike_gemm(left, right).execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)‘ to the caller

of the function _dynamic_binary_gemm, which handles dynamic binary gemm, ending this invocation. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold; left: the value named 'left' in this block; right: the value named 'right' in this block.

Line 1168: `self.driver, timeout_seconds=self.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 1167. Its exact fragment is 'self.driver, timeout_seconds=self.layer_timeout_seconds)'. The important operands are driver: the value named 'driver' in this block; timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This returns 'compile_spike_gemm(left, right).execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)' to the caller of the function _dynamic_binary_gemm, which handles dynamic binary gemm, ending this invocation. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold; left: the value named 'left' in this block; right: the value named 'right' in this block.

Line 1170: `def _dynamic_u8_gemm(self, left: np.ndarray, right: np.ndarray) -> np.ndarray:` This starts function _dynamic_u8_gemm. Its indented body performs dynamic u8 gemm as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1171: `return compile_u8_gemm(left, right).execute_batched(` This statement continues through source line 1172; the explanation uses the complete expression rather than only the first visual fragment. This returns 'compile_u8_gemm(left, right).execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)' to the caller of the function _dynamic_u8_gemm, which handles dynamic u8 gemm, ending this invocation. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; left: the value named 'left' in this block; right: the value named 'right' in this block.

Line 1172: `self.driver, timeout_seconds=self.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 1171. Its exact fragment is 'self.driver, timeout_seconds=self.layer_timeout_seconds)'. The important operands are driver: the value named 'driver' in this block; timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This returns 'compile_u8_gemm(left, right).execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)' to the caller of the function _dynamic_u8_gemm, which handles dynamic u8 gemm, ending this invocation. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; left: the value named 'left' in this block; right: the value named 'right' in this block.

Line 1174: `def _global_sum(self, value: np.ndarray) -> np.ndarray:` This starts function _global_sum. Its indented body performs global sum as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1175: `"""Reduce time and space through a pairwise fabric ADD tree."""` This is executable documentation attached to the function _global_sum, which handles global sum. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1177: `timesteps, batch, channels, height, width = value.shape` This assigns 'value.shape' to 'timesteps, batch, channels, height, width ' in host memory. Here timesteps means one discrete SNN update instant. The important operands are value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function _global_sum, which handles global sum.

Line 1178: `reduction = value.transpose(0, 3, 4, 1, 2).reshape(` This statement continues through source line 1179; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'value.transpose(0, 3, 4, 1, 2).reshape(timesteps * height * width, batch, channels)' to 'reduction ' in host memory. Here reduction means the value named 'reduction' in this block. The important operands are value: the value named 'value' in this block; transpose: the value named 'transpose' in this block; reshape: the value named 'reshape' in this block. Context: the function _global_sum, which handles global sum.

Line 1179: `timesteps * height * width, batch, channels)` This is continuation line 2 of the 2-line logical statement that begins at source line 1178. Its exact fragment is ‘timesteps * height * width, batch, channels)‘. The important operands are timesteps: one discrete SNN update instant; height: the value named ’height’ in this block; width: the value named ’width’ in this block. Read together, the complete operation is: This assigns ‘value.transpose(0, 3, 4, 1, 2).reshape(timesteps * height * width, batch, channels)‘ to ‘reduction ‘ in host memory. Here reduction means the value named ’reduction’ in this block. The important operands are value: the value named ’value’ in this block; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _global_sum, which handles global sum.

Line 1180: `conservative_bound = int(np.abs(reduction).max(initial=0)) * reduction.shape[0]` This assigns ‘int(np.abs(reduction).max(initial=0)) * reduction.shape[0]‘ to ‘conservative_bound ‘ in host memory. Here conservative_bound means the value named ’conservative bound’ in this block. The important operands are abs: the value named ’abs’ in this block; reduction: the value named ’reduction’ in this block; max: the value named ’max’ in this block. Context: the function _global_sum, which handles global sum.

Line 1181: `if conservative_bound > (1 << 31) - 1:` This executes the following indented host-code branch only when ‘conservative_bound > (1 << 31) - 1‘ is true. The important operands are conservative_bound: the value named ’conservative bound’ in this block. Context: the function _global_sum, which handles global sum.

Line 1182: `raise OverflowError("global pooling could saturate the signed-32 vector tree")` This rejects an invalid state by raising ‘OverflowError("global pooling could saturate the signed-32 vector tree")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1183: `candidates = [reduction[index] for index in range(reduction.shape[0])]` This assigns ‘[reduction[index] for index in range(reduction.shape[0])]‘ to ‘candidates ‘ in host memory. Here candidates means the value named ’candidates’ in this block. The important operands are reduction: the value named ’reduction’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block. Context: the function _global_sum, which handles global sum.

Line 1184: `while len(candidates) > 1:` This repeats the following host-code block while ‘len(candidates) > 1‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 1185: `reduced: list[np.ndarray] = []` This assigns ‘[]‘ to ‘reduced‘ in host memory. Here reduced means the value named ’reduced’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _global_sum, which handles global sum.

Line 1186: `for index in range(0, len(candidates) - 1, 2):` This iterates index over ‘range(0, len(candidates) - 1, 2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _global_sum, which handles global sum.

Line 1187: `reduced.append(execute_vector_tensor(` This statement continues through source line 1193; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘reduced.append‘ with ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=self.layer_)‘ inside the function _global_sum, which handles global sum. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1188: `self.driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1187. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This calls ‘reduced.append‘ with ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=self.layer_timeout_seconds,)‘ inside the function _global_sum, which handles global sum. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1189: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 1187. Its exact fragment is 'VectorOperation.ADD_SAT,'. The important operands are VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named 'add sat' in this block. Read together, the complete operation is: This calls 'reduced.append' with 'execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=self.layer_timeout_seconds,)' inside the function _global_sum, which handles global sum. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1190: `candidates[index],` This is continuation line 4 of the 7-line logical statement that begins at source line 1187. Its exact fragment is 'candidates[index],'. The important operands are candidates: the value named 'candidates' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'reduced.append' with 'execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=self.layer_timeout_seconds,)' inside the function _global_sum, which handles global sum. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1191: `candidates[index + 1],` This is continuation line 5 of the 7-line logical statement that begins at source line 1187. Its exact fragment is 'candidates[index + 1],'. The important operands are candidates: the value named 'candidates' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'reduced.append' with 'execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=self.layer_timeout_seconds,)' inside the function _global_sum, which handles global sum. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1192: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1187. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This calls 'reduced.append' with 'execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=self.layer_timeout_seconds,)' inside the function _global_sum, which handles global sum. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1193: `))` This is continuation line 7 of the 7-line logical statement that begins at source line 1187. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'reduced.append' with 'execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, candidates[index], candidates[index + 1], timeout_seconds=self.layer_timeout_seconds,)' inside the function _global_sum, which handles global sum. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1194: `if len(candidates) & 1:` This executes the following indented host-code branch only when 'len(candidates) & 1' is true. The important operands are len: the value named 'len' in this block; candidates: the value named 'candidates' in this block. Context: the function _global_sum, which handles global sum.

Line 1195: `reduced.append(candidates[-1])` This calls 'reduced.append' with 'candidates[-1]' inside the function _global_sum, which handles global sum. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1196: `candidates = reduced` This assigns 'reduced' to 'candidates ' in host memory. Here candidates means the value named 'candidates' in this block. The important operands are reduced: the value named 'reduced' in this block. Context: the function _global_sum, which handles global sum.

Line 1197: `return candidates[0]` This returns 'candidates[0]' to the caller of the function _global_sum, which handles global sum, ending this invocation. The important operands are candidates: the value named 'candidates' in this block.

Line 1199: `def _tokenizer_forward(` This starts function _tokenizer_forward. Its indented body performs tokenizer forward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1200: `self, image_u8: np.ndarray, image_centered: np.ndarray` Inside the function _tokenizer_forward, which handles tokenizer forward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are image_u8: the value named 'image u8' in this block; ndarray: the value named 'ndarray' in this block; image_centered: the value named 'image centered' in this block.

Line 1201: `) -> tuple[np.ndarray, tuple[TokenizerLayerCache, ...]]:` Inside the function _tokenizer_forward, which handles tokenizer forward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named 'tuple' in this block; ndarray: the value named 'ndarray' in this block; TokenizerLayerCache: the value named 'tokenizer layer cache' in this block.

Line 1202: `batch = image_u8.shape[0]` This assigns ‘image_u8.shape[0]‘ to ‘batch ‘ in host memory. Here batch means the value named 'batch' in this block. The important operands are image_u8: the value named 'image u8' in this block; shape: the value named 'shape' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1203: `first = self.projections.conv2d(` This statement continues through source line 1209; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeou)‘ to ‘first ‘ in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1204: `"patch_embed.block0",` This is continuation line 2 of the 7-line logical statement that begins at source line 1203. Its exact fragment is ‘"patch_embed.block0",‘. The important operands are patch_embed: the value named 'patch embed' in this block; block0: the value named 'block0' in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘first ‘ in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1205: `image_u8,` This is continuation line 3 of the 7-line logical statement that begins at source line 1203. Its exact fragment is ‘image_u8,‘. The important operands are image_u8: the value named 'image u8' in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘first ‘ in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1206: `activation_zero_point=self.image_zero_point,` This is continuation line 4 of the 7-line logical statement that begins at source line 1203. Its exact fragment is ‘activation_zero_point=self.image_zero_point,‘. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; image_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns ‘self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘first ‘ in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1207: `padding=1,` This is continuation line 5 of the 7-line logical statement that begins at source line 1203. Its exact fragment is 'padding=1,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'first ' in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1208: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1203. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeou)' to 'first ' in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1209: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1203. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self.projections.conv2d("patch_embed.block0", image_u8, activation_zero_point=self.image_zero_point, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'first ' in host memory. Here first means the value named 'first' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1210: `caches: list[TokenizerLayerCache] = [` This statement continues through source line 1212; the explanation uses the complete expression rather than only the first visual fragment. This assigns '[TokenizerLayerCache(0, image_centered, first, None, False)]' to 'caches' in host memory. Here caches means the value named 'caches' in this block. The important operands are TokenizerLayerCache: the value named 'tokenizer layer cache' in this block; image_centered: the value named 'image centered' in this block; first: the value named 'first' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1211: `TokenizerLayerCache(0, image_centered, first, None, False)` This is continuation line 2 of the 3-line logical statement that begins at source line 1210. Its exact fragment is 'TokenizerLayerCache(0, image_centered, first, None, False)'. The important operands are TokenizerLayerCache: the value named 'tokenizer layer cache' in this block; image_centered: the value named 'image centered' in this block; first: the value named 'first' in this block. Read together, the complete operation is: This assigns '[TokenizerLayerCache(0, image_centered, first, None, False)]' to 'caches' in host memory. Here caches means the value named 'caches' in this block. The important operands are TokenizerLayerCache: the value named 'tokenizer layer cache' in this block; image_centered: the value named 'image centered' in this block; first: the value named 'first' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1212: `]` This is continuation line 3 of the 3-line logical statement that begins at source line 1210. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '[TokenizerLayerCache(0, image_centered, first, None, False)]' to 'caches' in host memory. Here caches means the value named 'caches' in this block. The important operands are TokenizerLayerCache: the value named 'tokenizer layer cache' in this block; image_centered: the value named 'image centered' in this block; first: the value named 'first' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1213: `value = np.repeat(first[np.newaxis], self.timesteps, axis=0)` This assigns 'np.repeat(first[np.newaxis], self.timesteps, axis=0)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are repeat: the value named 'repeat' in this block; first: the value named 'first' in this block; newaxis: the value named 'newaxis' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1214: `for index in (1, 2, 3, 4):` This iterates index over ‘(1, 2, 3, 4)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _tokenizer_forward, which handles tokenizer forward.

Line 1215: `lif = self._lif_trace(value)` This assigns ‘self._lif_trace(value)‘ to ‘lif ‘ in host memory. Here lif means the value named ’lif’ in this block. The important operands are _lif_trace: the value named ’lif trace’ in this block; value: the value named ’value’ in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1216: `flat_spikes = lif.spikes.reshape(-1, *lif.spikes.shape[2:])` This assigns ‘lif.spikes.reshape(-1, *lif.spikes.shape[2:])‘ to ‘flat_spikes ‘ in host memory. Here flat_spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif: the value named ’lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1217: `if index >= 3:` This executes the following indented host-code branch only when ‘index >= 3‘ is true. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1218: `projection_input = binary_max_pool2d_on_accelerator(` This statement continues through source line 1222; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘binary_max_pool2d_on_accelerator(self.driver, flat_spikes, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projection_input ‘ in host memory. Here projection_input means the value named ’projection input’ in this block. The important operands are binary_max_pool2d_on_accelerator: the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1219: `self.driver,` This is continuation line 2 of the 5-line logical statement that begins at source line 1218. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘binary_max_pool2d_on_accelerator(self.driver, flat_spikes, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projection_input ‘ in host memory. Here projection_input means the value named ’projection input’ in this block. The important operands are binary_max_pool2d_on_accelerator: the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1220: `flat_spikes,` This is continuation line 3 of the 5-line logical statement that begins at source line 1218. Its exact fragment is ‘flat_spikes,‘. The important operands are flat_spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns ‘binary_max_pool2d_on_accelerator(self.driver, flat_spikes, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projection_input ‘ in host memory. Here projection_input means the value named ’projection input’ in this block. The important operands are binary_max_pool2d_on_accelerato the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1221: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 4 of the 5-line logical statement that begins at source line 1218. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘binary_max_pool2d_on_accelerator(self.driver, flat_spikes, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projection_input ‘ in host memory. Here projection_input means the value named ’projection input’ in this block. The important operands are binary_max_pool2d_on_accelerator: the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1222: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1218. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘binary_max_pool2d_on_accelerator(self.driver, flat_spikes, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projection_input ‘ in host memory. Here projection_input means the value named ’projection input’ in this block. The important operands are binary_max_pool2d_on_accelerator: the value named ’binary max pool2d on accelerator’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1223: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function _tokenizer_forward, which handles tokenizer forward.

Line 1224: `projection_input = flat_spikes` This assigns ‘flat_spikes‘ to ‘projection_input ‘ in host memory. Here projection_input means the value named ’projection input’ in this block. The important operands are flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1225: `projected = self.projections.conv2d(` This statement continues through source line 1230; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", projection_input, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1226: `f"patch_embed.block{index}",` This is continuation line 2 of the 6-line logical statement that begins at source line 1225. Its exact fragment is ‘f"patch_embed.block{index}",‘. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", projection_input, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1227: `projection_input,` This is continuation line 3 of the 6-line logical statement that begins at source line 1225. Its exact fragment is ‘projection_input,‘. The important operands are projection_input: the value named ’projection input’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index projection_input, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1228: `padding=1,` This is continuation line 4 of the 6-line logical statement that begins at source line 1225. Its exact fragment is ‘padding=1,‘. The important operands are padding: the value named ’padding’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", projection_input, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are projections: the value named ’projections’ in this block; conv2d: the value named ’conv2d’ in this block; f: the value named ’f’ in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1229: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1225. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘self.projections.conv2d(f"patch_embed.block{index}", projection_input, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘projected ‘ in host memory. Here projected means the value named ’projected’ in this block. The important operands are

projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; f: the value named 'f' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1230: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1225. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self.projections.conv2d(f"patch_embed.block{index}", projection_input, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are projections: the value named 'projections' in this block; conv2d: the value named 'conv2d' in this block; f: the value named 'f' in this block. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1231: `caches.append(TokenizerLayerCache(` This statement continues through source line 1237; the explanation uses the complete expression rather than only the first visual fragment. This calls 'caches.append' with 'TokenizerLayerCache(index, projection_input, projected, lif, index >= 3,)' inside the function _tokenizer_forward, which handles tokenizer forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1232: `index,` This is continuation line 2 of the 7-line logical statement that begins at source line 1231. Its exact fragment is 'index,'. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'caches.append' with 'TokenizerLayerCache(index, projection_input, projected, lif, index >= 3,)' inside the function _tokenizer_forward, which handles tokenizer forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1233: `projection_input,` This is continuation line 3 of the 7-line logical statement that begins at source line 1231. Its exact fragment is 'projection_input,'. The important operands are projection_input: the value named 'projection input' in this block. Read together, the complete operation is: This calls 'caches.append' with 'TokenizerLayerCache(index, projection_input, projected, lif, index >= 3,)' inside the function _tokenizer_forward, which handles tokenizer forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1234: `projected,` This is continuation line 4 of the 7-line logical statement that begins at source line 1231. Its exact fragment is 'projected,'. The important operands are projected: the value named 'projected' in this block. Read together, the complete operation is: This calls 'caches.append' with 'TokenizerLayerCache(index, projection_input, projected, lif, index >= 3,)' inside the function _tokenizer_forward, which handles tokenizer forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1235: `lif,` This is continuation line 5 of the 7-line logical statement that begins at source line 1231. Its exact fragment is 'lif,'. The important operands are lif: the value named 'lif' in this block. Read together, the complete operation is: This calls 'caches.append' with 'TokenizerLayerCache(index, projection_input, projected, lif, index >= 3,)' inside the function _tokenizer_forward, which handles tokenizer forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1236: `index >= 3,` This is continuation line 6 of the 7-line logical statement that begins at source line 1231. Its exact fragment is 'index >= 3,'. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'caches.append' with 'TokenizerLayerCache(index, projection_input, projected, lif, index >= 3,)' inside the function _tokenizer_forward, which handles tokenizer forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1237: `))` This is continuation line 7 of the 7-line logical statement that begins at source line 1231. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'caches.append' with 'TokenizerLayerCache(index, projection_input, projected, lif, index >= 3,)' inside the function _tokenizer_forward, which handles tokenizer forward. The call advances the

host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1238: `value = projected.reshape(` This statement continues through source line 1239; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘projected.reshape(self.timesteps, batch, *projected.shape[1:])‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are projected: the value named ’projected’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1239: `self.timesteps, batch, *projected.shape[1:])` This is continuation line 2 of the 2-line logical statement that begins at source line 1238. Its exact fragment is ‘self.timesteps, batch, *projected.shape[1:])‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; projected: the value named ’projected’ in this block. Read together, the complete operation is: This assigns ‘projected.reshape(self.timesteps, batch, *projected.shape[1:])‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are projected: the value named ’projected’ in this block; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _tokenizer_forward, which handles tokenizer forward.

Line 1240: `return value, tuple(caches)` This returns ‘value, tuple(caches)‘ to the caller of the function _tokenizer_forward, which handles tokenizer forward, ending this invocation. The important operands are value: the value named ’value’ in this block; tuple: the value named ’tuple’ in this block; caches: the value named ’caches’ in this block.

Line 1242: `def _attention_forward(self, block_index: int, value: np.ndarray) -> AttentionCache:` This starts function _attention_forward. Its indented body performs attention forward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1243: `timesteps, batch, channels, height, width = value.shape` This assigns ‘value.shape‘ to ‘timesteps, batch, channels, height, width ‘ in host memory. Here timesteps means one discrete SNN update instant. The important operands are value: the value named ’value’ in this block; shape: the value named ’shape’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1244: `tokens = height * width` This assigns ‘height * width‘ to ‘tokens ‘ in host memory. Here tokens means the value named ’tokens’ in this block. The important operands are height: the value named ’height’ in this block; width: the value named ’width’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1245: `projection_lif = self._lif_trace(value)` This assigns ‘self._lif_trace(value)‘ to ‘projection_lif ‘ in host memory. Here projection_lif means the value named ’projection lif’ in this block. The important operands are _lif_trace: the value named ’lif trace’ in this block; value: the value named ’value’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1246: `token_spikes = projection_lif.spikes.reshape(` This statement continues through source line 1247; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘projection_lif.spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)‘ to ‘token_spikes ‘ in host memory. Here token_spikes means a binary event produced when a neuron crosses its threshold. The important operands are projection_lif: the value named ’projection lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1247: `timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)` This is continuation line 2 of the 2-line logical statement that begins at source line 1246. Its exact fragment is ‘timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘projection_lif.spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2)‘ to ‘token_spikes ‘ in host memory. Here token_spikes means a binary event produced when a neuron crosses its threshold. The important operands are projection_lif: the value named ’projection lif’ in this block; spikes: a binary event produced when a neuron crosses its

threshold; reshape: the value named 'reshape' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1248: `rows = token_spikes.reshape(timesteps * batch * tokens, channels)` This assigns 'token_spikes.reshape(tim * batch * tokens, channels)' to 'rows ' in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are token_spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _attention_forward, which handles attention forward.

Line 1249: `prefix = f"block.{block_index}.attn"` This assigns 'f"block.{block_index}.attn"' to 'prefix ' in host memory. Here prefix means the value named 'prefix' in this block. The important operands are f: the value named 'f' in this block; block: the value named 'block' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function _attention_forward, which handles attention forward.

Line 1251: `def projected_lif(name: str) -> LifTraceCache:` This starts function projected_lif. Its indented body performs projected lif as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1252: `projected = self.projections.matrix(` This statement continues through source line 1254; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.projections.matrix(f"{prefix}.{name}", rows, timeout_seconds=self.layer_timeout_seconds)' to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function projected_lif, which handles projected lif.

Line 1253: `f"{prefix}.{name}", rows,` This is continuation line 2 of the 3-line logical statement that begins at source line 1252. Its exact fragment is 'f"{prefix}.{name}", rows,'. The important operands are f: the value named 'f' in this block; prefix: the value named 'prefix' in this block; name: the value named 'name' in this block. Read together, the complete operation is: This assigns 'self.projections.matrix(f"{prefix}.{name}", rows, timeout_seconds=self.layer_timeout_seconds to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function projected_lif, which handles projected lif.

Line 1254: `timeout_seconds=self.layer_timeout_seconds)` This is continuation line 3 of the 3-line logical statement that begins at source line 1252. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds)'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'self.projections.matrix(f"{prefix}.{name}", rows, timeout_seconds=self.layer_timeout_seconds)' to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function projected_lif, which handles projected lif.

Line 1255: `shaped = projected.reshape(timesteps, batch, tokens, channels)` This assigns 'projected.reshape(timesteps batch, tokens, channels)' to 'shaped ' in host memory. Here shaped means the value named 'shaped' in this block. The important operands are projected: the value named 'projected' in this block; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function projected_lif, which handles projected lif.

Line 1256: `return self._lif_trace(shaped)` This returns 'self._lif_trace(shaped)' to the caller of the function projected_lif, which handles projected lif, ending this invocation. The important operands are _lif_trace: the value named 'lif trace' in this block; shaped: the value named 'shaped' in this block.

Line 1258: `query_lif = projected_lif("q")` This assigns 'projected_lif("q")' to 'query_lif ' in host memory. Here query_lif means the value named 'query lif' in this block. The important operands are projected_lif: the value named 'projected lif' in this block; q: the value named 'q' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1259: `key_lif = projected_lif("k")` This assigns ‘projected_lif("k")‘ to ‘key_lif ‘ in host memory. Here key_lif means the value named ’key lif’ in this block. The important operands are projected_lif: the value named ’projected lif’ in this block; k: the value named ’k’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1260: `value_lif = projected_lif("v")` This assigns ‘projected_lif("v")‘ to ‘value_lif ‘ in host memory. Here value_lif means the value named ’value lif’ in this block. The important operands are projected_lif: the value named ’projected lif’ in this block; v: the value named ’v’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1261: `if channels % self.attention_heads:` This executes the following indented host-code branch only when ‘channels % self.attention_heads‘ is true. The important operands are channels: the value named ’channels’ in this block; attention_heads: the value named ’attention heads’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1262: `raise ValueError("attention channels must divide evenly into the head count")` This rejects an invalid state by raising ‘ValueError("attention channels must divide evenly into the head count")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1263: `head_dim = channels // self.attention_heads` This assigns ‘channels // self.attention_heads‘ to ‘head_dim ‘ in host memory. Here head_dim means the value named ’head dim’ in this block. The important operands are channels: the value named ’channels’ in this block; attention_heads: the value named ’attention heads’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1265: `def heads(spikes: np.ndarray) -> np.ndarray:` This starts function heads. Its indented body performs heads as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1266: `return spikes.reshape(` This statement continues through source line 1268; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘spikes.reshape(timesteps, batch, tokens, self.attention_heads, head_dim).transpose(0, 1, 3, 2, 4)‘ to the caller of the function heads, which handles heads, ending this invocation. The important operands are spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant.

Line 1267: `timesteps, batch, tokens, self.attention_heads, head_dim` This is continuation line 2 of the 3-line logical statement that begins at source line 1266. Its exact fragment is ‘timesteps, batch, tokens, self.attention_heads, head_dim‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This returns ‘spikes.reshape(timesteps, batch, tokens, self.attention_heads, head_dim).transpose(0, 1, 3, 2, 4)‘ to the caller of the function heads, which handles heads, ending this invocation. The important operands are spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant.

Line 1268: `).transpose(0, 1, 3, 2, 4)` This is continuation line 3 of the 3-line logical statement that begins at source line 1266. Its exact fragment is ‘).transpose(0, 1, 3, 2, 4)‘. The important operands are transpose: the value named ’transpose’ in this block. Read together, the complete operation is: This returns ‘spikes.reshape(timesteps, batch, tokens, self.attention_heads, head_dim).transpose(0, 1, 3, 2, 4)‘ to the caller of the function heads, which handles heads, ending this invocation. The important operands are spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant.

Line 1270: `query_heads = heads(query_lif.spikes)` This assigns ‘heads(query_lif.spikes)‘ to ‘query_heads ‘ in host memory. Here query_heads means the value named ’query heads’ in this block. The important operands are heads: the value named ’heads’ in this block; query_lif: the value named ’query lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold. Context: the function _attention_forward, which handles attention forward.

Line 1271: `key_heads = heads(key_lif.spikes)` This assigns ‘heads(key_lif.spikes)‘ to ‘key_heads ‘ in host memory. Here key_heads means the value named ’key heads’ in this block. The important operands are heads: the value named ’heads’ in this block; key_lif: the value named ’key lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold. Context: the function _attention_forward, which handles attention forward.

Line 1272: `value_heads = heads(value_lif.spikes)` This assigns ‘heads(value_lif.spikes)‘ to ‘value_heads ‘ in host memory. Here value_heads means the value named ’value heads’ in this block. The important operands are heads: the value named ’heads’ in this block; value_lif: the value named ’value lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold. Context: the function _attention_forward, which handles attention forward.

Line 1273: `scores = np.empty(` This statement continues through source line 1276; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.empty((timesteps, batch, self.attention_heads, tokens, tokens), dtype=np.int64,)‘ to ‘scores ‘ in host memory. Here scores means the value named ’scores’ in this block. The important operands are empty: the value named ’empty’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1274: `(timesteps, batch, self.attention_heads, tokens, tokens),` This is continuation line 2 of the 4-line logical statement that begins at source line 1273. Its exact fragment is ‘(timesteps, batch, self.attention_heads, tokens, tokens),‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; attention_heads: the value named ’attention heads’ in this block. Read together, the complete operation is: This assigns ‘np.empty((timesteps, batch, self.attention_heads, tokens, tokens), dtype=np.int64,)‘ to ‘scores ‘ in host memory. Here scores means the value named ’scores’ in this block. The important operands are empty: the value named ’empty’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1275: `dtype=np.int64,` This is continuation line 3 of the 4-line logical statement that begins at source line 1273. Its exact fragment is ‘dtype=np.int64,‘. The important operands are dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This assigns ‘np.empty((timesteps, batch, self.attention_heads, tokens, tokens), dtype=np.int64,)‘ to ‘scores ‘ in host memory. Here scores means the value named ’scores’ in this block. The important operands are empty: the value named ’empty’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1276: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1273. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.empty((timesteps, batch, self.attention_heads, tokens, tokens), dtype=np.int64,)‘ to ‘scores ‘ in host memory. Here scores means the value named ’scores’ in this block. The important operands are empty: the value named ’empty’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1277: `attention_values = np.empty(` This statement continues through source line 1280; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.empty((timesteps, batch, self.attention_heads, tokens, head_dim), dtype=np.int64,)‘ to ‘attention_values ‘ in host memory. Here attention_values means the value named ’attention values’ in this block. The important operands are empty: the value named ’empty’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1278: `(timesteps, batch, self.attention_heads, tokens, head_dim),` This is continuation line 2 of the 4-line logical statement that begins at source line 1277. Its exact fragment is ‘(timesteps, batch, self.attention_heads, tokens, head_dim),‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; attention_heads: the value named ’attention heads’ in this block. Read together, the complete operation is: This assigns ‘np.empty((timesteps, batch, self.attention_heads, tokens, head_dim), dtype=np.int64,)‘ to ‘attention_values ‘ in host memory. Here attention_values means the value named ’attention values’ in this block. The important operands are empty: the value named ’empty’ in this block; timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1279: `dtype=np.int64,` This is continuation line 3 of the 4-line logical statement that begins at source line 1277. Its exact fragment is 'dtype=np.int64,'. The important operands are dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'np.empty((timesteps, batch, self.attention_heads, tokens, head_dim), dtype=np.int64,)' to 'attention_values ' in host memory. Here attention_values means the value named 'attention values' in this block. The important operands are empty: the value named 'empty' in this block; timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1280: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1277. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.empty((timesteps, batch, self.attention_heads, tokens, head_dim), dtype=np.int64,)' to 'attention_values ' in host memory. Here attention_values means the value named 'attention values' in this block. The important operands are empty: the value named 'empty' in this block; timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1281: `for timestep in range(timesteps):` This iterates timestep over 'range(timesteps)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention_forward, which handles attention forward.

Line 1282: `for batch_index in range(batch):` This iterates batch_index over 'range(batch)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention_forward, which handles attention forward.

Line 1283: `for head in range(self.attention_heads):` This iterates head over 'range(self.attention_heads)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention_forward, which handles attention forward.

Line 1284: `score = self._dynamic_binary_gemm(` This statement continues through source line 1287; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._dynamic_binary_gemm(query_heads[timestep, batch_index, head], key_heads[timestep, batch_index, head].T,)' to 'score ' in host memory. Here score means the value named 'score' in this block. The important operands are _dynamic_binary_gemm: the value named 'dynamic binary gemm' in this block; query_heads: the value named 'query heads' in this block; timestep: one discrete SNN update instant. Context: the function _attention_forward, which handles attention forward.

Line 1285: `query_heads[timestep, batch_index, head],` This is continuation line 2 of the 4-line logical statement that begins at source line 1284. Its exact fragment is 'query_heads[timestep, batch_index, head],'. The important operands are query_heads: the value named 'query heads' in this block; timestep: one discrete SNN update instant; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'self._dynamic_binary_gemm(query_heads[timestep, batch_index, head], key_heads[timestep, batch_index, head].T,)' to 'score ' in host memory. Here score means the value named 'score' in this block. The important operands are _dynamic_binary_gemm: the value named 'dynamic binary gemm' in this block; query_heads: the value named 'query heads' in this block; timestep: one discrete SNN update instant. Context: the function _attention_forward, which handles attention forward.

Line 1286: `key_heads[timestep, batch_index, head].T,` This is continuation line 3 of the 4-line logical statement that begins at source line 1284. Its exact fragment is 'key_heads[timestep, batch_index, head].T,'. The important operands are key_heads: the value named 'key heads' in this block; timestep: one discrete SNN update instant; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'self._dynamic_binary_gemm(query_heads[timestep, batch_index, head], key_heads[timestep, batch_index, head].T,)' to 'score ' in host memory. Here score means the value named 'score' in this block. The important operands are _dynamic_binary_gemm: the value named 'dynamic binary gemm' in this block; query_heads: the value named 'query heads' in this block; timestep: one discrete SNN update instant. Context: the function _attention_forward, which handles attention forward.

Line 1287: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1284. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self._dynamic_binary_gemm(query_heads[timestep, batch_index, head], key_heads[timestep, batch_index, head].T,)' to 'score ' in host memory. Here score means the value named 'score' in this block. The important operands are _dynamic_binary_gemm: the value named 'dynamic binary gemm' in this block; query_heads: the value named 'query heads' in this block; timestep: one discrete SNN update instant. Context: the function _attention_forward, which handles attention forward.

Line 1288: `if score.min() < 0 or score.max() > 255:` This executes the following indented host-code branch only when 'score.min() $<$ 0 or score.max() $>$ 255' is true. The important operands are score: the value named 'score' in this block; min: the value named 'min' in this block; or: the value named 'or' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1289: `raise OverflowError("binary QK score exceeds the unsigned AV contract")` This rejects an invalid state by raising 'OverflowError("binary QK score exceeds the unsigned AV contract")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1290: `scores[timestep, batch_index, head] = score` This assigns 'score' to 'scores[timestep, batch_index, head] ' in host memory. Here scores means the value named 'scores' in this block. The important operands are score: the value named 'score' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1291: `attention_values[timestep, batch_index, head] = self._dynamic_u8_gemm(` This statement continues through source line 1294; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._dynamic_u8_gemm(score.astype(np.uint8), value_heads[timestep, batch_index, head].astype(np.uint8),)' to 'attention_values[timestep, batch_index, head] ' in host memory. Here attention_values means the value named 'attention values' in this block. The important operands are _dynamic_u8_gemm: the value named 'dynamic u8 gemm' in this block; score: the value named 'score' in this block; astype: the value named 'astype' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1292: `score.astype(np.uint8),` This is continuation line 2 of the 4-line logical statement that begins at source line 1291. Its exact fragment is 'score.astype(np.uint8),'. The important operands are score: the value named 'score' in this block; astype: the value named 'astype' in this block; uint8: the value named 'uint8' in this block. Read together, the complete operation is: This assigns 'self._dynamic_u8_gemm(score.astype(np.uint8), value_heads[timestep, batch_index, head].astype(np.uint8),)' to 'attention_values[timestep, batch_index, head] ' in host memory. Here attention_values means the value named 'attention values' in this block. The important operands are _dynamic_u8_gemm: the value named 'dynamic u8 gemm' in this block; score: the value named 'score' in this block; astype: the value named 'astype' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1293: `value_heads[timestep, batch_index, head].astype(np.uint8),` This is continuation line 3 of the 4-line logical statement that begins at source line 1291. Its exact fragment is 'value_heads[timestep, batch_index, head].astype(np.uint8),'. The important operands are value_heads: the value named 'value heads' in this block; timestep: one discrete SNN update instant; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'self._dynamic_u8_gemm(score.astype(np.uint8), value_heads[timestep, batch_index, head].astype(np.uint8),)' to 'attention_values[timestep, batch_index, head] ' in host memory. Here attention_values means the value named 'attention values' in this block. The important operands are _dynamic_u8_gemm: the value named 'dynamic u8 gemm' in this block; score: the value named 'score' in this block; astype: the value named 'astype' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1294: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1291. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self._dynamic_u8_gemm(score.astype(np.uint8), value_heads[timestep, batch_index, head].astype(np.uint8),)' to 'attention_values[timestep, batch_index, head] ' in host memory. Here attention_values means the value named 'attention values' in this block. The important operands are _dynamic_u8_gemm: the value named 'dynamic u8 gemm' in this block; score: the value named 'score' in this block; astype: the value named 'astype' in this block. Context: the function _attention_forward, which handles attention forward.

Line 1295: `attention_q10 = attention_values * (1 << self.attention_scale_shift)` This assigns ‘attention_values * (1 << self.attention_scale_shift)‘ to ‘attention_q10 ‘ in host memory. Here attention_q10 means the value named ’attention q10’ in this block. The important operands are attention_values: the value named ’attention values’ in this block; attention_scale_shift: the real-value spacing represented by one quantized integer step. Context: the function _attention_forward, which handles attention forward.

Line 1296: `merged = attention_q10.transpose(0, 1, 2, 4, 3).reshape(` This statement continues through source line 1297; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘attention_q10.transpose(0, 1, 2, 4, 3).reshape(timesteps, batch, channels, tokens)‘ to ‘merged ‘ in host memory. Here merged means the value named ’merged’ in this block. The important operands are attention_q10: the value named ’attention q10’ in this block; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1297: `timesteps, batch, channels, tokens)` This is continuation line 2 of the 2-line logical statement that begins at source line 1296. Its exact fragment is ‘timesteps, batch, channels, tokens)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘attention_q10.transpose(0, 1, 2, 4, 3).reshape(timesteps, batch, channels, tokens)‘ to ‘merged ‘ in host memory. Here merged means the value named ’merged’ in this block. The important operands are attention_q10: the value named ’attention q10’ in this block; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1298: `attention_lif = self._lif_trace(merged, threshold=0.5)` This assigns ‘self._lif_trace(merged, threshold=0.5)‘ to ‘attention_lif ‘ in host memory. Here attention_lif means the value named ’attention lif’ in this block. The important operands are _lif_trace: the value named ’lif trace’ in this block; merged: the value named ’merged’ in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function _attention_forward, which handles attention forward.

Line 1299: `projection_input = attention_lif.spikes.transpose(0, 1, 3, 2).reshape(` This statement continues through source line 1300; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘attention_lif.spikes.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)‘ to ‘projection_input ‘ in host memory. Here projection_input means the value named ’projection input’ in this block. The important operands are attention_lif: the value named ’attention lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold; transpose: the value named ’transpose’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1300: `timesteps * batch * tokens, channels)` This is continuation line 2 of the 2-line logical statement that begins at source line 1299. Its exact fragment is ‘timesteps * batch * tokens, channels)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This assigns ‘attention_lif.spikes.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)‘ to ‘projection_input ‘ in host memory. Here projection_input means the value named ’projection input’ in this block. The important operands are attention_lif: the value named ’attention lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold; transpose: the value named ’transpose’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1301: `output_rows = self.projections.matrix(` This statement continues through source line 1305; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.projections.matrix(f"{prefix}.proj", projection_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1302: `f"{prefix}.proj",` This is continuation line 2 of the 5-line logical statement that begins at source line 1301. Its exact fragment is ‘f"{prefix}.proj",‘. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; proj: the value named ’proj’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"{prefix}.proj", projection_input, timeout_seconds=self.layer_timeout_seconds,

)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1303: `projection_input,` This is continuation line 3 of the 5-line logical statement that begins at source line 1301. Its exact fragment is ‘projection_input,‘. The important operands are projection_input: the value named ’projection input’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"{prefix}.proj", projection_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1304: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 4 of the 5-line logical statement that begins at source line 1301. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"{prefix}.proj", projection_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1305: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1301. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"{prefix}.proj", projection_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _attention_forward, which handles attention forward.

Line 1306: `output = output_rows.reshape(` This statement continues through source line 1308; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘output_rows.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are output_rows: one output-row position processed by the tensor tile; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_forward, which handles attention forward.

Line 1307: `timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(` This is continuation line 2 of the 3-line logical statement that begins at source line 1306. Its exact fragment is ‘timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This assigns ‘output_rows.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are output_rows: one output-row position processed by the tensor tile; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_forward, which handles attention forward.

Line 1308: `timesteps, batch, channels, height, width)` This is continuation line 3 of the 3-line logical statement that begins at source line 1306. Its exact fragment is ‘timesteps, batch, channels, height, width)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘output_rows.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are output_rows: one output-row position processed by the tensor tile; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_forward, which handles attention forward.

Line 1309: `return AttentionCache(` This statement continues through source line 1325; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named ’attention cache’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named ’tuple’ in this block.

Line 1310: `block_index,` This is continuation line 2 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘block_index,‘. The important operands are block_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named ’attention cache’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named ’tuple’ in this block.

Line 1311: `tuple(value.shape),` This is continuation line 3 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘tuple(value.shape),‘. The important operands are tuple: the value named ’tuple’ in this block; value: the value named ’value’ in this block; shape: the value named ’shape’ in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named ’attention cache’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named ’tuple’ in this block.

Line 1312: `projection_lif,` This is continuation line 4 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘projection_lif,‘. The important operands are projection_lif: the value named ’projection lif’ in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named ’attention cache’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named ’tuple’ in this block.

Line 1313: `token_spikes,` This is continuation line 5 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘token_spikes,‘. The important operands are token_spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named ’attention cache’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named ’tuple’ in this block.

Line 1314: `query_lif,` This is continuation line 6 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘query_lif,‘. The important operands are query_lif: the value named ’query lif’ in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named ’attention cache’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named ’tuple’ in this block.

Line 1315: `key_lif,` This is continuation line 7 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘key_lif,‘. The important operands are key_lif: the value named ’key lif’ in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes,

query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1316: `value_lif,` This is continuation line 8 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘value_lif,‘. The important operands are value_lif: the value named 'value lif' in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1317: `query_heads,` This is continuation line 9 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘query_heads,‘. The important operands are query_heads: the value named 'query heads' in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1318: `key_heads,` This is continuation line 10 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘key_heads,‘. The important operands are key_heads: the value named 'key heads' in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1319: `value_heads,` This is continuation line 11 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘value_heads,‘. The important operands are value_heads: the value named 'value heads' in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1320: `scores,` This is continuation line 12 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘scores,‘. The important operands are scores: the value named 'scores' in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1321: `attention_values,` This is continuation line 13 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ‘attention_values,‘. The important operands are attention_values: the value named 'attention values' in this block. Read together, the complete operation is: This returns ‘AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)‘ to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1322: `attention_lif,` This is continuation line 14 of the 17-line logical statement that begins at source line 1309. Its exact fragment is 'attention_lif,'. The important operands are attention_lif: the value named 'attention lif' in this block. Read together, the complete operation is: This returns 'AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)' to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1323: `projection_input,` This is continuation line 15 of the 17-line logical statement that begins at source line 1309. Its exact fragment is 'projection_input,'. The important operands are projection_input: the value named 'projection input' in this block. Read together, the complete operation is: This returns 'AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)' to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1324: `output,` This is continuation line 16 of the 17-line logical statement that begins at source line 1309. Its exact fragment is 'output,'. The important operands are output: the value named 'output' in this block. Read together, the complete operation is: This returns 'AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)' to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1325: `)` This is continuation line 17 of the 17-line logical statement that begins at source line 1309. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'AttentionCache(block_index, tuple(value.shape), projection_lif, token_spikes, query_lif, key_lif, value_lif, query_heads, key_heads, value_heads, scores, attention_values, attention_lif, projection_input, output,)' to the caller of the function _attention_forward, which handles attention forward, ending this invocation. The important operands are AttentionCache: the value named 'attention cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1327: `def _mlp_forward(self, block_index: int, value: np.ndarray) -> MlpCache:` This starts function _mlp_forward. Its indented body performs mlp forward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1328: `timesteps, batch, channels, height, width = value.shape` This assigns 'value.shape' to 'timesteps, batch, channels, height, width ' in host memory. Here timesteps means one discrete SNN update instant. The important operands are value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1329: `tokens = height * width` This assigns 'height * width' to 'tokens ' in host memory. Here tokens means the value named 'tokens' in this block. The important operands are height: the value named 'height' in this block; width: the value named 'width' in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1330: `input_lif = self._lif_trace(value)` This assigns 'self._lif_trace(value)' to 'input_lif ' in host memory. Here input_lif means the value named 'input lif' in this block. The important operands are _lif_trace: the value named 'lif trace' in this block; value: the value named 'value' in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1331: `first_input = input_lif.spikes.reshape(` This statement continues through source line 1333; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'input_lif.spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)' to 'first_input

‘ in host memory. Here first_input means the value named ’first input’ in this block. The important operands are input_lif: the value named ’input lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1332: `timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(` This is continuation line 2 of the 3-line logical statement that begins at source line 1331. Its exact fragment is ‘timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘input_lif.spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)‘ to ‘first_input ‘ in host memory. Here first_input means the value named ’first input’ in this block. The important operands are input_lif: the value named ’input lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1333: `timesteps * batch * tokens, channels)` This is continuation line 3 of the 3-line logical statement that begins at source line 1331. Its exact fragment is ‘timesteps * batch * tokens, channels)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This assigns ‘input_lif.spikes.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)‘ to ‘first_input ‘ in host memory. Here first_input means the value named ’first input’ in this block. The important operands are input_lif: the value named ’input lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named ’reshape’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1334: `hidden_rows = self.projections.matrix(` This statement continues through source line 1338; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self.projections.matrix(f"block.{block_index}.mlp.fc1", first_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘hidden_rows ‘ in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1335: `f"block.{block_index}.mlp.fc1",` This is continuation line 2 of the 5-line logical statement that begins at source line 1334. Its exact fragment is ‘f"block.{block_index}.mlp.fc1",‘. The important operands are f: the value named ’f’ in this block; block: the value named ’block’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"block.{block_index}.mlp.fc1", first_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘hidden_rows ‘ in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1336: `first_input,` This is continuation line 3 of the 5-line logical statement that begins at source line 1334. Its exact fragment is ‘first_input,‘. The important operands are first_input: the value named ’first input’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"block.{block_index}.mlp.fc1", first_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘hidden_rows ‘ in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1337: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 4 of the 5-line logical statement that begins at source line 1334. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"block.{block_index}.mlp.fc1", first_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘hidden_rows ‘ in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1338: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1334. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self.projections.matrix(f"block.{block_index}.mlp.fc1", first_input, timeout_seconds=self.layer_timeout_seconds,)' to 'hidden_rows ' in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1339: `hidden_channels = hidden_rows.shape[1]` This assigns 'hidden_rows.shape[1]' to 'hidden_channels ' in host memory. Here hidden_channels means the value named 'hidden channels' in this block. The important operands are hidden_rows: one output-row position processed by the tensor tile; shape: the value named 'shape' in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1340: `hidden = hidden_rows.reshape(` This statement continues through source line 1341; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'hidden_rows.reshape(timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)' to 'hidden ' in host memory. Here hidden means the value named 'hidden' in this block. The important operands are hidden_rows: one output-row position processed by the tensor tile; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _mlp_forward, which handles mlp forward.

Line 1341: `timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)` This is continuation line 2 of the 2-line logical statement that begins at source line 1340. Its exact fragment is 'timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This assigns 'hidden_rows.reshape(timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)' to 'hidden ' in host memory. Here hidden means the value named 'hidden' in this block. The important operands are hidden_rows: one output-row position processed by the tensor tile; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _mlp_forward, which handles mlp forward.

Line 1342: `hidden_lif = self._lif_trace(hidden)` This assigns 'self._lif_trace(hidden)' to 'hidden_lif ' in host memory. Here hidden_lif means the value named 'hidden lif' in this block. The important operands are _lif_trace: the value named 'lif trace' in this block; hidden: the value named 'hidden' in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1343: `second_input = hidden_lif.spikes.transpose(0, 1, 3, 2).reshape(` This statement continues through source line 1344; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'hidden_lif.spikes.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, hidden_channels)' to 'second_input ' in host memory. Here second_input means the value named 'second input' in this block. The important operands are hidden_lif: the value named 'hidden lif' in this block; spikes: a binary event produced when a neuron crosses its threshold; transpose: the value named 'transpose' in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1344: `timesteps * batch * tokens, hidden_channels)` This is continuation line 2 of the 2-line logical statement that begins at source line 1343. Its exact fragment is 'timesteps * batch * tokens, hidden_channels)'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This assigns 'hidden_lif.spikes.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, hidden_channels)' to 'second_input ' in host memory. Here second_input means the value named 'second input' in this block. The important operands are hidden_lif: the value named 'hidden lif' in this block; spikes: a binary event produced when a neuron crosses its threshold; transpose: the value named 'transpose' in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1345: `output_rows = self.projections.matrix(` This statement continues through source line 1349; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.projections.matrix(f"block.{block_index}.mlp.fc2", second_input, timeout_seconds=self.layer_timeout_seconds,)' to 'output_rows ' in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named 'projections' in this block; matrix: the value named 'matrix' in this block; f: the value named 'f' in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1346: `f"block.{block_index}.mlp.fc2",` This is continuation line 2 of the 5-line logical statement that begins at source line 1345. Its exact fragment is ‘f"block.{block_index}.mlp.fc2",‘. The important operands are f: the value named ’f’ in this block; block: the value named ’block’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"block.{block_index}.mlp.fc2", second_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1347: `second_input,` This is continuation line 3 of the 5-line logical statement that begins at source line 1345. Its exact fragment is ‘second_input,‘. The important operands are second_input: the value named ’second input’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"block.{block_index}.mlp.fc2", second_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1348: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 4 of the 5-line logical statement that begins at source line 1345. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"block.{block_index}.mlp.fc2", second_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1349: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1345. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self.projections.matrix(f"block.{block_index}.mlp.fc2", second_input, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘output_rows ‘ in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are projections: the value named ’projections’ in this block; matrix: the value named ’matrix’ in this block; f: the value named ’f’ in this block. Context: the function _mlp_forward, which handles mlp forward.

Line 1350: `output = output_rows.reshape(` This statement continues through source line 1352; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘output_rows.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are output_rows: one output-row position processed by the tensor tile; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _mlp_forward, which handles mlp forward.

Line 1351: `timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(` This is continuation line 2 of the 3-line logical statement that begins at source line 1350. Its exact fragment is ‘timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This assigns ‘output_rows.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are output_rows: one output-row position processed by the tensor tile; reshape: the value named ’reshape’ in this block; timesteps: one discrete SNN update instant. Context: the function _mlp_forward, which handles mlp forward.

Line 1352: `timesteps, batch, channels, height, width)` This is continuation line 3 of the 3-line logical statement that begins at source line 1350. Its exact fragment is ‘timesteps, batch, channels, height, width)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘output_rows.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(timesteps, batch, channels, height, width)‘ to ‘output ‘ in host memory.

Here output means the value named 'output' in this block. The important operands are output_rows: one output-row position processed by the tensor tile; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _mlp_forward, which handles mlp forward.

Line 1353: `return MlpCache(` This statement continues through source line 1361; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘MlpCache(block_index, tuple(value.shape), input_lif, first_input, hidden_lif, second_input, output,)‘ to the caller of the function _mlp_forward, which handles mlp forward, ending this invocation. The important operands are MlpCache: the value named 'mlp cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1354: `block_index,` This is continuation line 2 of the 9-line logical statement that begins at source line 1353. Its exact fragment is ‘block_index,‘. The important operands are block_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This returns ‘MlpCache(block_index, tuple(value.shape), input_lif, first_input, hidden_lif, second_input, output,)‘ to the caller of the function _mlp_forward, which handles mlp forward, ending this invocation. The important operands are MlpCache: the value named 'mlp cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1355: `tuple(value.shape),` This is continuation line 3 of the 9-line logical statement that begins at source line 1353. Its exact fragment is ‘tuple(value.shape),‘. The important operands are tuple: the value named 'tuple' in this block; value: the value named 'value' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This returns ‘MlpCache(block_index, tuple(value.shape), input_lif, first_input, hidden_lif, second_input, output,)‘ to the caller of the function _mlp_forward, which handles mlp forward, ending this invocation. The important operands are MlpCache: the value named 'mlp cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1356: `input_lif,` This is continuation line 4 of the 9-line logical statement that begins at source line 1353. Its exact fragment is ‘input_lif,‘. The important operands are input_lif: the value named 'input lif' in this block. Read together, the complete operation is: This returns ‘MlpCache(block_index, tuple(value.shape), input_lif, first_input, hidden_lif, second_input, output,)‘ to the caller of the function _mlp_forward, which handles mlp forward, ending this invocation. The important operands are MlpCache: the value named 'mlp cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1357: `first_input,` This is continuation line 5 of the 9-line logical statement that begins at source line 1353. Its exact fragment is ‘first_input,‘. The important operands are first_input: the value named 'first input' in this block. Read together, the complete operation is: This returns ‘MlpCache(block_index, tuple(value.shape), input_lif, first_input, hidden_lif, second_input, output,)‘ to the caller of the function _mlp_forward, which handles mlp forward, ending this invocation. The important operands are MlpCache: the value named 'mlp cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1358: `hidden_lif,` This is continuation line 6 of the 9-line logical statement that begins at source line 1353. Its exact fragment is ‘hidden_lif,‘. The important operands are hidden_lif: the value named 'hidden lif' in this block. Read together, the complete operation is: This returns ‘MlpCache(block_index, tuple(value.shape), input_lif, first_input, hidden_lif, second_input, output,)‘ to the caller of the function _mlp_forward, which handles mlp forward, ending this invocation. The important operands are MlpCache: the value named 'mlp cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1359: `second_input,` This is continuation line 7 of the 9-line logical statement that begins at source line 1353. Its exact fragment is ‘second_input,‘. The important operands are second_input: the value named 'second input' in this block. Read together, the complete operation is: This returns ‘MlpCache(block_index, tuple(value.shape), input_lif, first_input, hidden_lif, second_input, output,)‘ to the caller of the function _mlp_forward, which handles mlp forward, ending this invocation. The important operands are MlpCache: the value named 'mlp cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1360: `output,` This is continuation line 8 of the 9-line logical statement that begins at source line 1353. Its exact fragment is 'output,'. The important operands are output: the value named 'output' in this block. Read together, the complete operation is: This returns 'MlpCache(block_index, tuple(value.shape), input_lif, first_input, hidden_lif, second_input, output,)' to the caller of the function _mlp_forward, which handles mlp forward, ending this invocation. The important operands are MlpCache: the value named 'mlp cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1361: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 1353. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'MlpCache(block_index, tuple(value.shape), input_lif, first_input, hidden_lif, second_input, output,)' to the caller of the function _mlp_forward, which handles mlp forward, ending this invocation. The important operands are MlpCache: the value named 'mlp cache' in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; tuple: the value named 'tuple' in this block.

Line 1363: `def forward(self, normalized_image) -> SpikingformerTrainingCache:` This starts function forward. Its indented body performs forward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1364: `"""Run the full graph and retain every state required by reverse mode."""` This is executable documentation attached to the function forward, which handles forward. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1366: `image = np.asarray(normalized_image, dtype=np.float32)` This assigns 'np.asarray(normalized_image, dtype=np.float32)' to 'image ' in host memory. Here image means the value named 'image' in this block. The important operands are asarray: the value named 'asarray' in this block; normalized_image: the value named 'normalized image' in this block; dtype: the value named 'dtype' in this block. Context: the function forward, which handles forward.

Line 1367: `if image.ndim != 4 or tuple(image.shape[1:]) != self.input_shape or image.shape[0] == 0:` This executes the following indented host-code branch only when 'image.ndim != 4 or tuple(image.shape[1:]) != self.input_shape or image.shape[0] == 0' is true. The important operands are image: the value named 'image' in this block; ndim: the value named 'ndim' in this block; or: the value named 'or' in this block. Context: the function forward, which handles forward.

Line 1368: `raise ValueError(f"input must have shape [batch,{','.join(map(str, self.input_shape))}]")` This rejects an invalid state by raising 'ValueError(f"input must have shape [batch,{','.join(map(str, self.input_shape))}]")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1369: `image_u8 = np.rint(image / self.image_scale + self.image_zero_point)` This assigns 'np.rint(image / self.image_scale + self.image_zero_point)' to 'image_u8 ' in host memory. Here image_u8 means the value named 'image u8' in this block. The important operands are rint: the value named 'rint' in this block; image: the value named 'image' in this block; image_scale: the real-value spacing represented by one quantized integer step. Context: the function forward, which handles forward.

Line 1370: `image_u8 = np.clip(image_u8, 0, 255).astype(np.uint8)` This assigns 'np.clip(image_u8, 0, 255).astype(np.ui to 'image_u8 ' in host memory. Here image_u8 means the value named 'image u8' in this block. The important operands are clip: the value named 'clip' in this block; image_u8: the value named 'image u8' in this block; astype: the value named 'astype' in this block. Context: the function forward, which handles forward.

Line 1371: `image_centered = image_u8.astype(np.int64) - self.image_zero_point` This assigns 'image_u8.astype(np.in - self.image_zero_point' to 'image_centered ' in host memory. Here image_centered means the value named 'image centered' in this block. The important operands are image_u8: the value named 'image u8' in this block; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Context: the function forward, which handles forward.

Line 1372: `self.driver.execute_program(` This statement continues through source line 1375; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.driver.execute_program‘ with ‘ (ProgramRecord(clear_status_command()),), timeout_seconds=self.layer_timeout_seconds, ‘ inside the function forward, which handles forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1373: `(ProgramRecord(clear_status_command()),),` This is continuation line 2 of the 4-line logical statement that begins at source line 1372. Its exact fragment is ‘(ProgramRecord(clear_status_command()),),‘. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘self.driver.execute_program‘ with ‘ (ProgramRecord(clear_status_command()),), timeout_seconds=self.layer_timeout_seconds, ‘ inside the function forward, which handles forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1374: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 3 of the 4-line logical statement that begins at source line 1372. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This calls ‘self.driver.execute_program‘ with ‘ (ProgramRecord(clear_status_command()),), timeout_seconds=self.layer_timeout_seconds, ‘ inside the function forward, which handles forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1375: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1372. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.driver.execute_program‘ with ‘ (ProgramRecord(clear_status_command()),), timeout_seconds=self.layer_timeout_seconds, ‘ inside the function forward, which handles forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1376: `value, tokenizer = self._tokenizer_forward(image_u8, image_centered)` This assigns ‘self._tokenizer_forw image_centered)‘ to ‘value, tokenizer ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are _tokenizer_forward: the value named ’tokenizer forward’ in this block; image_u8: the value named ’image u8’ in this block; image_centered: the value named ’image centered’ in this block. Context: the function forward, which handles forward.

Line 1377: `block_caches: list[TransformerBlockCache] = []` This assigns ‘[]‘ to ‘block_caches‘ in host memory. Here block_caches means the value named ’block caches’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function forward, which handles forward.

Line 1378: `for block_index in range(self.transformer_blocks):` This iterates block_index over ‘range(self.transformer_b Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function forward, which handles forward.

Line 1379: `block_input = value` This assigns ‘value‘ to ‘block_input ‘ in host memory. Here block_input means the value named ’block input’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function forward, which handles forward.

Line 1380: `attention = self._attention_forward(block_index, block_input)` This assigns ‘self._attention_forward(bloc block_input)‘ to ‘attention ‘ in host memory. Here attention means the value named ’attention’ in this block. The important operands are _attention_forward: the value named ’attention forward’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; block_input: the value named ’block input’ in this block. Context: the function forward, which handles forward.

Line 1381: `after_attention = execute_vector_tensor(` This statement continues through source line 1387; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, block_input, attention.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention ‘ in host memory. Here after_attention means the value named ’after attention’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1382: `self.driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1381. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, block_input, attention.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention ‘ in host memory. Here after_attention means the value named ’after attention’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1383: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 1381. Its exact fragment is ‘VectorOperation.ADD_SAT,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; ADD_SAT: the value named ’add sat’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, block_input, attention.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention ‘ in host memory. Here after_attention means the value named ’after attention’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1384: `block_input,` This is continuation line 4 of the 7-line logical statement that begins at source line 1381. Its exact fragment is ‘block_input,‘. The important operands are block_input: the value named ’block input’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, block_input, attention.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention ‘ in host memory. Here after_attention means the value named ’after attention’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1385: `attention.output,` This is continuation line 5 of the 7-line logical statement that begins at source line 1381. Its exact fragment is ‘attention.output,‘. The important operands are attention: the value named ’attention’ in this block; output: the value named ’output’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, block_input, attention.output, timeout_seconds=self.layer_timeout_)‘ to ‘after_attention ‘ in host memory. Here after_attention means the value named ’after attention’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1386: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1381. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, block_input, attention.output, timeout_seconds=self.layer_timeout_)‘ to ‘after_attention ‘ in host memory. Here after_attention means the value named ’after attention’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1387: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1381. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete op-

eration is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, block_input, attention.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention ‘ in host memory. Here after_attention means the value named ’after attention’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1388: `mlp = self._mlp_forward(block_index, after_attention)` This assigns ‘self._mlp_forward(block_index, after_attention)‘ to ‘mlp ‘ in host memory. Here mlp means the value named ’mlp’ in this block. The important operands are _mlp_forward: the value named ’mlp forward’ in this block; block_index: an integer selecting one tensor element, lane, bank, or state entry; after_attention: the value named ’after attention’ in this block. Context: the function forward, which handles forward.

Line 1389: `value = execute_vector_tensor(` This statement continues through source line 1395; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, after_attention, mlp.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1390: `self.driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1389. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, after_attention, mlp.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1391: `VectorOperation.ADD_SAT,` This is continuation line 3 of the 7-line logical statement that begins at source line 1389. Its exact fragment is ‘VectorOperation.ADD_SAT,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; ADD_SAT: the value named ’add sat’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, after_attention, mlp.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1392: `after_attention,` This is continuation line 4 of the 7-line logical statement that begins at source line 1389. Its exact fragment is ‘after_attention,‘. The important operands are after_attention: the value named ’after attention’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, after_attention, mlp.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1393: `mlp.output,` This is continuation line 5 of the 7-line logical statement that begins at source line 1389. Its exact fragment is ‘mlp.output,‘. The important operands are mlp: the value named ’mlp’ in this block; output: the value named ’output’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, after_attention, mlp.output, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘value ‘ in host memory. Here value means the value named ’value’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1394: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1389. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The

important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, after_attention, mlp.output, timeout_seconds=self.layer_timeout_s)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function forward, which handles forward.

Line 1395: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1389. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.ADD_SAT, after_attention, mlp.output, timeout_seconds=self.layer_timeout_seconds,)' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function forward, which handles forward.

Line 1396: `block_caches.append(TransformerBlockCache(` This statement continues through source line 1397; the explanation uses the complete expression rather than only the first visual fragment. This calls 'block_caches.append' with 'TransformerBlockCache(block_input, attention, after_attention, mlp, value)' inside the function forward, which handles forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1397: `block_input, attention, after_attention, mlp, value))` This is continuation line 2 of the 2-line logical statement that begins at source line 1396. Its exact fragment is 'block_input, attention, after_attention, mlp, value))'. The important operands are block_input: the value named 'block input' in this block; attention: the value named 'attention' in this block; after_attention: the value named 'after attention' in this block. Read together, the complete operation is: This calls 'block_caches.append' with 'TransformerBlockCache(block_input, attention, after_attention, mlp, value)' inside the function forward, which handles forward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1399: `pooled_sum = self._global_sum(value.astype(np.int64))` This assigns 'self._global_sum(value.astype(np.int6 to 'pooled_sum ' in host memory. Here pooled_sum means the value named 'pooled sum' in this block. The important operands are _global_sum: the value named 'global sum' in this block; value: the value named 'value' in this block; astype: the value named 'astype' in this block. Context: the function forward, which handles forward.

Line 1400: `divisor = value.shape[0] * value.shape[3] * value.shape[4]` This assigns 'value.shape[0] * value.shape[3] * value.shape[4]' to 'divisor ' in host memory. Here divisor means the value named 'divisor' in this block. The important operands are value: the value named 'value' in this block; shape: the value named 'shape' in this block. Context: the function forward, which handles forward.

Line 1401: `denominator = divisor * self.fixed_scale * self.head_activation_scale` This assigns 'divisor * self.fixed_scale * self.head_activation_scale' to 'denominator ' in host memory. Here denominator means the value named 'denominator' in this block. The important operands are divisor: the value named 'divisor' in this block; fixed_scale: the real-value spacing represented by one quantized integer step; head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function forward, which handles forward.

Line 1402: `rounded_denominator = int(round(denominator))` This assigns 'int(round(denominator))' to 'rounded_denomina ' in host memory. Here rounded_denominator means the value named 'rounded denominator' in this block. The important operands are round: the value named 'round' in this block; denominator: the value named 'denominator' in this block. Context: the function forward, which handles forward.

Line 1403: `if not math.isclose(denominator, rounded_denominator, rel_tol=0.0, abs_tol=1e-9):` This executes the following indented host-code branch only when 'not math.isclose(denominator, rounded_denominator, rel_tol=0.0, abs_tol=1e-9)' is true. The important operands are not: the value named 'not' in this block; math: the value named 'math' in this block; isclose: the value named 'isclose' in this block. Context: the function forward, which handles forward.

Line 1404: `raise ValueError("head pooling denominator is not an exact integer")` This rejects an invalid state by raising ‘ValueError("head pooling denominator is not an exact integer")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1405: `if rounded_denominator <= 0 or rounded_denominator & (rounded_denominator - 1):` This executes the following indented host-code branch only when ‘rounded_denominator <= 0 or rounded_denominator & (rounded_denominator - 1)‘ is true. The important operands are rounded_denominator: the value named ’rounded denominator’ in this block; or: the value named ’or’ in this block. Context: the function forward, which handles forward.

Line 1406: `raise ValueError("head pooling denominator must be a positive power of two")` This rejects an invalid state by raising ‘ValueError("head pooling denominator must be a positive power of two")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1407: `head_preclip = execute_power_of_two_scale(` This statement continues through source line 1412; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_power_of_two_scale(self.driver, pooled_sum, -(rounded_denominator.bit_length() - 1), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_preclip ‘ in host memory. Here head_preclip means the value named ’head preclip’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; pooled_sum: the value named ’pooled sum’ in this block. Context: the function forward, which handles forward.

Line 1408: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1407. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(self.driver, pooled_sum, -(rounded_denominator.bit_length() - 1), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_preclip ‘ in host memory. Here head_preclip means the value named ’head preclip’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; pooled_sum: the value named ’pooled sum’ in this block. Context: the function forward, which handles forward.

Line 1409: `pooled_sum,` This is continuation line 3 of the 6-line logical statement that begins at source line 1407. Its exact fragment is ‘pooled_sum,‘. The important operands are pooled_sum: the value named ’pooled sum’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(self.driver, pooled_sum, -(rounded_denominator.bit_length() - 1), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_preclip ‘ in host memory. Here head_preclip means the value named ’head preclip’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; pooled_sum: the value named ’pooled sum’ in this block. Context: the function forward, which handles forward.

Line 1410: `-(rounded_denominator.bit_length() - 1),` This is continuation line 4 of the 6-line logical statement that begins at source line 1407. Its exact fragment is ‘-(rounded_denominator.bit_length() - 1),‘. The important operands are rounded_denominator: the value named ’rounded denominator’ in this block; bit_length: the value named ’bit length’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(self.driver, pooled_sum, -(rounded_denominator.bit_length() - 1), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_preclip ‘ in host memory. Here head_preclip means the value named ’head preclip’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; pooled_sum: the value named ’pooled sum’ in this block. Context: the function forward, which handles forward.

Line 1411: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1407. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(self.driver, pooled_sum, -(rounded_denominator.bit_length() - 1), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_preclip ‘ in host memory. Here head_preclip means the value named ’head preclip’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step;

driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function forward, which handles forward.

Line 1412: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1407. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_power_of_two_scale(self.driver, pooled_sum, -(rounded_denominator.bit_length() - 1), timeout_seconds=self.layer_timeout_seconds,)' to 'head_preclip ' in host memory. Here head_preclip means the value named 'head preclip' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; pooled_sum: the value named 'pooled sum' in this block. Context: the function forward, which handles forward.

Line 1413: `head_input = execute_vector_tensor(` This statement continues through source line 1420; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_vector_tensor(self.driver, VectorOperation.CLAMP, head_preclip, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_t)' to 'head_input ' in host memory. Here head_input means the value named 'head input' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function forward, which handles forward.

Line 1414: `self.driver,` This is continuation line 2 of the 8-line logical statement that begins at source line 1413. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.CLAMP, head_preclip, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input ' in host memory. Here head_input means the value named 'head input' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function forward, which handles forward.

Line 1415: `VectorOperation.CLAMP,` This is continuation line 3 of the 8-line logical statement that begins at source line 1413. Its exact fragment is 'VectorOperation.CLAMP,'. The important operands are VectorOperation: the value named 'vector operation' in this block; CLAMP: the value named 'clamp' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.CLAMP, head_preclip, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input ' in host memory. Here head_input means the value named 'head input' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function forward, which handles forward.

Line 1416: `head_preclip,` This is continuation line 4 of the 8-line logical statement that begins at source line 1413. Its exact fragment is 'head_preclip,'. The important operands are head_preclip: the value named 'head preclip' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.CLAMP, head_preclip, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input ' in host memory. Here head_input means the value named 'head input' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function forward, which handles forward.

Line 1417: `clamp_minimum=-128,` This is continuation line 5 of the 8-line logical statement that begins at source line 1413. Its exact fragment is 'clamp_minimum=-128,'. The important operands are clamp_minimum: the value named 'clamp minimum' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.CLAMP, head_preclip, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_t)' to 'head_input ' in host memory. Here head_input means the value named 'head input' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function forward, which handles forward.

Line 1418: `clamp_maximum=127,` This is continuation line 6 of the 8-line logical statement that begins at source line 1413. Its exact fragment is ‘clamp_maximum=127,‘. The important operands are clamp_maximum: the value named ’clamp maximum’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.CLAMP, head_preclip, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_t)‘ to ‘head_input ‘ in host memory. Here head_input means the value named ’head input’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1419: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 7 of the 8-line logical statement that begins at source line 1413. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.CLAMP, head_preclip, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_input ‘ in host memory. Here head_input means the value named ’head input’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1420: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 1413. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.CLAMP, head_preclip, clamp_minimum=-128, clamp_maximum=127, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_input ‘ in host memory. Here head_input means the value named ’head input’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function forward, which handles forward.

Line 1421: `head = self.state.head` This assigns ‘self.state.head‘ to ‘head ‘ in host memory. Here head means the value named ’head’ in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; head: the value named ’head’ in this block. Context: the function forward, which handles forward.

Line 1422: `head_program = compile_u8_gemm(` This statement continues through source line 1428; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_u8_gemm((head_input + 128).astype(np.uint8), (head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)‘ to ‘head_program ‘ in host memory. Here head_program means the value named ’head program’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; head_input: the value named ’head input’ in this block; astype: the value named ’astype’ in this block. Context: the function forward, which handles forward.

Line 1423: `(head_input + 128).astype(np.uint8),` This is continuation line 2 of the 7-line logical statement that begins at source line 1422. Its exact fragment is ‘(head_input + 128).astype(np.uint8),‘. The important operands are head_input: the value named ’head input’ in this block; astype: the value named ’astype’ in this block; uint8: the value named ’uint8’ in this block. Read together, the complete operation is: This assigns ‘compile_u8_gemm((head_input + 128).astype(np.uint8), (head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)‘ to ‘head_program ‘ in host memory. Here head_program means the value named ’head program’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; head_input: the value named ’head input’ in this block; astype: the value named ’astype’ in this block. Context: the function forward, which handles forward.

Line 1424: `(head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8),` This is continuation line 3 of the 7-line logical statement that begins at source line 1422. Its exact fragment is ‘(head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8),‘. The important operands are head: the value named ’head’ in this block; weight_k_by_n: a learned coefficient multiplied by an activation; astype: the value named ’astype’ in this block. Read together, the complete operation is: This assigns ‘compile_u8_gemm((head_input + 128).astype(np.uint8), (head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)‘

to ‘head_program ‘ in host memory. Here head_program means the value named ’head program’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; head_input: the value named ’head input’ in this block; astype: the value named ’astype’ in this block. Context: the function forward, which handles forward.

Line 1425: `activation_zero_point=128,` This is continuation line 4 of the 7-line logical statement that begins at source line 1422. Its exact fragment is ‘activation_zero_point=128,‘. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘compile_u8_gemm((head_input + 128).astype(np.uint8), (head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)‘ to ‘head_program ‘ in host memory. Here head_program means the value named ’head program’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; head_input: the value named ’head input’ in this block; astype: the value named ’astype’ in this block. Context: the function forward, which handles forward.

Line 1426: `weight_zero_point=128,` This is continuation line 5 of the 7-line logical statement that begins at source line 1422. Its exact fragment is ‘weight_zero_point=128,‘. The important operands are weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘compile_u8_gemm((head_input + 128).astype(np.uint8), (head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)‘ to ‘head_program ‘ in host memory. Here head_program means the value named ’head program’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; head_input: the value named ’head input’ in this block; astype: the value named ’astype’ in this block. Context: the function forward, which handles forward.

Line 1427: `bias=head.bias_accumulator_units,` This is continuation line 6 of the 7-line logical statement that begins at source line 1422. Its exact fragment is ‘bias=head.bias_accumulator_units,‘. The important operands are bias: an additive learned offset applied after accumulation; head: the value named ’head’ in this block; bias_accumulator_units: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘compile_u8_gemm((head_input + 128).astype(np.uint8), (head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)‘ to ‘head_program ‘ in host memory. Here head_program means the value named ’head program’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; head_input: the value named ’head input’ in this block; astype: the value named ’astype’ in this block. Context: the function forward, which handles forward.

Line 1428: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1422. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘compile_u8_gemm((head_input + 128).astype(np.uint8), (head.weight_k_by_n.astype(np.int16) + 128).astype(np.uint8), activation_zero_point=128, weight_zero_point=128, bias=head.bias_accumulator_units,)‘ to ‘head_program ‘ in host memory. Here head_program means the value named ’head program’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; head_input: the value named ’head input’ in this block; astype: the value named ’astype’ in this block. Context: the function forward, which handles forward.

Line 1429: `logits_accumulator = head_program.execute_batched(` This statement continues through source line 1430; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘head_program.execute_b self.driver, timeout_seconds=self.layer_timeout_seconds)‘ to ‘logits_accumulator ‘ in host memory. Here logits_accumulator means a widened running sum that prevents intermediate overflow. The important operands are head_program: the value named ’head program’ in this block; execute_batched: the value named ’execute batched’ in this block; driver: the value named ’driver’ in this block. Context: the function forward, which handles forward.

Line 1430: `self.driver, timeout_seconds=self.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 1429. Its exact fragment is ‘self.driver, timeout_seconds=self.layer_timeout_seconds)‘. The important operands are driver: the value named ’driver’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘head_program.execute_batched(self.driver, timeout_seconds=self.layer_timeout_seconds)‘

to ‘logits_accumulator ‘ in host memory. Here logits_accumulator means a widened running sum that prevents intermediate overflow. The important operands are head_program: the value named ’head program’ in this block; execute_batched: the value named ’execute batched’ in this block; driver: the value named ’driver’ in this block. Context: the function forward, which handles forward.

Line 1431: `accumulator_scale = float(self.state.manifest["head"]["accumulator_scale"])` This assigns ‘float(self.sta
to ‘accumulator_scale ‘ in host memory. Here accumulator_scale means a widened running sum that prevents intermediate overflow. The important operands are float: the value named ’float’ in this block; state: the finite-state-machine phase retained in flip-flops; manifest: the value named ’manifest’ in this block. Context: the function forward, which handles forward.

Line 1432: `logits = logits_accumulator.astype(np.float64) * accumulator_scale` This assigns ‘logits_accumulator.asty
* accumulator_scale‘ to ‘logits ‘ in host memory. Here logits means the ten unnormalized classifier scores used for argmax. The important operands are logits_accumulator: a widened running sum that prevents intermediate overflow; astype: the value named ’astype’ in this block; float64: the value named ’float64’ in this block. Context: the function forward, which handles forward.

Line 1433: `return SpikingformerTrainingCache(` This statement continues through source line 1444; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1434: `image,` This is continuation line 2 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘image,‘. The important operands are image: the value named ’image’ in this block. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1435: `image_u8,` This is continuation line 3 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘image_u8,‘. The important operands are image_u8: the value named ’image u8’ in this block. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1436: `image_centered,` This is continuation line 4 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘image_centered,‘. The important operands are image_centered: the value named ’image centered’ in this block. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1437: `tokenizer,` This is continuation line 5 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘tokenizer,‘. The important operands are tokenizer: the value named ’tokenizer’ in this block. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1438: `tuple(block_caches),` This is continuation line 6 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘tuple(block_caches),‘. The important operands are tuple: the value named ’tuple’ in this block; block_caches: the value named ’block caches’ in this block. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1439: `value,` This is continuation line 7 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘value,‘. The important operands are value: the value named ’value’ in this block. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1440: `head_preclip,` This is continuation line 8 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘head_preclip,‘. The important operands are head_preclip: the value named ’head preclip’ in this block. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1441: `head_input,` This is continuation line 9 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘head_input,‘. The important operands are head_input: the value named ’head input’ in this block. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1442: `logits_accumulator,` This is continuation line 10 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘logits_accumulator,‘. The important operands are logits_accumulator: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1443: `logits,` This is continuation line 11 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘logits,‘. The important operands are logits: the ten unnormalized classifier scores used for argmax. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1444: `)` This is continuation line 12 of the 12-line logical statement that begins at source line 1433. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘SpikingformerTrainingCache(image, image_u8, image_centered, tokenizer, tuple(block_caches), value, head_preclip, head_input, logits_accumulator, logits,)‘ to the caller of the function forward, which handles forward, ending this invocation. The important operands are SpikingformerTrainingCache: the value named ’spikingformer training cache’ in this block; image: the value named ’image’ in this block; image_u8: the value named ’image u8’ in this block.

Line 1446: `def _lif_backward(` This starts function _lif_backward. Its indented body performs lif backward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1447: `self, cache: LifTraceCache, output_gradient: BlockGradient` Inside the function _lif_backward, which handles lif backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are cache: the value named 'cache' in this block; LifTraceCache: the value named 'lif trace cache' in this block; output_gradient: the signed loss derivative used to update a learned parameter.

Line 1448: `) -> BlockGradient:` Inside the function _lif_backward, which handles lif backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1449: `if output_gradient.values.shape != cache.spikes.shape:` This executes the following indented host-code branch only when 'output_gradient.values.shape != cache.spikes.shape' is true. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block; shape: the value named 'shape' in this block. Context: the function _lif_backward, which handles lif backward.

Line 1450: `raise ValueError("LIF output gradient does not match its forward trace")` This rejects an invalid state by raising 'ValueError("LIF output gradient does not match its forward trace")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1451: `timesteps = cache.spikes.shape[0]` This assigns 'cache.spikes.shape[0]' to 'timesteps ' in host memory. Here timesteps means one discrete SNN update instant. The important operands are cache: the value named 'cache' in this block; spikes: a binary event produced when a neuron crosses its threshold; shape: the value named 'shape' in this block. Context: the function _lif_backward, which handles lif backward.

Line 1452: `membrane = cache.pre_reset_membrane.reshape(timesteps, -1)` This assigns 'cache.pre_reset_membrane.resha -1)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are cache: the value named 'cache' in this block; pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; reshape: the value named 'reshape' in this block. Context: the function _lif_backward, which handles lif backward.

Line 1453: `spikes = cache.spikes.reshape(timesteps, -1)` This assigns 'cache.spikes.reshape(timesteps, -1)' to 'spikes ' in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are cache: the value named 'cache' in this block; spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block. Context: the function _lif_backward, which handles lif backward.

Line 1454: `gradient = output_gradient.values.reshape(timesteps, -1)` This assigns 'output_gradient.values.reshape(tim -1)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block; reshape: the value named 'reshape' in this block. Context: the function _lif_backward, which handles lif backward.

Line 1455: `input_gradient, _ = execute_lif_bptt(` This statement continues through source line 1465; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.frac - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1456: `self.driver,` This is continuation line 2 of the 11-line logical statement that begins at source line 1455. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1457: `membrane,` This is continuation line 3 of the 11-line logical statement that begins at source line 1455. Its exact fragment is 'membrane,'. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1458: `spikes,` This is continuation line 4 of the 11-line logical statement that begins at source line 1455. Its exact fragment is 'spikes,'. The important operands are spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1459: `gradient,` This is continuation line 5 of the 11-line logical statement that begins at source line 1455. Its exact fragment is 'gradient,'. The important operands are gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1460: `threshold=cache.threshold_q10,` This is continuation line 6 of the 11-line logical statement that begins at source line 1455. Its exact fragment is 'threshold=cache.threshold_q10,'. The important operands are threshold: the membrane level at which a neuron emits a spike; cache: the value named 'cache' in this block; threshold_q10: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1461: `reciprocal_tau_q15=1 << 14,` This is continuation line 7 of the 11-line logical statement that begins at source line 1455. Its exact fragment is 'reciprocal_tau_q15=1 << 14,'. The important operands are reciprocal_tau_q15: the value named 'reciprocal tau q15' in this block. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means

the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1462: `surrogate_address_shift=max(self.fractional_bits - 5, 0),` This is continuation line 8 of the 11-line logical statement that begins at source line 1455. Its exact fragment is 'surrogate_address_shift=max(self.fractional_bits - 5, 0),'. The important operands are surrogate_address_shift: the value named 'surrogate address shift' in this block; max: the value named 'max' in this block; fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1463: `load_surrogate_table=not self._surrogate_lut_loaded,` This is continuation line 9 of the 11-line logical statement that begins at source line 1455. Its exact fragment is 'load_surrogate_table=not self._surrogate_lut_loaded,'. The important operands are load_surrogate_table: the value named 'load surrogate table' in this block; not: the value named 'not' in this block; _surrogate_lut_loaded: the value named 'surrogate lut loaded' in this block. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1464: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 10 of the 11-line logical statement that begins at source line 1455. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1465: `)` This is continuation line 11 of the 11-line logical statement that begins at source line 1455. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_lif_bptt(self.driver, membrane, spikes, gradient, threshold=cache.threshold_q10, reciprocal_tau_q15=1 << 14, surrogate_address_shift=max(self.fractional_bits - 5, 0), load_surrogate_table=not self._surrogate_lut_loaded, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, _ ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; driver: the value named 'driver' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function _lif_backward, which handles lif backward.

Line 1466: `self._surrogate_lut_loaded = True` This assigns 'True' to 'self._surrogate_lut_loaded ' in host memory. Here _surrogate_lut_loaded means the value named 'surrogate lut loaded' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _lif_backward, which handles lif backward.

Line 1467: `return BlockGradient(` This statement continues through source line 1468; the explanation uses the complete expression rather than only the first visual fragment. This returns 'BlockGradient(input_gradient.reshape(cache.input_q10.sh

output_gradient.exponent)‘ to the caller of the function _lif_backward, which handles lif backward, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; input_gradient: the signed loss derivative used to update a learned parameter; reshape: the value named ’reshape’ in this block.

Line 1468: `input_gradient.reshape(cache.input_q10.shape), output_gradient.exponent)` This is continuation line 2 of the 2-line logical statement that begins at source line 1467. Its exact fragment is ‘input_gradient.reshape(cache.input_q
output_gradient.exponent)‘. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; reshape: the value named ’reshape’ in this block; cache: the value named ’cache’ in this block. Read together, the complete operation is: This returns ‘BlockGradient(input_gradient.reshape(cache.input_q10.shape), output_gradient.exponent)‘ to the caller of the function _lif_backward, which handles lif backward, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; input_gradient: the signed loss derivative used to update a learned parameter; reshape: the value named ’reshape’ in this block.

Line 1470: `def _bias_gradient(self, output_gradient: BlockGradient) -> BlockGradient:` This starts function _bias_gradient. Its indented body performs bias gradient as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1471: `if output_gradient.values.ndim != 2:` This executes the following indented host-code branch only when ‘output_gradient.values.ndim != 2‘ is true. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; ndim: the value named ’ndim’ in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1472: `raise ValueError("bias reduction expects a [rows,channels] gradient")` This rejects an invalid state by raising ‘ValueError("bias reduction expects a [rows,channels] gradient")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1473: `rows, channels = output_gradient.values.shape` This assigns ‘output_gradient.values.shape‘ to ‘rows, channels ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1474: `ones = np.ones((rows, 1), dtype=np.int64)` This assigns ‘np.ones((rows, 1), dtype=np.int64)‘ to ‘ones ‘ in host memory. Here ones means the value named ’ones’ in this block. The important operands are ones: the value named ’ones’ in this block; rows: one output-row position processed by the tensor tile; dtype: the value named ’dtype’ in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1475: `zeros = np.zeros((1, channels), dtype=np.int64)` This assigns ‘np.zeros((1, channels), dtype=np.int64)‘ to ‘zeros ‘ in host memory. Here zeros means the value named ’zeros’ in this block. The important operands are zeros: the value named ’zeros’ in this block; channels: the value named ’channels’ in this block; dtype: the value named ’dtype’ in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1476: `_, gradient = execute_linear_backward_block(` This statement continues through source line 1482; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_linear_backward_bloc
self.driver, ones, zeros, output_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘_, gradient ‘ in host memory. Here _ means the value named ” in this block. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; ones: the value named ’ones’ in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1477: `self.driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1476. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, ones, zeros, output_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘_, gradient ‘ in host memory. Here _ means the value named ”

in this block. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; ones: the value named 'ones' in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1478: `ones,` This is continuation line 3 of the 7-line logical statement that begins at source line 1476. Its exact fragment is 'ones,'. The important operands are ones: the value named 'ones' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, ones, zeros, output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to '_, gradient ' in host memory. Here _ means the value named '' in this block. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; ones: the value named 'ones' in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1479: `zeros,` This is continuation line 4 of the 7-line logical statement that begins at source line 1476. Its exact fragment is 'zeros,'. The important operands are zeros: the value named 'zeros' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, ones, zeros, output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to '_, gradient ' in host memory. Here _ means the value named '' in this block. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; ones: the value named 'ones' in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1480: `output_gradient,` This is continuation line 5 of the 7-line logical statement that begins at source line 1476. Its exact fragment is 'output_gradient,'. The important operands are output_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, ones, zeros, output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to '_, gradient ' in host memory. Here _ means the value named '' in this block. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; ones: the value named 'ones' in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1481: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1476. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, ones, zeros, output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to '_, gradient ' in host memory. Here _ means the value named '' in this block. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; ones: the value named 'ones' in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1482: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1476. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, ones, zeros, output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to '_, gradient ' in host memory. Here _ means the value named '' in this block. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; ones: the value named 'ones' in this block. Context: the function _bias_gradient, which handles bias gradient.

Line 1483: `return BlockGradient(gradient.values.reshape(channels), gradient.exponent)` This returns 'BlockGradient(gradient.values.reshape(channels), gradient.exponent)' to the caller of the function _bias_gradient, which handles bias gradient, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block.

Line 1485: `def _projection_matrix_backward(` This starts function _projection_matrix_backward. Its indented body performs projection matrix backward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1486: `self,` Inside the function _projection_matrix_backward, which handles projection matrix backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1487: `name: str,` Inside the function _projection_matrix_backward, which handles projection matrix backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are name: the value named 'name' in this block; str: the value named 'str' in this block.

Line 1488: `activations: np.ndarray,` Inside the function _projection_matrix_backward, which handles projection matrix backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named 'ndarray' in this block.

Line 1489: `output_gradient: BlockGradient,` Inside the function _projection_matrix_backward, which handles projection matrix backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1490: `) -> tuple[BlockGradient, ParameterGradient]:` Inside the function _projection_matrix_backward, which handles projection matrix backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named 'tuple' in this block; BlockGradient: the signed loss derivative used to update a learned parameter; ParameterGradient: the signed loss derivative used to update a learned parameter.

Line 1491: `parameters = self.state.projection(name)` This assigns ‘self.state.projection(name)‘ to ‘parameters ‘ in host memory. Here parameters means the value named 'parameters' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; projection: the value named 'projection' in this block; name: the value named 'name' in this block. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1492: `scaled_gradient = backward_requantize_projection(` This statement continues through source line 1497; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘backward_requantize_projection(self.driver, output_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘scaled_gradient ‘ in host memory. Here scaled_gradient means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1493: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1492. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns ‘backward_requantize_projection(self.driver, output_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘scaled_gradient ‘ in host memory. Here scaled_gradient means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1494: `output_gradient,` This is continuation line 3 of the 6-line logical statement that begins at source line 1492. Its exact fragment is ‘output_gradient,‘. The important operands are output_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘backward_requantize_projection(self.driver, output_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘scaled_gradient ‘ in host

memory. Here scaled_gradient means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1495: `parameters,` This is continuation line 4 of the 6-line logical statement that begins at source line 1492. Its exact fragment is 'parameters,'. The important operands are parameters: the value named 'parameters' in this block. Read together, the complete operation is: This assigns 'backward_requantize_projection(self.driver, output_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)' to 'scaled_gradient ' in host memory. Here scaled_gradient means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1496: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1492. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'backward_requantize_projection(self.driver, output_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)' to 'scaled_gradient ' in host memory. Here scaled_gradient means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1497: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1492. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'backward_requantize_projection(self.driver, output_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)' to 'scaled_gradient ' in host memory. Here scaled_gradient means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1498: `input_gradient, weight_gradient = execute_linear_backward_block(` This statement continues through source line 1504; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_linear_backward_block(self.driver, activations, parameters.weight_k_by_n, scaled_gradient, timeout_seconds=self.layer_t)' to 'input_gradient, weight_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1499: `self.driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1498. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, activations, parameters.weight_k_by_n, scaled_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, weight_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1500: `activations,` This is continuation line 3 of the 7-line logical statement that begins at source line 1498. Its exact fragment is 'activations,'. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, activations, parameters.weight_k_by_n, scaled_gradient, timeout_seconds=self.layer_timeout_seconds,)'

to ‘input_gradient, weight_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1501: `parameters.weight_k_by_n,` This is continuation line 4 of the 7-line logical statement that begins at source line 1498. Its exact fragment is ‘parameters.weight_k_by_n,‘. The important operands are parameters: the value named ’parameters’ in this block; weight_k_by_n: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, activations, parameters.weight_k_by_n, scaled_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘input_gradient, weight_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1502: `scaled_gradient,` This is continuation line 5 of the 7-line logical statement that begins at source line 1498. Its exact fragment is ‘scaled_gradient,‘. The important operands are scaled_gradient: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, activations, parameters.weight_k_by_n, scaled_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘input_gradient, weight_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1503: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1498. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, activations, parameters.weight_k_by_n, scaled_gradient, timeout_seconds=self.layer_t)‘ to ‘input_gradient, weight_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1504: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1498. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, activations, parameters.weight_k_by_n, scaled_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘input_gradient, weight_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1505: `bias_gradient = self._bias_gradient(output_gradient)` This assigns ‘self._bias_gradient(output_gradient)‘ to ‘bias_gradient ‘ in host memory. Here bias_gradient means an additive learned offset applied after accumulation. The important operands are _bias_gradient: an additive learned offset applied after accumulation; output_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_matrix_backward, which handles projection matrix backward.

Line 1506: `return input_gradient, ParameterGradient(weight_gradient, bias_gradient)` This returns ‘input_gradient, ParameterGradient(weight_gradient, bias_gradient)‘ to the caller of the function _projection_matrix_backward, which handles projection matrix backward, ending this invocation. The important operands are input_gradient: the

signed loss derivative used to update a learned parameter; ParameterGradient: the signed loss derivative used to update a learned parameter; weight_gradient: a learned coefficient multiplied by an activation.

Line 1508: `def _projection_conv2d_backward(` This starts function _projection_conv2d_backward. Its indented body performs projection conv2d backward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1509: `self,` Inside the function _projection_conv2d_backward, which handles projection conv2d backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1510: `name: str,` Inside the function _projection_conv2d_backward, which handles projection conv2d backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are name: the value named 'name' in this block; str: the value named 'str' in this block.

Line 1511: `activations: np.ndarray,` Inside the function _projection_conv2d_backward, which handles projection conv2d backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named 'ndarray' in this block.

Line 1512: `output_gradient: BlockGradient,` Inside the function _projection_conv2d_backward, which handles projection conv2d backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1513: `*,` Inside the function _projection_conv2d_backward, which handles projection conv2d backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1514: `stride=1,` This assigns ‘1,‘ to ‘stride‘ in host memory. Here stride means the value named 'stride' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1515: `padding=0,` This assigns ‘0,‘ to ‘padding‘ in host memory. Here padding means the value named 'padding' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1516: `dilation=1,` This assigns ‘1,‘ to ‘dilation‘ in host memory. Here dilation means the value named 'dilation' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1517: `groups: int = 1,` This assigns ‘1,‘ to ‘groups‘ in host memory. Here groups means the value named 'groups' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1518: `) -> tuple[BlockGradient, ParameterGradient]:` Inside the function _projection_conv2d_backward, which handles projection conv2d backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named 'tuple' in this block; BlockGradient: the signed loss derivative used to update a learned parameter; ParameterGradient: the signed loss derivative used to update a learned parameter.

Line 1519: `parameters = self.state.projection(name)` This assigns ‘self.state.projection(name)‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; projection: the value named ’projection’ in this block; name: the value named ’name’ in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1520: `values = output_gradient.values` This assigns ‘output_gradient.values‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1521: `if values.ndim != 4:` This executes the following indented host-code branch only when ‘values.ndim != 4‘ is true. The important operands are values: the value named ’values’ in this block; ndim: the value named ’ndim’ in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1522: `raise ValueError("projection convolution gradient must be NCHW")` This rejects an invalid state by raising ‘ValueError("projection convolution gradient must be NCHW")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1523: `batch, channels, height, width = values.shape` This assigns ‘values.shape‘ to ‘batch, channels, height, width ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1524: `matrix_gradient = output_gradient.transpose(0, 2, 3, 1).reshape(` This statement continues through source line 1525; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘output_gradient.transpose(0, 2, 3, 1).reshape(batch * height * width, channels)‘ to ‘matrix_gradient ‘ in host memory. Here matrix_gradient means the signed loss derivative used to update a learned parameter. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1525: `batch * height * width, channels)` This is continuation line 2 of the 2-line logical statement that begins at source line 1524. Its exact fragment is ‘batch * height * width, channels)‘. The important operands are batch: the value named ’batch’ in this block; height: the value named ’height’ in this block; width: the value named ’width’ in this block. Read together, the complete operation is: This assigns ‘output_gradient.transpose(0, 2, 3, 1).reshape(batch * height * width, channels)‘ to ‘matrix_gradient ‘ in host memory. Here matrix_gradient means the signed loss derivative used to update a learned parameter. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; transpose: the value named ’transpose’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1526: `scaled_matrix = backward_requantize_projection(` This statement continues through source line 1531; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘backward_requantize_projection(self.driver, matrix_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘scaled_matrix ‘ in host memory. Here scaled_matrix means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named ’driver’ in this block; matrix_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1527: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1526. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘backward_requantize_projection(self.driver, matrix_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘scaled_matrix ‘ in host memory. Here scaled_matrix means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection:

the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; matrix_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1528: `matrix_gradient,` This is continuation line 3 of the 6-line logical statement that begins at source line 1526. Its exact fragment is 'matrix_gradient,'. The important operands are matrix_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'backward_requantize_projection(self.driver, matrix_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)' to 'scaled_matrix ' in host memory. Here scaled_matrix means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; matrix_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1529: `parameters,` This is continuation line 4 of the 6-line logical statement that begins at source line 1526. Its exact fragment is 'parameters,'. The important operands are parameters: the value named 'parameters' in this block. Read together, the complete operation is: This assigns 'backward_requantize_projection(self.driver, matrix_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)' to 'scaled_matrix ' in host memory. Here scaled_matrix means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projectio the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; matrix_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1530: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1526. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'backward_requantize_projection(self.driver, matrix_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)' to 'scaled_matrix ' in host memory. Here scaled_matrix means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; matrix_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1531: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1526. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'backward_requantize_projection(self.driver, matrix_gradient, parameters, timeout_seconds=self.layer_timeout_seconds,)' to 'scaled_matrix ' in host memory. Here scaled_matrix means the real-value spacing represented by one quantized integer step. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named 'driver' in this block; matrix_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1532: `scaled_nchw = scaled_matrix.reshape(` This statement continues through source line 1533; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'scaled_matrix.reshape(batch, height, width, channels).transpose(0, 3, 1, 2)' to 'scaled_nchw ' in host memory. Here scaled_nchw means the real-value spacing represented by one quantized integer step. The important operands are scaled_matrix: the real-value spacing represented by one quantized integer step; reshape: the value named 'reshape' in this block; batch: the value named 'batch' in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1533: `batch, height, width, channels).transpose(0, 3, 1, 2)` This is continuation line 2 of the 2-line logical statement that begins at source line 1532. Its exact fragment is 'batch, height, width, channels).transpose(0, 3, 1, 2)'. The important operands are batch: the value named 'batch' in this block; height: the value named 'height' in this block; width: the value named 'width' in this block. Read together, the complete operation is: This assigns 'scaled_matrix.reshape(batch, height, width, channels).transpose(0, 3, 1, 2)' to 'scaled_nchw ' in host memory. Here scaled_nchw means the real-value spacing represented by one quantized integer step. The important operands are

scaled_matrix: the real-value spacing represented by one quantized integer step; reshape: the value named 'reshape' in this block; batch: the value named 'batch' in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1534: `weight_oihw = parameters.weight_k_by_n.T.reshape(parameters.source_weight_shape)` This assigns 'parameters.weight_k_by_n.T.reshape(parameters.source_weight_shape)' to 'weight_oihw ' in host memory. Here weight_oihw means a learned coefficient multiplied by an activation. The important operands are parameters: the value named 'parameters' in this block; weight_k_by_n: a learned coefficient multiplied by an activation; T: the value named 't' in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1535: `input_gradient, accumulator_parameter = execute_conv2d_backward_block(` This statement continues through source line 1546; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1536: `self.driver,` This is continuation line 2 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1537: `activations,` This is continuation line 3 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'activations,'. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1538: `weight_oihw,` This is continuation line 4 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'weight_oihw,'. The important operands are weight_oihw: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1539: `parameters.bias_q10,` This is continuation line 5 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'parameters.bias_q10,'. The important operands are parameters: the value named 'parameters' in this block; bias_q10: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative

used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1540: `scaled_nchw,` This is continuation line 6 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'scaled_nchw,'. The important operands are scaled_nchw: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1541: `stride=stride,` This is continuation line 7 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'stride=stride,'. The important operands are stride: the value named 'stride' in this block. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1542: `padding=padding,` This is continuation line 8 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'padding=padding,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1543: `dilation=dilation,` This is continuation line 9 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'dilation=dilation,'. The important operands are dilation: the value named 'dilation' in this block. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1544: `groups=groups,` This is continuation line 10 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'groups=groups,'. The important operands are groups: the value named 'groups' in this block. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1545: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 11 of the 12-line logical statement that begins at source line 1535. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1546: `)` This is continuation line 12 of the 12-line logical statement that begins at source line 1535. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_conv2d_backward_block(self.driver, activations, weight_oihw, parameters.bias_q10, scaled_nchw, stride=stride, padding=padding, dilation=dilation, groups=groups, timeout_seconds=self.layer_timeo)' to 'input_gradient, accumulator_parameter ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_conv2d_backward_block: the value named 'execute conv2d backward block' in this block; driver: the value named 'driver' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1547: `weight_gradient = accumulator_parameter.weight.values.reshape(` This statement continues through source line 1548; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'accumulator_parameter.weight.values.reshape(parameters.source_weight_shape[0], -1).T' to 'weight_gradient ' in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are accumulator_parameter: a widened running sum that prevents intermediate overflow; weight: a learned coefficient multiplied by an activation; values: the value named 'values' in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1548: `parameters.source_weight_shape[0], -1).T` This is continuation line 2 of the 2-line logical statement that begins at source line 1547. Its exact fragment is 'parameters.source_weight_shape[0], -1).T'. The important operands are parameters: the value named 'parameters' in this block; source_weight_shape: a learned coefficient multiplied by an activation; T: the value named 't' in this block. Read together, the complete operation is: This assigns 'accumulator_parameter.weight.values.reshape(parameters.source_weight_shape[0], -1).T' to 'weight_gradient ' in host memory. Here weight_gradient means a learned coefficient multiplied by an activation. The important operands are accumulator_parameter: a widened running sum that prevents intermediate overflow; weight: a learned coefficient multiplied by an activation; values: the value named 'values' in this block. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1549: `bias_gradient = self._bias_gradient(matrix_gradient)` This assigns 'self._bias_gradient(matrix_gradient)' to 'bias_gradient ' in host memory. Here bias_gradient means an additive learned offset applied after accumulation. The important operands are _bias_gradient: an additive learned offset applied after accumulation; matrix_gradient: the signed loss derivative used to update a learned parameter. Context: the function _projection_conv2d_backward, which handles projection conv2d backward.

Line 1550: `return input_gradient, ParameterGradient(` This statement continues through source line 1553; the explanation uses the complete expression rather than only the first visual fragment. This returns 'input_gradient, ParameterGradient(BlockGradient(weight_gradient, accumulator_parameter.weight.exponent), bias_gradient,)' to the caller of the function _projection_conv2d_backward, which handles projection conv2d backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1551: `BlockGradient(weight_gradient, accumulator_parameter.weight.exponent),` This is continuation line 2 of the 4-line logical statement that begins at source line 1550. Its exact fragment is 'BlockGradient(weight_gradient,

accumulator_parameter.weight.exponent),‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; weight_gradient: a learned coefficient multiplied by an activation; accumulator_parameter: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This returns ‘input_gradient, ParameterGradient(BlockGradient(weight_gradient, accumulator_parameter.weight.exponent), bias_gradient,)‘ to the caller of the function _projection_conv2d_backward, which handles projection conv2d backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1552: `bias_gradient,` This is continuation line 3 of the 4-line logical statement that begins at source line 1550. Its exact fragment is ‘bias_gradient,‘. The important operands are bias_gradient: an additive learned offset applied after accumulation. Read together, the complete operation is: This returns ‘input_gradient, ParameterGradient(BlockGradient(weight_gradient, accumulator_parameter.weight.exponent), bias_gradient,)‘ to the caller of the function _projection_conv2d_backward, which handles projection conv2d backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1553: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1550. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘input_gradient, ParameterGradient(BlockGradient(weight_gradient, accumulator_parameter.weight.exponent), bias_gradient,)‘ to the caller of the function _projection_conv2d_backward, which handles projection conv2d backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1555: `def _token_rows_gradient(` This starts function _token_rows_gradient. Its indented body performs token rows gradient as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1556: `self, gradient: BlockGradient, channels: int` Inside the function _token_rows_gradient, which handles token rows gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter; channels: the value named ’channels’ in this block.

Line 1557: `) -> BlockGradient:` Inside the function _token_rows_gradient, which handles token rows gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1558: `timesteps, batch, _, height, width = gradient.values.shape` This assigns ‘gradient.values.shape‘ to ‘timesteps, batch, _, height, width ‘ in host memory. Here timesteps means one discrete SNN update instant. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; shape: the value named ’shape’ in this block. Context: the function _token_rows_gradient, which handles token rows gradient.

Line 1559: `tokens = height * width` This assigns ‘height * width‘ to ‘tokens ‘ in host memory. Here tokens means the value named ’tokens’ in this block. The important operands are height: the value named ’height’ in this block; width: the value named ’width’ in this block. Context: the function _token_rows_gradient, which handles token rows gradient.

Line 1560: `values = gradient.values.reshape(` This statement continues through source line 1562; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘gradient.values.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are gradient: the signed loss derivative

used to update a learned parameter; values: the value named 'values' in this block; reshape: the value named 'reshape' in this block. Context: the function _token_rows_gradient, which handles token rows gradient.

Line 1561: `timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(` This is continuation line 2 of the 3-line logical statement that begins at source line 1560. Its exact fragment is 'timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape('. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; channels: the value named 'channels' in this block. Read together, the complete operation is: This assigns 'gradient.values.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block; reshape: the value named 'reshape' in this block. Context: the function _token_rows_gradient, which handles token rows gradient.

Line 1562: `timesteps * batch * tokens, channels)` This is continuation line 3 of the 3-line logical statement that begins at source line 1560. Its exact fragment is 'timesteps * batch * tokens, channels)'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This assigns 'gradient.values.reshape(timesteps, batch, channels, tokens).transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, channels)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block; reshape: the value named 'reshape' in this block. Context: the function _token_rows_gradient, which handles token rows gradient.

Line 1563: `return BlockGradient(values, gradient.exponent)` This returns 'BlockGradient(values, gradient.exponent)' to the caller of the function _token_rows_gradient, which handles token rows gradient, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block; gradient: the signed loss derivative used to update a learned parameter.

Line 1565: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class SpikingformerOnDeviceTrainer, which handles spikingformer on device trainer; it creates no FPGA circuit by itself.

Line 1566: `def _rows_to_channel_first(` This starts function _rows_to_channel_first. Its indented body performs rows to channel first as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1567: `gradient: BlockGradient,` Inside the function _rows_to_channel_first, which handles rows to channel first, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1568: `shape: tuple[int, ...],` Inside the function _rows_to_channel_first, which handles rows to channel first, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are shape: the value named 'shape' in this block; tuple: the value named 'tuple' in this block.

Line 1569: `) -> BlockGradient:` Inside the function _rows_to_channel_first, which handles rows to channel first, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1570: `timesteps, batch, channels, height, width = shape` This assigns 'shape' to 'timesteps, batch, channels, height, width ' in host memory. Here timesteps means one discrete SNN update instant. The important operands are shape: the value named 'shape' in this block. Context: the function _rows_to_channel_first, which handles rows to channel first.

Line 1571: `tokens = height * width` This assigns ‘height * width‘ to ‘tokens ‘ in host memory. Here tokens means the value named ’tokens’ in this block. The important operands are height: the value named ’height’ in this block; width: the value named ’width’ in this block. Context: the function _rows_to_channel_first, which handles rows to channel first.

Line 1572: `values = gradient.values.reshape(` This statement continues through source line 1573; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘gradient.values.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(shape)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _rows_to_channel_first, which handles rows to channel first.

Line 1573: `timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(shape)` This is continuation line 2 of the 2-line logical statement that begins at source line 1572. Its exact fragment is ‘timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(shape)‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This assigns ‘gradient.values.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2).reshape(shape)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; reshape: the value named ’reshape’ in this block. Context: the function _rows_to_channel_first, which handles rows to channel first.

Line 1574: `return BlockGradient(values, gradient.exponent)` This returns ‘BlockGradient(values, gradient.exponent)‘ to the caller of the function _rows_to_channel_first, which handles rows to channel first, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; gradient: the signed loss derivative used to update a learned parameter.

Line 1576: `def _stack_disjoint_gradients(` This starts function _stack_disjoint_gradients. Its indented body performs stack disjoint gradients as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1577: `self,` Inside the function _stack_disjoint_gradients, which handles stack disjoint gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1578: `gradients: Sequence[BlockGradient],` Inside the function _stack_disjoint_gradients, which handles stack disjoint gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are gradients: the signed loss derivative used to update a learned parameter; Sequence: the value named ’sequence’ in this block; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1579: `shape: tuple[int, ...],` Inside the function _stack_disjoint_gradients, which handles stack disjoint gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are shape: the value named ’shape’ in this block; tuple: the value named ’tuple’ in this block.

Line 1580: `) -> BlockGradient:` Inside the function _stack_disjoint_gradients, which handles stack disjoint gradients, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1581: `if not gradients:` This executes the following indented host-code branch only when ‘not gradients‘ is true. The important operands are not: the value named ’not’ in this block; gradients: the signed loss derivative used to update a learned parameter. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1582: `raise ValueError("at least one gradient is required for stacking")` This rejects an invalid state by raising ‘ValueError("at least one gradient is required for stacking")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1583: `exponent = max(gradient.exponent for gradient in gradients)` This assigns ‘max(gradient.exponent for gradient in gradients)‘ to ‘exponent ‘ in host memory. Here exponent means the value named ’exponent’ in this block. The important operands are max: the value named ’max’ in this block; gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1584: `aligned: list[np.ndarray] = []` This assigns ‘[]‘ to ‘aligned‘ in host memory. Here aligned means the value named ’aligned’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1585: `for gradient in gradients:` This iterates gradient over ‘gradients‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1586: `values = gradient.values` This assigns ‘gradient.values‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1587: `if gradient.exponent != exponent:` This executes the following indented host-code branch only when ‘gradient.exponent != exponent‘ is true. The important operands are gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1588: `values = execute_power_of_two_scale(` This statement continues through source line 1593; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_power_of_two_scale(self.driver, values, gradient.exponent - exponent, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1589: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1588. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(self.driver, values, gradient.exponent - exponent, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1590: `values,` This is continuation line 3 of the 6-line logical statement that begins at source line 1588. Its exact fragment is ‘values,‘. The important operands are values: the value named ’values’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(self.driver, values, gradient.exponent - exponent, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1591: `gradient.exponent - exponent,` This is continuation line 4 of the 6-line logical statement that begins at source line 1588. Its exact fragment is ‘gradient.exponent - exponent,‘. The important operands are gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block. Read together,

the complete operation is: This assigns ‘execute_power_of_two_scale(self.driver, values, gradient.exponent - exponent, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1592: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1588. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(self.driver, values, gradient.exponent - exponent, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1593: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1588. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_power_of_two_scale(self.driver, values, gradient.exponent - exponent, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named ’driver’ in this block; values: the value named ’values’ in this block. Context: the function _stack_disjoint_gradients, which handles stack disjoint gradients.

Line 1594: `aligned.append(values)` This calls ‘aligned.append‘ with ‘values‘ inside the function _stack_disjoint_gradients, which handles stack disjoint gradients. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1595: `return BlockGradient(np.stack(aligned).reshape(shape), exponent)` This returns ‘BlockGradient(np.stack(aligned).reshape(shape), exponent)‘ to the caller of the function _stack_disjoint_gradients, which handles stack disjoint gradients, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; stack: the value named ’stack’ in this block; aligned: the value named ’aligned’ in this block.

Line 1597: `def _gate_saturating_add_gradient(` This starts function _gate_saturating_add_gradient. Its indented body performs gate saturating add gradient as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1598: `self,` Inside the function _gate_saturating_add_gradient, which handles gate saturating add gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1599: `output_gradient: BlockGradient,` Inside the function _gate_saturating_add_gradient, which handles gate saturating add gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1600: `left: np.ndarray,` Inside the function _gate_saturating_add_gradient, which handles gate saturating add gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are left: the value named ’left’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 1601: `right: np.ndarray,` Inside the function _gate_saturating_add_gradient, which handles gate saturating add gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are right: the value named 'right' in this block; ndarray: the value named 'ndarray' in this block.

Line 1602: `) -> BlockGradient:` Inside the function _gate_saturating_add_gradient, which handles gate saturating add gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1603: `total = np.asarray(left, dtype=np.int64) + np.asarray(right, dtype=np.int64)` This assigns 'np.asarray(left, dtype=np.int64) + np.asarray(right, dtype=np.int64)' to 'total ' in host memory. Here total means the value named 'total' in this block. The important operands are asarray: the value named 'asarray' in this block; left: the value named 'left' in this block; dtype: the value named 'dtype' in this block. Context: the function _gate_saturating_add_gradient, which handles gate saturating add gradient.

Line 1604: `mask = ((total >= -(1 << 31)) & (total <= (1 << 31) - 1)).astype(np.int64)` Inside the function _gate_saturating_add_gradient, which handles gate saturating add gradient, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are mask: the value named 'mask' in this block; total: the value named 'total' in this block; astype: the value named 'astype' in this block.

Line 1605: `gated = execute_vector_tensor(` This statement continues through source line 1611; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, mask, output_gradient.values, timeout_seconds=self.layer_timeout_seconds,)' to 'gated ' in host memory. Here gated means the value named 'gated' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function _gate_saturating_add_gradient, which handles gate saturating add gradient.

Line 1606: `self.driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1605. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, mask, output_gradient.values, timeout_seconds=self.layer_timeout_seconds,)' to 'gated ' in host memory. Here gated means the value named 'gated' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function _gate_saturating_add_gradient, which handles gate saturating add gradient.

Line 1607: `VectorOperation.SELECT_NONZERO,` This is continuation line 3 of the 7-line logical statement that begins at source line 1605. Its exact fragment is 'VectorOperation.SELECT_NONZERO,'. The important operands are VectorOperation: the value named 'vector operation' in this block; SELECT_NONZERO: the value named 'select nonzero' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, mask, output_gradient.values, timeout_seconds=self.layer_timeout_seconds,)' to 'gated ' in host memory. Here gated means the value named 'gated' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function _gate_saturating_add_gradient, which handles gate saturating add gradient.

Line 1608: `mask,` This is continuation line 4 of the 7-line logical statement that begins at source line 1605. Its exact fragment is 'mask,'. The important operands are mask: the value named 'mask' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, mask, output_gradient.values, timeout_seconds=self.layer_timeout_seconds,)' to 'gated ' in host memory. Here gated means the value named 'gated' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector

operation' in this block. Context: the function _gate_saturating_add_gradient, which handles gate saturating add gradient.

Line 1609: `output_gradient.values,` This is continuation line 5 of the 7-line logical statement that begins at source line 1605. Its exact fragment is 'output_gradient.values,'. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, mask, output_gradient.values, timeout_seconds=self.layer_timeout_seconds,)' to 'gated ' in host memory. Here gated means the value named 'gated' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function _gate_saturating_add_gradient, which handles gate saturating add gradient.

Line 1610: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1605. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, mask, output_gradient.values, timeout_seconds=self.laye)' to 'gated ' in host memory. Here gated means the value named 'gated' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function _gate_saturating_add_gradient, which handles gate saturating add gradient.

Line 1611: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1605. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, mask, output_gradient.values, timeout_seconds=self.layer_timeout_seconds,)' to 'gated ' in host memory. Here gated means the value named 'gated' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function _gate_saturating_add_gradient, which handles gate saturating add gradient.

Line 1612: `return BlockGradient(gated, output_gradient.exponent)` This returns 'BlockGradient(gated, output_gradient.exponent)' to the caller of the function _gate_saturating_add_gradient, which handles gate saturating add gradient, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gated: the value named 'gated' in this block; output_gradient: the signed loss derivative used to update a learned parameter.

Line 1614: `def _mlp_backward(` This starts function _mlp_backward. Its indented body performs mlp backward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1615: `self,` Inside the function _mlp_backward, which handles mlp backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1616: `cache: MlpCache,` Inside the function _mlp_backward, which handles mlp backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are cache: the value named 'cache' in this block; MlpCache: the value named 'mlp cache' in this block.

Line 1617: `output_gradient: BlockGradient,` Inside the function _mlp_backward, which handles mlp backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1618: `) -> tuple[BlockGradient, dict[str, ParameterGradient]]:` Inside the function _mlp_backward, which handles mlp backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named 'tuple' in this block; BlockGradient: the signed loss derivative used to update a learned parameter; dict: the value named 'dict' in this block.

Line 1619: `timesteps, batch, channels, height, width = cache.input_shape` This assigns 'cache.input_shape' to 'timesteps, batch, channels, height, width ' in host memory. Here timesteps means one discrete SNN update instant. The important operands are cache: the value named 'cache' in this block; input_shape: the value named 'input shape' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1620: `tokens = height * width` This assigns 'height * width' to 'tokens ' in host memory. Here tokens means the value named 'tokens' in this block. The important operands are height: the value named 'height' in this block; width: the value named 'width' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1621: `output_rows = self._token_rows_gradient(output_gradient, channels)` This assigns 'self._token_rows_grad channels)' to 'output_rows ' in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are _token_rows_gradient: the signed loss derivative used to update a learned parameter; output_gradient: the signed loss derivative used to update a learned parameter; channels: the value named 'channels' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1622: `second_name = f"block.{cache.block_index}.mlp.fc2"` This assigns 'f"block.{cache.block_index}.mlp.fc2"' to 'second_name ' in host memory. Here second_name means the value named 'second name' in this block. The important operands are f: the value named 'f' in this block; block: the value named 'block' in this block; cache: the value named 'cache' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1623: `hidden_spike_rows, second_parameter = self._projection_matrix_backward(` This statement continues through source line 1624; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._projection_matrix_backward(second_name, cache.second_projection_input, output_rows)' to 'hidden_spike_rows, second_parameter ' in host memory. Here hidden_spike_rows means a binary event produced when a neuron crosses its threshold. The important operands are _projection_matrix_backward: the value named 'projection matrix backward' in this block; second_name: the value named 'second name' in this block; cache: the value named 'cache' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1624: `second_name, cache.second_projection_input, output_rows)` This is continuation line 2 of the 2-line logical statement that begins at source line 1623. Its exact fragment is 'second_name, cache.second_projection_input, output_rows)'. The important operands are second_name: the value named 'second name' in this block; cache: the value named 'cache' in this block; second_projection_input: the value named 'second projection input' in this block. Read together, the complete operation is: This assigns 'self._projection_matrix_backward(second_name, cache.second_projection_input, output_rows)' to 'hidden_spike_rows, second_parameter ' in host memory. Here hidden_spike_rows means a binary event produced when a neuron crosses its threshold. The important operands are _projection_matrix_backward: the value named 'projection matrix backward' in this block; second_name: the value named 'second name' in this block; cache: the value named 'cache' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1625: `hidden_channels = cache.hidden_lif.spikes.shape[2]` This assigns 'cache.hidden_lif.spikes.shape[2]' to 'hidden_channels ' in host memory. Here hidden_channels means the value named 'hidden channels' in this block. The important operands are cache: the value named 'cache' in this block; hidden_lif: the value named 'hidden lif' in this block; spikes: a binary event produced when a neuron crosses its threshold. Context: the function _mlp_backward, which handles mlp backward.

Line 1626: `hidden_spike_gradient = hidden_spike_rows.reshape(` This statement continues through source line 1627; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'hidden_spike_rows.reshape(timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)' to 'hidden_spike_gradient ' in host memory. Here hidden_spike_gradient means a binary event produced when a neuron crosses its threshold. The

important operands are hidden_spike_rows: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _mlp_backward, which handles mlp backward.

Line 1627: `timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)` This is continuation line 2 of the 2-line logical statement that begins at source line 1626. Its exact fragment is 'timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This assigns 'hidden_spike_rows.reshape(timesteps, batch, tokens, hidden_channels).transpose(0, 1, 3, 2)' to 'hidden_spike_gradient ' in host memory. Here hidden_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are hidden_spike_rows: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block; timesteps: one discrete SNN update instant. Context: the function _mlp_backward, which handles mlp backward.

Line 1628: `hidden_gradient = self._lif_backward(` This statement continues through source line 1631; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._lif_backward(cache.hidden_lif, hidden_spike_gradient,)' to 'hidden_gradient ' in host memory. Here hidden_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; hidden_lif: the value named 'hidden lif' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1629: `cache.hidden_lif,` This is continuation line 2 of the 4-line logical statement that begins at source line 1628. Its exact fragment is 'cache.hidden_lif,'. The important operands are cache: the value named 'cache' in this block; hidden_lif: the value named 'hidden lif' in this block. Read together, the complete operation is: This assigns 'self._lif_backward(cache.hidden_lif, hidden_spike_gradient,)' to 'hidden_gradient ' in host memory. Here hidden_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; hidden_lif: the value named 'hidden lif' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1630: `hidden_spike_gradient,` This is continuation line 3 of the 4-line logical statement that begins at source line 1628. Its exact fragment is 'hidden_spike_gradient,'. The important operands are hidden_spike_gradient: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns 'self._lif_backward(cache.hidden_lif, hidden_spike_gradient,)' to 'hidden_gradient ' in host memory. Here hidden_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; hidden_lif: the value named 'hidden lif' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1631: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1628. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self._lif_backward(cache.hidden_lif, hidden_spike_gradient,)' to 'hidden_gradient ' in host memory. Here hidden_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; hidden_lif: the value named 'hidden lif' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1632: `hidden_rows = BlockGradient(` This statement continues through source line 1636; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'BlockGradient(hidden_gradient.values.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, hidden_channels), hidden_gradient.exponent,)' to 'hidden_rows ' in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; hidden_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1633: `hidden_gradient.values.transpose(0, 1, 3, 2).reshape(` This is continuation line 2 of the 5-line logical statement that begins at source line 1632. Its exact fragment is 'hidden_gradient.values.transpose(0, 1, 3, 2).reshape('. The important operands are hidden_gradient: the signed loss derivative used to update a learned parameter;

values: the value named 'values' in this block; transpose: the value named 'transpose' in this block. Read together, the complete operation is: This assigns 'BlockGradient(hidden_gradient.values.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, hidden_channels), hidden_gradient.exponent,)' to 'hidden_rows ' in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; hidden_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1634: `timesteps * batch * tokens, hidden_channels),` This is continuation line 3 of the 5-line logical statement that begins at source line 1632. Its exact fragment is 'timesteps * batch * tokens, hidden_channels),'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This assigns 'BlockGradient(hidden_gradient.values.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, hidden_channels), hidden_gradient.exponent,)' to 'hidden_rows ' in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; hidden_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1635: `hidden_gradient.exponent,` This is continuation line 4 of the 5-line logical statement that begins at source line 1632. Its exact fragment is 'hidden_gradient.exponent,'. The important operands are hidden_gradient: the signed loss derivative used to update a learned parameter; exponent: the value named 'exponent' in this block. Read together, the complete operation is: This assigns 'BlockGradient(hidden_gradient.values.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, hidden_channels), hidden_gradient.exponent,)' to 'hidden_rows ' in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; hidden_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1636: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1632. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'BlockGradient(hidden_gradient.values.transpose(0, 1, 3, 2).reshape(timesteps * batch * tokens, hidden_channels), hidden_gradient.exponent,)' to 'hidden_rows ' in host memory. Here hidden_rows means one output-row position processed by the tensor tile. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; hidden_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1637: `first_name = f"block.{cache.block_index}.mlp.fc1"` This assigns 'f"block.{cache.block_index}.mlp.fc1"' to 'first_name ' in host memory. Here first_name means the active-low synchronous reset. The important operands are f: the value named 'f' in this block; block: the value named 'block' in this block; cache: the value named 'cache' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1638: `input_spike_rows, first_parameter = self._projection_matrix_backward(` This statement continues through source line 1639; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._projection_matrix_backward(first_name, cache.first_projection_input, hidden_rows)' to 'input_spike_rows, first_parameter ' in host memory. Here input_spike_rows means a binary event produced when a neuron crosses its threshold. The important operands are _projection_matrix_backward: the value named 'projection matrix backward' in this block; first_name: the active-low synchronous reset; cache: the value named 'cache' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1639: `first_name, cache.first_projection_input, hidden_rows)` This is continuation line 2 of the 2-line logical statement that begins at source line 1638. Its exact fragment is 'first_name, cache.first_projection_input, hidden_rows)'. The important operands are first_name: the active-low synchronous reset; cache: the value named 'cache' in this block; first_projection_input: the value named 'first projection input' in this block. Read together, the complete operation is: This assigns 'self._projection_matrix_backward(first_name, cache.first_projection_input, hidden_rows)' to 'input_spike_rows, first_parameter ' in host memory. Here input_spike_rows means a binary event produced when

a neuron crosses its threshold. The important operands are _projection_matrix_backward: the value named 'projection matrix backward' in this block; first_name: the active-low synchronous reset; cache: the value named 'cache' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1640: `input_spike_gradient = self._rows_to_channel_first(` This statement continues through source line 1641; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._rows_to_channel_first(input_spike_rows, cache.input_shape)' to 'input_spike_gradient ' in host memory. Here input_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are _rows_to_channel_first: one output-row position processed by the tensor tile; input_spike_rows: a binary event produced when a neuron crosses its threshold; cache: the value named 'cache' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1641: `input_spike_rows, cache.input_shape)` This is continuation line 2 of the 2-line logical statement that begins at source line 1640. Its exact fragment is 'input_spike_rows, cache.input_shape)'. The important operands are input_spike_rows: a binary event produced when a neuron crosses its threshold; cache: the value named 'cache' in this block; input_shape: the value named 'input shape' in this block. Read together, the complete operation is: This assigns 'self._rows_to_channel_first(input_spike_rows, cache.input_shape)' to 'input_spike_gradient ' in host memory. Here input_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are _rows_to_channel_first: one output-row position processed by the tensor tile; input_spike_rows: a binary event produced when a neuron crosses its threshold; cache: the value named 'cache' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1642: `input_gradient = self._lif_backward(cache.input_lif, input_spike_gradient)` This assigns 'self._lif_back input_spike_gradient)' to 'input_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; input_lif: the value named 'input lif' in this block. Context: the function _mlp_backward, which handles mlp backward.

Line 1643: `return input_gradient, {` This statement continues through source line 1646; the explanation uses the complete expression rather than only the first visual fragment. This returns 'input_gradient, { first_name: first_parameter, second_name: second_parameter, }' to the caller of the function _mlp_backward, which handles mlp backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; first_name: the active-low synchronous reset; first_parameter: the value named 'first parameter' in this block.

Line 1644: `first_name: first_parameter,` This is continuation line 2 of the 4-line logical statement that begins at source line 1643. Its exact fragment is 'first_name: first_parameter,'. The important operands are first_name: the active-low synchronous reset; first_parameter: the value named 'first parameter' in this block. Read together, the complete operation is: This returns 'input_gradient, { first_name: first_parameter, second_name: second_parameter, }' to the caller of the function _mlp_backward, which handles mlp backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; first_name: the active-low synchronous reset; first_parameter: the value named 'first parameter' in this block.

Line 1645: `second_name: second_parameter,` This is continuation line 3 of the 4-line logical statement that begins at source line 1643. Its exact fragment is 'second_name: second_parameter,'. The important operands are second_name: the value named 'second name' in this block; second_parameter: the value named 'second parameter' in this block. Read together, the complete operation is: This returns 'input_gradient, { first_name: first_parameter, second_name: second_parameter, }' to the caller of the function _mlp_backward, which handles mlp backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; first_name: the active-low synchronous reset; first_parameter: the value named 'first parameter' in this block.

Line 1646: `}` This is continuation line 4 of the 4-line logical statement that begins at source line 1643. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'input_gradient, { first_name: first_parameter, second_name: second_parameter, }' to the caller of the function _mlp_backward, which handles mlp backward, ending this invocation. The important

operands are input_gradient: the signed loss derivative used to update a learned parameter; first_name: the active-low synchronous reset; first_parameter: the value named 'first parameter' in this block.

Line 1648: `def _attention_backward(` This starts function _attention_backward. Its indented body performs attention backward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1649: `self,` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1650: `cache: AttentionCache,` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are cache: the value named 'cache' in this block; AttentionCache: the value named 'attention cache' in this block.

Line 1651: `output_gradient: BlockGradient,` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1652: `) -> tuple[BlockGradient, dict[str, ParameterGradient]]:` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named 'tuple' in this block; BlockGradient: the signed loss derivative used to update a learned parameter; dict: the value named 'dict' in this block.

Line 1653: `timesteps, batch, channels, height, width = cache.input_shape` This assigns 'cache.input_shape' to 'timesteps, batch, channels, height, width ' in host memory. Here timesteps means one discrete SNN update instant. The important operands are cache: the value named 'cache' in this block; input_shape: the value named 'input shape' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1654: `tokens = height * width` This assigns 'height * width' to 'tokens ' in host memory. Here tokens means the value named 'tokens' in this block. The important operands are height: the value named 'height' in this block; width: the value named 'width' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1655: `head_dim = channels // self.attention_heads` This assigns 'channels // self.attention_heads' to 'head_dim ' in host memory. Here head_dim means the value named 'head dim' in this block. The important operands are channels: the value named 'channels' in this block; attention_heads: the value named 'attention heads' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1656: `prefix = f"block.{cache.block_index}.attn"` This assigns 'f"block.{cache.block_index}.attn"' to 'prefix ' in host memory. Here prefix means the value named 'prefix' in this block. The important operands are f: the value named 'f' in this block; block: the value named 'block' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1657: `output_rows = self._token_rows_gradient(output_gradient, channels)` This assigns 'self._token_rows_grad channels)' to 'output_rows ' in host memory. Here output_rows means one output-row position processed by the tensor tile. The important operands are _token_rows_gradient: the signed loss derivative used to update a learned parameter; output_gradient: the signed loss derivative used to update a learned parameter; channels: the value named 'channels' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1658: `attention_spike_rows, projection_parameter = self._projection_matrix_backward(` This statement continues through source line 1659; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._projection_matrix_backward(f"{prefix}.proj", cache.projection_input, output_rows)‘ to ‘attention_spike_rows, projection_parameter ‘ in host memory. Here attention_spike_rows means a binary event produced when a neuron crosses its threshold. The important operands are _projection_matrix_backward: the value named ’projection matrix backward’ in this block; f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1659: `f"{prefix}.proj", cache.projection_input, output_rows)` This is continuation line 2 of the 2-line logical statement that begins at source line 1658. Its exact fragment is ‘f"{prefix}.proj", cache.projection_input, output_rows)‘. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; proj: the value named ’proj’ in this block. Read together, the complete operation is: This assigns ‘self._projection_matrix_backward(f"{prefix}.proj", cache.projection_input, output_rows)‘ to ‘attention_spike_rows, projection_parameter ‘ in host memory. Here attention_spike_rows means a binary event produced when a neuron crosses its threshold. The important operands are _projection_matrix_backward: the value named ’projection matrix backward’ in this block; f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1660: `attention_spike_gradient = BlockGradient(` This statement continues through source line 1664; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘BlockGradient(attention_spike_rows.values.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2), attention_spike_rows.exponent,)‘ to ‘attention_spike_gradient ‘ in host memory. Here attention_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; attention_spike_rows: a binary event produced when a neuron crosses its threshold; values: the value named ’values’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1661: `attention_spike_rows.values.reshape(` This is continuation line 2 of the 5-line logical statement that begins at source line 1660. Its exact fragment is ‘attention_spike_rows.values.reshape(‘. The important operands are attention_spike_rows: a binary event produced when a neuron crosses its threshold; values: the value named ’values’ in this block; reshape: the value named ’reshape’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(attention_spike_rows.values.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2), attention_spike_rows.exponent,)‘ to ‘attention_spike_gradient ‘ in host memory. Here attention_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; attention_spike_rows: a binary event produced when a neuron crosses its threshold; values: the value named ’values’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1662: `timesteps, batch, tokens, channels).transpose(0, 1, 3, 2),` This is continuation line 3 of the 5-line logical statement that begins at source line 1660. Its exact fragment is ‘timesteps, batch, tokens, channels).transpose(0, 1, 3, 2),‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; tokens: the value named ’tokens’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(attention_spike_rows.values.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2), attention_spike_rows.exponent,)‘ to ‘attention_spike_gradient ‘ in host memory. Here attention_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; attention_spike_rows: a binary event produced when a neuron crosses its threshold; values: the value named ’values’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1663: `attention_spike_rows.exponent,` This is continuation line 4 of the 5-line logical statement that begins at source line 1660. Its exact fragment is ‘attention_spike_rows.exponent,‘. The important operands are attention_spike_rows: a binary event produced when a neuron crosses its threshold; exponent: the value named ’exponent’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(attention_spike_rows.values.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2), attention_spike_rows.exponent,)‘ to ‘attention_spike_gradient ‘ in host memory. Here attention_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; attention_spike_rows: a binary event produced when a neuron crosses its threshold; values: the value named ’values’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1664: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1660. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'BlockGradient(attention_spike_rows.values.reshape(timesteps, batch, tokens, channels).transpose(0, 1, 3, 2), attention_spike_rows.exponent,)' to 'attention_spike_gradient ' in host memory. Here attention_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; attention_spike_rows: a binary event produced when a neuron crosses its threshold; values: the value named 'values' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1665: `merged_gradient = self._lif_backward(` This statement continues through source line 1666; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._lif_backward(cache.attention_lif, attention_spike_gradient)' to 'merged_gradient ' in host memory. Here merged_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; attention_lif: the value named 'attention lif' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1666: `cache.attention_lif, attention_spike_gradient)` This is continuation line 2 of the 2-line logical statement that begins at source line 1665. Its exact fragment is 'cache.attention_lif, attention_spike_gradient)'. The important operands are cache: the value named 'cache' in this block; attention_lif: the value named 'attention lif' in this block; attention_spike_gradient: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns 'self._lif_backward(cache.attention_lif, attention_spike_gradient)' to 'merged_gradient ' in host memory. Here merged_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; attention_lif: the value named 'attention lif' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1667: `attention_value_gradient = BlockGradient(` This statement continues through source line 1672; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'BlockGradient(merged_gradient.values.reshape(timesteps, batch, self.attention_heads, head_dim, tokens).transpose(0, 1, 2, 4, 3), merged_gradient.exponent + self.attention_scale_shift,)' to 'attention_value_gradient ' in host memory. Here attention_value_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; merged_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1668: `merged_gradient.values.reshape(` This is continuation line 2 of the 6-line logical statement that begins at source line 1667. Its exact fragment is 'merged_gradient.values.reshape('. The important operands are merged_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block; reshape: the value named 'reshape' in this block. Read together, the complete operation is: This assigns 'BlockGradient(merged_gradient.values.reshape(timesteps, batch, self.attention_heads, head_dim, tokens).transpose(0, 1, 2, 4, 3), merged_gradient.exponent + self.attention_scale_shift,)' to 'attention_value_gradient ' in host memory. Here attention_value_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; merged_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1669: `timesteps, batch, self.attention_heads, head_dim, tokens` This is continuation line 3 of the 6-line logical statement that begins at source line 1667. Its exact fragment is 'timesteps, batch, self.attention_heads, head_dim, tokens'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; attention_heads: the value named 'attention heads' in this block. Read together, the complete operation is: This assigns 'BlockGradient(merged_gradient.values.reshape(timesteps, batch, self.attention_heads, head_dim, tokens).transpose(0, 1, 2, 4, 3), merged_gradient.exponent + self.attention_scale_shift,)' to 'attention_value_gradient ' in host memory. Here attention_value_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; merged_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1670: `).transpose(0, 1, 2, 4, 3),` This is continuation line 4 of the 6-line logical statement that begins at source line 1667. Its exact fragment is ‘).transpose(0, 1, 2, 4, 3),‘. The important operands are transpose: the value named ’transpose’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(merged_gradient.values.reshape(timesteps, batch, self.attention_heads, head_dim, tokens).transpose(0, 1, 2, 4, 3), merged_gradient.exponent + self.attention_scale_shift,)‘ to ‘attention_value_gradient ‘ in host memory. Here attention_value_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; merged_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1671: `merged_gradient.exponent + self.attention_scale_shift,` This is continuation line 5 of the 6-line logical statement that begins at source line 1667. Its exact fragment is ‘merged_gradient.exponent + self.attention_scale_shift,‘. The important operands are merged_gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block; attention_scale_shift: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns ‘BlockGradient(merged_gradient.values.reshape(timesteps, batch, self.attention_heads, head_dim, tokens).transpose(0, 1, 2, 4, 3), merged_gradient.exponent + self.attention_scale)‘ to ‘attention_value_gradient ‘ in host memory. Here attention_value_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; merged_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1672: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1667. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘BlockGradient(merged_gradient.values.reshape(timesteps, batch, self.attention_heads, head_dim, tokens).transpose(0, 1, 2, 4, 3), merged_gradient.exponent + self.attention_scale_shift,)‘ to ‘attention_value_gradient ‘ in host memory. Here attention_value_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; merged_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1674: `score_parts: list[BlockGradient] = []` This assigns ‘[]‘ to ‘score_parts‘ in host memory. Here score_parts means the value named ’score parts’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _attention_backward, which handles attention backward.

Line 1675: `value_parts: list[BlockGradient] = []` This assigns ‘[]‘ to ‘value_parts‘ in host memory. Here value_parts means the value named ’value parts’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _attention_backward, which handles attention backward.

Line 1676: `for timestep in range(timesteps):` This iterates timestep over ‘range(timesteps)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention_backward, which handles attention backward.

Line 1677: `for batch_index in range(batch):` This iterates batch_index over ‘range(batch)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention_backward, which handles attention backward.

Line 1678: `for head in range(self.attention_heads):` This iterates head over ‘range(self.attention_heads)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention_backward, which handles attention backward.

Line 1679: `score_gradient, value_gradient = execute_linear_backward_block(` This statement continues through source line 1689; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.exp), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘score_gradient, value_gradient ‘ in host memory. Here score_gradient

means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1680: `self.driver,` This is continuation line 2 of the 11-line logical statement that begins at source line 1679. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1681: `cache.scores[timestep, batch_index, head],` This is continuation line 3 of the 11-line logical statement that begins at source line 1679. Its exact fragment is 'cache.scores[timestep, batch_index, head],'. The important operands are cache: the value named 'cache' in this block; scores: the value named 'scores' in this block; timestep: one discrete SNN update instant. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.exponent,), timeout_seconds=self.layer)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1682: `cache.value_heads[timestep, batch_index, head],` This is continuation line 4 of the 11-line logical statement that begins at source line 1679. Its exact fragment is 'cache.value_heads[timestep, batch_index, head],'. The important operands are cache: the value named 'cache' in this block; value_heads: the value named 'value heads' in this block; timestep: one discrete SNN update instant. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.expo), timeout_seconds=self.layer_timeout_seconds,)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1683: `BlockGradient(` This is continuation line 5 of the 11-line logical statement that begins at source line 1679. Its exact fragment is 'BlockGradient('. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.exponent,), timeout_seconds=self.layer)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1684: `attention_value_gradient.values[` This is continuation line 6 of the 11-line logical statement that begins at source line 1679. Its exact fragment is 'attention_value_gradient.values['. The important operands are attention_value_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradi timestep, batch_index, head], attention_value_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1685: `timestep, batch_index, head],` This is continuation line 7 of the 11-line logical statement that begins at source line 1679. Its exact fragment is 'timestep, batch_index, head],'. The important operands are timestep: one discrete SNN update instant; batch_index: an integer selecting one tensor element, lane, bank, or state entry; head: the value named 'head' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.exponent,), timeout_seconds=self.layer)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1686: `attention_value_gradient.exponent,` This is continuation line 8 of the 11-line logical statement that begins at source line 1679. Its exact fragment is 'attention_value_gradient.exponent,'. The important operands are attention_value_gradient: the signed loss derivative used to update a learned parameter; exponent: the value named 'exponent' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.exponent,), timeout_seconds=self.layer)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1687: `),` This is continuation line 9 of the 11-line logical statement that begins at source line 1679. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1688: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 10 of the 11-line logical statement that begins at source line 1679. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1689: `)` This is continuation line 11 of the 11-line logical statement that begins at source line 1679. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.scores[timestep, batch_index, head], cache.value_heads[timestep, batch_index, head], BlockGradient(attention_value_gradient.values[timestep, batch_index, head], attention_value_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)' to 'score_gradient, value_gradient ' in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1690: `score_parts.append(score_gradient)` This calls 'score_parts.append' with 'score_gradient' inside the function _attention_backward, which handles attention backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1691: `value_parts.append(value_gradient)` This calls ‘value_parts.append‘ with ‘value_gradient‘ inside the function _attention_backward, which handles attention backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1692: `score_gradient = self._stack_disjoint_gradients(` This statement continues through source line 1695; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._stack_disjoint_grad score_parts, (timesteps, batch, self.attention_heads, tokens, tokens),)‘ to ‘score_gradient ‘ in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; score_parts: the value named ’score parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1693: `score_parts,` This is continuation line 2 of the 4-line logical statement that begins at source line 1692. Its exact fragment is ‘score_parts,‘. The important operands are score_parts: the value named ’score parts’ in this block. Read together, the complete operation is: This assigns ‘self._stack_disjoint_gradients(score_parts, (timesteps, batch, self.attention_heads, tokens, tokens),)‘ to ‘score_gradient ‘ in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; score_parts: the value named ’score parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1694: `(timesteps, batch, self.attention_heads, tokens, tokens),` This is continuation line 3 of the 4-line logical statement that begins at source line 1692. Its exact fragment is ‘(timesteps, batch, self.attention_heads, tokens, tokens),‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; attention_heads: the value named ’attention heads’ in this block. Read together, the complete operation is: This assigns ‘self._stack_disjoint_gradients(score_parts, (timesteps, batch, self.attention_heads, tokens, tokens),)‘ to ‘score_gradient ‘ in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; score_parts: the value named ’score parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1695: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1692. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self._stack_disjoint_gradients(score_parts, (timesteps, batch, self.attention_heads, tokens, tokens),)‘ to ‘score_gradient ‘ in host memory. Here score_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; score_parts: the value named ’score parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1696: `value_head_gradient = self._stack_disjoint_gradients(` This statement continues through source line 1699; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._stack_disjoint_gradients(value_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)‘ to ‘value_head_gradie ‘ in host memory. Here value_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; value_parts: the value named ’value parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1697: `value_parts,` This is continuation line 2 of the 4-line logical statement that begins at source line 1696. Its exact fragment is ‘value_parts,‘. The important operands are value_parts: the value named ’value parts’ in this block. Read together, the complete operation is: This assigns ‘self._stack_disjoint_gradients(value_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)‘ to ‘value_head_gradient ‘ in host memory. Here value_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; value_parts: the value named ’value parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1698: `(timesteps, batch, self.attention_heads, tokens, head_dim),` This is continuation line 3 of the 4-line logical statement that begins at source line 1696. Its exact fragment is '(timesteps, batch, self.attention_heads, tokens, head_dim),'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; attention_heads: the value named 'attention heads' in this block. Read together, the complete operation is: This assigns 'self._stack_disjoint_gradients(value_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)' to 'value_head_gradient ' in host memory. Here value_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; value_parts: the value named 'value parts' in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1699: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1696. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self._stack_disjoint_gradients(value_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)' to 'value_head_gradient ' in host memory. Here value_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; value_parts: the value named 'value parts' in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1701: `query_parts: list[BlockGradient] = []` This assigns '[]' to 'query_parts' in host memory. Here query_parts means the value named 'query parts' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _attention_backward, which handles attention backward.

Line 1702: `key_parts: list[BlockGradient] = []` This assigns '[]' to 'key_parts' in host memory. Here key_parts means the value named 'key parts' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _attention_backward, which handles attention backward.

Line 1703: `part_index = 0` This assigns '0' to 'part_index ' in host memory. Here part_index means an integer selecting one tensor element, lane, bank, or state entry. The expression is a literal or structural operator and has no hidden software state. Context: the function _attention_backward, which handles attention backward.

Line 1704: `for timestep in range(timesteps):` This iterates timestep over 'range(timesteps)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention_backward, which handles attention backward.

Line 1705: `for batch_index in range(batch):` This iterates batch_index over 'range(batch)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention_backward, which handles attention backward.

Line 1706: `for head in range(self.attention_heads):` This iterates head over 'range(self.attention_heads)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _attention_backward, which handles attention backward.

Line 1707: `query_gradient, transposed_key_gradient = execute_linear_backward_block(` This statement continues through source line 1716; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index, head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)' to 'query_gradient, transposed_key_gradient ' in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1708: `self.driver,` This is continuation line 2 of the 10-line logical statement that begins at source line 1707. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index,

head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘query_gradient, transposed_key_gradient ‘ in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; cache: the value named ’cache’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1709: `cache.query_heads[timestep, batch_index, head],` This is continuation line 3 of the 10-line logical statement that begins at source line 1707. Its exact fragment is ‘cache.query_heads[timestep, batch_index, head],‘. The important operands are cache: the value named ’cache’ in this block; query_heads: the value named ’query heads’ in this block; timestep: one discrete SNN update instant. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index, head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘query_gradient, transposed_key_gradient ‘ in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; cache: the value named ’cache’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1710: `cache.key_heads[timestep, batch_index, head].T,` This is continuation line 4 of the 10-line logical statement that begins at source line 1707. Its exact fragment is ‘cache.key_heads[timestep, batch_index, head].T,‘. The important operands are cache: the value named ’cache’ in this block; key_heads: the value named ’key heads’ in this block; timestep: one discrete SNN update instant. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index, head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘query_gradient, transposed_key_gradient ‘ in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; cache: the value named ’cache’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1711: `BlockGradient(` This is continuation line 5 of the 10-line logical statement that begins at source line 1707. Its exact fragment is ‘BlockGradient(‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index, head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_se)‘ to ‘query_gradient, transposed_key_gradient ‘ in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; cache: the value named ’cache’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1712: `score_gradient.values[timestep, batch_index, head],` This is continuation line 6 of the 10-line logical statement that begins at source line 1707. Its exact fragment is ‘score_gradient.values[timestep, batch_index, head],‘. The important operands are score_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; timestep: one discrete SNN update instant. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index, head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘query_gradient, transposed_key_gradient ‘ in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named ’execute linear backward block’ in this block; driver: the value named ’driver’ in this block; cache: the value named ’cache’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1713: `score_gradient.exponent,` This is continuation line 7 of the 10-line logical statement that begins at source line 1707. Its exact fragment is ‘score_gradient.exponent,‘. The important operands are score_gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index,

head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘query_gradient, transposed_key_gradient ‘ in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1714: `),` This is continuation line 8 of the 10-line logical statement that begins at source line 1707. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index, head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘query_gradient, transposed_key_gradient ‘ in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1715: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 9 of the 10-line logical statement that begins at source line 1707. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index, head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘query_gradient, transposed_key_gradient ‘ in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1716: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 1707. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_linear_backward_block(self.driver, cache.query_heads[timestep, batch_index, head], cache.key_heads[timestep, batch_index, head].T, BlockGradient(score_gradient.values[timestep, batch_index, head], score_gradient.exponent,), timeout_seconds=self.layer_timeout_seconds,)‘ to ‘query_gradient, transposed_key_gradient ‘ in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_linear_backward_block: the value named 'execute linear backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1717: `query_parts.append(query_gradient)` This calls ‘query_parts.append‘ with ‘query_gradient‘ inside the function _attention_backward, which handles attention backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1718: `key_parts.append(BlockGradient(` This statement continues through source line 1721; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘key_parts.append‘ with ‘BlockGradient(transposed_key_gradient.values.T, transposed_key_gradient.exponent,)‘ inside the function _attention_backward, which handles attention backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1719: `transposed_key_gradient.values.T,` This is continuation line 2 of the 4-line logical statement that begins at source line 1718. Its exact fragment is ‘transposed_key_gradient.values.T,‘. The important operands are transposed_key_gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block; T: the value named 't' in this block. Read together, the complete operation is: This calls ‘key_parts.append‘ with ‘BlockGradient(transposed_key_gradient.values.T, transposed_key_gradient.exponent,)‘ inside the function _attention_backward, which handles attention backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1720: `transposed_key_gradient.exponent,` This is continuation line 3 of the 4-line logical statement that begins at source line 1718. Its exact fragment is 'transposed_key_gradient.exponent,'. The important operands are transposed_key_gradient: the signed loss derivative used to update a learned parameter; exponent: the value named 'exponent' in this block. Read together, the complete operation is: This calls 'key_parts.append' with 'BlockGradient(transposed_key_gradient.values.T, transposed_key_gradient.exponent,)' inside the function _attention_backward, which handles attention backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1721: `))` This is continuation line 4 of the 4-line logical statement that begins at source line 1718. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'key_parts.append' with 'BlockGradient(transposed_key_gradient.values.T, transposed_key_gradient.exponent,)' inside the function _attention_backward, which handles attention backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1722: `part_index += 1` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are part_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 1723: `query_head_gradient = self._stack_disjoint_gradients(` This statement continues through source line 1726; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._stack_disjoint_gradients(query_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)' to 'query_head_gradie
' in host memory. Here query_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; query_parts: the value named 'query parts' in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1724: `query_parts,` This is continuation line 2 of the 4-line logical statement that begins at source line 1723. Its exact fragment is 'query_parts,'. The important operands are query_parts: the value named 'query parts' in this block. Read together, the complete operation is: This assigns 'self._stack_disjoint_gradients(query_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)' to 'query_head_gradient ' in host memory. Here query_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; query_parts: the value named 'query parts' in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1725: `(timesteps, batch, self.attention_heads, tokens, head_dim),` This is continuation line 3 of the 4-line logical statement that begins at source line 1723. Its exact fragment is '(timesteps, batch, self.attention_heads, tokens, head_dim),'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; attention_heads: the value named 'attention heads' in this block. Read together, the complete operation is: This assigns 'self._stack_disjoint_gradients(query_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)' to 'query_head_gradient ' in host memory. Here query_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; query_parts: the value named 'query parts' in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1726: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1723. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self._stack_disjoint_gradients(query_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)' to 'query_head_gradient ' in host memory. Here query_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; query_parts: the value named 'query parts' in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1727: `key_head_gradient = self._stack_disjoint_gradients(` This statement continues through source line 1730; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._stack_disjoint_gradients(key_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)‘ to ‘key_head_gradient ‘ in host memory. Here key_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; key_parts: the value named ’key parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1728: `key_parts,` This is continuation line 2 of the 4-line logical statement that begins at source line 1727. Its exact fragment is ‘key_parts,‘. The important operands are key_parts: the value named ’key parts’ in this block. Read together, the complete operation is: This assigns ‘self._stack_disjoint_gradients(key_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)‘ to ‘key_head_gradient ‘ in host memory. Here key_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; key_parts: the value named ’key parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1729: `(timesteps, batch, self.attention_heads, tokens, head_dim),` This is continuation line 3 of the 4-line logical statement that begins at source line 1727. Its exact fragment is ‘(timesteps, batch, self.attention_heads, tokens, head_dim),‘. The important operands are timesteps: one discrete SNN update instant; batch: the value named ’batch’ in this block; attention_heads: the value named ’attention heads’ in this block. Read together, the complete operation is: This assigns ‘self._stack_disjoint_gradients(key_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)‘ to ‘key_head_gradient ‘ in host memory. Here key_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; key_parts: the value named ’key parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1730: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1727. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self._stack_disjoint_gradients(key_parts, (timesteps, batch, self.attention_heads, tokens, head_dim),)‘ to ‘key_head_gradient ‘ in host memory. Here key_head_gradient means the signed loss derivative used to update a learned parameter. The important operands are _stack_disjoint_gradients: the signed loss derivative used to update a learned parameter; key_parts: the value named ’key parts’ in this block; timesteps: one discrete SNN update instant. Context: the function _attention_backward, which handles attention backward.

Line 1732: `def merge_heads(gradient: BlockGradient) -> BlockGradient:` This starts function merge_heads. Its indented body performs merge heads as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1733: `return BlockGradient(` This statement continues through source line 1737; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘BlockGradient(gradient.values.transpose(0, 1, 3, 2, 4).reshape(timesteps, batch, tokens, channels), gradient.exponent,)‘ to the caller of the function merge_heads, which handles merge heads, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block.

Line 1734: `gradient.values.transpose(0, 1, 3, 2, 4).reshape(` This is continuation line 2 of the 5-line logical statement that begins at source line 1733. Its exact fragment is ‘gradient.values.transpose(0, 1, 3, 2, 4).reshape(‘. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; transpose: the value named ’transpose’ in this block. Read together, the complete operation is: This returns ‘BlockGradient(gradient.values.transpose(0, 1, 3, 2, 4).reshape(timesteps, batch, tokens, channels), gradient.exponent,)‘ to the caller of the function merge_heads, which handles merge heads, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block.

Line 1735: `timesteps, batch, tokens, channels),` This is continuation line 3 of the 5-line logical statement that begins at source line 1733. Its exact fragment is 'timesteps, batch, tokens, channels),'. The important operands are timesteps: one discrete SNN update instant; batch: the value named 'batch' in this block; tokens: the value named 'tokens' in this block. Read together, the complete operation is: This returns 'BlockGradient(gradient.values.transpose(0, 1, 3, 2, 4).reshape(timesteps, batch, tokens, channels), gradient.exponent,)' to the caller of the function merge_heads, which handles merge heads, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block.

Line 1736: `gradient.exponent,` This is continuation line 4 of the 5-line logical statement that begins at source line 1733. Its exact fragment is 'gradient.exponent,'. The important operands are gradient: the signed loss derivative used to update a learned parameter; exponent: the value named 'exponent' in this block. Read together, the complete operation is: This returns 'BlockGradient(gradient.values.transpose(0, 1, 3, 2, 4).reshape(timesteps, batch, tokens, channels), gradient.exponent,)' to the caller of the function merge_heads, which handles merge heads, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block.

Line 1737: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1733. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'BlockGradient(gradient.values.transpose(0, 1, 3, 2, 4).reshape(timesteps, batch, tokens, channels), gradient.exponent,)' to the caller of the function merge_heads, which handles merge heads, ending this invocation. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block.

Line 1739: `query_gradient = self._lif_backward(cache.query_lif, merge_heads(query_head_gradient))` This assigns 'self._lif_backward(cache.query_lif, merge_heads(query_head_gradient))' to 'query_gradient ' in host memory. Here query_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; query_lif: the value named 'query lif' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1740: `key_gradient = self._lif_backward(cache.key_lif, merge_heads(key_head_gradient))` This assigns 'self._lif_backward(cache.key_lif, merge_heads(key_head_gradient))' to 'key_gradient ' in host memory. Here key_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; key_lif: the value named 'key lif' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1741: `value_gradient = self._lif_backward(cache.value_lif, merge_heads(value_head_gradient))` This assigns 'self._lif_backward(cache.value_lif, merge_heads(value_head_gradient))' to 'value_gradient ' in host memory. Here value_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named 'lif backward' in this block; cache: the value named 'cache' in this block; value_lif: the value named 'value lif' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1742: `token_input = cache.token_spikes.reshape(timesteps * batch * tokens, channels)` This assigns 'cache.token_spikes.reshape(timesteps * batch * tokens, channels)' to 'token_input ' in host memory. Here token_input means the value named 'token input' in this block. The important operands are cache: the value named 'cache' in this block; token_spikes: a binary event produced when a neuron crosses its threshold; reshape: the value named 'reshape' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1743: `branch_input_gradients: list[BlockGradient] = []` This assigns '[]' to 'branch_input_gradients' in host memory. Here branch_input_gradients means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function _attention_backward, which handles attention backward.

Line 1744: `parameters: dict[str, ParameterGradient] = {` This statement continues through source line 1746; the explanation uses the complete expression rather than only the first visual fragment. This assigns '{ f"{prefix}.proj":

projection_parameter, }‘ to ‘parameters‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; proj: the value named ’proj’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1745: `f"{prefix}.proj": projection_parameter,` This is continuation line 2 of the 3-line logical statement that begins at source line 1744. Its exact fragment is ‘f"{prefix}.proj": projection_parameter,‘. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; proj: the value named ’proj’ in this block. Read together, the complete operation is: This assigns ‘{ f"{prefix}.proj": projection_parameter, }‘ to ‘parameters‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; proj: the value named ’proj’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1746: `}` This is continuation line 3 of the 3-line logical statement that begins at source line 1744. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ f"{prefix}.proj": projection_parameter, }‘ to ‘parameters‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are f: the value named ’f’ in this block; prefix: the value named ’prefix’ in this block; proj: the value named ’proj’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1747: `for name, gradient in (` This line opens or closes a multi-line Python expression inside the function _attention_backward, which handles attention backward; the neighboring lines supply its arguments or elements.

Line 1748: `("q", query_gradient),` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are q: the value named ’q’ in this block; query_gradient: the signed loss derivative used to update a learned parameter.

Line 1749: `("k", key_gradient),` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are k: the value named ’k’ in this block; key_gradient: the signed loss derivative used to update a learned parameter.

Line 1750: `("v", value_gradient),` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are v: the value named ’v’ in this block; value_gradient: the signed loss derivative used to update a learned parameter.

Line 1751: `):` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1752: `rows = BlockGradient(` This statement continues through source line 1755; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘BlockGradient(gradient.values.reshape(timesteps * batch * tokens, channels), gradient.exponent,)‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1753: `gradient.values.reshape(timesteps * batch * tokens, channels),` This is continuation line 2 of the 4-line logical statement that begins at source line 1752. Its exact fragment is ‘gradient.values.reshape(timesteps * batch * tokens, channels),‘. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; reshape: the value named ’reshape’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(gradient.values.reshape(timesteps * batch * tokens, channels), gradient.exponent,)‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter;

gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1754: `gradient.exponent,` This is continuation line 3 of the 4-line logical statement that begins at source line 1752. Its exact fragment is 'gradient.exponent,'. The important operands are gradient: the signed loss derivative used to update a learned parameter; exponent: the value named 'exponent' in this block. Read together, the complete operation is: This assigns 'BlockGradient(gradient.values.reshape(timesteps * batch * tokens, channels), gradient.exponent,)' to 'rows ' in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1755: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1752. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'BlockGradient(gradient.values.reshape(timesteps * batch * tokens, channels), gradient.exponent,)' to 'rows ' in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1756: `input_rows, parameter = self._projection_matrix_backward(` This statement continues through source line 1757; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._projection_matrix_backward(f"{prefix}.{name}", token_input, rows)' to 'input_rows, parameter ' in host memory. Here input_rows means one output-row position processed by the tensor tile. The important operands are _projection_matrix_backward: the value named 'projection matrix backward' in this block; f: the value named 'f' in this block; prefix: the value named 'prefix' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1757: `f"{prefix}.{name}", token_input, rows)` This is continuation line 2 of the 2-line logical statement that begins at source line 1756. Its exact fragment is 'f"{prefix}.{name}", token_input, rows)'. The important operands are f: the value named 'f' in this block; prefix: the value named 'prefix' in this block; name: the value named 'name' in this block. Read together, the complete operation is: This assigns 'self._projection_matrix_backward(f"{prefix}.{name}", token_input, rows)' to 'input_rows, parameter ' in host memory. Here input_rows means one output-row position processed by the tensor tile. The important operands are _projection_matrix_backward: the value named 'projection matrix backward' in this block; f: the value named 'f' in this block; prefix: the value named 'prefix' in this block. Context: the function _attention_backward, which handles attention backward.

Line 1758: `branch_input_gradients.append(input_rows)` This calls 'branch_input_gradients.append' with 'input_rows' inside the function _attention_backward, which handles attention backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1759: `parameters[f"{prefix}.{name}"] = parameter` Inside the function _attention_backward, which handles attention backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are parameters: the value named 'parameters' in this block; f: the value named 'f' in this block; prefix: the value named 'prefix' in this block.

Line 1760: `token_gradient = add_block_gradients(` This statement continues through source line 1765; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'add_block_gradients(self.driver, branch_input_gradients[0], branch_input_gradients[1], timeout_seconds=self.layer_timeout_seconds,)' to 'token_gradient ' in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; branch_input_gradients: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1761: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1760. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, branch_input_gradients[0], branch_input_gradients[1], timeout_seconds=self.layer_timeout_seconds,)' to 'token_gradient ' in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; branch_input_gradients: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1762: `branch_input_gradients[0],` This is continuation line 3 of the 6-line logical statement that begins at source line 1760. Its exact fragment is 'branch_input_gradients[0],'. The important operands are branch_input_gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, branch_input_gradients[0], branch_input_gradients[1], timeout_seconds=self.layer_timeout_s)' to 'token_gradient ' in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; branch_input_gradients: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1763: `branch_input_gradients[1],` This is continuation line 4 of the 6-line logical statement that begins at source line 1760. Its exact fragment is 'branch_input_gradients[1],'. The important operands are branch_input_gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, branch_input_gradients[0], branch_input_gradients[1], timeout_seconds=self.layer_timeout_s)' to 'token_gradient ' in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; branch_input_gradients: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1764: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1760. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, branch_input_gradients[0], branch_input_gradients[1], timeout_seconds=self.layer_timeout_seconds,)' to 'token_gradient ' in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; branch_input_gradients: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1765: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1760. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, branch_input_gradients[0], branch_input_gradients[1], timeout_seconds=self.layer_timeout_seconds,)' to 'token_gradient ' in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; branch_input_gradients: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1766: `token_gradient = add_block_gradients(` This statement continues through source line 1771; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'add_block_gradients(self.driver, token_gradient, branch_input_gradients[2], timeout_seconds=self.layer_timeout_seconds,)' to 'token_gradient ' in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; token_gradient: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1767: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1766. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together,

the complete operation is: This assigns ‘add_block_gradients(self.driver, token_gradient, branch_input_gradients[2], timeout_seconds=self.layer_timeout_seconds,)‘ to ‘token_gradient ‘ in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; token_gradient: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1768: `token_gradient,` This is continuation line 3 of the 6-line logical statement that begins at source line 1766. Its exact fragment is ‘token_gradient,‘. The important operands are token_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘add_block_gradients(self.driver, token_gradient, branch_input_gradients[2], timeout_seconds=self.layer_timeout_seconds,)‘ to ‘token_gradient ‘ in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; token_gradient: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1769: `branch_input_gradients[2],` This is continuation line 4 of the 6-line logical statement that begins at source line 1766. Its exact fragment is ‘branch_input_gradients[2],‘. The important operands are branch_input_gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘add_block_gradients(self.driver, token_gradient, branch_input_gradients[2], timeout_seconds=self.layer_timeout_seconds,)‘ to ‘token_gradient ‘ in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; token_gradient: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1770: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1766. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘add_block_gradients(self.driver, token_gradient, branch_input_gradients[2], timeout_seconds=self.layer_timeout_seconds,)‘ to ‘token_gradient ‘ in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; token_gradient: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1771: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1766. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘add_block_gradients(self.driver, token_gradient, branch_input_gradients[2], timeout_seconds=self.layer_timeout_seconds,)‘ to ‘token_gradient ‘ in host memory. Here token_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; token_gradient: the signed loss derivative used to update a learned parameter. Context: the function _attention_backward, which handles attention backward.

Line 1772: `projection_spike_gradient = self._rows_to_channel_first(` This statement continues through source line 1773; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._rows_to_channel_first(token_gradient, cache.input_shape)‘ to ‘projection_spike_gradient ‘ in host memory. Here projection_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are _rows_to_channel_first: one output-row position processed by the tensor tile; token_gradient: the signed loss derivative used to update a learned parameter; cache: the value named ’cache’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1773: `token_gradient, cache.input_shape)` This is continuation line 2 of the 2-line logical statement that begins at source line 1772. Its exact fragment is ‘token_gradient, cache.input_shape)‘. The important operands are token_gradient: the signed loss derivative used to update a learned parameter; cache: the value named ’cache’ in this block; input_shape: the value named ’input shape’ in this block. Read together, the complete operation is: This assigns

‘self._rows_to_channel_first(token_gradient, cache.input_shape)‘ to ‘projection_spike_gradient ‘ in host memory. Here projection_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are _rows_to_channel_first: one output-row position processed by the tensor tile; token_gradient: the signed loss derivative used to update a learned parameter; cache: the value named ’cache’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1774: `input_gradient = self._lif_backward(` This statement continues through source line 1775; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._lif_backward(cache.projection_lif, projection_spike_gradient)‘ to ‘input_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named ’lif backward’ in this block; cache: the value named ’cache’ in this block; projection_lif: the value named ’projection lif’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1775: `cache.projection_lif, projection_spike_gradient)` This is continuation line 2 of the 2-line logical statement that begins at source line 1774. Its exact fragment is ‘cache.projection_lif, projection_spike_gradient)‘. The important operands are cache: the value named ’cache’ in this block; projection_lif: the value named ’projection lif’ in this block; projection_spike_gradient: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns ‘self._lif_backward(cache.projection_lif, projection_spike_gradient)‘ to ‘input_gradient ‘ in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named ’lif backward’ in this block; cache: the value named ’cache’ in this block; projection_lif: the value named ’projection lif’ in this block. Context: the function _attention_backward, which handles attention backward.

Line 1776: `return input_gradient, parameters` This returns ‘input_gradient, parameters‘ to the caller of the function _attention_backward, which handles attention backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; parameters: the value named ’parameters’ in this block.

Line 1778: `def _block_backward(` This starts function _block_backward. Its indented body performs block backward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1779: `self,` Inside the function _block_backward, which handles block backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1780: `cache: TransformerBlockCache,` Inside the function _block_backward, which handles block backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are cache: the value named ’cache’ in this block; TransformerBlockCache: the value named ’transformer block cache’ in this block.

Line 1781: `output_gradient: BlockGradient,` Inside the function _block_backward, which handles block backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1782: `) -> tuple[BlockGradient, dict[str, ParameterGradient]]:` Inside the function _block_backward, which handles block backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named ’tuple’ in this block; BlockGradient: the signed loss derivative used to update a learned parameter; dict: the value named ’dict’ in this block.

Line 1783: `residual_two_gradient = self._gate_saturating_add_gradient(` This statement continues through source line 1784; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._gate_saturating_add_gradient(output_gradient, cache.after_attention, cache.mlp.output)‘ to ‘residual_two_gradient ‘ in host memory. Here residual_two_gradient means the signed loss derivative used to update a learned parameter. The important operands are _gate_saturating_add_gradient: the signed loss derivative used to update a learned parameter; output_gradient: the signed loss derivative used to update a learned parameter; cache: the value named ’cache’ in this block. Context: the function _block_backward, which handles block backward.

Line 1784: `output_gradient, cache.after_attention, cache.mlp.output)` This is continuation line 2 of the 2-line logical statement that begins at source line 1783. Its exact fragment is ‘output_gradient, cache.after_attention, cache.mlp.output)‘. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; cache: the value named ’cache’ in this block; after_attention: the value named ’after attention’ in this block. Read together, the complete operation is: This assigns ‘self._gate_saturating_add_gradient(output_gradient, cache.after_attention, cache.mlp.output)‘ to ‘residual_two_gradient ‘ in host memory. Here residual_two_gradient means the signed loss derivative used to update a learned parameter. The important operands are _gate_saturating_add_gradient: the signed loss derivative used to update a learned parameter; output_gradient: the signed loss derivative used to update a learned parameter; cache: the value named ’cache’ in this block. Context: the function _block_backward, which handles block backward.

Line 1785: `mlp_input_gradient, mlp_parameters = self._mlp_backward(` This statement continues through source line 1786; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._mlp_backward(cache.mlp, residual_two_gradient)‘ to ‘mlp_input_gradient, mlp_parameters ‘ in host memory. Here mlp_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _mlp_backward: the value named ’mlp backward’ in this block; cache: the value named ’cache’ in this block; mlp: the value named ’mlp’ in this block. Context: the function _block_backward, which handles block backward.

Line 1786: `cache.mlp, residual_two_gradient)` This is continuation line 2 of the 2-line logical statement that begins at source line 1785. Its exact fragment is ‘cache.mlp, residual_two_gradient)‘. The important operands are cache: the value named ’cache’ in this block; mlp: the value named ’mlp’ in this block; residual_two_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘self._mlp_backward(cache.mlp, residual_two_gradient)‘ to ‘mlp_input_gradient, mlp_parameters ‘ in host memory. Here mlp_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _mlp_backward: the value named ’mlp backward’ in this block; cache: the value named ’cache’ in this block; mlp: the value named ’mlp’ in this block. Context: the function _block_backward, which handles block backward.

Line 1787: `after_attention_gradient = add_block_gradients(` This statement continues through source line 1792; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘add_block_gradients(self.driver, residual_two_gradient, mlp_input_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention_grad ‘ in host memory. Here after_attention_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; residual_two_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1788: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1787. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘add_block_gradients(self.driver, residual_two_gradient, mlp_input_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention_gradient ‘ in host memory. Here after_attention_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; residual_two_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1789: `residual_two_gradient,` This is continuation line 3 of the 6-line logical statement that begins at source line 1787. Its exact fragment is ‘residual_two_gradient,‘. The important operands are residual_two_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘add_block_gradients(self.driver, residual_two_gradient, mlp_input_gradient, timeout_seconds=self.layer_timeout_seconds,

)‘ to ‘after_attention_gradient ‘ in host memory. Here after_attention_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; residual_two_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1790: `mlp_input_gradient,` This is continuation line 4 of the 6-line logical statement that begins at source line 1787. Its exact fragment is ‘mlp_input_gradient,‘. The important operands are mlp_input_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘add_block_gradients(self.driver, residual_two_gradient, mlp_input_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention_grad ‘ in host memory. Here after_attention_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; residual_two_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1791: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1787. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘add_block_gradients(self.driver, residual_two_gradient, mlp_input_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention_grad ‘ in host memory. Here after_attention_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; residual_two_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1792: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1787. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘add_block_gradients(self.driver, residual_two_gradient, mlp_input_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘after_attention_gradient ‘ in host memory. Here after_attention_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; residual_two_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1793: `residual_one_gradient = self._gate_saturating_add_gradient(` This statement continues through source line 1794; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._gate_saturating_add_gradient(after_attention_gradient, cache.input_value, cache.attention.output)‘ to ‘residual_one_gradient ‘ in host memory. Here residual_one_gradient means the signed loss derivative used to update a learned parameter. The important operands are _gate_saturating_add_gradient: the signed loss derivative used to update a learned parameter; after_attention_gradient: the signed loss derivative used to update a learned parameter; cache: the value named ’cache’ in this block. Context: the function _block_backward, which handles block backward.

Line 1794: `after_attention_gradient, cache.input_value, cache.attention.output)` This is continuation line 2 of the 2-line logical statement that begins at source line 1793. Its exact fragment is ‘after_attention_gradient, cache.input_value, cache.attention.output)‘. The important operands are after_attention_gradient: the signed loss derivative used to update a learned parameter; cache: the value named ’cache’ in this block; input_value: the value named ’input value’ in this block. Read together, the complete operation is: This assigns ‘self._gate_saturating_add_gradient(after_attention_gradient, cache.input_value, cache.attention.output)‘ to ‘residual_one_gradient ‘ in host memory. Here residual_one_gradient means the signed loss derivative used to update a learned parameter. The important operands are _gate_saturating_add_gradient: the signed loss derivative used to update a learned parameter; after_attention_gradient: the signed loss derivative used to update a learned parameter; cache: the value named ’cache’ in this block. Context: the function _block_backward, which handles block backward.

Line 1795: `attention_input_gradient, attention_parameters = self._attention_backward(` This statement continues through source line 1796; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._attention_backward(cache.attention, residual_one_gradient)‘ to ‘attention_input_gradient, attention_parameters ‘ in host memory. Here attention_input_gradient means the signed loss derivative used to update

a learned parameter. The important operands are _attention_backward: the value named 'attention backward' in this block; cache: the value named 'cache' in this block; attention: the value named 'attention' in this block. Context: the function _block_backward, which handles block backward.

Line 1796: `cache.attention, residual_one_gradient)` This is continuation line 2 of the 2-line logical statement that begins at source line 1795. Its exact fragment is 'cache.attention, residual_one_gradient)'. The important operands are cache: the value named 'cache' in this block; attention: the value named 'attention' in this block; residual_one_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'self._attention_backward(cache.attention, residual_one_gradient)' to 'attention_input_gradient, attention_parameters ' in host memory. Here attention_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _attention_backward: the value named 'attention backward' in this block; cache: the value named 'cache' in this block; attention: the value named 'attention' in this block. Context: the function _block_backward, which handles block backward.

Line 1797: `input_gradient = add_block_gradients(` This statement continues through source line 1802; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'add_block_gradients(self.driver, residual_one_gradient, attention_input_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; residual_one_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1798: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1797. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, residual_one_gradient, attention_input_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; residual_one_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1799: `residual_one_gradient,` This is continuation line 3 of the 6-line logical statement that begins at source line 1797. Its exact fragment is 'residual_one_gradient,'. The important operands are residual_one_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, residual_one_gradient, attention_input_gradient, timeout_seconds=self.layer_timeout_second)' to 'input_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; residual_one_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1800: `attention_input_gradient,` This is continuation line 4 of the 6-line logical statement that begins at source line 1797. Its exact fragment is 'attention_input_gradient,'. The important operands are attention_input_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, residual_one_gradient, attention_input_gradient, timeout_seconds=self.layer_timeout_second)' to 'input_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; residual_one_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1801: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1797. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, residual_one_gradient, attention_input_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver:

the value named 'driver' in this block; residual_one_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1802: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1797. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'add_block_gradients(self.driver, residual_one_gradient, attention_input_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'input_gradient ' in host memory. Here input_gradient means the signed loss derivative used to update a learned parameter. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; residual_one_gradient: the signed loss derivative used to update a learned parameter. Context: the function _block_backward, which handles block backward.

Line 1803: `return input_gradient, {**attention_parameters, **mlp_parameters}` This returns 'input_gradient, {**attention_parameters, **mlp_parameters}' to the caller of the function _block_backward, which handles block backward, ending this invocation. The important operands are input_gradient: the signed loss derivative used to update a learned parameter; attention_parameters: the value named 'attention parameters' in this block; mlp_parameters: the value named 'mlp parameters' in this block.

Line 1805: `def _sum_timestep_gradients(self, gradient: BlockGradient) -> BlockGradient:` This starts function _sum_timestep_gradients. Its indented body performs sum timestep gradients as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1806: `parts = [` This statement continues through source line 1809; the explanation uses the complete expression rather than only the first visual fragment. This assigns '[BlockGradient(gradient.values[timestep], gradient.exponent) for timestep in range(gradient.values.shape[0])]' to 'parts ' in host memory. Here parts means the value named 'parts' in this block. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _sum_timestep_gradients, which handles sum timestep gradients.

Line 1807: `BlockGradient(gradient.values[timestep], gradient.exponent)` This is continuation line 2 of the 4-line logical statement that begins at source line 1806. Its exact fragment is 'BlockGradient(gradient.values[timestep], gradient.exponent)'. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Read together, the complete operation is: This assigns '[BlockGradient(gradient.values[timestep], gradient.exponent) for timestep in range(gradient.values.shape[0])]' to 'parts ' in host memory. Here parts means the value named 'parts' in this block. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _sum_timestep_gradients, which handles sum timestep gradients.

Line 1808: `for timestep in range(gradient.values.shape[0])` This is continuation line 3 of the 4-line logical statement that begins at source line 1806. Its exact fragment is 'for timestep in range(gradient.values.shape[0])'. The important operands are for: the value named 'for' in this block; timestep: one discrete SNN update instant; in: the value named 'in' in this block. Read together, the complete operation is: This assigns '[BlockGradient(gradient.values[timestep], gradient.exponent) for timestep in range(gradient.values.shape[0])]' to 'parts ' in host memory. Here parts means the value named 'parts' in this block. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _sum_timestep_gradients, which handles sum timestep gradients.

Line 1809: `]` This is continuation line 4 of the 4-line logical statement that begins at source line 1806. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '[BlockGradient(gradient.values[timestep], gradient.exponent) for timestep in range(gradient.values.shape[0])]' to 'parts ' in host memory. Here parts means the value named 'parts' in this block. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function _sum_timestep_gradients, which handles sum timestep gradients.

Line 1810: `while len(parts) > 1:` This repeats the following host-code block while ‘len(parts) > 1‘ remains true; surrounding timeout/error logic prevents an unbounded hardware wait.

Line 1811: `reduced: list[BlockGradient] = []` This assigns ‘[]‘ to ‘reduced‘ in host memory. Here reduced means the value named ’reduced’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _sum_timestep_gradients, which handles sum timestep gradients.

Line 1812: `for index in range(0, len(parts) - 1, 2):` This iterates index over ‘range(0, len(parts) - 1, 2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _sum_timestep_gradients, which handles sum timestep gradients.

Line 1813: `reduced.append(add_block_gradients(` This statement continues through source line 1818; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘reduced.append‘ with ‘add_block_gradients(self.driver, parts[index], parts[index + 1], timeout_seconds=self.layer_timeout_seconds,)‘ inside the function _sum_timestep_gradients, which handles sum timestep gradients. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1814: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1813. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This calls ‘reduced.append‘ with ‘add_block_gradients(self.driver, parts[index], parts[index + 1], timeout_seconds=self.layer_timeout_seconds,)‘ inside the function _sum_timestep_gradients, which handles sum timestep gradients. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1815: `parts[index],` This is continuation line 3 of the 6-line logical statement that begins at source line 1813. Its exact fragment is ‘parts[index],‘. The important operands are parts: the value named ’parts’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘reduced.append‘ with ‘add_block_gradients(self.driver, parts[index], parts[index + 1], timeout_seconds=self.layer_timeout_seconds)‘ inside the function _sum_timestep_gradients, which handles sum timestep gradients. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1816: `parts[index + 1],` This is continuation line 4 of the 6-line logical statement that begins at source line 1813. Its exact fragment is ‘parts[index + 1],‘. The important operands are parts: the value named ’parts’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘reduced.append‘ with ‘add_block_gradients(self.driver, parts[index], parts[index + 1], timeout_seconds=self.layer_timeout_seconds,)‘ inside the function _sum_timestep_gradients, which handles sum timestep gradients. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1817: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1813. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This calls ‘reduced.append‘ with ‘add_block_gradients(self.driver, parts[index], parts[index + 1], timeout_seconds=self.layer_timeout_seconds,)‘ inside the function _sum_timestep_gradients, which handles sum timestep gradients. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1818: `))` This is continuation line 6 of the 6-line logical statement that begins at source line 1813. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘reduced.append‘ with ‘add_block_gradients(self.driver, parts[index], parts[index + 1], timeout_seconds=self.layer_timeout_seconds,)‘ inside the function _sum_timestep_gradients, which handles sum timestep gradients. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1819: `if len(parts) & 1:` This executes the following indented host-code branch only when ‘len(parts) & 1‘ is true. The important operands are len: the value named ’len’ in this block; parts: the value named ’parts’ in this block. Context: the function _sum_timestep_gradients, which handles sum timestep gradients.

Line 1820: `reduced.append(parts[-1])` This calls ‘reduced.append‘ with ‘parts[-1]‘ inside the function _sum_timestep_gradien which handles sum timestep gradients. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1821: `parts = reduced` This assigns ‘reduced‘ to ‘parts ‘ in host memory. Here parts means the value named ’parts’ in this block. The important operands are reduced: the value named ’reduced’ in this block. Context: the function _sum_timestep_gradients, which handles sum timestep gradients.

Line 1822: `return parts[0]` This returns ‘parts[0]‘ to the caller of the function _sum_timestep_gradients, which handles sum timestep gradients, ending this invocation. The important operands are parts: the value named ’parts’ in this block.

Line 1824: `def _tokenizer_backward(` This starts function _tokenizer_backward. Its indented body performs tokenizer backward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1825: `self,` Inside the function _tokenizer_backward, which handles tokenizer backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1826: `caches: tuple[TokenizerLayerCache, ...],` Inside the function _tokenizer_backward, which handles tokenizer backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are caches: the value named ’caches’ in this block; tuple: the value named ’tuple’ in this block; TokenizerLayerCache: the value named ’tokenizer layer cache’ in this block.

Line 1827: `output_gradient: BlockGradient,` Inside the function _tokenizer_backward, which handles tokenizer backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter.

Line 1828: `) -> tuple[BlockGradient, dict[str, ParameterGradient]]:` Inside the function _tokenizer_backward, which handles tokenizer backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are tuple: the value named ’tuple’ in this block; BlockGradient: the signed loss derivative used to update a learned parameter; dict: the value named ’dict’ in this block.

Line 1829: `gradient = output_gradient` This assigns ‘output_gradient‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are output_gradient: the signed loss derivative used to update a learned parameter. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1830: `parameters: dict[str, ParameterGradient] = {}` This assigns ‘{}‘ to ‘parameters‘ in host memory. Here parameters means the value named ’parameters’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1831: `for index in range(4, 0, -1):` This iterates index over ‘range(4, 0, -1)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _tokenizer_backward, which handles tokenizer backward.

Line 1832: `cache = caches[index]` This assigns ‘caches[index]‘ to ‘cache ‘ in host memory. Here cache means the value named ’cache’ in this block. The important operands are caches: the value named ’caches’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1833: `if cache.lif is None:` This executes the following indented host-code branch only when ‘cache.lif is None‘ is true. The important operands are cache: the value named ’cache’ in this block; lif: the value named ’lif’ in this block; is: the value named ’is’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1834: `raise RuntimeError("nonzero tokenizer layer has no LIF trace")` This rejects an invalid state by raising ‘RuntimeError("nonzero tokenizer layer has no LIF trace")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1835: `flat_gradient = BlockGradient(` This statement continues through source line 1838; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘BlockGradient(gradient.values.reshape(-1, *gradient.values.shape[2:]), gradient.exponent,)‘ to ‘flat_gradient ‘ in host memory. Here flat_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1836: `gradient.values.reshape(-1, *gradient.values.shape[2:]),` This is continuation line 2 of the 4-line logical statement that begins at source line 1835. Its exact fragment is ‘gradient.values.reshape(-1, *gradient.values.shape[2:]),‘. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; reshape: the value named ’reshape’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(gradient.values.reshape(-1, *gradient.values.shape[2:]), gradient.exponent,)‘ to ‘flat_gradient ‘ in host memory. Here flat_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1837: `gradient.exponent,` This is continuation line 3 of the 4-line logical statement that begins at source line 1835. Its exact fragment is ‘gradient.exponent,‘. The important operands are gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(gradient.values.reshape(-1, *gradient.values.shape[2:]), gradient.exponent,)‘ to ‘flat_gradient ‘ in host memory. Here flat_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1838: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1835. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘BlockGradient(gradient.values.reshape(-1, *gradient.values.shape[2:]), gradient.exponent,)‘ to ‘flat_gradient ‘ in host memory. Here flat_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1839: `name = f"patch_embed.block{index}"` This assigns ‘f"patch_embed.block{index}"‘ to ‘name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are f: the value named ’f’ in this block; patch_embed: the value named ’patch embed’ in this block; block: the value named ’block’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1840: `projection_input_gradient, parameter = self._projection_conv2d_backward(` This statement continues through source line 1845; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._projection_conv2d_backward(name, cache.projection_input, flat_gradient, padding=1,)‘ to ‘projection_input_gradient, parameter ‘ in host memory. Here projection_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; name: the value named ’name’ in this block; cache: the value named ’cache’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1841: `name,` This is continuation line 2 of the 6-line logical statement that begins at source line 1840. Its exact fragment is ‘name,‘. The important operands are name: the value named ’name’ in this block. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward(name, cache.projection_input, flat_gradient, padding=1,)‘ to ‘projection_input_gradient, parameter ‘ in host memory. Here projection_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; name: the value named ’name’ in this block; cache: the value named ’cache’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1842: `cache.projection_input,` This is continuation line 3 of the 6-line logical statement that begins at source line 1840. Its exact fragment is ‘cache.projection_input,‘. The important operands are cache: the value named ’cache’ in this block; projection_input: the value named ’projection input’ in this block. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward(name, cache.projection_input, flat_gradient, padding=1,)‘ to ‘projection_input_gradient, parameter ‘ in host memory. Here projection_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; name: the value named ’name’ in this block; cache: the value named ’cache’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1843: `flat_gradient,` This is continuation line 4 of the 6-line logical statement that begins at source line 1840. Its exact fragment is ‘flat_gradient,‘. The important operands are flat_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward(name, cache.projection_input, flat_gradient, padding=1,)‘ to ‘projection_input_gradient, parameter ‘ in host memory. Here projection_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; name: the value named ’name’ in this block; cache: the value named ’cache’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1844: `padding=1,` This is continuation line 5 of the 6-line logical statement that begins at source line 1840. Its exact fragment is ‘padding=1,‘. The important operands are padding: the value named ’padding’ in this block. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward(name, cache.projection_input, flat_gradient, padding=1,)‘ to ‘projection_input_gradient, parameter ‘ in host memory. Here projection_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; name: the value named ’name’ in this block; cache: the value named ’cache’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1845: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1840. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward(name, cache.projection_input, flat_gradient, padding=1,)‘ to ‘projection_input_gradient, parameter ‘ in host memory. Here projection_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; name: the value named ’name’ in this block; cache: the value named ’cache’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1846: `parameters[name] = parameter` This assigns ‘parameter‘ to ‘parameters[name] ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are parameter: the value named ’parameter’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1847: `if cache.pooled:` This executes the following indented host-code branch only when ‘cache.pooled‘ is true. The important operands are cache: the value named ’cache’ in this block; pooled: the value named ’pooled’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1848: `flat_spikes = cache.lif.spikes.reshape(-1, *cache.lif.spikes.shape[2:])` This assigns ‘cache.lif.spikes.re
1, *cache.lif.spikes.shape[2:])‘ to ‘flat_spikes ‘ in host memory. Here flat_spikes means a binary event produced when a neuron crosses its threshold. The important operands are cache: the value named ’cache’ in this block; lif: the value named ’lif’ in this block; spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1849: `spike_gradient, _ = execute_maxpool2d_backward(` This statement continues through source line 1857; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_maxpool2d_backward(self.driver, flat_spikes, projection_input_gradient.values, kernel_size=3, stride=2, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spike_gradient, _ ‘ in host memory. Here spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are execute_maxpool2d_backward: the value named ’execute maxpool2d backward’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1850: `self.driver,` This is continuation line 2 of the 9-line logical statement that begins at source line 1849. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘execute_maxpool2d_backward(self.driver, flat_spikes, projection_input_gradient.values, kernel_size=3, stride=2, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spike_gradient, _ ‘ in host memory. Here spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are execute_maxpool2d_backward: the value named ’execute maxpool2d backward’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1851: `flat_spikes,` This is continuation line 3 of the 9-line logical statement that begins at source line 1849. Its exact fragment is ‘flat_spikes,‘. The important operands are flat_spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns ‘execute_maxpool2d_backward(self.driver, flat_spikes, projection_input_gradient.values, kernel_size=3, stride=2, padding=1, timeout_seconds=self.layer_timeout_seconds
)‘ to ‘spike_gradient, _ ‘ in host memory. Here spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are execute_maxpool2d_backward: the value named ’execute maxpool2d backward’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1852: `projection_input_gradient.values,` This is continuation line 4 of the 9-line logical statement that begins at source line 1849. Its exact fragment is ‘projection_input_gradient.values,‘. The important operands are projection_input_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Read together, the complete operation is: This assigns ‘execute_maxpool2d_backward(self.driver, flat_spikes, projection_input_gradient.values, kernel_size=3, stride=2, padding=1, timeout_seconds=self.layer_timeout_seconds
)‘ to ‘spike_gradient, _ ‘ in host memory. Here spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are execute_maxpool2d_backward: the value named ’execute maxpool2d backward’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1853: `kernel_size=3,` This is continuation line 5 of the 9-line logical statement that begins at source line 1849. Its exact fragment is ‘kernel_size=3,‘. The important operands are kernel_size: the value named ’kernel size’ in this block. Read together, the complete operation is: This assigns ‘execute_maxpool2d_backward(self.driver, flat_spikes, projection_input_gradient.values, kernel_size=3, stride=2, padding=1, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘spike_gradient, _ ‘ in host memory. Here spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are execute_maxpool2d_backward: the value named ’execute maxpool2d backward’ in this block; driver: the value named ’driver’ in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1854: `stride=2,` This is continuation line 6 of the 9-line logical statement that begins at source line 1849. Its exact fragment is 'stride=2,'. The important operands are stride: the value named 'stride' in this block. Read together, the complete operation is: This assigns 'execute_maxpool2d_backward(self.driver, flat_spikes, projection_input_gradient.values, kernel_size=3, stride=2, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'spike_gradient, _ ' in host memory. Here spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; driver: the value named 'driver' in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1855: `padding=1,` This is continuation line 7 of the 9-line logical statement that begins at source line 1849. Its exact fragment is 'padding=1,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'execute_maxpool2d_backward(self.driver, flat_spikes, projection_input_gradient.values, kernel_size=3, stride=2, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'spike_gradient, _ ' in host memory. Here spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; driver: the value named 'driver' in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1856: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 8 of the 9-line logical statement that begins at source line 1849. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_maxpool2d_backward(self.driver, flat_spikes, projection_input_gradient.values, kernel_size=3, stride=2, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'spike_gradient, _ ' in host memory. Here spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; driver: the value named 'driver' in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1857: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 1849. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_maxpool2d_backward(self.driver, flat_spikes, projection_input_gradient.values, kernel_size=3, stride=2, padding=1, timeout_seconds=self.layer_timeout_seconds,)' to 'spike_gradient, _ ' in host memory. Here spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; driver: the value named 'driver' in this block; flat_spikes: a binary event produced when a neuron crosses its threshold. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1858: `projection_input_gradient = BlockGradient(` This statement continues through source line 1859; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'BlockGradient(spike_gradient, projection_input_gradient.exponent)' to 'projection_input_gradient ' in host memory. Here projection_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; spike_gradient: a binary event produced when a neuron crosses its threshold; projection_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1859: `spike_gradient, projection_input_gradient.exponent)` This is continuation line 2 of the 2-line logical statement that begins at source line 1858. Its exact fragment is 'spike_gradient, projection_input_gradient.exponent)'. The important operands are spike_gradient: a binary event produced when a neuron crosses its threshold; projection_input_gradient: the signed loss derivative used to update a learned parameter; exponent: the value named 'exponent' in this block. Read together, the complete operation is: This assigns 'BlockGradient(spike_gradient, projection_input_gradient.exponent)' to 'projection_input_gradient ' in host memory. Here projection_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; spike_gradient: a binary event produced when a neuron crosses its threshold; projection_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1860: `lif_spike_gradient = BlockGradient(` This statement continues through source line 1863; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘BlockGradient(projection_input_gradient.values.reshape(cache.lif.spikes.shape), projection_input_gradient.exponent,)‘ to ‘lif_spike_gradient ‘ in host memory. Here lif_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; projection_input_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1861: `projection_input_gradient.values.reshape(cache.lif.spikes.shape),` This is continuation line 2 of the 4-line logical statement that begins at source line 1860. Its exact fragment is ‘projection_input_gradient.values.reshape(cache.l The important operands are projection_input_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; reshape: the value named ’reshape’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(projection_input_gradient.values.reshape(cache.lif.spikes.shape), projection_input_gradient.exponent,)‘ to ‘lif_spike_gradient ‘ in host memory. Here lif_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; projection_input_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1862: `projection_input_gradient.exponent,` This is continuation line 3 of the 4-line logical statement that begins at source line 1860. Its exact fragment is ‘projection_input_gradient.exponent,‘. The important operands are projection_input_gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(projection_input_gradient.values.reshape projection_input_gradient.exponent,)‘ to ‘lif_spike_gradient ‘ in host memory. Here lif_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; projection_input_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1863: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 1860. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘BlockGradient(projection_input_gradient.values.reshape(cache.lif.spikes.shape), projection_input_gradient.exponent,)‘ to ‘lif_spike_gradient ‘ in host memory. Here lif_spike_gradient means a binary event produced when a neuron crosses its threshold. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; projection_input_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1864: `gradient = self._lif_backward(cache.lif, lif_spike_gradient)` This assigns ‘self._lif_backward(cache.lif, lif_spike_gradient)‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are _lif_backward: the value named ’lif backward’ in this block; cache: the value named ’cache’ in this block; lif: the value named ’lif’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1866: `first_output_gradient = self._sum_timestep_gradients(gradient)` This assigns ‘self._sum_timestep_gradie to ‘first_output_gradient ‘ in host memory. Here first_output_gradient means the signed loss derivative used to update a learned parameter. The important operands are _sum_timestep_gradients: the signed loss derivative used to update a learned parameter; gradient: the signed loss derivative used to update a learned parameter. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1867: `first_cache = caches[0]` This assigns ‘caches[0]‘ to ‘first_cache ‘ in host memory. Here first_cache means the value named ’first cache’ in this block. The important operands are caches: the value named ’caches’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1868: `image_gradient, first_parameter = self._projection_conv2d_backward(` This statement continues through source line 1873; the explanation uses the complete expression rather than only the first visual fragment. This as-

signs ‘self._projection_conv2d_backward("patch_embed.block0", first_cache.projection_input, first_output_gradient, padding=1,)‘ to ‘image_gradient, first_parameter ‘ in host memory. Here image_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; patch_embed: the value named ’patch embed’ in this block; block0: the value named ’block0’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1869: `"patch_embed.block0",` This is continuation line 2 of the 6-line logical statement that begins at source line 1868. Its exact fragment is ‘"patch_embed.block0",‘. The important operands are patch_embed: the value named ’patch embed’ in this block; block0: the value named ’block0’ in this block. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward("patch_embed.block0", first_cache.projection_input, first_output_gradient, padding=1,)‘ to ‘image_gradient, first_parameter ‘ in host memory. Here image_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; patch_embed: the value named ’patch embed’ in this block; block0: the value named ’block0’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1870: `first_cache.projection_input,` This is continuation line 3 of the 6-line logical statement that begins at source line 1868. Its exact fragment is ‘first_cache.projection_input,‘. The important operands are first_cache: the value named ’first cache’ in this block; projection_input: the value named ’projection input’ in this block. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward("patch_embed.block0", first_cache.projection_input, first_output_gradient, padding=1,)‘ to ‘image_gradient, first_parameter ‘ in host memory. Here image_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; patch_embed: the value named ’patch embed’ in this block; block0: the value named ’block0’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1871: `first_output_gradient,` This is continuation line 4 of the 6-line logical statement that begins at source line 1868. Its exact fragment is ‘first_output_gradient,‘. The important operands are first_output_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward("patch_embed.block0", first_cache.projection_input, first_output_gradient, padding=1,)‘ to ‘image_gradient, first_parameter ‘ in host memory. Here image_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; patch_embed: the value named ’patch embed’ in this block; block0: the value named ’block0’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1872: `padding=1,` This is continuation line 5 of the 6-line logical statement that begins at source line 1868. Its exact fragment is ‘padding=1,‘. The important operands are padding: the value named ’padding’ in this block. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward("patch_embed.block0", first_cache.projection_input, first_output_gradient, padding=1,)‘ to ‘image_gradient, first_parameter ‘ in host memory. Here image_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; patch_embed: the value named ’patch embed’ in this block; block0: the value named ’block0’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1873: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1868. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self._projection_conv2d_backward("patch_embed.block0", first_cache.projection_input, first_output_gradient, padding=1,)‘ to ‘image_gradient, first_parameter ‘ in host memory. Here image_gradient means the signed loss derivative used to update a learned parameter. The important operands are _projection_conv2d_backward: the value named ’projection conv2d backward’ in this block; patch_embed: the value named ’patch embed’ in this block; block0: the value named ’block0’ in this block. Context: the function _tokenizer_backward, which handles tokenizer backward.

Line 1874: `parameters["patch_embed.block0"] = first_parameter` Inside the function _tokenizer_backward, which handles tokenizer backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are

parameters: the value named 'parameters' in this block; patch_embed: the value named 'patch embed' in this block; block0: the value named 'block0' in this block.

Line 1875: `return image_gradient, parameters` This returns 'image_gradient, parameters' to the caller of the function _tokenizer_backward, which handles tokenizer backward, ending this invocation. The important operands are image_gradient: the signed loss derivative used to update a learned parameter; parameters: the value named 'parameters' in this block.

Line 1877: `def backward(` This starts function backward. Its indented body performs backward as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1878: `self,` Inside the function backward, which handles backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1879: `cache: SpikingformerTrainingCache,` Inside the function backward, which handles backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are cache: the value named 'cache' in this block; SpikingformerTrainingCache: the value named 'spikingformer training cache' in this block.

Line 1880: `labels,` Inside the function backward, which handles backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are labels: the value named 'labels' in this block.

Line 1881: `) -> TrainingBackwardResult:` Inside the function backward, which handles backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are TrainingBackwardResult: the value named 'training backward result' in this block.

Line 1882: `"""Reverse every exported operator and produce all trainable gradients."""` This is executable documentation attached to the function backward, which handles backward. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1884: `labels_array = np.asarray(labels, dtype=np.int64)` This assigns 'np.asarray(labels, dtype=np.int64)' to 'labels_array ' in host memory. Here labels_array means the value named 'labels array' in this block. The important operands are asarray: the value named 'asarray' in this block; labels: the value named 'labels' in this block; dtype: the value named 'dtype' in this block. Context: the function backward, which handles backward.

Line 1885: `probabilities = execute_softmax_q15(` This statement continues through source line 1890; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_softmax_q15(self.driver, cache.logits_accumulator, difference_shift=self.softmax_difference_shift, timeout_seconds=self.layer_timeout_second)' to 'probabilities ' in host memory. Here probabilities means the value named 'probabilities' in this block. The important operands are execute_softmax_q15: the value named 'execute softmax q15' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1886: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1885. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_softmax_q15(self.driver, cache.logits_accumulator, difference_shift=self.softmax_difference_shift, timeout_seconds=self.layer_timeout_seconds,)' to 'probabilities ' in host memory. Here probabilities means the value named 'probabilities' in this block. The important operands are execute_softmax_q15: the value named 'execute softmax q15' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1887: `cache.logits_accumulator,` This is continuation line 3 of the 6-line logical statement that begins at source line 1885. Its exact fragment is 'cache.logits_accumulator,'. The important operands are cache: the value named 'cache' in this block; logits_accumulator: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This assigns 'execute_softmax_q15(self.driver, cache.logits_accumulator, difference_shift=self.softmax_difference_shift, timeout_seconds=self.layer_timeout_seconds,)' to 'probabilities ' in host memory. Here probabilities means the value named 'probabilities' in this block. The important operands are execute_softmax_q15: the value named 'execute softmax q15' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1888: `difference_shift=self.softmax_difference_shift,` This is continuation line 4 of the 6-line logical statement that begins at source line 1885. Its exact fragment is 'difference_shift=self.softmax_difference_shift,'. The important operands are difference_shift: the value named 'difference shift' in this block; softmax_difference_shift: the value named 'softmax difference shift' in this block. Read together, the complete operation is: This assigns 'execute_softmax_q15(self.driver, cache.logits_accumulator, difference_shift=self.softmax_difference_shift, timeout_seconds=self.lay
)' to 'probabilities ' in host memory. Here probabilities means the value named 'probabilities' in this block. The important operands are execute_softmax_q15: the value named 'execute softmax q15' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1889: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1885. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_softmax_q15(self.driver, cache.logits_accumulator, difference_shift=self.softmax_difference_shift, timeout_seconds=self.lay
)' to 'probabilities ' in host memory. Here probabilities means the value named 'probabilities' in this block. The important operands are execute_softmax_q15: the value named 'execute softmax q15' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1890: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1885. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_softmax_q15(self.driver, cache.logits_accumulator, difference_shift=self.softmax_difference_shift, timeout_seconds=self.layer_timeout_seconds,)' to 'probabilities ' in host memory. Here probabilities means the value named 'probabilities' in this block. The important operands are execute_softmax_q15: the value named 'execute softmax q15' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1891: `head_output_gradient = execute_cross_entropy_gradient_q15(` This statement continues through source line 1896; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_cross_entropy_gradient_q15(self.driver, probabilities, labels_array, timeout_seconds=self.layer_timeout_seconds,)' to 'head_output_gradient ' in host memory. Here head_output_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_cross_entropy_gradient_q15: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; probabilities: the value named 'probabilities' in this block. Context: the function backward, which handles backward.

Line 1892: `self.driver,` This is continuation line 2 of the 6-line logical statement that begins at source line 1891. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_cross_entropy_gradient_q15(self.driver, probabilities, labels_array, timeout_seconds=self.layer_timeout_seconds,)' to 'head_output_gradient ' in host memory. Here head_output_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_cross_entropy_gradient_q15: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; probabilities: the value named 'probabilities' in this block. Context: the function backward, which handles backward.

Line 1893: `probabilities,` This is continuation line 3 of the 6-line logical statement that begins at source line 1891. Its exact fragment is 'probabilities,'. The important operands are probabilities: the value named 'probabilities' in

this block. Read together, the complete operation is: This assigns ‘execute_cross_entropy_gradient_q15(self.driver, probabilities, labels_array, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_output_gradient ‘ in host memory. Here head_output_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_cross_entropy_gradient_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; probabilities: the value named ’probabilities’ in this block. Context: the function backward, which handles backward.

Line 1894: `labels_array,` This is continuation line 4 of the 6-line logical statement that begins at source line 1891. Its exact fragment is ‘labels_array,‘. The important operands are labels_array: the value named ’labels array’ in this block. Read together, the complete operation is: This assigns ‘execute_cross_entropy_gradient_q15(self.driver, probabilities, labels_array, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_output_gradient ‘ in host memory. Here head_output_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_cross_entropy_gradient_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; probabilities: the value named ’probabilities’ in this block. Context: the function backward, which handles backward.

Line 1895: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 1891. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘execute_cross_entropy_gradient_q15(self.driver, probabilities, labels_array, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_output_gradient ‘ in host memory. Here head_output_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_cross_entropy_gradient_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; probabilities: the value named ’probabilities’ in this block. Context: the function backward, which handles backward.

Line 1896: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1891. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_cross_entropy_gradient_q15(self.driver, probabilities, labels_array, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_output_gradient ‘ in host memory. Here head_output_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_cross_entropy_gradient_ the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; probabilities: the value named ’probabilities’ in this block. Context: the function backward, which handles backward.

Line 1897: `head = self.state.head` This assigns ‘self.state.head‘ to ‘head ‘ in host memory. Here head means the value named ’head’ in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; head: the value named ’head’ in this block. Context: the function backward, which handles backward.

Line 1898: `head_input_gradient, head_parameter = execute_affine_backward_block(` This statement continues through source line 1905; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_affine_backward_block(self.driver, cache.head_input, head.weight_k_by_n, head.bias_accumulator_units, head_output_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_input_gradient, head_parameter ‘ in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_affine_backward_block: the value named ’execute affine backward block’ in this block; driver: the value named ’driver’ in this block; cache: the value named ’cache’ in this block. Context: the function backward, which handles backward.

Line 1899: `self.driver,` This is continuation line 2 of the 8-line logical statement that begins at source line 1898. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘execute_affine_backward_block(self.driver, cache.head_input, head.weight_k_by_n, head.bias_accumulator_units, head_output_gradient, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_input_gradient, head_parameter ‘ in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_affine_backward_block: the value named ’execute affine backward block’ in this block; driver: the value named ’driver’ in this block; cache: the value named ’cache’ in this block. Context: the function backward, which handles backward.

Line 1900: `cache.head_input,` This is continuation line 3 of the 8-line logical statement that begins at source line 1898. Its exact fragment is 'cache.head_input,'. The important operands are cache: the value named 'cache' in this block; head_input: the value named 'head input' in this block. Read together, the complete operation is: This assigns 'execute_affine_backward_block(self.driver, cache.head_input, head.weight_k_by_n, head.bias_accumulator_units, head_output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input_gradient, head_parameter ' in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_affine_backward_block: the value named 'execute affine backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1901: `head.weight_k_by_n,` This is continuation line 4 of the 8-line logical statement that begins at source line 1898. Its exact fragment is 'head.weight_k_by_n,'. The important operands are head: the value named 'head' in this block; weight_k_by_n: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'execute_affine_backward_block(self.driver, cache.head_input, head.weight_k_by_n, head.bias_accumulator_units, head_output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input_gradient, head_parameter ' in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_affine_backward_block: the value named 'execute affine backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1902: `head.bias_accumulator_units,` This is continuation line 5 of the 8-line logical statement that begins at source line 1898. Its exact fragment is 'head.bias_accumulator_units,'. The important operands are head: the value named 'head' in this block; bias_accumulator_units: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'execute_affine_backward_block(self.driver, cache.head_input, head.weight_k_by_n, head.bias_accumulator_units, head_output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input_gradient, head_parameter ' in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_affine_backward_block: the value named 'execute affine backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1903: `head_output_gradient,` This is continuation line 6 of the 8-line logical statement that begins at source line 1898. Its exact fragment is 'head_output_gradient,'. The important operands are head_output_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'execute_affine_backward_block(self.driver, cache.head_input, head.weight_k_by_n, head.bias_accumulator_units, head_output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input_gradient, head_parameter ' in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_affine_backward_block: the value named 'execute affine backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1904: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 7 of the 8-line logical statement that begins at source line 1898. Its exact fragment is 'timeout_seconds=self.layer_timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'execute_affine_backward_block(self.driver, cache.head_input, head.weight_k_by_n, head.bias_accumulator_units, head_output_g timeout_seconds=self.layer_timeout_seconds,)' to 'head_input_gradient, head_parameter ' in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_affine_backward_block: the value named 'execute affine backward block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1905: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 1898. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_affine_backward_block(self.driver, cache.head_input, head.weight_k_by_n, head.bias_accumulator_units, head_output_gradient, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input_gradient, head_parameter ' in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_affine_backward_block: the value named 'execute affine backward

block' in this block; driver: the value named 'driver' in this block; cache: the value named 'cache' in this block. Context: the function backward, which handles backward.

Line 1906: `clip_mask = (` This statement continues through source line 1908; the explanation uses the complete expression rather than only the first visual fragment. Inside the function backward, which handles backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are clip_mask: the value named 'clip mask' in this block; cache: the value named 'cache' in this block; head_preclip: the value named 'head preclip' in this block.

Line 1907: `(cache.head_preclip >= -128) & (cache.head_preclip <= 127)` This is continuation line 2 of the 3-line logical statement that begins at source line 1906. Its exact fragment is '(cache.head_preclip >= -128) & (cache.head_preclip <= 127)'. The important operands are cache: the value named 'cache' in this block; head_preclip: the value named 'head preclip' in this block. Read together, the complete operation is: Inside the function backward, which handles backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are clip_mask: the value named 'clip mask' in this block; cache: the value named 'cache' in this block; head_preclip: the value named 'head preclip' in this block.

Line 1908: `).astype(np.int64)` This is continuation line 3 of the 3-line logical statement that begins at source line 1906. Its exact fragment is ').astype(np.int64)'. The important operands are astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: Inside the function backward, which handles backward, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are clip_mask: the value named 'clip mask' in this block; cache: the value named 'cache' in this block; head_preclip: the value named 'head preclip' in this block.

Line 1909: `head_input_values = execute_vector_tensor(` This statement continues through source line 1915; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, clip_mask, head_input_gradient.values, timeout_seconds=self.layer_timeout_)' to 'head_input_values ' in host memory. Here head_input_values means the value named 'head input values' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function backward, which handles backward.

Line 1910: `self.driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 1909. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, clip_mask, head_input_gradient.values, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input_values ' in host memory. Here head_input_values means the value named 'head input values' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function backward, which handles backward.

Line 1911: `VectorOperation.SELECT_NONZERO,` This is continuation line 3 of the 7-line logical statement that begins at source line 1909. Its exact fragment is 'VectorOperation.SELECT_NONZERO,'. The important operands are VectorOperation: the value named 'vector operation' in this block; SELECT_NONZERO: the value named 'select nonzero' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, clip_mask, head_input_gradient.values, timeout_seconds=self.layer_timeout_seconds,)' to 'head_input_values ' in host memory. Here head_input_values means the value named 'head input values' in this block. The important operands are execute_vector_tensor: the value named 'execute vector tensor' in this block; driver: the value named 'driver' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function backward, which handles backward.

Line 1912: `clip_mask,` This is continuation line 4 of the 7-line logical statement that begins at source line 1909. Its exact fragment is 'clip_mask,'. The important operands are clip_mask: the value named 'clip mask' in this block. Read together, the complete operation is: This assigns 'execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO,

clip_mask, head_input_gradient.values, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_input_values ‘ in host memory. Here head_input_values means the value named ’head input values’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function backward, which handles backward.

Line 1913: `head_input_gradient.values,` This is continuation line 5 of the 7-line logical statement that begins at source line 1909. Its exact fragment is ‘head_input_gradient.values,‘. The important operands are head_input_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, clip_mask, head_input_gradient.values, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_input_values ‘ in host memory. Here head_input_values means the value named ’head input values’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function backward, which handles backward.

Line 1914: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 6 of the 7-line logical statement that begins at source line 1909. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, clip_mask, head_input_gradient.values, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_input_values ‘ in host memory. Here head_input_values means the value named ’head input values’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function backward, which handles backward.

Line 1915: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1909. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_vector_tensor(self.driver, VectorOperation.SELECT_NONZERO, clip_mask, head_input_gradient.values, timeout_seconds=self.layer_timeout_seconds,)‘ to ‘head_input_values ‘ in host memory. Here head_input_values means the value named ’head input values’ in this block. The important operands are execute_vector_tensor: the value named ’execute vector tensor’ in this block; driver: the value named ’driver’ in this block; VectorOperation: the value named ’vector operation’ in this block. Context: the function backward, which handles backward.

Line 1916: `head_input_gradient = BlockGradient(` This statement continues through source line 1917; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘BlockGradient(head_input_values, head_input_gradient.exponent)‘ to ‘head_input_gradient ‘ in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; head_input_values: the value named ’head input values’ in this block; head_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1917: `head_input_values, head_input_gradient.exponent)` This is continuation line 2 of the 2-line logical statement that begins at source line 1916. Its exact fragment is ‘head_input_values, head_input_gradient.exponent)‘. The important operands are head_input_values: the value named ’head input values’ in this block; head_input_gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(head_input_values, head_input_gradient.exponent)‘ to ‘head_input_gradient ‘ in host memory. Here head_input_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; head_input_values: the value named ’head input values’ in this block; head_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1918: `divisor = (` This statement continues through source line 1922; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(cache.final_value.shape[0] * cache.final_value.shape[3] * cache.final_value.shape[4])‘ to ‘divisor ‘ in host memory. Here divisor means the value named ’divisor’ in this block.

The important operands are cache: the value named 'cache' in this block; final_value: the value named 'final value' in this block; shape: the value named 'shape' in this block. Context: the function backward, which handles backward.

Line 1919: `cache.final_value.shape[0]` This is continuation line 2 of the 5-line logical statement that begins at source line 1918. Its exact fragment is 'cache.final_value.shape[0]'. The important operands are cache: the value named 'cache' in this block; final_value: the value named 'final value' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns '(cache.final_value.shape[0] * cache.final_value.shape[3] * cache.final_value.shape[4])' to 'divisor ' in host memory. Here divisor means the value named 'divisor' in this block. The important operands are cache: the value named 'cache' in this block; final_value: the value named 'final value' in this block; shape: the value named 'shape' in this block. Context: the function backward, which handles backward.

Line 1920: `* cache.final_value.shape[3]` This is continuation line 3 of the 5-line logical statement that begins at source line 1918. Its exact fragment is '* cache.final_value.shape[3]'. The important operands are cache: the value named 'cache' in this block; final_value: the value named 'final value' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns '(cache.final_value.shape[0] * cache.final_value.shape[3] * cache.final_value.shape[4])' to 'divisor ' in host memory. Here divisor means the value named 'divisor' in this block. The important operands are cache: the value named 'cache' in this block; final_value: the value named 'final value' in this block; shape: the value named 'shape' in this block. Context: the function backward, which handles backward.

Line 1921: `* cache.final_value.shape[4]` This is continuation line 4 of the 5-line logical statement that begins at source line 1918. Its exact fragment is '* cache.final_value.shape[4]'. The important operands are cache: the value named 'cache' in this block; final_value: the value named 'final value' in this block; shape: the value named 'shape' in this block. Read together, the complete operation is: This assigns '(cache.final_value.shape[0] * cache.final_value.shape[3] * cache.final_value.shape[4])' to 'divisor ' in host memory. Here divisor means the value named 'divisor' in this block. The important operands are cache: the value named 'cache' in this block; final_value: the value named 'final value' in this block; shape: the value named 'shape' in this block. Context: the function backward, which handles backward.

Line 1922: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 1918. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(cache.final_value.shape[0] * cache.final_value.shape[3] * cache.final_value.shape[4])' to 'divisor ' in host memory. Here divisor means the value named 'divisor' in this block. The important operands are cache: the value named 'cache' in this block; final_value: the value named 'final value' in this block; shape: the value named 'shape' in this block. Context: the function backward, which handles backward.

Line 1923: `denominator = int(round(divisor * self.fixed_scale * self.head_activation_scale))` This assigns 'int(round(divisor * self.fixed_scale * self.head_activation_scale))' to 'denominator ' in host memory. Here denominator means the value named 'denominator' in this block. The important operands are round: the value named 'round' in this block; divisor: the value named 'divisor' in this block; fixed_scale: the real-value spacing represented by one quantized integer step. Context: the function backward, which handles backward.

Line 1924: `if denominator <= 0 or denominator & (denominator - 1):` This executes the following indented host-code branch only when 'denominator <= 0 or denominator & (denominator - 1)' is true. The important operands are denominator: the value named 'denominator' in this block; or: the value named 'or' in this block. Context: the function backward, which handles backward.

Line 1925: `raise ValueError("the exported head pooling denominator must be a power of two")` This rejects an invalid state by raising 'ValueError("the exported head pooling denominator must be a power of two")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1926: `pooling_shift = denominator.bit_length() - 1` This assigns 'denominator.bit_length() - 1' to 'pooling_shift ' in host memory. Here pooling_shift means the value named 'pooling shift' in this block. The important operands are denominator: the value named 'denominator' in this block; bit_length: the value named 'bit length' in this block. Context: the function backward, which handles backward.

Line 1927: `final_gradient = BlockGradient(` This statement continues through source line 1933; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘BlockGradient(np.broadcast_to(head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis], cache.final_value.shape,).copy(), head_input_gradient.expo - pooling_shift,)‘ to ‘final_gradient ‘ in host memory. Here final_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; broadcast_to: the value named ’broadcast to’ in this block; head_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1928: `np.broadcast_to(` This is continuation line 2 of the 7-line logical statement that begins at source line 1927. Its exact fragment is ‘np.broadcast_to(‘. The important operands are broadcast_to: the value named ’broadcast to’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(np.broadcast_to(head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis], cache.final_value.shape,).copy(), head_input_gradient.expo - pooling_shift,)‘ to ‘final_gradient ‘ in host memory. Here final_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; broadcast_to: the value named ’broadcast to’ in this block; head_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1929: `head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis],` This is continuation line 3 of the 7-line logical statement that begins at source line 1927. Its exact fragment is ‘head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis],‘. The important operands are head_input_gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block; newaxis: the value named ’newaxis’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(np.broadcast_to(head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis], cache.final_value.shape,).copy(), head_input_gradient.exponent - pooling_shift,)‘ to ‘final_gradient ‘ in host memory. Here final_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; broadcast_to: the value named ’broadcast to’ in this block; head_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1930: `cache.final_value.shape,` This is continuation line 4 of the 7-line logical statement that begins at source line 1927. Its exact fragment is ‘cache.final_value.shape,‘. The important operands are cache: the value named ’cache’ in this block; final_value: the value named ’final value’ in this block; shape: the value named ’shape’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(np.broadcast_to(head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis], cache.final_value.shape,).copy(), head_input_gradient.exponent - pooling_shift,)‘ to ‘final_gradient ‘ in host memory. Here final_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; broadcast_to: the value named ’broadcast to’ in this block; head_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1931: `).copy(),` This is continuation line 5 of the 7-line logical statement that begins at source line 1927. Its exact fragment is ‘).copy(),‘. The important operands are copy: the value named ’copy’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(np.broadcast_to(head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis], cache.final_value.shape,).copy(), head_input_gradient.exponent - pooling_shift,)‘ to ‘final_gradient ‘ in host memory. Here final_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; broadcast_to: the value named ’broadcast to’ in this block; head_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1932: `head_input_gradient.exponent - pooling_shift,` This is continuation line 6 of the 7-line logical statement that begins at source line 1927. Its exact fragment is ‘head_input_gradient.exponent - pooling_shift,‘. The important operands are head_input_gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block; pooling_shift: the value named ’pooling shift’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(np.broadcast_to(head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis], cache.final_value.shape,).copy(), head_input_gradient.exponent - pooling_shift,)‘ to ‘final_gradient ‘ in host memory. Here final_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; broadcast_to: the value named ’broadcast to’ in this block; head_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1933: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1927. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'BlockGradient(np.broadcast_to(head_input_gradient.values[np.newaxis, :, :, np.newaxis, np.newaxis], cache.final_value.shape,).copy(), head_input_gradient.exponent - pooling_shift,)' to 'final_gradient ' in host memory. Here final_gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; broadcast_to: the value named 'broadcast to' in this block; head_input_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1935: `parameter_gradients: dict[str, ParameterGradient] = {"head": head_parameter}` This assigns '{"head": head_parameter}' to 'parameter_gradients' in host memory. Here parameter_gradients means the signed loss derivative used to update a learned parameter. The important operands are head: the value named 'head' in this block; head_parameter: the value named 'head parameter' in this block. Context: the function backward, which handles backward.

Line 1936: `gradient = final_gradient` This assigns 'final_gradient' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are final_gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1937: `for block_cache in reversed(cache.blocks):` This iterates block_cache over 'reversed(cache.blocks)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function backward, which handles backward.

Line 1938: `gradient, block_parameters = self._block_backward(block_cache, gradient)` This assigns 'self._block_bac gradient)' to 'gradient, block_parameters ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are _block_backward: the value named 'block backward' in this block; block_cache: the value named 'block cache' in this block; gradient: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1939: `overlap = set(parameter_gradients) & set(block_parameters)` This assigns 'set(parameter_gradients) & set(block_parameters)' to 'overlap ' in host memory. Here overlap means the value named 'overlap' in this block. The important operands are set: the value named 'set' in this block; parameter_gradients: the signed loss derivative used to update a learned parameter; block_parameters: the value named 'block parameters' in this block. Context: the function backward, which handles backward.

Line 1940: `if overlap:` This executes the following indented host-code branch only when 'overlap' is true. The important operands are overlap: the value named 'overlap' in this block. Context: the function backward, which handles backward.

Line 1941: `raise RuntimeError(f"duplicate block parameter gradients: {sorted(overlap)}")` This rejects an invalid state by raising 'RuntimeError(f"duplicate block parameter gradients: {sorted(overlap)}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1942: `parameter_gradients.update(block_parameters)` This calls 'parameter_gradients.update' with 'block_parameter inside the function backward, which handles backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1943: `image_gradient, tokenizer_parameters = self._tokenizer_backward(` This statement continues through source line 1944; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._tokenizer_backward(cache.tokenizer, gradient)' to 'image_gradient, tokenizer_parameters ' in host memory. Here image_gradient means the signed loss derivative used to update a learned parameter. The important operands are _tokenizer_backward: the value named 'tokenizer backward' in this block; cache: the value named 'cache' in this block; tokenizer: the value named 'tokenizer' in this block. Context: the function backward, which handles backward.

Line 1944: `cache.tokenizer, gradient)` This is continuation line 2 of the 2-line logical statement that begins at source line 1943. Its exact fragment is ‘cache.tokenizer, gradient)‘. The important operands are cache: the value named ’cache’ in this block; tokenizer: the value named ’tokenizer’ in this block; gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘self._tokenizer_backward(cache.tokenizer, gradient)‘ to ‘image_gradient, tokenizer_parameters ‘ in host memory. Here image_gradient means the signed loss derivative used to update a learned parameter. The important operands are _tokenizer_backward: the value named ’tokenizer backward’ in this block; cache: the value named ’cache’ in this block; tokenizer: the value named ’tokenizer’ in this block. Context: the function backward, which handles backward.

Line 1945: `overlap = set(parameter_gradients) & set(tokenizer_parameters)` This assigns ‘set(parameter_gradients) & set(tokenizer_parameters)‘ to ‘overlap ‘ in host memory. Here overlap means the value named ’overlap’ in this block. The important operands are set: the value named ’set’ in this block; parameter_gradients: the signed loss derivative used to update a learned parameter; tokenizer_parameters: the value named ’tokenizer parameters’ in this block. Context: the function backward, which handles backward.

Line 1946: `if overlap:` This executes the following indented host-code branch only when ‘overlap‘ is true. The important operands are overlap: the value named ’overlap’ in this block. Context: the function backward, which handles backward.

Line 1947: `raise RuntimeError(f"duplicate tokenizer parameter gradients: {sorted(overlap)}")` This rejects an invalid state by raising ‘RuntimeError(f"duplicate tokenizer parameter gradients: {sorted(overlap)}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1948: `parameter_gradients.update(tokenizer_parameters)` This calls ‘parameter_gradients.update‘ with ‘tokenizer_parameters‘ inside the function backward, which handles backward. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1949: `expected = set(self.state.projection_names) | {"head"}` This assigns ‘set(self.state.projection_names) | {"head"}‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are set: the value named ’set’ in this block; state: the finite-state-machine phase retained in flip-flops; projection_names: the value named ’projection names’ in this block. Context: the function backward, which handles backward.

Line 1950: `if set(parameter_gradients) != expected:` This executes the following indented host-code branch only when ‘set(parameter_gradients) != expected‘ is true. The important operands are set: the value named ’set’ in this block; parameter_gradients: the signed loss derivative used to update a learned parameter; expected: the value named ’expected’ in this block. Context: the function backward, which handles backward.

Line 1951: `missing = sorted(expected - set(parameter_gradients))` This assigns ‘sorted(expected - set(parameter_gradie to ‘missing ‘ in host memory. Here missing means the value named ’missing’ in this block. The important operands are sorted: the value named ’sorted’ in this block; expected: the value named ’expected’ in this block; set: the value named ’set’ in this block. Context: the function backward, which handles backward.

Line 1952: `extra = sorted(set(parameter_gradients) - expected)` This assigns ‘sorted(set(parameter_gradients) - expected)‘ to ‘extra ‘ in host memory. Here extra means the value named ’extra’ in this block. The important operands are sorted: the value named ’sorted’ in this block; set: the value named ’set’ in this block; parameter_gradients: the signed loss derivative used to update a learned parameter. Context: the function backward, which handles backward.

Line 1953: `raise RuntimeError(f"reverse graph coverage mismatch: missing={missing}, extra={extra}")` This rejects an invalid state by raising ‘RuntimeError(f"reverse graph coverage mismatch: missing={missing}, extra={extra}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 1955: `selected = probabilities[np.arange(probabilities.shape[0]), labels_array]` This assigns ‘probabilities[np.arange(probabilities.shape[0]), labels_array]‘ to ‘selected ‘ in host memory. Here selected means the value named ’selected’ in this block. The important operands are probabilities: the value named ’probabilities’ in this block; arange: the value named ’arange’ in this block; shape: the value named ’shape’ in this block. Context: the function backward, which handles backward.

Line 1956: `loss = float(-np.log(np.maximum(selected, 1) / float(1 << 15)).mean())` This assigns ‘float(-np.log(np.maximum(selected, 1) / float(1 << 15)).mean())‘ to ‘loss ‘ in host memory. Here loss means the value named ’loss’ in this block. The important operands are float: the value named ’float’ in this block; log: the value named ’log’ in this block; maximum: the value named ’maximum’ in this block. Context: the function backward, which handles backward.

Line 1957: `return TrainingBackwardResult(` This statement continues through source line 1963; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘TrainingBackwardResult(loss, probabilities, np.argmax(cache.logits_accumulator, axis=1), parameter_gradients, image_gradient,)‘ to the caller of the function backward, which handles backward, ending this invocation. The important operands are TrainingBackwardResult: the value named ’training backward result’ in this block; loss: the value named ’loss’ in this block; probabilities: the value named ’probabilities’ in this block.

Line 1958: `loss,` This is continuation line 2 of the 7-line logical statement that begins at source line 1957. Its exact fragment is ‘loss,‘. The important operands are loss: the value named ’loss’ in this block. Read together, the complete operation is: This returns ‘TrainingBackwardResult(loss, probabilities, np.argmax(cache.logits_accumulator, axis=1), parameter_gradients, image_gradient,)‘ to the caller of the function backward, which handles backward, ending this invocation. The important operands are TrainingBackwardResult: the value named ’training backward result’ in this block; loss: the value named ’loss’ in this block; probabilities: the value named ’probabilities’ in this block.

Line 1959: `probabilities,` This is continuation line 3 of the 7-line logical statement that begins at source line 1957. Its exact fragment is ‘probabilities,‘. The important operands are probabilities: the value named ’probabilities’ in this block. Read together, the complete operation is: This returns ‘TrainingBackwardResult(loss, probabilities, np.argmax(cache.logits_accumulator, axis=1), parameter_gradients, image_gradient,)‘ to the caller of the function backward, which handles backward, ending this invocation. The important operands are TrainingBackwardResult: the value named ’training backward result’ in this block; loss: the value named ’loss’ in this block; probabilities: the value named ’probabilities’ in this block.

Line 1960: `np.argmax(cache.logits_accumulator, axis=1),` This is continuation line 4 of the 7-line logical statement that begins at source line 1957. Its exact fragment is ‘np.argmax(cache.logits_accumulator, axis=1),‘. The important operands are argmax: the value named ’argmax’ in this block; cache: the value named ’cache’ in this block; logits_accumulator: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This returns ‘TrainingBackwardResult(loss, probabilities, np.argmax(cache.logits_accumulator, axis=1), parameter_gradients, image_gradient,)‘ to the caller of the function backward, which handles backward, ending this invocation. The important operands are TrainingBackwardResult: the value named ’training backward result’ in this block; loss: the value named ’loss’ in this block; probabilities: the value named ’probabilities’ in this block.

Line 1961: `parameter_gradients,` This is continuation line 5 of the 7-line logical statement that begins at source line 1957. Its exact fragment is ‘parameter_gradients,‘. The important operands are parameter_gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This returns ‘TrainingBackwardResult(loss, probabilities, np.argmax(cache.logits_accumulator, axis=1), parameter_gradients, image_gradient,)‘ to the caller of the function backward, which handles backward, ending this invocation. The important operands are TrainingBackwardResult: the value named ’training backward result’ in this block; loss: the value named ’loss’ in this block; probabilities: the value named ’probabilities’ in this block.

Line 1962: `image_gradient,` This is continuation line 6 of the 7-line logical statement that begins at source line 1957. Its exact fragment is ‘image_gradient,‘. The important operands are image_gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This returns ‘TrainingBackwardResult(loss, probabilities, np.argmax(cache.logits_accumulator, axis=1), parameter_gradients, image_gradient,)‘ to the caller

of the function backward, which handles backward, ending this invocation. The important operands are TrainingBackwardResult: the value named 'training backward result' in this block; loss: the value named 'loss' in this block; probabilities: the value named 'probabilities' in this block.

Line 1963: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 1957. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'TrainingBackwardResult(loss, probabilities, np.argmax(cache.logits_accumulator, axis=1), parameter_gradients, image_gradient,)' to the caller of the function backward, which handles backward, ending this invocation. The important operands are TrainingBackwardResult: the value named 'training backward result' in this block; loss: the value named 'loss' in this block; probabilities: the value named 'probabilities' in this block.

Line 1965: `def train_step(` This starts function train_step. Its indented body performs train step as part of the file that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation; it runs only when called.

Line 1966: `self,` Inside the function train_step, which handles train step, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1967: `normalized_image,` Inside the function train_step, which handles train step, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are normalized_image: the value named 'normalized image' in this block.

Line 1968: `labels,` Inside the function train_step, which handles train step, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are labels: the value named 'labels' in this block.

Line 1969: `*,` Inside the function train_step, which handles train step, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The expression is a literal or structural operator and has no hidden software state.

Line 1970: `learning_rate: float,` Inside the function train_step, which handles train step, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude; float: the value named 'float' in this block.

Line 1971: `momentum: float = 0.9,` This assigns '0.9,' to 'momentum' in host memory. Here momentum means the value named 'momentum' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function train_step, which handles train step.

Line 1972: `weight_decay: float = 5e-4,` This assigns '5e-4,' to 'weight_decay' in host memory. Here weight_decay means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function train_step, which handles train step.

Line 1973: `gradient_clip_q15: int = 4 << 15,` This assigns '4 << 15,' to 'gradient_clip_q15' in host memory. Here gradient_clip_q15 means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function train_step, which handles train step.

Line 1974: `) -> TrainingStepResult:` Inside the function train_step, which handles train step, this statement advances the host implementation that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation. The important operands are TrainingStepResult: the value named 'training step result' in this block.

Line 1975: `"""Execute one full forward, reverse, and all-parameter SGDW update."""` This is executable documentation attached to the function train_step, which handles train step. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 1977: `cache = self.forward(normalized_image)` This assigns 'self.forward(normalized_image)' to 'cache ' in host memory. Here cache means the value named 'cache' in this block. The important operands are forward: the value named 'forward' in this block; normalized_image: the value named 'normalized image' in this block. Context: the function train_step, which handles train step.

Line 1978: `backward = self.backward(cache, labels)` This assigns 'self.backward(cache, labels)' to 'backward ' in host memory. Here backward means the value named 'backward' in this block. The important operands are backward: the value named 'backward' in this block; cache: the value named 'cache' in this block; labels: the value named 'labels' in this block. Context: the function train_step, which handles train step.

Line 1979: `self.state.apply_gradients(` This statement continues through source line 1987; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=self.layer_timeout_seconds, ' inside the function train_step, which handles train step. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1980: `self.driver,` This is continuation line 2 of the 9-line logical statement that begins at source line 1979. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This calls 'self.state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=self.layer_timeout_seconds, ' inside the function train_step, which handles train step. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1981: `backward.parameter_gradients,` This is continuation line 3 of the 9-line logical statement that begins at source line 1979. Its exact fragment is 'backward.parameter_gradients,'. The important operands are backward: the value named 'backward' in this block; parameter_gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls 'self.state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=self.layer_timeout_seconds, ' inside the function train_step, which handles train step. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1982: `learning_rate=learning_rate,` This is continuation line 4 of the 9-line logical statement that begins at source line 1979. Its exact fragment is 'learning_rate=learning_rate,'. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This calls 'self.state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=self.layer_timeout_sec
' inside the function train_step, which handles train step. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1983: `momentum=momentum,` This is continuation line 5 of the 9-line logical statement that begins at source line 1979. Its exact fragment is 'momentum=momentum,'. The important operands are momentum: the value named 'momentum' in this block. Read together, the complete operation is: This calls 'self.state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=self.layer_timeout_seconds, ' inside the function train_step, which handles train step. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1984: `weight_decay=weight_decay,` This is continuation line 6 of the 9-line logical statement that begins at source line 1979. Its exact fragment is ‘weight_decay=weight_decay,‘. The important operands are weight_decay: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls ‘self.state.apply_gradients‘ with ‘ self.driver, backward.parameter_gradients, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_d gradient_clip_q15=gradient_clip_q15, timeout_seconds=self.layer_timeout_seconds, ‘ inside the function train_step, which handles train step. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1985: `gradient_clip_q15=gradient_clip_q15,` This is continuation line 7 of the 9-line logical statement that begins at source line 1979. Its exact fragment is ‘gradient_clip_q15=gradient_clip_q15,‘. The important operands are gradient_clip_q15: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls ‘self.state.apply_gradients‘ with ‘ self.driver, backward.parameter_gradients, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=self.layer_timeo ‘ inside the function train_step, which handles train step. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1986: `timeout_seconds=self.layer_timeout_seconds,` This is continuation line 8 of the 9-line logical statement that begins at source line 1979. Its exact fragment is ‘timeout_seconds=self.layer_timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This calls ‘self.state.apply_gradients‘ with ‘ self.driver, backward.parameter_gradients, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_d gradient_clip_q15=gradient_clip_q15, timeout_seconds=self.layer_timeout_seconds, ‘ inside the function train_step, which handles train step. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1987: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 1979. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.state.apply_gradients‘ with ‘ self.driver, backward.parameter_gradients, learning_rate=learning_rate, momentum=momentum, weight_decay=weight_decay, gradient_clip_q15=gradient_clip_q15, timeout_seconds=self.layer_timeout_seconds, ‘ inside the function train_step, which handles train step. The call advances the host-side job that runs resumable CIFAR-10 training through the accelerator command ABI with atomic checkpoints and validation.

Line 1988: `return TrainingStepResult(` This statement continues through source line 1993; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘TrainingStepResult(backward.loss, backward.probabilities_q15, backward.predicted_class, self.state.optimizer_step,)‘ to the caller of the function train_step, which handles train step, ending this invocation. The important operands are TrainingStepResult: the value named ’training step result’ in this block; backward: the value named ’backward’ in this block; loss: the value named ’loss’ in this block.

Line 1989: `backward.loss,` This is continuation line 2 of the 6-line logical statement that begins at source line 1988. Its exact fragment is ‘backward.loss,‘. The important operands are backward: the value named ’backward’ in this block; loss: the value named ’loss’ in this block. Read together, the complete operation is: This returns ‘TrainingStepResult(backward.loss, backward.probabilities_q15, backward.predicted_class, self.state.optimizer_step,)‘ to the caller of the function train_step, which handles train step, ending this invocation. The important operands are TrainingStepResult: the value named ’training step result’ in this block; backward: the value named ’backward’ in this block; loss: the value named ’loss’ in this block.

Line 1990: `backward.probabilities_q15,` This is continuation line 3 of the 6-line logical statement that begins at source line 1988. Its exact fragment is ‘backward.probabilities_q15,‘. The important operands are backward: the value named ’backward’ in this block; probabilities_q15: the value named ’probabilities q15’ in this block. Read together, the complete operation is: This returns ‘TrainingStepResult(backward.loss, backward.probabilities_q15, backward.predicted_class, self.state.optimizer_step,)‘ to the caller of the function train_step, which handles train step, ending this invocation. The important operands are TrainingStepResult: the value named ’training step result’ in this block; backward: the value named ’backward’ in this block; loss: the value named ’loss’ in this block.

Line 1991: `backward.predicted_class,` This is continuation line 4 of the 6-line logical statement that begins at source line 1988. Its exact fragment is ‘backward.predicted_class,‘. The important operands are backward: the value named ’backward’ in this block; predicted_class: the value named ’predicted class’ in this block. Read together, the complete operation is: This returns ‘TrainingStepResult(backward.loss, backward.probabilities_q15, backward.predicted_class, self.state.optimizer_step,)‘ to the caller of the function train_step, which handles train step, ending this invocation. The important operands are TrainingStepResult: the value named ’training step result’ in this block; backward: the value named ’backward’ in this block; loss: the value named ’loss’ in this block.

Line 1992: `self.state.optimizer_step,` This is continuation line 5 of the 6-line logical statement that begins at source line 1988. Its exact fragment is ‘self.state.optimizer_step,‘. The important operands are state: the finite-state-machine phase retained in flip-flops; optimizer_step: the value named ’optimizer step’ in this block. Read together, the complete operation is: This returns ‘TrainingStepResult(backward.loss, backward.probabilities_q15, backward.predicted_class, self.state.optimizer_step,)‘ to the caller of the function train_step, which handles train step, ending this invocation. The important operands are TrainingStepResult: the value named ’training step result’ in this block; backward: the value named ’backward’ in this block; loss: the value named ’loss’ in this block.

Line 1993: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 1988. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘TrainingStepResult(backward.loss, backward.probabilities_q15, backward.predicted_class, self.state.optimizer_step,)‘ to the caller of the function train_step, which handles train step, ending this invocation. The important operands are TrainingStepResult: the value named ’training step result’ in this block; backward: the value named ’backward’ in this block; loss: the value named ’loss’ in this block.

45 software/train_spikingformer_pynq.py

Role in the complete system: provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

45.1 Beginner architecture map

Dataflow responsibility The command-line entry point validates the overlay, parameter archive, dataset shards, and ABI before constructing the runtime. It translates pause-file and resume options into explicit checkpoint behavior, reports board telemetry and validation accuracy, and exits nonzero when required physical artifacts or numerical checks fail.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: Cifar10ShardSet. Functions: parse_arguments, atomic_json, file_sha256, augment_cifar10, cosine_learning_rate, evaluate, ___init___, batches. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

45.2 Complete source

```
"""Run resumable full-backbone Spikingformer fine-tuning on a PYNQ overlay."""

from __future__ import annotations

import argparse
import hashlib
import json
import math
import os
import shutil
import time
from dataclasses import asdict
from datetime import datetime, timezone
from pathlib import Path
from typing import Iterator

import numpy as np

from neuraedge_pynq import NeuraEdgeDriver
from run_spikingformer_pynq import prepare_input
from spikingformer_accelerator import SpikingformerAcceleratorModel
from spikingformer_parameters import SpikingformerParameterArchive
from spikingformer_training_runtime import (
    SpikingformerOnDeviceTrainer,
    TrainableSpikingformerState,
)


def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument("--bitfile", type=Path, required=True)
    parser.add_argument("--manifest", type=Path, required=True)
    parser.add_argument("--data-manifest", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--epochs", type=int, default=400)
    parser.add_argument("--batch-size", type=int, default=1)
    parser.add_argument("--learning-rate", type=float, default=0.25)
    parser.add_argument("--minimum-learning-rate", type=float, default=0.01)
    parser.add_argument("--momentum", type=float, default=0.9)
    parser.add_argument("--weight-decay", type=float, default=5e-4)
    parser.add_argument("--gradient-clip-q15", type=int, default=4 << 15)
    parser.add_argument("--seed", type=int, default=20260713)
    parser.add_argument("--checkpoint-interval", type=int, default=1)
    parser.add_argument("--layer-timeout-seconds", type=float, default=600.0)
    parser.add_argument("--resume", action="store_true")
    parser.add_argument("--no-download", action="store_true")
    return parser.parse_args()


```

```
def atomic_json(path: Path, value: dict[str, object]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_name(path.name + ".tmp")
    temporary.write_text(json.dumps(value, indent=2) + "\n", encoding="ascii")
    os.replace(temporary, path)


def file_sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b""):
            digest.update(chunk)
    return digest.hexdigest()


class Cifar10ShardSet:
    """Validated deterministic traversal of one manifest split."""

    def __init__(self, manifest_path: Path, split: str) -> None:
        manifest_path = manifest_path.expanduser().resolve()
        manifest = json.loads(manifest_path.read_text(encoding="ascii"))
        if manifest.get("format") != "neuraedge-official-cifar10-shards-v1":
            raise ValueError("unsupported CIFAR-10 shard manifest")
        entries = manifest.get("splits", {}).get(split)
        if not isinstance(entries, list) or not entries:
            raise ValueError(f"CIFAR-10 manifest has no {split} shards")
        self.entries = tuple(entries)
        self.root = manifest_path.parent
        self.split = split
        self.sample_count = sum(int(entry["samples"]) for entry in self.entries)
        for entry in self.entries:
            path = self.root / str(entry["file"])
            if not path.is_file() or file_sha256(path) != entry["sha256"]:
                raise ValueError(f"CIFAR-10 shard integrity failure: {path}")

    def batches(
        self,
        *,
        epoch: int,
        batch_size: int,
        seed: int,
        shuffle: bool,
        start_batch: int = 0,
    ) -> Iterator[tuple[int, np.ndarray, np.ndarray]]:
        rng = np.random.default_rng(seed + epoch * 1_000_003 + (0 if self.split == "train" else 97))
        shard_order = np.arange(len(self.entries))
        if shuffle:
            rng.shuffle(shard_order)
        global_batch = 0
        for shard_index in shard_order:
            entry = self.entries[int(shard_index)]
            with np.load(self.root / str(entry["file"]), allow_pickle=False) as shard:
                images = np.asarray(shard["images"], dtype=np.uint8)
                labels = np.asarray(shard["labels"], dtype=np.int64)
            order = np.arange(len(labels))
            if shuffle:
                rng.shuffle(order)
            for start in range(0, len(order), batch_size):
                selected = order[start:start + batch_size]
                if global_batch >= start_batch:
                    yield global_batch, images[selected], labels[selected]
                global_batch += 1


def augment_cifar10(images: np.ndarray, seed: int) -> np.ndarray:
    """Deterministic reflect-pad crop and horizontal flip for one batch."""

    rng = np.random.default_rng(seed)
    output = np.empty_like(images)
    for index, image in enumerate(images):
        padded = np.pad(image, ((4, 4), (4, 4), (0, 0)), mode="reflect")
        y = int(rng.integers(0, 9))
        x = int(rng.integers(0, 9))
        crop = padded[y:y + 32, x:x + 32]
        output[index] = crop[:, ::-1] if rng.random() < 0.5 else crop
    return output


def cosine_learning_rate(
    epoch: int,
    epochs: int,
    maximum: float,
    minimum: float,
) -> float:
    if epochs == 1:
        return minimum
    phase = epoch / float(epochs - 1)
```

```
    return minimum + 0.5 * (maximum - minimum) * (1.0 + math.cos(math.pi * phase))


def evaluate(
    model: SpikingformerAcceleratorModel,
    dataset: Cifar10ShardSet,
    *,
    batch_size: int,
    seed: int,
) -> tuple[float, int, int]:
    correct = 0
    total = 0
    for _, raw_images, labels in dataset.batches(
        epoch=0, batch_size=batch_size, seed=seed, shuffle=False):
        images = prepare_input(raw_images, "raw-u8")
        prediction = model.infer(images).predicted_class
        correct += int(np.count_nonzero(prediction == labels))
        total += len(labels)
    return 100.0 * correct / total, correct, total


if __name__ == "__main__":
    arguments = parse_arguments()
    if arguments.epochs <= 0 or arguments.batch_size <= 0:
        raise ValueError("epochs and batch-size must be positive")
    if arguments.checkpoint_interval <= 0:
        raise ValueError("checkpoint-interval must be positive")
    output_dir = arguments.output_dir.expanduser().resolve()
    output_dir.mkdir(parents=True, exist_ok=True)
    pause_path = output_dir / "PAUSE"
    status_path = output_dir / "training_status.json"
    latest_path = output_dir / "latest_checkpoint.npz"
    best_path = output_dir / "best_checkpoint.npz"
    history_path = output_dir / "training_history.jsonl"

    archive = SpikingformerParameterArchive(arguments.manifest)
    state = TrainableSpikingformerState(archive)
    start_epoch = 0
    start_batch = 0
    best_accuracy = 0.0
    if arguments.resume:
        if not latest_path.is_file():
            raise FileNotFoundError(f"resume checkpoint does not exist: {latest_path}")
        metadata = state.load_checkpoint(latest_path)
        start_epoch = int(metadata["epoch"])
        start_batch = int(metadata["batch_index"])
        best_accuracy = float(metadata["best_accuracy_percent"])

    train_data = Cifar10ShardSet(arguments.data_manifest, "train")
    test_data = Cifar10ShardSet(arguments.data_manifest, "test")
    driver = NeuraEdgeDriver.from_bitfile(
        arguments.bitfile, download=not arguments.no_download)
    trainer = SpikingformerOnDeviceTrainer(
        state, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)
    inference_model = SpikingformerAcceleratorModel(
        state, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)

    job_started = time.time()
    for epoch in range(start_epoch, arguments.epochs):
        epoch_batch_start = start_batch if epoch == start_epoch else 0
        learning_rate = cosine_learning_rate(
            epoch,
            arguments.epochs,
            arguments.learning_rate,
            arguments.minimum_learning_rate,
        )
        epoch_loss = 0.0
        epoch_correct = 0
        epoch_samples = 0
        last_batch = epoch_batch_start - 1
        for batch_index, raw_images, labels in train_data.batches(
            epoch=epoch,
            batch_size=arguments.batch_size,
            seed=arguments.seed,
            shuffle=True,
            start_batch=epoch_batch_start,
        ):
            if pause_path.exists():
                state.save_checkpoint(
                    latest_path,
                    epoch=epoch,
                    batch_index=batch_index,
                    best_accuracy_percent=best_accuracy,
                    extra={"learning_rate": learning_rate, "pause_boundary": True},
                )
                atomic_json(status_path, {
                    "status": "paused",
```

```
                    "epoch": epoch,
                    "batch_index": batch_index,
                    "optimizer_step": state.optimizer_step,
                    "latest_checkpoint": str(latest_path),
                    "updated_utc": datetime.now(timezone.utc).isoformat(),
                })
                raise SystemExit(0)
            augmented = augment_cifar10(
                raw_images, arguments.seed + epoch * 10_000_019 + batch_index)
            images = prepare_input(augmented, "raw-u8")
            batch_started = time.perf_counter()
            result = trainer.train_step(
                images,
                labels,
                learning_rate=learning_rate,
                momentum=arguments.momentum,
                weight_decay=arguments.weight_decay,
                gradient_clip_q15=arguments.gradient_clip_q15,
            )
            batch_seconds = time.perf_counter() - batch_started
            epoch_loss += result.loss * len(labels)
            epoch_correct += int(np.count_nonzero(result.predicted_class == labels))
            epoch_samples += len(labels)
            last_batch = batch_index
            if state.optimizer_step % arguments.checkpoint_interval == 0:
                checkpoint_hash = state.save_checkpoint(
                    latest_path,
                    epoch=epoch,
                    batch_index=batch_index + 1,
                    best_accuracy_percent=best_accuracy,
                    extra={"learning_rate": learning_rate},
                )
            else:
                checkpoint_hash = None
            event = {
                "event": "train_batch",
                "utc": datetime.now(timezone.utc).isoformat(),
                "epoch": epoch,
                "batch_index": batch_index,
                "optimizer_step": result.optimizer_step,
                "samples": len(labels),
                "loss": result.loss,
                "batch_accuracy_percent": 100.0 * float(np.mean(result.predicted_class == labels)),
                "learning_rate": learning_rate,
                "elapsed_seconds": batch_seconds,
                "checkpoint_sha256": checkpoint_hash,
            }
            with history_path.open("a", encoding="ascii") as history:
                history.write(json.dumps(event, separators=(",", ":")) + "\n")
                history.flush()
                os.fsync(history.fileno())
            atomic_json(status_path, {
                "status": "running",
                "epoch": epoch,
                "epochs": arguments.epochs,
                "batch_index": batch_index + 1,
                "optimizer_step": state.optimizer_step,
                "running_loss": epoch_loss / epoch_samples,
                "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples,
                "best_accuracy_percent": best_accuracy,
                "last_batch_seconds": batch_seconds,
                "latest_checkpoint": str(latest_path),
                "pause_file": str(pause_path),
                "updated_utc": datetime.now(timezone.utc).isoformat(),
            })

        evaluation_accuracy, evaluation_correct, evaluation_total = evaluate(
            inference_model,
            test_data,
            batch_size=arguments.batch_size,
            seed=arguments.seed,
        )
        improved = evaluation_accuracy > best_accuracy
        if improved:
            best_accuracy = evaluation_accuracy
        state.save_checkpoint(
            latest_path,
            epoch=epoch + 1,
            batch_index=0,
            best_accuracy_percent=best_accuracy,
            extra={"learning_rate": learning_rate},
        )
        if improved:
            temporary_best = best_path.with_name(best_path.name + ".tmp")
            shutil.copyfile(latest_path, temporary_best)
            os.replace(temporary_best, best_path)
        epoch_event = {
```

```
            "event": "epoch_complete",
            "utc": datetime.now(timezone.utc).isoformat(),
            "epoch": epoch,
            "last_batch_index": last_batch,
            "train_loss": epoch_loss / max(epoch_samples, 1),
            "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1),
            "evaluation_accuracy_percent": evaluation_accuracy,
            "evaluation_correct": evaluation_correct,
            "evaluation_samples": evaluation_total,
            "best_accuracy_percent": best_accuracy,
            "optimizer_step": state.optimizer_step,
            "performance_counters": asdict(driver.performance_snapshot()),
        }
        with history_path.open("a", encoding="ascii") as history:
            history.write(json.dumps(epoch_event, separators=(",", ":")) + "\n")
            history.flush()
            os.fsync(history.fileno())
        start_batch = 0

    atomic_json(status_path, {
        "status": "complete",
        "epochs": arguments.epochs,
        "optimizer_step": state.optimizer_step,
        "best_accuracy_percent": best_accuracy,
        "elapsed_seconds": time.time() - job_started,
        "latest_checkpoint": str(latest_path),
        "best_checkpoint": str(best_path),
        "updated_utc": datetime.now(timezone.utc).isoformat(),
    })
```

45.3 Line annotations

Line 1: `"""Run resumable full-backbone Spikingformer fine-tuning on a PYNQ overlay."""` This is executable documentation attached to the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import argparse` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import hashlib` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `import json` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `import math` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import os` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `import shutil` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `import time` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `from dataclasses import asdict` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `from datetime import datetime, timezone` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `from pathlib import Path` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `from typing import Iterator` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `import numpy as np` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `from neuraedge_pynq import NeuraEdgeDriver` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `from run_spikingformer_pynq import prepare_input` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `from spikingformer_accelerator import SpikingformerAcceleratorModel` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `from spikingformer_parameters import SpikingformerParameterArchive` This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `from spikingformer_training_runtime import (` This statement continues through source line 26; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `SpikingformerOnDeviceTrainer,` This is continuation line 2 of the 4-line logical statement that begins at source line 23. Its exact fragment is ‘SpikingformerOnDeviceTrainer,‘. The important operands are SpikingformerOnDeviceTrainer: the value named ’spikingformer on device trainer’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 25: `TrainableSpikingformerState,` This is continuation line 3 of the 4-line logical statement that begins at source line 23. Its exact fragment is 'TrainableSpikingformerState,'. The important operands are TrainableSpikingformerState: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 26: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 23. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 29: `def parse_arguments() -> argparse.Namespace:` This starts function parse_arguments. Its indented body performs parse arguments as part of the file that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports; it runs only when called.

Line 30: `parser = argparse.ArgumentParser()` This assigns 'argparse.ArgumentParser()' to 'parser ' in host memory. Here parser means the value named 'parser' in this block. The important operands are argparse: the value named 'argparse' in this block; ArgumentParser: the value named 'argument parser' in this block. Context: the function parse_arguments, which handles parse arguments.

Line 31: `parser.add_argument("-bitfile", type=Path, required=True)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 32: `parser.add_argument("-manifest", type=Path, required=True)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 33: `parser.add_argument("-data-manifest", type=Path, required=True)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 34: `parser.add_argument("-output-dir", type=Path, required=True)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 35: `parser.add_argument("-epochs", type=int, default=400)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 36: `parser.add_argument("-batch-size", type=int, default=1)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 37: `parser.add_argument("-learning-rate", type=float, default=0.25)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 38: `parser.add_argument("-minimum-learning-rate", type=float, default=0.01)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 39: `parser.add_argument("-momentum", type=float, default=0.9)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 40: `parser.add_argument("-weight-decay", type=float, default=5e-4)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 41: `parser.add_argument("-gradient-clip-q15", type=int, default=4 << 15)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 42: `parser.add_argument("-seed", type=int, default=20260713)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 43: `parser.add_argument("-checkpoint-interval", type=int, default=1)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 44: `parser.add_argument("-layer-timeout-seconds", type=float, default=600.0)` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 45: `parser.add_argument("-resume", action="store_true")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 46: `parser.add_argument("-no-download", action="store_true")` This declares a command-line option for the function parse_arguments, which handles parse arguments. Its name, type, default, and required status make the run configuration explicit and reproducible instead of relying on an invisible interactive setting.

Line 47: `return parser.parse_args()` This returns ‘parser.parse_args()‘ to the caller of the function parse_arguments, which handles parse arguments, ending this invocation. The important operands are parser: the value named ’parser’ in this block; parse_args: the value named ’parse args’ in this block.

Line 50: `def atomic_json(path: Path, value: dict[str, object]) -> None:` This starts function atomic_json. Its indented body performs atomic json as part of the file that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports; it runs only when called.

Line 51: `path.parent.mkdir(parents=True, exist_ok=True)` This calls ‘path.parent.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the function atomic_json, which handles atomic json. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 52: `temporary = path.with_name(path.name + ".tmp")` This assigns ‘path.with_name(path.name + ".tmp")‘ to ‘temporary ‘ in host memory. Here temporary means the value named ’temporary’ in this block. The important operands are path: the value named ’path’ in this block; with_name: the value named ’with name’ in this block; name: the value named ’name’ in this block. Context: the function atomic_json, which handles atomic json.

Line 53: `temporary.write_text(json.dumps(value, indent=2) + "\n", encoding="ascii")` This calls ‘temporary.write_text‘ with ‘json.dumps(value, indent=2) + "\n", encoding="ascii"‘ inside the function atomic_json, which handles atomic json. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 54: `os.replace(temporary, path)` This calls ‘os.replace‘ with ‘temporary, path‘ inside the function atomic_json, which handles atomic json. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 57: `def file_sha256(path: Path) -> str:` This starts function file_sha256. Its indented body performs file sha256 as part of the file that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports; it runs only when called.

Line 58: `digest = hashlib.sha256()` This assigns ‘hashlib.sha256()‘ to ‘digest ‘ in host memory. Here digest means the value named ’digest’ in this block. The important operands are hashlib: the value named ’hashlib’ in this block; sha256: the value named ’sha256’ in this block. Context: the function file_sha256, which handles file sha256.

Line 59: `with path.open("rb") as stream:` This opens a managed resource scope for ‘path.open("rb") as stream‘ so files, tracing, or temporary state are finalized even if the function file_sha256, which handles file sha256 raises an error.

Line 60: `for chunk in iter(lambda: stream.read(1 << 20), b""):` This iterates chunk over ‘iter(lambda: stream.read(1 << 20), b"")‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function file_sha256, which handles file sha256.

Line 61: `digest.update(chunk)` This calls ‘digest.update‘ with ‘chunk‘ inside the function file_sha256, which handles file sha256. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 62: `return digest.hexdigest()` This returns ‘digest.hexdigest()‘ to the caller of the function file_sha256, which handles file sha256, ending this invocation. The important operands are digest: the value named ’digest’ in this block; hexdigest: the value named ’hexdigest’ in this block.

Line 65: `class Cifar10ShardSet:` This defines class Cifar10ShardSet, which groups data and methods for cifar10 shard set. Instances exist in host memory and organize the flow that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 66: `"""Validated deterministic traversal of one manifest split."""` This is executable documentation attached to the class Cifar10ShardSet, which handles cifar10 shard set. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 68: `def __init__(self, manifest_path: Path, split: str) -> None:` This starts function ___init___. Its indented body performs init as part of the file that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports; it runs only when called.

Line 69: `manifest_path = manifest_path.expanduser().resolve()` This assigns ‘manifest_path.expanduser().resolve()‘ to ‘manifest_path ‘ in host memory. Here manifest_path means the value named ’manifest path’ in this block. The important operands are manifest_path: the value named ’manifest path’ in this block; expanduser: the value named ’expanduser’ in this block; resolve: the value named ’resolve’ in this block. Context: the function ___init___, which handles init.

Line 70: `manifest = json.loads(manifest_path.read_text(encoding="ascii"))` This assigns ‘json.loads(manifest_path.rea to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are json: the value named ’json’ in this block; loads: the value named ’loads’ in this block; manifest_path: the value named ’manifest path’ in this block. Context: the function ___init___, which handles init.

Line 71: `if manifest.get("format") != "neuraedge-official-cifar10-shards-v1":` This executes the following indented host-code branch only when ‘manifest.get("format") != "neuraedge-official-cifar10-shards-v1"‘ is true. The important operands are manifest: the value named ’manifest’ in this block; get: the value named ’get’ in this block; format: the value named ’format’ in this block. Context: the function ___init___, which handles init.

Line 72: `raise ValueError("unsupported CIFAR-10 shard manifest")` This rejects an invalid state by raising ‘ValueError("unsupported CIFAR-10 shard manifest")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 73: `entries = manifest.get("splits", {}).get(split)` This assigns ‘manifest.get("splits", {}).get(split)‘ to ‘entries ‘ in host memory. Here entries means the value named ’entries’ in this block. The important operands are manifest: the value named ’manifest’ in this block; get: the value named ’get’ in this block; splits: the value named ’splits’ in this block. Context: the function ___init___, which handles init.

Line 74: `if not isinstance(entries, list) or not entries:` This executes the following indented host-code branch only when ‘not isinstance(entries, list) or not entries‘ is true. The important operands are not: the value named ’not’ in this block; isinstance: the value named ’isinstance’ in this block; entries: the value named ’entries’ in this block. Context: the function ___init___, which handles init.

Line 75: `raise ValueError(f"CIFAR-10 manifest has no {split} shards")` This rejects an invalid state by raising ‘ValueError(f"CIFAR-10 manifest has no {split} shards")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 76: `self.entries = tuple(entries)` This assigns ‘tuple(entries)‘ to ‘self.entries ‘ in host memory. Here entries means the value named ’entries’ in this block. The important operands are tuple: the value named ’tuple’ in this block; entries: the value named ’entries’ in this block. Context: the function ___init___, which handles init.

Line 77: `self.root = manifest_path.parent` This assigns ‘manifest_path.parent‘ to ‘self.root ‘ in host memory. Here root means the value named ’root’ in this block. The important operands are manifest_path: the value named ’manifest path’ in this block; parent: the value named ’parent’ in this block. Context: the function ___init___, which handles init.

Line 78: `self.split = split` This assigns ‘split‘ to ‘self.split ‘ in host memory. Here split means the value named ’split’ in this block. The important operands are split: the value named ’split’ in this block. Context: the function ___init___, which handles init.

Line 79: `self.sample_count = sum(int(entry["samples"]) for entry in self.entries)` This assigns ‘sum(int(entry["sam
for entry in self.entries)‘ to ‘self.sample_count ‘ in host memory. Here sample_count means an integer count retained for control or measurable telemetry. The important operands are sum: the value named ’sum’ in this block; entry: the value named ’entry’ in this block; samples: the value named ’samples’ in this block. Context: the function ___init___, which handles init.

Line 80: `for entry in self.entries:` This iterates entry over ‘self.entries‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function ___init___, which handles init.

Line 81: `path = self.root / str(entry["file"])` This assigns ‘self.root / str(entry["file"])‘ to ‘path ‘ in host memory. Here path means the value named ’path’ in this block. The important operands are root: the value named ’root’ in this block; str: the value named ’str’ in this block; entry: the value named ’entry’ in this block. Context: the function ___init___, which handles init.

Line 82: `if not path.is_file() or file_sha256(path) != entry["sha256"]:` This executes the following indented host-code branch only when ‘not path.is_file() or file_sha256(path) != entry["sha256"]‘ is true. The important operands are not: the value named ’not’ in this block; path: the value named ’path’ in this block; is_file: the value named ’is file’ in this block. Context: the function ___init___, which handles init.

Line 83: `raise ValueError(f"CIFAR-10 shard integrity failure: {path}")` This rejects an invalid state by raising ‘ValueError(f"CIFAR-10 shard integrity failure: {path}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 85: `def batches(` This starts function batches. Its indented body performs batches as part of the file that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports; it runs only when called.

Line 86: `self,` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The expression is a literal or structural operator and has no hidden software state.

Line 87: `*,` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The expression is a literal or structural operator and has no hidden software state.

Line 88: `epoch: int,` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are epoch: the value named ’epoch’ in this block.

Line 89: `batch_size: int,` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are batch_size: the value named ’batch size’ in this block.

Line 90: `seed: int,` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are seed: the value named ’seed’ in this block.

Line 91: `shuffle: bool,` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are shuffle: the value named ’shuffle’ in this block; bool: the value named ’bool’ in this block.

Line 92: `start_batch: int = 0,` This assigns ‘0,‘ to ‘start_batch‘ in host memory. Here start_batch means the value named ’start batch’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function batches, which handles batches.

Line 93: `) -> Iterator[tuple[int, np.ndarray, np.ndarray]:` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are Iterator: the value named ’iterator’ in this block; tuple: the value named ’tuple’ in this block; ndarray: the value named ’ndarray’ in this block.

Line 94: `rng = np.random.default_rng(seed + epoch * 1_000_003 + (0 if self.split == "train" else 97))` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are rng: the value named ’rng’ in this block; random: the value named ’random’ in this block; default_rng: the value named ’default rng’ in this block.

Line 95: `shard_order = np.arange(len(self.entries))` This assigns ‘np.arange(len(self.entries))‘ to ‘shard_order ‘ in host memory. Here shard_order means the value named ’shard order’ in this block. The important operands are arange: the value named ’arange’ in this block; len: the value named ’len’ in this block; entries: the value named ’entries’ in this block. Context: the function batches, which handles batches.

Line 96: `if shuffle:` This executes the following indented host-code branch only when ‘shuffle‘ is true. The important operands are shuffle: the value named ’shuffle’ in this block. Context: the function batches, which handles batches.

Line 97: `rng.shuffle(shard_order)` This calls ‘rng.shuffle‘ with ‘shard_order‘ inside the function batches, which handles batches. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 98: `global_batch = 0` This assigns ‘0‘ to ‘global_batch ‘ in host memory. Here global_batch means the value named ’global batch’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function batches, which handles batches.

Line 99: `for shard_index in shard_order:` This iterates shard_index over ‘shard_order‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function batches, which handles batches.

Line 100: `entry = self.entries[int(shard_index)]` This assigns ‘self.entries[int(shard_index)]‘ to ‘entry ‘ in host memory. Here entry means the value named ’entry’ in this block. The important operands are entries: the value named ’entries’ in this block; shard_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function batches, which handles batches.

Line 101: `with np.load(self.root / str(entry["file"]), allow_pickle=False) as shard:` This opens a managed resource scope for ‘np.load(self.root / str(entry["file"]), allow_pickle=False) as shard‘ so files, tracing, or temporary state are finalized even if the function batches, which handles batches raises an error.

Line 102: `images = np.asarray(shard["images"], dtype=np.uint8)` This assigns ‘np.asarray(shard["images"], dtype=np.uin
to ‘images ‘ in host memory. Here images means the value named ’images’ in this block. The important operands are asarray: the value named ’asarray’ in this block; shard: the value named ’shard’ in this block; images: the value named ’images’ in this block. Context: the function batches, which handles batches.

Line 103: `labels = np.asarray(shard["labels"], dtype=np.int64)` This assigns ‘np.asarray(shard["labels"], dtype=np.int6
to ‘labels ‘ in host memory. Here labels means the value named ’labels’ in this block. The important operands are asarray: the value named ’asarray’ in this block; shard: the value named ’shard’ in this block; labels: the value named ’labels’ in this block. Context: the function batches, which handles batches.

Line 104: `order = np.arange(len(labels))` This assigns ‘np.arange(len(labels))‘ to ‘order ‘ in host memory. Here order means the value named ’order’ in this block. The important operands are arange: the value named ’arange’ in this block; len: the value named ’len’ in this block; labels: the value named ’labels’ in this block. Context: the function batches, which handles batches.

Line 105: `if shuffle:` This executes the following indented host-code branch only when ‘shuffle‘ is true. The important operands are shuffle: the value named ’shuffle’ in this block. Context: the function batches, which handles batches.

Line 106: `rng.shuffle(order)` This calls ‘rng.shuffle‘ with ‘order‘ inside the function batches, which handles batches. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 107: `for start in range(0, len(order), batch_size):` This iterates start over ‘range(0, len(order), batch_size)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function batches, which handles batches.

Line 108: `selected = order[start:start + batch_size]` This assigns ‘order[start:start + batch_size]‘ to ‘selected ‘ in host memory. Here selected means the value named ’selected’ in this block. The important operands are order: the value named ’order’ in this block; start: the value named ’start’ in this block; batch_size: the value named ’batch size’ in this block. Context: the function batches, which handles batches.

Line 109: `if global_batch >= start_batch:` This executes the following indented host-code branch only when ‘global_batch >= start_batch‘ is true. The important operands are global_batch: the value named ’global batch’ in this block; start_batch: the value named ’start batch’ in this block. Context: the function batches, which handles batches.

Line 110: `yield global_batch, images[selected], labels[selected]` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are yield: the value named ’yield’ in this block; global_batch: the value named ’global batch’ in this block; images: the value named ’images’ in this block.

Line 111: `global_batch += 1` Inside the function batches, which handles batches, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are global_batch: the value named ’global batch’ in this block.

Line 114: `def augment_cifar10(images: np.ndarray, seed: int) -> np.ndarray:` This starts function augment_cifar10. Its indented body performs augment cifar10 as part of the file that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports; it runs only when called.

Line 115: `"""Deterministic reflect-pad crop and horizontal flip for one batch."""` This is executable documentation attached to the function augment_cifar10, which handles augment cifar10. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 117: `rng = np.random.default_rng(seed)` This assigns ‘np.random.default_rng(seed)‘ to ‘rng ‘ in host memory. Here rng means the value named ’rng’ in this block. The important operands are random: the value named ’random’ in this block; default_rng: the value named ’default rng’ in this block; seed: the value named ’seed’ in this block. Context: the function augment_cifar10, which handles augment cifar10.

Line 118: `output = np.empty_like(images)` This assigns ‘np.empty_like(images)‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are empty_like: the value named ’empty like’ in this block; images: the value named ’images’ in this block. Context: the function augment_cifar10, which handles augment cifar10.

Line 119: `for index, image in enumerate(images):` This iterates index, image over ‘enumerate(images)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function augment_cifar10, which handles augment cifar10.

Line 120: `padded = np.pad(image, ((4, 4), (4, 4), (0, 0)), mode="reflect")` This assigns ‘np.pad(image, ((4, 4), (4, 4), (0, 0)), mode="reflect")‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are pad: the value named ’pad’ in this block; image: the value named ’image’ in this block; mode: the value named ’mode’ in this block. Context: the function augment_cifar10, which handles augment cifar10.

Line 121: `y = int(rng.integers(0, 9))` This assigns ‘int(rng.integers(0, 9))‘ to ‘y ‘ in host memory. Here y means the value named ’y’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function augment_cifar10, which handles augment cifar10.

Line 122: `x = int(rng.integers(0, 9))` This assigns ‘int(rng.integers(0, 9))‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function augment_cifar10, which handles augment cifar10.

Line 123: `crop = padded[y:y + 32, x:x + 32]` This assigns ‘padded[y:y + 32, x:x + 32]‘ to ‘crop ‘ in host memory. Here crop means the value named ’crop’ in this block. The important operands are padded: the value named ’padded’ in this block; y: the value named ’y’ in this block; x: the value named ’x’ in this block. Context: the function augment_cifar10, which handles augment cifar10.

Line 124: `output[index] = crop[:, ::-1] if rng.random() < 0.5 else crop` This assigns ‘crop[:, ::-1] if rng.random() < 0.5 else crop‘ to ‘output[index] ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are crop: the value named ’crop’ in this block; rng: the value named ’rng’ in this block; random: the value named ’random’ in this block. Context: the function augment_cifar10, which handles augment cifar10.

Line 125: `return output` This returns ‘output‘ to the caller of the function augment_cifar10, which handles augment cifar10, ending this invocation. The important operands are output: the value named ’output’ in this block.

Line 128: `def cosine_learning_rate(` This starts function cosine_learning_rate. Its indented body performs cosine learning rate as part of the file that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports; it runs only when called.

Line 129: `epoch: int,` Inside the function cosine_learning_rate, which handles cosine learning rate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are epoch: the value named ’epoch’ in this block.

Line 130: `epochs: int,` Inside the function cosine_learning_rate, which handles cosine learning rate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are epochs: the value named ’epochs’ in this block.

Line 131: `maximum: float,` Inside the function cosine_learning_rate, which handles cosine learning rate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are maximum: the value named ’maximum’ in this block; float: the value named ’float’ in this block.

Line 132: `minimum: float,` Inside the function cosine_learning_rate, which handles cosine learning rate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are minimum: the value named ’minimum’ in this block; float: the value named ’float’ in this block.

Line 133: `) -> float:` Inside the function cosine_learning_rate, which handles cosine learning rate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are float: the value named ’float’ in this block.

Line 134: `if epochs == 1:` This executes the following indented host-code branch only when ‘epochs == 1‘ is true. The important operands are epochs: the value named ’epochs’ in this block. Context: the function cosine_learning_rate, which handles cosine learning rate.

Line 135: `return minimum` This returns ‘minimum‘ to the caller of the function cosine_learning_rate, which handles cosine learning rate, ending this invocation. The important operands are minimum: the value named ’minimum’ in this block.

Line 136: `phase = epoch / float(epochs - 1)` This assigns ‘epoch / float(epochs - 1)‘ to ‘phase ‘ in host memory. Here phase means the value named ’phase’ in this block. The important operands are epoch: the value named ’epoch’ in this block; float: the value named ’float’ in this block; epochs: the value named ’epochs’ in this block. Context: the function cosine_learning_rate, which handles cosine learning rate.

Line 137: `return minimum + 0.5 * (maximum - minimum) * (1.0 + math.cos(math.pi * phase))` This returns ‘minimum + 0.5 * (maximum - minimum) * (1.0 + math.cos(math.pi * phase))‘ to the caller of the function cosine_learning_rate, which handles cosine learning rate, ending this invocation. The important operands are minimum: the value named ’minimum’ in this block; maximum: the value named ’maximum’ in this block; math: the value named ’math’ in this block.

Line 140: `def evaluate(` This starts function evaluate. Its indented body performs evaluate as part of the file that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports; it runs only when called.

Line 141: `model: SpikingformerAcceleratorModel,` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are model: the value named ’model’ in this block; SpikingformerAcceleratorModel: the value named ’spikingformer accelerator model’ in this block.

Line 142: `dataset: Cifar10ShardSet,` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are dataset: the value named ’dataset’ in this block; Cifar10ShardSet: the value named ’cifar10 shard set’ in this block.

Line 143: `*,` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The expression is a literal or structural operator and has no hidden software state.

Line 144: `batch_size: int,` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are batch_size: the value named ’batch size’ in this block.

Line 145: `seed: int,` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are seed: the value named ’seed’ in this block.

Line 146: `) -> tuple[float, int, int]:` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are tuple: the value named ’tuple’ in this block; float: the value named ’float’ in this block.

Line 147: `correct = 0` This assigns ‘0‘ to ‘correct ‘ in host memory. Here correct means the value named ’correct’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function evaluate, which handles evaluate.

Line 148: `total = 0` This assigns ‘0‘ to ‘total ‘ in host memory. Here total means the value named ’total’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function evaluate, which handles evaluate.

Line 149: `for _, raw_images, labels in dataset.batches(` This line opens or closes a multi-line Python expression inside the function evaluate, which handles evaluate; the neighboring lines supply its arguments or elements.

Line 150: `epoch=0, batch_size=batch_size, seed=seed, shuffle=False):` This assigns ‘0, batch_size=batch_size, seed=seed, shuffle=False):‘ to ‘epoch‘ in host memory. Here epoch means the value named ’epoch’ in this block. The important operands are batch_size: the value named ’batch size’ in this block; seed: the value named ’seed’ in this block; shuffle: the value named ’shuffle’ in this block. Context: the function evaluate, which handles evaluate.

Line 151: `images = prepare_input(raw_images, "raw-u8")` This assigns ‘prepare_input(raw_images, "raw-u8")‘ to ‘images ‘ in host memory. Here images means the value named ’images’ in this block. The important operands are prepare_input: the value named ’prepare input’ in this block; raw_images: the value named ’raw images’ in this block; raw: the value named ’raw’ in this block. Context: the function evaluate, which handles evaluate.

Line 152: `prediction = model.infer(images).predicted_class` This assigns ‘model.infer(images).predicted_class‘ to ‘prediction ‘ in host memory. Here prediction means the class index with the greatest classifier score. The important operands are model: the value named ’model’ in this block; infer: the value named ’infer’ in this block; images: the value named ’images’ in this block. Context: the function evaluate, which handles evaluate.

Line 153: `correct += int(np.count_nonzero(prediction == labels))` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are correct: the value named ’correct’ in this block; count_nonzero: the value named ’count nonzero’ in this block; prediction: the class index with the greatest classifier score.

Line 154: `total += len(labels)` Inside the function evaluate, which handles evaluate, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are total: the value named ’total’ in this block; len: the value named ’len’ in this block; labels: the value named ’labels’ in this block.

Line 155: `return 100.0 * correct / total, correct, total` This returns ‘100.0 * correct / total, correct, total‘ to the caller of the function evaluate, which handles evaluate, ending this invocation. The important operands are correct: the value named ’correct’ in this block; total: the value named ’total’ in this block.

Line 158: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 159: `arguments = parse_arguments()` This assigns ‘parse_arguments()‘ to ‘arguments ‘ in host memory. Here arguments means the value named ’arguments’ in this block. The important operands are parse_arguments: the value named ’parse arguments’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 160: `if arguments.epochs <= 0 or arguments.batch_size <= 0:` This executes the following indented host-code branch only when ‘arguments.epochs <= 0 or arguments.batch_size <= 0‘ is true. The important operands are arguments: the value named ’arguments’ in this block; epochs: the value named ’epochs’ in this block; or: the value named ’or’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 161: `raise ValueError("epochs and batch-size must be positive")` This rejects an invalid state by raising ‘ValueError("epochs and batch-size must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 162: `if arguments.checkpoint_interval <= 0:` This executes the following indented host-code branch only when ‘arguments.checkpoint_interval <= 0‘ is true. The important operands are arguments: the value named ’arguments’ in this block; checkpoint_interval: the value named ’checkpoint interval’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 163: `raise ValueError("checkpoint-interval must be positive")` This rejects an invalid state by raising ‘ValueError("checkpoint-interval must be positive")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 164: `output_dir = arguments.output_dir.expanduser().resolve()` This assigns ‘arguments.output_dir.expanduser().
to ‘output_dir ‘ in host memory. Here output_dir means the value named ’output dir’ in this block. The important operands are arguments: the value named ’arguments’ in this block; output_dir: the value named ’output dir’ in this block; expanduser: the value named ’expanduser’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 165: `output_dir.mkdir(parents=True, exist_ok=True)` This calls ‘output_dir.mkdir‘ with ‘parents=True, exist_ok=True‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 166: `pause_path = output_dir / "PAUSE"` This assigns ‘output_dir / "PAUSE"‘ to ‘pause_path ‘ in host memory. Here pause_path means the value named ’pause path’ in this block. The important operands are output_dir: the value named ’output dir’ in this block; PAUSE: the value named ’pause’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 167: `status_path = output_dir / "training_status.json"` This assigns ‘output_dir / "training_status.json"‘ to ‘status_path ‘ in host memory. Here status_path means the value named ’status path’ in this block. The important operands are output_dir: the value named ’output dir’ in this block; training_status: the value named ’training status’ in this block; json: the value named ’json’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 168: `latest_path = output_dir / "latest_checkpoint.npz"` This assigns ‘output_dir / "latest_checkpoint.npz"‘ to ‘latest_path ‘ in host memory. Here latest_path means the value named ’latest path’ in this block. The important operands are output_dir: the value named ’output dir’ in this block; latest_checkpoint: the value named ’latest checkpoint’ in this block; npz: the value named ’npz’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 169: `best_path = output_dir / "best_checkpoint.npz"` This assigns ‘output_dir / "best_checkpoint.npz"‘ to ‘best_path ‘ in host memory. Here best_path means the value named ’best path’ in this block. The important operands are output_dir: the value named ’output dir’ in this block; best_checkpoint: the value named ’best checkpoint’ in this block; npz: the value named ’npz’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 170: `history_path = output_dir / "training_history.jsonl"` This assigns ‘output_dir / "training_history.jsonl"‘ to ‘history_path ‘ in host memory. Here history_path means the value named ’history path’ in this block. The important operands are output_dir: the value named ’output dir’ in this block; training_history: the value named ’training history’ in this block; jsonl: the value named ’jsonl’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 172: `archive = SpikingformerParameterArchive(arguments.manifest)` This assigns ‘SpikingformerParameterArchive(arguments.manifest)‘ to ‘archive ‘ in host memory. Here archive means the value named ’archive’ in this block. The important operands are SpikingformerParameterArchive: the value named ’spikingformer parameter archive’ in this block; arguments: the value named ’arguments’ in this block; manifest: the value named ’manifest’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 173: `state = TrainableSpikingformerState(archive)` This assigns ‘TrainableSpikingformerState(archive)‘ to ‘state ‘ in host memory. Here state means the finite-state-machine phase retained in flip-flops. The important operands are TrainableSpikingformerState: the finite-state-machine phase retained in flip-flops; archive: the value named ’archive’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 174: `start_epoch = 0` This assigns ‘0‘ to ‘start_epoch ‘ in host memory. Here start_epoch means the value named ’start epoch’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 175: `start_batch = 0` This assigns ‘0‘ to ‘start_batch ‘ in host memory. Here start_batch means the value named ’start batch’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 176: `best_accuracy = 0.0` This assigns ‘0.0‘ to ‘best_accuracy ‘ in host memory. Here best_accuracy means the value named ’best accuracy’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 177: `if arguments.resume:` This executes the following indented host-code branch only when ‘arguments.resume‘ is true. The important operands are arguments: the value named ’arguments’ in this block; resume: the value named ’resume’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 178: `if not latest_path.is_file():` This executes the following indented host-code branch only when ‘not latest_path.is_file()‘ is true. The important operands are not: the value named ’not’ in this block; latest_path: the value named ’latest path’ in this block; is_file: the value named ’is file’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 179: `raise FileNotFoundError(f"resume checkpoint does not exist: {latest_path}")` This rejects an invalid state by raising ‘FileNotFoundError(f"resume checkpoint does not exist: {latest_path}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 180: `metadata = state.load_checkpoint(latest_path)` This assigns ‘state.load_checkpoint(latest_path)‘ to ‘metadata ‘ in host memory. Here metadata means the value named ’metadata’ in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; load_checkpoint: the value named ’load checkpoint’ in this block; latest_path: the value named ’latest path’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 181: `start_epoch = int(metadata["epoch"])` This assigns ‘int(metadata["epoch"])‘ to ‘start_epoch ‘ in host memory. Here start_epoch means the value named ’start epoch’ in this block. The important operands are metadata: the value named ’metadata’ in this block; epoch: the value named ’epoch’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 182: `start_batch = int(metadata["batch_index"])` This assigns ‘int(metadata["batch_index"])‘ to ‘start_batch ‘ in host memory. Here start_batch means the value named ’start batch’ in this block. The important operands are metadata: the value named ’metadata’ in this block; batch_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 183: `best_accuracy = float(metadata["best_accuracy_percent"])` This assigns ‘float(metadata["best_accuracy_perc to ‘best_accuracy ‘ in host memory. Here best_accuracy means the value named ’best accuracy’ in this block. The important operands are float: the value named ’float’ in this block; metadata: the value named ’metadata’ in this block; best_accuracy_percent: the value named ’best accuracy percent’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 185: `train_data = Cifar10ShardSet(arguments.data_manifest, "train")` This assigns ‘Cifar10ShardSet(arguments.d "train")‘ to ‘train_data ‘ in host memory. Here train_data means the payload carried with the surrounding valid/ready handshake. The important operands are Cifar10ShardSet: the value named ’cifar10 shard set’ in this block; arguments: the value named ’arguments’ in this block; data_manifest: the value named ’data manifest’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 186: `test_data = Cifar10ShardSet(arguments.data_manifest, "test")` This assigns ‘Cifar10ShardSet(arguments.dat "test")‘ to ‘test_data ‘ in host memory. Here test_data means the payload carried with the surrounding valid/ready handshake. The important operands are Cifar10ShardSet: the value named ’cifar10 shard set’ in this block; arguments: the value named ’arguments’ in this block; data_manifest: the value named ’data manifest’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 187: `driver = NeuraEdgeDriver.from_bitfile(` This statement continues through source line 188; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘NeuraEdgeDriver.from_bitfile(arguments.bitfile, download=not arguments.no_download)‘ to ‘driver ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are NeuraEdgeDriver: the value named ’neura edge driver’ in this block; from_bitfile: the value named ’from bitfile’ in this block; arguments: the value named ’arguments’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 188: `arguments.bitfile, download=not arguments.no_download)` This is continuation line 2 of the 2-line logical statement that begins at source line 187. Its exact fragment is ‘arguments.bitfile, download=not arguments.no_download)‘. The important operands are arguments: the value named ’arguments’ in this block; bitfile: the value named ’bitfile’ in this block; download: the value named ’download’ in this block. Read together, the complete operation is: This assigns ‘NeuraEdgeDriver.from_bitfile(arguments.bitfile, download=not arguments.no_download)‘ to ‘driver ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are NeuraEdgeDriver: the value named ’neura edge driver’ in this block; from_bitfile: the value named ’from bitfile’ in this block; arguments: the value named ’arguments’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 189: `trainer = SpikingformerOnDeviceTrainer(` This statement continues through source line 190; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘SpikingformerOnDeviceTrainer(state, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)‘ to ‘trainer ‘ in host memory. Here trainer means the value named ’trainer’ in this block. The important operands are SpikingformerOnDeviceTrainer: the value named ’spikingformer on device trainer’ in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named ’driver’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 190: `state, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 189. Its exact fragment is ‘state, driver, layer_timeout_seconds=arguments The important operands are state: the finite-state-machine phase retained in flip-flops; driver: the value named ’driver’ in this block; layer_timeout_seconds: the value named ’layer timeout seconds’ in this block. Read together, the complete operation is: This assigns ‘SpikingformerOnDeviceTrainer(state, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)‘ to ‘trainer ‘ in host memory. Here trainer means the value named ’trainer’ in this block. The important operands are SpikingformerOnDeviceTrainer: the value named ’spikingformer on device trainer’ in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named ’driver’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 191: `inference_model = SpikingformerAcceleratorModel(` This statement continues through source line 192; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘SpikingformerAcceleratorModel(state, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)‘ to ‘inference_model ‘ in host memory. Here inference_model means the value named ’inference model’ in this block. The important operands are SpikingformerAcceleratorModel: the value named ’spikingformer accelerator model’ in this block; state: the finite-

state-machine phase retained in flip-flops; driver: the value named 'driver' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 192: `state, driver, layer_timeout_seconds=arguments.layer_timeout_seconds)` This is continuation line 2 of the 2-line logical statement that begins at source line 191. Its exact fragment is 'state, driver, layer_timeout_seconds=arguments The important operands are state: the finite-state-machine phase retained in flip-flops; driver: the value named 'driver' in this block; layer_timeout_seconds: the value named 'layer timeout seconds' in this block. Read together, the complete operation is: This assigns 'SpikingformerAcceleratorModel(state, driver, layer_timeout_seconds=arguments.layer_timeout_seconds) to 'inference_model ' in host memory. Here inference_model means the value named 'inference model' in this block. The important operands are SpikingformerAcceleratorModel: the value named 'spikingformer accelerator model' in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named 'driver' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 194: `job_started = time.time()` This assigns 'time.time()' to 'job_started ' in host memory. Here job_started means the value named 'job started' in this block. The important operands are time: the value named 'time' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 195: `for epoch in range(start_epoch, arguments.epochs):` This iterates epoch over 'range(start_epoch, arguments.epochs)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 196: `epoch_batch_start = start_batch if epoch == start_epoch else 0` Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are epoch_batch_start: the value named 'epoch batch start' in this block; start_batch: the value named 'start batch' in this block; epoch: the value named 'epoch' in this block.

Line 197: `learning_rate = cosine_learning_rate(` This statement continues through source line 202; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'cosine_learning_rate(epoch, arguments.epochs, arguments.learning_rate, arguments.minimum_learning_rate,)' to 'learning_rate ' in host memory. Here learning_rate means the fixed-point multiplier controlling update magnitude. The important operands are cosine_learning_rate: the fixed-point multiplier controlling update magnitude; epoch: the value named 'epoch' in this block; arguments: the value named 'arguments' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 198: `epoch,` This is continuation line 2 of the 6-line logical statement that begins at source line 197. Its exact fragment is 'epoch,'. The important operands are epoch: the value named 'epoch' in this block. Read together, the complete operation is: This assigns 'cosine_learning_rate(epoch, arguments.epochs, arguments.learning_rate, arguments.minimum_learning_rate,)' to 'learning_rate ' in host memory. Here learning_rate means the fixed-point multiplier controlling update magnitude. The important operands are cosine_learning_rate: the fixed-point multiplier controlling update magnitude; epoch: the value named 'epoch' in this block; arguments: the value named 'arguments' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 199: `arguments.epochs,` This is continuation line 3 of the 6-line logical statement that begins at source line 197. Its exact fragment is 'arguments.epochs,'. The important operands are arguments: the value named 'arguments' in this block; epochs: the value named 'epochs' in this block. Read together, the complete operation is: This assigns 'cosine_learning_rate(epoch, arguments.epochs, arguments.learning_rate, arguments.minimum_learning_rate,)' to 'learning_rate ' in host memory. Here learning_rate means the fixed-point multiplier controlling update magnitude. The important operands are cosine_learning_rate: the fixed-point multiplier controlling update magnitude; epoch: the value named 'epoch' in this block; arguments: the value named 'arguments' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 200: `arguments.learning_rate,` This is continuation line 4 of the 6-line logical statement that begins at source line 197. Its exact fragment is 'arguments.learning_rate,'. The important operands are arguments: the value named 'arguments' in this block; learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns 'cosine_learning_rate(epoch, arguments.epochs, arguments.learning_rate, arguments.minimum_learning_rate,)' to 'learning_rate ' in host memory. Here learning_rate means the fixed-point multiplier controlling update magnitude. The important operands are cosine_learning_rate: the fixed-point multiplier controlling update magnitude; epoch: the value named 'epoch' in this block; arguments: the value named 'arguments' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 201: `arguments.minimum_learning_rate,` This is continuation line 5 of the 6-line logical statement that begins at source line 197. Its exact fragment is 'arguments.minimum_learning_rate,'. The important operands are arguments: the value named 'arguments' in this block; minimum_learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns 'cosine_learning_rate(epoch, arguments.epochs, arguments.learning_rate, arguments.minimum_learning_rate,)' to 'learning_rate ' in host memory. Here learning_rate means the fixed-point multiplier controlling update magnitude. The important operands are cosine_learning_rate: the fixed-point multiplier controlling update magnitude; epoch: the value named 'epoch' in this block; arguments: the value named 'arguments' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 202: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 197. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'cosine_learning_rate(epoch, arguments.epochs, arguments.learning_rate, arguments.minimum_learning_rate,)' to 'learning_rate ' in host memory. Here learning_rate means the fixed-point multiplier controlling update magnitude. The important operands are cosine_learning_rate: the fixed-point multiplier controlling update magnitude; epoch: the value named 'epoch' in this block; arguments: the value named 'arguments' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 203: `epoch_loss = 0.0` This assigns '0.0' to 'epoch_loss ' in host memory. Here epoch_loss means the value named 'epoch loss' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 204: `epoch_correct = 0` This assigns '0' to 'epoch_correct ' in host memory. Here epoch_correct means the value named 'epoch correct' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 205: `epoch_samples = 0` This assigns '0' to 'epoch_samples ' in host memory. Here epoch_samples means the value named 'epoch samples' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 206: `last_batch = epoch_batch_start - 1` This assigns 'epoch_batch_start - 1' to 'last_batch ' in host memory. Here last_batch means the AXI-Stream marker for the final beat of a packet or command payload. The important operands are epoch_batch_start: the value named 'epoch batch start' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 207: `for batch_index, raw_images, labels in train_data.batches(` This line opens or closes a multi-line Python expression inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports; the neighboring lines supply its arguments or elements.

Line 208: `epoch=epoch,` This assigns ‘epoch,‘ to ‘epoch‘ in host memory. Here epoch means the value named ’epoch’ in this block. The important operands are epoch: the value named ’epoch’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 209: `batch_size=arguments.batch_size,` This assigns ‘arguments.batch_size,‘ to ‘batch_size‘ in host memory. Here batch_size means the value named ’batch size’ in this block. The important operands are arguments: the value named ’arguments’ in this block; batch_size: the value named ’batch size’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 210: `seed=arguments.seed,` This assigns ‘arguments.seed,‘ to ‘seed‘ in host memory. Here seed means the value named ’seed’ in this block. The important operands are arguments: the value named ’arguments’ in this block; seed: the value named ’seed’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 211: `shuffle=True,` This assigns ‘True,‘ to ‘shuffle‘ in host memory. Here shuffle means the value named ’shuffle’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 212: `start_batch=epoch_batch_start,` This assigns ‘epoch_batch_start,‘ to ‘start_batch‘ in host memory. Here start_batch means the value named ’start batch’ in this block. The important operands are epoch_batch_start: the value named ’epoch batch start’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 213: `):` Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The expression is a literal or structural operator and has no hidden software state.

Line 214: `if pause_path.exists():` This executes the following indented host-code branch only when ‘pause_path.exists()‘ is true. The important operands are pause_path: the value named ’pause path’ in this block; exists: the value named ’exists’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 215: `state.save_checkpoint(` This statement continues through source line 221; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘state.save_checkpoint‘ with ‘ latest_path, epoch=epoch, batch_index=batch_index, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate, "pause_boundary": True}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 216: `latest_path,` This is continuation line 2 of the 7-line logical statement that begins at source line 215. Its exact fragment is ‘latest_path,‘. The important operands are latest_path: the value named ’latest path’ in this block. Read together, the complete operation is: This calls ‘state.save_checkpoint‘ with ‘ latest_path, epoch=epoch, batch_index=batch_index, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate, "pause_boundary": True}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 217: `epoch=epoch,` This is continuation line 3 of the 7-line logical statement that begins at source line 215. Its exact fragment is ‘epoch=epoch,‘. The important operands are epoch: the value named ’epoch’ in this block. Read together, the complete operation is: This calls ‘state.save_checkpoint‘ with ‘ latest_path, epoch=epoch, batch_index=batch_index, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate, "pause_boundary": True}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs,

and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 218: `batch_index=batch_index,` This is continuation line 4 of the 7-line logical statement that begins at source line 215. Its exact fragment is ‘batch_index=batch_index,‘. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘state.save_checkpoint‘ with ‘ latest_path, epoch=epoch, batch_index=batch_index, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate, "pause_boundary": True}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 219: `best_accuracy_percent=best_accuracy,` This is continuation line 5 of the 7-line logical statement that begins at source line 215. Its exact fragment is ‘best_accuracy_percent=best_accuracy,‘. The important operands are best_accuracy_percent: the value named ’best accuracy percent’ in this block; best_accuracy: the value named ’best accuracy’ in this block. Read together, the complete operation is: This calls ‘state.save_checkpoint‘ with ‘ latest_path, epoch=epoch, batch_index=batch_index, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate, "pause_boundary": True}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 220: `extra={"learning_rate": learning_rate, "pause_boundary": True},` This is continuation line 6 of the 7-line logical statement that begins at source line 215. Its exact fragment is ‘extra={"learning_rate": learning_rate, "pause_boundary": True},‘. The important operands are extra: the value named ’extra’ in this block; learning_rate: the fixed-point multiplier controlling update magnitude; pause_boundary: the value named ’pause boundary’ in this block. Read together, the complete operation is: This calls ‘state.save_checkpoint‘ with ‘ latest_path, epoch=epoch, batch_index=batch_index, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate, "pause_boundary": True}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 221: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 215. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘state.save_checkpoint‘ with ‘ latest_path, epoch=epoch, batch_index=batch_index, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate, "pause_boundary": True}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 222: `atomic_json(status_path, {` This statement continues through source line 229; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘atomic_json‘ with ‘status_path, { "status": "paused", "epoch": epoch, "batch_index": batch_index, "optimizer_step": state.optimizer_step, "latest_checkpoint": str(latest_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 223: `"status": "paused",` This is continuation line 2 of the 8-line logical statement that begins at source line 222. Its exact fragment is ‘"status": "paused",‘. The important operands are status: the value named ’status’ in this block; paused: the value named ’paused’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "paused", "epoch": epoch, "batch_index": batch_index, "optimizer_step": state.optimizer_step, "latest_checkpoint": str(latest_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 224: `"epoch": epoch,` This is continuation line 3 of the 8-line logical statement that begins at source line 222. Its exact fragment is ‘"epoch": epoch,‘. The important operands are epoch: the value named ’epoch’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "paused", "epoch": epoch, "batch_index": batch_index, "optimizer_step": state.optimizer_step, "latest_checkpoint": str(latest_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 225: `"batch_index": batch_index,` This is continuation line 4 of the 8-line logical statement that begins at source line 222. Its exact fragment is ‘"batch_index": batch_index,‘. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "paused", "epoch": epoch, "batch_index": batch_index, "optimizer_step": state.optimizer_step, "latest_checkpoint": str(latest_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 226: `"optimizer_step": state.optimizer_step,` This is continuation line 5 of the 8-line logical statement that begins at source line 222. Its exact fragment is ‘"optimizer_step": state.optimizer_step,‘. The important operands are optimizer_step: the value named ’optimizer step’ in this block; state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "paused", "epoch": epoch, "batch_index": batch_index, "optimizer_step": state.optimizer_step, "latest_checkpoint": str(latest_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 227: `"latest_checkpoint": str(latest_path),` This is continuation line 6 of the 8-line logical statement that begins at source line 222. Its exact fragment is ‘"latest_checkpoint": str(latest_path),‘. The important operands are latest_checkpoint: the value named ’latest checkpoint’ in this block; str: the value named ’str’ in this block; latest_path: the value named ’latest path’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "paused", "epoch": epoch, "batch_index": batch_index, "optimizer_step": state.optimizer_step, "latest_checkpoint": str(latest_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 228: `"updated_utc": datetime.now(timezone.utc).isoformat(),` This is continuation line 7 of the 8-line logical statement that begins at source line 222. Its exact fragment is ‘"updated_utc": datetime.now(timezone.utc).isoformat(),‘. The important operands are updated_utc: the value named ’updated utc’ in this block; datetime: the value named ’datetime’ in this block; now: the value named ’now’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "paused", "epoch": epoch, "batch_index": batch_index, "optimizer_step": state.optimizer_step, "latest_checkpoint": str(latest_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 229: `})` This is continuation line 8 of the 8-line logical statement that begins at source line 222. Its exact fragment is ‘})‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "paused", "epoch": epoch, "batch_index": batch_index, "optimizer_step": state.optimizer_step, "latest_checkpoint": str(latest_path), "updated_utc": datetime.now(timezone.utc).isoform
}‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 230: `raise SystemExit(0)` This rejects an invalid state by raising ‘SystemExit(0)‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 231: `augmented = augment_cifar10(` This statement continues through source line 232; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘augment_cifar10(raw_images, arguments.seed + epoch * 10_000_019 + batch_index)‘ to ‘augmented ‘ in host memory. Here augmented means the value named ’augmented’ in this block. The important operands are augment_cifar10: the value named ’augment cifar10’ in this block; raw_images: the value named ’raw images’ in this block; arguments: the value named ’arguments’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 232: `raw_images, arguments.seed + epoch * 10_000_019 + batch_index)` This is continuation line 2 of the 2-line logical statement that begins at source line 231. Its exact fragment is ‘raw_images, arguments.seed + epoch * 10_000_019 + batch_index)‘. The important operands are raw_images: the value named ’raw images’ in this block; arguments: the value named ’arguments’ in this block; seed: the value named ’seed’ in this block. Read together, the complete operation is: This assigns ‘augment_cifar10(raw_images, arguments.seed + epoch * 10_000_019 + batch_index)‘ to ‘augmented ‘ in host memory. Here augmented means the value named ’augmented’ in this block. The important operands are augment_cifar10: the value named ’augment cifar10’ in this block; raw_images: the value named ’raw images’ in this block; arguments: the value named ’arguments’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 233: `images = prepare_input(augmented, "raw-u8")` This assigns ‘prepare_input(augmented, "raw-u8")‘ to ‘images ‘ in host memory. Here images means the value named ’images’ in this block. The important operands are prepare_input: the value named ’prepare input’ in this block; augmented: the value named ’augmented’ in this block; raw: the value named ’raw’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 234: `batch_started = time.perf_counter()` This assigns ‘time.perf_counter()‘ to ‘batch_started ‘ in host memory. Here batch_started means the value named ’batch started’ in this block. The important operands are time: the value named ’time’ in this block; perf_counter: a hardware telemetry register that accumulates measured events. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 235: `result = trainer.train_step(` This statement continues through source line 242; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘trainer.train_step(images, labels, learning_rate=learning_rate, momentum=arguments.momentum, weight_decay=arguments.weight_decay, gradient_clip_q15=argume
)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are trainer: the value named ’trainer’ in this block; train_step: the value named ’train step’ in this block; images: the value named ’images’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 236: `images,` This is continuation line 2 of the 8-line logical statement that begins at source line 235. Its exact fragment is ‘images,‘. The important operands are images: the value named ’images’ in this block. Read together, the complete operation is: This assigns ‘trainer.train_step(images, labels, learning_rate=learning_rate, momentum=arguments.momentum, weight_decay=arguments.weight_decay, gradient_clip_q15=arguments.gradient_clip_q15,
)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are trainer: the value named ’trainer’ in this block; train_step: the value named ’train step’ in this block; images: the value named ’images’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 237: `labels,` This is continuation line 3 of the 8-line logical statement that begins at source line 235. Its exact fragment is ‘labels,‘. The important operands are labels: the value named ’labels’ in this block. Read together, the complete operation is: This assigns ‘trainer.train_step(images, labels, learning_rate=learning_rate, momentum=arguments.momentum, weight_decay=arguments.weight_decay, gradient_clip_q15=arguments.gradient_clip_q15,
)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are trainer: the value named ’trainer’ in this block; train_step: the value named ’train step’ in this block; images: the value named ’images’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 238: `learning_rate=learning_rate,` This is continuation line 4 of the 8-line logical statement that begins at source line 235. Its exact fragment is 'learning_rate=learning_rate,'. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns 'trainer.train_step(images, labels, learning_rate=learning_rate, momentum=arguments.momentum, weight_decay=arguments.we gradient_clip_q15=arguments.gradient_clip_q15,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are trainer: the value named 'trainer' in this block; train_step: the value named 'train step' in this block; images: the value named 'images' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 239: `momentum=arguments.momentum,` This is continuation line 5 of the 8-line logical statement that begins at source line 235. Its exact fragment is 'momentum=arguments.momentum,'. The important operands are momentum: the value named 'momentum' in this block; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns 'trainer.train_step(images, labels, learning_rate=learning_rate, momentum=arguments.momentum, weight_decay=arguments.weight_decay, gradient_clip_q15=arguments.gradient_clip_q15,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are trainer: the value named 'trainer' in this block; train_step: the value named 'train step' in this block; images: the value named 'images' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 240: `weight_decay=arguments.weight_decay,` This is continuation line 6 of the 8-line logical statement that begins at source line 235. Its exact fragment is 'weight_decay=arguments.weight_decay,'. The important operands are weight_decay: a learned coefficient multiplied by an activation; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns 'trainer.train_step(images, labels, learning_rate=learning_rate, momentum=arguments.momentum, weight_decay=arguments.weight_decay, gradient_clip_q15=arguments.gradient_clip_q15,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are trainer: the value named 'trainer' in this block; train_step: the value named 'train step' in this block; images: the value named 'images' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 241: `gradient_clip_q15=arguments.gradient_clip_q15,` This is continuation line 7 of the 8-line logical statement that begins at source line 235. Its exact fragment is 'gradient_clip_q15=arguments.gradient_clip_q15,'. The important operands are gradient_clip_q15: the signed loss derivative used to update a learned parameter; arguments: the value named 'arguments' in this block. Read together, the complete operation is: This assigns 'trainer.train_step(images, labels, learning_rate=learning_rate, momentum=arguments.momentum, weight_decay=arguments.weight_decay, gradient_clip_q15=arguments.gradient_clip_q15,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are trainer: the value named 'trainer' in this block; train_step: the value named 'train step' in this block; images: the value named 'images' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 242: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 235. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'trainer.train_step(images, labels, learning_rate=learning_rate, momentum=arguments.momentum, weight_decay=arguments.weight_decay, gradient_clip_q15=arguments.gradient_clip_q15,)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are trainer: the value named 'trainer' in this block; train_step: the value named 'train step' in this block; images: the value named 'images' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 243: `batch_seconds = time.perf_counter() - batch_started` This assigns 'time.perf_counter() - batch_started' to 'batch_seconds ' in host memory. Here batch_seconds means the value named 'batch seconds' in this block. The important operands are time: the value named 'time' in this block; perf_counter: a hardware telemetry register that accumulates measured events; batch_started: the value named 'batch started' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 244: `epoch_loss += result.loss * len(labels)` Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the

host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are epoch_loss: the value named 'epoch loss' in this block; result: the value named 'result' in this block; loss: the value named 'loss' in this block.

Line 245: `epoch_correct += int(np.count_nonzero(result.predicted_class == labels))` Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are epoch_correct: the value named 'epoch correct' in this block; count_nonzero: the value named 'count nonzero' in this block; result: the value named 'result' in this block.

Line 246: `epoch_samples += len(labels)` Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are epoch_samples: the value named 'epoch samples' in this block; len: the value named 'len' in this block; labels: the value named 'labels' in this block.

Line 247: `last_batch = batch_index` This assigns 'batch_index' to 'last_batch ' in host memory. Here last_batch means the AXI-Stream marker for the final beat of a packet or command payload. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 248: `if state.optimizer_step % arguments.checkpoint_interval == 0:` This executes the following indented host-code branch only when 'state.optimizer_step % arguments.checkpoint_interval == 0' is true. The important operands are state: the finite-state-machine phase retained in flip-flops; optimizer_step: the value named 'optimizer step' in this block; arguments: the value named 'arguments' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 249: `checkpoint_hash = state.save_checkpoint(` This statement continues through source line 255; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'state.save_checkpoint(latest_path, epoch=epoch, batch_index=batch_index + 1, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate},)' to 'checkpoint_hash ' in host memory. Here checkpoint_hash means the value named 'checkpoint hash' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; latest_path: the value named 'latest path' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 250: `latest_path,` This is continuation line 2 of the 7-line logical statement that begins at source line 249. Its exact fragment is 'latest_path,'. The important operands are latest_path: the value named 'latest path' in this block. Read together, the complete operation is: This assigns 'state.save_checkpoint(latest_path, epoch=epoch, batch_index=batch_index + 1, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate},)' to 'checkpoint_hash ' in host memory. Here checkpoint_hash means the value named 'checkpoint hash' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; latest_path: the value named 'latest path' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 251: `epoch=epoch,` This is continuation line 3 of the 7-line logical statement that begins at source line 249. Its exact fragment is 'epoch=epoch,'. The important operands are epoch: the value named 'epoch' in this block. Read together, the complete operation is: This assigns 'state.save_checkpoint(latest_path, epoch=epoch, batch_index=batch_index + 1, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate},)' to 'checkpoint_hash ' in host memory. Here checkpoint_hash means the value named 'checkpoint hash' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; latest_path: the value named 'latest path' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 252: `batch_index=batch_index + 1,` This is continuation line 4 of the 7-line logical statement that begins at source line 249. Its exact fragment is 'batch_index=batch_index + 1,'. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'state.save_checkpoint(latest_path, epoch=epoch, batch_index=batch_index + 1, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate},)' to 'checkpoint_hash ' in host memory. Here checkpoint_hash means the value named 'checkpoint hash' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; latest_path: the value named 'latest path' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 253: `best_accuracy_percent=best_accuracy,` This is continuation line 5 of the 7-line logical statement that begins at source line 249. Its exact fragment is 'best_accuracy_percent=best_accuracy,'. The important operands are best_accuracy_percent: the value named 'best accuracy percent' in this block; best_accuracy: the value named 'best accuracy' in this block. Read together, the complete operation is: This assigns 'state.save_checkpoint(latest_path, epoch=epoch, batch_index=batch_index + 1, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate},)' to 'checkpoint_hash ' in host memory. Here checkpoint_hash means the value named 'checkpoint hash' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; latest_path: the value named 'latest path' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 254: `extra={"learning_rate": learning_rate},` This is continuation line 6 of the 7-line logical statement that begins at source line 249. Its exact fragment is 'extra={"learning_rate": learning_rate},'. The important operands are extra: the value named 'extra' in this block; learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns 'state.save_checkpoint(latest_path, epoch=epoch, batch_index=batch_index + 1, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate},)' to 'checkpoint_hash ' in host memory. Here checkpoint_hash means the value named 'checkpoint hash' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; latest_path: the value named 'latest path' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 255: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 249. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'state.save_checkpoint(latest_path, epoch=epoch, batch_index=batch_index + 1, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate},)' to 'checkpoint_hash ' in host memory. Here checkpoint_hash means the value named 'checkpoint hash' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; latest_path: the value named 'latest path' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 256: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 257: `checkpoint_hash = None` This assigns 'None' to 'checkpoint_hash ' in host memory. Here checkpoint_hash means the value named 'checkpoint hash' in this block. The important operands are None: the value named 'none' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 258: `event = {` This statement continues through source line 270; the explanation uses the complete expression rather than only the first visual fragment. Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named 'event' in this block; train_batch: the value named 'train batch' in this block; utc: the value named 'utc' in this block.

Line 259: `"event": "train_batch",` This is continuation line 2 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"event": "train_batch",‘. The important operands are event: the value named ’event’ in this block; train_batch: the value named ’train batch’ in this block. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named ’event’ in this block; train_batch: the value named ’train batch’ in this block; utc: the value named ’utc’ in this block.

Line 260: `"utc": datetime.now(timezone.utc).isoformat(),` This is continuation line 3 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"utc": datetime.now(timezone.utc).isoformat(),‘. The important operands are utc: the value named ’utc’ in this block; datetime: the value named ’datetime’ in this block; now: the value named ’now’ in this block. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named ’event’ in this block; train_batch: the value named ’train batch’ in this block; utc: the value named ’utc’ in this block.

Line 261: `"epoch": epoch,` This is continuation line 4 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"epoch": epoch,‘. The important operands are epoch: the value named ’epoch’ in this block. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named ’event’ in this block; train_batch: the value named ’train batch’ in this block; utc: the value named ’utc’ in this block.

Line 262: `"batch_index": batch_index,` This is continuation line 5 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"batch_index": batch_index,‘. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named ’event’ in this block; train_batch: the value named ’train batch’ in this block; utc: the value named ’utc’ in this block.

Line 263: `"optimizer_step": result.optimizer_step,` This is continuation line 6 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"optimizer_step": result.optimizer_step,‘. The important operands are optimizer_step: the value named ’optimizer step’ in this block; result: the value named ’result’ in this block. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named ’event’ in this block; train_batch: the value named ’train batch’ in this block; utc: the value named ’utc’ in this block.

Line 264: `"samples": len(labels),` This is continuation line 7 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"samples": len(labels),‘. The important operands are samples: the value named ’samples’ in this block; len: the value named ’len’ in this block; labels: the value named ’labels’ in this block. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named ’event’ in this block; train_batch: the value named ’train batch’ in this block; utc: the value named ’utc’ in this block.

Line 265: `"loss": result.loss,` This is continuation line 8 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"loss": result.loss,‘. The important operands are loss: the value named ’loss’ in this block; result: the value named ’result’ in this block. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports,

this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named 'event' in this block; train_batch: the value named 'train batch' in this block; utc: the value named 'utc' in this block.

Line 266: `"batch_accuracy_percent": 100.0 * float(np.mean(result.predicted_class == labels)),` This is continuation line 9 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"batch_accuracy_percent": 100.0 * float(np.mean(result.predicted_class == labels)),‘. The important operands are batch_accuracy_percent: the value named 'batch accuracy percent' in this block; float: the value named 'float' in this block; mean: the arithmetic average used by normalization. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named 'event' in this block; train_batch: the value named 'train batch' in this block; utc: the value named 'utc' in this block.

Line 267: `"learning_rate": learning_rate,` This is continuation line 10 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"learning_rate": learning_rate,‘. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named 'event' in this block; train_batch: the value named 'train batch' in this block; utc: the value named 'utc' in this block.

Line 268: `"elapsed_seconds": batch_seconds,` This is continuation line 11 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"elapsed_seconds": batch_seconds,‘. The important operands are elapsed_seconds: the value named 'elapsed seconds' in this block; batch_seconds: the value named 'batch seconds' in this block. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named 'event' in this block; train_batch: the value named 'train batch' in this block; utc: the value named 'utc' in this block.

Line 269: `"checkpoint_sha256": checkpoint_hash,` This is continuation line 12 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘"checkpoint_sha256": checkpoint_hash,‘. The important operands are checkpoint_sha256: the value named 'checkpoint sha256' in this block; checkpoint_hash: the value named 'checkpoint hash' in this block. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named 'event' in this block; train_batch: the value named 'train batch' in this block; utc: the value named 'utc' in this block.

Line 270: `}` This is continuation line 13 of the 13-line logical statement that begins at source line 258. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports, this statement advances the host implementation that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The important operands are event: the value named 'event' in this block; train_batch: the value named 'train batch' in this block; utc: the value named 'utc' in this block.

Line 271: `with history_path.open("a", encoding="ascii") as history:` This opens a managed resource scope for ‘history_path.open("a", encoding="ascii") as history‘ so files, tracing, or temporary state are finalized even if the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports raises an error.

Line 272: `history.write(json.dumps(event, separators=(",", ":")) + "\n")` This calls ‘history.write‘ with ‘json.dumps(event, separators=(",", ":")) + "\n"‘ inside the file whose role is to provides the board-training command

line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 273: `history.flush()` This calls ‘history.flush‘ with “ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 274: `os.fsync(history.fileno())` This calls ‘os.fsync‘ with ‘history.fileno()‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 275: `atomic_json(status_path, {` This statement continues through source line 288; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 276: `"status": "running",` This is continuation line 2 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"status": "running",‘. The important operands are status: the value named ’status’ in this block; running: the value named ’running’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 277: `"epoch": epoch,` This is continuation line 3 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"epoch": epoch,‘. The important operands are epoch: the value named ’epoch’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 278: `"epochs": arguments.epochs,` This is continuation line 4 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"epochs": arguments.epochs,‘. The important operands are epochs: the value named ’epochs’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 279: `"batch_index": batch_index + 1,` This is continuation line 5 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"batch_index": batch_index + 1,‘. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seco batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc). }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 280: `"optimizer_step": state.optimizer_step,` This is continuation line 6 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"optimizer_step": state.optimizer_step,‘. The important operands are optimizer_step: the value named 'optimizer step' in this block; state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 281: `"running_loss": epoch_loss / epoch_samples,` This is continuation line 7 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"running_loss": epoch_loss / epoch_samples,‘. The important operands are running_loss: the value named 'running loss' in this block; epoch_loss: the value named 'epoch loss' in this block; epoch_samples: the value named 'epoch samples' in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 282: `"running_accuracy_percent": 100.0 * epoch_correct / epoch_samples,` This is continuation line 8 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"running_accuracy_percent": 100.0 * epoch_correct / epoch_samples,‘. The important operands are running_accuracy_percent: the value named 'running accuracy percent' in this block; epoch_correct: the value named 'epoch correct' in this block; epoch_samples: the value named 'epoch samples' in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 283: `"best_accuracy_percent": best_accuracy,` This is continuation line 9 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"best_accuracy_percent": best_accuracy,‘. The important operands are best_accuracy_percent: the value named 'best accuracy percent' in this block; best_accuracy: the value named 'best accuracy' in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume,

pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 284: `"last_batch_seconds": batch_seconds,` This is continuation line 10 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"last_batch_seconds": batch_seconds,‘. The important operands are last_batch_seconds: the AXI-Stream marker for the final beat of a packet or command payload; batch_seconds: the value named ’batch seconds’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 285: `"latest_checkpoint": str(latest_path),` This is continuation line 11 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"latest_checkpoint": str(latest_path),‘. The important operands are latest_checkpoint: the value named ’latest checkpoint’ in this block; str: the value named ’str’ in this block; latest_path: the value named ’latest path’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 286: `"pause_file": str(pause_path),` This is continuation line 12 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"pause_file": str(pause_path),‘. The important operands are pause_file: the value named ’pause file’ in this block; str: the value named ’str’ in this block; pause_path: the value named ’pause path’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 287: `"updated_utc": datetime.now(timezone.utc).isoformat(),` This is continuation line 13 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘"updated_utc": datetime.now(timezone.utc).isoformat(),‘. The important operands are updated_utc: the value named ’updated utc’ in this block; datetime: the value named ’datetime’ in this block; now: the value named ’now’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 288: `})` This is continuation line 14 of the 14-line logical statement that begins at source line 275. Its exact fragment is ‘})‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "running", "epoch": epoch, "epochs": arguments.epochs, "batch_index": batch_index + 1, "optimizer_step": state.optimizer_step, "running_loss": epoch_loss / epoch_samples, "running_accuracy_percent": 100.0 * epoch_correct / epoch_samples, "best_accuracy_percent": best_accuracy, "last_batch_seconds": batch_seconds, "latest_checkpoint": str(latest_path), "pause_file": str(pause_path),

"updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 290: `evaluation_accuracy, evaluation_correct, evaluation_total = evaluate(` This statement continues through source line 295; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘evaluate(inference_model, test_data, batch_size=arguments.batch_size, seed=arguments.seed,)‘ to ‘evaluation_accuracy, evaluation_correct, evaluation_total ‘ in host memory. Here evaluation_accuracy means the value named ’evaluation accuracy’ in this block. The important operands are evaluate: the value named ’evaluate’ in this block; inference_model: the value named ’inference model’ in this block; test_data: the payload carried with the surrounding valid/ready handshake. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 291: `inference_model,` This is continuation line 2 of the 6-line logical statement that begins at source line 290. Its exact fragment is ‘inference_model,‘. The important operands are inference_model: the value named ’inference model’ in this block. Read together, the complete operation is: This assigns ‘evaluate(inference_model, test_data, batch_size=arguments.batch_size, seed=arguments.seed,)‘ to ‘evaluation_accuracy, evaluation_correct, evaluation_total ‘ in host memory. Here evaluation_accuracy means the value named ’evaluation accuracy’ in this block. The important operands are evaluate: the value named ’evaluate’ in this block; inference_model: the value named ’inference model’ in this block; test_data: the payload carried with the surrounding valid/ready handshake. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 292: `test_data,` This is continuation line 3 of the 6-line logical statement that begins at source line 290. Its exact fragment is ‘test_data,‘. The important operands are test_data: the payload carried with the surrounding valid/ready handshake. Read together, the complete operation is: This assigns ‘evaluate(inference_model, test_data, batch_size=arguments.batch_size, seed=arguments.seed,)‘ to ‘evaluation_accuracy, evaluation_correct, evaluation_total ‘ in host memory. Here evaluation_accuracy means the value named ’evaluation accuracy’ in this block. The important operands are evaluate: the value named ’evaluate’ in this block; inference_model: the value named ’inference model’ in this block; test_data: the payload carried with the surrounding valid/ready handshake. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 293: `batch_size=arguments.batch_size,` This is continuation line 4 of the 6-line logical statement that begins at source line 290. Its exact fragment is ‘batch_size=arguments.batch_size,‘. The important operands are batch_size: the value named ’batch size’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This assigns ‘evaluate(inference_model, test_data, batch_size=arguments.batch_size, seed=arguments.seed,)‘ to ‘evaluation_accuracy, evaluation_correct, evaluation_total ‘ in host memory. Here evaluation_accuracy means the value named ’evaluation accuracy’ in this block. The important operands are evaluate: the value named ’evaluate’ in this block; inference_model: the value named ’inference model’ in this block; test_data: the payload carried with the surrounding valid/ready handshake. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 294: `seed=arguments.seed,` This is continuation line 5 of the 6-line logical statement that begins at source line 290. Its exact fragment is ‘seed=arguments.seed,‘. The important operands are seed: the value named ’seed’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This assigns ‘evaluate(inference_model, test_data, batch_size=arguments.batch_size, seed=arguments.seed,)‘ to ‘evaluation_accuracy, evaluation_correct, evaluation_total ‘ in host memory. Here evaluation_accuracy means the value named ’evaluation accuracy’ in this block. The important operands are evaluate: the value named ’evaluate’ in this block; inference_model: the value named ’inference model’ in this block; test_data: the payload carried with the surrounding valid/ready handshake. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 295: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 290. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘evaluate(inference_model, test_data, batch_size=arguments.batch_size, seed=arguments.seed,)‘ to ‘evaluation_accuracy, evaluation_correct, evaluation_total ‘ in host memory. Here evaluation_accuracy means the

value named 'evaluation accuracy' in this block. The important operands are evaluate: the value named 'evaluate' in this block; inference_model: the value named 'inference model' in this block; test_data: the payload carried with the surrounding valid/ready handshake. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 296: `improved = evaluation_accuracy > best_accuracy` This assigns 'evaluation_accuracy > best_accuracy' to 'improved ' in host memory. Here improved means the value named 'improved' in this block. The important operands are evaluation_accuracy: the value named 'evaluation accuracy' in this block; best_accuracy: the value named 'best accuracy' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 297: `if improved:` This executes the following indented host-code branch only when 'improved' is true. The important operands are improved: the value named 'improved' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 298: `best_accuracy = evaluation_accuracy` This assigns 'evaluation_accuracy' to 'best_accuracy ' in host memory. Here best_accuracy means the value named 'best accuracy' in this block. The important operands are evaluation_accuracy: the value named 'evaluation accuracy' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 299: `state.save_checkpoint(` This statement continues through source line 305; the explanation uses the complete expression rather than only the first visual fragment. This calls 'state.save_checkpoint' with ' latest_path, epoch=epoch + 1, batch_index=0, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate}, ' inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 300: `latest_path,` This is continuation line 2 of the 7-line logical statement that begins at source line 299. Its exact fragment is 'latest_path,'. The important operands are latest_path: the value named 'latest path' in this block. Read together, the complete operation is: This calls 'state.save_checkpoint' with ' latest_path, epoch=epoch + 1, batch_index=0, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate}, ' inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 301: `epoch=epoch + 1,` This is continuation line 3 of the 7-line logical statement that begins at source line 299. Its exact fragment is 'epoch=epoch + 1,'. The important operands are epoch: the value named 'epoch' in this block. Read together, the complete operation is: This calls 'state.save_checkpoint' with ' latest_path, epoch=epoch + 1, batch_index=0, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate}, ' inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 302: `batch_index=0,` This is continuation line 4 of the 7-line logical statement that begins at source line 299. Its exact fragment is 'batch_index=0,'. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'state.save_checkpoint' with ' latest_path, epoch=epoch + 1, batch_index=0, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate}, ' inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 303: `best_accuracy_percent=best_accuracy,` This is continuation line 5 of the 7-line logical statement that begins at source line 299. Its exact fragment is 'best_accuracy_percent=best_accuracy,'. The important operands are best_accuracy_percent: the value named 'best accuracy percent' in this block; best_accuracy: the value named 'best accuracy' in this block. Read together, the complete operation is: This calls 'state.save_checkpoint' with ' latest_path,

epoch=epoch + 1, batch_index=0, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 304: `extra={"learning_rate": learning_rate},` This is continuation line 6 of the 7-line logical statement that begins at source line 299. Its exact fragment is ‘extra={"learning_rate": learning_rate},‘. The important operands are extra: the value named ’extra’ in this block; learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This calls ‘state.save_checkpoint‘ with ‘ latest_path, epoch=epoch + 1, batch_index=0, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 305: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 299. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘state.save_checkpoint‘ with ‘ latest_path, epoch=epoch + 1, batch_index=0, best_accuracy_percent=best_accuracy, extra={"learning_rate": learning_rate}, ‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 306: `if improved:` This executes the following indented host-code branch only when ‘improved‘ is true. The important operands are improved: the value named ’improved’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 307: `temporary_best = best_path.with_name(best_path.name + ".tmp")` This assigns ‘best_path.with_name(best_p + ".tmp")‘ to ‘temporary_best ‘ in host memory. Here temporary_best means the value named ’temporary best’ in this block. The important operands are best_path: the value named ’best path’ in this block; with_name: the value named ’with name’ in this block; name: the value named ’name’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 308: `shutil.copyfile(latest_path, temporary_best)` This calls ‘shutil.copyfile‘ with ‘latest_path, temporary_best‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 309: `os.replace(temporary_best, best_path)` This calls ‘os.replace‘ with ‘temporary_best, best_path‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 310: `epoch_event = {` This statement continues through source line 323; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isof "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percent": evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named ’epoch event’ in this block. The important operands are event: the value named ’event’ in this block; epoch_complete: the value named ’epoch complete’ in this block; utc: the value named ’utc’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 311: `"event": "epoch_complete",` This is continuation line 2 of the 14-line logical statement that begins at source line 310. Its exact fragment is ‘"event": "epoch_complete",‘. The important operands are event: the value named

'event' in this block; epoch_complete: the value named 'epoch complete' in this block. Read together, the complete operation is: This assigns '{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percent": evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }' to 'epoch_event ' in host memory. Here epoch_event means the value named 'epoch event' in this block. The important operands are event: the value named 'event' in this block; epoch_complete: the value named 'epoch complete' in this block; utc: the value named 'utc' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 312: `"utc": datetime.now(timezone.utc).isoformat(),` This is continuation line 3 of the 14-line logical statement that begins at source line 310. Its exact fragment is '"utc": datetime.now(timezone.utc).isoformat(),'. The important operands are utc: the value named 'utc' in this block; datetime: the value named 'datetime' in this block; now: the value named 'now' in this block. Read together, the complete operation is: This assigns '{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percent": evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }' to 'epoch_event ' in host memory. Here epoch_event means the value named 'epoch event' in this block. The important operands are event: the value named 'event' in this block; epoch_complete: the value named 'epoch complete' in this block; utc: the value named 'utc' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 313: `"epoch": epoch,` This is continuation line 4 of the 14-line logical statement that begins at source line 310. Its exact fragment is '"epoch": epoch,'. The important operands are epoch: the value named 'epoch' in this block. Read together, the complete operation is: This assigns '{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samp
1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percent": evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }' to 'epoch_event ' in host memory. Here epoch_event means the value named 'epoch event' in this block. The important operands are event: the value named 'event' in this block; epoch_complete: the value named 'epoch complete' in this block; utc: the value named 'utc' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 314: `"last_batch_index": last_batch,` This is continuation line 5 of the 14-line logical statement that begins at source line 310. Its exact fragment is '"last_batch_index": last_batch,'. The important operands are last_batch_index: the AXI-Stream marker for the final beat of a packet or command payload; last_batch: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This assigns '{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percent": evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }' to 'epoch_event ' in host memory. Here epoch_event means the value named 'epoch event' in this block. The important operands are event: the value named 'event' in this block; epoch_complete: the value named 'epoch complete' in this block; utc: the value named 'utc' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 315: `"train_loss": epoch_loss / max(epoch_samples, 1),` This is continuation line 6 of the 14-line logical statement that begins at source line 310. Its exact fragment is '"train_loss": epoch_loss / max(epoch_samples, 1),'. The important operands are train_loss: the value named 'train loss' in this block; epoch_loss: the value named 'epoch loss' in this block; max: the value named 'max' in this block. Read together, the complete operation is: This assigns '{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percent": evaluation_accuracy, "evaluation_correct": evaluation_correct,

"evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named 'epoch event' in this block. The important operands are event: the value named 'event' in this block; epoch_complete: the value named 'epoch complete' in this block; utc: the value named 'utc' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 316: `"train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1),` This is continuation line 7 of the 14-line logical statement that begins at source line 310. Its exact fragment is ‘"train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1),‘. The important operands are train_accuracy_percent: the value named 'train accuracy percent' in this block; epoch_correct: the value named 'epoch correct' in this block; max: the value named 'max' in this block. Read together, the complete operation is: This assigns ‘{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percen evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named 'epoch event' in this block. The important operands are event: the value named 'event' in this block; epoch_complete: the value named 'epoch complete' in this block; utc: the value named 'utc' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 317: `"evaluation_accuracy_percent": evaluation_accuracy,` This is continuation line 8 of the 14-line logical statement that begins at source line 310. Its exact fragment is ‘"evaluation_accuracy_percent": evaluation_accuracy,‘. The important operands are evaluation_accuracy_percent: the value named 'evaluation accuracy percent' in this block; evaluation_accuracy: the value named 'evaluation accuracy' in this block. Read together, the complete operation is: This assigns ‘{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percent": evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named 'epoch event' in this block. The important operands are event: the value named 'event' in this block; epoch_complete: the value named 'epoch complete' in this block; utc: the value named 'utc' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 318: `"evaluation_correct": evaluation_correct,` This is continuation line 9 of the 14-line logical statement that begins at source line 310. Its exact fragment is ‘"evaluation_correct": evaluation_correct,‘. The important operands are evaluation_correct: the value named 'evaluation correct' in this block. Read together, the complete operation is: This assigns ‘{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percent": evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named 'epoch event' in this block. The important operands are event: the value named 'event' in this block; epoch_complete: the value named 'epoch complete' in this block; utc: the value named 'utc' in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 319: `"evaluation_samples": evaluation_total,` This is continuation line 10 of the 14-line logical statement that begins at source line 310. Its exact fragment is ‘"evaluation_samples": evaluation_total,‘. The important operands are evaluation_samples: the value named 'evaluation samples' in this block; evaluation_total: the value named 'evaluation total' in this block. Read together, the complete operation is: This assigns ‘{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percen evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()),

}‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named ’epoch event’ in this block. The important operands are event: the value named ’event’ in this block; epoch_complete: the value named ’epoch complete’ in this block; utc: the value named ’utc’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 320: `"best_accuracy_percent": best_accuracy,` This is continuation line 11 of the 14-line logical statement that begins at source line 310. Its exact fragment is ‘"best_accuracy_percent": best_accuracy,‘. The important operands are best_accuracy_percent: the value named ’best accuracy percent’ in this block; best_accuracy: the value named ’best accuracy’ in this block. Read together, the complete operation is: This assigns ‘{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percen evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named ’epoch event’ in this block. The important operands are event: the value named ’event’ in this block; epoch_complete: the value named ’epoch complete’ in this block; utc: the value named ’utc’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 321: `"optimizer_step": state.optimizer_step,` This is continuation line 12 of the 14-line logical statement that begins at source line 310. Its exact fragment is ‘"optimizer_step": state.optimizer_step,‘. The important operands are optimizer_step: the value named ’optimizer step’ in this block; state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns ‘{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percen evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named ’epoch event’ in this block. The important operands are event: the value named ’event’ in this block; epoch_complete: the value named ’epoch complete’ in this block; utc: the value named ’utc’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 322: `"performance_counters": asdict(driver.performance_snapshot()),` This is continuation line 13 of the 14-line logical statement that begins at source line 310. Its exact fragment is ‘"performance_counters": asdict(driver.performance_snapshot()),‘. The important operands are performance_counters: a hardware telemetry register that accumulates measured events; asdict: the value named ’asdict’ in this block; driver: the value named ’driver’ in this block. Read together, the complete operation is: This assigns ‘{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percen evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named ’epoch event’ in this block. The important operands are event: the value named ’event’ in this block; epoch_complete: the value named ’epoch complete’ in this block; utc: the value named ’utc’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 323: `}` This is continuation line 14 of the 14-line logical statement that begins at source line 310. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "event": "epoch_complete", "utc": datetime.now(timezone.utc).isoformat(), "epoch": epoch, "last_batch_index": last_batch, "train_loss": epoch_loss / max(epoch_samples, 1), "train_accuracy_percent": 100.0 * epoch_correct / max(epoch_samples, 1), "evaluation_accuracy_percent": evaluation_accuracy, "evaluation_correct": evaluation_correct, "evaluation_samples": evaluation_total, "best_accuracy_percent": best_accuracy, "optimizer_step": state.optimizer_step, "performance_counters": asdict(driver.performance_snapshot()), }‘ to ‘epoch_event ‘ in host memory. Here epoch_event means the value named ’epoch event’ in this block. The important operands are event: the value named ’event’ in this block; epoch_complete: the value named ’epoch complete’ in this block; utc: the value named ’utc’ in this block. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 324: `with history_path.open("a", encoding="ascii") as history:` This opens a managed resource scope for ‘history_path.open("a", encoding="ascii") as history‘ so files, tracing, or temporary state are finalized even if the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports raises an error.

Line 325: `history.write(json.dumps(epoch_event, separators=(",", ":")) + "\n")` This calls ‘history.write‘ with ‘json.dumps(epoch_event, separators=(",", ":")) + "\n"‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 326: `history.flush()` This calls ‘history.flush‘ with “ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 327: `os.fsync(history.fileno())` This calls ‘os.fsync‘ with ‘history.fileno()‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 328: `start_batch = 0` This assigns ‘0‘ to ‘start_batch ‘ in host memory. Here start_batch means the value named ’start batch’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 330: `atomic_json(status_path, {` This statement continues through source line 339; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step, "best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 331: `"status": "complete",` This is continuation line 2 of the 10-line logical statement that begins at source line 330. Its exact fragment is ‘"status": "complete",‘. The important operands are status: the value named ’status’ in this block; complete: the value named ’complete’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step, "best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 332: `"epochs": arguments.epochs,` This is continuation line 3 of the 10-line logical statement that begins at source line 330. Its exact fragment is ‘"epochs": arguments.epochs,‘. The important operands are epochs: the value named ’epochs’ in this block; arguments: the value named ’arguments’ in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step, "best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 333: `"optimizer_step": state.optimizer_step,` This is continuation line 4 of the 10-line logical statement that begins at source line 330. Its exact fragment is ‘"optimizer_step": state.optimizer_step,‘. The important operands

are optimizer_step: the value named 'optimizer step' in this block; state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step, "best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 334: `"best_accuracy_percent": best_accuracy,` This is continuation line 5 of the 10-line logical statement that begins at source line 330. Its exact fragment is ‘"best_accuracy_percent": best_accuracy,‘. The important operands are best_accuracy_percent: the value named 'best accuracy percent' in this block; best_accuracy: the value named 'best accuracy' in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step, "best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 335: `"elapsed_seconds": time.time() - job_started,` This is continuation line 6 of the 10-line logical statement that begins at source line 330. Its exact fragment is ‘"elapsed_seconds": time.time() - job_started,‘. The important operands are elapsed_seconds: the value named 'elapsed seconds' in this block; time: the value named 'time' in this block; job_started: the value named 'job started' in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step, "best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 336: `"latest_checkpoint": str(latest_path),` This is continuation line 7 of the 10-line logical statement that begins at source line 330. Its exact fragment is ‘"latest_checkpoint": str(latest_path),‘. The important operands are latest_checkpoint: the value named 'latest checkpoint' in this block; str: the value named 'str' in this block; latest_path: the value named 'latest path' in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step, "best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 337: `"best_checkpoint": str(best_path),` This is continuation line 8 of the 10-line logical statement that begins at source line 330. Its exact fragment is ‘"best_checkpoint": str(best_path),‘. The important operands are best_checkpoint: the value named 'best checkpoint' in this block; str: the value named 'str' in this block; best_path: the value named 'best path' in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step, "best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 338: `"updated_utc": datetime.now(timezone.utc).isoformat(),` This is continuation line 9 of the 10-line logical statement that begins at source line 330. Its exact fragment is ‘"updated_utc": datetime.now(timezone.utc).isoformat(),‘. The important operands are updated_utc: the value named 'updated utc' in this block; datetime: the value named 'datetime' in this block; now: the value named 'now' in this block. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step,

"best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

Line 339: `})` This is continuation line 10 of the 10-line logical statement that begins at source line 330. Its exact fragment is ‘})‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘atomic_json‘ with ‘status_path, { "status": "complete", "epochs": arguments.epochs, "optimizer_step": state.optimizer_step, "best_accuracy_percent": best_accuracy, "elapsed_seconds": time.time() - job_started, "latest_checkpoint": str(latest_path), "best_checkpoint": str(best_path), "updated_utc": datetime.now(timezone.utc).isoformat(), }‘ inside the file whose role is to provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports. The call advances the host-side job that provides the board-training command line, validates artifacts, and coordinates resume, pause, epochs, and reports.

46 tb/tb_neuraedge_activation_lut8.sv

Role in the complete system: drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation.

46.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation.

Simulation-only status tb_neuraedge_activation_lut8 is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

46.2 Complete source

```
`timescale 1ns / 1ps
`default_nettype none

module tb_neuraedge_activation_lut8;
    logic clk = 1'b0;
    logic rst_n = 1'b0;
    logic clear_table = 1'b0;
    logic load_valid = 1'b0;
    logic load_ready;
    logic [7:0] load_address = 8'd0;
    logic signed [15:0] load_value = 16'sd0;
    logic input_valid = 1'b0;
    logic input_ready;
    logic [63:0] input_indices = 64'd0;
    logic output_valid;
    logic output_ready = 1'b0;
    logic [127:0] output_values;
    logic config_error;
    logic [8:0] loaded_entries;
    int errors = 0;
    int checked = 0;

    neuraedge_activation_lut8 dut (
        .clk(clk), .rst_n(rst_n), .clear_table(clear_table),
        .load_valid(load_valid), .load_ready(load_ready),
        .load_address(load_address), .load_value(load_value),
        .input_valid(input_valid), .input_ready(input_ready),
        .input_indices(input_indices), .output_valid(output_valid),
        .output_ready(output_ready), .output_values(output_values),
        .config_error(config_error), .loaded_entries(loaded_entries)
    );

    always #2.5 clk = ~clk;

    function automatic logic signed [15:0] table_function(input int address);
        int centered;
        int value;
        begin
            centered = address - 128;
            value = centered < 0 ? -(centered * centered) : centered * centered;
            table_function = 16'(value);
        end
    endfunction

    initial begin
        int seed;
        seed = 32'h1a7a2026;
        repeat (5) @(negedge clk);
        rst_n = 1'b1;

        for (int address = 0; address < 256; address++) begin
            @(negedge clk);
            if (!load_ready) begin
                errors++;
                $display("ERROR LUT load ready address=%0d", address);
            end
            load_address = 8'(address);
            load_value = table_function(address);
            load_valid = 1'b1;
            @(posedge clk);
```

```
        end
        @(negedge clk);
        load_valid = 1'b0;
        @(posedge clk);
        @(negedge clk);
        if (loaded_entries != 256) begin
            errors++;
            $display("ERROR LUT loaded entries=%0d", loaded_entries);
        end

        // Rewriting an existing address must update its value without changing
        // the unique-entry count. The count commits one cycle after acceptance.
        load_address = 8'd42;
        load_value = table_function(42);
        load_valid = 1'b1;
        @(posedge clk);
        @(negedge clk);
        load_valid = 1'b0;
        @(posedge clk);
        @(negedge clk);
        if (loaded_entries != 256) begin
            errors++;
            $display("ERROR LUT duplicate changed entries=%0d", loaded_entries);
        end

        for (int test_index = 0; test_index < 100; test_index++) begin
            while (!input_ready) @(negedge clk);
            @(negedge clk);
            for (int lane = 0; lane < 8; lane++) begin
                input_indices[lane*8 +: 8] = $urandom(seed);
            end
            input_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            input_valid = 1'b0;
            repeat (test_index % 4) @(negedge clk);
            while (!output_valid) @(negedge clk);
            if (config_error) begin
                errors++;
                $display("ERROR LUT output validity test=%0d", test_index);
            end
            for (int lane = 0; lane < 8; lane++) begin
                if ($signed(output_values[lane*16 +: 16])
                        !== table_function(input_indices[lane*8 +: 8])) begin
                    errors++;
                    $display("ERROR LUT test=%0d lane=%0d address=%0d got=%0d expected=%0d",
                             test_index, lane, input_indices[lane*8 +: 8],
                             $signed(output_values[lane*16 +: 16]),
                             table_function(input_indices[lane*8 +: 8]));
                end
            end
            checked++;
            output_ready = 1'b1;
            @(posedge clk);
            @(negedge clk);
            output_ready = 1'b0;
        end

        // Clearing the table makes missing entries explicit rather than stale.
        clear_table = 1'b1;
        @(posedge clk);
        @(negedge clk);
        clear_table = 1'b0;
        input_indices = 64'h0706050403020100;
        input_valid = 1'b1;
        @(posedge clk);
        @(negedge clk);
        input_valid = 1'b0;
        while (!output_valid) @(negedge clk);
        if (!config_error || output_values != 0 || loaded_entries != 0) begin
            errors++;
            $display("ERROR LUT clear/missing-entry behavior");
        end

        if (errors != 0) begin
            $display("tb_neuraedge_activation_lut8 FAILED errors=%0d checked=%0d", errors, checked);
            $finish;
        end else begin
            $display("tb_neuraedge_activation_lut8 PASSED checked=%0d", checked);
            $finish;
        end
    end

    initial begin
        #100_000;
        $fatal(1, "tb_neuraedge_activation_lut8 watchdog timeout");
    end
```

```
endmodule

`default_nettype wire
```

46.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module tb_neuraedge_activation_lut8;` This opens tb_neuraedge_activation_lut8;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation.

Line 5: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 6: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 7: `logic clear_table = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clear_table is the value named 'clear table' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 8: `logic load_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic load_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic [7:0] load_address = 8'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_address is the value named 'load address' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic signed [15:0] load_value = 16'sd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, load_value is the value named 'load value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic input_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic [63:0] input_indices = 64'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_indices is the value named 'input indices' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic output_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [127:0] output_values;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_values is the value named 'output values' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, config_error is the value named 'config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic [8:0] loaded_entries;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, loaded_entries is the value named 'loaded entries' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 21: `int checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 23: `neuraedge_activation_lut8 dut (` This instantiates module neuraedge_activation_lut8 as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 24: `.clk(clk), .rst_n(rst_n), .clear_table(clear_table),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n), .clear_table(clear_table'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 25: `.load_valid(load_valid), .load_ready(load_ready),` This connects the child instance's named port 'load_valid' to parent expression 'load_valid), .load_ready(load_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 26: `.load_address(load_address), .load_value(load_value),` This connects the child instance's named port 'load_address' to parent expression 'load_address), .load_value(load_value'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'load address), .load value(load value' in this block.

Line 27: `.input_valid(input_valid), .input_ready(input_ready),` This connects the child instance's named port 'input_valid' to parent expression 'input_valid), .input_ready(input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 28: `.input_indices(input_indices), .output_valid(output_valid),` This connects the child instance's named port 'input_indices' to parent expression 'input_indices), .output_valid(output_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 29: `.output_ready(output_ready), .output_values(output_values),` This connects the child instance's named port 'output_ready' to parent expression 'output_ready), .output_values(output_values'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 30: `.config_error(config_error), .loaded_entries(loaded_entries)` This connects the child instance's named port 'config_error' to parent expression 'config_error), .loaded_entries(loaded_entries'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'config error), .loaded entries(loaded entries' in this block.

Line 31: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 33: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 35: `function automatic logic signed [15:0] table_function(input int address);` Within the combinational helper table function, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; table_function: the value named 'table function' in this block.

Line 36: `int centered;` Within the combinational helper table function, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are centered: the value named 'centered' in this block.

Line 37: `int value;` Within the combinational helper table function, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are value: the value named 'value' in this block.

Line 38: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper table function; it has no arithmetic value of its own.

Line 39: `centered = address - 128;` This makes centered, the value named 'centered' in this block, equal to 'address - 128'. It is driven by combinational or procedural logic in the current evaluation. The important operands are address: the value named 'address' in this block. Context: the combinational helper table function.

Line 40: `value = centered < 0 ? -(centered * centered) : centered * centered;` This makes value, the value named 'value' in this block, equal to 'centered < 0 ? -(centered * centered) : centered * centered'. It is driven by combinational or procedural logic in the current evaluation. The important operands are centered: the value named 'centered' in this block. Context: the combinational helper table function.

Line 41: `table_function = 16'(value);` This makes table_function, the value named 'table function' in this block, equal to '16'(value)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper table function.

Line 42: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper table function; it has no arithmetic value of its own.

Line 43: `endfunction` Within the combinational helper table function, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are endfunction: the value named 'endfunction' in this block.

Line 45: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 46: `int seed;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are seed: the value named 'seed' in this block.

Line 47: `seed = 32'h1a7a2026;` This makes seed, the value named 'seed' in this block, equal to '32'h1a7a2026'. It is driven by combinational or procedural logic in the current evaluation. The important operands are h1a7a2026: the value named 'h1a7a2026' in this block. Context: the module's structural declarations.

Line 48: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 49: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 51: `for (int address = 0; address < 256; address++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 52: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 53: `if (!load_ready) begin` This condition selects the following hardware branch only when '!load_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are load_ready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 54: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 55: `$display("ERROR LUT load ready address=%0d", address);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 56: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 57: `load_address = 8'(address);` This makes load_address, the value named 'load address' in this block, equal to '8'(address)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are address: the value named 'address' in this block. Context: the module's structural declarations.

Line 58: `load_value = table_function(address);` This makes load_value, the value named 'load value' in this block, equal to 'table_function(address)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are table_function: the value named 'table function' in this block; address: the value named 'address' in this block. Context: the module's structural declarations.

Line 59: `load_valid = 1'b1;` This makes load_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 60: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 61: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 62: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 63: `load_valid = 1'b0;` This makes load_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 64: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 65: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 66: `if (loaded_entries != 256) begin` This condition selects the following hardware branch only when 'loaded_entries != 256' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are loaded_entries: the value named 'loaded entries' in this block. Context: the module's structural declarations.

Line 67: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 68: `$display("ERROR LUT loaded entries=%0d", loaded_entries);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 69: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 71: `// Rewriting an existing address must update its value without changing` Design intent: Rewriting an existing address must update its value without changing This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation.

Line 72: `// the unique-entry count. The count commits one cycle after acceptance.` Design intent: the unique-entry count. The count commits one cycle after acceptance. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation.

Line 73: `load_address = 8'd42;` This makes load_address, the value named 'load address' in this block, equal to '8'd42'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d42: the value named 'd42' in this block. Context: the module's structural declarations.

Line 74: `load_value = table_function(42);` This makes load_value, the value named 'load value' in this block, equal to 'table_function(42)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are table_function: the value named 'table function' in this block. Context: the module's structural declarations.

Line 75: `load_valid = 1'b1;` This makes load_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 76: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 77: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 78: `load_valid = 1'b0;` This makes load_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 79: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 80: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 81: `if (loaded_entries != 256) begin` This condition selects the following hardware branch only when 'loaded_entries != 256' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are loaded_entries: the value named 'loaded entries' in this block. Context: the module's structural declarations.

Line 82: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 83: `$display("ERROR LUT duplicate changed entries=%0d", loaded_entries);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 84: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 86: `for (int test_index = 0; test_index < 100; test_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 87: `while (!input_ready) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are while: the value named 'while' in this block; input_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 88: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 89: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 90: `input_indices[lane*8 +: 8] = $urandom(seed);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are input_indices: the value named 'input indices' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; urandom: the value named 'urandom' in this block.

Line 91: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 92: `input_valid = 1'b1;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 93: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 94: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 95: `input_valid = 1'b0;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 96: `repeat (test_index % 4) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; test_index: an integer selecting one tensor element, lane, bank, or state entry; negedge: the value named 'negedge' in this block.

Line 97: `while (!output_valid) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are while: the value named 'while' in this block; output_valid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 98: `if (config_error) begin` This condition selects the following hardware branch only when 'config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are config_error: the value named 'config error' in this block. Context: the module's structural declarations.

Line 99: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 100: `$display("ERROR LUT output validity test=%0d", test_index);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 101: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 102: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 103: `if ($signed(output_values[lane*16 +: 16])` This condition selects the following hardware branch only when '$signed(output_values[lane*16 +: 16]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_values: the value named 'output values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the module's structural declarations.

Line 104: `!== table_function(input_indices[lane*8 +: 8])) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are table_function: the value named 'table function' in this block; input_indices: the value named 'input indices' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 105: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 106: `$display("ERROR LUT test=%0d lane=%0d address=%0d got=%0d expected=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 107: `test_index, lane, input_indices[lane*8 +: 8],` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are test_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath; input_indices: the value named 'input indices' in this block.

Line 108: `$signed(output_values[lane*16 +: 16]),` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are output_values: the value named 'output values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 109: `table_function(input_indices[lane*8 +: 8]));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are table_function: the value named 'table function' in this block; input_indices: the value named 'input indices' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 110: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 111: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 112: `checked++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 113: `output_ready = 1'b1;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 114: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 115: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 116: `output_ready = 1'b0;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 117: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 119: `// Clearing the table makes missing entries explicit rather than stale.` Design intent: Clearing the table makes missing entries explicit rather than stale. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation.

Line 120: `clear_table = 1'b1;` This makes clear_table, the value named 'clear table' in this block, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 121: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 122: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 123: `clear_table = 1'b0;` This makes clear_table, the value named 'clear table' in this block, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 124: `input_indices = 64'h0706050403020100;` This makes input_indices, the value named 'input indices' in this block, equal to '64'h0706050403020100'. It is driven by combinational or procedural logic in the current evaluation. The important operands are h0706050403020100: the value named 'h0706050403020100' in this block. Context: the module's structural declarations.

Line 125: `input_valid = 1'b1;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 126: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 127: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 128: `input_valid = 1'b0;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 129: `while (!output_valid) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are while: the value named 'while' in this block; output_valid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 130: `if (!config_error || output_values != 0 || loaded_entries != 0) begin` This condition selects the following hardware branch only when '!config_error || output_values != 0 || loaded_entries != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are config_error: the value named 'config error' in this block; output_values: the value named 'output values' in this block; loaded_entries: the value named 'loaded entries' in this block. Context: the module's structural declarations.

Line 131: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 132: `$display("ERROR LUT clear/missing-entry behavior");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 133: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 135: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 136: `$display("tb_neuraedge_activation_lut8 FAILED errors=%0d checked=%0d", errors, checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 137: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 138: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 139: `$display("tb_neuraedge_activation_lut8 PASSED checked=%0d", checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 140: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 141: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 142: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 144: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 145: `#100_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 146: `$fatal(1, "tb_neuraedge_activation_lut8 watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 147: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 148: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 150: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge activation lut8 RTL block in simulation.

47 tb/tb_neuraedge_alpha_xnor.sv

Role in the complete system: drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation.

47.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation.

Simulation-only status tb_neuraedge_alpha_xnor is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 4 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

47.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module tb_neuraedge_alpha_xnor;
5      logic clk = 1'b0;
6      logic rst_n = 1'b0;
7      logic input_valid = 1'b0;
8      logic input_ready;
9      logic [255:0] query_spikes = '0;
10     logic [255:0] key_spikes = '0;
11     logic [8:0] active_bits = 9'd1;
12     logic [8:0] alpha_q8 = 9'd0;
13     logic output_valid;
14     logic output_ready = 1'b0;
15     logic [17:0] score_q8;
16     logic [8:0] spike_matches;
17     logic [8:0] zero_matches;
18     logic config_error;
19
20     int errors = 0;
21     int checked = 0;
22     logic held_valid = 1'b0;
23     logic [17:0] held_score;
24     logic [8:0] held_spikes;
25     logic [8:0] held_zeros;
26
27     neuraedge_alpha_xnor dut (
28         .clk(clk),
29         .rst_n(rst_n),
30         .input_valid(input_valid),
31         .input_ready(input_ready),
32         .query_spikes(query_spikes),
33         .key_spikes(key_spikes),
34         .active_bits(active_bits),
35         .alpha_q8(alpha_q8),
36         .output_valid(output_valid),
37         .output_ready(output_ready),
38         .score_q8(score_q8),
39         .spike_matches(spike_matches),
40         .zero_matches(zero_matches),
41         .config_error(config_error)
42     );
43
44     always #2.5 clk = ~clk;
45
46     task automatic run_case(
47         input logic [255:0] query_value,
48         input logic [255:0] key_value,
49         input int bit_count,
50         input int alpha_value,
51         input int stall_cycles
52     );
53         int expected_spikes;
54         int expected_zeros;
55         int expected_score;
56         begin
57             expected_spikes = 0;
58             expected_zeros = 0;
59             for (int bit_index = 0; bit_index < bit_count; bit_index++) begin
60                 if (query_value[bit_index] && key_value[bit_index]) expected_spikes++;
```

```
                if (!query_value[bit_index] && !key_value[bit_index]) expected_zeros++;
            end
            expected_score = expected_spikes * 256 + expected_zeros * alpha_value;

            while (!input_ready) @(negedge clk);
            @(negedge clk);
            query_spikes = query_value;
            key_spikes = key_value;
            active_bits = 9'(bit_count);
            alpha_q8 = 9'(alpha_value);
            input_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            input_valid = 1'b0;

            while (!output_valid) @(negedge clk);
            repeat (stall_cycles) @(negedge clk);
            output_ready = 1'b1;
            @(posedge clk);
            if (config_error || spike_matches !== expected_spikes
                    || zero_matches !== expected_zeros || score_q8 !== expected_score) begin
                errors++;
                $display("ERROR alpha-XNOR bits=%0d alpha=%0d got=(%0d,%0d,%0d) expected=(%0d,%0d,%0d)",
                         bit_count, alpha_value, spike_matches, zero_matches, score_q8,
                         expected_spikes, expected_zeros, expected_score);
            end
            checked++;
            @(negedge clk);
            output_ready = 1'b0;
        end
    endtask

    always @(posedge clk) begin
        if (held_valid && (!output_valid || score_q8 !== held_score
                || spike_matches !== held_spikes || zero_matches !== held_zeros)) begin
            errors++;
            $display("ERROR alpha-XNOR output changed under backpressure");
        end
        held_valid <= output_valid && !output_ready;
        held_score <= score_q8;
        held_spikes <= spike_matches;
        held_zeros <= zero_matches;
    end

    initial begin
        int seed;
        logic [255:0] query_random;
        logic [255:0] key_random;
        int boundary_bits [0:7];
        int boundary_alpha [0:4];
        seed = 32'h51a7c0de;
        boundary_bits = '{1, 31, 32, 33, 127, 128, 255, 256};
        boundary_alpha = '{0, 1, 127, 255, 256};

        repeat (5) @(negedge clk);
        rst_n = 1'b1;

        for (int boundary = 0; boundary < 8; boundary++) begin
            query_random = '0;
            key_random = '0;
            for (int word_index = 0; word_index < 8; word_index++) begin
                query_random[word_index*32 +: 32] = $urandom(seed);
                key_random[word_index*32 +: 32] = $urandom(seed);
            end
            for (int alpha_index = 0; alpha_index < 5; alpha_index++) begin
                run_case(query_random, key_random, boundary_bits[boundary],
                         boundary_alpha[alpha_index], (boundary + alpha_index) % 4);
            end
        end

        for (int random_case = 0; random_case < 100; random_case++) begin
            for (int word_index = 0; word_index < 8; word_index++) begin
                query_random[word_index*32 +: 32] = $urandom(seed);
                key_random[word_index*32 +: 32] = $urandom(seed);
            end
            run_case(query_random, key_random, 1 + ($urandom(seed) % 256),
                     $urandom(seed) % 257, random_case % 3);
        end

        // An illegal active length must produce a flagged, deterministic result.
        while (!input_ready) @(negedge clk);
        @(negedge clk);
        active_bits = 9'd0;
        alpha_q8 = 9'd0;
        input_valid = 1'b1;
        @(posedge clk);
        @(negedge clk);
```

```
        input_valid = 1'b0;
        while (!output_valid) @(negedge clk);
        if (!config_error || score_q8 != 0) begin
            errors++;
            $display("ERROR alpha-XNOR illegal configuration was not rejected");
        end
        output_ready = 1'b1;
        @(posedge clk);
        @(negedge clk);
        output_ready = 1'b0;

        if (errors != 0) begin
            $display("tb_neuraedge_alpha_xnor FAILED errors=%0d checked=%0d", errors, checked);
            $fatal(1);
        end else begin
            $display("tb_neuraedge_alpha_xnor PASSED checked=%0d", checked);
            $finish;
        end
    end

    initial begin
        #100_000;
        $fatal(1, "tb_neuraedge_alpha_xnor watchdog timeout");
    end
endmodule

`default_nettype wire
```

47.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module tb_neuraedge_alpha_xnor;` This opens tb_neuraedge_alpha_xnor;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation.

Line 5: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 6: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 7: `logic input_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 8: `logic input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic [255:0] query_spikes = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, query_spikes is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic [255:0] key_spikes = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, key_spikes is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic [8:0] active_bits = 9'd1;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_bits is the value named 'active bits' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic [8:0] alpha_q8 = 9'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_q8 is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic output_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic [17:0] score_q8;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, score_q8 is the value named 'score q8' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic [8:0] spike_matches;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, spike_matches is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [8:0] zero_matches;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, zero_matches is the value named 'zero matches' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, config_error is the value named 'config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 21: `int checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 22: `logic held_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic [17:0] held_score;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_score is the value named 'held score' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic [8:0] held_spikes;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_spikes is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic [8:0] held_zeros;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_zeros is the value named 'held zeros' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `neuraedge_alpha_xnor dut (` This instantiates module neuraedge_alpha_xnor as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 28: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 29: `.rst_n(rst_n),` This connects the child instance's named port 'rst_n' to parent expression 'rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 30: `.input_valid(input_valid),` This connects the child instance's named port 'input_valid' to parent expression 'input_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 31: `.input_ready(input_ready),` This connects the child instance's named port 'input_ready' to parent expression 'input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 32: `.query_spikes(query_spikes),` This connects the child instance's named port 'query_spikes' to parent expression 'query_spikes'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 33: `.key_spikes(key_spikes),` This connects the child instance's named port 'key_spikes' to parent expression 'key_spikes'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 34: `.active_bits(active_bits),` This connects the child instance's named port 'active_bits' to parent expression 'active_bits'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'active bits' in this block.

Line 35: `.alpha_q8(alpha_q8),` This connects the child instance's named port 'alpha_q8' to parent expression 'alpha_q8'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 36: `.output_valid(output_valid),` This connects the child instance's named port 'output_valid' to parent expression 'output_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 37: `.output_ready(output_ready),` This connects the child instance's named port 'output_ready' to parent expression 'output_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 38: `.score_q8(score_q8),` This connects the child instance's named port 'score_q8' to parent expression 'score_q8'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'score q8' in this block.

Line 39: `.spike_matches(spike_matches),` This connects the child instance's named port 'spike_matches' to parent expression 'spike_matches'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 40: `.zero_matches(zero_matches),` This connects the child instance's named port 'zero_matches' to parent expression 'zero_matches'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'zero matches' in this block.

Line 41: `.config_error(config_error)` This connects the child instance's named port 'config_error' to parent expression 'config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'config error' in this block.

Line 42: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 44: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 46: `task automatic run_case(` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; run_case: the value named 'run case' in this block.

Line 47: `input logic [255:0] query_value,` This declares query_value as a input port with width [255:0]. It carries the value named 'query value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 48: `input logic [255:0] key_value,` This declares key_value as a input port with width [255:0]. It carries the value named 'key value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 49: `input int bit_count,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 50: `input int alpha_value,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 51: `input int stall_cycles` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 52: `);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 53: `int expected_spikes;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are expected_spikes: a binary event produced when a neuron crosses its threshold.

Line 54: `int expected_zeros;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are expected_zeros: the value named 'expected zeros' in this block.

Line 55: `int expected_score;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are expected_score: the value named 'expected score' in this block.

Line 56: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 57: `expected_spikes = 0;` This makes expected_spikes, a binary event produced when a neuron crosses its threshold, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 58: `expected_zeros = 0;` This makes expected_zeros, the value named 'expected zeros' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 59: `for (int bit_index = 0; bit_index < bit_count; bit_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 60: `if (query_value[bit_index] && key_value[bit_index]) expected_spikes++;` This condition selects the following hardware branch only when 'query_value[bit_index] && key_value[bit_index]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are query_value: the value named 'query value' in this block; bit_index: an integer selecting one tensor element, lane, bank, or state entry; key_value: the value named 'key value' in this block. Context: the simulation task run case.

Line 61: `if (!query_value[bit_index] && !key_value[bit_index]) expected_zeros++;` This condition selects the following hardware branch only when '!query_value[bit_index] && !key_value[bit_index]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are query_value: the value named 'query value' in this block; bit_index: an integer selecting one tensor element, lane, bank, or state entry; key_value: the value named 'key value' in this block. Context: the simulation task run case.

Line 62: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 63: `expected_score = expected_spikes * 256 + expected_zeros * alpha_value;` This makes expected_score, the value named 'expected score' in this block, equal to 'expected_spikes * 256 + expected_zeros * alpha_value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are expected_spikes: a binary event produced when a neuron crosses its threshold; expected_zeros: the value named 'expected zeros' in this block; alpha_value: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the simulation task run case.

Line 65: `while (!input_ready) @(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are while: the value named 'while' in this block; input_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 66: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 67: `query_spikes = query_value;` This makes query_spikes, a binary event produced when a neuron crosses its threshold, equal to 'query_value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are query_value: the value named 'query value' in this block. Context: the simulation task run case.

Line 68: `key_spikes = key_value;` This makes key_spikes, a binary event produced when a neuron crosses its threshold, equal to 'key_value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are key_value: the value named 'key value' in this block. Context: the simulation task run case.

Line 69: `active_bits = 9'(bit_count);` This makes active_bits, the value named 'active bits' in this block, equal to '9'(bit_count)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are bit_count: an integer count retained for control or measurable telemetry. Context: the simulation task run case.

Line 70: `alpha_q8 = 9'(alpha_value);` This makes alpha_q8, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to '9'(alpha_value)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are alpha_value: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the simulation task run case.

Line 71: `input_valid = 1'b1;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task run case.

Line 72: `@(posedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 73: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 74: `input_valid = 1'b0;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run case.

Line 76: `while (!output_valid) @(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are while: the value named 'while' in this block; output_valid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 77: `repeat (stall_cycles) @(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; stall_cycles: the value named 'stall cycles' in this block; negedge: the value named 'negedge' in this block.

Line 78: `output_ready = 1'b1;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task run case.

Line 79: `@(posedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 80: `if (config_error || spike_matches !== expected_spikes` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are config_error: the value named 'config error' in this block; spike_matches: a binary event produced when a neuron crosses its threshold; expected_spikes: a binary event produced when a neuron crosses its threshold.

Line 81: `|| zero_matches !== expected_zeros || score_q8 !== expected_score) begin` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are zero_matches: the value named 'zero matches' in this block; expected_zeros: the value named 'expected zeros' in this block; score_q8: the value named 'score q8' in this block.

Line 82: `errors++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 83: `$display("ERROR alpha-XNOR bits=%0d alpha=%0d got=(%0d,%0d,%0d) expected=(%0d,%0d,%0d)",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 84: `bit_count, alpha_value, spike_matches, zero_matches, score_q8,` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are bit_count: an integer count retained for control or measurable telemetry; alpha_value: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; spike_matches: a binary event produced when a neuron crosses its threshold.

Line 85: `expected_spikes, expected_zeros, expected_score);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are expected_spikes: a binary event produced when a neuron crosses its threshold; expected_zeros: the value named 'expected zeros' in this block; expected_score: the value named 'expected score' in this block.

Line 86: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 87: `checked++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 88: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 89: `output_ready = 1'b0;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run case.

Line 90: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 91: `endtask` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 93: `always @(posedge clk) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are always: the value named 'always' in this block; posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 94: `if (held_valid && (!output_valid || score_q8 !== held_score` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are held_valid: the producer's statement that its current payload is meaningful; output_valid: the producer's statement that its current payload is meaningful; score_q8: the value named 'score q8' in this block.

Line 95: `|| spike_matches !== held_spikes || zero_matches !== held_zeros)) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are spike_matches: a binary event produced when a neuron crosses its threshold; held_spikes: a binary event produced when a neuron crosses its threshold; zero_matches: the value named 'zero matches' in this block.

Line 96: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 97: `$display("ERROR alpha-XNOR output changed under backpressure");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 98: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 99: `held_valid <= output_valid && !output_ready;` This makes held_valid, the producer's statement that its current payload is meaningful, equal to 'output_valid && !output_ready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_valid: the producer's statement that its current payload is meaningful; output_ready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 100: `held_score <= score_q8;` This makes held_score, the value named 'held score' in this block, equal to 'score_q8'. It updates after the active clock edge together with other nonblocking assignments. The important operands are score_q8: the value named 'score q8' in this block. Context: the module's structural declarations.

Line 101: `held_spikes <= spike_matches;` This makes held_spikes, a binary event produced when a neuron crosses its threshold, equal to 'spike_matches'. It updates after the active clock edge together with other nonblocking assignments. The important operands are spike_matches: a binary event produced when a neuron crosses its threshold. Context: the module's structural declarations.

Line 102: `held_zeros <= zero_matches;` This makes held_zeros, the value named 'held zeros' in this block, equal to 'zero_matches'. It updates after the active clock edge together with other nonblocking assignments. The important operands are zero_matches: the value named 'zero matches' in this block. Context: the module's structural declarations.

Line 103: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 105: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 106: `int seed;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are seed: the value named 'seed' in this block.

Line 107: `logic [255:0] query_random;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, query_random is the value named 'query random' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 108: `logic [255:0] key_random;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, key_random is the value named 'key random' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 109: `int boundary_bits [0:7];` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are boundary_bits: the value named 'boundary bits' in this block.

Line 110: `int boundary_alpha [0:4];` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are boundary_alpha: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 111: `seed = 32'h51a7c0de;` This makes seed, the value named 'seed' in this block, equal to '32'h51a7c0de'. It is driven by combinational or procedural logic in the current evaluation. The important operands are h51a7c0de: the value named 'h51a7c0de' in this block. Context: the module's structural declarations.

Line 112: `boundary_bits = '{1, 31, 32, 33, 127, 128, 255, 256};` This makes boundary_bits, the value named 'boundary bits' in this block, equal to ''{1, 31, 32, 33, 127, 128, 255, 256}'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 113: `boundary_alpha = '{0, 1, 127, 255, 256};` This makes boundary_alpha, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to ''{0, 1, 127, 255, 256}'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 115: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 116: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 118: `for (int boundary = 0; boundary < 8; boundary++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 119: `query_random = '0;` This makes query_random, the value named 'query random' in this block, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 120: `key_random = '0;` This makes key_random, the value named 'key random' in this block, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 121: `for (int word_index = 0; word_index < 8; word_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 122: `query_random[word_index*32 +:  32] = $urandom(seed);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are query_random: the value named 'query random' in this block; word_index: an integer selecting one tensor element, lane, bank, or state entry; urandom: the value named 'urandom' in this block.

Line 123: `key_random[word_index*32 +: 32] = $urandom(seed);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are key_random: the value named 'key random' in this block; word_index: an integer selecting one tensor element, lane, bank, or state entry; urandom: the value named 'urandom' in this block.

Line 124: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 125: `for (int alpha_index = 0; alpha_index < 5; alpha_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 126: `run_case(query_random, key_random, boundary_bits[boundary],` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are run_case: the value named 'run case' in this block; query_random: the value named 'query random' in this block; key_random: the value named 'key random' in this block.

Line 127: `boundary_alpha[alpha_index], (boundary + alpha_index) % 4);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are boundary_alpha: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; alpha_index: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; boundary: the value named 'boundary' in this block.

Line 128: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 129: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 131: `for (int random_case = 0; random_case < 100; random_case++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 132: `for (int word_index = 0; word_index < 8; word_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 133: `query_random[word_index*32 +: 32] = $urandom(seed);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are query_random: the value named 'query random' in this block; word_index: an integer selecting one tensor element, lane, bank, or state entry; urandom: the value named 'urandom' in this block.

Line 134: `key_random[word_index*32 +: 32] = $urandom(seed);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are key_random: the value named 'key random' in this block; word_index: an integer selecting one tensor element, lane, bank, or state entry; urandom: the value named 'urandom' in this block.

Line 135: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 136: `run_case(query_random, key_random, 1 + ($urandom(seed) % 256),` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are run_case: the value named 'run case' in this block; query_random: the value named 'query random' in this block; key_random: the value named 'key random' in this block.

Line 137: `$urandom(seed) % 257, random_case % 3);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are urandom: the value named 'urandom' in this block; seed: the value named 'seed' in this block; random_case: the value named 'random case' in this block.

Line 138: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 140: `// An illegal active length must produce a flagged, deterministic result.` Design intent: An illegal active length must produce a flagged, deterministic result. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation.

Line 141: `while (!input_ready) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are while: the value named 'while' in this block; input_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 142: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 143: `active_bits = 9'd0;` This makes active_bits, the value named 'active bits' in this block, equal to '9'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 144: `alpha_q8 = 9'd0;` This makes alpha_q8, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to '9'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 145: `input_valid = 1'b1;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 146: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 147: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 148: `input_valid = 1'b0;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 149: `while (!output_valid) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are while: the value named 'while' in this block; output_valid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 150: `if (!config_error || score_q8 != 0) begin` This condition selects the following hardware branch only when '!config_error || score_q8 != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are config_error: the value named 'config error' in this block; score_q8: the value named 'score q8' in this block. Context: the module's structural declarations.

Line 151: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 152: `$display("ERROR alpha-XNOR illegal configuration was not rejected");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 153: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 154: `output_ready = 1'b1;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 155: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 156: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 157: `output_ready = 1'b0;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 159: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 160: `$display("tb_neuraedge_alpha_xnor FAILED errors=%0d checked=%0d", errors, checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 161: `$fatal(1);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 162: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 163: `$display("tb_neuraedge_alpha_xnor PASSED checked=%0d", checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 164: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 165: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 166: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 168: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 169: `#100_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 170: `$fatal(1, "tb_neuraedge_alpha_xnor watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 171: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 172: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 174: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge alpha xnor RTL block in simulation.

48 tb/tb_neuraedge_buffered_tensor_core.sv

Role in the complete system: drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation.

48.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation.

Simulation-only status tb_neuraedge_buffered_tensor_core is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 4 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

48.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3  `include "tb/fixtures/tensor_tile_params.svh"
4
5  module tb_neuraedge_buffered_tensor_core;
6      import neuraedge_command_pkg::*;
7
8      localparam int CASES = `NEURAEDGE_TENSOR_CASES;
9      localparam int ISSUES = `NEURAEDGE_TENSOR_ISSUES;
10     localparam int RESULTS = `NEURAEDGE_TENSOR_RESULTS;
11     localparam int COLUMN_LANES = `NEURAEDGE_COLUMN_LANES;
12
13     logic clk = 1'b0;
14     logic rst_n = 1'b0;
15     logic command_valid = 1'b0;
16     logic command_ready;
17     logic [127:0] command_data = 128'd0;
18     logic command_done;
19     logic status_error;
20     logic status_busy;
21     logic s_axis_tvalid = 1'b0;
22     logic s_axis_tready;
23     logic [63:0] s_axis_tdata = 64'd0;
24     logic s_axis_tlast = 1'b0;
25     logic m_axis_tvalid;
26     logic m_axis_tready = 1'b0;
27     logic [63:0] m_axis_tdata;
28     logic m_axis_tlast;
29     logic [11:0] status_loaded_activations;
30     logic [11:0] status_loaded_weights;
31     logic [11:0] status_active_issues;
32     logic [31:0] status_input_beats;
33     logic [31:0] status_output_beats;
34     logic [31:0] status_compute_cycles;
35     logic [31:0] status_skipped_issues;
36     logic [63:0] status_dense_macs;
37     logic [63:0] status_spike_adds;
38
39     logic [63:0] case_mem [0:CASES-1];
40     logic [63:0] activation_mem [0:ISSUES-1];
41     logic [COLUMN_LANES*8*8-1:0] weight_mem [0:ISSUES-1];
42     logic signed [63:0] expected_mem [0:RESULTS-1];
43
44     int issue_pointer = 0;
45     int expected_pointer = 0;
46     int checked = 0;
47     int errors = 0;
48     int cycle = 0;
49     logic held_valid = 1'b0;
50     logic [63:0] held_data;
51     logic held_last;
52
53     neuraedge_buffered_tensor_core dut (
54         .clk(clk),
55         .rst_n(rst_n),
56         .command_valid(command_valid),
57         .command_ready(command_ready),
```

```
        .command_data(command_data[41:0]),
        .command_done(command_done),
        .status_error(status_error),
        .status_busy(status_busy),
        .s_axis_tvalid(s_axis_tvalid),
        .s_axis_tready(s_axis_tready),
        .s_axis_tdata(s_axis_tdata),
        .s_axis_tlast(s_axis_tlast),
        .m_axis_tvalid(m_axis_tvalid),
        .m_axis_tready(m_axis_tready),
        .m_axis_tdata(m_axis_tdata),
        .m_axis_tlast(m_axis_tlast),
        .status_loaded_activations(status_loaded_activations),
        .status_loaded_weights(status_loaded_weights),
        .status_active_issues(status_active_issues),
        .status_input_beats(status_input_beats),
        .status_output_beats(status_output_beats),
        .status_compute_cycles(status_compute_cycles),
        .status_skipped_issues(status_skipped_issues),
        .status_dense_macs(status_dense_macs),
        .status_spike_adds(status_spike_adds)
    );

    always #2.5 clk = ~clk;

    function automatic logic [127:0] make_command(
        input logic [3:0] opcode,
        input int rows,
        input int columns,
        input int issue_count,
        input int activation_zp,
        input int weight_zp
    );
        logic [127:0] value;
        begin
            value = 128'd0;
            value[3:0] = opcode;
            value[7:4] = 4'(rows);
            value[13:8] = 6'(columns);
            value[25:14] = 12'(issue_count);
            value[33:26] = 8'(activation_zp);
            value[41:34] = 8'(weight_zp);
            make_command = value;
        end
    endfunction

    function automatic logic activation_is_zero(
        input logic [63:0] values,
        input logic [7:0] zero_point
    );
        logic is_zero;
        begin
            is_zero = 1'b1;
            for (int lane = 0; lane < 8; lane++) begin
                is_zero = is_zero && values[lane*8 +: 8] == zero_point;
            end
            activation_is_zero = is_zero;
        end
    endfunction

    task automatic submit_command(input logic [127:0] value);
        begin
            while (!command_ready) @(negedge clk);
            @(negedge clk);
            command_data = value;
            command_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            command_valid = 1'b0;
            command_data = 128'd0;
        end
    endtask

    task automatic stream_activation_case(
        input int first_issue,
        input int issue_count
    );
        begin
            for (int issue = 0; issue < issue_count; issue++) begin
                @(negedge clk);
                s_axis_tvalid = 1'b1;
                s_axis_tdata = activation_mem[first_issue + issue];
                s_axis_tlast = (issue == issue_count - 1);
                while (!s_axis_tready) @(negedge clk);
                @(posedge clk);
            end
            @(negedge clk);
```

```
            s_axis_tvalid = 1'b0;
            s_axis_tlast = 1'b0;
            s_axis_tdata = 64'd0;
        end
    endtask

    task automatic stream_weight_case(
        input int first_issue,
        input int issue_count,
        input logic dot8
    );
        int beats_per_issue;
        begin
            beats_per_issue = dot8 ? COLUMN_LANES : COLUMN_LANES / 8;
            for (int issue = 0; issue < issue_count; issue++) begin
                for (int beat = 0; beat < beats_per_issue; beat++) begin
                    @(negedge clk);
                    s_axis_tvalid = 1'b1;
                    s_axis_tdata = weight_mem[first_issue + issue][beat*64 +: 64];
                    s_axis_tlast = issue == issue_count - 1 && beat == beats_per_issue - 1;
                    while (!s_axis_tready) @(negedge clk);
                    @(posedge clk);
                end
            end
            @(negedge clk);
            s_axis_tvalid = 1'b0;
            s_axis_tlast = 1'b0;
            s_axis_tdata = 64'd0;
        end
    endtask

    task automatic consume_results(
        input int case_index,
        input int result_count
    );
        int received;
        int output_cycle;
        logic signed [63:0] result_extended;
        begin
            received = 0;
            output_cycle = 0;
            while (received < result_count && output_cycle < 20000) begin
                @(negedge clk);
                m_axis_tready = ((output_cycle + case_index) % 7 != 2);
                @(posedge clk);
                if (m_axis_tvalid && m_axis_tready) begin
                    result_extended = {{16{m_axis_tdata[47]}}, m_axis_tdata[47:0]};
                    if (result_extended !== expected_mem[expected_pointer]) begin
                        errors++;
                        if (errors <= 30) begin
                            $display("ERROR buffered case=%0d row=%0d col=%0d got=%0d expected=%0d",
                                     case_index, m_axis_tdata[51:48], m_axis_tdata[57:52],
                                     result_extended, expected_mem[expected_pointer]);
                        end
                    end
                    if (m_axis_tlast !== (received == result_count - 1)) begin
                        errors++;
                        $display("ERROR buffered last mismatch case=%0d result=%0d", case_index, received);
                    end
                    expected_pointer++;
                    received++;
                    checked++;
                end
                output_cycle++;
            end
            @(negedge clk);
            m_axis_tready = 1'b0;
            if (received != result_count) begin
                errors++;
                $display("ERROR buffered output timeout case=%0d got=%0d expected=%0d",
                         case_index, received, result_count);
            end
        end
    endtask

    task automatic run_fixture_case(input int case_index);
        logic [63:0] descriptor;
        logic [3:0] load_weight_opcode;
        logic [3:0] compute_opcode;
        int mode;
        int rows;
        int columns;
        int issue_count;
        int activation_zp;
        int weight_zp;
        int active_count_expected;
        longint unsigned dense_macs_expected;
```

```
        longint unsigned spike_adds_expected;
        begin
            descriptor = case_mem[case_index];
            mode = descriptor[1:0];
            rows = descriptor[7:4];
            columns = descriptor[13:8];
            issue_count = descriptor[25:14];
            activation_zp = descriptor[33:26];
            weight_zp = descriptor[41:34];

            load_weight_opcode = mode == MODE_DENSE_DOT8
                ? CMD_LOAD_WEIGHTS_DOT8 : CMD_LOAD_WEIGHTS_OUTER;
            unique case (mode)
                MODE_DENSE_OUTER: compute_opcode = CMD_COMPUTE_DENSE_OUTER;
                MODE_DENSE_DOT8: compute_opcode = CMD_COMPUTE_DENSE_DOT8;
                MODE_BINARY_WEIGHTED: compute_opcode = CMD_COMPUTE_BINARY;
                default: compute_opcode = CMD_COMPUTE_TERNARY;
            endcase

            submit_command(make_command(CMD_CLEAR_STATUS, 0, 0, 0, 0, 0));
            submit_command(make_command(CMD_LOAD_ACTIVATIONS, 0, 0,
                                        issue_count, activation_zp, 0));
            stream_activation_case(issue_pointer, issue_count);
            // Compaction is a registered BRAM write pipeline. Status becomes
            // architecturally visible when the load command commits.
            wait (command_done);
            @(negedge clk);

            active_count_expected = 0;
            dense_macs_expected = 0;
            spike_adds_expected = 0;
            for (int issue = 0; issue < issue_count; issue++) begin
                if (!activation_is_zero(activation_mem[issue_pointer + issue], activation_zp)) begin
                    active_count_expected++;
                    if (mode == MODE_DENSE_OUTER) begin
                        dense_macs_expected += rows * columns;
                    end else if (mode == MODE_DENSE_DOT8) begin
                        dense_macs_expected += 8 * columns;
                    end else begin
                        int active_spikes;
                        active_spikes = 0;
                        for (int row = 0; row < 8; row++) begin
                            if (row < rows
                                    && activation_mem[issue_pointer + issue][row*8 +: 8] != 0) begin
                                active_spikes++;
                            end
                        end
                        spike_adds_expected += active_spikes * columns;
                    end
                end
            end
            if (status_loaded_activations != issue_count
                    || status_active_issues != active_count_expected) begin
                errors++;
                $display("ERROR compaction case=%0d loaded=%0d/%0d active=%0d/%0d",
                         case_index, status_loaded_activations, issue_count,
                         status_active_issues, active_count_expected);
            end

            submit_command(make_command(load_weight_opcode, 0, 0,
                                        issue_count, 0, 0));
            stream_weight_case(issue_pointer, issue_count, mode == MODE_DENSE_DOT8);
            if (status_loaded_weights != issue_count) begin
                errors++;
                $display("ERROR weight count case=%0d loaded=%0d expected=%0d",
                         case_index, status_loaded_weights, issue_count);
            end

            submit_command(make_command(compute_opcode, rows, columns,
                                        issue_count, activation_zp, weight_zp));
            consume_results(case_index, rows * columns);

            // Completion is a registered architectural event.  It follows the
            // final output transfer so status_error is stable in the same cycle.
            wait (command_done);
            @(negedge clk);

            if (!command_done || status_busy || status_error
                    || status_skipped_issues != issue_count - active_count_expected
                    || status_dense_macs != dense_macs_expected
                    || status_spike_adds != spike_adds_expected) begin
                errors++;
                $display("ERROR completion case=%0d done=%0d busy=%0d error=%0d skipped=%0d/%0d dense=%0d/%0d spike=%0d/%0d",
                         case_index, command_done, status_busy, status_error,
                         status_skipped_issues, issue_count - active_count_expected,
                         status_dense_macs, dense_macs_expected,
                         status_spike_adds, spike_adds_expected);
```

```
            end
            $display("buffered_case=%0d mode=%0d results=%0d active=%0d/%0d input_beats=%0d compute_cycles=%0d",
                     case_index, mode, rows * columns, active_count_expected,
                     issue_count, status_input_beats, status_compute_cycles);
            issue_pointer += issue_count;
        end
    endtask

    task automatic check_counter_add(
        input logic [63:0] value,
        input logic [18:0] increment
    );
        logic [63:0] actual;
        logic [63:0] expected;
        logic [24:0] low_sum;
        begin
            low_sum = dut.add_u19_to_u24(value[23:0], increment);
            actual = {
                value[63:24] + {{39{1'b0}}, low_sum[24]},
                low_sum[23:0]
            };
            expected = value + {{45{1'b0}}, increment};
            if (actual !== expected) begin
                errors++;
                $display("ERROR counter add value=%h increment=%h actual=%h expected=%h",
                         value, increment, actual, expected);
            end
        end
    endtask

    task automatic verify_counter_adder;
        logic [63:0] random_value;
        logic [18:0] random_increment;
        int unsigned random_state;
        begin
            check_counter_add(64'd0, 19'd0);
            check_counter_add(64'd0, 19'h7ffff);
            check_counter_add(64'h0000000000ffffff, 19'd1);
            check_counter_add(64'h00000000ffffffff, 19'd1);
            check_counter_add(64'h0000ffffffffffff, 19'd1);
            check_counter_add(64'h00ffffffffffffff, 19'd1);
            check_counter_add(64'hffffffffffffffff, 19'h7ffff);
            check_counter_add(64'h55ffffffffffeaa, 19'h10155);

            random_state = 32'h4e455552;
            for (int vector_index = 0; vector_index < 1024;
                 vector_index++) begin
                random_state = random_state * 32'd1664525 + 32'd1013904223;
                random_value[31:0] = random_state;
                random_state = random_state * 32'd1664525 + 32'd1013904223;
                random_value[63:32] = random_state;
                random_state = random_state * 32'd1664525 + 32'd1013904223;
                random_increment = random_state[18:0];
                check_counter_add(random_value, random_increment);
            end
        end
    endtask

    always @(posedge clk) begin
        cycle <= cycle + 1;
        if (held_valid && (!m_axis_tvalid || m_axis_tdata !== held_data
                || m_axis_tlast !== held_last)) begin
            errors++;
            $display("ERROR buffered output changed while backpressured");
        end
        held_valid <= m_axis_tvalid && !m_axis_tready;
        held_data <= m_axis_tdata;
        held_last <= m_axis_tlast;
    end

    initial begin
        $readmemh("tb/fixtures/tensor_cases.mem", case_mem);
        $readmemh("tb/fixtures/tensor_activations.mem", activation_mem);
        $readmemh("tb/fixtures/tensor_weights.mem", weight_mem);
        $readmemh("tb/fixtures/tensor_expected.mem", expected_mem);

        repeat (5) @(negedge clk);
        rst_n = 1'b1;
        repeat (2) @(negedge clk);

        verify_counter_adder();

        for (int case_index = 0; case_index < CASES; case_index++) begin
            run_fixture_case(case_index);
        end

        if (issue_pointer != ISSUES || expected_pointer != RESULTS || checked != RESULTS) begin
```

```
406             errors++;
407             $display("ERROR buffered accounting issue=%0d/%0d expected=%0d/%0d checked=%0d",
408                      issue_pointer, ISSUES, expected_pointer, RESULTS, checked);
409         end
410
411         if (errors != 0) begin
412             $display("tb_neuraedge_buffered_tensor_core FAILED errors=%0d checked=%0d", errors, checked);
413             $fatal(1);
414         end else begin
415             $display("tb_neuraedge_buffered_tensor_core PASSED cases=%0d checked=%0d", CASES, checked);
416         end
417         $finish;
418     end
419
420     initial begin
421         #200_000;
422         $display("ERROR buffered core watchdog state=%0d busy=%0d cmd_ready=%0d s_ready=%0d m_valid=%0d",
423                  dut.state, status_busy, command_ready, s_axis_tready, m_axis_tvalid);
424         $fatal(1);
425     end
426 endmodule
427
428 `default_nettype wire
```

48.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 3: `` `include "tb/fixtures/tensor_tile_params.svh" `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation.

Line 5: `module tb_neuraedge_buffered_tensor_core;` This opens tb_neuraedge_buffered_tensor_core;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation.

Line 6: `import neuraedge_command_pkg::*;` This imports shared package names into the current module so opcodes and mode constants have exactly one definition. The import changes compile-time name visibility; it does not create registers or consume a clock cycle.

**Line 8: `localparam int CASES = `NEURAEDGE_TENSOR_CASES;`** This fixes CASES to `NEURAEDGE_TENSOR_CASES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CASES denotes the value named 'cases' in this block.

**Line 9: `localparam int ISSUES = `NEURAEDGE_TENSOR_ISSUES;`** This fixes ISSUES to `NEURAEDGE_TENSOR_ISSUES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ISSUES denotes the value named 'issues' in this block.

**Line 10: `localparam int RESULTS = `NEURAEDGE_TENSOR_RESULTS;`** This fixes RESULTS to `NEURAEDGE_TENSOR_RES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; RESULTS denotes the value named 'results' in this block.

**Line 11: `localparam int COLUMN_LANES = `NEURAEDGE_COLUMN_LANES;`** This fixes COLUMN_LANES to `NEURAEDGE_COLUMN_LANES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; COLUMN_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 13: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic command_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic command_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [127:0] command_data = 128'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_data is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic command_done;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_done is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic status_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_error is the value named 'status error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic status_busy;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_busy is the value named 'status busy' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic s_axis_tvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic s_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic [63:0] s_axis_tdata = 64'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tdata is the value named 's axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic s_axis_tlast = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic m_axis_tvalid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `logic m_axis_tready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic [63:0] m_axis_tdata;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tdata is the value named 'm axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 28: `logic m_axis_tlast;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `logic [11:0] status_loaded_activations;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_loaded_activations is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic [11:0] status_loaded_weights;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_loaded_weights is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `logic [11:0] status_active_issues;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_active_issues is the value named 'status active issues' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 32: `logic [31:0] status_input_beats;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_input_beats is the value named 'status input beats' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `logic [31:0] status_output_beats;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_output_beats is the value named 'status output beats' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic [31:0] status_compute_cycles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_compute_cycles is the value named 'status compute cycles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 35: `logic [31:0] status_skipped_issues;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_skipped_issues is the value named 'status skipped issues' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `logic [63:0] status_dense_macs;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_dense_macs is a count of ordinary activation-times-weight operations actually issued. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 37: `logic [63:0] status_spike_adds;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_spike_adds is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 39: `logic [63:0] case_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, case_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 40: `logic [63:0] activation_mem [0:ISSUES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_mem is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 41: `logic [COLUMN_LANES*8*8-1:0] weight_mem [0:ISSUES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_mem is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 42: `logic signed [63:0] expected_mem [0:RESULTS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, expected_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 44: `int issue_pointer = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are issue_pointer: the value named 'issue pointer' in this block.

Line 45: `int expected_pointer = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are expected_pointer: the value named 'expected pointer' in this block.

Line 46: `int checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 47: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 48: `int cycle = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are cycle: the value named 'cycle' in this block.

Line 49: `logic held_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 50: `logic [63:0] held_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 51: `logic held_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 53: `neuraedge_buffered_tensor_core dut (` This instantiates module neuraedge_buffered_tensor_core as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 54: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 55: `.rst_n(rst_n),` This connects the child instance's named port 'rst_n' to parent expression 'rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 56: `.command_valid(command_valid),` This connects the child instance's named port 'command_valid' to parent expression 'command_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 57: `.command_ready(command_ready),` This connects the child instance's named port 'command_ready' to parent expression 'command_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 58: `.command_data(command_data[41:0]),` This connects the child instance's named port 'command_data' to parent expression 'command_data[41:0]'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 59: `.command_done(command_done),` This connects the child instance's named port 'command_done' to parent expression 'command_done'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 60: `.status_error(status_error),` This connects the child instance's named port 'status_error' to parent expression 'status_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status error' in this block.

Line 61: `.status_busy(status_busy),` This connects the child instance's named port 'status_busy' to parent expression 'status_busy'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status busy' in this block.

Line 62: `.s_axis_tvalid(s_axis_tvalid),` This connects the child instance's named port 's_axis_tvalid' to parent expression 's_axis_tvalid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 63: `.s_axis_tready(s_axis_tready),` This connects the child instance's named port 's_axis_tready' to parent expression 's_axis_tready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 64: `.s_axis_tdata(s_axis_tdata),` This connects the child instance's named port 's_axis_tdata' to parent expression 's_axis_tdata'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 's axis tdata' in this block.

Line 65: `.s_axis_tlast(s_axis_tlast),` This connects the child instance's named port 's_axis_tlast' to parent expression 's_axis_tlast'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 66: `.m_axis_tvalid(m_axis_tvalid),` This connects the child instance's named port 'm_axis_tvalid' to parent expression 'm_axis_tvalid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 67: `.m_axis_tready(m_axis_tready),` This connects the child instance's named port 'm_axis_tready' to parent expression 'm_axis_tready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 68: `.m_axis_tdata(m_axis_tdata),` This connects the child instance's named port 'm_axis_tdata' to parent expression 'm_axis_tdata'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'm axis tdata' in this block.

Line 69: `.m_axis_tlast(m_axis_tlast),` This connects the child instance's named port 'm_axis_tlast' to parent expression 'm_axis_tlast'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 70: `.status_loaded_activations(status_loaded_activations),` This connects the child instance's named port ‘status_loaded_activations‘ to parent expression ‘status_loaded_activations‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 71: `.status_loaded_weights(status_loaded_weights),` This connects the child instance's named port ‘status_loaded_weights‘ to parent expression ‘status_loaded_weights‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 72: `.status_active_issues(status_active_issues),` This connects the child instance's named port ‘status_active_issues‘ to parent expression ‘status_active_issues‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status active issues' in this block.

Line 73: `.status_input_beats(status_input_beats),` This connects the child instance's named port ‘status_input_beats‘ to parent expression ‘status_input_beats‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status input beats' in this block.

Line 74: `.status_output_beats(status_output_beats),` This connects the child instance's named port ‘status_output_beats‘ to parent expression ‘status_output_beats‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status output beats' in this block.

Line 75: `.status_compute_cycles(status_compute_cycles),` This connects the child instance's named port ‘status_compute_cycles‘ to parent expression ‘status_compute_cycles‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status compute cycles' in this block.

Line 76: `.status_skipped_issues(status_skipped_issues),` This connects the child instance's named port ‘status_skipped_issues‘ to parent expression ‘status_skipped_issues‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status skipped issues' in this block.

Line 77: `.status_dense_macs(status_dense_macs),` This connects the child instance's named port ‘status_dense_macs‘ to parent expression ‘status_dense_macs‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a count of ordinary activation-times-weight operations actually issued.

Line 78: `.status_spike_adds(status_spike_adds)` This connects the child instance's named port ‘status_spike_adds‘ to parent expression ‘status_spike_adds‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 79: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 81: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 83: `function automatic logic [127:0] make_command(` Within the combinational helper make command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; make_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 84: `input logic [3:0] opcode,` This declares opcode as a input port with width [3:0]. It carries the value named 'opcode' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 85: `input int rows,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 86: `input int columns,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 87: `input int issue_count,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 88: `input int activation_zp,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 89: `input int weight_zp` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 90: `);` Within the combinational helper make command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 91: `logic [127:0] value;` This declares hardware signal storage or wiring in the combinational helper make command. Specifically, value is the value named 'value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 92: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper make command; it has no arithmetic value of its own.

Line 93: `value = 128'd0;` This makes value, the value named 'value' in this block, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the combinational helper make command.

Line 94: `value[3:0] = opcode;` This makes value[3:0], the value named 'value' in this block, equal to 'opcode'. It is driven by combinational or procedural logic in the current evaluation. The important operands are opcode: the value named 'opcode' in this block. Context: the combinational helper make command.

Line 95: `value[7:4] = 4'(rows);` This makes value[7:4], the value named 'value' in this block, equal to '4'(rows)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are rows: one output-row position processed by the tensor tile. Context: the combinational helper make command.

Line 96: `value[13:8] = 6'(columns);` This makes value[13:8], the value named 'value' in this block, equal to '6'(columns)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are columns: one output-feature position processed in parallel. Context: the combinational helper make command.

Line 97: `value[25:14] = 12'(issue_count);` This makes value[25:14], the value named 'value' in this block, equal to '12'(issue_count)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_count: an integer count retained for control or measurable telemetry. Context: the combinational helper make command.

Line 98: `value[33:26] = 8'(activation_zp);` This makes value[33:26], the value named 'value' in this block, equal to '8'(activation_zp)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are activation_zp: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the combinational helper make command.

Line 99: `value[41:34] = 8'(weight_zp);` This makes value[41:34], the value named 'value' in this block, equal to '8'(weight_zp)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_zp: a learned coefficient multiplied by an activation. Context: the combinational helper make command.

Line 100: `make_command = value;` This makes make_command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper make command.

Line 101: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper make command; it has no arithmetic value of its own.

Line 102: `endfunction` Within the combinational helper make command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are endfunction: the value named 'endfunction' in this block.

Line 104: `function automatic logic activation_is_zero(` Within the combinational helper activation is zero, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; activation_is_zero: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 105: `input logic [63:0] values,` This declares values as a input port with width [63:0]. It carries the value named 'values' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 106: `input logic [7:0] zero_point` This declares zero_point as a input port with width [7:0]. It carries the integer code representing real numeric zero in affine quantization across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 107: `);` Within the combinational helper activation is zero, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 108: `logic is_zero;` This declares hardware signal storage or wiring in the combinational helper activation is zero. Specifically, is_zero is the value named 'is zero' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 109: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper activation is zero; it has no arithmetic value of its own.

Line 110: `is_zero = 1'b1;` This makes is_zero, the value named 'is zero' in this block, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the combinational helper activation is zero.

Line 111: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the combinational helper activation is zero.

Line 112: `is_zero = is_zero && values[lane*8 +:  8] == zero_point;` This makes is_zero, the value named 'is zero' in this block, equal to 'is_zero && values[lane*8 +: 8] == zero_point'. It is driven by combinational or procedural logic in the current evaluation. The important operands are is_zero: the value named 'is zero' in this block; values: the value named 'values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the combinational helper activation is zero.

Line 113: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper activation is zero; it has no arithmetic value of its own.

Line 114: `activation_is_zero = is_zero;` This makes activation_is_zero, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'is_zero'. It is driven by combinational or procedural logic in the current evaluation. The important operands are is_zero: the value named 'is zero' in this block. Context: the combinational helper activation is zero.

Line 115: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper activation is zero; it has no arithmetic value of its own.

Line 116: `endfunction` Within the combinational helper activation is zero, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are endfunction: the value named 'endfunction' in this block.

Line 118: `task automatic submit_command(input logic [127:0] value);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 119: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task submit command; it has no arithmetic value of its own.

Line 120: `while (!command_ready) @(negedge clk);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are while: the value named 'while' in this block; command_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 121: `@(negedge clk);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 122: `command_data = value;` This makes command_data, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the simulation task submit command.

Line 123: `command_valid = 1'b1;` This makes command_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task submit command.

Line 124: `@(posedge clk);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 125: `@(negedge clk);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 126: `command_valid = 1'b0;` This makes command_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task submit command.

Line 127: `command_data = 128'd0;` This makes command_data, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task submit command.

Line 128: `end` This structural keyword opens or closes the branch/process scope described as the simulation task submit command; it has no arithmetic value of its own.

Line 129: `endtask` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 131: `task automatic stream_activation_case(` Within the simulation task stream activation case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; stream_activation_case: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 132: `input int first_issue,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 133: `input int issue_count` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 134: `);` Within the simulation task stream activation case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 135: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task stream activation case; it has no arithmetic value of its own.

Line 136: `for (int issue = 0; issue < issue_count; issue++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task stream activation case.

Line 137: `@(negedge clk);` Within the simulation task stream activation case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 138: `s_axis_tvalid = 1'b1;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task stream activation case.

Line 139: `s_axis_tdata = activation_mem[first_issue + issue];` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to 'activation_mem[first_issue + issue]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are activation_mem: an input feature value consumed by a neuron or multiply-accumulate operation; first_issue: the value named 'first issue' in this block; issue: the value named 'issue' in this block. Context: the simulation task stream activation case.

Line 140: `s_axis_tlast = (issue == issue_count - 1);` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘(issue == issue_count - 1)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue: the value named ’issue’ in this block; issue_count: an integer count retained for control or measurable telemetry. Context: the simulation task stream activation case.

Line 141: `while (!s_axis_tready) @(negedge clk);` Within the simulation task stream activation case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are while: the value named ’while’ in this block; s_axis_tready: the consumer’s statement that it can accept a payload this cycle; negedge: the value named ’negedge’ in this block.

Line 142: `@(posedge clk);` Within the simulation task stream activation case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 143: `end` This structural keyword opens or closes the branch/process scope described as the simulation task stream activation case; it has no arithmetic value of its own.

Line 144: `@(negedge clk);` Within the simulation task stream activation case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 145: `s_axis_tvalid = 1’b0;` This makes s_axis_tvalid, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task stream activation case.

Line 146: `s_axis_tlast = 1’b0;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task stream activation case.

Line 147: `s_axis_tdata = 64’d0;` This makes s_axis_tdata, the value named ’s axis tdata’ in this block, equal to ‘64’d0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named ’d0’ in this block. Context: the simulation task stream activation case.

Line 148: `end` This structural keyword opens or closes the branch/process scope described as the simulation task stream activation case; it has no arithmetic value of its own.

Line 149: `endtask` Within the simulation task stream activation case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are endtask: the value named ’endtask’ in this block.

Line 151: `task automatic stream_weight_case(` Within the simulation task stream weight case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are task: the value named ’task’ in this block; automatic: the value named ’automatic’ in this block; stream_weight_case: a learned coefficient multiplied by an activation.

Line 152: `input int first_issue,` This declares int as a input port with width one bit. It carries the value named ’int’ in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 153: `input int issue_count,` This declares int as a input port with width one bit. It carries the value named ’int’ in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 154: `input logic dot8` This declares dot8 as a input port with width one bit. It carries the value named ’dot8’ in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 155: `);` Within the simulation task stream weight case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 156: `int beats_per_issue;` Within the simulation task stream weight case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are beats_per_issue: the value named 'beats per issue' in this block.

Line 157: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task stream weight case; it has no arithmetic value of its own.

Line 158: `beats_per_issue = dot8 ? COLUMN_LANES : COLUMN_LANES / 8;` This makes beats_per_issue, the value named 'beats per issue' in this block, equal to 'dot8 ? COLUMN_LANES : COLUMN_LANES / 8'. It is driven by combinational or procedural logic in the current evaluation. The important operands are dot8: the value named 'dot8' in this block; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task stream weight case.

Line 159: `for (int issue = 0; issue < issue_count; issue++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task stream weight case.

Line 160: `for (int beat = 0; beat < beats_per_issue; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task stream weight case.

Line 161: `@(negedge clk);` Within the simulation task stream weight case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 162: `s_axis_tvalid = 1'b1;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task stream weight case.

Line 163: `s_axis_tdata = weight_mem[first_issue + issue][beat*64 +: 64];` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to 'weight_mem[first_issue + issue][beat*64 +: 64]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_mem: a learned coefficient multiplied by an activation; first_issue: the value named 'first issue' in this block; issue: the value named 'issue' in this block. Context: the simulation task stream weight case.

Line 164: `s_axis_tlast = issue == issue_count - 1 && beat == beats_per_issue - 1;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'issue == issue_count - 1 && beat == beats_per_issue - 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue: the value named 'issue' in this block; issue_count: an integer count retained for control or measurable telemetry; beat: the value named 'beat' in this block. Context: the simulation task stream weight case.

Line 165: `while (!s_axis_tready) @(negedge clk);` Within the simulation task stream weight case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axis_tready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 166: `@(posedge clk);` Within the simulation task stream weight case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 167: `end` This structural keyword opens or closes the branch/process scope described as the simulation task stream weight case; it has no arithmetic value of its own.

Line 168: `end` This structural keyword opens or closes the branch/process scope described as the simulation task stream weight case; it has no arithmetic value of its own.

Line 169: `@(negedge clk);` Within the simulation task stream weight case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 170: `s_axis_tvalid = 1'b0;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task stream weight case.

Line 171: `s_axis_tlast = 1'b0;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task stream weight case.

Line 172: `s_axis_tdata = 64'd0;` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task stream weight case.

Line 173: `end` This structural keyword opens or closes the branch/process scope described as the simulation task stream weight case; it has no arithmetic value of its own.

Line 174: `endtask` Within the simulation task stream weight case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 176: `task automatic consume_results(` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; consume_results: the value named 'consume results' in this block.

Line 177: `input int case_index,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 178: `input int result_count` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 179: `);` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 180: `int received;` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are received: the value named 'received' in this block.

Line 181: `int output_cycle;` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are output_cycle: the value named 'output cycle' in this block.

Line 182: `logic signed [63:0] result_extended;` This declares hardware signal storage or wiring in the simulation task consume results. Specifically, result_extended is the value named 'result extended' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 183: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task consume results; it has no arithmetic value of its own.

Line 184: `received = 0;` This makes received, the value named 'received' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task consume results.

Line 185: `output_cycle = 0;` This makes output_cycle, the value named 'output cycle' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task consume results.

Line 186: `while (received < result_count && output_cycle < 20000) begin` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are while: the value named 'while' in this block; received: the value named 'received' in this block; result_count: an integer count retained for control or measurable telemetry.

Line 187: `@(negedge clk);` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 188: `m_axis_tready = ((output_cycle + case_index) % 7 != 2);` This makes m_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '((output_cycle + case_index) % 7 != 2)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_cycle: the value named 'output cycle' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task consume results.

Line 189: `@(posedge clk);` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 190: `if (m_axis_tvalid && m_axis_tready) begin` This condition selects the following hardware branch only when 'm_axis_tvalid && m_axis_tready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are m_axis_tvalid: the producer's statement that its current payload is meaningful; m_axis_tready: the consumer's statement that it can accept a payload this cycle. Context: the simulation task consume results.

Line 191: `result_extended = {{16{m_axis_tdata[47]}}, m_axis_tdata[47:0]};` This makes result_extended, the value named 'result extended' in this block, equal to '{{16{m_axis_tdata[47]}}, m_axis_tdata[47:0]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are m_axis_tdata: the value named 'm axis tdata' in this block. Context: the simulation task consume results.

Line 192: `if (result_extended !== expected_mem[expected_pointer]) begin` This condition selects the following hardware branch only when 'result_extended !== expected_mem[expected_pointer]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_extended: the value named 'result extended' in this block; expected_mem: local FPGA storage addressed by surrounding index logic; expected_pointer: the value named 'expected pointer' in this block. Context: the simulation task consume results.

Line 193: `errors++;` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 194: `if (errors <= 30) begin` This statement continues through source line 197; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when ‘errors <= 30) begin $display("ERROR buffered case=%0d row=%0d col=%0d got=%0d expected=%0d", case_index, m_axis_tdata[51:48], m_axis_tdata[57:52], result_extended, expected_mem[expected_pointer]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task consume results.

Line 195: `$display("ERROR buffered case=%0d row=%0d col=%0d got=%0d expected=%0d",` This is continuation line 2 of the 4-line logical statement that begins at source line 194. Its exact fragment is ‘$display("ERROR buffered case=%0d row=%0d col=%0d got=%0d expected=%0d",‘. The important operands are display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block; buffered: local storage that decouples producer and consumer timing. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 30) begin $display("ERROR buffered case=%0d row=%0d col=%0d got=%0d expected=%0d", case_index, m_axis_tdata[51:48], m_axis_tdata[57:52], result_extended, expected_mem[expected_pointer]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task consume results.

Line 196: `case_index, m_axis_tdata[51:48], m_axis_tdata[57:52],` This is continuation line 3 of the 4-line logical statement that begins at source line 194. Its exact fragment is ‘case_index, m_axis_tdata[51:48], m_axis_tdata[57:52],‘. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; m_axis_tdata: the value named ’m axis tdata’ in this block. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 30) begin $display("ERROR buffered case=%0d row=%0d col=%0d got=%0d expected=%0d", case_index, m_axis_tdata[51:48], m_axis_tdata[57:52], result_extended, expected_mem[expected_pointer]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task consume results.

Line 197: `result_extended, expected_mem[expected_pointer]);` This is continuation line 4 of the 4-line logical statement that begins at source line 194. Its exact fragment is ‘result_extended, expected_mem[expected_pointer]);‘. The important operands are result_extended: the value named ’result extended’ in this block; expected_mem: local FPGA storage addressed by surrounding index logic; expected_pointer: the value named ’expected pointer’ in this block. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 30) begin $display("ERROR buffered case=%0d row=%0d col=%0d got=%0d expected=%0d", case_index, m_axis_tdata[51:48], m_axis_tdata[57:52], result_extended, expected_mem[expected_pointer]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task consume results.

Line 198: `end` This structural keyword opens or closes the branch/process scope described as the simulation task consume results; it has no arithmetic value of its own.

Line 199: `end` This structural keyword opens or closes the branch/process scope described as the simulation task consume results; it has no arithmetic value of its own.

Line 200: `if (m_axis_tlast !== (received == result_count - 1)) begin` This condition selects the following hardware branch only when ‘m_axis_tlast !== (received == result_count - 1)‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; received: the value named ’received’ in this block; result_count: an integer count retained for control or measurable telemetry. Context: the simulation task consume results.

Line 201: `errors++;` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 202: `$display("ERROR buffered last mismatch case=%0d result=%0d", case_index, received);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 203: `end` This structural keyword opens or closes the branch/process scope described as the simulation task consume results; it has no arithmetic value of its own.

Line 204: `expected_pointer++;` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are expected_pointer: the value named 'expected pointer' in this block.

Line 205: `received++;` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are received: the value named 'received' in this block.

Line 206: `checked++;` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 207: `end` This structural keyword opens or closes the branch/process scope described as the simulation task consume results; it has no arithmetic value of its own.

Line 208: `output_cycle++;` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are output_cycle: the value named 'output cycle' in this block.

Line 209: `end` This structural keyword opens or closes the branch/process scope described as the simulation task consume results; it has no arithmetic value of its own.

Line 210: `@(negedge clk);` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 211: `m_axis_tready = 1'b0;` This makes m_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task consume results.

Line 212: `if (received != result_count) begin` This condition selects the following hardware branch only when 'received != result_count' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received: the value named 'received' in this block; result_count: an integer count retained for control or measurable telemetry. Context: the simulation task consume results.

Line 213: `errors++;` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 214: `$display("ERROR buffered output timeout case=%0d got=%0d expected=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 215: `case_index, received, result_count);` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; received: the value named 'received' in this block; result_count: an integer count retained for control or measurable telemetry.

Line 216: `end` This structural keyword opens or closes the branch/process scope described as the simulation task consume results; it has no arithmetic value of its own.

Line 217: `end` This structural keyword opens or closes the branch/process scope described as the simulation task consume results; it has no arithmetic value of its own.

Line 218: `endtask` Within the simulation task consume results, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 220: `task automatic run_fixture_case(input int case_index);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; run_fixture_case: the value named 'run fixture case' in this block.

Line 221: `logic [63:0] descriptor;` This declares hardware signal storage or wiring in the simulation task run fixture case. Specifically, descriptor is a DMA record header plus one command and its declared payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 222: `logic [3:0] load_weight_opcode;` This declares hardware signal storage or wiring in the simulation task run fixture case. Specifically, load_weight_opcode is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 223: `logic [3:0] compute_opcode;` This declares hardware signal storage or wiring in the simulation task run fixture case. Specifically, compute_opcode is the value named 'compute opcode' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 224: `int mode;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are mode: the value named 'mode' in this block.

Line 225: `int rows;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are rows: one output-row position processed by the tensor tile.

Line 226: `int columns;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are columns: one output-feature position processed in parallel.

Line 227: `int issue_count;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are issue_count: an integer count retained for control or measurable telemetry.

Line 228: `int activation_zp;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are activation_zp: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 229: `int weight_zp;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are weight_zp: a learned coefficient multiplied by an activation.

Line 230: `int active_count_expected;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are active_count_expected: the value named 'active count expected' in this block.

Line 231: `longint unsigned dense_macs_expected;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are longint: the value named 'longint' in this block; dense_macs_expected: a count of ordinary activation-times-weight operations actually issued.

Line 232: `longint unsigned spike_adds_expected;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are longint: the value named 'longint' in this block; spike_adds_expected: a binary event produced when a neuron crosses its threshold.

Line 233: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 234: `descriptor = case_mem[case_index];` This makes descriptor, a DMA record header plus one command and its declared payload, equal to ‘case_mem[case_index]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are case_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run fixture case.

Line 235: `mode = descriptor[1:0];` This makes mode, the value named 'mode' in this block, equal to ‘descriptor[1:0]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are descriptor: a DMA record header plus one command and its declared payload. Context: the simulation task run fixture case.

Line 236: `rows = descriptor[7:4];` This makes rows, one output-row position processed by the tensor tile, equal to ‘descriptor[7:4]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are descriptor: a DMA record header plus one command and its declared payload. Context: the simulation task run fixture case.

Line 237: `columns = descriptor[13:8];` This makes columns, one output-feature position processed in parallel, equal to ‘descriptor[13:8]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are descriptor: a DMA record header plus one command and its declared payload. Context: the simulation task run fixture case.

Line 238: `issue_count = descriptor[25:14];` This makes issue_count, an integer count retained for control or measurable telemetry, equal to ‘descriptor[25:14]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are descriptor: a DMA record header plus one command and its declared payload. Context: the simulation task run fixture case.

Line 239: `activation_zp = descriptor[33:26];` This makes activation_zp, an input feature value consumed by a neuron or multiply-accumulate operation, equal to ‘descriptor[33:26]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are descriptor: a DMA record header plus one command and its declared payload. Context: the simulation task run fixture case.

Line 240: `weight_zp = descriptor[41:34];` This makes weight_zp, a learned coefficient multiplied by an activation, equal to ‘descriptor[41:34]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are descriptor: a DMA record header plus one command and its declared payload. Context: the simulation task run fixture case.

Line 242: `load_weight_opcode = mode == MODE_DENSE_DOT8` This statement continues through source line 243; the explanation uses the complete expression rather than only the first visual fragment. This makes load_weight_opcode, a learned coefficient multiplied by an activation, equal to ‘mode == MODE_DENSE_DOT8 ? CMD_LOAD_WEIGHTS_DOT8 : CMD_LOAD_WEIGHTS_OUTER‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are mode: the value named 'mode' in this block; MODE_DENSE_DOT8: the value named 'mode

dense dot8' in this block; CMD_LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation. Context: the simulation task run fixture case.

Line 243: `? CMD_LOAD_WEIGHTS_DOT8 : CMD_LOAD_WEIGHTS_OUTER;` This is continuation line 2 of the 2-line logical statement that begins at source line 242. Its exact fragment is '? CMD_LOAD_WEIGHTS_DOT8 : CMD_LOAD_WEIGHTS_O The important operands are CMD_LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation; CMD_LOAD_WE a learned coefficient multiplied by an activation. Read together, the complete operation is: This makes load_weight_opcode, a learned coefficient multiplied by an activation, equal to 'mode == MODE_DENSE_DOT8 ? CMD_LOAD_WEIGHTS_DOT8 : CMD_LOAD_WEIGHTS_OUTER'. It is driven by combinational or procedural logic in the current evaluation. The important operands are mode: the value named 'mode' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block; CMD_LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation. Context: the simulation task run fixture case.

Line 244: `unique case (mode)` This starts a multiplexer/state decode selected by 'mode'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 245: `MODE_DENSE_OUTER: compute_opcode = CMD_COMPUTE_DENSE_OUTER;` This is the MODE_DENSE_OUTER case branch, which performs this explicitly named phase. The same-cycle action begins with 'compute_opcode = CMD_COMPUTE_DENSE_OUTER;'; context is the simulation task run fixture case Within state MODE_DENSE_OUTER, the machine performs the named control phase..

Line 246: `MODE_DENSE_DOT8: compute_opcode = CMD_COMPUTE_DENSE_DOT8;` This is the MODE_DENSE_DOT8 case branch, which performs this explicitly named phase. The same-cycle action begins with 'compute_opcode = CMD_COMPUTE_DENSE_DOT8;'; context is the simulation task run fixture case Within state MODE_DENSE_DOT8, the machine performs the named control phase..

Line 247: `MODE_BINARY_WEIGHTED: compute_opcode = CMD_COMPUTE_BINARY;` This is the MODE_BINARY_WEIGHTED case branch, which performs this explicitly named phase. The same-cycle action begins with 'compute_opcode = CMD_COMPUTE_BINARY;'; context is the simulation task run fixture case Within state MODE_BINARY_WEIGHTED, the machine performs the named control phase..

Line 248: `default: compute_opcode = CMD_COMPUTE_TERNARY;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with 'compute_opcode = CMD_COMPUTE_TERNARY;'; context is the simulation task run fixture case Within state MODE_BINARY_WEIGHTED, the machine performs the named control phase..

Line 249: `endcase` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case Within state MODE_BINARY_WEIGHTED, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 251: `submit_command(make_command(CMD_CLEAR_STATUS, 0, 0, 0, 0, 0));` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; make_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; CMD_CLEAR_STATUS: the value named 'cmd clear status' in this block.

Line 252: `submit_command(make_command(CMD_LOAD_ACTIVATIONS, 0, 0,` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; make_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; CMD_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 253: `issue_count, activation_zp, 0));` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are issue_count: an integer count retained for control or measurable telemetry; activation_zp: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 254: `stream_activation_case(issue_pointer, issue_count);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are stream_activation_case: an input feature value consumed by a neuron or multiply-accumulate operation; issue_pointer: the value named 'issue pointer' in this block; issue_count: an integer count retained for control or measurable telemetry.

Line 255: `// Compaction is a registered BRAM write pipeline. Status becomes` Design intent: Compaction is a registered BRAM write pipeline. Status becomes This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation.

Line 256: `// architecturally visible when the load command commits.` Design intent: architecturally visible when the load command commits. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation.

Line 257: `wait (command_done);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are wait: the value named 'wait' in this block; command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 258: `@(negedge clk);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 260: `active_count_expected = 0;` This makes active_count_expected, the value named 'active count expected' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run fixture case.

Line 261: `dense_macs_expected = 0;` This makes dense_macs_expected, a count of ordinary activation-times-weight operations actually issued, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run fixture case.

Line 262: `spike_adds_expected = 0;` This makes spike_adds_expected, a binary event produced when a neuron crosses its threshold, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run fixture case.

Line 263: `for (int issue = 0; issue < issue_count; issue++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run fixture case.

Line 264: `if (!activation_is_zero(activation_mem[issue_pointer + issue], activation_zp)) begin` This condition selects the following hardware branch only when '!activation_is_zero(activation_mem[issue_pointer + issue], activation_zp)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are activation_is_zero: an input feature value consumed by a neuron or multiply-accumulate operation; activation_mem: an input feature value consumed by a neuron or multiply-accumulate operation; issue_pointer: the value named 'issue pointer' in this block. Context: the simulation task run fixture case.

Line 265: `active_count_expected++;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are active_count_expected: the value named 'active count expected' in this block.

Line 266: `if (mode == MODE_DENSE_OUTER) begin` This condition selects the following hardware branch only when 'mode == MODE_DENSE_OUTER' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are mode: the value named 'mode' in this block; MODE_DENSE_OUTER: the value named 'mode dense outer' in this block. Context: the simulation task run fixture case.

Line 267: `dense_macs_expected += rows * columns;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are dense_macs_expected: a count of ordinary activation-times-weight operations actually issued; rows: one output-row position processed by the tensor tile; columns: one output-feature position processed in parallel.

Line 268: `end else if (mode == MODE_DENSE_DOT8) begin` This condition selects the following hardware branch only when 'mode == MODE_DENSE_DOT8' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are mode: the value named 'mode' in this block; MODE_DENSE_DOT8: the value named 'mode dense dot8' in this block. Context: the simulation task run fixture case.

Line 269: `dense_macs_expected += 8 * columns;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are dense_macs_expected: a count of ordinary activation-times-weight operations actually issued; columns: one output-feature position processed in parallel.

Line 270: `end else begin` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 271: `int active_spikes;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are active_spikes: a binary event produced when a neuron crosses its threshold.

Line 272: `active_spikes = 0;` This makes active_spikes, a binary event produced when a neuron crosses its threshold, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run fixture case.

Line 273: `for (int row = 0; row < 8; row++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run fixture case.

Line 274: `if (row < rows` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are row: one output-row position processed by the tensor tile; rows: one output-row position processed by the tensor tile.

Line 275: `&& activation_mem[issue_pointer + issue][row*8 +: 8] != 0) begin` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are activation_mem: an input feature value consumed by a neuron or multiply-accumulate operation; issue_pointer: the value named 'issue pointer' in this block; issue: the value named 'issue' in this block.

Line 276: `active_spikes++;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are active_spikes: a binary event produced when a neuron crosses its threshold.

Line 277: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 278: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 279: `spike_adds_expected += active_spikes * columns;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are spike_adds_expected: a binary event produced when a neuron crosses its threshold; active_spikes: a binary event produced when a neuron crosses its threshold; columns: one output-feature position processed in parallel.

Line 280: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 281: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 282: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 283: `if (status_loaded_activations != issue_count` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are status_loaded_activations: an input feature value consumed by a neuron or multiply-accumulate operation; issue_count: an integer count retained for control or measurable telemetry.

Line 284: `|| status_active_issues != active_count_expected) begin` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are status_active_issues: the value named 'status active issues' in this block; active_count_expected: the value named 'active count expected' in this block.

Line 285: `errors++;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 286: `$display("ERROR compaction case=%0d loaded=%0d/%0d active=%0d/%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 287: `case_index, status_loaded_activations, issue_count,` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; status_loaded_activations: an input feature value consumed by a neuron or multiply-accumulate operation; issue_count: an integer count retained for control or measurable telemetry.

Line 288: `status_active_issues, active_count_expected);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are status_active_issues: the value named 'status active issues' in this block; active_count_expected: the value named 'active count expected' in this block.

Line 289: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 291: `submit_command(make_command(load_weight_opcode, 0, 0,` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; make_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; load_weight_opcode: a learned coefficient multiplied by an activation.

Line 292: `issue_count, 0, 0));` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are issue_count: an integer count retained for control or measurable telemetry.

Line 293: `stream_weight_case(issue_pointer, issue_count, mode == MODE_DENSE_DOT8);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are stream_weight_case: a learned coefficient multiplied by an activation; issue_pointer: the value named 'issue pointer' in this block; issue_count: an integer count retained for control or measurable telemetry.

Line 294: `if (status_loaded_weights != issue_count) begin` This condition selects the following hardware branch only when 'status_loaded_weights != issue_count' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are status_loaded_weights: a learned coefficient multiplied by an activation; issue_count: an integer count retained for control or measurable telemetry. Context: the simulation task run fixture case.

Line 295: `errors++;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 296: `$display("ERROR weight count case=%0d loaded=%0d expected=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 297: `case_index, status_loaded_weights, issue_count);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; status_loaded_weights: a learned coefficient multiplied by an activation; issue_count: an integer count retained for control or measurable telemetry.

Line 298: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 300: `submit_command(make_command(compute_opcode, rows, columns,` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; make_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; compute_opcode: the value named 'compute opcode' in this block.

Line 301: `issue_count, activation_zp, weight_zp));` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are issue_count: an integer count retained for control or measurable telemetry; activation_zp: an input feature value consumed by a neuron or multiply-accumulate operation; weight_zp: a learned coefficient multiplied by an activation.

Line 302: `consume_results(case_index, rows * columns);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are consume_results: the value named 'consume results' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry; rows: one output-row position processed by the tensor tile.

Line 304: `// Completion is a registered architectural event. It follows the` Design intent: Completion is a registered architectural event. It follows the This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation.

Line 305: `// final output transfer so status_error is stable in the same cycle.` Design intent: final output transfer so status_error is stable in the same cycle. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation.

Line 306: `wait (command_done);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are wait: the value named 'wait' in this block; command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 307: `@(negedge clk);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 309: `if (!command_done || status_busy || status_error` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; status_busy: the value named 'status busy' in this block; status_error: the value named 'status error' in this block.

Line 310: `|| status_skipped_issues != issue_count - active_count_expected` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are status_skipped_issues: the value named 'status skipped issues' in this block; issue_count: an integer count retained for control or measurable telemetry; active_count_expected: the value named 'active count expected' in this block.

Line 311: `|| status_dense_macs != dense_macs_expected` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are status_dense_macs: a count of ordinary activation-times-weight operations actually issued; dense_macs_expected: a count of ordinary activation-times-weight operations actually issued.

Line 312: `|| status_spike_adds != spike_adds_expected) begin` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are status_spike_adds: a binary event produced when a neuron crosses its threshold; spike_adds_expected: a binary event produced when a neuron crosses its threshold.

Line 313: `errors++;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 314: `$display("ERROR completion case=%0d done=%0d busy=%0d error=%0d skipped=%0d/%0d dense=%0d/%0d spike=%0d/%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 315: `case_index, command_done, status_busy, status_error,` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; status_busy: the value named 'status busy' in this block.

Line 316: `status_skipped_issues, issue_count - active_count_expected,` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are status_skipped_issues: the value named 'status skipped issues' in this block; issue_count: an integer count retained for control or measurable telemetry; active_count_expected: the value named 'active count expected' in this block.

Line 317: `status_dense_macs, dense_macs_expected,` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are status_dense_macs: a count of ordinary activation-times-weight operations actually issued; dense_macs_expected: a count of ordinary activation-times-weight operations actually issued.

Line 318: `status_spike_adds, spike_adds_expected);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are status_spike_adds: a binary event produced when a neuron crosses its threshold; spike_adds_expected: a binary event produced when a neuron crosses its threshold.

Line 319: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 320: `$display("buffered_case=%0d mode=%0d results=%0d active=%0d/%0d input_beats=%0d compute_cycles=%0d"` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 321: `case_index, mode, rows * columns, active_count_expected,` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; mode: the value named 'mode' in this block; rows: one output-row position processed by the tensor tile.

Line 322: `issue_count, status_input_beats, status_compute_cycles);` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are issue_count: an integer count retained for control or measurable telemetry; status_input_beats: the value named 'status input beats' in this block; status_compute_cycles: the value named 'status compute cycles' in this block.

Line 323: `issue_pointer += issue_count;` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are issue_pointer: the value named 'issue pointer' in this block; issue_count: an integer count retained for control or measurable telemetry.

Line 324: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run fixture case; it has no arithmetic value of its own.

Line 325: `endtask` Within the simulation task run fixture case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 327: `task automatic check_counter_add(` Within the simulation task check counter add, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; check_counter_add: a hardware telemetry register that accumulates measured events.

Line 328: `input logic [63:0] value,` This declares value as a input port with width [63:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 329: `input logic [18:0] increment` This declares increment as a input port with width [18:0]. It carries the value named 'increment' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 330: `);` Within the simulation task check counter add, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 331: `logic [63:0] actual;` This declares hardware signal storage or wiring in the simulation task check counter add. Specifically, actual is the value named 'actual' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 332: `logic [63:0] expected;` This declares hardware signal storage or wiring in the simulation task check counter add. Specifically, expected is the value named 'expected' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 333: `logic [24:0] low_sum;` This declares hardware signal storage or wiring in the simulation task check counter add. Specifically, low_sum is the value named 'low sum' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 334: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task check counter add; it has no arithmetic value of its own.

Line 335: `low_sum = dut.add_u19_to_u24(value[23:0], increment);` This makes low_sum, the value named 'low sum' in this block, equal to 'dut.add_u19_to_u24(value[23:0], increment)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are dut: the value named 'dut' in this block; add_u19_to_u24: the value named 'add u19 to u24' in this block; value: the value named 'value' in this block. Context: the simulation task check counter add.

Line 336: `actual = {` This statement continues through source line 339; the explanation uses the complete expression rather than only the first visual fragment. This makes actual, the value named 'actual' in this block, equal to '{ value[63:24] + {{39{1'b0}}, low_sum[24]}, low_sum[23:0] }'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block; b0: the value named 'b0' in this block; low_sum: the value named 'low sum' in this block. Context: the simulation task check counter add.

Line 337: `value[63:24] + {{39{1'b0}}, low_sum[24]},` This is continuation line 2 of the 4-line logical statement that begins at source line 336. Its exact fragment is 'value[63:24] + {{39{1'b0}}, low_sum[24]},'. The important operands are value: the value named 'value' in this block; b0: the value named 'b0' in this block; low_sum: the value named 'low sum' in this block. Read together, the complete operation is: This makes actual, the value named 'actual' in this block, equal to '{ value[63:24] + {{39{1'b0}}, low_sum[24]}, low_sum[23:0] }'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block; b0: the value named 'b0' in this block; low_sum: the value named 'low sum' in this block. Context: the simulation task check counter add.

Line 338: `low_sum[23:0]` This is continuation line 3 of the 4-line logical statement that begins at source line 336. Its exact fragment is 'low_sum[23:0]'. The important operands are low_sum: the value named 'low sum' in this block. Read together, the complete operation is: This makes actual, the value named 'actual' in this block, equal to '{ value[63:24] + {{39{1'b0}}, low_sum[24]}, low_sum[23:0] }'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block; b0: the value named 'b0' in this block; low_sum: the value named 'low sum' in this block. Context: the simulation task check counter add.

Line 339: `};` This is continuation line 4 of the 4-line logical statement that begins at source line 336. Its exact fragment is '};'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This makes actual, the value named 'actual' in this block, equal to '{ value[63:24] + {{39{1'b0}},

low_sum[24]}, low_sum[23:0] }‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named ’value’ in this block; b0: the value named ’b0’ in this block; low_sum: the value named ’low sum’ in this block. Context: the simulation task check counter add.

Line 340: `expected = value + {{45{1'b0}}, increment};` This makes expected, the value named ’expected’ in this block, equal to ‘value + {{45{1’b0}}, increment}‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named ’value’ in this block; b0: the value named ’b0’ in this block; increment: the value named ’increment’ in this block. Context: the simulation task check counter add.

Line 341: `if (actual !== expected) begin` This condition selects the following hardware branch only when ‘actual !== expected‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are actual: the value named ’actual’ in this block; expected: the value named ’expected’ in this block. Context: the simulation task check counter add.

Line 342: `errors++;` Within the simulation task check counter add, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 343: `$display("ERROR counter add value=%h increment=%h actual=%h expected=%h",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 344: `value, increment, actual, expected);` Within the simulation task check counter add, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are value: the value named ’value’ in this block; increment: the value named ’increment’ in this block; actual: the value named ’actual’ in this block.

Line 345: `end` This structural keyword opens or closes the branch/process scope described as the simulation task check counter add; it has no arithmetic value of its own.

Line 346: `end` This structural keyword opens or closes the branch/process scope described as the simulation task check counter add; it has no arithmetic value of its own.

Line 347: `endtask` Within the simulation task check counter add, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are endtask: the value named ’endtask’ in this block.

Line 349: `task automatic verify_counter_adder;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are task: the value named ’task’ in this block; automatic: the value named ’automatic’ in this block; verify_counter_adder: a hardware telemetry register that accumulates measured events.

Line 350: `logic [63:0] random_value;` This declares hardware signal storage or wiring in the simulation task task. Specifically, random_value is the value named ’random value’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 351: `logic [18:0] random_increment;` This declares hardware signal storage or wiring in the simulation task task. Specifically, random_increment is the value named ’random increment’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 352: `int unsigned random_state;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are random_state: the finite-state-machine phase retained in flip-flops.

Line 353: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 354: `check_counter_add(64'd0, 19'd0);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are check_counter_add: a hardware telemetry register that accumulates measured events; d0: the value named 'd0' in this block.

Line 355: `check_counter_add(64'd0, 19'h7ffff);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are check_counter_add: a hardware telemetry register that accumulates measured events; d0: the value named 'd0' in this block; h7ffff: the value named 'h7ffff' in this block.

Line 356: `check_counter_add(64'h0000000000ffffff, 19'd1);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are check_counter_add: a hardware telemetry register that accumulates measured events; h0000000000ffffff: the value named 'h0000000000ffffff' in this block; d1: the value named 'd1' in this block.

Line 357: `check_counter_add(64'h00000000ffffffff, 19'd1);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are check_counter_add: a hardware telemetry register that accumulates measured events; h00000000ffffffff: the value named 'h00000000ffffffff' in this block; d1: the value named 'd1' in this block.

Line 358: `check_counter_add(64'h0000fffffffffffff, 19'd1);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are check_counter_add: a hardware telemetry register that accumulates measured events; h0000fffffffffff: the value named 'h0000fffffffffff' in this block; d1: the value named 'd1' in this block.

Line 359: `check_counter_add(64'h00fffffffffffff, 19'd1);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are check_counter_add: a hardware telemetry register that accumulates measured events; h00fffffffffffff: the value named 'h00fffffffffffff' in this block; d1: the value named 'd1' in this block.

Line 360: `check_counter_add(64'hffffffffffffffff, 19'h7ffff);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are check_counter_add: a hardware telemetry register that accumulates measured events; hffffffffffffff: the value named 'hffffffffffffff' in this block; h7ffff: the value named 'h7ffff' in this block.

Line 361: `check_counter_add(64'h55fffffffffffeaa, 19'h10155);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are check_counter_add: a hardware telemetry register that accumulates measured events; h55fffffffffeaa: the value named 'h55fffffffffeaa' in this block; h10155: the value named 'h10155' in this block.

Line 363: `random_state = 32'h4e455552;` This makes random_state, the finite-state-machine phase retained in flip-flops, equal to '32'h4e455552'. It is driven by combinational or procedural logic in the current evaluation. The important operands are h4e455552: the value named 'h4e455552' in this block. Context: the simulation task task.

Line 364: `for (int vector_index = 0; vector_index < 1024;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 365: `vector_index++) begin` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are vector_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 366: `random_state = random_state * 32'd1664525 + 32'd1013904223;` This makes random_state, the finite-state-machine phase retained in flip-flops, equal to 'random_state * 32'd1664525 + 32'd1013904223'. It is driven by combinational or procedural logic in the current evaluation. The important operands are random_state: the finite-state-machine phase retained in flip-flops; d1664525: the value named 'd1664525' in this block; d1013904223: the value named 'd1013904223' in this block. Context: the simulation task task.

Line 367: `random_value[31:0] = random_state;` This makes random_value[31:0], the value named 'random value' in this block, equal to 'random_state'. It is driven by combinational or procedural logic in the current evaluation. The important operands are random_state: the finite-state-machine phase retained in flip-flops. Context: the simulation task task.

Line 368: `random_state = random_state * 32'd1664525 + 32'd1013904223;` This makes random_state, the finite-state-machine phase retained in flip-flops, equal to 'random_state * 32'd1664525 + 32'd1013904223'. It is driven by combinational or procedural logic in the current evaluation. The important operands are random_state: the finite-state-machine phase retained in flip-flops; d1664525: the value named 'd1664525' in this block; d1013904223: the value named 'd1013904223' in this block. Context: the simulation task task.

Line 369: `random_value[63:32] = random_state;` This makes random_value[63:32], the value named 'random value' in this block, equal to 'random_state'. It is driven by combinational or procedural logic in the current evaluation. The important operands are random_state: the finite-state-machine phase retained in flip-flops. Context: the simulation task task.

Line 370: `random_state = random_state * 32'd1664525 + 32'd1013904223;` This makes random_state, the finite-state-machine phase retained in flip-flops, equal to 'random_state * 32'd1664525 + 32'd1013904223'. It is driven by combinational or procedural logic in the current evaluation. The important operands are random_state: the finite-state-machine phase retained in flip-flops; d1664525: the value named 'd1664525' in this block; d1013904223: the value named 'd1013904223' in this block. Context: the simulation task task.

Line 371: `random_increment = random_state[18:0];` This makes random_increment, the value named 'random increment' in this block, equal to 'random_state[18:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are random_state: the finite-state-machine phase retained in flip-flops. Context: the simulation task task.

Line 372: `check_counter_add(random_value, random_increment);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are check_counter_add: a hardware telemetry register that accumulates measured events; random_value: the value named 'random value' in this block; random_increment: the value named 'random increment' in this block.

Line 373: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 374: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 375: `endtask` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 377: `always @(posedge clk) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are always: the value named 'always' in this block; posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 378: `cycle <= cycle + 1;` This makes cycle, the value named 'cycle' in this block, equal to 'cycle + 1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are cycle: the value named 'cycle' in this block. Context: the module's structural declarations.

Line 379: `if (held_valid && (!m_axis_tvalid || m_axis_tdata !== held_data` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are held_valid: the producer's statement that its current payload is meaningful; m_axis_tvalid: the producer's statement that its current payload is meaningful; m_axis_tdata: the value named 'm axis tdata' in this block.

Line 380: `|| m_axis_tlast !== held_last)) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; held_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 381: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 382: `$display("ERROR buffered output changed while backpressured");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 383: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 384: `held_valid <= m_axis_tvalid && !m_axis_tready;` This makes held_valid, the producer's statement that its current payload is meaningful, equal to 'm_axis_tvalid && !m_axis_tready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are m_axis_tvalid: the producer's statement that its current payload is meaningful; m_axis_tready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 385: `held_data <= m_axis_tdata;` This makes held_data, the payload carried with the surrounding valid/ready handshake, equal to 'm_axis_tdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are m_axis_tdata: the value named 'm axis tdata' in this block. Context: the module's structural declarations.

Line 386: `held_last <= m_axis_tlast;` This makes held_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'm_axis_tlast'. It updates after the active clock edge together with other nonblocking assignments. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the module's structural declarations.

Line 387: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 389: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 390: `$readmemh("tb/fixtures/tensor_cases.mem", case_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 391: `$readmemh("tb/fixtures/tensor_activations.mem", activation_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 392: `$readmemh("tb/fixtures/tensor_weights.mem", weight_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 393: `$readmemh("tb/fixtures/tensor_expected.mem", expected_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 395: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 396: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 397: `repeat (2) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 399: `verify_counter_adder();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are verify_counter_adder: a hardware telemetry register that accumulates measured events.

Line 401: `for (int case_index = 0; case_index < CASES; case_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 402: `run_fixture_case(case_index);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are run_fixture_case: the value named 'run fixture case' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 403: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 405: `if (issue_pointer != ISSUES || expected_pointer != RESULTS || checked != RESULTS) begin` This condition selects the following hardware branch only when 'issue_pointer != ISSUES || expected_pointer != RESULTS || checked != RESULTS' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are issue_pointer: the value named 'issue pointer' in this block; ISSUES: the value named 'issues' in this block; expected_pointer: the value named 'expected pointer' in this block. Context: the module's structural declarations.

Line 406: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 407: `$display("ERROR buffered accounting issue=%0d/%0d expected=%0d/%0d checked=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 408: `issue_pointer, ISSUES, expected_pointer, RESULTS, checked);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are issue_pointer: the value named 'issue pointer' in this block; ISSUES: the value named 'issues' in this block; expected_pointer: the value named 'expected pointer' in this block.

Line 409: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 411: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 412: `$display("tb_neuraedge_buffered_tensor_core FAILED errors=%0d checked=%0d", errors, checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 413: `$fatal(1);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 414: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 415: `$display("tb_neuraedge_buffered_tensor_core PASSED cases=%0d checked=%0d", CASES, checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 416: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 417: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 418: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 420: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 421: `#200_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 422: `$display("ERROR buffered core watchdog state=%0d busy=%0d cmd_ready=%0d s_ready=%0d m_valid=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 423: `dut.state, status_busy, command_ready, s_axis_tready, m_axis_tvalid);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation. The important operands are dut: the value named 'dut' in this block; state: the finite-state-machine phase retained in flip-flops; status_busy: the value named 'status busy' in this block.

Line 424: `$fatal(1);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 425: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 426: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 428: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge buffered tensor core RTL block in simulation.

49 tb/tb_neuraedge_descriptor_sequencer.sv

Role in the complete system: drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

49.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Simulation-only status tb_neuraedge_descriptor_sequencer is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

49.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module tb_neuraedge_descriptor_sequencer;
5      logic clk = 1'b0;
6      logic rst_n = 1'b0;
7      logic program_start = 1'b0;
8      logic clear_status = 1'b0;
9      logic [31:0] program_output_beats = 32'd0;
10     logic program_active;
11     logic program_done;
12     logic program_error;
13     logic [31:0] program_records;
14     logic s_axis_tvalid = 1'b0;
15     logic s_axis_tready;
16     logic [63:0] s_axis_tdata = 64'd0;
17     logic s_axis_tlast = 1'b0;
18     logic m_axis_tvalid;
19     logic m_axis_tready = 1'b1;
20     logic [63:0] m_axis_tdata;
21     logic m_axis_tlast;
22     logic core_command_valid;
23     logic core_command_ready;
24     logic [127:0] core_command_data;
25     logic core_command_done = 1'b0;
26     logic core_status_error = 1'b0;
27     logic core_s_valid;
28     logic core_s_ready;
29     logic [63:0] core_s_data;
30     logic core_s_last;
31     logic core_m_valid;
32     logic core_m_ready;
33     logic [63:0] core_m_data;
34     logic core_m_last;
35
36     typedef enum logic [1:0] {
37         CORE_IDLE = 2'd0,
38         CORE_INPUT = 2'd1,
39         CORE_OUTPUT = 2'd2,
40         CORE_DONE = 2'd3
41     } core_state_t;
42     core_state_t core_state = CORE_IDLE;
43     logic core_output_index = 1'b0;
44     int errors = 0;
45     int output_count = 0;
46     int program_output_index = 0;
47     int expected_program_outputs = 2;
48     int input_count = 0;
49
50     neuraedge_descriptor_sequencer dut (
51         .clk(clk), .rst_n(rst_n),
52         .program_start(program_start),
53         .clear_status(clear_status),
54         .program_output_beats(program_output_beats),
55         .program_active(program_active), .program_done(program_done),
56         .program_error(program_error), .program_records(program_records),
57         .s_axis_tvalid(s_axis_tvalid), .s_axis_tready(s_axis_tready),
```

```
        .s_axis_tdata(s_axis_tdata), .s_axis_tlast(s_axis_tlast),
        .m_axis_tvalid(m_axis_tvalid), .m_axis_tready(m_axis_tready),
        .m_axis_tdata(m_axis_tdata), .m_axis_tlast(m_axis_tlast),
        .core_command_valid(core_command_valid),
        .core_command_ready(core_command_ready),
        .core_command_data(core_command_data),
        .core_command_done(core_command_done),
        .core_status_error(core_status_error),
        .core_s_valid(core_s_valid), .core_s_ready(core_s_ready),
        .core_s_data(core_s_data), .core_s_last(core_s_last),
        .core_m_valid(core_m_valid), .core_m_ready(core_m_ready),
        .core_m_data(core_m_data), .core_m_last(core_m_last)
    );

    always #2.5 clk = ~clk;

    always_comb begin
        core_command_ready = core_state == CORE_IDLE;
        core_s_ready = core_state == CORE_INPUT;
        core_m_valid = core_state == CORE_OUTPUT;
        core_m_data = core_output_index ? 64'h2222 : 64'h1111;
        core_m_last = core_output_index;
    end

    always @(posedge clk) begin
        core_command_done <= 1'b0;
        if (!rst_n) begin
            core_state <= CORE_IDLE;
            core_output_index <= 1'b0;
        end else begin
            unique case (core_state)
                CORE_IDLE: if (core_command_valid && core_command_ready) begin
                    unique case (core_command_data[3:0])
                        4'h1: core_state <= CORE_INPUT;
                        4'h4: begin
                            core_output_index <= 1'b0;
                            core_state <= CORE_OUTPUT;
                        end
                        default: core_state <= CORE_DONE;
                    endcase
                end
                CORE_INPUT: if (core_s_valid && core_s_ready) begin
                    input_count <= input_count + 1;
                    if ((input_count == 0 && core_s_last)
                            || (input_count == 1 && !core_s_last)) begin
                        errors <= errors + 1;
                    end
                    if (core_s_last) core_state <= CORE_DONE;
                end
                CORE_OUTPUT: if (core_m_valid && core_m_ready) begin
                    if (core_output_index) core_state <= CORE_DONE;
                    else core_output_index <= 1'b1;
                end
                CORE_DONE: begin
                    core_command_done <= 1'b1;
                    core_state <= CORE_IDLE;
                end
                default: core_state <= CORE_IDLE;
            endcase
        end
    end

    always @(posedge clk) begin
        if (m_axis_tvalid && m_axis_tready) begin
            if (m_axis_tdata !== (program_output_index == 0 ? 64'h1111 : 64'h2222)
                    || m_axis_tlast !== (program_output_index
                        == expected_program_outputs - 1)) begin
                errors <= errors + 1;
                $display("ERROR output=%0d data=%h last=%0d",
                         program_output_index, m_axis_tdata, m_axis_tlast);
            end
            output_count <= output_count + 1;
            program_output_index <= program_output_index + 1;
        end
    end

    task automatic send_word(input logic [63:0] data, input logic last);
        begin
            while (!s_axis_tready) @(negedge clk);
            s_axis_tdata = data;
            s_axis_tlast = last;
            s_axis_tvalid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            s_axis_tvalid = 1'b0;
            s_axis_tlast = 1'b0;
        end
```

```
    endtask

    initial begin
        repeat (5) @(negedge clk);
        rst_n = 1'b1;
        @(negedge clk);
        program_output_beats = 2;
        program_start = 1'b1;
        @(posedge clk);
        @(negedge clk);
        program_start = 1'b0;

        // Load command: two payload beats, no output, not the final record.
        send_word({1'b0, 31'd0, 32'd2}, 1'b0);
        send_word(64'h1, 1'b0);
        send_word(64'd0, 1'b0);
        send_word(64'habc, 1'b0);
        send_word(64'hdef, 1'b0);

        // Compute command: no input and two outputs. It is deliberately not
        // the final input record; global output count must still generate TLAST.
        send_word({1'b0, 31'd2, 32'd0}, 1'b0);
        send_word(64'h4, 1'b0);
        send_word(64'd0, 1'b0);

        // A trailing no-output command proves output TLAST does not depend on
        // which descriptor carries the final input marker.
        send_word({1'b1, 31'd0, 32'd0}, 1'b0);
        send_word(64'hf, 1'b0);
        send_word(64'd0, 1'b1);

        wait (program_done);
        @(negedge clk);
        if (program_error || program_active || program_records != 3
                || input_count != 2 || output_count != 2) begin
            errors++;
            $display("ERROR final active=%0d error=%0d records=%0d input=%0d output=%0d",
                     program_active, program_error, program_records,
                     input_count, output_count);
        end

        // Declare one output for a core command that produces two. The first
        // beat is legal and gets TLAST; the second must be discarded while the
        // descriptor sequencer reports an overrun instead of deadlocking.
        @(negedge clk);
        program_output_index = 0;
        expected_program_outputs = 1;
        program_output_beats = 1;
        program_start = 1'b1;
        @(posedge clk);
        @(negedge clk);
        program_start = 1'b0;
        send_word({1'b1, 31'd1, 32'd0}, 1'b0);
        send_word(64'h4, 1'b0);
        send_word(64'd0, 1'b1);
        wait (program_done);
        @(negedge clk);
        if (!program_error || program_active || program_records != 1
                || output_count != 3) begin
            errors++;
            $display("ERROR overrun active=%0d error=%0d records=%0d output=%0d",
                     program_active, program_error, program_records, output_count);
        end
        if (errors != 0) begin
            $display("tb_neuraedge_descriptor_sequencer FAILED errors=%0d", errors);
        end else begin
            $display("tb_neuraedge_descriptor_sequencer PASSED records=%0d", program_records);
        end
        $finish;
    end

    initial begin
        #200_000;
        $fatal(1, "descriptor sequencer watchdog timeout");
    end
endmodule

`default_nettype wire
```

49.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `‘default_nettype none` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module tb_neuraedge_descriptor_sequencer;` This opens tb_neuraedge_descriptor_sequencer;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Line 5: `logic clk = 1’b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 6: `logic rst_n = 1’b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 7: `logic program_start = 1’b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_start is the value named 'program start' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 8: `logic clear_status = 1’b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clear_status is the value named 'clear status' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic [31:0] program_output_beats = 32’d0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_output_beats is the value named 'program output beats' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic program_active;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_active is the value named 'program active' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic program_done;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_done is the value named 'program done' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic program_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_error is the value named 'program error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic [31:0] program_records;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, program_records is the number of descriptor records accepted since program start. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic s_axis_tvalid = 1’b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic s_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic [63:0] s_axis_tdata = 64’d0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tdata is the value named 's axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic s_axis_tlast = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic m_axis_tvalid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic m_axis_tready = 1'b1;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic [63:0] m_axis_tdata;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tdata is the value named 'm axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic m_axis_tlast;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic core_command_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_command_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic core_command_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_command_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic [127:0] core_command_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_command_data is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic core_command_done = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_command_done is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `logic core_status_error = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_status_error is the value named 'core status error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic core_s_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 28: `logic core_s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `logic [63:0] core_s_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic core_s_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_s_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `logic core_m_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 32: `logic core_m_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `logic [63:0] core_m_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic core_m_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_m_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `typedef enum logic [1:0] {` This begins a named finite-state-machine type. The encoded bits are stored in flip-flops, while symbolic names make each cycle-level phase reviewable.

Line 37: `CORE_IDLE = 2'd0,` This assigns a fixed binary encoding to state CORE_IDLE, which waits for a new operation and does not consume payload data. The encoding is visible in waveforms even though the source uses the readable name.

Line 38: `CORE_INPUT = 2'd1,` This assigns a fixed binary encoding to state CORE_INPUT, which identifies one explicit control phase. The encoding is visible in waveforms even though the source uses the readable name.

Line 39: `CORE_OUTPUT = 2'd2,` This assigns a fixed binary encoding to state CORE_OUTPUT, which presents computed results under valid/ready backpressure. The encoding is visible in waveforms even though the source uses the readable name.

Line 40: `CORE_DONE = 2'd3` This statement continues through source line 41; the explanation uses the complete expression rather than only the first visual fragment. This assigns a fixed binary encoding to state CORE_DONE, which raises completion for one cycle and returns control to idle. The encoding is visible in waveforms even though the source uses the readable name.

Line 41: `} core_state_t;` This is continuation line 2 of the 2-line logical statement that begins at source line 40. Its exact fragment is '} core_state_t;'. The important operands are core_state_t: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns a fixed binary encoding to state CORE_DONE, which raises completion for one cycle and returns control to idle. The encoding is visible in waveforms even though the source uses the readable name.

Line 42: `core_state_t core_state = CORE_IDLE;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are core_state_t: the finite-state-machine phase retained in flip-flops; core_state: the finite-state-machine phase retained in flip-flops; CORE_IDLE: the value named 'core idle' in this block.

Line 43: `logic core_output_index = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, core_output_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 44: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 45: `int output_count = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are output_count: an integer count retained for control or measurable telemetry.

Line 46: `int program_output_index = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are program_output_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 47: `int expected_program_outputs = 2;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are expected_program_outputs: the value named 'expected program outputs' in this block.

Line 48: `int input_count = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are input_count: an integer count retained for control or measurable telemetry.

Line 50: `neuraedge_descriptor_sequencer dut (` This instantiates module neuraedge_descriptor_sequencer as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 51: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 52: `.program_start(program_start),` This connects the child instance's named port 'program_start' to parent expression 'program_start'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'program start' in this block.

Line 53: `.clear_status(clear_status),` This connects the child instance's named port 'clear_status' to parent expression 'clear_status'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'clear status' in this block.

Line 54: `.program_output_beats(program_output_beats),` This connects the child instance's named port 'program_output_beats' to parent expression 'program_output_beats'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'program output beats' in this block.

Line 55: `.program_active(program_active), .program_done(program_done),` This connects the child instance's named port 'program_active' to parent expression 'program_active), .program_done(program_done'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'program active), .program done(program done' in this block.

Line 56: `.program_error(program_error), .program_records(program_records),` This connects the child instance's named port 'program_error' to parent expression 'program_error), .program_records(program_records'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the number of descriptor records accepted since program start.

Line 57: `.s_axis_tvalid(s_axis_tvalid), .s_axis_tready(s_axis_tready),` This connects the child instance's named port 's_axis_tvalid' to parent expression 's_axis_tvalid), .s_axis_tready(s_axis_tready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 58: `.s_axis_tdata(s_axis_tdata), .s_axis_tlast(s_axis_tlast),` This connects the child instance's named port ‘s_axis_tdata‘ to parent expression ‘s_axis_tdata), .s_axis_tlast(s_axis_tlast‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 59: `.m_axis_tvalid(m_axis_tvalid), .m_axis_tready(m_axis_tready),` This connects the child instance's named port ‘m_axis_tvalid‘ to parent expression ‘m_axis_tvalid), .m_axis_tready(m_axis_tready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 60: `.m_axis_tdata(m_axis_tdata), .m_axis_tlast(m_axis_tlast),` This connects the child instance's named port ‘m_axis_tdata‘ to parent expression ‘m_axis_tdata), .m_axis_tlast(m_axis_tlast‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 61: `.core_command_valid(core_command_valid),` This connects the child instance's named port ‘core_command_valid‘ to parent expression ‘core_command_valid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 62: `.core_command_ready(core_command_ready),` This connects the child instance's named port ‘core_command_ready‘ to parent expression ‘core_command_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 63: `.core_command_data(core_command_data),` This connects the child instance's named port ‘core_command_data‘ to parent expression ‘core_command_data‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 64: `.core_command_done(core_command_done),` This connects the child instance's named port ‘core_command_done‘ to parent expression ‘core_command_done‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 65: `.core_status_error(core_status_error),` This connects the child instance's named port ‘core_status_error‘ to parent expression ‘core_status_error‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'core status error' in this block.

Line 66: `.core_s_valid(core_s_valid), .core_s_ready(core_s_ready),` This connects the child instance's named port ‘core_s_valid‘ to parent expression ‘core_s_valid), .core_s_ready(core_s_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 67: `.core_s_data(core_s_data), .core_s_last(core_s_last),` This connects the child instance's named port ‘core_s_data‘ to parent expression ‘core_s_data), .core_s_last(core_s_last‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 68: `.core_m_valid(core_m_valid), .core_m_ready(core_m_ready),` This connects the child instance's named port ‘core_m_valid‘ to parent expression ‘core_m_valid), .core_m_ready(core_m_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 69: `.core_m_data(core_m_data), .core_m_last(core_m_last)` This connects the child instance's named port ‘core_m_data‘ to parent expression ‘core_m_data), .core_m_last(core_m_last‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 70: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 72: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 74: `always_comb begin` This opens combinational logic. Vivado builds gates and multiplexers whose outputs continuously follow the inputs, and the language requires every assigned output to receive a value on every path.

Line 75: `core_command_ready = core_state == CORE_IDLE;` This makes core_command_ready, the consumer's statement that it can accept a payload this cycle, equal to 'core_state == CORE_IDLE'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_state: the finite-state-machine phase retained in flip-flops; CORE_IDLE: the value named 'core idle' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 76: `core_s_ready = core_state == CORE_INPUT;` This makes core_s_ready, the consumer's statement that it can accept a payload this cycle, equal to 'core_state == CORE_INPUT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_state: the finite-state-machine phase retained in flip-flops; CORE_INPUT: the value named 'core input' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 77: `core_m_valid = core_state == CORE_OUTPUT;` This makes core_m_valid, the producer's statement that its current payload is meaningful, equal to 'core_state == CORE_OUTPUT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_state: the finite-state-machine phase retained in flip-flops; CORE_OUTPUT: the value named 'core output' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 78: `core_m_data = core_output_index ? 64'h2222 : 64'h1111;` This makes core_m_data, the payload carried with the surrounding valid/ready handshake, equal to 'core_output_index ? 64'h2222 : 64'h1111'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_output_index: an integer selecting one tensor element, lane, bank, or state entry; h2222: the value named 'h2222' in this block; h1111: the value named 'h1111' in this block. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 79: `core_m_last = core_output_index;` This makes core_m_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'core_output_index'. It is driven by combinational or procedural logic in the current evaluation. The important operands are core_output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the combinational process whose outputs respond without waiting for a clock edge.

Line 80: `end` This structural keyword opens or closes the branch/process scope described as the combinational process whose outputs respond without waiting for a clock edge; it has no arithmetic value of its own.

Line 82: `always @(posedge clk) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are always: the value named 'always' in this block; posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 83: `core_command_done <= 1'b0;` This makes core_command_done, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 84: `if (!rst_n) begin` This is the reset branch for '(!rst_n'. While reset is asserted, the following assignments force control state, handshakes, counters, and error flags to known values so no stale pre-reset transaction can be mistaken for valid work. Context: the module's structural declarations.

Line 85: `core_state <= CORE_IDLE;` This makes core_state, the finite-state-machine phase retained in flip-flops, equal to 'CORE_IDLE'. It updates after the active clock edge together with other nonblocking assignments. The important operands are CORE_IDLE: the value named 'core idle' in this block. Context: the module's structural declarations.

Line 86: `core_output_index <= 1'b0;` This makes core_output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 87: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 88: `unique case (core_state)` This starts a multiplexer/state decode selected by 'core_state'. 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 89: `CORE_IDLE: if (core_command_valid && core_command_ready) begin` This is the CORE_IDLE case branch, which waits for a new operation and does not consume payload data. The same-cycle action begins with 'if (core_command_valid && core_command_ready) begin'; context is the module's structural declarations Within state CORE_IDLE, the machine waits for a new operation and does not consume payload data..

Line 90: `unique case (core_command_data[3:0])` This starts a multiplexer/state decode selected by 'core_command_data[3:0 'unique' allows tools to diagnose overlapping or missing legal choices and optimize on the expectation that one branch matches.

Line 91: `4'h1: core_state <= CORE_INPUT;` Within the module's structural declarations Within state CORE_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are h1: the value named 'h1' in this block; core_state: the finite-state-machine phase retained in flip-flops; CORE_INPUT: the value named 'core input' in this block.

Line 92: `4'h4: begin` Within the module's structural declarations Within state CORE_IDLE, the machine waits for a new operation and does not consume payload data., this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are h4: the value named 'h4' in this block.

Line 93: `core_output_index <= 1'b0;` This makes core_output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '1'b0'. It updates after the active clock edge together with other nonblocking assignments. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations Within state CORE_IDLE, the machine waits for a new operation and does not consume payload data..

Line 94: `core_state <= CORE_OUTPUT;` This makes core_state, the finite-state-machine phase retained in flip-flops, equal to 'CORE_OUTPUT'. It updates after the active clock edge together with other nonblocking assignments. The important operands are CORE_OUTPUT: the value named 'core output' in this block. Context: the module's structural declarations Within state CORE_IDLE, the machine waits for a new operation and does not consume payload data..

Line 95: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations Within state CORE_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 96: `default: core_state <= CORE_DONE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘core_state <= CORE_DONE;‘; context is the module’s structural declarations Within state CORE_IDLE, the machine waits for a new operation and does not consume payload data..

Line 97: `endcase` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations Within state CORE_IDLE, the machine waits for a new operation and does not consume payload data.; it has no arithmetic value of its own.

Line 98: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 99: `CORE_INPUT: if (core_s_valid && core_s_ready) begin` This is the CORE_INPUT case branch, which performs this explicitly named phase. The same-cycle action begins with ‘if (core_s_valid && core_s_ready) begin‘; context is the module’s structural declarations Within state CORE_INPUT, the machine performs the named control phase..

Line 100: `input_count <= input_count + 1;` This makes input_count, an integer count retained for control or measurable telemetry, equal to ‘input_count + 1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are input_count: an integer count retained for control or measurable telemetry. Context: the module’s structural declarations Within state CORE_INPUT, the machine performs the named control phase..

Line 101: `if ((input_count == 0 && core_s_last)` This condition selects the following hardware branch only when ‘(input_count == 0 && core_s_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_count: an integer count retained for control or measurable telemetry; core_s_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the module’s structural declarations Within state CORE_INPUT, the machine performs the named control phase..

Line 102: `|| (input_count == 1 && !core_s_last)) begin` Within the module’s structural declarations Within state CORE_INPUT, the machine performs the named control phase., this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are input_count: an integer count retained for control or measurable telemetry; core_s_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 103: `errors <= errors + 1;` This makes errors, the value named ’errors’ in this block, equal to ‘errors + 1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are errors: the value named ’errors’ in this block. Context: the module’s structural declarations Within state CORE_INPUT, the machine performs the named control phase..

Line 104: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations Within state CORE_INPUT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 105: `if (core_s_last) core_state <= CORE_DONE;` This condition selects the following hardware branch only when ‘core_s_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are core_s_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the module’s structural declarations Within state CORE_INPUT, the machine performs the named control phase..

Line 106: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations Within state CORE_INPUT, the machine performs the named control phase.; it has no arithmetic value of its own.

Line 107: `CORE_OUTPUT: if (core_m_valid && core_m_ready) begin` This is the CORE_OUTPUT case branch, which presents computed results under valid/ready backpressure. The same-cycle action begins with ‘if (core_m_valid && core_m_ready) begin‘; context is the module’s structural declarations Within state CORE_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 108: `if (core_output_index) core_state <= CORE_DONE;` This condition selects the following hardware branch only when ‘core_output_index‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are core_output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module’s structural declarations Within state CORE_OUTPUT, the machine presents computed results under valid/ready backpressure..

Line 109: `else core_output_index <= 1'b1;` Within the module’s structural declarations Within state CORE_OUTPUT, the machine presents computed results under valid/ready backpressure., this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are core_output_index: an integer selecting one tensor element, lane, bank, or state entry; b1: the value named ’b1’ in this block.

Line 110: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations Within state CORE_OUTPUT, the machine presents computed results under valid/ready backpressure.; it has no arithmetic value of its own.

Line 111: `CORE_DONE: begin` This is the CORE_DONE case branch, which raises completion for one cycle and returns control to idle. The same-cycle action begins with ‘begin‘; context is the module’s structural declarations Within state CORE_DONE, the machine raises completion for one cycle and returns control to idle..

Line 112: `core_command_done <= 1'b1;` This makes core_command_done, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘1’b1‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are b1: the value named ’b1’ in this block. Context: the module’s structural declarations Within state CORE_DONE, the machine raises completion for one cycle and returns control to idle..

Line 113: `core_state <= CORE_IDLE;` This makes core_state, the finite-state-machine phase retained in flip-flops, equal to ‘CORE_IDLE‘. It updates after the active clock edge together with other nonblocking assignments. The important operands are CORE_IDLE: the value named ’core idle’ in this block. Context: the module’s structural declarations Within state CORE_DONE, the machine raises completion for one cycle and returns control to idle..

Line 114: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations Within state CORE_DONE, the machine raises completion for one cycle and returns control to idle.; it has no arithmetic value of its own.

Line 115: `default: core_state <= CORE_IDLE;` This is the default case branch, which performs this explicitly named phase. The same-cycle action begins with ‘core_state <= CORE_IDLE;‘; context is the module’s structural declarations Within state CORE_DONE, the machine raises completion for one cycle and returns control to idle..

Line 116: `endcase` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations Within state CORE_DONE, the machine raises completion for one cycle and returns control to idle.; it has no arithmetic value of its own.

Line 117: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 118: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 120: `always @(posedge clk) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are always: the value named 'always' in this block; posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 121: `if (m_axis_tvalid && m_axis_tready) begin` This condition selects the following hardware branch only when 'm_axis_tvalid && m_axis_tready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are m_axis_tvalid: the producer's statement that its current payload is meaningful; m_axis_tready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 122: `if (m_axis_tdata !== (program_output_index == 0 ? 64'h1111 : 64'h2222)` This condition selects the following hardware branch only when 'm_axis_tdata !== (program_output_index == 0 ? 64'h1111 : 64'h2222' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are m_axis_tdata: the value named 'm axis tdata' in this block; program_output_index: an integer selecting one tensor element, lane, bank, or state entry; h1111: the value named 'h1111' in this block. Context: the module's structural declarations.

Line 123: `|| m_axis_tlast !== (program_output_index` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; program_output_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 124: `== expected_program_outputs - 1)) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are expected_program_outputs: the value named 'expected program outputs' in this block.

Line 125: `errors <= errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 126: `$display("ERROR output=%0d data=%h last=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 127: `program_output_index, m_axis_tdata, m_axis_tlast);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are program_output_index: an integer selecting one tensor element, lane, bank, or state entry; m_axis_tdata: the value named 'm axis tdata' in this block; m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload.

Line 128: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 129: `output_count <= output_count + 1;` This makes output_count, an integer count retained for control or measurable telemetry, equal to 'output_count + 1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_count: an integer count retained for control or measurable telemetry. Context: the module's structural declarations.

Line 130: `program_output_index <= program_output_index + 1;` This makes program_output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'program_output_index + 1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are program_output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 131: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 132: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 134: `task automatic send_word(input logic [63:0] data, input logic last);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; send_word: the value named 'send word' in this block.

Line 135: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task send word; it has no arithmetic value of its own.

Line 136: `while (!s_axis_tready) @(negedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axis_tready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 137: `s_axis_tdata = data;` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to 'data'. It is driven by combinational or procedural logic in the current evaluation. The important operands are data: the value named 'data' in this block. Context: the simulation task send word.

Line 138: `s_axis_tlast = last;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'last'. It is driven by combinational or procedural logic in the current evaluation. The important operands are last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the simulation task send word.

Line 139: `s_axis_tvalid = 1'b1;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task send word.

Line 140: `@(posedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 141: `@(negedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 142: `s_axis_tvalid = 1'b0;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send word.

Line 143: `s_axis_tlast = 1'b0;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send word.

Line 144: `end` This structural keyword opens or closes the branch/process scope described as the simulation task send word; it has no arithmetic value of its own.

Line 145: `endtask` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 147: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 148: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 149: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 150: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 151: `program_output_beats = 2;` This makes program_output_beats, the value named 'program output beats' in this block, equal to '2'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 152: `program_start = 1'b1;` This makes program_start, the value named 'program start' in this block, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 153: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 154: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 155: `program_start = 1'b0;` This makes program_start, the value named 'program start' in this block, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 157: `// Load command: two payload beats, no output, not the final record.` Design intent: Load command: two payload beats, no output, not the final record. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Line 158: `send_word({1'b0, 31'd0, 32'd2}, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; b0: the value named 'b0' in this block; d0: the value named 'd0' in this block.

Line 159: `send_word(64'h1, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h1: the value named 'h1' in this block; b0: the value named 'b0' in this block.

Line 160: `send_word(64'd0, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; d0: the value named 'd0' in this block; b0: the value named 'b0' in this block.

Line 161: `send_word(64'habc, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; habc: the value named 'habc' in this block; b0: the value named 'b0' in this block.

Line 162: `send_word(64'hdef, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; hdef: the value named 'hdef' in this block; b0: the value named 'b0' in this block.

Line 164: `// Compute command: no input and two outputs. It is deliberately not` Design intent: Compute command: no input and two outputs. It is deliberately not This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Line 165: `// the final input record; global output count must still generate TLAST.` Design intent: the final input record; global output count must still generate TLAST. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Line 166: `send_word({1'b0, 31'd2, 32'd0}, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; b0: the value named 'b0' in this block; d2: the value named 'd2' in this block.

Line 167: `send_word(64'h4, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h4: the value named 'h4' in this block; b0: the value named 'b0' in this block.

Line 168: `send_word(64'd0, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; d0: the value named 'd0' in this block; b0: the value named 'b0' in this block.

Line 170: `// A trailing no-output command proves output TLAST does not depend on` Design intent: A trailing no-output command proves output TLAST does not depend on This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Line 171: `// which descriptor carries the final input marker.` Design intent: which descriptor carries the final input marker. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Line 172: `send_word({1'b1, 31'd0, 32'd0}, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; b1: the value named 'b1' in this block; d0: the value named 'd0' in this block.

Line 173: `send_word(64'hf, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; hf: the value named 'hf' in this block; b0: the value named 'b0' in this block.

Line 174: `send_word(64'd0, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; d0: the value named 'd0' in this block; b1: the value named 'b1' in this block.

Line 176: `wait (program_done);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are wait: the value named 'wait' in this block; program_done: the value named 'program done' in this block.

Line 177: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 178: `if (program_error || program_active || program_records != 3` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are program_error: the value named 'program error' in this block; program_active: the value named 'program active' in this block; program_records: the number of descriptor records accepted since program start.

Line 179: `|| input_count != 2 || output_count != 2) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are input_count: an integer count retained for control or measurable telemetry; output_count: an integer count retained for control or measurable telemetry.

Line 180: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 181: `$display("ERROR final active=%0d error=%0d records=%0d input=%0d output=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 182: `program_active, program_error, program_records,` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are program_active: the value named 'program active' in this block; program_error: the value named 'program error' in this block; program_records: the number of descriptor records accepted since program start.

Line 183: `input_count, output_count);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are input_count: an integer count retained for control or measurable telemetry; output_count: an integer count retained for control or measurable telemetry.

Line 184: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 186: `// Declare one output for a core command that produces two. The first` Design intent: Declare one output for a core command that produces two. The first This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Line 187: `// beat is legal and gets TLAST; the second must be discarded while the` Design intent: beat is legal and gets TLAST; the second must be discarded while the This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Line 188: `// descriptor sequencer reports an overrun instead of deadlocking.` Design intent: descriptor sequencer reports an overrun instead of deadlocking. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

Line 189: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 190: `program_output_index = 0;` This makes program_output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 191: `expected_program_outputs = 1;` This makes expected_program_outputs, the value named 'expected program outputs' in this block, equal to '1'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 192: `program_output_beats = 1;` This makes program_output_beats, the value named 'program output beats' in this block, equal to '1'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 193: `program_start = 1'b1;` This makes program_start, the value named 'program start' in this block, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 194: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 195: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 196: `program_start = 1'b0;` This makes program_start, the value named 'program start' in this block, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 197: `send_word({1'b1, 31'd1, 32'd0}, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; b1: the value named 'b1' in this block; d1: the value named 'd1' in this block.

Line 198: `send_word(64'h4, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h4: the value named 'h4' in this block; b0: the value named 'b0' in this block.

Line 199: `send_word(64'd0, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; d0: the value named 'd0' in this block; b1: the value named 'b1' in this block.

Line 200: `wait (program_done);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are wait: the value named 'wait' in this block; program_done: the value named 'program done' in this block.

Line 201: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 202: `if (!program_error || program_active || program_records != 1` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are program_error: the value named 'program error' in this block; program_active: the value named 'program active' in this block; program_records: the number of descriptor records accepted since program start.

Line 203: `|| output_count != 3) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are output_count: an integer count retained for control or measurable telemetry.

Line 204: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 205: `$display("ERROR overrun active=%0d error=%0d records=%0d output=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 206: `program_active, program_error, program_records, output_count);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The important operands are program_active: the value named 'program active' in this block; program_error: the value named 'program error' in this block; program_records: the number of descriptor records accepted since program start.

Line 207: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 208: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 209: `$display("tb_neuraedge_descriptor_sequencer FAILED errors=%0d", errors);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 210: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 211: `$display("tb_neuraedge_descriptor_sequencer PASSED records=%0d", program_records);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 212: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 213: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 214: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 216: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 217: `#200_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 218: `$fatal(1, "descriptor sequencer watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 219: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 220: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 222: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge descriptor sequencer RTL block in simulation.

50 tb/tb_neuraedge_integer_primitives.sv

Role in the complete system: drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation.

50.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation.

Simulation-only status tb_neuraedge_integer_primitives is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

50.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module tb_neuraedge_integer_primitives;
5      logic clk = 1'b0;
6      logic rst_n = 1'b0;
7
8      logic div_input_valid = 1'b0;
9      logic div_input_ready;
10     logic [63:0] div_numerator = 64'd0;
11     logic [31:0] div_denominator = 32'd1;
12     logic div_output_valid;
13     logic div_output_ready = 1'b0;
14     logic [63:0] div_quotient;
15     logic [31:0] div_remainder;
16     logic divide_by_zero;
17
18     logic sqrt_input_valid = 1'b0;
19     logic sqrt_input_ready;
20     logic [63:0] sqrt_radicand = 64'd0;
21     logic sqrt_output_valid;
22     logic sqrt_output_ready = 1'b0;
23     logic [31:0] sqrt_root;
24     logic [32:0] sqrt_remainder;
25
26     int errors = 0;
27     int div_checked = 0;
28     int sqrt_checked = 0;
29
30     neuraedge_udivider u_divider (
31         .clk(clk), .rst_n(rst_n),
32         .input_valid(div_input_valid), .input_ready(div_input_ready),
33         .numerator(div_numerator), .denominator(div_denominator),
34         .output_valid(div_output_valid), .output_ready(div_output_ready),
35         .quotient(div_quotient), .remainder(div_remainder),
36         .divide_by_zero(divide_by_zero)
37     );
38
39     neuraedge_isqrt64 u_sqrt (
40         .clk(clk), .rst_n(rst_n),
41         .input_valid(sqrt_input_valid), .input_ready(sqrt_input_ready),
42         .radicand(sqrt_radicand), .output_valid(sqrt_output_valid),
43         .output_ready(sqrt_output_ready), .root(sqrt_root),
44         .remainder(sqrt_remainder)
45     );
46
47     always #2.5 clk = ~clk;
48
49     task automatic check_division(input logic [63:0] n, input logic [31:0] d);
50         logic [63:0] expected_q;
51         logic [31:0] expected_r;
52         begin
53             expected_q = d == 0 ? 64'hffffffffffffffff : n / d;
54             expected_r = d == 0 ? n[31:0] : n % d;
55             while (!div_input_ready) @(negedge clk);
56             @(negedge clk);
57             div_numerator = n;
58             div_denominator = d;
59             div_input_valid = 1'b1;
```

```
            @(posedge clk);
            @(negedge clk);
            div_input_valid = 1'b0;
            while (!div_output_valid) @(negedge clk);
            repeat (div_checked % 3) @(negedge clk);
            if (div_quotient !== expected_q || div_remainder !== expected_r
                    || divide_by_zero !== (d == 0)) begin
                errors++;
                $display("ERROR divider n=%0d d=%0d got=%0d r=%0d expected=%0d r=%0d",
                         n, d, div_quotient, div_remainder, expected_q, expected_r);
            end
            div_output_ready = 1'b1;
            @(posedge clk);
            @(negedge clk);
            div_output_ready = 1'b0;
            div_checked++;
        end
    endtask

    task automatic check_sqrt(input logic [63:0] value);
        logic [63:0] expected_root;
        begin
            expected_root = 0;
            // The testbench oracle uses binary search, independent of the
            // restoring square-root datapath in the DUT.
            begin
                logic [63:0] low;
                logic [63:0] high;
                low = 0;
                high = 64'h0000000100000000;
                while (low + 1 < high) begin
                    logic [63:0] middle;
                    middle = (low + high) >> 1;
                    if (middle <= value / middle) low = middle;
                    else high = middle;
                end
                expected_root = low;
            end
            while (!sqrt_input_ready) @(negedge clk);
            @(negedge clk);
            sqrt_radicand = value;
            sqrt_input_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            sqrt_input_valid = 1'b0;
            while (!sqrt_output_valid) @(negedge clk);
            repeat (sqrt_checked % 4) @(negedge clk);
            if (sqrt_root !== expected_root[31:0]
                    || sqrt_remainder !== value - expected_root * expected_root) begin
                errors++;
                $display("ERROR sqrt value=%0d got=%0d rem=%0d expected=%0d rem=%0d",
                         value, sqrt_root, sqrt_remainder, expected_root,
                         value - expected_root * expected_root);
            end
            sqrt_output_ready = 1'b1;
            @(posedge clk);
            @(negedge clk);
            sqrt_output_ready = 1'b0;
            sqrt_checked++;
        end
    endtask

    initial begin
        int seed;
        logic [63:0] random_value;
        seed = 32'hd171de42;
        repeat (5) @(negedge clk);
        rst_n = 1'b1;

        check_division(0, 1);
        check_division(1, 1);
        check_division(64'hffffffffffffffff, 32'hffffffff);
        check_division(64'd123456789012345, 32'd97);
        check_division(1234, 0);
        for (int test_index = 0; test_index < 100; test_index++) begin
            random_value = {$urandom(seed), $urandom(seed)};
            check_division(random_value, 1 + ($urandom(seed) % 32'hfffffffe));
        end

        check_sqrt(0);
        check_sqrt(1);
        check_sqrt(2);
        check_sqrt(3);
        check_sqrt(4);
        check_sqrt(64'hffffffffffffffff);
        for (int test_index = 0; test_index < 100; test_index++) begin
            random_value = {$urandom(seed), $urandom(seed)};
```

```
            check_sqrt(random_value);
        end

        if (errors != 0) begin
            $display("tb_neuraedge_integer_primitives FAILED errors=%0d", errors);
            $finish;
        end else begin
            $display("tb_neuraedge_integer_primitives PASSED divisions=%0d square_roots=%0d",
                     div_checked, sqrt_checked);
            $finish;
        end
    end

    initial begin
        #500_000;
        $fatal(1, "tb_neuraedge_integer_primitives watchdog timeout");
    end
endmodule

`default_nettype wire
```

50.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module tb_neuraedge_integer_primitives;` This opens tb_neuraedge_integer_primitives;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation.

Line 5: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 6: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 8: `logic div_input_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, div_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic div_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, div_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic [63:0] div_numerator = 64'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, div_numerator is the value named 'div numerator' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic [31:0] div_denominator = 32'd1;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, div_denominator is the value named 'div denominator' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic div_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, div_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic div_output_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, div_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic [63:0] div_quotient;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, div_quotient is the value named 'div quotient' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic [31:0] div_remainder;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, div_remainder is the value named 'div remainder' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic divide_by_zero;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, divide_by_zero is the value named 'divide by zero' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic sqrt_input_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic sqrt_input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic [63:0] sqrt_radicand = 64'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_radicand is the value named 'sqrt radicand' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic sqrt_output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic sqrt_output_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic [31:0] sqrt_root;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_root is the value named 'sqrt root' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic [32:0] sqrt_remainder;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sqrt_remainder is the value named 'sqrt remainder' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 27: `int div_checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are div_checked: the value named 'div checked' in this block.

Line 28: `int sqrt_checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are sqrt_checked: the value named 'sqrt checked' in this block.

Line 30: `neuraedge_udivider u_divider (` This instantiates module neuraedge_udivider as u_divider. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 31: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 32: `.input_valid(div_input_valid), .input_ready(div_input_ready),` This connects the child instance's named port 'input_valid' to parent expression 'div_input_valid), .input_ready(div_input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 33: `.numerator(div_numerator), .denominator(div_denominator),` This connects the child instance's named port 'numerator' to parent expression 'div_numerator), .denominator(div_denominator'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'div numerator), .denominator(div denominator' in this block.

Line 34: `.output_valid(div_output_valid), .output_ready(div_output_ready),` This connects the child instance's named port 'output_valid' to parent expression 'div_output_valid), .output_ready(div_output_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 35: `.quotient(div_quotient), .remainder(div_remainder),` This connects the child instance's named port 'quotient' to parent expression 'div_quotient), .remainder(div_remainder'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'div quotient), .remainder(div remainder' in this block.

Line 36: `.divide_by_zero(divide_by_zero)` This connects the child instance's named port 'divide_by_zero' to parent expression 'divide_by_zero'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'divide by zero' in this block.

Line 37: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 39: `neuraedge_isqrt64 u_sqrt (` This instantiates module neuraedge_isqrt64 as u_sqrt. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 40: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 41: `.input_valid(sqrt_input_valid), .input_ready(sqrt_input_ready),` This connects the child instance's named port 'input_valid' to parent expression 'sqrt_input_valid), .input_ready(sqrt_input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 42: `.radicand(sqrt_radicand), .output_valid(sqrt_output_valid),` This connects the child instance's named port ‘radicand‘ to parent expression ‘sqrt_radicand), .output_valid(sqrt_output_valid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 43: `.output_ready(sqrt_output_ready), .root(sqrt_root),` This connects the child instance's named port ‘output_ready‘ to parent expression ‘sqrt_output_ready), .root(sqrt_root‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 44: `.remainder(sqrt_remainder)` This connects the child instance's named port ‘remainder‘ to parent expression ‘sqrt_remainder‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'sqrt remainder' in this block.

Line 45: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 47: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 49: `task automatic check_division(input logic [63:0] n, input logic [31:0] d);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; check_division: the value named 'check division' in this block.

Line 50: `logic [63:0] expected_q;` This declares hardware signal storage or wiring in the simulation task check division. Specifically, expected_q is the value named 'expected q' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 51: `logic [31:0] expected_r;` This declares hardware signal storage or wiring in the simulation task check division. Specifically, expected_r is the value named 'expected r' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 52: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task check division; it has no arithmetic value of its own.

Line 53: `expected_q = d == 0 ? 64'hffffffffffffffff : n / d;` This makes expected_q, the value named 'expected q' in this block, equal to ‘d == 0 ? 64'hffffffffffffffff : n / d‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d: the value named 'd' in this block; hffffffffffffff: the value named 'hffffffffffffff' in this block; n: the value named 'n' in this block. Context: the simulation task check division.

Line 54: `expected_r = d == 0 ? n[31:0] : n % d;` This makes expected_r, the value named 'expected r' in this block, equal to ‘d == 0 ? n[31:0] : n % d‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d: the value named 'd' in this block; n: the value named 'n' in this block. Context: the simulation task check division.

Line 55: `while (!div_input_ready) @(negedge clk);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are while: the value named 'while' in this block; div_input_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 56: `@(negedge clk);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 57: `div_numerator = n;` This makes div_numerator, the value named 'div numerator' in this block, equal to 'n'. It is driven by combinational or procedural logic in the current evaluation. The important operands are n: the value named 'n' in this block. Context: the simulation task check division.

Line 58: `div_denominator = d;` This makes div_denominator, the value named 'div denominator' in this block, equal to 'd'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d: the value named 'd' in this block. Context: the simulation task check division.

Line 59: `div_input_valid = 1'b1;` This makes div_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task check division.

Line 60: `@(posedge clk);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 61: `@(negedge clk);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 62: `div_input_valid = 1'b0;` This makes div_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task check division.

Line 63: `while (!div_output_valid) @(negedge clk);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are while: the value named 'while' in this block; div_output_valid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 64: `repeat (div_checked % 3) @(negedge clk);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; div_checked: the value named 'div checked' in this block; negedge: the value named 'negedge' in this block.

Line 65: `if (div_quotient !== expected_q || div_remainder !== expected_r` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are div_quotient: the value named 'div quotient' in this block; expected_q: the value named 'expected q' in this block; div_remainder: the value named 'div remainder' in this block.

Line 66: `|| divide_by_zero !== (d == 0)) begin` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are divide_by_zero: the value named 'divide by zero' in this block; d: the value named 'd' in this block.

Line 67: `errors++;` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 68: `$display("ERROR divider n=%0d d=%0d got=%0d r=%0d expected=%0d r=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 69: `n, d, div_quotient, div_remainder, expected_q, expected_r);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are n: the value named 'n' in this block; d: the value named 'd' in this block; div_quotient: the value named 'div quotient' in this block.

Line 70: `end` This structural keyword opens or closes the branch/process scope described as the simulation task check division; it has no arithmetic value of its own.

Line 71: `div_output_ready = 1'b1;` This makes div_output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task check division.

Line 72: `@(posedge clk);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 73: `@(negedge clk);` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 74: `div_output_ready = 1'b0;` This makes div_output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task check division.

Line 75: `div_checked++;` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are div_checked: the value named 'div checked' in this block.

Line 76: `end` This structural keyword opens or closes the branch/process scope described as the simulation task check division; it has no arithmetic value of its own.

Line 77: `endtask` Within the simulation task check division, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 79: `task automatic check_sqrt(input logic [63:0] value);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; check_sqrt: the value named 'check sqrt' in this block.

Line 80: `logic [63:0] expected_root;` This declares hardware signal storage or wiring in the simulation task check sqrt. Specifically, expected_root is the value named 'expected root' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 81: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task check sqrt; it has no arithmetic value of its own.

Line 82: `expected_root = 0;` This makes expected_root, the value named 'expected root' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task check sqrt.

Line 83: `// The testbench oracle uses binary search, independent of the` Design intent: The testbench oracle uses binary search, independent of the This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation.

Line 84: `// restoring square-root datapath in the DUT.` Design intent: restoring square-root datapath in the DUT. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation.

Line 85: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task check sqrt; it has no arithmetic value of its own.

Line 86: `logic [63:0] low;` This declares hardware signal storage or wiring in the simulation task check sqrt. Specifically, low is the value named 'low' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 87: `logic [63:0] high;` This declares hardware signal storage or wiring in the simulation task check sqrt. Specifically, high is the value named 'high' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 88: `low = 0;` This makes low, the value named 'low' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task check sqrt.

Line 89: `high = 64'h0000000100000000;` This makes high, the value named 'high' in this block, equal to '64'h0000000100000000'
It is driven by combinational or procedural logic in the current evaluation. The important operands are h0000000100000000:
the value named 'h0000000100000000' in this block. Context: the simulation task check sqrt.

Line 90: `while (low + 1 < high) begin` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are while: the value named 'while' in this block; low: the value named 'low' in this block; high: the value named 'high' in this block.

Line 91: `logic [63:0] middle;` This declares hardware signal storage or wiring in the simulation task check sqrt. Specifically, middle is the value named 'middle' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 92: `middle = (low + high) >> 1;` This makes middle, the value named 'middle' in this block, equal to '(low + high) >> 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are low: the value named 'low' in this block; high: the value named 'high' in this block. Context: the simulation task check sqrt.

Line 93: `if (middle <= value / middle) low = middle;` This condition selects the following hardware branch only when 'middle <= value / middle' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are middle: the value named 'middle' in this block; value: the value named 'value' in this block. Context: the simulation task check sqrt.

Line 94: `else high = middle;` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are high: the value named 'high' in this block; middle: the value named 'middle' in this block.

Line 95: `end` This structural keyword opens or closes the branch/process scope described as the simulation task check sqrt; it has no arithmetic value of its own.

Line 96: `expected_root = low;` This makes expected_root, the value named 'expected root' in this block, equal to 'low'. It is driven by combinational or procedural logic in the current evaluation. The important operands are low: the value named 'low' in this block. Context: the simulation task check sqrt.

Line 97: `end` This structural keyword opens or closes the branch/process scope described as the simulation task check sqrt; it has no arithmetic value of its own.

Line 98: `while (!sqrt_input_ready) @(negedge clk);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are while: the value named 'while' in this block; sqrt_input_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 99: `@(negedge clk);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 100: `sqrt_radicand = value;` This makes sqrt_radicand, the value named 'sqrt radicand' in this block, equal to 'value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the simulation task check sqrt.

Line 101: `sqrt_input_valid = 1'b1;` This makes sqrt_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task check sqrt.

Line 102: `@(posedge clk);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 103: `@(negedge clk);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 104: `sqrt_input_valid = 1'b0;` This makes sqrt_input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task check sqrt.

Line 105: `while (!sqrt_output_valid) @(negedge clk);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are while: the value named 'while' in this block; sqrt_output_valid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 106: `repeat (sqrt_checked % 4) @(negedge clk);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; sqrt_checked: the value named 'sqrt checked' in this block; negedge: the value named 'negedge' in this block.

Line 107: `if (sqrt_root !== expected_root[31:0]` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are sqrt_root: the value named 'sqrt root' in this block; expected_root: the value named 'expected root' in this block.

Line 108: `|| sqrt_remainder !== value - expected_root * expected_root) begin` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are sqrt_remainder: the value named 'sqrt remainder' in this block; value: the value named 'value' in this block; expected_root: the value named 'expected root' in this block.

Line 109: `errors++;` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 110: `$display("ERROR sqrt value=%0d got=%0d rem=%0d expected=%0d rem=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 111: `value, sqrt_root, sqrt_remainder, expected_root,` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are value: the value named 'value' in this block; sqrt_root: the value named 'sqrt root' in this block; sqrt_remainder: the value named 'sqrt remainder' in this block.

Line 112: `value - expected_root * expected_root);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are value: the value named 'value' in this block; expected_root: the value named 'expected root' in this block.

Line 113: `end` This structural keyword opens or closes the branch/process scope described as the simulation task check sqrt; it has no arithmetic value of its own.

Line 114: `sqrt_output_ready = 1'b1;` This makes sqrt_output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task check sqrt.

Line 115: `@(posedge clk);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 116: `@(negedge clk);` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 117: `sqrt_output_ready = 1'b0;` This makes sqrt_output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task check sqrt.

Line 118: `sqrt_checked++;` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are sqrt_checked: the value named 'sqrt checked' in this block.

Line 119: `end` This structural keyword opens or closes the branch/process scope described as the simulation task check sqrt; it has no arithmetic value of its own.

Line 120: `endtask` Within the simulation task check sqrt, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 122: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 123: `int seed;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are seed: the value named 'seed' in this block.

Line 124: `logic [63:0] random_value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, random_value is the value named 'random value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 125: `seed = 32'hd171de42;` This makes seed, the value named 'seed' in this block, equal to '32'hd171de42'. It is driven by combinational or procedural logic in the current evaluation. The important operands are hd171de42: the value named 'hd171de42' in this block. Context: the module's structural declarations.

Line 126: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 127: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 129: `check_division(0, 1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_division: the value named 'check division' in this block.

Line 130: `check_division(1, 1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_division: the value named 'check division' in this block.

Line 131: `check_division(64'hffffffffffffffff, 32'hffffffff);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_division: the value named 'check division' in this block; hffffffffffffffff: the value named 'hffffffffffffffff' in this block; hffffffff: the value named 'hffffffff' in this block.

Line 132: `check_division(64'd123456789012345, 32'd97);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_division: the value named 'check division' in this block; d123456789012345: the value named 'd123456789012345' in this block; d97: the value named 'd97' in this block.

Line 133: `check_division(1234, 0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_division: the value named 'check division' in this block.

Line 134: `for (int test_index = 0; test_index < 100; test_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 135: `random_value = {$urandom(seed), $urandom(seed)};` This makes random_value, the value named 'random value' in this block, equal to '{$urandom(seed), $urandom(seed)}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are urandom: the value named 'urandom' in this block; seed: the value named 'seed' in this block. Context: the module's structural declarations.

Line 136: `check_division(random_value, 1 + ($urandom(seed) % 32'hfffffffe));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_division: the value named 'check division' in this block; random_value: the value named 'random value' in this block; urandom: the value named 'urandom' in this block.

Line 137: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 139: `check_sqrt(0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_sqrt: the value named 'check sqrt' in this block.

Line 140: `check_sqrt(1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_sqrt: the value named 'check sqrt' in this block.

Line 141: `check_sqrt(2);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_sqrt: the value named 'check sqrt' in this block.

Line 142: `check_sqrt(3);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_sqrt: the value named 'check sqrt' in this block.

Line 143: `check_sqrt(4);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_sqrt: the value named 'check sqrt' in this block.

Line 144: `check_sqrt(64'hffffffffffffffff);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_sqrt: the value named 'check sqrt' in this block; hffffffffffffffff: the value named 'hffffffffffffffff' in this block.

Line 145: `for (int test_index = 0; test_index < 100; test_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 146: `random_value = {$urandom(seed), $urandom(seed)};` This makes random_value, the value named 'random value' in this block, equal to '{$urandom(seed), $urandom(seed)}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are urandom: the value named 'urandom' in this block; seed: the value named 'seed' in this block. Context: the module's structural declarations.

Line 147: `check_sqrt(random_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are check_sqrt: the value named 'check sqrt' in this block; random_value: the value named 'random value' in this block.

Line 148: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 150: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 151: `$display("tb_neuraedge_integer_primitives FAILED errors=%0d", errors);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 152: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 153: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 154: `$display("tb_neuraedge_integer_primitives PASSED divisions=%0d square_roots=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 155: `div_checked, sqrt_checked);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The important operands are div_checked: the value named 'div checked' in this block; sqrt_checked: the value named 'sqrt checked' in this block.

Line 156: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 157: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 158: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 160: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 161: `#500_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 162: `$fatal(1, "tb_neuraedge_integer_primitives watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 163: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 164: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 166: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge integer primitives RTL block in simulation.

51 tb/tb_neuraedge_lif_bank.sv

Role in the complete system: drives, backpressures, and checks the neuraedge lif bank RTL block in simulation.

51.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge lif bank RTL block in simulation.

Simulation-only status tb_neuraedge_lif_bank is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

51.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module tb_neuraedge_lif_bank;
5      localparam int LANES = 32;
6      localparam int MEMBRANE_BITS = 24;
7      localparam int CURRENT_BITS = 16;
8      localparam int MAXIMUM_MEMBRANE = (1 << 23) - 1;
9      localparam int MINIMUM_MEMBRANE = -(1 << 23);
10
11     logic clk = 1'b0;
12     logic rst_n = 1'b0;
13     logic clear_state = 1'b0;
14     logic clear_counters = 1'b0;
15     logic input_valid = 1'b0;
16     logic input_ready;
17     logic [LANES*CURRENT_BITS-1:0] current_vector = '0;
18     logic [4:0] leak_shift = 5'd2;
19     logic signed [MEMBRANE_BITS-1:0] threshold = 24'sd100;
20     logic signed [MEMBRANE_BITS-1:0] reset_value = 24'sd0;
21     logic output_valid;
22     logic output_ready = 1'b0;
23     logic [LANES-1:0] spikes;
24     logic [LANES*MEMBRANE_BITS-1:0] membrane_vector;
25     logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector;
26     logic [31:0] timestep_count;
27     logic [63:0] total_spike_count;
28
29     integer reference_membrane [0:LANES-1];
30     int reference_timesteps = 0;
31     int reference_spikes = 0;
32     int errors = 0;
33     int checked = 0;
34
35     neuraedge_lif_bank dut (
36         .clk(clk),
37         .rst_n(rst_n),
38         .clear_state(clear_state),
39         .clear_counters(clear_counters),
40         .input_valid(input_valid),
41         .input_ready(input_ready),
42         .current_vector(current_vector),
43         .leak_shift(leak_shift),
44         .threshold(threshold),
45         .reset_value(reset_value),
46         .output_valid(output_valid),
47         .output_ready(output_ready),
48         .spikes(spikes),
49         .membrane_vector(membrane_vector),
50         .pre_reset_membrane_vector(pre_reset_membrane_vector),
51         .timestep_count(timestep_count),
52         .total_spike_count(total_spike_count)
53     );
54
55     always #2.5 clk = ~clk;
56
57     function automatic int saturate_reference(input longint signed value);
58         begin
59             if (value > MAXIMUM_MEMBRANE) saturate_reference = MAXIMUM_MEMBRANE;
60             else if (value < MINIMUM_MEMBRANE) saturate_reference = MINIMUM_MEMBRANE;
```

```
            else saturate_reference = int'(value);
        end
    endfunction

    task automatic clear_reference_and_dut(input int reset_level);
        begin
            reset_value = 24'(reset_level);
            @(negedge clk);
            clear_state = 1'b1;
            @(posedge clk);
            for (int lane = 0; lane < LANES; lane++) reference_membrane[lane] = reset_level;
            reference_timesteps = 0;
            @(negedge clk);
            clear_state = 1'b0;
        end
    endtask

    task automatic run_step(input int stall_cycles);
        logic [LANES-1:0] expected_spikes;
        integer expected_membrane [0:LANES-1];
        integer expected_pre_reset_membrane [0:LANES-1];
        begin
            expected_spikes = '0;
            for (int lane = 0; lane < LANES; lane++) begin
                int current_value;
                longint signed integrated;
                current_value = $signed(current_vector[lane*CURRENT_BITS +: CURRENT_BITS]);
                integrated = reference_membrane[lane]
                    - (reference_membrane[lane] >>> leak_shift) + current_value;
                expected_membrane[lane] = saturate_reference(integrated);
                expected_pre_reset_membrane[lane] = expected_membrane[lane];
                if (expected_membrane[lane] >= $signed(threshold)) begin
                    expected_spikes[lane] = 1'b1;
                    expected_membrane[lane] = $signed(reset_value);
                end
            end

            while (!input_ready) @(negedge clk);
            @(negedge clk);
            input_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            input_valid = 1'b0;
            while (!output_valid) @(negedge clk);
            repeat (stall_cycles) begin
                if (input_ready) begin
                    errors++;
                    $display("ERROR LIF accepted a new step while output was stalled");
                end
                @(negedge clk);
            end

            if (!output_valid || spikes !== expected_spikes) begin
                errors++;
                $display("ERROR LIF spike mismatch step=%0d got=%h expected=%h",
                         reference_timesteps, spikes, expected_spikes);
            end
            for (int lane = 0; lane < LANES; lane++) begin
                logic signed [MEMBRANE_BITS-1:0] got_membrane;
                logic signed [MEMBRANE_BITS-1:0] got_pre_reset_membrane;
                got_membrane = membrane_vector[lane*MEMBRANE_BITS +: MEMBRANE_BITS];
                got_pre_reset_membrane = pre_reset_membrane_vector[
                    lane*MEMBRANE_BITS +: MEMBRANE_BITS];
                if ($signed(got_membrane) !== expected_membrane[lane]) begin
                    errors++;
                    if (errors <= 20) begin
                        $display("ERROR LIF membrane step=%0d lane=%0d got=%0d expected=%0d",
                                 reference_timesteps, lane, $signed(got_membrane), expected_membrane[lane]);
                    end
                end
                if ($signed(got_pre_reset_membrane)
                        !== expected_pre_reset_membrane[lane]) begin
                    errors++;
                    if (errors <= 20) begin
                        $display("ERROR LIF pre-reset membrane step=%0d lane=%0d got=%0d expected=%0d",
                                 reference_timesteps, lane,
                                 $signed(got_pre_reset_membrane),
                                 expected_pre_reset_membrane[lane]);
                    end
                end
                reference_membrane[lane] = expected_membrane[lane];
                reference_spikes += expected_spikes[lane];
            end
            reference_timesteps++;

            output_ready = 1'b1;
            @(posedge clk);
```

```
            @(negedge clk);
            output_ready = 1'b0;
            if (timestep_count != reference_timesteps || total_spike_count != reference_spikes) begin
                errors++;
                $display("ERROR LIF counters got=(%0d,%0d) expected=(%0d,%0d)",
                         timestep_count, total_spike_count, reference_timesteps, reference_spikes);
            end
            checked++;
        end
    endtask

    initial begin
        int seed;
        seed = 32'h1f1f2026;
        for (int lane = 0; lane < LANES; lane++) reference_membrane[lane] = 0;

        repeat (5) @(negedge clk);
        rst_n = 1'b1;
        clear_reference_and_dut(0);

        for (int step = 0; step < 100; step++) begin
            for (int lane = 0; lane < LANES; lane++) begin
                current_vector[lane*CURRENT_BITS +: CURRENT_BITS] =
                    16'($signed(($urandom(seed) % 241) - 120));
            end
            leak_shift = 5'(1 + step % 5);
            threshold = 24'sd100;
            run_step(step % 4);
        end

        // Explicit positive saturation boundary.
        clear_reference_and_dut(MAXIMUM_MEMBRANE - 4);
        leak_shift = 5'd31;
        threshold = 24'sd8388607;
        current_vector = '0;
        current_vector[CURRENT_BITS-1:0] = 16'sd32767;
        run_step(2);

        @(negedge clk);
        clear_counters = 1'b1;
        @(posedge clk);
        @(negedge clk);
        clear_counters = 1'b0;
        reference_spikes = 0;
        if (total_spike_count != 0) begin
            errors++;
            $display("ERROR LIF spike counter did not clear independently");
        end

        if (errors != 0) begin
            $display("tb_neuraedge_lif_bank FAILED errors=%0d checked=%0d", errors, checked);
            $finish;
        end else begin
            $display("tb_neuraedge_lif_bank PASSED checked=%0d timesteps=%0d spikes=%0d",
                     checked, reference_timesteps, reference_spikes);
            $finish;
        end
    end

    initial begin
        #200_000;
        $fatal(1, "tb_neuraedge_lif_bank watchdog timeout");
    end
endmodule

`default_nettype wire
```

51.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module tb_neuraedge_lif_bank;` This opens tb_neuraedge_lif_bank;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation.

Line 5: `localparam int LANES = 32;` This fixes LANES to 32 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 6: `localparam int MEMBRANE_BITS = 24;` This fixes MEMBRANE_BITS to 24 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MEMBRANE_BITS denotes the persistent LIF neuron state carrying charge between timesteps.

Line 7: `localparam int CURRENT_BITS = 16;` This fixes CURRENT_BITS to 16 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CURRENT_BITS denotes the value named 'current bits' in this block.

Line 8: `localparam int MAXIMUM_MEMBRANE = (1 << 23) - 1;` This fixes MAXIMUM_MEMBRANE to (1 << 23) - 1 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MAXIMUM_MEMBRANE denotes the persistent LIF neuron state carrying charge between timesteps.

Line 9: `localparam int MINIMUM_MEMBRANE = -(1 << 23);` This fixes MINIMUM_MEMBRANE to -(1 << 23) during elaboration. Vivado uses that constant to shape or simplify the generated circuit; MINIMUM_MEMBRANE denotes the persistent LIF neuron state carrying charge between timesteps.

Line 11: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic clear_state = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clear_state is the finite-state-machine phase retained in flip-flops. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic clear_counters = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clear_counters is a hardware telemetry register that accumulates measured events. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic input_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [LANES*CURRENT_BITS-1:0] current_vector = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, current_vector is the value named 'current vector' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic [4:0] leak_shift = 5'd2;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, leak_shift is the retained fraction of prior membrane state. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic signed [MEMBRANE_BITS-1:0] threshold = 24'sd100;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, threshold is the membrane level at which a neuron emits a spike. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic signed [MEMBRANE_BITS-1:0] reset_value = 24'sd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reset_value is the value named 'reset value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic output_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic [LANES-1:0] spikes;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, spikes is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic [LANES*MEMBRANE_BITS-1:0] membrane_vector;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, membrane_vector is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic [LANES*MEMBRANE_BITS-1:0] pre_reset_membrane_vector;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, pre_reset_membrane_vector is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `logic [31:0] timestep_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, timestep_count is one discrete SNN update instant. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic [63:0] total_spike_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, total_spike_count is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `integer reference_membrane [0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reference_membrane is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `int reference_timesteps = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are reference_timesteps: one discrete SNN update instant.

Line 31: `int reference_spikes = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are reference_spikes: a binary event produced when a neuron crosses its threshold.

Line 32: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 33: `int checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 35: `neuraedge_lif_bank dut (` This instantiates module neuraedge_lif_bank as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 36: `.clk(clk),` This connects the child instance's named port ‘clk‘ to parent expression ‘clk‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 37: `.rst_n(rst_n),` This connects the child instance's named port ‘rst_n‘ to parent expression ‘rst_n‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 38: `.clear_state(clear_state),` This connects the child instance's named port ‘clear_state‘ to parent expression ‘clear_state‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the finite-state-machine phase retained in flip-flops.

Line 39: `.clear_counters(clear_counters),` This connects the child instance's named port ‘clear_counters‘ to parent expression ‘clear_counters‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a hardware telemetry register that accumulates measured events.

Line 40: `.input_valid(input_valid),` This connects the child instance's named port ‘input_valid‘ to parent expression ‘input_valid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 41: `.input_ready(input_ready),` This connects the child instance's named port ‘input_ready‘ to parent expression ‘input_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 42: `.current_vector(current_vector),` This connects the child instance's named port ‘current_vector‘ to parent expression ‘current_vector‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'current vector' in this block.

Line 43: `.leak_shift(leak_shift),` This connects the child instance's named port ‘leak_shift‘ to parent expression ‘leak_shift‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the retained fraction of prior membrane state.

Line 44: `.threshold(threshold),` This connects the child instance's named port ‘threshold‘ to parent expression ‘threshold‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the membrane level at which a neuron emits a spike.

Line 45: `.reset_value(reset_value),` This connects the child instance's named port ‘reset_value‘ to parent expression ‘reset_value‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'reset value' in this block.

Line 46: `.output_valid(output_valid),` This connects the child instance's named port ‘output_valid‘ to parent expression ‘output_valid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 47: `.output_ready(output_ready),` This connects the child instance's named port ‘output_ready‘ to parent expression ‘output_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 48: `.spikes(spikes),` This connects the child instance's named port ‘spikes‘ to parent expression ‘spikes‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 49: `.membrane_vector(membrane_vector),` This connects the child instance's named port 'membrane_vector' to parent expression 'membrane_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the persistent LIF neuron state carrying charge between timesteps.

Line 50: `.pre_reset_membrane_vector(pre_reset_membrane_vector),` This connects the child instance's named port 'pre_reset_membrane_vector' to parent expression 'pre_reset_membrane_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the persistent LIF neuron state carrying charge between timesteps.

Line 51: `.timestep_count(timestep_count),` This connects the child instance's named port 'timestep_count' to parent expression 'timestep_count'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one discrete SNN update instant.

Line 52: `.total_spike_count(total_spike_count)` This connects the child instance's named port 'total_spike_count' to parent expression 'total_spike_count'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 53: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 55: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 57: `function automatic int saturate_reference(input longint signed value);` Within the combinational helper saturate reference, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_reference: the value named 'saturate reference' in this block.

Line 58: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate reference; it has no arithmetic value of its own.

Line 59: `if (value > MAXIMUM_MEMBRANE) saturate_reference = MAXIMUM_MEMBRANE;` This condition selects the following hardware branch only when 'value > MAXIMUM_MEMBRANE' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; MAXIMUM_MEMBRANE: the persistent LIF neuron state carrying charge between timesteps. Context: the combinational helper saturate reference.

Line 60: `else if (value < MINIMUM_MEMBRANE) saturate_reference = MINIMUM_MEMBRANE;` This condition selects the following hardware branch only when 'value < MINIMUM_MEMBRANE' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; MINIMUM_MEMBRANE: the persistent LIF neuron state carrying charge between timesteps. Context: the combinational helper saturate reference.

Line 61: `else saturate_reference = int'(value);` Within the combinational helper saturate reference, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are saturate_reference: the value named 'saturate reference' in this block; value: the value named 'value' in this block.

Line 62: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate reference; it has no arithmetic value of its own.

Line 63: `endfunction` Within the combinational helper saturate reference, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are endfunction: the value named 'endfunction' in this block.

Line 65: `task automatic clear_reference_and_dut(input int reset_level);` Within the simulation task clear reference and dut, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; clear_reference_and_dut: the value named 'clear reference and dut' in this block.

Line 66: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task clear reference and dut; it has no arithmetic value of its own.

Line 67: `reset_value = 24'(reset_level);` This makes reset_value, the value named 'reset value' in this block, equal to '24'(reset_level)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are reset_level: the value named 'reset level' in this block. Context: the simulation task clear reference and dut.

Line 68: `@(negedge clk);` Within the simulation task clear reference and dut, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 69: `clear_state = 1'b1;` This makes clear_state, the finite-state-machine phase retained in flip-flops, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task clear reference and dut.

Line 70: `@(posedge clk);` Within the simulation task clear reference and dut, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 71: `for (int lane = 0; lane < LANES; lane++) reference_membrane[lane] = reset_level;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task clear reference and dut.

Line 72: `reference_timesteps = 0;` This makes reference_timesteps, one discrete SNN update instant, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task clear reference and dut.

Line 73: `@(negedge clk);` Within the simulation task clear reference and dut, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 74: `clear_state = 1'b0;` This makes clear_state, the finite-state-machine phase retained in flip-flops, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task clear reference and dut.

Line 75: `end` This structural keyword opens or closes the branch/process scope described as the simulation task clear reference and dut; it has no arithmetic value of its own.

Line 76: `endtask` Within the simulation task clear reference and dut, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 78: `task automatic run_step(input int stall_cycles);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; run_step: the value named 'run step' in this block.

Line 79: `logic [LANES-1:0] expected_spikes;` This declares hardware signal storage or wiring in the simulation task run step. Specifically, expected_spikes is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 80: `integer expected_membrane [0:LANES-1];` This declares hardware signal storage or wiring in the simulation task run step. Specifically, expected_membrane is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 81: `integer expected_pre_reset_membrane [0:LANES-1];` This declares hardware signal storage or wiring in the simulation task run step. Specifically, expected_pre_reset_membrane is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 82: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 83: `expected_spikes = '0;` This makes expected_spikes, a binary event produced when a neuron crosses its threshold, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run step.

Line 84: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run step.

Line 85: `int current_value;` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are current_value: the value named 'current value' in this block.

Line 86: `longint signed integrated;` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are longint: the value named 'longint' in this block; integrated: the value named 'integrated' in this block.

Line 87: `current_value = $signed(current_vector[lane*CURRENT_BITS +: CURRENT_BITS]);` This makes current_value, the value named 'current value' in this block, equal to '$signed(current_vector[lane*CURRENT_BITS +: CURRENT_BITS])'. It is driven by combinational or procedural logic in the current evaluation. The important operands are current_vector: the value named 'current vector' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; CURRENT_BITS: the value named 'current bits' in this block. Context: the simulation task run step.

Line 88: `integrated = reference_membrane[lane]` This statement continues through source line 89; the explanation uses the complete expression rather than only the first visual fragment. This makes integrated, the value named 'integrated' in this block, equal to 'reference_membrane[lane] - (reference_membrane[lane] >>> leak_shift) + current_value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are reference_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath; leak_shift: the retained fraction of prior membrane state. Context: the simulation task run step.

Line 89: `- (reference_membrane[lane] >>> leak_shift) + current_value;` This is continuation line 2 of the 2-line logical statement that begins at source line 88. Its exact fragment is '- (reference_membrane[lane] >>> leak_shift) + current_value;'. The important operands are reference_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath; leak_shift: the retained fraction of prior membrane state. Read together, the complete operation is: This makes integrated, the value named 'integrated' in this block, equal to 'reference_membrane[lane] - (reference_membrane[lane] >>> leak_shift) + current_value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are reference_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath; leak_shift: the retained fraction of prior membrane state. Context: the simulation task run step.

Line 90: `expected_membrane[lane] = saturate_reference(integrated);` This makes expected_membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to 'saturate_reference(integrated)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_reference: the value named 'saturate reference' in this block; integrated: the value named 'integrated' in this block. Context: the simulation task run step.

Line 91: `expected_pre_reset_membrane[lane] = expected_membrane[lane];` This makes expected_pre_reset_membrane[la the persistent LIF neuron state carrying charge between timesteps, equal to 'expected_membrane[lane]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are expected_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task run step.

Line 92: `if (expected_membrane[lane] >= $signed(threshold)) begin` This condition selects the following hardware branch only when 'expected_membrane[lane] >= $signed(threshold)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are expected_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath; threshold: the membrane level at which a neuron emits a spike. Context: the simulation task run step.

Line 93: `expected_spikes[lane] = 1'b1;` This makes expected_spikes[lane], a binary event produced when a neuron crosses its threshold, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task run step.

Line 94: `expected_membrane[lane] = $signed(reset_value);` This makes expected_membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to '$signed(reset_value)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are reset_value: the value named 'reset value' in this block. Context: the simulation task run step.

Line 95: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 96: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 98: `while (!input_ready) @(negedge clk);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are while: the value named 'while' in this block; input_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 99: `@(negedge clk);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 100: `input_valid = 1'b1;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task run step.

Line 101: `@(posedge clk);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 102: `@(negedge clk);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 103: `input_valid = 1'b0;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run step.

Line 104: `while (!output_valid) @(negedge clk);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are while: the value named 'while' in this block; output_valid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 105: `repeat (stall_cycles) begin` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; stall_cycles: the value named 'stall cycles' in this block.

Line 106: `if (input_ready) begin` This condition selects the following hardware branch only when 'input_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_ready: the consumer's statement that it can accept a payload this cycle. Context: the simulation task run step.

Line 107: `errors++;` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 108: `$display("ERROR LIF accepted a new step while output was stalled");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 109: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 110: `@(negedge clk);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 111: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 113: `if (!output_valid || spikes !== expected_spikes) begin` This condition selects the following hardware branch only when '!output_valid || spikes !== expected_spikes' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_valid: the producer's statement that its current payload is meaningful; spikes: a binary event produced when a neuron crosses its threshold; expected_spikes: a binary event produced when a neuron crosses its threshold. Context: the simulation task run step.

Line 114: `errors++;` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 115: `$display("ERROR LIF spike mismatch step=%0d got=%h expected=%h",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 116: `reference_timesteps, spikes, expected_spikes);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are reference_timesteps: one discrete SNN update instant; spikes: a binary event produced when a neuron crosses its threshold; expected_spikes: a binary event produced when a neuron crosses its threshold.

Line 117: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 118: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run step.

Line 119: `logic signed [MEMBRANE_BITS-1:0] got_membrane;` This declares hardware signal storage or wiring in the simulation task run step. Specifically, got_membrane is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 120: `logic signed [MEMBRANE_BITS-1:0] got_pre_reset_membrane;` This declares hardware signal storage or wiring in the simulation task run step. Specifically, got_pre_reset_membrane is the persistent LIF neuron state carrying charge between timesteps. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 121: `got_membrane = membrane_vector[lane*MEMBRANE_BITS +: MEMBRANE_BITS];` This makes got_membrane, the persistent LIF neuron state carrying charge between timesteps, equal to 'membrane_vector[lane*MEMBRANE_BITS +: MEMBRANE_BITS]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are membrane_vector: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the simulation task run step.

Line 122: `got_pre_reset_membrane = pre_reset_membrane_vector[` This statement continues through source line 123; the explanation uses the complete expression rather than only the first visual fragment. This makes got_pre_reset_membrane, the persistent LIF neuron state carrying charge between timesteps, equal to 'pre_reset_membrane_vector[lane*MEMBRANE_BIT +: MEMBRANE_BITS]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pre_reset_membrane_vector: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the simulation task run step.

Line 123: `lane*MEMBRANE_BITS +: MEMBRANE_BITS];` This is continuation line 2 of the 2-line logical statement that begins at source line 122. Its exact fragment is 'lane*MEMBRANE_BITS +: MEMBRANE_BITS];'. The important operands are lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This makes got_pre_reset_membrane, the persistent LIF neuron state carrying charge between timesteps, equal to 'pre_reset_membrane_vecto lane*MEMBRANE_BITS +: MEMBRANE_BITS]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are pre_reset_membrane_vector: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath; MEMBRANE_BITS: the persistent LIF neuron state carrying charge between timesteps. Context: the simulation task run step.

Line 124: `if ($signed(got_membrane) !== expected_membrane[lane]) begin` This condition selects the following hardware branch only when ‘$signed(got_membrane) !== expected_membrane[lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are got_membrane: the persistent LIF neuron state carrying charge between timesteps; expected_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task run step.

Line 125: `errors++;` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 126: `if (errors <= 20) begin` This statement continues through source line 128; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LIF membrane step=%0d lane=%0d got=%0d expected=%0d", reference_timesteps, lane, $signed(got_membrane), expected_membrane[lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task run step.

Line 127: `$display("ERROR LIF membrane step=%0d lane=%0d got=%0d expected=%0d",` This is continuation line 2 of the 3-line logical statement that begins at source line 126. Its exact fragment is ‘$display("ERROR LIF membrane step=%0d lane=%0d got=%0d expected=%0d",‘. The important operands are display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block; LIF: the value named ’lif’ in this block. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LIF membrane step=%0d lane=%0d got=%0d expected=%0d", reference_timesteps, lane, $signed(got_membrane), expected_membrane[lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task run step.

Line 128: `reference_timesteps, lane, $signed(got_membrane), expected_membrane[lane]);` This is continuation line 3 of the 3-line logical statement that begins at source line 126. Its exact fragment is ‘reference_timesteps, lane, $signed(got_membrane), expected_membrane[lane]);‘. The important operands are reference_timesteps: one discrete SNN update instant; lane: one parallel arithmetic element within a vector or tensor datapath; got_membrane: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LIF membrane step=%0d lane=%0d got=%0d expected=%0d", reference_timesteps, lane, $signed(got_membrane), expected_membrane[lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task run step.

Line 129: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 130: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 131: `if ($signed(got_pre_reset_membrane)` This condition selects the following hardware branch only when ‘$signed(got_pre_reset_membrane‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are got_pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the simulation task run step.

Line 132: `!== expected_pre_reset_membrane[lane]) begin` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are expected_pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 133: `errors++;` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 134: `if (errors <= 20) begin` This statement continues through source line 138; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LIF pre-reset membrane step=%0d lane=%0d got=%0d expected=%0d", reference_timesteps, lane, $signed(got_pre_reset_membrane), expected_pre_reset_membrane[lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run step.

Line 135: `$display("ERROR LIF pre-reset membrane step=%0d lane=%0d got=%0d expected=%0d",` This is continuation line 2 of the 5-line logical statement that begins at source line 134. Its exact fragment is ‘$display("ERROR LIF pre-reset membrane step=%0d lane=%0d got=%0d expected=%0d",‘. The important operands are display: the value named 'display' in this block; ERROR: the value named 'error' in this block; LIF: the value named 'lif' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LIF pre-reset membrane step=%0d lane=%0d got=%0d expected=%0d", reference_timesteps, lane, $signed(got_pre_reset_membrane), expected_pre_reset_membrane[lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run step.

Line 136: `reference_timesteps, lane,` This is continuation line 3 of the 5-line logical statement that begins at source line 134. Its exact fragment is ‘reference_timesteps, lane,‘. The important operands are reference_timesteps: one discrete SNN update instant; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LIF pre-reset membrane step=%0d lane=%0d got=%0d expected=%0d", reference_timesteps, lane, $signed(got_pre_reset_membrane), expected_pre_reset_membrane[lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run step.

Line 137: `$signed(got_pre_reset_membrane),` This is continuation line 4 of the 5-line logical statement that begins at source line 134. Its exact fragment is ‘$signed(got_pre_reset_membrane),‘. The important operands are got_pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LIF pre-reset membrane step=%0d lane=%0d got=%0d expected=%0d", reference_timesteps, lane, $signed(got_pre_reset_membra expected_pre_reset_membrane[lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run step.

Line 138: `expected_pre_reset_membrane[lane]);` This is continuation line 5 of the 5-line logical statement that begins at source line 134. Its exact fragment is ‘expected_pre_reset_membrane[lane]);‘. The important operands are expected_pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LIF pre-reset membrane step=%0d lane=%0d got=%0d expected=%0d", reference_timesteps, lane, $signed(got_pre_reset_membrane), expected_pre_reset_membrane[lane]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run step.

Line 139: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 140: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 141: `reference_membrane[lane] = expected_membrane[lane];` This makes reference_membrane[lane], the persistent LIF neuron state carrying charge between timesteps, equal to ‘expected_membrane[lane]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are expected_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task run step.

Line 142: `reference_spikes += expected_spikes[lane];` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are reference_spikes: a binary event produced when a neuron crosses its threshold; expected_spikes: a binary event produced when a neuron crosses its threshold; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 143: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 144: `reference_timesteps++;` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are reference_timesteps: one discrete SNN update instant.

Line 146: `output_ready = 1'b1;` This makes output_ready, the consumer’s statement that it can accept a payload this cycle, equal to ‘1’b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named ’b1’ in this block. Context: the simulation task run step.

Line 147: `@(posedge clk);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 148: `@(negedge clk);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 149: `output_ready = 1'b0;` This makes output_ready, the consumer’s statement that it can accept a payload this cycle, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task run step.

Line 150: `if (timestep_count != reference_timesteps || total_spike_count != reference_spikes) begin` This condition selects the following hardware branch only when ‘timestep_count != reference_timesteps || total_spike_count != reference_spikes‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are timestep_count: one discrete SNN update instant; reference_timesteps: one discrete SNN update instant; total_spike_count: a binary event produced when a neuron crosses its threshold. Context: the simulation task run step.

Line 151: `errors++;` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 152: `$display("ERROR LIF counters got=(%0d,%0d) expected=(%0d,%0d)",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 153: `timestep_count, total_spike_count, reference_timesteps, reference_spikes);` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are timestep_count: one discrete SNN update instant; total_spike_count: a binary event produced when a neuron crosses its threshold; reference_timesteps: one discrete SNN update instant.

Line 154: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 155: `checked++;` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 156: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run step; it has no arithmetic value of its own.

Line 157: `endtask` Within the simulation task run step, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 159: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 160: `int seed;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are seed: the value named 'seed' in this block.

Line 161: `seed = 32'h1f1f2026;` This makes seed, the value named 'seed' in this block, equal to '32'h1f1f2026'. It is driven by combinational or procedural logic in the current evaluation. The important operands are h1f1f2026: the value named 'h1f1f2026' in this block. Context: the module's structural declarations.

Line 162: `for (int lane = 0; lane < LANES; lane++) reference_membrane[lane] = 0;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 164: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 165: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 166: `clear_reference_and_dut(0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are clear_reference_and_dut: the value named 'clear reference and dut' in this block.

Line 168: `for (int step = 0; step < 100; step++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 169: `for (int lane = 0; lane < LANES; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 170: `current_vector[lane*CURRENT_BITS +: CURRENT_BITS] =` This statement continues through source line 171; the explanation uses the complete expression rather than only the first visual fragment. Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are current_vector: the value named 'current vector' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; CURRENT_BITS: the value named 'current bits' in this block.

Line 171: `16'($signed(($urandom(seed) % 241) - 120));` This is continuation line 2 of the 2-line logical statement that begins at source line 170. Its exact fragment is '16'($signed(($urandom(seed) % 241) - 120));'. The important operands are urandom: the value named 'urandom' in this block; seed: the value named 'seed' in this block. Read together, the complete operation is: Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are current_vector: the value named 'current vector' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; CURRENT_BITS: the value named 'current bits' in this block.

Line 172: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 173: `leak_shift = 5'(1 + step % 5);` This makes leak_shift, the retained fraction of prior membrane state, equal to '5'(1 + step % 5)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are step: the value named 'step' in this block. Context: the module's structural declarations.

Line 174: `threshold = 24'sd100;` This makes threshold, the membrane level at which a neuron emits a spike, equal to '24'sd100'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd100: the value named 'sd100' in this block. Context: the module's structural declarations.

Line 175: `run_step(step % 4);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are run_step: the value named 'run step' in this block; step: the value named 'step' in this block.

Line 176: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 178: `// Explicit positive saturation boundary.` Design intent: Explicit positive saturation boundary. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation.

Line 179: `clear_reference_and_dut(MAXIMUM_MEMBRANE - 4);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are clear_reference_and_dut: the value named 'clear reference and dut' in this block; MAXIMUM_MEMBRANE: the persistent LIF neuron state carrying charge between timesteps.

Line 180: `leak_shift = 5'd31;` This makes leak_shift, the retained fraction of prior membrane state, equal to '5'd31'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d31: the value named 'd31' in this block. Context: the module's structural declarations.

Line 181: `threshold = 24'sd8388607;` This makes threshold, the membrane level at which a neuron emits a spike, equal to '24'sd8388607'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd8388607: the value named 'sd8388607' in this block. Context: the module's structural declarations.

Line 182: `current_vector = '0;` This makes current_vector, the value named 'current vector' in this block, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 183: `current_vector[CURRENT_BITS-1:0] = 16'sd32767;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are current_vector: the value named 'current vector' in this block; CURRENT_BITS: the value named 'current bits' in this block; sd32767: the value named 'sd32767' in this block.

Line 184: `run_step(2);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are run_step: the value named 'run step' in this block.

Line 186: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 187: `clear_counters = 1'b1;` This makes clear_counters, a hardware telemetry register that accumulates measured events, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 188: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 189: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 190: `clear_counters = 1'b0;` This makes clear_counters, a hardware telemetry register that accumulates measured events, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 191: `reference_spikes = 0;` This makes reference_spikes, a binary event produced when a neuron crosses its threshold, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 192: `if (total_spike_count != 0) begin` This condition selects the following hardware branch only when 'total_spike_count != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are total_spike_count: a binary event produced when a neuron crosses its threshold. Context: the module's structural declarations.

Line 193: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 194: `$display("ERROR LIF spike counter did not clear independently");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 195: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 197: `if (errors != 0) begin` This condition selects the following hardware branch only when ‘errors != 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block. Context: the module’s structural declarations.

Line 198: `$display("tb_neuraedge_lif_bank FAILED errors=%0d checked=%0d", errors, checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 199: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 200: `end else begin` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 201: `$display("tb_neuraedge_lif_bank PASSED checked=%0d timesteps=%0d spikes=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 202: `checked, reference_timesteps, reference_spikes);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The important operands are checked: the value named ’checked’ in this block; reference_timesteps: one discrete SNN update instant; reference_spikes: a binary event produced when a neuron crosses its threshold.

Line 203: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 204: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 205: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 207: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 208: `#200_000;` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 209: `$fatal(1, "tb_neuraedge_lif_bank watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 210: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 211: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 213: `‘default_nettype wire` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge lif bank RTL block in simulation.

52 tb/tb_neuraedge_linear_backward8.sv

Role in the complete system: drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

52.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Simulation-only status tb_neuraedge_linear_backward8 is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

52.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module tb_neuraedge_linear_backward8;
5      localparam int LANES = 8;
6      localparam int RANDOM_CASES = 48;
7
8      logic clk = 1'b0;
9      logic rst_n = 1'b0;
10     logic clear_counters = 1'b0;
11     logic start_valid = 1'b0;
12     logic start_ready;
13     logic [3:0] active_rows = 4'd0;
14     logic [3:0] active_inputs = 4'd0;
15     logic [3:0] active_outputs = 4'd0;
16     logic [4:0] input_gradient_shift = 5'd0;
17     logic [4:0] weight_gradient_shift = 5'd0;
18     logic [1:0] gradient_mode = 2'd0;
19     logic accumulate_result = 1'b0;
20     logic suppress_output = 1'b0;
21     logic reserved_nonzero = 1'b0;
22     logic s_axis_tvalid = 1'b0;
23     logic s_axis_tready;
24     logic [63:0] s_axis_tdata = 64'd0;
25     logic s_axis_tlast = 1'b0;
26     logic m_axis_tvalid;
27     logic m_axis_tready = 1'b0;
28     logic [63:0] m_axis_tdata;
29     logic m_axis_tlast;
30     logic busy;
31     logic done;
32     logic config_error;
33     logic [31:0] completed_tiles;
34     logic [31:0] active_cycles;
35
36     integer activation [0:LANES-1][0:LANES-1];
37     integer weight [0:LANES-1][0:LANES-1];
38     integer output_gradient [0:LANES-1][0:LANES-1];
39     logic signed [31:0] expected_input_gradient [0:LANES-1][0:LANES-1];
40     logic signed [23:0] expected_weight_gradient [0:LANES-1][0:LANES-1];
41
42     integer errors = 0;
43     integer checked_tiles = 0;
44     integer seed;
45     logic held_valid = 1'b0;
46     logic [63:0] held_data;
47     logic held_last;
48
49     neuraedge_linear_backward8 dut (
50         .clk(clk),
51         .rst_n(rst_n),
52         .clear_counters(clear_counters),
53         .start_valid(start_valid),
54         .start_ready(start_ready),
55         .active_rows(active_rows),
56         .active_inputs(active_inputs),
57         .active_outputs(active_outputs),
58         .input_gradient_shift(input_gradient_shift),
59         .weight_gradient_shift(weight_gradient_shift),
```

```
        .gradient_mode(gradient_mode),
        .accumulate_result(accumulate_result),
        .suppress_output(suppress_output),
        .reserved_nonzero(reserved_nonzero),
        .s_axis_tvalid(s_axis_tvalid),
        .s_axis_tready(s_axis_tready),
        .s_axis_tdata(s_axis_tdata),
        .s_axis_tlast(s_axis_tlast),
        .m_axis_tvalid(m_axis_tvalid),
        .m_axis_tready(m_axis_tready),
        .m_axis_tdata(m_axis_tdata),
        .m_axis_tlast(m_axis_tlast),
        .busy(busy),
        .done(done),
        .config_error(config_error),
        .completed_tiles(completed_tiles),
        .active_cycles(active_cycles)
    );

    always #2.5 clk = ~clk;

    always @(posedge clk) begin
        if (held_valid && (!m_axis_tvalid || m_axis_tdata !== held_data
                || m_axis_tlast !== held_last)) begin
            errors = errors + 1;
            $display("ERROR backward output changed while stalled");
        end
        held_valid <= m_axis_tvalid && !m_axis_tready;
        held_data <= m_axis_tdata;
        held_last <= m_axis_tlast;
    end

    function automatic signed [63:0] round_shift_ref(
        input signed [63:0] value,
        input integer shift
    );
        reg signed [63:0] magnitude;
        reg signed [63:0] rounded;
        begin
            if (shift == 0) begin
                round_shift_ref = value;
            end else begin
                magnitude = value < 0 ? -value : value;
                rounded = (magnitude + (64'sd1 << (shift - 1))) >>> shift;
                round_shift_ref = value < 0 ? -rounded : rounded;
            end
        end
    endfunction

    function automatic signed [31:0] saturate_s32_ref(
        input signed [63:0] value
    );
        begin
            if (value > 64'sd2147483647) saturate_s32_ref = 32'sh7fffffff;
            else if (value < -64'sd2147483648) saturate_s32_ref = 32'sh80000000;
            else saturate_s32_ref = value[31:0];
        end
    endfunction

    function automatic signed [23:0] saturate_s24_ref(
        input signed [63:0] value
    );
        begin
            if (value > 64'sd8388607) saturate_s24_ref = 24'sh7fffff;
            else if (value < -64'sd8388608) saturate_s24_ref = 24'sh800000;
            else saturate_s24_ref = value[23:0];
        end
    endfunction

    task automatic send_word(input logic [63:0] value, input logic last_value);
        begin
            while (!s_axis_tready) @(negedge clk);
            s_axis_tdata = value;
            s_axis_tlast = last_value;
            s_axis_tvalid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            s_axis_tvalid = 1'b0;
            s_axis_tlast = 1'b0;
            s_axis_tdata = 64'd0;
        end
    endtask

    task automatic receive_word(
        output logic [63:0] value,
        output logic last_value,
        input integer stall_cycles
```

```
    );
        begin
            while (!m_axis_tvalid) @(negedge clk);
            repeat (stall_cycles) @(negedge clk);
            value = m_axis_tdata;
            last_value = m_axis_tlast;
            m_axis_tready = 1'b1;
            @(posedge clk);
            @(negedge clk);
            m_axis_tready = 1'b0;
        end
    endtask

    task automatic start_tile(
        input integer rows,
        input integer inputs,
        input integer outputs,
        input integer dx_shift,
        input integer dw_shift,
        input logic bad_reserved
    );
        begin
            while (!start_ready) @(negedge clk);
            active_rows = rows;
            active_inputs = inputs;
            active_outputs = outputs;
            input_gradient_shift = dx_shift;
            weight_gradient_shift = dw_shift;
            gradient_mode = 2'd0;
            accumulate_result = 1'b0;
            suppress_output = 1'b0;
            reserved_nonzero = bad_reserved;
            start_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            start_valid = 1'b0;
            reserved_nonzero = 1'b0;
        end
    endtask

    task automatic start_partial_tile(
        input integer rows,
        input integer inputs,
        input integer outputs,
        input integer dx_shift,
        input integer dw_shift,
        input logic [1:0] mode_value,
        input logic accumulate_value,
        input logic suppress_value
    );
        begin
            while (!start_ready) @(negedge clk);
            active_rows = rows;
            active_inputs = inputs;
            active_outputs = outputs;
            input_gradient_shift = dx_shift;
            weight_gradient_shift = dw_shift;
            gradient_mode = mode_value;
            accumulate_result = accumulate_value;
            suppress_output = suppress_value;
            reserved_nonzero = 1'b0;
            start_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            start_valid = 1'b0;
        end
    endtask

    task automatic wait_suppressed_done;
        integer timeout;
        begin
            timeout = 0;
            while (!done && timeout < 10000) begin
                @(negedge clk);
                if (m_axis_tvalid) begin
                    errors = errors + 1;
                    $display("ERROR suppressed backward command emitted output");
                end
                timeout = timeout + 1;
            end
            if (!done) begin
                errors = errors + 1;
                $display("ERROR suppressed backward command timed out");
            end
            if (config_error) begin
                errors = errors + 1;
                $display("ERROR suppressed backward command reported configuration failure");
```

```
            end
            checked_tiles = checked_tiles + 1;
            @(posedge clk);
            @(negedge clk);
        end
    endtask

    task automatic run_exact_accumulation_modes;
        integer row;
        integer input_index;
        integer output_index;
        integer chunk;
        integer chunk_size;
        integer global_index;
        integer half;
        integer lane;
        integer beat_index;
        integer word_index;
        integer timeout;
        reg signed [63:0] accumulator;
        logic [63:0] stream_word;
        logic [63:0] received_word;
        logic received_last;
        logic [191:0] packed_weight_row;
        begin
            // dX is a reduction over output features. Two commands cover
            // thirteen features while one 47-bit accumulator preserves the
            // exact sum until the final round-by-16 operation.
            for (row = 0; row < LANES; row = row + 1) begin
                for (input_index = 0; input_index < LANES;
                        input_index = input_index + 1) begin
                    expected_input_gradient[row][input_index] = 32'sd0;
                end
            end
            for (row = 0; row < 5; row = row + 1) begin
                for (input_index = 0; input_index < 6;
                        input_index = input_index + 1) begin
                    accumulator = 64'sd0;
                    for (output_index = 0; output_index < 13;
                            output_index = output_index + 1) begin
                        accumulator = accumulator
                            + ((((row*71 + output_index*43 + 11) % 1801) - 900)
                            * (((input_index*97 + output_index*53 + 17) % 2001) - 1000));
                    end
                    expected_input_gradient[row][input_index] = saturate_s32_ref(
                        round_shift_ref(accumulator, 4));
                end
            end
            for (chunk = 0; chunk < 2; chunk = chunk + 1) begin
                chunk_size = chunk == 0 ? 8 : 5;
                start_partial_tile(5, 6, chunk_size, 4, 0, 2'd1,
                                   chunk != 0, chunk == 0);
                for (input_index = 0; input_index < 6;
                        input_index = input_index + 1) begin
                    for (half = 0; half < 2; half = half + 1) begin
                        stream_word = 64'd0;
                        for (lane = 0; lane < 4; lane = lane + 1) begin
                            output_index = half * 4 + lane;
                            if (output_index < chunk_size) begin
                                global_index = chunk * 8 + output_index;
                                stream_word[lane*16 +: 16] =
                                    ((input_index*97 + global_index*53 + 17) % 2001) - 1000;
                            end
                        end
                        send_word(stream_word, 1'b0);
                    end
                end
                for (row = 0; row < 5; row = row + 1) begin
                    for (half = 0; half < 2; half = half + 1) begin
                        stream_word = 64'd0;
                        for (lane = 0; lane < 4; lane = lane + 1) begin
                            output_index = half * 4 + lane;
                            if (output_index < chunk_size) begin
                                global_index = chunk * 8 + output_index;
                                stream_word[lane*16 +: 16] =
                                    ((row*71 + global_index*43 + 11) % 1801) - 900;
                            end
                        end
                        send_word(stream_word, row == 4 && half == 1);
                    end
                end
                if (chunk == 0) begin
                    wait_suppressed_done();
                end else begin
                    for (beat_index = 0; beat_index < 20;
                            beat_index = beat_index + 1) begin
                        receive_word(received_word, received_last, beat_index % 3);
```

```
                        row = beat_index / 4;
                        word_index = beat_index % 4;
                        for (lane = 0; lane < 2; lane = lane + 1) begin
                            input_index = word_index * 2 + lane;
                            if ($signed(received_word[lane*32 +: 32])
                                    !== expected_input_gradient[row][input_index]) begin
                                errors = errors + 1;
                                $display("ERROR accumulated dX row=%0d lane=%0d got=%0d expected=%0d",
                                        row, input_index,
                                        $signed(received_word[lane*32 +: 32]),
                                        expected_input_gradient[row][input_index]);
                            end
                        end
                        if (received_last != (beat_index == 19)) errors = errors + 1;
                    end
                    timeout = 0;
                    while (!done && timeout < 20) begin
                        @(negedge clk);
                        timeout = timeout + 1;
                    end
                    if (!done || config_error) errors = errors + 1;
                    checked_tiles = checked_tiles + 1;
                    @(posedge clk);
                    @(negedge clk);
                end
            end

            // dW is a reduction over rows. Thirteen rows are split 8+5; the
            // dW-only mode consequently omits every weight input beat.
            for (input_index = 0; input_index < LANES;
                    input_index = input_index + 1) begin
                for (output_index = 0; output_index < LANES;
                        output_index = output_index + 1) begin
                    expected_weight_gradient[input_index][output_index] = 24'sd0;
                end
            end
            for (input_index = 0; input_index < 6;
                    input_index = input_index + 1) begin
                for (output_index = 0; output_index < 7;
                        output_index = output_index + 1) begin
                    accumulator = 64'sd0;
                    for (row = 0; row < 13; row = row + 1) begin
                        accumulator = accumulator
                            + ((((row*29 + input_index*13 + 7) % 255) - 127)
                            * (((row*71 + output_index*43 + 11) % 1801) - 900));
                    end
                    expected_weight_gradient[input_index][output_index]
                        = saturate_s24_ref(round_shift_ref(accumulator, 3));
                end
            end
            for (chunk = 0; chunk < 2; chunk = chunk + 1) begin
                chunk_size = chunk == 0 ? 8 : 5;
                start_partial_tile(chunk_size, 6, 7, 0, 3, 2'd2,
                                   chunk != 0, chunk == 0);
                for (row = 0; row < chunk_size; row = row + 1) begin
                    global_index = chunk * 8 + row;
                    for (half = 0; half < 2; half = half + 1) begin
                        stream_word = 64'd0;
                        for (lane = 0; lane < 4; lane = lane + 1) begin
                            input_index = half * 4 + lane;
                            if (input_index < 6) begin
                                stream_word[lane*16 +: 16] =
                                    ((global_index*29 + input_index*13 + 7) % 255) - 127;
                            end
                        end
                        send_word(stream_word, 1'b0);
                    end
                end
                for (row = 0; row < chunk_size; row = row + 1) begin
                    global_index = chunk * 8 + row;
                    for (half = 0; half < 2; half = half + 1) begin
                        stream_word = 64'd0;
                        for (lane = 0; lane < 4; lane = lane + 1) begin
                            output_index = half * 4 + lane;
                            if (output_index < 7) begin
                                stream_word[lane*16 +: 16] =
                                    ((global_index*71 + output_index*43 + 11) % 1801) - 900;
                            end
                        end
                        send_word(stream_word,
                                  row == chunk_size - 1 && half == 1);
                    end
                end
                if (chunk == 0) begin
                    wait_suppressed_done();
                end else begin
                    packed_weight_row = 192'd0;
```

```
                    for (beat_index = 0; beat_index < 18;
                            beat_index = beat_index + 1) begin
                        receive_word(received_word, received_last, beat_index % 4);
                        input_index = beat_index / 3;
                        word_index = beat_index % 3;
                        packed_weight_row[word_index*64 +: 64] = received_word;
                        if (word_index == 2) begin
                            for (lane = 0; lane < LANES; lane = lane + 1) begin
                                if ($signed(packed_weight_row[lane*24 +: 24])
                                        !== expected_weight_gradient[input_index][lane]) begin
                                    errors = errors + 1;
                                    $display("ERROR accumulated dW row=%0d lane=%0d got=%0d expected=%0d",
                                             input_index, lane,
                                             $signed(packed_weight_row[lane*24 +: 24]),
                                             expected_weight_gradient[input_index][lane]);
                                end
                            end
                            packed_weight_row = 192'd0;
                        end
                        if (received_last != (beat_index == 17)) errors = errors + 1;
                    end
                    timeout = 0;
                    while (!done && timeout < 20) begin
                        @(negedge clk);
                        timeout = timeout + 1;
                    end
                    if (!done || config_error) errors = errors + 1;
                    checked_tiles = checked_tiles + 1;
                    @(posedge clk);
                    @(negedge clk);
                end
            end

            // Emitting the final result invalidates the retained shape.
            start_partial_tile(5, 6, 5, 4, 0, 2'd1, 1'b1, 1'b1);
            while (!done) @(negedge clk);
            if (!config_error) begin
                errors = errors + 1;
                $display("ERROR stale dX accumulator remained valid after final output");
            end
            @(posedge clk);
            @(negedge clk);
        end
    endtask

    task automatic prepare_case(
        input integer rows,
        input integer inputs,
        input integer outputs,
        input integer pattern,
        input integer dx_shift,
        input integer dw_shift
    );
        integer row;
        integer column;
        integer reduction;
        reg signed [63:0] accumulator;
        begin
            for (row = 0; row < LANES; row = row + 1) begin
                for (column = 0; column < LANES; column = column + 1) begin
                    activation[row][column] = 0;
                    weight[row][column] = 0;
                    output_gradient[row][column] = 0;
                    expected_input_gradient[row][column] = 32'sd0;
                    expected_weight_gradient[row][column] = 24'sd0;
                end
            end

            for (row = 0; row < rows; row = row + 1) begin
                for (column = 0; column < inputs; column = column + 1) begin
                    if (pattern == 1) activation[row][column] = 127;
                    else if (pattern == 2) activation[row][column] = -128;
                    else if (pattern == 3) activation[row][column] = 127;
                    else activation[row][column] = $urandom_range(255, 0) - 128;
                end
            end
            for (row = 0; row < inputs; row = row + 1) begin
                for (column = 0; column < outputs; column = column + 1) begin
                    if (pattern == 1) weight[row][column] = -32768;
                    else if (pattern == 2) weight[row][column] = 32767;
                    else if (pattern == 3) weight[row][column] = 32767;
                    else weight[row][column] = $urandom_range(65535, 0) - 32768;
                end
            end
            for (row = 0; row < rows; row = row + 1) begin
                for (column = 0; column < outputs; column = column + 1) begin
                    if (pattern == 1) output_gradient[row][column] = -32768;
```

```
                    else if (pattern == 2) output_gradient[row][column] = 32767;
                    else if (pattern == 3) output_gradient[row][column] = -32768;
                    else output_gradient[row][column] = $urandom_range(65535, 0) - 32768;
                end
            end

            for (row = 0; row < rows; row = row + 1) begin
                for (column = 0; column < inputs; column = column + 1) begin
                    accumulator = 64'sd0;
                    for (reduction = 0; reduction < outputs; reduction = reduction + 1) begin
                        accumulator = accumulator
                            + output_gradient[row][reduction] * weight[column][reduction];
                    end
                    expected_input_gradient[row][column] = saturate_s32_ref(
                        round_shift_ref(accumulator, dx_shift));
                end
            end
            for (row = 0; row < inputs; row = row + 1) begin
                for (column = 0; column < outputs; column = column + 1) begin
                    accumulator = 64'sd0;
                    for (reduction = 0; reduction < rows; reduction = reduction + 1) begin
                        accumulator = accumulator
                            + activation[reduction][row] * output_gradient[reduction][column];
                    end
                    expected_weight_gradient[row][column] = saturate_s24_ref(
                        round_shift_ref(accumulator, dw_shift));
                end
            end
        end
    endtask

    task automatic run_case(
        input integer rows,
        input integer inputs,
        input integer outputs,
        input integer pattern,
        input integer dx_shift,
        input integer dw_shift,
        input logic malformed_tlast
    );
        integer row;
        integer lane;
        integer half;
        integer beat_index;
        integer row_index;
        integer word_index;
        integer lane_index;
        integer total_output_beats;
        integer timeout;
        logic [63:0] stream_word;
        logic [63:0] received_word;
        logic received_last;
        logic [191:0] packed_weight_row;
        begin
            prepare_case(rows, inputs, outputs, pattern, dx_shift, dw_shift);
            start_tile(rows, inputs, outputs, dx_shift, dw_shift, 1'b0);

            for (row = 0; row < rows; row = row + 1) begin
                for (half = 0; half < 2; half = half + 1) begin
                    stream_word = 64'd0;
                    for (lane = 0; lane < 4; lane = lane + 1) begin
                        stream_word[lane*16 +: 16] = activation[row][half*4 + lane];
                    end
                    send_word(stream_word,
                              malformed_tlast && row == 0 && half == 0);
                end
            end
            for (row = 0; row < inputs; row = row + 1) begin
                for (half = 0; half < 2; half = half + 1) begin
                    stream_word = 64'd0;
                    for (lane = 0; lane < 4; lane = lane + 1) begin
                        stream_word[lane*16 +: 16] = weight[row][half*4 + lane];
                    end
                    send_word(stream_word, 1'b0);
                end
            end
            for (row = 0; row < rows; row = row + 1) begin
                for (half = 0; half < 2; half = half + 1) begin
                    stream_word = 64'd0;
                    for (lane = 0; lane < 4; lane = lane + 1) begin
                        stream_word[lane*16 +: 16] = output_gradient[row][half*4 + lane];
                    end
                    send_word(stream_word, row == rows - 1 && half == 1);
                end
            end

            total_output_beats = 4 * rows + 3 * inputs;
```

```
            packed_weight_row = 192'd0;
            for (beat_index = 0; beat_index < total_output_beats;
                    beat_index = beat_index + 1) begin
                receive_word(received_word, received_last, beat_index % 4);
                if (received_last != (beat_index == total_output_beats - 1)) begin
                    errors = errors + 1;
                    $display("ERROR backward TLAST case=%0d beat=%0d got=%0b",
                             checked_tiles, beat_index, received_last);
                end
                if (beat_index < 4 * rows) begin
                    row_index = beat_index / 4;
                    word_index = beat_index % 4;
                    for (lane = 0; lane < 2; lane = lane + 1) begin
                        lane_index = word_index * 2 + lane;
                        if ($signed(received_word[lane*32 +: 32])
                                !== expected_input_gradient[row_index][lane_index]) begin
                            errors = errors + 1;
                            if (errors <= 30) begin
                                $display("ERROR dX tile=%0d row=%0d lane=%0d got=%0d expected=%0d",
                                         checked_tiles, row_index, lane_index,
                                         $signed(received_word[lane*32 +: 32]),
                                         expected_input_gradient[row_index][lane_index]);
                            end
                        end
                    end
                end else begin
                    row_index = (beat_index - 4 * rows) / 3;
                    word_index = (beat_index - 4 * rows) % 3;
                    packed_weight_row[word_index*64 +: 64] = received_word;
                    if (word_index == 2) begin
                        for (lane = 0; lane < LANES; lane = lane + 1) begin
                            if ($signed(packed_weight_row[lane*24 +: 24])
                                    !== expected_weight_gradient[row_index][lane]) begin
                                errors = errors + 1;
                                if (errors <= 30) begin
                                    $display("ERROR dW tile=%0d row=%0d lane=%0d got=%0d expected=%0d",
                                             checked_tiles, row_index, lane,
                                             $signed(packed_weight_row[lane*24 +: 24]),
                                             expected_weight_gradient[row_index][lane]);
                                end
                            end
                        end
                        packed_weight_row = 192'd0;
                    end
                end
            end

            timeout = 0;
            while (!done && timeout < 20) begin
                @(negedge clk);
                timeout = timeout + 1;
            end
            if (!done) begin
                errors = errors + 1;
                $display("ERROR backward done timeout");
            end
            if (config_error !== malformed_tlast) begin
                errors = errors + 1;
                $display("ERROR backward config_error tile=%0d got=%0b expected=%0b",
                         checked_tiles, config_error, malformed_tlast);
            end
            checked_tiles = checked_tiles + 1;
            @(posedge clk);
            @(negedge clk);
        end
    endtask

    initial begin
        integer case_index;
        integer rows;
        integer inputs;
        integer outputs;
        integer dx_shift;
        integer dw_shift;

        seed = 32'h17bacc8d;
        seed = $urandom(seed);
        repeat (5) @(negedge clk);
        rst_n = 1'b1;

        run_case(8, 8, 8, 1, 0, 0, 1'b0);
        run_case(8, 8, 8, 2, 0, 0, 1'b0);
        run_case(8, 8, 8, 3, 3, 2, 1'b0);
        run_case(1, 1, 1, 0, 5, 7, 1'b0);
        for (case_index = 0; case_index < RANDOM_CASES; case_index = case_index + 1) begin
            rows = $urandom_range(8, 1);
            inputs = $urandom_range(8, 1);
```

```
            outputs = $urandom_range(8, 1);
            dx_shift = $urandom_range(12, 0);
            dw_shift = $urandom_range(12, 0);
            run_case(rows, inputs, outputs, 0, dx_shift, dw_shift, 1'b0);
        end

        // A premature TLAST must be reported while arithmetic remains exact.
        run_case(3, 5, 7, 0, 4, 3, 1'b1);

        run_exact_accumulation_modes();

        // Invalid dimensions and nonzero reserved bits complete without data.
        start_tile(0, 1, 1, 0, 0, 1'b0);
        while (!done) @(negedge clk);
        if (!config_error) begin
            errors = errors + 1;
            $display("ERROR backward accepted zero rows");
        end
        @(posedge clk);
        @(negedge clk);
        start_tile(1, 1, 1, 0, 0, 1'b1);
        while (!done) @(negedge clk);
        if (!config_error) begin
            errors = errors + 1;
            $display("ERROR backward accepted reserved command bits");
        end
        @(posedge clk);
        @(negedge clk);

        if (completed_tiles != checked_tiles || active_cycles == 0) begin
            errors = errors + 1;
            $display("ERROR backward counters tiles=%0d/%0d cycles=%0d",
                     completed_tiles, checked_tiles, active_cycles);
        end
        clear_counters = 1'b1;
        @(posedge clk);
        @(negedge clk);
        clear_counters = 1'b0;
        if (completed_tiles != 0 || active_cycles != 0) begin
            errors = errors + 1;
            $display("ERROR backward counters did not clear");
        end

        if (errors != 0) begin
            $display("tb_neuraedge_linear_backward8 FAILED errors=%0d tiles=%0d",
                     errors, checked_tiles);
            $finish;
        end
        $display("tb_neuraedge_linear_backward8 PASSED tiles=%0d random=%0d",
                 checked_tiles, RANDOM_CASES);
        $finish;
    end

    initial begin
        #10_000_000;
        $fatal(1, "tb_neuraedge_linear_backward8 watchdog timeout state=%0d", dut.state);
    end
endmodule

`default_nettype wire
```

52.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module tb_neuraedge_linear_backward8;` This opens tb_neuraedge_linear_backward8;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Line 5: `localparam int LANES = 8;` This fixes LANES to 8 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 6: `localparam int RANDOM_CASES = 48;` This fixes RANDOM_CASES to 48 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; RANDOM_CASES denotes the value named 'random cases' in this block.

Line 8: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic clear_counters = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clear_counters is a hardware telemetry register that accumulates measured events. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic start_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, start_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic start_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, start_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic [3:0] active_rows = 4'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_rows is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic [3:0] active_inputs = 4'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_inputs is the value named 'active inputs' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic [3:0] active_outputs = 4'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_outputs is the value named 'active outputs' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic [4:0] input_gradient_shift = 5'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_gradient_shift is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [4:0] weight_gradient_shift = 5'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_gradient_shift is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic [1:0] gradient_mode = 2'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gradient_mode is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic accumulate_result = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, accumulate_result is the value named 'accumulate result' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic suppress_output = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, suppress_output is the value named 'suppress output' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic reserved_nonzero = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, reserved_nonzero is the value named 'reserved nonzero' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic s_axis_tvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic s_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic [63:0] s_axis_tdata = 64'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tdata is the value named 's axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic s_axis_tlast = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `logic m_axis_tvalid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic m_axis_tready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 28: `logic [63:0] m_axis_tdata;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tdata is the value named 'm axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `logic m_axis_tlast;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic busy;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, busy is the value named 'busy' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `logic done;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, done is the value named 'done' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 32: `logic config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, config_error is the value named 'config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `logic [31:0] completed_tiles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, completed_tiles is the value named 'completed tiles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic [31:0] active_cycles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_cycles is the value named 'active cycles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `integer activation [0:LANES-1][0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 37: `integer weight [0:LANES-1][0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 38: `integer output_gradient [0:LANES-1][0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_gradient is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 39: `logic signed [31:0] expected_input_gradient [0:LANES-1][0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, expected_input_gradient is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 40: `logic signed [23:0] expected_weight_gradient [0:LANES-1][0:LANES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, expected_weight_gradient is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 42: `integer errors = 0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, errors is the value named 'errors' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 43: `integer checked_tiles = 0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, checked_tiles is the value named 'checked tiles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 44: `integer seed;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, seed is the value named 'seed' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 45: `logic held_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 46: `logic [63:0] held_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 47: `logic held_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 49: `neuraedge_linear_backward8 dut (` This instantiates module neuraedge_linear_backward8 as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 50: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 51: `.rst_n(rst_n),` This connects the child instance's named port 'rst_n' to parent expression 'rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 52: `.clear_counters(clear_counters),` This connects the child instance's named port 'clear_counters' to parent expression 'clear_counters'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a hardware telemetry register that accumulates measured events.

Line 53: `.start_valid(start_valid),` This connects the child instance's named port 'start_valid' to parent expression 'start_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 54: `.start_ready(start_ready),` This connects the child instance's named port 'start_ready' to parent expression 'start_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 55: `.active_rows(active_rows),` This connects the child instance's named port 'active_rows' to parent expression 'active_rows'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one output-row position processed by the tensor tile.

Line 56: `.active_inputs(active_inputs),` This connects the child instance's named port 'active_inputs' to parent expression 'active_inputs'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'active inputs' in this block.

Line 57: `.active_outputs(active_outputs),` This connects the child instance's named port 'active_outputs' to parent expression 'active_outputs'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'active outputs' in this block.

Line 58: `.input_gradient_shift(input_gradient_shift),` This connects the child instance's named port 'input_gradient_shift' to parent expression 'input_gradient_shift'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the signed loss derivative used to update a learned parameter.

Line 59: `.weight_gradient_shift(weight_gradient_shift),` This connects the child instance's named port 'weight_gradient_ to parent expression 'weight_gradient_shift'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 60: `.gradient_mode(gradient_mode),` This connects the child instance's named port 'gradient_mode' to parent expression 'gradient_mode'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the signed loss derivative used to update a learned parameter.

Line 61: `.accumulate_result(accumulate_result),` This connects the child instance's named port 'accumulate_result' to parent expression 'accumulate_result'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'accumulate result' in this block.

Line 62: `.suppress_output(suppress_output),` This connects the child instance's named port 'suppress_output' to parent expression 'suppress_output'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'suppress output' in this block.

Line 63: `.reserved_nonzero(reserved_nonzero),` This connects the child instance's named port 'reserved_nonzero' to parent expression 'reserved_nonzero'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'reserved nonzero' in this block.

Line 64: `.s_axis_tvalid(s_axis_tvalid),` This connects the child instance's named port 's_axis_tvalid' to parent expression 's_axis_tvalid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 65: `.s_axis_tready(s_axis_tready),` This connects the child instance's named port 's_axis_tready' to parent expression 's_axis_tready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 66: `.s_axis_tdata(s_axis_tdata),` This connects the child instance's named port 's_axis_tdata' to parent expression 's_axis_tdata'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 's axis tdata' in this block.

Line 67: `.s_axis_tlast(s_axis_tlast),` This connects the child instance's named port 's_axis_tlast' to parent expression 's_axis_tlast'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 68: `.m_axis_tvalid(m_axis_tvalid),` This connects the child instance's named port 'm_axis_tvalid' to parent expression 'm_axis_tvalid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 69: `.m_axis_tready(m_axis_tready),` This connects the child instance's named port 'm_axis_tready' to parent expression 'm_axis_tready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 70: `.m_axis_tdata(m_axis_tdata),` This connects the child instance's named port 'm_axis_tdata' to parent expression 'm_axis_tdata'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'm axis tdata' in this block.

Line 71: `.m_axis_tlast(m_axis_tlast),` This connects the child instance's named port 'm_axis_tlast' to parent expression 'm_axis_tlast'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 72: `.busy(busy),` This connects the child instance's named port 'busy' to parent expression 'busy'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'busy' in this block.

Line 73: `.done(done),` This connects the child instance's named port 'done' to parent expression 'done'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'done' in this block.

Line 74: `.config_error(config_error),` This connects the child instance's named port 'config_error' to parent expression 'config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'config error' in this block.

Line 75: `.completed_tiles(completed_tiles),` This connects the child instance's named port 'completed_tiles' to parent expression 'completed_tiles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'completed tiles' in this block.

Line 76: `.active_cycles(active_cycles)` This connects the child instance's named port 'active_cycles' to parent expression 'active_cycles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'active cycles' in this block.

Line 77: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 79: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 81: `always @(posedge clk) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are always: the value named 'always' in this block; posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 82: `if (held_valid && (!m_axis_tvalid || m_axis_tdata !== held_data` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are held_valid: the producer's statement that its current payload is meaningful; m_axis_tvalid: the producer's statement that its current payload is meaningful; m_axis_tdata: the value named 'm axis tdata' in this block.

Line 83: `|| m_axis_tlast !== held_last)) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; held_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 84: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 85: `$display("ERROR backward output changed while stalled");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 86: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 87: `held_valid <= m_axis_tvalid && !m_axis_tready;` This makes held_valid, the producer's statement that its current payload is meaningful, equal to 'm_axis_tvalid && !m_axis_tready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are m_axis_tvalid: the producer's statement that its current payload is meaningful; m_axis_tready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 88: `held_data <= m_axis_tdata;` This makes held_data, the payload carried with the surrounding valid/ready handshake, equal to 'm_axis_tdata'. It updates after the active clock edge together with other nonblocking assignments. The important operands are m_axis_tdata: the value named 'm axis tdata' in this block. Context: the module's structural declarations.

Line 89: `held_last <= m_axis_tlast;` This makes held_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'm_axis_tlast'. It updates after the active clock edge together with other nonblocking assignments. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the module's structural declarations.

Line 90: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 92: `function automatic signed [63:0] round_shift_ref(` Within the combinational helper round shift ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; round_shift_ref: the value named 'round shift ref' in this block.

Line 93: `input signed [63:0] value,` This declares value as a input port with width [63:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 94: `input integer shift` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 95: `);` Within the combinational helper round shift ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 96: `reg signed [63:0] magnitude;` This declares hardware signal storage or wiring in the combinational helper round shift ref. Specifically, magnitude is the value named 'magnitude' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 97: `reg signed [63:0] rounded;` This declares hardware signal storage or wiring in the combinational helper round shift ref. Specifically, rounded is the value named 'rounded' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 98: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper round shift ref; it has no arithmetic value of its own.

Line 99: `if (shift == 0) begin` This condition selects the following hardware branch only when 'shift == 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are shift: the value named 'shift' in this block. Context: the combinational helper round shift ref.

Line 100: `round_shift_ref = value;` This makes round_shift_ref, the value named 'round shift ref' in this block, equal to 'value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper round shift ref.

Line 101: `end else begin` This structural keyword opens or closes the branch/process scope described as the combinational helper round shift ref; it has no arithmetic value of its own.

Line 102: `magnitude = value < 0 ? -value : value;` This makes magnitude, the value named 'magnitude' in this block, equal to 'value < 0 ? -value : value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper round shift ref.

Line 103: `rounded = (magnitude + (64'sd1 << (shift - 1))) >>> shift;` This makes rounded, the value named 'rounded' in this block, equal to '(magnitude + (64'sd1 << (shift - 1))) >>> shift'. It is driven by combinational or procedural logic in the current evaluation. The important operands are magnitude: the value named 'magnitude' in this block; sd1: the value named 'sd1' in this block; shift: the value named 'shift' in this block. Context: the combinational helper round shift ref.

Line 104: `round_shift_ref = value < 0 ? -rounded : rounded;` This makes round_shift_ref, the value named 'round shift ref' in this block, equal to 'value < 0 ? -rounded : rounded'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block; rounded: the value named 'rounded' in this block. Context: the combinational helper round shift ref.

Line 105: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper round shift ref; it has no arithmetic value of its own.

Line 106: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper round shift ref; it has no arithmetic value of its own.

Line 107: `endfunction` Within the combinational helper round shift ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endfunction: the value named 'endfunction' in this block.

Line 109: `function automatic signed [31:0] saturate_s32_ref(` Within the combinational helper saturate s32 ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_s32_ref: the value named 'saturate s32 ref' in this block.

Line 110: `input signed [63:0] value` This declares value as a input port with width [63:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 111: `);` Within the combinational helper saturate s32 ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 112: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32 ref; it has no arithmetic value of its own.

Line 113: `if (value > 64'sd2147483647) saturate_s32_ref = 32'sh7fffffff;` This condition selects the following hardware branch only when 'value > 64'sd2147483647' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; sd2147483647: the value named 'sd2147483647' in this block. Context: the combinational helper saturate s32 ref.

Line 114: `else if (value < -64'sd2147483648) saturate_s32_ref = 32'sh80000000;` This condition selects the following hardware branch only when 'value < -64'sd2147483648' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; sd2147483648: the value named 'sd2147483648' in this block. Context: the combinational helper saturate s32 ref.

Line 115: `else saturate_s32_ref = value[31:0];` Within the combinational helper saturate s32 ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are saturate_s32_ref: the value named 'saturate s32 ref' in this block; value: the value named 'value' in this block.

Line 116: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s32 ref; it has no arithmetic value of its own.

Line 117: `endfunction` Within the combinational helper saturate s32 ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endfunction: the value named 'endfunction' in this block.

Line 119: `function automatic signed [23:0] saturate_s24_ref(` Within the combinational helper saturate s24 ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; saturate_s24_ref: the value named 'saturate s24 ref' in this block.

Line 120: `input signed [63:0] value` This declares value as a input port with width [63:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 121: `);` Within the combinational helper saturate s24 ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 122: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24 ref; it has no arithmetic value of its own.

Line 123: `if (value > 64'sd8388607) saturate_s24_ref = 24'sh7fffff;` This condition selects the following hardware branch only when 'value > 64'sd8388607' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; sd8388607: the value named 'sd8388607' in this block. Context: the combinational helper saturate s24 ref.

Line 124: `else if (value < -64'sd8388608) saturate_s24_ref = 24'sh800000;` This condition selects the following hardware branch only when 'value < -64'sd8388608' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; sd8388608: the value named 'sd8388608' in this block. Context: the combinational helper saturate s24 ref.

Line 125: `else saturate_s24_ref = value[23:0];` Within the combinational helper saturate s24 ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are saturate_s24_ref: the value named 'saturate s24 ref' in this block; value: the value named 'value' in this block.

Line 126: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper saturate s24 ref; it has no arithmetic value of its own.

Line 127: `endfunction` Within the combinational helper saturate s24 ref, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endfunction: the value named 'endfunction' in this block.

Line 129: `task automatic send_word(input logic [63:0] value, input logic last_value);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; send_word: the value named 'send word' in this block.

Line 130: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task send word; it has no arithmetic value of its own.

Line 131: `while (!s_axis_tready) @(negedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axis_tready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 132: `s_axis_tdata = value;` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to 'value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the simulation task send word.

Line 133: `s_axis_tlast = last_value;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘last_value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are last_value: the AXI-Stream marker for the final beat of a packet or command payload. Context: the simulation task send word.

Line 134: `s_axis_tvalid = 1'b1;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to ‘1'b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task send word.

Line 135: `@(posedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 136: `@(negedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 137: `s_axis_tvalid = 1'b0;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send word.

Line 138: `s_axis_tlast = 1'b0;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1'b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send word.

Line 139: `s_axis_tdata = 64'd0;` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to ‘64'd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task send word.

Line 140: `end` This structural keyword opens or closes the branch/process scope described as the simulation task send word; it has no arithmetic value of its own.

Line 141: `endtask` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 143: `task automatic receive_word(` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; receive_word: the value named 'receive word' in this block.

Line 144: `output logic [63:0] value,` This declares value as a output port with width [63:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 145: `output logic last_value,` This declares last_value as a output port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 146: `input integer stall_cycles` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 147: `);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 148: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task receive word; it has no arithmetic value of its own.

Line 149: `while (!m_axis_tvalid) @(negedge clk);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; m_axis_tvalid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 150: `repeat (stall_cycles) @(negedge clk);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; stall_cycles: the value named 'stall cycles' in this block; negedge: the value named 'negedge' in this block.

Line 151: `value = m_axis_tdata;` This makes value, the value named 'value' in this block, equal to 'm_axis_tdata'. It is driven by combinational or procedural logic in the current evaluation. The important operands are m_axis_tdata: the value named 'm axis tdata' in this block. Context: the simulation task receive word.

Line 152: `last_value = m_axis_tlast;` This makes last_value, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'm_axis_tlast'. It is driven by combinational or procedural logic in the current evaluation. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the simulation task receive word.

Line 153: `m_axis_tready = 1'b1;` This makes m_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task receive word.

Line 154: `@(posedge clk);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 155: `@(negedge clk);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 156: `m_axis_tready = 1'b0;` This makes m_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task receive word.

Line 157: `end` This structural keyword opens or closes the branch/process scope described as the simulation task receive word; it has no arithmetic value of its own.

Line 158: `endtask` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 160: `task automatic start_tile(` Within the simulation task start tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; start_tile: the value named 'start tile' in this block.

Line 161: `input integer rows,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 162: `input integer inputs,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 163: `input integer outputs,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 164: `input integer dx_shift,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 165: `input integer dw_shift,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 166: `input logic bad_reserved` This declares bad_reserved as a input port with width one bit. It carries the value named 'bad reserved' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 167: `);` Within the simulation task start tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 168: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task start tile; it has no arithmetic value of its own.

Line 169: `while (!start_ready) @(negedge clk);` Within the simulation task start tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; start_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 170: `active_rows = rows;` This makes active_rows, one output-row position processed by the tensor tile, equal to ‘rows‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are rows: one output-row position processed by the tensor tile. Context: the simulation task start tile.

Line 171: `active_inputs = inputs;` This makes active_inputs, the value named 'active inputs' in this block, equal to ‘inputs‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are inputs: the value named 'inputs' in this block. Context: the simulation task start tile.

Line 172: `active_outputs = outputs;` This makes active_outputs, the value named 'active outputs' in this block, equal to ‘outputs‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are outputs: the value named 'outputs' in this block. Context: the simulation task start tile.

Line 173: `input_gradient_shift = dx_shift;` This makes input_gradient_shift, the signed loss derivative used to update a learned parameter, equal to ‘dx_shift‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are dx_shift: the value named 'dx shift' in this block. Context: the simulation task start tile.

Line 174: `weight_gradient_shift = dw_shift;` This makes weight_gradient_shift, a learned coefficient multiplied by an activation, equal to ‘dw_shift‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are dw_shift: the value named ’dw shift’ in this block. Context: the simulation task start tile.

Line 175: `gradient_mode = 2'd0;` This makes gradient_mode, the signed loss derivative used to update a learned parameter, equal to ‘2’d0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named ’d0’ in this block. Context: the simulation task start tile.

Line 176: `accumulate_result = 1'b0;` This makes accumulate_result, the value named ’accumulate result’ in this block, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task start tile.

Line 177: `suppress_output = 1'b0;` This makes suppress_output, the value named ’suppress output’ in this block, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task start tile.

Line 178: `reserved_nonzero = bad_reserved;` This makes reserved_nonzero, the value named ’reserved nonzero’ in this block, equal to ‘bad_reserved‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are bad_reserved: the value named ’bad reserved’ in this block. Context: the simulation task start tile.

Line 179: `start_valid = 1'b1;` This makes start_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named ’b1’ in this block. Context: the simulation task start tile.

Line 180: `@(posedge clk);` Within the simulation task start tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 181: `@(negedge clk);` Within the simulation task start tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 182: `start_valid = 1'b0;` This makes start_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task start tile.

Line 183: `reserved_nonzero = 1'b0;` This makes reserved_nonzero, the value named ’reserved nonzero’ in this block, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task start tile.

Line 184: `end` This structural keyword opens or closes the branch/process scope described as the simulation task start tile; it has no arithmetic value of its own.

Line 185: `endtask` Within the simulation task start tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endtask: the value named ’endtask’ in this block.

Line 187: `task automatic start_partial_tile(` Within the simulation task start partial tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are task: the value named ’task’ in this block; automatic: the value named ’automatic’ in this block; start_partial_tile: the value named ’start partial tile’ in this block.

Line 188: `input integer rows,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 189: `input integer inputs,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 190: `input integer outputs,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 191: `input integer dx_shift,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 192: `input integer dw_shift,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 193: `input logic [1:0] mode_value,` This declares mode_value as a input port with width [1:0]. It carries the value named 'mode value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 194: `input logic accumulate_value,` This declares accumulate_value as a input port with width one bit. It carries the value named 'accumulate value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 195: `input logic suppress_value` This declares suppress_value as a input port with width one bit. It carries the value named 'suppress value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 196: `);` Within the simulation task start partial tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 197: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task start partial tile; it has no arithmetic value of its own.

Line 198: `while (!start_ready) @(negedge clk);` Within the simulation task start partial tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; start_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 199: `active_rows = rows;` This makes active_rows, one output-row position processed by the tensor tile, equal to ‘rows‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are rows: one output-row position processed by the tensor tile. Context: the simulation task start partial tile.

Line 200: `active_inputs = inputs;` This makes active_inputs, the value named 'active inputs' in this block, equal to ‘inputs‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are inputs: the value named 'inputs' in this block. Context: the simulation task start partial tile.

Line 201: `active_outputs = outputs;` This makes active_outputs, the value named 'active outputs' in this block, equal to ‘outputs‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are outputs: the value named 'outputs' in this block. Context: the simulation task start partial tile.

Line 202: `input_gradient_shift = dx_shift;` This makes input_gradient_shift, the signed loss derivative used to update a learned parameter, equal to ‘dx_shift‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are dx_shift: the value named ’dx shift’ in this block. Context: the simulation task start partial tile.

Line 203: `weight_gradient_shift = dw_shift;` This makes weight_gradient_shift, a learned coefficient multiplied by an activation, equal to ‘dw_shift‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are dw_shift: the value named ’dw shift’ in this block. Context: the simulation task start partial tile.

Line 204: `gradient_mode = mode_value;` This makes gradient_mode, the signed loss derivative used to update a learned parameter, equal to ‘mode_value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are mode_value: the value named ’mode value’ in this block. Context: the simulation task start partial tile.

Line 205: `accumulate_result = accumulate_value;` This makes accumulate_result, the value named ’accumulate result’ in this block, equal to ‘accumulate_value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are accumulate_value: the value named ’accumulate value’ in this block. Context: the simulation task start partial tile.

Line 206: `suppress_output = suppress_value;` This makes suppress_output, the value named ’suppress output’ in this block, equal to ‘suppress_value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are suppress_value: the value named ’suppress value’ in this block. Context: the simulation task start partial tile.

Line 207: `reserved_nonzero = 1'b0;` This makes reserved_nonzero, the value named ’reserved nonzero’ in this block, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task start partial tile.

Line 208: `start_valid = 1'b1;` This makes start_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named ’b1’ in this block. Context: the simulation task start partial tile.

Line 209: `@(posedge clk);` Within the simulation task start partial tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 210: `@(negedge clk);` Within the simulation task start partial tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 211: `start_valid = 1'b0;` This makes start_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task start partial tile.

Line 212: `end` This structural keyword opens or closes the branch/process scope described as the simulation task start partial tile; it has no arithmetic value of its own.

Line 213: `endtask` Within the simulation task start partial tile, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endtask: the value named ’endtask’ in this block.

Line 215: `task automatic wait_suppressed_done;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; wait_suppressed_done: the value named 'wait suppressed done' in this block.

Line 216: `integer timeout;` This declares hardware signal storage or wiring in the simulation task task. Specifically, timeout is the value named 'timeout' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 217: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 218: `timeout = 0;` This makes timeout, the value named 'timeout' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task task.

Line 219: `while (!done && timeout < 10000) begin` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; done: the value named 'done' in this block; timeout: the value named 'timeout' in this block.

Line 220: `@(negedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 221: `if (m_axis_tvalid) begin` This condition selects the following hardware branch only when 'm_axis_tvalid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are m_axis_tvalid: the producer's statement that its current payload is meaningful. Context: the simulation task task.

Line 222: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task task.

Line 223: `$display("ERROR suppressed backward command emitted output");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 224: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 225: `timeout = timeout + 1;` This makes timeout, the value named 'timeout' in this block, equal to 'timeout + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are timeout: the value named 'timeout' in this block. Context: the simulation task task.

Line 226: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 227: `if (!done) begin` This condition selects the following hardware branch only when '!done' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are done: the value named 'done' in this block. Context: the simulation task task.

Line 228: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task task.

Line 229: `$display("ERROR suppressed backward command timed out");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 230: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 231: `if (config_error) begin` This condition selects the following hardware branch only when ‘config_error‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are config_error: the value named 'config error' in this block. Context: the simulation task task.

Line 232: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to ‘errors + 1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task task.

Line 233: `$display("ERROR suppressed backward command reported configuration failure");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 234: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 235: `checked_tiles = checked_tiles + 1;` This makes checked_tiles, the value named 'checked tiles' in this block, equal to ‘checked_tiles + 1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are checked_tiles: the value named 'checked tiles' in this block. Context: the simulation task task.

Line 236: `@(posedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 237: `@(negedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 238: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 239: `endtask` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 241: `task automatic run_exact_accumulation_modes;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; run_exact_accumulation_modes: the value named 'run exact accumulation modes' in this block.

Line 242: `integer row;` This declares hardware signal storage or wiring in the simulation task task. Specifically, row is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 243: `integer input_index;` This declares hardware signal storage or wiring in the simulation task task. Specifically, input_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 244: `integer output_index;` This declares hardware signal storage or wiring in the simulation task task. Specifically, output_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 245: `integer chunk;` This declares hardware signal storage or wiring in the simulation task task. Specifically, chunk is the value named 'chunk' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 246: `integer chunk_size;` This declares hardware signal storage or wiring in the simulation task task. Specifically, chunk_size is the value named 'chunk size' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 247: `integer global_index;` This declares hardware signal storage or wiring in the simulation task task. Specifically, global_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 248: `integer half;` This declares hardware signal storage or wiring in the simulation task task. Specifically, half is the value named 'half' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 249: `integer lane;` This declares hardware signal storage or wiring in the simulation task task. Specifically, lane is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 250: `integer beat_index;` This declares hardware signal storage or wiring in the simulation task task. Specifically, beat_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 251: `integer word_index;` This declares hardware signal storage or wiring in the simulation task task. Specifically, word_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 252: `integer timeout;` This declares hardware signal storage or wiring in the simulation task task. Specifically, timeout is the value named 'timeout' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 253: `reg signed [63:0] accumulator;` This declares hardware signal storage or wiring in the simulation task task. Specifically, accumulator is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 254: `logic [63:0] stream_word;` This declares hardware signal storage or wiring in the simulation task task. Specifically, stream_word is the value named 'stream word' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 255: `logic [63:0] received_word;` This declares hardware signal storage or wiring in the simulation task task. Specifically, received_word is the value named 'received word' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 256: `logic received_last;` This declares hardware signal storage or wiring in the simulation task task. Specifically, received_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 257: `logic [191:0] packed_weight_row;` This declares hardware signal storage or wiring in the simulation task task. Specifically, packed_weight_row is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 258: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 259: `// dX is a reduction over output features. Two commands cover` Design intent: dX is a reduction over output features. Two commands cover This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Line 260: `// thirteen features while one 47-bit accumulator preserves the` Design intent: thirteen features while one 47-bit accumulator preserves the This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Line 261: `// exact sum until the final round-by-16 operation.` Design intent: exact sum until the final round-by-16 operation. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Line 262: `for (row = 0; row < LANES; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 263: `for (input_index = 0; input_index < LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 264: `input_index = input_index + 1) begin` This statement continues through source line 265; the explanation uses the complete expression rather than only the first visual fragment. This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘input_index + 1) begin expected_input_gradient[row][input_index] = 32’sd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; expected_input_gradient: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile. Context: the simulation task task.

Line 265: `expected_input_gradient[row][input_index] = 32’sd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 264. Its exact fragment is ‘expected_input_gradient[row][input_index] = 32’sd0;‘. The important operands are expected_input_gradient: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘input_index + 1) begin expected_input_gradient[row][input_index] = 32’sd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; expected_input_gradient: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile. Context: the simulation task task.

Line 266: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 267: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 268: `for (row = 0; row < 5; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 269: `for (input_index = 0; input_index < 6;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 270: `input_index = input_index + 1) begin` This statement continues through source line 271; the explanation uses the complete expression rather than only the first visual fragment. This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘input_index + 1) begin accumulator = 64’sd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; accumulator: a widened running sum that prevents intermediate overflow; sd0: the value named ’sd0’ in this block. Context: the simulation task task.

Line 271: `accumulator = 64’sd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 270. Its exact fragment is ‘accumulator = 64’sd0;‘. The important operands are accumulator: a widened running sum that prevents intermediate overflow; sd0: the value named ’sd0’ in this block. Read together, the complete operation is: This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘input_index + 1) begin accumulator = 64’sd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; accumulator: a widened running sum that prevents intermediate overflow; sd0: the value named ’sd0’ in this block. Context: the simulation task task.

Line 272: `for (output_index = 0; output_index < 13;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 273: `output_index = output_index + 1) begin` This statement continues through source line 276; the explanation uses the complete expression rather than only the first visual fragment. This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index + 1) begin accumulator = accumulator + ((((row*71 + output_index*43 + 11) % 1801) - 900) * (((input_index*97 + output_index*53 + 17) % 2001) - 1000))‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry; accumulator: a widened running sum that prevents intermediate overflow; row: one output-row position processed by the tensor tile. Context: the simulation task task.

Line 274: `accumulator = accumulator` This is continuation line 2 of the 4-line logical statement that begins at source line 273. Its exact fragment is ‘accumulator = accumulator‘. The important operands are accumulator: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index + 1) begin accumulator = accumulator + ((((row*71 + output_index*43 + 11) % 1801) - 900) * (((input_index*97 + output_index*53 + 17) % 2001) - 1000))‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry; accumulator: a widened running sum that prevents intermediate overflow; row: one output-row position processed by the tensor tile. Context: the simulation task task.

Line 275: `+ ((((row*71 + output_index*43 + 11) % 1801) - 900)` This is continuation line 3 of the 4-line logical statement that begins at source line 273. Its exact fragment is ‘+ ((((row*71 + output_index*43 + 11) % 1801) - 900)‘. The important operands are row: one output-row position processed by the tensor tile; output_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index + 1) begin accumulator = accumulator + ((((row*71 + output_index*43 + 11) % 1801) - 900) * (((input_index*97 + output_index*53 + 17) % 2001) - 1000))‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry; accumulator: a widened running sum that prevents intermediate overflow; row: one output-row position processed by the tensor tile. Context: the simulation task task.

Line 276: `* (((input_index*97 + output_index*53 + 17) % 2001) - 1000));` This is continuation line 4 of the 4-line logical statement that begins at source line 273. Its exact fragment is ‘* (((input_index*97 + output_index*53 + 17) % 2001) - 1000));‘. The important operands are input_index: an integer selecting one tensor element, lane, bank,

or state entry; output_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index + 1) begin accumulator = accumulator + ((((row*71 + output_index*43 + 11) % 1801) - 900) * (((input_index*97 + output_index*53 + 17) % 2001) - 1000))‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry; accumulator: a widened running sum that prevents intermediate overflow; row: one output-row position processed by the tensor tile. Context: the simulation task task.

Line 277: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 278: `expected_input_gradient[row][input_index] = saturate_s32_ref(` This statement continues through source line 279; the explanation uses the complete expression rather than only the first visual fragment. This makes expected_input_gradient[row][input_index], the signed loss derivative used to update a learned parameter, equal to ‘saturate_s32_ref(round_shift_ref(accumulator, 4))‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_s32_ref: the value named ’saturate s32 ref’ in this block; round_shift_ref: the value named ’round shift ref’ in this block; accumulator: a widened running sum that prevents intermediate overflow. Context: the simulation task task.

Line 279: `round_shift_ref(accumulator, 4));` This is continuation line 2 of the 2-line logical statement that begins at source line 278. Its exact fragment is ‘round_shift_ref(accumulator, 4));‘. The important operands are round_shift_ref: the value named ’round shift ref’ in this block; accumulator: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This makes expected_input_gradient[row][input_index], the signed loss derivative used to update a learned parameter, equal to ‘saturate_s32_ref(round_shift_ref(accumulator, 4))‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_s32_ref: the value named ’saturate s32 ref’ in this block; round_shift_ref: the value named ’round shift ref’ in this block; accumulator: a widened running sum that prevents intermediate overflow. Context: the simulation task task.

Line 280: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 281: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 282: `for (chunk = 0; chunk < 2; chunk = chunk + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 283: `chunk_size = chunk == 0 ?  8 :  5;` This makes chunk_size, the value named ’chunk size’ in this block, equal to ‘chunk == 0 ? 8 : 5‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are chunk: the value named ’chunk’ in this block. Context: the simulation task task.

Line 284: `start_partial_tile(5, 6, chunk_size, 4, 0, 2'd1,` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are start_partial_tile: the value named ’start partial tile’ in this block; chunk_size: the value named ’chunk size’ in this block; d1: the value named ’d1’ in this block.

Line 285: `chunk != 0, chunk == 0);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are chunk: the value named ’chunk’ in this block.

Line 286: `for (input_index = 0; input_index < 6;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 287: `input_index = input_index + 1) begin` This statement continues through source line 288; the explanation uses the complete expression rather than only the first visual fragment. This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'input_index + 1) begin for (half = 0; half < 2; half = half + 1) begin'. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block; half: the value named 'half' in this block. Context: the simulation task task.

Line 288: `for (half = 0; half < 2; half = half + 1) begin` This is continuation line 2 of the 2-line logical statement that begins at source line 287. Its exact fragment is 'for (half = 0; half < 2; half = half + 1) begin'. The important operands are for: the value named 'for' in this block; half: the value named 'half' in this block. Read together, the complete operation is: This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'input_index + 1) begin for (half = 0; half < 2; half = half + 1) begin'. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block; half: the value named 'half' in this block. Context: the simulation task task.

Line 289: `stream_word = 64'd0;` This makes stream_word, the value named 'stream word' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task task.

Line 290: `for (lane = 0; lane < 4; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 291: `output_index = half * 4 + lane;` This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'half * 4 + lane'. It is driven by combinational or procedural logic in the current evaluation. The important operands are half: the value named 'half' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task task.

Line 292: `if (output_index < chunk_size) begin` This condition selects the following hardware branch only when 'output_index < chunk_size' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry; chunk_size: the value named 'chunk size' in this block. Context: the simulation task task.

Line 293: `global_index = chunk * 8 + output_index;` This makes global_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'chunk * 8 + output_index'. It is driven by combinational or procedural logic in the current evaluation. The important operands are chunk: the value named 'chunk' in this block; output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 294: `stream_word[lane*16 +:  16] =` This statement continues through source line 295; the explanation uses the complete expression rather than only the first visual fragment. Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; input_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 295: `((input_index*97 + global_index*53 + 17) % 2001) - 1000;` This is continuation line 2 of the 2-line logical statement that begins at source line 294. Its exact fragment is '((input_index*97 + global_index*53 + 17) % 2001) - 1000;'. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; global_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; input_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 296: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 297: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 298: `send_word(stream_word, 1'b0);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; stream_word: the value named 'stream word' in this block; b0: the value named 'b0' in this block.

Line 299: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 300: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 301: `for (row = 0; row < 5; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 302: `for (half = 0; half < 2; half = half + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 303: `stream_word = 64'd0;` This makes stream_word, the value named 'stream word' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task task.

Line 304: `for (lane = 0; lane < 4; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 305: `output_index = half * 4 + lane;` This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'half * 4 + lane'. It is driven by combinational or procedural logic in the current evaluation. The important operands are half: the value named 'half' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task task.

Line 306: `if (output_index < chunk_size) begin` This condition selects the following hardware branch only when 'output_index < chunk_size' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry; chunk_size: the value named 'chunk size' in this block. Context: the simulation task task.

Line 307: `global_index = chunk * 8 + output_index;` This makes global_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'chunk * 8 + output_index'. It is driven by combinational or procedural logic in the current evaluation. The important operands are chunk: the value named 'chunk' in this block; output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 308: `stream_word[lane*16 +:  16] =` This statement continues through source line 309; the explanation uses the complete expression rather than only the first visual fragment. Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; row: one output-row position processed by the tensor tile.

Line 309: `((row*71 + global_index*43 + 11) % 1801) - 900;` This is continuation line 2 of the 2-line logical statement that begins at source line 308. Its exact fragment is ‘((row*71 + global_index*43 + 11) % 1801) - 900;‘. The important operands are row: one output-row position processed by the tensor tile; global_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named ’stream word’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; row: one output-row position processed by the tensor tile.

Line 310: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 311: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 312: `send_word(stream_word, row == 4 && half == 1);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are send_word: the value named ’send word’ in this block; stream_word: the value named ’stream word’ in this block; row: one output-row position processed by the tensor tile.

Line 313: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 314: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 315: `if (chunk == 0) begin` This condition selects the following hardware branch only when ‘chunk == 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are chunk: the value named ’chunk’ in this block. Context: the simulation task task.

Line 316: `wait_suppressed_done();` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are wait_suppressed_done: the value named ’wait suppressed done’ in this block.

Line 317: `end else begin` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 318: `for (beat_index = 0; beat_index < 20;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 319: `beat_index = beat_index + 1) begin` This statement continues through source line 320; the explanation uses the complete expression rather than only the first visual fragment. This makes beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘beat_index + 1) begin receive_word(received_word, received_last, beat_index % 3)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry; receive_word: the value named ’receive word’ in this block; received_word: the value named ’received word’ in this block. Context: the simulation task task.

Line 320: `receive_word(received_word, received_last, beat_index % 3);` This is continuation line 2 of the 2-line logical statement that begins at source line 319. Its exact fragment is ‘receive_word(received_word, received_last, beat_index % 3);‘. The important operands are receive_word: the value named ’receive word’ in this block; received_word: the value named ’received word’ in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This makes beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘beat_index + 1) begin receive_word(received_word, received_last,

beat_index % 3)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry; receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block. Context: the simulation task task.

Line 321: `row = beat_index / 4;` This makes row, one output-row position processed by the tensor tile, equal to ‘beat_index / 4‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 322: `word_index = beat_index % 4;` This makes word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘beat_index % 4‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 323: `for (lane = 0; lane < 2; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 324: `input_index = word_index * 2 + lane;` This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘word_index * 2 + lane‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are word_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task task.

Line 325: `if ($signed(received_word[lane*32 +: 32])` This condition selects the following hardware branch only when ‘$signed(received_word[lane*32 +: 32]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_word: the value named 'received word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task task.

Line 326: `!== expected_input_gradient[row][input_index]) begin` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are expected_input_gradient: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 327: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to ‘errors + 1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task task.

Line 328: `$display("ERROR accumulated dX row=%0d lane=%0d got=%0d expected=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 329: `row, input_index,` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are row: one output-row position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 330: `$signed(received_word[lane*32 +: 32]),` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are received_word: the value named 'received word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 331: `expected_input_gradient[row][input_index]);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are expected_input_gradient: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 332: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 333: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 334: `if (received_last != (beat_index == 19)) errors = errors + 1;` This condition selects the following hardware branch only when ‘received_last != (beat_index == 19)‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 335: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 336: `timeout = 0;` This makes timeout, the value named ’timeout’ in this block, equal to ‘0‘. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task task.

Line 337: `while (!done && timeout < 20) begin` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named ’while’ in this block; done: the value named ’done’ in this block; timeout: the value named ’timeout’ in this block.

Line 338: `@(negedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 339: `timeout = timeout + 1;` This makes timeout, the value named ’timeout’ in this block, equal to ‘timeout + 1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are timeout: the value named ’timeout’ in this block. Context: the simulation task task.

Line 340: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 341: `if (!done || config_error) errors = errors + 1;` This condition selects the following hardware branch only when ‘!done || config_error‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are done: the value named ’done’ in this block; config_error: the value named ’config error’ in this block. Context: the simulation task task.

Line 342: `checked_tiles = checked_tiles + 1;` This makes checked_tiles, the value named ’checked tiles’ in this block, equal to ‘checked_tiles + 1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are checked_tiles: the value named ’checked tiles’ in this block. Context: the simulation task task.

Line 343: `@(posedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 344: `@(negedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 345: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 346: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 348: `// dW is a reduction over rows. Thirteen rows are split 8+5; the` Design intent: dW is a reduction over rows. Thirteen rows are split 8+5; the This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Line 349: `// dW-only mode consequently omits every weight input beat.` Design intent: dW-only mode consequently omits every weight input beat. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Line 350: `for (input_index = 0; input_index < LANES;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 351: `input_index = input_index + 1) begin` This statement continues through source line 352; the explanation uses the complete expression rather than only the first visual fragment. This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘input_index + 1) begin for (output_index = 0; output_index < LANES‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block; output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 352: `for (output_index = 0; output_index < LANES;` This is continuation line 2 of the 2-line logical statement that begins at source line 351. Its exact fragment is ‘for (output_index = 0; output_index < LANES;‘. The important operands are for: the value named 'for' in this block; output_index: an integer selecting one tensor element, lane, bank, or state entry; LANES: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘input_index + 1) begin for (output_index = 0; output_index < LANES‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named 'for' in this block; output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 353: `output_index = output_index + 1) begin` This statement continues through source line 354; the explanation uses the complete expression rather than only the first visual fragment. This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index + 1) begin expected_weight_gradient[input_index][output = 24’sd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry; expected_weight_gradient: a learned coefficient multiplied by an activation; input_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 354: `expected_weight_gradient[input_index][output_index] = 24’sd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 353. Its exact fragment is ‘expected_weight_gradient[input_index][output_index] = 24’sd0;‘. The important operands are expected_weight_gradient: a learned coefficient multiplied by an activation; input_index: an integer selecting one tensor element, lane, bank, or state entry; output_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index + 1) begin expected_weight_gradient[input_index][output_index] = 24’sd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index: an integer selecting one tensor element, lane, bank,

or state entry; expected_weight_gradient: a learned coefficient multiplied by an activation; input_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 355: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 356: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 357: `for (input_index = 0; input_index < 6;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 358: `input_index = input_index + 1) begin` This statement continues through source line 359; the explanation uses the complete expression rather than only the first visual fragment. This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘input_index + 1) begin for (output_index = 0; output_index < 7‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block; output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 359: `for (output_index = 0; output_index < 7;` This is continuation line 2 of the 2-line logical statement that begins at source line 358. Its exact fragment is ‘for (output_index = 0; output_index < 7;‘. The important operands are for: the value named ’for’ in this block; output_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘input_index + 1) begin for (output_index = 0; output_index < 7‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block; output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 360: `output_index = output_index + 1) begin` This statement continues through source line 361; the explanation uses the complete expression rather than only the first visual fragment. This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index + 1) begin accumulator = 64’sd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry; accumulator: a widened running sum that prevents intermediate overflow; sd0: the value named ’sd0’ in this block. Context: the simulation task task.

Line 361: `accumulator = 64'sd0;` This is continuation line 2 of the 2-line logical statement that begins at source line 360. Its exact fragment is ‘accumulator = 64’sd0;‘. The important operands are accumulator: a widened running sum that prevents intermediate overflow; sd0: the value named ’sd0’ in this block. Read together, the complete operation is: This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘output_index + 1) begin accumulator = 64’sd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry; accumulator: a widened running sum that prevents intermediate overflow; sd0: the value named ’sd0’ in this block. Context: the simulation task task.

Line 362: `for (row = 0; row < 13; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 363: `accumulator = accumulator` This statement continues through source line 365; the explanation uses the complete expression rather than only the first visual fragment. This makes accumulator, a widened running sum that prevents intermediate overflow, equal to ‘accumulator + ((((row*29 + input_index*13 + 7) % 255) - 127) * (((row*71 + output_index*43 + 11) % 1801) - 900))‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are accumulator: a widened running sum that prevents intermediate overflow; row: one output-row

position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 364: `+ ((((row*29 + input_index*13 + 7) % 255) - 127)` This is continuation line 2 of the 3-line logical statement that begins at source line 363. Its exact fragment is '+ ((((row*29 + input_index*13 + 7) % 255) - 127)'. The important operands are row: one output-row position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes accumulator, a widened running sum that prevents intermediate overflow, equal to 'accumulator + ((((row*29 + input_index*13 + 7) % 255) - 127) * (((row*71 + output_index*43 + 11) % 1801) - 900))'. It is driven by combinational or procedural logic in the current evaluation. The important operands are accumulator: a widened running sum that prevents intermediate overflow; row: one output-row position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 365: `* (((row*71 + output_index*43 + 11) % 1801) - 900));` This is continuation line 3 of the 3-line logical statement that begins at source line 363. Its exact fragment is '* (((row*71 + output_index*43 + 11) % 1801) - 900));'. The important operands are row: one output-row position processed by the tensor tile; output_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This makes accumulator, a widened running sum that prevents intermediate overflow, equal to 'accumulator + ((((row*29 + input_index*13 + 7) % 255) - 127) * (((row*71 + output_index*43 + 11) % 1801) - 900))'. It is driven by combinational or procedural logic in the current evaluation. The important operands are accumulator: a widened running sum that prevents intermediate overflow; row: one output-row position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 366: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 367: `expected_weight_gradient[input_index][output_index]` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are expected_weight_gradient: a learned coefficient multiplied by an activation; input_index: an integer selecting one tensor element, lane, bank, or state entry; output_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 368: `= saturate_s24_ref(round_shift_ref(accumulator, 3));` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are saturate_s24_ref: the value named 'saturate s24 ref' in this block; round_shift_ref: the value named 'round shift ref' in this block; accumulator: a widened running sum that prevents intermediate overflow.

Line 369: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 370: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 371: `for (chunk = 0; chunk < 2; chunk = chunk + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 372: `chunk_size = chunk == 0 ?  8 :  5;` This makes chunk_size, the value named 'chunk size' in this block, equal to 'chunk == 0 ? 8 : 5'. It is driven by combinational or procedural logic in the current evaluation. The important operands are chunk: the value named 'chunk' in this block. Context: the simulation task task.

Line 373: `start_partial_tile(chunk_size, 6, 7, 0, 3, 2'd2,` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are start_partial_tile: the value named 'start partial tile' in this block; chunk_size: the value named 'chunk size' in this block; d2: the value named 'd2' in this block.

Line 374: `chunk != 0, chunk == 0);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are chunk: the value named 'chunk' in this block.

Line 375: `for (row = 0; row < chunk_size; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 376: `global_index = chunk * 8 + row;` This makes global_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'chunk * 8 + row'. It is driven by combinational or procedural logic in the current evaluation. The important operands are chunk: the value named 'chunk' in this block; row: one output-row position processed by the tensor tile. Context: the simulation task task.

Line 377: `for (half = 0; half < 2; half = half + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 378: `stream_word = 64'd0;` This makes stream_word, the value named 'stream word' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task task.

Line 379: `for (lane = 0; lane < 4; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 380: `input_index = half * 4 + lane;` This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'half * 4 + lane'. It is driven by combinational or procedural logic in the current evaluation. The important operands are half: the value named 'half' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task task.

Line 381: `if (input_index < 6) begin` This condition selects the following hardware branch only when 'input_index < 6' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 382: `stream_word[lane*16 +: 16] =` This statement continues through source line 383; the explanation uses the complete expression rather than only the first visual fragment. Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; global_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 383: `((global_index*29 + input_index*13 + 7) % 255) - 127;` This is continuation line 2 of the 2-line logical statement that begins at source line 382. Its exact fragment is '((global_index*29 + input_index*13 + 7) % 255) - 127;'. The important operands are global_index: an integer selecting one tensor element, lane, bank, or state entry; input_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; global_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 384: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 385: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 386: `send_word(stream_word, 1'b0);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; stream_word: the value named 'stream word' in this block; b0: the value named 'b0' in this block.

Line 387: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 388: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 389: `for (row = 0; row < chunk_size; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 390: `global_index = chunk * 8 + row;` This makes global_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'chunk * 8 + row'. It is driven by combinational or procedural logic in the current evaluation. The important operands are chunk: the value named 'chunk' in this block; row: one output-row position processed by the tensor tile. Context: the simulation task task.

Line 391: `for (half = 0; half < 2; half = half + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 392: `stream_word = 64'd0;` This makes stream_word, the value named 'stream word' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task task.

Line 393: `for (lane = 0; lane < 4; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 394: `output_index = half * 4 + lane;` This makes output_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'half * 4 + lane'. It is driven by combinational or procedural logic in the current evaluation. The important operands are half: the value named 'half' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task task.

Line 395: `if (output_index < 7) begin` This condition selects the following hardware branch only when 'output_index < 7' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 396: `stream_word[lane*16 +: 16] =` This statement continues through source line 397; the explanation uses the complete expression rather than only the first visual fragment. Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; global_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 397: `((global_index*71 + output_index*43 + 11) % 1801) - 900;` This is continuation line 2 of the 2-line logical statement that begins at source line 396. Its exact fragment is '((global_index*71 + output_index*43 + 11) % 1801) - 900;'. The important operands are global_index: an integer selecting one tensor element, lane, bank, or state entry; output_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; global_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 398: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 399: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 400: `send_word(stream_word,` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; stream_word: the value named 'stream word' in this block.

Line 401: `row == chunk_size - 1 && half == 1);` This makes row, one output-row position processed by the tensor tile, equal to '= chunk_size - 1 && half == 1)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are chunk_size: the value named 'chunk size' in this block; half: the value named 'half' in this block. Context: the simulation task task.

Line 402: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 403: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 404: `if (chunk == 0) begin` This condition selects the following hardware branch only when 'chunk == 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are chunk: the value named 'chunk' in this block. Context: the simulation task task.

Line 405: `wait_suppressed_done();` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are wait_suppressed_done: the value named 'wait suppressed done' in this block.

Line 406: `end else begin` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 407: `packed_weight_row = 192'd0;` This makes packed_weight_row, a learned coefficient multiplied by an activation, equal to '192'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task task.

Line 408: `for (beat_index = 0; beat_index < 18;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 409: `beat_index = beat_index + 1) begin` This statement continues through source line 410; the explanation uses the complete expression rather than only the first visual fragment. This makes beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'beat_index + 1) begin receive_word(received_word, received_last, beat_index % 4)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry; receive_word: the value named

'receive word' in this block; received_word: the value named 'received word' in this block. Context: the simulation task task.

Line 410: `receive_word(received_word, received_last, beat_index % 4);` This is continuation line 2 of the 2-line logical statement that begins at source line 409. Its exact fragment is 'receive_word(received_word, received_last, beat_index % 4);'. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This makes beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'beat_index + 1) begin receive_word(received_word, received_last, beat_index % 4)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry; receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block. Context: the simulation task task.

Line 411: `input_index = beat_index / 3;` This makes input_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'beat_index / 3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 412: `word_index = beat_index % 3;` This makes word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'beat_index % 3'. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 413: `packed_weight_row[word_index*64 +: 64] = received_word;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are packed_weight_row: a learned coefficient multiplied by an activation; word_index: an integer selecting one tensor element, lane, bank, or state entry; received_word: the value named 'received word' in this block.

Line 414: `if (word_index == 2) begin` This condition selects the following hardware branch only when 'word_index == 2' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 415: `for (lane = 0; lane < LANES; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task task.

Line 416: `if ($signed(packed_weight_row[lane*24 +: 24])` This condition selects the following hardware branch only when '$signed(packed_weight_row[lane*24 +: 24]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are packed_weight_row: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task task.

Line 417: `!== expected_weight_gradient[input_index][lane]) begin` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are expected_weight_gradient: a learned coefficient multiplied by an activation; input_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 418: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task task.

Line 419: `$display("ERROR accumulated dW row=%0d lane=%0d got=%0d expected=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 420: `input_index, lane,` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are input_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 421: `$signed(packed_weight_row[lane*24 +: 24]),` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are packed_weight_row: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 422: `expected_weight_gradient[input_index][lane]);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are expected_weight_gradient: a learned coefficient multiplied by an activation; input_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 423: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 424: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 425: `packed_weight_row = 192'd0;` This makes packed_weight_row, a learned coefficient multiplied by an activation, equal to '192'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task task.

Line 426: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 427: `if (received_last != (beat_index == 17)) errors = errors + 1;` This condition selects the following hardware branch only when 'received_last != (beat_index == 17)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task task.

Line 428: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 429: `timeout = 0;` This makes timeout, the value named 'timeout' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task task.

Line 430: `while (!done && timeout < 20) begin` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; done: the value named 'done' in this block; timeout: the value named 'timeout' in this block.

Line 431: `@(negedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 432: `timeout = timeout + 1;` This makes timeout, the value named 'timeout' in this block, equal to 'timeout + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are timeout: the value named 'timeout' in this block. Context: the simulation task task.

Line 433: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 434: `if (!done || config_error) errors = errors + 1;` This condition selects the following hardware branch only when '!done || config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are done: the value named 'done' in this block; config_error: the value named 'config error' in this block. Context: the simulation task task.

Line 435: `checked_tiles = checked_tiles + 1;` This makes checked_tiles, the value named 'checked tiles' in this block, equal to 'checked_tiles + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are checked_tiles: the value named 'checked tiles' in this block. Context: the simulation task task.

Line 436: `@(posedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 437: `@(negedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 438: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 439: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 441: `// Emitting the final result invalidates the retained shape.` Design intent: Emitting the final result invalidates the retained shape. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Line 442: `start_partial_tile(5, 6, 5, 4, 0, 2'd1, 1'b1, 1'b1);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are start_partial_tile: the value named 'start partial tile' in this block; d1: the value named 'd1' in this block; b1: the value named 'b1' in this block.

Line 443: `while (!done) @(negedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; done: the value named 'done' in this block; negedge: the value named 'negedge' in this block.

Line 444: `if (!config_error) begin` This condition selects the following hardware branch only when '!config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are config_error: the value named 'config error' in this block. Context: the simulation task task.

Line 445: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task task.

Line 446: `$display("ERROR stale dX accumulator remained valid after final output");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 447: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 448: `@(posedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 449: `@(negedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 450: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 451: `endtask` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 453: `task automatic prepare_case(` Within the simulation task prepare case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; prepare_case: the value named 'prepare case' in this block.

Line 454: `input integer rows,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 455: `input integer inputs,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 456: `input integer outputs,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 457: `input integer pattern,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 458: `input integer dx_shift,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 459: `input integer dw_shift` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 460: `);` Within the simulation task prepare case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 461: `integer row;` This declares hardware signal storage or wiring in the simulation task prepare case. Specifically, row is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 462: `integer column;` This declares hardware signal storage or wiring in the simulation task prepare case. Specifically, column is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 463: `integer reduction;` This declares hardware signal storage or wiring in the simulation task prepare case. Specifically, reduction is the value named 'reduction' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 464: `reg signed [63:0] accumulator;` This declares hardware signal storage or wiring in the simulation task prepare case. Specifically, accumulator is a widened running sum that prevents intermediate overflow. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 465: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 466: `for (row = 0; row < LANES; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 467: `for (column = 0; column < LANES; column = column + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 468: `activation[row][column] = 0;` This makes activation[row][column], an input feature value consumed by a neuron or multiply-accumulate operation, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task prepare case.

Line 469: `weight[row][column] = 0;` This makes weight[row][column], a learned coefficient multiplied by an activation, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task prepare case.

Line 470: `output_gradient[row][column] = 0;` This makes output_gradient[row][column], the signed loss derivative used to update a learned parameter, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task prepare case.

Line 471: `expected_input_gradient[row][column] = 32'sd0;` This makes expected_input_gradient[row][column], the signed loss derivative used to update a learned parameter, equal to '32'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the simulation task prepare case.

Line 472: `expected_weight_gradient[row][column] = 24'sd0;` This makes expected_weight_gradient[row][column], a learned coefficient multiplied by an activation, equal to '24'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the simulation task prepare case.

Line 473: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 474: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 476: `for (row = 0; row < rows; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 477: `for (column = 0; column < inputs; column = column + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 478: `if (pattern == 1) activation[row][column] = 127;` This condition selects the following hardware branch only when ‘pattern == 1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pattern: the value named 'pattern' in this block. Context: the simulation task prepare case.

Line 479: `else if (pattern == 2) activation[row][column] = -128;` This condition selects the following hardware branch only when ‘pattern == 2‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pattern: the value named 'pattern' in this block. Context: the simulation task prepare case.

Line 480: `else if (pattern == 3) activation[row][column] = 127;` This condition selects the following hardware branch only when ‘pattern == 3‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pattern: the value named 'pattern' in this block. Context: the simulation task prepare case.

Line 481: `else activation[row][column] = $urandom_range(255, 0) - 128;` Within the simulation task prepare case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are activation: an input feature value consumed by a neuron or multiply-accumulate operation; row: one output-row position processed by the tensor tile; column: one output-feature position processed in parallel.

Line 482: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 483: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 484: `for (row = 0; row < inputs; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 485: `for (column = 0; column < outputs; column = column + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 486: `if (pattern == 1) weight[row][column] = -32768;` This condition selects the following hardware branch only when ‘pattern == 1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pattern: the value named 'pattern' in this block. Context: the simulation task prepare case.

Line 487: `else if (pattern == 2) weight[row][column] = 32767;` This condition selects the following hardware branch only when ‘pattern == 2‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pattern: the value named ’pattern’ in this block. Context: the simulation task prepare case.

Line 488: `else if (pattern == 3) weight[row][column] = 32767;` This condition selects the following hardware branch only when ‘pattern == 3‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pattern: the value named ’pattern’ in this block. Context: the simulation task prepare case.

Line 489: `else weight[row][column] = $urandom_range(65535, 0) - 32768;` Within the simulation task prepare case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are weight: a learned coefficient multiplied by an activation; row: one output-row position processed by the tensor tile; column: one output-feature position processed in parallel.

Line 490: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 491: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 492: `for (row = 0; row < rows; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 493: `for (column = 0; column < outputs; column = column + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 494: `if (pattern == 1) output_gradient[row][column] = -32768;` This condition selects the following hardware branch only when ‘pattern == 1‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pattern: the value named ’pattern’ in this block. Context: the simulation task prepare case.

Line 495: `else if (pattern == 2) output_gradient[row][column] = 32767;` This condition selects the following hardware branch only when ‘pattern == 2‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pattern: the value named ’pattern’ in this block. Context: the simulation task prepare case.

Line 496: `else if (pattern == 3) output_gradient[row][column] = -32768;` This condition selects the following hardware branch only when ‘pattern == 3‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are pattern: the value named ’pattern’ in this block. Context: the simulation task prepare case.

Line 497: `else output_gradient[row][column] = $urandom_range(65535, 0) - 32768;` Within the simulation task prepare case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile; column: one output-feature position processed in parallel.

Line 498: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 499: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 501: `for (row = 0; row < rows; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 502: `for (column = 0; column < inputs; column = column + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 503: `accumulator = 64'sd0;` This makes accumulator, a widened running sum that prevents intermediate overflow, equal to '64'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the simulation task prepare case.

Line 504: `for (reduction = 0; reduction < outputs; reduction = reduction + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 505: `accumulator = accumulator` This statement continues through source line 506; the explanation uses the complete expression rather than only the first visual fragment. This makes accumulator, a widened running sum that prevents intermediate overflow, equal to 'accumulator + output_gradient[row][reduction] * weight[column][reduction]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are accumulator: a widened running sum that prevents intermediate overflow; output_gradient: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile. Context: the simulation task prepare case.

Line 506: `+ output_gradient[row][reduction] * weight[column][reduction];` This is continuation line 2 of the 2-line logical statement that begins at source line 505. Its exact fragment is '+ output_gradient[row][reduction] * weight[column][reduction];'. The important operands are output_gradient: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile; reduction: the value named 'reduction' in this block. Read together, the complete operation is: This makes accumulator, a widened running sum that prevents intermediate overflow, equal to 'accumulator + output_gradient[row][reduction] * weight[column][reduction]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are accumulator: a widened running sum that prevents intermediate overflow; output_gradient: the signed loss derivative used to update a learned parameter; row: one output-row position processed by the tensor tile. Context: the simulation task prepare case.

Line 507: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 508: `expected_input_gradient[row][column] = saturate_s32_ref(` This statement continues through source line 509; the explanation uses the complete expression rather than only the first visual fragment. This makes expected_input_gradient[row][column], the signed loss derivative used to update a learned parameter, equal to 'saturate_s32_ref(round_shift_ref(accumulator, dx_shift))'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_s32_ref: the value named 'saturate s32 ref' in this block; round_shift_ref: the value named 'round shift ref' in this block; accumulator: a widened running sum that prevents intermediate overflow. Context: the simulation task prepare case.

Line 509: `round_shift_ref(accumulator, dx_shift));` This is continuation line 2 of the 2-line logical statement that begins at source line 508. Its exact fragment is 'round_shift_ref(accumulator, dx_shift));'. The important operands are round_shift_ref: the value named 'round shift ref' in this block; accumulator: a widened running sum that prevents intermediate overflow; dx_shift: the value named 'dx shift' in this block. Read together, the complete operation is: This makes expected_input_gradient[row][column], the signed loss derivative used to update a learned parameter, equal to 'saturate_s32_ref(round_shift_ref(accumulator, dx_shift))'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_s32_ref: the value named 'saturate s32 ref' in this block;

round_shift_ref: the value named 'round shift ref' in this block; accumulator: a widened running sum that prevents intermediate overflow. Context: the simulation task prepare case.

Line 510: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 511: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 512: `for (row = 0; row < inputs; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 513: `for (column = 0; column < outputs; column = column + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 514: `accumulator = 64'sd0;` This makes accumulator, a widened running sum that prevents intermediate overflow, equal to '64'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the simulation task prepare case.

Line 515: `for (reduction = 0; reduction < rows; reduction = reduction + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task prepare case.

Line 516: `accumulator = accumulator` This statement continues through source line 517; the explanation uses the complete expression rather than only the first visual fragment. This makes accumulator, a widened running sum that prevents intermediate overflow, equal to 'accumulator + activation[reduction][row] * output_gradient[reduction][column]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are accumulator: a widened running sum that prevents intermediate overflow; activation: an input feature value consumed by a neuron or multiply-accumulate operation; reduction: the value named 'reduction' in this block. Context: the simulation task prepare case.

Line 517: `+ activation[reduction][row] * output_gradient[reduction][column];` This is continuation line 2 of the 2-line logical statement that begins at source line 516. Its exact fragment is '+ activation[reduction][row] * output_gradient[reduction][column];'. The important operands are activation: an input feature value consumed by a neuron or multiply-accumulate operation; reduction: the value named 'reduction' in this block; row: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes accumulator, a widened running sum that prevents intermediate overflow, equal to 'accumulator + activation[reduction][row] * output_gradient[reduction][column]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are accumulator: a widened running sum that prevents intermediate overflow; activation: an input feature value consumed by a neuron or multiply-accumulate operation; reduction: the value named 'reduction' in this block. Context: the simulation task prepare case.

Line 518: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 519: `expected_weight_gradient[row][column] = saturate_s24_ref(` This statement continues through source line 520; the explanation uses the complete expression rather than only the first visual fragment. This makes expected_weight_gradient[row][column], a learned coefficient multiplied by an activation, equal to 'saturate_s24_ref(round_shift_ref(accumulator, dw_shift))'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_s24_ref: the value named 'saturate s24 ref' in this block; round_shift_ref: the value named 'round shift ref' in this block; accumulator: a widened running sum that prevents intermediate overflow. Context: the simulation task prepare case.

Line 520: `round_shift_ref(accumulator, dw_shift));` This is continuation line 2 of the 2-line logical statement that begins at source line 519. Its exact fragment is 'round_shift_ref(accumulator, dw_shift));'. The important operands are round_shift_ref: the value named 'round shift ref' in this block; accumulator: a widened running sum that prevents intermediate overflow; dw_shift: the value named 'dw shift' in this block. Read together, the complete operation is: This makes expected_weight_gradient[row][column], a learned coefficient multiplied by an activation, equal to 'saturate_s24_ref(round_shift_ref(accumulator, dw_shift))'. It is driven by combinational or procedural logic in the current evaluation. The important operands are saturate_s24_ref: the value named 'saturate s24 ref' in this block; round_shift_ref: the value named 'round shift ref' in this block; accumulator: a widened running sum that prevents intermediate overflow. Context: the simulation task prepare case.

Line 521: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 522: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 523: `end` This structural keyword opens or closes the branch/process scope described as the simulation task prepare case; it has no arithmetic value of its own.

Line 524: `endtask` Within the simulation task prepare case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 526: `task automatic run_case(` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; run_case: the value named 'run case' in this block.

Line 527: `input integer rows,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 528: `input integer inputs,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 529: `input integer outputs,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 530: `input integer pattern,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 531: `input integer dx_shift,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 532: `input integer dw_shift,` This declares integer as a input port with width one bit. It carries the value named 'integer' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 533: `input logic malformed_tlast` This declares malformed_tlast as a input port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 534: `);` Within the simulation task run case, this exact statement contributes to the block that drives, back-pressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 535: `integer row;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, row is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 536: `integer lane;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, lane is one parallel arithmetic element within a vector or tensor datapath. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 537: `integer half;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, half is the value named 'half' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 538: `integer beat_index;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, beat_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 539: `integer row_index;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, row_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 540: `integer word_index;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, word_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 541: `integer lane_index;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, lane_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 542: `integer total_output_beats;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, total_output_beats is the value named 'total output beats' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 543: `integer timeout;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, timeout is the value named 'timeout' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 544: `logic [63:0] stream_word;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, stream_word is the value named 'stream word' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 545: `logic [63:0] received_word;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, received_word is the value named 'received word' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 546: `logic received_last;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, received_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 547: `logic [191:0] packed_weight_row;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, packed_weight_row is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 548: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 549: `prepare_case(rows, inputs, outputs, pattern, dx_shift, dw_shift);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are prepare_case: the value named 'prepare case' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 550: `start_tile(rows, inputs, outputs, dx_shift, dw_shift, 1'b0);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are start_tile: the value named 'start tile' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 552: `for (row = 0; row < rows; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 553: `for (half = 0; half < 2; half = half + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 554: `stream_word = 64'd0;` This makes stream_word, the value named 'stream word' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task run case.

Line 555: `for (lane = 0; lane < 4; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 556: `stream_word[lane*16 +: 16] = activation[row][half*4 + lane];` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; activation: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 557: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 558: `send_word(stream_word,` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; stream_word: the value named 'stream word' in this block.

Line 559: `malformed_tlast && row == 0 && half == 0);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are malformed_tlast: the AXI-Stream marker for the final beat of a packet or command payload; row: one output-row position processed by the tensor tile; half: the value named 'half' in this block.

Line 560: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 561: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 562: `for (row = 0; row < inputs; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 563: `for (half = 0; half < 2; half = half + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 564: `stream_word = 64'd0;` This makes stream_word, the value named 'stream word' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task run case.

Line 565: `for (lane = 0; lane < 4; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 566: `stream_word[lane*16 +: 16] = weight[row][half*4 + lane];` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; weight: a learned coefficient multiplied by an activation.

Line 567: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 568: `send_word(stream_word, 1'b0);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; stream_word: the value named 'stream word' in this block; b0: the value named 'b0' in this block.

Line 569: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 570: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 571: `for (row = 0; row < rows; row = row + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 572: `for (half = 0; half < 2; half = half + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 573: `stream_word = 64'd0;` This makes stream_word, the value named 'stream word' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task run case.

Line 574: `for (lane = 0; lane < 4; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 575: `stream_word[lane*16 +:  16] = output_gradient[row][half*4 + lane];` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are stream_word: the value named 'stream word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; output_gradient: the signed loss derivative used to update a learned parameter.

Line 576: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 577: `send_word(stream_word, row == rows - 1 && half == 1);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; stream_word: the value named 'stream word' in this block; row: one output-row position processed by the tensor tile.

Line 578: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 579: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 581: `total_output_beats = 4 * rows + 3 * inputs;` This makes total_output_beats, the value named 'total output beats' in this block, equal to '4 * rows + 3 * inputs'. It is driven by combinational or procedural logic in the current evaluation. The important operands are rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block. Context: the simulation task run case.

Line 582: `packed_weight_row = 192'd0;` This makes packed_weight_row, a learned coefficient multiplied by an activation, equal to '192'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task run case.

Line 583: `for (beat_index = 0; beat_index < total_output_beats;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 584: `beat_index = beat_index + 1) begin` This statement continues through source line 585; the explanation uses the complete expression rather than only the first visual fragment. This makes beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'beat_index + 1) begin receive_word(received_word, received_last, beat_index % 4)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry; receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block. Context: the simulation task run case.

Line 585: `receive_word(received_word, received_last, beat_index % 4);` This is continuation line 2 of the 2-line logical statement that begins at source line 584. Its exact fragment is 'receive_word(received_word, received_last, beat_index % 4);'. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload. Read together, the complete operation is: This makes beat_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'beat_index + 1) begin receive_word(received_word, received_last, beat_index % 4)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry; receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block. Context: the simulation task run case.

Line 586: `if (received_last != (beat_index == total_output_beats - 1)) begin` This condition selects the following hardware branch only when 'received_last != (beat_index == total_output_beats - 1)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; beat_index: an integer selecting one tensor element, lane, bank, or state entry; total_output_beats: the value named 'total output beats' in this block. Context: the simulation task run case.

Line 587: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task run case.

Line 588: `$display("ERROR backward TLAST case=%0d beat=%0d got=%0b",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 589: `checked_tiles, beat_index, received_last);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are checked_tiles: the value named 'checked tiles' in this block; beat_index: an integer selecting one tensor element, lane, bank, or state entry; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 590: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 591: `if (beat_index < 4 * rows) begin` This condition selects the following hardware branch only when 'beat_index $<$ 4 * rows' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry; rows: one output-row position processed by the tensor tile. Context: the simulation task run case.

Line 592: `row_index = beat_index / 4;` This makes row_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'beat_index / 4'. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run case.

Line 593: `word_index = beat_index % 4;` This makes word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'beat_index % 4'. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run case.

Line 594: `for (lane = 0; lane < 2; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 595: `lane_index = word_index * 2 + lane;` This makes lane_index, an integer selecting one tensor element, lane, bank, or state entry, equal to 'word_index * 2 + lane'. It is driven by combinational or procedural logic in the current evaluation. The important operands are word_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task run case.

Line 596: `if ($signed(received_word[lane*32 +: 32])` This condition selects the following hardware branch only when '$signed(received_word[lane*32 +: 32]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_word: the value named 'received word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task run case.

Line 597: `!== expected_input_gradient[row_index][lane_index]) begin` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are expected_input_gradient: the signed loss derivative used to update a learned parameter; row_index: an integer selecting one tensor element, lane, bank, or state entry; lane_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 598: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task run case.

Line 599: `if (errors <= 30) begin` This statement continues through source line 603; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dX tile=%0d row=%0d lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane_index, $signed(received_word[lane*32 +: 32]), expected_input_gradient[row_index][lane_index]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 600: `$display("ERROR dX tile=%0d row=%0d lane=%0d got=%0d expected=%0d",` This is continuation line 2 of the 5-line logical statement that begins at source line 599. Its exact fragment is '$display("ERROR dX tile=%0d row=%0d lane=%0d got=%0d expected=%0d",'. The important operands are display: the value named 'display' in this block; ERROR: the value named 'error' in this block; dX: the value named 'd x' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dX tile=%0d row=%0d lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane_index, $signed(received_word[lane*32 +: 32]), expected_input_gradient[row_index][lane_index]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 601: `checked_tiles, row_index, lane_index,` This is continuation line 3 of the 5-line logical statement that begins at source line 599. Its exact fragment is 'checked_tiles, row_index, lane_index,'. The important operands are checked_tiles: the value named 'checked tiles' in this block; row_index: an integer selecting one tensor element, lane, bank, or state entry; lane_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dX tile=%0d row=%0d lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane_index, $signed(received_word[lane*32 +: 32]), expected_input_gradient[row_index][lane_index]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 602: `$signed(received_word[lane*32 +: 32]),` This is continuation line 4 of the 5-line logical statement that begins at source line 599. Its exact fragment is '$signed(received_word[lane*32 +: 32]),'. The important operands are received_word: the value named 'received word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dX tile=%0d row=%0d lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane_index, $signed(received_word[lane*32 +: 32]), expected_input_gradient[row_index][lane_index]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 603: `expected_input_gradient[row_index][lane_index]);` This is continuation line 5 of the 5-line logical statement that begins at source line 599. Its exact fragment is 'expected_input_gradient[row_index][lane_index]);'. The important operands are expected_input_gradient: the signed loss derivative used to update a learned parameter; row_index: an integer selecting one tensor element, lane, bank, or state entry; lane_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dX tile=%0d row=%0d

lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane_index, $signed(received_word[lane*32 +: 32]), expected_input_gradient[row_index][lane_index]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 604: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 605: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 606: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 607: `end else begin` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 608: `row_index = (beat_index - 4 * rows) / 3;` This makes row_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘(beat_index - 4 * rows) / 3‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry; rows: one output-row position processed by the tensor tile. Context: the simulation task run case.

Line 609: `word_index = (beat_index - 4 * rows) % 3;` This makes word_index, an integer selecting one tensor element, lane, bank, or state entry, equal to ‘(beat_index - 4 * rows) % 3‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat_index: an integer selecting one tensor element, lane, bank, or state entry; rows: one output-row position processed by the tensor tile. Context: the simulation task run case.

Line 610: `packed_weight_row[word_index*64 +: 64] = received_word;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are packed_weight_row: a learned coefficient multiplied by an activation; word_index: an integer selecting one tensor element, lane, bank, or state entry; received_word: the value named 'received word' in this block.

Line 611: `if (word_index == 2) begin` This condition selects the following hardware branch only when ‘word_index == 2‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are word_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run case.

Line 612: `for (lane = 0; lane < LANES; lane = lane + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 613: `if ($signed(packed_weight_row[lane*24 +: 24])` This condition selects the following hardware branch only when ‘$signed(packed_weight_row[lane*24 +: 24]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are packed_weight_row: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task run case.

Line 614: `!== expected_weight_gradient[row_index][lane]) begin` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are expected_weight_gradient: a learned coefficient multiplied by an activation; row_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 615: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task run case.

Line 616: `if (errors <= 30) begin` This statement continues through source line 620; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dW tile=%0d row=%0d lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane, $signed(packed_weight_row[lane*24 +: 24]), expected_weight_gradient[row_index][lane]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 617: `$display("ERROR dW tile=%0d row=%0d lane=%0d got=%0d expected=%0d",` This is continuation line 2 of the 5-line logical statement that begins at source line 616. Its exact fragment is '$display("ERROR dW tile=%0d row=%0d lane=%0d got=%0d expected=%0d",'. The important operands are display: the value named 'display' in this block; ERROR: the value named 'error' in this block; dW: the value named 'd w' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dW tile=%0d row=%0d lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane, $signed(packed_weight_row[lane*24 +: 24]), expected_weight_gradient[row_index][lane]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 618: `checked_tiles, row_index, lane,` This is continuation line 3 of the 5-line logical statement that begins at source line 616. Its exact fragment is 'checked_tiles, row_index, lane,'. The important operands are checked_tiles: the value named 'checked tiles' in this block; row_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dW tile=%0d row=%0d lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane, $signed(packed_weight_row[lane*24 +: 24]), expected_weight_gradient[row_index][lane]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 619: `$signed(packed_weight_row[lane*24 +: 24]),` This is continuation line 4 of the 5-line logical statement that begins at source line 616. Its exact fragment is '$signed(packed_weight_row[lane*24 +: 24]),'. The important operands are packed_weight_row: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dW tile=%0d row=%0d lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane, $signed(packed_weight_row[lane*24 +: 24]), expected_weight_gradient[row_index is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 620: `expected_weight_gradient[row_index][lane]);` This is continuation line 5 of the 5-line logical statement that begins at source line 616. Its exact fragment is 'expected_weight_gradient[row_index][lane]);'. The important operands are expected_weight_gradient: a learned coefficient multiplied by an activation; row_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR dW tile=%0d row=%0d lane=%0d got=%0d expected=%0d", checked_tiles, row_index, lane, $signed(packed_weight_row[lane*24 +: 24]), expected_weight_gradient[row_index][lane]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 621: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 622: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 623: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 624: `packed_weight_row = 192'd0;` This makes packed_weight_row, a learned coefficient multiplied by an activation, equal to '192'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task run case.

Line 625: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 626: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 627: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 629: `timeout = 0;` This makes timeout, the value named 'timeout' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 630: `while (!done && timeout < 20) begin` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; done: the value named 'done' in this block; timeout: the value named 'timeout' in this block.

Line 631: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 632: `timeout = timeout + 1;` This makes timeout, the value named 'timeout' in this block, equal to 'timeout + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are timeout: the value named 'timeout' in this block. Context: the simulation task run case.

Line 633: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 634: `if (!done) begin` This condition selects the following hardware branch only when '!done' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are done: the value named 'done' in this block. Context: the simulation task run case.

Line 635: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the simulation task run case.

Line 636: `$display("ERROR backward done timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 637: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 638: `if (config_error !== malformed_tlast) begin` This condition selects the following hardware branch only when ‘config_error !== malformed_tlast‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are config_error: the value named ’config error’ in this block; malformed_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the simulation task run case.

Line 639: `errors = errors + 1;` This makes errors, the value named ’errors’ in this block, equal to ‘errors + 1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named ’errors’ in this block. Context: the simulation task run case.

Line 640: `$display("ERROR backward config_error tile=%0d got=%0b expected=%0b",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 641: `checked_tiles, config_error, malformed_tlast);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are checked_tiles: the value named ’checked tiles’ in this block; config_error: the value named ’config error’ in this block; malformed_tlast: the AXI-Stream marker for the final beat of a packet or command payload.

Line 642: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 643: `checked_tiles = checked_tiles + 1;` This makes checked_tiles, the value named ’checked tiles’ in this block, equal to ‘checked_tiles + 1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are checked_tiles: the value named ’checked tiles’ in this block. Context: the simulation task run case.

Line 644: `@(posedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 645: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 646: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 647: `endtask` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are endtask: the value named ’endtask’ in this block.

Line 649: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 650: `integer case_index;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, case_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 651: `integer rows;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rows is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 652: `integer inputs;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, inputs is the value named 'inputs' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 653: `integer outputs;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, outputs is the value named 'outputs' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 654: `integer dx_shift;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dx_shift is the value named 'dx shift' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 655: `integer dw_shift;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dw_shift is the value named 'dw shift' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 657: `seed = 32'h17bacc8d;` This makes seed, the value named 'seed' in this block, equal to '32'h17bacc8d'. It is driven by combinational or procedural logic in the current evaluation. The important operands are h17bacc8d: the value named 'h17bacc8d' in this block. Context: the module's structural declarations.

Line 658: `seed = $urandom(seed);` This makes seed, the value named 'seed' in this block, equal to '$urandom(seed)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are urandom: the value named 'urandom' in this block; seed: the value named 'seed' in this block. Context: the module's structural declarations.

Line 659: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 660: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 662: `run_case(8, 8, 8, 1, 0, 0, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are run_case: the value named 'run case' in this block; b0: the value named 'b0' in this block.

Line 663: `run_case(8, 8, 8, 2, 0, 0, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are run_case: the value named 'run case' in this block; b0: the value named 'b0' in this block.

Line 664: `run_case(8, 8, 8, 3, 3, 2, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are run_case: the value named 'run case' in this block; b0: the value named 'b0' in this block.

Line 665: `run_case(1, 1, 1, 0, 5, 7, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are run_case: the value named 'run case' in this block; b0: the value named 'b0' in this block.

Line 666: `for (case_index = 0; case_index < RANDOM_CASES; case_index = case_index + 1) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 667: `rows = $urandom_range(8, 1);` This makes rows, one output-row position processed by the tensor tile, equal to '$urandom_range(8, 1)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are urandom_range: the value named 'urandom range' in this block. Context: the module's structural declarations.

Line 668: `inputs = $urandom_range(8, 1);` This makes inputs, the value named 'inputs' in this block, equal to '$urandom_range(8, 1)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are urandom_range: the value named 'urandom range' in this block. Context: the module's structural declarations.

Line 669: `outputs = $urandom_range(8, 1);` This makes outputs, the value named 'outputs' in this block, equal to '$urandom_range(8, 1)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are urandom_range: the value named 'urandom range' in this block. Context: the module's structural declarations.

Line 670: `dx_shift = $urandom_range(12, 0);` This makes dx_shift, the value named 'dx shift' in this block, equal to '$urandom_range(12, 0)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are urandom_range: the value named 'urandom range' in this block. Context: the module's structural declarations.

Line 671: `dw_shift = $urandom_range(12, 0);` This makes dw_shift, the value named 'dw shift' in this block, equal to '$urandom_range(12, 0)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are urandom_range: the value named 'urandom range' in this block. Context: the module's structural declarations.

Line 672: `run_case(rows, inputs, outputs, 0, dx_shift, dw_shift, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are run_case: the value named 'run case' in this block; rows: one output-row position processed by the tensor tile; inputs: the value named 'inputs' in this block.

Line 673: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 675: `// A premature TLAST must be reported while arithmetic remains exact.` Design intent: A premature TLAST must be reported while arithmetic remains exact. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Line 676: `run_case(3, 5, 7, 0, 4, 3, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are run_case: the value named 'run case' in this block; b1: the value named 'b1' in this block.

Line 678: `run_exact_accumulation_modes();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are run_exact_accumulation_modes: the value named 'run exact accumulation modes' in this block.

Line 680: `// Invalid dimensions and nonzero reserved bits complete without data.` Design intent: Invalid dimensions and nonzero reserved bits complete without data. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

Line 681: `start_tile(0, 1, 1, 0, 0, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are start_tile: the value named 'start tile' in this block; b0: the value named 'b0' in this block.

Line 682: `while (!done) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; done: the value named 'done' in this block; negedge: the value named 'negedge' in this block.

Line 683: `if (!config_error) begin` This condition selects the following hardware branch only when '!config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are config_error: the value named 'config error' in this block. Context: the module's structural declarations.

Line 684: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 685: `$display("ERROR backward accepted zero rows");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 686: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 687: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 688: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 689: `start_tile(1, 1, 1, 0, 0, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are start_tile: the value named 'start tile' in this block; b1: the value named 'b1' in this block.

Line 690: `while (!done) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are while: the value named 'while' in this block; done: the value named 'done' in this block; negedge: the value named 'negedge' in this block.

Line 691: `if (!config_error) begin` This condition selects the following hardware branch only when '!config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are config_error: the value named 'config error' in this block. Context: the module's structural declarations.

Line 692: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 693: `$display("ERROR backward accepted reserved command bits");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 694: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 695: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 696: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 698: `if (completed_tiles != checked_tiles || active_cycles == 0) begin` This condition selects the following hardware branch only when 'completed_tiles != checked_tiles || active_cycles == 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are completed_tiles: the value named 'completed tiles' in this block; checked_tiles: the value named 'checked tiles' in this block; active_cycles: the value named 'active cycles' in this block. Context: the module's structural declarations.

Line 699: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 700: `$display("ERROR backward counters tiles=%0d/%0d cycles=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 701: `completed_tiles, checked_tiles, active_cycles);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are completed_tiles: the value named 'completed tiles' in this block; checked_tiles: the value named 'checked tiles' in this block; active_cycles: the value named 'active cycles' in this block.

Line 702: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 703: `clear_counters = 1'b1;` This makes clear_counters, a hardware telemetry register that accumulates measured events, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 704: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 705: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 706: `clear_counters = 1'b0;` This makes clear_counters, a hardware telemetry register that accumulates measured events, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 707: `if (completed_tiles != 0 || active_cycles != 0) begin` This condition selects the following hardware branch only when 'completed_tiles != 0 || active_cycles != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are completed_tiles: the value named 'completed tiles' in this block; active_cycles: the value named 'active cycles' in this block. Context: the module's structural declarations.

Line 708: `errors = errors + 1;` This makes errors, the value named 'errors' in this block, equal to 'errors + 1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 709: `$display("ERROR backward counters did not clear");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 710: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 712: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 713: `$display("tb_neuraedge_linear_backward8 FAILED errors=%0d tiles=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 714: `errors, checked_tiles);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block; checked_tiles: the value named 'checked tiles' in this block.

Line 715: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 716: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 717: `$display("tb_neuraedge_linear_backward8 PASSED tiles=%0d random=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 718: `checked_tiles, RANDOM_CASES);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The important operands are checked_tiles: the value named 'checked tiles' in this block; RANDOM_CASES: the value named 'random cases' in this block.

Line 719: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 720: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 722: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 723: `#10_000_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 724: `$fatal(1, "tb_neuraedge_linear_backward8 watchdog timeout state=%0d", dut.state);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 725: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 726: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 728: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge linear backward8 RTL block in simulation.

53 tb/tb_neuraedge_mac2_u8.sv

Role in the complete system: drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation.

53.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation.

Simulation-only status tb_neuraedge_mac2_u8 is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 4 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

53.2 Complete source

```
`timescale 1ns / 1ps
`default_nettype none

module tb_neuraedge_mac2_u8;
    logic clk;
    logic rst_n = 1'b0;
    logic enable = 1'b0;
    logic [7:0] activation = '0;
    logic [7:0] weight_lo = '0;
    logic [7:0] weight_hi = '0;
    logic product_valid;
    logic [15:0] product_lo;
    logic [15:0] product_hi;

    int errors = 0;
    int checks = 0;

    neuraedge_mac2_u8 dut (
        .clk(clk),
        .rst_n(rst_n),
        .enable(enable),
        .activation(activation),
        .weight_lo(weight_lo),
        .weight_hi(weight_hi),
        .product_valid(product_valid),
        .product_lo(product_lo),
        .product_hi(product_hi)
    );

    initial clk = 1'b0;
    always #5 clk = ~clk;

    task automatic apply_and_check(
        input logic [7:0] a,
        input logic [7:0] w0,
        input logic [7:0] w1
    );
        logic [15:0] expected_lo;
        logic [15:0] expected_hi;
        begin
            expected_lo = a * w0;
            expected_hi = a * w1;
            @(negedge clk);
            enable = 1'b1;
            activation = a;
            weight_lo = w0;
            weight_hi = w1;
            @(posedge clk);
            @(posedge clk);
            @(posedge clk);
            #1;
            checks++;
            if (!product_valid || product_lo !== expected_lo || product_hi !== expected_hi) begin
                errors++;
                if (errors <= 20) begin
                    $display("ERROR a=%0d w0=%0d w1=%0d got=(%0d,%0d) expected=(%0d,%0d)",
                             a, w0, w1, product_lo, product_hi, expected_lo, expected_hi);
                end
            end
        end
```

```
61      endtask
62
63      initial begin
64          repeat (3) @(negedge clk);
65          rst_n = 1'b1;
66
67          // Exhaust every activation/low-weight combination while sweeping the
68          // upper field through a deterministic permutation of all byte values.
69          for (int a = 0; a < 256; a++) begin
70              for (int w0 = 0; w0 < 256; w0++) begin
71                  apply_and_check(8'(a), 8'(w0), 8'((w0 * 73 + a * 29 + 17) & 255));
72              end
73          end
74
75          // Exhaust every activation/high-weight combination independently.
76          for (int a = 0; a < 256; a++) begin
77              for (int w1 = 0; w1 < 256; w1++) begin
78                  apply_and_check(8'(a), 8'((w1 * 151 + a * 11 + 3) & 255), 8'(w1));
79              end
80          end
81
82          @(negedge clk);
83          enable = 1'b0;
84          @(posedge clk);
85          @(posedge clk);
86          @(posedge clk);
87          #1;
88          if (product_valid) begin
89              errors++;
90              $display("ERROR product_valid did not follow enable low");
91          end
92
93          if (errors != 0) begin
94              $display("tb_neuraedge_mac2_u8 FAILED checks=%0d errors=%0d", checks, errors);
95              $fatal(1);
96          end
97          $display("tb_neuraedge_mac2_u8 PASSED checks=%0d errors=0", checks);
98          $finish;
99      end
100
101     initial begin
102         #6_000_000;
103         $display("ERROR tb_neuraedge_mac2_u8 watchdog timeout");
104         $fatal(1);
105     end
106 endmodule
107
108 `default_nettype wire
```

53.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module tb_neuraedge_mac2_u8;` This opens tb_neuraedge_mac2_u8;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation.

Line 5: `logic clk;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 6: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 7: `logic enable = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, enable is the value named 'enable' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 8: `logic [7:0] activation = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic [7:0] weight_lo = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_lo is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic [7:0] weight_hi = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_hi is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic product_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic [15:0] product_lo;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_lo is the value named 'product lo' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic [15:0] product_hi;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, product_hi is the value named 'product hi' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 16: `int checks = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are checks: the value named 'checks' in this block.

Line 18: `neuraedge_mac2_u8 dut (` This instantiates module neuraedge_mac2_u8 as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 19: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 20: `.rst_n(rst_n),` This connects the child instance's named port 'rst_n' to parent expression 'rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 21: `.enable(enable),` This connects the child instance's named port 'enable' to parent expression 'enable'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'enable' in this block.

Line 22: `.activation(activation),` This connects the child instance's named port 'activation' to parent expression 'activation'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 23: `.weight_lo(weight_lo),` This connects the child instance's named port 'weight_lo' to parent expression 'weight_lo'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 24: `.weight_hi(weight_hi),` This connects the child instance's named port 'weight_hi' to parent expression 'weight_hi'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 25: `.product_valid(product_valid),` This connects the child instance's named port 'product_valid' to parent expression 'product_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 26: `.product_lo(product_lo),` This connects the child instance's named port 'product_lo' to parent expression 'product_lo'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'product lo' in this block.

Line 27: `.product_hi(product_hi)` This connects the child instance's named port 'product_hi' to parent expression 'product_hi'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'product hi' in this block.

Line 28: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 30: `initial clk = 1'b0;` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 31: `always #5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 33: `task automatic apply_and_check(` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; apply_and_check: the value named 'apply and check' in this block.

Line 34: `input logic [7:0] a,` This declares a as a input port with width [7:0]. It carries the value named 'a' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 35: `input logic [7:0] w0,` This declares w0 as a input port with width [7:0]. It carries the value named 'w0' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 36: `input logic [7:0] w1` This declares w1 as a input port with width [7:0]. It carries the value named 'w1' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 37: `);` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 38: `logic [15:0] expected_lo;` This declares hardware signal storage or wiring in the simulation task apply and check. Specifically, expected_lo is the value named 'expected lo' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 39: `logic [15:0] expected_hi;` This declares hardware signal storage or wiring in the simulation task apply and check. Specifically, expected_hi is the value named 'expected hi' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 40: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task apply and check; it has no arithmetic value of its own.

Line 41: `expected_lo = a * w0;` This makes expected_lo, the value named 'expected lo' in this block, equal to 'a * w0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are a: the value named 'a' in this block; w0: the value named 'w0' in this block. Context: the simulation task apply and check.

Line 42: `expected_hi = a * w1;` This makes expected_hi, the value named 'expected hi' in this block, equal to 'a * w1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are a: the value named 'a' in this block; w1: the value named 'w1' in this block. Context: the simulation task apply and check.

Line 43: `@(negedge clk);` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 44: `enable = 1'b1;` This makes enable, the value named 'enable' in this block, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task apply and check.

Line 45: `activation = a;` This makes activation, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'a'. It is driven by combinational or procedural logic in the current evaluation. The important operands are a: the value named 'a' in this block. Context: the simulation task apply and check.

Line 46: `weight_lo = w0;` This makes weight_lo, a learned coefficient multiplied by an activation, equal to 'w0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are w0: the value named 'w0' in this block. Context: the simulation task apply and check.

Line 47: `weight_hi = w1;` This makes weight_hi, a learned coefficient multiplied by an activation, equal to 'w1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are w1: the value named 'w1' in this block. Context: the simulation task apply and check.

Line 48: `@(posedge clk);` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 49: `@(posedge clk);` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 50: `@(posedge clk);` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 51: `#1;` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 52: `checks++;` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are checks: the value named 'checks' in this block.

Line 53: `if (!product_valid || product_lo !== expected_lo || product_hi !== expected_hi) begin` This condition selects the following hardware branch only when ‘!product_valid || product_lo !== expected_lo || product_hi !== expected_hi‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are product_valid: the producer’s statement that its current payload is meaningful; product_lo: the value named ’product lo’ in this block; expected_lo: the value named ’expected lo’ in this block. Context: the simulation task apply and check.

Line 54: `errors++;` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 55: `if (errors <= 20) begin` This statement continues through source line 57; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR a=%0d w0=%0d w1=%0d got=(%0d,%0d) expected=(%0d,%0d)", a, w0, w1, product_lo, product_hi, expected_lo, expected_hi‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task apply and check.

Line 56: `$display("ERROR a=%0d w0=%0d w1=%0d got=(%0d,%0d) expected=(%0d,%0d)",` This is continuation line 2 of the 3-line logical statement that begins at source line 55. Its exact fragment is ‘$display("ERROR a=%0d w0=%0d w1=%0d got=(%0d,%0d) expected=(%0d,%0d)",‘. The important operands are display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block; a: the value named ’a’ in this block. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR a=%0d w0=%0d w1=%0d got=(%0d,%0d) expected=(%0d,%0d)", a, w0, w1, product_lo, product_hi, expected_lo, expected_hi‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task apply and check.

Line 57: `a, w0, w1, product_lo, product_hi, expected_lo, expected_hi);` This is continuation line 3 of the 3-line logical statement that begins at source line 55. Its exact fragment is ‘a, w0, w1, product_lo, product_hi, expected_lo, expected_hi);‘. The important operands are a: the value named ’a’ in this block; w0: the value named ’w0’ in this block; w1: the value named ’w1’ in this block. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR a=%0d w0=%0d w1=%0d got=(%0d,%0d) expected=(%0d,%0d)", a, w0, w1, product_lo, product_hi, expected_lo, expected_hi‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task apply and check.

Line 58: `end` This structural keyword opens or closes the branch/process scope described as the simulation task apply and check; it has no arithmetic value of its own.

Line 59: `end` This structural keyword opens or closes the branch/process scope described as the simulation task apply and check; it has no arithmetic value of its own.

Line 60: `end` This structural keyword opens or closes the branch/process scope described as the simulation task apply and check; it has no arithmetic value of its own.

Line 61: `endtask` Within the simulation task apply and check, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are endtask: the value named ’endtask’ in this block.

Line 63: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 64: `repeat (3) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 65: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 67: `// Exhaust every activation/low-weight combination while sweeping the` Design intent: Exhaust every activation/low-weight combination while sweeping the This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation.

Line 68: `// upper field through a deterministic permutation of all byte values.` Design intent: upper field through a deterministic permutation of all byte values. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation.

Line 69: `for (int a = 0; a < 256; a++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 70: `for (int w0 = 0; w0 < 256; w0++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 71: `apply_and_check(8'(a), 8'(w0), 8'((w0 * 73 + a * 29 + 17) & 255));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are apply_and_check: the value named 'apply and check' in this block; a: the value named 'a' in this block; w0: the value named 'w0' in this block.

Line 72: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 73: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 75: `// Exhaust every activation/high-weight combination independently.` Design intent: Exhaust every activation/high-weight combination independently. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation.

Line 76: `for (int a = 0; a < 256; a++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 77: `for (int w1 = 0; w1 < 256; w1++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 78: `apply_and_check(8'(a), 8'((w1 * 151 + a * 11 + 3) & 255), 8'(w1));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are apply_and_check: the value named 'apply and check' in this block; a: the value named 'a' in this block; w1: the value named 'w1' in this block.

Line 79: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 80: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 82: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 83: `enable = 1'b0;` This makes enable, the value named 'enable' in this block, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 84: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 85: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 86: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 87: `#1;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 88: `if (product_valid) begin` This condition selects the following hardware branch only when 'product_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are product_valid: the producer's statement that its current payload is meaningful. Context: the module's structural declarations.

Line 89: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 90: `$display("ERROR product_valid did not follow enable low");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 91: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 93: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 94: `$display("tb_neuraedge_mac2_u8 FAILED checks=%0d errors=%0d", checks, errors);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 95: `$fatal(1);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 96: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 97: `$display("tb_neuraedge_mac2_u8 PASSED checks=%0d errors=0", checks);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 98: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 99: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 101: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 102: `#6_000_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 103: `$display("ERROR tb_neuraedge_mac2_u8 watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 104: `$fatal(1);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 105: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 106: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 108: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge mac2 u8 RTL block in simulation.

54 tb/tb_neuraedge_normalization_units.sv

Role in the complete system: drives, backpressures, and checks the neuraedge normalization units RTL block in simulation.

54.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge normalization units RTL block in simulation.

Simulation-only status tb_neuraedge_normalization_units is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

54.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3  `include "tb/fixtures/operator_fixture_params.svh"
4
5  module tb_neuraedge_normalization_units;
6      localparam int LN_CASES = `NEURAEDGE_LAYERNORM_CASES;
7      localparam int LN_SAMPLES = `NEURAEDGE_LAYERNORM_SAMPLES;
8      localparam int SM_CASES = `NEURAEDGE_SOFTMAX_CASES;
9      localparam int SM_SAMPLES = `NEURAEDGE_SOFTMAX_SAMPLES;
10
11     logic clk = 1'b0;
12     logic rst_n = 1'b0;
13
14     logic ln_start_valid = 1'b0;
15     logic ln_start_ready;
16     logic [11:0] ln_feature_count = 12'd0;
17     logic [31:0] ln_epsilon = 32'd1;
18     logic ln_sample_valid = 1'b0;
19     logic ln_sample_ready;
20     logic signed [23:0] ln_sample_value = 24'sd0;
21     logic signed [23:0] ln_sample_gamma = 24'sd0;
22     logic signed [23:0] ln_sample_beta = 24'sd0;
23     logic ln_sample_last = 1'b0;
24     logic ln_result_valid;
25     logic ln_result_ready = 1'b0;
26     logic signed [31:0] ln_result_value;
27     logic [11:0] ln_result_index;
28     logic ln_result_last;
29     logic ln_config_error;
30     logic [31:0] ln_cycle_count;
31     logic signed [31:0] ln_debug_mean;
32     logic [31:0] ln_debug_variance;
33     logic [31:0] ln_debug_sigma;
34     logic [31:0] ln_debug_reciprocal;
35
36     logic sm_start_valid = 1'b0;
37     logic sm_start_ready;
38     logic [8:0] sm_item_count = 9'd0;
39     logic [4:0] sm_difference_shift = 5'd0;
40     logic sm_score_valid = 1'b0;
41     logic sm_score_ready;
42     logic signed [23:0] sm_score_value = 24'sd0;
43     logic sm_score_last = 1'b0;
44     logic sm_probability_valid;
45     logic sm_probability_ready = 1'b0;
46     logic [15:0] sm_probability;
47     logic [8:0] sm_probability_index;
48     logic sm_probability_last;
49     logic sm_config_error;
50     logic [31:0] sm_cycle_count;
51     logic signed [23:0] sm_debug_maximum;
52     logic [23:0] sm_debug_denominator;
53     logic [31:0] sm_debug_reciprocal;
54
55     logic [63:0] ln_case_mem [0:LN_CASES-1];
56     logic signed [23:0] ln_value_mem [0:LN_SAMPLES-1];
57     logic signed [23:0] ln_gamma_mem [0:LN_SAMPLES-1];
```

```
    logic signed [23:0] ln_beta_mem [0:LN_SAMPLES-1];
    logic signed [31:0] ln_expected_mem [0:LN_SAMPLES-1];
    logic [127:0] ln_debug_mem [0:LN_CASES-1];
    logic [31:0] sm_case_mem [0:SM_CASES-1];
    logic signed [23:0] sm_score_mem [0:SM_SAMPLES-1];
    logic [15:0] sm_expected_mem [0:SM_SAMPLES-1];
    logic [79:0] sm_debug_mem [0:SM_CASES-1];

    int ln_pointer = 0;
    int sm_pointer = 0;
    int errors = 0;
    int ln_checked = 0;
    int sm_checked = 0;

    neuraedge_integer_layernorm u_layernorm (
        .clk(clk), .rst_n(rst_n),
        .start_valid(ln_start_valid), .start_ready(ln_start_ready),
        .feature_count(ln_feature_count), .epsilon(ln_epsilon),
        .sample_valid(ln_sample_valid), .sample_ready(ln_sample_ready),
        .sample_value(ln_sample_value), .sample_gamma_q15(ln_sample_gamma),
        .sample_beta(ln_sample_beta), .sample_last(ln_sample_last),
        .result_valid(ln_result_valid), .result_ready(ln_result_ready),
        .result_value(ln_result_value), .result_index(ln_result_index),
        .result_last(ln_result_last), .config_error(ln_config_error),
        .cycle_count(ln_cycle_count), .debug_mean(ln_debug_mean),
        .debug_variance(ln_debug_variance), .debug_sigma(ln_debug_sigma),
        .debug_reciprocal_q31(ln_debug_reciprocal)
    );

    neuraedge_log_softmax u_softmax (
        .clk(clk), .rst_n(rst_n),
        .start_valid(sm_start_valid), .start_ready(sm_start_ready),
        .item_count(sm_item_count), .difference_shift(sm_difference_shift),
        .score_valid(sm_score_valid), .score_ready(sm_score_ready),
        .score_value(sm_score_value), .score_last(sm_score_last),
        .probability_valid(sm_probability_valid), .probability_ready(sm_probability_ready),
        .probability_q15(sm_probability), .probability_index(sm_probability_index),
        .probability_last(sm_probability_last), .config_error(sm_config_error),
        .cycle_count(sm_cycle_count), .debug_maximum_score(sm_debug_maximum),
        .debug_denominator(sm_debug_denominator),
        .debug_reciprocal_q31(sm_debug_reciprocal)
    );

    always #2.5 clk = ~clk;

    task automatic run_layernorm_case(input int case_index);
        int count;
        int epsilon_value;
        int received;
        int output_cycle;
        int first_sample;
        begin
            count = ln_case_mem[case_index][11:0];
            epsilon_value = ln_case_mem[case_index][43:12];
            first_sample = ln_pointer;
            while (!ln_start_ready) @(negedge clk);
            @(negedge clk);
            ln_feature_count = 12'(count);
            ln_epsilon = epsilon_value;
            ln_start_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            ln_start_valid = 1'b0;

            for (int sample = 0; sample < count; sample++) begin
                if (!ln_sample_ready) begin
                    errors++;
                    $display("ERROR LayerNorm sample ready case=%0d sample=%0d", case_index, sample);
                end
                ln_sample_value = ln_value_mem[first_sample + sample];
                ln_sample_gamma = ln_gamma_mem[first_sample + sample];
                ln_sample_beta = ln_beta_mem[first_sample + sample];
                ln_sample_last = sample == count - 1;
                ln_sample_valid = 1'b1;
                @(posedge clk);
                @(negedge clk);
            end
            ln_sample_valid = 1'b0;
            ln_sample_last = 1'b0;

            received = 0;
            output_cycle = 0;
            while (received < count && output_cycle < 100000) begin
                ln_result_ready = ((output_cycle + case_index) % 6 != 1);
                @(posedge clk);
                if (ln_result_valid && ln_result_ready) begin
                    if (ln_result_value !== ln_expected_mem[first_sample + received]
```

```
                            || ln_result_index !== received
                            || ln_result_last !== (received == count - 1)) begin
                        errors++;
                        if (errors <= 20) begin
                            $display("ERROR LayerNorm case=%0d index=%0d got=%0d expected=%0d rtl_index=%0d last=%0d",
                                    case_index, received, ln_result_value,
                                    ln_expected_mem[first_sample + received], ln_result_index, ln_result_last);
                        end
                    end
                    received++;
                    ln_checked++;
                end
                @(negedge clk);
                output_cycle++;
            end
            ln_result_ready = 1'b0;
            if (received != count || ln_config_error
                    || ln_debug_mean !== ln_debug_mem[case_index][31:0]
                    || ln_debug_variance !== ln_debug_mem[case_index][63:32]
                    || ln_debug_sigma !== ln_debug_mem[case_index][95:64]
                    || ln_debug_reciprocal !== ln_debug_mem[case_index][127:96]) begin
                errors++;
                $display("ERROR LayerNorm completion case=%0d received=%0d/%0d error=%0d debug=(%0d,%0d,%0d,%0d) expected=(%0d,%0d,%0d,%0d)",
                         case_index, received, count, ln_config_error,
                         ln_debug_mean, ln_debug_variance, ln_debug_sigma, ln_debug_reciprocal,
                         $signed(ln_debug_mem[case_index][31:0]), ln_debug_mem[case_index][63:32],
                         ln_debug_mem[case_index][95:64], ln_debug_mem[case_index][127:96]);
            end
            $display("layernorm_case=%0d features=%0d cycles=%0d", case_index, count, ln_cycle_count);
            ln_pointer += count;
        end
    endtask

    task automatic run_softmax_case(input int case_index);
        int count;
        int shift_value;
        int received;
        int output_cycle;
        int first_sample;
        begin
            count = sm_case_mem[case_index][8:0];
            shift_value = sm_case_mem[case_index][13:9];
            first_sample = sm_pointer;
            while (!sm_start_ready) @(negedge clk);
            @(negedge clk);
            sm_item_count = 9'(count);
            sm_difference_shift = 5'(shift_value);
            sm_start_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            sm_start_valid = 1'b0;

            for (int sample = 0; sample < count; sample++) begin
                if (!sm_score_ready) begin
                    errors++;
                    $display("ERROR softmax score ready case=%0d sample=%0d", case_index, sample);
                end
                sm_score_value = sm_score_mem[first_sample + sample];
                sm_score_last = sample == count - 1;
                sm_score_valid = 1'b1;
                @(posedge clk);
                @(negedge clk);
            end
            sm_score_valid = 1'b0;
            sm_score_last = 1'b0;

            received = 0;
            output_cycle = 0;
            while (received < count && output_cycle < 100000) begin
                sm_probability_ready = ((output_cycle + case_index) % 5 != 2);
                @(posedge clk);
                if (sm_probability_valid && sm_probability_ready) begin
                    if (sm_probability !== sm_expected_mem[first_sample + received]
                            || sm_probability_index !== received
                            || sm_probability_last !== (received == count - 1)) begin
                        errors++;
                        if (errors <= 20) begin
                            $display("ERROR softmax case=%0d index=%0d got=%0d expected=%0d",
                                    case_index, received, sm_probability,
                                    sm_expected_mem[first_sample + received]);
                        end
                    end
                    received++;
                    sm_checked++;
                end
                @(negedge clk);
                output_cycle++;
```

```
            end
            sm_probability_ready = 1'b0;
            if (received != count || sm_config_error
                    || sm_debug_maximum !== sm_debug_mem[case_index][23:0]
                    || sm_debug_denominator !== sm_debug_mem[case_index][47:24]
                    || sm_debug_reciprocal !== sm_debug_mem[case_index][79:48]) begin
                errors++;
                $display("ERROR softmax completion case=%0d received=%0d/%0d error=%0d", case_index,
                        received, count, sm_config_error);
            end
            $display("softmax_case=%0d items=%0d cycles=%0d", case_index, count, sm_cycle_count);
            sm_pointer += count;
        end
    endtask

    initial begin
        $readmemh("tb/fixtures/layernorm_cases.mem", ln_case_mem);
        $readmemh("tb/fixtures/layernorm_values.mem", ln_value_mem);
        $readmemh("tb/fixtures/layernorm_gammas.mem", ln_gamma_mem);
        $readmemh("tb/fixtures/layernorm_betas.mem", ln_beta_mem);
        $readmemh("tb/fixtures/layernorm_expected.mem", ln_expected_mem);
        $readmemh("tb/fixtures/layernorm_debug.mem", ln_debug_mem);
        $readmemh("tb/fixtures/softmax_cases.mem", sm_case_mem);
        $readmemh("tb/fixtures/softmax_scores.mem", sm_score_mem);
        $readmemh("tb/fixtures/softmax_expected.mem", sm_expected_mem);
        $readmemh("tb/fixtures/softmax_debug.mem", sm_debug_mem);

        repeat (5) @(negedge clk);
        rst_n = 1'b1;

        for (int case_index = 0; case_index < LN_CASES; case_index++) begin
            run_layernorm_case(case_index);
        end
        for (int case_index = 0; case_index < SM_CASES; case_index++) begin
            run_softmax_case(case_index);
        end

        if (ln_pointer != LN_SAMPLES || sm_pointer != SM_SAMPLES
                || ln_checked != LN_SAMPLES || sm_checked != SM_SAMPLES) begin
            errors++;
            $display("ERROR normalization accounting ln=%0d/%0d checked=%0d sm=%0d/%0d checked=%0d",
                    ln_pointer, LN_SAMPLES, ln_checked, sm_pointer, SM_SAMPLES, sm_checked);
        end
        if (errors != 0) begin
            $display("tb_neuraedge_normalization_units FAILED errors=%0d", errors);
            $finish;
        end else begin
            $display("tb_neuraedge_normalization_units PASSED layernorm=%0d softmax=%0d",
                    ln_checked, sm_checked);
            $finish;
        end
    end

    initial begin
        #2_000_000;
        $fatal(1, "tb_neuraedge_normalization_units watchdog timeout");
    end
endmodule

`default_nettype wire
```

54.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 3: `` `include "tb/fixtures/operator_fixture_params.svh" `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation.

Line 5: `module tb_neuraedge_normalization_units;` This opens tb_neuraedge_normalization_units;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation.

Line 6: `localparam int LN_CASES = ‘NEURAEDGE_LAYERNORM_CASES;` This fixes LN_CASES to ‘NEURAEDGE_LAYERNOR during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LN_CASES denotes the value named ’ln cases’ in this block.

Line 7: `localparam int LN_SAMPLES = ‘NEURAEDGE_LAYERNORM_SAMPLES;` This fixes LN_SAMPLES to ‘NEURAEDGE_LAY during elaboration. Vivado uses that constant to shape or simplify the generated circuit; LN_SAMPLES denotes the value named ’ln samples’ in this block.

Line 8: `localparam int SM_CASES = ‘NEURAEDGE_SOFTMAX_CASES;` This fixes SM_CASES to ‘NEURAEDGE_SOFTMAX_C during elaboration. Vivado uses that constant to shape or simplify the generated circuit; SM_CASES denotes the value named ’sm cases’ in this block.

Line 9: `localparam int SM_SAMPLES = ‘NEURAEDGE_SOFTMAX_SAMPLES;` This fixes SM_SAMPLES to ‘NEURAEDGE_SOFTM during elaboration. Vivado uses that constant to shape or simplify the generated circuit; SM_SAMPLES denotes the value named ’sm samples’ in this block.

Line 11: `logic clk = 1’b0;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic rst_n = 1’b0;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic ln_start_valid = 1’b0;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, ln_start_valid is the producer’s statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic ln_start_ready;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, ln_start_ready is the consumer’s statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic [11:0] ln_feature_count = 12’d0;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, ln_feature_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [31:0] ln_epsilon = 32’d1;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, ln_epsilon is the value named ’ln epsilon’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic ln_sample_valid = 1’b0;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, ln_sample_valid is the producer’s statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic ln_sample_ready;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, ln_sample_ready is the consumer’s statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic signed [23:0] ln_sample_value = 24’sd0;` This declares hardware signal storage or wiring in the module’s structural declarations. Specifically, ln_sample_value is the value named ’ln sample value’ in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic signed [23:0] ln_sample_gamma = 24'sd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_gamma is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic signed [23:0] ln_sample_beta = 24'sd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_beta is the learned additive LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic ln_sample_last = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_sample_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic ln_result_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic ln_result_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `logic signed [31:0] ln_result_value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_value is the value named 'ln result value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic [11:0] ln_result_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 28: `logic ln_result_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_result_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `logic ln_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_config_error is the value named 'ln config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic [31:0] ln_cycle_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_cycle_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `logic signed [31:0] ln_debug_mean;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_debug_mean is the arithmetic average used by normalization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 32: `logic [31:0] ln_debug_variance;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_debug_variance is the average squared distance from the mean. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `logic [31:0] ln_debug_sigma;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_debug_sigma is the value named 'ln debug sigma' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic [31:0] ln_debug_reciprocal;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_debug_reciprocal is the value named 'ln debug reciprocal' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `logic sm_start_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_start_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 37: `logic sm_start_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_start_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 38: `logic [8:0] sm_item_count = 9'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_item_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 39: `logic [4:0] sm_difference_shift = 5'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_difference_shift is the value named 'sm difference shift' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 40: `logic sm_score_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_score_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 41: `logic sm_score_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_score_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 42: `logic signed [23:0] sm_score_value = 24'sd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_score_value is the value named 'sm score value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 43: `logic sm_score_last = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_score_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 44: `logic sm_probability_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_probability_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 45: `logic sm_probability_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_probability_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 46: `logic [15:0] sm_probability;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_probability is the value named 'sm probability' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 47: `logic [8:0] sm_probability_index;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_probability_index is an integer selecting one tensor element, lane, bank, or state entry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 48: `logic sm_probability_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_probability_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 49: `logic sm_config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_config_error is the value named 'sm config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 50: `logic [31:0] sm_cycle_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_cycle_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 51: `logic signed [23:0] sm_debug_maximum;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_debug_maximum is the value named 'sm debug maximum' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 52: `logic [23:0] sm_debug_denominator;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_debug_denominator is the value named 'sm debug denominator' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 53: `logic [31:0] sm_debug_reciprocal;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_debug_reciprocal is the value named 'sm debug reciprocal' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 55: `logic [63:0] ln_case_mem [0:LN_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_case_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 56: `logic signed [23:0] ln_value_mem [0:LN_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_value_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 57: `logic signed [23:0] ln_gamma_mem [0:LN_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_gamma_mem is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 58: `logic signed [23:0] ln_beta_mem [0:LN_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_beta_mem is the learned additive LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 59: `logic signed [31:0] ln_expected_mem [0:LN_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_expected_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 60: `logic [127:0] ln_debug_mem [0:LN_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_debug_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 61: `logic [31:0] sm_case_mem [0:SM_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_case_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 62: `logic signed [23:0] sm_score_mem [0:SM_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_score_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 63: `logic [15:0] sm_expected_mem [0:SM_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_expected_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 64: `logic [79:0] sm_debug_mem [0:SM_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_debug_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 66: `int ln_pointer = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_pointer: the value named 'ln pointer' in this block.

Line 67: `int sm_pointer = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_pointer: the value named 'sm pointer' in this block.

Line 68: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 69: `int ln_checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_checked: the value named 'ln checked' in this block.

Line 70: `int sm_checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_checked: the value named 'sm checked' in this block.

Line 72: `neuraedge_integer_layernorm u_layernorm (` This instantiates module neuraedge_integer_layernorm as u_layernorm. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 73: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 74: `.start_valid(ln_start_valid), .start_ready(ln_start_ready),` This connects the child instance's named port 'start_valid' to parent expression 'ln_start_valid), .start_ready(ln_start_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 75: `.feature_count(ln_feature_count), .epsilon(ln_epsilon),` This connects the child instance's named port 'feature_count' to parent expression 'ln_feature_count), .epsilon(ln_epsilon'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'ln feature count), .epsilon(ln epsilon' in this block.

Line 76: `.sample_valid(ln_sample_valid), .sample_ready(ln_sample_ready),` This connects the child instance's named port 'sample_valid' to parent expression 'ln_sample_valid), .sample_ready(ln_sample_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 77: `.sample_value(ln_sample_value), .sample_gamma_q15(ln_sample_gamma),` This connects the child instance's named port 'sample_value' to parent expression 'ln_sample_value), .sample_gamma_q15(ln_sample_gamma'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the learned multiplicative LayerNorm parameter.

Line 78: `.sample_beta(ln_sample_beta), .sample_last(ln_sample_last),` This connects the child instance's named port 'sample_beta' to parent expression 'ln_sample_beta), .sample_last(ln_sample_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 79: `.result_valid(ln_result_valid), .result_ready(ln_result_ready),` This connects the child instance's named port 'result_valid' to parent expression 'ln_result_valid), .result_ready(ln_result_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 80: `.result_value(ln_result_value), .result_index(ln_result_index),` This connects the child instance's named port 'result_value' to parent expression 'ln_result_value), .result_index(ln_result_index'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an integer selecting one tensor element, lane, bank, or state entry.

Line 81: `.result_last(ln_result_last), .config_error(ln_config_error),` This connects the child instance's named port 'result_last' to parent expression 'ln_result_last), .config_error(ln_config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 82: `.cycle_count(ln_cycle_count), .debug_mean(ln_debug_mean),` This connects the child instance's named port 'cycle_count' to parent expression 'ln_cycle_count), .debug_mean(ln_debug_mean'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the arithmetic average used by normalization.

Line 83: `.debug_variance(ln_debug_variance), .debug_sigma(ln_debug_sigma),` This connects the child instance's named port 'debug_variance' to parent expression 'ln_debug_variance), .debug_sigma(ln_debug_sigma'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the average squared distance from the mean.

Line 84: `.debug_reciprocal_q31(ln_debug_reciprocal)` This connects the child instance's named port 'debug_reciprocal_q31 to parent expression 'ln_debug_reciprocal'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'ln debug reciprocal' in this block.

Line 85: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 87: `neuraedge_log_softmax u_softmax (` This instantiates module neuraedge_log_softmax as u_softmax. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 88: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 89: `.start_valid(sm_start_valid), .start_ready(sm_start_ready),` This connects the child instance's named port 'start_valid' to parent expression 'sm_start_valid), .start_ready(sm_start_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 90: `.item_count(sm_item_count), .difference_shift(sm_difference_shift),` This connects the child instance's named port ‘item_count‘ to parent expression ‘sm_item_count), .difference_shift(sm_difference_shift‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named ’sm item count), .difference shift(sm difference shift’ in this block.

Line 91: `.score_valid(sm_score_valid), .score_ready(sm_score_ready),` This connects the child instance's named port ‘score_valid‘ to parent expression ‘sm_score_valid), .score_ready(sm_score_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 92: `.score_value(sm_score_value), .score_last(sm_score_last),` This connects the child instance's named port ‘score_value‘ to parent expression ‘sm_score_value), .score_last(sm_score_last‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 93: `.probability_valid(sm_probability_valid), .probability_ready(sm_probability_ready),` This connects the child instance's named port ‘probability_valid‘ to parent expression ‘sm_probability_valid), .probability_ready(sm_proba The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 94: `.probability_q15(sm_probability), .probability_index(sm_probability_index),` This connects the child instance's named port ‘probability_q15‘ to parent expression ‘sm_probability), .probability_index(sm_probability_index‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an integer selecting one tensor element, lane, bank, or state entry.

Line 95: `.probability_last(sm_probability_last), .config_error(sm_config_error),` This connects the child instance's named port ‘probability_last‘ to parent expression ‘sm_probability_last), .config_error(sm_config_error‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 96: `.cycle_count(sm_cycle_count), .debug_maximum_score(sm_debug_maximum),` This connects the child instance's named port ‘cycle_count‘ to parent expression ‘sm_cycle_count), .debug_maximum_score(sm_debug_maximum‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named ’sm cycle count), .debug maximum score(sm debug maximum’ in this block.

Line 97: `.debug_denominator(sm_debug_denominator),` This connects the child instance's named port ‘debug_denominator‘ to parent expression ‘sm_debug_denominator‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named ’sm debug denominator’ in this block.

Line 98: `.debug_reciprocal_q31(sm_debug_reciprocal)` This connects the child instance's named port ‘debug_reciprocal_q31 to parent expression ‘sm_debug_reciprocal‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named ’sm debug reciprocal’ in this block.

Line 99: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 101: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are always: the value named ’always’ in this block; clk: the clock that makes all registers advance together.

Line 103: `task automatic run_layernorm_case(input int case_index);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; run_layernorm_case: the value named 'run layernorm case' in this block.

Line 104: `int count;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are count: the value named 'count' in this block.

Line 105: `int epsilon_value;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are epsilon_value: the value named 'epsilon value' in this block.

Line 106: `int received;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are received: the value named 'received' in this block.

Line 107: `int output_cycle;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are output_cycle: the value named 'output cycle' in this block.

Line 108: `int first_sample;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are first_sample: the value named 'first sample' in this block.

Line 109: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task run layernorm case; it has no arithmetic value of its own.

Line 110: `count = ln_case_mem[case_index][11:0];` This makes count, the value named 'count' in this block, equal to 'ln_case_mem[case_index][11:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are ln_case_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run layernorm case.

Line 111: `epsilon_value = ln_case_mem[case_index][43:12];` This makes epsilon_value, the value named 'epsilon value' in this block, equal to 'ln_case_mem[case_index][43:12]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are ln_case_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run layernorm case.

Line 112: `first_sample = ln_pointer;` This makes first_sample, the value named 'first sample' in this block, equal to 'ln_pointer'. It is driven by combinational or procedural logic in the current evaluation. The important operands are ln_pointer: the value named 'ln pointer' in this block. Context: the simulation task run layernorm case.

Line 113: `while (!ln_start_ready) @(negedge clk);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are while: the value named 'while' in this block; ln_start_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 114: `@(negedge clk);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 115: `ln_feature_count = 12'(count);` This makes ln_feature_count, an integer count retained for control or measurable telemetry, equal to ‘12’(count)‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are count: the value named ’count’ in this block. Context: the simulation task run layernorm case.

Line 116: `ln_epsilon = epsilon_value;` This makes ln_epsilon, the value named ’ln epsilon’ in this block, equal to ‘epsilon_value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are epsilon_value: the value named ’epsilon value’ in this block. Context: the simulation task run layernorm case.

Line 117: `ln_start_valid = 1'b1;` This makes ln_start_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named ’b1’ in this block. Context: the simulation task run layernorm case.

Line 118: `@(posedge clk);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 119: `@(negedge clk);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 120: `ln_start_valid = 1'b0;` This makes ln_start_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task run layernorm case.

Line 122: `for (int sample = 0; sample < count; sample++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run layernorm case.

Line 123: `if (!ln_sample_ready) begin` This condition selects the following hardware branch only when ‘!ln_sample_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are ln_sample_ready: the consumer’s statement that it can accept a payload this cycle. Context: the simulation task run layernorm case.

Line 124: `errors++;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 125: `$display("ERROR LayerNorm sample ready case=%0d sample=%0d", case_index, sample);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 126: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run layernorm case; it has no arithmetic value of its own.

Line 127: `ln_sample_value = ln_value_mem[first_sample + sample];` This makes ln_sample_value, the value named ’ln sample value’ in this block, equal to ‘ln_value_mem[first_sample + sample]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are ln_value_mem: local FPGA storage addressed by surrounding index logic; first_sample: the value named ’first sample’ in this block; sample: the value named ’sample’ in this block. Context: the simulation task run layernorm case.

Line 128: `ln_sample_gamma = ln_gamma_mem[first_sample + sample];` This makes ln_sample_gamma, the learned multiplicative LayerNorm parameter, equal to ‘ln_gamma_mem[first_sample + sample]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are ln_gamma_mem: the learned multiplicative LayerNorm parameter; first_sample: the value named ’first sample’ in this block; sample: the value named ’sample’ in this block. Context: the simulation task run layernorm case.

Line 129: `ln_sample_beta = ln_beta_mem[first_sample + sample];` This makes ln_sample_beta, the learned additive LayerNorm parameter, equal to ‘ln_beta_mem[first_sample + sample]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are ln_beta_mem: the learned additive LayerNorm parameter; first_sample: the value named ’first sample’ in this block; sample: the value named ’sample’ in this block. Context: the simulation task run layernorm case.

Line 130: `ln_sample_last = sample == count - 1;` This makes ln_sample_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘sample == count - 1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are sample: the value named ’sample’ in this block; count: the value named ’count’ in this block. Context: the simulation task run layernorm case.

Line 131: `ln_sample_valid = 1'b1;` This makes ln_sample_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named ’b1’ in this block. Context: the simulation task run layernorm case.

Line 132: `@(posedge clk);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 133: `@(negedge clk);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 134: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run layernorm case; it has no arithmetic value of its own.

Line 135: `ln_sample_valid = 1'b0;` This makes ln_sample_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task run layernorm case.

Line 136: `ln_sample_last = 1'b0;` This makes ln_sample_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task run layernorm case.

Line 138: `received = 0;` This makes received, the value named ’received’ in this block, equal to ‘0‘. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run layernorm case.

Line 139: `output_cycle = 0;` This makes output_cycle, the value named ’output cycle’ in this block, equal to ‘0‘. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run layernorm case.

Line 140: `while (received < count && output_cycle < 100000) begin` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are while: the value named ’while’ in this block; received: the value named ’received’ in this block; count: the value named ’count’ in this block.

Line 141: `ln_result_ready = ((output_cycle + case_index) % 6 != 1);` This makes ln_result_ready, the consumer's statement that it can accept a payload this cycle, equal to '((output_cycle + case_index) % 6 != 1)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_cycle: the value named 'output cycle' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run layernorm case.

Line 142: `@(posedge clk);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 143: `if (ln_result_valid && ln_result_ready) begin` This condition selects the following hardware branch only when 'ln_result_valid && ln_result_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are ln_result_valid: the producer's statement that its current payload is meaningful; ln_result_ready: the consumer's statement that it can accept a payload this cycle. Context: the simulation task run layernorm case.

Line 144: `if (ln_result_value !== ln_expected_mem[first_sample + received]` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_result_value: the value named 'ln result value' in this block; ln_expected_mem: local FPGA storage addressed by surrounding index logic; first_sample: the value named 'first sample' in this block.

Line 145: `|| ln_result_index !== received` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_result_index: an integer selecting one tensor element, lane, bank, or state entry; received: the value named 'received' in this block.

Line 146: `|| ln_result_last !== (received == count - 1)) begin` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_result_last: the AXI-Stream marker for the final beat of a packet or command payload; received: the value named 'received' in this block; count: the value named 'count' in this block.

Line 147: `errors++;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 148: `if (errors <= 20) begin` This statement continues through source line 151; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR LayerNorm case=%0d index=%0d got=%0d expected=%0d rtl_index=%0d last=%0d", case_index, received, ln_result_value, ln_expected_mem[first_sample + received], ln_result_index, ln_result_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run layernorm case.

Line 149: `$display("ERROR LayerNorm case=%0d index=%0d got=%0d expected=%0d rtl_index=%0d last=%0d",` This is continuation line 2 of the 4-line logical statement that begins at source line 148. Its exact fragment is '$display("ERROR LayerNorm case=%0d index=%0d got=%0d expected=%0d rtl_index=%0d last=%0d",'. The important operands are display: the value named 'display' in this block; ERROR: the value named 'error' in this block; LayerNorm: the value named 'layer norm' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR LayerNorm case=%0d index=%0d got=%0d expected=%0d rtl_index=%0d last=%0d", case_index, received, ln_result_value, ln_expected_mem[first_sample + received], ln_result_index, ln_result_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run layernorm case.

Line 150: `case_index, received, ln_result_value,` This is continuation line 3 of the 4-line logical statement that begins at source line 148. Its exact fragment is ‘case_index, received, ln_result_value,‘. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; received: the value named ’received’ in this block; ln_result_value: the value named ’ln result value’ in this block. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LayerNorm case=%0d index=%0d got=%0d expected=%0d rtl_index=%0d last=%0d", case_index, received, ln_result_value, ln_expected_mem[first_sample + received], ln_result_index, ln_result_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task run layernorm case.

Line 151: `ln_expected_mem[first_sample + received], ln_result_index, ln_result_last);` This is continuation line 4 of the 4-line logical statement that begins at source line 148. Its exact fragment is ‘ln_expected_mem[first_sample + received], ln_result_index, ln_result_last);‘. The important operands are ln_expected_mem: local FPGA storage addressed by surrounding index logic; first_sample: the value named ’first sample’ in this block; received: the value named ’received’ in this block. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR LayerNorm case=%0d index=%0d got=%0d expected=%0d rtl_index=%0d last=%0d", case_index, received, ln_result_value, ln_expected_mem[first_sample + received], ln_result_index, ln_result_last‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the simulation task run layernorm case.

Line 152: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run layernorm case; it has no arithmetic value of its own.

Line 153: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run layernorm case; it has no arithmetic value of its own.

Line 154: `received++;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are received: the value named ’received’ in this block.

Line 155: `ln_checked++;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_checked: the value named ’ln checked’ in this block.

Line 156: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run layernorm case; it has no arithmetic value of its own.

Line 157: `@(negedge clk);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 158: `output_cycle++;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are output_cycle: the value named ’output cycle’ in this block.

Line 159: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run layernorm case; it has no arithmetic value of its own.

Line 160: `ln_result_ready = 1'b0;` This makes ln_result_ready, the consumer’s statement that it can accept a payload this cycle, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task run layernorm case.

Line 161: `if (received != count || ln_config_error` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are received: the value named 'received' in this block; count: the value named 'count' in this block; ln_config_error: the value named 'ln config error' in this block.

Line 162: `|| ln_debug_mean !== ln_debug_mem[case_index][31:0]` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_debug_mean: the arithmetic average used by normalization; ln_debug_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 163: `|| ln_debug_variance !== ln_debug_mem[case_index][63:32]` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_debug_variance: the average squared distance from the mean; ln_debug_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 164: `|| ln_debug_sigma !== ln_debug_mem[case_index][95:64]` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_debug_sigma: the value named 'ln debug sigma' in this block; ln_debug_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 165: `|| ln_debug_reciprocal !== ln_debug_mem[case_index][127:96]) begin` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_debug_reciprocal: the value named 'ln debug reciprocal' in this block; ln_debug_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 166: `errors++;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 167: `$display("ERROR LayerNorm completion case=%0d received=%0d/%0d error=%0d debug=(%0d,%0d,%0d,%0d) expected=(...` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 168: `case_index, received, count, ln_config_error,` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; received: the value named 'received' in this block; count: the value named 'count' in this block.

Line 169: `ln_debug_mean, ln_debug_variance, ln_debug_sigma, ln_debug_reciprocal,` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_debug_mean: the arithmetic average used by normalization; ln_debug_variance: the average squared distance from the mean; ln_debug_sigma: the value named 'ln debug sigma' in this block.

Line 170: `$signed(ln_debug_mem[case_index][31:0]), ln_debug_mem[case_index][63:32],` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_debug_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 171: `ln_debug_mem[case_index][95:64], ln_debug_mem[case_index][127:96]);` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_debug_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 172: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run layernorm case; it has no arithmetic value of its own.

Line 173: `$display("layernorm_case=%0d features=%0d cycles=%0d", case_index, count, ln_cycle_count);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 174: `ln_pointer += count;` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_pointer: the value named 'ln pointer' in this block; count: the value named 'count' in this block.

Line 175: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run layernorm case; it has no arithmetic value of its own.

Line 176: `endtask` Within the simulation task run layernorm case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 178: `task automatic run_softmax_case(input int case_index);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; run_softmax_case: the value named 'run softmax case' in this block.

Line 179: `int count;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are count: the value named 'count' in this block.

Line 180: `int shift_value;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are shift_value: the value named 'shift value' in this block.

Line 181: `int received;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are received: the value named 'received' in this block.

Line 182: `int output_cycle;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are output_cycle: the value named 'output cycle' in this block.

Line 183: `int first_sample;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are first_sample: the value named 'first sample' in this block.

Line 184: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task run softmax case; it has no arithmetic value of its own.

Line 185: `count = sm_case_mem[case_index][8:0];` This makes count, the value named 'count' in this block, equal to 'sm_case_mem[case_index][8:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sm_case_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run softmax case.

Line 186: `shift_value = sm_case_mem[case_index][13:9];` This makes shift_value, the value named 'shift value' in this block, equal to 'sm_case_mem[case_index][13:9]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sm_case_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run softmax case.

Line 187: `first_sample = sm_pointer;` This makes first_sample, the value named 'first sample' in this block, equal to 'sm_pointer'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sm_pointer: the value named 'sm pointer' in this block. Context: the simulation task run softmax case.

Line 188: `while (!sm_start_ready) @(negedge clk);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are while: the value named 'while' in this block; sm_start_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 189: `@(negedge clk);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 190: `sm_item_count = 9'(count);` This makes sm_item_count, an integer count retained for control or measurable telemetry, equal to '9'(count)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are count: the value named 'count' in this block. Context: the simulation task run softmax case.

Line 191: `sm_difference_shift = 5'(shift_value);` This makes sm_difference_shift, the value named 'sm difference shift' in this block, equal to '5'(shift_value)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are shift_value: the value named 'shift value' in this block. Context: the simulation task run softmax case.

Line 192: `sm_start_valid = 1'b1;` This makes sm_start_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task run softmax case.

Line 193: `@(posedge clk);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 194: `@(negedge clk);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 195: `sm_start_valid = 1'b0;` This makes sm_start_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run softmax case.

Line 197: `for (int sample = 0; sample < count; sample++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run softmax case.

Line 198: `if (!sm_score_ready) begin` This condition selects the following hardware branch only when ‘!sm_score_ready‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sm_score_ready: the consumer’s statement that it can accept a payload this cycle. Context: the simulation task run softmax case.

Line 199: `errors++;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 200: `$display("ERROR softmax score ready case=%0d sample=%0d", case_index, sample);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 201: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run softmax case; it has no arithmetic value of its own.

Line 202: `sm_score_value = sm_score_mem[first_sample + sample];` This makes sm_score_value, the value named ’sm score value’ in this block, equal to ‘sm_score_mem[first_sample + sample]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are sm_score_mem: local FPGA storage addressed by surrounding index logic; first_sample: the value named ’first sample’ in this block; sample: the value named ’sample’ in this block. Context: the simulation task run softmax case.

Line 203: `sm_score_last = sample == count - 1;` This makes sm_score_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘sample == count - 1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are sample: the value named ’sample’ in this block; count: the value named ’count’ in this block. Context: the simulation task run softmax case.

Line 204: `sm_score_valid = 1'b1;` This makes sm_score_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named ’b1’ in this block. Context: the simulation task run softmax case.

Line 205: `@(posedge clk);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 206: `@(negedge clk);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 207: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run softmax case; it has no arithmetic value of its own.

Line 208: `sm_score_valid = 1'b0;` This makes sm_score_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task run softmax case.

Line 209: `sm_score_last = 1'b0;` This makes sm_score_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the simulation task run softmax case.

Line 211: `received = 0;` This makes received, the value named ’received’ in this block, equal to ‘0‘. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run softmax case.

Line 212: `output_cycle = 0;` This makes output_cycle, the value named 'output cycle' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run softmax case.

Line 213: `while (received < count && output_cycle < 100000) begin` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are while: the value named 'while' in this block; received: the value named 'received' in this block; count: the value named 'count' in this block.

Line 214: `sm_probability_ready = ((output_cycle + case_index) % 5 != 2);` This makes sm_probability_ready, the consumer's statement that it can accept a payload this cycle, equal to '((output_cycle + case_index) % 5 != 2)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_cycle: the value named 'output cycle' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run softmax case.

Line 215: `@(posedge clk);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 216: `if (sm_probability_valid && sm_probability_ready) begin` This condition selects the following hardware branch only when 'sm_probability_valid && sm_probability_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are sm_probability_valid: the producer's statement that its current payload is meaningful; sm_probability_ready: the consumer's statement that it can accept a payload this cycle. Context: the simulation task run softmax case.

Line 217: `if (sm_probability !== sm_expected_mem[first_sample + received]` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_probability: the value named 'sm probability' in this block; sm_expected_mem: local FPGA storage addressed by surrounding index logic; first_sample: the value named 'first sample' in this block.

Line 218: `|| sm_probability_index !== received` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_probability_index: an integer selecting one tensor element, lane, bank, or state entry; received: the value named 'received' in this block.

Line 219: `|| sm_probability_last !== (received == count - 1)) begin` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_probability_last: the AXI-Stream marker for the final beat of a packet or command payload; received: the value named 'received' in this block; count: the value named 'count' in this block.

Line 220: `errors++;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 221: `if (errors <= 20) begin` This statement continues through source line 224; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR softmax case=%0d index=%0d got=%0d expected=%0d", case_index, received, sm_probability, sm_expected_mem[first_sample + received]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run softmax case.

Line 222: `$display("ERROR softmax case=%0d index=%0d got=%0d expected=%0d",` This is continuation line 2 of the 4-line logical statement that begins at source line 221. Its exact fragment is '$display("ERROR softmax case=%0d index=%0d got=%0d expected=%0d",'. The important operands are display: the value named 'display' in this block; ERROR: the value named 'error' in this block; softmax: the value named 'softmax' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR softmax case=%0d index=%0d got=%0d expected=%0d", case_index, received, sm_probability, sm_expected_mem[first_sample + received]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run softmax case.

Line 223: `case_index, received, sm_probability,` This is continuation line 3 of the 4-line logical statement that begins at source line 221. Its exact fragment is 'case_index, received, sm_probability,'. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; received: the value named 'received' in this block; sm_probability: the value named 'sm probability' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR softmax case=%0d index=%0d got=%0d expected=%0d", case_index, received, sm_probability, sm_expected_mem[first_sample + received]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run softmax case.

Line 224: `sm_expected_mem[first_sample + received]);` This is continuation line 4 of the 4-line logical statement that begins at source line 221. Its exact fragment is 'sm_expected_mem[first_sample + received]);'. The important operands are sm_expected_mem: local FPGA storage addressed by surrounding index logic; first_sample: the value named 'first sample' in this block; received: the value named 'received' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR softmax case=%0d index=%0d got=%0d expected=%0d", case_index, received, sm_probability, sm_expected_mem[first_sample + received]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run softmax case.

Line 225: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run softmax case; it has no arithmetic value of its own.

Line 226: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run softmax case; it has no arithmetic value of its own.

Line 227: `received++;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are received: the value named 'received' in this block.

Line 228: `sm_checked++;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_checked: the value named 'sm checked' in this block.

Line 229: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run softmax case; it has no arithmetic value of its own.

Line 230: `@(negedge clk);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 231: `output_cycle++;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are output_cycle: the value named 'output cycle' in this block.

Line 232: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run softmax case; it has no arithmetic value of its own.

Line 233: `sm_probability_ready = 1'b0;` This makes sm_probability_ready, the consumer's statement that it can accept a payload this cycle, equal to `1'b0`. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run softmax case.

Line 234: `if (received != count || sm_config_error` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are received: the value named 'received' in this block; count: the value named 'count' in this block; sm_config_error: the value named 'sm config error' in this block.

Line 235: `|| sm_debug_maximum !== sm_debug_mem[case_index][23:0]` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_debug_maximum: the value named 'sm debug maximum' in this block; sm_debug_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 236: `|| sm_debug_denominator !== sm_debug_mem[case_index][47:24]` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_debug_denominator: the value named 'sm debug denominator' in this block; sm_debug_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 237: `|| sm_debug_reciprocal !== sm_debug_mem[case_index][79:48]) begin` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_debug_reciprocal: the value named 'sm debug reciprocal' in this block; sm_debug_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 238: `errors++;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 239: `$display("ERROR softmax completion case=%0d received=%0d/%0d error=%0d", case_index,` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 240: `received, count, sm_config_error);` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are received: the value named 'received' in this block; count: the value named 'count' in this block; sm_config_error: the value named 'sm config error' in this block.

Line 241: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run softmax case; it has no arithmetic value of its own.

Line 242: `$display("softmax_case=%0d items=%0d cycles=%0d", case_index, count, sm_cycle_count);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 243: `sm_pointer += count;` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are sm_pointer: the value named 'sm pointer' in this block; count: the value named 'count' in this block.

Line 244: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run softmax case; it has no arithmetic value of its own.

Line 245: `endtask` Within the simulation task run softmax case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 247: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 248: `$readmemh("tb/fixtures/layernorm_cases.mem", ln_case_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 249: `$readmemh("tb/fixtures/layernorm_values.mem", ln_value_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 250: `$readmemh("tb/fixtures/layernorm_gammas.mem", ln_gamma_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 251: `$readmemh("tb/fixtures/layernorm_betas.mem", ln_beta_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 252: `$readmemh("tb/fixtures/layernorm_expected.mem", ln_expected_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 253: `$readmemh("tb/fixtures/layernorm_debug.mem", ln_debug_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 254: `$readmemh("tb/fixtures/softmax_cases.mem", sm_case_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 255: `$readmemh("tb/fixtures/softmax_scores.mem", sm_score_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 256: `$readmemh("tb/fixtures/softmax_expected.mem", sm_expected_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 257: `$readmemh("tb/fixtures/softmax_debug.mem", sm_debug_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 259: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 260: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 262: `for (int case_index = 0; case_index < LN_CASES; case_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 263: `run_layernorm_case(case_index);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are run_layernorm_case: the value named 'run layernorm case' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 264: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 265: `for (int case_index = 0; case_index < SM_CASES; case_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 266: `run_softmax_case(case_index);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are run_softmax_case: the value named 'run softmax case' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 267: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 269: `if (ln_pointer != LN_SAMPLES || sm_pointer != SM_SAMPLES` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_pointer: the value named 'ln pointer' in this block; LN_SAMPLES: the value named 'ln samples' in this block; sm_pointer: the value named 'sm pointer' in this block.

Line 270: `|| ln_checked != LN_SAMPLES || sm_checked != SM_SAMPLES) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_checked: the value named 'ln checked' in this block; LN_SAMPLES: the value named 'ln samples' in this block; sm_checked: the value named 'sm checked' in this block.

Line 271: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 272: `$display("ERROR normalization accounting ln=%0d/%0d checked=%0d sm=%0d/%0d checked=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 273: `ln_pointer, LN_SAMPLES, ln_checked, sm_pointer, SM_SAMPLES, sm_checked);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_pointer: the value named 'ln pointer' in this block; LN_SAMPLES: the value named 'ln samples' in this block; ln_checked: the value named 'ln checked' in this block.

Line 274: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 275: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 276: `$display("tb_neuraedge_normalization_units FAILED errors=%0d", errors);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 277: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 278: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 279: `$display("tb_neuraedge_normalization_units PASSED layernorm=%0d softmax=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 280: `ln_checked, sm_checked);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The important operands are ln_checked: the value named 'ln checked' in this block; sm_checked: the value named 'sm checked' in this block.

Line 281: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 282: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 283: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 285: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 286: `#2_000_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 287: `$fatal(1, "tb_neuraedge_normalization_units watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 288: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 289: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 291: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge normalization units RTL block in simulation.

55 tb/tb_neuraedge_pynq_axi_top.sv

Role in the complete system: drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

55.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

Simulation-only status tb_neuraedge_pynq_axi_top is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

55.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module tb_neuraedge_pynq_axi_top;
5      logic aclk = 1'b0;
6      logic aresetn = 1'b0;
7      logic [7:0] s_axi_awaddr = 8'd0;
8      logic s_axi_awvalid = 1'b0;
9      logic s_axi_awready;
10     logic [31:0] s_axi_wdata = 32'd0;
11     logic [3:0] s_axi_wstrb = 4'd0;
12     logic s_axi_wvalid = 1'b0;
13     logic s_axi_wready;
14     logic [1:0] s_axi_bresp;
15     logic s_axi_bvalid;
16     logic s_axi_bready = 1'b0;
17     logic [7:0] s_axi_araddr = 8'd0;
18     logic s_axi_arvalid = 1'b0;
19     logic s_axi_arready;
20     logic [31:0] s_axi_rdata;
21     logic [1:0] s_axi_rresp;
22     logic s_axi_rvalid;
23     logic s_axi_rready = 1'b0;
24     logic s_axis_tvalid = 1'b0;
25     logic s_axis_tready;
26     logic [63:0] s_axis_tdata = 64'd0;
27     logic [7:0] s_axis_tkeep = 8'hff;
28     logic s_axis_tlast = 1'b0;
29     logic m_axis_tvalid;
30     logic m_axis_tready = 1'b0;
31     logic [63:0] m_axis_tdata;
32     logic [7:0] m_axis_tkeep;
33     logic m_axis_tlast;
34     logic interrupt;
35     int errors = 0;
36
37     neuraedge_pynq_axi_top dut (
38         .aclk(aclk), .aresetn(aresetn),
39         .s_axi_awaddr(s_axi_awaddr), .s_axi_awvalid(s_axi_awvalid),
40         .s_axi_awready(s_axi_awready), .s_axi_wdata(s_axi_wdata),
41         .s_axi_wstrb(s_axi_wstrb), .s_axi_wvalid(s_axi_wvalid),
42         .s_axi_wready(s_axi_wready), .s_axi_bresp(s_axi_bresp),
43         .s_axi_bvalid(s_axi_bvalid), .s_axi_bready(s_axi_bready),
44         .s_axi_araddr(s_axi_araddr), .s_axi_arvalid(s_axi_arvalid),
45         .s_axi_arready(s_axi_arready), .s_axi_rdata(s_axi_rdata),
46         .s_axi_rresp(s_axi_rresp), .s_axi_rvalid(s_axi_rvalid),
47         .s_axi_rready(s_axi_rready),
48         .s_axis_tvalid(s_axis_tvalid), .s_axis_tready(s_axis_tready),
49         .s_axis_tdata(s_axis_tdata), .s_axis_tkeep(s_axis_tkeep),
50         .s_axis_tlast(s_axis_tlast), .m_axis_tvalid(m_axis_tvalid),
51         .m_axis_tready(m_axis_tready), .m_axis_tdata(m_axis_tdata),
52         .m_axis_tkeep(m_axis_tkeep), .m_axis_tlast(m_axis_tlast),
53         .interrupt(interrupt)
54     );
55
56     always #2.5 aclk = ~aclk;
57
58     task automatic send_aw(input logic [7:0] address);
59         begin
60             @(negedge aclk);
```

```
            s_axi_awaddr = address;
            s_axi_awvalid = 1'b1;
            while (!s_axi_awready) @(negedge aclk);
            @(posedge aclk);
            @(negedge aclk);
            s_axi_awvalid = 1'b0;
        end
    endtask

    task automatic send_w(input logic [31:0] value, input logic [3:0] strobes);
        begin
            @(negedge aclk);
            s_axi_wdata = value;
            s_axi_wstrb = strobes;
            s_axi_wvalid = 1'b1;
            while (!s_axi_wready) @(negedge aclk);
            @(posedge aclk);
            @(negedge aclk);
            s_axi_wvalid = 1'b0;
        end
    endtask

    task automatic axi_write(
        input logic [7:0] address,
        input logic [31:0] value,
        input logic [3:0] strobes,
        input logic address_first
    );
        begin
            if (address_first) begin
                send_aw(address);
                repeat (2) @(negedge aclk);
                send_w(value, strobes);
            end else begin
                send_w(value, strobes);
                repeat (3) @(negedge aclk);
                send_aw(address);
            end
            while (!s_axi_bvalid) @(negedge aclk);
            if (s_axi_bresp != 2'b00) begin
                errors++;
                $display("ERROR AXI write response address=%h response=%b", address, s_axi_bresp);
            end
            s_axi_bready = 1'b1;
            @(posedge aclk);
            @(negedge aclk);
            s_axi_bready = 1'b0;
        end
    endtask

    task automatic axi_read(input logic [7:0] address, output logic [31:0] value);
        begin
            @(negedge aclk);
            s_axi_araddr = address;
            s_axi_arvalid = 1'b1;
            while (!s_axi_arready) @(negedge aclk);
            @(posedge aclk);
            @(negedge aclk);
            s_axi_arvalid = 1'b0;
            while (!s_axi_rvalid) @(negedge aclk);
            value = s_axi_rdata;
            if (s_axi_rresp != 2'b00) begin
                errors++;
                $display("ERROR AXI read response address=%h response=%b", address, s_axi_rresp);
            end
            repeat (2) @(negedge aclk);
            if (!s_axi_rvalid || s_axi_rdata !== value) begin
                errors++;
                $display("ERROR AXI read data did not remain stable under backpressure");
            end
            s_axi_rready = 1'b1;
            @(posedge aclk);
            @(negedge aclk);
            s_axi_rready = 1'b0;
        end
    endtask

    task automatic wait_interrupt;
        int timeout;
        begin
            timeout = 0;
            while (!interrupt && timeout < 10000) begin
                @(negedge aclk);
                timeout++;
            end
            if (!interrupt) begin
                errors++;
```

```
148                 $display("ERROR AXI interrupt timeout");
149             end
150         end
151     endtask
152
153     task automatic send_axis_word(
154         input logic [63:0] value,
155         input logic last_value
156     );
157         begin
158             @(negedge aclk);
159             s_axis_tdata = value;
160             s_axis_tlast = last_value;
161             s_axis_tvalid = 1'b1;
162             while (!s_axis_tready) @(negedge aclk);
163             @(posedge aclk);
164             @(negedge aclk);
165             s_axis_tvalid = 1'b0;
166             s_axis_tlast = 1'b0;
167             s_axis_tdata = 64'd0;
168         end
169     endtask
170
171     initial begin
172         logic [31:0] read_value;
173         repeat (5) @(negedge aclk);
174         aresetn = 1'b1;
175
176         axi_read(8'h34, read_value);
177         if (read_value != 32'h4e455552) begin
178             errors++;
179             $display("ERROR AXI identity got=%h", read_value);
180         end
181         axi_read(8'h2c, read_value);
182         if (read_value != 32'h1ff) errors++;
183         axi_read(8'h30, read_value);
184         if (read_value != 32'h00010007) errors++;
185         axi_read(8'h28, read_value);
186         if (read_value != 32'd200000000) errors++;
187         axi_read(8'h48, read_value);
188         if (read_value != 0) errors++;
189         axi_read(8'h74, read_value);
190         if (read_value != 0) errors++;
191         axi_read(8'h80, read_value);
192         if (read_value != 0) errors++;
193         axi_read(8'h94, read_value);
194         if (read_value != 0) errors++;
195         axi_read(8'h98, read_value);
196         if (read_value != 0) errors++;
197
198         // Write address and data channels in both legal arrival orders.
199         axi_write(8'h10, 32'h0000000f, 4'hf, 1'b1);
200         axi_write(8'h14, 32'h00000000, 4'hf, 1'b0);
201         axi_write(8'h18, 32'h00000000, 4'hf, 1'b1);
202         axi_write(8'h1c, 32'h00000000, 4'hf, 1'b0);
203         axi_write(8'h00, 32'h00000101, 4'hf, 1'b1);
204         wait_interrupt();
205         axi_read(8'h04, read_value);
206         if (!read_value[4] || !read_value[2] || read_value[1] || read_value[0]) begin
207             errors++;
208             $display("ERROR AXI successful status=%h", read_value);
209         end
210         axi_read(8'h0c, read_value);
211         if (read_value != 1) begin
212             errors++;
213             $display("ERROR AXI command count=%0d", read_value);
214         end
215
216         // Four output words fill the wrapper's two-entry FIFO plus capture
217         // register. Hold the external sink stalled long enough to prove that
218         // registered credit stops the core without dropping or changing data.
219         axi_write(8'h10, 32'h00000009, 4'hf, 1'b1);
220         axi_write(8'h14, 32'h00000000, 4'hf, 1'b0);
221         axi_write(8'h18, 32'h00000000, 4'hf, 1'b1);
222         axi_write(8'h1c, 32'h00000000, 4'hf, 1'b0);
223         axi_write(8'h00, 32'h00000101, 4'hf, 1'b1);
224         send_axis_word(64'h7fffffff80000000, 1'b0);
225         for (int beat = 0; beat < 4; beat++) begin
226             send_axis_word({32'(11 + 2*beat), 32'(10 + 2*beat)}, 1'b0);
227         end
228         for (int beat = 0; beat < 4; beat++) begin
229             send_axis_word({32'(3 * (2*beat + 1)), 32'(3 * (2*beat))}, beat == 3);
230         end
231         while (!m_axis_tvalid) @(negedge aclk);
232         begin
233             logic [63:0] stalled_data;
234             logic stalled_last;
```

```
            stalled_data = m_axis_tdata;
            stalled_last = m_axis_tlast;
            repeat (12) begin
                @(negedge aclk);
                if (!m_axis_tvalid || m_axis_tdata !== stalled_data
                        || m_axis_tlast !== stalled_last) begin
                    errors++;
                    $display("ERROR AXI output changed while backpressured");
                end
            end
        end
        for (int beat = 0; beat < 4; beat++) begin
            logic [63:0] expected_word;
            expected_word = {32'(14 + 8*beat), 32'(10 + 8*beat)};
            while (!m_axis_tvalid) @(negedge aclk);
            if (m_axis_tdata !== expected_word || m_axis_tlast != (beat == 3)
                    || m_axis_tkeep != 8'hff) begin
                errors++;
                $display("ERROR AXI vector beat=%0d got=%h last=%0d expected=%h",
                         beat, m_axis_tdata, m_axis_tlast, expected_word);
            end
            m_axis_tready = 1'b1;
            @(posedge aclk);
            @(negedge aclk);
            m_axis_tready = 1'b0;
        end
        wait_interrupt();
        axi_read(8'h20, read_value);
        if (read_value != 9) begin
            errors++;
            $display("ERROR AXI vector input-beat count=%0d", read_value);
        end
        axi_read(8'h24, read_value);
        if (read_value != 4) begin
            errors++;
            $display("ERROR AXI vector output-beat count=%0d", read_value);
        end

        // Byte strobes update only the selected command byte.
        axi_write(8'h10, 32'hdeadbe00, 4'b0001, 1'b0);
        axi_read(8'h10, read_value);
        if (read_value != 32'h00000000) begin
            errors++;
            $display("ERROR AXI write strobe result=%h", read_value);
        end

        // Opcode zero is illegal and must set the core error bit.
        axi_write(8'h00, 32'h00000102, 4'hf, 1'b1);
        axi_write(8'h00, 32'h00000101, 4'hf, 1'b0);
        wait_interrupt();
        axi_read(8'h04, read_value);
        if (!read_value[1] || !read_value[2]) begin
            errors++;
            $display("ERROR AXI invalid-command status=%h", read_value);
        end

        // Restore opcode F and clear all nested sticky errors once more.
        axi_write(8'h10, 32'h0000000f, 4'b0001, 1'b1);
        axi_write(8'h00, 32'h00000102, 4'hf, 1'b0);
        axi_write(8'h00, 32'h00000101, 4'hf, 1'b1);
        wait_interrupt();
        axi_read(8'h04, read_value);
        if (read_value[1] || !read_value[2]) begin
            errors++;
            $display("ERROR AXI final clear status=%h", read_value);
        end
        axi_read(8'h0c, read_value);
        if (read_value != 4) begin
            errors++;
            $display("ERROR AXI final command count=%0d", read_value);
        end

        if (errors != 0) begin
            $display("tb_neuraedge_pynq_axi_top FAILED errors=%0d", errors);
            $finish;
        end else begin
            $display("tb_neuraedge_pynq_axi_top PASSED commands=%0d", read_value);
            $finish;
        end
    end

    initial begin
        #20_000;
        $fatal(1, "tb_neuraedge_pynq_axi_top watchdog timeout");
    end
endmodule
```

```
322 `default_nettype wire
```

55.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module tb_neuraedge_pynq_axi_top;` This opens tb_neuraedge_pynq_axi_top;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

Line 5: `logic aclk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, aclk is the AXI and accelerator clock. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 6: `logic aresetn = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, aresetn is the active-low AXI reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 7: `logic [7:0] s_axi_awaddr = 8'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_awaddr is the value named 's axi awaddr' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 8: `logic s_axi_awvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_awvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic s_axi_awready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_awready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic [31:0] s_axi_wdata = 32'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_wdata is the value named 's axi wdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic [3:0] s_axi_wstrb = 4'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_wstrb is the value named 's axi wstrb' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic s_axi_wvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_wvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic s_axi_wready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_wready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic [1:0] s_axi_bresp;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_bresp is the value named 's axi bresp' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic s_axi_bvalid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_bvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic s_axi_bready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_bready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [7:0] s_axi_araddr = 8'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_araddr is the value named 's axi araddr' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic s_axi_arvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_arvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic s_axi_arready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_arready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic [31:0] s_axi_rdata;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_rdata is the value named 's axi rdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic [1:0] s_axi_rresp;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_rresp is the value named 's axi rresp' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic s_axi_rvalid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_rvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic s_axi_rready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axi_rready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic s_axis_tvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic s_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `logic [63:0] s_axis_tdata = 64'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tdata is the value named 's axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic [7:0] s_axis_tkeep = 8'hff;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tkeep is the value named 's axis tkeep' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 28: `logic s_axis_tlast = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `logic m_axis_tvalid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic m_axis_tready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `logic [63:0] m_axis_tdata;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tdata is the value named 'm axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 32: `logic [7:0] m_axis_tkeep;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tkeep is the value named 'm axis tkeep' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `logic m_axis_tlast;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic interrupt;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, interrupt is the hardware notification that wakes software after completion or error. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 35: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 37: `neuraedge_pynq_axi_top dut (` This instantiates module neuraedge_pynq_axi_top as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 38: `.aclk(aclk), .aresetn(aresetn),` This connects the child instance's named port ‘aclk‘ to parent expression ‘aclk), .aresetn(aresetn‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 39: `.s_axi_awaddr(s_axi_awaddr), .s_axi_awvalid(s_axi_awvalid),` This connects the child instance's named port ‘s_axi_awaddr‘ to parent expression ‘s_axi_awaddr), .s_axi_awvalid(s_axi_awvalid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 40: `.s_axi_awready(s_axi_awready), .s_axi_wdata(s_axi_wdata),` This connects the child instance's named port ‘s_axi_awready‘ to parent expression ‘s_axi_awready), .s_axi_wdata(s_axi_wdata‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 41: `.s_axi_wstrb(s_axi_wstrb), .s_axi_wvalid(s_axi_wvalid),` This connects the child instance's named port ‘s_axi_wstrb‘ to parent expression ‘s_axi_wstrb), .s_axi_wvalid(s_axi_wvalid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 42: `.s_axi_wready(s_axi_wready), .s_axi_bresp(s_axi_bresp),` This connects the child instance's named port ‘s_axi_wready‘ to parent expression ‘s_axi_wready), .s_axi_bresp(s_axi_bresp‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 43: `.s_axi_bvalid(s_axi_bvalid), .s_axi_bready(s_axi_bready),` This connects the child instance's named port ‘s_axi_bvalid‘ to parent expression ‘s_axi_bvalid), .s_axi_bready(s_axi_bready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 44: `.s_axi_araddr(s_axi_araddr), .s_axi_arvalid(s_axi_arvalid),` This connects the child instance's named port ‘s_axi_araddr‘ to parent expression ‘s_axi_araddr), .s_axi_arvalid(s_axi_arvalid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 45: `.s_axi_arready(s_axi_arready), .s_axi_rdata(s_axi_rdata),` This connects the child instance's named port ‘s_axi_arready‘ to parent expression ‘s_axi_arready), .s_axi_rdata(s_axi_rdata‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 46: `.s_axi_rresp(s_axi_rresp), .s_axi_rvalid(s_axi_rvalid),` This connects the child instance's named port ‘s_axi_rresp‘ to parent expression ‘s_axi_rresp), .s_axi_rvalid(s_axi_rvalid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 47: `.s_axi_rready(s_axi_rready),` This connects the child instance's named port ‘s_axi_rready‘ to parent expression ‘s_axi_rready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 48: `.s_axis_tvalid(s_axis_tvalid), .s_axis_tready(s_axis_tready),` This connects the child instance's named port ‘s_axis_tvalid‘ to parent expression ‘s_axis_tvalid), .s_axis_tready(s_axis_tready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 49: `.s_axis_tdata(s_axis_tdata), .s_axis_tkeep(s_axis_tkeep),` This connects the child instance's named port ‘s_axis_tdata‘ to parent expression ‘s_axis_tdata), .s_axis_tkeep(s_axis_tkeep‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 's axis tdata), .s axis tkeep(s axis tkeep' in this block.

Line 50: `.s_axis_tlast(s_axis_tlast), .m_axis_tvalid(m_axis_tvalid),` This connects the child instance's named port ‘s_axis_tlast‘ to parent expression ‘s_axis_tlast), .m_axis_tvalid(m_axis_tvalid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 51: `.m_axis_tready(m_axis_tready), .m_axis_tdata(m_axis_tdata),` This connects the child instance's named port ‘m_axis_tready‘ to parent expression ‘m_axis_tready), .m_axis_tdata(m_axis_tdata‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 52: `.m_axis_tkeep(m_axis_tkeep), .m_axis_tlast(m_axis_tlast),` This connects the child instance's named port ‘m_axis_tkeep‘ to parent expression ‘m_axis_tkeep), .m_axis_tlast(m_axis_tlast‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 53: `.interrupt(interrupt)` This connects the child instance's named port 'interrupt' to parent expression 'interrupt'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the hardware notification that wakes software after completion or error.

Line 54: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 56: `always #2.5 aclk = ~aclk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are always: the value named 'always' in this block; aclk: the AXI and accelerator clock.

Line 58: `task automatic send_aw(input logic [7:0] address);` Within the simulation task send aw, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; send_aw: the value named 'send aw' in this block.

Line 59: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task send aw; it has no arithmetic value of its own.

Line 60: `@(negedge aclk);` Within the simulation task send aw, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 61: `s_axi_awaddr = address;` This makes s_axi_awaddr, the value named 's axi awaddr' in this block, equal to 'address'. It is driven by combinational or procedural logic in the current evaluation. The important operands are address: the value named 'address' in this block. Context: the simulation task send aw.

Line 62: `s_axi_awvalid = 1'b1;` This makes s_axi_awvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task send aw.

Line 63: `while (!s_axi_awready) @(negedge aclk);` Within the simulation task send aw, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axi_awready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 64: `@(posedge aclk);` Within the simulation task send aw, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; aclk: the AXI and accelerator clock.

Line 65: `@(negedge aclk);` Within the simulation task send aw, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 66: `s_axi_awvalid = 1'b0;` This makes s_axi_awvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send aw.

Line 67: `end` This structural keyword opens or closes the branch/process scope described as the simulation task send aw; it has no arithmetic value of its own.

Line 68: `endtask` Within the simulation task send aw, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 70: `task automatic send_w(input logic [31:0] value, input logic [3:0] strobes);` Within the simulation task send w, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; send_w: the value named 'send w' in this block.

Line 71: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task send w; it has no arithmetic value of its own.

Line 72: `@(negedge aclk);` Within the simulation task send w, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 73: `s_axi_wdata = value;` This makes s_axi_wdata, the value named 's axi wdata' in this block, equal to ‘value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the simulation task send w.

Line 74: `s_axi_wstrb = strobes;` This makes s_axi_wstrb, the value named 's axi wstrb' in this block, equal to ‘strobes‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are strobes: the value named 'strobes' in this block. Context: the simulation task send w.

Line 75: `s_axi_wvalid = 1'b1;` This makes s_axi_wvalid, the producer's statement that its current payload is meaningful, equal to ‘1'b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task send w.

Line 76: `while (!s_axi_wready) @(negedge aclk);` Within the simulation task send w, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axi_wready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 77: `@(posedge aclk);` Within the simulation task send w, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; aclk: the AXI and accelerator clock.

Line 78: `@(negedge aclk);` Within the simulation task send w, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 79: `s_axi_wvalid = 1'b0;` This makes s_axi_wvalid, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send w.

Line 80: `end` This structural keyword opens or closes the branch/process scope described as the simulation task send w; it has no arithmetic value of its own.

Line 81: `endtask` Within the simulation task send w, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 83: `task automatic axi_write(` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; axi_write: the value named 'axi write' in this block.

Line 84: `input logic [7:0] address,` This declares address as a input port with width [7:0]. It carries the value named 'address' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 85: `input logic [31:0] value,` This declares value as a input port with width [31:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 86: `input logic [3:0] strobes,` This declares strobes as a input port with width [3:0]. It carries the value named 'strobes' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 87: `input logic address_first` This declares address_first as a input port with width one bit. It carries the value named 'address first' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 88: `);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 89: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task axi write; it has no arithmetic value of its own.

Line 90: `if (address_first) begin` This condition selects the following hardware branch only when ‘address_first‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are address_first: the value named 'address first' in this block. Context: the simulation task axi write.

Line 91: `send_aw(address);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are send_aw: the value named 'send aw' in this block; address: the value named 'address' in this block.

Line 92: `repeat (2) @(negedge aclk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 93: `send_w(value, strobes);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are send_w: the value named 'send w' in this block; value: the value named 'value' in this block; strobes: the value named 'strobes' in this block.

Line 94: `end else begin` This structural keyword opens or closes the branch/process scope described as the simulation task axi write; it has no arithmetic value of its own.

Line 95: `send_w(value, strobes);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are send_w: the value named 'send w' in this block; value: the value named 'value' in this block; strobes: the value named 'strobes' in this block.

Line 96: `repeat (3) @(negedge aclk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 97: `send_aw(address);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are send_aw: the value named 'send aw' in this block; address: the value named 'address' in this block.

Line 98: `end` This structural keyword opens or closes the branch/process scope described as the simulation task axi write; it has no arithmetic value of its own.

Line 99: `while (!s_axi_bvalid) @(negedge aclk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axi_bvalid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 100: `if (s_axi_bresp != 2'b00) begin` This condition selects the following hardware branch only when ‘s_axi_bresp != 2'b00‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axi_bresp: the value named 's axi bresp' in this block; b00: the value named 'b00' in this block. Context: the simulation task axi write.

Line 101: `errors++;` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 102: `$display("ERROR AXI write response address=%h response=%b", address, s_axi_bresp);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 103: `end` This structural keyword opens or closes the branch/process scope described as the simulation task axi write; it has no arithmetic value of its own.

Line 104: `s_axi_bready = 1'b1;` This makes s_axi_bready, the consumer's statement that it can accept a payload this cycle, equal to ‘1'b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task axi write.

Line 105: `@(posedge aclk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; aclk: the AXI and accelerator clock.

Line 106: `@(negedge aclk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 107: `s_axi_bready = 1'b0;` This makes s_axi_bready, the consumer's statement that it can accept a payload this cycle, equal to ‘1'b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task axi write.

Line 108: `end` This structural keyword opens or closes the branch/process scope described as the simulation task axi write; it has no arithmetic value of its own.

Line 109: `endtask` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 111: `task automatic axi_read(input logic [7:0] address, output logic [31:0] value);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; axi_read: the value named 'axi read' in this block.

Line 112: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task axi read; it has no arithmetic value of its own.

Line 113: `@(negedge aclk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 114: `s_axi_araddr = address;` This makes s_axi_araddr, the value named 's axi araddr' in this block, equal to ‘address‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are address: the value named 'address' in this block. Context: the simulation task axi read.

Line 115: `s_axi_arvalid = 1'b1;` This makes s_axi_arvalid, the producer's statement that its current payload is meaningful, equal to ‘1'b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task axi read.

Line 116: `while (!s_axi_arready) @(negedge aclk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axi_arready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 117: `@(posedge aclk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; aclk: the AXI and accelerator clock.

Line 118: `@(negedge aclk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 119: `s_axi_arvalid = 1'b0;` This makes s_axi_arvalid, the producer's statement that its current payload is meaningful, equal to ‘1'b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task axi read.

Line 120: `while (!s_axi_rvalid) @(negedge aclk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axi_rvalid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 121: `value = s_axi_rdata;` This makes value, the value named 'value' in this block, equal to ‘s_axi_rdata‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are s_axi_rdata: the value named 's axi rdata' in this block. Context: the simulation task axi read.

Line 122: `if (s_axi_rresp != 2'b00) begin` This condition selects the following hardware branch only when ‘s_axi_rresp != 2'b00‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axi_rresp: the value named 's axi rresp' in this block; b00: the value named 'b00' in this block. Context: the simulation task axi read.

Line 123: `errors++;` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 124: `$display("ERROR AXI read response address=%h response=%b", address, s_axi_rresp);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 125: `end` This structural keyword opens or closes the branch/process scope described as the simulation task axi read; it has no arithmetic value of its own.

Line 126: `repeat (2) @(negedge aclk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 127: `if (!s_axi_rvalid || s_axi_rdata !== value) begin` This condition selects the following hardware branch only when ‘!s_axi_rvalid || s_axi_rdata !== value‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are s_axi_rvalid: the producer's statement that its current payload is meaningful; s_axi_rdata: the value named 's axi rdata' in this block; value: the value named 'value' in this block. Context: the simulation task axi read.

Line 128: `errors++;` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 129: `$display("ERROR AXI read data did not remain stable under backpressure");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 130: `end` This structural keyword opens or closes the branch/process scope described as the simulation task axi read; it has no arithmetic value of its own.

Line 131: `s_axi_rready = 1'b1;` This makes s_axi_rready, the consumer's statement that it can accept a payload this cycle, equal to ‘1'b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task axi read.

Line 132: `@(posedge aclk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; aclk: the AXI and accelerator clock.

Line 133: `@(negedge aclk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 134: `s_axi_rready = 1'b0;` This makes s_axi_rready, the consumer's statement that it can accept a payload this cycle, equal to ‘1'b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task axi read.

Line 135: `end` This structural keyword opens or closes the branch/process scope described as the simulation task axi read; it has no arithmetic value of its own.

Line 136: `endtask` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 138: `task automatic wait_interrupt;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; wait_interrupt: the hardware notification that wakes software after completion or error.

Line 139: `int timeout;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are timeout: the value named 'timeout' in this block.

Line 140: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 141: `timeout = 0;` This makes timeout, the value named 'timeout' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task task.

Line 142: `while (!interrupt && timeout < 10000) begin` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are while: the value named 'while' in this block; interrupt: the hardware notification that wakes software after completion or error; timeout: the value named 'timeout' in this block.

Line 143: `@(negedge aclk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 144: `timeout++;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are timeout: the value named 'timeout' in this block.

Line 145: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 146: `if (!interrupt) begin` This condition selects the following hardware branch only when '!interrupt' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are interrupt: the hardware notification that wakes software after completion or error. Context: the simulation task task.

Line 147: `errors++;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 148: `$display("ERROR AXI interrupt timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 149: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 150: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 151: `endtask` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 153: `task automatic send_axis_word(` Within the simulation task send axis word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; send_axis_word: the value named 'send axis word' in this block.

Line 154: `input logic [63:0] value,` This declares value as a input port with width [63:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 155: `input logic last_value` This declares last_value as a input port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 156: `);` Within the simulation task send axis word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 157: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task send axis word; it has no arithmetic value of its own.

Line 158: `@(negedge aclk);` Within the simulation task send axis word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 159: `s_axis_tdata = value;` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to ‘value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the simulation task send axis word.

Line 160: `s_axis_tlast = last_value;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to ‘last_value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are last_value: the AXI-Stream marker for the final beat of a packet or command payload. Context: the simulation task send axis word.

Line 161: `s_axis_tvalid = 1'b1;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to ‘1'b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task send axis word.

Line 162: `while (!s_axis_tready) @(negedge aclk);` Within the simulation task send axis word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axis_tready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 163: `@(posedge aclk);` Within the simulation task send axis word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; aclk: the AXI and accelerator clock.

Line 164: `@(negedge aclk);` Within the simulation task send axis word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 165: `s_axis_tvalid = 1'b0;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send axis word.

Line 166: `s_axis_tlast = 1'b0;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send axis word.

Line 167: `s_axis_tdata = 64'd0;` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task send axis word.

Line 168: `end` This structural keyword opens or closes the branch/process scope described as the simulation task send axis word; it has no arithmetic value of its own.

Line 169: `endtask` Within the simulation task send axis word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 171: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 172: `logic [31:0] read_value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, read_value is the value named 'read value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 173: `repeat (5) @(negedge aclk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 174: `aresetn = 1'b1;` This makes aresetn, the active-low AXI reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 176: `axi_read(8'h34, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h34: the value named 'h34' in this block; read_value: the value named 'read value' in this block.

Line 177: `if (read_value != 32'h4e455552) begin` This condition selects the following hardware branch only when 'read_value != 32'h4e455552' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block; h4e455552: the value named 'h4e455552' in this block. Context: the module's structural declarations.

Line 178: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 179: `$display("ERROR AXI identity got=%h", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 180: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 181: `axi_read(8'h2c, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h2c: the value named 'h2c' in this block; read_value: the value named 'read value' in this block.

Line 182: `if (read_value != 32'h1ff) errors++;` This condition selects the following hardware branch only when 'read_value != 32'h1ff' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block; h1ff: the value named 'h1ff' in this block. Context: the module's structural declarations.

Line 183: `axi_read(8'h30, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h30: the value named 'h30' in this block; read_value: the value named 'read value' in this block.

Line 184: `if (read_value != 32'h00010007) errors++;` This condition selects the following hardware branch only when 'read_value != 32'h00010007' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block; h00010007: the value named 'h00010007' in this block. Context: the module's structural declarations.

Line 185: `axi_read(8'h28, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h28: the value named 'h28' in this block; read_value: the value named 'read value' in this block.

Line 186: `if (read_value != 32'd200000000) errors++;` This condition selects the following hardware branch only when 'read_value != 32'd200000000' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block; d200000000: the value named 'd200000000' in this block. Context: the module's structural declarations.

Line 187: `axi_read(8'h48, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h48: the value named 'h48' in this block; read_value: the value named 'read value' in this block.

Line 188: `if (read_value != 0) errors++;` This condition selects the following hardware branch only when 'read_value != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block. Context: the module's structural declarations.

Line 189: `axi_read(8'h74, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h74: the value named 'h74' in this block; read_value: the value named 'read value' in this block.

Line 190: `if (read_value != 0) errors++;` This condition selects the following hardware branch only when 'read_value != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block. Context: the module's structural declarations.

Line 191: `axi_read(8'h80, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h80: the value named 'h80' in this block; read_value: the value named 'read value' in this block.

Line 192: `if (read_value != 0) errors++;` This condition selects the following hardware branch only when ‘read_value != 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named ’read value’ in this block. Context: the module’s structural declarations.

Line 193: `axi_read(8’h94, read_value);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named ’axi read’ in this block; h94: the value named ’h94’ in this block; read_value: the value named ’read value’ in this block.

Line 194: `if (read_value != 0) errors++;` This condition selects the following hardware branch only when ‘read_value != 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named ’read value’ in this block. Context: the module’s structural declarations.

Line 195: `axi_read(8’h98, read_value);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named ’axi read’ in this block; h98: the value named ’h98’ in this block; read_value: the value named ’read value’ in this block.

Line 196: `if (read_value != 0) errors++;` This condition selects the following hardware branch only when ‘read_value != 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named ’read value’ in this block. Context: the module’s structural declarations.

Line 198: `// Write address and data channels in both legal arrival orders.` Design intent: Write address and data channels in both legal arrival orders. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

Line 199: `axi_write(8’h10, 32’h0000000f, 4’hf, 1’b1);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named ’axi write’ in this block; h10: the value named ’h10’ in this block; h0000000f: the value named ’h0000000f’ in this block.

Line 200: `axi_write(8’h14, 32’h00000000, 4’hf, 1’b0);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named ’axi write’ in this block; h14: the value named ’h14’ in this block; h00000000: the value named ’h00000000’ in this block.

Line 201: `axi_write(8’h18, 32’h00000000, 4’hf, 1’b1);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named ’axi write’ in this block; h18: the value named ’h18’ in this block; h00000000: the value named ’h00000000’ in this block.

Line 202: `axi_write(8’h1c, 32’h00000000, 4’hf, 1’b0);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named ’axi write’ in this block; h1c: the value named ’h1c’ in this block; h00000000: the value named ’h00000000’ in this block.

Line 203: `axi_write(8’h00, 32’h00000101, 4’hf, 1’b1);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named ’axi write’ in this block; h00: the value named ’h00’ in this block; h00000101: the value named ’h00000101’ in this block.

Line 204: `wait_interrupt();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are wait_interrupt: the hardware notification that wakes software after completion or error.

Line 205: `axi_read(8'h04, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h04: the value named 'h04' in this block; read_value: the value named 'read value' in this block.

Line 206: `if (!read_value[4] || !read_value[2] || read_value[1] || read_value[0]) begin` This condition selects the following hardware branch only when '!read_value[4] || !read_value[2] || read_value[1] || read_value[0]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block. Context: the module's structural declarations.

Line 207: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 208: `$display("ERROR AXI successful status=%h", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 209: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 210: `axi_read(8'h0c, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h0c: the value named 'h0c' in this block; read_value: the value named 'read value' in this block.

Line 211: `if (read_value != 1) begin` This condition selects the following hardware branch only when 'read_value != 1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block. Context: the module's structural declarations.

Line 212: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 213: `$display("ERROR AXI command count=%0d", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 214: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 216: `// Four output words fill the wrapper's two-entry FIFO plus capture` Design intent: Four output words fill the wrapper's two-entry FIFO plus capture This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

Line 217: `// register. Hold the external sink stalled long enough to prove that` Design intent: register. Hold the external sink stalled long enough to prove that This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

Line 218: `// registered credit stops the core without dropping or changing data.` Design intent: registered credit stops the core without dropping or changing data. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

Line 219: `axi_write(8'h10, 32'h00000009, 4'hf, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h10: the value named 'h10' in this block; h00000009: the value named 'h00000009' in this block.

Line 220: `axi_write(8'h14, 32'h00000000, 4'hf, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h14: the value named 'h14' in this block; h00000000: the value named 'h00000000' in this block.

Line 221: `axi_write(8'h18, 32'h00000000, 4'hf, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h18: the value named 'h18' in this block; h00000000: the value named 'h00000000' in this block.

Line 222: `axi_write(8'h1c, 32'h00000000, 4'hf, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h1c: the value named 'h1c' in this block; h00000000: the value named 'h00000000' in this block.

Line 223: `axi_write(8'h00, 32'h00000101, 4'hf, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h00: the value named 'h00' in this block; h00000101: the value named 'h00000101' in this block.

Line 224: `send_axis_word(64'h7fffffff80000000, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are send_axis_word: the value named 'send axis word' in this block; h7fffffff80000000: the value named 'h7fffffff80000000' in this block; b0: the value named 'b0' in this block.

Line 225: `for (int beat = 0; beat < 4; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 226: `send_axis_word({32'(11 + 2*beat), 32'(10 + 2*beat)}, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are send_axis_word: the value named 'send axis word' in this block; beat: the value named 'beat' in this block; b0: the value named 'b0' in this block.

Line 227: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 228: `for (int beat = 0; beat < 4; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 229: `send_axis_word({32'(3 * (2*beat + 1)), 32'(3 * (2*beat))}, beat == 3);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are send_axis_word: the value named 'send axis word' in this block; beat: the value named 'beat' in this block.

Line 230: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 231: `while (!m_axis_tvalid) @(negedge aclk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are while: the value named 'while' in this block; m_axis_tvalid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 232: `begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 233: `logic [63:0] stalled_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stalled_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 234: `logic stalled_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stalled_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 235: `stalled_data = m_axis_tdata;` This makes stalled_data, the payload carried with the surrounding valid/ready handshake, equal to 'm_axis_tdata'. It is driven by combinational or procedural logic in the current evaluation. The important operands are m_axis_tdata: the value named 'm axis tdata' in this block. Context: the module's structural declarations.

Line 236: `stalled_last = m_axis_tlast;` This makes stalled_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'm_axis_tlast'. It is driven by combinational or procedural logic in the current evaluation. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the module's structural declarations.

Line 237: `repeat (12) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block.

Line 238: `@(negedge aclk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 239: `if (!m_axis_tvalid || m_axis_tdata !== stalled_data` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are m_axis_tvalid: the producer's statement that its current payload is meaningful; m_axis_tdata: the value named 'm axis tdata' in this block; stalled_data: the payload carried with the surrounding valid/ready handshake.

Line 240: `|| m_axis_tlast !== stalled_last) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload; stalled_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 241: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 242: `$display("ERROR AXI output changed while backpressured");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 243: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 244: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 245: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 246: `for (int beat = 0; beat < 4; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 247: `logic [63:0] expected_word;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, expected_word is the value named 'expected word' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 248: `expected_word = {32'(14 + 8*beat), 32'(10 + 8*beat)};` This makes expected_word, the value named 'expected word' in this block, equal to '{32'(14 + 8*beat), 32'(10 + 8*beat)}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are beat: the value named 'beat' in this block. Context: the module's structural declarations.

Line 249: `while (!m_axis_tvalid) @(negedge aclk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are while: the value named 'while' in this block; m_axis_tvalid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 250: `if (m_axis_tdata !== expected_word || m_axis_tlast != (beat == 3)` This condition selects the following hardware branch only when 'm_axis_tdata !== expected_word || m_axis_tlast != (beat == 3' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are m_axis_tdata: the value named 'm axis tdata' in this block; expected_word: the value named 'expected word' in this block; m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the module's structural declarations.

Line 251: `|| m_axis_tkeep != 8'hff) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are m_axis_tkeep: the value named 'm axis tkeep' in this block; hff: the value named 'hff' in this block.

Line 252: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 253: `$display("ERROR AXI vector beat=%0d got=%h last=%0d expected=%h",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 254: `beat, m_axis_tdata, m_axis_tlast, expected_word);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are beat: the value named 'beat' in this block; m_axis_tdata: the value named 'm axis tdata' in this block; m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload.

Line 255: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 256: `m_axis_tready = 1'b1;` This makes m_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 257: `@(posedge aclk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; aclk: the AXI and accelerator clock.

Line 258: `@(negedge aclk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; aclk: the AXI and accelerator clock.

Line 259: `m_axis_tready = 1'b0;` This makes m_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 260: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 261: `wait_interrupt();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are wait_interrupt: the hardware notification that wakes software after completion or error.

Line 262: `axi_read(8'h20, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h20: the value named 'h20' in this block; read_value: the value named 'read value' in this block.

Line 263: `if (read_value != 9) begin` This condition selects the following hardware branch only when 'read_value != 9' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block. Context: the module's structural declarations.

Line 264: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 265: `$display("ERROR AXI vector input-beat count=%0d", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 266: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 267: `axi_read(8'h24, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h24: the value named 'h24' in this block; read_value: the value named 'read value' in this block.

Line 268: `if (read_value != 4) begin` This condition selects the following hardware branch only when 'read_value != 4' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block. Context: the module's structural declarations.

Line 269: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 270: `$display("ERROR AXI vector output-beat count=%0d", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 271: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 273: `// Byte strobes update only the selected command byte.` Design intent: Byte strobes update only the selected command byte. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

Line 274: `axi_write(8'h10, 32'hdeadbe00, 4'b0001, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h10: the value named 'h10' in this block; hdeadbe00: the value named 'hdeadbe00' in this block.

Line 275: `axi_read(8'h10, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h10: the value named 'h10' in this block; read_value: the value named 'read value' in this block.

Line 276: `if (read_value != 32'h00000000) begin` This condition selects the following hardware branch only when 'read_value != 32'h00000000' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block; h00000000: the value named 'h00000000' in this block. Context: the module's structural declarations.

Line 277: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 278: `$display("ERROR AXI write strobe result=%h", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 279: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 281: `// Opcode zero is illegal and must set the core error bit.` Design intent: Opcode zero is illegal and must set the core error bit. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

Line 282: `axi_write(8'h00, 32'h00000102, 4'hf, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h00: the value named 'h00' in this block; h00000102: the value named 'h00000102' in this block.

Line 283: `axi_write(8'h00, 32'h00000101, 4'hf, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h00: the value named 'h00' in this block; h00000101: the value named 'h00000101' in this block.

Line 284: `wait_interrupt();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are wait_interrupt: the hardware notification that wakes software after completion or error.

Line 285: `axi_read(8'h04, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h04: the value named 'h04' in this block; read_value: the value named 'read value' in this block.

Line 286: `if (!read_value[1] || !read_value[2]) begin` This condition selects the following hardware branch only when '!read_value[1] || !read_value[2]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named 'read value' in this block. Context: the module's structural declarations.

Line 287: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 288: `$display("ERROR AXI invalid-command status=%h", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 289: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 291: `// Restore opcode F and clear all nested sticky errors once more.` Design intent: Restore opcode F and clear all nested sticky errors once more. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

Line 292: `axi_write(8'h10, 32'h0000000f, 4'b0001, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h10: the value named 'h10' in this block; h0000000f: the value named 'h0000000f' in this block.

Line 293: `axi_write(8'h00, 32'h00000102, 4'hf, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h00: the value named 'h00' in this block; h00000102: the value named 'h00000102' in this block.

Line 294: `axi_write(8'h00, 32'h00000101, 4'hf, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h00: the value named 'h00' in this block; h00000101: the value named 'h00000101' in this block.

Line 295: `wait_interrupt();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are wait_interrupt: the hardware notification that wakes software after completion or error.

Line 296: `axi_read(8'h04, read_value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h04: the value named 'h04' in this block; read_value: the value named 'read value' in this block.

Line 297: `if (read_value[1] || !read_value[2]) begin` This condition selects the following hardware branch only when ‘read_value[1] || !read_value[2]‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named ’read value’ in this block. Context: the module’s structural declarations.

Line 298: `errors++;` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 299: `$display("ERROR AXI final clear status=%h", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 300: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 301: `axi_read(8'h0c, read_value);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are axi_read: the value named ’axi read’ in this block; h0c: the value named ’h0c’ in this block; read_value: the value named ’read value’ in this block.

Line 302: `if (read_value != 4) begin` This condition selects the following hardware branch only when ‘read_value != 4‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are read_value: the value named ’read value’ in this block. Context: the module’s structural declarations.

Line 303: `errors++;` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 304: `$display("ERROR AXI final command count=%0d", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 305: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 307: `if (errors != 0) begin` This condition selects the following hardware branch only when ‘errors != 0‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block. Context: the module’s structural declarations.

Line 308: `$display("tb_neuraedge_pynq_axi_top FAILED errors=%0d", errors);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 309: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 310: `end else begin` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 311: `$display("tb_neuraedge_pynq_axi_top PASSED commands=%0d", read_value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 312: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 313: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 314: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 316: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 317: `#20_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 318: `$fatal(1, "tb_neuraedge_pynq_axi_top watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 319: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 320: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 322: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge pynq axi top RTL block in simulation.

56 tb/tb_neuraedge_pynq_descriptor.sv

Role in the complete system: drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation.

56.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation.

Simulation-only status tb_neuraedge_pynq_descriptor is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

56.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module tb_neuraedge_pynq_descriptor;
5      logic clk = 1'b0;
6      logic rst_n = 1'b0;
7      logic [7:0] awaddr = 8'd0;
8      logic awvalid = 1'b0;
9      wire awready;
10     logic [31:0] wdata = 32'd0;
11     logic [3:0] wstrb = 4'hf;
12     logic wvalid = 1'b0;
13     wire wready;
14     wire [1:0] bresp;
15     wire bvalid;
16     logic bready = 1'b0;
17     logic [7:0] araddr = 8'd0;
18     logic arvalid = 1'b0;
19     wire arready;
20     wire [31:0] rdata;
21     wire [1:0] rresp;
22     wire rvalid;
23     logic rready = 1'b0;
24     logic s_valid = 1'b0;
25     wire s_ready;
26     logic [63:0] s_data = 64'd0;
27     logic [7:0] s_keep = 8'hff;
28     logic s_last = 1'b0;
29     wire m_valid;
30     logic m_ready = 1'b0;
31     wire [63:0] m_data;
32     wire [7:0] m_keep;
33     wire m_last;
34     wire interrupt;
35     int errors = 0;
36
37     neuraedge_pynq_axi_top dut (
38         .aclk(clk), .aresetn(rst_n),
39         .s_axi_awaddr(awaddr), .s_axi_awvalid(awvalid), .s_axi_awready(awready),
40         .s_axi_wdata(wdata), .s_axi_wstrb(wstrb), .s_axi_wvalid(wvalid),
41         .s_axi_wready(wready), .s_axi_bresp(bresp), .s_axi_bvalid(bvalid),
42         .s_axi_bready(bready), .s_axi_araddr(araddr), .s_axi_arvalid(arvalid),
43         .s_axi_arready(arready), .s_axi_rdata(rdata), .s_axi_rresp(rresp),
44         .s_axi_rvalid(rvalid), .s_axi_rready(rready),
45         .s_axis_tvalid(s_valid), .s_axis_tready(s_ready), .s_axis_tdata(s_data),
46         .s_axis_tkeep(s_keep), .s_axis_tlast(s_last),
47         .m_axis_tvalid(m_valid), .m_axis_tready(m_ready), .m_axis_tdata(m_data),
48         .m_axis_tkeep(m_keep), .m_axis_tlast(m_last), .interrupt(interrupt)
49     );
50
51     always #2.5 clk = ~clk;
52
53     task automatic axi_write(input logic [7:0] address, input logic [31:0] value);
54         begin
55             @(negedge clk);
56             awaddr = address;
57             wdata = value;
58             awvalid = 1'b1;
59             wvalid = 1'b1;
```

```
            while (!(awready && wready)) @(negedge clk);
            @(posedge clk);
            @(negedge clk);
            awvalid = 1'b0;
            wvalid = 1'b0;
            while (!bvalid) @(negedge clk);
            if (bresp != 0) errors++;
            bready = 1'b1;
            @(posedge clk);
            @(negedge clk);
            bready = 1'b0;
        end
    endtask

    task automatic axi_read(input logic [7:0] address, output logic [31:0] value);
        begin
            @(negedge clk);
            araddr = address;
            arvalid = 1'b1;
            while (!arready) @(negedge clk);
            @(posedge clk);
            @(negedge clk);
            arvalid = 1'b0;
            while (!rvalid) @(negedge clk);
            value = rdata;
            if (rresp != 0) errors++;
            rready = 1'b1;
            @(posedge clk);
            @(negedge clk);
            rready = 1'b0;
        end
    endtask

    task automatic send_word(input logic [63:0] value, input logic last);
        begin
            @(negedge clk);
            s_data = value;
            s_last = last;
            s_valid = 1'b1;
            while (!s_ready) @(negedge clk);
            @(posedge clk);
            @(negedge clk);
            s_valid = 1'b0;
            s_last = 1'b0;
        end
    endtask

    initial begin
        logic [31:0] value;
        repeat (5) @(negedge clk);
        rst_n = 1'b1;
        axi_read(8'h2c, value);
        if (value != 32'h1ff) errors++;
        axi_read(8'h30, value);
        if (value != 32'h00010007) errors++;
        axi_read(8'h94, value);
        if (value != 0) errors++;
        axi_read(8'h98, value);
        if (value != 0) errors++;

        axi_write(8'h90, 32'd0);
        axi_write(8'h00, 32'h00000104);
        send_word({1'b1, 31'd0, 32'd0}, 1'b0);
        send_word(64'h000000000000000f, 1'b0);
        send_word(64'd0, 1'b1);

        wait (interrupt);
        axi_read(8'h04, value);
        if (!value[2] || value[5] || value[1] || value[0]) begin
            errors++;
            $display("ERROR descriptor status=%h", value);
        end
        axi_read(8'h8c, value);
        if (value != 1) begin
            errors++;
            $display("ERROR descriptor records=%0d", value);
        end
        axi_read(8'h0c, value);
        if (value != 1) begin
            errors++;
            $display("ERROR descriptor commands=%0d", value);
        end
        if (errors != 0) begin
            $display("tb_neuraedge_pynq_descriptor FAILED errors=%0d", errors);
        end else begin
            $display("tb_neuraedge_pynq_descriptor PASSED records=1");
        end
```

```
        $finish;
    end

    initial begin
        #20_000;
        $fatal(1, "PYNQ descriptor watchdog timeout");
    end
endmodule

`default_nettype wire
```

56.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 4: `module tb_neuraedge_pynq_descriptor;` This opens tb_neuraedge_pynq_descriptor;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation.

Line 5: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 6: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 7: `logic [7:0] awaddr = 8'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, awaddr is the value named 'awaddr' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 8: `logic awvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, awvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `wire awready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, awready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic [31:0] wdata = 32'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wdata is the value named 'wdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic [3:0] wstrb = 4'hf;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wstrb is the value named 'wstrb' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic wvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `wire wready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, wready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `wire [1:0] bresp;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, bresp is the value named 'bresp' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `wire bvalid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, bvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic bready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, bready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [7:0] araddr = 8'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, araddr is the value named 'araddr' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic arvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, arvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `wire arready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, arready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `wire [31:0] rdata;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rdata is the value named 'rdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `wire [1:0] rresp;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rresp is the value named 'rresp' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `wire rvalid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic rready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic s_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `wire s_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `logic [63:0] s_data = 64'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic [7:0] s_keep = 8'hff;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_keep is the value named 's keep' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 28: `logic s_last = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `wire m_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic m_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `wire [63:0] m_data;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_data is the payload carried with the surrounding valid/ready handshake. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 32: `wire [7:0] m_keep;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_keep is the value named 'm keep' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `wire m_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `wire interrupt;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, interrupt is the hardware notification that wakes software after completion or error. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 35: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 37: `neuraedge_pynq_axi_top dut (` This instantiates module neuraedge_pynq_axi_top as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 38: `.aclk(clk), .aresetn(rst_n),` This connects the child instance's named port 'aclk' to parent expression 'clk), .aresetn(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 39: `.s_axi_awaddr(awaddr), .s_axi_awvalid(awvalid), .s_axi_awready(awready),` This connects the child instance's named port 's_axi_awaddr' to parent expression 'awaddr), .s_axi_awvalid(awvalid), .s_axi_awready(awready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 40: `.s_axi_wdata(wdata), .s_axi_wstrb(wstrb), .s_axi_wvalid(wvalid),` This connects the child instance's named port 's_axi_wdata' to parent expression 'wdata), .s_axi_wstrb(wstrb), .s_axi_wvalid(wvalid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 41: `.s_axi_wready(wready), .s_axi_bresp(bresp), .s_axi_bvalid(bvalid),` This connects the child instance's named port 's_axi_wready' to parent expression 'wready), .s_axi_bresp(bresp), .s_axi_bvalid(bvalid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 42: `.s_axi_bready(bready), .s_axi_araddr(araddr), .s_axi_arvalid(arvalid),` This connects the child instance's named port 's_axi_bready' to parent expression 'bready), .s_axi_araddr(araddr), .s_axi_arvalid(arvalid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 43: `.s_axi_arready(arready), .s_axi_rdata(rdata), .s_axi_rresp(rresp),` This connects the child instance's named port 's_axi_arready' to parent expression 'arready), .s_axi_rdata(rdata), .s_axi_rresp(rresp'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 44: `.s_axi_rvalid(rvalid), .s_axi_rready(rready),` This connects the child instance's named port 's_axi_rvalid' to parent expression 'rvalid), .s_axi_rready(rready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 45: `.s_axis_tvalid(s_valid), .s_axis_tready(s_ready), .s_axis_tdata(s_data),` This connects the child instance's named port 's_axis_tvalid' to parent expression 's_valid), .s_axis_tready(s_ready), .s_axis_tdata(s_data'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 46: `.s_axis_tkeep(s_keep), .s_axis_tlast(s_last),` This connects the child instance's named port 's_axis_tkeep' to parent expression 's_keep), .s_axis_tlast(s_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 47: `.m_axis_tvalid(m_valid), .m_axis_tready(m_ready), .m_axis_tdata(m_data),` This connects the child instance's named port 'm_axis_tvalid' to parent expression 'm_valid), .m_axis_tready(m_ready), .m_axis_tdata(m_data'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 48: `.m_axis_tkeep(m_keep), .m_axis_tlast(m_last), .interrupt(interrupt)` This connects the child instance's named port 'm_axis_tkeep' to parent expression 'm_keep), .m_axis_tlast(m_last), .interrupt(interrupt'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 49: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 51: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 53: `task automatic axi_write(input logic [7:0] address, input logic [31:0] value);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; axi_write: the value named 'axi write' in this block.

Line 54: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task axi write; it has no arithmetic value of its own.

Line 55: `@(negedge clk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 56: `awaddr = address;` This makes awaddr, the value named 'awaddr' in this block, equal to 'address'. It is driven by combinational or procedural logic in the current evaluation. The important operands are address: the value named 'address' in this block. Context: the simulation task axi write.

Line 57: `wdata = value;` This makes wdata, the value named 'wdata' in this block, equal to 'value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the simulation task axi write.

Line 58: `awvalid = 1'b1;` This makes awvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task axi write.

Line 59: `wvalid = 1'b1;` This makes wvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task axi write.

Line 60: `while (!(awready && wready)) @(negedge clk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are while: the value named 'while' in this block; awready: the consumer's statement that it can accept a payload this cycle; wready: the consumer's statement that it can accept a payload this cycle.

Line 61: `@(posedge clk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 62: `@(negedge clk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 63: `awvalid = 1'b0;` This makes awvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task axi write.

Line 64: `wvalid = 1'b0;` This makes wvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task axi write.

Line 65: `while (!bvalid) @(negedge clk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are while: the value named 'while' in this block; bvalid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 66: `if (bresp != 0) errors++;` This condition selects the following hardware branch only when 'bresp != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are bresp: the value named 'bresp' in this block. Context: the simulation task axi write.

Line 67: `bready = 1'b1;` This makes bready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task axi write.

Line 68: `@(posedge clk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 69: `@(negedge clk);` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 70: `bready = 1'b0;` This makes bready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task axi write.

Line 71: `end` This structural keyword opens or closes the branch/process scope described as the simulation task axi write; it has no arithmetic value of its own.

Line 72: `endtask` Within the simulation task axi write, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 74: `task automatic axi_read(input logic [7:0] address, output logic [31:0] value);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; axi_read: the value named 'axi read' in this block.

Line 75: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task axi read; it has no arithmetic value of its own.

Line 76: `@(negedge clk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 77: `araddr = address;` This makes araddr, the value named 'araddr' in this block, equal to 'address'. It is driven by combinational or procedural logic in the current evaluation. The important operands are address: the value named 'address' in this block. Context: the simulation task axi read.

Line 78: `arvalid = 1'b1;` This makes arvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task axi read.

Line 79: `while (!aready) @(negedge clk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are while: the value named 'while' in this block; aready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 80: `@(posedge clk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 81: `@(negedge clk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 82: `arvalid = 1'b0;` This makes arvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task axi read.

Line 83: `while (!rvalid) @(negedge clk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are while: the value named 'while' in this block; rvalid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 84: `value = rdata;` This makes value, the value named 'value' in this block, equal to 'rdata'. It is driven by combinational or procedural logic in the current evaluation. The important operands are rdata: the value named 'rdata' in this block. Context: the simulation task axi read.

Line 85: `if (rresp != 0) errors++;` This condition selects the following hardware branch only when 'rresp != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are rresp: the value named 'rresp' in this block. Context: the simulation task axi read.

Line 86: `rready = 1'b1;` This makes rready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task axi read.

Line 87: `@(posedge clk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 88: `@(negedge clk);` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 89: `rready = 1'b0;` This makes rready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task axi read.

Line 90: `end` This structural keyword opens or closes the branch/process scope described as the simulation task axi read; it has no arithmetic value of its own.

Line 91: `endtask` Within the simulation task axi read, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 93: `task automatic send_word(input logic [63:0] value, input logic last);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; send_word: the value named 'send word' in this block.

Line 94: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task send word; it has no arithmetic value of its own.

Line 95: `@(negedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 96: `s_data = value;` This makes s_data, the payload carried with the surrounding valid/ready handshake, equal to 'value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the simulation task send word.

Line 97: `s_last = last;` This makes s_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'last'. It is driven by combinational or procedural logic in the current evaluation. The important operands are last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the simulation task send word.

Line 98: `s_valid = 1'b1;` This makes s_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task send word.

Line 99: `while (!s_ready) @(negedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are while: the value named 'while' in this block; s_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 100: `@(posedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 101: `@(negedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 102: `s_valid = 1'b0;` This makes s_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send word.

Line 103: `s_last = 1'b0;` This makes s_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send word.

Line 104: `end` This structural keyword opens or closes the branch/process scope described as the simulation task send word; it has no arithmetic value of its own.

Line 105: `endtask` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 107: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 108: `logic [31:0] value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, value is the value named 'value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 109: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 110: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 111: `axi_read(8'h2c, value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h2c: the value named 'h2c' in this block; value: the value named 'value' in this block.

Line 112: `if (value != 32'h1ff) errors++;` This condition selects the following hardware branch only when 'value != 32'h1ff' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; h1ff: the value named 'h1ff' in this block. Context: the module's structural declarations.

Line 113: `axi_read(8'h30, value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h30: the value named 'h30' in this block; value: the value named 'value' in this block.

Line 114: `if (value != 32'h00010007) errors++;` This condition selects the following hardware branch only when 'value != 32'h00010007' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block; h00010007: the value named 'h00010007' in this block. Context: the module's structural declarations.

Line 115: `axi_read(8'h94, value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h94: the value named 'h94' in this block; value: the value named 'value' in this block.

Line 116: `if (value != 0) errors++;` This condition selects the following hardware branch only when 'value != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the module's structural declarations.

Line 117: `axi_read(8'h98, value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h98: the value named 'h98' in this block; value: the value named 'value' in this block.

Line 118: `if (value != 0) errors++;` This condition selects the following hardware branch only when 'value != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the module's structural declarations.

Line 120: `axi_write(8'h90, 32'd0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h90: the value named 'h90' in this block; d0: the value named 'd0' in this block.

Line 121: `axi_write(8'h00, 32'h00000104);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are axi_write: the value named 'axi write' in this block; h00: the value named 'h00' in this block; h00000104: the value named 'h00000104' in this block.

Line 122: `send_word({1'b1, 31'd0, 32'd0}, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; b1: the value named 'b1' in this block; d0: the value named 'd0' in this block.

Line 123: `send_word(64'h000000000000000f, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h000000000000000f: the value named 'h000000000000000f' in this block; b0: the value named 'b0' in this block.

Line 124: `send_word(64'd0, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; d0: the value named 'd0' in this block; b1: the value named 'b1' in this block.

Line 126: `wait (interrupt);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are wait: the value named 'wait' in this block; interrupt: the hardware notification that wakes software after completion or error.

Line 127: `axi_read(8'h04, value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h04: the value named 'h04' in this block; value: the value named 'value' in this block.

Line 128: `if (!value[2] || value[5] || value[1] || value[0]) begin` This condition selects the following hardware branch only when '!value[2] || value[5] || value[1] || value[0]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the module's structural declarations.

Line 129: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 130: `$display("ERROR descriptor status=%h", value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 131: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 132: `axi_read(8'h8c, value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h8c: the value named 'h8c' in this block; value: the value named 'value' in this block.

Line 133: `if (value != 1) begin` This condition selects the following hardware branch only when 'value != 1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the module's structural declarations.

Line 134: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 135: `$display("ERROR descriptor records=%0d", value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 136: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 137: `axi_read(8'h0c, value);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are axi_read: the value named 'axi read' in this block; h0c: the value named 'h0c' in this block; value: the value named 'value' in this block.

Line 138: `if (value != 1) begin` This condition selects the following hardware branch only when 'value != 1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are value: the value named 'value' in this block. Context: the module's structural declarations.

Line 139: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 140: `$display("ERROR descriptor commands=%0d", value);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 141: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 142: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 143: `$display("tb_neuraedge_pynq_descriptor FAILED errors=%0d", errors);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 144: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 145: `$display("tb_neuraedge_pynq_descriptor PASSED records=1");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 146: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 147: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 148: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 150: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 151: `#20_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 152: `$fatal(1, "PYNQ descriptor watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 153: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 154: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 156: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge pynq descriptor RTL block in simulation.

57 tb/tb_neuraedge_sgd_optimizer8.sv

Role in the complete system: drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation.

57.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation.

Simulation-only status tb_neuraedge_sgd_optimizer8 is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

57.2 Complete source

```
`timescale 1ns / 1ps
`default_nettype none
`include "tb/fixtures/operator_fixture_params.svh"

module tb_neuraedge_sgd_optimizer8;
    localparam int CASES = `NEURAEDGE_OPTIMIZER_CASES;

    logic clk = 1'b0;
    logic rst_n = 1'b0;
    logic clear_counters = 1'b0;
    logic input_valid = 1'b0;
    logic input_ready;
    logic [127:0] weight_vector = '0;
    logic [191:0] gradient_vector = '0;
    logic [191:0] velocity_vector = '0;
    logic [17:0] learning_rate_q16 = 18'd0;
    logic [16:0] momentum_q16 = 17'd0;
    logic [16:0] weight_decay_q16 = 17'd0;
    logic [4:0] update_right_shift = 5'd16;
    logic output_valid;
    logic output_ready = 1'b0;
    logic [127:0] updated_weight_vector;
    logic [191:0] updated_velocity_vector;
    logic config_error;
    logic [31:0] update_count;

    logic [63:0] parameter_mem [0:CASES-1];
    logic [127:0] weight_mem [0:CASES-1];
    logic [191:0] gradient_mem [0:CASES-1];
    logic [191:0] velocity_mem [0:CASES-1];
    logic [127:0] expected_weight_mem [0:CASES-1];
    logic [191:0] expected_velocity_mem [0:CASES-1];
    int errors = 0;
    int checked = 0;
    logic held_valid = 1'b0;
    logic [127:0] held_weight;
    logic [191:0] held_velocity;

    neuraedge_sgd_optimizer8 dut (
        .clk(clk),
        .rst_n(rst_n),
        .clear_counters(clear_counters),
        .input_valid(input_valid),
        .input_ready(input_ready),
        .weight_vector(weight_vector),
        .gradient_vector(gradient_vector),
        .velocity_vector(velocity_vector),
        .learning_rate_q16(learning_rate_q16),
        .momentum_q16(momentum_q16),
        .weight_decay_q16(weight_decay_q16),
        .update_right_shift(update_right_shift),
        .output_valid(output_valid),
        .output_ready(output_ready),
        .updated_weight_vector(updated_weight_vector),
        .updated_velocity_vector(updated_velocity_vector),
        .config_error(config_error),
        .update_count(update_count)
    );

    always #2.5 clk = ~clk;
```

```
61
62      always @(posedge clk) begin
63          if (held_valid && (!output_valid || updated_weight_vector !== held_weight
64                  || updated_velocity_vector !== held_velocity)) begin
65              errors++;
66              $display("ERROR optimizer output changed while stalled");
67          end
68          held_valid <= output_valid && !output_ready;
69          held_weight <= updated_weight_vector;
70          held_velocity <= updated_velocity_vector;
71      end
72
73      initial begin
74          $readmemh("tb/fixtures/optimizer_parameters.mem", parameter_mem);
75          $readmemh("tb/fixtures/optimizer_weights.mem", weight_mem);
76          $readmemh("tb/fixtures/optimizer_gradients.mem", gradient_mem);
77          $readmemh("tb/fixtures/optimizer_velocities.mem", velocity_mem);
78          $readmemh("tb/fixtures/optimizer_expected_weights.mem", expected_weight_mem);
79          $readmemh("tb/fixtures/optimizer_expected_velocities.mem", expected_velocity_mem);
80
81          repeat (5) @(negedge clk);
82          rst_n = 1'b1;
83
84          for (int case_index = 0; case_index < CASES; case_index++) begin
85              while (!input_ready) @(negedge clk);
86              @(negedge clk);
87              learning_rate_q16 = parameter_mem[case_index][17:0];
88              momentum_q16 = parameter_mem[case_index][34:18];
89              weight_decay_q16 = parameter_mem[case_index][51:35];
90              update_right_shift = parameter_mem[case_index][56:52];
91              weight_vector = weight_mem[case_index];
92              gradient_vector = gradient_mem[case_index];
93              velocity_vector = velocity_mem[case_index];
94              input_valid = 1'b1;
95              @(posedge clk);
96              @(negedge clk);
97              input_valid = 1'b0;
98
99              while (!output_valid) @(negedge clk);
100             repeat (case_index % 4) @(negedge clk);
101             if (config_error || updated_weight_vector !== expected_weight_mem[case_index]
102                     || updated_velocity_vector !== expected_velocity_mem[case_index]) begin
103                 errors++;
104                 if (errors <= 20) begin
105                     $display("ERROR optimizer case=%0d config=%0d", case_index, config_error);
106                     for (int lane = 0; lane < 8; lane++) begin
107                         if (updated_weight_vector[lane*16 +: 16]
108                                 !== expected_weight_mem[case_index][lane*16 +: 16]) begin
109                             $display("  weight lane=%0d got=%0d expected=%0d", lane,
110                                     $signed(updated_weight_vector[lane*16 +: 16]),
111                                     $signed(expected_weight_mem[case_index][lane*16 +: 16]));
112                         end
113                         if (updated_velocity_vector[lane*24 +: 24]
114                                 !== expected_velocity_mem[case_index][lane*24 +: 24]) begin
115                             $display("  velocity lane=%0d got=%0d expected=%0d", lane,
116                                     $signed(updated_velocity_vector[lane*24 +: 24]),
117                                     $signed(expected_velocity_mem[case_index][lane*24 +: 24]));
118                         end
119                     end
120                 end
121             end
122             checked++;
123             output_ready = 1'b1;
124             @(posedge clk);
125             @(negedge clk);
126             output_ready = 1'b0;
127         end
128
129         if (update_count != CASES * 8) begin
130             errors++;
131             $display("ERROR optimizer update count got=%0d expected=%0d", update_count, CASES * 8);
132         end
133
134         @(negedge clk);
135         clear_counters = 1'b1;
136         @(posedge clk);
137         @(negedge clk);
138         clear_counters = 1'b0;
139         if (update_count != 0) begin
140             errors++;
141             $display("ERROR optimizer update counter did not clear independently");
142         end
143
144         if (errors != 0) begin
145             $display("tb_neuraedge_sgd_optimizer8 FAILED errors=%0d checked=%0d", errors, checked);
146             $finish;
147         end else begin
```

```
            $display("tb_neuraedge_sgd_optimizer8 PASSED checked=%0d updates=%0d", checked, update_count);
            $finish;
        end
    end

    initial begin
        #200_000;
        $fatal(1, "tb_neuraedge_sgd_optimizer8 watchdog timeout");
    end
endmodule

`default_nettype wire
```

57.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 3: `` `include "tb/fixtures/operator_fixture_params.svh" `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation.

Line 5: `module tb_neuraedge_sgd_optimizer8;` This opens tb_neuraedge_sgd_optimizer8;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation.

**Line 6: `localparam int CASES = `NEURAEDGE_OPTIMIZER_CASES;`** This fixes CASES to `NEURAEDGE_OPTIMIZER_CASES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CASES denotes the value named 'cases' in this block.

Line 8: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic clear_counters = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clear_counters is a hardware telemetry register that accumulates measured events. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic input_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic [127:0] weight_vector = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_vector is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic [191:0] gradient_vector = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gradient_vector is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic [191:0] velocity_vector = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, velocity_vector is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic [17:0] learning_rate_q16 = 18'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, learning_rate_q16 is the fixed-point multiplier controlling update magnitude. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [16:0] momentum_q16 = 17'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, momentum_q16 is the value named 'momentum q16' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic [16:0] weight_decay_q16 = 17'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_decay_q16 is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic [4:0] update_right_shift = 5'd16;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, update_right_shift is the value named 'update right shift' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic output_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic [127:0] updated_weight_vector;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, updated_weight_vector is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic [191:0] updated_velocity_vector;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, updated_velocity_vector is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, config_error is the value named 'config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic [31:0] update_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, update_count is an integer count retained for control or measurable telemetry. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic [63:0] parameter_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, parameter_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 28: `logic [127:0] weight_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_mem is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `logic [191:0] gradient_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, gradient_mem is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic [191:0] velocity_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, velocity_mem is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `logic [127:0] expected_weight_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, expected_weight_mem is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 32: `logic [191:0] expected_velocity_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, expected_velocity_mem is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 34: `int checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 35: `logic held_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `logic [127:0] held_weight;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_weight is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 37: `logic [191:0] held_velocity;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_velocity is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 39: `neuraedge_sgd_optimizer8 dut (` This instantiates module neuraedge_sgd_optimizer8 as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 40: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 41: `.rst_n(rst_n),` This connects the child instance's named port 'rst_n' to parent expression 'rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 42: `.clear_counters(clear_counters),` This connects the child instance's named port 'clear_counters' to parent expression 'clear_counters'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a hardware telemetry register that accumulates measured events.

Line 43: `.input_valid(input_valid),` This connects the child instance's named port 'input_valid' to parent expression 'input_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 44: `.input_ready(input_ready),` This connects the child instance's named port 'input_ready' to parent expression 'input_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 45: `.weight_vector(weight_vector),` This connects the child instance's named port 'weight_vector' to parent expression 'weight_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 46: `.gradient_vector(gradient_vector),` This connects the child instance's named port 'gradient_vector' to parent expression 'gradient_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the signed loss derivative used to update a learned parameter.

Line 47: `.velocity_vector(velocity_vector),` This connects the child instance's named port 'velocity_vector' to parent expression 'velocity_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the momentum history retained by the SGD optimizer.

Line 48: `.learning_rate_q16(learning_rate_q16),` This connects the child instance's named port 'learning_rate_q16' to parent expression 'learning_rate_q16'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the fixed-point multiplier controlling update magnitude.

Line 49: `.momentum_q16(momentum_q16),` This connects the child instance's named port 'momentum_q16' to parent expression 'momentum_q16'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'momentum q16' in this block.

Line 50: `.weight_decay_q16(weight_decay_q16),` This connects the child instance's named port 'weight_decay_q16' to parent expression 'weight_decay_q16'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 51: `.update_right_shift(update_right_shift),` This connects the child instance's named port 'update_right_shift' to parent expression 'update_right_shift'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'update right shift' in this block.

Line 52: `.output_valid(output_valid),` This connects the child instance's named port 'output_valid' to parent expression 'output_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 53: `.output_ready(output_ready),` This connects the child instance's named port 'output_ready' to parent expression 'output_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 54: `.updated_weight_vector(updated_weight_vector),` This connects the child instance's named port 'updated_weight_vector' to parent expression 'updated_weight_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 55: `.updated_velocity_vector(updated_velocity_vector),` This connects the child instance's named port 'updated_velocity_vector' to parent expression 'updated_velocity_vector'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the momentum history retained by the SGD optimizer.

Line 56: `.config_error(config_error),` This connects the child instance's named port 'config_error' to parent expression 'config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'config error' in this block.

Line 57: `.update_count(update_count)` This connects the child instance's named port 'update_count' to parent expression 'update_count'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an integer count retained for control or measurable telemetry.

Line 58: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 60: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 62: `always @(posedge clk) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are always: the value named 'always' in this block; posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 63: `if (held_valid && (!output_valid || updated_weight_vector !== held_weight` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are held_valid: the producer's statement that its current payload is meaningful; output_valid: the producer's statement that its current payload is meaningful; updated_weight_vector: a learned coefficient multiplied by an activation.

Line 64: `|| updated_velocity_vector !== held_velocity)) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are updated_velocity_vector: the momentum history retained by the SGD optimizer; held_velocity: the momentum history retained by the SGD optimizer.

Line 65: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 66: `$display("ERROR optimizer output changed while stalled");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 67: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 68: `held_valid <= output_valid && !output_ready;` This makes held_valid, the producer's statement that its current payload is meaningful, equal to 'output_valid && !output_ready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_valid: the producer's statement that its current payload is meaningful; output_ready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 69: `held_weight <= updated_weight_vector;` This makes held_weight, a learned coefficient multiplied by an activation, equal to 'updated_weight_vector'. It updates after the active clock edge together with other nonblocking assignments. The important operands are updated_weight_vector: a learned coefficient multiplied by an activation. Context: the module's structural declarations.

Line 70: `held_velocity <= updated_velocity_vector;` This makes held_velocity, the momentum history retained by the SGD optimizer, equal to 'updated_velocity_vector'. It updates after the active clock edge together with other nonblocking assignments. The important operands are updated_velocity_vector: the momentum history retained by the SGD optimizer. Context: the module's structural declarations.

Line 71: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 73: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 74: `$readmemh("tb/fixtures/optimizer_parameters.mem", parameter_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 75: `$readmemh("tb/fixtures/optimizer_weights.mem", weight_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 76: `$readmemh("tb/fixtures/optimizer_gradients.mem", gradient_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 77: `$readmemh("tb/fixtures/optimizer_velocities.mem", velocity_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 78: `$readmemh("tb/fixtures/optimizer_expected_weights.mem", expected_weight_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 79: `$readmemh("tb/fixtures/optimizer_expected_velocities.mem", expected_velocity_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 81: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 82: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 84: `for (int case_index = 0; case_index < CASES; case_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 85: `while (!input_ready) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are while: the value named 'while' in this block; input_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 86: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 87: `learning_rate_q16 = parameter_mem[case_index][17:0];` This makes learning_rate_q16, the fixed-point multiplier controlling update magnitude, equal to 'parameter_mem[case_index][17:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are parameter_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 88: `momentum_q16 = parameter_mem[case_index][34:18];` This makes momentum_q16, the value named 'momentum q16' in this block, equal to 'parameter_mem[case_index][34:18]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are parameter_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 89: `weight_decay_q16 = parameter_mem[case_index][51:35];` This makes weight_decay_q16, a learned coefficient multiplied by an activation, equal to 'parameter_mem[case_index][51:35]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are parameter_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 90: `update_right_shift = parameter_mem[case_index][56:52];` This makes update_right_shift, the value named 'update right shift' in this block, equal to 'parameter_mem[case_index][56:52]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are parameter_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 91: `weight_vector = weight_mem[case_index];` This makes weight_vector, a learned coefficient multiplied by an activation, equal to 'weight_mem[case_index]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_mem: a learned coefficient multiplied by an activation; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 92: `gradient_vector = gradient_mem[case_index];` This makes gradient_vector, the signed loss derivative used to update a learned parameter, equal to 'gradient_mem[case_index]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are gradient_mem: the signed loss derivative used to update a learned parameter; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 93: `velocity_vector = velocity_mem[case_index];` This makes velocity_vector, the momentum history retained by the SGD optimizer, equal to 'velocity_mem[case_index]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are velocity_mem: the momentum history retained by the SGD optimizer; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 94: `input_valid = 1'b1;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 95: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 96: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 97: `input_valid = 1'b0;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 99: `while (!output_valid) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are while: the value named 'while' in this block; output_valid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 100: `repeat (case_index % 4) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry; negedge: the value named 'negedge' in this block.

Line 101: `if (config_error || updated_weight_vector !== expected_weight_mem[case_index]` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are config_error: the value named 'config error' in this block; updated_weight_vector: a learned coefficient multiplied by an activation; expected_weight_mem: a learned coefficient multiplied by an activation.

Line 102: `|| updated_velocity_vector !== expected_velocity_mem[case_index]) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are updated_velocity_vector: the momentum history retained by the SGD optimizer; expected_velocity_mem: the momentum history retained by the SGD optimizer; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 103: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 104: `if (errors <= 20) begin` This statement continues through source line 105; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR optimizer case=%0d config=%0d", case_index, config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the module's structural declarations.

Line 105: `$display("ERROR optimizer case=%0d config=%0d", case_index, config_error);` This is continuation line 2 of the 2-line logical statement that begins at source line 104. Its exact fragment is '$display("ERROR optimizer case=%0d config=%0d", case_index, config_error);'. The important operands are display: the value named 'display' in this block; ERROR: the value named 'error' in this block; optimizer: the value named 'optimizer' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR optimizer case=%0d config=%0d", case_index, config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the module's structural declarations.

Line 106: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 107: `if (updated_weight_vector[lane*16 +: 16]` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are updated_weight_vector: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 108: `!== expected_weight_mem[case_index][lane*16 +: 16]) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are expected_weight_mem: a learned coefficient multiplied by an activation; case_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 109: `$display(" weight lane=%0d got=%0d expected=%0d", lane,` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 110: `$signed(updated_weight_vector[lane*16 +: 16]),` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are updated_weight_vector: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 111: `$signed(expected_weight_mem[case_index][lane*16 +: 16]));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are expected_weight_mem: a learned coefficient multiplied by an activation; case_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 112: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 113: `if (updated_velocity_vector[lane*24 +: 24]` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are updated_velocity_vector: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 114: `!== expected_velocity_mem[case_index][lane*24 +: 24]) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are expected_velocity_mem: the momentum history retained by the SGD optimizer; case_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 115: `$display(" velocity lane=%0d got=%0d expected=%0d", lane,` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 116: `$signed(updated_velocity_vector[lane*24 +: 24]),` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are updated_velocity_vector: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 117: `$signed(expected_velocity_mem[case_index][lane*24 +: 24]));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are expected_velocity_mem: the momentum history retained by the SGD optimizer; case_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 118: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 119: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 120: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 121: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 122: `checked++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 123: `output_ready = 1'b1;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 124: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 125: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 126: `output_ready = 1'b0;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 127: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 129: `if (update_count != CASES * 8) begin` This condition selects the following hardware branch only when 'update_count != CASES * 8' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are update_count: an integer count retained for control or measurable telemetry; CASES: the value named 'cases' in this block. Context: the module's structural declarations.

Line 130: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 131: `$display("ERROR optimizer update count got=%0d expected=%0d", update_count, CASES * 8);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 132: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 134: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 135: `clear_counters = 1'b1;` This makes clear_counters, a hardware telemetry register that accumulates measured events, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 136: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 137: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 138: `clear_counters = 1'b0;` This makes clear_counters, a hardware telemetry register that accumulates measured events, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 139: `if (update_count != 0) begin` This condition selects the following hardware branch only when 'update_count != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are update_count: an integer count retained for control or measurable telemetry. Context: the module's structural declarations.

Line 140: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 141: `$display("ERROR optimizer update counter did not clear independently");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 142: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 144: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 145: `$display("tb_neuraedge_sgd_optimizer8 FAILED errors=%0d checked=%0d", errors, checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 146: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 147: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 148: `$display("tb_neuraedge_sgd_optimizer8 PASSED checked=%0d updates=%0d", checked, update_count);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 149: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 150: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 151: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 153: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 154: `#200_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 155: `$fatal(1, "tb_neuraedge_sgd_optimizer8 watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 156: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 157: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 159: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge sgd optimizer8 RTL block in simulation.

58 tb/tb_neuraedge_tensor_tile.sv

Role in the complete system: drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

58.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Simulation-only status tb_neuraedge_tensor_tile is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 4 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

58.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3  `include "tb/fixtures/tensor_tile_params.svh"
4
5  module tb_neuraedge_tensor_tile;
6      localparam int CASES = `NEURAEDGE_TENSOR_CASES;
7      localparam int ISSUES = `NEURAEDGE_TENSOR_ISSUES;
8      localparam int RESULTS = `NEURAEDGE_TENSOR_RESULTS;
9      localparam int ROW_LANES = 8;
10     localparam int COLUMN_LANES = `NEURAEDGE_COLUMN_LANES;
11
12     logic clk;
13     logic rst_n = 1'b0;
14     logic start = 1'b0;
15     logic [1:0] compute_mode = 2'd0;
16     logic [3:0] active_rows = 4'd1;
17     logic [5:0] active_columns = 6'd1;
18     logic [7:0] activation_zero_point = 8'd0;
19     logic [7:0] weight_zero_point = 8'd0;
20     logic config_error;
21     logic issue_valid = 1'b0;
22     logic issue_ready;
23     logic [63:0] activation_vector = 64'd0;
24     logic [COLUMN_LANES*8-1:0] outer_weight_vector = '0;
25     logic [COLUMN_LANES*8*8-1:0] dot_weight_vector = '0;
26     logic issue_last = 1'b0;
27     logic busy;
28     logic result_valid;
29     logic result_ready = 1'b0;
30     logic signed [47:0] result_value;
31     logic [3:0] result_row;
32     logic [5:0] result_column;
33     logic result_last;
34     logic [31:0] debug_accepted_issues;
35     logic [31:0] debug_skipped_issues;
36     logic [31:0] debug_retired_issues;
37     logic [63:0] debug_dense_macs;
38     logic [63:0] debug_spike_adds;
39
40     logic [63:0] case_mem [0:CASES-1];
41     logic [63:0] activation_mem [0:ISSUES-1];
42     logic [COLUMN_LANES*8*8-1:0] weight_mem [0:ISSUES-1];
43     logic signed [63:0] expected_mem [0:RESULTS-1];
44
45     int issue_pointer = 0;
46     int expected_pointer = 0;
47     int errors = 0;
48     int checked = 0;
49     int cycle = 0;
50     logic stalled_valid = 1'b0;
51     logic signed [47:0] stalled_value;
52     logic [3:0] stalled_row;
53     logic [5:0] stalled_column;
54     logic stalled_last;
55
56     neuraedge_tensor_tile dut (
57         .clk(clk),
58         .rst_n(rst_n),
59         .start(start),
60         .compute_mode(compute_mode),
```

```
        .active_rows(active_rows),
        .active_columns(active_columns),
        .activation_zero_point(activation_zero_point),
        .weight_zero_point(weight_zero_point),
        .config_error(config_error),
        .issue_valid(issue_valid),
        .issue_ready(issue_ready),
        .activation_vector(activation_vector),
        .outer_weight_vector(outer_weight_vector),
        .dot_weight_vector(dot_weight_vector),
        .issue_last(issue_last),
        .busy(busy),
        .result_valid(result_valid),
        .result_ready(result_ready),
        .result_value(result_value),
        .result_row(result_row),
        .result_column(result_column),
        .result_last(result_last),
        .debug_accepted_issues(debug_accepted_issues),
        .debug_skipped_issues(debug_skipped_issues),
        .debug_retired_issues(debug_retired_issues),
        .debug_dense_macs(debug_dense_macs),
        .debug_spike_adds(debug_spike_adds)
    );

    initial clk = 1'b0;
    always #2.5 clk = ~clk;

    task automatic run_case(input int case_index);
        logic [63:0] command;
        int case_issues;
        int case_results;
        int received;
        int watchdog;
        int output_cycle;
        longint unsigned expected_dense_macs;
        longint unsigned expected_spike_adds;
        logic signed [63:0] result_extended;
        begin
            command = case_mem[case_index];
            compute_mode = command[1:0];
            active_rows = command[7:4];
            active_columns = command[13:8];
            case_issues = command[25:14];
            activation_zero_point = command[33:26];
            weight_zero_point = command[41:34];
            // Cast before multiplying. Without these casts, SystemVerilog keeps
            // a full tile product can wrap before assignment to an integer.
            case_results = int'(active_rows) * int'(active_columns);
            expected_dense_macs = 0;
            expected_spike_adds = 0;

            @(negedge clk);
            start = 1'b1;
            @(negedge clk);
            start = 1'b0;
            if (config_error) begin
                errors++;
                $display("ERROR case=%0d unexpectedly reported config_error", case_index);
            end

            for (int issue = 0; issue < case_issues; issue++) begin
                while (!issue_ready) @(negedge clk);
                activation_vector = activation_mem[issue_pointer];
                if (compute_mode == 2'd0 || compute_mode == 2'd1) begin
                    logic issue_nonzero;
                    issue_nonzero = 1'b0;
                    for (int row = 0; row < ROW_LANES; row++) begin
                        issue_nonzero = issue_nonzero
                            || activation_mem[issue_pointer][row*8 +: 8]
                                != activation_zero_point;
                    end
                    if (issue_nonzero) begin
                        expected_dense_macs += compute_mode == 2'd0
                            ? int'(active_rows) * int'(active_columns)
                            : ROW_LANES * int'(active_columns);
                    end
                end else begin
                    int active_spikes;
                    active_spikes = 0;
                    for (int row = 0; row < ROW_LANES; row++) begin
                        if (row < active_rows
                                && activation_mem[issue_pointer][row*8 +: 8] != 0) begin
                            active_spikes++;
                        end
                    end
                    expected_spike_adds += active_spikes * int'(active_columns);
```

```
148                 end
149                 dot_weight_vector = weight_mem[issue_pointer];
150                 outer_weight_vector = weight_mem[issue_pointer][COLUMN_LANES*8-1:0];
151                 issue_last = (issue == case_issues - 1);
152                 issue_valid = 1'b1;
153                 @(posedge clk);
154                 @(negedge clk);
155                 issue_valid = 1'b0;
156                 issue_last = 1'b0;
157                 issue_pointer++;
158             end
159 
160             received = 0;
161             watchdog = 0;
162             output_cycle = 0;
163             if (case_index == 0) begin
164                 // The first fixture is the maximum 8x16 tile. Keep READY low
165                 // until all 128 records are resident, proving that compute can
166                 // finish without a downstream consumer and without overflow.
167                 result_ready = 1'b0;
168                 while (int'(dut.result_fifo_count_r)
169                             + int'(dut.result_stage_valid_r) < case_results
170                         && watchdog < 2000) begin
171                     @(negedge clk);
172                     watchdog++;
173                 end
174                 if (int'(dut.result_fifo_count_r)
175                             + int'(dut.result_stage_valid_r) != case_results
176                         || !dut.result_fifo_nonempty_r
177                         || !dut.result_fifo_has_space_r
178                         || !dut.result_stage_valid_r
179                         || dut.p5_valid_r) begin
180                     errors++;
181                     $display("ERROR maximum result buffer did not fill exactly memory_count=%0d stage_valid=%0d nonempty=%0d space=%0d
        p5_valid=%0d",
182                              dut.result_fifo_count_r, dut.result_stage_valid_r,
183                              dut.result_fifo_nonempty_r,
184                              dut.result_fifo_has_space_r, dut.p5_valid_r);
185                 end
186                 watchdog = 0;
187             end
188             while (received < case_results && watchdog < 20000) begin
189                 @(negedge clk);
190                 // Drive READY independently of the result count. Tying stalls to
191                 // `received` can deadlock a ready/valid test when a stalled count
192                 // selects the same false value forever.
193                 result_ready = ((output_cycle + case_index) % 5 != 1);
194                 // READY is now stable before the active edge. Sample the result
195                 // on the same rising edge on which the DUT commits the transfer.
196                 @(posedge clk);
197                 if (result_valid && result_ready) begin
198                     result_extended = {{16{result_value[47]}}, result_value};
199                     if (result_extended !== expected_mem[expected_pointer]) begin
200                         errors++;
201                         if (errors <= 30) begin
202                             $display("ERROR case=%0d row=%0d col=%0d got=%0d expected=%0d",
203                                      case_index, result_row, result_column,
204                                      result_extended, expected_mem[expected_pointer]);
205                         end
206                     end
207                     if (result_last !== (received == case_results - 1)) begin
208                         errors++;
209                         $display("ERROR case=%0d result_last mismatch at result=%0d", case_index, received);
210                     end
211                     expected_pointer++;
212                     received++;
213                     checked++;
214                 end
215                 output_cycle++;
216                 watchdog++;
217             end
218             @(negedge clk);
219             result_ready = 1'b0;
220             if (received != case_results) begin
221                 errors++;
222                 $display("ERROR case=%0d output timeout received=%0d expected=%0d", case_index, received, case_results);
223             end
224             while (busy) @(negedge clk);
225             if (debug_dense_macs !== expected_dense_macs
226                     || debug_spike_adds !== expected_spike_adds) begin
227                 errors++;
228                 $display("ERROR case=%0d counter mismatch dense=%0d/%0d spike=%0d/%0d",
229                          case_index, debug_dense_macs, expected_dense_macs,
230                          debug_spike_adds, expected_spike_adds);
231             end
232             $display("case=%0d mode=%0d rows=%0d columns=%0d issues=%0d skips=%0d dense_macs=%0d spike_adds=%0d",
233                      case_index, compute_mode, active_rows, active_columns, case_issues,
```

```
                    debug_skipped_issues, debug_dense_macs, debug_spike_adds);
        end
    endtask

    always @(posedge clk) begin
        cycle <= cycle + 1;
        if (stalled_valid) begin
            if (!result_valid || result_value !== stalled_value || result_row !== stalled_row
                    || result_column !== stalled_column || result_last !== stalled_last) begin
                errors++;
                $display("ERROR result changed while backpressured");
            end
        end
        stalled_valid <= result_valid && !result_ready;
        stalled_value <= result_value;
        stalled_row <= result_row;
        stalled_column <= result_column;
        stalled_last <= result_last;
    end

    initial begin
        $readmemh("tb/fixtures/tensor_cases.mem", case_mem);
        $readmemh("tb/fixtures/tensor_activations.mem", activation_mem);
        $readmemh("tb/fixtures/tensor_weights.mem", weight_mem);
        $readmemh("tb/fixtures/tensor_expected.mem", expected_mem);
        // The complete numerical regression deliberately runs without a VCD:
        // 128 MAC instances make an unrestricted trace needlessly enormous.
        // A bounded, purpose-built waveform test generates the review VCD.

        repeat (5) @(negedge clk);
        rst_n = 1'b1;
        repeat (2) @(negedge clk);

        for (int case_index = 0; case_index < CASES; case_index++) begin
            run_case(case_index);
        end

        // Synchronous reset recovery is checked after all numerical cases so it cannot
        // conceal a fixture mismatch.
        compute_mode = 2'd0;
        active_rows = 4'd1;
        active_columns = 6'd1;
        @(negedge clk);
        start = 1'b1;
        @(negedge clk);
        start = 1'b0;
        if (!busy) begin
            errors++;
            $display("ERROR reset-recovery setup did not enter busy state");
        end
        rst_n = 1'b0;
        repeat (2) @(negedge clk);
        if (busy || result_valid || issue_ready) begin
            errors++;
            $display("ERROR external reset did not return tile to idle outputs");
        end
        rst_n = 1'b1;

        if (issue_pointer != ISSUES || expected_pointer != RESULTS || checked != RESULTS) begin
            errors++;
            $display("ERROR fixture accounting issue_ptr=%0d/%0d result_ptr=%0d/%0d checked=%0d",
                     issue_pointer, ISSUES, expected_pointer, RESULTS, checked);
        end
        if (errors != 0) begin
            $display("tb_neuraedge_tensor_tile FAILED errors=%0d checked=%0d", errors, checked);
            $fatal(1);
        end
        $display("tb_neuraedge_tensor_tile PASSED cases=%0d checked=%0d", CASES, checked);
        $finish;
    end

    initial begin
        #100_000;
        $display("ERROR tb_neuraedge_tensor_tile watchdog timeout state=%0d busy=%0d issue_ready=%0d result_valid=%0d result_ready=%0d
     accepted=%0d retired=%0d mac_issued=%0d mac_retired=%0d target=%0d",
                 dut.state, busy, issue_ready, result_valid, result_ready,
                 debug_accepted_issues, debug_retired_issues,
                 dut.mac_issue_count, dut.mac_retire_count, dut.target_mac_retires);
        $fatal(1);
    end
endmodule

`default_nettype wire
```

58.3 Line annotations

Line 1: `‘timescale 1ns / 1ps` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `‘default_nettype none` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 3: `‘include "tb/fixtures/tensor_tile_params.svh"` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 5: `module tb_neuraedge_tensor_tile;` This opens tb_neuraedge_tensor_tile;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 6: `localparam int CASES = ‘NEURAEDGE_TENSOR_CASES;` This fixes CASES to ‘NEURAEDGE_TENSOR_CASES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CASES denotes the value named 'cases' in this block.

Line 7: `localparam int ISSUES = ‘NEURAEDGE_TENSOR_ISSUES;` This fixes ISSUES to ‘NEURAEDGE_TENSOR_ISSUES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ISSUES denotes the value named 'issues' in this block.

Line 8: `localparam int RESULTS = ‘NEURAEDGE_TENSOR_RESULTS;` This fixes RESULTS to ‘NEURAEDGE_TENSOR_RESU during elaboration. Vivado uses that constant to shape or simplify the generated circuit; RESULTS denotes the value named 'results' in this block.

Line 9: `localparam int ROW_LANES = 8;` This fixes ROW_LANES to 8 during elaboration. Vivado uses that constant to shape or simplify the generated circuit; ROW_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 10: `localparam int COLUMN_LANES = ‘NEURAEDGE_COLUMN_LANES;` This fixes COLUMN_LANES to ‘NEURAEDGE_COLUMN_LANES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; COLUMN_LANES denotes one parallel arithmetic element within a vector or tensor datapath.

Line 12: `logic clk;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic rst_n = 1’b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic start = 1’b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, start is the value named 'start' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic [1:0] compute_mode = 2’d0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, compute_mode is the value named 'compute mode' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic [3:0] active_rows = 4’d1;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_rows is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [5:0] active_columns = 6'd1;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, active_columns is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic [7:0] activation_zero_point = 8'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_zero_point is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic [7:0] weight_zero_point = 8'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_zero_point is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, config_error is the value named 'config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic issue_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic issue_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic [63:0] activation_vector = 64'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_vector is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic [COLUMN_LANES*8-1:0] outer_weight_vector = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, outer_weight_vector is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic [COLUMN_LANES*8*8-1:0] dot_weight_vector = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, dot_weight_vector is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `logic issue_last = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, issue_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic busy;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, busy is the value named 'busy' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 28: `logic result_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `logic result_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic signed [47:0] result_value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_value is the value named 'result value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `logic [3:0] result_row;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_row is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 32: `logic [5:0] result_column;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_column is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `logic result_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic [31:0] debug_accepted_issues;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, debug_accepted_issues is the value named 'debug accepted issues' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 35: `logic [31:0] debug_skipped_issues;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, debug_skipped_issues is the value named 'debug skipped issues' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `logic [31:0] debug_retired_issues;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, debug_retired_issues is the value named 'debug retired issues' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 37: `logic [63:0] debug_dense_macs;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, debug_dense_macs is a count of ordinary activation-times-weight operations actually issued. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 38: `logic [63:0] debug_spike_adds;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, debug_spike_adds is a binary event produced when a neuron crosses its threshold. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 40: `logic [63:0] case_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, case_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 41: `logic [63:0] activation_mem [0:ISSUES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, activation_mem is an input feature value consumed by a neuron or multiply-accumulate operation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 42: `logic [COLUMN_LANES*8*8-1:0] weight_mem [0:ISSUES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, weight_mem is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 43: `logic signed [63:0] expected_mem [0:RESULTS-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, expected_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 45: `int issue_pointer = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are issue_pointer: the value named 'issue pointer' in this block.

Line 46: `int expected_pointer = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are expected_pointer: the value named 'expected pointer' in this block.

Line 47: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 48: `int checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 49: `int cycle = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are cycle: the value named 'cycle' in this block.

Line 50: `logic stalled_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stalled_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 51: `logic signed [47:0] stalled_value;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stalled_value is the value named 'stalled value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 52: `logic [3:0] stalled_row;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stalled_row is one output-row position processed by the tensor tile. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 53: `logic [5:0] stalled_column;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stalled_column is one output-feature position processed in parallel. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 54: `logic stalled_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, stalled_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 56: `neuraedge_tensor_tile dut (` This instantiates module neuraedge_tensor_tile as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 57: `.clk(clk),` This connects the child instance's named port 'clk' to parent expression 'clk'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 58: `.rst_n(rst_n),` This connects the child instance's named port 'rst_n' to parent expression 'rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 59: `.start(start),` This connects the child instance's named port 'start' to parent expression 'start'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'start' in this block.

Line 60: `.compute_mode(compute_mode),` This connects the child instance's named port ‘compute_mode‘ to parent expression ‘compute_mode‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'compute mode' in this block.

Line 61: `.active_rows(active_rows),` This connects the child instance's named port ‘active_rows‘ to parent expression ‘active_rows‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one output-row position processed by the tensor tile.

Line 62: `.active_columns(active_columns),` This connects the child instance's named port ‘active_columns‘ to parent expression ‘active_columns‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one output-feature position processed in parallel.

Line 63: `.activation_zero_point(activation_zero_point),` This connects the child instance's named port ‘activation_zero_point‘ to parent expression ‘activation_zero_point‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 64: `.weight_zero_point(weight_zero_point),` This connects the child instance's named port ‘weight_zero_point‘ to parent expression ‘weight_zero_point‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 65: `.config_error(config_error),` This connects the child instance's named port ‘config_error‘ to parent expression ‘config_error‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'config error' in this block.

Line 66: `.issue_valid(issue_valid),` This connects the child instance's named port ‘issue_valid‘ to parent expression ‘issue_valid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 67: `.issue_ready(issue_ready),` This connects the child instance's named port ‘issue_ready‘ to parent expression ‘issue_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 68: `.activation_vector(activation_vector),` This connects the child instance's named port ‘activation_vector‘ to parent expression ‘activation_vector‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and an input feature value consumed by a neuron or multiply-accumulate operation.

Line 69: `.outer_weight_vector(outer_weight_vector),` This connects the child instance's named port ‘outer_weight_vector‘ to parent expression ‘outer_weight_vector‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 70: `.dot_weight_vector(dot_weight_vector),` This connects the child instance's named port ‘dot_weight_vector‘ to parent expression ‘dot_weight_vector‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a learned coefficient multiplied by an activation.

Line 71: `.issue_last(issue_last),` This connects the child instance's named port ‘issue_last‘ to parent expression ‘issue_last‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 72: `.busy(busy),` This connects the child instance's named port ‘busy‘ to parent expression ‘busy‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'busy' in this block.

Line 73: `.result_valid(result_valid),` This connects the child instance's named port 'result_valid' to parent expression 'result_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 74: `.result_ready(result_ready),` This connects the child instance's named port 'result_ready' to parent expression 'result_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 75: `.result_value(result_value),` This connects the child instance's named port 'result_value' to parent expression 'result_value'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'result value' in this block.

Line 76: `.result_row(result_row),` This connects the child instance's named port 'result_row' to parent expression 'result_row'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one output-row position processed by the tensor tile.

Line 77: `.result_column(result_column),` This connects the child instance's named port 'result_column' to parent expression 'result_column'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and one output-feature position processed in parallel.

Line 78: `.result_last(result_last),` This connects the child instance's named port 'result_last' to parent expression 'result_last'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 79: `.debug_accepted_issues(debug_accepted_issues),` This connects the child instance's named port 'debug_accepted_issues' to parent expression 'debug_accepted_issues'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug accepted issues' in this block.

Line 80: `.debug_skipped_issues(debug_skipped_issues),` This connects the child instance's named port 'debug_skipped_issues' to parent expression 'debug_skipped_issues'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug skipped issues' in this block.

Line 81: `.debug_retired_issues(debug_retired_issues),` This connects the child instance's named port 'debug_retired_issues' to parent expression 'debug_retired_issues'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug retired issues' in this block.

Line 82: `.debug_dense_macs(debug_dense_macs),` This connects the child instance's named port 'debug_dense_macs' to parent expression 'debug_dense_macs'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a count of ordinary activation-times-weight operations actually issued.

Line 83: `.debug_spike_adds(debug_spike_adds)` This connects the child instance's named port 'debug_spike_adds' to parent expression 'debug_spike_adds'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and a binary event produced when a neuron crosses its threshold.

Line 84: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 86: `initial clk = 1'b0;` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 87: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 89: `task automatic run_case(input int case_index);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; run_case: the value named 'run case' in this block.

Line 90: `logic [63:0] command;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, command is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 91: `int case_issues;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are case_issues: the value named 'case issues' in this block.

Line 92: `int case_results;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are case_results: the value named 'case results' in this block.

Line 93: `int received;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are received: the value named 'received' in this block.

Line 94: `int watchdog;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are watchdog: the value named 'watchdog' in this block.

Line 95: `int output_cycle;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are output_cycle: the value named 'output cycle' in this block.

Line 96: `longint unsigned expected_dense_macs;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are longint: the value named 'longint' in this block; expected_dense_macs: a count of ordinary activation-times-weight operations actually issued.

Line 97: `longint unsigned expected_spike_adds;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are longint: the value named 'longint' in this block; expected_spike_adds: a binary event produced when a neuron crosses its threshold.

Line 98: `logic signed [63:0] result_extended;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, result_extended is the value named 'result extended' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 99: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 100: `command = case_mem[case_index];` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘case_mem[case_index]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are case_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run case.

Line 101: `compute_mode = command[1:0];` This makes compute_mode, the value named 'compute mode' in this block, equal to 'command[1:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the simulation task run case.

Line 102: `active_rows = command[7:4];` This makes active_rows, one output-row position processed by the tensor tile, equal to 'command[7:4]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the simulation task run case.

Line 103: `active_columns = command[13:8];` This makes active_columns, one output-feature position processed in parallel, equal to 'command[13:8]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the simulation task run case.

Line 104: `case_issues = command[25:14];` This makes case_issues, the value named 'case issues' in this block, equal to 'command[25:14]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the simulation task run case.

Line 105: `activation_zero_point = command[33:26];` This makes activation_zero_point, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'command[33:26]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the simulation task run case.

Line 106: `weight_zero_point = command[41:34];` This makes weight_zero_point, a learned coefficient multiplied by an activation, equal to 'command[41:34]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the simulation task run case.

Line 107: `// Cast before multiplying. Without these casts, SystemVerilog keeps` Design intent: Cast before multiplying. Without these casts, SystemVerilog keeps This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 108: `// a full tile product can wrap before assignment to an integer.` Design intent: a full tile product can wrap before assignment to an integer. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 109: `case_results = int'(active_rows) * int'(active_columns);` This makes case_results, the value named 'case results' in this block, equal to 'int'(active_rows) * int'(active_columns)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are active_rows: one output-row position processed by the tensor tile; active_columns: one output-feature position processed in parallel. Context: the simulation task run case.

Line 110: `expected_dense_macs = 0;` This makes expected_dense_macs, a count of ordinary activation-times-weight operations actually issued, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 111: `expected_spike_adds = 0;` This makes expected_spike_adds, a binary event produced when a neuron crosses its threshold, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 113: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 114: `start = 1'b1;` This makes start, the value named 'start' in this block, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task run case.

Line 115: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 116: `start = 1'b0;` This makes start, the value named 'start' in this block, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run case.

Line 117: `if (config_error) begin` This condition selects the following hardware branch only when 'config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are config_error: the value named 'config error' in this block. Context: the simulation task run case.

Line 118: `errors++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 119: `$display("ERROR case=%0d unexpectedly reported config_error", case_index);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 120: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 122: `for (int issue = 0; issue < case_issues; issue++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 123: `while (!issue_ready) @(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are while: the value named 'while' in this block; issue_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 124: `activation_vector = activation_mem[issue_pointer];` This makes activation_vector, an input feature value consumed by a neuron or multiply-accumulate operation, equal to 'activation_mem[issue_pointer]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are activation_mem: an input feature value consumed by a neuron or multiply-accumulate operation; issue_pointer: the value named 'issue pointer' in this block. Context: the simulation task run case.

Line 125: `if (compute_mode == 2'd0 || compute_mode == 2'd1) begin` This condition selects the following hardware branch only when 'compute_mode == 2'd0 || compute_mode == 2'd1' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are compute_mode: the value named 'compute mode' in this block; d0: the value named 'd0' in this block; d1: the value named 'd1' in this block. Context: the simulation task run case.

Line 126: `logic issue_nonzero;` This declares hardware signal storage or wiring in the simulation task run case. Specifically, issue_nonzero is the value named 'issue nonzero' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 127: `issue_nonzero = 1'b0;` This makes issue_nonzero, the value named 'issue nonzero' in this block, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run case.

Line 128: `for (int row = 0; row < ROW_LANES; row++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 129: `issue_nonzero = issue_nonzero` This statement continues through source line 131; the explanation uses the complete expression rather than only the first visual fragment. This makes issue_nonzero, the value named 'issue nonzero' in this block, equal to 'issue_nonzero || activation_mem[issue_pointer][row*8 +: 8] != activation_zero_point'. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_nonzero: the value named 'issue nonzero' in this block; activation_mem: an input feature value consumed by a neuron or multiply-accumulate operation; issue_pointer: the value named 'issue pointer' in this block. Context: the simulation task run case.

Line 130: `|| activation_mem[issue_pointer][row*8 +: 8]` This is continuation line 2 of the 3-line logical statement that begins at source line 129. Its exact fragment is '|| activation_mem[issue_pointer][row*8 +: 8]'. The important operands are activation_mem: an input feature value consumed by a neuron or multiply-accumulate operation; issue_pointer: the value named 'issue pointer' in this block; row: one output-row position processed by the tensor tile. Read together, the complete operation is: This makes issue_nonzero, the value named 'issue nonzero' in this block, equal to 'issue_nonzero || activation_mem[issue_pointer][row*8 +: 8] != activation_zero_point'. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_nonzero: the value named 'issue nonzero' in this block; activation_mem: an input feature value consumed by a neuron or multiply-accumulate operation; issue_pointer: the value named 'issue pointer' in this block. Context: the simulation task run case.

Line 131: `!= activation_zero_point;` This is continuation line 3 of the 3-line logical statement that begins at source line 129. Its exact fragment is '!= activation_zero_point;'. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This makes issue_nonzero, the value named 'issue nonzero' in this block, equal to 'issue_nonzero || activation_mem[issue_pointer][row*8 +: 8] != activation_zero_point'. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue_nonzero: the value named 'issue nonzero' in this block; activation_mem: an input feature value consumed by a neuron or multiply-accumulate operation; issue_pointer: the value named 'issue pointer' in this block. Context: the simulation task run case.

Line 132: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 133: `if (issue_nonzero) begin` This condition selects the following hardware branch only when 'issue_nonzero' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are issue_nonzero: the value named 'issue nonzero' in this block. Context: the simulation task run case.

Line 134: `expected_dense_macs += compute_mode == 2'd0` This statement continues through source line 136; the explanation uses the complete expression rather than only the first visual fragment. Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are expected_dense_macs: a count of ordinary activation-times-weight operations actually issued; compute_mode: the value named 'compute mode' in this block; d0: the value named 'd0' in this block.

Line 135: `? int'(active_rows) * int'(active_columns)` This is continuation line 2 of the 3-line logical statement that begins at source line 134. Its exact fragment is '? int'(active_rows) * int'(active_columns)'. The important operands are active_rows: one output-row position processed by the tensor tile; active_columns: one output-feature position processed in parallel. Read together, the complete operation is: Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in

simulation. The important operands are expected_dense_macs: a count of ordinary activation-times-weight operations actually issued; compute_mode: the value named 'compute mode' in this block; d0: the value named 'd0' in this block.

Line 136: `: ROW_LANES * int'(active_columns);` This is continuation line 3 of the 3-line logical statement that begins at source line 134. Its exact fragment is ': ROW_LANES * int'(active_columns);'. The important operands are ROW_LANES: one parallel arithmetic element within a vector or tensor datapath; active_columns: one output-feature position processed in parallel. Read together, the complete operation is: Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are expected_dense_macs: a count of ordinary activation-times-weight operations actually issued; compute_mode: the value named 'compute mode' in this block; d0: the value named 'd0' in this block.

Line 137: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 138: `end else begin` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 139: `int active_spikes;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are active_spikes: a binary event produced when a neuron crosses its threshold.

Line 140: `active_spikes = 0;` This makes active_spikes, a binary event produced when a neuron crosses its threshold, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 141: `for (int row = 0; row < ROW_LANES; row++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the simulation task run case.

Line 142: `if (row < active_rows` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are row: one output-row position processed by the tensor tile; active_rows: one output-row position processed by the tensor tile.

Line 143: `&& activation_mem[issue_pointer][row*8 +: 8] != 0) begin` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are activation_mem: an input feature value consumed by a neuron or multiply-accumulate operation; issue_pointer: the value named 'issue pointer' in this block; row: one output-row position processed by the tensor tile.

Line 144: `active_spikes++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are active_spikes: a binary event produced when a neuron crosses its threshold.

Line 145: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 146: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 147: `expected_spike_adds += active_spikes * int'(active_columns);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are expected_spike_adds: a binary event produced when a neuron crosses its threshold; active_spikes: a binary event produced when a neuron crosses its threshold; active_columns: one output-feature position processed in parallel.

Line 148: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 149: `dot_weight_vector = weight_mem[issue_pointer];` This makes dot_weight_vector, a learned coefficient multiplied by an activation, equal to 'weight_mem[issue_pointer]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_mem: a learned coefficient multiplied by an activation; issue_pointer: the value named 'issue pointer' in this block. Context: the simulation task run case.

Line 150: `outer_weight_vector = weight_mem[issue_pointer][COLUMN_LANES*8-1:0];` This makes outer_weight_vector, a learned coefficient multiplied by an activation, equal to 'weight_mem[issue_pointer][COLUMN_LANES*8-1:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_mem: a learned coefficient multiplied by an activation; issue_pointer: the value named 'issue pointer' in this block; COLUMN_LANES: one parallel arithmetic element within a vector or tensor datapath. Context: the simulation task run case.

Line 151: `issue_last = (issue == case_issues - 1);` This makes issue_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '(issue == case_issues - 1)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are issue: the value named 'issue' in this block; case_issues: the value named 'case issues' in this block. Context: the simulation task run case.

Line 152: `issue_valid = 1'b1;` This makes issue_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task run case.

Line 153: `@(posedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 154: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 155: `issue_valid = 1'b0;` This makes issue_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run case.

Line 156: `issue_last = 1'b0;` This makes issue_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run case.

Line 157: `issue_pointer++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are issue_pointer: the value named 'issue pointer' in this block.

Line 158: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 160: `received = 0;` This makes received, the value named 'received' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 161: `watchdog = 0;` This makes watchdog, the value named 'watchdog' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 162: `output_cycle = 0;` This makes output_cycle, the value named 'output cycle' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 163: `if (case_index == 0) begin` This condition selects the following hardware branch only when 'case_index == 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run case.

Line 164: `// The first fixture is the maximum 8x16 tile. Keep READY low` Design intent: The first fixture is the maximum 8x16 tile. Keep READY low This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 165: `// until all 128 records are resident, proving that compute can` Design intent: until all 128 records are resident, proving that compute can This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 166: `// finish without a downstream consumer and without overflow.` Design intent: finish without a downstream consumer and without overflow. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 167: `result_ready = 1'b0;` This makes result_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run case.

Line 168: `while (int'(dut.result_fifo_count_r)` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are while: the value named 'while' in this block; dut: the value named 'dut' in this block; result_fifo_count_r: an integer count retained for control or measurable telemetry.

Line 169: `+ int'(dut.result_stage_valid_r) < case_results` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; result_stage_valid_r: the producer's statement that its current payload is meaningful; case_results: the value named 'case results' in this block.

Line 170: `&& watchdog < 2000) begin` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are watchdog: the value named 'watchdog' in this block.

Line 171: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 172: `watchdog++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are watchdog: the value named 'watchdog' in this block.

Line 173: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 174: `if (int'(dut.result_fifo_count_r)` This condition selects the following hardware branch only when 'int'(dut.result_fifo_count_r' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are dut: the value named 'dut' in this block; result_fifo_count_r: an integer count retained for control or measurable telemetry. Context: the simulation task run case.

Line 175: `+ int'(dut.result_stage_valid_r) != case_results` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; result_stage_valid_r: the producer's statement that its current payload is meaningful; case_results: the value named 'case results' in this block.

Line 176: `|| !dut.result_fifo_nonempty_r` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; result_fifo_nonempty_r: the value named 'result fifo nonempty r' in this block.

Line 177: `|| !dut.result_fifo_has_space_r` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; result_fifo_has_space_r: the value named 'result fifo has space r' in this block.

Line 178: `|| !dut.result_stage_valid_r` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; result_stage_valid_r: the producer's statement that its current payload is meaningful.

Line 179: `|| dut.p5_valid_r) begin` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; p5_valid_r: the producer's statement that its current payload is meaningful.

Line 180: `errors++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 181: `$display("ERROR maximum result buffer did not fill exactly memory_count=%0d stage_valid=%0d nonempty=%0d sp...` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 182: `dut.result_fifo_count_r, dut.result_stage_valid_r,` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; result_fifo_count_r: an integer count retained for control or measurable telemetry; result_stage_valid_r: the producer's statement that its current payload is meaningful.

Line 183: `dut.result_fifo_nonempty_r,` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; result_fifo_nonempty_r: the value named 'result fifo nonempty r' in this block.

Line 184: `dut.result_fifo_has_space_r, dut.p5_valid_r);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; result_fifo_has_space_r: the value named 'result fifo has space r' in this block; p5_valid_r: the producer's statement that its current payload is meaningful.

Line 185: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 186: `watchdog = 0;` This makes watchdog, the value named 'watchdog' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task run case.

Line 187: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 188: `while (received < case_results && watchdog < 20000) begin` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are while: the value named 'while' in this block; received: the value named 'received' in this block; case_results: the value named 'case results' in this block.

Line 189: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 190: `// Drive READY independently of the result count. Tying stalls to` Design intent: Drive READY independently of the result count. Tying stalls to This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 191: `// 'received' can deadlock a ready/valid test when a stalled count` Design intent: 'received' can deadlock a ready/valid test when a stalled count This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 192: `// selects the same false value forever.` Design intent: selects the same false value forever. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 193: `result_ready = ((output_cycle + case_index) % 5 != 1);` This makes result_ready, the consumer's statement that it can accept a payload this cycle, equal to '((output_cycle + case_index) % 5 != 1)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are output_cycle: the value named 'output cycle' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the simulation task run case.

Line 194: `// READY is now stable before the active edge. Sample the result` Design intent: READY is now stable before the active edge. Sample the result This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 195: `// on the same rising edge on which the DUT commits the transfer.` Design intent: on the same rising edge on which the DUT commits the transfer. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 196: `@(posedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 197: `if (result_valid && result_ready) begin` This condition selects the following hardware branch only when 'result_valid && result_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_valid: the producer's statement that its current payload is meaningful; result_ready: the consumer's statement that it can accept a payload this cycle. Context: the simulation task run case.

Line 198: `result_extended = {{16{result_value[47]}}, result_value};` This makes result_extended, the value named 'result extended' in this block, equal to '{{16{result_value[47]}}, result_value}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are result_value: the value named 'result value' in this block. Context: the simulation task run case.

Line 199: `if (result_extended !== expected_mem[expected_pointer]) begin` This condition selects the following hardware branch only when 'result_extended !== expected_mem[expected_pointer]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_extended: the value named 'result extended' in this block; expected_mem: local FPGA storage addressed by surrounding index logic; expected_pointer: the value named 'expected pointer' in this block. Context: the simulation task run case.

Line 200: `errors++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 201: `if (errors <= 30) begin` This statement continues through source line 204; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR case=%0d row=%0d col=%0d got=%0d expected=%0d", case_index, result_row, result_column, result_extended, expected_mem[expected_pointer]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 202: `$display("ERROR case=%0d row=%0d col=%0d got=%0d expected=%0d",` This is continuation line 2 of the 4-line logical statement that begins at source line 201. Its exact fragment is '$display("ERROR case=%0d row=%0d col=%0d got=%0d expected=%0d",'. The important operands are display: the value named 'display' in this block; ERROR: the value named 'error' in this block; case: the value named 'case' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR case=%0d row=%0d col=%0d got=%0d expected=%0d", case_index, result_row, result_column, result_extended, expected_mem[expected_pointer]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 203: `case_index, result_row, result_column,` This is continuation line 3 of the 4-line logical statement that begins at source line 201. Its exact fragment is 'case_index, result_row, result_column,'. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; result_row: one output-row position processed by the tensor tile; result_column: one output-feature position processed in parallel. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR case=%0d row=%0d col=%0d got=%0d expected=%0d", case_index, result_row, result_column, result_extended, expected_mem[expected_pointer]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 204: `result_extended, expected_mem[expected_pointer]);` This is continuation line 4 of the 4-line logical statement that begins at source line 201. Its exact fragment is 'result_extended, expected_mem[expected_pointer]);'. The important operands are result_extended: the value named 'result extended' in this block; expected_mem: local FPGA storage addressed by surrounding index logic; expected_pointer: the value named 'expected pointer' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 30) begin $display("ERROR case=%0d row=%0d col=%0d got=%0d expected=%0d", case_index, result_row, result_column, result_extended, expected_mem[expected_pointer]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the simulation task run case.

Line 205: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 206: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 207: `if (result_last !== (received == case_results - 1)) begin` This condition selects the following hardware branch only when ‘result_last !== (received == case_results - 1)‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result_last: the AXI-Stream marker for the final beat of a packet or command payload; received: the value named ’received’ in this block; case_results: the value named ’case results’ in this block. Context: the simulation task run case.

Line 208: `errors++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 209: `$display("ERROR case=%0d result_last mismatch at result=%0d", case_index, received);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 210: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 211: `expected_pointer++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are expected_pointer: the value named ’expected pointer’ in this block.

Line 212: `received++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are received: the value named ’received’ in this block.

Line 213: `checked++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are checked: the value named ’checked’ in this block.

Line 214: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 215: `output_cycle++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are output_cycle: the value named ’output cycle’ in this block.

Line 216: `watchdog++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are watchdog: the value named ’watchdog’ in this block.

Line 217: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 218: `@(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 219: `result_ready = 1'b0;` This makes result_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task run case.

Line 220: `if (received != case_results) begin` This condition selects the following hardware branch only when 'received != case_results' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received: the value named 'received' in this block; case_results: the value named 'case results' in this block. Context: the simulation task run case.

Line 221: `errors++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 222: `$display("ERROR case=%0d output timeout received=%0d expected=%0d", case_index, received, case_results);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 223: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 224: `while (busy) @(negedge clk);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are while: the value named 'while' in this block; busy: the value named 'busy' in this block; negedge: the value named 'negedge' in this block.

Line 225: `if (debug_dense_macs !== expected_dense_macs` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are debug_dense_macs: a count of ordinary activation-times-weight operations actually issued; expected_dense_macs: a count of ordinary activation-times-weight operations actually issued.

Line 226: `|| debug_spike_adds !== expected_spike_adds) begin` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are debug_spike_adds: a binary event produced when a neuron crosses its threshold; expected_spike_adds: a binary event produced when a neuron crosses its threshold.

Line 227: `errors++;` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 228: `$display("ERROR case=%0d counter mismatch dense=%0d/%0d spike=%0d/%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 229: `case_index, debug_dense_macs, expected_dense_macs,` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; debug_dense_macs: a count of ordinary activation-times-weight operations actually issued; expected_dense_macs: a count of ordinary activation-times-weight operations actually issued.

Line 230: `debug_spike_adds, expected_spike_adds);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are debug_spike_adds: a binary event produced when a neuron crosses its threshold; expected_spike_adds: a binary event produced when a neuron crosses its threshold.

Line 231: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 232: `$display("case=%0d mode=%0d rows=%0d columns=%0d issues=%0d skips=%0d dense_macs=%0d spike_adds=%0d` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 233: `case_index, compute_mode, active_rows, active_columns, case_issues,` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are case_index: an integer selecting one tensor element, lane, bank, or state entry; compute_mode: the value named 'compute mode' in this block; active_rows: one output-row position processed by the tensor tile.

Line 234: `debug_skipped_issues, debug_dense_macs, debug_spike_adds);` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are debug_skipped_issues: the value named 'debug skipped issues' in this block; debug_dense_macs: a count of ordinary activation-times-weight operations actually issued; debug_spike_adds: a binary event produced when a neuron crosses its threshold.

Line 235: `end` This structural keyword opens or closes the branch/process scope described as the simulation task run case; it has no arithmetic value of its own.

Line 236: `endtask` Within the simulation task run case, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 238: `always @(posedge clk) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are always: the value named 'always' in this block; posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 239: `cycle <= cycle + 1;` This makes cycle, the value named 'cycle' in this block, equal to 'cycle + 1'. It updates after the active clock edge together with other nonblocking assignments. The important operands are cycle: the value named 'cycle' in this block. Context: the module's structural declarations.

Line 240: `if (stalled_valid) begin` This condition selects the following hardware branch only when 'stalled_valid' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are stalled_valid: the producer's statement that its current payload is meaningful. Context: the module's structural declarations.

Line 241: `if (!result_valid || result_value !== stalled_value || result_row !== stalled_row` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are result_valid: the producer's statement that its current payload is meaningful; result_value: the value named 'result value' in this block; stalled_value: the value named 'stalled value' in this block.

Line 242: `|| result_column !== stalled_column || result_last !== stalled_last) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are result_column: one output-feature position processed in parallel; stalled_column: one output-feature position processed in parallel; result_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 243: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 244: `$display("ERROR result changed while backpressured");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 245: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 246: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 247: `stalled_valid <= result_valid && !result_ready;` This makes stalled_valid, the producer's statement that its current payload is meaningful, equal to 'result_valid && !result_ready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_valid: the producer's statement that its current payload is meaningful; result_ready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 248: `stalled_value <= result_value;` This makes stalled_value, the value named 'stalled value' in this block, equal to 'result_value'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_value: the value named 'result value' in this block. Context: the module's structural declarations.

Line 249: `stalled_row <= result_row;` This makes stalled_row, one output-row position processed by the tensor tile, equal to 'result_row'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_row: one output-row position processed by the tensor tile. Context: the module's structural declarations.

Line 250: `stalled_column <= result_column;` This makes stalled_column, one output-feature position processed in parallel, equal to 'result_column'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_column: one output-feature position processed in parallel. Context: the module's structural declarations.

Line 251: `stalled_last <= result_last;` This makes stalled_last, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'result_last'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the module's structural declarations.

Line 252: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 254: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 255: `$readmemh("tb/fixtures/tensor_cases.mem", case_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 256: `$readmemh("tb/fixtures/tensor_activations.mem", activation_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 257: `$readmemh("tb/fixtures/tensor_weights.mem", weight_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 258: `$readmemh("tb/fixtures/tensor_expected.mem", expected_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 259: `// The complete numerical regression deliberately runs without a VCD:` Design intent: The complete numerical regression deliberately runs without a VCD: This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 260: `// 128 MAC instances make an unrestricted trace needlessly enormous.` Design intent: 128 MAC instances make an unrestricted trace needlessly enormous. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 261: `// A bounded, purpose-built waveform test generates the review VCD.` Design intent: A bounded, purpose-built waveform test generates the review VCD. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 263: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 264: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 265: `repeat (2) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 267: `for (int case_index = 0; case_index < CASES; case_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 268: `run_case(case_index);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are run_case: the value named 'run case' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 269: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 271: `// Synchronous reset recovery is checked after all numerical cases so it cannot` Design intent: Synchronous reset recovery is checked after all numerical cases so it cannot This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 272: `// conceal a fixture mismatch.` Design intent: conceal a fixture mismatch. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

Line 273: `compute_mode = 2'd0;` This makes compute_mode, the value named 'compute mode' in this block, equal to '2'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 274: `active_rows = 4'd1;` This makes active_rows, one output-row position processed by the tensor tile, equal to '4'd1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d1: the value named 'd1' in this block. Context: the module's structural declarations.

Line 275: `active_columns = 6'd1;` This makes active_columns, one output-feature position processed in parallel, equal to '6'd1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d1: the value named 'd1' in this block. Context: the module's structural declarations.

Line 276: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 277: `start = 1'b1;` This makes start, the value named 'start' in this block, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 278: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 279: `start = 1'b0;` This makes start, the value named 'start' in this block, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 280: `if (!busy) begin` This condition selects the following hardware branch only when '!busy' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are busy: the value named 'busy' in this block. Context: the module's structural declarations.

Line 281: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 282: `$display("ERROR reset-recovery setup did not enter busy state");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 283: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 284: `rst_n = 1'b0;` This makes rst_n, the active-low synchronous reset, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 285: `repeat (2) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 286: `if (busy || result_valid || issue_ready) begin` This condition selects the following hardware branch only when 'busy || result_valid || issue_ready' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are busy: the value named 'busy' in this block; result_valid: the producer's statement that its current payload is meaningful; issue_ready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 287: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 288: `$display("ERROR external reset did not return tile to idle outputs");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 289: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 290: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 292: `if (issue_pointer != ISSUES || expected_pointer != RESULTS || checked != RESULTS) begin` This condition selects the following hardware branch only when 'issue_pointer != ISSUES || expected_pointer != RESULTS || checked != RESULTS' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are issue_pointer: the value named 'issue pointer' in this block; ISSUES: the value named 'issues' in this block; expected_pointer: the value named 'expected pointer' in this block. Context: the module's structural declarations.

Line 293: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 294: `$display("ERROR fixture accounting issue_ptr=%0d/%0d result_ptr=%0d/%0d checked=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 295: `issue_pointer, ISSUES, expected_pointer, RESULTS, checked);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are issue_pointer: the value named 'issue pointer' in this block; ISSUES: the value named 'issues' in this block; expected_pointer: the value named 'expected pointer' in this block.

Line 296: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 297: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 298: `$display("tb_neuraedge_tensor_tile FAILED errors=%0d checked=%0d", errors, checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 299: `$fatal(1);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 300: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 301: `$display("tb_neuraedge_tensor_tile PASSED cases=%0d checked=%0d", CASES, checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 302: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 303: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 305: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 306: `#100_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 307: `$display("ERROR tb_neuraedge_tensor_tile watchdog timeout state=%0d busy=%0d issue_ready=%0d result_valid=%...` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 308: `dut.state, busy, issue_ready, result_valid, result_ready,` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; state: the finite-state-machine phase retained in flip-flops; busy: the value named 'busy' in this block.

Line 309: `debug_accepted_issues, debug_retired_issues,` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are debug_accepted_issues: the value named 'debug accepted issues' in this block; debug_retired_issues: the value named 'debug retired issues' in this block.

Line 310: `dut.mac_issue_count, dut.mac_retire_count, dut.target_mac_retires);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation. The important operands are dut: the value named 'dut' in this block; mac_issue_count: an integer count retained for control or measurable telemetry; mac_retire_count: an integer count retained for control or measurable telemetry.

Line 311: `$fatal(1);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 312: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 313: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 315: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge tensor tile RTL block in simulation.

59 tb/tb_neuraedge_universal_core.sv

Role in the complete system: drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

59.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Simulation-only status tb_neuraedge_universal_core is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

59.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3  `include "tb/fixtures/operator_fixture_params.svh"
4
5  module tb_neuraedge_universal_core;
6      import neuraedge_command_pkg::*;
7
8      logic clk = 1'b0;
9      logic rst_n = 1'b0;
10     logic command_valid = 1'b0;
11     logic command_ready;
12     logic [127:0] command_data = 128'd0;
13     logic command_done;
14     logic status_busy;
15     logic status_error;
16     logic [3:0] status_active_opcode;
17     logic [31:0] status_command_count;
18     logic [31:0] status_input_beats;
19     logic [31:0] status_output_beats;
20     logic s_axis_tvalid = 1'b0;
21     logic s_axis_tready;
22     logic [63:0] s_axis_tdata = 64'd0;
23     logic s_axis_tlast = 1'b0;
24     logic m_axis_tvalid;
25     logic m_axis_tready = 1'b0;
26     logic [63:0] m_axis_tdata;
27     logic m_axis_tlast;
28
29     logic [191:0] vector_command_mem [0:`NEURAEDGE_VECTOR_CASES-1];
30     logic [255:0] vector_a_mem [0:`NEURAEDGE_VECTOR_CASES-1];
31     logic [255:0] vector_b_mem [0:`NEURAEDGE_VECTOR_CASES-1];
32     logic [255:0] vector_expected_mem [0:`NEURAEDGE_VECTOR_CASES-1];
33     logic [63:0] optimizer_parameter_mem [0:`NEURAEDGE_OPTIMIZER_CASES-1];
34     logic [127:0] optimizer_weight_mem [0:`NEURAEDGE_OPTIMIZER_CASES-1];
35     logic [191:0] optimizer_gradient_mem [0:`NEURAEDGE_OPTIMIZER_CASES-1];
36     logic [191:0] optimizer_velocity_mem [0:`NEURAEDGE_OPTIMIZER_CASES-1];
37     logic [127:0] optimizer_expected_weight_mem [0:`NEURAEDGE_OPTIMIZER_CASES-1];
38     logic [191:0] optimizer_expected_velocity_mem [0:`NEURAEDGE_OPTIMIZER_CASES-1];
39     logic signed [23:0] ln_value_mem [0:`NEURAEDGE_LAYERNORM_SAMPLES-1];
40     logic signed [23:0] ln_gamma_mem [0:`NEURAEDGE_LAYERNORM_SAMPLES-1];
41     logic signed [23:0] ln_beta_mem [0:`NEURAEDGE_LAYERNORM_SAMPLES-1];
42     logic signed [31:0] ln_expected_mem [0:`NEURAEDGE_LAYERNORM_SAMPLES-1];
43     logic signed [23:0] sm_score_mem [0:`NEURAEDGE_SOFTMAX_SAMPLES-1];
44     logic [15:0] sm_expected_mem [0:`NEURAEDGE_SOFTMAX_SAMPLES-1];
45     integer train_expected_dx [0:1][0:7];
46     integer train_expected_dw [0:2][0:7];
47     logic [31:0] debug_training_tiles;
48     logic [31:0] debug_training_cycles;
49
50     int errors = 0;
51     int commands_expected = 0;
52
53     neuraedge_universal_core dut (
54         .clk(clk), .rst_n(rst_n),
55         .command_valid(command_valid), .command_ready(command_ready),
56         .command_data(command_data), .command_done(command_done),
57         .status_busy(status_busy), .status_error(status_error),
58         .status_active_opcode(status_active_opcode),
59         .status_command_count(status_command_count),
60         .status_input_beats(status_input_beats),
```

```
        .status_output_beats(status_output_beats),
        .debug_training_tiles(debug_training_tiles),
        .debug_training_cycles(debug_training_cycles),
        .s_axis_tvalid(s_axis_tvalid), .s_axis_tready(s_axis_tready),
        .s_axis_tdata(s_axis_tdata), .s_axis_tlast(s_axis_tlast),
        .m_axis_tvalid(m_axis_tvalid), .m_axis_tready(m_axis_tready),
        .m_axis_tdata(m_axis_tdata), .m_axis_tlast(m_axis_tlast)
    );

    always #2.5 clk = ~clk;

    task automatic submit_command(input logic [127:0] value);
        begin
            while (!command_ready) @(negedge clk);
            @(negedge clk);
            command_data = value;
            command_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            command_valid = 1'b0;
            command_data = 128'd0;
            commands_expected++;
        end
    endtask

    task automatic send_word(input logic [63:0] value, input logic last_value);
        begin
            while (!s_axis_tready) @(negedge clk);
            s_axis_tdata = value;
            s_axis_tlast = last_value;
            s_axis_tvalid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            s_axis_tvalid = 1'b0;
            s_axis_tlast = 1'b0;
            s_axis_tdata = 64'd0;
        end
    endtask

    task automatic receive_word(
        output logic [63:0] value,
        output logic last_value,
        input int stall_cycles
    );
        begin
            while (!m_axis_tvalid) @(negedge clk);
            repeat (stall_cycles) @(negedge clk);
            value = m_axis_tdata;
            last_value = m_axis_tlast;
            m_axis_tready = 1'b1;
            @(posedge clk);
            @(negedge clk);
            m_axis_tready = 1'b0;
        end
    endtask

    task automatic wait_done;
        int timeout;
        begin
            timeout = 0;
            while (!command_done && timeout < 100000) begin
                @(negedge clk);
                timeout++;
            end
            if (!command_done) begin
                errors++;
                $display("ERROR universal command timeout opcode=%0d state=%0d",
                         status_active_opcode, dut.state);
            end
        end
    endtask

    function automatic logic [127:0] tensor_command(
        input logic [3:0] opcode,
        input int rows,
        input int columns,
        input int count,
        input int activation_zp,
        input int weight_zp
    );
        logic [127:0] value;
        begin
            value = 128'd0;
            value[3:0] = opcode;
            value[7:4] = 4'(rows);
            value[13:8] = 6'(columns);
            value[25:14] = 12'(count);
```

```
            value[33:26] = 8'(activation_zp);
            value[41:34] = 8'(weight_zp);
            tensor_command = value;
        end
    endfunction

    initial begin
        logic [127:0] command;
        logic [63:0] received_word;
        logic received_last;
        logic [255:0] received_vector;
        logic [127:0] received_weights;
        logic [191:0] received_velocities;
        logic [767:0] received_lif_trace;
        logic [511:0] received_train_dx;
        logic [575:0] received_train_dw;
        logic [255:0] alpha_query;
        logic [255:0] alpha_key;
        int alpha_spikes;
        int alpha_zeros;
        int alpha_expected;
        int seed;

        $readmemh("tb/fixtures/vector_commands.mem", vector_command_mem);
        $readmemh("tb/fixtures/vector_a.mem", vector_a_mem);
        $readmemh("tb/fixtures/vector_b.mem", vector_b_mem);
        $readmemh("tb/fixtures/vector_expected.mem", vector_expected_mem);
        $readmemh("tb/fixtures/optimizer_parameters.mem", optimizer_parameter_mem);
        $readmemh("tb/fixtures/optimizer_weights.mem", optimizer_weight_mem);
        $readmemh("tb/fixtures/optimizer_gradients.mem", optimizer_gradient_mem);
        $readmemh("tb/fixtures/optimizer_velocities.mem", optimizer_velocity_mem);
        $readmemh("tb/fixtures/optimizer_expected_weights.mem", optimizer_expected_weight_mem);
        $readmemh("tb/fixtures/optimizer_expected_velocities.mem", optimizer_expected_velocity_mem);
        $readmemh("tb/fixtures/layernorm_values.mem", ln_value_mem);
        $readmemh("tb/fixtures/layernorm_gammas.mem", ln_gamma_mem);
        $readmemh("tb/fixtures/layernorm_betas.mem", ln_beta_mem);
        $readmemh("tb/fixtures/layernorm_expected.mem", ln_expected_mem);
        $readmemh("tb/fixtures/softmax_scores.mem", sm_score_mem);
        $readmemh("tb/fixtures/softmax_expected.mem", sm_expected_mem);

        seed = 32'hc01ec7ed;
        repeat (5) @(negedge clk);
        rst_n = 1'b1;

        // Clear every sticky status source, including the nested tensor core.
        submit_command({124'd0, CMD_CLEAR_STATUS});
        wait_done();

        // Empty centered tensor tile: compaction emits one sentinel issue and
        // the exact corrected result remains zero.
        submit_command(tensor_command(CMD_LOAD_ACTIVATIONS, 0, 0, 1, 128, 0));
        send_word(64'h8080808080808080, 1'b1);
        wait_done();
        submit_command(tensor_command(CMD_LOAD_WEIGHTS_OUTER, 0, 0, 1, 0, 0));
        send_word(64'h0102030405060708, 1'b0);
        send_word(64'h1112131415161718, 1'b1);
        wait_done();
        submit_command(tensor_command(CMD_COMPUTE_DENSE_OUTER, 1, 1, 1, 128, 0));
        receive_word(received_word, received_last, 2);
        if (!received_last || $signed(received_word[47:0]) != 0) begin
            errors++;
            $display("ERROR universal tensor result=%0d last=%0d", $signed(received_word[47:0]), received_last);
        end

        // Alpha-XNOR command and eight-beat binary vector transfer.
        for (int word_index = 0; word_index < 8; word_index++) begin
            alpha_query[word_index*32 +: 32] = $urandom(seed);
            alpha_key[word_index*32 +: 32] = $urandom(seed);
        end
        alpha_spikes = 0;
        alpha_zeros = 0;
        for (int bit_index = 0; bit_index < 256; bit_index++) begin
            alpha_spikes += alpha_query[bit_index] && alpha_key[bit_index];
            alpha_zeros += !alpha_query[bit_index] && !alpha_key[bit_index];
        end
        alpha_expected = alpha_spikes * 256 + alpha_zeros * 127;
        command = 128'd0;
        command[3:0] = CMD_ALPHA_XNOR;
        command[50:42] = 9'd256;
        command[59:51] = 9'd127;
        submit_command(command);
        for (int beat = 0; beat < 4; beat++) send_word(alpha_query[beat*64 +: 64], 1'b0);
        for (int beat = 0; beat < 4; beat++) send_word(alpha_key[beat*64 +: 64], beat == 3);
        receive_word(received_word, received_last, 3);
        if (!received_last || received_word[17:0] != alpha_expected
                || received_word[26:18] != alpha_spikes || received_word[35:27] != alpha_zeros) begin
            errors++;
```

```
            $display("ERROR universal alpha-XNOR result");
        end

        // Vector command uses fixture case 50 (REQUANTIZE) through the shared stream.
        command = 128'd0;
        command[3:0] = CMD_VECTOR_ALU;
        command[108:4] = vector_command_mem[50][104:0];
        submit_command(command);
        send_word({vector_command_mem[50][168:137], vector_command_mem[50][136:105]}, 1'b0);
        for (int beat = 0; beat < 4; beat++) send_word(vector_a_mem[50][beat*64 +: 64], 1'b0);
        for (int beat = 0; beat < 4; beat++) send_word(vector_b_mem[50][beat*64 +: 64], beat == 3);
        for (int beat = 0; beat < 4; beat++) begin
            receive_word(received_word, received_last, beat % 2);
            received_vector[beat*64 +: 64] = received_word;
            if (received_last != (beat == 3)) errors++;
        end
        if (received_vector !== vector_expected_mem[50]) begin
            errors++;
            $display("ERROR universal vector result");
        end

        // LayerNorm fixture case 2 starts after the first five samples.
        command = 128'd0;
        command[3:0] = CMD_LAYERNORM;
        command[15:4] = 12'd7;
        command[47:16] = 32'd7;
        submit_command(command);
        for (int sample = 0; sample < 7; sample++) begin
            logic [63:0] first_word;
            first_word = {ln_beta_mem[5+sample][15:0], ln_gamma_mem[5+sample], ln_value_mem[5+sample]};
            send_word(first_word, 1'b0);
            send_word({56'd0, ln_beta_mem[5+sample][23:16]}, sample == 6);
        end
        for (int sample = 0; sample < 7; sample++) begin
            receive_word(received_word, received_last, sample % 2);
            if ($signed(received_word[31:0]) !== ln_expected_mem[5+sample]
                    || received_last != (sample == 6)) begin
                errors++;
                $display("ERROR universal LayerNorm sample=%0d got=%0d expected=%0d",
                         sample, $signed(received_word[31:0]), ln_expected_mem[5+sample]);
            end
        end

        // Softmax fixture case 2 also starts after five entries.
        command = 128'd0;
        command[3:0] = CMD_LOG_SOFTMAX;
        command[12:4] = 9'd7;
        command[17:13] = 5'd2;
        submit_command(command);
        for (int sample = 0; sample < 7; sample++) begin
            send_word({40'd0, sm_score_mem[5+sample]}, sample == 6);
        end
        for (int sample = 0; sample < 7; sample++) begin
            receive_word(received_word, received_last, 1);
            if (received_word[15:0] !== sm_expected_mem[5+sample]
                    || received_last != (sample == 6)) begin
                errors++;
                $display("ERROR universal softmax sample=%0d", sample);
            end
        end

        // One state-clearing LIF timestep with analytically obvious currents.
        command = 128'd0;
        command[3:0] = CMD_LIF_STEP;
        command[8:4] = 5'd2;
        command[32:9] = 24'sd50;
        command[56:33] = 24'sd0;
        command[57] = 1'b1;
        submit_command(command);
        for (int beat = 0; beat < 8; beat++) begin
            logic [63:0] currents;
            currents = 64'd0;
            for (int lane = 0; lane < 4; lane++) begin
                currents[lane*16 +: 16] = 16'(5 * (beat*4 + lane));
            end
            send_word(currents, beat == 7);
        end
        receive_word(received_word, received_last, 2);
        if (!received_last || received_word[63:32] != 1
                || received_word[31:0] != 32'hfffffc00) begin
            errors++;
            $display("ERROR universal LIF got=%h", received_word);
        end

        // Trace mode returns the same metadata plus every pre-reset membrane.
        // These are the h[t] values required by surrogate-gradient BPTT.
        command[58] = 1'b1;
```

```
        submit_command(command);
        for (int beat = 0; beat < 8; beat++) begin
            logic [63:0] currents;
            currents = 64'd0;
            for (int lane = 0; lane < 4; lane++) begin
                currents[lane*16 +: 16] = 16'(5 * (beat*4 + lane));
            end
            send_word(currents, beat == 7);
        end
        receive_word(received_word, received_last, 1);
        if (received_last || received_word[63:32] != 1
                || received_word[31:0] != 32'hfffffc00) begin
            errors++;
            $display("ERROR universal traced LIF metadata got=%h last=%b",
                     received_word, received_last);
        end
        received_lif_trace = '0;
        for (int beat = 0; beat < 12; beat++) begin
            receive_word(received_word, received_last, beat % 3);
            received_lif_trace[beat*64 +: 64] = received_word;
            if (received_last != (beat == 11)) errors++;
        end
        for (int lane = 0; lane < 32; lane++) begin
            if ($signed(received_lif_trace[lane*24 +: 24]) != 5 * lane) begin
                errors++;
                if (errors <= 20) begin
                    $display("ERROR universal traced LIF lane=%0d got=%0d expected=%0d",
                             lane,
                             $signed(received_lif_trace[lane*24 +: 24]),
                             5 * lane);
                end
            end
        end

        // Full eight-lane on-chip SGDW update using fixture case zero.
        command = 128'd0;
        command[3:0] = CMD_SGD_UPDATE;
        command[21:4] = optimizer_parameter_mem[0][17:0];
        command[38:22] = optimizer_parameter_mem[0][34:18];
        command[55:39] = optimizer_parameter_mem[0][51:35];
        command[60:56] = optimizer_parameter_mem[0][56:52];
        submit_command(command);
        for (int lane = 0; lane < 8; lane++) begin
            send_word({optimizer_velocity_mem[0][lane*24 +: 24],
                       optimizer_gradient_mem[0][lane*24 +: 24],
                       optimizer_weight_mem[0][lane*16 +: 16]}, lane == 7);
        end
        for (int lane = 0; lane < 8; lane++) begin
            receive_word(received_word, received_last, lane % 3);
            received_weights[lane*16 +: 16] = received_word[15:0];
            received_velocities[lane*24 +: 24] = received_word[39:16];
            if (received_last != (lane == 7)) errors++;
        end
        if (received_weights !== optimizer_expected_weight_mem[0]
                || received_velocities !== optimizer_expected_velocity_mem[0]) begin
            errors++;
            $display("ERROR universal optimizer result");
        end

        // Backpropagate one complete Y=XW tile through the universal stream.
        // X, W, and dY are s16; dX is shifted once and dW is unshifted.
        command = 128'd0;
        command[3:0] = CMD_TRAIN_LINEAR_BACKWARD;
        command[7:4] = 4'd2;
        command[11:8] = 4'd3;
        command[15:12] = 4'd2;
        command[20:16] = 5'd1;
        command[25:21] = 5'd0;
        submit_command(command);
        send_word(64'h00000003fffe0001, 1'b0);
        send_word(64'h0000000000000000, 1'b0);
        send_word(64'h0000fffa0005fffc, 1'b0);
        send_word(64'h0000000000000000, 1'b0);
        send_word(64'h00000000fff80007, 1'b0);
        send_word(64'h0000000000000000, 1'b0);
        send_word(64'h000000000000a0009, 1'b0);
        send_word(64'h0000000000000000, 1'b0);
        send_word(64'h000000000000cfff5, 1'b0);
        send_word(64'h0000000000000000, 1'b0);
        send_word(64'h00000000fff2000d, 1'b0);
        send_word(64'h0000000000000000, 1'b0);
        send_word(64'h000000000010fff1, 1'b0);
        send_word(64'h0000000000000000, 1'b1);
        received_train_dx = '0;
        for (int beat = 0; beat < 8; beat++) begin
            receive_word(received_word, received_last, beat % 3);
            received_train_dx[beat*64 +: 64] = received_word;
```

```
            if (received_last) errors++;
        end
        received_train_dw = '0;
        for (int beat = 0; beat < 9; beat++) begin
            receive_word(received_word, received_last, (beat + 1) % 3);
            received_train_dw[beat*64 +: 64] = received_word;
            if (received_last != (beat == 8)) errors++;
        end
        train_expected_dx[0][0] = 102;
        train_expected_dx[0][1] = -12;
        train_expected_dx[0][2] = -156;
        train_expected_dx[1][0] = -117;
        train_expected_dx[1][1] = 13;
        train_expected_dx[1][2] = 179;
        for (int row = 0; row < 2; row++) begin
            for (int lane = 3; lane < 8; lane++) train_expected_dx[row][lane] = 0;
            for (int lane = 0; lane < 8; lane++) begin
                if ($signed(received_train_dx[(row*8+lane)*32 +: 32])
                        != train_expected_dx[row][lane]) begin
                    errors++;
                    $display("ERROR universal backward dX row=%0d lane=%0d got=%0d expected=%0d",
                             row, lane,
                             $signed(received_train_dx[(row*8+lane)*32 +: 32]),
                             train_expected_dx[row][lane]);
                end
            end
        end
        train_expected_dw[0][0] = 73;
        train_expected_dw[0][1] = -78;
        train_expected_dw[1][0] = -101;
        train_expected_dw[1][1] = 108;
        train_expected_dw[2][0] = 129;
        train_expected_dw[2][1] = -138;
        for (int row = 0; row < 3; row++) begin
            for (int lane = 2; lane < 8; lane++) train_expected_dw[row][lane] = 0;
            for (int lane = 0; lane < 8; lane++) begin
                if ($signed(received_train_dw[row*192+lane*24 +: 24])
                        != train_expected_dw[row][lane]) begin
                    errors++;
                    $display("ERROR universal backward dW row=%0d lane=%0d got=%0d expected=%0d",
                             row, lane,
                             $signed(received_train_dw[row*192+lane*24 +: 24]),
                             train_expected_dw[row][lane]);
                end
            end
        end
        wait_done();

        // Program and evaluate all eight ports of the activation LUT.
        command = 128'd0;
        command[3:0] = CMD_ACTIVATION_LUT;
        command[7:4] = 4'd0;
        submit_command(command);
        wait_done();
        command[7:4] = 4'd1;
        command[19:8] = 12'd8;
        submit_command(command);
        for (int address = 0; address < 8; address++) begin
            send_word({40'd0, 16'(address*address-3), 8'(address)}, address == 7);
        end
        wait_done();
        command = 128'd0;
        command[3:0] = CMD_ACTIVATION_LUT;
        command[7:4] = 4'd2;
        submit_command(command);
        send_word(64'h0706050403020100, 1'b1);
        received_vector = '0;
        for (int beat = 0; beat < 2; beat++) begin
            receive_word(received_word, received_last, beat);
            received_vector[beat*64 +: 64] = received_word;
        end
        for (int lane = 0; lane < 8; lane++) begin
            if ($signed(received_vector[lane*16 +: 16]) != lane*lane-3) begin
                errors++;
                $display("ERROR universal LUT lane=%0d", lane);
            end
        end
        wait_done();

        if (status_error || status_busy || status_command_count != commands_expected
                || status_input_beats != 88 || status_output_beats != 61
                || debug_training_tiles != 1 || debug_training_cycles == 0) begin
            errors++;
            $display("ERROR universal final status error=%0d busy=%0d commands=%0d/%0d inputs=%0d outputs=%0d training=%0d/%0d",
                     status_error, status_busy, status_command_count, commands_expected,
                     status_input_beats, status_output_beats,
                     debug_training_tiles, debug_training_cycles);
```

```
        end
        if (errors != 0) begin
            $display("tb_neuraedge_universal_core FAILED errors=%0d", errors);
            $finish;
        end else begin
            $display("tb_neuraedge_universal_core PASSED commands=%0d input_beats=%0d output_beats=%0d",
                     status_command_count, status_input_beats, status_output_beats);
            $finish;
        end
    end

    initial begin
        #2_000_000;
        $fatal(1, "tb_neuraedge_universal_core watchdog timeout state=%0d opcode=%0d",
               dut.state, status_active_opcode);
    end
endmodule

`default_nettype wire
```

59.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 3: `` `include "tb/fixtures/operator_fixture_params.svh" `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 5: `module tb_neuraedge_universal_core;` This opens tb_neuraedge_universal_core;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 6: `import neuraedge_command_pkg::*;` This imports shared package names into the current module so opcodes and mode constants have exactly one definition. The import changes compile-time name visibility; it does not create registers or consume a clock cycle.

Line 8: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic command_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic command_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic [127:0] command_data = 128'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_data is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic command_done;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_done is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic status_busy;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_busy is the value named 'status busy' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic status_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_error is the value named 'status error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic [3:0] status_active_opcode;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_active_opcode is the value named 'status active opcode' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic [31:0] status_command_count;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_command_count is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic [31:0] status_input_beats;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_input_beats is the value named 'status input beats' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic [31:0] status_output_beats;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, status_output_beats is the value named 'status output beats' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic s_axis_tvalid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic s_axis_tready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic [63:0] s_axis_tdata = 64'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tdata is the value named 's axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic s_axis_tlast = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, s_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic m_axis_tvalid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tvalid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic m_axis_tready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 26: `logic [63:0] m_axis_tdata;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tdata is the value named 'm axis tdata' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic m_axis_tlast;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, m_axis_tlast is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `logic [191:0] vector_command_mem [0:‘NEURAEDGE_VECTOR_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_command_mem is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic [255:0] vector_a_mem [0:‘NEURAEDGE_VECTOR_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_a_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `logic [255:0] vector_b_mem [0:‘NEURAEDGE_VECTOR_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_b_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 32: `logic [255:0] vector_expected_mem [0:‘NEURAEDGE_VECTOR_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, vector_expected_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 33: `logic [63:0] optimizer_parameter_mem [0:‘NEURAEDGE_OPTIMIZER_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_parameter_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic [127:0] optimizer_weight_mem [0:‘NEURAEDGE_OPTIMIZER_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_weight_mem is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 35: `logic [191:0] optimizer_gradient_mem [0:‘NEURAEDGE_OPTIMIZER_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_gradient_mem is the signed loss derivative used to update a learned parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `logic [191:0] optimizer_velocity_mem [0:‘NEURAEDGE_OPTIMIZER_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_velocity_mem is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 37: `logic [127:0] optimizer_expected_weight_mem [0:‘NEURAEDGE_OPTIMIZER_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_expected_weight_mem is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 38: `logic [191:0] optimizer_expected_velocity_mem [0:‘NEURAEDGE_OPTIMIZER_CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, optimizer_expected_velocity_mem is the momentum history retained by the SGD optimizer. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 39: `logic signed [23:0] ln_value_mem [0:‘NEURAEDGE_LAYERNORM_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_value_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 40: `logic signed [23:0] ln_gamma_mem [0:‘NEURAEDGE_LAYERNORM_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_gamma_mem is the learned multiplicative LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 41: `logic signed [23:0] ln_beta_mem [0:‘NEURAEDGE_LAYERNORM_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_beta_mem is the learned additive LayerNorm parameter. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 42: `logic signed [31:0] ln_expected_mem [0:‘NEURAEDGE_LAYERNORM_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, ln_expected_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 43: `logic signed [23:0] sm_score_mem [0:‘NEURAEDGE_SOFTMAX_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_score_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 44: `logic [15:0] sm_expected_mem [0:‘NEURAEDGE_SOFTMAX_SAMPLES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, sm_expected_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 45: `integer train_expected_dx [0:1][0:7];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_expected_dx is the value named 'train expected dx' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 46: `integer train_expected_dw [0:2][0:7];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, train_expected_dw is the value named 'train expected dw' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 47: `logic [31:0] debug_training_tiles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, debug_training_tiles is the value named 'debug training tiles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 48: `logic [31:0] debug_training_cycles;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, debug_training_cycles is the value named 'debug training cycles' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 50: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 51: `int commands_expected = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are commands_expected: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 53: `neuraedge_universal_core dut (` This instantiates module neuraedge_universal_core as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 54: `.clk(clk), .rst_n(rst_n),` This connects the child instance's named port 'clk' to parent expression 'clk), .rst_n(rst_n'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 55: `.command_valid(command_valid), .command_ready(command_ready),` This connects the child instance's named port 'command_valid' to parent expression 'command_valid), .command_ready(command_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 56: `.command_data(command_data), .command_done(command_done),` This connects the child instance's named port 'command_data' to parent expression 'command_data), .command_done(command_done'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 57: `.status_busy(status_busy), .status_error(status_error),` This connects the child instance's named port 'status_busy' to parent expression 'status_busy), .status_error(status_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status busy), .status error(status error' in this block.

Line 58: `.status_active_opcode(status_active_opcode),` This connects the child instance's named port 'status_active_opcode' to parent expression 'status_active_opcode'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status active opcode' in this block.

Line 59: `.status_command_count(status_command_count),` This connects the child instance's named port 'status_command_count' to parent expression 'status_command_count'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 60: `.status_input_beats(status_input_beats),` This connects the child instance's named port 'status_input_beats' to parent expression 'status_input_beats'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status input beats' in this block.

Line 61: `.status_output_beats(status_output_beats),` This connects the child instance's named port 'status_output_beats' to parent expression 'status_output_beats'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'status output beats' in this block.

Line 62: `.debug_training_tiles(debug_training_tiles),` This connects the child instance's named port 'debug_training_tiles' to parent expression 'debug_training_tiles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug training tiles' in this block.

Line 63: `.debug_training_cycles(debug_training_cycles),` This connects the child instance's named port 'debug_training_cycles' to parent expression 'debug_training_cycles'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'debug training cycles' in this block.

Line 64: `.s_axis_tvalid(s_axis_tvalid), .s_axis_tready(s_axis_tready),` This connects the child instance's named port ‘s_axis_tvalid‘ to parent expression ‘s_axis_tvalid), .s_axis_tready(s_axis_tready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 65: `.s_axis_tdata(s_axis_tdata), .s_axis_tlast(s_axis_tlast),` This connects the child instance's named port ‘s_axis_tdata‘ to parent expression ‘s_axis_tdata), .s_axis_tlast(s_axis_tlast‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 66: `.m_axis_tvalid(m_axis_tvalid), .m_axis_tready(m_axis_tready),` This connects the child instance's named port ‘m_axis_tvalid‘ to parent expression ‘m_axis_tvalid), .m_axis_tready(m_axis_tready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 67: `.m_axis_tdata(m_axis_tdata), .m_axis_tlast(m_axis_tlast)` This connects the child instance's named port ‘m_axis_tdata‘ to parent expression ‘m_axis_tdata), .m_axis_tlast(m_axis_tlast‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the AXI-Stream marker for the final beat of a packet or command payload.

Line 68: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 70: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 72: `task automatic submit_command(input logic [127:0] value);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 73: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task submit command; it has no arithmetic value of its own.

Line 74: `while (!command_ready) @(negedge clk);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are while: the value named 'while' in this block; command_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 75: `@(negedge clk);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 76: `command_data = value;` This makes command_data, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘value‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the simulation task submit command.

Line 77: `command_valid = 1'b1;` This makes command_valid, the producer's statement that its current payload is meaningful, equal to ‘1'b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task submit command.

Line 78: `@(posedge clk);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 79: `@(negedge clk);` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 80: `command_valid = 1'b0;` This makes command_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task submit command.

Line 81: `command_data = 128'd0;` This makes command_data, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task submit command.

Line 82: `commands_expected++;` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are commands_expected: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 83: `end` This structural keyword opens or closes the branch/process scope described as the simulation task submit command; it has no arithmetic value of its own.

Line 84: `endtask` Within the simulation task submit command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 86: `task automatic send_word(input logic [63:0] value, input logic last_value);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; send_word: the value named 'send word' in this block.

Line 87: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task send word; it has no arithmetic value of its own.

Line 88: `while (!s_axis_tready) @(negedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are while: the value named 'while' in this block; s_axis_tready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 89: `s_axis_tdata = value;` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to 'value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the simulation task send word.

Line 90: `s_axis_tlast = last_value;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'last_value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are last_value: the AXI-Stream marker for the final beat of a packet or command payload. Context: the simulation task send word.

Line 91: `s_axis_tvalid = 1'b1;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task send word.

Line 92: `@(posedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 93: `@(negedge clk);` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 94: `s_axis_tvalid = 1'b0;` This makes s_axis_tvalid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send word.

Line 95: `s_axis_tlast = 1'b0;` This makes s_axis_tlast, the AXI-Stream marker for the final beat of a packet or command payload, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task send word.

Line 96: `s_axis_tdata = 64'd0;` This makes s_axis_tdata, the value named 's axis tdata' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the simulation task send word.

Line 97: `end` This structural keyword opens or closes the branch/process scope described as the simulation task send word; it has no arithmetic value of its own.

Line 98: `endtask` Within the simulation task send word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 100: `task automatic receive_word(` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; receive_word: the value named 'receive word' in this block.

Line 101: `output logic [63:0] value,` This declares value as a output port with width [63:0]. It carries the value named 'value' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 102: `output logic last_value,` This declares last_value as a output port with width one bit. It carries the AXI-Stream marker for the final beat of a packet or command payload across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 103: `input int stall_cycles` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 104: `);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 105: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task receive word; it has no arithmetic value of its own.

Line 106: `while (!m_axis_tvalid) @(negedge clk);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are while: the value named 'while' in this block; m_axis_tvalid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 107: `repeat (stall_cycles) @(negedge clk);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; stall_cycles: the value named 'stall cycles' in this block; negedge: the value named 'negedge' in this block.

Line 108: `value = m_axis_tdata;` This makes value, the value named 'value' in this block, equal to 'm_axis_tdata'. It is driven by combinational or procedural logic in the current evaluation. The important operands are m_axis_tdata: the value named 'm axis tdata' in this block. Context: the simulation task receive word.

Line 109: `last_value = m_axis_tlast;` This makes last_value, the AXI-Stream marker for the final beat of a packet or command payload, equal to 'm_axis_tlast'. It is driven by combinational or procedural logic in the current evaluation. The important operands are m_axis_tlast: the AXI-Stream marker for the final beat of a packet or command payload. Context: the simulation task receive word.

Line 110: `m_axis_tready = 1'b1;` This makes m_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the simulation task receive word.

Line 111: `@(posedge clk);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 112: `@(negedge clk);` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 113: `m_axis_tready = 1'b0;` This makes m_axis_tready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the simulation task receive word.

Line 114: `end` This structural keyword opens or closes the branch/process scope described as the simulation task receive word; it has no arithmetic value of its own.

Line 115: `endtask` Within the simulation task receive word, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 117: `task automatic wait_done;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are task: the value named 'task' in this block; automatic: the value named 'automatic' in this block; wait_done: the value named 'wait done' in this block.

Line 118: `int timeout;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are timeout: the value named 'timeout' in this block.

Line 119: `begin` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 120: `timeout = 0;` This makes timeout, the value named 'timeout' in this block, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the simulation task task.

Line 121: `while (!command_done && timeout < 100000) begin` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are while: the value named 'while' in this block; command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; timeout: the value named 'timeout' in this block.

Line 122: `@(negedge clk);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 123: `timeout++;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are timeout: the value named 'timeout' in this block.

Line 124: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 125: `if (!command_done) begin` This condition selects the following hardware branch only when '!command_done' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are command_done: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the simulation task task.

Line 126: `errors++;` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 127: `$display("ERROR universal command timeout opcode=%0d state=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 128: `status_active_opcode, dut.state);` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are status_active_opcode: the value named 'status active opcode' in this block; dut: the value named 'dut' in this block; state: the finite-state-machine phase retained in flip-flops.

Line 129: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 130: `end` This structural keyword opens or closes the branch/process scope described as the simulation task task; it has no arithmetic value of its own.

Line 131: `endtask` Within the simulation task task, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are endtask: the value named 'endtask' in this block.

Line 133: `function automatic logic [127:0] tensor_command(` Within the combinational helper tensor command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are function: the value named 'function' in this block; automatic: the value named 'automatic' in this block; tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 134: `input logic [3:0] opcode,` This declares opcode as a input port with width [3:0]. It carries the value named 'opcode' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 135: `input int rows,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 136: `input int columns,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 137: `input int count,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 138: `input int activation_zp,` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 139: `input int weight_zp` This declares int as a input port with width one bit. It carries the value named 'int' in this block across the module boundary; a port is a bundle of physical FPGA nets, not a software argument.

Line 140: `);` Within the combinational helper tensor command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 141: `logic [127:0] value;` This declares hardware signal storage or wiring in the combinational helper tensor command. Specifically, value is the value named 'value' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 142: `begin` This structural keyword opens or closes the branch/process scope described as the combinational helper tensor command; it has no arithmetic value of its own.

Line 143: `value = 128'd0;` This makes value, the value named 'value' in this block, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the combinational helper tensor command.

Line 144: `value[3:0] = opcode;` This makes value[3:0], the value named 'value' in this block, equal to 'opcode'. It is driven by combinational or procedural logic in the current evaluation. The important operands are opcode: the value named 'opcode' in this block. Context: the combinational helper tensor command.

Line 145: `value[7:4] = 4'(rows);` This makes value[7:4], the value named 'value' in this block, equal to '4'(rows)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are rows: one output-row position processed by the tensor tile. Context: the combinational helper tensor command.

Line 146: `value[13:8] = 6'(columns);` This makes value[13:8], the value named 'value' in this block, equal to '6'(columns)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are columns: one output-feature position processed in parallel. Context: the combinational helper tensor command.

Line 147: `value[25:14] = 12'(count);` This makes value[25:14], the value named 'value' in this block, equal to '12'(count)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are count: the value named 'count' in this block. Context: the combinational helper tensor command.

Line 148: `value[33:26] = 8'(activation_zp);` This makes value[33:26], the value named 'value' in this block, equal to '8'(activation_zp)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are activation_zp: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the combinational helper tensor command.

Line 149: `value[41:34] = 8'(weight_zp);` This makes value[41:34], the value named 'value' in this block, equal to '8'(weight_zp)'. It is driven by combinational or procedural logic in the current evaluation. The important operands are weight_zp: a learned coefficient multiplied by an activation. Context: the combinational helper tensor command.

Line 150: `tensor_command = value;` This makes tensor_command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'value'. It is driven by combinational or procedural logic in the current evaluation. The important operands are value: the value named 'value' in this block. Context: the combinational helper tensor command.

Line 151: `end` This structural keyword opens or closes the branch/process scope described as the combinational helper tensor command; it has no arithmetic value of its own.

Line 152: `endfunction` Within the combinational helper tensor command, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are endfunction: the value named 'endfunction' in this block.

Line 154: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 155: `logic [127:0] command;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 156: `logic [63:0] received_word;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, received_word is the value named 'received word' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 157: `logic received_last;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, received_last is the AXI-Stream marker for the final beat of a packet or command payload. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 158: `logic [255:0] received_vector;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, received_vector is the value named 'received vector' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 159: `logic [127:0] received_weights;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, received_weights is a learned coefficient multiplied by an activation. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 160: `logic [191:0] received_velocities;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, received_velocities is the value named 'received velocities' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 161: `logic [767:0] received_lif_trace;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, received_lif_trace is the value named 'received lif trace' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 162: `logic [511:0] received_train_dx;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, received_train_dx is the value named 'received train dx' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 163: `logic [575:0] received_train_dw;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, received_train_dw is the value named 'received train dw' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 164: `logic [255:0] alpha_query;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_query is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 165: `logic [255:0] alpha_key;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, alpha_key is the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 166: `int alpha_spikes;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are alpha_spikes: a binary event produced when a neuron crosses its threshold.

Line 167: `int alpha_zeros;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are alpha_zeros: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 168: `int alpha_expected;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are alpha_expected: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 169: `int seed;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are seed: the value named 'seed' in this block.

Line 171: `$readmemh("tb/fixtures/vector_commands.mem", vector_command_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 172: `$readmemh("tb/fixtures/vector_a.mem", vector_a_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 173: `$readmemh("tb/fixtures/vector_b.mem", vector_b_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 174: `$readmemh("tb/fixtures/vector_expected.mem", vector_expected_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 175: `$readmemh("tb/fixtures/optimizer_parameters.mem", optimizer_parameter_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 176: `$readmemh("tb/fixtures/optimizer_weights.mem", optimizer_weight_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 177: `$readmemh("tb/fixtures/optimizer_gradients.mem", optimizer_gradient_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 178: `$readmemh("tb/fixtures/optimizer_velocities.mem", optimizer_velocity_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 179: `$readmemh("tb/fixtures/optimizer_expected_weights.mem", optimizer_expected_weight_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 180: `$readmemh("tb/fixtures/optimizer_expected_velocities.mem", optimizer_expected_velocity_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 181: `$readmemh("tb/fixtures/layernorm_values.mem", ln_value_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 182: `$readmemh("tb/fixtures/layernorm_gammas.mem", ln_gamma_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 183: `$readmemh("tb/fixtures/layernorm_betas.mem", ln_beta_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 184: `$readmemh("tb/fixtures/layernorm_expected.mem", ln_expected_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 185: `$readmemh("tb/fixtures/softmax_scores.mem", sm_score_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 186: `$readmemh("tb/fixtures/softmax_expected.mem", sm_expected_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 188: `seed = 32'hc01ec7ed;` This makes seed, the value named 'seed' in this block, equal to '32'hc01ec7ed'. It is driven by combinational or procedural logic in the current evaluation. The important operands are hc01ec7ed: the value named 'hc01ec7ed' in this block. Context: the module's structural declarations.

Line 189: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 190: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 192: `// Clear every sticky status source, including the nested tensor core.` Design intent: Clear every sticky status source, including the nested tensor core. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 193: `submit_command({124'd0, CMD_CLEAR_STATUS});` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; d0: the value named 'd0' in this block; CMD_CLEAR_STATUS: the value named 'cmd clear status' in this block.

Line 194: `wait_done();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are wait_done: the value named 'wait done' in this block.

Line 196: `// Empty centered tensor tile: compaction emits one sentinel issue and` Design intent: Empty centered tensor tile: compaction emits one sentinel issue and This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 197: `// the exact corrected result remains zero.` Design intent: the exact corrected result remains zero. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 198: `submit_command(tensor_command(CMD_LOAD_ACTIVATIONS, 0, 0, 1, 128, 0));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; CMD_LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 199: `send_word(64'h8080808080808080, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h8080808080808080: the value named 'h8080808080808080' in this block; b1: the value named 'b1' in this block.

Line 200: `wait_done();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are wait_done: the value named 'wait done' in this block.

Line 201: `submit_command(tensor_command(CMD_LOAD_WEIGHTS_OUTER, 0, 0, 1, 0, 0));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; CMD_LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation.

Line 202: `send_word(64'h0102030405060708, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0102030405060708: the value named 'h0102030405060708' in this block; b0: the value named 'b0' in this block.

Line 203: `send_word(64'h1112131415161718, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h1112131415161718: the value named 'h1112131415161718' in this block; b1: the value named 'b1' in this block.

Line 204: `wait_done();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are wait_done: the value named 'wait done' in this block.

Line 205: `submit_command(tensor_command(CMD_COMPUTE_DENSE_OUTER, 1, 1, 1, 128, 0));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; CMD_COMPUTE_DENSE_OUTER: the value named 'cmd compute dense outer' in this block.

Line 206: `receive_word(received_word, received_last, 2);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 207: `if (!received_last || $signed(received_word[47:0]) != 0) begin` This condition selects the following hardware branch only when '!received_last || $signed(received_word[47:0]) != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; received_word: the value named 'received word' in this block. Context: the module's structural declarations.

Line 208: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 209: `$display("ERROR universal tensor result=%0d last=%0d", $signed(received_word[47:0]), received_last)` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 210: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 212: `// Alpha-XNOR command and eight-beat binary vector transfer.` Design intent: Alpha-XNOR command and eight-beat binary vector transfer. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 213: `for (int word_index = 0; word_index < 8; word_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 214: `alpha_query[word_index*32 +: 32] = $urandom(seed);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are alpha_query: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; word_index: an integer selecting one tensor element, lane, bank, or state entry; urandom: the value named 'urandom' in this block.

Line 215: `alpha_key[word_index*32 +: 32] = $urandom(seed);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are alpha_key: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; word_index: an integer selecting one tensor element, lane, bank, or state entry; urandom: the value named 'urandom' in this block.

Line 216: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 217: `alpha_spikes = 0;` This makes alpha_spikes, a binary event produced when a neuron crosses its threshold, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 218: `alpha_zeros = 0;` This makes alpha_zeros, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to '0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 219: `for (int bit_index = 0; bit_index < 256; bit_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 220: `alpha_spikes += alpha_query[bit_index] && alpha_key[bit_index];` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are alpha_spikes: a binary event produced when a neuron crosses its threshold; alpha_query: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; bit_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 221: `alpha_zeros += !alpha_query[bit_index] && !alpha_key[bit_index];` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are alpha_zeros: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; alpha_query: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; bit_index: an integer selecting one tensor element, lane, bank, or state entry.

Line 222: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 223: `alpha_expected = alpha_spikes * 256 + alpha_zeros * 127;` This makes alpha_expected, the programmable weight assigned to matching zero spikes in alpha-XNOR attention, equal to 'alpha_spikes * 256 + alpha_zeros * 127'. It is driven by combinational or procedural logic in the current evaluation. The important operands are alpha_spikes: a binary event produced when a neuron crosses its threshold; alpha_zeros: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the module's structural declarations.

Line 224: `command = 128'd0;` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 225: `command[3:0] = CMD_ALPHA_XNOR;` This makes command[3:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘CMD_ALPHA_XNOR‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are CMD_ALPHA_XNOR: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Context: the module’s structural declarations.

Line 226: `command[50:42] = 9'd256;` This makes command[50:42], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘9’d256‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d256: the value named ’d256’ in this block. Context: the module’s structural declarations.

Line 227: `command[59:51] = 9'd127;` This makes command[59:51], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘9’d127‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d127: the value named ’d127’ in this block. Context: the module’s structural declarations.

Line 228: `submit_command(command);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 229: `for (int beat = 0; beat < 4; beat++) send_word(alpha_query[beat*64 +: 64], 1'b0);` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module’s structural declarations.

Line 230: `for (int beat = 0; beat < 4; beat++) send_word(alpha_key[beat*64 +: 64], beat == 3);` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module’s structural declarations.

Line 231: `receive_word(received_word, received_last, 3);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named ’receive word’ in this block; received_word: the value named ’received word’ in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 232: `if (!received_last || received_word[17:0] != alpha_expected` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; received_word: the value named ’received word’ in this block; alpha_expected: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 233: `|| received_word[26:18] != alpha_spikes || received_word[35:27] != alpha_zeros) begin` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_word: the value named ’received word’ in this block; alpha_spikes: a binary event produced when a neuron crosses its threshold; alpha_zeros: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 234: `errors++;` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named ’errors’ in this block.

Line 235: `$display("ERROR universal alpha-XNOR result");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 236: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 238: `// Vector command uses fixture case 50 (REQUANTIZE) through the shared stream.` Design intent: Vector command uses fixture case 50 (REQUANTIZE) through the shared stream. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 239: `command = 128'd0;` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 240: `command[3:0] = CMD_VECTOR_ALU;` This makes command[3:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'CMD_VECTOR_ALU'. It is driven by combinational or procedural logic in the current evaluation. The important operands are CMD_VECTOR_ALU: the value named 'cmd vector alu' in this block. Context: the module's structural declarations.

Line 241: `command[108:4] = vector_command_mem[50][104:0];` This makes command[108:4], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'vector_command_mem[50][104:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are vector_command_mem: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the module's structural declarations.

Line 242: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 243: `send_word({vector_command_mem[50][168:137], vector_command_mem[50][136:105]}, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; vector_command_mem: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; b0: the value named 'b0' in this block.

Line 244: `for (int beat = 0; beat < 4; beat++) send_word(vector_a_mem[50][beat*64 +:  64], 1'b0);` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 245: `for (int beat = 0; beat < 4; beat++) send_word(vector_b_mem[50][beat*64 +:  64], beat == 3);` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 246: `for (int beat = 0; beat < 4; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 247: `receive_word(received_word, received_last, beat % 2);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 248: `received_vector[beat*64 +: 64] = received_word;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_vector: the value named 'received vector' in this block; beat: the value named 'beat' in this block; received_word: the value named 'received word' in this block.

Line 249: `if (received_last != (beat == 3)) errors++;` This condition selects the following hardware branch only when 'received_last != (beat == 3)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; beat: the value named 'beat' in this block. Context: the module's structural declarations.

Line 250: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 251: `if (received_vector !== vector_expected_mem[50]) begin` This condition selects the following hardware branch only when 'received_vector !== vector_expected_mem[50]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_vector: the value named 'received vector' in this block; vector_expected_mem: local FPGA storage addressed by surrounding index logic. Context: the module's structural declarations.

Line 252: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 253: `$display("ERROR universal vector result");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 254: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 256: `// LayerNorm fixture case 2 starts after the first five samples.` Design intent: LayerNorm fixture case 2 starts after the first five samples. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 257: `command = 128'd0;` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 258: `command[3:0] = CMD_LAYERNORM;` This makes command[3:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'CMD_LAYERNORM'. It is driven by combinational or procedural logic in the current evaluation. The important operands are CMD_LAYERNORM: the value named 'cmd layernorm' in this block. Context: the module's structural declarations.

Line 259: `command[15:4] = 12'd7;` This makes command[15:4], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '12'd7'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d7: the value named 'd7' in this block. Context: the module's structural declarations.

Line 260: `command[47:16] = 32'd7;` This makes command[47:16], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '32'd7'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d7: the value named 'd7' in this block. Context: the module's structural declarations.

Line 261: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 262: `for (int sample = 0; sample < 7; sample++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 263: `logic [63:0] first_word;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, first_word is the value named 'first word' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 264: `first_word = {ln_beta_mem[5+sample][15:0], ln_gamma_mem[5+sample], ln_value_mem[5+sample]};` This makes first_word, the value named 'first word' in this block, equal to '{ln_beta_mem[5+sample][15:0], ln_gamma_mem[5+san ln_value_mem[5+sample]}'. It is driven by combinational or procedural logic in the current evaluation. The important operands are ln_beta_mem: the learned additive LayerNorm parameter; sample: the value named 'sample' in this block; ln_gamma_mem: the learned multiplicative LayerNorm parameter. Context: the module's structural declarations.

Line 265: `send_word(first_word, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; first_word: the value named 'first word' in this block; b0: the value named 'b0' in this block.

Line 266: `send_word({56'd0, ln_beta_mem[5+sample][23:16]}, sample == 6);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; d0: the value named 'd0' in this block; ln_beta_mem: the learned additive LayerNorm parameter.

Line 267: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 268: `for (int sample = 0; sample < 7; sample++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 269: `receive_word(received_word, received_last, sample % 2);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 270: `if ($signed(received_word[31:0]) !== ln_expected_mem[5+sample]` This condition selects the following hardware branch only when '$signed(received_word[31:0]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_word: the value named 'received word' in this block. Context: the module's structural declarations.

Line 271: `|| received_last != (sample == 6)) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; sample: the value named 'sample' in this block.

Line 272: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 273: `$display("ERROR universal LayerNorm sample=%0d got=%0d expected=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 274: `sample, $signed(received_word[31:0]), ln_expected_mem[5+sample]);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are sample: the value named 'sample' in this block; received_word: the value named 'received word' in this block; ln_expected_mem: local FPGA storage addressed by surrounding index logic.

Line 275: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 276: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 278: `// Softmax fixture case 2 also starts after five entries.` Design intent: Softmax fixture case 2 also starts after five entries. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 279: `command = 128'd0;` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 280: `command[3:0] = CMD_LOG_SOFTMAX;` This makes command[3:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'CMD_LOG_SOFTMAX'. It is driven by combinational or procedural logic in the current evaluation. The important operands are CMD_LOG_SOFTMAX: the value named 'cmd log softmax' in this block. Context: the module's structural declarations.

Line 281: `command[12:4] = 9'd7;` This makes command[12:4], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '9'd7'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d7: the value named 'd7' in this block. Context: the module's structural declarations.

Line 282: `command[17:13] = 5'd2;` This makes command[17:13], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '5'd2'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d2: the value named 'd2' in this block. Context: the module's structural declarations.

Line 283: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 284: `for (int sample = 0; sample < 7; sample++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 285: `send_word({40'd0, sm_score_mem[5+sample]}, sample == 6);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; d0: the value named 'd0' in this block; sm_score_mem: local FPGA storage addressed by surrounding index logic.

Line 286: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 287: `for (int sample = 0; sample < 7; sample++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 288: `receive_word(received_word, received_last, 1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 289: `if (received_word[15:0] !== sm_expected_mem[5+sample]` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_word: the value named 'received word' in this block; sm_expected_mem: local FPGA storage addressed by surrounding index logic; sample: the value named 'sample' in this block.

Line 290: `|| received_last != (sample == 6)) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; sample: the value named 'sample' in this block.

Line 291: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 292: `$display("ERROR universal softmax sample=%0d", sample);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 293: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 294: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 296: `// One state-clearing LIF timestep with analytically obvious currents.` Design intent: One state-clearing LIF timestep with analytically obvious currents. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 297: `command = 128'd0;` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 298: `command[3:0] = CMD_LIF_STEP;` This makes command[3:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'CMD_LIF_STEP'. It is driven by combinational or procedural logic in the current evaluation. The important operands are CMD_LIF_STEP: the value named 'cmd lif step' in this block. Context: the module's structural declarations.

Line 299: `command[8:4] = 5'd2;` This makes command[8:4], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '5'd2'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d2: the value named 'd2' in this block. Context: the module's structural declarations.

Line 300: `command[32:9] = 24'sd50;` This makes command[32:9], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '24'sd50'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd50: the value named 'sd50' in this block. Context: the module's structural declarations.

Line 301: `command[56:33] = 24'sd0;` This makes command[56:33], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '24'sd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are sd0: the value named 'sd0' in this block. Context: the module's structural declarations.

Line 302: `command[57] = 1'b1;` This makes command[57], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 303: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 304: `for (int beat = 0; beat < 8; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 305: `logic [63:0] currents;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, currents is the value named 'currents' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 306: `currents = 64'd0;` This makes currents, the value named 'currents' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 307: `for (int lane = 0; lane < 4; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 308: `currents[lane*16 +:  16] = 16'(5 * (beat*4 + lane));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are currents: the value named 'currents' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; beat: the value named 'beat' in this block.

Line 309: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 310: `send_word(currents, beat == 7);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; currents: the value named 'currents' in this block; beat: the value named 'beat' in this block.

Line 311: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 312: `receive_word(received_word, received_last, 2);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 313: `if (!received_last || received_word[63:32] != 1` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; received_word: the value named 'received word' in this block.

Line 314: `|| received_word[31:0] != 32'hfffffc00) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_word: the value named 'received word' in this block; hfffffc00: the value named 'hfffffc00' in this block.

Line 315: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 316: `$display("ERROR universal LIF got=%h", received_word);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 317: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 319: `// Trace mode returns the same metadata plus every pre-reset membrane.` Design intent: Trace mode returns the same metadata plus every pre-reset membrane. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 320: `// These are the h[t] values required by surrogate-gradient BPTT.` Design intent: These are the h[t] values required by surrogate-gradient BPTT. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 321: `command[58] = 1'b1;` This makes command[58], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 322: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 323: `for (int beat = 0; beat < 8; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 324: `logic [63:0] currents;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, currents is the value named 'currents' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 325: `currents = 64'd0;` This makes currents, the value named 'currents' in this block, equal to '64'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 326: `for (int lane = 0; lane < 4; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 327: `currents[lane*16 +:  16] = 16'(5 * (beat*4 + lane));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are currents: the value named 'currents' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; beat: the value named 'beat' in this block.

Line 328: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 329: `send_word(currents, beat == 7);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; currents: the value named 'currents' in this block; beat: the value named 'beat' in this block.

Line 330: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 331: `receive_word(received_word, received_last, 1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 332: `if (received_last || received_word[63:32] != 1` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; received_word: the value named 'received word' in this block.

Line 333: `|| received_word[31:0] != 32'hfffffc00) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_word: the value named 'received word' in this block; hfffffc00: the value named 'hfffffc00' in this block.

Line 334: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 335: `$display("ERROR universal traced LIF metadata got=%h last=%b",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 336: `received_word, received_last);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 337: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 338: `received_lif_trace = '0;` This makes received_lif_trace, the value named 'received lif trace' in this block, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 339: `for (int beat = 0; beat < 12; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 340: `receive_word(received_word, received_last, beat % 3);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 341: `received_lif_trace[beat*64 +: 64] = received_word;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_lif_trace: the value named 'received lif trace' in this block; beat: the value named 'beat' in this block; received_word: the value named 'received word' in this block.

Line 342: `if (received_last != (beat == 11)) errors++;` This condition selects the following hardware branch only when ‘received_last != (beat == 11)‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; beat: the value named 'beat' in this block. Context: the module's structural declarations.

Line 343: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 344: `for (int lane = 0; lane < 32; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 345: `if ($signed(received_lif_trace[lane*24 +: 24]) != 5 * lane) begin` This condition selects the following hardware branch only when ‘$signed(received_lif_trace[lane*24 +: 24]) != 5 * lane‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_lif_trace: the value named 'received lif trace' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the module's structural declarations.

Line 346: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 347: `if (errors <= 20) begin` This statement continues through source line 351; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR universal traced LIF lane=%0d got=%0d expected=%0d", lane, $signed(received_lif_trace[lane*24 +: 24]), 5 * lane‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the module's structural declarations.

Line 348: `$display("ERROR universal traced LIF lane=%0d got=%0d expected=%0d",` This is continuation line 2 of the 5-line logical statement that begins at source line 347. Its exact fragment is ‘$display("ERROR universal traced LIF lane=%0d got=%0d expected=%0d",‘. The important operands are display: the value named 'display' in this block; ERROR: the value named 'error' in this block; universal: the value named 'universal' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR universal traced LIF lane=%0d got=%0d expected=%0d", lane, $signed(received_lif_trace[lane*24 +: 24]), 5 * lane‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the module's structural declarations.

Line 349: `lane,` This is continuation line 3 of the 5-line logical statement that begins at source line 347. Its exact fragment is ‘lane,‘. The important operands are lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR universal traced LIF lane=%0d got=%0d expected=%0d", lane, $signed(received_lif_trace[lane*24 +: 24]), 5 * lane‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the module’s structural declarations.

Line 350: `$signed(received_lif_trace[lane*24 +: 24]),` This is continuation line 4 of the 5-line logical statement that begins at source line 347. Its exact fragment is ‘$signed(received_lif_trace[lane*24 +: 24]),‘. The important operands are received_lif_trace: the value named ’received lif trace’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR universal traced LIF lane=%0d got=%0d expected=%0d", lane, $signed(received_lif_trace[lane*24 +: 24]), 5 * lane‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the module’s structural declarations.

Line 351: `5 * lane);` This is continuation line 5 of the 5-line logical statement that begins at source line 347. Its exact fragment is ‘5 * lane);‘. The important operands are lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This condition selects the following hardware branch only when ‘errors <= 20) begin $display("ERROR universal traced LIF lane=%0d got=%0d expected=%0d", lane, $signed(received_lif_trace[lane*24 +: 24]), 5 * lane‘ is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named ’errors’ in this block; display: the value named ’display’ in this block; ERROR: the value named ’error’ in this block. Context: the module’s structural declarations.

Line 352: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 353: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 354: `end` This structural keyword opens or closes the branch/process scope described as the module’s structural declarations; it has no arithmetic value of its own.

Line 356: `// Full eight-lane on-chip SGDW update using fixture case zero.` Design intent: Full eight-lane on-chip SGDW update using fixture case zero. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 357: `command = 128’d0;` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘128’d0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named ’d0’ in this block. Context: the module’s structural declarations.

Line 358: `command[3:0] = CMD_SGD_UPDATE;` This makes command[3:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘CMD_SGD_UPDATE‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are CMD_SGD_UPDATE: the value named ’cmd sgd update’ in this block. Context: the module’s structural declarations.

Line 359: `command[21:4] = optimizer_parameter_mem[0][17:0];` This makes command[21:4], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘optimizer_parameter_mem[0][17:0]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are optimizer_parameter_mem: local FPGA storage addressed by surrounding index logic. Context: the module’s structural declarations.

Line 360: `command[38:22] = optimizer_parameter_mem[0][34:18];` This makes command[38:22], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'optimizer_parameter_mem[0][34:18]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are optimizer_parameter_mem: local FPGA storage addressed by surrounding index logic. Context: the module's structural declarations.

Line 361: `command[55:39] = optimizer_parameter_mem[0][51:35];` This makes command[55:39], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'optimizer_parameter_mem[0][51:35]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are optimizer_parameter_mem: local FPGA storage addressed by surrounding index logic. Context: the module's structural declarations.

Line 362: `command[60:56] = optimizer_parameter_mem[0][56:52];` This makes command[60:56], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'optimizer_parameter_mem[0][56:52]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are optimizer_parameter_mem: local FPGA storage addressed by surrounding index logic. Context: the module's structural declarations.

Line 363: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 364: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 365: `send_word({optimizer_velocity_mem[0][lane*24 +: 24],` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; optimizer_velocity_mem: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 366: `optimizer_gradient_mem[0][lane*24 +: 24],` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are optimizer_gradient_mem: the signed loss derivative used to update a learned parameter; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 367: `optimizer_weight_mem[0][lane*16 +: 16]}, lane == 7);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are optimizer_weight_mem: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 368: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 369: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 370: `receive_word(received_word, received_last, lane % 3);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 371: `received_weights[lane*16 +: 16] = received_word[15:0];` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_weights: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath; received_word: the value named 'received word' in this block.

Line 372: `received_velocities[lane*24 +: 24] = received_word[39:16];` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_velocities: the value named 'received velocities' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; received_word: the value named 'received word' in this block.

Line 373: `if (received_last != (lane == 7)) errors++;` This condition selects the following hardware branch only when 'received_last != (lane == 7)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the module's structural declarations.

Line 374: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 375: `if (received_weights !== optimizer_expected_weight_mem[0]` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_weights: a learned coefficient multiplied by an activation; optimizer_expected_weight_mem: a learned coefficient multiplied by an activation.

Line 376: `|| received_velocities !== optimizer_expected_velocity_mem[0]) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_velocities: the value named 'received velocities' in this block; optimizer_expected_velocity_mem: the momentum history retained by the SGD optimizer.

Line 377: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 378: `$display("ERROR universal optimizer result");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 379: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 381: `// Backpropagate one complete Y=XW tile through the universal stream.` Design intent: Backpropagate one complete Y=XW tile through the universal stream. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 382: `// X, W, and dY are s16; dX is shifted once and dW is unshifted.` Design intent: X, W, and dY are s16; dX is shifted once and dW is unshifted. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 383: `command = 128'd0;` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 384: `command[3:0] = CMD_TRAIN_LINEAR_BACKWARD;` This makes command[3:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘CMD_TRAIN_LINEAR_BACKWARD‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are CMD_TRAIN_LINEAR_BACKWARD: the value named 'cmd train linear backward' in this block. Context: the module's structural declarations.

Line 385: `command[7:4] = 4'd2;` This makes command[7:4], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘4'd2‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d2: the value named 'd2' in this block. Context: the module's structural declarations.

Line 386: `command[11:8] = 4'd3;` This makes command[11:8], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘4'd3‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d3: the value named 'd3' in this block. Context: the module's structural declarations.

Line 387: `command[15:12] = 4'd2;` This makes command[15:12], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘4'd2‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d2: the value named 'd2' in this block. Context: the module's structural declarations.

Line 388: `command[20:16] = 5'd1;` This makes command[20:16], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘5'd1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d1: the value named 'd1' in this block. Context: the module's structural declarations.

Line 389: `command[25:21] = 5'd0;` This makes command[25:21], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to ‘5'd0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 390: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 391: `send_word(64'h00000003fffe0001, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h00000003fffe0001: the value named 'h00000003fffe0001' in this block; b0: the value named 'b0' in this block.

Line 392: `send_word(64'h0000000000000000, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0000000000000000: the value named 'h0000000000000000' in this block; b0: the value named 'b0' in this block.

Line 393: `send_word(64'h0000fffa0005fffc, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0000fffa0005fffc: the value named 'h0000fffa0005fffc' in this block; b0: the value named 'b0' in this block.

Line 394: `send_word(64'h0000000000000000, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0000000000000000: the value named 'h0000000000000000' in this block; b0: the value named 'b0' in this block.

Line 395: `send_word(64'h00000000fff80007, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h00000000fff80007: the value named 'h00000000fff80007' in this block; b0: the value named 'b0' in this block.

Line 396: `send_word(64'h0000000000000000, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0000000000000000: the value named 'h0000000000000000' in this block; b0: the value named 'b0' in this block.

Line 397: `send_word(64'h00000000000a0009, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h00000000000a0009: the value named 'h00000000000a0009' in this block; b0: the value named 'b0' in this block.

Line 398: `send_word(64'h0000000000000000, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0000000000000000: the value named 'h0000000000000000' in this block; b0: the value named 'b0' in this block.

Line 399: `send_word(64'h00000000000cfff5, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h00000000000cfff5: the value named 'h00000000000cfff5' in this block; b0: the value named 'b0' in this block.

Line 400: `send_word(64'h0000000000000000, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0000000000000000: the value named 'h0000000000000000' in this block; b0: the value named 'b0' in this block.

Line 401: `send_word(64'h00000000fff2000d, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h00000000fff2000d: the value named 'h00000000fff2000d' in this block; b0: the value named 'b0' in this block.

Line 402: `send_word(64'h0000000000000000, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0000000000000000: the value named 'h0000000000000000' in this block; b0: the value named 'b0' in this block.

Line 403: `send_word(64'h000000000010fff1, 1'b0);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h000000000010fff1: the value named 'h000000000010fff1' in this block; b0: the value named 'b0' in this block.

Line 404: `send_word(64'h0000000000000000, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0000000000000000: the value named 'h0000000000000000' in this block; b1: the value named 'b1' in this block.

Line 405: `received_train_dx = '0;` This makes received_train_dx, the value named 'received train dx' in this block, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 406: `for (int beat = 0; beat < 8; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 407: `receive_word(received_word, received_last, beat % 3);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 408: `received_train_dx[beat*64 +: 64] = received_word;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_train_dx: the value named 'received train dx' in this block; beat: the value named 'beat' in this block; received_word: the value named 'received word' in this block.

Line 409: `if (received_last) errors++;` This condition selects the following hardware branch only when 'received_last' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload. Context: the module's structural declarations.

Line 410: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 411: `received_train_dw = '0;` This makes received_train_dw, the value named 'received train dw' in this block, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 412: `for (int beat = 0; beat < 9; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 413: `receive_word(received_word, received_last, (beat + 1) % 3);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 414: `received_train_dw[beat*64 +: 64] = received_word;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_train_dw: the value named 'received train dw' in this block; beat: the value named 'beat' in this block; received_word: the value named 'received word' in this block.

Line 415: `if (received_last != (beat == 8)) errors++;` This condition selects the following hardware branch only when 'received_last != (beat == 8)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_last: the AXI-Stream marker for the final beat of a packet or command payload; beat: the value named 'beat' in this block. Context: the module's structural declarations.

Line 416: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 417: `train_expected_dx[0][0] = 102;` This makes train_expected_dx[0][0], the value named 'train expected dx' in this block, equal to '102'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 418: `train_expected_dx[0][1] = -12;` This makes train_expected_dx[0][1], the value named 'train expected dx' in this block, equal to '-12'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 419: `train_expected_dx[0][2] = -156;` This makes train_expected_dx[0][2], the value named 'train expected dx' in this block, equal to '-156'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 420: `train_expected_dx[1][0] = -117;` This makes train_expected_dx[1][0], the value named 'train expected dx' in this block, equal to '-117'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 421: `train_expected_dx[1][1] = 13;` This makes train_expected_dx[1][1], the value named 'train expected dx' in this block, equal to '13'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 422: `train_expected_dx[1][2] = 179;` This makes train_expected_dx[1][2], the value named 'train expected dx' in this block, equal to '179'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 423: `for (int row = 0; row < 2; row++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 424: `for (int lane = 3; lane < 8; lane++) train_expected_dx[row][lane] = 0;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 425: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 426: `if ($signed(received_train_dx[(row*8+lane)*32 +: 32])` This condition selects the following hardware branch only when '$signed(received_train_dx[(row*8+lane)*32 +: 32]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_train_dx: the value named 'received train dx' in this block; row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the module's structural declarations.

Line 427: `!= train_expected_dx[row][lane]) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are train_expected_dx: the value named 'train expected dx' in this block; row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 428: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 429: `$display("ERROR universal backward dX row=%0d lane=%0d got=%0d expected=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 430: `row, lane,` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 431: `$signed(received_train_dx[(row*8+lane)*32 +: 32]),` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_train_dx: the value named 'received train dx' in this block; row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 432: `train_expected_dx[row][lane]);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are train_expected_dx: the value named 'train expected dx' in this block; row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 433: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 434: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 435: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 436: `train_expected_dw[0][0] = 73;` This makes train_expected_dw[0][0], the value named 'train expected dw' in this block, equal to '73'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 437: `train_expected_dw[0][1] = -78;` This makes train_expected_dw[0][1], the value named 'train expected dw' in this block, equal to '-78'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 438: `train_expected_dw[1][0] = -101;` This makes train_expected_dw[1][0], the value named 'train expected dw' in this block, equal to '-101'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 439: `train_expected_dw[1][1] = 108;` This makes train_expected_dw[1][1], the value named 'train expected dw' in this block, equal to '108'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 440: `train_expected_dw[2][0] = 129;` This makes train_expected_dw[2][0], the value named 'train expected dw' in this block, equal to '129'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 441: `train_expected_dw[2][1] = -138;` This makes train_expected_dw[2][1], the value named 'train expected dw' in this block, equal to '-138'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 442: `for (int row = 0; row < 3; row++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 443: `for (int lane = 2; lane < 8; lane++) train_expected_dw[row][lane] = 0;` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 444: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 445: `if ($signed(received_train_dw[row*192+lane*24 +: 24])` This condition selects the following hardware branch only when '$signed(received_train_dw[row*192+lane*24 +: 24]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_train_dw: the value named 'received train dw' in this block; row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the module's structural declarations.

Line 446: `!= train_expected_dw[row][lane]) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are train_expected_dw: the value named 'train expected dw' in this block; row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 447: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 448: `$display("ERROR universal backward dW row=%0d lane=%0d got=%0d expected=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 449: `row, lane,` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 450: `$signed(received_train_dw[row*192+lane*24 +: 24]),` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_train_dw: the value named 'received train dw' in this block; row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 451: `train_expected_dw[row][lane]);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are train_expected_dw: the value named 'train expected dw' in this block; row: one output-row position processed by the tensor tile; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 452: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 453: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 454: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 455: `wait_done();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are wait_done: the value named 'wait done' in this block.

Line 457: `// Program and evaluate all eight ports of the activation LUT.` Design intent: Program and evaluate all eight ports of the activation LUT. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

Line 458: `command = 128'd0;` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 459: `command[3:0] = CMD_ACTIVATION_LUT;` This makes command[3:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'CMD_ACTIVATION_LUT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are CMD_ACTIVATION_LUT: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the module's structural declarations.

Line 460: `command[7:4] = 4'd0;` This makes command[7:4], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '4'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 461: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 462: `wait_done();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are wait_done: the value named 'wait done' in this block.

Line 463: `command[7:4] = 4'd1;` This makes command[7:4], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '4'd1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d1: the value named 'd1' in this block. Context: the module's structural declarations.

Line 464: `command[19:8] = 12'd8;` This makes command[19:8], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '12'd8'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d8: the value named 'd8' in this block. Context: the module's structural declarations.

Line 465: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 466: `for (int address = 0; address < 8; address++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 467: `send_word({40'd0, 16'(address*address-3), 8'(address)}, address == 7);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; d0: the value named 'd0' in this block; address: the value named 'address' in this block.

Line 468: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 469: `wait_done();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are wait_done: the value named 'wait done' in this block.

Line 470: `command = 128'd0;` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '128'd0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d0: the value named 'd0' in this block. Context: the module's structural declarations.

Line 471: `command[3:0] = CMD_ACTIVATION_LUT;` This makes command[3:0], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'CMD_ACTIVATION_LUT'. It is driven by combinational or procedural logic in the current evaluation. The important operands are CMD_ACTIVATION_LUT: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the module's structural declarations.

Line 472: `command[7:4] = 4'd2;` This makes command[7:4], the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to '4'd2'. It is driven by combinational or procedural logic in the current evaluation. The important operands are d2: the value named 'd2' in this block. Context: the module's structural declarations.

Line 473: `submit_command(command);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are submit_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 474: `send_word(64'h0706050403020100, 1'b1);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are send_word: the value named 'send word' in this block; h0706050403020100: the value named 'h0706050403020100' in this block; b1: the value named 'b1' in this block.

Line 475: `received_vector = '0;` This makes received_vector, the value named 'received vector' in this block, equal to ''0'. It is driven by combinational or procedural logic in the current evaluation. The expression is a literal or structural operator and has no hidden software state. Context: the module's structural declarations.

Line 476: `for (int beat = 0; beat < 2; beat++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 477: `receive_word(received_word, received_last, beat);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are receive_word: the value named 'receive word' in this block; received_word: the value named 'received word' in this block; received_last: the AXI-Stream marker for the final beat of a packet or command payload.

Line 478: `received_vector[beat*64 +: 64] = received_word;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are received_vector: the value named 'received vector' in this block; beat: the value named 'beat' in this block; received_word: the value named 'received word' in this block.

Line 479: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 480: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 481: `if ($signed(received_vector[lane*16 +: 16]) != lane*lane-3) begin` This condition selects the following hardware branch only when '$signed(received_vector[lane*16 +: 16]) != lane*lane-3' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are received_vector: the value named 'received vector' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the module's structural declarations.

Line 482: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 483: `$display("ERROR universal LUT lane=%0d", lane);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 484: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 485: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 486: `wait_done();` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are wait_done: the value named 'wait done' in this block.

Line 488: `if (status_error || status_busy || status_command_count != commands_expected` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are status_error: the value named 'status error' in this block; status_busy: the value named 'status busy' in this block; status_command_count: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 489: `|| status_input_beats != 88 || status_output_beats != 61` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are status_input_beats: the value named 'status input beats' in this block; status_output_beats: the value named 'status output beats' in this block.

Line 490: `|| debug_training_tiles != 1 || debug_training_cycles == 0) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are debug_training_tiles: the value named 'debug training tiles' in this block; debug_training_cycles: the value named 'debug training cycles' in this block.

Line 491: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 492: `$display("ERROR universal final status error=%0d busy=%0d commands=%0d/%0d inputs=%0d outputs=%0d training=...` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 493: `status_error, status_busy, status_command_count, commands_expected,` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are status_error: the value named 'status error' in this block; status_busy: the value named 'status busy' in this block; status_command_count: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 494: `status_input_beats, status_output_beats,` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are status_input_beats: the value named 'status input beats' in this block; status_output_beats: the value named 'status output beats' in this block.

Line 495: `debug_training_tiles, debug_training_cycles);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are debug_training_tiles: the value named 'debug training tiles' in this block; debug_training_cycles: the value named 'debug training cycles' in this block.

Line 496: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 497: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 498: `$display("tb_neuraedge_universal_core FAILED errors=%0d", errors);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 499: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 500: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 501: `$display("tb_neuraedge_universal_core PASSED commands=%0d input_beats=%0d output_beats=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 502: `status_command_count, status_input_beats, status_output_beats);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are status_command_count: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; status_input_beats: the value named 'status input beats' in this block; status_output_beats: the value named 'status output beats' in this block.

Line 503: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 504: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 505: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 507: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 508: `#2_000_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 509: `$fatal(1, "tb_neuraedge_universal_core watchdog timeout state=%0d opcode=%0d",` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 510: `dut.state, status_active_opcode);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation. The important operands are dut: the value named 'dut' in this block; state: the finite-state-machine phase retained in flip-flops; status_active_opcode: the value named 'status active opcode' in this block.

Line 511: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 512: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 514: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge universal core RTL block in simulation.

60 tb/tb_neuraedge_vector_alu.sv

Role in the complete system: drives, backpressures, and checks the neuraedge vector alu RTL block in simulation.

60.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to drives, backpressures, and checks the neuraedge vector alu RTL block in simulation.

Simulation-only status tb_neuraedge_vector_alu is not synthesized into the FPGA. It generates clocks, reset, commands, stalls, and expected values around the design under test, then terminates with an explicit result marker.

Evidence produced The file contains approximately 3 explicit assertions, failure diagnostics, or pass/fail markers. Its VCD/WDB trace is useful evidence only when the final pass marker is present; a simulator exit code by itself is insufficient.

60.2 Complete source

```
1  `timescale 1ns / 1ps
2  `default_nettype none
3  `include "tb/fixtures/operator_fixture_params.svh"
4
5  module tb_neuraedge_vector_alu;
6      localparam int CASES = `NEURAEDGE_VECTOR_CASES;
7
8      logic clk = 1'b0;
9      logic rst_n = 1'b0;
10     logic input_valid = 1'b0;
11     logic input_ready;
12     logic [3:0] operation = 4'd0;
13     logic [255:0] operand_a = '0;
14     logic [255:0] operand_b = '0;
15     logic signed [15:0] multiplier_q15 = 16'sd0;
16     logic [4:0] right_shift = 5'd0;
17     logic signed [15:0] output_zero_point = 16'sd0;
18     logic signed [31:0] threshold = 32'sd0;
19     logic [31:0] surrogate_window = 32'd0;
20     logic signed [31:0] clamp_minimum = 32'sh80000000;
21     logic signed [31:0] clamp_maximum = 32'sh7fffffff;
22     logic output_valid;
23     logic output_ready = 1'b0;
24     logic [255:0] result;
25     logic config_error;
26
27     logic [191:0] command_mem [0:CASES-1];
28     logic [255:0] operand_a_mem [0:CASES-1];
29     logic [255:0] operand_b_mem [0:CASES-1];
30     logic [255:0] expected_mem [0:CASES-1];
31     int errors = 0;
32     int checked = 0;
33     logic held_valid = 1'b0;
34     logic [255:0] held_result;
35
36     neuraedge_vector_alu dut (
37         .clk(clk),
38         .rst_n(rst_n),
39         .input_valid(input_valid),
40         .input_ready(input_ready),
41         .operation(operation),
42         .operand_a(operand_a),
43         .operand_b(operand_b),
44         .multiplier_q15(multiplier_q15),
45         .right_shift(right_shift),
46         .output_zero_point(output_zero_point),
47         .threshold(threshold),
48         .surrogate_window(surrogate_window),
49         .clamp_minimum(clamp_minimum),
50         .clamp_maximum(clamp_maximum),
51         .output_valid(output_valid),
52         .output_ready(output_ready),
53         .result(result),
54         .config_error(config_error)
55     );
56
57     always #2.5 clk = ~clk;
58
59     always @(posedge clk) begin
60         if (held_valid && (!output_valid || result !== held_result)) begin
```

```
            errors++;
            $display("ERROR vector ALU output changed while stalled");
        end
        held_valid <= output_valid && !output_ready;
        held_result <= result;
    end

    initial begin
        $readmemh("tb/fixtures/vector_commands.mem", command_mem);
        $readmemh("tb/fixtures/vector_a.mem", operand_a_mem);
        $readmemh("tb/fixtures/vector_b.mem", operand_b_mem);
        $readmemh("tb/fixtures/vector_expected.mem", expected_mem);

        repeat (5) @(negedge clk);
        rst_n = 1'b1;

        for (int case_index = 0; case_index < CASES; case_index++) begin
            logic [191:0] command;
            command = command_mem[case_index];
            while (!input_ready) @(negedge clk);
            @(negedge clk);
            operation = command[3:0];
            multiplier_q15 = command[19:4];
            right_shift = command[24:20];
            output_zero_point = command[40:25];
            threshold = command[72:41];
            surrogate_window = command[104:73];
            clamp_minimum = command[136:105];
            clamp_maximum = command[168:137];
            operand_a = operand_a_mem[case_index];
            operand_b = operand_b_mem[case_index];
            input_valid = 1'b1;
            @(posedge clk);
            @(negedge clk);
            input_valid = 1'b0;

            while (!output_valid) @(negedge clk);
            repeat (case_index % 5) @(negedge clk);
            if (!output_valid || config_error || result !== expected_mem[case_index]) begin
                errors++;
                if (errors <= 20) begin
                    $display("ERROR vector case=%0d op=%0d config=%0d", case_index, operation, config_error);
                    for (int lane = 0; lane < 8; lane++) begin
                        if (result[lane*32 +: 32] !== expected_mem[case_index][lane*32 +: 32]) begin
                            $display("  lane=%0d got=%0d expected=%0d", lane,
                                     $signed(result[lane*32 +: 32]),
                                     $signed(expected_mem[case_index][lane*32 +: 32]));
                        end
                    end
                end
            end
            checked++;
            output_ready = 1'b1;
            @(posedge clk);
            @(negedge clk);
            output_ready = 1'b0;
        end

        // Unsupported operations are visible as deterministic configuration errors.
        while (!input_ready) @(negedge clk);
        @(negedge clk);
        operation = 4'hf;
        input_valid = 1'b1;
        @(posedge clk);
        @(negedge clk);
        input_valid = 1'b0;
        while (!output_valid) @(negedge clk);
        if (!output_valid || !config_error || result != 0) begin
            errors++;
            $display("ERROR vector ALU invalid opcode handling");
        end
        output_ready = 1'b1;
        @(posedge clk);
        @(negedge clk);

        if (errors != 0) begin
            $display("tb_neuraedge_vector_alu FAILED errors=%0d checked=%0d", errors, checked);
            $finish;
        end else begin
            $display("tb_neuraedge_vector_alu PASSED checked=%0d", checked);
            $finish;
        end
    end

    initial begin
        #500_000;
        $fatal(1, "tb_neuraedge_vector_alu watchdog timeout");
```

```
148     end
149 endmodule
150
151 `default_nettype wire
```

60.3 Line annotations

Line 1: `` `timescale 1ns / 1ps `` This simulation directive selects nanoseconds as the time unit and picoseconds as the timestamp precision; it does not create FPGA logic.

Line 2: `` `default_nettype none `` This compiler directive forbids accidental implicit one-bit wires, turning misspelled signal names into errors instead of unintended hardware.

Line 3: `` `include "tb/fixtures/operator_fixture_params.svh" `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation.

Line 5: `module tb_neuraedge_vector_alu;` This opens tb_neuraedge_vector_alu;, an independently compiled hardware definition. Its ports and statements collectively describe circuitry that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation.

**Line 6: `localparam int CASES = `NEURAEDGE_VECTOR_CASES;`** This fixes CASES to `NEURAEDGE_VECTOR_CASES during elaboration. Vivado uses that constant to shape or simplify the generated circuit; CASES denotes the value named 'cases' in this block.

Line 8: `logic clk = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clk is the clock that makes all registers advance together. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 9: `logic rst_n = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, rst_n is the active-low synchronous reset. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 10: `logic input_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 11: `logic input_ready;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, input_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 12: `logic [3:0] operation = 4'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, operation is the value named 'operation' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 13: `logic [255:0] operand_a = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, operand_a is the value named 'operand a' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 14: `logic [255:0] operand_b = '0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, operand_b is the value named 'operand b' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 15: `logic signed [15:0] multiplier_q15 = 16'sd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, multiplier_q15 is the value named 'multiplier q15' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 16: `logic [4:0] right_shift = 5'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, right_shift is the value named 'right shift' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 17: `logic signed [15:0] output_zero_point = 16'sd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_zero_point is the integer code representing real numeric zero in affine quantization. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 18: `logic signed [31:0] threshold = 32'sd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, threshold is the membrane level at which a neuron emits a spike. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 19: `logic [31:0] surrogate_window = 32'd0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, surrogate_window is the value named 'surrogate window' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 20: `logic signed [31:0] clamp_minimum = 32'sh80000000;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clamp_minimum is the value named 'clamp minimum' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 21: `logic signed [31:0] clamp_maximum = 32'sh7fffffff;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, clamp_maximum is the value named 'clamp maximum' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 22: `logic output_valid;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 23: `logic output_ready = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, output_ready is the consumer's statement that it can accept a payload this cycle. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 24: `logic [255:0] result;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, result is the value named 'result' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 25: `logic config_error;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, config_error is the value named 'config error' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 27: `logic [191:0] command_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command_mem is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 28: `logic [255:0] operand_a_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, operand_a_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 29: `logic [255:0] operand_b_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, operand_b_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 30: `logic [255:0] expected_mem [0:CASES-1];` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, expected_mem is local FPGA storage addressed by surrounding index logic. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 31: `int errors = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 32: `int checked = 0;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 33: `logic held_valid = 1'b0;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_valid is the producer's statement that its current payload is meaningful. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 34: `logic [255:0] held_result;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, held_result is the value named 'held result' in this block. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 36: `neuraedge_vector_alu dut (` This instantiates module neuraedge_vector_alu as dut. Synthesis creates that subordinate circuit and connects its ports in the lines that follow.

Line 37: `.clk(clk),` This connects the child instance's named port ‘clk‘ to parent expression ‘clk‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the clock that makes all registers advance together.

Line 38: `.rst_n(rst_n),` This connects the child instance's named port ‘rst_n‘ to parent expression ‘rst_n‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the active-low synchronous reset.

Line 39: `.input_valid(input_valid),` This connects the child instance's named port ‘input_valid‘ to parent expression ‘input_valid‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 40: `.input_ready(input_ready),` This connects the child instance's named port ‘input_ready‘ to parent expression ‘input_ready‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 41: `.operation(operation),` This connects the child instance's named port ‘operation‘ to parent expression ‘operation‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'operation' in this block.

Line 42: `.operand_a(operand_a),` This connects the child instance's named port ‘operand_a‘ to parent expression ‘operand_a‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'operand a' in this block.

Line 43: `.operand_b(operand_b),` This connects the child instance's named port ‘operand_b‘ to parent expression ‘operand_b‘. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'operand b' in this block.

Line 44: `.multiplier_q15(multiplier_q15),` This connects the child instance's named port 'multiplier_q15' to parent expression 'multiplier_q15'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'multiplier q15' in this block.

Line 45: `.right_shift(right_shift),` This connects the child instance's named port 'right_shift' to parent expression 'right_shift'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'right shift' in this block.

Line 46: `.output_zero_point(output_zero_point),` This connects the child instance's named port 'output_zero_point' to parent expression 'output_zero_point'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the integer code representing real numeric zero in affine quantization.

Line 47: `.threshold(threshold),` This connects the child instance's named port 'threshold' to parent expression 'threshold'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the membrane level at which a neuron emits a spike.

Line 48: `.surrogate_window(surrogate_window),` This connects the child instance's named port 'surrogate_window' to parent expression 'surrogate_window'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'surrogate window' in this block.

Line 49: `.clamp_minimum(clamp_minimum),` This connects the child instance's named port 'clamp_minimum' to parent expression 'clamp_minimum'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'clamp minimum' in this block.

Line 50: `.clamp_maximum(clamp_maximum),` This connects the child instance's named port 'clamp_maximum' to parent expression 'clamp_maximum'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'clamp maximum' in this block.

Line 51: `.output_valid(output_valid),` This connects the child instance's named port 'output_valid' to parent expression 'output_valid'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the producer's statement that its current payload is meaningful.

Line 52: `.output_ready(output_ready),` This connects the child instance's named port 'output_ready' to parent expression 'output_ready'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the consumer's statement that it can accept a payload this cycle.

Line 53: `.result(result),` This connects the child instance's named port 'result' to parent expression 'result'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'result' in this block.

Line 54: `.config_error(config_error)` This connects the child instance's named port 'config_error' to parent expression 'config_error'. The connection is a physical net bundle after synthesis; direction is declared in the child module, and the value named 'config error' in this block.

Line 55: `);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 57: `always #2.5 clk = ~clk;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are always: the value named 'always' in this block; clk: the clock that makes all registers advance together.

Line 59: `always @(posedge clk) begin` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are always: the value named 'always' in this block; posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 60: `if (held_valid && (!output_valid || result !== held_result)) begin` This condition selects the following hardware branch only when 'held_valid && (!output_valid || result !== held_result)' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are held_valid: the producer's statement that its current payload is meaningful; output_valid: the producer's statement that its current payload is meaningful; result: the value named 'result' in this block. Context: the module's structural declarations.

Line 61: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 62: `$display("ERROR vector ALU output changed while stalled");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 63: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 64: `held_valid <= output_valid && !output_ready;` This makes held_valid, the producer's statement that its current payload is meaningful, equal to 'output_valid && !output_ready'. It updates after the active clock edge together with other nonblocking assignments. The important operands are output_valid: the producer's statement that its current payload is meaningful; output_ready: the consumer's statement that it can accept a payload this cycle. Context: the module's structural declarations.

Line 65: `held_result <= result;` This makes held_result, the value named 'held result' in this block, equal to 'result'. It updates after the active clock edge together with other nonblocking assignments. The important operands are result: the value named 'result' in this block. Context: the module's structural declarations.

Line 66: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 68: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 69: `$readmemh("tb/fixtures/vector_commands.mem", command_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 70: `$readmemh("tb/fixtures/vector_a.mem", operand_a_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 71: `$readmemh("tb/fixtures/vector_b.mem", operand_b_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 72: `$readmemh("tb/fixtures/vector_expected.mem", expected_mem);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are readmemh: the value named 'readmemh' in this block; tb: the value named 'tb' in this block; fixtures: the value named 'fixtures' in this block.

Line 74: `repeat (5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 75: `rst_n = 1'b1;` This makes rst_n, the active-low synchronous reset, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 77: `for (int case_index = 0; case_index < CASES; case_index++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 78: `logic [191:0] command;` This declares hardware signal storage or wiring in the module's structural declarations. Specifically, command is the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Widths in square brackets count from the stated most-significant bit down to the least-significant bit.

Line 79: `command = command_mem[case_index];` This makes command, the 128-bit instruction selecting an engine, shape, mode, and numeric parameters, equal to 'command_mem[case_index]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command_mem: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module's structural declarations.

Line 80: `while (!input_ready) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are while: the value named 'while' in this block; input_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 81: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 82: `operation = command[3:0];` This makes operation, the value named 'operation' in this block, equal to 'command[3:0]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the module's structural declarations.

Line 83: `multiplier_q15 = command[19:4];` This makes multiplier_q15, the value named 'multiplier q15' in this block, equal to 'command[19:4]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the module's structural declarations.

Line 84: `right_shift = command[24:20];` This makes right_shift, the value named 'right shift' in this block, equal to 'command[24:20]'. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the module's structural declarations.

Line 85: `output_zero_point = command[40:25];` This makes output_zero_point, the integer code representing real numeric zero in affine quantization, equal to ‘command[40:25]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the module’s structural declarations.

Line 86: `threshold = command[72:41];` This makes threshold, the membrane level at which a neuron emits a spike, equal to ‘command[72:41]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the module’s structural declarations.

Line 87: `surrogate_window = command[104:73];` This makes surrogate_window, the value named ’surrogate window’ in this block, equal to ‘command[104:73]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the module’s structural declarations.

Line 88: `clamp_minimum = command[136:105];` This makes clamp_minimum, the value named ’clamp minimum’ in this block, equal to ‘command[136:105]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the module’s structural declarations.

Line 89: `clamp_maximum = command[168:137];` This makes clamp_maximum, the value named ’clamp maximum’ in this block, equal to ‘command[168:137]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the module’s structural declarations.

Line 90: `operand_a = operand_a_mem[case_index];` This makes operand_a, the value named ’operand a’ in this block, equal to ‘operand_a_mem[case_index]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are operand_a_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module’s structural declarations.

Line 91: `operand_b = operand_b_mem[case_index];` This makes operand_b, the value named ’operand b’ in this block, equal to ‘operand_b_mem[case_index]‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are operand_b_mem: local FPGA storage addressed by surrounding index logic; case_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the module’s structural declarations.

Line 92: `input_valid = 1'b1;` This makes input_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b1‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named ’b1’ in this block. Context: the module’s structural declarations.

Line 93: `@(posedge clk);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together.

Line 94: `@(negedge clk);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are negedge: the value named ’negedge’ in this block; clk: the clock that makes all registers advance together.

Line 95: `input_valid = 1'b0;` This makes input_valid, the producer’s statement that its current payload is meaningful, equal to ‘1’b0‘. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named ’b0’ in this block. Context: the module’s structural declarations.

Line 97: `while (!output_valid) @(negedge clk);` Within the module’s structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are while: the value named ’while’ in this block; output_valid: the producer’s statement that its current payload is meaningful; negedge: the value named ’negedge’ in this block.

Line 98: `repeat (case_index % 5) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are repeat: the value named 'repeat' in this block; case_index: an integer selecting one tensor element, lane, bank, or state entry; negedge: the value named 'negedge' in this block.

Line 99: `if (!output_valid || config_error || result !== expected_mem[case_index]) begin` This condition selects the following hardware branch only when '!output_valid || config_error || result !== expected_mem[case_index]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_valid: the producer's statement that its current payload is meaningful; config_error: the value named 'config error' in this block; result: the value named 'result' in this block. Context: the module's structural declarations.

Line 100: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 101: `if (errors <= 20) begin` This statement continues through source line 102; the explanation uses the complete expression rather than only the first visual fragment. This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR vector case=%0d op=%0d config=%0d", case_index, operation, config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the module's structural declarations.

Line 102: `$display("ERROR vector case=%0d op=%0d config=%0d", case_index, operation, config_error);` This is continuation line 2 of the 2-line logical statement that begins at source line 101. Its exact fragment is '$display("ERROR vector case=%0d op=%0d config=%0d", case_index, operation, config_error);'. The important operands are display: the value named 'display' in this block; ERROR: the value named 'error' in this block; vector: the value named 'vector' in this block. Read together, the complete operation is: This condition selects the following hardware branch only when 'errors <= 20) begin $display("ERROR vector case=%0d op=%0d config=%0d", case_index, operation, config_error' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block; display: the value named 'display' in this block; ERROR: the value named 'error' in this block. Context: the module's structural declarations.

Line 103: `for (int lane = 0; lane < 8; lane++) begin` This generate/procedural loop repeats the following operation across lanes or entries. In synthesizable RTL, a fixed loop is expanded into parallel or scheduled hardware rather than executed by a CPU. Context: the module's structural declarations.

Line 104: `if (result[lane*32 +: 32] !== expected_mem[case_index][lane*32 +: 32]) begin` This condition selects the following hardware branch only when 'result[lane*32 +: 32] !== expected_mem[case_index][lane*32 +: 32]' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are result: the value named 'result' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; expected_mem: local FPGA storage addressed by surrounding index logic. Context: the module's structural declarations.

Line 105: `$display(" lane=%0d got=%0d expected=%0d", lane,` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 106: `$signed(result[lane*32 +: 32]),` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are result: the value named 'result' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 107: `$signed(expected_mem[case_index][lane*32 +: 32]));` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are expected_mem: local FPGA storage addressed by surrounding index logic;

case_index: an integer selecting one tensor element, lane, bank, or state entry; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 108: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 109: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 110: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 111: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 112: `checked++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are checked: the value named 'checked' in this block.

Line 113: `output_ready = 1'b1;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 114: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 115: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 116: `output_ready = 1'b0;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 117: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 119: `// Unsupported operations are visible as deterministic configuration errors.` Design intent: Unsupported operations are visible as deterministic configuration errors. This comment anchors the surrounding logic in the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation.

Line 120: `while (!input_ready) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are while: the value named 'while' in this block; input_ready: the consumer's statement that it can accept a payload this cycle; negedge: the value named 'negedge' in this block.

Line 121: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 122: `operation = 4'hf;` This makes operation, the value named 'operation' in this block, equal to '4'hf'. It is driven by combinational or procedural logic in the current evaluation. The important operands are hf: the value named 'hf' in this block. Context: the module's structural declarations.

Line 123: `input_valid = 1'b1;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 124: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 125: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 126: `input_valid = 1'b0;` This makes input_valid, the producer's statement that its current payload is meaningful, equal to '1'b0'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b0: the value named 'b0' in this block. Context: the module's structural declarations.

Line 127: `while (!output_valid) @(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are while: the value named 'while' in this block; output_valid: the producer's statement that its current payload is meaningful; negedge: the value named 'negedge' in this block.

Line 128: `if (!output_valid || !config_error || result != 0) begin` This condition selects the following hardware branch only when '!output_valid || !config_error || result != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are output_valid: the producer's statement that its current payload is meaningful; config_error: the value named 'config error' in this block; result: the value named 'result' in this block. Context: the module's structural declarations.

Line 129: `errors++;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are errors: the value named 'errors' in this block.

Line 130: `$display("ERROR vector ALU invalid opcode handling");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 131: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 132: `output_ready = 1'b1;` This makes output_ready, the consumer's statement that it can accept a payload this cycle, equal to '1'b1'. It is driven by combinational or procedural logic in the current evaluation. The important operands are b1: the value named 'b1' in this block. Context: the module's structural declarations.

Line 133: `@(posedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together.

Line 134: `@(negedge clk);` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The important operands are negedge: the value named 'negedge' in this block; clk: the clock that makes all registers advance together.

Line 136: `if (errors != 0) begin` This condition selects the following hardware branch only when 'errors != 0' is true. Both branches exist as gates; the condition controls a multiplexer rather than making software jump. The important operands are errors: the value named 'errors' in this block. Context: the module's structural declarations.

Line 137: `$display("tb_neuraedge_vector_alu FAILED errors=%0d checked=%0d", errors, checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 138: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 139: `end else begin` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 140: `$display("tb_neuraedge_vector_alu PASSED checked=%0d", checked);` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 141: `$finish;` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 142: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 143: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 145: `initial begin` This opens simulation initialization. In a testbench it drives starting values and stimulus; project synthesis does not rely on this block as a replacement for explicit reset behavior.

Line 146: `#500_000;` Within the module's structural declarations, this exact statement contributes to the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation. The expression is a literal or structural operator and has no hidden software state.

Line 147: `$fatal(1, "tb_neuraedge_vector_alu watchdog timeout");` This is a simulation check or diagnostic. It can stop a failing test and print evidence, but it is not part of the synthesized PYNQ circuit.

Line 148: `end` This structural keyword opens or closes the branch/process scope described as the module's structural declarations; it has no arithmetic value of its own.

Line 149: `endmodule` This closes the current hardware definition; no circuit behavior is attached to the keyword itself.

Line 151: `` `default_nettype wire `` This SystemVerilog preprocessor line controls compilation of the block that drives, backpressures, and checks the neuraedge vector alu RTL block in simulation.

61 tests/test_build_contract.py

Role in the complete system: checks build contract behavior independently with Python assertions.

61.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks build contract behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: PynqBuildContractTests. Functions: setUp, test_dma_accepts_a_complete_spikingformer_layer_pr test_accelerator_and_dma_address_map_is_stable, test_three_clock_domains_are_explicit, test_control_and_stream_ test_descriptor_rtl_is_packaged, test_deployment_artifacts_require_physical_acceptance. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

61.2 Complete source

```
"""Static checks for board-build settings that simulations cannot observe."""

from __future__ import annotations

import re
import unittest
from pathlib import Path


ROOT = Path(__file__).resolve().parents[1]


class PynqBuildContractTests(unittest.TestCase):
    def setUp(self) -> None:
        self.script = (ROOT / "scripts" / "build_pynq_full.tcl").read_text(
            encoding="ascii")

    def test_dma_accepts_a_complete_spikingformer_layer_program(self) -> None:
        match = re.search(r"CONFIG\.c_sg_length_width\s+\{(\d+)\}", self.script)
        self.assertIsNotNone(match, "AXI DMA length width must be explicit")
        self.assertEqual(int(match.group(1)), 26)
        self.assertGreater((1 << int(match.group(1))) - 1, 43_300_000)

    def test_accelerator_and_dma_address_map_is_stable(self) -> None:
        self.assertIn("0x43C00000", self.script)
        self.assertIn("0x40400000", self.script)

    def test_three_clock_domains_are_explicit(self) -> None:
        expected_frequencies = {
            0: "100.000000",
            1: "125.000000",
            2: "200.000000",
        }
        for clock_index, frequency in expected_frequencies.items():
            self.assertIn(
                f"CONFIG.PCW_FPGA{clock_index}_PERIPHERAL_FREQMHZ "
                f"{{{frequency}}}",
                self.script,
            )

        self.assertIn("[get_bd_pins ps7/FCLK_CLK0]", self.script)
        self.assertIn("[get_bd_pins ps7/FCLK_CLK1]", self.script)
        self.assertIn("[get_bd_pins ps7/FCLK_CLK2]", self.script)
        self.assertIn("[get_bd_pins accelerator/aclk]", self.script)
        self.assertIn("[get_bd_pins dma/m_axi_mm2s_aclk]", self.script)

    def test_control_and_stream_clock_crossings_are_explicit(self) -> None:
        for instance_name in ("accelerator_control_cdc", "dma_control_cdc"):
            self.assertRegex(
                self.script,
```

```
                rf"set {instance_name} .*axi_clock_converter:2\.1",
            )
        for instance_name in ("mm2s_cdc", "s2mm_cdc"):
            self.assertRegex(
                self.script,
                rf"set {instance_name} .*axis_clock_converter:1\.1",
            )

        self.assertIn("CONFIG.IS_ACLK_ASYNC {1}", self.script)
        self.assertIn("CONFIG.SYNCHRONIZATION_STAGES {3}", self.script)
        self.assertIn("mm2s_cdc/S_AXIS", self.script)
        self.assertIn("mm2s_cdc/M_AXIS", self.script)
        self.assertIn("s2mm_cdc/S_AXIS", self.script)
        self.assertIn("s2mm_cdc/M_AXIS", self.script)

    def test_descriptor_rtl_is_packaged(self) -> None:
        self.assertIn('"neuraedge_descriptor_sequencer.sv"', self.script)
        self.assertIn('"neuraedge_gradient_accumulator_bank.sv"', self.script)
        self.assertIn('"neuraedge_linear_backward8.sv"', self.script)
        self.assertIn("set_property range 256", self.script)

    def test_deployment_artifacts_require_physical_acceptance(self) -> None:
        self.assertIn(
            "STEPS.POST_ROUTE_PHYS_OPT_DESIGN.ARGS.DIRECTIVE Explore",
            self.script,
        )
        self.assertIn("route_error_count != 0", self.script)
        self.assertIn("drc_error_count != 0", self.script)
        self.assertIn("setup_wns < 0.0", self.script)
        self.assertIn("hold_whs < 0.0", self.script)
        self.assertLess(
            self.script.index("if {$hold_whs < 0.0}"),
            self.script.index("file copy -force $generated_bit"),
            "the accepted bitstream must be published only after timing gates",
        )


if __name__ == "__main__":
    unittest.main()
```

61.3 Line annotations

Line 1: `"""Static checks for board-build settings that simulations cannot observe."""` This is executable documentation attached to the file whose role is to checks build contract behavior independently with Python assertions. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 3: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks build contract behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import re` This imports the named Python API so the file whose role is to checks build contract behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import unittest` This imports the named Python API so the file whose role is to checks build contract behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from pathlib import Path` This imports the named Python API so the file whose role is to checks build contract behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `ROOT = Path(__file__).resolve().parents[1]` This assigns ‘Path(___file___).resolve().parents[1]‘ to ‘ROOT ‘ in host memory. Here ROOT means the value named ’root’ in this block. The important operands are Path: the value named ’path’ in this block; ___file___: the value named ’file’ in this block; resolve: the value named ’resolve’ in this block. Context: the file whose role is to checks build contract behavior independently with Python assertions.

Line 13: `class PynqBuildContractTests(unittest.TestCase):` This defines class PynqBuildContractTests, which groups data and methods for pynq build contract tests. Instances exist in host memory and organize the flow that checks build contract behavior independently with Python assertions.

Line 14: `def setUp(self) -> None:` This starts function setUp. Its indented body performs set up as part of the file that checks build contract behavior independently with Python assertions; it runs only when called.

Line 15: `self.script = (ROOT / "scripts" / "build_pynq_full.tcl").read_text(` This statement continues through source line 16; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(ROOT / "scripts" / "build_pynq_full.tcl").read_text(encoding="ascii")‘ to ‘self.script ‘ in host memory. Here script means the value named ’script’ in this block. The important operands are ROOT: the value named ’root’ in this block; scripts: the value named ’scripts’ in this block; build_pynq_full: the value named ’build pynq full’ in this block. Context: the function setUp, which handles set up.

Line 16: `encoding="ascii")` This is continuation line 2 of the 2-line logical statement that begins at source line 15. Its exact fragment is ‘encoding="ascii")‘. The important operands are encoding: the value named ’encoding’ in this block; ascii: the value named ’ascii’ in this block. Read together, the complete operation is: This assigns ‘(ROOT / "scripts" / "build_pynq_full.tcl").read_text(encoding="ascii")‘ to ‘self.script ‘ in host memory. Here script means the value named ’script’ in this block. The important operands are ROOT: the value named ’root’ in this block; scripts: the value named ’scripts’ in this block; build_pynq_full: the value named ’build pynq full’ in this block. Context: the function setUp, which handles set up.

Line 18: `def test_dma_accepts_a_complete_spikingformer_layer_program(self) -> None:` This starts function test_dma_accepts_a_complete_spikingformer_layer_program. Its indented body performs test dma accepts a complete spikingformer layer program as part of the file that checks build contract behavior independently with Python assertions; it runs only when called.

Line 19: `match = re.search(r"CONFIG\.c_sg_length_width\s+\{(\d+)\}", self.script)` This assigns ‘re.search(r"CONFI self.script)‘ to ‘match ‘ in host memory. Here match means the value named ’match’ in this block. The important operands are re: the value named ’re’ in this block; search: the value named ’search’ in this block; r: the value named ’r’ in this block. Context: the function test_dma_accepts_a_complete_spikingformer_layer_program, which handles test dma accepts a complete spikingformer layer program.

Line 20: `self.assertIsNotNone(match, "AXI DMA length width must be explicit")` This calls ‘self.assertIsNotNone‘ with ‘match, "AXI DMA length width must be explicit"‘ inside the function test_dma_accepts_a_complete_spikingformer_layer_p which handles test dma accepts a complete spikingformer layer program. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 21: `self.assertEqual(int(match.group(1)), 26)` This calls ‘self.assertEqual‘ with ‘int(match.group(1)), 26‘ inside the function test_dma_accepts_a_complete_spikingformer_layer_program, which handles test dma accepts a complete spikingformer layer program. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 22: `self.assertGreater((1 << int(match.group(1))) - 1, 43_300_000)` This calls ‘self.assertGreater‘ with ‘(1 << int(match.group(1))) - 1, 43_300_000‘ inside the function test_dma_accepts_a_complete_spikingformer_layer_program, which handles test dma accepts a complete spikingformer layer program. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 24: `def test_accelerator_and_dma_address_map_is_stable(self) -> None:` This starts function test_accelerator_an Its indented body performs test accelerator and dma address map is stable as part of the file that checks build contract behavior independently with Python assertions; it runs only when called.

Line 25: `self.assertIn("0x43C00000", self.script)` This calls ‘self.assertIn‘ with ‘"0x43C00000", self.script‘ inside the function test_accelerator_and_dma_address_map_is_stable, which handles test accelerator and dma address map is stable. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 26: `self.assertIn("0x40400000", self.script)` This calls ‘self.assertIn‘ with ‘"0x40400000", self.script‘ inside the function test_accelerator_and_dma_address_map_is_stable, which handles test accelerator and dma address map is stable. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 28: `def test_three_clock_domains_are_explicit(self) -> None:` This starts function test_three_clock_domains_ar Its indented body performs test three clock domains are explicit as part of the file that checks build contract behavior independently with Python assertions; it runs only when called.

Line 29: `expected_frequencies = {` This statement continues through source line 33; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ 0: "100.000000", 1: "125.000000", 2: "200.000000", }‘ to ‘expected_frequencies ‘ in host memory. Here expected_frequencies means the value named ’expected frequencies’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit.

Line 30: `0: "100.000000",` This is continuation line 2 of the 5-line logical statement that begins at source line 29. Its exact fragment is ‘0: "100.000000",‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ 0: "100.000000", 1: "125.000000", 2: "200.000000", }‘ to ‘expected_frequencies ‘ in host memory. Here expected_frequencies means the value named ’expected frequencies’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit.

Line 31: `1: "125.000000",` This is continuation line 3 of the 5-line logical statement that begins at source line 29. Its exact fragment is ‘1: "125.000000",‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ 0: "100.000000", 1: "125.000000", 2: "200.000000", }‘ to ‘expected_frequencies ‘ in host memory. Here expected_frequencies means the value named ’expected frequencies’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit.

Line 32: `2: "200.000000",` This is continuation line 4 of the 5-line logical statement that begins at source line 29. Its exact fragment is ‘2: "200.000000",‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ 0: "100.000000", 1: "125.000000", 2: "200.000000", }‘ to ‘expected_frequencies ‘ in host memory. Here expected_frequencies means the value named ’expected frequencies’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit.

Line 33: `}` This is continuation line 5 of the 5-line logical statement that begins at source line 29. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ 0: "100.000000", 1: "125.000000", 2: "200.000000", }‘ to ‘expected_frequencies ‘ in host memory. Here expected_frequencies means the value named ’expected frequencies’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit.

Line 34: `for clock_index, frequency in expected_frequencies.items():` This iterates clock_index, frequency over ‘expected_frequencies.items()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit.

Line 35: `self.assertIn(` This statement continues through source line 39; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertIn‘ with ‘ f"CONFIG.PCW_FPGA{clock_index}_PERIPHERA " f"{{{frequency}}}", self.script, ‘ inside the function test_three_clock_domains_are_explicit, which handles test three

clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 36: `f"CONFIG.PCW_FPGA{clock_index}_PERIPHERAL_FREQMHZ "` This is continuation line 2 of the 5-line logical statement that begins at source line 35. Its exact fragment is ‘f"CONFIG.PCW_FPGA{clock_index}_PERIPHERAL_FREQMHZ "‘. The important operands are f: the value named ’f’ in this block; CONFIG: the value named ’config’ in this block; PCW_FPGA: the value named ’pcw fpga’ in this block. Read together, the complete operation is: This calls ‘self.assertIn‘ with ‘ f"CONFIG.PCW_FPGA{clock_index}_PERIPHERAL_FREQMHZ " f"{{{frequency}}}", self.script, ‘ inside the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 37: `f"{{{frequency}}}",` This is continuation line 3 of the 5-line logical statement that begins at source line 35. Its exact fragment is ‘f"{{{frequency}}}",‘. The important operands are f: the value named ’f’ in this block; frequency: the value named ’frequency’ in this block. Read together, the complete operation is: This calls ‘self.assertIn‘ with ‘ f"CONFIG.PCW_FPGA{clock_index}_PERIPHERAL_FREQMHZ " f"{{{frequency}}}", self.script, ‘ inside the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 38: `self.script,` This is continuation line 4 of the 5-line logical statement that begins at source line 35. Its exact fragment is ‘self.script,‘. The important operands are script: the value named ’script’ in this block. Read together, the complete operation is: This calls ‘self.assertIn‘ with ‘ f"CONFIG.PCW_FPGA{clock_index}_PERIPHERAL_FREQMHZ " f"{{{frequency}}}", self.script, ‘ inside the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 39: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 35. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertIn‘ with ‘ f"CONFIG.PCW_FPGA{clock_index}_PERIPHERAL_FREQMHZ " f"{{{frequency}}}", self.script, ‘ inside the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 41: `self.assertIn("[get_bd_pins ps7/FCLK_CLK0]", self.script)` This calls ‘self.assertIn‘ with ‘"[get_bd_pins ps7/FCLK_CLK0]", self.script‘ inside the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 42: `self.assertIn("[get_bd_pins ps7/FCLK_CLK1]", self.script)` This calls ‘self.assertIn‘ with ‘"[get_bd_pins ps7/FCLK_CLK1]", self.script‘ inside the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 43: `self.assertIn("[get_bd_pins ps7/FCLK_CLK2]", self.script)` This calls ‘self.assertIn‘ with ‘"[get_bd_pins ps7/FCLK_CLK2]", self.script‘ inside the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 44: `self.assertIn("[get_bd_pins accelerator/aclk]", self.script)` This calls ‘self.assertIn‘ with ‘"[get_bd_pins accelerator/aclk]", self.script‘ inside the function test_three_clock_domains_are_explicit, which handles test three clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 45: `self.assertIn("[get_bd_pins dma/m_axi_mm2s_aclk]", self.script)` This calls ‘self.assertIn‘ with ‘"[get_bd_pin dma/m_axi_mm2s_aclk]", self.script‘ inside the function test_three_clock_domains_are_explicit, which handles test

three clock domains are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 47: `def test_control_and_stream_clock_crossings_are_explicit(self) -> None:` This starts function test_control_and_stream_clock_crossings_are_explicit. Its indented body performs test control and stream clock crossings are explicit as part of the file that checks build contract behavior independently with Python assertions; it runs only when called.

Line 48: `for instance_name in ("accelerator_control_cdc", "dma_control_cdc"):` This iterates instance_name over ‘("accelerator_control_cdc", "dma_control_cdc")‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit.

Line 49: `self.assertRegex(` This statement continues through source line 52; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertRegex‘ with ‘ self.script, rf"set {instance_name} .*axi_clock_converter:2\.1", ‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 50: `self.script,` This is continuation line 2 of the 4-line logical statement that begins at source line 49. Its exact fragment is ‘self.script,‘. The important operands are script: the value named ’script’ in this block. Read together, the complete operation is: This calls ‘self.assertRegex‘ with ‘ self.script, rf"set {instance_name} .*axi_clock_converter:2\.1", ‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 51: `rf"set {instance_name} .*axi_clock_converter:2\.1",` This is continuation line 3 of the 4-line logical statement that begins at source line 49. Its exact fragment is ‘rf"set {instance_name} .*axi_clock_converter:2\.1",‘. The important operands are rf: the value named ’rf’ in this block; set: the value named ’set’ in this block; instance_name: the value named ’instance name’ in this block. Read together, the complete operation is: This calls ‘self.assertRegex‘ with ‘ self.script, rf"set {instance_name} .*axi_clock_converter:2\.1", ‘ inside the function test_control_and_stream_clock_crossings_ar which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 52: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 49. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertRegex‘ with ‘ self.script, rf"set {instance_name} .*axi_clock_converter:2\.1", ‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 53: `for instance_name in ("mm2s_cdc", "s2mm_cdc"):` This iterates instance_name over ‘("mm2s_cdc", "s2mm_cdc")‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit.

Line 54: `self.assertRegex(` This statement continues through source line 57; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertRegex‘ with ‘ self.script, rf"set {instance_name} .*axis_clock_converter:1\.1", ‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 55: `self.script,` This is continuation line 2 of the 4-line logical statement that begins at source line 54. Its exact fragment is ‘self.script,‘. The important operands are script: the value named ’script’ in this block. Read together, the complete operation is: This calls ‘self.assertRegex‘ with ‘ self.script, rf"set {instance_name} .*axis_clock_converter:1\.1",

‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 56: `rf"set {instance_name} .*axis_clock_converter:1\.1",` This is continuation line 3 of the 4-line logical statement that begins at source line 54. Its exact fragment is ‘rf"set {instance_name} .*axis_clock_converter:1\.1",‘. The important operands are rf: the value named ’rf’ in this block; set: the value named ’set’ in this block; instance_name: the value named ’instance name’ in this block. Read together, the complete operation is: This calls ‘self.assertRegex‘ with ‘ self.script, rf"set {instance_name} .*axis_clock_converter:1\.1", ‘ inside the function test_control_and_stream_clock_crossings_a which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 57: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 54. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertRegex‘ with ‘ self.script, rf"set {instance_name} .*axis_clock_converter:1\.1", ‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 59: `self.assertIn("CONFIG.IS_ACLK_ASYNC {1}", self.script)` This calls ‘self.assertIn‘ with ‘"CONFIG.IS_ACLK_A {1}", self.script‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 60: `self.assertIn("CONFIG.SYNCHRONIZATION_STAGES {3}", self.script)` This calls ‘self.assertIn‘ with ‘"CONFIG.SYNCHRONIZATION_STAGES {3}", self.script‘ inside the function test_control_and_stream_clock_crossings_are_explicit which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 61: `self.assertIn("mm2s_cdc/S_AXIS", self.script)` This calls ‘self.assertIn‘ with ‘"mm2s_cdc/S_AXIS", self.script‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 62: `self.assertIn("mm2s_cdc/M_AXIS", self.script)` This calls ‘self.assertIn‘ with ‘"mm2s_cdc/M_AXIS", self.script‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 63: `self.assertIn("s2mm_cdc/S_AXIS", self.script)` This calls ‘self.assertIn‘ with ‘"s2mm_cdc/S_AXIS", self.script‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 64: `self.assertIn("s2mm_cdc/M_AXIS", self.script)` This calls ‘self.assertIn‘ with ‘"s2mm_cdc/M_AXIS", self.script‘ inside the function test_control_and_stream_clock_crossings_are_explicit, which handles test control and stream clock crossings are explicit. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 66: `def test_descriptor_rtl_is_packaged(self) -> None:` This starts function test_descriptor_rtl_is_packaged. Its indented body performs test descriptor rtl is packaged as part of the file that checks build contract behavior independently with Python assertions; it runs only when called.

Line 67: `self.assertIn('"neuraedge_descriptor_sequencer.sv"', self.script)` This calls ‘self.assertIn‘ with ‘’"neuraedge_descriptor_sequencer.sv"’, self.script‘ inside the function test_descriptor_rtl_is_packaged, which handles test descriptor rtl is packaged. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 68: `self.assertIn('"neuraedge_gradient_accumulator_bank.sv"', self.script)` This calls ‘self.assertIn‘ with ‘’"neuraedge_gradient_accumulator_bank.sv"’, self.script‘ inside the function test_descriptor_rtl_is_packaged, which handles test descriptor rtl is packaged. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 69: `self.assertIn('"neuraedge_linear_backward8.sv"', self.script)` This calls ‘self.assertIn‘ with ‘’"neuraedge_linear_backward8.sv"’, self.script‘ inside the function test_descriptor_rtl_is_packaged, which handles test descriptor rtl is packaged. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 70: `self.assertIn("set_property range 256", self.script)` This calls ‘self.assertIn‘ with ‘"set_property range 256", self.script‘ inside the function test_descriptor_rtl_is_packaged, which handles test descriptor rtl is packaged. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 72: `def test_deployment_artifacts_require_physical_acceptance(self) -> None:` This starts function test_deployment_artifacts_require_physical_acceptance. Its indented body performs test deployment artifacts require physical acceptance as part of the file that checks build contract behavior independently with Python assertions; it runs only when called.

Line 73: `self.assertIn(` This statement continues through source line 76; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertIn‘ with ‘ "STEPS.POST_ROUTE_PHYS_OPT_DESIGN.ARG Explore", self.script, ‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 74: `"STEPS.POST_ROUTE_PHYS_OPT_DESIGN.ARGS.DIRECTIVE Explore",` This is continuation line 2 of the 4-line logical statement that begins at source line 73. Its exact fragment is ‘"STEPS.POST_ROUTE_PHYS_OPT_DESIGN.ARGS.DIRE Explore",‘. The important operands are STEPS: the value named ’steps’ in this block; POST_ROUTE_PHYS_OPT_DESIGN: the value named ’post route phys opt design’ in this block; ARGS: the value named ’args’ in this block. Read together, the complete operation is: This calls ‘self.assertIn‘ with ‘ "STEPS.POST_ROUTE_PHYS_OPT_DESIGN.ARGS.DIRECTIVE Explore", self.script, ‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 75: `self.script,` This is continuation line 3 of the 4-line logical statement that begins at source line 73. Its exact fragment is ‘self.script,‘. The important operands are script: the value named ’script’ in this block. Read together, the complete operation is: This calls ‘self.assertIn‘ with ‘ "STEPS.POST_ROUTE_PHYS_OPT_DESIGN.ARGS.DIRECTIVE Explore", self.script, ‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 76: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 73. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertIn‘ with ‘ "STEPS.POST_ROUTE_PHYS_OPT_DESIGN.ARGS.DIRECTIVE Explore", self.script, ‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 77: `self.assertIn("route_error_count != 0", self.script)` This calls ‘self.assertIn‘ with ‘"route_error_count != 0", self.script‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 78: `self.assertIn("drc_error_count != 0", self.script)` This calls ‘self.assertIn‘ with ‘"drc_error_count != 0", self.script‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 79: `self.assertIn("setup_wns < 0.0", self.script)` This calls ‘self.assertIn‘ with ‘"setup_wns < 0.0", self.script‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 80: `self.assertIn("hold_whs < 0.0", self.script)` This calls ‘self.assertIn‘ with ‘"hold_whs < 0.0", self.script‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 81: `self.assertLess(` This statement continues through source line 85; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertLess‘ with ‘ self.script.index("if {$hold_whs < 0.0}"), self.script.index("file copy -force $generated_bit"), "the accepted bitstream must be published only after timing gates", ‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 82: `self.script.index("if {$hold_whs < 0.0}"),` This is continuation line 2 of the 5-line logical statement that begins at source line 81. Its exact fragment is ‘self.script.index("if {$hold_whs < 0.0}"),‘. The important operands are script: the value named ’script’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; hold_whs: the value named ’hold whs’ in this block. Read together, the complete operation is: This calls ‘self.assertLess‘ with ‘ self.script.index("if {$hold_whs < 0.0}"), self.script.index("file copy -force $generated_bit"), "the accepted bitstream must be published only after timing gates", ‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 83: `self.script.index("file copy -force $generated_bit"),` This is continuation line 3 of the 5-line logical statement that begins at source line 81. Its exact fragment is ‘self.script.index("file copy -force $generated_bit"),‘. The important operands are script: the value named ’script’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; file: the value named ’file’ in this block. Read together, the complete operation is: This calls ‘self.assertLess‘ with ‘ self.script.index("if {$hold_whs < 0.0}"), self.script.index("file copy -force $generated_bit"), "the accepted bitstream must be published only after timing gates", ‘ inside the function test_deployment_artifacts_require_physical_ac which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 84: `"the accepted bitstream must be published only after timing gates",` This is continuation line 4 of the 5-line logical statement that begins at source line 81. Its exact fragment is ‘"the accepted bitstream must be published only after timing gates",‘. The important operands are the: the value named ’the’ in this block; accepted: the value named ’accepted’ in this block; bitstream: the value named ’bitstream’ in this block. Read together, the complete operation is: This calls ‘self.assertLess‘ with ‘ self.script.index("if {$hold_whs < 0.0}"), self.script.index("file copy -force $generated_bit"), "the accepted bitstream must be published only after timing gates", ‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 85: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 81. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertLess‘ with ‘ self.script.index("if {$hold_whs < 0.0}"), self.script.index("file copy -force $generated_bit"), "the accepted bitstream must be published only after timing gates", ‘ inside the function test_deployment_artifacts_require_physical_acceptance, which handles test deployment artifacts require physical acceptance. The call advances the host-side job that checks build contract behavior independently with Python assertions.

Line 88: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks build contract behavior independently with Python assertions.

Line 89: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks build contract behavior independently with Python assertions. The call advances the host-side job that checks build contract behavior independently with Python assertions.

62 tests/test_debug_report.py

Role in the complete system: checks debug report behavior independently with Python assertions.

62.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks debug report behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: DebugReportTests. Functions: test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, test_final_vivado_report_parsers_recover_verified_metrics. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

62.2 Complete source

```
from __future__ import annotations

import tempfile
import unittest
from pathlib import Path

from generate_debug_report import (
    choose_wave_signals,
    parse_cdc_summary,
    parse_clock_timing,
    parse_drc_summary,
    parse_implementation_summary,
    parse_power_report,
    parse_utilization_hierarchy,
    parse_vcd,
)


class DebugReportTests(unittest.TestCase):
    def test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state(self) -> None:
        fixture = """$timescale 1ps $end
$scope module top $end
$var reg 1 ! clk $end
$var reg 1 \" clear_state $end
$scope module dut $end
$var reg 3 # state [2:0] $end
$var reg 1 $ program_active $end
$upscope $end
$upscope $end
$enddefinitions $end
#0
0!
0\"
b0 #
0$
#5
1!
b1 #
1$
"""
        with tempfile.TemporaryDirectory() as temporary_directory:
            path = Path(temporary_directory) / "fixture.vcd"
            path.write_text(fixture, encoding="utf-8")
            timescale, signals = parse_vcd(path)

        selected = choose_wave_signals(signals)
        leaves = [signal.name.rsplit("/", 1)[-1] for signal in selected]
        self.assertEqual(timescale, "1ps")
        self.assertIn("state", leaves)
        self.assertIn("program_active", leaves)
        self.assertNotIn("clear_state", leaves)
        self.assertEqual(signals["/top/dut/state"].transitions[-1], (5, 1.0))

```

```
    def test_final_vivado_report_parsers_recover_verified_metrics(self) -> None:
        root = Path(__file__).resolve().parents[1]
        reports = root / "outputs" / "pynq_full"
        implementation = parse_implementation_summary(
            reports / "full_implementation_summary.txt"
        )
        clocks = parse_clock_timing(reports / "full_routed_timing_summary.rpt")
        utilization = parse_utilization_hierarchy(
            reports / "full_routed_utilization_hierarchical.rpt"
        )
        drc = parse_drc_summary(reports / "full_drc.rpt")
        cdc = parse_cdc_summary(reports / "full_cdc.rpt")
        power = parse_power_report(
            root / "outputs" / "power_activity_full" / "vivado_reports"
            / "routed_saif_power.rpt"
        )

        self.assertEqual(
            clocks["clk_fpga_2"],
            (implementation["setup_wns_ns"], implementation["hold_whs_ns"]),
        )
        self.assertEqual(
            utilization["system_wrapper"]["dsp"],
            int(implementation["dsp48_count"]),
        )
        self.assertFalse(any(row[1] == "Error" for row in drc))
        self.assertGreater(sum(row[3] for row in drc), 0)
        self.assertTrue(any(row[0] == "CDC-15" for row in cdc))
        self.assertGreater(power["total"], 0.0)
        self.assertRegex(power["matched"], r"\d+%\s+\(\d+/\d+\)")


if __name__ == "__main__":
    unittest.main()
```

62.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import tempfile` This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 4: `import unittest` This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `from pathlib import Path` This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from generate_debug_report import (` This statement continues through source line 16; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `choose_wave_signals,` This is continuation line 2 of the 10-line logical statement that begins at source line 7. Its exact fragment is ‘choose_wave_signals,‘. The important operands are choose_wave_signals: the value named ’choose wave signals’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `parse_cdc_summary,` This is continuation line 3 of the 10-line logical statement that begins at source line 7. Its exact fragment is ‘parse_cdc_summary,‘. The important operands are parse_cdc_summary: the value named ’parse

cdc summary' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `parse_clock_timing,` This is continuation line 4 of the 10-line logical statement that begins at source line 7. Its exact fragment is 'parse_clock_timing,'. The important operands are parse_clock_timing: the value named 'parse clock timing' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `parse_drc_summary,` This is continuation line 5 of the 10-line logical statement that begins at source line 7. Its exact fragment is 'parse_drc_summary,'. The important operands are parse_drc_summary: the value named 'parse drc summary' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `parse_implementation_summary,` This is continuation line 6 of the 10-line logical statement that begins at source line 7. Its exact fragment is 'parse_implementation_summary,'. The important operands are parse_implementation_summar the value named 'parse implementation summary' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `parse_power_report,` This is continuation line 7 of the 10-line logical statement that begins at source line 7. Its exact fragment is 'parse_power_report,'. The important operands are parse_power_report: the value named 'parse power report' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `parse_utilization_hierarchy,` This is continuation line 8 of the 10-line logical statement that begins at source line 7. Its exact fragment is 'parse_utilization_hierarchy,'. The important operands are parse_utilization_hierarchy: the value named 'parse utilization hierarchy' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `parse_vcd,` This is continuation line 9 of the 10-line logical statement that begins at source line 7. Its exact fragment is 'parse_vcd,'. The important operands are parse_vcd: the value named 'parse vcd' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 7. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks debug report behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `class DebugReportTests(unittest.TestCase):` This defines class DebugReportTests, which groups data and methods for debug report tests. Instances exist in host memory and organize the flow that checks debug report behavior independently with Python assertions.

Line 20: `def test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state(self) -> None:` This starts function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state. Its indented body performs test vcd parser keeps exact fsm state ahead of clear state as part of the file that checks debug report behavior independently with Python assertions; it runs only when called.

Line 21: `fixture = """$timescale 1ps $end` This statement continues through source line 40; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 22: `$scope module top $end` This is continuation line 2 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘$scope module top $end‘. The important operands are scope: the value named ’scope’ in this block; module: the value named ’module’ in this block; top: the value named ’top’ in this block. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 23: `$var reg 1 ! clk $end` This is continuation line 3 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘$var reg 1 ! clk $end‘. The important operands are var: the value named ’var’ in this block; reg: the value named ’reg’ in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 24: `$var reg 1 \" clear_state $end` This is continuation line 4 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘$var reg 1 \" clear_state $end‘. The important operands are var: the value named ’var’ in this block; reg: the value named ’reg’ in this block; clear_state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 25: `$scope module dut $end` This is continuation line 5 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘$scope module dut $end‘. The important operands are scope: the value named ’scope’ in this block; module: the value named ’module’ in this block; dut: the value named ’dut’ in this block. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 26: `$var reg 3 # state [2:0] $end` This is continuation line 6 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘$var reg 3 # state [2:0] $end‘. The important operands are var: the value named ’var’ in this block; reg: the value named ’reg’ in this block; state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk

$end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 27: `$var reg 1 $ program_active $end` This is continuation line 7 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘$var reg 1 $ program_active $end‘. The important operands are var: the value named ’var’ in this block; reg: the value named ’reg’ in this block; program_active: the value named ’program active’ in this block. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 28: `$upscope $end` This is continuation line 8 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘$upscope $end‘. The important operands are upscope: the value named ’upscope’ in this block. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 29: `$upscope $end` This is continuation line 9 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘$upscope $end‘. The important operands are upscope: the value named ’upscope’ in this block. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 30: `$enddefinitions $end` This is continuation line 10 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘$enddefinitions $end‘. The important operands are enddefinitions: the value named ’enddefinitions’ in this block. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 31: `#0` This is continuation line 11 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘#0‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 32: `0!` This is continuation line 12 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘0¡. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 33: `0\"` This is continuation line 13 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘0\"‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 34: `b0 #` This is continuation line 14 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘b0 #‘. The important operands are b0: the value named ’b0’ in this block. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 35: `0$` This is continuation line 15 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘0$‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 36: `#5` This is continuation line 16 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘#5‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 37: `1!` This is continuation line 17 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘1¡. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block.

Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 38: `b1 #` This is continuation line 18 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘b1 #‘. The important operands are b1: the value named ’b1’ in this block. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 39: `1$` This is continuation line 19 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘1$‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 40: `"""` This is continuation line 20 of the 20-line logical statement that begins at source line 21. Its exact fragment is ‘"""‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘"""$timescale 1ps $end $scope module top $end $var reg 1 ! clk $end $var reg 1 \" clear_state $end $scope module dut $end $var reg 3 # state [2:0] $end $var reg 1 $ program_active $end $upscope $end $upscope $end $enddefinitions $end #0 0! 0\" b0 # 0$ #5 1! b1 # 1$ """‘ to ‘fixture ‘ in host memory. Here fixture means the value named ’fixture’ in this block. The important operands are timescale: the real-value spacing represented by one quantized integer step; scope: the value named ’scope’ in this block; module: the value named ’module’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 41: `with tempfile.TemporaryDirectory() as temporary_directory:` This opens a managed resource scope for ‘tempfile.TemporaryDirectory() as temporary_directory‘ so files, tracing, or temporary state are finalized even if the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state raises an error.

Line 42: `path = Path(temporary_directory) / "fixture.vcd"` This assigns ‘Path(temporary_directory) / "fixture.vcd"‘ to ‘path ‘ in host memory. Here path means the value named ’path’ in this block. The important operands are Path: the value named ’path’ in this block; temporary_directory: the value named ’temporary directory’ in this block; fixture: the value named ’fixture’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_sta which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 43: `path.write_text(fixture, encoding="utf-8")` This calls ‘path.write_text‘ with ‘fixture, encoding="utf-8"‘ inside the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 44: `timescale, signals = parse_vcd(path)` This assigns ‘parse_vcd(path)‘ to ‘timescale, signals ‘ in host memory. Here timescale means the real-value spacing represented by one quantized integer step. The important operands are parse_vcd: the value named ’parse vcd’ in this block; path: the value named ’path’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 46: `selected = choose_wave_signals(signals)` This assigns ‘choose_wave_signals(signals)‘ to ‘selected ‘ in host memory. Here selected means the value named ’selected’ in this block. The important operands are choose_wave_signals: the value named ’choose wave signals’ in this block; signals: the value named ’signals’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 47: `leaves = [signal.name.rsplit("/", 1)[-1] for signal in selected]` This assigns ‘[signal.name.rsplit("/", 1)[-1] for signal in selected]‘ to ‘leaves ‘ in host memory. Here leaves means the value named ’leaves’ in this block. The important operands are signal: the value named ’signal’ in this block; name: the value named ’name’ in this block; rsplit: the value named ’rsplit’ in this block. Context: the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state.

Line 48: `self.assertEqual(timescale, "1ps")` This calls ‘self.assertEqual‘ with ‘timescale, "1ps"‘ inside the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 49: `self.assertIn("state", leaves)` This calls ‘self.assertIn‘ with ‘"state", leaves‘ inside the function test_vcd_parser_k which handles test vcd parser keeps exact fsm state ahead of clear state. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 50: `self.assertIn("program_active", leaves)` This calls ‘self.assertIn‘ with ‘"program_active", leaves‘ inside the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 51: `self.assertNotIn("clear_state", leaves)` This calls ‘self.assertNotIn‘ with ‘"clear_state", leaves‘ inside the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_clear_state, which handles test vcd parser keeps exact fsm state ahead of clear state. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 52: `self.assertEqual(signals["/top/dut/state"].transitions[-1], (5, 1.0))` This calls ‘self.assertEqual‘ with ‘signals["/top/dut/state"].transitions[-1], (5, 1.0)‘ inside the function test_vcd_parser_keeps_exact_fsm_state_ahead_of_cle which handles test vcd parser keeps exact fsm state ahead of clear state. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 54: `def test_final_vivado_report_parsers_recover_verified_metrics(self) -> None:` This starts function test_final_vivado_report_parsers_recover_verified_metrics. Its indented body performs test final vivado report parsers recover verified metrics as part of the file that checks debug report behavior independently with Python assertions; it runs only when called.

Line 55: `root = Path(__file__).resolve().parents[1]` This assigns ‘Path(___file___).resolve().parents[1]‘ to ‘root ‘ in host memory. Here root means the value named ’root’ in this block. The important operands are Path: the value named ’path’ in this block; ___file___: the value named ’file’ in this block; resolve: the value named ’resolve’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 56: `reports = root / "outputs" / "pynq_full"` This assigns ‘root / "outputs" / "pynq_full"‘ to ‘reports ‘ in host memory. Here reports means the value named ’reports’ in this block. The important operands are root: the value named ’root’ in this block; outputs: the value named ’outputs’ in this block; pynq_full: the value named ’pynq full’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 57: `implementation = parse_implementation_summary(` This statement continues through source line 59; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘parse_implementation_summary(reports / "full_implementation_summary.txt")‘ to ‘implementation ‘ in host memory. Here implementation means the value named ’implementation’ in this block. The important operands are parse_implementation_summary: the value named ’parse implementation summary’ in this block; reports: the value named ’reports’ in this block; full_implementation_summar the value named ’full implementation summary’ in this block. Context: the function test_final_vivado_report_parsers_recover_ver which handles test final vivado report parsers recover verified metrics.

Line 58: `reports / "full_implementation_summary.txt"` This is continuation line 2 of the 3-line logical statement that begins at source line 57. Its exact fragment is ‘reports / "full_implementation_summary.txt"‘. The important operands are reports: the value named ’reports’ in this block; full_implementation_summary: the value named ’full implementation summary’ in this block; txt: the value named ’txt’ in this block. Read together, the complete operation is: This assigns ‘parse_implementation_summary(reports / "full_implementation_summary.txt")‘ to ‘implementation ‘ in host memory. Here implementation means the value named ’implementation’ in this block. The important operands are parse_implementation_summary: the value named ’parse implementation summary’ in this block; reports: the value named ’reports’ in this block; full_implementation_summary: the value named ’full implementation summary’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 59: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 57. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘parse_implementation_summary(reports / "full_implementation_summary.txt")‘ to ‘implementation ‘ in host memory. Here implementation means the value named ’implementation’ in this block. The important operands are parse_implementation_summary: the value named ’parse implementation summary’ in this block; reports: the value named ’reports’ in this block; full_implementation_summary: the value named ’full implementation summary’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 60: `clocks = parse_clock_timing(reports / "full_routed_timing_summary.rpt")` This assigns ‘parse_clock_timing / "full_routed_timing_summary.rpt")‘ to ‘clocks ‘ in host memory. Here clocks means the value named ’clocks’ in this block. The important operands are parse_clock_timing: the value named ’parse clock timing’ in this block; reports: the value named ’reports’ in this block; full_routed_timing_summary: the value named ’full routed timing summary’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 61: `utilization = parse_utilization_hierarchy(` This statement continues through source line 63; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘parse_utilization_hierarchy(reports / "full_routed_utilization_hierarchical.rpt")‘ to ‘utilization ‘ in host memory. Here utilization means the value named ’utilization’ in this block. The important operands are parse_utilization_hierarchy: the value named ’parse utilization hierarchy’ in this block; reports: the value named ’reports’ in this block; full_routed_utilization_hierarchical: the value named ’full routed utilization hierarchical’ in this block. Context: the function test_final_vivado_report_parsers_recover_ver which handles test final vivado report parsers recover verified metrics.

Line 62: `reports / "full_routed_utilization_hierarchical.rpt"` This is continuation line 2 of the 3-line logical statement that begins at source line 61. Its exact fragment is ‘reports / "full_routed_utilization_hierarchical.rpt"‘. The important operands are reports: the value named ’reports’ in this block; full_routed_utilization_hierarchical: the value named ’full routed utilization hierarchical’ in this block; rpt: the value named ’rpt’ in this block. Read together, the complete operation is: This assigns ‘parse_utilization_hierarchy(reports / "full_routed_utilization_hierarchical.rpt")‘ to ‘utilization ‘ in host memory. Here utilization means the value named ’utilization’ in this block. The important operands are parse_utilization_hierarchy: the value named ’parse utilization hierarchy’ in this block; reports: the value named ’reports’ in this block; full_routed_utilization_hierarchical: the value named ’full routed utilization hierarchical’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 63: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 61. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the

complete operation is: This assigns ‘parse_utilization_hierarchy(reports / "full_routed_utilization_hierarchical.rpt")‘ to ‘utilization ‘ in host memory. Here utilization means the value named ’utilization’ in this block. The important operands are parse_utilization_hierarchy: the value named ’parse utilization hierarchy’ in this block; reports: the value named ’reports’ in this block; full_routed_utilization_hierarchical: the value named ’full routed utilization hierarchical’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 64: `drc = parse_drc_summary(reports / "full_drc.rpt")` This assigns ‘parse_drc_summary(reports / "full_drc.rpt")‘ to ‘drc ‘ in host memory. Here drc means the value named ’drc’ in this block. The important operands are parse_drc_summary: the value named ’parse drc summary’ in this block; reports: the value named ’reports’ in this block; full_drc: the value named ’full drc’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 65: `cdc = parse_cdc_summary(reports / "full_cdc.rpt")` This assigns ‘parse_cdc_summary(reports / "full_cdc.rpt")‘ to ‘cdc ‘ in host memory. Here cdc means the value named ’cdc’ in this block. The important operands are parse_cdc_summary: the value named ’parse cdc summary’ in this block; reports: the value named ’reports’ in this block; full_cdc: the value named ’full cdc’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 66: `power = parse_power_report(` This statement continues through source line 69; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘parse_power_report(root / "outputs" / "power_activity_full" / "vivado_reports" / "routed_saif_power.rpt")‘ to ‘power ‘ in host memory. Here power means the value named ’power’ in this block. The important operands are parse_power_report: the value named ’parse power report’ in this block; root: the value named ’root’ in this block; outputs: the value named ’outputs’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 67: `root / "outputs" / "power_activity_full" / "vivado_reports"` This is continuation line 2 of the 4-line logical statement that begins at source line 66. Its exact fragment is ‘root / "outputs" / "power_activity_full" / "vivado_reports"‘. The important operands are root: the value named ’root’ in this block; outputs: the value named ’outputs’ in this block; power_activity_full: the value named ’power activity full’ in this block. Read together, the complete operation is: This assigns ‘parse_power_report(root / "outputs" / "power_activity_full" / "vivado_reports" / "routed_saif_power.rpt")‘ to ‘power ‘ in host memory. Here power means the value named ’power’ in this block. The important operands are parse_power_report: the value named ’parse power report’ in this block; root: the value named ’root’ in this block; outputs: the value named ’outputs’ in this block. Context: the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics.

Line 68: `/ "routed_saif_power.rpt"` This is continuation line 3 of the 4-line logical statement that begins at source line 66. Its exact fragment is ‘/ "routed_saif_power.rpt"‘. The important operands are routed_saif_power: the value named ’routed saif power’ in this block; rpt: the value named ’rpt’ in this block. Read together, the complete operation is: This assigns ‘parse_power_report(root / "outputs" / "power_activity_full" / "vivado_reports" / "routed_saif_power.rpt")‘ to ‘power ‘ in host memory. Here power means the value named ’power’ in this block. The important operands are parse_power_report: the value named ’parse power report’ in this block; root: the value named ’root’ in this block; outputs: the value named ’outputs’ in this block. Context: the function test_final_vivado_report_parsers_recov which handles test final vivado report parsers recover verified metrics.

Line 69: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 66. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘parse_power_report(root / "outputs" / "power_activity_full" / "vivado_reports" / "routed_saif_power.rpt")‘ to ‘power ‘ in host memory. Here power means the value named ’power’ in this block. The important operands are parse_power_report: the value named ’parse power report’ in this block; root: the value named ’root’ in this block; outputs: the value named ’outputs’ in this block. Context: the function test_final_vivado_report_parsers_recov which handles test final vivado report parsers recover verified metrics.

Line 71: `self.assertEqual(` This statement continues through source line 74; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertEqual‘ with ‘ clocks["clk_fpga_2"], (implementation["setup_wns_ns"], implementation["hold_whs_ns"]), ‘ inside the function test_final_vivado_report_parsers_recover_verified_ which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 72: `clocks["clk_fpga_2"],` This is continuation line 2 of the 4-line logical statement that begins at source line 71. Its exact fragment is ‘clocks["clk_fpga_2"],‘. The important operands are clocks: the value named ’clocks’ in this block; clk_fpga_2: the clock that makes all registers advance together. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ clocks["clk_fpga_2"], (implementation["setup_wns_ns"], implementation["hold_whs_ns"]), ‘ inside the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 73: `(implementation["setup_wns_ns"], implementation["hold_whs_ns"]),` This is continuation line 3 of the 4-line logical statement that begins at source line 71. Its exact fragment is ‘(implementation["setup_wns_ns"], implementation["hold_whs_ns"]),‘. The important operands are implementation: the value named ’implementation’ in this block; setup_wns_ns: the value named ’setup wns ns’ in this block; hold_whs_ns: the value named ’hold whs ns’ in this block. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ clocks["clk_fpga_2"], (implementation["setup_wns_ns"], implementation["hold_whs_ns"]), ‘ inside the function test_final_vivado_report_parsers_recover_verified_ which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 74: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 71. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ clocks["clk_fpga_2"], (implementation["setup_wns_ns"], implementation["hold_whs_ns"]), ‘ inside the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 75: `self.assertEqual(` This statement continues through source line 78; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertEqual‘ with ‘ utilization["system_wrapper"]["dsp"], int(implementation["dsp48_count"]), ‘ inside the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 76: `utilization["system_wrapper"]["dsp"],` This is continuation line 2 of the 4-line logical statement that begins at source line 75. Its exact fragment is ‘utilization["system_wrapper"]["dsp"],‘. The important operands are utilization: the value named ’utilization’ in this block; system_wrapper: the value named ’system wrapper’ in this block; dsp: the value named ’dsp’ in this block. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ utilization["system_wrapper"]["dsp"], int(implementation["dsp48_count"]), ‘ inside the function test_final_vivado_report_parsers_recove which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 77: `int(implementation["dsp48_count"]),` This is continuation line 3 of the 4-line logical statement that begins at source line 75. Its exact fragment is ‘int(implementation["dsp48_count"]),‘. The important operands are implementation: the value named ’implementation’ in this block; dsp48_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ utilization["system_wrapper"]["dsp"], int(implementation["dsp48_count"]), ‘ inside the function test_final_vivado_report_parsers_recove which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 78: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 75. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ utilization["system_wrapper"]["dsp"], int(implementation["dsp48_count"]), ‘ inside the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers

recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 79: `self.assertFalse(any(row[1] == "Error" for row in drc))` This calls ‘self.assertFalse‘ with ‘any(row[1] == "Error" for row in drc)‘ inside the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 80: `self.assertGreater(sum(row[3] for row in drc), 0)` This calls ‘self.assertGreater‘ with ‘sum(row[3] for row in drc), 0‘ inside the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 81: `self.assertTrue(any(row[0] == "CDC-15" for row in cdc))` This calls ‘self.assertTrue‘ with ‘any(row[0] == "CDC-15" for row in cdc)‘ inside the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 82: `self.assertGreater(power["total"], 0.0)` This calls ‘self.assertGreater‘ with ‘power["total"], 0.0‘ inside the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 83: `self.assertRegex(power["matched"], r"\d+%\s+\(\d+/\d+\)")` This calls ‘self.assertRegex‘ with ‘power["matched" r"\d+%\s+\(\d+/\d+\)"‘ inside the function test_final_vivado_report_parsers_recover_verified_metrics, which handles test final vivado report parsers recover verified metrics. The call advances the host-side job that checks debug report behavior independently with Python assertions.

Line 86: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks debug report behavior independently with Python assertions.

Line 87: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks debug report behavior independently with Python assertions. The call advances the host-side job that checks debug report behavior independently with Python assertions.

63 tests/test_documentation_generator.py

Role in the complete system: checks documentation generator behavior independently with Python assertions.

63.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks documentation generator behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: DocumentationGeneratorTests. Functions: test_every_nonblank_source_line_is_covered_and_hash
test_rtl_context_returns_to_structure_after_clocked_process, test_multiline_statements_are_explained_as_complete_
Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

63.2 Complete source

```
from __future__ import annotations

import json
import tempfile
import unittest
from pathlib import Path

from generate_source_reference import (
    generate,
    project_root,
    python_statement_spans,
    source_entries,
    sv_contexts,
    sv_statement_spans,
)


class DocumentationGeneratorTests(unittest.TestCase):
    def test_every_nonblank_source_line_is_covered_and_hashed(self) -> None:
        with tempfile.TemporaryDirectory() as temporary_directory:
            tex_path, manifest_path = generate(Path(temporary_directory))
            manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
            entries = source_entries(project_root())

            self.assertEqual(manifest["summary"]["file_count"], len(entries))
            self.assertEqual(
                manifest["summary"]["nonblank_lines"],
                manifest["summary"]["explained_nonblank_lines"],
            )
            self.assertTrue(tex_path.read_text(encoding="utf-8").endswith("\\end{document}\n"))
            generated_tex = tex_path.read_text(encoding="utf-8")
            self.assertIn("Beginner architecture map", generated_tex)
            self.assertIn("What becomes FPGA hardware", generated_tex)
            self.assertIn("Two unsigned byte weights", generated_tex)
            listing_paths = [
                line.removeprefix("\\lstinputlisting{").removesuffix("}")
                for line in generated_tex.splitlines()
                if line.startswith("\\lstinputlisting{")
            ]
            self.assertEqual(len(listing_paths), len(entries))
            for listing_path in listing_paths:
                self.assertTrue(
                    (tex_path.parent / listing_path).resolve().is_file(),
                    listing_path,
                )
            for file_record in manifest["files"]:
                self.assertEqual(
                    file_record["nonblank_lines"],
                    file_record["explained_nonblank_lines"],
                )
                self.assertEqual(len(file_record["sha256"]), 64)

```

```
    def test_rtl_context_returns_to_structure_after_clocked_process(self) -> None:
        lines = [
            "always_ff @(posedge clk) begin",
            "  state <= next_state;",
            "end",
            "logic later_signal;",
        ]
        contexts = sv_contexts(lines)
        self.assertIn("clocked process", contexts[2])
        self.assertEqual(contexts[4], "the module's structural declarations")

    def test_multiline_statements_are_explained_as_complete_operations(self) -> None:
        rtl_lines = [
            "always_ff @(posedge clk) begin",
            "  result_r <= left_r",
            "      + right_r;",
            "end",
        ]
        rtl_spans = sv_statement_spans(rtl_lines)
        self.assertEqual((rtl_spans[2].start, rtl_spans[2].end), (2, 3))
        self.assertIn("left_r + right_r", rtl_spans[3].text)

        python_text = "value = function(\n    operand_a,\n    operand_b,\n)\n"
        python_spans = python_statement_spans(python_text)
        self.assertEqual((python_spans[1].start, python_spans[1].end), (1, 4))
        self.assertIn("operand_a", python_spans[3].text)


if __name__ == "__main__":
    unittest.main()
```

63.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import json` This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 4: `import tempfile` This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import unittest` This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `from pathlib import Path` This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `from generate_source_reference import (` This statement continues through source line 15; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `generate,` This is continuation line 2 of the 8-line logical statement that begins at source line 8. Its exact fragment is ‘generate,‘. The important operands are generate: the value named ’generate’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `project_root,` This is continuation line 3 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'project_root,'. The important operands are project_root: the value named 'project root' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `python_statement_spans,` This is continuation line 4 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'python_statement_spans,'. The important operands are python_statement_spans: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `source_entries,` This is continuation line 5 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'source_entries,'. The important operands are source_entries: the value named 'source entries' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `sv_contexts,` This is continuation line 6 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'sv_contexts,'. The important operands are sv_contexts: the value named 'sv contexts' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `sv_statement_spans,` This is continuation line 7 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'sv_statement_spans,'. The important operands are sv_statement_spans: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 8. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks documentation generator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `class DocumentationGeneratorTests(unittest.TestCase):` This defines class DocumentationGeneratorTests, which groups data and methods for documentation generator tests. Instances exist in host memory and organize the flow that checks documentation generator behavior independently with Python assertions.

Line 19: `def test_every_nonblank_source_line_is_covered_and_hashed(self) -> None:` This starts function test_every_nonblank_source_line_is_covered_and_hashed. Its indented body performs test every nonblank source line is covered and hashed as part of the file that checks documentation generator behavior independently with Python assertions; it runs only when called.

Line 20: `with tempfile.TemporaryDirectory() as temporary_directory:` This opens a managed resource scope for 'tempfile.TemporaryDirectory() as temporary_directory' so files, tracing, or temporary state are finalized even if the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed raises an error.

Line 21: `tex_path, manifest_path = generate(Path(temporary_directory))` This assigns 'generate(Path(temporary_direc to 'tex_path, manifest_path ' in host memory. Here tex_path means the value named 'tex path' in this block. The important operands are generate: the value named 'generate' in this block; Path: the value named 'path' in this block; temporary_directory: the value named 'temporary directory' in this block. Context: the function test_every_nonblank_source_line_is_c which handles test every nonblank source line is covered and hashed.

Line 22: `manifest = json.loads(manifest_path.read_text(encoding="utf-8"))` This assigns ‘json.loads(manifest_path.rea 8"))‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are json: the value named ’json’ in this block; loads: the value named ’loads’ in this block; manifest_path: the value named ’manifest path’ in this block. Context: the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed.

Line 23: `entries = source_entries(project_root())` This assigns ‘source_entries(project_root())‘ to ‘entries ‘ in host memory. Here entries means the value named ’entries’ in this block. The important operands are source_entries: the value named ’source entries’ in this block; project_root: the value named ’project root’ in this block. Context: the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed.

Line 25: `self.assertEqual(manifest["summary"]["file_count"], len(entries))` This calls ‘self.assertEqual‘ with ‘manifest["summary"]["file_count"], len(entries)‘ inside the function test_every_nonblank_source_line_is_covered_and_hash which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 26: `self.assertEqual(` This statement continues through source line 29; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertEqual‘ with ‘ manifest["summary"]["nonblank_lines"], manifest["summary"]["explained_nonblank_lines"], ‘ inside the function test_every_nonblank_source_line_is_covered_and_hashe which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 27: `manifest["summary"]["nonblank_lines"],` This is continuation line 2 of the 4-line logical statement that begins at source line 26. Its exact fragment is ‘manifest["summary"]["nonblank_lines"],‘. The important operands are manifest: the value named ’manifest’ in this block; summary: the value named ’summary’ in this block; nonblank_lines: the value named ’nonblank lines’ in this block. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ manifest["summary"]["nonblank_lines"], manifest["summary"]["explained_nonblank_lines"], ‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 28: `manifest["summary"]["explained_nonblank_lines"],` This is continuation line 3 of the 4-line logical statement that begins at source line 26. Its exact fragment is ‘manifest["summary"]["explained_nonblank_lines"],‘. The important operands are manifest: the value named ’manifest’ in this block; summary: the value named ’summary’ in this block; explained_nonblank_lines: the value named ’explained nonblank lines’ in this block. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ manifest["summary"]["nonblank_lines"], manifest["summary"]["explained_nonblank_lines"], ‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 29: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 26. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ manifest["summary"]["nonblank_lines"], manifest["summary"]["explained_nonblank_lines"], ‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 30: `self.assertTrue(tex_path.read_text(encoding="utf-8").endswith("\\end{document}\n"))` This calls ‘self.assertTrue‘ with ‘tex_path.read_text(encoding="utf-8").endswith("\\end{document}\n")‘ inside the function test_every_nonb which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 31: `generated_tex = tex_path.read_text(encoding="utf-8")` This assigns ‘tex_path.read_text(encoding="utf-8")‘ to ‘generated_tex ‘ in host memory. Here generated_tex means the value named ’generated tex’ in this block. The

important operands are tex_path: the value named 'tex path' in this block; read_text: the value named 'read text' in this block; encoding: the value named 'encoding' in this block. Context: the function test_every_nonblank_source_line_is_covered_an
which handles test every nonblank source line is covered and hashed.

Line 32: `self.assertIn("Beginner architecture map", generated_tex)` This calls ‘self.assertIn‘ with ‘"Beginner architecture map", generated_tex‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 33: `self.assertIn("What becomes FPGA hardware", generated_tex)` This calls ‘self.assertIn‘ with ‘"What becomes FPGA hardware", generated_tex‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 34: `self.assertIn("Two unsigned byte weights", generated_tex)` This calls ‘self.assertIn‘ with ‘"Two unsigned byte weights", generated_tex‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 35: `listing_paths = [` This statement continues through source line 39; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘[line.removeprefix("\\lstinputlisting{").removesuffix("}") for line in generated_tex.splitlines() if line.startswith("\\lstinputlisting{")]‘ to ‘listing_paths ‘ in host memory. Here listing_paths means the value named 'listing paths' in this block. The important operands are line: the value named 'line' in this block; removeprefix: the value named 'removeprefix' in this block; lstinputlisting: the value named 'lstinputlisting' in this block. Context: the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed.

Line 36: `line.removeprefix("\\lstinputlisting{").removesuffix("}")` This is continuation line 2 of the 5-line logical statement that begins at source line 35. Its exact fragment is ‘line.removeprefix("\\lstinputlisting{").removesuffix("}")‘. The important operands are line: the value named 'line' in this block; removeprefix: the value named 'removeprefix' in this block; lstinputlisting: the value named 'lstinputlisting' in this block. Read together, the complete operation is: This assigns ‘[line.removeprefix("\\lstinputlisting{").removesuffix("}") for line in generated_tex.splitlines() if line.startswith("\\lstinputlisting{")]‘ to ‘listing_paths ‘ in host memory. Here listing_paths means the value named 'listing paths' in this block. The important operands are line: the value named 'line' in this block; removeprefix: the value named 'removeprefix' in this block; lstinputlisting: the value named 'lstinputlisting' in this block. Context: the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed.

Line 37: `for line in generated_tex.splitlines()` This is continuation line 3 of the 5-line logical statement that begins at source line 35. Its exact fragment is ‘for line in generated_tex.splitlines()‘. The important operands are for: the value named 'for' in this block; line: the value named 'line' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns ‘[line.removeprefix("\\lstinputlisting{").removesuffix("}") for line in generated_tex.splitlines() if line.startswith("\\lstinputlisting{")]‘ to ‘listing_paths ‘ in host memory. Here listing_paths means the value named 'listing paths' in this block. The important operands are line: the value named 'line' in this block; removeprefix: the value named 'removeprefix' in this block; lstinputlisting: the value named 'lstinputlisting' in this block. Context: the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed.

Line 38: `if line.startswith("\\lstinputlisting{")` This is continuation line 4 of the 5-line logical statement that begins at source line 35. Its exact fragment is ‘if line.startswith("\\lstinputlisting{")‘. The important operands are line: the value named 'line' in this block; startswith: the value named 'startswith' in this block; lstinputlisting: the value named 'lstinputlisting' in this block. Read together, the complete operation is: This assigns ‘[line.removeprefix("\\lstinputlisting{").removes
for line in generated_tex.splitlines() if line.startswith("\\lstinputlisting{")]‘ to ‘listing_paths ‘ in host memory. Here listing_paths means the value named 'listing paths' in this block. The important operands are line: the value named

'line' in this block; removeprefix: the value named 'removeprefix' in this block; lstinputlisting: the value named 'lstinputlisting' in this block. Context: the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed.

Line 39: `]` This is continuation line 5 of the 5-line logical statement that begins at source line 35. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '[line.removeprefix("\\lstinputlisting{").removesuffix("}") for line in generated_tex.splitlines() if line.startswith("\\lstinputlisting{")]' to 'listing_paths ' in host memory. Here listing_paths means the value named 'listing paths' in this block. The important operands are line: the value named 'line' in this block; removeprefix: the value named 'removeprefix' in this block; lstinputlisting: the value named 'lstinputlisting' in this block. Context: the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed.

Line 40: `self.assertEqual(len(listing_paths), len(entries))` This calls 'self.assertEqual' with 'len(listing_paths), len(entries)' inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 41: `for listing_path in listing_paths:` This iterates listing_path over 'listing_paths'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_every_nonblank_source_ which handles test every nonblank source line is covered and hashed.

Line 42: `self.assertTrue(` This statement continues through source line 45; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertTrue' with ' (tex_path.parent / listing_path).resolve().is_file(), listing_path, ' inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 43: `(tex_path.parent / listing_path).resolve().is_file(),` This is continuation line 2 of the 4-line logical statement that begins at source line 42. Its exact fragment is '(tex_path.parent / listing_path).resolve().is_file(),'. The important operands are tex_path: the value named 'tex path' in this block; parent: the value named 'parent' in this block; listing_path: the value named 'listing path' in this block. Read together, the complete operation is: This calls 'self.assertTrue' with ' (tex_path.parent / listing_path).resolve().is_file(), listing_path, ' inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 44: `listing_path,` This is continuation line 3 of the 4-line logical statement that begins at source line 42. Its exact fragment is 'listing_path,'. The important operands are listing_path: the value named 'listing path' in this block. Read together, the complete operation is: This calls 'self.assertTrue' with ' (tex_path.parent / listing_path).resolve().is_file(), listing_path, ' inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 45: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 42. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertTrue' with ' (tex_path.parent / listing_path).resolve().is_file(), listing_path, ' inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 46: `for file_record in manifest["files"]:` This iterates file_record over 'manifest["files"]'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_every_nonblank_source which handles test every nonblank source line is covered and hashed.

Line 47: `self.assertEqual(` This statement continues through source line 50; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertEqual‘ with ‘ file_record["nonblank_lines"], file_record["explained_nonblank_lines"], ‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 48: `file_record["nonblank_lines"],` This is continuation line 2 of the 4-line logical statement that begins at source line 47. Its exact fragment is ‘file_record["nonblank_lines"],‘. The important operands are file_record: the value named ’file record’ in this block; nonblank_lines: the value named ’nonblank lines’ in this block. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ file_record["nonblank_lines"], file_record["explained_nonblank_lines"], ‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 49: `file_record["explained_nonblank_lines"],` This is continuation line 3 of the 4-line logical statement that begins at source line 47. Its exact fragment is ‘file_record["explained_nonblank_lines"],‘. The important operands are file_record: the value named ’file record’ in this block; explained_nonblank_lines: the value named ’explained nonblank lines’ in this block. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ file_record["nonblank_lines"], file_record["explained_nonblank_lines"], ‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 50: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 47. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ file_record["nonblank_lines"], file_record["explained_nonblank_lines"], ‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 51: `self.assertEqual(len(file_record["sha256"]), 64)` This calls ‘self.assertEqual‘ with ‘len(file_record["sha256"]), 64‘ inside the function test_every_nonblank_source_line_is_covered_and_hashed, which handles test every nonblank source line is covered and hashed. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 53: `def test_rtl_context_returns_to_structure_after_clocked_process(self) -> None:` This starts function test_rtl_context_returns_to_structure_after_clocked_process. Its indented body performs test rtl context returns to structure after clocked process as part of the file that checks documentation generator behavior independently with Python assertions; it runs only when called.

Line 54: `lines = [` This statement continues through source line 59; the explanation uses the complete expression rather than only the first visual fragment. Inside the function test_rtl_context_returns_to_structure_after_clocked_process, which handles test rtl context returns to structure after clocked process, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are lines: the value named ’lines’ in this block; always_ff: the value named ’always ff’ in this block; posedge: the value named ’posedge’ in this block.

Line 55: `"always_ff @(posedge clk) begin",` This is continuation line 2 of the 6-line logical statement that begins at source line 54. Its exact fragment is ‘"always_ff @(posedge clk) begin",‘. The important operands are always_ff: the value named ’always ff’ in this block; posedge: the value named ’posedge’ in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: Inside the function test_rtl_context_returns_to_structure_after_ which handles test rtl context returns to structure after clocked process, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are lines: the value named ’lines’ in this block; always_ff: the value named ’always ff’ in this block; posedge: the value named ’posedge’ in this block.

Line 56: `" state <= next_state;",` This is continuation line 3 of the 6-line logical statement that begins at source line 54. Its exact fragment is ‘" state <= next_state;",‘. The important operands are state: the finite-state-machine phase retained in flip-flops; next_state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: Inside the function test_rtl_context_returns_to_structure_after_clocked_process, which handles test rtl context returns to structure after clocked process, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are lines: the value named 'lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 57: `"end",` This is continuation line 4 of the 6-line logical statement that begins at source line 54. Its exact fragment is ‘"end",‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function test_rtl_context_returns_to_structure_after_clocked_process, which handles test rtl context returns to structure after clocked process, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are lines: the value named 'lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 58: `"logic later_signal;",` This is continuation line 5 of the 6-line logical statement that begins at source line 54. Its exact fragment is ‘"logic later_signal;",‘. The important operands are later_signal: the value named 'later signal' in this block. Read together, the complete operation is: Inside the function test_rtl_context_returns_to_structure_after_clocked_pr which handles test rtl context returns to structure after clocked process, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are lines: the value named 'lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 59: `]` This is continuation line 6 of the 6-line logical statement that begins at source line 54. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function test_rtl_context_returns_to_structure_after_clocked_process, which handles test rtl context returns to structure after clocked process, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are lines: the value named 'lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 60: `contexts = sv_contexts(lines)` This assigns ‘sv_contexts(lines)‘ to ‘contexts ‘ in host memory. Here contexts means the value named 'contexts' in this block. The important operands are sv_contexts: the value named 'sv contexts' in this block; lines: the value named 'lines' in this block. Context: the function test_rtl_context_returns_to_structure_after_ which handles test rtl context returns to structure after clocked process.

Line 61: `self.assertIn("clocked process", contexts[2])` This calls ‘self.assertIn‘ with ‘"clocked process", contexts[2]‘ inside the function test_rtl_context_returns_to_structure_after_clocked_process, which handles test rtl context returns to structure after clocked process. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 62: `self.assertEqual(contexts[4], "the module's structural declarations")` This calls ‘self.assertEqual‘ with ‘contexts[4], "the module's structural declarations"‘ inside the function test_rtl_context_returns_to_structure_after_clocked_ which handles test rtl context returns to structure after clocked process. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 64: `def test_multiline_statements_are_explained_as_complete_operations(self) -> None:` This starts function test_multiline_statements_are_explained_as_complete_operations. Its indented body performs test multiline statements are explained as complete operations as part of the file that checks documentation generator behavior independently with Python assertions; it runs only when called.

Line 65: `rtl_lines = [` This statement continues through source line 70; the explanation uses the complete expression rather than only the first visual fragment. Inside the function test_multiline_statements_are_explained_as_complete_operations,

which handles test multiline statements are explained as complete operations, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are rtl_lines: the value named 'rtl lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 66: `"always_ff @(posedge clk) begin",` This is continuation line 2 of the 6-line logical statement that begins at source line 65. Its exact fragment is '"always_ff @(posedge clk) begin",'. The important operands are always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block; clk: the clock that makes all registers advance together. Read together, the complete operation is: Inside the function test_multiline_statements_are_explained_as_ which handles test multiline statements are explained as complete operations, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are rtl_lines: the value named 'rtl lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 67: `" result_r <= left_r",` This is continuation line 3 of the 6-line logical statement that begins at source line 65. Its exact fragment is '" result_r <= left_r",'. The important operands are result_r: the value named 'result r' in this block; left_r: the value named 'left r' in this block. Read together, the complete operation is: Inside the function test_multiline_statements_are_explained_as_complete_operations, which handles test multiline statements are explained as complete operations, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are rtl_lines: the value named 'rtl lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 68: `" + right_r;",` This is continuation line 4 of the 6-line logical statement that begins at source line 65. Its exact fragment is '" + right_r;",'. The important operands are right_r: the value named 'right r' in this block. Read together, the complete operation is: Inside the function test_multiline_statements_are_explained_as_complete_operations, which handles test multiline statements are explained as complete operations, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are rtl_lines: the value named 'rtl lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 69: `"end",` This is continuation line 5 of the 6-line logical statement that begins at source line 65. Its exact fragment is '"end",'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function test_multiline_statements_are_explained_as_complete_operations, which handles test multiline statements are explained as complete operations, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are rtl_lines: the value named 'rtl lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 70: `]` This is continuation line 6 of the 6-line logical statement that begins at source line 65. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function test_multiline_statements_are_explained_as_complete_operations, which handles test multiline statements are explained as complete operations, this statement advances the host implementation that checks documentation generator behavior independently with Python assertions. The important operands are rtl_lines: the value named 'rtl lines' in this block; always_ff: the value named 'always ff' in this block; posedge: the value named 'posedge' in this block.

Line 71: `rtl_spans = sv_statement_spans(rtl_lines)` This assigns 'sv_statement_spans(rtl_lines)' to 'rtl_spans ' in host memory. Here rtl_spans means the value named 'rtl spans' in this block. The important operands are sv_statement_spans: the finite-state-machine phase retained in flip-flops; rtl_lines: the value named 'rtl lines' in this block. Context: the function test_multiline_statements_are_explained_as_complete_operations, which handles test multiline statements are explained as complete operations.

Line 72: `self.assertEqual((rtl_spans[2].start, rtl_spans[2].end), (2, 3))` This calls 'self.assertEqual' with '(rtl_spans[2].start, rtl_spans[2].end), (2, 3)' inside the function test_multiline_statements_are_explained_as_complete_operatio which handles test multiline statements are explained as complete operations. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 73: `self.assertIn("left_r + right_r", rtl_spans[3].text)` This calls ‘self.assertIn‘ with ‘"left_r + right_r", rtl_spans[3].text‘ inside the function test_multiline_statements_are_explained_as_complete_operations, which handles test multiline statements are explained as complete operations. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 75: `python_text = "value = function(\n operand_a,\n operand_b,\n)\n"` This assigns ‘"value = function(\n operand_a,\n operand_b,\n)\n"‘ to ‘python_text ‘ in host memory. Here python_text means the value named ’python text’ in this block. The important operands are value: the value named ’value’ in this block; function: the value named ’function’ in this block; n: the value named ’n’ in this block. Context: the function test_multiline_statements_are_explained which handles test multiline statements are explained as complete operations.

Line 76: `python_spans = python_statement_spans(python_text)` This assigns ‘python_statement_spans(python_text)‘ to ‘python_spans ‘ in host memory. Here python_spans means the value named ’python spans’ in this block. The important operands are python_statement_spans: the finite-state-machine phase retained in flip-flops; python_text: the value named ’python text’ in this block. Context: the function test_multiline_statements_are_explained_as_complete_operations, which handles test multiline statements are explained as complete operations.

Line 77: `self.assertEqual((python_spans[1].start, python_spans[1].end), (1, 4))` This calls ‘self.assertEqual‘ with ‘(python_spans[1].start, python_spans[1].end), (1, 4)‘ inside the function test_multiline_statements_are_explained_as_comp which handles test multiline statements are explained as complete operations. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 78: `self.assertIn("operand_a", python_spans[3].text)` This calls ‘self.assertIn‘ with ‘"operand_a", python_spans[3].t inside the function test_multiline_statements_are_explained_as_complete_operations, which handles test multiline statements are explained as complete operations. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

Line 81: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks documentation generator behavior independently with Python assertions.

Line 82: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks documentation generator behavior independently with Python assertions. The call advances the host-side job that checks documentation generator behavior independently with Python assertions.

64 tests/test_neuraedge_compiler.py

Role in the complete system: checks neuraedge compiler behavior independently with Python assertions.

64.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks neuraedge compiler behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: TensorTransactionEmulator, CompilerTests. Functions: _decode_bytes, direct_conv2d, ___init___, execute, execute_program, assert_shape, setUp, check_case, test_outer_mode_irregular_tiles_both_dataflows, test_dot8_single_row_with_padding, test_reduction_larger_than_local_memory, test_zero_skipping_and_boundary_va test_binary_and_ternary_sparse_paths, test_grouped_u8_convolution_with_stride_padding_and_dilation, test_depthw test_validation. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

64.2 Complete source

```
from __future__ import annotations

import unittest

import numpy as np

from neuraedge_compiler import (
    compile_spike_conv2d,
    compile_spike_gemm,
    compile_u8_conv2d,
    compile_u8_gemm,
)
from neuraedge_golden import Dataflow, direct_centered_gemm
from neuraedge_protocol import Command128, Opcode, StreamBeat


def _decode_bytes(word: int) -> list[int]:
    return [(word >> (8 * lane)) & 0xFF for lane in range(8)]


class TensorTransactionEmulator:
    """Independent stateful interpretation of the public tensor stream ABI."""

    def __init__(self) -> None:
        self.activations: np.ndarray | None = None
        self.weights: np.ndarray | None = None

    def execute(
        self,
        command: Command128,
        input_beats: tuple[StreamBeat, ...] = (),
        *,
        output_beats: int = 0,
        timeout_seconds: float = 5.0,
    ) -> tuple[int, ...]:
        del timeout_seconds
        value = command.value
        opcode = command.opcode
        issues = (value >> 14) & 0xFFF

        if opcode == Opcode.LOAD_ACTIVATIONS:
            self.activations = np.asarray([_decode_bytes(beat.data) for beat in input_beats], dtype=np.int64)
            self.assert_shape(self.activations, (issues, 8))
            return ()
        if opcode == Opcode.LOAD_WEIGHTS_OUTER:
            words = [_decode_bytes(beat.data) for beat in input_beats]
            self.weights = np.asarray(
                [words[2 * issue] + words[2 * issue + 1] for issue in range(issues)],
                dtype=np.int64,
```

```
            )
            return ()
        if opcode == Opcode.LOAD_WEIGHTS_DOT8:
            words = [_decode_bytes(beat.data) for beat in input_beats]
            self.weights = np.asarray([
                [words[16 * issue + 2 * lane] + words[16 * issue + 2 * lane + 1]
                 for lane in range(8)]
                for issue in range(issues)
            ], dtype=np.int64)
            return ()

        if self.activations is None or self.weights is None:
            raise AssertionError("compute was issued before operand loads")
        rows = (value >> 4) & 0xF
        columns = (value >> 8) & 0x3F
        za = (value >> 26) & 0xFF
        zb = (value >> 34) & 0xFF
        if opcode == Opcode.COMPUTE_DENSE_OUTER:
            result = (self.activations[:, :rows].T - za) @ (self.weights[:, :columns] - zb)
        elif opcode == Opcode.COMPUTE_DENSE_DOT8:
            lhs = self.activations.reshape(-1) - za
            rhs = self.weights.reshape(-1, 16)[:, :columns] - zb
            result = (lhs @ rhs).reshape(1, columns)
        elif opcode in {Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY}:
            lhs = self.activations[:, :rows].T.copy()
            if opcode == Opcode.COMPUTE_TERNARY:
                lhs[lhs >= 128] -= 256
            rhs = self.weights[:, :columns].copy()
            rhs[rhs >= 128] -= 256
            result = lhs @ rhs
        else:
            raise AssertionError(f"unexpected opcode {opcode}")

        words = []
        for row in range(rows):
            for column in range(columns):
                encoded = int(result[row, column]) & ((1 << 48) - 1)
                encoded |= row << 48
                encoded |= column << 52
                encoded |= int(len(words) == output_beats - 1) << 58
                words.append(encoded)
        if len(words) != output_beats:
            raise AssertionError("compiler requested the wrong output beat count")
        return tuple(words)

    def execute_program(
        self,
        records,
        *,
        timeout_seconds: float = 30.0,
    ) -> tuple[int, ...]:
        words: list[int] = []
        for record in records:
            words.extend(self.execute(
                record.command,
                record.input_beats,
                output_beats=record.output_beats,
                timeout_seconds=timeout_seconds,
            ))
        return tuple(words)

    @staticmethod
    def assert_shape(array: np.ndarray, shape: tuple[int, ...]) -> None:
        if array.shape != shape:
            raise AssertionError(f"stream shape {array.shape} != {shape}")


def direct_conv2d(
    activations: np.ndarray,
    weights: np.ndarray,
    bias: np.ndarray,
    *,
    activation_zero_point: int = 0,
    weight_zero_point: int = 0,
    stride: tuple[int, int] = (1, 1),
    padding: tuple[int, int] = (0, 0),
    dilation: tuple[int, int] = (1, 1),
    groups: int = 1,
) -> np.ndarray:
    batch, input_channels, height, width = activations.shape
    output_channels, channels_per_group, kernel_height, kernel_width = weights.shape
    outputs_per_group = output_channels // groups
    output_height = (height + 2 * padding[0] - dilation[0] * (kernel_height - 1) - 1) // stride[0] + 1
    output_width = (width + 2 * padding[1] - dilation[1] * (kernel_width - 1) - 1) // stride[1] + 1
    output = np.zeros((batch, output_channels, output_height, output_width), dtype=np.int64)
    for n in range(batch):
        for output_channel in range(output_channels):
```

```
            group = output_channel // outputs_per_group
            input_start = group * channels_per_group
            for output_y in range(output_height):
                for output_x in range(output_width):
                    accumulator = int(bias[output_channel])
                    for local_channel in range(channels_per_group):
                        input_channel = input_start + local_channel
                        for kernel_y in range(kernel_height):
                            input_y = output_y * stride[0] + kernel_y * dilation[0] - padding[0]
                            for kernel_x in range(kernel_width):
                                input_x = output_x * stride[1] + kernel_x * dilation[1] - padding[1]
                                activation = activation_zero_point
                                if 0 <= input_y < height and 0 <= input_x < width:
                                    activation = int(activations[n, input_channel, input_y, input_x])
                                weight = int(weights[output_channel, local_channel, kernel_y, kernel_x])
                                accumulator += ((activation - activation_zero_point)
                                                * (weight - weight_zero_point))
                    output[n, output_channel, output_y, output_x] = accumulator
    return output


class CompilerTests(unittest.TestCase):
    def setUp(self) -> None:
        self.rng = np.random.default_rng(20260713)

    def check_case(self, rows: int, reduction: int, columns: int, dataflow: Dataflow | None = None) -> None:
        a = self.rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)
        b = self.rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)
        bias = self.rng.integers(-100000, 100001, size=columns, dtype=np.int64)
        za = int(self.rng.integers(0, 256))
        zb = int(self.rng.integers(0, 256))
        program = compile_u8_gemm(
            a, b, activation_zero_point=za, weight_zero_point=zb, bias=bias,
            dataflow=dataflow,
        )
        actual = program.execute(TensorTransactionEmulator())
        batched = program.execute_batched(TensorTransactionEmulator())
        expected = direct_centered_gemm(a, b, za, zb, bias)
        np.testing.assert_array_equal(actual, expected)
        np.testing.assert_array_equal(batched, expected)
        self.assertEqual(program.mathematical_macs, rows * reduction * columns)
        self.assertGreater(program.estimated_input_bytes, 0)

    def test_outer_mode_irregular_tiles_both_dataflows(self) -> None:
        self.check_case(13, 37, 29, Dataflow.INPUT_BROADCAST)
        self.check_case(17, 33, 19, Dataflow.WEIGHT_BROADCAST)

    def test_dot8_single_row_with_padding(self) -> None:
        self.check_case(1, 19, 35)

    def test_reduction_larger_than_local_memory(self) -> None:
        self.check_case(9, 2053, 17)

    def test_zero_skipping_and_boundary_values(self) -> None:
        a = np.full((8, 9), 128, dtype=np.uint8)
        b = np.full((9, 16), 129, dtype=np.uint8)
        a[3, 4] = 255
        b[4, 7] = 0
        program = compile_u8_gemm(a, b, activation_zero_point=128, weight_zero_point=129)
        np.testing.assert_array_equal(
            program.execute(TensorTransactionEmulator()),
            direct_centered_gemm(a, b, 128, 129),
        )

    def test_binary_and_ternary_sparse_paths(self) -> None:
        weights = self.rng.integers(-128, 128, size=(41, 23), dtype=np.int16)
        binary = self.rng.integers(0, 2, size=(11, 41), dtype=np.int8)
        ternary = self.rng.integers(-1, 2, size=(9, 41), dtype=np.int8)
        binary_program = compile_spike_gemm(binary, weights)
        ternary_program = compile_spike_gemm(
            ternary, weights, ternary=True, dataflow=Dataflow.WEIGHT_BROADCAST,
        )
        np.testing.assert_array_equal(
            binary_program.execute(TensorTransactionEmulator()),
            binary.astype(np.int64) @ weights.astype(np.int64),
        )
        np.testing.assert_array_equal(
            ternary_program.execute(TensorTransactionEmulator()),
            ternary.astype(np.int64) @ weights.astype(np.int64),
        )
        self.assertEqual(binary_program.estimated_spike_adds, int(np.count_nonzero(binary)) * 23)
        self.assertEqual(ternary_program.estimated_spike_adds, int(np.count_nonzero(ternary)) * 23)

    def test_grouped_u8_convolution_with_stride_padding_and_dilation(self) -> None:
        activations = self.rng.integers(0, 256, size=(1, 4, 5, 6), dtype=np.uint8)
        weights = self.rng.integers(0, 256, size=(6, 2, 2, 3), dtype=np.uint8)
        bias = self.rng.integers(-5000, 5001, size=6, dtype=np.int64)
```

```
        program = compile_u8_conv2d(
            activations,
            weights,
            activation_zero_point=117,
            weight_zero_point=133,
            bias=bias,
            stride=(2, 1),
            padding=(1, 2),
            dilation=(2, 1),
            groups=2,
        )
        actual = program.execute(TensorTransactionEmulator())
        expected = direct_conv2d(
            activations, weights, bias,
            activation_zero_point=117, weight_zero_point=133,
            stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,
        )
        np.testing.assert_array_equal(actual, expected)
        self.assertEqual(actual.shape, (1, 6, 3, 8))

    def test_depthwise_binary_and_ternary_spike_convolution(self) -> None:
        binary = self.rng.integers(0, 2, size=(1, 3, 4, 4), dtype=np.int8)
        ternary = self.rng.integers(-1, 2, size=(1, 3, 4, 4), dtype=np.int8)
        weights = self.rng.integers(-128, 128, size=(3, 1, 3, 3), dtype=np.int16)
        bias = self.rng.integers(-50, 51, size=3, dtype=np.int64)
        for spikes, is_ternary in ((binary, False), (ternary, True)):
            program = compile_spike_conv2d(
                spikes, weights, ternary=is_ternary, bias=bias,
                padding=1, groups=3,
            )
            actual = program.execute(TensorTransactionEmulator())
            expected = direct_conv2d(
                spikes, weights, bias, padding=(1, 1), groups=3,
            )
            np.testing.assert_array_equal(actual, expected)
            self.assertEqual(program.estimated_spike_adds,
                             sum(group.gemm.estimated_spike_adds for group in program.group_programs))

    def test_validation(self) -> None:
        with self.assertRaises(ValueError):
            compile_u8_gemm([[1, 2]], [[1, 2]])
        with self.assertRaises(ValueError):
            compile_u8_gemm([[256]], [[1]])
        with self.assertRaises(ValueError):
            compile_u8_gemm([[1]], [[1]], bias=[1, 2])
        with self.assertRaises(ValueError):
            compile_spike_gemm([[0, 2]], [[1], [2]])
        with self.assertRaises(ValueError):
            compile_spike_gemm([[0, -1]], [[1], [2]], ternary=False)
        with self.assertRaises(ValueError):
            compile_u8_conv2d(np.zeros((1, 3, 4, 4), dtype=np.uint8),
                              np.zeros((4, 2, 3, 3), dtype=np.uint8), groups=2)


if __name__ == "__main__":
    unittest.main()
```

64.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import unittest` This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import numpy as np` This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from neuraedge_compiler import (` This statement continues through source line 12; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `compile_spike_conv2d,` This is continuation line 2 of the 6-line logical statement that begins at source line 7. Its exact fragment is 'compile_spike_conv2d,'. The important operands are compile_spike_conv2d: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `compile_spike_gemm,` This is continuation line 3 of the 6-line logical statement that begins at source line 7. Its exact fragment is 'compile_spike_gemm,'. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `compile_u8_conv2d,` This is continuation line 4 of the 6-line logical statement that begins at source line 7. Its exact fragment is 'compile_u8_conv2d,'. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `compile_u8_gemm,` This is continuation line 5 of the 6-line logical statement that begins at source line 7. Its exact fragment is 'compile_u8_gemm,'. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 7. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `from neuraedge_golden import Dataflow, direct_centered_gemm` This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `from neuraedge_protocol import Command128, Opcode, StreamBeat` This imports the named Python API so the file whose role is to checks neuraedge compiler behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `def _decode_bytes(word: int) -> list[int]:` This starts function _decode_bytes. Its indented body performs decode bytes as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 18: `return [(word >> (8 * lane)) & 0xFF for lane in range(8)]` This returns '[(word >> (8 * lane)) & 0xFF for lane in range(8)]' to the caller of the function _decode_bytes, which handles decode bytes, ending this invocation. The important operands are word: the value named 'word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; for: the value named 'for' in this block.

Line 21: `class TensorTransactionEmulator:` This defines class TensorTransactionEmulator, which groups data and methods for tensor transaction emulator. Instances exist in host memory and organize the flow that checks neuraedge compiler behavior independently with Python assertions.

Line 22: `"""Independent stateful interpretation of the public tensor stream ABI."""` This is executable documentation attached to the class TensorTransactionEmulator, which handles tensor transaction emulator. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 24: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 25: `self.activations: np.ndarray | None = None` This assigns 'None' to 'self.activations' in host memory. Here activations means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are None: the value named 'none' in this block. Context: the function ___init___, which handles init.

Line 26: `self.weights: np.ndarray | None = None` This assigns 'None' to 'self.weights' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are None: the value named 'none' in this block. Context: the function ___init___, which handles init.

Line 28: `def execute(` This starts function execute. Its indented body performs execute as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 29: `self,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 30: `command: Command128,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 31: `input_beats: tuple[StreamBeat, ...] = (),` This assigns '(),' to 'input_beats' in host memory. Here input_beats means the value named 'input beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 32: `*,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 33: `output_beats: int = 0,` This assigns '0,' to 'output_beats' in host memory. Here output_beats means the value named 'output beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 34: `timeout_seconds: float = 5.0,` This assigns '5.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 35: `) -> tuple[int, ...]:` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are tuple: the value named 'tuple' in this block.

Line 36: `del timeout_seconds` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are del: the value named 'del' in this block; timeout_seconds: the value named 'timeout seconds' in this block.

Line 37: `value = command.value` This assigns 'command.value' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 38: `opcode = command.opcode` This assigns ‘command.opcode‘ to ‘opcode ‘ in host memory. Here opcode means the value named ’opcode’ in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; opcode: the value named ’opcode’ in this block. Context: the function execute, which handles execute.

Line 39: `issues = (value >> 14) & 0xFFF` This assigns ‘(value >> 14) & 0xFFF‘ to ‘issues ‘ in host memory. Here issues means the value named ’issues’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 41: `if opcode == Opcode.LOAD_ACTIVATIONS:` This executes the following indented host-code branch only when ‘opcode == Opcode.LOAD_ACTIVATIONS‘ is true. The important operands are opcode: the value named ’opcode’ in this block; Opcode: the value named ’opcode’ in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function execute, which handles execute.

Line 42: `self.activations = np.asarray([_decode_bytes(beat.data) for beat in input_beats], dtype=np.int64)` This assigns ‘np.asarray([_decode_bytes(beat.data) for beat in input_beats], dtype=np.int64)‘ to ‘self.activations ‘ in host memory. Here activations means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are asarray: the value named ’asarray’ in this block; _decode_bytes: the value named ’decode bytes’ in this block; beat: the value named ’beat’ in this block. Context: the function execute, which handles execute.

Line 43: `self.assert_shape(self.activations, (issues, 8))` This calls ‘self.assert_shape‘ with ‘self.activations, (issues, 8)‘ inside the function execute, which handles execute. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 44: `return ()` This returns ‘()‘ to the caller of the function execute, which handles execute, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 45: `if opcode == Opcode.LOAD_WEIGHTS_OUTER:` This executes the following indented host-code branch only when ‘opcode == Opcode.LOAD_WEIGHTS_OUTER‘ is true. The important operands are opcode: the value named ’opcode’ in this block; Opcode: the value named ’opcode’ in this block; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation. Context: the function execute, which handles execute.

Line 46: `words = [_decode_bytes(beat.data) for beat in input_beats]` This assigns ‘[_decode_bytes(beat.data) for beat in input_beats]‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are _decode_bytes: the value named ’decode bytes’ in this block; beat: the value named ’beat’ in this block; data: the value named ’data’ in this block. Context: the function execute, which handles execute.

Line 47: `self.weights = np.asarray(` This statement continues through source line 50; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.asarray([words[2 * issue] + words[2 * issue + 1] for issue in range(issues)], dtype=np.int64,)‘ to ‘self.weights ‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; words: the value named ’words’ in this block; issue: the value named ’issue’ in this block. Context: the function execute, which handles execute.

Line 48: `[words[2 * issue] + words[2 * issue + 1] for issue in range(issues)],` This is continuation line 2 of the 4-line logical statement that begins at source line 47. Its exact fragment is ‘[words[2 * issue] + words[2 * issue + 1] for issue in range(issues)],‘. The important operands are words: the value named ’words’ in this block; issue: the value named ’issue’ in this block; for: the value named ’for’ in this block. Read together, the complete operation is: This assigns ‘np.asarray([words[2 * issue] + words[2 * issue + 1] for issue in range(issues)], dtype=np.int64,)‘ to ‘self.weights ‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; words: the value named ’words’ in this block; issue: the value named ’issue’ in this block. Context: the function execute, which handles execute.

Line 49: `dtype=np.int64,` This is continuation line 3 of the 4-line logical statement that begins at source line 47. Its exact fragment is 'dtype=np.int64,'. The important operands are dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'np.asarray([words[2 * issue] + words[2 * issue + 1] for issue in range(issues)], dtype=np.int64,)' to 'self.weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are asarray: the value named 'asarray' in this block; words: the value named 'words' in this block; issue: the value named 'issue' in this block. Context: the function execute, which handles execute.

Line 50: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 47. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'np.asarray([words[2 * issue] + words[2 * issue + 1] for issue in range(issues)], dtype=np.int64,)' to 'self.weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are asarray: the value named 'asarray' in this block; words: the value named 'words' in this block; issue: the value named 'issue' in this block. Context: the function execute, which handles execute.

Line 51: `return ()` This returns '()' to the caller of the function execute, which handles execute, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 52: `if opcode == Opcode.LOAD_WEIGHTS_DOT8:` This executes the following indented host-code branch only when 'opcode == Opcode.LOAD_WEIGHTS_DOT8' is true. The important operands are opcode: the value named 'opcode' in this block; Opcode: the value named 'opcode' in this block; LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation. Context: the function execute, which handles execute.

Line 53: `words = [_decode_bytes(beat.data) for beat in input_beats]` This assigns '[_decode_bytes(beat.data) for beat in input_beats]' to 'words ' in host memory. Here words means the value named 'words' in this block. The important operands are _decode_bytes: the value named 'decode bytes' in this block; beat: the value named 'beat' in this block; data: the value named 'data' in this block. Context: the function execute, which handles execute.

Line 54: `self.weights = np.asarray([` This statement continues through source line 58; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.asarray([[words[16 * issue + 2 * lane] + words[16 * issue + 2 * lane + 1] for lane in range(8)] for issue in range(issues)], dtype=np.int64)' to 'self.weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are asarray: the value named 'asarray' in this block; words: the value named 'words' in this block; issue: the value named 'issue' in this block. Context: the function execute, which handles execute.

Line 55: `[words[16 * issue + 2 * lane] + words[16 * issue + 2 * lane + 1]` This is continuation line 2 of the 5-line logical statement that begins at source line 54. Its exact fragment is '[words[16 * issue + 2 * lane] + words[16 * issue + 2 * lane + 1]'. The important operands are words: the value named 'words' in this block; issue: the value named 'issue' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'np.asarray([[words[16 * issue + 2 * lane] + words[16 * issue + 2 * lane + 1] for lane in range(8)] for issue in range(issues)], dtype=np.int64)' to 'self.weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are asarray: the value named 'asarray' in this block; words: the value named 'words' in this block; issue: the value named 'issue' in this block. Context: the function execute, which handles execute.

Line 56: `for lane in range(8)]` This is continuation line 3 of the 5-line logical statement that begins at source line 54. Its exact fragment is 'for lane in range(8)]'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'np.asarray([[words[16 * issue + 2 * lane] + words[16 * issue + 2 * lane + 1] for lane in range(8)] for issue in range(issues)], dtype=np.int64)' to 'self.weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are asarray: the value named 'asarray' in this block; words: the value named 'words' in this block; issue: the value named 'issue' in this block. Context: the function execute, which handles execute.

Line 57: `for issue in range(issues)` This is continuation line 4 of the 5-line logical statement that begins at source line 54. Its exact fragment is ‘for issue in range(issues)‘. The important operands are for: the value named ’for’ in this block; issue: the value named ’issue’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘np.asarray([[words[16 * issue + 2 * lane] + words[16 * issue + 2 * lane + 1] for lane in range(8)] for issue in range(issues)], dtype=np.int64)‘ to ‘self.weights ‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; words: the value named ’words’ in this block; issue: the value named ’issue’ in this block. Context: the function execute, which handles execute.

Line 58: `], dtype=np.int64)` This is continuation line 5 of the 5-line logical statement that begins at source line 54. Its exact fragment is ‘], dtype=np.int64)‘. The important operands are dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This assigns ‘np.asarray([[words[16 * issue + 2 * lane] + words[16 * issue + 2 * lane + 1] for lane in range(8)] for issue in range(issues)], dtype=np.int64)‘ to ‘self.weights ‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are asarray: the value named ’asarray’ in this block; words: the value named ’words’ in this block; issue: the value named ’issue’ in this block. Context: the function execute, which handles execute.

Line 59: `return ()` This returns ‘()‘ to the caller of the function execute, which handles execute, ending this invocation. The expression is a literal or structural operator and has no hidden software state.

Line 61: `if self.activations is None or self.weights is None:` This executes the following indented host-code branch only when ‘self.activations is None or self.weights is None‘ is true. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function execute, which handles execute.

Line 62: `raise AssertionError("compute was issued before operand loads")` This rejects an invalid state by raising ‘AssertionError("compute was issued before operand loads")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 63: `rows = (value >> 4) & 0xF` This assigns ‘(value >> 4) & 0xF‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 64: `columns = (value >> 8) & 0x3F` This assigns ‘(value >> 8) & 0x3F‘ to ‘columns ‘ in host memory. Here columns means one output-feature position processed in parallel. The important operands are value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 65: `za = (value >> 26) & 0xFF` This assigns ‘(value >> 26) & 0xFF‘ to ‘za ‘ in host memory. Here za means the value named ’za’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 66: `zb = (value >> 34) & 0xFF` This assigns ‘(value >> 34) & 0xFF‘ to ‘zb ‘ in host memory. Here zb means the value named ’zb’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 67: `if opcode == Opcode.COMPUTE_DENSE_OUTER:` This executes the following indented host-code branch only when ‘opcode == Opcode.COMPUTE_DENSE_OUTER‘ is true. The important operands are opcode: the value named ’opcode’ in this block; Opcode: the value named ’opcode’ in this block; COMPUTE_DENSE_OUTER: the value named ’compute dense outer’ in this block. Context: the function execute, which handles execute.

Line 68: `result = (self.activations[:, :rows].T - za) @ (self.weights[:, :columns] - zb)` This assigns ‘(self.activations[:, :rows].T - za) @ (self.weights[:, :columns] - zb)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; rows: one output-row position processed by the tensor tile; T: the value named ’t’ in this block. Context: the function execute, which handles execute.

Line 69: `elif opcode == Opcode.COMPUTE_DENSE_DOT8:` This executes the following indented host-code branch only when ‘opcode == Opcode.COMPUTE_DENSE_DOT8‘ is true. The important operands are opcode: the value named ’opcode’ in this block; Opcode: the value named ’opcode’ in this block; COMPUTE_DENSE_DOT8: the value named ’compute dense dot8’ in this block. Context: the function execute, which handles execute.

Line 70: `lhs = self.activations.reshape(-1) - za` This assigns ‘self.activations.reshape(-1) - za‘ to ‘lhs ‘ in host memory. Here lhs means the value named ’lhs’ in this block. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; reshape: the value named ’reshape’ in this block; za: the value named ’za’ in this block. Context: the function execute, which handles execute.

Line 71: `rhs = self.weights.reshape(-1, 16)[:, :columns] - zb` This assigns ‘self.weights.reshape(-1, 16)[:, :columns] - zb‘ to ‘rhs ‘ in host memory. Here rhs means the value named ’rhs’ in this block. The important operands are weights: a learned coefficient multiplied by an activation; reshape: the value named ’reshape’ in this block; columns: one output-feature position processed in parallel. Context: the function execute, which handles execute.

Line 72: `result = (lhs @ rhs).reshape(1, columns)` This assigns ‘(lhs @ rhs).reshape(1, columns)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are lhs: the value named ’lhs’ in this block; rhs: the value named ’rhs’ in this block; reshape: the value named ’reshape’ in this block. Context: the function execute, which handles execute.

Line 73: `elif opcode in {Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY}:` This executes the following indented host-code branch only when ‘opcode in {Opcode.COMPUTE_BINARY, Opcode.COMPUTE_TERNARY}‘ is true. The important operands are opcode: the value named ’opcode’ in this block; in: the value named ’in’ in this block; Opcode: the value named ’opcode’ in this block. Context: the function execute, which handles execute.

Line 74: `lhs = self.activations[:, :rows].T.copy()` This assigns ‘self.activations[:, :rows].T.copy()‘ to ‘lhs ‘ in host memory. Here lhs means the value named ’lhs’ in this block. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; rows: one output-row position processed by the tensor tile; T: the value named ’t’ in this block. Context: the function execute, which handles execute.

Line 75: `if opcode == Opcode.COMPUTE_TERNARY:` This executes the following indented host-code branch only when ‘opcode == Opcode.COMPUTE_TERNARY‘ is true. The important operands are opcode: the value named ’opcode’ in this block; Opcode: the value named ’opcode’ in this block; COMPUTE_TERNARY: the value named ’compute ternary’ in this block. Context: the function execute, which handles execute.

Line 76: `lhs[lhs >= 128] -= 256` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are lhs: the value named ’lhs’ in this block.

Line 77: `rhs = self.weights[:, :columns].copy()` This assigns ‘self.weights[:, :columns].copy()‘ to ‘rhs ‘ in host memory. Here rhs means the value named ’rhs’ in this block. The important operands are weights: a learned coefficient multiplied by an activation; columns: one output-feature position processed in parallel; copy: the value named ’copy’ in this block. Context: the function execute, which handles execute.

Line 78: `rhs[rhs >= 128] -= 256` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are rhs: the value named ’rhs’ in this block.

Line 79: `result = lhs @ rhs` This assigns ‘lhs @ rhs‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are lhs: the value named ’lhs’ in this block; rhs: the value named ’rhs’ in this block. Context: the function execute, which handles execute.

Line 80: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute, which handles execute.

Line 81: `raise AssertionError(f"unexpected opcode {opcode}")` This rejects an invalid state by raising ‘AssertionError(f"unexpected opcode {opcode}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 83: `words = []` This assigns ‘[]‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 84: `for row in range(rows):` This iterates row over ‘range(rows)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 85: `for column in range(columns):` This iterates column over ‘range(columns)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 86: `encoded = int(result[row, column]) & ((1 << 48) - 1)` This assigns ‘int(result[row, column]) & ((1 << 48) - 1)‘ to ‘encoded ‘ in host memory. Here encoded means the value named ’encoded’ in this block. The important operands are result: the value named ’result’ in this block; row: one output-row position processed by the tensor tile; column: one output-feature position processed in parallel. Context: the function execute, which handles execute.

Line 87: `encoded |= row << 48` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are encoded: the value named ’encoded’ in this block; row: one output-row position processed by the tensor tile.

Line 88: `encoded |= column << 52` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are encoded: the value named ’encoded’ in this block; column: one output-feature position processed in parallel.

Line 89: `encoded |= int(len(words) == output_beats - 1) << 58` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are encoded: the value named ’encoded’ in this block; len: the value named ’len’ in this block; words: the value named ’words’ in this block.

Line 90: `words.append(encoded)` This calls ‘words.append‘ with ‘encoded‘ inside the function execute, which handles execute. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 91: `if len(words) != output_beats:` This executes the following indented host-code branch only when ‘len(words) != output_beats‘ is true. The important operands are len: the value named ’len’ in this block; words: the value named ’words’ in this block; output_beats: the value named ’output beats’ in this block. Context: the function execute, which handles execute.

Line 92: `raise AssertionError("compiler requested the wrong output beat count")` This rejects an invalid state by raising ‘AssertionError("compiler requested the wrong output beat count")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 93: `return tuple(words)` This returns ‘tuple(words)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; words: the value named ’words’ in this block.

Line 95: `def execute_program(` This starts function execute_program. Its indented body performs execute program as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 96: `self,` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 97: `records,` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are records: the number of descriptor records accepted since program start.

Line 98: `*,` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 99: `timeout_seconds: float = 30.0,` This assigns ‘30.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 100: `) -> tuple[int, ...]:` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are tuple: the value named ’tuple’ in this block.

Line 101: `words: list[int] = []` This assigns ‘[]‘ to ‘words‘ in host memory. Here words means the value named ’words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 102: `for record in records:` This iterates record over ‘records‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_program, which handles execute program.

Line 103: `words.extend(self.execute(` This statement continues through source line 108; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘words.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 104: `record.command,` This is continuation line 2 of the 6-line logical statement that begins at source line 103. Its exact fragment is ‘record.command,‘. The important operands are record: the value named ’record’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘words.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 105: `record.input_beats,` This is continuation line 3 of the 6-line logical statement that begins at source line 103. Its exact fragment is ‘record.input_beats,‘. The important operands are record: the value named ’record’ in this block; input_beats: the value named ’input beats’ in this block. Read together, the complete operation is: This calls ‘words.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 106: `output_beats=record.output_beats,` This is continuation line 4 of the 6-line logical statement that begins at source line 103. Its exact fragment is ‘output_beats=record.output_beats,‘. The important operands are output_beats: the value named ’output beats’ in this block; record: the value named ’record’ in this block. Read together, the complete operation is: This calls ‘words.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which

handles execute program. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 107: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 103. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This calls 'words.extend' with 'self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)' inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 108: `))` This is continuation line 6 of the 6-line logical statement that begins at source line 103. Its exact fragment is '))'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'words.extend' with 'self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)' inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 109: `return tuple(words)` This returns 'tuple(words)' to the caller of the function execute_program, which handles execute program, ending this invocation. The important operands are tuple: the value named 'tuple' in this block; words: the value named 'words' in this block.

Line 111: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class TensorTransactionEmulator, which handles tensor transaction emulator; it creates no FPGA circuit by itself.

Line 112: `def assert_shape(array: np.ndarray, shape: tuple[int, ...]) -> None:` This starts function assert_shape. Its indented body performs assert shape as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 113: `if array.shape != shape:` This executes the following indented host-code branch only when 'array.shape != shape' is true. The important operands are array: the value named 'array' in this block; shape: the value named 'shape' in this block. Context: the function assert_shape, which handles assert shape.

Line 114: `raise AssertionError(f"stream shape {array.shape} != {shape}")` This rejects an invalid state by raising 'AssertionError(f"stream shape {array.shape} != {shape}")'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 117: `def direct_conv2d(` This starts function direct_conv2d. Its indented body performs direct conv2d as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 118: `activations: np.ndarray,` Inside the function direct_conv2d, which handles direct conv2d, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; ndarray: the value named 'ndarray' in this block.

Line 119: `weights: np.ndarray,` Inside the function direct_conv2d, which handles direct conv2d, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are weights: a learned coefficient multiplied by an activation; ndarray: the value named 'ndarray' in this block.

Line 120: `bias: np.ndarray,` Inside the function direct_conv2d, which handles direct conv2d, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are bias: an additive learned offset applied after accumulation; ndarray: the value named 'ndarray' in this block.

Line 121: `*,` Inside the function direct_conv2d, which handles direct conv2d, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 122: `activation_zero_point: int = 0,` This assigns ‘0,‘ to ‘activation_zero_point‘ in host memory. Here activation_zero_point means an input feature value consumed by a neuron or multiply-accumulate operation. The expression is a literal or structural operator and has no hidden software state. Context: the function direct_conv2d, which handles direct conv2d.

Line 123: `weight_zero_point: int = 0,` This assigns ‘0,‘ to ‘weight_zero_point‘ in host memory. Here weight_zero_point means a learned coefficient multiplied by an activation. The expression is a literal or structural operator and has no hidden software state. Context: the function direct_conv2d, which handles direct conv2d.

Line 124: `stride: tuple[int, int] = (1, 1),` This assigns ‘(1, 1),‘ to ‘stride‘ in host memory. Here stride means the value named ’stride’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function direct_conv2d, which handles direct conv2d.

Line 125: `padding: tuple[int, int] = (0, 0),` This assigns ‘(0, 0),‘ to ‘padding‘ in host memory. Here padding means the value named ’padding’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function direct_conv2d, which handles direct conv2d.

Line 126: `dilation: tuple[int, int] = (1, 1),` This assigns ‘(1, 1),‘ to ‘dilation‘ in host memory. Here dilation means the value named ’dilation’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function direct_conv2d, which handles direct conv2d.

Line 127: `groups: int = 1,` This assigns ‘1,‘ to ‘groups‘ in host memory. Here groups means the value named ’groups’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function direct_conv2d, which handles direct conv2d.

Line 128: `) -> np.ndarray:` Inside the function direct_conv2d, which handles direct conv2d, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are ndarray: the value named ’ndarray’ in this block.

Line 129: `batch, input_channels, height, width = activations.shape` This assigns ‘activations.shape‘ to ‘batch, input_channels, height, width ‘ in host memory. Here batch means the value named ’batch’ in this block. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; shape: the value named ’shape’ in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 130: `output_channels, channels_per_group, kernel_height, kernel_width = weights.shape` This assigns ‘weights.shape‘ to ‘output_channels, channels_per_group, kernel_height, kernel_width ‘ in host memory. Here output_channels means the value named ’output channels’ in this block. The important operands are weights: a learned coefficient multiplied by an activation; shape: the value named ’shape’ in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 131: `outputs_per_group = output_channels // groups` This assigns ‘output_channels // groups‘ to ‘outputs_per_group ‘ in host memory. Here outputs_per_group means the value named ’outputs per group’ in this block. The important operands are output_channels: the value named ’output channels’ in this block; groups: the value named ’groups’ in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 132: `output_height = (height + 2 * padding[0] - dilation[0] * (kernel_height - 1) - 1) // stride[0] + 1` This assigns ‘(height + 2 * padding[0] - dilation[0] * (kernel_height - 1) - 1) // stride[0] + 1‘ to ‘output_height ‘ in host memory. Here output_height means the value named ’output height’ in this block. The important operands are height: the value named ’height’ in this block; padding: the value named ’padding’ in this block; dilation: the value named ’dilation’ in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 133: `output_width = (width + 2 * padding[1] - dilation[1] * (kernel_width - 1) - 1) // stride[1] + 1` This assigns '(width + 2 * padding[1] - dilation[1] * (kernel_width - 1) - 1) // stride[1] + 1' to 'output_width ' in host memory. Here output_width means the value named 'output width' in this block. The important operands are width: the value named 'width' in this block; padding: the value named 'padding' in this block; dilation: the value named 'dilation' in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 134: `output = np.zeros((batch, output_channels, output_height, output_width), dtype=np.int64)` This assigns 'np.zeros((batch, output_channels, output_height, output_width), dtype=np.int64)' to 'output ' in host memory. Here output means the value named 'output' in this block. The important operands are zeros: the value named 'zeros' in this block; batch: the value named 'batch' in this block; output_channels: the value named 'output channels' in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 135: `for n in range(batch):` This iterates n over 'range(batch)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function direct_conv2d, which handles direct conv2d.

Line 136: `for output_channel in range(output_channels):` This iterates output_channel over 'range(output_channels)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function direct_conv2d, which handles direct conv2d.

Line 137: `group = output_channel // outputs_per_group` This assigns 'output_channel // outputs_per_group' to 'group ' in host memory. Here group means the value named 'group' in this block. The important operands are output_channel: the value named 'output channel' in this block; outputs_per_group: the value named 'outputs per group' in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 138: `input_start = group * channels_per_group` This assigns 'group * channels_per_group' to 'input_start ' in host memory. Here input_start means the value named 'input start' in this block. The important operands are group: the value named 'group' in this block; channels_per_group: the value named 'channels per group' in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 139: `for output_y in range(output_height):` This iterates output_y over 'range(output_height)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function direct_conv2d, which handles direct conv2d.

Line 140: `for output_x in range(output_width):` This iterates output_x over 'range(output_width)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function direct_conv2d, which handles direct conv2d.

Line 141: `accumulator = int(bias[output_channel])` This assigns 'int(bias[output_channel])' to 'accumulator ' in host memory. Here accumulator means a widened running sum that prevents intermediate overflow. The important operands are bias: an additive learned offset applied after accumulation; output_channel: the value named 'output channel' in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 142: `for local_channel in range(channels_per_group):` This iterates local_channel over 'range(channels_per_group)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function direct_conv2d, which handles direct conv2d.

Line 143: `input_channel = input_start + local_channel` This assigns 'input_start + local_channel' to 'input_channel ' in host memory. Here input_channel means the value named 'input channel' in this block. The important operands are input_start: the value named 'input start' in this block; local_channel: the value named 'local channel' in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 144: `for kernel_y in range(kernel_height):` This iterates kernel_y over 'range(kernel_height)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function direct_conv2d, which handles direct conv2d.

Line 145: `input_y = output_y * stride[0] + kernel_y * dilation[0] - padding[0]` This assigns ‘output_y * stride[0] + kernel_y * dilation[0] - padding[0]‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; stride: the value named ’stride’ in this block; kernel_y: the value named ’kernel y’ in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 146: `for kernel_x in range(kernel_width):` This iterates kernel_x over ‘range(kernel_width)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function direct_conv2d, which handles direct conv2d.

Line 147: `input_x = output_x * stride[1] + kernel_x * dilation[1] - padding[1]` This assigns ‘output_x * stride[1] + kernel_x * dilation[1] - padding[1]‘ to ‘input_x ‘ in host memory. Here input_x means the value named ’input x’ in this block. The important operands are output_x: the value named ’output x’ in this block; stride: the value named ’stride’ in this block; kernel_x: the value named ’kernel x’ in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 148: `activation = activation_zero_point` This assigns ‘activation_zero_point‘ to ‘activation ‘ in host memory. Here activation means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function direct_conv2d, which handles direct conv2d.

Line 149: `if 0 <= input_y < height and 0 <= input_x < width:` This executes the following indented host-code branch only when ‘0 <= input_y < height and 0 <= input_x < width‘ is true. The important operands are input_y: the value named ’input y’ in this block; height: the value named ’height’ in this block; and: the value named ’and’ in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 150: `activation = int(activations[n, input_channel, input_y, input_x])` This assigns ‘int(activations[n, input_channel, input_y, input_x])‘ to ‘activation ‘ in host memory. Here activation means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; n: the value named ’n’ in this block; input_channel: the value named ’input channel’ in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 151: `weight = int(weights[output_channel, local_channel, kernel_y, kernel_x])` This assigns ‘int(weights[outpu
local_channel, kernel_y, kernel_x])‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are weights: a learned coefficient multiplied by an activation; output_channel: the value named ’output channel’ in this block; local_channel: the value named ’local channel’ in this block. Context: the function direct_conv2d, which handles direct conv2d.

Line 152: `accumulator += ((activation - activation_zero_point)` This statement continues through source line 153; the explanation uses the complete expression rather than only the first visual fragment. Inside the function direct_conv2d, which handles direct conv2d, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are accumulator: a widened running sum that prevents intermediate overflow; activation: an input feature value consumed by a neuron or multiply-accumulate operation; activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 153: `* (weight - weight_zero_point))` This is continuation line 2 of the 2-line logical statement that begins at source line 152. Its exact fragment is ‘* (weight - weight_zero_point))‘. The important operands are weight: a learned coefficient multiplied by an activation; weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: Inside the function direct_conv2d, which handles direct conv2d, this statement advances the host implementation that checks neuraedge compiler behavior independently with Python assertions. The important operands are accumulator: a widened running sum that prevents intermediate overflow; activation: an input feature value consumed by a neuron or multiply-accumulate operation; activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 154: `output[n, output_channel, output_y, output_x] = accumulator` This assigns ‘accumulator‘ to ‘output[n, output_channel, output_y, output_x] ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are accumulator: a widened running sum that prevents intermediate overflow. Context: the function direct_conv2d, which handles direct conv2d.

Line 155: `return output` This returns ‘output‘ to the caller of the function direct_conv2d, which handles direct conv2d, ending this invocation. The important operands are output: the value named ’output’ in this block.

Line 158: `class CompilerTests(unittest.TestCase):` This defines class CompilerTests, which groups data and methods for compiler tests. Instances exist in host memory and organize the flow that checks neuraedge compiler behavior independently with Python assertions.

Line 159: `def setUp(self) -> None:` This starts function setUp. Its indented body performs set up as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 160: `self.rng = np.random.default_rng(20260713)` This assigns ‘np.random.default_rng(20260713)‘ to ‘self.rng ‘ in host memory. Here rng means the value named ’rng’ in this block. The important operands are random: the value named ’random’ in this block; default_rng: the value named ’default rng’ in this block. Context: the function setUp, which handles set up.

Line 162: `def check_case(self, rows: int, reduction: int, columns: int, dataflow: Dataflow | None = None) -> None:` This starts function check_case. Its indented body performs check case as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 163: `a = self.rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)` This assigns ‘self.rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)‘ to ‘a ‘ in host memory. Here a means the value named ’a’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function check_case, which handles check case.

Line 164: `b = self.rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)` This assigns ‘self.rng.integers(0 256, size=(reduction, columns), dtype=np.uint8)‘ to ‘b ‘ in host memory. Here b means the value named ’b’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function check_case, which handles check case.

Line 165: `bias = self.rng.integers(-100000, 100001, size=columns, dtype=np.int64)` This assigns ‘self.rng.integers(-100000, 100001, size=columns, dtype=np.int64)‘ to ‘bias ‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function check_case, which handles check case.

Line 166: `za = int(self.rng.integers(0, 256))` This assigns ‘int(self.rng.integers(0, 256))‘ to ‘za ‘ in host memory. Here za means the value named ’za’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function check_case, which handles check case.

Line 167: `zb = int(self.rng.integers(0, 256))` This assigns ‘int(self.rng.integers(0, 256))‘ to ‘zb ‘ in host memory. Here zb means the value named ’zb’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function check_case, which handles check case.

Line 168: `program = compile_u8_gemm(` This statement continues through source line 171; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_u8_gemm(a, b, activation_zero_point=za, weight_zero_point=zb, bias=bias, dataflow=dataflow,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; a: the value named ’a’ in this block; b: the value named ’b’ in this block. Context: the function check_case, which handles check case.

Line 169: `a, b, activation_zero_point=za, weight_zero_point=zb, bias=bias,` This is continuation line 2 of the 4-line logical statement that begins at source line 168. Its exact fragment is 'a, b, activation_zero_point=za, weight_zero_point=zb, bias=bias,'. The important operands are a: the value named 'a' in this block; b: the value named 'b' in this block; activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns 'compile_u8_gemm(a, b, activation_zero_point=za, weight_zero_point=zb, bias=bias, dataflow=dataflow,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; a: the value named 'a' in this block; b: the value named 'b' in this block. Context: the function check_case, which handles check case.

Line 170: `dataflow=dataflow,` This is continuation line 3 of the 4-line logical statement that begins at source line 168. Its exact fragment is 'dataflow=dataflow,'. The important operands are dataflow: the value named 'dataflow' in this block. Read together, the complete operation is: This assigns 'compile_u8_gemm(a, b, activation_zero_point=za, weight_zero_point=zb, bias=bias, dataflow=dataflow,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; a: the value named 'a' in this block; b: the value named 'b' in this block. Context: the function check_case, which handles check case.

Line 171: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 168. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'compile_u8_gemm(a, b, activation_zero_point=za, weight_zero_point=zb, bias=bias, dataflow=dataflow,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_u8_gemm: the value named 'compile u8 gemm' in this block; a: the value named 'a' in this block; b: the value named 'b' in this block. Context: the function check_case, which handles check case.

Line 172: `actual = program.execute(TensorTransactionEmulator())` This assigns 'program.execute(TensorTransactionEmu to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are program: the value named 'program' in this block; execute: the value named 'execute' in this block; TensorTransactionEmulator: the value named 'tensor transaction emulator' in this block. Context: the function check_case, which handles check case.

Line 173: `batched = program.execute_batched(TensorTransactionEmulator())` This assigns 'program.execute_batched(Te to 'batched ' in host memory. Here batched means the value named 'batched' in this block. The important operands are program: the value named 'program' in this block; execute_batched: the value named 'execute batched' in this block; TensorTransactionEmulator: the value named 'tensor transaction emulator' in this block. Context: the function check_case, which handles check case.

Line 174: `expected = direct_centered_gemm(a, b, za, zb, bias)` This assigns 'direct_centered_gemm(a, b, za, zb, bias)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are direct_centered_gemm: the value named 'direct centered gemm' in this block; a: the value named 'a' in this block; b: the value named 'b' in this block. Context: the function check_case, which handles check case.

Line 175: `np.testing.assert_array_equal(actual, expected)` This calls 'np.testing.assert_array_equal' with 'actual, expected' inside the function check_case, which handles check case. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 176: `np.testing.assert_array_equal(batched, expected)` This calls 'np.testing.assert_array_equal' with 'batched, expected' inside the function check_case, which handles check case. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 177: `self.assertEqual(program.mathematical_macs, rows * reduction * columns)` This calls 'self.assertEqual' with 'program.mathematical_macs, rows * reduction * columns' inside the function check_case, which handles check case. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 178: `self.assertGreater(program.estimated_input_bytes, 0)` This calls ‘self.assertGreater‘ with ‘program.estimated_ 0‘ inside the function check_case, which handles check case. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 180: `def test_outer_mode_irregular_tiles_both_dataflows(self) -> None:` This starts function test_outer_mode_ Its indented body performs test outer mode irregular tiles both dataflows as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 181: `self.check_case(13, 37, 29, Dataflow.INPUT_BROADCAST)` This calls ‘self.check_case‘ with ‘13, 37, 29, Dataflow.INPUT_BROADCAST‘ inside the function test_outer_mode_irregular_tiles_both_dataflows, which handles test outer mode irregular tiles both dataflows. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 182: `self.check_case(17, 33, 19, Dataflow.WEIGHT_BROADCAST)` This calls ‘self.check_case‘ with ‘17, 33, 19, Dataflow.WEIGHT_BROADCAST‘ inside the function test_outer_mode_irregular_tiles_both_dataflows, which handles test outer mode irregular tiles both dataflows. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 184: `def test_dot8_single_row_with_padding(self) -> None:` This starts function test_dot8_single_row_with_pad Its indented body performs test dot8 single row with padding as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 185: `self.check_case(1, 19, 35)` This calls ‘self.check_case‘ with ‘1, 19, 35‘ inside the function test_dot8_single_row_w which handles test dot8 single row with padding. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 187: `def test_reduction_larger_than_local_memory(self) -> None:` This starts function test_reduction_larger_tha Its indented body performs test reduction larger than local memory as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 188: `self.check_case(9, 2053, 17)` This calls ‘self.check_case‘ with ‘9, 2053, 17‘ inside the function test_reduction_large which handles test reduction larger than local memory. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 190: `def test_zero_skipping_and_boundary_values(self) -> None:` This starts function test_zero_skipping_and_bo Its indented body performs test zero skipping and boundary values as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 191: `a = np.full((8, 9), 128, dtype=np.uint8)` This assigns ‘np.full((8, 9), 128, dtype=np.uint8)‘ to ‘a ‘ in host memory. Here a means the value named ’a’ in this block. The important operands are full: the value named ’full’ in this block; dtype: the value named ’dtype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function test_zero_skipping_and_boundary_values, which handles test zero skipping and boundary values.

Line 192: `b = np.full((9, 16), 129, dtype=np.uint8)` This assigns ‘np.full((9, 16), 129, dtype=np.uint8)‘ to ‘b ‘ in host memory. Here b means the value named ’b’ in this block. The important operands are full: the value named ’full’ in this block; dtype: the value named ’dtype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function test_zero_skipping_and_boundary_values, which handles test zero skipping and boundary values.

Line 193: `a[3, 4] = 255` This assigns ‘255‘ to ‘a[3, 4] ‘ in host memory. Here a means the value named ’a’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_zero_skipping_and_boundary_values, which handles test zero skipping and boundary values.

Line 194: `b[4, 7] = 0` This assigns ‘0‘ to ‘b[4, 7] ‘ in host memory. Here b means the value named ’b’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_zero_skipping_and_boundary_values, which handles test zero skipping and boundary values.

Line 195: `program = compile_u8_gemm(a, b, activation_zero_point=128, weight_zero_point=129)` This assigns ‘compile_u8_gemm(a, b, activation_zero_point=128, weight_zero_point=129)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_u8_gemm: the value named ’compile u8 gemm’ in this block; a: the value named ’a’ in this block; b: the value named ’b’ in this block. Context: the function test_zero_skipping_and_boundary_values, which handles test zero skipping and boundary values.

Line 196: `np.testing.assert_array_equal(` This statement continues through source line 199; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.testing.assert_array_equal‘ with ‘ program.execute(TensorTransactionEmulator()), direct_centered_gemm(a, b, 128, 129), ‘ inside the function test_zero_skipping_and_boundary_values, which handles test zero skipping and boundary values. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 197: `program.execute(TensorTransactionEmulator()),` This is continuation line 2 of the 4-line logical statement that begins at source line 196. Its exact fragment is ‘program.execute(TensorTransactionEmulator()),‘. The important operands are program: the value named ’program’ in this block; execute: the value named ’execute’ in this block; TensorTransactionEmulator: the value named ’tensor transaction emulator’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ program.execute(TensorTransactionEmulator()), direct_centered_gemm(a, b, 128, 129), ‘ inside the function test_zero_skipping_and_boundary_values, which handles test zero skipping and boundary values. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 198: `direct_centered_gemm(a, b, 128, 129),` This is continuation line 3 of the 4-line logical statement that begins at source line 196. Its exact fragment is ‘direct_centered_gemm(a, b, 128, 129),‘. The important operands are direct_centered_gemm: the value named ’direct centered gemm’ in this block; a: the value named ’a’ in this block; b: the value named ’b’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ program.execute(TensorTransactionEmulator()), direct_centered_gemm(a, b, 128, 129), ‘ inside the function test_zero_skipping_and_boundary_values, which handles test zero skipping and boundary values. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 199: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 196. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ program.execute(TensorTransactionEmulator()), direct_centered_gemm(a, b, 128, 129), ‘ inside the function test_zero_skipping_and_boundary_values, which handles test zero skipping and boundary values. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 201: `def test_binary_and_ternary_sparse_paths(self) -> None:` This starts function test_binary_and_ternary_spa
Its indented body performs test binary and ternary sparse paths as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 202: `weights = self.rng.integers(-128, 128, size=(41, 23), dtype=np.int16)` This assigns ‘self.rng.integers(-128, 128, size=(41, 23), dtype=np.int16)‘ to ‘weights ‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths.

Line 203: `binary = self.rng.integers(0, 2, size=(11, 41), dtype=np.int8)` This assigns ‘self.rng.integers(0, 2, size=(11, 41), dtype=np.int8)‘ to ‘binary ‘ in host memory. Here binary means the value named ’binary’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths.

Line 204: `ternary = self.rng.integers(-1, 2, size=(9, 41), dtype=np.int8)` This assigns ‘self.rng.integers(-1, 2, size=(9, 41), dtype=np.int8)‘ to ‘ternary ‘ in host memory. Here ternary means the value named ’ternary’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths.

Line 205: `binary_program = compile_spike_gemm(binary, weights)` This assigns ‘compile_spike_gemm(binary, weights)‘ to ‘binary_program ‘ in host memory. Here binary_program means the value named ’binary program’ in this block. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold; binary: the value named ’binary’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths.

Line 206: `ternary_program = compile_spike_gemm(` This statement continues through source line 208; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_spike_gemm(ternary, weights, ternary=True, dataflow=Dataflow.WEIGHT_BROADCAST,)‘ to ‘ternary_program ‘ in host memory. Here ternary_program means the value named ’ternary program’ in this block. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold; ternary: the value named ’ternary’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths.

Line 207: `ternary, weights, ternary=True, dataflow=Dataflow.WEIGHT_BROADCAST,` This is continuation line 2 of the 3-line logical statement that begins at source line 206. Its exact fragment is ‘ternary, weights, ternary=True, dataflow=Dataflow.WEIGHT_BROADCAST,‘. The important operands are ternary: the value named ’ternary’ in this block; weights: a learned coefficient multiplied by an activation; dataflow: the value named ’dataflow’ in this block. Read together, the complete operation is: This assigns ‘compile_spike_gemm(ternary, weights, ternary=True, dataflow=Dataflow.WEIGHT_BROADCAST,)‘ to ‘ternary_program ‘ in host memory. Here ternary_program means the value named ’ternary program’ in this block. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold; ternary: the value named ’ternary’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths.

Line 208: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 206. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘compile_spike_gemm(ternary, weights, ternary=True, dataflow=Dataflow.WEIGHT_BROADCAST,)‘ to ‘ternary_program ‘ in host memory. Here ternary_program means the value named ’ternary program’ in this block. The important operands are compile_spike_gemm: a binary event produced when a neuron crosses its threshold; ternary: the value named ’ternary’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths.

Line 209: `np.testing.assert_array_equal(` This statement continues through source line 212; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.testing.assert_array_equal‘ with ‘ binary_program.execute(TensorTransactionEmulator()), binary.astype(np.int64) @ weights.astype(np.int64), ‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 210: `binary_program.execute(TensorTransactionEmulator()),` This is continuation line 2 of the 4-line logical statement that begins at source line 209. Its exact fragment is ‘binary_program.execute(TensorTransactionEmulator()),‘. The important operands are binary_program: the value named ’binary program’ in this block; execute: the value named ’execute’ in this block; TensorTransactionEmulator: the value named ’tensor transaction emulator’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ binary_program.execute(TensorTransactionEmul binary.astype(np.int64) @ weights.astype(np.int64), ‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 211: `binary.astype(np.int64) @ weights.astype(np.int64),` This is continuation line 3 of the 4-line logical statement that begins at source line 209. Its exact fragment is ‘binary.astype(np.int64) @ weights.astype(np.int64),‘.

The important operands are binary: the value named 'binary' in this block; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ binary_program.execute(TensorTransactionEmulator()), binary.astype(np.int64) @ weights.astype(np.int64), ‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 212: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 209. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ binary_program.execute(TensorTransactionEmulator()), binary.astype(np.int64) @ weights.astype(np.int64), ‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 213: `np.testing.assert_array_equal(` This statement continues through source line 216; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.testing.assert_array_equal‘ with ‘ ternary_program.execute(TensorTransactionEmulator()), ternary.astype(np.int64) @ weights.astype(np.int64), ‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 214: `ternary_program.execute(TensorTransactionEmulator()),` This is continuation line 2 of the 4-line logical statement that begins at source line 213. Its exact fragment is ‘ternary_program.execute(TensorTransactionEmulator()),‘. The important operands are ternary_program: the value named 'ternary program' in this block; execute: the value named 'execute' in this block; TensorTransactionEmulator: the value named 'tensor transaction emulator' in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ ternary_program.execute(TensorTransactionEmu ternary.astype(np.int64) @ weights.astype(np.int64), ‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 215: `ternary.astype(np.int64) @ weights.astype(np.int64),` This is continuation line 3 of the 4-line logical statement that begins at source line 213. Its exact fragment is ‘ternary.astype(np.int64) @ weights.astype(np.int64),‘. The important operands are ternary: the value named 'ternary' in this block; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ ternary_program.execute(TensorTransactionEmulator()), ternary.astype(np.int64) @ weights.astype(np.int64), ‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 216: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 213. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ ternary_program.execute(TensorTransactionEmulator()), ternary.astype(np.int64) @ weights.astype(np.int64), ‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 217: `self.assertEqual(binary_program.estimated_spike_adds, int(np.count_nonzero(binary)) * 23)` This calls ‘self.assertEqual‘ with ‘binary_program.estimated_spike_adds, int(np.count_nonzero(binary)) * 23‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 218: `self.assertEqual(ternary_program.estimated_spike_adds, int(np.count_nonzero(ternary)) * 23)` This calls ‘self.assertEqual‘ with ‘ternary_program.estimated_spike_adds, int(np.count_nonzero(ternary)) * 23‘ inside the function test_binary_and_ternary_sparse_paths, which handles test binary and ternary sparse paths. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 220: `def test_grouped_u8_convolution_with_stride_padding_and_dilation(self) -> None:` This starts function test_grouped_u8_convolution_with_stride_padding_and_dilation. Its indented body performs test grouped u8 convolution with stride padding and dilation as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 221: `activations = self.rng.integers(0, 256, size=(1, 4, 5, 6), dtype=np.uint8)` This assigns 'self.rng.integers 256, size=(1, 4, 5, 6), dtype=np.uint8)' to 'activations ' in host memory. Here activations means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 222: `weights = self.rng.integers(0, 256, size=(6, 2, 2, 3), dtype=np.uint8)` This assigns 'self.rng.integers(0, 256, size=(6, 2, 2, 3), dtype=np.uint8)' to 'weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_grouped_u8_convolution_with_stride_padding_ which handles test grouped u8 convolution with stride padding and dilation.

Line 223: `bias = self.rng.integers(-5000, 5001, size=6, dtype=np.int64)` This assigns 'self.rng.integers(-5000, 5001, size=6, dtype=np.int64)' to 'bias ' in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_grouped_u8_convolution_with_stride_padding_a which handles test grouped u8 convolution with stride padding and dilation.

Line 224: `program = compile_u8_conv2d(` This statement continues through source line 234; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 225: `activations,` This is continuation line 2 of the 11-line logical statement that begins at source line 224. Its exact fragment is 'activations,'. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns 'compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 226: `weights,` This is continuation line 3 of the 11-line logical statement that begins at source line 224. Its exact fragment is 'weights,'. The important operands are weights: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 227: `activation_zero_point=117,` This is continuation line 4 of the 11-line logical statement that begins at source line 224. Its exact fragment is 'activation_zero_point=117,'. The important operands are activation_zero_point:

an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_ which handles test grouped u8 convolution with stride padding and dilation.

Line 228: `weight_zero_point=133,` This is continuation line 5 of the 11-line logical statement that begins at source line 224. Its exact fragment is ‘weight_zero_point=133,‘. The important operands are weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 229: `bias=bias,` This is continuation line 6 of the 11-line logical statement that begins at source line 224. Its exact fragment is ‘bias=bias,‘. The important operands are bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 230: `stride=(2, 1),` This is continuation line 7 of the 11-line logical statement that begins at source line 224. Its exact fragment is ‘stride=(2, 1),‘. The important operands are stride: the value named ’stride’ in this block. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 231: `padding=(1, 2),` This is continuation line 8 of the 11-line logical statement that begins at source line 224. Its exact fragment is ‘padding=(1, 2),‘. The important operands are padding: the value named ’padding’ in this block. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 232: `dilation=(2, 1),` This is continuation line 9 of the 11-line logical statement that begins at source line 224. Its exact fragment is ‘dilation=(2, 1),‘. The important operands are dilation: the value named ’dilation’ in this block. Read together, the complete operation is: This assigns ‘compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_u8_conv2d: the value named ’compile u8 conv2d’ in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 233: `groups=2,` This is continuation line 10 of the 11-line logical statement that begins at source line 224. Its exact fragment is 'groups=2,'. The important operands are groups: the value named 'groups' in this block. Read together, the complete operation is: This assigns 'compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 234: `)` This is continuation line 11 of the 11-line logical statement that begins at source line 224. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'compile_u8_conv2d(activations, weights, activation_zero_point=117, weight_zero_point=133, bias=bias, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_u8_conv2d: the value named 'compile u8 conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 235: `actual = program.execute(TensorTransactionEmulator())` This assigns 'program.execute(TensorTransactionEmu to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are program: the value named 'program' in this block; execute: the value named 'execute' in this block; TensorTransactionEmulator: the value named 'tensor transaction emulator' in this block. Context: the function test_grouped_u8_convolution_with_stride_ which handles test grouped u8 convolution with stride padding and dilation.

Line 236: `expected = direct_conv2d(` This statement continues through source line 240; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'direct_conv2d(activations, weights, bias, activation_zero_point=117, weight_zero_point=133, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are direct_conv2d: the value named 'direct conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 237: `activations, weights, bias,` This is continuation line 2 of the 5-line logical statement that begins at source line 236. Its exact fragment is 'activations, weights, bias,'. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation; bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'direct_conv2d(activations, weights, bias, activation_zero_point=117, weight_zero_point=133, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are direct_conv2d: the value named 'direct conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 238: `activation_zero_point=117, weight_zero_point=133,` This is continuation line 3 of the 5-line logical statement that begins at source line 236. Its exact fragment is 'activation_zero_point=117, weight_zero_point=133,'. The important operands are activation_zero_point: an input feature value consumed by a neuron or multiply-accumulate operation; weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'direct_conv2d(activations, weights, bias, activation_zero_point=117, weight_zero_point=133, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are direct_conv2d: the value named 'direct conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 239: `stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,` This is continuation line 4 of the 5-line logical statement that begins at source line 236. Its exact fragment is 'stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,'. The important operands are stride: the value named 'stride' in this block; padding: the value named 'padding' in this block; dilation: the value named 'dilation' in this block. Read together, the complete operation is: This assigns 'direct_conv2d(activations, weights, bias, activation_zero_point=117, weight_zero_point=133, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are direct_conv2d: the value named 'direct conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 240: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 236. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'direct_conv2d(activations, weights, bias, activation_zero_point=117, weight_zero_point=133, stride=(2, 1), padding=(1, 2), dilation=(2, 1), groups=2,)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are direct_conv2d: the value named 'direct conv2d' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation; weights: a learned coefficient multiplied by an activation. Context: the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation.

Line 241: `np.testing.assert_array_equal(actual, expected)` This calls 'np.testing.assert_array_equal' with 'actual, expected' inside the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 242: `self.assertEqual(actual.shape, (1, 6, 3, 8))` This calls 'self.assertEqual' with 'actual.shape, (1, 6, 3, 8)' inside the function test_grouped_u8_convolution_with_stride_padding_and_dilation, which handles test grouped u8 convolution with stride padding and dilation. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 244: `def test_depthwise_binary_and_ternary_spike_convolution(self) -> None:` This starts function test_depthwise_binary_and_ternary_spike_convolution. Its indented body performs test depthwise binary and ternary spike convolution as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 245: `binary = self.rng.integers(0, 2, size=(1, 3, 4, 4), dtype=np.int8)` This assigns 'self.rng.integers(0, 2, size=(1, 3, 4, 4), dtype=np.int8)' to 'binary ' in host memory. Here binary means the value named 'binary' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 246: `ternary = self.rng.integers(-1, 2, size=(1, 3, 4, 4), dtype=np.int8)` This assigns 'self.rng.integers(-1, 2, size=(1, 3, 4, 4), dtype=np.int8)' to 'ternary ' in host memory. Here ternary means the value named 'ternary' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 247: `weights = self.rng.integers(-128, 128, size=(3, 1, 3, 3), dtype=np.int16)` This assigns 'self.rng.integers(128, 128, size=(3, 1, 3, 3), dtype=np.int16)' to 'weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 248: `bias = self.rng.integers(-50, 51, size=3, dtype=np.int64)` This assigns ‘self.rng.integers(-50, 51, size=3, dtype=np.int64)‘ to ‘bias ‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 249: `for spikes, is_ternary in ((binary, False), (ternary, True)):` This iterates spikes, is_ternary over ‘((binary, False), (ternary, True))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 250: `program = compile_spike_conv2d(` This statement continues through source line 253; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_spike_conv2d(spikes, weights, ternary=is_ternary, bias=bias, padding=1, groups=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_spike_conv2d: a binary event produced when a neuron crosses its threshold; spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 251: `spikes, weights, ternary=is_ternary, bias=bias,` This is continuation line 2 of the 4-line logical statement that begins at source line 250. Its exact fragment is ‘spikes, weights, ternary=is_ternary, bias=bias,‘. The important operands are spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation; ternary: the value named ’ternary’ in this block. Read together, the complete operation is: This assigns ‘compile_spike_conv2d(spikes, weights, ternary=is_ternary, bias=bias, padding=1, groups=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_spike_conv2d: a binary event produced when a neuron crosses its threshold; spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 252: `padding=1, groups=3,` This is continuation line 3 of the 4-line logical statement that begins at source line 250. Its exact fragment is ‘padding=1, groups=3,‘. The important operands are padding: the value named ’padding’ in this block; groups: the value named ’groups’ in this block. Read together, the complete operation is: This assigns ‘compile_spike_conv2d(spikes, weights, ternary=is_ternary, bias=bias, padding=1, groups=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_spike_conv2d: a binary event produced when a neuron crosses its threshold; spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 253: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 250. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘compile_spike_conv2d(spikes, weights, ternary=is_ternary, bias=bias, padding=1, groups=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_spike_conv2d: a binary event produced when a neuron crosses its threshold; spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 254: `actual = program.execute(TensorTransactionEmulator())` This assigns ‘program.execute(TensorTransactionEmu
to ‘actual ‘ in host memory. Here actual means the value named ’actual’ in this block. The important operands are program: the value named ’program’ in this block; execute: the value named ’execute’ in this block; TensorTransactionEmulator: the value named ’tensor transaction emulator’ in this block. Context: the function test_depthwise_binary_and_ternary_spike_
which handles test depthwise binary and ternary spike convolution.

Line 255: `expected = direct_conv2d(` This statement continues through source line 257; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘direct_conv2d(spikes, weights, bias, padding=(1, 1), groups=3,)‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are direct_conv2d: the value named ’direct conv2d’ in this block; spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 256: `spikes, weights, bias, padding=(1, 1), groups=3,` This is continuation line 2 of the 3-line logical statement that begins at source line 255. Its exact fragment is ‘spikes, weights, bias, padding=(1, 1), groups=3,‘. The important operands are spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation; bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘direct_conv2d(spikes, weights, bias, padding=(1, 1), groups=3,)‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are direct_conv2d: the value named ’direct conv2d’ in this block; spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 257: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 255. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘direct_conv2d(spikes, weights, bias, padding=(1, 1), groups=3,)‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are direct_conv2d: the value named ’direct conv2d’ in this block; spikes: a binary event produced when a neuron crosses its threshold; weights: a learned coefficient multiplied by an activation. Context: the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution.

Line 258: `np.testing.assert_array_equal(actual, expected)` This calls ‘np.testing.assert_array_equal‘ with ‘actual, expected‘ inside the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 259: `self.assertEqual(program.estimated_spike_adds,` This statement continues through source line 260; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertEqual‘ with ‘program.estimated_spike_adds, sum(group.gemm.estimated_spike_adds for group in program.group_programs)‘ inside the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 260: `sum(group.gemm.estimated_spike_adds for group in program.group_programs))` This is continuation line 2 of the 2-line logical statement that begins at source line 259. Its exact fragment is ‘sum(group.gemm.estimated_spike_adds for group in program.group_programs))‘. The important operands are sum: the value named ’sum’ in this block; group: the value named ’group’ in this block; gemm: the value named ’gemm’ in this block. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘program.estimated_spike_adds, sum(group.gemm.estimated_spike_adds for group in program.group_programs)‘ inside the function test_depthwise_binary_and_ternary_spike_convolution, which handles test depthwise binary and ternary spike convolution. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 262: `def test_validation(self) -> None:` This starts function test_validation. Its indented body performs test validation as part of the file that checks neuraedge compiler behavior independently with Python assertions; it runs only when called.

Line 263: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_validation, which handles test validation raises an error.

Line 264: `compile_u8_gemm([[1, 2]], [[1, 2]])` This calls ‘compile_u8_gemm‘ with ‘[[1, 2]], [[1, 2]]‘ inside the function test_validation, which handles test validation. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 265: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_validation, which handles test validation raises an error.

Line 266: `compile_u8_gemm([[256]], [[1]])` This calls ‘compile_u8_gemm‘ with ‘[[256]], [[1]]‘ inside the function test_validation, which handles test validation. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 267: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_validation, which handles test validation raises an error.

Line 268: `compile_u8_gemm([[1]], [[1]], bias=[1, 2])` This calls ‘compile_u8_gemm‘ with ‘[[1]], [[1]], bias=[1, 2]‘ inside the function test_validation, which handles test validation. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 269: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_validation, which handles test validation raises an error.

Line 270: `compile_spike_gemm([[0, 2]], [[1], [2]])` This calls ‘compile_spike_gemm‘ with ‘[[0, 2]], [[1], [2]]‘ inside the function test_validation, which handles test validation. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 271: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_validation, which handles test validation raises an error.

Line 272: `compile_spike_gemm([[0, -1]], [[1], [2]], ternary=False)` This calls ‘compile_spike_gemm‘ with ‘[[0, -1]], [[1], [2]], ternary=False‘ inside the function test_validation, which handles test validation. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 273: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_validation, which handles test validation raises an error.

Line 274: `compile_u8_conv2d(np.zeros((1, 3, 4, 4), dtype=np.uint8),` This statement continues through source line 275; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘compile_u8_conv2d‘ with ‘np.zeros((1, 3, 4, 4), dtype=np.uint8), np.zeros((4, 2, 3, 3), dtype=np.uint8), groups=2‘ inside the function test_validation, which handles test validation. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 275: `np.zeros((4, 2, 3, 3), dtype=np.uint8), groups=2)` This is continuation line 2 of the 2-line logical statement that begins at source line 274. Its exact fragment is ‘np.zeros((4, 2, 3, 3), dtype=np.uint8), groups=2)‘. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; uint8: the value named ’uint8’ in this block. Read together, the complete operation is: This calls ‘compile_u8_conv2d‘ with ‘np.zeros((1, 3, 4, 4), dtype=np.uint8), np.zeros((4, 2, 3, 3), dtype=np.uint8), groups=2‘ inside the function test_validation, which handles test validation. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

Line 278: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks neuraedge compiler behavior independently with Python assertions.

Line 279: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks neuraedge compiler behavior independently with Python assertions. The call advances the host-side job that checks neuraedge compiler behavior independently with Python assertions.

65 tests/test_neuraedge_golden.py

Role in the complete system: checks neuraedge golden behavior independently with Python assertions.

65.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks neuraedge golden behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: GoldenModelTests. Functions: setUp, test_signed_encoding_round_trip, test_packed_mac_bounda test_zero_point_correction_matches_direct_expression, test_sparse_index_compaction, test_binary_and_ternary_produ test_alpha_xnor_special_cases, test_log_integer_softmax_properties, test_integer_layer_norm_constant_and_nonconst test_lif_and_ternary_firing, test_dataflow_selection_and_peak_rate, test_signed_rounding_and_saturation, test_linear_backward_matches_independent_scalar_reference, test_sgd_momentum_update, test_sgd_weight_decay_is_ test_vector_alu_operations, rounded, clipped. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

65.2 Complete source

```
from __future__ import annotations

import unittest

import numpy as np

from neuraedge_golden import (
    Dataflow,
    VectorOperation,
    active_dot8_indices,
    active_outer_indices,
    alpha_xnor_similarity,
    binary_weighted_gemm,
    choose_dataflow,
    decode_uint8_as_int8,
    direct_centered_gemm,
    encode_int8_as_uint8,
    gemm_u8_zero_point,
    integer_layer_norm,
    linear_backward_tile,
    lif_step,
    log_int_softmax_q15,
    packed_mac2_u8,
    peak_dense_gops,
    quantize_flexible_scale,
    round_shift_signed,
    saturate_signed,
    sgd_momentum_step,
    ternary_spike,
    ternary_weighted_gemm,
    vector_alu,
)


class GoldenModelTests(unittest.TestCase):
    def setUp(self) -> None:
        self.rng = np.random.default_rng(20260711)

    def test_signed_encoding_round_trip(self) -> None:
        values = np.arange(-128, 128, dtype=np.int16)
        np.testing.assert_array_equal(decode_uint8_as_int8(encode_int8_as_uint8(values)), values)

    def test_packed_mac_boundaries(self) -> None:
        for activation in (0, 1, 127, 128, 254, 255):
            for low in (0, 1, 127, 128, 254, 255):
                for high in (0, 1, 127, 128, 254, 255):
                    got_low, got_high = packed_mac2_u8(activation, low, high)
```

```
                    self.assertEqual(got_low, activation * low)
                    self.assertEqual(got_high, activation * high)

    def test_zero_point_correction_matches_direct_expression(self) -> None:
        for _ in range(100):
            rows = int(self.rng.integers(1, 9))
            columns = int(self.rng.integers(1, 33))
            reduction = int(self.rng.integers(1, 129))
            a = self.rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)
            b = self.rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)
            za = int(self.rng.integers(0, 256))
            zb = int(self.rng.integers(0, 256))
            bias = self.rng.integers(-(1 << 20), 1 << 20, size=columns, dtype=np.int64)
            corrected = gemm_u8_zero_point(a, b, za, zb, bias)
            direct = direct_centered_gemm(a, b, za, zb, bias)
            np.testing.assert_array_equal(corrected, direct)

    def test_sparse_index_compaction(self) -> None:
        zero = 128
        a = np.full((3, 19), zero, dtype=np.uint8)
        a[0, 1] = 129
        a[2, 9] = 127
        a[1, 18] = 0
        np.testing.assert_array_equal(active_outer_indices(a, zero), np.array([1, 9, 18]))
        np.testing.assert_array_equal(active_dot8_indices(a[0], zero), np.array([0]))
        np.testing.assert_array_equal(active_dot8_indices(a[2], zero), np.array([1]))

    def test_binary_and_ternary_products(self) -> None:
        weights = self.rng.integers(-127, 128, size=(17, 11), dtype=np.int16)
        binary = self.rng.integers(0, 2, size=(5, 17), dtype=np.int8)
        ternary = self.rng.integers(-1, 2, size=(5, 17), dtype=np.int8)
        np.testing.assert_array_equal(binary_weighted_gemm(binary, weights), binary.astype(np.int64) @ weights)
        np.testing.assert_array_equal(ternary_weighted_gemm(ternary, weights), ternary.astype(np.int64) @ weights)

    def test_alpha_xnor_special_cases(self) -> None:
        q = np.array([[1, 0, 1, 0], [0, 0, 0, 0]], dtype=np.uint8)
        k = np.array([[1, 0, 0, 0], [0, 0, 0, 0]], dtype=np.uint8)
        dot_q8 = alpha_xnor_similarity(q, k, 0)
        xnor_q8 = alpha_xnor_similarity(q, k, 256)
        np.testing.assert_array_equal(dot_q8 // 256, q.astype(np.int64) @ k.astype(np.int64).T)
        expected_xnor = np.array([[3, 2], [3, 4]], dtype=np.int64)
        np.testing.assert_array_equal(xnor_q8 // 256, expected_xnor)

    def test_log_integer_softmax_properties(self) -> None:
        scores = np.array([91, 83, 50, -30], dtype=np.int64)
        probabilities = log_int_softmax_q15(scores, difference_shift=3)
        translated = log_int_softmax_q15(scores + 123456, difference_shift=3)
        np.testing.assert_array_equal(probabilities, translated)
        self.assertGreaterEqual(int(probabilities[0]), int(probabilities[1]))
        self.assertGreaterEqual(int(probabilities[1]), int(probabilities[2]))
        self.assertLessEqual(abs(int(probabilities.sum()) - (1 << 15)), 2)

    def test_integer_layer_norm_constant_and_nonconstant(self) -> None:
        gamma = np.full(4, 1 << 15, dtype=np.int64)
        beta = np.array([1, 2, 3, 4], dtype=np.int64)
        np.testing.assert_array_equal(integer_layer_norm(np.full(4, 12), gamma, beta), beta)
        output = integer_layer_norm(np.array([-9, -3, 3, 9]), gamma, np.zeros(4, dtype=np.int64))
        self.assertEqual(int(output.sum()), 0)
        self.assertTrue(np.all(np.diff(output) >= 0))

    def test_lif_and_ternary_firing(self) -> None:
        next_membrane, spikes = lif_step(np.array([0, 8, 20]), np.array([4, 8, 1]), 2, 12)
        np.testing.assert_array_equal(spikes, np.array([0, 1, 1], dtype=np.uint8))
        np.testing.assert_array_equal(next_membrane, np.array([4, 0, 0]))
        np.testing.assert_array_equal(
            ternary_spike(np.array([-8, -7, 0, 7, 8]), 8),
            np.array([-1, 0, 0, 0, 1], dtype=np.int8),
        )

    def test_dataflow_selection_and_peak_rate(self) -> None:
        wide = choose_dataflow(rows=8, columns=768, reduction=384)
        tall = choose_dataflow(rows=4096, columns=16, reduction=144)
        self.assertEqual(wide.mode, Dataflow.INPUT_BROADCAST)
        self.assertEqual(wide.input_broadcast_bytes, 297984)
        self.assertEqual(wide.weight_broadcast_bytes, 442368)
        self.assertEqual(tall.mode, Dataflow.WEIGHT_BROADCAST)
        self.assertEqual(tall.input_broadcast_bytes, 1769472)
        self.assertEqual(tall.weight_broadcast_bytes, 592128)
        self.assertAlmostEqual(peak_dense_gops(200e6, 64), 51.2)

    def test_signed_rounding_and_saturation(self) -> None:
        values = np.array([-9, -8, -7, -1, 0, 1, 7, 8, 9], dtype=np.int64)
        np.testing.assert_array_equal(
            round_shift_signed(values, 3),
            np.array([-1, -1, -1, 0, 0, 0, 1, 1, 1], dtype=np.int64),
        )
        np.testing.assert_array_equal(
```

```
            saturate_signed(np.array([-40000, -32768, 32767, 40000]), 16),
            np.array([-32768, -32768, 32767, 32767]),
        )

    def test_linear_backward_matches_independent_scalar_reference(self) -> None:
        def rounded(value: int, shift: int) -> int:
            if shift == 0:
                return value
            magnitude = abs(value)
            result = (magnitude + (1 << (shift - 1))) >> shift
            return -result if value < 0 else result

        def clipped(value: int, bits: int) -> int:
            return max(-(1 << (bits - 1)), min((1 << (bits - 1)) - 1, value))

        for _ in range(100):
            rows = int(self.rng.integers(1, 9))
            inputs = int(self.rng.integers(1, 9))
            outputs = int(self.rng.integers(1, 9))
            dx_shift = int(self.rng.integers(0, 13))
            dw_shift = int(self.rng.integers(0, 13))
            x = self.rng.integers(-128, 128, size=(rows, inputs), dtype=np.int64)
            w = self.rng.integers(-32768, 32768, size=(inputs, outputs), dtype=np.int64)
            dy = self.rng.integers(-32768, 32768, size=(rows, outputs), dtype=np.int64)
            expected_dx = np.zeros((rows, inputs), dtype=np.int64)
            expected_dw = np.zeros((inputs, outputs), dtype=np.int64)
            for row in range(rows):
                for input_index in range(inputs):
                    total = sum(
                        int(dy[row, output_index]) * int(w[input_index, output_index])
                        for output_index in range(outputs)
                    )
                    expected_dx[row, input_index] = clipped(rounded(total, dx_shift), 32)
            for input_index in range(inputs):
                for output_index in range(outputs):
                    total = sum(
                        int(x[row, input_index]) * int(dy[row, output_index])
                        for row in range(rows)
                    )
                    expected_dw[input_index, output_index] = clipped(rounded(total, dw_shift), 24)
            got_dx, got_dw = linear_backward_tile(
                x,
                w,
                dy,
                input_gradient_shift=dx_shift,
                weight_gradient_shift=dw_shift,
            )
            np.testing.assert_array_equal(got_dx, expected_dx)
            np.testing.assert_array_equal(got_dw, expected_dw)

        x = np.full((8, 8), 127, dtype=np.int64)
        w = np.full((8, 8), -32768, dtype=np.int64)
        dy = np.full((8, 8), -32768, dtype=np.int64)
        got_dx, got_dw = linear_backward_tile(x, w, dy)
        np.testing.assert_array_equal(got_dx, np.full((8, 8), (1 << 31) - 1))
        np.testing.assert_array_equal(got_dw, np.full((8, 8), -(1 << 23)))

    def test_sgd_momentum_update(self) -> None:
        for _ in range(100):
            weights = self.rng.integers(-32768, 32768, size=8, dtype=np.int64)
            gradients = self.rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)
            velocities = self.rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)
            next_weight, next_velocity = sgd_momentum_step(
                weights, gradients, velocities, 655, 58982, 66
            )
            self.assertTrue(np.all(next_weight >= -32768))
            self.assertTrue(np.all(next_weight <= 32767))
            self.assertTrue(np.all(next_velocity >= -(1 << 23)))
            self.assertTrue(np.all(next_velocity <= (1 << 23) - 1))

    def test_sgd_weight_decay_is_decoupled_from_velocity(self) -> None:
        next_weight, next_velocity = sgd_momentum_step(
            np.full(8, 1000, dtype=np.int64),
            np.zeros(8, dtype=np.int64),
            np.full(8, 1000, dtype=np.int64),
            65536,
            65536,
            65536,
        )
        np.testing.assert_array_equal(next_velocity, np.full(8, 1000))
        np.testing.assert_array_equal(next_weight, np.full(8, -1000))

    def test_vector_alu_operations(self) -> None:
        a = np.array([-(1 << 31), -100, -8, -1, 0, 7, 100, (1 << 31) - 1])
        b = np.array([-1, 300, 8, -2, 1, 9, -300, 1])
        added = vector_alu(VectorOperation.ADD_SAT, a, b)
        self.assertEqual(int(added[0]), -(1 << 31))
```

```
        self.assertEqual(int(added[-1]), (1 << 31) - 1)
        np.testing.assert_array_equal(
            vector_alu(VectorOperation.TERNARY_SPIKE, a, b, threshold=8),
            np.array([-1, -1, -1, 0, 0, 0, 1, 1]),
        )
        requantized = vector_alu(
            VectorOperation.REQUANTIZE,
            a,
            b,
            multiplier_q15=16384,
            right_shift=1,
            output_zero_point=3,
            clamp_minimum=-128,
            clamp_maximum=127,
        )
        self.assertTrue(np.all(requantized >= -128))
        self.assertTrue(np.all(requantized <= 127))

        multiplier, shift, represented = quantize_flexible_scale(51.03228)
        self.assertGreater(multiplier, 0)
        self.assertLess(shift, 0)
        self.assertAlmostEqual(represented, 51.03228, delta=0.002)
        flexible = vector_alu(
            VectorOperation.FLEX_REQUANTIZE,
            np.array([1, -1, 100, -100], dtype=np.int64),
            np.zeros(4, dtype=np.int64),
            multiplier_q15=multiplier,
            right_shift=shift,
        )
        np.testing.assert_allclose(
            flexible,
            np.array([51, -51, 5103, -5103], dtype=np.int64),
            atol=1,
        )
        selector = np.array([0, 1, -1, 0], dtype=np.int64)
        payload = np.array([10, 20, 30, 40], dtype=np.int64)
        np.testing.assert_array_equal(
            vector_alu(VectorOperation.SELECT_ZERO, selector, payload),
            np.array([10, 0, 0, 40]),
        )
        np.testing.assert_array_equal(
            vector_alu(VectorOperation.SELECT_NONZERO, selector, payload),
            np.array([0, 20, 30, 0]),
        )
        np.testing.assert_array_equal(
            vector_alu(
                VectorOperation.COMPARE_GE,
                np.array([1, 2, 2, -1]),
                np.array([2, 2, 1, -1]),
            ),
            np.array([0, 1, 1, 1]),
        )


if __name__ == "__main__":
    unittest.main()
```

65.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import unittest` This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import numpy as np` This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from neuraedge_golden import (` This statement continues through source line 32; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `Dataflow,` This is continuation line 2 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'Dataflow,'. The important operands are Dataflow: the value named 'dataflow' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `VectorOperation,` This is continuation line 3 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'VectorOperation,'. The important operands are VectorOperation: the value named 'vector operation' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `active_dot8_indices,` This is continuation line 4 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'active_dot8_indices,'. The important operands are active_dot8_indices: the value named 'active dot8 indices' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `active_outer_indices,` This is continuation line 5 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'active_outer_indices,'. The important operands are active_outer_indices: the value named 'active outer indices' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `alpha_xnor_similarity,` This is continuation line 6 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'alpha_xnor_similarity,'. The important operands are alpha_xnor_similarity: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `binary_weighted_gemm,` This is continuation line 7 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'binary_weighted_gemm,'. The important operands are binary_weighted_gemm: a learned coefficient multiplied by an activation. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `choose_dataflow,` This is continuation line 8 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'choose_dataflow,'. The important operands are choose_dataflow: the value named 'choose dataflow' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `decode_uint8_as_int8,` This is continuation line 9 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'decode_uint8_as_int8,'. The important operands are decode_uint8_as_int8: the value named 'decode uint8 as int8' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `direct_centered_gemm,` This is continuation line 10 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'direct_centered_gemm,'. The important operands are direct_centered_gemm: the value named 'direct centered gemm' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `encode_int8_as_uint8,` This is continuation line 11 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'encode_int8_as_uint8,'. The important operands are encode_int8_as_uint8: the value named 'encode int8 as uint8' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `gemm_u8_zero_point,` This is continuation line 12 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'gemm_u8_zero_point,'. The important operands are gemm_u8_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `integer_layer_norm,` This is continuation line 13 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'integer_layer_norm,'. The important operands are integer_layer_norm: the value named 'integer layer norm' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `linear_backward_tile,` This is continuation line 14 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'linear_backward_tile,'. The important operands are linear_backward_tile: the value named 'linear backward tile' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `lif_step,` This is continuation line 15 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'lif_step,'. The important operands are lif_step: the value named 'lif step' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `log_int_softmax_q15,` This is continuation line 16 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'log_int_softmax_q15,'. The important operands are log_int_softmax_q15: the value named 'log int softmax q15' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `packed_mac2_u8,` This is continuation line 17 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'packed_mac2_u8,'. The important operands are packed_mac2_u8: the value named 'packed mac2 u8' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `peak_dense_gops,` This is continuation line 18 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'peak_dense_gops,'. The important operands are peak_dense_gops: the value named 'peak dense gops' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 25: `quantize_flexible_scale,` This is continuation line 19 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'quantize_flexible_scale,'. The important operands are quantize_flexible_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 26: `round_shift_signed,` This is continuation line 20 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'round_shift_signed,'. The important operands are round_shift_signed: the value named 'round shift signed' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 27: `saturate_signed,` This is continuation line 21 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'saturate_signed,'. The important operands are saturate_signed: the value named 'saturate signed' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 28: `sgd_momentum_step,` This is continuation line 22 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'sgd_momentum_step,'. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 29: `ternary_spike,` This is continuation line 23 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'ternary_spike,'. The important operands are ternary_spike: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 30: `ternary_weighted_gemm,` This is continuation line 24 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'ternary_weighted_gemm,'. The important operands are ternary_weighted_gemm: a learned coefficient multiplied by an activation. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 31: `vector_alu,` This is continuation line 25 of the 26-line logical statement that begins at source line 7. Its exact fragment is 'vector_alu,'. The important operands are vector_alu: the value named 'vector alu' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 32: `)` This is continuation line 26 of the 26-line logical statement that begins at source line 7. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge golden behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 35: `class GoldenModelTests(unittest.TestCase):` This defines class GoldenModelTests, which groups data and methods for golden model tests. Instances exist in host memory and organize the flow that checks neuraedge golden behavior independently with Python assertions.

Line 36: `def setUp(self) -> None:` This starts function setUp. Its indented body performs set up as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 37: `self.rng = np.random.default_rng(20260711)` This assigns 'np.random.default_rng(20260711)' to 'self.rng ' in host memory. Here rng means the value named 'rng' in this block. The important operands are random: the value named 'random' in this block; default_rng: the value named 'default rng' in this block. Context: the function setUp, which handles set up.

Line 39: `def test_signed_encoding_round_trip(self) -> None:` This starts function test_signed_encoding_round_trip. Its indented body performs test signed encoding round trip as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 40: `values = np.arange(-128, 128, dtype=np.int16)` This assigns ‘np.arange(-128, 128, dtype=np.int16)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are arange: the value named ’arange’ in this block; dtype: the value named ’dtype’ in this block; int16: the value named ’int16’ in this block. Context: the function test_signed_encoding_round_trip, which handles test signed encoding round trip.

Line 41: `np.testing.assert_array_equal(decode_uint8_as_int8(encode_int8_as_uint8(values)), values)` This calls ‘np.testing.assert_array_equal‘ with ‘decode_uint8_as_int8(encode_int8_as_uint8(values)), values‘ inside the function test_signed_encoding_round_trip, which handles test signed encoding round trip. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 43: `def test_packed_mac_boundaries(self) -> None:` This starts function test_packed_mac_boundaries. Its indented body performs test packed mac boundaries as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 44: `for activation in (0, 1, 127, 128, 254, 255):` This iterates activation over ‘(0, 1, 127, 128, 254, 255)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_packed_mac_boundaries, which handles test packed mac boundaries.

Line 45: `for low in (0, 1, 127, 128, 254, 255):` This iterates low over ‘(0, 1, 127, 128, 254, 255)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_packed_mac_boundaries, which handles test packed mac boundaries.

Line 46: `for high in (0, 1, 127, 128, 254, 255):` This iterates high over ‘(0, 1, 127, 128, 254, 255)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_packed_mac_boundaries, which handles test packed mac boundaries.

Line 47: `got_low, got_high = packed_mac2_u8(activation, low, high)` This assigns ‘packed_mac2_u8(activation, low, high)‘ to ‘got_low, got_high ‘ in host memory. Here got_low means the value named ’got low’ in this block. The important operands are packed_mac2_u8: the value named ’packed mac2 u8’ in this block; activation: an input feature value consumed by a neuron or multiply-accumulate operation; low: the value named ’low’ in this block. Context: the function test_packed_mac_boundaries, which handles test packed mac boundaries.

Line 48: `self.assertEqual(got_low, activation * low)` This calls ‘self.assertEqual‘ with ‘got_low, activation * low‘ inside the function test_packed_mac_boundaries, which handles test packed mac boundaries. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 49: `self.assertEqual(got_high, activation * high)` This calls ‘self.assertEqual‘ with ‘got_high, activation * high‘ inside the function test_packed_mac_boundaries, which handles test packed mac boundaries. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 51: `def test_zero_point_correction_matches_direct_expression(self) -> None:` This starts function test_zero_point_correction_matches_direct_expression. Its indented body performs test zero point correction matches direct expression as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 52: `for _ in range(100):` This iterates _ over ‘range(100)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_zero_point_correction_matches_direct_expression, which handles test zero point correction matches direct expression.

Line 53: `rows = int(self.rng.integers(1, 9))` This assigns ‘int(self.rng.integers(1, 9))‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_zero_point_correction_matches_di which handles test zero point correction matches direct expression.

Line 54: `columns = int(self.rng.integers(1, 33))` This assigns ‘int(self.rng.integers(1, 33))‘ to ‘columns ‘ in host memory. Here columns means one output-feature position processed in parallel. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_zero_point_correction_matches_direct_expression, which handles test zero point correction matches direct expression.

Line 55: `reduction = int(self.rng.integers(1, 129))` This assigns ‘int(self.rng.integers(1, 129))‘ to ‘reduction ‘ in host memory. Here reduction means the value named ’reduction’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_zero_point_correction_matches_direct_expression, which handles test zero point correction matches direct expression.

Line 56: `a = self.rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)` This assigns ‘self.rng.integers(0, 256, size=(rows, reduction), dtype=np.uint8)‘ to ‘a ‘ in host memory. Here a means the value named ’a’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_zero_point_correction_matches_direct_expression, which handles test zero point correction matches direct expression.

Line 57: `b = self.rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)` This assigns ‘self.rng.integers(0, 256, size=(reduction, columns), dtype=np.uint8)‘ to ‘b ‘ in host memory. Here b means the value named ’b’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_zero_point_correction_matches_direct_expression, which handles test zero point correction matches direct expression.

Line 58: `za = int(self.rng.integers(0, 256))` This assigns ‘int(self.rng.integers(0, 256))‘ to ‘za ‘ in host memory. Here za means the value named ’za’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_zero_point_correction_matches_direct_expression, which handles test zero point correction matches direct expression.

Line 59: `zb = int(self.rng.integers(0, 256))` This assigns ‘int(self.rng.integers(0, 256))‘ to ‘zb ‘ in host memory. Here zb means the value named ’zb’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_zero_point_correction_matches_direct_expression, which handles test zero point correction matches direct expression.

Line 60: `bias = self.rng.integers(-(1 << 20), 1 << 20, size=columns, dtype=np.int64)` This assigns ‘self.rng.integers (1 << 20), 1 << 20, size=columns, dtype=np.int64)‘ to ‘bias ‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_zero_point_correction_matches which handles test zero point correction matches direct expression.

Line 61: `corrected = gemm_u8_zero_point(a, b, za, zb, bias)` This assigns ‘gemm_u8_zero_point(a, b, za, zb, bias)‘ to ‘corrected ‘ in host memory. Here corrected means the value named ’corrected’ in this block. The important operands are gemm_u8_zero_point: the integer code representing real numeric zero in affine quantization; a: the value named ’a’ in this block; b: the value named ’b’ in this block. Context: the function test_zero_point_correction_matches_direct_ex which handles test zero point correction matches direct expression.

Line 62: `direct = direct_centered_gemm(a, b, za, zb, bias)` This assigns ‘direct_centered_gemm(a, b, za, zb, bias)‘ to ‘direct ‘ in host memory. Here direct means the value named ’direct’ in this block. The important operands are direct_centered_gemm: the value named ’direct centered gemm’ in this block; a: the value named ’a’ in this block;

b: the value named 'b' in this block. Context: the function test_zero_point_correction_matches_direct_expression, which handles test zero point correction matches direct expression.

Line 63: `np.testing.assert_array_equal(corrected, direct)` This calls 'np.testing.assert_array_equal' with 'corrected, direct' inside the function test_zero_point_correction_matches_direct_expression, which handles test zero point correction matches direct expression. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 65: `def test_sparse_index_compaction(self) -> None:` This starts function test_sparse_index_compaction. Its indented body performs test sparse index compaction as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 66: `zero = 128` This assigns '128' to 'zero ' in host memory. Here zero means the value named 'zero' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_sparse_index_compaction, which handles test sparse index compaction.

Line 67: `a = np.full((3, 19), zero, dtype=np.uint8)` This assigns 'np.full((3, 19), zero, dtype=np.uint8)' to 'a ' in host memory. Here a means the value named 'a' in this block. The important operands are full: the value named 'full' in this block; zero: the value named 'zero' in this block; dtype: the value named 'dtype' in this block. Context: the function test_sparse_index_compaction, which handles test sparse index compaction.

Line 68: `a[0, 1] = 129` This assigns '129' to 'a[0, 1] ' in host memory. Here a means the value named 'a' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_sparse_index_compaction, which handles test sparse index compaction.

Line 69: `a[2, 9] = 127` This assigns '127' to 'a[2, 9] ' in host memory. Here a means the value named 'a' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_sparse_index_compaction, which handles test sparse index compaction.

Line 70: `a[1, 18] = 0` This assigns '0' to 'a[1, 18] ' in host memory. Here a means the value named 'a' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_sparse_index_compaction, which handles test sparse index compaction.

Line 71: `np.testing.assert_array_equal(active_outer_indices(a, zero), np.array([1, 9, 18]))` This calls 'np.testing.assert_array_equal' with 'active_outer_indices(a, zero), np.array([1, 9, 18])' inside the function test_sparse_index_com which handles test sparse index compaction. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 72: `np.testing.assert_array_equal(active_dot8_indices(a[0], zero), np.array([0]))` This calls 'np.testing.asser with 'active_dot8_indices(a[0], zero), np.array([0])' inside the function test_sparse_index_compaction, which handles test sparse index compaction. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 73: `np.testing.assert_array_equal(active_dot8_indices(a[2], zero), np.array([1]))` This calls 'np.testing.asser with 'active_dot8_indices(a[2], zero), np.array([1])' inside the function test_sparse_index_compaction, which handles test sparse index compaction. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 75: `def test_binary_and_ternary_products(self) -> None:` This starts function test_binary_and_ternary_products. Its indented body performs test binary and ternary products as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 76: `weights = self.rng.integers(-127, 128, size=(17, 11), dtype=np.int16)` This assigns ‘self.rng.integers(-127, 128, size=(17, 11), dtype=np.int16)‘ to ‘weights ‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_binary_and_ternary_products, which handles test binary and ternary products.

Line 77: `binary = self.rng.integers(0, 2, size=(5, 17), dtype=np.int8)` This assigns ‘self.rng.integers(0, 2, size=(5, 17), dtype=np.int8)‘ to ‘binary ‘ in host memory. Here binary means the value named ’binary’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_binary_and_ternary_products, which handles test binary and ternary products.

Line 78: `ternary = self.rng.integers(-1, 2, size=(5, 17), dtype=np.int8)` This assigns ‘self.rng.integers(-1, 2, size=(5, 17), dtype=np.int8)‘ to ‘ternary ‘ in host memory. Here ternary means the value named ’ternary’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_binary_and_ternary_products, which handles test binary and ternary products.

Line 79: `np.testing.assert_array_equal(binary_weighted_gemm(binary, weights), binary.astype(np.int64) @ weights)` This calls ‘np.testing.assert_array_equal‘ with ‘binary_weighted_gemm(binary, weights), binary.astype(np.int64) @ weights‘ inside the function test_binary_and_ternary_products, which handles test binary and ternary products. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 80: `np.testing.assert_array_equal(ternary_weighted_gemm(ternary, weights), ternary.astype(np.int64) @ weights)` This calls ‘np.testing.assert_array_equal‘ with ‘ternary_weighted_gemm(ternary, weights), ternary.astype(np.int64) @ weights‘ inside the function test_binary_and_ternary_products, which handles test binary and ternary products. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 82: `def test_alpha_xnor_special_cases(self) -> None:` This starts function test_alpha_xnor_special_cases. Its indented body performs test alpha xnor special cases as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 83: `q = np.array([[1, 0, 1, 0], [0, 0, 0, 0]], dtype=np.uint8)` This assigns ‘np.array([[1, 0, 1, 0], [0, 0, 0, 0]], dtype=np.uint8)‘ to ‘q ‘ in host memory. Here q means the value named ’q’ in this block. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function test_alpha_xnor_special_cases, which handles test alpha xnor special cases.

Line 84: `k = np.array([[1, 0, 0, 0], [0, 0, 0, 0]], dtype=np.uint8)` This assigns ‘np.array([[1, 0, 0, 0], [0, 0, 0, 0]], dtype=np.uint8)‘ to ‘k ‘ in host memory. Here k means the value named ’k’ in this block. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function test_alpha_xnor_special_cases, which handles test alpha xnor special cases.

Line 85: `dot_q8 = alpha_xnor_similarity(q, k, 0)` This assigns ‘alpha_xnor_similarity(q, k, 0)‘ to ‘dot_q8 ‘ in host memory. Here dot_q8 means the value named ’dot q8’ in this block. The important operands are alpha_xnor_similarity: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; q: the value named ’q’ in this block; k: the value named ’k’ in this block. Context: the function test_alpha_xnor_special_cases, which handles test alpha xnor special cases.

Line 86: `xnor_q8 = alpha_xnor_similarity(q, k, 256)` This assigns ‘alpha_xnor_similarity(q, k, 256)‘ to ‘xnor_q8 ‘ in host memory. Here xnor_q8 means the value named ’xnor q8’ in this block. The important operands are alpha_xnor_similarity: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; q: the value named ’q’ in this block; k: the value named ’k’ in this block. Context: the function test_alpha_xnor_special_cases, which handles test alpha xnor special cases.

Line 87: `np.testing.assert_array_equal(dot_q8 // 256, q.astype(np.int64) @ k.astype(np.int64).T)` This calls ‘np.testing.assert_array_equal‘ with ‘dot_q8 // 256, q.astype(np.int64) @ k.astype(np.int64).T‘ inside the function test_alpha_xnor_special_cases, which handles test alpha xnor special cases. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 88: `expected_xnor = np.array([[3, 2], [3, 4]], dtype=np.int64)` This assigns ‘np.array([[3, 2], [3, 4]], dtype=np.int64)‘ to ‘expected_xnor ‘ in host memory. Here expected_xnor means the value named ’expected xnor’ in this block. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_alpha_xnor_special_cases, which handles test alpha xnor special cases.

Line 89: `np.testing.assert_array_equal(xnor_q8 // 256, expected_xnor)` This calls ‘np.testing.assert_array_equal‘ with ‘xnor_q8 // 256, expected_xnor‘ inside the function test_alpha_xnor_special_cases, which handles test alpha xnor special cases. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 91: `def test_log_integer_softmax_properties(self) -> None:` This starts function test_log_integer_softmax_prope Its indented body performs test log integer softmax properties as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 92: `scores = np.array([91, 83, 50, -30], dtype=np.int64)` This assigns ‘np.array([91, 83, 50, -30], dtype=np.int64)‘ to ‘scores ‘ in host memory. Here scores means the value named ’scores’ in this block. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_log_integer_softmax_properties, which handles test log integer softmax properties.

Line 93: `probabilities = log_int_softmax_q15(scores, difference_shift=3)` This assigns ‘log_int_softmax_q15(scores, difference_shift=3)‘ to ‘probabilities ‘ in host memory. Here probabilities means the value named ’probabilities’ in this block. The important operands are log_int_softmax_q15: the value named ’log int softmax q15’ in this block; scores: the value named ’scores’ in this block; difference_shift: the value named ’difference shift’ in this block. Context: the function test_log_integer_softmax_properties, which handles test log integer softmax properties.

Line 94: `translated = log_int_softmax_q15(scores + 123456, difference_shift=3)` This assigns ‘log_int_softmax_q15 + 123456, difference_shift=3)‘ to ‘translated ‘ in host memory. Here translated means the value named ’translated’ in this block. The important operands are log_int_softmax_q15: the value named ’log int softmax q15’ in this block; scores: the value named ’scores’ in this block; difference_shift: the value named ’difference shift’ in this block. Context: the function test_log_integer_softmax_properties, which handles test log integer softmax properties.

Line 95: `np.testing.assert_array_equal(probabilities, translated)` This calls ‘np.testing.assert_array_equal‘ with ‘probabilities, translated‘ inside the function test_log_integer_softmax_properties, which handles test log integer softmax properties. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 96: `self.assertGreaterEqual(int(probabilities[0]), int(probabilities[1]))` This calls ‘self.assertGreaterEqual‘ with ‘int(probabilities[0]), int(probabilities[1])‘ inside the function test_log_integer_softmax_properties, which handles test log integer softmax properties. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 97: `self.assertGreaterEqual(int(probabilities[1]), int(probabilities[2]))` This calls ‘self.assertGreaterEqual‘ with ‘int(probabilities[1]), int(probabilities[2])‘ inside the function test_log_integer_softmax_properties, which handles test log integer softmax properties. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 98: `self.assertLessEqual(abs(int(probabilities.sum()) - (1 << 15)), 2)` This calls ‘self.assertLessEqual‘ with ‘abs(int(probabilities.sum()) - (1 << 15)), 2‘ inside the function test_log_integer_softmax_properties, which handles test log integer softmax properties. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 100: `def test_integer_layer_norm_constant_and_nonconstant(self) -> None:` This starts function test_integer_lay Its indented body performs test integer layer norm constant and nonconstant as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 101: `gamma = np.full(4, 1 << 15, dtype=np.int64)` This assigns ‘np.full(4, 1 << 15, dtype=np.int64)‘ to ‘gamma ‘ in host memory. Here gamma means the learned multiplicative LayerNorm parameter. The important operands are full: the value named ’full’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_integer_layer_norm_constant_and_nonconstant, which handles test integer layer norm constant and nonconstant.

Line 102: `beta = np.array([1, 2, 3, 4], dtype=np.int64)` This assigns ‘np.array([1, 2, 3, 4], dtype=np.int64)‘ to ‘beta ‘ in host memory. Here beta means the learned additive LayerNorm parameter. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_integer_layer_norm_constant_and_nonconstant, which handles test integer layer norm constant and nonconstant.

Line 103: `np.testing.assert_array_equal(integer_layer_norm(np.full(4, 12), gamma, beta), beta)` This calls ‘np.testing.assert_array_equal‘ with ‘integer_layer_norm(np.full(4, 12), gamma, beta), beta‘ inside the function test_integer_layer_norm_constant_and_nonconstant, which handles test integer layer norm constant and nonconstant. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 104: `output = integer_layer_norm(np.array([-9, -3, 3, 9]), gamma, np.zeros(4, dtype=np.int64))` This assigns ‘integer_layer_norm(np.array([-9, -3, 3, 9]), gamma, np.zeros(4, dtype=np.int64))‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are integer_layer_norm: the value named ’integer layer norm’ in this block; array: the value named ’array’ in this block; gamma: the learned multiplicative LayerNorm parameter. Context: the function test_integer_layer_norm_constant_and_nonconstant, which handles test integer layer norm constant and nonconstant.

Line 105: `self.assertEqual(int(output.sum()), 0)` This calls ‘self.assertEqual‘ with ‘int(output.sum()), 0‘ inside the function test_integer_layer_norm_constant_and_nonconstant, which handles test integer layer norm constant and nonconstant. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 106: `self.assertTrue(np.all(np.diff(output) >= 0))` This calls ‘self.assertTrue‘ with ‘np.all(np.diff(output) >= 0)‘ inside the function test_integer_layer_norm_constant_and_nonconstant, which handles test integer layer norm constant and nonconstant. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 108: `def test_lif_and_ternary_firing(self) -> None:` This starts function test_lif_and_ternary_firing. Its indented body performs test lif and ternary firing as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 109: `next_membrane, spikes = lif_step(np.array([0, 8, 20]), np.array([4, 8, 1]), 2, 12)` This assigns ‘lif_step(np.array([0, 8, 20]), np.array([4, 8, 1]), 2, 12)‘ to ‘next_membrane, spikes ‘ in host memory. Here next_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are lif_step: the value named ’lif step’ in this block; array: the value named ’array’ in this block. Context: the function test_lif_and_ternary_firing, which handles test lif and ternary firing.

Line 110: `np.testing.assert_array_equal(spikes, np.array([0, 1, 1], dtype=np.uint8))` This calls ‘np.testing.assert_ with ‘spikes, np.array([0, 1, 1], dtype=np.uint8)‘ inside the function test_lif_and_ternary_firing, which handles test lif and ternary firing. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 111: `np.testing.assert_array_equal(next_membrane, np.array([4, 0, 0]))` This calls ‘np.testing.assert_array_equ with ‘next_membrane, np.array([4, 0, 0])‘ inside the function test_lif_and_ternary_firing, which handles test lif and ternary firing. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 112: `np.testing.assert_array_equal(` This statement continues through source line 115; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.testing.assert_array_equal‘ with ‘ ternary_spike(np.array([-8, -7, 0, 7, 8]), 8), np.array([-1, 0, 0, 0, 1], dtype=np.int8), ‘ inside the function test_lif_and_ternary_firing, which handles test lif and ternary firing. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 113: `ternary_spike(np.array([-8, -7, 0, 7, 8]), 8),` This is continuation line 2 of the 4-line logical statement that begins at source line 112. Its exact fragment is ‘ternary_spike(np.array([-8, -7, 0, 7, 8]), 8),‘. The important operands are ternary_spike: a binary event produced when a neuron crosses its threshold; array: the value named ’array’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ ternary_spike(np.array([-8, -7, 0, 7, 8]), 8), np.array([-1, 0, 0, 0, 1], dtype=np.int8), ‘ inside the function test_lif_and_ternary_firing, which handles test lif and ternary firing. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 114: `np.array([-1, 0, 0, 0, 1], dtype=np.int8),` This is continuation line 3 of the 4-line logical statement that begins at source line 112. Its exact fragment is ‘np.array([-1, 0, 0, 0, 1], dtype=np.int8),‘. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int8: the value named ’int8’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ ternary_spike(np.array([-8, -7, 0, 7, 8]), 8), np.array([-1, 0, 0, 0, 1], dtype=np.int8), ‘ inside the function test_lif_and_ternary_firing, which handles test lif and ternary firing. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 115: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 112. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ ternary_spike(np.array([-8, -7, 0, 7, 8]), 8), np.array([-1, 0, 0, 0, 1], dtype=np.int8), ‘ inside the function test_lif_and_ternary_firing, which handles test lif and ternary firing. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 117: `def test_dataflow_selection_and_peak_rate(self) -> None:` This starts function test_dataflow_selection_and_ Its indented body performs test dataflow selection and peak rate as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 118: `wide = choose_dataflow(rows=8, columns=768, reduction=384)` This assigns ‘choose_dataflow(rows=8, columns=768, reduction=384)‘ to ‘wide ‘ in host memory. Here wide means the value named ’wide’ in this block. The important operands are choose_dataflow: the value named ’choose dataflow’ in this block; rows: one output-row position processed by the tensor tile; columns: one output-feature position processed in parallel. Context: the function test_dataflow_selection_and_peak_rate, which handles test dataflow selection and peak rate.

Line 119: `tall = choose_dataflow(rows=4096, columns=16, reduction=144)` This assigns ‘choose_dataflow(rows=4096, columns=16, reduction=144)‘ to ‘tall ‘ in host memory. Here tall means the value named ’tall’ in this block. The important operands are choose_dataflow: the value named ’choose dataflow’ in this block; rows: one output-row position processed by the tensor tile; columns: one output-feature position processed in parallel. Context: the function test_dataflow_selection_and_peak_rate, which handles test dataflow selection and peak rate.

Line 120: `self.assertEqual(wide.mode, Dataflow.INPUT_BROADCAST)` This calls ‘self.assertEqual‘ with ‘wide.mode, Dataflow.INPUT_BROADCAST‘ inside the function test_dataflow_selection_and_peak_rate, which handles test dataflow selection and peak rate. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 121: `self.assertEqual(wide.input_broadcast_bytes, 297984)` This calls ‘self.assertEqual‘ with ‘wide.input_broadcast 297984‘ inside the function test_dataflow_selection_and_peak_rate, which handles test dataflow selection and peak rate. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 122: `self.assertEqual(wide.weight_broadcast_bytes, 442368)` This calls ‘self.assertEqual‘ with ‘wide.weight_broadca 442368‘ inside the function test_dataflow_selection_and_peak_rate, which handles test dataflow selection and peak rate. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 123: `self.assertEqual(tall.mode, Dataflow.WEIGHT_BROADCAST)` This calls ‘self.assertEqual‘ with ‘tall.mode, Dataflow.WEIGHT_BROADCAST‘ inside the function test_dataflow_selection_and_peak_rate, which handles test dataflow selection and peak rate. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 124: `self.assertEqual(tall.input_broadcast_bytes, 1769472)` This calls ‘self.assertEqual‘ with ‘tall.input_broadcast 1769472‘ inside the function test_dataflow_selection_and_peak_rate, which handles test dataflow selection and peak rate. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 125: `self.assertEqual(tall.weight_broadcast_bytes, 592128)` This calls ‘self.assertEqual‘ with ‘tall.weight_broadcas 592128‘ inside the function test_dataflow_selection_and_peak_rate, which handles test dataflow selection and peak rate. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 126: `self.assertAlmostEqual(peak_dense_gops(200e6, 64), 51.2)` This calls ‘self.assertAlmostEqual‘ with ‘peak_dense_gops(200e6, 64), 51.2‘ inside the function test_dataflow_selection_and_peak_rate, which handles test dataflow selection and peak rate. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 128: `def test_signed_rounding_and_saturation(self) -> None:` This starts function test_signed_rounding_and_sat Its indented body performs test signed rounding and saturation as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 129: `values = np.array([-9, -8, -7, -1, 0, 1, 7, 8, 9], dtype=np.int64)` This assigns ‘np.array([-9, -8, -7, -1, 0, 1, 7, 8, 9], dtype=np.int64)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_signed_rounding_and_saturation, which handles test signed rounding and saturation.

Line 130: `np.testing.assert_array_equal(` This statement continues through source line 133; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.testing.assert_array_equal‘ with ‘ round_shift_signed(values, 3), np.array([-1, -1, -1, 0, 0, 0, 1, 1, 1], dtype=np.int64), ‘ inside the function test_signed_rounding_and_saturation, which handles test signed rounding and saturation. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 131: `round_shift_signed(values, 3),` This is continuation line 2 of the 4-line logical statement that begins at source line 130. Its exact fragment is ‘round_shift_signed(values, 3),‘. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; values: the value named ’values’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ round_shift_signed(values, 3), np.array([-1, -1, -1, 0, 0, 0, 1, 1, 1], dtype=np.int64), ‘ inside the function test_signed_rounding_and_saturation, which handles test signed rounding and saturation. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 132: `np.array([-1, -1, -1, 0, 0, 0, 1, 1, 1], dtype=np.int64),` This is continuation line 3 of the 4-line logical statement that begins at source line 130. Its exact fragment is 'np.array([-1, -1, -1, 0, 0, 0, 1, 1, 1], dtype=np.int64),'. The important operands are array: the value named 'array' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' round_shift_signed(values, 3), np.array([-1, -1, -1, 0, 0, 0, 1, 1, 1], dtype=np.int64), ' inside the function test_signed_rounding_and_saturation, which handles test signed rounding and saturation. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 133: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 130. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' round_shift_signed(values, 3), np.array([-1, -1, -1, 0, 0, 0, 1, 1, 1], dtype=np.int64), ' inside the function test_signed_rounding_and_saturation, which handles test signed rounding and saturation. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 134: `np.testing.assert_array_equal(` This statement continues through source line 137; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_array_equal' with ' saturate_signed(np.array([-40000, -32768, 32767, 40000]), 16), np.array([-32768, -32768, 32767, 32767]), ' inside the function test_signed_rounding_and_saturation, which handles test signed rounding and saturation. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 135: `saturate_signed(np.array([-40000, -32768, 32767, 40000]), 16),` This is continuation line 2 of the 4-line logical statement that begins at source line 134. Its exact fragment is 'saturate_signed(np.array([-40000, -32768, 32767, 40000]), 16),'. The important operands are saturate_signed: the value named 'saturate signed' in this block; array: the value named 'array' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' saturate_signed(np.array([-40000, -32768, 32767, 40000]), 16), np.array([-32768, -32768, 32767, 32767]), ' inside the function test_signed_rounding_and_saturation, which handles test signed rounding and saturation. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 136: `np.array([-32768, -32768, 32767, 32767]),` This is continuation line 3 of the 4-line logical statement that begins at source line 134. Its exact fragment is 'np.array([-32768, -32768, 32767, 32767]),'. The important operands are array: the value named 'array' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' saturate_signed(np.array([-40000, -32768, 32767, 40000]), 16), np.array([-32768, -32768, 32767, 32767]), ' inside the function test_signed_rounding_and_saturation, which handles test signed rounding and saturation. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 137: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 134. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' saturate_signed(np.array([-40000, -32768, 32767, 40000]), 16), np.array([-32768, -32768, 32767, 32767]), ' inside the function test_signed_rounding_and_saturation, which handles test signed rounding and saturation. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 139: `def test_linear_backward_matches_independent_scalar_reference(self) -> None:` This starts function test_linear_backward_matches_independent_scalar_reference. Its indented body performs test linear backward matches independent scalar reference as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 140: `def rounded(value: int, shift: int) -> int:` This starts function rounded. Its indented body performs rounded as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 141: `if shift == 0:` This executes the following indented host-code branch only when 'shift == 0' is true. The important operands are shift: the value named 'shift' in this block. Context: the function rounded, which handles rounded.

Line 142: `return value` This returns ‘value‘ to the caller of the function rounded, which handles rounded, ending this invocation. The important operands are value: the value named ’value’ in this block.

Line 143: `magnitude = abs(value)` This assigns ‘abs(value)‘ to ‘magnitude ‘ in host memory. Here magnitude means the value named ’magnitude’ in this block. The important operands are abs: the value named ’abs’ in this block; value: the value named ’value’ in this block. Context: the function rounded, which handles rounded.

Line 144: `result = (magnitude + (1 << (shift - 1))) >> shift` This assigns ‘(magnitude + (1 << (shift - 1))) >> shift‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are magnitude: the value named ’magnitude’ in this block; shift: the value named ’shift’ in this block. Context: the function rounded, which handles rounded.

Line 145: `return -result if value < 0 else result` This returns ‘-result if value < 0 else result‘ to the caller of the function rounded, which handles rounded, ending this invocation. The important operands are result: the value named ’result’ in this block; value: the value named ’value’ in this block.

Line 147: `def clipped(value: int, bits: int) -> int:` This starts function clipped. Its indented body performs clipped as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 148: `return max(-(1 << (bits - 1)), min((1 << (bits - 1)) - 1, value))` This returns ‘max(-(1 << (bits - 1)), min((1 << (bits - 1)) - 1, value))‘ to the caller of the function clipped, which handles clipped, ending this invocation. The important operands are max: the value named ’max’ in this block; bits: the value named ’bits’ in this block; min: the value named ’min’ in this block.

Line 150: `for _ in range(100):` This iterates _ over ‘range(100)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_linear_backward_matches_independent_scalar_refer which handles test linear backward matches independent scalar reference.

Line 151: `rows = int(self.rng.integers(1, 9))` This assigns ‘int(self.rng.integers(1, 9))‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_linear_backward_matches_i which handles test linear backward matches independent scalar reference.

Line 152: `inputs = int(self.rng.integers(1, 9))` This assigns ‘int(self.rng.integers(1, 9))‘ to ‘inputs ‘ in host memory. Here inputs means the value named ’inputs’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_linear_backward_matches_independent_ which handles test linear backward matches independent scalar reference.

Line 153: `outputs = int(self.rng.integers(1, 9))` This assigns ‘int(self.rng.integers(1, 9))‘ to ‘outputs ‘ in host memory. Here outputs means the value named ’outputs’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_linear_backward_matches_indepen which handles test linear backward matches independent scalar reference.

Line 154: `dx_shift = int(self.rng.integers(0, 13))` This assigns ‘int(self.rng.integers(0, 13))‘ to ‘dx_shift ‘ in host memory. Here dx_shift means the value named ’dx shift’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_linear_backward_matches_i which handles test linear backward matches independent scalar reference.

Line 155: `dw_shift = int(self.rng.integers(0, 13))` This assigns ‘int(self.rng.integers(0, 13))‘ to ‘dw_shift ‘ in host memory. Here dw_shift means the value named ’dw shift’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 156: `x = self.rng.integers(-128, 128, size=(rows, inputs), dtype=np.int64)` This assigns ‘self.rng.integers(-128, 128, size=(rows, inputs), dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 157: `w = self.rng.integers(-32768, 32768, size=(inputs, outputs), dtype=np.int64)` This assigns ‘self.rng.integ 32768, 32768, size=(inputs, outputs), dtype=np.int64)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_linear_backward_matches_independent_scalar_referen which handles test linear backward matches independent scalar reference.

Line 158: `dy = self.rng.integers(-32768, 32768, size=(rows, outputs), dtype=np.int64)` This assigns ‘self.rng.intege 32768, 32768, size=(rows, outputs), dtype=np.int64)‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_linear_backward_matches_independent_scalar_referen which handles test linear backward matches independent scalar reference.

Line 159: `expected_dx = np.zeros((rows, inputs), dtype=np.int64)` This assigns ‘np.zeros((rows, inputs), dtype=np.int6 to ‘expected_dx ‘ in host memory. Here expected_dx means the value named ’expected dx’ in this block. The important operands are zeros: the value named ’zeros’ in this block; rows: one output-row position processed by the tensor tile; inputs: the value named ’inputs’ in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 160: `expected_dw = np.zeros((inputs, outputs), dtype=np.int64)` This assigns ‘np.zeros((inputs, outputs), dtype=np.int64)‘ to ‘expected_dw ‘ in host memory. Here expected_dw means the value named ’expected dw’ in this block. The important operands are zeros: the value named ’zeros’ in this block; inputs: the value named ’inputs’ in this block; outputs: the value named ’outputs’ in this block. Context: the function test_linear_backward_matches_independent_sc which handles test linear backward matches independent scalar reference.

Line 161: `for row in range(rows):` This iterates row over ‘range(rows)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_linear_backward_matches_independent_scalar_re which handles test linear backward matches independent scalar reference.

Line 162: `for input_index in range(inputs):` This iterates input_index over ‘range(inputs)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_linear_backward_matche which handles test linear backward matches independent scalar reference.

Line 163: `total = sum(` This statement continues through source line 166; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sum(int(dy[row, output_index]) * int(w[input_index, output_index]) for output_index in range(outputs))‘ to ‘total ‘ in host memory. Here total means the value named ’total’ in this block. The important operands are sum: the value named ’sum’ in this block; dy: the value named ’dy’ in this block; row: one output-row position processed by the tensor tile. Context: the function test_linear_backward_matches_independent_scal which handles test linear backward matches independent scalar reference.

Line 164: `int(dy[row, output_index]) * int(w[input_index, output_index])` This is continuation line 2 of the 4-line logical statement that begins at source line 163. Its exact fragment is ‘int(dy[row, output_index]) * int(w[input_index, output_index])‘. The important operands are dy: the value named ’dy’ in this block; row: one output-row position processed by the tensor tile; output_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns ‘sum(int(dy[row, output_index]) * int(w[input_index, output_index]) for output_index in range(outputs))‘ to ‘total ‘ in host memory. Here total means the value named ’total’ in this block. The important operands are sum: the value named ’sum’ in this block; dy: the value named ’dy’ in this block; row: one output-row position processed by the tensor tile. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 165: `for output_index in range(outputs)` This is continuation line 3 of the 4-line logical statement that begins at source line 163. Its exact fragment is 'for output_index in range(outputs)'. The important operands are for: the value named 'for' in this block; output_index: an integer selecting one tensor element, lane, bank, or state entry; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'sum(int(dy[row, output_index]) * int(w[input_index, output_index]) for output_index in range(outputs))' to 'total ' in host memory. Here total means the value named 'total' in this block. The important operands are sum: the value named 'sum' in this block; dy: the value named 'dy' in this block; row: one output-row position processed by the tensor tile. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 166: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 163. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sum(int(dy[row, output_index]) * int(w[input_index, output_index]) for output_index in range(outputs))' to 'total ' in host memory. Here total means the value named 'total' in this block. The important operands are sum: the value named 'sum' in this block; dy: the value named 'dy' in this block; row: one output-row position processed by the tensor tile. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 167: `expected_dx[row, input_index] = clipped(rounded(total, dx_shift), 32)` This assigns 'clipped(rounded(tota dx_shift), 32)' to 'expected_dx[row, input_index] ' in host memory. Here expected_dx means the value named 'expected dx' in this block. The important operands are clipped: the value named 'clipped' in this block; rounded: the value named 'rounded' in this block; total: the value named 'total' in this block. Context: the function test_linear_backward_matches_independ which handles test linear backward matches independent scalar reference.

Line 168: `for input_index in range(inputs):` This iterates input_index over 'range(inputs)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_linear_backward_matche which handles test linear backward matches independent scalar reference.

Line 169: `for output_index in range(outputs):` This iterates output_index over 'range(outputs)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_linear_backward_matc which handles test linear backward matches independent scalar reference.

Line 170: `total = sum(` This statement continues through source line 173; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sum(int(x[row, input_index]) * int(dy[row, output_index]) for row in range(rows))' to 'total ' in host memory. Here total means the value named 'total' in this block. The important operands are sum: the value named 'sum' in this block; x: the value named 'x' in this block; row: one output-row position processed by the tensor tile. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 171: `int(x[row, input_index]) * int(dy[row, output_index])` This is continuation line 2 of the 4-line logical statement that begins at source line 170. Its exact fragment is 'int(x[row, input_index]) * int(dy[row, output_index])'. The important operands are x: the value named 'x' in this block; row: one output-row position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'sum(int(x[row, input_index]) * int(dy[row, output_index]) for row in range(rows))' to 'total ' in host memory. Here total means the value named 'total' in this block. The important operands are sum: the value named 'sum' in this block; x: the value named 'x' in this block; row: one output-row position processed by the tensor tile. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 172: `for row in range(rows)` This is continuation line 3 of the 4-line logical statement that begins at source line 170. Its exact fragment is 'for row in range(rows)'. The important operands are for: the value named 'for' in this block; row: one output-row position processed by the tensor tile; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'sum(int(x[row, input_index]) * int(dy[row, output_index]) for row in range(rows))' to 'total ' in host memory. Here total means the value named 'total' in this block. The important operands are sum: the value named 'sum' in this block; x: the value named 'x' in this block; row: one output-row position processed by the

tensor tile. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 173: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 170. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sum(int(x[row, input_index]) * int(dy[row, output_index]) for row in range(rows))' to 'total ' in host memory. Here total means the value named 'total' in this block. The important operands are sum: the value named 'sum' in this block; x: the value named 'x' in this block; row: one output-row position processed by the tensor tile. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 174: `expected_dw[input_index, output_index] = clipped(rounded(total, dw_shift), 24)` This assigns 'clipped(rounded(total, dw_shift), 24)' to 'expected_dw[input_index, output_index] ' in host memory. Here expected_dw means the value named 'expected dw' in this block. The important operands are clipped: the value named 'clipped' in this block; rounded: the value named 'rounded' in this block; total: the value named 'total' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 175: `got_dx, got_dw = linear_backward_tile(` This statement continues through source line 181; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'linear_backward_tile(x, w, dy, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'got_dx, got_dw ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are linear_backward_tile: the value named 'linear backward tile' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 176: `x,` This is continuation line 2 of the 7-line logical statement that begins at source line 175. Its exact fragment is 'x,'. The important operands are x: the value named 'x' in this block. Read together, the complete operation is: This assigns 'linear_backward_tile(x, w, dy, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'got_dx, got_dw ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are linear_backward_tile: the value named 'linear backward tile' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 177: `w,` This is continuation line 3 of the 7-line logical statement that begins at source line 175. Its exact fragment is 'w,'. The important operands are w: the value named 'w' in this block. Read together, the complete operation is: This assigns 'linear_backward_tile(x, w, dy, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'got_dx, got_dw ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are linear_backward_tile: the value named 'linear backward tile' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 178: `dy,` This is continuation line 4 of the 7-line logical statement that begins at source line 175. Its exact fragment is 'dy,'. The important operands are dy: the value named 'dy' in this block. Read together, the complete operation is: This assigns 'linear_backward_tile(x, w, dy, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'got_dx, got_dw ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are linear_backward_tile: the value named 'linear backward tile' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 179: `input_gradient_shift=dx_shift,` This is continuation line 5 of the 7-line logical statement that begins at source line 175. Its exact fragment is 'input_gradient_shift=dx_shift,'. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter; dx_shift: the value named 'dx shift' in this block. Read together, the complete operation is: This assigns 'linear_backward_tile(x, w, dy, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'got_dx, got_dw ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are linear_backward_tile: the value named 'linear

backward tile' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 180: `weight_gradient_shift=dw_shift,` This is continuation line 6 of the 7-line logical statement that begins at source line 175. Its exact fragment is 'weight_gradient_shift=dw_shift,'. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation; dw_shift: the value named 'dw shift' in this block. Read together, the complete operation is: This assigns 'linear_backward_tile(x, w, dy, input_gradient_shift=dx_shift, weight_gradient_shift=dw_sh)' to 'got_dx, got_dw ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are linear_backward_tile: the value named 'linear backward tile' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 181: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 175. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'linear_backward_tile(x, w, dy, input_gradient_shift=dx_shift, weight_gradient_shift=dw_shift,)' to 'got_dx, got_dw ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are linear_backward_tile: the value named 'linear backward tile' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 182: `np.testing.assert_array_equal(got_dx, expected_dx)` This calls 'np.testing.assert_array_equal' with 'got_dx, expected_dx' inside the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 183: `np.testing.assert_array_equal(got_dw, expected_dw)` This calls 'np.testing.assert_array_equal' with 'got_dw, expected_dw' inside the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 185: `x = np.full((8, 8), 127, dtype=np.int64)` This assigns 'np.full((8, 8), 127, dtype=np.int64)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are full: the value named 'full' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 186: `w = np.full((8, 8), -32768, dtype=np.int64)` This assigns 'np.full((8, 8), -32768, dtype=np.int64)' to 'w ' in host memory. Here w means the value named 'w' in this block. The important operands are full: the value named 'full' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 187: `dy = np.full((8, 8), -32768, dtype=np.int64)` This assigns 'np.full((8, 8), -32768, dtype=np.int64)' to 'dy ' in host memory. Here dy means the value named 'dy' in this block. The important operands are full: the value named 'full' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 188: `got_dx, got_dw = linear_backward_tile(x, w, dy)` This assigns 'linear_backward_tile(x, w, dy)' to 'got_dx, got_dw ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are linear_backward_tile: the value named 'linear backward tile' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference.

Line 189: `np.testing.assert_array_equal(got_dx, np.full((8, 8), (1 << 31) - 1))` This calls ‘np.testing.assert_array
with ‘got_dx, np.full((8, 8), (1 << 31) - 1)‘ inside the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 190: `np.testing.assert_array_equal(got_dw, np.full((8, 8), -(1 << 23)))` This calls ‘np.testing.assert_array_eq
with ‘got_dw, np.full((8, 8), -(1 << 23))‘ inside the function test_linear_backward_matches_independent_scalar_reference, which handles test linear backward matches independent scalar reference. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 192: `def test_sgd_momentum_update(self) -> None:` This starts function test_sgd_momentum_update. Its indented body performs test sgd momentum update as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 193: `for _ in range(100):` This iterates _ over ‘range(100)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_sgd_momentum_update, which handles test sgd momentum update.

Line 194: `weights = self.rng.integers(-32768, 32768, size=8, dtype=np.int64)` This assigns ‘self.rng.integers(-32768, 32768, size=8, dtype=np.int64)‘ to ‘weights ‘ in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_sgd_momentum_update, which handles test sgd momentum update.

Line 195: `gradients = self.rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)` This assigns ‘self.rng.integers
(1 << 20), 1 << 20, size=8, dtype=np.int64)‘ to ‘gradients ‘ in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_sgd_momentum_update, which handles test sgd momentum update.

Line 196: `velocities = self.rng.integers(-(1 << 20), 1 << 20, size=8, dtype=np.int64)` This assigns ‘self.rng.intege
(1 << 20), 1 << 20, size=8, dtype=np.int64)‘ to ‘velocities ‘ in host memory. Here velocities means the value named ’velocities’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_sgd_momentum_update, which handles test sgd momentum update.

Line 197: `next_weight, next_velocity = sgd_momentum_step(` This statement continues through source line 199; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66)‘ to ‘next_weight, next_velocity ‘ in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named ’sgd momentum step’ in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_sgd_momentum_update, which handles test sgd momentum update.

Line 198: `weights, gradients, velocities, 655, 58982, 66` This is continuation line 2 of the 3-line logical statement that begins at source line 197. Its exact fragment is ‘weights, gradients, velocities, 655, 58982, 66‘. The important operands are weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter; velocities: the value named ’velocities’ in this block. Read together, the complete operation is: This assigns ‘sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66)‘ to ‘next_weight, next_velocity ‘ in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named ’sgd momentum step’ in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_sgd_momentum_update, which handles test sgd momentum update.

Line 199: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 197. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_sgd_momentum_update, which handles test sgd momentum update.

Line 200: `self.assertTrue(np.all(next_weight >= -32768))` This calls 'self.assertTrue' with 'np.all(next_weight >= -32768)' inside the function test_sgd_momentum_update, which handles test sgd momentum update. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 201: `self.assertTrue(np.all(next_weight <= 32767))` This calls 'self.assertTrue' with 'np.all(next_weight <= 32767)' inside the function test_sgd_momentum_update, which handles test sgd momentum update. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 202: `self.assertTrue(np.all(next_velocity >= -(1 << 23)))` This calls 'self.assertTrue' with 'np.all(next_velocity >= -(1 << 23))' inside the function test_sgd_momentum_update, which handles test sgd momentum update. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 203: `self.assertTrue(np.all(next_velocity <= (1 << 23) - 1))` This calls 'self.assertTrue' with 'np.all(next_velocit <= (1 << 23) - 1)' inside the function test_sgd_momentum_update, which handles test sgd momentum update. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 205: `def test_sgd_weight_decay_is_decoupled_from_velocity(self) -> None:` This starts function test_sgd_weight Its indented body performs test sgd weight decay is decoupled from velocity as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 206: `next_weight, next_velocity = sgd_momentum_step(` This statement continues through source line 213; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sgd_momentum_step(np.full(8, 1000, dtype=np.int64), np.zeros(8, dtype=np.int64), np.full(8, 1000, dtype=np.int64), 65536, 65536, 65536,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; full: the value named 'full' in this block; dtype: the value named 'dtype' in this block. Context: the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity.

Line 207: `np.full(8, 1000, dtype=np.int64),` This is continuation line 2 of the 8-line logical statement that begins at source line 206. Its exact fragment is 'np.full(8, 1000, dtype=np.int64),'. The important operands are full: the value named 'full' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'sgd_momentum_step(np.full(8, 1000, dtype=np.int64), np.zeros(8, dtype=np.int64), np.full(8, 1000, dtype=np.int64), 65536, 65536, 65536,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; full: the value named 'full' in this block; dtype: the value named 'dtype' in this block. Context: the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity.

Line 208: `np.zeros(8, dtype=np.int64),` This is continuation line 3 of the 8-line logical statement that begins at source line 206. Its exact fragment is 'np.zeros(8, dtype=np.int64),'. The important operands are zeros: the value named 'zeros' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'sgd_momentum_step(np.full(8, 1000, dtype=np.int64), np.zeros(8, dtype=np.int64), np.full(8, 1000, dtype=np.int64), 65536, 65536, 65536,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; full: the value named 'full' in this block; dtype: the value named 'dtype' in this block. Context: the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity.

Line 209: `np.full(8, 1000, dtype=np.int64),` This is continuation line 4 of the 8-line logical statement that begins at source line 206. Its exact fragment is 'np.full(8, 1000, dtype=np.int64),'. The important operands are full: the value named 'full' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'sgd_momentum_step(np.full(8, 1000, dtype=np.int64), np.zeros(8, dtype=np.int64), np.full(8, 1000, dtype=np.int64), 65536, 65536, 65536,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; full: the value named 'full' in this block; dtype: the value named 'dtype' in this block. Context: the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity.

Line 210: `65536,` This is continuation line 5 of the 8-line logical statement that begins at source line 206. Its exact fragment is '65536,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(np.full(8, 1000, dtype=np.int64), np.zeros(8, dtype=np.int64), np.full(8, 1000, dtype=np.int64), 65536, 65536, 65536,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; full: the value named 'full' in this block; dtype: the value named 'dtype' in this block. Context: the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity.

Line 211: `65536,` This is continuation line 6 of the 8-line logical statement that begins at source line 206. Its exact fragment is '65536,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(np.full(8, 1000, dtype=np.int64), np.zeros(8, dtype=np.int64), np.full(8, 1000, dtype=np.int64), 65536, 65536, 65536,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; full: the value named 'full' in this block; dtype: the value named 'dtype' in this block. Context: the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity.

Line 212: `65536,` This is continuation line 7 of the 8-line logical statement that begins at source line 206. Its exact fragment is '65536,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(np.full(8, 1000, dtype=np.int64), np.zeros(8, dtype=np.int64), np.full(8, 1000, dtype=np.int64), 65536, 65536, 65536,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; full: the value named 'full' in this block; dtype: the value named 'dtype' in this block. Context: the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity.

Line 213: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 206. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(np.full(8, 1000, dtype=np.int64), np.zeros(8, dtype=np.int64), np.full(8, 1000, dtype=np.int64), 65536, 65536, 65536,)' to 'next_weight, next_velocity ' in host memory. Here next_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; full: the value named 'full' in this block; dtype: the value named 'dtype' in this block. Context: the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity.

Line 214: `np.testing.assert_array_equal(next_velocity, np.full(8, 1000))` This calls 'np.testing.assert_array_equal' with 'next_velocity, np.full(8, 1000)' inside the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 215: `np.testing.assert_array_equal(next_weight, np.full(8, -1000))` This calls 'np.testing.assert_array_equal' with 'next_weight, np.full(8, -1000)' inside the function test_sgd_weight_decay_is_decoupled_from_velocity, which handles test sgd weight decay is decoupled from velocity. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 217: `def test_vector_alu_operations(self) -> None:` This starts function test_vector_alu_operations. Its indented body performs test vector alu operations as part of the file that checks neuraedge golden behavior independently with Python assertions; it runs only when called.

Line 218: `a = np.array([-(1 << 31), -100, -8, -1, 0, 7, 100, (1 << 31) - 1])` This assigns 'np.array([-(1 << 31), -100, -8, -1, 0, 7, 100, (1 << 31) - 1])' to 'a ' in host memory. Here a means the value named 'a' in this block. The important operands are array: the value named 'array' in this block. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 219: `b = np.array([-1, 300, 8, -2, 1, 9, -300, 1])` This assigns 'np.array([-1, 300, 8, -2, 1, 9, -300, 1])' to 'b ' in host memory. Here b means the value named 'b' in this block. The important operands are array: the value named 'array' in this block. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 220: `added = vector_alu(VectorOperation.ADD_SAT, a, b)` This assigns 'vector_alu(VectorOperation.ADD_SAT, a, b)' to 'added ' in host memory. Here added means the value named 'added' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named 'add sat' in this block. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 221: `self.assertEqual(int(added[0]), -(1 << 31))` This calls 'self.assertEqual' with 'int(added[0]), -(1 << 31)' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 222: `self.assertEqual(int(added[-1]), (1 << 31) - 1)` This calls 'self.assertEqual' with 'int(added[-1]), (1 << 31) - 1' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 223: `np.testing.assert_array_equal(` This statement continues through source line 226; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.TERNARY_SPIKE, a, b, threshold=8), np.array([-1, -1, -1, 0, 0, 0, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 224: `vector_alu(VectorOperation.TERNARY_SPIKE, a, b, threshold=8),` This is continuation line 2 of the 4-line logical statement that begins at source line 223. Its exact fragment is 'vector_alu(VectorOperation.TERNARY_SPIKE, a, b, threshold=8),'. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; TERNARY_SPIKE: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.TERNARY_SPIKE, a, b, threshold=8), np.array([-1, -1, -1, 0, 0, 0, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 225: `np.array([-1, -1, -1, 0, 0, 0, 1, 1]),` This is continuation line 3 of the 4-line logical statement that begins at source line 223. Its exact fragment is 'np.array([-1, -1, -1, 0, 0, 0, 1, 1]),'. The important operands are array: the value named 'array' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.TERNARY_SPIKE, a, b, threshold=8), np.array([-1, -1, -1, 0, 0, 0, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 226: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 223. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.TERNARY_SPIKE, a, b, threshold=8), np.array([-1, -1, -1, 0, 0, 0, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 227: `requantized = vector_alu(` This statement continues through source line 236; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum=127,)‘ to ‘requantized ‘ in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 228: `VectorOperation.REQUANTIZE,` This is continuation line 2 of the 10-line logical statement that begins at source line 227. Its exact fragment is ‘VectorOperation.REQUANTIZE,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum=127,)‘ to ‘requantized ‘ in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 229: `a,` This is continuation line 3 of the 10-line logical statement that begins at source line 227. Its exact fragment is ‘a,‘. The important operands are a: the value named ’a’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum=127,)‘ to ‘requantized ‘ in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 230: `b,` This is continuation line 4 of the 10-line logical statement that begins at source line 227. Its exact fragment is ‘b,‘. The important operands are b: the value named ’b’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum=127,)‘ to ‘requantized ‘ in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 231: `multiplier_q15=16384,` This is continuation line 5 of the 10-line logical statement that begins at source line 227. Its exact fragment is ‘multiplier_q15=16384,‘. The important operands are multiplier_q15: the value named ’multiplier q15’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum)‘ to ‘requantized ‘ in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 232: `right_shift=1,` This is continuation line 6 of the 10-line logical statement that begins at source line 227. Its exact fragment is ‘right_shift=1,‘. The important operands are right_shift: the value named ’right shift’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum=127,)‘ to ‘requantized ‘ in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 233: `output_zero_point=3,` This is continuation line 7 of the 10-line logical statement that begins at source line 227. Its exact fragment is 'output_zero_point=3,'. The important operands are output_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns 'vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum=127,)' to 'requantized ' in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 234: `clamp_minimum=-128,` This is continuation line 8 of the 10-line logical statement that begins at source line 227. Its exact fragment is 'clamp_minimum=-128,'. The important operands are clamp_minimum: the value named 'clamp minimum' in this block. Read together, the complete operation is: This assigns 'vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum=127,)' to 'requantized ' in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 235: `clamp_maximum=127,` This is continuation line 9 of the 10-line logical statement that begins at source line 227. Its exact fragment is 'clamp_maximum=127,'. The important operands are clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This assigns 'vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum=127,)' to 'requantized ' in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 236: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 227. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'vector_alu(VectorOperation.REQUANTIZE, a, b, multiplier_q15=16384, right_shift=1, output_zero_point=3, clamp_minimum=-128, clamp_maximum=127,)' to 'requantized ' in host memory. Here requantized means the conversion of a wide accumulated result back into a bounded activation code. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 237: `self.assertTrue(np.all(requantized >= -128))` This calls 'self.assertTrue' with 'np.all(requantized >= -128)' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 238: `self.assertTrue(np.all(requantized <= 127))` This calls 'self.assertTrue' with 'np.all(requantized <= 127)' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 240: `multiplier, shift, represented = quantize_flexible_scale(51.03228)` This assigns 'quantize_flexible_scale(to 'multiplier, shift, represented ' in host memory. Here multiplier means the value named 'multiplier' in this block. The important operands are quantize_flexible_scale: the real-value spacing represented by one quantized integer step. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 241: `self.assertGreater(multiplier, 0)` This calls 'self.assertGreater' with 'multiplier, 0' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 242: `self.assertLess(shift, 0)` This calls ‘self.assertLess‘ with ‘shift, 0‘ inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 243: `self.assertAlmostEqual(represented, 51.03228, delta=0.002)` This calls ‘self.assertAlmostEqual‘ with ‘represented, 51.03228, delta=0.002‘ inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 244: `flexible = vector_alu(` This statement continues through source line 250; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘vector_alu(VectorOperation.FLEX_REQUANTIZE, np.array([1, -1, 100, -100], dtype=np.int64), np.zeros(4, dtype=np.int64), multiplier_q15=multiplier, right_shift=shift,)‘ to ‘flexible ‘ in host memory. Here flexible means the value named ’flexible’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 245: `VectorOperation.FLEX_REQUANTIZE,` This is continuation line 2 of the 7-line logical statement that begins at source line 244. Its exact fragment is ‘VectorOperation.FLEX_REQUANTIZE,‘. The important operands are VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation.FLEX_REQUANTIZE, np.array([1, -1, 100, -100], dtype=np.int64), np.zeros(4, dtype=np.int64), multiplier_q15=multiplier, right_shift=shift,)‘ to ‘flexible ‘ in host memory. Here flexible means the value named ’flexible’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 246: `np.array([1, -1, 100, -100], dtype=np.int64),` This is continuation line 3 of the 7-line logical statement that begins at source line 244. Its exact fragment is ‘np.array([1, -1, 100, -100], dtype=np.int64),‘. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation.FLEX_REQUANTIZE, np.array([1, -1, 100, -100], dtype=np.int64), np.zeros(4, dtype=np.int64), multiplier_q15=multiplier, right_shift=shift,)‘ to ‘flexible ‘ in host memory. Here flexible means the value named ’flexible’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 247: `np.zeros(4, dtype=np.int64),` This is continuation line 4 of the 7-line logical statement that begins at source line 244. Its exact fragment is ‘np.zeros(4, dtype=np.int64),‘. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation.FLEX_REQUANTIZE, np.array([1, -1, 100, -100], dtype=np.int64), np.zeros(4, dtype=np.int64), multiplier_q15=multiplier, right_shift=shift,)‘ to ‘flexible ‘ in host memory. Here flexible means the value named ’flexible’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 248: `multiplier_q15=multiplier,` This is continuation line 5 of the 7-line logical statement that begins at source line 244. Its exact fragment is ‘multiplier_q15=multiplier,‘. The important operands are multiplier_q15: the value named ’multiplier q15’ in this block; multiplier: the value named ’multiplier’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation.FLEX_REQUANTIZE, np.array([1, -1, 100, -100], dtype=np.int64), np.zeros(4, dtype=np.int64), multiplier_q15=multiplier, right_shift=shift,)‘ to ‘flexible ‘ in host memory. Here flexible means the value named ’flexible’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; FLEX_REQUANTIZE: the conversion

of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 249: `right_shift=shift,` This is continuation line 6 of the 7-line logical statement that begins at source line 244. Its exact fragment is 'right_shift=shift,'. The important operands are right_shift: the value named 'right shift' in this block; shift: the value named 'shift' in this block. Read together, the complete operation is: This assigns 'vector_alu(VectorOperation.FLEX_REQUANTIZE, np.array([1, -1, 100, -100], dtype=np.int64), np.zeros(4, dtype=np.int64), multiplier_q15=multiplier, right_shift=shift,)' to 'flexible ' in host memory. Here flexible means the value named 'flexible' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 250: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 244. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'vector_alu(VectorOperation.FLEX_REQUANTIZE, np.array([1, -1, 100, -100], dtype=np.int64), np.zeros(4, dtype=np.int64), multiplier_q15=multiplier, right_shift=shift,)' to 'flexible ' in host memory. Here flexible means the value named 'flexible' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; FLEX_REQUANTIZE: the conversion of a wide accumulated result back into a bounded activation code. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 251: `np.testing.assert_allclose(` This statement continues through source line 255; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_allclose' with ' flexible, np.array([51, -51, 5103, -5103], dtype=np.int64), atol=1, ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 252: `flexible,` This is continuation line 2 of the 5-line logical statement that begins at source line 251. Its exact fragment is 'flexible,'. The important operands are flexible: the value named 'flexible' in this block. Read together, the complete operation is: This calls 'np.testing.assert_allclose' with ' flexible, np.array([51, -51, 5103, -5103], dtype=np.int64), atol=1, ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 253: `np.array([51, -51, 5103, -5103], dtype=np.int64),` This is continuation line 3 of the 5-line logical statement that begins at source line 251. Its exact fragment is 'np.array([51, -51, 5103, -5103], dtype=np.int64),'. The important operands are array: the value named 'array' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This calls 'np.testing.assert_allclose' with ' flexible, np.array([51, -51, 5103, -5103], dtype=np.int64), atol=1, ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 254: `atol=1,` This is continuation line 4 of the 5-line logical statement that begins at source line 251. Its exact fragment is 'atol=1,'. The important operands are atol: the value named 'atol' in this block. Read together, the complete operation is: This calls 'np.testing.assert_allclose' with ' flexible, np.array([51, -51, 5103, -5103], dtype=np.int64), atol=1, ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 255: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 251. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_allclose' with ' flexible, np.array([51, -51, 5103, -5103], dtype=np.int64), atol=1, ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 256: `selector = np.array([0, 1, -1, 0], dtype=np.int64)` This assigns ‘np.array([0, 1, -1, 0], dtype=np.int64)‘ to ‘selector ‘ in host memory. Here selector means the value named ’selector’ in this block. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 257: `payload = np.array([10, 20, 30, 40], dtype=np.int64)` This assigns ‘np.array([10, 20, 30, 40], dtype=np.int64)‘ to ‘payload ‘ in host memory. Here payload means stream data belonging to the current command. The important operands are array: the value named ’array’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_vector_alu_operations, which handles test vector alu operations.

Line 258: `np.testing.assert_array_equal(` This statement continues through source line 261; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.testing.assert_array_equal‘ with ‘ vector_alu(VectorOperation.SELECT_ZERO, selector, payload), np.array([10, 0, 0, 40]), ‘ inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 259: `vector_alu(VectorOperation.SELECT_ZERO, selector, payload),` This is continuation line 2 of the 4-line logical statement that begins at source line 258. Its exact fragment is ‘vector_alu(VectorOperation.SELECT_ZERO, selector, payload),‘. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; SELECT_ZERO: the value named ’select zero’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ vector_alu(VectorOperation.SELECT_ZERO, selector, payload), np.array([10, 0, 0, 40]), ‘ inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 260: `np.array([10, 0, 0, 40]),` This is continuation line 3 of the 4-line logical statement that begins at source line 258. Its exact fragment is ‘np.array([10, 0, 0, 40]),‘. The important operands are array: the value named ’array’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ vector_alu(VectorOperation.SELECT_ZERO, selector, payload), np.array([10, 0, 0, 40]), ‘ inside the function test_vector_alu_operati which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 261: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 258. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ vector_alu(VectorOperation.SELECT_ZERO, selector, payload), np.array([10, 0, 0, 40]), ‘ inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 262: `np.testing.assert_array_equal(` This statement continues through source line 265; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.testing.assert_array_equal‘ with ‘ vector_alu(VectorOperation.SELECT_NONZERO, selector, payload), np.array([0, 20, 30, 0]), ‘ inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 263: `vector_alu(VectorOperation.SELECT_NONZERO, selector, payload),` This is continuation line 2 of the 4-line logical statement that begins at source line 262. Its exact fragment is ‘vector_alu(VectorOperation.SELECT_NONZERO, selector, payload),‘. The important operands are vector_alu: the value named ’vector alu’ in this block; VectorOperation: the value named ’vector operation’ in this block; SELECT_NONZERO: the value named ’select nonzero’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ vector_alu(VectorOperation.SELECT_N selector, payload), np.array([0, 20, 30, 0]), ‘ inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 264: `np.array([0, 20, 30, 0]),` This is continuation line 3 of the 4-line logical statement that begins at source line 262. Its exact fragment is 'np.array([0, 20, 30, 0]),'. The important operands are array: the value named 'array' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.SELECT_NONZERO, selector, payload), np.array([0, 20, 30, 0]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 265: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 262. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.SELECT_NONZERO, selector, payload), np.array([0, 20, 30, 0]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 266: `np.testing.assert_array_equal(` This statement continues through source line 273; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.COMPARE_GE, np.array([1, 2, 2, -1]), np.array([2, 2, 1, -1]),), np.array([0, 1, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 267: `vector_alu(` This is continuation line 2 of the 8-line logical statement that begins at source line 266. Its exact fragment is 'vector_alu('. The important operands are vector_alu: the value named 'vector alu' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.COMPARE_GE, np.array([1, 2, 2, -1]), np.array([2, 2, 1, -1]),), np.array([0, 1, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 268: `VectorOperation.COMPARE_GE,` This is continuation line 3 of the 8-line logical statement that begins at source line 266. Its exact fragment is 'VectorOperation.COMPARE_GE,'. The important operands are VectorOperation: the value named 'vector operation' in this block; COMPARE_GE: the value named 'compare ge' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.COMPARE_GE, np.array([1, 2, 2, -1]), np.array([2, 2, 1, -1]),), np.array([0, 1, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 269: `np.array([1, 2, 2, -1]),` This is continuation line 4 of the 8-line logical statement that begins at source line 266. Its exact fragment is 'np.array([1, 2, 2, -1]),'. The important operands are array: the value named 'array' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.COMPARE_GE, np.array([1, 2, 2, -1]), np.array([2, 2, 1, -1]),), np.array([0, 1, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 270: `np.array([2, 2, 1, -1]),` This is continuation line 5 of the 8-line logical statement that begins at source line 266. Its exact fragment is 'np.array([2, 2, 1, -1]),'. The important operands are array: the value named 'array' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.COMPARE_GE, np.array([1, 2, 2, -1]), np.array([2, 2, 1, -1]),), np.array([0, 1, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 271: `),` This is continuation line 6 of the 8-line logical statement that begins at source line 266. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' vector_alu(VectorOperation.COMPARE_GE, np.array([1, 2, 2, -1]), np.array([2, 2, 1, -1]),), np.array([0, 1, 1, 1]), ' inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 272: `np.array([0, 1, 1, 1]),` This is continuation line 7 of the 8-line logical statement that begins at source line 266. Its exact fragment is ‘np.array([0, 1, 1, 1]),‘. The important operands are array: the value named ’array’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ vector_alu(VectorOperation.COMPARE_GE, np.array([1, 2, 2, -1]), np.array([2, 2, 1, -1]),), np.array([0, 1, 1, 1]), ‘ inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 273: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 266. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ vector_alu(VectorOperation.COMPARE_GE, np.array([1, 2, 2, -1]), np.array([2, 2, 1, -1]),), np.array([0, 1, 1, 1]), ‘ inside the function test_vector_alu_operations, which handles test vector alu operations. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

Line 276: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks neuraedge golden behavior independently with Python assertions.

Line 277: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks neuraedge golden behavior independently with Python assertions. The call advances the host-side job that checks neuraedge golden behavior independently with Python assertions.

66 tests/test_neuraedge_protocol.py

Role in the complete system: checks neuraedge protocol behavior independently with Python assertions.

66.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks neuraedge protocol behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: ProtocolTests. Functions: test_command_words_round_trip, test_tensor_command_fields_and_li test_non_tensor_command_fields, test_tensor_stream_layouts, test_descriptor_program_layout, test_operator_stream_ test_result_decoders. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

66.2 Complete source

```
import unittest

from neuraedge_protocol import (
    Command128,
    ProgramRecord,
    StreamBeat,
    LinearGradientMode,
    LutSuboperation,
    Opcode,
    alpha_xnor_command,
    clear_status_command,
    layernorm_command,
    linear_backward_command,
    lif_command,
    log_softmax_command,
    lut_command,
    pack_activation_beats,
    pack_alpha_beats,
    pack_dot8_weight_beats,
    pack_layernorm_beats,
    pack_linear_backward_beats,
    pack_lif_beats,
    pack_lut_eval_beat,
    pack_lut_load_beats,
    pack_optimizer_beats,
    pack_outer_weight_beats,
    pack_softmax_beats,
    pack_vector_beats,
    pack_descriptor_program,
    sgd_command,
    tensor_command,
    unpack_alpha_result,
    unpack_layernorm_result,
    unpack_linear_backward_result,
    unpack_lif_result,
    unpack_lif_trace,
    unpack_lut_result,
    unpack_optimizer_result,
    unpack_softmax_result,
    unpack_tensor_result,
    unpack_vector_result,
    vector_command,
)


class ProtocolTests(unittest.TestCase):
    def test_command_words_round_trip(self):
        command = Command128(0xFEDCBA98765432100123456789ABCDEF)
        self.assertEqual(command.words_le, (0x89ABCDEF, 0x01234567, 0x76543210, 0xFEDCBA98))
        self.assertEqual(Command128.from_words_le(command.words_le), command)
        self.assertEqual(clear_status_command().opcode, Opcode.CLEAR_STATUS)

```

```
    def test_tensor_command_fields_and_limits(self):
        command = tensor_command(
            Opcode.COMPUTE_DENSE_OUTER,
            rows=8,
            columns=16,
            issue_count=2048,
            activation_zero_point=128,
            weight_zero_point=129,
        ).value
        self.assertEqual(command & 0xF, Opcode.COMPUTE_DENSE_OUTER)
        self.assertEqual((command >> 4) & 0xF, 8)
        self.assertEqual((command >> 8) & 0x3F, 16)
        self.assertEqual((command >> 14) & 0xFFF, 2048)
        self.assertEqual((command >> 26) & 0xFF, 128)
        self.assertEqual((command >> 34) & 0xFF, 129)
        with self.assertRaises(ValueError):
            tensor_command(Opcode.COMPUTE_DENSE_DOT8, rows=2, columns=16, issue_count=1)
        with self.assertRaises(ValueError):
            tensor_command(Opcode.LOAD_WEIGHTS_DOT8, issue_count=257)

    def test_non_tensor_command_fields(self):
        alpha = alpha_xnor_command(256, 127).value
        self.assertEqual((alpha >> 42) & 0x1FF, 256)
        self.assertEqual((alpha >> 51) & 0x1FF, 127)
        vector = vector_command(8, multiplier_q15=-3, right_shift=31,
                                output_zero_point=-8, threshold=-123,
                                surrogate_window=456).value
        self.assertEqual((vector >> 4) & 0xF, 8)
        self.assertEqual((vector >> 8) & 0xFFFF, 0xFFFD)
        self.assertEqual((vector >> 24) & 0x1F, 31)
        self.assertEqual((vector >> 29) & 0xFFFF, 0xFFF8)
        self.assertEqual((vector >> 45) & 0xFFFFFFFF, (-123) & 0xFFFFFFFF)
        self.assertEqual((vector >> 77) & 0xFFFFFFFF, 456)
        flexible = vector_command(9, multiplier_q15=24576, right_shift=-6).value
        self.assertEqual((flexible >> 4) & 0xF, 9)
        self.assertEqual((flexible >> 24) & 0x1F, (-6) & 0x1F)
        self.assertEqual((layernorm_command(2048, 17).value >> 4) & 0xFFF, 2048)
        self.assertEqual((log_softmax_command(256, 15).value >> 4) & 0x1FF, 256)
        lif = lif_command(4, -2, -3, clear_state=True, trace_state=True).value
        self.assertEqual((lif >> 57) & 1, 1)
        self.assertEqual((lif >> 58) & 1, 1)
        optimizer = sgd_command(
            131071, 65536, 32768, update_right_shift=29).value
        self.assertEqual((optimizer >> 4) & 0x3FFFF, 131071)
        self.assertEqual((optimizer >> 22) & 0x1FFFF, 65536)
        self.assertEqual((optimizer >> 39) & 0x1FFFF, 32768)
        self.assertEqual((optimizer >> 56) & 0x1F, 29)
        self.assertEqual(lut_command(LutSuboperation.EVALUATE, 8).opcode, Opcode.ACTIVATION_LUT)

        backward = linear_backward_command(
            8,
            7,
            6,
            input_gradient_shift=31,
            weight_gradient_shift=17,
            mode=LinearGradientMode.WEIGHT,
            accumulate=True,
            suppress_output=True,
        ).value
        self.assertEqual(backward & 0xF, Opcode.TRAIN_LINEAR_BACKWARD)
        self.assertEqual((backward >> 4) & 0xF, 8)
        self.assertEqual((backward >> 8) & 0xF, 7)
        self.assertEqual((backward >> 12) & 0xF, 6)
        self.assertEqual((backward >> 16) & 0x1F, 31)
        self.assertEqual((backward >> 21) & 0x1F, 17)
        self.assertEqual((backward >> 26) & 0x3, LinearGradientMode.WEIGHT)
        self.assertEqual((backward >> 28) & 1, 1)
        self.assertEqual((backward >> 29) & 1, 1)
        with self.assertRaises(ValueError):
            linear_backward_command(0, 1, 1)
        with self.assertRaises(ValueError):
            linear_backward_command(1, 9, 1)

    def test_tensor_stream_layouts(self):
        activations = pack_activation_beats([[0, 1, 2, 3, 4, 5, 6, 255]])
        self.assertEqual(activations[0].data, 0xFF06050403020100)
        self.assertTrue(activations[0].last)
        outer = pack_outer_weight_beats([list(range(16))])
        self.assertEqual([beat.data for beat in outer], [0x0706050403020100, 0x0F0E0D0C0B0A0908])
        self.assertEqual([beat.last for beat in outer], [False, True])
        matrix = [[16 * row + column for column in range(16)] for row in range(8)]
        dot8 = pack_dot8_weight_beats([matrix])
        self.assertEqual(len(dot8), 16)
        self.assertEqual(dot8[0].data, 0x0706050403020100)
        self.assertEqual(dot8[1].data, 0x0F0E0D0C0B0A0908)
        self.assertEqual(dot8[2].data, 0x1716151413121110)
        self.assertTrue(dot8[-1].last)
```

```
    def test_descriptor_program_layout(self):
        records = (
            ProgramRecord(
                clear_status_command(),
                (StreamBeat(0xAA, False), StreamBeat(0xBB, True)),
                0,
            ),
            ProgramRecord(clear_status_command(), (), 3),
        )
        beats, outputs = pack_descriptor_program(records)
        self.assertEqual(outputs, 3)
        self.assertEqual(len(beats), 8)
        self.assertEqual(beats[0].data, 2)
        self.assertEqual(beats[5].data, (1 << 63) | (3 << 32))
        self.assertEqual([beat.last for beat in beats], [False] * 7 + [True])

    def test_operator_stream_layouts(self):
        query = [index & 1 for index in range(256)]
        key = [1 - bit for bit in query]
        alpha = pack_alpha_beats(query, key)
        self.assertEqual(len(alpha), 8)
        self.assertEqual(alpha[0].data, 0xAAAAAAAAAAAAAAAA)
        self.assertEqual(alpha[4].data, 0x5555555555555555)
        self.assertTrue(alpha[-1].last)

        vector = pack_vector_beats(range(-4, 4), range(10, 18), clamp_minimum=-7, clamp_maximum=8)
        self.assertEqual(len(vector), 9)
        self.assertEqual(vector[0].data, 0x00000008FFFFFFF9)
        self.assertEqual(vector[1].data, 0xFFFFFFFDFFFFFFFC)
        self.assertTrue(vector[-1].last)

        layernorm = pack_layernorm_beats([-1], [-2], [-3])
        self.assertEqual(layernorm[0].data, 0xFFFDFFFFFEFFFFFF)
        self.assertEqual(layernorm[1].data, 0xFF)
        self.assertEqual([beat.last for beat in layernorm], [False, True])
        self.assertEqual(pack_softmax_beats([-1, 2])[0].data, 0xFFFFFF)

        lif = pack_lif_beats(range(-16, 16))
        self.assertEqual(len(lif), 8)
        self.assertEqual(lif[0].data, 0xFFF3FFF2FFF1FFF0)
        optimizer = pack_optimizer_beats(range(-4, 4), range(-8, 0), range(8))
        self.assertEqual(len(optimizer), 8)
        self.assertEqual(optimizer[0].data, 0x000000FFFFF8FFFC)
        self.assertTrue(optimizer[-1].last)

        load = pack_lut_load_beats([(7, -2), (8, 3)])
        self.assertEqual(load[0].data, 0x00FFFE07)
        self.assertEqual([beat.last for beat in load], [False, True])
        self.assertEqual(pack_lut_eval_beat(range(8))[0].data, 0x0706050403020100)

        backward = pack_linear_backward_beats(
            [[-128, -1, 0, 1], [127, 2, -3, 4]],
            [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]],
            [[-32768, -1, 32767], [10, 11, 12]],
        )
        self.assertEqual(len(backward), 16)
        self.assertEqual(backward[0].data, 0x00010000FFFFFF80)
        self.assertEqual(backward[1].data, 0)
        self.assertEqual(backward[2].data, 0x0004FFFD0002007F)
        self.assertEqual(backward[4].data, 0x00007FFFFFFF8000)
        self.assertEqual(backward[5].data, 0)
        self.assertEqual([beat.last for beat in backward], [False] * 15 + [True])
        input_only = pack_linear_backward_beats(
            [[-128, -1, 0, 1], [127, 2, -3, 4]],
            [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]],
            [[-32768, -1, 32767], [10, 11, 12]],
            mode=LinearGradientMode.INPUT,
        )
        weight_only = pack_linear_backward_beats(
            [[-128, -1, 0, 1], [127, 2, -3, 4]],
            [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]],
            [[-32768, -1, 32767], [10, 11, 12]],
            mode=LinearGradientMode.WEIGHT,
        )
        self.assertEqual(len(input_only), 12)
        self.assertEqual(input_only[0].data, backward[4].data)
        self.assertEqual(len(weight_only), 8)
        self.assertEqual(weight_only[0].data, backward[0].data)

    def test_result_decoders(self):
        tensor_word = ((-7) & ((1 << 48) - 1)) | (5 << 48) | (12 << 52) | (1 << 58)
        self.assertEqual(unpack_tensor_result(tensor_word), (-7, 5, 12, True))
        self.assertEqual(unpack_alpha_result(11 | (12 << 18) | (13 << 27)), (11, 12, 13))
        self.assertEqual(unpack_vector_result([0xFFFFFFFF00000001, 0, 0, 0])[:2], (1, -1))
        self.assertEqual(unpack_layernorm_result(0xFFFFFFFF | (9 << 32) | (1 << 44)), (-1, 9, True))
        self.assertEqual(unpack_softmax_result(123 | (7 << 16) | (1 << 25)), (123, 7, True))
```

```
        self.assertEqual(unpack_lif_result(0x12345678ABCDEF01), (0xABCDEF01, 0x12345678))
        expected_membranes = tuple(
            (-1 if lane % 3 == 0 else (lane * 7919) % (1 << 23))
            for lane in range(32)
        )
        packed_membranes = sum(
            (value & 0xFFFFFF) << (24 * lane)
            for lane, value in enumerate(expected_membranes)
        )
        trace_words = [0x12345678ABCDEF01]
        trace_words.extend(
            (packed_membranes >> (64 * index)) & 0xFFFFFFFFFFFFFFFF
            for index in range(12)
        )
        self.assertEqual(
            unpack_lif_trace(trace_words),
            (0xABCDEF01, 0x12345678, expected_membranes),
        )
        with self.assertRaises(ValueError):
            unpack_lif_trace(trace_words[:-1])
        weights, velocities = unpack_optimizer_result([0xFFFFFF000001FFFF] * 8)
        self.assertEqual(weights[0], -1)
        self.assertEqual(velocities[0], 1)
        self.assertEqual(unpack_lut_result([0x0003FFFE0001FFFF, 0]), (-1, 1, -2, 3, 0, 0, 0, 0))

        expected_dx = ((-1, 2, -(1 << 31), (1 << 31) - 1), (9, -10, 11, -12))
        expected_dw = (
            (-1, 2, -3),
            ((1 << 23) - 1, -(1 << 23), 7),
            (8, 9, 10),
            (-11, -12, -13),
        )
        backward_words = []
        for row in expected_dx:
            padded = row + (0,) * (8 - len(row))
            backward_words.extend(
                (padded[lane] & 0xFFFFFFFF)
                | ((padded[lane + 1] & 0xFFFFFFFF) << 32)
                for lane in range(0, 8, 2)
            )
        for row in expected_dw:
            padded = row + (0,) * (8 - len(row))
            packed = sum((value & 0xFFFFFF) << (24 * lane)
                         for lane, value in enumerate(padded))
            backward_words.extend(
                (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3)
            )
        self.assertEqual(
            unpack_linear_backward_result(backward_words, 2, 4, 3),
            (expected_dx, expected_dw),
        )
        self.assertEqual(
            unpack_linear_backward_result(
                backward_words[:8], 2, 4, 3, mode=LinearGradientMode.INPUT
            ),
            (expected_dx, ()),
        )
        self.assertEqual(
            unpack_linear_backward_result(
                backward_words[8:], 2, 4, 3, mode=LinearGradientMode.WEIGHT
            ),
            ((), expected_dw),
        )
        with self.assertRaises(ValueError):
            unpack_linear_backward_result(backward_words[:-1], 2, 4, 3)


if __name__ == "__main__":
    unittest.main()
```

66.3 Line annotations

Line 1: `import unittest` This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `from neuraedge_protocol import (` This statement continues through source line 43; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 4: `Command128,` This is continuation line 2 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'Command128,'. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `ProgramRecord,` This is continuation line 3 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'ProgramRecord,'. The important operands are ProgramRecord: the value named 'program record' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `StreamBeat,` This is continuation line 4 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'StreamBeat,'. The important operands are StreamBeat: the value named 'stream beat' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `LinearGradientMode,` This is continuation line 5 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'LinearGradientMode,'. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `LutSuboperation,` This is continuation line 6 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'LutSuboperation,'. The important operands are LutSuboperation: the value named 'lut suboperation' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `Opcode,` This is continuation line 7 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'Opcode,'. The important operands are Opcode: the value named 'opcode' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `alpha_xnor_command,` This is continuation line 8 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'alpha_xnor_command,'. The important operands are alpha_xnor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `clear_status_command,` This is continuation line 9 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'clear_status_command,'. The important operands are clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `layernorm_command,` This is continuation line 10 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'layernorm_command,'. The important operands are layernorm_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `linear_backward_command,` This is continuation line 11 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'linear_backward_command,'. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `lif_command,` This is continuation line 12 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'lif_command,'. The important operands are lif_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `log_softmax_command,` This is continuation line 13 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'log_softmax_command,'. The important operands are log_softmax_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `lut_command,` This is continuation line 14 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'lut_command,'. The important operands are lut_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `pack_activation_beats,` This is continuation line 15 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_activation_beats,'. The important operands are pack_activation_beats: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `pack_alpha_beats,` This is continuation line 16 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_alpha_beats,'. The important operands are pack_alpha_beats: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `pack_dot8_weight_beats,` This is continuation line 17 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_dot8_weight_beats,'. The important operands are pack_dot8_weight_beats: a learned coefficient multiplied by an activation. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `pack_layernorm_beats,` This is continuation line 18 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_layernorm_beats,'. The important operands are pack_layernorm_beats: the value named 'pack layernorm beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `pack_linear_backward_beats,` This is continuation line 19 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_linear_backward_beats,'. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `pack_lif_beats,` This is continuation line 20 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_lif_beats,'. The important operands are pack_lif_beats: the value named 'pack lif beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `pack_lut_eval_beat,` This is continuation line 21 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_lut_eval_beat,'. The important operands are pack_lut_eval_beat: the value named 'pack lut eval beat' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `pack_lut_load_beats,` This is continuation line 22 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_lut_load_beats,'. The important operands are pack_lut_load_beats: the value named 'pack lut load beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 25: `pack_optimizer_beats,` This is continuation line 23 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_optimizer_beats,'. The important operands are pack_optimizer_beats: the value named 'pack optimizer beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 26: `pack_outer_weight_beats,` This is continuation line 24 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_outer_weight_beats,'. The important operands are pack_outer_weight_beats: a learned coefficient multiplied by an activation. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 27: `pack_softmax_beats,` This is continuation line 25 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_softmax_beats,'. The important operands are pack_softmax_beats: the value named 'pack softmax beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 28: `pack_vector_beats,` This is continuation line 26 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_vector_beats,'. The important operands are pack_vector_beats: the value named 'pack vector beats' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 29: `pack_descriptor_program,` This is continuation line 27 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'pack_descriptor_program,'. The important operands are pack_descriptor_program: a DMA record header plus one command and its declared payload. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 30: `sgd_command,` This is continuation line 28 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'sgd_command,'. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 31: `tensor_command,` This is continuation line 29 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'tensor_command,'. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 32: `unpack_alpha_result,` This is continuation line 30 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_alpha_result,'. The important operands are unpack_alpha_result: the programmable weight assigned to matching zero spikes in alpha-XNOR attention. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 33: `unpack_layernorm_result,` This is continuation line 31 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_layernorm_result,'. The important operands are unpack_layernorm_result: the value named 'unpack layernorm result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 34: `unpack_linear_backward_result,` This is continuation line 32 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_linear_backward_result,'. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 35: `unpack_lif_result,` This is continuation line 33 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_lif_result,'. The important operands are unpack_lif_result: the value named 'unpack lif result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 36: `unpack_lif_trace,` This is continuation line 34 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_lif_trace,'. The important operands are unpack_lif_trace: the value named 'unpack lif trace' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 37: `unpack_lut_result,` This is continuation line 35 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_lut_result,'. The important operands are unpack_lut_result: the value named 'unpack lut result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 38: `unpack_optimizer_result,` This is continuation line 36 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_optimizer_result,'. The important operands are unpack_optimizer_result: the value named 'unpack optimizer result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 39: `unpack_softmax_result,` This is continuation line 37 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_softmax_result,'. The important operands are unpack_softmax_result: the value named 'unpack softmax result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 40: `unpack_tensor_result,` This is continuation line 38 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_tensor_result,'. The important operands are unpack_tensor_result: the value named 'unpack tensor result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 41: `unpack_vector_result,` This is continuation line 39 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'unpack_vector_result,'. The important operands are unpack_vector_result: the value named 'unpack vector result' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 42: `vector_command,` This is continuation line 40 of the 41-line logical statement that begins at source line 3. Its exact fragment is 'vector_command,'. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 43: `)` This is continuation line 41 of the 41-line logical statement that begins at source line 3. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge protocol behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 46: `class ProtocolTests(unittest.TestCase):` This defines class ProtocolTests, which groups data and methods for protocol tests. Instances exist in host memory and organize the flow that checks neuraedge protocol behavior independently with Python assertions.

Line 47: `def test_command_words_round_trip(self):` This starts function test_command_words_round_trip. Its indented body performs test command words round trip as part of the file that checks neuraedge protocol behavior independently with Python assertions; it runs only when called.

Line 48: `command = Command128(0xFEDCBA98765432100123456789ABCDEF)` This assigns 'Command128(0xFEDCBA9876543210 to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are Command128: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_command_words_round_trip, which handles test command words round trip.

Line 49: `self.assertEqual(command.words_le, (0x89ABCDEF, 0x01234567, 0x76543210, 0xFEDCBA98))` This calls 'self.assertEqual' with 'command.words_le, (0x89ABCDEF, 0x01234567, 0x76543210, 0xFEDCBA98)' inside the function test_command_words_round_trip, which handles test command words round trip. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 50: `self.assertEqual(Command128.from_words_le(command.words_le), command)` This calls 'self.assertEqual' with 'Command128.from_words_le(command.words_le), command' inside the function test_command_words_round_trip, which handles test command words round trip. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 51: `self.assertEqual(clear_status_command().opcode, Opcode.CLEAR_STATUS)` This calls 'self.assertEqual' with 'clear_status_command().opcode, Opcode.CLEAR_STATUS' inside the function test_command_words_round_trip, which handles test command words round trip. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 53: `def test_tensor_command_fields_and_limits(self):` This starts function test_tensor_command_fields_and_limi Its indented body performs test tensor command fields and limits as part of the file that checks neuraedge protocol behavior independently with Python assertions; it runs only when called.

Line 54: `command = tensor_command(` This statement continues through source line 61; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'tensor_command(Opcode.COMPUTE_DENSE_OUTER, rows=8, columns=16, issue_count=2048, activation_zero_point=128, weight_zero_point=129,).value' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block. Context: the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits.

Line 55: `Opcode.COMPUTE_DENSE_OUTER,` This is continuation line 2 of the 8-line logical statement that begins at source line 54. Its exact fragment is 'Opcode.COMPUTE_DENSE_OUTER,'. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block. Read together, the complete operation is: This assigns 'tensor_command(Opcode.COMPUTE_DENSE_OUTER, rows=8, columns=16, issue_count=2048, activation_zero_point=128, weight_zero_point=129,).value' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block. Context: the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits.

Line 56: `rows=8,` This is continuation line 3 of the 8-line logical statement that begins at source line 54. Its exact fragment is 'rows=8,'. The important operands are rows: one output-row position processed by the tensor tile. Read together, the complete operation is: This assigns 'tensor_command(Opcode.COMPUTE_DENSE_OUTER, rows=8, columns=16, issue_count=2048, activation_zero_point=128, weight_zero_point=129,).value' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block. Context: the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits.

Line 57: `columns=16,` This is continuation line 4 of the 8-line logical statement that begins at source line 54. Its exact fragment is 'columns=16,'. The important operands are columns: one output-feature position processed in parallel. Read together, the complete operation is: This assigns 'tensor_command(Opcode.COMPUTE_DENSE_OUTER, rows=8, columns=16, issue_count=2048, activation_zero_point=128, weight_zero_point=129,).value' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block. Context: the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits.

Line 58: `issue_count=2048,` This is continuation line 5 of the 8-line logical statement that begins at source line 54. Its exact fragment is 'issue_count=2048,'. The important operands are issue_count: an integer count retained for control or measurable telemetry. Read together, the complete operation is: This assigns 'tensor_command(Opcode.COMPUTE_DENSE_OUTER, rows=8, columns=16, issue_count=2048, activation_zero_point=128, weight_zero_point=1).value' to 'command ' in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block. Context: the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits.

Line 59: `activation_zero_point=128,` This is continuation line 6 of the 8-line logical statement that begins at source line 54. Its exact fragment is 'activation_zero_point=128,'. The important operands are activation_zero_point:

an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘tensor_command(Opcode.COMPUTE_DENSE_OUTER, rows=8, columns=16, issue_count=2048, activation_zero_point=128, weight_zero_point=129,).value‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named ’opcode’ in this block; COMPUTE_DENSE_OUTER: the value named ’compute dense outer’ in this block. Context: the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits.

Line 60: `weight_zero_point=129,` This is continuation line 7 of the 8-line logical statement that begins at source line 54. Its exact fragment is ‘weight_zero_point=129,‘. The important operands are weight_zero_point: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘tensor_command(Opcode.COMPUTE_DENSE_OUTER, rows=8, columns=16, issue_count=2048, activation_zero_point=128, weight_zero_point=1).value‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named ’opcode’ in this block; COMPUTE_DENSE_OUTER: the value named ’compute dense outer’ in this block. Context: the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits.

Line 61: `).value` This is continuation line 8 of the 8-line logical statement that begins at source line 54. Its exact fragment is ‘).value‘. The important operands are value: the value named ’value’ in this block. Read together, the complete operation is: This assigns ‘tensor_command(Opcode.COMPUTE_DENSE_OUTER, rows=8, columns=16, issue_count=2048, activation_zero_point=128, weight_zero_point=129,).value‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are tensor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Opcode: the value named ’opcode’ in this block; COMPUTE_DENSE_OUTER: the value named ’compute dense outer’ in this block. Context: the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits.

Line 62: `self.assertEqual(command & 0xF, Opcode.COMPUTE_DENSE_OUTER)` This calls ‘self.assertEqual‘ with ‘command & 0xF, Opcode.COMPUTE_DENSE_OUTER‘ inside the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 63: `self.assertEqual((command >> 4) & 0xF, 8)` This calls ‘self.assertEqual‘ with ‘(command >> 4) & 0xF, 8‘ inside the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 64: `self.assertEqual((command >> 8) & 0x3F, 16)` This calls ‘self.assertEqual‘ with ‘(command >> 8) & 0x3F, 16‘ inside the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 65: `self.assertEqual((command >> 14) & 0xFFF, 2048)` This calls ‘self.assertEqual‘ with ‘(command >> 14) & 0xFFF, 2048‘ inside the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 66: `self.assertEqual((command >> 26) & 0xFF, 128)` This calls ‘self.assertEqual‘ with ‘(command >> 26) & 0xFF, 128‘ inside the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 67: `self.assertEqual((command >> 34) & 0xFF, 129)` This calls ‘self.assertEqual‘ with ‘(command >> 34) & 0xFF, 129‘ inside the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 68: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits raises an error.

Line 69: `tensor_command(Opcode.COMPUTE_DENSE_DOT8, rows=2, columns=16, issue_count=1)` This calls ‘tensor_command‘ with ‘Opcode.COMPUTE_DENSE_DOT8, rows=2, columns=16, issue_count=1‘ inside the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 70: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits raises an error.

Line 71: `tensor_command(Opcode.LOAD_WEIGHTS_DOT8, issue_count=257)` This calls ‘tensor_command‘ with ‘Opcode.LOAD_WEIGHTS_DOT8, issue_count=257‘ inside the function test_tensor_command_fields_and_limits, which handles test tensor command fields and limits. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 73: `def test_non_tensor_command_fields(self):` This starts function test_non_tensor_command_fields. Its indented body performs test non tensor command fields as part of the file that checks neuraedge protocol behavior independently with Python assertions; it runs only when called.

Line 74: `alpha = alpha_xnor_command(256, 127).value` This assigns ‘alpha_xnor_command(256, 127).value‘ to ‘alpha ‘ in host memory. Here alpha means the programmable weight assigned to matching zero spikes in alpha-XNOR attention. The important operands are alpha_xnor_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 75: `self.assertEqual((alpha >> 42) & 0x1FF, 256)` This calls ‘self.assertEqual‘ with ‘(alpha >> 42) & 0x1FF, 256‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 76: `self.assertEqual((alpha >> 51) & 0x1FF, 127)` This calls ‘self.assertEqual‘ with ‘(alpha >> 51) & 0x1FF, 127‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 77: `vector = vector_command(8, multiplier_q15=-3, right_shift=31,` This statement continues through source line 79; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘vector_command(8, multiplier_q15=-3, right_shift=31, output_zero_point=-8, threshold=-123, surrogate_window=456).value‘ to ‘vector ‘ in host memory. Here vector means the value named ’vector’ in this block. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; multiplier_q15: the value named ’multiplier q15’ in this block; right_shift: the value named ’right shift’ in this block. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 78: `output_zero_point=-8, threshold=-123,` This is continuation line 2 of the 3-line logical statement that begins at source line 77. Its exact fragment is ‘output_zero_point=-8, threshold=-123,‘. The important operands are output_zero_point: the integer code representing real numeric zero in affine quantization; threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns ‘vector_command(8, multiplier_q15=-3, right_shift=31, output_zero_point=-8, threshold=-123, surrogate_window=456).value‘ to ‘vector ‘ in host memory. Here vector means the value named ’vector’ in this block. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; multiplier_q15: the value named ’multiplier q15’ in this block; right_shift: the value named ’right shift’ in this block. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 79: `surrogate_window=456).value` This is continuation line 3 of the 3-line logical statement that begins at source line 77. Its exact fragment is 'surrogate_window=456).value'. The important operands are surrogate_window: the value named 'surrogate window' in this block; value: the value named 'value' in this block. Read together, the complete operation is: This assigns 'vector_command(8, multiplier_q15=-3, right_shift=31, output_zero_point=-8, threshold=-123, surrogate_window=456).value' to 'vector ' in host memory. Here vector means the value named 'vector' in this block. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; multiplier_q15: the value named 'multiplier q15' in this block; right_shift: the value named 'right shift' in this block. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 80: `self.assertEqual((vector >> 4) & 0xF, 8)` This calls 'self.assertEqual' with '(vector >> 4) & 0xF, 8' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 81: `self.assertEqual((vector >> 8) & 0xFFFF, 0xFFFD)` This calls 'self.assertEqual' with '(vector >> 8) & 0xFFFF, 0xFFFD' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 82: `self.assertEqual((vector >> 24) & 0x1F, 31)` This calls 'self.assertEqual' with '(vector >> 24) & 0x1F, 31' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 83: `self.assertEqual((vector >> 29) & 0xFFFF, 0xFFF8)` This calls 'self.assertEqual' with '(vector >> 29) & 0xFFFF, 0xFFF8' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 84: `self.assertEqual((vector >> 45) & 0xFFFFFFFF, (-123) & 0xFFFFFFFF)` This calls 'self.assertEqual' with '(vector >> 45) & 0xFFFFFFFF, (-123) & 0xFFFFFFFF' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 85: `self.assertEqual((vector >> 77) & 0xFFFFFFFF, 456)` This calls 'self.assertEqual' with '(vector >> 77) & 0xFFFFFFFF, 456' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 86: `flexible = vector_command(9, multiplier_q15=24576, right_shift=-6).value` This assigns 'vector_command(9 multiplier_q15=24576, right_shift=-6).value' to 'flexible ' in host memory. Here flexible means the value named 'flexible' in this block. The important operands are vector_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; multiplier_q15: the value named 'multiplier q15' in this block; right_shift: the value named 'right shift' in this block. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 87: `self.assertEqual((flexible >> 4) & 0xF, 9)` This calls 'self.assertEqual' with '(flexible >> 4) & 0xF, 9' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 88: `self.assertEqual((flexible >> 24) & 0x1F, (-6) & 0x1F)` This calls 'self.assertEqual' with '(flexible >> 24) & 0x1F, (-6) & 0x1F' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 89: `self.assertEqual((layernorm_command(2048, 17).value >> 4) & 0xFFF, 2048)` This calls 'self.assertEqual' with '(layernorm_command(2048, 17).value >> 4) & 0xFFF, 2048' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 90: `self.assertEqual((log_softmax_command(256, 15).value >> 4) & 0x1FF, 256)` This calls ‘self.assertEqual‘ with ‘(log_softmax_command(256, 15).value >> 4) & 0x1FF, 256‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 91: `lif = lif_command(4, -2, -3, clear_state=True, trace_state=True).value` This assigns ‘lif_command(4, -2, -3, clear_state=True, trace_state=True).value‘ to ‘lif ‘ in host memory. Here lif means the value named ’lif’ in this block. The important operands are lif_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; clear_state: the finite-state-machine phase retained in flip-flops; trace_state: the finite-state-machine phase retained in flip-flops. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 92: `self.assertEqual((lif >> 57) & 1, 1)` This calls ‘self.assertEqual‘ with ‘(lif >> 57) & 1, 1‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 93: `self.assertEqual((lif >> 58) & 1, 1)` This calls ‘self.assertEqual‘ with ‘(lif >> 58) & 1, 1‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 94: `optimizer = sgd_command(` This statement continues through source line 95; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sgd_command(131071, 65536, 32768, update_right_shift=29).value‘ to ‘optimizer ‘ in host memory. Here optimizer means the value named ’optimizer’ in this block. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; update_right_shift: the value named ’update right shift’ in this block; value: the value named ’value’ in this block. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 95: `131071, 65536, 32768, update_right_shift=29).value` This is continuation line 2 of the 2-line logical statement that begins at source line 94. Its exact fragment is ‘131071, 65536, 32768, update_right_shift=29).value‘. The important operands are update_right_shift: the value named ’update right shift’ in this block; value: the value named ’value’ in this block. Read together, the complete operation is: This assigns ‘sgd_command(131071, 65536, 32768, update_right_shift=29).value‘ to ‘optimizer ‘ in host memory. Here optimizer means the value named ’optimizer’ in this block. The important operands are sgd_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; update_right_shift: the value named ’update right shift’ in this block; value: the value named ’value’ in this block. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 96: `self.assertEqual((optimizer >> 4) & 0x3FFFF, 131071)` This calls ‘self.assertEqual‘ with ‘(optimizer >> 4) & 0x3FFFF, 131071‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 97: `self.assertEqual((optimizer >> 22) & 0x1FFFF, 65536)` This calls ‘self.assertEqual‘ with ‘(optimizer >> 22) & 0x1FFFF, 65536‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 98: `self.assertEqual((optimizer >> 39) & 0x1FFFF, 32768)` This calls ‘self.assertEqual‘ with ‘(optimizer >> 39) & 0x1FFFF, 32768‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 99: `self.assertEqual((optimizer >> 56) & 0x1F, 29)` This calls ‘self.assertEqual‘ with ‘(optimizer >> 56) & 0x1F, 29‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 100: `self.assertEqual(lut_command(LutSuboperation.EVALUATE, 8).opcode, Opcode.ACTIVATION_LUT)` This calls ‘self.assertEqual‘ with ‘lut_command(LutSuboperation.EVALUATE, 8).opcode, Opcode.ACTIVATION_LUT‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 102: `backward = linear_backward_command(` This statement continues through source line 111; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value‘ to ‘backward ‘ in host memory. Here backward means the value named ’backward’ in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 103: `8,` This is continuation line 2 of the 10-line logical statement that begins at source line 102. Its exact fragment is ‘8,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value‘ to ‘backward ‘ in host memory. Here backward means the value named ’backward’ in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 104: `7,` This is continuation line 3 of the 10-line logical statement that begins at source line 102. Its exact fragment is ‘7,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value‘ to ‘backward ‘ in host memory. Here backward means the value named ’backward’ in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 105: `6,` This is continuation line 4 of the 10-line logical statement that begins at source line 102. Its exact fragment is ‘6,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value‘ to ‘backward ‘ in host memory. Here backward means the value named ’backward’ in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 106: `input_gradient_shift=31,` This is continuation line 5 of the 10-line logical statement that begins at source line 102. Its exact fragment is ‘input_gradient_shift=31,‘. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value‘ to ‘backward ‘ in host memory. Here backward means the value named ’backward’ in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 107: `weight_gradient_shift=17,` This is continuation line 6 of the 10-line logical statement that begins at source line 102. Its exact fragment is 'weight_gradient_shift=17,'. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value' to 'backward ' in host memory. Here backward means the value named 'backward' in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 108: `mode=LinearGradientMode.WEIGHT,` This is continuation line 7 of the 10-line logical statement that begins at source line 102. Its exact fragment is 'mode=LinearGradientMode.WEIGHT,'. The important operands are mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value' to 'backward ' in host memory. Here backward means the value named 'backward' in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 109: `accumulate=True,` This is continuation line 8 of the 10-line logical statement that begins at source line 102. Its exact fragment is 'accumulate=True,'. The important operands are accumulate: the value named 'accumulate' in this block. Read together, the complete operation is: This assigns 'linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value' to 'backward ' in host memory. Here backward means the value named 'backward' in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 110: `suppress_output=True,` This is continuation line 9 of the 10-line logical statement that begins at source line 102. Its exact fragment is 'suppress_output=True,'. The important operands are suppress_output: the value named 'suppress output' in this block. Read together, the complete operation is: This assigns 'linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value' to 'backward ' in host memory. Here backward means the value named 'backward' in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 111: `).value` This is continuation line 10 of the 10-line logical statement that begins at source line 102. Its exact fragment is ').value'. The important operands are value: the value named 'value' in this block. Read together, the complete operation is: This assigns 'linear_backward_command(8, 7, 6, input_gradient_shift=31, weight_gradient_shift=17, mode=LinearGradientMode.WEIGHT, accumulate=True, suppress_output=True,).value' to 'backward ' in host memory. Here backward means the value named 'backward' in this block. The important operands are linear_backward_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_gradient_shift: the signed loss derivative used to update a learned parameter; weight_gradient_shift: a learned coefficient multiplied by an activation. Context: the function test_non_tensor_command_fields, which handles test non tensor command fields.

Line 112: `self.assertEqual(backward & 0xF, Opcode.TRAIN_LINEAR_BACKWARD)` This calls 'self.assertEqual' with 'backward & 0xF, Opcode.TRAIN_LINEAR_BACKWARD' inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 113: `self.assertEqual((backward >> 4) & 0xF, 8)` This calls ‘self.assertEqual‘ with ‘(backward >> 4) & 0xF, 8‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 114: `self.assertEqual((backward >> 8) & 0xF, 7)` This calls ‘self.assertEqual‘ with ‘(backward >> 8) & 0xF, 7‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 115: `self.assertEqual((backward >> 12) & 0xF, 6)` This calls ‘self.assertEqual‘ with ‘(backward >> 12) & 0xF, 6‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 116: `self.assertEqual((backward >> 16) & 0x1F, 31)` This calls ‘self.assertEqual‘ with ‘(backward >> 16) & 0x1F, 31‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 117: `self.assertEqual((backward >> 21) & 0x1F, 17)` This calls ‘self.assertEqual‘ with ‘(backward >> 21) & 0x1F, 17‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 118: `self.assertEqual((backward >> 26) & 0x3, LinearGradientMode.WEIGHT)` This calls ‘self.assertEqual‘ with ‘(backward >> 26) & 0x3, LinearGradientMode.WEIGHT‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 119: `self.assertEqual((backward >> 28) & 1, 1)` This calls ‘self.assertEqual‘ with ‘(backward >> 28) & 1, 1‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 120: `self.assertEqual((backward >> 29) & 1, 1)` This calls ‘self.assertEqual‘ with ‘(backward >> 29) & 1, 1‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 121: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_non_tensor_command_fields, which handles test non tensor command fields raises an error.

Line 122: `linear_backward_command(0, 1, 1)` This calls ‘linear_backward_command‘ with ‘0, 1, 1‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 123: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_non_tensor_command_fields, which handles test non tensor command fields raises an error.

Line 124: `linear_backward_command(1, 9, 1)` This calls ‘linear_backward_command‘ with ‘1, 9, 1‘ inside the function test_non_tensor_command_fields, which handles test non tensor command fields. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 126: `def test_tensor_stream_layouts(self):` This starts function test_tensor_stream_layouts. Its indented body performs test tensor stream layouts as part of the file that checks neuraedge protocol behavior independently with Python assertions; it runs only when called.

Line 127: `activations = pack_activation_beats([[0, 1, 2, 3, 4, 5, 6, 255]])` This assigns 'pack_activation_beats([[0, 1, 2, 3, 4, 5, 6, 255]])' to 'activations ' in host memory. Here activations means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are pack_activation_beats: an input feature value consumed by a neuron or multiply-accumulate operation. Context: the function test_tensor_stream_layouts, which handles test tensor stream layouts.

Line 128: `self.assertEqual(activations[0].data, 0xFF06050403020100)` This calls 'self.assertEqual' with 'activations[0].data, 0xFF06050403020100' inside the function test_tensor_stream_layouts, which handles test tensor stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 129: `self.assertTrue(activations[0].last)` This calls 'self.assertTrue' with 'activations[0].last' inside the function test_tensor_stream_layouts, which handles test tensor stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 130: `outer = pack_outer_weight_beats([list(range(16))])` This assigns 'pack_outer_weight_beats([list(range(16))])' to 'outer ' in host memory. Here outer means the value named 'outer' in this block. The important operands are pack_outer_weight_beats: a learned coefficient multiplied by an activation; list: the value named 'list' in this block; range: the value named 'range' in this block. Context: the function test_tensor_stream_layouts, which handles test tensor stream layouts.

Line 131: `self.assertEqual([beat.data for beat in outer], [0x0706050403020100, 0x0F0E0D0C0B0A0908])` This calls 'self.assertEqual' with '[beat.data for beat in outer], [0x0706050403020100, 0x0F0E0D0C0B0A0908]' inside the function test_tensor_stream_layouts, which handles test tensor stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 132: `self.assertEqual([beat.last for beat in outer], [False, True])` This calls 'self.assertEqual' with '[beat.last for beat in outer], [False, True]' inside the function test_tensor_stream_layouts, which handles test tensor stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 133: `matrix = [[16 * row + column for column in range(16)] for row in range(8)]` This assigns '[[16 * row + column for column in range(16)] for row in range(8)]' to 'matrix ' in host memory. Here matrix means the value named 'matrix' in this block. The important operands are row: one output-row position processed by the tensor tile; column: one output-feature position processed in parallel; for: the value named 'for' in this block. Context: the function test_tensor_stream_layouts, which handles test tensor stream layouts.

Line 134: `dot8 = pack_dot8_weight_beats([matrix])` This assigns 'pack_dot8_weight_beats([matrix])' to 'dot8 ' in host memory. Here dot8 means the value named 'dot8' in this block. The important operands are pack_dot8_weight_beats: a learned coefficient multiplied by an activation; matrix: the value named 'matrix' in this block. Context: the function test_tensor_stream_layouts, which handles test tensor stream layouts.

Line 135: `self.assertEqual(len(dot8), 16)` This calls 'self.assertEqual' with 'len(dot8), 16' inside the function test_tensor_stream_layouts, which handles test tensor stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 136: `self.assertEqual(dot8[0].data, 0x0706050403020100)` This calls 'self.assertEqual' with 'dot8[0].data, 0x0706050403020100' inside the function test_tensor_stream_layouts, which handles test tensor stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 137: `self.assertEqual(dot8[1].data, 0x0F0E0D0C0B0A0908)` This calls 'self.assertEqual' with 'dot8[1].data, 0x0F0E0D0C0B0A0908' inside the function test_tensor_stream_layouts, which handles test tensor stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 138: `self.assertEqual(dot8[2].data, 0x1716151413121110)` This calls ‘self.assertEqual‘ with ‘dot8[2].data, 0x1716151413121110‘ inside the function test_tensor_stream_layouts, which handles test tensor stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 139: `self.assertTrue(dot8[-1].last)` This calls ‘self.assertTrue‘ with ‘dot8[-1].last‘ inside the function test_tensor_strea which handles test tensor stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 141: `def test_descriptor_program_layout(self):` This starts function test_descriptor_program_layout. Its indented body performs test descriptor program layout as part of the file that checks neuraedge protocol behavior independently with Python assertions; it runs only when called.

Line 142: `records = (` This statement continues through source line 149; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0xAA, False), StreamBeat(0xBB, True)), 0,), ProgramRecord(clear_status_command(), (), 3),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_layout, which handles test descriptor program layout.

Line 143: `ProgramRecord(` This is continuation line 2 of the 8-line logical statement that begins at source line 142. Its exact fragment is ‘ProgramRecord(‘. The important operands are ProgramRecord: the value named ’program record’ in this block. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0xAA, False), StreamBeat(0xBB, True)), 0,), ProgramRecord(clear_status_command(), (), 3),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_layout, which handles test descriptor program layout.

Line 144: `clear_status_command(),` This is continuation line 3 of the 8-line logical statement that begins at source line 142. Its exact fragment is ‘clear_status_command(),‘. The important operands are clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0xAA, False), StreamBeat(0xBB, True)), 0,), ProgramRecord(clear_status_command(), (), 3),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_layout, which handles test descriptor program layout.

Line 145: `(StreamBeat(0xAA, False), StreamBeat(0xBB, True)),` This is continuation line 4 of the 8-line logical statement that begins at source line 142. Its exact fragment is ‘(StreamBeat(0xAA, False), StreamBeat(0xBB, True)),‘. The important operands are StreamBeat: the value named ’stream beat’ in this block. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0xAA, False), StreamBeat(0xBB, True)), 0,), ProgramRecord(clear_status_command(), (), 3),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_layout, which handles test descriptor program layout.

Line 146: `0,` This is continuation line 5 of the 8-line logical statement that begins at source line 142. Its exact fragment is ‘0,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0xAA, False), StreamBeat(0xBB, True)), 0,), ProgramRecord(clear_status_command(), (), 3),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape,

mode, and numeric parameters; StreamBeat: the value named 'stream beat' in this block. Context: the function test_descriptor_program_layout, which handles test descriptor program layout.

Line 147: `),` This is continuation line 6 of the 8-line logical statement that begins at source line 142. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(ProgramRecord(clear_status_command(), (StreamBeat(0xAA, False), StreamBeat(0xBB, True)), 0,), ProgramRecord(clear_status_command(), (), 3),)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named 'stream beat' in this block. Context: the function test_descriptor_program_layout, which handles test descriptor program layout.

Line 148: `ProgramRecord(clear_status_command(), (), 3),` This is continuation line 7 of the 8-line logical statement that begins at source line 142. Its exact fragment is 'ProgramRecord(clear_status_command(), (), 3),'. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns '(ProgramRecord(clear_status_command(), (StreamBeat(0xAA, False), StreamBeat(0xBB, True)), 0,), ProgramRecord(clear_status_command(), (), 3),)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named 'stream beat' in this block. Context: the function test_descriptor_program_layout, which handles test descriptor program layout.

Line 149: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 142. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(ProgramRecord(clear_status_command(), (StreamBeat(0xAA, False), StreamBeat(0xBB, True)), 0,), ProgramRecord(clear_status_command(), (), 3),)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named 'stream beat' in this block. Context: the function test_descriptor_program_layout, which handles test descriptor program layout.

Line 150: `beats, outputs = pack_descriptor_program(records)` This assigns 'pack_descriptor_program(records)' to 'beats, outputs ' in host memory. Here beats means the value named 'beats' in this block. The important operands are pack_descriptor_program: a DMA record header plus one command and its declared payload; records: the number of descriptor records accepted since program start. Context: the function test_descriptor_program_layout, which handles test descriptor program layout.

Line 151: `self.assertEqual(outputs, 3)` This calls 'self.assertEqual' with 'outputs, 3' inside the function test_descriptor_prog which handles test descriptor program layout. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 152: `self.assertEqual(len(beats), 8)` This calls 'self.assertEqual' with 'len(beats), 8' inside the function test_descriptor_program_layout, which handles test descriptor program layout. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 153: `self.assertEqual(beats[0].data, 2)` This calls 'self.assertEqual' with 'beats[0].data, 2' inside the function test_descriptor_program_layout, which handles test descriptor program layout. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 154: `self.assertEqual(beats[5].data, (1 << 63) | (3 << 32))` This calls 'self.assertEqual' with 'beats[5].data, (1 << 63) | (3 << 32)' inside the function test_descriptor_program_layout, which handles test descriptor program layout. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 155: `self.assertEqual([beat.last for beat in beats], [False] * 7 + [True])` This calls ‘self.assertEqual‘ with ‘[beat.last for beat in beats], [False] * 7 + [True]‘ inside the function test_descriptor_program_layout, which handles test descriptor program layout. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 157: `def test_operator_stream_layouts(self):` This starts function test_operator_stream_layouts. Its indented body performs test operator stream layouts as part of the file that checks neuraedge protocol behavior independently with Python assertions; it runs only when called.

Line 158: `query = [index & 1 for index in range(256)]` This assigns ‘[index & 1 for index in range(256)]‘ to ‘query ‘ in host memory. Here query means the value named ’query’ in this block. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 159: `key = [1 - bit for bit in query]` This assigns ‘[1 - bit for bit in query]‘ to ‘key ‘ in host memory. Here key means the value named ’key’ in this block. The important operands are bit: the value named ’bit’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 160: `alpha = pack_alpha_beats(query, key)` This assigns ‘pack_alpha_beats(query, key)‘ to ‘alpha ‘ in host memory. Here alpha means the programmable weight assigned to matching zero spikes in alpha-XNOR attention. The important operands are pack_alpha_beats: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; query: the value named ’query’ in this block; key: the value named ’key’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 161: `self.assertEqual(len(alpha), 8)` This calls ‘self.assertEqual‘ with ‘len(alpha), 8‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 162: `self.assertEqual(alpha[0].data, 0xAAAAAAAAAAAAAAAA)` This calls ‘self.assertEqual‘ with ‘alpha[0].data, 0xAAAAAAAAAAAAAAAA‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 163: `self.assertEqual(alpha[4].data, 0x5555555555555555)` This calls ‘self.assertEqual‘ with ‘alpha[4].data, 0x5555555555555555‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 164: `self.assertTrue(alpha[-1].last)` This calls ‘self.assertTrue‘ with ‘alpha[-1].last‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 166: `vector = pack_vector_beats(range(-4, 4), range(10, 18), clamp_minimum=-7, clamp_maximum=8)` This assigns ‘pack_vector_beats(range(-4, 4), range(10, 18), clamp_minimum=-7, clamp_maximum=8)‘ to ‘vector ‘ in host memory. Here vector means the value named ’vector’ in this block. The important operands are pack_vector_beats: the value named ’pack vector beats’ in this block; range: the value named ’range’ in this block; clamp_minimum: the value named ’clamp minimum’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 167: `self.assertEqual(len(vector), 9)` This calls ‘self.assertEqual‘ with ‘len(vector), 9‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 168: `self.assertEqual(vector[0].data, 0x00000008FFFFFFF9)` This calls ‘self.assertEqual‘ with ‘vector[0].data, 0x00000008FFFFFFF9‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 169: `self.assertEqual(vector[1].data, 0xFFFFFFFDFFFFFFFC)` This calls ‘self.assertEqual‘ with ‘vector[1].data, 0xFFFFFFFDFFFFFFFC‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 170: `self.assertTrue(vector[-1].last)` This calls ‘self.assertTrue‘ with ‘vector[-1].last‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 172: `layernorm = pack_layernorm_beats([-1], [-2], [-3])` This assigns ‘pack_layernorm_beats([-1], [-2], [-3])‘ to ‘layernorm ‘ in host memory. Here layernorm means the value named ’layernorm’ in this block. The important operands are pack_layernorm_beats: the value named ’pack layernorm beats’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 173: `self.assertEqual(layernorm[0].data, 0xFFFDFFFFFEFFFFFF)` This calls ‘self.assertEqual‘ with ‘layernorm[0].data, 0xFFFDFFFFFEFFFFFF‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 174: `self.assertEqual(layernorm[1].data, 0xFF)` This calls ‘self.assertEqual‘ with ‘layernorm[1].data, 0xFF‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 175: `self.assertEqual([beat.last for beat in layernorm], [False, True])` This calls ‘self.assertEqual‘ with ‘[beat.last for beat in layernorm], [False, True]‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 176: `self.assertEqual(pack_softmax_beats([-1, 2])[0].data, 0xFFFFFF)` This calls ‘self.assertEqual‘ with ‘pack_softmax_beats([-1, 2])[0].data, 0xFFFFFF‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 178: `lif = pack_lif_beats(range(-16, 16))` This assigns ‘pack_lif_beats(range(-16, 16))‘ to ‘lif ‘ in host memory. Here lif means the value named ’lif’ in this block. The important operands are pack_lif_beats: the value named ’pack lif beats’ in this block; range: the value named ’range’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 179: `self.assertEqual(len(lif), 8)` This calls ‘self.assertEqual‘ with ‘len(lif), 8‘ inside the function test_operator_strea which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 180: `self.assertEqual(lif[0].data, 0xFFF3FFF2FFF1FFF0)` This calls ‘self.assertEqual‘ with ‘lif[0].data, 0xFFF3FFF2FFF1FFF0‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 181: `optimizer = pack_optimizer_beats(range(-4, 4), range(-8, 0), range(8))` This assigns ‘pack_optimizer_be 4, 4), range(-8, 0), range(8))‘ to ‘optimizer ‘ in host memory. Here optimizer means the value named ’optimizer’ in this block. The important operands are pack_optimizer_beats: the value named ’pack optimizer beats’ in this block; range: the value named ’range’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 182: `self.assertEqual(len(optimizer), 8)` This calls ‘self.assertEqual‘ with ‘len(optimizer), 8‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 183: `self.assertEqual(optimizer[0].data, 0x000000FFFFF8FFFC)` This calls ‘self.assertEqual‘ with ‘optimizer[0].data, 0x000000FFFFF8FFFC‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 184: `self.assertTrue(optimizer[-1].last)` This calls ‘self.assertTrue‘ with ‘optimizer[-1].last‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 186: `load = pack_lut_load_beats([(7, -2), (8, 3)])` This assigns ‘pack_lut_load_beats([(7, -2), (8, 3)])‘ to ‘load ‘ in host memory. Here load means the value named ’load’ in this block. The important operands are pack_lut_load_beats: the value named ’pack lut load beats’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 187: `self.assertEqual(load[0].data, 0x00FFFE07)` This calls ‘self.assertEqual‘ with ‘load[0].data, 0x00FFFE07‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 188: `self.assertEqual([beat.last for beat in load], [False, True])` This calls ‘self.assertEqual‘ with ‘[beat.last for beat in load], [False, True]‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 189: `self.assertEqual(pack_lut_eval_beat(range(8))[0].data, 0x0706050403020100)` This calls ‘self.assertEqual‘ with ‘pack_lut_eval_beat(range(8))[0].data, 0x0706050403020100‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 191: `backward = pack_linear_backward_beats(` This statement continues through source line 195; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]],)‘ to ‘backward ‘ in host memory. Here backward means the value named ’backward’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 192: `[[-128, -1, 0, 1], [127, 2, -3, 4]],` This is continuation line 2 of the 5-line logical statement that begins at source line 191. Its exact fragment is ‘[[-128, -1, 0, 1], [127, 2, -3, 4]],‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]],)‘ to ‘backward ‘ in host memory. Here backward means the value named ’backward’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 193: `[[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]],` This is continuation line 3 of the 5-line logical statement that begins at source line 191. Its exact fragment is ‘[[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]],‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]],)‘ to ‘backward ‘ in host memory. Here backward means the value named ’backward’ in this block. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 194: `[[-32768, -1, 32767], [10, 11, 12]],` This is continuation line 4 of the 5-line logical statement that begins at source line 191. Its exact fragment is '[[-32768, -1, 32767], [10, 11, 12]],'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]],)' to 'backward ' in host memory. Here backward means the value named 'backward' in this block. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 195: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 191. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]],)' to 'backward ' in host memory. Here backward means the value named 'backward' in this block. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 196: `self.assertEqual(len(backward), 16)` This calls 'self.assertEqual' with 'len(backward), 16' inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 197: `self.assertEqual(backward[0].data, 0x00010000FFFFFF80)` This calls 'self.assertEqual' with 'backward[0].data, 0x00010000FFFFFF80' inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 198: `self.assertEqual(backward[1].data, 0)` This calls 'self.assertEqual' with 'backward[1].data, 0' inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 199: `self.assertEqual(backward[2].data, 0x0004FFFD0002007F)` This calls 'self.assertEqual' with 'backward[2].data, 0x0004FFFD0002007F' inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 200: `self.assertEqual(backward[4].data, 0x00007FFFFFFF8000)` This calls 'self.assertEqual' with 'backward[4].data, 0x00007FFFFFFF8000' inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 201: `self.assertEqual(backward[5].data, 0)` This calls 'self.assertEqual' with 'backward[5].data, 0' inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 202: `self.assertEqual([beat.last for beat in backward], [False] * 15 + [True])` This calls 'self.assertEqual' with '[beat.last for beat in backward], [False] * 15 + [True]' inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 203: `input_only = pack_linear_backward_beats(` This statement continues through source line 208; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.INPUT,)' to 'input_only ' in host memory. Here input_only means the value named 'input only' in this block. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block; mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to

update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 204: `[[-128, -1, 0, 1], [127, 2, -3, 4]],` This is continuation line 2 of the 6-line logical statement that begins at source line 203. Its exact fragment is '[[-128, -1, 0, 1], [127, 2, -3, 4]],'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.INPUT,)' to 'input_only ' in host memory. Here input_only means the value named 'input only' in this block. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block; mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 205: `[[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]],` This is continuation line 3 of the 6-line logical statement that begins at source line 203. Its exact fragment is '[[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]],'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.INPUT,)' to 'input_only ' in host memory. Here input_only means the value named 'input only' in this block. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block; mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 206: `[[-32768, -1, 32767], [10, 11, 12]],` This is continuation line 4 of the 6-line logical statement that begins at source line 203. Its exact fragment is '[[-32768, -1, 32767], [10, 11, 12]],'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.INPUT,)' to 'input_only ' in host memory. Here input_only means the value named 'input only' in this block. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block; mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 207: `mode=LinearGradientMode.INPUT,` This is continuation line 5 of the 6-line logical statement that begins at source line 203. Its exact fragment is 'mode=LinearGradientMode.INPUT,'. The important operands are mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; INPUT: the value named 'input' in this block. Read together, the complete operation is: This assigns 'pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.INPUT,)' to 'input_only ' in host memory. Here input_only means the value named 'input only' in this block. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block; mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 208: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 203. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.INPUT,)' to 'input_only ' in host memory. Here input_only means the value named 'input only' in this block. The important operands are pack_linear_backward_beats: the value named 'pack linear backward beats' in this block; mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 209: `weight_only = pack_linear_backward_beats(` This statement continues through source line 214; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]],

mode=LinearGradientMode.WEIGHT,)‘ to ‘weight_only ‘ in host memory. Here weight_only means a learned coefficient multiplied by an activation. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 210: `[[-128, -1, 0, 1], [127, 2, -3, 4]],` This is continuation line 2 of the 6-line logical statement that begins at source line 209. Its exact fragment is ‘[[-128, -1, 0, 1], [127, 2, -3, 4]],‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.WEIGHT,)‘ to ‘weight_only ‘ in host memory. Here weight_only means a learned coefficient multiplied by an activation. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 211: `[[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]],` This is continuation line 3 of the 6-line logical statement that begins at source line 209. Its exact fragment is ‘[[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]],‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.WEIGHT,)‘ to ‘weight_only ‘ in host memory. Here weight_only means a learned coefficient multiplied by an activation. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 212: `[[-32768, -1, 32767], [10, 11, 12]],` This is continuation line 4 of the 6-line logical statement that begins at source line 209. Its exact fragment is ‘[[-32768, -1, 32767], [10, 11, 12]],‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.WEIGHT,)‘ to ‘weight_only ‘ in host memory. Here weight_only means a learned coefficient multiplied by an activation. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 213: `mode=LinearGradientMode.WEIGHT,` This is continuation line 5 of the 6-line logical statement that begins at source line 209. Its exact fragment is ‘mode=LinearGradientMode.WEIGHT,‘. The important operands are mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.WEIGHT,)‘ to ‘weight_only ‘ in host memory. Here weight_only means a learned coefficient multiplied by an activation. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 214: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 209. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘pack_linear_backward_beats([[-128, -1, 0, 1], [127, 2, -3, 4]], [[-32768, -1, 32767], [1, 2, 3], [4, 5, 6], [7, 8, 9]], [[-32768, -1, 32767], [10, 11, 12]], mode=LinearGradientMode.WEIGHT,)‘ to ‘weight_only ‘ in host memory. Here weight_only means a learned coefficient multiplied by an activation. The important operands are pack_linear_backward_beats: the value named ’pack linear backward beats’ in this block; mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Context: the function test_operator_stream_layouts, which handles test operator stream layouts.

Line 215: `self.assertEqual(len(input_only), 12)` This calls ‘self.assertEqual‘ with ‘len(input_only), 12‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 216: `self.assertEqual(input_only[0].data, backward[4].data)` This calls ‘self.assertEqual‘ with ‘input_only[0].data, backward[4].data‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 217: `self.assertEqual(len(weight_only), 8)` This calls ‘self.assertEqual‘ with ‘len(weight_only), 8‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 218: `self.assertEqual(weight_only[0].data, backward[0].data)` This calls ‘self.assertEqual‘ with ‘weight_only[0].data, backward[0].data‘ inside the function test_operator_stream_layouts, which handles test operator stream layouts. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 220: `def test_result_decoders(self):` This starts function test_result_decoders. Its indented body performs test result decoders as part of the file that checks neuraedge protocol behavior independently with Python assertions; it runs only when called.

Line 221: `tensor_word = ((-7) & ((1 << 48) - 1)) | (5 << 48) | (12 << 52) | (1 << 58)` This assigns ‘((-7) & ((1 << 48) - 1)) | (5 << 48) | (12 << 52) | (1 << 58)‘ to ‘tensor_word ‘ in host memory. Here tensor_word means the value named ’tensor word’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 222: `self.assertEqual(unpack_tensor_result(tensor_word), (-7, 5, 12, True))` This calls ‘self.assertEqual‘ with ‘unpack_tensor_result(tensor_word), (-7, 5, 12, True)‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 223: `self.assertEqual(unpack_alpha_result(11 | (12 << 18) | (13 << 27)), (11, 12, 13))` This calls ‘self.assertEqual‘ with ‘unpack_alpha_result(11 | (12 << 18) | (13 << 27)), (11, 12, 13)‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 224: `self.assertEqual(unpack_vector_result([0xFFFFFFFF00000001, 0, 0, 0])[:2], (1, -1))` This calls ‘self.assertEqual‘ with ‘unpack_vector_result([0xFFFFFFFF00000001, 0, 0, 0])[:2], (1, -1)‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 225: `self.assertEqual(unpack_layernorm_result(0xFFFFFFFF | (9 << 32) | (1 << 44)), (-1, 9, True))` This calls ‘self.assertEqual‘ with ‘unpack_layernorm_result(0xFFFFFFFF | (9 << 32) | (1 << 44)), (-1, 9, True)‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 226: `self.assertEqual(unpack_softmax_result(123 | (7 << 16) | (1 << 25)), (123, 7, True))` This calls ‘self.assertEqual‘ with ‘unpack_softmax_result(123 | (7 << 16) | (1 << 25)), (123, 7, True)‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 227: `self.assertEqual(unpack_lif_result(0x12345678ABCDEF01), (0xABCDEF01, 0x12345678))` This calls ‘self.assertEqual‘ with ‘unpack_lif_result(0x12345678ABCDEF01), (0xABCDEF01, 0x12345678)‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 228: `expected_membranes = tuple(` This statement continues through source line 231; the explanation uses the complete expression rather than only the first visual fragment. Inside the function test_result_decoders, which handles test result decoders, this statement advances the host implementation that checks neuraedge protocol behavior independently with Python assertions. The important operands are expected_membranes: the persistent LIF neuron state carrying charge between timesteps; tuple: the value named 'tuple' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 229: `(-1 if lane % 3 == 0 else (lane * 7919) % (1 << 23))` This is continuation line 2 of the 4-line logical statement that begins at source line 228. Its exact fragment is '(-1 if lane % 3 == 0 else (lane * 7919) % (1 << 23))'. The important operands are lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: Inside the function test_result_decoders, which handles test result decoders, this statement advances the host implementation that checks neuraedge protocol behavior independently with Python assertions. The important operands are expected_membranes: the persistent LIF neuron state carrying charge between timesteps; tuple: the value named 'tuple' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 230: `for lane in range(32)` This is continuation line 3 of the 4-line logical statement that begins at source line 228. Its exact fragment is 'for lane in range(32)'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named 'in' in this block. Read together, the complete operation is: Inside the function test_result_decoders, which handles test result decoders, this statement advances the host implementation that checks neuraedge protocol behavior independently with Python assertions. The important operands are expected_membranes: the persistent LIF neuron state carrying charge between timesteps; tuple: the value named 'tuple' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 231: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 228. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function test_result_decoders, which handles test result decoders, this statement advances the host implementation that checks neuraedge protocol behavior independently with Python assertions. The important operands are expected_membranes: the persistent LIF neuron state carrying charge between timesteps; tuple: the value named 'tuple' in this block; lane: one parallel arithmetic element within a vector or tensor datapath.

Line 232: `packed_membranes = sum(` This statement continues through source line 235; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sum((value & 0xFFFFFF) << (24 * lane) for lane, value in enumerate(expected_membranes))' to 'packed_membranes ' in host memory. Here packed_membranes means the persistent LIF neuron state carrying charge between timesteps. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function test_result_decoders, which handles test result decoders.

Line 233: `(value & 0xFFFFFF) << (24 * lane)` This is continuation line 2 of the 4-line logical statement that begins at source line 232. Its exact fragment is '(value & 0xFFFFFF) << (24 * lane)'. The important operands are value: the value named 'value' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'sum((value & 0xFFFFFF) << (24 * lane) for lane, value in enumerate(expected_membranes))' to 'packed_membranes ' in host memory. Here packed_membranes means the persistent LIF neuron state carrying charge between timesteps. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function test_result_decoders, which handles test result decoders.

Line 234: `for lane, value in enumerate(expected_membranes)` This is continuation line 3 of the 4-line logical statement that begins at source line 232. Its exact fragment is 'for lane, value in enumerate(expected_membranes)'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; value: the value named 'value' in this block. Read together, the complete operation is: This assigns 'sum((value & 0xFFFFFF) << (24 * lane) for lane, value in enumerate(expected_membranes))' to 'packed_membranes ' in host memory. Here packed_membranes means the persistent LIF neuron state carrying charge between timesteps. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function test_result_decoders, which handles test result decoders.

Line 235: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 232. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sum((value & 0xFFFFFF) << (24 * lane) for lane, value in enumerate(expected_membranes))' to 'packed_membranes ' in host memory. Here packed_membranes means the persistent LIF neuron state carrying charge between timesteps. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function test_result_decoders, which handles test result decoders.

Line 236: `trace_words = [0x12345678ABCDEF01]` This assigns '[0x12345678ABCDEF01]' to 'trace_words ' in host memory. Here trace_words means the value named 'trace words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 237: `trace_words.extend(` This statement continues through source line 240; the explanation uses the complete expression rather than only the first visual fragment. This calls 'trace_words.extend' with ' (packed_membranes >> (64 * index)) & 0xFFFFFFFFFFFFFFFF for index in range(12) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 238: `(packed_membranes >> (64 * index)) & 0xFFFFFFFFFFFFFFFF` This is continuation line 2 of the 4-line logical statement that begins at source line 237. Its exact fragment is '(packed_membranes >> (64 * index)) & 0xFFFFFFFFFFFFFFFF'. The important operands are packed_membranes: the persistent LIF neuron state carrying charge between timesteps; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'trace_words.extend' with ' (packed_membranes >> (64 * index)) & 0xFFFFFFFFFFFFFFFF for index in range(12) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 239: `for index in range(12)` This is continuation line 3 of the 4-line logical statement that begins at source line 237. Its exact fragment is 'for index in range(12)'. The important operands are for: the value named 'for' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; in: the value named 'in' in this block. Read together, the complete operation is: This calls 'trace_words.extend' with ' (packed_membranes >> (64 * index)) & 0xFFFFFFFFFFFFFFFF for index in range(12) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 240: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 237. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'trace_words.extend' with ' (packed_membranes >> (64 * index)) & 0xFFFFFFFFFFFFFFFF for index in range(12) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 241: `self.assertEqual(` This statement continues through source line 244; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertEqual' with ' unpack_lif_trace(trace_words), (0xABCDEF01, 0x12345678, expected_membranes), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 242: `unpack_lif_trace(trace_words),` This is continuation line 2 of the 4-line logical statement that begins at source line 241. Its exact fragment is 'unpack_lif_trace(trace_words),'. The important operands are unpack_lif_trace: the value named 'unpack lif trace' in this block; trace_words: the value named 'trace words' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_lif_trace(trace_words), (0xABCDEF01, 0x12345678, expected_membranes), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 243: `(0xABCDEF01, 0x12345678, expected_membranes),` This is continuation line 3 of the 4-line logical statement that begins at source line 241. Its exact fragment is '(0xABCDEF01, 0x12345678, expected_membranes),'. The important operands are expected_membranes: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_lif_trace(trace_words), (0xABCDEF01, 0x12345678, expected_membranes), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 244: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 241. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_lif_trace(trace_words), (0xABCDEF01, 0x12345678, expected_membranes), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 245: `with self.assertRaises(ValueError):` This opens a managed resource scope for 'self.assertRaises(ValueError)' so files, tracing, or temporary state are finalized even if the function test_result_decoders, which handles test result decoders raises an error.

Line 246: `unpack_lif_trace(trace_words[:-1])` This calls 'unpack_lif_trace' with 'trace_words[:-1]' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 247: `weights, velocities = unpack_optimizer_result([0xFFFFFF000001FFFF] * 8)` This assigns 'unpack_optimizer_ * 8)' to 'weights, velocities ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are unpack_optimizer_result: the value named 'unpack optimizer result' in this block. Context: the function test_result_decoders, which handles test result decoders.

Line 248: `self.assertEqual(weights[0], -1)` This calls 'self.assertEqual' with 'weights[0], -1' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 249: `self.assertEqual(velocities[0], 1)` This calls 'self.assertEqual' with 'velocities[0], 1' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 250: `self.assertEqual(unpack_lut_result([0x0003FFFE0001FFFF, 0]), (-1, 1, -2, 3, 0, 0, 0, 0))` This calls 'self.assertEqual' with 'unpack_lut_result([0x0003FFFE0001FFFF, 0]), (-1, 1, -2, 3, 0, 0, 0, 0)' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 252: `expected_dx = ((-1, 2, -(1 << 31), (1 << 31) - 1), (9, -10, 11, -12))` This assigns '((-1, 2, -(1 << 31), (1 << 31) - 1), (9, -10, 11, -12))' to 'expected_dx ' in host memory. Here expected_dx means the value named 'expected dx' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 253: `expected_dw = (` This statement continues through source line 258; the explanation uses the complete expression rather than only the first visual fragment. This assigns '((-1, 2, -3), ((1 << 23) - 1, -(1 << 23), 7), (8, 9, 10), (-11, -12, -13),)' to 'expected_dw ' in host memory. Here expected_dw means the value named 'expected dw' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 254: `(-1, 2, -3),` This is continuation line 2 of the 6-line logical statement that begins at source line 253. Its exact fragment is '(-1, 2, -3),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '((-1, 2, -3), ((1 << 23) - 1, -(1 << 23), 7), (8, 9, 10), (-11, -12, -13),)' to 'expected_dw ' in host memory. Here expected_dw means the value named 'expected dw' in this block. The expression

is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 255: `((1 << 23) - 1, -(1 << 23), 7),` This is continuation line 3 of the 6-line logical statement that begins at source line 253. Its exact fragment is '((1 << 23) - 1, -(1 << 23), 7),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '((-1, 2, -3), ((1 << 23) - 1, -(1 << 23), 7), (8, 9, 10), (-11, -12, -13),)' to 'expected_dw ' in host memory. Here expected_dw means the value named 'expected dw' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 256: `(8, 9, 10),` This is continuation line 4 of the 6-line logical statement that begins at source line 253. Its exact fragment is '(8, 9, 10),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '((-1, 2, -3), ((1 << 23) - 1, -(1 << 23), 7), (8, 9, 10), (-11, -12, -13),)' to 'expected_dw ' in host memory. Here expected_dw means the value named 'expected dw' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 257: `(-11, -12, -13),` This is continuation line 5 of the 6-line logical statement that begins at source line 253. Its exact fragment is '(-11, -12, -13),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '((-1, 2, -3), ((1 << 23) - 1, -(1 << 23), 7), (8, 9, 10), (-11, -12, -13),)' to 'expected_dw ' in host memory. Here expected_dw means the value named 'expected dw' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 258: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 253. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '((-1, 2, -3), ((1 << 23) - 1, -(1 << 23), 7), (8, 9, 10), (-11, -12, -13),)' to 'expected_dw ' in host memory. Here expected_dw means the value named 'expected dw' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 259: `backward_words = []` This assigns '[]' to 'backward_words ' in host memory. Here backward_words means the value named 'backward words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_result_decoders, which handles test result decoders.

Line 260: `for row in expected_dx:` This iterates row over 'expected_dx'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_result_decoders, which handles test result decoders.

Line 261: `padded = row + (0,) * (8 - len(row))` This assigns 'row + (0,) * (8 - len(row))' to 'padded ' in host memory. Here padded means the value named 'padded' in this block. The important operands are row: one output-row position processed by the tensor tile; len: the value named 'len' in this block. Context: the function test_result_decoders, which handles test result decoders.

Line 262: `backward_words.extend(` This statement continues through source line 266; the explanation uses the complete expression rather than only the first visual fragment. This calls 'backward_words.extend' with ' (padded[lane] & 0xFFFFFFFF) | ((padded[lane + 1] & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 263: `(padded[lane] & 0xFFFFFFFF)` This is continuation line 2 of the 5-line logical statement that begins at source line 262. Its exact fragment is '(padded[lane] & 0xFFFFFFFF)'. The important operands are padded: the value named 'padded' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls 'backward_words.extend' with ' (padded[lane] & 0xFFFFFFFF) | ((padded[lane + 1] & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2) ' inside the function test_result_decoders, which handles test

result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 264: `| ((padded[lane + 1] & 0xFFFFFFFF) << 32)` This is continuation line 3 of the 5-line logical statement that begins at source line 262. Its exact fragment is '| ((padded[lane + 1] & 0xFFFFFFFF) << 32)'. The important operands are padded: the value named 'padded' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls 'backward_words.extend' with ' (padded[lane] & 0xFFFFFFFF) | ((padded[lane + 1] & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 265: `for lane in range(0, 8, 2)` This is continuation line 4 of the 5-line logical statement that begins at source line 262. Its exact fragment is 'for lane in range(0, 8, 2)'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named 'in' in this block. Read together, the complete operation is: This calls 'backward_words.extend' with ' (padded[lane] & 0xFFFFFFFF) | ((padded[lane + 1] & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 266: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 262. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'backward_words.extend' with ' (padded[lane] & 0xFFFFFFFF) | ((padded[lane + 1] & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 267: `for row in expected_dw:` This iterates row over 'expected_dw'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_result_decoders, which handles test result decoders.

Line 268: `padded = row + (0,) * (8 - len(row))` This assigns 'row + (0,) * (8 - len(row))' to 'padded ' in host memory. Here padded means the value named 'padded' in this block. The important operands are row: one output-row position processed by the tensor tile; len: the value named 'len' in this block. Context: the function test_result_decoders, which handles test result decoders.

Line 269: `packed = sum((value & 0xFFFFFF) << (24 * lane)` This statement continues through source line 270; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sum((value & 0xFFFFFF) << (24 * lane) for lane, value in enumerate(padded))' to 'packed ' in host memory. Here packed means the value named 'packed' in this block. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function test_result_decoders, which handles test result decoders.

Line 270: `for lane, value in enumerate(padded))` This is continuation line 2 of the 2-line logical statement that begins at source line 269. Its exact fragment is 'for lane, value in enumerate(padded))'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; value: the value named 'value' in this block. Read together, the complete operation is: This assigns 'sum((value & 0xFFFFFF) << (24 * lane) for lane, value in enumerate(padded))' to 'packed ' in host memory. Here packed means the value named 'packed' in this block. The important operands are sum: the value named 'sum' in this block; value: the value named 'value' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function test_result_decoders, which handles test result decoders.

Line 271: `backward_words.extend(` This statement continues through source line 273; the explanation uses the complete expression rather than only the first visual fragment. This calls 'backward_words.extend' with ' (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 272: `(packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3)` This is continuation line 2 of the 3-line logical statement that begins at source line 271. Its exact fragment is '(packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3)'. The important operands are packed: the value named 'packed' in this block; word: the value named 'word' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This calls 'backward_words.extend' with ' (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 273: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 271. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'backward_words.extend' with ' (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3) ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 274: `self.assertEqual(` This statement continues through source line 277; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_wo 2, 4, 3), (expected_dx, expected_dw), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 275: `unpack_linear_backward_result(backward_words, 2, 4, 3),` This is continuation line 2 of the 4-line logical statement that begins at source line 274. Its exact fragment is 'unpack_linear_backward_result(backward_words, 2, 4, 3),'. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block; backward_words: the value named 'backward words' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words, 2, 4, 3), (expected_dx, expected_dw), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 276: `(expected_dx, expected_dw),` This is continuation line 3 of the 4-line logical statement that begins at source line 274. Its exact fragment is '(expected_dx, expected_dw),'. The important operands are expected_dx: the value named 'expected dx' in this block; expected_dw: the value named 'expected dw' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words, 2, 4, 3), (expected_dx, expected_dw), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 277: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 274. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words, 2, 4, 3), (expected_dx, expected_dw), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 278: `self.assertEqual(` This statement continues through source line 283; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[:8], 2, 4, 3, mode=LinearGradientMode.INPUT), (expected_dx, ()), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 279: `unpack_linear_backward_result(` This is continuation line 2 of the 6-line logical statement that begins at source line 278. Its exact fragment is 'unpack_linear_backward_result('. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[:8], 2, 4, 3, mode=LinearGradientMode.INPUT), (expected_dx, ()), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 280: `backward_words[:8], 2, 4, 3, mode=LinearGradientMode.INPUT` This is continuation line 3 of the 6-line logical statement that begins at source line 278. Its exact fragment is 'backward_words[:8], 2, 4, 3, mode=LinearGradientMode.I The important operands are backward_words: the value named 'backward words' in this block; mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[:8], 2, 4, 3, mode=LinearGradientMode.INPUT), (expected_dx, ()), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 281: `),` This is continuation line 4 of the 6-line logical statement that begins at source line 278. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[:8], 2, 4, 3, mode=LinearGradientMode.INPUT), (expected_dx, ()), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 282: `(expected_dx, ()),` This is continuation line 5 of the 6-line logical statement that begins at source line 278. Its exact fragment is '(expected_dx, ()),'. The important operands are expected_dx: the value named 'expected dx' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[:8], 2, 4, 3, mode=LinearGradientMode.INPUT), (expected_dx, ()), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 283: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 278. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[:8], 2, 4, 3, mode=LinearGradientMode.INPUT), (expected_dx, ()), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 284: `self.assertEqual(` This statement continues through source line 289; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[8:], 2, 4, 3, mode=LinearGradientMode.WEIGHT), ((), expected_dw), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 285: `unpack_linear_backward_result(` This is continuation line 2 of the 6-line logical statement that begins at source line 284. Its exact fragment is 'unpack_linear_backward_result('. The important operands are unpack_linear_backward_result: the value named 'unpack linear backward result' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[8:], 2, 4, 3, mode=LinearGradientMode.WEIGHT), ((), expected_dw), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 286: `backward_words[8:], 2, 4, 3, mode=LinearGradientMode.WEIGHT` This is continuation line 3 of the 6-line logical statement that begins at source line 284. Its exact fragment is 'backward_words[8:], 2, 4, 3, mode=LinearGradientMode.W The important operands are backward_words: the value named 'backward words' in this block; mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[8:], 2, 4, 3, mode=LinearGradientMode.WEIGHT), ((), expected_dw), ' inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 287: `),` This is continuation line 4 of the 6-line logical statement that begins at source line 284. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with ' unpack_linear_backward_result(backward_words[8:], 2, 4, 3,

mode=LinearGradientMode.WEIGHT), (), expected_dw), ‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 288: `(), expected_dw),` This is continuation line 5 of the 6-line logical statement that begins at source line 284. Its exact fragment is ‘(), expected_dw),‘. The important operands are expected_dw: the value named ’expected dw’ in this block. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ unpack_linear_backward_result(backward_words[8:], 2, 4, 3, mode=LinearGradientMode.WEIGHT), (), expected_dw), ‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 289: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 284. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘ unpack_linear_backward_result(backward_words[8:], 2, 4, 3, mode=LinearGradientMode.WEIGHT), (), expected_dw), ‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 290: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_result_decoders, which handles test result decoders raises an error.

Line 291: `unpack_linear_backward_result(backward_words[:-1], 2, 4, 3)` This calls ‘unpack_linear_backward_result‘ with ‘backward_words[:-1], 2, 4, 3‘ inside the function test_result_decoders, which handles test result decoders. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

Line 294: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks neuraedge protocol behavior independently with Python assertions.

Line 295: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks neuraedge protocol behavior independently with Python assertions. The call advances the host-side job that checks neuraedge protocol behavior independently with Python assertions.

67 tests/test_neuraedge_pynq.py

Role in the complete system: checks neuraedge pynq behavior independently with Python assertions.

67.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks neuraedge pynq behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: FakeMmio, FakeBuffer, FakeChannel, FakeDma, PynqDriverTests. Functions: ___init___, read, write, ___new___, flush, invalidate, freebuffer, ___init___, transfer, wait, ___init___, make_driver, test_identity_and_m test_performance_snapshot_decodes_hardware_registers, test_receive_is_armed_before_submit_and_send, test_rejects_ test_descriptor_program_uses_one_dma_pair, test_descriptor_program_partitions_at_dma_packet_boundary. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

67.2 Complete source

```
import unittest

import numpy as np

from neuraedge_protocol import ProgramRecord, StreamBeat, clear_status_command
from neuraedge_pynq import NeuraEdgeDriver, NeuraEdgeHardwareError


class FakeMmio:
    def __init__(self, events, *, error=False):
        self.events = events
        self.error = error
        self.registers = {0x30: 0x00010007, 0x34: 0x4E455552,
                          0x28: 200000000, 0x2C: 0x1FF, 0x04: 0}

    def read(self, address):
        return self.registers.get(address, 0)

    def write(self, address, value):
        self.events.append(("mmio", address, value))
        self.registers[address] = value
        if address == 0 and value & 0x5:
            self.registers[4] = 0x2 if self.error else 0x4


class FakeBuffer(np.ndarray):
    def __new__(cls, shape, dtype):
        return np.zeros(shape, dtype=dtype).view(cls)

    def flush(self):
        pass

    def invalidate(self):
        pass

    def freebuffer(self):
        self.freed = True


class FakeChannel:
    def __init__(self, events, name, receive_words=()):
        self.events = events
        self.name = name
        self.receive_words = receive_words
        self.idle = True

    def transfer(self, buffer, nbytes):
        self.events.append((self.name, "transfer", nbytes))
        if self.name == "recv":
```

```
            buffer[:len(self.receive_words)] = self.receive_words

    def wait(self):
        self.events.append((self.name, "wait"))


class FakeDma:
    def __init__(self, events, receive_words=()):
        self.sendchannel = FakeChannel(events, "send")
        self.recvchannel = FakeChannel(events, "recv", receive_words)


class PynqDriverTests(unittest.TestCase):
    def make_driver(self, *, receive_words=(), error=False):
        events = []
        mmio = FakeMmio(events, error=error)
        dma = FakeDma(events, receive_words)
        driver = NeuraEdgeDriver(mmio, dma, lambda *, shape, dtype: FakeBuffer(shape, dtype))
        return driver, events

    def test_identity_and_metadata(self):
        driver, _ = self.make_driver()
        self.assertEqual(driver.target_clock_hz, 200000000)
        self.assertEqual(driver.capabilities, 0x1FF)

    def test_performance_snapshot_decodes_hardware_registers(self):
        driver, _ = self.make_driver()
        driver.mmio.registers.update({
            0x38: (29 << 16) | 31,
            0x3C: 17,
            0x40: 1234,
            0x44: 14,
            0x48: 0x89ABCDEF,
            0x4C: 0x01234567,
            0x50: 0x76543210,
            0x54: 0xFEDCBA98,
            0x58: 222,
            0x5C: 0xFFFFFFF9,
            0x60: 81,
            0x64: 9,
            0x68: 333,
            0x6C: 0xFFFFFFF5,
            0x70: 0xABCDEF,
            0x74: 0x11111111,
            0x78: 0x22222222,
            0x7C: 44,
            0x80: 256,
            0x84: 0x12345678,
            0x88: 0x87654321,
            0x94: 19,
            0x98: 9876,
        })
        snapshot = driver.performance_snapshot()
        self.assertEqual(snapshot.tensor_loaded_activations, 31)
        self.assertEqual(snapshot.tensor_loaded_weights, 29)
        self.assertEqual(snapshot.tensor_active_issues, 17)
        self.assertEqual(snapshot.dense_macs, 0x0123456789ABCDEF)
        self.assertEqual(snapshot.spike_adds, 0xFEDCBA9876543210)
        self.assertEqual(snapshot.layernorm_mean, -7)
        self.assertEqual(snapshot.softmax_maximum, -11)
        self.assertEqual(snapshot.lif_total_spikes, 0x2222222211111111)
        self.assertEqual(snapshot.lut_loaded_entries, 256)
        self.assertEqual(snapshot.training_tiles, 19)
        self.assertEqual(snapshot.training_cycles, 9876)

    def test_receive_is_armed_before_submit_and_send(self):
        driver, events = self.make_driver(receive_words=(0x1234, 0x5678))
        results = driver.execute(
            clear_status_command(),
            [StreamBeat(1, False), StreamBeat(2, True)],
            output_beats=2,
        )
        self.assertEqual(results, (0x1234, 0x5678))
        receive_index = events.index(("recv", "transfer", 16))
        submit_index = next(index for index, event in enumerate(events)
                            if event[0] == "mmio" and event[1] == 0 and event[2] & 1)
        send_index = events.index(("send", "transfer", 16))
        self.assertLess(receive_index, submit_index)
        self.assertLess(submit_index, send_index)

    def test_rejects_bad_last_and_hardware_error(self):
        driver, _ = self.make_driver()
        with self.assertRaises(ValueError):
            driver.execute(clear_status_command(), [StreamBeat(1, False)])
        driver, _ = self.make_driver(error=True)
        with self.assertRaises(NeuraEdgeHardwareError):
            driver.execute(clear_status_command())
```

```
137
138     def test_descriptor_program_uses_one_dma_pair(self):
139         driver, events = self.make_driver(receive_words=(0xAA, 0xBB))
140         driver.mmio.registers[0x8C] = 2
141         records = (
142             ProgramRecord(
143                 clear_status_command(),
144                 (StreamBeat(0x11, False), StreamBeat(0x22, True)),
145                 0,
146             ),
147             ProgramRecord(clear_status_command(), (), 2),
148         )
149         result = driver.execute_program(records)
150         self.assertEqual(result, (0xAA, 0xBB))
151         self.assertIn(("recv", "transfer", 16), events)
152         self.assertIn(("send", "transfer", 64), events)
153         self.assertIn(("mmio", 0x90, 2), events)
154         self.assertTrue(any(event[0] == "mmio" and event[1] == 0
155                             and event[2] & 4 for event in events))
156
157     def test_descriptor_program_partitions_at_dma_packet_boundary(self):
158         driver, events = self.make_driver(receive_words=(0xAA,))
159         driver.MAX_DMA_TRANSFER_BYTES = 32
160         driver.mmio.registers[0x8C] = 1
161         records = (
162             ProgramRecord(clear_status_command(), (), 1),
163             ProgramRecord(clear_status_command(), (), 1),
164         )
165         result = driver.execute_program(records)
166         self.assertEqual(result, (0xAA, 0xAA))
167         self.assertEqual(events.count(("send", "transfer", 24)), 2)
168         self.assertEqual(events.count(("recv", "transfer", 8)), 2)
169
170
171 if __name__ == "__main__":
172     unittest.main()
```

67.3 Line annotations

Line 1: `import unittest` This imports the named Python API so the file whose role is to checks neuraedge pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import numpy as np` This imports the named Python API so the file whose role is to checks neuraedge pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `from neuraedge_protocol import ProgramRecord, StreamBeat, clear_status_command` This imports the named Python API so the file whose role is to checks neuraedge pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `from neuraedge_pynq import NeuraEdgeDriver, NeuraEdgeHardwareError` This imports the named Python API so the file whose role is to checks neuraedge pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `class FakeMmio:` This defines class FakeMmio, which groups data and methods for fake mmio. Instances exist in host memory and organize the flow that checks neuraedge pynq behavior independently with Python assertions.

Line 10: `def __init__(self, events, *, error=False):` This starts function ___init___. Its indented body performs init as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 11: `self.events = events` This assigns ‘events‘ to ‘self.events ‘ in host memory. Here events means the value named ’events’ in this block. The important operands are events: the value named ’events’ in this block. Context: the function ___init___, which handles init.

Line 12: `self.error = error` This assigns ‘error‘ to ‘self.error ‘ in host memory. Here error means the value named ’error’ in this block. The important operands are error: the value named ’error’ in this block. Context: the function ___init___, which handles init.

Line 13: `self.registers = {0x30: 0x00010007, 0x34: 0x4E455552,` This statement continues through source line 14; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{0x30: 0x00010007, 0x34: 0x4E455552, 0x28: 200000000, 0x2C: 0x1FF, 0x04: 0}‘ to ‘self.registers ‘ in host memory. Here registers means the value named ’registers’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 14: `0x28: 200000000, 0x2C: 0x1FF, 0x04: 0}` This is continuation line 2 of the 2-line logical statement that begins at source line 13. Its exact fragment is ‘0x28: 200000000, 0x2C: 0x1FF, 0x04: 0}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{0x30: 0x00010007, 0x34: 0x4E455552, 0x28: 200000000, 0x2C: 0x1FF, 0x04: 0}‘ to ‘self.registers ‘ in host memory. Here registers means the value named ’registers’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 16: `def read(self, address):` This starts function read. Its indented body performs read as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 17: `return self.registers.get(address, 0)` This returns ‘self.registers.get(address, 0)‘ to the caller of the function read, which handles read, ending this invocation. The important operands are registers: the value named ’registers’ in this block; get: the value named ’get’ in this block; address: the value named ’address’ in this block.

Line 19: `def write(self, address, value):` This starts function write. Its indented body performs write as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 20: `self.events.append(("mmio", address, value))` This calls ‘self.events.append‘ with ‘("mmio", address, value)‘ inside the function write, which handles write. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 21: `self.registers[address] = value` This assigns ‘value‘ to ‘self.registers[address] ‘ in host memory. Here registers means the value named ’registers’ in this block. The important operands are value: the value named ’value’ in this block. Context: the function write, which handles write.

Line 22: `if address == 0 and value & 0x5:` This executes the following indented host-code branch only when ‘address == 0 and value & 0x5‘ is true. The important operands are address: the value named ’address’ in this block; and: the value named ’and’ in this block; value: the value named ’value’ in this block. Context: the function write, which handles write.

Line 23: `self.registers[4] = 0x2 if self.error else 0x4` This assigns ‘0x2 if self.error else 0x4‘ to ‘self.registers[4] ‘ in host memory. Here registers means the value named ’registers’ in this block. The important operands are error: the value named ’error’ in this block. Context: the function write, which handles write.

Line 26: `class FakeBuffer(np.ndarray):` This defines class FakeBuffer, which groups data and methods for fake buffer. Instances exist in host memory and organize the flow that checks neuraedge pynq behavior independently with Python assertions.

Line 27: `def __new__(cls, shape, dtype):` This starts function ___new___. Its indented body performs new as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 28: `return np.zeros(shape, dtype=dtype).view(cls)` This returns ‘np.zeros(shape, dtype=dtype).view(cls)‘ to the caller of the function ___new___, which handles new, ending this invocation. The important operands are zeros: the value named ’zeros’ in this block; shape: the value named ’shape’ in this block; dtype: the value named ’dtype’ in this block.

Line 30: `def flush(self):` This starts function flush. Its indented body performs flush as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 31: `pass` Inside the function flush, which handles flush, this statement advances the host implementation that checks neuraedge pynq behavior independently with Python assertions. The important operands are pass: the value named ’pass’ in this block.

Line 33: `def invalidate(self):` This starts function invalidate. Its indented body performs invalidate as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 34: `pass` Inside the function invalidate, which handles invalidate, this statement advances the host implementation that checks neuraedge pynq behavior independently with Python assertions. The important operands are pass: the value named ’pass’ in this block.

Line 36: `def freebuffer(self):` This starts function freebuffer. Its indented body performs freebuffer as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 37: `self.freed = True` This assigns ‘True‘ to ‘self.freed ‘ in host memory. Here freed means the value named ’freed’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function freebuffer, which handles freebuffer.

Line 40: `class FakeChannel:` This defines class FakeChannel, which groups data and methods for fake channel. Instances exist in host memory and organize the flow that checks neuraedge pynq behavior independently with Python assertions.

Line 41: `def __init__(self, events, name, receive_words=()):` This starts function ___init___. Its indented body performs init as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 42: `self.events = events` This assigns ‘events‘ to ‘self.events ‘ in host memory. Here events means the value named ’events’ in this block. The important operands are events: the value named ’events’ in this block. Context: the function ___init___, which handles init.

Line 43: `self.name = name` This assigns ‘name‘ to ‘self.name ‘ in host memory. Here name means the value named ’name’ in this block. The important operands are name: the value named ’name’ in this block. Context: the function ___init___, which handles init.

Line 44: `self.receive_words = receive_words` This assigns ‘receive_words‘ to ‘self.receive_words ‘ in host memory. Here receive_words means the value named ’receive words’ in this block. The important operands are receive_words: the value named ’receive words’ in this block. Context: the function ___init___, which handles init.

Line 45: `self.idle = True` This assigns ‘True‘ to ‘self.idle ‘ in host memory. Here idle means the value named ’idle’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 47: `def transfer(self, buffer, nbytes):` This starts function transfer. Its indented body performs transfer as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 48: `self.events.append((self.name, "transfer", nbytes))` This calls ‘self.events.append‘ with ‘(self.name, "transfer", nbytes)‘ inside the function transfer, which handles transfer. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 49: `if self.name == "recv":` This executes the following indented host-code branch only when ‘self.name == "recv"‘ is true. The important operands are name: the value named ’name’ in this block; recv: the value named ’recv’ in this block. Context: the function transfer, which handles transfer.

Line 50: `buffer[:len(self.receive_words)] = self.receive_words` This assigns ‘self.receive_words‘ to ‘buffer[‘ in host memory. Here buffer means local storage that decouples producer and consumer timing. The important operands are receive_words: the value named ’receive words’ in this block. Context: the function transfer, which handles transfer.

Line 52: `def wait(self):` This starts function wait. Its indented body performs wait as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 53: `self.events.append((self.name, "wait"))` This calls ‘self.events.append‘ with ‘(self.name, "wait")‘ inside the function wait, which handles wait. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 56: `class FakeDma:` This defines class FakeDma, which groups data and methods for fake dma. Instances exist in host memory and organize the flow that checks neuraedge pynq behavior independently with Python assertions.

Line 57: `def __init__(self, events, receive_words=()):` This starts function ___init___. Its indented body performs init as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 58: `self.sendchannel = FakeChannel(events, "send")` This assigns ‘FakeChannel(events, "send")‘ to ‘self.sendchannel ‘ in host memory. Here sendchannel means the value named ’sendchannel’ in this block. The important operands are FakeChannel: the value named ’fake channel’ in this block; events: the value named ’events’ in this block; send: the value named ’send’ in this block. Context: the function ___init___, which handles init.

Line 59: `self.recvchannel = FakeChannel(events, "recv", receive_words)` This assigns ‘FakeChannel(events, "recv", receive_words)‘ to ‘self.recvchannel ‘ in host memory. Here recvchannel means the value named ’recvchannel’ in this block. The important operands are FakeChannel: the value named ’fake channel’ in this block; events: the value named ’events’ in this block; recv: the value named ’recv’ in this block. Context: the function ___init___, which handles init.

Line 62: `class PynqDriverTests(unittest.TestCase):` This defines class PynqDriverTests, which groups data and methods for pynq driver tests. Instances exist in host memory and organize the flow that checks neuraedge pynq behavior independently with Python assertions.

Line 63: `def make_driver(self, *, receive_words=(), error=False):` This starts function make_driver. Its indented body performs make driver as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 64: `events = []` This assigns ‘[]‘ to ‘events ‘ in host memory. Here events means the value named ’events’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function make_driver, which handles make driver.

Line 65: `mmio = FakeMmio(events, error=error)` This assigns ‘FakeMmio(events, error=error)‘ to ‘mmio ‘ in host memory. Here mmio means the value named ’mmio’ in this block. The important operands are FakeMmio: the value named ’fake mmio’ in this block; events: the value named ’events’ in this block; error: the value named ’error’ in this block. Context: the function make_driver, which handles make driver.

Line 66: `dma = FakeDma(events, receive_words)` This assigns ‘FakeDma(events, receive_words)‘ to ‘dma ‘ in host memory. Here dma means the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. The important operands are FakeDma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; events: the value named ’events’ in this block; receive_words: the value named ’receive words’ in this block. Context: the function make_driver, which handles make driver.

Line 67: `driver = NeuraEdgeDriver(mmio, dma, lambda *, shape, dtype: FakeBuffer(shape, dtype))` This assigns ‘NeuraEdgeDriver(mmio, dma, lambda *, shape, dtype: FakeBuffer(shape, dtype))‘ to ‘driver ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are NeuraEdgeDriver: the value named ’neura edge driver’ in this block; mmio: the value named ’mmio’ in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. Context: the function make_driver, which handles make driver.

Line 68: `return driver, events` This returns ‘driver, events‘ to the caller of the function make_driver, which handles make driver, ending this invocation. The important operands are driver: the value named ’driver’ in this block; events: the value named ’events’ in this block.

Line 70: `def test_identity_and_metadata(self):` This starts function test_identity_and_metadata. Its indented body performs test identity and metadata as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 71: `driver, _ = self.make_driver()` This assigns ‘self.make_driver()‘ to ‘driver, _ ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are make_driver: the value named ’make driver’ in this block. Context: the function test_identity_and_metadata, which handles test identity and metadata.

Line 72: `self.assertEqual(driver.target_clock_hz, 200000000)` This calls ‘self.assertEqual‘ with ‘driver.target_clock_hz, 200000000‘ inside the function test_identity_and_metadata, which handles test identity and metadata. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 73: `self.assertEqual(driver.capabilities, 0x1FF)` This calls ‘self.assertEqual‘ with ‘driver.capabilities, 0x1FF‘ inside the function test_identity_and_metadata, which handles test identity and metadata. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 75: `def test_performance_snapshot_decodes_hardware_registers(self):` This starts function test_performance_sna
Its indented body performs test performance snapshot decodes hardware registers as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 76: `driver, _ = self.make_driver()` This assigns ‘self.make_driver()‘ to ‘driver, _ ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are make_driver: the value named ’make driver’ in this block. Context: the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers.

Line 77: `driver.mmio.registers.update({` This statement continues through source line 101; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 78: `0x38: (29 << 16) | 31,` This is continuation line 2 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x38: (29 << 16) | 31,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38:

(29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 79: `0x3C: 17,` This is continuation line 3 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x3C: 17,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 80: `0x40: 1234,` This is continuation line 4 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x40: 1234,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 81: `0x44: 14,` This is continuation line 5 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x44: 14,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 82: `0x48: 0x89ABCDEF,` This is continuation line 6 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x48: 0x89ABCDEF,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 83: `0x4C: 0x01234567,` This is continuation line 7 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x4C: 0x01234567,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 84: `0x50: 0x76543210,` This is continuation line 8 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x50: 0x76543210,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58:

222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 85: `0x54: 0xFEDCBA98,` This is continuation line 9 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x54: 0xFEDCBA98,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 86: `0x58: 222,` This is continuation line 10 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x58: 222,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 87: `0x5C: 0xFFFFFFF9,` This is continuation line 11 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x5C: 0xFFFFFFF9,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 88: `0x60: 81,` This is continuation line 12 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x60: 81,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 89: `0x64: 9,` This is continuation line 13 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x64: 9,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 90: `0x68: 333,` This is continuation line 14 of the 25-line logical statement that begins at source line 77. Its exact fragment is ‘0x68: 333,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘driver.mmio.registers.update‘ with ‘{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }‘ inside the function

test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 91: `0x6C: 0xFFFFFFF5,` This is continuation line 15 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x6C: 0xFFFFFFF5,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 92: `0x70: 0xABCDEF,` This is continuation line 16 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x70: 0xABCDEF,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 93: `0x74: 0x11111111,` This is continuation line 17 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x74: 0x11111111,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 94: `0x78: 0x22222222,` This is continuation line 18 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x78: 0x22222222,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 95: `0x7C: 44,` This is continuation line 19 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x7C: 44,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 96: `0x80: 256,` This is continuation line 20 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x80: 256,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 97: `0x84: 0x12345678,` This is continuation line 21 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x84: 0x12345678,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 98: `0x88: 0x87654321,` This is continuation line 22 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x88: 0x87654321,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 99: `0x94: 19,` This is continuation line 23 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x94: 19,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 100: `0x98: 9876,` This is continuation line 24 of the 25-line logical statement that begins at source line 77. Its exact fragment is '0x98: 9876,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 101: `})` This is continuation line 25 of the 25-line logical statement that begins at source line 77. Its exact fragment is '})'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'driver.mmio.registers.update' with '{ 0x38: (29 << 16) | 31, 0x3C: 17, 0x40: 1234, 0x44: 14, 0x48: 0x89ABCDEF, 0x4C: 0x01234567, 0x50: 0x76543210, 0x54: 0xFEDCBA98, 0x58: 222, 0x5C: 0xFFFFFFF9, 0x60: 81, 0x64: 9, 0x68: 333, 0x6C: 0xFFFFFFF5, 0x70: 0xABCDEF, 0x74: 0x11111111, 0x78: 0x22222222, 0x7C: 44, 0x80: 256, 0x84: 0x12345678, 0x88: 0x87654321, 0x94: 19, 0x98: 9876, }' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 102: `snapshot = driver.performance_snapshot()` This assigns 'driver.performance_snapshot()' to 'snapshot ' in host memory. Here snapshot means the value named 'snapshot' in this block. The important operands are driver: the value named 'driver' in this block; performance_snapshot: the value named 'performance snapshot' in this block. Context: the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers.

Line 103: `self.assertEqual(snapshot.tensor_loaded_activations, 31)` This calls 'self.assertEqual' with 'snapshot.tensor_loaded_activations, 31' inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 104: `self.assertEqual(snapshot.tensor_loaded_weights, 29)` This calls ‘self.assertEqual‘ with ‘snapshot.tensor_load 29‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 105: `self.assertEqual(snapshot.tensor_active_issues, 17)` This calls ‘self.assertEqual‘ with ‘snapshot.tensor_active 17‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 106: `self.assertEqual(snapshot.dense_macs, 0x0123456789ABCDEF)` This calls ‘self.assertEqual‘ with ‘snapshot.dense_macs, 0x0123456789ABCDEF‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 107: `self.assertEqual(snapshot.spike_adds, 0xFEDCBA9876543210)` This calls ‘self.assertEqual‘ with ‘snapshot.spike_adds, 0xFEDCBA9876543210‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 108: `self.assertEqual(snapshot.layernorm_mean, -7)` This calls ‘self.assertEqual‘ with ‘snapshot.layernorm_mean, -7‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 109: `self.assertEqual(snapshot.softmax_maximum, -11)` This calls ‘self.assertEqual‘ with ‘snapshot.softmax_maximu -11‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 110: `self.assertEqual(snapshot.lif_total_spikes, 0x2222222211111111)` This calls ‘self.assertEqual‘ with ‘snapshot.lif_total_spikes, 0x2222222211111111‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 111: `self.assertEqual(snapshot.lut_loaded_entries, 256)` This calls ‘self.assertEqual‘ with ‘snapshot.lut_loaded_en 256‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 112: `self.assertEqual(snapshot.training_tiles, 19)` This calls ‘self.assertEqual‘ with ‘snapshot.training_tiles, 19‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 113: `self.assertEqual(snapshot.training_cycles, 9876)` This calls ‘self.assertEqual‘ with ‘snapshot.training_cycles, 9876‘ inside the function test_performance_snapshot_decodes_hardware_registers, which handles test performance snapshot decodes hardware registers. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 115: `def test_receive_is_armed_before_submit_and_send(self):` This starts function test_receive_is_armed_before Its indented body performs test receive is armed before submit and send as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 116: `driver, events = self.make_driver(receive_words=(0x1234, 0x5678))` This assigns ‘self.make_driver(receive_ 0x5678))‘ to ‘driver, events ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are make_driver: the value named ’make driver’ in this block; receive_words: the value named ’receive words’ in this block. Context: the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send.

Line 117: `results = driver.execute(` This statement continues through source line 121; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘driver.execute(clear_status_command(), [StreamBeat(1, False), StreamBeat(2, True)], output_beats=2,)‘ to ‘results ‘ in host memory. Here results means the value named ’results’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send.

Line 118: `clear_status_command(),` This is continuation line 2 of the 5-line logical statement that begins at source line 117. Its exact fragment is ‘clear_status_command(),‘. The important operands are clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘driver.execute(clear_status_command(), [StreamBeat(1, False), StreamBeat(2, True)], output_beats=2,)‘ to ‘results ‘ in host memory. Here results means the value named ’results’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send.

Line 119: `[StreamBeat(1, False), StreamBeat(2, True)],` This is continuation line 3 of the 5-line logical statement that begins at source line 117. Its exact fragment is ‘[StreamBeat(1, False), StreamBeat(2, True)],‘. The important operands are StreamBeat: the value named ’stream beat’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(clear_status_command(), [StreamBeat(1, False), StreamBeat(2, True)], output_beats=2,)‘ to ‘results ‘ in host memory. Here results means the value named ’results’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send.

Line 120: `output_beats=2,` This is continuation line 4 of the 5-line logical statement that begins at source line 117. Its exact fragment is ‘output_beats=2,‘. The important operands are output_beats: the value named ’output beats’ in this block. Read together, the complete operation is: This assigns ‘driver.execute(clear_status_command(), [StreamBeat(1, False), StreamBeat(2, True)], output_beats=2,)‘ to ‘results ‘ in host memory. Here results means the value named ’results’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send.

Line 121: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 117. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘driver.execute(clear_status_command(), [StreamBeat(1, False), StreamBeat(2, True)], output_beats=2,)‘ to ‘results ‘ in host memory. Here results means the value named ’results’ in this block. The important operands are driver: the value named ’driver’ in this block; execute: the value named ’execute’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send.

Line 122: `self.assertEqual(results, (0x1234, 0x5678))` This calls ‘self.assertEqual‘ with ‘results, (0x1234, 0x5678)‘ inside the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 123: `receive_index = events.index(("recv", "transfer", 16))` This assigns ‘events.index(("recv", "transfer", 16))‘ to ‘receive_index ‘ in host memory. Here receive_index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are events: the value named ’events’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; recv: the value named ’recv’ in this block. Context: the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send.

Line 124: `submit_index = next(index for index, event in enumerate(events)` This statement continues through source line 125; the explanation uses the complete expression rather than only the first visual fragment. Inside the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send, this statement advances the host implementation that checks neuraedge pynq behavior independently with Python assertions. The important operands are submit_index: an integer selecting one tensor element, lane, bank, or state entry; next: the value named ’next’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry.

Line 125: `if event[0] == "mmio" and event[1] == 0 and event[2] & 1)` This is continuation line 2 of the 2-line logical statement that begins at source line 124. Its exact fragment is ‘if event[0] == "mmio" and event[1] == 0 and event[2] & 1)‘. The important operands are event: the value named ’event’ in this block; mmio: the value named ’mmio’ in this block; and: the value named ’and’ in this block. Read together, the complete operation is: Inside the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send, this statement advances the host implementation that checks neuraedge pynq behavior independently with Python assertions. The important operands are submit_index: an integer selecting one tensor element, lane, bank, or state entry; next: the value named ’next’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry.

Line 126: `send_index = events.index(("send", "transfer", 16))` This assigns ‘events.index(("send", "transfer", 16))‘ to ‘send_index ‘ in host memory. Here send_index means an integer selecting one tensor element, lane, bank, or state entry. The important operands are events: the value named ’events’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; send: the value named ’send’ in this block. Context: the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send.

Line 127: `self.assertLess(receive_index, submit_index)` This calls ‘self.assertLess‘ with ‘receive_index, submit_index‘ inside the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 128: `self.assertLess(submit_index, send_index)` This calls ‘self.assertLess‘ with ‘submit_index, send_index‘ inside the function test_receive_is_armed_before_submit_and_send, which handles test receive is armed before submit and send. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 130: `def test_rejects_bad_last_and_hardware_error(self):` This starts function test_rejects_bad_last_and_hardw Its indented body performs test rejects bad last and hardware error as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 131: `driver, _ = self.make_driver()` This assigns ‘self.make_driver()‘ to ‘driver, _ ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are make_driver: the value named ’make driver’ in this block. Context: the function test_rejects_bad_last_and_hardware_error, which handles test rejects bad last and hardware error.

Line 132: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_rejects_bad_last_and_hardware_error, which handles test rejects bad last and hardware error raises an error.

Line 133: `driver.execute(clear_status_command(), [StreamBeat(1, False)])` This calls ‘driver.execute‘ with ‘clear_status_command(), [StreamBeat(1, False)]‘ inside the function test_rejects_bad_last_and_hardware_error, which handles test rejects bad last and hardware error. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 134: `driver, _ = self.make_driver(error=True)` This assigns ‘self.make_driver(error=True)‘ to ‘driver, _ ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are make_driver: the value named ’make driver’ in this block; error: the value named ’error’ in this block. Context: the function test_rejects_bad_last_and_hardware_error, which handles test rejects bad last and hardware error.

Line 135: `with self.assertRaises(NeuraEdgeHardwareError):` This opens a managed resource scope for ‘self.assertRaises(Ne so files, tracing, or temporary state are finalized even if the function test_rejects_bad_last_and_hardware_error, which handles test rejects bad last and hardware error raises an error.

Line 136: `driver.execute(clear_status_command())` This calls ‘driver.execute‘ with ‘clear_status_command()‘ inside the function test_rejects_bad_last_and_hardware_error, which handles test rejects bad last and hardware error. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 138: `def test_descriptor_program_uses_one_dma_pair(self):` This starts function test_descriptor_program_uses_o Its indented body performs test descriptor program uses one dma pair as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 139: `driver, events = self.make_driver(receive_words=(0xAA, 0xBB))` This assigns ‘self.make_driver(receive_word 0xBB))‘ to ‘driver, events ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are make_driver: the value named ’make driver’ in this block; receive_words: the value named ’receive words’ in this block. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 140: `driver.mmio.registers[0x8C] = 2` This assigns ‘2‘ to ‘driver.mmio.registers[0x8C] ‘ in host memory. Here registers means the value named ’registers’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 141: `records = (` This statement continues through source line 148; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0x11, False), StreamBeat(0x22, True)), 0,), ProgramRecord(clear_status_command(), (), 2),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 142: `ProgramRecord(` This is continuation line 2 of the 8-line logical statement that begins at source line 141. Its exact fragment is ‘ProgramRecord(‘. The important operands are ProgramRecord: the value named ’program record’ in this block. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0x11, False), StreamBeat(0x22, True)), 0,), ProgramRecord(clear_status_command(), (), 2),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 143: `clear_status_command(),` This is continuation line 3 of the 8-line logical statement that begins at source line 141. Its exact fragment is ‘clear_status_command(),‘. The important operands are clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0x11, False), StreamBeat(0x22, True)), 0,), ProgramRecord(clear_status_command(), (), 2),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 144: `(StreamBeat(0x11, False), StreamBeat(0x22, True)),` This is continuation line 4 of the 8-line logical statement that begins at source line 141. Its exact fragment is ‘(StreamBeat(0x11, False), StreamBeat(0x22, True)),‘. The important operands are StreamBeat: the value named ’stream beat’ in this block. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0x11, False), StreamBeat(0x22, True)), 0,), ProgramRecord(clear_status_command(), (), 2),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 145: `0,` This is continuation line 5 of the 8-line logical statement that begins at source line 141. Its exact fragment is ‘0,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0x11, False), StreamBeat(0x22, True)), 0,), ProgramRecord(clear_status_command(), (), 2),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 146: `),` This is continuation line 6 of the 8-line logical statement that begins at source line 141. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0x11, False), StreamBeat(0x22, True)), 0,), ProgramRecord(clear_status_command(), (), 2),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 147: `ProgramRecord(clear_status_command(), (), 2),` This is continuation line 7 of the 8-line logical statement that begins at source line 141. Its exact fragment is ‘ProgramRecord(clear_status_command(), (), 2),‘. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0x11, False), StreamBeat(0x22, True)), 0,), ProgramRecord(clear_status_command(), (), 2),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 148: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 141. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘(ProgramRecord(clear_status_command(), (StreamBeat(0x11, False), StreamBeat(0x22, True)), 0,), ProgramRecord(clear_status_command(), (), 2),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named ’program record’ in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; StreamBeat: the value named ’stream beat’ in this block. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 149: `result = driver.execute_program(records)` This assigns ‘driver.execute_program(records)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are driver: the value named ’driver’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start. Context: the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

Line 150: `self.assertEqual(result, (0xAA, 0xBB))` This calls ‘self.assertEqual‘ with ‘result, (0xAA, 0xBB)‘ inside the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair.

The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 151: `self.assertIn(("recv", "transfer", 16), events)` This calls ‘self.assertIn‘ with ‘("recv", "transfer", 16), events‘ inside the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 152: `self.assertIn(("send", "transfer", 64), events)` This calls ‘self.assertIn‘ with ‘("send", "transfer", 64), events‘ inside the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 153: `self.assertIn(("mmio", 0x90, 2), events)` This calls ‘self.assertIn‘ with ‘("mmio", 0x90, 2), events‘ inside the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 154: `self.assertTrue(any(event[0] == "mmio" and event[1] == 0` This statement continues through source line 155; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertTrue‘ with ‘any(event[0] == "mmio" and event[1] == 0 and event[2] & 4 for event in events)‘ inside the function test_descriptor_program_u which handles test descriptor program uses one dma pair. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 155: `and event[2] & 4 for event in events))` This is continuation line 2 of the 2-line logical statement that begins at source line 154. Its exact fragment is ‘and event[2] & 4 for event in events))‘. The important operands are and: the value named ’and’ in this block; event: the value named ’event’ in this block; for: the value named ’for’ in this block. Read together, the complete operation is: This calls ‘self.assertTrue‘ with ‘any(event[0] == "mmio" and event[1] == 0 and event[2] & 4 for event in events)‘ inside the function test_descriptor_program_uses_one_dma_pair, which handles test descriptor program uses one dma pair. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 157: `def test_descriptor_program_partitions_at_dma_packet_boundary(self):` This starts function test_descriptor Its indented body performs test descriptor program partitions at dma packet boundary as part of the file that checks neuraedge pynq behavior independently with Python assertions; it runs only when called.

Line 158: `driver, events = self.make_driver(receive_words=(0xAA,))` This assigns ‘self.make_driver(receive_words=(0x to ‘driver, events ‘ in host memory. Here driver means the value named ’driver’ in this block. The important operands are make_driver: the value named ’make driver’ in this block; receive_words: the value named ’receive words’ in this block. Context: the function test_descriptor_program_partitions_at_dma_packet_boundary, which handles test descriptor program partitions at dma packet boundary.

Line 159: `driver.MAX_DMA_TRANSFER_BYTES = 32` This assigns ‘32‘ to ‘driver.MAX_DMA_TRANSFER_BYTES ‘ in host memory. Here MAX_DMA_TRANSFER_BYTES means the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic. The expression is a literal or structural operator and has no hidden software state. Context: the function test_descriptor_program_partitions_at_dma_packet_boundary, which handles test descriptor program partitions at dma packet boundary.

Line 160: `driver.mmio.registers[0x8C] = 1` This assigns ‘1‘ to ‘driver.mmio.registers[0x8C] ‘ in host memory. Here registers means the value named ’registers’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_descriptor_program_partitions_at_dma_packet_boundary, which handles test descriptor program partitions at dma packet boundary.

Line 161: `records = (` This statement continues through source line 164; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘(ProgramRecord(clear_status_command(), (), 1), ProgramRecord(clear_status_command(), (), 1),)‘ to ‘records ‘ in host memory. Here records means the number of descriptor

records accepted since program start. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_descriptor_program_partitions_at_dma_packet_boundary, which handles test descriptor program partitions at dma packet boundary.

Line 162: `ProgramRecord(clear_status_command(), (), 1),` This is continuation line 2 of the 4-line logical statement that begins at source line 161. Its exact fragment is 'ProgramRecord(clear_status_command(), (), 1),'. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns '(ProgramRecord(clear_status_command(), (), 1), ProgramRecord(clear_status_command(), (), 1),)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_descriptor_program_partitions_at which handles test descriptor program partitions at dma packet boundary.

Line 163: `ProgramRecord(clear_status_command(), (), 1),` This is continuation line 3 of the 4-line logical statement that begins at source line 161. Its exact fragment is 'ProgramRecord(clear_status_command(), (), 1),'. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This assigns '(ProgramRecord(clear_status_command(), (), 1), ProgramRecord(clear_status_command(), (), 1),)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_descriptor_program_partitions_at which handles test descriptor program partitions at dma packet boundary.

Line 164: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 161. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '(ProgramRecord(clear_status_command(), (), 1), ProgramRecord(clear_status_command(), (), 1),)' to 'records ' in host memory. Here records means the number of descriptor records accepted since program start. The important operands are ProgramRecord: the value named 'program record' in this block; clear_status_command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function test_descriptor_program_part which handles test descriptor program partitions at dma packet boundary.

Line 165: `result = driver.execute_program(records)` This assigns 'driver.execute_program(records)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are driver: the value named 'driver' in this block; execute_program: the value named 'execute program' in this block; records: the number of descriptor records accepted since program start. Context: the function test_descriptor_program_partitions_at_dma_packet_bou which handles test descriptor program partitions at dma packet boundary.

Line 166: `self.assertEqual(result, (0xAA, 0xAA))` This calls 'self.assertEqual' with 'result, (0xAA, 0xAA)' inside the function test_descriptor_program_partitions_at_dma_packet_boundary, which handles test descriptor program partitions at dma packet boundary. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 167: `self.assertEqual(events.count(("send", "transfer", 24)), 2)` This calls 'self.assertEqual' with 'events.count(("send", "transfer", 24)), 2' inside the function test_descriptor_program_partitions_at_dma_packet_boundary, which handles test descriptor program partitions at dma packet boundary. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 168: `self.assertEqual(events.count(("recv", "transfer", 8)), 2)` This calls 'self.assertEqual' with 'events.count(("recv", "transfer", 8)), 2' inside the function test_descriptor_program_partitions_at_dma_packet_boundary, which handles test descriptor program partitions at dma packet boundary. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

Line 171: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks neuraedge pynq behavior independently with Python assertions.

Line 172: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks neuraedge pynq behavior independently with Python assertions. The call advances the host-side job that checks neuraedge pynq behavior independently with Python assertions.

68 tests/test_neuraedge_training.py

Role in the complete system: checks neuraedge training behavior independently with Python assertions.

68.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks neuraedge training behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: BackwardRtlEmulator, FabricOperatorEmulator, FullTrainingEmulator, TrainingCompilerTests. Functions: _signed, ___init___, _round_and_clip, _decode, _pack_dx, _pack_dw, execute, execute_program, ___init___, execute, ___init___, execute, execute_program, setUp, test_arbitrary_linear_shape_uses_exact_ test_affine_augmentation_returns_exact_bias_gradient, test_grouped_convolution_backward_and_fabric_col2im, test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, test_rejects_misaligned_or_out_of_range_operar test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, test_maxpool_backward_matches_first_winner_and_o test_real_spikingformer_manifest_has_no_training_coverage_holes. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

68.2 Complete source

```
from __future__ import annotations

import unittest
from pathlib import Path

import numpy as np

from neuraedge_golden import (
    VectorOperation,
    log_int_softmax_q15,
    round_shift_signed,
    saturate_signed,
    sgd_momentum_step,
    vector_alu,
)
from neuraedge_protocol import LinearGradientMode, Opcode
from neuraedge_training import (
    compile_affine_backward,
    compile_conv2d_backward,
    compile_linear_backward,
    build_spikingformer_training_coverage,
    execute_lif_bptt,
    execute_int8_weight_projection,
    execute_maxpool2d_backward,
    execute_sgdw_update,
    lif_bptt_fixed,
    maxpool2d_backward_fixed,
    sigmoid_surrogate_lut_q15,
    surrogate_lut_addresses,
)
from tests.test_neuraedge_compiler import TensorTransactionEmulator


def _signed(value: int, bits: int) -> int:
    masked = value & ((1 << bits) - 1)
    return masked - (1 << bits) if masked & (1 << (bits - 1)) else masked


class BackwardRtlEmulator:
    """Independent command-level model of retained raw gradient sums."""

    def __init__(self) -> None:
        self.input_accumulator: np.ndarray | None = None
        self.weight_accumulator: np.ndarray | None = None

    @staticmethod
```

```
    def _round_and_clip(values: np.ndarray, shift: int, bits: int) -> np.ndarray:
        return saturate_signed(round_shift_signed(values, shift), bits)

    @staticmethod
    def _decode(command, beats):
        value = command.value
        rows = (value >> 4) & 0xF
        inputs = (value >> 8) & 0xF
        outputs = (value >> 12) & 0xF
        mode = LinearGradientMode((value >> 26) & 0x3)
        words = [beat.data for beat in beats]
        cursor = 0
        x = np.zeros((rows, inputs), dtype=np.int64)
        w = np.zeros((inputs, outputs), dtype=np.int64)
        if mode != LinearGradientMode.INPUT:
            for row in range(rows):
                packed = words[cursor] | (words[cursor + 1] << 64)
                cursor += 2
                for lane in range(inputs):
                    x[row, lane] = _signed(packed >> (16 * lane), 16)
        if mode != LinearGradientMode.WEIGHT:
            for row in range(inputs):
                packed = words[cursor] | (words[cursor + 1] << 64)
                cursor += 2
                for lane in range(outputs):
                    w[row, lane] = _signed(packed >> (16 * lane), 16)
        dy = np.zeros((rows, outputs), dtype=np.int64)
        for row in range(rows):
            packed = words[cursor] | (words[cursor + 1] << 64)
            cursor += 2
            for lane in range(outputs):
                dy[row, lane] = _signed(packed >> (16 * lane), 16)
        if cursor != len(words):
            raise AssertionError("compiler emitted an unexpected backward stream length")
        return rows, inputs, outputs, mode, x, w, dy

    @staticmethod
    def _pack_dx(values: np.ndarray) -> tuple[int, ...]:
        words: list[int] = []
        for row in values:
            padded = list(map(int, row)) + [0] * (8 - len(row))
            for lane in range(0, 8, 2):
                words.append(
                    (padded[lane] & 0xFFFFFFFF)
                    | ((padded[lane + 1] & 0xFFFFFFFF) << 32)
                )
        return tuple(words)

    @staticmethod
    def _pack_dw(values: np.ndarray) -> tuple[int, ...]:
        words: list[int] = []
        for row in values:
            padded = list(map(int, row)) + [0] * (8 - len(row))
            packed = sum(
                (item & 0xFFFFFF) << (24 * lane)
                for lane, item in enumerate(padded)
            )
            words.extend(
                (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3)
            )
        return tuple(words)

    def execute(
        self,
        command,
        input_beats=(),
        *,
        output_beats=0,
        timeout_seconds=5.0,
    ) -> tuple[int, ...]:
        del timeout_seconds
        rows, inputs, outputs, mode, x, w, dy = self._decode(command, input_beats)
        accumulate = bool((command.value >> 28) & 1)
        suppress = bool((command.value >> 29) & 1)
        dx_shift = (command.value >> 16) & 0x1F
        dw_shift = (command.value >> 21) & 0x1F
        result: tuple[int, ...]
        if mode == LinearGradientMode.INPUT:
            partial = np.zeros((rows, inputs), dtype=np.int64)
            for row in range(rows):
                for input_index in range(inputs):
                    partial[row, input_index] = sum(
                        int(dy[row, output]) * int(w[input_index, output])
                        for output in range(outputs)
                    )
            if accumulate:
                if self.input_accumulator is None:
```

```
                    raise AssertionError("input accumulation began without retained state")
                self.input_accumulator += partial
            else:
                self.input_accumulator = partial
            if suppress:
                result = ()
            else:
                result = self._pack_dx(
                    self._round_and_clip(self.input_accumulator, dx_shift, 32)
                )
                self.input_accumulator = None
        elif mode == LinearGradientMode.WEIGHT:
            partial = np.zeros((inputs, outputs), dtype=np.int64)
            for input_index in range(inputs):
                for output in range(outputs):
                    partial[input_index, output] = sum(
                        int(x[row, input_index]) * int(dy[row, output])
                        for row in range(rows)
                    )
            if accumulate:
                if self.weight_accumulator is None:
                    raise AssertionError("weight accumulation began without retained state")
                self.weight_accumulator += partial
            else:
                self.weight_accumulator = partial
            if suppress:
                result = ()
            else:
                result = self._pack_dw(
                    self._round_and_clip(self.weight_accumulator, dw_shift, 24)
                )
                self.weight_accumulator = None
        else:
            raise AssertionError("the tiled compiler must use one-gradient passes")
        if len(result) != output_beats:
            raise AssertionError(f"emulator produced {len(result)} words, expected {output_beats}")
        return result

    def execute_program(self, records, *, timeout_seconds=30.0) -> tuple[int, ...]:
        del timeout_seconds
        output: list[int] = []
        for record in records:
            output.extend(self.execute(
                record.command,
                record.input_beats,
                output_beats=record.output_beats,
            ))
        return tuple(output)


class FabricOperatorEmulator:
    def __init__(self) -> None:
        self.lut = np.zeros(256, dtype=np.int64)
        self.lif_membrane = np.zeros(32, dtype=np.int64)
        self.lif_timestep = 0
        self.command_count = 0

    def execute(
        self,
        command,
        input_beats=(),
        *,
        output_beats=0,
        timeout_seconds=5.0,
    ) -> tuple[int, ...]:
        del timeout_seconds
        self.command_count += 1
        words = [beat.data for beat in input_beats]
        if command.opcode == Opcode.ACTIVATION_LUT:
            suboperation = (command.value >> 4) & 0xF
            if suboperation == 1:
                for word in words:
                    self.lut[word & 0xFF] = _signed(word >> 8, 16)
                result: tuple[int, ...] = ()
            elif suboperation == 2:
                indices = [(words[0] >> (8 * lane)) & 0xFF for lane in range(8)]
                values = [int(self.lut[index]) for index in indices]
                result = tuple(
                    sum((values[word * 4 + lane] & 0xFFFF) << (16 * lane)
                        for lane in range(4))
                    for word in range(2)
                )
            else:
                self.lut.fill(0)
                result = ()
        elif command.opcode == Opcode.VECTOR_ALU:
            operation = VectorOperation((command.value >> 4) & 0xF)
```

```
            multiplier = _signed(command.value >> 8, 16)
            shift_bits = (command.value >> 24) & 0x1F
            right_shift = _signed(shift_bits, 5) if operation == VectorOperation.FLEX_REQUANTIZE else shift_bits
            output_zero_point = _signed(command.value >> 29, 16)
            threshold = _signed(command.value >> 45, 32)
            surrogate_window = (command.value >> 77) & 0xFFFFFFFF
            clamp_minimum = _signed(words[0], 32)
            clamp_maximum = _signed(words[0] >> 32, 32)
            operand_a = np.array([
                _signed(words[1 + lane // 2] >> (32 * (lane % 2)), 32)
                for lane in range(8)
            ], dtype=np.int64)
            operand_b = np.array([
                _signed(words[5 + lane // 2] >> (32 * (lane % 2)), 32)
                for lane in range(8)
            ], dtype=np.int64)
            values = vector_alu(
                operation,
                operand_a,
                operand_b,
                multiplier_q15=multiplier,
                right_shift=right_shift,
                output_zero_point=output_zero_point,
                threshold=threshold,
                surrogate_window=surrogate_window,
                clamp_minimum=clamp_minimum,
                clamp_maximum=clamp_maximum,
            )
            result = tuple(
                (int(values[lane]) & 0xFFFFFFFF)
                | ((int(values[lane + 1]) & 0xFFFFFFFF) << 32)
                for lane in range(0, 8, 2)
            )
        elif command.opcode == Opcode.SGD_UPDATE:
            weights = np.array([_signed(word, 16) for word in words], dtype=np.int64)
            gradients = np.array([_signed(word >> 16, 24) for word in words], dtype=np.int64)
            velocities = np.array([_signed(word >> 40, 24) for word in words], dtype=np.int64)
            learning_rate = (command.value >> 4) & 0x3FFFF
            momentum = (command.value >> 22) & 0x1FFFF
            weight_decay = (command.value >> 39) & 0x1FFFF
            update_right_shift = (command.value >> 56) & 0x1F
            updated_weight, updated_velocity = sgd_momentum_step(
                weights,
                gradients,
                velocities,
                learning_rate,
                momentum,
                weight_decay,
                update_right_shift,
            )
            result = tuple(
                (int(updated_weight[lane]) & 0xFFFF)
                | ((int(updated_velocity[lane]) & 0xFFFFFF) << 16)
                for lane in range(8)
            )
        elif command.opcode == Opcode.LOG_SOFTMAX:
            count = (command.value >> 4) & 0x1FF
            difference_shift = (command.value >> 13) & 0x1F
            scores = np.asarray(
                [_signed(word, 24) for word in words[:count]], dtype=np.int64)
            probabilities = log_int_softmax_q15(scores, difference_shift)
            result = tuple(
                int(probabilities[index])
                | (index << 16)
                | (int(index == count - 1) << 25)
                for index in range(count)
            )
        elif command.opcode == Opcode.LIF_STEP:
            leak_shift = (command.value >> 4) & 0x1F
            threshold = _signed(command.value >> 9, 24)
            reset = _signed(command.value >> 33, 24)
            if (command.value >> 57) & 1:
                self.lif_membrane.fill(reset)
                self.lif_timestep = 0
            currents = np.asarray([
                _signed(word >> (16 * lane), 16)
                for word in words for lane in range(4)
            ], dtype=np.int64)
            self.lif_membrane = saturate_signed(
                self.lif_membrane - (self.lif_membrane >> leak_shift) + currents,
                24,
            )
            pre_reset = self.lif_membrane.copy()
            fired = pre_reset >= threshold
            self.lif_membrane = np.where(fired, reset, pre_reset)
            self.lif_timestep += 1
            spike_word = sum(int(fired[lane]) << lane for lane in range(32))
```

```
            output = [spike_word | (self.lif_timestep << 32)]
            if (command.value >> 58) & 1:
                packed = sum(
                    (int(pre_reset[lane]) & 0xFFFFFF) << (24 * lane)
                    for lane in range(32)
                )
                output.extend(
                    (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF
                    for word in range(12)
                )
            result = tuple(output)
        elif command.opcode == Opcode.CLEAR_STATUS:
            result = ()
        else:
            raise AssertionError(f"unexpected fabric opcode {command.opcode}")
        if len(result) != output_beats:
            raise AssertionError(f"fabric emulator returned {len(result)} words, expected {output_beats}")
        return result


class FullTrainingEmulator:
    def __init__(self) -> None:
        self.backward = BackwardRtlEmulator()
        self.fabric = FabricOperatorEmulator()
        self.tensor = TensorTransactionEmulator()

    def execute(self, command, input_beats=(), *, output_beats=0, timeout_seconds=5.0):
        if command.opcode == Opcode.TRAIN_LINEAR_BACKWARD:
            return self.backward.execute(
                command,
                input_beats,
                output_beats=output_beats,
                timeout_seconds=timeout_seconds,
            )
        if command.opcode in {
            Opcode.LOAD_ACTIVATIONS,
            Opcode.LOAD_WEIGHTS_OUTER,
            Opcode.LOAD_WEIGHTS_DOT8,
            Opcode.COMPUTE_DENSE_OUTER,
            Opcode.COMPUTE_DENSE_DOT8,
            Opcode.COMPUTE_BINARY,
            Opcode.COMPUTE_TERNARY,
        }:
            return self.tensor.execute(
                command,
                input_beats,
                output_beats=output_beats,
                timeout_seconds=timeout_seconds,
            )
        return self.fabric.execute(
            command,
            input_beats,
            output_beats=output_beats,
            timeout_seconds=timeout_seconds,
        )

    def execute_program(self, records, *, timeout_seconds=30.0) -> tuple[int, ...]:
        output: list[int] = []
        for record in records:
            output.extend(self.execute(
                record.command,
                record.input_beats,
                output_beats=record.output_beats,
                timeout_seconds=timeout_seconds,
            ))
        return tuple(output)


class TrainingCompilerTests(unittest.TestCase):
    def setUp(self) -> None:
        self.rng = np.random.default_rng(20260713)

    def test_arbitrary_linear_shape_uses_exact_two_pass_accumulation(self) -> None:
        x = self.rng.integers(-128, 128, size=(13, 11), dtype=np.int64)
        w = self.rng.integers(-32768, 32768, size=(11, 17), dtype=np.int64)
        dy = self.rng.integers(-32768, 32768, size=(13, 17), dtype=np.int64)
        program = compile_linear_backward(
            x,
            w,
            dy,
            input_gradient_shift=4,
            weight_gradient_shift=3,
        )
        self.assertEqual(program.shape, (13, 11, 17))
        self.assertEqual(program.partial_tile_count, 24)
        self.assertEqual(program.mathematical_multiplications, 2 * 13 * 11 * 17)
        self.assertEqual(sum(
```

```
            instruction.mode == LinearGradientMode.INPUT
            for instruction in program.instructions
        ), 12)
        self.assertEqual(sum(
            instruction.mode == LinearGradientMode.WEIGHT
            for instruction in program.instructions
        ), 12)
        for instruction in program.instructions:
            self.assertEqual(
                instruction.record.output_beats == 0,
                instruction.suppress_output,
            )
            self.assertEqual(
                bool((instruction.record.command.value >> 28) & 1),
                instruction.accumulate,
            )

        expected_dx = saturate_signed(round_shift_signed(dy @ w.T, 4), 32)
        expected_dw = saturate_signed(round_shift_signed(x.T @ dy, 3), 24)
        got_dx, got_dw = program.execute(BackwardRtlEmulator())
        np.testing.assert_array_equal(got_dx, expected_dx)
        np.testing.assert_array_equal(got_dw, expected_dw)
        got_dx, got_dw = program.execute_batched(BackwardRtlEmulator())
        np.testing.assert_array_equal(got_dx, expected_dx)
        np.testing.assert_array_equal(got_dw, expected_dw)

    def test_affine_augmentation_returns_exact_bias_gradient(self) -> None:
        x = self.rng.integers(-32, 33, size=(9, 10), dtype=np.int64)
        w = self.rng.integers(-2000, 2001, size=(10, 7), dtype=np.int64)
        bias = self.rng.integers(-3000, 3001, size=7, dtype=np.int64)
        dy = self.rng.integers(-1000, 1001, size=(9, 7), dtype=np.int64)
        program = compile_affine_backward(
            x,
            w,
            bias,
            dy,
            input_gradient_shift=2,
            weight_gradient_shift=1,
        )
        got_dx, got_dw, got_db = program.execute_batched(BackwardRtlEmulator())
        np.testing.assert_array_equal(
            got_dx,
            saturate_signed(round_shift_signed(dy @ w.T, 2), 32),
        )
        np.testing.assert_array_equal(
            got_dw,
            saturate_signed(round_shift_signed(x.T @ dy, 1), 24),
        )
        np.testing.assert_array_equal(
            got_db,
            saturate_signed(round_shift_signed(dy.sum(axis=0), 1), 24),
        )

    def test_grouped_convolution_backward_and_fabric_col2im(self) -> None:
        x = self.rng.integers(-8, 9, size=(1, 4, 4, 5), dtype=np.int64)
        w = self.rng.integers(-20, 21, size=(4, 2, 3, 3), dtype=np.int64)
        bias = self.rng.integers(-30, 31, size=4, dtype=np.int64)
        dy = self.rng.integers(-12, 13, size=(1, 4, 2, 3), dtype=np.int64)
        program = compile_conv2d_backward(
            x,
            w,
            bias,
            dy,
            stride=2,
            padding=1,
            groups=2,
        )
        self.assertEqual(program.output_shape, (1, 4, 2, 3))
        self.assertEqual(program.partial_tile_count, 12)
        got_dx, got_dw, got_db = program.execute_batched(FullTrainingEmulator())

        expected_dx = np.zeros_like(x)
        expected_dw = np.zeros_like(w)
        expected_db = np.zeros_like(bias)
        for output_channel in range(4):
            group = output_channel // 2
            input_start = group * 2
            for output_y in range(2):
                for output_x in range(3):
                    gradient = int(dy[0, output_channel, output_y, output_x])
                    expected_db[output_channel] += gradient
                    for local_input in range(2):
                        for kernel_y in range(3):
                            input_y = output_y * 2 - 1 + kernel_y
                            for kernel_x in range(3):
                                input_x = output_x * 2 - 1 + kernel_x
                                if 0 <= input_y < 4 and 0 <= input_x < 5:
```

```
                                    expected_dw[output_channel, local_input, kernel_y, kernel_x] += (
                                        int(x[0, input_start + local_input, input_y, input_x])
                                        * gradient
                                    )
                                    expected_dx[0, input_start + local_input, input_y, input_x] += (
                                        int(w[output_channel, local_input, kernel_y, kernel_x])
                                        * gradient
                                    )
        np.testing.assert_array_equal(got_dx, expected_dx)
        np.testing.assert_array_equal(got_dw, expected_dw)
        np.testing.assert_array_equal(got_db, expected_db)

    def test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric(self) -> None:
        weights = self.rng.integers(-500, 501, size=(3, 11), dtype=np.int64)
        gradients = self.rng.integers(-(1 << 20), 1 << 20, size=(3, 11), dtype=np.int64)
        velocities = self.rng.integers(-(1 << 20), 1 << 20, size=(3, 11), dtype=np.int64)
        expected_weight, expected_velocity = sgd_momentum_step(
            weights,
            gradients,
            velocities,
            655,
            58982,
            66,
        )
        emulator = FabricOperatorEmulator()
        got_weight, got_velocity = execute_sgdw_update(
            emulator,
            weights,
            gradients,
            velocities,
            learning_rate_q16=655,
            momentum_q16=58982,
            weight_decay_q16=66,
        )
        np.testing.assert_array_equal(got_weight, expected_weight)
        np.testing.assert_array_equal(got_velocity, expected_velocity)
        projected = execute_int8_weight_projection(emulator, got_weight)
        np.testing.assert_array_equal(projected, np.clip(got_weight, -128, 127).astype(np.int8))
        self.assertEqual(emulator.command_count, 10)

    def test_rejects_misaligned_or_out_of_range_operands(self) -> None:
        with self.assertRaises(ValueError):
            compile_linear_backward(np.zeros((2, 3)), np.zeros((4, 5)), np.zeros((2, 5)))
        with self.assertRaises(ValueError):
            compile_linear_backward(np.array([[32768]]), np.array([[1]]), np.array([[1]]))
        with self.assertRaises(ValueError):
            compile_affine_backward(
                np.zeros((1, 1), dtype=np.int64),
                np.zeros((1, 2), dtype=np.int64),
                np.zeros(3, dtype=np.int64),
                np.zeros((1, 2), dtype=np.int64),
            )

    def test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence(self) -> None:
        time_steps = 4
        neurons = 19
        threshold = 1 << 10
        membrane = self.rng.integers(
            threshold - 4096,
            threshold + 4097,
            size=(time_steps, neurons),
            dtype=np.int64,
        )
        spikes = (membrane >= threshold).astype(np.int64)
        grad_spikes = self.rng.integers(
            -(1 << 20), 1 << 20, size=(time_steps, neurons), dtype=np.int64
        )
        grad_membrane = self.rng.integers(
            -(1 << 18), 1 << 18, size=(time_steps, neurons), dtype=np.int64
        )
        table, address_shift = sigmoid_surrogate_lut_q15()
        self.assertEqual(int(table[128]), 32767)
        addresses = surrogate_lut_addresses(membrane, threshold, address_shift)
        self.assertTrue(np.all(addresses >= 0))
        self.assertTrue(np.all(addresses <= 255))
        expected_input, expected_h = lif_bptt_fixed(
            membrane,
            spikes,
            grad_spikes,
            threshold=threshold,
            membrane_gradients=grad_membrane,
            surrogate_table_q15=table,
            surrogate_address_shift=address_shift,
        )
        emulator = FabricOperatorEmulator()
        got_input, got_h = execute_lif_bptt(
            emulator,
```

```
            membrane,
            spikes,
            grad_spikes,
            threshold=threshold,
            membrane_gradients=grad_membrane,
            surrogate_table_q15=table,
            surrogate_address_shift=address_shift,
        )
        np.testing.assert_array_equal(got_input, expected_input)
        np.testing.assert_array_equal(got_h, expected_h)
        self.assertEqual(emulator.command_count, 1 + time_steps * 3 * 7)

    def test_maxpool_backward_matches_first_winner_and_overlap_accumulation(self) -> None:
        values = self.rng.integers(-20, 21, size=(1, 2, 5, 6), dtype=np.int64)
        values[:, :, 1:4, 1:5] = 7
        values[0, 0, 0, 0] = -(1 << 31)
        gradients = self.rng.integers(-1000, 1001, size=(1, 2, 3, 3), dtype=np.int64)
        expected_gradient, expected_winners = maxpool2d_backward_fixed(
            values,
            gradients,
            kernel_size=3,
            stride=2,
            padding=1,
        )
        emulator = FabricOperatorEmulator()
        got_gradient, got_winners = execute_maxpool2d_backward(
            emulator,
            values,
            gradients,
            kernel_size=3,
            stride=2,
            padding=1,
        )
        np.testing.assert_array_equal(got_winners, expected_winners)
        np.testing.assert_array_equal(got_gradient, expected_gradient)
        self.assertGreater(emulator.command_count, 100)

    def test_real_spikingformer_manifest_has_no_training_coverage_holes(self) -> None:
        root = Path(__file__).resolve().parents[1]
        result = build_spikingformer_training_coverage(
            root / "outputs" / "spikingformer_reference" / "spikingformer_execution_manifest.json",
            root / "outputs" / "spikingformer_hardware_parameters" / "spikingformer_neuraedge_weights.json",
        )
        summary = result["summary"]
        self.assertEqual(summary["execution_operator_count"], 43)
        self.assertEqual(summary["covered_execution_operator_count"], 43)
        self.assertEqual(summary["learned_affine_group_count"], 30)
        self.assertEqual(summary["backbone_projection_count"], 29)
        self.assertEqual(summary["dynamic_attention_matmul_count"], 8)
        self.assertEqual(summary["lif_node_count"], 32)
        self.assertEqual(summary["trainable_deployed_parameter_elements"], 9_295_978)
        self.assertEqual(summary["uncovered_operator_count"], 0)
        self.assertTrue(summary["full_deployed_backbone_covered"])
        self.assertEqual(
            {item["name"] for item in result["operator_coverage"]},
            {item["name"] for item in result["learned_affine_coverage"]}
            | {item["name"] for item in result["dynamic_attention_coverage"]}
            | {f"block.{index}.residuals" for index in range(4)}
            | {"global_average_pool"},
        )


if __name__ == "__main__":
    unittest.main()
```

68.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import unittest` This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 4: `from pathlib import Path` This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import numpy as np` This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `from neuraedge_golden import (` This statement continues through source line 15; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `VectorOperation,` This is continuation line 2 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'VectorOperation,'. The important operands are VectorOperation: the value named 'vector operation' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `log_int_softmax_q15,` This is continuation line 3 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'log_int_softmax_q15,'. The important operands are log_int_softmax_q15: the value named 'log int softmax q15' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `round_shift_signed,` This is continuation line 4 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'round_shift_signed,'. The important operands are round_shift_signed: the value named 'round shift signed' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `saturate_signed,` This is continuation line 5 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'saturate_signed,'. The important operands are saturate_signed: the value named 'saturate signed' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `sgd_momentum_step,` This is continuation line 6 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'sgd_momentum_step,'. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `vector_alu,` This is continuation line 7 of the 8-line logical statement that begins at source line 8. Its exact fragment is 'vector_alu,'. The important operands are vector_alu: the value named 'vector alu' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 8. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `from neuraedge_protocol import LinearGradientMode, Opcode` This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `from neuraedge_training import (` This statement continues through source line 30; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `compile_affine_backward,` This is continuation line 2 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'compile_affine_backward,'. The important operands are compile_affine_backward: the value named 'compile affine backward' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `compile_conv2d_backward,` This is continuation line 3 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'compile_conv2d_backward,'. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `compile_linear_backward,` This is continuation line 4 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'compile_linear_backward,'. The important operands are compile_linear_backward: the value named 'compile linear backward' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `build_spikingformer_training_coverage,` This is continuation line 5 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'build_spikingformer_training_coverage,'. The important operands are build_spikingformer_training_coverage: the value named 'build spikingformer training coverage' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `execute_lif_bptt,` This is continuation line 6 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'execute_lif_bptt,'. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `execute_int8_weight_projection,` This is continuation line 7 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'execute_int8_weight_projection,'. The important operands are execute_int8_weight_projection: a learned coefficient multiplied by an activation. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `execute_maxpool2d_backward,` This is continuation line 8 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'execute_maxpool2d_backward,'. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 25: `execute_sgdw_update,` This is continuation line 9 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'execute_sgdw_update,'. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 26: `lif_bptt_fixed,` This is continuation line 10 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'lif_bptt_fixed,'. The important operands are lif_bptt_fixed: the value named 'lif bptt fixed' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 27: `maxpool2d_backward_fixed,` This is continuation line 11 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'maxpool2d_backward_fixed,'. The important operands are maxpool2d_backward_fixed: the value named 'maxpool2d backward fixed' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 28: `sigmoid_surrogate_lut_q15,` This is continuation line 12 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'sigmoid_surrogate_lut_q15,'. The important operands are sigmoid_surrogate_lut_q15: the value named 'sigmoid surrogate lut q15' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 29: `surrogate_lut_addresses,` This is continuation line 13 of the 14-line logical statement that begins at source line 17. Its exact fragment is 'surrogate_lut_addresses,'. The important operands are surrogate_lut_addresses: the value named 'surrogate lut addresses' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 30: `)` This is continuation line 14 of the 14-line logical statement that begins at source line 17. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 31: `from tests.test_neuraedge_compiler import TensorTransactionEmulator` This imports the named Python API so the file whose role is to checks neuraedge training behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 34: `def _signed(value: int, bits: int) -> int:` This starts function _signed. Its indented body performs signed as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 35: `masked = value & ((1 << bits) - 1)` This assigns 'value & ((1 << bits) - 1)' to 'masked ' in host memory. Here masked means the value named 'masked' in this block. The important operands are value: the value named 'value' in this block; bits: the value named 'bits' in this block. Context: the function _signed, which handles signed.

Line 36: `return masked - (1 << bits) if masked & (1 << (bits - 1)) else masked` This returns 'masked - (1 << bits) if masked & (1 << (bits - 1)) else masked' to the caller of the function _signed, which handles signed, ending this invocation. The important operands are masked: the value named 'masked' in this block; bits: the value named 'bits' in this block.

Line 39: `class BackwardRtlEmulator:` This defines class BackwardRtlEmulator, which groups data and methods for backward rtl emulator. Instances exist in host memory and organize the flow that checks neuraedge training behavior independently with Python assertions.

Line 40: `"""Independent command-level model of retained raw gradient sums."""` This is executable documentation attached to the class BackwardRtlEmulator, which handles backward rtl emulator. It states the intended contract that tests and callers should enforce, but the text itself does not transform data or issue hardware work.

Line 42: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 43: `self.input_accumulator: np.ndarray | None = None` This assigns 'None' to 'self.input_accumulator' in host memory. Here input_accumulator means a widened running sum that prevents intermediate overflow. The important operands are None: the value named 'none' in this block. Context: the function ___init___, which handles init.

Line 44: `self.weight_accumulator: np.ndarray | None = None` This assigns 'None' to 'self.weight_accumulator' in host memory. Here weight_accumulator means a learned coefficient multiplied by an activation. The important operands are None: the value named 'none' in this block. Context: the function ___init___, which handles init.

Line 46: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class BackwardRtlEmulator, which handles backward rtl emulator; it creates no FPGA circuit by itself.

Line 47: `def _round_and_clip(values: np.ndarray, shift: int, bits: int) -> np.ndarray:` This starts function _round_and_clip. Its indented body performs round and clip as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 48: `return saturate_signed(round_shift_signed(values, shift), bits)` This returns 'saturate_signed(round_shift_ shift), bits)' to the caller of the function _round_and_clip, which handles round and clip, ending this invocation. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; values: the value named 'values' in this block.

Line 50: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class BackwardRtlEmulator, which handles backward rtl emulator; it creates no FPGA circuit by itself.

Line 51: `def _decode(command, beats):` This starts function _decode. Its indented body performs decode as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 52: `value = command.value` This assigns 'command.value' to 'value ' in host memory. Here value means the value named 'value' in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function _decode, which handles decode.

Line 53: `rows = (value >> 4) & 0xF` This assigns '(value >> 4) & 0xF' to 'rows ' in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are value: the value named 'value' in this block. Context: the function _decode, which handles decode.

Line 54: `inputs = (value >> 8) & 0xF` This assigns '(value >> 8) & 0xF' to 'inputs ' in host memory. Here inputs means the value named 'inputs' in this block. The important operands are value: the value named 'value' in this block. Context: the function _decode, which handles decode.

Line 55: `outputs = (value >> 12) & 0xF` This assigns '(value >> 12) & 0xF' to 'outputs ' in host memory. Here outputs means the value named 'outputs' in this block. The important operands are value: the value named 'value' in this block. Context: the function _decode, which handles decode.

Line 56: `mode = LinearGradientMode((value >> 26) & 0x3)` This assigns 'LinearGradientMode((value >> 26) & 0x3)' to 'mode ' in host memory. Here mode means the value named 'mode' in this block. The important operands are LinearGradientMode: the signed loss derivative used to update a learned parameter; value: the value named 'value' in this block. Context: the function _decode, which handles decode.

Line 57: `words = [beat.data for beat in beats]` This assigns ‘[beat.data for beat in beats]‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are beat: the value named ’beat’ in this block; data: the value named ’data’ in this block; for: the value named ’for’ in this block. Context: the function _decode, which handles decode.

Line 58: `cursor = 0` This assigns ‘0‘ to ‘cursor ‘ in host memory. Here cursor means the value named ’cursor’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _decode, which handles decode.

Line 59: `x = np.zeros((rows, inputs), dtype=np.int64)` This assigns ‘np.zeros((rows, inputs), dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are zeros: the value named ’zeros’ in this block; rows: one output-row position processed by the tensor tile; inputs: the value named ’inputs’ in this block. Context: the function _decode, which handles decode.

Line 60: `w = np.zeros((inputs, outputs), dtype=np.int64)` This assigns ‘np.zeros((inputs, outputs), dtype=np.int64)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are zeros: the value named ’zeros’ in this block; inputs: the value named ’inputs’ in this block; outputs: the value named ’outputs’ in this block. Context: the function _decode, which handles decode.

Line 61: `if mode != LinearGradientMode.INPUT:` This executes the following indented host-code branch only when ‘mode != LinearGradientMode.INPUT‘ is true. The important operands are mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; INPUT: the value named ’input’ in this block. Context: the function _decode, which handles decode.

Line 62: `for row in range(rows):` This iterates row over ‘range(rows)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _decode, which handles decode.

Line 63: `packed = words[cursor] | (words[cursor + 1] << 64)` This assigns ‘words[cursor] | (words[cursor + 1] << 64)‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are words: the value named ’words’ in this block; cursor: the value named ’cursor’ in this block. Context: the function _decode, which handles decode.

Line 64: `cursor += 2` Inside the function _decode, which handles decode, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are cursor: the value named ’cursor’ in this block.

Line 65: `for lane in range(inputs):` This iterates lane over ‘range(inputs)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _decode, which handles decode.

Line 66: `x[row, lane] = _signed(packed >> (16 * lane), 16)` This assigns ‘_signed(packed >> (16 * lane), 16)‘ to ‘x[row, lane] ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are _signed: the value named ’signed’ in this block; packed: the value named ’packed’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function _decode, which handles decode.

Line 67: `if mode != LinearGradientMode.WEIGHT:` This executes the following indented host-code branch only when ‘mode != LinearGradientMode.WEIGHT‘ is true. The important operands are mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Context: the function _decode, which handles decode.

Line 68: `for row in range(inputs):` This iterates row over ‘range(inputs)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _decode, which handles decode.

Line 69: `packed = words[cursor] | (words[cursor + 1] << 64)` This assigns ‘words[cursor] | (words[cursor + 1] << 64)‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are words: the value named ’words’ in this block; cursor: the value named ’cursor’ in this block. Context: the function _decode, which handles decode.

Line 70: `cursor += 2` Inside the function _decode, which handles decode, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are cursor: the value named ’cursor’ in this block.

Line 71: `for lane in range(outputs):` This iterates lane over ‘range(outputs)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _decode, which handles decode.

Line 72: `w[row, lane] = _signed(packed >> (16 * lane), 16)` This assigns ‘_signed(packed >> (16 * lane), 16)‘ to ‘w[row, lane] ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are _signed: the value named ’signed’ in this block; packed: the value named ’packed’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function _decode, which handles decode.

Line 73: `dy = np.zeros((rows, outputs), dtype=np.int64)` This assigns ‘np.zeros((rows, outputs), dtype=np.int64)‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are zeros: the value named ’zeros’ in this block; rows: one output-row position processed by the tensor tile; outputs: the value named ’outputs’ in this block. Context: the function _decode, which handles decode.

Line 74: `for row in range(rows):` This iterates row over ‘range(rows)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _decode, which handles decode.

Line 75: `packed = words[cursor] | (words[cursor + 1] << 64)` This assigns ‘words[cursor] | (words[cursor + 1] << 64)‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are words: the value named ’words’ in this block; cursor: the value named ’cursor’ in this block. Context: the function _decode, which handles decode.

Line 76: `cursor += 2` Inside the function _decode, which handles decode, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are cursor: the value named ’cursor’ in this block.

Line 77: `for lane in range(outputs):` This iterates lane over ‘range(outputs)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _decode, which handles decode.

Line 78: `dy[row, lane] = _signed(packed >> (16 * lane), 16)` This assigns ‘_signed(packed >> (16 * lane), 16)‘ to ‘dy[row, lane] ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are _signed: the value named ’signed’ in this block; packed: the value named ’packed’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function _decode, which handles decode.

Line 79: `if cursor != len(words):` This executes the following indented host-code branch only when ‘cursor != len(words)‘ is true. The important operands are cursor: the value named ’cursor’ in this block; len: the value named ’len’ in this block; words: the value named ’words’ in this block. Context: the function _decode, which handles decode.

Line 80: `raise AssertionError("compiler emitted an unexpected backward stream length")` This rejects an invalid state by raising ‘AssertionError("compiler emitted an unexpected backward stream length")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 81: `return rows, inputs, outputs, mode, x, w, dy` This returns ‘rows, inputs, outputs, mode, x, w, dy‘ to the caller of the function _decode, which handles decode, ending this invocation. The important operands are rows: one output-row position processed by the tensor tile; inputs: the value named ’inputs’ in this block; outputs: the value named ’outputs’ in this block.

Line 83: `@staticmethod` This decorator changes how the following class or function is constructed. ‘@staticmethod‘ is applied at import time on the host and is part of the class BackwardRtlEmulator, which handles backward rtl emulator; it creates no FPGA circuit by itself.

Line 84: `def _pack_dx(values: np.ndarray) -> tuple[int, ...]:` This starts function _pack_dx. Its indented body performs pack dx as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 85: `words: list[int] = []` This assigns ‘[]‘ to ‘words‘ in host memory. Here words means the value named ’words’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _pack_dx, which handles pack dx.

Line 86: `for row in values:` This iterates row over ‘values‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _pack_dx, which handles pack dx.

Line 87: `padded = list(map(int, row)) + [0] * (8 - len(row))` This assigns ‘list(map(int, row)) + [0] * (8 - len(row))‘ to ‘padded ‘ in host memory. Here padded means the value named ’padded’ in this block. The important operands are list: the value named ’list’ in this block; map: the value named ’map’ in this block; row: one output-row position processed by the tensor tile. Context: the function _pack_dx, which handles pack dx.

Line 88: `for lane in range(0, 8, 2):` This iterates lane over ‘range(0, 8, 2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _pack_dx, which handles pack dx.

Line 89: `words.append(` This statement continues through source line 92; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘words.append‘ with ‘ (padded[lane] & 0xFFFFFFFF) | ((padded[lane + 1] & 0xFFFFFFFF) << 32) ‘ inside the function _pack_dx, which handles pack dx. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 90: `(padded[lane] & 0xFFFFFFFF)` This is continuation line 2 of the 4-line logical statement that begins at source line 89. Its exact fragment is ‘(padded[lane] & 0xFFFFFFFF)‘. The important operands are padded: the value named ’padded’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘words.append‘ with ‘ (padded[lane] & 0xFFFFFFFF) | ((padded[lane + 1] & 0xFFFFFFFF) << 32) ‘ inside the function _pack_dx, which handles pack dx. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 91: `| ((padded[lane + 1] & 0xFFFFFFFF) << 32)` This is continuation line 3 of the 4-line logical statement that begins at source line 89. Its exact fragment is ‘| ((padded[lane + 1] & 0xFFFFFFFF) << 32)‘. The important operands are padded: the value named ’padded’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘words.append‘ with ‘ (padded[lane] & 0xFFFFFFFF) | ((padded[lane + 1] & 0xFFFFFFFF) << 32) ‘ inside the function _pack_dx, which handles pack dx. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 92: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 89. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘words.append‘ with ‘ (padded[lane] & 0xFFFFFFFF) | ((padded[lane + 1] & 0xFFFFFFFF) << 32) ‘ inside the function _pack_dx, which handles pack dx. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 93: `return tuple(words)` This returns ‘tuple(words)‘ to the caller of the function _pack_dx, which handles pack dx, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; words: the value named ’words’ in this block.

Line 95: `@staticmethod` This decorator changes how the following class or function is constructed. ‘@staticmethod‘ is applied at import time on the host and is part of the class BackwardRtlEmulator, which handles backward rtl emulator; it creates no FPGA circuit by itself.

Line 96: `def _pack_dw(values: np.ndarray) -> tuple[int, ...]:` This starts function _pack_dw. Its indented body performs pack dw as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 97: `words: list[int] = []` This assigns '[]' to 'words' in host memory. Here words means the value named 'words' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function _pack_dw, which handles pack dw.

Line 98: `for row in values:` This iterates row over 'values'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function _pack_dw, which handles pack dw.

Line 99: `padded = list(map(int, row)) + [0] * (8 - len(row))` This assigns 'list(map(int, row)) + [0] * (8 - len(row))' to 'padded ' in host memory. Here padded means the value named 'padded' in this block. The important operands are list: the value named 'list' in this block; map: the value named 'map' in this block; row: one output-row position processed by the tensor tile. Context: the function _pack_dw, which handles pack dw.

Line 100: `packed = sum(` This statement continues through source line 103; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sum((item & 0xFFFFFF) << (24 * lane) for lane, item in enumerate(padded))' to 'packed ' in host memory. Here packed means the value named 'packed' in this block. The important operands are sum: the value named 'sum' in this block; item: the value named 'item' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function _pack_dw, which handles pack dw.

Line 101: `(item & 0xFFFFFF) << (24 * lane)` This is continuation line 2 of the 4-line logical statement that begins at source line 100. Its exact fragment is '(item & 0xFFFFFF) << (24 * lane)'. The important operands are item: the value named 'item' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'sum((item & 0xFFFFFF) << (24 * lane) for lane, item in enumerate(padded))' to 'packed ' in host memory. Here packed means the value named 'packed' in this block. The important operands are sum: the value named 'sum' in this block; item: the value named 'item' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function _pack_dw, which handles pack dw.

Line 102: `for lane, item in enumerate(padded)` This is continuation line 3 of the 4-line logical statement that begins at source line 100. Its exact fragment is 'for lane, item in enumerate(padded)'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; item: the value named 'item' in this block. Read together, the complete operation is: This assigns 'sum((item & 0xFFFFFF) << (24 * lane) for lane, item in enumerate(padded))' to 'packed ' in host memory. Here packed means the value named 'packed' in this block. The important operands are sum: the value named 'sum' in this block; item: the value named 'item' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function _pack_dw, which handles pack dw.

Line 103: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 100. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sum((item & 0xFFFFFF) << (24 * lane) for lane, item in enumerate(padded))' to 'packed ' in host memory. Here packed means the value named 'packed' in this block. The important operands are sum: the value named 'sum' in this block; item: the value named 'item' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function _pack_dw, which handles pack dw.

Line 104: `words.extend(` This statement continues through source line 106; the explanation uses the complete expression rather than only the first visual fragment. This calls 'words.extend' with ' (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3) ' inside the function _pack_dw, which handles pack dw. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 105: `(packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3)` This is continuation line 2 of the 3-line logical statement that begins at source line 104. Its exact fragment is '(packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3)'. The important operands are packed: the value named 'packed' in

this block; word: the value named 'word' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This calls 'words.extend' with ' (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3) ' inside the function _pack_dw, which handles pack dw. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 106: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 104. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'words.extend' with ' (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(3) ' inside the function _pack_dw, which handles pack dw. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 107: `return tuple(words)` This returns 'tuple(words)' to the caller of the function _pack_dw, which handles pack dw, ending this invocation. The important operands are tuple: the value named 'tuple' in this block; words: the value named 'words' in this block.

Line 109: `def execute(` This starts function execute. Its indented body performs execute as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 110: `self,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 111: `command,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 112: `input_beats=(),` This assigns '(),' to 'input_beats' in host memory. Here input_beats means the value named 'input beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 113: `*,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 114: `output_beats=0,` This assigns '0,' to 'output_beats' in host memory. Here output_beats means the value named 'output beats' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 115: `timeout_seconds=5.0,` This assigns '5.0,' to 'timeout_seconds' in host memory. Here timeout_seconds means the value named 'timeout seconds' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 116: `) -> tuple[int, ...]:` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are tuple: the value named 'tuple' in this block.

Line 117: `del timeout_seconds` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are del: the value named 'del' in this block; timeout_seconds: the value named 'timeout seconds' in this block.

Line 118: `rows, inputs, outputs, mode, x, w, dy = self._decode(command, input_beats)` This assigns ‘self._decode(co input_beats)‘ to ‘rows, inputs, outputs, mode, x, w, dy ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The important operands are _decode: the value named ’decode’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; input_beats: the value named ’input beats’ in this block. Context: the function execute, which handles execute.

Line 119: `accumulate = bool((command.value >> 28) & 1)` This assigns ‘bool((command.value >> 28) & 1)‘ to ‘accumulate ‘ in host memory. Here accumulate means the value named ’accumulate’ in this block. The important operands are bool: the value named ’bool’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 120: `suppress = bool((command.value >> 29) & 1)` This assigns ‘bool((command.value >> 29) & 1)‘ to ‘suppress ‘ in host memory. Here suppress means the value named ’suppress’ in this block. The important operands are bool: the value named ’bool’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 121: `dx_shift = (command.value >> 16) & 0x1F` This assigns ‘(command.value >> 16) & 0x1F‘ to ‘dx_shift ‘ in host memory. Here dx_shift means the value named ’dx shift’ in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 122: `dw_shift = (command.value >> 21) & 0x1F` This assigns ‘(command.value >> 21) & 0x1F‘ to ‘dw_shift ‘ in host memory. Here dw_shift means the value named ’dw shift’ in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 123: `result: tuple[int, ...]` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are result: the value named ’result’ in this block; tuple: the value named ’tuple’ in this block.

Line 124: `if mode == LinearGradientMode.INPUT:` This executes the following indented host-code branch only when ‘mode == LinearGradientMode.INPUT‘ is true. The important operands are mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; INPUT: the value named ’input’ in this block. Context: the function execute, which handles execute.

Line 125: `partial = np.zeros((rows, inputs), dtype=np.int64)` This assigns ‘np.zeros((rows, inputs), dtype=np.int64)‘ to ‘partial ‘ in host memory. Here partial means the value named ’partial’ in this block. The important operands are zeros: the value named ’zeros’ in this block; rows: one output-row position processed by the tensor tile; inputs: the value named ’inputs’ in this block. Context: the function execute, which handles execute.

Line 126: `for row in range(rows):` This iterates row over ‘range(rows)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 127: `for input_index in range(inputs):` This iterates input_index over ‘range(inputs)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 128: `partial[row, input_index] = sum(` This statement continues through source line 131; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sum(int(dy[row, output]) * int(w[input_index, output]) for output in range(outputs))‘ to ‘partial[row, input_index] ‘ in host memory. Here partial means the value named ’partial’ in this block. The important operands are sum: the value named ’sum’ in this block; dy: the value named ’dy’ in this block; row: one output-row position processed by the tensor tile. Context: the function execute, which handles execute.

Line 129: `int(dy[row, output]) * int(w[input_index, output])` This is continuation line 2 of the 4-line logical statement that begins at source line 128. Its exact fragment is ‘int(dy[row, output]) * int(w[input_index, output])‘. The important operands are dy: the value named ’dy’ in this block; row: one output-row position processed by the tensor tile; output: the value named ’output’ in this block. Read together, the complete operation is: This assigns ‘sum(int(dy[row, output]) * int(w[input_index, output]) for output in range(outputs))‘ to ‘partial[row, input_index] ‘ in host memory. Here partial means the value named ’partial’ in this block. The important operands are sum: the value named ’sum’ in this block; dy: the value named ’dy’ in this block; row: one output-row position processed by the tensor tile. Context: the function execute, which handles execute.

Line 130: `for output in range(outputs)` This is continuation line 3 of the 4-line logical statement that begins at source line 128. Its exact fragment is ‘for output in range(outputs)‘. The important operands are for: the value named ’for’ in this block; output: the value named ’output’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘sum(int(dy[row, output]) * int(w[input_index, output]) for output in range(outputs))‘ to ‘partial[row, input_index] ‘ in host memory. Here partial means the value named ’partial’ in this block. The important operands are sum: the value named ’sum’ in this block; dy: the value named ’dy’ in this block; row: one output-row position processed by the tensor tile. Context: the function execute, which handles execute.

Line 131: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 128. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘sum(int(dy[row, output]) * int(w[input_index, output]) for output in range(outputs))‘ to ‘partial[row, input_index] ‘ in host memory. Here partial means the value named ’partial’ in this block. The important operands are sum: the value named ’sum’ in this block; dy: the value named ’dy’ in this block; row: one output-row position processed by the tensor tile. Context: the function execute, which handles execute.

Line 132: `if accumulate:` This executes the following indented host-code branch only when ‘accumulate‘ is true. The important operands are accumulate: the value named ’accumulate’ in this block. Context: the function execute, which handles execute.

Line 133: `if self.input_accumulator is None:` This executes the following indented host-code branch only when ‘self.input_accumulator is None‘ is true. The important operands are input_accumulator: a widened running sum that prevents intermediate overflow; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function execute, which handles execute.

Line 134: `raise AssertionError("input accumulation began without retained state")` This rejects an invalid state by raising ‘AssertionError("input accumulation began without retained state")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 135: `self.input_accumulator += partial` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are input_accumulator: a widened running sum that prevents intermediate overflow; partial: the value named ’partial’ in this block.

Line 136: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute, which handles execute.

Line 137: `self.input_accumulator = partial` This assigns ‘partial‘ to ‘self.input_accumulator ‘ in host memory. Here input_accumulator means a widened running sum that prevents intermediate overflow. The important operands are partial: the value named ’partial’ in this block. Context: the function execute, which handles execute.

Line 138: `if suppress:` This executes the following indented host-code branch only when ‘suppress‘ is true. The important operands are suppress: the value named ’suppress’ in this block. Context: the function execute, which handles execute.

Line 139: `result = ()` This assigns '()' to 'result ' in host memory. Here result means the value named 'result' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 140: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute, which handles execute.

Line 141: `result = self._pack_dx(` This statement continues through source line 143; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self._pack_dx(self._round_and_clip(self.input_accumulat dx_shift, 32))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are _pack_dx: the value named 'pack dx' in this block; _round_and_clip: the value named 'round and clip' in this block; input_accumulator: a widened running sum that prevents intermediate overflow. Context: the function execute, which handles execute.

Line 142: `self._round_and_clip(self.input_accumulator, dx_shift, 32)` This is continuation line 2 of the 3-line logical statement that begins at source line 141. Its exact fragment is 'self._round_and_clip(self.input_accumulator, dx_shift, 32)'. The important operands are _round_and_clip: the value named 'round and clip' in this block; input_accumulator: a widened running sum that prevents intermediate overflow; dx_shift: the value named 'dx shift' in this block. Read together, the complete operation is: This assigns 'self._pack_dx(self._round_and_clip(self.input_accumulator, dx_shift, 32))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are _pack_dx: the value named 'pack dx' in this block; _round_and_clip: the value named 'round and clip' in this block; input_accumulator: a widened running sum that prevents intermediate overflow. Context: the function execute, which handles execute.

Line 143: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 141. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self._pack_dx(self._round_and_clip(self.input_accumulator, dx_shift, 32))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are _pack_dx: the value named 'pack dx' in this block; _round_and_clip: the value named 'round and clip' in this block; input_accumulator: a widened running sum that prevents intermediate overflow. Context: the function execute, which handles execute.

Line 144: `self.input_accumulator = None` This assigns 'None' to 'self.input_accumulator ' in host memory. Here input_accumulator means a widened running sum that prevents intermediate overflow. The important operands are None: the value named 'none' in this block. Context: the function execute, which handles execute.

Line 145: `elif mode == LinearGradientMode.WEIGHT:` This executes the following indented host-code branch only when 'mode == LinearGradientMode.WEIGHT' is true. The important operands are mode: the value named 'mode' in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter; WEIGHT: a learned coefficient multiplied by an activation. Context: the function execute, which handles execute.

Line 146: `partial = np.zeros((inputs, outputs), dtype=np.int64)` This assigns 'np.zeros((inputs, outputs), dtype=np.int64)' to 'partial ' in host memory. Here partial means the value named 'partial' in this block. The important operands are zeros: the value named 'zeros' in this block; inputs: the value named 'inputs' in this block; outputs: the value named 'outputs' in this block. Context: the function execute, which handles execute.

Line 147: `for input_index in range(inputs):` This iterates input_index over 'range(inputs)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 148: `for output in range(outputs):` This iterates output over 'range(outputs)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 149: `partial[input_index, output] = sum(` This statement continues through source line 152; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sum(int(x[row, input_index]) * int(dy[row, output]) for row in range(rows))‘ to ‘partial[input_index, output] ‘ in host memory. Here partial means the value named ’partial’ in this block. The important operands are sum: the value named ’sum’ in this block; x: the value named ’x’ in this block; row: one output-row position processed by the tensor tile. Context: the function execute, which handles execute.

Line 150: `int(x[row, input_index]) * int(dy[row, output])` This is continuation line 2 of the 4-line logical statement that begins at source line 149. Its exact fragment is ‘int(x[row, input_index]) * int(dy[row, output])‘. The important operands are x: the value named ’x’ in this block; row: one output-row position processed by the tensor tile; input_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns ‘sum(int(x[row, input_index]) * int(dy[row, output]) for row in range(rows))‘ to ‘partial[input_index, output] ‘ in host memory. Here partial means the value named ’partial’ in this block. The important operands are sum: the value named ’sum’ in this block; x: the value named ’x’ in this block; row: one output-row position processed by the tensor tile. Context: the function execute, which handles execute.

Line 151: `for row in range(rows)` This is continuation line 3 of the 4-line logical statement that begins at source line 149. Its exact fragment is ‘for row in range(rows)‘. The important operands are for: the value named ’for’ in this block; row: one output-row position processed by the tensor tile; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘sum(int(x[row, input_index]) * int(dy[row, output]) for row in range(rows))‘ to ‘partial[input_index, output] ‘ in host memory. Here partial means the value named ’partial’ in this block. The important operands are sum: the value named ’sum’ in this block; x: the value named ’x’ in this block; row: one output-row position processed by the tensor tile. Context: the function execute, which handles execute.

Line 152: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 149. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘sum(int(x[row, input_index]) * int(dy[row, output]) for row in range(rows))‘ to ‘partial[input_index, output] ‘ in host memory. Here partial means the value named ’partial’ in this block. The important operands are sum: the value named ’sum’ in this block; x: the value named ’x’ in this block; row: one output-row position processed by the tensor tile. Context: the function execute, which handles execute.

Line 153: `if accumulate:` This executes the following indented host-code branch only when ‘accumulate‘ is true. The important operands are accumulate: the value named ’accumulate’ in this block. Context: the function execute, which handles execute.

Line 154: `if self.weight_accumulator is None:` This executes the following indented host-code branch only when ‘self.weight_accumulator is None‘ is true. The important operands are weight_accumulator: a learned coefficient multiplied by an activation; is: the value named ’is’ in this block; None: the value named ’none’ in this block. Context: the function execute, which handles execute.

Line 155: `raise AssertionError("weight accumulation began without retained state")` This rejects an invalid state by raising ‘AssertionError("weight accumulation began without retained state")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 156: `self.weight_accumulator += partial` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are weight_accumulator: a learned coefficient multiplied by an activation; partial: the value named ’partial’ in this block.

Line 157: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute, which handles execute.

Line 158: `self.weight_accumulator = partial` This assigns ‘partial‘ to ‘self.weight_accumulator ‘ in host memory. Here weight_accumulator means a learned coefficient multiplied by an activation. The important operands are partial: the value named ’partial’ in this block. Context: the function execute, which handles execute.

Line 159: `if suppress:` This executes the following indented host-code branch only when ‘suppress‘ is true. The important operands are suppress: the value named ’suppress’ in this block. Context: the function execute, which handles execute.

Line 160: `result = ()` This assigns ‘()‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 161: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute, which handles execute.

Line 162: `result = self._pack_dw(` This statement continues through source line 164; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘self._pack_dw(self._round_and_clip(self.weight_accumula dw_shift, 24))‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are _pack_dw: the value named ’pack dw’ in this block; _round_and_clip: the value named ’round and clip’ in this block; weight_accumulator: a learned coefficient multiplied by an activation. Context: the function execute, which handles execute.

Line 163: `self._round_and_clip(self.weight_accumulator, dw_shift, 24)` This is continuation line 2 of the 3-line logical statement that begins at source line 162. Its exact fragment is ‘self._round_and_clip(self.weight_accumulator, dw_shift, 24)‘. The important operands are _round_and_clip: the value named ’round and clip’ in this block; weight_accumulator: a learned coefficient multiplied by an activation; dw_shift: the value named ’dw shift’ in this block. Read together, the complete operation is: This assigns ‘self._pack_dw(self._round_and_clip(self.weight_accumulator, dw_shift, 24))‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are _pack_dw: the value named ’pack dw’ in this block; _round_and_clip: the value named ’round and clip’ in this block; weight_accumulator: a learned coefficient multiplied by an activation. Context: the function execute, which handles execute.

Line 164: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 162. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘self._pack_dw(self._round_and_clip(self.weight_accumulator, dw_shift, 24))‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are _pack_dw: the value named ’pack dw’ in this block; _round_and_clip: the value named ’round and clip’ in this block; weight_accumulator: a learned coefficient multiplied by an activation. Context: the function execute, which handles execute.

Line 165: `self.weight_accumulator = None` This assigns ‘None‘ to ‘self.weight_accumulator ‘ in host memory. Here weight_accumulator means a learned coefficient multiplied by an activation. The important operands are None: the value named ’none’ in this block. Context: the function execute, which handles execute.

Line 166: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute, which handles execute.

Line 167: `raise AssertionError("the tiled compiler must use one-gradient passes")` This rejects an invalid state by raising ‘AssertionError("the tiled compiler must use one-gradient passes")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 168: `if len(result) != output_beats:` This executes the following indented host-code branch only when ‘len(result) != output_beats‘ is true. The important operands are len: the value named ’len’ in this block; result: the value named ’result’ in this block; output_beats: the value named ’output beats’ in this block. Context: the function execute, which handles execute.

Line 169: `raise AssertionError(f"emulator produced {len(result)} words, expected {output_beats}")` This rejects an invalid state by raising ‘AssertionError(f"emulator produced {len(result)} words, expected {output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 170: `return result` This returns ‘result‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are result: the value named ’result’ in this block.

Line 172: `def execute_program(self, records, *, timeout_seconds=30.0) -> tuple[int, ...]:` This starts function execute_program. Its indented body performs execute program as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 173: `del timeout_seconds` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are del: the value named ’del’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block.

Line 174: `output: list[int] = []` This assigns ‘[]‘ to ‘output‘ in host memory. Here output means the value named ’output’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 175: `for record in records:` This iterates record over ‘records‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_program, which handles execute program.

Line 176: `output.extend(self.execute(` This statement continues through source line 180; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 177: `record.command,` This is continuation line 2 of the 5-line logical statement that begins at source line 176. Its exact fragment is ‘record.command,‘. The important operands are record: the value named ’record’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 178: `record.input_beats,` This is continuation line 3 of the 5-line logical statement that begins at source line 176. Its exact fragment is ‘record.input_beats,‘. The important operands are record: the value named ’record’ in this block; input_beats: the value named ’input beats’ in this block. Read together, the complete operation is: This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 179: `output_beats=record.output_beats,` This is continuation line 4 of the 5-line logical statement that begins at source line 176. Its exact fragment is ‘output_beats=record.output_beats,‘. The important operands are output_beats: the value named ’output beats’ in this block; record: the value named ’record’ in this block. Read together, the complete operation is: This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 180: `))` This is continuation line 5 of the 5-line logical statement that begins at source line 176. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 181: `return tuple(output)` This returns ‘tuple(output)‘ to the caller of the function execute_program, which handles execute program, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; output: the value named ’output’ in this block.

Line 184: `class FabricOperatorEmulator:` This defines class FabricOperatorEmulator, which groups data and methods for fabric operator emulator. Instances exist in host memory and organize the flow that checks neuraedge training behavior independently with Python assertions.

Line 185: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 186: `self.lut = np.zeros(256, dtype=np.int64)` This assigns ‘np.zeros(256, dtype=np.int64)‘ to ‘self.lut ‘ in host memory. Here lut means the value named ’lut’ in this block. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function ___init___, which handles init.

Line 187: `self.lif_membrane = np.zeros(32, dtype=np.int64)` This assigns ‘np.zeros(32, dtype=np.int64)‘ to ‘self.lif_membrane ‘ in host memory. Here lif_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function ___init___, which handles init.

Line 188: `self.lif_timestep = 0` This assigns ‘0‘ to ‘self.lif_timestep ‘ in host memory. Here lif_timestep means one discrete SNN update instant. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 189: `self.command_count = 0` This assigns ‘0‘ to ‘self.command_count ‘ in host memory. Here command_count means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 191: `def execute(` This starts function execute. Its indented body performs execute as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 192: `self,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 193: `command,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 194: `input_beats=(),` This assigns ‘(),‘ to ‘input_beats‘ in host memory. Here input_beats means the value named ’input beats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 195: `*,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 196: `output_beats=0,` This assigns ‘0,‘ to ‘output_beats‘ in host memory. Here output_beats means the value named ’output beats’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 197: `timeout_seconds=5.0,` This assigns ‘5.0,‘ to ‘timeout_seconds‘ in host memory. Here timeout_seconds means the value named ’timeout seconds’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 198: `) -> tuple[int, ...]:` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are tuple: the value named ’tuple’ in this block.

Line 199: `del timeout_seconds` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are del: the value named ’del’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block.

Line 200: `self.command_count += 1` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are command_count: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 201: `words = [beat.data for beat in input_beats]` This assigns ‘[beat.data for beat in input_beats]‘ to ‘words ‘ in host memory. Here words means the value named ’words’ in this block. The important operands are beat: the value named ’beat’ in this block; data: the value named ’data’ in this block; for: the value named ’for’ in this block. Context: the function execute, which handles execute.

Line 202: `if command.opcode == Opcode.ACTIVATION_LUT:` This executes the following indented host-code branch only when ‘command.opcode == Opcode.ACTIVATION_LUT‘ is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; opcode: the value named ’opcode’ in this block; Opcode: the value named ’opcode’ in this block. Context: the function execute, which handles execute.

Line 203: `suboperation = (command.value >> 4) & 0xF` This assigns ‘(command.value >> 4) & 0xF‘ to ‘suboperation ‘ in host memory. Here suboperation means the value named ’suboperation’ in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 204: `if suboperation == 1:` This executes the following indented host-code branch only when ‘suboperation == 1‘ is true. The important operands are suboperation: the value named ’suboperation’ in this block. Context: the function execute, which handles execute.

Line 205: `for word in words:` This iterates word over ‘words‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute, which handles execute.

Line 206: `self.lut[word & 0xFF] = _signed(word >> 8, 16)` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are lut: the value named ’lut’ in this block; word: the value named ’word’ in this block; _signed: the value named ’signed’ in this block.

Line 207: `result: tuple[int, ...] = ()` This assigns ‘()‘ to ‘result‘ in host memory. Here result means the value named ’result’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 208: `elif suboperation == 2:` This executes the following indented host-code branch only when ‘suboperation == 2‘ is true. The important operands are suboperation: the value named ’suboperation’ in this block. Context: the function execute, which handles execute.

Line 209: `indices = [(words[0] >> (8 * lane)) & 0xFF for lane in range(8)]` This assigns ‘[(words[0] >> (8 * lane)) & 0xFF for lane in range(8)]‘ to ‘indices ‘ in host memory. Here indices means the value named ’indices’ in this block. The important operands are words: the value named ’words’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; for: the value named ’for’ in this block. Context: the function execute, which handles execute.

Line 210: `values = [int(self.lut[index]) for index in indices]` This assigns ‘[int(self.lut[index]) for index in indices]‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are lut: the value named ’lut’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry; for: the value named ’for’ in this block. Context: the function execute, which handles execute.

Line 211: `result = tuple(` This statement continues through source line 215; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘tuple(sum((values[word * 4 + lane] & 0xFFFF) << (16 * lane) for lane in range(4)) for word in range(2))‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are tuple: the value named ’tuple’ in this block; sum: the value named ’sum’ in this block; values: the value named ’values’ in this block. Context: the function execute, which handles execute.

Line 212: `sum((values[word * 4 + lane] & 0xFFFF) << (16 * lane)` This is continuation line 2 of the 5-line logical statement that begins at source line 211. Its exact fragment is ‘sum((values[word * 4 + lane] & 0xFFFF) << (16 * lane)‘. The important operands are sum: the value named ’sum’ in this block; values: the value named ’values’ in this block; word: the value named ’word’ in this block. Read together, the complete operation is: This assigns ‘tuple(sum((values[word * 4 + lane] & 0xFFFF) << (16 * lane) for lane in range(4)) for word in range(2))‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are tuple: the value named ’tuple’ in this block; sum: the value named ’sum’ in this block; values: the value named ’values’ in this block. Context: the function execute, which handles execute.

Line 213: `for lane in range(4))` This is continuation line 3 of the 5-line logical statement that begins at source line 211. Its exact fragment is ‘for lane in range(4))‘. The important operands are for: the value named ’for’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘tuple(sum((values[word * 4 + lane] & 0xFFFF) << (16 * lane) for lane in range(4)) for word in range(2))‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are tuple: the value named ’tuple’ in this block; sum: the value named ’sum’ in this block; values: the value named ’values’ in this block. Context: the function execute, which handles execute.

Line 214: `for word in range(2)` This is continuation line 4 of the 5-line logical statement that begins at source line 211. Its exact fragment is ‘for word in range(2)‘. The important operands are for: the value named ’for’ in this block; word: the value named ’word’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘tuple(sum((values[word * 4 + lane] & 0xFFFF) << (16 * lane) for lane in range(4)) for word in range(2))‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are tuple: the value named ’tuple’ in this block; sum: the value named ’sum’ in this block; values: the value named ’values’ in this block. Context: the function execute, which handles execute.

Line 215: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 211. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘tuple(sum((values[word * 4 + lane] & 0xFFFF) << (16 * lane) for lane in range(4)) for word in range(2))‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are tuple: the value named ’tuple’ in this block; sum: the value named ’sum’ in this block; values: the value named ’values’ in this block. Context: the function execute, which handles execute.

Line 216: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute, which handles execute.

Line 217: `self.lut.fill(0)` This calls ‘self.lut.fill‘ with ‘0‘ inside the function execute, which handles execute. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 218: `result = ()` This assigns '()' to 'result ' in host memory. Here result means the value named 'result' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 219: `elif command.opcode == Opcode.VECTOR_ALU:` This executes the following indented host-code branch only when 'command.opcode == Opcode.VECTOR_ALU' is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; opcode: the value named 'opcode' in this block; Opcode: the value named 'opcode' in this block. Context: the function execute, which handles execute.

Line 220: `operation = VectorOperation((command.value >> 4) & 0xF)` This assigns 'VectorOperation((command.value >> 4) & 0xF)' to 'operation ' in host memory. Here operation means the value named 'operation' in this block. The important operands are VectorOperation: the value named 'vector operation' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 221: `multiplier = _signed(command.value >> 8, 16)` This assigns '_signed(command.value >> 8, 16)' to 'multiplier ' in host memory. Here multiplier means the value named 'multiplier' in this block. The important operands are _signed: the value named 'signed' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 222: `shift_bits = (command.value >> 24) & 0x1F` This assigns '(command.value >> 24) & 0x1F' to 'shift_bits ' in host memory. Here shift_bits means the value named 'shift bits' in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 223: `right_shift = _signed(shift_bits, 5) if operation == VectorOperation.FLEX_REQUANTIZE else shift_bits` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are right_shift: the value named 'right shift' in this block; _signed: the value named 'signed' in this block; shift_bits: the value named 'shift bits' in this block.

Line 224: `output_zero_point = _signed(command.value >> 29, 16)` This assigns '_signed(command.value >> 29, 16)' to 'output_zero_point ' in host memory. Here output_zero_point means the integer code representing real numeric zero in affine quantization. The important operands are _signed: the value named 'signed' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 225: `threshold = _signed(command.value >> 45, 32)` This assigns '_signed(command.value >> 45, 32)' to 'threshold ' in host memory. Here threshold means the membrane level at which a neuron emits a spike. The important operands are _signed: the value named 'signed' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 226: `surrogate_window = (command.value >> 77) & 0xFFFFFFFF` This assigns '(command.value >> 77) & 0xFFFFFFFF' to 'surrogate_window ' in host memory. Here surrogate_window means the value named 'surrogate window' in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 227: `clamp_minimum = _signed(words[0], 32)` This assigns '_signed(words[0], 32)' to 'clamp_minimum ' in host memory. Here clamp_minimum means the value named 'clamp minimum' in this block. The important operands are _signed: the value named 'signed' in this block; words: the value named 'words' in this block. Context: the function execute, which handles execute.

Line 228: `clamp_maximum = _signed(words[0] >> 32, 32)` This assigns '_signed(words[0] >> 32, 32)' to 'clamp_maximum ' in host memory. Here clamp_maximum means the value named 'clamp maximum' in this block. The important operands are _signed: the value named 'signed' in this block; words: the value named 'words' in this block. Context: the function execute, which handles execute.

Line 229: `operand_a = np.array([` This statement continues through source line 232; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.array([_signed(words[1 + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(8)], dtype=np.int64)' to 'operand_a ' in host memory. Here operand_a means the value named 'operand a' in this block. The important operands are array: the value named 'array' in this block; _signed: the value named 'signed' in this block; words: the value named 'words' in this block. Context: the function execute, which handles execute.

Line 230: `_signed(words[1 + lane // 2] >> (32 * (lane % 2)), 32)` This is continuation line 2 of the 4-line logical statement that begins at source line 229. Its exact fragment is '_signed(words[1 + lane // 2] >> (32 * (lane % 2)), 32)'. The important operands are _signed: the value named 'signed' in this block; words: the value named 'words' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'np.array([_signed(words[1 + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(8)], dtype=np.int64)' to 'operand_a ' in host memory. Here operand_a means the value named 'operand a' in this block. The important operands are array: the value named 'array' in this block; _signed: the value named 'signed' in this block; words: the value named 'words' in this block. Context: the function execute, which handles execute.

Line 231: `for lane in range(8)` This is continuation line 3 of the 4-line logical statement that begins at source line 229. Its exact fragment is 'for lane in range(8)'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'np.array([_signed(words[1 + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(8)], dtype=np.int64)' to 'operand_a ' in host memory. Here operand_a means the value named 'operand a' in this block. The important operands are array: the value named 'array' in this block; _signed: the value named 'signed' in this block; words: the value named 'words' in this block. Context: the function execute, which handles execute.

Line 232: `], dtype=np.int64)` This is continuation line 4 of the 4-line logical statement that begins at source line 229. Its exact fragment is '], dtype=np.int64)'. The important operands are dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'np.array([_signed(words[1 + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(8)], dtype=np.int64)' to 'operand_a ' in host memory. Here operand_a means the value named 'operand a' in this block. The important operands are array: the value named 'array' in this block; _signed: the value named 'signed' in this block; words: the value named 'words' in this block. Context: the function execute, which handles execute.

Line 233: `operand_b = np.array([` This statement continues through source line 236; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.array([_signed(words[5 + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(8)], dtype=np.int64)' to 'operand_b ' in host memory. Here operand_b means the value named 'operand b' in this block. The important operands are array: the value named 'array' in this block; _signed: the value named 'signed' in this block; words: the value named 'words' in this block. Context: the function execute, which handles execute.

Line 234: `_signed(words[5 + lane // 2] >> (32 * (lane % 2)), 32)` This is continuation line 2 of the 4-line logical statement that begins at source line 233. Its exact fragment is '_signed(words[5 + lane // 2] >> (32 * (lane % 2)), 32)'. The important operands are _signed: the value named 'signed' in this block; words: the value named 'words' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'np.array([_signed(words[5 + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(8)], dtype=np.int64)' to 'operand_b ' in host memory. Here operand_b means the value named 'operand b' in this block. The important operands are array: the value named 'array' in this block; _signed: the value named 'signed' in this block; words: the value named 'words' in this block. Context: the function execute, which handles execute.

Line 235: `for lane in range(8)` This is continuation line 3 of the 4-line logical statement that begins at source line 233. Its exact fragment is 'for lane in range(8)'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named 'in' in this block. Read together,

the complete operation is: This assigns ‘np.array([_signed(words[5 + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(8)], dtype=np.int64)‘ to ‘operand_b ‘ in host memory. Here operand_b means the value named ’operand b’ in this block. The important operands are array: the value named ’array’ in this block; _signed: the value named ’signed’ in this block; words: the value named ’words’ in this block. Context: the function execute, which handles execute.

Line 236: `], dtype=np.int64)` This is continuation line 4 of the 4-line logical statement that begins at source line 233. Its exact fragment is ‘], dtype=np.int64)‘. The important operands are dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This assigns ‘np.array([_signed(words[5 + lane // 2] >> (32 * (lane % 2)), 32) for lane in range(8)], dtype=np.int64)‘ to ‘operand_b ‘ in host memory. Here operand_b means the value named ’operand b’ in this block. The important operands are array: the value named ’array’ in this block; _signed: the value named ’signed’ in this block; words: the value named ’words’ in this block. Context: the function execute, which handles execute.

Line 237: `values = vector_alu(` This statement continues through source line 248; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; operand_a: the value named ’operand a’ in this block. Context: the function execute, which handles execute.

Line 238: `operation,` This is continuation line 2 of the 12-line logical statement that begins at source line 237. Its exact fragment is ‘operation,‘. The important operands are operation: the value named ’operation’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; operand_a: the value named ’operand a’ in this block. Context: the function execute, which handles execute.

Line 239: `operand_a,` This is continuation line 3 of the 12-line logical statement that begins at source line 237. Its exact fragment is ‘operand_a,‘. The important operands are operand_a: the value named ’operand a’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; operand_a: the value named ’operand a’ in this block. Context: the function execute, which handles execute.

Line 240: `operand_b,` This is continuation line 4 of the 12-line logical statement that begins at source line 237. Its exact fragment is ‘operand_b,‘. The important operands are operand_b: the value named ’operand b’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are vector_alu: the value named ’vector alu’ in this block; operation: the value named ’operation’ in this block; operand_a: the value named ’operand a’ in this block. Context: the function execute, which handles execute.

Line 241: `multiplier_q15=multiplier,` This is continuation line 5 of the 12-line logical statement that begins at source line 237. Its exact fragment is ‘multiplier_q15=multiplier,‘. The important operands are multiplier_q15: the value named ’multiplier q15’ in this block; multiplier: the value named ’multiplier’ in this block. Read together, the complete operation is: This assigns ‘vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are vector_alu: the value named ’vector alu’ in

this block; operation: the value named 'operation' in this block; operand_a: the value named 'operand a' in this block. Context: the function execute, which handles execute.

Line 242: `right_shift=right_shift,` This is continuation line 6 of the 12-line logical statement that begins at source line 237. Its exact fragment is 'right_shift=right_shift,'. The important operands are right_shift: the value named 'right shift' in this block. Read together, the complete operation is: This assigns 'vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; operand_a: the value named 'operand a' in this block. Context: the function execute, which handles execute.

Line 243: `output_zero_point=output_zero_point,` This is continuation line 7 of the 12-line logical statement that begins at source line 237. Its exact fragment is 'output_zero_point=output_zero_point,'. The important operands are output_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns 'vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_
clamp_maximum=clamp_maximum,)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; operand_a: the value named 'operand a' in this block. Context: the function execute, which handles execute.

Line 244: `threshold=threshold,` This is continuation line 8 of the 12-line logical statement that begins at source line 237. Its exact fragment is 'threshold=threshold,'. The important operands are threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns 'vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_max
)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; operand_a: the value named 'operand a' in this block. Context: the function execute, which handles execute.

Line 245: `surrogate_window=surrogate_window,` This is continuation line 9 of the 12-line logical statement that begins at source line 237. Its exact fragment is 'surrogate_window=surrogate_window,'. The important operands are surrogate_window: the value named 'surrogate window' in this block. Read together, the complete operation is: This assigns 'vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_min
clamp_maximum=clamp_maximum,)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; operand_a: the value named 'operand a' in this block. Context: the function execute, which handles execute.

Line 246: `clamp_minimum=clamp_minimum,` This is continuation line 10 of the 12-line logical statement that begins at source line 237. Its exact fragment is 'clamp_minimum=clamp_minimum,'. The important operands are clamp_minimum: the value named 'clamp minimum' in this block. Read together, the complete operation is: This assigns 'vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_z
threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_max
)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; operand_a: the value named 'operand a' in this block. Context: the function execute, which handles execute.

Line 247: `clamp_maximum=clamp_maximum,` This is continuation line 11 of the 12-line logical statement that begins at source line 237. Its exact fragment is 'clamp_maximum=clamp_maximum,'. The important operands are clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This assigns 'vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=outp
threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_max
)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are

vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; operand_a: the value named 'operand a' in this block. Context: the function execute, which handles execute.

Line 248: `)` This is continuation line 12 of the 12-line logical statement that begins at source line 237. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'vector_alu(operation, operand_a, operand_b, multiplier_q15=multiplier, right_shift=right_shift, output_zero_point=output_zero_point, threshold=threshold, surrogate_window=surrogate_window, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum,)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are vector_alu: the value named 'vector alu' in this block; operation: the value named 'operation' in this block; operand_a: the value named 'operand a' in this block. Context: the function execute, which handles execute.

Line 249: `result = tuple(` This statement continues through source line 253; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'tuple((int(values[lane]) & 0xFFFFFFFF) | ((int(values[lane + 1]) & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; values: the value named 'values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 250: `(int(values[lane]) & 0xFFFFFFFF)` This is continuation line 2 of the 5-line logical statement that begins at source line 249. Its exact fragment is '(int(values[lane]) & 0xFFFFFFFF)'. The important operands are values: the value named 'values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'tuple((int(values[lane]) & 0xFFFFFFFF) | ((int(values[lane + 1]) & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; values: the value named 'values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 251: `| ((int(values[lane + 1]) & 0xFFFFFFFF) << 32)` This is continuation line 3 of the 5-line logical statement that begins at source line 249. Its exact fragment is '| ((int(values[lane + 1]) & 0xFFFFFFFF) << 32)'. The important operands are values: the value named 'values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'tuple((int(values[lane]) & 0xFFFFFFFF) | ((int(values[lane + 1]) & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; values: the value named 'values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 252: `for lane in range(0, 8, 2)` This is continuation line 4 of the 5-line logical statement that begins at source line 249. Its exact fragment is 'for lane in range(0, 8, 2)'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'tuple((int(values[lane]) & 0xFFFFFFFF) | ((int(values[lane + 1]) & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; values: the value named 'values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 253: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 249. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'tuple((int(values[lane]) & 0xFFFFFFFF) | ((int(values[lane + 1]) & 0xFFFFFFFF) << 32) for lane in range(0, 8, 2))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; values: the value named 'values' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 254: `elif command.opcode == Opcode.SGD_UPDATE:` This executes the following indented host-code branch only when 'command.opcode == Opcode.SGD_UPDATE' is true. The important operands are command: the 128-bit

instruction selecting an engine, shape, mode, and numeric parameters; opcode: the value named 'opcode' in this block; Opcode: the value named 'opcode' in this block. Context: the function execute, which handles execute.

Line 255: `weights = np.array([_signed(word, 16) for word in words], dtype=np.int64)` This assigns 'np.array([_signe 16) for word in words], dtype=np.int64)' to 'weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are array: the value named 'array' in this block; _signed: the value named 'signed' in this block; word: the value named 'word' in this block. Context: the function execute, which handles execute.

Line 256: `gradients = np.array([_signed(word >> 16, 24) for word in words], dtype=np.int64)` This assigns 'np.array([_signed(word >> 16, 24) for word in words], dtype=np.int64)' to 'gradients ' in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are array: the value named 'array' in this block; _signed: the value named 'signed' in this block; word: the value named 'word' in this block. Context: the function execute, which handles execute.

Line 257: `velocities = np.array([_signed(word >> 40, 24) for word in words], dtype=np.int64)` This assigns 'np.array([_signed(word >> 40, 24) for word in words], dtype=np.int64)' to 'velocities ' in host memory. Here velocities means the value named 'velocities' in this block. The important operands are array: the value named 'array' in this block; _signed: the value named 'signed' in this block; word: the value named 'word' in this block. Context: the function execute, which handles execute.

Line 258: `learning_rate = (command.value >> 4) & 0x3FFFF` This assigns '(command.value >> 4) & 0x3FFFF' to 'learning_rate ' in host memory. Here learning_rate means the fixed-point multiplier controlling update magnitude. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 259: `momentum = (command.value >> 22) & 0x1FFFF` This assigns '(command.value >> 22) & 0x1FFFF' to 'momentum ' in host memory. Here momentum means the value named 'momentum' in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 260: `weight_decay = (command.value >> 39) & 0x1FFFF` This assigns '(command.value >> 39) & 0x1FFFF' to 'weight_decay ' in host memory. Here weight_decay means a learned coefficient multiplied by an activation. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 261: `update_right_shift = (command.value >> 56) & 0x1F` This assigns '(command.value >> 56) & 0x1F' to 'update_right_shift ' in host memory. Here update_right_shift means the value named 'update right shift' in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 262: `updated_weight, updated_velocity = sgd_momentum_step(` This statement continues through source line 270; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sgd_momentum_step(weights, gradients, velocities, learning_rate, momentum, weight_decay, update_right_shift,)' to 'updated_weight, updated_velocity ' in host memory. Here updated_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function execute, which handles execute.

Line 263: `weights,` This is continuation line 2 of the 9-line logical statement that begins at source line 262. Its exact fragment is 'weights,'. The important operands are weights: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, learning_rate, momentum, weight_decay, update_right_shift,)' to 'updated_weight, updated_velocity ' in host memory. Here updated_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function execute, which handles execute.

Line 264: `gradients,` This is continuation line 3 of the 9-line logical statement that begins at source line 262. Its exact fragment is 'gradients,'. The important operands are gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, learning_rate, momentum, weight_decay, update_right_shift,)' to 'updated_weight, updated_velocity ' in host memory. Here updated_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function execute, which handles execute.

Line 265: `velocities,` This is continuation line 4 of the 9-line logical statement that begins at source line 262. Its exact fragment is 'velocities,'. The important operands are velocities: the value named 'velocities' in this block. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, learning_rate, momentum, weight_decay, update_right_shift,)' to 'updated_weight, updated_velocity ' in host memory. Here updated_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function execute, which handles execute.

Line 266: `learning_rate,` This is continuation line 5 of the 9-line logical statement that begins at source line 262. Its exact fragment is 'learning_rate,'. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, learning_rate, momentum, weight_decay, update_right_shift,)' to 'updated_weight, updated_velocity ' in host memory. Here updated_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function execute, which handles execute.

Line 267: `momentum,` This is continuation line 6 of the 9-line logical statement that begins at source line 262. Its exact fragment is 'momentum,'. The important operands are momentum: the value named 'momentum' in this block. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, learning_rate, momentum, weight_decay, update_right_shift,)' to 'updated_weight, updated_velocity ' in host memory. Here updated_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function execute, which handles execute.

Line 268: `weight_decay,` This is continuation line 7 of the 9-line logical statement that begins at source line 262. Its exact fragment is 'weight_decay,'. The important operands are weight_decay: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, learning_rate, momentum, weight_decay, update_right_shift,)' to 'updated_weight, updated_velocity ' in host memory. Here updated_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function execute, which handles execute.

Line 269: `update_right_shift,` This is continuation line 8 of the 9-line logical statement that begins at source line 262. Its exact fragment is 'update_right_shift,'. The important operands are update_right_shift: the value named 'update right shift' in this block. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, learning_rate, momentum, weight_decay, update_right_shift,)' to 'updated_weight, updated_velocity ' in host memory. Here updated_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function execute, which handles execute.

Line 270: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 262. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, learning_rate, momentum, weight_decay, update_right_shift,)' to 'updated_weight, updated_velocity ' in host memory. Here updated_weight

means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function execute, which handles execute.

Line 271: `result = tuple(` This statement continues through source line 275; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'tuple((int(updated_weight[lane]) & 0xFFFF) | ((int(updated_velocity[lane]) & 0xFFFFFF) << 16) for lane in range(8))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; updated_weight: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 272: `(int(updated_weight[lane]) & 0xFFFF)` This is continuation line 2 of the 5-line logical statement that begins at source line 271. Its exact fragment is '(int(updated_weight[lane]) & 0xFFFF)'. The important operands are updated_weight: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'tuple((int(updated_weight[lane]) & 0xFFFF) | ((int(updated_velocity[lane]) & 0xFFFFFF) << 16) for lane in range(8))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; updated_weight: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 273: `| ((int(updated_velocity[lane]) & 0xFFFFFF) << 16)` This is continuation line 3 of the 5-line logical statement that begins at source line 271. Its exact fragment is '| ((int(updated_velocity[lane]) & 0xFFFFFF) << 16)'. The important operands are updated_velocity: the momentum history retained by the SGD optimizer; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'tuple((int(updated_weight[lane]) & 0xFFFF) | ((int(updated_velocity[lane]) & 0xFFFFFF) << 16) for lane in range(8))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; updated_weight: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 274: `for lane in range(8)` This is continuation line 4 of the 5-line logical statement that begins at source line 271. Its exact fragment is 'for lane in range(8)'. The important operands are for: the value named 'for' in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'tuple((int(updated_weight[lane]) & 0xFFFF) | ((int(updated_velocity[lane]) & 0xFFFFFF) << 16) for lane in range(8))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; updated_weight: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 275: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 271. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'tuple((int(updated_weight[lane]) & 0xFFFF) | ((int(updated_velocity[lane]) & 0xFFFFFF) << 16) for lane in range(8))' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are tuple: the value named 'tuple' in this block; updated_weight: a learned coefficient multiplied by an activation; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 276: `elif command.opcode == Opcode.LOG_SOFTMAX:` This executes the following indented host-code branch only when 'command.opcode == Opcode.LOG_SOFTMAX' is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; opcode: the value named 'opcode' in this block; Opcode: the value named 'opcode' in this block. Context: the function execute, which handles execute.

Line 277: `count = (command.value >> 4) & 0x1FF` This assigns '(command.value >> 4) & 0x1FF' to 'count ' in host memory. Here count means the value named 'count' in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 278: `difference_shift = (command.value >> 13) & 0x1F` This assigns ‘(command.value >> 13) & 0x1F‘ to ‘difference_shift ‘ in host memory. Here difference_shift means the value named ’difference shift’ in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 279: `scores = np.asarray(` This statement continues through source line 280; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.asarray([_signed(word, 24) for word in words[:count]], dtype=np.int64)‘ to ‘scores ‘ in host memory. Here scores means the value named ’scores’ in this block. The important operands are asarray: the value named ’asarray’ in this block; _signed: the value named ’signed’ in this block; word: the value named ’word’ in this block. Context: the function execute, which handles execute.

Line 280: `[_signed(word, 24) for word in words[:count]], dtype=np.int64)` This is continuation line 2 of the 2-line logical statement that begins at source line 279. Its exact fragment is ‘[_signed(word, 24) for word in words[:count]], dtype=np.int64)‘. The important operands are _signed: the value named ’signed’ in this block; word: the value named ’word’ in this block; for: the value named ’for’ in this block. Read together, the complete operation is: This assigns ‘np.asarray([_signed(word, 24) for word in words[:count]], dtype=np.int64)‘ to ‘scores ‘ in host memory. Here scores means the value named ’scores’ in this block. The important operands are asarray: the value named ’asarray’ in this block; _signed: the value named ’signed’ in this block; word: the value named ’word’ in this block. Context: the function execute, which handles execute.

Line 281: `probabilities = log_int_softmax_q15(scores, difference_shift)` This assigns ‘log_int_softmax_q15(scores, difference_shift)‘ to ‘probabilities ‘ in host memory. Here probabilities means the value named ’probabilities’ in this block. The important operands are log_int_softmax_q15: the value named ’log int softmax q15’ in this block; scores: the value named ’scores’ in this block; difference_shift: the value named ’difference shift’ in this block. Context: the function execute, which handles execute.

Line 282: `result = tuple(` This statement continues through source line 287; the explanation uses the complete expression rather than only the first visual fragment. Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are result: the value named ’result’ in this block; tuple: the value named ’tuple’ in this block; probabilities: the value named ’probabilities’ in this block.

Line 283: `int(probabilities[index])` This is continuation line 2 of the 6-line logical statement that begins at source line 282. Its exact fragment is ‘int(probabilities[index])‘. The important operands are probabilities: the value named ’probabilities’ in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are result: the value named ’result’ in this block; tuple: the value named ’tuple’ in this block; probabilities: the value named ’probabilities’ in this block.

Line 284: `| (index << 16)` This is continuation line 3 of the 6-line logical statement that begins at source line 282. Its exact fragment is ‘| (index << 16)‘. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are result: the value named ’result’ in this block; tuple: the value named ’tuple’ in this block; probabilities: the value named ’probabilities’ in this block.

Line 285: `| (int(index == count - 1) << 25)` This is continuation line 4 of the 6-line logical statement that begins at source line 282. Its exact fragment is ‘| (int(index == count - 1) << 25)‘. The important operands are index: an integer selecting one tensor element, lane, bank, or state entry; count: the value named ’count’ in this block. Read together, the complete operation is: Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are result: the value named ’result’ in this block; tuple: the value named ’tuple’ in this block; probabilities: the value named ’probabilities’ in this block.

Line 286: `for index in range(count)` This is continuation line 5 of the 6-line logical statement that begins at source line 282. Its exact fragment is 'for index in range(count)'. The important operands are for: the value named 'for' in this block; index: an integer selecting one tensor element, lane, bank, or state entry; in: the value named 'in' in this block. Read together, the complete operation is: Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are result: the value named 'result' in this block; tuple: the value named 'tuple' in this block; probabilities: the value named 'probabilities' in this block.

Line 287: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 282. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are result: the value named 'result' in this block; tuple: the value named 'tuple' in this block; probabilities: the value named 'probabilities' in this block.

Line 288: `elif command.opcode == Opcode.LIF_STEP:` This executes the following indented host-code branch only when 'command.opcode == Opcode.LIF_STEP' is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; opcode: the value named 'opcode' in this block; Opcode: the value named 'opcode' in this block. Context: the function execute, which handles execute.

Line 289: `leak_shift = (command.value >> 4) & 0x1F` This assigns '(command.value >> 4) & 0x1F' to 'leak_shift ' in host memory. Here leak_shift means the retained fraction of prior membrane state. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 290: `threshold = _signed(command.value >> 9, 24)` This assigns '_signed(command.value >> 9, 24)' to 'threshold ' in host memory. Here threshold means the membrane level at which a neuron emits a spike. The important operands are _signed: the value named 'signed' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 291: `reset = _signed(command.value >> 33, 24)` This assigns '_signed(command.value >> 33, 24)' to 'reset ' in host memory. Here reset means the value named 'reset' in this block. The important operands are _signed: the value named 'signed' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 292: `if (command.value >> 57) & 1:` This executes the following indented host-code branch only when '(command.value >> 57) & 1' is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named 'value' in this block. Context: the function execute, which handles execute.

Line 293: `self.lif_membrane.fill(reset)` This calls 'self.lif_membrane.fill' with 'reset' inside the function execute, which handles execute. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 294: `self.lif_timestep = 0` This assigns '0' to 'self.lif_timestep ' in host memory. Here lif_timestep means one discrete SNN update instant. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 295: `currents = np.asarray([` This statement continues through source line 298; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.asarray([_signed(word >> (16 * lane), 16) for word in words for lane in range(4)], dtype=np.int64)' to 'currents ' in host memory. Here currents means the value named 'currents' in this block. The important operands are asarray: the value named 'asarray' in this block; _signed: the value named 'signed' in this block; word: the value named 'word' in this block. Context: the function execute, which handles execute.

Line 296: `_signed(word >> (16 * lane), 16)` This is continuation line 2 of the 4-line logical statement that begins at source line 295. Its exact fragment is '_signed(word >> (16 * lane), 16)'. The important operands are _signed: the value named 'signed' in this block; word: the value named 'word' in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns 'np.asarray([_signed(word >> (16 * lane), 16) for word in words for lane in range(4)], dtype=np.int64)' to 'currents ' in host memory. Here currents means the value named 'currents' in this block. The important operands are asarray: the value named 'asarray' in this block; _signed: the value named 'signed' in this block; word: the value named 'word' in this block. Context: the function execute, which handles execute.

Line 297: `for word in words for lane in range(4)` This is continuation line 3 of the 4-line logical statement that begins at source line 295. Its exact fragment is 'for word in words for lane in range(4)'. The important operands are for: the value named 'for' in this block; word: the value named 'word' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns 'np.asarray([_signed(word >> (16 * lane), 16) for word in words for lane in range(4)], dtype=np.int64)' to 'currents ' in host memory. Here currents means the value named 'currents' in this block. The important operands are asarray: the value named 'asarray' in this block; _signed: the value named 'signed' in this block; word: the value named 'word' in this block. Context: the function execute, which handles execute.

Line 298: `], dtype=np.int64)` This is continuation line 4 of the 4-line logical statement that begins at source line 295. Its exact fragment is '], dtype=np.int64)'. The important operands are dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'np.asarray([_signed(word >> (16 * lane), 16) for word in words for lane in range(4)], dtype=np.int64)' to 'currents ' in host memory. Here currents means the value named 'currents' in this block. The important operands are asarray: the value named 'asarray' in this block; _signed: the value named 'signed' in this block; word: the value named 'word' in this block. Context: the function execute, which handles execute.

Line 299: `self.lif_membrane = saturate_signed(` This statement continues through source line 302; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'saturate_signed(self.lif_membrane - (self.lif_membrane >> leak_shift) + currents, 24,)' to 'self.lif_membrane ' in host memory. Here lif_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are saturate_signed: the value named 'saturate signed' in this block; lif_membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function execute, which handles execute.

Line 300: `self.lif_membrane - (self.lif_membrane >> leak_shift) + currents,` This is continuation line 2 of the 4-line logical statement that begins at source line 299. Its exact fragment is 'self.lif_membrane - (self.lif_membrane >> leak_shift) + currents,'. The important operands are lif_membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state; currents: the value named 'currents' in this block. Read together, the complete operation is: This assigns 'saturate_signed(self.lif_membrane - (self.lif_membrane >> leak_shift) + currents, 24,)' to 'self.lif_membrane ' in host memory. Here lif_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are saturate_signed: the value named 'saturate signed' in this block; lif_membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function execute, which handles execute.

Line 301: `24,` This is continuation line 3 of the 4-line logical statement that begins at source line 299. Its exact fragment is '24,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'saturate_signed(self.lif_membrane - (self.lif_membrane >> leak_shift) + currents, 24,)' to 'self.lif_membrane ' in host memory. Here lif_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are saturate_signed: the value named 'saturate signed' in this block; lif_membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function execute, which handles execute.

Line 302: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 299. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'saturate_signed(self.lif_membrane - (self.lif_membrane >> leak_shift) + currents, 24,)' to 'self.lif_membrane ' in host memory. Here lif_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are saturate_signed: the value named 'saturate signed' in this block; lif_membrane:

the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function execute, which handles execute.

Line 303: `pre_reset = self.lif_membrane.copy()` This assigns ‘self.lif_membrane.copy()‘ to ‘pre_reset ‘ in host memory. Here pre_reset means the value named ’pre reset’ in this block. The important operands are lif_membrane: the persistent LIF neuron state carrying charge between timesteps; copy: the value named ’copy’ in this block. Context: the function execute, which handles execute.

Line 304: `fired = pre_reset >= threshold` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are fired: the value named ’fired’ in this block; pre_reset: the value named ’pre reset’ in this block; threshold: the membrane level at which a neuron emits a spike.

Line 305: `self.lif_membrane = np.where(fired, reset, pre_reset)` This assigns ‘np.where(fired, reset, pre_reset)‘ to ‘self.lif_membrane ‘ in host memory. Here lif_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are where: the value named ’where’ in this block; fired: the value named ’fired’ in this block; reset: the value named ’reset’ in this block. Context: the function execute, which handles execute.

Line 306: `self.lif_timestep += 1` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are lif_timestep: one discrete SNN update instant.

Line 307: `spike_word = sum(int(fired[lane]) << lane for lane in range(32))` This assigns ‘sum(int(fired[lane]) << lane for lane in range(32))‘ to ‘spike_word ‘ in host memory. Here spike_word means a binary event produced when a neuron crosses its threshold. The important operands are sum: the value named ’sum’ in this block; fired: the value named ’fired’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 308: `output = [spike_word | (self.lif_timestep << 32)]` This assigns ‘[spike_word | (self.lif_timestep << 32)]‘ to ‘output ‘ in host memory. Here output means the value named ’output’ in this block. The important operands are spike_word: a binary event produced when a neuron crosses its threshold; lif_timestep: one discrete SNN update instant. Context: the function execute, which handles execute.

Line 309: `if (command.value >> 58) & 1:` This executes the following indented host-code branch only when ‘(command.value >> 58) & 1‘ is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function execute, which handles execute.

Line 310: `packed = sum(` This statement continues through source line 313; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sum((int(pre_reset[lane]) & 0xFFFFFF) << (24 * lane) for lane in range(32))‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are sum: the value named ’sum’ in this block; pre_reset: the value named ’pre reset’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 311: `(int(pre_reset[lane]) & 0xFFFFFF) << (24 * lane)` This is continuation line 2 of the 4-line logical statement that begins at source line 310. Its exact fragment is ‘(int(pre_reset[lane]) & 0xFFFFFF) << (24 * lane)‘. The important operands are pre_reset: the value named ’pre reset’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns ‘sum((int(pre_reset[lane]) & 0xFFFFFF) << (24 * lane) for lane in range(32))‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are sum: the value named ’sum’ in this block; pre_reset: the value named ’pre reset’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 312: `for lane in range(32)` This is continuation line 3 of the 4-line logical statement that begins at source line 310. Its exact fragment is ‘for lane in range(32)‘. The important operands are for: the value named ’for’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘sum((int(pre_reset[lane]) & 0xFFFFFF) << (24 * lane) for lane in range(32))‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are sum: the value named ’sum’ in this block; pre_reset: the value named ’pre reset’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 313: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 310. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘sum((int(pre_reset[lane]) & 0xFFFFFF) << (24 * lane) for lane in range(32))‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are sum: the value named ’sum’ in this block; pre_reset: the value named ’pre reset’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute, which handles execute.

Line 314: `output.extend(` This statement continues through source line 317; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘output.extend‘ with ‘ (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(12) ‘ inside the function execute, which handles execute. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 315: `(packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF` This is continuation line 2 of the 4-line logical statement that begins at source line 314. Its exact fragment is ‘(packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF‘. The important operands are packed: the value named ’packed’ in this block; word: the value named ’word’ in this block. Read together, the complete operation is: This calls ‘output.extend‘ with ‘ (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(12) ‘ inside the function execute, which handles execute. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 316: `for word in range(12)` This is continuation line 3 of the 4-line logical statement that begins at source line 314. Its exact fragment is ‘for word in range(12)‘. The important operands are for: the value named ’for’ in this block; word: the value named ’word’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This calls ‘output.extend‘ with ‘ (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(12) ‘ inside the function execute, which handles execute. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 317: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 314. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘output.extend‘ with ‘ (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(12) ‘ inside the function execute, which handles execute. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 318: `result = tuple(output)` This assigns ‘tuple(output)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are tuple: the value named ’tuple’ in this block; output: the value named ’output’ in this block. Context: the function execute, which handles execute.

Line 319: `elif command.opcode == Opcode.CLEAR_STATUS:` This executes the following indented host-code branch only when ‘command.opcode == Opcode.CLEAR_STATUS‘ is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; opcode: the value named ’opcode’ in this block; Opcode: the value named ’opcode’ in this block. Context: the function execute, which handles execute.

Line 320: `result = ()` This assigns ‘()‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute, which handles execute.

Line 321: `else:` This begins the alternative host-code branch when the preceding condition was false, still inside the function execute, which handles execute.

Line 322: `raise AssertionError(f"unexpected fabric opcode {command.opcode}")` This rejects an invalid state by raising ‘AssertionError(f"unexpected fabric opcode {command.opcode}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 323: `if len(result) != output_beats:` This executes the following indented host-code branch only when ‘len(result) != output_beats‘ is true. The important operands are len: the value named ’len’ in this block; result: the value named ’result’ in this block; output_beats: the value named ’output beats’ in this block. Context: the function execute, which handles execute.

Line 324: `raise AssertionError(f"fabric emulator returned {len(result)} words, expected {output_beats}")` This rejects an invalid state by raising ‘AssertionError(f"fabric emulator returned {len(result)} words, expected {output_beats}")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 325: `return result` This returns ‘result‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are result: the value named ’result’ in this block.

Line 328: `class FullTrainingEmulator:` This defines class FullTrainingEmulator, which groups data and methods for full training emulator. Instances exist in host memory and organize the flow that checks neuraedge training behavior independently with Python assertions.

Line 329: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 330: `self.backward = BackwardRtlEmulator()` This assigns ‘BackwardRtlEmulator()‘ to ‘self.backward ‘ in host memory. Here backward means the value named ’backward’ in this block. The important operands are BackwardRtlEmulator: the value named ’backward rtl emulator’ in this block. Context: the function ___init___, which handles init.

Line 331: `self.fabric = FabricOperatorEmulator()` This assigns ‘FabricOperatorEmulator()‘ to ‘self.fabric ‘ in host memory. Here fabric means the value named ’fabric’ in this block. The important operands are FabricOperatorEmulator: the value named ’fabric operator emulator’ in this block. Context: the function ___init___, which handles init.

Line 332: `self.tensor = TensorTransactionEmulator()` This assigns ‘TensorTransactionEmulator()‘ to ‘self.tensor ‘ in host memory. Here tensor means the value named ’tensor’ in this block. The important operands are TensorTransactionEmulator: the value named ’tensor transaction emulator’ in this block. Context: the function ___init___, which handles init.

Line 334: `def execute(self, command, input_beats=(), *, output_beats=0, timeout_seconds=5.0):` This starts function execute. Its indented body performs execute as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 335: `if command.opcode == Opcode.TRAIN_LINEAR_BACKWARD:` This executes the following indented host-code branch only when ‘command.opcode == Opcode.TRAIN_LINEAR_BACKWARD‘ is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; opcode: the value named ’opcode’ in this block; Opcode: the value named ’opcode’ in this block. Context: the function execute, which handles execute.

Line 336: `return self.backward.execute(` This statement continues through source line 341; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘self.backward.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are backward: the value named ’backward’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 337: `command,` This is continuation line 2 of the 6-line logical statement that begins at source line 336. Its exact fragment is 'command,'. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This returns 'self.backward.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)' to the caller of the function execute, which handles execute, ending this invocation. The important operands are backward: the value named 'backward' in this block; execute: the value named 'execute' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 338: `input_beats,` This is continuation line 3 of the 6-line logical statement that begins at source line 336. Its exact fragment is 'input_beats,'. The important operands are input_beats: the value named 'input beats' in this block. Read together, the complete operation is: This returns 'self.backward.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)' to the caller of the function execute, which handles execute, ending this invocation. The important operands are backward: the value named 'backward' in this block; execute: the value named 'execute' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 339: `output_beats=output_beats,` This is continuation line 4 of the 6-line logical statement that begins at source line 336. Its exact fragment is 'output_beats=output_beats,'. The important operands are output_beats: the value named 'output beats' in this block. Read together, the complete operation is: This returns 'self.backward.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)' to the caller of the function execute, which handles execute, ending this invocation. The important operands are backward: the value named 'backward' in this block; execute: the value named 'execute' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 340: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 336. Its exact fragment is 'timeout_seconds=timeout_seconds,'. The important operands are timeout_seconds: the value named 'timeout seconds' in this block. Read together, the complete operation is: This returns 'self.backward.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)' to the caller of the function execute, which handles execute, ending this invocation. The important operands are backward: the value named 'backward' in this block; execute: the value named 'execute' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 341: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 336. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns 'self.backward.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)' to the caller of the function execute, which handles execute, ending this invocation. The important operands are backward: the value named 'backward' in this block; execute: the value named 'execute' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 342: `if command.opcode in {` This line opens or closes a multi-line Python expression inside the function execute, which handles execute; the neighboring lines supply its arguments or elements.

Line 343: `Opcode.LOAD_ACTIVATIONS,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are Opcode: the value named 'opcode' in this block; LOAD_ACTIVATIONS: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 344: `Opcode.LOAD_WEIGHTS_OUTER,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are Opcode: the value named 'opcode' in this block; LOAD_WEIGHTS_OUTER: a learned coefficient multiplied by an activation.

Line 345: `Opcode.LOAD_WEIGHTS_DOT8,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are Opcode: the value named 'opcode' in this block; LOAD_WEIGHTS_DOT8: a learned coefficient multiplied by an activation.

Line 346: `Opcode.COMPUTE_DENSE_OUTER,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block.

Line 347: `Opcode.COMPUTE_DENSE_DOT8,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_DENSE_DOT8: the value named 'compute dense dot8' in this block.

Line 348: `Opcode.COMPUTE_BINARY,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_BINARY: the value named 'compute binary' in this block.

Line 349: `Opcode.COMPUTE_TERNARY,` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are Opcode: the value named 'opcode' in this block; COMPUTE_TERNARY: the value named 'compute ternary' in this block.

Line 350: `}:` Inside the function execute, which handles execute, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The expression is a literal or structural operator and has no hidden software state.

Line 351: `return self.tensor.execute(` This statement continues through source line 356; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘self.tensor.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are tensor: the value named 'tensor' in this block; execute: the value named 'execute' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 352: `command,` This is continuation line 2 of the 6-line logical statement that begins at source line 351. Its exact fragment is ‘command,‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This returns ‘self.tensor.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are tensor: the value named 'tensor' in this block; execute: the value named 'execute' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 353: `input_beats,` This is continuation line 3 of the 6-line logical statement that begins at source line 351. Its exact fragment is ‘input_beats,‘. The important operands are input_beats: the value named 'input beats' in this block. Read together, the complete operation is: This returns ‘self.tensor.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are tensor: the value named 'tensor' in this block; execute: the value named 'execute' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 354: `output_beats=output_beats,` This is continuation line 4 of the 6-line logical statement that begins at source line 351. Its exact fragment is ‘output_beats=output_beats,‘. The important operands are output_beats: the value named 'output beats' in this block. Read together, the complete operation is: This returns ‘self.tensor.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are tensor: the value named 'tensor' in this block; execute: the value named 'execute' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 355: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 351. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This returns ‘self.tensor.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are tensor: the value named ’tensor’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 356: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 351. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘self.tensor.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are tensor: the value named ’tensor’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 357: `return self.fabric.execute(` This statement continues through source line 362; the explanation uses the complete expression rather than only the first visual fragment. This returns ‘self.fabric.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are fabric: the value named ’fabric’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 358: `command,` This is continuation line 2 of the 6-line logical statement that begins at source line 357. Its exact fragment is ‘command,‘. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This returns ‘self.fabric.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are fabric: the value named ’fabric’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 359: `input_beats,` This is continuation line 3 of the 6-line logical statement that begins at source line 357. Its exact fragment is ‘input_beats,‘. The important operands are input_beats: the value named ’input beats’ in this block. Read together, the complete operation is: This returns ‘self.fabric.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are fabric: the value named ’fabric’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 360: `output_beats=output_beats,` This is continuation line 4 of the 6-line logical statement that begins at source line 357. Its exact fragment is ‘output_beats=output_beats,‘. The important operands are output_beats: the value named ’output beats’ in this block. Read together, the complete operation is: This returns ‘self.fabric.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are fabric: the value named ’fabric’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 361: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 357. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This returns ‘self.fabric.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are fabric: the value named ’fabric’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 362: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 357. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the

complete operation is: This returns ‘self.fabric.execute(command, input_beats, output_beats=output_beats, timeout_seconds=timeout_seconds,)‘ to the caller of the function execute, which handles execute, ending this invocation. The important operands are fabric: the value named ’fabric’ in this block; execute: the value named ’execute’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 364: `def execute_program(self, records, *, timeout_seconds=30.0) -> tuple[int, ...]:` This starts function execute_program. Its indented body performs execute program as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 365: `output: list[int] = []` This assigns ‘[]‘ to ‘output‘ in host memory. Here output means the value named ’output’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 366: `for record in records:` This iterates record over ‘records‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_program, which handles execute program.

Line 367: `output.extend(self.execute(` This statement continues through source line 372; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 368: `record.command,` This is continuation line 2 of the 6-line logical statement that begins at source line 367. Its exact fragment is ‘record.command,‘. The important operands are record: the value named ’record’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Read together, the complete operation is: This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 369: `record.input_beats,` This is continuation line 3 of the 6-line logical statement that begins at source line 367. Its exact fragment is ‘record.input_beats,‘. The important operands are record: the value named ’record’ in this block; input_beats: the value named ’input beats’ in this block. Read together, the complete operation is: This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 370: `output_beats=record.output_beats,` This is continuation line 4 of the 6-line logical statement that begins at source line 367. Its exact fragment is ‘output_beats=record.output_beats,‘. The important operands are output_beats: the value named ’output beats’ in this block; record: the value named ’record’ in this block. Read together, the complete operation is: This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 371: `timeout_seconds=timeout_seconds,` This is continuation line 5 of the 6-line logical statement that begins at source line 367. Its exact fragment is ‘timeout_seconds=timeout_seconds,‘. The important operands are timeout_seconds: the value named ’timeout seconds’ in this block. Read together, the complete operation is: This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 372: `))` This is continuation line 6 of the 6-line logical statement that begins at source line 367. Its exact fragment is ‘))‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘output.extend‘ with ‘self.execute(record.command, record.input_beats, output_beats=record.output_beats, timeout_seconds=timeout_seconds,)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 373: `return tuple(output)` This returns ‘tuple(output)‘ to the caller of the function execute_program, which handles execute program, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; output: the value named ’output’ in this block.

Line 376: `class TrainingCompilerTests(unittest.TestCase):` This defines class TrainingCompilerTests, which groups data and methods for training compiler tests. Instances exist in host memory and organize the flow that checks neuraedge training behavior independently with Python assertions.

Line 377: `def setUp(self) -> None:` This starts function setUp. Its indented body performs set up as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 378: `self.rng = np.random.default_rng(20260713)` This assigns ‘np.random.default_rng(20260713)‘ to ‘self.rng ‘ in host memory. Here rng means the value named ’rng’ in this block. The important operands are random: the value named ’random’ in this block; default_rng: the value named ’default rng’ in this block. Context: the function setUp, which handles set up.

Line 380: `def test_arbitrary_linear_shape_uses_exact_two_pass_accumulation(self) -> None:` This starts function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation. Its indented body performs test arbitrary linear shape uses exact two pass accumulation as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 381: `x = self.rng.integers(-128, 128, size=(13, 11), dtype=np.int64)` This assigns ‘self.rng.integers(-128, 128, size=(13, 11), dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 382: `w = self.rng.integers(-32768, 32768, size=(11, 17), dtype=np.int64)` This assigns ‘self.rng.integers(-32768, 32768, size=(11, 17), dtype=np.int64)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 383: `dy = self.rng.integers(-32768, 32768, size=(13, 17), dtype=np.int64)` This assigns ‘self.rng.integers(-32768, 32768, size=(13, 17), dtype=np.int64)‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation which handles test arbitrary linear shape uses exact two pass accumulation.

Line 384: `program = compile_linear_backward(` This statement continues through source line 390; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_linear_backward(x, w, dy, input_gradient_shift=4, weight_gradient_shift=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 385: `x,` This is continuation line 2 of the 7-line logical statement that begins at source line 384. Its exact fragment is ‘x,‘. The important operands are x: the value named ’x’ in this block. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy, input_gradient_shift=4, weight_gradient_shift=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 386: `w,` This is continuation line 3 of the 7-line logical statement that begins at source line 384. Its exact fragment is ‘w,‘. The important operands are w: the value named ’w’ in this block. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy, input_gradient_shift=4, weight_gradient_shift=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 387: `dy,` This is continuation line 4 of the 7-line logical statement that begins at source line 384. Its exact fragment is ‘dy,‘. The important operands are dy: the value named ’dy’ in this block. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy, input_gradient_shift=4, weight_gradient_shift=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 388: `input_gradient_shift=4,` This is continuation line 5 of the 7-line logical statement that begins at source line 384. Its exact fragment is ‘input_gradient_shift=4,‘. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy, input_gradient_shift=4, weight_gradient_shift=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 389: `weight_gradient_shift=3,` This is continuation line 6 of the 7-line logical statement that begins at source line 384. Its exact fragment is ‘weight_gradient_shift=3,‘. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy, input_gradient_shift=4, weight_gradient_shift=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 390: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 384. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘compile_linear_backward(x, w, dy, input_gradient_shift=4, weight_gradient_shift=3,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_linear_backward: the value named ’compile linear backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 391: `self.assertEqual(program.shape, (13, 11, 17))` This calls ‘self.assertEqual‘ with ‘program.shape, (13, 11, 17)‘ inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 392: `self.assertEqual(program.partial_tile_count, 24)` This calls ‘self.assertEqual‘ with ‘program.partial_tile_coun 24‘ inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 393: `self.assertEqual(program.mathematical_multiplications, 2 * 13 * 11 * 17)` This calls ‘self.assertEqual‘ with ‘program.mathematical_multiplications, 2 * 13 * 11 * 17‘ inside the function test_arbitrary_linear_shape_uses_exact_two_p which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 394: `self.assertEqual(sum(` This statement continues through source line 397; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertEqual‘ with ‘sum(instruction.mode == LinearGradientMode.INPUT for instruction in program.instructions), 12‘ inside the function test_arbitrary_linear_shape_uses_ex which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 395: `instruction.mode == LinearGradientMode.INPUT` This is continuation line 2 of the 4-line logical statement that begins at source line 394. Its exact fragment is ‘instruction.mode == LinearGradientMode.INPUT‘. The important operands are instruction: the value named ’instruction’ in this block; mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘sum(instruction.mode == LinearGradientMode.INPUT for instruction in program.instructions), 12‘ inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 396: `for instruction in program.instructions` This is continuation line 3 of the 4-line logical statement that begins at source line 394. Its exact fragment is ‘for instruction in program.instructions‘. The important operands are for: the value named ’for’ in this block; instruction: the value named ’instruction’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘sum(instruction.mode == LinearGradientMode.INPUT for instruction in program.instructions), 12‘ inside the function test_arbitrary_linear_shape_uses_exact_two_ which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 397: `), 12)` This is continuation line 4 of the 4-line logical statement that begins at source line 394. Its exact fragment is ‘), 12)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘sum(instruction.mode == LinearGradientMode.INPUT for instruction in program.instructions), 12‘ inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 398: `self.assertEqual(sum(` This statement continues through source line 401; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘self.assertEqual‘ with ‘sum(instruction.mode == LinearGradientMode.WEIGHT for instruction in program.instructions), 12‘ inside the function test_arbitrary_linear_shape_uses_ which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 399: `instruction.mode == LinearGradientMode.WEIGHT` This is continuation line 2 of the 4-line logical statement that begins at source line 398. Its exact fragment is ‘instruction.mode == LinearGradientMode.WEIGHT‘. The important operands are instruction: the value named ’instruction’ in this block; mode: the value named ’mode’ in this block; LinearGradientMode: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls ‘self.assertEqual‘ with ‘sum(instruction.mode == LinearGradientMode.WEIGHT for instruction in program.instructions), 12‘ inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 400: `for instruction in program.instructions` This is continuation line 3 of the 4-line logical statement that begins at source line 398. Its exact fragment is 'for instruction in program.instructions'. The important operands are for: the value named 'for' in this block; instruction: the value named 'instruction' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with 'sum(instruction.mode == LinearGradientMode.WEIGHT for instruction in program.instructions), 12' inside the function test_arbitrary_linear_shape_uses_exact_two which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 401: `), 12)` This is continuation line 4 of the 4-line logical statement that begins at source line 398. Its exact fragment is '), 12)'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with 'sum(instruction.mode == LinearGradientMode.WEIGHT for instruction in program.instructions), 12' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 402: `for instruction in program.instructions:` This iterates instruction over 'program.instructions'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 403: `self.assertEqual(` This statement continues through source line 406; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertEqual' with ' instruction.record.output_beats == 0, instruction.suppress_output, ' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 404: `instruction.record.output_beats == 0,` This is continuation line 2 of the 4-line logical statement that begins at source line 403. Its exact fragment is 'instruction.record.output_beats == 0,'. The important operands are instruction: the value named 'instruction' in this block; record: the value named 'record' in this block; output_beats: the value named 'output beats' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' instruction.record.output_beats == 0, instruction.suppress_output, ' inside the function test_arbitrary_linear_shape_uses_exact_t which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 405: `instruction.suppress_output,` This is continuation line 3 of the 4-line logical statement that begins at source line 403. Its exact fragment is 'instruction.suppress_output,'. The important operands are instruction: the value named 'instruction' in this block; suppress_output: the value named 'suppress output' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' instruction.record.output_beats == 0, instruction.suppress_output, ' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 406: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 403. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with ' instruction.record.output_beats == 0, instruction.suppress_output, ' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 407: `self.assertEqual(` This statement continues through source line 410; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertEqual' with ' bool((instruction.record.command.value >> 28) & 1), instruction.accumulate, ' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 408: `bool((instruction.record.command.value >> 28) & 1),` This is continuation line 2 of the 4-line logical statement that begins at source line 407. Its exact fragment is 'bool((instruction.record.command.value >> 28) & 1),'. The important operands are bool: the value named 'bool' in this block; instruction: the value named 'instruction' in this block; record: the value named 'record' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' bool((instruction.record.command.value >> 28) & 1), instruction.accumulate, ' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 409: `instruction.accumulate,` This is continuation line 3 of the 4-line logical statement that begins at source line 407. Its exact fragment is 'instruction.accumulate,'. The important operands are instruction: the value named 'instruction' in this block; accumulate: the value named 'accumulate' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' bool((instruction.record.command.value >> 28) & 1), instruction.accumulate, ' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 410: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 407. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with ' bool((instruction.record.command.value >> 28) & 1), instruction.accumulate, ' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 412: `expected_dx = saturate_signed(round_shift_signed(dy @ w.T, 4), 32)` This assigns 'saturate_signed(round_ @ w.T, 4), 32)' to 'expected_dx ' in host memory. Here expected_dx means the value named 'expected dx' in this block. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; dy: the value named 'dy' in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 413: `expected_dw = saturate_signed(round_shift_signed(x.T @ dy, 3), 24)` This assigns 'saturate_signed(round_ @ dy, 3), 24)' to 'expected_dw ' in host memory. Here expected_dw means the value named 'expected dw' in this block. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; x: the value named 'x' in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 414: `got_dx, got_dw = program.execute(BackwardRtlEmulator())` This assigns 'program.execute(BackwardRtlEmulat to 'got_dx, got_dw ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are program: the value named 'program' in this block; execute: the value named 'execute' in this block; BackwardRtlEmulator: the value named 'backward rtl emulator' in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_ which handles test arbitrary linear shape uses exact two pass accumulation.

Line 415: `np.testing.assert_array_equal(got_dx, expected_dx)` This calls 'np.testing.assert_array_equal' with 'got_dx, expected_dx' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 416: `np.testing.assert_array_equal(got_dw, expected_dw)` This calls 'np.testing.assert_array_equal' with 'got_dw, expected_dw' inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 417: `got_dx, got_dw = program.execute_batched(BackwardRtlEmulator())` This assigns ‘program.execute_batched(B to ‘got_dx, got_dw ‘ in host memory. Here got_dx means the value named ’got dx’ in this block. The important operands are program: the value named ’program’ in this block; execute_batched: the value named ’execute batched’ in this block; BackwardRtlEmulator: the value named ’backward rtl emulator’ in this block. Context: the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation.

Line 418: `np.testing.assert_array_equal(got_dx, expected_dx)` This calls ‘np.testing.assert_array_equal‘ with ‘got_dx, expected_dx‘ inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 419: `np.testing.assert_array_equal(got_dw, expected_dw)` This calls ‘np.testing.assert_array_equal‘ with ‘got_dw, expected_dw‘ inside the function test_arbitrary_linear_shape_uses_exact_two_pass_accumulation, which handles test arbitrary linear shape uses exact two pass accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 421: `def test_affine_augmentation_returns_exact_bias_gradient(self) -> None:` This starts function test_affine_augmentation_returns_exact_bias_gradient. Its indented body performs test affine augmentation returns exact bias gradient as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 422: `x = self.rng.integers(-32, 33, size=(9, 10), dtype=np.int64)` This assigns ‘self.rng.integers(-32, 33, size=(9, 10), dtype=np.int64)‘ to ‘x ‘ in host memory. Here x means the value named ’x’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 423: `w = self.rng.integers(-2000, 2001, size=(10, 7), dtype=np.int64)` This assigns ‘self.rng.integers(-2000, 2001, size=(10, 7), dtype=np.int64)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 424: `bias = self.rng.integers(-3000, 3001, size=7, dtype=np.int64)` This assigns ‘self.rng.integers(-3000, 3001, size=7, dtype=np.int64)‘ to ‘bias ‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient which handles test affine augmentation returns exact bias gradient.

Line 425: `dy = self.rng.integers(-1000, 1001, size=(9, 7), dtype=np.int64)` This assigns ‘self.rng.integers(-1000, 1001, size=(9, 7), dtype=np.int64)‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 426: `program = compile_affine_backward(` This statement continues through source line 433; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘compile_affine_backward(x, w, bias, dy, input_gradient_shift=2, weight_gradient_shift=1,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_affine_backward: the value named ’compile affine backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 427: `x,` This is continuation line 2 of the 8-line logical statement that begins at source line 426. Its exact fragment is ‘x,‘. The important operands are x: the value named ’x’ in this block. Read together, the complete operation is: This assigns ‘compile_affine_backward(x, w, bias, dy, input_gradient_shift=2, weight_gradient_shift=1,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_affine_backward: the value named ’compile affine backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 428: `w,` This is continuation line 3 of the 8-line logical statement that begins at source line 426. Its exact fragment is ‘w,‘. The important operands are w: the value named ’w’ in this block. Read together, the complete operation is: This assigns ‘compile_affine_backward(x, w, bias, dy, input_gradient_shift=2, weight_gradient_shift=1,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_affine_backward: the value named ’compile affine backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 429: `bias,` This is continuation line 4 of the 8-line logical statement that begins at source line 426. Its exact fragment is ‘bias,‘. The important operands are bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns ‘compile_affine_backward(x, w, bias, dy, input_gradient_shift=2, weight_gradient_shift=1,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_affine_backward: the value named ’compile affine backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_affine_augmentation_returns_exact_ which handles test affine augmentation returns exact bias gradient.

Line 430: `dy,` This is continuation line 5 of the 8-line logical statement that begins at source line 426. Its exact fragment is ‘dy,‘. The important operands are dy: the value named ’dy’ in this block. Read together, the complete operation is: This assigns ‘compile_affine_backward(x, w, bias, dy, input_gradient_shift=2, weight_gradient_shift=1,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_affine_backward: the value named ’compile affine backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 431: `input_gradient_shift=2,` This is continuation line 6 of the 8-line logical statement that begins at source line 426. Its exact fragment is ‘input_gradient_shift=2,‘. The important operands are input_gradient_shift: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘compile_affine_backward(x, w, bias, dy, input_gradient_shift=2, weight_gradient_shift=1,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_affine_backward: the value named ’compile affine backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 432: `weight_gradient_shift=1,` This is continuation line 7 of the 8-line logical statement that begins at source line 426. Its exact fragment is ‘weight_gradient_shift=1,‘. The important operands are weight_gradient_shift: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘compile_affine_backward(x, w, bias, dy, input_gradient_shift=2, weight_gradient_shift=1,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_affine_backward: the value named ’compile affine backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 433: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 426. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘compile_affine_backward(x, w, bias, dy, input_gradient_shift=2, weight_gradient_shift=1,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_affine_backward: the value named ’compile affine backward’ in this block; x: the value named ’x’ in this block;

w: the value named 'w' in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 434: `got_dx, got_dw, got_db = program.execute_batched(BackwardRtlEmulator())` This assigns 'program.execute_ to 'got_dx, got_dw, got_db ' in host memory. Here got_dx means the value named 'got dx' in this block. The important operands are program: the value named 'program' in this block; execute_batched: the value named 'execute batched' in this block; BackwardRtlEmulator: the value named 'backward rtl emulator' in this block. Context: the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient.

Line 435: `np.testing.assert_array_equal(` This statement continues through source line 438; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_array_equal' with ' got_dx, saturate_signed(round_shift_signed(dy @ w.T, 2), 32), ' inside the function test_affine_augmentation_returns_exact_bia which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 436: `got_dx,` This is continuation line 2 of the 4-line logical statement that begins at source line 435. Its exact fragment is 'got_dx,'. The important operands are got_dx: the value named 'got dx' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' got_dx, saturate_signed(round_shift_signed(dy @ w.T, 2), 32), ' inside the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 437: `saturate_signed(round_shift_signed(dy @ w.T, 2), 32),` This is continuation line 3 of the 4-line logical statement that begins at source line 435. Its exact fragment is 'saturate_signed(round_shift_signed(dy @ w.T, 2), 32),'. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; dy: the value named 'dy' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' got_dx, saturate_signed(round_shift_signed(dy @ w.T, 2), 32), ' inside the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 438: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 435. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' got_dx, saturate_signed(round_shift_signed(dy @ w.T, 2), 32), ' inside the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 439: `np.testing.assert_array_equal(` This statement continues through source line 442; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_array_equal' with ' got_dw, saturate_signed(round_shift_signed(x.T @ dy, 1), 24), ' inside the function test_affine_augmentation_returns_exact_bia which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 440: `got_dw,` This is continuation line 2 of the 4-line logical statement that begins at source line 439. Its exact fragment is 'got_dw,'. The important operands are got_dw: the value named 'got dw' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' got_dw, saturate_signed(round_shift_signed(x.T @ dy, 1), 24), ' inside the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 441: `saturate_signed(round_shift_signed(x.T @ dy, 1), 24),` This is continuation line 3 of the 4-line logical statement that begins at source line 439. Its exact fragment is 'saturate_signed(round_shift_signed(x.T @ dy, 1), 24),'. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed:

the value named 'round shift signed' in this block; x: the value named 'x' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' got_dw, saturate_signed(round_shift_signed(x.T @ dy, 1), 24), ' inside the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 442: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 439. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' got_dw, saturate_signed(round_shift_signed(x.T @ dy, 1), 24), ' inside the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 443: `np.testing.assert_array_equal(` This statement continues through source line 446; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_array_equal' with ' got_db, saturate_signed(round_shift_signed(dy.sum(axis=0), 1), 24), ' inside the function test_affine_augmentation_returns_exa which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 444: `got_db,` This is continuation line 2 of the 4-line logical statement that begins at source line 443. Its exact fragment is 'got_db,'. The important operands are got_db: the value named 'got db' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' got_db, saturate_signed(round_shift_signed(dy.sum(axis= 1), 24), ' inside the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 445: `saturate_signed(round_shift_signed(dy.sum(axis=0), 1), 24),` This is continuation line 3 of the 4-line logical statement that begins at source line 443. Its exact fragment is 'saturate_signed(round_shift_signed(dy.sum(axis=0), 1), 24),'. The important operands are saturate_signed: the value named 'saturate signed' in this block; round_shift_signed: the value named 'round shift signed' in this block; dy: the value named 'dy' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' got_db, saturate_signed(round_shift_signed(dy.sum(axis=0), 1), 24), ' inside the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 446: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 443. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' got_db, saturate_signed(round_shift_signed(dy.sum(axis=0), 1), 24), ' inside the function test_affine_augmentation_returns_exact_bias_gradient, which handles test affine augmentation returns exact bias gradient. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 448: `def test_grouped_convolution_backward_and_fabric_col2im(self) -> None:` This starts function test_grouped_convolution_backward_and_fabric_col2im. Its indented body performs test grouped convolution backward and fabric col2im as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 449: `x = self.rng.integers(-8, 9, size=(1, 4, 4, 5), dtype=np.int64)` This assigns 'self.rng.integers(-8, 9, size=(1, 4, 4, 5), dtype=np.int64)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 450: `w = self.rng.integers(-20, 21, size=(4, 2, 3, 3), dtype=np.int64)` This assigns 'self.rng.integers(-20, 21, size=(4, 2, 3, 3), dtype=np.int64)' to 'w ' in host memory. Here w means the value named 'w' in this block. The

important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 451: `bias = self.rng.integers(-30, 31, size=4, dtype=np.int64)` This assigns 'self.rng.integers(-30, 31, size=4, dtype=np.int64)' to 'bias ' in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 452: `dy = self.rng.integers(-12, 13, size=(1, 4, 2, 3), dtype=np.int64)` This assigns 'self.rng.integers(-12, 13, size=(1, 4, 2, 3), dtype=np.int64)' to 'dy ' in host memory. Here dy means the value named 'dy' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 453: `program = compile_conv2d_backward(` This statement continues through source line 461; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'compile_conv2d_backward(x, w, bias, dy, stride=2, padding=1, groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 454: `x,` This is continuation line 2 of the 9-line logical statement that begins at source line 453. Its exact fragment is 'x,'. The important operands are x: the value named 'x' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, bias, dy, stride=2, padding=1, groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 455: `w,` This is continuation line 3 of the 9-line logical statement that begins at source line 453. Its exact fragment is 'w,'. The important operands are w: the value named 'w' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, bias, dy, stride=2, padding=1, groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 456: `bias,` This is continuation line 4 of the 9-line logical statement that begins at source line 453. Its exact fragment is 'bias,'. The important operands are bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, bias, dy, stride=2, padding=1, groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 457: `dy,` This is continuation line 5 of the 9-line logical statement that begins at source line 453. Its exact fragment is 'dy,'. The important operands are dy: the value named 'dy' in this block. Read together, the complete operation is: This assigns 'compile_conv2d_backward(x, w, bias, dy, stride=2, padding=1, groups=2,)' to 'program ' in host memory. Here program means the value named 'program' in this block. The important operands are compile_conv2d_backward: the value named 'compile conv2d backward' in this block; x: the value named 'x' in this block; w: the value named 'w' in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 458: `stride=2,` This is continuation line 6 of the 9-line logical statement that begins at source line 453. Its exact fragment is ‘stride=2,‘. The important operands are stride: the value named ’stride’ in this block. Read together, the complete operation is: This assigns ‘compile_conv2d_backward(x, w, bias, dy, stride=2, padding=1, groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_conv2d_backward: the value named ’compile conv2d backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 459: `padding=1,` This is continuation line 7 of the 9-line logical statement that begins at source line 453. Its exact fragment is ‘padding=1,‘. The important operands are padding: the value named ’padding’ in this block. Read together, the complete operation is: This assigns ‘compile_conv2d_backward(x, w, bias, dy, stride=2, padding=1, groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_conv2d_backward: the value named ’compile conv2d backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 460: `groups=2,` This is continuation line 8 of the 9-line logical statement that begins at source line 453. Its exact fragment is ‘groups=2,‘. The important operands are groups: the value named ’groups’ in this block. Read together, the complete operation is: This assigns ‘compile_conv2d_backward(x, w, bias, dy, stride=2, padding=1, groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_conv2d_backward: the value named ’compile conv2d backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 461: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 453. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘compile_conv2d_backward(x, w, bias, dy, stride=2, padding=1, groups=2,)‘ to ‘program ‘ in host memory. Here program means the value named ’program’ in this block. The important operands are compile_conv2d_backward: the value named ’compile conv2d backward’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 462: `self.assertEqual(program.output_shape, (1, 4, 2, 3))` This calls ‘self.assertEqual‘ with ‘program.output_shap (1, 4, 2, 3)‘ inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 463: `self.assertEqual(program.partial_tile_count, 12)` This calls ‘self.assertEqual‘ with ‘program.partial_tile_coun 12‘ inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 464: `got_dx, got_dw, got_db = program.execute_batched(FullTrainingEmulator())` This assigns ‘program.execute_batched(FullTrainingEmulator())‘ to ‘got_dx, got_dw, got_db ‘ in host memory. Here got_dx means the value named ’got dx’ in this block. The important operands are program: the value named ’program’ in this block; execute_batched: the value named ’execute batched’ in this block; FullTrainingEmulator: the value named ’full training emulator’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 466: `expected_dx = np.zeros_like(x)` This assigns ‘np.zeros_like(x)‘ to ‘expected_dx ‘ in host memory. Here expected_dx means the value named ’expected dx’ in this block. The important operands are zeros_like: the value named ’zeros like’ in this block; x: the value named ’x’ in this block. Context: the function test_grouped_convolution_backward_and_fabr which handles test grouped convolution backward and fabric col2im.

Line 467: `expected_dw = np.zeros_like(w)` This assigns ‘np.zeros_like(w)‘ to ‘expected_dw ‘ in host memory. Here expected_dw means the value named ’expected dw’ in this block. The important operands are zeros_like: the value named ’zeros like’ in this block; w: the value named ’w’ in this block. Context: the function test_grouped_convolution_backward_a which handles test grouped convolution backward and fabric col2im.

Line 468: `expected_db = np.zeros_like(bias)` This assigns ‘np.zeros_like(bias)‘ to ‘expected_db ‘ in host memory. Here expected_db means the value named ’expected db’ in this block. The important operands are zeros_like: the value named ’zeros like’ in this block; bias: an additive learned offset applied after accumulation. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 469: `for output_channel in range(4):` This iterates output_channel over ‘range(4)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_grouped_convolution_backwar which handles test grouped convolution backward and fabric col2im.

Line 470: `group = output_channel // 2` This assigns ‘output_channel // 2‘ to ‘group ‘ in host memory. Here group means the value named ’group’ in this block. The important operands are output_channel: the value named ’output channel’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 471: `input_start = group * 2` This assigns ‘group * 2‘ to ‘input_start ‘ in host memory. Here input_start means the value named ’input start’ in this block. The important operands are group: the value named ’group’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 472: `for output_y in range(2):` This iterates output_y over ‘range(2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_grouped_convolution_backward_and_fabri which handles test grouped convolution backward and fabric col2im.

Line 473: `for output_x in range(3):` This iterates output_x over ‘range(3)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_grouped_convolution_backward_and_fabri which handles test grouped convolution backward and fabric col2im.

Line 474: `gradient = int(dy[0, output_channel, output_y, output_x])` This assigns ‘int(dy[0, output_channel, output_y, output_x])‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are dy: the value named ’dy’ in this block; output_channel: the value named ’output channel’ in this block; output_y: the value named ’output y’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 475: `expected_db[output_channel] += gradient` Inside the function test_grouped_convolution_backward_and_fabric which handles test grouped convolution backward and fabric col2im, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are expected_db: the value named ’expected db’ in this block; output_channel: the value named ’output channel’ in this block; gradient: the signed loss derivative used to update a learned parameter.

Line 476: `for local_input in range(2):` This iterates local_input over ‘range(2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_grouped_convolution_backward_an which handles test grouped convolution backward and fabric col2im.

Line 477: `for kernel_y in range(3):` This iterates kernel_y over ‘range(3)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_grouped_convolution_backward_and_fabri which handles test grouped convolution backward and fabric col2im.

Line 478: `input_y = output_y * 2 - 1 + kernel_y` This assigns ‘output_y * 2 - 1 + kernel_y‘ to ‘input_y ‘ in host memory. Here input_y means the value named ’input y’ in this block. The important operands are output_y: the value named ’output y’ in this block; kernel_y: the value named ’kernel y’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 479: `for kernel_x in range(3):` This iterates kernel_x over ‘range(3)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_grouped_convolution_backward_and_fabri which handles test grouped convolution backward and fabric col2im.

Line 480: `input_x = output_x * 2 - 1 + kernel_x` This assigns ‘output_x * 2 - 1 + kernel_x‘ to ‘input_x ‘ in host memory. Here input_x means the value named ’input x’ in this block. The important operands are output_x: the value named ’output x’ in this block; kernel_x: the value named ’kernel x’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 481: `if 0 <= input_y < 4 and 0 <= input_x < 5:` This executes the following indented host-code branch only when ‘0 <= input_y < 4 and 0 <= input_x < 5‘ is true. The important operands are input_y: the value named ’input y’ in this block; and: the value named ’and’ in this block; input_x: the value named ’input x’ in this block. Context: the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im.

Line 482: `expected_dw[output_channel, local_input, kernel_y, kernel_x] += (` This statement continues through source line 485; the explanation uses the complete expression rather than only the first visual fragment. Inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are expected_dw: the value named ’expected dw’ in this block; output_channel: the value named ’output channel’ in this block; local_input: the value named ’local input’ in this block.

Line 483: `int(x[0, input_start + local_input, input_y, input_x])` This is continuation line 2 of the 4-line logical statement that begins at source line 482. Its exact fragment is ‘int(x[0, input_start + local_input, input_y, input_x])‘. The important operands are x: the value named ’x’ in this block; input_start: the value named ’input start’ in this block; local_input: the value named ’local input’ in this block. Read together, the complete operation is: Inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are expected_dw: the value named ’expected dw’ in this block; output_channel: the value named ’output channel’ in this block; local_input: the value named ’local input’ in this block.

Line 484: `* gradient` This is continuation line 3 of the 4-line logical statement that begins at source line 482. Its exact fragment is ‘* gradient‘. The important operands are gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: Inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are expected_dw: the value named ’expected dw’ in this block; output_channel: the value named ’output channel’ in this block; local_input: the value named ’local input’ in this block.

Line 485: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 482. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are expected_dw: the value named ’expected dw’ in this block; output_channel: the value named ’output channel’ in this block; local_input: the value named ’local input’ in this block.

Line 486: `expected_dx[0, input_start + local_input, input_y, input_x] += (` This statement continues through source line 489; the explanation uses the complete expression rather than only the first visual fragment. Inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are expected_dx: the value named 'expected dx' in this block; input_start: the value named 'input start' in this block; local_input: the value named 'local input' in this block.

Line 487: `int(w[output_channel, local_input, kernel_y, kernel_x])` This is continuation line 2 of the 4-line logical statement that begins at source line 486. Its exact fragment is 'int(w[output_channel, local_input, kernel_y, kernel_x])'. The important operands are w: the value named 'w' in this block; output_channel: the value named 'output channel' in this block; local_input: the value named 'local input' in this block. Read together, the complete operation is: Inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are expected_dx: the value named 'expected dx' in this block; input_start: the value named 'input start' in this block; local_input: the value named 'local input' in this block.

Line 488: `* gradient` This is continuation line 3 of the 4-line logical statement that begins at source line 486. Its exact fragment is '* gradient'. The important operands are gradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: Inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are expected_dx: the value named 'expected dx' in this block; input_start: the value named 'input start' in this block; local_input: the value named 'local input' in this block.

Line 489: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 486. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: Inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are expected_dx: the value named 'expected dx' in this block; input_start: the value named 'input start' in this block; local_input: the value named 'local input' in this block.

Line 490: `np.testing.assert_array_equal(got_dx, expected_dx)` This calls 'np.testing.assert_array_equal' with 'got_dx, expected_dx' inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 491: `np.testing.assert_array_equal(got_dw, expected_dw)` This calls 'np.testing.assert_array_equal' with 'got_dw, expected_dw' inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 492: `np.testing.assert_array_equal(got_db, expected_db)` This calls 'np.testing.assert_array_equal' with 'got_db, expected_db' inside the function test_grouped_convolution_backward_and_fabric_col2im, which handles test grouped convolution backward and fabric col2im. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 494: `def test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric(self) -> None:` This starts function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric. Its indented body performs test arbitrary parameter sgdw and int8 projection run in fabric as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 495: `weights = self.rng.integers(-500, 501, size=(3, 11), dtype=np.int64)` This assigns 'self.rng.integers(-500, 501, size=(3, 11), dtype=np.int64)' to 'weights ' in host memory. Here weights means a learned coefficient multiplied by an activation. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers'

in this block; size: the value named 'size' in this block. Context: the function test_arbitrary_parameter_sgdw_and_int8_projectio which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 496: `gradients = self.rng.integers(-(1 << 20), 1 << 20, size=(3, 11), dtype=np.int64)` This assigns 'self.rng.integers(-(1 << 20), 1 << 20, size=(3, 11), dtype=np.int64)' to 'gradients ' in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 497: `velocities = self.rng.integers(-(1 << 20), 1 << 20, size=(3, 11), dtype=np.int64)` This assigns 'self.rng.integers(-(1 << 20), 1 << 20, size=(3, 11), dtype=np.int64)' to 'velocities ' in host memory. Here velocities means the value named 'velocities' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 498: `expected_weight, expected_velocity = sgd_momentum_step(` This statement continues through source line 505; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66,)' to 'expected_weight, expected_velocity ' in host memory. Here expected_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_arbitrary_parameter_sgdw_and_int8_pr which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 499: `weights,` This is continuation line 2 of the 8-line logical statement that begins at source line 498. Its exact fragment is 'weights,'. The important operands are weights: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66,)' to 'expected_weight, expected_velocity ' in host memory. Here expected_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 500: `gradients,` This is continuation line 3 of the 8-line logical statement that begins at source line 498. Its exact fragment is 'gradients,'. The important operands are gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66,)' to 'expected_weight, expected_velocity ' in host memory. Here expected_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 501: `velocities,` This is continuation line 4 of the 8-line logical statement that begins at source line 498. Its exact fragment is 'velocities,'. The important operands are velocities: the value named 'velocities' in this block. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66,)' to 'expected_weight, expected_velocity ' in host memory. Here expected_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 502: `655,` This is continuation line 5 of the 8-line logical statement that begins at source line 498. Its exact fragment is '655,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66,)' to 'expected_weight, expected_velocity ' in host memory. Here expected_weight means a learned coefficient multiplied

by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 503: `58982,` This is continuation line 6 of the 8-line logical statement that begins at source line 498. Its exact fragment is '58982,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66,)' to 'expected_weight, expected_velocity ' in host memory. Here expected_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 504: `66,` This is continuation line 7 of the 8-line logical statement that begins at source line 498. Its exact fragment is '66,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66,)' to 'expected_weight, expected_velocity ' in host memory. Here expected_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 505: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 498. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'sgd_momentum_step(weights, gradients, velocities, 655, 58982, 66,)' to 'expected_weight, expected_velocity ' in host memory. Here expected_weight means a learned coefficient multiplied by an activation. The important operands are sgd_momentum_step: the value named 'sgd momentum step' in this block; weights: a learned coefficient multiplied by an activation; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 506: `emulator = FabricOperatorEmulator()` This assigns 'FabricOperatorEmulator()' to 'emulator ' in host memory. Here emulator means the value named 'emulator' in this block. The important operands are FabricOperatorEmulator: the value named 'fabric operator emulator' in this block. Context: the function test_arbitrary_parameter_sgdw_and_int8_ which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 507: `got_weight, got_velocity = execute_sgdw_update(` This statement continues through source line 515; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_sgdw_update(emulator, weights, gradients, velocities, learning_rate_q16=655, momentum_q16=58982, weight_decay_q16=66,)' to 'got_weight, got_velocity ' in host memory. Here got_weight means a learned coefficient multiplied by an activation. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; emulator: the value named 'emulator' in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 508: `emulator,` This is continuation line 2 of the 9-line logical statement that begins at source line 507. Its exact fragment is 'emulator,'. The important operands are emulator: the value named 'emulator' in this block. Read together, the complete operation is: This assigns 'execute_sgdw_update(emulator, weights, gradients, velocities, learning_rate_q16=655, momentum_q16=58982, weight_decay_q16=66,)' to 'got_weight, got_velocity ' in host memory. Here got_weight means a learned coefficient multiplied by an activation. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; emulator: the value named 'emulator' in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_ which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 509: `weights,` This is continuation line 3 of the 9-line logical statement that begins at source line 507. Its exact fragment is ‘weights,‘. The important operands are weights: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘execute_sgdw_update(emulator, weights, gradients, velocities, learning_rate_q16=655, momentum_q16=58982, weight_decay_q16=66,)‘ to ‘got_weight, got_velocity ‘ in host memory. Here got_weight means a learned coefficient multiplied by an activation. The important operands are execute_sgdw_update: the value named ’execute sgdw update’ in this block; emulator: the value named ’emulator’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_ which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 510: `gradients,` This is continuation line 4 of the 9-line logical statement that begins at source line 507. Its exact fragment is ‘gradients,‘. The important operands are gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns ‘execute_sgdw_update(emulator, weights, gradients, velocities, learning_rate_q16=655, momentum_q16=58982, weight_decay_q16=66,)‘ to ‘got_weight, got_velocity ‘ in host memory. Here got_weight means a learned coefficient multiplied by an activation. The important operands are execute_sgdw_update: the value named ’execute sgdw update’ in this block; emulator: the value named ’emulator’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_ which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 511: `velocities,` This is continuation line 5 of the 9-line logical statement that begins at source line 507. Its exact fragment is ‘velocities,‘. The important operands are velocities: the value named ’velocities’ in this block. Read together, the complete operation is: This assigns ‘execute_sgdw_update(emulator, weights, gradients, velocities, learning_rate_q16=655, momentum_q16=58982, weight_decay_q16=66,)‘ to ‘got_weight, got_velocity ‘ in host memory. Here got_weight means a learned coefficient multiplied by an activation. The important operands are execute_sgdw_update: the value named ’execute sgdw update’ in this block; emulator: the value named ’emulator’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_ which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 512: `learning_rate_q16=655,` This is continuation line 6 of the 9-line logical statement that begins at source line 507. Its exact fragment is ‘learning_rate_q16=655,‘. The important operands are learning_rate_q16: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This assigns ‘execute_sgdw_update(emulator, weights, gradients, velocities, learning_rate_q16=655, momentum_q16=58982, weight_decay_q16=66,)‘ to ‘got_weight, got_velocity ‘ in host memory. Here got_weight means a learned coefficient multiplied by an activation. The important operands are execute_sgdw_update: the value named ’execute sgdw update’ in this block; emulator: the value named ’emulator’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 513: `momentum_q16=58982,` This is continuation line 7 of the 9-line logical statement that begins at source line 507. Its exact fragment is ‘momentum_q16=58982,‘. The important operands are momentum_q16: the value named ’momentum q16’ in this block. Read together, the complete operation is: This assigns ‘execute_sgdw_update(emulator, weights, gradients, velocities, learning_rate_q16=655, momentum_q16=58982, weight_decay_q16=66,)‘ to ‘got_weight, got_velocity ‘ in host memory. Here got_weight means a learned coefficient multiplied by an activation. The important operands are execute_sgdw_update: the value named ’execute sgdw update’ in this block; emulator: the value named ’emulator’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 514: `weight_decay_q16=66,` This is continuation line 8 of the 9-line logical statement that begins at source line 507. Its exact fragment is ‘weight_decay_q16=66,‘. The important operands are weight_decay_q16: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘execute_sgdw_update(emulator, weights, gradients, velocities, learning_rate_q16=655, momentum_q16=58982, weight_decay_q16=66,)‘ to ‘got_weight, got_velocity ‘ in host memory. Here got_weight means a learned coefficient multiplied by an activation. The important operands are execute_sgdw_update: the value named ’execute sgdw update’ in this block; emulator: the value named ’emulator’ in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 515: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 507. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_sgdw_update(emulator, weights, gradients, velocities, learning_rate_q16=655, momentum_q16=58982, weight_decay_q16=66,)' to 'got_weight, got_velocity ' in host memory. Here got_weight means a learned coefficient multiplied by an activation. The important operands are execute_sgdw_update: the value named 'execute sgdw update' in this block; emulator: the value named 'emulator' in this block; weights: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 516: `np.testing.assert_array_equal(got_weight, expected_weight)` This calls 'np.testing.assert_array_equal' with 'got_weight, expected_weight' inside the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 517: `np.testing.assert_array_equal(got_velocity, expected_velocity)` This calls 'np.testing.assert_array_equal' with 'got_velocity, expected_velocity' inside the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 518: `projected = execute_int8_weight_projection(emulator, got_weight)` This assigns 'execute_int8_weight_proj got_weight)' to 'projected ' in host memory. Here projected means the value named 'projected' in this block. The important operands are execute_int8_weight_projection: a learned coefficient multiplied by an activation; emulator: the value named 'emulator' in this block; got_weight: a learned coefficient multiplied by an activation. Context: the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric.

Line 519: `np.testing.assert_array_equal(projected, np.clip(got_weight, -128, 127).astype(np.int8))` This calls 'np.testing.assert_array_equal' with 'projected, np.clip(got_weight, -128, 127).astype(np.int8)' inside the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 520: `self.assertEqual(emulator.command_count, 10)` This calls 'self.assertEqual' with 'emulator.command_count, 10' inside the function test_arbitrary_parameter_sgdw_and_int8_projection_run_in_fabric, which handles test arbitrary parameter sgdw and int8 projection run in fabric. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 522: `def test_rejects_misaligned_or_out_of_range_operands(self) -> None:` This starts function test_rejects_mis Its indented body performs test rejects misaligned or out of range operands as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 523: `with self.assertRaises(ValueError):` This opens a managed resource scope for 'self.assertRaises(ValueError)' so files, tracing, or temporary state are finalized even if the function test_rejects_misaligned_or_out_of_range_operands, which handles test rejects misaligned or out of range operands raises an error.

Line 524: `compile_linear_backward(np.zeros((2, 3)), np.zeros((4, 5)), np.zeros((2, 5)))` This calls 'compile_linear_backward' with 'np.zeros((2, 3)), np.zeros((4, 5)), np.zeros((2, 5))' inside the function test_rejects_misaligned_or_out_ which handles test rejects misaligned or out of range operands. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 525: `with self.assertRaises(ValueError):` This opens a managed resource scope for 'self.assertRaises(ValueError)' so files, tracing, or temporary state are finalized even if the function test_rejects_misaligned_or_out_of_range_operands, which handles test rejects misaligned or out of range operands raises an error.

Line 526: `compile_linear_backward(np.array([[32768]]), np.array([[1]]), np.array([[1]]))` This calls ‘compile_linear_backward‘ with ‘np.array([[32768]]), np.array([[1]]), np.array([[1]])‘ inside the function test_rejects_misaligned_or_out which handles test rejects misaligned or out of range operands. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 527: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_rejects_misaligned_or_out_of_range_operands, which handles test rejects misaligned or out of range operands raises an error.

Line 528: `compile_affine_backward(` This statement continues through source line 533; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘compile_affine_backward‘ with ‘ np.zeros((1, 1), dtype=np.int64), np.zeros((1, 2), dtype=np.int64), np.zeros(3, dtype=np.int64), np.zeros((1, 2), dtype=np.int64), ‘ inside the function test_rejects_misaligned_or_out_of_range_operands, which handles test rejects misaligned or out of range operands. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 529: `np.zeros((1, 1), dtype=np.int64),` This is continuation line 2 of the 6-line logical statement that begins at source line 528. Its exact fragment is ‘np.zeros((1, 1), dtype=np.int64),‘. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This calls ‘compile_affine_backward‘ with ‘ np.zeros((1, 1), dtype=np.int64), np.zeros((1, 2), dtype=np.int64), np.zeros(3, dtype=np.int64), np.zeros((1, 2), dtype=np.int64), ‘ inside the function test_rejects_misaligned_or_out_of_range_operands, which handles test rejects misaligned or out of range operands. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 530: `np.zeros((1, 2), dtype=np.int64),` This is continuation line 3 of the 6-line logical statement that begins at source line 528. Its exact fragment is ‘np.zeros((1, 2), dtype=np.int64),‘. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This calls ‘compile_affine_backward‘ with ‘ np.zeros((1, 1), dtype=np.int64), np.zeros((1, 2), dtype=np.int64), np.zeros(3, dtype=np.int64), np.zeros((1, 2), dtype=np.int64), ‘ inside the function test_rejects_misaligned_or_out_of_range_operands, which handles test rejects misaligned or out of range operands. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 531: `np.zeros(3, dtype=np.int64),` This is continuation line 4 of the 6-line logical statement that begins at source line 528. Its exact fragment is ‘np.zeros(3, dtype=np.int64),‘. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This calls ‘compile_affine_backward‘ with ‘ np.zeros((1, 1), dtype=np.int64), np.zeros((1, 2), dtype=np.int64), np.zeros(3, dtype=np.int64), np.zeros((1, 2), dtype=np.int64), ‘ inside the function test_rejects_misaligned_or_out_of_range_operands, which handles test rejects misaligned or out of range operands. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 532: `np.zeros((1, 2), dtype=np.int64),` This is continuation line 5 of the 6-line logical statement that begins at source line 528. Its exact fragment is ‘np.zeros((1, 2), dtype=np.int64),‘. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This calls ‘compile_affine_backward‘ with ‘ np.zeros((1, 1), dtype=np.int64), np.zeros((1, 2), dtype=np.int64), np.zeros(3, dtype=np.int64), np.zeros((1, 2), dtype=np.int64), ‘ inside the function test_rejects_misaligned_or_out_of_range_operands, which handles test rejects misaligned or out of range operands. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 533: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 528. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘compile_affine_backward‘ with ‘ np.zeros((1, 1), dtype=np.int64), np.zeros((1, 2), dtype=np.int64), np.zeros(3, dtype=np.int64), np.zeros((1, 2), dtype=np.int64), ‘ inside the function test_rejects_misaligned_or_out_of_range_ope which handles test rejects misaligned or out of range operands. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 535: `def test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence(self) -> None:` This starts function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence. Its indented body performs test lif bptt fabric schedule matches fixed point recurrence as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 536: `time_steps = 4` This assigns '4' to 'time_steps ' in host memory. Here time_steps means the value named 'time steps' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 537: `neurons = 19` This assigns '19' to 'neurons ' in host memory. Here neurons means the value named 'neurons' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 538: `threshold = 1 << 10` This assigns '1 << 10' to 'threshold ' in host memory. Here threshold means the membrane level at which a neuron emits a spike. The expression is a literal or structural operator and has no hidden software state. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 539: `membrane = self.rng.integers(` This statement continues through source line 544; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.rng.integers(threshold - 4096, threshold + 4097, size=(time_steps, neurons), dtype=np.int64,)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 540: `threshold - 4096,` This is continuation line 2 of the 6-line logical statement that begins at source line 539. Its exact fragment is 'threshold - 4096,'. The important operands are threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns 'self.rng.integers(threshold - 4096, threshold + 4097, size=(time_steps, neurons), dtype=np.int64,)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 541: `threshold + 4097,` This is continuation line 3 of the 6-line logical statement that begins at source line 539. Its exact fragment is 'threshold + 4097,'. The important operands are threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns 'self.rng.integers(threshold - 4096, threshold + 4097, size=(time_steps, neurons), dtype=np.int64,)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 542: `size=(time_steps, neurons),` This is continuation line 4 of the 6-line logical statement that begins at source line 539. Its exact fragment is 'size=(time_steps, neurons),'. The important operands are size: the value named 'size' in this block; time_steps: the value named 'time steps' in this block; neurons: the value named 'neurons' in this block. Read together, the complete operation is: This assigns 'self.rng.integers(threshold - 4096, threshold + 4097, size=(time_steps, neurons), dtype=np.int64,)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 543: `dtype=np.int64,` This is continuation line 5 of the 6-line logical statement that begins at source line 539. Its exact fragment is 'dtype=np.int64,'. The important operands are dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns 'self.rng.integers(threshold - 4096, threshold + 4097, size=(time_steps, neurons), dtype=np.int64,)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 544: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 539. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self.rng.integers(threshold - 4096, threshold + 4097, size=(time_steps, neurons), dtype=np.int64,)' to 'membrane ' in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 545: `spikes = (membrane >= threshold).astype(np.int64)` Inside the function test_lif_bptt_fabric_schedule_matche which handles test lif bptt fabric schedule matches fixed point recurrence, this statement advances the host implementation that checks neuraedge training behavior independently with Python assertions. The important operands are spikes: a binary event produced when a neuron crosses its threshold; membrane: the persistent LIF neuron state carrying charge between timesteps; threshold: the membrane level at which a neuron emits a spike.

Line 546: `grad_spikes = self.rng.integers(` This statement continues through source line 548; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.rng.integers(-(1 << 20), 1 << 20, size=(time_steps, neurons), dtype=np.int64)' to 'grad_spikes ' in host memory. Here grad_spikes means a binary event produced when a neuron crosses its threshold. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 547: `-(1 << 20), 1 << 20, size=(time_steps, neurons), dtype=np.int64` This is continuation line 2 of the 3-line logical statement that begins at source line 546. Its exact fragment is '-(1 << 20), 1 << 20, size=(time_steps, neurons), dtype=np.int64'. The important operands are size: the value named 'size' in this block; time_steps: the value named 'time steps' in this block; neurons: the value named 'neurons' in this block. Read together, the complete operation is: This assigns 'self.rng.integers(-(1 << 20), 1 << 20, size=(time_steps, neurons), dtype=np.int64)' to 'grad_spikes ' in host memory. Here grad_spikes means a binary event produced when a neuron crosses its threshold. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 548: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 546. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self.rng.integers(-(1 << 20), 1 << 20, size=(time_steps, neurons), dtype=np.int64)' to 'grad_spikes ' in host memory. Here grad_spikes means a binary event produced when a neuron crosses its threshold. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 549: `grad_membrane = self.rng.integers(` This statement continues through source line 551; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'self.rng.integers(-(1 << 18), 1 << 18, size=(time_steps, neurons), dtype=np.int64)' to 'grad_membrane ' in host memory. Here grad_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context:

the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 550: `-(1 << 18), 1 << 18, size=(time_steps, neurons), dtype=np.int64` This is continuation line 2 of the 3-line logical statement that begins at source line 549. Its exact fragment is '-(1 << 18), 1 << 18, size=(time_steps, neurons), dtype=np.int64'. The important operands are size: the value named 'size' in this block; time_steps: the value named 'time steps' in this block; neurons: the value named 'neurons' in this block. Read together, the complete operation is: This assigns 'self.rng.integers(-(1 << 18), 1 << 18, size=(time_steps, neurons), dtype=np.int64)' to 'grad_membrane ' in host memory. Here grad_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurr which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 551: `)` This is continuation line 3 of the 3-line logical statement that begins at source line 549. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'self.rng.integers(-(1 << 18), 1 << 18, size=(time_steps, neurons), dtype=np.int64)' to 'grad_membrane ' in host memory. Here grad_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 552: `table, address_shift = sigmoid_surrogate_lut_q15()` This assigns 'sigmoid_surrogate_lut_q15()' to 'table, address_shift ' in host memory. Here table means the value named 'table' in this block. The important operands are sigmoid_surrogate_lut_q15: the value named 'sigmoid surrogate lut q15' in this block. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 553: `self.assertEqual(int(table[128]), 32767)` This calls 'self.assertEqual' with 'int(table[128]), 32767' inside the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 554: `addresses = surrogate_lut_addresses(membrane, threshold, address_shift)` This assigns 'surrogate_lut_addresses(membrane, threshold, address_shift)' to 'addresses ' in host memory. Here addresses means the value named 'addresses' in this block. The important operands are surrogate_lut_addresses: the value named 'surrogate lut addresses' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; threshold: the membrane level at which a neuron emits a spike. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 555: `self.assertTrue(np.all(addresses >= 0))` This calls 'self.assertTrue' with 'np.all(addresses >= 0)' inside the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 556: `self.assertTrue(np.all(addresses <= 255))` This calls 'self.assertTrue' with 'np.all(addresses <= 255)' inside the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 557: `expected_input, expected_h = lif_bptt_fixed(` This statement continues through source line 565; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'lif_bptt_fixed(membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'expected_input, expected_h ' in host memory. Here expected_input means the value named 'expected input' in this block. The important operands are lif_bptt_fixed: the value named 'lif

bptt fixed' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_re which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 558: `membrane,` This is continuation line 2 of the 9-line logical statement that begins at source line 557. Its exact fragment is 'membrane,'. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns 'lif_bptt_fixed(membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'expected_input, expected_h ' in host memory. Here expected_input means the value named 'expected input' in this block. The important operands are lif_bptt_fixed: the value named 'lif bptt fixed' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 559: `spikes,` This is continuation line 3 of the 9-line logical statement that begins at source line 557. Its exact fragment is 'spikes,'. The important operands are spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns 'lif_bptt_fixed(membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'expected_input, expected_h ' in host memory. Here expected_input means the value named 'expected input' in this block. The important operands are lif_bptt_fixed: the value named 'lif bptt fixed' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 560: `grad_spikes,` This is continuation line 4 of the 9-line logical statement that begins at source line 557. Its exact fragment is 'grad_spikes,'. The important operands are grad_spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns 'lif_bptt_fixed(membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'expected_input, expected_h ' in host memory. Here expected_input means the value named 'expected input' in this block. The important operands are lif_bptt_fixed: the value named 'lif bptt fixed' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 561: `threshold=threshold,` This is continuation line 5 of the 9-line logical statement that begins at source line 557. Its exact fragment is 'threshold=threshold,'. The important operands are threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns 'lif_bptt_fixed(membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'expected_input, expected_h ' in host memory. Here expected_input means the value named 'expected input' in this block. The important operands are lif_bptt_fixed: the value named 'lif bptt fixed' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 562: `membrane_gradients=grad_membrane,` This is continuation line 6 of the 9-line logical statement that begins at source line 557. Its exact fragment is 'membrane_gradients=grad_membrane,'. The important operands are membrane_gradients: the persistent LIF neuron state carrying charge between timesteps; grad_membrane: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns 'lif_bptt_fixed(membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'expected_input, expected_h ' in host memory. Here expected_input means the value named 'expected input' in this block. The important operands are lif_bptt_fixed: the value named 'lif bptt fixed' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_lif_bptt_fabric_schedule which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 563: `surrogate_table_q15=table,` This is continuation line 7 of the 9-line logical statement that begins at source line 557. Its exact fragment is 'surrogate_table_q15=table,'. The important operands are surrogate_table_q15: the value named 'surrogate table q15' in this block; table: the value named 'table' in this block. Read together, the complete operation is: This assigns 'lif_bptt_fixed(membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'expected_input, expected_h ' in host memory. Here expected_input means the value named 'expected input' in this block. The important operands are lif_bptt_fixed: the value named 'lif bptt fixed' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 564: `surrogate_address_shift=address_shift,` This is continuation line 8 of the 9-line logical statement that begins at source line 557. Its exact fragment is 'surrogate_address_shift=address_shift,'. The important operands are surrogate_address_shift: the value named 'surrogate address shift' in this block; address_shift: the value named 'address shift' in this block. Read together, the complete operation is: This assigns 'lif_bptt_fixed(membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'expected_input, expected_h ' in host memory. Here expected_input means the value named 'expected input' in this block. The important operands are lif_bptt_fixed: the value named 'lif bptt fixed' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 565: `)` This is continuation line 9 of the 9-line logical statement that begins at source line 557. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'lif_bptt_fixed(membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'expected_input, expected_h ' in host memory. Here expected_input means the value named 'expected input' in this block. The important operands are lif_bptt_fixed: the value named 'lif bptt fixed' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 566: `emulator = FabricOperatorEmulator()` This assigns 'FabricOperatorEmulator()' to 'emulator ' in host memory. Here emulator means the value named 'emulator' in this block. The important operands are FabricOperatorEmulator: the value named 'fabric operator emulator' in this block. Context: the function test_lif_bptt_fabric_schedule_matches_fixe which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 567: `got_input, got_h = execute_lif_bptt(` This statement continues through source line 576; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 568: `emulator,` This is continuation line 2 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'emulator,'. The important operands are emulator: the value named 'emulator' in this block. Read together, the complete operation is: This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 569: `membrane,` This is continuation line 3 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'membrane,'. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 570: `spikes,` This is continuation line 4 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'spikes,'. The important operands are spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shif)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 571: `grad_spikes,` This is continuation line 5 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'grad_spikes,'. The important operands are grad_spikes: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 572: `threshold=threshold,` This is continuation line 6 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'threshold=threshold,'. The important operands are threshold: the membrane level at which a neuron emits a spike. Read together, the complete operation is: This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 573: `membrane_gradients=grad_membrane,` This is continuation line 7 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'membrane_gradients=grad_membrane,'. The important operands are membrane_gradients: the persistent LIF neuron state carrying charge between timesteps; grad_membrane: the persistent LIF neuron state carrying charge between timesteps. Read together, the complete operation is: This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recur which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 574: `surrogate_table_q15=table,` This is continuation line 8 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'surrogate_table_q15=table,'. The important operands are surrogate_table_q15: the value named 'surrogate table q15' in this block; table: the value named 'table' in this block. Read together, the complete operation is: This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands

are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 575: `surrogate_address_shift=address_shift,` This is continuation line 9 of the 10-line logical statement that begins at source line 567. Its exact fragment is 'surrogate_address_shift=address_shift,'. The important operands are surrogate_address_shift: the value named 'surrogate address shift' in this block; address_shift: the value named 'address shift' in this block. Read together, the complete operation is: This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 576: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 567. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'execute_lif_bptt(emulator, membrane, spikes, grad_spikes, threshold=threshold, membrane_gradients=grad_membrane, surrogate_table_q15=table, surrogate_address_shift=address_shift,)' to 'got_input, got_h ' in host memory. Here got_input means the value named 'got input' in this block. The important operands are execute_lif_bptt: the value named 'execute lif bptt' in this block; emulator: the value named 'emulator' in this block; membrane: the persistent LIF neuron state carrying charge between timesteps. Context: the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence.

Line 577: `np.testing.assert_array_equal(got_input, expected_input)` This calls 'np.testing.assert_array_equal' with 'got_input, expected_input' inside the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 578: `np.testing.assert_array_equal(got_h, expected_h)` This calls 'np.testing.assert_array_equal' with 'got_h, expected_h' inside the function test_lif_bptt_fabric_schedule_matches_fixed_point_recurrence, which handles test lif bptt fabric schedule matches fixed point recurrence. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 579: `self.assertEqual(emulator.command_count, 1 + time_steps * 3 * 7)` This calls 'self.assertEqual' with 'emulator.command_count, 1 + time_steps * 3 * 7' inside the function test_lif_bptt_fabric_schedule_matches_fixed_point_ which handles test lif bptt fabric schedule matches fixed point recurrence. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 581: `def test_maxpool_backward_matches_first_winner_and_overlap_accumulation(self) -> None:` This starts function test_maxpool_backward_matches_first_winner_and_overlap_accumulation. Its indented body performs test maxpool backward matches first winner and overlap accumulation as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 582: `values = self.rng.integers(-20, 21, size=(1, 2, 5, 6), dtype=np.int64)` This assigns 'self.rng.integers(-20, 21, size=(1, 2, 5, 6), dtype=np.int64)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overla which handles test maxpool backward matches first winner and overlap accumulation.

Line 583: `values[:, :, 1:4, 1:5] = 7` This assigns '7' to 'values[' in host memory. Here values means the value named 'values' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 584: `values[0, 0, 0, 0] = -(1 << 31)` This assigns '-(1 << 31)' to 'values[0, 0, 0, 0] ' in host memory. Here values means the value named 'values' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 585: `gradients = self.rng.integers(-1000, 1001, size=(1, 2, 3, 3), dtype=np.int64)` This assigns 'self.rng.integers(-1000, 1001, size=(1, 2, 3, 3), dtype=np.int64)' to 'gradients ' in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 586: `expected_gradient, expected_winners = maxpool2d_backward_fixed(` This statement continues through source line 592; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'maxpool2d_backward_fixed(values, gradients, kernel_size=3, stride=2, padding=1,)' to 'expected_gradient, expected_winners ' in host memory. Here expected_gradient means the signed loss derivative used to update a learned parameter. The important operands are maxpool2d_backward_fixed: the value named 'maxpool2d backward fixed' in this block; values: the value named 'values' in this block; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 587: `values,` This is continuation line 2 of the 7-line logical statement that begins at source line 586. Its exact fragment is 'values,'. The important operands are values: the value named 'values' in this block. Read together, the complete operation is: This assigns 'maxpool2d_backward_fixed(values, gradients, kernel_size=3, stride=2, padding=1,)' to 'expected_gradient, expected_winners ' in host memory. Here expected_gradient means the signed loss derivative used to update a learned parameter. The important operands are maxpool2d_backward_fixed: the value named 'maxpool2d backward fixed' in this block; values: the value named 'values' in this block; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 588: `gradients,` This is continuation line 3 of the 7-line logical statement that begins at source line 586. Its exact fragment is 'gradients,'. The important operands are gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'maxpool2d_backward_fixed(values, gradients, kernel_size=3, stride=2, padding=1,)' to 'expected_gradient, expected_winners ' in host memory. Here expected_gradient means the signed loss derivative used to update a learned parameter. The important operands are maxpool2d_backward_fixed: the value named 'maxpool2d backward fixed' in this block; values: the value named 'values' in this block; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 589: `kernel_size=3,` This is continuation line 4 of the 7-line logical statement that begins at source line 586. Its exact fragment is 'kernel_size=3,'. The important operands are kernel_size: the value named 'kernel size' in this block. Read together, the complete operation is: This assigns 'maxpool2d_backward_fixed(values, gradients, kernel_size=3, stride=2, padding=1,)' to 'expected_gradient, expected_winners ' in host memory. Here expected_gradient means the signed loss derivative used to update a learned parameter. The important operands are maxpool2d_backward_fixed: the value named 'maxpool2d backward fixed' in this block; values: the value named 'values' in this block; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 590: `stride=2,` This is continuation line 5 of the 7-line logical statement that begins at source line 586. Its exact fragment is 'stride=2,'. The important operands are stride: the value named 'stride' in this block. Read together, the complete operation is: This assigns 'maxpool2d_backward_fixed(values, gradients, kernel_size=3, stride=2, padding=1,)' to 'expected_gradient, expected_winners ' in host memory. Here expected_gradient means the signed loss derivative used to update a learned parameter. The important operands are maxpool2d_backward_fixed: the value named 'maxpool2d backward fixed' in this block; values: the value named 'values' in this block; gradients: the signed loss derivative used to

update a learned parameter. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 591: `padding=1,` This is continuation line 6 of the 7-line logical statement that begins at source line 586. Its exact fragment is 'padding=1,'. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns 'maxpool2d_backward_fixed(values, gradients, kernel_size=3, stride=2, padding=1,)' to 'expected_gradient, expected_winners ' in host memory. Here expected_gradient means the signed loss derivative used to update a learned parameter. The important operands are maxpool2d_backward_fixed: the value named 'maxpool2d backward fixed' in this block; values: the value named 'values' in this block; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_maxpool_backward_matches_first_winner_and_overla which handles test maxpool backward matches first winner and overlap accumulation.

Line 592: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 586. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'maxpool2d_backward_fixed(values, gradients, kernel_size=3, stride=2, padding=1,)' to 'expected_gradient, expected_winners ' in host memory. Here expected_gradient means the signed loss derivative used to update a learned parameter. The important operands are maxpool2d_backward_fixed: the value named 'maxpool2d backward fixed' in this block; values: the value named 'values' in this block; gradients: the signed loss derivative used to update a learned parameter. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 593: `emulator = FabricOperatorEmulator()` This assigns 'FabricOperatorEmulator()' to 'emulator ' in host memory. Here emulator means the value named 'emulator' in this block. The important operands are FabricOperatorEmulator: the value named 'fabric operator emulator' in this block. Context: the function test_maxpool_backward_matches_first_wi which handles test maxpool backward matches first winner and overlap accumulation.

Line 594: `got_gradient, got_winners = execute_maxpool2d_backward(` This statement continues through source line 601; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'execute_maxpool2d_backward(emulator, values, gradients, kernel_size=3, stride=2, padding=1,)' to 'got_gradient, got_winners ' in host memory. Here got_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; emulator: the value named 'emulator' in this block; values: the value named 'values' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 595: `emulator,` This is continuation line 2 of the 8-line logical statement that begins at source line 594. Its exact fragment is 'emulator,'. The important operands are emulator: the value named 'emulator' in this block. Read together, the complete operation is: This assigns 'execute_maxpool2d_backward(emulator, values, gradients, kernel_size=3, stride=2, padding=1,)' to 'got_gradient, got_winners ' in host memory. Here got_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; emulator: the value named 'emulator' in this block; values: the value named 'values' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 596: `values,` This is continuation line 3 of the 8-line logical statement that begins at source line 594. Its exact fragment is 'values,'. The important operands are values: the value named 'values' in this block. Read together, the complete operation is: This assigns 'execute_maxpool2d_backward(emulator, values, gradients, kernel_size=3, stride=2, padding=1,)' to 'got_gradient, got_winners ' in host memory. Here got_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; emulator: the value named 'emulator' in this block; values: the value named 'values' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 597: `gradients,` This is continuation line 4 of the 8-line logical statement that begins at source line 594. Its exact fragment is 'gradients,'. The important operands are gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns 'execute_maxpool2d_backward(emulator, values, gradients,

kernel_size=3, stride=2, padding=1,)‘ to ‘got_gradient, got_winners ‘ in host memory. Here got_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; emulator: the value named 'emulator' in this block; values: the value named 'values' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accum which handles test maxpool backward matches first winner and overlap accumulation.

Line 598: `kernel_size=3,` This is continuation line 5 of the 8-line logical statement that begins at source line 594. Its exact fragment is ‘kernel_size=3,‘. The important operands are kernel_size: the value named 'kernel size' in this block. Read together, the complete operation is: This assigns ‘execute_maxpool2d_backward(emulator, values, gradients, kernel_size=3, stride=2, padding=1,)‘ to ‘got_gradient, got_winners ‘ in host memory. Here got_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; emulator: the value named 'emulator' in this block; values: the value named 'values' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumula which handles test maxpool backward matches first winner and overlap accumulation.

Line 599: `stride=2,` This is continuation line 6 of the 8-line logical statement that begins at source line 594. Its exact fragment is ‘stride=2,‘. The important operands are stride: the value named 'stride' in this block. Read together, the complete operation is: This assigns ‘execute_maxpool2d_backward(emulator, values, gradients, kernel_size=3, stride=2, padding=1,)‘ to ‘got_gradient, got_winners ‘ in host memory. Here got_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; emulator: the value named 'emulator' in this block; values: the value named 'values' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 600: `padding=1,` This is continuation line 7 of the 8-line logical statement that begins at source line 594. Its exact fragment is ‘padding=1,‘. The important operands are padding: the value named 'padding' in this block. Read together, the complete operation is: This assigns ‘execute_maxpool2d_backward(emulator, values, gradients, kernel_size=3, stride=2, padding=1,)‘ to ‘got_gradient, got_winners ‘ in host memory. Here got_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; emulator: the value named 'emulator' in this block; values: the value named 'values' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 601: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 594. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘execute_maxpool2d_backward(emulator, values, gradients, kernel_size=3, stride=2, padding=1,)‘ to ‘got_gradient, got_winners ‘ in host memory. Here got_gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_maxpool2d_backward: the value named 'execute maxpool2d backward' in this block; emulator: the value named 'emulator' in this block; values: the value named 'values' in this block. Context: the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation.

Line 602: `np.testing.assert_array_equal(got_winners, expected_winners)` This calls ‘np.testing.assert_array_equal‘ with ‘got_winners, expected_winners‘ inside the function test_maxpool_backward_matches_first_winner_and_overlap_accumul which handles test maxpool backward matches first winner and overlap accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 603: `np.testing.assert_array_equal(got_gradient, expected_gradient)` This calls ‘np.testing.assert_array_equal‘ with ‘got_gradient, expected_gradient‘ inside the function test_maxpool_backward_matches_first_winner_and_overlap_accumu which handles test maxpool backward matches first winner and overlap accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 604: `self.assertGreater(emulator.command_count, 100)` This calls ‘self.assertGreater‘ with ‘emulator.command_count 100‘ inside the function test_maxpool_backward_matches_first_winner_and_overlap_accumulation, which handles test maxpool backward matches first winner and overlap accumulation. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 606: `def test_real_spikingformer_manifest_has_no_training_coverage_holes(self) -> None:` This starts function test_real_spikingformer_manifest_has_no_training_coverage_holes. Its indented body performs test real spikingformer manifest has no training coverage holes as part of the file that checks neuraedge training behavior independently with Python assertions; it runs only when called.

Line 607: `root = Path(__file__).resolve().parents[1]` This assigns 'Path(___file___).resolve().parents[1]' to 'root ' in host memory. Here root means the value named 'root' in this block. The important operands are Path: the value named 'path' in this block; ___file___: the value named 'file' in this block; resolve: the value named 'resolve' in this block. Context: the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes.

Line 608: `result = build_spikingformer_training_coverage(` This statement continues through source line 611; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'build_spikingformer_training_
root / "outputs" / "spikingformer_reference" / "spikingformer_execution_manifest.json", root / "outputs" / "spikingformer_hardware_parameters" / "spikingformer_neuraedge_weights.json",)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are build_spikingformer_training_coverage: the value named 'build spikingformer training coverage' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes.

Line 609: `root / "outputs" / "spikingformer_reference" / "spikingformer_execution_manifest.json",` This is continuation line 2 of the 4-line logical statement that begins at source line 608. Its exact fragment is 'root / "outputs" / "spikingformer_reference" / "spikingformer_execution_manifest.json",'. The important operands are root: the value named 'root' in this block; outputs: the value named 'outputs' in this block; spikingformer_reference: the value named 'spikingformer reference' in this block. Read together, the complete operation is: This assigns 'build_spikingformer_training_covera
root / "outputs" / "spikingformer_reference" / "spikingformer_execution_manifest.json", root / "outputs" / "spikingformer_hardware_parameters" / "spikingformer_neuraedge_weights.json",)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are build_spikingformer_training_coverage: the value named 'build spikingformer training coverage' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes.

Line 610: `root / "outputs" / "spikingformer_hardware_parameters" / "spikingformer_neuraedge_weights.json",` This is continuation line 3 of the 4-line logical statement that begins at source line 608. Its exact fragment is 'root / "outputs" / "spikingformer_hardware_parameters" / "spikingformer_neuraedge_weights.json",'. The important operands are root: the value named 'root' in this block; outputs: the value named 'outputs' in this block; spikingformer_hardware_parameters: the value named 'spikingformer hardware parameters' in this block. Read together, the complete operation is: This assigns 'build_spikingformer_training_coverage(root / "outputs" / "spikingformer_reference" / "spikingformer_execution_manifest.json", root / "outputs" / "spikingformer_hardware_parameters" / "spikingformer_neuraedge_weights.json",)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are build_spikingformer_training_c
the value named 'build spikingformer training coverage' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function test_real_spikingformer_manifest_has_no_training_coverage_hole
which handles test real spikingformer manifest has no training coverage holes.

Line 611: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 608. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'build_spikingformer_training_coverage(root / "outputs" / "spikingformer_reference" / "spikingformer_execution_manifest.json", root / "outputs" / "spikingformer_hardware_parameters" / "spikingformer_neuraedge_weights.json",)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are build_spikingformer_training_coverage: the value named 'build spikingformer training coverage' in this block; root: the value named 'root' in this block; outputs: the value named 'outputs' in this block. Context: the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes.

Line 612: `summary = result["summary"]` This assigns 'result["summary"]' to 'summary ' in host memory. Here summary means the value named 'summary' in this block. The important operands are result: the value named 'result' in this

block; summary: the value named 'summary' in this block. Context: the function test_real_spikingformer_manifest_has_no_train which handles test real spikingformer manifest has no training coverage holes.

Line 613: `self.assertEqual(summary["execution_operator_count"], 43)` This calls 'self.assertEqual' with 'summary["execution_operator_count"], 43' inside the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 614: `self.assertEqual(summary["covered_execution_operator_count"], 43)` This calls 'self.assertEqual' with 'summary["covered_execution_operator_count"], 43' inside the function test_real_spikingformer_manifest_has_no_training which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 615: `self.assertEqual(summary["learned_affine_group_count"], 30)` This calls 'self.assertEqual' with 'summary["learned_affine_group_count"], 30' inside the function test_real_spikingformer_manifest_has_no_training_coverage_ which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 616: `self.assertEqual(summary["backbone_projection_count"], 29)` This calls 'self.assertEqual' with 'summary["backbone_projection_count"], 29' inside the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 617: `self.assertEqual(summary["dynamic_attention_matmul_count"], 8)` This calls 'self.assertEqual' with 'summary["dynamic_attention_matmul_count"], 8' inside the function test_real_spikingformer_manifest_has_no_training_cove which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 618: `self.assertEqual(summary["lif_node_count"], 32)` This calls 'self.assertEqual' with 'summary["lif_node_count"] 32' inside the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 619: `self.assertEqual(summary["trainable_deployed_parameter_elements"], 9_295_978)` This calls 'self.assertEqu with 'summary["trainable_deployed_parameter_elements"], 9_295_978' inside the function test_real_spikingformer_manifest_ha which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 620: `self.assertEqual(summary["uncovered_operator_count"], 0)` This calls 'self.assertEqual' with 'summary["uncovered_operator_count"], 0' inside the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 621: `self.assertTrue(summary["full_deployed_backbone_covered"])` This calls 'self.assertTrue' with 'summary["full_deployed_backbone_covered"]' inside the function test_real_spikingformer_manifest_has_no_training_coverage_hol which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 622: `self.assertEqual(` This statement continues through source line 628; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertEqual' with ' {item["name"] for item in result["operator_coverage"]}, {item["name"] for item in result["learned_affine_coverage"]} | {item["name"] for item in result["dynamic_attention_coverage"]} | {f"block.{index}.residuals" for index in range(4)} | {"global_average_pool"}, ' inside the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 623: `{item["name"] for item in result["operator_coverage"]},` This is continuation line 2 of the 7-line logical statement that begins at source line 622. Its exact fragment is '{item["name"] for item in result["operator_coverage"]},'. The important operands are item: the value named 'item' in this block; name: the value named 'name' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' {item["name"] for item in result["operator_coverage"]}, {item["name"] for item in result["learned_affine_coverage"]} | {item["name"] for item in result["dynamic_attention_coverage"]} | {f"block.{index}.residuals" for index in range(4)} | {"global_average_pool"}, ' inside the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 624: `{item["name"] for item in result["learned_affine_coverage"]}` This is continuation line 3 of the 7-line logical statement that begins at source line 622. Its exact fragment is '{item["name"] for item in result["learned_affine_coverage"]' The important operands are item: the value named 'item' in this block; name: the value named 'name' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' {item["name"] for item in result["operator_coverage"]}, {item["name"] for item in result["learned_affine_coverage"]} | {item["name"] for item in result["dynamic_attention_coverage"]} | {f"block.{index}.residuals" for index in range(4)} | {"global_average_pool"}, ' inside the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 625: `| {item["name"] for item in result["dynamic_attention_coverage"]}` This is continuation line 4 of the 7-line logical statement that begins at source line 622. Its exact fragment is '| {item["name"] for item in result["dynamic_attention_coverage"]}'. The important operands are item: the value named 'item' in this block; name: the value named 'name' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' {item["name"] for item in result["operator_coverage"]}, {item["name"] for item in result["learned_affine_coverage"]} | {item["name"] for item in result["dynamic_attention_coverage"]} | {f"block.{index}.residuals" for index in range(4)} | {"global_average_pool"}, ' inside the function test_real_spikingformer_manifest_has_no_training_covera which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 626: `| {f"block.{index}.residuals" for index in range(4)}` This is continuation line 5 of the 7-line logical statement that begins at source line 622. Its exact fragment is '| {f"block.{index}.residuals" for index in range(4)}'. The important operands are f: the value named 'f' in this block; block: the value named 'block' in this block; index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls 'self.assertEqual' with ' {item["name"] for item in result["operator_coverage"]}, {item["name"] for item in result["learned_affine_coverage"]} | {item["name"] for item in result["dynamic_attention_coverage"]} | {f"block.{index}.residuals" for index in range(4)} | {"global_average_pool"}, ' inside the function test_real_spikingformer_manifest_has_no_training_covera which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 627: `| {"global_average_pool"},` This is continuation line 6 of the 7-line logical statement that begins at source line 622. Its exact fragment is '| {"global_average_pool"},'. The important operands are global_average_pool: the value named 'global average pool' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' {item["name"] for item in result["operator_coverage"]}, {item["name"] for item in result["learned_affine_coverage"]} | {item["name"] for item in result["dynamic_attention_coverage"]} | {f"block.{index}.residuals" for index in range(4)} | {"global_average_pool"}, ' inside the function test_real_spikingformer_manifest_has_no_training_coverage_holes, which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 628: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 622. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with ' {item["name"] for item in result["operator_coverage"]}, {item["name"] for item in result["learned_affine_coverage"]} | {item["name"] for item in result["dynamic_attention_coverage"]} | {f"block.{index}.residuals" for index in range(4)} | {"global_average_pool"}, ' inside the function test_real_spikingformer_manifest_has_no_training_covera which handles test real spikingformer manifest has no training coverage holes. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

Line 631: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks neuraedge training behavior independently with Python assertions.

Line 632: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks neuraedge training behavior independently with Python assertions. The call advances the host-side job that checks neuraedge training behavior independently with Python assertions.

69 tests/test_run_spikingformer_pynq.py

Role in the complete system: checks run spikingformer pynq behavior independently with Python assertions.

69.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks run spikingformer pynq behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: PynqInferenceInputTests. Functions: test_raw_nhwc_and_nchw_normalization, test_rejects_wrong Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

69.2 Complete source

```
from __future__ import annotations

import unittest

import numpy as np

from run_spikingformer_pynq import CIFAR10_MEAN, CIFAR10_STD, prepare_input


class PynqInferenceInputTests(unittest.TestCase):
    def test_raw_nhwc_and_nchw_normalization(self) -> None:
        raw_nhwc = np.zeros((32, 32, 3), dtype=np.uint8)
        raw_nhwc[..., 1] = 255
        result = prepare_input(raw_nhwc, "raw-u8")
        self.assertEqual(result.shape, (1, 3, 32, 32))
        self.assertAlmostEqual(float(result[0, 0, 0, 0]), -float(CIFAR10_MEAN[0] / CIFAR10_STD[0]), places=6)
        self.assertAlmostEqual(float(result[0, 1, 0, 0]), float((1 - CIFAR10_MEAN[1]) / CIFAR10_STD[1]), places=6)
        np.testing.assert_array_equal(
            prepare_input(result, "normalized"), result.astype(np.float32))

    def test_rejects_wrong_shape_and_out_of_range_raw_values(self) -> None:
        with self.assertRaises(ValueError):
            prepare_input(np.zeros((28, 28), dtype=np.uint8), "raw-u8")
        with self.assertRaises(ValueError):
            prepare_input(np.full((3, 32, 32), 256), "raw-u8")


if __name__ == "__main__":
    unittest.main()
```

69.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks run spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import unittest` This imports the named Python API so the file whose role is to checks run spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import numpy as np` This imports the named Python API so the file whose role is to checks run spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from run_spikingformer_pynq import CIFAR10_MEAN, CIFAR10_STD, prepare_input` This imports the named Python API so the file whose role is to checks run spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `class PynqInferenceInputTests(unittest.TestCase):` This defines class PynqInferenceInputTests, which groups data and methods for pynq inference input tests. Instances exist in host memory and organize the flow that checks run spikingformer pynq behavior independently with Python assertions.

Line 11: `def test_raw_nhwc_and_nchw_normalization(self) -> None:` This starts function test_raw_nhwc_and_nchw_no Its indented body performs test raw nhwc and nchw normalization as part of the file that checks run spikingformer pynq behavior independently with Python assertions; it runs only when called.

Line 12: `raw_nhwc = np.zeros((32, 32, 3), dtype=np.uint8)` This assigns 'np.zeros((32, 32, 3), dtype=np.uint8)' to 'raw_nhwc ' in host memory. Here raw_nhwc means the value named 'raw nhwc' in this block. The important operands are zeros: the value named 'zeros' in this block; dtype: the value named 'dtype' in this block; uint8: the value named 'uint8' in this block. Context: the function test_raw_nhwc_and_nchw_normalization, which handles test raw nhwc and nchw normalization.

Line 13: `raw_nhwc[..., 1] = 255` This assigns '255' to 'raw_nhwc[..., 1] ' in host memory. Here raw_nhwc means the value named 'raw nhwc' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_raw_nhwc_and_nchw_normalization, which handles test raw nhwc and nchw normalization.

Line 14: `result = prepare_input(raw_nhwc, "raw-u8")` This assigns 'prepare_input(raw_nhwc, "raw-u8")' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are prepare_input: the value named 'prepare input' in this block; raw_nhwc: the value named 'raw nhwc' in this block; raw: the value named 'raw' in this block. Context: the function test_raw_nhwc_and_nchw_normalization, which handles test raw nhwc and nchw normalization.

Line 15: `self.assertEqual(result.shape, (1, 3, 32, 32))` This calls 'self.assertEqual' with 'result.shape, (1, 3, 32, 32)' inside the function test_raw_nhwc_and_nchw_normalization, which handles test raw nhwc and nchw normalization. The call advances the host-side job that checks run spikingformer pynq behavior independently with Python assertions.

Line 16: `self.assertAlmostEqual(float(result[0, 0, 0, 0]), -float(CIFAR10_MEAN[0] / CIFAR10_STD[0]), places=6)` This calls 'self.assertAlmostEqual' with 'float(result[0, 0, 0, 0]), -float(CIFAR10_MEAN[0] / CIFAR10_STD[0]), places=6' inside the function test_raw_nhwc_and_nchw_normalization, which handles test raw nhwc and nchw normalization. The call advances the host-side job that checks run spikingformer pynq behavior independently with Python assertions.

Line 17: `self.assertAlmostEqual(float(result[0, 1, 0, 0]), float((1 - CIFAR10_MEAN[1]) / CIFAR10_STD[1]), places=6)` This calls 'self.assertAlmostEqual' with 'float(result[0, 1, 0, 0]), float((1 - CIFAR10_MEAN[1]) / CIFAR10_STD[1]), places=6' inside the function test_raw_nhwc_and_nchw_normalization, which handles test raw nhwc and nchw normalization. The call advances the host-side job that checks run spikingformer pynq behavior independently with Python assertions.

Line 18: `np.testing.assert_array_equal(` This statement continues through source line 19; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_array_equal' with ' prepare_input(result, "normalized"), result.astype(np.float32)' inside the function test_raw_nhwc_and_nchw_normalization, which handles test raw nhwc and nchw normalization. The call advances the host-side job that checks run spikingformer pynq behavior independently with Python assertions.

Line 19: `prepare_input(result, "normalized"), result.astype(np.float32))` This is continuation line 2 of the 2-line logical statement that begins at source line 18. Its exact fragment is 'prepare_input(result, "normalized"), result.astype(np.float32))'. The important operands are prepare_input: the value named 'prepare input' in this block;

result: the value named 'result' in this block; normalized: the value named 'normalized' in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ prepare_input(result, "normalized"), result.astype(np.float32)‘ inside the function test_raw_nhwc_and_nchw_normalization, which handles test raw nhwc and nchw normalization. The call advances the host-side job that checks run spikingformer pynq behavior independently with Python assertions.

Line 21: `def test_rejects_wrong_shape_and_out_of_range_raw_values(self) -> None:` This starts function test_rejects_wrong_shape_and_out_of_range_raw_values. Its indented body performs test rejects wrong shape and out of range raw values as part of the file that checks run spikingformer pynq behavior independently with Python assertions; it runs only when called.

Line 22: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_rejects_wrong_shape_and_out_of_range_raw_values, which handles test rejects wrong shape and out of range raw values raises an error.

Line 23: `prepare_input(np.zeros((28, 28), dtype=np.uint8), "raw-u8")` This calls ‘prepare_input‘ with ‘np.zeros((28, 28), dtype=np.uint8), "raw-u8"‘ inside the function test_rejects_wrong_shape_and_out_of_range_raw_values, which handles test rejects wrong shape and out of range raw values. The call advances the host-side job that checks run spikingformer pynq behavior independently with Python assertions.

Line 24: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_rejects_wrong_shape_and_out_of_range_raw_values, which handles test rejects wrong shape and out of range raw values raises an error.

Line 25: `prepare_input(np.full((3, 32, 32), 256), "raw-u8")` This calls ‘prepare_input‘ with ‘np.full((3, 32, 32), 256), "raw-u8"‘ inside the function test_rejects_wrong_shape_and_out_of_range_raw_values, which handles test rejects wrong shape and out of range raw values. The call advances the host-side job that checks run spikingformer pynq behavior independently with Python assertions.

Line 28: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named 'name' in this block; ___main___: the value named 'main' in this block. Context: the file whose role is to checks run spikingformer pynq behavior independently with Python assertions.

Line 29: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks run spikingformer pynq behavior independently with Python assertions. The call advances the host-side job that checks run spikingformer pynq behavior independently with Python assertions.

70 tests/test_spikingformer_accelerator.py

Role in the complete system: checks spikingformer accelerator behavior independently with Python assertions.

70.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks spikingformer accelerator behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: VectorProgramEmulator, CombinedProgramEmulator, OneProjectionArchive, LifProgramEmulator, SpikingformerAcceleratorTests. Functions: _signed, execute_program, assert_vector_record, ___init___, execute_program, ___init___, projection, ___init___, execute_program, parameters, test_fabric_projection_requantizer_ma test_binary_projection_executes_tensor_and_vector_programs, test_asymmetric_dense_projection_executes_exact_zer test_chunked_lif_scheduler_preserves_every_logical_membrane, test_traced_lif_scheduler_returns_exact_pre_reset_m test_real_block0_archive_executes_exact_exported_integer_equation, test_vector_residual_and_binary_max_pool_cov test_global_pooling_and_head_division_execute_in_vector_fabric. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

70.2 Complete source

```
from __future__ import annotations

import unittest
from pathlib import Path

import numpy as np

from neuraedge_golden import VectorOperation, round_shift_signed, vector_alu
from neuraedge_protocol import Opcode, ProgramRecord, unpack_vector_result
from spikingformer_accelerator import (
    SpikingformerProjectionExecutor,
    binary_max_pool2d_on_accelerator,
    global_spatiotemporal_sum_on_accelerator,
    lif_sequence_on_accelerator,
    lif_sequence_reference,
    lif_sequence_with_trace_on_accelerator,
    lif_sequence_with_trace_reference,
    power_of_two_requantize_on_accelerator,
    requantize_projection_on_accelerator,
    requantize_projection_reference,
    vector_binary_on_accelerator,
)
from spikingformer_parameters import ProjectionParameters, SpikingformerParameterArchive
from tests.test_neuraedge_compiler import TensorTransactionEmulator


def _signed(value: int, bits: int) -> int:
    value &= (1 << bits) - 1
    return value - (1 << bits) if value & (1 << (bits - 1)) else value


class VectorProgramEmulator:
    def execute_program(self, records, *, timeout_seconds: float = 30.0):
        del timeout_seconds
        output: list[int] = []
        for record in records:
            self.assert_vector_record(record)
            command = record.command.value
            operation = (command >> 4) & 0xF
            multiplier = _signed(command >> 8, 16)
            shift = _signed(command >> 24, 5) if operation == 9 else (command >> 24) & 0x1F
            offset = _signed(command >> 29, 16)
            clamp_word = record.input_beats[0].data
            clamp_minimum = _signed(clamp_word, 32)
            clamp_maximum = _signed(clamp_word >> 32, 32)
            a = unpack_vector_result([beat.data for beat in record.input_beats[1:5]])
```

```
            b = unpack_vector_result([beat.data for beat in record.input_beats[5:9]])
            lanes = vector_alu(
                VectorOperation(operation), np.asarray(a), np.asarray(b),
                multiplier_q15=multiplier, right_shift=shift,
                output_zero_point=offset, clamp_minimum=clamp_minimum,
                clamp_maximum=clamp_maximum)
            for lane in range(0, 8, 2):
                output.append(
                    (int(lanes[lane]) & 0xFFFFFFFF)
                    | ((int(lanes[lane + 1]) & 0xFFFFFFFF) << 32))
        return tuple(output)

    @staticmethod
    def assert_vector_record(record: ProgramRecord) -> None:
        if len(record.input_beats) != 9 or record.output_beats != 4:
            raise AssertionError("invalid vector record")


class CombinedProgramEmulator:
    def __init__(self) -> None:
        self.tensor = TensorTransactionEmulator()
        self.vector = VectorProgramEmulator()

    def execute_program(self, records, *, timeout_seconds: float = 30.0):
        if records and all(record.command.opcode == Opcode.VECTOR_ALU for record in records):
            return self.vector.execute_program(records, timeout_seconds=timeout_seconds)
        return self.tensor.execute_program(records, timeout_seconds=timeout_seconds)


class OneProjectionArchive:
    def __init__(self, parameters: ProjectionParameters) -> None:
        self.parameters = parameters

    def projection(self, name: str) -> ProjectionParameters:
        if name != self.parameters.name:
            raise KeyError(name)
        return self.parameters


class LifProgramEmulator:
    def __init__(self) -> None:
        self.membrane = np.zeros(32, dtype=np.int64)
        self.timesteps = 0

    def execute_program(self, records, *, timeout_seconds: float = 30.0):
        del timeout_seconds
        output: list[int] = []
        for record in records:
            command = record.command.value
            if record.command.opcode != Opcode.LIF_STEP:
                raise AssertionError("expected an LIF record")
            leak_shift = (command >> 4) & 0x1F
            threshold = _signed(command >> 9, 24)
            reset = _signed(command >> 33, 24)
            if (command >> 57) & 1:
                self.membrane.fill(reset)
                self.timesteps = 0
            currents: list[int] = []
            for beat in record.input_beats:
                currents.extend(_signed(beat.data >> (16 * lane), 16) for lane in range(4))
            self.membrane = np.clip(
                self.membrane - (self.membrane >> leak_shift) + np.asarray(currents),
                -(1 << 23),
                (1 << 23) - 1,
            )
            pre_reset_membrane = self.membrane.copy()
            fired = self.membrane >= threshold
            self.membrane = np.where(fired, reset, self.membrane)
            self.timesteps += 1
            spike_word = sum(int(fired[lane]) << lane for lane in range(32))
            output.append(spike_word | (self.timesteps << 32))
            if (command >> 58) & 1:
                packed = sum(
                    (int(pre_reset_membrane[lane]) & 0xFFFFFF) << (24 * lane)
                    for lane in range(32)
                )
                output.extend(
                    (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF
                    for word in range(12)
                )
                if record.output_beats != 13:
                    raise AssertionError("traced LIF record must request thirteen beats")
            elif record.output_beats != 1:
                raise AssertionError("untraced LIF record must request one beat")
        return tuple(output)


```

```
class SpikingformerAcceleratorTests(unittest.TestCase):
    @staticmethod
    def parameters(name: str, weight: np.ndarray, opcode: str) -> ProjectionParameters:
        channels = weight.shape[1]
        return ProjectionParameters(
            name,
            weight.astype(np.int8),
            np.arange(channels, dtype=np.int32) * 17 - 31,
            np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16),
            np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8),
            opcode,
            (channels, weight.shape[0], 1),
            10,
        )

    def test_fabric_projection_requantizer_matches_independent_reference(self) -> None:
        rng = np.random.default_rng(20260713)
        rows = 19
        channels = 13
        parameters = ProjectionParameters(
            "test",
            np.zeros((17, channels), dtype=np.int8),
            rng.integers(-3000, 3001, size=channels, dtype=np.int32),
            rng.integers(-32768, 32768, size=channels, dtype=np.int16),
            rng.integers(-10, 11, size=channels, dtype=np.int8),
            "COMPUTE_BINARY",
            (channels, 17, 1, 1),
            10,
        )
        accumulators = rng.integers(-500_000, 500_001, size=(rows, channels), dtype=np.int64)
        expected = requantize_projection_reference(accumulators, parameters)
        actual = requantize_projection_on_accelerator(
            VectorProgramEmulator(), accumulators, parameters)
        np.testing.assert_array_equal(actual, expected)

    def test_binary_projection_executes_tensor_and_vector_programs(self) -> None:
        rng = np.random.default_rng(11)
        spikes = rng.integers(0, 2, size=(11, 7), dtype=np.uint8)
        weight = rng.integers(-127, 128, size=(7, 5), dtype=np.int16).astype(np.int8)
        parameters = self.parameters("binary", weight, "COMPUTE_BINARY")
        executor = SpikingformerProjectionExecutor(
            OneProjectionArchive(parameters), CombinedProgramEmulator())
        actual = executor.matrix("binary", spikes)
        expected_accumulator = spikes.astype(np.int64) @ weight.astype(np.int64)
        expected = requantize_projection_reference(expected_accumulator, parameters)
        np.testing.assert_array_equal(actual, expected)

    def test_asymmetric_dense_projection_executes_exact_zero_point_math(self) -> None:
        rng = np.random.default_rng(12)
        activations = rng.integers(0, 256, size=(9, 13), dtype=np.uint8)
        weight = rng.integers(-127, 128, size=(13, 6), dtype=np.int16).astype(np.int8)
        parameters = self.parameters("dense", weight, "COMPUTE_DENSE_OUTER")
        executor = SpikingformerProjectionExecutor(
            OneProjectionArchive(parameters), CombinedProgramEmulator())
        actual = executor.matrix("dense", activations, activation_zero_point=123)
        expected_accumulator = (
            activations.astype(np.int64) - 123) @ weight.astype(np.int64)
        expected = requantize_projection_reference(expected_accumulator, parameters)
        np.testing.assert_array_equal(actual, expected)

    def test_chunked_lif_scheduler_preserves_every_logical_membrane(self) -> None:
        rng = np.random.default_rng(13)
        values = rng.integers(-5000, 5001, size=(4, 3, 5, 7), dtype=np.int64)
        expected = lif_sequence_reference(
            values, threshold_q10=512, leak_shift=1, reset_q10=0)
        actual = lif_sequence_on_accelerator(
            LifProgramEmulator(), values,
            threshold_q10=512, leak_shift=1, reset_q10=0)
        np.testing.assert_array_equal(actual, expected)

    def test_traced_lif_scheduler_returns_exact_pre_reset_membrane(self) -> None:
        rng = np.random.default_rng(1301)
        values = rng.integers(-9000, 9001, size=(5, 2, 7, 5), dtype=np.int64)
        expected_spikes, expected_membrane = lif_sequence_with_trace_reference(
            values, threshold_q10=700, leak_shift=1, reset_q10=-17)
        actual_spikes, actual_membrane = lif_sequence_with_trace_on_accelerator(
            LifProgramEmulator(), values,
            threshold_q10=700, leak_shift=1, reset_q10=-17)
        np.testing.assert_array_equal(actual_spikes, expected_spikes)
        np.testing.assert_array_equal(actual_membrane, expected_membrane)
        self.assertTrue(np.any(actual_spikes))
        self.assertTrue(np.any(actual_membrane[actual_spikes.astype(bool)] >= 700))

    def test_real_block0_archive_executes_exact_exported_integer_equation(self) -> None:
        manifest = Path(__file__).resolve().parents[1] / (
            "outputs/spikingformer_hardware_parameters/"
            "spikingformer_neuraedge_weights.json")
```

```
        if not manifest.exists():
            self.skipTest("full trained parameter archive is not present")
        archive = SpikingformerParameterArchive(manifest)
        parameters = archive.projection("patch_embed.block0")
        rng = np.random.default_rng(14)
        activations = rng.integers(
            0, 256, size=(3, parameters.weight_k_by_n.shape[0]), dtype=np.uint8)
        executor = SpikingformerProjectionExecutor(archive, CombinedProgramEmulator())
        actual = executor.matrix(
            "patch_embed.block0", activations, activation_zero_point=123)
        accumulators = (
            activations.astype(np.int64) - 123
        ) @ parameters.weight_k_by_n.astype(np.int64)
        expected = requantize_projection_reference(accumulators, parameters)
        np.testing.assert_array_equal(actual, expected)

    def test_vector_residual_and_binary_max_pool_cover_edge_lanes(self) -> None:
        left = np.asarray([-(1 << 31), -3, 0, 7, (1 << 31) - 1, 4, 5, 6, 7])
        right = np.asarray([-1, 8, 0, -2, 1, 3, 2, 1, 0])
        added = vector_binary_on_accelerator(
            VectorProgramEmulator(), VectorOperation.ADD_SAT, left, right)
        np.testing.assert_array_equal(
            added, np.clip(left.astype(np.int64) + right, -(1 << 31), (1 << 31) - 1))

        rng = np.random.default_rng(15)
        spikes = rng.integers(0, 2, size=(2, 3, 5, 7), dtype=np.uint8)
        actual = binary_max_pool2d_on_accelerator(VectorProgramEmulator(), spikes)
        padded = np.pad(spikes, ((0, 0), (0, 0), (1, 1), (1, 1)))
        candidates = [
            padded[:, :, y:y + 5:2, x:x + 7:2]
            for y in range(3) for x in range(3)
        ]
        expected = np.maximum.reduce(candidates)
        np.testing.assert_array_equal(actual, expected)

    def test_global_pooling_and_head_division_execute_in_vector_fabric(self) -> None:
        values = np.arange(-120, 120, dtype=np.int64).reshape(3, 2, 5, 4, 2)
        pooled = global_spatiotemporal_sum_on_accelerator(
            VectorProgramEmulator(), values)
        np.testing.assert_array_equal(pooled, values.sum(axis=(0, 3, 4)))
        divided = power_of_two_requantize_on_accelerator(
            VectorProgramEmulator(),
            pooled,
            division_shift=5,
            clamp_minimum=-17,
            clamp_maximum=19,
        )
        np.testing.assert_array_equal(
            divided,
            np.clip(round_shift_signed(pooled, 5), -17, 19),
        )


if __name__ == "__main__":
    unittest.main()
```

70.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import unittest` This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 4: `from pathlib import Path` This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import numpy as np` This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `from neuraedge_golden import VectorOperation, round_shift_signed, vector_alu` This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `from neuraedge_protocol import Opcode, ProgramRecord, unpack_vector_result` This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `from spikingformer_accelerator import (` This statement continues through source line 22; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `SpikingformerProjectionExecutor,` This is continuation line 2 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'SpikingformerProjectionExecutor,'. The important operands are SpikingformerProjectionExecutor: the value named 'spikingformer projection executor' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `binary_max_pool2d_on_accelerator,` This is continuation line 3 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'binary_max_pool2d_on_accelerator,'. The important operands are binary_max_pool2d_on_accelerator: the value named 'binary max pool2d on accelerator' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `global_spatiotemporal_sum_on_accelerator,` This is continuation line 4 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'global_spatiotemporal_sum_on_accelerator,'. The important operands are global_spatiotemporal_sum_on_accelerator: the value named 'global spatiotemporal sum on accelerator' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `lif_sequence_on_accelerator,` This is continuation line 5 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'lif_sequence_on_accelerator,'. The important operands are lif_sequence_on_accelerator: the value named 'lif sequence on accelerator' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `lif_sequence_reference,` This is continuation line 6 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'lif_sequence_reference,'. The important operands are lif_sequence_reference: the value named 'lif sequence reference' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `lif_sequence_with_trace_on_accelerator,` This is continuation line 7 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'lif_sequence_with_trace_on_accelerator,'. The important operands are lif_sequence_with_trace_on_accelerator: the value named 'lif sequence with trace on accelerator' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `lif_sequence_with_trace_reference,` This is continuation line 8 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'lif_sequence_with_trace_reference,'. The important operands are lif_sequence_with_trace_reference: the value named 'lif sequence with trace reference' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `power_of_two_requantize_on_accelerator,` This is continuation line 9 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'power_of_two_requantize_on_accelerator,'. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `requantize_projection_on_accelerator,` This is continuation line 10 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'requantize_projection_on_accelerator,'. The important operands are requantize_projection_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `requantize_projection_reference,` This is continuation line 11 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'requantize_projection_reference,'. The important operands are requantize_projection_reference: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `vector_binary_on_accelerator,` This is continuation line 12 of the 13-line logical statement that begins at source line 10. Its exact fragment is 'vector_binary_on_accelerator,'. The important operands are vector_binary_on_accelerator: the value named 'vector binary on accelerator' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `)` This is continuation line 13 of the 13-line logical statement that begins at source line 10. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `from spikingformer_parameters import ProjectionParameters, SpikingformerParameterArchive` This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `from tests.test_neuraedge_compiler import TensorTransactionEmulator` This imports the named Python API so the file whose role is to checks spikingformer accelerator behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 27: `def _signed(value: int, bits: int) -> int:` This starts function _signed. Its indented body performs signed as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 28: `value &= (1 << bits) - 1` Inside the function _signed, which handles signed, this statement advances the host implementation that checks spikingformer accelerator behavior independently with Python assertions. The important operands are value: the value named 'value' in this block; bits: the value named 'bits' in this block.

Line 29: `return value - (1 << bits) if value & (1 << (bits - 1)) else value` This returns ‘value - (1 << bits) if value & (1 << (bits - 1)) else value‘ to the caller of the function _signed, which handles signed, ending this invocation. The important operands are value: the value named ’value’ in this block; bits: the value named ’bits’ in this block.

Line 32: `class VectorProgramEmulator:` This defines class VectorProgramEmulator, which groups data and methods for vector program emulator. Instances exist in host memory and organize the flow that checks spikingformer accelerator behavior independently with Python assertions.

Line 33: `def execute_program(self, records, *, timeout_seconds: float = 30.0):` This starts function execute_program. Its indented body performs execute program as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 34: `del timeout_seconds` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks spikingformer accelerator behavior independently with Python assertions. The important operands are del: the value named ’del’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block.

Line 35: `output: list[int] = []` This assigns ‘[]‘ to ‘output‘ in host memory. Here output means the value named ’output’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 36: `for record in records:` This iterates record over ‘records‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_program, which handles execute program.

Line 37: `self.assert_vector_record(record)` This calls ‘self.assert_vector_record‘ with ‘record‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 38: `command = record.command.value` This assigns ‘record.command.value‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are record: the value named ’record’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function execute_program, which handles execute program.

Line 39: `operation = (command >> 4) & 0xF` This assigns ‘(command >> 4) & 0xF‘ to ‘operation ‘ in host memory. Here operation means the value named ’operation’ in this block. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_program, which handles execute program.

Line 40: `multiplier = _signed(command >> 8, 16)` This assigns ‘_signed(command >> 8, 16)‘ to ‘multiplier ‘ in host memory. Here multiplier means the value named ’multiplier’ in this block. The important operands are _signed: the value named ’signed’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_program, which handles execute program.

Line 41: `shift = _signed(command >> 24, 5) if operation == 9 else (command >> 24) & 0x1F` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks spikingformer accelerator behavior independently with Python assertions. The important operands are shift: the value named ’shift’ in this block; _signed: the value named ’signed’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 42: `offset = _signed(command >> 29, 16)` This assigns '_signed(command >> 29, 16)' to 'offset ' in host memory. Here offset means the value named 'offset' in this block. The important operands are _signed: the value named 'signed' in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_program, which handles execute program.

Line 43: `clamp_word = record.input_beats[0].data` This assigns 'record.input_beats[0].data' to 'clamp_word ' in host memory. Here clamp_word means the value named 'clamp word' in this block. The important operands are record: the value named 'record' in this block; input_beats: the value named 'input beats' in this block; data: the value named 'data' in this block. Context: the function execute_program, which handles execute program.

Line 44: `clamp_minimum = _signed(clamp_word, 32)` This assigns '_signed(clamp_word, 32)' to 'clamp_minimum ' in host memory. Here clamp_minimum means the value named 'clamp minimum' in this block. The important operands are _signed: the value named 'signed' in this block; clamp_word: the value named 'clamp word' in this block. Context: the function execute_program, which handles execute program.

Line 45: `clamp_maximum = _signed(clamp_word >> 32, 32)` This assigns '_signed(clamp_word >> 32, 32)' to 'clamp_maximum ' in host memory. Here clamp_maximum means the value named 'clamp maximum' in this block. The important operands are _signed: the value named 'signed' in this block; clamp_word: the value named 'clamp word' in this block. Context: the function execute_program, which handles execute program.

Line 46: `a = unpack_vector_result([beat.data for beat in record.input_beats[1:5]])` This assigns 'unpack_vector_r for beat in record.input_beats[1:5]])' to 'a ' in host memory. Here a means the value named 'a' in this block. The important operands are unpack_vector_result: the value named 'unpack vector result' in this block; beat: the value named 'beat' in this block; data: the value named 'data' in this block. Context: the function execute_program, which handles execute program.

Line 47: `b = unpack_vector_result([beat.data for beat in record.input_beats[5:9]])` This assigns 'unpack_vector_r for beat in record.input_beats[5:9]])' to 'b ' in host memory. Here b means the value named 'b' in this block. The important operands are unpack_vector_result: the value named 'unpack vector result' in this block; beat: the value named 'beat' in this block; data: the value named 'data' in this block. Context: the function execute_program, which handles execute program.

Line 48: `lanes = vector_alu(` This statement continues through source line 52; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'vector_alu(VectorOperation(operation), np.asarray(a), np.asarray(b), multiplier_q15=multiplier, right_shift=shift, output_zero_point=offset, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum)' to 'lanes ' in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; operation: the value named 'operation' in this block. Context: the function execute_program, which handles execute program.

Line 49: `VectorOperation(operation), np.asarray(a), np.asarray(b),` This is continuation line 2 of the 5-line logical statement that begins at source line 48. Its exact fragment is 'VectorOperation(operation), np.asarray(a), np.asarray(b),'. The important operands are VectorOperation: the value named 'vector operation' in this block; operation: the value named 'operation' in this block; asarray: the value named 'asarray' in this block. Read together, the complete operation is: This assigns 'vector_alu(VectorOperation(operation), np.asarray(a), np.asarray(b), multiplier_q15=multiplier, right_shift=shift, output_zero_point=offset, clamp_minimum=clamp_minimum, clamp_maximum=clamp to 'lanes ' in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; operation: the value named 'operation' in this block. Context: the function execute_program, which handles execute program.

Line 50: `multiplier_q15=multiplier, right_shift=shift,` This is continuation line 3 of the 5-line logical statement that begins at source line 48. Its exact fragment is 'multiplier_q15=multiplier, right_shift=shift,'. The important operands are multiplier_q15: the value named 'multiplier q15' in this block; multiplier: the value named 'multiplier' in this

block; right_shift: the value named 'right shift' in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation(operation), np.asarray(a), np.asarray(b), multiplier_q15=multiplier, right_shift=shift, output_zero_point=offset, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum)‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; operation: the value named 'operation' in this block. Context: the function execute_program, which handles execute program.

Line 51: `output_zero_point=offset, clamp_minimum=clamp_minimum,` This is continuation line 4 of the 5-line logical statement that begins at source line 48. Its exact fragment is ‘output_zero_point=offset, clamp_minimum=clamp_minimum,‘. The important operands are output_zero_point: the integer code representing real numeric zero in affine quantization; offset: the value named 'offset' in this block; clamp_minimum: the value named 'clamp minimum' in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation(operation), np.asarray(a), np.asarray(b), multiplier_q15=multiplier, right_shift=shift, output_zero_point=offset, clamp_minimum=clamp_minimum, clamp_maximum=
to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; operation: the value named 'operation' in this block. Context: the function execute_program, which handles execute program.

Line 52: `clamp_maximum=clamp_maximum)` This is continuation line 5 of the 5-line logical statement that begins at source line 48. Its exact fragment is ‘clamp_maximum=clamp_maximum)‘. The important operands are clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This assigns ‘vector_alu(VectorOperation(operation), np.asarray(a), np.asarray(b), multiplier_q15=multiplier, right_shift=shift, output_zero_point=offset, clamp_minimum=clamp_minimum, clamp_maximum=clamp_maximum)‘ to ‘lanes ‘ in host memory. Here lanes means one parallel arithmetic element within a vector or tensor datapath. The important operands are vector_alu: the value named 'vector alu' in this block; VectorOperation: the value named 'vector operation' in this block; operation: the value named 'operation' in this block. Context: the function execute_program, which handles execute program.

Line 53: `for lane in range(0, 8, 2):` This iterates lane over ‘range(0, 8, 2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_program, which handles execute program.

Line 54: `output.append(` This statement continues through source line 56; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘output.append‘ with ‘ (int(lanes[lane]) & 0xFFFFFFFF) | ((int(lanes[lane + 1]) & 0xFFFFFFFF) << 32)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 55: `(int(lanes[lane]) & 0xFFFFFFFF)` This is continuation line 2 of the 3-line logical statement that begins at source line 54. Its exact fragment is ‘(int(lanes[lane]) & 0xFFFFFFFF)‘. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘output.append‘ with ‘ (int(lanes[lane]) & 0xFFFFFFFF) | ((int(lanes[lane + 1]) & 0xFFFFFFFF) << 32)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 56: `| ((int(lanes[lane + 1]) & 0xFFFFFFFF) << 32))` This is continuation line 3 of the 3-line logical statement that begins at source line 54. Its exact fragment is ‘| ((int(lanes[lane + 1]) & 0xFFFFFFFF) << 32))‘. The important operands are lanes: one parallel arithmetic element within a vector or tensor datapath; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This calls ‘output.append‘ with ‘ (int(lanes[lane]) & 0xFFFFFFFF) | ((int(lanes[lane + 1]) & 0xFFFFFFFF) << 32)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 57: `return tuple(output)` This returns ‘tuple(output)‘ to the caller of the function execute_program, which handles execute program, ending this invocation. The important operands are tuple: the value named 'tuple' in this block; output: the value named 'output' in this block.

Line 59: `@staticmethod` This decorator changes how the following class or function is constructed. ‘@staticmethod‘ is applied at import time on the host and is part of the class VectorProgramEmulator, which handles vector program emulator; it creates no FPGA circuit by itself.

Line 60: `def assert_vector_record(record: ProgramRecord) -> None:` This starts function assert_vector_record. Its indented body performs assert vector record as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 61: `if len(record.input_beats) != 9 or record.output_beats != 4:` This executes the following indented host-code branch only when ‘len(record.input_beats) != 9 or record.output_beats != 4‘ is true. The important operands are len: the value named ’len’ in this block; record: the value named ’record’ in this block; input_beats: the value named ’input beats’ in this block. Context: the function assert_vector_record, which handles assert vector record.

Line 62: `raise AssertionError("invalid vector record")` This rejects an invalid state by raising ‘AssertionError("invalid vector record")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 65: `class CombinedProgramEmulator:` This defines class CombinedProgramEmulator, which groups data and methods for combined program emulator. Instances exist in host memory and organize the flow that checks spikingformer accelerator behavior independently with Python assertions.

Line 66: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 67: `self.tensor = TensorTransactionEmulator()` This assigns ‘TensorTransactionEmulator()‘ to ‘self.tensor ‘ in host memory. Here tensor means the value named ’tensor’ in this block. The important operands are TensorTransactionEmulator: the value named ’tensor transaction emulator’ in this block. Context: the function ___init___, which handles init.

Line 68: `self.vector = VectorProgramEmulator()` This assigns ‘VectorProgramEmulator()‘ to ‘self.vector ‘ in host memory. Here vector means the value named ’vector’ in this block. The important operands are VectorProgramEmulator: the value named ’vector program emulator’ in this block. Context: the function ___init___, which handles init.

Line 70: `def execute_program(self, records, *, timeout_seconds: float = 30.0):` This starts function execute_program. Its indented body performs execute program as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 71: `if records and all(record.command.opcode == Opcode.VECTOR_ALU for record in records):` This executes the following indented host-code branch only when ‘records and all(record.command.opcode == Opcode.VECTOR_ALU for record in records)‘ is true. The important operands are records: the number of descriptor records accepted since program start; and: the value named ’and’ in this block; all: the value named ’all’ in this block. Context: the function execute_program, which handles execute program.

Line 72: `return self.vector.execute_program(records, timeout_seconds=timeout_seconds)` This returns ‘self.vector.exe timeout_seconds=timeout_seconds)‘ to the caller of the function execute_program, which handles execute program, ending this invocation. The important operands are vector: the value named ’vector’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start.

Line 73: `return self.tensor.execute_program(records, timeout_seconds=timeout_seconds)` This returns ‘self.tensor.exe timeout_seconds=timeout_seconds)‘ to the caller of the function execute_program, which handles execute program, ending this invocation. The important operands are tensor: the value named ’tensor’ in this block; execute_program: the value named ’execute program’ in this block; records: the number of descriptor records accepted since program start.

Line 76: `class OneProjectionArchive:` This defines class OneProjectionArchive, which groups data and methods for one projection archive. Instances exist in host memory and organize the flow that checks spikingformer accelerator behavior independently with Python assertions.

Line 77: `def __init__(self, parameters: ProjectionParameters) -> None:` This starts function ___init___. Its indented body performs init as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 78: `self.parameters = parameters` This assigns ‘parameters‘ to ‘self.parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are parameters: the value named ’parameters’ in this block. Context: the function ___init___, which handles init.

Line 80: `def projection(self, name: str) -> ProjectionParameters:` This starts function projection. Its indented body performs projection as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 81: `if name != self.parameters.name:` This executes the following indented host-code branch only when ‘name != self.parameters.name‘ is true. The important operands are name: the value named ’name’ in this block; parameters: the value named ’parameters’ in this block. Context: the function projection, which handles projection.

Line 82: `raise KeyError(name)` This rejects an invalid state by raising ‘KeyError(name)‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 83: `return self.parameters` This returns ‘self.parameters‘ to the caller of the function projection, which handles projection, ending this invocation. The important operands are parameters: the value named ’parameters’ in this block.

Line 86: `class LifProgramEmulator:` This defines class LifProgramEmulator, which groups data and methods for lif program emulator. Instances exist in host memory and organize the flow that checks spikingformer accelerator behavior independently with Python assertions.

Line 87: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 88: `self.membrane = np.zeros(32, dtype=np.int64)` This assigns ‘np.zeros(32, dtype=np.int64)‘ to ‘self.membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function ___init___, which handles init.

Line 89: `self.timesteps = 0` This assigns ‘0‘ to ‘self.timesteps ‘ in host memory. Here timesteps means one discrete SNN update instant. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 91: `def execute_program(self, records, *, timeout_seconds: float = 30.0):` This starts function execute_program. Its indented body performs execute program as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 92: `del timeout_seconds` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks spikingformer accelerator behavior independently with Python assertions. The important operands are del: the value named ’del’ in this block; timeout_seconds: the value named ’timeout seconds’ in this block.

Line 93: `output: list[int] = []` This assigns ‘[]‘ to ‘output‘ in host memory. Here output means the value named ’output’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 94: `for record in records:` This iterates record over ‘records‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_program, which handles execute program.

Line 95: `command = record.command.value` This assigns ‘record.command.value‘ to ‘command ‘ in host memory. Here command means the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. The important operands are record: the value named ’record’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; value: the value named ’value’ in this block. Context: the function execute_program, which handles execute program.

Line 96: `if record.command.opcode != Opcode.LIF_STEP:` This executes the following indented host-code branch only when ‘record.command.opcode != Opcode.LIF_STEP‘ is true. The important operands are record: the value named ’record’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; opcode: the value named ’opcode’ in this block. Context: the function execute_program, which handles execute program.

Line 97: `raise AssertionError("expected an LIF record")` This rejects an invalid state by raising ‘AssertionError("expected an LIF record")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 98: `leak_shift = (command >> 4) & 0x1F` This assigns ‘(command >> 4) & 0x1F‘ to ‘leak_shift ‘ in host memory. Here leak_shift means the retained fraction of prior membrane state. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_program, which handles execute program.

Line 99: `threshold = _signed(command >> 9, 24)` This assigns ‘_signed(command >> 9, 24)‘ to ‘threshold ‘ in host memory. Here threshold means the membrane level at which a neuron emits a spike. The important operands are _signed: the value named ’signed’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_program, which handles execute program.

Line 100: `reset = _signed(command >> 33, 24)` This assigns ‘_signed(command >> 33, 24)‘ to ‘reset ‘ in host memory. Here reset means the value named ’reset’ in this block. The important operands are _signed: the value named ’signed’ in this block; command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_program, which handles execute program.

Line 101: `if (command >> 57) & 1:` This executes the following indented host-code branch only when ‘(command >> 57) & 1‘ is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_program, which handles execute program.

Line 102: `self.membrane.fill(reset)` This calls ‘self.membrane.fill‘ with ‘reset‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 103: `self.timesteps = 0` This assigns ‘0‘ to ‘self.timesteps ‘ in host memory. Here timesteps means one discrete SNN update instant. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 104: `currents: list[int] = []` This assigns ‘[]‘ to ‘currents‘ in host memory. Here currents means the value named ’currents’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function execute_program, which handles execute program.

Line 105: `for beat in record.input_beats:` This iterates beat over ‘record.input_beats‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function execute_program, which handles execute program.

Line 106: `currents.extend(_signed(beat.data >> (16 * lane), 16) for lane in range(4))` This calls ‘currents.extend‘ with ‘_signed(beat.data >> (16 * lane), 16) for lane in range(4)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 107: `self.membrane = np.clip(` This statement continues through source line 111; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘np.clip(self.membrane - (self.membrane >> leak_shift) + np.asarray(currents), -(1 << 23), (1 << 23) - 1,)‘ to ‘self.membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are clip: the value named ’clip’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function execute_program, which handles execute program.

Line 108: `self.membrane - (self.membrane >> leak_shift) + np.asarray(currents),` This is continuation line 2 of the 5-line logical statement that begins at source line 107. Its exact fragment is ‘self.membrane - (self.membrane >> leak_shift) + np.asarray(currents),‘. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state; asarray: the value named ’asarray’ in this block. Read together, the complete operation is: This assigns ‘np.clip(self.membrane - (self.membrane >> leak_shift) + np.asarray(currents), -(1 << 23), (1 << 23) - 1,)‘ to ‘self.membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are clip: the value named ’clip’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function execute_program, which handles execute program.

Line 109: `-(1 << 23),` This is continuation line 3 of the 5-line logical statement that begins at source line 107. Its exact fragment is ‘-(1 << 23),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.clip(self.membrane - (self.membrane >> leak_shift) + np.asarray(currents), -(1 << 23), (1 << 23) - 1,)‘ to ‘self.membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are clip: the value named ’clip’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function execute_program, which handles execute program.

Line 110: `(1 << 23) - 1,` This is continuation line 4 of the 5-line logical statement that begins at source line 107. Its exact fragment is ‘(1 << 23) - 1,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.clip(self.membrane - (self.membrane >> leak_shift) + np.asarray(currents), -(1 << 23), (1 << 23) - 1,)‘ to ‘self.membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are clip: the value named ’clip’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function execute_program, which handles execute program.

Line 111: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 107. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘np.clip(self.membrane - (self.membrane >> leak_shift) + np.asarray(currents), -(1 << 23), (1 << 23) - 1,)‘ to ‘self.membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are clip: the value named ’clip’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; leak_shift: the retained fraction of prior membrane state. Context: the function execute_program, which handles execute program.

Line 112: `pre_reset_membrane = self.membrane.copy()` This assigns ‘self.membrane.copy()‘ to ‘pre_reset_membrane ‘ in host memory. Here pre_reset_membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are membrane: the persistent LIF neuron state carrying charge between timesteps; copy: the value named ’copy’ in this block. Context: the function execute_program, which handles execute program.

Line 113: `fired = self.membrane >= threshold` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks spikingformer accelerator behavior independently with Python assertions. The important operands are fired: the value named ’fired’ in this block; membrane: the persistent LIF neuron state carrying charge between timesteps; threshold: the membrane level at which a neuron emits a spike.

Line 114: `self.membrane = np.where(fired, reset, self.membrane)` This assigns ‘np.where(fired, reset, self.membrane)‘ to ‘self.membrane ‘ in host memory. Here membrane means the persistent LIF neuron state carrying charge between timesteps. The important operands are where: the value named ’where’ in this block; fired: the value named ’fired’ in this block; reset: the value named ’reset’ in this block. Context: the function execute_program, which handles execute program.

Line 115: `self.timesteps += 1` Inside the function execute_program, which handles execute program, this statement advances the host implementation that checks spikingformer accelerator behavior independently with Python assertions. The important operands are timesteps: one discrete SNN update instant.

Line 116: `spike_word = sum(int(fired[lane]) << lane for lane in range(32))` This assigns ‘sum(int(fired[lane]) << lane for lane in range(32))‘ to ‘spike_word ‘ in host memory. Here spike_word means a binary event produced when a neuron crosses its threshold. The important operands are sum: the value named ’sum’ in this block; fired: the value named ’fired’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute_program, which handles execute program.

Line 117: `output.append(spike_word | (self.timesteps << 32))` This calls ‘output.append‘ with ‘spike_word | (self.timesteps << 32)‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 118: `if (command >> 58) & 1:` This executes the following indented host-code branch only when ‘(command >> 58) & 1‘ is true. The important operands are command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters. Context: the function execute_program, which handles execute program.

Line 119: `packed = sum(` This statement continues through source line 122; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘sum((int(pre_reset_membrane[lane]) & 0xFFFFFF) << (24 * lane) for lane in range(32))‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are sum: the value named ’sum’ in this block; pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute_program, which handles execute program.

Line 120: `(int(pre_reset_membrane[lane]) & 0xFFFFFF) << (24 * lane)` This is continuation line 2 of the 4-line logical statement that begins at source line 119. Its exact fragment is ‘(int(pre_reset_membrane[lane]) & 0xFFFFFF) << (24 * lane)‘. The important operands are pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Read together, the complete operation is: This assigns ‘sum((int(pre_reset_membrane[lane]) & 0xFFFFFF) << (24 * lane) for lane in range(32))‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are sum: the value named ’sum’ in this block; pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute_program, which handles execute program.

Line 121: `for lane in range(32)` This is continuation line 3 of the 4-line logical statement that begins at source line 119. Its exact fragment is ‘for lane in range(32)‘. The important operands are for: the value named ’for’ in this block; lane: one parallel arithmetic element within a vector or tensor datapath; in: the value named ’in’ in this block. Read together, the complete operation is: This assigns ‘sum((int(pre_reset_membrane[lane]) & 0xFFFFFF) << (24 * lane) for lane in range(32))‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are sum: the value named ’sum’ in this block; pre_reset_membrane: the persistent LIF neuron state carrying charge between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute_program, which handles execute program.

Line 122: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 119. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘sum((int(pre_reset_membrane[lane]) & 0xFFFFFF) << (24 * lane) for lane in range(32))‘ to ‘packed ‘ in host memory. Here packed means the value named ’packed’ in this block. The important operands are sum: the value named ’sum’ in this block; pre_reset_membrane: the persistent LIF neuron state carrying charge

between timesteps; lane: one parallel arithmetic element within a vector or tensor datapath. Context: the function execute_program, which handles execute program.

Line 123: `output.extend(` This statement continues through source line 126; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘output.extend‘ with ‘ (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(12) ‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 124: `(packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF` This is continuation line 2 of the 4-line logical statement that begins at source line 123. Its exact fragment is ‘(packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF‘. The important operands are packed: the value named ’packed’ in this block; word: the value named ’word’ in this block. Read together, the complete operation is: This calls ‘output.extend‘ with ‘ (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(12) ‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 125: `for word in range(12)` This is continuation line 3 of the 4-line logical statement that begins at source line 123. Its exact fragment is ‘for word in range(12)‘. The important operands are for: the value named ’for’ in this block; word: the value named ’word’ in this block; in: the value named ’in’ in this block. Read together, the complete operation is: This calls ‘output.extend‘ with ‘ (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(12) ‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 126: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 123. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘output.extend‘ with ‘ (packed >> (64 * word)) & 0xFFFFFFFFFFFFFFFF for word in range(12) ‘ inside the function execute_program, which handles execute program. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 127: `if record.output_beats != 13:` This executes the following indented host-code branch only when ‘record.output_be
!= 13‘ is true. The important operands are record: the value named ’record’ in this block; output_beats: the value named ’output beats’ in this block. Context: the function execute_program, which handles execute program.

Line 128: `raise AssertionError("traced LIF record must request thirteen beats")` This rejects an invalid state by raising ‘AssertionError("traced LIF record must request thirteen beats")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 129: `elif record.output_beats != 1:` This executes the following indented host-code branch only when ‘record.output_beats != 1‘ is true. The important operands are record: the value named ’record’ in this block; output_beats: the value named ’output beats’ in this block. Context: the function execute_program, which handles execute program.

Line 130: `raise AssertionError("untraced LIF record must request one beat")` This rejects an invalid state by raising ‘AssertionError("untraced LIF record must request one beat")‘. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 131: `return tuple(output)` This returns ‘tuple(output)‘ to the caller of the function execute_program, which handles execute program, ending this invocation. The important operands are tuple: the value named ’tuple’ in this block; output: the value named ’output’ in this block.

Line 134: `class SpikingformerAcceleratorTests(unittest.TestCase):` This defines class SpikingformerAcceleratorTests, which groups data and methods for spikingformer accelerator tests. Instances exist in host memory and organize the flow that checks spikingformer accelerator behavior independently with Python assertions.

Line 135: `@staticmethod` This decorator changes how the following class or function is constructed. '@staticmethod' is applied at import time on the host and is part of the class SpikingformerAcceleratorTests, which handles spikingformer accelerator tests; it creates no FPGA circuit by itself.

Line 136: `def parameters(name: str, weight: np.ndarray, opcode: str) -> ProjectionParameters:` This starts function parameters. Its indented body performs parameters as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 137: `channels = weight.shape[1]` This assigns 'weight.shape[1]' to 'channels ' in host memory. Here channels means the value named 'channels' in this block. The important operands are weight: a learned coefficient multiplied by an activation; shape: the value named 'shape' in this block. Context: the function parameters, which handles parameters.

Line 138: `return ProjectionParameters(` This statement continues through source line 147; the explanation uses the complete expression rather than only the first visual fragment. This returns 'ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)' to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 139: `name,` This is continuation line 2 of the 10-line logical statement that begins at source line 138. Its exact fragment is 'name,'. The important operands are name: the value named 'name' in this block. Read together, the complete operation is: This returns 'ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)' to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 140: `weight.astype(np.int8),` This is continuation line 3 of the 10-line logical statement that begins at source line 138. Its exact fragment is 'weight.astype(np.int8),'. The important operands are weight: a learned coefficient multiplied by an activation; astype: the value named 'astype' in this block; int8: the value named 'int8' in this block. Read together, the complete operation is: This returns 'ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)' to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 141: `np.arange(channels, dtype=np.int32) * 17 - 31,` This is continuation line 4 of the 10-line logical statement that begins at source line 138. Its exact fragment is 'np.arange(channels, dtype=np.int32) * 17 - 31,'. The important operands are arange: the value named 'arange' in this block; channels: the value named 'channels' in this block; dtype: the value named 'dtype' in this block. Read together, the complete operation is: This returns 'ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)' to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 142: `np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16),` This is continuation line 5 of the 10-line logical statement that begins at source line 138. Its exact fragment is 'np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16),'. The important operands are asarray: the value named 'asarray' in this block; channel: the value named 'channel' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This returns 'ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)],

dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)‘ to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 143: `np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8),` This is continuation line 6 of the 10-line logical statement that begins at source line 138. Its exact fragment is ‘np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8),‘. The important operands are asarray: the value named 'asarray' in this block; channel: the value named 'channel' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This returns ‘ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)‘ to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 144: `opcode,` This is continuation line 7 of the 10-line logical statement that begins at source line 138. Its exact fragment is ‘opcode,‘. The important operands are opcode: the value named 'opcode' in this block. Read together, the complete operation is: This returns ‘ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)‘ to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 145: `(channels, weight.shape[0], 1),` This is continuation line 8 of the 10-line logical statement that begins at source line 138. Its exact fragment is ‘(channels, weight.shape[0], 1),‘. The important operands are channels: the value named 'channels' in this block; weight: a learned coefficient multiplied by an activation; shape: the value named 'shape' in this block. Read together, the complete operation is: This returns ‘ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)‘ to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 146: `10,` This is continuation line 9 of the 10-line logical statement that begins at source line 138. Its exact fragment is ‘10,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)‘ to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 147: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 138. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This returns ‘ProjectionParameters(name, weight.astype(np.int8), np.arange(channels, dtype=np.int32) * 17 - 31, np.asarray([16384 + 733 * channel for channel in range(channels)], dtype=np.int16), np.asarray([(channel % 3) - 1 for channel in range(channels)], dtype=np.int8), opcode, (channels, weight.shape[0], 1), 10,)‘ to the caller of the function parameters, which handles parameters, ending this invocation. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation.

Line 149: `def test_fabric_projection_requantizer_matches_independent_reference(self) -> None:` This starts function test_fabric_projection_requantizer_matches_independent_reference. Its indented body performs test fabric

projection requantizer matches independent reference as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 150: `rng = np.random.default_rng(20260713)` This assigns ‘np.random.default_rng(20260713)‘ to ‘rng ‘ in host memory. Here rng means the value named ’rng’ in this block. The important operands are random: the value named ’random’ in this block; default_rng: the value named ’default rng’ in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 151: `rows = 19` This assigns ‘19‘ to ‘rows ‘ in host memory. Here rows means one output-row position processed by the tensor tile. The expression is a literal or structural operator and has no hidden software state. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 152: `channels = 13` This assigns ‘13‘ to ‘channels ‘ in host memory. Here channels means the value named ’channels’ in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 153: `parameters = ProjectionParameters(` This statement continues through source line 162; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels, dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; test: the value named ’test’ in this block; zeros: the value named ’zeros’ in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 154: `"test",` This is continuation line 2 of the 10-line logical statement that begins at source line 153. Its exact fragment is ‘"test",‘. The important operands are test: the value named ’test’ in this block. Read together, the complete operation is: This assigns ‘ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels, dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; test: the value named ’test’ in this block; zeros: the value named ’zeros’ in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 155: `np.zeros((17, channels), dtype=np.int8),` This is continuation line 3 of the 10-line logical statement that begins at source line 153. Its exact fragment is ‘np.zeros((17, channels), dtype=np.int8),‘. The important operands are zeros: the value named ’zeros’ in this block; channels: the value named ’channels’ in this block; dtype: the value named ’dtype’ in this block. Read together, the complete operation is: This assigns ‘ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels, dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; test: the value named ’test’ in this block; zeros: the value named ’zeros’ in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 156: `rng.integers(-3000, 3001, size=channels, dtype=np.int32),` This is continuation line 4 of the 10-line logical statement that begins at source line 153. Its exact fragment is ‘rng.integers(-3000, 3001, size=channels, dtype=np.int32),‘. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Read together, the complete operation is: This assigns ‘ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels,

dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; test: the value named ’test’ in this block; zeros: the value named ’zeros’ in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 157: `rng.integers(-32768, 32768, size=channels, dtype=np.int16),` This is continuation line 5 of the 10-line logical statement that begins at source line 153. Its exact fragment is ‘rng.integers(-32768, 32768, size=channels, dtype=np.int16),‘. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Read together, the complete operation is: This assigns ‘ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels, dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; test: the value named ’test’ in this block; zeros: the value named ’zeros’ in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 158: `rng.integers(-10, 11, size=channels, dtype=np.int8),` This is continuation line 6 of the 10-line logical statement that begins at source line 153. Its exact fragment is ‘rng.integers(-10, 11, size=channels, dtype=np.int8),‘. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Read together, the complete operation is: This assigns ‘ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels, dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; test: the value named ’test’ in this block; zeros: the value named ’zeros’ in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 159: `"COMPUTE_BINARY",` This is continuation line 7 of the 10-line logical statement that begins at source line 153. Its exact fragment is ‘"COMPUTE_BINARY",‘. The important operands are COMPUTE_BINARY: the value named ’compute binary’ in this block. Read together, the complete operation is: This assigns ‘ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels, dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; test: the value named ’test’ in this block; zeros: the value named ’zeros’ in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 160: `(channels, 17, 1, 1),` This is continuation line 8 of the 10-line logical statement that begins at source line 153. Its exact fragment is ‘(channels, 17, 1, 1),‘. The important operands are channels: the value named ’channels’ in this block. Read together, the complete operation is: This assigns ‘ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels, dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; test: the value named ’test’ in this block; zeros: the value named ’zeros’ in this block. Context: the function test_fabric_projection_requantizer_matches_indepen which handles test fabric projection requantizer matches independent reference.

Line 161: `10,` This is continuation line 9 of the 10-line logical statement that begins at source line 153. Its exact fragment is ‘10,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels, dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)‘ to ‘parameters ‘ in host

memory. Here parameters means the value named 'parameters' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; test: the value named 'test' in this block; zeros: the value named 'zeros' in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 162: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 153. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'ProjectionParameters("test", np.zeros((17, channels), dtype=np.int8), rng.integers(-3000, 3001, size=channels, dtype=np.int32), rng.integers(-32768, 32768, size=channels, dtype=np.int16), rng.integers(-10, 11, size=channels, dtype=np.int8), "COMPUTE_BINARY", (channels, 17, 1, 1), 10,)' to 'parameters ' in host memory. Here parameters means the value named 'parameters' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; test: the value named 'test' in this block; zeros: the value named 'zeros' in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 163: `accumulators = rng.integers(-500_000, 500_001, size=(rows, channels), dtype=np.int64)` This assigns 'rng.integers(-500_000, 500_001, size=(rows, channels), dtype=np.int64)' to 'accumulators ' in host memory. Here accumulators means a widened running sum that prevents intermediate overflow. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 164: `expected = requantize_projection_reference(accumulators, parameters)` This assigns 'requantize_projection parameters)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are requantize_projection_reference: the conversion of a wide accumulated result back into a bounded activation code; accumulators: a widened running sum that prevents intermediate overflow; parameters: the value named 'parameters' in this block. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 165: `actual = requantize_projection_on_accelerator(` This statement continues through source line 166; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'requantize_projection_on_acce VectorProgramEmulator(), accumulators, parameters)' to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are requantize_projection_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; VectorProgramEmulator: the value named 'vector program emulator' in this block; accumulators: a widened running sum that prevents intermediate overflow. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 166: `VectorProgramEmulator(), accumulators, parameters)` This is continuation line 2 of the 2-line logical statement that begins at source line 165. Its exact fragment is 'VectorProgramEmulator(), accumulators, parameters)'. The important operands are VectorProgramEmulator: the value named 'vector program emulator' in this block; accumulators: a widened running sum that prevents intermediate overflow; parameters: the value named 'parameters' in this block. Read together, the complete operation is: This assigns 'requantize_projection_on_accelerator(VectorProgramEmulator(), accumulators, parameters)' to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are requantize_projection_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; VectorProgramEmulator: the value named 'vector program emulator' in this block; accumulators: a widened running sum that prevents intermediate overflow. Context: the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference.

Line 167: `np.testing.assert_array_equal(actual, expected)` This calls 'np.testing.assert_array_equal' with 'actual, expected' inside the function test_fabric_projection_requantizer_matches_independent_reference, which handles test fabric projection requantizer matches independent reference. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 169: `def test_binary_projection_executes_tensor_and_vector_programs(self) -> None:` This starts function test_binary_projection_executes_tensor_and_vector_programs. Its indented body performs test binary projection executes tensor and vector programs as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 170: `rng = np.random.default_rng(11)` This assigns ‘np.random.default_rng(11)‘ to ‘rng ‘ in host memory. Here rng means the value named ’rng’ in this block. The important operands are random: the value named ’random’ in this block; default_rng: the value named ’default rng’ in this block. Context: the function test_binary_projection_executes_tensor_and which handles test binary projection executes tensor and vector programs.

Line 171: `spikes = rng.integers(0, 2, size=(11, 7), dtype=np.uint8)` This assigns ‘rng.integers(0, 2, size=(11, 7), dtype=np.uint8)‘ to ‘spikes ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_binary_projection_executes_tensor_and_vector_progr which handles test binary projection executes tensor and vector programs.

Line 172: `weight = rng.integers(-127, 128, size=(7, 5), dtype=np.int16).astype(np.int8)` This assigns ‘rng.integers(-127, 128, size=(7, 5), dtype=np.int16).astype(np.int8)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_binary_projection_executes_tensor_and_vector_programs, which handles test binary projection executes tensor and vector programs.

Line 173: `parameters = self.parameters("binary", weight, "COMPUTE_BINARY")` This assigns ‘self.parameters("binary", weight, "COMPUTE_BINARY")‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are parameters: the value named ’parameters’ in this block; binary: the value named ’binary’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function test_binary_projection_executes_tensor_and_vector_programs, which handles test binary projection executes tensor and vector programs.

Line 174: `executor = SpikingformerProjectionExecutor(` This statement continues through source line 175; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘SpikingformerProjectionExecutor(OneProjectionArchive(parameters), CombinedProgramEmulator())‘ to ‘executor ‘ in host memory. Here executor means the value named ’executor’ in this block. The important operands are SpikingformerProjectionExecutor: the value named ’spikingformer projection executor’ in this block; OneProjectionArchive: the value named ’one projection archive’ in this block; parameters: the value named ’parameters’ in this block. Context: the function test_binary_projection_executes_tensor_and_vector_programs, which handles test binary projection executes tensor and vector programs.

Line 175: `OneProjectionArchive(parameters), CombinedProgramEmulator())` This is continuation line 2 of the 2-line logical statement that begins at source line 174. Its exact fragment is ‘OneProjectionArchive(parameters), CombinedProgramEmulator())‘. The important operands are OneProjectionArchive: the value named ’one projection archive’ in this block; parameters: the value named ’parameters’ in this block; CombinedProgramEmulator: the value named ’combined program emulator’ in this block. Read together, the complete operation is: This assigns ‘SpikingformerProjectionExecutor(OneProjectionArchive(parameters), CombinedProgramEmulator())‘ to ‘executor ‘ in host memory. Here executor means the value named ’executor’ in this block. The important operands are SpikingformerProjectionExecutor: the value named ’spikingformer projection executor’ in this block; OneProjectionArchive: the value named ’one projection archive’ in this block; parameters: the value named ’parameters’ in this block. Context: the function test_binary_projection_executes_tensor_and_vector_programs, which handles test binary projection executes tensor and vector programs.

Line 176: `actual = executor.matrix("binary", spikes)` This assigns ‘executor.matrix("binary", spikes)‘ to ‘actual ‘ in host memory. Here actual means the value named ’actual’ in this block. The important operands are executor: the value named ’executor’ in this block; matrix: the value named ’matrix’ in this block; binary: the value named ’binary’ in this block. Context: the function test_binary_projection_executes_tensor_and_vector_programs, which handles test binary projection executes tensor and vector programs.

Line 177: `expected_accumulator = spikes.astype(np.int64) @ weight.astype(np.int64)` This assigns ‘spikes.astype(np.i @ weight.astype(np.int64)‘ to ‘expected_accumulator ‘ in host memory. Here expected_accumulator means a widened running sum that prevents intermediate overflow. The important operands are spikes: a binary event produced when a neuron crosses its threshold; astype: the value named ’astype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_binary_projection_executes_tensor_and_vector_programs, which handles test binary projection executes tensor and vector programs.

Line 178: `expected = requantize_projection_reference(expected_accumulator, parameters)` This assigns ‘requantize_projection_reference(expected_accumulator, parameters)‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are requantize_projection_reference: the conversion of a wide accumulated result back into a bounded activation code; expected_accumulator: a widened running sum that prevents intermediate overflow; parameters: the value named ’parameters’ in this block. Context: the function test_binary_projection_executes_tensor_and_vector_programs, which handles test binary projection executes tensor and vector programs.

Line 179: `np.testing.assert_array_equal(actual, expected)` This calls ‘np.testing.assert_array_equal‘ with ‘actual, expected‘ inside the function test_binary_projection_executes_tensor_and_vector_programs, which handles test binary projection executes tensor and vector programs. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 181: `def test_asymmetric_dense_projection_executes_exact_zero_point_math(self) -> None:` This starts function test_asymmetric_dense_projection_executes_exact_zero_point_math. Its indented body performs test asymmetric dense projection executes exact zero point math as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 182: `rng = np.random.default_rng(12)` This assigns ‘np.random.default_rng(12)‘ to ‘rng ‘ in host memory. Here rng means the value named ’rng’ in this block. The important operands are random: the value named ’random’ in this block; default_rng: the value named ’default rng’ in this block. Context: the function test_asymmetric_dense_projection_executes which handles test asymmetric dense projection executes exact zero point math.

Line 183: `activations = rng.integers(0, 256, size=(9, 13), dtype=np.uint8)` This assigns ‘rng.integers(0, 256, size=(9, 13), dtype=np.uint8)‘ to ‘activations ‘ in host memory. Here activations means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math.

Line 184: `weight = rng.integers(-127, 128, size=(13, 6), dtype=np.int16).astype(np.int8)` This assigns ‘rng.integers(-127, 128, size=(13, 6), dtype=np.int16).astype(np.int8)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are rng: the value named ’rng’ in this block; integers: the value named ’integers’ in this block; size: the value named ’size’ in this block. Context: the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math.

Line 185: `parameters = self.parameters("dense", weight, "COMPUTE_DENSE_OUTER")` This assigns ‘self.parameters("dense weight, "COMPUTE_DENSE_OUTER")‘ to ‘parameters ‘ in host memory. Here parameters means the value named ’parameters’ in this block. The important operands are parameters: the value named ’parameters’ in this block; dense: the value named ’dense’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math.

Line 186: `executor = SpikingformerProjectionExecutor(` This statement continues through source line 187; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘SpikingformerProjectionExecutor(OneProjectionArchive(parameters), CombinedProgramEmulator())‘ to ‘executor ‘ in host memory. Here

executor means the value named 'executor' in this block. The important operands are SpikingformerProjectionExecutor: the value named 'spikingformer projection executor' in this block; OneProjectionArchive: the value named 'one projection archive' in this block; parameters: the value named 'parameters' in this block. Context: the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math.

Line 187: `OneProjectionArchive(parameters), CombinedProgramEmulator())` This is continuation line 2 of the 2-line logical statement that begins at source line 186. Its exact fragment is 'OneProjectionArchive(parameters), CombinedProgramEmulator())'. The important operands are OneProjectionArchive: the value named 'one projection archive' in this block; parameters: the value named 'parameters' in this block; CombinedProgramEmulator: the value named 'combined program emulator' in this block. Read together, the complete operation is: This assigns 'SpikingformerProjectionExecutor(OneProjectionArchive(parameters), CombinedProgramEmulator())' to 'executor ' in host memory. Here executor means the value named 'executor' in this block. The important operands are SpikingformerProjectionExecutor: the value named 'spikingformer projection executor' in this block; OneProjectionArchive: the value named 'one projection archive' in this block; parameters: the value named 'parameters' in this block. Context: the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math.

Line 188: `actual = executor.matrix("dense", activations, activation_zero_point=123)` This assigns 'executor.matrix("dense", activations, activation_zero_point=123)' to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are executor: the value named 'executor' in this block; matrix: the value named 'matrix' in this block; dense: the value named 'dense' in this block. Context: the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math.

Line 189: `expected_accumulator = (` This statement continues through source line 190; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(activations.astype(np.int64) - 123) @ weight.astype(np.int64)' to 'expected_accumulator ' in host memory. Here expected_accumulator means a widened running sum that prevents intermediate overflow. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Context: the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math.

Line 190: `activations.astype(np.int64) - 123) @ weight.astype(np.int64)` This is continuation line 2 of the 2-line logical statement that begins at source line 189. Its exact fragment is 'activations.astype(np.int64) - 123) @ weight.astype(np.int64)'. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns '(activations.astype(np.int64) - 123) @ weight.astype(np.int64)' to 'expected_accumulator ' in host memory. Here expected_accumulator means a widened running sum that prevents intermediate overflow. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Context: the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math.

Line 191: `expected = requantize_projection_reference(expected_accumulator, parameters)` This assigns 'requantize_projection_reference(expected_accumulator, parameters)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are requantize_projection_reference: the conversion of a wide accumulated result back into a bounded activation code; expected_accumulator: a widened running sum that prevents intermediate overflow; parameters: the value named 'parameters' in this block. Context: the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math.

Line 192: `np.testing.assert_array_equal(actual, expected)` This calls 'np.testing.assert_array_equal' with 'actual, expected' inside the function test_asymmetric_dense_projection_executes_exact_zero_point_math, which handles test asymmetric dense projection executes exact zero point math. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 194: `def test_chunked_lif_scheduler_preserves_every_logical_membrane(self) -> None:` This starts function test_chunked_lif_scheduler_preserves_every_logical_membrane. Its indented body performs test chunked lif scheduler preserves every logical membrane as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 195: `rng = np.random.default_rng(13)` This assigns 'np.random.default_rng(13)' to 'rng ' in host memory. Here rng means the value named 'rng' in this block. The important operands are random: the value named 'random' in this block; default_rng: the value named 'default rng' in this block. Context: the function test_chunked_lif_scheduler_preserves_every which handles test chunked lif scheduler preserves every logical membrane.

Line 196: `values = rng.integers(-5000, 5001, size=(4, 3, 5, 7), dtype=np.int64)` This assigns 'rng.integers(-5000, 5001, size=(4, 3, 5, 7), dtype=np.int64)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_chunked_lif_scheduler_preserves_every_logical_memb which handles test chunked lif scheduler preserves every logical membrane.

Line 197: `expected = lif_sequence_reference(` This statement continues through source line 198; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'lif_sequence_reference(values, threshold_q10=512, leak_shift=1, reset_q10=0)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are lif_sequence_reference: the value named 'lif sequence reference' in this block; values: the value named 'values' in this block; threshold_q10: the membrane level at which a neuron emits a spike. Context: the function test_chunked_lif_scheduler_preserves_every_logical_membrane, which handles test chunked lif scheduler preserves every logical membrane.

Line 198: `values, threshold_q10=512, leak_shift=1, reset_q10=0)` This is continuation line 2 of the 2-line logical statement that begins at source line 197. Its exact fragment is 'values, threshold_q10=512, leak_shift=1, reset_q10=0)'. The important operands are values: the value named 'values' in this block; threshold_q10: the membrane level at which a neuron emits a spike; leak_shift: the retained fraction of prior membrane state. Read together, the complete operation is: This assigns 'lif_sequence_reference(values, threshold_q10=512, leak_shift=1, reset_q10=0)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are lif_sequence_reference: the value named 'lif sequence reference' in this block; values: the value named 'values' in this block; threshold_q10: the membrane level at which a neuron emits a spike. Context: the function test_chunked_lif_scheduler_preserves_every_logical_membrane, which handles test chunked lif scheduler preserves every logical membrane.

Line 199: `actual = lif_sequence_on_accelerator(` This statement continues through source line 201; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'lif_sequence_on_accelerator(LifProgramEmulator(), values, threshold_q10=512, leak_shift=1, reset_q10=0)' to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are lif_sequence_on_accelerator: the value named 'lif sequence on accelerator' in this block; LifProgramEmulator: the value named 'lif program emulator' in this block; values: the value named 'values' in this block. Context: the function test_chunked_lif_scheduler_preserves_every_logical_membrane which handles test chunked lif scheduler preserves every logical membrane.

Line 200: `LifProgramEmulator(), values,` This is continuation line 2 of the 3-line logical statement that begins at source line 199. Its exact fragment is 'LifProgramEmulator(), values,'. The important operands are LifProgramEmulator: the value named 'lif program emulator' in this block; values: the value named 'values' in this block. Read together, the complete operation is: This assigns 'lif_sequence_on_accelerator(LifProgramEmulator(), values, threshold_q10=512, leak_shift=1, reset_q10=0)' to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are lif_sequence_on_accelerator: the value named 'lif sequence on accelerator' in this block; LifProgramEmulator: the value named 'lif program emulator' in this block; values: the value named 'values' in this block. Context: the function test_chunked_lif_scheduler_preserves_every_logical_membrane, which handles test chunked lif scheduler preserves every logical membrane.

Line 201: `threshold_q10=512, leak_shift=1, reset_q10=0)` This is continuation line 3 of the 3-line logical statement that begins at source line 199. Its exact fragment is 'threshold_q10=512, leak_shift=1, reset_q10=0)'. The important operands are threshold_q10: the membrane level at which a neuron emits a spike; leak_shift: the retained

fraction of prior membrane state; reset_q10: the value named 'reset q10' in this block. Read together, the complete operation is: This assigns 'lif_sequence_on_accelerator(LifProgramEmulator(), values, threshold_q10=512, leak_shift=1, reset_q10=0)' to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are lif_sequence_on_accelerator: the value named 'lif sequence on accelerator' in this block; LifProgramEmulator: the value named 'lif program emulator' in this block; values: the value named 'values' in this block. Context: the function test_chunked_lif_scheduler_preserves_every_logical_membrane, which handles test chunked lif scheduler preserves every logical membrane.

Line 202: `np.testing.assert_array_equal(actual, expected)` This calls 'np.testing.assert_array_equal' with 'actual, expected' inside the function test_chunked_lif_scheduler_preserves_every_logical_membrane, which handles test chunked lif scheduler preserves every logical membrane. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 204: `def test_traced_lif_scheduler_returns_exact_pre_reset_membrane(self) -> None:` This starts function test_traced_lif_scheduler_returns_exact_pre_reset_membrane. Its indented body performs test traced lif scheduler returns exact pre reset membrane as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 205: `rng = np.random.default_rng(1301)` This assigns 'np.random.default_rng(1301)' to 'rng ' in host memory. Here rng means the value named 'rng' in this block. The important operands are random: the value named 'random' in this block; default_rng: the value named 'default rng' in this block. Context: the function test_traced_lif_scheduler_returns_exa which handles test traced lif scheduler returns exact pre reset membrane.

Line 206: `values = rng.integers(-9000, 9001, size=(5, 2, 7, 5), dtype=np.int64)` This assigns 'rng.integers(-9000, 9001, size=(5, 2, 7, 5), dtype=np.int64)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_traced_lif_scheduler_returns_exact_pre_reset_membr which handles test traced lif scheduler returns exact pre reset membrane.

Line 207: `expected_spikes, expected_membrane = lif_sequence_with_trace_reference(` This statement continues through source line 208; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'lif_sequence_with_trace_reference(values, threshold_q10=700, leak_shift=1, reset_q10=-17)' to 'expected_spikes, expected_membrane ' in host memory. Here expected_spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_reference: the value named 'lif sequence with trace reference' in this block; values: the value named 'values' in this block; threshold_q10: the membrane level at which a neuron emits a spike. Context: the function test_traced_lif_scheduler_returns_exact_pre_reset_membrane, which handles test traced lif scheduler returns exact pre reset membrane.

Line 208: `values, threshold_q10=700, leak_shift=1, reset_q10=-17)` This is continuation line 2 of the 2-line logical statement that begins at source line 207. Its exact fragment is 'values, threshold_q10=700, leak_shift=1, reset_q10=-17)'. The important operands are values: the value named 'values' in this block; threshold_q10: the membrane level at which a neuron emits a spike; leak_shift: the retained fraction of prior membrane state. Read together, the complete operation is: This assigns 'lif_sequence_with_trace_reference(values, threshold_q10=700, leak_shift=1, reset_q10=-17)' to 'expected_spikes, expected_membrane ' in host memory. Here expected_spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_reference: the value named 'lif sequence with trace reference' in this block; values: the value named 'values' in this block; threshold_q10: the membrane level at which a neuron emits a spike. Context: the function test_traced_lif_scheduler_returns_exact_pre_reset_memb which handles test traced lif scheduler returns exact pre reset membrane.

Line 209: `actual_spikes, actual_membrane = lif_sequence_with_trace_on_accelerator(` This statement continues through source line 211; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'lif_sequence_with_trace_on_accelerator(LifProgramEmulator(), values, threshold_q10=700, leak_shift=1, reset_q10=-17)' to 'actual_spikes, actual_membrane ' in host memory. Here actual_spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the

value named 'lif sequence with trace on accelerator' in this block; LifProgramEmulator: the value named 'lif program emulator' in this block; values: the value named 'values' in this block. Context: the function test_traced_lif_scheduler_returns_exact_p which handles test traced lif scheduler returns exact pre reset membrane.

Line 210: `LifProgramEmulator(), values,` This is continuation line 2 of the 3-line logical statement that begins at source line 209. Its exact fragment is 'LifProgramEmulator(), values,'. The important operands are LifProgramEmulator: the value named 'lif program emulator' in this block; values: the value named 'values' in this block. Read together, the complete operation is: This assigns 'lif_sequence_with_trace_on_accelerator(LifProgramEmulator(), values, threshold_q10=700, leak_shift=1, reset_q10=-17)' to 'actual_spikes, actual_membrane ' in host memory. Here actual_spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named 'lif sequence with trace on accelerator' in this block; LifProgramEmulator: the value named 'lif program emulator' in this block; values: the value named 'values' in this block. Context: the function test_traced_lif_scheduler_returns_exact_pre_reset_membrane, which handles test traced lif scheduler returns exact pre reset membrane.

Line 211: `threshold_q10=700, leak_shift=1, reset_q10=-17)` This is continuation line 3 of the 3-line logical statement that begins at source line 209. Its exact fragment is 'threshold_q10=700, leak_shift=1, reset_q10=-17)'. The important operands are threshold_q10: the membrane level at which a neuron emits a spike; leak_shift: the retained fraction of prior membrane state; reset_q10: the value named 'reset q10' in this block. Read together, the complete operation is: This assigns 'lif_sequence_with_trace_on_accelerator(LifProgramEmulator(), values, threshold_q10=700, leak_shift=1, reset_q10=-17)' to 'actual_spikes, actual_membrane ' in host memory. Here actual_spikes means a binary event produced when a neuron crosses its threshold. The important operands are lif_sequence_with_trace_on_accelerator: the value named 'lif sequence with trace on accelerator' in this block; LifProgramEmulator: the value named 'lif program emulator' in this block; values: the value named 'values' in this block. Context: the function test_traced_lif_scheduler_returns_exa which handles test traced lif scheduler returns exact pre reset membrane.

Line 212: `np.testing.assert_array_equal(actual_spikes, expected_spikes)` This calls 'np.testing.assert_array_equal' with 'actual_spikes, expected_spikes' inside the function test_traced_lif_scheduler_returns_exact_pre_reset_membrane, which handles test traced lif scheduler returns exact pre reset membrane. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 213: `np.testing.assert_array_equal(actual_membrane, expected_membrane)` This calls 'np.testing.assert_array_equ with 'actual_membrane, expected_membrane' inside the function test_traced_lif_scheduler_returns_exact_pre_reset_membrane which handles test traced lif scheduler returns exact pre reset membrane. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 214: `self.assertTrue(np.any(actual_spikes))` This calls 'self.assertTrue' with 'np.any(actual_spikes)' inside the function test_traced_lif_scheduler_returns_exact_pre_reset_membrane, which handles test traced lif scheduler returns exact pre reset membrane. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 215: `self.assertTrue(np.any(actual_membrane[actual_spikes.astype(bool)] >= 700))` This calls 'self.assertTrue' with 'np.any(actual_membrane[actual_spikes.astype(bool)] >= 700)' inside the function test_traced_lif_scheduler_returns_exact which handles test traced lif scheduler returns exact pre reset membrane. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 217: `def test_real_block0_archive_executes_exact_exported_integer_equation(self) -> None:` This starts function test_real_block0_archive_executes_exact_exported_integer_equation. Its indented body performs test real block0 archive executes exact exported integer equation as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 218: `manifest = Path(__file__).resolve().parents[1] / (` This statement continues through source line 220; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'Path(___file___).resolve().p / ("outputs/spikingformer_hardware_parameters/" "spikingformer_neuraedge_weights.json")' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are Path: the value

named 'path' in this block; ___file___: the value named 'file' in this block; resolve: the value named 'resolve' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 219: `"outputs/spikingformer_hardware_parameters/"` This is continuation line 2 of the 3-line logical statement that begins at source line 218. Its exact fragment is '"outputs/spikingformer_hardware_parameters/"'. The important operands are outputs: the value named 'outputs' in this block; spikingformer_hardware_parameters: the value named 'spikingformer hardware parameters' in this block. Read together, the complete operation is: This assigns 'Path(___file___).resolve().parents[1] / ("outputs/spikingformer_hardware_parameters/" "spikingformer_neuraedge_weights.json" to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are Path: the value named 'path' in this block; ___file___: the value named 'file' in this block; resolve: the value named 'resolve' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 220: `"spikingformer_neuraedge_weights.json")` This is continuation line 3 of the 3-line logical statement that begins at source line 218. Its exact fragment is '"spikingformer_neuraedge_weights.json")'. The important operands are spikingformer_neuraedge_weights: a learned coefficient multiplied by an activation; json: the value named 'json' in this block. Read together, the complete operation is: This assigns 'Path(___file___).resolve().parents[1] / ("outputs/spikingformer_hardware_parameters/" "spikingformer_neuraedge_weights.json")' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are Path: the value named 'path' in this block; ___file___: the value named 'file' in this block; resolve: the value named 'resolve' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 221: `if not manifest.exists():` This executes the following indented host-code branch only when 'not manifest.exists()' is true. The important operands are not: the value named 'not' in this block; manifest: the value named 'manifest' in this block; exists: the value named 'exists' in this block. Context: the function test_real_block0_archive_executes_exact_e which handles test real block0 archive executes exact exported integer equation.

Line 222: `self.skipTest("full trained parameter archive is not present")` This calls 'self.skipTest' with '"full trained parameter archive is not present"' inside the function test_real_block0_archive_executes_exact_exported_integer_eq which handles test real block0 archive executes exact exported integer equation. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 223: `archive = SpikingformerParameterArchive(manifest)` This assigns 'SpikingformerParameterArchive(manifest)' to 'archive ' in host memory. Here archive means the value named 'archive' in this block. The important operands are SpikingformerParameterArchive: the value named 'spikingformer parameter archive' in this block; manifest: the value named 'manifest' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 224: `parameters = archive.projection("patch_embed.block0")` This assigns 'archive.projection("patch_embed.block0 to 'parameters ' in host memory. Here parameters means the value named 'parameters' in this block. The important operands are archive: the value named 'archive' in this block; projection: the value named 'projection' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function test_real_block0_archive_executes_exact_expo which handles test real block0 archive executes exact exported integer equation.

Line 225: `rng = np.random.default_rng(14)` This assigns 'np.random.default_rng(14)' to 'rng ' in host memory. Here rng means the value named 'rng' in this block. The important operands are random: the value named 'random' in this block; default_rng: the value named 'default rng' in this block. Context: the function test_real_block0_archive_executes_exact_e which handles test real block0 archive executes exact exported integer equation.

Line 226: `activations = rng.integers(` This statement continues through source line 227; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'rng.integers(0, 256, size=(3, parameters.weight_k_by_n.shape[0]), dtype=np.uint8)' to 'activations ' in host memory. Here activations means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are rng: the value

named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 227: `0, 256, size=(3, parameters.weight_k_by_n.shape[0]), dtype=np.uint8)` This is continuation line 2 of the 2-line logical statement that begins at source line 226. Its exact fragment is '0, 256, size=(3, parameters.weight_k_by_n.shape[0]), dtype=np.uint8)'. The important operands are size: the value named 'size' in this block; parameters: the value named 'parameters' in this block; weight_k_by_n: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'rng.integers(0, 256, size=(3, parameters.weight_k_by_n.shape[0]), dtype=np.uint8)' to 'activations ' in host memory. Here activations means an input feature value consumed by a neuron or multiply-accumulate operation. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 228: `executor = SpikingformerProjectionExecutor(archive, CombinedProgramEmulator())` This assigns 'SpikingformerProjectionExecutor(archive, CombinedProgramEmulator())' to 'executor ' in host memory. Here executor means the value named 'executor' in this block. The important operands are SpikingformerProjectionExecutor: the value named 'spikingformer projection executor' in this block; archive: the value named 'archive' in this block; CombinedProgramEmulator: the value named 'combined program emulator' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 229: `actual = executor.matrix(` This statement continues through source line 230; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'executor.matrix("patch_embed.block0", activations, activation_zero_point=123)' to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are executor: the value named 'executor' in this block; matrix: the value named 'matrix' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 230: `"patch_embed.block0", activations, activation_zero_point=123)` This is continuation line 2 of the 2-line logical statement that begins at source line 229. Its exact fragment is '"patch_embed.block0", activations, activation_zero_point=123)'. The important operands are patch_embed: the value named 'patch embed' in this block; block0: the value named 'block0' in this block; activations: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns 'executor.matrix("patch_embed.block0", activations, activation_zero_point=123)' to 'actual ' in host memory. Here actual means the value named 'actual' in this block. The important operands are executor: the value named 'executor' in this block; matrix: the value named 'matrix' in this block; patch_embed: the value named 'patch embed' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 231: `accumulators = (` This statement continues through source line 233; the explanation uses the complete expression rather than only the first visual fragment. This assigns '(activations.astype(np.int64) - 123) @ parameters.weight_k_by_n.astype(np.int64)' to 'accumulators ' in host memory. Here accumulators means a widened running sum that prevents intermediate overflow. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 232: `activations.astype(np.int64) - 123` This is continuation line 2 of the 3-line logical statement that begins at source line 231. Its exact fragment is 'activations.astype(np.int64) - 123'. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Read together, the complete operation is: This assigns '(activations.astype(np.int64) - 123) @ parameters.weight_k_by_n.astype(np.int64)' to 'accumulators ' in host memory. Here accumulators means a widened running sum that prevents intermediate overflow. The important operands are activations:

an input feature value consumed by a neuron or multiply-accumulate operation; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Context: the function test_real_block0_archive_executes_exact_exported_inte which handles test real block0 archive executes exact exported integer equation.

Line 233: `) @ parameters.weight_k_by_n.astype(np.int64)` This is continuation line 3 of the 3-line logical statement that begins at source line 231. Its exact fragment is ‘) @ parameters.weight_k_by_n.astype(np.int64)‘. The important operands are parameters: the value named 'parameters' in this block; weight_k_by_n: a learned coefficient multiplied by an activation; astype: the value named 'astype' in this block. Read together, the complete operation is: This assigns ‘(activations.astype(np.int64) - 123) @ parameters.weight_k_by_n.astype(np.int64)‘ to ‘accumulators ‘ in host memory. Here accumulators means a widened running sum that prevents intermediate overflow. The important operands are activations: an input feature value consumed by a neuron or multiply-accumulate operation; astype: the value named 'astype' in this block; int64: the value named 'int64' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 234: `expected = requantize_projection_reference(accumulators, parameters)` This assigns ‘requantize_projection parameters)‘ to ‘expected ‘ in host memory. Here expected means the value named 'expected' in this block. The important operands are requantize_projection_reference: the conversion of a wide accumulated result back into a bounded activation code; accumulators: a widened running sum that prevents intermediate overflow; parameters: the value named 'parameters' in this block. Context: the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation.

Line 235: `np.testing.assert_array_equal(actual, expected)` This calls ‘np.testing.assert_array_equal‘ with ‘actual, expected‘ inside the function test_real_block0_archive_executes_exact_exported_integer_equation, which handles test real block0 archive executes exact exported integer equation. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 237: `def test_vector_residual_and_binary_max_pool_cover_edge_lanes(self) -> None:` This starts function test_vector_residual_and_binary_max_pool_cover_edge_lanes. Its indented body performs test vector residual and binary max pool cover edge lanes as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 238: `left = np.asarray([-(1 << 31), -3, 0, 7, (1 << 31) - 1, 4, 5, 6, 7])` This assigns ‘np.asarray([-(1 << 31), -3, 0, 7, (1 << 31) - 1, 4, 5, 6, 7])‘ to ‘left ‘ in host memory. Here left means the value named 'left' in this block. The important operands are asarray: the value named 'asarray' in this block. Context: the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes.

Line 239: `right = np.asarray([-1, 8, 0, -2, 1, 3, 2, 1, 0])` This assigns ‘np.asarray([-1, 8, 0, -2, 1, 3, 2, 1, 0])‘ to ‘right ‘ in host memory. Here right means the value named 'right' in this block. The important operands are asarray: the value named 'asarray' in this block. Context: the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes.

Line 240: `added = vector_binary_on_accelerator(` This statement continues through source line 241; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘vector_binary_on_accelerator(VectorProgramEmulator(), VectorOperation.ADD_SAT, left, right)‘ to ‘added ‘ in host memory. Here added means the value named 'added' in this block. The important operands are vector_binary_on_accelerator: the value named 'vector binary on accelerator' in this block; VectorProgramEmulator: the value named 'vector program emulator' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function test_vector_residual_and_binary_max_ which handles test vector residual and binary max pool cover edge lanes.

Line 241: `VectorProgramEmulator(), VectorOperation.ADD_SAT, left, right)` This is continuation line 2 of the 2-line logical statement that begins at source line 240. Its exact fragment is ‘VectorProgramEmulator(), VectorOperation.ADD_SAT, left, right)‘. The important operands are VectorProgramEmulator: the value named 'vector program emulator' in this block; VectorOperation: the value named 'vector operation' in this block; ADD_SAT: the value named

'add sat' in this block. Read together, the complete operation is: This assigns ‘vector_binary_on_accelerator(VectorProgramEmulator(), VectorOperation.ADD_SAT, left, right)‘ to ‘added ‘ in host memory. Here added means the value named 'added' in this block. The important operands are vector_binary_on_accelerator: the value named 'vector binary on accelerator' in this block; VectorProgramEmulator: the value named 'vector program emulator' in this block; VectorOperation: the value named 'vector operation' in this block. Context: the function test_vector_residual_and_binary_max_pool_c which handles test vector residual and binary max pool cover edge lanes.

Line 242: `np.testing.assert_array_equal(` This statement continues through source line 243; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.testing.assert_array_equal‘ with ‘ added, np.clip(left.astype(np.int64) + right, -(1 << 31), (1 << 31) - 1)‘ inside the function test_vector_residual_and_binary_max_pool_c which handles test vector residual and binary max pool cover edge lanes. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 243: `added, np.clip(left.astype(np.int64) + right, -(1 << 31), (1 << 31) - 1))` This is continuation line 2 of the 2-line logical statement that begins at source line 242. Its exact fragment is ‘added, np.clip(left.astype(np.int64) + right, -(1 << 31), (1 << 31) - 1))‘. The important operands are added: the value named 'added' in this block; clip: the value named 'clip' in this block; left: the value named 'left' in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ added, np.clip(left.astype(np.int64) + right, -(1 << 31), (1 << 31) - 1)‘ inside the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 245: `rng = np.random.default_rng(15)` This assigns ‘np.random.default_rng(15)‘ to ‘rng ‘ in host memory. Here rng means the value named 'rng' in this block. The important operands are random: the value named 'random' in this block; default_rng: the value named 'default rng' in this block. Context: the function test_vector_residual_and_binary_max_pool which handles test vector residual and binary max pool cover edge lanes.

Line 246: `spikes = rng.integers(0, 2, size=(2, 3, 5, 7), dtype=np.uint8)` This assigns ‘rng.integers(0, 2, size=(2, 3, 5, 7), dtype=np.uint8)‘ to ‘spikes ‘ in host memory. Here spikes means a binary event produced when a neuron crosses its threshold. The important operands are rng: the value named 'rng' in this block; integers: the value named 'integers' in this block; size: the value named 'size' in this block. Context: the function test_vector_residual_and_binary_max_pool_co which handles test vector residual and binary max pool cover edge lanes.

Line 247: `actual = binary_max_pool2d_on_accelerator(VectorProgramEmulator(), spikes)` This assigns ‘binary_max_pool2d_on_accelerator(VectorProgramEmulator(), spikes)‘ to ‘actual ‘ in host memory. Here actual means the value named 'actual' in this block. The important operands are binary_max_pool2d_on_accelerator: the value named 'binary max pool2d on accelerator' in this block; VectorProgramEmulator: the value named 'vector program emulator' in this block; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes.

Line 248: `padded = np.pad(spikes, ((0, 0), (0, 0), (1, 1), (1, 1)))` This assigns ‘np.pad(spikes, ((0, 0), (0, 0), (1, 1), (1, 1)))‘ to ‘padded ‘ in host memory. Here padded means the value named 'padded' in this block. The important operands are pad: the value named 'pad' in this block; spikes: a binary event produced when a neuron crosses its threshold. Context: the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes.

Line 249: `candidates = [` This statement continues through source line 252; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘[padded[:, :, y:y + 5:2, x:x + 7:2] for y in range(3) for x in range(3)]‘ to ‘candidates ‘ in host memory. Here candidates means the value named 'candidates' in this block. The important operands are padded: the value named 'padded' in this block; y: the value named 'y' in this block; x: the value named 'x' in this block. Context: the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes.

Line 250: `padded[:, :, y:y + 5:2, x:x + 7:2]` This is continuation line 2 of the 4-line logical statement that begins at source line 249. Its exact fragment is 'padded[:, :, y:y + 5:2, x:x + 7:2]'. The important operands are padded: the value named 'padded' in this block; y: the value named 'y' in this block; x: the value named 'x' in this block. Read together, the complete operation is: This assigns '[padded[:, :, y:y + 5:2, x:x + 7:2] for y in range(3) for x in range(3)]' to 'candidates ' in host memory. Here candidates means the value named 'candidates' in this block. The important operands are padded: the value named 'padded' in this block; y: the value named 'y' in this block; x: the value named 'x' in this block. Context: the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes.

Line 251: `for y in range(3) for x in range(3)` This is continuation line 3 of the 4-line logical statement that begins at source line 249. Its exact fragment is 'for y in range(3) for x in range(3)'. The important operands are for: the value named 'for' in this block; y: the value named 'y' in this block; in: the value named 'in' in this block. Read together, the complete operation is: This assigns '[padded[:, :, y:y + 5:2, x:x + 7:2] for y in range(3) for x in range(3)]' to 'candidates ' in host memory. Here candidates means the value named 'candidates' in this block. The important operands are padded: the value named 'padded' in this block; y: the value named 'y' in this block; x: the value named 'x' in this block. Context: the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes.

Line 252: `]` This is continuation line 4 of the 4-line logical statement that begins at source line 249. Its exact fragment is ']'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '[padded[:, :, y:y + 5:2, x:x + 7:2] for y in range(3) for x in range(3)]' to 'candidates ' in host memory. Here candidates means the value named 'candidates' in this block. The important operands are padded: the value named 'padded' in this block; y: the value named 'y' in this block; x: the value named 'x' in this block. Context: the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes.

Line 253: `expected = np.maximum.reduce(candidates)` This assigns 'np.maximum.reduce(candidates)' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are maximum: the value named 'maximum' in this block; reduce: the value named 'reduce' in this block; candidates: the value named 'candidates' in this block. Context: the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes.

Line 254: `np.testing.assert_array_equal(actual, expected)` This calls 'np.testing.assert_array_equal' with 'actual, expected' inside the function test_vector_residual_and_binary_max_pool_cover_edge_lanes, which handles test vector residual and binary max pool cover edge lanes. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 256: `def test_global_pooling_and_head_division_execute_in_vector_fabric(self) -> None:` This starts function test_global_pooling_and_head_division_execute_in_vector_fabric. Its indented body performs test global pooling and head division execute in vector fabric as part of the file that checks spikingformer accelerator behavior independently with Python assertions; it runs only when called.

Line 257: `values = np.arange(-120, 120, dtype=np.int64).reshape(3, 2, 5, 4, 2)` This assigns 'np.arange(-120, 120, dtype=np.int64).reshape(3, 2, 5, 4, 2)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are arange: the value named 'arange' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function test_global_pooling_and_head_division_execute_in which handles test global pooling and head division execute in vector fabric.

Line 258: `pooled = global_spatiotemporal_sum_on_accelerator(` This statement continues through source line 259; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'global_spatiotemporal_su VectorProgramEmulator(), values)' to 'pooled ' in host memory. Here pooled means the value named 'pooled' in this block. The important operands are global_spatiotemporal_sum_on_accelerator: the value named 'global spatiotemporal sum on accelerator' in this block; VectorProgramEmulator: the value named 'vector program emulator' in this block; values: the value named 'values' in this block. Context: the function test_global_pooling_and_head_division_execute_in_vector_ which handles test global pooling and head division execute in vector fabric.

Line 259: `VectorProgramEmulator(), values)` This is continuation line 2 of the 2-line logical statement that begins at source line 258. Its exact fragment is 'VectorProgramEmulator(), values)'. The important operands are VectorProgramEmulator: the value named 'vector program emulator' in this block; values: the value named 'values' in this block. Read together, the complete operation is: This assigns 'global_spatiotemporal_sum_on_accelerator(VectorProgramEmulator(), values)' to 'pooled ' in host memory. Here pooled means the value named 'pooled' in this block. The important operands are global_spatiotemporal_sum_on_accelerator: the value named 'global spatiotemporal sum on accelerator' in this block; VectorProgramEmulator: the value named 'vector program emulator' in this block; values: the value named 'values' in this block. Context: the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric.

Line 260: `np.testing.assert_array_equal(pooled, values.sum(axis=(0, 3, 4)))` This calls 'np.testing.assert_array_equ with 'pooled, values.sum(axis=(0, 3, 4))' inside the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 261: `divided = power_of_two_requantize_on_accelerator(` This statement continues through source line 267; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'power_of_two_requantize_ VectorProgramEmulator(), pooled, division_shift=5, clamp_minimum=-17, clamp_maximum=19,)' to 'divided ' in host memory. Here divided means the value named 'divided' in this block. The important operands are power_of_two_requantize_o the conversion of a wide accumulated result back into a bounded activation code; VectorProgramEmulator: the value named 'vector program emulator' in this block; pooled: the value named 'pooled' in this block. Context: the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric.

Line 262: `VectorProgramEmulator(),` This is continuation line 2 of the 7-line logical statement that begins at source line 261. Its exact fragment is 'VectorProgramEmulator(),'. The important operands are VectorProgramEmulator: the value named 'vector program emulator' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(VectorProgramEmulator(), pooled, division_shift=5, clamp_minimum=-17, clamp_maximum=19,)' to 'divided ' in host memory. Here divided means the value named 'divided' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; VectorProgramEmulator: the value named 'vector program emulator' in this block; pooled: the value named 'pooled' in this block. Context: the function test_global_pooling_and_head_division_execute_in_vector which handles test global pooling and head division execute in vector fabric.

Line 263: `pooled,` This is continuation line 3 of the 7-line logical statement that begins at source line 261. Its exact fragment is 'pooled,'. The important operands are pooled: the value named 'pooled' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(VectorProgramEmulator(), pooled, division_shift=5, clamp_minimum=-17, clamp_maximum=19,)' to 'divided ' in host memory. Here divided means the value named 'divided' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; VectorProgramEmulator: the value named 'vector program emulator' in this block; pooled: the value named 'pooled' in this block. Context: the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric.

Line 264: `division_shift=5,` This is continuation line 4 of the 7-line logical statement that begins at source line 261. Its exact fragment is 'division_shift=5,'. The important operands are division_shift: the value named 'division shift' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(VectorProgramEmulator(), pooled, division_shift=5, clamp_minimum=-17, clamp_maximum=19,)' to 'divided ' in host memory. Here divided means the value named 'divided' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; VectorProgramEmulator: the value named 'vector program emulator' in this block; pooled: the value named 'pooled' in this block. Context: the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric.

Line 265: `clamp_minimum=-17,` This is continuation line 5 of the 7-line logical statement that begins at source line 261. Its exact fragment is 'clamp_minimum=-17,'. The important operands are clamp_minimum: the value named 'clamp

minimum' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(VectorProgramEmulator(), pooled, division_shift=5, clamp_minimum=-17, clamp_maximum=19,)' to 'divided ' in host memory. Here divided means the value named 'divided' in this block. The important operands are power_of_two_requantize_o the conversion of a wide accumulated result back into a bounded activation code; VectorProgramEmulator: the value named 'vector program emulator' in this block; pooled: the value named 'pooled' in this block. Context: the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric.

Line 266: `clamp_maximum=19,` This is continuation line 6 of the 7-line logical statement that begins at source line 261. Its exact fragment is 'clamp_maximum=19,'. The important operands are clamp_maximum: the value named 'clamp maximum' in this block. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(VectorProgramEmulator(), pooled, division_shift=5, clamp_minimum=-17, clamp_maximum=19,)' to 'divided ' in host memory. Here divided means the value named 'divided' in this block. The important operands are power_of_two_requantize_o the conversion of a wide accumulated result back into a bounded activation code; VectorProgramEmulator: the value named 'vector program emulator' in this block; pooled: the value named 'pooled' in this block. Context: the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric.

Line 267: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 261. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'power_of_two_requantize_on_accelerator(VectorProgramEmulator(), pooled, division_shift=5, clamp_minimum=-17, clamp_maximum=19,)' to 'divided ' in host memory. Here divided means the value named 'divided' in this block. The important operands are power_of_two_requantize_on_accelerator: the conversion of a wide accumulated result back into a bounded activation code; VectorProgramEmulator: the value named 'vector program emulator' in this block; pooled: the value named 'pooled' in this block. Context: the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric.

Line 268: `np.testing.assert_array_equal(` This statement continues through source line 271; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_array_equal' with ' divided, np.clip(round_shift_signed(pooled, 5), -17, 19), ' inside the function test_global_pooling_and_head_division_execute_in_ which handles test global pooling and head division execute in vector fabric. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 269: `divided,` This is continuation line 2 of the 4-line logical statement that begins at source line 268. Its exact fragment is 'divided,'. The important operands are divided: the value named 'divided' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' divided, np.clip(round_shift_signed(pooled, 5), -17, 19), ' inside the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 270: `np.clip(round_shift_signed(pooled, 5), -17, 19),` This is continuation line 3 of the 4-line logical statement that begins at source line 268. Its exact fragment is 'np.clip(round_shift_signed(pooled, 5), -17, 19),'. The important operands are clip: the value named 'clip' in this block; round_shift_signed: the value named 'round shift signed' in this block; pooled: the value named 'pooled' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' divided, np.clip(round_shift_signed(pooled, 5), -17, 19), ' inside the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 271: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 268. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' divided, np.clip(round_shift_signed(pooled, 5), -17, 19), ' inside the function test_global_pooling_and_head_division_execute_in_vector_fabric, which handles test global pooling and head division execute in vector fabric. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

Line 274: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks spikingformer accelerator behavior independently with Python assertions.

Line 275: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks spikingformer accelerator behavior independently with Python assertions. The call advances the host-side job that checks spikingformer accelerator behavior independently with Python assertions.

71 tests/test_spikingformer_export.py

Role in the complete system: checks spikingformer export behavior independently with Python assertions.

71.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks spikingformer export behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: HardwareExportTests. Functions: test_aligned_writer_records_exact_payload_hash, test_tensor_w test_projection_export_matches_folded_float_equation. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

71.2 Complete source

```
from __future__ import annotations

import hashlib
import unittest

import numpy as np
import torch
import torch.nn as nn

from export_spikingformer_hardware import (
    ALIGNMENT,
    AlignedBinaryWriter,
    export_projection,
    quantize_weight_tensor,
)


class HardwareExportTests(unittest.TestCase):
    def test_aligned_writer_records_exact_payload_hash(self) -> None:
        writer = AlignedBinaryWriter()
        first = writer.append("first", np.arange(7, dtype=np.int8))
        second_value = np.arange(5, dtype="<i4")
        second = writer.append("second", second_value)
        self.assertEqual(first.offset, 0)
        self.assertEqual(second.offset % ALIGNMENT, 0)
        payload = second_value.tobytes()
        self.assertEqual(second.sha256, hashlib.sha256(payload).hexdigest())
        self.assertEqual(bytes(writer.data[second.offset:second.offset + second.length]), payload)

    def test_tensor_weight_quantization_uses_complete_int8_range(self) -> None:
        weight = torch.tensor([[-2.0, 0.0, 1.0, 2.0]])
        integer, scale = quantize_weight_tensor(weight)
        self.assertEqual(scale, 2.0 / 127.0)
        torch.testing.assert_close(integer, torch.tensor([[-127, 0, 64, 127]], dtype=torch.int8))

    def test_projection_export_matches_folded_float_equation(self) -> None:
        torch.manual_seed(19)
        convolution = nn.Conv1d(4, 3, kernel_size=1, bias=True).eval()
        batch_norm = nn.BatchNorm1d(3).eval()
        batch_norm.running_mean.copy_(torch.randn(3))
        batch_norm.running_var.copy_(torch.rand(3) + 0.5)
        writer = AlignedBinaryWriter()
        exported = export_projection(
            writer,
            name="probe",
            convolution=convolution,
            batch_norm=batch_norm,
            input_scale=1.0,
            fractional_bits=10,
            input_encoding="binary spike 0/1",
            compute_opcode="COMPUTE_BINARY",
        )
        self.assertEqual(exported.hardware_matrix_shape, (4, 3))
```

```
        self.assertEqual(len(exported.sections), 4)
        self.assertLess(exported.represented_factor_maximum_error, 0.001)
        for section in exported.sections:
            self.assertEqual(section.offset % ALIGNMENT, 0)


if __name__ == "__main__":
    unittest.main()
```

71.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import hashlib` This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 4: `import unittest` This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import numpy as np` This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `import torch` This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `import torch.nn as nn` This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `from export_spikingformer_hardware import (` This statement continues through source line 15; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `ALIGNMENT,` This is continuation line 2 of the 6-line logical statement that begins at source line 10. Its exact fragment is 'ALIGNMENT,'. The important operands are ALIGNMENT: the value named 'alignment' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `AlignedBinaryWriter,` This is continuation line 3 of the 6-line logical statement that begins at source line 10. Its exact fragment is 'AlignedBinaryWriter,'. The important operands are AlignedBinaryWriter: the value named 'aligned binary writer' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `export_projection,` This is continuation line 4 of the 6-line logical statement that begins at source line 10. Its exact fragment is 'export_projection,'. The important operands are export_projection: the value named 'export projection' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `quantize_weight_tensor,` This is continuation line 5 of the 6-line logical statement that begins at source line 10. Its exact fragment is ‘quantize_weight_tensor,‘. The important operands are quantize_weight_tensor: a learned coefficient multiplied by an activation. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 10. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer export behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `class HardwareExportTests(unittest.TestCase):` This defines class HardwareExportTests, which groups data and methods for hardware export tests. Instances exist in host memory and organize the flow that checks spikingformer export behavior independently with Python assertions.

Line 19: `def test_aligned_writer_records_exact_payload_hash(self) -> None:` This starts function test_aligned_writer Its indented body performs test aligned writer records exact payload hash as part of the file that checks spikingformer export behavior independently with Python assertions; it runs only when called.

Line 20: `writer = AlignedBinaryWriter()` This assigns ‘AlignedBinaryWriter()‘ to ‘writer ‘ in host memory. Here writer means the value named 'writer' in this block. The important operands are AlignedBinaryWriter: the value named 'aligned binary writer' in this block. Context: the function test_aligned_writer_records_exact_payload_hash, which handles test aligned writer records exact payload hash.

Line 21: `first = writer.append("first", np.arange(7, dtype=np.int8))` This assigns ‘writer.append("first", np.arange(7, dtype=np.int8))‘ to ‘first ‘ in host memory. Here first means the value named 'first' in this block. The important operands are writer: the value named 'writer' in this block; append: the value named 'append' in this block; first: the value named 'first' in this block. Context: the function test_aligned_writer_records_exact_payload_hash, which handles test aligned writer records exact payload hash.

Line 22: `second_value = np.arange(5, dtype="<i4")` This assigns ‘np.arange(5, dtype="<i4")‘ to ‘second_value ‘ in host memory. Here second_value means the value named 'second value' in this block. The important operands are arange: the value named 'arange' in this block; dtype: the value named 'dtype' in this block; i4: the value named 'i4' in this block. Context: the function test_aligned_writer_records_exact_payload_hash, which handles test aligned writer records exact payload hash.

Line 23: `second = writer.append("second", second_value)` This assigns ‘writer.append("second", second_value)‘ to ‘second ‘ in host memory. Here second means the value named 'second' in this block. The important operands are writer: the value named 'writer' in this block; append: the value named 'append' in this block; second: the value named 'second' in this block. Context: the function test_aligned_writer_records_exact_payload_hash, which handles test aligned writer records exact payload hash.

Line 24: `self.assertEqual(first.offset, 0)` This calls ‘self.assertEqual‘ with ‘first.offset, 0‘ inside the function test_aligned_writer_records_exact_payload_hash, which handles test aligned writer records exact payload hash. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 25: `self.assertEqual(second.offset % ALIGNMENT, 0)` This calls ‘self.assertEqual‘ with ‘second.offset % ALIGNMENT, 0‘ inside the function test_aligned_writer_records_exact_payload_hash, which handles test aligned writer records exact payload hash. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 26: `payload = second_value.tobytes()` This assigns ‘second_value.tobytes()‘ to ‘payload ‘ in host memory. Here payload means stream data belonging to the current command. The important operands are second_value: the value named ’second value’ in this block; tobytes: the value named ’tobytes’ in this block. Context: the function test_aligned_writer_records_exact_payload_hash, which handles test aligned writer records exact payload hash.

Line 27: `self.assertEqual(second.sha256, hashlib.sha256(payload).hexdigest())` This calls ‘self.assertEqual‘ with ‘second.sha256, hashlib.sha256(payload).hexdigest()‘ inside the function test_aligned_writer_records_exact_payload_hash, which handles test aligned writer records exact payload hash. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 28: `self.assertEqual(bytes(writer.data[second.offset:second.offset + second.length]), payload)` This calls ‘self.assertEqual‘ with ‘bytes(writer.data[second.offset:second.offset + second.length]), payload‘ inside the function test_aligned_writer_records_exact_payload_hash, which handles test aligned writer records exact payload hash. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 30: `def test_tensor_weight_quantization_uses_complete_int8_range(self) -> None:` This starts function test_tensor_weight_quantization_uses_complete_int8_range. Its indented body performs test tensor weight quantization uses complete int8 range as part of the file that checks spikingformer export behavior independently with Python assertions; it runs only when called.

Line 31: `weight = torch.tensor([[-2.0, 0.0, 1.0, 2.0]])` This assigns ‘torch.tensor([[-2.0, 0.0, 1.0, 2.0]])‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block. Context: the function test_tensor_weight_quantization_uses_complete_int8_range, which handles test tensor weight quantization uses complete int8 range.

Line 32: `integer, scale = quantize_weight_tensor(weight)` This assigns ‘quantize_weight_tensor(weight)‘ to ‘integer, scale ‘ in host memory. Here integer means the value named ’integer’ in this block. The important operands are quantize_weight_tensor: a learned coefficient multiplied by an activation; weight: a learned coefficient multiplied by an activation. Context: the function test_tensor_weight_quantization_uses_complete_int8_range, which handles test tensor weight quantization uses complete int8 range.

Line 33: `self.assertEqual(scale, 2.0 / 127.0)` This calls ‘self.assertEqual‘ with ‘scale, 2.0 / 127.0‘ inside the function test_tensor_weight_quantization_uses_complete_int8_range, which handles test tensor weight quantization uses complete int8 range. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 34: `torch.testing.assert_close(integer, torch.tensor([[-127, 0, 64, 127]], dtype=torch.int8))` This calls ‘torch.testing.assert_close‘ with ‘integer, torch.tensor([[-127, 0, 64, 127]], dtype=torch.int8)‘ inside the function test_tensor_weight_quantization_uses_complete_int8_range, which handles test tensor weight quantization uses complete int8 range. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 36: `def test_projection_export_matches_folded_float_equation(self) -> None:` This starts function test_projection_export_matches_folded_float_equation. Its indented body performs test projection export matches folded float equation as part of the file that checks spikingformer export behavior independently with Python assertions; it runs only when called.

Line 37: `torch.manual_seed(19)` This calls ‘torch.manual_seed‘ with ‘19‘ inside the function test_projection_export_matches_ which handles test projection export matches folded float equation. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 38: `convolution = nn.Conv1d(4, 3, kernel_size=1, bias=True).eval()` This assigns ‘nn.Conv1d(4, 3, kernel_size=1, bias=True).eval()‘ to ‘convolution ‘ in host memory. Here convolution means the value named ’convolution’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv1d: the value named ’conv1d’ in this block; kernel_size: the value named ’kernel size’ in this block. Context: the function test_projection_export_matches_folded_float which handles test projection export matches folded float equation.

Line 39: `batch_norm = nn.BatchNorm1d(3).eval()` This assigns ‘nn.BatchNorm1d(3).eval()‘ to ‘batch_norm ‘ in host memory. Here batch_norm means the value named ’batch norm’ in this block. The important operands are nn: the value named ’nn’ in this block; BatchNorm1d: the value named ’batch norm1d’ in this block; eval: the value named ’eval’ in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 40: `batch_norm.running_mean.copy_(torch.randn(3))` This calls ‘batch_norm.running_mean.copy_‘ with ‘torch.randn(3)‘ inside the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 41: `batch_norm.running_var.copy_(torch.rand(3) + 0.5)` This calls ‘batch_norm.running_var.copy_‘ with ‘torch.rand(3) + 0.5‘ inside the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 42: `writer = AlignedBinaryWriter()` This assigns ‘AlignedBinaryWriter()‘ to ‘writer ‘ in host memory. Here writer means the value named ’writer’ in this block. The important operands are AlignedBinaryWriter: the value named ’aligned binary writer’ in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 43: `exported = export_projection(` This statement continues through source line 52; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPUTE_BINARY",)‘ to ‘exported ‘ in host memory. Here exported means the value named ’exported’ in this block. The important operands are export_projection: the value named ’export projection’ in this block; writer: the value named ’writer’ in this block; name: the value named ’name’ in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 44: `writer,` This is continuation line 2 of the 10-line logical statement that begins at source line 43. Its exact fragment is ‘writer,‘. The important operands are writer: the value named ’writer’ in this block. Read together, the complete operation is: This assigns ‘export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPU)‘ to ‘exported ‘ in host memory. Here exported means the value named ’exported’ in this block. The important operands are export_projection: the value named ’export projection’ in this block; writer: the value named ’writer’ in this block; name: the value named ’name’ in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 45: `name="probe",` This is continuation line 3 of the 10-line logical statement that begins at source line 43. Its exact fragment is ‘name="probe",‘. The important operands are name: the value named ’name’ in this block; probe: the value named ’probe’ in this block. Read together, the complete operation is: This assigns ‘export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPUTE_BINARY",)‘ to ‘exported ‘ in host memory. Here exported means the value named ’exported’ in this block. The important operands are export_projection: the value named ’export projection’ in this block; writer: the value named ’writer’ in this block; name: the value named ’name’ in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 46: `convolution=convolution,` This is continuation line 4 of the 10-line logical statement that begins at source line 43. Its exact fragment is 'convolution=convolution,'. The important operands are convolution: the value named 'convolution' in this block. Read together, the complete operation is: This assigns 'export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPUTE_BINARY",)' to 'exported ' in host memory. Here exported means the value named 'exported' in this block. The important operands are export_projection: the value named 'export projection' in this block; writer: the value named 'writer' in this block; name: the value named 'name' in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 47: `batch_norm=batch_norm,` This is continuation line 5 of the 10-line logical statement that begins at source line 43. Its exact fragment is 'batch_norm=batch_norm,'. The important operands are batch_norm: the value named 'batch norm' in this block. Read together, the complete operation is: This assigns 'export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPUTE_BINARY",)' to 'exported ' in host memory. Here exported means the value named 'exported' in this block. The important operands are export_projection: the value named 'export projection' in this block; writer: the value named 'writer' in this block; name: the value named 'name' in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 48: `input_scale=1.0,` This is continuation line 6 of the 10-line logical statement that begins at source line 43. Its exact fragment is 'input_scale=1.0,'. The important operands are input_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns 'export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPUTE_BINARY",)' to 'exported ' in host memory. Here exported means the value named 'exported' in this block. The important operands are export_projection: the value named 'export projection' in this block; writer: the value named 'writer' in this block; name: the value named 'name' in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 49: `fractional_bits=10,` This is continuation line 7 of the 10-line logical statement that begins at source line 43. Its exact fragment is 'fractional_bits=10,'. The important operands are fractional_bits: the value named 'fractional bits' in this block. Read together, the complete operation is: This assigns 'export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPUTE_BINARY",)' to 'exported ' in host memory. Here exported means the value named 'exported' in this block. The important operands are export_projection: the value named 'export projection' in this block; writer: the value named 'writer' in this block; name: the value named 'name' in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 50: `input_encoding="binary spike 0/1",` This is continuation line 8 of the 10-line logical statement that begins at source line 43. Its exact fragment is 'input_encoding="binary spike 0/1",'. The important operands are input_encoding: the value named 'input encoding' in this block; binary: the value named 'binary' in this block; spike: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns 'export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPUTE_BINARY",)' to 'exported ' in host memory. Here exported means the value named 'exported' in this block. The important operands are export_projection: the value named 'export projection' in this block; writer: the value named 'writer' in this block; name: the value named 'name' in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 51: `compute_opcode="COMPUTE_BINARY",` This is continuation line 9 of the 10-line logical statement that begins at source line 43. Its exact fragment is 'compute_opcode="COMPUTE_BINARY",'. The important operands are compute_opcode: the value named 'compute opcode' in this block; COMPUTE_BINARY: the value named 'compute binary' in this block. Read together, the complete operation is: This assigns 'export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPUTE_BINARY",)' to 'exported ' in host memory. Here exported means the value

named 'exported' in this block. The important operands are export_projection: the value named 'export projection' in this block; writer: the value named 'writer' in this block; name: the value named 'name' in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 52: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 43. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'export_projection(writer, name="probe", convolution=convolution, batch_norm=batch_norm, input_scale=1.0, fractional_bits=10, input_encoding="binary spike 0/1", compute_opcode="COMPUTE_BINARY",)' to 'exported ' in host memory. Here exported means the value named 'exported' in this block. The important operands are export_projection: the value named 'export projection' in this block; writer: the value named 'writer' in this block; name: the value named 'name' in this block. Context: the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation.

Line 53: `self.assertEqual(exported.hardware_matrix_shape, (4, 3))` This calls 'self.assertEqual' with 'exported.hardware_matrix_shape, (4, 3)' inside the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 54: `self.assertEqual(len(exported.sections), 4)` This calls 'self.assertEqual' with 'len(exported.sections), 4' inside the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 55: `self.assertLess(exported.represented_factor_maximum_error, 0.001)` This calls 'self.assertLess' with 'exported.represented_factor_maximum_error, 0.001' inside the function test_projection_export_matches_folded_float_equation which handles test projection export matches folded float equation. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 56: `for section in exported.sections:` This iterates section over 'exported.sections'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_projection_export_matches_fo which handles test projection export matches folded float equation.

Line 57: `self.assertEqual(section.offset % ALIGNMENT, 0)` This calls 'self.assertEqual' with 'section.offset % ALIGNMENT, 0' inside the function test_projection_export_matches_folded_float_equation, which handles test projection export matches folded float equation. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

Line 60: `if __name__ == "__main__":` This executes the following indented host-code branch only when '___name___ == "___main___"' is true. The important operands are ___name___: the value named 'name' in this block; ___main___: the value named 'main' in this block. Context: the file whose role is to checks spikingformer export behavior independently with Python assertions.

Line 61: `unittest.main()` This calls 'unittest.main' with " inside the file whose role is to checks spikingformer export behavior independently with Python assertions. The call advances the host-side job that checks spikingformer export behavior independently with Python assertions.

72 tests/test_spikingformer_manifest.py

Role in the complete system: checks spikingformer manifest behavior independently with Python assertions.

72.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks spikingformer manifest behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: MatrixScheduleTests, SpikingformerManifestTests. Functions: test_outer_product_schedule_counts_ test_dot8_schedule_pads_reduction_without_changing_mac_count, test_large_reduction_is_split_at_hardware_limit, test_manifest_covers_every_learned_projection, test_temporal_hoist_counts_block0_once. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

72.2 Complete source

```
from __future__ import annotations

import unittest

from generate_spikingformer_manifest import build_operator_manifest, matrix_schedule


class MatrixScheduleTests(unittest.TestCase):
    def test_outer_product_schedule_counts_tiles_and_commands(self) -> None:
        schedule = matrix_schedule(
            9, 17, 18, arithmetic="binary_spike_add")
        self.assertEqual(schedule.compute_tiles, 4)
        self.assertEqual(schedule.commands, 10)
        self.assertEqual(schedule.mac_equivalents, 9 * 17 * 18)
        self.assertEqual(schedule.reduction_chunks, 1)

    def test_dot8_schedule_pads_reduction_without_changing_mac_count(self) -> None:
        schedule = matrix_schedule(1, 17, 18, arithmetic="u8_mac")
        self.assertEqual(schedule.compute_tiles, 2)
        self.assertEqual(schedule.commands, 5)
        self.assertEqual(schedule.mac_equivalents, 1 * 17 * 18)

    def test_large_reduction_is_split_at_hardware_limit(self) -> None:
        schedule = matrix_schedule(8, 3456, 16, arithmetic="binary_spike_add")
        self.assertEqual(schedule.reduction_chunks, 2)
        self.assertEqual(schedule.compute_tiles, 2)
        self.assertEqual(schedule.commands, 6)


class SpikingformerManifestTests(unittest.TestCase):
    def test_manifest_covers_every_learned_projection(self) -> None:
        operators = build_operator_manifest()
        matrix_operators = [item for item in operators if item.matrix is not None]
        self.assertEqual(len(matrix_operators), 5 + 4 * 8 + 1)
        names = {item.name for item in matrix_operators}
        self.assertIn("patch_embed.block0", names)
        self.assertIn("block.3.attn.qk", names)
        self.assertIn("block.3.mlp.fc2", names)
        self.assertIn("head", names)

    def test_temporal_hoist_counts_block0_once(self) -> None:
        operators = {item.name: item for item in build_operator_manifest()}
        block0 = operators["patch_embed.block0"]
        self.assertEqual(block0.matrix.repetitions, 1)
        self.assertIsNotNone(block0.exact_optimization)


if __name__ == "__main__":
    unittest.main()
```

72.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks spikingformer manifest behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import unittest` This imports the named Python API so the file whose role is to checks spikingformer manifest behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `from generate_spikingformer_manifest import build_operator_manifest, matrix_schedule` This imports the named Python API so the file whose role is to checks spikingformer manifest behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `class MatrixScheduleTests(unittest.TestCase):` This defines class MatrixScheduleTests, which groups data and methods for matrix schedule tests. Instances exist in host memory and organize the flow that checks spikingformer manifest behavior independently with Python assertions.

Line 9: `def test_outer_product_schedule_counts_tiles_and_commands(self) -> None:` This starts function test_outer_product_schedule_counts_tiles_and_commands. Its indented body performs test outer product schedule counts tiles and commands as part of the file that checks spikingformer manifest behavior independently with Python assertions; it runs only when called.

Line 10: `schedule = matrix_schedule(` This statement continues through source line 11; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘matrix_schedule(9, 17, 18, arithmetic="binary_spike_add")‘ to ‘schedule ‘ in host memory. Here schedule means the value named ’schedule’ in this block. The important operands are matrix_schedule: the value named ’matrix schedule’ in this block; arithmetic: the value named ’arithmetic’ in this block; binary_spike_add: a binary event produced when a neuron crosses its threshold. Context: the function test_outer_product_schedule_counts_tiles_and_commands, which handles test outer product schedule counts tiles and commands.

Line 11: `9, 17, 18, arithmetic="binary_spike_add")` This is continuation line 2 of the 2-line logical statement that begins at source line 10. Its exact fragment is ‘9, 17, 18, arithmetic="binary_spike_add")‘. The important operands are arithmetic: the value named ’arithmetic’ in this block; binary_spike_add: a binary event produced when a neuron crosses its threshold. Read together, the complete operation is: This assigns ‘matrix_schedule(9, 17, 18, arithmetic="binary_spike_add")‘ to ‘schedule ‘ in host memory. Here schedule means the value named ’schedule’ in this block. The important operands are matrix_schedule: the value named ’matrix schedule’ in this block; arithmetic: the value named ’arithmetic’ in this block; binary_spike_add: a binary event produced when a neuron crosses its threshold. Context: the function test_outer_product_schedule_counts_tiles_and_commands, which handles test outer product schedule counts tiles and commands.

Line 12: `self.assertEqual(schedule.compute_tiles, 4)` This calls ‘self.assertEqual‘ with ‘schedule.compute_tiles, 4‘ inside the function test_outer_product_schedule_counts_tiles_and_commands, which handles test outer product schedule counts tiles and commands. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 13: `self.assertEqual(schedule.commands, 10)` This calls ‘self.assertEqual‘ with ‘schedule.commands, 10‘ inside the function test_outer_product_schedule_counts_tiles_and_commands, which handles test outer product schedule counts tiles and commands. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 14: `self.assertEqual(schedule.mac_equivalents, 9 * 17 * 18)` This calls ‘self.assertEqual‘ with ‘schedule.mac_equivalents, 9 * 17 * 18‘ inside the function test_outer_product_schedule_counts_tiles_and_commands, which handles test outer product schedule counts tiles and commands. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 15: `self.assertEqual(schedule.reduction_chunks, 1)` This calls ‘self.assertEqual‘ with ‘schedule.reduction_chunks, 1‘ inside the function test_outer_product_schedule_counts_tiles_and_commands, which handles test outer product schedule counts tiles and commands. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 17: `def test_dot8_schedule_pads_reduction_without_changing_mac_count(self) -> None:` This starts function test_dot8_schedule_pads_reduction_without_changing_mac_count. Its indented body performs test dot8 schedule pads reduction without changing mac count as part of the file that checks spikingformer manifest behavior independently with Python assertions; it runs only when called.

Line 18: `schedule = matrix_schedule(1, 17, 18, arithmetic="u8_mac")` This assigns ‘matrix_schedule(1, 17, 18, arithmetic="u8_mac")‘ to ‘schedule ‘ in host memory. Here schedule means the value named ’schedule’ in this block. The important operands are matrix_schedule: the value named ’matrix schedule’ in this block; arithmetic: the value named ’arithmetic’ in this block; u8_mac: the value named ’u8 mac’ in this block. Context: the function test_dot8_schedule_pads_reduction_without_changing_mac_count, which handles test dot8 schedule pads reduction without changing mac count.

Line 19: `self.assertEqual(schedule.compute_tiles, 2)` This calls ‘self.assertEqual‘ with ‘schedule.compute_tiles, 2‘ inside the function test_dot8_schedule_pads_reduction_without_changing_mac_count, which handles test dot8 schedule pads reduction without changing mac count. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 20: `self.assertEqual(schedule.commands, 5)` This calls ‘self.assertEqual‘ with ‘schedule.commands, 5‘ inside the function test_dot8_schedule_pads_reduction_without_changing_mac_count, which handles test dot8 schedule pads reduction without changing mac count. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 21: `self.assertEqual(schedule.mac_equivalents, 1 * 17 * 18)` This calls ‘self.assertEqual‘ with ‘schedule.mac_equivalents, 1 * 17 * 18‘ inside the function test_dot8_schedule_pads_reduction_without_changing_mac_count, which handles test dot8 schedule pads reduction without changing mac count. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 23: `def test_large_reduction_is_split_at_hardware_limit(self) -> None:` This starts function test_large_reducti Its indented body performs test large reduction is split at hardware limit as part of the file that checks spikingformer manifest behavior independently with Python assertions; it runs only when called.

Line 24: `schedule = matrix_schedule(8, 3456, 16, arithmetic="binary_spike_add")` This assigns ‘matrix_schedule(8, 3456, 16, arithmetic="binary_spike_add")‘ to ‘schedule ‘ in host memory. Here schedule means the value named ’schedule’ in this block. The important operands are matrix_schedule: the value named ’matrix schedule’ in this block; arithmetic: the value named ’arithmetic’ in this block; binary_spike_add: a binary event produced when a neuron crosses its threshold. Context: the function test_large_reduction_is_split_at_hardware_limit, which handles test large reduction is split at hardware limit.

Line 25: `self.assertEqual(schedule.reduction_chunks, 2)` This calls ‘self.assertEqual‘ with ‘schedule.reduction_chunks, 2‘ inside the function test_large_reduction_is_split_at_hardware_limit, which handles test large reduction is split at hardware limit. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 26: `self.assertEqual(schedule.compute_tiles, 2)` This calls ‘self.assertEqual‘ with ‘schedule.compute_tiles, 2‘ inside the function test_large_reduction_is_split_at_hardware_limit, which handles test large reduction is split at hardware limit. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 27: `self.assertEqual(schedule.commands, 6)` This calls ‘self.assertEqual‘ with ‘schedule.commands, 6‘ inside the function test_large_reduction_is_split_at_hardware_limit, which handles test large reduction is split at hardware limit. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 30: `class SpikingformerManifestTests(unittest.TestCase):` This defines class SpikingformerManifestTests, which groups data and methods for spikingformer manifest tests. Instances exist in host memory and organize the flow that checks spikingformer manifest behavior independently with Python assertions.

Line 31: `def test_manifest_covers_every_learned_projection(self) -> None:` This starts function test_manifest_covers Its indented body performs test manifest covers every learned projection as part of the file that checks spikingformer manifest behavior independently with Python assertions; it runs only when called.

Line 32: `operators = build_operator_manifest()` This assigns ‘build_operator_manifest()‘ to ‘operators ‘ in host memory. Here operators means the value named ’operators’ in this block. The important operands are build_operator_manifest: the value named ’build operator manifest’ in this block. Context: the function test_manifest_covers_every_learned_projection, which handles test manifest covers every learned projection.

Line 33: `matrix_operators = [item for item in operators if item.matrix is not None]` This assigns ‘[item for item in operators if item.matrix is not None]‘ to ‘matrix_operators ‘ in host memory. Here matrix_operators means the value named ’matrix operators’ in this block. The important operands are item: the value named ’item’ in this block; for: the value named ’for’ in this block; in: the value named ’in’ in this block. Context: the function test_manifest_covers_every_learned_projection, which handles test manifest covers every learned projection.

Line 34: `self.assertEqual(len(matrix_operators), 5 + 4 * 8 + 1)` This calls ‘self.assertEqual‘ with ‘len(matrix_operator 5 + 4 * 8 + 1‘ inside the function test_manifest_covers_every_learned_projection, which handles test manifest covers every learned projection. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 35: `names = {item.name for item in matrix_operators}` This assigns ‘{item.name for item in matrix_operators}‘ to ‘names ‘ in host memory. Here names means the value named ’names’ in this block. The important operands are item: the value named ’item’ in this block; name: the value named ’name’ in this block; for: the value named ’for’ in this block. Context: the function test_manifest_covers_every_learned_projection, which handles test manifest covers every learned projection.

Line 36: `self.assertIn("patch_embed.block0", names)` This calls ‘self.assertIn‘ with ‘"patch_embed.block0", names‘ inside the function test_manifest_covers_every_learned_projection, which handles test manifest covers every learned projection. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 37: `self.assertIn("block.3.attn.qk", names)` This calls ‘self.assertIn‘ with ‘"block.3.attn.qk", names‘ inside the function test_manifest_covers_every_learned_projection, which handles test manifest covers every learned projection. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 38: `self.assertIn("block.3.mlp.fc2", names)` This calls ‘self.assertIn‘ with ‘"block.3.mlp.fc2", names‘ inside the function test_manifest_covers_every_learned_projection, which handles test manifest covers every learned projection. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 39: `self.assertIn("head", names)` This calls ‘self.assertIn‘ with ‘"head", names‘ inside the function test_manifest_covers which handles test manifest covers every learned projection. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 41: `def test_temporal_hoist_counts_block0_once(self) -> None:` This starts function test_temporal_hoist_counts_ Its indented body performs test temporal hoist counts block0 once as part of the file that checks spikingformer manifest behavior independently with Python assertions; it runs only when called.

Line 42: `operators = {item.name: item for item in build_operator_manifest()}` This assigns '{item.name: item for item in build_operator_manifest()}' to 'operators ' in host memory. Here operators means the value named 'operators' in this block. The important operands are item: the value named 'item' in this block; name: the value named 'name' in this block; for: the value named 'for' in this block. Context: the function test_temporal_hoist_counts_block0_once, which handles test temporal hoist counts block0 once.

Line 43: `block0 = operators["patch_embed.block0"]` This assigns 'operators["patch_embed.block0"]' to 'block0 ' in host memory. Here block0 means the value named 'block0' in this block. The important operands are operators: the value named 'operators' in this block; patch_embed: the value named 'patch embed' in this block; block0: the value named 'block0' in this block. Context: the function test_temporal_hoist_counts_block0_once, which handles test temporal hoist counts block0 once.

Line 44: `self.assertEqual(block0.matrix.repetitions, 1)` This calls 'self.assertEqual' with 'block0.matrix.repetitions, 1' inside the function test_temporal_hoist_counts_block0_once, which handles test temporal hoist counts block0 once. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 45: `self.assertIsNotNone(block0.exact_optimization)` This calls 'self.assertIsNotNone' with 'block0.exact_optimizati inside the function test_temporal_hoist_counts_block0_once, which handles test temporal hoist counts block0 once. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

Line 48: `if __name__ == "__main__":` This executes the following indented host-code branch only when '___name___ == "___main___"' is true. The important operands are ___name___: the value named 'name' in this block; ___main___: the value named 'main' in this block. Context: the file whose role is to checks spikingformer manifest behavior independently with Python assertions.

Line 49: `unittest.main()` This calls 'unittest.main' with " inside the file whose role is to checks spikingformer manifest behavior independently with Python assertions. The call advances the host-side job that checks spikingformer manifest behavior independently with Python assertions.

73 tests/test_spikingformer_parameters.py

Role in the complete system: checks spikingformer parameters behavior independently with Python assertions.

73.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks spikingformer parameters behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: ParameterArchiveTests. Functions: _write_archive, test_loads_typed_read_only_projection_and_h test_rejects_one_corrupted_parameter_byte, test_rejects_overlapping_section_metadata_without_hash_checks. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

73.2 Complete source

```
from __future__ import annotations

import hashlib
import json
import tempfile
import unittest
from pathlib import Path

import numpy as np

from export_spikingformer_hardware import AlignedBinaryWriter
from spikingformer_parameters import FORMAT, SpikingformerParameterArchive


class ParameterArchiveTests(unittest.TestCase):
    def _write_archive(self, directory: Path) -> Path:
        writer = AlignedBinaryWriter()
        sections = [
            writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)),
            writer.append("unit.bias_q10", np.arange(4, dtype="<i4")),
            writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")),
            writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)),
            writer.append("head.weight_k_by_n_int8", np.arange(8, dtype=np.int8).reshape(4, 2)),
            writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),
        ]
        binary = directory / "parameters.bin"
        binary.write_bytes(writer.data)
        manifest = {
            "format": FORMAT,
            "binary_file": binary.name,
            "binary_bytes": len(writer.data),
            "binary_sha256": hashlib.sha256(writer.data).hexdigest(),
            "alignment_bytes": 64,
            "projections": [{
                "name": "unit",
                "compute_opcode": "COMPUTE_BINARY",
                "source_weight_shape": [4, 3, 1, 1],
                "output_fractional_bits": 10,
                "sections": [section.__dict__ for section in sections[:4]],
            }],
            "head": {"sections": [section.__dict__ for section in sections[4:]]},
            "sections": [section.__dict__ for section in sections],
        }
        path = directory / "parameters.json"
        path.write_text(json.dumps(manifest), encoding="ascii")
        return path

    def test_loads_typed_read_only_projection_and_head(self) -> None:
        with tempfile.TemporaryDirectory() as temporary:
            archive = SpikingformerParameterArchive(self._write_archive(Path(temporary)))
            projection = archive.projection("unit")
            self.assertEqual(projection.weight_k_by_n.shape, (3, 4))
```

```
            self.assertEqual(projection.weight_k_by_n.dtype, np.dtype("int8"))
            self.assertFalse(projection.weight_k_by_n.flags.writeable)
            self.assertEqual(archive.head.weight_k_by_n.shape, (4, 2))
            self.assertEqual(archive.projection_names, ("unit",))

    def test_rejects_one_corrupted_parameter_byte(self) -> None:
        with tempfile.TemporaryDirectory() as temporary:
            directory = Path(temporary)
            manifest = self._write_archive(directory)
            binary = directory / "parameters.bin"
            payload = bytearray(binary.read_bytes())
            payload[0] ^= 0x80
            binary.write_bytes(payload)
            with self.assertRaisesRegex(ValueError, "SHA-256 mismatch"):
                SpikingformerParameterArchive(manifest)

    def test_rejects_overlapping_section_metadata_without_hash_checks(self) -> None:
        with tempfile.TemporaryDirectory() as temporary:
            directory = Path(temporary)
            manifest_path = self._write_archive(directory)
            manifest = json.loads(manifest_path.read_text(encoding="ascii"))
            manifest["sections"][1]["offset"] = manifest["sections"][0]["offset"]
            manifest_path.write_text(json.dumps(manifest), encoding="ascii")
            with self.assertRaisesRegex(ValueError, "overlaps"):
                SpikingformerParameterArchive(manifest_path, verify_hashes=False)


if __name__ == "__main__":
    unittest.main()
```

73.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks spikingformer parameters behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import hashlib` This imports the named Python API so the file whose role is to checks spikingformer parameters behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 4: `import json` This imports the named Python API so the file whose role is to checks spikingformer parameters behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import tempfile` This imports the named Python API so the file whose role is to checks spikingformer parameters behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import unittest` This imports the named Python API so the file whose role is to checks spikingformer parameters behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from pathlib import Path` This imports the named Python API so the file whose role is to checks spikingformer parameters behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import numpy as np` This imports the named Python API so the file whose role is to checks spikingformer parameters behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `from export_spikingformer_hardware import AlignedBinaryWriter` This imports the named Python API so the file whose role is to checks spikingformer parameters behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `from spikingformer_parameters import FORMAT, SpikingformerParameterArchive` This imports the named Python API so the file whose role is to checks spikingformer parameters behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `class ParameterArchiveTests(unittest.TestCase):` This defines class ParameterArchiveTests, which groups data and methods for parameter archive tests. Instances exist in host memory and organize the flow that checks spikingformer parameters behavior independently with Python assertions.

Line 16: `def _write_archive(self, directory: Path) -> Path:` This starts function _write_archive. Its indented body performs write archive as part of the file that checks spikingformer parameters behavior independently with Python assertions; it runs only when called.

Line 17: `writer = AlignedBinaryWriter()` This assigns ‘AlignedBinaryWriter()‘ to ‘writer ‘ in host memory. Here writer means the value named ’writer’ in this block. The important operands are AlignedBinaryWriter: the value named ’aligned binary writer’ in this block. Context: the function _write_archive, which handles write archive.

Line 18: `sections = [` This statement continues through source line 25; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘[writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)), writer.append("unit.bias_q10", np.arange(4, dtype="<i4")), writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")), writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)), writer.append("head.weight_
np.arange(8, dtype=np.int8).reshape(4, 2)), writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),]‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Context: the function _write_archive, which handles write archive.

Line 19: `writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)),` This is continuation line 2 of the 8-line logical statement that begins at source line 18. Its exact fragment is ‘writer.append("unit.weight_k_
np.arange(12, dtype=np.int8).reshape(3, 4)),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Read together, the complete operation is: This assigns ‘[writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)), writer.append("unit.bias_q10", np.arange(4, dtype="<i4")), writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")), writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)), writer.append("head.weight_k_by_n_int8", np.arange(8, dtype=np.int8).reshape(4, 2)), writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),]‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Context: the function _write_archive, which handles write archive.

Line 20: `writer.append("unit.bias_q10", np.arange(4, dtype="<i4")),` This is continuation line 3 of the 8-line logical statement that begins at source line 18. Its exact fragment is ‘writer.append("unit.bias_q10", np.arange(4, dtype="<i4")),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Read together, the complete operation is: This assigns ‘[writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)), writer.append("unit.bias_q10", np.arange(4, dtype="<i4")), writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")), writer.append("unit.binary_point_s
np.arange(4, dtype=np.int8)), writer.append("head.weight_k_by_n_int8", np.arange(8, dtype=np.int8).reshape(4, 2)), writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),]‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Context: the function _write_archive, which handles write archive.

Line 21: `writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")),` This is continuation line 4 of the 8-line logical statement that begins at source line 18. Its exact fragment is ‘writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Read together, the complete operation is: This assigns ‘[writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)), writer.append("unit.bias_q10", np.arange(4, dtype="<i4")), writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")), writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)), writer.append("head.weight_k_by_n_int8", np.arange(8,

dtype=np.int8).reshape(4, 2)), writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),]‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Context: the function _write_archive, which handles write archive.

Line 22: `writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)),` This is continuation line 5 of the 8-line logical statement that begins at source line 18. Its exact fragment is ‘writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Read together, the complete operation is: This assigns ‘[writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)), writer.append("unit.bias_q10", np.arange(4, dtype="<i4")), writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")), writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)), writer.append("head.weight_k_by_n_int8", np.arange(8, dtype=np.int8).reshape(4, 2)), writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),]‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Context: the function _write_archive, which handles write archive.

Line 23: `writer.append("head.weight_k_by_n_int8", np.arange(8, dtype=np.int8).reshape(4, 2)),` This is continuation line 6 of the 8-line logical statement that begins at source line 18. Its exact fragment is ‘writer.append("head.weight_k_b np.arange(8, dtype=np.int8).reshape(4, 2)),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; head: the value named ’head’ in this block. Read together, the complete operation is: This assigns ‘[writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)), writer.append("unit.bias_q10", np.arange(4, dtype="<i4")), writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")), writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)), writer.append("head.weight_k_by_n_int8", np.arange(8, dtype=np.int8).reshape(4, 2)), writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),]‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Context: the function _write_archive, which handles write archive.

Line 24: `writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),` This is continuation line 7 of the 8-line logical statement that begins at source line 18. Its exact fragment is ‘writer.append("head.bias_accumulator_units np.arange(2, dtype="<i4")),‘. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; head: the value named ’head’ in this block. Read together, the complete operation is: This assigns ‘[writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)), writer.append("unit.bias_q10", np.arange(4, dtype="<i4")), writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")), writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)), writer.append("head.weight_k_by_n_int8", np.arange(8, dtype=np.int8).reshape(4, 2)), writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),]‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Context: the function _write_archive, which handles write archive.

Line 25: `]` This is continuation line 8 of the 8-line logical statement that begins at source line 18. Its exact fragment is ‘]‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘[writer.append("unit.weight_k_by_n_int8", np.arange(12, dtype=np.int8).reshape(3, 4)), writer.append("unit.bias_q10", np.arange(4, dtype="<i4")), writer.append("unit.multiplier_q15", np.arange(4, dtype="<i2")), writer.append("unit.binary_point_shift", np.arange(4, dtype=np.int8)), writer.append("head.weight_k_by_n_int8", np.arange(8, dtype=np.int8).reshape(4, 2)), writer.append("head.bias_accumulator_units", np.arange(2, dtype="<i4")),]‘ to ‘sections ‘ in host memory. Here sections means the value named ’sections’ in this block. The important operands are writer: the value named ’writer’ in this block; append: the value named ’append’ in this block; unit: the value named ’unit’ in this block. Context: the function _write_archive, which handles write archive.

Line 26: `binary = directory / "parameters.bin"` This assigns ‘directory / "parameters.bin"‘ to ‘binary ‘ in host memory. Here binary means the value named ’binary’ in this block. The important operands are directory: the value named ’directory’ in this block; parameters: the value named ’parameters’ in this block; bin: the value named ’bin’ in this block. Context: the function _write_archive, which handles write archive.

Line 27: `binary.write_bytes(writer.data)` This calls ‘binary.write_bytes‘ with ‘writer.data‘ inside the function _write_archive, which handles write archive. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 28: `manifest = {` This statement continues through source line 43; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 29: `"format": FORMAT,` This is continuation line 2 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"format": FORMAT,‘. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 30: `"binary_file": binary.name,` This is continuation line 3 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"binary_file": binary.name,‘. The important operands are binary_file: the value named ’binary file’ in this block; binary: the value named ’binary’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 31: `"binary_bytes": len(writer.data),` This is continuation line 4 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"binary_bytes": len(writer.data),‘. The important operands are binary_bytes: the value named ’binary bytes’ in this block; len: the value named ’len’ in this block; writer: the value named ’writer’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 32: `"binary_sha256": hashlib.sha256(writer.data).hexdigest(),` This is continuation line 5 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"binary_sha256": hashlib.sha256(writer.data).hexdigest(),‘. The important operands are binary_sha256: the value named ’binary sha256’ in this block; hashlib: the value named ’hashlib’ in this block; sha256: the value named ’sha256’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___

for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 33: `"alignment_bytes": 64,` This is continuation line 6 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"alignment_bytes": 64,‘. The important operands are alignment_bytes: the value named ’alignment bytes’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 34: `"projections": [{` This is continuation line 7 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"projections": [{‘. The important operands are projections: the value named ’projections’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 35: `"name": "unit",` This is continuation line 8 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"name": "unit",‘. The important operands are name: the value named ’name’ in this block; unit: the value named ’unit’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 36: `"compute_opcode": "COMPUTE_BINARY",` This is continuation line 9 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"compute_opcode": "COMPUTE_BINARY",‘. The important operands are compute_opcode: the value named ’compute opcode’ in this block; COMPUTE_BINARY: the value named ’compute binary’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 37: `"source_weight_shape": [4, 3, 1, 1],` This is continuation line 10 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"source_weight_shape": [4, 3, 1, 1],‘. The important operands are source_weight_shape: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___

for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 38: `"output_fractional_bits": 10,` This is continuation line 11 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"output_fractional_bits": 10,‘. The important operands are output_fractional_bits: the value named ’output fractional bits’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 39: `"sections": [section.__dict__ for section in sections[:4]],` This is continuation line 12 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"sections": [section.___dict___ for section in sections[:4]],‘. The important operands are sections: the value named ’sections’ in this block; section: the value named ’section’ in this block; ___dict___: the value named ’dict’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 40: `}],` This is continuation line 13 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘}],‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 41: `"head": {"sections": [section.__dict__ for section in sections[4:]]},` This is continuation line 14 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"head": {"sections": [section.___dict___ for section in sections[4:]]},‘. The important operands are head: the value named ’head’ in this block; sections: the value named ’sections’ in this block; section: the value named ’section’ in this block. Read together, the complete operation is: This assigns ‘{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are format: the value named ’format’ in this block; FORMAT: the value named ’format’ in this block; binary_file: the value named ’binary file’ in this block. Context: the function _write_archive, which handles write archive.

Line 42: `"sections": [section.__dict__ for section in sections],` This is continuation line 15 of the 16-line logical statement that begins at source line 28. Its exact fragment is ‘"sections": [section.___dict___ for section in sections],‘. The important operands are sections: the value named ’sections’ in this block; section: the value named

'section' in this block; ___dict___: the value named 'dict' in this block. Read together, the complete operation is: This assigns '{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; FORMAT: the value named 'format' in this block; binary_file: the value named 'binary file' in this block. Context: the function _write_archive, which handles write archive.

Line 43: `}` This is continuation line 16 of the 16-line logical statement that begins at source line 28. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "format": FORMAT, "binary_file": binary.name, "binary_bytes": len(writer.data), "binary_sha256": hashlib.sha256(writer.data).hexdigest(), "alignment_bytes": 64, "projections": [{ "name": "unit", "compute_opcode": "COMPUTE_BINARY", "source_weight_shape": [4, 3, 1, 1], "output_fractional_bits": 10, "sections": [section.___dict___ for section in sections[:4]], }], "head": {"sections": [section.___dict___ for section in sections[4:]]}, "sections": [section.___dict___ for section in sections], }' to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; FORMAT: the value named 'format' in this block; binary_file: the value named 'binary file' in this block. Context: the function _write_archive, which handles write archive.

Line 44: `path = directory / "parameters.json"` This assigns 'directory / "parameters.json"' to 'path ' in host memory. Here path means the value named 'path' in this block. The important operands are directory: the value named 'directory' in this block; parameters: the value named 'parameters' in this block; json: the value named 'json' in this block. Context: the function _write_archive, which handles write archive.

Line 45: `path.write_text(json.dumps(manifest), encoding="ascii")` This calls 'path.write_text' with 'json.dumps(manife encoding="ascii"' inside the function _write_archive, which handles write archive. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 46: `return path` This returns 'path' to the caller of the function _write_archive, which handles write archive, ending this invocation. The important operands are path: the value named 'path' in this block.

Line 48: `def test_loads_typed_read_only_projection_and_head(self) -> None:` This starts function test_loads_typed_r Its indented body performs test loads typed read only projection and head as part of the file that checks spikingformer parameters behavior independently with Python assertions; it runs only when called.

Line 49: `with tempfile.TemporaryDirectory() as temporary:` This opens a managed resource scope for 'tempfile.TemporaryDirectory() as temporary' so files, tracing, or temporary state are finalized even if the function test_loads_typed_read which handles test loads typed read only projection and head raises an error.

Line 50: `archive = SpikingformerParameterArchive(self._write_archive(Path(temporary)))` This assigns 'SpikingformerParameterArchive(self._write_archive(Path(temporary)))' to 'archive ' in host memory. Here archive means the value named 'archive' in this block. The important operands are SpikingformerParameterArchive: the value named 'spikingformer parameter archive' in this block; _write_archive: the value named 'write archive' in this block; Path: the value named 'path' in this block. Context: the function test_loads_typed_read_only_projection_and_head, which handles test loads typed read only projection and head.

Line 51: `projection = archive.projection("unit")` This assigns 'archive.projection("unit")' to 'projection ' in host memory. Here projection means the value named 'projection' in this block. The important operands are archive: the value named 'archive' in this block; projection: the value named 'projection' in this block; unit: the value named 'unit' in this block. Context: the function test_loads_typed_read_only_projection_and_head, which handles test loads typed read only projection and head.

Line 52: `self.assertEqual(projection.weight_k_by_n.shape, (3, 4))` This calls ‘self.assertEqual‘ with ‘projection.weight_k_by_n.shape, (3, 4)‘ inside the function test_loads_typed_read_only_projection_and_head, which handles test loads typed read only projection and head. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 53: `self.assertEqual(projection.weight_k_by_n.dtype, np.dtype("int8"))` This calls ‘self.assertEqual‘ with ‘projection.weight_k_by_n.dtype, np.dtype("int8")‘ inside the function test_loads_typed_read_only_projection_and_head, which handles test loads typed read only projection and head. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 54: `self.assertFalse(projection.weight_k_by_n.flags.writeable)` This calls ‘self.assertFalse‘ with ‘projection.weight_k_by_n.flags.writeable‘ inside the function test_loads_typed_read_only_projection_and_head, which handles test loads typed read only projection and head. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 55: `self.assertEqual(archive.head.weight_k_by_n.shape, (4, 2))` This calls ‘self.assertEqual‘ with ‘archive.head.we (4, 2)‘ inside the function test_loads_typed_read_only_projection_and_head, which handles test loads typed read only projection and head. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 56: `self.assertEqual(archive.projection_names, ("unit",))` This calls ‘self.assertEqual‘ with ‘archive.projection_na ("unit",)‘ inside the function test_loads_typed_read_only_projection_and_head, which handles test loads typed read only projection and head. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 58: `def test_rejects_one_corrupted_parameter_byte(self) -> None:` This starts function test_rejects_one_corrupt Its indented body performs test rejects one corrupted parameter byte as part of the file that checks spikingformer parameters behavior independently with Python assertions; it runs only when called.

Line 59: `with tempfile.TemporaryDirectory() as temporary:` This opens a managed resource scope for ‘tempfile.TemporaryDirectory() as temporary‘ so files, tracing, or temporary state are finalized even if the function test_rejects_one_corru which handles test rejects one corrupted parameter byte raises an error.

Line 60: `directory = Path(temporary)` This assigns ‘Path(temporary)‘ to ‘directory ‘ in host memory. Here directory means the value named ’directory’ in this block. The important operands are Path: the value named ’path’ in this block; temporary: the value named ’temporary’ in this block. Context: the function test_rejects_one_corrupted_parameter_byte, which handles test rejects one corrupted parameter byte.

Line 61: `manifest = self._write_archive(directory)` This assigns ‘self._write_archive(directory)‘ to ‘manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are _write_archive: the value named ’write archive’ in this block; directory: the value named ’directory’ in this block. Context: the function test_rejects_one_corrupted_parameter_byte, which handles test rejects one corrupted parameter byte.

Line 62: `binary = directory / "parameters.bin"` This assigns ‘directory / "parameters.bin"‘ to ‘binary ‘ in host memory. Here binary means the value named ’binary’ in this block. The important operands are directory: the value named ’directory’ in this block; parameters: the value named ’parameters’ in this block; bin: the value named ’bin’ in this block. Context: the function test_rejects_one_corrupted_parameter_byte, which handles test rejects one corrupted parameter byte.

Line 63: `payload = bytearray(binary.read_bytes())` This assigns ‘bytearray(binary.read_bytes())‘ to ‘payload ‘ in host memory. Here payload means stream data belonging to the current command. The important operands are bytearray: the value named ’bytearray’ in this block; binary: the value named ’binary’ in this block; read_bytes: the value named ’read bytes’ in this block. Context: the function test_rejects_one_corrupted_parameter_byte, which handles test rejects one corrupted parameter byte.

Line 64: `payload[0] ^= 0x80` Inside the function test_rejects_one_corrupted_parameter_byte, which handles test rejects one corrupted parameter byte, this statement advances the host implementation that checks spikingformer parameters behavior independently with Python assertions. The important operands are payload: stream data belonging to the current command.

Line 65: `binary.write_bytes(payload)` This calls 'binary.write_bytes' with 'payload' inside the function test_rejects_one_cor which handles test rejects one corrupted parameter byte. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 66: `with self.assertRaisesRegex(ValueError, "SHA-256 mismatch"):` This opens a managed resource scope for 'self.assertRaisesRegex(ValueError, "SHA-256 mismatch")' so files, tracing, or temporary state are finalized even if the function test_rejects_one_corrupted_parameter_byte, which handles test rejects one corrupted parameter byte raises an error.

Line 67: `SpikingformerParameterArchive(manifest)` This calls 'SpikingformerParameterArchive' with 'manifest' inside the function test_rejects_one_corrupted_parameter_byte, which handles test rejects one corrupted parameter byte. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 69: `def test_rejects_overlapping_section_metadata_without_hash_checks(self) -> None:` This starts function test_rejects_overlapping_section_metadata_without_hash_checks. Its indented body performs test rejects overlapping section metadata without hash checks as part of the file that checks spikingformer parameters behavior independently with Python assertions; it runs only when called.

Line 70: `with tempfile.TemporaryDirectory() as temporary:` This opens a managed resource scope for 'tempfile.TemporaryDirectory() as temporary' so files, tracing, or temporary state are finalized even if the function test_rejects_overlappin which handles test rejects overlapping section metadata without hash checks raises an error.

Line 71: `directory = Path(temporary)` This assigns 'Path(temporary)' to 'directory ' in host memory. Here directory means the value named 'directory' in this block. The important operands are Path: the value named 'path' in this block; temporary: the value named 'temporary' in this block. Context: the function test_rejects_overlapping_section_metadata_w which handles test rejects overlapping section metadata without hash checks.

Line 72: `manifest_path = self._write_archive(directory)` This assigns 'self._write_archive(directory)' to 'manifest_path ' in host memory. Here manifest_path means the value named 'manifest path' in this block. The important operands are _write_archive: the value named 'write archive' in this block; directory: the value named 'directory' in this block. Context: the function test_rejects_overlapping_section_metadata_without_hash_checks, which handles test rejects overlapping section metadata without hash checks.

Line 73: `manifest = json.loads(manifest_path.read_text(encoding="ascii"))` This assigns 'json.loads(manifest_path.rea to 'manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are json: the value named 'json' in this block; loads: the value named 'loads' in this block; manifest_path: the value named 'manifest path' in this block. Context: the function test_rejects_overlapping_section_metadata_without_hash_checks, which handles test rejects overlapping section metadata without hash checks.

Line 74: `manifest["sections"][1]["offset"] = manifest["sections"][0]["offset"]` Inside the function test_rejects_ove which handles test rejects overlapping section metadata without hash checks, this statement advances the host implementation that checks spikingformer parameters behavior independently with Python assertions. The important operands are manifest: the value named 'manifest' in this block; sections: the value named 'sections' in this block; offset: the value named 'offset' in this block.

Line 75: `manifest_path.write_text(json.dumps(manifest), encoding="ascii")` This calls 'manifest_path.write_text' with 'json.dumps(manifest), encoding="ascii"' inside the function test_rejects_overlapping_section_metadata_without_hash_che which handles test rejects overlapping section metadata without hash checks. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 76: `with self.assertRaisesRegex(ValueError, "overlaps"):` This opens a managed resource scope for ‘self.assertRaisesRegex(ValueError, "overlaps")‘ so files, tracing, or temporary state are finalized even if the function test_rejects_overlapping_section_metadata_without_hash_checks, which handles test rejects overlapping section metadata without hash checks raises an error.

Line 77: `SpikingformerParameterArchive(manifest_path, verify_hashes=False)` This calls ‘SpikingformerParameterArchive‘ with ‘manifest_path, verify_hashes=False‘ inside the function test_rejects_overlapping_section_metadata_witho which handles test rejects overlapping section metadata without hash checks. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

Line 80: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks spikingformer parameters behavior independently with Python assertions.

Line 81: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks spikingformer parameters behavior independently with Python assertions. The call advances the host-side job that checks spikingformer parameters behavior independently with Python assertions.

74 tests/test_spikingformer_reference.py

Role in the complete system: checks spikingformer reference behavior independently with Python assertions.

74.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks spikingformer reference behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: MultiStepLIFTests, BatchNormFoldingTests, ReferenceGraphTests. Functions: test_hard_reset_sequ test_fixed_point_lif_matches_rtl_step_order, test_attention_threshold_is_inclusive, test_conv2d_fold_matches_eval_g test_conv1d_fold_matches_eval_graph, test_small_graph_shape_and_binary_observations, test_calibration_observer_a test_hardware_emulation_quantizes_all_learned_operators, observer. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

74.2 Complete source

```
from __future__ import annotations

import unittest

import torch
import torch.nn as nn

from spikingformer_reference import (
    MultiStepHardResetLIF,
    Spikingformer,
    fold_batch_norm,
)
from evaluate_spikingformer import CalibrationObserver
from spikingformer_quantization import (
    FixedPointHardResetLIF,
    fake_fixed_point,
    prepare_hardware_emulation,
    quantize_cifar_input_u8,
)


class MultiStepLIFTests(unittest.TestCase):
    def test_hard_reset_sequence(self) -> None:
        neuron = MultiStepHardResetLIF(tau=2.0, threshold=1.0)
        sequence = torch.tensor([[1.0], [1.0], [1.0], [2.0], [1.0]])
        expected = torch.tensor([[0.0], [0.0], [0.0], [1.0], [0.0]])
        torch.testing.assert_close(neuron(sequence), expected)

    def test_fixed_point_lif_matches_rtl_step_order(self) -> None:
        neuron = FixedPointHardResetLIF(threshold=1.0, fractional_bits=4)
        sequence = torch.tensor([[1.0], [1.0], [1.0], [2.0], [1.0]])
        expected = torch.tensor([[0.0], [0.0], [0.0], [1.0], [0.0]])
        torch.testing.assert_close(neuron(sequence), expected)

    def test_attention_threshold_is_inclusive(self) -> None:
        neuron = MultiStepHardResetLIF(tau=2.0, threshold=0.5)
        sequence = torch.tensor([[1.0], [0.0]])
        expected = torch.tensor([[1.0], [0.0]])
        torch.testing.assert_close(neuron(sequence), expected)


class BatchNormFoldingTests(unittest.TestCase):
    def test_conv2d_fold_matches_eval_graph(self) -> None:
        torch.manual_seed(7)
        convolution = nn.Conv2d(3, 5, kernel_size=3, padding=1, bias=False)
        batch_norm = nn.BatchNorm2d(5)
        batch_norm.running_mean.copy_(torch.randn(5))
        batch_norm.running_var.copy_(torch.rand(5) + 0.2)
        batch_norm.weight.data.copy_(torch.randn(5))
        batch_norm.bias.data.copy_(torch.randn(5))
```

```
        convolution.eval()
        batch_norm.eval()
        image = torch.randn(2, 3, 7, 7)

        folded_weight, folded_bias = fold_batch_norm(convolution, batch_norm)
        folded = nn.Conv2d(3, 5, kernel_size=3, padding=1, bias=True)
        folded.weight.data.copy_(folded_weight)
        folded.bias.data.copy_(folded_bias)
        folded.eval()

        torch.testing.assert_close(
            folded(image), batch_norm(convolution(image)), rtol=2e-5, atol=2e-6)

    def test_conv1d_fold_matches_eval_graph(self) -> None:
        torch.manual_seed(11)
        convolution = nn.Conv1d(4, 6, kernel_size=1, bias=True)
        batch_norm = nn.BatchNorm1d(6)
        batch_norm.running_mean.copy_(torch.randn(6))
        batch_norm.running_var.copy_(torch.rand(6) + 0.2)
        convolution.eval()
        batch_norm.eval()
        values = torch.randn(3, 4, 9)

        folded_weight, folded_bias = fold_batch_norm(convolution, batch_norm)
        folded = nn.Conv1d(4, 6, kernel_size=1, bias=True)
        folded.weight.data.copy_(folded_weight)
        folded.bias.data.copy_(folded_bias)
        folded.eval()

        torch.testing.assert_close(
            folded(values), batch_norm(convolution(values)), rtol=2e-5, atol=2e-6)


class ReferenceGraphTests(unittest.TestCase):
    def test_small_graph_shape_and_binary_observations(self) -> None:
        torch.manual_seed(13)
        model = Spikingformer(
            time_steps=2,
            input_channels=3,
            classes=10,
            embed_dim=24,
            heads=3,
            depth=1,
            mlp_ratio=2,
        ).eval()
        observed_spikes: list[torch.Tensor] = []

        def observer(_name: str, value: torch.Tensor, is_spike: bool) -> None:
            if is_spike:
                observed_spikes.append(value.detach())

        with torch.no_grad():
            result = model(torch.randn(2, 3, 8, 8), observer=observer)
        self.assertEqual(tuple(result.shape), (2, 10))
        self.assertGreater(len(observed_spikes), 0)
        for spike in observed_spikes:
            self.assertTrue(torch.all((spike == 0) | (spike == 1)).item())

    def test_calibration_observer_aggregates_without_feature_storage(self) -> None:
        observer = CalibrationObserver()
        observer("analog", torch.tensor([[-1.0, 1.0], [2.0, 0.0]]), False)
        observer("analog", torch.tensor([[3.0, 1.0]]), False)
        report = observer.as_dict()["analog"]
        self.assertEqual(report["elements"], 6)
        self.assertEqual(report["batches"], 2)
        self.assertEqual(report["minimum"], -1.0)
        self.assertEqual(report["maximum"], 3.0)
        self.assertAlmostEqual(report["mean"], 1.0)
        self.assertAlmostEqual(report["nonzero_fraction"], 5.0 / 6.0)

    def test_hardware_emulation_quantizes_all_learned_operators(self) -> None:
        torch.manual_seed(17)
        model = Spikingformer(
            time_steps=2,
            input_channels=3,
            classes=10,
            embed_dim=24,
            heads=3,
            depth=1,
            mlp_ratio=2,
        ).eval()
        quantized, report = prepare_hardware_emulation(
            model, fractional_bits=8, weight_granularity="tensor")
        self.assertEqual(len(report.quantized_weights), 12)
        self.assertTrue(all(item.integer_minimum >= -127 for item in report.quantized_weights))
        self.assertTrue(all(item.integer_maximum <= 127 for item in report.quantized_weights))
        image = torch.randn(2, 3, 8, 8)
```

```
        image_quantized, scale, zero_point = quantize_cifar_input_u8(image)
        self.assertGreater(scale, 0.0)
        self.assertTrue(0 <= zero_point <= 255)
        self.assertEqual(report.head_activation_scale, 0.125)
        self.assertGreater(report.head_accumulator_scale, 0.0)
        with torch.no_grad():
            result = quantized(image_quantized)
        self.assertEqual(tuple(result.shape), (2, 10))
        fixed = fake_fixed_point(torch.tensor([-1.1, 0.1, 1.1]), 4)
        torch.testing.assert_close(fixed * 16, torch.round(fixed * 16))


if __name__ == "__main__":
    unittest.main()
```

74.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import unittest` This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import torch` This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import torch.nn as nn` This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 8: `from spikingformer_reference import (` This statement continues through source line 12; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `MultiStepHardResetLIF,` This is continuation line 2 of the 5-line logical statement that begins at source line 8. Its exact fragment is ‘MultiStepHardResetLIF,‘. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `Spikingformer,` This is continuation line 3 of the 5-line logical statement that begins at source line 8. Its exact fragment is ‘Spikingformer,‘. The important operands are Spikingformer: the value named ’spikingformer’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `fold_batch_norm,` This is continuation line 4 of the 5-line logical statement that begins at source line 8. Its exact fragment is ‘fold_batch_norm,‘. The important operands are fold_batch_norm: the value named ’fold batch norm’ in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 8. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `from evaluate_spikingformer import CalibrationObserver` This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `from spikingformer_quantization import (` This statement continues through source line 19; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `FixedPointHardResetLIF,` This is continuation line 2 of the 6-line logical statement that begins at source line 14. Its exact fragment is 'FixedPointHardResetLIF,'. The important operands are FixedPointHardResetLIF: the value named 'fixed point hard reset lif' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `fake_fixed_point,` This is continuation line 3 of the 6-line logical statement that begins at source line 14. Its exact fragment is 'fake_fixed_point,'. The important operands are fake_fixed_point: the value named 'fake fixed point' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `prepare_hardware_emulation,` This is continuation line 4 of the 6-line logical statement that begins at source line 14. Its exact fragment is 'prepare_hardware_emulation,'. The important operands are prepare_hardware_emulation: the value named 'prepare hardware emulation' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `quantize_cifar_input_u8,` This is continuation line 5 of the 6-line logical statement that begins at source line 14. Its exact fragment is 'quantize_cifar_input_u8,'. The important operands are quantize_cifar_input_u8: the value named 'quantize cifar input u8' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `)` This is continuation line 6 of the 6-line logical statement that begins at source line 14. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer reference behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `class MultiStepLIFTests(unittest.TestCase):` This defines class MultiStepLIFTests, which groups data and methods for multi step liftests. Instances exist in host memory and organize the flow that checks spikingformer reference behavior independently with Python assertions.

Line 23: `def test_hard_reset_sequence(self) -> None:` This starts function test_hard_reset_sequence. Its indented body performs test hard reset sequence as part of the file that checks spikingformer reference behavior independently with Python assertions; it runs only when called.

Line 24: `neuron = MultiStepHardResetLIF(tau=2.0, threshold=1.0)` This assigns ‘MultiStepHardResetLIF(tau=2.0, threshold=1.0)‘ to ‘neuron ‘ in host memory. Here neuron means the value named ’neuron’ in this block. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block; tau: the value named ’tau’ in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function test_hard_reset_sequence, which handles test hard reset sequence.

Line 25: `sequence = torch.tensor([[1.0], [1.0], [1.0], [2.0], [1.0]])` This assigns ‘torch.tensor([[1.0], [1.0], [1.0], [2.0], [1.0]])‘ to ‘sequence ‘ in host memory. Here sequence means the value named ’sequence’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block. Context: the function test_hard_reset_sequence, which handles test hard reset sequence.

Line 26: `expected = torch.tensor([[0.0], [0.0], [0.0], [1.0], [0.0]])` This assigns ‘torch.tensor([[0.0], [0.0], [0.0], [1.0], [0.0]])‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block. Context: the function test_hard_reset_sequence, which handles test hard reset sequence.

Line 27: `torch.testing.assert_close(neuron(sequence), expected)` This calls ‘torch.testing.assert_close‘ with ‘neuron(sequence), expected‘ inside the function test_hard_reset_sequence, which handles test hard reset sequence. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 29: `def test_fixed_point_lif_matches_rtl_step_order(self) -> None:` This starts function test_fixed_point_lif_m
Its indented body performs test fixed point lif matches rtl step order as part of the file that checks spikingformer reference behavior independently with Python assertions; it runs only when called.

Line 30: `neuron = FixedPointHardResetLIF(threshold=1.0, fractional_bits=4)` This assigns ‘FixedPointHardResetLIF(t
fractional_bits=4)‘ to ‘neuron ‘ in host memory. Here neuron means the value named ’neuron’ in this block. The important operands are FixedPointHardResetLIF: the value named ’fixed point hard reset lif’ in this block; threshold: the membrane level at which a neuron emits a spike; fractional_bits: the value named ’fractional bits’ in this block. Context: the function test_fixed_point_lif_matches_rtl_step_order, which handles test fixed point lif matches rtl step order.

Line 31: `sequence = torch.tensor([[1.0], [1.0], [1.0], [2.0], [1.0]])` This assigns ‘torch.tensor([[1.0], [1.0], [1.0], [2.0], [1.0]])‘ to ‘sequence ‘ in host memory. Here sequence means the value named ’sequence’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block. Context: the function test_fixed_point_lif_matches_rtl_step_order, which handles test fixed point lif matches rtl step order.

Line 32: `expected = torch.tensor([[0.0], [0.0], [0.0], [1.0], [0.0]])` This assigns ‘torch.tensor([[0.0], [0.0], [0.0], [1.0], [0.0]])‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block. Context: the function test_fixed_point_lif_matches_rtl_step_order, which handles test fixed point lif matches rtl step order.

Line 33: `torch.testing.assert_close(neuron(sequence), expected)` This calls ‘torch.testing.assert_close‘ with ‘neuron(sequence), expected‘ inside the function test_fixed_point_lif_matches_rtl_step_order, which handles test fixed point lif matches rtl step order. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 35: `def test_attention_threshold_is_inclusive(self) -> None:` This starts function test_attention_threshold_is_i
Its indented body performs test attention threshold is inclusive as part of the file that checks spikingformer reference behavior independently with Python assertions; it runs only when called.

Line 36: `neuron = MultiStepHardResetLIF(tau=2.0, threshold=0.5)` This assigns ‘MultiStepHardResetLIF(tau=2.0, threshold=0.5)‘ to ‘neuron ‘ in host memory. Here neuron means the value named ’neuron’ in this block. The important operands are MultiStepHardResetLIF: the value named ’multi step hard reset lif’ in this block; tau: the value

named 'tau' in this block; threshold: the membrane level at which a neuron emits a spike. Context: the function test_attention_threshold_is_inclusive, which handles test attention threshold is inclusive.

Line 37: `sequence = torch.tensor([[1.0], [0.0]])` This assigns 'torch.tensor([[1.0], [0.0]])' to 'sequence ' in host memory. Here sequence means the value named 'sequence' in this block. The important operands are torch: the value named 'torch' in this block; tensor: the value named 'tensor' in this block. Context: the function test_attention_threshold_is_inclus which handles test attention threshold is inclusive.

Line 38: `expected = torch.tensor([[1.0], [0.0]])` This assigns 'torch.tensor([[1.0], [0.0]])' to 'expected ' in host memory. Here expected means the value named 'expected' in this block. The important operands are torch: the value named 'torch' in this block; tensor: the value named 'tensor' in this block. Context: the function test_attention_threshold_is_inclus which handles test attention threshold is inclusive.

Line 39: `torch.testing.assert_close(neuron(sequence), expected)` This calls 'torch.testing.assert_close' with 'neuron(sequence), expected' inside the function test_attention_threshold_is_inclusive, which handles test attention threshold is inclusive. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 42: `class BatchNormFoldingTests(unittest.TestCase):` This defines class BatchNormFoldingTests, which groups data and methods for batch norm folding tests. Instances exist in host memory and organize the flow that checks spikingformer reference behavior independently with Python assertions.

Line 43: `def test_conv2d_fold_matches_eval_graph(self) -> None:` This starts function test_conv2d_fold_matches_eval Its indented body performs test conv2d fold matches eval graph as part of the file that checks spikingformer reference behavior independently with Python assertions; it runs only when called.

Line 44: `torch.manual_seed(7)` This calls 'torch.manual_seed' with '7' inside the function test_conv2d_fold_matches_eval_gr which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 45: `convolution = nn.Conv2d(3, 5, kernel_size=3, padding=1, bias=False)` This assigns 'nn.Conv2d(3, 5, kernel_size=3, padding=1, bias=False)' to 'convolution ' in host memory. Here convolution means the value named 'convolution' in this block. The important operands are nn: the value named 'nn' in this block; Conv2d: the value named 'conv2d' in this block; kernel_size: the value named 'kernel size' in this block. Context: the function test_conv2d_fold_matches_eva which handles test conv2d fold matches eval graph.

Line 46: `batch_norm = nn.BatchNorm2d(5)` This assigns 'nn.BatchNorm2d(5)' to 'batch_norm ' in host memory. Here batch_norm means the value named 'batch norm' in this block. The important operands are nn: the value named 'nn' in this block; BatchNorm2d: the value named 'batch norm2d' in this block. Context: the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph.

Line 47: `batch_norm.running_mean.copy_(torch.randn(5))` This calls 'batch_norm.running_mean.copy_' with 'torch.randn(5)' inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 48: `batch_norm.running_var.copy_(torch.rand(5) + 0.2)` This calls 'batch_norm.running_var.copy_' with 'torch.rand(5) + 0.2' inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 49: `batch_norm.weight.data.copy_(torch.randn(5))` This calls 'batch_norm.weight.data.copy_' with 'torch.randn(5)' inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 50: `batch_norm.bias.data.copy_(torch.randn(5))` This calls ‘batch_norm.bias.data.copy_‘ with ‘torch.randn(5)‘ inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 51: `convolution.eval()` This calls ‘convolution.eval‘ with “ inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 52: `batch_norm.eval()` This calls ‘batch_norm.eval‘ with “ inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 53: `image = torch.randn(2, 3, 7, 7)` This assigns ‘torch.randn(2, 3, 7, 7)‘ to ‘image ‘ in host memory. Here image means the value named ’image’ in this block. The important operands are torch: the value named ’torch’ in this block; randn: the value named ’randn’ in this block. Context: the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph.

Line 55: `folded_weight, folded_bias = fold_batch_norm(convolution, batch_norm)` This assigns ‘fold_batch_norm(con batch_norm)‘ to ‘folded_weight, folded_bias ‘ in host memory. Here folded_weight means a learned coefficient multiplied by an activation. The important operands are fold_batch_norm: the value named ’fold batch norm’ in this block; convolution: the value named ’convolution’ in this block; batch_norm: the value named ’batch norm’ in this block. Context: the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph.

Line 56: `folded = nn.Conv2d(3, 5, kernel_size=3, padding=1, bias=True)` This assigns ‘nn.Conv2d(3, 5, kernel_size=3, padding=1, bias=True)‘ to ‘folded ‘ in host memory. Here folded means the value named ’folded’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv2d: the value named ’conv2d’ in this block; kernel_size: the value named ’kernel size’ in this block. Context: the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph.

Line 57: `folded.weight.data.copy_(folded_weight)` This calls ‘folded.weight.data.copy_‘ with ‘folded_weight‘ inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 58: `folded.bias.data.copy_(folded_bias)` This calls ‘folded.bias.data.copy_‘ with ‘folded_bias‘ inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 59: `folded.eval()` This calls ‘folded.eval‘ with “ inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 61: `torch.testing.assert_close(` This statement continues through source line 62; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘torch.testing.assert_close‘ with ‘ folded(image), batch_norm(convolution(image)), rtol=2e-5, atol=2e-6‘ inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 62: `folded(image), batch_norm(convolution(image)), rtol=2e-5, atol=2e-6)` This is continuation line 2 of the 2-line logical statement that begins at source line 61. Its exact fragment is ‘folded(image), batch_norm(convolution(image)), rtol=2e-5, atol=2e-6)‘. The important operands are folded: the value named ’folded’ in this block; image: the value named ’image’ in this block; batch_norm: the value named ’batch norm’ in this block. Read together, the complete operation is: This calls ‘torch.testing.assert_close‘ with ‘ folded(image), batch_norm(convolution(image)), rtol=2e-5, atol=2e-6‘ inside the function test_conv2d_fold_matches_eval_graph, which handles test conv2d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 64: `def test_conv1d_fold_matches_eval_graph(self) -> None:` This starts function test_conv1d_fold_matches_eval
Its indented body performs test conv1d fold matches eval graph as part of the file that checks spikingformer reference behavior independently with Python assertions; it runs only when called.

Line 65: `torch.manual_seed(11)` This calls ‘torch.manual_seed‘ with ‘11‘ inside the function test_conv1d_fold_matches_eval_
which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 66: `convolution = nn.Conv1d(4, 6, kernel_size=1, bias=True)` This assigns ‘nn.Conv1d(4, 6, kernel_size=1, bias=True)‘ to ‘convolution ‘ in host memory. Here convolution means the value named ’convolution’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv1d: the value named ’conv1d’ in this block; kernel_size: the value named ’kernel size’ in this block. Context: the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph.

Line 67: `batch_norm = nn.BatchNorm1d(6)` This assigns ‘nn.BatchNorm1d(6)‘ to ‘batch_norm ‘ in host memory. Here batch_norm means the value named ’batch norm’ in this block. The important operands are nn: the value named ’nn’ in this block; BatchNorm1d: the value named ’batch norm1d’ in this block. Context: the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph.

Line 68: `batch_norm.running_mean.copy_(torch.randn(6))` This calls ‘batch_norm.running_mean.copy_‘ with ‘torch.randn(6)‘ inside the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 69: `batch_norm.running_var.copy_(torch.rand(6) + 0.2)` This calls ‘batch_norm.running_var.copy_‘ with ‘torch.rand(6) + 0.2‘ inside the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 70: `convolution.eval()` This calls ‘convolution.eval‘ with “ inside the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 71: `batch_norm.eval()` This calls ‘batch_norm.eval‘ with “ inside the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 72: `values = torch.randn(3, 4, 9)` This assigns ‘torch.randn(3, 4, 9)‘ to ‘values ‘ in host memory. Here values means the value named ’values’ in this block. The important operands are torch: the value named ’torch’ in this block; randn: the value named ’randn’ in this block. Context: the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph.

Line 74: `folded_weight, folded_bias = fold_batch_norm(convolution, batch_norm)` This assigns ‘fold_batch_norm(con
batch_norm)‘ to ‘folded_weight, folded_bias ‘ in host memory. Here folded_weight means a learned coefficient multiplied by an activation. The important operands are fold_batch_norm: the value named ’fold batch norm’ in this block; convolution: the value named ’convolution’ in this block; batch_norm: the value named ’batch norm’ in this block. Context: the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph.

Line 75: `folded = nn.Conv1d(4, 6, kernel_size=1, bias=True)` This assigns ‘nn.Conv1d(4, 6, kernel_size=1, bias=True)‘ to ‘folded ‘ in host memory. Here folded means the value named ’folded’ in this block. The important operands are nn: the value named ’nn’ in this block; Conv1d: the value named ’conv1d’ in this block; kernel_size: the value named ’kernel size’ in this block. Context: the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph.

Line 76: `folded.weight.data.copy_(folded_weight)` This calls ‘folded.weight.data.copy_‘ with ‘folded_weight‘ inside the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 77: `folded.bias.data.copy_(folded_bias)` This calls ‘folded.bias.data.copy_‘ with ‘folded_bias‘ inside the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 78: `folded.eval()` This calls ‘folded.eval‘ with “ inside the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 80: `torch.testing.assert_close(` This statement continues through source line 81; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘torch.testing.assert_close‘ with ‘ folded(values), batch_norm(convolution(values)), rtol=2e-5, atol=2e-6‘ inside the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 81: `folded(values), batch_norm(convolution(values)), rtol=2e-5, atol=2e-6)` This is continuation line 2 of the 2-line logical statement that begins at source line 80. Its exact fragment is ‘folded(values), batch_norm(convolution(values)), rtol=2e-5, atol=2e-6)‘. The important operands are folded: the value named ’folded’ in this block; values: the value named ’values’ in this block; batch_norm: the value named ’batch norm’ in this block. Read together, the complete operation is: This calls ‘torch.testing.assert_close‘ with ‘ folded(values), batch_norm(convolution(values)), rtol=2e-5, atol=2e-6‘ inside the function test_conv1d_fold_matches_eval_graph, which handles test conv1d fold matches eval graph. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 84: `class ReferenceGraphTests(unittest.TestCase):` This defines class ReferenceGraphTests, which groups data and methods for reference graph tests. Instances exist in host memory and organize the flow that checks spikingformer reference behavior independently with Python assertions.

Line 85: `def test_small_graph_shape_and_binary_observations(self) -> None:` This starts function test_small_graph_s Its indented body performs test small graph shape and binary observations as part of the file that checks spikingformer reference behavior independently with Python assertions; it runs only when called.

Line 86: `torch.manual_seed(13)` This calls ‘torch.manual_seed‘ with ‘13‘ inside the function test_small_graph_shape_and_bi which handles test small graph shape and binary observations. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 87: `model = Spikingformer(` This statement continues through source line 95; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 88: `time_steps=2,` This is continuation line 2 of the 9-line logical statement that begins at source line 87. Its exact fragment is ‘time_steps=2,‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 89: `input_channels=3,` This is continuation line 3 of the 9-line logical statement that begins at source line 87. Its exact fragment is 'input_channels=3,'. The important operands are input_channels: the value named 'input channels' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 90: `classes=10,` This is continuation line 4 of the 9-line logical statement that begins at source line 87. Its exact fragment is 'classes=10,'. The important operands are classes: the value named 'classes' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 91: `embed_dim=24,` This is continuation line 5 of the 9-line logical statement that begins at source line 87. Its exact fragment is 'embed_dim=24,'. The important operands are embed_dim: the value named 'embed dim' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 92: `heads=3,` This is continuation line 6 of the 9-line logical statement that begins at source line 87. Its exact fragment is 'heads=3,'. The important operands are heads: the value named 'heads' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 93: `depth=1,` This is continuation line 7 of the 9-line logical statement that begins at source line 87. Its exact fragment is 'depth=1,'. The important operands are depth: the value named 'depth' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 94: `mlp_ratio=2,` This is continuation line 8 of the 9-line logical statement that begins at source line 87. Its exact fragment is 'mlp_ratio=2,'. The important operands are mlp_ratio: the value named 'mlp ratio' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 95: `).eval()` This is continuation line 9 of the 9-line logical statement that begins at source line 87. Its exact fragment is ').eval()'. The important operands are eval: the value named 'eval' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value

named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 96: `observed_spikes: list[torch.Tensor] = []` This assigns '[]' to 'observed_spikes' in host memory. Here observed_spikes means a binary event produced when a neuron crosses its threshold. The expression is a literal or structural operator and has no hidden software state. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 98: `def observer(_name: str, value: torch.Tensor, is_spike: bool) -> None:` This starts function observer. Its indented body performs observer as part of the file that checks spikingformer reference behavior independently with Python assertions; it runs only when called.

Line 99: `if is_spike:` This executes the following indented host-code branch only when 'is_spike' is true. The important operands are is_spike: a binary event produced when a neuron crosses its threshold. Context: the function observer, which handles observer.

Line 100: `observed_spikes.append(value.detach())` This calls 'observed_spikes.append' with 'value.detach()' inside the function observer, which handles observer. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 102: `with torch.no_grad():` This opens a managed resource scope for 'torch.no_grad()' so files, tracing, or temporary state are finalized even if the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations raises an error.

Line 103: `result = model(torch.randn(2, 3, 8, 8), observer=observer)` This assigns 'model(torch.randn(2, 3, 8, 8), observer=observer)' to 'result ' in host memory. Here result means the value named 'result' in this block. The important operands are model: the value named 'model' in this block; torch: the value named 'torch' in this block; randn: the value named 'randn' in this block. Context: the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations.

Line 104: `self.assertEqual(tuple(result.shape), (2, 10))` This calls 'self.assertEqual' with 'tuple(result.shape), (2, 10)' inside the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 105: `self.assertGreater(len(observed_spikes), 0)` This calls 'self.assertGreater' with 'len(observed_spikes), 0' inside the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 106: `for spike in observed_spikes:` This iterates spike over 'observed_spikes'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_small_graph_shape_and_binary_ob which handles test small graph shape and binary observations.

Line 107: `self.assertTrue(torch.all((spike == 0) | (spike == 1)).item())` This calls 'self.assertTrue' with 'torch.all((spike == 0) | (spike == 1)).item()' inside the function test_small_graph_shape_and_binary_observations, which handles test small graph shape and binary observations. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 109: `def test_calibration_observer_aggregates_without_feature_storage(self) -> None:` This starts function test_calibration_observer_aggregates_without_feature_storage. Its indented body performs test calibration observer aggregates without feature storage as part of the file that checks spikingformer reference behavior independently with Python assertions; it runs only when called.

Line 110: `observer = CalibrationObserver()` This assigns ‘CalibrationObserver()‘ to ‘observer ‘ in host memory. Here observer means the value named ’observer’ in this block. The important operands are CalibrationObserver: the value named ’calibration observer’ in this block. Context: the function test_calibration_observer_aggregates_without_feature_storage, which handles test calibration observer aggregates without feature storage.

Line 111: `observer("analog", torch.tensor([[-1.0, 1.0], [2.0, 0.0]]), False)` This calls ‘observer‘ with ‘"analog", torch.tensor([[-1.0, 1.0], [2.0, 0.0]]), False‘ inside the function test_calibration_observer_aggregates_without_feature_sto which handles test calibration observer aggregates without feature storage. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 112: `observer("analog", torch.tensor([[3.0, 1.0]]), False)` This calls ‘observer‘ with ‘"analog", torch.tensor([[3.0, 1.0]]), False‘ inside the function test_calibration_observer_aggregates_without_feature_storage, which handles test calibration observer aggregates without feature storage. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 113: `report = observer.as_dict()["analog"]` This assigns ‘observer.as_dict()["analog"]‘ to ‘report ‘ in host memory. Here report means the value named ’report’ in this block. The important operands are observer: the value named ’observer’ in this block; as_dict: the value named ’as dict’ in this block; analog: the value named ’analog’ in this block. Context: the function test_calibration_observer_aggregates_without_feature_storage, which handles test calibration observer aggregates without feature storage.

Line 114: `self.assertEqual(report["elements"], 6)` This calls ‘self.assertEqual‘ with ‘report["elements"], 6‘ inside the function test_calibration_observer_aggregates_without_feature_storage, which handles test calibration observer aggregates without feature storage. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 115: `self.assertEqual(report["batches"], 2)` This calls ‘self.assertEqual‘ with ‘report["batches"], 2‘ inside the function test_calibration_observer_aggregates_without_feature_storage, which handles test calibration observer aggregates without feature storage. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 116: `self.assertEqual(report["minimum"], -1.0)` This calls ‘self.assertEqual‘ with ‘report["minimum"], -1.0‘ inside the function test_calibration_observer_aggregates_without_feature_storage, which handles test calibration observer aggregates without feature storage. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 117: `self.assertEqual(report["maximum"], 3.0)` This calls ‘self.assertEqual‘ with ‘report["maximum"], 3.0‘ inside the function test_calibration_observer_aggregates_without_feature_storage, which handles test calibration observer aggregates without feature storage. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 118: `self.assertAlmostEqual(report["mean"], 1.0)` This calls ‘self.assertAlmostEqual‘ with ‘report["mean"], 1.0‘ inside the function test_calibration_observer_aggregates_without_feature_storage, which handles test calibration observer aggregates without feature storage. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 119: `self.assertAlmostEqual(report["nonzero_fraction"], 5.0 / 6.0)` This calls ‘self.assertAlmostEqual‘ with ‘report["nonzero_fraction"], 5.0 / 6.0‘ inside the function test_calibration_observer_aggregates_without_feature_storage, which handles test calibration observer aggregates without feature storage. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 121: `def test_hardware_emulation_quantizes_all_learned_operators(self) -> None:` This starts function test_hardware_emulation_quantizes_all_learned_operators. Its indented body performs test hardware emulation quantizes all learned operators as part of the file that checks spikingformer reference behavior independently with Python assertions; it runs only when called.

Line 122: `torch.manual_seed(17)` This calls ‘torch.manual_seed‘ with ‘17‘ inside the function test_hardware_emulation_quant which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 123: `model = Spikingformer(` This statement continues through source line 131; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 124: `time_steps=2,` This is continuation line 2 of the 9-line logical statement that begins at source line 123. Its exact fragment is ‘time_steps=2,‘. The important operands are time_steps: the value named ’time steps’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 125: `input_channels=3,` This is continuation line 3 of the 9-line logical statement that begins at source line 123. Its exact fragment is ‘input_channels=3,‘. The important operands are input_channels: the value named ’input channels’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 126: `classes=10,` This is continuation line 4 of the 9-line logical statement that begins at source line 123. Its exact fragment is ‘classes=10,‘. The important operands are classes: the value named ’classes’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 127: `embed_dim=24,` This is continuation line 5 of the 9-line logical statement that begins at source line 123. Its exact fragment is ‘embed_dim=24,‘. The important operands are embed_dim: the value named ’embed dim’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 128: `heads=3,` This is continuation line 6 of the 9-line logical statement that begins at source line 123. Its exact fragment is ‘heads=3,‘. The important operands are heads: the value named ’heads’ in this block. Read together, the complete operation is: This assigns ‘Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()‘ to ‘model ‘ in host memory. Here model means the value named ’model’ in this block. The important operands are Spikingformer: the value named ’spikingformer’ in this block; time_steps: the value named ’time steps’ in this block; input_channels: the value named ’input channels’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 129: `depth=1,` This is continuation line 7 of the 9-line logical statement that begins at source line 123. Its exact fragment is 'depth=1,'. The important operands are depth: the value named 'depth' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 130: `mlp_ratio=2,` This is continuation line 8 of the 9-line logical statement that begins at source line 123. Its exact fragment is 'mlp_ratio=2,'. The important operands are mlp_ratio: the value named 'mlp ratio' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 131: `).eval()` This is continuation line 9 of the 9-line logical statement that begins at source line 123. Its exact fragment is ').eval()'. The important operands are eval: the value named 'eval' in this block. Read together, the complete operation is: This assigns 'Spikingformer(time_steps=2, input_channels=3, classes=10, embed_dim=24, heads=3, depth=1, mlp_ratio=2,).eval()' to 'model ' in host memory. Here model means the value named 'model' in this block. The important operands are Spikingformer: the value named 'spikingformer' in this block; time_steps: the value named 'time steps' in this block; input_channels: the value named 'input channels' in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 132: `quantized, report = prepare_hardware_emulation(` This statement continues through source line 133; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'prepare_hardware_emulation(model, fractional_bits=8, weight_granularity="tensor")' to 'quantized, report ' in host memory. Here quantized means the value named 'quantized' in this block. The important operands are prepare_hardware_emulation: the value named 'prepare hardware emulation' in this block; model: the value named 'model' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 133: `model, fractional_bits=8, weight_granularity="tensor")` This is continuation line 2 of the 2-line logical statement that begins at source line 132. Its exact fragment is 'model, fractional_bits=8, weight_granularity="tensor")'. The important operands are model: the value named 'model' in this block; fractional_bits: the value named 'fractional bits' in this block; weight_granularity: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'prepare_hardware_emulation(model, fractional_bits=8, weight_granularity="tensor")' to 'quantized, report ' in host memory. Here quantized means the value named 'quantized' in this block. The important operands are prepare_hardware_emulation: the value named 'prepare hardware emulation' in this block; model: the value named 'model' in this block; fractional_bits: the value named 'fractional bits' in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 134: `self.assertEqual(len(report.quantized_weights), 12)` This calls 'self.assertEqual' with 'len(report.quantized_w 12' inside the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 135: `self.assertTrue(all(item.integer_minimum >= -127 for item in report.quantized_weights))` This calls 'self.assertTrue' with 'all(item.integer_minimum >= -127 for item in report.quantized_weights)' inside the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 136: `self.assertTrue(all(item.integer_maximum <= 127 for item in report.quantized_weights))` This calls ‘self.assertTrue‘ with ‘all(item.integer_maximum <= 127 for item in report.quantized_weights)‘ inside the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 137: `image = torch.randn(2, 3, 8, 8)` This assigns ‘torch.randn(2, 3, 8, 8)‘ to ‘image ‘ in host memory. Here image means the value named ’image’ in this block. The important operands are torch: the value named ’torch’ in this block; randn: the value named ’randn’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_opera which handles test hardware emulation quantizes all learned operators.

Line 138: `image_quantized, scale, zero_point = quantize_cifar_input_u8(image)` This assigns ‘quantize_cifar_input_ to ‘image_quantized, scale, zero_point ‘ in host memory. Here image_quantized means the value named ’image quantized’ in this block. The important operands are quantize_cifar_input_u8: the value named ’quantize cifar input u8’ in this block; image: the value named ’image’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_ which handles test hardware emulation quantizes all learned operators.

Line 139: `self.assertGreater(scale, 0.0)` This calls ‘self.assertGreater‘ with ‘scale, 0.0‘ inside the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 140: `self.assertTrue(0 <= zero_point <= 255)` This calls ‘self.assertTrue‘ with ‘0 <= zero_point <= 255‘ inside the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 141: `self.assertEqual(report.head_activation_scale, 0.125)` This calls ‘self.assertEqual‘ with ‘report.head_activati 0.125‘ inside the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 142: `self.assertGreater(report.head_accumulator_scale, 0.0)` This calls ‘self.assertGreater‘ with ‘report.head_accumulator_scale, 0.0‘ inside the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 143: `with torch.no_grad():` This opens a managed resource scope for ‘torch.no_grad()‘ so files, tracing, or temporary state are finalized even if the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators raises an error.

Line 144: `result = quantized(image_quantized)` This assigns ‘quantized(image_quantized)‘ to ‘result ‘ in host memory. Here result means the value named ’result’ in this block. The important operands are quantized: the value named ’quantized’ in this block; image_quantized: the value named ’image quantized’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators.

Line 145: `self.assertEqual(tuple(result.shape), (2, 10))` This calls ‘self.assertEqual‘ with ‘tuple(result.shape), (2, 10)‘ inside the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 146: `fixed = fake_fixed_point(torch.tensor([-1.1, 0.1, 1.1]), 4)` This assigns ‘fake_fixed_point(torch.tensor([-1.1, 0.1, 1.1]), 4)‘ to ‘fixed ‘ in host memory. Here fixed means the value named ’fixed’ in this block. The important operands are fake_fixed_point: the value named ’fake fixed point’ in this block; torch: the value named ’torch’ in this block; tensor: the value named ’tensor’ in this block. Context: the function test_hardware_emulation_quantizes_all_learned_oper which handles test hardware emulation quantizes all learned operators.

Line 147: `torch.testing.assert_close(fixed * 16, torch.round(fixed * 16))` This calls ‘torch.testing.assert_close‘ with ‘fixed * 16, torch.round(fixed * 16)‘ inside the function test_hardware_emulation_quantizes_all_learned_operators, which handles test hardware emulation quantizes all learned operators. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

Line 150: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks spikingformer reference behavior independently with Python assertions.

Line 151: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks spikingformer reference behavior independently with Python assertions. The call advances the host-side job that checks spikingformer reference behavior independently with Python assertions.

75 tests/test_spikingformer_training_runtime.py

Role in the complete system: checks spikingformer training runtime behavior independently with Python assertions.

75.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks spikingformer training runtime behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: TinyParameterArchive, TinyFullGraphArchive, SpikingformerTrainingRuntimeTests. Functions: ___init___, projection_names, projection, head, ___init___, projection_names, projection, head, setUp, test_block_floating_shift_normalize_and_add_are_exact, test_projection_derivative_uses_exported_channel_scales, test_affine_backward_returns_exact_dx_dw_and_bias, test_hardware_softmax_and_batch_mean_cross_entropy_grad test_low_rate_bias_update_retains_q16_learning_rate_resolution, test_full_parameter_update_and_atomic_checkpoin test_complete_spikingformer_topology_forward_backward_and_update, add_conv, add_matrix. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

75.2 Complete source

```
from __future__ import annotations

import tempfile
import unittest
from pathlib import Path

import numpy as np

from neuraedge_golden import round_shift_signed
from spikingformer_parameters import HeadParameters, ProjectionParameters
from spikingformer_training_runtime import (
    BlockGradient,
    MasterTensor,
    ParameterGradient,
    TrainableSpikingformerState,
    SpikingformerOnDeviceTrainer,
    add_block_gradients,
    backward_requantize_projection,
    execute_affine_backward_block,
    execute_cross_entropy_gradient_q15,
    execute_power_of_two_scale,
    execute_softmax_q15,
    normalize_gradient_s16,
)
from tests.test_neuraedge_training import FullTrainingEmulator


class TinyParameterArchive:
    def __init__(self) -> None:
        self.manifest = {
            "binary_sha256": "a" * 64,
            "fixed_point": {
                "activation_fractional_bits": 10,
                "image_input_scale": 0.02,
                "image_input_zero_point": 123,
                "head_activation_scale": 0.125,
            },
        }
        self._projection = ProjectionParameters(
            "tiny",
            np.asarray([[3, -4], [-2, 5]], dtype=np.int8),
            np.asarray([7, -9], dtype=np.int32),
            np.asarray([16384, 24576], dtype=np.int16),
            np.asarray([-1, 1], dtype=np.int8),
            "COMPUTE_BINARY",
            (2, 2, 1),
```

```
            10,
        )
        self._head = HeadParameters(
            np.asarray([[6, -7], [8, 9]], dtype=np.int8),
            np.asarray([11, -13], dtype=np.int32),
        )

    @property
    def projection_names(self) -> tuple[str, ...]:
        return ("tiny",)

    def projection(self, name: str) -> ProjectionParameters:
        if name != "tiny":
            raise KeyError(name)
        return self._projection

    @property
    def head(self) -> HeadParameters:
        return self._head


class TinyFullGraphArchive:
    def __init__(self) -> None:
        self.manifest = {
            "binary_sha256": "b" * 64,
            "fixed_point": {
                "activation_fractional_bits": 10,
                "image_input_scale": 0.02,
                "image_input_zero_point": 123,
                "head_activation_scale": 0.125,
            },
            "head": {"accumulator_scale": 1.0 / 1024.0},
        }
        self._projections: dict[str, ProjectionParameters] = {}

        def add_conv(name: str, input_channels: int, output_channels: int, dense=False) -> None:
            reduction = input_channels * 3 * 3
            weight = np.fromfunction(
                lambda row, column: ((row + 2 * column) % 3) - 1,
                (reduction, output_channels),
                dtype=int,
            ).astype(np.int8)
            self._projections[name] = ProjectionParameters(
                name,
                weight,
                np.full(output_channels, 2200, dtype=np.int32),
                np.full(output_channels, 16384, dtype=np.int16),
                np.full(output_channels, -1, dtype=np.int8),
                "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY",
                (output_channels, input_channels, 3, 3),
                10,
            )

        add_conv("patch_embed.block0", 3, 2, dense=True)
        for index in range(1, 5):
            add_conv(f"patch_embed.block{index}", 2, 2)

        def add_matrix(name: str, inputs: int, outputs: int, bias: int = 2200) -> None:
            weight = np.fromfunction(
                lambda row, column: ((2 * row + column) % 3) - 1,
                (inputs, outputs),
                dtype=int,
            ).astype(np.int8)
            self._projections[name] = ProjectionParameters(
                name,
                weight,
                np.full(outputs, bias, dtype=np.int32),
                np.full(outputs, 16384, dtype=np.int16),
                np.full(outputs, -1, dtype=np.int8),
                "COMPUTE_BINARY",
                (outputs, inputs, 1),
                10,
            )

        for name in ("q", "k", "v"):
            add_matrix(f"block.0.attn.{name}", 2, 2)
        add_matrix("block.0.attn.proj", 2, 2, bias=0)
        add_matrix("block.0.mlp.fc1", 2, 3)
        add_matrix("block.0.mlp.fc2", 3, 2, bias=0)
        self._head = HeadParameters(
            np.asarray([[3, -2, 1], [-1, 2, 4]], dtype=np.int8),
            np.asarray([17, -11, 5], dtype=np.int32),
        )

    @property
    def projection_names(self) -> tuple[str, ...]:
        return tuple(self._projections)
```

```
    def projection(self, name: str) -> ProjectionParameters:
        return self._projections[name]

    @property
    def head(self) -> HeadParameters:
        return self._head


class SpikingformerTrainingRuntimeTests(unittest.TestCase):
    def setUp(self) -> None:
        self.driver = FullTrainingEmulator()

    def test_block_floating_shift_normalize_and_add_are_exact(self) -> None:
        values = np.asarray([[-2_000_000, -65537, 0, 65537, 2_000_000]], dtype=np.int64)
        shifted = execute_power_of_two_scale(self.driver, values, -4)
        np.testing.assert_array_equal(shifted, round_shift_signed(values, 4))
        normalized = normalize_gradient_s16(self.driver, BlockGradient(values, -15))
        self.assertGreater(normalized.exponent, -15)
        self.assertLessEqual(int(np.abs(normalized.values).max()), 32767)
        left = BlockGradient(np.asarray([100, -200, 300]), -12)
        right = BlockGradient(np.asarray([800, 400, -1200]), -15)
        added = add_block_gradients(self.driver, left, right)
        self.assertEqual(added.exponent, -12)
        np.testing.assert_array_equal(
            added.values,
            left.values + round_shift_signed(right.values, 3),
        )

    def test_projection_derivative_uses_exported_channel_scales(self) -> None:
        parameters = TinyParameterArchive().projection("tiny")
        gradient = BlockGradient(
            np.asarray([[31000, -27000], [-11000, 9000], [777, -333]], dtype=np.int64),
            -15,
        )
        actual = backward_requantize_projection(self.driver, gradient, parameters)
        boost = gradient.exponent - actual.exponent
        expected = np.empty_like(gradient.values)
        for channel in range(2):
            expected[:, channel] = round_shift_signed(
                gradient.values[:, channel] * int(parameters.multiplier_q15[channel]),
                15 + int(parameters.binary_point_shift[channel]) - boost,
            )
        np.testing.assert_array_equal(actual.values, expected)
        self.assertLessEqual(int(np.abs(actual.values).max()), 32767)

    def test_affine_backward_returns_exact_dx_dw_and_bias(self) -> None:
        x = np.asarray([[3, -2, 1], [-4, 5, 2], [7, 1, -3]], dtype=np.int64)
        w = np.asarray([[2, -1], [4, 3], [-5, 6]], dtype=np.int64)
        bias = np.asarray([9, -8], dtype=np.int64)
        dy = np.asarray([[11, -7], [5, 13], [-3, 2]], dtype=np.int64)
        dx, parameter = execute_affine_backward_block(
            self.driver, x, w, bias, BlockGradient(dy, -15))
        self.assertEqual(dx.exponent, -15)
        self.assertEqual(parameter.weight.exponent, -15)
        np.testing.assert_array_equal(dx.values, dy @ w.T)
        np.testing.assert_array_equal(parameter.weight.values, x.T @ dy)
        np.testing.assert_array_equal(parameter.bias.values, dy.sum(axis=0))

    def test_hardware_softmax_and_batch_mean_cross_entropy_gradient(self) -> None:
        logits = np.asarray([[1200, -300, 77], [-80, 900, 100]], dtype=np.int64)
        probabilities = execute_softmax_q15(
            self.driver, logits, difference_shift=8)
        gradient = execute_cross_entropy_gradient_q15(
            self.driver, probabilities, np.asarray([0, 2]))
        self.assertEqual(gradient.exponent, -15)
        expected = probabilities.copy()
        expected[0, 0] -= 1 << 15
        expected[1, 2] -= 1 << 15
        expected = round_shift_signed(expected, 1)
        np.testing.assert_array_equal(gradient.values, expected)

    def test_low_rate_bias_update_retains_q16_learning_rate_resolution(self) -> None:
        tensor = MasterTensor(
            np.asarray([0], dtype=np.int16),
            np.asarray([0], dtype=np.int32),
            2,
            -(1 << 15),
            (1 << 15) - 1,
        )
        coefficients = TrainableSpikingformerState._optimizer_coefficients(
            tensor, 0.01, 0.9, 0.0)
        self.assertEqual(coefficients, (655, 58982, 0, 29))
        with self.assertRaises(ValueError):
            TrainableSpikingformerState._optimizer_coefficients(
                tensor, 1.0e-7, 0.9, 0.0)

```

```
    def test_full_parameter_update_and_atomic_checkpoint_round_trip(self) -> None:
        state = TrainableSpikingformerState(TinyParameterArchive())
        projection = state.projection("tiny")
        head = state.head
        gradients = {
            "tiny": ParameterGradient(
                BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15),
                BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),
            ),
            "head": ParameterGradient(
                BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15),
                BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),
            ),
        }
        state.apply_gradients(
            self.driver,
            gradients,
            learning_rate=0.25,
            momentum=0.0,
            weight_decay=0.0,
        )
        self.assertEqual(state.optimizer_step, 1)
        self.assertFalse(np.array_equal(state.projection("tiny").weight_k_by_n,
                                        projection.weight_k_by_n))
        self.assertFalse(np.array_equal(state.head.weight_k_by_n, head.weight_k_by_n))

        with tempfile.TemporaryDirectory() as temporary:
            checkpoint = Path(temporary) / "latest_checkpoint.npz"
            digest = state.save_checkpoint(
                checkpoint,
                epoch=3,
                batch_index=17,
                best_accuracy_percent=95.75,
                extra={"reason": "unit-test"},
            )
            self.assertEqual(len(digest), 64)
            restored = TrainableSpikingformerState(TinyParameterArchive())
            metadata = restored.load_checkpoint(checkpoint)
            self.assertEqual(metadata["epoch"], 3)
            self.assertEqual(metadata["batch_index"], 17)
            self.assertEqual(restored.optimizer_step, 1)
            np.testing.assert_array_equal(
                restored.projection("tiny").weight_k_by_n,
                state.projection("tiny").weight_k_by_n,
            )
            np.testing.assert_array_equal(restored.head.weight_k_by_n, state.head.weight_k_by_n)

    def test_complete_spikingformer_topology_forward_backward_and_update(self) -> None:
        state = TrainableSpikingformerState(TinyFullGraphArchive())
        trainer = SpikingformerOnDeviceTrainer(
            state,
            self.driver,
            transformer_blocks=1,
            attention_heads=1,
            timesteps=2,
            input_shape=(3, 8, 8),
        )
        self.assertEqual(trainer.attention_scale_shift, 7)
        image = np.zeros((1, 3, 8, 8), dtype=np.float32)
        cache = trainer.forward(image)
        self.assertEqual(cache.final_value.shape, (2, 1, 2, 2, 2))
        self.assertEqual(len(cache.tokenizer), 5)
        self.assertEqual(len(cache.blocks), 1)
        self.assertEqual(cache.logits_accumulator.shape, (1, 3))
        backward = trainer.backward(cache, np.asarray([1]))
        self.assertTrue(np.isfinite(backward.loss))
        self.assertEqual(backward.input_gradient.values.shape, image.shape)
        self.assertEqual(
            set(backward.parameter_gradients),
            set(state.projection_names) | {"head"},
        )
        nonzero_gradients = 0
        for name, gradient in backward.parameter_gradients.items():
            expected_weight = state.head.weight_k_by_n.shape if name == "head" else (
                state.projection(name).weight_k_by_n.shape)
            expected_bias = state.head.bias_accumulator_units.shape if name == "head" else (
                state.projection(name).bias_q10.shape)
            self.assertEqual(gradient.weight.values.shape, expected_weight)
            self.assertEqual(gradient.bias.values.shape, expected_bias)
            nonzero_gradients += int(np.count_nonzero(gradient.weight.values))
            nonzero_gradients += int(np.count_nonzero(gradient.bias.values))
        self.assertGreater(nonzero_gradients, 0)
        state.apply_gradients(
            self.driver,
            backward.parameter_gradients,
            learning_rate=0.25,
            momentum=0.0,
```

```
            weight_decay=0.0,
        )
        self.assertEqual(state.optimizer_step, 1)


if __name__ == "__main__":
    unittest.main()
```

75.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import tempfile` This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 4: `import unittest` This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `from pathlib import Path` This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `import numpy as np` This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `from neuraedge_golden import round_shift_signed` This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 10: `from spikingformer_parameters import HeadParameters, ProjectionParameters` This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `from spikingformer_training_runtime import (` This statement continues through source line 24; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `BlockGradient,` This is continuation line 2 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'BlockGradient,'. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `MasterTensor,` This is continuation line 3 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'MasterTensor,'. The important operands are MasterTensor: the value named 'master tensor' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `ParameterGradient,` This is continuation line 4 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'ParameterGradient,'. The important operands are ParameterGradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `TrainableSpikingformerState,` This is continuation line 5 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'TrainableSpikingformerState,'. The important operands are TrainableSpikingformerState: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 16: `SpikingformerOnDeviceTrainer,` This is continuation line 6 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'SpikingformerOnDeviceTrainer,'. The important operands are SpikingformerOnDeviceTrainer: the value named 'spikingformer on device trainer' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 17: `add_block_gradients,` This is continuation line 7 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'add_block_gradients,'. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `backward_requantize_projection,` This is continuation line 8 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'backward_requantize_projection,'. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 19: `execute_affine_backward_block,` This is continuation line 9 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'execute_affine_backward_block,'. The important operands are execute_affine_backward_block: the value named 'execute affine backward block' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 20: `execute_cross_entropy_gradient_q15,` This is continuation line 10 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'execute_cross_entropy_gradient_q15,'. The important operands are execute_cross_entropy_gradient_q15: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 21: `execute_power_of_two_scale,` This is continuation line 11 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'execute_power_of_two_scale,'. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 22: `execute_softmax_q15,` This is continuation line 12 of the 14-line logical statement that begins at source line 11. Its exact fragment is 'execute_softmax_q15,'. The important operands are execute_softmax_q15: the value named 'execute softmax q15' in this block. Read together, the complete operation is: This imports the named Python API so

the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 23: `normalize_gradient_s16,` This is continuation line 13 of the 14-line logical statement that begins at source line 11. Its exact fragment is ‘normalize_gradient_s16,‘. The important operands are normalize_gradient_s16: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 24: `)` This is continuation line 14 of the 14-line logical statement that begins at source line 11. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 25: `from tests.test_neuraedge_training import FullTrainingEmulator` This imports the named Python API so the file whose role is to checks spikingformer training runtime behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 28: `class TinyParameterArchive:` This defines class TinyParameterArchive, which groups data and methods for tiny parameter archive. Instances exist in host memory and organize the flow that checks spikingformer training runtime behavior independently with Python assertions.

Line 29: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 30: `self.manifest = {` This statement continues through source line 38; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "binary_sha256": "a" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, }‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block; fixed_point: the value named ’fixed point’ in this block. Context: the function ___init___, which handles init.

Line 31: `"binary_sha256": "a" * 64,` This is continuation line 2 of the 9-line logical statement that begins at source line 30. Its exact fragment is ‘"binary_sha256": "a" * 64,‘. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block. Read together, the complete operation is: This assigns ‘{ "binary_sha256": "a" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, }‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block; fixed_point: the value named ’fixed point’ in this block. Context: the function ___init___, which handles init.

Line 32: `"fixed_point": {` This is continuation line 3 of the 9-line logical statement that begins at source line 30. Its exact fragment is ‘"fixed_point": {‘. The important operands are fixed_point: the value named ’fixed point’ in this block. Read together, the complete operation is: This assigns ‘{ "binary_sha256": "a" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, }‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block; fixed_point: the value named ’fixed point’ in this block. Context: the function ___init___, which handles init.

Line 33: `"activation_fractional_bits": 10,` This is continuation line 4 of the 9-line logical statement that begins at source line 30. Its exact fragment is ‘"activation_fractional_bits": 10,‘. The important operands are activation_fractional_bits: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘{ "binary_sha256": "a" * 64, "fixed_point": { "activation_fractional_bits": 10,

"image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, }‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block; fixed_point: the value named ’fixed point’ in this block. Context: the function ___init___, which handles init.

Line 34: `"image_input_scale": 0.02,` This is continuation line 5 of the 9-line logical statement that begins at source line 30. Its exact fragment is ‘"image_input_scale": 0.02,‘. The important operands are image_input_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns ‘{ "binary_sha256": "a" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, }‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block; fixed_point: the value named ’fixed point’ in this block. Context: the function ___init___, which handles init.

Line 35: `"image_input_zero_point": 123,` This is continuation line 6 of the 9-line logical statement that begins at source line 30. Its exact fragment is ‘"image_input_zero_point": 123,‘. The important operands are image_input_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns ‘{ "binary_sha256": "a" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, }‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block; fixed_point: the value named ’fixed point’ in this block. Context: the function ___init___, which handles init.

Line 36: `"head_activation_scale": 0.125,` This is continuation line 7 of the 9-line logical statement that begins at source line 30. Its exact fragment is ‘"head_activation_scale": 0.125,‘. The important operands are head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns ‘{ "binary_sha256": "a" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, }‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block; fixed_point: the value named ’fixed point’ in this block. Context: the function ___init___, which handles init.

Line 37: `},` This is continuation line 8 of the 9-line logical statement that begins at source line 30. Its exact fragment is ‘},‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "binary_sha256": "a" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, }‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block; fixed_point: the value named ’fixed point’ in this block. Context: the function ___init___, which handles init.

Line 38: `}` This is continuation line 9 of the 9-line logical statement that begins at source line 30. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "binary_sha256": "a" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, }‘ to ‘self.manifest ‘ in host memory. Here manifest means the value named ’manifest’ in this block. The important operands are binary_sha256: the value named ’binary sha256’ in this block; a: the value named ’a’ in this block; fixed_point: the value named ’fixed point’ in this block. Context: the function ___init___, which handles init.

Line 39: `self._projection = ProjectionParameters(` This statement continues through source line 48; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)‘ to ‘self._projection ‘ in host memory. Here _projection means the value named ’projection’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; tiny: the value named ’tiny’ in this block; asarray: the value named ’asarray’ in this block. Context: the function ___init___, which handles init.

Line 40: `"tiny",` This is continuation line 2 of the 10-line logical statement that begins at source line 39. Its exact fragment is '"tiny",'. The important operands are tiny: the value named 'tiny' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)' to 'self._projection ' in host memory. Here _projection means the value named 'projection' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; tiny: the value named 'tiny' in this block; asarray: the value named 'asarray' in this block. Context: the function ___init___, which handles init.

Line 41: `np.asarray([[3, -4], [-2, 5]], dtype=np.int8),` This is continuation line 3 of the 10-line logical statement that begins at source line 39. Its exact fragment is 'np.asarray([[3, -4], [-2, 5]], dtype=np.int8),'. The important operands are asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block; int8: the value named 'int8' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)' to 'self._projection ' in host memory. Here _projection means the value named 'projection' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; tiny: the value named 'tiny' in this block; asarray: the value named 'asarray' in this block. Context: the function ___init___, which handles init.

Line 42: `np.asarray([7, -9], dtype=np.int32),` This is continuation line 4 of the 10-line logical statement that begins at source line 39. Its exact fragment is 'np.asarray([7, -9], dtype=np.int32),'. The important operands are asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block; int32: the value named 'int32' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)' to 'self._projection ' in host memory. Here _projection means the value named 'projection' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; tiny: the value named 'tiny' in this block; asarray: the value named 'asarray' in this block. Context: the function ___init___, which handles init.

Line 43: `np.asarray([16384, 24576], dtype=np.int16),` This is continuation line 5 of the 10-line logical statement that begins at source line 39. Its exact fragment is 'np.asarray([16384, 24576], dtype=np.int16),'. The important operands are asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block; int16: the value named 'int16' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)' to 'self._projection ' in host memory. Here _projection means the value named 'projection' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; tiny: the value named 'tiny' in this block; asarray: the value named 'asarray' in this block. Context: the function ___init___, which handles init.

Line 44: `np.asarray([-1, 1], dtype=np.int8),` This is continuation line 6 of the 10-line logical statement that begins at source line 39. Its exact fragment is 'np.asarray([-1, 1], dtype=np.int8),'. The important operands are asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block; int8: the value named 'int8' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)' to 'self._projection ' in host memory. Here _projection means the value named 'projection' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; tiny: the value named 'tiny' in this block; asarray: the value named 'asarray' in this block. Context: the function ___init___, which handles init.

Line 45: `"COMPUTE_BINARY",` This is continuation line 7 of the 10-line logical statement that begins at source line 39. Its exact fragment is '"COMPUTE_BINARY",'. The important operands are COMPUTE_BINARY: the value named 'compute binary' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)' to 'self._projection ' in host memory. Here _projection means the value named 'projection' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; tiny: the value named 'tiny' in this block; asarray: the value named 'asarray' in this block. Context: the function ___init___, which handles init.

Line 46: `(2, 2, 1),` This is continuation line 8 of the 10-line logical statement that begins at source line 39. Its exact fragment is '(2, 2, 1),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)' to 'self._projection ' in host memory. Here _projection means the value named 'projection' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; tiny: the value named 'tiny' in this block; asarray: the value named 'asarray' in this block. Context: the function ___init___, which handles init.

Line 47: `10,` This is continuation line 9 of the 10-line logical statement that begins at source line 39. Its exact fragment is '10,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)' to 'self._projection ' in host memory. Here _projection means the value named 'projection' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; tiny: the value named 'tiny' in this block; asarray: the value named 'asarray' in this block. Context: the function ___init___, which handles init.

Line 48: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 39. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'ProjectionParameters("tiny", np.asarray([[3, -4], [-2, 5]], dtype=np.int8), np.asarray([7, -9], dtype=np.int32), np.asarray([16384, 24576], dtype=np.int16), np.asarray([-1, 1], dtype=np.int8), "COMPUTE_BINARY", (2, 2, 1), 10,)' to 'self._projection ' in host memory. Here _projection means the value named 'projection' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; tiny: the value named 'tiny' in this block; asarray: the value named 'asarray' in this block. Context: the function ___init___, which handles init.

Line 49: `self._head = HeadParameters(` This statement continues through source line 52; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'HeadParameters(np.asarray([[6, -7], [8, 9]], dtype=np.int8), np.asarray([11, -13], dtype=np.int32),)' to 'self._head ' in host memory. Here _head means the value named 'head' in this block. The important operands are HeadParameters: the value named 'head parameters' in this block; asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block. Context: the function ___init___, which handles init.

Line 50: `np.asarray([[6, -7], [8, 9]], dtype=np.int8),` This is continuation line 2 of the 4-line logical statement that begins at source line 49. Its exact fragment is 'np.asarray([[6, -7], [8, 9]], dtype=np.int8),'. The important operands are asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block; int8: the value named 'int8' in this block. Read together, the complete operation is: This assigns 'HeadParameters(np.asarray([[6, -7], [8, 9]], dtype=np.int8), np.asarray([11, -13], dtype=np.int32),)' to 'self._head ' in host memory. Here _head means the value named 'head' in this block. The important operands are HeadParameters: the value named 'head parameters' in this block; asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block. Context: the function ___init___, which handles init.

Line 51: `np.asarray([11, -13], dtype=np.int32),` This is continuation line 3 of the 4-line logical statement that begins at source line 49. Its exact fragment is 'np.asarray([11, -13], dtype=np.int32),'. The important operands are asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block; int32: the value named 'int32' in this block. Read together, the complete operation is: This assigns 'HeadParameters(np.asarray([[6, -7], [8, 9]], dtype=np.int8), np.asarray([11, -13], dtype=np.int32),)' to 'self._head ' in host memory. Here _head means the value named 'head' in this block. The important operands are HeadParameters: the value named 'head parameters' in this block; asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block. Context: the function ___init___, which handles init.

Line 52: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 49. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'HeadParameters(np.asarray([[6, -7], [8, 9]], dtype=np.int8), np.asarray([11, -13], dtype=np.int32),)' to 'self._head ' in host memory. Here _head means the value named 'head' in this block. The important operands are

HeadParameters: the value named 'head parameters' in this block; asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block. Context: the function ___init___, which handles init.

Line 54: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class TinyParameterArchive, which handles tiny parameter archive; it creates no FPGA circuit by itself.

Line 55: `def projection_names(self) -> tuple[str, ...]:` This starts function projection_names. Its indented body performs projection names as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 56: `return ("tiny",)` This returns '("tiny",)' to the caller of the function projection_names, which handles projection names, ending this invocation. The important operands are tiny: the value named 'tiny' in this block.

Line 58: `def projection(self, name: str) -> ProjectionParameters:` This starts function projection. Its indented body performs projection as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 59: `if name != "tiny":` This executes the following indented host-code branch only when 'name != "tiny"' is true. The important operands are name: the value named 'name' in this block; tiny: the value named 'tiny' in this block. Context: the function projection, which handles projection.

Line 60: `raise KeyError(name)` This rejects an invalid state by raising 'KeyError(name)'. Failing before DMA prevents malformed shapes or artifacts from silently becoming hardware results.

Line 61: `return self._projection` This returns 'self._projection' to the caller of the function projection, which handles projection, ending this invocation. The important operands are _projection: the value named 'projection' in this block.

Line 63: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class TinyParameterArchive, which handles tiny parameter archive; it creates no FPGA circuit by itself.

Line 64: `def head(self) -> HeadParameters:` This starts function head. Its indented body performs head as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 65: `return self._head` This returns 'self._head' to the caller of the function head, which handles head, ending this invocation. The important operands are _head: the value named 'head' in this block.

Line 68: `class TinyFullGraphArchive:` This defines class TinyFullGraphArchive, which groups data and methods for tiny full graph archive. Instances exist in host memory and organize the flow that checks spikingformer training runtime behavior independently with Python assertions.

Line 69: `def __init__(self) -> None:` This starts function ___init___. Its indented body performs init as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 70: `self.manifest = {` This statement continues through source line 79; the explanation uses the complete expression rather than only the first visual fragment. This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are binary_sha256: the value named 'binary sha256' in

this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 71: `"binary_sha256": "b" * 64,` This is continuation line 2 of the 10-line logical statement that begins at source line 70. Its exact fragment is '"binary_sha256": "b" * 64,'. The important operands are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block. Read together, the complete operation is: This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 72: `"fixed_point": {` This is continuation line 3 of the 10-line logical statement that begins at source line 70. Its exact fragment is '"fixed_point": {'. The important operands are fixed_point: the value named 'fixed point' in this block. Read together, the complete operation is: This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 73: `"activation_fractional_bits": 10,` This is continuation line 4 of the 10-line logical statement that begins at source line 70. Its exact fragment is '"activation_fractional_bits": 10,'. The important operands are activation_fractional_bits: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 74: `"image_input_scale": 0.02,` This is continuation line 5 of the 10-line logical statement that begins at source line 70. Its exact fragment is '"image_input_scale": 0.02,'. The important operands are image_input_scale: the real-value spacing represented by one quantized integer step. Read together, the complete operation is: This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 75: `"image_input_zero_point": 123,` This is continuation line 6 of the 10-line logical statement that begins at source line 70. Its exact fragment is '"image_input_zero_point": 123,'. The important operands are image_input_zero_point: the integer code representing real numeric zero in affine quantization. Read together, the complete operation is: This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 76: `"head_activation_scale": 0.125,` This is continuation line 7 of the 10-line logical statement that begins at source line 70. Its exact fragment is '"head_activation_scale": 0.125,'. The important operands are head_activation_scale: an input feature value consumed by a neuron or multiply-accumulate operation. Read together, the complete operation is: This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands

are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 77: `},` This is continuation line 8 of the 10-line logical statement that begins at source line 70. Its exact fragment is '},'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 78: `"head": {"accumulator_scale": 1.0 / 1024.0},` This is continuation line 9 of the 10-line logical statement that begins at source line 70. Its exact fragment is '"head": {"accumulator_scale": 1.0 / 1024.0},'. The important operands are head: the value named 'head' in this block; accumulator_scale: a widened running sum that prevents intermediate overflow. Read together, the complete operation is: This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 79: `}` This is continuation line 10 of the 10-line logical statement that begins at source line 70. Its exact fragment is '}'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "binary_sha256": "b" * 64, "fixed_point": { "activation_fractional_bits": 10, "image_input_scale": 0.02, "image_input_zero_point": 123, "head_activation_scale": 0.125, }, "head": {"accumulator_scale": 1.0 / 1024.0}, }' to 'self.manifest ' in host memory. Here manifest means the value named 'manifest' in this block. The important operands are binary_sha256: the value named 'binary sha256' in this block; b: the value named 'b' in this block; fixed_point: the value named 'fixed point' in this block. Context: the function ___init___, which handles init.

Line 80: `self._projections: dict[str, ProjectionParameters] = {}` This assigns '{}' to 'self._projections' in host memory. Here _projections means the value named 'projections' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function ___init___, which handles init.

Line 82: `def add_conv(name: str, input_channels: int, output_channels: int, dense=False) -> None:` This starts function add_conv. Its indented body performs add conv as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 83: `reduction = input_channels * 3 * 3` This assigns 'input_channels * 3 * 3' to 'reduction ' in host memory. Here reduction means the value named 'reduction' in this block. The important operands are input_channels: the value named 'input channels' in this block. Context: the function add_conv, which handles add conv.

Line 84: `weight = np.fromfunction(` This statement continues through source line 88; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.fromfunction(lambda row, column: ((row + 2 * column) % 3) - 1, (reduction, output_channels), dtype=int,).astype(np.int8)' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named 'fromfunction' in this block; lambda: the value named 'lambda' in this block; row: one output-row position processed by the tensor tile. Context: the function add_conv, which handles add conv.

Line 85: `lambda row, column: ((row + 2 * column) % 3) - 1,` This is continuation line 2 of the 5-line logical statement that begins at source line 84. Its exact fragment is 'lambda row, column: ((row + 2 * column) % 3) - 1,'. The important operands are lambda: the value named 'lambda' in this block; row: one output-row position processed by the tensor tile; column: one output-feature position processed in parallel. Read together, the complete operation is: This assigns 'np.fromfunction(lambda row, column: ((row + 2 * column) % 3) - 1, (reduction, output_channels), dtype=int,

).astype(np.int8)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named ’fromfunction’ in this block; lambda: the value named ’lambda’ in this block; row: one output-row position processed by the tensor tile. Context: the function add_conv, which handles add conv.

Line 86: `(reduction, output_channels),` This is continuation line 3 of the 5-line logical statement that begins at source line 84. Its exact fragment is ‘(reduction, output_channels),‘. The important operands are reduction: the value named ’reduction’ in this block; output_channels: the value named ’output channels’ in this block. Read together, the complete operation is: This assigns ‘np.fromfunction(lambda row, column: ((row + 2 * column) % 3) - 1, (reduction, output_channels), dtype=int,).astype(np.int8)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named ’fromfunction’ in this block; lambda: the value named ’lambda’ in this block; row: one output-row position processed by the tensor tile. Context: the function add_conv, which handles add conv.

Line 87: `dtype=int,` This is continuation line 4 of the 5-line logical statement that begins at source line 84. Its exact fragment is ‘dtype=int,‘. The important operands are dtype: the value named ’dtype’ in this block. Read together, the complete operation is: This assigns ‘np.fromfunction(lambda row, column: ((row + 2 * column) % 3) - 1, (reduction, output_channels), dtype=int,).astype(np.int8)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named ’fromfunction’ in this block; lambda: the value named ’lambda’ in this block; row: one output-row position processed by the tensor tile. Context: the function add_conv, which handles add conv.

Line 88: `).astype(np.int8)` This is continuation line 5 of the 5-line logical statement that begins at source line 84. Its exact fragment is ‘).astype(np.int8)‘. The important operands are astype: the value named ’astype’ in this block; int8: the value named ’int8’ in this block. Read together, the complete operation is: This assigns ‘np.fromfunction(lambda row, column: ((row + 2 * column) % 3) - 1, (reduction, output_channels), dtype=int,).astype(np.int8)‘ to ‘weight ‘ in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named ’fromfunction’ in this block; lambda: the value named ’lambda’ in this block; row: one output-row position processed by the tensor tile. Context: the function add_conv, which handles add conv.

Line 89: `self._projections[name] = ProjectionParameters(` This statement continues through source line 98; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)‘ to ‘self._projections[name] ‘ in host memory. Here _projections means the value named ’projections’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; name: the value named ’name’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 90: `name,` This is continuation line 2 of the 10-line logical statement that begins at source line 89. Its exact fragment is ‘name,‘. The important operands are name: the value named ’name’ in this block. Read together, the complete operation is: This assigns ‘ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)‘ to ‘self._projections[name] ‘ in host memory. Here _projections means the value named ’projections’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; name: the value named ’name’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 91: `weight,` This is continuation line 3 of the 10-line logical statement that begins at source line 89. Its exact fragment is ‘weight,‘. The important operands are weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns ‘ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)‘ to ‘self._projections[name] ‘ in host memory. Here _projections means the value named ’projections’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; name: the value

named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 92: `np.full(output_channels, 2200, dtype=np.int32),` This is continuation line 4 of the 10-line logical statement that begins at source line 89. Its exact fragment is 'np.full(output_channels, 2200, dtype=np.int32),'. The important operands are full: the value named 'full' in this block; output_channels: the value named 'output channels' in this block; dtype: the value named 'dtype' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 93: `np.full(output_channels, 16384, dtype=np.int16),` This is continuation line 5 of the 10-line logical statement that begins at source line 89. Its exact fragment is 'np.full(output_channels, 16384, dtype=np.int16),'. The important operands are full: the value named 'full' in this block; output_channels: the value named 'output channels' in this block; dtype: the value named 'dtype' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 94: `np.full(output_channels, -1, dtype=np.int8),` This is continuation line 6 of the 10-line logical statement that begins at source line 89. Its exact fragment is 'np.full(output_channels, -1, dtype=np.int8),'. The important operands are full: the value named 'full' in this block; output_channels: the value named 'output channels' in this block; dtype: the value named 'dtype' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 95: `"COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY",` This is continuation line 7 of the 10-line logical statement that begins at source line 89. Its exact fragment is '"COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY",'. The important operands are COMPUTE_DENSE_OUTER: the value named 'compute dense outer' in this block; dense: the value named 'dense' in this block; COMPUTE_BINARY: the value named 'compute binary' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 96: `(output_channels, input_channels, 3, 3),` This is continuation line 8 of the 10-line logical statement that begins at source line 89. Its exact fragment is '(output_channels, input_channels, 3, 3),'. The important operands are output_channels: the value named 'output channels' in this block; input_channels: the value named 'input channels' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'pro-

jection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 97: `10,` This is continuation line 9 of the 10-line logical statement that begins at source line 89. Its exact fragment is '10,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 98: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 89. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(output_channels, 2200, dtype=np.int32), np.full(output_channels, 16384, dtype=np.int16), np.full(output_channels, -1, dtype=np.int8), "COMPUTE_DENSE_OUTER" if dense else "COMPUTE_BINARY", (output_channels, input_channels, 3, 3), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_conv, which handles add conv.

Line 100: `add_conv("patch_embed.block0", 3, 2, dense=True)` This calls 'add_conv' with '"patch_embed.block0", 3, 2, dense=True' inside the function ___init___, which handles init. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 101: `for index in range(1, 5):` This iterates index over 'range(1, 5)'. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function ___init___, which handles init.

Line 102: `add_conv(f"patch_embed.block{index}", 2, 2)` This calls 'add_conv' with 'f"patch_embed.block{index}", 2, 2' inside the function ___init___, which handles init. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 104: `def add_matrix(name: str, inputs: int, outputs: int, bias: int = 2200) -> None:` This starts function add_matrix. Its indented body performs add matrix as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 105: `weight = np.fromfunction(` This statement continues through source line 109; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'np.fromfunction(lambda row, column: ((2 * row + column) % 3) - 1, (inputs, outputs), dtype=int,).astype(np.int8)' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named 'fromfunction' in this block; lambda: the value named 'lambda' in this block; row: one output-row position processed by the tensor tile. Context: the function add_matrix, which handles add matrix.

Line 106: `lambda row, column: ((2 * row + column) % 3) - 1,` This is continuation line 2 of the 5-line logical statement that begins at source line 105. Its exact fragment is 'lambda row, column: ((2 * row + column) % 3) - 1,'. The important operands are lambda: the value named 'lambda' in this block; row: one output-row position processed by the tensor tile; column: one output-feature position processed in parallel. Read together, the complete operation is: This assigns 'np.fromfunction(lambda row, column: ((2 * row + column) % 3) - 1, (inputs, outputs), dtype=int,).astype(np.int8)' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named 'fromfunction' in this block; lambda: the value named 'lambda' in this block; row: one output-row position processed by the tensor tile. Context: the function add_matrix, which handles add matrix.

Line 107: `(inputs, outputs),` This is continuation line 3 of the 5-line logical statement that begins at source line 105. Its exact fragment is '(inputs, outputs),'. The important operands are inputs: the value named 'inputs' in this block; outputs: the value named 'outputs' in this block. Read together, the complete operation is: This assigns 'np.fromfunction(lambda row, column: ((2 * row + column) % 3) - 1, (inputs, outputs), dtype=int,).astype(np.int8)' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named 'fromfunction' in this block; lambda: the value named 'lambda' in this block; row: one output-row position processed by the tensor tile. Context: the function add_matrix, which handles add matrix.

Line 108: `dtype=int,` This is continuation line 4 of the 5-line logical statement that begins at source line 105. Its exact fragment is 'dtype=int,'. The important operands are dtype: the value named 'dtype' in this block. Read together, the complete operation is: This assigns 'np.fromfunction(lambda row, column: ((2 * row + column) % 3) - 1, (inputs, outputs), dtype=int,).astype(np.int8)' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named 'fromfunction' in this block; lambda: the value named 'lambda' in this block; row: one output-row position processed by the tensor tile. Context: the function add_matrix, which handles add matrix.

Line 109: `).astype(np.int8)` This is continuation line 5 of the 5-line logical statement that begins at source line 105. Its exact fragment is ').astype(np.int8)'. The important operands are astype: the value named 'astype' in this block; int8: the value named 'int8' in this block. Read together, the complete operation is: This assigns 'np.fromfunction(lambda row, column: ((2 * row + column) % 3) - 1, (inputs, outputs), dtype=int,).astype(np.int8)' to 'weight ' in host memory. Here weight means a learned coefficient multiplied by an activation. The important operands are fromfunction: the value named 'fromfunction' in this block; lambda: the value named 'lambda' in this block; row: one output-row position processed by the tensor tile. Context: the function add_matrix, which handles add matrix.

Line 110: `self._projections[name] = ProjectionParameters(` This statement continues through source line 119; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs, 1), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 111: `name,` This is continuation line 2 of the 10-line logical statement that begins at source line 110. Its exact fragment is 'name,'. The important operands are name: the value named 'name' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs, 1), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 112: `weight,` This is continuation line 3 of the 10-line logical statement that begins at source line 110. Its exact fragment is 'weight,'. The important operands are weight: a learned coefficient multiplied by an activation. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs, 1), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 113: `np.full(outputs, bias, dtype=np.int32),` This is continuation line 4 of the 10-line logical statement that begins at source line 110. Its exact fragment is 'np.full(outputs, bias, dtype=np.int32),'. The important operands are full: the value named 'full' in this block; outputs: the value named 'outputs' in this block; bias: an additive learned offset applied after accumulation. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs, 1), 10,)' to 'self._projections[name] ' in host memory. Here _projections

means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 114: `np.full(outputs, 16384, dtype=np.int16),` This is continuation line 5 of the 10-line logical statement that begins at source line 110. Its exact fragment is 'np.full(outputs, 16384, dtype=np.int16),'. The important operands are full: the value named 'full' in this block; outputs: the value named 'outputs' in this block; dtype: the value named 'dtype' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs, 1), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 115: `np.full(outputs, -1, dtype=np.int8),` This is continuation line 6 of the 10-line logical statement that begins at source line 110. Its exact fragment is 'np.full(outputs, -1, dtype=np.int8),'. The important operands are full: the value named 'full' in this block; outputs: the value named 'outputs' in this block; dtype: the value named 'dtype' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs, 1), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 116: `"COMPUTE_BINARY",` This is continuation line 7 of the 10-line logical statement that begins at source line 110. Its exact fragment is '"COMPUTE_BINARY",'. The important operands are COMPUTE_BINARY: the value named 'compute binary' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs, 1), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 117: `(outputs, inputs, 1),` This is continuation line 8 of the 10-line logical statement that begins at source line 110. Its exact fragment is '(outputs, inputs, 1),'. The important operands are outputs: the value named 'outputs' in this block; inputs: the value named 'inputs' in this block. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs, 1), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 118: `10,` This is continuation line 9 of the 10-line logical statement that begins at source line 110. Its exact fragment is '10,'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs, 1), 10,)' to 'self._projections[name] ' in host memory. Here _projections means the value named 'projections' in this block. The important operands are ProjectionParameters: the value named 'projection parameters' in this block; name: the value named 'name' in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 119: `)` This is continuation line 10 of the 10-line logical statement that begins at source line 110. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'ProjectionParameters(name, weight, np.full(outputs, bias, dtype=np.int32), np.full(outputs, 16384, dtype=np.int16), np.full(outputs, -1, dtype=np.int8), "COMPUTE_BINARY", (outputs, inputs,

1), 10,)‘ to ‘self._projections[name] ‘ in host memory. Here _projections means the value named ’projections’ in this block. The important operands are ProjectionParameters: the value named ’projection parameters’ in this block; name: the value named ’name’ in this block; weight: a learned coefficient multiplied by an activation. Context: the function add_matrix, which handles add matrix.

Line 121: `for name in ("q", "k", "v"):` This iterates name over ‘("q", "k", "v")‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function ___init___, which handles init.

Line 122: `add_matrix(f"block.0.attn.{name}", 2, 2)` This calls ‘add_matrix‘ with ‘f"block.0.attn.{name}", 2, 2‘ inside the function ___init___, which handles init. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 123: `add_matrix("block.0.attn.proj", 2, 2, bias=0)` This calls ‘add_matrix‘ with ‘"block.0.attn.proj", 2, 2, bias=0‘ inside the function ___init___, which handles init. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 124: `add_matrix("block.0.mlp.fc1", 2, 3)` This calls ‘add_matrix‘ with ‘"block.0.mlp.fc1", 2, 3‘ inside the function ___init___, which handles init. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 125: `add_matrix("block.0.mlp.fc2", 3, 2, bias=0)` This calls ‘add_matrix‘ with ‘"block.0.mlp.fc2", 3, 2, bias=0‘ inside the function ___init___, which handles init. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 126: `self._head = HeadParameters(` This statement continues through source line 129; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘HeadParameters(np.asarray([[3, -2, 1], [-1, 2, 4]], dtype=np.int8), np.asarray([17, -11, 5], dtype=np.int32),)‘ to ‘self._head ‘ in host memory. Here _head means the value named ’head’ in this block. The important operands are HeadParameters: the value named ’head parameters’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function ___init___, which handles init.

Line 127: `np.asarray([[3, -2, 1], [-1, 2, 4]], dtype=np.int8),` This is continuation line 2 of the 4-line logical statement that begins at source line 126. Its exact fragment is ‘np.asarray([[3, -2, 1], [-1, 2, 4]], dtype=np.int8),‘. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; int8: the value named ’int8’ in this block. Read together, the complete operation is: This assigns ‘HeadParameters(np.asarray([[3, -2, 1], [-1, 2, 4]], dtype=np.int8), np.asarray([17, -11, 5], dtype=np.int32),)‘ to ‘self._head ‘ in host memory. Here _head means the value named ’head’ in this block. The important operands are HeadParameters: the value named ’head parameters’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function ___init___, which handles init.

Line 128: `np.asarray([17, -11, 5], dtype=np.int32),` This is continuation line 3 of the 4-line logical statement that begins at source line 126. Its exact fragment is ‘np.asarray([17, -11, 5], dtype=np.int32),‘. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; int32: the value named ’int32’ in this block. Read together, the complete operation is: This assigns ‘HeadParameters(np.asarray([[3, -2, 1], [-1, 2, 4]], dtype=np.int8), np.asarray([17, -11, 5], dtype=np.int32),)‘ to ‘self._head ‘ in host memory. Here _head means the value named ’head’ in this block. The important operands are HeadParameters: the value named ’head parameters’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function ___init___, which handles init.

Line 129: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 126. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘HeadParameters(np.asarray([[3, -2, 1], [-1, 2, 4]], dtype=np.int8), np.asarray([17, -11, 5], dtype=np.int32),)‘ to ‘self._head ‘ in host memory. Here _head means the value named ’head’ in this block. The important operands are HeadParameters: the value named ’head parameters’ in this block; asarray: the value named

'asarray' in this block; dtype: the value named 'dtype' in this block. Context: the function ___init___, which handles init.

Line 131: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class TinyFullGraphArchive, which handles tiny full graph archive; it creates no FPGA circuit by itself.

Line 132: `def projection_names(self) -> tuple[str, ...]:` This starts function projection_names. Its indented body performs projection names as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 133: `return tuple(self._projections)` This returns 'tuple(self._projections)' to the caller of the function projection_names, which handles projection names, ending this invocation. The important operands are tuple: the value named 'tuple' in this block; _projections: the value named 'projections' in this block.

Line 135: `def projection(self, name: str) -> ProjectionParameters:` This starts function projection. Its indented body performs projection as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 136: `return self._projections[name]` This returns 'self._projections[name]' to the caller of the function projection, which handles projection, ending this invocation. The important operands are _projections: the value named 'projections' in this block; name: the value named 'name' in this block.

Line 138: `@property` This decorator changes how the following class or function is constructed. '@property' is applied at import time on the host and is part of the class TinyFullGraphArchive, which handles tiny full graph archive; it creates no FPGA circuit by itself.

Line 139: `def head(self) -> HeadParameters:` This starts function head. Its indented body performs head as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 140: `return self._head` This returns 'self._head' to the caller of the function head, which handles head, ending this invocation. The important operands are _head: the value named 'head' in this block.

Line 143: `class SpikingformerTrainingRuntimeTests(unittest.TestCase):` This defines class SpikingformerTrainingRuntimeTests, which groups data and methods for spikingformer training runtime tests. Instances exist in host memory and organize the flow that checks spikingformer training runtime behavior independently with Python assertions.

Line 144: `def setUp(self) -> None:` This starts function setUp. Its indented body performs set up as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 145: `self.driver = FullTrainingEmulator()` This assigns 'FullTrainingEmulator()' to 'self.driver ' in host memory. Here driver means the value named 'driver' in this block. The important operands are FullTrainingEmulator: the value named 'full training emulator' in this block. Context: the function setUp, which handles set up.

Line 147: `def test_block_floating_shift_normalize_and_add_are_exact(self) -> None:` This starts function test_block_floating_shift_normalize_and_add_are_exact. Its indented body performs test block floating shift normalize and add are exact as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 148: `values = np.asarray([[-2_000_000, -65537, 0, 65537, 2_000_000]], dtype=np.int64)` This assigns 'np.asarray([[-2_000_000, -65537, 0, 65537, 2_000_000]], dtype=np.int64)' to 'values ' in host memory. Here values means the value named 'values' in this block. The important operands are asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact.

Line 149: `shifted = execute_power_of_two_scale(self.driver, values, -4)` This assigns 'execute_power_of_two_scale values, -4)' to 'shifted ' in host memory. Here shifted means the value named 'shifted' in this block. The important operands are execute_power_of_two_scale: the real-value spacing represented by one quantized integer step; driver: the value named 'driver' in this block; values: the value named 'values' in this block. Context: the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact.

Line 150: `np.testing.assert_array_equal(shifted, round_shift_signed(values, 4))` This calls 'np.testing.assert_array with 'shifted, round_shift_signed(values, 4)' inside the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 151: `normalized = normalize_gradient_s16(self.driver, BlockGradient(values, -15))` This assigns 'normalize_gradient_s16(self.driver, BlockGradient(values, -15))' to 'normalized ' in host memory. Here normalized means the value named 'normalized' in this block. The important operands are normalize_gradient_s16: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact.

Line 152: `self.assertGreater(normalized.exponent, -15)` This calls 'self.assertGreater' with 'normalized.exponent, -15' inside the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 153: `self.assertLessEqual(int(np.abs(normalized.values).max()), 32767)` This calls 'self.assertLessEqual' with 'int(np.abs(normalized.values).max()), 32767' inside the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 154: `left = BlockGradient(np.asarray([100, -200, 300]), -12)` This assigns 'BlockGradient(np.asarray([100, -200, 300]), -12)' to 'left ' in host memory. Here left means the value named 'left' in this block. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; asarray: the value named 'asarray' in this block. Context: the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact.

Line 155: `right = BlockGradient(np.asarray([800, 400, -1200]), -15)` This assigns 'BlockGradient(np.asarray([800, 400, -1200]), -15)' to 'right ' in host memory. Here right means the value named 'right' in this block. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; asarray: the value named 'asarray' in this block. Context: the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact.

Line 156: `added = add_block_gradients(self.driver, left, right)` This assigns 'add_block_gradients(self.driver, left, right)' to 'added ' in host memory. Here added means the value named 'added' in this block. The important operands are add_block_gradients: the signed loss derivative used to update a learned parameter; driver: the value named 'driver' in this block; left: the value named 'left' in this block. Context: the function test_block_floating_shift_normalize_and_add_are_e which handles test block floating shift normalize and add are exact.

Line 157: `self.assertEqual(added.exponent, -12)` This calls 'self.assertEqual' with 'added.exponent, -12' inside the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 158: `np.testing.assert_array_equal(` This statement continues through source line 161; the explanation uses the complete expression rather than only the first visual fragment. This calls 'np.testing.assert_array_equal' with ' added.values, left.values + round_shift_signed(right.values, 3), ' inside the function test_block_floating_shift_normalize_and_ad which handles test block floating shift normalize and add are exact. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 159: `added.values,` This is continuation line 2 of the 4-line logical statement that begins at source line 158. Its exact fragment is 'added.values,'. The important operands are added: the value named 'added' in this block; values: the value named 'values' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' added.values, left.values + round_shift_signed(right.values, 3), ' inside the function test_block_floating_shift_normalize_an which handles test block floating shift normalize and add are exact. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 160: `left.values + round_shift_signed(right.values, 3),` This is continuation line 3 of the 4-line logical statement that begins at source line 158. Its exact fragment is 'left.values + round_shift_signed(right.values, 3),'. The important operands are left: the value named 'left' in this block; values: the value named 'values' in this block; round_shift_signed: the value named 'round shift signed' in this block. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' added.values, left.values + round_shift_signed(right.values, 3), ' inside the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 161: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 158. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'np.testing.assert_array_equal' with ' added.values, left.values + round_shift_signed(right.values, 3), ' inside the function test_block_floating_shift_normalize_and_add_are_exact, which handles test block floating shift normalize and add are exact. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 163: `def test_projection_derivative_uses_exported_channel_scales(self) -> None:` This starts function test_projection_derivative_uses_exported_channel_scales. Its indented body performs test projection derivative uses exported channel scales as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 164: `parameters = TinyParameterArchive().projection("tiny")` This assigns 'TinyParameterArchive().projection("ti to 'parameters ' in host memory. Here parameters means the value named 'parameters' in this block. The important operands are TinyParameterArchive: the value named 'tiny parameter archive' in this block; projection: the value named 'projection' in this block; tiny: the value named 'tiny' in this block. Context: the function test_projection_derivative_uses_exported which handles test projection derivative uses exported channel scales.

Line 165: `gradient = BlockGradient(` This statement continues through source line 168; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'BlockGradient(np.asarray([[31000, -27000], [-11000, 9000], [777, -333]], dtype=np.int64), -15,)' to 'gradient ' in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block. Context: the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales.

Line 166: `np.asarray([[31000, -27000], [-11000, 9000], [777, -333]], dtype=np.int64),` This is continuation line 2 of the 4-line logical statement that begins at source line 165. Its exact fragment is 'np.asarray([[31000,

-27000], [-11000, 9000], [777, -333]], dtype=np.int64),‘. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Read together, the complete operation is: This assigns ‘BlockGradient(np.asarray([[31000, -27000], [-11000, 9000], [777, -333]], dtype=np.int64), -15,)‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales.

Line 167: `-15,` This is continuation line 3 of the 4-line logical statement that begins at source line 165. Its exact fragment is ‘-15,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘BlockGradient(np.asarray([[31000, -27000], [-11000, 9000], [777, -333]], dtype=np.int64), -15,)‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales.

Line 168: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 165. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘BlockGradient(np.asarray([[31000, -27000], [-11000, 9000], [777, -333]], dtype=np.int64), -15,)‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales.

Line 169: `actual = backward_requantize_projection(self.driver, gradient, parameters)` This assigns ‘backward_requantize_projection(self.driver, gradient, parameters)‘ to ‘actual ‘ in host memory. Here actual means the value named ’actual’ in this block. The important operands are backward_requantize_projection: the conversion of a wide accumulated result back into a bounded activation code; driver: the value named ’driver’ in this block; gradient: the signed loss derivative used to update a learned parameter. Context: the function test_projection_derivative_uses_exported_channel_scal which handles test projection derivative uses exported channel scales.

Line 170: `boost = gradient.exponent - actual.exponent` This assigns ‘gradient.exponent - actual.exponent‘ to ‘boost ‘ in host memory. Here boost means the value named ’boost’ in this block. The important operands are gradient: the signed loss derivative used to update a learned parameter; exponent: the value named ’exponent’ in this block; actual: the value named ’actual’ in this block. Context: the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales.

Line 171: `expected = np.empty_like(gradient.values)` This assigns ‘np.empty_like(gradient.values)‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are empty_like: the value named ’empty like’ in this block; gradient: the signed loss derivative used to update a learned parameter; values: the value named ’values’ in this block. Context: the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales.

Line 172: `for channel in range(2):` This iterates channel over ‘range(2)‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_projection_derivative_uses_exported_channel_ which handles test projection derivative uses exported channel scales.

Line 173: `expected[:, channel] = round_shift_signed(` This statement continues through source line 176; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘round_shift_signed(gradient.values[:, channel] * int(parameters.multiplier_q15[channel]), 15 + int(parameters.binary_point_shift[channel]) - boost,)‘ to ‘expected[‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; gradient: the signed loss

derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales.

Line 174: `gradient.values[:, channel] * int(parameters.multiplier_q15[channel]),` This is continuation line 2 of the 4-line logical statement that begins at source line 173. Its exact fragment is 'gradient.values[:, channel] * int(parameters.multiplier_q15[channel]),'. The important operands are gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block; channel: the value named 'channel' in this block. Read together, the complete operation is: This assigns 'round_shift_signed(gradient.values[:, channel] * int(parameters.multiplier_q15[channel]), 15 + int(parameters.binary_point_shift[channel]) - boost,)' to 'expected[' in host memory. Here expected means the value named 'expected' in this block. The important operands are round_shift_signed: the value named 'round shift signed' in this block; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function test_projection_derivative_uses_exported which handles test projection derivative uses exported channel scales.

Line 175: `15 + int(parameters.binary_point_shift[channel]) - boost,` This is continuation line 3 of the 4-line logical statement that begins at source line 173. Its exact fragment is '15 + int(parameters.binary_point_shift[channel]) - boost,'. The important operands are parameters: the value named 'parameters' in this block; binary_point_shift: the value named 'binary point shift' in this block; channel: the value named 'channel' in this block. Read together, the complete operation is: This assigns 'round_shift_signed(gradient.values[:, channel] * int(parameters.multiplier_q15[channel]), 15 + int(parameters.binary_point_shift[channel]) - boost,)' to 'expected[' in host memory. Here expected means the value named 'expected' in this block. The important operands are round_shift_signed: the value named 'round shift signed' in this block; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales.

Line 176: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 173. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'round_shift_signed(gradient.values[:, channel] * int(parameters.multiplier_q15[channel]), 15 + int(parameters.binary_point_shift[channel]) - boost,)' to 'expected[' in host memory. Here expected means the value named 'expected' in this block. The important operands are round_shift_signed: the value named 'round shift signed' in this block; gradient: the signed loss derivative used to update a learned parameter; values: the value named 'values' in this block. Context: the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales.

Line 177: `np.testing.assert_array_equal(actual.values, expected)` This calls 'np.testing.assert_array_equal' with 'actual.values, expected' inside the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 178: `self.assertLessEqual(int(np.abs(actual.values).max()), 32767)` This calls 'self.assertLessEqual' with 'int(np.abs(actual.values).max()), 32767' inside the function test_projection_derivative_uses_exported_channel_scales, which handles test projection derivative uses exported channel scales. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 180: `def test_affine_backward_returns_exact_dx_dw_and_bias(self) -> None:` This starts function test_affine_bac Its indented body performs test affine backward returns exact dx dw and bias as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 181: `x = np.asarray([[3, -2, 1], [-4, 5, 2], [7, 1, -3]], dtype=np.int64)` This assigns 'np.asarray([[3, -2, 1], [-4, 5, 2], [7, 1, -3]], dtype=np.int64)' to 'x ' in host memory. Here x means the value named 'x' in this block. The important operands are asarray: the value named 'asarray' in this block; dtype: the value named 'dtype' in this block; int64: the value named 'int64' in this block. Context: the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias.

Line 182: `w = np.asarray([[2, -1], [4, 3], [-5, 6]], dtype=np.int64)` This assigns ‘np.asarray([[2, -1], [4, 3], [-5, 6]], dtype=np.int64)‘ to ‘w ‘ in host memory. Here w means the value named ’w’ in this block. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias.

Line 183: `bias = np.asarray([9, -8], dtype=np.int64)` This assigns ‘np.asarray([9, -8], dtype=np.int64)‘ to ‘bias ‘ in host memory. Here bias means an additive learned offset applied after accumulation. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias.

Line 184: `dy = np.asarray([[11, -7], [5, 13], [-3, 2]], dtype=np.int64)` This assigns ‘np.asarray([[11, -7], [5, 13], [-3, 2]], dtype=np.int64)‘ to ‘dy ‘ in host memory. Here dy means the value named ’dy’ in this block. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias.

Line 185: `dx, parameter = execute_affine_backward_block(` This statement continues through source line 186; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_affine_backward_bloc self.driver, x, w, bias, BlockGradient(dy, -15))‘ to ‘dx, parameter ‘ in host memory. Here dx means the value named ’dx’ in this block. The important operands are execute_affine_backward_block: the value named ’execute affine backward block’ in this block; driver: the value named ’driver’ in this block; x: the value named ’x’ in this block. Context: the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias.

Line 186: `self.driver, x, w, bias, BlockGradient(dy, -15))` This is continuation line 2 of the 2-line logical statement that begins at source line 185. Its exact fragment is ‘self.driver, x, w, bias, BlockGradient(dy, -15))‘. The important operands are driver: the value named ’driver’ in this block; x: the value named ’x’ in this block; w: the value named ’w’ in this block. Read together, the complete operation is: This assigns ‘execute_affine_backward_block(self.driver, x, w, bias, BlockGradient(dy, -15))‘ to ‘dx, parameter ‘ in host memory. Here dx means the value named ’dx’ in this block. The important operands are execute_affine_backward_block: the value named ’execute affine backward block’ in this block; driver: the value named ’driver’ in this block; x: the value named ’x’ in this block. Context: the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias.

Line 187: `self.assertEqual(dx.exponent, -15)` This calls ‘self.assertEqual‘ with ‘dx.exponent, -15‘ inside the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 188: `self.assertEqual(parameter.weight.exponent, -15)` This calls ‘self.assertEqual‘ with ‘parameter.weight.exponent -15‘ inside the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 189: `np.testing.assert_array_equal(dx.values, dy @ w.T)` This calls ‘np.testing.assert_array_equal‘ with ‘dx.values, dy @ w.T‘ inside the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 190: `np.testing.assert_array_equal(parameter.weight.values, x.T @ dy)` This calls ‘np.testing.assert_array_equa with ‘parameter.weight.values, x.T @ dy‘ inside the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 191: `np.testing.assert_array_equal(parameter.bias.values, dy.sum(axis=0))` This calls ‘np.testing.assert_array_ with ‘parameter.bias.values, dy.sum(axis=0)‘ inside the function test_affine_backward_returns_exact_dx_dw_and_bias, which handles test affine backward returns exact dx dw and bias. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 193: `def test_hardware_softmax_and_batch_mean_cross_entropy_gradient(self) -> None:` This starts function test_hardware_softmax_and_batch_mean_cross_entropy_gradient. Its indented body performs test hardware softmax and batch mean cross entropy gradient as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 194: `logits = np.asarray([[1200, -300, 77], [-80, 900, 100]], dtype=np.int64)` This assigns ‘np.asarray([[1200 -300, 77], [-80, 900, 100]], dtype=np.int64)‘ to ‘logits ‘ in host memory. Here logits means the ten unnormalized classifier scores used for argmax. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; int64: the value named ’int64’ in this block. Context: the function test_hardware_softmax_and_batch_mean_cross_entropy_gradient, which handles test hardware softmax and batch mean cross entropy gradient.

Line 195: `probabilities = execute_softmax_q15(` This statement continues through source line 196; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_softmax_q15(self.driver, logits, difference_shift=8)‘ to ‘probabilities ‘ in host memory. Here probabilities means the value named ’probabilities’ in this block. The important operands are execute_softmax_q15: the value named ’execute softmax q15’ in this block; driver: the value named ’driver’ in this block; logits: the ten unnormalized classifier scores used for argmax. Context: the function test_hardware_softmax_and_batch_mean_cross_entropy_gradient, which handles test hardware softmax and batch mean cross entropy gradient.

Line 196: `self.driver, logits, difference_shift=8)` This is continuation line 2 of the 2-line logical statement that begins at source line 195. Its exact fragment is ‘self.driver, logits, difference_shift=8)‘. The important operands are driver: the value named ’driver’ in this block; logits: the ten unnormalized classifier scores used for argmax; difference_shift: the value named ’difference shift’ in this block. Read together, the complete operation is: This assigns ‘execute_softmax_q15(self.driver, logits, difference_shift=8)‘ to ‘probabilities ‘ in host memory. Here probabilities means the value named ’probabilities’ in this block. The important operands are execute_softmax_q15: the value named ’execute softmax q15’ in this block; driver: the value named ’driver’ in this block; logits: the ten unnormalized classifier scores used for argmax. Context: the function test_hardware_softmax_and_batch_mean_cross_entropy_gradient, which handles test hardware softmax and batch mean cross entropy gradient.

Line 197: `gradient = execute_cross_entropy_gradient_q15(` This statement continues through source line 198; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘execute_cross_entropy_gradien self.driver, probabilities, np.asarray([0, 2]))‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_cross_entropy_gradient_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; probabilities: the value named ’probabilities’ in this block. Context: the function test_hardware_softmax_and_batch_mean_cross_entropy_gradient, which handles test hardware softmax and batch mean cross entropy gradient.

Line 198: `self.driver, probabilities, np.asarray([0, 2]))` This is continuation line 2 of the 2-line logical statement that begins at source line 197. Its exact fragment is ‘self.driver, probabilities, np.asarray([0, 2]))‘. The important operands are driver: the value named ’driver’ in this block; probabilities: the value named ’probabilities’ in this block; asarray: the value named ’asarray’ in this block. Read together, the complete operation is: This assigns ‘execute_cross_entropy_gradient_q15(self.driver, probabilities, np.asarray([0, 2]))‘ to ‘gradient ‘ in host memory. Here gradient means the signed loss derivative used to update a learned parameter. The important operands are execute_cross_entropy_gradient_q15: the signed loss derivative used to update a learned parameter; driver: the value named ’driver’ in this block; probabilities: the value named ’probabilities’ in this block. Context: the function test_hardware_softmax_and_batch_mean_cross_entropy_gradient, which handles test hardware softmax and batch mean cross entropy gradient.

Line 199: `self.assertEqual(gradient.exponent, -15)` This calls ‘self.assertEqual‘ with ‘gradient.exponent, -15‘ inside the function test_hardware_softmax_and_batch_mean_cross_entropy_gradient, which handles test hardware

softmax and batch mean cross entropy gradient. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 200: `expected = probabilities.copy()` This assigns ‘probabilities.copy()‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are probabilities: the value named ’probabilities’ in this block; copy: the value named ’copy’ in this block. Context: the function test_hardware_softmax_and_batch_
which handles test hardware softmax and batch mean cross entropy gradient.

Line 201: `expected[0, 0] -= 1 << 15` Inside the function test_hardware_softmax_and_batch_mean_cross_entropy_gradie
which handles test hardware softmax and batch mean cross entropy gradient, this statement advances the host implementation that checks spikingformer training runtime behavior independently with Python assertions. The important operands are expected: the value named ’expected’ in this block.

Line 202: `expected[1, 2] -= 1 << 15` Inside the function test_hardware_softmax_and_batch_mean_cross_entropy_gradie
which handles test hardware softmax and batch mean cross entropy gradient, this statement advances the host implementation that checks spikingformer training runtime behavior independently with Python assertions. The important operands are expected: the value named ’expected’ in this block.

Line 203: `expected = round_shift_signed(expected, 1)` This assigns ‘round_shift_signed(expected, 1)‘ to ‘expected ‘ in host memory. Here expected means the value named ’expected’ in this block. The important operands are round_shift_signed: the value named ’round shift signed’ in this block; expected: the value named ’expected’ in this block. Context: the function test_hardware_softmax_and_batch_mean_cross_entropy_gradient, which handles test hardware softmax and batch mean cross entropy gradient.

Line 204: `np.testing.assert_array_equal(gradient.values, expected)` This calls ‘np.testing.assert_array_equal‘ with ‘gradient.values, expected‘ inside the function test_hardware_softmax_and_batch_mean_cross_entropy_gradient, which handles test hardware softmax and batch mean cross entropy gradient. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 206: `def test_low_rate_bias_update_retains_q16_learning_rate_resolution(self) -> None:` This starts function test_low_rate_bias_update_retains_q16_learning_rate_resolution. Its indented body performs test low rate bias update retains q16 learning rate resolution as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 207: `tensor = MasterTensor(` This statement continues through source line 213; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘MasterTensor(np.asarray([0], dtype=np.int16), np.asarray([0], dtype=np.int32), 2, -(1 << 15), (1 << 15) - 1,)‘ to ‘tensor ‘ in host memory. Here tensor means the value named ’tensor’ in this block. The important operands are MasterTensor: the value named ’master tensor’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_low_rate_bias_update_retains_q16_learning_rate_resolution, which handles test low rate bias update retains q16 learning rate resolution.

Line 208: `np.asarray([0], dtype=np.int16),` This is continuation line 2 of the 7-line logical statement that begins at source line 207. Its exact fragment is ‘np.asarray([0], dtype=np.int16),‘. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; int16: the value named ’int16’ in this block. Read together, the complete operation is: This assigns ‘MasterTensor(np.asarray([0], dtype=np.int16), np.asarray([0], dtype=np.int32), 2, -(1 << 15), (1 << 15) - 1,)‘ to ‘tensor ‘ in host memory. Here tensor means the value named ’tensor’ in this block. The important operands are MasterTensor: the value named ’master tensor’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_low_rate_bias_update_retains_q16_learning_rate_resolution, which handles test low rate bias update retains q16 learning rate resolution.

Line 209: `np.asarray([0], dtype=np.int32),` This is continuation line 3 of the 7-line logical statement that begins at source line 207. Its exact fragment is ‘np.asarray([0], dtype=np.int32),‘. The important operands are asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block; int32: the value named ’int32’ in

this block. Read together, the complete operation is: This assigns ‘MasterTensor(np.asarray([0], dtype=np.int16), np.asarray([0], dtype=np.int32), 2, -(1 << 15), (1 << 15) - 1,)‘ to ‘tensor ‘ in host memory. Here tensor means the value named ’tensor’ in this block. The important operands are MasterTensor: the value named ’master tensor’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_low_rate_bias_update_retains_q16_learning_rate_resolution, which handles test low rate bias update retains q16 learning rate resolution.

Line 210: `2,` This is continuation line 4 of the 7-line logical statement that begins at source line 207. Its exact fragment is ‘2,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(np.asarray([0], dtype=np.int16), np.asarray([0], dtype=np.int32), 2, -(1 << 15), (1 << 15) - 1,)‘ to ‘tensor ‘ in host memory. Here tensor means the value named ’tensor’ in this block. The important operands are MasterTensor: the value named ’master tensor’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_low_rate_bias_update_retains_q16_learning_rate_resol which handles test low rate bias update retains q16 learning rate resolution.

Line 211: `-(1 << 15),` This is continuation line 5 of the 7-line logical statement that begins at source line 207. Its exact fragment is ‘-(1 << 15),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(np.asarray([0], dtype=np.int16), np.asarray([0], dtype=np.int32), 2, -(1 << 15), (1 << 15) - 1,)‘ to ‘tensor ‘ in host memory. Here tensor means the value named ’tensor’ in this block. The important operands are MasterTensor: the value named ’master tensor’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_low_rate_bias_update_retains_q16_learning_rate_resolution, which handles test low rate bias update retains q16 learning rate resolution.

Line 212: `(1 << 15) - 1,` This is continuation line 6 of the 7-line logical statement that begins at source line 207. Its exact fragment is ‘(1 << 15) - 1,‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(np.asarray([0], dtype=np.int16), np.asarray([0], dtype=np.int32), 2, -(1 << 15), (1 << 15) - 1,)‘ to ‘tensor ‘ in host memory. Here tensor means the value named ’tensor’ in this block. The important operands are MasterTensor: the value named ’master tensor’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_low_rate_bias_update_retains_q16_learning_rate_resolution, which handles test low rate bias update retains q16 learning rate resolution.

Line 213: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 207. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘MasterTensor(np.asarray([0], dtype=np.int16), np.asarray([0], dtype=np.int32), 2, -(1 << 15), (1 << 15) - 1,)‘ to ‘tensor ‘ in host memory. Here tensor means the value named ’tensor’ in this block. The important operands are MasterTensor: the value named ’master tensor’ in this block; asarray: the value named ’asarray’ in this block; dtype: the value named ’dtype’ in this block. Context: the function test_low_rate_bias_update_retains_q16_learning_rate_resol which handles test low rate bias update retains q16 learning rate resolution.

Line 214: `coefficients = TrainableSpikingformerState._optimizer_coefficients(` This statement continues through source line 215; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘TrainableSpikingformerState._optimizer_coefficients(tensor, 0.01, 0.9, 0.0)‘ to ‘coefficients ‘ in host memory. Here coefficients means the value named ’coefficients’ in this block. The important operands are TrainableSpikingformerState: the finite-state-machine phase retained in flip-flops; _optimizer_coefficients: the value named ’optimizer coefficients’ in this block; tensor: the value named ’tensor’ in this block. Context: the function test_low_rate_bias_update_retains_q16_learning which handles test low rate bias update retains q16 learning rate resolution.

Line 215: `tensor, 0.01, 0.9, 0.0)` This is continuation line 2 of the 2-line logical statement that begins at source line 214. Its exact fragment is ‘tensor, 0.01, 0.9, 0.0)‘. The important operands are tensor: the value named ’tensor’ in this block. Read together, the complete operation is: This assigns ‘TrainableSpikingformerState._optimizer_coefficients(tensor, 0.01, 0.9, 0.0)‘ to ‘coefficients ‘ in host memory. Here coefficients means the value named ’coefficients’ in this block. The important operands are TrainableSpikingformerState: the finite-state-machine phase retained in flip-flops; _optimizer_coefficients: the value named ’optimizer coefficients’ in this block; tensor: the value named ’tensor’ in this

block. Context: the function test_low_rate_bias_update_retains_q16_learning_rate_resolution, which handles test low rate bias update retains q16 learning rate resolution.

Line 216: `self.assertEqual(coefficients, (655, 58982, 0, 29))` This calls 'self.assertEqual' with 'coefficients, (655, 58982, 0, 29)' inside the function test_low_rate_bias_update_retains_q16_learning_rate_resolution, which handles test low rate bias update retains q16 learning rate resolution. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 217: `with self.assertRaises(ValueError):` This opens a managed resource scope for 'self.assertRaises(ValueError)' so files, tracing, or temporary state are finalized even if the function test_low_rate_bias_update_retains_q16_learning_rate_resol which handles test low rate bias update retains q16 learning rate resolution raises an error.

Line 218: `TrainableSpikingformerState._optimizer_coefficients(` This statement continues through source line 219; the explanation uses the complete expression rather than only the first visual fragment. This calls 'TrainableSpikingformerState._optimizer_coefficients' with ' tensor, 1.0e-7, 0.9, 0.0' inside the function test_low_rate_bias_update_retains_q16_le which handles test low rate bias update retains q16 learning rate resolution. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 219: `tensor, 1.0e-7, 0.9, 0.0)` This is continuation line 2 of the 2-line logical statement that begins at source line 218. Its exact fragment is 'tensor, 1.0e-7, 0.9, 0.0)'. The important operands are tensor: the value named 'tensor' in this block. Read together, the complete operation is: This calls 'TrainableSpikingformerState._optimizer_coefficients' with ' tensor, 1.0e-7, 0.9, 0.0' inside the function test_low_rate_bias_update_retains_q16_learning_rate_resolution, which handles test low rate bias update retains q16 learning rate resolution. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 221: `def test_full_parameter_update_and_atomic_checkpoint_round_trip(self) -> None:` This starts function test_full_parameter_update_and_atomic_checkpoint_round_trip. Its indented body performs test full parameter update and atomic checkpoint round trip as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 222: `state = TrainableSpikingformerState(TinyParameterArchive())` This assigns 'TrainableSpikingformerState(TinyParameterArchive())' to 'state ' in host memory. Here state means the finite-state-machine phase retained in flip-flops. The important operands are TrainableSpikingformerState: the finite-state-machine phase retained in flip-flops; TinyParameterArchive: the value named 'tiny parameter archive' in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 223: `projection = state.projection("tiny")` This assigns 'state.projection("tiny")' to 'projection ' in host memory. Here projection means the value named 'projection' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; projection: the value named 'projection' in this block; tiny: the value named 'tiny' in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 224: `head = state.head` This assigns 'state.head' to 'head ' in host memory. Here head means the value named 'head' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; head: the value named 'head' in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 225: `gradients = {` This statement continues through source line 234; the explanation uses the complete expression rather than only the first visual fragment. This assigns '{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k 65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }' to 'gradients ' in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named 'tiny' in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a

learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 226: `"tiny": ParameterGradient(` This is continuation line 2 of the 10-line logical statement that begins at source line 225. Its exact fragment is '"tiny": ParameterGradient('. The important operands are tiny: the value named 'tiny' in this block; ParameterGradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns '{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }' to 'gradients ' in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named 'tiny' in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 227: `BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15),` This is continuation line 3 of the 10-line logical statement that begins at source line 225. Its exact fragment is 'BlockGradient(np.full(projection.weight_k_by_n
65536), -15),'. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; full: the value named 'full' in this block; projection: the value named 'projection' in this block. Read together, the complete operation is: This assigns '{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }' to 'gradients ' in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named 'tiny' in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 228: `BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),` This is continuation line 4 of the 10-line logical statement that begins at source line 225. Its exact fragment is 'BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),'. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; full: the value named 'full' in this block; projection: the value named 'projection' in this block. Read together, the complete operation is: This assigns '{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }' to 'gradients ' in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named 'tiny' in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 229: `),` This is continuation line 5 of the 10-line logical statement that begins at source line 225. Its exact fragment is '),'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns '{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }' to 'gradients ' in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named 'tiny' in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 230: `"head": ParameterGradient(` This is continuation line 6 of the 10-line logical statement that begins at source line 225. Its exact fragment is '"head": ParameterGradient('. The important operands are head: the value named 'head' in this block; ParameterGradient: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This assigns '{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k_by_n.shape,

65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }‘ to ‘gradients ‘ in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named ’tiny’ in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 231: `BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15),` This is continuation line 7 of the 10-line logical statement that begins at source line 225. Its exact fragment is ‘BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15),‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; full: the value named ’full’ in this block; head: the value named ’head’ in this block. Read together, the complete operation is: This assigns ‘{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }‘ to ‘gradients ‘ in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named ’tiny’ in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 232: `BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),` This is continuation line 8 of the 10-line logical statement that begins at source line 225. Its exact fragment is ‘BlockGradient(np.full(head.bias_accumulat
32768), -15),‘. The important operands are BlockGradient: the signed loss derivative used to update a learned parameter; full: the value named ’full’ in this block; head: the value named ’head’ in this block. Read together, the complete operation is: This assigns ‘{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }‘ to ‘gradients ‘ in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named ’tiny’ in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 233: `),` This is continuation line 9 of the 10-line logical statement that begins at source line 225. Its exact fragment is ‘),‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }‘ to ‘gradients ‘ in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named ’tiny’ in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 234: `}` This is continuation line 10 of the 10-line logical statement that begins at source line 225. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "tiny": ParameterGradient(BlockGradient(np.full(projection.weight_k_by_n.shape, 65536), -15), BlockGradient(np.full(projection.bias_q10.shape, -32768), -15),), "head": ParameterGradient(BlockGradient(np.full(head.weight_k_by_n.shape, -65536), -15), BlockGradient(np.full(head.bias_accumulator_units.shape, 32768), -15),), }‘ to ‘gradients ‘ in host memory. Here gradients means the signed loss derivative used to update a learned parameter. The important operands are tiny: the value named ’tiny’ in this block; ParameterGradient: the signed loss derivative used to update a learned parameter; BlockGradient: the signed loss derivative used to update a learned parameter. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 235: `state.apply_gradients(` This statement continues through source line 241; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘state.apply_gradients‘ with ‘ self.driver, gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ‘ inside the function test_full_parameter_update_and_atomic_checkpoi which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 236: `self.driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 235. Its exact fragment is ‘self.driver,‘. The important operands are driver: the value named ’driver’ in this block. Read together, the complete operation is: This calls ‘state.apply_gradients‘ with ‘ self.driver, gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 237: `gradients,` This is continuation line 3 of the 7-line logical statement that begins at source line 235. Its exact fragment is ‘gradients,‘. The important operands are gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls ‘state.apply_gradients‘ with ‘ self.driver, gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ‘ inside the function test_full_parameter_update_and_atomic_checkpoint_r which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 238: `learning_rate=0.25,` This is continuation line 4 of the 7-line logical statement that begins at source line 235. Its exact fragment is ‘learning_rate=0.25,‘. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This calls ‘state.apply_gradients‘ with ‘ self.driver, gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ‘ inside the function test_full_parameter_update_a which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 239: `momentum=0.0,` This is continuation line 5 of the 7-line logical statement that begins at source line 235. Its exact fragment is ‘momentum=0.0,‘. The important operands are momentum: the value named ’momentum’ in this block. Read together, the complete operation is: This calls ‘state.apply_gradients‘ with ‘ self.driver, gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ‘ inside the function test_full_parameter_update_and_atomic_checkpoint_r which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 240: `weight_decay=0.0,` This is continuation line 6 of the 7-line logical statement that begins at source line 235. Its exact fragment is ‘weight_decay=0.0,‘. The important operands are weight_decay: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls ‘state.apply_gradients‘ with ‘ self.driver, gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ‘ inside the function test_full_parameter_update_and_atomic_checkpoi which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 241: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 235. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘state.apply_gradients‘ with ‘ self.driver, gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 242: `self.assertEqual(state.optimizer_step, 1)` This calls ‘self.assertEqual‘ with ‘state.optimizer_step, 1‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 243: `self.assertFalse(np.array_equal(state.projection("tiny").weight_k_by_n,` This statement continues through source line 244; the explanation uses the complete expression rather than only the first visual fragment.

This calls ‘self.assertFalse‘ with ‘np.array_equal(state.projection("tiny").weight_k_by_n, projection.weight_k_by_n)‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 244: `projection.weight_k_by_n))` This is continuation line 2 of the 2-line logical statement that begins at source line 243. Its exact fragment is ‘projection.weight_k_by_n))‘. The important operands are projection: the value named ’projection’ in this block; weight_k_by_n: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls ‘self.assertFalse‘ with ‘np.array_equal(state.projection("tiny").weight_k_by_n, projection.weight_k_by_n)‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 245: `self.assertFalse(np.array_equal(state.head.weight_k_by_n, head.weight_k_by_n))` This calls ‘self.assertFal with ‘np.array_equal(state.head.weight_k_by_n, head.weight_k_by_n)‘ inside the function test_full_parameter_update_and_a which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 247: `with tempfile.TemporaryDirectory() as temporary:` This opens a managed resource scope for ‘tempfile.TemporaryDirectory() as temporary‘ so files, tracing, or temporary state are finalized even if the function test_full_parameter_u which handles test full parameter update and atomic checkpoint round trip raises an error.

Line 248: `checkpoint = Path(temporary) / "latest_checkpoint.npz"` This assigns ‘Path(temporary) / "latest_checkpoint.n to ‘checkpoint ‘ in host memory. Here checkpoint means the value named ’checkpoint’ in this block. The important operands are Path: the value named ’path’ in this block; temporary: the value named ’temporary’ in this block; latest_checkpoint: the value named ’latest checkpoint’ in this block. Context: the function test_full_parameter_update_and_atomic which handles test full parameter update and atomic checkpoint round trip.

Line 249: `digest = state.save_checkpoint(` This statement continues through source line 255; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘state.save_checkpoint(checkpoint, epoch=3, batch_index=17, best_accuracy_percent=95.75, extra={"reason": "unit-test"},)‘ to ‘digest ‘ in host memory. Here digest means the value named ’digest’ in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named ’save checkpoint’ in this block; checkpoint: the value named ’checkpoint’ in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 250: `checkpoint,` This is continuation line 2 of the 7-line logical statement that begins at source line 249. Its exact fragment is ‘checkpoint,‘. The important operands are checkpoint: the value named ’checkpoint’ in this block. Read together, the complete operation is: This assigns ‘state.save_checkpoint(checkpoint, epoch=3, batch_index=17, best_accuracy_percent=95.75, extra={"reason": "unit-test"},)‘ to ‘digest ‘ in host memory. Here digest means the value named ’digest’ in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named ’save checkpoint’ in this block; checkpoint: the value named ’checkpoint’ in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 251: `epoch=3,` This is continuation line 3 of the 7-line logical statement that begins at source line 249. Its exact fragment is ‘epoch=3,‘. The important operands are epoch: the value named ’epoch’ in this block. Read together, the complete operation is: This assigns ‘state.save_checkpoint(checkpoint, epoch=3, batch_index=17, best_accuracy_percent=95.75, extra={"reason": "unit-test"},)‘ to ‘digest ‘ in host memory. Here digest means the value named ’digest’ in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named ’save checkpoint’ in this block; checkpoint: the value named ’checkpoint’ in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 252: `batch_index=17,` This is continuation line 4 of the 7-line logical statement that begins at source line 249. Its exact fragment is 'batch_index=17,'. The important operands are batch_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This assigns 'state.save_checkpoint(checkpoint, epoch=3, batch_index=17, best_accuracy_percent=95.75, extra={"reason": "unit-test"},)' to 'digest ' in host memory. Here digest means the value named 'digest' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; checkpoint: the value named 'checkpoint' in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 253: `best_accuracy_percent=95.75,` This is continuation line 5 of the 7-line logical statement that begins at source line 249. Its exact fragment is 'best_accuracy_percent=95.75,'. The important operands are best_accuracy_percent: the value named 'best accuracy percent' in this block. Read together, the complete operation is: This assigns 'state.save_checkpoint(checkpoint, epoch=3, batch_index=17, best_accuracy_percent=95.75, extra={"reason": "unit-test"},)' to 'digest ' in host memory. Here digest means the value named 'digest' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; checkpoint: the value named 'checkpoint' in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 254: `extra={"reason": "unit-test"},` This is continuation line 6 of the 7-line logical statement that begins at source line 249. Its exact fragment is 'extra={"reason": "unit-test"},'. The important operands are extra: the value named 'extra' in this block; reason: the value named 'reason' in this block; unit: the value named 'unit' in this block. Read together, the complete operation is: This assigns 'state.save_checkpoint(checkpoint, epoch=3, batch_index=17, best_accuracy_percent=95.75, extra={"reason": "unit-test"},)' to 'digest ' in host memory. Here digest means the value named 'digest' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; checkpoint: the value named 'checkpoint' in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 255: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 249. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns 'state.save_checkpoint(checkpoint, epoch=3, batch_index=17, best_accuracy_percent=95.75, extra={"reason": "unit-test"},)' to 'digest ' in host memory. Here digest means the value named 'digest' in this block. The important operands are state: the finite-state-machine phase retained in flip-flops; save_checkpoint: the value named 'save checkpoint' in this block; checkpoint: the value named 'checkpoint' in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 256: `self.assertEqual(len(digest), 64)` This calls 'self.assertEqual' with 'len(digest), 64' inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 257: `restored = TrainableSpikingformerState(TinyParameterArchive())` This assigns 'TrainableSpikingformerState(TinyParameterArchive())' to 'restored ' in host memory. Here restored means the value named 'restored' in this block. The important operands are TrainableSpikingformerState: the finite-state-machine phase retained in flip-flops; TinyParameterArchive: the value named 'tiny parameter archive' in this block. Context: the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip.

Line 258: `metadata = restored.load_checkpoint(checkpoint)` This assigns 'restored.load_checkpoint(checkpoint)' to 'metadata ' in host memory. Here metadata means the value named 'metadata' in this block. The important operands are restored: the value named 'restored' in this block; load_checkpoint: the value named 'load checkpoint' in this block; checkpoint: the value named 'checkpoint' in this block. Context: the function test_full_parameter_update_and_atomic_checkpoin which handles test full parameter update and atomic checkpoint round trip.

Line 259: `self.assertEqual(metadata["epoch"], 3)` This calls ‘self.assertEqual‘ with ‘metadata["epoch"], 3‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 260: `self.assertEqual(metadata["batch_index"], 17)` This calls ‘self.assertEqual‘ with ‘metadata["batch_index"], 17‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 261: `self.assertEqual(restored.optimizer_step, 1)` This calls ‘self.assertEqual‘ with ‘restored.optimizer_step, 1‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 262: `np.testing.assert_array_equal(` This statement continues through source line 265; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘np.testing.assert_array_equal‘ with ‘ restored.projection("tiny").weight_k_by_n, state.projection("tiny").weight_k_by_n, ‘ inside the function test_full_parameter_upda which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 263: `restored.projection("tiny").weight_k_by_n,` This is continuation line 2 of the 4-line logical statement that begins at source line 262. Its exact fragment is ‘restored.projection("tiny").weight_k_by_n,‘. The important operands are restored: the value named ’restored’ in this block; projection: the value named ’projection’ in this block; tiny: the value named ’tiny’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ restored.projection("tiny").weight_k_by_n, state.projection("tiny").weight_k_by_n, ‘ inside the function test_full_paramet which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 264: `state.projection("tiny").weight_k_by_n,` This is continuation line 3 of the 4-line logical statement that begins at source line 262. Its exact fragment is ‘state.projection("tiny").weight_k_by_n,‘. The important operands are state: the finite-state-machine phase retained in flip-flops; projection: the value named ’projection’ in this block; tiny: the value named ’tiny’ in this block. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ restored.projection("tiny").weight_k_by_n, state.projection("tiny").weight_k_by_n, ‘ inside the function test_full_parameter_u which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 265: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 262. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘np.testing.assert_array_equal‘ with ‘ restored.projection("tiny").weight_k_by_n, state.projection("tiny").weight_k_by_n, ‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 266: `np.testing.assert_array_equal(restored.head.weight_k_by_n, state.head.weight_k_by_n)` This calls ‘np.testing.assert_array_equal‘ with ‘restored.head.weight_k_by_n, state.head.weight_k_by_n‘ inside the function test_full_parameter_update_and_atomic_checkpoint_round_trip, which handles test full parameter update and atomic checkpoint round trip. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 268: `def test_complete_spikingformer_topology_forward_backward_and_update(self) -> None:` This starts function test_complete_spikingformer_topology_forward_backward_and_update. Its indented body performs test complete spikingformer topology forward backward and update as part of the file that checks spikingformer training runtime behavior independently with Python assertions; it runs only when called.

Line 269: `state = TrainableSpikingformerState(TinyFullGraphArchive())` This assigns 'TrainableSpikingformerState(TinyFullGraphArchive())' to 'state ' in host memory. Here state means the finite-state-machine phase retained in flip-flops. The important operands are TrainableSpikingformerState: the finite-state-machine phase retained in flip-flops; TinyFullGraphArchive: the value named 'tiny full graph archive' in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 270: `trainer = SpikingformerOnDeviceTrainer(` This statement continues through source line 277; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'SpikingformerOnDeviceTrainer(state, self.driver, transformer_blocks=1, attention_heads=1, timesteps=2, input_shape=(3, 8, 8),)' to 'trainer ' in host memory. Here trainer means the value named 'trainer' in this block. The important operands are SpikingformerOnDeviceTrainer: the value named 'spikingformer on device trainer' in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named 'driver' in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 271: `state,` This is continuation line 2 of the 8-line logical statement that begins at source line 270. Its exact fragment is 'state,'. The important operands are state: the finite-state-machine phase retained in flip-flops. Read together, the complete operation is: This assigns 'SpikingformerOnDeviceTrainer(state, self.driver, transformer_blocks=1, attention_heads=1, timesteps=2, input_shape=(3, 8, 8),)' to 'trainer ' in host memory. Here trainer means the value named 'trainer' in this block. The important operands are SpikingformerOnDeviceTrainer: the value named 'spikingformer on device trainer' in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named 'driver' in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 272: `self.driver,` This is continuation line 3 of the 8-line logical statement that begins at source line 270. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This assigns 'SpikingformerOnDeviceTrainer(state, self.driver, transformer_blocks=1, attention_heads=1, timesteps=2, input_shape=(3, 8, 8),)' to 'trainer ' in host memory. Here trainer means the value named 'trainer' in this block. The important operands are SpikingformerOnDeviceTrainer: the value named 'spikingformer on device trainer' in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named 'driver' in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 273: `transformer_blocks=1,` This is continuation line 4 of the 8-line logical statement that begins at source line 270. Its exact fragment is 'transformer_blocks=1,'. The important operands are transformer_blocks: the value named 'transformer blocks' in this block. Read together, the complete operation is: This assigns 'SpikingformerOnDeviceTrainer(state, self.driver, transformer_blocks=1, attention_heads=1, timesteps=2, input_shape=(3, 8, 8),)' to 'trainer ' in host memory. Here trainer means the value named 'trainer' in this block. The important operands are SpikingformerOnDeviceTrainer: the value named 'spikingformer on device trainer' in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named 'driver' in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 274: `attention_heads=1,` This is continuation line 5 of the 8-line logical statement that begins at source line 270. Its exact fragment is 'attention_heads=1,'. The important operands are attention_heads: the value named 'attention heads' in this block. Read together, the complete operation is: This assigns 'SpikingformerOnDeviceTrainer(state, self.driver, transformer_blocks=1, attention_heads=1, timesteps=2, input_shape=(3, 8, 8),)' to 'trainer ' in host memory. Here trainer means the value named 'trainer' in this block. The important operands are SpikingformerOnDeviceTrainer: the value named 'spikingformer on device trainer' in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named 'driver' in this block. Context: the function test_complete_spikingformer_topology_forward_ba
which handles test complete spikingformer topology forward backward and update.

Line 275: `timesteps=2,` This is continuation line 6 of the 8-line logical statement that begins at source line 270. Its exact fragment is 'timesteps=2,'. The important operands are timesteps: one discrete SNN update instant. Read together, the complete operation is: This assigns 'SpikingformerOnDeviceTrainer(state, self.driver, transformer_blocks=1,

attention_heads=1, timesteps=2, input_shape=(3, 8, 8),)‘ to ‘trainer ‘ in host memory. Here trainer means the value named ’trainer’ in this block. The important operands are SpikingformerOnDeviceTrainer: the value named ’spikingformer on device trainer’ in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named ’driver’ in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 276: `input_shape=(3, 8, 8),` This is continuation line 7 of the 8-line logical statement that begins at source line 270. Its exact fragment is ‘input_shape=(3, 8, 8),‘. The important operands are input_shape: the value named ’input shape’ in this block. Read together, the complete operation is: This assigns ‘SpikingformerOnDeviceTrainer(state, self.driver, transformer_blocks=1, attention_heads=1, timesteps=2, input_shape=(3, 8, 8),)‘ to ‘trainer ‘ in host memory. Here trainer means the value named ’trainer’ in this block. The important operands are SpikingformerOnDeviceTrainer: the value named ’spikingformer on device trainer’ in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named ’driver’ in this block. Context: the function test_complete_spikingformer_topology_forward_ba which handles test complete spikingformer topology forward backward and update.

Line 277: `)` This is continuation line 8 of the 8-line logical statement that begins at source line 270. Its exact fragment is ‘)‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘SpikingformerOnDeviceTrainer(state, self.driver, transformer_blocks=1, attention_heads=1, timesteps=2, input_shape=(3, 8, 8),)‘ to ‘trainer ‘ in host memory. Here trainer means the value named ’trainer’ in this block. The important operands are SpikingformerOnDeviceTrainer: the value named ’spikingformer on device trainer’ in this block; state: the finite-state-machine phase retained in flip-flops; driver: the value named ’driver’ in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 278: `self.assertEqual(trainer.attention_scale_shift, 7)` This calls ‘self.assertEqual‘ with ‘trainer.attention_scale_ 7‘ inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 279: `image = np.zeros((1, 3, 8, 8), dtype=np.float32)` This assigns ‘np.zeros((1, 3, 8, 8), dtype=np.float32)‘ to ‘image ‘ in host memory. Here image means the value named ’image’ in this block. The important operands are zeros: the value named ’zeros’ in this block; dtype: the value named ’dtype’ in this block; float32: the value named ’float32’ in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 280: `cache = trainer.forward(image)` This assigns ‘trainer.forward(image)‘ to ‘cache ‘ in host memory. Here cache means the value named ’cache’ in this block. The important operands are trainer: the value named ’trainer’ in this block; forward: the value named ’forward’ in this block; image: the value named ’image’ in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 281: `self.assertEqual(cache.final_value.shape, (2, 1, 2, 2, 2))` This calls ‘self.assertEqual‘ with ‘cache.final_va (2, 1, 2, 2, 2)‘ inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 282: `self.assertEqual(len(cache.tokenizer), 5)` This calls ‘self.assertEqual‘ with ‘len(cache.tokenizer), 5‘ inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 283: `self.assertEqual(len(cache.blocks), 1)` This calls ‘self.assertEqual‘ with ‘len(cache.blocks), 1‘ inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 284: `self.assertEqual(cache.logits_accumulator.shape, (1, 3))` This calls 'self.assertEqual' with 'cache.logits_accu (1, 3)' inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 285: `backward = trainer.backward(cache, np.asarray([1]))` This assigns 'trainer.backward(cache, np.asarray([1]))' to 'backward ' in host memory. Here backward means the value named 'backward' in this block. The important operands are trainer: the value named 'trainer' in this block; backward: the value named 'backward' in this block; cache: the value named 'cache' in this block. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 286: `self.assertTrue(np.isfinite(backward.loss))` This calls 'self.assertTrue' with 'np.isfinite(backward.loss)' inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 287: `self.assertEqual(backward.input_gradient.values.shape, image.shape)` This calls 'self.assertEqual' with 'backward.input_gradient.values.shape, image.shape' inside the function test_complete_spikingformer_topology_forward_ba which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 288: `self.assertEqual(` This statement continues through source line 291; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertEqual' with ' set(backward.parameter_gradients), set(state.projection_names) | {"head"}, ' inside the function test_complete_spikingformer_topology_forward_backward_and_upd which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 289: `set(backward.parameter_gradients),` This is continuation line 2 of the 4-line logical statement that begins at source line 288. Its exact fragment is 'set(backward.parameter_gradients),'. The important operands are set: the value named 'set' in this block; backward: the value named 'backward' in this block; parameter_gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls 'self.assertEqual' with ' set(backward.parameter_gradients), set(state.projection_names) | {"head"}, ' inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 290: `set(state.projection_names) | {"head"},` This is continuation line 3 of the 4-line logical statement that begins at source line 288. Its exact fragment is 'set(state.projection_names) | {"head"},'. The important operands are set: the value named 'set' in this block; state: the finite-state-machine phase retained in flip-flops; projection_names: the value named 'projection names' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' set(backward.parameter_gradients), set(state.projection_names) | {"head"}, ' inside the function test_complete_spikingformer_top which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 291: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 288. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'self.assertEqual' with ' set(backward.parameter_gradients), set(state.projection_names) | {"head"}, ' inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 292: `nonzero_gradients = 0` This assigns '0' to 'nonzero_gradients ' in host memory. Here nonzero_gradients means the signed loss derivative used to update a learned parameter. The expression is a literal or structural operator and has no hidden software state. Context: the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 293: `for name, gradient in backward.parameter_gradients.items():` This iterates name, gradient over ‘backward.parameter_gradients.items()‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update.

Line 294: `expected_weight = state.head.weight_k_by_n.shape if name == "head" else (` This statement continues through source line 295; the explanation uses the complete expression rather than only the first visual fragment. Inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update, this statement advances the host implementation that checks spikingformer training runtime behavior independently with Python assertions. The important operands are expected_weight: a learned coefficient multiplied by an activation; state: the finite-state-machine phase retained in flip-flops; head: the value named ’head’ in this block.

Line 295: `state.projection(name).weight_k_by_n.shape)` This is continuation line 2 of the 2-line logical statement that begins at source line 294. Its exact fragment is ‘state.projection(name).weight_k_by_n.shape)‘. The important operands are state: the finite-state-machine phase retained in flip-flops; projection: the value named ’projection’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: Inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update, this statement advances the host implementation that checks spikingformer training runtime behavior independently with Python assertions. The important operands are expected_weight: a learned coefficient multiplied by an activation; state: the finite-state-machine phase retained in flip-flops; head: the value named ’head’ in this block.

Line 296: `expected_bias = state.head.bias_accumulator_units.shape if name == "head" else (` This statement continues through source line 297; the explanation uses the complete expression rather than only the first visual fragment. Inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update, this statement advances the host implementation that checks spikingformer training runtime behavior independently with Python assertions. The important operands are expected_bias: an additive learned offset applied after accumulation; state: the finite-state-machine phase retained in flip-flops; head: the value named ’head’ in this block.

Line 297: `state.projection(name).bias_q10.shape)` This is continuation line 2 of the 2-line logical statement that begins at source line 296. Its exact fragment is ‘state.projection(name).bias_q10.shape)‘. The important operands are state: the finite-state-machine phase retained in flip-flops; projection: the value named ’projection’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: Inside the function test_complete_spikingformer_to which handles test complete spikingformer topology forward backward and update, this statement advances the host implementation that checks spikingformer training runtime behavior independently with Python assertions. The important operands are expected_bias: an additive learned offset applied after accumulation; state: the finite-state-machine phase retained in flip-flops; head: the value named ’head’ in this block.

Line 298: `self.assertEqual(gradient.weight.values.shape, expected_weight)` This calls ‘self.assertEqual‘ with ‘gradient.weight.values.shape, expected_weight‘ inside the function test_complete_spikingformer_topology_forward_backward_a which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 299: `self.assertEqual(gradient.bias.values.shape, expected_bias)` This calls ‘self.assertEqual‘ with ‘gradient.bias.values.shape, expected_bias‘ inside the function test_complete_spikingformer_topology_forward_backward_and_u which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 300: `nonzero_gradients += int(np.count_nonzero(gradient.weight.values))` Inside the function test_complete_spi which handles test complete spikingformer topology forward backward and update, this statement advances the host implementation that checks spikingformer training runtime behavior independently with Python assertions. The important operands are nonzero_gradients: the signed loss derivative used to update a learned parameter; count_nonzero: the value named ’count nonzero’ in this block; gradient: the signed loss derivative used to update a learned parameter.

Line 301: `nonzero_gradients += int(np.count_nonzero(gradient.bias.values))` Inside the function test_complete_spiki which handles test complete spikingformer topology forward backward and update, this statement advances the host implementation that checks spikingformer training runtime behavior independently with Python assertions. The important operands are nonzero_gradients: the signed loss derivative used to update a learned parameter; count_nonzero: the value named 'count nonzero' in this block; gradient: the signed loss derivative used to update a learned parameter.

Line 302: `self.assertGreater(nonzero_gradients, 0)` This calls 'self.assertGreater' with 'nonzero_gradients, 0' inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 303: `state.apply_gradients(` This statement continues through source line 309; the explanation uses the complete expression rather than only the first visual fragment. This calls 'state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ' inside the function test_complete_spikingfor which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 304: `self.driver,` This is continuation line 2 of the 7-line logical statement that begins at source line 303. Its exact fragment is 'self.driver,'. The important operands are driver: the value named 'driver' in this block. Read together, the complete operation is: This calls 'state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ' inside the function test_complete_spikingformer_topology_forward_backwa which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 305: `backward.parameter_gradients,` This is continuation line 3 of the 7-line logical statement that begins at source line 303. Its exact fragment is 'backward.parameter_gradients,'. The important operands are backward: the value named 'backward' in this block; parameter_gradients: the signed loss derivative used to update a learned parameter. Read together, the complete operation is: This calls 'state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ' inside the function test_complete_spikingformer_topology_forward_ba which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 306: `learning_rate=0.25,` This is continuation line 4 of the 7-line logical statement that begins at source line 303. Its exact fragment is 'learning_rate=0.25,'. The important operands are learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This calls 'state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ' inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 307: `momentum=0.0,` This is continuation line 5 of the 7-line logical statement that begins at source line 303. Its exact fragment is 'momentum=0.0,'. The important operands are momentum: the value named 'momentum' in this block. Read together, the complete operation is: This calls 'state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ' inside the function test_complete_spikingfor which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 308: `weight_decay=0.0,` This is continuation line 6 of the 7-line logical statement that begins at source line 303. Its exact fragment is 'weight_decay=0.0,'. The important operands are weight_decay: a learned coefficient multiplied by an activation. Read together, the complete operation is: This calls 'state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ' inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 309: `)` This is continuation line 7 of the 7-line logical statement that begins at source line 303. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls 'state.apply_gradients' with ' self.driver, backward.parameter_gradients, learning_rate=0.25, momentum=0.0, weight_decay=0.0, ' inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 310: `self.assertEqual(state.optimizer_step, 1)` This calls 'self.assertEqual' with 'state.optimizer_step, 1' inside the function test_complete_spikingformer_topology_forward_backward_and_update, which handles test complete spikingformer topology forward backward and update. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

Line 313: `if __name__ == "__main__":` This executes the following indented host-code branch only when '___name___ == "___main___"' is true. The important operands are ___name___: the value named 'name' in this block; ___main___: the value named 'main' in this block. Context: the file whose role is to checks spikingformer training runtime behavior independently with Python assertions.

Line 314: `unittest.main()` This calls 'unittest.main' with " inside the file whose role is to checks spikingformer training runtime behavior independently with Python assertions. The call advances the host-side job that checks spikingformer training runtime behavior independently with Python assertions.

76 tests/test_train_spikingformer_pynq.py

Role in the complete system: checks train spikingformer pynq behavior independently with Python assertions.

76.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to checks train spikingformer pynq behavior independently with Python assertions.

Where it runs This is sequential host software. Python objects live in CPU memory; they become FPGA work only when the compiler packs a command or payload and the PYNQ driver transfers those bytes through AXI DMA or AXI-Lite.

Public structure Classes: PynqTrainingJobTests. Functions: sha256, test_shards_are_integrity_checked_deterministic_and_r test_augmentation_and_cosine_schedule_are_reproducible. Each function annotation below names its enclosing function so an array operation is tied to the model, compiler, driver, test, or report phase that uses it.

Numeric and failure boundary Shapes, signedness, byte order, scaling, and saturation are part of the hardware contract. Validation exceptions are functional safety checks: rejecting malformed input before DMA is different from silently clipping, reshaping, or continuing with an incompatible artifact.

76.2 Complete source

```
from __future__ import annotations

import hashlib
import json
import tempfile
import unittest
from pathlib import Path

import numpy as np

from train_spikingformer_pynq import (
    Cifar10ShardSet,
    augment_cifar10,
    cosine_learning_rate,
)


def sha256(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()


class PynqTrainingJobTests(unittest.TestCase):
    def test_shards_are_integrity_checked_deterministic_and_resumable(self) -> None:
        with tempfile.TemporaryDirectory() as temporary:
            root = Path(temporary)
            entries = []
            first_index = 0
            for shard_index, samples in enumerate((5, 4)):
                path = root / f"cifar10_train_{shard_index:03d}.npz"
                images = np.full((samples, 32, 32, 3), shard_index * 40, dtype=np.uint8)
                labels = np.arange(first_index, first_index + samples, dtype=np.uint8) % 10
                np.savez_compressed(path, images=images, labels=labels)
                entries.append({
                    "file": path.name,
                    "samples": samples,
                    "first_index": first_index,
                    "last_index_exclusive": first_index + samples,
                    "bytes": path.stat().st_size,
                    "sha256": sha256(path),
                })
                first_index += samples
            manifest = {
                "format": "neuraedge-official-cifar10-shards-v1",
                "splits": {"train": entries, "test": entries},
            }
            manifest_path = root / "cifar10_shards_manifest.json"
            manifest_path.write_text(json.dumps(manifest), encoding="ascii")
            dataset = Cifar10ShardSet(manifest_path, "train")
            first = list(dataset.batches(
                epoch=7, batch_size=2, seed=19, shuffle=True))
            second = list(dataset.batches(
                epoch=7, batch_size=2, seed=19, shuffle=True))
            self.assertEqual([item[0] for item in first], list(range(len(first))))
```

```
            self.assertEqual(
                [item[2].tolist() for item in first],
                [item[2].tolist() for item in second],
            )
            resumed = list(dataset.batches(
                epoch=7, batch_size=2, seed=19, shuffle=True, start_batch=3))
            self.assertEqual([item[0] for item in resumed], list(range(3, len(first))))
            self.assertEqual(sum(len(item[2]) for item in first), 9)

            manifest["splits"]["train"][0]["sha256"] = "0" * 64
            manifest_path.write_text(json.dumps(manifest), encoding="ascii")
            with self.assertRaises(ValueError):
                Cifar10ShardSet(manifest_path, "train")

    def test_augmentation_and_cosine_schedule_are_reproducible(self) -> None:
        images = np.arange(2 * 32 * 32 * 3, dtype=np.uint16).reshape(2, 32, 32, 3)
        images = (images % 256).astype(np.uint8)
        first = augment_cifar10(images, 1234)
        second = augment_cifar10(images, 1234)
        np.testing.assert_array_equal(first, second)
        self.assertEqual(first.shape, images.shape)
        self.assertEqual(cosine_learning_rate(0, 400, 0.25, 0.01), 0.25)
        self.assertAlmostEqual(cosine_learning_rate(399, 400, 0.25, 0.01), 0.01)


if __name__ == "__main__":
    unittest.main()
```

76.3 Line annotations

Line 1: `from __future__ import annotations` This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 3: `import hashlib` This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 4: `import json` This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 5: `import tempfile` This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 6: `import unittest` This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 7: `from pathlib import Path` This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 9: `import numpy as np` This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 11: `from train_spikingformer_pynq import (` This statement continues through source line 15; the explanation uses the complete expression rather than only the first visual fragment. This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 12: `Cifar10ShardSet,` This is continuation line 2 of the 5-line logical statement that begins at source line 11. Its exact fragment is 'Cifar10ShardSet,'. The important operands are Cifar10ShardSet: the value named 'cifar10 shard set' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 13: `augment_cifar10,` This is continuation line 3 of the 5-line logical statement that begins at source line 11. Its exact fragment is 'augment_cifar10,'. The important operands are augment_cifar10: the value named 'augment cifar10' in this block. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 14: `cosine_learning_rate,` This is continuation line 4 of the 5-line logical statement that begins at source line 11. Its exact fragment is 'cosine_learning_rate,'. The important operands are cosine_learning_rate: the fixed-point multiplier controlling update magnitude. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 15: `)` This is continuation line 5 of the 5-line logical statement that begins at source line 11. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This imports the named Python API so the file whose role is to checks train spikingformer pynq behavior independently with Python assertions can use it explicitly. Imports execute on the host processor and create no FPGA hardware.

Line 18: `def sha256(path: Path) -> str:` This starts function sha256. Its indented body performs sha256 as part of the file that checks train spikingformer pynq behavior independently with Python assertions; it runs only when called.

Line 19: `return hashlib.sha256(path.read_bytes()).hexdigest()` This returns 'hashlib.sha256(path.read_bytes()).hexdige
to the caller of the function sha256, which handles sha256, ending this invocation. The important operands are hashlib: the value named 'hashlib' in this block; sha256: the value named 'sha256' in this block; path: the value named 'path' in this block.

Line 22: `class PynqTrainingJobTests(unittest.TestCase):` This defines class PynqTrainingJobTests, which groups data and methods for pynq training job tests. Instances exist in host memory and organize the flow that checks train spikingformer pynq behavior independently with Python assertions.

Line 23: `def test_shards_are_integrity_checked_deterministic_and_resumable(self) -> None:` This starts function test_shards_are_integrity_checked_deterministic_and_resumable. Its indented body performs test shards are integrity checked deterministic and resumable as part of the file that checks train spikingformer pynq behavior independently with Python assertions; it runs only when called.

Line 24: `with tempfile.TemporaryDirectory() as temporary:` This opens a managed resource scope for 'tempfile.TemporaryDirectory() as temporary' so files, tracing, or temporary state are finalized even if the function test_shards_are_integr
which handles test shards are integrity checked deterministic and resumable raises an error.

Line 25: `root = Path(temporary)` This assigns 'Path(temporary)' to 'root ' in host memory. Here root means the value named 'root' in this block. The important operands are Path: the value named 'path' in this block; temporary: the value named 'temporary' in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 26: `entries = []` This assigns '[]' to 'entries ' in host memory. Here entries means the value named 'entries' in this block. The expression is a literal or structural operator and has no hidden software state. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 27: `first_index = 0` This assigns ‘0‘ to ‘first_index ‘ in host memory. Here first_index means an integer selecting one tensor element, lane, bank, or state entry. The expression is a literal or structural operator and has no hidden software state. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 28: `for shard_index, samples in enumerate((5, 4)):` This iterates shard_index, samples over ‘enumerate((5, 4))‘. Each iteration repeats the indented host operation, commonly once per layer, tile, timestep, or test case in the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 29: `path = root / f"cifar10_train_{shard_index:03d}.npz"` This assigns ‘root / f"cifar10_train_{shard_index:03d}.
to ‘path ‘ in host memory. Here path means the value named ’path’ in this block. The important operands are root: the value named ’root’ in this block; f: the value named ’f’ in this block; cifar10_train_: the value named ’cifar10 train’ in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 30: `images = np.full((samples, 32, 32, 3), shard_index * 40, dtype=np.uint8)` This assigns ‘np.full((samples, 32, 32, 3), shard_index * 40, dtype=np.uint8)‘ to ‘images ‘ in host memory. Here images means the value named ’images’ in this block. The important operands are full: the value named ’full’ in this block; samples: the value named ’samples’ in this block; shard_index: an integer selecting one tensor element, lane, bank, or state entry. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 31: `labels = np.arange(first_index, first_index + samples, dtype=np.uint8) % 10` This assigns ‘np.arange(first_
first_index + samples, dtype=np.uint8) % 10‘ to ‘labels ‘ in host memory. Here labels means the value named ’labels’ in this block. The important operands are arange: the value named ’arange’ in this block; first_index: an integer selecting one tensor element, lane, bank, or state entry; samples: the value named ’samples’ in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 32: `np.savez_compressed(path, images=images, labels=labels)` This calls ‘np.savez_compressed‘ with ‘path, images=images, labels=labels‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 33: `entries.append({` This statement continues through source line 40; the explanation uses the complete expression rather than only the first visual fragment. This calls ‘entries.append‘ with ‘{ "file": path.name, "samples": samples, "first_index": first_index, "last_index_exclusive": first_index + samples, "bytes": path.stat().st_size, "sha256": sha256(path), }‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 34: `"file": path.name,` This is continuation line 2 of the 8-line logical statement that begins at source line 33. Its exact fragment is ‘"file": path.name,‘. The important operands are file: the value named ’file’ in this block; path: the value named ’path’ in this block; name: the value named ’name’ in this block. Read together, the complete operation is: This calls ‘entries.append‘ with ‘{ "file": path.name, "samples": samples, "first_index": first_index, "last_index_exclusive": first_index + samples, "bytes": path.stat().st_size, "sha256": sha256(path), }‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 35: `"samples": samples,` This is continuation line 3 of the 8-line logical statement that begins at source line 33. Its exact fragment is ‘"samples": samples,‘. The important operands are samples: the value named ’samples’ in this block. Read together, the complete operation is: This calls ‘entries.append‘ with ‘{ "file": path.name, "samples": samples, "first_index": first_index, "last_index_exclusive": first_index + samples, "bytes": path.stat().st_size, "sha256":

sha256(path), }‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 36: `"first_index": first_index,` This is continuation line 4 of the 8-line logical statement that begins at source line 33. Its exact fragment is ‘"first_index": first_index,‘. The important operands are first_index: an integer selecting one tensor element, lane, bank, or state entry. Read together, the complete operation is: This calls ‘entries.append‘ with ‘{ "file": path.name, "samples": samples, "first_index": first_index, "last_index_exclusive": first_index + samples, "bytes": path.stat().st_size, "sha256": sha256(path), }‘ inside the function test_shards_are_integrity_checked_deterministic_and_ which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 37: `"last_index_exclusive": first_index + samples,` This is continuation line 5 of the 8-line logical statement that begins at source line 33. Its exact fragment is ‘"last_index_exclusive": first_index + samples,‘. The important operands are last_index_exclusive: the AXI-Stream marker for the final beat of a packet or command payload; first_index: an integer selecting one tensor element, lane, bank, or state entry; samples: the value named 'samples' in this block. Read together, the complete operation is: This calls ‘entries.append‘ with ‘{ "file": path.name, "samples": samples, "first_index": first_index, "last_index_exclusive": first_index + samples, "bytes": path.stat().st_size, "sha256": sha256(path), }‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 38: `"bytes": path.stat().st_size,` This is continuation line 6 of the 8-line logical statement that begins at source line 33. Its exact fragment is ‘"bytes": path.stat().st_size,‘. The important operands are bytes: the value named 'bytes' in this block; path: the value named 'path' in this block; stat: the value named 'stat' in this block. Read together, the complete operation is: This calls ‘entries.append‘ with ‘{ "file": path.name, "samples": samples, "first_index": first_index, "last_index_exclusive": first_index + samples, "bytes": path.stat().st_size, "sha256": sha256(path), }‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 39: `"sha256": sha256(path),` This is continuation line 7 of the 8-line logical statement that begins at source line 33. Its exact fragment is ‘"sha256": sha256(path),‘. The important operands are sha256: the value named 'sha256' in this block; path: the value named 'path' in this block. Read together, the complete operation is: This calls ‘entries.append‘ with ‘{ "file": path.name, "samples": samples, "first_index": first_index, "last_index_exclusive": first_index + samples, "bytes": path.stat().st_size, "sha256": sha256(path), }‘ inside the function test_shards_are_integrity_checked_deterministic_and_ which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 40: `})` This is continuation line 8 of the 8-line logical statement that begins at source line 33. Its exact fragment is ‘})‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This calls ‘entries.append‘ with ‘{ "file": path.name, "samples": samples, "first_index": first_index, "last_index_exclusive": first_index + samples, "bytes": path.stat().st_size, "sha256": sha256(path), }‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 41: `first_index += samples` Inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable, this statement advances the host implementation that checks train spikingformer pynq behavior independently with Python assertions. The important operands are first_index: an integer selecting one tensor element, lane, bank, or state entry; samples: the value named 'samples' in this block.

Line 42: `manifest = {` This statement continues through source line 45; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "splits": {"train": entries, "test": entries}, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named

'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; official: the value named 'official' in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 43: `"format": "neuraedge-official-cifar10-shards-v1",` This is continuation line 2 of the 4-line logical statement that begins at source line 42. Its exact fragment is ‘"format": "neuraedge-official-cifar10-shards-v1",‘. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; official: the value named 'official' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "splits": {"train": entries, "test": entries}, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; official: the value named 'official' in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 44: `"splits": {"train": entries, "test": entries},` This is continuation line 3 of the 4-line logical statement that begins at source line 42. Its exact fragment is ‘"splits": {"train": entries, "test": entries},‘. The important operands are splits: the value named 'splits' in this block; train: the value named 'train' in this block; entries: the value named 'entries' in this block. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "splits": {"train": entries, "test": entries}, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; official: the value named 'official' in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 45: `}` This is continuation line 4 of the 4-line logical statement that begins at source line 42. Its exact fragment is ‘}‘. The expression is a literal or structural operator and has no hidden software state. Read together, the complete operation is: This assigns ‘{ "format": "neuraedge-official-cifar10-shards-v1", "splits": {"train": entries, "test": entries}, }‘ to ‘manifest ‘ in host memory. Here manifest means the value named 'manifest' in this block. The important operands are format: the value named 'format' in this block; neuraedge: the value named 'neuraedge' in this block; official: the value named 'official' in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 46: `manifest_path = root / "cifar10_shards_manifest.json"` This assigns ‘root / "cifar10_shards_manifest.json"‘ to ‘manifest_path ‘ in host memory. Here manifest_path means the value named 'manifest path' in this block. The important operands are root: the value named 'root' in this block; cifar10_shards_manifest: the value named 'cifar10 shards manifest' in this block; json: the value named 'json' in this block. Context: the function test_shards_are_integrity_checked_determ which handles test shards are integrity checked deterministic and resumable.

Line 47: `manifest_path.write_text(json.dumps(manifest), encoding="ascii")` This calls ‘manifest_path.write_text‘ with ‘json.dumps(manifest), encoding="ascii"‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumab which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 48: `dataset = Cifar10ShardSet(manifest_path, "train")` This assigns ‘Cifar10ShardSet(manifest_path, "train")‘ to ‘dataset ‘ in host memory. Here dataset means the value named 'dataset' in this block. The important operands are Cifar10ShardSet: the value named 'cifar10 shard set' in this block; manifest_path: the value named 'manifest path' in this block; train: the value named 'train' in this block. Context: the function test_shards_are_integrity_checked_deterministic_an which handles test shards are integrity checked deterministic and resumable.

Line 49: `first = list(dataset.batches(` This statement continues through source line 50; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘list(dataset.batches(epoch=7, batch_size=2, seed=19, shuffle=True))‘ to ‘first ‘ in host memory. Here first means the value named 'first' in this block. The important operands are list: the value named 'list' in this block; dataset: the value named 'dataset' in this block; batches: the value named 'batches' in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 50: `epoch=7, batch_size=2, seed=19, shuffle=True))` This is continuation line 2 of the 2-line logical statement that begins at source line 49. Its exact fragment is 'epoch=7, batch_size=2, seed=19, shuffle=True))'. The important operands are epoch: the value named 'epoch' in this block; batch_size: the value named 'batch size' in this block; seed: the value named 'seed' in this block. Read together, the complete operation is: This assigns 'list(dataset.batches(epoch=7, batch_size=2, seed=19, shuffle=True))' to 'first ' in host memory. Here first means the value named 'first' in this block. The important operands are list: the value named 'list' in this block; dataset: the value named 'dataset' in this block; batches: the value named 'batches' in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 51: `second = list(dataset.batches(` This statement continues through source line 52; the explanation uses the complete expression rather than only the first visual fragment. This assigns 'list(dataset.batches(epoch=7, batch_size=2, seed=19, shuffle=True))' to 'second ' in host memory. Here second means the value named 'second' in this block. The important operands are list: the value named 'list' in this block; dataset: the value named 'dataset' in this block; batches: the value named 'batches' in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 52: `epoch=7, batch_size=2, seed=19, shuffle=True))` This is continuation line 2 of the 2-line logical statement that begins at source line 51. Its exact fragment is 'epoch=7, batch_size=2, seed=19, shuffle=True))'. The important operands are epoch: the value named 'epoch' in this block; batch_size: the value named 'batch size' in this block; seed: the value named 'seed' in this block. Read together, the complete operation is: This assigns 'list(dataset.batches(epoch=7, batch_size=2, seed=19, shuffle=True))' to 'second ' in host memory. Here second means the value named 'second' in this block. The important operands are list: the value named 'list' in this block; dataset: the value named 'dataset' in this block; batches: the value named 'batches' in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 53: `self.assertEqual([item[0] for item in first], list(range(len(first))))` This calls 'self.assertEqual' with '[item[0] for item in first], list(range(len(first)))' inside the function test_shards_are_integrity_checked_deterministic_and_re which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 54: `self.assertEqual(` This statement continues through source line 57; the explanation uses the complete expression rather than only the first visual fragment. This calls 'self.assertEqual' with ' [item[2].tolist() for item in first], [item[2].tolist() for item in second], ' inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 55: `[item[2].tolist() for item in first],` This is continuation line 2 of the 4-line logical statement that begins at source line 54. Its exact fragment is '[item[2].tolist() for item in first],'. The important operands are item: the value named 'item' in this block; tolist: the value named 'tolist' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' [item[2].tolist() for item in first], [item[2].tolist() for item in second], ' inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 56: `[item[2].tolist() for item in second],` This is continuation line 3 of the 4-line logical statement that begins at source line 54. Its exact fragment is '[item[2].tolist() for item in second],'. The important operands are item: the value named 'item' in this block; tolist: the value named 'tolist' in this block; for: the value named 'for' in this block. Read together, the complete operation is: This calls 'self.assertEqual' with ' [item[2].tolist() for item in first], [item[2].tolist() for item in second], ' inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 57: `)` This is continuation line 4 of the 4-line logical statement that begins at source line 54. Its exact fragment is ')'. The expression is a literal or structural operator and has no hidden software state. Read together, the complete

operation is: This calls ‘self.assertEqual‘ with ‘ [item[2].tolist() for item in first], [item[2].tolist() for item in second], ‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 58: `resumed = list(dataset.batches(` This statement continues through source line 59; the explanation uses the complete expression rather than only the first visual fragment. This assigns ‘list(dataset.batches(epoch=7, batch_size=2, seed=19, shuffle=True, start_batch=3))‘ to ‘resumed ‘ in host memory. Here resumed means the value named ’resumed’ in this block. The important operands are list: the value named ’list’ in this block; dataset: the value named ’dataset’ in this block; batches: the value named ’batches’ in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 59: `epoch=7, batch_size=2, seed=19, shuffle=True, start_batch=3))` This is continuation line 2 of the 2-line logical statement that begins at source line 58. Its exact fragment is ‘epoch=7, batch_size=2, seed=19, shuffle=True, start_batch=3))‘. The important operands are epoch: the value named ’epoch’ in this block; batch_size: the value named ’batch size’ in this block; seed: the value named ’seed’ in this block. Read together, the complete operation is: This assigns ‘list(dataset.batches(epoch=7, batch_size=2, seed=19, shuffle=True, start_batch=3))‘ to ‘resumed ‘ in host memory. Here resumed means the value named ’resumed’ in this block. The important operands are list: the value named ’list’ in this block; dataset: the value named ’dataset’ in this block; batches: the value named ’batches’ in this block. Context: the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable.

Line 60: `self.assertEqual([item[0] for item in resumed], list(range(3, len(first))))` This calls ‘self.assertEqual‘ with ‘[item[0] for item in resumed], list(range(3, len(first)))‘ inside the function test_shards_are_integrity_checked_deterministic_a which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 61: `self.assertEqual(sum(len(item[2]) for item in first), 9)` This calls ‘self.assertEqual‘ with ‘sum(len(item[2]) for item in first), 9‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 63: `manifest["splits"]["train"][0]["sha256"] = "0" * 64` Inside the function test_shards_are_integrity_checked_ which handles test shards are integrity checked deterministic and resumable, this statement advances the host implementation that checks train spikingformer pynq behavior independently with Python assertions. The important operands are manifest: the value named ’manifest’ in this block; splits: the value named ’splits’ in this block; train: the value named ’train’ in this block.

Line 64: `manifest_path.write_text(json.dumps(manifest), encoding="ascii")` This calls ‘manifest_path.write_text‘ with ‘json.dumps(manifest), encoding="ascii"‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumab which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 65: `with self.assertRaises(ValueError):` This opens a managed resource scope for ‘self.assertRaises(ValueError)‘ so files, tracing, or temporary state are finalized even if the function test_shards_are_integrity_checked_deterministic_and_resuma which handles test shards are integrity checked deterministic and resumable raises an error.

Line 66: `Cifar10ShardSet(manifest_path, "train")` This calls ‘Cifar10ShardSet‘ with ‘manifest_path, "train"‘ inside the function test_shards_are_integrity_checked_deterministic_and_resumable, which handles test shards are integrity checked deterministic and resumable. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 68: `def test_augmentation_and_cosine_schedule_are_reproducible(self) -> None:` This starts function test_augmentation_and_cosine_schedule_are_reproducible. Its indented body performs test augmentation and cosine schedule are reproducible as part of the file that checks train spikingformer pynq behavior independently with Python assertions; it runs only when called.

Line 69: `images = np.arange(2 * 32 * 32 * 3, dtype=np.uint16).reshape(2, 32, 32, 3)` This assigns ‘np.arange(2 * 32 * 32 * 3, dtype=np.uint16).reshape(2, 32, 32, 3)‘ to ‘images ‘ in host memory. Here images means the value named ’images’ in this block. The important operands are arange: the value named ’arange’ in this block; dtype: the value named ’dtype’ in this block; uint16: the value named ’uint16’ in this block. Context: the function test_augmentation_and_cosine_schedule which handles test augmentation and cosine schedule are reproducible.

Line 70: `images = (images % 256).astype(np.uint8)` This assigns ‘(images % 256).astype(np.uint8)‘ to ‘images ‘ in host memory. Here images means the value named ’images’ in this block. The important operands are images: the value named ’images’ in this block; astype: the value named ’astype’ in this block; uint8: the value named ’uint8’ in this block. Context: the function test_augmentation_and_cosine_schedule_are_reproducible, which handles test augmentation and cosine schedule are reproducible.

Line 71: `first = augment_cifar10(images, 1234)` This assigns ‘augment_cifar10(images, 1234)‘ to ‘first ‘ in host memory. Here first means the value named ’first’ in this block. The important operands are augment_cifar10: the value named ’augment cifar10’ in this block; images: the value named ’images’ in this block. Context: the function test_augmentation_and_cosine_schedule_are_reproducible, which handles test augmentation and cosine schedule are reproducible.

Line 72: `second = augment_cifar10(images, 1234)` This assigns ‘augment_cifar10(images, 1234)‘ to ‘second ‘ in host memory. Here second means the value named ’second’ in this block. The important operands are augment_cifar10: the value named ’augment cifar10’ in this block; images: the value named ’images’ in this block. Context: the function test_augmentation_and_cosine_schedule_are_reproducible, which handles test augmentation and cosine schedule are reproducible.

Line 73: `np.testing.assert_array_equal(first, second)` This calls ‘np.testing.assert_array_equal‘ with ‘first, second‘ inside the function test_augmentation_and_cosine_schedule_are_reproducible, which handles test augmentation and cosine schedule are reproducible. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 74: `self.assertEqual(first.shape, images.shape)` This calls ‘self.assertEqual‘ with ‘first.shape, images.shape‘ inside the function test_augmentation_and_cosine_schedule_are_reproducible, which handles test augmentation and cosine schedule are reproducible. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 75: `self.assertEqual(cosine_learning_rate(0, 400, 0.25, 0.01), 0.25)` This calls ‘self.assertEqual‘ with ‘cosine_learning_rate(0, 400, 0.25, 0.01), 0.25‘ inside the function test_augmentation_and_cosine_schedule_are_reproducible, which handles test augmentation and cosine schedule are reproducible. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 76: `self.assertAlmostEqual(cosine_learning_rate(399, 400, 0.25, 0.01), 0.01)` This calls ‘self.assertAlmostEqua with ‘cosine_learning_rate(399, 400, 0.25, 0.01), 0.01‘ inside the function test_augmentation_and_cosine_schedule_are_reproduci which handles test augmentation and cosine schedule are reproducible. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

Line 79: `if __name__ == "__main__":` This executes the following indented host-code branch only when ‘___name___ == "___main___"‘ is true. The important operands are ___name___: the value named ’name’ in this block; ___main___: the value named ’main’ in this block. Context: the file whose role is to checks train spikingformer pynq behavior independently with Python assertions.

Line 80: `unittest.main()` This calls ‘unittest.main‘ with “ inside the file whose role is to checks train spikingformer pynq behavior independently with Python assertions. The call advances the host-side job that checks train spikingformer pynq behavior independently with Python assertions.

77 scripts/capture_power_activity.ps1

Role in the complete system: orchestrates the repeatable Windows capture power activity command-line flow.

77.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows capture power activity command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

77.2 Complete source

```
[CmdletBinding()]
param(
    [string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",
    [string]$OutputDirectory = ""
)

$ErrorActionPreference = "Stop"
Set-StrictMode -Version Latest

$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path
if ([string]::IsNullOrWhiteSpace($OutputDirectory)) {
    $OutputDirectory = Join-Path $ProjectRoot "outputs\power_activity"
}
$OutputDirectory = [System.IO.Path]::GetFullPath($OutputDirectory)
$WorkDirectory = Join-Path $OutputDirectory "xsim_work"
$LogDirectory = Join-Path $OutputDirectory "logs"
$SaifDirectory = Join-Path $OutputDirectory "saif"
foreach ($Directory in @($OutputDirectory, $WorkDirectory, $LogDirectory, $SaifDirectory)) {
    New-Item -ItemType Directory -Force -Path $Directory | Out-Null
}
$FixtureStage = Join-Path $WorkDirectory "tb\fixtures"
New-Item -ItemType Directory -Force -Path $FixtureStage | Out-Null
Copy-Item -Path (Join-Path $ProjectRoot "tb\fixtures\*") -Destination $FixtureStage -Force

$VivadoBin = Join-Path $VivadoRoot "bin"
$Xvlog = Join-Path $VivadoBin "xvlog.bat"
$Xelab = Join-Path $VivadoBin "xelab.bat"
$Xsim = Join-Path $VivadoBin "xsim.bat"
foreach ($Tool in @($Xvlog, $Xelab, $Xsim)) {
    if (-not (Test-Path -LiteralPath $Tool)) {
        throw "Required Vivado simulator tool was not found: $Tool"
    }
}

$PackageFile = Join-Path $ProjectRoot "rtl\neuraedge_command_pkg.sv"
$RtlFiles = @($PackageFile) + @(
    Get-ChildItem -LiteralPath (Join-Path $ProjectRoot "rtl") -Filter "*.sv" -File |
        Where-Object { $_.FullName -ne $PackageFile } |
        Sort-Object Name |
        ForEach-Object FullName
)
$CaptureTcl = (Join-Path $PSScriptRoot "capture_power_activity.tcl").Replace('\', '/')
$Scenarios = @(
    [PSCustomObject]@{
        Top = "tb_neuraedge_pynq_axi_top"
        Scope = "/tb_neuraedge_pynq_axi_top/dut"
        Amount = 5
        Unit = "us"
    },
    [PSCustomObject]@{
        Top = "tb_neuraedge_universal_core"
        Scope = "/tb_neuraedge_universal_core/dut"
        Amount = 50
        Unit = "us"
    }
)
$ManifestRows = [System.Collections.Generic.List[object]]::new()

Push-Location $WorkDirectory
try {
```

```
    & $Xvlog -sv -i $ProjectRoot @RtlFiles 2>&1 |
        Tee-Object -FilePath (Join-Path $LogDirectory "rtl_compile.log")
    if ($LASTEXITCODE -ne 0) {
        throw "RTL xvlog compilation failed with exit code $LASTEXITCODE"
    }

    foreach ($Scenario in $Scenarios) {
        $Top = $Scenario.Top
        $Snapshot = "${Top}_power_snapshot"
        $SaifPath = Join-Path $SaifDirectory "$Top.saif"
        Remove-Item -LiteralPath $SaifPath -Force -ErrorAction SilentlyContinue
        & $Xvlog -sv -i $ProjectRoot (Join-Path $ProjectRoot "tb\$Top.sv") 2>&1 |
            Tee-Object -FilePath (Join-Path $LogDirectory "$Top.compile.log")
        if ($LASTEXITCODE -ne 0) {
            throw "$Top xvlog compilation failed with exit code $LASTEXITCODE"
        }
        & $Xelab -debug typical -timescale 1ns/1ps -snapshot $Snapshot "work.$Top" 2>&1 |
            Tee-Object -FilePath (Join-Path $LogDirectory "$Top.elaborate.log")
        if ($LASTEXITCODE -ne 0) {
            throw "$Top xelab failed with exit code $LASTEXITCODE"
        }

        $env:NEURAEDGE_SAIF_PATH = $SaifPath
        $env:NEURAEDGE_SAIF_SCOPE = $Scenario.Scope
        $env:NEURAEDGE_SAIF_AMOUNT = [string]$Scenario.Amount
        $env:NEURAEDGE_SAIF_UNIT = $Scenario.Unit
        & $Xsim $Snapshot -onerror quit -onfinish quit -tclbatch $CaptureTcl 2>&1 |
            Tee-Object -FilePath (Join-Path $LogDirectory "$Top.simulate.log")
        if ($LASTEXITCODE -ne 0) {
            throw "$Top XSim SAIF capture failed with exit code $LASTEXITCODE"
        }
        if (-not (Test-Path -LiteralPath $SaifPath) -or
                (Get-Item -LiteralPath $SaifPath).Length -eq 0) {
            throw "$Top did not produce a nonempty SAIF"
        }
        $ManifestRows.Add([PSCustomObject]@{
            testbench = $Top
            scope = $Scenario.Scope
            duration = $Scenario.Amount
            unit = $Scenario.Unit
            saif = [System.IO.Path]::GetRelativePath($OutputDirectory, $SaifPath)
            saif_bytes = (Get-Item -LiteralPath $SaifPath).Length
        })
    }
} finally {
    foreach ($Name in @("NEURAEDGE_SAIF_PATH", "NEURAEDGE_SAIF_SCOPE",
            "NEURAEDGE_SAIF_AMOUNT", "NEURAEDGE_SAIF_UNIT")) {
        Remove-Item "Env:$Name" -ErrorAction SilentlyContinue
    }
    Pop-Location
}

$Manifest = [ordered]@{
    format = "neuraedge-xsim-saif-v1"
    vivado_root = $VivadoRoot
    target_part = "xc7z020clg400-1"
    target_clock_mhz = 200
    captures = @($ManifestRows)
}
$ManifestPath = Join-Path $OutputDirectory "power_activity_manifest.json"
$Manifest | ConvertTo-Json -Depth 5 | Set-Content -LiteralPath $ManifestPath -Encoding utf8
Write-Output "NEURAEDGE_POWER_ACTIVITY_CAPTURED count=$($ManifestRows.Count) manifest=$ManifestPath"
```

77.3 Line annotations

Line 1: `[CmdletBinding()]` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are CmdletBinding: the value named 'cmdlet binding' in this block.

Line 2: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `[string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 4: `[string]$OutputDirectory = ""` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 5: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 7: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 8: `Set-StrictMode -Version Latest` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Set: the value named 'set' in this block; StrictMode: the value named 'strict mode' in this block; Version: the value named 'version' in this block.

Line 10: `$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows capture power activity command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 11: `if ([string]::IsNullOrWhiteSpace($OutputDirectory)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are string: the value named 'string' in this block; IsNullOrWhiteSpace: the value named 'is null or white space' in this block; OutputDirectory: the value named 'output directory' in this block.

Line 12: `$OutputDirectory = Join-Path $ProjectRoot "outputs\power_activity"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are OutputDirectory: the value named 'output directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 13: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 14: `$OutputDirectory = [System.IO.Path]::GetFullPath($OutputDirectory)` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows capture power activity command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 15: `$WorkDirectory = Join-Path $OutputDirectory "xsim_work"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 16: `$LogDirectory = Join-Path $OutputDirectory "logs"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are LogDirectory: the value named 'log directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 17: `$SaifDirectory = Join-Path $OutputDirectory "saif"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are SaifDirectory: the value named 'saif directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 18: `foreach ($Directory in @($OutputDirectory, $WorkDirectory, $LogDirectory, $SaifDirectory)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are foreach: the value named 'foreach' in this block; Directory: the value named 'directory' in this block; in: the value named 'in' in this block.

Line 19: `New-Item -ItemType Directory -Force -Path $Directory | Out-Null` This creates the declared output directory before a tool writes into it. '-Force' permits an existing directory but does not make an external tool failure succeed.

Line 20: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 21: `$FixtureStage = Join-Path $WorkDirectory "tb\fixtures"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are FixtureStage: the value named 'fixture stage' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 22: `New-Item -ItemType Directory -Force -Path $FixtureStage | Out-Null` This creates the declared output directory before a tool writes into it. '-Force' permits an existing directory but does not make an external tool failure succeed.

Line 23: `Copy-Item -Path (Join-Path $ProjectRoot "tb\fixtures\*") -Destination $FixtureStage -Force` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Copy: the value named 'copy' in this block; Item: the value named 'item' in this block; Path: the value named 'path' in this block.

Line 25: `$VivadoBin = Join-Path $VivadoRoot "bin"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 26: `$Xvlog = Join-Path $VivadoBin "xvlog.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 27: `$Xelab = Join-Path $VivadoBin "xelab.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 28: `$Xsim = Join-Path $VivadoBin "xsim.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 29: `foreach ($Tool in @($Xvlog, $Xelab, $Xsim)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 30: `if (-not (Test-Path -LiteralPath $Tool)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are not: the value named 'not' in this block; Test: the value named 'test' in this block; Path: the value named 'path' in this block.

Line 31: `throw "Required Vivado simulator tool was not found: $Tool"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 32: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 33: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 35: `$PackageFile = Join-Path $ProjectRoot "rtl\neuraedge_command_pkg.sv"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are PackageFile: the value named 'package file' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 36: `$RtlFiles = @($PackageFile) + @(` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are RtlFiles: the value named 'rtl files' in this block; PackageFile: the value named 'package file' in this block.

Line 37: `Get-ChildItem -LiteralPath (Join-Path $ProjectRoot "rtl") -Filter "*.sv" -File |` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Get: the value named 'get' in this block; ChildItem: the value named 'child item' in this block; LiteralPath: the value named 'literal path' in this block.

Line 38: `Where-Object { $_.FullName -ne $PackageFile } |` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Where: the value named 'where' in this block; Object: the value named 'object' in this block; _: the value named '' in this block.

Line 39: `Sort-Object Name |` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Sort: the value named 'sort' in this block; Object: the value named 'object' in this block; Name: the value named 'name' in this block.

Line 40: `ForEach-Object FullName` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are ForEach: the value named 'for each' in this block; Object: the value named 'object' in this block; FullName: the value named 'full name' in this block.

Line 41: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 42: `$CaptureTcl = (Join-Path $PSScriptRoot "capture_power_activity.tcl").Replace('\', '/')` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are CaptureTcl: the value named 'capture tcl' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 43: `$Scenarios = @(` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Scenarios: the value named 'scenarios' in this block.

Line 44: `[PSCustomObject]@{` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are PSCustomObject: the value named 'pscustom object' in this block.

Line 45: `Top = "tb_neuraedge_pynq_axi_top"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Top: the value named 'top' in this block; tb_neuraedge_pynq_axi_top: the value named 'tb neuraedge pynq axi top' in this block.

Line 46: `Scope = "/tb_neuraedge_pynq_axi_top/dut"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Scope: the value named 'scope' in this block; tb_neuraedge_pynq_axi_top: the value named 'tb neuraedge pynq axi top' in this block; dut: the value named 'dut' in this block.

Line 47: `Amount = 5` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Amount: the value named 'amount' in this block.

Line 48: `Unit = "us"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Unit: the value named 'unit' in this block; us: the value named 'us' in this block.

Line 49: `},` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 50: `[PSCustomObject]@{` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are PSCustomObject: the value named 'pscustom object' in this block.

Line 51: `Top = "tb_neuraedge_universal_core"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Top: the value named 'top' in this block; tb_neuraedge_universal_core: the value named 'tb neuraedge universal core' in this block.

Line 52: `Scope = "/tb_neuraedge_universal_core/dut"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Scope: the value named 'scope' in this block; tb_neuraedge_universal_core: the value named 'tb neuraedge universal core' in this block; dut: the value named 'dut' in this block.

Line 53: `Amount = 50` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Amount: the value named 'amount' in this block.

Line 54: `Unit = "us"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Unit: the value named 'unit' in this block; us: the value named 'us' in this block.

Line 55: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 56: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 57: `$ManifestRows = [System.Collections.Generic.List[object]]::new()` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are ManifestRows: one output-row position processed by the tensor tile; System: the value named 'system' in this block; Collections: the value named 'collections' in this block.

Line 59: `Push-Location $WorkDirectory` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Push: the value named 'push' in this block; Location: the value named 'location' in this block; WorkDirectory: the value named 'work directory' in this block.

Line 60: `try {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are try: the value named 'try' in this block.

Line 61: `& $Xvlog -sv -i $ProjectRoot @RtlFiles 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 62: `Tee-Object -FilePath (Join-Path $LogDirectory "rtl_compile.log")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 63: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 64: `throw "RTL xvlog compilation failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 65: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 67: `foreach ($Scenario in $Scenarios) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are foreach: the value named 'foreach' in this block; Scenario: the value named 'scenario' in this block; in: the value named 'in' in this block.

Line 68: `$Top = $Scenario.Top` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Top: the value named 'top' in this block; Scenario: the value named 'scenario' in this block.

Line 69: `$Snapshot = "${Top}_power_snapshot"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Snapshot: the value named 'snapshot' in this block; Top: the value named 'top' in this block; _power_snapshot: the value named 'power snapshot' in this block.

Line 70: `$SaifPath = Join-Path $SaifDirectory "$Top.saif"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are SaifPath: the value named 'saif path' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 71: `Remove-Item -LiteralPath $SaifPath -Force -ErrorAction SilentlyContinue` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 72: `& $Xvlog -sv -i $ProjectRoot (Join-Path $ProjectRoot "tb\$Top.sv") 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 73: `Tee-Object -FilePath (Join-Path $LogDirectory "$Top.compile.log")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 74: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 75: `throw "$Top xvlog compilation failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 76: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 77: `& $Xelab -debug typical -timescale 1ns/1ps -snapshot $Snapshot "work.$Top" 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 78: `Tee-Object -FilePath (Join-Path $LogDirectory "$Top.elaborate.log")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 79: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 80: `throw "$Top xelab failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 81: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 83: `$env:NEURAEDGE_SAIF_PATH = $SaifPath` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are env: the value named 'env' in this block; NEURAEDGE_SAIF_PATH: the value named 'neuraedge saif path' in this block; SaifPath: the value named 'saif path' in this block.

Line 84: `$env:NEURAEDGE_SAIF_SCOPE = $Scenario.Scope` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are env: the value named 'env' in this block; NEURAEDGE_SAIF_SCOPE: the value named 'neuraedge saif scope' in this block; Scenario: the value named 'scenario' in this block.

Line 85: `$env:NEURAEDGE_SAIF_AMOUNT = [string]$Scenario.Amount` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are env: the value named 'env' in this block; NEURAEDGE_SAIF_AMOUNT: the value named 'neuraedge saif amount' in this block; string: the value named 'string' in this block.

Line 86: `$env:NEURAEDGE_SAIF_UNIT = $Scenario.Unit` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are env: the value named 'env' in this block; NEURAEDGE_SAIF_UNIT: the value named 'neuraedge saif unit' in this block; Scenario: the value named 'scenario' in this block.

Line 87: `& $Xsim $Snapshot -onerror quit -onfinish quit -tclbatch $CaptureTcl 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 88: `Tee-Object -FilePath (Join-Path $LogDirectory "$Top.simulate.log")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 89: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 90: `throw "$Top XSim SAIF capture failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 91: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 92: `if (-not (Test-Path -LiteralPath $SaifPath) -or` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are not: the value named 'not' in this block; Test: the value named 'test' in this block; Path: the value named 'path' in this block.

Line 93: `(Get-Item -LiteralPath $SaifPath).Length -eq 0) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Get: the value named 'get' in this block; Item: the value named 'item' in this block; LiteralPath: the value named 'literal path' in this block.

Line 94: `throw "$Top did not produce a nonempty SAIF"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are throw: one output-row position processed by the tensor tile; Top: the value named 'top' in this block; did: the value named 'did' in this block.

Line 95: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 96: `$ManifestRows.Add([PSCustomObject]@{` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are ManifestRows: one output-row position processed by the tensor tile; Add: the value named 'add' in this block; PSCustomObject: the value named 'pscustom object' in this block.

Line 97: `testbench = $Top` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are testbench: the value named 'testbench' in this block; Top: the value named 'top' in this block.

Line 98: `scope = $Scenario.Scope` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are scope: the value named 'scope' in this block; Scenario: the value named 'scenario' in this block; Scope: the value named 'scope' in this block.

Line 99: `duration = $Scenario.Amount` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are duration: the value named 'duration' in this block; Scenario: the value named 'scenario' in this block; Amount: the value named 'amount' in this block.

Line 100: `unit = $Scenario.Unit` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are unit: the value named 'unit' in this block; Scenario: the value named 'scenario' in this block; Unit: the value named 'unit' in this block.

Line 101: `saif = [System.IO.Path]::GetRelativePath($OutputDirectory, $SaifPath)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are saif: the value named 'saif' in this block; System: the value named 'system' in this block; IO: the value named 'io' in this block.

Line 102: `saif_bytes = (Get-Item -LiteralPath $SaifPath).Length` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are saif_bytes: the value named 'saif bytes' in this block; Get: the value named 'get' in this block; Item: the value named 'item' in this block.

Line 103: `})` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 104: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 105: `} finally {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are finally: the value named 'finally' in this block.

Line 106: `foreach ($Name in @("NEURAEDGE_SAIF_PATH", "NEURAEDGE_SAIF_SCOPE",` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are foreach: the value named 'foreach' in this block; Name: the value named 'name' in this block; in: the value named 'in' in this block.

Line 107: `"NEURAEDGE_SAIF_AMOUNT", "NEURAEDGE_SAIF_UNIT")) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are NEURAEDGE_SAIF_AMOUNT: the value named 'neuraedge saif amount' in this block; NEURAEDGE_SAIF_UNIT: the value named 'neuraedge saif unit' in this block.

Line 108: `Remove-Item "Env:$Name" -ErrorAction SilentlyContinue` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 109: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 110: `Pop-Location` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Pop: the value named 'pop' in this block; Location: the value named 'location' in this block.

Line 111: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 113: `$Manifest = [ordered]@{` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Manifest: the value named 'manifest' in this block; ordered: the value named 'ordered' in this block.

Line 114: `format = "neuraedge-xsim-saif-v1"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 115: `vivado_root = $VivadoRoot` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture power activity command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 116: `target_part = "xc7z020clg400-1"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are target_part: the value named 'target part' in this block; xc7z020clg400: the value named 'xc7z020clg400' in this block.

Line 117: `target_clock_mhz = 200` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are target_clock_mhz: the value named 'target clock mhz' in this block.

Line 118: `captures = @($ManifestRows)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are captures: the value named 'captures' in this block; ManifestRows: one output-row position processed by the tensor tile.

Line 119: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 120: `$ManifestPath = Join-Path $OutputDirectory "power_activity_manifest.json"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are ManifestPath: the value named 'manifest path' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 121: `$Manifest | ConvertTo-Json -Depth 5 | Set-Content -LiteralPath $ManifestPath -Encoding utf8` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Manifest: the value named 'manifest' in this block; ConvertTo: the value named 'convert to' in this block; Json: the value named 'json' in this block.

Line 122: `Write-Output "NEURAEDGE_POWER_ACTIVITY_CAPTURED count=$($ManifestRows.Count) manifest=$ManifestPath` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture power activity command-line flow. The important operands are Write: the value named 'write' in this block; Output: the value named 'output' in this block; NEURAEDGE_POWER_ACTIVITY_CAPTURED: the value named 'neuraedge power activity captured' in this block.

78 scripts/capture_waveforms.ps1

Role in the complete system: orchestrates the repeatable Windows capture waveforms command-line flow.

78.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows capture waveforms command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

78.2 Complete source

```
[CmdletBinding()]
param(
    [string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",
    [string]$OutputDirectory = "",
    [string[]]$Testbench = @()
)

$ErrorActionPreference = "Stop"
Set-StrictMode -Version Latest

$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path
if ([string]::IsNullOrWhiteSpace($OutputDirectory)) {
    $OutputDirectory = Join-Path $ProjectRoot "outputs\waveforms"
}
$OutputDirectory = [System.IO.Path]::GetFullPath($OutputDirectory)
$WorkDirectory = Join-Path $OutputDirectory "xsim_work"
$VcdDirectory = Join-Path $OutputDirectory "vcd"
$WdbDirectory = Join-Path $OutputDirectory "wdb"
$LogDirectory = Join-Path $OutputDirectory "logs"
foreach ($Directory in @($OutputDirectory, $WorkDirectory, $VcdDirectory, $WdbDirectory, $LogDirectory)) {
    New-Item -ItemType Directory -Force -Path $Directory | Out-Null
}
$FixtureStage = Join-Path $WorkDirectory "tb\fixtures"
New-Item -ItemType Directory -Force -Path $FixtureStage | Out-Null
Copy-Item -Path (Join-Path $ProjectRoot "tb\fixtures\*") -Destination $FixtureStage -Force

$VivadoBin = Join-Path $VivadoRoot "bin"
$Xvlog = Join-Path $VivadoBin "xvlog.bat"
$Xelab = Join-Path $VivadoBin "xelab.bat"
$Xsim = Join-Path $VivadoBin "xsim.bat"
foreach ($Tool in @($Xvlog, $Xelab, $Xsim)) {
    if (-not (Test-Path -LiteralPath $Tool)) {
        throw "Required Vivado simulator tool was not found: $Tool"
    }
}

$Windows = [ordered]@{
    "tb_neuraedge_mac2_u8" = @(2, "us")
    "tb_neuraedge_tensor_tile" = @(3, "us")
    "tb_neuraedge_buffered_tensor_core" = @(4, "us")
    "tb_neuraedge_alpha_xnor" = @(40, "us")
    "tb_neuraedge_vector_alu" = @(170, "us")
    "tb_neuraedge_integer_primitives" = @(90, "us")
    "tb_neuraedge_normalization_units" = @(405, "us")
    "tb_neuraedge_lif_bank" = @(20, "us")
    "tb_neuraedge_linear_backward8" = @(1, "ms")
    "tb_neuraedge_sgd_optimizer8" = @(20, "us")
    "tb_neuraedge_activation_lut8" = @(20, "us")
    "tb_neuraedge_universal_core" = @(50, "us")
    "tb_neuraedge_descriptor_sequencer" = @(2, "us")
    "tb_neuraedge_pynq_descriptor" = @(2, "us")
    "tb_neuraedge_pynq_axi_top" = @(5, "us")
}

if ($Testbench.Count -eq 0) {
    $Selected = @($Windows.Keys)
} else {
    $Selected = @($Testbench)
    foreach ($Top in $Selected) {
        if (-not $Windows.Contains($Top)) {
```

```
            throw "Unknown waveform testbench '$Top'"
        }
    }
}

$PackageFile = Join-Path $ProjectRoot "rtl\neuraedge_command_pkg.sv"
$RtlFiles = @($PackageFile) + @(
    Get-ChildItem -LiteralPath (Join-Path $ProjectRoot "rtl") -Filter "*.sv" -File |
        Where-Object { $_.FullName -ne $PackageFile } |
        Sort-Object Name |
        ForEach-Object FullName
)
$CaptureTcl = Join-Path $PSScriptRoot "capture_waveform.tcl"
$CaptureTclArgument = $CaptureTcl.Replace('\', '/')
$ManifestRows = [System.Collections.Generic.List[object]]::new()

Push-Location $WorkDirectory
try {
    & $Xvlog -sv -i $ProjectRoot @RtlFiles 2>&1 |
        Tee-Object -FilePath (Join-Path $LogDirectory "rtl_compile.log")
    if ($LASTEXITCODE -ne 0) {
        throw "RTL xvlog compilation failed with exit code $LASTEXITCODE"
    }

    foreach ($Top in $Selected) {
        $TestbenchFile = Join-Path $ProjectRoot "tb\$Top.sv"
        $CompileLog = Join-Path $LogDirectory "$Top.compile.log"
        $ElaborateLog = Join-Path $LogDirectory "$Top.elaborate.log"
        $SimulationLog = Join-Path $LogDirectory "$Top.simulate.log"
        $Snapshot = "${Top}_wave_snapshot"
        $VcdPath = Join-Path $VcdDirectory "$Top.vcd"
        $WdbPath = Join-Path $WdbDirectory "$Top.wdb"
        $Amount = [int]$Windows[$Top][0]
        $Unit = [string]$Windows[$Top][1]
        Remove-Item -LiteralPath $VcdPath -Force -ErrorAction SilentlyContinue
        Remove-Item -LiteralPath $WdbPath -Force -ErrorAction SilentlyContinue

        & $Xvlog -sv -i $ProjectRoot $TestbenchFile 2>&1 |
            Tee-Object -FilePath $CompileLog
        if ($LASTEXITCODE -ne 0) {
            throw "$Top xvlog compilation failed with exit code $LASTEXITCODE"
        }

        & $Xelab -debug typical -timescale 1ns/1ps -snapshot $Snapshot "work.$Top" 2>&1 |
            Tee-Object -FilePath $ElaborateLog
        if ($LASTEXITCODE -ne 0) {
            throw "$Top xelab failed with exit code $LASTEXITCODE"
        }

        $env:NEURAEDGE_VCD_PATH = $VcdPath
        $env:NEURAEDGE_WAVE_AMOUNT = [string]$Amount
        $env:NEURAEDGE_WAVE_UNIT = $Unit
        & $Xsim $Snapshot -onerror quit -onfinish quit -wdb $WdbPath `
            -tclbatch $CaptureTclArgument 2>&1 |
            Tee-Object -FilePath $SimulationLog
        if ($LASTEXITCODE -ne 0) {
            throw "$Top XSim waveform capture failed with exit code $LASTEXITCODE"
        }
        if (-not (Test-Path -LiteralPath $VcdPath) -or (Get-Item -LiteralPath $VcdPath).Length -eq 0) {
            throw "$Top did not produce a nonempty VCD"
        }

        $ManifestRows.Add([PSCustomObject]@{
            testbench = $Top
            duration = $Amount
            unit = $Unit
            vcd = [System.IO.Path]::GetRelativePath($OutputDirectory, $VcdPath)
            vcd_bytes = (Get-Item -LiteralPath $VcdPath).Length
            wdb = [System.IO.Path]::GetRelativePath($OutputDirectory, $WdbPath)
            wdb_bytes = (Get-Item -LiteralPath $WdbPath).Length
            simulation_log = [System.IO.Path]::GetRelativePath($OutputDirectory, $SimulationLog)
        })
    }
} finally {
    Remove-Item Env:NEURAEDGE_VCD_PATH -ErrorAction SilentlyContinue
    Remove-Item Env:NEURAEDGE_WAVE_AMOUNT -ErrorAction SilentlyContinue
    Remove-Item Env:NEURAEDGE_WAVE_UNIT -ErrorAction SilentlyContinue
    Pop-Location
}

$Manifest = [ordered]@{
    format = "neuraedge-xsim-waveforms-v1"
    vivado_root = $VivadoRoot
    target_part = "xc7z020clg400-1"
    target_clock_mhz = 200
    captures = @($ManifestRows)
}
```

```
$ManifestPath = Join-Path $OutputDirectory "waveform_manifest.json"
$Manifest | ConvertTo-Json -Depth 5 | Set-Content -LiteralPath $ManifestPath -Encoding utf8
Write-Output "NEURAEDGE_WAVEFORMS_CAPTURED count=$($ManifestRows.Count) manifest=$ManifestPath"
```

78.3 Line annotations

Line 1: `[CmdletBinding()]` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are CmdletBinding: the value named 'cmdlet binding' in this block.

Line 2: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `[string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 4: `[string]$OutputDirectory = "",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 5: `[string[]]$Testbench = @()` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are string: the value named 'string' in this block; Testbench: the value named 'testbench' in this block.

Line 6: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 8: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 9: `Set-StrictMode -Version Latest` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Set: the value named 'set' in this block; StrictMode: the value named 'strict mode' in this block; Version: the value named 'version' in this block.

Line 11: `$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows capture waveforms command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 12: `if ([string]::IsNullOrWhiteSpace($OutputDirectory)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are string: the value named 'string' in this block; IsNullOrWhiteSpace: the value named 'is null or white space' in this block; OutputDirectory: the value named 'output directory' in this block.

Line 13: `$OutputDirectory = Join-Path $ProjectRoot "outputs\waveforms"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are OutputDirectory: the value named 'output directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 14: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 15: `$OutputDirectory = [System.IO.Path]::GetFullPath($OutputDirectory)` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows capture waveforms command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 16: `$WorkDirectory = Join-Path $OutputDirectory "xsim_work"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 17: `$VcdDirectory = Join-Path $OutputDirectory "vcd"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are VcdDirectory: the value named 'vcd directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 18: `$WdbDirectory = Join-Path $OutputDirectory "wdb"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are WdbDirectory: the value named 'wdb directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 19: `$LogDirectory = Join-Path $OutputDirectory "logs"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are LogDirectory: the value named 'log directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 20: `foreach ($Directory in @($OutputDirectory, $WorkDirectory, $VcdDirectory, $WdbDirectory, $LogDirecto`
`{` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are foreach: the value named 'foreach' in this block; Directory: the value named 'directory' in this block; in: the value named 'in' in this block.

Line 21: `New-Item -ItemType Directory -Force -Path $Directory | Out-Null` This creates the declared output directory before a tool writes into it. '-Force' permits an existing directory but does not make an external tool failure succeed.

Line 22: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 23: `$FixtureStage = Join-Path $WorkDirectory "tb\fixtures"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are FixtureStage: the value named 'fixture stage' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 24: `New-Item -ItemType Directory -Force -Path $FixtureStage | Out-Null` This creates the declared output directory before a tool writes into it. '-Force' permits an existing directory but does not make an external tool failure succeed.

Line 25: `Copy-Item -Path (Join-Path $ProjectRoot "tb\fixtures\*") -Destination $FixtureStage -Force`
This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Copy: the value named 'copy' in this block; Item: the value named 'item' in this block; Path: the value named 'path' in this block.

Line 27: `$VivadoBin = Join-Path $VivadoRoot "bin"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 28: `$Xvlog = Join-Path $VivadoBin "xvlog.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 29: `$Xelab = Join-Path $VivadoBin "xelab.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 30: `$Xsim = Join-Path $VivadoBin "xsim.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 31: `foreach ($Tool in @($Xvlog, $Xelab, $Xsim)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 32: `if (-not (Test-Path -LiteralPath $Tool)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are not: the value named 'not' in this block; Test: the value named 'test' in this block; Path: the value named 'path' in this block.

Line 33: `throw "Required Vivado simulator tool was not found: $Tool"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 34: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 35: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 37: `$Windows = [ordered]@{` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Windows: the value named 'windows' in this block; ordered: the value named 'ordered' in this block.

Line 38: `"tb_neuraedge_mac2_u8" = @(2, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_mac2_u8: the value named 'tb neuraedge mac2 u8' in this block; us: the value named 'us' in this block.

Line 39: `"tb_neuraedge_tensor_tile" = @(3, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_tensor_tile: the value named 'tb neuraedge tensor tile' in this block; us: the value named 'us' in this block.

Line 40: `"tb_neuraedge_buffered_tensor_core" = @(4, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_buffered_te local storage that decouples producer and consumer timing; us: the value named 'us' in this block.

Line 41: `"tb_neuraedge_alpha_xnor" = @(40, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_alpha_xnor: the programmable weight assigned to matching zero spikes in alpha-XNOR attention; us: the value named 'us' in this block.

Line 42: `"tb_neuraedge_vector_alu" = @(170, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_vector_alu: the value named 'tb neuraedge vector alu' in this block; us: the value named 'us' in this block.

Line 43: `"tb_neuraedge_integer_primitives" = @(90, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_integer_pri the value named 'tb neuraedge integer primitives' in this block; us: the value named 'us' in this block.

Line 44: `"tb_neuraedge_normalization_units" = @(405, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_norma the value named 'tb neuraedge normalization units' in this block; us: the value named 'us' in this block.

Line 45: `"tb_neuraedge_lif_bank" = @(20, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_lif_bank: the value named 'tb neuraedge lif bank' in this block; us: the value named 'us' in this block.

Line 46: `"tb_neuraedge_linear_backward8" = @(1, "ms")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_linear_backw the value named 'tb neuraedge linear backward8' in this block; ms: the value named 'ms' in this block.

Line 47: `"tb_neuraedge_sgd_optimizer8" = @(20, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_sgd_optimize the value named 'tb neuraedge sgd optimizer8' in this block; us: the value named 'us' in this block.

Line 48: `"tb_neuraedge_activation_lut8" = @(20, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_activation_lu an input feature value consumed by a neuron or multiply-accumulate operation; us: the value named 'us' in this block.

Line 49: `"tb_neuraedge_universal_core" = @(50, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_universal_co the value named 'tb neuraedge universal core' in this block; us: the value named 'us' in this block.

Line 50: `"tb_neuraedge_descriptor_sequencer" = @(2, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_descriptor_ a DMA record header plus one command and its declared payload; us: the value named 'us' in this block.

Line 51: `"tb_neuraedge_pynq_descriptor" = @(2, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_pynq_descrip a DMA record header plus one command and its declared payload; us: the value named 'us' in this block.

Line 52: `"tb_neuraedge_pynq_axi_top" = @(5, "us")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tb_neuraedge_pynq_axi_top: the value named 'tb neuraedge pynq axi top' in this block; us: the value named 'us' in this block.

Line 53: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 55: `if ($Testbench.Count -eq 0) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Testbench: the value named 'testbench' in this block; Count: the value named 'count' in this block; eq: the value named 'eq' in this block.

Line 56: `$Selected = @($Windows.Keys)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Selected: the value named 'selected' in this block; Windows: the value named 'windows' in this block; Keys: the value named 'keys' in this block.

Line 57: `} else {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 58: `$Selected = @($Testbench)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Selected: the value named 'selected' in this block; Testbench: the value named 'testbench' in this block.

Line 59: `foreach ($Top in $Selected) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are foreach: the value named 'foreach' in this block; Top: the value named 'top' in this block; in: the value named 'in' in this block.

Line 60: `if (-not $Windows.Contains($Top)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are not: the value named 'not' in this block; Windows: the value named 'windows' in this block; Contains: the value named 'contains' in this block.

Line 61: `throw "Unknown waveform testbench '$Top'"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are throw: one output-row position processed by the tensor tile; Unknown: the value named 'unknown' in this block; waveform: the value named 'waveform' in this block.

Line 62: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 63: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 64: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 66: `$PackageFile = Join-Path $ProjectRoot "rtl\neuraedge_command_pkg.sv"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are PackageFile: the value named 'package file' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 67: `$RtlFiles = @($PackageFile) + @(` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are RtlFiles: the value named 'rtl files' in this block; PackageFile: the value named 'package file' in this block.

Line 68: `Get-ChildItem -LiteralPath (Join-Path $ProjectRoot "rtl") -Filter "*.sv" -File |` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Get: the value named 'get' in this block; ChildItem: the value named 'child item' in this block; LiteralPath: the value named 'literal path' in this block.

Line 69: `Where-Object { $_.FullName -ne $PackageFile } |` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Where: the value named 'where' in this block; Object: the value named 'object' in this block; _: the value named '' in this block.

Line 70: `Sort-Object Name |` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Sort: the value named 'sort' in this block; Object: the value named 'object' in this block; Name: the value named 'name' in this block.

Line 71: `ForEach-Object FullName` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are ForEach: the value named 'for each' in this block; Object: the value named 'object' in this block; FullName: the value named 'full name' in this block.

Line 72: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 73: `$CaptureTcl = Join-Path $PSScriptRoot "capture_waveform.tcl"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are CaptureTcl: the value named 'capture tcl' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 74: `$CaptureTclArgument = $CaptureTcl.Replace('\', '/')` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are CaptureTclArgument: the value named 'capture tcl argument' in this block; CaptureTcl: the value named 'capture tcl' in this block; Replace: the value named 'replace' in this block.

Line 75: `$ManifestRows = [System.Collections.Generic.List[object]]::new()` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are ManifestRows: one output-row position processed by the tensor tile; System: the value named 'system' in this block; Collections: the value named 'collections' in this block.

Line 77: `Push-Location $WorkDirectory` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Push: the value named 'push' in this block; Location: the value named 'location' in this block; WorkDirectory: the value named 'work directory' in this block.

Line 78: `try {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are try: the value named 'try' in this block.

Line 79: `& $Xvlog -sv -i $ProjectRoot @RtlFiles 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 80: `Tee-Object -FilePath (Join-Path $LogDirectory "rtl_compile.log")` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 81: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 82: `throw "RTL xvlog compilation failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 83: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 85: `foreach ($Top in $Selected) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are foreach: the value named 'foreach' in this block; Top: the value named 'top' in this block; in: the value named 'in' in this block.

Line 86: `$TestbenchFile = Join-Path $ProjectRoot "tb\$Top.sv"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are TestbenchFile: the value named 'testbench file' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 87: `$CompileLog = Join-Path $LogDirectory "$Top.compile.log"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are CompileLog: the value named 'compile log' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 88: `$ElaborateLog = Join-Path $LogDirectory "$Top.elaborate.log"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are ElaborateLog: the value named 'elaborate log' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 89: `$SimulationLog = Join-Path $LogDirectory "$Top.simulate.log"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are SimulationLog: the value named 'simulation log' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 90: `$Snapshot = "${Top}_wave_snapshot"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Snapshot: the value named 'snapshot' in this block; Top: the value named 'top' in this block; _wave_snapshot: the value named 'wave snapshot' in this block.

Line 91: `$VcdPath = Join-Path $VcdDirectory "$Top.vcd"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are VcdPath: the value named 'vcd path' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 92: `$WdbPath = Join-Path $WdbDirectory "$Top.wdb"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are WdbPath: the value named 'wdb path' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 93: `$Amount = [int]$Windows[$Top][0]` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Amount: the value named 'amount' in this block; Windows: the value named 'windows' in this block; Top: the value named 'top' in this block.

Line 94: `$Unit = [string]$Windows[$Top][1]` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Unit: the value named 'unit' in this block; string: the value named 'string' in this block; Windows: the value named 'windows' in this block.

Line 95: `Remove-Item -LiteralPath $VcdPath -Force -ErrorAction SilentlyContinue` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 96: `Remove-Item -LiteralPath $WdbPath -Force -ErrorAction SilentlyContinue` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 98: `& $Xvlog -sv -i $ProjectRoot $TestbenchFile 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 99: `Tee-Object -FilePath $CompileLog` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 100: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 101: `throw "$Top xvlog compilation failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 102: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 104: `& $Xelab -debug typical -timescale 1ns/1ps -snapshot $Snapshot "work.$Top" 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 105: `Tee-Object -FilePath $ElaborateLog` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 106: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 107: `throw "$Top xelab failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 108: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 110: `$env:NEURAEDGE_VCD_PATH = $VcdPath` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are env: the value named 'env' in this block; NEURAEDGE_VCD_PATH: the value named 'neuraedge vcd path' in this block; VcdPath: the value named 'vcd path' in this block.

Line 111: `$env:NEURAEDGE_WAVE_AMOUNT = [string]$Amount` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are env: the value named 'env' in this block; NEURAEDGE_WAVE_AMOUNT: the value named 'neuraedge wave amount' in this block; string: the value named 'string' in this block.

Line 112: `$env:NEURAEDGE_WAVE_UNIT = $Unit` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are env: the value named 'env' in this block; NEURAEDGE_WAVE_UNIT: the value named 'neuraedge wave unit' in this block; Unit: the value named 'unit' in this block.

Line 113: `& $Xsim $Snapshot -onerror quit -onfinish quit -wdb $WdbPath ‘` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 114: `-tclbatch $CaptureTclArgument 2>&1 |` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are tclbatch: the value named 'tclbatch' in this block; CaptureTclArgument: the value named 'capture tcl argument' in this block.

Line 115: `Tee-Object -FilePath $SimulationLog` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 116: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 117: `throw "$Top XSim waveform capture failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 118: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 119: `if (-not (Test-Path -LiteralPath $VcdPath) -or (Get-Item -LiteralPath $VcdPath).Length -eq 0) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are not: the value named 'not' in this block; Test: the value named 'test' in this block; Path: the value named 'path' in this block.

Line 120: `throw "$Top did not produce a nonempty VCD"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are throw: one output-row position processed by the tensor tile; Top: the value named 'top' in this block; did: the value named 'did' in this block.

Line 121: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 123: `$ManifestRows.Add([PSCustomObject]@{` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are ManifestRows: one output-row position processed by the tensor tile; Add: the value named 'add' in this block; PSCustomObject: the value named 'pscustom object' in this block.

Line 124: `testbench = $Top` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are testbench: the value named 'testbench' in this block; Top: the value named 'top' in this block.

Line 125: `duration = $Amount` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are duration: the value named 'duration' in this block; Amount: the value named 'amount' in this block.

Line 126: `unit = $Unit` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are unit: the value named 'unit' in this block; Unit: the value named 'unit' in this block.

Line 127: `vcd = [System.IO.Path]::GetRelativePath($OutputDirectory, $VcdPath)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are vcd: the value named 'vcd' in this block; System: the value named 'system' in this block; IO: the value named 'io' in this block.

Line 128: `vcd_bytes = (Get-Item -LiteralPath $VcdPath).Length` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are vcd_bytes: the value named 'vcd bytes' in this block; Get: the value named 'get' in this block; Item: the value named 'item' in this block.

Line 129: `wdb = [System.IO.Path]::GetRelativePath($OutputDirectory, $WdbPath)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are wdb: the value named 'wdb' in this block; System: the value named 'system' in this block; IO: the value named 'io' in this block.

Line 130: `wdb_bytes = (Get-Item -LiteralPath $WdbPath).Length` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are wdb_bytes: the value named 'wdb bytes' in this block; Get: the value named 'get' in this block; Item: the value named 'item' in this block.

Line 131: `simulation_log = [System.IO.Path]::GetRelativePath($OutputDirectory, $SimulationLog)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are simulation_log: the value named 'simulation log' in this block; System: the value named 'system' in this block; IO: the value named 'io' in this block.

Line 132: `})` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 133: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 134: `} finally {` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are finally: the value named 'finally' in this block.

Line 135: `Remove-Item Env:NEURAEDGE_VCD_PATH -ErrorAction SilentlyContinue` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 136: `Remove-Item Env:NEURAEDGE_WAVE_AMOUNT -ErrorAction SilentlyContinue` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 137: `Remove-Item Env:NEURAEDGE_WAVE_UNIT -ErrorAction SilentlyContinue` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 138: `Pop-Location` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Pop: the value named 'pop' in this block; Location: the value named 'location' in this block.

Line 139: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 141: `$Manifest = [ordered]@{` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Manifest: the value named 'manifest' in this block; ordered: the value named 'ordered' in this block.

Line 142: `format = "neuraedge-xsim-waveforms-v1"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 143: `vivado_root = $VivadoRoot` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows capture waveforms command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 144: `target_part = "xc7z020clg400-1"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are target_part: the value named 'target part' in this block; xc7z020clg400: the value named 'xc7z020clg400' in this block.

Line 145: `target_clock_mhz = 200` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are target_clock_mhz: the value named 'target clock mhz' in this block.

Line 146: `captures = @($ManifestRows)` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are captures: the value named 'captures' in this block; ManifestRows: one output-row position processed by the tensor tile.

Line 147: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 148: `$ManifestPath = Join-Path $OutputDirectory "waveform_manifest.json"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are ManifestPath: the value named 'manifest path' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 149: `$Manifest | ConvertTo-Json -Depth 5 | Set-Content -LiteralPath $ManifestPath -Encoding utf8` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Manifest: the value named 'manifest' in this block; ConvertTo: the value named 'convert to' in this block; Json: the value named 'json' in this block.

Line 150: `Write-Output "NEURAEDGE_WAVEFORMS_CAPTURED count=$($ManifestRows.Count) manifest=$ManifestPath"` This batch-control line contributes to the flow that orchestrates the repeatable Windows capture waveforms command-line flow. The important operands are Write: the value named 'write' in this block; Output: the value named 'output' in this block; NEURAEDGE_WAVEFORMS_CAPTURED: the value named 'neuraedge waveforms captured' in this block.

79 scripts/deploy_to_pynq.ps1

Role in the complete system: orchestrates the repeatable Windows deploy to pynq command-line flow.

79.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows deploy to pynq command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

79.2 Complete source

```
param(
    [string]$BoardHost = "xilinx@192.168.2.99",
    [string]$RemoteRoot = "/home/xilinx/neuraedge_universal",
    [string]$DataDirectory = ""
)

$ErrorActionPreference = "Stop"
Set-StrictMode -Version Latest

$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path
if ([string]::IsNullOrWhiteSpace($DataDirectory)) {
    $DataDirectory = Join-Path $ProjectRoot "outputs\cifar10_shards"
}
$DataDirectory = (Resolve-Path -LiteralPath $DataDirectory).Path
$Bitfile = Join-Path $ProjectRoot "outputs\pynq_full\neuraedge_pynq_full.bit"
$Hwh = Join-Path $ProjectRoot "outputs\pynq_full\neuraedge_pynq_full.hwh"
$ParameterDirectory = Join-Path $ProjectRoot "outputs\spikingformer_hardware_parameters"
$ParameterManifest = Join-Path $ParameterDirectory "spikingformer_neuraedge_weights.json"
$ParameterBinary = Join-Path $ParameterDirectory "spikingformer_neuraedge_weights.bin"
$DataManifest = Join-Path $DataDirectory "cifar10_shards_manifest.json"
foreach ($Required in @($Bitfile, $Hwh, $ParameterManifest, $ParameterBinary, $DataManifest)) {
    if (-not (Test-Path -LiteralPath $Required -PathType Leaf)) {
        throw "Required deployment artifact is missing: $Required"
    }
}

$RemoteDirectories = @(
    "$RemoteRoot/software",
    "$RemoteRoot/outputs/pynq_full",
    "$RemoteRoot/outputs/spikingformer_hardware_parameters",
    "$RemoteRoot/outputs/cifar10_shards",
    "$RemoteRoot/outputs/pynq_training"
) -join " "
& ssh $BoardHost "mkdir -p $RemoteDirectories"
if ($LASTEXITCODE -ne 0) { throw "Could not create remote PYNQ directories" }

$SoftwareFiles = Get-ChildItem -LiteralPath (Join-Path $ProjectRoot "software") -Filter "*.py" -File |
    Sort-Object Name |
    ForEach-Object FullName
& scp @SoftwareFiles "${BoardHost}:$RemoteRoot/software/"
if ($LASTEXITCODE -ne 0) { throw "PYNQ software transfer failed" }
& scp $Bitfile $Hwh "${BoardHost}:$RemoteRoot/outputs/pynq_full/"
if ($LASTEXITCODE -ne 0) { throw "PYNQ overlay transfer failed" }
& scp $ParameterManifest $ParameterBinary `
    "${BoardHost}:$RemoteRoot/outputs/spikingformer_hardware_parameters/"
if ($LASTEXITCODE -ne 0) { throw "PYNQ parameter transfer failed" }
$DataFiles = Get-ChildItem -LiteralPath $DataDirectory -Filter "*.npz" -File |
    Sort-Object Name |
    ForEach-Object FullName
& scp $DataManifest @DataFiles "${BoardHost}:$RemoteRoot/outputs/cifar10_shards/"
if ($LASTEXITCODE -ne 0) { throw "PYNQ CIFAR-10 transfer failed" }
Write-Host "NEURAEDGE_PYNQ_DEPLOYED host=$BoardHost root=$RemoteRoot"
```

79.3 Line annotations

Line 1: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 2: `[string]$BoardHost = "xilinx@192.168.2.99",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `[string]$RemoteRoot = "/home/xilinx/neuraedge_universal",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 4: `[string]$DataDirectory = ""` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 5: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 7: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 8: `Set-StrictMode -Version Latest` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are Set: the value named 'set' in this block; StrictMode: the value named 'strict mode' in this block; Version: the value named 'version' in this block.

Line 10: `$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 11: `if ([string]::IsNullOrWhiteSpace($DataDirectory)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are string: the value named 'string' in this block; IsNullOrWhiteSpace: the value named 'is null or white space' in this block; DataDirectory: the value named 'data directory' in this block.

Line 12: `$DataDirectory = Join-Path $ProjectRoot "outputs\cifar10_shards"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are DataDirectory: the value named 'data directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 13: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 14: `$DataDirectory = (Resolve-Path -LiteralPath $DataDirectory).Path` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are DataDirectory: the value named 'data directory' in this block; Resolve: the value named 'resolve' in this block; Path: the value named 'path' in this block.

Line 15: `$Bitfile = Join-Path $ProjectRoot "outputs\pynq_full\neuraedge_pynq_full.bit"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are Bitfile: the value named 'bitfile' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 16: `$Hwh = Join-Path $ProjectRoot "outputs\pynq_full\neuraedge_pynq_full.hwh"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are Hwh: the value named 'hwh' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 17: `$ParameterDirectory = Join-Path $ProjectRoot "outputs\spikingformer_hardware_parameters"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are ParameterDirectory: the value named 'parameter directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 18: `$ParameterManifest = Join-Path $ParameterDirectory "spikingformer_neuraedge_weights.json"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are ParameterManifest: the value named 'parameter manifest' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 19: `$ParameterBinary = Join-Path $ParameterDirectory "spikingformer_neuraedge_weights.bin"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are ParameterBinary: the value named 'parameter binary' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 20: `$DataManifest = Join-Path $DataDirectory "cifar10_shards_manifest.json"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are DataManifest: the value named 'data manifest' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 21: `foreach ($Required in @($Bitfile, $Hwh, $ParameterManifest, $ParameterBinary, $DataManifest)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are foreach: the value named 'foreach' in this block; Required: the value named 'required' in this block; in: the value named 'in' in this block.

Line 22: `if (-not (Test-Path -LiteralPath $Required -PathType Leaf)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are not: the value named 'not' in this block; Test: the value named 'test' in this block; Path: the value named 'path' in this block.

Line 23: `throw "Required deployment artifact is missing: $Required"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are throw: one output-row position processed by the tensor tile; Required: the value named 'required' in this block; deployment: the value named 'deployment' in this block.

Line 24: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 25: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 27: `$RemoteDirectories = @(` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are RemoteDirectories: the value named 'remote directories' in this block.

Line 28: `"$RemoteRoot/software",` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are RemoteRoot: the value named 'remote root' in this block; software: the value named 'software' in this block.

Line 29: `"$RemoteRoot/outputs/pynq_full",` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are RemoteRoot: the value named 'remote root' in this block; outputs: the value named 'outputs' in this block; pynq_full: the value named 'pynq full' in this block.

Line 30: `"$RemoteRoot/outputs/spikingformer_hardware_parameters",` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are RemoteRoot: the value named 'remote root' in this block; outputs: the value named 'outputs' in this block; spikingformer_hardware_parameters: the value named 'spikingformer hardware parameters' in this block.

Line 31: `"$RemoteRoot/outputs/cifar10_shards",` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are RemoteRoot: the value named 'remote root' in this block; outputs: the value named 'outputs' in this block; cifar10_shards: the value named 'cifar10 shards' in this block.

Line 32: `"$RemoteRoot/outputs/pynq_training"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are RemoteRoot: the value named 'remote root' in this block; outputs: the value named 'outputs' in this block; pynq_training: the value named 'pynq training' in this block.

Line 33: `) -join " "` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are join: the value named 'join' in this block.

Line 34: `& ssh $BoardHost "mkdir -p $RemoteDirectories"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are ssh: the value named 'ssh' in this block; BoardHost: the value named 'board host' in this block; mkdir: the value named 'mkdir' in this block.

Line 35: `if ($LASTEXITCODE -ne 0) { throw "Could not create remote PYNQ directories" }` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 37: `$SoftwareFiles = Get-ChildItem -LiteralPath (Join-Path $ProjectRoot "software") -Filter "*.py" -File |` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are SoftwareFiles: the value named 'software files' in this block; Get: the value named 'get' in this block; ChildItem: the value named 'child item' in this block.

Line 38: `Sort-Object Name |` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are Sort: the value named 'sort' in this block; Object: the value named 'object' in this block; Name: the value named 'name' in this block.

Line 39: `ForEach-Object FullName` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are ForEach: the value named 'for each' in this block; Object: the value named 'object' in this block; FullName: the value named 'full name' in this block.

Line 40: `& scp @SoftwareFiles "${BoardHost}:$RemoteRoot/software/"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are scp: the value named 'scp' in this block; SoftwareFiles: the value named 'software files' in this block; BoardHost: the value named 'board host' in this block.

Line 41: `if ($LASTEXITCODE -ne 0) { throw "PYNQ software transfer failed" }` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 42: `& scp $Bitfile $Hwh "${BoardHost}:$RemoteRoot/outputs/pynq_full/"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are scp: the value named 'scp' in this block; Bitfile: the value named 'bitfile' in this block; Hwh: the value named 'hwh' in this block.

Line 43: `if ($LASTEXITCODE -ne 0) { throw "PYNQ overlay transfer failed" }` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 44: `& scp $ParameterManifest $ParameterBinary ‘` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are scp: the value named 'scp' in this block; ParameterManifest: the value named 'parameter manifest' in this block; ParameterBinary: the value named 'parameter binary' in this block.

Line 45: `"${BoardHost}:$RemoteRoot/outputs/spikingformer_hardware_parameters/"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are BoardHost: the value named 'board host' in this block; RemoteRoot: the value named 'remote root' in this block; outputs: the value named 'outputs' in this block.

Line 46: `if ($LASTEXITCODE -ne 0) { throw "PYNQ parameter transfer failed" }` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 47: `$DataFiles = Get-ChildItem -LiteralPath $DataDirectory -Filter "*.npz" -File |` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are DataFiles: the value named 'data files' in this block; Get: the value named 'get' in this block; ChildItem: the value named 'child item' in this block.

Line 48: `Sort-Object Name |` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are Sort: the value named 'sort' in this block; Object: the value named 'object' in this block; Name: the value named 'name' in this block.

Line 49: `ForEach-Object FullName` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are ForEach: the value named 'for each' in this block; Object: the value named 'object' in this block; FullName: the value named 'full name' in this block.

Line 50: `& scp $DataManifest @DataFiles "${BoardHost}:$RemoteRoot/outputs/cifar10_shards/"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are scp: the value named 'scp' in this block; DataManifest: the value named 'data manifest' in this block; DataFiles: the value named 'data files' in this block.

Line 51: `if ($LASTEXITCODE -ne 0) { throw "PYNQ CIFAR-10 transfer failed" }` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 52: `Write-Host "NEURAEDGE_PYNQ_DEPLOYED host=$BoardHost root=$RemoteRoot"` This batch-control line contributes to the flow that orchestrates the repeatable Windows deploy to pynq command-line flow. The important operands are Write: the value named 'write' in this block; Host: the value named 'host' in this block; NEURAEDGE_PYNQ_DEPLOYED: the value named 'neuraedge pynq deployed' in this block.

80 scripts/export_cifar10_shards.ps1

Role in the complete system: orchestrates the repeatable Windows export cifar10 shards command-line flow.

80.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows export cifar10 shards command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

80.2 Complete source

```
param(
    [string]$DatasetRoot = "",
    [string]$OutputDirectory = "",
    [int]$ShardSize = 1000
)

$ErrorActionPreference = "Stop"
Set-StrictMode -Version Latest

$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path
if ([string]::IsNullOrWhiteSpace($DatasetRoot)) {
    $DatasetRoot = (Resolve-Path (Join-Path $ProjectRoot "..\datasets")).Path
}
if ([string]::IsNullOrWhiteSpace($OutputDirectory)) {
    $OutputDirectory = Join-Path $ProjectRoot "outputs\cifar10_shards"
}
$Python = (Resolve-Path (Join-Path $ProjectRoot "..\external_research\spikingformer_py310\Scripts\python.exe")).Path
& $Python (Join-Path $ProjectRoot "software\export_cifar10_shards.py") `
    --dataset-root $DatasetRoot `
    --output-dir $OutputDirectory `
    --shard-size $ShardSize `
    --no-download
if ($LASTEXITCODE -ne 0) {
    throw "CIFAR-10 shard export failed with exit code $LASTEXITCODE"
}
Write-Host "NEURAEDGE_CIFAR10_SHARDS_READY directory=$OutputDirectory"
```

80.3 Line annotations

Line 1: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 2: `[string]$DatasetRoot = "",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `[string]$OutputDirectory = "",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 4: `[int]$ShardSize = 1000` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 5: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 7: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 8: `Set-StrictMode -Version Latest` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are Set: the value named 'set' in this block; StrictMode: the value named 'strict mode' in this block; Version: the value named 'version' in this block.

Line 10: `$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 11: `if ([string]::IsNullOrWhiteSpace($DatasetRoot)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are string: the value named 'string' in this block; IsNullOrWhiteSpace: the value named 'is null or white space' in this block; DatasetRoot: the value named 'dataset root' in this block.

Line 12: `$DatasetRoot = (Resolve-Path (Join-Path $ProjectRoot "..\datasets")).Path` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are DatasetRoot: the value named 'dataset root' in this block; Resolve: the value named 'resolve' in this block; Path: the value named 'path' in this block.

Line 13: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 14: `if ([string]::IsNullOrWhiteSpace($OutputDirectory)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are string: the value named 'string' in this block; IsNullOrWhiteSpace: the value named 'is null or white space' in this block; OutputDirectory: the value named 'output directory' in this block.

Line 15: `$OutputDirectory = Join-Path $ProjectRoot "outputs\cifar10_shards"` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are OutputDirectory: the value named 'output directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 16: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 17: `$Python = (Resolve-Path (Join-Path $ProjectRoot "..\external_research\spikingformer_py310\Scripts\py`
This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows export cifar10 shards command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 18: `& $Python (Join-Path $ProjectRoot "software\export_cifar10_shards.py") ‘` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows export cifar10 shards command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 19: `-dataset-root $DatasetRoot ‘` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are dataset: the value named 'dataset' in this block; root: the value named 'root' in this block; DatasetRoot: the value named 'dataset root' in this block.

Line 20: `-output-dir $OutputDirectory ‘` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are output: the value named 'output' in this block; dir: the value named 'dir' in this block; OutputDirectory: the value named 'output directory' in this block.

Line 21: `-shard-size $ShardSize ‘` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are shard: the value named 'shard' in this block; size: the value named 'size' in this block; ShardSize: the value named 'shard size' in this block.

Line 22: `-no-download` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are no: the value named 'no' in this block; download: the value named 'download' in this block.

Line 23: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 24: `throw "CIFAR-10 shard export failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 25: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 26: `Write-Host "NEURAEDGE_CIFAR10_SHARDS_READY directory=$OutputDirectory"` This batch-control line contributes to the flow that orchestrates the repeatable Windows export cifar10 shards command-line flow. The important operands are Write: the value named 'write' in this block; Host: the value named 'host' in this block; NEURAEDGE_CIFAR10_SHARDS_READY: the consumer's statement that it can accept a payload this cycle.

81 scripts/manage_pynq_training.ps1

Role in the complete system: orchestrates the repeatable Windows manage pynq training command-line flow.

81.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows manage pynq training command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

81.2 Complete source

```
param(
    [ValidateSet("start", "status", "pause", "resume")]
    [string]$Action = "status",
    [string]$BoardHost = "xilinx@192.168.2.99",
    [string]$RemoteRoot = "/home/xilinx/neuraedge_universal",
    [string]$Python = "python3",
    [int]$Epochs = 400,
    [int]$BatchSize = 1,
    [double]$LearningRate = 0.25,
    [double]$MinimumLearningRate = 0.01,
    [int]$CheckpointInterval = 1
)

$ErrorActionPreference = "Stop"
Set-StrictMode -Version Latest

$Output = "$RemoteRoot/outputs/pynq_training"
$PidFile = "$Output/training.pid"
$PauseFile = "$Output/PAUSE"
$StatusFile = "$Output/training_status.json"
$OutLog = "$Output/train.out.log"
$ErrLog = "$Output/train.err.log"

function Invoke-Remote([string]$Command) {
    & ssh $BoardHost "sh -lc '$Command'"
    if ($LASTEXITCODE -ne 0) {
        throw "Remote PYNQ command failed with exit code $LASTEXITCODE"
    }
}

function Start-Training([switch]$Resume) {
    $ResumeArgument = if ($Resume) { "--resume" } else { "" }
    $Command = "mkdir -p $Output; " +
        "if [ -f $PidFile ] && kill -0 `$(cat $PidFile) 2>/dev/null; then " +
        "echo already_running pid=`$(cat $PidFile); exit 2; fi; " +
        "rm -f $PauseFile; " +
        "cd $RemoteRoot; " +
        "nohup $Python software/train_spikingformer_pynq.py " +
        "--bitfile outputs/pynq_full/neuraedge_pynq_full.bit " +
        "--manifest outputs/spikingformer_hardware_parameters/spikingformer_neuraedge_weights.json " +
        "--data-manifest outputs/cifar10_shards/cifar10_shards_manifest.json " +
        "--output-dir outputs/pynq_training --epochs $Epochs --batch-size $BatchSize " +
        "--learning-rate $LearningRate --minimum-learning-rate $MinimumLearningRate " +
        "--checkpoint-interval $CheckpointInterval $ResumeArgument " +
        ">$OutLog 2>$ErrLog </dev/null & echo `$! >$PidFile; " +
        "echo started pid=`$(cat $PidFile)"
    Invoke-Remote $Command
}

switch ($Action) {
    "start" { Start-Training }
    "resume" { Start-Training -Resume }
    "pause" {
        Invoke-Remote "mkdir -p $Output; touch $PauseFile; echo pause_requested=$PauseFile"
    }
    "status" {
        $Command = "if [ -f $PidFile ]; then pid=`$(cat $PidFile); else pid=none; fi; " +
            "if [ `$pid != none ] && kill -0 `$pid 2>/dev/null; then alive=true; else alive=false; fi; " +
            "echo pid=`$pid alive=`$alive; " +
            "if [ -f $StatusFile ]; then cat $StatusFile; else echo no_status_file; fi; " +
```

```
            "echo out_log=$OutLog; echo err_log=$ErrLog"
        Invoke-Remote $Command
    }
}
```

81.3 Line annotations

Line 1: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 2: `[ValidateSet("start", "status", "pause", "resume")]` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are ValidateSet: the producer's statement that its current payload is meaningful; start: the value named 'start' in this block; status: the value named 'status' in this block.

Line 3: `[string]$Action = "status",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 4: `[string]$BoardHost = "xilinx@192.168.2.99",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 5: `[string]$RemoteRoot = "/home/xilinx/neuraedge_universal",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 6: `[string]$Python = "python3",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 7: `[int]$Epochs = 400,` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 8: `[int]$BatchSize = 1,` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 9: `[double]$LearningRate = 0.25,` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 10: `[double]$MinimumLearningRate = 0.01,` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 11: `[int]$CheckpointInterval = 1` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 12: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 14: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 15: `Set-StrictMode -Version Latest` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are Set: the value named 'set' in this block; StrictMode: the value named 'strict mode' in this block; Version: the value named 'version' in this block.

Line 17: `$Output = "$RemoteRoot/outputs/pynq_training"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are Output: the value named 'output' in this block; RemoteRoot: the value named 'remote root' in this block; outputs: the value named 'outputs' in this block.

Line 18: `$PidFile = "$Output/training.pid"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are PidFile: the value named 'pid file' in this block; Output: the value named 'output' in this block; training: the value named 'training' in this block.

Line 19: `$PauseFile = "$Output/PAUSE"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are PauseFile: the value named 'pause file' in this block; Output: the value named 'output' in this block; PAUSE: the value named 'pause' in this block.

Line 20: `$StatusFile = "$Output/training_status.json"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are StatusFile: the value named 'status file' in this block; Output: the value named 'output' in this block; training_status: the value named 'training status' in this block.

Line 21: `$OutLog = "$Output/train.out.log"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are OutLog: the value named 'out log' in this block; Output: the value named 'output' in this block; train: the value named 'train' in this block.

Line 22: `$ErrLog = "$Output/train.err.log"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are ErrLog: the value named 'err log' in this block; Output: the value named 'output' in this block; train: the value named 'train' in this block.

Line 24: `function Invoke-Remote([string]$Command) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are function: the value named 'function' in this block; Invoke: the value named 'invoke' in this block; Remote: the value named 'remote' in this block.

Line 25: `& ssh $BoardHost "sh -lc '$Command'"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are ssh: the value named 'ssh' in this block; BoardHost: the value named 'board host' in this block; sh: the value named 'sh' in this block.

Line 26: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 27: `throw "Remote PYNQ command failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 28: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 29: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 31: `function Start-Training([switch]$Resume) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are function: the value named 'function' in this block; Start: the value named 'start' in this block; Training: the value named 'training' in this block.

Line 32: `$ResumeArgument = if ($Resume) { "-resume" } else { "" }` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are ResumeArgument: the value named 'resume argument' in this block; Resume: the value named 'resume' in this block; resume: the value named 'resume' in this block.

Line 33: `$Command = "mkdir -p $Output; " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are Command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; mkdir: the value named 'mkdir' in this block; p: the value named 'p' in this block.

Line 34: `"if [ -f $PidFile ] && kill -0 ‘$(cat $PidFile) 2>/dev/null; then " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are f: the value named 'f' in this block; PidFile: the value named 'pid file' in this block; kill: the value named 'kill' in this block.

Line 35: `"echo already_running pid=‘$(cat $PidFile); exit 2; fi; " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are echo: the value named 'echo' in this block; already_running: the consumer's statement that it can accept a payload this cycle; pid: the value named 'pid' in this block.

Line 36: `"rm -f $PauseFile; " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are rm: the value named 'rm' in this block; f: the value named 'f' in this block; PauseFile: the value named 'pause file' in this block.

Line 37: `"cd $RemoteRoot; " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are cd: the value named 'cd' in this block; RemoteRoot: the value named 'remote root' in this block.

Line 38: `"nohup $Python software/train_spikingformer_pynq.py " +` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows manage pynq training command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 39: `"-bitfile outputs/pynq_full/neuraedge_pynq_full.bit " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are bitfile: the value named 'bitfile' in this block; outputs: the value named 'outputs' in this block; pynq_full: the value named 'pynq full' in this block.

Line 40: `"-manifest outputs/spikingformer_hardware_parameters/spikingformer_neuraedge_weights.json " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are manifest: the value named 'manifest' in this block; outputs: the value named 'outputs' in this block; spikingformer_hardware_parameters: the value named 'spikingformer hardware parameters' in this block.

Line 41: `"-data-manifest outputs/cifar10_shards/cifar10_shards_manifest.json " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are data: the value named 'data' in this block; manifest: the value named 'manifest' in this block; outputs: the value named 'outputs' in this block.

Line 42: `"-output-dir outputs/pynq_training -epochs $Epochs -batch-size $BatchSize " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are output: the value named 'output' in this block; dir: the value named 'dir' in this block; outputs: the value named 'outputs' in this block.

Line 43: `"-learning-rate $LearningRate -minimum-learning-rate $MinimumLearningRate " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are learning: the value named 'learning' in this block; rate: the value named 'rate' in this block; LearningRate: the value named 'learning rate' in this block.

Line 44: `"-checkpoint-interval $CheckpointInterval $ResumeArgument " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are checkpoint: the value named 'checkpoint' in this block; interval: the value named 'interval' in this block; CheckpointInterval: the value named 'checkpoint interval' in this block.

Line 45: `">$OutLog 2>$ErrLog </dev/null & echo '$! >$PidFile; " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are OutLog: the value named 'out log' in this block; ErrLog: the value named 'err log' in this block; dev: the value named 'dev' in this block.

Line 46: `"echo started pid='$(cat $PidFile)"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are echo: the value named 'echo' in this block; started: the value named 'started' in this block; pid: the value named 'pid' in this block.

Line 47: `Invoke-Remote $Command` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are Invoke: the value named 'invoke' in this block; Remote: the value named 'remote' in this block; Command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 48: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 50: `switch ($Action) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are switch: the value named 'switch' in this block; Action: the value named 'action' in this block.

Line 51: `"start" { Start-Training }` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are start: the value named 'start' in this block; Start: the value named 'start' in this block; Training: the value named 'training' in this block.

Line 52: `"resume" { Start-Training -Resume }` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are resume: the value named 'resume' in this block; Start: the value named 'start' in this block; Training: the value named 'training' in this block.

Line 53: `"pause" {` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are pause: the value named 'pause' in this block.

Line 54: `Invoke-Remote "mkdir -p $Output; touch $PauseFile; echo pause_requested=$PauseFile"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are Invoke: the value named 'invoke' in this block; Remote: the value named 'remote' in this block; mkdir: the value named 'mkdir' in this block.

Line 55: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 56: `"status" {` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are status: the value named 'status' in this block.

Line 57: `$Command = "if [ -f $PidFile ]; then pid=‘$(cat $PidFile); else pid=none; fi; " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are Command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; f: the value named ’f’ in this block; PidFile: the value named ’pid file’ in this block.

Line 58: `"if [ ‘$pid != none ] && kill -0 ‘$pid 2>/dev/null; then alive=true; else alive=false; fi; " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are pid: the value named ’pid’ in this block; none: the value named ’none’ in this block; kill: the value named ’kill’ in this block.

Line 59: `"echo pid=‘$pid alive=‘$alive; " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are echo: the value named ’echo’ in this block; pid: the value named ’pid’ in this block; alive: the value named ’alive’ in this block.

Line 60: `"if [ -f $StatusFile ]; then cat $StatusFile; else echo no_status_file; fi; " +` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are f: the value named ’f’ in this block; StatusFile: the value named ’status file’ in this block; then: the value named ’then’ in this block.

Line 61: `"echo out_log=$OutLog; echo err_log=$ErrLog"` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are echo: the value named ’echo’ in this block; out_log: the value named ’out log’ in this block; OutLog: the value named ’out log’ in this block.

Line 62: `Invoke-Remote $Command` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The important operands are Invoke: the value named ’invoke’ in this block; Remote: the value named ’remote’ in this block; Command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 63: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 64: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows manage pynq training command-line flow. The expression is a literal or structural operator and has no hidden software state.

82 scripts/run_mac2_probe.ps1

Role in the complete system: orchestrates the repeatable Windows run mac2 probe command-line flow.

82.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows run mac2 probe command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

82.2 Complete source

```
param(
    [string]$VivadoBin = "D:\AMDDesignTools\2026.1\Vivado\bin"
)

$ErrorActionPreference = "Stop"
$ProjectDir = Resolve-Path (Join-Path $PSScriptRoot "..")
$Xvlog = Join-Path $VivadoBin "xvlog.bat"
$Xelab = Join-Path $VivadoBin "xelab.bat"
$Xsim = Join-Path $VivadoBin "xsim.bat"
$Vivado = Join-Path $VivadoBin "vivado.bat"

foreach ($tool in @($Xvlog, $Xelab, $Xsim, $Vivado)) {
    if (!(Test-Path $tool)) { throw "Required Vivado tool not found: $tool" }
}

Push-Location $ProjectDir
try {
    & $Xvlog -sv rtl\neuraedge_mac2_u8.sv tb\tb_neuraedge_mac2_u8.sv
    if ($LASTEXITCODE -ne 0) { throw "xvlog failed with exit code $LASTEXITCODE" }
    & $Xelab tb_neuraedge_mac2_u8 -debug typical
    if ($LASTEXITCODE -ne 0) { throw "xelab failed with exit code $LASTEXITCODE" }
    & $Xsim tb_neuraedge_mac2_u8 -runall
    if ($LASTEXITCODE -ne 0) { throw "xsim failed with exit code $LASTEXITCODE" }
    & $Vivado -mode batch -source scripts\synth_mac2_probe.tcl -notrace
    if ($LASTEXITCODE -ne 0) { throw "Vivado probe implementation failed with exit code $LASTEXITCODE" }
} finally {
    Pop-Location
}

Write-Host "Packed MAC arithmetic and 128-DSP mapping probe passed."
```

82.3 Line annotations

Line 1: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 2: `[string]$VivadoBin = "D:\AMDDesignTools\2026.1\Vivado\bin"` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows run mac2 probe command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 5: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 6: `$ProjectDir = Resolve-Path (Join-Path $PSScriptRoot "..")` This batch-control line contributes to the flow that orchestrates the repeatable Windows run mac2 probe command-line flow. The important operands are ProjectDir: the value named 'project dir' in this block; Resolve: the value named 'resolve' in this block; Path: the value named 'path' in this block.

Line 7: `$Xvlog = Join-Path $VivadoBin "xvlog.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 8: `$Xelab = Join-Path $VivadoBin "xelab.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 9: `$Xsim = Join-Path $VivadoBin "xsim.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 10: `$Vivado = Join-Path $VivadoBin "vivado.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 12: `foreach ($tool in @($Xvlog, $Xelab, $Xsim, $Vivado)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 13: `if (!(Test-Path $tool)) { throw "Required Vivado tool not found: $tool" }` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 14: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run mac2 probe command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 16: `Push-Location $ProjectDir` This batch-control line contributes to the flow that orchestrates the repeatable Windows run mac2 probe command-line flow. The important operands are Push: the value named 'push' in this block; Location: the value named 'location' in this block; ProjectDir: the value named 'project dir' in this block.

Line 17: `try {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run mac2 probe command-line flow. The important operands are try: the value named 'try' in this block.

Line 18: `& $Xvlog -sv rtl\neuraedge_mac2_u8.sv tb\tb_neuraedge_mac2_u8.sv` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 19: `if ($LASTEXITCODE -ne 0) { throw "xvlog failed with exit code $LASTEXITCODE" }` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 20: `& $Xelab tb_neuraedge_mac2_u8 -debug typical` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 21: `if ($LASTEXITCODE -ne 0) { throw "xelab failed with exit code $LASTEXITCODE" }` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 22: `& $Xsim tb_neuraedge_mac2_u8 -runall` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 23: `if ($LASTEXITCODE -ne 0) { throw "xsim failed with exit code $LASTEXITCODE" }` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 24: `& $Vivado -mode batch -source scripts\synth_mac2_probe.tcl -notrace` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run mac2 probe command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 25: `if ($LASTEXITCODE -ne 0) { throw "Vivado probe implementation failed with exit code $LASTEXITCODE" }` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 26: `} finally {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run mac2 probe command-line flow. The important operands are finally: the value named 'finally' in this block.

Line 27: `Pop-Location` This batch-control line contributes to the flow that orchestrates the repeatable Windows run mac2 probe command-line flow. The important operands are Pop: the value named 'pop' in this block; Location: the value named 'location' in this block.

Line 28: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run mac2 probe command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 30: `Write-Host "Packed MAC arithmetic and 128-DSP mapping probe passed."` This batch-control line contributes to the flow that orchestrates the repeatable Windows run mac2 probe command-line flow. The important operands are Write: the value named 'write' in this block; Host: the value named 'host' in this block; Packed: the value named 'packed' in this block.

83 scripts/run_pynq_full.ps1

Role in the complete system: orchestrates the repeatable Windows run pynq full command-line flow.

83.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows run pynq full command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

83.2 Complete source

```
[CmdletBinding()]
param(
    [string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",
    [ValidateSet("validate", "full")]
    [string]$Mode = "full",
    [switch]$CleanLogs
)

$ErrorActionPreference = "Stop"
$ProjectRoot = Split-Path -Parent $PSScriptRoot
$Vivado = Join-Path $VivadoRoot "bin\vivado.bat"
$Tcl = Join-Path $PSScriptRoot "build_pynq_full.tcl"
$DiagramTcl = Join-Path $PSScriptRoot "export_pynq_diagrams.tcl"
$OutputRoot = Join-Path $ProjectRoot "outputs\pynq_full"
$Summary = Join-Path $ProjectRoot "outputs\pynq_full\full_implementation_summary.txt"
$BuildRoot = if ([string]::IsNullOrWhiteSpace($env:NEURAEDGE_VIVADO_BUILD_ROOT)) {
    "D:\NeuraEdgeBuild"
} else {
    [System.IO.Path]::GetFullPath($env:NEURAEDGE_VIVADO_BUILD_ROOT)
}
$VivadoProject = Join-Path $BuildRoot "pynq_full\neuraedge_pynq_full.xpr"

if (-not (Test-Path -LiteralPath $Vivado)) {
    throw "Vivado was not found at $Vivado"
}

if ($CleanLogs) {
    foreach ($Name in @("vivado.jou", "vivado.log")) {
        $Path = Join-Path $ProjectRoot $Name
        if (Test-Path -LiteralPath $Path) {
            Remove-Item -LiteralPath $Path -Force
        }
    }
}

Push-Location $ProjectRoot
try {
    New-Item -ItemType Directory -Force -Path $OutputRoot | Out-Null
    $Log = Join-Path $OutputRoot "vivado_${Mode}.log"
    $Journal = Join-Path $OutputRoot "vivado_${Mode}.jou"
    & $Vivado -mode batch -log $Log -journal $Journal -source $Tcl -tclargs $Mode
    if ($LASTEXITCODE -ne 0) {
        throw "PYNQ $Mode build failed with exit code $LASTEXITCODE. See $Log"
    }

    $DiagramLog = Join-Path $OutputRoot "vivado_diagrams.log"
    $DiagramJournal = Join-Path $OutputRoot "vivado_diagrams.jou"
    & $Vivado -mode gui -log $DiagramLog -journal $DiagramJournal `
        -source $DiagramTcl -tclargs $VivadoProject $OutputRoot
    if ($LASTEXITCODE -ne 0) {
        throw "PYNQ diagram export failed with exit code $LASTEXITCODE. See $DiagramLog"
    }
    foreach ($Diagram in @("neuraedge_pynq_block_design.pdf",
            "neuraedge_pynq_block_design.svg")) {
        $DiagramPath = Join-Path $OutputRoot $Diagram
        if (-not (Test-Path -LiteralPath $DiagramPath) -or
                (Get-Item -LiteralPath $DiagramPath).Length -eq 0) {
            throw "PYNQ diagram export did not create $DiagramPath"
        }
    }
```

```
} finally {
    Pop-Location
}

if ($Mode -eq "full") {
    Get-Content -LiteralPath $Summary
} else {
    Write-Host "NEURAEDGE_PYNQ_VALIDATION_PASSED report=$OutputRoot\validated_ip_status.rpt"
}
```

83.3 Line annotations

Line 1: `[CmdletBinding()]` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are CmdletBinding: the value named 'cmdlet binding' in this block.

Line 2: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `[string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 4: `[ValidateSet("validate", "full")]` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are ValidateSet: the producer's statement that its current payload is meaningful; validate: the producer's statement that its current payload is meaningful; full: the value named 'full' in this block.

Line 5: `[string]$Mode = "full",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 6: `[switch]$CleanLogs` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 7: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 9: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 10: `$ProjectRoot = Split-Path -Parent $PSScriptRoot` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows run pynq full command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 11: `$Vivado = Join-Path $VivadoRoot "bin\vivado.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 12: `$Tcl = Join-Path $PSScriptRoot "build_pynq_full.tcl"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Tcl: the value named 'tcl' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 13: `$DiagramTcl = Join-Path $PSScriptRoot "export_pynq_diagrams.tcl"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are DiagramTcl: the value named 'diagram tcl' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 14: `$OutputRoot = Join-Path $ProjectRoot "outputs\pynq_full"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are OutputRoot: the value named 'output root' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 15: `$Summary = Join-Path $ProjectRoot "outputs\pynq_full\full_implementation_summary.txt"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Summary: the value named 'summary' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 16: `$BuildRoot = if ([string]::IsNullOrWhiteSpace($env:NEURAEDGE_VIVADO_BUILD_ROOT)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 17: `"D:\NeuraEdgeBuild"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are D: the value named 'd' in this block; NeuraEdgeBuild: the value named 'neura edge build' in this block.

Line 18: `} else {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 19: `[System.IO.Path]::GetFullPath($env:NEURAEDGE_VIVADO_BUILD_ROOT)` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 20: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 21: `$VivadoProject = Join-Path $BuildRoot "pynq_full\neuraedge_pynq_full.xpr"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 23: `if (-not (Test-Path -LiteralPath $Vivado)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 24: `throw "Vivado was not found at $Vivado"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 25: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 27: `if ($CleanLogs) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are CleanLogs: the value named 'clean logs' in this block.

Line 28: `foreach ($Name in @("vivado.jou", "vivado.log")) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 29: `$Path = Join-Path $ProjectRoot $Name` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Path: the value named 'path' in this block; Join: the value named 'join' in this block; ProjectRoot: the value named 'project root' in this block.

Line 30: `if (Test-Path -LiteralPath $Path) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Test: the value named 'test' in this block; Path: the value named 'path' in this block; LiteralPath: the value named 'literal path' in this block.

Line 31: `Remove-Item -LiteralPath $Path -Force` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 32: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 33: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 34: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 36: `Push-Location $ProjectRoot` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Push: the value named 'push' in this block; Location: the value named 'location' in this block; ProjectRoot: the value named 'project root' in this block.

Line 37: `try {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are try: the value named 'try' in this block.

Line 38: `New-Item -ItemType Directory -Force -Path $OutputRoot | Out-Null` This creates the declared output directory before a tool writes into it. '-Force' permits an existing directory but does not make an external tool failure succeed.

Line 39: `$Log = Join-Path $OutputRoot "vivado_${Mode}.log"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 40: `$Journal = Join-Path $OutputRoot "vivado_${Mode}.jou"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 41: `& $Vivado -mode batch -log $Log -journal $Journal -source $Tcl -tclargs $Mode` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 42: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 43: `throw "PYNQ $Mode build failed with exit code $LASTEXITCODE. See $Log"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 44: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 46: `$DiagramLog = Join-Path $OutputRoot "vivado_diagrams.log"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 47: `$DiagramJournal = Join-Path $OutputRoot "vivado_diagrams.jou"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 48: `& $Vivado -mode gui -log $DiagramLog -journal $DiagramJournal ‘` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 49: `-source $DiagramTcl -tclargs $VivadoProject $OutputRoot` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run pynq full command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 50: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 51: `throw "PYNQ diagram export failed with exit code $LASTEXITCODE. See $DiagramLog"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 52: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 53: `foreach ($Diagram in @("neuraedge_pynq_block_design.pdf",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are foreach: the value named 'foreach' in this block; Diagram: the value named 'diagram' in this block; in: the value named 'in' in this block.

Line 54: `"neuraedge_pynq_block_design.svg")) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are neuraedge_pynq_block_design: the value named 'neuraedge pynq block design' in this block; svg: the value named 'svg' in this block.

Line 55: `$DiagramPath = Join-Path $OutputRoot $Diagram` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are DiagramPath: the value named 'diagram path' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 56: `if (-not (Test-Path -LiteralPath $DiagramPath) -or` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are not: the value named 'not' in this block; Test: the value named 'test' in this block; Path: the value named 'path' in this block.

Line 57: `(Get-Item -LiteralPath $DiagramPath).Length -eq 0) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Get: the value named 'get' in this block; Item: the value named 'item' in this block; LiteralPath: the value named 'literal path' in this block.

Line 58: `throw "PYNQ diagram export did not create $DiagramPath"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are throw: one output-row position processed by the tensor tile; PYNQ: the value named 'pynq' in this block; diagram: the value named 'diagram' in this block.

Line 59: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 60: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 61: `} finally {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are finally: the value named 'finally' in this block.

Line 62: `Pop-Location` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Pop: the value named 'pop' in this block; Location: the value named 'location' in this block.

Line 63: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 65: `if ($Mode -eq "full") {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Mode: the value named 'mode' in this block; eq: the value named 'eq' in this block; full: the value named 'full' in this block.

Line 66: `Get-Content -LiteralPath $Summary` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Get: the value named 'get' in this block; Content: the value named 'content' in this block; LiteralPath: the value named 'literal path' in this block.

Line 67: `} else {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 68: `Write-Host "NEURAEDGE_PYNQ_VALIDATION_PASSED report=$OutputRoot\validated_ip_status.rpt"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The important operands are Write: the value named 'write' in this block; Host: the value named 'host' in this block; NEURAEDGE_PYNQ_VALIDATION_PASSED: the producer's statement that its current payload is meaningful.

Line 69: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run pynq full command-line flow. The expression is a literal or structural operator and has no hidden software state.

84 scripts/run_regression.ps1

Role in the complete system: orchestrates the repeatable Windows run regression command-line flow.

84.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows run regression command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

84.2 Complete source

```
param(
    [string]$IcarusRoot = "C:\msys64\ucrt64\bin",
    [string]$MsysRoot = "C:\msys64",
    [string]$PythonExe = "",
    [switch]$KeepExecutables
)

$ErrorActionPreference = "Stop"
Set-StrictMode -Version Latest

$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path
$OutputRoot = Join-Path $ProjectRoot "outputs\regression"
New-Item -ItemType Directory -Force -Path $OutputRoot | Out-Null
$SummaryPath = Join-Path $OutputRoot "regression_summary.txt"
@(
    "result=RUNNING",
    "started_utc=$([DateTime]::UtcNow.ToString('o'))"
) | Set-Content -LiteralPath $SummaryPath -Encoding ascii

$Iverilog = Join-Path $IcarusRoot "iverilog.exe"
$Vvp = Join-Path $IcarusRoot "vvp.exe"
if (-not (Test-Path -LiteralPath $Iverilog) -or -not (Test-Path -LiteralPath $Vvp)) {
    throw "Icarus Verilog was not found under $IcarusRoot"
}

function Test-RegressionPython {
    param([string]$Candidate)
    if ([string]::IsNullOrWhiteSpace($Candidate)) {
        return $false
    }
    $Command = Get-Command $Candidate -ErrorAction SilentlyContinue
    if ($null -eq $Command) {
        return $false
    }
    & $Command.Source -c "import numpy, torch" *> $null
    return $LASTEXITCODE -eq 0
}

if ([string]::IsNullOrWhiteSpace($PythonExe)) {
    $Candidates = [System.Collections.Generic.List[string]]::new()
    $SearchRoot = Get-Item -LiteralPath $ProjectRoot
    for ($Depth = 0; $Depth -lt 5 -and $null -ne $SearchRoot; $Depth++) {
        $PreferredPython = Join-Path $SearchRoot.FullName `
            "external_research\spikingformer_py310\Scripts\python.exe"
        if (Test-Path -LiteralPath $PreferredPython) {
            $Candidates.Add((Resolve-Path $PreferredPython).Path)
        }
        $SearchRoot = $SearchRoot.Parent
    }
    $Launcher = Get-Command py.exe -ErrorAction SilentlyContinue
    if ($null -ne $Launcher) {
        foreach ($Version in @("3.14", "3.10")) {
            $Discovered = & $Launcher.Source "-$Version" -c "import sys; print(sys.executable)" 2>$null
            if ($LASTEXITCODE -eq 0 -and -not [string]::IsNullOrWhiteSpace($Discovered)) {
                $Candidates.Add([string]($Discovered | Select-Object -Last 1))
            }
        }
    }
    $PathPython = Get-Command python.exe -ErrorAction SilentlyContinue
    if ($null -ne $PathPython) {
```

```
        $Candidates.Add($PathPython.Source)
    }
    foreach ($Candidate in $Candidates | Select-Object -Unique) {
        if (Test-RegressionPython $Candidate) {
            $PythonExe = (Get-Command $Candidate -ErrorAction Stop).Source
            break
        }
    }
    if ([string]::IsNullOrWhiteSpace($PythonExe)) {
        throw "No Python interpreter with NumPy and PyTorch was found. Pass -PythonExe explicitly."
    }
} elseif (-not (Test-RegressionPython $PythonExe)) {
    throw "The requested Python interpreter is unavailable or cannot import NumPy and PyTorch: $PythonExe"
}

$PackageFile = Join-Path $ProjectRoot "rtl\neuraedge_command_pkg.sv"
$RtlFiles = @($PackageFile) + @(
    Get-ChildItem -LiteralPath (Join-Path $ProjectRoot "rtl") -Filter "*.sv" -File |
        Where-Object { $_.FullName -ne $PackageFile } |
        Sort-Object Name |
        ForEach-Object FullName
)
$Testbenches = @(
    "tb_neuraedge_mac2_u8",
    "tb_neuraedge_tensor_tile",
    "tb_neuraedge_buffered_tensor_core",
    "tb_neuraedge_alpha_xnor",
    "tb_neuraedge_vector_alu",
    "tb_neuraedge_integer_primitives",
    "tb_neuraedge_normalization_units",
    "tb_neuraedge_lif_bank",
    "tb_neuraedge_sgd_optimizer8",
    "tb_neuraedge_linear_backward8",
    "tb_neuraedge_activation_lut8",
    "tb_neuraedge_universal_core",
    "tb_neuraedge_descriptor_sequencer",
    "tb_neuraedge_pynq_descriptor",
    "tb_neuraedge_pynq_axi_top"
)

Push-Location $ProjectRoot
try {
    $env:PYTHONPATH = (Join-Path $ProjectRoot "software")
    & $PythonExe (Join-Path $ProjectRoot "software\generate_tensor_tile_fixture.py") `
        --output-dir (Join-Path $ProjectRoot "tb\fixtures") 2>&1 |
        Tee-Object -FilePath (Join-Path $OutputRoot "tensor_fixture_generation.log")
    if ($LASTEXITCODE -ne 0) {
        throw "Tensor fixture generation failed with exit code $LASTEXITCODE"
    }
    & $PythonExe (Join-Path $ProjectRoot "software\generate_operator_fixtures.py") `
        --output-dir (Join-Path $ProjectRoot "tb\fixtures") 2>&1 |
        Tee-Object -FilePath (Join-Path $OutputRoot "operator_fixture_generation.log")
    if ($LASTEXITCODE -ne 0) {
        throw "Operator fixture generation failed with exit code $LASTEXITCODE"
    }

    & (Join-Path $PSScriptRoot "run_verilator_lint.ps1") -MsysRoot $MsysRoot 2>&1 |
        Tee-Object -FilePath (Join-Path $OutputRoot "verilator_driver.log")
    if ($LASTEXITCODE -ne 0) {
        throw "Verilator lint failed with exit code $LASTEXITCODE"
    }

    & $PythonExe -m unittest discover -s tests -v 2>&1 |
        Tee-Object -FilePath (Join-Path $OutputRoot "python_unittest.log")
    if ($LASTEXITCODE -ne 0) {
        throw "Python regression failed with exit code $LASTEXITCODE"
    }

    foreach ($Top in $Testbenches) {
        $TestbenchFile = Join-Path $ProjectRoot "tb\$Top.sv"
        $Executable = Join-Path $OutputRoot "$Top.vvp"
        $CompileLog = Join-Path $OutputRoot "$Top.compile.log"
        $RunLog = Join-Path $OutputRoot "$Top.run.log"

        & $Iverilog -g2012 -I $ProjectRoot -s $Top -o $Executable @RtlFiles $TestbenchFile 2>&1 |
            Tee-Object -FilePath $CompileLog
        if ($LASTEXITCODE -ne 0) {
            throw "$Top compilation failed with exit code $LASTEXITCODE"
        }

        & $Vvp $Executable 2>&1 | Tee-Object -FilePath $RunLog
        if ($LASTEXITCODE -ne 0) {
            throw "$Top simulation failed with exit code $LASTEXITCODE"
        }
        $RunText = Get-Content -LiteralPath $RunLog -Raw
        $PassPattern = "(?m)^$([regex]::Escape($Top)) PASSED\b"
        if ($RunText -notmatch $PassPattern) {
```

```
            throw "$Top simulation did not emit its required PASSED marker"
        }
        if ($RunText -match "(?im)^\s*ERROR\b|^.*\bFAILED\b") {
            throw "$Top simulation emitted an ERROR or FAILED marker"
        }
        if (-not $KeepExecutables) {
            Remove-Item -LiteralPath $Executable -Force
        }
    }
} catch {
    @(
        "result=FAIL",
        "message=$($_.Exception.Message)",
        "completed_utc=$([DateTime]::UtcNow.ToString('o'))"
    ) | Set-Content -LiteralPath $SummaryPath -Encoding ascii
    throw
} finally {
    Pop-Location
}

@(
    "result=PASS",
    "python_executable=$PythonExe",
    "iverilog_executable=$Iverilog",
    "verilator_environment=$MsysRoot",
    "rtl_testbench_count=$($Testbenches.Count)",
    "completed_utc=$([DateTime]::UtcNow.ToString('o'))"
) | Set-Content -LiteralPath $SummaryPath -Encoding ascii
Write-Host "NEURAEDGE_REGRESSION_PASSED tests=$($Testbenches.Count) summary=$SummaryPath"
```

84.3 Line annotations

Line 1: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 2: `[string]$IcarusRoot = "C:\msys64\ucrt64\bin",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `[string]$MsysRoot = "C:\msys64",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 4: `[string]$PythonExe = "",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 5: `[switch]$KeepExecutables` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 6: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 8: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 9: `Set-StrictMode -Version Latest` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Set: the value named 'set' in this block; StrictMode: the value named 'strict mode' in this block; Version: the value named 'version' in this block.

Line 11: `$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows run regression command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 12: `$OutputRoot = Join-Path $ProjectRoot "outputs\regression"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are OutputRoot: the value named 'output root' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 13: `New-Item -ItemType Directory -Force -Path $OutputRoot | Out-Null` This creates the declared output directory before a tool writes into it. '-Force' permits an existing directory but does not make an external tool failure succeed.

Line 14: `$SummaryPath = Join-Path $OutputRoot "regression_summary.txt"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are SummaryPath: the value named 'summary path' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 15: `@(` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 16: `"result=RUNNING",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are result: the value named 'result' in this block; RUNNING: the value named 'running' in this block.

Line 17: `"started_utc=$([DateTime]::UtcNow.ToString('o'))"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are started_utc: the value named 'started utc' in this block; DateTime: the value named 'date time' in this block; UtcNow: the value named 'utc now' in this block.

Line 18: `) | Set-Content -LiteralPath $SummaryPath -Encoding ascii` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Set: the value named 'set' in this block; Content: the value named 'content' in this block; LiteralPath: the value named 'literal path' in this block.

Line 20: `$Iverilog = Join-Path $IcarusRoot "iverilog.exe"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 21: `$Vvp = Join-Path $IcarusRoot "vvp.exe"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Vvp: the value named 'vvp' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 22: `if (-not (Test-Path -LiteralPath $Iverilog) -or -not (Test-Path -LiteralPath $Vvp)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 23: `throw "Icarus Verilog was not found under $IcarusRoot"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are throw: one output-row position processed by the tensor tile; Icarus: the value named 'icarus' in this block; Verilog: the value named 'verilog' in this block.

Line 24: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 26: `function Test-RegressionPython {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 27: `param([string]$Candidate)` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 28: `if ([string]::IsNullOrWhiteSpace($Candidate)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are string: the value named 'string' in this block; IsNullOrWhiteSpace: the value named 'is null or white space' in this block; Candidate: the value named 'candidate' in this block.

Line 29: `return $false` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 30: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 31: `$Command = Get-Command $Candidate -ErrorAction SilentlyContinue` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Get: the value named 'get' in this block; Candidate: the value named 'candidate' in this block.

Line 32: `if ($null -eq $Command) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are null: the value named 'null' in this block; eq: the value named 'eq' in this block; Command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 33: `return $false` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 34: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 35: `& $Command.Source -c "import numpy, torch" *> $null` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters; Source: the value named 'source' in this block; c: the value named 'c' in this block.

Line 36: `return $LASTEXITCODE -eq 0` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 37: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 39: `if ([string]::IsNullOrWhiteSpace($PythonExe)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 40: `$Candidates = [System.Collections.Generic.List[string]]::new()` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Candidates: the value named 'candidates' in this block; System: the value named 'system' in this block; Collections: the value named 'collections' in this block.

Line 41: `$SearchRoot = Get-Item -LiteralPath $ProjectRoot` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are SearchRoot: the value named 'search root' in this block; Get: the value named 'get' in this block; Item: the value named 'item' in this block.

Line 42: `for ($Depth = 0; $Depth -lt 5 -and $null -ne $SearchRoot; $Depth++) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are for: the value named 'for' in this block; Depth: the value named 'depth' in this block; lt: the value named 'lt' in this block.

Line 43: `$PreferredPython = Join-Path $SearchRoot.FullName '` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 44: `"external_research\spikingformer_py310\Scripts\python.exe"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 45: `if (Test-Path -LiteralPath $PreferredPython) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 46: `$Candidates.Add((Resolve-Path $PreferredPython).Path)` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 47: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 48: `$SearchRoot = $SearchRoot.Parent` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are SearchRoot: the value named 'search root' in this block; Parent: the value named 'parent' in this block.

Line 49: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 50: `$Launcher = Get-Command py.exe -ErrorAction SilentlyContinue` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Launcher: the value named 'launcher' in this block; Get: the value named 'get' in this block; Command: the 128-bit instruction selecting an engine, shape, mode, and numeric parameters.

Line 51: `if ($null -ne $Launcher) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are null: the value named 'null' in this block; ne: the value named 'ne' in this block; Launcher: the value named 'launcher' in this block.

Line 52: `foreach ($Version in @("3.14", "3.10")) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are foreach: the value named 'foreach' in this block; Version: the value named 'version' in this block; in: the value named 'in' in this block.

Line 53: `$Discovered = & $Launcher.Source "-$Version" -c "import sys; print(sys.executable)" 2>$null` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Discovered: the value named 'discovered' in this block; Launcher: the value named 'launcher' in this block; Source: the value named 'source' in this block.

Line 54: `if ($LASTEXITCODE -eq 0 -and -not [string]::IsNullOrWhiteSpace($Discovered)) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 55: `$Candidates.Add([string]($Discovered | Select-Object -Last 1))` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Candidates: the value named 'candidates' in this block; Add: the value named 'add' in this block; string: the value named 'string' in this block.

Line 56: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 57: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 58: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 59: `$PathPython = Get-Command python.exe -ErrorAction SilentlyContinue` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 60: `if ($null -ne $PathPython) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 61: `$Candidates.Add($PathPython.Source)` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 62: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 63: `foreach ($Candidate in $Candidates | Select-Object -Unique) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are foreach: the value named 'foreach' in this block; Candidate: the value named 'candidate' in this block; in: the value named 'in' in this block.

Line 64: `if (Test-RegressionPython $Candidate) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 65: `$PythonExe = (Get-Command $Candidate -ErrorAction Stop).Source` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 66: `break` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are break: the value named 'break' in this block.

Line 67: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 68: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 69: `if ([string]::IsNullOrWhiteSpace($PythonExe)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 70: `throw "No Python interpreter with NumPy and PyTorch was found. Pass -PythonExe explicitly."` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 71: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 72: `} elseif (-not (Test-RegressionPython $PythonExe)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 73: `throw "The requested Python interpreter is unavailable or cannot import NumPy and PyTorch: $PythonExe"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 74: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 76: `$PackageFile = Join-Path $ProjectRoot "rtl\neuraedge_command_pkg.sv"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are PackageFile: the value named 'package file' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 77: `$RtlFiles = @($PackageFile) + @(` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are RtlFiles: the value named 'rtl files' in this block; PackageFile: the value named 'package file' in this block.

Line 78: `Get-ChildItem -LiteralPath (Join-Path $ProjectRoot "rtl") -Filter "*.sv" -File |` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Get: the value named 'get' in this block; ChildItem: the value named 'child item' in this block; LiteralPath: the value named 'literal path' in this block.

Line 79: `Where-Object { $_.FullName -ne $PackageFile } |` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Where: the value named 'where' in this block; Object: the value named 'object' in this block; _: the value named '' in this block.

Line 80: `Sort-Object Name |` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Sort: the value named 'sort' in this block; Object: the value named 'object' in this block; Name: the value named 'name' in this block.

Line 81: `ForEach-Object FullName` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are ForEach: the value named 'for each' in this block; Object: the value named 'object' in this block; FullName: the value named 'full name' in this block.

Line 82: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 83: `$Testbenches = @(` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Testbenches: the value named 'testbenches' in this block.

Line 84: `"tb_neuraedge_mac2_u8",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_mac2_u8: the value named 'tb neuraedge mac2 u8' in this block.

Line 85: `"tb_neuraedge_tensor_tile",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_tensor_tile: the value named 'tb neuraedge tensor tile' in this block.

Line 86: `"tb_neuraedge_buffered_tensor_core",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_buffered_tensor_core: local storage that decouples producer and consumer timing.

Line 87: `"tb_neuraedge_alpha_xnor",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_alpha_xnor: the programmable weight assigned to matching zero spikes in alpha-XNOR attention.

Line 88: `"tb_neuraedge_vector_alu",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_vector_alu: the value named 'tb neuraedge vector alu' in this block.

Line 89: `"tb_neuraedge_integer_primitives",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_integer_primitives: the value named 'tb neuraedge integer primitives' in this block.

Line 90: `"tb_neuraedge_normalization_units",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_normalization_units: the value named 'tb neuraedge normalization units' in this block.

Line 91: `"tb_neuraedge_lif_bank",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_lif_bank: the value named 'tb neuraedge lif bank' in this block.

Line 92: `"tb_neuraedge_sgd_optimizer8",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_sgd_optimizer8: the value named 'tb neuraedge sgd optimizer8' in this block.

Line 93: `"tb_neuraedge_linear_backward8",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_linear_backward8: the value named 'tb neuraedge linear backward8' in this block.

Line 94: `"tb_neuraedge_activation_lut8",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_activation_lut8: an input feature value consumed by a neuron or multiply-accumulate operation.

Line 95: `"tb_neuraedge_universal_core",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_universal_core: the value named 'tb neuraedge universal core' in this block.

Line 96: `"tb_neuraedge_descriptor_sequencer",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_descriptor_sequencer: a DMA record header plus one command and its declared payload.

Line 97: `"tb_neuraedge_pynq_descriptor",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_pynq_descriptor: a DMA record header plus one command and its declared payload.

Line 98: `"tb_neuraedge_pynq_axi_top"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are tb_neuraedge_pynq_axi_top: the value named 'tb neuraedge pynq axi top' in this block.

Line 99: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 101: `Push-Location $ProjectRoot` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Push: the value named 'push' in this block; Location: the value named 'location' in this block; ProjectRoot: the value named 'project root' in this block.

Line 102: `try {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are try: the value named 'try' in this block.

Line 103: `$env:PYTHONPATH = (Join-Path $ProjectRoot "software")` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 104: `& $PythonExe (Join-Path $ProjectRoot "software\generate_tensor_tile_fixture.py") ‘` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 105: `-output-dir (Join-Path $ProjectRoot "tb\fixtures") 2>&1 |` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are output: the value named 'output' in this block; dir: the value named 'dir' in this block; Join: the value named 'join' in this block.

Line 106: `Tee-Object -FilePath (Join-Path $OutputRoot "tensor_fixture_generation.log")` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 107: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 108: `throw "Tensor fixture generation failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 109: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 110: `& $PythonExe (Join-Path $ProjectRoot "software\generate_operator_fixtures.py") ‘` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 111: `-output-dir (Join-Path $ProjectRoot "tb\fixtures") 2>&1 |` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are output: the value named 'output' in this block; dir: the value named 'dir' in this block; Join: the value named 'join' in this block.

Line 112: `Tee-Object -FilePath (Join-Path $OutputRoot "operator_fixture_generation.log")` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 113: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 114: `throw "Operator fixture generation failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 115: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 117: `& (Join-Path $PSScriptRoot "run_verilator_lint.ps1") -MsysRoot $MsysRoot 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 118: `Tee-Object -FilePath (Join-Path $OutputRoot "verilator_driver.log")` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 119: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 120: `throw "Verilator lint failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 121: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 123: `& $PythonExe -m unittest discover -s tests -v 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 124: `Tee-Object -FilePath (Join-Path $OutputRoot "python_unittest.log")` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 125: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 126: `throw "Python regression failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 127: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 129: `foreach ($Top in $Testbenches) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are foreach: the value named 'foreach' in this block; Top: the value named 'top' in this block; in: the value named 'in' in this block.

Line 130: `$TestbenchFile = Join-Path $ProjectRoot "tb\$Top.sv"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are TestbenchFile: the value named 'testbench file' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 131: `$Executable = Join-Path $OutputRoot "$Top.vvp"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Executable: the value named 'executable' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 132: `$CompileLog = Join-Path $OutputRoot "$Top.compile.log"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are CompileLog: the value named 'compile log' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 133: `$RunLog = Join-Path $OutputRoot "$Top.run.log"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are RunLog: the value named 'run log' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 135: `& $Iverilog -g2012 -I $ProjectRoot -s $Top -o $Executable @RtlFiles $TestbenchFile 2>&1 |` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 136: `Tee-Object -FilePath $CompileLog` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Tee: the value named 'tee' in this block; Object: the value named 'object' in this block; FilePath: the value named 'file path' in this block.

Line 137: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 138: `throw "$Top compilation failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 139: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 141: `& $Vvp $Executable 2>&1 | Tee-Object -FilePath $RunLog` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Vvp: the value named 'vvp' in this block; Executable: the value named 'executable' in this block; Tee: the value named 'tee' in this block.

Line 142: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 143: `throw "$Top simulation failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 144: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 145: `$RunText = Get-Content -LiteralPath $RunLog -Raw` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are RunText: the value named 'run text' in this block; Get: the value named 'get' in this block; Content: the value named 'content' in this block.

Line 146: `$PassPattern = "(?m)^$([regex]::Escape($Top)) PASSED\b"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are PassPattern: the value named 'pass pattern' in this block; m: the value named 'm' in this block; regex: the value named 'regex' in this block.

Line 147: `if ($RunText -notmatch $PassPattern) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are RunText: the value named 'run text' in this block; notmatch: the value named 'notmatch' in this block; PassPattern: the value named 'pass pattern' in this block.

Line 148: `throw "$Top simulation did not emit its required PASSED marker"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are throw: one output-row position processed by the tensor tile; Top: the value named 'top' in this block; simulation: the value named 'simulation' in this block.

Line 149: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 150: `if ($RunText -match "(?im)^\s*ERROR\b|^.*\bFAILED\b") {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are RunText: the value named 'run text' in this block; match: the value named 'match' in this block; im: the value named 'im' in this block.

Line 151: `throw "$Top simulation emitted an ERROR or FAILED marker"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are throw: one output-row position processed by the tensor tile; Top: the value named 'top' in this block; simulation: the value named 'simulation' in this block.

Line 152: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 153: `if (-not $KeepExecutables) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are not: the value named 'not' in this block; KeepExecutables: the value named 'keep executables' in this block.

Line 154: `Remove-Item -LiteralPath $Executable -Force` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 155: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 156: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 157: `} catch {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are catch: the value named 'catch' in this block.

Line 158: `@(` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 159: `"result=FAIL",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are result: the value named 'result' in this block; FAIL: the value named 'fail' in this block.

Line 160: `"message=$($_.Exception.Message)",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are message: the value named 'message' in this block; _: the value named '' in this block; Exception: the value named 'exception' in this block.

Line 161: `"completed_utc=$([DateTime]::UtcNow.ToString('o'))"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are completed_utc: the value named 'completed utc' in this block; DateTime: the value named 'date time' in this block; UtcNow: the value named 'utc now' in this block.

Line 162: `) | Set-Content -LiteralPath $SummaryPath -Encoding ascii` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Set: the value named 'set' in this block; Content: the value named 'content' in this block; LiteralPath: the value named 'literal path' in this block.

Line 163: `throw` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are throw: one output-row position processed by the tensor tile.

Line 164: `} finally {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are finally: the value named 'finally' in this block.

Line 165: `Pop-Location` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Pop: the value named 'pop' in this block; Location: the value named 'location' in this block.

Line 166: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 168: `@(` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 169: `"result=PASS",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are result: the value named 'result' in this block; PASS: the value named 'pass' in this block.

Line 170: `"python_executable=$PythonExe",` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 171: `"iverilog_executable=$Iverilog",` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 172: `"verilator_environment=$MsysRoot",` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run regression command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 173: `"rtl_testbench_count=$($Testbenches.Count)",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are rtl_testbench_count: an integer count retained for control or measurable telemetry; Testbenches: the value named 'testbenches' in this block; Count: the value named 'count' in this block.

Line 174: `"completed_utc=$([DateTime]::UtcNow.ToString('o'))"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are completed_utc: the value named 'completed utc' in this block; DateTime: the value named 'date time' in this block; UtcNow: the value named 'utc now' in this block.

Line 175: `) | Set-Content -LiteralPath $SummaryPath -Encoding ascii` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Set: the value named 'set' in this block; Content: the value named 'content' in this block; LiteralPath: the value named 'literal path' in this block.

Line 176: `Write-Host "NEURAEDGE_REGRESSION_PASSED tests=$($Testbenches.Count) summary=$SummaryPath"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run regression command-line flow. The important operands are Write: the value named 'write' in this block; Host: the value named 'host' in this block; NEURAEDGE_REGRESSION_PASSED: the value named 'neuraedge regression passed' in this block.

85 scripts/run_saif_power.ps1

Role in the complete system: orchestrates the repeatable Windows run saif power command-line flow.

85.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows run saif power command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

85.2 Complete source

```
[CmdletBinding()]
param(
    [string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",
    [string]$Checkpoint = "",
    [string]$OutputDirectory = "",
    [string]$SaifDirectory = "",
    [string]$AcceleratorInstance = "__ROOT__",
    [switch]$ReuseSaif
)

$ErrorActionPreference = "Stop"
Set-StrictMode -Version Latest
$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path
if ([string]::IsNullOrWhiteSpace($Checkpoint)) {
    $Checkpoint = Join-Path $ProjectRoot "outputs\pynq_ooc\neuraedge_pynq_axi_top_routed.dcp"
}
if ([string]::IsNullOrWhiteSpace($OutputDirectory)) {
    $OutputDirectory = Join-Path $ProjectRoot "outputs\power_activity"
}
$Checkpoint = [System.IO.Path]::GetFullPath($Checkpoint)
$OutputDirectory = [System.IO.Path]::GetFullPath($OutputDirectory)
if ([string]::IsNullOrWhiteSpace($SaifDirectory)) {
    $SaifDirectory = $OutputDirectory
}
$SaifDirectory = [System.IO.Path]::GetFullPath($SaifDirectory)

$Vivado = Join-Path $VivadoRoot "bin\vivado.bat"
$Tcl = Join-Path $PSScriptRoot "annotate_saif_power.tcl"
$WrapperSaif = Join-Path $SaifDirectory "saif\tb_neuraedge_pynq_axi_top.saif"
$CoreSaif = Join-Path $SaifDirectory "saif\tb_neuraedge_universal_core.saif"
if (-not $ReuseSaif) {
    & (Join-Path $PSScriptRoot "capture_power_activity.ps1") `
        -VivadoRoot $VivadoRoot -OutputDirectory $SaifDirectory
    if ($LASTEXITCODE -ne 0) {
        throw "SAIF capture failed with exit code $LASTEXITCODE"
    }
}
foreach ($Saif in @($WrapperSaif, $CoreSaif)) {
    if (-not (Test-Path -LiteralPath $Saif -PathType Leaf)) {
        throw "Required SAIF does not exist: $Saif"
    }
    if ((Get-Item -LiteralPath $Saif).Length -eq 0) {
        throw "Required SAIF is empty: $Saif"
    }
}
$ReportDirectory = Join-Path $OutputDirectory "vivado_reports"
& $Vivado -mode batch -nojournal -nolog -source $Tcl -tclargs `
    $Checkpoint $WrapperSaif $CoreSaif $ReportDirectory $AcceleratorInstance
if ($LASTEXITCODE -ne 0) {
    throw "Vivado SAIF power analysis failed with exit code $LASTEXITCODE"
}
Write-Output "NEURAEDGE_SAIF_POWER_COMPLETE report=$ReportDirectory\routed_saif_power.rpt"
```

85.3 Line annotations

Line 1: `[CmdletBinding()]` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are CmdletBinding: the value named 'cmdlet binding' in

this block.

Line 2: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `[string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 4: `[string]$Checkpoint = "",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 5: `[string]$OutputDirectory = "",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 6: `[string]$SaifDirectory = "",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 7: `[string]$AcceleratorInstance = "__ROOT__",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 8: `[switch]$ReuseSaif` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 9: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 11: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 12: `Set-StrictMode -Version Latest` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are Set: the value named 'set' in this block; StrictMode: the value named 'strict mode' in this block; Version: the value named 'version' in this block.

Line 13: `$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows run saif power command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 14: `if ([string]::IsNullOrWhiteSpace($Checkpoint)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are string: the value named 'string' in this block; IsNullOrWhiteSpace: the value named 'is null or white space' in this block; Checkpoint: the value named 'checkpoint' in this block.

Line 15: `$Checkpoint = Join-Path $ProjectRoot "outputs\pynq_ooc\neuraedge_pynq_axi_top_routed.dcp"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are Checkpoint: the value named 'checkpoint' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 16: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 17: `if ([string]::IsNullOrWhiteSpace($OutputDirectory)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are string: the value named 'string' in this block; IsNullOrWhiteSpace: the value named 'is null or white space' in this block; OutputDirectory: the value named 'output directory' in this block.

Line 18: `$OutputDirectory = Join-Path $ProjectRoot "outputs\power_activity"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are OutputDirectory: the value named 'output directory' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 19: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 20: `$Checkpoint = [System.IO.Path]::GetFullPath($Checkpoint)` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows run saif power command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 21: `$OutputDirectory = [System.IO.Path]::GetFullPath($OutputDirectory)` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows run saif power command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 22: `if ([string]::IsNullOrWhiteSpace($SaifDirectory)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are string: the value named 'string' in this block; IsNullOrWhiteSpace: the value named 'is null or white space' in this block; SaifDirectory: the value named 'saif directory' in this block.

Line 23: `$SaifDirectory = $OutputDirectory` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are SaifDirectory: the value named 'saif directory' in this block; OutputDirectory: the value named 'output directory' in this block.

Line 24: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 25: `$SaifDirectory = [System.IO.Path]::GetFullPath($SaifDirectory)` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows run saif power command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 27: `$Vivado = Join-Path $VivadoRoot "bin\vivado.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run saif power command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 28: `$Tcl = Join-Path $PSScriptRoot "annotate_saif_power.tcl"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are Tcl: the value named 'tcl' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 29: `$WrapperSaif = Join-Path $SaifDirectory "saif\tb_neuraedge_pynq_axi_top.saif"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are WrapperSaif: the value named 'wrapper saif' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 30: `$CoreSaif = Join-Path $SaifDirectory "saif\tb_neuraedge_universal_core.saif"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are CoreSaif: the value named 'core saif' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 31: `if (-not $ReuseSaif) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are not: the value named 'not' in this block; ReuseSaif: the value named 'reuse saif' in this block.

Line 32: `& (Join-Path $PSScriptRoot "capture_power_activity.ps1") ‘` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are Join: the value named 'join' in this block; Path: the value named 'path' in this block; PSScriptRoot: the value named 'psscript root' in this block.

Line 33: `-VivadoRoot $VivadoRoot -OutputDirectory $SaifDirectory` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run saif power command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 34: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 35: `throw "SAIF capture failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 36: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 37: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 38: `foreach ($Saif in @($WrapperSaif, $CoreSaif)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are foreach: the value named 'foreach' in this block; Saif: the value named 'saif' in this block; in: the value named 'in' in this block.

Line 39: `if (-not (Test-Path -LiteralPath $Saif -PathType Leaf)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are not: the value named 'not' in this block; Test: the value named 'test' in this block; Path: the value named 'path' in this block.

Line 40: `throw "Required SAIF does not exist: $Saif"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are throw: one output-row position processed by the tensor tile; Required: the value named 'required' in this block; SAIF: the value named 'saif' in this block.

Line 41: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 42: `if ((Get-Item -LiteralPath $Saif).Length -eq 0) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are Get: the value named 'get' in this block; Item: the value named 'item' in this block; LiteralPath: the value named 'literal path' in this block.

Line 43: `throw "Required SAIF is empty: $Saif"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are throw: one output-row position processed by the tensor tile; Required: the value named 'required' in this block; SAIF: the value named 'saif' in this block.

Line 44: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 45: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 46: `$ReportDirectory = Join-Path $OutputDirectory "vivado_reports"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run saif power command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 47: `& $Vivado -mode batch -nojournal -nolog -source $Tcl -tclargs ‘` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run saif power command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 48: `$Checkpoint $WrapperSaif $CoreSaif $ReportDirectory $AcceleratorInstance` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are Checkpoint: the value named 'checkpoint' in this block; WrapperSaif: the value named 'wrapper saif' in this block; CoreSaif: the value named 'core saif' in this block.

Line 49: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 50: `throw "Vivado SAIF power analysis failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 51: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 52: `Write-Output "NEURAEDGE_SAIF_POWER_COMPLETE report=$ReportDirectory\routed_saif_power.rpt"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run saif power command-line flow. The important operands are Write: the value named 'write' in this block; Output: the value named 'output' in this block; NEURAEDGE_SAIF_POWER_COMPLETE: the value named 'neuraedge saif power complete' in this block.

86 scripts/run_verilator_lint.ps1

Role in the complete system: orchestrates the repeatable Windows run verilator lint command-line flow.

86.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows run verilator lint command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

86.2 Complete source

```
param(
    [string]$MsysRoot = "C:\msys64"
)

$ErrorActionPreference = "Stop"
Set-StrictMode -Version Latest

$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path
$MsysShell = Join-Path $MsysRoot "msys2_shell.cmd"
if (-not (Test-Path -LiteralPath $MsysShell)) {
    throw "MSYS2 shell was not found at $MsysShell"
}

Push-Location $ProjectRoot
try {
    & $MsysShell -defterm -no-start -ucrt64 -here -c "bash scripts/run_verilator_lint.sh"
    if ($LASTEXITCODE -ne 0) {
        throw "Verilator lint failed with exit code $LASTEXITCODE"
    }
} finally {
    Pop-Location
}

Write-Host "NEURAEDGE_VERILATOR_LINT_PASSED"
```

86.3 Line annotations

Line 1: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 2: `[string]$MsysRoot = "C:\msys64"` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 5: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 6: `Set-StrictMode -Version Latest` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The important operands are Set: the value named 'set' in this block; StrictMode: the value named 'strict mode' in this block; Version: the value named 'version' in this block.

Line 8: `$ProjectRoot = (Resolve-Path (Join-Path $PSScriptRoot "..")).Path` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows run verilator lint command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 9: `$MsysShell = Join-Path $MsysRoot "msys2_shell.cmd"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The important operands are MsysShell: the value named 'msys shell' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 10: `if (-not (Test-Path -LiteralPath $MsysShell)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The important operands are not: the value named 'not' in this block; Test: the value named 'test' in this block; Path: the value named 'path' in this block.

Line 11: `throw "MSYS2 shell was not found at $MsysShell"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The important operands are throw: one output-row position processed by the tensor tile; MSYS2: the value named 'msys2' in this block; shell: the value named 'shell' in this block.

Line 12: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 14: `Push-Location $ProjectRoot` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The important operands are Push: the value named 'push' in this block; Location: the value named 'location' in this block; ProjectRoot: the value named 'project root' in this block.

Line 15: `try {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The important operands are try: the value named 'try' in this block.

Line 16: `& $MsysShell -defterm -no-start -ucrt64 -here -c "bash scripts/run_verilator_lint.sh"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run verilator lint command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 17: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 18: `throw "Verilator lint failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 19: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 20: `} finally {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The important operands are finally: the value named 'finally' in this block.

Line 21: `Pop-Location` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The important operands are Pop: the value named 'pop' in this block; Location: the value named 'location' in this block.

Line 22: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run verilator lint command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 24: `Write-Host "NEURAEDGE_VERILATOR_LINT_PASSED"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run verilator lint command-line flow; the exact command and arguments remain visible here for reproducibility.

87 scripts/run_vivado_build.ps1

Role in the complete system: orchestrates the repeatable Windows run vivado build command-line flow.

87.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to orchestrates the repeatable Windows run vivado build command-line flow.

Where it runs This is PowerShell orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

87.2 Complete source

```
[CmdletBinding()]
param(
    [string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",
    [switch]$CleanLogs,
    [switch]$Resume,
    [ValidateSet("Explore", "AggressiveExplore", "Default")]
    [string]$ResumeDirective = "Explore"
)

$ErrorActionPreference = "Stop"
$ProjectRoot = Split-Path -Parent $PSScriptRoot
$Vivado = Join-Path $VivadoRoot "bin\vivado.bat"
$Tcl = Join-Path $PSScriptRoot "build_pynq_ooc.tcl"
$ResumeTcl = Join-Path $PSScriptRoot "resume_pynq_ooc.tcl"
$OutputRoot = Join-Path $ProjectRoot "outputs\pynq_ooc"
$ResumeCheckpoint = Join-Path $OutputRoot "pre_postroute_physopt.dcp"

if (-not (Test-Path -LiteralPath $Vivado)) {
    throw "Vivado was not found at $Vivado"
}

if ($CleanLogs) {
    foreach ($Name in @("vivado.jou", "vivado.log")) {
        $Path = Join-Path $ProjectRoot $Name
        if (Test-Path -LiteralPath $Path) {
            Remove-Item -LiteralPath $Path -Force
        }
    }
    if (-not $Resume) {
        foreach ($Name in @(
            "pre_postroute_physopt.dcp",
            "postroute_physopt_unrouted.dcp",
            "implementation_summary.txt"
        )) {
            $Path = Join-Path $OutputRoot $Name
            if (Test-Path -LiteralPath $Path) {
                Remove-Item -LiteralPath $Path -Force
            }
        }
    }
}

Push-Location $ProjectRoot
try {
    if ($Resume) {
        if (-not (Test-Path -LiteralPath $ResumeCheckpoint)) {
            throw "No routed resume checkpoint was found at $ResumeCheckpoint"
        }
        & $Vivado -mode batch -nojournal -nolog -source $ResumeTcl `
            -tclargs pynq_ooc $ResumeDirective
        if ($LASTEXITCODE -ne 0) {
            throw "Vivado resume failed with exit code $LASTEXITCODE"
        }
    } else {
        & $Vivado -mode batch -nojournal -nolog -source $Tcl
        $BuildExitCode = $LASTEXITCODE
        if ($BuildExitCode -ne 0 -and (Test-Path -LiteralPath $ResumeCheckpoint)) {
            Write-Host "Primary Vivado flow exited with $BuildExitCode; resuming the saved route with $ResumeDirective."
            & $Vivado -mode batch -nojournal -nolog -source $ResumeTcl `
                -tclargs pynq_ooc $ResumeDirective
```

```
            if ($LASTEXITCODE -ne 0) {
                throw "Vivado implementation and routed-checkpoint resume both failed; primary=$BuildExitCode resume=$LASTEXITCODE"
            }
        } elseif ($BuildExitCode -ne 0) {
            throw "Vivado implementation failed with exit code $BuildExitCode before a routed checkpoint was available"
        }
    }
} finally {
    Pop-Location
}

Get-Content -LiteralPath (Join-Path $ProjectRoot "outputs\pynq_ooc\implementation_summary.txt")
```

87.3 Line annotations

Line 1: `[CmdletBinding()]` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are CmdletBinding: the value named 'cmdlet binding' in this block.

Line 2: `param(` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 3: `[string]$VivadoRoot = "D:\AMDDesignTools\2026.1\Vivado",` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 4: `[switch]$CleanLogs,` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 5: `[switch]$Resume,` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 6: `[ValidateSet("Explore", "AggressiveExplore", "Default")]` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are ValidateSet: the producer's statement that its current payload is meaningful; Explore: the value named 'explore' in this block; AggressiveExplore: the value named 'aggressive explore' in this block.

Line 7: `[string]$ResumeDirective = "Explore"` This declares a typed command-line parameter so the flow can be rerun with explicit, validated paths and options.

Line 8: `)` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 10: `$ErrorActionPreference = "Stop"` This makes PowerShell stop on errors so a failed prerequisite cannot be hidden by later successful commands.

Line 11: `$ProjectRoot = Split-Path -Parent $PSScriptRoot` This resolves a deterministic filesystem location for the flow that orchestrates the repeatable Windows run vivado build command-line flow. Literal/full paths prevent a different working directory from silently selecting the wrong source, checkpoint, or output.

Line 12: `$Vivado = Join-Path $VivadoRoot "bin\vivado.bat"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run vivado build command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 13: `$Tcl = Join-Path $PSScriptRoot "build_pynq_ooc.tcl"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are Tcl: the value named 'tcl' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 14: `$ResumeTcl = Join-Path $PSScriptRoot "resume_pynq_ooc.tcl"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are ResumeTcl: the value named 'resume tcl' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 15: `$OutputRoot = Join-Path $ProjectRoot "outputs\pynq_ooc"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are OutputRoot: the value named 'output root' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 16: `$ResumeCheckpoint = Join-Path $OutputRoot "pre_postroute_physopt.dcp"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are ResumeCheckpoint: the value named 'resume checkpoint' in this block; Join: the value named 'join' in this block; Path: the value named 'path' in this block.

Line 18: `if (-not (Test-Path -LiteralPath $Vivado)) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run vivado build command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 19: `throw "Vivado was not found at $Vivado"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run vivado build command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 20: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 22: `if ($CleanLogs) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are CleanLogs: the value named 'clean logs' in this block.

Line 23: `foreach ($Name in @("vivado.jou", "vivado.log")) {` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run vivado build command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 24: `$Path = Join-Path $ProjectRoot $Name` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are Path: the value named 'path' in this block; Join: the value named 'join' in this block; ProjectRoot: the value named 'project root' in this block.

Line 25: `if (Test-Path -LiteralPath $Path) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are Test: the value named 'test' in this block; Path: the value named 'path' in this block; LiteralPath: the value named 'literal path' in this block.

Line 26: `Remove-Item -LiteralPath $Path -Force` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 27: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 28: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 29: `if (-not $Resume) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are not: the value named 'not' in this block; Resume: the value named 'resume' in this block.

Line 30: `foreach ($Name in @(` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are foreach: the value named 'foreach' in this block; Name: the value named 'name' in this block; in: the value named 'in' in this block.

Line 31: `"pre_postroute_physopt.dcp",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are pre_postroute_physopt: the value named 'pre postroute physopt' in this block; dcp: the value named 'dcp' in this block.

Line 32: `"postroute_physopt_unrouted.dcp",` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are postroute_physopt_unrouted: the value named 'postroute physopt unrouted' in this block; dcp: the value named 'dcp' in this block.

Line 33: `"implementation_summary.txt"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are implementation_summary: the value named 'implementation summary' in this block; txt: the value named 'txt' in this block.

Line 34: `)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 35: `$Path = Join-Path $OutputRoot $Name` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are Path: the value named 'path' in this block; Join: the value named 'join' in this block; OutputRoot: the value named 'output root' in this block.

Line 36: `if (Test-Path -LiteralPath $Path) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are Test: the value named 'test' in this block; Path: the value named 'path' in this block; LiteralPath: the value named 'literal path' in this block.

Line 37: `Remove-Item -LiteralPath $Path -Force` This removes the explicitly named rebuildable artifact. The script uses a literal path so wildcard expansion cannot delete an unintended neighboring file.

Line 38: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 39: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 40: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 41: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 43: `Push-Location $ProjectRoot` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are Push: the value named 'push' in this block; Location: the value named 'location' in this block; ProjectRoot: the value named 'project root' in this block.

Line 44: `try {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are try: the value named 'try' in this block.

Line 45: `if ($Resume) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are Resume: the value named 'resume' in this block.

Line 46: `if (-not (Test-Path -LiteralPath $ResumeCheckpoint)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are not: the value named 'not' in this block; Test: the value named 'test' in this block; Path: the value named 'path' in this block.

Line 47: `throw "No routed resume checkpoint was found at $ResumeCheckpoint"` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are throw: one output-row position processed by the tensor tile; No: the value named 'no' in this block; routed: the value named 'routed' in this block.

Line 48: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 49: `& $Vivado -mode batch -nojournal -nolog -source $ResumeTcl ‘` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run vivado build command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 50: `-tclargs pynq_ooc $ResumeDirective` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are tclargs: the value named 'tclargs' in this block; pynq_ooc: the value named 'pynq ooc' in this block; ResumeDirective: the value named 'resume directive' in this block.

Line 51: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 52: `throw "Vivado resume failed with exit code $LASTEXITCODE"` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 53: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 54: `} else {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 55: `& $Vivado -mode batch -nojournal -nolog -source $Tcl` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run vivado build command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 56: `$BuildExitCode = $LASTEXITCODE` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 57: `if ($BuildExitCode -ne 0 -and (Test-Path -LiteralPath $ResumeCheckpoint)) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are BuildExitCode: the value named 'build exit code' in this block; ne: the value named 'ne' in this block; and: the value named 'and' in this block.

Line 58: `Write-Host "Primary Vivado flow exited with $BuildExitCode; resuming the saved route with $ResumeDir` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run vivado build command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 59: `& $Vivado -mode batch -nojournal -nolog -source $ResumeTcl ‘` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run vivado build command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 60: `-tclargs pynq_ooc $ResumeDirective` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are tclargs: the value named 'tclargs' in this block; pynq_ooc: the value named 'pynq ooc' in this block; ResumeDirective: the value named 'resume directive' in this block.

Line 61: `if ($LASTEXITCODE -ne 0) {` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 62: `throw "Vivado implementation and routed-checkpoint resume both failed; primary=$BuildExitCode resume=$LASTE...` This checks the external program's process exit code and turns a failed compiler, simulator, or Vivado run into a failed pipeline.

Line 63: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 64: `} elseif ($BuildExitCode -ne 0) {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are elseif: the value named 'elseif' in this block; BuildExitCode: the value named 'build exit code' in this block; ne: the value named 'ne' in this block.

Line 65: `throw "Vivado implementation failed with exit code $BuildExitCode before a routed checkpoint was available"` This locates or invokes a required tool as part of the script that orchestrates the repeatable Windows run vivado build command-line flow; the exact command and arguments remain visible here for reproducibility.

Line 66: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 67: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 68: `} finally {` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are finally: the value named 'finally' in this block.

Line 69: `Pop-Location` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are Pop: the value named 'pop' in this block; Location: the value named 'location' in this block.

Line 70: `}` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The expression is a literal or structural operator and has no hidden software state.

Line 72: `Get-Content -LiteralPath (Join-Path $ProjectRoot "outputs\pynq_ooc\implementation_summary.txt")` This batch-control line contributes to the flow that orchestrates the repeatable Windows run vivado build command-line flow. The important operands are Get: the value named 'get' in this block; Content: the value named 'content' in this block; LiteralPath: the value named 'literal path' in this block.

88 scripts/analyze_routed_timing.tcl

Role in the complete system: runs the reproducible Vivado analyze routed timing hardware flow.

88.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado analyze routed timing hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

88.2 Complete source

```
set script_dir [file dirname [file normalize [info script]]]
set project_dir [file dirname $script_dir]
set output_dir [file join $project_dir "outputs" "pynq_ooc"]

open_checkpoint [file join $output_dir "neuraedge_pynq_axi_top_routed.dcp"]
report_timing -delay_type max -max_paths 250 -nworst 1 -input_pins \
    -file [file join $output_dir "routed_timing_paths_250.rpt"]
report_methodology -file [file join $output_dir "routed_methodology.rpt"]
close_design
```

88.3 Line annotations

Line 1: `set script_dir [file dirname [file normalize [info script]]]` This batch-control line contributes to the flow that runs the reproducible Vivado analyze routed timing hardware flow. The important operands are set: the value named 'set' in this block; script_dir: the value named 'script dir' in this block; file: the value named 'file' in this block.

Line 2: `set project_dir [file dirname $script_dir]` This batch-control line contributes to the flow that runs the reproducible Vivado analyze routed timing hardware flow. The important operands are set: the value named 'set' in this block; project_dir: the value named 'project dir' in this block; file: the value named 'file' in this block.

Line 3: `set output_dir [file join $project_dir "outputs" "pynq_ooc"]` This batch-control line contributes to the flow that runs the reproducible Vivado analyze routed timing hardware flow. The important operands are set: the value named 'set' in this block; output_dir: the value named 'output dir' in this block; file: the value named 'file' in this block.

Line 5: `open_checkpoint [file join $output_dir "neuraedge_pynq_axi_top_routed.dcp"]` This batch-control line contributes to the flow that runs the reproducible Vivado analyze routed timing hardware flow. The important operands are open_checkpoint: the value named 'open checkpoint' in this block; file: the value named 'file' in this block; join: the value named 'join' in this block.

Line 6: `report_timing -delay_type max -max_paths 250 -nworst 1 -input_pins \` This writes an auditable Vivado artifact using 'report_timing -delay_type max -max_paths 250 -nworst 1 -input_pins \'; later documentation reads the resulting report rather than inventing a metric.

Line 7: `-file [file join $output_dir "routed_timing_paths_250.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado analyze routed timing hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 8: `report_methodology -file [file join $output_dir "routed_methodology.rpt"]` This writes an auditable Vivado artifact using ‘report_methodology -file [file join $output_dir "routed_methodology.rpt"]‘; later documentation reads the resulting report rather than inventing a metric.

Line 9: `close_design` This batch-control line contributes to the flow that runs the reproducible Vivado analyze routed timing hardware flow. The important operands are close_design: the value named ’close design’ in this block.

89 scripts/annotate_saif_power.tcl

Role in the complete system: runs the reproducible Vivado annotate saif power hardware flow.

89.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado annotate saif power hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

89.2 Complete source

```
if {$argc != 5} {
    error "Usage: annotate_saif_power.tcl <routed.dcp> <wrapper.saif> <core.saif> <output_dir> <accelerator_instance|__ROOT__>"
}
set checkpoint [file normalize [lindex $argv 0]]
set wrapper_saif [file normalize [lindex $argv 1]]
set core_saif [file normalize [lindex $argv 2]]
set output_dir [file normalize [lindex $argv 3]]
set accelerator_instance [lindex $argv 4]
file mkdir $output_dir
foreach required_file [list $checkpoint $wrapper_saif $core_saif] {
    if {![file exists $required_file]} {
        error "Required power-analysis input does not exist: $required_file"
    }
}

proc relocate_saif {source_path destination_path target_path} {
    set target_parts [split $target_path "/"]
    if {[llength $target_parts] < 2} {
        error "Relocated SAIF target must contain at least two hierarchy levels"
    }
    set source_handle [open $source_path "r"]
    set destination_handle [open $destination_path "w"]
    set replaced_instances 0
    try {
        while {[gets $source_handle line] >= 0} {
            if {$replaced_instances < 2 &&
                    [regexp {^(\s*)\(INSTANCE\s+\S+\s*$} $line -> indent]} {
                puts $destination_handle "${indent}(INSTANCE  [lindex $target_parts $replaced_instances]"
                incr replaced_instances
                if {$replaced_instances == 2} {
                    for {set index 2} {$index < [llength $target_parts]} {incr index} {
                        append indent "   "
                        puts $destination_handle "${indent}(INSTANCE  [lindex $target_parts $index]"
                    }
                }
            } else {
                puts $destination_handle $line
            }
        }
        if {$replaced_instances != 2} {
            error "Could not find the two simulated SAIF wrapper instances in $source_path"
        }
        for {set index 2} {$index < [llength $target_parts]} {incr index} {
            puts $destination_handle ")"
        }
    } finally {
        close $source_handle
        close $destination_handle
    }
}

open_checkpoint $checkpoint
if {$accelerator_instance eq "__ROOT__"} {
    read_saif -out_file [file join $output_dir "wrapper_saif_match.rpt"] \
        -strip_path tb_neuraedge_pynq_axi_top/dut -file $wrapper_saif
    current_instance u_core
    read_saif -out_file [file join $output_dir "core_saif_match.rpt"] \
        -strip_path tb_neuraedge_universal_core/dut -file $core_saif
    current_instance
} else {
```

```
    set accelerator_cells [get_cells -quiet -hierarchical -filter \
        "NAME == $accelerator_instance"]
    if {[llength $accelerator_cells] != 1} {
        error "Accelerator instance does not resolve uniquely: $accelerator_instance"
    }
    set relocated_wrapper [file join $output_dir "relocated_wrapper.saif"]
    set relocated_core [file join $output_dir "relocated_core.saif"]
    relocate_saif $wrapper_saif $relocated_wrapper $accelerator_instance
    relocate_saif $core_saif $relocated_core ${accelerator_instance}/u_core
    try {
        read_saif -out_file [file join $output_dir "wrapper_saif_match.rpt"] \
            -no_strip -file $relocated_wrapper
        read_saif -out_file [file join $output_dir "core_saif_match.rpt"] \
            -no_strip -file $relocated_core
    } finally {
        file delete -force $relocated_wrapper $relocated_core
    }
}
set_switching_activity -deassert_resets
if {$accelerator_instance eq "__ROOT__"} {
    set routed_nets [get_nets -hierarchical]
} else {
    current_instance $accelerator_instance
    set routed_nets [get_nets -hierarchical]
    current_instance
}
if {[llength $routed_nets] == 0} {
    error "The routed checkpoint contains no hierarchical nets to report"
}
report_switching_activity -file \
    [file join $output_dir "annotated_switching_activity.rpt"] $routed_nets
report_power -file [file join $output_dir "routed_saif_power.rpt"]
write_checkpoint -force [file join $output_dir "neuraedge_saif_annotated.dcp"]
close_design
```

89.3 Line annotations

Line 1: `if {$argc != 5} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are argc: the value named 'argc' in this block.

Line 2: `error "Usage: annotate_saif_power.tcl <routed.dcp> <wrapper.saif> <core.saif> <output_dir> <accelerat` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are error: the value named 'error' in this block; Usage: the value named 'usage' in this block; annotate_saif_power: the value named 'annotate saif power' in this block.

Line 3: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 4: `set checkpoint [file normalize [lindex $argv 0]]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; checkpoint: the value named 'checkpoint' in this block; file: the value named 'file' in this block.

Line 5: `set wrapper_saif [file normalize [lindex $argv 1]]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; wrapper_saif: the value named 'wrapper saif' in this block; file: the value named 'file' in this block.

Line 6: `set core_saif [file normalize [lindex $argv 2]]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; core_saif: the value named 'core saif' in this block; file: the value named 'file' in this block.

Line 7: `set output_dir [file normalize [lindex $argv 3]]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; output_dir: the value named 'output dir' in this block; file: the value named 'file' in this block.

Line 8: `set accelerator_instance [lindex $argv 4]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; accelerator_instance: the value named 'accelerator instance' in this block; lindex: an integer selecting one tensor element, lane, bank, or state entry.

Line 9: `file mkdir $output_dir` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are file: the value named 'file' in this block; mkdir: the value named 'mkdir' in this block; output_dir: the value named 'output dir' in this block.

Line 10: `foreach required_file [list $checkpoint $wrapper_saif $core_saif] {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are foreach: the value named 'foreach' in this block; required_file: the value named 'required file' in this block; list: the value named 'list' in this block.

Line 11: `if {![file exists $required_file]} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are file: the value named 'file' in this block; exists: the value named 'exists' in this block; required_file: the value named 'required file' in this block.

Line 12: `error "Required power-analysis input does not exist: $required_file"` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are error: the value named 'error' in this block; Required: the value named 'required' in this block; power: the value named 'power' in this block.

Line 13: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 14: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 16: `proc relocate_saif {source_path destination_path target_path} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are proc: the value named 'proc' in this block; relocate_saif: the value named 'relocate saif' in this block; source_path: the value named 'source path' in this block.

Line 17: `set target_parts [split $target_path "/"]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; target_parts: the value named 'target parts' in this block; split: the value named 'split' in this block.

Line 18: `if {[llength $target_parts] < 2} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are llength: the value named 'llength' in this block; target_parts: the value named 'target parts' in this block.

Line 19: `error "Relocated SAIF target must contain at least two hierarchy levels"` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are error: the value named 'error' in this block; Relocated: the value named 'relocated' in this block; SAIF: the value named 'saif' in this block.

Line 20: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 21: `set source_handle [open $source_path "r"]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; source_handle: the value named 'source handle' in this block; open: the value named 'open' in this block.

Line 22: `set destination_handle [open $destination_path "w"]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; destination_handle: the value named 'destination handle' in this block; open: the value named 'open' in this block.

Line 23: `set replaced_instances 0` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; replaced_instances: the value named 'replaced instances' in this block.

Line 24: `try {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are try: the value named 'try' in this block.

Line 25: `while {[gets $source_handle line] >= 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are while: the value named 'while' in this block; gets: the value named 'gets' in this block; source_handle: the value named 'source handle' in this block.

Line 26: `if {$replaced_instances < 2 &&` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are replaced_instances: the value named 'replaced instances' in this block.

Line 27: `[regexp {^(\s*)\(INSTANCE\s+\S+\s*$} $line -> indent]} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are regexp: the value named 'regexp' in this block; s: the value named 's' in this block; INSTANCE: the value named 'instance' in this block.

Line 28: `puts $destination_handle "${indent}(INSTANCE [lindex $target_parts $replaced_instances]"` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are puts: the value named 'puts' in this block; destination_handle: the value named 'destination handle' in this block; indent: the value named 'indent' in this block.

Line 29: `incr replaced_instances` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are incr: the value named 'incr' in this block; replaced_instances: the value named 'replaced instances' in this block.

Line 30: `if {$replaced_instances == 2} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are replaced_instances: the value named 'replaced instances' in this block.

Line 31: `for {set index 2} {$index < [llength $target_parts]} {incr index} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are for: the value named 'for' in this block; set: the value named 'set' in this block; index: an integer selecting one tensor element, lane, bank, or state entry.

Line 32: `append indent " "` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are append: the value named 'append' in this block; indent: the value named 'indent' in this block.

Line 33: `puts $destination_handle "${indent}(INSTANCE [lindex $target_parts $index]"` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are puts: the value named 'puts' in this block; destination_handle: the value named 'destination handle' in this block; indent: the value named 'indent' in this block.

Line 34: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 35: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 36: `} else {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 37: `puts $destination_handle $line` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are puts: the value named 'puts' in this block; destination_handle: the value named 'destination handle' in this block; line: the value named 'line' in this block.

Line 38: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 39: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 40: `if {$replaced_instances != 2} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are replaced_instances: the value named 'replaced instances' in this block.

Line 41: `error "Could not find the two simulated SAIF wrapper instances in $source_path"` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are error: the value named 'error' in this block; Could: the value named 'could' in this block; not: the value named 'not' in this block.

Line 42: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 43: `for {set index 2} {$index < [llength $target_parts]} {incr index} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are for: the value named 'for' in this block; set: the value named 'set' in this block; index: an integer selecting one tensor element, lane, bank, or state entry.

Line 44: `puts $destination_handle ")"` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are puts: the value named 'puts' in this block; destination_handle: the value named 'destination handle' in this block.

Line 45: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 46: `} finally {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are finally: the value named 'finally' in this block.

Line 47: `close $source_handle` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are close: the value named 'close' in this block; source_handle: the value named 'source handle' in this block.

Line 48: `close $destination_handle` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are close: the value named 'close' in this block; destination_handle: the value named 'destination handle' in this block.

Line 49: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 50: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 52: `open_checkpoint $checkpoint` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are open_checkpoint: the value named 'open checkpoint' in this block; checkpoint: the value named 'checkpoint' in this block.

Line 53: `if {$accelerator_instance eq "__ROOT__"} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are accelerator_instance: the value named 'accelerator instance' in this block; eq: the value named 'eq' in this block; ___ROOT___: the value named 'root' in this block.

Line 54: `read_saif -out_file [file join $output_dir "wrapper_saif_match.rpt"] \` This annotates implemented nets with measured XSim switching activity from the named SAIF file. The retained match report quantifies which simulated names mapped to routed nets: 'read_saif -out_file [file join $output_dir "wrapper_saif_match.rpt"] \'.

Line 55: `-strip_path tb_neuraedge_pynq_axi_top/dut -file $wrapper_saif` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are strip_path: the value named 'strip path' in this block; tb_neuraedge_pynq_axi_top: the value named 'tb neuraedge pynq axi top' in this block; dut: the value named 'dut' in this block.

Line 56: `current_instance u_core` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are current_instance: the value named 'current instance' in this block; u_core: the value named 'u core' in this block.

Line 57: `read_saif -out_file [file join $output_dir "core_saif_match.rpt"] \` This annotates implemented nets with measured XSim switching activity from the named SAIF file. The retained match report quantifies which simulated names mapped to routed nets: 'read_saif -out_file [file join $output_dir "core_saif_match.rpt"] \'.

Line 58: `-strip_path tb_neuraedge_universal_core/dut -file $core_saif` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are strip_path: the value named 'strip path' in this block; tb_neuraedge_universal_core: the value named 'tb neuraedge universal core' in this block; dut: the value named 'dut' in this block.

Line 59: `current_instance` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are current_instance: the value named 'current instance' in this block.

Line 60: `} else {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 61: `set accelerator_cells [get_cells -quiet -hierarchical -filter \` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; accelerator_cells: the value named 'accelerator cells' in this block; get_cells: the value named 'get cells' in this block.

Line 62: `"NAME == $accelerator_instance"]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are NAME: the value named 'name' in this block; accelerator_instance: the value named 'accelerator instance' in this block.

Line 63: `if {[llength $accelerator_cells] != 1} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are llength: the value named 'llength' in this block; accelerator_cells: the value named 'accelerator cells' in this block.

Line 64: `error "Accelerator instance does not resolve uniquely: $accelerator_instance"` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are error: the value named 'error' in this block; Accelerator: the value named 'accelerator' in this block; instance: the value named 'instance' in this block.

Line 65: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 66: `set relocated_wrapper [file join $output_dir "relocated_wrapper.saif"]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; relocated_wrapper: the value named 'relocated wrapper' in this block; file: the value named 'file' in this block.

Line 67: `set relocated_core [file join $output_dir "relocated_core.saif"]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; relocated_core: the value named 'relocated core' in this block; file: the value named 'file' in this block.

Line 68: `relocate_saif $wrapper_saif $relocated_wrapper $accelerator_instance` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are relocate_saif: the value named 'relocate saif' in this block; wrapper_saif: the value named 'wrapper saif' in this block; relocated_wrapper: the value named 'relocated wrapper' in this block.

Line 69: `relocate_saif $core_saif $relocated_core ${accelerator_instance}/u_core` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are relocate_saif: the value named 'relocate saif' in this block; core_saif: the value named 'core saif' in this block; relocated_core: the value named 'relocated core' in this block.

Line 70: `try {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are try: the value named 'try' in this block.

Line 71: `read_saif -out_file [file join $output_dir "wrapper_saif_match.rpt"] \` This annotates implemented nets with measured XSim switching activity from the named SAIF file. The retained match report quantifies which simulated names mapped to routed nets: 'read_saif -out_file [file join $output_dir "wrapper_saif_match.rpt"] \'.

Line 72: `-no_strip -file $relocated_wrapper` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are no_strip: the value named 'no strip' in this block; file: the value named 'file' in this block; relocated_wrapper: the value named 'relocated wrapper' in this block.

Line 73: `read_saif -out_file [file join $output_dir "core_saif_match.rpt"] \` This annotates implemented nets with measured XSim switching activity from the named SAIF file. The retained match report quantifies which simulated names mapped to routed nets: 'read_saif -out_file [file join $output_dir "core_saif_match.rpt"] \'.

Line 74: `-no_strip -file $relocated_core` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are no_strip: the value named 'no strip' in this block; file: the value named 'file' in this block; relocated_core: the value named 'relocated core' in this block.

Line 75: `} finally {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are finally: the value named 'finally' in this block.

Line 76: `file delete -force $relocated_wrapper $relocated_core` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are file: the value named 'file' in this block; delete: the value named 'delete' in this block; force: the value named 'force' in this block.

Line 77: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 78: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 79: `set_switching_activity -deassert_resets` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set_switching_activity: the value named 'set switching activity' in this block; deassert_resets: the value named 'deassert resets' in this block.

Line 80: `if {$accelerator_instance eq "__ROOT__"} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are accelerator_instance: the value named 'accelerator instance' in this block; eq: the value named 'eq' in this block; ___ROOT___: the value named 'root' in this block.

Line 81: `set routed_nets [get_nets -hierarchical]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; routed_nets: the value named 'routed nets' in this block; get_nets: the value named 'get nets' in this block.

Line 82: `} else {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 83: `current_instance $accelerator_instance` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are current_instance: the value named 'current instance' in this block; accelerator_instance: the value named 'accelerator instance' in this block.

Line 84: `set routed_nets [get_nets -hierarchical]` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are set: the value named 'set' in this block; routed_nets: the value named 'routed nets' in this block; get_nets: the value named 'get nets' in this block.

Line 85: `current_instance` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are current_instance: the value named 'current instance' in this block.

Line 86: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 87: `if {[llength $routed_nets] == 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are llength: the value named 'llength' in this block; routed_nets: the value named 'routed nets' in this block.

Line 88: `error "The routed checkpoint contains no hierarchical nets to report"` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are error: the value named 'error' in this block; The: the value named 'the' in this block; routed: the value named 'routed' in this block.

Line 89: `}` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 90: `report_switching_activity -file \` This writes an auditable Vivado artifact using 'report_switching_activity -file \'; later documentation reads the resulting report rather than inventing a metric.

Line 91: `[file join $output_dir "annotated_switching_activity.rpt"] $routed_nets` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 92: `report_power -file [file join $output_dir "routed_saif_power.rpt"]` This writes an auditable Vivado artifact using 'report_power -file [file join $output_dir "routed_saif_power.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 93: `write_checkpoint -force [file join $output_dir "neuraedge_saif_annotated.dcp"]` This writes an auditable Vivado artifact using 'write_checkpoint -force [file join $output_dir "neuraedge_saif_annotated.dcp"]'; later documentation reads the resulting report rather than inventing a metric.

Line 94: `close_design` This batch-control line contributes to the flow that runs the reproducible Vivado annotate saif power hardware flow. The important operands are close_design: the value named 'close design' in this block.

90 scripts/build_pynq_full.tcl

Role in the complete system: runs the reproducible Vivado build pynq full hardware flow.

90.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado build pynq full hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

90.2 Complete source

```
# Build the complete PYNQ-Z2 processing-system, DMA, and NeuraEdge overlay.
# The board preset is vendored from the AMD/Xilinx Board Store and configures
# the physical DDR3 and MIO pins. FCLK0 supplies 100 MHz control, FCLK1 supplies
# a timing-safe 125 MHz DMA/DDR path, and FCLK2 supplies the 200 MHz accelerator.

set script_dir [file normalize [file dirname [info script]]]
set project_dir [file normalize [file join $script_dir ".."]]
set board_store [file join $project_dir "third_party" "XilinxBoardStore" "boards"]
set output_dir [file join $project_dir "outputs" "pynq_full"]
if {[info exists ::env(NEURAEDGE_VIVADO_BUILD_ROOT)]} {
    set build_root [file normalize $::env(NEURAEDGE_VIVADO_BUILD_ROOT)]
} else {
    set build_root [file normalize "D:/NeuraEdgeBuild"]
}
set vivado_project_dir [file join $build_root "pynq_full"]
set build_mode "full"
if {$argc >= 1} {
    set build_mode [lindex $argv 0]
}
if {$build_mode ni {validate full}} {
    error "Build mode must be validate or full"
}
file mkdir $output_dir
if {$build_mode eq "full"} {
    # A failed rebuild must never leave an older deployable image looking like
    # the result of the current source tree. Diagnostic reports are overwritten
    # in place, while accepted deployment artifacts are republished only after
    # route, timing, and DRC gates pass below.
    foreach stale_artifact {
        neuraedge_pynq_full.bit
        neuraedge_pynq_full.hwh
        neuraedge_pynq_full.xsa
        full_implementation_summary.txt
    } {
        set stale_path [file join $output_dir $stale_artifact]
        if {[file exists $stale_path]} {
            file delete -force $stale_path
        }
    }
}

set_param board.repoPaths [list $board_store]
set board_part "tul.com.tw:pynq-z2:part0:1.0"
if {[llength [get_board_parts -quiet $board_part]] != 1} {
    error "Vendored PYNQ-Z2 board part was not discovered at $board_store"
}

create_project neuraedge_pynq_full $vivado_project_dir -force -part xc7z020clg400-1
set_property board_part $board_part [current_project]
set_property target_language Verilog [current_project]
set_property simulator_language Mixed [current_project]

set rtl_files [list \
    [file join $project_dir "rtl" "neuraedge_command_pkg.sv"] \
    [file join $project_dir "rtl" "neuraedge_mac2_u8.sv"] \
    [file join $project_dir "rtl" "neuraedge_tensor_tile.sv"] \
    [file join $project_dir "rtl" "neuraedge_active_index_buffer.sv"] \
    [file join $project_dir "rtl" "neuraedge_activation_buffer.sv"] \
    [file join $project_dir "rtl" "neuraedge_weight_bank.sv"] \
    [file join $project_dir "rtl" "neuraedge_buffered_tensor_core.sv"] \
```

```
    [file join $project_dir "rtl" "neuraedge_alpha_xnor.sv"] \
    [file join $project_dir "rtl" "neuraedge_vector_alu.sv"] \
    [file join $project_dir "rtl" "neuraedge_udivider.sv"] \
    [file join $project_dir "rtl" "neuraedge_isqrt64.sv"] \
    [file join $project_dir "rtl" "neuraedge_integer_layernorm.sv"] \
    [file join $project_dir "rtl" "neuraedge_log_softmax.sv"] \
    [file join $project_dir "rtl" "neuraedge_lif_bank.sv"] \
    [file join $project_dir "rtl" "neuraedge_sgd_optimizer8.sv"] \
    [file join $project_dir "rtl" "neuraedge_gradient_accumulator_bank.sv"] \
    [file join $project_dir "rtl" "neuraedge_linear_backward8.sv"] \
    [file join $project_dir "rtl" "neuraedge_activation_lut8.sv"] \
    [file join $project_dir "rtl" "neuraedge_universal_core.sv"] \
    [file join $project_dir "rtl" "neuraedge_descriptor_sequencer.sv"] \
    [file join $project_dir "rtl" "neuraedge_pynq_axi_top.sv"]]
add_files -norecurse $rtl_files
set_property file_type SystemVerilog [get_files $rtl_files]
set_property top neuraedge_pynq_axi_top [current_fileset]
update_compile_order -fileset sources_1

# Package the verified RTL with an explicit IP-XACT memory map. A raw module
# reference exposes AXI pins but has no address block, which prevents Vivado's
# Address Editor and PYNQ's HWH parser from describing the control registers.
set ip_repo [file join $output_dir "ip_repo" "neuraedge_universal_1_0"]
file mkdir $ip_repo
ipx::package_project -root_dir $ip_repo -vendor neuraedge.local \
    -library accelerator -taxonomy /UserIP -import_files -set_current true
set packaged_core [ipx::current_core]
set_property name neuraedge_universal $packaged_core
set_property display_name "NeuraEdge Universal Accelerator" $packaged_core
set_property description "INT8, sparse, spiking, transformer, and on-fabric optimizer accelerator" $packaged_core
set_property version 1.0 $packaged_core
set_property core_revision 1 $packaged_core
set s_axi_interface [ipx::get_bus_interfaces S_AXI -of_objects $packaged_core]
ipx::add_memory_map S_AXI $packaged_core
set_property slave_memory_map_ref S_AXI $s_axi_interface
set memory_map [ipx::get_memory_maps S_AXI -of_objects $packaged_core]
ipx::add_address_block Reg $memory_map
set register_block [ipx::get_address_blocks Reg -of_objects $memory_map]
set_property range 256 $register_block
set_property width 32 $register_block
ipx::associate_bus_interfaces -busif S_AXI -clock aclk $packaged_core
ipx::associate_bus_interfaces -busif S_AXIS -clock aclk $packaged_core
ipx::associate_bus_interfaces -busif M_AXIS -clock aclk $packaged_core
ipx::create_xgui_files $packaged_core
ipx::update_checksums $packaged_core
ipx::save_core $packaged_core
set_property ip_repo_paths [list [file dirname $ip_repo]] [current_project]
update_ip_catalog

create_bd_design system

set ps7 [create_bd_cell -type ip -vlnv xilinx.com:ip:processing_system7:5.5 ps7]
apply_bd_automation -rule xilinx.com:bd_rule:processing_system7 \
    -config {apply_board_preset "1" make_external "FIXED_IO, DDR" Master "Disable" Slave "Disable"} \
    $ps7
set_property -dict [list \
    CONFIG.PCW_USE_M_AXI_GP0 {1} \
    CONFIG.PCW_USE_S_AXI_HP0 {1} \
    CONFIG.PCW_S_AXI_HP0_DATA_WIDTH {64} \
    CONFIG.PCW_S_AXI_HP0_ID_WIDTH {1} \
    CONFIG.PCW_USE_FABRIC_INTERRUPT {1} \
    CONFIG.PCW_IRQ_F2P_INTR {1} \
    CONFIG.PCW_EN_CLK0_PORT {1} \
    CONFIG.PCW_EN_CLK1_PORT {1} \
    CONFIG.PCW_EN_CLK2_PORT {1} \
    CONFIG.PCW_FPGA0_PERIPHERAL_FREQMHZ {100.000000} \
    CONFIG.PCW_FPGA1_PERIPHERAL_FREQMHZ {125.000000} \
    CONFIG.PCW_FPGA2_PERIPHERAL_FREQMHZ {200.000000}] $ps7
# The board preset assigns 10 MHz to this disabled-by-default interface. Once
# GP0 is enabled, let block-design propagation derive its frequency from the
# connected 100 MHz FCLK0 instead of retaining that stale user-strength value.
set_property CONFIG.PCW_M_AXI_GP0_FREQMHZ.VALUE_SRC auto $ps7

set reset_100 [create_bd_cell -type ip -vlnv xilinx.com:ip:proc_sys_reset:5.0 reset_100]
set reset_125 [create_bd_cell -type ip -vlnv xilinx.com:ip:proc_sys_reset:5.0 reset_125]
set reset_200 [create_bd_cell -type ip -vlnv xilinx.com:ip:proc_sys_reset:5.0 reset_200]
set accelerator_control_cdc [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_clock_converter:2.1 accelerator_control_cdc]
set dma_control_cdc [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_clock_converter:2.1 dma_control_cdc]
set control_interconnect [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_interconnect:2.1 control_interconnect]
set_property -dict [list CONFIG.NUM_SI {1} CONFIG.NUM_MI {2}] $control_interconnect

set dma [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_dma:7.1 dma]
set_property -dict [list \
    CONFIG.c_include_sg {0} \
    CONFIG.c_sg_length_width {26} \
    CONFIG.c_include_mm2s {1} \
    CONFIG.c_include_s2mm {1} \
```

```
    CONFIG.c_include_mm2s_dre {1} \
    CONFIG.c_include_s2mm_dre {1} \
    CONFIG.c_m_axi_mm2s_data_width {64} \
    CONFIG.c_m_axi_s2mm_data_width {64} \
    CONFIG.c_m_axis_mm2s_tdata_width {64} \
    CONFIG.c_s_axis_s2mm_tdata_width {64} \
    CONFIG.c_mm2s_burst_size {256} \
    CONFIG.c_s2mm_burst_size {256}] $dma

set memory_interconnect [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_interconnect:2.1 memory_interconnect]
set_property -dict [list CONFIG.NUM_SI {2} CONFIG.NUM_MI {1}] $memory_interconnect

set mm2s_cdc [create_bd_cell -type ip -vlnv xilinx.com:ip:axis_clock_converter:1.1 mm2s_cdc]
set s2mm_cdc [create_bd_cell -type ip -vlnv xilinx.com:ip:axis_clock_converter:1.1 s2mm_cdc]
foreach stream_cdc [list $mm2s_cdc $s2mm_cdc] {
    set_property -dict [list \
        CONFIG.TDATA_NUM_BYTES {8} \
        CONFIG.HAS_TKEEP {1} \
        CONFIG.HAS_TLAST {1} \
        CONFIG.IS_ACLK_ASYNC {1} \
        CONFIG.SYNCHRONIZATION_STAGES {3}] $stream_cdc
}

set accelerator [create_bd_cell -type ip \
    -vlnv neuraedge.local:accelerator:neuraedge_universal:1.0 accelerator]
set interrupt_concat [create_bd_cell -type inline_hdl \
    -vlnv xilinx.com:inline_hdl:ilconcat:1.0 interrupt_concat]
set_property CONFIG.NUM_PORTS {16} $interrupt_concat
set interrupt_zero [create_bd_cell -type inline_hdl \
    -vlnv xilinx.com:inline_hdl:ilconstant:1.0 interrupt_zero]
set_property -dict [list CONFIG.CONST_WIDTH {1} CONFIG.CONST_VAL {0}] $interrupt_zero

# Control path: the 100 MHz GP0/interconnect fans out through explicit CDCs to
# the 200 MHz accelerator and the 125 MHz DMA register interface.
connect_bd_intf_net [get_bd_intf_pins ps7/M_AXI_GP0] [get_bd_intf_pins control_interconnect/S00_AXI]
connect_bd_intf_net [get_bd_intf_pins control_interconnect/M00_AXI] [get_bd_intf_pins accelerator_control_cdc/S_AXI]
connect_bd_intf_net [get_bd_intf_pins accelerator_control_cdc/M_AXI] [get_bd_intf_pins accelerator/S_AXI]
connect_bd_intf_net [get_bd_intf_pins control_interconnect/M01_AXI] [get_bd_intf_pins dma_control_cdc/S_AXI]
connect_bd_intf_net [get_bd_intf_pins dma_control_cdc/M_AXI] [get_bd_intf_pins dma/S_AXI_LITE]

# Streaming path: 125 MHz DDR/DMA -> asynchronous stream CDC -> 200 MHz
# accelerator -> asynchronous stream CDC -> 125 MHz DMA/DDR.
connect_bd_intf_net [get_bd_intf_pins dma/M_AXIS_MM2S] [get_bd_intf_pins mm2s_cdc/S_AXIS]
connect_bd_intf_net [get_bd_intf_pins mm2s_cdc/M_AXIS] [get_bd_intf_pins accelerator/S_AXIS]
connect_bd_intf_net [get_bd_intf_pins accelerator/M_AXIS] [get_bd_intf_pins s2mm_cdc/S_AXIS]
connect_bd_intf_net [get_bd_intf_pins s2mm_cdc/M_AXIS] [get_bd_intf_pins dma/S_AXIS_S2MM]
connect_bd_intf_net [get_bd_intf_pins dma/M_AXI_MM2S] [get_bd_intf_pins memory_interconnect/S00_AXI]
connect_bd_intf_net [get_bd_intf_pins dma/M_AXI_S2MM] [get_bd_intf_pins memory_interconnect/S01_AXI]
connect_bd_intf_net [get_bd_intf_pins memory_interconnect/M00_AXI] [get_bd_intf_pins ps7/S_AXI_HP0]

# Clock and reset domains are explicit; there is no unconstrained interface CDC.
connect_bd_net [get_bd_pins ps7/FCLK_CLK0] \
    [get_bd_pins ps7/M_AXI_GP0_ACLK] \
    [get_bd_pins reset_100/slowest_sync_clk] \
    [get_bd_pins control_interconnect/ACLK] \
    [get_bd_pins control_interconnect/S00_ACLK] \
    [get_bd_pins control_interconnect/M00_ACLK] \
    [get_bd_pins control_interconnect/M01_ACLK] \
    [get_bd_pins accelerator_control_cdc/s_axi_aclk] \
    [get_bd_pins dma_control_cdc/s_axi_aclk]
connect_bd_net [get_bd_pins ps7/FCLK_CLK1] \
    [get_bd_pins ps7/S_AXI_HP0_ACLK] \
    [get_bd_pins reset_125/slowest_sync_clk] \
    [get_bd_pins memory_interconnect/ACLK] \
    [get_bd_pins memory_interconnect/S00_ACLK] \
    [get_bd_pins memory_interconnect/S01_ACLK] \
    [get_bd_pins memory_interconnect/M00_ACLK] \
    [get_bd_pins dma/s_axi_lite_aclk] \
    [get_bd_pins dma/m_axi_mm2s_aclk] \
    [get_bd_pins dma/m_axi_s2mm_aclk] \
    [get_bd_pins dma_control_cdc/m_axi_aclk] \
    [get_bd_pins mm2s_cdc/s_axis_aclk] \
    [get_bd_pins s2mm_cdc/m_axis_aclk]
connect_bd_net [get_bd_pins ps7/FCLK_CLK2] \
    [get_bd_pins reset_200/slowest_sync_clk] \
    [get_bd_pins accelerator_control_cdc/m_axi_aclk] \
    [get_bd_pins mm2s_cdc/m_axis_aclk] \
    [get_bd_pins s2mm_cdc/s_axis_aclk] \
    [get_bd_pins accelerator/aclk]
connect_bd_net [get_bd_pins ps7/FCLK_RESET0_N] \
    [get_bd_pins reset_100/ext_reset_in] \
    [get_bd_pins reset_125/ext_reset_in] \
    [get_bd_pins reset_200/ext_reset_in]
connect_bd_net [get_bd_pins reset_100/interconnect_aresetn] \
    [get_bd_pins control_interconnect/ARESETN] \
    [get_bd_pins control_interconnect/S00_ARESETN] \
    [get_bd_pins control_interconnect/M00_ARESETN] \
```

```
    [get_bd_pins control_interconnect/M01_ARESETN] \
    [get_bd_pins accelerator_control_cdc/s_axi_aresetn] \
    [get_bd_pins dma_control_cdc/s_axi_aresetn]
connect_bd_net [get_bd_pins reset_125/interconnect_aresetn] \
    [get_bd_pins dma_control_cdc/m_axi_aresetn] \
    [get_bd_pins memory_interconnect/ARESETN] \
    [get_bd_pins memory_interconnect/S00_ARESETN] \
    [get_bd_pins memory_interconnect/S01_ARESETN] \
    [get_bd_pins memory_interconnect/M00_ARESETN]
connect_bd_net [get_bd_pins reset_125/peripheral_aresetn] \
    [get_bd_pins dma/axi_resetn] \
    [get_bd_pins mm2s_cdc/s_axis_aresetn] \
    [get_bd_pins s2mm_cdc/m_axis_aresetn]
connect_bd_net [get_bd_pins reset_200/interconnect_aresetn] \
    [get_bd_pins accelerator_control_cdc/m_axi_aresetn]
connect_bd_net [get_bd_pins reset_200/peripheral_aresetn] \
    [get_bd_pins mm2s_cdc/m_axis_aresetn] \
    [get_bd_pins s2mm_cdc/s_axis_aresetn] \
    [get_bd_pins accelerator/aresetn]

# Accelerator completion and both DMA channels are independently visible to Linux.
connect_bd_net [get_bd_pins accelerator/interrupt] [get_bd_pins interrupt_concat/In0]
connect_bd_net [get_bd_pins dma/mm2s_introut] [get_bd_pins interrupt_concat/In1]
connect_bd_net [get_bd_pins dma/s2mm_introut] [get_bd_pins interrupt_concat/In2]
for {set interrupt_index 3} {$interrupt_index < 16} {incr interrupt_index} {
    connect_bd_net [get_bd_pins interrupt_zero/dout] \
        [get_bd_pins interrupt_concat/In$interrupt_index]
}
connect_bd_net [get_bd_pins interrupt_concat/dout] [get_bd_pins ps7/IRQ_F2P]

assign_bd_address -offset 0x43C00000 -range 0x00010000 \
    -target_address_space [get_bd_addr_spaces ps7/Data] \
    [get_bd_addr_segs accelerator/S_AXI/Reg] -force
assign_bd_address -offset 0x40400000 -range 0x00010000 \
    -target_address_space [get_bd_addr_spaces ps7/Data] \
    [get_bd_addr_segs dma/S_AXI_LITE/Reg] -force
assign_bd_address -offset 0x00000000 -range 0x20000000 \
    -target_address_space [get_bd_addr_spaces dma/Data_MM2S] \
    [get_bd_addr_segs ps7/S_AXI_HP0/HP0_DDR_LOWOCM] -force
assign_bd_address -offset 0x00000000 -range 0x20000000 \
    -target_address_space [get_bd_addr_spaces dma/Data_S2MM] \
    [get_bd_addr_segs ps7/S_AXI_HP0/HP0_DDR_LOWOCM] -force

validate_bd_design
save_bd_design
set bd_file [get_files system.bd]
generate_target all $bd_file
make_wrapper -files $bd_file -top
set wrapper_file [file join $vivado_project_dir "neuraedge_pynq_full.gen" "sources_1" "bd" "system" "hdl" "system_wrapper.v"]
add_files -norecurse $wrapper_file
set_property top system_wrapper [current_fileset]
update_compile_order -fileset sources_1

write_bd_tcl -force [file join $output_dir "generated_system_bd.tcl"]
if {$build_mode eq "validate"} {
    report_ip_status -file [file join $output_dir "validated_ip_status.rpt"]
    puts "NEURAEDGE_PYNQ_BD_VALIDATION_PASSED"
    close_project
    exit 0
}

set synth_run [get_runs synth_1]
set_property STEPS.SYNTH_DESIGN.ARGS.DIRECTIVE PerformanceOptimized $synth_run
set impl_run [get_runs impl_1]
set_property STEPS.OPT_DESIGN.ARGS.DIRECTIVE Explore $impl_run
set_property STEPS.PLACE_DESIGN.ARGS.DIRECTIVE ExtraNetDelay_high $impl_run
set_property STEPS.PHYS_OPT_DESIGN.IS_ENABLED true $impl_run
set_property STEPS.PHYS_OPT_DESIGN.ARGS.DIRECTIVE AggressiveExplore $impl_run
set_property STEPS.ROUTE_DESIGN.ARGS.DIRECTIVE AggressiveExplore $impl_run
set_property STEPS.POST_ROUTE_PHYS_OPT_DESIGN.IS_ENABLED true $impl_run
# Vivado 2026.1 on Windows reproducibly faults in AggressiveExplore's
# post-route hold-fix phase for this design. Explore retains post-route setup
# and hold repair without entering that unstable phase.
set_property STEPS.POST_ROUTE_PHYS_OPT_DESIGN.ARGS.DIRECTIVE Explore $impl_run

launch_runs synth_1 -jobs 2
wait_on_run synth_1
if {[get_property PROGRESS $synth_run] ne "100%"
        || ![string match "*Complete*" [get_property STATUS $synth_run]]} {
    error "Full PYNQ synthesis failed: [get_property STATUS $synth_run]"
}
launch_runs impl_1 -to_step write_bitstream -jobs 2
wait_on_run impl_1
if {[get_property PROGRESS $impl_run] ne "100%"
        || ![string match "*Complete*" [get_property STATUS $impl_run]]} {
    error "Full PYNQ implementation failed: [get_property STATUS $impl_run]"
}
```

```
322
323 open_run impl_1
324 report_utilization -hierarchical -file [file join $output_dir "full_routed_utilization_hierarchical.rpt"]
325 report_timing_summary -delay_type min_max -max_paths 50 \
326     -file [file join $output_dir "full_routed_timing_summary.rpt"]
327 report_timing -delay_type max -max_paths 250 -sort_by group \
328     -file [file join $output_dir "full_routed_timing_paths_250.rpt"]
329 report_clock_interaction -file [file join $output_dir "full_clock_interaction.rpt"]
330 report_cdc -details -file [file join $output_dir "full_cdc.rpt"]
331 set full_route_status_text [report_route_status -return_string]
332 set route_status_file [open [file join $output_dir "full_route_status.rpt"] "w"]
333 puts -nonewline $route_status_file $full_route_status_text
334 close $route_status_file
335 report_drc -file [file join $output_dir "full_drc.rpt"]
336 report_methodology -file [file join $output_dir "full_methodology.rpt"]
337 report_design_analysis -congestion -file [file join $output_dir "full_congestion.rpt"]
338 report_power -file [file join $output_dir "full_vectorless_power.rpt"]
339 write_checkpoint -force [file join $output_dir "neuraedge_pynq_full_routed.dcp"]
340
341 set setup_path [get_timing_paths -delay_type max -max_paths 1]
342 set hold_path [get_timing_paths -delay_type min -max_paths 1]
343 set setup_wns [get_property SLACK $setup_path]
344 set hold_whs [get_property SLACK $hold_path]
345 set route_error_count -1
346 if {[regexp {# of nets with routing errors\.*\s*:\s*([0-9]+)} \
347         $full_route_status_text route_match route_count]} {
348     set route_error_count $route_count
349 }
350 set drc_error_count [llength [get_drc_violations -quiet -filter {SEVERITY == Error}]]
351 set summary_file [open [file join $output_dir "full_implementation_summary.txt"] "w"]
352 puts $summary_file "board_part=$board_part"
353 puts $summary_file "part=xc7z020clg400-1"
354 puts $summary_file "accelerator_clock_mhz=200.0"
355 puts $summary_file "dma_clock_mhz=125.0"
356 puts $summary_file "control_clock_mhz=100.0"
357 puts $summary_file "lut_count=[llength [get_cells -hierarchical -filter {REF_NAME =~ LUT*}]]"
358 puts $summary_file "ff_count=[llength [get_cells -hierarchical -filter {REF_NAME =~ FD*}]]"
359 puts $summary_file "dsp48_count=[llength [get_cells -hierarchical -filter {REF_NAME =~ DSP48*}]]"
360 puts $summary_file "bram36_count=[llength [get_cells -hierarchical -filter {REF_NAME == RAMB36E1}]]"
361 puts $summary_file "bram18_count=[llength [get_cells -hierarchical -filter {REF_NAME == RAMB18E1}]]"
362 puts $summary_file "setup_wns_ns=$setup_wns"
363 puts $summary_file "hold_whs_ns=$hold_whs"
364 puts $summary_file "route_error_count=$route_error_count"
365 puts $summary_file "drc_error_count=$drc_error_count"
366 close $summary_file
367
368 if {$route_error_count != 0} {
369     error "Full PYNQ routing is incomplete: route_error_count=$route_error_count"
370 }
371 if {$drc_error_count != 0} {
372     error "Full PYNQ design has DRC errors: drc_error_count=$drc_error_count"
373 }
374 if {$setup_wns < 0.0} {
375     error "Full PYNQ design missed setup timing: WNS=$setup_wns ns"
376 }
377 if {$hold_whs < 0.0} {
378     error "Full PYNQ design missed hold timing: WHS=$hold_whs ns"
379 }
380
381 set generated_bit [file join $vivado_project_dir \
382     "neuraedge_pynq_full.runs" "impl_1" "system_wrapper.bit"]
383 if {![file exists $generated_bit]} {
384     error "Implementation completed but bitstream was not found at $generated_bit"
385 }
386 file copy -force $generated_bit [file join $output_dir "neuraedge_pynq_full.bit"]
387 set generated_hwh [file join $vivado_project_dir "neuraedge_pynq_full.gen" "sources_1" "bd" "system" "hw_handoff" "system.hwh"]
388 if {![file exists $generated_hwh]} {
389     error "Block design completed but hardware handoff was not found at $generated_hwh"
390 }
391 file copy -force $generated_hwh [file join $output_dir "neuraedge_pynq_full.hwh"]
392 write_hw_platform -fixed -include_bit -force \
393     -file [file join $output_dir "neuraedge_pynq_full.xsa"]
```

90.3 Line annotations

Line 1: `# Build the complete PYNQ-Z2 processing-system, DMA, and NeuraEdge overlay.` This comment documents the batch-flow intent: Build the complete PYNQ-Z2 processing-system, DMA, and NeuraEdge overlay.

Line 2: `# The board preset is vendored from the AMD/Xilinx Board Store and configures` This comment documents the batch-flow intent: The board preset is vendored from the AMD/Xilinx Board Store and configures

Line 3: `# the physical DDR3 and MIO pins. FCLK0 supplies 100 MHz control, FCLK1 supplies` This comment documents the batch-flow intent: the physical DDR3 and MIO pins. FCLK0 supplies 100 MHz control, FCLK1 supplies

Line 4: `# a timing-safe 125 MHz DMA/DDR path, and FCLK2 supplies the 200 MHz accelerator.` This comment documents the batch-flow intent: a timing-safe 125 MHz DMA/DDR path, and FCLK2 supplies the 200 MHz accelerator.

Line 6: `set script_dir [file normalize [file dirname [info script]]]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; script_dir: the value named 'script dir' in this block; file: the value named 'file' in this block.

Line 7: `set project_dir [file normalize [file join $script_dir ".."]]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; project_dir: the value named 'project dir' in this block; file: the value named 'file' in this block.

Line 8: `set board_store [file join $project_dir "third_party" "XilinxBoardStore" "boards"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; board_store: the value named 'board store' in this block; file: the value named 'file' in this block.

Line 9: `set output_dir [file join $project_dir "outputs" "pynq_full"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; output_dir: the value named 'output dir' in this block; file: the value named 'file' in this block.

Line 10: `if {[info exists ::env(NEURAEDGE_VIVADO_BUILD_ROOT)]} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are info: the value named 'info' in this block; exists: the value named 'exists' in this block; env: the value named 'env' in this block.

Line 11: `set build_root [file normalize $::env(NEURAEDGE_VIVADO_BUILD_ROOT)]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; build_root: the value named 'build root' in this block; file: the value named 'file' in this block.

Line 12: `} else {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 13: `set build_root [file normalize "D:/NeuraEdgeBuild"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; build_root: the value named 'build root' in this block; file: the value named 'file' in this block.

Line 14: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 15: `set vivado_project_dir [file join $build_root "pynq_full"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; vivado_project_dir: the value named 'vivado project dir' in this block; file: the value named 'file' in this block.

Line 16: `set build_mode "full"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; build_mode: the value named 'build mode' in this block; full: the value named 'full' in this block.

Line 17: `if {$argc >= 1} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are argc: the value named 'argc' in this block.

Line 18: `set build_mode [lindex $argv 0]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; build_mode: the value named 'build mode' in this block; lindex: an integer selecting one tensor element, lane, bank, or state entry.

Line 19: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 20: `if {$build_mode ni {validate full}} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are build_mode: the value named 'build mode' in this block; ni: the value named 'ni' in this block; validate: the producer's statement that its current payload is meaningful.

Line 21: `error "Build mode must be validate or full"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Build: the value named 'build' in this block; mode: the value named 'mode' in this block.

Line 22: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 23: `file mkdir $output_dir` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; mkdir: the value named 'mkdir' in this block; output_dir: the value named 'output dir' in this block.

Line 24: `if {$build_mode eq "full"} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are build_mode: the value named 'build mode' in this block; eq: the value named 'eq' in this block; full: the value named 'full' in this block.

Line 25: `# A failed rebuild must never leave an older deployable image looking like` This comment documents the batch-flow intent: A failed rebuild must never leave an older deployable image looking like

Line 26: `# the result of the current source tree. Diagnostic reports are overwritten` This comment documents the batch-flow intent: the result of the current source tree. Diagnostic reports are overwritten

Line 27: `# in place, while accepted deployment artifacts are republished only after` This comment documents the batch-flow intent: in place, while accepted deployment artifacts are republished only after

Line 28: `# route, timing, and DRC gates pass below.` This comment documents the batch-flow intent: route, timing, and DRC gates pass below.

Line 29: `foreach stale_artifact {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are foreach: the value named 'foreach' in this block; stale_artifact: the value named 'stale artifact' in this block.

Line 30: `neuraedge_pynq_full.bit` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are neuraedge_pynq_full: the value named 'neuraedge pynq full' in this block; bit: the value named 'bit' in this block.

Line 31: `neuraedge_pynq_full.hwh` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are neuraedge_pynq_full: the value named 'neuraedge pynq full' in this block; hwh: the value named 'hwh' in this block.

Line 32: `neuraedge_pynq_full.xsa` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are neuraedge_pynq_full: the value named 'neuraedge pynq full' in this block; xsa: the value named 'xsa' in this block.

Line 33: `full_implementation_summary.txt` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are full_implementation_summary: the value named 'full implementation summary' in this block; txt: the value named 'txt' in this block.

Line 34: `} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 35: `set stale_path [file join $output_dir $stale_artifact]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; stale_path: the value named 'stale path' in this block; file: the value named 'file' in this block.

Line 36: `if {[file exists $stale_path]} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; exists: the value named 'exists' in this block; stale_path: the value named 'stale path' in this block.

Line 37: `file delete -force $stale_path` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; delete: the value named 'delete' in this block; force: the value named 'force' in this block.

Line 38: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 39: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 40: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 42: `set_param board.repoPaths [list $board_store]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_param: the value named 'set param' in this block; board: the value named 'board' in this block; repoPaths: the value named 'repo paths' in this block.

Line 43: `set board_part "tul.com.tw:pynq-z2:part0:1.0"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; board_part: the value named 'board part' in this block; tul: the value named 'tul' in this block.

Line 44: `if {[llength [get_board_parts -quiet $board_part]] != 1} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are llength: the value named 'llength' in this block; get_board_parts: the value named 'get board parts' in this block; quiet: the value named 'quiet' in this block.

Line 45: `error "Vendored PYNQ-Z2 board part was not discovered at $board_store"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Vendored: the value named 'vendored' in this block; PYNQ: the value named 'pynq' in this block.

Line 46: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 48: `create_project neuraedge_pynq_full $vivado_project_dir -force -part xc7z020clg400-1` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are create_project: the value named 'create project' in this block; neuraedge_pynq_full: the value named 'neuraedge pynq full' in this block; vivado_project_dir: the value named 'vivado project dir' in this block.

Line 49: `set_property board_part $board_part [current_project]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; board_part: the value named 'board part' in this block; current_project: the value named 'current project' in this block.

Line 50: `set_property target_language Verilog [current_project]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; target_language: the value named 'target language' in this block; Verilog: the value named 'verilog' in this block.

Line 51: `set_property simulator_language Mixed [current_project]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; simulator_language: the value named 'simulator language' in this block; Mixed: the value named 'mixed' in this block.

Line 53: `set rtl_files [list \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; rtl_files: the value named 'rtl files' in this block; list: the value named 'list' in this block.

Line 54: `[file join $project_dir "rtl" "neuraedge_command_pkg.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 55: `[file join $project_dir "rtl" "neuraedge_mac2_u8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 56: `[file join $project_dir "rtl" "neuraedge_tensor_tile.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 57: `[file join $project_dir "rtl" "neuraedge_active_index_buffer.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 58: `[file join $project_dir "rtl" "neuraedge_activation_buffer.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 59: `[file join $project_dir "rtl" "neuraedge_weight_bank.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 60: `[file join $project_dir "rtl" "neuraedge_buffered_tensor_core.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 61: `[file join $project_dir "rtl" "neuraedge_alpha_xnor.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 62: `[file join $project_dir "rtl" "neuraedge_vector_alu.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 63: `[file join $project_dir "rtl" "neuraedge_udivider.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 64: `[file join $project_dir "rtl" "neuraedge_isqrt64.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 65: `[file join $project_dir "rtl" "neuraedge_integer_layernorm.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 66: `[file join $project_dir "rtl" "neuraedge_log_softmax.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 67: `[file join $project_dir "rtl" "neuraedge_lif_bank.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 68: `[file join $project_dir "rtl" "neuraedge_sgd_optimizer8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 69: `[file join $project_dir "rtl" "neuraedge_gradient_accumulator_bank.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 70: `[file join $project_dir "rtl" "neuraedge_linear_backward8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 71: `[file join $project_dir "rtl" "neuraedge_activation_lut8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 72: `[file join $project_dir "rtl" "neuraedge_universal_core.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 73: `[file join $project_dir "rtl" "neuraedge_descriptor_sequencer.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 74: `[file join $project_dir "rtl" "neuraedge_pynq_axi_top.sv"]]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 75: `add_files -norecurse $rtl_files` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are add_files: the value named 'add files' in this block; norecurse: the value named 'norecurse' in this block; rtl_files: the value named 'rtl files' in this block.

Line 76: `set_property file_type SystemVerilog [get_files $rtl_files]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; file_type: the value named 'file type' in this block; SystemVerilog: the value named 'system verilog' in this block.

Line 77: `set_property top neuraedge_pynq_axi_top [current_fileset]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; top: the value named 'top' in this block; neuraedge_pynq_axi_top: the value named 'neuraedge pynq axi top' in this block.

Line 78: `update_compile_order -fileset sources_1` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are update_compile_order: the value named 'update compile order' in this block; fileset: the value named 'fileset' in this block; sources_1: the value named 'sources 1' in this block.

Line 80: `# Package the verified RTL with an explicit IP-XACT memory map. A raw module` This comment documents the batch-flow intent: Package the verified RTL with an explicit IP-XACT memory map. A raw module

Line 81: `# reference exposes AXI pins but has no address block, which prevents Vivado's` This comment documents the batch-flow intent: reference exposes AXI pins but has no address block, which prevents Vivado's

Line 82: `# Address Editor and PYNQ's HWH parser from describing the control registers.` This comment documents the batch-flow intent: Address Editor and PYNQ's HWH parser from describing the control registers.

Line 83: `set ip_repo [file join $output_dir "ip_repo" "neuraedge_universal_1_0"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; ip_repo: the value named 'ip repo' in this block; file: the value named 'file' in this block.

Line 84: `file mkdir $ip_repo` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; mkdir: the value named 'mkdir' in this block; ip_repo: the value named 'ip repo' in this block.

Line 85: `ipx::package_project -root_dir $ip_repo -vendor neuraedge.local \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ipx: the value named 'ipx' in this block; package_project: the value named 'package project' in this block; root_dir: the value named 'root dir' in this block.

Line 86: `-library accelerator -taxonomy /UserIP -import_files -set_current true` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are library: the value named 'library' in this block; accelerator: the value named 'accelerator' in this block; taxonomy: the value named 'taxonomy' in this block.

Line 87: `set packaged_core [ipx::current_core]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; packaged_core: the value named 'packaged core' in this block; ipx: the value named 'ipx' in this block.

Line 88: `set_property name neuraedge_universal $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; name: the value named 'name' in this block; neuraedge_universal: the value named 'neuraedge universal' in this block.

Line 89: `set_property display_name "NeuraEdge Universal Accelerator" $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; display_name: the value named 'display name' in this block; NeuraEdge: the value named 'neura edge' in this block.

Line 90: `set_property description "INT8, sparse, spiking, transformer, and on-fabric optimizer accelerator" $package...` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; description: the value named 'description' in this block; INT8: the value named 'int8' in this block.

Line 91: `set_property version 1.0 $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; version: the value named 'version' in this block; packaged_core: the value named 'packaged core' in this block.

Line 92: `set_property core_revision 1 $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; core_revision: the value named 'core revision' in this block; packaged_core: the value named 'packaged core' in this block.

Line 93: `set s_axi_interface [ipx::get_bus_interfaces S_AXI -of_objects $packaged_core]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; s_axi_interface: the value named 's axi interface' in this block; ipx: the value named 'ipx' in this block.

Line 94: `ipx::add_memory_map S_AXI $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ipx: the value named 'ipx' in this block; add_memory_map: local FPGA storage addressed by surrounding index logic; S_AXI: the value named 's axi' in this block.

Line 95: `set_property slave_memory_map_ref S_AXI $s_axi_interface` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; slave_memory_map_ref: local FPGA storage addressed by surrounding index logic; S_AXI: the value named 's axi' in this block.

Line 96: `set memory_map [ipx::get_memory_maps S_AXI -of_objects $packaged_core]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; memory_map: local FPGA storage addressed by surrounding index logic; ipx: the value named 'ipx' in this block.

Line 97: `ipx::add_address_block Reg $memory_map` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ipx: the value named 'ipx' in this block; add_address_block: the value named 'add address block' in this block; Reg: the value named 'reg' in this block.

Line 98: `set register_block [ipx::get_address_blocks Reg -of_objects $memory_map]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; register_block: the value named 'register block' in this block; ipx: the value named 'ipx' in this block.

Line 99: `set_property range 256 $register_block` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; range: the value named 'range' in this block; register_block: the value named 'register block' in this block.

Line 100: `set_property width 32 $register_block` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; width: the value named 'width' in this block; register_block: the value named 'register block' in this block.

Line 101: `ipx::associate_bus_interfaces -busif S_AXI -clock aclk $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ipx: the value named 'ipx' in this block; associate_bus_interfaces: the value named 'associate bus interfaces' in this block; busif: the value named 'busif' in this block.

Line 102: `ipx::associate_bus_interfaces -busif S_AXIS -clock aclk $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ipx: the value named 'ipx' in this block; associate_bus_interfaces: the value named 'associate bus interfaces' in this block; busif: the value named 'busif' in this block.

Line 103: `ipx::associate_bus_interfaces -busif M_AXIS -clock aclk $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ipx: the value named 'ipx' in this block; associate_bus_interfaces: the value named 'associate bus interfaces' in this block; busif: the value named 'busif' in this block.

Line 104: `ipx::create_xgui_files $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ipx: the value named 'ipx' in this block; create_xgui_files: the value named 'create xgui files' in this block; packaged_core: the value named 'packaged core' in this block.

Line 105: `ipx::update_checksums $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ipx: the value named 'ipx' in this block; update_checksums: the value named 'update checksums' in this block; packaged_core: the value named 'packaged core' in this block.

Line 106: `ipx::save_core $packaged_core` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ipx: the value named 'ipx' in this block; save_core: the value named 'save core' in this block; packaged_core: the value named 'packaged core' in this block.

Line 107: `set_property ip_repo_paths [list [file dirname $ip_repo]] [current_project]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; ip_repo_paths: the value named 'ip repo paths' in this block; list: the value named 'list' in this block.

Line 108: `update_ip_catalog` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are update_ip_catalog: the value named 'update ip catalog' in this block.

Line 110: `create_bd_design system` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are create_bd_design: the value named 'create bd design' in this block; system: the value named 'system' in this block.

Line 112: `set ps7 [create_bd_cell -type ip -vlnv xilinx.com:ip:processing_system7:5.5 ps7]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; ps7: the value named 'ps7' in this block; create_bd_cell: the value named 'create bd cell' in this block.

Line 113: `apply_bd_automation -rule xilinx.com:bd_rule:processing_system7 \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are apply_bd_automation: the value named 'apply bd automation' in this block; rule: the value named 'rule' in this block; xilinx: the value named 'xilinx' in this block.

Line 114: `-config {apply_board_preset "1" make_external "FIXED_IO, DDR" Master "Disable" Slave "Disable"} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are config: the value named 'config' in this block; apply_board_preset: the value named 'apply board preset' in this block; make_external: the value named 'make external' in this block.

Line 115: `$ps7` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are ps7: the value named 'ps7' in this block.

Line 116: `set_property -dict [list \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; dict: the value named 'dict' in this block; list: the value named 'list' in this block.

Line 117: `CONFIG.PCW_USE_M_AXI_GP0 {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_USE_M_AXI_GP0: the value named 'pcw use m axi gp0' in this block.

Line 118: `CONFIG.PCW_USE_S_AXI_HP0 {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_USE_S_AXI_HP0: the value named 'pcw use s axi hp0' in this block.

Line 119: `CONFIG.PCW_S_AXI_HP0_DATA_WIDTH {64} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_S_AXI_HP0_DATA_WIDTH: the value named 'pcw s axi hp0 data width' in this block.

Line 120: `CONFIG.PCW_S_AXI_HP0_ID_WIDTH {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_S_AXI_HP0_ID_WIDTH: the value named 'pcw s axi hp0 id width' in this block.

Line 121: `CONFIG.PCW_USE_FABRIC_INTERRUPT {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_USE_FABRIC_INTERRUPT: the hardware notification that wakes software after completion or error.

Line 122: `CONFIG.PCW_IRQ_F2P_INTR {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_IRQ_F2P_INTR: the value named 'pcw irq f2 p intr' in this block.

Line 123: `CONFIG.PCW_EN_CLK0_PORT {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_EN_CLK0_PORT: the clock that makes all registers advance together.

Line 124: `CONFIG.PCW_EN_CLK1_PORT {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_EN_CLK1_PORT: the clock that makes all registers advance together.

Line 125: `CONFIG.PCW_EN_CLK2_PORT {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_EN_CLK2_PORT: the clock that makes all registers advance together.

Line 126: `CONFIG.PCW_FPGA0_PERIPHERAL_FREQMHZ {100.000000} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_FPGA0_PERIPHERAL_FREQMHZ: the value named 'pcw fpga0 peripheral freqmhz' in this block.

Line 127: `CONFIG.PCW_FPGA1_PERIPHERAL_FREQMHZ {125.000000} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_FPGA1_PERIPHERAL_FREQMHZ: the value named 'pcw fpga1 peripheral freqmhz' in this block.

Line 128: `CONFIG.PCW_FPGA2_PERIPHERAL_FREQMHZ {200.000000}] $ps7` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; PCW_FPGA2_PERIPHERAL_FREQMHZ: the value named 'pcw fpga2 peripheral freqmhz' in this block; ps7: the value named 'ps7' in this block.

Line 129: `# The board preset assigns 10 MHz to this disabled-by-default interface. Once` This comment documents the batch-flow intent: The board preset assigns 10 MHz to this disabled-by-default interface. Once

Line 130: `# GP0 is enabled, let block-design propagation derive its frequency from the` This comment documents the batch-flow intent: GP0 is enabled, let block-design propagation derive its frequency from the

Line 131: `# connected 100 MHz FCLK0 instead of retaining that stale user-strength value.` This comment documents the batch-flow intent: connected 100 MHz FCLK0 instead of retaining that stale user-strength value.

Line 132: `set_property CONFIG.PCW_M_AXI_GP0_FREQMHZ.VALUE_SRC auto $ps7` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; CONFIG: the value named 'config' in this block; PCW_M_AXI_GP0_FREQMHZ: the value named 'pcw m axi gp0 freqmhz' in this block.

Line 134: `set reset_100 [create_bd_cell -type ip -vlnv xilinx.com:ip:proc_sys_reset:5.0 reset_100]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; reset_100: the value named 'reset 100' in this block; create_bd_cell: the value named 'create bd cell' in this block.

Line 135: `set reset_125 [create_bd_cell -type ip -vlnv xilinx.com:ip:proc_sys_reset:5.0 reset_125]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; reset_125: the value named 'reset 125' in this block; create_bd_cell: the value named 'create bd cell' in this block.

Line 136: `set reset_200 [create_bd_cell -type ip -vlnv xilinx.com:ip:proc_sys_reset:5.0 reset_200]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; reset_200: the value named 'reset 200' in this block; create_bd_cell: the value named 'create bd cell' in this block.

Line 137: `set accelerator_control_cdc [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_clock_converter:2.1 accelerato...` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; accelerator_control_cdc: the value named 'accelerator control cdc' in this block; create_bd_cell: the value named 'create bd cell' in this block.

Line 138: `set dma_control_cdc [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_clock_converter:2.1 dma_contro` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; dma_control_cdc: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; create_bd_cell: the value named 'create bd cell' in this block.

Line 139: `set control_interconnect [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_interconnect:2.1 control_intercon...` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; control_interconnect: the value named 'control interconnect' in this block; create_bd_cell: the value named 'create bd cell' in this block.

Line 140: `set_property -dict [list CONFIG.NUM_SI {1} CONFIG.NUM_MI {2}] $control_interconnect` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; dict: the value named 'dict' in this block; list: the value named 'list' in this block.

Line 142: `set dma [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_dma:7.1 dma]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; create_bd_cell: the value named 'create bd cell' in this block.

Line 143: `set_property -dict [list \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; dict: the value named 'dict' in this block; list: the value named 'list' in this block.

Line 144: `CONFIG.c_include_sg {0} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_include_sg: the value named 'c include sg' in this block.

Line 145: `CONFIG.c_sg_length_width {26} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_sg_length_width: the value named 'c sg length width' in this block.

Line 146: `CONFIG.c_include_mm2s {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_include_mm2s: the value named 'c include mm2s' in this block.

Line 147: `CONFIG.c_include_s2mm {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_include_s2mm: the value named 'c include s2mm' in this block.

Line 148: `CONFIG.c_include_mm2s_dre {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_include_mm2s_dre: the value named 'c include mm2s dre' in this block.

Line 149: `CONFIG.c_include_s2mm_dre {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_include_s2mm_dre: the value named 'c include s2mm dre' in this block.

Line 150: `CONFIG.c_m_axi_mm2s_data_width {64} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_m_axi_mm2s_data_width: the value named 'c m axi mm2s data width' in this block.

Line 151: `CONFIG.c_m_axi_s2mm_data_width {64} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_m_axi_s2mm_data_width: the value named 'c m axi s2mm data width' in this block.

Line 152: `CONFIG.c_m_axis_mm2s_tdata_width {64} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_m_axis_mm2s_tdata_width: the value named 'c m axis mm2s tdata width' in this block.

Line 153: `CONFIG.c_s_axis_s2mm_tdata_width {64} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_s_axis_s2mm_tdata_width: the value named 'c s axis s2mm tdata width' in this block.

Line 154: `CONFIG.c_mm2s_burst_size {256} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_mm2s_burst_size: the value named 'c mm2s burst size' in this block.

Line 155: `CONFIG.c_s2mm_burst_size {256}] $dma` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; c_s2mm_burst_size: the value named 'c s2mm burst size' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic.

Line 157: `set memory_interconnect [create_bd_cell -type ip -vlnv xilinx.com:ip:axi_interconnect:2.1 memory_interconnect]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; memory_interconnect: local FPGA storage addressed by surrounding index logic; create_bd_cell: the value named 'create bd cell' in this block.

Line 158: `set_property -dict [list CONFIG.NUM_SI {2} CONFIG.NUM_MI {1}] $memory_interconnect` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; dict: the value named 'dict' in this block; list: the value named 'list' in this block.

Line 160: `set mm2s_cdc [create_bd_cell -type ip -vlnv xilinx.com:ip:axis_clock_converter:1.1 mm2s_cdc]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; mm2s_cdc: the value named 'mm2s cdc' in this block; create_bd_cell: the value named 'create bd cell' in this block.

Line 161: `set s2mm_cdc [create_bd_cell -type ip -vlnv xilinx.com:ip:axis_clock_converter:1.1 s2mm_cdc]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; s2mm_cdc: the value named 's2mm cdc' in this block; create_bd_cell: the value named 'create bd cell' in this block.

Line 162: `foreach stream_cdc [list $mm2s_cdc $s2mm_cdc] {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are foreach: the value named 'foreach' in this block; stream_cdc: the value named 'stream cdc' in this block; list: the value named 'list' in this block.

Line 163: `set_property -dict [list \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; dict: the value named 'dict' in this block; list: the value named 'list' in this block.

Line 164: `CONFIG.TDATA_NUM_BYTES {8} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; TDATA_NUM_BYTES: the value named 'tdata num bytes' in this block.

Line 165: `CONFIG.HAS_TKEEP {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; HAS_TKEEP: the value named 'has tkeep' in this block.

Line 166: `CONFIG.HAS_TLAST {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; HAS_TLAST: the AXI-Stream marker for the final beat of a packet or command payload.

Line 167: `CONFIG.IS_ACLK_ASYNC {1} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; IS_ACLK_ASYNC: the clock that makes all registers advance together.

Line 168: `CONFIG.SYNCHRONIZATION_STAGES {3}] $stream_cdc` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are CONFIG: the value named 'config' in this block; SYNCHRONIZATION_STAGES: the value named 'synchronization stages' in this block; stream_cdc: the value named 'stream cdc' in this block.

Line 169: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 171: `set accelerator [create_bd_cell -type ip \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; accelerator: the value named 'accelerator' in this block; create_bd_cell: the value named 'create bd cell' in this block.

Line 172: `-vlnv neuraedge.local:accelerator:neuraedge_universal:1.0 accelerator]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are vlnv: the value named 'vlnv' in this block; neuraedge: the value named 'neuraedge' in this block; local: the value named 'local' in this block.

Line 173: `set interrupt_concat [create_bd_cell -type inline_hdl \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; interrupt_concat: the hardware notification that wakes software after completion or error; create_bd_cell: the value named 'create bd cell' in this block.

Line 174: `-vlnv xilinx.com:inline_hdl:ilconcat:1.0 interrupt_concat]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are vlnv: the value named 'vlnv' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 175: `set_property CONFIG.NUM_PORTS {16} $interrupt_concat` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; CONFIG: the value named 'config' in this block; NUM_PORTS: the value named 'num ports' in this block.

Line 176: `set interrupt_zero [create_bd_cell -type inline_hdl \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; interrupt_zero: the hardware notification that wakes software after completion or error; create_bd_cell: the value named 'create bd cell' in this block.

Line 177: `-vlnv xilinx.com:inline_hdl:ilconstant:1.0 interrupt_zero]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are vlnv: the value named 'vlnv' in this block; xilinx: the value named 'xilinx' in this block; com: the value named 'com' in this block.

Line 178: `set_property -dict [list CONFIG.CONST_WIDTH {1} CONFIG.CONST_VAL {0}] $interrupt_zero` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; dict: the value named 'dict' in this block; list: the value named 'list' in this block.

Line 180: `# Control path: the 100 MHz GP0/interconnect fans out through explicit CDCs to` This comment documents the batch-flow intent: Control path: the 100 MHz GP0/interconnect fans out through explicit CDCs to

Line 181: `# the 200 MHz accelerator and the 125 MHz DMA register interface.` This comment documents the batch-flow intent: the 200 MHz accelerator and the 125 MHz DMA register interface.

Line 182: `connect_bd_intf_net [get_bd_intf_pins ps7/M_AXI_GP0] [get_bd_intf_pins control_interconnect/S00_AXI`
This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins ps7/M_AXI_GP0] [get_bd_intf_pins control_interconnect/S00_AXI]'.

Line 183: `connect_bd_intf_net [get_bd_intf_pins control_interconnect/M00_AXI] [get_bd_intf_pins accelerator_c`
This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins control_interconnect/M00_AXI] [get_bd_intf_pins accelerator_control_cdc/S_AXI]'.

Line 184: `connect_bd_intf_net [get_bd_intf_pins accelerator_control_cdc/M_AXI] [get_bd_intf_pins accelerator/`
This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins accelerator_control_cdc/M_AXI] [get_bd_intf_pins accelerator/S_AXI]'.

Line 185: `connect_bd_intf_net [get_bd_intf_pins control_interconnect/M01_AXI] [get_bd_intf_pins dma_control_c`
This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins control_interconnect/M01_AXI] [get_bd_intf_pins dma_control_cdc/S_AXI]'.

Line 186: `connect_bd_intf_net [get_bd_intf_pins dma_control_cdc/M_AXI] [get_bd_intf_pins dma/S_AXI_LITE]`
This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins dma_control_cdc/M_AX
[get_bd_intf_pins dma/S_AXI_LITE]'.

Line 188: `# Streaming path: 125 MHz DDR/DMA -> asynchronous stream CDC -> 200 MHz` This comment documents the batch-flow intent: Streaming path: 125 MHz DDR/DMA -> asynchronous stream CDC -> 200 MHz

Line 189: `# accelerator -> asynchronous stream CDC -> 125 MHz DMA/DDR.` This comment documents the batch-flow intent: accelerator -> asynchronous stream CDC -> 125 MHz DMA/DDR.

Line 190: `connect_bd_intf_net [get_bd_intf_pins dma/M_AXIS_MM2S] [get_bd_intf_pins mm2s_cdc/S_AXIS]` This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins dma/M_AXIS_MM2S] [get_bd_intf_pins mm2s_cdc/S_AXIS]'.

Line 191: `connect_bd_intf_net [get_bd_intf_pins mm2s_cdc/M_AXIS] [get_bd_intf_pins accelerator/S_AXIS]` This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins mm2s_cdc/M_AXIS] [get_bd_intf_pins accelerator/S_AXIS]'.

Line 192: `connect_bd_intf_net [get_bd_intf_pins accelerator/M_AXIS] [get_bd_intf_pins s2mm_cdc/S_AXIS]` This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins accelerator/M_AXIS] [get_bd_intf_pins s2mm_cdc/S_AXIS]'.

Line 193: `connect_bd_intf_net [get_bd_intf_pins s2mm_cdc/M_AXIS] [get_bd_intf_pins dma/S_AXIS_S2MM]` This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins s2mm_cdc/M_AXIS] [get_bd_intf_pins dma/S_AXIS_S2MM]'.

Line 194: `connect_bd_intf_net [get_bd_intf_pins dma/M_AXI_MM2S] [get_bd_intf_pins memory_interconnect/S00_AXI` This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins dma/M_AXI_MM2S] [get_bd_intf_pins memory_interconnect/S00_AXI]'.

Line 195: `connect_bd_intf_net [get_bd_intf_pins dma/M_AXI_S2MM] [get_bd_intf_pins memory_interconnect/S01_AXI` This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins dma/M_AXI_S2MM] [get_bd_intf_pins memory_interconnect/S01_AXI]'.

Line 196: `connect_bd_intf_net [get_bd_intf_pins memory_interconnect/M00_AXI] [get_bd_intf_pins ps7/S_AXI_HP0]` This connects two complete block-design interfaces, carrying all AXI channel signals defined by that interface. Vivado validates widths, directions, clocks, and protocol compatibility for 'connect_bd_intf_net [get_bd_intf_pins memory_interconnect/M00_AXI] [get_bd_intf_pins ps7/S_AXI_HP0]'.

Line 198: `# Clock and reset domains are explicit; there is no unconstrained interface CDC.` This comment documents the batch-flow intent: Clock and reset domains are explicit; there is no unconstrained interface CDC.

Line 199: `connect_bd_net [get_bd_pins ps7/FCLK_CLK0] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins ps7/FCLK_CLK0] \'.

Line 200: `[get_bd_pins ps7/M_AXI_GP0_ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; ps7: the value named 'ps7' in this block; M_AXI_GP0_ACLK: the clock that makes all registers advance together.

Line 201: `[get_bd_pins reset_100/slowest_sync_clk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; reset_100: the value named 'reset 100' in this block; slowest_sync_clk: the clock that makes all registers advance together.

Line 202: `[get_bd_pins control_interconnect/ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; control_interconnect: the value named 'control interconnect' in this block; ACLK: the AXI and accelerator clock.

Line 203: `[get_bd_pins control_interconnect/S00_ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; control_interconnect: the value named 'control interconnect' in this block; S00_ACLK: the clock that makes all registers advance together.

Line 204: `[get_bd_pins control_interconnect/M00_ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; control_interconnect: the value named 'control interconnect' in this block; M00_ACLK: the clock that makes all registers advance together.

Line 205: `[get_bd_pins control_interconnect/M01_ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; control_interconnect: the value named 'control interconnect' in this block; M01_ACLK: the clock that makes all registers advance together.

Line 206: `[get_bd_pins accelerator_control_cdc/s_axi_aclk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; accelerator_control_cdc: the value named 'accelerator control cdc' in this block; s_axi_aclk: the clock that makes all registers advance together.

Line 207: `[get_bd_pins dma_control_cdc/s_axi_aclk]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; dma_control_cdc: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; s_axi_aclk: the clock that makes all registers advance together.

Line 208: `connect_bd_net [get_bd_pins ps7/FCLK_CLK1] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: ‘connect_bd_net [get_bd_pins ps7/FCLK_CLK1] \‘.

Line 209: `[get_bd_pins ps7/S_AXI_HP0_ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; ps7: the value named 'ps7' in this block; S_AXI_HP0_ACLK: the clock that makes all registers advance together.

Line 210: `[get_bd_pins reset_125/slowest_sync_clk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; reset_125: the value named 'reset 125' in this block; slowest_sync_clk: the clock that makes all registers advance together.

Line 211: `[get_bd_pins memory_interconnect/ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; memory_interconnect: local FPGA storage addressed by surrounding index logic; ACLK: the AXI and accelerator clock.

Line 212: `[get_bd_pins memory_interconnect/S00_ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; memory_interconnect: local FPGA storage addressed by surrounding index logic; S00_ACLK: the clock that makes all registers advance together.

Line 213: `[get_bd_pins memory_interconnect/S01_ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; memory_interconnect: local FPGA storage addressed by surrounding index logic; S01_ACLK: the clock that makes all registers advance together.

Line 214: `[get_bd_pins memory_interconnect/M00_ACLK] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; memory_interconnect: local FPGA storage addressed by surrounding index logic; M00_ACLK: the clock that makes all registers advance together.

Line 215: `[get_bd_pins dma/s_axi_lite_aclk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; s_axi_lite_aclk: the clock that makes all registers advance together.

Line 216: `[get_bd_pins dma/m_axi_mm2s_aclk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; m_axi_mm2s_aclk: the clock that makes all registers advance together.

Line 217: `[get_bd_pins dma/m_axi_s2mm_aclk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; m_axi_s2mm_aclk: the clock that makes all registers advance together.

Line 218: `[get_bd_pins dma_control_cdc/m_axi_aclk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; dma_control_cdc: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; m_axi_aclk: the clock that makes all registers advance together.

Line 219: `[get_bd_pins mm2s_cdc/s_axis_aclk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; mm2s_cdc: the value named 'mm2s cdc' in this block; s_axis_aclk: the clock that makes all registers advance together.

Line 220: `[get_bd_pins s2mm_cdc/m_axis_aclk]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; s2mm_cdc: the value named 's2mm cdc' in this block; m_axis_aclk: the clock that makes all registers advance together.

Line 221: `connect_bd_net [get_bd_pins ps7/FCLK_CLK2] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins ps7/FCLK_CLK2] \'.

Line 222: `[get_bd_pins reset_200/slowest_sync_clk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; reset_200: the value named 'reset 200' in this block; slowest_sync_clk: the clock that makes all registers advance together.

Line 223: `[get_bd_pins accelerator_control_cdc/m_axi_aclk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; accelerator_control_cdc: the value named 'accelerator control cdc' in this block; m_axi_aclk: the clock that makes all registers advance together.

Line 224: `[get_bd_pins mm2s_cdc/m_axis_aclk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; mm2s_cdc: the value named 'mm2s cdc' in this block; m_axis_aclk: the clock that makes all registers advance together.

Line 225: `[get_bd_pins s2mm_cdc/s_axis_aclk] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; s2mm_cdc: the value named 's2mm cdc' in this block; s_axis_aclk: the clock that makes all registers advance together.

Line 226: `[get_bd_pins accelerator/aclk]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; accelerator: the value named 'accelerator' in this block; aclk: the AXI and accelerator clock.

Line 227: `connect_bd_net [get_bd_pins ps7/FCLK_RESET0_N] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins ps7/FCLK_RESET0_N] \'.

Line 228: `[get_bd_pins reset_100/ext_reset_in] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; reset_100: the value named 'reset 100' in this block; ext_reset_in: the value named 'ext reset in' in this block.

Line 229: `[get_bd_pins reset_125/ext_reset_in] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; reset_125: the value named 'reset 125' in this block; ext_reset_in: the value named 'ext reset in' in this block.

Line 230: `[get_bd_pins reset_200/ext_reset_in]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; reset_200: the value named 'reset 200' in this block; ext_reset_in: the value named 'ext reset in' in this block.

Line 231: `connect_bd_net [get_bd_pins reset_100/interconnect_aresetn] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins reset_100/interconnect_aresetn] \'.

Line 232: `[get_bd_pins control_interconnect/ARESETN] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; control_interconnect: the value named 'control interconnect' in this block; ARESETN: the active-low AXI reset.

Line 233: `[get_bd_pins control_interconnect/S00_ARESETN] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; control_interconnect: the value named 'control interconnect' in this block; S00_ARESETN: the active-low AXI reset.

Line 234: `[get_bd_pins control_interconnect/M00_ARESETN] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; control_interconnect: the value named 'control interconnect' in this block; M00_ARESETN: the active-low AXI reset.

Line 235: `[get_bd_pins control_interconnect/M01_ARESETN] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; control_interconnect: the value named 'control interconnect' in this block; M01_ARESETN: the active-low AXI reset.

Line 236: `[get_bd_pins accelerator_control_cdc/s_axi_aresetn] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; accelerator_control_cdc: the value named 'accelerator control cdc' in this block; s_axi_aresetn: the active-low AXI reset.

Line 237: `[get_bd_pins dma_control_cdc/s_axi_aresetn]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; dma_control_cdc: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; s_axi_aresetn: the active-low AXI reset.

Line 238: `connect_bd_net [get_bd_pins reset_125/interconnect_aresetn] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins reset_125/interconnect_aresetn] \'.

Line 239: `[get_bd_pins dma_control_cdc/m_axi_aresetn] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; dma_control_cdc: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; m_axi_aresetn: the active-low AXI reset.

Line 240: `[get_bd_pins memory_interconnect/ARESETN] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; memory_interconnect: local FPGA storage addressed by surrounding index logic; ARESETN: the active-low AXI reset.

Line 241: `[get_bd_pins memory_interconnect/S00_ARESETN] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; memory_interconnect: local FPGA storage addressed by surrounding index logic; S00_ARESETN: the active-low AXI reset.

Line 242: `[get_bd_pins memory_interconnect/S01_ARESETN] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; memory_interconnect: local FPGA storage addressed by surrounding index logic; S01_ARESETN: the active-low AXI reset.

Line 243: `[get_bd_pins memory_interconnect/M00_ARESETN]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; memory_interconnect: local FPGA storage addressed by surrounding index logic; M00_ARESETN: the active-low AXI reset.

Line 244: `connect_bd_net [get_bd_pins reset_125/peripheral_aresetn] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins reset_125/peripheral_aresetn] \'.

Line 245: `[get_bd_pins dma/axi_resetn] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; axi_resetn: the value named 'axi resetn' in this block.

Line 246: `[get_bd_pins mm2s_cdc/s_axis_aresetn] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; mm2s_cdc: the value named 'mm2s cdc' in this block; s_axis_aresetn: the active-low AXI reset.

Line 247: `[get_bd_pins s2mm_cdc/m_axis_aresetn]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; s2mm_cdc: the value named 's2mm cdc' in this block; m_axis_aresetn: the active-low AXI reset.

Line 248: `connect_bd_net [get_bd_pins reset_200/interconnect_aresetn] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins reset_200/interconnect_aresetn] \'.

Line 249: `[get_bd_pins accelerator_control_cdc/m_axi_aresetn]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; accelerator_control_cdc: the value named 'accelerator control cdc' in this block; m_axi_aresetn: the active-low AXI reset.

Line 250: `connect_bd_net [get_bd_pins reset_200/peripheral_aresetn] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins reset_200/peripheral_aresetn] \'.

Line 251: `[get_bd_pins mm2s_cdc/m_axis_aresetn] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; mm2s_cdc: the value named 'mm2s cdc' in this block; m_axis_aresetn: the active-low AXI reset.

Line 252: `[get_bd_pins s2mm_cdc/s_axis_aresetn] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; s2mm_cdc: the value named 's2mm cdc' in this block; s_axis_aresetn: the active-low AXI reset.

Line 253: `[get_bd_pins accelerator/aresetn]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; accelerator: the value named 'accelerator' in this block; aresetn: the active-low AXI reset.

Line 255: `# Accelerator completion and both DMA channels are independently visible to Linux.` This comment documents the batch-flow intent: Accelerator completion and both DMA channels are independently visible to Linux.

Line 256: `connect_bd_net [get_bd_pins accelerator/interrupt] [get_bd_pins interrupt_concat/In0]` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins accelerator/interrupt] [get_bd_pins interrupt_concat/In0]'.

Line 257: `connect_bd_net [get_bd_pins dma/mm2s_introut] [get_bd_pins interrupt_concat/In1]` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins dma/mm2s_introut] [get_bd_pins interrupt_concat/In1]'.

Line 258: `connect_bd_net [get_bd_pins dma/s2mm_introut] [get_bd_pins interrupt_concat/In2]` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins dma/s2mm_introut] [get_bd_pins interrupt_concat/In2]'.

Line 259: `for {set interrupt_index 3} {$interrupt_index < 16} {incr interrupt_index} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are for: the value named 'for' in this block; set: the value named 'set' in this block; interrupt_index: the hardware notification that wakes software after completion or error.

Line 260: `connect_bd_net [get_bd_pins interrupt_zero/dout] \` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins interrupt_zero/dout] \'.

Line 261: `[get_bd_pins interrupt_concat/In$interrupt_index]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_pins: the value named 'get bd pins' in this block; interrupt_concat: the hardware notification that wakes software after completion or error; In$interrupt_index: the hardware notification that wakes software after completion or error.

Line 262: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 263: `connect_bd_net [get_bd_pins interrupt_concat/dout] [get_bd_pins ps7/IRQ_F2P]` This connects scalar or bus pins in the block design. For clocks and resets, this exact connection decides the timing domain and reset island of every attached register: 'connect_bd_net [get_bd_pins interrupt_concat/dout] [get_bd_pins ps7/IRQ_F2P]'.

Line 265: `assign_bd_address -offset 0x43C00000 -range 0x00010000 \` This asks Vivado to establish an AXI address map; following explicit offset/range settings make the software-visible register addresses stable across rebuilds.

Line 266: `-target_address_space [get_bd_addr_spaces ps7/Data] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are target_address_space: the value named 'target address space' in this block; get_bd_addr_spaces: the value named 'get bd addr spaces' in this block; ps7: the value named 'ps7' in this block.

Line 267: `[get_bd_addr_segs accelerator/S_AXI/Reg] -force` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_addr_segs: the value named 'get bd addr segs' in this block; accelerator: the value named 'accelerator' in this block; S_AXI: the value named 's axi' in this block.

Line 268: `assign_bd_address -offset 0x40400000 -range 0x00010000 \` This asks Vivado to establish an AXI address map; following explicit offset/range settings make the software-visible register addresses stable across rebuilds.

Line 269: `-target_address_space [get_bd_addr_spaces ps7/Data] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are target_address_space: the value named 'target address space' in this block; get_bd_addr_spaces: the value named 'get bd addr spaces' in this block; ps7: the value named 'ps7' in this block.

Line 270: `[get_bd_addr_segs dma/S_AXI_LITE/Reg] -force` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_addr_segs: the value named 'get bd addr segs' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic; S_AXI_LITE: the value named 's axi lite' in this block.

Line 271: `assign_bd_address -offset 0x00000000 -range 0x20000000 \` This asks Vivado to establish an AXI address map; following explicit offset/range settings make the software-visible register addresses stable across rebuilds.

Line 272: `-target_address_space [get_bd_addr_spaces dma/Data_MM2S] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are target_address_space: the value named 'target address space' in this block; get_bd_addr_spaces: the value named 'get bd addr spaces' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic.

Line 273: `[get_bd_addr_segs ps7/S_AXI_HP0/HP0_DDR_LOWOCM] -force` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_addr_segs: the value named 'get bd addr segs' in this block; ps7: the value named 'ps7' in this block; S_AXI_HP0: the value named 's axi hp0' in this block.

Line 274: `assign_bd_address -offset 0x00000000 -range 0x20000000 \` This asks Vivado to establish an AXI address map; following explicit offset/range settings make the software-visible register addresses stable across rebuilds.

Line 275: `-target_address_space [get_bd_addr_spaces dma/Data_S2MM] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are target_address_space: the value named 'target address space' in this block; get_bd_addr_spaces: the value named 'get bd addr spaces' in this block; dma: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic.

Line 276: `[get_bd_addr_segs ps7/S_AXI_HP0/HP0_DDR_LOWOCM] -force` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_bd_addr_segs: the value named 'get bd addr segs' in this block; ps7: the value named 'ps7' in this block; S_AXI_HP0: the value named 's axi hp0' in this block.

Line 278: `validate_bd_design` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are validate_bd_design: the producer's statement that its current payload is meaningful.

Line 279: `save_bd_design` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are save_bd_design: the value named 'save bd design' in this block.

Line 280: `set bd_file [get_files system.bd]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; bd_file: the value named 'bd file' in this block; get_files: the value named 'get files' in this block.

Line 281: `generate_target all $bd_file` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are generate_target: the value named 'generate target' in this block; all: the value named 'all' in this block; bd_file: the value named 'bd file' in this block.

Line 282: `make_wrapper -files $bd_file -top` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are make_wrapper: the value named 'make wrapper' in this block; files: the value named 'files' in this block; bd_file: the value named 'bd file' in this block.

Line 283: `set wrapper_file [file join $vivado_project_dir "neuraedge_pynq_full.gen" "sources_1" "bd" "system" "hdl" "...` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; wrapper_file: the value named 'wrapper file' in this block; file: the value named 'file' in this block.

Line 284: `add_files -norecurse $wrapper_file` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are add_files: the value named 'add files' in this block; norecurse: the value named 'norecurse' in this block; wrapper_file: the value named 'wrapper file' in this block.

Line 285: `set_property top system_wrapper [current_fileset]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; top: the value named 'top' in this block; system_wrapper: the value named 'system wrapper' in this block.

Line 286: `update_compile_order -fileset sources_1` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are update_compile_order: the value named 'update compile order' in this block; fileset: the value named 'fileset' in this block; sources_1: the value named 'sources 1' in this block.

Line 288: `write_bd_tcl -force [file join $output_dir "generated_system_bd.tcl"]` This writes an auditable Vivado artifact using 'write_bd_tcl -force [file join $output_dir "generated_system_bd.tcl"]'; later documentation reads the resulting report rather than inventing a metric.

Line 289: `if {$build_mode eq "validate"} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are build_mode: the value named 'build mode' in this block; eq: the value named 'eq' in this block; validate: the producer's statement that its current payload is meaningful.

Line 290: `report_ip_status -file [file join $output_dir "validated_ip_status.rpt"]` This writes an auditable Vivado artifact using 'report_ip_status -file [file join $output_dir "validated_ip_status.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 291: `puts "NEURAEDGE_PYNQ_BD_VALIDATION_PASSED"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; NEURAEDGE_PYNQ_BD_VALIDATION_PASSED: the producer's statement that its current payload is meaningful.

Line 292: `close_project` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are close_project: the value named 'close project' in this block.

Line 293: `exit 0` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are exit: the value named 'exit' in this block.

Line 294: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 296: `set synth_run [get_runs synth_1]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; synth_run: the value named 'synth run' in this block; get_runs: the value named 'get runs' in this block.

Line 297: `set_property STEPS.SYNTH_DESIGN.ARGS.DIRECTIVE PerformanceOptimized $synth_run` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; STEPS: the value named 'steps' in this block; SYNTH_DESIGN: the value named 'synth design' in this block.

Line 298: `set impl_run [get_runs impl_1]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; impl_run: the value named 'impl run' in this block; get_runs: the value named 'get runs' in this block.

Line 299: `set_property STEPS.OPT_DESIGN.ARGS.DIRECTIVE Explore $impl_run` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; STEPS: the value named 'steps' in this block; OPT_DESIGN: the value named 'opt design' in this block.

Line 300: `set_property STEPS.PLACE_DESIGN.ARGS.DIRECTIVE ExtraNetDelay_high $impl_run` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; STEPS: the value named 'steps' in this block; PLACE_DESIGN: the value named 'place design' in this block.

Line 301: `set_property STEPS.PHYS_OPT_DESIGN.IS_ENABLED true $impl_run` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; STEPS: the value named 'steps' in this block; PHYS_OPT_DESIGN: the value named 'phys opt design' in this block.

Line 302: `set_property STEPS.PHYS_OPT_DESIGN.ARGS.DIRECTIVE AggressiveExplore $impl_run` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; STEPS: the value named 'steps' in this block; PHYS_OPT_DESIGN: the value named 'phys opt design' in this block.

Line 303: `set_property STEPS.ROUTE_DESIGN.ARGS.DIRECTIVE AggressiveExplore $impl_run` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; STEPS: the value named 'steps' in this block; ROUTE_DESIGN: the value named 'route design' in this block.

Line 304: `set_property STEPS.POST_ROUTE_PHYS_OPT_DESIGN.IS_ENABLED true $impl_run` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; STEPS: the value named 'steps' in this block; POST_ROUTE_PHYS_O the value named 'post route phys opt design' in this block.

Line 305: `# Vivado 2026.1 on Windows reproducibly faults in AggressiveExplore's` This comment documents the batch-flow intent: Vivado 2026.1 on Windows reproducibly faults in AggressiveExplore's

Line 306: `# post-route hold-fix phase for this design. Explore retains post-route setup` This comment documents the batch-flow intent: post-route hold-fix phase for this design. Explore retains post-route setup

Line 307: `# and hold repair without entering that unstable phase.` This comment documents the batch-flow intent: and hold repair without entering that unstable phase.

Line 308: `set_property STEPS.POST_ROUTE_PHYS_OPT_DESIGN.ARGS.DIRECTIVE Explore $impl_run` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set_property: the value named 'set property' in this block; STEPS: the value named 'steps' in this block; POST_ROUTE_PHYS_OPT_DESIGN: the value named 'post route phys opt design' in this block.

Line 310: `launch_runs synth_1 -jobs 2` This controls a named Vivado synthesis or implementation run with 'launch_runs synth_1 -jobs 2'. Waiting and opening the completed run prevents reports from reading an earlier or unfinished netlist.

Line 311: `wait_on_run synth_1` This controls a named Vivado synthesis or implementation run with 'wait_on_run synth_1'. Waiting and opening the completed run prevents reports from reading an earlier or unfinished netlist.

Line 312: `if {[get_property PROGRESS $synth_run] ne "100%"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_property: the value named 'get property' in this block; PROGRESS: the value named 'progress' in this block; synth_run: the value named 'synth run' in this block.

Line 313: `|| ![string match "*Complete*" [get_property STATUS $synth_run]]} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are string: the value named 'string' in this block; match: the value named 'match' in this block; Complete: the value named 'complete' in this block.

Line 314: `error "Full PYNQ synthesis failed: [get_property STATUS $synth_run]"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Full: the value named 'full' in this block; PYNQ: the value named 'pynq' in this block.

Line 315: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 316: `launch_runs impl_1 -to_step write_bitstream -jobs 2` This controls a named Vivado synthesis or implementation run with 'launch_runs impl_1 -to_step write_bitstream -jobs 2'. Waiting and opening the completed run prevents reports from reading an earlier or unfinished netlist.

Line 317: `wait_on_run impl_1` This controls a named Vivado synthesis or implementation run with 'wait_on_run impl_1'. Waiting and opening the completed run prevents reports from reading an earlier or unfinished netlist.

Line 318: `if {[get_property PROGRESS $impl_run] ne "100%"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are get_property: the value named 'get property' in this block; PROGRESS: the value named 'progress' in this block; impl_run: the value named 'impl run' in this block.

Line 319: `|| ![string match "*Complete*" [get_property STATUS $impl_run]]} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are string: the value named 'string' in this block; match: the value named 'match' in this block; Complete: the value named 'complete' in this block.

Line 320: `error "Full PYNQ implementation failed: [get_property STATUS $impl_run]"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Full: the value named 'full' in this block; PYNQ: the value named 'pynq' in this block.

Line 321: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 323: `open_run impl_1` This controls a named Vivado synthesis or implementation run with 'open_run impl_1'. Waiting and opening the completed run prevents reports from reading an earlier or unfinished netlist.

Line 324: `report_utilization -hierarchical -file [file join $output_dir "full_routed_utilization_hierarchical` This writes an auditable Vivado artifact using 'report_utilization -hierarchical -file [file join $output_dir "full_routed_utilization_hi later documentation reads the resulting report rather than inventing a metric.

Line 325: `report_timing_summary -delay_type min_max -max_paths 50 \` This writes an auditable Vivado artifact using 'report_timing_summary -delay_type min_max -max_paths 50 \'; later documentation reads the resulting report rather than inventing a metric.

Line 326: `-file [file join $output_dir "full_routed_timing_summary.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 327: `report_timing -delay_type max -max_paths 250 -sort_by group \` This writes an auditable Vivado artifact using 'report_timing -delay_type max -max_paths 250 -sort_by group \'; later documentation reads the resulting report rather than inventing a metric.

Line 328: `-file [file join $output_dir "full_routed_timing_paths_250.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 329: `report_clock_interaction -file [file join $output_dir "full_clock_interaction.rpt"]` This writes an auditable Vivado artifact using 'report_clock_interaction -file [file join $output_dir "full_clock_interaction.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 330: `report_cdc -details -file [file join $output_dir "full_cdc.rpt"]` This writes an auditable Vivado artifact using 'report_cdc -details -file [file join $output_dir "full_cdc.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 331: `set full_route_status_text [report_route_status -return_string]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; full_route_status_text: the value named 'full route status text' in this block; report_route_status: the value named 'report route status' in this block.

Line 332: `set route_status_file [open [file join $output_dir "full_route_status.rpt"] "w"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; route_status_file: the value named 'route status file' in this block; open: the value named 'open' in this block.

Line 333: `puts -nonewline $route_status_file $full_route_status_text` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; nonewline: the value named 'nonewline' in this block; route_status_file: the value named 'route status file' in this block.

Line 334: `close $route_status_file` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are close: the value named 'close' in this block; route_status_file: the value named 'route status file' in this block.

Line 335: `report_drc -file [file join $output_dir "full_drc.rpt"]` This writes an auditable Vivado artifact using 'report_drc -file [file join $output_dir "full_drc.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 336: `report_methodology -file [file join $output_dir "full_methodology.rpt"]` This writes an auditable Vivado artifact using 'report_methodology -file [file join $output_dir "full_methodology.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 337: `report_design_analysis -congestion -file [file join $output_dir "full_congestion.rpt"]` This writes an auditable Vivado artifact using 'report_design_analysis -congestion -file [file join $output_dir "full_congestion.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 338: `report_power -file [file join $output_dir "full_vectorless_power.rpt"]` This writes an auditable Vivado artifact using 'report_power -file [file join $output_dir "full_vectorless_power.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 339: `write_checkpoint -force [file join $output_dir "neuraedge_pynq_full_routed.dcp"]` This writes an auditable Vivado artifact using 'write_checkpoint -force [file join $output_dir "neuraedge_pynq_full_routed.dcp"]'; later documentation reads the resulting report rather than inventing a metric.

Line 341: `set setup_path [get_timing_paths -delay_type max -max_paths 1]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; setup_path: the value named 'setup path' in this block; get_timing_paths: the value named 'get timing paths' in this block.

Line 342: `set hold_path [get_timing_paths -delay_type min -max_paths 1]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; hold_path: the value named 'hold path' in this block; get_timing_paths: the value named 'get timing paths' in this block.

Line 343: `set setup_wns [get_property SLACK $setup_path]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; setup_wns: the value named 'setup wns' in this block; get_property: the value named 'get property' in this block.

Line 344: `set hold_whs [get_property SLACK $hold_path]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; hold_whs: the value named 'hold whs' in this block; get_property: the value named 'get property' in this block.

Line 345: `set route_error_count -1` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; route_error_count: an integer count retained for control or measurable telemetry.

Line 346: `if {[regexp {# of nets with routing errors\.*\s*:\s*([0-9]+)} \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are regexp: the value named 'regexp' in this block; of: the value named 'of' in this block; nets: the value named 'nets' in this block.

Line 347: `$full_route_status_text route_match route_count]} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are full_route_status_text: the value named 'full route status text' in this block; route_match: the value named 'route match' in this block; route_count: an integer count retained for control or measurable telemetry.

Line 348: `set route_error_count $route_count` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; route_error_count: an integer count retained for control or measurable telemetry; route_count: an integer count retained for control or measurable telemetry.

Line 349: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 350: `set drc_error_count [llength [get_drc_violations -quiet -filter {SEVERITY == Error}]]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; drc_error_count: an integer count retained for control or measurable telemetry; llength: the value named 'llength' in this block.

Line 351: `set summary_file [open [file join $output_dir "full_implementation_summary.txt"] "w"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; summary_file: the value named 'summary file' in this block; open: the value named 'open' in this block.

Line 352: `puts $summary_file "board_part=$board_part"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; board_part: the value named 'board part' in this block.

Line 353: `puts $summary_file "part=xc7z020clg400-1"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; part: the value named 'part' in this block.

Line 354: `puts $summary_file "accelerator_clock_mhz=200.0"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; accelerator_clock_mhz: the value named 'accelerator clock mhz' in this block.

Line 355: `puts $summary_file "dma_clock_mhz=125.0"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; dma_clock_mhz: the direct-memory-access engine moving contiguous buffers between DDR and FPGA logic.

Line 356: `puts $summary_file "control_clock_mhz=100.0"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; control_clock_mhz: the value named 'control clock mhz' in this block.

Line 357: `puts $summary_file "lut_count=[llength [get_cells -hierarchical -filter {REF_NAME =~ LUT*}]]"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; lut_count: an integer count retained for control or measurable telemetry.

Line 358: `puts $summary_file "ff_count=[llength [get_cells -hierarchical -filter {REF_NAME =~ FD*}]]"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; ff_count: an integer count retained for control or measurable telemetry.

Line 359: `puts $summary_file "dsp48_count=[llength [get_cells -hierarchical -filter {REF_NAME =~ DSP48*}]]"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; dsp48_count: an integer count retained for control or measurable telemetry.

Line 360: `puts $summary_file "bram36_count=[llength [get_cells -hierarchical -filter {REF_NAME == RAMB36E1}]]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; bram36_count: an integer count retained for control or measurable telemetry.

Line 361: `puts $summary_file "bram18_count=[llength [get_cells -hierarchical -filter {REF_NAME == RAMB18E1}]]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; bram18_count: an integer count retained for control or measurable telemetry.

Line 362: `puts $summary_file "setup_wns_ns=$setup_wns"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; setup_wns_ns: the value named 'setup wns ns' in this block.

Line 363: `puts $summary_file "hold_whs_ns=$hold_whs"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; hold_whs_ns: the value named 'hold whs ns' in this block.

Line 364: `puts $summary_file "route_error_count=$route_error_count"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; route_error_count: an integer count retained for control or measurable telemetry.

Line 365: `puts $summary_file "drc_error_count=$drc_error_count"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; drc_error_count: an integer count retained for control or measurable telemetry.

Line 366: `close $summary_file` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are close: the value named 'close' in this block; summary_file: the value named 'summary file' in this block.

Line 368: `if {$route_error_count != 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are route_error_count: an integer count retained for control or measurable telemetry.

Line 369: `error "Full PYNQ routing is incomplete: route_error_count=$route_error_count"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Full: the value named 'full' in this block; PYNQ: the value named 'pynq' in this block.

Line 370: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 371: `if {$drc_error_count != 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are drc_error_count: an integer count retained for control or measurable telemetry.

Line 372: `error "Full PYNQ design has DRC errors: drc_error_count=$drc_error_count"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Full: the value named 'full' in this block; PYNQ: the value named 'pynq' in this block.

Line 373: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 374: `if {$setup_wns < 0.0} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are setup_wns: the value named 'setup wns' in this block.

Line 375: `error "Full PYNQ design missed setup timing: WNS=$setup_wns ns"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Full: the value named 'full' in this block; PYNQ: the value named 'pynq' in this block.

Line 376: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 377: `if {$hold_whs < 0.0} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are hold_whs: the value named 'hold whs' in this block.

Line 378: `error "Full PYNQ design missed hold timing: WHS=$hold_whs ns"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Full: the value named 'full' in this block; PYNQ: the value named 'pynq' in this block.

Line 379: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 381: `set generated_bit [file join $vivado_project_dir \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; generated_bit: the value named 'generated bit' in this block; file: the value named 'file' in this block.

Line 382: `"neuraedge_pynq_full.runs" "impl_1" "system_wrapper.bit"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are neuraedge_pynq_full: the value named 'neuraedge pynq full' in this block; runs: the value named 'runs' in this block; impl_1: the value named 'impl 1' in this block.

Line 383: `if {![file exists $generated_bit]} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; exists: the value named 'exists' in this block; generated_bit: the value named 'generated bit' in this block.

Line 384: `error "Implementation completed but bitstream was not found at $generated_bit"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Implementation: the value named 'implementation' in this block; completed: the value named 'completed' in this block.

Line 385: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 386: `file copy -force $generated_bit [file join $output_dir "neuraedge_pynq_full.bit"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; copy: the value named 'copy' in this block; force: the value named 'force' in this block.

Line 387: `set generated_hwh [file join $vivado_project_dir "neuraedge_pynq_full.gen" "sources_1" "bd" "system" "hw_ha...` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are set: the value named 'set' in this block; generated_hwh: the value named 'generated hwh' in this block; file: the value named 'file' in this block.

Line 388: `if {![file exists $generated_hwh]} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; exists: the value named 'exists' in this block; generated_hwh: the value named 'generated hwh' in this block.

Line 389: `error "Block design completed but hardware handoff was not found at $generated_hwh"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are error: the value named 'error' in this block; Block: the value named 'block' in this block; design: the value named 'design' in this block.

Line 390: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 391: `file copy -force $generated_hwh [file join $output_dir "neuraedge_pynq_full.hwh"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; copy: the value named 'copy' in this block; force: the value named 'force' in this block.

Line 392: `write_hw_platform -fixed -include_bit -force \` This writes an auditable Vivado artifact using 'write_hw_platform -fixed -include_bit -force \'; later documentation reads the resulting report rather than inventing a metric.

Line 393: `-file [file join $output_dir "neuraedge_pynq_full.xsa"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq full hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

91 scripts/build_pynq_ooc.tcl

Role in the complete system: runs the reproducible Vivado build pynq ooc hardware flow.

91.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado build pynq ooc hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

91.2 Complete source

```
set script_dir [file normalize [file dirname [info script]]]
set project_dir [file normalize [file join $script_dir ".."]]
set output_name "pynq_ooc"
if {$argc >= 1} {
    set output_name [lindex $argv 0]
}
set output_dir [file join $project_dir "outputs" $output_name]
file mkdir $output_dir

set rtl_files [list \
    [file join $project_dir "rtl" "neuraedge_command_pkg.sv"] \
    [file join $project_dir "rtl" "neuraedge_mac2_u8.sv"] \
    [file join $project_dir "rtl" "neuraedge_tensor_tile.sv"] \
    [file join $project_dir "rtl" "neuraedge_active_index_buffer.sv"] \
    [file join $project_dir "rtl" "neuraedge_activation_buffer.sv"] \
    [file join $project_dir "rtl" "neuraedge_weight_bank.sv"] \
    [file join $project_dir "rtl" "neuraedge_buffered_tensor_core.sv"] \
    [file join $project_dir "rtl" "neuraedge_alpha_xnor.sv"] \
    [file join $project_dir "rtl" "neuraedge_vector_alu.sv"] \
    [file join $project_dir "rtl" "neuraedge_udivider.sv"] \
    [file join $project_dir "rtl" "neuraedge_isqrt64.sv"] \
    [file join $project_dir "rtl" "neuraedge_integer_layernorm.sv"] \
    [file join $project_dir "rtl" "neuraedge_log_softmax.sv"] \
    [file join $project_dir "rtl" "neuraedge_lif_bank.sv"] \
    [file join $project_dir "rtl" "neuraedge_sgd_optimizer8.sv"] \
    [file join $project_dir "rtl" "neuraedge_gradient_accumulator_bank.sv"] \
    [file join $project_dir "rtl" "neuraedge_linear_backward8.sv"] \
    [file join $project_dir "rtl" "neuraedge_activation_lut8.sv"] \
    [file join $project_dir "rtl" "neuraedge_universal_core.sv"] \
    [file join $project_dir "rtl" "neuraedge_descriptor_sequencer.sv"] \
    [file join $project_dir "rtl" "neuraedge_pynq_axi_top.sv"]]

read_verilog -sv $rtl_files
synth_design -top neuraedge_pynq_axi_top -part xc7z020clg400-1 \
    -mode out_of_context -flatten_hierarchy rebuilt -directive PerformanceOptimized
create_clock -name aclk -period 5.000 [get_ports aclk]
set_clock_uncertainty 0.100 [get_clocks aclk]
set_property HD.CLK_SRC BUFGCTRL_X0Y0 [get_ports aclk]
set_false_path -from [get_ports aresetn]
set data_inputs [get_ports -filter {DIRECTION == IN && NAME != "aclk" && NAME != "aresetn"}]
set data_outputs [get_ports -filter {DIRECTION == OUT}]
set_input_delay -clock aclk 1.000 $data_inputs
set_output_delay -clock aclk 1.000 $data_outputs

report_utilization -hierarchical -file [file join $output_dir "post_synth_utilization_hierarchical.rpt"]
report_timing_summary -file [file join $output_dir "post_synth_timing.rpt"]
write_checkpoint -force [file join $output_dir "post_synth.dcp"]

opt_design -directive Explore
place_design -directive ExtraNetDelay_high
phys_opt_design -directive AggressiveExplore
route_design -directive AggressiveExplore
# Persist the expensive placed-and-routed solution before post-route physical
# synthesis. Vivado can then resume without repeating placement if its
# optimizer process terminates unexpectedly.
write_checkpoint -force [file join $output_dir "pre_postroute_physopt.dcp"]
# Vivado 2026.1 on Windows reproducibly faults in AggressiveExplore's
# post-route hold-fix phase. Explore performs the same critical-path repair
# without entering that faulty phase; aggressive optimization remains enabled
# before routing, where it completes normally.
```

```
phys_opt_design -directive Explore
write_checkpoint -force [file join $output_dir "postroute_physopt_unrouted.dcp"]
# Post-route physical optimization may move cells or insert hold buffers. Route
# those edits before reporting timing so setup and hold are both physically real.
route_design -directive Quick
source [file join $script_dir "finalize_pynq_ooc.tcl"]
```

91.3 Line annotations

Line 1: `set script_dir [file normalize [file dirname [info script]]]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; script_dir: the value named 'script dir' in this block; file: the value named 'file' in this block.

Line 2: `set project_dir [file normalize [file join $script_dir ".."]]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; project_dir: the value named 'project dir' in this block; file: the value named 'file' in this block.

Line 3: `set output_name "pynq_ooc"` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; output_name: the value named 'output name' in this block; pynq_ooc: the value named 'pynq ooc' in this block.

Line 4: `if {$argc >= 1} {` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are argc: the value named 'argc' in this block.

Line 5: `set output_name [lindex $argv 0]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; output_name: the value named 'output name' in this block; lindex: an integer selecting one tensor element, lane, bank, or state entry.

Line 6: `}` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 7: `set output_dir [file join $project_dir "outputs" $output_name]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; output_dir: the value named 'output dir' in this block; file: the value named 'file' in this block.

Line 8: `file mkdir $output_dir` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; mkdir: the value named 'mkdir' in this block; output_dir: the value named 'output dir' in this block.

Line 10: `set rtl_files [list \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; rtl_files: the value named 'rtl files' in this block; list: the value named 'list' in this block.

Line 11: `[file join $project_dir "rtl" "neuraedge_command_pkg.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 12: `[file join $project_dir "rtl" "neuraedge_mac2_u8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 13: `[file join $project_dir "rtl" "neuraedge_tensor_tile.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 14: `[file join $project_dir "rtl" "neuraedge_active_index_buffer.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 15: `[file join $project_dir "rtl" "neuraedge_activation_buffer.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 16: `[file join $project_dir "rtl" "neuraedge_weight_bank.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 17: `[file join $project_dir "rtl" "neuraedge_buffered_tensor_core.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 18: `[file join $project_dir "rtl" "neuraedge_alpha_xnor.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 19: `[file join $project_dir "rtl" "neuraedge_vector_alu.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 20: `[file join $project_dir "rtl" "neuraedge_udivider.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 21: `[file join $project_dir "rtl" "neuraedge_isqrt64.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 22: `[file join $project_dir "rtl" "neuraedge_integer_layernorm.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 23: `[file join $project_dir "rtl" "neuraedge_log_softmax.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 24: `[file join $project_dir "rtl" "neuraedge_lif_bank.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 25: `[file join $project_dir "rtl" "neuraedge_sgd_optimizer8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 26: `[file join $project_dir "rtl" "neuraedge_gradient_accumulator_bank.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 27: `[file join $project_dir "rtl" "neuraedge_linear_backward8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 28: `[file join $project_dir "rtl" "neuraedge_activation_lut8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 29: `[file join $project_dir "rtl" "neuraedge_universal_core.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 30: `[file join $project_dir "rtl" "neuraedge_descriptor_sequencer.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 31: `[file join $project_dir "rtl" "neuraedge_pynq_axi_top.sv"]]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 33: `read_verilog -sv $rtl_files` This loads the listed RTL into Vivado's in-memory design so the subsequent synthesis uses the repository source.

Line 34: `synth_design -top neuraedge_pynq_axi_top -part xc7z020clg400-1 \` This invokes Vivado synthesis for the named top and FPGA part, mapping behavioral RTL into LUT, flip-flop, BRAM, and DSP primitives.

Line 35: `-mode out_of_context -flatten_hierarchy rebuilt -directive PerformanceOptimized` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are mode: the value named 'mode' in this block; out_of_context: the value named 'out of context' in this block; flatten_hierarchy: the value named 'flatten hierarchy' in this block.

Line 36: `create_clock -name aclk -period 5.000 [get_ports aclk]` This declares the clock period against which Vivado checks every synchronous setup and hold path.

Line 37: `set_clock_uncertainty 0.100 [get_clocks aclk]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set_clock_uncertainty: the value named 'set clock uncertainty' in this block; get_clocks: the value named 'get clocks' in this block; aclk: the AXI and accelerator clock.

Line 38: `set_property HD.CLK_SRC BUFGCTRL_X0Y0 [get_ports aclk]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set_property: the value named 'set property' in this block; HD: the value named 'hd' in this block; CLK_SRC: the clock that makes all registers advance together.

Line 39: `set_false_path -from [get_ports aresetn]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set_false_path: the value named 'set false path' in this block; from: the value named 'from' in this block; get_ports: the value named 'get ports' in this block.

Line 40: `set data_inputs [get_ports -filter {DIRECTION == IN && NAME != "aclk" && NAME != "aresetn"}]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; data_inputs: the value named 'data inputs' in this block; get_ports: the value named 'get ports' in this block.

Line 41: `set data_outputs [get_ports -filter {DIRECTION == OUT}]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; data_outputs: the value named 'data outputs' in this block; get_ports: the value named 'get ports' in this block.

Line 42: `set_input_delay -clock aclk 1.000 $data_inputs` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set_input_delay: the value named 'set input delay' in this block; clock: the value named 'clock' in this block; aclk: the AXI and accelerator clock.

Line 43: `set_output_delay -clock aclk 1.000 $data_outputs` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are set_output_delay: the value named 'set output delay' in this block; clock: the value named 'clock' in this block; aclk: the AXI and accelerator clock.

Line 45: `report_utilization -hierarchical -file [file join $output_dir "post_synth_utilization_hierarchical.r`
This writes an auditable Vivado artifact using ‘report_utilization -hierarchical -file [file join $output_dir "post_synth_utilization_hi later documentation reads the resulting report rather than inventing a metric.

Line 46: `report_timing_summary -file [file join $output_dir "post_synth_timing.rpt"]` This writes an auditable Vivado artifact using ‘report_timing_summary -file [file join $output_dir "post_synth_timing.rpt"]‘; later documentation reads the resulting report rather than inventing a metric.

Line 47: `write_checkpoint -force [file join $output_dir "post_synth.dcp"]` This writes an auditable Vivado artifact using ‘write_checkpoint -force [file join $output_dir "post_synth.dcp"]‘; later documentation reads the resulting report rather than inventing a metric.

Line 49: `opt_design -directive Explore` This runs Vivado implementation command ‘opt_design -directive Explore‘, changing the physical netlist or placement/routing needed for real timing.

Line 50: `place_design -directive ExtraNetDelay_high` This runs Vivado implementation command ‘place_design -directive ExtraNetDelay_high‘, changing the physical netlist or placement/routing needed for real timing.

Line 51: `phys_opt_design -directive AggressiveExplore` This runs Vivado implementation command ‘phys_opt_design -directive AggressiveExplore‘, changing the physical netlist or placement/routing needed for real timing.

Line 52: `route_design -directive AggressiveExplore` This runs Vivado implementation command 'route_design -directive AggressiveExplore', changing the physical netlist or placement/routing needed for real timing.

Line 53: `# Persist the expensive placed-and-routed solution before post-route physical` This comment documents the batch-flow intent: Persist the expensive placed-and-routed solution before post-route physical

Line 54: `# synthesis. Vivado can then resume without repeating placement if its` This comment documents the batch-flow intent: synthesis. Vivado can then resume without repeating placement if its

Line 55: `# optimizer process terminates unexpectedly.` This comment documents the batch-flow intent: optimizer process terminates unexpectedly.

Line 56: `write_checkpoint -force [file join $output_dir "pre_postroute_physopt.dcp"]` This writes an auditable Vivado artifact using 'write_checkpoint -force [file join $output_dir "pre_postroute_physopt.dcp"]'; later documentation reads the resulting report rather than inventing a metric.

Line 57: `# Vivado 2026.1 on Windows reproducibly faults in AggressiveExplore's` This comment documents the batch-flow intent: Vivado 2026.1 on Windows reproducibly faults in AggressiveExplore's

Line 58: `# post-route hold-fix phase. Explore performs the same critical-path repair` This comment documents the batch-flow intent: post-route hold-fix phase. Explore performs the same critical-path repair

Line 59: `# without entering that faulty phase; aggressive optimization remains enabled` This comment documents the batch-flow intent: without entering that faulty phase; aggressive optimization remains enabled

Line 60: `# before routing, where it completes normally.` This comment documents the batch-flow intent: before routing, where it completes normally.

Line 61: `phys_opt_design -directive Explore` This runs Vivado implementation command 'phys_opt_design -directive Explore', changing the physical netlist or placement/routing needed for real timing.

Line 62: `write_checkpoint -force [file join $output_dir "postroute_physopt_unrouted.dcp"]` This writes an auditable Vivado artifact using 'write_checkpoint -force [file join $output_dir "postroute_physopt_unrouted.dcp"]'; later documentation reads the resulting report rather than inventing a metric.

Line 63: `# Post-route physical optimization may move cells or insert hold buffers. Route` This comment documents the batch-flow intent: Post-route physical optimization may move cells or insert hold buffers. Route

Line 64: `# those edits before reporting timing so setup and hold are both physically real.` This comment documents the batch-flow intent: those edits before reporting timing so setup and hold are both physically real.

Line 65: `route_design -directive Quick` This runs Vivado implementation command 'route_design -directive Quick', changing the physical netlist or placement/routing needed for real timing.

Line 66: `source [file join $script_dir "finalize_pynq_ooc.tcl"]` This batch-control line contributes to the flow that runs the reproducible Vivado build pynq ooc hardware flow. The important operands are source: the value named 'source' in this block; file: the value named 'file' in this block; join: the value named 'join' in this block.

92 scripts/capture_power_activity.tcl

Role in the complete system: runs the reproducible Vivado capture power activity hardware flow.

92.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado capture power activity hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

92.2 Complete source

```
foreach required_variable {
    NEURAEDGE_SAIF_PATH
    NEURAEDGE_SAIF_SCOPE
    NEURAEDGE_SAIF_AMOUNT
    NEURAEDGE_SAIF_UNIT
} {
    if {![info exists ::env($required_variable)]} {
        error "$required_variable is required"
    }
}

set saif_path [file normalize $::env(NEURAEDGE_SAIF_PATH)]
set saif_scope $::env(NEURAEDGE_SAIF_SCOPE)
set run_amount $::env(NEURAEDGE_SAIF_AMOUNT)
set run_unit $::env(NEURAEDGE_SAIF_UNIT)
set activity_objects [get_objects -r "${saif_scope}/*"]
if {[llength $activity_objects] == 0} {
    error "No simulation objects matched SAIF scope $saif_scope"
}

open_saif $saif_path
log_saif $activity_objects
run $run_amount $run_unit
close_saif
quit
```

92.3 Line annotations

Line 1: `foreach required_variable {` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are foreach: the value named 'foreach' in this block; required_variable: the value named 'required variable' in this block.

Line 2: `NEURAEDGE_SAIF_PATH` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are NEURAEDGE_SAIF_PATH: the value named 'neuraedge saif path' in this block.

Line 3: `NEURAEDGE_SAIF_SCOPE` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are NEURAEDGE_SAIF_SCOPE: the value named 'neuraedge saif scope' in this block.

Line 4: `NEURAEDGE_SAIF_AMOUNT` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are NEURAEDGE_SAIF_AMOUNT: the value named 'neuraedge saif amount' in this block.

Line 5: `NEURAEDGE_SAIF_UNIT` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are NEURAEDGE_SAIF_UNIT: the value named 'neuraedge saif unit' in this block.

Line 6: `} {` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 7: `if {![info exists ::env($required_variable)]} {` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are info: the value named 'info' in this block; exists: the value named 'exists' in this block; env: the value named 'env' in this block.

Line 8: `error "$required_variable is required"` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are error: the value named 'error' in this block; required_variable: the value named 'required variable' in this block; is: the value named 'is' in this block.

Line 9: `}` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 10: `}` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 12: `set saif_path [file normalize $::env(NEURAEDGE_SAIF_PATH)]` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are set: the value named 'set' in this block; saif_path: the value named 'saif path' in this block; file: the value named 'file' in this block.

Line 13: `set saif_scope $::env(NEURAEDGE_SAIF_SCOPE)` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are set: the value named 'set' in this block; saif_scope: the value named 'saif scope' in this block; env: the value named 'env' in this block.

Line 14: `set run_amount $::env(NEURAEDGE_SAIF_AMOUNT)` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are set: the value named 'set' in this block; run_amount: the value named 'run amount' in this block; env: the value named 'env' in this block.

Line 15: `set run_unit $::env(NEURAEDGE_SAIF_UNIT)` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are set: the value named 'set' in this block; run_unit: the value named 'run unit' in this block; env: the value named 'env' in this block.

Line 16: `set activity_objects [get_objects -r "${saif_scope}/*"]` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are set: the value named 'set' in this block; activity_objects: the value named 'activity objects' in this block; get_objects: the value named 'get objects' in this block.

Line 17: `if {[llength $activity_objects] == 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are llength: the value named 'llength' in this block; activity_objects: the value named 'activity objects' in this block.

Line 18: `error "No simulation objects matched SAIF scope $saif_scope"` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are error: the value named 'error' in this block; No: the value named 'no' in this block; simulation: the value named 'simulation' in this block.

Line 19: `}` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 21: `open_saif $saif_path` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are open_saif: the value named 'open saif' in this block; saif_path: the value named 'saif path' in this block.

Line 22: `log_saif $activity_objects` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are log_saif: the value named 'log saif' in this block; activity_objects: the value named 'activity objects' in this block.

Line 23: `run $run_amount $run_unit` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are run: the value named 'run' in this block; run_amount: the value named 'run amount' in this block; run_unit: the value named 'run unit' in this block.

Line 24: `close_saif` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are close_saif: the value named 'close saif' in this block.

Line 25: `quit` This batch-control line contributes to the flow that runs the reproducible Vivado capture power activity hardware flow. The important operands are quit: the value named 'quit' in this block.

93 scripts/capture_waveform.tcl

Role in the complete system: runs the reproducible Vivado capture waveform hardware flow.

93.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado capture waveform hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

93.2 Complete source

```
if {![info exists ::env(NEURAEDGE_VCD_PATH)]} {
    error "NEURAEDGE_VCD_PATH is required"
}
if {![info exists ::env(NEURAEDGE_WAVE_AMOUNT)]} {
    error "NEURAEDGE_WAVE_AMOUNT is required"
}
if {![info exists ::env(NEURAEDGE_WAVE_UNIT)]} {
    error "NEURAEDGE_WAVE_UNIT is required"
}

set vcd_path [file normalize $::env(NEURAEDGE_VCD_PATH)]
set run_amount $::env(NEURAEDGE_WAVE_AMOUNT)
set run_unit $::env(NEURAEDGE_WAVE_UNIT)

open_vcd $vcd_path
log_vcd [get_objects -r /*]
log_wave -r /*
run $run_amount $run_unit
close_vcd
quit
```

93.3 Line annotations

Line 1: `if {![info exists ::env(NEURAEDGE_VCD_PATH)]} {` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are info: the value named 'info' in this block; exists: the value named 'exists' in this block; env: the value named 'env' in this block.

Line 2: `error "NEURAEDGE_VCD_PATH is required"` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are error: the value named 'error' in this block; NEURAEDGE_VCD_PATH: the value named 'neuraedge vcd path' in this block; is: the value named 'is' in this block.

Line 3: `}` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 4: `if {![info exists ::env(NEURAEDGE_WAVE_AMOUNT)]} {` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are info: the value named 'info' in this block; exists: the value named 'exists' in this block; env: the value named 'env' in this block.

Line 5: `error "NEURAEDGE_WAVE_AMOUNT is required"` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are error: the value named 'error' in this block; NEURAEDGE_WAVE_AMOUNT: the value named 'neuraedge wave amount' in this block; is: the value named 'is' in this block.

Line 6: `}` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 7: `if {![info exists ::env(NEURAEDGE_WAVE_UNIT)]} {` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are info: the value named 'info' in this block; exists: the value named 'exists' in this block; env: the value named 'env' in this block.

Line 8: `error "NEURAEDGE_WAVE_UNIT is required"` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are error: the value named 'error' in this block; NEURAEDGE_WAVE_UNIT: the value named 'neuraedge wave unit' in this block; is: the value named 'is' in this block.

Line 9: `}` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 11: `set vcd_path [file normalize $::env(NEURAEDGE_VCD_PATH)]` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are set: the value named 'set' in this block; vcd_path: the value named 'vcd path' in this block; file: the value named 'file' in this block.

Line 12: `set run_amount $::env(NEURAEDGE_WAVE_AMOUNT)` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are set: the value named 'set' in this block; run_amount: the value named 'run amount' in this block; env: the value named 'env' in this block.

Line 13: `set run_unit $::env(NEURAEDGE_WAVE_UNIT)` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are set: the value named 'set' in this block; run_unit: the value named 'run unit' in this block; env: the value named 'env' in this block.

Line 15: `open_vcd $vcd_path` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are open_vcd: the value named 'open vcd' in this block; vcd_path: the value named 'vcd path' in this block.

Line 16: `log_vcd [get_objects -r /*]` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are log_vcd: the value named 'log vcd' in this block; get_objects: the value named 'get objects' in this block; r: the value named 'r' in this block.

Line 17: `log_wave -r /*` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are log_wave: the value named 'log wave' in this block; r: the value named 'r' in this block.

Line 18: `run $run_amount $run_unit` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are run: the value named 'run' in this block; run_amount: the value named 'run amount' in this block; run_unit: the value named 'run unit' in this block.

Line 19: `close_vcd` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are close_vcd: the value named 'close vcd' in this block.

Line 20: `quit` This batch-control line contributes to the flow that runs the reproducible Vivado capture waveform hardware flow. The important operands are quit: the value named 'quit' in this block.

94 scripts/export_pynq_diagrams.tcl

Role in the complete system: runs the reproducible Vivado export pynq diagrams hardware flow.

94.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado export pynq diagrams hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

94.2 Complete source

```
if {$argc != 2} {
    error "Usage: export_pynq_diagrams.tcl <project.xpr> <output_dir>"
}

set project_file [file normalize [lindex $argv 0]]
set output_dir [file normalize [lindex $argv 1]]
if {![file exists $project_file]} {
    error "Vivado project does not exist: $project_file"
}
file mkdir $output_dir

open_project $project_file
set bd_files [get_files -quiet system.bd]
if {[llength $bd_files] != 1} {
    error "Expected exactly one system.bd, found [llength $bd_files]"
}
open_bd_design [lindex $bd_files 0]

set pdf_path [file join $output_dir "neuraedge_pynq_block_design.pdf"]
set svg_path [file join $output_dir "neuraedge_pynq_block_design.svg"]
write_bd_layout -force -format pdf -orientation landscape -scope all $pdf_path
write_bd_layout -force -format svg -orientation landscape -scope all $svg_path
foreach diagram [list $pdf_path $svg_path] {
    if {![file exists $diagram] || [file size $diagram] == 0} {
        error "Block-design diagram was not generated: $diagram"
    }
}

puts "NEURAEDGE_PYNQ_DIAGRAMS_EXPORTED pdf=$pdf_path svg=$svg_path"
close_project
exit 0
```

94.3 Line annotations

Line 1: `if {$argc != 2} {` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are argc: the value named 'argc' in this block.

Line 2: `error "Usage: export_pynq_diagrams.tcl <project.xpr> <output_dir>"` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are error: the value named 'error' in this block; Usage: the value named 'usage' in this block; export_pynq_diagrams: the value named 'export pynq diagrams' in this block.

Line 3: `}` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 5: `set project_file [file normalize [lindex $argv 0]]` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are set: the value named 'set' in this block; project_file: the value named 'project file' in this block; file: the value named 'file' in this block.

Line 6: `set output_dir [file normalize [lindex $argv 1]]` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are set: the value named 'set' in this block; output_dir: the value named 'output dir' in this block; file: the value named 'file' in this block.

Line 7: `if {![file exists $project_file]} {` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are file: the value named 'file' in this block; exists: the value named 'exists' in this block; project_file: the value named 'project file' in this block.

Line 8: `error "Vivado project does not exist: $project_file"` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are error: the value named 'error' in this block; Vivado: the value named 'vivado' in this block; project: the value named 'project' in this block.

Line 9: `}` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 10: `file mkdir $output_dir` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are file: the value named 'file' in this block; mkdir: the value named 'mkdir' in this block; output_dir: the value named 'output dir' in this block.

Line 12: `open_project $project_file` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are open_project: the value named 'open project' in this block; project_file: the value named 'project file' in this block.

Line 13: `set bd_files [get_files -quiet system.bd]` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are set: the value named 'set' in this block; bd_files: the value named 'bd files' in this block; get_files: the value named 'get files' in this block.

Line 14: `if {[llength $bd_files] != 1} {` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are llength: the value named 'llength' in this block; bd_files: the value named 'bd files' in this block.

Line 15: `error "Expected exactly one system.bd, found [llength $bd_files]"` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are error: the value named 'error' in this block; Expected: the value named 'expected' in this block; exactly: the value named 'exactly' in this block.

Line 16: `}` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 17: `open_bd_design [lindex $bd_files 0]` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are open_bd_design: the value named 'open bd design' in this block; lindex: an integer selecting one tensor element, lane, bank, or state entry; bd_files: the value named 'bd files' in this block.

Line 19: `set pdf_path [file join $output_dir "neuraedge_pynq_block_design.pdf"]` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are set: the value named 'set' in this block; pdf_path: the value named 'pdf path' in this block; file: the value named 'file' in this block.

Line 20: `set svg_path [file join $output_dir "neuraedge_pynq_block_design.svg"]` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are set: the value named 'set' in this block; svg_path: the value named 'svg path' in this block; file: the value named 'file' in this block.

Line 21: `write_bd_layout -force -format pdf -orientation landscape -scope all $pdf_path` This writes an auditable Vivado artifact using ‘write_bd_layout -force -format pdf -orientation landscape -scope all $pdf_path‘; later documentation reads the resulting report rather than inventing a metric.

Line 22: `write_bd_layout -force -format svg -orientation landscape -scope all $svg_path` This writes an auditable Vivado artifact using ‘write_bd_layout -force -format svg -orientation landscape -scope all $svg_path‘; later documentation reads the resulting report rather than inventing a metric.

Line 23: `foreach diagram [list $pdf_path $svg_path] {` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are foreach: the value named ’foreach’ in this block; diagram: the value named ’diagram’ in this block; list: the value named ’list’ in this block.

Line 24: `if {![file exists $diagram] || [file size $diagram] == 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are file: the value named ’file’ in this block; exists: the value named ’exists’ in this block; diagram: the value named ’diagram’ in this block.

Line 25: `error "Block-design diagram was not generated: $diagram"` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are error: the value named ’error’ in this block; Block: the value named ’block’ in this block; design: the value named ’design’ in this block.

Line 26: `}` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 27: `}` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 29: `puts "NEURAEDGE_PYNQ_DIAGRAMS_EXPORTED pdf=$pdf_path svg=$svg_path"` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are puts: the value named ’puts’ in this block; NEURAEDGE_PYNQ_DIAGRAMS_EXPORTED: the value named ’neuraedge pynq diagrams exported’ in this block; pdf: the value named ’pdf’ in this block.

Line 30: `close_project` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are close_project: the value named ’close project’ in this block.

Line 31: `exit 0` This batch-control line contributes to the flow that runs the reproducible Vivado export pynq diagrams hardware flow. The important operands are exit: the value named ’exit’ in this block.

95 scripts/finalize_pynq_ooc.tcl

Role in the complete system: runs the reproducible Vivado finalize pynq ooc hardware flow.

95.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado finalize pynq ooc hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

95.2 Complete source

```
# Shared final evidence and acceptance gates for fresh and resumed OOC builds.
# The caller must define output_dir and have an open, fully routed design.
report_utilization -hierarchical -file [file join $output_dir "routed_utilization_hierarchical.rpt"]
report_timing_summary -delay_type min_max -max_paths 20 \
    -file [file join $output_dir "routed_timing_summary.rpt"]
report_timing -delay_type max -max_paths 250 -sort_by group \
    -file [file join $output_dir "routed_timing_paths_250.rpt"]
report_clock_utilization -file [file join $output_dir "routed_clock_utilization.rpt"]
set route_status_text [report_route_status -return_string]
set route_file [open [file join $output_dir "routed_route_status.rpt"] "w"]
puts -nonewline $route_file $route_status_text
close $route_file
report_drc -file [file join $output_dir "routed_drc.rpt"]
report_methodology -file [file join $output_dir "routed_methodology.rpt"]
report_design_analysis -congestion \
    -file [file join $output_dir "routed_congestion.rpt"]
report_power -file [file join $output_dir "routed_vectorless_power.rpt"]
write_checkpoint -force [file join $output_dir "neuraedge_pynq_axi_top_routed.dcp"]

set lut_count [llength [get_cells -hierarchical -filter {REF_NAME =~ LUT*}]]
set ff_count [llength [get_cells -hierarchical -filter {REF_NAME =~ FD*}]]
set dsp_count [llength [get_cells -hierarchical -filter {REF_NAME =~ DSP48*}]]
set bram36_count [llength [get_cells -hierarchical -filter {REF_NAME == RAMB36E1}]]
set bram18_count [llength [get_cells -hierarchical -filter {REF_NAME == RAMB18E1}]]
set setup_path [get_timing_paths -delay_type max -max_paths 1]
set hold_path [get_timing_paths -delay_type min -max_paths 1]
set setup_wns [get_property SLACK $setup_path]
set hold_whs [get_property SLACK $hold_path]
set route_error_count -1
if {[regexp {# of nets with routing errors\.*\s*:\s*([0-9]+)} \
        $route_status_text route_match route_count]} {
    set route_error_count $route_count
}
set drc_error_count [llength [get_drc_violations -quiet -filter {SEVERITY == Error}]]

set summary_file [open [file join $output_dir "implementation_summary.txt"] "w"]
puts $summary_file "part=xc7z020clg400-1"
puts $summary_file "target_clock_mhz=200.0"
puts $summary_file "lut_count=$lut_count"
puts $summary_file "ff_count=$ff_count"
puts $summary_file "dsp48_count=$dsp_count"
puts $summary_file "bram36_count=$bram36_count"
puts $summary_file "bram18_count=$bram18_count"
puts $summary_file "setup_wns_ns=$setup_wns"
puts $summary_file "hold_whs_ns=$hold_whs"
puts $summary_file "route_error_count=$route_error_count"
puts $summary_file "drc_error_count=$drc_error_count"
puts $summary_file "hold_context=ooc_top_ports_without_hd_partpin_locs"
close $summary_file

if {$route_error_count != 0} {
    error "Routed design is incomplete: route_error_count=$route_error_count"
}
if {$drc_error_count != 0} {
    error "Routed design has DRC errors: drc_error_count=$drc_error_count"
}
if {$setup_wns < 0.0} {
    error "Routed design missed the 5.0 ns target: WNS=$setup_wns ns"
}
# Min-delay paths at OOC AXI/stream ports are inaccurate without physical
```

```
# partition pins. Preserve the diagnostic and enforce hold in the full shell.
if {$hold_whs < 0.0} {
    puts "NEURAEDGE_OOC_HOLD_CONTEXT_ONLY WHS=$hold_whs ns"
}
```

95.3 Line annotations

Line 1: `# Shared final evidence and acceptance gates for fresh and resumed OOC builds.` This comment documents the batch-flow intent: Shared final evidence and acceptance gates for fresh and resumed OOC builds.

Line 2: `# The caller must define output_dir and have an open, fully routed design.` This comment documents the batch-flow intent: The caller must define output_dir and have an open, fully routed design.

Line 3: `report_utilization -hierarchical -file [file join $output_dir "routed_utilization_hierarchical.rpt"]` This writes an auditable Vivado artifact using ‘report_utilization -hierarchical -file [file join $output_dir "routed_utilization_hierarc later documentation reads the resulting report rather than inventing a metric.

Line 4: `report_timing_summary -delay_type min_max -max_paths 20 \` This writes an auditable Vivado artifact using ‘report_timing_summary -delay_type min_max -max_paths 20 \‘; later documentation reads the resulting report rather than inventing a metric.

Line 5: `-file [file join $output_dir "routed_timing_summary.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are file: the value named ’file’ in this block; join: the value named ’join’ in this block; output_dir: the value named ’output dir’ in this block.

Line 6: `report_timing -delay_type max -max_paths 250 -sort_by group \` This writes an auditable Vivado artifact using ‘report_timing -delay_type max -max_paths 250 -sort_by group \‘; later documentation reads the resulting report rather than inventing a metric.

Line 7: `-file [file join $output_dir "routed_timing_paths_250.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are file: the value named ’file’ in this block; join: the value named ’join’ in this block; output_dir: the value named ’output dir’ in this block.

Line 8: `report_clock_utilization -file [file join $output_dir "routed_clock_utilization.rpt"]` This writes an auditable Vivado artifact using ‘report_clock_utilization -file [file join $output_dir "routed_clock_utilization.rpt"]‘; later documentation reads the resulting report rather than inventing a metric.

Line 9: `set route_status_text [report_route_status -return_string]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named ’set’ in this block; route_status_text: the value named ’route status text’ in this block; report_route_status: the value named ’report route status’ in this block.

Line 10: `set route_file [open [file join $output_dir "routed_route_status.rpt"] "w"]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named ’set’ in this block; route_file: the value named ’route file’ in this block; open: the value named ’open’ in this block.

Line 11: `puts -nonewline $route_file $route_status_text` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named ’puts’ in this block; nonewline: the value named ’nonewline’ in this block; route_file: the value named ’route file’ in this block.

Line 12: `close $route_file` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are close: the value named ’close’ in this block; route_file: the value named ’route file’ in this block.

Line 13: `report_drc -file [file join $output_dir "routed_drc.rpt"]` This writes an auditable Vivado artifact using ‘report_drc -file [file join $output_dir "routed_drc.rpt"]‘; later documentation reads the resulting report rather than inventing a metric.

Line 14: `report_methodology -file [file join $output_dir "routed_methodology.rpt"]` This writes an auditable Vivado artifact using ‘report_methodology -file [file join $output_dir "routed_methodology.rpt"]‘; later documentation reads the resulting report rather than inventing a metric.

Line 15: `report_design_analysis -congestion \` This writes an auditable Vivado artifact using ‘report_design_analysis -congestion \‘; later documentation reads the resulting report rather than inventing a metric.

Line 16: `-file [file join $output_dir "routed_congestion.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are file: the value named ’file’ in this block; join: the value named ’join’ in this block; output_dir: the value named ’output dir’ in this block.

Line 17: `report_power -file [file join $output_dir "routed_vectorless_power.rpt"]` This writes an auditable Vivado artifact using ‘report_power -file [file join $output_dir "routed_vectorless_power.rpt"]‘; later documentation reads the resulting report rather than inventing a metric.

Line 18: `write_checkpoint -force [file join $output_dir "neuraedge_pynq_axi_top_routed.dcp"]` This writes an auditable Vivado artifact using ‘write_checkpoint -force [file join $output_dir "neuraedge_pynq_axi_top_routed.dcp"]‘; later documentation reads the resulting report rather than inventing a metric.

Line 20: `set lut_count [llength [get_cells -hierarchical -filter {REF_NAME =~ LUT*}]]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named ’set’ in this block; lut_count: an integer count retained for control or measurable telemetry; llength: the value named ’llength’ in this block.

Line 21: `set ff_count [llength [get_cells -hierarchical -filter {REF_NAME =~ FD*}]]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named ’set’ in this block; ff_count: an integer count retained for control or measurable telemetry; llength: the value named ’llength’ in this block.

Line 22: `set dsp_count [llength [get_cells -hierarchical -filter {REF_NAME =~ DSP48*}]]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named ’set’ in this block; dsp_count: an integer count retained for control or measurable telemetry; llength: the value named ’llength’ in this block.

Line 23: `set bram36_count [llength [get_cells -hierarchical -filter {REF_NAME == RAMB36E1}]]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named ’set’ in this block; bram36_count: an integer count retained for control or measurable telemetry; llength: the value named ’llength’ in this block.

Line 24: `set bram18_count [llength [get_cells -hierarchical -filter {REF_NAME == RAMB18E1}]]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named ’set’ in this block; bram18_count: an integer count retained for control or measurable telemetry; llength: the value named ’llength’ in this block.

Line 25: `set setup_path [get_timing_paths -delay_type max -max_paths 1]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named ’set’ in this block; setup_path: the value named ’setup path’ in this block; get_timing_paths: the value named ’get timing paths’ in this block.

Line 26: `set hold_path [get_timing_paths -delay_type min -max_paths 1]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; hold_path: the value named 'hold path' in this block; get_timing_paths: the value named 'get timing paths' in this block.

Line 27: `set setup_wns [get_property SLACK $setup_path]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; setup_wns: the value named 'setup wns' in this block; get_property: the value named 'get property' in this block.

Line 28: `set hold_whs [get_property SLACK $hold_path]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; hold_whs: the value named 'hold whs' in this block; get_property: the value named 'get property' in this block.

Line 29: `set route_error_count -1` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; route_error_count: an integer count retained for control or measurable telemetry.

Line 30: `if {[regexp {# of nets with routing errors\.*\s*:\s*([0-9]+)} \` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are regexp: the value named 'regexp' in this block; of: the value named 'of' in this block; nets: the value named 'nets' in this block.

Line 31: `$route_status_text route_match route_count]} {` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are route_status_text: the value named 'route status text' in this block; route_match: the value named 'route match' in this block; route_count: an integer count retained for control or measurable telemetry.

Line 32: `set route_error_count $route_count` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; route_error_count: an integer count retained for control or measurable telemetry; route_count: an integer count retained for control or measurable telemetry.

Line 33: `}` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 34: `set drc_error_count [llength [get_drc_violations -quiet -filter {SEVERITY == Error}]]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; drc_error_count: an integer count retained for control or measurable telemetry; llength: the value named 'llength' in this block.

Line 36: `set summary_file [open [file join $output_dir "implementation_summary.txt"] "w"]` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; summary_file: the value named 'summary file' in this block; open: the value named 'open' in this block.

Line 37: `puts $summary_file "part=xc7z020clg400-1"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; part: the value named 'part' in this block.

Line 38: `puts $summary_file "target_clock_mhz=200.0"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; target_clock_mhz: the value named 'target clock mhz' in this block.

Line 39: `puts $summary_file "lut_count=$lut_count"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; lut_count: an integer count retained for control or measurable telemetry.

Line 40: `puts $summary_file "ff_count=$ff_count"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; ff_count: an integer count retained for control or measurable telemetry.

Line 41: `puts $summary_file "dsp48_count=$dsp_count"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; dsp48_count: an integer count retained for control or measurable telemetry.

Line 42: `puts $summary_file "bram36_count=$bram36_count"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; bram36_count: an integer count retained for control or measurable telemetry.

Line 43: `puts $summary_file "bram18_count=$bram18_count"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; bram18_count: an integer count retained for control or measurable telemetry.

Line 44: `puts $summary_file "setup_wns_ns=$setup_wns"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; setup_wns_ns: the value named 'setup wns ns' in this block.

Line 45: `puts $summary_file "hold_whs_ns=$hold_whs"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; hold_whs_ns: the value named 'hold whs ns' in this block.

Line 46: `puts $summary_file "route_error_count=$route_error_count"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; route_error_count: an integer count retained for control or measurable telemetry.

Line 47: `puts $summary_file "drc_error_count=$drc_error_count"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; drc_error_count: an integer count retained for control or measurable telemetry.

Line 48: `puts $summary_file "hold_context=ooc_top_ports_without_hd_partpin_locs"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; summary_file: the value named 'summary file' in this block; hold_context: the value named 'hold context' in this block.

Line 49: `close $summary_file` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are close: the value named 'close' in this block; summary_file: the value named 'summary file' in this block.

Line 51: `if {$route_error_count != 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are route_error_count: an integer count retained for control or measurable telemetry.

Line 52: `error "Routed design is incomplete: route_error_count=$route_error_count"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are error: the value named 'error' in this block; Routed: the value named 'routed' in this block; design: the value named 'design' in this block.

Line 53: `}` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 54: `if {$drc_error_count != 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are drc_error_count: an integer count retained for control or measurable telemetry.

Line 55: `error "Routed design has DRC errors: drc_error_count=$drc_error_count"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are error: the value named 'error' in this block; Routed: the value named 'routed' in this block; design: the value named 'design' in this block.

Line 56: `}` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 57: `if {$setup_wns < 0.0} {` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are setup_wns: the value named 'setup wns' in this block.

Line 58: `error "Routed design missed the 5.0 ns target: WNS=$setup_wns ns"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are error: the value named 'error' in this block; Routed: the value named 'routed' in this block; design: the value named 'design' in this block.

Line 59: `}` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 60: `# Min-delay paths at OOC AXI/stream ports are inaccurate without physical` This comment documents the batch-flow intent: Min-delay paths at OOC AXI/stream ports are inaccurate without physical

Line 61: `# partition pins. Preserve the diagnostic and enforce hold in the full shell.` This comment documents the batch-flow intent: partition pins. Preserve the diagnostic and enforce hold in the full shell.

Line 62: `if {$hold_whs < 0.0} {` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are hold_whs: the value named 'hold whs' in this block.

Line 63: `puts "NEURAEDGE_OOC_HOLD_CONTEXT_ONLY WHS=$hold_whs ns"` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The important operands are puts: the value named 'puts' in this block; NEURAEDGE_OOC_HOLD_CONTEXT_ONLY: the value named 'neuraedge ooc hold context only' in this block; WHS: the value named 'whs' in this block.

Line 64: `}` This batch-control line contributes to the flow that runs the reproducible Vivado finalize pynq ooc hardware flow. The expression is a literal or structural operator and has no hidden software state.

96 scripts/report_checkpoint_hierarchy.tcl

Role in the complete system: runs the reproducible Vivado report checkpoint hierarchy hardware flow.

96.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado report checkpoint hierarchy hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

96.2 Complete source

```
# Report module-level area and control-set pressure from any synthesized DCP.
if {$argc != 2} {
    error "Usage: report_checkpoint_hierarchy.tcl <checkpoint.dcp> <output_dir>"
}

set checkpoint [file normalize [lindex $argv 0]]
set output_dir [file normalize [lindex $argv 1]]
if {![file exists $checkpoint]} {
    error "Checkpoint does not exist: $checkpoint"
}
file mkdir $output_dir

open_checkpoint $checkpoint
report_utilization -hierarchical -hierarchical_depth 8 \
    -file [file join $output_dir "hierarchical_utilization.rpt"]
report_control_sets -verbose \
    -file [file join $output_dir "control_sets.rpt"]
report_design_analysis -complexity -congestion \
    -file [file join $output_dir "design_complexity_congestion.rpt"]
puts "NEURAEDGE_CHECKPOINT_HIERARCHY_REPORTED output=$output_dir"
close_design
exit 0
```

96.3 Line annotations

Line 1: `# Report module-level area and control-set pressure from any synthesized DCP.` This comment documents the batch-flow intent: Report module-level area and control-set pressure from any synthesized DCP.

Line 2: `if {$argc != 2} {` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are argc: the value named 'argc' in this block.

Line 3: `error "Usage: report_checkpoint_hierarchy.tcl <checkpoint.dcp> <output_dir>"` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are error: the value named 'error' in this block; Usage: the value named 'usage' in this block; report_checkpoint_hierarchy: the value named 'report checkpoint hierarchy' in this block.

Line 4: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 6: `set checkpoint [file normalize [lindex $argv 0]]` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are set: the value named 'set' in this block; checkpoint: the value named 'checkpoint' in this block; file: the value named 'file' in this block.

Line 7: `set output_dir [file normalize [lindex $argv 1]]` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are set: the value named 'set' in this block; output_dir: the value named 'output dir' in this block; file: the value named 'file' in this block.

Line 8: `if {![file exists $checkpoint]} {` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are file: the value named 'file' in this block; exists: the value named 'exists' in this block; checkpoint: the value named 'checkpoint' in this block.

Line 9: `error "Checkpoint does not exist: $checkpoint"` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are error: the value named 'error' in this block; Checkpoint: the value named 'checkpoint' in this block; does: the value named 'does' in this block.

Line 10: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 11: `file mkdir $output_dir` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are file: the value named 'file' in this block; mkdir: the value named 'mkdir' in this block; output_dir: the value named 'output dir' in this block.

Line 13: `open_checkpoint $checkpoint` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are open_checkpoint: the value named 'open checkpoint' in this block; checkpoint: the value named 'checkpoint' in this block.

Line 14: `report_utilization -hierarchical -hierarchical_depth 8 \` This writes an auditable Vivado artifact using 'report_utilization -hierarchical -hierarchical_depth 8 \'; later documentation reads the resulting report rather than inventing a metric.

Line 15: `-file [file join $output_dir "hierarchical_utilization.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 16: `report_control_sets -verbose \` This writes an auditable Vivado artifact using 'report_control_sets -verbose \'; later documentation reads the resulting report rather than inventing a metric.

Line 17: `-file [file join $output_dir "control_sets.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 18: `report_design_analysis -complexity -congestion \` This writes an auditable Vivado artifact using 'report_design_analysis -complexity -congestion \'; later documentation reads the resulting report rather than inventing a metric.

Line 19: `-file [file join $output_dir "design_complexity_congestion.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 20: `puts "NEURAEDGE_CHECKPOINT_HIERARCHY_REPORTED output=$output_dir"` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are puts: the value named 'puts' in this block; NEURAEDGE_CHECKPOINT_HIERARCHY_REPORTED: the value named 'neuraedge checkpoint hierarchy reported' in this block; output: the value named 'output' in this block.

Line 21: `close_design` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are close_design: the value named 'close design' in this block.

Line 22: `exit 0` This batch-control line contributes to the flow that runs the reproducible Vivado report checkpoint hierarchy hardware flow. The important operands are exit: the value named 'exit' in this block.

97 scripts/report_optimizer_dsp.tcl

Role in the complete system: runs the reproducible Vivado report optimizer dsp hardware flow.

97.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado report optimizer dsp hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

97.2 Complete source

```
# Inspect the implemented optimizer multipliers without modifying the checkpoint.
if {$argc != 2} {
    error "Usage: report_optimizer_dsp.tcl <checkpoint.dcp> <output_report>"
}

set checkpoint [file normalize [lindex $argv 0]]
set output_report [file normalize [lindex $argv 1]]
if {![file exists $checkpoint]} {
    error "Checkpoint does not exist: $checkpoint"
}
file mkdir [file dirname $output_report]

open_checkpoint $checkpoint
set optimizer_dsps [get_cells -hierarchical -filter {
    REF_NAME == DSP48E1 && NAME =~ *u_optimizer*
}]

set report_handle [open $output_report w]
puts $report_handle "checkpoint=$checkpoint"
puts $report_handle "optimizer_dsp_count=[llength $optimizer_dsps]"
foreach dsp $optimizer_dsps {
    puts $report_handle "\n=== [get_property NAME $dsp] ==="
    foreach property [lsort [list_property $dsp]] {
        puts $report_handle "$property=[get_property $property $dsp]"
    }
    foreach pin_name {A B C D M P CLK CEA1 CEA2 CEB1 CEB2 CEM CEP RSTA RSTB RSTM RSTP} {
        set pins [get_pins -quiet "$dsp/$pin_name*"]
        if {[llength $pins] == 0} {
            continue
        }
        foreach pin $pins {
            set nets [get_nets -quiet -of_objects $pin]
            if {[llength $nets] > 0} {
                puts $report_handle \
                    "PIN [get_property NAME $pin] NET [join [get_property NAME $nets] {, }]"
            }
        }
    }
}
close $report_handle

puts "NEURAEDGE_OPTIMIZER_DSP_REPORTED count=[llength $optimizer_dsps] report=$output_report"
close_design
exit 0
```

97.3 Line annotations

Line 1: `# Inspect the implemented optimizer multipliers without modifying the checkpoint.` This comment documents the batch-flow intent: Inspect the implemented optimizer multipliers without modifying the checkpoint.

Line 2: `if {$argc != 2} {` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are argc: the value named 'argc' in this block.

Line 3: `error "Usage: report_optimizer_dsp.tcl <checkpoint.dcp> <output_report>"` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are error: the value named 'error' in this block; Usage: the value named 'usage' in this block; report_optimizer_dsp: the value named 'report optimizer dsp' in this block.

Line 4: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 6: `set checkpoint [file normalize [lindex $argv 0]]` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are set: the value named 'set' in this block; checkpoint: the value named 'checkpoint' in this block; file: the value named 'file' in this block.

Line 7: `set output_report [file normalize [lindex $argv 1]]` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are set: the value named 'set' in this block; output_report: the value named 'output report' in this block; file: the value named 'file' in this block.

Line 8: `if {![file exists $checkpoint]} {` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are file: the value named 'file' in this block; exists: the value named 'exists' in this block; checkpoint: the value named 'checkpoint' in this block.

Line 9: `error "Checkpoint does not exist: $checkpoint"` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are error: the value named 'error' in this block; Checkpoint: the value named 'checkpoint' in this block; does: the value named 'does' in this block.

Line 10: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 11: `file mkdir [file dirname $output_report]` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are file: the value named 'file' in this block; mkdir: the value named 'mkdir' in this block; dirname: the value named 'dirname' in this block.

Line 13: `open_checkpoint $checkpoint` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are open_checkpoint: the value named 'open checkpoint' in this block; checkpoint: the value named 'checkpoint' in this block.

Line 14: `set optimizer_dsps [get_cells -hierarchical -filter {` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are set: the value named 'set' in this block; optimizer_dsps: the value named 'optimizer dsps' in this block; get_cells: the value named 'get cells' in this block.

Line 15: `REF_NAME == DSP48E1 && NAME =~ *u_optimizer*` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are REF_NAME: the value named 'ref name' in this block; DSP48E1: the value named 'dsp48 e1' in this block; NAME: the value named 'name' in this block.

Line 16: `}]` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 18: `set report_handle [open $output_report w]` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are set: the value named 'set' in this block; report_handle: the value named 'report handle' in this block; open: the value named 'open' in this block.

Line 19: `puts $report_handle "checkpoint=$checkpoint"` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are puts: the value named 'puts' in this block; report_handle: the value named 'report handle' in this block; checkpoint: the value named 'checkpoint' in this block.

Line 20: `puts $report_handle "optimizer_dsp_count=[llength $optimizer_dsps]"` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are puts: the value named 'puts' in this block; report_handle: the value named 'report handle' in this block; optimizer_dsp_count: an integer count retained for control or measurable telemetry.

Line 21: `foreach dsp $optimizer_dsps {` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are foreach: the value named 'foreach' in this block; dsp: the value named 'dsp' in this block; optimizer_dsps: the value named 'optimizer dsps' in this block.

Line 22: `puts $report_handle "\n=== [get_property NAME $dsp] ==="` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are puts: the value named 'puts' in this block; report_handle: the value named 'report handle' in this block; n: the value named 'n' in this block.

Line 23: `foreach property [lsort [list_property $dsp]] {` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are foreach: the value named 'foreach' in this block; property: the value named 'property' in this block; lsort: the value named 'lsort' in this block.

Line 24: `puts $report_handle "$property=[get_property $property $dsp]"` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are puts: the value named 'puts' in this block; report_handle: the value named 'report handle' in this block; property: the value named 'property' in this block.

Line 25: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 26: `foreach pin_name {A B C D M P CLK CEA1 CEA2 CEB1 CEB2 CEM CEP RSTA RSTB RSTM RSTP} {` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are foreach: the value named 'foreach' in this block; pin_name: the value named 'pin name' in this block; A: the value named 'a' in this block.

Line 27: `set pins [get_pins -quiet "$dsp/$pin_name*"]` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are set: the value named 'set' in this block; pins: the value named 'pins' in this block; get_pins: the value named 'get pins' in this block.

Line 28: `if {[llength $pins] == 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are llength: the value named 'llength' in this block; pins: the value named 'pins' in this block.

Line 29: `continue` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are continue: the value named 'continue' in this block.

Line 30: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 31: `foreach pin $pins {` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are foreach: the value named 'foreach' in this block; pin: the value named 'pin' in this block; pins: the value named 'pins' in this block.

Line 32: `set nets [get_nets -quiet -of_objects $pin]` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are set: the value named 'set' in this block; nets: the value named 'nets' in this block; get_nets: the value named 'get nets' in this block.

Line 33: `if {[llength $nets] > 0} {` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are llength: the value named 'llength' in this block; nets: the value named 'nets' in this block.

Line 34: `puts $report_handle \` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are puts: the value named 'puts' in this block; report_handle: the value named 'report handle' in this block.

Line 35: `"PIN [get_property NAME $pin] NET [join [get_property NAME $nets] {, }]"` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are PIN: the value named 'pin' in this block; get_property: the value named 'get property' in this block; NAME: the value named 'name' in this block.

Line 36: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 37: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 38: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 39: `}` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 40: `close $report_handle` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are close: the value named 'close' in this block; report_handle: the value named 'report handle' in this block.

Line 42: `puts "NEURAEDGE_OPTIMIZER_DSP_REPORTED count=[llength $optimizer_dsps] report=$output_report"` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are puts: the value named 'puts' in this block; NEURAEDGE_OPTIMIZER_DSP_REPORTED: the value named 'neuraedge optimizer dsp reported' in this block; count: the value named 'count' in this block.

Line 43: `close_design` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are close_design: the value named 'close design' in this block.

Line 44: `exit 0` This batch-control line contributes to the flow that runs the reproducible Vivado report optimizer dsp hardware flow. The important operands are exit: the value named 'exit' in this block.

98 scripts/resume_pynq_ooc.tcl

Role in the complete system: runs the reproducible Vivado resume pynq ooc hardware flow.

98.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado resume pynq ooc hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

98.2 Complete source

```
set script_dir [file normalize [file dirname [info script]]]
set project_dir [file normalize [file join $script_dir ".."]]
set output_name "pynq_ooc"
set directive "Explore"
if {$argc >= 1} {
    set output_name [lindex $argv 0]
}
if {$argc >= 2} {
    set directive [lindex $argv 1]
}
set output_dir [file join $project_dir "outputs" $output_name]
set checkpoint [file join $output_dir "pre_postroute_physopt.dcp"]

if {![file exists $checkpoint]} {
    error "Routed resume checkpoint does not exist: $checkpoint"
}

open_checkpoint $checkpoint
phys_opt_design -directive $directive
write_checkpoint -force [file join $output_dir "postroute_physopt_unrouted.dcp"]
route_design -directive Quick
source [file join $script_dir "finalize_pynq_ooc.tcl"]
```

98.3 Line annotations

Line 1: `set script_dir [file normalize [file dirname [info script]]]` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; script_dir: the value named 'script dir' in this block; file: the value named 'file' in this block.

Line 2: `set project_dir [file normalize [file join $script_dir ".."]]` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; project_dir: the value named 'project dir' in this block; file: the value named 'file' in this block.

Line 3: `set output_name "pynq_ooc"` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; output_name: the value named 'output name' in this block; pynq_ooc: the value named 'pynq ooc' in this block.

Line 4: `set directive "Explore"` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; directive: the value named 'directive' in this block; Explore: the value named 'explore' in this block.

Line 5: `if {$argc >= 1} {` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are argc: the value named 'argc' in this block.

Line 6: `set output_name [lindex $argv 0]` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; output_name: the value named 'output name' in this block; lindex: an integer selecting one tensor element, lane, bank, or state entry.

Line 7: `}` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 8: `if {$argc >= 2} {` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are argc: the value named 'argc' in this block.

Line 9: `set directive [lindex $argv 1]` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; directive: the value named 'directive' in this block; lindex: an integer selecting one tensor element, lane, bank, or state entry.

Line 10: `}` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 11: `set output_dir [file join $project_dir "outputs" $output_name]` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; output_dir: the value named 'output dir' in this block; file: the value named 'file' in this block.

Line 12: `set checkpoint [file join $output_dir "pre_postroute_physopt.dcp"]` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are set: the value named 'set' in this block; checkpoint: the value named 'checkpoint' in this block; file: the value named 'file' in this block.

Line 14: `if {![file exists $checkpoint]} {` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are file: the value named 'file' in this block; exists: the value named 'exists' in this block; checkpoint: the value named 'checkpoint' in this block.

Line 15: `error "Routed resume checkpoint does not exist: $checkpoint"` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are error: the value named 'error' in this block; Routed: the value named 'routed' in this block; resume: the value named 'resume' in this block.

Line 16: `}` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 18: `open_checkpoint $checkpoint` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are open_checkpoint: the value named 'open checkpoint' in this block; checkpoint: the value named 'checkpoint' in this block.

Line 19: `phys_opt_design -directive $directive` This runs Vivado implementation command 'phys_opt_design -directive $directive', changing the physical netlist or placement/routing needed for real timing.

Line 20: `write_checkpoint -force [file join $output_dir "postroute_physopt_unrouted.dcp"]` This writes an auditable Vivado artifact using 'write_checkpoint -force [file join $output_dir "postroute_physopt_unrouted.dcp"]'; later documentation reads the resulting report rather than inventing a metric.

Line 21: `route_design -directive Quick` This runs Vivado implementation command 'route_design -directive Quick', changing the physical netlist or placement/routing needed for real timing.

Line 22: `source [file join $script_dir "finalize_pynq_ooc.tcl"]` This batch-control line contributes to the flow that runs the reproducible Vivado resume pynq ooc hardware flow. The important operands are source: the value named 'source' in this block; file: the value named 'file' in this block; join: the value named 'join' in this block.

99 scripts/synth_mac2_probe.tcl

Role in the complete system: runs the reproducible Vivado synth mac2 probe hardware flow.

99.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado synth mac2 probe hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

99.2 Complete source

```
set script_dir [file normalize [file dirname [info script]]]
set project_dir [file normalize [file join $script_dir ".."]]
set out_dir [file join $project_dir "outputs" "mac2_probe"]
file mkdir $out_dir

read_verilog -sv [file join $project_dir "rtl" "neuraedge_mac2_u8.sv"]
read_verilog -sv [file join $project_dir "rtl" "neuraedge_mac2_array_probe.sv"]
synth_design -top neuraedge_mac2_array_probe -part xc7z020clg400-1 -generic PAIRS=128
create_clock -period 5.0 -name clk [get_ports clk]

opt_design
place_design
route_design

report_utilization -file [file join $out_dir "mac2_probe_utilization.rpt"]
report_timing_summary -file [file join $out_dir "mac2_probe_timing.rpt"]
report_drc -file [file join $out_dir "mac2_probe_drc.rpt"]
write_checkpoint -force [file join $out_dir "mac2_probe_routed.dcp"]

set dsp_count [llength [get_cells -hierarchical -filter {REF_NAME =~ DSP48*}]]
set wns [get_property SLACK [get_timing_paths -max_paths 1 -setup]]
set fh [open [file join $out_dir "mac2_probe_summary.txt"] "w"]
puts $fh "pairs=128"
puts $fh "products_per_cycle=256"
puts $fh "dsp48_count=$dsp_count"
puts $fh "wns_ns=$wns"
close $fh

if {$dsp_count != 128} {
    error "Packed MAC probe expected exactly 128 DSP48 cells, got $dsp_count"
}
if {$wns < 0.0} {
    error "Packed MAC probe failed the 5.0 ns timing target: WNS=$wns"
}
```

99.3 Line annotations

Line 1: `set script_dir [file normalize [file dirname [info script]]]` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are set: the value named 'set' in this block; script_dir: the value named 'script dir' in this block; file: the value named 'file' in this block.

Line 2: `set project_dir [file normalize [file join $script_dir ".."]]` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are set: the value named 'set' in this block; project_dir: the value named 'project dir' in this block; file: the value named 'file' in this block.

Line 3: `set out_dir [file join $project_dir "outputs" "mac2_probe"]` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are set: the value named 'set' in this block; out_dir: the value named 'out dir' in this block; file: the value named 'file' in this block.

Line 4: `file mkdir $out_dir` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are file: the value named 'file' in this block; mkdir: the value named 'mkdir' in this block; out_dir: the value named 'out dir' in this block.

Line 6: `read_verilog -sv [file join $project_dir "rtl" "neuraedge_mac2_u8.sv"]` This loads the listed RTL into Vivado's in-memory design so the subsequent synthesis uses the repository source.

Line 7: `read_verilog -sv [file join $project_dir "rtl" "neuraedge_mac2_array_probe.sv"]` This loads the listed RTL into Vivado's in-memory design so the subsequent synthesis uses the repository source.

Line 8: `synth_design -top neuraedge_mac2_array_probe -part xc7z020clg400-1 -generic PAIRS=128` This invokes Vivado synthesis for the named top and FPGA part, mapping behavioral RTL into LUT, flip-flop, BRAM, and DSP primitives.

Line 9: `create_clock -period 5.0 -name clk [get_ports clk]` This declares the clock period against which Vivado checks every synchronous setup and hold path.

Line 11: `opt_design` This runs Vivado implementation command 'opt_design', changing the physical netlist or placement/routing needed for real timing.

Line 12: `place_design` This runs Vivado implementation command 'place_design', changing the physical netlist or placement/routing needed for real timing.

Line 13: `route_design` This runs Vivado implementation command 'route_design', changing the physical netlist or placement/routing needed for real timing.

Line 15: `report_utilization -file [file join $out_dir "mac2_probe_utilization.rpt"]` This writes an auditable Vivado artifact using 'report_utilization -file [file join $out_dir "mac2_probe_utilization.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 16: `report_timing_summary -file [file join $out_dir "mac2_probe_timing.rpt"]` This writes an auditable Vivado artifact using 'report_timing_summary -file [file join $out_dir "mac2_probe_timing.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 17: `report_drc -file [file join $out_dir "mac2_probe_drc.rpt"]` This writes an auditable Vivado artifact using 'report_drc -file [file join $out_dir "mac2_probe_drc.rpt"]'; later documentation reads the resulting report rather than inventing a metric.

Line 18: `write_checkpoint -force [file join $out_dir "mac2_probe_routed.dcp"]` This writes an auditable Vivado artifact using 'write_checkpoint -force [file join $out_dir "mac2_probe_routed.dcp"]'; later documentation reads the resulting report rather than inventing a metric.

Line 20: `set dsp_count [llength [get_cells -hierarchical -filter {REF_NAME =~ DSP48*}]]` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are set: the value named 'set' in this block; dsp_count: an integer count retained for control or measurable telemetry; llength: the value named 'llength' in this block.

Line 21: `set wns [get_property SLACK [get_timing_paths -max_paths 1 -setup]]` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are set: the value named 'set' in this block; wns: the value named 'wns' in this block; get_property: the value named 'get property' in this block.

Line 22: `set fh [open [file join $out_dir "mac2_probe_summary.txt"] "w"]` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are set: the value named 'set' in this block; fh: the value named 'fh' in this block; open: the value named 'open' in this block.

Line 23: `puts $fh "pairs=128"` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are puts: the value named 'puts' in this block; fh: the value named 'fh' in this block; pairs: the value named 'pairs' in this block.

Line 24: `puts $fh "products_per_cycle=256"` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are puts: the value named 'puts' in this block; fh: the value named 'fh' in this block; products_per_cycle: the value named 'products per cycle' in this block.

Line 25: `puts $fh "dsp48_count=$dsp_count"` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are puts: the value named 'puts' in this block; fh: the value named 'fh' in this block; dsp48_count: an integer count retained for control or measurable telemetry.

Line 26: `puts $fh "wns_ns=$wns"` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are puts: the value named 'puts' in this block; fh: the value named 'fh' in this block; wns_ns: the value named 'wns ns' in this block.

Line 27: `close $fh` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are close: the value named 'close' in this block; fh: the value named 'fh' in this block.

Line 29: `if {$dsp_count != 128} {` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are dsp_count: an integer count retained for control or measurable telemetry.

Line 30: `error "Packed MAC probe expected exactly 128 DSP48 cells, got $dsp_count"` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are error: the value named 'error' in this block; Packed: the value named 'packed' in this block; MAC: the value named 'mac' in this block.

Line 31: `}` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 32: `if {$wns < 0.0} {` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are wns: the value named 'wns' in this block.

Line 33: `error "Packed MAC probe failed the 5.0 ns timing target: WNS=$wns"` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The important operands are error: the value named 'error' in this block; Packed: the value named 'packed' in this block; MAC: the value named 'mac' in this block.

Line 34: `}` This batch-control line contributes to the flow that runs the reproducible Vivado synth mac2 probe hardware flow. The expression is a literal or structural operator and has no hidden software state.

100 scripts/synth_timing_iteration.tcl

Role in the complete system: runs the reproducible Vivado synth timing iteration hardware flow.

100.1 Beginner architecture map

Dataflow responsibility This file's end-to-end responsibility is to runs the reproducible Vivado synth timing iteration hardware flow.

Where it runs This is Vivado Tcl orchestration, not accelerator datapath hardware. It makes tool inputs, versions, paths, stages, and failure propagation repeatable.

Reproducibility contract Every external command must be checked, every required artifact must exist and be nonempty, and measured values must be read from generated reports. Rerunning this file should either reproduce its declared outputs or stop at the first failed gate.

100.2 Complete source

```
set script_dir [file normalize [file dirname [info script]]]
set project_dir [file normalize [file join $script_dir ".."]]
set output_name "timing_iteration"
if {$argc >= 1} {
    set output_name [lindex $argv 0]
}
set output_dir [file join $project_dir "outputs" $output_name]
file mkdir $output_dir

set rtl_files [list \
    [file join $project_dir "rtl" "neuraedge_command_pkg.sv"] \
    [file join $project_dir "rtl" "neuraedge_mac2_u8.sv"] \
    [file join $project_dir "rtl" "neuraedge_tensor_tile.sv"] \
    [file join $project_dir "rtl" "neuraedge_active_index_buffer.sv"] \
    [file join $project_dir "rtl" "neuraedge_activation_buffer.sv"] \
    [file join $project_dir "rtl" "neuraedge_weight_bank.sv"] \
    [file join $project_dir "rtl" "neuraedge_buffered_tensor_core.sv"] \
    [file join $project_dir "rtl" "neuraedge_alpha_xnor.sv"] \
    [file join $project_dir "rtl" "neuraedge_vector_alu.sv"] \
    [file join $project_dir "rtl" "neuraedge_udivider.sv"] \
    [file join $project_dir "rtl" "neuraedge_isqrt64.sv"] \
    [file join $project_dir "rtl" "neuraedge_integer_layernorm.sv"] \
    [file join $project_dir "rtl" "neuraedge_log_softmax.sv"] \
    [file join $project_dir "rtl" "neuraedge_lif_bank.sv"] \
    [file join $project_dir "rtl" "neuraedge_sgd_optimizer8.sv"] \
    [file join $project_dir "rtl" "neuraedge_gradient_accumulator_bank.sv"] \
    [file join $project_dir "rtl" "neuraedge_linear_backward8.sv"] \
    [file join $project_dir "rtl" "neuraedge_activation_lut8.sv"] \
    [file join $project_dir "rtl" "neuraedge_universal_core.sv"] \
    [file join $project_dir "rtl" "neuraedge_descriptor_sequencer.sv"] \
    [file join $project_dir "rtl" "neuraedge_pynq_axi_top.sv"]]

read_verilog -sv $rtl_files
synth_design -top neuraedge_pynq_axi_top -part xc7z020clg400-1 \
    -mode out_of_context -flatten_hierarchy rebuilt -directive PerformanceOptimized
create_clock -name aclk -period 5.000 [get_ports aclk]
set_clock_uncertainty 0.100 [get_clocks aclk]
set_property HD.CLK_SRC BUFGCTRL_X0Y0 [get_ports aclk]
set_false_path -from [get_ports aresetn]
set data_inputs [get_ports -filter {DIRECTION == IN && NAME != "aclk" && NAME != "aresetn"}]
set data_outputs [get_ports -filter {DIRECTION == OUT}]
set_input_delay -clock aclk 1.000 $data_inputs
set_output_delay -clock aclk 1.000 $data_outputs

report_utilization -hierarchical \
    -file [file join $output_dir "post_synth_utilization_hierarchical.rpt"]
report_timing -delay_type max -max_paths 250 -nworst 1 -input_pins \
    -file [file join $output_dir "post_synth_timing_paths_250.rpt"]
report_timing_summary -delay_type min_max -max_paths 20 \
    -file [file join $output_dir "post_synth_timing_summary.rpt"]
write_checkpoint -force [file join $output_dir "post_synth.dcp"]
```

100.3 Line annotations

Line 1: `set script_dir [file normalize [file dirname [info script]]]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set: the

value named 'set' in this block; script_dir: the value named 'script dir' in this block; file: the value named 'file' in this block.

Line 2: `set project_dir [file normalize [file join $script_dir ".."]]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set: the value named 'set' in this block; project_dir: the value named 'project dir' in this block; file: the value named 'file' in this block.

Line 3: `set output_name "timing_iteration"` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set: the value named 'set' in this block; output_name: the value named 'output name' in this block; timing_iteration: the value named 'timing iteration' in this block.

Line 4: `if {$argc >= 1} {` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are argc: the value named 'argc' in this block.

Line 5: `set output_name [lindex $argv 0]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set: the value named 'set' in this block; output_name: the value named 'output name' in this block; lindex: an integer selecting one tensor element, lane, bank, or state entry.

Line 6: `}` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The expression is a literal or structural operator and has no hidden software state.

Line 7: `set output_dir [file join $project_dir "outputs" $output_name]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set: the value named 'set' in this block; output_dir: the value named 'output dir' in this block; file: the value named 'file' in this block.

Line 8: `file mkdir $output_dir` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; mkdir: the value named 'mkdir' in this block; output_dir: the value named 'output dir' in this block.

Line 10: `set rtl_files [list \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set: the value named 'set' in this block; rtl_files: the value named 'rtl files' in this block; list: the value named 'list' in this block.

Line 11: `[file join $project_dir "rtl" "neuraedge_command_pkg.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 12: `[file join $project_dir "rtl" "neuraedge_mac2_u8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 13: `[file join $project_dir "rtl" "neuraedge_tensor_tile.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 14: `[file join $project_dir "rtl" "neuraedge_active_index_buffer.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 15: `[file join $project_dir "rtl" "neuraedge_activation_buffer.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 16: `[file join $project_dir "rtl" "neuraedge_weight_bank.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 17: `[file join $project_dir "rtl" "neuraedge_buffered_tensor_core.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 18: `[file join $project_dir "rtl" "neuraedge_alpha_xnor.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 19: `[file join $project_dir "rtl" "neuraedge_vector_alu.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 20: `[file join $project_dir "rtl" "neuraedge_udivider.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 21: `[file join $project_dir "rtl" "neuraedge_isqrt64.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 22: `[file join $project_dir "rtl" "neuraedge_integer_layernorm.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 23: `[file join $project_dir "rtl" "neuraedge_log_softmax.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 24: `[file join $project_dir "rtl" "neuraedge_lif_bank.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 25: `[file join $project_dir "rtl" "neuraedge_sgd_optimizer8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 26: `[file join $project_dir "rtl" "neuraedge_gradient_accumulator_bank.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 27: `[file join $project_dir "rtl" "neuraedge_linear_backward8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 28: `[file join $project_dir "rtl" "neuraedge_activation_lut8.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 29: `[file join $project_dir "rtl" "neuraedge_universal_core.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 30: `[file join $project_dir "rtl" "neuraedge_descriptor_sequencer.sv"] \` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 31: `[file join $project_dir "rtl" "neuraedge_pynq_axi_top.sv"]]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; project_dir: the value named 'project dir' in this block.

Line 33: `read_verilog -sv $rtl_files` This loads the listed RTL into Vivado's in-memory design so the subsequent synthesis uses the repository source.

Line 34: `synth_design -top neuraedge_pynq_axi_top -part xc7z020clg400-1 \` This invokes Vivado synthesis for the named top and FPGA part, mapping behavioral RTL into LUT, flip-flop, BRAM, and DSP primitives.

Line 35: `-mode out_of_context -flatten_hierarchy rebuilt -directive PerformanceOptimized` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are mode: the value named 'mode' in this block; out_of_context: the value named 'out of context' in this block; flatten_hierarchy: the value named 'flatten hierarchy' in this block.

Line 36: `create_clock -name aclk -period 5.000 [get_ports aclk]` This declares the clock period against which Vivado checks every synchronous setup and hold path.

Line 37: `set_clock_uncertainty 0.100 [get_clocks aclk]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set_clock_uncertainty: the value named 'set clock uncertainty' in this block; get_clocks: the value named 'get clocks' in this block; aclk: the AXI and accelerator clock.

Line 38: `set_property HD.CLK_SRC BUFGCTRL_X0Y0 [get_ports aclk]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set_property: the value named 'set property' in this block; HD: the value named 'hd' in this block; CLK_SRC: the clock that makes all registers advance together.

Line 39: `set_false_path -from [get_ports aresetn]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set_false_path: the value named 'set false path' in this block; from: the value named 'from' in this block; get_ports: the value named 'get ports' in this block.

Line 40: `set data_inputs [get_ports -filter {DIRECTION == IN && NAME != "aclk" && NAME != "aresetn"}]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set: the value named 'set' in this block; data_inputs: the value named 'data inputs' in this block; get_ports: the value named 'get ports' in this block.

Line 41: `set data_outputs [get_ports -filter {DIRECTION == OUT}]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set: the value named 'set' in this block; data_outputs: the value named 'data outputs' in this block; get_ports: the value named 'get ports' in this block.

Line 42: `set_input_delay -clock aclk 1.000 $data_inputs` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set_input_delay: the value named 'set input delay' in this block; clock: the value named 'clock' in this block; aclk: the AXI and accelerator clock.

Line 43: `set_output_delay -clock aclk 1.000 $data_outputs` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are set_output_delay: the value named 'set output delay' in this block; clock: the value named 'clock' in this block; aclk: the AXI and accelerator clock.

Line 45: `report_utilization -hierarchical \` This writes an auditable Vivado artifact using 'report_utilization -hierarchical \'; later documentation reads the resulting report rather than inventing a metric.

Line 46: `-file [file join $output_dir "post_synth_utilization_hierarchical.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 47: `report_timing -delay_type max -max_paths 250 -nworst 1 -input_pins \` This writes an auditable Vivado artifact using 'report_timing -delay_type max -max_paths 250 -nworst 1 -input_pins \'; later documentation reads the resulting report rather than inventing a metric.

Line 48: `-file [file join $output_dir "post_synth_timing_paths_250.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 49: `report_timing_summary -delay_type min_max -max_paths 20 \` This writes an auditable Vivado artifact using 'report_timing_summary -delay_type min_max -max_paths 20 \'; later documentation reads the resulting report rather than inventing a metric.

Line 50: `-file [file join $output_dir "post_synth_timing_summary.rpt"]` This batch-control line contributes to the flow that runs the reproducible Vivado synth timing iteration hardware flow. The important operands are file: the value named 'file' in this block; join: the value named 'join' in this block; output_dir: the value named 'output dir' in this block.

Line 51: `write_checkpoint -force [file join $output_dir "post_synth.dcp"]` This writes an auditable Vivado artifact using ‘write_checkpoint -force [file join $output_dir "post_synth.dcp"]‘; later documentation reads the resulting report rather than inventing a metric.

101 Coverage statement

This volume contains 99 source files, 28,915 physical source lines, and an explanation for all 26,565 nonblank lines. The companion JSON manifest records each source SHA-256 digest.